Infoblox NIOS WAPI 9.x Reference Guide

Infoblox API Documentation

Table of Contents

- **Infoblox**
  - **CiscoISE**
    - EAAssociation
    - Endpoint
    - PublishSetting
    - SubscribeSetting
  - **Cursor**
  - **DHCP**
    - DDNS
    - ExclusionRange
    - ExclusionRangeTemplate
    - ExpandNetwork
    - FailOver
    - Filter
      - Fingerprint
      - MAC
      - NAC
      - Option
      - RelayAgent
    - FilterRule
      - Fingerprint
      - MAC
      - NAC
      - Option
      - RelayAgent
    - Fingerprint
    - FixedAddr
    - FixedAddrTemplate
    - HostAddr
    - IPv6FixedAddr
    - IPv6FixedAddrTemplate
    - IPv6HostAddr
    - IPv6Network
    - IPv6NetworkContainer
    - IPv6NetworkTemplate
    - IPv6OptionDefinition
    - IPv6OptionSpace
    - IPv6Range
    - IPv6RangeTemplate
    - IPv6SharedNetwork
    - Lease
    - MAC
    - Member
    - MSOption
    - MSServer
    - MSSuperscope
    - Network
    - NetworkContainer
    - NetworkTemplate
    - NetworkUser
    - Option
    - Option60MatchRule
    - OptionDefinition
    - OptionSpace
    - OrderedRanges
    - Range
    - RangeTemplate
    - RoamingHost
    - SharedNetwork
    - SplitNetwork
    - Statistics
    - Template
    - View
  - **DNS**

Infoblox - API Installation and Usage Instructions

- NAME
- VERSION
- Infoblox API Installation Instructions
  - Windows Management System
    - Requirements for Windows Management Systems and IPv4 connections
    - Requirements for Windows Management Systems and IPv6 connections
    - 1a. Install Crypt::SSLeay and libwww-perl.
    - 1b. Install the supporting libraries.
    - 2. PPM 3.x: Add the Infoblox device as a repository.
    - 3. PPM 3.x: Search the Infoblox device for Infoblox packages.
    - 4. PPM 3.x: Install Infoblox packages on your management system.
    - 5. PPM 4.0: Fetch and install Infoblox packages.
  - Unix Management System
    - Requirements for Unix Management Systems
    - DIRECT INSTALLATION
- Infoblox API Usage Guidelines
  - Library options
  - UTF8 Considerations
  - `search()` Considerations
  - Object inheritance
- Alternate installation instructions
    - CPAN INSTALLATION (obsolete, only for CPAN::Site pre 1.02)
      - First Time CPAN Installation
      - 1. Install Crypt::SSLeay and CPAN::Site on your management system.
      - 2. Run CPAN::Site, instead of CPAN.
      - 3. Add the Infoblox device URL to the top of the CPAN URL list.
      - 4. (Optional) Reduce the number of CPAN fetch methods.
      - 5. Reload the CPAN::Site index.
      - 6. Install the Infoblox package on your management system.

NAME

Infoblox - API Installation and Usage Instructions

VERSION

```
9.0.2-50196-23939c893787
```

Infoblox API Installation Instructions

Infoblox provides a Perl API (application programming interface) to help facilitate the integration of the Infoblox device into network environments. The Infoblox DMAPI (Data and Management API) is a set of packages delivered with the Infoblox device to install the package. Use the usual Perl module installation tools on your management system to install the package. For Windows systems using the ActiveState build of Perl, the package is called PPM ("Perl Package Manager" or "Programmer's Package Manager"). For UNIX systems, the package is called CPAN, a global archive of Perl resources.

Windows Management System

Unix Management System

Windows Management System

To install the Infoblox DMAPI packages on a Windows management system, download and install PPM (from http://www.activestate.com/). Then do the following:

1a. Install Crypt::SSLeay and libwww-perl. for IPv4 connections

1b. Install the supporting libraries. for IPv6 connections

- AllRecords
- AllRPZRecords
- BulkHost
- DDNS
  - PrincipalCluster
    - Group
- DNSSecKey
- DNSSecKeyAlgorithm
- DnssecTrustedKey
- FireEye
  - AlertMap
  - RuleMapping
- GlueRecordAddr
- Host
- Member
  - SoaMname
  - SoaSerial
- MSServer
- Nameserver
  - Address
- OrderedResponsePolicyZones
- Record
  - A
  - AAAA
  - Alias
  - CNAME
  - DHCID
  - DNAME
  - DNSKEY
  - DS
  - DTCLBDN
  - MX
  - NAPTR
  - NS
  - NSEC
  - NSEC3
  - NSEC3PARAM
  - PTR
  - RRSIG
  - SRV
  - TLSA
  - TXT
- RootNameServer
- RPZRecord
  - A
  - AAAA
  - AAAAIpAddress
  - AIpAddress
  - CNAME
    - ClientIpAddress
    - ClientIpAddressDN
    - IpAddress
    - IpAddressDN
  - MX
  - NAPTR
  - PTR
  - SRV
  - TXT
- Ruleset
  - NxdomainRule
- SharedRecord
  - A
  - AAAA
  - CNAME
  - MX
  - SRV
  - TXT
- Sortlist
- SRG
- TSIGKey
- View
- Zone
  - Discrepancy

DTC

2. PPM 3.x: Add the Infoblox device as a repository.

3. PPM 3.x: Search the Infoblox device for Infoblox packages.

4. PPM 3.x: Install Infoblox packages on your management system.

5. PPM 4.0: Fetch and install Infoblox packages.

Requirements for Windows Management Systems and IPv4 connections

- Perl 5.8.8 or later

- For Windows management systems: the ActiveState version of Perl 5.8.8 or later with PPM 3.x or later

- Crypt::SSLeay version 0.51 or later (and its dependencies: ssleay32.dll and libeay32.dll)

- LWP::UserAgent version 5.813 or later

- XML::Parser

Requirements for Windows Management Systems and IPv6 connections

- Perl 5.14.2 or later

- For Windows management systems: the ActiveState version of Perl 5.14.2 or later with PPM 3.x or later

- LWP::UserAgent version 6.02 or later and relevant dependencies, including LWP::Protocol::https

- Net::INET6Glue

- XML::Parser

1a. Install Crypt::SSLeay and libwww-perl.

1.0 Check your Perl version.

The Crypt::SSLeay package is Perl version sensitive. Perl version 5.8.x requires different packages than 5.10.x. To check the Perl version installed, enter the command: perl -v.

If Perl 5.8.x is installed, the command will return the following:

```
C:\>perl -v
This is perl, v5.8.8 built for MSWin32-x86-multi-thread...
```

If Perl 5.10.x is installed, the command will return the following:

```
C:\>perl -v
This is perl, v5.10.0 built for MSWin32-x86-multi-thread...
```

1.1 Install Crypt-SSLeay.

The Crypt::SSLeay package allows you to make an HTTPS connection to the Infoblox device. Crypt::SSLeay is not part of the core Perl distribution. To check if it is installed, enter the command: ppm query Crypt-SSLeay. If the Crypt-SSLeay package is already installed, the query returns the following result:

```
C:\>ppm query Crypt-SSLeay
Querying target 1 (ActivePerl 5.8.8.817)
1. Crypt-SSLeay [0.51] OpenSSL glue that provides LWP https
```

If the Crypt::SSLeay package is not installed, the query returns the following:

```
C:\>ppm query Crypt-SSLeay
Querying target 1 (ActivePerl 5.8.8.817)
No matches for 'Crypt-SSLeay; see 'help query'.
```

To install Crypt::SSLeay (and its dependencies ssleay32.dll and libeay32.dll) one needs to either build it from the source distribution, or locate a precompiled Crypt-SSLeay.ppd archive.

Note that Crypt::SSLeay does not support TLSv1.1 and higher until version 0.58. To use TLS v1.1 and higher, you must install Crypt::SSLeay version 0.58 and higher.

1.2 Install libwww-perl.

If you need to update LWP::UserAgent, then you will have to install the latest libwww-perl

- AllRecords
- Certificate
- Health
- LBDN
- Monitor
  - HTTP
  - ICMP
  - PDP
  - SIP
  - SNMP
    - OID
  - TCP
- Object
- Pool
  - DynamicRatioSetting
  - Link
- Query
  - Parameters
  - Response
  - Result
- Record
  - A
  - AAAA
  - CNAME
  - NAPTR
- Server
  - Link
  - Monitor
- Topology
  - Label
  - Rule
    - Source

DXL

- Endpoint
  - Broker

Grid

- Admin
  - AdAuthServer
  - AdAuthService
  - AuthPolicy
  - CertificateAuthService
  - Group
  - LocalUserAuthService
  - Permission
  - RadiusAuthServer
  - RadiusAuthService
  - Role
  - TACACSPlusAuthServer
  - TACACSPlusAuthService
  - User
  - UserProfile
- AllEndpoints
- ApprovalWorkflow
- BFD
  - Template
- CACertificate
- CloudAPI
  - CloudStatistics
  - Info
  - Member
  - Tenant
  - User
  - VMAddress
- ConsentBannerSetting
- CSVImportStatus
- Dashboard
- DataCollection
  - BlacklistedCluster
  - Cluster
  - IPInfo
- DateTime
- DBSnapshot
- DHCP
- Discovery

package.

If the PPM version is 3.x, you will first need to add a PPM repository:

```
C:\> ppm repo add bribes http://www.bribes.org/perl/ppm
```

To install libwww-perl:

```
C:\> ppm install libwww-perl -force
```

1b. Install the supporting libraries.

Infoblox recommends using the ActiveState ppm tool to install the needed modules and dependencies, which may already be installed as part of the perl distribution. For some platforms, some Perl modules may not be available at the time of this writing. an example: Net::INET6Glue, which depends on IO::Socket::INET6; Net::INET6Glue can be compiled following the instructions at

```
http://www.activestate.com/blog/2010/10/how-install-cpan-modules-activeperl
```

using the `Microsoft(r)` SDK section of the instructions. Please refer to ActiveState's website and documentation for further questions.

2. PPM 3.x: Add the Infoblox device as a repository.

The following steps add the Infoblox device as a repository in your PPM 3.x configuration, included with Active Perl from version 5.8.0.802 to 5.8.8.817.

(For PPM 4.0, included with ActivePerl 5.8.8.818 and later, see "5. PPM 4.0: Fetch and install Infoblox packages.")

2.1 Add a PPM repository to the configuration by issuing the ppm repository command;

* For <repo_name>, type a meaningful name for the repository, such as 'Infoblox-device'. * For <ip_addr>, type the IP address that you use for managing the Infoblox device or a resolvable domain name (for a grid, use the address or domain name of the grid master).

```
C:\> ppm repo add <repo_name> https://<ip_addr>/api/dist/ppm
Repositories:
[1] <repo_name>
[2] ActiveState Package Repository
```

(Note that the packages are connected via HTTPS, not HTTP. You only need to add the repository once.)

2.2 To ensure that you have the correct URL, open a Web browser and visit the URL you specified for the repository. If the URL is correct, you can see a directory listing.

3. PPM 3.x: Search the Infoblox device for Infoblox packages.

Start PPM from a command shell:

```
C:\> ppm
PPM - Programmer's Package Manager version 3.4.
Copyright (c) 2001 ActiveState Software Inc.  All Rights Reserved.

Entering interactive shell. Using Term::ReadLine::Perl as readline
library.

Type 'help' to get started.

ppm>

Use PPM to find the Infoblox package:

ppm> search Infoblox
Searching in Active Repositories
1. Infoblox [4.9999990014709] Infoblox Data and Management API 4.x
```

4. PPM 3.x: Install Infoblox packages on your management system.

Install the Infoblox package with PPM:

```
ppm> install Infoblox
====================
Install 'Infoblox' version 4.9999990014709 in ActivePerl 5.8.8.817.
====================
Downloaded 167481 bytes.
...
```

- AdvancedPollSettings
- AutoConversionSetting
- BasicPollSettings
- CLICredential
- ConversionAttributes
- Data
- Device
  - PortStatistics
- DeviceComponent
- DeviceDataCollectionStatus
- DeviceInterface
- DeviceNeighbor
- DeviceSupportInfo
- DeviceSupportInfoResponse
- DiagnosticsStatus
- DiagnosticTask
- DiscoveryDataConversionResult
- DSBundle
- IFAddrInfo
- JobProcessDetails
- NetworkDeprovisionInfo
- Port
  - Config
    - AdminStatus
    - Description
    - VlanInfo
  - Control
    - Info
    - TaskDetails
- Properties
  - BlackoutSetting
- SeedRouter
- SNMP3Credential
- SNMPCredential
- Status
- StatusInfo
- TestCredential
- VLANInfo
- VRFMappingRule
- DistributionSchedule
- DNS
- AllNsgroups
- AttackDetect
- AttackMitigation
- AutoBlackHole
- BulkHostNameTemplate
- ClientSubnetDomain
- DNS64Group
- FileTransferSetting
- FixedRRSetOrderFQDN
- Nsgroup
  - DelegationMember
  - ForwardingMember
  - ForwardStubServer
  - StubMember
- ResponseRateLimiting
- ScavengingSetting
- ScavengingTask
- EAExpressionOp
- ExpressionOp
- Extattr
- ExtensibleAttributeDef
- Descendants
- ListValue
- FileDistribution
- FileDistributionDir
- GlobalSmartFolder
- HostNameRewritePolicy
- HSM
- AllGroup
- SafeNet
- SafeNetGroup
- Thales


```
 Successfully installed Infoblox version 4.9999990014709 in ActivePerl
5.8.8.817.
```

5. PPM 4.0: Fetch and install Infoblox packages.

There is a bug in PPM version 4.0 which is included in ActivePerl 5.8.8.818 and 5.8.8.819 which prevents PPM 4.0 from accessing a module archive with https. This bug, and ActiveState's workaround is described at:

http://bugs.activestate.com/show_bug.cgi?id=52508

If you have ActivePerl 5.8.8.818 or 5.8.8.819 (use perl -v at a command line to see), either upgrade to ActivePerl 5.8.8.820 or use the workaround described in the bug:
http://bugs.activestate.com/show_bug.cgi?id=52508

You can also work around the bug by following the steps in section 5.3.

If you have ActivePerl 5.8.8.820 installed, skip the patch step in 5.1 and immediately install the package by following the steps in section 5.2.

5.1 PPM 4.0: Patch lib/ActivePerl/PPM/limited_inc.pm

If you have ActivePerl 5.8.8.818 or 5.8.8.819, please patch your PPM installation so PPM will be able to find modules stored in repositories accessed by https.

As shown in

http://bugs.activestate.com/show_bug.cgi?id=52508

apply this patch one time to your ActiveState PPM installation for each win32 client:

```
 Index: lib/ActivePerl/PPM/limited_inc.pm
 --- lib/ActivePerl/PPM/limited_inc.pm.~1~ Tue Nov 28 16:01:09 2006
 +++ lib/ActivePerl/PPM/limited_inc.pm Tue Nov 28 16:01:09 2006
 @@ -9,7 +9,7 @@
 (my $dir = __FILE__) =~ s,/ActivePerl/PPM/.*,,;
 Make a new directory, naming it something like "ibpack", and then change
the command prompt to that directory.
 if (grep $_ eq $dir, @INC) {
 - shift(@INC) while $INC[0] ne $dir;
 + push(@INC, shift(@INC)) while $INC[0] ne $dir;
 }

 1;
```

To find the full path to this file on your system, use perl -V at the command line. The contents of @INC are at the end of the output. The limited_inc.pm file is in the lib directory, not in the site/lib directory.

If you are unfamiliar with diffs, the + and - notation just means to replace the line:

```
 shift(@INC) while $INC[0] ne $dir;
```

line with the

```
 push(@INC, shift(@INC)) while $INC[0] ne $dir;
```

When complete, the lines of the patched limited_inc.pm should look like:

```
 if (grep $_ eq $dir, @INC) {
 # patch from http://bugs.activestate.com/show_bug.cgi?id=52508
 #     shift(@INC) while $INC[0] ne $dir;
 push(@INC, shift(@INC)) while $INC[0] ne $dir;
 }
```

Continue on to step 5.2

5.2 PPM 4.0: Install the Infoblox modules

Enter the following command, substituting the IP address or resolvable hostname of the Infoblox device for <ip_addr>:

C:\tmp>ppm install https://<ip_addr>/api/dist/ppm/Infoblox.ppd

or you can use the GUI version of PPM by running the GUI, searching for the Infoblox module and installing it in the usual way.

5.3 PPM 4.0 Alternate Work Around to Install

- ThalesGroup
- HTTPProxyServerSetting
- InformationalBannerSetting
- IPValidationInfo
- KerberosKey
- LicenseGridWide
- LicensePool
- LicensePoolContainer
- LicenseSubPool
- LOM
  - User
- MaxMindDBInfo
- Member
  - BGP
    - AS
    - Neighbor
  - CapacityReport
  - CaptivePortal
    - File
  - Capture
    - Control
    - Status
  - Communication
  - DHCP
  - Discovery
    - ScanInterface
    - SDNConfig
    - VRF
  - DiscoveryProperties
  - DNS
    - IP
    - ViewAddressSetting
  - FileDistribution
  - Interface
  - IPv6StaticRoute
  - License
  - OSPF
  - ParentalControl
  - PNodeToken
  - PreProvisioning
    - Hardware
  - QueryFQDNParameter
  - QueryFQDNResponse
  - RestartServiceStatus
  - ServiceStatus
  - StaticRoute
  - Taxii
    - RPZConfig
  - ThreatProtection
    - Rule
- MSServer
  - AdSites
    - Domain
    - Site
  - AdUser
    - Data
  - DCNSRecordCreation
  - DHCP
  - DNS
- NamedACL
- NatGroup
- NTPAccess
- NTPKey
- NTPServer
- ObjectsChangesTrackingSetting
- ParentalControl
  - MSP
  - NAS
    - AVP
    - Gateway
  - SiteMember
  - Subscriber
  - SubscriberSite
- PersonalSmartFolder

Make a new directory and name the directory, for example, "ibpack". Then change the command prompt to that directory.

Make a new directory, naming it something like "ibpack", and then change the command prompt to that directory.

```
C:\> md ibpack

C:\> cd ibpack

C:\ibpack>
```

Next, transfer the files that contain the module distribution from the Infoblox device to your management system using 'lwp-request'. You must transfer two files: a description file, and the main distribution. The description file is always named "Infoblox.ppd".

Enter the following command, substituting the IP address or resolvable hostname of the Infoblox device for <ip_addr>:

```
C:\ibpack> lwp-request https://<ip_addr>/api/dist/ppm/Infoblox.ppd >
Infoblox.ppd
```

View the Infoblox.ppd file:

```
C:\ibpack> type Infoblox.ppd
<?xml version="1.0" encoding="UTF-8"?>
<SOFTPKG NAME="Infoblox" VERSION="4,999999001478,0,0">
<TITLE>Infoblox</TITLE>
<ABSTRACT>Infoblox Data and Management API 4.x</ABSTRACT>
<AUTHOR>Infoblox, Inc.</AUTHOR>
<IMPLEMENTATION>
<CODEBASE HREF="Infoblox-4.999999001478.tar.gz" />
</IMPLEMENTATION>
</SOFTPKG>
```

The file name of the main distribution package is contained in the HREF attributes of the CODEBASE element. In the output shown above, it is "Infoblox-4.999999001478.tar.gz". Use lwp-request to fetch that file.

```
C:\ibpack> lwp-request https://<ip_addr>/api/dist/ppm/Infoblox-
4.999999001478.tar.gz > Infoblox-4.999999001478.tar.gz
```

Use PPM to install the Infoblox DMAPI from the local file system:

```
C:\ibpack> ppm install Infoblox.ppd
Unpacking Infoblox-4.999999001478...done
Generating HTML for Infoblox-4.999999001478...done
Installing to site area...done
64 files installed
```

The installation is complete.

Uninstalling and Reinstalling the Infoblox Package

After you configure PPM to search the Infoblox repository, use the appropriate command to uninstall and reinstall the Infoblox package:

```
ppm> uninstall Infoblox
====================
Remove 'Infoblox' version 4.9999990014709 in ActivePerl 5.8.8.817.
====================
...
Successfully removed Infoblox version 4.9999990014709 from ActivePerl
5.8.8.817.

ppm> install Infoblox
====================
Install 'Infoblox' version 4.9999990014709 in ActivePerl 5.8.8.817.
====================
...
Successfully installed Infoblox version 4.9999990014709 in ActivePerl
5.8.8.817.
```

Unix Management System

Requirements for Unix Management Systems

For IPv4 connections the following are required

- Perl 5.8.8 or later

- Crypt::SSLeay version 0.51 or later

- RecordNamePolicy
- RestartBannerSetting
- RIR
  - Organization
- RootNameServer
- RPZ
  - ThreatDetails
- ScheduledBackup
- ScheduledTask
  - ChangedObject
- ScheduleSetting
- ServiceRestart
  - Group
    - Order
    - Schedule
  - Request
    - ChangedObject
  - Status
- SmartFolder
  - GroupBy
  - GroupByValue
  - QueryItem
- SmartFolderChildren
- SNMP
  - Admin
  - IPAMThreshold
  - IPAMTrap
  - QueriesUser
  - ThresholdTrap
  - TrapNotification
  - TrapReceiver
  - User
- SyslogBackupServer
- SyslogServer
- TestResult
- ThreatAnalytics
  - Member
  - ModuleSet
  - WhiteList
- ThreatInsight
  - CloudClient
- ThreatProtection
  - NATPort
  - NATRule
  - Profile
    - Rule
  - Rule
  - RuleCategory
  - RuleConfig
  - RuleParam
  - Ruleset
  - RuleTemplate
  - StatInfo
  - Statistics
- TimeZone
- UpdatesDownloadMemberConfig
- UpgradeGroup
  - Member
- UpgradeSchedule
- UpgradeStatus
- VDiscoveryTask
- X509Certificate

IFMap

- CACertificate
- Client
  - Credentials

IPAM

- Address
- DiscoveryTask
  - VServer
- Statistics
- TCPPort

LDAP

- AuthService

- LWP::UserAgent version 5.813 or later

- XML::Parser

For IPv6 connections the following are required

- Perl 5.14.2 or later

- LWP::UserAgent version 6.02 or later and relevant dependencies, including LWP::Protocol::https

- Net::INET6Glue

- XML::Parser

to easily install the required libraries it is recommented to use CPAN, after installing perl run

```
perl -e shell -MCPAN
```

and once configured simply run

```
install LWP::UserAgent
install XML::Parser
```

and the other packages you need depending on IPv4 or IPv6 accessibility.

DIRECT INSTALLATION

To install the Infoblox DMAPI packages on a UNIX management system, first download and install the API package from:

```
https://<ip_addr>/api/dist/CPAN/authors/id/INFOBLOX/
```

where ip_addr is the IP address of the appliance. Then locate and download the file Infoblox-xxxxxxx.tar.gz where xxxxxxx is an integer depending on your API package version.

After you download the package, extract it to a temporary directory with:

```
tar xvfz Infoblox-xxxxxxx.tar.gz
```

Then execute the following commands:

```
cd Infoblox-xxxxxxx/
perl Makefile.PL
make
make install
```

Optionally, before you install, test the package by running:

```
make test
```

The installation is complete.

Infoblox API Usage Guidelines

The usage instructions are provided to assist you in using the Infoblox API.

Library options

The Infoblox library can optionally be invoked by supplying the following options

```
  :accessrights     - If this is specified,
'Infoblox::Grid::Admin::Group' sends a request for
'Infoblox::Grid::Admin::Permission' to set the member 'permission_list'.
  :noaccessrights   - If this is specified,
'Infoblox::Grid::Admin::Group' does not send a request for
'Infoblox::Grid::Admin::Permission' and the member 'permission_list' is
not set. This is specified by default.
  :ipv6connection   - This will allow the API to connect to IPv6 members.
  :hostaddress      - if this is specified the new
'Infoblox::DHCP::HostAddr' and 'Infoblox::DHCP::IPv6HostAddr' types will
be supported, see 'Infoblox::DNS::Host' for more information
```

for example, to enable :ipv6connection the use line would be

```
  use Infoblox qw ( :ipv6connection );
```

UTF8 Considerations

- EA_Mapping
- Server
- **MasterGrid**
- Grid
- **Notification**
- REST
  - Endpoint
  - Template
  - TemplateInstance
  - TemplateParameter
- Rule
- RuleExpressionOp
- **OCSP**
- Responder
- **Session**
- **Util**

If there are utf-8 characters in your script, remember to use the pragma 'use utf8'. If you need more information, please see http://search.cpan.org/~nwclark/perl-5.8.8/lib/utf8.pm on the utf8 pragma.

`search()` Considerations

When using the `search()` method, you can retrieve more information than expected. Include additional checks for exact matches.

Example

```
my @result_array = $session->search(
       "object" => "Infoblox::DNS::Record::A",
       "name" => "infoblox.com" );

my $obj;

foreach my $match (@result_array)
{
   if( ($match->name() eq "infoblox.com") && ($match->ipv4addr()
eq "2.2.2.2") )
   {
      print "Matches\n";
      $obj = $match;
   }
}
```

Object inheritance

Values set in PAPI object members might differ from the effective value used for that particular member during product operation, which could be a value inherited from the Grid or the Grid Member depending on the particular object in question and the state of the object override flags.

Alternate installation instructions

The following is provided for reference purposes only, as CPAN::Site has not supported this installation method for some time.

CPAN INSTALLATION (obsolete, only for CPAN::Site pre 1.02)

To install the Infoblox DMAPI packages on a UNIX management system, first download and install ActivePerl (from www.activestate.com), and then do the following steps:

1. Install Crypt::SSLeay and CPAN::Site on your management system.

2. Run CPAN::Site, instead of CPAN.

3. Add the Infoblox device URL to the top of the CPAN URL list.

4. (Optional) Reduce the number of CPAN fetch methods.

5. Reload the CPAN::Site index.

6. Install the Infoblox package on your management system.

Typically, the installation is done at the root level. See perlfaq8 under the question "How do I keep my own module/library directory?" for instructions on how to install a module as a user other than root.

If you have never used CPAN on the client UNIX machine, you will need to follow the instructions, when you run it for the first time, you will be prompted for many configuration variables. Follow the procedure at 0. First Time CPAN Installation

CPAN::Site Overview

The Infoblox modules are installed using a variant of CPAN named CPAN::Site. See

http://search.cpan.org/search?mode=module&query=CPAN%3A%3ASite

for more information on CPAN::Site.

In the usual CPAN model, there is a single CPAN repository, and every copy is a full and complete copy of every available package. In the CPAN::Site model, there are multiple repositories which may have a given module. As the Infoblox modules are deployed with

CPAN::Site, you must use CPAN::Site instead of CPAN to install them. This is covered in more detail in step 2. If you use CPAN instead of CPAN::Site, typically the installation process will not find the Infoblox module in step 5.

One consequence of using CPAN::Site is the installation searches multiple sites for the same set of files, finding some of them in some locations but not in others. This is a normal result. Additionally, CPAN has the ability to search for and fetch files using several different mechanisms: curl, FTP, lynx, ncftpget, wget, and LWP. Both CPAN and CPAN::Site use each of these methods in turn when it tries to find files. If the CPAN configuration lists many sites in its URL list and many access methods, the installation may take a long time while CPAN::Site tries each of the access methods on each of the listed sites. For this reason, Infoblox recommends that you reduce the number of sites in the URL list and the number of access methods as described in step 4.

If you are installing a new version of the Infoblox modules and your urllist configuration still includes the Infoblox appliance, skip directly to steps 5 and 6.

First Time CPAN Installation

If you have never used CPAN on this client machine before, enter the following command:

```
[root@lx2 ~]# perl -MCPAN -e shell
```

and follow the prompts through the CPAN configuration. You can accept all of the defaults, except for the list of CPAN sites close to your physical location. Navigate the menus and choose 1 or 2 CPAN sites that are close to your site. When your configuration choices are complete, exit CPAN by typing exit at the prompt.

1. Install Crypt::SSLeay and CPAN::Site on your management system.

The Crypt::SSLeay package allows you to make an HTTPS connection to the Infoblox device. If Crypt::SSLeay is not already installed, open a terminal window, login as the root user, and use CPAN to install the Crypt::SSLeay package and all its dependencies:

```
lx> perl -MCPAN -e 'install Crypt::SSLeay'
```

Install the CPAN::Site module:

```
lx> perl -MCPAN -e 'install CPAN::Site'
```

When the system prompts: "Running client or server?", enter 'client'. The installation process is asking if you want to install CPAN::Site as a server or as a client. Since CPAN::Site retrieves the Infoblox modules from a server, CPAN::Site acts as a client.

2. Run CPAN::Site, instead of CPAN.

Use CPAN::Site instead of CPAN:

```
lx> perl -MCPAN::Site -e shell
CPAN: File::HomeDir loaded ok

cpan shell -- CPAN exploration and modules installation (v1.87)
ReadLine support enabled
```

You must use CPAN::Site every time to access the Infoblox modules.

3. Add the Infoblox device URL to the top of the CPAN URL list.

3.0 You can manipulate the contents of the urllist with the usual perl functions push, pop, shift and unshift. push and pop operate on the end of the list, while shift and unshift operate on the beginning of the list. To empty the urllist, issue repeated shift or pop operands like this:

```
cpan> o conf urllist
urllist
[ftp://mirrors.kernel.org/pub/CPAN]
[http://mirrors.kernel.org/pub/CPAN]
Type 'o conf' to view all configuration items

cpan> o conf urllist shift

cpan> o conf urllist shift

cpan> o conf urllist
urllist
Type 'o conf' to view all configuration items
```

You can put an item on the list at the beginning with unshift and at the end of the list with

push:

```
 cpan> o conf urllist unshift ftp://mirrors.kernel.org/pub/CPAN

 cpan> o conf urllist push ftp://bogus.site.at.end.of.list/CPAN

 cpan> o conf urllist
 urllist
  [ftp://mirrors.kernel.org/pub/CPAN]
  [ftp://bogus.site.at.end.of.list/CPAN]
Type 'o conf' to view all configuration items
```

When you finish configuring the urllist, you must have the Infoblox appliance as the first item in the list. This is a requirement of CPAN::Site. The best configuration of urllist is a two-item list with the Infoblox appliance first and one other close, fast CPAN site second. For example, at the Infoblox headquarters a good set of choices for urllist is:

```
 cpan> o conf urllist
 urllist
  [https://<ip_addr>/api/dist/CPAN]
  [ftp://mirrors.kernel.org/pub/CPAN]
Type 'o conf' to view all configuration items
```

The complete list of CPAN sites can be found at:

http://www.cpan.org/SITES.html

3.1 Configure the Infoblox device first in the CPAN URL list, so that CPAN::Site finds it first when searching for packages. Type the IP address or domain name of your Infoblox device for <ip_addr> (for a grid, use the address of the grid master):

cpan> o conf urllist unshift https://<ip_addr>/api/dist/CPAN

(Note: You only need to add the repository to the URL list once.)

3.2 You must have at least one other normal CPAN site in the list in case your client needs other public packages. The following is a sample URL list for a set of CPAN sites in North America:

```
 cpan> o conf urllist
 urllist
  [https://<ip_addr>/api/dist/CPAN]
  [ftp://mirrors.kernel.org/pub/CPAN]
  [ftp://cpan.pair.com/pub/CPAN/]
Type 'o conf' to view all configuration items
```

If you do not have any other CPAN sites in your URL list, add at least one before continuing. Use the command

o conf urllist push <url_string>

3.3 To ensure that you have the correct URL for the Infoblox modules, open a Web browser and visit the URL you specified for the repository, in this example https://<ip_addr>/api/dist/CPAN. If the URL is correct, you can see a directory listing of the module files.

3.4 Save your CPAN configuration:

```
 cpan> o conf commit
commit: wrote '/usr/lib/perl5/5.8.8/CPAN/Config.pm'
```

4. (Optional) Reduce the number of CPAN fetch methods.

By reducing the number of CPAN fetch methods, you can shorten the installation process. For an explanation why Infoblox recommends this step, see the note in the introduction (before step 1).

```
 CPAN (and CPAN::Site) supports up to six different ways to search for
and fetch files: curl, FTP, lynx, ncftpget, wget, and LWP. Infoblox
recommends that you choose the method that work best for you and only
configure those methods. For the typical cases, Infoblox recommends
keeping LWP, FTP, and ncftpget enabled and disabling curl, lynx and wget.
To disable a fetch method, set it to an empty string and then commit the
configuration. In the example below, you first display the configuration
parameters for curl, lynx, and wget, and then set them to an empty
string:

 cpan> o conf curl
 curl                                [/usr/bin/curl]

 cpan> o conf curl ""
 curl                                []

 cpan> o conf lynx
```

```
 lynx                            [/usr/bin/lynx]

 cpan> o conf lynx ""
 lynx                            []

 cpan> o conf wget
 wget                            [/usr/bin/wget]

 cpan> o conf wget ""
 wget                            []

 cpan> o conf commit
 commit: wrote '/usr/lib/perl5/5.8.8/CPAN/Config.pm'
```

5. Reload the CPAN::Site index.

Use CPAN to reload the CPAN::Site index:

```
 cpan> reload index
 CPAN: Storable loaded ok
 Going to read /root/.cpan/Metadata
 Database was generated on Sat, 09 Sep 2006 22:32:52 GMT
 CPAN: LWP::UserAgent loaded ok
 Fetching with LWP:
 https://<ip_addr>/api/dist/CPAN/authors/01mailrc.txt.gz
 ...
 Fetching with LWP:
 https://<ip_addr>/api/dist/CPAN/modules/03modlist.data.gz
 LWP failed with code[404] message[Not Found]
 Fetching with LWP:
 ftp://mirrors.kernel.org/pub/CPAN/modules/03modlist.data.gz
 Going to read /root/.cpan/sources/modules/03modlist.data.gz
 Going to write /root/.cpan/Metadata
```

Note: Unlike CPAN, CPAN::Site is designed so that some of the files that describe the available packages are not found in all of the locations in your URL list. It is normal to see some "file not found" errors during the index reload.

6. Install the Infoblox package on your management system.

The usual CPAN process for installing Perl modules is:

6.1 Search

6.2 Test

6.3 Install

6.4 Uninstall and Reinstall

6.1 Search

Use the m operator to search for the Infoblox module:

```
 cpan> m /Infoblox/
 CPAN: Storable loaded ok
 Going to read /root/.cpan/Metadata
 Database was generated on Mon, 18 Sep 2006 06:29:55 GMT
 CPAN: LWP::UserAgent loaded ok
 . . .
 Fetching with LWP:
 https://<ip_addr>/api/dist/CPAN/site/01mailrc.txt.gz
 Going to read /root/.cpan/sources/site/01mailrc.txt.gz
 Module id = Infoblox
 CPAN_USERID  INFOBLOX (Infoblox Technical Support
<support@infoblox.com>)
 CPAN_VERSION 4.9999990014709
 CPAN_FILE    INFOBLOX/Infoblox-4.9999990014709.tar.gz
 INST_FILE    (not installed)
```

If the search cannot find the Infoblox modules:

a. Ensure that you are using CPAN::Site, not CPAN as shown in 2. Run CPAN::Site, instead of CPAN. b. Check the Infoblox appliance in the urllist as shown in step 3.3.

6.2 Test

You can optionally test the package before the installation:

```
 cpan> test Infoblox
 Running test for module Infoblox
 Running make for Infoblox-4.999990014709.tar.gz
 ...
 All tests successful.
 Files=1, Tests=1,  0 wallclock secs ( 0.74 cusr +  0.08 csys =  0.82
CPU)
```

```
 /usr/bin/make test - OK
```

6.3 Install

Install the Infoblox package:

```
 cpan> install Infoblox
 Running install for module Infoblox
 Running make for INFOBLOX/Infoblox-4.9999990014709.tar.gz
 ...
 Writing /usr/lib/perl5/site_perl/5.8.8/i386-linux-thread-
multi/auto/Infoblox/.packlist
 Appending installation info to /usr/lib/perl5/5.8.8/i386-linux-thread-
multi/perllocal.pod
 /usr/bin/make install  -- OK
```

To exit the CPAN::Site shell:

```
 cpan> quit
```

The installation is complete.

6.4 Uninstalling and Reinstalling the Infoblox Package

Although not directly supported by CPAN, it is simple to uninstall the Infoblox package using published techniques. One way to do it is provided by the modrm program that is described in the ExtUtils::Packlist documentation. See http://search.cpan.org/~yves/ExtUtils-Install-1.41/lib/ExtUtils/Packlist.pm#EXAMPLE for the modrm program. To reinstall it, repeat steps 5 and 6 above.

Infoblox - API Installation and Usage Instructions

- NAME
- VERSION
- Infoblox API Installation Instructions
  - Windows Management System
    - Requirements for Windows Management Systems and IPv4 connections
    - Requirements for Windows Management Systems and IPv6 connections
    - 1a. Install Crypt::SSLeay and libwww-perl.
    - 1b. Install the supporting libraries.
    - 2. PPM 3.x: Add the Infoblox device as a repository.
    - 3. PPM 3.x: Search the Infoblox device for Infoblox packages.
    - 4. PPM 3.x: Install Infoblox packages on your management system.
    - 5. PPM 4.0: Fetch and install Infoblox packages.
  - Unix Management System
    - Requirements for Unix Management Systems
    - DIRECT INSTALLATION
- Infoblox API Usage Guidelines
  - Library options
  - UTF8 Considerations
  - `search()` Considerations
  - Object inheritance
- Alternate installation instructions
    - CPAN INSTALLATION (obsolete, only for CPAN::Site pre 1.02)
      - First Time CPAN Installation
      - 1. Install Crypt::SSLeay and CPAN::Site on your management system.
      - 2. Run CPAN::Site, instead of CPAN.
      - 3. Add the Infoblox device URL to the top of the CPAN URL list.
      - 4. (Optional) Reduce the number of CPAN fetch methods.
      - 5. Reload the CPAN::Site index.
      - 6. Install the Infoblox package on your management system.

NAME

Infoblox - API Installation and Usage Instructions

VERSION

```
9.0.2-50196-23939c893787
```

Infoblox API Installation Instructions

Infoblox provides a Perl API (application programming interface) to help facilitate the integration of the Infoblox device into network environments. The Infoblox DMAPI (Data and Management API) is a set of packages delivered with the Infoblox device to install the package. Use the usual Perl module installation tools on your management system to install the package. For Windows systems using the ActiveState build of Perl, the package is called PPM ("Perl Package Manager" or "Programmer's Package Manager"). For UNIX systems, the package is called CPAN, a global archive of Perl resources.

Windows Management System

Unix Management System

Windows Management System

To install the Infoblox DMAPI packages on a Windows management system, download and install PPM (from http://www.activestate.com/). Then do the following:

1a. Install Crypt::SSLeay and libwww-perl. for IPv4 connections

1b. Install the supporting libraries. for IPv6 connections

2. PPM 3.x: Add the Infoblox device as a repository.

3. PPM 3.x: Search the Infoblox device for Infoblox packages.

4. PPM 3.x: Install Infoblox packages on your management system.

5. PPM 4.0: Fetch and install Infoblox packages.

Requirements for Windows Management Systems and IPv4 connections

- Perl 5.8.8 or later

- For Windows management systems: the ActiveState version of Perl 5.8.8 or later with PPM 3.x or later

- Crypt::SSLeay version 0.51 or later (and its dependencies: ssleay32.dll and libeay32.dll)

- LWP::UserAgent version 5.813 or later

- XML::Parser

Requirements for Windows Management Systems and IPv6 connections

- Perl 5.14.2 or later

- For Windows management systems: the ActiveState version of Perl 5.14.2 or later with PPM 3.x or later

- LWP::UserAgent version 6.02 or later and relevant dependencies, including LWP::Protocol::https

- Net::INET6Glue

- XML::Parser

1a. Install Crypt::SSLeay and libwww-perl.

1.0 Check your Perl version.

The Crypt::SSLeay package is Perl version sensitive. Perl version 5.8.x requires different packages than 5.10.x. To check the Perl version installed, enter the command: perl -v.

If Perl 5.8.x is installed, the command will return the following:

```
C:\>perl -v
This is perl, v5.8.8 built for MSWin32-x86-multi-thread...
```

If Perl 5.10.x is installed, the command will return the following:

```
C:\>perl -v
This is perl, v5.10.0 built for MSWin32-x86-multi-thread...
```

1.1 Install Crypt-SSLeay.

The Crypt::SSLeay package allows you to make an HTTPS connection to the Infoblox device. Crypt::SSLeay is not part of the core Perl distribution. To check if it is installed, enter the command: ppm query Crypt-SSLeay. If the Crypt-SSLeay package is already installed, the query returns the following result:

```
C:\>ppm query Crypt-SSLeay
Querying target 1 (ActivePerl 5.8.8.817)
1. Crypt-SSLeay [0.51] OpenSSL glue that provides LWP https
```

If the Crypt::SSLeay package is not installed, the query returns the following:

```
C:\>ppm query Crypt-SSLeay
Querying target 1 (ActivePerl 5.8.8.817)
No matches for 'Crypt-SSLeay; see 'help query'.
```

To install Crypt::SSLeay (and its dependencies ssleay32.dll and libeay32.dll) one needs to either build it from the source distribution, or locate a precompiled Crypt-SSLeay.ppd archive.

Note that Crypt::SSLeay does not support TLSv1.1 and higher until version 0.58. To use TLS v1.1 and higher, you must install Crypt::SSLeay version 0.58 and higher.

1.2 Install libwww-perl.

If you need to update LWP::UserAgent, then you will have to install the latest libwww-perl package.

If the PPM version is 3.x, you will first need to add a PPM repository:

```
C:\> ppm repo add bribes http://www.bribes.org/perl/ppm
```

To install libwww-perl:

```
C:\> ppm install libwww-perl -force
```

1b. Install the supporting libraries.

Infoblox recommends using the ActiveState ppm tool to install the needed modules and dependencies, which may already be installed as part of the perl distribution. For some platforms, some Perl modules may not be available at the time of this writing. an example:

Net::INET6Glue, which depends on IO::Socket::INET6; Net::INET6Glue can be compiled following the instructions at

http://www.activestate.com/blog/2010/10/how-install-cpan-modules-activeperl

using the Microsoft(r) SDK section of the instructions. Please refer to ActiveState's website and documentation for further questions.

2. PPM 3.x: Add the Infoblox device as a repository.

The following steps add the Infoblox device as a repository in your PPM 3.x configuration, included with Active Perl from version 5.8.0.802 to 5.8.8.817.

(For PPM 4.0, included with ActivePerl 5.8.8.818 and later, see "5. PPM 4.0: Fetch and install Infoblox packages.")

2.1 Add a PPM repository to the configuration by issuing the ppm repository command;

* For <repo_name>, type a meaningful name for the repository, such as 'Infoblox-device'. * For <ip_addr>, type the IP address that you use for managing the Infoblox device or a resolvable domain name (for a grid, use the address or domain name of the grid master).

```
C:\> ppm repo add <repo_name> https://<ip_addr>/api/dist/ppm
Repositories:
[1] <repo_name>
[2] ActiveState Package Repository
```

(Note that the packages are connected via HTTPS, not HTTP. You only need to add the repository once.)

2.2 To ensure that you have the correct URL, open a Web browser and visit the URL you specified for the repository. If the URL is correct, you can see a directory listing.

3. PPM 3.x: Search the Infoblox device for Infoblox packages.

Start PPM from a command shell:

```
C:\> ppm
PPM - Programmer's Package Manager version 3.4.
Copyright (c) 2001 ActiveState Software Inc.  All Rights Reserved.

Entering interactive shell. Using Term::ReadLine::Perl as readline library.

Type 'help' to get started.

ppm>

Use PPM to find the Infoblox package:

ppm> search Infoblox
Searching in Active Repositories
1. Infoblox [4.9999990014709] Infoblox Data and Management API 4.x
```

4. PPM 3.x: Install Infoblox packages on your management system.

Install the Infoblox package with PPM:

```
ppm> install Infoblox
====================
Install 'Infoblox' version 4.9999990014709 in ActivePerl 5.8.8.817.
====================
Downloaded 167481 bytes.
...
Successfully installed Infoblox version 4.9999990014709 in ActivePerl 5.8.8.817.
```

5. PPM 4.0: Fetch and install Infoblox packages.

There is a bug in PPM version 4.0 which is included in ActivePerl 5.8.8.818 and 5.8.8.819 which prevents PPM 4.0 from accessing a module archive with https. This bug, and ActiveState's workaround is described at:

http://bugs.activestate.com/show_bug.cgi?id=52508

If you have ActivePerl 5.8.8.818 or 5.8.8.819 (use perl -v at a command line to see), either upgrade to ActivePerl 5.8.8.820 or use the workaround described in the bug: http://bugs.activestate.com/show_bug.cgi?id=52508

You can also work around the bug by following the steps in section 5.3.

If you have ActivePerl 5.8.8.820 installed, skip the patch step in 5.1 and immediately install the package by following the steps in section 5.2.

5.1 PPM 4.0: Patch lib/ActivePerl/PPM/limited_inc.pm

If you have ActivePerl 5.8.8.818 or 5.8.8.819, please patch your PPM installation so PPM will be able to find modules stored in repositories accessed by https.

As shown in

http://bugs.activestate.com/show_bug.cgi?id=52508

apply this patch one time to your ActiveState PPM installation for each win32 client:

```
Index: lib/ActivePerl/PPM/limited_inc.pm
--- lib/ActivePerl/PPM/limited_inc.pm.~1~ Tue Nov 28 16:01:09 2006
+++ lib/ActivePerl/PPM/limited_inc.pm Tue Nov 28 16:01:09 2006
@@ -9,7 +9,7 @@
(my $dir = __FILE__) =~ s,/ActivePerl/PPM/.*,,;
Make a new directory, naming it something like "ibpack", and then change the command prompt to that
directory.
if (grep $_ eq $dir, @INC) {
- shift(@INC) while $INC[0] ne $dir;
+ push(@INC, shift(@INC)) while $INC[0] ne $dir;
}

1;
```

To find the full path to this file on your system, use perl -V at the command line. The contents of @INC are at the end of the output. The limited_inc.pm file is in the lib directory, not in the site/lib directory.

If you are unfamiliar with diffs, the + and - notation just means to replace the line:

```
shift(@INC) while $INC[0] ne $dir;
```

line with the

```
push(@INC, shift(@INC)) while $INC[0] ne $dir;
```

When complete, the lines of the patched limited_inc.pm should look like:

```
if (grep $_ eq $dir, @INC) {
# patch from http://bugs.activestate.com/show_bug.cgi?id=52508
#    shift(@INC) while $INC[0] ne $dir;
push(@INC, shift(@INC)) while $INC[0] ne $dir;
}
```

Continue on to step 5.2

5.2 PPM 4.0: Install the Infoblox modules

Enter the following command, substituting the IP address or resolvable hostname of the Infoblox device for <ip_addr>:

C:\tmp>ppm install https://<ip_addr>/api/dist/ppm/Infoblox.ppd

or you can use the GUI version of PPM by running the GUI, searching for the Infoblox module and installing it in the usual way.

5.3 PPM 4.0 Alternate Work Around to Install

Make a new directory and name the directory, for example, "ibpack". Then change the command prompt to that directory.

Make a new directory, naming it something like "ibpack", and then change the command prompt to that directory.

```
C:\> md ibpack

C:\> cd ibpack

C:\ibpack>
```

Next, transfer the files that contain the module distribution from the Infoblox device to your management system using 'lwp-request'. You must transfer two files: a description file, and the main distribution. The description file is always named "Infoblox.ppd".

Enter the following command, substituting the IP address or resolvable hostname of the Infoblox device for <ip_addr>:

```
C:\ibpack> lwp-request https://<ip_addr>/api/dist/ppm/Infoblox.ppd > Infoblox.ppd
```

View the Infoblox.ppd file:

```
C:\ibpack> type Infoblox.ppd
<?xml version="1.0" encoding="UTF-8"?>
<SOFTPKG NAME="Infoblox" VERSION="4,999999001478,0,0">
<TITLE>Infoblox</TITLE>
<ABSTRACT>Infoblox Data and Management API 4.x</ABSTRACT>
<AUTHOR>Infoblox, Inc.</AUTHOR>
<IMPLEMENTATION>
<CODEBASE HREF="Infoblox-4.999999001478.tar.gz" />
</IMPLEMENTATION>
</SOFTPKG>
```

The file name of the main distribution package is contained in the HREF attributes of the CODEBASE element. In the output shown above,

it is "Infoblox-4.999999001478.tar.gz". Use lwp-request to fetch that file.

```
 C:\ibpack> lwp-request https://<ip_addr>/api/dist/ppm/Infoblox-4.999999001478.tar.gz > Infoblox-
4.999999001478.tar.gz
```

Use PPM to install the Infoblox DMAPI from the local file system:

```
 C:\ibpack> ppm install Infoblox.ppd
 Unpacking Infoblox-4.999999001478...done
 Generating HTML for Infoblox-4.999999001478...done
 Installing to site area...done
 64 files installed
```

The installation is complete.

Uninstalling and Reinstalling the Infoblox Package

After you configure PPM to search the Infoblox repository, use the appropriate command to uninstall and reinstall the Infoblox package:

```
 ppm> uninstall Infoblox
 ====================
 Remove 'Infoblox' version 4.9999990014709 in ActivePerl 5.8.8.817.
 ====================
 ...
 Successfully removed Infoblox version 4.9999990014709 from ActivePerl 5.8.8.817.

 ppm> install Infoblox
 ====================
 Install 'Infoblox' version 4.9999990014709 in ActivePerl 5.8.8.817.
 ====================
 ...
 Successfully installed Infoblox version 4.9999990014709 in ActivePerl 5.8.8.817.
```

Unix Management System

Requirements for Unix Management Systems

For IPv4 connections the following are required

- Perl 5.8.8 or later

- Crypt::SSLeay version 0.51 or later

- LWP::UserAgent version 5.813 or later

- XML::Parser

For IPv6 connections the following are required

- Perl 5.14.2 or later

- LWP::UserAgent version 6.02 or later and relevant dependencies, including LWP::Protocol::https

- Net::INET6Glue

- XML::Parser

to easily install the required libraries it is recommented to use CPAN, after installing perl run

```
 perl -e shell -MCPAN
```

and once configured simply run

```
 install LWP::UserAgent
 install XML::Parser
```

and the other packages you need depending on IPv4 or IPv6 accessibility.

DIRECT INSTALLATION

To install the Infoblox DMAPI packages on a UNIX management system, first download and install the API package from:

```
 https://<ip_addr>/api/dist/CPAN/authors/id/INFOBLOX/
```

where ip_addr is the IP address of the appliance. Then locate and download the file Infoblox-xxxxxxx.tar.gz where xxxxxxx is an integer depending on your API package version.

After you download the package, extract it to a temporary directory with:

```
 tar xvfz Infoblox-xxxxxxx.tar.gz
```

Then execute the following commands:

```
cd Infoblox-xxxxxxx/
perl Makefile.PL
make
make install
```

Optionally, before you install, test the package by running:

```
make test
```

The installation is complete.

Infoblox API Usage Guidelines

The usage instructions are provided to assist you in using the Infoblox API.

Library options

The Infoblox library can optionally be invoked by supplying the following options

```
  :accessrights     - If this is specified, 'Infoblox::Grid::Admin::Group' sends a request for
'Infoblox::Grid::Admin::Permission' to set the member 'permission_list'.
  :noaccessrights   - If this is specified, 'Infoblox::Grid::Admin::Group' does not send a request for
'Infoblox::Grid::Admin::Permission' and the member 'permission_list' is not set. This is specified by
default.
  :ipv6connection   - This will allow the API to connect to IPv6 members.
  :hostaddress      - if this is specified the new 'Infoblox::DHCP::HostAddr' and
'Infoblox::DHCP::IPv6HostAddr' types will be supported, see 'Infoblox::DNS::Host' for more information
```

for example, to enable :ipv6connection the use line would be

```
  use Infoblox qw ( :ipv6connection );
```

UTF8 Considerations

If there are utf-8 characters in your script, remember to use the pragma 'use utf8'. If you need more information, please see http://search.cpan.org/~nwclark/perl-5.8.8/lib/utf8.pm on the utf8 pragma.

`search()` Considerations

When using the `search()` method, you can retrieve more information than expected. Include additional checks for exact matches.

Example

```
my @result_array = $session->search(
       "object" => "Infoblox::DNS::Record::A",
       "name" => "infoblox.com" );

my $obj;

foreach my $match (@result_array)
{
   if( ($match->name() eq "infoblox.com") && ($match->ipv4addr() eq "2.2.2.2") )
   {
      print "Matches\n";
      $obj = $match;
   }
}
```

Object inheritance

Values set in PAPI object members might differ from the effective value used for that particular member during product operation, which could be a value inherited from the Grid or the Grid Member depending on the particular object in question and the state of the object override flags.

Alternate installation instructions

The following is provided for reference purposes only, as CPAN::Site has not supported this installation method for some time.

CPAN INSTALLATION (obsolete, only for CPAN::Site pre 1.02)

To install the Infoblox DMAPI packages on a UNIX management system, first download and install ActivePerl (from www.activestate.com), and then do the following steps:

1. Install Crypt::SSLeay and CPAN::Site on your management system.

2. Run CPAN::Site, instead of CPAN.

3. Add the Infoblox device URL to the top of the CPAN URL list.

4. (Optional) Reduce the number of CPAN fetch methods.

5. Reload the CPAN::Site index.

6. Install the Infoblox package on your management system.

Typically, the installation is done at the root level. See perlfaq8 under the question "How do I keep my own module/library directory?" for instructions on how to install a module as a user other than root.

If you have never used CPAN on the client UNIX machine, you will need to follow the instructions, when you run it for the first time, you will be prompted for many configuration variables. Follow the procedure at 0. First Time CPAN Installation

CPAN::Site Overview

The Infoblox modules are installed using a variant of CPAN named CPAN::Site. See

http://search.cpan.org/search?mode=module&query=CPAN%3A%3ASite

for more information on CPAN::Site.

In the usual CPAN model, there is a single CPAN repository, and every copy is a full and complete copy of every available package. In the CPAN::Site model, there are multiple repositories which may have a given module. As the Infoblox modules are deployed with CPAN::Site, you must use CPAN::Site instead of CPAN to install them. This is covered in more detail in step 2. If you use CPAN instead of CPAN::Site, typically the installation process will not find the Infoblox module in step 5.

One consequence of using CPAN::Site is the installation searches multiple sites for the same set of files, finding some of them in some locations but not in others. This is a normal result. Additionally, CPAN has the ability to search for and fetch files using several different mechanisms: curl, FTP, lynx, ncftpget, wget, and LWP. Both CPAN and CPAN::Site use each of these methods in turn when it tries to find files. If the CPAN configuration lists many sites in its URL list and many access methods, the installation may take a long time while CPAN::Site tries each of the access methods on each of the listed sites. For this reason, Infoblox recommends that you reduce the number of sites in the URL list and the number of access methods as described in step 4.

If you are installing a new version of the Infoblox modules and your urllist configuration still includes the Infoblox appliance, skip directly to steps 5 and 6.

First Time CPAN Installation

If you have never used CPAN on this client machine before, enter the following command:

```
[root@lx2 ~]# perl -MCPAN -e shell
```

and follow the prompts through the CPAN configuration. You can accept all of the defaults, except for the list of CPAN sites close to your physical location. Navigate the menus and choose 1 or 2 CPAN sites that are close to your site. When your configuration choices are complete, exit CPAN by typing exit at the prompt.

1. Install Crypt::SSLeay and CPAN::Site on your management system.

The Crypt::SSLeay package allows you to make an HTTPS connection to the Infoblox device. If Crypt::SSLeay is not already installed, open a terminal window, login as the root user, and use CPAN to install the Crypt::SSLeay package and all its dependencies:

```
lx> perl -MCPAN -e 'install Crypt::SSLeay'
```

Install the CPAN::Site module:

```
lx> perl -MCPAN -e 'install CPAN::Site'
```

When the system prompts: "Running client or server?", enter 'client'. The installation process is asking if you want to install CPAN::Site as a server or as a client. Since CPAN::Site retrieves the Infoblox modules from a server, CPAN::Site acts as a client.

2. Run CPAN::Site, instead of CPAN.

Use CPAN::Site instead of CPAN:

```
lx> perl -MCPAN::Site -e shell
CPAN: File::HomeDir loaded ok

cpan shell -- CPAN exploration and modules installation (v1.87)
ReadLine support enabled
```

You must use CPAN::Site every time to access the Infoblox modules.

3. Add the Infoblox device URL to the top of the CPAN URL list.

3.0 You can manipulate the contents of the urllist with the usual perl functions push, pop, shift and unshift. push and pop operate on the end of the list, while shift and unshift operate on the beginning of the list. To empty the urllist, issue repeated shift or pop operands like this:

```
 cpan> o conf urllist
 urllist
  [ftp://mirrors.kernel.org/pub/CPAN]
  [http://mirrors.kernel.org/pub/CPAN]
 Type 'o conf' to view all configuration items

 cpan> o conf urllist shift

 cpan> o conf urllist shift

 cpan> o conf urllist
 urllist
Type 'o conf' to view all configuration items
```

You can put an item on the list at the beginning with unshift and at the end of the list with push:

```
 cpan> o conf urllist unshift ftp://mirrors.kernel.org/pub/CPAN

 cpan> o conf urllist push ftp://bogus.site.at.end.of.list/CPAN

 cpan> o conf urllist
 urllist
  [ftp://mirrors.kernel.org/pub/CPAN]
  [ftp://bogus.site.at.end.of.list/CPAN]
Type 'o conf' to view all configuration items
```

When you finish configuring the urllist, you must have the Infoblox appliance as the first item in the list. This is a requirement of CPAN::Site. The best configuration of urllist is a two-item list with the Infoblox appliance first and one other close, fast CPAN site second. For example, at the Infoblox headquarters a good set of choices for urllist is:

```
 cpan> o conf urllist
 urllist
  [https://<ip_addr>/api/dist/CPAN]
  [ftp://mirrors.kernel.org/pub/CPAN]
Type 'o conf' to view all configuration items
```

The complete list of CPAN sites can be found at:

http://www.cpan.org/SITES.html

3.1 Configure the Infoblox device first in the CPAN URL list, so that CPAN::Site finds it first when searching for packages. Type the IP address or domain name of your Infoblox device for <ip_addr> (for a grid, use the address of the grid master):

cpan> o conf urllist unshift https://<ip_addr>/api/dist/CPAN

(Note: You only need to add the repository to the URL list once.)

3.2 You must have at least one other normal CPAN site in the list in case your client needs other public packages. The following is a sample URL list for a set of CPAN sites in North America:

```
 cpan> o conf urllist
 urllist
  [https://<ip_addr>/api/dist/CPAN]
  [ftp://mirrors.kernel.org/pub/CPAN]
  [ftp://cpan.pair.com/pub/CPAN/]
Type 'o conf' to view all configuration items
```

If you do not have any other CPAN sites in your URL list, add at least one before continuing. Use the command

o conf urllist push <url_string>

3.3 To ensure that you have the correct URL for the Infoblox modules, open a Web browser and visit the URL you specified for the repository, in this example https://<ip_addr>/api/dist/CPAN. If the URL is correct, you can see a directory listing of the module files.

3.4 Save your CPAN configuration:

```
 cpan> o conf commit
commit: wrote '/usr/lib/perl5/5.8.8/CPAN/Config.pm'
```

4. (Optional) Reduce the number of CPAN fetch methods.

By reducing the number of CPAN fetch methods, you can shorten the installation process. For an explanation why Infoblox recommends this step, see the note in the introduction (before step 1).

```
 CPAN (and CPAN::Site) supports up to six different ways to search for and fetch files: curl, FTP, lynx,
ncftpget, wget, and LWP. Infoblox recommends that you choose the method that work best for you and only
configure those methods. For the typical cases, Infoblox recommends keeping LWP, FTP, and ncftpget enabled
and disabling curl, lynx and wget. To disable a fetch method, set it to an empty string and then commit the
configuration. In the example below, you first display the configuration parameters for curl, lynx, and
```

```
wget, and then set them to an empty string:

 cpan> o conf curl
 curl                           [/usr/bin/curl]

 cpan> o conf curl ""
 curl                           []

 cpan> o conf lynx
 lynx                           [/usr/bin/lynx]

 cpan> o conf lynx ""
 lynx                           []

 cpan> o conf wget
 wget                           [/usr/bin/wget]

 cpan> o conf wget ""
 wget                           []

 cpan> o conf commit
 commit: wrote '/usr/lib/perl5/5.8.8/CPAN/Config.pm'
```

5. Reload the CPAN::Site index.

Use CPAN to reload the CPAN::Site index:

```
 cpan> reload index
 CPAN: Storable loaded ok
 Going to read /root/.cpan/Metadata
 Database was generated on Sat, 09 Sep 2006 22:32:52 GMT
 CPAN: LWP::UserAgent loaded ok
 Fetching with LWP:
 https://<ip_addr>/api/dist/CPAN/authors/01mailrc.txt.gz
 ...
 Fetching with LWP:
 https://<ip_addr>/api/dist/CPAN/modules/03modlist.data.gz
 LWP failed with code[404] message[Not Found]
 Fetching with LWP:
 ftp://mirrors.kernel.org/pub/CPAN/modules/03modlist.data.gz
 Going to read /root/.cpan/sources/modules/03modlist.data.gz
 Going to write /root/.cpan/Metadata
```

Note: Unlike CPAN, CPAN::Site is designed so that some of the files that describe the available packages are not found in all of the locations in your URL list. It is normal to see some "file not found" errors during the index reload.

6. Install the Infoblox package on your management system.

The usual CPAN process for installing Perl modules is:

6.1 Search

6.2 Test

6.3 Install

6.4 Uninstall and Reinstall

6.1 Search

Use the m operator to search for the Infoblox module:

```
 cpan> m /Infoblox/
 CPAN: Storable loaded ok
 Going to read /root/.cpan/Metadata
 Database was generated on Mon, 18 Sep 2006 06:29:55 GMT
 CPAN: LWP::UserAgent loaded ok
 . . .
 Fetching with LWP:
 https://<ip_addr>/api/dist/CPAN/site/01mailrc.txt.gz
 Going to read /root/.cpan/sources/site/01mailrc.txt.gz
 Module id = Infoblox
 CPAN_USERID  INFOBLOX (Infoblox Technical Support <support@infoblox.com>)
 CPAN_VERSION 4.9999990014709
 CPAN_FILE    INFOBLOX/Infoblox-4.9999990014709.tar.gz
 INST_FILE    (not installed)
```

If the search cannot find the Infoblox modules:

a. Ensure that you are using CPAN::Site, not CPAN as shown in 2. Run CPAN::Site, instead of CPAN. b. Check the Infoblox appliance in the urllist as shown in step 3.3.

6.2 Test

You can optionally test the package before the installation:

```
cpan> test Infoblox
Running test for module Infoblox
Running make for Infoblox-4.999990014709.tar.gz
...
All tests successful.
Files=1, Tests=1,  0 wallclock secs ( 0.74 cusr +  0.08 csys =  0.82 CPU)
/usr/bin/make test - OK
```

6.3 Install

Install the Infoblox package:

```
cpan> install Infoblox
Running install for module Infoblox
Running make for INFOBLOX/Infoblox-4.9999990014709.tar.gz
...
Writing /usr/lib/perl5/site_perl/5.8.8/i386-linux-thread-multi/auto/Infoblox/.packlist
Appending installation info to /usr/lib/perl5/5.8.8/i386-linux-thread-multi/perllocal.pod
/usr/bin/make install  -- OK
```

To exit the CPAN::Site shell:

```
cpan> quit
```

The installation is complete.

6.4 Uninstalling and Reinstalling the Infoblox Package

Although not directly supported by CPAN, it is simple to uninstall the Infoblox package using published techniques. One way to do it is provided by the modrm program that is described in the ExtUtils::Packlist documentation. See http://search.cpan.org/~yves/ExtUtils-Install-1.41/lib/ExtUtils/Packlist.pm#EXAMPLE for the modrm program. To reinstall it, repeat steps 5 and 6 above.

Infoblox::CiscoISE::EAAssociation - A Cisco ISE extensible attributes association.

- NAME
- DESCRIPTION
- CONSTRUCTOR
- SESSION METHODS
- MODULE METHODS
  - Infoblox::CiscoISE::SubscribeSetting->mapped_ea_attributes( )
- METHODS
  - name( )
  - mapped_ea( )
- AUTHOR
- SEE ALSO
- COPYRIGHT

NAME

Infoblox::CiscoISE::EAAssociation - A Cisco ISE extensible attributes association.

DESCRIPTION

The Cisco ISE extensible attribute association provides information on NIOS extensible attributes mapping on Cisco ISE attributes.

CONSTRUCTOR

```
my $ea_assoc = Infoblox::CiscoISE::EAAssociation->new(
    name      => 'ACCOUNT_SESSION_ID' | 'AUDIT_SESSION_ID' |
                 'EPS_STATUS' | 'IP_ADDRESS' |'MAC' |'NAS_IP_ADDRESS' | 'NAS_PORT_ID' |
                 'POSTURE_STATUS' |'POSTURE_TIMESTAMP',                                  #Required
    mapped_ea => $string,                                                                #Required
```

SESSION METHODS

The object does not have any session methods.

MODULE METHODS

This section describes all the functions that you can apply to Cisco ISE extensible attributes association object.

Infoblox::CiscoISE::SubscribeSetting->mapped_ea_attributes( )

Use this method to set or retrieve the list of extensible attribute associations that are mapped to Cisco ISE attributes. See Infoblox::CiscoISE::SubscribeSetting->mapped_ea_attributes() for parameters and return values.

Example

```
#Get mapped_ea_attributes value
my $mapped_ea_attributes = $object->mapped_ea_attributes();

#Modify mapped_ea_attributes value
$object->mapped_ea_attributes([$ea_assoc1, $ea_assoc2]);
```

METHODS

This section describes all the methods that you can use to configure and retrieve the attribute values of a Cisco ISE extensible attribute association object.

name( )

Use this method to set or retrieve the Cisco ISE attribute name.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

The valid values are 'ACCOUNT_SESSION_ID', 'AUDIT_SESSION_ID', 'EPS_STATUS', 'IP_ADDRESS', 'MAC', 'NAS_IP_ADDRESS', 'NAS_PORT_ID', 'POSTURE_STATUS' and 'POSTURE_TIMESTAMP',

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get name value
my $name = $object->name();

#Modify name value
$object->name('EPS_STATUS');
```

mapped_ea( )

Use this method to set or retrieve the NIOS extensible attribute definition name.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

The valid value is a desired extensible attribute definition name in string format.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get mapped_ea value
my $mapped_ea = $object->mapped_ea();

#Modify mapped_ea value
$object->mapped_ea('nios_ead');
```

AUTHOR

Infoblox Inc. http://www.infoblox.com/

SEE ALSO

Infoblox::CiscoISE::SubscribeSetting, Infoblox::CiscoISE::SubscribeSetting->mapped_ea_attributes(),

---

COPYRIGHT

Copyright (c) 2017 Infoblox Inc.

Infoblox::CiscoISE::Endpoint - A Cisco ISE endpoint object.

- NAME
- DESCRIPTION
- CONSTRUCTOR
- SESSION METHODS
  - Infoblox::Session->add( )
  - Infoblox::Session->get( )
  - Infoblox::Session->modify( )
  - Infoblox::Session->remove( )
  - Infoblox::Session->search( )
- METHODS
  - address( )
  - bulk_download_certificate( )
  - bulk_download_certificate_subject( )
  - bulk_download_certificate_valid_from( )
  - bulk_download_certificate_valid_to( )
  - client_certificate( )
  - client_certificate_subject( )
  - client_certificate_valid_from( )
  - client_certificate_valid_to( )
  - comment( )
  - connection_status( )
  - connection_timeout( )
  - disable( )
  - extattrs( )
  - extensible_attributes( )
  - network_view( )
  - publish_settings( )
  - resolved_address( )
  - resolved_secondary_address( )
  - secondary_address( )
  - subscribe_settings( )
  - subscribing_member( )
  - type( )
  - version( )
- AUTHOR
- SEE ALSO
- COPYRIGHT

NAME

Infoblox::CiscoISE::Endpoint - A Cisco ISE endpoint object.

DESCRIPTION

The CiscoISE endpoint object provides configuration for third party Cisco ISE servers integration.

CONSTRUCTOR

```
 my $ise_endpoint = Infoblox::CiscoISE::Endpoint->new(
     address                  => $fqdn | $ipaddr,                                                  #
Required
     bulk_download_certificate => $path,                                                           #
Required
     client_certificate       => $path,                                                            #
Required
     subscribe_settings       => $subscribe_settings,                                              #
Required
     subscribing_member       => $string,                                                          #
Required
     version                  => 'VERSION_1_3' | 'VERSION_1_4' | 'VERSION_2_0',                    #
Required
     comment                  => $string,                                                          #
Optional / Default is undefined
     connection_timeout       => $uint,                                                            #
Optional / Default is 10
     disable                  => 'true' | 'false',                                                 #
Optional / Default is 'true'
     extattrs                 => { $string => $extattr, ... },                                     #
Optional / Default is undefined
     extensible_attributes    => {$string => $string | $num, $string => [ $string | $num, ... ], ... } #
```

```
Optional / Default is undefined
     network_view              => $string,                                    #
Optional / Default is 'default'
     publish_settings          => $publish_settings,                          #
Optional / Default is undefined
     secondary_address         => $fqdn | $ipaddr,                            #
Optional / Default is undefined
     type                      => 'TYPE_CISCO',                               #
Optional / Default is 'TYPE_CISCO',
 );
```

You cannot set both extattrs and extensible_attributes attributes at the same time.

SESSION METHODS

This section describes all the methods in an Infoblox::Session module that you can apply to a Cisco ISE endpoint object.

Infoblox::Session->add( )

Use this method to add the object to the Infoblox appliance. See Infoblox::Session->add() for parameters and return values.

Example

```
#Construct an object
my $object = Infoblox::CiscoISE::Endpoint->new(
    address                   => '10.0.0.1',
    bulk_download_certificate => '/path/to/bulk_download_cert.pem',
    client_certificate        => '/path/to/client_cert.pem',
    subscribe_settings        => $subscribe_settings,
    subscribing_member        => 'infoblox.com',
    version                   => 'VERSION_1_3',
);

#Submit for addition
my $response = $session->add($object);
```

Infoblox::Session->get( )

Use this method to retrieve all the matching objects from the Infoblox appliance. See Infoblox::Session->get() for parameters and return values.

Key References

```
Apply the following attributes to get a specific Cisco ISE endpoint object:

 address                   - Optional. The FQDN or IP address of the Cisco ISE endpoint.
 comment                   - Optional. The Cisco ISE endpoint comment in string format
 extattrs                  - Optional. A hash reference containing Infoblox::Grid::Extattr object.
 extensible_attributes     - Optional. A hash reference containing extensible attributes.
 network_view              - Optional. The Cisco ISE endpoint network view name.
 resolved_address          - Optional. The resolved IP address of the Cisco ISE endpoint.
 resolved_secondary_address - Optional. The resolved secondary IP address of the Cisco ISE endpoint.
 secondary_address         - Optional. The secondary FQDN or IP address of the Cisco ISE endpoint.
 subscribing_member        - Optional. The Cisco ISE endpoint subscribing member name.
 type                      - Optional. The Cisco ISE endpoint type.
 version                   - Optional. The Cisco ISE endpoint version.
```

Examples

```
my @retrieved_objs = $session->get(
    object  => 'Infoblox::CiscoISE::Endpoint',
    address => '10.0.0.1',
);

my @retrieved_objs = $session->get(
    object               => 'Infoblox::CiscoISE::Endpoint',
    extensible_attributes => {'Site' => 'Santa Clara'},
);
```

Infoblox::Session->modify( )

Use this method to modify the object in the Infoblox appliance. See Infoblox::Session->modify() for parameters and return values.

Example

```
#Modify comment value
$object->comment('this is a modified comment');

#Submit modification
```

```
my $response = $session->modify($object);
```

Infoblox::Session->remove( )

Use this method to remove the object from the Infoblox appliance. See Infoblox::Session->remove() for parameters and return values.

To remove a specific object, first use `get()` or `search()` to retrieve the specific DHCP range object, and then submit this object for removal.

Example

```
#Get the objects
my @retrieved_objs = $session->get(
    object       => 'Infoblox::CiscoISE::Endpoint',
    address      => '10.0.0.1',
    newtork_view => 'default',
);

#Find the desired object from the retrieved list
my $desired_object = $retrieved_objs[0];

#Submit for removal
my $response = $session->remove($desired_object);
```

Infoblox::Session->search( )

Use this method to search for objects in the Infoblox appliance. See Infoblox::Session->search() for parameters and return values.

Key References

```
Apply the following attributes to search for the Cisco ISE endpoint object:

 address                    - Optional. The FQDN or IP address of the Cisco ISE endpoint (regexp).
 comment                    - Optional. The Cisco ISE endpoint comment in string format (regexp).
 extattrs                   - Optional. A hash reference containing Infoblox::Grid::Extattr object.
 extensible_attributes      - Optional. A hash reference containing extensible attributes.
 network_view               - Optional. The Cisco ISE endpoint network view name (exact).
 resolved_address           - Optional. The resolved IP address of the Cisco ISE endpoint (regexp).
 resolved_secondary_address - Optional. The resolved secondary IP address of the Cisco ISE endpoint
(regexp).
 secondary_address          - Optional. The secondary FQDN or IP address of the Cisco ISE endpoint
(regexp).
 subscribing_member         - Optional. The Cisco ISE endpoint subscribing member name (exact).
 type                       - Optional. The Cisco ISE endpoint type (exact).
 version                    - Optional. The Cisco ISE endpoint version (exact).
```

For more information about searching extensible attributes, see Infoblox::Grid::ExtensibleAttributeDef/Searching Extensible Attributes.

Examples

```
# search for objects
my @retrieved_objs = $session->search(
    object       => 'Infoblox::CiscoISE::Endpoint',
    network_view => 'default',
    address      => '10.*',
    comment      => '.*comment',
);

# search for objects defining 'Santa Clara' for 'Site' extensible attribute
my @retrieved_objs = $session->search(
    object               => 'Infoblox::DHCP::Range',
    extensible_attributes => {'Site' => 'Santa Clara'},
);
```

METHODS

This section describes all the methods that you can use to configure and retrieve the attribute values of a Cisco ISE endpoint.

address( )

Use this method to set or retrieve the fully-qualified domain name (FQDN) or IP address of the Cisco ISE endpoint.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

The valid value is a desired IP address or FQDN in string format.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get address value
my $address = $object->address();

#Modify address value
$object->address('foo.com');
```

bulk_download_certificate( )

Use this method to set the path to a bulk download certificate file. This is a write-only attribute.

Include the specified parameter to set the attribute value.

Parameter

The valid value is a path to a certificate in string format.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

Example

```
#Modify bulk_download_certificate value
$object->bulk_download_certificate('/path/to/cert.pem');
```

bulk_download_certificate_subject( )

Use this method to retrieve the bulk download certificate subject. This is a read-only attrbiute.

Omit the parameter to retrieve the attribute value.

Parameter

None

Returns

The method returns the attribute value.

Example

```
#Get bulk_download_certificate_subject value
my $bulk_download_certificate_subject = $object->bulk_download_certificate_subject();
```

bulk_download_certificate_valid_from( )

Use this method to retrieve the time from which the bulk download certificate is valid. This is a read-only attrbiute.

Omit the parameter to retrieve the attribute value.

Parameter

None

Returns

The method returns the attribute value. The number of seconds that have elapsed since January 1st, 1970 UTC.

Example

```
#Get bulk_download_certificate_valid_from value
my $bulk_download_certificate_valid_from = $object->bulk_download_certificate_valid_from();
```

bulk_download_certificate_valid_to( )

Use this method to retrieve the bulk download certificate expiration time. This is a read-only attrbiute.

Omit the parameter to retrieve the attribute value.

Parameter

None

Returns

The method returns the attribute value. The number of seconds that have elapsed since January 1st, 1970 UTC.

Example

```
#Get bulk_download_certificate_valid_to value
my $bulk_download_certificate_valid_to = $object->bulk_download_certificate_valid_to();
```

client_certificate( )

Use this method to set the path to a client certificate file. This is a write-only attribute.

Include the specified parameter to set the attribute value.

Parameter

The valid value is a path to a certificate in string format.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

Example

```
#Modify client_certificate value
$object->client_certificate('/path/to/cert.pem');
```

client_certificate_subject( )

Use this method to retrieve the client certificate subject. This is a read-only attrbiute.

Omit the parameter to retrieve the attribute value.

Parameter

None

Returns

The method returns the attribute value.

Example

```
#Get client_certificate_subject value
my $client_certificate_subject = $object->client_certificate_subject();
```

client_certificate_valid_from( )

Use this method to retrieve the time from which the client certificate is valid. This is a read-only attrbiute.

Omit the parameter to retrieve the attribute value.

Parameter

None

Returns

The method returns the attribute value. The number of seconds that have elapsed since January 1st, 1970 UTC.

Example

```
#Get client_certificate_valid_from value
my $client_certificate_valid_from = $object->client_certificate_valid_from();
```

client_certificate_valid_to( )

Use this method to retrieve the bulk client certificate expiration time. This is a read-only attrbiute.

Omit the parameter to retrieve the attribute value.

Parameter

None

Returns

The method returns the attribute value. The number of seconds that have elapsed since January 1st, 1970 UTC.

Example

```
#Get client_certificate_valid_to value
my $client_certificate_valid_to = $object->client_certificate_valid_to();
```

comment( )

Use this method to set or retrieve the Cisco ISE endpoint comment.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

The valid value is a desired comment in string format.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get comment value
my $comment = $object->comment();

#Modify comment value
$object->comment('ise comment');
```

connection_status( )

Use this method to retrieve the Cisco ISE endpoint connection status. This is a read-only attrbiute.

Omit the parameter to retrieve the attribute value.

Parameter

None

Returns

The method returns the attribute value.

Example

```
#Get connection_status value
my $connection_status = $object->connection_status();
```

connection_timeout( )

Use this method to set or retrieve the timeout value (in seconds) for the Cisco ISE endpoint connections.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

The valid value is an unsigned integer between 1 and 180. The default value is 10.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get connection_timeout value
my $connection_timeout = $object->connection_timeout();

#Modify connection_timeout value
$object->connection_timeout('20');
```

disable( )

Use this method to set or retrieve the flag that indicates whether the Cisco ISE endpoint is enabled or disabled.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

Specify 'true' to disable the Cisco ISE endpoint or 'false' to enable it. The default value is 'true'.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get disable value
my $disable = $object->disable();

#Modify disable value
$object->disable('false');
```

extattrs( )

Use this method to set or retrieve the extensible attributes associated with a Cisco ISE endpoint object.

Parameter

Valid value is a hash reference that contains the names of extensible attributes and their associated values (Infoblox::Grid::Extattr objects).

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get extattrs value
my $extattrs = $object->extattrs();

#Modify extattrs value
$dhcp_range->extattrs({'Site' => $extattr1, 'Administrator' => $extattr2});
```

extensible_attributes( )

Use this method to set or retrieve the extensible attributes associated with a Cisco ISE endpoint object.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

For valid values for extensible attributes, see Infoblox::Grid::ExtensibleAttributeDef/Extensible Attribute Values.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get extensible attributes value
my $extensible_attributes = $object->extensible_attributes();
#Modify extensible attributes
$object->extensible_attributes({'Site' => 'Santa Clara', 'Administrator' => ['Peter', 'Tom']});
```

network_view( )

Use this method to set or retrieve the Cisco ISE endpoint network view name.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

The valid value is a desired network view name in string format. The default value is 'default'.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get network_view value
my $network_view = $object->network_view();

#Modify network_view value
$object->network_view('internal');
```

publish_settings( )

Use this method to set or retrieve the Cisco ISE endpoint publish settings.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

The valid value is an Infoblox::CiscoISE::PublishSetting object.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get publish_settings value
my $publish_settings = $object->publish_settings();

#Modify publish_settings value
$object->publish_settings($publish_settings);
```

resolved_address( )

Use this method to retrieve the resolved IP address of the Cisco ISE endpoint. This is a read-only attribute.

Omit the parameter to retrieve the attribute value.

Parameter

None

Returns

The method returns the attribute value.

Example

```
#Get resolved_address value
my $resolved_address = $object->resolved_address();
```

resolved_secondary_address( )

Use this method to retrieve the resolved secondary IP address of the Cisco ISE endpoint. This is a read-only attribute.

Omit the parameter to retrieve the attribute value.

Parameter

None

Returns

The method returns the attribute value.

Example

```
#Get resolved_secondary_address value
my $resolved_secondary_address = $object->resolved_secondary_address();
```

secondary_address( )

Use this method to set or retrieve the secondary fully-qualified domain name (FQDN) or IP address of the Cisco ISE endpoint.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

The valid value is a desired IP address or FQDN in string format.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get secondary_address value
my $secondary_address = $object->secondary_address();

#Modify secondary_address value
$object->secondary_address('foo.com');
```

subscribe_settings( )

Use this method to set or retrieve the Cisco ISE endpoint subscribe settings.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

The valid value is an Infoblox::CiscoISE::SubscribeSetting object.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get subscribe_settings value
my $subscribe_settings = $object->subscribe_settings();

#Modify subscribe_settings value
$object->subscribe_settings($subscribe_settings);
```

subscribing_member( )

Use this method to set or retrieve the Cisco ISE endpoint subscribing member host name.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

The valid value is a desired subscribing member host name in string format. The default value is 'default'.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get subscribing_member value
my $subscribing_member = $object->subscribing_member();

#Modify subscribing_member value
$object->subscribing_member('infoblox.com');
```

type( )

Use this method to set or retrieve the Cisco ISE endpoint type.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

The valid value is 'TYPE_CISCO'.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get type value
my $type = $object->type();

#Modify type value
$object->type('TYPE_CISCO');
```

version( )

Use this method to set or retrieve the Cisco ISE endpoint version.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

The valid values are 'VERSION_1_3', 'VERSION_1_4' and 'VERSION_2_0'.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get version value
my $version = $object->version();

#Modify version value
$object->version('VERSION_2_0');
```

AUTHOR

Infoblox Inc. http://www.infoblox.com/

SEE ALSO

Infoblox::Session, Infoblox::Session->add(), Infoblox::Session->get(), Infoblox::Session->modify(), Infoblox::Session->remove(), Infoblox::Session->search(), Infoblox::Grid::Extattr, Infoblox::Grid::ExtensibleAttributeDef/Searching Extensible Attributes, Infoblox::CiscoISE::PublishSetting, Infoblox::CiscoISE::SubscribeSetting,

---

COPYRIGHT

Copyright (c) 2017 Infoblox Inc.

Infoblox::CiscoISE::PublishSetting - A Cisco ISE publish settings.

- NAME
- DESCRIPTION
- CONSTRUCTOR
- SESSION METHODS
- MODULE METHODS
  - Infoblox::CiscoISE::Endpoint->publish_settings( )
  - Infoblox::Notification::Rule->publish_settings( )
- METHODS
  - enabled_attributes( )
- AUTHOR
- SEE ALSO
- COPYRIGHT

NAME

Infoblox::CiscoISE::PublishSetting - A Cisco ISE publish settings.

DESCRIPTION

The Cisco ISE publish setting contains a list of extensible attributes that are enabled for publishing to a Cisco ISE server.

CONSTRUCTOR

```
my $publish_setting = Infoblox::CiscoISE::PublishSetting->new(
    enabled_attributes => ['CLIENT_ID' | 'FINGERPRINT' | 'HOSTNAME' | 'IPADDRESS' |
                           'INFOBLOX_MEMBER' | 'LEASE_STATE' | 'LEASE_START_TIME' |
                           'LEASE_END_TIME' | 'MAC_OR_DUID' | 'NETBIOS_NAME', ...],
);
```

SESSION METHODS

The object does not have any session methods.

MODULE METHODS

This section describes all the functions that you can apply to Cisco ISE publish settings object.

Infoblox::CiscoISE::Endpoint->publish_settings( )

Use this method to set or retrieve the Cisco ISE endpoint publish settings. See Infoblox::CiscoISE::Endpoint->publish_settings() for parameters and return values.

Example

```
#Get publish_settings value
my $publish_settings = $object->publish_settings();

#Modify publish_settings value
$object->publish_settings($publish_settings);
```

Infoblox::Notification::Rule->publish_settings( )

Use this method to set or retrieve the notification rule publish settings. See Infoblox::Notification::Rule->publish_settings() for parameters and return values.

Example

```
#Get publish_settings value
my $publish_settings = $object->publish_settings();

#Modify publish_settings value
$object->publish_settings($publish_settings);
```

METHODS

This section describes all the methods that you can use to configure and retrieve the attribute values of a Cisco ISE publish settings.

enabled_attributes( )

Use this method to set or retrieve the list of extensible attributes that are enabled for publishing to a Cisco ISE server.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

The valid value is an array that contains following values: 'CLIENT_ID', 'FINGERPRINT', 'HOSTNAME', 'IPADDRESS', 'INFOBLOX_MEMBER', 'LEASE_STATE', 'LEASE_START_TIME', 'LEASE_END_TIME', 'MAC_OR_DUID', 'NETBIOS_NAME'.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get enabled_attributes value
my $enabled_attributes = $object->enabled_attributes();

#Modify enabled_attributes value
$object->enabled_attributes(['CLIENT_ID', 'IPADDRESS']);
```

AUTHOR

Infoblox Inc. http://www.infoblox.com/

SEE ALSO

Infoblox::CiscoISE::Endpoint, Infoblox::CiscoISE::Endpoint->publish_settings(), Infoblox::Notification::Rule, Infoblox::Notification::Rule->publish_settings(),

---

COPYRIGHT

Copyright (c) 2017 Infoblox Inc.

Infoblox::CiscoISE::PublishSetting - A Cisco ISE subscribe settings.

- NAME
- DESCRIPTION
- CONSTRUCTOR
- SESSION METHODS
- MODULE METHODS
  - Infoblox::CiscoISE::Endpoint->subscribe_settings( )
- METHODS
  - enabled_attributes( )
  - mapped_ea_attributes( )
- AUTHOR
- SEE ALSO
- COPYRIGHT

NAME

Infoblox::CiscoISE::PublishSetting - A Cisco ISE subscribe settings.

DESCRIPTION

The Cisco ISE subscribe setting contains list of extensible attributes that are enabled for subscription from Cisco ISE server.

CONSTRUCTOR

```
 my $subscribe_setting = Infoblox::CiscoISE::PublishSetting->new(
     enabled_attributes   => ['DOMAINNAME' | 'ENDPOINT_PROFILE' | 'SECURITY_GROUP' | 'SESSION_STATE' |
'SSID' | 'USERNAME' | 'VLAN', ...], #Required
     mapped_ea_attributes => [$ea_assoc1, $ea_assoc2, ...],
#Optional / Default is undefined
 );
```

SESSION METHODS

The object does not have any session methods.

MODULE METHODS

This section describes all the functions that you can apply to Cisco ISE subscribe settings object.

Infoblox::CiscoISE::Endpoint->subscribe_settings( )

Use this method to set or retrieve the Cisco ISE endpoint subscribe settings. See Infoblox::CiscoISE::Endpoint->subscribe_settings() for parameters and return values.

Example

```
#Get subscribe_settings value
my $subscribe_settings = $object->subscribe_settings();

#Modify subscribe_settings value
$object->subscribe_settings($subscribe_settings);
```

METHODS

This section describes all the methods that you can use to configure and retrieve the attribute values of a Cisco ISE subscribe settings.

enabled_attributes( )

Use this method to set or retrieve the list of attributes that are enabled for subscription from the Cisco ISE server.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

The valid value is an array that contains following values: 'DOMAINNAME', 'ENDPOINT_PROFILE', 'SECURITY_GROUP', 'SESSION_STATE', 'SSID', 'USERNAME', 'VLAN'.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get enabled_attributes value
my $enabled_attributes = $object->enabled_attributes();

#Modify enabled_attributes value
$object->enabled_attributes(['DOMAINNAME', 'SECURITY_GROUP']);
```

mapped_ea_attributes( )

Use this method to set or retrieve the list of extensible attribute associations that are mapped to Cisco ISE attributes.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

The valid value is an array of Infoblox::CiscoISE::EAAssociation objects.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get mapped_ea_attributes value
my $mapped_ea_attributes = $object->mapped_ea_attributes();

#Modify mapped_ea_attributes value
$object->mapped_ea_attributes([$ea_assoc1, $ea_assoc2]);
```

AUTHOR

Infoblox Inc. http://www.infoblox.com/

SEE ALSO

Infoblox::CiscoISE::Endpoint, Infoblox::CiscoISE::Endpoint->subscribe_settings(), Infoblox::CiscoISE::EAAssociation,

---

COPYRIGHT

Copyright (c) 2017 Infoblox Inc.

Infoblox::Cursor - Manages paged searches.

- NAME
- DESCRIPTION
- CONSTRUCTOR
- METHODS
  - fetch_size( )
  - other parameters
  - Supported object types
- SAMPLE CODE
- AUTHOR
- SEE ALSO
- COPYRIGHT

NAME

Infoblox::Cursor - Manages paged searches.

DESCRIPTION

This class enables users to conduct database searches returning large datasets in an object-by-object manner.

CONSTRUCTOR

```
my $cursor=$session->new_cursor(
                                fetch_size     => 200,
                                object         => "Infoblox::DNS::AllRecords",
                                return_methods => ['name'],
                                name           => ".*"
                               );
```

METHODS

This section describes all the methods that you can use when creating a cursor used for paged searches.

fetch_size( )

This method is used to set the amount of records returned from the back-end to the client for every request. Larger values will cause less requests to be sent to the back-end and more memory used on the client-side; smaller values will cause more requests.

The default value is 100.

other parameters

Other parameters for this constructor are any valid search parameters as described in Infoblox::Session->search()

Supported object types

Paged search is supported for the following object types:

Infoblox::DNS::AllRecords
Infoblox::DNS::Zone
Infoblox::DHCP::FixedAddr
Infoblox::DHCP::Network
Infoblox::DNS::Host
Infoblox::DNS::Record::A
Infoblox::DNS::Record::AAAA
Infoblox::DNS::Record::PTR
Infoblox::DNS::Record::TXT
Infoblox::DNS::Record::MX
Infoblox::DNS::Record::SRV
Infoblox::DNS::Record::CNAME
Infoblox::DNS::Record::DNAME

SAMPLE CODE

The following sample code demonstrates how to execute a paged search.

```
#PROGRAM STARTS: Include all the modules that will be used
```

```
$cursor=$session->new_cursor(
                             fetch_size => 100,
                             object => "Infoblox::DNS::AllRecords",
                             name   => '.*',
                            );

while ($current_record = $cursor->fetch()) {
    # $current_record will contain a matching record object
}
```

AUTHOR

Infoblox Inc. http://www.infoblox.com/

SEE ALSO

Infoblox::Session, Infoblox::Session->search()

COPYRIGHT

Copyright (c) 2017 Infoblox Inc.

Infoblox::DHCP::DDNS - DHCP DDNS object.

- NAME
- DESCRIPTION
- CONSTRUCTOR
- SESSION METHODS
- METHODS
  - dns_ext_zone( )
  - dns_grid_zone( )
  - view( )
  - external_primary( )
  - grid_primary( )
  - zone_match( )
- AUTHOR
- SEE ALSO
- COPYRIGHT

NAME

Infoblox::DHCP::DDNS - DHCP DDNS object.

DESCRIPTION

The object is used to configure Multi-Grid Master preferences for updates from DHCP to Multi-Grid Master zones.

CONSTRUCTOR

```
my $ddns = Infoblox::DHCP::DDNS->new(
    dns_ext_zone      => $string,           # required if zone_match is 'EXTERNAL'
    dns_grid_zone     => $string,           # required if zone_match is 'GRID'
    view              => $string,           # optional / The default value is the name of the default view
    external_primary  => $ipaddr,           # required if zone_match is 'EXTERNAL' or 'ANY_EXTERNAL'
    grid_primary      => $string,           # required if zone_match is 'GRID' or 'ANY_GRID'
    zone_match        => 'ANY_EXTERNAL' | 'ANY_GRID' | 'EXTERNAL' | 'GRID',  # required
);
```

SESSION METHODS

The object doesn't support any session methods.

METHODS

This section describes all the methods that you can use to configure and retrieve a DHCP DDNS object.

dns_ext_zone( )

Use this method to set or retrieve a zone name, in FQDN (Fully Qualified Domain Name) format, that is contained in an external member.

Parameter

A zone name in FQDN format.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
# Get the attribute value
my $value = $ddns->dns_ext_zone();
# Modify the attribute value
$ddns->dns_ext_zone('exmmdns.com');
```

dns_grid_zone( )

Use this method to set or retrieve a zone name, in FQDN (Fully Qualified Domain Name) format, that is contained in a Grid member.

Parameter

A zone name in FQDN format.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
# Get the attribute value
my $value = $ddns->dns_grid_zone();
# Modify the attribute value
$ddns->dns_grid_zone('mmdns.com');
```

view( )

Use this method to set or retrieve the name of the DNS view in which the zone is located.

Parameter

The name of the DNS view in string format. The default value is the name of the default view.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
# Get the attribute value
my $value = $ddns->view();
# Modify the attribute value
$ddns->view('dnsview');
```

external_primary( )

Use this method to set or retrieve the IP address of an external member.

Parameter

The IP address of an external member.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
# Get the attribute value
my $value = $ddns->external_primary();
# Modify the attribute value
$ddns->external_primary('10.0.0.1');
```

grid_primary( )

Use this method to set or retrieve the name of the primary Grid member.

Parameter

The name of the primary Grid member.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
# Get the attribute value
my $value = $ddns->grid_primary();
# Modify the attribute value
$ddns->grid_primary('grid.com');
```

zone_match( )

Use this method to set or retrieve the matching type.

Parameter

The valid value is 'ANY_EXTERNAL', 'ANY_GRID', 'EXTERNAL' or 'GRID'.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
# Get the attribute value
my $value = $ddns->zone_match();
# Modify the attribute value
$ddns->zone_match('GRID');
```

AUTHOR

Infoblox Inc. http://www.infoblox.com/

SEE ALSO

Infoblox::DHCP::View, Infoblox::Grid::Member::DHCP

COPYRIGHT

Copyright (c) 2017 Infoblox Inc.

Infoblox::DHCP::ExclusionRange - DHCP ExclusionRange object.

- NAME
- DESCRIPTION
- CONSTRUCTOR
- MODULE METHODS
  - Infoblox::DHCP::Range->exclude( )
- METHODS
  - comment( )
  - end_address( )
  - start_address( )
- SAMPLE CODE
- AUTHOR
- SEE ALSO
- COPYRIGHT

NAME

Infoblox::DHCP::ExclusionRange - DHCP ExclusionRange object.

DESCRIPTION

The DHCP exclusion range describes the range of IP addresses that can be excluded in an address range. If static addresses are assigned to certain hosts in the middle of an address range, those addresses can be excluded from the address range so the DHCP server does not assign those IP addresses to clients.

CONSTRUCTOR

```
my $exclusion_range = Infoblox::DHCP::ExclusionRange->new(
      end_address   => $ipv4addr,      #Required
      start_address => $ipv4addr,      #Required
      comment       => $string,        #Optional / Default is empty
);
```

MODULE METHODS

The following functions are available to apply to a DHCP exclusion range object.

Infoblox::DHCP::Range->exclude( )

Use this function to specify exclusion range for the DHCP range. See Infoblox::DHCP::Range->exclude() for parameters and return values.

Example

```
#Construct a DHCP ExclusionRange object
my $exclusion_range = Infoblox::DHCP::ExclusionRange->new(
         end_address   => "10.0.0.8",
         start_address => "10.0.0.5",
         comment       => "Exclude four IP addresses"
);

# Configure DHCP ExclusionRange on the DHCP Range object
my $response = $dhcp_range->exclude([$exclusion_range]);
```

METHODS

This section describes all the methods that can be used to configure and retrieve the attribute values of a DHCP exclusion range object.

comment( )

Use this method to set or retrieve a descriptive comment of a DHCP exclusion range object.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

Desired comment in string format with a maximum of 256 bytes.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get comment
my $comment = $exclusion_range->comment();
#Modify comment
$exclusion_range->comment("Modifying the DHCP exclusion range comment");
```

end_address( )

Use this method to set or retrieve the end address of a DHCP exclusion range object.

An end address is the last IP address in the DHCP exclusion range which is not available for the clients.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

Specify the end address of the DHCP exclusion range in IPv4 address format. An IPv4 address is a 32-bit number in dotted decimal notation. It consists of four 8-bit groups of decimal digits separated by decimal points (example: 192.168.1.2).

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get end_address
my $end_address = $exclusion_range->end_address();
#Modify end_address
$exclusion_range->end_address("10.0.0.9");
```

start_address( )

Use this method to set or retrieve the start address of a DHCP exclusion range object.

A start address is the first IP address in the DHCP exclusion range which is not available for the clients.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

Specify the start address of the DHCP exclusion range in IPv4 address format. An IPv4 address is a 32-bit number in dotted decimal notation. It consists of four 8-bit groups of decimal digits separated by decimal points (example: 192.168.1.2).

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get start_address
my $start_address = $exclusion_range->start_address();
#Modify start_address
$exclusion_range->start_address("10.0.0.6");
```

SAMPLE CODE

The following sample code demonstrates the different functions that can be applied to a DHCP exclusion range object such as add, modify, and remove. Also, this sample includes error handling for the operations.

#Preparation prior to a DHCP exclusion range object insertion

```
#PROGRAM STARTS: Include all the modules that will be used
use strict;
use Infoblox;

#Create a session to the Infoblox appliance
my $session = Infoblox::Session->new(
              master   => "192.168.1.2", #appliance host ip
                                         #appliance user login
```

```
                username => "admin",
                password => "infoblox"     #appliance password
);
unless ($session) {
       die("Construct session failed: ",
            Infoblox::status_code() . ":" . Infoblox::status_detail());
}
print "Session created successfully\n";

#Create the Network object
my $network = Infoblox::DHCP::Network->new(
    network => "10.0.0.0/8",
    comment => "add network",
);
unless($network) {
       die("Construct Network object failed: ",
            Infoblox::status_code() . ":" . Infoblox::status_detail());
}
print "Network object created successfully\n";

#Add the Network object into the Infoblox appliance through a session
$session->add($network)
    or die("Add Network object failed: ",
            $session->status_code() . ":" . $session->status_detail());
print "Network object added to Infoblox appliance successfully\n";

#Create the DHCP Range object
my $dhcp_range = Infoblox::DHCP::Range->new(
        end_addr   => "10.0.0.10",
        network    => "10.0.0.0/8",
        start_addr => "10.0.0.1",
);
unless($dhcp_range) {
       die("Construct DHCP Range object failed: ",
            Infoblox::status_code() . ":" . Infoblox::status_detail());
}
print "DHCP Range object created successfully\n";

#Add the DHCP Range object into the Infoblox appliance through a session
$session->add($dhcp_range)
    or die("Add DHCP Range object failed: ",
            $session->status_code() . ":" . $session->status_detail());
print "DHCP Range object added to Infoblox appliance successfully\n";
```

#Create a DHCP exclusion range object

```
my $exclusion_range = Infoblox::DHCP::ExclusionRange->new(
         end_address   => "10.0.0.8",
         start_address => "10.0.0.5",
         comment       => "Exclude four IP addresses"
);
unless($exclusion_range) {
       die("Construct DHCP exclusion range object failed: ",
            Infoblox::status_code() . ":" . Infoblox::status_detail());
}
print "DHCP exclusion range object created successfully\n";
```

#Add DHCP exclusion range to DHCP Range

```
#Get the DHCP Range object
my @retrieved_objs = $session->get(
    object     => "Infoblox::DHCP::Range",
    start_addr => "10.0.0.1"
);
my $object = $retrieved_objs[0];
unless ($object) {
       die("Get DHCP Range object failed: ",
            $session->status_code() . ":" . $session->status_detail());
}
print "Get DHCP Range object found at least 1 matching entry\n";

#Apply the changes to the DHCP Range object
$object->exclude([$exclusion_range]);

#Submit the changes to the Session
$session->modify($object)
    or die("Adding DHCP exclusion range to DHCP Range object failed: ",
```

```
            $session->status_code() . ":" . $session->status_detail());
print "DHCP Range with exclusion range updated to Infoblox appliance successfully\n";
```

#Modify DHCP exclusion range

```
#Modifying the end address of the DHCP exclusion range object.
$exclusion_range->end_address("10.0.0.9");

#Modifying the start address of the DHCP exclusion range object.
$exclusion_range->start_address("10.0.0.6");

#Modifying the comment of the DHCP exclusion range object.
$exclusion_range->comment("Modified DHCP Exclusion Range");

#Apply changes to the DHCP Range object.
$object->exclude([$exclusion_range]);

#Update DHCP Range object through the Infoblox session.
$session->modify($object)
  or die("Modify DHCP Range object failed: ",
  $session->status_code() . ":" . $session->status_detail());
print "DHCP Range object with modified DHCP exclusion range updated to Infoblox appliance successfully\n";
```

#Remove DHCP exclusion range

```
#Remove the exclusion range from the DHCP Range object.
$object->exclude([]);

#Update DHCP Range object through the Infoblox session.
$session->modify($object)
  or die("Modify DHCP Range object failed: ",
  $session->status_code() . ":" . $session->status_detail());
print "Removed DHCP exclusion range from DHCP Range successfully\n";
```

####PROGRAM ENDS####

AUTHOR

Infoblox Inc. http://www.infoblox.com/

SEE ALSO

Infoblox::Session, Infoblox::Session->add(),Infoblox::Session->get(),Infoblox::Session->modify(),Infoblox::DHCP::Range,Infoblox::DHCP::Network

---

COPYRIGHT

Copyright (c) 2017 Infoblox Inc.

Infoblox::DHCP::ExclusionRangeTemplate - DHCP ExclusionRangeTemplate object.

- NAME
- DESCRIPTION
- CONSTRUCTOR
- MODULE METHODS
  - Infoblox::DHCP::RangeTemplate->exclude( )
- METHODS
  - comment( )
  - number_of_addresses( )
  - offset( )
- SAMPLE CODE
- AUTHOR
- SEE ALSO
- COPYRIGHT

NAME

Infoblox::DHCP::ExclusionRangeTemplate - DHCP ExclusionRangeTemplate object.

DESCRIPTION

The DHCP exclusion range template describes the range of IP addresses that can be excluded in an address range template. If static IP addresses are assigned to certain hosts in the middle of an address range, those addresses can be excluded from the address range so the DHCP server does not assign those IP addresses to the clients.

CONSTRUCTOR

```
my $exclusion_range_template = Infoblox::DHCP::ExclusionRangeTemplate->new(
      offset               => $num,          #Required
      number_of_addresses  => $num,          #Required
      comment              => $string,       #Optional / Default is empty
);
```

MODULE METHODS

The following functions are available to be applied to a DHCP exclusion range template object.

Infoblox::DHCP::RangeTemplate->exclude( )

Use this function to specify exclusion range for the DHCP range template object. See Infoblox::DHCP::RangeTemplate->exclude() for parameters and return values.

Example

```
#Construct a DHCP ExclusionRangeTemplate object
my $exclusion_range_template = Infoblox::DHCP::ExclusionRangeTemplate->new(
        offset               => "10",
        number_of_addresses  => "5",
        comment              => "Exclude five IP addresses"
);

# Configure DHCP ExclusionRangeTemplate on the DHCP Range template object
my $response = $dhcp_range_template->exclude([$exclusion_range_template]);
```

METHODS

This section describes all the methods that can be used to configure and retrieve the attribute values of a DHCP exclusion range template object.

comment( )

Use this method to set or retrieve a descriptive comment of a DHCP exclusion range template object.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

Desired comment in string format with a maximum of 256 bytes.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get comment
my $comment = $exclusion_range_template->comment();
#Modify comment
$exclusion_range_template->comment("Modifying the DHCP exclusion range template comment");
```

number_of_addresses( )

Use this method to set or retrieve the number of addresses in the DHCP exclusion range template object.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

The number of addresses in this exclusion range template.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get number_of_addresses
my $number_of_addresses = $exclusion_range_template->number_of_addresses();
#Modify number_of_addresses
$exclusion_range_template->number_of_addresses("10");
```

offset( )

Use this method to set or retrieve the address offset of the DHCP exclusion range template object.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

The address offset of this exclusion range template. The offset is calculated from the start address of network template to which the DHCP range template is assigned to. For example, if offset is set to 10 and number_of_addresses is set to 10, and network is set to 10.0.0.0/8, then start address of exclusion range created from this exclusion range template will be 10.0.0.10 and end address will be 10.0.0.20.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get offset
my $offset = $exclusion_range_template->offset();
#Modify offset
$exclusion_range_template->offset("10");
```

SAMPLE CODE

The following sample code demonstrates the different functions that can be applied to a DHCP exclusion range object such as add, modify, and remove. Also, this sample includes error handling for the operations.

#PROGRAM STARTS: Include all the modules that will be used

```
use strict;
use Infoblox;

#Create a session to the Infoblox appliance
my $session = Infoblox::Session->new(
              master   => "192.168.1.2", #appliance host ip
              username => "admin",       #appliance user login
              password => "infoblox"     #appliance password
```

```
);
unless ($session) {
       die("Construct session failed: ",
            Infoblox::status_code() . ":" . Infoblox::status_detail());
}
print "Session created successfully\n";
```

#Create the DHCP Range template object

```
my $dhcp_range_template = Infoblox::DHCP::RangeTemplate->new(
        name                    => "custom_range_template",
        offset                  => "10",
        number_of_addresses     => "10",
        authority               => "TRUE",
        bootfile                => "bootfile1.com",
);
unless($dhcp_range_template) {
       die("Construct DHCP Range template object failed: ",
            Infoblox::status_code() . ":" . Infoblox::status_detail());
}
print "DHCP Range template object created successfully\n";

#Add the DHCP Range object into the Infoblox appliance through a session
$session->add($dhcp_range_template)
    or die("Add DHCP Range template object failed: ",
            $session->status_code() . ":" . $session->status_detail());
print "DHCP Range object added to Infoblox appliance successfully\n";
```

#Create a DHCP exclusion range templat object

```
my $exclusion_range_template = Infoblox::DHCP::ExclusionRangeTemplate->new(
         offset              => "10",
         number_of_addresses => "5",
         comment             => "Exclude five IP addresses"
);
unless($exclusion_range_template) {
       die("Construct DHCP exclusion range template object failed: ",
            Infoblox::status_code() . ":" . Infoblox::status_detail());
}
print "DHCP exclusion range template object created successfully\n";
```

#Add DHCP exclusion range template to DHCP Range template

```
#Get the DHCP Range template object
my @retrieved_objs = $session->get(
    object     => "Infoblox::DHCP::RangeTemplate",
    name       => "custom_range_template"
);
my $object = $retrieved_objs[0];
unless ($object) {
       die("Get DHCP Range template object failed: ",
            $session->status_code() . ":" . $session->status_detail());
}
print "Get DHCP Range template object found at least 1 matching entry\n";

#Apply the changes to the DHCP Range template object
$object->exclude([$exclusion_range_template]);

#Submit the changes to the Session
$session->modify($object)
    or die("Adding DHCP exclusion range to DHCP Range template object failed: ",
            $session->status_code() . ":" . $session->status_detail());
print "DHCP Range template with exclusion range template updated to Infoblox appliance successfully\n";
```

#Remove DHCP exclusion range template

```
#Remove the exclusion range template from the DHCP Range template object.
$object->exclude([]);

#Update DHCP Range template object through the Infoblox session.
$session->modify($object)
  or die("Modify DHCP Range template object failed: ",
  $session->status_code() . ":" . $session->status_detail());
print "Removed DHCP exclusion range template from DHCP Range successfully\n";

####PROGRAM ENDS####
```

AUTHOR

Infoblox Inc. http://www.infoblox.com/

SEE ALSO

Infoblox::Session,Infoblox::DHCP::RangeTemplate,Infoblox::DHCP::NetworkTemplate

---

COPYRIGHT

Copyright (c) 2017 Infoblox Inc.

Infoblox::DHCP::ExpandNetwork - Join or Expand an existing network

- NAME
- DESCRIPTION
- CONSTRUCTOR
- SESSION METHODS
  - Infoblox::Session->add( )
- METHODS
  - auto_create_reversezone( )
  - network( )
  - option_delete_ea( )
  - prefix( )
- SAMPLE CODE
- AUTHOR
- SEE ALSO
- COPYRIGHT

NAME

Infoblox::DHCP::ExpandNetwork - Join or Expand an existing network

---

DESCRIPTION

Expanding/Joining multiple networks into a larger network is the opposite of spliting a network. You can select a network and expand it into a larger network with smaller netmask.

CONSTRUCTOR

```
#Constructor a DHCP Expand Network object
my $expand_network = Infoblox::DHCP::ExpandNetwork->new (
       network                => $Network|$IPv6Network, #Required
       prefix                 => $num,                 #Required
       auto_create_reversezone => "true" | "false",      #Optional / Default is "false"
       option_delete_ea        => "RETAIN" | "REMOVE",   #Optional / Default is "REMOVE"
    );
```

SESSION METHODS

This section describes all the methods in Infoblox::Session that you can apply to a DHCP ExpandNetwork object.

Infoblox::Session->add( )

Use this method to expand an existing network on the Infoblox appliance. See Infoblox::Session->`add()` for parameters and return values

Example

```
# construct the network to be expanded
my $network = Infoblox::DHCP::Network->new(
         network => "123.0.0.0/16",
           );

# construct a Expand Network object
my $expand_network = Infoblox::DHCP::ExpandNetwork->new (
        network                => $network,
        prefix                 => "13",
        auto_create_reversezone => "false"
);

# Submit the Expand network operation.
$session->add($expand_network);
```

METHODS

This section describes all the methods that you can use to configure and retrieve the attribute value of a Expand Network.

auto_create_reversezone( )

Use this method to set the auto_create_reversezone flag. When set, this flag will automatically create reverse-mapping zones for the

subnet.

Include the specified parameter to set the attribute value.

Parameter

Specify "true" to set auto_create_reversezone flag or "false" to deactivate/unset it. The default value is "false".

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

Example

```
#Set auto_create_reversezone
$split_network->auto_create_reversezone("true");
```

network( )

Use this method to specify the network to be expanded.

Include the specified parameter to set the attribute value.

Parameter

The valid values are either Infoblox::DHCP::Network or Infoblox::DHCP::IPv6Network objects.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

Example

```
#Specify network
my $network = Infoblox::DHCP::Network->new(
                    network => "123.0.0.0/16",
                    );
$expand_network->network($network);

#Specify IPv6 network
my $network = Infoblox::DHCP::IPv6Network->new(
                    network => "12::/68",
                    );
$expand_network->network($network);
```

option_delete_ea( )

Use this method to specify the action for inheritable extensible attributes that exist on descendants if a parent network is deleted.

Parameter

Valid value is "RETAIN" or "REMOVE".

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Specify option_delete_ea
$expand_network->option_delete_ea('RETAIN');
```

prefix( )

Use this method to specify the appropriate subnet mask to be expanded on the network.

Include the specified parameter to set the attribute value.

Parameter

Number with the subnet mask value. This value must be 1 smaller then the subnet mask of the network to be expanded. The minimuim value is 8.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

Example

```
#Specify prefix
$expand_network->prefix(13);
```

SAMPLE CODE

The following sample code demonstrates different operations that can be applied to an object such as create, modify, and remove an object. This sample code also includes error handling for the operations.

```
#PROGRAM STARTS: Include all the modules that will be used
use strict;
use Infoblox;

#Create a session to the Infoblox appliance
my $session = Infoblox::Session->new(
    master   => "192.168.1.2",
    username => "admin",
    password => "infoblox"
);
unless ($session) {
   die("Construct session failed: ",
       Infoblox::status_code() . ":" . Infoblox::status_detail());
}
print "Session created successfully\n";
```

#Create network prior to splitting the network my $network = Infoblox::DHCP::Network->new(network => "123.0.0.0/16", comment => "add network",); unless($network){ die("Construct network object fail: ", Infoblox::status_code(). ":" .Infoblox::status_detail()); }

```
print"Network object created successfully.\n";

#Verify if the network exists
my $object = $session->get(
               object  => "Infoblox::DHCP::Network",
               network => "123.0.0.0/16"
               );
unless($object){
       print"Network does not exist on server, safe to add the network.\n";
       $session->add($network)
               or die("Add network failed: ",
                       $session->status_code(). ":" .$session->status_detail());
       }

print"Network added successfully.\n";
```

#Expand the network

```
my $expand_network = Infoblox::DHCP::ExpandNetwork->new (
                        network                => $network,
                        prefix                 => "13",
                        auto_create_reversezone => "false",
                      );
unless($expand_network){
       die("Construct expand network object fail: ",
               Infoblox::status_code(). ":" .Infoblox::status_detail());
       }

print"Expand network object created successfully.\n";

#Expand the network
$session->add( $expand_network )
         or die("Expand network fail:",
                     $session->status_code(). ":" .$session->status_detail());

print "Expand network successfully.\n";

####PROGRAM ENDS####
```

AUTHOR

Infoblox Inc. http://www.infoblox.com/

SEE ALSO

Infoblox::Session, Infoblox::Session->add(), Infoblox::DHCP::Network

---

COPYRIGHT

Copyright (c) 2017 Infoblox Inc.

Infoblox::DHCP::FailOver - DHCP Failover object.

- NAME
- DESCRIPTION
- CONSTRUCTOR
- SESSION METHODS
  - Infoblox::Session->add( )
  - Infoblox::Session->get( )
  - Infoblox::Session->modify( )
  - Infoblox::Session->remove( )
  - Infoblox::Session->search( )
- METHODS
  - association_type( )
  - ms_association_mode( )
  - ms_enable_authentication( )
  - ms_enable_switchover_interval( )
  - ms_failover_mode( )
  - ms_failover_partner( )
  - ms_hotstandby_partner_role( )
  - ms_is_conflict( )
  - ms_server( )
  - ms_shared_secret( )
  - ms_state( )
  - ms_switchover_interval( )
  - ms_previous_state( )
  - name( )
  - primary( )
  - secondary( )
  - comment( )
  - extattrs( )
  - extensible_attributes( )
  - load_balance_split( )
  - max_load_balance_delay( )
  - max_client_lead_time( )
  - max_unacked_updates( )
  - max_response_delay( )
  - primary_state( )
  - recycle_leases( )
  - secondary_state( )
- SAMPLE CODE
- AUTHOR
- SEE ALSO
- COPYRIGHT

NAME

Infoblox::DHCP::FailOver - DHCP Failover object.

DESCRIPTION

DHCP failover is a protocol designed to allow a backup DHCP server to take over for a main server if the main server is taken off the network for any reason. DHCP failover can be used to configure two DHCP servers to operate as a redundant pair.

CONSTRUCTOR

```
 my $dhcp_failover = Infoblox::DHCP::FailOver->new(
     name                   => $string,                              #Required
     primary                => $ipv4addr | $Member                   #Required
     secondary              => $ipv4addr | $Member                   #Required
     comment                => $string,                              #Optional / Default is empty
     extattrs               => { $string => $extattr, ... },         #Optional / Default is undefined
     extensible_attributes  => { $string => $string | $num, $string => [ $string | $num, ... ], ... },
#Optional / Default is undefined
     load_balance_split     => $num,                                 #Optional / Default value is 128
     max_load_balance_delay => $num,                                 #Optional / Default value is 3
     max_client_lead_time   => $num,                                 #Optional / Default value is 3600
     max_unacked_updates    => $num,                                 #Optional / Default value is 10
     max_response_delay     => $num,                                 #Optional / Default value is 60
     recycle_leases         => "true" | "false" | undef,             #Optional / Default value is undefined
 );
```

You cannot set both extattrs and extensible_attributes attributes at the same time.

SESSION METHODS

This section describes all the methods in Infoblox::Session module that can be applied to a DHCP failover object.

Infoblox::Session->add( )

Use this method to add a DHCP failover object to the Infoblox appliance. See Infoblox::Session->add() for parameters and return values.

Example

```
#Create a DHCP Member object
my $memberdhcp = Infoblox::DHCP::Member->new(
      name     => "infoblox.localdomain",
      ipv4addr => "192.168.1.2"
);

#Construct an object
my $dhcp_failover = Infoblox::DHCP::FailOver->new(
    name                  => "peer1",
    primary               => $memberdhcp,
    secondary             => "1.1.1.1",
    load_balance_split    => 128,
    max_load_balance_delay => 3,
    max_client_lead_time  => 3600,
    max_unacked_updates   => 10,
    max_response_delay    => 60,
    recycle_leases        => "false",
);

# Submit for addition
my $response = $session->add( $dhcp_failover );
```

Infoblox::Session->get( )

Use this method to retrieve all the matching DHCP failover objects from the Infoblox appliance. See Infoblox::Session->get() for parameters and return values.

Key References

```
Apply the following attributes to get a specific DHCP failover object:

name - Required. Name of a DHCP failover in string format.
extattrs     - Optional. A hash reference containing extensible attributes.
```

Example

```
my @retrieved_objs = $session->get(
                     object     => "Infoblox::DHCP::FailOver",
                     name       => "peer1" );
```

Infoblox::Session->modify( )

Use this method to modify a DHCP failover object in the Infoblox appliance. See Infoblox::Session->modify() for parameters and return values.

Example

```
# Use method to modify the load_balance_split.
$dhcp_failover->load_balance_split(130);
# Submit modification
my $response = $session->modify( $dhcp_failover );
```

Infoblox::Session->remove( )

Use this method to remove a DHCP failover object from the Infoblox appliance. See Infoblox::Session->remove() for parameters and return values.

To remove a specific object, first use `get()` or `search()` to retrieve the specific DHCP failover object, and then submit this object for removal.

Example

```
# Get the objects with the same name
my @retrieved objs   $session  get
```

```
                    =         ->    (
    object  => "Infoblox::DHCP::FailOver",
    name    => "peer1" );
# Find the desired object from the retrieved list.
my $desired_dhcp_failover = $retrieved_objs[0];
# Submit for removal
my $response = $session->remove( $desired_dhcp_failover );
```

Infoblox::Session->search( )

Use this method to search for DHCP failover objects in the Infoblox appliance. See Infoblox::Session->search() for parameters and return values.

Key References

Apply the following attributes, to search for a DHCP failover object:

```
name    - Optional. The name of a DHCP failover object. Regular expression supported.
comment - Optional. The comment of the DHCP failover object. Regular expression supported.
extattrs      - Optional. A hash reference containing extensible attributes.
extensible_attributes - Optional. A hash reference containing extensible attributes.
```

For more information about searching extensible attributes, see Infoblox::Grid::ExtensibleAttributeDef/Searching Extensible Attributes.

Example

```
# search for DHCP failover object with name as peer1
my @retrieved_objs = $session->search(
    object  => "Infoblox::DHCP::FailOver",
    name    => "pe.*" );

# search all DHCP failover associations with the extensible attribute 'Site'
my @retrieved_objs = $session->search(
   object => "Infoblox::DHCP::FailOver",
   extensible_attributes => { 'Site' => 'Santa Clara' });
```

METHODS

This section describes all the methods that can be used to configure and retrieve the attribute values of a DHCP failover object.

association_type( )

Use this method to retrieve the value indicating whether the failover assocation is Microsoft or Grid based. This is a read-only attribute.

Parameter

None

Returns

The return value can be "MS" or "GRID".

Example

```
#Get association type
my $association_type = $failover->association_type();
```

ms_association_mode( )

Use this method to retrieve the value that indicates whether the failover association is read-write or read-only. This is a read-only attribute.

Parameter

None

Returns

The valid return values are "RO" and "RW".

Example

```
#Get association_mode
my $association_mode = $failover->ms_association_mode();
```

ms_enable_authentication( )

Use this method to enable or disable authentication for the failover association.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

Specify "true" to enable authentication for the failover association or "false" to disable it. The default value is "false".

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get ms_enable_authentication value
my $enable_authentication_value = $failover->ms_enable_authentication();
#Modify ms_enable_authentication value
$failover->ms_enable_authentication("true");
```

ms_enable_switchover_interval( )

Use this method to enable or disable the switchover interval.

The ms_enable_switchover_interval attribute can be specified explicitly. It is also set implicitly when ms_switchover_interval is set to a defined value.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

Specify "true" to enable the switchover interval or "false" to disable it. The default value is "false".

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get ms_enable_switchover_interval value
my $enable_switchover_interval = $failover->ms_enable_switchover_interval();
#Modify ms_enable_switchover_interval value
$failover->ms_enable_switchover_interval("true");
```

ms_failover_mode( )

Use this method to set or retrieve the mode for the failover association.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

The valid parameters are "LOADBALANCE" and "HOTSTANDBY".

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

```
=item B<Example>

#Get ms_failover_mode
my $failover_mode = $failover->ms_failover_mode();
#Modify ms_failover_mode
$failover->ms_failover_mode("HOTSTANDBY");
```

ms_failover_partner( )

Use this method to set or retrieve the failover partner defined in the association for the Microsoft Server.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Note that the failover partner must be retrieved from the server settings and cannot be changed after being written.

Parameter

The valid parameter is an Infoblox::Grid::MSServer object.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get ms_failover_partner
my $failover_partner = $failover->ms_failover_partner();
#Modify ms_failover_partner
$failover->ms_failover_partner($failover_partner);
```

ms_hotstandby_partner_role( )

Use this method to set or retrieve the partner role in the case of HotStandby.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

The valid values are "ACTIVE" and "PASSIVE".

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get ms_hotstandby_partner_role
my $partner_role = $failover->ms_hotstandby_partner_role();
#Modify ms_hotstandby_partner_role
$failover->ms_hotstandby_partner_role("ACTIVE");

=back
```

ms_is_conflict( )

Use this method to retrieve the flag used to indicate whether the matching Microsfot failover association (if any) is in synchronization (False) or not (True). The method returns False if there is no matching failover association. This is a read-only attribute.

Parameter

None

Returns

The valid return values are "true" and "false".

Example

```
#Get ms_is_conflict value
my $is_conflict = $failover->ms_is_conflict();
```

ms_server( )

Use this method to set or retrieve the primary Microsoft Server object.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Note that the Microsoft server must be retrieved from the server settings and cannot be changed after being written.

Parameter

The valid parameter is an Infoblox::Grid::MSServer object.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get ms_server
my $server = $failover->ms_server();
#Modify ms_failover_partner
$failover->ms_server($server);
```

ms_shared_secret( )

Use this method to set the shared secret for the failover association authentication. This is a write-only attribute.

Parameter

The valid value is a string that contains the shared secret for the failover association authentication.

Returns

The method returns true when the modification succeeds, and returns false when the operation fails.

Example

```
#Set ms_shared_secret value
$failover->ms_shared_secret($shared_secret);
```

ms_state( )

Use this method to retrieve the failover association state. This is a read-only attribute.

Parameter

None

Returns

The valid return values are "NO_STATE", "INIT", "STARTUP", "NORMAL", "COMMUNICATION_INIT", "PARTNER_DOWN", "POTENTIAL_CONFLICT", "CONFLICT_DONE", "RESOLUTION_INIT", "RECOVER", "RECOVER_DONE", "RECOVER_WAIT".

Example

```
#Get ms_state value
my $state = $failover->ms_state();
```

ms_switchover_interval( )

Use this method to set or retrieve the time (in seconds) that DHCPv4 server will wait before transitioning the server from the COMMUNICATION-INT state to PARTNER-DOWN state.

Setting this method to a defined value implicitly sets the ms_enable_switchover_interval method to "true". Setting the parameter to undefined causes the appliance to use the grid default and automatically resets the ms_enable_switchover_interval method to "false".

Note that when ms_switchover_interval is set to a defined value and ms_enable_switchover_interval is set to "false", the last operation takes precedence. Thus the sequence $object->ms_switchover_interval(20); $object->ms_enable_switchover_interval("false"); will set ms_enable_switchover_interval to "false", and the sequence $object->ms_enable_switchover_interval("false"); $object->ms_switchover_interval(20); will result in ms_enable_switchover_interval="true".

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

The valid value are integers between 1 and 65535.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get ms_switchover_interval value
my $so_interval = $failover->ms_switchover_interval();
#Modify ms_switchover_interval value
$failover->ms_switchover_interval(1000);
```

ms_previous_state( )

Use this method to retrieve the previous failover association state. This is a read-only attribute.

Parameter

None

Returns

The valid return values are "NO_STATE", "INIT", "STARTUP", "NORMAL", "COMMUNICATION_INIT", "PARTNER_DOWN", "POTENTIAL_CONFLICT", "CONFLICT_DONE", "RESOLUTION_INIT", "RECOVER", "RECOVER_DONE", "RECOVER_WAIT".

Example

```
#Get ms_previous_state value
my $previous_state = $failover->ms_previous_state();
```

name( )

Use this method to set or retrieve the name of a DHCP failover object.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

Specify the name in string format.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get name
my $name = $dhcp_failover->name();
#Modify name
$dhcp_failover->name("peer2");
```

primary( )

Use this method to set or retrieve primary server of a DHCP failover object.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

Specify external primary server in IPv4 address format or grid primary server in a defined Infoblox::DHCP::Member object. An IPv4 address is a 32-bit number in dotted decimal notation. It consists of four 8-bit groups of decimal digits separated by decimal points (example: 192.168.1.2).

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get primary
my $primary = $dhcp_failover->primary();
#Modify primary
$dhcp_failover->primary($member); #$member is a Infoblox::DHCP::Member object
```

secondary( )

Use this method to set or retrieve secondary server of a DHCP failover object.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

Specify external secondary server in IPv4 address format or grid secondary server in a defined Infoblox::DHCP::Member object. An IPv4 address is a 32-bit number in dotted decimal notation. It consists of four 8-bit groups of decimal digits separated by decimal points (example: 192.168.1.2).

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get secondary
my $secondary = $dhcp_failover->secondary();
#Modify secondary
$dhcp_failover->secondary("2.2.2.2");
```

comment( )

Use this method to set or retrieve a descriptive comment about a DHCP failover object.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

Enter a descriptive comment for the network in string format with a maximum of 256 bytes.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
# Get comment
my $comment = $dhcp_failover->comment();
# Modify comment
$dhcp_failover->comment("add a network");
```

extattrs( )

Use this method to set or retrieve the extensible attributes associated with a DHCP failover object.

Parameter

Valid value is a hash reference containing the names of extensible attributes and their associated values (Infoblox::Grid::Extattr objects).

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get extattrs
my $ref_extattrs = $dhcp_failover->extattrs();
#Modify extattrs
$dhcp_failover->extattrs({ 'Site' => $extattr1, 'Administrator' => $extattr2 });
```

extensible_attributes( )

Use this method to set or retrieve the extensible attributes associated with a DHCP failover object.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

For valid values for extensible attributes, see Infoblox::Grid::ExtensibleAttributeDef/Extensible Attribute Values.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get extensible attributes
my $ref_extensible_attributes = $dhcp_failover->extensible_attributes();
#Modify extensible attributes
$dhcp_failover->extensible_attributes({ 'Site' => 'Santa Clara', 'Administrator' => [ 'Peter', 'Tom'
] });
```

load_balance_split( )

Use this method to set or retrieve a load balancing split value of a DHCP failover object.

Load balancing split determines which server handles IP address requests. This parameter is specified on the primary only.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

Specify the value of the maximum load balancing delay in a 8-bit integer format (range from 0 to 255). The default value is 128.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get load_balance_split
my $load_balance_split = $dhcp_failover->load_balance_split();
#Modify load_balance_split
$dhcp_failover->load_balance_split(200);
```

max_load_balance_delay( )

Use this method to set or retrieve the maximum load balancing delay value of a DHCP failover object.

Maximum load balancing delay specifies the cutoff after which load balancing is disabled. The cutoff is based on the number of seconds since the client sent its first DHCPDISCOVER or DHCPREQUEST message. For instance, if one of the failover peers gets into a state where it is responding to failover messages, but not responding to some client requests, the other failover peer will take over its client load automatically as the clients retry.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

Specify the value of the maximum load balancing delay in a 32-bit integer format (range from 0 to 4294967295) that represents the duration in seconds. The default value is 3.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get max_load_balance_delay
my $max_load_balance_delay = $dhcp_failover->max_load_balance_delay();
#Modify max_load_balance_delay
$dhcp_failover->max_load_balance_delay("500");
```

max_client_lead_time( )

Use this method to set or retrieve the maximum client lead time value of a DHCP failover object.

Maximum client lead time value specifies the length of time that a failover peer can renew a lease without contacting the other peer. The larger the number, the longer it takes for the peer server to recover IP addresses after moving to the Partner Down mode. The smaller the number, the more load your servers experience when they are not communicating. This is specified on the primary server only.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

Specify the value of the maximum client lead time in a 32-bit integer format (range from 0 to 4294967295) that represents the duration in seconds. The default value is 3600.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get max_client_lead_time
my $max_client_lead_time = $dhcp_failover->max_client_lead_time();
```

```
#Modify max_client_lead_time
$dhcp_failover->max_client_lead_time(600);
```

max_unacked_updates( )

Use this method to set or retrieve the maximum number of unacked updates value of a DHCP failover object.

Maximum number of unacked updates value specifies how many update messages the server can send before it should receive an ACK from the failover peer. If no ACK is received after these messages are sent, failover occurs.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

Specify the value of the maximum number of unacked updates in a 32-bit integer format (range from 0 to 4294967295) that represents the number of messages. The default value is 10.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get max_unacked_updates
my $max_unacked_updates = $dhcp_failover->max_unacked_updates();
#Modify max_unacked_updates
$dhcp_failover->max_unacked_updates(700);
```

max_response_delay( )

Use this method to set or retrieve the maximum response delay value of a DHCP failover object.

Maximum response delay value before failover specifies how many seconds can transpire before the primary server assumes its peer (the secondary server) is not sending messages due to failure.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

Specify the value of the maximum response delay in a 32-bit integer format (range from 0 to 4294967295) that represents the duration in seconds. The default value is 60.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get max_response_delay
my $max_response_delay = $dhcp_failover->max_response_delay();
#Modify max_response_delay
$dhcp_failover->max_response_delay(800);
```

primary_state( )

Use this method to retrieve the primary server status of a DHCP failover object.

Parameter

None

Returns

The method returns the attribute value.

Example

```
#Get primary_state
my $primary_state = $dhcp_failover->primary_state();
```

recycle_leases( )

Use this method to set or retrieve the recycle leases flag of a DHCP failover object.

If the flag is enabled, the leases are kept in recycle bin until one week after expiration and when the flag is disabled, the leases are irrecoverably deleted.

Parameter

Specify "true" to set the recycle_leases flag or "false" to deactivate/unset it. The default value for this field is undefined.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get recycle_leases
my $recycle_leases = $dhcp_failover->recycle_leases();
#Modify recycle_leases
$dhcp_failover->recycle_leases("true");
#Un-override recycle_leases
$dhcp_failover->recycle_leases(undef);
```

secondary_state( )

Use this method to retrieve the secondary server status of a DHCP failover object.

Parameter

None

Returns

The method returns the attribute value.

Example

```
#Get secondary_state
my $secondary_state = $dhcp_failover->secondary_state();
```

SAMPLE CODE

The following sample code demonstrates the different functions that can be applied to a DHCP failover object such as add, search, modify, and remove. Also, this sample includes error handling for the operations.

#Preparation prior to a DHCP failover object insertion

```
#PROGRAM STARTS: Include all the modules that will be used
use strict;
use Infoblox;

#Create a session to the Infoblox appliance
my $session = Infoblox::Session->new(
               master   => "192.168.1.2", #appliance host ip
               username => "admin",       #appliance user login
               password => "infoblox"     #appliance password
);
unless ($session) {
       die("Construct session failed: ",
            Infoblox::status_code() . ":" . Infoblox::status_detail());
}
print "Session created successfully\n";

#Create a DHCP Member object
my $memberdhcp = Infoblox::DHCP::Member->new(
    name     => "infoblox.localdomain",
    ipv4addr => "192.168.1.2"
);
unless($memberdhcp) {
       die("Construct member failed: ",
            Infoblox::status_code() . ":" . Infoblox::status_detail());
}
print "DHCP Member object created successfully\n";

#Create the Network object with member
my $network = Infoblox::DHCP::Network->new(
    network => "10.0.0.0/8",
    comment => "add network",
```

```
    members => [ $memberdhcp ]
);
unless($network) {
       die("Construct Network object failed: ",
            Infoblox::status_code() . ":" . Infoblox::status_detail());
}
print "Network object created successfully\n";

#Add the Network object into the Infoblox appliance through a session
$session->add($network)
    or die("Add Network object failed: ",
            $session->status_code() . ":" . $session->status_detail());
print "Network object added to Infoblox appliance successfully\n";
```

#Create a DHCP failover object

```
my $dhcp_failover = Infoblox::DHCP::FailOver->new(
    name                  => "peer1",
    primary               => $memberdhcp,
    secondary             => "1.1.1.1",
    load_balance_split    => 128,
    max_load_balance_delay => 3,
    max_client_lead_time  => 3600,
    max_unacked_updates   => 10,
    max_response_delay    => 60,
    recycle_leases        => "true",
);
unless($dhcp_failover) {
       die("Construct DHCP failover object failed: ",
            Infoblox::status_code() . ":" . Infoblox::status_detail());
}
print "DHCP failover object created successfully\n";

#Add a DHCP failover object into the Infoblox appliance through a session
$session->add($dhcp_failover)
    or die("Add DHCP failover object failed: ",
            $session->status_code() . ":" . $session->status_detail());
print "DHCP failover object added to the network successfully\n";
```

#Get and modify a DHCP failover

```
#Get a DHCP failover through the session
my @retrieved_objs = $session->get(
    object  => "Infoblox::DHCP::FailOver",
    name    => "peer1",
);
my $object = $retrieved_objs[0];
unless ($object) {
       die("Get DHCP failover failed: ",
            $session->status_code() . ":" . $session->status_detail());
}
print "Get DHCP failover object found at least 1 matching entry\n";

#Modify one of the attributes of the specified DHCP failover
$object->max_client_lead_time(3500);

#Apply the changes
$session->modify($object)
    or die("Modify DHCP failover failed: ",
            $session->status_code() . ":" . $session->status_detail());
print "DHCP failover object modified successfully \n";
```

#Search a DHCP failover

```
# search for DHCP failover object with name starting with letter "pe"
my @retrieved_objs = $session->search(
    object => "Infoblox::DHCP::FailOver",
    name   => "pe.*"
);

my $object = $retrieved_objs[0];
unless ($object) {
    die("Search DHCP failover failed: ",
        $session->status_code() . ":" . $session->status_detail());
}
print "Search DHCP failover object found at least 1 matching entry\n";
```

#Add a DHCP range with failover

```
#Construct the DHCP range with failover peer1
my $dhcp_range = Infoblox::DHCP::Range->new(
        end_addr       => "10.0.0.10",
        network        => "10.0.0.0/8",
        start_addr     => "10.0.0.1",
        failover_assoc => "peer1"
);
unless($dhcp_range) {
       die("Construct DHCP Range object with failover failed: ",
            Infoblox::status_code() . ":" . Infoblox::status_detail());
}
print "DHCP Range object with failover peer1 created successfully\n";

#Add the DHCP Range object with failover into the Infoblox appliance through a session
$session->add($dhcp_range)
    or die("Add DHCP Range object with failover failed: ",
            $session->status_code() . ":" . $session->status_detail());
print "DHCP Range object with failover added to the network successfully\n";

#submit the network object for removal
$session->remove($network)
    or die("Remove Network failed: ",
            $session->status_code() . ":" . $session->status_detail());
print "Network removed successfully \n";
```

#Remove a DHCP failover

```
#Get a DHCP failover through the session
my @retrieved_objs = $session->get(
    object => "Infoblox::DHCP::FailOver",
    name   => "peer1"
);

my $object = $retrieved_objs[0];
unless ($object) {
       die("Get DHCP failover failed: ",
            $session->status_code() . ":" . $session->status_detail());
}
print "Get DHCP failover object found at least 1 matching entry\n";

#submit the object for removal
$session->remove($object)
    or die("Remove DHCP failover failed: ",
            $session->status_code() . ":" . $session->status_detail());
print "DHCP failover object removed successfully \n";

####PROGRAM ENDS####
```

AUTHOR

Infoblox Inc. http://www.infoblox.com/

SEE ALSO

Infoblox::Session, Infoblox::Session->get(), Infoblox::Session->search(), Infoblox::Session->add(), Infoblox::Session->remove(), Infoblox::Session->modify(), Infoblox::DHCP::Member

---

COPYRIGHT

Copyright (c) 2017 Infoblox Inc.

Infoblox::DHCP::Filter::Fingerprint - DHCP Fingerprint Filter object.

- NAME
- DESCRIPTION
- CONSTRUCTOR
- SESSION METHODS
  - Infoblox::Session->add( )
  - Infoblox::Session->get( )
  - Infoblox::Session->modify( )
  - Infoblox::Session->remove( )
  - Infoblox::Session->search( )
- METHODS
  - comment( )
  - extattrs( )
  - extensible_attributes( )
  - fingerprint( )
  - name( )
- SAMPLE CODE
- AUTHOR
- SEE ALSO
- COPYRIGHT

NAME

Infoblox::DHCP::Filter::Fingerprint - DHCP Fingerprint Filter object.

DESCRIPTION

DHCP Fingerprint Filter object.

CONSTRUCTOR

```
 my $filter = Infoblox::DHCP::Filter::Fingerprint->new(
    name            => $string,                                  # required
    fingerprint     => [$fingerprint1, $fingerprint2, ...],      # required, reference to an array of
Infoblox::DHCP::Fingerprint objects
    comment         => $string,                                  # optional
    extattrs        => { $string => $extattr, ... },             # optional / Default is undefined
    extensible_attributes => { $string => $string | $num, $string => [ $string | $num, ... ], ... },
#Optional / Default is undefined
 );
```

You cannot set both extattrs and extensible_attributes attributes at the same time.

SESSION METHODS

This section describes all the methods in an Infoblox::Session module that you can apply to DHCP Fingerprint Filter object.

Infoblox::Session->add( )

Use this method to add an object to the Infoblox appliance. See Infoblox::Session->add() for parameters and return values.

Example

```
#Construct an object
my $filter = Infoblox::DHCP::Filter::Fingerprint->new(
   name        => 'fingerprint_filter',
   fingerprint => [$fingerprint],
);

#Submit for addition
my $response = $session->add( $filter );
```

Infoblox::Session->get( )

Use this method to retrieve all the matching objects from the Infoblox appliance. See Infoblox::Session->get() for parameters and return values.

Key References

```
Apply the following attributes to get a specific DHCP Fingerprint Filter object:

 name       - Optional. The name of DHCP Fingerprint Filter object.
 comment    - Optional. The comment.
 extattrs   - Optional. A hash reference containing extensible attributes.
 extensible_attributes - Optional. A hash reference containing extensible attributes.
```

Example

```
# Get DHCP Fingerprint Filter object by name
my @retrieved_objs = $session->get(
   object    => 'Infoblox::DHCP::Filter::Fingerprint',
   name      => 'fingerprint_filter',
);
```

Infoblox::Session->modify( )

Use this method to modify an object in the Infoblox appliance. See Infoblox::Session->modify() for parameters and return values.

Example

```
# Modify comment
$filter->comment("This is modified object");
# Submit modification
my $response = $session->modify( $filter );
```

Infoblox::Session->remove( )

Use this method to remove an object from the Infoblox appliance. See Infoblox::Session->remove() for parameters and return values.

To remove a specific object, first use `get()` or `search()` to retrieve the specific object, and then submit this object for removal.

Example

```
# Get the object by name
my @retrieved_objs = $session->get(
   object   => 'Infoblox::DHCP::Filter::Fingerprint',
   name     => 'fingerprint_filter',
);
#Find the desired object from the retrieved list.
my $desired_filter = $retrieved_objs[0];
# Submit for removal
my $response = $session->remove( $desired_filter );
```

Infoblox::Session->search( )

Use this method to search for DHCP Fingerprint Filter objects in the Infoblox appliance. See Infoblox::Session->search() for parameters and return values.

Key References

```
Apply the following attributes to search for specific DHCP Fingerprint Filter objects:

 name       - Optional. The name of DHCP Fingerprint Filter object (regular expression).
 comment    - Optional. The comment (regular expression).
 extattrs   - Optional. A hash reference containing extensible attributes.
 extensible_attributes - Optional. A hash reference containing extensible attributes.
```

Example

```
# Search DHCP Fingerprint Filter object by name
my @retrieved_objs = $session->search(
   object    => 'Infoblox::DHCP::Filter::Fingerprint',
   name      => 'fingerprint_filter',
);
```

METHODS

This section describes all the methods that you can use to configure and retrieve the attribute values of DHCP Fingerprint Filter objects.

comment( )

Use this method to set or retrieve a descriptive comment.

Parameter

Desired comment in string format with a maximum of 256 bytes.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get attribute value
my $value = $filter->comment();
#Modify attribute value
$filter->comment('new comment');
```

extattrs( )

Use this method to set or retrieve the extensible attributes associated with a DHCP Fingerprint Filter object.

Parameter

Valid value is a hash reference containing the names of extensible attributes and their associated values (Infoblox::Grid::Extattr objects).

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get extattrs
my $ref_extattrs = $filter->extattrs();
#Modify extattrs
$filter->extattrs({ 'Site' => $extattr1, 'Administrator' => $extattr2 });
```

extensible_attributes( )

Use this method to set or retrieve the extensible attributes associated with a DHCP Fingerprint Filter object.

Parameter

For valid values for extensible attributes, see Infoblox::Grid::ExtensibleAttributeDef/Extensible Attribute Values.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get extensible attributes
my $extensible_attributes = $filter->extensible_attributes();
#Modify extensible attributes
$filter->extensible_attributes({ 'Site' => 'Santa Clara', 'Administrator' => [ 'Peter', 'Tom' ] });
```

fingerprint( )

Use this method to set or retrieve a list of DHCP Fingerprint objects.

Parameter

Reference to an array of Infoblox::DHCP::Fingerprint objects.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get attribute value
my $value = $filter->fingerprint();
#Modify attribute value
$filter->fingerprint([$fingerprint]);
```

name( )

Use this method to set or retrieve the name of a DHCP Fingerprint Filter object.

Parameter

The name in string format with a maximum of 256 bytes.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get attribute value
my $value = $filter->name();
#Modify attribute value
$filter->name('new_fingerprint_filter');
```

SAMPLE CODE

The following sample code demonstrates the different functions that can be applied to an object such as add, get, modify, search and remove. This sample also includes error handling for the operations.

#Preparation prior to getting and modifying DHCP Fingerprint Filter object

```
use strict;
use Infoblox;

#refers to Infoblox Appliance IP address
my $host_ip = "192.168.1.2";

#Create a session to the Infoblox appliance

my $session = Infoblox::Session->new(
    master   => $host_ip,
    username => "admin",
    password => "infoblox"
);
unless ($session) {
       die("Construct session failed: ",
               Infoblox::status_code() . ":" . Infoblox::status_detail());
}
print "Session created successfully\n";
```

#Create DHCP Fingerprint Filter object

```
my $fingerprint = Infoblox::DHCP::Fingerprint->new('name' => 'Microsoft Windows 8');
unless ($fingerprint) {
       die("Construct DHCP Fingerprint object failed: ",
               Infoblox::status_code() . ":" . Infoblox::status_detail());
}
print "DHCP Fingerprint object created successfully\n";

my $filter = Infoblox::DHCP::Filter::Fingerprint->new(
   'name'        => 'fingerprint_filter',
   'fingerprint' => [$fingerprint],
);

unless ($filter) {
       die("Construct DHCP Fingerprint Filter object failed: ",
               Infoblox::status_code() . ":" . Infoblox::status_detail());
}
print "DHCP Fingerprint Filter object created successfully\n";

my $response = $session->add($filter);

unless ($response) {
    die("Add DHCP Fingerprint Filter object failed: ",
           $session->status_code() . ":" . $session->status_detail());
}
print"DHCP Fingerprint Filter object added successfully \n";
```

#Search for DHCP Fingerprint Filter object

```
my @retrieved_objs = $session->search(
```

```
    object  => 'Infoblox::DHCP::Filter::Fingerprint',
    name    => 'fingerprint_filter',
);

$filter = $retrieved_objs[0];

unless ($filter) {
        die("Search DHCP Fingerprint Filter object object failed: ",
                $session->status_code() . ":" . $session->status_detail());
}
print "Search DHCP Fingerprint Filter object found at least 1 matching entry\n";
```

#Get and modify DHCP Fingerprint Filter object

```
@retrieved_objs = $session->get(
    object  => 'Infoblox::DHCP::Filter::Fingerprint',
    name    => 'fingerprint_filter',
);

$filter = $retrieved_objs[0];

unless ($filter) {
    die("Get DHCP Fingerprint Filter object failed: ",
            $session->status_code() . ":" . $session->status_detail());
}
print"Get DHCP Fingerprint Filter object successful \n";

$filter->comment("This is modified object");

$session->modify($filter)
    or die("Modify DHCP Fingerprint Filter object failed",
             $session->status_code() . ":" . $session->status_detail());
print "DHCP Fingerprint Filter object modified successfully \n";
```

#Remove DHCP Fingerprint Filter object

```
@retrieved_objs = $session->get(
    object  => 'Infoblox::DHCP::Filter::Fingerprint',
    name    => 'fingerprint_filter',
);

$filter = $retrieved_objs[0];

unless ($filter) {
    die("Get DHCP Fingerprint Filter object failed: ",
            $session->status_code() . ":" . $session->status_detail());
}
print"Get DHCP Fingerprint Filter object successful \n";

$session->remove( $filter )
    or die("Remove DHCP Fingerprint Filter object failed",
            $session->status_code() . ":" . $session->status_detail());
print"DHCP Fingerprint Filter object removed successfully \n";

####PROGRAM ENDS####
```

AUTHOR

Infoblox Inc. http://www.infoblox.com/

SEE ALSO

Infoblox::Session, Infoblox::DHCP::Fingerprint

COPYRIGHT

Copyright (c) 2017 Infoblox Inc.

Infoblox::DHCP::Filter::MAC - DHCP MAC Address Filter object.

- NAME
- DESCRIPTION
- CONSTRUCTOR
- SESSION METHODS
  - Infoblox::Session->add( )
  - Infoblox::Session->get( )
  - Infoblox::Session->modify( )
  - Infoblox::Session->remove( )
  - Infoblox::Session->search( )
- METHODS
  - comment( )
  - default_mac_address_expiration( )
  - disable( )
  - enforce_expiration_times( )
  - extattrs( )
  - extensible_attributes( )
  - lease_time( )
  - name( )
  - option_list( )
  - reserved_for_infoblox( )
- SAMPLE CODE
- AUTHOR
- SEE ALSO
- COPYRIGHT

NAME

Infoblox::DHCP::Filter::MAC - DHCP MAC Address Filter object.

DESCRIPTION

An Infoblox appliance can filter address requests by the MAC address and/or vendor prefix (i.e., the first 6 hexadecimal characters in MAC address) of a requesting host. The filter instructs the infoblox appliance either to grant or deny an address request if the requesting host matches the filter.

CONSTRUCTOR

```
 my $mac_filter = Infoblox::DHCP::Filter::MAC->new(
       name                            => $string,                    #Required
       comment                         => $string,                    #Optional / Default is empty
       default_mac_address_expiration => $num,                        #Optional / Default is 0 for never
expire
       enforce_expiration_times        => "true" | "false",           #Optional / Default is "true"
       extattrs                        => { $string => $extattr, ... },      #Optional / Default is
undefined
       extensible_attributes           => { $string => $string | $num, $string => [ $string | $num, ... ],
... }, #Optional / Default is undefined
       lease_time                      => $time,                      #Optional / Default is empty
       option_list                     => [$Option1, $Option2,...],   #Optional / Default is empty
       reserved_for_infoblox           => $string,                    #Optional / Default is empty
 );
```

You cannot set both extattrs and extensible_attributes attributes at the same time.

SESSION METHODS

This section describes all the methods in Infoblox::Session module that can be applied to a DHCP MAC Address Filter object.

Infoblox::Session->add( )

Use this method to add a DHCP MAC Address Filter object to the Infoblox appliance. See Infoblox::Session->add() for parameters and return values.

Example

```
       #Construct a DHCP MAC Address Filter object
       my $mac_filter = Infoblox::DHCP::Filter::MAC->new (
             name                              => "filter_script",
```

```
        comment                         => "Created through script",
        default_mac_address_expiration => "70",
        enforce_expiration_times        => "true",
        reserved_for_infoblox           => "This is reserved for infoblox",
);
#Submit for addition
my $response = $session->add($mac_filter)
```

Infoblox::Session->get( )

Use this method to retrieve all the matching objects from the Infoblox appliance. See Infoblox::Session->get() for parameters and return values.

Key References

```
Apply the following attributes to get a specific DHCP MAC Address Filter object:

 name - Required. Name of the MAC Address Filter.
 extattrs      - Optional. A hash reference containing extensible attributes.
 extensible_attributes - Optional. A hash reference containing extensible attributes.
```

Example

```
#Get DHCP MAC Address Filter object through the session
my @retrieved_objs = $session->get(
    object => "Infoblox::DHCP::Filter::MAC",
    name   => "filter_script",
);

# get all objects with a given extensible attribute
my @retrieved_objs = $session->get(
    object                => "Infoblox::DHCP::Filter::MAC",
    extensible_attributes => { 'Site' => 'Santa Clara' }
);
```

Infoblox::Session->modify( )

Use this method to modify a DHCP MAC Address Filter object in the Infoblox appliance. See Infoblox::Session->modify() for parameters and return values.

Example

```
#Use method to modify the default_mac_address_expiration
$mac_filter->default_mac_address_expiration("1234");
#Submit modification
my $response = $session->modify( $mac_filter );
```

Infoblox::Session->remove( )

Use this method to remove a DHCP MAC Address Filter object from the Infoblox appliance. See Infoblox::Session->remove() for parameters and return values.

To remove a specific object, first use `get()` or `search()` to retrieve the specific DHCP MAC Address Filter object, then submit this object for removal.

Example

```
#Get DHCP MAC Address Filter object through the session
my @retrieved_objs = $session->get(
    object => "Infoblox::DHCP::Filter::MAC",
    name   => "filter_script",
);
#Find the desired object from the retrieved list.
my $desired_mac_filter = $retrieved_objs[0];
#Submit for removal
my $response = $session->remove( $desired_mac_filter );
```

Infoblox::Session->search( )

Use this method to search for DHCP MAC Address Filter objects in the Infoblox appliance. See Infoblox::Session->search() for parameters and return values.

Key References

```
Apply the following attributes to search for a specific DHCP MAC Address Filter object:

 name - Name of the MAC Address Filter (regular expression).
 comment - Comment string for the MAC Address Filter (regular expression).
```

```
 extattrs     - Optional. A hash reference containing extensible attributes.
 extensible_attributes - Optional. A hash reference containing extensible attributes.

Either name or comment must be specified.
```

For more information about searching extensible attributes, see Infoblox::Grid::ExtensibleAttributeDef/Searching Extensible Attributes.

Example

```
# search for all DHCP MAC Address Filter objects whose name starts with "filter".
my @retrieved_objs = $session->search(
    object  => "Infoblox::DHCP::Filter::MAC",
    name    => "filter.*",
);

# search for all DHCP MAC Address Filter objects whose comment starts with "user1".
my @retrieved_objs = $session->search(
    object  => "Infoblox::DHCP::Filter::MAC",
    comment    => "user1.*",
);

# search all DHCP MAC Address Filter associations with the extensible attribute 'Site'
my @retrieved_objs = $session->search(
   object => "Infoblox::DHCP::Filter::MAC",
   extensible_attributes => { 'Site' => 'Santa Clara' });
```

METHODS

This section describes all the methods that can be used to set and retrieve the attribute values of a DHCP MAC Address Filter object.

comment( )

Use this method to set or retrieve a descriptive comment of a DHCP MAC Address Filter object.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

Desired comment in string format with a maximum of 256 bytes.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get comment
my $comment = $mac_filter->comment();
#Modify comment
$mac_filter->comment("Modified DHCP MAC Address Filter comment");
```

default_mac_address_expiration( )

Use this method to set or retrieve the default mac address expiration time of a DHCP MAC Address Filter object.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

By default, the MAC address filter never expires; otherwise, it is the absolute interval when the MAC address filter expires. The maximum value can extend up to 4294967295 secs. The minimum value is 60 secs (1 min).

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get default_mac_address_expiration
my $default_mac_address_expiration = $mac_filter->default_mac_address_expiration();
#Modify default_mac_address_expiration
$mac_filter->default_mac_address_expiration("1234");
```

disable( )

Use this method to set or retrieve the disable flag of a DHCP MAC Address Filter object.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

Specify "true" to set the disable flag or "false" to deactivate/unset it. The default value for this field is false.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get disable
my $disable = $mac_filter->disable();
#Modify disable
$mac_filter->disable("true");
```

enforce_expiration_times( )

Use this method to set or retrieve the enforce_expiration_times flag of a DHCP MAC Address Filter object.

Determines whether lease requests will be denied from expired MAC Addresses.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

Specify "true" to set the expired flag or "false" to deactivate/unset it. Default value is "true".

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get enforce_expiration_times
my $enforce_expiration_times = $mac_filter->enforce_expiration_times();
#Modify enforce_expiration_times
$mac_filter->enforce_expiration_times("false");
```

extattrs( )

Use this method to set or retrieve the extensible attributes associated with a DHCP MAC Address Filter object.

Parameter

Valid value is a hash reference containing the names of extensible attributes and their associated values (Infoblox::Grid::Extattr objects).

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get extattrs
my $ref_extattrs = $mac_filter->extattrs();
#Modify extattrs
$mac_filter->extattrs({ 'Site' => $extattr1, 'Administrator' => $extattr2 });
```

extensible_attributes( )

Use this method to set or retrieve the extensible attributes associated with a DHCP MAC Address Filter object.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

For valid values for extensible attributes, see Infoblox::Grid::ExtensibleAttributeDef/Extensible Attribute Values.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get extensible attributes
my $ref_extensible_attributes = $mac_filter->extensible_attributes();
#Modify extensible attributes
$mac_filter->extensible_attributes({ 'Site' => 'Santa Clara', 'Administrator' => [ 'Peter', 'Tom' ]
});
```

lease_time( )

Use this method to set or retrieve a lease_time attribute of a DHCP MAC address filter object.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

Enter appropriate values in seconds.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
# Get lease_time
my $lease_time = $mac_filter->lease_time();
# Modify lease_time
$mac_filter->lease_time("7200");
```

name( )

Use this method to set or retrieve the name of a DHCP MAC Address Filter object.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

Name of the DHCP MAC Address filter. Maximum length up to 1024 bytes is supported.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get name
my $name = $mac_filter->name();
#Modify name
$mac_filter->name("filter1");
```

option_list( )

Use this method to set or retrieve the option_list of a DHCP MAC address filter object.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

The valid value is an array reference that contains the Infoblox::DHCP::Option objects. The option list describes filter option configuration settings and various services.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get option_list
my $options = $mac_filter->option_list();
#Modify option_list
$mac_filter->option_list([$option1]);
```

reserved_for_infoblox( )

Set this flag to reserve the MAC address filter for an Infoblox appliance.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

Reserved to write comment related to this particular MAC address filter. The length of comment cannot exceed 1024 bytes long.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get reserved_for_infoblox
my $reserved_for_infoblox = $mac_filter->reserved_for_infoblox();
#Modify reserved_for_infoblox
$mac_filter->reserved_for_infoblox("reserved for Infoblox");
```

SAMPLE CODE

The following sample code demonstrates the different functions that can be applied to an object such as add, search, modify, and remove. Also, this sample includes error handling for the operations.

#Preparation prior to a DHCP MAC Address Filter object insertion

```
#PROGRAM STARTS: Include all the modules that will be used
use strict;
use Infoblox;

#Create a session to the Infoblox appliance

my $session = Infoblox::Session->new(
               master   => "192.168.1.2", #appliance host ip
               username => "admin",       #appliance user login
               password => "infoblox"     #appliance password
);

unless ($session) {
       die("Construct session failed: ",
               Infoblox::status_code() . ":" . Infoblox::status_detail());
}
print "Session created successfully\n";
```

#Create a DHCP MAC Address Filter object

```
#Create a DHCP MAC Address Filter object
my $mac_filter = Infoblox::DHCP::Filter::MAC->new (
       name                          => "script_filter",
       comment                       => "Create through script",
       default_mac_address_expiration => "70",
       enforce_expiration_times      => "true",
       reserved_for_infoblox         => "This is reserved for infoblox",
);

unless($mac_filter) {
     die("Construct mac address filter failed: ",
           Infoblox::status_code() . ":" . Infoblox::status_detail());
}
print "MAC Address Filter object created successfully\n";

#Verify if the DHCP MAC Address Filter exists in the Infoblox appliance
my $object = $session->get(object => "Infoblox::DHCP::Filter::MAC", name => "script_filter");
unless ($object) {
   print "Mac address filter does not exist on server, safe to add the mac address filter\n";
   $session->add($mac_filter)
      or die("Add mac address filter failed: ",
```

```
            $session->status_code() . ":" . $session->status_detail());
}
print "DHCP MAC Address Filter added successfully\n";
```

#Search for a specific DHCP MAC Address Filter object

```
#Search for all DHCP MAC address filter objects
my @retrieved_objs = $session->search(
    object => "Infoblox::DHCP::Filter::MAC",
    name   => ".*" );
my $object = $retrieved_objs[0];

unless ($object) {
       die("Search for MAC Address Filter object failed: ",
              $session->status_code() . ":" . $session->status_detail());
}
print "Search MAC Address Filter object found at least 1 matching entry\n";
```

#Get and modify a MAC Address Filter object

```
#Get MAC Address Filter object through the session
my @retrieved_objs = $session->get(
    object => "Infoblox::DHCP::Filter::MAC",
    name   => "script_filter" );

my $object = $retrieved_objs[0];
unless ($object) {
    die("Get MAC Address Filter object failed: ",
        $session->status_code() . ":" . $session->status_detail());
}
print "Get MAC Address Filter object found at least 1 matching entry\n";

#Modify one of the attributes of the obtained MAC Address Filter object
$object->default_mac_address_expiration("1234");

#Apply the changes
$session->modify($object)
    or die("Modify MAC Address Filter object failed: ",
           $session->status_code() . ":" . $session->status_detail());
print "MAC Address Filter object modified successfully \n";
```

#Remove a MAC Address Filter object

```
#Get MAC Address Filter object through the session
my @retrieved_objs = $session->get(
    object => "Infoblox::DHCP::Filter::MAC",
    name   => "script_filter",
);
my $object = $retrieved_objs[0];
unless ($object) {
    die("Get MAC Address Filter object failed: ",
        $session->status_code() . ":" . $session->status_detail());
}
print "Get MAC Address Filter object found at least 1 matching entry\n";

#Submit the object for removal
$session->remove($object)
    or die("Remove MAC Address Filter object failed: ",
        $session->status_code() . ":" . $session->status_detail());
print "MAC Address Filter object removed successfully \n";

####PROGRAM ENDS####
```

AUTHOR

Infoblox Inc. http://www.infoblox.com/

SEE ALSO

Infoblox::DHCP::MAC, Infoblox::DHCP::Filter::RelayAgent, Infoblox::DHCP::Lease, Infoblox::Session,Infoblox::Session->get(),Infoblox::Session->search(),Infoblox::Session->modify(),Infoblox::Session->remove()

COPYRIGHT

Infoblox::DHCP::Filter::MAC - DHCP MAC Address Filter object.

Copyright (c) 2017 Infoblox Inc.

Infoblox::DHCP::Filter::NAC - DHCP NAC Filter object.

- NAME
- DESCRIPTION
- CONSTRUCTOR
- SESSION METHODS
  - Infoblox::Session->add( )
  - Infoblox::Session->get( )
  - Infoblox::Session->modify( )
  - Infoblox::Session->remove( )
  - Infoblox::Session->search( )
- METHODS
  - comment( )
  - expression( )
  - extattrs( )
  - extensible_attributes( )
  - lease_time( )
  - name( )
  - option_list( )
- SAMPLE CODE
- AUTHOR
- SEE ALSO
- COPYRIGHT

NAME

Infoblox::DHCP::Filter::NAC - DHCP NAC Filter object.

DESCRIPTION

If NAC authentication is configured, the appliance receives authentication responses from NAC authentication servers, and it grants or denies a lease request if the authentication response matches conditions defined by the NAC filters.

CONSTRUCTOR

```
 my $nac_filter = Infoblox::DHCP::Filter::NAC->new(
       name                          => $string,                  #Required
       expression                    => $string,                  #Optional / Default is empty
       extattrs                      => { $string => $extattr, ... },      #Optional / Default is
undefined
       extensible_attributes         => { $string => $string | $num, $string => [ $string | $num, ... ],
... }, #Optional / Default is undefined
       comment                       => $string,                  #Optional / Default is empty
       lease_time                    => $time,                    #Optional / Default is empty
       option_list                   => [$Option1, $Option2,...],  #Optional / Default is empty
 );
```

You cannot set both extattrs and extensible_attributes attributes at the same time.

SESSION METHODS

This section describes all the methods in the Infoblox::Session module that can be applied to a DHCP NAC Filter object.

Infoblox::Session->add( )

Use this method to add a DHCP NAC Filter object to the Infoblox appliance. See Infoblox::Session->add() for parameters and return values.

Example

```
       #Construct a DHCP NAC Filter object
       my $nac_filter = Infoblox::DHCP::Filter::NAC->new (
              name                          => "filter_script",
              comment                       => "Created through script",
              expression                    => 'Sophos.ComplianceState="PartialCompliant"',
       );
       #Submit for addition
       my $response = $session->add($nac_filter)
```

Infoblox::Session->get( )

Use this method to retrieve all the matching objects from the Infoblox appliance. See Infoblox::Session->get() for parameters and return values.

Key References

```
Apply the following attributes to get a specific DHCP NAC Filter object:

 name - Required. Name of the NAC Filter.
 extattrs     - Optional. A hash reference containing extensible attributes.
 extensible_attributes - Optional. A hash reference containing extensible attributes.
```

Example

```
#Get DHCP NAC Filter object through the session
my @retrieved_objs = $session->get(
    object => "Infoblox::DHCP::Filter::NAC",
    name   => "filter_script",
);

# get all objects with a given extensible attribute
my @retrieved_objs = $session->get(
    object               => "Infoblox::DHCP::Filter::NAC",
    extensible_attributes => { 'Site' => 'Santa Clara' }
);
```

Infoblox::Session->modify( )

Use this method to modify a DHCP NAC Filter object in the Infoblox appliance. See Infoblox::Session->modify() for parameters and return values.

Example

```
#Use method to modify the default_nac_address_expiration
$nac_filter->comment("modified comment");
#Submit modification
my $response = $session->modify( $nac_filter );
```

Infoblox::Session->remove( )

Use this method to remove a DHCP NAC Filter object from the Infoblox appliance. See Infoblox::Session->remove() for parameters and return values.

To remove a specific object, first use `get()` or `search()` to retrieve the specific DHCP NAC Filter object, then submit this object for removal.

Example

```
#Get DHCP NAC Filter object through the session
my @retrieved_objs = $session->get(
    object => "Infoblox::DHCP::Filter::NAC",
    name   => "filter_script",
);
#Find the desired object from the retrieved list.
my $desired_nac_filter = $retrieved_objs[0];
#Submit for removal
my $response = $session->remove( $desired_nac_filter );
```

Infoblox::Session->search( )

Use this method to search for DHCP NAC Filter objects in the Infoblox appliance. See Infoblox::Session->search() for parameters and return values.

Key References

```
Apply the following attributes to search for a specific DHCP NAC Filter object:

 name - Name of the NAC filter (regular expression).
 comment - Comment string for the NAC filter (regular expression).
 extattrs     - Optional. A hash reference containing extensible attributes.
 extensible_attributes - Optional. A hash reference that contains extensible attributes.

Either a name or comment must be specified.
```

For more information about searching extensible attributes, see Infoblox::Grid::ExtensibleAttributeDef/Searching Extensible Attributes.

Example

```
# search for all DHCP NAC Filter objects whose name starts with "filter".
my @retrieved_objs = $session->search(
    object  => "Infoblox::DHCP::Filter::NAC",
    name    => "filter.*",
);

# search for all DHCP NAC Filter objects whose comment starts with "user1".
my @retrieved_objs = $session->search(
    object  => "Infoblox::DHCP::Filter::NAC",
    comment    => "user1.*",
);

# search all DHCP NAC Filter associations with the extensible attribute 'Site'
my @retrieved_objs = $session->search(
   object => "Infoblox::DHCP::Filter::NAC",
   extensible_attributes => { 'Site' => 'Santa Clara' });
```

METHODS

This section describes all the methods that can be used to set and retrieve the attribute values of a DHCP NAC Filter object.

comment( )

Use this method to set or retrieve a descriptive comment of a DHCP NAC Filter object.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

Desired comment in string format, with a maximum of 256 bytes.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get comment
my $comment = $nac_filter->comment();
#Modify comment
$nac_filter->comment("Modified DHCP NAC Filter comment");
```

expression( )

Use this method to set or retrieve the conditional expression of a DHCP NAC Filter object.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

An expression in string format, with a maximum of 4096 characters. The expression must comply with the NAC filter expression syntax.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Getting expression
my $expression = $nac_filter->expression( );
#Modifying expression
$nac_filter->expression('Sophos.ComplianceState="PartialCompliant"');
```

extattrs( )

Use this method to set or retrieve the extensible attributes associated with a DHCP NAC Filter object.

Parameter

Valid value is a hash reference containing the names of extensible attributes and their associated values (Infoblox::Grid::Extattr objects).

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get extattrs
my $ref_extattrs = $nac_filter->extattrs();
#Modify extattrs
$nac_filter->extattrs({ 'Site' => $extattr1, 'Administrator' => $extattr2 });
```

extensible_attributes( )

Use this method to set or retrieve the extensible attributes associated with a DHCP NAC Filter object.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

For valid values for extensible attributes, see Infoblox::Grid::ExtensibleAttributeDef/Extensible Attribute Values.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get extensible attributes
my $ref_extensible_attributes = $nac_filter->extensible_attributes();
#Modify extensible attributes
$nac_filter->extensible_attributes({ 'Site' => 'Santa Clara', 'Administrator' => [ 'Peter', 'Tom' ]
});
```

lease_time( )

Use this method to set or retrieve a lease_time attribute of a DHCP NAC filter object.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

Enter appropriate values in seconds.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
# Get lease_time
my $lease_time = $nac_filter->lease_time();
# Modify lease_time
$nac_filter->lease_time("7200");
```

name( )

Use this method to set or retrieve the name of a DHCP NAC Filter object.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

The name of the DHCP NAC filter. The maximum length supported is 1024 bytes.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get name
```

```
my $name = $nac_filter->name();
#Modify name
$nac_filter->name("filter1");
```

option_list( )

Use this method to set or retrieve the option_list of a DHCP NAC filter object.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

The valid value is an array reference that contains the Infoblox::DHCP::Option objects. The option list describes filter option configuration settings and various services.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get option_list
my $options = $nac_filter->option_list();
#Modify option_list
$nac_filter->option_list([$option1]);
```

SAMPLE CODE

The following sample code demonstrates the different functions that can be applied to an object such as add, search, modify, and remove. Also, this sample includes error handling for the operations.

#Preparation prior to a DHCP NAC Filter object insertion

```
#PROGRAM STARTS: Include all the modules that will be used
use strict;
use Infoblox;

#Create a session to the Infoblox appliance

my $session = Infoblox::Session->new(
               master   => "192.168.1.2", #appliance host ip
               username => "admin",       #appliance user login
               password => "infoblox"     #appliance password
);

unless ($session) {
       die("Construct session failed: ",
               Infoblox::status_code() . ":" . Infoblox::status_detail());
}
print "Session created successfully\n";
```

#Create a DHCP NAC Filter object

```
#Create a DHCP NAC Filter object
my $nac_filter = Infoblox::DHCP::Filter::NAC->new (
       name                          => "script_filter",
       comment                       => "Create through script",
       expression                    => 'Sophos.ComplianceState="PartialCompliant"',
);

unless($nac_filter) {
     die("Construct nac filter failed: ",
           Infoblox::status_code() . ":" . Infoblox::status_detail());
}
print "NAC Filter object created successfully\n";

#Verify if the DHCP NAC Filter exists in the Infoblox appliance
my $object = $session->get(object => "Infoblox::DHCP::Filter::NAC", name => "script_filter");
unless ($object) {
   print "nac filter does not exist on server, safe to add the nac filter\n";
   $session->add($nac_filter)
      or die("Add nac filter failed: ",
             $session->status_code() . ":" . $session->status_detail());
}
print "DHCP NAC Filter added successfully\n";
```

#Search for a specific DHCP NAC Filter object

```
#Search for all DHCP NAC filter objects
my @retrieved_objs = $session->search(
    object => "Infoblox::DHCP::Filter::NAC",
    name   => ".*" );
my $object = $retrieved_objs[0];

unless ($object) {
       die("Search for NAC Filter object failed: ",
               $session->status_code() . ":" . $session->status_detail());
}
print "Search NAC Filter object found at least 1 matching entry\n";
```

#Get and modify a NAC Filter object

```
#Get a NAC Filter object through the session
my @retrieved_objs = $session->get(
    object => "Infoblox::DHCP::Filter::NAC",
    name   => "script_filter" );

my $object = $retrieved_objs[0];
unless ($object) {
    die("Get NAC Filter object failed: ",
        $session->status_code() . ":" . $session->status_detail());
}
print "Get NAC Filter object found at least 1 matching entry\n";

#Modify one of the attributes of the obtained NAC Filter object
$object->comment("Modified comment");

#Apply the changes
$session->modify($object)
    or die("Modify NAC Filter object failed: ",
           $session->status_code() . ":" . $session->status_detail());
print "NAC Filter object modified successfully \n";
```

#Remove a NAC Filter object

```
#Get a NAC Filter object through the session
my @retrieved_objs = $session->get(
    object => "Infoblox::DHCP::Filter::NAC",
    name   => "script_filter",
);
my $object = $retrieved_objs[0];
unless ($object) {
    die("Get NAC Filter object failed: ",
        $session->status_code() . ":" . $session->status_detail());
}
print "Get NAC Filter object found at least 1 matching entry\n";

#Submit the object for removal
$session->remove($object)
    or die("Remove NAC Filter object failed: ",
        $session->status_code() . ":" . $session->status_detail());
print "NAC Filter object removed successfully \n";

####PROGRAM ENDS####
```

AUTHOR

Infoblox Inc. http://www.infoblox.com/

SEE ALSO

Infoblox::DHCP::Filter::RelayAgent, Infoblox::DHCP::Lease, Infoblox::Session,Infoblox::Session->get(),Infoblox::Session->search(),Infoblox::Session->modify(),Infoblox::Session->remove()

COPYRIGHT

Copyright (c) 2017 Infoblox Inc.

Infoblox::DHCP::Filter::Option - DHCP filter option object

- NAME
- DESCRIPTION
- CONSTRUCTOR
- SESSION METHODS
  - Infoblox::Session->add( )
  - Infoblox::Session->get( )
  - Infoblox::Session->modify( )
  - Infoblox::Session->remove( )
  - Infoblox::Session->search( )
- METHODS
  - name( )
  - apply_as_class( )
  - boot_file( )
  - boot_server( )
  - comment( )
  - expression( )
  - extattrs( )
  - extensible_attributes( )
  - lease_time( )
  - next_server( )
  - option_list( )
  - option_space( )
  - pxe_lease_time( )
- SAMPLE CODE
- AUTHOR
- SEE ALSO
- COPYRIGHT

NAME

Infoblox::DHCP::Filter::Option - DHCP filter option object

DESCRIPTION

A Filter::Option defines a DHCP option filter. In the ISC DHCP terms, it defines a class of clients that match a particular (option, value) pair. To define an option filter, add Option to the DHCP Filter object.

CONSTRUCTOR

```
  my $optionFilter = Infoblox::DHCP::Filter::Option->new(
                 name                   => $string,                          #Required
                 apply_as_class         => "true" | "false",                 #Optional / Default is "true"
                 comment                => $string,                          #Optional / Default is empty
                 boot_file              => $filename | undef,                #Optional / Default is
undefined
                 boot_server            => $fqdn | undef,                    #Optional / Default is
undefined
                 extattrs               => { $string => $extattr, ... },     #Optional / Default is
undefined
                 extensible_attributes  => { $string => $string | $num, $string => [ $string | $num, ... ],
... }, #Optional / Default is undefined
                 expression             => $string,                          #Optional / Default is empty
                 lease_time             => $time,                            #Optional / Default is empty
                 next_server            => $fqdn | undef,                    #Optional / Default is
undefined
                 option_list            => [$Option1, $Option2,...],         #Optional / Default is empty
                 option_space           => $string,                          #Optional / Default is "DHCP"
                 pxe_lease_time         => $time,                            #Optional / Default is empty
                );
```

You cannot set both extattrs and extensible_attributes attributes at the same time.

SESSION METHODS

This section describes all the methods in Infoblox::Session module that can be applied to a DHCP filter option object.

Infoblox::Session->add( )

Use this method to add a DHCP filter option object to the Infoblox device. See Infoblox::Session->add() for parameters and return values.

Example

```
#Construct an object
my  $optionFilter = Infoblox::DHCP::Filter::Option->new(
       name              => "option_filter",

        );

# Submit for adding a option Filter
my $response = $session->add( $optionFilter );
```

Infoblox::Session->get( )

Use this method to retrieve all the matching DHCP option filter objects from the Infoblox device. See Infoblox::Session->get() for parameters and return values.

Key References

```
Apply the following attributes to get a specific DHCP relayagent filter object:

name - Required. An option filter name in string format.
extattrs              - Optional. A hash reference containing extensible attributes.
extensible_attributes - Optional. A hash reference containing extensible attributes.
```

Example

```
my  @retrieved_objs = $session->get(
    object => "Infoblox::DHCP::Filter::Option" ,
    name   => "option_filter",
);

# get all objects with a given extensible attribute
my @retrieved_objs = $session->get(
    object               => "Infoblox::DHCP::Filter::Option",
    extensible_attributes => { 'Site' => 'Santa Clara' }
);
```

Infoblox::Session->modify( )

Use this method to modify a DHCP option filter object in the Infoblox device. See Infoblox::Session->modify() for parameters and return values.

Example

```
#Use method to modify boot_server method.
$object->boot_server("myboot_server.com");
# Submit modification
my $response = $session->modify( $object );
```

Infoblox::Session->remove( )

Use this method to remove a DHCP option filter object from the Infoblox device. See Infoblox::Session->remove() for parameters and return values.

To remove a specifc object, first use `get()` or `search()` to retrieve the specific object, and then submit this object for removal.

Example

```
# Get the object under the same name
my @retrieved_objs = $session->get(
           object  => "Infoblox::DHCP::Filter::Option" ,
           "name"  => "option_filter",
       );

# find the desired object from retrieved list.
my $desired_obj = $retrieved_objs[0];

# Submit for removal
my $response = $session->remove( $desired_obj);
```

Infoblox::Session->search( )

Use this method to search for DHCP option filter object in the Infoblox device. See Infoblox::Session->search() for parameters and return values.

Key References

```
Apply the following attributes to search for a DHCP Network object:

 name - Required. An option filter name in string format (regular expression).
 comment - Optional . A comment in string format (regular expression).
 extattrs              - Optional. A hash reference containing extensible attributes.
 extensible_attributes - Optional. A hash reference containing extensible attributes.
```

For more information about searching extensible attributes, see Infoblox::Grid::ExtensibleAttributeDef/Searching Extensible Attributes.

Example

```
# search for all DHCP option filter objects
     my @retrieved_objs = $session->search(
    object => "Infoblox::DHCP::Filter::Option",
    name   => "option.*",
);

# search all DHCP option filter associations with the extensible attribute 'Site'
my @retrieved_objs = $session->search(
   object => "Infoblox::DHCP::Filter::Option",
   extensible_attributes => { 'Site' => 'Santa Clara' });
```

METHODS

This section describes all the methods that can be used to configure and retrieve the attribute values of DHCP option filter object

name( )

Use this method to set or retrieve the name of a DHCP option filter object.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

Specify the name in string format. The default value for this field is empty.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
# Get name
my $name = $optionFilter->name();
#Modify name
$optionFilter->name("option_filter");
```

apply_as_class( )

Use this method to set or retrieve the "apply_as_class" flag of a DHCP filter option object. If this flag is set to "true" the filter is treated as global DHCP class, e.g it is written to dhcpd config file even if it is not present in any DHCP range.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

Specify "true" to set the "apply_as_class" flag or "false" to deactivate/unset it. The default value is "true".

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get apply_as_class flag
my $apply_as_class = $optionFilter->apply_as_class();
#Modify apply_as_class flag
$optionFilter->apply_as_class("true");
```

boot_file( )

Use this method to set or retrieve a boot_file attribute of a DHCP filter option object. If this attribute is not set, it will inherit the member level setting.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

The name of the file that the client must download.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
# Get boot_file
my $boot_file = $optionFilter->boot_file();
# Modify boot_file
$optionFilter->boot_file("bootfile1");
#Un-override boot_file
$optionFilter->boot_file(undef);
```

boot_server( )

Use this method to set or retrieve a bootserver attribute of a DHCP filter option object. You can specify the name and/or IP address of the boot server that host needs to boot. If this attribute is not set, it will inherit the member level setting.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

The boot server IP address or name in FQDN (Fully Qualified Domain Name) format. The FQDN consists of the host name followed by the domain name (example: abc.com). A boot server name can have a maximum of 256 bytes. The default value is undefined.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
# Get boot_server
my $bootserver = $optionFilter->bootserver();
# Modify boot_server
$optionFilter->boot_server("abc.domain.com");
#Un-override boot_server
$optionFilter->boot_server(undef);
```

comment( )

Use this method to set or retrieve the descriptive comment of a DHCP filter option object.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

Desired comment in string format with a maximum of 256 bytes.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get comment
my $comment = $optionFilter->comment();
#Modify comment
$optionFilter->comment("Modified DHCP filter option object comment");
```

expression( )

Use this method to set or retrieve the conditional expression of a DHCP filter option object.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

An expression in string format, with a maximum of 32768 characters. The expression must comply with the option filter expression syntax.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Getting expression
my $expression = $optionFilter->expression( );
#Modifying expression
$optionFilter->expression('Sophos.ComplianceState="PartialCompliant"');
```

extattrs( )

Use this method to set or retrieve the extensible attributes associated with a DHCP Filter Option object.

Parameter

Valid value is a hash reference containing the names of extensible attributes and their associated values (Infoblox::Grid::Extattr objects).

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get extattrs
my $ref_extattrs = $optionFilter->extattrs();
#Modify extattrs
$optionFilter->extattrs({ 'Site' => $extattr1, 'Administrator' => $extattr2 });
```

extensible_attributes( )

Use this method to set or retrieve the extensible attributes associated with a DHCP Filter Option object.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

For valid values for extensible attributes, see Infoblox::Grid::ExtensibleAttributeDef/Extensible Attribute Values.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get extensible attributes
my $ref_extensible_attributes = $optionFilter->extensible_attributes();
#Modify extensible attributes
$optionFilter->extensible_attributes({ 'Site' => 'Santa Clara', 'Administrator' => [ 'Peter', 'Tom'
] });
```

lease_time( )

Use this method to set or retrieve a lease_time attribute of a DHCP filter option object.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

Enter appropriate values in seconds.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
# Get lease_time
my $lease_time = $optionFilter->lease_time();
# Modify lease_time
$optionFilter->lease_time("7200");
```

next_server( )

Use this method to set or retrieve a next_server attribute of a DHCP filter option object. You can specify the name and/or IP address of the next server that the host needs to boot. If this attribute is not set, it will inherit the member level setting.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

The next server IP address or name in FQDN (Fully Qualified Domain Name) format. The FQDN consists of the host name followed by the domain name (example: abc.com). The next server name can have a maximum of 256 bytes. The default value is undefined.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
# Get next_server
my $nextserver = $optionFilter->next_server();
# Modify next_server
$optionFilter->next_server("blue.domain.com");
#Un-override next_server
$optionFilter->next_server(undef);
```

option_list( )

Use this method to set or retrieve the option_list of a DHCP filter option object.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

The valid value is an array reference that contains Infoblox::DHCP::Option objects. Option list describe filter option configuration settings and various services.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get option_list
my $options = $optionFilter->option_list();
#Modify option_list
$optionFilter->option_list([$option1]);
```

option_space( )

Use this method to retrieve the option_space of a DHCP filter option object.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

Specify the option space value in string format. The default value of option_space is DHCP.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get option_space
my $options = $optionFilter->option_space();
#Modify option_space
```

```
$optionFilter->option_space("Infoblox_DHCP");
```

pxe_lease_time( )

Use this method to set or retrieve a pxe_lease_time attribute of a DHCP filter option object.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

Enter appropriate values in seconds.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
# Get pxe_lease_time
my $pxe_lease_time = $optionFilter->pxe_lease_time();
# Modify pxe_lease_time
$optionFilter->pxe_lease_time("3600");
```

SAMPLE CODE

The following sample code demonstrates the different functions that can be applied to an object such as add, search, modify, and remove. Also, this sample includes error handling for the operations.

#Preparation prior to a DHCP option filter object insertion

```
#PROGRAM STARTS: Include all the modules that will be used
use strict;
use Infoblox;

#Create a session to the Infoblox device

my $session = Infoblox::Session->new(
    master   => "192.168.1.2",
    username => "admin",
    password => "infoblox"
);
unless ($session) {
   die("Construct session failed: ",
       Infoblox::status_code() . ":" . Infoblox::status_detail());
}
print "Session created successfully\n";
```

#Create a DHCP Filter Option object

```
my  $optionfilter = Infoblox::DHCP::Filter::Option->new(

       name             => "option_filter",
       next_server      => "abc_server.com" ,
       boot_file        => "file1" ,
       boot_server      => "boot_server.com" ,
       lease_time       => "7200",
       pxe_lease_time   => "3600",
              );

unless ($optionfilter) {
   die("Construct DHCP Filter::Option failed: ",
       Infoblox::status_code() . ":" . Infoblox::status_detail());
}
print "DHCP Filter::Option object created successfully\n";
my $response = $session->add($optionfilter)
       or die("Add DHCP Filter Option failed: ",
             $session->status_code() . ":" . $session->status_detail());

print "DHCP Filter Option added successfully\n";
```

#Search for a specific DHCP Filter Option

```
#Search the  DHCP Filter Option
my @retrieved_objs = $session->search(
    object             => "Infoblox::DHCP::Filter::Option",
```

```
    name               => "option.*",
);
my $object = $retrieved_objs[0];
unless ($object) {
    die("Search DHCP Filter Option  failed: ",
        $session->status_code() . ":" . $session->status_detail());
}

print "Search DHCP Filter Option found at least 1 matching entry\n";
```

#Get and modify a DHCP Filter Option attribute

```
#Get  DHCP Filter Option through the session
my  @retrieved_objs = $session->get(

              object      =>"Infoblox::DHCP::Filter::Option" ,
              name        => "option_filter",
                      );

my $object = $retrieved_objs[0];
unless ($object) {
    die("Get option filter object failed: ",
        $session->status_code() . ":" . $session->status_detail());
}

print "Get DHCP Filter Option found at least 1 matching entry\n";

#Modify one of the attributes of the specified DHCP Filter Option attribute

$object->boot_server("another_Server");

#Apply the changes
$session->modify($object)
    or die("Modify DHCP Filter Option attribute failed: ",
           $session->status_code() . ":" . $session->status_detail());

print "DHCP Filter Option object modified successfully \n";
```

#Remove a DHCP Filter Option Object

```
 #Get object through the session
 my @retrieved_objs = $session->get(

           object  => "Infoblox::DHCP::Filter::Option" ,
           "name"  => "option_filter",
                   );

   my $object = $retrieved_objs[0];

   unless ($object) {
    die("Get a DHCP Filter Optionobject failed: ",
        $session->status_code() . ":" . $session->status_detail());
    }

print "Get a DHCP Filter Option object found at least 1 matching entry\n";

#Submit the object for removal
$session->remove($object)
    or die("Remove a DHCP Filter Option failed: ",
        $session->status_code() . ":" . $session->status_detail());

print "DHCP Filter Option removed successfully \n";

####PROGRAM ENDS####
```

AUTHOR

Infoblox Inc. http://www.infoblox.com/

SEE ALSO

Infoblox::Session, Infoblox::Session->get(), Infoblox::Session->search(), Infoblox::Session->add(), Infoblox::Session->remove(), Infoblox::Session->modify(), Infoblox::Grid::Extattr

---

COPYRIGHT

Copyright (c) 2017 Infoblox Inc.

Infoblox::DHCP::Filter::RelayAgent - DHCP relay agent filter object

- NAME
- DESCRIPTION
- CONSTRUCTOR
- SESSION METHODS
  - Infoblox::Session->add( )
  - Infoblox::Session->get( )
  - Infoblox::Session->modify( )
  - Infoblox::Session->remove( )
  - Infoblox::Session->search( )
- METHODS
  - name( )
  - comment( )
  - circuit_id_name( )
  - circuit_id_substring_length( )
  - circuit_id_substring_offset( )
  - extattrs( )
  - extensible_attributes( )
  - remote_id_name( )
  - remote_id_substring_length( )
  - remote_id_substring_offset( )
  - is_circuit_id( )
  - is_circuit_id_substring( )
  - is_remote_id( )
  - is_remote_id_substring( )
- SAMPLE CODE
- AUTHOR
- SEE ALSO
- COPYRIGHT

NAME

Infoblox::DHCP::Filter::RelayAgent - DHCP relay agent filter object

---

DESCRIPTION

The Infoblox appliance can screen address requests through relay agent filters (DHCP option 82) that assist the agents in forwarding address assignments across the proper circuit. When a relay agent receives the DHCPDISCOVER message, it can add one or two agent IDs in the DHCP option 82 suboption fields to the message. If the agent ID strings match those defined in a relay agent filter applied to a DHCP address range, the Infoblox appliance either assigns addresses from that range or denies the request (based on previously configured parameters; that is, the Grant lease and Deny lease parameters).

CONSTRUCTOR

```
  my $relayagentFilter = Infoblox::DHCP::Filter::RelayAgent->new(
                  name                        => $string | $ipv4addr,
# Required
                  comment                     => $string,
# Optional / Default is empty
                  circuit_id_name             => $string,
# Optional / Default is empty
                  extattrs                    => { $string => $extattr, ... },
# Optional / Default is undefined
                  extensible_attributes       => { $string => $string | $num, $string => [ $string | $num,
... ], ... }, # Optional / Default is undefined
                  remote_id_name              => $string,
# Optional / Default is empty
                  is_circuit_id               => $string,
# Optional / Default is empty
                  is_remote_id                => $string,
# Optional / Default is empty
                  is_circuit_id_substring     => 'true' | 'false',
# Optional / Default is 'false'
                  circuit_id_substring_offset => $uint,
# Optional / Default is undefined
                  circuit_id_substring_length => $uint,
# Optional / Default is undefined
                  is_remote_id_substring      => 'true' | 'false',
# Optional / Default is 'false'
                  remote_id_substring_offset  => $uint,
# Optional / Default is undefined
                  remote_id_substring_length  => $uint,
```

```
# Optional / Default is undefined
                );
```

You cannot set both extattrs and extensible_attributes attributes at the same time.

SESSION METHODS

This section describes all the methods in Infoblox::Session module that can be applied to a DHCP relay agent filter object.

Infoblox::Session->add( )

Use this method to add a DHCP relay agent filter object to the Infoblox appliance. See Infoblox::Session->add() for parameters and return values.

Example

```
#Construct an object
my  $relayagentFilter = Infoblox::DHCP::Filter::RelayAgent->new(
       name             => "relayagent_filter",
       circuit_id_name  => "circuit",
       remote_id_name   => "remote"
        );

# Submit for adding a relay agent filter
my $response = $session->add( $relayagentFilter );
```

Infoblox::Session->get( )

Use this method to retrieve all the matching DHCP relay agent filter objects from the Infoblox appliance. See Infoblox::Session->get() for parameters and return values.

Key References

```
Apply the following attributes to get a specific DHCP relay agent filter object:

 name - Required. A relay agent filter name in string format or in ipv4address format.
 extattrs     - Optional. A hash reference containing extensible attributes.
 extensible_attributes - Optional. A hash reference containing extensible attributes.
```

Example

```
my  @retrieved_objs = $session->get(
    object => "Infoblox::DHCP::Filter::RelayAgent" ,
    name   => "relayagent_filter",
);

# get all objects with a given extensible attribute
my @retrieved_objs = $session->get(
    object               => "Infoblox::DHCP::Filter::RelayAgent",
    extensible_attributes => { 'Site' => 'Santa Clara' }
);
```

Infoblox::Session->modify( )

Use this method to modify a DHCP relay agent filter object in the Infoblox appliance. See Infoblox::Session->modify() for parameters and return values.

Example

```
#Use method to modify circuit id name method.
$object->circuit_id_name("diff_Circuit");
# Submit modification
my $response = $session->modify( $object );
```

Infoblox::Session->remove( )

Use this method to remove a DHCP relay agent filter object from the Infoblox appliance. See Infoblox::Session->remove() for parameters and return values.

To remove a specifc object, first use `get()` or `search()` to retrieve the specific object, then submit this object for removal.

Example

```
# Get the object under the same name
my @retrieved_objs = $session->get(
           object  => "Infoblox::DHCP::Filter::RelayAgent" ,
```

```
            "name"   => "relayagent_filter",
        );

    # find the desired object from retrieved list.
    my $desired_obj = $retrieved_objs[0];

    # Submit for removal
    my $response = $session->remove( $desired_obj);
```

Infoblox::Session->search( )

Use this method to search for a DHCP relay agent filter object in the Infoblox appliance. See Infoblox::Session->search() for parameters and return values.

Key References

```
Apply the following attributes to search for a DHCP Network object:

 name - Required. A relay agent filter name in string format (regular expression).
 extattrs     - Optional. A hash reference containing extensible attributes.
 extensible_attributes - Optional. A hash reference containing extensible attributes.
 comment - Optional . A comment in string format (regular expression).
```

For more information about searching extensible attributes, see Infoblox::Grid::ExtensibleAttributeDef/Searching Extensible Attributes.

Example

```
# search for all DHCP relayagent objects
     my @retrieved_objs = $session->search(
               object   => "Infoblox::DHCP::Filter::RelayAgent",
               name     => 'relayagent\..*',
               );

# search all DHCP relay agent associations with the extensible attribute 'Site'
my @retrieved_objs = $session->search(
   object => "Infoblox::DHCP::Filter::RelayAgent",
   extensible_attributes => { 'Site' => 'Santa Clara' });
```

METHODS

This section describes all the methods that can be used to configure and retrieve the attribute values of a DHCP relay agent filter object.

name( )

Use this method to set or retrieve the name of a DHCP relay agent filter object. The name can be an IP address or name of the router acting as relay agent.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

Specify the name in string format or in ipv4addr format. An IPv4 address is a 32-bit number in dotted decimal notation. It consists of four 8-bit groups of decimal digits separated by decimal points (example: 192.168.1.2). The default value for this field is empty.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
# Get name
my $name = $relayagentFilter->name()
# Modify name
$relayagentFilter->name("relay_filter");
```

comment( )

Use this method to set or retrieve the descriptive comment of a DHCP relay agent filter object.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

Desired comment in string format with a maximum of 256 bytes.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get comment
my $comment = $relayagentFilter->comment();
#Modify comment
$relayagentFilter->comment("Modified DHCP relay agent filter object comment");
```

circuit_id_name( )

Use this method to set or retrieve a circuit_id_name of a DHCP relay agent filter object. This filter identifies the circuit between the remote host and the relay agent. For example, the identifier can be the ingress interface number of the circuit access unit, perhaps concatenated with the unit ID number and slot number. Also, the circuit ID can be an ATM virtual circuit ID or cable data virtual circuit ID.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

Specify circuit id name in string format. example: Text String: 001122334455 Binary String: \x00\x11\x22\x33\x44\x55. The default value for this field is empty.

To reset this value, set it to an empty string together with the is_circuit_id flag (anything except "Matches_Value").

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
# Get circuit_id_name
my $circuit_id_name = $relayagentFilter->circuit_id_name();
# Modify circuit_id_name
$relayagentFilter->circuit_id_name("001122334455");
# Set is_circuit_id to "Not_Set"
$relayagentFilter->is_circuit_id("Not_Set");
# Set circuit_id_name to empty string
$relayagentFilter->circuit_id_name("");
```

circuit_id_substring_length( )

Use this method to set or retrive the circuit ID substring length.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

The valid value is an unsigned integer.

Returns

If you specified a parameter, the method returns 'true' when the modification succeeds, and returns 'false' when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#get circuit_id_substring_length value
my $circuit_id_substring_length = $relayagentfilter->circuit_id_substring_length();

#modify circuit_id_substring_length value
$relayagentfilter->circuit_id_substring_length(10);
```

circuit_id_substring_offset( )

Use this method to set or retrive the circuit ID substring offset.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

The valid value is an unsigned integer.

Returns

If you specified a parameter, the method returns 'true' when the modification succeeds, and returns 'false' when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#get circuit_id_substring_offset value
my $circuit_id_substring_offset = $relayagentfilter->circuit_id_substring_offset();

#modify circuit_id_substring_offset value
$relayagentfilter->circuit_id_substring_offset(10);
```

extattrs( )

Use this method to set or retrieve the extensible attributes associated with a DHCP relay agent filter object.

Parameter

Valid value is a hash reference containing the names of extensible attributes and their associated values (Infoblox::Grid::Extattr objects).

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get extattrs
my $ref_extattrs = $relayagentFilter->extattrs();
#Modify extattrs
$relayagentFilter->extattrs({ 'Site' => $extattr1, 'Administrator' => $extattr2 });
```

extensible_attributes( )

Use this method to set or retrieve the extensible attributes associated with a DHCP relay agent filter object.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

For valid values for extensible attributes, see Infoblox::Grid::ExtensibleAttributeDef/Extensible Attribute Values.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get extensible attributes
my $ref_extensible_attributes = $relayagentFilter->extensible_attributes();
#Modify extensible attributes
$relayagentFilter->extensible_attributes({ 'Site' => 'Santa Clara', 'Administrator' => [ 'Peter',
'Tom' ] });
```

remote_id_name( )

Use this method to set or retrieve a remote ID name attribute of a relay agent filter object. This filter identifies the remote host. The remote ID name can represent many different things such as the caller ID telephone number for a dial-up connection, a user name for logging in to the ISP, a modem ID, etc. When the remote ID name is defined on the relay agent, the DHCP server will have a trusted relationship to identify the remote host. The remote ID name is considered as a trusted identifier.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

Specify remote_id_name in string format. example: Text String: 001122334455 Binary String: \x00\x11\x22\x33\x44\x55. The default value for this field is empty.

To reset this value, set it to an empty string together with the is_remote_id flag (anything except "Matches_Value").

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
# Get remote_id_name
my $remote_id_name = $relayagentFilter->remote_id_name();
# Modify remote_id_name
$relayagentFilter->remote_id_name("\x00\x11\x22\x33\x44\x55");
# Set is_remote_id to "Not_Set"
$relayagentFilter->is_remote_id("Not_Set");
# Set remote_id_name to empty string
$relayagentFilter->remote_id_name("");
```

remote_id_substring_length( )

Use this method to set or retrive the remote ID substring length.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

The valid value is an unsigned integer.

Returns

If you specified a parameter, the method returns 'true' when the modification succeeds, and returns 'false' when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#get remote_id_substring_length value
my $remote_id_substring_length = $relayagentfilter->remote_id_substring_length();

#modify remote_id_substring_length value
$relayagentfilter->remote_id_substring_length(10);
```

remote_id_substring_offset( )

Use this method to set or retrive the remote ID substring offset.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

The valid value is an unsigned integer.

Returns

If you specified a parameter, the method returns 'true' when the modification succeeds, and returns 'false' when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#get remote_id_substring_offset value
my $remote_id_substring_offset = $relayagentfilter->remote_id_substring_offset();

#modify remote_id_substring_offset value
$relayagentfilter->remote_id_substring_offset(10);
```

is_circuit_id( )

Use this method to set or retrieve the is_circuit_id flag of a DHCP relay agent filter object. The possible values for the is_circuit_id flag are Any, Not_Set, Matches_Value. The circuit_id value takes effect only if the value is Matches_Value.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

Specify is_circuit_id in string format. example: Text String: Any

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
# Get is_circuit_id
my $is_circuit_id = $relayagentFilter->is_circuit_id();
# Modify is_circuit_id
$relayagentFilter->is_circuit_id("Any");
```

is_circuit_id_substring( )

Use this method to set or retrive the flag that indicates whether the substring of circuit ID, instead of the full circuit ID, is matched.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

Specify 'true' to enable the circuit ID substring match or 'false' to disable it. The default value is 'false'.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
# Get is_circuit_id_substring
my $is_circuit_id_substring = $relayagentFilter->is_circuit_id_substring();
# Modify is_circuit_id_substring
$relayagentFilter->is_circuit_id_substring("true");
```

is_remote_id( )

Use this method to set or retrieve the is_remote_id flag of a DHCP relay agent filter object. The possible values for the is_remote_id flag are Any, Not_Set, Matches_Value. The remote_id value takes effect only if the value is Matches_Value.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

Specify is_remote_id in string format. example: Text String: Not_Set

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
# Get is_remote_id
my $is_remote_id = $relayagentFilter->is_remote_id();
# Modify is_remote_id
$relayagentFilter->is_remote_id("Not_Set");
```

is_remote_id_substring( )

Use this method to set or retrive the flag that indicates whether the substring of the remote ID, instead of the full remote ID, is matched.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

Specify 'true' to enable remote id substring match or 'false' to disable it. The default value is 'false'.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
# Get is_remote_id_substring
my $is_remote_id_substring = $relayagentFilter->is_remote_id_substring();
# Modify is_remote_id_substring
```

```
$relayagentFilter->is_remote_id_substring("true");
```

SAMPLE CODE

The following sample code demonstrates the different functions that can be applied to an object such as add, search, modify, and remove. Also, this sample includes error handling for the operations.

#Preparation prior to a DHCP relay agent filter object insertion

```
#PROGRAM STARTS: Include all the modules that will be used
use strict;
use Infoblox;

#Create a session to the Infoblox appliance

my $session = Infoblox::Session->new(
    master   => "192.168.1.2",
    username => "admin",
    password => "infoblox"
);
unless ($session) {
   die("Construct session failed: ",
       Infoblox::status_code() . ":" . Infoblox::status_detail());
}
print "Session created successfully\n";

#Create a FilterRule relayagent filter object>

my $filter = Infoblox::DHCP::FilterRule::RelayAgent->new(
               filter_name => "relayagent_filter",
               permission => "grant" );
```

#Create a DHCP relay agent filter object

```
my  $relayagentfilter = Infoblox::DHCP::Filter::RelayAgent->new(

       name             => "relayagent_filter",
       circuit_id_name  => "circuit",
       remote_id_name   => "remote"
             );
        my $response = $session->add($relayagentfilter)
       or die("Add DHCP Relay Agent Filter failed: ",
             $session->status_code() . ":" . $session->status_detail());

print "DHCP Relay Agent Filter added successfully\n";
```

#Search for a specific DHCP relay agent filter

```
#Search the DHCP Relay Agent Filter
my @retrieved_objs = $session->search(
               object => "Infoblox::DHCP::Filter::RelayAgent",
               name   => "relayagent.*",
);
my $object = $retrieved_objs[0];
unless ($object) {
    die("Search DHCP Relay Agent Filter  failed: ",
        $session->status_code() . ":" . $session->status_detail());
}

print "Search DHCP relay agent filter found at least 1 matching entry\n";
```

#Get and modify a DHCP relay agent filter attribute

```
#Get  DHCP Relay Agent Filter through the session
my  @retrieved_objs = $session->get(
              object => "Infoblox::DHCP::Filter::RelayAgent" ,
              name   => "relayagent_filter",
       );
my $object = $retrieved_objs[0];
unless ($object) {
    die("Get Network object failed: ",
        $session->status_code() . ":" . $session->status_detail());
}

print "Get DHCP relay agent filter found at least 1 matching entry\n";
```

```
#Modify the attributes of the specified DHCP Relay Agent Filter object
$object->circuit_id_name("diff_Circuit");
$object->remote_id_name("diff_remote");
$session->modify($object)
    or die("Modify DHCP Relay Agent Filter attribute failed: ",
           $session->status_code() . ":" . $session->status_detail());

print "DHCP Relay Agent Filter object modified successfully \n";

#Create the Network object with the member
my $network = Infoblox::DHCP::Network->new(
    network => "10.0.0.0/8",
    comment => "add network",
);
unless($network) {
       die("Construct Network object failed: ",
            Infoblox::status_code() . ":" . Infoblox::status_detail());
}
print "Network object created successfully\n";

#Add the Network object into the Infoblox appliance through a session
$session->add($network)
    or die("Add Network object failed: ",
            $session->status_code() . ":" . $session->status_detail());

print "Network object added to Infoblox appliance successfully\n";
```

#Create a DHCP Range object that uses the relayagent filter

```
my $dhcp_range = Infoblox::DHCP::Range->new(
        end_addr                => "10.0.0.10",
        network                 => "10.0.0.0/8",
        start_addr              => "10.0.0.1",
        authority               => "true",
        filters                 => [$filter],
);
unless($dhcp_range) {
       die("Construct DHCP Range object failed: ",
            Infoblox::status_code() . ":" . Infoblox::status_detail());
}

print "DHCP Range object created successfully\n";

#Add the DHCP Range object into the Infoblox appliance through a session
$session->add($dhcp_range)
    or die("Add DHCP Range object failed: ",
            $session->status_code() . ":" . $session->status_detail());
print "DHCP Range object added to the network successfully\n";

#Remove the Network object into the Infoblox appliance through a session
$session->remove($network)
    or die("Remove Network object failed: ",
            $session->status_code() . ":" . $session->status_detail());
print "Network object removed from Infoblox appliance successfully\n";
```

#Remove a DHCP relay agent filter Object

```
 #Get object through the session
 my @retrieved_objs = $session->get(
         object   => "Infoblox::DHCP::Filter::RelayAgent" ,
         name     => "relayagent_filter",
       );
  my $object = $retrieved_objs[0];    unless ($object) {
    die("Get a DHCP Relay Agent Filterobject failed: ",
        $session->status_code() . ":" . $session->status_detail());
  }

print "Get a DHCP relay agent filter object found at least 1 matching entry\n";

#Submit the object for removal
$session->remove($object)
    or die("Remove a DHCP Relay Agent Filter failed: ",
        $session->status_code() . ":" . $session->status_detail());

print "DHCP relay agent filter removed successfully \n";

####PROGRAM ENDS####
```

AUTHOR

Infoblox Inc. http://www.infoblox.com/

SEE ALSO

Infoblox::Session, Infoblox::DHCP::Network, Infoblox::DHCP::Range, Infoblox::DHCP::FilterRule::RelayAgent, Infoblox::Session->get(), Infoblox::Session->search(), Infoblox::Session->add(), Infoblox::Session->remove(), Infoblox::Session->modify()

COPYRIGHT

Copyright (c) 2017 Infoblox Inc.

Infoblox::DHCP::FilterRule::Fingerprint - DHCP Fingerprint Filter Rule object.

- NAME
- DESCRIPTION
- CONSTRUCTOR
- MODULE METHODS
  - Infoblox::DHCP::Range->filters( )
  - Infoblox::DHCP::RangeTemplate->filters( )
- METHODS
  - filter_name( )
  - permission( )
- AUTHOR
- SEE ALSO
- COPYRIGHT

NAME

Infoblox::DHCP::FilterRule::Fingerprint - DHCP Fingerprint Filter Rule object.

DESCRIPTION

DHCP Fingerprint Filter Rule object.

CONSTRUCTOR

```
my $filter_rule = Infoblox::DHCP::FilterRule::Fingerprint->new(
      filter_name    => $string,           #Required
      permission     => "grant" | "deny", #Required
);
```

MODULE METHODS

The following functions are available to apply to a DHCP Fingerprint Filter Rule object.

Infoblox::DHCP::Range->filters( )

Use this method to add a DHCP fingerprint Filter Rule object to the DHCP Range on the Infoblox appliance. See Infoblox::DHCP::Range->filters() for parameters and return values.

Example

```
#Construct a DHCP Fingerprint Filter Rule object
my $filter_rule = Infoblox::DHCP::FilterRule::Fingerprint->new(
      filter_name    => "filter",
      permission     => "grant",
);

#Add Fingerprint Filter Rule to the DHCP Range object.
my $response = $dhcp_range->filters([$filter_rule]);
```

Infoblox::DHCP::RangeTemplate->filters( )

Use this method to add a DHCP fingerprint Filter Rule object to the DHCP Range Template on the Infoblox appliance. See Infoblox::DHCP::RangeTemplate->filters() for parameters and return values.

Example

```
#Construct a DHCP Fingerprint Filter Rule object
my $filter_rule = Infoblox::DHCP::FilterRule::Fingerprint->new(
      filter_name    => "filter",
      permission     => "grant",
);

#Add Fingerprint Filter Rule to the DHCP Range Template object.
my $response = $dhcp_range_template->filters([$filter_rule]);
```

METHODS

This section describes all the methods that can be used to set and retrieve the attribute values of a DHCP Fingerprint Filter Rule object.

filter_name( )

Use this method to set or retrieve the filter_name of a DHCP Fingerprint Filter Rule object.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

Name of the DHCP Fingerprint filter. Maximum length up to 1024 bytes is supported.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get filter_name
my $filter_name = $filter_rule->filter_name();
#Modify filter_name
$filter_rule->filter_name("fingerprint filter");
```

permission( )

Use this method to set or retrieve the permission of a DHCP Fingerprint Filter Rule object.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

Specify the permission in string format. The permission can be either "grant" or "deny".

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get permission
my $permission = $filter_rule->permission();
#Modify permission
$filter_rule->permission("deny");
```

AUTHOR

Infoblox Inc. http://www.infoblox.com/

SEE ALSO

Infoblox::DHCP::Fingerprint, Infoblox::DHCP::Filter::Fingerprint, Infoblox::DHCP::Range, Infoblox::DHCP::RangeTemplate

---

COPYRIGHT

Copyright (c) 2017 Infoblox Inc.

Infoblox::DHCP::FilterRule::MAC - DHCP MAC Address Filter Rule object.

- NAME
- DESCRIPTION
- CONSTRUCTOR
- MODULE METHODS
  - Infoblox::DHCP::Range->filters( )
- METHODS
  - filter_name( )
  - permission( )
- SAMPLE CODE
- AUTHOR
- SEE ALSO
- COPYRIGHT

NAME

Infoblox::DHCP::FilterRule::MAC - DHCP MAC Address Filter Rule object.

DESCRIPTION

MAC Address Filter Rule allows a user to grant/deny the lease of an IP defined in DHCP Range.

CONSTRUCTOR

```
my $mac_filter_rule = Infoblox::DHCP::FilterRule::MAC->new(
      filter_name    => $string,          #Required
      permission     => "grant" | "deny", #Required
);
```

MODULE METHODS

The following functions can be applied to a DHCP MAC Filter Rule object.

Infoblox::DHCP::Range->filters( )

Use this method to add a DHCP MAC Address Filter Rule object to the DHCP Range on an Infoblox appliance. See Infoblox::DHCP::Range->filters() for parameters and return values.

Example

```
#Construct a DHCP MAC Address Filter Rule object
my $mac_filter_rule1 = Infoblox::DHCP::FilterRule::MAC->new(
      filter_name    => "filter1",
      permission     => "grant",
);

#Add MAC Address Filter Rule to the DHCP Range object.
my $response = $dhcp_range->filters([$mac_filter_rule1]);
```

METHODS

This section describes all the methods that can be used to set and retrieve the attribute values of a DHCP MAC Address Filter Rule object.

filter_name( )

Use this method to set or retrieve the filter_name of a DHCP MAC Address Filter Rule object.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

Name of the DHCP MAC Address filter. Maximum length up to 1024 bytes is supported.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get filter_name
my $filter_name = $mac_filter_rule->filter_name();
#Modify filter_name
$mac_filter_rule->filter_name("filter2");
```

permission( )

Use this method to set or retrieve the permission of a DHCP MAC Address Filter Rule object.

Permission indicates whether members of this MAC address filter are allowed or denied an IP address from the defined DHCP range.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

Permission value can be either "grant" or "deny".

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get permission
my $permission = $mac_filter_rule->permission();
#Modify permission
$mac_filter_rule->permission("deny");
```

SAMPLE CODE

The following sample code demonstrates different functions that can be applied to an object such as modify. Also, this sample includes error handling for the operations.

#Preparation prior to a DHCP MAC Address Filter Rule object insertion

```
#PROGRAM STARTS: Include all the modules that will be used
use strict;
use Infoblox;

#Create a session to the Infoblox appliance

my $session = Infoblox::Session->new(
               master   => "192.168.1.2", #appliance host ip
               username => "admin",       #appliance user login
               password => "infoblox"     #appliance password
);

unless ($session) {
       die("Construct session failed: ",
               Infoblox::status_code() . ":" . Infoblox::status_detail());
}
print "Session created successfully\n";

#Create the Network object
my $network = Infoblox::DHCP::Network->new(
    network => "10.0.0.0/8",
    comment => "add network",
);
unless($network) {
       die("Construct Network object failed: ",
            Infoblox::status_code() . ":" . Infoblox::status_detail());
}
print "Network object created successfully\n";

#Add the Network object into the Infoblox appliance through a session
$session->add($network)
    or die("Add Network object failed: ",
            $session->status_code() . ":" . $session->status_detail());
print "Network object added to Infoblox appliance successfully\n";

#Construct a DHCP MAC Address Filter object
my $mac_filter = Infoblox::DHCP::Filter::MAC->new(
       name => "filter1",
```

```
);

unless ( $mac_filter ) {
       die("Construct MAC Address Filter failed: ",
               Infoblox::status_code() . ":" . Infoblox::status_detail() );
}
print "MAC Address Filter object created successfully\n";

#Add the MAC Address Filter to the Infoblox appliance through a session
$session->add($mac_filter)
    or die("Add MAC Address Filter object failed: ",
            $session->status_code() . ":" . $session->status_detail());
print "MAC Address Filter object added to Infoblox appliance successfully\n";
```

#Create a DHCP MAC Address Filter Rule object

```
#Construct a DHCP MAC Address Filter Rule object
my $mac_filter_rule1 = Infoblox::DHCP::FilterRule::MAC->new(
      filter_name   => "filter1",
      permission    => "grant",
);

unless ( $mac_filter_rule1 ) {
       die("Construct MAC Address Filter Rule failed: ",
               Infoblox::status_code() . ":" . Infoblox::status_detail() );
}
print "MAC Address Filter Rule object created successfully\n";

#Create the DHCP Range object with this MAC Address Filter Rule.
my $dhcp_range = Infoblox::DHCP::Range->new(
        end_addr   => "10.0.0.10",
        network    => "10.0.0.0/8",
        start_addr => "10.0.0.1",
        filters    => [ $mac_filter_rule1 ],
);
unless($dhcp_range) {
       die("Construct DHCP Range object failed: ",
            Infoblox::status_code() . ":" . Infoblox::status_detail());
}
print "DHCP Range object created successfully\n";

#Add the DHCP Range object into the Infoblox appliance through a session
$session->add($dhcp_range)
    or die("Add DHCP Range object failed: ",
            $session->status_code() . ":" . $session->status_detail());
print "DHCP Range object added to Infoblox appliance successfully\n";
```

#Get and modify a MAC Address Filter Rule object

```
#Get MAC Address Filter Rule object from the DHCP Range object through session
my @retrieved_objs = $session->get(
    object      => "Infoblox::DHCP::Range",
    start_addr  => "10.0.0.1",
);

my $object = $retrieved_objs[0];
my $filter = $object->filters();

my @filters = @{$filter};

my $mac_filterrule = $filters[0];

#Modify one of the attributes of the obtained MAC Address Filter Rule object
$mac_filterrule->permission("deny");
$object->filters([$mac_filterrule]);

#Apply the changes
$session->modify($object)
    or die("Modify MAC Address Filter Rule object failed: ",
           $session->status_code() . ":" . $session->status_detail());
print "MAC Address Filter Rule object modified and added to the DHCP Range object successfully \n";
```

#Remove a MAC Address Filter Rule object

```
$object->filters([]);

my $response = $session->modify($object);
```

```
unless($response) {
       die("Remove MAC Address Filter Rule failed: ",
                session->status_code() . ":" . session->status_detail() );
}
print "MAC Address Filter Rule object removed successfully \n";

#Submit the network object for removal
$session->remove($network)
    or die("Remove Network object failed: ",
        $session->status_code() . ":" . $session->status_detail());
print "Network object removed successfully \n";

#Submit the mac address filter object for removal
$session->remove($mac_filter)
    or die("Remove MAC Address Filter object failed: ",
        $session->status_code() . ":" . $session->status_detail());
print "MAC Address Filter object removed successfully \n";

####PROGRAM ENDS####
```

AUTHOR

Infoblox Inc. http://www.infoblox.com/

SEE ALSO

Infoblox::DHCP::MAC, Infoblox::DHCP::FilterRule::RelayAgent, Infoblox::DHCP::Network, Infoblox::DHCP::Range, Infoblox::Session->add(),Infoblox::Session->modify(), Infoblox::Session->remove(),Infoblox::Session

COPYRIGHT

Copyright (c) 2017 Infoblox Inc.

Infoblox::DHCP::FilterRule::NAC - DHCP NAC Filter Rule object.

- NAME
- DESCRIPTION
- CONSTRUCTOR
- MODULE METHODS
  - Infoblox::DHCP::Range->filters( )
- METHODS
  - filter_name( )
  - permission( )
- SAMPLE CODE
- AUTHOR
- SEE ALSO
- COPYRIGHT

NAME

Infoblox::DHCP::FilterRule::NAC - DHCP NAC Filter Rule object.

DESCRIPTION

A NAC Filter Rule object is used to grant or deny the lease of an IP address in a DHCP range based on whether the authentication response matches the NAC filter.

CONSTRUCTOR

```
my $nac_filter_rule = Infoblox::DHCP::FilterRule::NAC->new(
      filter_name    => $string,          #Required
      permission     => "grant" | "deny", #Required
);
```

MODULE METHODS

The following functions can be applied to a DHCP NAC Filter Rule object.

Infoblox::DHCP::Range->filters( )

Use this method to add a DHCP NAC Filter Rule object to a DHCP range. See Infoblox::DHCP::Range->filters() for parameters and return values.

Example

```
#Construct a DHCP NAC Filter Rule object
my $nac_filter_rule1 = Infoblox::DHCP::FilterRule::NAC->new(
      filter_name    => "filter1",
      permission     => "grant",
);

#Add a NAC Filter Rule to the DHCP Range object.
my $response = $dhcp_range->filters([$nac_filter_rule1]);
```

METHODS

This section describes all the methods that can be used to set and retrieve the attribute values of a DHCP NAC Filter Rule object.

filter_name( )

Use this method to set or retrieve the filter_name of a DHCP NAC Filter Rule object.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

The name of the DHCP NAC filter. The maximum length supported is 1024 bytes.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get filter_name
my $filter_name = $nac_filter_rule->filter_name();
#Modify filter_name
$nac_filter_rule->filter_name("filter2");
```

permission( )

Use this method to set or retrieve the permission of a DHCP NAC Filter Rule object.

The permission indicates whether a DHCP client whose NAC authentication response matches this NAC filter is allowed or denied a lease from the defined DHCP range.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

The permission value can be either "grant" or "deny".

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get permission
my $permission = $nac_filter_rule->permission();
#Modify permission
$nac_filter_rule->permission("deny");
```

SAMPLE CODE

The following sample code demonstrates different functions that can be applied to an object such as modify. Also, this sample includes error handling for the operations.

#Preparation prior to a DHCP NAC Filter Rule object insertion

```
#PROGRAM STARTS: Include all the modules that will be used
use strict;
use Infoblox;

#Create a session to the Infoblox appliance

my $session = Infoblox::Session->new(
               master   => "192.168.1.2", #appliance host ip
               username => "admin",       #appliance user login
               password => "infoblox"     #appliance password
);

unless ($session) {
       die("Construct session failed: ",
               Infoblox::status_code() . ":" . Infoblox::status_detail());
}
print "Session created successfully\n";

#Create the Network object
my $network = Infoblox::DHCP::Network->new(
    network => "10.0.0.0/8",
    comment => "add network",
);
unless($network) {
       die("Construct Network object failed: ",
            Infoblox::status_code() . ":" . Infoblox::status_detail());
}
print "Network object created successfully\n";

#Add the Network object into the Infoblox appliance through a session
$session->add($network)
    or die("Add Network object failed: ",
            $session->status_code() . ":" . $session->status_detail());
print "Network object added to Infoblox appliance successfully\n";
```

```
#Construct a DHCP NAC Filter object
my $nac_filter = Infoblox::DHCP::Filter::NAC->new(
       name => "filter1",
);

unless ( $nac_filter ) {
       die("Construct NAC Filter failed: ",
               Infoblox::status_code() . ":" . Infoblox::status_detail() );
}
print "NAC Filter object created successfully\n";

#Add the NAC Filter to the Infoblox appliance through a session
$session->add($nac_filter)
    or die("Add NAC Filter object failed: ",
            $session->status_code() . ":" . $session->status_detail());
print "NAC Filter object added to Infoblox appliance successfully\n";
```

#Create a DHCP NAC Filter Rule object

```
#Construct a DHCP NAC Filter Rule object
my $nac_filter_rule1 = Infoblox::DHCP::FilterRule::NAC->new(
      filter_name   => "filter1",
      permission    => "grant",
);

unless ( $nac_filter_rule1 ) {
       die("Construct NAC Filter Rule failed: ",
               Infoblox::status_code() . ":" . Infoblox::status_detail() );
}
print "NAC Filter Rule object created successfully\n";

#Create the DHCP Range object with this NAC Filter Rule.
my $dhcp_range = Infoblox::DHCP::Range->new(
        end_addr   => "10.0.0.10",
        network    => "10.0.0.0/8",
        start_addr => "10.0.0.1",
        filters    => [ $nac_filter_rule1 ],
);
unless($dhcp_range) {
       die("Construct DHCP Range object failed: ",
            Infoblox::status_code() . ":" . Infoblox::status_detail());
}
print "DHCP Range object created successfully\n";

#Add the DHCP Range object through a session
$session->add($dhcp_range)
    or die("Add DHCP Range object failed: ",
            $session->status_code() . ":" . $session->status_detail());
print "DHCP Range object added to Infoblox appliance successfully\n";
```

#Get and modify a NAC Filter Rule object

```
#Get NAC Filter Rule object from the DHCP Range object through session
my @retrieved_objs = $session->get(
    object      => "Infoblox::DHCP::Range",
    start_addr  => "10.0.0.1",
);

my $object = $retrieved_objs[0];
my $filter = $object->filters();

my @filters = @{$filter};

my $nac_filterrule = $filters[0];

#Modify one of the attributes of the obtained NAC Filter Rule object
$nac_filterrule->permission("deny");
$object->filters([$nac_filterrule]);

#Apply the changes
$session->modify($object)
    or die("Modify NAC Filter Rule object failed: ",
           $session->status_code() . ":" . $session->status_detail());
print "NAC Filter Rule object modified and added to the DHCP Range object successfully \n";
```

#Remove a NAC Filter Rule object

```
$object->filters([]);

my $response = $session->modify($object);
unless($response) {
       die("Remove NAC Filter Rule failed: ",
               session->status_code() . ":" . session->status_detail() );
}
print "NAC Filter Rule object removed successfully \n";

#Submit the network object for removal
$session->remove($network)
    or die("Remove Network object failed: ",
        $session->status_code() . ":" . $session->status_detail());
print "Network object removed successfully \n";

#Submit the NAC Address Filter object for removal
$session->remove($nac_filter)
    or die("Remove NAC Filter object failed: ",
        $session->status_code() . ":" . $session->status_detail());
print "NAC Filter object removed successfully \n";

####PROGRAM ENDS####
```

AUTHOR

Infoblox Inc. http://www.infoblox.com/

SEE ALSO

Infoblox::DHCP::FilterRule::RelayAgent, Infoblox::DHCP::Network, Infoblox::DHCP::Range, Infoblox::Session->add(),Infoblox::Session->modify(), Infoblox::Session->remove(),Infoblox::Session

COPYRIGHT

Copyright (c) 2017 Infoblox Inc.

Infoblox::DHCP::FilterRule::Option - DHCP filter rule option object.

- NAME
- DESCRIPTION
- CONSTRUCTOR
- MODULE METHODS
  - Infoblox::DHCP::Range->filters( )
- METHODS
  - filter_name( )
  - permission( )
- SAMPLE CODE
- AUTHOR
- SEE ALSO
- COPYRIGHT

NAME

Infoblox::DHCP::FilterRule::Option - DHCP filter rule option object.

DESCRIPTION

A DHCP filter rule option object gives the ability to grant and/or deny a lease of an IP defined in DHCP Range.

CONSTRUCTOR

```
my $FilterRuleOption = Infoblox::DHCP::FilterRule::Option->new(
              filter_name   =>  $string,          #Required
              permission    =>  "grant" | "deny", #Required
          );
```

MODULE METHODS

The following functions are available to apply to a DHCP filter rule option object.

Infoblox::DHCP::Range->filters( )

Use this function to specify filters at the DHCP range level. See details for Infoblox::DHCP::Range objects

Example

```
#Construct DHCP filter rule option object
my $filter = Infoblox::DHCP::FilterRule::Option->new(
    filter_name => "filter1",
    permission  => "grant"
);

# Configure  filters
my $response = $range->filters([$res]);
```

METHODS

This section describes all the methods that can be used to configure and retrieve the attribute values of a DHCP filter rule option object.

filter_name( )

Use this method to set or retrieve the filter name of a DHCP filter rule option object.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

Specify filter name in string format.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
# Get filters from  range
my $resp = $range->filters();
# Modify filters
$range->filters([$res]);
```

permission( )

Use this method to set or retrieve the permission of a DHCP filter rule option object. When selecting Grant lease, you need to assign addresses from the address range to requesting hosts whose user class (as defined on their network adapter) matches an entry in the option filter rules. When selecting Deny lease, you need to refuse an address request from a host whose user class matches an entry in the option filter rules.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

The permission can be either "grant" or "deny". The default value is "grant" lease.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
# Get permission
 $res->permission();
#modify permission value
$res->permission("deny");
```

SAMPLE CODE

The following sample code demonstrates the different functions that can be applied to an object such as add, modify. Also, this sample includes error handling for the operations.

##Preparation prior to a DHCP filter rule option object insertion

```
#PROGRAM STARTS: Include all the modules that will be used
use strict;
use Infoblox;

#Create a session to the Infoblox appliance
my $session = Infoblox::Session->new(
    master   => "192.168.1.2",
    username => "admin",
    password => "infoblox"
);
unless ($session) {
   die("Construct session failed: ",
       Infoblox::status_code() . ":" . Infoblox::status_detail());
}
print "Session created successfully\n";

  #Create a DHCP Member object
   my $memberdhcp = Infoblox::DHCP::Member->new(
    name     => "infoblox.localdomain",
    ipv4addr => "192.168.1.2"
   );
   unless($memberdhcp) {
       die("Construct member failed: ",
            Infoblox::status_code() . ":" . Infoblox::status_detail());
   }
   print "DHCP Member object created successfully\n";

#Create the Network object with the member
my $network = Infoblox::DHCP::Network->new(
    network => "10.0.0.0/8",
    comment => "add network",
    members => [ $memberdhcp ]
);
unless($network) {
       die("Construct Network object failed: ",
            Infoblox::status_code() . ":" . Infoblox::status_detail());
}
```

```
print "Network object created successfully\n";
```

#Add the Network object into the Infoblox appliance through a session

```
$session->add($network)
    or die("Add Network object failed: ",
            $session->status_code() . ":" . $session->status_detail());
print "Network object added to Infoblox appliance successfully\n";

my $opt1 = Infoblox::DHCP::Option->new(
      name => "user-class" ,
      value => "77"
   );
```

#Create the Option Filter object

```
my $option_filter = Infoblox::DHCP::Filter::Option->new(
      name => 'filter1' ,
      option_list => [$opt1],

);

unless($option_filter) {
       die("Construct Option Filter object failed: ",
            Infoblox::status_code() . ":" . Infoblox::status_detail());
}
print "Option Filter object created successfully\n";
```

#Add the filter rule option object into the Infoblox appliance

```
$session->add($option_filter)
    or die("Add option Filter object failed: ",
            $session->status_code() . ":" . $session->status_detail());
print "Option Filter object added to Infoblox appliance successfully\n";
```

#Construct DHCP Filter rule Option object

```
my $filter = Infoblox::DHCP::FilterRule::Option->new(
    filter_name => "filter1",
    permission  => "grant"
);
```

#Create a DHCP Range object

```
my $dhcp_range = Infoblox::DHCP::Range->new(
        end_addr     => "10.0.0.10",
        network      => "10.0.0.0/8",
        start_addr   => "10.0.0.1",
        authority    => "true",
        comment      => "add range",
        filters      => [$filter],
        member       => $memberdhcp,
);
unless($dhcp_range) {
       die("Construct DHCP Range object failed: ",
            Infoblox::status_code() . ":" . Infoblox::status_detail());
}
print "DHCP Range object created successfully\n";

#Add the DHCP Range object into the Infoblox appliance through a session
$session->add($dhcp_range)
    or die("Add DHCP Range object failed: ",
            $session->status_code() . ":" . $session->status_detail());
print "DHCP Range object added to the network successfully\n";
```

#Get and modify the filter rule option object

```
#get  object from session
my @result = $session->get(
    object => "Infoblox::DHCP::Range",
    start_addr => "10.0.0.1"
);
unless (scalar(@result) == 0) {
    my $range = $result[0];

    if ($range) {
    my $resp = $range->filters();
```

```
    my @arr = @{$resp};
    my $res = $arr[0];

        #modify permission value
        $res->permission("deny");
        $range->filters([$res]);
        #update session
        $session->modify($range)
            or die("modify failed: ",
               $session->status_code(), $session->status_detail());

            print "Modify filter permisson successful\n";
                        }
                    } else {
             print "No range found.";
                        }
             $session->remove($option_filter);
             print "Remove filter1 successfull\n";

  ####PROGRAM ENDS####
```

AUTHOR

Infoblox Inc. http://www.infoblox.com/

SEE ALSO

Infoblox::Session, Infoblox::DHCP::Range, Infoblox::DHCP::Network, Infoblox::DHCP::Member, Infoblox::DHCP::Option, Infoblox::Session->get(), Infoblox::Session->add(),

COPYRIGHT

Copyright (c) 2017 Infoblox Inc.

Infoblox::DHCP::FilterRule::RelayAgent - DHCP RelayAgent Filter Rule object.

- NAME
- DESCRIPTION
- CONSTRUCTOR
- MODULE METHODS
  - Infoblox::DHCP::Range->filters( )
- METHODS
  - filter_name( )
  - permission( )
- SAMPLE CODE
- AUTHOR
- SEE ALSO
- COPYRIGHT

NAME

Infoblox::DHCP::FilterRule::RelayAgent - DHCP RelayAgent Filter Rule object.

DESCRIPTION

A relay agent adds the circuit ID and Remote ID information to the client request and sends it to DHCP Server. Infoblox appliance uses Relay Agent Filter Rules to grant/deny the lease to a request that comes from those Relay Agents.

CONSTRUCTOR

```
my $ra_filter_rule = Infoblox::DHCP::FilterRule::RelayAgent->new(
      filter_name    => $string,          #Required
      permission     => "grant" | "deny", #Required
);
```

MODULE METHODS

The following functions are available to apply to a DHCP RelayAgent Filter Rule object.

Infoblox::DHCP::Range->filters( )

Use this method to add a DHCP RelayAgent Filter Rule object to the DHCP Range on the Infoblox appliance. See Infoblox::DHCP::Range->filters() for parameters and return values.

Example

```
#Construct a DHCP RelayAgent Filter Rule object
my $ra_filter_rule1 = Infoblox::DHCP::FilterRule::RelayAgent->new(
      filter_name    => "filter1",
      permission     => "grant",
);

#Add RelayAgent Filter Rule to the DHCP Range object.
my $response = $dhcp_range->filters([$ra_filter_rule1]);
```

METHODS

This section describes all the methods that can be used to set and retrieve the attribute values of a DHCP RelayAgent Filter Rule object.

filter_name( )

Use this method to set or retrieve the filter_name of a DHCP RelayAgent Filter Rule object.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

Name of the DHCP RelayAgent filter. Maximum length up to 1024 bytes is supported.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get filter_name
my $filter_name = $ra_filter_rule->filter_name();
#Modify filter_name
$ra_filter_rule->filter_name("filter2");
```

permission( )

Use this method to set or retrieve the permission of a DHCP RelayAgent Filter Rule object.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

Specify the permission in string format. The permission can be either "grant" or "deny".

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get permission
my $permission = $ra_filter_rule->permission();
#Modify permission
$ra_filter_rule->permission("deny");
```

SAMPLE CODE

The following sample code demonstrates the different functions that can be applied to an object such as modify. Also, this sample includes error handling for the operations.

#Preparation prior to a DHCP RelayAgent Filter Rule object insertion

```
#PROGRAM STARTS: Include all the modules that will be used
use strict;
use Infoblox;

#Create a session to the Infoblox appliance

my $session = Infoblox::Session->new(
               master   => "192.168.1.2", #appliance host ip
               username => "admin",       #appliance user login
               password => "infoblox"     #appliance password
);

unless ($session) {
       die("Construct session failed: ",
               Infoblox::status_code() . ":" . Infoblox::status_detail());
}
print "Session created successfully\n";

#Create the Network object
my $network = Infoblox::DHCP::Network->new(
    network => "10.0.0.0/8",
    comment => "add network",
);
unless($network) {
       die("Construct Network object failed: ",
            Infoblox::status_code() . ":" . Infoblox::status_detail());
}
print "Network object created successfully\n";

#Add the Network object into the Infoblox appliance through a session
$session->add($network)
    or die("Add Network object failed: ",
            $session->status_code() . ":" . $session->status_detail());
print "Network object added to Infoblox appliance successfully\n";

#Construct a DHCP RelayAgent Filter object
my $ra_filter = Infoblox::DHCP::Filter::RelayAgent->new(
       name            => "filter1",
```

```
        circuit_id_name => "circuit",
        remote_id_name  => "remote"
);

unless ( $ra_filter ) {
        die("Construct RelayAgent Filter failed: ",
                Infoblox::status_code() . ":" . Infoblox::status_detail() );
}
print "RelayAgent Filter object created successfully\n";

#Add the RelayAgent Filter to the Infoblox appliance through a session
$session->add($ra_filter)
    or die("Add RelayAgent Filter object failed: ",
            $session->status_code() . ":" . $session->status_detail());
print "RelayAgent Filter object added to Infoblox appliance successfully\n";
```

#Create a DHCP RelayAgent Filter Rule object

```
#Construct a DHCP RelayAgent Filter Rule object
my $ra_filter_rule1 = Infoblox::DHCP::FilterRule::RelayAgent->new(
      filter_name   => "filter1",
      permission    => "grant",
);

unless ( $ra_filter_rule1 ) {
        die("Construct RelayAgent Filter Rule failed: ",
                Infoblox::status_code() . ":" . Infoblox::status_detail() );
}
print "RelayAgent Filter Rule object created successfully\n";

#Create the DHCP Range object with this RelayAgent Filter Rule.
my $dhcp_range = Infoblox::DHCP::Range->new(
        end_addr   => "10.0.0.10",
        network    => "10.0.0.0/8",
        start_addr => "10.0.0.1",
        filters    => [ $ra_filter_rule1 ],
);
unless($dhcp_range) {
        die("Construct DHCP Range object failed: ",
            Infoblox::status_code() . ":" . Infoblox::status_detail());
}
print "DHCP Range object created successfully\n";

#Add the DHCP Range object into the Infoblox appliance through a session
$session->add($dhcp_range)
    or die("Add DHCP Range object failed: ",
            $session->status_code() . ":" . $session->status_detail());
print "DHCP Range object added to Infoblox appliance successfully\n";
```

#Get and modify a RelayAgent Filter Rule object

```
#Get RelayAgent Filter Rule object from the DHCP Range object through session
my @retrieved_objs = $session->get(
    object      => "Infoblox::DHCP::Range",
    start_addr  => "10.0.0.1",
);

my $object = $retrieved_objs[0];
my $filter = $object->filters();

my @filters = @{$filter};

my $ra_filterrule = $filters[0];

#Modify one of the attributes of the obtained RelayAgent Filter Rule object
$ra_filterrule->permission("deny");
$object->filters([$ra_filterrule]);

#Apply the changes
$session->modify($object)
    or die("Modify RelayAgent Filter Rule object failed: ",
           $session->status_code() . ":" . $session->status_detail());
print "RelayAgent Filter Rule object modified and added to the DHCP Range object successfully \n";
```

#Remove a RelayAgent Filter Rule object

```
$object->filters([]);
```

```
my $response = $session->modify($object);
unless($response) {
       die("Remove RelayAgent Filter Rule failed: ",
               session->status_code() . ":" . session->status_detail() );
}
print "RelayAgent Filter Rule object removed successfully \n";

#Submit the network object for removal
$session->remove($network)
    or die("Remove Network object failed: ",
        $session->status_code() . ":" . $session->status_detail());
print "Network object removed successfully \n";

#Submit the relay agent filter object for removal
$session->remove($ra_filter)
    or die("Remove RelayAgent Filter object failed: ",
        $session->status_code() . ":" . $session->status_detail());
print "RelayAgent Filter object removed successfully \n";

####PROGRAM ENDS####
```

AUTHOR

Infoblox Inc. http://www.infoblox.com/

SEE ALSO

Infoblox::DHCP::MAC, Infoblox::DHCP::FilterRule::RelayAgent, Infoblox::DHCP::Network, Infoblox::DHCP::Range, Infoblox::Session->add(), Infoblox::Session->get(), Infoblox::Session->modify(), Infoblox::Session->remove(),Infoblox::Session

---

COPYRIGHT

Copyright (c) 2017 Infoblox Inc.

Infoblox::DHCP::Fingerprint - DHCP Fingerprint object.

- NAME
- DESCRIPTION
- CONSTRUCTOR
- SESSION METHODS
  - Infoblox::Session->add( )
  - Infoblox::Session->get( )
  - Infoblox::Session->modify( )
  - Infoblox::Session->remove( )
  - Infoblox::Session->search( )
- METHODS
  - comment( )
  - device_class( )
  - name( )
  - disable( )
  - extattrs( )
  - extensible_attributes( )
  - option_sequence( )
  - ipv6_option_sequence( )
  - type( )
  - vendor_id( )
- SAMPLE CODE
- AUTHOR
- SEE ALSO
- COPYRIGHT

NAME

Infoblox::DHCP::Fingerprint - DHCP Fingerprint object.

DESCRIPTION

DHCP Fingerprint object.

CONSTRUCTOR

```
my $fingerprint = Infoblox::DHCP::Fingerprint->new(
   name                  => $string,                              # required
   device_class          => $string,                              # required
   type                  => 'CUSTOM' | 'STANDARD',                # optional, default is 'CUSTOM'
   comment               => $string,                              # optional
   disable               => 'true' | 'false',                     # optional, default is 'false'
   vendor_id             => [$string1, $string2, ...],            # optional
   option_sequence       => [$string1, $string2, ...],            # optional
   ipv6_option_sequence  => [$string1, $string2, ...],            # optional
   extattrs              => { $string => $extattr, ... },         # optional / Default is undefined
   extensible_attributes => $extattrs,                            # optional
);
```

You cannot set both extattrs and extensible_attributes attributes at the same time.

At least one of option_sequence, ipv6_option_sequence or vendor_id must be defined.

SESSION METHODS

This section describes all the methods in an Infoblox::Session module that you can apply to DHCP Fingerprint objects.

Infoblox::Session->add( )

Use this method to add an object to the Infoblox appliance if the type value is 'CUSTOM'. Otherwise this method is not supported. See Infoblox::Session->add() for parameters and return values.

Example

```
#Construct an object
my $fingerprint = Infoblox::DHCP::Fingerprint->new(
   name => 'Infoblox',
   device_class => 'Inc',
```

```
    option_sequence => ['1'],
);

#Submit for addition
my $response = $session->add( $fingerprint );
```

Infoblox::Session->get( )

Use this method to retrieve all the matching objects from the Infoblox appliance. See Infoblox::Session->get() for parameters and return values.

Key References

```
Apply the following attributes to get a specific DHCP Fingerprint object:

 name                       - Optional. The name of the DHPC Fingerprint object.
 device_class               - Optional. The class of DHCP Fingerprint object.
 type                       - Optional. The type of DHCP Fingerprint object.
 comment                    - Optional. The comment.
 extattrs                   - Optional. A hash reference containing extensible attributes.
 extensible_attributes      - Optional. A hash reference containing extensible attributes.
```

Example

```
# Get DHCP Fingerprint object by name
my @retrieved_objs = $session->get(
    object  => 'Infoblox::DHCP::Fingerprint',
    name    => 'Infoblox',
);

# Get all base DHCP Fingerprint objects
my @retrieved_objs = $session->get(
    object  => 'Infoblox::DHCP::Fingerprint',
    type    => 'STANDARD',
);
```

Infoblox::Session->modify( )

Use this method to modify an object in the Infoblox appliance. See Infoblox::Session->modify() for parameters and return values.

Example

```
# Modify comment
$fingerprint->comment("This is modified object");
# Submit modification
my $response = $session->modify( $fingerprint );
```

Infoblox::Session->remove( )

Use this method to remove an object from the Infoblox appliance if the type value is 'CUSTOM'. Otherwise this method is not supported. See Infoblox::Session->remove() for parameters and return values.

To remove a specific object, first use `get()` or `search()` to retrieve the specific object, and then submit this object for removal.

Example

```
# Get the object by name
my @retrieved_objs = $session->get(
    object    => 'Infoblox::DHCP::Fingerprint',
    name      => 'Infoblox',
);
#Find the desired object from the retrieved list.
my $desired_fingerprint = $retrieved_objs[0];
# Submit for removal
my $response = $session->remove( $desired_fingerprint );
```

Infoblox::Session->search( )

Use this method to search for DHCP Fingerprint objects in the Infoblox appliance. See Infoblox::Session->search() for parameters and return values.

Key References

```
Apply the following attributes to search for specific DHCP Fingerprint objects:

  name                       - Optional. The name of the DHPC Fingerprint object (regular
expression).
  device_class               - Optional. The class of DHCP Fingerprint object (regular expression).
  type                       - Optional. The type of DHCP Fingerprint object.
```

```
 comment                    - Optional. The comment (regular expression).
 extattrs                   - Optional. A hash reference containing extensible attributes.
 extensible_attributes      - Optional. A hash reference containing extensible attributes.
```

Example

```
# Search DHCP Fingerprint object by name
my @retrieved_objs = $session->search(
   object  => 'Infoblox::DHCP::Fingerprint',
   name    => 'Infoblox',
);

# Search all base DHCP Fingerprint objects
my @retrieved_objs = $session->search(
   object  => 'Infoblox::DHCP::Fingerprint',
   type    => 'STANDARD',
);
```

METHODS

This section describes all the methods that you can use to configure and retrieve the attribute values of DHCP Fingerprint objects.

comment( )

Use this method to set or retrieve a descriptive comment.

Parameter

Desired comment in string format with a maximum of 256 bytes.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get attribute value
my $value = $fingerprint->comment();
#Modify attribute value
$fingerprint->comment('new comment');
```

device_class( )

Use this method to set or retrieve a class of DHCP Fingerprint object.

Parameter

The class in string format with a maximum of 256 bytes.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get attribute value
my $value = $fingerprint->device_class();
#Modify attribute value
$fingerprint->device_class('Infoblox Inc.');
```

name( )

Use this method to set or retrieve a name of DHCP Fingerprint object.

Parameter

The name in string format with a maximum of 256 bytes.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get attribute value
my $value = $fingerprint->name();
#Modify attribute value
$fingerprint->name('custom fingerprint');
```

disable( )

Use this method to enable or disable a DHCP Fingerprint object.

If a DHCP Fingerprint object is enabled, the option number sequence and vendor_ids for this operating system or device will be included in the list of fingerprints to match incoming lease requests.

Parameter

Specify "true" to disable DHCP Fingerprint object or "false" to enable it. The default value is 'false'.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get attribute value
my $value = $fingerprint->disable();
#Modify attribute value
$fingerprint->disable('false');
```

extattrs( )

Use this method to set or retrieve the extensible attributes associated with a DHCP Fingerprint object.

Parameter

Valid value is a hash reference containing the names of extensible attributes and their associated values (Infoblox::Grid::Extattr objects).

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get extattrs
my $ref_extattrs = $fingerprint->extattrs();
#Modify extattrs
$fingerprint->extattrs({ 'Site' => $extattr1, 'Administrator' => $extattr2 });
```

extensible_attributes( )

Use this method to set or retrieve the extensible attributes associated with DHCP Fingerprint objects.

Parameter

For valid values for extensible attributes, see Infoblox::Grid::ExtensibleAttributeDef/Extensible Attribute Values.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get attribute value
my $value = $fingerprint->extensible_attributes();
#Modify attribute value
$fingerprint->extensible_attributes({ 'Site' => 'Santa Clara', 'Administrator' => [ 'Peter', 'Tom' ]
});
```

option_sequence( )

Use this method to set or retrieve a list (comma separated list) of IPv4 option number sequences of the device or operating system.

At least one of option_sequence, ipv6_option_sequence or vendor_id must be defined.

Parameter

Reference to an array of IPv4 option number sequence (comma separated list) in string format.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get attribute value
my $value = $fingerprint->option_sequence();
#Modify attribute value
$fingerprint->option_sequence(['1,2,3']);
```

ipv6_option_sequence( )

Use this method to set or retrieve a list (comma separated list) of IPv6 option number sequences of the device or operating system.

At least one of option_sequence, ipv6_option_sequence or vendor_id must be defined.

Parameter

Reference to an array of IPv6 option number sequence (comma separated list) in string format.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get attribute value
my $value = $fingerprint->ipv6_option_sequence();
#Modify attribute value
$fingerprint->ipv6_option_sequence(['1,2,3']);
```

type( )

Use this method to set or retrieve the type of the DHCP Fingerprint object.

Parameter

Valid value can be 'STANDARD' or 'CUSTOM'. Standard fingerprints are the fingerprints from fingerbank. Custom fingerprints are user defined fingerprints.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get attribute value
my $value = $fingerprint->type();
#Modify attribute value
$fingerprint->type('CUSTOM');
```

vendor_id( )

Use this method to set or retrieve a list of vendor IDs of the device or operating system.

At least one of option_sequence, ipv6_option_sequence or vendor_id must be defined.

Parameter

Reference to an array of vendor ID in string format.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get attribute value
my $value = $fingerprint->vendor_id();
#Modify attribute value
$fingerprint->vendor_id(['3','4']);
```

SAMPLE CODE

The following sample code demonstrates the different functions that can be applied to an object such as add, get, modify, search and remove. This sample also includes error handling for the operations.

#Preparation prior to getting and modifying DHCP Fingerprint object

```
use strict;
use Infoblox;

#refers to Infoblox Appliance IP address
my $host_ip = "192.168.1.2";

#Create a session to the Infoblox appliance

my $session = Infoblox::Session->new(
    master   => $host_ip,
    username => "admin",
    password => "infoblox"
);
unless ($session) {
       die("Construct session failed: ",
               Infoblox::status_code() . ":" . Infoblox::status_detail());
}
print "Session created successfully\n";
```

#Create DHCP Fingerprint object

```
my $fingerprint = Infoblox::DHCP::Fingerprint->new(
   'name' => 'Infoblox',
   'device_class' => 'Inc',
   'option_sequence' => ['1'],
);

unless ($fingerprint) {
       die("Construct DHCP Fingerprint object failed: ",
               Infoblox::status_code() . ":" . Infoblox::status_detail());
}
print "DHCP Fingerprint object created successfully\n";

my $response = $session->add($fingerprint);

unless ($response) {
    die("Add DHCP Fingerprint object failed: ",
           $session->status_code() . ":" . $session->status_detail());
}
print"DHCP Fingerprint object added successfully \n";
```

#Search for DHCP Fingerprint object

```
my @retrieved_objs = $session->search(
   object   => 'Infoblox::DHCP::Fingerprint',
   name     => 'Infoblox',
);

$fingerprint = $retrieved_objs[0];

unless ($fingerprint) {
        die("Search DHCP Fingerprint object object failed: ",
                $session->status_code() . ":" . $session->status_detail());
}
print "Search DHCP Fingerprint object found at least 1 matching entry\n";
```

#Get and modify DHCP Fingerprint object

```
@retrieved_objs = $session->get(
   object  => 'Infoblox::DHCP::Fingerprint',
   name    => 'Infoblox',
```

```
);

$fingerprint = $retrieved_objs[0];

unless ($fingerprint) {
    die("Get DHCP Fingerprint object failed: ",
            $session->status_code() . ":" . $session->status_detail());
}
print"Get DHCP Fingerprint object successful \n";

$fingerprint->comment("This is modified object");

$session->modify($fingerprint)
   or die("Modify DHCP Fingerprint object failed",
             $session->status_code() . ":" . $session->status_detail());
print "DHCP Fingerprint object modified successfully \n";
```

#Remove DHCP Fingerprint object

```
@retrieved_objs = $session->get(
   object   => 'Infoblox::DHCP::Fingerprint',
   name     => 'Infoblox',
);

$fingerprint = $retrieved_objs[0];

unless ($fingerprint) {
    die("Get DHCP Fingerprint object failed: ",
            $session->status_code() . ":" . $session->status_detail());
}
print"Get DHCP Fingerprint object successful \n";

$session->remove( $fingerprint )
    or die("Remove DHCP Fingerprint object failed",
            $session->status_code() . ":" . $session->status_detail());
print"DHCP Fingerprint object removed successfully \n";

####PROGRAM ENDS####
```

AUTHOR

Infoblox Inc. http://www.infoblox.com/

SEE ALSO

Infoblox::Session

COPYRIGHT

Copyright (c) 2017 Infoblox Inc.

Infoblox::DHCP::FixedAddr - DHCP Fixed Address object.

- NAME
- DESCRIPTION
- CONSTRUCTOR
- SESSION METHODS
  - Infoblox::Session->add( )
  - Infoblox::Session->get( )
  - Infoblox::Session->modify( )
  - Infoblox::Session->remove( )
  - Infoblox::Session->search( )
- METHODS
  - allow_telnet( )
  - bootfile( )
  - agent_circuit_id( )
  - agent_remote_id( )
  - always_update_dns( )
  - bootserver( )
  - client_identifier_prepend_zero( )
  - cli_credentials( )
  - comment( )
  - configure_for_dhcp( )
  - cloud_info( )
  - ddns_domainname( )
  - ddns_hostname( )
  - deny_bootp( )
  - device_description( )
  - device_location( )
  - device_type( )
  - device_vendor( )
  - dhcp_client_identifier( )
  - disable( )
  - discover_now_status( )
  - discovered_data( )
  - discovered_mac( )
  - discovered_name( )
  - discoverer( )
  - enable_ddns( )
  - extattrs( )
  - extensible_attributes( )
  - first_discovered( )
  - ignore_dhcp_option_list_request( )
  - ipv4addr( )
  - is_invalid_mac( )
  - last_discovered( )
  - logic_filters( )
  - mac( )
  - match_client( )
  - member( )
  - ms_ad_user_data( )
  - name( )
  - netbios( )
  - ms_options( )
  - network( )
  - network_component_description( )
  - network_component_ip( )
  - network_component_name( )
  - network_component_port_description( )
  - network_component_port_name( )
  - network_component_port_number( )
  - network_component_type( )
  - network_view( )
  - nextserver( )
  - options( )
  - os( )
  - override_ddns_domainname( )
  - override_cli_credentials( )
  - override_logic_filters( )
  - port_duplex( )
  - port_link_status( )
  - port_speed( )
  - port_status( )
  - port_vlan_description( )

- port_vlan_name( )
- port_vlan_number( )
- pxe_lease_time( )
- reserved_interface( )
- template( )
- v_cluster( )
- v_datacenter( )
- v_host( )
- v_name( )
- v_netadapter( )
- v_switch( )
- v_type( )

- SAMPLE CODE
- AUTHOR
- SEE ALSO
- COPYRIGHT

NAME

Infoblox::DHCP::FixedAddr - DHCP Fixed Address object.

DESCRIPTION

A Fixed Address is a specific IP address that a DHCP server always assigns when a lease request comes from a particular MAC address of the client.

CONSTRUCTOR

```
 my $fixed_addr = Infoblox::DHCP::FixedAddr -> new(
       ipv4addr                          => $ipv4addr,                  #Required
       agent_circuit_id                  => $circuit_id,                #Optional / Default is undefined
       agent_remote_id                   => $remote_id,                 #Optional / Default is undefined
       always_update_dns                 => "true" | "false",           #Optional / Default is "false"
       bootfile                          => $filename | undef,          #Optional / Default is undefined
       bootserver                        => $ipv4addr | $fqdn | undef,  #Optional / Default is undefined
       client_identifier_prepend_zero    => "true" | "false",           #Optional / Default is "false"
       comment                           => $string,                    #Optional / Default is empty
       configure_for_dhcp                => "true" | "false",           #Optional / Default is "false"
                                                                        # When both the IP address and MAC
address are present, the default value is "true"
                                                                        # Applicable only when adding this
fixed address to a host
       ddns_domainname                   => $string,                    #Optional / Default is empty
       ddns_hostname                     => $string,                    #Optional / Default is empty
       deny_bootp                        => "true" | "false" | undef,   #Optional / Default is undefined
       dhcp_client_identifier            => $string,                    #Optional / Default is empty
       disable                           => "true" | "false",           #Optional / Default is "false"
       enable_ddns                       => "true" | "false",           #Optional / Default is "false"
       extattrs                          => { $string => $extattr, ... },  #Optional / Default is undefined
       extensible_attributes             => { $string => $string | $num, $string => [ $string | $num, ... ],
... } #Optional / Default is undefined
       ignore_dhcp_option_list_request   => "true" | "false" | undef,   #Optional / Default is "false"
       last_discovered                   => $string,                    #Optional / Default is empty
       mac                               => $mac_addr,                  #Optional / Default is empty
       match_client                      => $string,                    #Optional / Default is "MAC"
                                                                        #Possible values are:
                                                                        #
"MAC","CLIENT_IDENTIFIER","RESERVED",
                                                                        # "CIRCUIT_ID", "REMOTE_ID"
       member                            => $msmember,                  #Optional
       ms_options                        => [$msopt1, $msopt2, ...]     #Optional / Default is empty
       name                              => $name,                      #Optional
       network                           => $network,                   #Optional
       network_view                      => $NetworkView,               #Optional / Default is "default"
network view
       nextserver                        => $ipv4addr | $fqdn | undef,  #Optional / Default is undefined
       options                           => [$option1, $option2, ...]   #Optional / Default is empty
       override_ddns_domainname          => "true" | "false",           #Optional / Default value is "false"
       pxe_lease_time                    => $time,                      #Optional / Default is empty
       template                          => $string,                    #Optional / Default is undefined
       cli_credentials                   => [$cli1, ...],               #Optional
       override_cli_credentials          => "true" | "false",           #Optional / Default is "false"
       reserved_interface                => $interface,                 #Optional
       device_type                       => $string,                    #Optional
```

```
        device_vendor                   => $string,                     #Optional
        device_location                 => $string,                     #Optional
        device_description              => $string,                     #Optional
        allow_telnet                    => "true" | "false",            #Optional / Default is "false"
        logic_filters                   => [ $filter_name | $filter_obj, ...], # Optional / Default is
undefined
        override_logic_filters          => "true' | 'false",                    # Optional / Default value is
"false"
 );
```

You cannot set both extattrs and extensible_attributes attributes at the same time.

SESSION METHODS

This section describes all the methods in Infoblox::Session module that you can apply to a DHCP Fixed Address object.

Infoblox::Session->add( )

Use this method to add a DHCP Fixed Address object to the Infoblox appliance. See Infoblox::Session->add() for parameters and return values.

Example

```
        #Construct a DHCP Fixed Address object
        my $fixed_addr = Infoblox::DHCP::FixedAddr->new(
                "ipv4addr"                          => "10.0.0.5",
                "bootfile"                          => "bootfile",
                "bootserver"                        => "bootserver.com",
                "client_identifier_prepend_zero"    => "true",
                "comment"                           => "add fixed address using uid",
                "ddns_domainname"                   => "ddns.domain.com",
                "ddns_hostname"                     => "ddns.host.domain.com",
                "deny_bootp"                        => "false",
                "dhcp_client_identifier"            => "foo",
                "disable"                           => "false",
                "enable_ddns"                       => "false",
                "match_client"                      => "MAC",
                "network"                           => "10.0.0.0/8",
                "network_view"                      => $nview,
                "nextserver"                        => "3.3.3.3",
                "options"                           => [ $option1,  $option2, $option3,  $option4, $option5,
      $option6 ],
                "pxe_lease_time"                    => "3600",
         );

         #Submit for addition
         my $response = $session->add($fixed_addr)
```

Infoblox::Session->get( )

Use this method to retrieve all the matching objects from the Infoblox appliance. See Infoblox::Session->get() for parameters and return values.

'discover_now_status' will not be returned by default, use 'return_methods' to request it if needed.

Key References

```
        Apply the following attributes to get a specific DHCP Fixed Address object:

          ipv4addr                - Optional. IPV4 address of the fixed address.
          mac                     - Optional. MAC address of the fixed address.
          dhcp_client_identifier  - Optional, DHCP option 61 client identifier.
          network                 - Optional. Network address of the fixed address.
          network_view            - Optional. Network view in which the fixed address is located. By default,
      all network views are searched.
          extattrs                - Optional. A hash reference containing extensible attributes.
          extensible_attributes   - Optional. A hash reference containing extensible attributes.
          v_cluster               - Optional. Discovered VMware cluster name.
          v_datacenter            - Optional. Discovered VMware datacenter name.
          v_host                  - Optional. Discovered VMware server name.
          v_name                  - Optional. Discovered VMware host or virtual machine name.
          v_netadapter            - Optional. Discovered VMware physical network adapter name.
          v_switch                - Optional. Discovered VMware virtual switch name.
          v_type                  - Optional. Discovered VMware entity type (possible values:
      "VirtualMachine", "HostSystem", "VirtualCenter").
```

Examples

```
        #Get DHCP Fixed Address object through the session
```

```
my @retrieved_objs = $session->get(
    object => "Infoblox::DHCP::FixedAddr",
    mac    => "11:11:11:11:11.11",
);
my @retrieved_objs = $session->get(
    object => "Infoblox::DHCP::FixedAddr",
    dhcp_client_identifier => "foo",
);
my @retrieved_objs = $session->get(
    object       => "Infoblox::DHCP::FixedAddr",
    ipv4addr     => "10.0.0.4",
    network_view => "default",
);

my @retrieved_objs = $session->get(
   object                => "Infoblox::DHCP::FixedAddr",
   extensible_attributes => { 'Site' => 'Santa Clara' });

 # get FixedAddr by v_type:
 my @retrieved_objs = $session->get(
   object    => 'Infoblox::DHCP::FixedAddr',
   v_type    => 'VirtualMachine');
```

Infoblox::Session->modify( )

Use this method to modify a DHCP Fixed Address object in the Infoblox appliance. See Infoblox::Session->modify() for parameters and return values.

Example

```
#Use method to modify the pxe_lease_time
$fixed_address->pxe_lease_time("1234");
#Submit modification
my $response = $session->modify( $fixed_address );
```

Infoblox::Session->remove( )

Use this method to remove a DHCP Fixed Address object from the Infoblox appliance. See Infoblox::Session->remove() for parameters and return values.

To remove a specific object, first use `get()` or `search()` to retrieve the specific DHCP Fixed Address object, and then submit this object for removal.

Example

```
#Get DHCP Fixed Address object through the session
my @retrieved_objs = $session->get(
    object       => "Infoblox::DHCP::FixedAddr",
    mac          => "11:11:11:11:11:11",
    network_view => "default"
);
#Find the desired object from the retrieved list.
my $desired_fixed_addr = $retrieved_objs[0];
#Submit for removal
my $response = $session->remove( $desired_fixed_addr );
```

Infoblox::Session->search( )

Use this method to search for DHCP Fixed Address object and matching Host object with MAC address in the Infoblox appliance (see below). See Infoblox::Session->search() for parameters and return values.

Note that if the Infoblox library is invoked with the :hostaddress option, only DHCP Fixed Address objects are searched.

'discover_now_status' will not be returned by default, use 'return_methods' to request it if needed.

Key References

```
Apply the following attributes to search for a specific DHCP Fixed Address object:

  name          - Optional. Name of the fixed address. If name is passed it implies no_hostaddresses
  ipv4addr      - Optional. IPV4 address of the fixed address (regular expression).
  mac           - Optional. MAC address of the fixed address (regular expression).
  network       - Optional. Network address of the fixed address.
  network_view  - Optional. Network view in which the fixed address is located. By default, all
network views are searched.
  extattrs      - Optional. A hash reference containing extensible attributes.
  extensible_attributes - Optional. A hash reference containing extensible attributes.
  v_cluster     - Optional. Discovered VMware cluster name (regular expression).
  v_datacenter  - Optional. Discovered VMware datacenter name (regular expression).
```

```
  v_host        - Optional. Discovered VMware server name (regular expression).
  v_name        - Optional. Discovered VMware host or virtual machine name (regular expression).
  v_netadapter  - Optional. Discovered VMware physical network adapter name (regular expression).
  v_switch      - Optional. Discovered VMware virtual switch name (regular expression).
  v_type        - Optional. Discovered VMware entity type (possible values: "VirtualMachine",
"HostSystem", "VirtualCenter").
  no_hostaddresses - Optional. If specified and set to "true", only Fixed Address objects will be
returned by the search.
```

For more information about searching extensible attributes, see Infoblox::Grid::ExtensibleAttributeDef/Searching Extensible Attributes.

Examples

```
# Search for all DHCP Fixed Address objects that are specified in the network "10.0.0.0/8" in the
"default" network view.
my @retrieved_objs = $session->search(
    object       => "Infoblox::DHCP::FixedAddr",
    mac          => "11:11:11:11:11.*",
    network      => "10.0.0.0/8",
    network_view => "default"
);

# Search for all DHCP Fixed Address objects with "Santa Clara" for the "Site" extensible attribute
my @retrieved_objs = $session->search(
    object               => "Infoblox::DHCP::FixedAddr",
    extensible_attributes => { 'Site' => 'Santa Clara' }
);

# search FixedAddr by v_datacenter:
my @retrieved_objs = $session->search(
   object               => 'Infoblox::DHCP::FixedAddr',
   v_datacenter => 'test.*');
```

METHODS

This section describes all the methods that you can use to set and retrieve the attribute values of a DHCP Fixed Address object.

allow_telnet( )

Use this method to set or retrieve the flag that enables copying SSH credential to TELNET.

Parameter

Specify 'true' to copy SSH credential to TELNET or 'false' to disable it. The default value is 'false'.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
# Get attribute value
my $value = $fixed_addr->allow_telnet();
# Modify attribute value
$fixed_addr->allow_telnet('true');
```

bootfile( )

Use this method to set or retrieve the bootfile value of a DHCP Fixed Address object.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

This option is overridden independently from Infoblox::DHCP::FixedAddr->bootserver() and Infoblox::DHCP::FixedAddr->nextserver().

Parameter

The name of the file that the client must download. The default value is undefined.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get bootfile
my $bootfile = $fixed_addr->bootfile();
#Modify bootfile
$fixed_addr->bootfile("boot_file");
#Un-override bootfile
$fixed_addr->bootfile(undef);
```

agent_circuit_id( )

Use this method to set or retrieve the agent circuit ID of a DHCP Fixed Address object.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

Use the agent_circuit_id to assign fixed addresses to DHCP clients from whom the member DHCP server receives DHCPDISCOVER and DHCPREQUEST messages that include option 82. To enable the member DHCP server to match the agent circuit ID, set the "match_client" field to "CIRCUIT_ID".

Returns

If you specified a parameter, the method returns true when the modification succeeds and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get agent_circuit_id
my $uid = $fixed_addr->agent_circuit_id();
#Modify agent_circuit_id
$fixed_addr->agent_circuit_id("foo");
```

agent_remote_id( )

Use this method to set or retrieve the agent remote ID of a DHCP Fixed Address object.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

Use the agent_remote_id to assign fixed addresses to DHCP clients from whom the member DHCP server receives DHCPDISCOVER and DHCPREQUEST messages that include option 82. To enable the member DHCP server to match the agent remote ID, set the "match_client" field to "REMOTE_ID".

Returns

If you specified a parameter, the method returns true when the modification succeeds and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get agent_remote_id
my $uid = $fixed_addr->agent_remote_id();
#Modify agent_remote_id
$fixed_addr->agent_remote_id("foo");
```

always_update_dns( )

Use this method to set or retrieve the always_update_dns flag of a DHCP Fixed Address object.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

This flag only applies if the fixed address is associated with a Microsoft (r) server. If it is set to "true", the DHCP server will always update DNS, regardless of the requests from the DHCP clients. If this flag is set to false, clients will be allowed to update DNS when they request it.

Parameter

Specify "true" to set the always_update_dns flag or "false" to deactivate/unset it. The default value is "false".

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get always_update_dns
my $always_update_dns = $fixed_addr->always_update_dns();
#Modify always_update_dns
$fixed_addr->always_update_dns("false");
```

bootserver( )

Use this method to set or retrieve the bootserver of a DHCP Fixed Address object.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

This option is overridden independently from Infoblox::DHCP::FixedAddr->bootfile() and Infoblox::DHCP::FixedAddr->nextserver().

Parameter

The boot server IP address or name in FQDN (Fully Qualified Domain Name) format. The FQDN consists of the host name followed by the domain name (example: abc.com). A boot server name can have a maximum of 256 bytes. The default value is undefined.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get bootserver
my $bootserver = $fixed_addr->bootserver();
#Modify bootserver
$fixed_addr->bootserver("bootserver2.com");
#Un-override bootserver
$fixed_addr->bootserver(undef);
```

client_identifier_prepend_zero( )

Use this method prepend "\000" to the dhcp-client-identifier of a fixed address.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

When you set the client_identifier_prepend_zero parameter to "true", it prefixes a 0 (zero) to the DHCP client identifier whenever option 61 is used to assign fixed addresses. The following is an excerpt from "man dhcp-options":

Please be aware that some DHCP clients, when configured with client identifiers that are ASCII text, will prepend a zero to the ASCII text. So you may need to write:

```
option dhcp-client-identifier "\000foo"
```

rather than:

```
option dhcp-client-identifier "foo".
```

Returns

If you specified a parameter, the method returns true when the modification succeeds and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get client_identifier_prepend_zero
my $client_identifier_prepend_zero = $fixed_addr->client_identifier_prepend_zero();
#Modify dhcp_client_identifier
$fixed_addr->client_identifier_prepend_zero("true");
```

cli_credentials( )

Use this method to set or retrieve the discovery CLI credentials.

Setting this method to a defined value implicitly sets the override_cli_credentials method to "true". Setting the parameter to undefined causes the appliance to use the grid default and automatically resets the override_cli_credentials attribute to "false".

Note that when cli_credentials is set to a defined value and override_cli_credentials is set to "false", the last operation takes precedence. Thus the sequence $fixed_addr->cli_credentials([$cli]); $fixed_addr->override_cli_credentials("false"); will set override_cli_credentials to "false", and the sequence $fixed_addr->override_cli_credentials("false"); $fixed_addr->cli_credentials([$cli]); will result in override_cli_credentials="true".

Parameter

Valid value is an array reference that contains Infoblox::Grid::Discovery::CLICredential objects.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get cli_credentials
my $cli_credentials = $fixed_addr->cli_credentials();
#Modify cli_credentials
$fixed_addr->cli_credentials([$cli1, $cli2]);
```

comment( )

Use this method to set or retrieve the descriptive comment of a DHCP Fixed Address object.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

Desired comment in string format with a maximum of 256 bytes.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get comment
my $comment = $fixed_addr->comment();
#Modify comment
$fixed_addr->comment("Modified DHCP Fixed Address object comment");
```

configure_for_dhcp( )

Use this method to set or retrieve the configure_for_dhcp flag of a DHCP Fixed Address object.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

Specify "true" to set the configure_for_dhcp flag or "false" to deactivate/unset it. The default value is "false".

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get configure_for_dhcp
my $configure_for_dhcp = $fixed_addr->configure_for_dhcp();
#Modify configure_for_dhcp
$fixed_addr->configure_for_dhcp("true");
```

cloud_info( )

Use this method to retrieve cloud API related information for the Infoblox::DHCP::FixedAddr object.

Parameter

None

Returns

The method returns the attribute value.

Example

```
# Get cloud_info
my $cloud_info = $object->cloud_info();
```

ddns_domainname( )

Use this method to set or retrieve the DDNS domain name value of a DHCP Fixed Address object.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Setting this method to a defined value implicitly sets the override_ddns_domainname method to "true". Setting the parameter to undefined causes the appliance to use the grid default and automatically resets the override_ddns_domainname attribute to "false".

Note that when ddns_domainname is set to a defined value and override_dddns_domainname is set to "false", the last operation takes precedence. Thus the sequence $object->ddns_domainname("testdomain"); $object->override_ddns_domainname("false"); will set override_ddns_domainname to "false", and the sequence $object->override_ddns_domainname("false"); $object->ddns_domainname("testdomain"); will result in override_ddns_domainname="true".

Parameter

Specify the DDNS domain name.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get the DDNS domain name
my $ddns_domainname = $fixed_addr->ddns_domainname();
#Modify the DDNS domain name
$fixed_addr->ddns_domainname("ddns.domain.com");
```

ddns_hostname( )

Use this method to set or retrieve the DDNS host name value of a DHCP Fixed Address object.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

Specify the DDNS host name.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get the DDNS host name
my $ddns_hostname = $fixed_addr->ddns_hostname();
#Modify the DDNS host name
$fixed_addr->ddns_hostname("host.ddns.domain.com");
```

deny_bootp( )

Use this method to set or retrieve the deny_bootp flag of a DHCP Fixed Address object.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

Specify "true" to set the deny_bootp flag or "false" to deactivate/unset it. The default value is undefined which indicates that this attribute inherits the upper-level setting

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get deny_bootp
my $deny_bootp = $fixed_addr->deny_bootp();
#Modify deny_bootp
$fixed_addr->deny_bootp("true");
```

device_description( )

Use this method to set or retrieve the description of the discovery device.

Parameter

The device description in string format.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get device_description
my $device_description = $fixed_addr->device_description();
#Modify device_description
$fixed_addr->device_description('example');
```

device_location( )

Use this method to set or retrieve the location of the discovery device.

Parameter

The device location in string format.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get device_location
my $device_location = $fixed_addr->device_location();
#Modify device_location
$fixed_addr->device_location('example');
```

device_type( )

Use this method to set or retrieve the type of the discovery device.

Parameter

The device type in string format.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get device_type
my $device_type = $fixed_addr->device_type();
#Modify device_type
$fixed_addr->device_type('example');
```

device_vendor( )

Use this method to set or retrieve the vendor name of the discovery device.

Parameter

The device vendor name in string format.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get device_vendor
my $device_vendor = $fixed_addr->device_vendor();
#Modify device_vendor
$fixed_addr->device_vendor('example');
```

dhcp_client_identifier( )

Use this method to set or retrieve the dhcp-client-identifier value of a DHCP Fixed Address object.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

Use the dhcp_client_identifier or uid to assign fixed addresses if the DHCP clients send option 61. A DHCP client identifier can be a string or a MAC address. Set the "match_client" field to "CLIENT_IDENTIFIER" for fixed addresses to use the dhcp_client_identifier instead of the mac_address.

Returns

If you specified a parameter, the method returns true when the modification succeeds and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get dhcp_client_identifier
my $uid = $fixed_addr->dhcp_client_identifier();
#Modify dhcp_client_identifier
$fixed_addr->dhcp_client_identifier("foo");
or
$fixed_addr->dhcp_client_identifier("1:11::11::11::11::11::12");
```

disable( )

Use this method to set or retrieve the disable flag of a DHCP Fixed Address object.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

Specify "true" to set the disable flag or "false" to deactivate/unset it. The default value is "false".

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get disable
my $disable = $fixed_addr->disable();
#Modify disable
$fixed_addr->disable("true");
```

discover_now_status( )

Use this method to retrieve the current discovery status. This attribute is read-only.

'discover_now_status' will not be returned by default, use 'return_methods' to request it if needed.

Parameter

None.

Returns

The method returns the attribute value that can be 'COMPLETE', 'FAILED', 'NONE', 'PENDING' or 'RUNNING'.

Example

```
#Get discover_now_status
my $discover_now_status = $fixed_addr->discover_now_status();
```

discovered_data( )

Use this method to retrieve the discovered data of a DHCP Fixed Address object. This is a read-only attribute.

Omit the parameter to retrieve the attribute value.

Parameter

None

Returns

The valid return value is an Infoblox::Grid::Discovery::Data object.

Example

```
#Get discovered_data value
my $discovered_data = $object->discovered_data();
```

discovered_mac( )

Use this method to retrieve the discovered MAC address of a DHCP Fixed Address object.

Parameter

none

Returns

The method returns the discovered MAC address.

Example

```
#Get discovered_mac
my $discovered_mac = $fixed_addr->discovered_mac();
```

discovered_name( )

Use this method to retrieve the discovered name of a DHCP Fixed Address object.

Parameter

none

Returns

The method returns the discovered name.

Example

```
#Get discovered_name
my $discovered_name = $fixed_addr->discovered_name();
```

discoverer( )

Use this method to retrieve the discoverer of a DHCP Fixed Address object.

Parameter

none

Returns

The method returns the discoverer value.

Example

```
#Get discoverer
my $discoverer = $fixed_addr->discoverer();
```

enable_ddns( )

Use this method to set or retrieve the enable_ddns flag of a DHCP Fixed Address object.

If enable_ddns value is specified as "true", then the A and PTR records corresponding to the Fixed Address object will be placed on the DNS server.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

Specify "true" to set the enable_ddns flag or "false" to deactivate/unset it. The default value is "false".

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get enable_ddns
my $enable_ddns = $fixed_addr->enable_ddns();
#Modify enable_ddns
$fixed_addr->enable_ddns("true");
```

extattrs( )

Use this method to set or retrieve the extensible attributes associated with a DHCP Fixed Address object.

Parameter

Valid value is a hash reference containing the names of extensible attributes and their associated values (Infoblox::Grid::Extattr objects).

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get extattrs
my $ref_extattrs = $fixed_addr->extattrs();
#Modify extattrs
$fixed_addr->extattrs({ 'Site' => $extattr1, 'Administrator' => $extattr2 });
```

extensible_attributes( )

Use this method to set or retrieve the extensible attributes associated with a DHCP Fixed Address object

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

For valid values for extensible attributes, see Infoblox::Grid::ExtensibleAttributeDef/Extensible Attribute Values.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get extensible attributes
my $ref_extensible_attributes = $fixed_addr->extensible_attributes();
#Modify extensible attributes
$fixed_addr->extensible_attributes({ 'Site' => 'Santa Clara', 'Administrator' => [ 'Peter', 'Tom' ]
});
```

first_discovered( )

Use this method to retrieve the date and time that the DHCP Fixed Address object was first discovered.

Parameter

none

Returns

The method returns the date and time that the DHCP Fixed Address object was first discovered.

Example

```
#Get first_discovered
my $first_discovered = $fixed_addr->first_discovered();
```

ignore_dhcp_option_list_request( )

Use this method to set or retrieve the ignore_dhcp_option_list_request flag of a DHCP Fixed address object. If this flag is set to false all the defined DHCP options will be returned to the client.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

Specify "true" to set the ignore_dhcp_option_list_request flag or "false" to deactivate/unset it. The default value is false which indicates that this attribute inherit the member level setting..

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
# Get ignore_dhcp_option_list_request
my $ignore_dhcp_option_list_request = $fixed_addr->ignore_dhcp_option_list_request();
# Un-override ignore_dhcp_option_list_request
$fixed_addr->ignore_dhcp_option_list_request(undef);
# Modify ignore_dhcp_option_list_request
$fixed_addr->ignore_dhcp_option_list_request("true");
```

ipv4addr( )

Use this method to set or retrieve the ipv4addr value of a DHCP Fixed Address object.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

An IPv4 address is a 32-bit number in dotted decimal notation. It consists of four 8-bit groups of decimal digits separated by decimal points (example: 192.168.1.2).

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get ipv4addr
my $ipv4addr = $fixed_addr->ipv4addr();
#Modify ipv4addr
$fixed_addr->ipv4addr("10.0.0.5");
```

is_invalid_mac( )

Use this method to retrieve the flag used to indicate whether an invalid MAC address is used. This is a read-only attribute.

Parameter

None

Returns

The method returns the attribute value.

Example

```
#Get is_invalid_mac value
my $is_invalid_mac = $fixed_addr->is_invalid_mac();
```

last_discovered( )

Use this method to retrieve the time this object was last seen by a discovery job.

The default value for this field is empty.

Parameter

N/A

Returns

The method returns the attribute value.

Example

```
#Get last_discovered
my $last_discovered = $fixed_addr->last_discovered();
```

logic_filters( )

Use this method to set or retrieve all the logic (ordered) filters associated with the DHCP Fixed Address.

Filters are used to screen address assignments for the DHCP Fixed Address if the addresses are to be assigned to specific hosts.

Setting this method to a defined value implicitly sets the override_logic_filters method to "true". Setting the parameter to undefined causes the appliance to use the grid default and automatically resets the override_logic_filters attribute to "false".

Note that when logic_filters is set to a defined value and override_dlogic_filters is set to "false", the last operation takes precedence. Thus the sequence $object->logic_filters($filters); $object->override_logic_filters("false"); will set override_logic_filters to "false", and the sequence $object->override_logic_filters("false"); $object->logic_filters($filters); will result in override_logic_filters="true".

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

The valid value is an array reference that contains either the Infoblox::DHCP::Filter::MAC, Infoblox::DHCP::Filter::NAC, Infoblox::DHCP::Filter::Option objects or the names of the corresponding filters. The filters in the list must satisfy the following conditions:

Filter name must be globally unique.
Filters without expressions must be at the end of the list.
The filters should not be disabled.

The default value of the parameter is undefined.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get logic_filters
my $ref_logic_filters = $fixed_addr->logic_filters();
#Modify logic_filters
$fixed_addr->logic_filters(["filter_1","filter_2",$exl1]); #$exl1 is an Infoblox::DHCP::Filter::MAC,
Infoblox::DHCP::Filter::NAC or Infoblox::DHCP::Filter::Option object.
```

mac( )

```
Use this method to set or retrieve the MAC address value of a DHCP Fixed Address object.
```

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

MAC Address of the DHCP Fixed address object. Format for the hexadecimal strings for MAC addresses and vendor prefixes with colons or dashes. Both of the following formats are acceptable: 11:11:11:11:11:11 and 11-11-11-11-11-11.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get mac
my $mac = $fixed_addr->mac();
#Modify mac
$fixed_addr->mac("11::11::11::11::11::11::12");
```

match_client( )

Use this method to set or retrieve the match_client value of a DHCP Fixed Address object.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

```
    match_client "MAC": The fixed IP address is leased to the matching MAC address.

    match_client "CLIENT_IDENTIFIER": The fixed IP address is leased to the matching DHCP client
identifier

    match_client "RESERVED": The fixed IP address is reserved for later use with a MAC address that
only has zeros.

    match_client "CIRCUIT_ID": The fixed IP address is leased to the DHCP client with a matching
circuit ID. Note that
                  the "agent_circuit_id" field must be set in this case.

    match_client "REMOTE_ID": The fixed IP address is leased to the DHCP client with a matching
remote ID. Note that
                  the "agent_remote_id" field must be set in this case.
```

Returns

```
    If you specified a parameter, the method returns true when the modification succeeds, and returns
false when the operation fails.

    If you did not specify a parameter, the method returns the attribute value.
```

Example

```
#Get match_client
my $match_client = $fixed_addr->match_client();
#Modify match_client
$fixed_addr->match_client("CLIENT_IDENTIFIER");
```

member( )

Use this method to set or retrieve the Microsoft server associated with this fixed address.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

The valid value is a defined Infoblox::DHCP::MSServer object. The default value is empty.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get member
my $ref_member = $dhcp_fixedaddr->member();
#Modify member
$dhcp_fixedaddr->member($member1); #$member1 is an Infoblox::DHCP::MSServer object
```

ms_ad_user_data( )

Use this method to retrieve Microsoft Active Directory users related information. This is a read-only attribute.

Parameter

None

Returns

The valid return value is an Infoblox::Grid::MSServer::AdUser::Data object.

Example

```
#Get ms_ad_user_data
my $ms_ad_user_data = $dhcp_fixedaddr->ms_ad_user_data();
```

name( )

```
Use this method to set or retrieve the name of a DHCP Fixed Address object.
```

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

A string specifying the name of the fixed address.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get name
my $name = $fixed_addr->name();
#Modify name
$fixed_addr->name("fxname");
```

netbios( )

Use this method to retrieve the name in the NetBIOS reply that responded to a NetBIOS query.

Returns

The method returns the attribute value.

Example

```
#Get netbios
my $netbios = $fixed_addr->netbios();
```

ms_options( )

Use this method to set or retrieve the Microsoft server custom DHCP options associated with this DHCP fixed address.

Note: You can set only options that are specified in both the Microsoft server and NIOS option definitions. Options that are not common to both the Microsoft server and NIOS cannot be modified.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

Valid value is an array reference that contains defined Infoblox::DHCP::MSOption objects. The default value is empty.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get options
my $ref_options = $dhcp_fixedaddr->ms_options();
#Modify options
$dhcp_fixedaddr->ms_options([$option12]); #$option12 is an Infoblox::DHCP::MSOption object
```

network( )

Use this method to retrieve the network value of a DHCP Fixed Address object.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

An network is a 32-bit number in dotted decimal notation. It consists of four 8-bit groups of decimal digits separated by decimal points followed by prefix (example: 1.0.0.0/8).

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get network
my $network = $fixed_addr->network();
```

network_component_description( )

Use this method to retrieve the description of the network device that is connected to the DHCP Fixed Address object.

Parameter

none

Returns

The method returns the network device description.

Example

```
#Get network_component_description
my $network_component_description = $fixed_addr->network_component_description();
```

network_component_ip( )

Use this method to retrieve the IP address of the network device that is connected to the DHCP Fixed Address object.

Parameter

none

Returns

The method returns the IP address of the network device.

Example

```
#Get network_component_ip
my $network_component_ip = $fixed_addr->network_component_ip();
```

network_component_name( )

Use this method to retrieve the name of the network device that is connected to the DHCP Fixed Address object.

Parameter

none

Returns

The method returns the network device name.

Example

```
#Get network_component_name
my $network_component_name = $fixed_addr->network_component_name();
```

network_component_port_description( )

Use this method to retrieve the description of the network device port that is connected to the DHCP Fixed Address object.

Parameter

none

Returns

The method returns the description of the port on the network device.

Example

```
#Get network_component_port_description
my $network_component_port_description = $fixed_addr->network_component_port_description();
```

network_component_port_name( )

Use this method to retrieve the name of the network device port that is connected to the DHCP Fixed Address object.

Parameter

none

Returns

The method returns the name of the port on the network device.

Example

```
#Get network_component_port_name
my $network_component_port_name = $fixed_addr->network_component_port_name();
```

network_component_port_number( )

Use this method to retrieve the number of the network device port that is connected to the DHCP Fixed Address object.

Parameter

none

Returns

The method returns the network device port number.

Example

```
#Get network_component_port_number
my $network_component_port_number = $fixed_addr->network_component_port_number();
```

network_component_type( )

Use this method to retrieve the type of the network device that is connected to DHCP Fixed Address object.

Parameter

none

Returns

The method returns the network device type.

Example

```
#Get network_component_type
my $network_component_type = $fixed_addr->network_component_type();
```

network_view( )

Use this method to set or retrieve the network view of the DHCP fixed address.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

The default value is the "default" network view, which means the fixed address is in the default network view.

Parameter

Valid value is a defined Infoblox::DHCP::View object. The default network view value is the system-defined default network view object.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get network view
my $nview = $fixed_addr->network_view();
#Modify network view, with an Infoblox::DHCP::View object
$fixed_addr->network_view($nview);
```

nextserver( )

Use this method to set or retrieve the nextserver value of a DHCP Fixed Address object.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

This option is overridden independently from Infoblox::DHCP::FixedAddr->bootfile() and Infoblox::DHCP::FixedAddr->bootserver().

Parameter

The next server IP address or name in FQDN (Fully Qualified Domain Name) format. The FQDN consists of the host name followed by the domain name (example: abc.com). A next server name can have a maximum of 256 bytes. The default value is undefined.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get nextserver
my $nextserver = $fixed_addr->nextserver();
#Modify nextserver
$fixed_addr->nextserver("3.3.3.4");
#Un-override nextserver
$fixed_addr->nextserver(undef);
```

options( )

Use this method to set or retrieve the options of a DHCP Fixed Address object.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

See Infoblox::DHCP::Option for parameters and methods available for this object.

Parameter

Options describe network configuration settings and various services available on the network. These options occur as variable-length fields at the end of DHCP messages.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get options
my $options = $fixed_addr->options();
#Modify options
$fixed_addr->options([$option1, $option2]);
```

os( )

Use this method to retrieve the operating system associated with this fixed address.

Returns

The method returns the attribute value.

Example

```
#Get os
my $os = $fixed_addr->os();
```

override_ddns_domainname( )

The override_ddns_domainname attribute controls whether the ddns_domainname value in the DHCP Fixed Address is used, instead of the grid default.

The override_ddns_domainname attribute can be specified explicitly. It is also set implicitly when ddns_domainname is set to a defined value.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

Set the parameter to "true" to override the grid-level setting for ddns_domainname. Set the parameter to "false" to inherit the grid-level setting for ddns_domainname.

The default value of this parameter is "false".

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Getting override_ddns_domainname
my $override_ddns_domainname=$fixed_addr->override_ddns_domainname( );
#Modifying override_ddns_domainname
$fixed_addr->override_ddns_domainname("true");
```

override_cli_credentials( )

The override_cli_credentials attribute controls whether the cli_credentials value in the object is used, instead of the grid default.

The override_cli_credentials attribute can be specified explicitly. It is also set implicitly when cli_credentials is set to a defined value.

Parameter

Specify "true" to set the override_cli_credentials flag or "false" to deactivate/unset it. The default value is "false".

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
# Get attribute value
my $value = $fixed_addr->override_cli_credentials();
# Modify attribute value
$fixed_addr->override_cli_credentials('true');
```

override_logic_filters( )

The override_logic_filters attribute controls whether the logic_filters value of the DHCP Fixed Address is used, instead of the Grid default.

The override_logic_filters attribute can be specified explicitly. It is also set implicitly when logic_filters is set to a defined value.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

Set the parameter to "true" to override the Grid-level setting for logic_filters. Set the parameter to "false" to inherit the Grid-level setting for logic_filters.

The default value of this parameter is "false".

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Getting override_logic_filters
my $override_logic_filters=$fixed_addr->override_logic_filters( );
#Modifying override_logic_filters
$fixed_addr->override_logic_filters("true");
```

port_duplex( )

Use this method to retrieve the duplex setting of the network device port that is connected to the DHCP Fixed Address object.

Parameter

none

Returns

The method returns the port duplex setting. Returned values, if any, are one of the following:

```
Full
Half
```

Example

```
#Get port_duplex
my $port_duplex = $fixed_addr->port_duplex();
```

port_link_status( )

Use this method to retrieve the link status of the network device port that is connected to the DHCP Fixed Address object.

Parameter

none

Returns

The method returns the network device port link status. Returned values, if any, are one of the following:

```
Connected
Not Connected
Unknown
```

Example

```
#Get port_link_status
my $port_link_status = $fixed_addr->port_link_status();
```

port_speed( )

Use this method to retrieve the speed of the network device port that is connected to the DHCP Fixed Address object.

Parameter

none

Returns

The method returns the network device port speed. Returned values, if any, are one of the following:

```
10M
100M
1G
10G
100G
Unknown
```

Example

```
#Get port_speed
my $port_speed = $fixed_addr->port_speed();
```

port_status( )

Use this method to retrieve the status of the network device port that is connected to the DHCP Fixed Address object.

Parameter

none

Returns

The method returns the network device port status. Returned values, if any, are one of the following:

```
Up
Down
Unknown
```

Example

```
#Get port_status
my $port_status = $fixed_addr->port_status();
```

port_vlan_description( )

Use this method to retrieve the VLAN description of the network device port that is connected to the DHCP Fixed Address object.

Parameter

none

Returns

The method returns the VLAN description of the network device port.

Example

```
#Get port_vlan_description
my $port_vlan_description = $fixed_addr->port_vlan_description();
```

port_vlan_name( )

Use this method to retrieve the VLAN name of the network device port that is connected to the DHCP Fixed Address object.

Parameter

none

Returns

The method returns the network device port VLAN name.

Example

```
#Get port_vlan_name
my $port_vlan_name = $fixed_addr->port_vlan_name();
```

port_vlan_number( )

Use this method to retrieve the VLAN number of the network device port that is connected to the DHCP Fixed Address object.

Parameter

none

Returns

The method returns the port VLAN number.

Example

```
#Get port_vlan_number
my $port_vlan_number = $fixed_addr->port_vlan_number();
```

pxe_lease_time( )

Use this method to set or retrieve the pxe_lease_time value of a DHCP Fixed Address object.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

Enter appropriate values in the Days, Hours, Mins and Secs fields. User can specify the duration of time it takes a host to connect to a boot server, such as a TFTP server, and download the file it needs to boot.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get pxe_lease_time
my $pxe_lease_time = $fixed_addr->pxe_lease_time();
#Modify pxe_lease_time
$fixed_addr->pxe_lease_time("369");
```

reserved_interface( )

Use this method to set or retrieve the reserved discovery interface.

Parameter

Valid value is an Infoblox::Grid::Discovery::DeviceInterface object.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get reserved_interface
my $reserved_interface = $fixed_addr->reserved_interface();
#Modify reserved_interface
$fixed_addr->reserved_interface($interface);
```

template( )

Use this method to set or retrieve the fixed address template name for a DHCP Fixed Address object.

Template is the name of a DHCP Fixed Address template which will be used to create the DHCP Fixed Address.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

The valid name in sting format.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

The template parameter can be used to only add a new DHCP Fixed Address and cannot be used during object modify or returned during object get.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get template
my $template = $fixed_addr->template();
#Modify template
$fixed_addr->template("Template_Name");
```

v_cluster( )

Use this method to retrieve the name of the VMware cluster associated with the DHCP Fixed Address object. This method is read-only.

Parameter

none

Returns

This method returns a string that contains the VMware cluster name. The default value is an empty string.

Example

```
#Get v_cluster
my $v_cluster = $fixed_addr->v_cluster();
```

v_datacenter( )

Use this method to retrieve the name of the VMware datacenter associated with the DHCP Fixed Address object. This method is read-only.

Parameter

none

Returns

This method returns a string that contains the VMware datacenter name. The default value is an empty string.

Example

```
#Get v_datacenter
my $v_datacenter = $fixed_addr->v_datacenter();
```

v_host( )

Use this method to retrieve the name of the VMware host associated with the DHCP Fixed Address object. This method is read-only.

Parameter

none

Returns

This method returns a string that contains the VMware host name. The default value is an empty string.

Example

```
#Get v_host
my $v_host = $fixed_addr->v_host();
```

v_name( )

Use this method to retrieve the name of the VMware entity associated with the DHCP Fixed Address object. This method is read-only.

Parameter

none

Returns

This method returns a string that contains the VMware entity name. The default value is an empty string.

Example

```
#Get v_name
my $v_name = $fixed_addr->v_name();
```

v_netadapter( )

Use this method to retrieve the name of the physical network adapter through which the VMware entity is connected to the appliance. This method is read-only.

Parameter

none

Returns

This method returns a string that contains the network adapter name. The default value is an empty string.

Example

```
#Get v_netadapter
my $v_netadapter = $fixed_addr->v_netadapter();
```

v_switch( )

Use this method to retrieve the name of the virtual switch through which the VMware entity is connected to the appliance. This method is read-only.

Parameter

none

Returns

This method returns a string that contains the virtual switch name. The default value is an empty string.

Example

```
#Get v_switch
my $v_switch = $fixed_addr->v_switch();
```

v_type( )

Use this method to retrieve the type of VMware entity associated with the DHCP Fixed Address object. This method is read-only.

Parameter

none

Returns

This method returns a string that contains the VMware entity type. The default value is an empty string. The returned value, if any, can be one of the following:

```
VirtualMachine
HostSystem
VirtualCenter
```

Example

```
#Get v_type
my $v_type = $fixed_addr->v_type();
```

SAMPLE CODE

The following sample code demonstrates the different functions that can be applied to an object such as add, search, modify, and remove. This sample also includes error handling for the operations.

#Preparation prior to a DHCP Fixed Address object insertion

```
#PROGRAM STARTS: Include all the modules that will be used
```

```
use strict;
use Infoblox;

#Create a session to the Infoblox appliance

my $session = Infoblox::Session->new(
               master   => "192.168.1.2", #appliance host ip
               username => "admin",       #appliance user login
               password => "infoblox"     #appliance password
);

unless ($session) {
       die("Construct session failed: ",
               Infoblox::status_code() . ":" . Infoblox::status_detail());
}
print "Session created successfully\n";

#Create a network object
my $network = Infoblox::DHCP::Network->new(
                                          "network" => "10.0.0.0/8",
                                         );
unless($network) {
     die("Construct Network object failed: ",
           Infoblox::status_code() . ":" . Infoblox::status_detail());
}
print "Network object created successfully\n";

$session->add($network)
    or die("Add network failed: ",
           $session->status_code() . ":" . $session->status_detail());
print "Network added successfully\n";

my $option1 = Infoblox::DHCP::Option->new(
                                          "name"  => "lease-time",
                                          "value" => "50"
                                         );
unless($option1) {
     die("Construct option1 object failed: ",
           Infoblox::status_code() . ":" . Infoblox::status_detail());
}
print "option1 object created successfully\n";

my $option2 = Infoblox::DHCP::Option->new(
                                          "type"      => "routers",
                                          "ipv4addrs" => [ "10.0.0.1", "10.0.0.255" ],
                                         );
unless($option2) {
     die("Construct option2 object failed: ",
           Infoblox::status_code() . ":" . Infoblox::status_detail());
}
print "option2 object created successfully\n";
```

#Create a DHCP Fixed address object

```
#Create a DHCP Fixed Address object
my $fixed_addr = Infoblox::DHCP::FixedAddr->new(
    "ipv4addr"       => "10.0.0.6",
    "mac"            => "00:00:00:11:11:11",
    "network"        => "10.0.0.0/8",
    "comment"        => "add fixed address",
    "enable_ddns"    => "true",
    "pxe_lease_time" => "3600",
    "bootserver"     => "someserver.com",
    "nextserver"     => "3.3.3.3",
    "bootfile"       => "bootfile.com",
    "options"        => [ $option1 ,  $option2 ],
);
unless($fixed_addr) {
     die("Construct fixed address failed: ",
           Infoblox::status_code() . ":" . Infoblox::status_detail());
}
print "DHCP Fixed address object created successfully\n";

#Verify if the DHCP Fixed Address exists in the Infoblox appliance
my $object = $session->get(
       object => "Infoblox::DHCP::FixedAddr",
```

```
        mac     => "00:00:00:11:11:11"
);
unless ($object) {
   print "fixed address does not exist on server, safe to add the fixed address\n";
   $session->add($fixed_addr)
      or die("Add fixed address failed: ",
             $session->status_code() . ":" . $session->status_detail());
}
print "DHCP Fixed address added successfully\n";
```

#Create a DHCP Fixed Address object using a template

```
#Create a DHCP Fixed Address template object
my $fixed_addr_template = Infoblox::DHCP::FixedAddrTemplate->new(
    "name"                  => "custom_fa_template",
    "offset"                => "10",
    "number_of_addresses"   => "1",
    "comment"               => "add fixed address template",
    "enable_ddns"           => "TRUE",
);
unless($fixed_addr_template) {
     die("Construct fixed address failed: ",
           Infoblox::status_code() . ":" . Infoblox::status_detail());
}
print "DHCP Fixed address template object created successfully\n";

$session->add($fixed_addr_template)
      or die("Add fixed address template failed: ",
             $session->status_code() . ":" . $session->status_detail());
}
print "DHCP Fixed address template added successfully\n";

#Create a DHCP Fixed Address object
my $fixed_addr = Infoblox::DHCP::FixedAddr->new(
    "network"        => "10.0.0.0/8",
    "ipv4addr"       => "10.0.0.7",
    "match_client"   => "RESERVED",
    "template"       => "custom_fa_template"
);
unless($fixed_addr) {
     die("Construct fixed address failed: ",
           Infoblox::status_code() . ":" . Infoblox::status_detail());
}
print "DHCP Fixed address object created successfully\n";

$session->add($fixed_addr)
      or die("Add fixed address failed: ",
             $session->status_code() . ":" . $session->status_detail());
}
print "DHCP Fixed address added successfully\n";
```

#Search for a specific DHCP Fixed Address object

```
#Search for all DHCP Fixed address objects under the network 10.0.0.0/8
my @retrieved_objs = $session->search(
    object  => "Infoblox::DHCP::FixedAddr",
    mac     => "00:00:00:11:11.*",
    network => "10.0.0.0/8" );
my $object = $retrieved_objs[0];

unless ($object) {
       die("Search for Fixed Address object failed: ",
              $session->status_code() . ":" . $session->status_detail());
}
print "Search Fixed Address object found at least 1 matching entry\n";
```

#Get and modify a DHCP Fixed Address object

```
#Get DHCP Fixed Address object through the session
my @retrieved_objs = $session->get(
    object => "Infoblox::DHCP::FixedAddr",
    mac    => "00:00:00:11:11:11",
);
my $object = $retrieved_objs[0];
unless ($object) {
    die("Get Fixed Address object failed: ",
```

```
        $session->status_code() . ":" . $session->status_detail());
}
print "Get Fixed Address object found at least 1 matching entry\n";

#Modify one of the attributes of the obtained Fixed Address object
$object->pxe_lease_time("1234");

#Un-override bootfile
$object->bootfile(undef);

#Un-override bootserver
$object->bootserver(undef);

#Un-override nextserver
$object->nextserver(undef);

#Apply the changes
$session->modify($object)
    or die("Modify Fixed Address object failed: ",
           $session->status_code() . ":" . $session->status_detail());
print "Fixed Address object modified successfully \n";
```

#Remove a Fixed Address object

```
#Get Fixed Address object through the session
my @retrieved_objs = $session->get(
    object => "Infoblox::DHCP::FixedAddr",
    mac    => "00:00:00:11:11:11",
);
my $object = $retrieved_objs[0];
unless ($object) {
    die("Get Fixed Address object failed: ",
        $session->status_code() . ":" . $session->status_detail());
}
print "Get Fixed Address object found at least 1 matching entry\n";

#Submit the object for removal
$session->remove($object)
    or die("Remove Fixed Address object failed: ",
        $session->status_code() . ":" . $session->status_detail());
print "Fixed Address object removed successfully \n";

#Submit the Network object for removal
$session->remove($network)
    or die("Remove Network object failed: ",
        $session->status_code() . ":" . $session->status_detail());
print "Network object removed successfully \n";

#Submit the Fixed Address Template object for removal
$session->remove($fixed_addr_template)
    or die("Remove Fixed Address Template object failed: ",
        $session->status_code() . ":" . $session->status_detail());
print "Fixed Address Template object removed successfully \n";

####PROGRAM ENDS####
```

AUTHOR

Infoblox Inc. http://www.infoblox.com/

SEE ALSO

Infoblox::DHCP::Network, Infoblox::DHCP::View, Infoblox::DHCP::Range, Infoblox::DHCP::Lease, Infoblox::Session->add(), Infoblox::Session->get(), Infoblox::Session->modify(), Infoblox::Session->remove(), Infoblox::Session->search(), Infoblox::Session, Infoblox::DHCP::FixedAddrTemplate, Infoblox::Grid::Discovery::Data

---

COPYRIGHT

Copyright (c) 2017 Infoblox Inc.

Infoblox::DHCP::FixedAddrTemplate - DHCP Fixed Address template object.

- NAME
- DESCRIPTION
- CONSTRUCTOR
- MODULE METHODS
  - Infoblox::DHCP::NetworkTemplate-> fixed_address_templates( )
- SESSION METHODS
  - Infoblox::Session->add( )
  - Infoblox::Session->get( )
  - Infoblox::Session->search( )
  - Infoblox::Session->modify( )
  - Infoblox::Session->remove( )
- METHODS
  - bootfile( )
  - bootserver( )
  - comment( )
  - deny_bootp( )
  - enable_ddns( )
  - extattrs( )
  - extensible_attributes( )
  - ddns_domainname( )
  - ddns_hostname( )
  - ignore_dhcp_option_list_request( )
  - logic_filters( )
  - name( )
  - nextserver( )
  - number_of_addresses( )
  - offset( )
  - options( )
  - override_ddns_domainname( )
  - override_logic_filters( )
  - pxe_lease_time( )
- SAMPLE CODE
- AUTHOR
- SEE ALSO
- COPYRIGHT

NAME

Infoblox::DHCP::FixedAddrTemplate - DHCP Fixed Address template object.

DESCRIPTION

A fixed address template works as a 'cookie cutter' and can be used to create a real fixed address object in a quick and consistant way. A Fixed address object created from a fixed address template will inherit most properties defined in fixed address template object so most of the fixed address template properties are the same as the fixed address object properties.

CONSTRUCTOR

```
 my $fixed_addr_template = Infoblox::DHCP::FixedAddrTemplate -> new(
       name                              => $string,                        #Required
       number_of_addresses               => $num | undef,                   #Optional / Default is undefined
       offset                            => $num | undef,                   #Optional / Default is undefined
       bootfile                          => $filename | undef,              #Optional / Default is undefined
       bootserver                        => $ipv4addr | $fqdn | undef,      #Optional / Default is undefined
       comment                           => $string,                        #Optional / Default is empty
       deny_bootp                        => "true" | "false" | undef,       #Optional / Default is undefined
       enable_ddns                       => "true" | "false",               #Optional / Default is false
       ddns_domainname                   => $string | undef,                #Optional / Default is undefined
       ddns_hostname                     => $string | undef,                #Optional / Default is undefined
       extattrs                          => { $string => $extattr, ... },      #Optional / Default is
undefined
       extensible_attributes             => { $string => $string | $num, $string => [ $string | $num, ... ],
... } #Optional / Default is undefined

       ignore_dhcp_option_list_request   => "true" | "false" | undef,       #Optional / Default is false
       nextserver                        => $ipv4addr | $fqdn | undef,      #Optional / Default is undefined
       options                           => [$option1, $option2, ...]       #Optional / Default is empty
```

```
        override_ddns_domainname          => "true" | "false",            #Optional / Default value is
"false"
        pxe_lease_time                    => $time,                       #Optional / Default is empty
        logic_filters                     => [ $filter_name | $filter_obj, ...], # Optional / Default is
undefined
        override_logic_filters            => "true' | 'false",                  # Optional / Default value
is "false"
         );
```

You cannot set both extattrs and extensible_attributes attributes at the same time.

MODULE METHODS

The following functions are available to be applied to a DHCP fixed address template object.

Infoblox::DHCP::NetworkTemplate-> fixed_address_templates( )

Use this function to specify a fixed address template at network template on the Infoblox appliance. See Infoblox::DHCP::NetworkTemplate->fixed_address_templates() for parameters and return values.

Example

```
        #Create the Network Template object with fixed address template
         my $template = Infoblox::DHCP::Template->new(
           name                    => "custom_fa_template"
         );
         my $network_template = Infoblox::DHCP::NetworkTemplate->new(
           name                    => "custom_network_template",
           comment                 => "add network",
           fixed_address_templates => [$template]
         );
```

SESSION METHODS

This section describes all the methods in Infoblox::Session module that can be applied to a DHCP Fixed Address template object.

Infoblox::Session->add( )

Use this method to add a DHCP Fixed Address template object to the Infoblox appliance. See Infoblox::Session->add() for parameters and return values.

Example

```
        #Construct a DHCP Fixed Address template object
        my $fixed_addr_template = Infoblox::DHCP::FixedAddrTemplate->new(
               "name"                  => "custom_fa_template",
               "offset"                => "10",
               "number_of_addresses"   => "1",
               "bootfile"              => "bootfile",
               "bootserver"            => "bootserver.com",
               "comment"               => "add fixed address",
               "deny_bootp"            => "false",
               "enable_ddns"           => "false",
               "nextserver"            => "3.3.3.3",
               "options"               => [ $option1,  $option2 ],
               "pxe_lease_time"        => "3600",
        );

        #Submit for addition
        my $response = $session->add($fixed_addr_template);
```

Infoblox::Session->get( )

Use this method to retrieve all the matching objects from the Infoblox appliance. See Infoblox::Session->get() for parameters and return values.

Key References

```
        Apply the following attributes to get a specific DHCP Fixed Address template object:

        name - Optional. The name of the Fixed address template.
        extattrs     - Optional. A hash reference containing extensible attributes.
        extensible_attributes - Optional. A hash reference containing extensible attributes.
```

Examples

```
#Get DHCP Fixed Address template object through the session
my @retrieved_objs = $session->get(
    object => "Infoblox::DHCP::FixedAddrTemplate",
    name   => "custom_fa_template",
);

my @retrieved_objs = $session->get(
  object               => "Infoblox::DHCP::FixedAddrTemplate",
  extensible_attributes => { 'Site' => 'Santa Clara' }
);
```

Infoblox::Session->search( )

Use this method to retrieve all the matching objects from the Infoblox appliance. See Infoblox::Session->search() for parameters and return values.

Key References

```
Apply the following attributes to search for a DHCP Fixed Address template object:

name    - Optional. The name of the fixed address template (regular expression).
comment - Optional. The comment of the fixed address template (regular expression).
extattrs     - Optional. A hash reference containing extensible attributes.
extensible_attributes - Optional. A hash reference containing extensible attributes.
```

For more information about searching extensible attributes, see Infoblox::Grid::ExtensibleAttributeDef/Searching Extensible Attributes.

Examples

```
#Search DHCP Fixed Address template object through the session
my @retrieved_objs = $session->search(
    object  => "Infoblox::DHCP::FixedAddrTemplate",
    name    => "custom_fa_template",
    comment => ".*",
);

#Search for DHCP Fixed Address template objects defining "Santa Clara" for "Site" extensible attribute
my @retrieved_objs = $session->search(
    object               => "Infoblox::DHCP::FixedAddrTemplate",
    extensible_attributes => { 'Site' => 'Santa Clara' }
);
```

Infoblox::Session->modify( )

Use this method to modify a DHCP Fixed Address template object in the Infoblox appliance. See Infoblox::Session->modify() for parameters and return values.

Example

```
#Use method to modify the pxe_lease_time
$fixed_addr_templateess->pxe_lease_time("1234");
#Submit modification
my $response = $session->modify( $fixed_addr_templateess );
```

Infoblox::Session->remove( )

Use this method to remove a DHCP Fixed Address template object from the Infoblox appliance. See Infoblox::Session->remove() for parameters and return values.

To remove a specific object, first use `get()` to retrieve the specific DHCP Fixed Address template object, and then submit this object for removal.

Example

```
#Get DHCP Fixed Address template object through the session
my @retrieved_objs = $session->get(
    object => "Infoblox::DHCP::FixedAddrTemplate",
    name   => "custom_fa_template",
);
#Find the desired object from the retrieved list.
my $desired_fixed_addr_template = $retrieved_objs[0];
#Submit for removal
my $response = $session->remove( $desired_fixed_addr_template );
```

METHODS

This section describes all the methods that can be used to set and retrieve the attribute values of a DHCP Fixed Address template object.

bootfile( )

Use this method to set or retrieve the bootfile value of a DHCP Fixed Address template object.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

This option is overridden independently from Infoblox::DHCP::FixedAddrTemplate->bootserver() and Infoblox::DHCP::FixedAddrTemplate->nextserver().

Parameter

The name of the file that the client must download. The default value is undefined.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails. If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get bootfile
my $bootfile = $fixed_addr_template->bootfile();
#Modify bootfile
$fixed_addr_template->bootfile("boot_file");
#Un-override bootfile
$fixed_addr_template->bootfile(undef);
```

bootserver( )

Use this method to set or retrieve the bootserver of a DHCP Fixed Address template object.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

This option is overridden independently from Infoblox::DHCP::FixedAddrTemplate->bootfile() and Infoblox::DHCP::FixedAddrTemplate->nextserver().

Parameter

The boot server IP address or name in FQDN (Fully Qualified Domain Name) format. The FQDN consists of the host name followed by the domain name (example: abc.com). A boot server name can have a maximum of 256 bytes. The default value is undefined.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails. If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get bootserver
my $bootserver = $fixed_addr_template->bootserver();
#Modify bootserver
$fixed_addr_template->bootserver("bootserver2.com");
#Un-override bootserver
$fixed_addr_template->bootserver(undef);
```

comment( )

Use this method to set or retrieve the descriptive comment of a DHCP Fixed Address template object.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

Desired comment in string format with a maximum of 256 bytes.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails. If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get comment
my $comment = $fixed_addr_template->comment();
#Modify comment
$fixed_addr_template->comment("Modified DHCP Fixed Address template object comment");
```

deny_bootp( )

Use this method to set or retrieve the deny_bootp flag of a DHCP Fixed Address template object.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

Specify "true" to set the deny_bootp flag or "false" to deactivate/unset it. The default value is undefined which indicates that this attribute inherits the upper-level setting.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails. If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get deny_bootp
my $deny_bootp = $fixed_addr_template->deny_bootp();
#Modify deny_bootp
$fixed_addr_template->deny_bootp("true");
```

enable_ddns( )

Use this method to set or retrieve the enable_ddns flag of a DHCP Fixed Address template object. If enable_ddns value is specified as "true", then the A and PTR records corresponding to the Fixed Address template object will be placed on the DNS server.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

Specify "true" to set the enable_ddns flag or "false" to deactivate/unset it. The default value is "false".

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails. If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get enable_ddns
my $enable_ddns = $fixed_addr_template->enable_ddns();
#Modify enable_ddns
$fixed_addr_template->enable_ddns("true");
```

extattrs( )

Use this method to set or retrieve the extensible attributes associated with a DHCP Fixed Address template object.

Parameter

Valid value is a hash reference containing the names of extensible attributes and their associated values (Infoblox::Grid::Extattr objects).

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get extattrs
my $ref_extattrs = $fixed_addr_template->extattrs();
#Modify extattrs
$fixed_addr_template->extattrs({ 'Site' => $extattr1, 'Administrator' => $extattr2 });
```

extensible_attributes( )

Use this method to set or retrieve the extensible attributes associated with a DHCP Fixed Address template object

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

For valid values for extensible attributes, see Infoblox::Grid::ExtensibleAttributeDef/Extensible Attribute Values.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get extensible attributes
my $ref_extensible_attributes = $fixed_addr_template->extensible_attributes();
#Modify extensible attributes
$fixed_addr_template->extensible_attributes({ 'Site' => 'Santa Clara', 'Administrator' => [ 'Peter',
'Tom' ] });
```

ddns_domainname( )

Use this method to set or retrieve the ddns_domainname value of a DHCP Fixed Address template object.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Setting this method to a defined value implicitly sets the override_ddns_domainname method to "true". Setting the parameter to undefined causes the appliance to use the grid default and automatically resets the override_ddns_domainname attribute to "false".

Note that when ddns_domainname is set to a defined value and override_dddns_domainname is set to "false", the last operation takes precedence. Thus the sequence $object->ddns_domainname("testdomain"); $object->override_ddns_domainname("false"); will set override_ddns_domainname to "false", and the sequence $object->override_ddns_domainname("false"); $object->ddns_domainname("testdomain"); will result in override_ddns_domainname="true".

Parameter

Desired name for the dynamic DNS domain name in string format.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get ddns_domainname
my $ddns_domainname = $fixed_addr->ddns_domainname();
#Modify ddns_domainname
$fixed_addr->ddns_domainname("test_domain.com");
```

ddns_hostname( )

Use this method to set or retrieve the ddns_hostname value of a DHCP Fixed Address template object.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

Desired name for the dynamic DNS host name in string format.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get ddns_hostname
my $ddns_hostname = $fixed_addr->ddns_hostname();
#Modify ddns_hostname
$fixed_addr->ddns_hostname("test_host");
```

ignore_dhcp_option_list_request( )

Use this method to set or retrieve the ignore_dhcp_option_list_request flag of a DHCP fixed address teplate object. If this flag is set to false all the defined DHCP options will be returned to the client. This setting overrides the upper-level settings.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

Specify "true" to set the ignore_dhcp_option_list_request flag or "false" to deactivate it. The default value is false.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
# Get ignore_dhcp_option_list_request
my $ignore_dhcp_option_list_request = $fixed_addr_template->ignore_dhcp_option_list_request();
# Modify ignore_dhcp_option_list_request
$fixed_addr_template->ignore_dhcp_option_list_request("true");
# Un-override ignore_dhcp_option_list_request
$fixed_addr_template->ignore_dhcp_option_list_request(undef);
```

logic_filters( )

Use this method to set or retrieve all the logic (ordered) filters associated with the DHCP Fixed Address Template.

Filters are used to screen address assignments for the DHCP Fixed Address Template if the addresses are to be assigned to specific hosts.

Setting this method to a defined value implicitly sets the override_logic_filters method to "true". Setting the parameter to undefined causes the appliance to use the grid default and automatically resets the override_logic_filters attribute to "false".

Note that when logic_filters is set to a defined value and override_dlogic_filters is set to "false", the last operation takes precedence. Thus the sequence $object->logic_filters($filters); $object->override_logic_filters("false"); will set override_logic_filters to "false", and the sequence $object->override_logic_filters("false"); $object->logic_filters($filters); will result in override_logic_filters="true".

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

The valid value is an array reference that contains either the Infoblox::DHCP::Filter::MAC, Infoblox::DHCP::Filter::NAC, Infoblox::DHCP::Filter::Option objects or the names of the corresponding filters. The filters in the list must satisfy the following conditions:

Filter name must be globally unique.
Filters without expressions must be at the end of the list.
The filters should not be disabled.

The default value of the parameter is undefined.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get logic_filters
my $ref_logic_filters = $fixed_addr_template->logic_filters();
#Modify logic_filters
$fixed_addr_template->logic_filters(["filter_1","filter_2",$exl1]); #$exl1 is an
Infoblox::DHCP::Filter::MAC, Infoblox::DHCP::Filter::NAC or Infoblox::DHCP::Filter::Option object.
```

name( )

Use this method to set or retrieve the name of a DHCP Fixed Address template object.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

Name of the fixed address template in string format with a maximum of 64 characters.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.
If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get name
my $name = $fixed_address_template->name();
#Modify name
$fixed_address_template->name("custom_fa_template");
```

nextserver( )

Use this method to set or retrieve the nextserver value of a DHCP Fixed Address template object.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

This option is overridden independently from Infoblox::DHCP::FixedAddrTemplate->bootfile() and Infoblox::DHCP::FixedAddrTemplate->bootserver().

Parameter

The next server IP address or name in FQDN (Fully Qualified Domain Name) format. The FQDN consists of the host name followed by the domain name (example: abc.com). A next server name can have a maximum of 256 bytes. The default value is undefined.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails. If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get nextserver
my $nextserver = $fixed_addr_template->nextserver();
#Modify nextserver
$fixed_addr_template->nextserver("3.3.3.4");
#Un-override nextserver
$fixed_addr_template->nextserver(undef);
```

number_of_addresses( )

Use this method to set or retrieve the number of addresses in a DHCP Fixed Address template object.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

The number of addresses in this fixed address template.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails. If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get number_of_addresses
my $number_of_addresses = $fixed_address_template-> number_of_addresses();
#Modify number_of_addresses
$fixed_address_template->number_of_addresses("10");
```

offset( )

Use this method to set or retrieve the address offset of a Fixed Address template object.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

The address offset of this fixed address template. The offset is calculated from the start address of network template to which the fixed address template is assigned to. For example, if offset is set to 10 and number_of_addresses is set to 10, and network is set to 10.0.0.0/8, then there will be 10 fixed address object created and start address will be 10.0.0.10 and end address will be 10.0.0.20.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails. If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get offset
my $offset = $fixed_address_template->offset();
#Modify offset
$fixed_address_template->offset("10");
```

options( )

Use this method to set or retrieve the options of a DHCP Fixed Address template object.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

See Infoblox::DHCP::Option for parameters and methods available for this object.

Parameter

Options describe network configuration settings and various services available on the network. These options occur as variable-length fields at the end of DHCP messages.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails. If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get options
my $options = $fixed_addr_template->options();
#Modify options
$fixed_addr_template->options([$option1, $option2]);
```

override_ddns_domainname( )

The override_ddns_domainname attribute controls whether the ddns_domainname value in the DHCP Fixed Address is used, instead of the grid default.

The override_ddns_domainname attribute can be specified explicitly. It is also set implicitly when ddns_domainname is set to a defined value.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

Set the parameter to "true" to override the grid-level setting for ddns_domainname. Set the parameter to "false" to inherit the grid-level setting for ddns_domainname.

The default value of this parameter is "false".

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Getting override_ddns_domainname
my $override_ddns_domainname=$fixed_addr_template->override_ddns_domainname( );
#Modifying override_ddns_domainname
$fixed_addr_template->override_ddns_domainname("true");
```

override_logic_filters( )

The override_logic_filters attribute controls whether the logic_filters value of the DHCP Fixed Address Template is used, instead of the Grid default.

The override_logic_filters attribute can be specified explicitly. It is also set implicitly when logic_filters is set to a defined value.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

Set the parameter to "true" to override the Grid-level setting for logic_filters. Set the parameter to "false" to inherit the Grid-level setting for logic_filters.

The default value of this parameter is "false".

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Getting override_logic_filters
my $override_logic_filters=$fixed_addr_template->override_logic_filters( );
#Modifying override_logic_filters
$fixed_addr_template->override_logic_filters("true");
```

pxe_lease_time( )

Use this method to set or retrieve the pxe_lease_time value of a DHCP Fixed Address template object.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

Enter appropriate values in the Days, Hours, Mins and Secs fields. User can specify the duration of time it takes a host to connect to a boot server, such as a TFTP server, and download the file it needs to boot.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails. If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get pxe_lease_time
my $pxe_lease_time = $fixed_addr_template->pxe_lease_time();
#Modify pxe_lease_time
$fixed_addr_template->pxe_lease_time("369");
```

SAMPLE CODE

The following sample code demonstrates the different functions that can be applied to an object such as add, get, modify, and remove. Also, this sample includes error handling for the operations.

```
#PROGRAM STARTS: Include all the modules that will be used
use strict;
use Infoblox;

#Create a session to the Infoblox appliance

my $session = Infoblox::Session->new(
               master   => "192.168.1.2", #appliance host ip
               username => "admin",       #appliance user login
               password => "infoblox"     #appliance password
);

unless ($session) {
       die("Construct session failed: ",
               Infoblox::status_code() . ":" . Infoblox::status_detail());
}
print "Session created successfully\n";

my $option1 = Infoblox::DHCP::Option->new(
                                          "name"  => "fqdn",
                                          "value" => "mydomain.com"
                                         );
unless($option1) {
     die("Construct option1 object failed: ",
           Infoblox::status_code() . ":" . Infoblox::status_detail());
}
print "option1 object created successfully\n";

my $option2 = Infoblox::DHCP::Option->new(
                                          "name"     => "merit-dump",
                                           "value"   => "sometextdump",
                                         );
unless($option2) {
     die("Construct option2 object failed: ",
           Infoblox::status_code() . ":" . Infoblox::status_detail());
}
print "option2 object created successfully\n";
```

#Create a DHCP Fixed Address template object

```
#Create a DHCP Fixed Address template object
my $fixed_addr_template = Infoblox::DHCP::FixedAddrTemplate->new(
    "name"                  => "custom_fa_template",
    "offset"                => "10",
    "number_of_addresses"   => "1",
    "comment"               => "add fixed address template",
    "enable_ddns"           => "TRUE",
    "ddns_domainname"       => "test_domain.com",
```

```
    "ddns_hostname"          => "test_host",
    "pxe_lease_time"         => "3600",
    "bootserver"             => "someserver.com",
    "nextserver"             => "3.3.3.3",
    "bootfile"               => "bootfile.com",
    "options"                => [ $option1, $option2 ],
);
unless($fixed_addr_template) {
     die("Construct Fixed Address template failed: ",
           Infoblox::status_code() . ":" . Infoblox::status_detail());
}
print "DHCP Fixed Address template object created successfully\n";

#Verify if the DHCP Fixed Address templateexists in the Infoblox appliance
my $object = $session->get(
       object => "Infoblox::DHCP::FixedAddrTemplate",
       name   => "custom_fa_template"
);

unless ($object) {
   print "Fixed Address template does not exist on server, safe to add the fixed address template\n";
   $session->add($fixed_addr_template)
      or die("Add Fixed Address template failed: ",
             $session->status_code() . ":" . $session->status_detail());
}
print "DHCP Fixed Address template added successfully\n";
```

#Search a DHCP Fixed Address template object

```
#Search DHCP Fixed Address template object through the session
my @retrieved_objs = $session->search(
    object => "Infoblox::DHCP::FixedAddrTemplate",
    name   => "custom_fa_template",
);
my $object = $retrieved_objs[0];

unless ($object) {
    die("Search Fixed Address template object failed: ",
        $session->status_code() . ":" . $session->status_detail());
}
print "Search Fixed Address template object found at least 1 matching entry\n";
```

#Get and modify a DHCP Fixed Address template object

```
#Get DHCP Fixed Address template object through the session
my @retrieved_objs = $session->get(
    object => "Infoblox::DHCP::FixedAddrTemplate",
    name   => "custom_fa_template",
);
my $object = $retrieved_objs[0];

unless ($object) {
    die("Get Fixed Address template object failed: ",
        $session->status_code() . ":" . $session->status_detail());
}
print "Get Fixed Address template object found at least 1 matching entry\n";

#Modify one of the attributes of the obtained Fixed Address template object
$object->pxe_lease_time("1234");

#Un-override bootfile
$object->bootfile(undef);

#Un-override bootserver
$object->bootserver(undef);

#Un-override nextserver
$object->nextserver(undef);

#Apply the changes
$session->modify($object)
  or die("Modify Fixed Address template failed: ",
             $session->status_code() . ":" . $session->status_detail());

print "DHCP Fixed Address template added successfully\n";
```

#Create the Network Template object with fixed address template

```
my $template = Infoblox::DHCP::Template->new(
    name => "custom_fa_template",
);

my $network_template = Infoblox::DHCP::NetworkTemplate->new(
    name    => "custom_network_template",
    comment => "add network",
    fixed_address_templates => [$template]
);

unless($network_template) {
       die("Construct Network template object failed: ",
            Infoblox::status_code() . ":" . Infoblox::status_detail());
}

print "Network template object created successfully\n";

#Add the Network template object into the Infoblox appliance through a session
my $resp = $session->add($network_template)
or die("Add Network object failed: ",
            $session->status_code() . ":" . $session->status_detail());

print "Network template object added to Infoblox appliance successfully\n";
```

#Remove a Fixed Address template object

```
#Remove a Network Template object
$session->remove($network_template)
    or die("Remove Network Template template object failed: ",
        $session->status_code() . ":" . $session->status_detail());
print "Network Template object removed successfully \n";

#Get Fixed Address template object through the session
my @retrieved_objs = $session->get(
    object => "Infoblox::DHCP::FixedAddrTemplate",
    name   => "custom_fa_template",
);
my $object = $retrieved_objs[0];
unless ($object) {
    die("Get Fixed Address template object failed: ",
        $session->status_code() . ":" . $session->status_detail());
}
print "Get Fixed Address template object found at least 1 matching entry\n";

#Submit the object for removal
$session->remove($object)
    or die("Remove Fixed Address template object failed: ",
        $session->status_code() . ":" . $session->status_detail());
print "Fixed Address template object removed successfully \n";

####PROGRAM ENDS####
```

AUTHOR

Infoblox Inc. http://www.infoblox.com/

SEE ALSO

Infoblox::Session, Infoblox::Session->get(), Infoblox::Session->add(), Infoblox::Session->remove(), Infoblox::Session->modify(), Infoblox::DHCP::NetworkTemplate, Infoblox::DHCP::RangeTemplate,Infoblox::DHCP::Template,Infoblox::Session, Infoblox::DHCP::Option

COPYRIGHT

Copyright (c) 2017 Infoblox Inc.

Infoblox::DHCP::HostAddr - DHCP Host Address object.

- NAME
- DESCRIPTION
- CONSTRUCTOR
- SESSION METHODS
  - Infoblox::DNS::Host->ipv4addrs( )
  - Infoblox::Session->get( )
  - Infoblox::Session->modify( )
  - Infoblox::Session->search( )
- METHODS
  - bootfile( )
  - bootserver( )
  - configure_for_dhcp( )
  - deny_bootp( )
  - discovered_data( )
  - discovered_mac( )
  - discovered_name( )
  - discoverer( )
  - discover_now_status( )
  - enable_pxe_lease_time( )
  - first_discovered( )
  - host( )
  - ignore_client_requested_options( )
  - ipv4addr( )
  - is_invalid_mac( )
  - last_discovered( )
  - last_queried( )
  - logic_filters( )
  - mac( )
  - match_client( )
  - ms_ad_user_data( )
  - netbios( )
  - network( )
  - network_component_description( )
  - network_component_ip( )
  - network_component_name( )
  - network_component_port_description( )
  - network_component_port_name( )
  - network_component_port_number( )
  - network_component_type( )
  - nextserver( )
  - options( )
  - os( )
  - override_bootfile( )
  - override_bootserver( )
  - override_deny_bootp( )
  - override_ignore_client_requested_option( )
  - override_logic_filters( )
  - override_nextserver( )
  - override_options( )
  - override_pxe_lease_time( )
  - port_duplex( )
  - port_link_status( )
  - port_speed( )
  - port_status( )
  - port_vlan_description( )
  - port_vlan_name( )
  - port_vlan_number( )
  - pxe_lease_time( )
  - reserved_interface( )
  - v_cluster( )
  - v_datacenter( )
  - v_host( )
  - v_name( )
  - v_netadapter( )
  - v_switch( )
  - v_type( )
  - use_for_ea_inheritance( )
- SAMPLE CODE
- AUTHOR
- SEE ALSO

- COPYRIGHT

NAME

Infoblox::DHCP::HostAddr - DHCP Host Address object.

DESCRIPTION

A Host Address is an object used to specify addresses in a DNS Host object.

CONSTRUCTOR

```
my $host_addr = Infoblox::DHCP::HostAddr -> new(
      ipv4addr                          => $ipv4addr,                  #Required
      bootfile                          => $filename,                  #Optional / Default is undefined
      bootserver                        => $ipv4addr | $fqdn,          #Optional / Default is undefined
      configure_for_dhcp                => "true" | "false",           #Optional / Default is "false"
      deny_bootp                        => "true" | "false",           #Optional / Default is "false"
      dhcp_client_identifier            => $string,                    #Optional / Default is empty
      enable_pxe_lease_time             => "true" | "false",           #Optional / Default is "false"
      ignore_client_requested_options   => "true" | "false",           #Optional / Default is "false"
      mac                               => $mac_addr,                  #Optional / Default is empty
      match_client                      => $string,                    #Optional / Default is "MAC"
                                                                       #Possible values are:
                                                                       #
"MAC","CLIENT_IDENTIFIER","RESERVED",
                                                                       # "CIRCUIT_ID", "REMOTE_ID"
      nextserver                        => $ipv4addr | $fqdn | undef,  #Optional / Default is undefined
      options                           => [$option1, $option2, ...]   #Optional / Default is empty
      override_bootfile                 => "true" | "false",           #Optional / Default is "false"
      override_bootserver               => "true" | "false",           #Optional / Default is "false"
      override_deny_bootp               => "true" | "false",           #Optional / Default is "false"
      override_ignore_client_requested_options => "true" | "false",    #Optional / Default is "false"
      override_nextserver               => "true" | "false",           #Optional / Default is "false"
      override_options                  => "true" | "false",           #Optional / Default is "false"
      override_pxe_lease_time           => "true" | "false",           #Optional / Default is "false"
      pxe_lease_time                    => $time,                      #Optional / Default is empty
      use_for_ea_inheritance            => "true" | "false",           #Optional / Default is "false"
      reserved_interface                => $interface,                 #Optional
      logic_filters                     => [ $filter_name | $filter_obj, ...], # Optional / Default is
undefined
      override_logic_filters            => "true' | 'false",                  # Optional / Default value is
"false"
 );
```

SESSION METHODS

This section describes all the methods in Infoblox::Session module that you can apply to a DHCP Host Address object.

Infoblox::DNS::Host->ipv4addrs( )

Use this function to specify the addresses associated with the Host record. See Infoblox::DNS::Host->ipv4addrs() for parameters and return values.

Example

```
      #Construct a Host Address object
      my $h1 = Infoblox::DHCP::HostAddr->new(
                  ipv4addr  => "10.0.0.1"
      );

      # Configure an address in the Host record object
      $host->ipv4addrs([$h1]);
```

Infoblox::Session->get( )

Use this method to retrieve all the matching objects from the Infoblox appliance. See Infoblox::Session->get() for parameters and return values.

'discover_now_status' will not be returned by default, use 'return_methods' to request it if needed.

Key References

```
    Apply the following attributes to get a specific DHCP Host Address object:

     ipv4addr                - Optional. IPV4 address of the host address.
     mac                     - Optional. MAC address of the host address.
     host                    - Optional, the name of the host to which this address belongs.

     discovered_mac                          - Optional. Discovered MAC address.
     discovered_name                         - Optional. Discovered name.
     discoverer                              - Optional. Discoverer name.
     first_discovered                        - Optional. Time of the first discovery.
     last_discovered                         - Optional. Time of the last discovery.
     netbios                                 - Optional. Discovered NetBIOS name.
     network_component_description           - Optional. Discovered network component description.
     network_component_ip                    - Optional. Discovered network component IP address.
     network_component_name                  - Optional. Discovered network component name.
     network_component_port_description      - Optional. Discovered network component port description.
     network_component_port_name             - Optional. Discovered network component port name.
     network_component_port_number           - Optional. Discovered network component port number.
     network_component_type                  - Optional. Discovered network component type.
     os                                      - Optional. Discovered OS.
     port_duplex                             - Optional. Discovered port duplex settings.
     port_link_status                        - Optional. Discovered port link status.
     port_speed                              - Optional. Discovered port speed settings.
     port_status                             - Optional. Discovered port status settings.
     port_vlan_description                   - Optional. Discovered port VLAN description.
     port_vlan_name                          - Optional. Discovered port VLAN name.
     port_vlan_number                        - Optional. Discovered port VLAN number.
     v_cluster                               - Optional. Discovered VMware cluster name.
     v_datacenter                            - Optional. Discovered VMware datacenter name.
     v_host                                  - Optional. Discovered VMware server name.
     v_name                                  - Optional. Discovered VMware host or virtual machine name.
     v_netadapter                            - Optional. Discovered VMware physical network adapter name.
     v_switch                                - Optional. Discovered VMware virtual switch name.
     v_type                                  - Optional. Discovered VMware entity type (possible values:
"VirtualMachine", "HostSystem", "VirtualCenter").
```

Examples

```
#Get DHCP Host Address object through the session
my @retrieved_objs = $session->get(
    object => "Infoblox::DHCP::HostAddr",
    mac    => "11:11:11:11:11.11",
);

# get HostAddr by v_type:
my @retrieved_objs = $session->get(
   object    => 'Infoblox::DHCP::HostAddr',
   v_type    => 'VirtualMachine');
```

Infoblox::Session->modify( )

Use this method to modify a DHCP Host Address object in the Infoblox appliance. See Infoblox::Session->modify() for parameters and return values.

Example

```
#Use method to modify the pxe_lease_time
$host_address->pxe_lease_time("1234");
#Submit modification
my $response = $session->modify( $host_address );
```

Infoblox::Session->search( )

Use this method to search for DHCP Host Address object. See Infoblox::Session->search() for parameters and return values.

'discover_now_status' will not be returned by default, use 'return_methods' to request it if needed.

Key References

```
    Apply the following attributes to search for a specific DHCP Host Address object:

     ipv4addr                - Optional. IPV4 address of the host address (regular expression).
     mac                     - Optional. MAC address of the host address (regular expression).
     host                    - Optional, the name of the host to which this address belongs (regular
expression).

     discovered_mac                          - Optional. Discovered MAC address.
     discovered_name                         - Optional. Discovered name (regular expression).
     discoverer                              - Optional. Discoverer name (regular expression).
     first_discovered                        - Optional. Time of the first discovery.
     last_discovered                         - Optional. Time of the last discovery.
     netbios                                 - Optional. Discovered NetBIOS name (regular expression).
     network_component_description           - Optional. Discovered network component description (regular
expression).
     network_component_ip                    - Optional. Discovered network component IP address (regular
expression).
     network_component_name                  - Optional. Discovered network component name (regular
```

```
expression).
  network_component_port_description    - Optional. Discovered network component port description
(regular expression).
  network_component_port_name           - Optional. Discovered network component port name (regular
expression).
  network_component_port_number         - Optional. Discovered network component port number.
  network_component_type                - Optional. Discovered network component type (regular
expression).
  os                                    - Optional. Discovered OS (regular expression).
  port_duplex                           - Optional. Discovered port duplex settings.
  port_link_status                      - Optional. Discovered port link status.
  port_speed                            - Optional. Discovered port speed settings.
  port_status                           - Optional. Discovered port status settings.
  port_vlan_description                 - Optional. Discovered port VLAN description (regular
expression).
  port_vlan_name                        - Optional. Discovered port VLAN name (regular expression).
  port_vlan_number                      - Optional. Discovered port VLAN number.
  v_cluster                             - Optional. Discovered VMware cluster name (regular
expression).
  v_datacenter                          - Optional. Discovered VMware datacenter name (regular
expression).
  v_host                                - Optional. Discovered VMware server name (regular
expression).
  v_name                                - Optional. Discovered VMware host or virtual machine name
(regular expression).
  v_netadapter                          - Optional. Discovered VMware physical network adapter name
(regular expression).
  v_switch                              - Optional. Discovered VMware virtual switch name (regular
expression).
  v_type                                - Optional. Discovered VMware entity type (possible values:
"VirtualMachine", "HostSystem", "VirtualCenter").
```

Examples

```
# Search DHCP Host Address object through the session
my @retrieved_objs = $session->search(
    object       => "Infoblox::DHCP::HostAddr",
    mac          => "11:11:11:11:11.*",
);

# search HostAddr by v_datacenter:
my @retrieved_objs = $session->search(
   object                => 'Infoblox::DHCP::HostAddr',
   v_datacenter => 'test.*');
```

METHODS

This section describes all the methods that you can use to set and retrieve the attribute values of a DHCP Host Address object.

bootfile( )

Use this method to set or retrieve the bootfile value of a DHCP Host Address object.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Note that when bootfile is set to a defined value and override_bootfile is set to "false", the last operation takes precedence. Thus the sequence $object->bootfile("boot_file"); $object->override_bootfile("false"); will set override_bootfile to "false", and the sequence $object->override_bootfile("false"); $object->bootfile("boot_file"); will set override_bootfile to "true".

Parameter

The name of the file that the client must download. The default value is undefined.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get bootfile
my $bootfile = $host_addr->bootfile();
#Modify bootfile
$host_addr->bootfile("boot_file");
```

bootserver( )

Use this method to set or retrieve the bootserver of a DHCP Host Address object.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Note that when bootserver is set to a defined value and override_bootserver is set to "false", the last operation takes precedence. Thus

the sequence $object->bootserver("bootserver2.com"); $object->override_bootserver("false"); will set override_bootserver to "false", and the sequence $object->override_bootserver("false"); $object->bootserver("bootserver2.com"); will set override_bootserver to "true".

Parameter

The boot server IP address or FQDN (Fully Qualified Domain Name). The FQDN consists of the host name followed by the domain name (example: abc.com). A boot server name can have a maximum of 256 bytes. The default value is undefined.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get bootserver
my $bootserver = $host_addr->bootserver();
#Modify bootserver
$host_addr->bootserver("bootserver2.com");
#Un-override bootserver
$host_addr->bootserver(undef);
```

configure_for_dhcp( )

Use this method to set or retrieve the configure_for_dhcp flag of a DHCP Host Address object.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

Specify "true" to set the configure_for_dhcp flag or "false" to deactivate/unset it. The default value is "false".

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get configure_for_dhcp
my $configure_for_dhcp = $host_addr->configure_for_dhcp();
#Modify configure_for_dhcp
$host_addr->configure_for_dhcp("true");
```

deny_bootp( )

Use this method to set or retrieve the deny_bootp flag of a DHCP Host Address object.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Note that when deny_bootp is set to a defined value and override_deny_bootp is set to "false", the last operation takes precedence. Thus the sequence $object->deny_bootp("true"); $object->override_deny_bootp("false"); will set override_deny_bootp to "false", and the sequence $object->override_deny_bootp("false"); $object->deny_bootp("true"); will result in override_deny_bootp="true".

Parameter

Specify "true" to set the deny_bootp flag or "false" to deactivate/unset it. The default value is undefined, which indicates that this attribute inherits the upper-level setting.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get deny_bootp
my $deny_bootp = $host_addr->deny_bootp();
#Modify deny_bootp
$host_addr->deny_bootp("true");
```

discovered_data( )

Use this method to retrieve the discovered data of a DHCP Host Address object. This is a read-only attribute.

Omit the parameter to retrieve the attribute value.

Parameter

None

Returns

The valid return value is an Infoblox::Grid::Discovery::Data object.

Example

```
#Get discovered_data value
my $discovered_data = $object->discovered_data();
```

discovered_mac( )

Use this method to retrieve the discovered MAC address of a DHCP Host Address object.

Parameter

none

Returns

The method returns the discovered MAC address.

Example

```
#Get discovered_mac
my $discovered_mac = $host_addr->discovered_mac();
```

discovered_name( )

Use this method to retrieve the discovered name of a DHCP Host Address object.

Parameter

none

Returns

The method returns the discovered name.

Example

```
#Get discovered_name
my $discovered_name = $host_addr->discovered_name();
```

discoverer( )

Use this method to retrieve the discoverer of a DHCP Host Address object.

Parameter

none

Returns

The method returns the discoverer value.

Example

```
#Get discoverer
my $discoverer = $host_addr->discoverer();
```

discover_now_status( )

Use this method to retrieve the current discovery status. This attribute is read-only.

'discover_now_status' will not be returned by default, use 'return_methods' to request it if needed.

Parameter

None.

Returns

The method returns the attribute value that can be 'COMPLETE', 'FAILED', 'NONE', 'PENDING' or 'RUNNING'.

Example

```
#Get discover_now_status
my $discover_now_status = $host_addr->discover_now_status();
```

enable_pxe_lease_time( )

Use this method to set or retrieve the enable_pxe_lease_time flag of a DHCP Host Address object.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

Specify "true" to set the enable_pxe_lease_time flag or "false" to deactivate/unset it. The default value is "false".

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get enable_pxe_lease_time
my $enable_pxe_lease_time = $host_addr->enable_pxe_lease_time();
#Modify enable_pxe_lease_time
$host_addr->enable_pxe_lease_time("true");
```

first_discovered( )

Use this method to retrieve the date and time that the DHCP Host Address object was first discovered.

Parameter

none

Returns

The method returns the date and time that the DHCP Host Address object was first discovered.

Example

```
#Get first_discovered
my $first_discovered = $host_addr->first_discovered();
```

host( )

Use this method to retrieve the name of the Host record this address belongs to.

Parameter

none

Returns

The method returns the host value.

Example

```
#Get host
my $host = $host_addr->host();
```

ignore_client_requested_options( )

Use this method to set or retrieve the ignore_client_requested_option flag of a DHCP Host address object. If this flag is set to false, all the defined DHCP options are returned to the client.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Note that when ignore_client_requested_options is set to a defined value and override_ignore_client_requested_options is set to "false", the last operation takes precedence. Thus the sequence $object->ignore_client_requested_options("true"); $object->override_ignore_client_requested_options("false"); will set override_ignore_client_requested_options to "false", and the sequence $object->override_ignore_client_requested_options("false"); $object->ignore_client_requested_options("true"); will set override_ignore_client_requested_options to "true".

Parameter

Specify "true" to set the ignore_client_requested_option flag or "false" to deactivate/unset it. The default value is false, which

indicates that this attribute inherits the member-level setting.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
# Get ignore_client_requested_option
my $ignore_client_requested_option = $host_addr->ignore_client_requested_options();
# Un-override ignore_client_requested_option
$host_addr->ignore_client_requested_options(undef);
# Modify ignore_client_requested_option
$host_addr->ignore_client_requested_options("true");
```

ipv4addr( )

Use this method to set or retrieve the ipv4addr value of a DHCP Host Address object.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

An IPv4 address is a 32-bit number in dotted decimal notation. It consists of four 8-bit groups of decimal digits separated by decimal points (example: 192.168.1.2).

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get ipv4addr
my $ipv4addr = $host_addr->ipv4addr();
#Modify ipv4addr
$host_addr->ipv4addr("10.0.0.5");
```

is_invalid_mac( )

Use this method to retrieve the flag used to indicate whether an invalid MAC address is used. This is a read-only attribute.

Parameter

None

Returns

The method returns the attribute value.

Example

```
#Get is_invalid_mac value
my $is_invalid_mac = $host_addr->is_invalid_mac();
```

last_discovered( )

Use this method to retrieve the time this object was last seen by a discovery job.

The default value for this field is empty.

Parameter

N/A

Returns

The method returns the attribute value.

Example

```
#Get last_discovered
my $last_discovered = $host_addr->last_discovered();
```

last_queried( )

Use this method to retrieve the time when the associated record was last queried. This is a read-only attribute.

Parameter

None

Returns

The method returns the attribute value. The number of seconds that have elapsed since January 1st, 1970 UTC.

Example

```
#Get last_queried
my $last_queried = $host_addr->last_queried();
```

logic_filters( )

Use this method to set or retrieve all the logic (ordered) filters associated with the DHCP Host Address.

Filters are used to screen address assignments for the DHCP Host Address if the addresses are to be assigned to specific hosts.

Setting this method to a defined value implicitly sets the override_logic_filters method to "true". Setting the parameter to undefined causes the appliance to use the grid default and automatically resets the override_logic_filters attribute to "false".

Note that when logic_filters is set to a defined value and override_dlogic_filters is set to "false", the last operation takes precedence. Thus the sequence $object->logic_filters($filters); $object->override_logic_filters("false"); will set override_logic_filters to "false", and the sequence $object->override_logic_filters("false"); $object->logic_filters($filters); will result in override_logic_filters="true".

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

The valid value is an array reference that contains either the Infoblox::DHCP::Filter::MAC, Infoblox::DHCP::Filter::NAC, Infoblox::DHCP::Filter::Option objects or the names of the corresponding filters. The filters in the list must satisfy the following conditions:

Filter name must be globally unique.
Filters without expressions must be at the end of the list.
The filters should not be disabled.

The default value of the parameter is undefined.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get logic_filters
my $ref_logic_filters = $host_addr->logic_filters();
#Modify logic_filters
$host_addr->logic_filters(["filter_1","filter_2",$exl1]); #$exl1 is an Infoblox::DHCP::Filter::MAC,
Infoblox::DHCP::Filter::NAC or Infoblox::DHCP::Filter::Option object.
```

mac( )

Use this method to set or retrieve the MAC address value of a DHCP Host Address object.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

MAC Address of the DHCP Host address object. The format for MAC addresses and vendor prefixes are hexadecimal strings separated by colons or dashes. The following formats are acceptable: 11:11:11:11:11:11 and 11-11-11-11-11-11.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get mac
my $mac = $host_addr->mac();
#Modify mac
$host_addr->mac("11::11::11::11::11::12");
```

match_client( )

Use this method to set or retrieve the match_client value of a DHCP Host Address object.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

```
    match_client "MAC": The fixed IP address is leased to the matching MAC address.

    match_client "CLIENT_IDENTIFIER": The fixed IP address is leased to the matching DHCP client
identifier

    match_client "RESERVED": The fixed IP address is reserved for later use with a MAC address that
only has zeros.

    match_client "CIRCUIT_ID": The fixed IP address is leased to the DHCP client with a matching
circuit ID. Note that
                  the "agent_circuit_id" field must be set in this case.

    match_client "REMOTE_ID": The fixed IP address is leased to the DHCP client with a matching
remote ID. Note that
                  the "agent_remote_id" field must be set in this case.
```

Returns

```
    If you specified a parameter, the method returns true when the modification succeeds, and returns
false when the operation fails.

    If you did not specify a parameter, the method returns the attribute value.
```

Example

```
#Get match_client
my $match_client = $host_addr->match_client();
#Modify match_client
$host_addr->match_client("CLIENT_IDENTIFIER");
```

ms_ad_user_data( )

Use this method to retrieve Microsoft Active Directory users related information. This is a read-only attribute.

Parameter

None

Returns

The valid return value is an Infoblox::Grid::MSServer::AdUser::Data object.

Example

```
#Get ms_ad_user_data value
my $value = $object->ms_ad_user_data();
```

netbios( )

Use this method to retrieve the name in the NetBIOS reply that responded to a NetBIOS query.

Returns

The method returns the attribute value.

Example

```
#Get netbios
my $netbios = $host_addr->netbios();
```

network( )

Use this method to retrieve the network value of a DHCP Host Address object. This is a read-only attribute.

Omit the parameter to retrieve the attribute value.

Parameter

None

Returns

The method returns the attribute value.

Example

```
#Get network
my $network = $host_addr->network();
```

network_component_description( )

Use this method to retrieve the description of the network device that is connected to the DHCP Host Address object.

Parameter

none

Returns

The method returns the network device description.

Example

```
#Get network_component_description
my $network_component_description = $host_addr->network_component_description();
```

network_component_ip( )

Use this method to retrieve the IP address of the network device that is connected to the DHCP Host Address object.

Parameter

none

Returns

The method returns the IP address of the network device.

Example

```
#Get network_component_ip
my $network_component_ip = $host_addr->network_component_ip();
```

network_component_name( )

Use this method to retrieve the name of the network device that is connected to the DHCP Host Address object.

Parameter

none

Returns

The method returns the network device name.

Example

```
#Get network_component_name
my $network_component_name = $host_addr->network_component_name();
```

network_component_port_description( )

Use this method to retrieve the description of the network device port that is connected to the DHCP Host Address object.

Parameter

none

Returns

The method returns the description of the port on the network device.

Example

```
#Get network_component_port_description
my $network_component_port_description = $host_addr->network_component_port_description();
```

network_component_port_name( )

Use this method to retrieve the name of the network device port that is connected to the DHCP Host Address object.

Parameter

none

Returns

The method returns the name of the port on the network device.

Example

```
#Get network_component_port_name
my $network_component_port_name = $host_addr->network_component_port_name();
```

network_component_port_number( )

Use this method to retrieve the number of the network device port that is connected to the DHCP Host Address object.

Parameter

none

Returns

The method returns the network device port number.

Example

```
#Get network_component_port_number
my $network_component_port_number = $host_addr->network_component_port_number();
```

network_component_type( )

Use this method to retrieve the type of the network device that is connected to DHCP Host Address object.

Parameter

none

Returns

The method returns the network device type.

Example

```
#Get network_component_type
my $network_component_type = $host_addr->network_component_type();
```

nextserver( )

Use this method to set or retrieve the nextserver value of a DHCP Host Address object.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Note that when nextserver is set to a defined value and override_nextserver is set to "false", the last operation takes precedence. Thus the sequence $object->nextserver("3.3.3.4"); $object->override_nextserver("false"); will set override_nextserver to "false", and the sequence $object->override_nextserver("false"); $object->nextserver("3.3.3.4"); will set override_nextserver to "true".

Parameter

The next server IP address or FQDN (Fully Qualified Domain Name). The FQDN consists of the host name followed by the domain name (example: abc.com). A next server name can have a maximum of 256 bytes. The default value is undefined.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get nextserver
my $nextserver = $host_addr->nextserver();
#Modify nextserver
$host_addr->nextserver("3.3.3.4");
#Un-override nextserver
$host_addr->nextserver(undef);
```

options( )

Use this method to set or retrieve the options of a DHCP Host Address object.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

See Infoblox::DHCP::Option for parameters and methods available for this object.

Note that when options is set to a defined value and override_options is set to "false", the last operation takes precedence. Thus the sequence $object->options([$o1,$o2]); $object->override_options("false"); will set override_options to "false", and the sequence $object->override_options("false"); $object->options([$o1,$o2]); will set override_options to "true".

Parameter

Options describe network configuration settings and various services available on the network. These options occur as variable-length fields at the end of DHCP messages.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get options
my $options = $host_addr->options();
#Modify options
$host_addr->options([$option1, $option2]);
```

os( )

Use this method to retrieve the operating system associated with this host address.

Returns

The method returns the attribute value.

Example

```
#Get os
my $os = $host_addr->os();
```

override_bootfile( )

The override_bootfile attribute controls whether the bootfile value in the member is used, instead of the grid default.

The override_bootfile attribute can be specified explicitly. It is also set implicitly when bootfile is set to a defined value.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

Specify "true" to set the override_bootfile flag or "false" to deactivate/unset it. The default value is "false".

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get override_bootfile
my $override = $host_addr->override_bootfile();
#Modify the override
$host_addr->override_bootfile("true");
```

override_bootserver( )

The override_bootserver attribute controls whether the bootserver value in the member is used, instead of the grid default.

The override_bootserver attribute can be specified explicitly. It is also set implicitly when bootserver is set to a defined value.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

Specify "true" to set the override_bootserver flag or "false" to deactivate/unset it. The default value is "false".

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get override_bootserver
my $override = $host_addr->override_bootserver();
#Modify the override
$host_addr->override_bootserver("true");
```

override_deny_bootp( )

The override_deny_bootp attribute controls whether the deny_bootp value in the member is used, instead of the grid default.

The override_deny_bootp attribute can be specified explicitly. It is also set implicitly when deny_bootp is set to a defined value.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

Specify "true" to set the override_deny_bootp flag or "false" to deactivate/unset it. The default value is "false".

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get override_deny_bootp
my $override = $host_addr->override_deny_bootp();
#Modify the override
$host_addr->override_deny_bootp("true");
```

override_ignore_client_requested_option( )

The override_ignore_client_requested_option attribute controls whether the ignore_client_requested_option value in the member is used, instead of the grid default.

The override_ignore_client_requested_option attribute can be specified explicitly. It is also set implicitly when ignore_client_requested_option is set to a defined value.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

Specify "true" to set the override_ignore_client_requested_option flag or "false" to deactivate/unset it. The default value is "false".

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get override_ignore_client_requested_option
my $override = $host_addr->override_ignore_client_requested_option();
#Modify the override
$host_addr->override_ignore_client_requested_option("true");
```

override_logic_filters( )

The override_logic_filters attribute controls whether the logic_filters value of the DHCP Host Address is used, instead of the Grid default.

The override_logic_filters attribute can be specified explicitly. It is also set implicitly when logic_filters is set to a defined value.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

Set the parameter to "true" to override the Grid-level setting for logic_filters. Set the parameter to "false" to inherit the Grid-level setting for logic_filters.

The default value of this parameter is "false".

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Getting override_logic_filters
my $override_logic_filters=$host_addr->override_logic_filters( );
#Modifying override_logic_filters
$host_addr->override_logic_filters("true");
```

override_nextserver( )

The override_nextserver attribute controls whether the nextserver value in the member is used, instead of the grid default.

The override_nextserver attribute can be specified explicitly. It is also set implicitly when nextserver is set to a defined value.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

Specify "true" to set the override_nextserver flag or "false" to deactivate/unset it. The default value is "false".

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get override_nextserver
my $override = $host_addr->override_nextserver();
#Modify the override
$host_addr->override_nextserver("true");
```

override_options( )

The override_options attribute controls whether the options value in the member is used, instead of the upper-level default.

The override_options attribute can be specified explicitly. It is also set implicitly when options is set to a defined value.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

Specify "true" to set the override_options flag or "false" to deactivate/unset it. The default value is "false".

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get override_options
my $override = $host_addr->override_options();
#Modify the override
$host_addr->override_options("true");
```

override_pxe_lease_time( )

The override_pxe_lease_time attribute controls whether the pxe_lease_time value in the member is used, instead of the grid default.

The override_pxe_lease_time attribute can be specified explicitly. It is also set implicitly when pxe_lease_time is set to a defined value.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

Specify "true" to set the override_pxe_lease_time flag or "false" to deactivate/unset it. The default value is "false".

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get override_pxe_lease_time
my $override = $host_addr->override_pxe_lease_time();
#Modify the override
$host_addr->override_pxe_lease_time("true");
```

port_duplex( )

Use this method to retrieve the duplex setting of the network device port that is connected to the DHCP Host Address object.

Parameter

none

Returns

The method returns the port duplex setting. The returned value, if any, is one of the following:

```
Full
Half
```

Example

```
#Get port_duplex
my $port_duplex = $host_addr->port_duplex();
```

port_link_status( )

Use this method to retrieve the link status of the network device port that is connected to the DHCP Host Address object.

Parameter

none

Returns

The method returns the network device port link status. The returned value, if any, is one of the following:

```
Connected
Not Connected
Unknown
```

Example

```
#Get port_link_status
my $port_link_status = $host_addr->port_link_status();
```

port_speed( )

Use this method to retrieve the speed of the network device port that is connected to the DHCP Host Address object.

Parameter

none

Returns

The method returns the network device port speed. The returned value, if any, is one of the following:

```
10M
100M
1G
10G
100G
Unknown
```

Example

```
#Get port_speed
my $port_speed = $host_addr->port_speed();
```

port_status( )

Use this method to retrieve the status of the network device port that is connected to the DHCP Host Address object.

Parameter

none

Returns

The method returns the network device port status. The returned value, if any, is one of the following:

```
Up
Down
Unknown
```

Example

```
#Get port_status
my $port_status = $host_addr->port_status();
```

port_vlan_description( )

Use this method to retrieve the VLAN description of the network device port that is connected to the DHCP Host Address object.

Parameter

none

Returns

The method returns the VLAN description of the network device port.

Example

```
#Get port_vlan_description
my $port_vlan_description = $host_addr->port_vlan_description();
```

port_vlan_name( )

Use this method to retrieve the VLAN name of the network device port that is connected to the DHCP Host Address object.

Parameter

none

Returns

The method returns the network device port VLAN name.

Example

```
#Get port_vlan_name
my $port_vlan_name = $host_addr->port_vlan_name();
```

port_vlan_number( )

Use this method to retrieve the VLAN number of the network device port that is connected to the DHCP Host Address object.

Parameter

none

Returns

The method returns the port VLAN number.

Example

```
#Get port_vlan_number
my $port_vlan_number = $host_addr->port_vlan_number();
```

pxe_lease_time( )

Use this method to set or retrieve the pxe_lease_time value of a DHCP Host Address object.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Note that when pxe_lease_time is set to a defined value and override_pxe_lease_time is set to "false", the last operation takes precedence. Thus the sequence $object->pxe_lease_time("369"); $object->override_pxe_lease_time("false"); will set override_pxe_lease_time to "false", and the sequence $object->override_pxe_lease_time("false"); $object->pxe_lease_time("369"); will set override_pxe_lease_time to "true".

Parameter

Enter appropriate values in the Days, Hours, Mins and Secs fields. User can specify the duration of time it takes a host to connect to a boot server, such as a TFTP server, and download the file it needs to boot.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get pxe_lease_time
my $pxe_lease_time = $host_addr->pxe_lease_time();
#Modify pxe_lease_time
$host_addr->pxe_lease_time("369");
```

reserved_interface( )

Use this method to set or retrieve the reserved discovery interface.

Parameter

Valid value is an Infoblox::Grid::Discovery::DeviceInterface object.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get reserved_interface
my $reserved_interface = $host_addr->reserved_interface();
#Modify reserved_interface
$host_addr->reserved_interface($interface);
```

v_cluster( )

Use this method to retrieve the name of the VMware cluster associated with the DHCP Host Address object. This method is read-only.

Parameter

none

Returns

This method returns a string that contains the VMware cluster name. The default value is an empty string.

Example

```
#Get v_cluster
my $v_cluster = $host_addr->v_cluster();
```

v_datacenter( )

Use this method to retrieve the name of the VMware datacenter associated with the DHCP Host Address object. This method is read-only.

Parameter

none

Returns

This method returns a string that contains the VMware datacenter name. The default value is an empty string.

Example

```
#Get v_datacenter
my $v_datacenter = $host_addr->v_datacenter();
```

v_host( )

Use this method to retrieve the name of the VMware host associated with the DHCP Host Address object. This method is read-only.

Parameter

none

Returns

This method returns a string that contains the VMware host name. The default value is an empty string.

Example

```
#Get v_host
my $v_host = $host_addr->v_host();
```

v_name( )

Use this method to retrieve the name of the VMware entity associated with the DHCP Host Address object. This method is read-only.

Parameter

none

Returns

This method returns a string that contains the VMware entity name. The default value is an empty string.

Example

```
#Get v_name
my $v_name = $host_addr->v_name();
```

v_netadapter( )

Use this method to retrieve the name of the physical network adapter through which the VMware entity is connected to the appliance. This method is read-only.

Parameter

none

Returns

This method returns a string that contains the network adapter name. The default value is an empty string.

Example

```
#Get v_netadapter
my $v_netadapter = $host_addr->v_netadapter();
```

v_switch( )

Use this method to retrieve the name of the virtual switch through which the VMware entity is connected to the appliance. This method is read-only.

Parameter

none

Returns

This method returns a string that contains the virtual switch name. The default value is an empty string.

Example

```
#Get v_switch
my $v_switch = $host_addr->v_switch();
```

v_type( )

Use this method to retrieve the type of VMware entity associated with the DHCP Host Address object. This method is read-only.

Parameter

none

Returns

This method returns a string that contains the VMware entity type. The default value is an empty string. The returned value, if any, can be one of the following:

```
VirtualMachine
```

```
HostSystem
VirtualCenter
```

Example

```
#Get v_type
my $v_type = $host_addr->v_type();
```

use_for_ea_inheritance( )

Use this method to set or retrieve the flag that indicates if this host address is used for inheritable extensible attributes.

Parameter

Specify "true" to enable the flag or "false" to disable it. The default value is "false".

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get use_for_ea_inheritance
my $use_for_ea_inheritance = $host_addr->use_for_ea_inheritance();
#Modify use_for_ea_inheritance
$host_addr->use_for_ea_inheritance("true");
```

SAMPLE CODE

The following sample code demonstrates the different functions that can be applied to an object such as search and modify. This sample also includes error handling for the operations.

#Preparation prior to a DHCP IPv4 Host Address object insertion

```
#PROGRAM STARTS: Include all the modules that will be used, note the hostaddress option
use strict;
use Infoblox qw (:hostaddress);

#Create a session to the Infoblox appliance

my $session = Infoblox::Session->new(
              master   => "192.168.1.2", #appliance host ip
              username => "admin",       #appliance user login
              password => "infoblox"     #appliance password
);

unless ($session) {
       die("Construct session failed: ",
              Infoblox::status_code() . ":" . Infoblox::status_detail());
}
print "Session created successfully\n";

#Create an IPv4 network object
my $network = Infoblox::DHCP::Network->new(
                                          "network" => "10.0.0.0/8",
                                         );
unless($network) {
     die("Construct IPv4 Network object failed: ",
           Infoblox::status_code() . ":" . Infoblox::status_detail());
}
print "IPv4 Network object created successfully\n";

$session->add($network)
    or die("Add network failed: ",
           $session->status_code() . ":" . $session->status_detail());
print "Network added successfully\n";

#Create a zone object
my $zone = Infoblox::DNS::Zone->new(
                                    name => "test.com",
                                   );
unless($zone) {
     die("Construct Zone object failed: ",
           Infoblox::status_code() . ":" . Infoblox::status_detail());
}
```

```
print "Zone object created successfully\n";

$session->add($zone)
    or die("Add zone failed: ",
           $session->status_code() . ":" . $session->status_detail());
print "Zone added successfully\n";

my $option1 = Infoblox::DHCP::Option->new(
                                          name  => "routers",
                                          value => "10.0.0.100,10.0.0.200"
                                         );

unless($option1) {
    die("Construct option1 object failed: ",
        Infoblox::status_code() . ":" . Infoblox::status_detail());
}
print "option1 object created successfully\n";

my $option2 = Infoblox::DHCP::Option->new(
                                          name  => "nameservers",
                                          value => "1.1.1.1,2.2.2.2,3.3.3.3"
                                         );
unless($option2) {
    die("Construct option2 object failed: ",
        Infoblox::status_code() . ":" . Infoblox::status_detail());
}
print "option2 object created successfully\n";
```

#Create a DHCP IPv4 Host address object

```
#Create a DHCP IPv4 Host Address object
my $host_addr = Infoblox::DHCP::HostAddr->new(
       "ipv4addr"                => "10.0.0.5",
       "mac"                     => "00:00:00:11:11:11",
       "pxe_lease_time"          => 1234,
       "override_pxe_lease_time" => "true",
       "options"                 => [ $option1 , $option2 ],
       );
unless($host_addr) {
     die("Construct IPv4 host address failed: ",
           Infoblox::status_code() . ":" . Infoblox::status_detail());
}
print "DHCP IPv4 Host address object created successfully\n";

my $host = Infoblox::DNS::Host->new(
                                    name         => "host.test.com",
                                    ipv4addrs    => [$host_addr],
                                   );

unless($host) {
     die("Construct IPv4 host record failed: ",
           Infoblox::status_code() . ":" . Infoblox::status_detail());
}

$session->add($host)
    or die("Add host failed: ",
           $session->status_code() . ":" . $session->status_detail());
print "Host added successfully\n";
```

#Get and modify a DHCP IPv4 Host Address object

```
#Get DHCP IPv4 Host Address object through the session
my @retrieved_objs = $session->get(
    object => "Infoblox::DHCP::HostAddr",
    mac    => "00:00:00:11:11:11",
);
my $object = $retrieved_objs[0];
unless ($object) {
    die("Get IPv4 Host Address object failed: ",
        $session->status_code() . ":" . $session->status_detail());
}
print "Get IPv4 Host Address object found at least 1 matching entry\n";

#Modify one of the attributes of the obtained IPv4 Host Address object
$object->pxe_lease_time(1235);
```

```
#Apply the changes
$session->modify($object)
    or die("Modify IPv4 Host Address object failed: ",
           $session->status_code() . ":" . $session->status_detail());
print "IPv4 Host Address object modified successfully \n";

#Verify that this was changed also when retrieving this via the host record
my @retrieved_objs = $session->get(
    object => "Infoblox::DNS::Host",
    name => "host.test.com",
);
my $object = $retrieved_objs[0];
unless ($object) {
    die("Get Host record failed: ",
        $session->status_code() . ":" . $session->status_detail());
}
print "Get host record object found at least 1 matching entry\n";

my $hostaddress = @{$object->ipv4addrs()}[0];

unless ($hostaddress->pxe_lease_time() == 1235) {
    die("The modification of host address has failed: ",
        $session->status_code() . ":" . $session->status_detail());
}
print "The change to preferred lifetime has succeeded correctly\n";

#The host address retrieved this way can also be modified without submitting the full host record
$hostaddress->pxe_lease_time(1111);

#Apply the changes
$session->modify($hostaddress)
    or die("Modify IPv4 Host Address object failed: ",
           $session->status_code() . ":" . $session->status_detail());
print "IPv4 Host Address object modified successfully \n";

#Verify the change was submitted successfully
@retrieved_objs = $session->get(
    object => "Infoblox::DHCP::HostAddr",
    mac    => "00:00:00:11:11:11",
);
$object = $retrieved_objs[0];
unless ($object) {
    die("Get IPv4 Host Address object failed: ",
        $session->status_code() . ":" . $session->status_detail());
}
print "Get IPv4 Host Address object found at least 1 matching entry\n";

unless ($hostaddress->pxe_lease_time() == 1111) {
    die("The modification of host address has failed: ",
        $session->status_code() . ":" . $session->status_detail());
}

#Submit the Host object for removal
$session->remove($host)
    or die("Remove Host object failed: ",
        $session->status_code() . ":" . $session->status_detail());
print "Host object removed successfully \n";

#Submit the Zone object for removal
$session->remove($zone)
    or die("Remove Zone object failed: ",
        $session->status_code() . ":" . $session->status_detail());
print "Zone object removed successfully \n";

#Submit the Network object for removal
$session->remove($network)
    or die("Remove Network object failed: ",
        $session->status_code() . ":" . $session->status_detail());
print "Network object removed successfully \n";
```

AUTHOR

Infoblox Inc. http://www.infoblox.com/

SEE ALSO

Infoblox::DHCP::Network, Infoblox::DHCP::View, Infoblox::DHCP::Range, Infoblox::DHCP::Lease, Infoblox::Session->add(), Infoblox::Session->get(), Infoblox::Session->modify(), Infoblox::Session->remove(), Infoblox::Session->search(), Infoblox::Session, Infoblox::Grid::MSServer::AdUser::Data, Infoblox::Grid::Discovery::Data

---

COPYRIGHT

Copyright (c) 2017 Infoblox Inc.

Infoblox::DHCP::IPv6FixedAddr - DHCP IPv6 Fixed Address object.

- NAME
- DESCRIPTION
- CONSTRUCTOR
- SESSION METHODS
  - Infoblox::Session->add( )
  - Infoblox::Session->get( )
  - Infoblox::Session->modify( )
  - Infoblox::Session->remove( )
  - Infoblox::Session->search( )
- METHODS
  - address_type( )
  - allow_telnet( )
  - comment( )
  - configure_for_dhcp( )
  - cli_credentials( )
  - cloud_info( )
  - device_description( )
  - device_location( )
  - device_type( )
  - device_vendor( )
  - disable( )
  - discovered_duid( )
  - discovered_name( )
  - discoverer( )
  - discover_now_status( )
  - domain_name( )
  - domain_name_servers( )
  - duid( )
  - extattrs( )
  - extensible_attributes( )
  - first_discovered( )
  - ipv6addr( )
  - ipv6prefix( )
  - ipv6prefix_bits( )
  - last_discovered( )
  - match_client( )
  - ms_ad_user_data( )
  - name( )
  - netbios( )
  - network( )
  - network_component_description( )
  - network_component_ip( )
  - network_component_name( )
  - network_component_port_description( )
  - network_component_port_name( )
  - network_component_port_number( )
  - network_component_type( )
  - network_view( )
  - options( )
  - os( )
  - override_domain_name( )
  - override_domain_name_servers( )
  - override_cli_credentials( )
  - override_options( )
  - override_preferred_lifetime( )
  - override_valid_lifetime( )
  - port_duplex( )
  - port_link_status( )
  - port_speed( )
  - port_status( )
  - port_vlan_description( )
  - port_vlan_name( )
  - port_vlan_number ( )
  - preferred_lifetime( )
  - template( )
  - valid_lifetime( )
- SAMPLE CODE
- AUTHOR
- SEE ALSO
- COPYRIGHT

NAME

Infoblox::DHCP::IPv6FixedAddr - DHCP IPv6 Fixed Address object.

DESCRIPTION

An IPv6 Fixed Address is a specific IP address that a DHCP server always assigns when a lease request comes from a particular MAC address of the client. This is an IPv6 representation of a fixed address.

CONSTRUCTOR

```
 my $ipv6fa = Infoblox::DHCP::IPv6FixedAddr -> new(
       duid                          => $string,                    # Required
       address_type                  => "ADDRESS" | "PREFIX" | "BOTH", # Optional / Default is "ADDRESS"
       comment                       => $string,                    # Optional / Default is empty
       configure_for_dhcp            => "true" | "false",           # Optional / Default is "false"
                                                                    # When both the IP address and DUID
are present, the default value is "true"
                                                                    # Applicable only when adding this
fixed address to a host
       disable                       => "true" | "false",           # Optional / Default is "false"
       domain_name                   => $string,                    # Optional / Default is undefined
       domain_name_servers           => "$ipv6addr1,$ipv6addr2,...", # Optional / Default is undefined
       extattrs                      => { $string => $extattr, ... },#Optional / Default is undefined
       extensible_attributes         => { $string => $string | $num, $string => [ $string | $num, ... ],
... } #Optional / Default is undefined
       ipv6addr                      => $ipv6addr,                  # Optional / one of ipv6addr or
ipv6prefix is required
       ipv6prefix                    => $ipv6addr,                  # Optional / one of ipv6addr or
ipv6prefix is required
       ipv6prefix_bits               => $ipv6addr,                  # Optional / Required if ipv6prefix
is specified
       match_client                  => $string,                    # Optional / Default is "DUID"
                                                                    # Possible values are:
                                                                    #   "DUID", "RESERVED",
       name                          => $string,                    # Optional / Default is undefined
       network                       => $ipv6network,               # Optional / Default is undefined
       network_view                  => $NetworkView,               # Optional / Default is "default"
network view
       options                       => [$option1, $option2, ...]   # Optional / Default is empty
       override_domain_name          => "true" | "false",           # Optional / Default value is
"false"
       override_domain_name_servers  => "true" | "false",           # Optional / Default value is
"false"
       override_options              => "true" | "false",           # Optional / Default value is
"false"
       override_preferred_lifetime   => "true" | "false",           # Optional / Default value is
"false"
       override_valid_lifetime       => "true" | "false",           # Optional / Default value is
"false"
       preferred_lifetime            => $num,                       # Optional / Default is "27000"
       template                      => $string,                    # Optional / Default is undefined
       valid_lifetime                => $num,                       # Optional / Default is 43200
       cli_credentials               => [$cli1, ...],               # Optional
       override_cli_credentials      => "true" | "false",           # Optional / Default is "false"
       reserved_interface            => $interface,                 # Optional
       device_type                   => $string,                    # Optional
       device_vendor                 => $string,                    # Optional
       device_location               => $string,                    # Optional
       device_description            => $string,                    # Optional
       allow_telnet                  => "true" | "false",           # Optional / Default is "false"
 );
```

You cannot set both extattrs and extensible_attributes attributes at the same time.

SESSION METHODS

This section describes all the methods in Infoblox::Session module that you can apply to a DHCP IPv6 Fixed Address object.

Infoblox::Session->add( )

Use this method to add a DHCP IPv6 Fixed Address object to the Infoblox appliance. See Infoblox::Session->add() for parameters and return values.

Example

```
#Construct a DHCP IPv6 Fixed Address object
my $ipv6fa = Infoblox::DHCP::IPv6FixedAddr->new(
       duid                    => "0001",
       comment                 => "add ipv6 fixed address using uid",
       disable                 => "false",
       match_client            => "DUID",
       network                 => "2001::/24",
       network_view            => $nview,
       options                 => [ $option1,  $option2, $option3,  $option4, $option5, $option6 ],
);

#Submit for addition
my $response = $session->add($ipv6fa)
```

Infoblox::Session->get( )

Use this method to retrieve all the matching objects from the Infoblox appliance. See Infoblox::Session->get() for parameters and return values.

'discover_now_status' will not be returned by default, use 'return_methods' to request it if needed.

Key References

```
Apply the following attributes to get a specific DHCP IPv6 Fixed Address object:

  comment                                 - Optional. User-specified comment for the fixed address.
  duid                                    - Optional. DUID of the fixed address.
  ipv6addr                                - Optional. IPv6 address of the fixed address.
  network                                 - Optional. Network address of the fixed address.
  network_view                            - Optional. Network view in which the fixed address is
located. By default, all network views are searched.
  extattrs                                - Optional. A hash reference containing extensible
attributes.
  extensible_attributes                   - Optional. A hash reference containing extensible
attributes.

  discovered_duid                         - Optional. Discovered DHCPv6 UID.
  discovered_name                         - Optional. Discovered name.
  discoverer                              - Optional. Discoverer name.
  first_discovered                        - Optional. Time of the first discovery.
  last_discovered                         - Optional. Time of the last discovery.
  netbios                                 - Optional. Discovered NetBIOS name.
  network_component_description           - Optional. Discovered network component description.
  network_component_ip                    - Optional. Discovered network component IP address.
  network_component_name                  - Optional. Discovered network component name.
  network_component_port_description      - Optional. Discovered network component port description.
  network_component_port_name             - Optional. Discovered network component port name.
  network_component_port_number           - Optional. Discovered network component port number.
  network_component_type                  - Optional. Discovered network component type.
  os                                      - Optional. Discovered OS.
  port_duplex                             - Optional. Discovered port duplex settings.
  port_link_status                        - Optional. Discovered port link status.
  port_speed                              - Optional. Discovered port speed settings.
  port_status                             - Optional. Discovered port status settings.
  port_vlan_description                   - Optional. Discovered port VLAN description.
  port_vlan_name                          - Optional. Discovered port VLAN name.
  port_vlan_number                        - Optional. Discovered port VLAN number.
```

Examples

```
#Get DHCP IPv6 Fixed Address object through the session
my @retrieved_objs = $session->get(
    object => "Infoblox::DHCP::IPv6FixedAddr",
    duid   => "0001",
);
my @retrieved_objs = $session->get(
    object       => "Infoblox::DHCP::IPv6FixedAddr",
    ipv6addr     => "2001::1",
    network_view => "default",
);

my @retrieved_objs = $session->get(
   object                => "Infoblox::DHCP::IPv6FixedAddr",
   extensible_attributes => { 'Site' => 'Santa Clara' }
);
```

Infoblox::Session->modify( )

Use this method to modify a DHCP IPv6 Fixed Address object in the Infoblox appliance. See Infoblox::Session->modify() for parameters and return values.

Example

```
#Use method to modify the preferred_lifetime
$ipv6fa->preferred_lifetime("1234567");
#Submit modification
my $response = $session->modify( $ipv6fa );
```

Infoblox::Session->remove( )

Use this method to remove a DHCP IPv6 Fixed Address object from the Infoblox appliance. See Infoblox::Session->remove() for parameters and return values.

To remove a specific object, first use `get()` or `search()` to retrieve the specific DHCP IPv6 Fixed Address object, and then submit this object for removal.

Example

```
#Get DHCP IPv6 Fixed Address object through the session
my @retrieved_objs = $session->get(
    object       => "Infoblox::DHCP::IPv6FixedAddr",
    duid         => "0001",
    network_view => "default"
);
#Find the desired object from the retrieved list.
my $desired_ipv6fa = $retrieved_objs[0];
#Submit for removal
my $response = $session->remove( $desired_ipv6fa );
```

Infoblox::Session->search( )

Use this method to search for DHCP IPv6 Fixed Address object and matching Host object with MAC address in the Infoblox appliance. See Infoblox::Session->search() for parameters and return values.

'discover_now_status' will not be returned by default, use 'return_methods' to request it if needed.

Key References

```
 Apply the following attributes to search for a specific DHCP IPv6 Fixed Address object:

  comment                                 - Optional. User-specified comment for the fixed address.
  duid                                    - Optional. DUID of the fixed address.
  ipv6addr                                - Optional. IPv6 address of the fixed address.
  network                                 - Optional. Network address of the fixed address.
  network_view                            - Optional. Network view in which the fixed address is
located. By default, all network views are searched.
  discovered_duid                         - Optional. Discovered DHCPv6 UID (exact search only).
  discovered_name                         - Optional. Discovered name(regex).
  discoverer                              - Optional. Discoverer name(regex).
  first_discovered                        - Optional. Time of the first discovery (in ISO format, exact
search only).
  last_discovered                         - Optional. Time of the last discovery (in ISO format, exact
search only).
  netbios                                 - Optional. Discovered NetBIOS name(regex).
  network_component_description           - Optional. Discovered network component description(regex).
  network_component_ip                    - Optional. Discovered network component IP address(regex).
  network_component_name                  - Optional. Discovered network component name(regex).
  network_component_port_description      - Optional. Discovered network component port
description(regex).
  network_component_port_name             - Optional. Discovered network component port name(regex).
  network_component_port_number           - Optional. Discovered network component port number (exact
search only).
  network_component_type                  - Optional. Discovered network component type(regex).
  os                                      - Optional. Discovered OS (regex).
  port_duplex                             - Optional. Discovered port duplex settings (exact search
only).
  port_link_status                        - Optional. Discovered port link status (exact search only).
  port_speed                              - Optional. Discovered port speed settings (exact search
only).
  port_status                             - Optional. Discovered port status settings (exact search
only).
  port_vlan_description                   - Optional. Discovered port VLAN description(regex).
  port_vlan_name                          - Optional. Discovered port VLAN name(regex).
  port_vlan_number                        - Optional. Discovered port VLAN number(exact search only).

  extattrs                                - Optional. A hash reference containing extensible
attributes.
  extensible_attributes                   - Optional. A hash reference containing extensible
attributes.
```

For more information about searching extensible attributes, see Infoblox::Grid::ExtensibleAttributeDef/Searching Extensible Attributes.

Examples

```
 # Search for all DHCP IPv6 Fixed Address objects that are specified in the network "2001::/24" in
the "default" network view.
```

```
my @retrieved_objs = $session->search(
    object       => "Infoblox::DHCP::IPv6FixedAddr",
    duid         => "0001",
    network      => "2001::/24",
    network_view => "default"
);

# Search for all DHCP IPv6 Fixed Address objects with "Santa Clara" for the "Site" extensible
attribute
my @retrieved_objs = $session->search(
    object                => "Infoblox::DHCP::IPv6FixedAddr",
    extensible_attributes => { 'Site' => 'Santa Clara' }
);
```

METHODS

This section describes all the methods that you can use to set and retrieve the attribute values of a DHCP IPv6 Fixed Address object.

address_type( )

Use this method to set or retrieve the address type of a DHCP IPv6 Fixed Address object.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

Valid values are "ADDRESS", "PREFIX", or "BOTH". When the address type is "ADDRESS", a value for the 'ipv6addr' member is required. When the address type is "PREFIX", values for 'ipv6prefix' and 'ipv6prefix_bits' are required. When the address type is "BOTH", values for 'ipv6addr', 'ipv6prefix', and 'ipv6prefix_bits' are all required.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get address_type
my $address_type = $ipv6fa->address_type();
#Modify address_type
$ipv6fa->address_type("PREFIX");
```

allow_telnet( )

Use this method to set or retrieve the flag that enables copying SSH credential to TELNET.

Parameter

Specify 'true' to copy SSH credential to TELNET or 'false' to disable it. The default value is 'false'.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
# Get attribute value
my $value = $ipv6fa->allow_telnet();
# Modify attribute value
$ipv6fa->allow_telnet('true');
```

comment( )

Use this method to set or retrieve the descriptive comment of a DHCP IPv6 Fixed Address object.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

Desired comment in string format with a maximum of 256 bytes.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get comment
my $comment = $ipv6fa->comment();
#Modify comment
$ipv6fa->comment("Modified DHCP IPv6 Fixed Address object comment");
```

configure_for_dhcp( )

Use this method to set or retrieve the configure_for_dhcp flag of a DHCP IPv6 Fixed Address object.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

Specify "true" to set the configure_for_dhcp flag or "false" to deactivate/unset it. The default value is "false".

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get configure_for_dhcp
my $configure_for_dhcp = $ipv6fa->configure_for_dhcp();
#Modify configure_for_dhcp
$ipv6fa->configure_for_dhcp("true");
```

cli_credentials( )

Use this method to set or retrieve the discovery CLI credentials.

Setting this method to a defined value implicitly sets the override_cli_credentials method to "true". Setting the parameter to undefined causes the appliance to use the grid default and automatically resets the override_cli_credentials attribute to "false".

Note that when cli_credentials is set to a defined value and override_cli_credentials is set to "false", the last operation takes precedence. Thus the sequence $ipv6fa->cli_credentials([$cli]); $ipv6fa->override_cli_credentials("false"); will set override_cli_credentials to "false", and the sequence $ipv6fa->override_cli_credentials("false"); $ipv6fa->cli_credentials([$cli]); will result in override_cli_credentials="true".

Parameter

Valid value is an array reference that contains Infoblox::Grid::Discovery::CLICredential objects.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get cli_credentials
my $cli_credentials = $ipv6fa->cli_credentials();
#Modify cli_credentials
$ipv6fa->cli_credentials([$cli1, $cli2]);
```

cloud_info( )

Use this method to retrieve cloud API related information for the Infoblox::DHCP::IPv6FixedAddr object.

Parameter

None

Returns

The method returns the attribute value.

Example

```
# Get cloud_info
my $cloud_info = $object->cloud_info();
```

device_description( )

Use this method to set or retrieve the description of the discovery device.

Parameter

The device description in string format.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get device_description
my $device_description = $ipv6fa->device_description();
#Modify device_description
$ipv6fa->device_description('example');
```

device_location( )

Use this method to set or retrieve the location of the discovery device.

Parameter

The device location in string format.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get device_location
my $device_location = $ipv6fa->device_location();
#Modify device_location
$ipv6fa->device_location('example');
```

device_type( )

Use this method to set or retrieve the type of the discovery device.

Parameter

The device type in string format.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get device_type
my $device_type = $ipv6fa->device_type();
#Modify device_type
$ipv6fa->device_type('example');
```

device_vendor( )

Use this method to set or retrieve the vendor name of the discovery device.

Parameter

The device vendor name in string format.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get device_vendor
```

```
my $device_vendor = $ipv6fa->device_vendor();
#Modify device_vendor
$ipv6fa->device_vendor('example');
```

disable( )

Use this method to set or retrieve the disable flag of a DHCP IPv6 Fixed Address object.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

Specify "true" to set the disable flag or "false" to deactivate/unset it. The default value is "false".

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get disable
my $disable = $ipv6fa->disable();
#Modify disable
$ipv6fa->disable("true");
```

=head2 discovered_data( )

Use this method to retrieve the discovered data of a DHCP IPv6 Fixed Address object. This is a read-only attribute.

Omit the parameter to retrieve the attribute value.

Parameter

None

Returns

The valid return value is an Infoblox::Grid::Discovery::Data object.

Example

```
#Get discovered_data value
my $discovered_data = $object->discovered_data();
```

discovered_duid( )

Use this method to retrieve the discovered DHCPv6 UID (DUID) of a DHCP IPv6 Fixed Address object.

Parameter

none

Returns

The method returns the discovered name.

Example

```
#Get discovered_duid
my $discovered_duid = $ipv6fa->discovered_duid();
```

discovered_name( )

Use this method to retrieve the discovered name of a DHCP IPv6 Fixed Address object.

Parameter

none

Returns

The method returns the discovered name.

Example

```
#Get discovered_name
```

```
my $discovered_name = $ipv6fa->discovered_name();
```

discoverer( )

Use this method to retrieve the discoverer of a DHCP IPv6 Fixed Address object.

Parameter

none

Returns

The method returns the discoverer value.

Example

```
#Get discoverer
my $discoverer = $ipv6fa->discoverer();
```

discover_now_status( )

Use this method to retrieve the current discovery status. This attribute is read-only.

'discover_now_status' will not be returned by default, use 'return_methods' to request it if needed.

Parameter

None.

Returns

The method returns the attribute value that can be 'COMPLETE', 'FAILED', 'NONE', 'PENDING' or 'RUNNING'.

Example

```
#Get discover_now_status
my $discover_now_status = $ipv6fa->discover_now_status();
```

domain_name( )

Use this method to set or retrieve the domain_name value of a DHCP IPv6 fixed address object.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Setting this method to a defined value implicitly sets the override_domain_name method to "true". Setting the parameter to undefined causes the appliance to use the upper-level default and automatically resets the override_domain_name attribute to "false".

Note that when domain_name is set to a defined value and override_domain_name is set to "false", the last operation takes precedence. Thus the sequence $object->domain_name("testdomain"); $object->override_domain_name("false"); will set override_domain_name to "false", and the sequence $object->override_domain_name("false"); $object->domain_name("testdomain"); will result in override_domain_name="true".

Parameter

The client's domain name in string format.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get domain_name
my $domain_name = $ipv6fa->domain_name();
#Modify domain_name
$ipv6fa->domain_name("test_domain.com");
```

domain_name_servers( )

Use this method to set or retrieve the domain_name_servers value of a DHCP IPv6 fixed address object.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Setting this method to a defined value implicitly sets the override_domain_name_servers method to "true". Setting the parameter to undefined causes the appliance to use the upper-level default and automatically resets the override_domain_name_servers attribute to "false".

Note that when domain_name_servers is set to a defined value and override_domain_name_servers is set to "false", the last operation takes precedence. Thus the sequence $object->domain_name_servers("2001::1,2001::2"); $object->override_domain_name_servers("false"); will set override_domain_name_servers to "false", and the sequence $object->override_domain_name_servers("false"); $object->domain_name_servers("2001::1,2001::2"); will result in override_domain_name_servers="true".

Parameter

The comma-separated list of domain name server addresses in IPv6 address format.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get domain_name_servers
my $domain_name_servers = $ipv6fa->domain_name_servers();
#Modify domain_name_servers
$ipv6fa->domain_name_servers("2001::1,2001::2");
```

duid( )

Use this method to set or retrieve the DHCPv6 Unique Identifier (DUID) of the IPv6 Fixed Address object.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

The DUID of the matching client in hexadecimal format.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get duid
my $duid = $ipv6fa->duid();
#Modify duid
$ipv6fa->duid("1c2edfaa");
```

extattrs( )

Use this method to set or retrieve the extensible attributes associated with an IPv6 Fixed Address object.

Parameter

Valid value is a hash reference containing the names of extensible attributes and their associated values (Infoblox::Grid::Extattr objects).

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get extattrs
my $ref_extattrs = $ipv6fa->extattrs();
#Modify extattrs
$ipv6fa->extattrs({ 'Site' => $extattr1, 'Administrator' => $extattr2 });
```

extensible_attributes( )

Use this method to set or retrieve the extensible attributes associated with an IPv6 Fixed Address object.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

For valid values for extensible attributes, see Infoblox::Grid::ExtensibleAttributeDef/Extensible Attribute Values.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get extensible attributes
my $ref_extensible_attributes = $ipv6fa->extensible_attributes();
#Modify extensible attributes
$ipv6fa->extensible_attributes({ 'Site' => 'Santa Clara', 'Administrator' => [ 'Peter', 'Tom' ] });
```

first_discovered( )

Use this method to retrieve the date and time that the DHCP IPv6 Fixed Address object was first discovered.

Parameter

none

Returns

The method returns the date and time that the DHCP IPv6 Fixed Address object was first discovered.

Example

```
#Get first_discovered
my $first_discovered = $ipv6fa->first_discovered();
```

ipv6addr( )

Use this method to set or retrieve the the IPv6 address of the IPv6 Fixed Address object.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

An IPv6 address is a 128-bit number represented as eight groups of four hexadecimal digits (i.e. two octets), each group separated by a `colon(:)` (example: 2001:0db8:85a3:0000:0000:8a2e:0370:7334). Leading zeroes and groups of consecutive zeros within the value can be omitted to produce a more compact representation (example: 2001::1, ::1), and dotted-quad notation can be used to express IPv4-mapped IPv6 addresses (example: ::ffff:192.0.2.128).

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get ipv6addr
my ipv6addr = $ipv6fa->ipv6addr();
#Modify ipv6addr
$ipv6fa->ipv6addr("aaaa:1111:bbbb:2222:cccc:3333:dddd:5555");
```

ipv6prefix( )

Use this method to set or retrieve the IPv6 prefix of a DHCP IPv6 Fixed Address object. The prefix must belong to the list of global prefixes defined in the Infoblox::Grid::DHCP object.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

IPv6 prefix from the list of global IPv6 prefixes.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.
ls.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get ipv6prefix
my $ipv6prefix = $ipv6fa->ipv6prefix();
#Modify ipv6prefix
```

```
$ipv6fa->ipv6prefix("2001::");
```

ipv6prefix_bits( )

Use this method to set or retrieve the prefix bits for the DHCP IPv6 Fixed Address.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

An unsigned integer between 2 and 128.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get ipv6prefix_bits
my $ipv6prefix_bits = $ipv6fa->ipv6prefix_bits();
#Modify ipv6prefix_bits
$ipv6fa->ipv6prefix_bits(32);
```

last_discovered( )

Use this method to retrieve the time this object was last seen by a discovery job.

The default value for this field is empty.

Parameter

N/A

Returns

The method returns the attribute value.

Example

```
#Get last_discovered
my $last_discovered = $ipv6fa->last_discovered();
```

match_client( )

Use this method to set or retrieve the match_client value of a DHCP Fixed Address object.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

```
match_client "DUID": The fixed IP address is leased to the matching DUID.
```

Returns

```
If you specified a parameter, the method returns true when the modification succeeds, and returns
false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.
```

Example

```
#Get match_client
my $match_client = $ipv6fa->match_client();
#Modify match_client
$ipv6fa->match_client("DUID");
```

ms_ad_user_data( )

Use this method to retrieve Microsoft Active Directory users related information. This is a read-only attribute.

Parameter

None

Returns

The valid return value is an Infoblox::Grid::MSServer::AdUser::Data object.

Example

```
#Get ms_ad_user_data
my $ms_ad_user_data = $ipv6fa->ms_ad_user_data();
```

name( )

Use this method to set or retrieve the name of a DHCP IPv6 Fixed Address object. The same name can be used for multiple fixed addresses.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

Specify a name in string format.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get the name
my $name = $ipv6fa->name();
#Modify the name
$ipv6fa->name("pop-server");
```

netbios( )

Use this method to retrieve the name in the NetBIOS reply that responded to a NetBIOS query.

Returns

The method returns the attribute value.

Example

```
#Get NetBIOS
my $netbios = $ipv6fa->netbios();
```

network( )

Use this method to retrieve the network value of a DHCP IPv6 Fixed Address object.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

Specify the address and netmask of the network in CIDR format (example: 2001::/64).

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get network
my $network = $ipv6addr->network();
#Modify network
$ipv6fa->network("2001::/24");
```

network_component_description( )

Use this method to retrieve the description of the network device that is connected to the DHCP IPv6 Fixed Address object.

Parameter

none

Returns

The method returns the network device description.

Example

```
#Get network_component_description
my $network_component_description = $ipv6fa->network_component_description();
```

network_component_ip( )

Use this method to retrieve the IP address of the network device that is connected to the DHCP IPv6 Fixed Address object.

Parameter

none

Returns

The method returns the IP address of the network device.

Example

```
#Get network_component_ip
my $network_component_ip = $ipv6fa->network_component_ip();
```

network_component_name( )

Use this method to retrieve the name of the network device that is connected to the DHCP IPv6 Fixed Address object.

Parameter

none

Returns

The method returns the network device name.

Example

```
#Get network_component_name
my $network_component_name = $ipv6fa->network_component_name();
```

network_component_port_description( )

Use this method to retrieve the description of the network device port that is connected to the DHCP IPv6 Fixed Address object.

Parameter

none

Returns

The method returns the description of the port on the network device.

Example

```
#Get network_component_port_description
my $network_component_port_description = $ipv6fa->network_component_port_description();
```

network_component_port_name( )

Use this method to retrieve the name of the network device port that is connected to the DHCP IPv6 Fixed Address object.

Parameter

none

Returns

The method returns the name of the port on the network device.

Example

```
#Get network_component_port_name
my $network_component_port_name = $ipv6fa->network_component_port_name();
```

network_component_port_number( )

Use this method to retrieve the number of the network device port that is connected to the DHCP IPv6 Fixed Address object.

Parameter

none

Returns

The method returns the network device port number.

Example

```
#Get network_component_port_number
my $network_component_port_number = $ipv6fa->network_component_port_number();
```

network_component_type( )

Use this method to retrieve the type of the network device that is connected to the DHCP IPv6 Fixed Address object.

Parameter

none

Returns

The method returns the network device type.

Example

```
#Get network_component_type
my $network_component_type = $ipv6fa->network_component_type();
```

network_view( )

Use this method to set or retrieve the network view of the DHCP fixed address.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

The default value is the "default" network view, which means the fixed address is in the default network view.

Parameter

Valid value is a defined Infoblox::DHCP::View object. The default network view value is the system-defined default network view object.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get network view
my $nview = $ipv6fa->network_view();
#Modify network view, with an Infoblox::DHCP::View object
$ipv6fa->network_view($nview);
```

options( )

Use this method to set or retrieve the options of a DHCP IPv6 Fixed Address object.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Setting this method to a defined value implicitly sets the override_options method to "true". Setting the parameter to undefined causes the appliance to use the grid default and automatically resets the override_options attribute to "false".

Note that when the options method is set to a defined value and override_options is set to "false", the last operation takes precedence. Thus the sequence $object->options([$opt]); $object->override_options("false"); will set override_options to "false", and the sequence $object->override_options("false"); $object->options([$opt]); will result in override_options="true".

Parameter

The valid value is an array reference that contains Infoblox::DHCP::Option object(s).

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get options
my $options = $ipv6fa->options();
#Modify options
my $option1 = Infoblox::DHCP::Option->new(
    name   => "dhcp6.rapid-commit",
    value  => "true"
);
$ipv6fa->options([$option1);
```

os( )

Use this method to retrieve the operating system associated with this fixed address.

Returns

The method returns the attribute value.

Example

```
#Get OS
my $os = $ipv6fa->os();
```

override_domain_name( )

The override_domain_name attribute controls whether the domain_name value in the DHCP IPv6 fixed address is used, instead of the upper-level default.

The override_domain_name attribute can be specified explicitly. It is also set implicitly when domain_name is set to a defined value.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

Set the parameter to "true" to override the upper-level setting for domain_name. Set the parameter to "false" to inherit the upper-level setting for domain_name.

The default value of this parameter is "false".

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get override_domain_name
my $override_domain_name=$ipv6fa->override_domain_name( );
#Modify override_domain_name
$ipv6fa->override_domain_name("true");
```

override_domain_name_servers( )

The override_domain_name_servers attribute controls whether the domain_name_servers value in the DHCP IPv6 fixed address is used, instead of the upper-level default.

The override_domain_name_servers attribute can be specified explicitly. It is also set implicitly when domain_name_servers is set to a defined value.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

Set the parameter to "true" to override the upper-level setting for domain_name_servers. Set the parameter to "false" to inherit the upper-level setting for domain_name_servers.

The default value of this parameter is "false".

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get override_domain_name_servers
my $override_domain_name_servers=$ipv6fa->override_domain_name_servers( );
```

```
#Modify override_domain_name_servers
$ipv6fa->override_domain_name_servers("true");
```

override_cli_credentials( )

The override_cli_credentials attribute controls whether the cli_credentials value in the object is used, instead of the grid default.

The override_cli_credentials attribute can be specified explicitly. It is also set implicitly when cli_credentials is set to a defined value.

Parameter

Specify "true" to set the override_cli_credentials flag or "false" to deactivate/unset it. The default value is "false".

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
# Get attribute value
my $value = $ipv6fa->override_cli_credentials();
# Modify attribute value
$ipv6fa->override_cli_credentials('true');
```

override_options( )

The override_options attribute controls whether the options in the DHCP IPv6 fixed address are used, instead of the upper-level default.

The override_options attribute can be specified explicitly. It is also set implicitly when options are set to defined values.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

Set the parameter to "true" to override the upper-level setting for valid_lifetime. Set the parameter to "false" to inherit the upper-level setting for valid_lifetime.

The default value of this parameter is "false".

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get override_options
my $override_options=$ipv6fa->override_options( );
#Modify override_options
$ipv6fa->override_options("true");
```

override_preferred_lifetime( )

The override_preferred_lifetime attribute controls whether the preferred_lifetime value in the DHCP IPv6 fixed address is used, instead of the upper-level default.

The override_preferred_lifetime attribute can be specified explicitly. It is also set implicitly when preferred_lifetime is set to a defined value.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

Set the parameter to "true" to override the upper-level setting for valid_lifetime. Set the parameter to "false" to inherit the upper-level setting for valid_lifetime.

The default value of this parameter is "false".

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get override_preferred_lifetime
my $override_preferred_lifetime=$ipv6fa->override_preferred_lifetime( );
#Modify override_preferred_lifetime
$ipv6fa->override_preferred_lifetime("true");
```

override_valid_lifetime( )

The override_valid_lifetime attribute controls whether the valid_lifetime value in the DHCP IPv6 fixed address is used, instead of the upper-level default.

The override_valid_lifetime attribute can be specified explicitly. It is also set implicitly when valid_lifetime is set to a defined value.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

Set the parameter to "true" to override the upper-level setting for valid_lifetime. Set the parameter to "false" to inherit the upper-level setting for valid_lifetime.

The default value of this parameter is "false".

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get override_valid_lifetime
my $override_valid_lifetime=$ipv6fa->override_valid_lifetime( );
#Modify override_valid_lifetime
$ipv6fa->override_valid_lifetime("true");
```

port_duplex( )

Use this method to retrieve the duplex setting of the network device port that is connected to the DHCP IPv6 Fixed Address object.

Parameter

none

Returns

The method returns the port duplex setting. The returned value, if any, is one of the following:

```
Full
Half
```

Example

```
#Get port_duplex
my $port_duplex = $ipv6fa->port_duplex();
```

port_link_status( )

Use this method to retrieve the link status of the network device port that is connected to the DHCP IPv6 Fixed Address object.

Parameter

none

Returns

This method returns the link status of the network device port. The returned value, if any, is one of the following:

```
Connected
Not Connected
Unknown
```

Example

```
#Get port_link_status
my $port_link_status = $ipv6fa->port_link_status();
```

port_speed( )

Use this method to retrieve the speed of the network device port that is connected to the DHCP IPv6 Fixed Address object.

Parameter

none

Returns

The method returns the network device port speed. The returned value, if any, is one of the following:

```
10M
100M
1G
10G
100G
Unknown
```

Example

```
#Get port_speed
my $port_speed = $ipv6fa->port_speed();
```

port_status( )

Use this method to retrieve the status of the network device port that is connected to the DHCP IPv6 Fixed Address object.

Parameter

none

Returns

The method returns the network device port status. The returned value, if any, is one of the following:

```
Up
Down
Unknown
```

Example

```
#Get port_status
my $port_status = $ipv6fa->port_status();
```

port_vlan_description( )

Use this method to retrieve the VLAN description of the network device port that is connected to the DHCP IPv6 Fixed Address object.

Parameter

none

Returns

The method returns the VLAN description of the network device port.

Example

```
#Get port_vlan_description
my $port_vlan_description = $ipv6fa->port_vlan_description();
```

port_vlan_name( )

Use this method to retrieve the VLAN name of the network device port that is connected to the DHCP IPv6 Fixed Address object.

Parameter

none

Returns

The method returns the network device port VLAN name.

Example

```
#Get port_vlan_name
my $port_vlan_name = $ipv6fa->port_vlan_name();
```

port_vlan_number ( )

Use this method to retrieve the VLAN number of the network device port that is connected to the DHCP IPv6 Fixed Address object.

Parameter

none

Returns

The method returns the port VLAN number.

Example

```
#Get port_vlan_number
my $port_vlan_number  = $ipv6fa->port_vlan_number ();
```

preferred_lifetime( )

Use this method to set or retrieve the preferred_lifetime value of a DHCP IPv6 Fixed Address object.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Setting this method to a defined value implicitly sets the override_preferred_lifetime method to "true". Setting the parameter to undefined causes the appliance to use the upper-level default and automatically resets the override_preferred_lifetime attribute to "false".

Note that when preferred_lifetime is set to a defined value and override_preferred_lifetime is set to "false", the last operation takes precedence. Thus the sequence $object->preferred_lifetime(20000); $object->override_preferred_lifetime("false"); will set override_preferred_lifetime to "false", and the sequence $object->override_preferred_lifetime("false"); $object->preferred_lifetime(20000); will result in override_preferred_lifetime="true".

Parameter

A 32-bit integer (valid range from 1 to 4294967295) that represents the lifetime, in seconds. The default value is 27000.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get preferred_lifetime
my $preferred_lifetime = $ipv6fa->preferred_lifetime();
#Modify preferred_lifetime
$ipv6fa->preferred_lifetime("369");
```

template( )

Use this method to set or retrieve the fixed address template name for a DHCP IPv6 Fixed Address object.

Template is the name of a DHCP IPv6 Fixed Address template which will be used to create the DHCP IPv6 Fixed Address.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

The valid name in string format.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

The template parameter can be used to only add a new DHCP IPv6 Fixed Address and cannot be used during object modify or returned during object get.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get template
my $template = $ipv6fa->template();
#Modify template
$ipv6fa->template("Template_Name");
```

valid_lifetime( )

Use this method to set or retrieve the valid_lifetime value of a DHCP IPv6 Fixed Address object.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

A 32-bit integer, from 1 to 4294967295, that represents the lifetime, in seconds. The default value is 43200.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get valid_lifetime
my $valid_lifetime = $ipv6fa->valid_lifetime();
#Modify valid_lifetime
$ipv6fa->valid_lifetime("36900");
```

SAMPLE CODE

The following sample code demonstrates the different functions that can be applied to an object such as add, search, modify, and remove. This sample also includes error handling for the operations.

#Preparation prior to a DHCP IPv6 Fixed Address object insertion

```
#PROGRAM STARTS: Include all the modules that will be used
use strict;
use Infoblox;

#Create a session to the Infoblox appliance

my $session = Infoblox::Session->new(
               master   => "192.168.1.2", #appliance host ip
               username => "admin",       #appliance user login
               password => "infoblox"     #appliance password
);

unless ($session) {
       die("Construct session failed: ",
               Infoblox::status_code() . ":" . Infoblox::status_detail());
}
print "Session created successfully\n";

#Create an IPv6 network object
my $network = Infoblox::DHCP::IPv6Network->new(
                                          "network" => "2001::/64",
                                         );
unless($network) {
     die("Construct IPv6 Network object failed: ",
          Infoblox::status_code() . ":" . Infoblox::status_detail());
}
print "IPv6 Network object created successfully\n";

$session->add($network)
    or die("Add network failed: ",
           $session->status_code() . ":" . $session->status_detail());
print "Network added successfully\n";

my $option1 = Infoblox::DHCP::Option->new(
    "name"  => "dhcp6.fqdn",
    "value" => "mydomain.com"
);
unless($option1) {
    die("Construct option1 object failed: ",
        Infoblox::status_code() . ":" . Infoblox::status_detail());
}
print "option1 object created successfully\n";

my $option2 = Infoblox::DHCP::Option->new(
    "name"     => "dhcp6.info-refresh-time",
    "value"    => "1000",
);
unless($option2) {
    die("Construct option2 object failed: ",
        Infoblox::status_code() . ":" . Infoblox::status_detail());
}
print "option2 object created successfully\n";
```

#Create a DHCP IPv6 Fixed address object

```
#Create a DHCP IPv6 Fixed Address object
my $ipv6fa = Infoblox::DHCP::IPv6FixedAddr->new(
       "ipv6addr"       => "2001::4",
       "network"        => "2001::/64",
       "duid"           => "000000111111",
       "comment"        => "add fixed address",
       "domain_name"    => "foo.com",
       "domain_name_servers" => "2002::1,2002::2",
       "preferred_lifetime" => 1234,
       "override_preferred_lifetime" => "true",
       "valid_lifetime" => 123456,
       "override_valid_lifetime" => "true",
       "options"        => [ $option1 , $option2 ],
       );
unless($ipv6fa) {
     die("Construct IPv6 fixed address failed: ",
           Infoblox::status_code() . ":" . Infoblox::status_detail());
}
print "DHCP IPv6 Fixed address object created successfully\n";

#Verify if the DHCP IPv6 Fixed Address exists in the Infoblox appliance
my $object = $session->get(object => "Infoblox::DHCP::IPv6FixedAddr", duid => "00:00:00:11:11:11");
unless ($object) {
   print "IPv6 fixed address does not exist on server, safe to add the fixed address\n";
   $session->add($ipv6fa)
      or die("Add IPv6 fixed address failed: ",
             $session->status_code() . ":" . $session->status_detail());
}
print "DHCP IPv6 Fixed address added successfully\n";
```

#Create a DHCP IPv6 Fixed address object using a template

```
#Create a DHCP IPv6 Fixed Address template object
my $ipv6fa_template = Infoblox::DHCP::IPv6FixedAddrTemplate->new(
    "name"                  => "custom_fa_template",
    "offset"                => "10",
    "number_of_addresses"   => "1",
    "comment"               => "add fixed address template",
    "valid_lifetime"        => 86400,
);
unless($ipv6fa_template) {
     die("Construct IPv6 fixed address failed: ",
           Infoblox::status_code() . ":" . Infoblox::status_detail());
}
print "DHCP IPv6 Fixed address template object created successfully\n";

$session->add($ipv6fa_template)
      or die("Add IPv6 fixed address template failed: ",
             $session->status_code() . ":" . $session->status_detail());
print "DHCP IPv6 Fixed address template added successfully\n";

#Create a DHCP IPv6 Fixed Address object
my $ipv6fa = Infoblox::DHCP::IPv6FixedAddr->new(
    "network"        => "2001::/64",
    "ipv6addr"       => "2001::2",
    "match_client"   => "DUID",
    "duid"           => "010203040505",
    "template"       => "custom_fa_template"
);
unless($ipv6fa) {
     die("Construct fixed address failed: ",
           Infoblox::status_code() . ":" . Infoblox::status_detail());
}
print "DHCP IPv6 Fixed address object created successfully\n";

$session->add($ipv6fa)
      or die("Add fixed address failed: ",
             $session->status_code() . ":" . $session->status_detail());
print "DHCP IPv6 Fixed address added successfully\n";
```

#Search for a specific DHCP IPv6 Fixed Address object

```
#Search for all DHCP IPv6 Fixed address objects under the network 2001::/64
my @retrieved_objs = $session->search(
    object  => "Infoblox::DHCP::IPv6FixedAddr",
    duid     => "0000001111.*",
```

```
    network => "2001::/64" );
my $object = $retrieved_objs[0];

unless ($object) {
       die("Search for IPv6 Fixed Address object failed: ",
               $session->status_code() . ":" . $session->status_detail());
}
print "Search IPv6 Fixed Address object found at least 1 matching entry\n";
```

#Get and modify a DHCP IPv6 Fixed Address object

```
#Get DHCP IPv6 Fixed Address object through the session
my @retrieved_objs = $session->get(
    object => "Infoblox::DHCP::IPv6FixedAddr",
    duid    => "000000111111",
);
my $object = $retrieved_objs[0];
unless ($object) {
    die("Get IPv6 Fixed Address object failed: ",
        $session->status_code() . ":" . $session->status_detail());
}
print "Get IPv6 Fixed Address object found at least 1 matching entry\n";

#Modify one of the attributes of the obtained IPv6 Fixed Address object
$object->preferred_lifetime("1235");

#Apply the changes
$session->modify($object)
    or die("Modify IPv6 Fixed Address object failed: ",
           $session->status_code() . ":" . $session->status_detail());
print "IPv6 Fixed Address object modified successfully \n";
```

#Remove an IPv6 Fixed Address object

```
#Get IPv6 Fixed Address object through the session
my @retrieved_objs = $session->get(
    object => "Infoblox::DHCP::IPv6FixedAddr",
    duid    => "000000111111",
);
my $object = $retrieved_objs[0];
unless ($object) {
    die("Get IPv6 Fixed Address object failed: ",
        $session->status_code() . ":" . $session->status_detail());
}
print "Get IPv6 Fixed Address object found at least 1 matching entry\n";

#Submit the object for removal
$session->remove($object)
    or die("Remove IPv6 Fixed Address object failed: ",
        $session->status_code() . ":" . $session->status_detail());
print "IPv6 Fixed Address object removed successfully \n";

#Submit the Network object for removal
$session->remove($network)
    or die("Remove Network object failed: ",
        $session->status_code() . ":" . $session->status_detail());
print "Network object removed successfully \n";

#Submit the Fixed Address Template object for removal
$session->remove($ipv6fa_template)
    or die("Remove IPv6 Fixed Address Template object failed: ",
        $session->status_code() . ":" . $session->status_detail());
print "IPv6 Fixed Address Template object removed successfully \n";
####PROGRAM ENDS####
```

AUTHOR

Infoblox Inc. http://www.infoblox.com/

SEE ALSO

Infoblox::DHCP::IPv6Network, Infoblox::DHCP::View, Infoblox::DHCP::IPv6Range, Infoblox::DHCP::IPv6Lease, Infoblox::Session->add(), Infoblox::Session->get(), Infoblox::Session->modify(), Infoblox::Session->remove(), Infoblox::Session->search(), Infoblox::Session, Infoblox::DHCP::IPv6FixedAddrTemplate, Infoblox::Grid::Discovery::Data

---

COPYRIGHT

Copyright (c) 2017 Infoblox Inc.

Infoblox::DHCP::IPv6FixedAddrTemplate - DHCP IPv6 Fixed Address Template object

- NAME
- DESCRIPTION
- CONSTRUCTOR
- MODULE METHODS
  - Infoblox::DHCP::NetworkTemplate-> fixed_address_templates( )
- SESSION METHODS
  - Infoblox::Session->add( )
  - Infoblox::Session->get( )
  - Infoblox::Session->search( )
  - Infoblox::Session->modify( )
  - Infoblox::Session->remove( )
- METHODS
  - comment( )
  - domain_name( )
  - domain_name_servers( )
  - extattrs( )
  - extensible_attributes( )
  - name( )
  - number_of_addresses( )
  - offset( )
  - options( )
  - override_domain_name( )
  - override_domain_name_servers( )
  - override_options( )
  - override_preferred_lifetime( )
  - override_valid_lifetime( )
  - preferred_lifetime( )
  - valid_lifetime( )
- SAMPLE CODE
- AUTHOR
- SEE ALSO
- COPYRIGHT

NAME

Infoblox::DHCP::IPv6FixedAddrTemplate - DHCP IPv6 Fixed Address Template object

---

DESCRIPTION

An IPv6 fixed address template contains a set of predefined properties and can be used to create an IPv6 fixed address object in a quick and consistent way. An IPv6 fixed address created from an IPv6 fixed address template inherits the properties defined in the template.

CONSTRUCTOR

```
my $fixed_addr_template = Infoblox::DHCP::IPv6FixedAddrTemplate -> new(
        name                          => $string,                                  #Required
        comment                       => $string,                                  #Optional / Default is
undefined
        domain_name                   => $fqdn,                                    #Optional / Default is
undefined
        domain_name_servers           => "$ipv6addr1,$ipv6addr2,...",              #Optional / Default is
undefined
        extattrs                      => { $string => $extattr, ... },             #Optional / Default is
undefined
        extensible_attributes         => { $string => $string | $num, $string => [ $string | $num, ... ],
... }, #Optional / Default is undefined
        number_of_addresses           => $num | undef,                             #Optional / Default is
undefined
        offset                        => $num | undef,                             #Optional / Default is
undefined
        options                       => [$option1, $option2],                     #Optional / Default is empty
        override_domain_name          => "true" | "false",                         #Optional / Default is
"false"
        override_domain_name_servers => "true" | "false",                          #Optional / Default is
"false"
        override_options              => "true" | "false",                         #Optional / Default is
"false"
        override_preferred_lifetime   => "true" | "false",                         #Optional / Default is
"false"
```

```
        override_valid_lifetime      => "true" | "false",                #Optional / Default is
"false"
        preferred_lifetime           => $num,                            #Optional / Default is 27000
        valid_lifetime               => $num,                            #Optional / Default is 43200
 );
```

You cannot set both extattrs and extensible_attributes attributes at the same time.

MODULE METHODS

The following functions can be applied to a DHCP IPv6 Fixed Address Template object.

Infoblox::DHCP::NetworkTemplate-> fixed_address_templates( )

Use this function to include a fixed address template in a network template on the Infoblox appliance. See Infoblox::DHCP::IPv6NetworkTemplate->fixed_address_templates() for parameters and return values.

Example

```
#Create the IPv6 network template that includes the IPv6 fixed address template
my $template = Infoblox::DHCP::Template->new(
  name                    => "custom_fa_template"
);
my $network_template = Infoblox::DHCP::IPv6NetworkTemplate->new(
  name                    => "custom_network_template",
  comment                 => "add network",
  fixed_address_templates => [$template]
);
```

SESSION METHODS

This section describes all the methods in the Infoblox::Session module that can be applied to a DHCP IPv6 fixed address template object.

Infoblox::Session->add( )

Use this method to add a DHCP IPv6 fixed address template object to the Infoblox appliance. See Infoblox::Session->add() for parameters and return values.

Example

```
#Construct a DHCP IPv6 Fixed Address Template object
my $fixed_addr_template = Infoblox::DHCP::IPv6FixedAddrTemplate->new(
);

#Submit for addition
my $response = $session->add($fixed_addr_template);
```

Infoblox::Session->get( )

Use this method to retrieve all the matching objects from the Infoblox appliance. See Infoblox::Session->get() for parameters and return values.

Key References

```
Apply the following attributes to get a specific DHCP IPv6 Fixed Address Template object:

name - Optional. The name of the fixed address template.
extattrs     - Optional. A hash reference containing extensible attributes.
extensible_attributes - Optional. A hash reference containing extensible attributes.
```

Examples

```
#Get DHCP IPv6 fixed address template object through the session
my @retrieved_objs = $session->get(
    object => "Infoblox::DHCP::IPv6FixedAddrTemplate",
    name   => "custom_fa_template",
);

my @retrieved_objs = $session->get(
  object                => "Infoblox::DHCP::IPv6FixedAddrTemplate",
  extensible_attributes => { 'Site' => 'Santa Clara' }
);
```

Infoblox::Session->search( )

Use this method to retrieve all the matching objects from the Infoblox appliance. See Infoblox::Session->search() for parameters and return values.

Key References

```
Apply the following attributes to search for a DHCP IPv6 Fixed Address Template object:

name    - Optional. The name of the fixed address template (regular expression).
comment - Optional. The comment of the fixed address template (regular expression).
extattrs     - Optional. A hash reference containing extensible attributes.
extensible_attributes - Optional. A hash reference containing extensible attributes.
```

For more information about searching extensible attributes, see Infoblox::Grid::ExtensibleAttributeDef/Searching Extensible Attributes.

Examples

```
#Search for a DHCP IPv6 fixed address template object through the session
my @retrieved_objs = $session->search(
    object  => "Infoblox::DHCP::IPv6FixedAddrTemplate",
    name    => "custom_fa_template",
    comment => ".*",
);

#Search for DHCP IPv6 Fixed Address Template objects that specify "Santa Clara" for the "Site"
extensible attribute
my @retrieved_objs = $session->search(
    object                => "Infoblox::DHCP::IPv6FixedAddrTemplate",
    extensible_attributes => { 'Site' => 'Santa Clara' }
);
```

Infoblox::Session->modify( )

Use this method to modify a DHCP IPv6 fixed address template object in the Infoblox appliance. See Infoblox::Session->modify() for parameters and return values.

Example

```
#Modify the pxe_lease_time
$fixed_addr_templateess->pxe_lease_time("1234");
#Submit modification
my $response = $session->modify( $fixed_addr_templateess );
```

Infoblox::Session->remove( )

Use this method to remove a DHCP IPv6 fixed address template object from the Infoblox appliance. See Infoblox::Session->remove() for parameters and return values.

To remove a specific object, first use `get()` to retrieve the DHCP IPv6 Fixed Address Template object, and then submit it for removal.

Example

```
#Get DHCP IPv6 fixed address template object through the session
my @retrieved_objs = $session->get(
    object => "Infoblox::DHCP::IPv6FixedAddrTemplate",
    name   => "custom_fa_template",
);
#Find the desired object from the retrieved list.
my $desired_fixed_addr_template = $retrieved_objs[0];
#Submit for removal
my $response = $session->remove( $desired_fixed_addr_template );
```

METHODS

This section describes all the methods that can be used to set and retrieve the attribute values of a DHCP IPv6 Fixed Address Template object.

comment( )

Use this method to set or retrieve the descriptive comment of a DHCP IPv6 Fixed Address Template object.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

Desired comment in string format with a maximum of 256 bytes.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails. If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get comment
my $comment = $fixed_addr_template->comment();
#Modify comment
$fixed_addr_template->comment("Modified DHCP Fixed Address template object comment");
```

domain_name( )

Use this method to set or retrieve the domain_name value of a DHCP IPv6 Fixed Address Template object.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Setting this method to a defined value implicitly sets the override_domain_name method to "true". Setting the parameter to undefined causes the appliance to use the grid default and automatically resets the override_domain_name attribute to "false".

Note that when domain_name is set to a defined value and override_domain_name is set to "false", the last operation takes precedence. Thus the sequence $object->domain_name("testdomain"); $object->override_domain_name("false"); will set override_domain_name to "false", and the sequence $object->override_domain_name("false"); $object->domain_name("testdomain"); will result in override_domain_name="true".

Parameter

The client's domain name in string format.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get domain_name
my $domain_name = $fixed_addr_template->domain_name();
#Modify domain_name
$fixed_addr_template->domain_name("test_domain.com");
```

domain_name_servers( )

Use this method to set or retrieve the domain_name_servers value of a DHCP IPv6 fixed address template.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Setting this method to a defined value implicitly sets the override_domain_name_servers method to "true". Setting the parameter to undefined causes the appliance to use the upper-level default and automatically resets the override_domain_name_servers attribute to "false".

Note that when domain_name_servers is set to a defined value and override_domain_name_servers is set to "false", the last operation takes precedence. Thus the sequence $object->domain_name_servers("2001::1,2001::2"); $object->override_domain_name_servers("false"); will set override_domain_name_servers to "false", and the sequence $object->override_domain_name_servers("false"); $object->domain_name_servers("2001::1,2001::2"); will result in override_domain_name_servers="true".

Parameter

Comma-separated list of the IPv6 addresses of the DNS servers.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get domain_name_servers
my $domain_name_servers = $fixed_addr_template->domain_name_servers();
#Modify domain_name_servers
$fixed_addr_template->domain_name_servers("2001::1,2001::2");
```

extattrs( )

Use this method to set or retrieve the extensible attributes associated with a DHCP IPv6 Fixed Address Template object.

Parameter

Valid value is a hash reference containing the names of extensible attributes and their associated values (Infoblox::Grid::Extattr objects).

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get extattrs
my $ref_extattrs = $fixed_addr_template->extattrs();
#Modify extattrs
$fixed_addr_template->extattrs({ 'Site' => $extattr1, 'Administrator' => $extattr2 });
```

extensible_attributes( )

Use this method to set or retrieve the extensible attributes associated with a DHCP IPv6 Fixed Address Template object.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

For valid values for extensible attributes, see Infoblox::Grid::ExtensibleAttributeDef/Extensible Attribute Values.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get extensible attributes
my $ref_extensible_attributes = $fixed_addr_template->extensible_attributes();
#Modify extensible attributes
$fixed_addr_template->extensible_attributes({ 'Site' => 'Santa Clara', 'Administrator' => [ 'Peter',
'Tom' ] });
```

name( )

Use this method to set or retrieve the name of a DHCP IPv6 Fixed Address Template object.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

Name of the IPv6 fixed address template in string format and with a maximum of 64 characters.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.
If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get name
my $name = $fixed_address_template->name();
#Modify name
$fixed_address_template->name("custom_fa_template");
```

number_of_addresses( )

Use this method to set or retrieve the number of addresses in a DHCP IPv6 fixed address template.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

The number of addresses in this fixed address template.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.
If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get number_of_addresses
my $number_of_addresses = $fixed_address_template-> number_of_addresses();
#Modify number_of_addresses
$fixed_address_template->number_of_addresses("10");
```

offset( )

Use this method to set or retrieve the address offset of an IPv6 fixed address template.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

The address offset of this IPv6 fixed address template. The offset is calculated from the start address of the IPv6 network template that includes the fixed address template. For example, in the IPv6 network 2001:0:0:10::/8, if the offset is set to 10 and number_of_addresses is set to 10, then 10 fixed address objects will be created. The start address will be 2001:0:0:10::A and the end address will be 2001:0:0:10::14.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails. If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get offset
my $offset = $fixed_address_template->offset();
#Modify offset
$fixed_address_template->offset("10");
```

options( )

Use this method to set or retrieve the options of a DHCP IPv6 fixed address template object.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

See Infoblox::DHCP::Option for parameters and methods available for this object.

Setting this method to a defined value implicitly sets the override_options method to "true". Setting the parameter to undefined causes the appliance to use the grid default and automatically resets the override_options attribute to "false".

Note that when the options method is set to a defined value and override_options is set to "false", the last operation takes precedence. Thus the sequence $object->options([$opt]); $object->override_options("false"); will set override_options to "false", and the sequence $object->override_options("false"); $object->options([$opt]); will result in override_options="true".

Parameter

Options describe network configuration settings and various services available on the network. These options occur as variable-length fields at the end of DHCP messages.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails. If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get options
my $options = $fixed_addr_template->options();
#Modify options
$fixed_addr_template->options([$option1, $option2]);
```

override_domain_name( )

The override_domain_name attribute controls whether the domain_name value in the DHCP IPv6 fixed address template is used, instead of the upper-level default.

The override_domain_name attribute can be specified explicitly. It is also set implicitly when domain_name is set to a defined value.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

Set the parameter to "true" to override the upper-level setting for domain_name. Set the parameter to "false" to inherit the upper-level setting for domain_name.

The default value of this parameter is "false".

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get override_domain_name
my $override_domain_name=$fixed_addr_template->override_domain_name( );
#Modify override_domain_name
$fixed_addr_template->override_domain_name("true");
```

override_domain_name_servers( )

The override_domain_name_servers attribute controls whether the domain_name_servers value in the DHCP IPv6 fixed address template is used, instead of the upper-level default.

The override_domain_name_servers attribute can be specified explicitly. It is also set implicitly when domain_name_servers is set to a defined value.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

Set the parameter to "true" to override the upper-level setting for domain_name_servers. Set the parameter to "false" to inherit the upper-level setting for domain_name_servers.

The default value of this parameter is "false".

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get override_domain_name_servers
my $override_domain_name_servers=$fixed_addr_template->override_domain_name_servers( );
#Modify override_domain_name_servers
$fixed_addr_template->override_domain_name_servers("true");
```

override_options( )

The override_options attribute controls whether the options in the DHCP IPv6 fixed address template are used, instead of the upper-level default.

The override_options attribute can be specified explicitly. It is also set implicitly when options are set to defined values.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

Set the parameter to "true" to override the upper-level setting for options. Set the parameter to "false" to inherit the upper-level setting for options.

The default value of this parameter is "false".

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get override_options
my $override_options=$fixed_addr_template->override_options( );
#Modify override_options
$fixed_addr_template->override_options("true");
```

override_preferred_lifetime( )

The override_preferred_lifetime attribute controls whether the preferred_lifetime value in the DHCP IPv6 fixed address template is used, instead of the upper-level default.

The override_preferred_lifetime attribute can be specified explicitly. It is also set implicitly when preferred_lifetime is set to a defined value.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

Set the parameter to "true" to override the upper-level setting for options. Set the parameter to "false" to inherit the upper-level setting for options.

The default value of this parameter is "false".

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Getting override_preferred_lifetime
my $override_preferred_lifetime=$fixed_addr_template->override_preferred_lifetime( );
#Modifying override_preferred_lifetime
$fixed_addr_template->override_preferred_lifetime("true");
```

override_valid_lifetime( )

The override_valid_lifetime attribute controls whether the valid_lifetime value in the DHCP IPv6 fixed address template object is used, instead of the upper-level default.

The override_valid_lifetime attribute can be specified explicitly. It is also set implicitly when valid_lifetime is set to a defined value.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

Set the parameter to "true" to override the upper-level setting for valid_lifetime. Set the parameter to "false" to inherit the upper-level setting for valid_lifetime.

The default value of this parameter is "false".

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Getting override_valid_lifetime
my $override_valid_lifetime=$fixed_addr_template->override_valid_lifetime( );
#Modifying override_valid_lifetime
$fixed_addr_template->override_valid_lifetime("true");
```

preferred_lifetime( )

Use this method to set or retrieve the preferred lifetime of the DHCP IPv6 fixed address template.

Setting this method to a defined value implicitly sets the override_preferred_lifetime method to "true". Setting the parameter to undefined causes the appliance to use the upper-level default and automatically resets the override_preferred_lifetime attribute to "false".

Note that when preferred_lifetime is set to a defined value and override_preferred_lifetime is set to "false", the last operation takes precedence. Thus the sequence $object->preferred_lifetime(20000); $object->override_preferred_lifetime("false"); will set override_preferred_lifetime to "false", and the sequence $object->override_preferred_lifetime("false"); $object->preferred_lifetime(20000); will result in override_preferred_lifetime="true".

Parameter

Preferred lifetime value in seconds. The default value is 27000. Note that the preferred lifetime cannot be greater than the valid lifetime.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get preferred_lifetime
my $preferred_lifetime = $fixed_addr_template->preferred_lifetime();
#Modify preferred_lifetime
$fixed_addr_template->preferred_lifetime(12345);
```

valid_lifetime( )

Use this method to set or retrieve the valid lifetime of the DHCP IPv6 fixed address template.

Setting this method to a defined value implicitly sets the override_valid_lifetime method to "true". Setting the parameter to undefined causes the appliance to use the upper-level default and automatically resets the override_valid_lifetime attribute to "false".

Note that when valid_lifetime is set to a defined value and override_valid_lifetime is set to "false", the last operation takes precedence. Thus the sequence $object->valid_lifetime(20000); $object->override_valid_lifetime("false"); will set override_valid_lifetime to "false", and the sequence $object->override_valid_lifetime("false"); $object->valid_lifetime(20000); will result in override_valid_lifetime="true".

Parameter

Valid lifetime value in seconds. The default value is 43200.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get valid_lifetime
my $valid_lifetime = $fixed_addr_template->valid_lifetime();
#Modify valid_lifetime
$fixed_addr_template->valid_lifetime(123456);
```

SAMPLE CODE

The following sample code demonstrates the different functions that can be applied to an object such as add, get, modify, and remove. Also, this sample includes error handling for the operations.

```
#PROGRAM STARTS: Include all the modules that will be used
use strict;
use Infoblox;

#Create a session to the Infoblox appliance
my $session = Infoblox::Session->new(
    master   => "192.168.1.2",
    username => "admin",
    password => "infoblox"
);

unless ($session) {
    die("Construct session failed: ",
        Infoblox::status_code() . ":" . Infoblox::status_detail());
}
print "Session created successfully\n";

my $option1 = Infoblox::DHCP::Option->new(
    "name"  => "dhcp6.fqdn",
    "value" => "mydomain.com"
);
unless($option1) {
    die("Construct option1 object failed: ",
        Infoblox::status_code() . ":" . Infoblox::status_detail());
}
print "option1 object created successfully\n";

my $option2 = Infoblox::DHCP::Option->new(
    "name"     => "dhcp6.info-refresh-time",
    "value"    => "1000",
);
unless($option2) {
    die("Construct option2 object failed: ",
        Infoblox::status_code() . ":" . Infoblox::status_detail());
}
print "option2 object created successfully\n";
```

#Create a DHCP IPv6 Fixed Address Template object

```
#Create a DHCP IPv6 Fixed Address Template object
my $fixed_addr_template = Infoblox::DHCP::IPv6FixedAddrTemplate->new(
    "name"                  => "custom_fa_template",
    "comment"               => "add IPv6 fixed address template",
    "domain_name"           => "test_domain.com",
    "number_of_addresses"   => 1,
    "offset"                => 10,
    "options"               => [ $option1, $option2 ],
    "preferred_lifetime"    => 1000,
    "valid_lifetime"        => 3600,
);
unless($fixed_addr_template) {
    die("Construct IPv6 Fixed Address template failed: ",
        Infoblox::status_code() . ":" . Infoblox::status_detail());
}
print "DHCP IPv6 Fixed Address template object created successfully\n";

#Verify if the DHCP IPv6 Fixed Address template exists in the Infoblox appliance
my $object = $session->get(
    object => "Infoblox::DHCP::IPv6FixedAddrTemplate",
    name   => "custom_fa_template"
);

unless ($object) {
    print "IPv6 Fixed Address template does not exist on server, safe to add the IPv6 fixed address
template\n";
    $session->add($fixed_addr_template)
        or die("Add IPv6 Fixed Address template failed: ",
        $session->status_code() . ":" . $session->status_detail());
}
print "DHCP IPv6 Fixed Address template added successfully\n";
```

#Search for a DHCP IPv6 Fixed Address Template object

```
#Search for a DHCP IPv6 fixed address template object through the session
my @retrieved_objs = $session->search(
    object => "Infoblox::DHCP::IPv6FixedAddrTemplate",
    name   => "custom_fa_template",
);
my $object = $retrieved_objs[0];

unless ($object) {
    die("Search IPv6 Fixed Address template object failed: ",
        $session->status_code() . ":" . $session->status_detail());
}
print "Search IPv6 Fixed Address template object found at least 1 matching entry\n";
```

#Get and modify a DHCP IPv6 Fixed Address Template object

```
#Get DHCP IPv6 fixed address template object through the session
my @retrieved_objs = $session->get(
    object => "Infoblox::DHCP::IPv6FixedAddrTemplate",
    name   => "custom_fa_template",
);
my $object = $retrieved_objs[0];

unless ($object) {
    die("Get IPv6 Fixed Address template object failed: ",
        $session->status_code() . ":" . $session->status_detail());
}
print "Get IPv6 Fixed Address template object found at least 1 matching entry\n";

#Modify one of the attributes of the obtained IPv6 Fixed Address Template object
$object->valid_lifetime(1234);

#Apply the changes
$session->modify($object)
    or die("Modify IPv6 Fixed Address template failed: ",
    $session->status_code() . ":" . $session->status_detail());

print "DHCP IPv6 Fixed Address template added successfully\n";
```

#Create the IPv6 network template object and include the IPv6 fixed address template

```
my $template = Infoblox::DHCP::Template->new(
    name => "custom_fa_template",
);
```

```
my $network_template = Infoblox::DHCP::IPv6NetworkTemplate->new(
    name     => "custom_network_template",
    comment => "add network",
    fixed_address_templates => [$template]
);

unless($network_template) {
    die("Construct Network template object failed: ",
        Infoblox::status_code() . ":" . Infoblox::status_detail());
}

print "Network template object created successfully\n";

#Add the IPv6 network template object to the Infoblox appliance through a session
my $resp = $session->add($network_template)
    or die("Add Network object failed: ",
    $session->status_code() . ":" . $session->status_detail());

print "Network template object added to Infoblox appliance successfully\n";
```

#Remove an IPv6 fixed address template object

```
#Remove an IPv6 network template object
$session->remove($network_template)
    or die("Remove Network Template template object failed: ",
    $session->status_code() . ":" . $session->status_detail());
print "Network Template object removed successfully \n";

#Get an IPv6 Fixed Address Template object through the session
my @retrieved_objs = $session->get(
    object => "Infoblox::DHCP::IPv6FixedAddrTemplate",
    name   => "custom_fa_template",
);
my $object = $retrieved_objs[0];
unless ($object) {
    die("Get IPv6 Fixed Address template object failed: ",
        $session->status_code() . ":" . $session->status_detail());
}
print "Get IPv6 Fixed Address template object found at least 1 matching entry\n";

#Submit the object for removal
$session->remove($object)
    or die("Remove IPv6 Fixed Address template object failed: ",
    $session->status_code() . ":" . $session->status_detail());
print "IPv6 Fixed Address template object removed successfully \n";

####PROGRAM ENDS####
```

AUTHOR

Infoblox Inc. http://www.infoblox.com/

SEE ALSO

Infoblox::Session, Infoblox::Session->get(), Infoblox::Session->add(), Infoblox::Session->remove(), Infoblox::Session->modify(), Infoblox::DHCP::IPv6NetworkTemplate, Infoblox::DHCP::IPv6FixedAddr, Infoblox::DHCP::IPv6RangeTemplate,Infoblox::DHCP::Template,Infoblox::Session, Infoblox::DHCP::Option

COPYRIGHT

Copyright (c) 2017 Infoblox Inc.

Infoblox::DHCP::IPv6HostAddr - DHCP IPv6 Host Address object.

- NAME
- DESCRIPTION
- CONSTRUCTOR
- SESSION METHODS
  - Infoblox::DNS::Host->ipv6addrs( )
  - Infoblox::Session->get( )
  - Infoblox::Session->modify( )
  - Infoblox::Session->search( )
- METHODS
  - address_type( )
  - configure_for_dhcp( )
  - discovered_data( )
  - discovered_duid( )
  - discovered_name( )
  - discoverer( )
  - discover_now_status( )
  - domain_name( )
  - domain_name_servers( )
  - duid( )
  - first_discovered( )
  - ipv6addr( )
  - ipv6prefix( )
  - ipv6prefix_bits( )
  - last_discovered( )
  - last_queried( )
  - match_client( )
  - ms_ad_user_data( )
  - name( )
  - netbios( )
  - network( )
  - network_component_description( )
  - network_component_ip( )
  - network_component_name( )
  - network_component_port_description( )
  - network_component_port_name( )
  - network_component_port_number( )
  - network_component_type( )
  - options( )
  - os( )
  - override_domain_name( )
  - override_domain_name_servers( )
  - override_options( )
  - override_preferred_lifetime( )
  - override_valid_lifetime( )
  - port_duplex( )
  - port_link_status( )
  - port_speed( )
  - port_status( )
  - port_vlan_description( )
  - port_vlan_name( )
  - port_vlan_number ( )
  - preferred_lifetime( )
  - reserved_interface( )
  - valid_lifetime( )
  - use_for_ea_inheritance( )
- SAMPLE CODE
- AUTHOR
- SEE ALSO
- COPYRIGHT

NAME

Infoblox::DHCP::IPv6HostAddr - DHCP IPv6 Host Address object.

DESCRIPTION

An IPv6 Host Address is an object used to specify IPv6 addresses in a DNS Host object.

CONSTRUCTOR

```
 my $host_addr = Infoblox::DHCP::IPv6HostAddr -> new(
       duid                          => $string,                       # Required
       address_type                  => "ADDRESS" | "PREFIX" | "BOTH", # Optional / Default is "ADDRESS"
       configure_for_dhcp            => "true" | "false",              # Optional / Default is "false"
                                                                       # When both the IP address and DUID
are present, the default value is "true"
                                                                       # Applicable only when adding this
host address to a host
       domain_name                   => $string,                       # Optional / Default is undefined
       domain_name_servers           => "$ipv6addr1,$ipv6addr2,...",   # Optional / Default is undefined
       ipv6addr                      => $ipv6addr,                     # Optional / one of ipv6addr or
ipv6prefix is required
       ipv6prefix                    => $ipv6addr,                     # Optional / one of ipv6addr or
ipv6prefix is required
       ipv6prefix_bits               => $ipv6addr,                     # Optional / Required if ipv6prefix
is specified
       match_client                  => $string,                       # Optional / Default is "DUID"
                                                                       # Possible values are:
                                                                       #  "DUID", "RESERVED",
       options                       => [$option1, $option2, ...]      # Optional / Default is empty
       override_domain_name          => "true" | "false",              # Optional / Default value is
"false"
       override_domain_name_servers  => "true" | "false",              # Optional / Default value is
"false"
       override_options              => "true" | "false",              # Optional / Default value is
"false"
       override_preferred_lifetime   => "true" | "false",              # Optional / Default value is
"false"
       override_valid_lifetime       => "true" | "false",              # Optional / Default value is
"false"
       preferred_lifetime            => $num,                          # Optional / Default is "27000"
       valid_lifetime                => $num,                          # Optional / Default is 43200
       use_for_ea_inheritance        => "true" | "false",              # Optional / Default value is
"false"
       reserved_interface            => $interface,                    # Optional
 );
```

SESSION METHODS

This section describes all the methods in Infoblox::Session module that you can apply to a DHCP IPv6 Host Address object.

Infoblox::DNS::Host->ipv6addrs( )

Use this function to specify the addresses associated with the Host record. See Infoblox::DNS::Host->ipv6addrs() for parameters and return values.

Example

```
       #Construct a Host Address object
       my $h1 = Infoblox::DHCP::HostAddr->new(
                                          duid     => '0cc084d303010003',
                                          ipv6addr => '2001::4',
       );

       # Configure an address in the Host record object
       $host->ipv6addrs([$h1]);
```

Infoblox::Session->get( )

Use this method to retrieve all the matching objects from the Infoblox appliance. See Infoblox::Session->get() for parameters and return values.

'discover_now_status' will not be returned by default, use 'return_methods' to request it if needed.

Key References

```
       Apply the following attributes to get a specific DHCP IPv6 Host Address object:

        comment                              - Optional. User-specified comment for the host address.
        duid                                 - Optional. DUID of the host address.
        ipv6addr                             - Optional. IPv6 address of the host address.
        network                              - Optional. Network address of the host address.
        network_view                         - Optional. Network view in which the host address is
      located. By default, all network views are searched.
        extensible_attributes                - Optional. A hash reference containing extensible
      attributes.

        discovered_duid                      - Optional. Discovered DHCPv6 UID.
```

```
  discovered_name                       - Optional. Discovered name.
  discoverer                            - Optional. Discoverer name.
  first_discovered                      - Optional. Time of the first discovery.
  last_discovered                       - Optional. Time of the last discovery.
  netbios                               - Optional. Discovered NetBIOS name.
  network_component_description         - Optional. Discovered network component description.
  network_component_ip                  - Optional. Discovered network component IP address.
  network_component_name                - Optional. Discovered network component name.
  network_component_port_description    - Optional. Discovered network component port description.
  network_component_port_name           - Optional. Discovered network component port name.
  network_component_port_number         - Optional. Discovered network component port number.
  network_component_type                - Optional. Discovered network component type.
  os                                    - Optional. Discovered OS.
  port_duplex                           - Optional. Discovered port duplex settings.
  port_link_status                      - Optional. Discovered port link status.
  port_speed                            - Optional. Discovered port speed settings.
  port_status                           - Optional. Discovered port status settings.
  port_vlan_description                 - Optional. Discovered port VLAN description.
  port_vlan_name                        - Optional. Discovered port VLAN name.
  port_vlan_number                      - Optional. Discovered port VLAN number.
```

Examples

```
#Get DHCP IPv6 Host Address object through the session
my @retrieved_objs = $session->get(
    object => "Infoblox::DHCP::IPv6HostAddr",
    duid   => "0001",
);
my @retrieved_objs = $session->get(
    object       => "Infoblox::DHCP::IPv6HostAddr",
    ipv6addr     => "2001::1",
);
```

Infoblox::Session->modify( )

Use this method to modify a DHCP IPv6 Host Address object in the Infoblox appliance. See Infoblox::Session->modify() for parameters and return values.

Example

```
#Use method to modify the preferred_lifetime
$ipv6ha->preferred_lifetime("1234567");
#Submit modification
my $response = $session->modify( $ipv6ha );
```

Infoblox::Session->search( )

Use this method to search for DHCP IPv6 Host Address object and matching Host object with MAC address in the Infoblox appliance. See Infoblox::Session->search() for parameters and return values.

'discover_now_status' will not be returned by default, use 'return_methods' to request it if needed.

Key References

```
  Apply the following attributes to search for a specific DHCP IPv6 Host Address object:

  comment                               - Optional. User-specified comment for the host address.
  duid                                  - Optional. DUID of the host address.
  ipv6addr                              - Optional. IPv6 address of the host address.
  network                               - Optional. Network address of the host address.
  network_view                          - Optional. Network view in which the host address is
located. By default, all network views are searched.

  discovered_duid                       - Optional. Discovered DHCPv6 UID.
  discovered_name                       - Optional. Discovered name (regular expression).
  discoverer                            - Optional. Discoverer name (regular expression).
  first_discovered                      - Optional. Time of the first discovery.
  last_discovered                       - Optional. Time of the last discovery.
  netbios                               - Optional. Discovered NetBIOS name (regular expression).
  network_component_description         - Optional. Discovered network component description (regular
expression).
  network_component_ip                  - Optional. Discovered network component IP address (regular
expression).
  network_component_name                - Optional. Discovered network component name (regular
expression).
  network_component_port_description    - Optional. Discovered network component port description
(regular expression).
  network_component_port_name           - Optional. Discovered network component port name (regular
expression).
  network_component_port_number         - Optional. Discovered network component port number.
  network_component_type                - Optional. Discovered network component type (regular
expression).
  os                                    - Optional. Discovered OS  (regular expression).
  port_duplex                           - Optional. Discovered port duplex settings.
  port_link_status                      - Optional. Discovered port link status.
  port_speed                            - Optional. Discovered port speed settings
  port_status                           - Optional. Discovered port status settings.
```

```
  port_vlan_description                - Optional. Discovered port VLAN description (regular
expression).
  port_vlan_name                       - Optional. Discovered port VLAN name (regular expression).
  port_vlan_number                     - Optional. Discovered port VLAN numbe.
```

Examples

```
 # Search for all DHCP IPv6 Host Address objects that are specified in the network "2001::/24" in the
"default" network view.
 my @retrieved_objs = $session->search(
     object       => "Infoblox::DHCP::IPv6HostAddr",
     duid         => "0001",
 );
```

METHODS

This section describes all the methods that you can use to set and retrieve the attribute values of a DHCP IPv6 Host Address object.

address_type( )

Use this method to set or retrieve the address type of a DHCP IPv6 Host Address object.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

Valid values are "ADDRESS", "PREFIX", or "BOTH". When the address type is "ADDRESS", a value for the 'ipv6addr' member is required. When the address type is "PREFIX", values for 'ipv6prefix' and 'ipv6prefix_bits' are required. When the address type is "BOTH", values for 'ipv6addr', 'ipv6prefix', and 'ipv6prefix_bits' are all required.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get address_type
my $address_type = $ipv6ha->address_type();
#Modify address_type
$ipv6ha->address_type("PREFIX");
```

configure_for_dhcp( )

Use this method to set or retrieve the configure_for_dhcp flag of a DHCP IPv6 Host Address object.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

Specify "true" to set the configure_for_dhcp flag or "false" to deactivate/unset it. The default value is "false".

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get configure_for_dhcp
my $configure_for_dhcp = $ipv6ha->configure_for_dhcp();
#Modify configure_for_dhcp
$ipv6ha->configure_for_dhcp("true");
```

discovered_data( )

Use this method to retrieve the discovered data of a DHCP IPv6 Host Address object. This is a read-only attribute.

Omit the parameter to retrieve the attribute value.

Parameter

None

Returns

The valid return value is an Infoblox::Grid::Discovery::Data object.

Example

```
#Get discovered_data value
my $discovered_data = $object->discovered_data();
```

discovered_duid( )

Use this method to retrieve the discovered DHCPv6 UID (DUID) of a DHCP IPv6 Host Address object.

Parameter

none

Returns

The method returns the discovered name.

Example

```
#Get discovered_duid
my $discovered_duid = $ipv6ha->discovered_duid();
```

discovered_name( )

Use this method to retrieve the discovered name of a DHCP IPv6 Host Address object.

Parameter

none

Returns

The method returns the discovered name.

Example

```
#Get discovered_name
my $discovered_name = $ipv6ha->discovered_name();
```

discoverer( )

Use this method to retrieve the discoverer of a DHCP IPv6 Host Address object.

Parameter

none

Returns

The method returns the discoverer value.

Example

```
#Get discoverer
my $discoverer = $ipv6ha->discoverer();
```

discover_now_status( )

Use this method to retrieve the current discovery status. This attribute is read-only.

'discover_now_status' will not be returned by default, use 'return_methods' to request it if needed.

Parameter

None.

Returns

The method returns the attribute value that can be 'COMPLETE', 'FAILED', 'NONE', 'PENDING' or 'RUNNING'.

Example

```
#Get discover_now_status
my $discover_now_status = $ipv6ha->discover_now_status();
```

domain_name( )

Use this method to set or retrieve the domain_name value of a DHCP IPv6 host address object.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Setting this method to a defined value implicitly sets the override_domain_name method to "true". Setting the parameter to undefined causes the appliance to use the upper-level default and automatically resets the override_domain_name attribute to "false".

Note that when domain_name is set to a defined value and override_domain_name is set to "false", the last operation takes precedence. Thus the sequence $object->domain_name("testdomain"); $object->override_domain_name("false"); will set override_domain_name to "false", and the sequence $object->override_domain_name("false"); $object->domain_name("testdomain"); will set override_domain_name to "true".

Parameter

The client's domain name in string format.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get domain_name
my $domain_name = $host_addr->domain_name();
#Modify domain_name
$host_addr->domain_name("test_domain.com");
```

domain_name_servers( )

Use this method to set or retrieve the domain_name_servers value of a DHCP IPv6 host address object.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Setting this method to a defined value implicitly sets the override_domain_name_servers method to "true". Setting the parameter to undefined causes the appliance to use the upper-level default and automatically resets the override_domain_name_servers attribute to "false".

Note that when domain_name_servers is set to a defined value and override_domain_name_servers is set to "false", the last operation takes precedence. Thus the sequence $object->domain_name_servers("2001::1,2001::2"); $object->override_domain_name_servers("false"); will set override_domain_name_servers to "false", and the sequence $object->override_domain_name_servers("false"); $object->domain_name_servers("2001::1,2001::2"); will set in override_domain_name_servers to "true".

Parameter

The comma-separated list of domain name server addresses in IPv6 address format.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get domain_name_servers
my $domain_name_servers = $host_addr->domain_name_servers();
#Modify domain_name_servers
$host_addr->domain_name_servers("2001::1,2001::2");
```

duid( )

Use this method to set or retrieve the DHCPv6 Unique Identifier (DUID) of the IPv6 Host Address object.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

The DUID of the matching client in hexadecimal format.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get duid
my $duid = $ipv6ha->duid();
#Modify duid
$ipv6ha->duid("1c2edfaa");
```

first_discovered( )

Use this method to retrieve the date and time that the DHCP IPv6 Host Address object was first discovered.

Parameter

none

Returns

The method returns the date and time that the DHCP IPv6 Host Address object was first discovered.

Example

```
#Get first_discovered
my $first_discovered = $ipv6ha->first_discovered();
```

ipv6addr( )

Use this method to set or retrieve the the IPv6 address of the IPv6 Host Address object.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

An IPv6 address is a 128-bit number represented as eight groups of four hexadecimal digits (i.e. two octets), with each group separated by a `colon(:)` (example: 2001:0db8:85a3:0000:0000:8a2e:0370:7334). Leading zeroes and groups of consecutive zeros within the value can be omitted to produce a more compact representation (example: 2001::1, ::1), and dotted-quad notations can be used to express IPv4-mapped IPv6 addresses (example: ::ffff:192.0.2.128).

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get ipv6addr
my ipv6addr = $ipv6ha->ipv6addr();
#Modify ipv6addr
$ipv6ha->ipv6addr("aaaa:1111:bbbb:2222:cccc:3333:dddd:5555");
```

ipv6prefix( )

Use this method to set or retrieve the IPv6 prefix of a DHCP IPv6 Host Address object. The prefix must belong to the list of global prefixes defined in the Infoblox::Grid::DHCP object.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

IPv6 prefix from the list of global IPv6 prefixes.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails. ls.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get ipv6prefix
my $ipv6prefix = $ipv6ha->ipv6prefix();
#Modify ipv6prefix
$ipv6ha->ipv6prefix("2001::");
```

ipv6prefix_bits( )

Use this method to set or retrieve the prefix bits for the DHCP IPv6 Host Address.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

An unsigned integer between 2 and 128.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get ipv6prefix_bits
my $ipv6prefix_bits = $ipv6ha->ipv6prefix_bits();
#Modify ipv6prefix_bits
$ipv6ha->ipv6prefix_bits(32);
```

last_discovered( )

Use this method to retrieve the time this object was last seen by a discovery job.

The default value for this field is empty.

Parameter

N/A

Returns

The method returns the attribute value.

Example

```
#Get last_discovered
my $last_discovered = $ipv6ha->last_discovered();
```

last_queried( )

Use this method to retrieve the time when the associated record was last queried. This is a read-only attribute.

Parameter

None

Returns

The method returns the attribute value. The number of seconds that have elapsed since January 1st, 1970 UTC.

Example

```
#Get last_queried
my $last_queried = $ipv6ha->last_queried();
```

match_client( )

Use this method to set or retrieve the match_client value of a DHCP Host Address object.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

```
match_client "DUID": The fixed IP address is leased to the matching DUID.
```

Returns

```
If you specified a parameter, the method returns true when the modification succeeds, and returns
false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.
```

Example

```
#Get match_client
my $match_client = $ipv6ha->match_client();
#Modify match_client
$ipv6ha->match_client("DUID");
```

ms_ad_user_data( )

Use this method to retrieve Microsoft Active Directory users related information. This is a read-only attribute.

Parameter

None

Returns

The valid return value is an Infoblox::Grid::MSServer::AdUser::Data object.

Example

```
#Get ms_ad_user_data
my $ms_ad_user_data = $host_addr->ms_ad_user_data();
```

name( )

Use this method to set or retrieve the name of a DHCP IPv6 Host Address object. The same name can be used for multiple host addresses.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

Specify a name in string format.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get the name
my $name = $ipv6ha->name();
#Modify the name
$ipv6ha->name("pop-server");
```

netbios( )

Use this method to retrieve the name in the NetBIOS reply that responded to a NetBIOS query.

Returns

The method returns the attribute value.

Example

```
#Get NetBIOS
my $netbios = $ipv6ha->netbios();
```

network( )

Use this method to retrieve the network value of a DHCP IPv6 Host Address object. This is a read-only attribute.

Omit the parameter to retrieve the attribute value.

Parameter

None

Returns

The method returns the attribute value.

Example

```
#Get network value
my $value = $object->network();
```

network_component_description( )

Use this method to retrieve the description of the network device that is connected to the DHCP IPv6 Host Address object.

Parameter

none

Returns

The method returns the network device description.

Example

```
#Get network_component_description
my $network_component_description = $ipv6ha->network_component_description();
```

network_component_ip( )

Use this method to retrieve the IP address of the network device that is connected to the DHCP IPv6 Host Address object.

Parameter

none

Returns

The method returns the IP address of the network device.

Example

```
#Get network_component_ip
my $network_component_ip = $ipv6ha->network_component_ip();
```

network_component_name( )

Use this method to retrieve the name of the network device that is connected to the DHCP IPv6 Host Address object.

Parameter

none

Returns

The method returns the network device name.

Example

```
#Get network_component_name
my $network_component_name = $ipv6ha->network_component_name();
```

network_component_port_description( )

Use this method to retrieve the description of the network device port that is connected to the DHCP IPv6 Host Address object.

Parameter

none

Returns

The method returns the description of the port on the network device.

Example

```
#Get network_component_port_description
my $network_component_port_description = $ipv6ha->network_component_port_description();
```

network_component_port_name( )

Use this method to retrieve the name of the network device port that is connected to the DHCP IPv6 Host Address object.

Parameter

none

Returns

The method returns the name of the port on the network device.

Example

```
#Get network_component_port_name
my $network_component_port_name = $ipv6ha->network_component_port_name();
```

network_component_port_number( )

Use this method to retrieve the number of the network device port that is connected to the DHCP IPv6 Host Address object.

Parameter

none

Returns

The method returns the network device port number.

Example

```
#Get network_component_port_number
my $network_component_port_number = $ipv6ha->network_component_port_number();
```

network_component_type( )

Use this method to retrieve the type of the network device that is connected to the DHCP IPv6 Host Address object.

Parameter

none

Returns

The method returns the network device type.

Example

```
#Get network_component_type
my $network_component_type = $ipv6ha->network_component_type();
```

options( )

Use this method to set or retrieve the options of a DHCP IPv6 Host Address object.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Setting this method to a defined value implicitly sets the override_options method to "true". Setting the parameter to undefined causes the appliance to use the grid default and automatically resets the override_options attribute to "false".

Note that when the options method is set to a defined value and override_options is set to "false", the last operation takes precedence. Thus the sequence $object->options([$opt]); $object->override_options("false"); will set override_options to "false", and the sequence $object->override_options("false"); $object->options([$opt]); will set override_options to "true".

Parameter

The valid value is an array reference that contains Infoblox::DHCP::Option object(s).

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get options
my $options = $ipv6ha->options();
#Modify options
my $option1 = Infoblox::DHCP::Option->new(
    name   => "dhcp6.rapid-commit",
    value  => "true"
);
$ipv6ha->options([$option1);
```

os( )

Use this method to retrieve the operating system associated with this host address.

Returns

The method returns the attribute value.

Example

```
#Get OS
my $os = $ipv6ha->os();
```

override_domain_name( )

The override_domain_name attribute controls whether the domain_name value in the DHCP IPv6 host address is used, instead of the upper-level default.

The override_domain_name attribute can be specified explicitly. It is also set implicitly when domain_name is set to a defined value.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

Set the parameter to "true" to override the upper-level setting for domain_name. Set the parameter to "false" to inherit the upper-level setting for domain_name.

The default value of this parameter is "false".

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get override_domain_name
my $override_domain_name=$ipv6ha->override_domain_name( );
#Modify override_domain_name
$ipv6ha->override_domain_name("true");
```

override_domain_name_servers( )

The override_domain_name_servers attribute controls whether the domain_name_servers value in the DHCP IPv6 host address is used, instead of the upper-level default.

The override_domain_name_servers attribute can be specified explicitly. It is also set implicitly when domain_name_servers is set to a defined value.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

Set the parameter to "true" to override the upper-level setting for domain_name_servers. Set the parameter to "false" to inherit the upper-level setting for domain_name_servers.

The default value of this parameter is "false".

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get override_domain_name_servers
my $override_domain_name_servers=$ipv6ha->override_domain_name_servers( );
#Modify override_domain_name_servers
$ipv6ha->override_domain_name_servers("true");
```

override_options( )

The override_options attribute controls whether the options in the DHCP IPv6 host address are used, instead of the upper-level default.

The override_options attribute can be specified explicitly. It is also set implicitly when options are set to defined values.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

Set the parameter to "true" to override the upper-level setting for valid_lifetime. Set the parameter to "false" to inherit the upper-level setting for valid_lifetime.

The default value of this parameter is "false".

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get override_options
my $override_options=$host_addr->override_options( );
#Modify override_options
$host_addr->override_options("true");
```

override_preferred_lifetime( )

The override_preferred_lifetime attribute controls whether the preferred_lifetime value in the DHCP IPv6 host address is used, instead of the upper-level default.

The override_preferred_lifetime attribute can be specified explicitly. It is also set implicitly when preferred_lifetime is set to a defined value.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

Set the parameter to "true" to override the upper-level setting for valid_lifetime. Set the parameter to "false" to inherit the upper-level setting for valid_lifetime.

The default value of this parameter is "false".

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get override_preferred_lifetime
my $override_preferred_lifetime=$host_addr->override_preferred_lifetime( );
#Modify override_preferred_lifetime
$host_addr->override_preferred_lifetime("true");
```

override_valid_lifetime( )

The override_valid_lifetime attribute controls whether the valid_lifetime value in the DHCP IPv6 host address is used, instead of the upper-level default.

The override_valid_lifetime attribute can be specified explicitly. It is also set implicitly when valid_lifetime is set to a defined value.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

Set the parameter to "true" to override the upper-level setting for valid_lifetime. Set the parameter to "false" to inherit the upper-level setting for valid_lifetime.

The default value of this parameter is "false".

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get override_valid_lifetime
my $override_valid_lifetime=$host_addr->override_valid_lifetime( );
#Modify override_valid_lifetime
$host_addr->override_valid_lifetime("true");
```

port_duplex( )

Use this method to retrieve the duplex setting of the network device port that is connected to the DHCP IPv6 Host Address object.

Parameter

none

Returns

The method returns the port duplex setting. The returned value, if any, is one of the following:

```
Full
Half
```

Example

```
#Get port_duplex
my $port_duplex = $ipv6ha->port_duplex();
```

port_link_status( )

Use this method to retrieve the link status of the network device port that is connected to the DHCP IPv6 Host Address object.

Parameter

none

Returns

This method returns the link status of the network device port. The returned value, if any, is one of the following:

```
Connected
Not Connected
Unknown
```

Example

```
#Get port_link_status
my $port_link_status = $ipv6ha->port_link_status();
```

port_speed( )

Use this method to retrieve the speed of the network device port that is connected to the DHCP IPv6 Host Address object.

Parameter

none

Returns

The method returns the network device port speed. The returned value, if any, is one of the following:

```
10M
100M
1G
10G
100G
Unknown
```

Example

```
#Get port_speed
my $port_speed = $ipv6ha->port_speed();
```

port_status( )

Use this method to retrieve the status of the network device port that is connected to the DHCP IPv6 Host Address object.

Parameter

none

Returns

The method returns the network device port status. The returned value, if any, is one of the following:

```
Up
Down
Unknown
```

Example

```
#Get port_status
my $port_status = $ipv6ha->port_status();
```

port_vlan_description( )

Use this method to retrieve the VLAN description of the network device port that is connected to the DHCP IPv6 Host Address object.

Parameter

none

Returns

The method returns the VLAN description of the network device port.

Example

```
#Get port_vlan_description
my $port_vlan_description = $ipv6ha->port_vlan_description();
```

port_vlan_name( )

Use this method to retrieve the VLAN name of the network device port that is connected to the DHCP IPv6 Host Address object.

Parameter

none

Returns

The method returns the network device port VLAN name.

Example

```
#Get port_vlan_name
my $port_vlan_name = $ipv6ha->port_vlan_name();
```

port_vlan_number ( )

Use this method to retrieve the VLAN number of the network device port that is connected to the DHCP IPv6 Host Address object.

Parameter

none

Returns

The method returns the port VLAN number.

Example

```
#Get port_vlan_number
my $port_vlan_number  = $ipv6ha->port_vlan_number ();
```

preferred_lifetime( )

Use this method to set or retrieve the preferred_lifetime value of a DHCP IPv6 Host Address object.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Setting this method to a defined value implicitly sets the override_preferred_lifetime method to "true". Setting the parameter to undefined causes the appliance to use the upper-level default and automatically resets the override_preferred_lifetime attribute to "false".

Note that when preferred_lifetime is set to a defined value and override_preferred_lifetime is set to "false", the last operation takes precedence. Thus the sequence $object->preferred_lifetime(20000); $object->override_preferred_lifetime("false"); will set override_preferred_lifetime to "false", and the sequence $object->override_preferred_lifetime("false"); $object->preferred_lifetime(20000); will set override_preferred_lifetime to "true".

Parameter

A 32-bit integer (valid range from 1 to 4294967295) that represents the lifetime, in seconds. The default value is 27000.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get preferred_lifetime
my $preferred_lifetime = $ipv6ha->preferred_lifetime();
#Modify preferred_lifetime
$ipv6ha->preferred_lifetime("369");
```

reserved_interface( )

Use this method to set or retrieve the reserved discovery interface.

Parameter

Valid value is an Infoblox::Grid::Discovery::DeviceInterface object.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get reserved_interface
my $reserved_interface = $ipv6ha->reserved_interface();
#Modify reserved_interface
$ipv6ha->reserved_interface($interface);
```

valid_lifetime( )

Use this method to set or retrieve the valid_lifetime value of a DHCP IPv6 Host Address object.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

A 32-bit integer, from 1 to 4294967295, that represents the lifetime, in seconds. The default value is 43200.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get valid_lifetime
my $valid_lifetime = $ipv6ha->valid_lifetime();
#Modify valid_lifetime
$ipv6ha->valid_lifetime("36900");
```

use_for_ea_inheritance( )

Use this method to set or retrieve the flag that indicates if this host address is used for inheritable extensible attributes.

Parameter

Specify "true" to enable the flag or "false" to disable it. The default value is "false".

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get use_for_ea_inheritance
my $use_for_ea_inheritance = $ipv6ha->use_for_ea_inheritance();
#Modify use_for_ea_inheritance
$ipv6ha->use_for_ea_inheritance("true");
```

SAMPLE CODE

The following sample code demonstrates the different functions that can be applied to an object such as search and modify. This sample also includes error handling for the operations.

#Preparation prior to a DHCP IPv6 Host Address object insertion

```
#PROGRAM STARTS: Include all the modules that will be used, note the hostaddress option
use strict;
use Infoblox qw (:hostaddress);

#Create a session to the Infoblox appliance
```

```
my $session = Infoblox::Session->new(
              master   => "192.168.1.2", #appliance host ip
              username => "admin",       #appliance user login
              password => "infoblox"     #appliance password
);

unless ($session) {
       die("Construct session failed: ",
              Infoblox::status_code() . ":" . Infoblox::status_detail());
}
print "Session created successfully\n";

#Create an IPv6 network object
my $network = Infoblox::DHCP::IPv6Network->new(
                                          "network" => "2001::/64",
                                         );
unless($network) {
     die("Construct IPv6 Network object failed: ",
           Infoblox::status_code() . ":" . Infoblox::status_detail());
}
print "IPv6 Network object created successfully\n";

$session->add($network)
    or die("Add network failed: ",
           $session->status_code() . ":" . $session->status_detail());
print "Network added successfully\n";

#Create a zone object
my $zone = Infoblox::DNS::Zone->new(
                                    name => "test.com",
                                   );
unless($zone) {
     die("Construct Zone object failed: ",
           Infoblox::status_code() . ":" . Infoblox::status_detail());
}
print "Zone object created successfully\n";

$session->add($zone)
    or die("Add zone failed: ",
           $session->status_code() . ":" . $session->status_detail());
print "Zone added successfully\n";

my $option1 = Infoblox::DHCP::Option->new(
    "name"  => "dhcp6.fqdn",
    "value" => "mydomain.com"
);
unless($option1) {
    die("Construct option1 object failed: ",
        Infoblox::status_code() . ":" . Infoblox::status_detail());
}
print "option1 object created successfully\n";

my $option2 = Infoblox::DHCP::Option->new(
    "name"     => "dhcp6.info-refresh-time",
    "value"    => "1000",
);
unless($option2) {
    die("Construct option2 object failed: ",
        Infoblox::status_code() . ":" . Infoblox::status_detail());
}
print "option2 object created successfully\n";
```

#Create a DHCP IPv6 Host address object

```
#Create a DHCP IPv6 Host Address object
my $host_addr = Infoblox::DHCP::IPv6HostAddr->new(
       "ipv6addr"                    => "2001::4",
       "duid"                        => "00:00:00:11:11:11",
       "domain_name"                 => "foo.com",
       "domain_name_servers"         => "2002::1,2002::2",
       "preferred_lifetime"          => 1234,
       "override_preferred_lifetime" => "true",
       "valid_lifetime"              => 123456,
       "override_valid_lifetime"     => "true",
       "options"                     => [ $option1 , $option2 ],
       );
```

```
unless($host_addr) {
     die("Construct IPv6 host address failed: ",
           Infoblox::status_code() . ":" . Infoblox::status_detail());
}
print "DHCP IPv6 Host address object created successfully\n";

my $host = Infoblox::DNS::Host->new(
                                    name         => "host.test.com",
                                    ipv6addrs    => [$host_addr],
                                   );

unless($host) {
     die("Construct IPv6 host record failed: ",
           Infoblox::status_code() . ":" . Infoblox::status_detail());
}

$session->add($host)
    or die("Add host failed: ",
           $session->status_code() . ":" . $session->status_detail());
print "Host added successfully\n";
```

#Get and modify a DHCP IPv6 Host Address object

```
#Get DHCP IPv6 Host Address object through the session
my @retrieved_objs = $session->get(
    object => "Infoblox::DHCP::IPv6HostAddr",
    duid    => "00:00:00:11:11:11",
);
my $object = $retrieved_objs[0];
unless ($object) {
    die("Get IPv6 Host Address object failed: ",
        $session->status_code() . ":" . $session->status_detail());
}
print "Get IPv6 Host Address object found at least 1 matching entry\n";

#Modify one of the attributes of the obtained IPv6 Host Address object
$object->preferred_lifetime(1235);

#Apply the changes
$session->modify($object)
    or die("Modify IPv6 Host Address object failed: ",
           $session->status_code() . ":" . $session->status_detail());
print "IPv6 Host Address object modified successfully \n";

#Verify that this was changed also when retrieving this via the host record
my @retrieved_objs = $session->get(
    object => "Infoblox::DNS::Host",
    name => "host.test.com",
);
my $object = $retrieved_objs[0];
unless ($object) {
    die("Get Host record failed: ",
        $session->status_code() . ":" . $session->status_detail());
}
print "Get host record object found at least 1 matching entry\n";

my $hostaddress = @{$object->ipv6addrs()}[0];

unless ($hostaddress->preferred_lifetime() == 1235) {
    die("The modification of host address has failed: ",
        $session->status_code() . ":" . $session->status_detail());
}
print "The change to preferred lifetime has succeeded correctly\n";

#The host address retrieved this way can also be modified without submitting the full host record
$hostaddress->preferred_lifetime(1111);

#Apply the changes
$session->modify($hostaddress)
    or die("Modify IPv6 Host Address object failed: ",
           $session->status_code() . ":" . $session->status_detail());
print "IPv6 Host Address object modified successfully \n";

#Verify the change was submitted successfully
@retrieved_objs = $session->get(
    object => "Infoblox::DHCP::IPv6HostAddr",
    duid    => "00:00:00:11:11:11",
```

```
);
$object = $retrieved_objs[0];
unless ($object) {
    die("Get IPv6 Host Address object failed: ",
        $session->status_code() . ":" . $session->status_detail());
}
print "Get IPv6 Host Address object found at least 1 matching entry\n";

unless ($hostaddress->preferred_lifetime() == 1111) {
    die("The modification of host address has failed: ",
        $session->status_code() . ":" . $session->status_detail());
}

#Submit the Host object for removal
$session->remove($host)
    or die("Remove Host object failed: ",
        $session->status_code() . ":" . $session->status_detail());
print "Host object removed successfully \n";

#Submit the Zone object for removal
$session->remove($zone)
    or die("Remove Zone object failed: ",
        $session->status_code() . ":" . $session->status_detail());
print "Zone object removed successfully \n";

#Submit the Network object for removal
$session->remove($network)
    or die("Remove Network object failed: ",
        $session->status_code() . ":" . $session->status_detail());
print "Network object removed successfully \n";
```

AUTHOR

Infoblox Inc. http://www.infoblox.com/

SEE ALSO

Infoblox::DHCP::IPv6Network, Infoblox::DHCP::View, Infoblox::DHCP::IPv6Range, Infoblox::Session->get(), Infoblox::Session->modify(), Infoblox::Session->search(), Infoblox::Session, Infoblox::Grid::Discovery::Data, Infoblox::Grid::MSServer::AdUser::Data

---

COPYRIGHT

Copyright (c) 2017 Infoblox Inc.

Infoblox::DHCP::IPv6Network - DHCP IPv6 Network object

- NAME
- DESCRIPTION
- CONSTRUCTOR
- SESSION METHODS
  - Infoblox::Session->add( )
  - Infoblox::Session->get( )
  - Infoblox::Session->modify( )
  - Infoblox::Session->remove( )
  - Infoblox::Session->search( )
- METHODS
  - auto_create_reversezone( )
  - comment( )
  - cloud_info( )
  - ddns_domainname( )
  - ddns_enable_option_fqdn( )
  - ddns_generate_hostname( )
  - ddns_server_always_updates( )
  - ddns_ttl( )
  - disable( )
  - discovered_bridge_domain( )
  - discovered_tenant( )
  - discovery_basic_poll_setting( )
  - discovery_member( )
  - discover_now_status( )
  - discovery_blackout_setting( )
  - discovery_engine_type( )
  - domain_name( )
  - domain_name_servers( )
  - enable_ddns( )
  - enable_discovery( )
  - enable_ifmap_publishing( )
  - enable_immediate_discovery( )
  - endpoint_sources( )
  - extattrs( )
  - extensible_attributes( )
  - members( )
  - mgm_private( )
  - mgm_private_overridable( )
  - ms_ad_user_data( )
  - network( )
  - network_container( )
  - network_view( )
  - next_available_ip( )
  - next_available_network( )
  - options( )
  - override_ddns_domainname( )
  - override_ddns_enable_option_fqdn( )
  - override_ddns_generate_hostname( )
  - override_ddns_ttl( )
  - override_domain_name( )
  - override_domain_name_servers( )
  - override_blackout_setting( )
  - override_enable_ddns( )
  - override_enable_ifmap_publishing( )
  - override_options( )
  - override_preferred_lifetime( )
  - override_recycle_leases( )
  - override_mgm_private( )
  - override_subscribe_settings( )
  - override_update_dns_on_lease_renewal( )
  - override_valid_lifetime( )
  - preferred_lifetime ( )
  - port_control_blackout_setting
  - recycle_leases( )
  - restart_if_needed( )
  - rir( )
  - rir_organization( )
  - rir_registration_action( )
  - rir_registration_status( )
  - last_rir_registration_update_sent( )
  - last_rir_registration_update_status( )

- same_port_control_discovery_blackout( )
- send_rir_request( )
- delete_reason( )
- shared_network_name( )
- subscribe_settings( )
- template( )
- unmanaged( )
- update_dns_on_lease_renewal( )
- valid_lifetime ( )
- zone_associations( )

- SAMPLE CODE
- AUTHOR
- SEE ALSO
- COPYRIGHT

NAME

Infoblox::DHCP::IPv6Network - DHCP IPv6 Network object

DESCRIPTION

When DHCP services are configured on an appliance, a network that it serves must be defined. After a network is created, you can then create all the subnetworks individually or create a parent network that encompasses all the subnetworks. This object allows the user to create IPv6 networks. For IPv4 networks, see Infoblox::DHCP::Network object.

CONSTRUCTOR

```
  my $network = Infoblox::DHCP::IPv6Network->new(
                network                             => $ipv6_subnet/prefix,             #Required,
when creating an IPv6 network
                auto_create_reversezone             => "true" | "false",                #Optional /
Default is undefined
                comment                             => $string| undef,                  #Optional /
Default is undefined
                ddns_domainname                     => $string| undef,                  #Optional /
Default is undefined
                ddns_enable_option_fqdn             => "true" | "false" ,               #Optional /
Default is "false"
                ddns_generate_hostname              => "true" | "false" | undef,        #Optional /
Default is undefined
                ddns_server_always_updates          => "true" | "false",                #Optional /
Default is "true"
                ddns_ttl                            => $num,                            #Optional /
Default is "0"
                disable                             => "true" | "false",                #Optional /
Default is "false"
                domain_name                         => $string,                         #Optional /
Default is undefined
                domain_name_servers                 => "$ipv6addr,$ipv6addr,...",       #Optional /
Default is undefined
                enable_ddns                         => "true" | "false" | undef,        #Optional /
Default is undefined
                enable_ifmap_publishing             => "true" | "false",                #Optional /
Default is undefined
                extattrs                            => { $string => $extattr, ... },    #Required if
'rir_organization' attribute is defined
                                                                                        Otherwise is
optional
                extensible_attributes               => { $string => $string | $num, $string => [
$string | $num, ... ], ... }, #Required if 'rir_organization' attribute is defined
#Otherwise is optional
                members                             => [ $Member1,$Member2,...],        #Optional /
Default is empty
                mgm_private                         => "true" | "false",                #Optional /
Default is "false"
                network_container                   => $ipv6_subnet/prefix,             #Optional /
Default is "/"
                network_view                        => $NetworkView,                    #Optional /
Default is "default" network view
                options                             => [$Option1,$Option2,...],         #Optional /
Default is empty list
                override_ddns_domainname            => "true" | "false",                #Optional /
Default value is "false"
                override_ddns_enable_option_fqdn    => "true" | "false",                #Optional /
Default value is "false"
                override_ddns_generate_hostname     => "true" | "false",                #Optional /
Default value is "false"
```

```
                override_ddns_ttl                   => "true" | "false",                #Optional /
Default value is "false"
                override_domain_name                => "true" | "false",                #Optional /
Default value is "false"
                override_domain_name_servers        => "true" | "false",                #Optional /
Default value is "false"
                override_mgm_private                => "true" | "false",                #Optional /
Default value is "false"
                override_enable_ddns                => "true" | "false",                #Optional /
Default value is "false"
                override_enable_ifmap_publishing    => "true" | "false",                #Optional /
Default value is "false"
                override_options                    => "true" | "false",                #Optional /
Default value is "false"
                override_preferred_lifetime         => "true" | "false",                #Optional /
Default value is "false"
                override_recycle_leases             => "true" | "false",                #Optional /
Default value is "false"
                override_update_dns_on_lease_renewal => "true" | "false",               #Optional /
Default value is "false"
                override_valid_lifetime             => "true" | "false",                #Optional /
Default value is "false"
                preferred_lifetime                  => $num,                            #Optional /
Default is "27000"
                recycle_leases                      => "true" | "false" | undef,        #Optional /
Default is undefined
                rir_organization                    => $rir_org,                        #Optional /
An Infoblox::Grid::RIR::Organization object
                rir_registration_action             => 'NONE' | 'CREATE' | 'MODIFY' | 'DELETE',  #
Optional / write-only
                rir_registration_status             => 'NOT_REGISTERED' | 'REGISTERED',  #Optional /
Default is 'NOT_REGISTERED'
                send_rir_request                    => 'true' | 'false',                #Optional /
write-only
                delete_reason                       => $string,                         #Optional /
write-only
                                                                                        #Required
only if 'rir_registration_action' is specified as 'DELETE'
                shared_network_name                 => $string| undef,                  #Optional /
Default is undefined
                template                            => $string| undef,                  #Optional /
Default is undefined
                update_dns_on_lease_renewal         => "true" | "false",                #Optional /
Default value is "false"
                valid_lifetime                      => $num,                            #Optional /
Default is "43200"
                zone_associations                   => [$zone1, $zone2, ... ] | undef,  #Optional /
Default is an empty list
                discovery_blackout_setting          => $blackout,                       #Optional
                port_control_blackout_setting       => $blackout,                       #Optional
                override_blackout_setting           => "true" | "false",                #Optional /
Default value is "false"
                same_port_control_discovery_blackout  => "true" | "false",              #Optional /
Default value is "false"
                subscribe_settings          => $subscribe_settings,                     #Optional /
Default is undefined
                override_subscribe_settings => 'true' | 'false',                        #Optional /
Default is 'false'
                discovery_engine_type       => 'NETMRI' | 'NETWORK_INSIGHT' | 'VDISCOVERY' | 'UNKNOWN' |
'NONE', #Optional / Default is 'NONE'
                discovered_bridge_domain    => $string,                                 #Optional /
Default is undefined
                discovered_tenant           => $string,                                 #Optional /
Default is undefined
              );
```

You cannot set both extattrs and extensible_attributes attributes at the same time.

SESSION METHODS

This section describes all the methods in an Infoblox::Session module that can apply to a DHCP IPv6 Network object.

Infoblox::Session->add( )

Use this method to add an IPv6 Network object to the Infoblox appliance. See Infoblox::Session->add() for parameters and return values.

Example

```
#Adding an IPv6 network.
#Construct an object
my $network = Infoblox::DHCP::IPv6Network->new(
```

```
        network => "2001::/64",
);

# Submit for adding an IPv6 network
my $response = $session->add( $network );
```

Infoblox::Session->get( )

Use this method to retrieve all the matching IPv6 Network objects from the Infoblox appliance. See Infoblox::Session->get() for parameters and return values.

'discover_now_status' will not be returned by default, use 'return_methods' to request it if needed.

Key References

```
Apply the following attributes to get a specific DHCP IPv6 network object:

  comment                  - Optional. A comment in string format.
  extattrs                 - Optional. A hash reference containing extensible attributes.
  extensible_attributes    - Optional. A hash reference containing extensible attributes.
  network                  - Optional. The IPv6 address and netmask of an IPv6 network in CIDR
format.
  network_view             - Optional. The network view in which the network is located. By default,
all network views are searched.
  discovered_bridge_domain - Optional. A bridge domain name in string format.
  discovered_tenant        - Optional. A tenant name in string format.
```

Examples

```
my  @result_array = $session->get(
     object       => "Infoblox::DHCP::IPv6Network" ,
     network      => "2001::/64",
     network_view => "default"
);

my  @result_array = $session->get(
     object               => "Infoblox::DHCP::IPv6Network" ,
     extensible_attributes => { 'Site' => 'Santa Clara' }
);

my  @result_array = $session->get(
     object  => "Infoblox::DHCP::IPv6Network" ,
     comment => "my-comment",
);

my @result_array = $session->get(
     object                  => "Infoblox::DHCP::IPv6Network" ,
     discovered_bridge_domain => "domain1",
);

my @result_array = $session->get(
     object           => "Infoblox::DHCP::IPv6Network" ,
     discovered_tenant => "tenant1",
);
```

Infoblox::Session->modify( )

Use this method to modify an IPv6 Network object in the Infoblox appliance. See Infoblox::Session->modify() for parameters and return values.

Example

```
#Use method to modify comment.
$object->comment("This is a new comment");
# Submit modification
my $response = $session->modify( $object );
```

Infoblox::Session->remove( )

Use this method to remove an object from the Infoblox appliance. See Infoblox::Session->remove() for parameters and return values.

To remove a specific object, first use `get()` or `search()` to retrieve the specific object, and then submit this object for removal.

Key References

```
remove_options - Optional. A hash reference containing the following attributes:

+-------------------------------------------------------------------------------------------+
| reason             - Optional. The reason for removing the network in string format with |
```

```
|                       a maximum of 256 bytes.                                                  |
|                       Used when 'rir_organization' attribute has been defined.                 |
| send_rir_request    - Optional. Boolean flag that indicates whether the appliance should       |
|                       send an API or email request for Regional Internet Registry (RIR).       |
|                       Used when 'rir_organization' attribute has been defined.                 |
+------------------------------------------------------------------------------------------------+
```

Example

```
 # Get the object under the same name
my  @result_array = $session->get(

       object        => "Infoblox::DHCP::IPv6Network" ,
       network       => "2001::/64",
       network_view  => "default"
         );

 # find the desired object from the retrieved list.
 my $desired_network = $result_array[0];

 # Submit for removal
 my $response = $session->remove(
     $desired_network,
     remove_options => {'reason' => 'Moved to B3', send_rir_request => 'true'} );
```

Infoblox::Session->search( )

Use this method to search for a DHCP IPv6 Network object in the Infoblox appliance. See Infoblox::Session->search() for parameters and return values.

'discover_now_status' will not be returned by default, use 'return_methods' to request it if needed.

Key References

```
 Apply the following attributes to search for a DHCP IPv6 Network object:

  comment                  - Optional. A comment in string format (regular expression).
  extattrs                 - Optional. A hash reference containing extensible attributes.
  extensible_attributes    - Optional. A hash reference containing extensible attributes.
  network                  - Optional. The IPv6 address and netmask of an IPv6 network in CIDR
format.
  network_view             - Optional. The network view in which the network is located. By default,
all network views are searched.
  discovered_bridge_domain - Optional. A bridge domain name in string format (regular expression).
  discovered_tenant        - Optional. A tenant name in string format (regular expression).
```

For more information about searching extensible attributes, see Infoblox::Grid::ExtensibleAttributeDef/Searching Extensible Attributes.

Examples

```
# search for all DHCP IPv6 Network objects
    my @retrieved_objs = $session->search(
      object       => "Infoblox::DHCP::IPv6Network",
      network      => "2001.*",
      network_view => "default"
    );

# search IPv6 network by comment
    my @retrieved_objs = $session->search(
      object    => "Infoblox::DHCP::IPv6Network",
      comment   => "my-comment",
    );

# search for all DHCP IPv6 Network objects defining "Santa Clara" for "Site" extensible attribute
    my @retrieved_objs = $session->search(
      object               => "Infoblox::DHCP::IPv6Network",
      extensible_attributes => { 'Site' => 'Santa Clara' }
    );

# search network by discovered bridge domain
 my @result_array = $session->search(
       object                   => "Infoblox::DHCP::IPv6Network" ,
       discovered_bridge_domain => "domain.*",
   );

 # search network by discovered tenant
 my @result_array = $session->search(
       object            => "Infoblox::DHCP::IPv6Network" ,
       discovered_tenant => "tenant.*",
   );
```

METHODS

This section describes all the methods that can be used to configure and retrieve the attribute values of a DHCP IPv6 Network object

auto_create_reversezone( )

Use this method to set the auto_create_reversezone flag of a DHCP IPv6 Network object.

Include the specified parameter to set the attribute value. This is a write-only attribute and cannot be retrieved after being set.

Parameter

Specify "true" to set the auto_create_reversezone flag or "false" to deactivate/unset it. The default value for this field is "false".

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

Example

```
# Modify auto_create_reversezone
$network_container->auto_create_reversezone("true");
```

comment( )

Use this method to set or retrieve a descriptive comment of an IPv6 Network object.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

Enter a descriptive comment for the network in string format with a maximum of 256 bytes.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
# Get comment
my $comment = $network->comment();
# Modify comment
$network->comment("add an IPv6 network");
```

cloud_info( )

Use this method to set or retrieve cloud API related information for the Infoblox::DHCP::IPv6Network object.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

The valid value is a Infoblox::Grid::CloudAPI::Info object.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
# Get cloud_info
my $cloud_info = $network_view->cloud_info();
# Modify cloud_info
$network_view->cloud_info($cloud_info_object);
```

ddns_domainname( )

Use this method to set or retrieve the ddns_domainname value of a DHCP IPv6 Network object.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Setting this method to a defined value implicitly sets the override_ddns_domainname method to "true". Setting the parameter to undefined causes the appliance to use the grid default and automatically resets the override_ddns_domainname attribute to "false".

Note that when ddns_domainname is set to a defined value and override_ddns_domainname is set to "false", the last operation takes precedence. Thus the sequence $object->ddns_domainname("testdomain"); $object->override_ddns_domainname("false"); will set override_ddns_domainname to "false", and the sequence $object->override_ddns_domainname("false"); $object->ddns_domainname("testdomain"); will result in override_ddns_domainname="true".

Parameter

The dynamic DNS domain name in string format.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get ddns_domainname
my $ddns_domainname = $network->ddns_domainname();
#Modify ddns_domainname
$network->ddns_domainname("test_domain.com");
```

ddns_enable_option_fqdn( )

Use this method to set or retrieve the ddns_enable_option_fqdn flag of a DHCP IPv6 Network object. This method controls whether the FQDN option sent by the client is to be used, or if the server can automatically generate the FQDN. This setting overrides the upper-level settings.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Setting this method to a defined value implicitly sets the override_ddns_enable_option_fqdn method to "true". Setting the parameter to undefined causes the appliance to use the upper-level setting and automatically resets the override_ddns_enable_option_fqdn attribute to "false".

Note that when ddns_enable_option_fqdn is set to a defined value and override_ddns_enable_option_fqdn is set to "false", the last operation takes precedence. Thus the sequence $object->ddns_enable_option_fqdn("true"); $object->override_ddns_enable_option_fqdn("false"); will set override_ddns_enable_option_fqdn to "false", and the sequence $object->override_ddns_enable_option_fqdn("false"); $object->ddns_enable_option_fqdn("true"); will result in override_ddns_enable_option_fqdn="true".

Parameter

Specify "true" to generate hostname or "false" to deactivate/unset it.The default value for this field is undefined, which indicates that this attribute inherit the member level setting.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
# Get ddns_enable_option_fqdn
my $ddns_enable_option_fqdn = $network->ddns_enable_option_fqdn()
# Modify ddns_enable_option_fqdn
$network->ddns_enable_option_fqdn("true");
#Un-override
$network->ddns_enable_option_fqdn(undef);
```

ddns_generate_hostname( )

Use this method to set or retrieve the ddns_generate_hostname flag of a DHCP IPv6 Network object. Enable this method to allow only the DHCP server for generating hostname. This setting overrides the upper-level settings.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Setting this method to a defined value implicitly sets the override_ddns_generate_hostname method to "true". Setting the parameter to undefined causes the appliance to use the upper-level setting and automatically resets the override_ddns_generate_hostname attribute to "false".

Note that when ddns_generate_hostname is set to a defined value and override_ddns_generate_hostname is set to "false", the last operation takes precedence. Thus the sequence $object->ddns_generate_hostname("true"); $object->override_ddns_generate_hostname("false"); will set override_ddns_generate_hostname to "false", and the sequence $object->override_ddns_generate_hostname("false"); $object->ddns_generate_hostname("true"); will result in override_ddns_generate_hostname="true".

Parameter

Specify "true" to generate a hostname or "false" to deactivate/unset it. The default value for this field is undefined, which indicates that this attribute inherits the member-level setting.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
# Get ddns_generate_hostname
my $ddns_generate_hostname = $network->ddns_generate_hostname()
# Modify ddns_generate_hostname
$network->ddns_generate_hostname("true");
#Un-override
$network->ddns_generate_hostname(undef);
```

ddns_server_always_updates( )

Use this method to set or retrieve the ddns_server_always_updates flag of a DHCP IPv6 Network object. Enable this method to allow only the DHCP server to update DNS, regardless of the requests from the DHCP clients.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

Specify "true" to set the flag that enables the DHCP server to always updates DNS or "false" to deactivate/unset it.The default value for this field is "true".

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
# Get ddns_server_always_updates
my $ddns_server_always_updates = $network->ddns_server_always_updates()
# Modify ddns_server_always_updates
$network->ddns_server_always_updates("true");
```

ddns_ttl( )

Use this method to set or retrieve the DNS update Time to Live (TTL) value of a DHCP IPv6 Network object.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Setting this method to a defined value implicitly sets the override_ddns_ttl method to "true". Setting the parameter to undefined causes the appliance to use the grid default and automatically resets the override_ddns_ttl attribute to "false".

Note that when ddns_ttl is set to a defined value and override_ddns_ttl is set to "false", the last operation takes precedence. Thus the sequence $object->ddns_ttl(12); $object->override_ddns_ttl("false"); will set override_ddns_ttl to "false", and the sequence $object->override_ddns_ttl("false"); $object->ddns_ttl(12); will result in override_ddns_ttl="true".

Parameter

A 32-bit integer (range from 0 to 4294967295) that represents the duration, in seconds, that the update is cached. Zero indicates that the update is not cached. The default value is 0.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get DNS Update ttl
my $dns_ttl = $network->ddns_ttl();
#Modify DNS Update ttl
$network->ddns_ttl(1200);
```

disable( )

Use this method to set or retrieve the disable flag of a DHCP IPv6 Network object.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

Specify "true" to set the disable flag or "false" to deactivate/unset it. The default value for this field is "false".

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
# Get disable
my $disable = $network->disable()
# Modify disable
$network->disable("true");
```

discovered_bridge_domain( )

Use this method to set or retrieve the discovered bridge domain name of a DHCP IPv6 Network object.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

The desired bridge domain name in a string format.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get bridge domain
my $bridge_domain = $network->discovered_bridge_domain();
#Modify bridge domain
$network->discovered_bridge_domain("Domain");
```

discovered_tenant( )

Use this method to set or retrieve the discovered tenant name of a DHCP IPv6 Network object.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

The desired tenant name in a string format.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get tenant
my $tenant = $network->discovered_tenant();
#Modify tenant
$network->discovered_tenant("Tenant");
```

discovery_basic_poll_setting( )

Use this method to set or retrieve the discovery_basic_poll_setting value of a DHCP IPv6 Network object.

Include the specified parameter to set the attribute value.

Parameter

The valid value is object of the Infoblox::Grid::Discovery::BasicPollSettings manpage.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get discovery_basic_poll_setting
my $discovery_basic_poll_setting = $network->discovery_basic_poll_setting();
#Modify discovery_basic_poll_setting
$network->discovery_basic_poll_setting($poll_setting);
```

discovery_member( )

Use this method to set or retrieve the discovery_member value of a DHCP IPv6 Network object.

Parameter

The discovery member name in string format.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get discovery_member
my $discovery_member = $network->discovery_member();
#Modify discovery_member
$network->discovery_member("test.localdomain");
```

discover_now_status( )

Use this method to retrieve the current discovery status. This attribute is read-only.

'discover_now_status' will not be returned by default, use 'return_methods' to request it if needed.

Parameter

None.

Returns

The method returns the attribute value that can be 'COMPLETE', 'FAILED', 'NONE', 'PENDING' or 'RUNNING'.

Example

```
#Get discover_now_status
my $discover_now_status = $network->discover_now_status();
```

discovery_blackout_setting( )

Use this method to set or retrieve the discovery blackout setting.

Setting this method to a defined value implicitly sets the override_blackout_setting method to "true". If port_control_blackout_setting is undefined, setting the parameter to undefined causes the appliance to use the grid default and automatically resets the override_blackout_setting attribute to "false".

Note that when discovery_blackout_setting is set to a defined value and override_blackout_setting is set to "false", the last operation takes precedence. Thus the sequence $network->discovery_blackout_setting($blackout); $network->override_blackout_setting("false"); will set override_blackout_setting to "false", and the sequence $network->override_blackout_setting("false"); $network->discovery_blackout_setting($blackout); will result in override_blackout_setting="true".

Parameter

Valid value is an Infoblox::Grid::Discovery::Properties::BlackoutSetting object.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get discovery_blackout_setting
my $discovery_blackout_setting = $network->discovery_blackout_setting();
#Modify discovery_blackout_setting
$network->discovery_blackout_setting($blackout);
```

discovery_engine_type( )

Use this method to retrieve the DHCP IPv6 Network discovery engine type. This attribute is read-only.

Parameter

None.

Returns

The method returns the attribute value that can be 'NETMRI', 'NETWORK_INSIGHT', 'VDISCOVERY', 'UNKNOWN' or 'NONE'.

Example

```
#Get discovery_engine_type
my $discovery_engine_type = $network->discovery_engine_type();
```

domain_name( )

Use this method to set or retrieve the domain_name value of a DHCP IPv6 Network object.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Setting this method to a defined value implicitly sets the override_domain_name method to "true". Setting the parameter to undefined causes the appliance to use the grid default and automatically resets the override_domain_name attribute to "false".

Note that when domain_name is set to a defined value and override_domain_name is set to "false", the last operation takes precedence. Thus the sequence $object->domain_name("testdomain"); $object->override_domain_name("false"); will set override_domain_name to "false", and the sequence $object->override_domain_name("false"); $object->domain_name("testdomain"); will result in override_domain_name="true".

Parameter

The name of the client's domain for resolving names in DNS in string format.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get domain_name
my $domain_name = $network->domain_name();
#Modify domain_name
$network->domain_name("test_domain.com");
```

domain_name_servers( )

Use this method to set or retrieve the domain_name_servers value of a DHCP IPv6 Network object.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Setting this method to a defined value implicitly sets the override_domain_name_servers method to "true". Setting the parameter to undefined causes the appliance to use the grid default and automatically resets the override_domain_name_servers attribute to "false".

Note that when domain_name_servers is set to a defined value and override_domain_name_servers is set to "false", the last operation takes precedence. Thus the sequence $object->domain_name_servers("2001::1,2001::2"); $object->override_domain_name_servers("false"); will set override_domain_name_servers to "false", and the sequence $object->override_domain_name_servers("false"); $object->domain_name_servers("2001::1,2001::2"); will result in override_domain_name_servers="true".

Parameter

The domain name server addresses in IPv6 address format.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get domain_name_servers
my $domain_name_servers = $network->domain_name_servers();
#Modify domain_name_servers
```

```
$network->domain_name_servers("2001::1,2001::2");
```

enable_ddns( )

Use this method to set or retrieve the dynamic DNS updates flag of a DHCP IPv6 Network object. The DHCP server can send DDNS updates to DNS servers in the same grid and to external DNS servers. This setting overrides the member level settings.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Setting this method to a defined value implicitly sets the override_enable_ddns method to "true". Setting the parameter to undefined causes the appliance to use the upper-level default and automatically resets the override_enable_ddns attribute to "false".

Note that when enable_ddns is set to a defined value and override_enable_ddns is set to "false", the last operation takes precedence. Thus the sequence $object->enable_ddns("true"); $object->override_enable_ddns("false"); will set override_enable_ddns to "false", and the sequence $object->override_enable_ddns("false"); $object->enable_ddns("true"); will result in override_enable_ddns="true".

Parameter

Specify "true" to set the dynamic DNS updates flag or "false" to deactivate/unset it. The default value for this field is undefined, which indicates that this attribute inherit the member level setting.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
# Get enable_ddns
my $enable_ddns = $network->enable_ddns()
# Modify enable_ddns
$network->enable_ddns("true");
# Un-override enable_ddns
$network->enable_ddns(undef);
```

enable_discovery( )

Use this method to set or retrieve the enable_discovery value of a DHCP IPv6 Network object. This setting overrides the member level settings.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Setting this method to a defined value implicitly sets the override_enable_discovery method to "true". Setting the parameter to undefined causes the appliance to use the default from the upper level and automatically resets the override_enable_discovery attribute to "false".

Enable this flag if you want to set discovery for this object.

Parameter

Specify "true" to set the enable_discovery flag or "false" to deactivate/unset it. The default value is undefined.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get enable_discovery
my $enable_discovery = $network->enable_discovery();
#Modify enable_discovery
$network->enable_discovery("false");
```

enable_ifmap_publishing( )

Use this method to set or retrieve the ifmap publishing option of a DHCP IPv6 Network object. This setting overrides the member level settings.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

Specify "true" to set ifmap publishing option or "false" to deactivate/unset it. The default value is 'false', which indicates that this attribute inherits the member level setting.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
# Get enable_ifmap_publishing
my $enable_ifmap_publishing = $network->enable_ifmap_publishing()
# Modify enable_ifmap_publishing
$network->enable_ifmap_publishing("true");
# Un-override enable_ifmap_publishing
$network->enable_ifmap_publishing(undef);
```

enable_immediate_discovery( )

Use this method to set or retrieve enable_immediate_discovery value of a DHCP IPv6 Network object.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

Specify "true" to set the enable_immediate_discovery flag or "false" to deactivate/unset it. The default value is undefined.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get enable_immediate_discovery
my $enable_immediate_discovery = $network->enable_immediate_discovery();
#Modify enable_immediate_discovery
$network->enable_immediate_discovery("false");
```

endpoint_sources( )

Use this method to retrieve the list of endpoints that provide object data. This is a read-only attribute.

Omit the parameter to retrieve the attribute value.

Parameter

None

Returns

The valid return value is an array of Infoblox::CiscoISE::Endpoint objects.

Example

```
#Get endpoint_sources value
my $endpoint_sources = $object->endpoint_sources();
```

extattrs( )

Use this method to set or retrieve the extensible attributes associated with an IPv6 Network object.

Parameter

Valid value is a hash reference containing the names of extensible attributes and their associated values (Infoblox::Grid::Extattr objects).

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get extattrs
my $ref_extattrs = $network->extattrs();
#Modify extattrs
$network->extattrs({ 'Site' => $extattr1, 'Administrator' => $extattr2 });
```

extensible_attributes( )

Use this method to set or retrieve the extensible attributes associated with an IPv6 Network object.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

For valid values for extensible attributes, see Infoblox::Grid::ExtensibleAttributeDef/Extensible Attribute Values.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get extensible attributes
my $ref_extensible_attributes = $network->extensible_attributes();
#Modify extensible attributes
$network->extensible_attributes({ 'Site' => 'Santa Clara', 'Administrator' => [ 'Peter', 'Tom' ] });
```

members( )

Use this method to set or retrieve the member that serves DHCP for an IPv6 Network object.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

The valid value is an array reference that contains Infoblox::DHCP::Member objects.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
my $member1 = Infoblox::DHCP::Member->new(
    name     => "infoblox.localdomain",
    ipv4addr => "192.168.1.2",
);

#Create the Network object with this member
my $network = Infoblox::DHCP::Network->new(
    network => "2001::/64",
    members => [$member1],
);
```

mgm_private( )

Use this method to set or retrieve the flag that controls whether the object should be synced to the MGM or not.

Setting this method to a defined value implicitly sets the override_blackout_setting method to "true". If discovery_blackout_setting is undefined, setting the parameter to undefined causes the appliance to use the grid default and automatically resets the override_blackout_setting attribute to "false".

Note that when mgm_private is set to a defined value and override_mgm_private is set to "false", the last operation takes precedence. Thus the sequence $network->mgm_private("true"); $network->override_mgm_private("false"); will set override_mgm_private to "false", and the sequence $network->override_mgm_private("false"); $network->mgm_private("true"); will result in override_mgm_private="true".

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

Specify 'true' to enable synchronization of the managed Grid with the MGM or 'false' to forbid it. The default value is 'false'.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Note that this member is loaded on on-demand basis, so to retrieve the value the object must have been retrieved from the session first.

Example

```
#Get mgm_private value
my $value = $object->mgm_private();

#Modify mgm_private value
$object->mgm_private('true');
```

mgm_private_overridable( )

Use this method to retrieve the flag that indicates whether the overriding of 'mgm_private' flag is allowed or not. This is a read-only attribute.

Parameter

None

Returns

The method returns the attribute value.

Note that this member is loaded on on-demand basis, so to retrieve the value the object must have been retrieved from the session first.

Example

```
#Get mgm_private_overridable value
my $value = $object->mgm_private_overridable();
```

ms_ad_user_data( )

Use this method to retrieve Microsoft Active Directory users related information. This is a read-only attribute.

Parameter

None

Returns

The valid return value is an Infoblox::Grid::MSServer::AdUser::Data object.

Example

```
#Get ms_ad_user_data
my $ms_ad_user_data = $v6network->ms_ad_user_data();
```

network( )

Use this method to retrieve the network attribute of a DHCP IPv6 Network object.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

Specify the address and netmask of the network in CIDR format (example: 2001::/64).

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get network
my $network = $network->network();
```

network_container( )

Use this method to set or retrieve the network container of a DHCP IPv6 Network object.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

Specify the address and netmask of the network in CIDR format (example: 2001::/64).

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get network container
my $network_container = $network->network_container();
#Modify network container
$network->network_container("2001::/64");
```

network_view( )

Use this method to set or retrieve the network view of the IPv6 Network object.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

The default value is the "default" network view, which means the network is in the default network view.

Parameter

Valid value is a defined Infoblox::DHCP::View object. The default network view value is the system-defined default network view object.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get network view
my $nview = $network->network_view();
#Modify network view, with an Infoblox::DHCP::View object
$network->network_view($nview);
```

next_available_ip( )

Use this method to retrieve the next available IPv6 address(es) in the network. Note that to use this method, the object MUST be retrieved through a get or search call, or created with an add call.

Parameter

The method accepts the following optional parameters:

```
  requested_num - The number of IP addresses requested. It must be a number between 1 and 20,
inclusive.
  excluded - A reference to an array of addresses to be excluded from the results.
```

Returns

This method returns the next available IPv6 address(es) in the network. If requested_num is not specified, a string with the next available address will be returned; otherwise, the method will return a reference to an array of addresses.

Within a DHCP IPv6Network object, the next available IP address(es) are the first addresses in this network that match all of the following rules:

```
   * Are not part of the passed 'excluded' parameter (if specified).
   * There is no AAAA, PTR, fixed address or host address record with this IP address
   * There is no bulk host record with this IP address
   * There is no unmanaged record with this IP address
   * The IP address is not the network or broadcast address of this network
   * The IP address is not in any IPv6 DHCP range within this network (including any exclusion range)
```

Example

```
# Retrieve a particular network
my @retrieved_objs = $session->get(
                          object   => "Infoblox::DHCP::IPv6Network",
                          network  => "10:9::/64"
);

$network = $retrieved_objs[0];

# Print the next available IPv6 address in the network
print $network->next_available_ip() . "\n";

# This will retrieve the next 10 available addresses and print the first one.
my $addressesref = $network->next_available_ip(requested_num => 10);
```

```
print @$addressesref[0] . "\n";

# This will retrieve the next 10 available addresses and exclude two addresses.
my $addressesref = $network->next_available_ip(requested_num => 10, excluded => ["10:9::2",
"10:9::3"]);
```

next_available_network( )

Use this method to retrieve the next available IPv6 network(s). Note that to use this method, the object MUST be retrieved through a get or search call, or it must have been created with an add call.

Parameter

The method accepts the following parameters:

```
  cidr - (required) The CIDR of the requested networks. It must be a number between 3 and 128,
inclusive.
  requested_num - (optional) The number of networks requested.  It must be a number between 1 and 20,
inclusive.
  excluded - (optional) A reference to an array of networks to be excluded from the results.
```

Returns

This method returns the next available IPv6 `network(s)` . If requested_num is not specified, a string with the next available network will be returned; otherwise, the method will return a reference to an array of networks.

A network cannot be returned as the next available network if it cannot be inserted; such as when any of the following conditions exist:

- It's parent network has a DHCP configuration assigned and therefore cannot be converted to a network container.

- It overlaps with an existing network or network container, regardless of whether the existing network has a greater or smaller CIDR than the requested CIDR.

- Its network address or broadcast address is used by a fixed address or a host configured for DHCP.

Example

```
# Retrieve a particular network
my @retrieved_objs = $session->get(
                          object   => "Infoblox::DHCP::IPv6Network",
                          network  => "2001::/64"
);

$network = $retrieved_objs[0];

# Print the next available IPv6 network in the network
print $network->next_available_network(cidr => 68) . "\n";

# This will retrieve the next 10 available networks and print the first one.
my $addressesref = $network->next_available_network(requested_num => 10, cidr => 68);
print @$addressesref[0] . "\n";

# This will retrieve the next 10 available networks and exclude two.
my $addressesref = $network->next_available_network(requested_num => 10, cidr => 72, excluded =>
['2001::100:0:0:0/72','2001::300:0:0:0/72']);
```

options( )

Use this method to set or retrieve the options value of a member DHCP server. DHCP options provide network configuration settings and various services available on the network for DHCP.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Setting this method to a defined value implicitly sets the override_options method to "true". Setting the parameter to undefined causes the appliance to use the upper-level default and automatically resets the override_options attribute to "false".

Note that when the options method is set to a defined value and override_options is set to "false", the last operation takes precedence. Thus the sequence $object->options([$opt]); $object->override_options("false"); will set override_options to "false", and the sequence $object->override_options("false"); $object->options([$opt]); will result in override_options="true".

Parameter

The valid value is an array reference that contains Infoblox::DHCP::Option objects.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get options
my $options = $network->options();
#Modify options
my $option1 = Infoblox::DHCP::Option->new(
    name => "dhcp6.info-refresh-time",
    value => 60
);

$network->options([$option1]);
```

override_ddns_domainname( )

The override_ddns_domainname attribute controls whether the ddns_domainname value in the DHCP IPv6 Network is used, instead of the grid default.

The override_ddns_domainname attribute can be specified explicitly. It is also set implicitly when ddns_domainname is set to a defined value.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

Set the parameter to "true" to override the grid-level setting for ddns_domainname. Set the parameter to "false" to inherit the grid-level setting for ddns_domainname.

The default value of this parameter is "false".

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Getting override_ddns_domainname
my $override_ddns_domainname=$network->override_ddns_domainname( );
#Modifying override_ddns_domainname
$network->override_ddns_domainname("true");
```

override_ddns_enable_option_fqdn( )

The override_ddns_enable_option_fqdn attribute controls whether the ddns_enable_option_fqdn value in the DHCP IPv6 Network is used, instead of the grid default.

The override_ddns_enable_option_fqdn attribute can be specified explicitly. It is also set implicitly when ddns_domainname is set to a defined value.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

Set the parameter to "true" to override the grid-level setting for ddns_enable_option_fqdn. Set the parameter to "false" to inherit the grid-level setting for ddns_enable_option_fqdn.

The default value of this parameter is "false".

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Getting override_ddns_enable_option_fqdn
my $override_ddns_enable_option_fqdn=$network->override_ddns_enable_option_fqdn( );
#Modifying override_ddns_enable_option_fqdn
$network->override_ddns_enable_option_fqdn("true");
```

override_ddns_generate_hostname( )

The override_ddns_generate_hostname attribute controls whether the ddns_generate_hostname value in the DHCP IPv6 Network is used, instead of the grid default.

The override_ddns_generate_hostname attribute can be specified explicitly. It is also set implicitly when ddns_generate_hostname is set to a defined value.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

Set the parameter to "true" to override the grid-level setting for ddns_generate_hostname. Set the parameter to "false" to inherit the grid-level setting for ddns_generate_hostname.

The default value of this parameter is "false".

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Getting override_ddns_generate_hostname
my $override_ddns_generate_hostname=$network->override_ddns_generate_hostname( );
#Modifying override_ddns_generate_hostname
$network->override_ddns_generate_hostname("true");
```

override_ddns_ttl( )

The override_ddns_ttl attribute controls whether the ddns_ttl value in the DHCP IPv6 Network is used, instead of the grid default.

The override_ddns_ttl attribute can be specified explicitly. It is also set implicitly when ddns_ttl is set to a defined value.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

Set the parameter to "true" to override the grid-level setting for ddns_ttl. Set the parameter to "false" to inherit the grid-level setting for ddns_ttl.

The default value of this parameter is "false".

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Getting override_ddns_ttl
my $override_ddns_ttl=$network->override_ddns_ttl( );
#Modifying override_ddns_ttl
$network->override_ddns_ttl("true");
```

override_domain_name( )

The override_domain_name attribute controls whether the domain_name value in the DHCP IPv6 Network is used, instead of the grid default.

The override_domain_name attribute can be specified explicitly. It is also set implicitly when domain_name is set to a defined value.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

Set the parameter to "true" to override the grid-level setting for domain_name. Set the parameter to "false" to inherit the grid-level setting for domain_name.

The default value of this parameter is "false".

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Getting override_domain_name
my $override_domain_name=$network->override_domain_name( );
```

```
#Modifying override_domain_name
$network->override_domain_name("true");
```

override_domain_name_servers( )

The override_domain_name_servers attribute controls whether the domain_name_servers value in the DHCP IPv6 Network is used, instead of the grid default.

The override_domain_name_servers attribute can be specified explicitly. It is also set implicitly when domain_name_servers is set to a defined value.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

Set the parameter to "true" to override the grid-level setting for domain_name_servers. Set the parameter to "false" to inherit the grid-level setting for domain_name_servers.

The default value of this parameter is "false".

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Getting override_domain_name_servers
my $override_domain_name_servers=$network->override_domain_name_servers( );
#Modifying override_domain_name_servers
$network->override_domain_name_servers("true");
```

override_blackout_setting( )

The override_blackout_setting attribute controls whether the discovery_blackout_setting and port_control_blackout_setting values in the object are used, instead of the grid default.

The override_blackout_setting attribute can be specified explicitly. It is also set implicitly when discovery_blackout_setting or port_control_blackout_setting is set to a defined value.

Parameter

Specify "true" to set the override_blackout_setting flag or "false" to deactivate/unset it. The default value is "false".

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
# Get attribute value
my $value = $network->override_blackout_setting();
# Modify attribute value
$network->override_blackout_setting('true');
```

override_enable_ddns( )

The override_enable_ddns attribute controls whether the enable_ddns value in the DHCP IPv6 Network is used, instead of the grid default.

The override_enable_ddns attribute can be specified explicitly. It is also set implicitly when enable_ddns is set to a defined value.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

Set the parameter to "true" to override the grid-level setting for enable_ddns. Set the parameter to "false" to inherit the grid-level setting for enable_ddns.

The default value of this parameter is "false".

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Getting override_enable_ddns
my $override_enable_ddns=$network->override_enable_ddns( );
#Modifying override_enable_ddns
$network->override_enable_ddns("true");
```

override_enable_ifmap_publishing( )

The override_enable_ifmap_publishing controls whether the enable_ifmap_publishing in the DHCP IPv6 Network is used, instead of the grid default.

The override_enable_ifmap_publishing can be specified explicitly. It is also set implicitly when enable_ifmap_publishing is set to a defined value.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

Set the parameter to "true" to override the grid-level setting for enable_ifmap_publishing. Set the parameter to "false" to inherit the grid-level setting for enable_ifmap_publishing.

The default value of this parameter is "false".

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Getting override_enable_ifmap_publishing
my $override_enable_ifmap_publishing=$network->override_enable_ifmap_publishing( );
#Modifying override_enable_ifmap_publishing
$network->override_enable_ifmap_publishing("true");
```

override_options( )

The override_options attribute controls whether the options value in the DHCP IPv6 Network is used, instead of the grid default.

The override_options attribute can be specified explicitly. It is also set implicitly when options is set to a defined value.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

Set the parameter to "true" to override the grid-level setting for options. Set the parameter to "false" to inherit the grid-level setting for options.

The default value of this parameter is "false".

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Getting override_options
my $override_options=$network->override_options( );
#Modifying override_options
$network->override_options("true");
```

override_preferred_lifetime( )

The override_preferred_lifetime attribute controls whether the preferred_lifetime value in the DHCP IPv6 Network is used, instead of the grid default.

The override_preferred_lifetime attribute can be specified explicitly. It is also set implicitly when preferred_lifetime is set to a defined value.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

Set the parameter to "true" to override the grid-level setting for options. Set the parameter to "false" to inherit the grid-level setting for options.

The default value of this parameter is "false".

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Getting override_preferred_lifetime
my $override_preferred_lifetime=$network->override_preferred_lifetime( );
#Modifying override_preferred_lifetime
$network->override_preferred_lifetime("true");
```

override_recycle_leases( )

The override_recycle_leases attribute controls whether the recycle_leases value in the DHCP IPv6 Network is used, instead of the grid default.

The override_recycle_leases attribute can be specified explicitly. It is also set implicitly when recycle_leases is set to a defined value.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

Set the parameter to "true" to override the grid-level setting for options. Set the parameter to "false" to inherit the grid-level setting for options.

The default value of this parameter is "false".

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Getting override_recycle_leases
my $override_recycle_leases=$network->override_recycle_leases( );
#Modifying override_recycle_leases
$network->override_recycle_leases("true");
```

override_mgm_private( )

The override_mgm_private attribute controls whether the mgm_private value in the object is used, instead of the parent-level default.

The override_mgm_private attribute can be specified explicitly. It is also set implicitly when mgm_private is set to a defined value.

Note that setting override_mgm_private to 'false' sets the original 'mgm_private' value to 'false'.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

Set the parameter to 'true' to override the parent-level setting for mgm_private. Set the parameter to 'false' to inherit the parent-level setting for mgm_private.

The default value of this parameter is 'false'.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Note that this member is loaded on on-demand basis, so to retrieve the value the object must have been retrieved from the session first.

Example

```
#Get override_mgm_private value
my $value = $object->override_mgm_private( );

#Modify override_mgm_private value
$object->override_mgm_private('true');
```

override_subscribe_settings( )

The override_subscribe_settings attribute controls whether the subscribe_settings method values of the member are used, instead of the Grid default.

The override_subscribe_settings attribute can be specified explicitly. It is also set implicitly when subscribe_settings is set to a defined value.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

Set the parameter to "true" to override the Grid-level setting for subscribe_settings. Set the parameter to "false" to inherit the Grid-level setting for subscribe_settings.

The default value of this parameter is "false".

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get override_subscribe_settings value
my $override_subscribe_settings = $object->override_subscribe_settings( );
#Modify override_subscribe_settings value
$object->override_subscribe_settings("true");
```

override_update_dns_on_lease_renewal( )

The override_update_dns_on_lease_renewal attribute controls whether the update_dns_on_lease_renewal value in the DHCP IPv6 Network is used, instead of the grid default.

The override_update_dns_on_lease_renewal attribute can be specified explicitly. It is also set implicitly when update_dns_on_lease_renewal is set to a defined value.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

Set the parameter to "true" to override the grid-level setting for update_dns_on_lease_renewal. Set the parameter to "false" to inherit the grid-level setting for update_dns_on_lease_renewal.

The default value of this parameter is "false".

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Getting override_update_dns_on_lease_renewal
my $override_update_dns_on_lease_renewal=$network->override_update_dns_on_lease_renewal( );
#Modifying override_update_dns_on_lease_renewal
$network->override_update_dns_on_lease_renewal("true");
```

override_valid_lifetime( )

The override_valid_lifetime attribute controls whether the valid_lifetime value in the DHCP IPv6 network is used, instead of the grid default.

The override_valid_lifetime attribute can be specified explicitly. It is also set implicitly when valid_lifetime is set to a defined value.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

Set the parameter to "true" to override the grid-level setting for options. Set the parameter to "false" to inherit the grid-level setting for options.

The default value of this parameter is "false".

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Getting override_valid_lifetime
my $override_valid_lifetime=$network->override_valid_lifetime( );
#Modifying override_valid_lifetime
$network->override_valid_lifetime("true");
```

preferred_lifetime ( )

Use this method to set or retrieve the preferred lifetime value of a DHCP IPv6 Network object.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Setting this method to a defined value implicitly sets the override_preferred_lifetime method to "true". Setting the parameter to undefined causes the appliance to use the grid default and automatically resets the override_preferred_lifetime attribute to "false".

Note that when preferred_lifetime is set to a defined value and override_preferred_lifetime is set to "false", the last operation takes precedence. Thus the sequence $object->preferred_lifetime(691200); $object->override_preferred_lifetime("false"); will set override_preferred_lifetime to "false", and the sequence $object->override_preferred_lifetime("false"); $object->preferred_lifetime(691200); will result in override_preferred_lifetime="true".

Parameter

A 32-bit integer (valid range from 1 to 4294967295) that represents the lifetime, in seconds. The default value is 27000.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get Preferred lifetime
my $preferred_lifetime = $network->preferred_lifetime();
#Modify Preferred lifetime
$network->preferred_lifetime(864000);
```

port_control_blackout_setting

Use this method to set or retrieve the port control blackout setting.

Setting this method to a defined value implicitly sets the override_blackout_setting method to "true". If discovery_blackout_setting is undefined, setting the parameter to undefined causes the appliance to use the grid default and automatically resets the override_blackout_setting attribute to "false".

Note that when port_control_blackout_setting is set to a defined value and override_blackout_setting is set to "false", the last operation takes precedence. Thus the sequence $network->port_control_blackout_setting($blackout); $network->override_blackout_setting("false"); will set override_blackout_setting to "false", and the sequence $network->override_blackout_setting("false"); $network->port_control_blackout_setting($blackout); will result in override_blackout_setting="true".

Parameter

Valid value is an Infoblox::Grid::Discovery::Properties::BlackoutSetting object.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get port_control_blackout_setting
my $port_control_blackout_setting = $network->port_control_blackout_setting();
#Modify port_control_blackout_setting
$network->port_control_blackout_setting($blackout);
```

recycle_leases( )

Use this method to set or retrieve the recycle_leases flag at the network level.

If the flag is enabled, the leases are kept in the recycle bin until one week after expiration. When the flag is disabled, the leases are irrecoverably deleted.

Setting this method to a defined value implicitly sets the override_recycle_leases method to "true". Setting the parameter to undefined causes the appliance to use the grid default and automatically resets the override_recycle_leases attribute to "false".

Note that when recycle_leases is set to a defined value and override_recycle_leases is set to "false", the last operation takes precedence. Thus the sequence $object->recycle_leases("true"); $object->override_recycle_leases("false"); will set override_recycle_leases to "false", and the sequence $objct->override_recycle_leases("false"); $object->recycle_leases("true"); will result in override_recycle_leases="true".

Parameter

Specify "true" to enable the recycle_leases flag or "false" to disable it. The default value is undefined.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get recycle_leases
my $recycle_leases = $network->recycle_leases();
#Modify recycle_leases
$network->recycle_leases("false");
```

restart_if_needed( )

Use this method to set the flag that indicates whether DHCP service should be restarted on a member that is serving DHCP for the changed object referenced in a cloud API request.

Parameter

Specify true to restart the DHCP service.

Returns

None

Example

```
# Modify
$network->restart_if_needed('true');
```

rir( )

Use this method to retrieve the name of the Regional Internet Registry (RIR) that allocated this address space. This is a read-only attribute.

Parameter

None

Returns

The method returns the attribute value that can be 'RIPE' or 'NONE'.

Example

```
#Get rir
my $rir = $network->rir();
```

rir_organization( )

Use this method to set or retrieve the Regional Internet Registry (RIR) organization object.

Parameter

An Infoblox::Grid::RIR::Organization object.

You must enable support for RIR/SWIP at the Grid level before you can set attribute values. See Infoblox::Grid->enable_rir_swip() method.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get rir_organization
my $rir_organization = $network->rir_organization();
#Modify rir_organization
$network->rir_organization($rir_org);
```

rir_registration_action( )

Use this method to set the registration action of Regional Internet Registry (RIR). This is a write-only attribute.

Parameter

Valid values are 'NONE', 'CREATE', 'MODIFY' or 'DELETE'.

Returns

The method returns true when the modification succeeds, and returns false when the operation fails.

Example

```
#Set rir_registration_action
$network->rir_registration_action('CREATE');
```

rir_registration_status( )

Use this method to set or retrieve the registration status of Regional Internet Registry (RIR).

Parameter

The registration status of Regional Internet Registry (RIR), which can be 'REGISTERED' or 'NOT_REGISTERED'.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get rir_registration_status
my $rir_registration_status = $network->rir_registration_status();
#Modify rir_registration_status
$network->rir_registration_status('NOT_REGISTERED');
```

last_rir_registration_update_sent( )

Use this method to retrieve the last registration update timestamp of Regional Internet Registry (RIR). This is a read-only attribute.

Parameter

None

Returns

The method returns the attribute value.

Example

```
#Get last_rir_registration_update_sent
my $last_rir_registration_update_sent = $network->last_rir_registration_update_sent();
```

last_rir_registration_update_status( )

Use this method to retrieve the last registration update status of Regional Internet Registry (RIR). This is a read-only attribute.

Parameter

None

Returns

The method returns the attribute value.

Example

```
#Get last_rir_registration_update_status
my $last_rir_registration_update_status = $network->last_rir_registration_update_status();
```

same_port_control_discovery_blackout( )

Use this method to set or retrieve the flag that enables using the same discovery blackout for port control blackout.

Parameter

Specify 'true' to use same discovery blackout for port control blackout or 'false' to disable it. The default value is 'false'.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
# Get attribute value
my $value = $network->same_port_control_discovery_blackout();
# Modify attribute value
$network->same_port_control_discovery_blackout('true');
```

send_rir_request( )

Use this method to set the flag that indicates whether the appliance should send an API or email request for Regional Internet Registry (RIR). This is a write-only attribute.

Parameter

Specify "true" to set the send_rir_request flag or "false" to unset it.

Returns

The method returns true when the modification succeeds, and returns false when the operation fails.

Example

```
#Set send_rir_request
$network->send_rir_request('true');
```

delete_reason( )

Use this method to specify the reason for deleting the Regional Internet Registry (RIR) registration request. This is a write-only attribute. This field is required only if 'rir_registration_action' is specified as 'DELETE'.

Parameter

The reason for deleting the RIR registration request in string format with a maximum of 256 bytes.

Returns

The method returns true when the modification succeeds, and returns false when the operation fails.

Example

```
#Set delete_reason
$network->delete_reason('invalid');
```

shared_network_name( )

Use this method to retrieve the name of the shared network associated with the DHCP IPv6 Network object.

Parameter

none

Returns

The method returns the name of shared network.

Example

```
#Get shared_network_name
my $shared_network_name = $network->shared_network_name();
```

subscribe_settings( )

Use this method to set or retrieve the Cisco ISE subscribe settings.

Setting this method to a defined value implicitly sets the override_subscribe_settings method to "true". Setting the parameter to

undefined causes the appliance to use the Cisco ISE endpoint default and automatically resets the override_subscribe_settings method to "false".

Note that when subscribe_settings is set to a defined value and override_subscribe_settings is set to "false", the last operation takes precedence. Thus the sequence $object->subscribe_settings($value); $object->override_subscribe_settings("false"); will set override_subscribe_settings to "false", and the sequence $object->override_subscribe_settings("false"); $object->subscribe_settings($value); will result in override_subscribe_settings="true".

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

The valid value is an Infoblox::CiscoISE::SubscribeSetting object.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get subscribe_settings value
my $subscribe_settings = $object->subscribe_settings();
#Modify subscribe_settings value
$object->subscribe_settings($settings);
```

template( )

Use this method to set or retrieve the network template name of a DHCP IPv6 Network object.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

The valid name in string format.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get template
my $template = $network->template();
#Modify template
$network->template("Template_Name");
```

unmanaged( )

Use this method to set or retrieve the network status, which indicates whether the network is unmanaged or not.

Parameter

Specify true to mark the network as unmanaged or false as managed.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
# Get network status
$unmanaged = $network->unmanaged();
# Modify network status
$network->unmanaged('true');
```

update_dns_on_lease_renewal( )

The update_dns_on_lease_renewal attribute controls whether the DHCP server updates DNS when a DHCP lease is renewed.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Setting this method to a defined value implicitly sets the override_update_dns_on_lease_renewal method to "true". Setting the parameter to undefined causes the appliance to use the grid default and automatically resets the override_update_dns_on_lease_renewal attribute to

"false".

Note that when update_dns_on_lease_renewal is set to a defined value and override_update_dns_on_lease_renewal is set to "false", the last operation takes precedence. Thus the sequence $object->update_dns_on_lease_renewal("true"); $object->override_update_dns_on_lease_renewal("false"); will set override_update_dns_on_lease_renewal to "false", and the sequence $object->override_update_dns_on_lease_renewal("false"); $object->update_dns_on_lease_renewal("true"); will result in override_update_dns_on_lease_renewal="true".

Parameter

Specify "true" to set the update_dns_on_lease_renewal flag or "false" to deactivate/unset it.

The default value of this parameter is "false".

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get update_dns_on_lease_renewal
my $update_dns_on_lease_renewal=$network->update_dns_on_lease_renewal( );
#Modify update_dns_on_lease_renewal
$network->update_dns_on_lease_renewal("true");
```

valid_lifetime ( )

Use this method to set or retrieve the valid lifetime value of a DHCP IPv6 Network object.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Setting this method to a defined value implicitly sets the override_valid_lifetime method to "true". Setting the parameter to undefined causes the appliance to use the grid default and automatically resets the override_valid_lifetime attribute to "false".

Note that when valid_lifetime is set to a defined value and override_valid_lifetime is set to "false", the last operation takes precedence. Thus the sequence $object->valid_lifetime(691200); $object->override_valid_lifetime("false"); will set override_valid_lifetime to "false", and the sequence $object->override_valid_lifetime("false"); $object->valid_lifetime(691200); will result in override_valid_lifetime="true".

Parameter

A 32-bit integer (valid range from 1 to 4294967295) that represents the valid lifetime, in seconds. The default value is 43200.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get valid lifetime
my $valid_lifetime = $network->valid_lifetime();
#Modify valid lifetime
$network->valid_lifetime(864000);
```

zone_associations( )

Use this method to set or retrieve the associated zone information.

Parameter

The valid value is an array reference that contains Infoblox::DNS::Zone object(s). The default value is an empty list. If the network has a parent, it is possible to use the zone associations of the parent by setting the parameter to undef.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails. If you did not specify a parameter, the method returns the attribute value.

Examples

```
#Getting zone associations
my $zone_associations = $network->zone_associations();
my $zone1 = Infoblox::DNS::Zone->new(
        name      => "test1.com",
);
my $zone2 = Infoblox::DNS::Zone->new(
```

```
            name     => "test2.com",
        );
    #Modifying zone_associations
    $network->zone_associations([$zone1, $zone2]);
    #Clear zone_assocations
    $network->zone_associations([]);
    ##Remove the override for zone_associations
    $network->zone_associations(undef)
```

SAMPLE CODE

The following sample code demonstrates the different functions that can be applied to an object such as add, search, modify, and remove. This sample also includes error handling for the operations.

#Preparation prior to a DHCP IPv6 Network insertion

```
#PROGRAM STARTS: Include all the modules that will be used
use strict;
use Infoblox;

#Create a session to the Infoblox appliance

my $session = Infoblox::Session->new(
    master   => "192.168.1.2",
    username => "admin",
    password => "infoblox"
);
unless ($session) {
   die("Construct session failed: ",
       Infoblox::status_code() . ":" . Infoblox::status_detail());
}
print "Session created successfully\n";
```

#Create a DHCP IPv6 Network object

```
my  $network = Infoblox::DHCP::IPv6Network->new(
              network => "2001::/64",
              comment => "add network",
            );

my $response = $session->add($network)
       or die("Add Network failed: ",
             $session->status_code() . ":" . $session->status_detail());

print "Network added successfully\n";

#Search for a specific Network

#Search the network
my @retrieved_objs = $session->search(
    object  => "Infoblox::DHCP::IPv6Network",
    network => "2001.*"
);
my $object = $retrieved_objs[0];
unless ($object) {
    die("Search Network failed: ",
        $session->status_code() . ":" . $session->status_detail());
}
print "Search Network found at least 1 matching entry\n";
```

#Get and modify a Network attribute

```
#Get IPv6 Network through the session
my @retrieved_objs = $session->get(
           object =>"Infoblox::DHCP::IPv6Network" ,
           network => "2001::"
        );

my $object = $retrieved_objs[0];

unless ($object) {
    die("Get Network object failed: ",
        $session->status_code() . ":" . $session->status_detail());
}
print "Get Network found at least 1 matching entry\n";
```

#Modify one of the attributes of the specified IPv6 Network

```
$object->comment("This is a new comment"); #Apply the changes

$session->modify($object)
    or die("Modify Network attribute failed: ",
            $session->status_code() . ":" . $session->status_detail());

print "Network object modified successfully \n";
```

#Remove an IPv6 Network Object

```
 #Get IPv6 Network object through the session
 my @retrieved_objs = $session->get(
           object  => "Infoblox::DHCP::IPv6Network" ,
           network => "2001::"
       );

   my $object = $retrieved_objs[0];

   unless ($object) {
    die("Get Network object failed: ",
        $session->status_code() . ":" . $session->status_detail());
    }

print "Get Network object found at least 1 matching entry\n";
```

#Submit the object for removal

```
$session->remove($object)
    or die("Remove Network failed: ",
        $session->status_code() . ":" . $session->status_detail());

print "Network removed successfully \n";

####PROGRAM ENDS####
```

AUTHOR

Infoblox Inc. http://www.infoblox.com/

SEE ALSO

Infoblox::Session, Infoblox::DHCP::View, Infoblox::DHCP::IPv6NetworkContainer, Infoblox::DHCP::Network, Infoblox::DHCP::NetworkContainer, Infoblox::Session->get(), Infoblox::Session->search(), Infoblox::Session->add(), Infoblox::Session->remove(), Infoblox::Session->modify(), Infoblox::Grid::MSServer::AdUser::Data

---

COPYRIGHT

Copyright (c) 2017 Infoblox Inc.

Infoblox::DHCP::IPv6NetworkContainer - DHCP IPv6 Network Container object

- NAME
- DESCRIPTION
- CONSTRUCTOR
- SESSION METHODS
  - Infoblox::Session->add( )
  - Infoblox::Session->get( )
  - Infoblox::Session->modify( )
  - Infoblox::Session->remove( )
  - Infoblox::Session->search( )
- METHODS
  - auto_create_reversezone( )
  - comment( )
  - cloud_info( )
  - ddns_domainname( )
  - ddns_enable_option_fqdn( )
  - ddns_generate_hostname( )
  - ddns_server_always_updates( )
  - ddns_ttl( )
  - discovery_basic_poll_setting( )
  - discovery_member( )
  - discover_now_status( )
  - discovery_blackout_setting( )
  - discovery_engine_type( )
  - domain_name( )
  - domain_name_servers( )
  - enable_ddns( )
  - enable_discovery( )
  - enable_immediate_discovery( )
  - endpoint_sources( )
  - extattrs( )
  - extensible_attributes( )
  - mgm_private( )
  - mgm_private_overridable( )
  - ms_ad_user_data( )
  - network( )
  - network_container( )
  - network_view( )
  - next_available_network( )
  - options( )
  - override_ddns_domainname( )
  - override_ddns_enable_option_fqdn( )
  - override_ddns_generate_hostname( )
  - override_ddns_ttl( )
  - override_domain_name( )
  - override_domain_name_servers( )
  - override_blackout_setting( )
  - override_enable_ddns( )
  - override_options( )
  - override_preferred_lifetime( )
  - override_recycle_leases( )
  - override_mgm_private( )
  - override_subscribe_settings( )
  - override_update_dns_on_lease_renewal( )
  - override_valid_lifetime( )
  - preferred_lifetime ( )
  - port_control_blackout_setting
  - recycle_leases( )
  - remove_subnets( )
  - restart_if_needed( )
  - rir( )
  - rir_organization( )
  - rir_registration_action( )
  - rir_registration_status( )
  - last_rir_registration_update_sent( )
  - last_rir_registration_update_status( )
  - same_port_control_discovery_blackout( )
  - send_rir_request( )
  - subscribe_settings( )
  - delete_reason( )
  - unmanaged( )
  - update_dns_on_lease_renewal( )

- valid_lifetime ( )
- zone_associations( )

- SAMPLE CODE
- AUTHOR
- SEE ALSO
- COPYRIGHT

NAME

Infoblox::DHCP::IPv6NetworkContainer - DHCP IPv6 Network Container object

DESCRIPTION

A network can contain child networks. The network that contains the child network is called a network container. This object encapsulates an IPv6 network container object.

CONSTRUCTOR

```
  my $network = Infoblox::DHCP::IPv6NetworkContainer->new(
                network                          => $ipv6_subnet/prefix,               #Required, when
creating a network container
                auto_create_reversezone          => "true" | "false",                  #Optional / Default
is "false"
                comment                          => $string| undef,                    #Optional / Default
is undefined
                remove_subnets                   => "true" | "false",                  #Optional / Default
is undefined
                rir_organization                 => $rir_org,                          #Optional / An
Infoblox::Grid::RIR::Organization object
                rir_registration_action          => 'NONE' | 'CREATE' | 'MODIFY' | 'DELETE',  # Optional /
write-only
                rir_registration_status          => 'NOT_REGISTERED' | 'REGISTERED',   #Optional / Default
is 'NOT_REGISTERED'
                send_rir_request                 => 'true' | 'false',                  #Optional / write-
only
                delete_reason                    => $string,                           #Optional / write-
only
                                                                                       #Required only if
'rir_registration_action' is specified as 'DELETE'
                mgm_private                      => "true" | "false",                  #Optional / Default
is "false"
                network_container                => $ipv6_subnet/prefix,               #Optional / Default
is "/"
                network_view                     => $NetworkView,                      #Optional / Default
is the system-defined default network view
                zone_associations                => [$zone1, $zone2, ... ] | undef,    #Optional / Default
is an empty list
                extattrs                         => { $string => $extattr, ... },      #Required if
'rir_organization' attribute is defined
                                                                                       #Otherwise is
optional
                extattrs                         => { $string => $extattr, ... },      #Required if
'rir_organization' attribute is defined
                                                                                       #Otherwise is
optional
                extensible_attributes            => { $string => $string | $num, $string => [ $string |
$num, ... ], ... } #Required if 'rir_organization' attribute is defined
#Otherwise is optional
                discovery_blackout_setting       => $blackout,                         #Optional
                port_control_blackout_setting    => $blackout,                         #Optional
                override_blackout_setting        => "true" | "false",                  #Optional / Default
value is "false"
                same_port_control_discovery_blackout => "true" | "false",              #Optional / Default
value is "false"

                ddns_domainname                      => $string| undef,                    #Optional /
Default is undefined
                ddns_enable_option_fqdn              => "true" | "false" ,                 #Optional /
Default is "false"
                ddns_generate_hostname               => "true" | "false" | undef,          #Optional /
Default is undefined
                ddns_server_always_updates           => "true" | "false",                  #Optional /
Default is "true"
                ddns_ttl                             => $num,                              #Optional /
Default is "0"
                domain_name                          => $string,                           #Optional /
Default is undefined
                domain_name_servers                  => "$ipv6addr,$ipv6addr,...",         #Optional /
```

```
Default is undefined
                enable_ddns                         => "true" | "false" | undef,          #Optional /
Default is undefined
                options                             => [$Option1,$Option2,...],           #Optional /
Default is empty list
                override_ddns_domainname            => "true" | "false",                  #Optional /
Default value is "false"
                override_ddns_enable_option_fqdn    => "true" | "false",                  #Optional /
Default value is "false"
                override_ddns_generate_hostname     => "true" | "false",                  #Optional /
Default value is "false"
                override_ddns_ttl                   => "true" | "false",                  #Optional /
Default value is "false"
                override_domain_name                => "true" | "false",                  #Optional /
Default value is "false"
                override_domain_name_servers        => "true" | "false",                  #Optional /
Default value is "false"
                override_enable_ddns                => "true" | "false",                  #Optional /
Default value is "false"
                override_mgm_private                => "true" | "false",                  #Optional /
Default value is "false"
                override_options                    => "true" | "false",                  #Optional /
Default value is "false"
                override_preferred_lifetime         => "true" | "false",                  #Optional /
Default value is "false"
                override_recycle_leases             => "true" | "false",                  #Optional /
Default value is "false"
                override_update_dns_on_lease_renewal  => "true" | "false",                #Optional /
Default value is "false"
                override_valid_lifetime             => "true" | "false",                  #Optional /
Default value is "false"
                preferred_lifetime                  => $num,                              #Optional /
Default is "27000"
                recycle_leases                      => "true" | "false" | undef,          #Optional /
Default is undefined
                update_dns_on_lease_renewal         => "true" | "false",                  #Optional /
Default value is "false"
                valid_lifetime                      => $num,                              #Optional /
Default is "43200"
                subscribe_settings          => $subscribe_settings,                       #Optional /
Default is undefined
                override_subscribe_settings => 'true' | 'false',                          #Optional /
Default is 'false'
                discovery_engine_type       => 'NETMRI' | 'NETWORK_INSIGHT' | 'VDISCOVERY' | 'UNKNOWN' |
'NONE', #Optional / Default is 'NONE'
                                );
```

You cannot set both extattrs and extensible_attributes at the same time.

SESSION METHODS

This section describes all the methods in an Infoblox::Session module that you can apply to a DHCP IPv6 Network Container object.

Infoblox::Session->add( )

Use this method to add a DHCP IPv6 Network Container object to the Infoblox appliance. See Infoblox::Session->add() for parameters and return values.

Example

```
#Adding an IPv6 network container.
#Construct an object
my $network_container = Infoblox::DHCP::IPv6NetworkContainer->new(
        network => "2001::/64",
);

# Submit for adding a network container
my $response = $session->add( $network_container );
```

Infoblox::Session->get( )

Use this method to retrieve all the matching DHCP IPv6 Network Container objects from the Infoblox appliance. See Infoblox::Session->get() for parameters and return values.

'discover_now_status' will not be returned by default, use 'return_methods' to request it if needed.

Key References

```
Apply the following attributes to get a specific DHCP IPv6 Network Container object:
```

```
  network               - Optional. The IPv6 address and netmask of a network in CIDR format.
  network_view          - Optional. The network view in which the network is located. By default, all
network views are searched.
  extattrs              - Optional. A hash reference containing extensible attributes.
  extensible_attributes - Optional. A hash reference containing extensible attributes.
  comment               - Optional. A comment in string format.
```

Examples

```
my  @result_array = $session->get(
     object        => "Infoblox::DHCP::IPv6NetworkContainer",
     network       => "2001::/64",
     network_view  => "default"
);

my  @result_array = $session->get(
     object                => "Infoblox::DHCP::IPv6NetworkContainer",
     extensible_attributes => { 'Site' => 'Santa Clara' }
);

my  @result_array = $session->get(
     object  => "Infoblox::DHCP::IPv6NetworkContainer",
     comment => "my-comment",
);
```

Infoblox::Session->modify( )

Use this method to modify a DHCP IPv6 Network Container object in the Infoblox appliance. See Infoblox::Session->modify() for parameters and return values.

Example

```
#Use method to modify comment.
$network_container->comment("modified comment");
# Submit modification
my $response = $session->modify( $network_container );
```

Infoblox::Session->remove( )

Use this method to remove an object from the Infoblox appliance. See Infoblox::Session->remove() for parameters and return values.

To remove a specifc object, first use `get()` or `search()` to retrieve the specific object, and then submit this object for removal.

Key References

```
remove_options - Optional. A hash reference containing the following attributes:

+-------------------------------------------------------------------------------------------+
| reason              - Optional. The reason for removing the network container in string   |
|                       format with a maximum of 256 bytes.                                 |
|                       Used when 'rir_organization' attribute has been defined.            |
| send_rir_request    - Optional. Boolean flag that indicates whether the appliance should |
|                       send an API or email request for Regional Internet Registry (RIR).  |
|                       Used when 'rir_organization' attribute has been defined.            |
+-------------------------------------------------------------------------------------------+
```

Example

```
 # Get the object under the same name
my  @result_array = $session->get(

      object        => "Infoblox::DHCP::IPv6NetworkContainer" ,
      network       => "2001::/64",
      network_view  => "default"
        );

# Find the desired object in the retrieved list.
my $network_container = $result_array[0];

# Submit for removal
my $response = $session->remove(
    $network_container,
    remove_options => {'reason' => 'Moved to B3', send_rir_request => 'true'} );
```

Infoblox::Session->search( )

Use this method to search for a DHCP IPv6 Network Container object in the Infoblox appliance. See Infoblox::Session->search() for parameters and return values.

'discover_now_status' will not be returned by default, use 'return_methods' to request it if needed.

Key References

```
 Apply the following attributes to search for a DHCP IPv6 Network Container object:

  network              - Optional. The IPv6 address and netmask of a network in CIDR format (regular
expression).
  network_view         - Optional. The network view in which the network is located. By default, all
network views are searched.
  extattrs             - Optional. A hash reference containing extensible attributes.
  extensible_attributes - Optional. A hash reference containing extensible attributes.
  comment              - Optional. A comment in string format (regular expression).
```

For more information about searching extensible attributes, see Infoblox::Grid::ExtensibleAttributeDef/Searching Extensible Attributes.

Examples

```
 # search for all DHCP IPv6 Network Container objects
     my @retrieved_objs = $session->search(
       object       => "Infoblox::DHCP::IPv6NetworkContainer",
       network      => "2001.*",
       network_view => "default"
     );

 # search network by comment
     my @retrieved_objs = $session->search(
       object    => "Infoblox::DHCP::IPv6NetworkContainer",
       comment   => "my-comment",
     );

 # search for all DHCP IPv6 Network Container objects defining "Santa Clara" for "Site" extensible
attribute
     my @retrieved_objs = $session->search(
       object                => "Infoblox::DHCP::IPv6NetworkContainer",
       extensible_attributes => { 'Site' => 'Santa Clara' }
     );
```

METHODS

This section describes all the methods that can be used to configure and retrieve the attribute values of a DHCP IPv6 Network Container object.

auto_create_reversezone( )

Use this method to set the auto_create_reversezone flag of a DHCP IPv6 Network Container object.

Include the specified parameter to set the attribute value. This is a write-only attribute and cannot be retrieved after it is set.

Parameter

Specify "true" to set the auto_create_reversezone flag or "false" to deactivate/unset it. The default value for this field is "false".

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

Example

```
 # Modify auto_create_reversezone
 $network_container->auto_create_reversezone("true");
```

comment( )

Use this method to set or retrieve a descriptive comment for a DHCP IPv6 Network Container object.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

Enter a descriptive comment for the network container in string format with a maximum of 256 bytes.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
# Get comment
my $comment = $network_container->comment();
# Modify comment
$network_container->comment("add a network");
```

cloud_info( )

Use this method to set or retrieve cloud API related information for the Infoblox::DHCP::IPv6NetworkContainer object.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

The valid value is a Infoblox::Grid::CloudAPI::Info object.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
# Get cloud_info
my $cloud_info = $network_view->cloud_info();
# Modify cloud_info
$network_view->cloud_info($cloud_info_object);
```

ddns_domainname( )

Use this method to set or retrieve the ddns_domainname value of a DHCP IPv6 Network Container object.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Setting this method to a defined value implicitly sets the override_ddns_domainname method to "true". Setting the parameter to undefined causes the appliance to use the Grid default and automatically resets the override_ddns_domainname attribute to "false".

Note that when ddns_domainname is set to a defined value and override_ddns_domainname is set to "false", the last operation takes precedence. Thus the sequence $object->ddns_domainname("testdomain"); $object->override_ddns_domainname("false"); will set override_ddns_domainname to "false", and the sequence $object->override_ddns_domainname("false"); $object->ddns_domainname("testdomain"); will result in override_ddns_domainname="true".

Parameter

The dynamic DNS domain name in string format.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get ddns_domainname
my $ddns_domainname = $network_container->ddns_domainname();
#Modify ddns_domainname
$network_container->ddns_domainname("test_domain.com");
```

ddns_enable_option_fqdn( )

Use this method to set or retrieve the ddns_enable_option_fqdn flag of a DHCP IPv6 Network Container object. This method controls whether the FQDN option sent by the client is to be used, or if the server can automatically generate the FQDN. This setting overrides the upper-level settings.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Setting this method to a defined value implicitly sets the override_ddns_enable_option_fqdn method to "true". Setting the parameter to undefined causes the appliance to use the upper-level setting and automatically resets the override_ddns_enable_option_fqdn attribute to "false".

Note that when ddns_enable_option_fqdn is set to a defined value and override_ddns_enable_option_fqdn is set to "false", the last operation takes precedence. Thus the sequence $object->ddns_enable_option_fqdn("true"); $object->override_ddns_enable_option_fqdn("false"); will set override_ddns_enable_option_fqdn to "false", and the sequence $object->override_ddns_enable_option_fqdn("false"); $object->ddns_enable_option_fqdn("true"); will result in override_ddns_enable_option_fqdn="true".

Parameter

Specify "true" to generate hostname or "false" to deactivate/unset it. The default value for this field is undefined, which indicates that this attribute inherits the member-level setting.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
# Get ddns_enable_option_fqdn
my $ddns_enable_option_fqdn = $network_container->ddns_enable_option_fqdn()
# Modify ddns_enable_option_fqdn
$network_container->ddns_enable_option_fqdn("true");
#Un-override
$network_container->ddns_enable_option_fqdn(undef);
```

ddns_generate_hostname( )

Use this method to set or retrieve the ddns_generate_hostname flag of a DHCP IPv6 Network Container object. Enable this method to allow only the DHCP server for generating hostname. This setting overrides the upper-level settings.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Setting this method to a defined value implicitly sets the override_ddns_generate_hostname method to "true". Setting the parameter to undefined causes the appliance to use the upper-level setting and automatically resets the override_ddns_generate_hostname attribute to "false".

Note that when ddns_generate_hostname is set to a defined value and override_ddns_generate_hostname is set to "false", the last operation takes precedence. Thus the sequence $object->ddns_generate_hostname("true"); $object->override_ddns_generate_hostname("false"); will set override_ddns_generate_hostname to "false", and the sequence $object->override_ddns_generate_hostname("false"); $object->ddns_generate_hostname("true"); will result in override_ddns_generate_hostname="true".

Parameter

Specify "true" to generate a hostname or "false" to deactivate/unset it. The default value for this field is undefined, which indicates that this attribute inherits the member-level setting.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
# Get ddns_generate_hostname
my $ddns_generate_hostname = $network_container->ddns_generate_hostname()
# Modify ddns_generate_hostname
$network_container->ddns_generate_hostname("true");
#Un-override
$network_container->ddns_generate_hostname(undef);
```

ddns_server_always_updates( )

Use this method to set or retrieve the ddns_server_always_updates flag of a DHCP IPv6 Network Container object. Enable this method to allow only the DHCP server to update DNS, regardless of the requests from the DHCP clients.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

Specify "true" to set the flag that enables the DHCP server to always updates DNS or "false" to deactivate/unset it.The default value for this field is "true".

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
# Get ddns_server_always_updates
my $ddns_server_always_updates = $network_container->ddns_server_always_updates()
# Modify ddns_server_always_updates
```

```
$network_container->ddns_server_always_updates("true");
```

ddns_ttl( )

Use this method to set or retrieve the DNS update Time to Live (TTL) value of a DHCP IPv6 Network Container object.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Setting this method to a defined value implicitly sets the override_ddns_ttl method to "true". Setting the parameter to undefined causes the appliance to use the Grid default and automatically resets the override_ddns_ttl attribute to "false".

Note that when ddns_ttl is set to a defined value and override_ddns_ttl is set to "false", the last operation takes precedence. Thus the sequence $object->ddns_ttl(12); $object->override_ddns_ttl("false"); will set override_ddns_ttl to "false", and the sequence $object->override_ddns_ttl("false"); $object->ddns_ttl(12); will result in override_ddns_ttl="true".

Parameter

A 32-bit integer (range from 0 to 4294967295) that represents the duration, in seconds, the update is cached. Zero indicates that the update is not cached. The default value is 0.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get DNS Update ttl
my $dns_ttl = $network_container->ddns_ttl();
#Modify DNS Update ttl
$network_container->ddns_ttl(1200);
```

discovery_basic_poll_setting( )

Use this method to set or retrieve the discovery_basic_poll_setting value of a DHCP IPv6 Network Container object.

Include the specified parameter to set the attribute value.

Parameter

The valid value is object of the Infoblox::Grid::Discovery::BasicPollSettings manpage.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get discovery_basic_poll_setting
my $discovery_basic_poll_setting = $network->discovery_basic_poll_setting();
#Modify discovery_basic_poll_setting
$network->discovery_basic_poll_setting($poll_setting);
```

discovery_member( )

Use this method to set or retrieve the discovery_member value of a DHCP IPv6 Network Container object.

Parameter

The discovery member name in string format.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get discovery_member
my $discovery_member = $network->discovery_member();
#Modify discovery_member
$network->discovery_member("test.localdomain");
```

discover_now_status( )

Use this method to retrieve the current discovery status. This attribute is read-only.

'discover_now_status' will not be returned by default, use 'return_methods' to request it if needed.

Parameter

None.

Returns

The method returns the attribute value that can be 'COMPLETE', 'FAILED', 'NONE', 'PENDING' or 'RUNNING'.

Example

```
#Get discover_now_status
my $discover_now_status = $network->discover_now_status();
```

discovery_blackout_setting( )

Use this method to set or retrieve the discovery blackout setting.

Setting this method to a defined value implicitly sets the override_blackout_setting method to "true". If port_control_blackout_setting is undefined, setting the parameter to undefined causes the appliance to use the Grid default and automatically resets the override_blackout_setting attribute to "false".

Note that when discovery_blackout_setting is set to a defined value and override_blackout_setting is set to "false", the last operation takes precedence. Thus the sequence $network->discovery_blackout_setting($blackout); $network->override_blackout_setting("false"); will set override_blackout_setting to "false", and the sequence $network->override_blackout_setting("false"); $network->discovery_blackout_setting($blackout); will result in override_blackout_setting="true".

Parameter

Valid value is an Infoblox::Grid::Discovery::Properties::BlackoutSetting object.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get discovery_blackout_setting
my $discovery_blackout_setting = $network->discovery_blackout_setting();
#Modify discovery_blackout_setting
$network->discovery_blackout_setting($blackout);
```

discovery_engine_type( )

Use this method to retrieve the DHCP IPv6 Network Container discovery engine type. This attribute is read-only.

Parameter

None.

Returns

The method returns the attribute value that can be 'NETMRI', 'NETWORK_INSIGHT', 'VDISCOVERY', 'UNKNOWN' or 'NONE'.

Example

```
#Get discovery_engine_type
my $discovery_engine_type = $network_container->discovery_engine_type();
```

domain_name( )

Use this method to set or retrieve the domain_name value of a DHCP IPv6 Network Container object.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Setting this method to a defined value implicitly sets the override_domain_name method to "true". Setting the parameter to undefined causes the appliance to use the Grid default and automatically resets the override_domain_name attribute to "false".

Note that when domain_name is set to a defined value and override_domain_name is set to "false", the last operation takes precedence. Thus the sequence $object->domain_name("testdomain"); $object->override_domain_name("false"); will set override_domain_name to "false", and the sequence $object->override_domain_name("false"); $object->domain_name("testdomain"); will result in override_domain_name="true".

Parameter

The name of the client domain in string format for resolving names in DNS.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get domain_name
my $domain_name = $network_container->domain_name();
#Modify domain_name
$network_container->domain_name("test_domain.com");
```

domain_name_servers( )

Use this method to set or retrieve the domain_name_servers value of a DHCP IPv6 Network Container object.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Setting this method to a defined value implicitly sets the override_domain_name_servers method to "true". Setting the parameter to undefined causes the appliance to use the Grid default and automatically resets the override_domain_name_servers attribute to "false".

Note that when domain_name_servers is set to a defined value and override_domain_name_servers is set to "false", the last operation takes precedence. Thus the sequence $object->domain_name_servers("2001::1,2001::2"); $object->override_domain_name_servers("false"); will set override_domain_name_servers to "false", and the sequence $object->override_domain_name_servers("false"); $object->domain_name_servers("2001::1,2001::2"); will result in override_domain_name_servers="true".

Parameter

The IPv6 addresses of the domain name servers.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get domain_name_servers
my $domain_name_servers = $network_container->domain_name_servers();
#Modify domain_name_servers
$network_container->domain_name_servers("2001::1,2001::2");
```

enable_ddns( )

Use this method to set or retrieve the dynamic DNS updates flag of a DHCP IPv6 Network Container object. The DHCP server can send DDNS updates to DNS servers in the same Grid and to external DNS servers. This setting overrides the member-level settings.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Setting this method to a defined value implicitly sets the override_enable_ddns method to "true". Setting the parameter to undefined causes the appliance to use the upper-level default and automatically resets the override_enable_ddns attribute to "false".

Note that when enable_ddns is set to a defined value and override_enable_ddns is set to "false", the last operation takes precedence. Thus the sequence $object->enable_ddns("true"); $object->override_enable_ddns("false"); will set override_enable_ddns to "false", and the sequence $object->override_enable_ddns("false"); $object->enable_ddns("true"); will result in override_enable_ddns="true".

Parameter

Specify "true" to set the dynamic DNS updates flag or "false" to deactivate/unset it. The default value for this field is undefined, which indicates that this attribute inherits the member-level setting.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
# Get enable_ddns
my $enable_ddns = $network_container->enable_ddns()
# Modify enable_ddns
```

```
$network_container->enable_ddns("true");
# Un-override enable_ddns
$network_container->enable_ddns(undef);
```

enable_discovery( )

Use this method to set or retrieve the enable_discovery value of a DHCP IPv6 Network Container object. This setting overrides the member level settings.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Setting this method to a defined value implicitly sets the override_enable_discovery method to "true". Setting the parameter to undefined causes the appliance to use the default from the upper level and automatically resets the override_enable_discovery attribute to "false".

Enable this flag if you want to set discovery for this object.

Parameter

Specify "true" to set the enable_discovery flag or "false" to deactivate/unset it. The default value is undefined.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get enable_discovery
my $enable_discovery = $network->enable_discovery();
#Modify enable_discovery
$network->enable_discovery("false");
```

enable_immediate_discovery( )

Use this method to set or retrieve enable_immediate_discovery value of a DHCP IPv6 Network Container object.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

Specify "true" to set the enable_immediate_discovery flag or "false" to deactivate/unset it. The default value is undefined.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get enable_immediate_discovery
my $enable_immediate_discovery = $network->enable_immediate_discovery();
#Modify enable_immediate_discovery
$network->enable_immediate_discovery("false");
```

endpoint_sources( )

Use this method to retrieve the list of endpoints that provide object data. This is a read-only attribute.

Omit the parameter to retrieve the attribute value.

Parameter

None

Returns

The valid return value is an array of Infoblox::CiscoISE::Endpoint objects.

Example

```
#Get endpoint_sources value
my $endpoint_sources = $object->endpoint_sources();
```

extattrs( )

Use this method to set or retrieve the extensible attributes associated with a DHCP IPv6 Network Container object.

Parameter

Valid value is a hash reference containing the names of extensible attributes and their associated values (Infoblox::Grid::Extattr objects).

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get extattrs
my $ref_extattrs = $network_container->extattrs();
#Modify extattrs
$network_container->extattrs({ 'Site' => $extattr1, 'Administrator' => $extattr2 });
```

extensible_attributes( )

Use this method to set or retrieve the extensible attributes associated with a DHCP IPv6 Network Container object

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

For valid values for extensible attributes, see Infoblox::Grid::ExtensibleAttributeDef/Extensible Attribute Values.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get extensible attributes
my $ref_extensible_attributes = $network_container->extensible_attributes();
#Modify extensible attributes
$network_container->extensible_attributes({ 'Site' => 'Santa Clara', 'Administrator' => [ 'Peter',
'Tom' ] });
```

mgm_private( )

Use this method to set or retrieve the flag that controls whether the object should be synced to the MGM or not.

Setting this method to a defined value implicitly sets the override_blackout_setting method to "true". If discovery_blackout_setting is undefined, setting the parameter to undefined causes the appliance to use the grid default and automatically resets the override_blackout_setting attribute to "false".

Note that when mgm_private is set to a defined value and override_mgm_private is set to "false", the last operation takes precedence. Thus the sequence $network->mgm_private("true"); $network->override_mgm_private("false"); will set override_mgm_private to "false", and the sequence $network->override_mgm_private("false"); $network->mgm_private("true"); will result in override_mgm_private="true".

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

Specify 'true' to enable synchronization of the managed Grid with the MGM or 'false' to forbid it. The default value is 'false'.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Note that this member is loaded on on-demand basis, so to retrieve the value the object must have been retrieved from the session first.

Example

```
#Get mgm_private value
my $value = $object->mgm_private();

#Modify mgm_private value
$object->mgm_private('true');
```

mgm_private_overridable( )

Use this method to retrieve the flag that indicates whether the overriding of 'mgm_private' flag is allowed or not. This is a read-only attribute.

Parameter

None

Returns

The method returns the attribute value.

Note that this member is loaded on on-demand basis, so to retrieve the value the object must have been retrieved from the session first.

Example

```
#Get mgm_private_overridable value
my $value = $object->mgm_private_overridable();
```

ms_ad_user_data( )

Use this method to retrieve Microsoft Active Directory users related information. This is a read-only attribute.

Parameter

None

Returns

The valid return value is an Infoblox::Grid::MSServer::AdUser::Data object.

Example

```
#Get ms_ad_user_data
my $ms_ad_user_data = $v6network_container->ms_ad_user_data();
```

network( )

Use this method to retrieve the network attribute of a DHCP IPv6 Network Container object.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

Specify the address and netmask of the network in CIDR format (example: 2001::/64).

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get network
my $network = $network_container->network();
```

network_container( )

Use this method to set or retrieve the network container of a DHCP IPv6 Network Container object.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

Specify the address and netmask of the network in CIDR format (example: 2001::/64).

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get network container
```

```
my $network_container = $network->network_container();
#Modify network container
$network->network_container("2001::/64");
```

network_view( )

Use this method to set or retrieve the network view of the DHCP IPv6 network container.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

The default value is the system-defined default network view, which means the network is in the default network view.

Parameter

Valid value is a defined Infoblox::DHCP::View object. The default network view value is the system-defined default network view object.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get network view
my $nview = $network_container->network_view();
#Modify network view, with an Infoblox::DHCP::View object
$network_container->network_view($nview);
```

next_available_network( )

Use this method to retrieve the next available IPv6 `network(s)` in the network container. Note that to use this method, the object MUST be retrieved through a get or search call, or it must have been created with an add call.

Parameter

The method accepts the following parameters:

```
  cidr - (required) The CIDR of the requested networks. It must be a number between 3 and 128,
inclusive.
  requested_num - (optional) The number of networks requested. It must be a number between 1 and 20,
inclusive.
  excluded - (optional) A reference to an array of networks to be excluded from the results.
```

Returns

This method returns the next available IPv6 `network(s)` in the network container. If requested_num is not specified, a string with the next available network will be returned; otherwise, the method will return a reference to an array of networks.

A network cannot be returned as the next available network if it cannot be inserted; such as when any of the following conditions exist:

- It's parent network has a DHCP configuration assigned and therefore cannot be converted to a network container.

- It overlaps with an existing network or network container, regardless of whether the existing network has a greater or smaller CIDR than the requested CIDR.

- Its network address or broadcast address is used by a fixed address or a host configured for DHCP.

Example

```
# Retrieve a particular network container
my @retrieved_objs = $session->get(
                        object   => "Infoblox::DHCP::IPv6NetworkContainer",
                        network  => "2001::/64"
);

$network_container = $retrieved_objs[0];

# Print the next available IPv6 network in the network container
print $network_container->next_available_network(cidr => 68) . "\n";

# This will retrieve the next 10 available networks and print the first one.
my $addressesref = $network_container->next_available_network(requested_num => 10, cidr => 68);
print @$addressesref[0] . "\n";

# This will retrieve the next 10 available networks and exclude two.
my $addressesref = $network_container->next_available_network(requested_num => 10, cidr => 72,
excluded => ['2001::100:0:0:0/72','2001::300:0:0:0/72']);
```

options( )

Use this method to set or retrieve the option values for a member DHCP server. DHCP options provide Network Container configuration settings and various services available for the Network Container.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Setting this method to a defined value implicitly sets the override_options method to "true". Setting the parameter to undefined causes the appliance to use the upper-level default and automatically resets the override_options attribute to "false".

Note that when the options method is set to a defined value and override_options is set to "false", the last operation takes precedence. Thus the sequence $object->options([$opt]); $object->override_options("false"); will set override_options to "false", and the sequence $object->override_options("false"); $object->options([$opt]); will result in override_options="true".

Parameter

The valid value is an array reference that contains Infoblox::DHCP::Option objects.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get options
my $options = $network_container->options();
#Modify options
my $option1 = Infoblox::DHCP::Option->new(
    name => "dhcp6.info-refresh-time",
    value => 60
);

$network_container->options([$option1]);
```

override_ddns_domainname( )

The override_ddns_domainname attribute controls whether the ddns_domainname value in the DHCP IPv6 Network Container is used, instead of the Grid default.

The override_ddns_domainname attribute can be specified explicitly. It is also set implicitly when ddns_domainname is set to a defined value.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

Set the parameter to "true" to override the Grid-level setting for ddns_domainname. Set the parameter to "false" to inherit the Grid-level setting for ddns_domainname.

The default value of this parameter is "false".

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Getting override_ddns_domainname
my $override_ddns_domainname=$network_container->override_ddns_domainname( );
#Modifying override_ddns_domainname
$network_container->override_ddns_domainname("true");
```

override_ddns_enable_option_fqdn( )

The override_ddns_enable_option_fqdn attribute controls whether the ddns_enable_option_fqdn value in the DHCP IPv6 Network Container is used, instead of the Grid default.

The override_ddns_enable_option_fqdn attribute can be specified explicitly. It is also set implicitly when ddns_domainname is set to a defined value.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

Set the parameter to "true" to override the Grid-level setting for ddns_enable_option_fqdn. Set the parameter to "false" to inherit the Grid-level setting for ddns_enable_option_fqdn.

The default value of this parameter is "false".

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Getting override_ddns_enable_option_fqdn
my $override_ddns_enable_option_fqdn=$network_container->override_ddns_enable_option_fqdn( );
#Modifying override_ddns_enable_option_fqdn
$network_container->override_ddns_enable_option_fqdn("true");
```

override_ddns_generate_hostname( )

The override_ddns_generate_hostname attribute controls whether the ddns_generate_hostname value in the DHCP IPv6 Network Container is used, instead of the Grid default.

The override_ddns_generate_hostname attribute can be specified explicitly. It is also set implicitly when ddns_generate_hostname is set to a defined value.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

Set the parameter to "true" to override the Grid-level setting for ddns_generate_hostname. Set the parameter to "false" to inherit the Grid-level setting for ddns_generate_hostname.

The default value of this parameter is "false".

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Getting override_ddns_generate_hostname
my $override_ddns_generate_hostname=$network_container->override_ddns_generate_hostname( );
#Modifying override_ddns_generate_hostname
$network_container->override_ddns_generate_hostname("true");
```

override_ddns_ttl( )

The override_ddns_ttl attribute controls whether the ddns_ttl value in the DHCP IPv6 Network Container is used, instead of the Grid default.

The override_ddns_ttl attribute can be specified explicitly. It is also set implicitly when ddns_ttl is set to a defined value.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

Set the parameter to "true" to override the Grid-level setting for ddns_ttl. Set the parameter to "false" to inherit the Grid-level setting for ddns_ttl.

The default value of this parameter is "false".

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Getting override_ddns_ttl
my $override_ddns_ttl=$network_container->override_ddns_ttl( );
#Modifying override_ddns_ttl
$network_container->override_ddns_ttl("true");
```

override_domain_name( )

The override_domain_name attribute controls whether the domain_name value in the DHCP IPv6 Network Container is used, instead of the Grid default.

The override_domain_name attribute can be specified explicitly. It is also set implicitly when domain_name is set to a defined value.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

Set the parameter to "true" to override the Grid-level setting for domain_name. Set the parameter to "false" to inherit the Grid-level setting for domain_name.

The default value of this parameter is "false".

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Getting override_domain_name
my $override_domain_name=$network_container->override_domain_name( );
#Modifying override_domain_name
$network_container->override_domain_name("true");
```

override_domain_name_servers( )

The override_domain_name_servers attribute controls whether the domain_name_servers value in the DHCP IPv6 Network Container is used, instead of the Grid default.

The override_domain_name_servers attribute can be specified explicitly. It is also set implicitly when domain_name_servers is set to a defined value.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

Set the parameter to "true" to override the Grid-level setting for domain_name_servers. Set the parameter to "false" to inherit the Grid-level setting for domain_name_servers.

The default value of this parameter is "false".

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Getting override_domain_name_servers
my $override_domain_name_servers=$network_container->override_domain_name_servers( );
#Modifying override_domain_name_servers
$network_container->override_domain_name_servers("true");
```

override_blackout_setting( )

The override_blackout_setting attribute controls whether the discovery_blackout_setting and port_control_blackout_setting values in the object are used, instead of the Grid default.

The override_blackout_setting attribute can be specified explicitly. It is also set implicitly when discovery_blackout_setting or port_control_blackout_setting is set to a defined value.

Parameter

Specify "true" to set the override_blackout_setting flag or "false" to deactivate/unset it. The default value is "false".

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
# Get attribute value
my $value = $network->override_blackout_setting();
```

```
# Modify attribute value
$network->override_blackout_setting('true');
```

override_enable_ddns( )

The override_enable_ddns attribute controls whether the enable_ddns value in the DHCP IPv6 Network Container is used, instead of the Grid default.

The override_enable_ddns attribute can be specified explicitly. It is also set implicitly when enable_ddns is set to a defined value.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

Set the parameter to "true" to override the Grid-level setting for enable_ddns. Set the parameter to "false" to inherit the Grid-level setting for enable_ddns.

The default value of this parameter is "false".

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Getting override_enable_ddns
my $override_enable_ddns=$network_container->override_enable_ddns( );
#Modifying override_enable_ddns
$network_container->override_enable_ddns("true");
```

override_options( )

The override_options attribute controls whether the options value in the DHCP IPv6 Network Container is used, instead of the Grid default.

The override_options attribute can be specified explicitly. It is also set implicitly when options is set to a defined value.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

Set the parameter to "true" to override the Grid-level setting for options. Set the parameter to "false" to inherit the Grid-level setting for options.

The default value of this parameter is "false".

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Getting override_options
my $override_options=$network_container->override_options( );
#Modifying override_options
$network_container->override_options("true");
```

override_preferred_lifetime( )

The override_preferred_lifetime attribute controls whether the preferred_lifetime value in the DHCP IPv6 Network Container is used, instead of the Grid default.

The override_preferred_lifetime attribute can be specified explicitly. It is also set implicitly when preferred_lifetime is set to a defined value.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

Set the parameter to "true" to override the Grid-level setting for options. Set the parameter to "false" to inherit the Grid-level setting for options.

The default value of this parameter is "false".

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Getting override_preferred_lifetime
my $override_preferred_lifetime=$network_container->override_preferred_lifetime( );
#Modifying override_preferred_lifetime
$network_container->override_preferred_lifetime("true");
```

override_recycle_leases( )

The override_recycle_leases attribute controls whether the recycle_leases value in the DHCP IPv6 Network Container is used, instead of the Grid default.

The override_recycle_leases attribute can be specified explicitly. It is also set implicitly when recycle_leases is set to a defined value.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

Set the parameter to "true" to override the Grid-level setting for options. Set the parameter to "false" to inherit the Grid-level setting for options.

The default value of this parameter is "false".

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Getting override_recycle_leases
my $override_recycle_leases=$network_container->override_recycle_leases( );
#Modifying override_recycle_leases
$network_container->override_recycle_leases("true");
```

override_mgm_private( )

The override_mgm_private attribute controls whether the mgm_private value in the object is used, instead of the parent-level default.

The override_mgm_private attribute can be specified explicitly. It is also set implicitly when mgm_private is set to a defined value.

Note that setting override_mgm_private to 'false' sets the original 'mgm_private' value to 'false'.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

Set the parameter to 'true' to override the parent-level setting for mgm_private. Set the parameter to 'false' to inherit the parent-level setting for mgm_private.

The default value of this parameter is 'false'.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Note that this member is loaded on on-demand basis, so to retrieve the value the object must have been retrieved from the session first.

Example

```
#Get override_mgm_private value
my $value = $object->override_mgm_private( );

#Modify override_mgm_private value
$object->override_mgm_private('true');
```

override_subscribe_settings( )

The override_subscribe_settings attribute controls whether the subscribe_settings method values of the member are used, instead of the Grid default.

The override_subscribe_settings attribute can be specified explicitly. It is also set implicitly when subscribe_settings is set to a defined value.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

Set the parameter to "true" to override the Grid-level setting for subscribe_settings. Set the parameter to "false" to inherit the Grid-level setting for subscribe_settings.

The default value of this parameter is "false".

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get override_subscribe_settings value
my $override_subscribe_settings = $object->override_subscribe_settings( );
#Modify override_subscribe_settings value
$object->override_subscribe_settings("true");
```

override_update_dns_on_lease_renewal( )

The override_update_dns_on_lease_renewal attribute controls whether the update_dns_on_lease_renewal value in the DHCP IPv6 Network Container is used, instead of the Grid default.

The override_update_dns_on_lease_renewal attribute can be specified explicitly. It is also set implicitly when update_dns_on_lease_renewal is set to a defined value.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

Set the parameter to "true" to override the Grid-level setting for update_dns_on_lease_renewal. Set the parameter to "false" to inherit the Grid-level setting for update_dns_on_lease_renewal.

The default value of this parameter is "false".

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
    #Getting override_update_dns_on_lease_renewal
    my $override_update_dns_on_lease_renewal=$network_container-
>override_update_dns_on_lease_renewal( );
    #Modifying override_update_dns_on_lease_renewal
    $network_container->override_update_dns_on_lease_renewal("true");
```

override_valid_lifetime( )

The override_valid_lifetime attribute controls whether the valid_lifetime value in the DHCP IPv6 Network Container is used, instead of the Grid default.

The override_valid_lifetime attribute can be specified explicitly. It is also set implicitly when valid_lifetime is set to a defined value.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

Set the parameter to "true" to override the Grid-level setting for options. Set the parameter to "false" to inherit the Grid-level setting for options.

The default value of this parameter is "false".

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Getting override_valid_lifetime
my $override_valid_lifetime=$network_container->override_valid_lifetime( );
#Modifying override_valid_lifetime
$network_container->override_valid_lifetime("true");
```

preferred_lifetime ( )

Use this method to set or retrieve the preferred lifetime value of a DHCP IPv6 Network Container object.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Setting this method to a defined value implicitly sets the override_preferred_lifetime method to "true". Setting the parameter to undefined causes the appliance to use the Grid default and automatically resets the override_preferred_lifetime attribute to "false".

Note that when preferred_lifetime is set to a defined value and override_preferred_lifetime is set to "false", the last operation takes precedence. Thus the sequence $object->preferred_lifetime(691200); $object->override_preferred_lifetime("false"); will set override_preferred_lifetime to "false", and the sequence $object->override_preferred_lifetime("false"); $object->preferred_lifetime(691200); will result in override_preferred_lifetime="true".

Parameter

A 32-bit integer (valid range from 1 to 4294967295) that represents the lifetime, in seconds. The default value is 27000.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get Preferred lifetime
my $preferred_lifetime = $network_container->preferred_lifetime();
#Modify Prefered lifetime
$network_container->preferred_lifetime(864000);
```

port_control_blackout_setting

Use this method to set or retrieve the port control blackout setting.

Setting this method to a defined value implicitly sets the override_blackout_setting method to "true". If discovery_blackout_setting is undefined, setting the parameter to undefined causes the appliance to use the Grid default and automatically resets the override_blackout_setting attribute to "false".

Note that when port_control_blackout_setting is set to a defined value and override_blackout_setting is set to "false", the last operation takes precedence. Thus the sequence $network->port_control_blackout_setting($blackout); $network->override_blackout_setting("false"); will set override_blackout_setting to "false", and the sequence $network->override_blackout_setting("false"); $network->port_control_blackout_setting($blackout); will result in override_blackout_setting="true".

Parameter

Valid value is an Infoblox::Grid::Discovery::Properties::BlackoutSetting object.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get port_control_blackout_setting
my $port_control_blackout_setting = $network->port_control_blackout_setting();
#Modify port_control_blackout_setting
$network->port_control_blackout_setting($blackout);
```

recycle_leases( )

Use this method to set or retrieve the recycle_leases flag at the network container level.

If the flag is enabled, the leases are kept in the recycle bin until one week after expiration. When the flag is disabled, the leases are irrecoverably deleted.

Setting this method to a defined value implicitly sets the override_recycle_leases method to "true". Setting the parameter to undefined causes the appliance to use the Grid default and automatically resets the override_recycle_leases attribute to "false".

Note that when recycle_leases is set to a defined value and override_recycle_leases is set to "false", the last operation takes precedence.

Thus the sequence $object->recycle_leases("true"); $object->override_recycle_leases("false"); will set override_recycle_leases to "false", and the sequence $objct->override_recycle_leases("false"); $object->recycle_leases("true"); will result in override_recycle_leases="true".

Parameter

Specify "true" to enable the recycle_leases flag or "false" to disable it. The default value is undefined.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get recycle_leases
my $recycle_leases = $network_container->recycle_leases();
#Modify recycle_leases
$network_container->recycle_leases("false");
```

remove_subnets( )

Use this method to set or retrieve the remove_subnets flag of a DHCP IPv6 Network Container object.

You can set this flag to false, if you want to remove the network, but keep all subnets under it. This flag is used only when removing DHCP IPv6 Network Container objects. It is not retrieved or used for any other operation.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

Specify "false" to keep the subnets, or "true" to remove them. The default value is undefined, which is to remove all subnets.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Set remove_subnets before submitting for removal
$network_container->remove_subnets("false");
#Remove the container of the subnets
$session->remove($network_container);
```

restart_if_needed( )

Use this method to set the flag that indicates whether DHCP service should be restarted on a member that is serving DHCP for the changed object referenced in a cloud API request.

Parameter

Specify true to restart the DHCP service.

Returns

None

Example

```
# Modify
$network->restart_if_needed('true');
```

rir( )

Use this method to retrieve the name of the Regional Internet Registry (RIR) that allocated this address space. This is a read-only attribute.

Parameter

None

Returns

The method returns the attribute value that can be 'RIPE' or 'NONE'.

Example

```
#Get rir
my $rir = $network_container->rir();
```

rir_organization( )

Use this method to set or retrieve the Regional Internet Registry (RIR) organization object.

Parameter

An Infoblox::Grid::RIR::Organization object.

You must enable support for RIR/SWIP at the Grid level before you can set attribute values. See Infoblox::Grid->enable_rir_swip() method.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get rir_organization
my $rir_organization = $network_container->rir_organization();
#Modify rir_organization
$network_container->rir_organization($rir_org);
```

rir_registration_action( )

Use this method to set the registration action of Regional Internet Registry (RIR). This is a write-only attribute.

Parameter

Valid values are 'NONE', 'CREATE', 'MODIFY' or 'DELETE'.

Returns

The method returns true when the modification succeeds, and returns false when the operation fails.

Example

```
#Set rir_registration_action
$network_container->rir_registration_action('CREATE');
```

rir_registration_status( )

Use this method to set or retrieve the registration status of Regional Internet Registry (RIR).

Parameter

The registration status of Regional Internet Registry (RIR), which can be 'REGISTERED' or 'NOT_REGISTERED'.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get rir_registration_status
my $rir_registration_status = $network_container->rir_registration_status();
#Modify rir_registration_status
$network_container->rir_registration_status('NOT_REGISTERED');
```

last_rir_registration_update_sent( )

Use this method to retrieve the last registration update timestamp of Regional Internet Registry (RIR). This is a read-only attribute.

Parameter

None

Returns

The method returns the attribute value.

Example

```
#Get last_rir_registration_update_sent
my $last_rir_registration_update_sent = $network_container->last_rir_registration_update_sent();
```

last_rir_registration_update_status( )

Use this method to retrieve the last registration update status of Regional Internet Registry (RIR). This is a read-only attribute.

Parameter

None

Returns

The method returns the attribute value.

Example

```
#Get last_rir_registration_update_status
my $last_rir_registration_update_status = $network_container->last_rir_registration_update_status();
```

same_port_control_discovery_blackout( )

Use this method to set or retrieve the flag that enables using the same discovery blackout for port control blackout.

Parameter

Specify 'true' to use same discovery blackout for port control blackout or 'false' to disable it. The default value is 'false'.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
# Get attribute value
my $value = $network_container->same_port_control_discovery_blackout();
# Modify attribute value
$network_container->same_port_control_discovery_blackout('true');
```

send_rir_request( )

Use this method to set the flag that indicates whether the appliance should send an API or email request for Regional Internet Registry (RIR). This is a write-only attribute.

Parameter

Specify "true" to set the send_rir_request flag or "false" to unset it.

Returns

The method returns true when the modification succeeds, and returns false when the operation fails.

Example

```
#Set send_rir_request
$network_container->send_rir_request('true');
```

subscribe_settings( )

Use this method to set or retrieve the Cisco ISE subscribe settings.

Setting this method to a defined value implicitly sets the override_subscribe_settings method to "true". Setting the parameter to undefined causes the appliance to use the Cisco ISE endpoint default and automatically resets the override_subscribe_settings method to "false".

Note that when subscribe_settings is set to a defined value and override_subscribe_settings is set to "false", the last operation takes precedence. Thus the sequence $object->subscribe_settings($value); $object->override_subscribe_settings("false"); will set override_subscribe_settings to "false", and the sequence $object->override_subscribe_settings("false"); $object->subscribe_settings($value); will result in override_subscribe_settings="true".

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

The valid value is an Infoblox::CiscoISE::SubscribeSetting object.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get subscribe_settings value
my $subscribe_settings = $object->subscribe_settings();
#Modify subscribe_settings value
$object->subscribe_settings($settings);
```

delete_reason( )

Use this method to specify the reason for deleting the Regional Internet Registry (RIR) registration request. This is a write-only attribute. This field is required only if 'rir_registration_action' is specified as 'DELETE'

Parameter

The reason for deleting the RIR registration request in string format with a maximum of 256 bytes.

Returns

The method returns true when the modification succeeds, and returns false when the operation fails.

Example

```
#Set delete_reason
$network_container->delete_reason('invalid');
```

unmanaged( )

Use this method to set or retrieve the network status, which indicates whether the network is unmanaged or not.

Parameter

Specify true to mark the network as unmanaged or false as managed.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
# Get network status
$unmanaged = $network_container->unmanaged();
# Modify network status
$network_container->unmanaged('true');
```

update_dns_on_lease_renewal( )

The update_dns_on_lease_renewal attribute controls whether the DHCP server updates DNS when a DHCP lease is renewed.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Setting this method to a defined value implicitly sets the override_update_dns_on_lease_renewal method to "true". Setting the parameter to undefined causes the appliance to use the Grid default and automatically resets the override_update_dns_on_lease_renewal attribute to "false".

Note that when update_dns_on_lease_renewal is set to a defined value and override_update_dns_on_lease_renewal is set to "false", the last operation takes precedence. Thus the sequence $object->update_dns_on_lease_renewal("true"); $object->override_update_dns_on_lease_renewal("false"); will set override_update_dns_on_lease_renewal to "false", and the sequence $object->override_update_dns_on_lease_renewal("false"); $object->update_dns_on_lease_renewal("true"); will result in override_update_dns_on_lease_renewal="true".

Parameter

Specify "true" to set the update_dns_on_lease_renewal flag or "false" to deactivate/unset it.

The default value of this parameter is "false".

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get update_dns_on_lease_renewal
my $update_dns_on_lease_renewal=$network_container->update_dns_on_lease_renewal( );
#Modify update_dns_on_lease_renewal
$network_container->update_dns_on_lease_renewal("true");
```

valid_lifetime ( )

Use this method to set or retrieve the valid lifetime value of a DHCP IPv6 Network Container object.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Setting this method to a defined value implicitly sets the override_valid_lifetime method to "true". Setting the parameter to undefined causes the appliance to use the Grid default and automatically resets the override_valid_lifetime attribute to "false".

Note that when valid_lifetime is set to a defined value and override_valid_lifetime is set to "false", the last operation takes precedence. Thus the sequence $object->valid_lifetime(691200); $object->override_valid_lifetime("false"); will set override_valid_lifetime to "false", and the sequence $object->override_valid_lifetime("false"); $object->valid_lifetime(691200); will result in override_valid_lifetime="true".

Parameter

A 32-bit integer (valid range from 1 to 4294967295) that represents the valid lifetime, in seconds. The default value is 43200.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get valid lifetime
my $valid_lifetime = $network_container->valid_lifetime();
#Modify valid lifetime
$network_container->valid_lifetime(864000);
```

zone_associations( )

Use this method to set or retrieve the associated zone information.

Parameter

The valid value is an array reference that contains Infoblox::DNS::Zone object(s). The default value is an empty list. If the network container has a parent, it is possible to use the zone associations of the parent by setting the parameter to undef.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails. If you did not specify a parameter, the method returns the attribute value.

Examples

```
#Getting zone associations
my $zone_associations = $network_container->zone_associations();
my $zone1 = Infoblox::DNS::Zone->new(
      name     => "test1.com",
);
my $zone2 = Infoblox::DNS::Zone->new(
      name     => "test2.com",
   );
#Modifying zone_associations
$network_container->zone_associations([$zone1, $zone2]);
#Clear zone_associations
$network_container->zone_associations([]);
##Remove the override for zone_associations
$network_container->zone_associations(undef)
```

SAMPLE CODE

The following sample code demonstrates the different functions that can be applied to an object such as add, search, modify, and remove. This sample also includes error handling for the operations.

#Preparation prior to a DHCP IPv6 Network Container object insertion

```
#PROGRAM STARTS: Include all the modules that will be used
```

```
use strict;
use Infoblox;

#Create a session to the Infoblox appliance

my $session = Infoblox::Session->new(
    master   => "192.168.1.2",
    username => "admin",
    password => "infoblox"
);
unless ($session) {
   die("Construct session failed: ",
       Infoblox::status_code() . ":" . Infoblox::status_detail());
}
print "Session created successfully\n";
```

#Create a DHCP IPv6 Network Container object

```
my $network_container = Infoblox::DHCP::IPv6NetworkContainer->new(
            network => "2001::/64",
            comment => "add network container",
             );

my $response = $session->add($network_container)
       or die("Add Network Container failed: ",
             $session->status_code() . ":" . $session->status_detail());

print "Network Container added successfully\n";

#Search for a specific Network Container

#Search the network container
my @retrieved_objs = $session->search(
    object  => "Infoblox::DHCP::IPv6NetworkContainer",
    network => "2001.*"
);
my $object = $retrieved_objs[0];
unless ($object) {
    die("Search Network Container failed: ",
        $session->status_code() . ":" . $session->status_detail());
}
print "Search Network Container found at least 1 matching entry\n";
```

#Get and modify an IPv6 Network Container attribute

```
#Get Network Container through the session
my @retrieved_objs = $session->get(
           object =>"Infoblox::DHCP::IPv6NetworkContainer" ,
           network => "2001::/64"
        );

my $object = $retrieved_objs[0];

unless ($object) {
    die("Get Network Container object failed: ",
        $session->status_code() . ":" . $session->status_detail());
}
print "Get Network Container found at least 1 matching entry\n";
```

#Modify one of the attributes of the specified IPv6 network container attribute

```
$object->comment("modified comment"); #Apply the changes

$session->modify($object)
    or die("Modify Network Container attribute failed: ",
           $session->status_code() . ":" . $session->status_detail());

print "Network Container object modified successfully \n";
```

#Remove a Network Container Object

```
 #Get Network Container object through the session
 my @retrieved_objs = $session->get(
           object  => "Infoblox::DHCP::IPv6NetworkContainer" ,
           network => "2001::"
       );
```

```
    my $object = $retrieved_objs[0];

    unless ($object) {
     die("Get Network Container object failed: ",
         $session->status_code() . ":" . $session->status_detail());
     }

 print "Get Network Container object found at least 1 matching entry\n";
```

#Submit the object for removal

```
 $session->remove($object)
     or die("Remove Network Container failed: ",
         $session->status_code() . ":" . $session->status_detail());

 print "Network Container removed successfully \n";
```

####PROGRAM ENDS####

AUTHOR

Infoblox Inc. http://www.infoblox.com/

SEE ALSO

Infoblox::Session, Infoblox::DHCP::View, Infoblox::DHCP::IPv6Network, Infoblox::DHCP::Network, Infoblox::DHCP::NetworkContainer, Infoblox::Session->get(), Infoblox::Session->search(), Infoblox::Session->add(), Infoblox::Session->remove(), Infoblox::Session->modify(), Infoblox::Grid::MSServer::AdUser::Data

---

COPYRIGHT

Copyright (c) 2017 Infoblox Inc.

Infoblox::DHCP::IPv6NetworkTemplate - DHCP IPv6 Network Template object

- NAME
- DESCRIPTION
- CONSTRUCTOR
- SESSION METHODS
  - Infoblox::Session->add( )
  - Infoblox::Session->get( )
  - Infoblox::Session->search( )
  - Infoblox::Session->modify( )
  - Infoblox::Session->remove( )
- METHODS
  - allow_any_netmask( )
  - auto_create_reversezone( )
  - cidr( )
  - comment( )
  - ddns_domainname( )
  - ddns_enable_option_fqdn( )
  - ddns_generate_hostname( )
  - ddns_server_always_updates( )
  - ddns_ttl( )
  - domain_name( )
  - domain_name_servers( )
  - enable_ddns( )
  - extattrs( )
  - extensible_attributes( )
  - fixed_address_templates( )
  - ipv6prefix( )
  - members( )
  - name( )
  - options( )
  - override_ddns_domainname( )
  - override_ddns_enable_option_fqdn( )
  - override_ddns_generate_hostname( )
  - override_ddns_ttl( )
  - override_domain_name( )
  - override_domain_name_servers( )
  - override_enable_ddns( )
  - override_options( )
  - override_preferred_lifetime( )
  - override_recycle_leases( )
  - override_update_dns_on_lease_renewal( )
  - override_valid_lifetime( )
  - preferred_lifetime( )
  - range_templates( )
  - recycle_leases( )
  - rir( )
  - rir_organization( )
  - rir_registration_action( )
  - rir_registration_status( )
  - send_rir_request( )
  - update_dns_on_lease_renewal( )
  - valid_lifetime( )
- SAMPLE CODE
- AUTHOR
- SEE ALSO
- COPYRIGHT

NAME

Infoblox::DHCP::IPv6NetworkTemplate - DHCP IPv6 Network Template object

---

DESCRIPTION

You can use a template to create IPv6 networks in a quick and consistent way. Networks created from a network template inherit all the properties defined in the network template, except for the comment and cidr if they are defined in the network.

CONSTRUCTOR

```
my $ipv6network_template = Infoblox::DHCP::IPv6NetworkTemplate->new(
```

```
        name                                => $string,
#Required
        allow_any_netmask                   => "true" | "false",
#Optional / Default is "false"
        auto_create_reversezone             => "true" | "false",
#Optional / Default is "false"
        cidr                                => $num,
#Optional / Default is undefined
        comment                             => $string,
#Optional / Default is empty
        ddns_domainname                     => $fqdn,
#Optional / Default is undefined
        ddns_enable_option_fqdn             => "true" | "false" ,
#Optional / Default is "false"
        ddns_generate_hostname              => "true" | "false" | undef,
#Optional / Default is undefined
        ddns_server_always_updates          => "true" | "false",
#Optional / Default is "true"
        ddns_ttl                            => $num,
#Optional / Default is 0
        domain_name                         => $fqdn,
#Optional / Default is undefined
        domain_name_servers                 => "$ipv6addr1,$ipv6addr2,...",
#Optional / Default is undefined
        enable_ddns                         => "true" | "false" | undef,
#Optional / Default is undefined
        extattrs                            => { $string => $extattr, ... },
#Required if 'rir_organization' attribute is defined

#Otherwise is optional
        extensible_attributes               => { $string => $string | $num, $string => [ $string |
$num, ... ], ... },

#Required if 'rir_organization' attribute is defined

#Otherwise is optional

        fixed_address_templates             => [$DhcpTemplate1,$DhcpTemplate2,...],
#Optional / Default is empty
        ipv6prefix                          => $ipv6prefix,
#Optional / Default is undefined
        members                             => [$Member1, $Member2, ...],
#Optional / Default is empty
        options                             => [$Option1, $Option2, ...],
#Optional / Default is empty list
        override_ddns_domainname            => "true" | "false",
#Optional / Default is "false"
        override_ddns_enable_option_fqdn    => "true" | "false",
#Optional / Default is "false"
        override_ddns_generate_hostname     => "true" | "false",
#Optional / Default is "false"
        override_ddns_ttl                   => "true" | "false",
#Optional / Default is "false"
        override_domain_name                => "true" | "false",
#Optional / Default is "false"
        override_domain_name_servers        => "true" | "false",
#Optional / Default is "false"
        override_enable_ddns                => "true" | "false",
#Optional / Default is "false"
        override_options                    => "true" | "false",
#Optional / Default is "false"
        override_preferred_lifetime         => "true" | "false",
#Optional / Default is "false"
        override_recycle_leases             => "true" | "false",
#Optional / Default is "false"
        override_update_dns_on_lease_renewal => "true" | "false",
#Optional / Default is "false"
        override_valid_lifetime             => "true" | "false",
#Optional / Default is "false"
        preferred_lifetime                  => $num,
#Optional / Default is 27000
        range_templates                     => [$RangeTemplate1,$RangeTemplate2,...],
#Optional / Default is empty
        recycle_leases                      => "true" | "false" | undef,
#Optional / Default is undefined
        rir_organization                    => $rir_org,
#Optional / An Infoblox::Grid::RIR::Organization object
        rir_registration_action             => 'NONE' | 'CREATE',
#Optional / Default is 'NONE'
        rir_registration_status             => 'NOT_REGISTERED' | 'REGISTERED',
#Optional / Default is 'NOT_REGISTERED'
        send_rir_request                    => 'true' | 'false',
#Optional / Default is 'false'
        update_dns_on_lease_renewal         => "true" | "false",
#Optional / Default is "false"
```

```
        valid_lifetime                          => $num,
#Optional / Default is 43200
        );
```

You cannot set both extattrs and extensible_attributes attributes at the same time.

SESSION METHODS

This section describes all the methods in an Infoblox::Session module that you can apply to a DHCP IPv6 network template object.

Infoblox::Session->add( )

Use this method to add an IPv6 network template object to the Infoblox appliance. See Infoblox::Session->add() for parameters and return values.

Example

```
#Construct an object
my $ipv6network_template = Infoblox::DHCP::IPv6NetworkTemplate->new(
       name               => "custom_network_template",
       allow_any_netmask => "true",
);

# Submit for addition
my $response = $session->add( $ipv6network_template );
```

Infoblox::Session->get( )

Use this method to retrieve the matching IPv6 network template objects from the Infoblox appliance. See Infoblox::Session->get() for parameters and return values.

Key References

```
Apply the following attributes to get a specific DHCP IPv6 network template object:

 name - Required. The name of network template.
 extattrs     - Optional. A hash reference containing extensible attributes.
```

Example

```
 my  @result_array = $session->get(
      object  => "Infoblox::DHCP::IPv6NetworkTemplate" ,
      name    => "custom_network_template"
 );
```

Infoblox::Session->search( )

Use this method to retrieve all the matching IPv6 network template objects from the Infoblox appliance. See Infoblox::Session->search() for parameters and return values.

Key References

```
Apply the following attributes to search for a specific DHCP IPv6 network template object:

 name     - Optional. The name of the network template (regular
            expression).
 comment - Optional. The comment of the network template (regular
            expression).
 extattrs     - Optional. A hash reference containing extensible attributes.
 extensible_attributes - Optional. A hash reference containing extensible attributes.
```

For more information about searching extensible attributes, see Infoblox::Grid::ExtensibleAttributeDef/Searching Extensible Attributes.

Example

```
 my  @result_array = $session->search(
      object  => "Infoblox::DHCP::IPv6NetworkTemplate" ,
      name    => "custom_network_template"
  comment => ".*",
 );

 # search all Network Templates with a given extensible attribute 'Site'
 my @retrieved_objs = $session->search(
   object => "Infoblox::DHCP::IPv6NetworkTemplate",
   extensible_attributes => { 'Site' => 'Santa Clara' });
```

Infoblox::Session->modify( )

Use this method to modify an IPv6 network template in the Infoblox appliance. See Infoblox::Session->modify() for parameters and return values.

Example

```
#Modify enable_ddns method
$object->enable_ddns("true");
# Submit modification
my $response = $session->modify( $object );
```

Infoblox::Session->remove( )

Use this method to remove an object from the Infoblox appliance. See Infoblox::Session->remove() for parameters and return values.

To remove a specific object, first use `get()` or `search()` to retrieve it, and then submit it for removal.

Example

```
# Get the object under the same name
my  @result_array = $session->get(

      object => "Infoblox::DHCP::IPv6NetworkTemplate" ,
      name   => "custom_network_template"
            );

# Find the desired object in the retrieved list
my $desired_network_template = $result_array[0];

# Submit for removal
my $response = $session->remove( $desired_network_template );
```

METHODS

This section describes all the methods that can be used to configure and retrieve the attribute values of a DHCP IPv6 network template object.

allow_any_netmask( )

Use this method to set or retrieve the allow_any_netmask parameter of a DHCP IPv6 network template object. If you set this parameter to "true", then the template allows any netmask. A user must specify a netmask when creating a network using this template. If you set this parameter to false, you must specify the "netmask" property for the network template object.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

Specify "true" to set the allow_any_netmask parameter or specify "false" if you want to specify a particular netmask. The default value is false.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
# Get allow_any_netmask
my $allow_any_netmask = $ipv6network_template->allow_any_netmask()
# Modify allow_any_netmask
$ipv6network_template->allow_any_netmask("true");
```

auto_create_reversezone( )

Use this method to set or retrieve the auto_create_reversezone flag of a DHCP IPv6 network template object. This flag indicates whether the appliance must automatically create the corresponding reverse zone.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

Specify "true" to create a reverse zone automatically or "false" to disable the automatic creation of a reverse zone.The default value for this field is false.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
# Get auto_create_reversezone
my $auto_create_reversezone = $ipv6network_template->auto_create_reversezone();
# Modify auto_create_reversezone
$ipv6network_template->auto_create_reversezone("true");
```

cidr( )

Use this method to set or retrieve the CIDR for the IPv6 network template object.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

Specify the CIDR as a number from 1 to 128.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get CIDR
my $cidr = $ipv6network_template->cidr();
#Modify CIDR
$ipv6network_template->cidr("68");
```

comment( )

Use this method to set or retrieve a descriptive comment about a DHCP IPv6 network template.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

Enter a descriptive comment for the network template in string format and with a maximum of 256 bytes.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
# Get comment
my $comment = $ipv6network_template->comment();
# Modify comment
$ipv6network_template->comment("add a network");
```

ddns_domainname( )

Use this method to set or retrieve the ddns_domainname value of a DHCP IPv6 Network Tempate object.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Setting this method to a defined value implicitly sets the override_ddns_domainname method to "true". Setting the parameter to undefined causes the appliance to use the grid default and automatically resets the override_ddns_domainname attribute to "false".

Note that when ddns_domainname is set to a defined value and override_ddns_domainname is set to "false", the last operation takes precedence. Thus the sequence $object->ddns_domainname("testdomain"); $object->override_ddns_domainname("false"); will set override_ddns_domainname to "false", and the sequence $object->override_ddns_domainname("false"); $object->ddns_domainname("testdomain"); will result in override_ddns_domainname="true".

Parameter

The dynamic DNS domain name in string format.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get ddns_domainname
my $ddns_domainname = $ipv6network_template->ddns_domainname();
#Modify ddns_domainname
$ipv6network_template->ddns_domainname("test_domain.com");
```

ddns_enable_option_fqdn( )

Use this method to set or retrieve the ddns_enable_option_fqdn flag of a DHCP IPv6 network template object. This method controls whether the FQDN option sent by the client is to be used, or if the server can automatically generate the FQDN. This setting overrides the upper-level settings.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Setting this method to a defined value implicitly sets the override_ddns_enable_option_fqdn method to "true". Setting the parameter to undefined causes the appliance to use the upper-level setting and automatically resets the override_ddns_enable_option_fqdn attribute to "false".

Note that when ddns_enable_option_fqdn is set to a defined value and override_ddns_enable_option_fqdn is set to "false", the last operation takes precedence. Thus the sequence $object->ddns_enable_option_fqdn("true"); $object->override_ddns_enable_option_fqdn("false"); will set override_ddns_enable_option_fqdn to "false", and the sequence $object->override_ddns_enable_option_fqdn("false"); $object->ddns_enable_option_fqdn("true"); will result in override_ddns_enable_option_fqdn="true".

Parameter

Specify "true" to generate the hostname or "false" to deactivate/unset it. The default value is undefined, which indicates that this attribute inherits the upper-level setting.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
# Get ddns_enable_option_fqdn
my $ddns_enable_option_fqdn = $ipv6network_template->ddns_enable_option_fqdn()
# Modify ddns_enable_option_fqdn
$ipv6network_template->ddns_enable_option_fqdn("true");
#Remove override
$network_template->ddns_enable_option_fqdn(undef);
```

ddns_generate_hostname( )

Use this method to set or retrieve the ddns_generate_hostname flag of a DHCP IPv6 network template object. Set this flag to allow only the DHCP server to generate a hostname. This setting overrides the upper-level settings.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Setting this method to a defined value implicitly sets the override_ddns_generate_hostname method to "true". Setting the parameter to undefined causes the appliance to use the upper-level setting and automatically resets the override_ddns_generate_hostname attribute to "false".

Note that when ddns_generate_hostname is set to a defined value and override_ddns_generate_hostname is set to "false", the last operation takes precedence. Thus the sequence $object->ddns_generate_hostname("true"); $object->override_ddns_generate_hostname("false"); will set override_ddns_generate_hostname to "false", and the sequence $object->override_ddns_generate_hostname("false"); $object->ddns_generate_hostname("true"); will result in override_ddns_generate_hostname="true".

Parameter

Specify "true" to generate the hostname or "false" to deactivate/unset it. The default value is undefined, which indicates that this attribute inherits the upper-level setting.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
# Get ddns_generate_hostname
```

```
my $ddns_generate_hostname = $ipv6network_template->ddns_generate_hostname()
# Modify ddns_generate_hostname
$ipv6network_template->ddns_generate_hostname("true");
#Remove override
$network_template->ddns_generate_hostname(undef);
```

ddns_server_always_updates( )

Use this method to set or retrieve the ddns_server_always_updates flag of a DHCP IPv6 network template object. Set this flag to allow the DHCP server only to update DNS, regardless of the requests from DHCP clients.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

Specify "true" to set this flag and allow only the DHCP server to update DNS or "false" to deactivate/unset the flag. The default value is true.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
# Get ddns_server_always_updates
my $ddns_server_always_updates = $ipv6network_template->ddns_server_always_updates()
# Modify ddns_server_always_updates
$ipv6network_template->ddns_server_always_updates("true");
```

ddns_ttl( )

Use this method to set or retrieve the DNS update Time to Live (TTL) value of a DHCP IPv6 network template object. The TTL value indicates the number of seconds the Infoblox appliance caches a DDNS update.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Setting this method to a defined value implicitly sets the override_ddns_ttl method to "true". Setting the parameter to undefined causes the appliance to use the grid default and automatically resets the override_ddns_ttl attribute to "false".

Note that when ddns_ttl is set to a defined value and override_dddns_ttl is set to "false", the last operation takes precedence. Thus the sequence $object->ddns_ttl(12); $object->override_ddns_ttl("false"); will set override_ddns_ttl to "false", and the sequence $object->override_ddns_ttl("false"); $object->ddns_ttl(12); will result in override_ddns_ttl="true".

Parameter

A 32-bit integer, from 0 to 4294967295, that represents the duration, in seconds, that the update is cached. Zero indicates that the update will not be cached. The default value is 0.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get DNS Update TTL
my $dns_ttl = $ipv6network_template->ddns_ttl();
#Modify DNS Update TTL
$ipv6network_template->ddns_ttl(1200);
```

domain_name( )

Use this method to set or retrieve the domain_name value of a DHCP IPv6 network template object.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Setting this method to a defined value implicitly sets the override_domain_name method to "true". Setting the parameter to undefined causes the appliance to use the grid default and automatically resets the override_domain_name attribute to "false".

Note that when domain_name is set to a defined value and override_domain_name is set to "false", the last operation takes precedence. Thus the sequence $object->domain_name("testdomain"); $object->override_domain_name("false"); will set override_domain_name to "false", and the sequence $object->override_domain_name("false"); $object->domain_name("testdomain"); will result in override_domain_name="true".

Parameter

The client's domain name in string format.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get domain_name
my $domain_name = $ipv6network_template->domain_name();
#Modify domain_name
$ipv6network_template->domain_name("test_domain.com");
```

domain_name_servers( )

Use this method to set or retrieve the domain_name_servers value of a DHCP IPv6 network template object.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Setting this method to a defined value implicitly sets the override_domain_name_servers method to "true". Setting the parameter to undefined causes the appliance to use the grid default and automatically resets the override_domain_name_servers attribute to "false".

Note that when domain_name_servers is set to a defined value and override_domain_name_servers is set to "false", the last operation takes precedence. Thus the sequence $object->domain_name_servers("2001::1,2001::2"); $object->override_domain_name_servers("false"); will set override_domain_name_servers to "false", and the sequence $object->override_domain_name_servers("false"); $object->domain_name_servers("2001::1,2001::2"); will result in override_domain_name_servers="true".

Parameter

The domain name server addresses in IPv6 address format.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get domain_name_servers
my $domain_name_servers = $ipv6network_template->domain_name_servers();
#Modify domain_name_servers
$ipv6network_template->domain_name_servers("2001::1,2001::2");
```

enable_ddns( )

Use this method to set or retrieve the dynamic DNS updates flag of a DHCP IPv6 network template object. The DHCP server can send DDNS updates to DNS servers in the same grid and to external DNS servers. This setting overrides the upper-level settings.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Setting this method to a defined value implicitly sets the override_enable_ddns method to "true". Setting the parameter to undefined causes the appliance to use the upper-level default and automatically resets the override_enable_ddns attribute to "false".

Note that when enable_ddns is set to a defined value and override_enable_ddns is set to "false", the last operation takes precedence. Thus the sequence $object->enable_ddns("true"); $object->override_enable_ddns("false"); will set override_enable_ddns to "false", and the sequence $object->override_enable_ddns("false"); $object->enable_ddns("true"); will result in override_enable_ddns="true".

Parameter

Specify "true" to set the dynamic DNS updates flag or "false" to deactivate/unset it. The default value is undefined, which indicates that this attribute inherits the upper-level setting.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
# Get enable_ddns
my $enable_ddns = $ipv6network_template->enable_ddns()
# Modify enable_ddns
$ipv6network_template->enable_ddns("true");
# Remove the enable_ddns override
```

```
$ipv6network_template->enable_ddns(undef);
```

extattrs( )

Use this method to set or retrieve the extensible attributes associated with an IPv6 network template object.

Parameter

Valid value is a hash reference containing the names of extensible attributes and their associated values (Infoblox::Grid::Extattr objects).

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get extattrs
my $ref_extattrs = $ipv6network_template->extattrs();
#Modify extattrs
$ipv6network_template->extattrs({ 'Site' => $extattr1, 'Administrator' => $extattr2 });
```

extensible_attributes( )

Use this method to set or retrieve the extensible attributes associated with an IPv6 network template object.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

For valid values for extensible attributes, see Infoblox::Grid::ExtensibleAttributeDef/Extensible Attribute Values.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get extensible attributes
my $ref_extensible_attributes = $ipv6network_template->extensible_attributes();
#Modify extensible attributes
$ipv6network_template->extensible_attributes({ 'Site' => 'Santa Clara', 'Administrator' => [
'Peter', 'Tom' ] });
```

fixed_address_templates( )

Use this method to set or retrieve the IPv6 fixed address template that is assigned to this IPv6 network template. When you create a network based on a network template that includes a fixed address template, the fixed addresses are created based on the associated fixed address template.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

The valid value is an array reference of Infoblox::DHCP::Template objects.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
# Get fixed_address_templates
my $fixed_address_templates = $ipv6network_template->fixed_address_templates();
# Modify fixed_address_templates
my $tmpl = Infoblox::DHCP::Template->new(
   name => "custom_fixed_address_template"
);
$network_template->fixed_address_templates([$tmpl]);
```

ipv6prefix( )

Use this method to set or retrieve the IPv6 prefix of a DHCP IPv6 network template object. The prefix must belong to the list of global prefixes defined in the Infoblox::Grid::DHCP object.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

IPv6 prefix from the list of global IPv6 prefixes.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails. ls.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get ipv6prefix
my $ipv6prefix = $ipv6network_template->ipv6prefix();
#Modify ipv6prefix
$griddhcp->ipv6prefix("2001::");
```

members( )

Use this method to set or retrieve the member assigned to the IPv6 network template object.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

The valid value is an array reference that contains Infoblox::DHCP::Member objects.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
my $member1 = Infoblox::DHCP::Member->new(
    name     => "infoblox.localdomain",
    ipv4addr => "192.168.1.2",
);

#Create a network template with this member
my $ipv6network_template = Infoblox::DHCP::IPv6NetworkTemplate->new(
    name    => "custom_network_template",
    members => [$member1],
);
```

name( )

Use this method to set or retrieve the name of the DHCP IPv6 network template.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

The network template name in string format and with a maximum of 64 bytes.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get name
my $name = $ipv6network_template->name();
#Modify name
$ipv6network_template->name("custom_network_template");
```

options( )

Use this method to set or retrieve the options value of an IPv6 network template object. DHCP options provide network configuration

settings and various services available on the network for DHCP. This setting overrides the upper-level setting. The default value is undefined which indicates that this attribute inherit the upper-level setting.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Setting this method to a defined value implicitly sets the override_options method to "true". Setting the parameter to undefined causes the appliance to use the upper-level default and automatically resets the override_options attribute to "false".

Note that when the options method is set to a defined value and override_options is set to "false", the last operation takes precedence. Thus the sequence $object->options([$opt]); $object->override_options("false"); will set override_options to "false", and the sequence $object->override_options("false"); $object->options([$opt]); will result in override_options="true".

Parameter

The valid value is an array reference that contains Infoblox::DHCP::Option objects.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get options
my $options = $ipv6network_template->options();
#Modify options
my $option1 = Infoblox::DHCP::Option->new(
    name   => "dhcp6.rapid-commit",
    value  => "true"
);
$ipv6network_template->options([$option1]);
```

override_ddns_domainname( )

The override_ddns_domainname attribute controls whether the ddns_domainname value in the DHCP IPv6 Network Template is used, instead of the grid default.

The override_ddns_domainname attribute can be specified explicitly. It is also set implicitly when ddns_domainname is set to a defined value.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

Set the parameter to "true" to override the grid-level setting for ddns_domainname. Set the parameter to "false" to inherit the grid-level setting for ddns_domainname.

The default value of this parameter is "false".

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Getting override_ddns_domainname
my $override_ddns_domainname=$ipv6network_template->override_ddns_domainname( );
#Modifying override_ddns_domainname
$ipv6network_template->override_ddns_domainname("true");
```

override_ddns_enable_option_fqdn( )

The override_ddns_enable_option_fqdn attribute controls whether the ddns_enable_option_fqdn value in the DHCP IPv6 network template is used, instead of the grid default.

The override_ddns_enable_option_fqdn attribute can be specified explicitly. It is also set implicitly when ddns_domainname is set to a defined value.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

Set the parameter to "true" to override the grid-level setting for ddns_enable_option_fqdn. Set the parameter to "false" to inherit the grid-level setting for ddns_enable_option_fqdn.

The default value of this parameter is "false".

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get override_ddns_enable_option_fqdn
my $override_ddns_enable_option_fqdn=$ipv6network_template->override_ddns_enable_option_fqdn( );
#Modify override_ddns_enable_option_fqdn
$ipv6network_template->override_ddns_enable_option_fqdn("true");
```

override_ddns_generate_hostname( )

The override_ddns_generate_hostname attribute controls whether the ddns_generate_hostname value in the DHCP IPv6 network template is used, instead of the grid default.

The override_ddns_generate_hostname attribute can be specified explicitly. It is also set implicitly when ddns_domainname is set to a defined value.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

Set the parameter to "true" to override the grid-level setting for ddns_generate_hostname. Set the parameter to "false" to inherit the grid-level setting for ddns_generate_hostname.

The default value of this parameter is "false".

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get override_ddns_generate_hostname
my $override_ddns_generate_hostname=$ipv6network_template->override_ddns_generate_hostname( );
#Modify override_ddns_generate_hostname
$ipv6network_template->override_ddns_generate_hostname("true");
```

override_ddns_ttl( )

The override_ddns_ttl attribute controls whether the ddns_ttl value in the DHCP IPv6 network template object is used, instead of the grid default.

The override_ddns_ttl attribute can be specified explicitly. It is also set implicitly when ddns_ttl is set to a defined value.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

Set the parameter to "true" to override the grid-level setting for ddns_ttl. Set the parameter to "false" to inherit the grid-level setting for ddns_ttl.

The default value of this parameter is "false".

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get override_ddns_ttl
my $override_ddns_ttl=$ipv6network_template->override_ddns_ttl( );
#Modify override_ddns_ttl
$ipv6network_template->override_ddns_ttl("true");
```

override_domain_name( )

The override_domain_name attribute controls whether the domain_name value in the DHCP IPv6 network template is used, instead of the grid default.

The override_domain_name attribute can be specified explicitly. It is also set implicitly when domain_name is set to a defined value.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

Set the parameter to "true" to override the grid-level setting for domain_name. Set the parameter to "false" to inherit the grid-level setting for domain_name.

The default value of this parameter is "false".

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Getting override_domain_name
my $override_domain_name=$ipv6network_template->override_domain_name( );
#Modifying override_domain_name
$ipv6network_template->override_domain_name("true");
```

override_domain_name_servers( )

The override_domain_name_servers attribute controls whether the domain_name_servers value in the DHCP IPv6 network template is used, instead of the grid default.

The override_domain_name_servers attribute can be specified explicitly. It is also set implicitly when domain_name_servers is set to a defined value.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

Set the parameter to "true" to override the grid-level setting for domain_name_servers. Set the parameter to "false" to inherit the grid-level setting for domain_name_servers.

The default value of this parameter is "false".

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get override_domain_name_servers
my $override_domain_name_servers=$ipv6network_template->override_domain_name_servers( );
#Modify override_domain_name_servers
$ipv6network_template->override_domain_name_servers("true");
```

override_enable_ddns( )

The override_enable_ddns attribute controls whether the enable_ddns value in the DHCP IPv6 network template is used, instead of the grid default.

The override_enable_ddns attribute can be specified explicitly. It is also set implicitly when enable_ddns is set to a defined value.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

Set the parameter to "true" to override the grid-level setting for enable_ddns. Set the parameter to "false" to inherit the grid-level setting for enable_ddns.

The default value of this parameter is "false".

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get override_enable_ddns
```

```
my $override_enable_ddns=$ipv6network_template->override_enable_ddns( );
#Modify override_enable_ddns
$ipv6network_template->override_enable_ddns("true");
```

override_options( )

The override_options attribute controls whether the options in the DHCP IPv6 network template are used, instead of the grid default.

The override_options attribute can be specified explicitly. It is also set implicitly when options is set to a defined value.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

Set the parameter to "true" to override the grid-level setting for options. Set the parameter to "false" to inherit the grid-level setting for options.

The default value of this parameter is "false".

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get override_options
my $override_options=$ipv6network_template->override_options( );
#Modify override_options
$ipv6network_template->override_options("true");
```

override_preferred_lifetime( )

The override_preferred_lifetime attribute controls whether the preferred_lifetime value in the DHCP IPv6 network template is used, instead of the grid default.

The override_preferred_lifetime attribute can be specified explicitly. It is also set implicitly when preferred_lifetime is set to a defined value.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

Set the parameter to "true" to override the grid-level setting for options. Set the parameter to "false" to inherit the grid-level setting for options.

The default value of this parameter is "false".

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get override_preferred_lifetime
my $override_preferred_lifetime=$ipv6network_template->override_preferred_lifetime( );
#Modify override_preferred_lifetime
$ipv6network_template->override_preferred_lifetime("true");
```

override_recycle_leases( )

The override_recycle_leases attribute controls whether the recycle_leases value in the DHCP IPv6 network template is used, instead of the grid default.

The override_recycle_leases attribute can be specified explicitly. It is also set implicitly when recycle_leases is set to a defined value.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

Set the parameter to "true" to override the grid-level setting for options. Set the parameter to "false" to inherit the grid-level setting for options.

The default value of this parameter is "false".

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get override_recycle_leases
my $override_recycle_leases=$ipv6network_template->override_recycle_leases( );
#Modify override_recycle_leases
$ipv6network_template->override_recycle_leases("true");
```

override_update_dns_on_lease_renewal( )

The override_update_dns_on_lease_renewal attribute controls whether the update_dns_on_lease_renewal value in the DHCP IPv6 network template is used, instead of the grid default.

The override_update_dns_on_lease_renewal attribute can be specified explicitly. It is also set implicitly when update_dns_on_lease_renewal is set to a defined value.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

Set the parameter to "true" to override the grid-level setting for update_dns_on_lease_renewal. Set the parameter to "false" to inherit the grid-level setting for update_dns_on_lease_renewal.

The default value of this parameter is "false".

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get override_update_dns_on_lease_renewal
my $override_update_dns_on_lease_renewal=$ipv6network_template-
>override_update_dns_on_lease_renewal( );
#Modify override_update_dns_on_lease_renewal
$ipv6network_template->override_update_dns_on_lease_renewal("true");
```

override_valid_lifetime( )

The override_valid_lifetime attribute controls whether the valid_lifetime value in the DHCP IPv6 network template object is used, instead of the upper-level default.

The override_valid_lifetime attribute can be specified explicitly. It is also set implicitly when valid_lifetime is set to a defined value.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

Set the parameter to "true" to override the grid-level setting for valid_lifetime. Set the parameter to "false" to inherit the upper-level setting for valid_lifetime.

The default value of this parameter is "false".

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get override_valid_lifetime
my $override_valid_lifetime=$ipv6network_template->override_valid_lifetime( );
#Modify override_valid_lifetime
$ipv6network_template->override_valid_lifetime("true");
```

preferred_lifetime( )

Use this method to set or retrieve the valid lifetime of the DHCP IPv6 network template object.

Setting this method to a defined value implicitly sets the override_preferred_lifetime method to "true". Setting the parameter to undefined

causes the appliance to use the upper-level default and automatically resets the override_preferred_lifetime attribute to "false".

Note that when preferred_lifetime is set to a defined value and override_preferred_lifetime is set to "false", the last operation takes precedence. Thus the sequence $object->preferred_lifetime(20000); $object->override_preferred_lifetime("false"); will set override_preferred_lifetime to "false", and the sequence $object->override_preferred_lifetime("false"); $object->preferred_lifetime(20000); will result in override_preferred_lifetime="true".

Parameter

Preferred lifetime value in seconds. The default value is 27000. Note that the preferred lifetime cannot be greater than the valid lifetime.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get preferred_lifetime
my $preferred_lifetime = $ipv6network_template->preferred_lifetime();
#Modify preferred_lifetime
$ipv6network_template->preferred_lifetime(12345);
```

range_templates( )

Use this method to set or retrieve the IPv6 address range template assigned to this IPv6 network template object. When you create a network based on a network template object with a range template, the IPv6 address ranges are created based on the associated IPv6 address range template.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

The valid value is an array reference of Infoblox::DHCP::Template objects.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get range_templates
my $range_templates = $ipv6network_template->range_templates();
#Modify range_templates
my $tmpl = Infoblox::DHCP::Template->new(
   name => "custom_range_template"
);
$ipv6network_template->range_templates([ $tmpl ]);
```

recycle_leases( )

Use this method to set or retrieve the recycle_leases flag for an IPv6 network template object.

If the flag is enabled, the leases are kept in the recycle bin until one week after expiration. When the flag is disabled, the leases are irrecoverably deleted.

Setting this method to a defined value implicitly sets the override_recycle_leases method to "true". Setting the parameter to undefined causes the appliance to use the grid default and automatically resets the override_recycle_leases attribute to "false".

Note that when recycle_leases is set to a defined value and override_recycle_leases is set to "false", the last operation takes precedence. Thus the sequence $object->recycle_leases("true"); $object->override_recycle_leases("false"); will set override_recycle_leases to "false", and the sequence $objct->override_recycle_leases("false"); $object->recycle_leases("true"); will result in override_recycle_leases="true".

Parameter

Specify "true" to enable the recycle_leases flag or "false" to disable it. The default value is undefined.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get recycle_leases
my $recycle_leases = $ipv6network_template->recycle_leases();
#Modify recycle_leases
$ipv6network_template->recycle_leases("false");
```

rir( )

Use this method to retrieve the name of the Regional Internet Registry (RIR) that allocated this address space. This is a read-only attribute.

Parameter

None

Returns

The method returns the attribute value that can be 'RIPE' or 'NONE'.

Example

```
#Get rir
my $rir = $ipv6network_template->rir();
```

rir_organization( )

Use this method to set or retrieve the Regional Internet Registry (RIR) organization object.

Parameter

An Infoblox::Grid::RIR::Organization object.

You must enable support for RIR/SWIP at the Grid level before you can set attribute values. See Infoblox::Grid->enable_rir_swip() method.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get rir_organization
my $rir_organization = $ipv6network_template->rir_organization();
#Modify rir_organization
$ipv6network_template->rir_organization($rir_org);
```

rir_registration_action( )

Use this method to set or retrieve the registration action of Regional Internet Registry (RIR).

Parameter

Valid values are 'NONE', 'CREATE'.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get rir_registration_action
my $rir_registration_action = $ipv6network_template->rir_registration_action();
#Modify rir_registration_action
$ipv6network_template->rir_registration_action('CREATE');
```

rir_registration_status( )

Use this method to set or retrieve the registration status of Regional Internet Registry (RIR).

Parameter

The registration status of Regional Internet Registry (RIR), which can be 'REGISTERED' or 'NOT_REGISTERED'.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get rir_registration_status
my $rir_registration_status = $ipv6network_template->rir_registration_status();
#Modify rir_registration_status
$ipv6network_template->rir_registration_status('NOT_REGISTERED');
```

send_rir_request( )

Use this method to set or retrieve the flag that indicates whether the appliance should send an API or email request for Regional Internet Registry (RIR).

Parameter

Specify "true" to set the send_rir_request flag or "false" to unset it.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get send_rir_request
my $send_rir_request = $ipv6network_template->send_rir_request();
#Modify send_rir_request
$ipv6network_template->send_rir_request('true');
```

update_dns_on_lease_renewal( )

The update_dns_on_lease_renewal attribute controls whether the DHCP server updates DNS when a DHCP lease is renewed.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Setting this method to a defined value implicitly sets the override_update_dns_on_lease_renewal method to "true". Setting the parameter to undefined causes the appliance to use the grid default and automatically resets the override_update_dns_on_lease_renewal attribute to "false".

Note that when update_dns_on_lease_renewal is set to a defined value and override_update_dns_on_lease_renewal is set to "false", the last operation takes precedence. Thus the sequence $object->update_dns_on_lease_renewal("true"); $object->override_update_dns_on_lease_renewal("false"); will set override_update_dns_on_lease_renewal to "false", and the sequence $object->override_update_dns_on_lease_renewal("false"); $object->update_dns_on_lease_renewal("true"); will result in override_update_dns_on_lease_renewal="true".

Parameter

Specify "true" to set the update_dns_on_lease_renewal flag or "false" to deactivate/unset it.

The default value of this parameter is "false".

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get update_dns_on_lease_renewal
my $update_dns_on_lease_renewal=$ipv6network_template->update_dns_on_lease_renewal( );
#Modify update_dns_on_lease_renewal
$ipv6network_template->update_dns_on_lease_renewal("true");
```

valid_lifetime( )

Use this method to set or retrieve the valid lifetime of the DHCP IPv6 network template object.

Setting this method to a defined value implicitly sets the override_valid_lifetime method to "true". Setting the parameter to undefined causes the appliance to use the upper-level default and automatically resets the override_valid_lifetime attribute to "false".

Note that when valid_lifetime is set to a defined value and override_valid_lifetime is set to "false", the last operation takes precedence. Thus the sequence $object->valid_lifetime(20000); $object->override_valid_lifetime("false"); will set override_valid_lifetime to "false", and the sequence $object->override_valid_lifetime("false"); $object->valid_lifetime(20000); will result in override_valid_lifetime="true".

Parameter

Valid lifetime value in seconds. The default value is 43200.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get valid_lifetime
my $valid_lifetime = $ipv6network_template->valid_lifetime();
#Modify valid_lifetime
$ipv6network_template->valid_lifetime(12345);
```

SAMPLE CODE

The following sample code demonstrates the different functions that can be applied to an object such as add, search, modify, and remove. This sample also includes error handling for the operations.

```
#Preparation prior to a DHCP IPv6 network template object insertion
```

#PROGRAM STARTS: Include all the modules that will be used use strict; use Infoblox; **#Create a session to the Infoblox DNSone appliance** my $session = Infoblox::Session->new(master => "192.168.1.2", username => "admin", password => "infoblox");

```
unless ($session) {
    die("Construct session failed: ",
        Infoblox::status_code() . ":" . Infoblox::status_detail());
}
print "Session created successfully\n";

my $memberdhcp = Infoblox::DHCP::Member->new(
    name     => $host_name,
    ipv4addr => $node_ip
);
```

#Add IPv6 network template object

```
#Create a DHCP network template object

my $network_template = Infoblox::DHCP::IPv6NetworkTemplate->new(
    name                   => "custom_network_template",
    cidr                   => 64,
    comment                => "add IPv6 network template",
    members                => [ $memberdhcp ],
    enable_ddns            => "false",
    valid_lifetime         => 99999,
    auto_create_reversezone => "true",
);

my $response = $session->add($network_template)
    or die("Add Network template failed: ",
    $session->status_code() . ":" . $session->status_detail());

print "Network template added successfully\n";
```

#Get and modify an IPv6 network template attribute

```
#Get Network template through the session
my  @retrieved_objs = $session->get(
    object => "Infoblox::DHCP::IPv6NetworkTemplate",
    name   => "custom_network_template"
);

my $object = $retrieved_objs[0];

unless ($object) {
    die("Get Network template object failed: ",
        $session->status_code() . ":" . $session->status_detail());
}
print "Get Network template found at least 1 matching entry\n";
```

#Search an IPv6 network template attribute

```
#Search Network template through the session
```

```
my  @retrieved_objs = $session->search(
    object => "Infoblox::DHCP::IPv6NetworkTemplate",
    name   => "custom_network_template"
);

my $object = $retrieved_objs[0];

unless ($object) {
    die("Search Network template object failed: ",
        $session->status_code() . ":" . $session->status_detail());
}
print "Search Network template found at least 1 matching entry\n";

#Modify attributes of the specified IPv6 network template

$object->enable_ddns("true"); #Apply the changes

$session->modify($object)
    or die("Modify Network template attribute failed: ",
    $session->status_code() . ":" . $session->status_detail());

print "Network template object modified successfully \n";
```

#Remove an IPv6 network template object

```
#Get IPv6 Network Template object through the session

my @retrieved_objs = $session->get(
    object  => "Infoblox::DHCP::IPv6NetworkTemplate",
    name    => "custom_network_template"
);

my $object = $retrieved_objs[0];

unless ($object) {
    die("Get IPv6 Network Template object failed: ",
        $session->status_code() . ":" . $session->status_detail());
}

print "Get IPv6 Network Template object found at least 1 matching entry\n";

#Submit the object for removal

$session->remove($object)
    or die("Remove IPv6 Network Template failed: ",
    $session->status_code() . ":" . $session->status_detail());

print "IPv6 Network Template removed successfully \n";

####PROGRAM ENDS####
```

AUTHOR

Infoblox Inc. http://www.infoblox.com/

SEE ALSO

Infoblox::Session, Infoblox::DHCP::IPv6Network, Infoblox::Grid::DHCP, Infoblox::DHCP::Template, Infoblox::DHCP::Member, Infoblox::Session->get(), Infoblox::Session->search(), Infoblox::Session->add(), Infoblox::Session->remove(), Infoblox::Session->modify()

COPYRIGHT

Copyright (c) 2017 Infoblox Inc.

Infoblox::DHCP::IPv6OptionDefinition - DHCP IPv6 option definition object

- NAME
- DESCRIPTION
- CONSTRUCTOR
- SESSION METHODS
  - Infoblox::Session->add( )
  - Infoblox::Session->get( )
  - Infoblox::Session->modify( )
  - Infoblox::Session->search( )
  - Infoblox::Session->remove( )
- METHODS
  - name( )
  - code( )
  - space( )
  - type( )
- SAMPLE CODE
- AUTHOR
- SEE ALSO
- COPYRIGHT

NAME

Infoblox::DHCP::IPv6OptionDefinition - DHCP IPv6 option definition object

---

DESCRIPTION

An IPv6OptionDefinition defines an IPv6 DHCP option within a specific IPv6 option space. A custom option can be defined in the predefined DHCP option space or in the user-defined vendor option space. To define an option, add the option definition to the required option space.

CONSTRUCTOR

```
my $ipv6od = Infoblox::DHCP::IPv6OptionDefinition->new(
                name  => $string, #Required
                code  => $num,    #Required
                space => $string, #Optional / Default is DHCP
                type  => $string, #Optional / Default is string
             );
```

SESSION METHODS

This section describes all the methods in an Infoblox::Session module that can be applied to a DHCP IPv6 option definition object.

Infoblox::Session->add( )

Use this method to add an IPv6 option definition object to the Infoblox appliance. See Infoblox::Session->add() for parameters and return values.

Example

```
#Create a DHCP IPv6OptionDefinition object.
my $ipv6od = Infoblox::DHCP::IPv6OptionDefinition->new(
                space => "SUNW",
                name  => "JumpStart-server",
                code  => "6",
                type  => "text"
             );
# Submit for adding
my $response = $session->add( $ipv6od );
```

Infoblox::Session->get( )

Use this method to retrieve all the matching DHCP IPv6 option definition objects from the Infoblox appliance. See Infoblox::Session->get() for parameters and return values.

Key References

```
Apply the following attributes to get a specific DHCP IPv6 option definition object:

 name  - A name for the option definition in string format (regular expression).
```

```
  code  - Option number. Either a name or code is required.
  space - Optional. The option space in which the option is defined. The default is the "DHCPv6"
option space.
```

Example

```
#Getting option by name
my  @result_array = $session->get(
                        object   => "Infoblox::DHCP::IPv6OptionDefinition",
                        space    => "SUNW",
                        name     => "JumpStart-server",
                        );

#Getting option by code
my @result_array = $session->get(
                        object   => "Infoblox::DHCP::IPv6OptionDefinition",
                        space    => "DHCPv6",
                        code     => 176,
                        );
```

Infoblox::Session->modify( )

Use this method to modify a DHCP IPv6 option definition object in the Infoblox appliance. See Infoblox::Session->modify() for parameters and return values.

Example

```
#Use method to modify code
$object->code(20);
# Submit modification
my $response = $session->modify($object);
```

Infoblox::Session->search( )

Use this method to retrieve all the matching DHCP IPv6 option definition objects from the Infoblox appliance. See Infoblox::Session->search() for parameters and return values.

Key References

```
Apply the following attributes to get a specific DHCP IPv6 option definition object:

  name  - A name for the option definition in string format (regular expression).
  code  - Option number. Either a name or code is required.
  space - Optional. The option space in which the option is defined. The default is the "DHCPv6"
option space.
```

Example

```
#Getting option by name
my  @result_array = $session->search(
                        object   => "Infoblox::DHCP::IPv6OptionDefinition",
                        space    => "SUNW",
                        name     => "JumpStart-server",
                        );

#Getting option by code
my @result_array = $session->search(
                        object   => "Infoblox::DHCP::IPv6OptionDefinition",
                        space    => "DHCPv6",
                        code     => 176,
                        );
```

Infoblox::Session->remove( )

Use this method to remove a DHCP IPv6 option definition object from the Infoblox appliance. See Infoblox::Session->remove() for parameters and return values.

To remove a specifc object, first use `get()` to retrieve the specific object, and then submit this object for removal.

Example

```
my  @result_array = $session->get(
                        object   => "Infoblox::DHCP::IPv6OptionDefinition",
                        space    => "SUNW",
                        name     => "JumpStart-server",
                        );

# find the desired object from retrieved list.
```

```
my $desired_obj = $result_array[0];

# Submit for removal
my $response = $session->remove($desired_obj);
```

METHODS

This section describes all the methods that can be used to configure and retrieve the attribute values of a DHCP IPv6 option definition object.

name( )

Use this method to set or retrieve the name of a DHCP IPv6 option definition object.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

Desired name in string format with a maximum of 256 bytes.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
# Get name
my $od_name = $ipv6od->name();
# Modify name
$ipv6od->name("Start-server");
```

code( )

Use this method to set or retrieve the code of a DHCP IPv6 option definition object. An option code number is used to identify the DHCP option.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

Number for the option code. Value must be between 1 and 254. Values 2, 7, 12, 14, 21-14, 27-34, and 39 are predefined. These option codes have corresponding predefined Types. Therefore, you cannot change the predefined Types.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
# Get code
my $od_code = $ipv6od->code();
# Modify code
$ipv6od->code(14);
```

space( )

Use this method to set or retrieve the space of a DHCP IPv6 option definition object.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

Specify the space name in string format.

Returns

If you did not specify a parameter, the method returns the attribute value.

Example

```
# Get space
my $od_space = $ipv6od->space();
# Modify space
$ipv6od->space("SUNW");
```

type( )

Use this method to set or retrieve the data type of the grid DHCP IPv6 option.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

The accepted types are the following:

```
  "boolean"                 - For "true" or "false" boolean values.

  "ip-address"              - For a single IPv6 address. An IPv6 address is a 128-bit number.
                              It consist of eight groups of hexadecimal digits separated
                              by colon (example: "2001:1dfc:2dfb:0:c1:0:a0:a2"). A group of all zeros
                              can be replaced by a double colon (e.g. 2001:1dfc:2dfb::1 is equivalent
                              to 2001:1dfc:2dfb:0:0:0:0:1).

  "array of ip-address"     - For a list of IPv6 addresses, each separated by a comma.
                              (example: "2002:d0:d2::1, 2002:d0:d2::3, 2001::3")

  "string"                  - For value in ASCII text string or
                              a list of hexadecimal characters (0-9, a-f, or A-F) without spaces and
separated by colons.

  "text"                    - For value in ASCII text string.

  "domain-name"             - For domain name in ASCII text format

  "domain-list"             - For a list of domain names, each separated by a comma.

  "8-bit unsigned integer"  - For numeric value between 0 and 255.

  "16-bit unsigned integer" - For numeric value between 0 and 65535.

  "32-bit unsigned integer" - For numeric value between 0 and 4294967295.

  "8-bit signed integer"    - For numeric value between -128 and 127.

  "16-bit signed integer"   - For numeric value between -32768 and 32767.

  "32-bit signed integer"   - For numeric value between -2147483648 and 2147483647.

  "array of 8-bit unsigned integer"  - For a list of numeric values between 0 and 255, each separated
by a comma.

  "array of 16-bit unsigned integer" - For a list of numeric values between 0 and 65535, each
separated by a comma.

  "array of 32-bit unsigned integer" - For a list of numeric values between 0 and 4294967295, each
separated by a comma.

  "array of 8-bit integer"           - For a list of numeric values between -128 and 127, each
separated by a comma.

  "array of 16-bit integer"          - For a list of numeric values between -32768 and 32767, each
separated by a comma.

  "array of 32-bit integer"          - For a list of numeric values between -2147483648 and
2147483647, each separated by a comma.
```

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
# Get type
my $od_type = $ipv6od->type();
# Modify type
$ipv6od->type("string");
```

SAMPLE CODE

The following sample code demonstrates the different functions that can be applied to an object such as add, modify, and remove. Also, this sample includes error handling for the operations.

#Preparation prior to a DHCP Network insertion

```
#PROGRAM STARTS: Include all the modules that will be used
use strict;
```

```
use Infoblox;

#Create a session to the Infoblox appliance

my $session = Infoblox::Session->new(
    master   => "192.168.1.2",
    username => "admin",
    password => "infoblox"
);
unless ($session) {
   die("Construct session failed: ",
       Infoblox::status_code() . ":" . Infoblox::status_detail());
}
print "Session created successfully\n";

my $ipv6od = Infoblox::DHCP::IPv6OptionSpace->new("name" => "SUNW");
my $response = $session->add($ipv6od);
print "Add option space operation success \n" if ($response);
print "Add option space operation failed  \n" if (not $response);
```

#Create a DHCP IPv6 OptionDefinition object.

```
my $ipv6od = Infoblox::DHCP::OptionDefinition->new(
                                space => "SUNW",
                                name  => "JumpStart-server",
                                code  => "6",
                                type  => "text"
                              );
unless ($ipv6od) {
    die("Construct DHCP IPv6OptionDefinition failed: ",
    Infoblox::status_code() . ":" . Infoblox::status_detail());
}
print "DHCP IPv6OptionDefinition object created successfully\n";

#Add the DHCP IPv6OptionDefinition object to Infoblox appliance through a session
my $response = $session->add( $ipv6od )
   or die("Add a DHCP IPv6OptionDefinition object failed: ",
          $session->status_code() . ":" . $session->status_detail());

print "Add a DHCP IPv6OptionDefinition object successfully\n";
```

#Get and modify a DHCP OptionDefinition object attribute

```
#Get Network through the session
my @retrieved_objs = $session->get(
               object   => "Infoblox::DHCP::IPv6OptionDefinition",
               space  => "SUNW",
               name   => "JumpStart-server",
            );

my $object = $retrieved_objs[0];

unless ($object) {
    die("Get DHCP IPv6OptionDefinition object failed: ",
        $session->status_code() . ":" . $session->status_detail());
}
print "Get DHCP IPv6OptionDefinition object found at least 1 matching entry\n";

#Modify one of the attributes of the specified Network attribute

$object->code(20);    #Apply the changes

$session->modify($object)
    or die("Modify DHCP IPv6OptionDefinition attribute failed: ",
           $session->status_code() . ":" . $session->status_detail());

print "DHCP IPv6OptionDefinition object modified successfully \n";
```

#Remove a DHCP IPv6OptionDefinition Object

```
#Get DHCP IPv6OptionDefinition object through the session
my @retrieved_objs = $session->get(
               object =>"Infoblox::DHCP::IPv6OptionDefinition",
               space  => "SUNW",
               name   => "JumpStart-server",
            );
```

```
my $object = $retrieved_objs[0];

unless ($object) {
    die("Get DHCP IPv6OptionDefinition object failed: ",
        $session->status_code() . ":" . $session->status_detail());
}

print "Get DHCP IPv6OptionDefinition object found at least 1 matching entry\n";
```

#Submit the object for removal

```
$session->remove($object)
    or die("Remove Network failed: ",
        $session->status_code() . ":" . $session->status_detail());

print "DHCP IPv6OptionDefinition removed successfully \n";

#Remove option space.

my $response = $session->remove( object => "Infoblox::DHCP::IPv6OptionSpace" ,
                                 name   => "SUNW",
                               );
print "Remove option space success \n" if ($response );
print "Remove option space failed  \n" if ( not $response );

####PROGRAM ENDS####
```

AUTHOR

Infoblox Inc. http://www.infoblox.com/

SEE ALSO

Infoblox::Session, Infoblox::DHCP::IPv6OptionSpace, Infoblox::DHCP::IPv6Option, Infoblox::Session->get(), Infoblox::Session->add(), Infoblox::Session->remove(), Infoblox::Session->modify()

COPYRIGHT

Copyright (c) 2017 Infoblox Inc.

Infoblox::DHCP::IPv6OptionSpace - DHCP IPv6 OptionSpace object.

- NAME
- DESCRIPTION
- CONSTRUCTOR
- SESSION METHODS
  - Infoblox::Session->add( )
  - Infoblox::Session->get( )
  - Infoblox::Session->modify( )
  - Infoblox::Session->remove( )
  - Infoblox::Session->search( )
- METHODS
  - name( )
  - comment( )
  - enterprise_number( )
- SAMPLE CODE
- AUTHOR
- SEE ALSO
- COPYRIGHT

NAME

Infoblox::DHCP::IPv6OptionSpace - DHCP IPv6 OptionSpace object.

DESCRIPTION

An Option Space defines a namespace in which vendor options can be defined. This object is for an IPv6 option space. To define a specific vendor option space, add an option space to DHCP.

CONSTRUCTOR

```
my $ipv6os = Infoblox::DHCP::IPv6OptionSpace->new(
      name              => $string, #Required
      enterprise_number  => $num,    #Required, if this is a vendor option space.  It is optional
otherwise.
      comment           => $string, #Optional, default is undefined
);
```

SESSION METHODS

This section describes all the methods in Infoblox::Session module that can be applied to a DHCP IPv6 OptionSpace object.

Infoblox::Session->add( )

Use this method to add a DHCP IPv6 OptionSpace object to the Infoblox appliance. See Infoblox::Session->add() for parameters and return values.

Example

```
#Construct a DHCP IPV6 OptionSpace object
my $ipv6os = Infoblox::DHCP::IPv6OptionSpace->new(
      name => "infospace",
      comment => "some comment",
      enterprise_number => 123,
);
#Submit for addition
my $response = $session->add($ipv6os)
```

Infoblox::Session->get( )

Use this method to retrieve all the matching objects from the Infoblox appliance. See Infoblox::Session->get() for parameters and return values.

Key References

```
Apply the following attributes to get a specific DHCP IPv6 OptionSpace object:

 name     - Required. Option space name in string format.
 comment  - Optional. Option space comment.
 enterprise_number - Optional. Enterprise number.
```

Example

```
#Get DHCP IPv6 OptionSpace object through the session
my @retrieved_objs = $session->get(
    object => "Infoblox::DHCP::IPv6OptionSpace",
    name   => "infospace",
    enterprise_number => 123,
);
```

Infoblox::Session->modify( )

Use this method to modify a DHCP IPv6 OptionSpace object in the Infoblox appliance. See Infoblox::Session->modify() for parameters and return values.

Example

```
#Use method to modify the name.
$ipv6os->name("info_space");
#Submit modification
my $response = $session->modify( $ipv6os );
```

Infoblox::Session->remove( )

Use this method to remove a DHCP IPv6 OptionSpace object from the Infoblox appliance. See Infoblox::Session->remove() for parameters and return values.

To remove a specific object, first use `get()` to retrieve the specific DHCP IPv6 OptionSpace object, and then submit this object for removal.

Example

```
#Get DHCP IPv6 OptionSpace object through the session
my @retrieved_objs = $session->get(
    object => "Infoblox::DHCP::IPv6OptionSpace",
    name   => "info_space",
    enterprise_number => 123,
);
#Find the desired object from the retrieved list.
my $desired_option_space = $retrieved_objs[0];
#Submit for removal
my $response = $session->remove( $desired_option_space );
```

Infoblox::Session->search( )

Use this method to retrieve all the matching objects from the Infoblox appliance. See Infoblox::Session->search() for parameters and return values.

Key References

```
Apply the following attributes to get a specific DHCP IPv6 OptionSpace object:

 name     - Optional. Option space name in string format.
 comment  - Optional. Option space comment (regular expression).
 enterprise_number - Optional. Enterprise number.
```

Example

```
#Get DHCP IPv6 OptionSpace object through the session
my @retrieved_objs = $session->search(
    object => "Infoblox::DHCP::IPv6OptionSpace",
    comment => ".*comm.*"
);
```

METHODS

This section describes all the methods that can used to set and retrieve the attribute values of a DHCP IPv6 OptionSpace object.

name( )

Use this method to set or retrieve name of IPv6 option space object.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

Desired name in string format with a maximum of 256 bytes.

Returns

If parameter is specified, the method returns true when the modification succeeds, and returns false when the operation fails.

If parameter is not specified, the method returns the attribute value.

Example

```
#Get name
my $name = $ipv6os->name();
#Modify name
$ipv6os->name("info_space");
```

comment( )

Use this method to set or retrieve a comment about the IPv6 option space object.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

Desired comment in string format with a maximum of 256 bytes.

Returns

If the parameter is specified, the method returns true when the modification succeeds, and returns false when the operation fails.

If the parameter is not specified, the method returns the attribute value.

Example

```
#Get comment
my $comment = $ipv6os->comment();
#Modify comment
$ipv6os->comment("my comment");
```

enterprise_number( )

Use this method to set or retrieve the enterprise number for the IPv6 option space object.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

An unsigned integer that represents the enterprise number.

Returns

If the parameter is specified, the method returns true when the modification succeeds, and returns false when the operation fails.

If the parameter is not specified, the method returns the attribute value.

Example

```
#Get enterprise_number
my $enterprise_number = $ipv6os->enterprise_number();
#Modify enterprise_number
$ipv6os->enterprise_number(345);
```

SAMPLE CODE

The following sample code demonstrates the different functions that can be applied to an IPv6OptionSpace object, such as get, add, modify and remove. Also sample includes error handling for the operations.

#Preparation prior to a DHCP IPv6OptionSpace object insertion

```
#PROGRAM STARTS: Include all the modules that will be used
use strict;
use Infoblox;

#Create a session to the Infoblox appliance

my $session = Infoblox::Session->new(
               master   => "192.168.1.2", #appliance host ip
               username => "admin",       #appliance user login
               password => "infoblox"     #appliance password
);
```

```
unless ($session) {
       die("Construct session failed: ",
               Infoblox::status_code() . ":" . Infoblox::status_detail());
}
print "Session created successfully\n";
```

#Create an IPv6OptionSpace object

```
my $ipv6os = Infoblox::DHCP::IPv6OptionSpace->new (
       "name" => "infospace"
);

unless($ipv6os) {
     die("Construct IPv6 option_space filter failed: ",
          Infoblox::status_code() . ":" . Infoblox::status_detail());
}
print "IPv6OptionSpace object created successfully\n";

#Verify if the DHCP IPv6OptionSpace exists in the Infoblox appliance
my $object = $session->get(
       object => "Infoblox::DHCP::IPv6OptionSpace",
       name   => "infospace"
       enterprise_number => 123,
);
unless ($object) {
   print "IPv6 option space does not exist on server, safe to add the option_space\n";
   $session->add($ipv6os)
      or die("Add IPv6 option space failed: ",
             $session->status_code() . ":" . $session->status_detail());
}
print "DHCP IPv6 OptionSpace added successfully\n";
```

#Get and Modify an IPv6OptionSpace object

```
#Get IPv6OptionSpace object through the session
my @retrieved_objs = $session->get(
    object => "Infoblox::DHCP::IPv6OptionSpace",
    name   => "infospace"
);
my $object = $retrieved_objs[0];
unless ($object) {
    die("Get IPv6OptionSpace object failed: ",
        $session->status_code() . ":" . $session->status_detail());
}
print "Get IPv6OptionSpace object found at least 1 matching entry\n";

#Modify name of the obtained IPv6OptionSpace object
$object->name("info_space");

#Apply the changes
$session->modify($object)
    or die("Modify IPv6OptionSpace object failed: ",
           $session->status_code() . ":" . $session->status_detail());
print "IPv6OptionSpace object modified successfully \n";
```

#Remove an IPv6OptionSpace object

```
#Get IPv6OptionSpace object through the session
my @retrieved_objs = $session->get(
    object => "Infoblox::DHCP::IPv6OptionSpace",
    name   => "info_space"
);
my $object = $retrieved_objs[0];
unless ($object) {
    die("Get IPv6OptionSpace object failed: ",
        $session->status_code() . ":" . $session->status_detail());
}
print "Get IPv6OptionSpace object found at least 1 matching entry\n";

#Submit the object for removal
$session->remove($object)
    or die("Remove IPv6OptionSpace object failed: ",
        $session->status_code() . ":" . $session->status_detail());
print "IPv6OptionSpace object removed successfully \n";

####PROGRAM ENDS####
```

AUTHOR

Infoblox Inc. http://www.infoblox.com/

SEE ALSO

Infoblox::DHCP::IPv6OptionDefinition,Infoblox::Session->get(), Infoblox::Session->modify(), Infoblox::Session->remove(), Infoblox::Session

---

COPYRIGHT

Copyright (c) 2017 Infoblox Inc.

Infoblox::DHCP::IPv6Range - DHCP IPv6 Range object.

- NAME
- DESCRIPTION
- CONSTRUCTOR
- SESSION METHODS
  - Infoblox::Session->add( )
  - Infoblox::Session->get( )
  - Infoblox::Session->modify( )
  - Infoblox::Session->remove( )
  - Infoblox::Session->search( )
- METHODS
  - address_type( )
  - comment( )
  - cloud_info( )
  - disable( )
  - discovery_basic_poll_setting( )
  - discovery_member( )
  - discover_now_status( )
  - discovery_blackout_setting( )
  - enable_discovery( )
  - enable_immediate_discovery( )
  - end_addr( )
  - endpoint_sources( )
  - exclude( )
  - extattrs( )
  - extensible_attributes( )
  - ipv6_end_prefix( )
  - ipv6_prefix_bits( )
  - ipv6_start_prefix( )
  - member( )
  - name( )
  - network( )
  - network_view( )
  - next_available_ip( )
  - override_blackout_setting( )
  - override_recycle_leases( )
  - override_subscribe_settings( )
  - port_control_blackout_setting
  - restart_if_needed( )
  - recycle_leases( )
  - same_port_control_discovery_blackout( )
  - server_association_type( )
  - start_addr( )
  - subscribe_settings( )
  - template( )
- SAMPLE CODE
- AUTHOR
- SEE ALSO
- COPYRIGHT

NAME

Infoblox::DHCP::IPv6Range - DHCP IPv6 Range object.

DESCRIPTION

A DHCP IPv6 range defines the specified range of IP addresses in an IPv6 network. A DHCP IPv6 range should be added for an IPv6 network so the Infoblox appliance can assign IP addresses within that specified range to DHCP clients. If the client is on an IPv6 network that is assigned a DHCP IPv6 range, the device distributes an available IP address from that range to the DHCP client, or to a DHCP relay agent if the request came through an agent. The DHCP IPv6 range should also be assigned with a device. If devices are in a grid, the particular member serving DHCP for the DHCP IPv6 range must be specified. If the server is an independent device, this device must be specified as the member that serves the DHCP IPv6 range.

CONSTRUCTOR

```
 my $dhcp_ipv6_range = Infoblox::DHCP::IPv6Range->new(
     network                          => $network,                                  #Required
     ipv6_end_prefix                  => $ipv6addr,                                 #Optional / See
NOTE below
```

```
     ipv6_prefix_bits                    => $num,                                  #Optional / See
NOTE below
     ipv6_start_prefix                   => $ipv6addr,                             #Optional / See
NOTE below
     end_addr                            => $ipv6addr,                             #Optional / See
NOTE below
     start_addr                          => $ipv6addr,                             #Optional / See
NOTE below
     address_type                        => "ADDRESS" | "PREFIX" | "BOTH",         #Optional / Default
is undefined
     comment                             => $string,                               #Optional / Default
is undefined
     disable                             => "true" | "false",                      #Optional / Default
is "false"
     exclude                             => [$ExclusionRange1,$ExclusionRange2,...], #Optional / Default
is empty
     extattrs                            => { $string => $extattr, ... },          #Optional / Default
is undefined
     extensible_attributes               => { $string => $string | $num, $string => [ $string | $num, ...
], ... } #Optional / Default is undefined
     member                              => $Member,                               #Optional / Default
is undefined
     name                                => $string,                               #Optional / Default
is undefined
     network_view                        => $NetworkView,                          #Optional / Default
is system-defined default network view
     override_recycle_leases             => "true" | "false"                       #Optional / Default
is "false"
     recycle_leases                      => "true" | "false",                      #Optional / Default
is "true"
     server_association_type             => "NONE" | "MEMBER"                      #Optional / Default
is undefined
     template                            => $string,                               #Optional / Default
is undefined
     discovery_blackout_setting          => $blackout,                             #Optional
     port_control_blackout_setting       => $blackout,                             #Optional
     override_blackout_setting           => "true" | "false",                      #Optional / Default
value is "false"
     same_port_control_discovery_blackout => "true" | "false",                     #Optional / Default
value is "false"
     subscribe_settings                  => $subscribe_settings,                   #Optional / Default
is undefined
     override_subscribe_settings         => 'true' | 'false',                      #Optional / Default
is 'false'
 );
```

NOTE: one or both of the start_addr/end_addr and ipv6_start_prefix/ipv6_end_prefix/ipv6_prefix_bits parameter sets is required.

You cannot set both extattrs and extensible_attributes attributes at the same time.

SESSION METHODS

This section describes all the methods in an Infoblox::Session module that you can apply to a DHCP IPv6 range object.

Infoblox::Session->add( )

Use this method to add a DHCP IPv6 range object to the Infoblox appliance. See Infoblox::Session->add() for parameters and return values.

Example

```
#Construct an object
my $dhcp_ipv6range = Infoblox::DHCP::IPv6Range->new(
                                                start_addr => "2001:db8:0:c000:0001::",
                                                end_addr   => "2001:db8:0:c000:0021::",
                                                network    => "2001:db8:0:c000::/51",
);

# Submit for addition
my $response = $session->add( $dhcp_ipv6range );
```

Infoblox::Session->get( )

Use this method to retrieve all the matching objects from the Infoblox appliance. See Infoblox::Session->get() for parameters and return values.

'discover_now_status' will not be returned by default, use 'return_methods' to request it if needed.

Key References

```
Apply the following attributes to get a specific DHCP IPv6 range object:

  start_addr             - Optional. Starting IP address of the DHCP IPv6 range within the network.
  end_addr               - Optional. Ending IP address of the DHCP IPv6 range within the network.
  ipv6_start_prefix      - Optional. Starting prefix of the DHCP IPv6 range.
  ipv6_end_prefix        - Optional. Ending prefix of the DHCP IPv6.
  network_view           - Optional. Network view in which the DHCP IPv6 range is located. By
default, all network views are searched.
  extattrs               - Optional. A hash reference containing extensible attributes.
  extensible_attributes  - Optional. A hash reference containing extensible attributes.
  member                 - Optional. The member serving the range. The valid value is an
Infoblox::DHCP::Member object.
  name                   - Optional. The name of the DHCP IPv6 Range.
  network                - Optional. The IPv6 Network the range resides in.
  comment                - Optional. The descriptive comment.
  server_association_type - Optional. The server association type for the range.
```

Examples

```
my @retrieved_objs = $session->get(
                     object     => "Infoblox::DHCP::IPv6Range",
                     start_addr => "2001:db8:0:c000:0001::" );

my @retrieved_objs = $session->get(
                     object       => "Infoblox::DHCP::IPv6Range",
                     start_addr   => "2001:db8:0:c000:0001::",
                     network_view => "default" );

my @retrieved_objs = $session->get(
                     object                => "Infoblox::DHCP::IPv6Range",
                     extensible_attributes => { 'Site' => 'Santa Clara' });
```

Infoblox::Session->modify( )

Use this method to modify a DHCP IPv6 range object in the Infoblox appliance. See Infoblox::Session->modify() for parameters and return values.

Example

```
# Use method to modify the comment.
$dhcp_ipv6range->comment("this is a modified comment");
# Submit modification
my $response = $session->modify( $dhcp_ipv6range );
```

Infoblox::Session->remove( )

Use this method to remove a DHCP IPv6 range object from the Infoblox appliance. See Infoblox::Session->remove() for parameters and return values.

To remove a specific object, first use `get()` or `search()` to retrieve the specific DHCP IPv6 range object, and then submit this object for removal.

Example

```
# Get the objects with the same name
my @retrieved_objs = $session->get(
    object       => "Infoblox::DHCP::IPv6Range",
    start_addr   => "2001:db8:0:c000:0001::",
    newtork_view => "default" );
# Find the desired object from the retrieved list.
my $desired_dhcp_range = $retrieved_objs[0];
# Submit for removal
my $response = $session->remove( $desired_dhcp_range );
```

Infoblox::Session->search( )

Use this method to search for DHCP IPv6 range objects in the Infoblox appliance. See Infoblox::Session->search() for parameters and return values.

'discover_now_status' will not be returned by default, use 'return_methods' to request it if needed.

Key References

```
Apply the following attributes to search for a DHCP IPv6 range object:

  start_addr             - Optional. Starting IP address of the DHCP IPv6 range within the network
(regular expression).
  end_addr               - Optional. Ending IP address of the DHCP IPv6 range within the network
(regular expression).
  ipv6_start_prefix      - Optional. Starting prefix of the DHCP IPv6 range (regular expression).
  ipv6_end_prefix        - Optional. Ending prefix of the DHCP IPv6 (regular expression).
```

```
  network_view           - Optional. Network view in which the DHCP IPv6 range is located. By
default, all network views are searched.
  extattrs               - Optional. A hash reference containing extensible attributes.
  extensible_attributes  - Optional. A hash reference containing extensible attributes.
  member                 - Optional. The member serving the range. The valid value is an
Infoblox::DHCP::Member object.
  name                   - Optional. The name of the DHCP IPv6 Range (regular expression).
  network                - Optional. The IPv6 Network the range resides in (regular expression).
  comment                - Optional. The descriptive comment (regular expression).
  server_association_type - Optional. The server association type for the range.
```

For more information about searching extensible attributes, see Infoblox::Grid::ExtensibleAttributeDef/Searching Extensible Attributes.

Examples

```
# search for DHCP IPv6 range object with start address having last octet as 1
my @retrieved_objs = $session->search(
    object     => "Infoblox::DHCP::IPv6Range",
    network       => "2001:db8:0:c000::/51",
    network_view  => "default",
    start_addr    => "0021",
    comment       => ".*comment" );

# search for DHCP IPv6 range objects defining "Santa Clara" for "Site" extensible attribute
my @retrieved_objs = $session->search(
    object                => "Infoblox::DHCP::IPv6Range",
    extensible_attributes => { 'Site' => 'Santa Clara' });
```

METHODS

This section describes all the methods that you can use to configure and retrieve the attribute values of a DHCP IPv6 range.

address_type( )

Use this method to set or retrieve the address type of a DHCP IPv6 Range object.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

Valid values are "ADDRESS", "PREFIX", or "BOTH". When the address type is "ADDRESS", values for the 'start_addr' and 'end_addr' members are required. When the address type is "PREFIX", values for 'ipv6_start_prefix', 'ipv6_end_prefix', and 'ipv6_prefix_bits' are required. When the address type is "BOTH", values for 'start_addr', 'end_addr', 'ipv6_start_prefix', 'ipv6_end_prefix', and 'ipv6_prefix_bits' are all required.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get address_type
my $address_type = $dhcp_ipv6range->address_type();
#Modify address_type
$dhcp_ipv6range->address_type("PREFIX");
```

comment( )

Use this method to set or retrieve a descriptive comment.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

Desired comment in string format with a maximum of 256 bytes. The default value is undefined.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get comment
my $comment = $dhcp_ipv6range->comment();
```

```
#Modify comment
$dhcp_ipv6range->comment("Modifying the DHCP IPv6 range comment");
```

cloud_info( )

Use this method to set or retrieve cloud API related information for the Infoblox::DHCP::IPv6Range object.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

The valid value is a Infoblox::Grid::CloudAPI::Info object.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
# Get cloud_info
my $cloud_info = $dhcp_ipv6range->cloud_info();
# Modify cloud_info
$dhcp_ipv6range->cloud_info($cloud_info_object);
```

disable( )

Use this method to set or retrieve the disable flag of a DHCP IPv6 range.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

Specify "true" to set the disable flag or "false" to deactivate/unset it. The default value for this field is false.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get disable
my $disable = $dhcp_ipv6range->disable();
#Modify disable
$dhcp_ipv6range->disable("false");
```

discovery_basic_poll_setting( )

Use this method to set or retrieve the discovery_basic_poll_setting value of a DHCP IPv6 range.

Include the specified parameter to set the attribute value.

Parameter

The valid value is object of the Infoblox::Grid::Discovery::BasicPollSettings manpage.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get discovery_basic_poll_setting
my $discovery_basic_poll_setting = $dhcp_ipv6range->discovery_basic_poll_setting();
#Modify discovery_basic_poll_setting
$dhcp_ipv6range->discovery_basic_poll_setting($poll_setting);
```

discovery_member( )

Use this method to set or retrieve the discovery_member value of a DHCP IPv6 range.

Parameter

The discovery member name in string format.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get discovery_member
my $discovery_member = $dhcp_ipv6range->discovery_member();
#Modify discovery_member
$dhcp_ipv6range->discovery_member("test.localdomain");
```

discover_now_status( )

Use this method to retrieve the current discovery status. This attribute is read-only.

'discover_now_status' will not be returned by default, use 'return_methods' to request it if needed.

Parameter

None.

Returns

The method returns the attribute value that can be 'COMPLETE', 'FAILED', 'NONE', 'PENDING' or 'RUNNING'.

Example

```
#Get discover_now_status
my $discover_now_status = $dhcp_ipv6range->discover_now_status();
```

discovery_blackout_setting( )

Use this method to set or retrieve the discovery blackout setting.

Setting this method to a defined value implicitly sets the override_blackout_setting method to "true". If port_control_blackout_setting is undefined, setting the parameter to undefined causes the appliance to use the grid default and automatically resets the override_blackout_setting attribute to "false".

Note that when discovery_blackout_setting is set to a defined value and override_blackout_setting is set to "false", the last operation takes precedence. Thus the sequence $network->discovery_blackout_setting($blackout); $network->override_blackout_setting("false"); will set override_blackout_setting to "false", and the sequence $network->override_blackout_setting("false"); $network->discovery_blackout_setting($blackout); will result in override_blackout_setting="true".

Parameter

Valid value is an Infoblox::Grid::Discovery::Properties::BlackoutSetting object.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get discovery_blackout_setting
my $discovery_blackout_setting = $dhcp_ipv6range->discovery_blackout_setting();
#Modify discovery_blackout_setting
$dhcp_ipv6range->discovery_blackout_setting($blackout);
```

enable_discovery( )

Use this method to set or retrieve the enable_discovery value of a DHCP IPv6 range. This setting overrides the member level settings.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Setting this method to a defined value implicitly sets the override_enable_discovery method to "true". Setting the parameter to undefined causes the appliance to use the default from the upper level and automatically resets the override_enable_discovery attribute to "false".

Enable this flag if you want to set discovery for this object.

Parameter

Specify "true" to set the enable_discovery flag or "false" to deactivate/unset it. The default value is undefined.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get enable_discovery
my $enable_discovery = $dhcp_ipv6range->enable_discovery();
#Modify enable_discovery
$dhcp_ipv6range->enable_discovery("false");
```

enable_immediate_discovery( )

Use this method to set or retrieve enable_immediate_discovery value of a DHCP IPv6 range.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

Specify "true" to set the enable_immediate_discovery flag or "false" to deactivate/unset it. The default value is undefined.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get enable_immediate_discovery
my $enable_immediate_discovery = $dhcp_ipv6range->enable_immediate_discovery();
#Modify enable_immediate_discovery
$dhcp_ipv6range->enable_immediate_discovery("false");
```

end_addr( )

Use this method to set or retrieve the end address of the DHCP IPv6 range.

An end address is the last IP address in the DHCP IPv6 range available for the clients.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

Specify the end address of the DHCP IPv6 range in lowercase.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get end_addr
my $end_addr = $dhcp_ipv6range->end_addr();
#Modify end_addr
$dhcp_ipv6range->end_addr("2001:db8:0:c000:0021::");
```

endpoint_sources( )

Use this method to retrieve the list of endpoints that provide object data. This is a read-only attribute.

Omit the parameter to retrieve the attribute value.

Parameter

None

Returns

The valid return value is an array of Infoblox::CiscoISE::Endpoint objects.

Example

```
#Get endpoint_sources value
my $endpoint_sources = $object->endpoint_sources();
```

exclude( )

Use this method to set or retrieve the exclusion range of IP addresses in the DHCP IPv6 range which should not be assigned to clients.

If static addresses in the middle of a DHCP IPv6 range are assigned to certain hosts, those addresses can be excluded from the DHCP IPv6 range using this method.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

Valid value is an array reference that contains defined Infoblox::DHCP::ExclusionRange objects. The default value is empty.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get exclude
my $ref_exclude = $dhcp_ipv6range->exclude();
#Modify exclude
$dhcp_ipv6range->exclude([$exl1]); #$exl1 is an Infoblox::DHCP::ExclusionRange object
```

extattrs( )

Use this method to set or retrieve the extensible attributes associated with a DHCP IPv6 range object.

Parameter

Valid value is a hash reference containing the names of extensible attributes and their associated values (Infoblox::Grid::Extattr objects).

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get extattrs
my $ref_extattrs = $dhcp_ipv6range->extattrs();
#Modify extattrs
$dhcp_ipv6range->extattrs({ 'Site' => $extattr1, 'Administrator' => $extattr2 });
```

extensible_attributes( )

Use this method to set or retrieve the extensible attributes associated with a DHCP IPv6 range object

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

For valid values for extensible attributes, see Infoblox::Grid::ExtensibleAttributeDef/Extensible Attribute Values.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get extensible attributes
my $ref_extensible_attributes = $dhcp_ipv6range->extensible_attributes();
#Modify extensible attributes
$dhcp_ipv6range->extensible_attributes({ 'Site' => 'Santa Clara', 'Administrator' => [ 'Peter',
'Tom' ] });
```

ipv6_end_prefix( )

Use this method to set or retrieve the end prefix of the DHCP IPv6 range.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

Specify the end prefix of the DHCP IPv6 range in lowercase.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get ipv6_end_prefix
my $ipv6_end_prefix = $dhcp_ipv6range->ipv6_end_prefix();
#Modify ipv6_end_prefix
$dhcp_ipv6range->ipv6_end_prefix("2001:db8:0:c000:0021::");
```

ipv6_prefix_bits( )

Use this method to set or retrieve the prefix bits for the DHCP IPv6 range.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

An unsigned integer between 2 and 128.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get ipv6_prefix_bits
my $ipv6_prefix_bits = $dhcp_ipv6range->ipv6_prefix_bits();
#Modify ipv6_prefix_bits
$dhcp_ipv6range->ipv6_prefix_bits(32);
```

ipv6_start_prefix( )

Use this method to set or retrieve the start prefix of the DHCP IPv6 range.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

Specify the start prefix of the DHCP IPv6 range in lowercase.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get ipv6_start_prefix
my $ipv6_start_prefix = $dhcp_ipv6range->ipv6_start_prefix();
#Modify ipv6_start_prefix
$dhcp_ipv6range->ipv6_start_prefix("2001:db8:0:c000:0011::");
```

member( )

Use this method to set or retrieve the member that serves DHCP for this IP address range.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

If the attribute is set, the server_association_type attribute will also be set: to "MEMBER" if setting member to a defined value, to "NONE" otherwise.

Parameter

The valid value is a defined Infoblox::DHCP::Member object. The default value is empty.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get member
my $ref_member = $dhcp_ipv6range->member();
#Modify member
$dhcp_ipv6range->member($member1); #$member is an Infoblox::DHCP::Member object
```

name( )

Use this method to set or retrieve a descriptive name for the DHCP IPv6 range.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

The desired name in string format with a maximum of 256 bytes. The default value is undefined.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get name
my $name = $dhcp_ipv6range->name();
#Modify name
$dhcp_ipv6range->name("external");
```

network( )

Use this method to set or retrieve the DHCP IPv6 network of the address range.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

The DHCP IPv6 Network.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get network
my $network = $dhcp_ipv6range->network();
#Set the network
$dhcp_ipv6range->network("2001:db8:0:c000::/51");
```

network_view( )

Use this method to set or retrieve the network view of the DHCP IPv6 range.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

The default value is the "default" network view, which means the DHCP IPv6 range is in the default network view.

Parameter

Valid value is a defined Infoblox::DHCP::View object. The default network view value is the system-defined default network view object.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get network view
my $nview = $dhcp_ipv6range->network_view();
#Modify network view, with an Infoblox::DHCP::View object
```

```
$dhcp_ipv6range->network_view($nview);
```

next_available_ip( )

Use this method to get the next available IPv6 address(es) in the DHCP IPv6 range object. Note that to use this method, the object MUST be retrieved via a get or search call, or created with an add call.

Parameter

The method accepts the following optional parameters:

```
  requested_num - The number of IP addresses requested. It must be a value between 1 and 20,
inclusive.
  excluded - A reference to an array of addresses to be excluded from the results.
```

Returns

This method returns the next available IPv6 address(es) in the range. If requested_num is not specified, a string with the next available address will be returned; otherwise, the method will return a reference to an array of addresses.

Within a DHCP Range object, the next available IP address(es) are the first addresses in this range that match all of the following rules:

```
   * Are not part of the passed 'excluded' parameter (if specified).
   * There is no fixed address or host address record with this IP address
   * There is no unmanaged record with this IP address
   * The IP address is not in any excluded range of this IPv6 DHCP range
   * There is no active lease with this IP address (Deleted leases in the recycle bin are not
considered.)
```

Example

```
# Retrieve a particular range
my @retrieved_objs = $session->get(
                        object     => "Infoblox::DHCP::IPv6Range",
                        start_addr => "10:9::"
);

$range = $retrieved_objs[0];

# Print the next available IPv6 address in the range
print $range->next_available_ip() . "\n";

# This will retrieve the next 10 available addresses and print the first one.
my $addressesref = $range->next_available_ip(requested_num => 10);
print @$addressesref[0] . "\n";

# This will retrieve the next 10 available addresses and exclude two.
my $addressesref = $range->next_available_ip(requested_num => 10, excluded => ["10:9::2",
"10:9::3"]);
```

override_blackout_setting( )

The override_blackout_setting attribute controls whether the discovery_blackout_setting and port_control_blackout_setting values in the object are used, instead of the grid default.

The override_blackout_setting attribute can be specified explicitly. It is also set implicitly when discovery_blackout_setting or port_control_blackout_setting is set to a defined value.

Parameter

Specify "true" to set the override_blackout_setting flag or "false" to deactivate/unset it. The default value is "false".

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
# Get attribute value
my $value = $dhcp_range->override_blackout_setting();
# Modify attribute value
$dhcp_range->override_blackout_setting('true');
```

override_recycle_leases( )

The override_recycle_leases attribute controls whether the recycle_leases value in the DHCP IPv6 Range is used, instead of the grid

default.

The override_recycle_leases attribute can be specified explicitly. It is also set implicitly when recycle_leases is set to a defined value.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

Set the parameter to "true" to override the grid-level setting for recycle_leases. Set the parameter to "false" to inherit the grid-level setting for recycle_leases.

The default value of this parameter is "false".

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Getting override_recycle_leases
my $override_recycle_leases=$dhcp_ipv6range->override_recycle_leases( );
#Modifying override_recycle_leases
$dhcp_ipv6range->override_recycle_leases("true");
```

override_subscribe_settings( )

The override_subscribe_settings attribute controls whether the subscribe_settings method values of the member are used, instead of the Grid default.

The override_subscribe_settings attribute can be specified explicitly. It is also set implicitly when subscribe_settings is set to a defined value.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

Set the parameter to "true" to override the Grid-level setting for subscribe_settings. Set the parameter to "false" to inherit the Grid-level setting for subscribe_settings.

The default value of this parameter is "false".

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get override_subscribe_settings value
my $override_subscribe_settings = $object->override_subscribe_settings( );
#Modify override_subscribe_settings value
$object->override_subscribe_settings("true");
```

port_control_blackout_setting

Use this method to set or retrieve the port control blackout setting.

Setting this method to a defined value implicitly sets the override_blackout_setting method to "true". If discovery_blackout_setting is undefined, setting the parameter to undefined causes the appliance to use the grid default and automatically resets the override_blackout_setting attribute to "false".

Note that when port_control_blackout_setting is set to a defined value and override_blackout_setting is set to "false", the last operation takes precedence. Thus the sequence $network->port_control_blackout_setting($blackout); $network->override_blackout_setting("false"); will set override_blackout_setting to "false", and the sequence $network->override_blackout_setting("false"); $network->port_control_blackout_setting($blackout); will result in override_blackout_setting="true".

Parameter

Valid value is an Infoblox::Grid::Discovery::Properties::BlackoutSetting object.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get port_control_blackout_setting
my $port_control_blackout_setting = $dhcp_ipv6range->port_control_blackout_setting();
#Modify port_control_blackout_setting
$dhcp_ipv6range->port_control_blackout_setting($blackout);
```

restart_if_needed( )

Use this method to set the flag that indicates whether DHCP service should be restarted on a member that is serving DHCP for the changed object referenced in a cloud API request.

Parameter

Specify true to restart the DHCP service.

Returns

None

Example

```
# Modify
$network->restart_if_needed('true');
```

recycle_leases( )

Use this method to set or retrieve the recycle_leases flag of the DHCP IPv6 range.

If the flag is enabled, the leases are kept in recycle bin until one week after expiration. When the flag is disabled, the leases are irrecoverably deleted.

Setting this method to a defined value implicitly sets the override_recycle_leases method to "true". Setting the parameter to undefined causes the appliance to use the grid default and automatically resets the override_recycle_leases attribute to "false".

Note that when recycle_leases is set to a defined value and override_recycle_leases is set to "false", the last operation takes precedence. Thus the sequence $object->recycle_leases("true"); $object->override_recycle_leases("false"); will set override_recycle_leases to "false", and the sequence $objct->override_recycle_leases("false"); $object->recycle_leases("true"); will result in override_recycle_leases="true".

Parameter

Specify "true" to set the recycle_leases flag or "false" to deactivate/unset it. The default value is undefined.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get recycle_leases
my $recycle_leases = $dhcp_ipv6range->recycle_leases();
#Modify recycle_leases
$dhcp_ipv6range->recycle_leases("false");
```

same_port_control_discovery_blackout( )

Use this method to set or retrieve the flag that enables using the same discovery blackout for port control blackout.

Parameter

Specify 'true' to use same discovery blackout for port control blackout or 'false' to disable it. The default value is 'false'.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
# Get attribute value
my $value = $dhcp_ipv6range->same_port_control_discovery_blackout();
# Modify attribute value
$dhcp_ipv6range->same_port_control_discovery_blackout('true');
```

server_association_type( )

Use this method to set or retrieve the DHCP IPv6 range server association type.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

The server_association_type attribute can be specified explicitly. It is also set implicitly when member is set.

Parameter

Valid values are "NONE" and "MEMBER".

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get server_association_type
my $server_association_type = $dhcp_ipv6range->server_association_type();
#Modify server_association_type
$dhcp_ipv6range->server_association_type("MEMBER");
```

start_addr( )

Use this method to set or retrieve the start address of the DHCP IPv6 range.

A start address is the first IP address in the DHCP IPv6 range available for the clients.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

Specify the start address of the DHCP IPv6 range in lowercase.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get start_addr
my $start_addr = $dhcp_ipv6range->start_addr();
#Modify start_addr
$dhcp_ipv6range->start_addr("2001:db8:0:c000:0001::");
```

subscribe_settings( )

Use this method to set or retrieve the Cisco ISE subscribe settings.

Setting this method to a defined value implicitly sets the override_subscribe_settings method to "true". Setting the parameter to undefined causes the appliance to use the Cisco ISE endpoint default and automatically resets the override_subscribe_settings method to "false".

Note that when subscribe_settings is set to a defined value and override_subscribe_settings is set to "false", the last operation takes precedence. Thus the sequence $object->subscribe_settings($value); $object->override_subscribe_settings("false"); will set override_subscribe_settings to "false", and the sequence $object->override_subscribe_settings("false"); $object->subscribe_settings($value); will result in override_subscribe_settings="true".

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

The valid value is an Infoblox::CiscoISE::SubscribeSetting object.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get subscribe_settings value
my $subscribe_settings = $object->subscribe_settings();
#Modify subscribe_settings value
$object->subscribe_settings($settings);
```

template( )

Use this method to set or retrieve the DHCP IPv6 range template name for a DHCP IPv6 Range object.

Template is the name of a DHCP IPv6 range template which will be used to create the DHCP IPv6 range.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

The valid name in string format.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

The template parameter can be used to only add a new DHCP IPv6 range and cannot be used during object modify or returned during object get.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get template
my $template = $dhcp_ipv6range->template();
#Modify template
$dhcp_ipv6range->template("Template_Name");
```

SAMPLE CODE

The following sample code demonstrates the different functions that can be applied to a DHCP IPv6 range object, such as add, search, modify, and remove. This sample also includes error handling for the operations.

#Preparation prior to a DHCP IPv6 Range object insertion

```
#PROGRAM STARTS: Include all the modules that will be used
use strict;
use Infoblox;

#Create a session to the Infoblox appliance
my $session = Infoblox::Session->new(
               master   => "192.168.1.2", #appliance host ip
               username => "admin",       #appliance user login
               password => "infoblox"     #appliance password
);
unless ($session) {
       die("Construct session failed: ",
            Infoblox::status_code() . ":" . Infoblox::status_detail());
}
print "Session created successfully\n";

#Create a DHCP Member object
my $memberdhcp = Infoblox::DHCP::Member->new(
    name     => "infoblox.localdomain",
    ipv4addr => "192.168.1.2"
);
unless($memberdhcp) {
       die("Construct member failed: ",
            Infoblox::status_code() . ":" . Infoblox::status_detail());
}
print "DHCP Member object created successfully\n";

#Create the Network object with the member
my $network = Infoblox::DHCP::IPv6Network->new(
    network => "2001:db8:0:a000::/51",
    comment => "add network",
    members => [ $memberdhcp ]
);
unless($network) {
       die("Construct Network object failed: ",
            Infoblox::status_code() . ":" . Infoblox::status_detail());
}
print "Network object created successfully\n";

#Add the Network object into the Infoblox appliance through a session
$session->add($network)
```

```
    or die("Add Network object failed: ",
            $session->status_code() . ":" . $session->status_detail());
print "Network object added to Infoblox appliance successfully\n";

#Construct exclusion range object
my $exclusion_range = Infoblox::DHCP::ExclusionRange->new(
    start_address => "2001:db8:0:a000:0001::1a",
    end_address   => "2001:db8:0:a000:0001::fa",
    comment       => "Excluded range"
);
```

#Create a DHCP IPv6 Range object

```
my $dhcp_range = Infoblox::DHCP::IPv6Range->new(
                                        start_addr => '2001:db8:0:a000:0001::',
                                        end_addr   => '2001:db8:0:a000:0021::',
                                        network    => '2001:db8:0:a000::/51',
                                        comment    => 'Random comment',
                                        disable    => 'true',
                                        exclude => [$exclusion_range],
                                        ipv6_start_prefix       => '2001:db8:0:c001:0001::',
                                        ipv6_end_prefix         => '2001:db8:0:c001:0021::',
                                        ipv6_prefix_bits        => 12,
                                        member                  => $memberdhcp,
                                        name                    => 'randomrange',
                                        override_recycle_leases => 'true',
                                        recycle_leases          => 'false',
                                        server_association_type => 'MEMBER',
);
unless($dhcp_range) {
       die("Construct DHCP IPv6 Range object failed: ",
            Infoblox::status_code() . ":" . Infoblox::status_detail());
}
print "DHCP IPv6 Range object created successfully\n";

#Add the DHCP IPv6 Range object into the Infoblox appliance through a session
$session->add($dhcp_range)
    or die("Add DHCP IPv6 Range object failed: ",
            $session->status_code() . ":" . $session->status_detail());
print "DHCP IPv6 Range object added to the network successfully\n";
```

#Create a DHCP IPv6 Range object using a template

```
my $dhcp_range_template = Infoblox::DHCP::IPv6RangeTemplate->new(
        name                    => "custom_range_template",
        offset                  => "10",
        number_of_addresses     => "90",
);
unless($dhcp_range_template) {
       die("Construct DHCP IPv6 Range template object failed: ",
            Infoblox::status_code() . ":" . Infoblox::status_detail());
}
print "DHCP IPv6 Range template object created successfully\n";

#Add the DHCP IPv6 Range template object into the Infoblox appliance through a session
$session->add($dhcp_range_template)
    or die("Add DHCP IPv6 Range template object failed: ",
            $session->status_code() . ":" . $session->status_detail());
print "DHCP IPv6 Range template object added to the network successfully\n";

my $dhcp_range = Infoblox::DHCP::IPv6Range->new(
        start_addr              => "2001:db8:0:a000:0100::",
        end_addr                => "2001:db8:0:a000:0200::",
        network                 => "2001:db8:0:a000::/51",
        template                => "custom_range_template"
);
unless($dhcp_range) {
       die("Construct DHCP IPv6 Range object failed: ",
            Infoblox::status_code() . ":" . Infoblox::status_detail());
}
print "DHCP IPv6 Range object created successfully\n";

#Add the DHCP IPv6 Range object into the Infoblox appliance through a session
$session->add($dhcp_range)
    or die("Add DHCP IPv6 Range object failed: ",
            $session->status_code() . ":" . $session->status_detail());
```

```
print "DHCP IPv6 Range object added to the network successfully\n";
```

#Get and modify a DHCP IPv6 Range

```
#Get a DHCP IPv6 Range through the session
my @retrieved_objs = $session->get(
    object     => "Infoblox::DHCP::IPv6Range",
    start_addr => "2001:db8:0:a000:0001::",
);
my $object = $retrieved_objs[0];
unless ($object) {
       die("Get DHCP IPv6 Range failed: ",
            $session->status_code() . ":" . $session->status_detail());
}
print "Get DHCP IPv6 Range object found at least 1 matching entry\n";

#Modify one of the attributes of the specified DHCP IPv6 Range
$object->comment ("modified comment");

#Apply the changes
$session->modify($object)
    or die("Modify DHCP IPv6 Range failed: ",
            $session->status_code() . ":" . $session->status_detail());
print "DHCP IPv6 Range object modified successfully \n";
```

#Search a DHCP IPv6 Range

```
# search for DHCP IPv6 range object with start address having last octet as 1
my @retrieved_objs = $session->search(
    object     => "Infoblox::DHCP::IPv6Range",
    network    => "2001:db8:0:a000::/51",
    comment    => "Modified.*"
);

my $object = $retrieved_objs[0];
unless ($object) {
    die("Search DHCP IPv6 range failed: ",
        $session->status_code() . ":" . $session->status_detail());
}
print "Search DHCP IPv6 range object found at least 1 matching entry\n";
```

#Remove a DHCP IPv6 Range

```
#Get a DHCP IPv6 Range through the session
my @retrieved_objs = $session->get(
    object     => "Infoblox::DHCP::IPv6Range",
    start_addr => "2001:db8:0:a000:0001::",
);

my $object = $retrieved_objs[0];
unless ($object) {
       die("Get DHCP IPv6 Range failed: ",
            $session->status_code() . ":" . $session->status_detail());
}
print "Get DHCP IPv6 Range object found at least 1 matching entry\n";

#submit the object for removal
$session->remove($object)
    or die("Remove DHCP IPv6 Range failed: ",
            $session->status_code() . ":" . $session->status_detail());
print "DHCP IPv6 Range object removed successfully \n";

#submit the Range Template object for removal
$session->remove($dhcp_range_template)
    or die("Remove DHCP IPv6 Range Tempate failed: ",
            $session->status_code() . ":" . $session->status_detail());
print "DHCP IPv6 Range Templateobject removed successfully \n";

#Get a DHCP IPv6 Range through the session
my @retrieved_objs = $session->get(
    object     => "Infoblox::DHCP::IPv6Range",
    start_addr => "2001:db8:0:a000:0100::",
);

my $object = $retrieved_objs[0];
unless ($object) {
       die("Get DHCP IPv6 Range failed: ",
```

```
            $session->status_code() . ":" . $session->status_detail());
}
print "Get DHCP IPv6 Range object found at least 1 matching entry\n";

#submit the object for removal
$session->remove($object)
    or die("Remove DHCP IPv6 Range failed: ",
            $session->status_code() . ":" . $session->status_detail());
print "DHCP IPv6 Range object removed successfully \n";

#Remove network

my @retrieved_objs = $session->get(
    object     => "Infoblox::DHCP::IPv6Network",
    network    => "2001:db8:0:a000::/51",
);

my $object = $retrieved_objs[0];
unless ($object) {
       die("Get DHCP IPv6 Network failed: ",
            $session->status_code() . ":" . $session->status_detail());
}
print "Get DHCP IPv6 Network object found at least 1 matching entry\n";

#submit the object for removal
$session->remove($object)
    or die("Remove DHCP IPv6 Network failed: ",
            $session->status_code() . ":" . $session->status_detail());
print "DHCP IPv6 Network object removed successfully \n";

####PROGRAM ENDS####
```

AUTHOR

Infoblox Inc. http://www.infoblox.com/

SEE ALSO

Infoblox::Session, Infoblox::Session->add(), Infoblox::Session->get(), Infoblox::Session->modify(), Infoblox::Session->remove(), Infoblox::Session->search(),Infoblox::DHCP::View, Infoblox::DHCP::ExclusionRange, Infoblox::DHCP::Member, Infoblox::DHCP::Option, Infoblox::DHCP::IPv6RangeTemplate

---

COPYRIGHT

Copyright (c) 2017 Infoblox Inc.

Infoblox::DHCP::IPv6RangeTemplate - DHCP IPv6 RangeTemplate object.

- NAME
- DESCRIPTION
- CONSTRUCTOR
- MODULE METHODS
  - Infoblox::DHCP::IPv6NetworkTemplate->range_templates( )
- SESSION METHODS
  - Infoblox::Session->add( )
  - Infoblox::Session->get( )
  - Infoblox::Session->search( )
  - Infoblox::Session->modify( )
  - Infoblox::Session->remove( )
- METHODS
  - comment( )
  - exclude( )
  - extattrs( )
  - extensible_attributes( )
  - member( )
  - name( )
  - number_of_addresses( )
  - offset( )
  - override_recycle_leases( )
  - recycle_leases( )
  - server_association_type( )
- SAMPLE CODE
- AUTHOR
- SEE ALSO
- COPYRIGHT

NAME

Infoblox::DHCP::IPv6RangeTemplate - DHCP IPv6 RangeTemplate object.

DESCRIPTION

A DHCP IPv6 range template contains a set of predefined properties and can be used to create DHCP IPv6 ranges in a quick and consistent way. The DHCP IPv6 range created from a DHCP IPv6 range template will inherit the properties defined in the template.

CONSTRUCTOR

```
 my $ipv6_range_template = Infoblox::DHCP::IPv6RangeTemplate->new(
             name                    => $string,                                    #Required
             number_of_addresses     => $num,                                       #Required
             offset                  => $num,                                       #Required
             comment                 => $string,                                    #Optional / Default is
undefined
             exclude                 => [$ExclusionRange1, $ExclusionRange2,...],   #Optional / Default is
empty
             extattrs                => { $string => $extattr, ... },               #Optional / Default is
undefined
             extensible_attributes   => { $string => $string | $num, $string => [ $string | $num, ... ],
... }, #Optional / Default is undefined
             member                  => $Member,                                    #Optional / Default is
undefined
             override_recycle_leases => "true" | "false",                           #Optional / Default is
"false"
             recycle_leases          => "true" | "false" | undef,                   #Optional / Default is
undefined
             server_association_type => "NONE" | "MEMBER",                          #Optional / Default is
"NONE"
  );
```

You cannot set both extattrs and extensible_attributes attributes at the same time.

MODULE METHODS

The following functions can be applied to a DHCP IPv6 range template object.

Infoblox::DHCP::IPv6NetworkTemplate->range_templates( )

Use this function to include a range template in an IPv6 network template on the Infoblox appliance. See Infoblox::DHCP::IPv6NetworkTemplate->range_templates() for parameters and return values.

Example

```
#Construct an IPv6 range template object
my $ipv6_range_template = Infoblox::DHCP::IPv6RangeTemplate->new(
            name                    => "custom_range_template",
            number_of_addresses     => "1000",
            offset                  => "10",
    );

my $template = Infoblox::DHCP::Template->new(
            name                    => "custom_range_template",
    );

#Configure a range template in a network template object
my $responce = $network_template->range_templates([$template]);
```

SESSION METHODS

This section describes all the methods in an Infoblox::Session module that can be applied to a DHCP IPv6 range template object.

Infoblox::Session->add( )

Use this method to add a DHCP IPv6 range template object to the Infoblox appliance. See Infoblox::Session->add() for parameters and return values.

Example

```
#Construct an object
my $ipv6_range_template = Infoblox::DHCP::IPv6RangeTemplate->new(
        name                    => "custom_range_template",
        offset                  => "20",
        number_of_addresses     => "100",
        comment                 => "add range",
        exclude                 => [$exclusion_range_template],
        member                  => $memberdhcp,
        recycle_leases          => "false",
        override_recycle_leases => "true",
        server_association_type => 'MEMBER',
);

# Submit for addition
my $response = $session->add( $ipv6_range_template );
```

Infoblox::Session->get( )

Use this method to retrieve all the matching objects from the Infoblox appliance. See Infoblox::Session->get() for parameters and return values.

Key References

```
Apply the following attributes to get a specific DHCP IPv6 range template object:

name - Optional. The name of the DHCP IPv6 range template.
extattrs     - Optional. A hash reference containing extensible attributes.
extensible_attributes - Optional. A hash reference containing extensible attributes.
server_association_type - Optional. The server association type for the range.
```

Examples

```
my @retrieved_objs = $session->get(
                      object => "Infoblox::DHCP::IPv6RangeTemplate",
                      name   => "custom_range_template"
                      );

my @retrieved_objs = $session->get(
                     object               => "Infoblox::DHCP::IPv6RangeTemplate",
                     extensible_attributes => { 'Site' => 'Santa Clara' }
                    );
```

Infoblox::Session->search( )

Use this method to retrieve all the matching objects from the Infoblox appliance. See Infoblox::Session->search() for parameters and

return values.

Key References

```
Apply the following attributes to search for a specific DHCP IPv6 range template object:

name    - Optional. The name of the DHCP IPv6 range template (regular expression).
comment - Optional. The comment of the DHCP IPv6 range template (regular expression).
extattrs     - Optional. A hash reference containing extensible attributes.
extensible_attributes - Optional. A hash reference containing extensible attributes.
server_association_type - Optional. The server association type for the range.
```

For more information about searching extensible attributes, see Infoblox::Grid::ExtensibleAttributeDef/Searching Extensible Attributes.

Examples

```
my @retrieved_objs = $session->search(
                        object  => "Infoblox::DHCP::IPv6RangeTemplate",
                        name    => "custom_range_template",
                        comment => ".*",
        );

my @retrieved_objs = $session->search(
                        object               => "Infoblox::DHCP::IPv6RangeTemplate",
                        extensible_attributes => { 'Site' => 'Santa Clara' }
        );
```

Infoblox::Session->modify( )

Use this method to modify a DHCP IPv6 range template object in the Infoblox appliance. See Infoblox::Session->modify() for parameters and return values.

Example

```
# Use this method to modify the comment.
$ipv6_range_template->comment("this is a modified comment");
# Submit modification
my $response = $session->modify( $ipv6_range_template );
```

Infoblox::Session->remove( )

Use this method to remove a DHCP IPv6 range template object from the Infoblox appliance. See Infoblox::Session->remove() for parameters and return values.

To remove a specific object, first use `get()` or `search()` to retrieve the specific DHCP IPv6 range template object, and then submit this object for removal.

Example

```
# Get the objects with the same name
my @retrieved_objs = $session->get(
    object => "Infoblox::DHCP::IPv6RangeTemplate",
    name   => "custom_range_template"
    );

# Find the desired object from the retrieved list.
my $desired_ipv6_range_template = $retrieved_objs[0];
# Submit for removal
my $response = $session->remove( $desired_ipv6_range_template );
```

METHODS

This section describes all the methods that can be used to configure and retrieve the attribute values of a DHCP IPv6 range template object.

comment( )

Use this method to set or retrieve a descriptive comment of a DHCP IPv6 range template.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

Desired comment in string format with a maximum of 256 bytes.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get comment
my $comment = $ipv6_range_template->comment();
#Modify comment
$ipv6_range_template->comment("Modifying the DHCP IPv6 range template comment");
```

exclude( )

Use this method to set or retrieve the exclusion range template in the DHCP IPv6 range template.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

Valid value is an array reference that contains defined Infoblox::DHCP::ExclusionRangeTemplate objects.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get exclude
my $ref_exclude = $ipv6_range_template->exclude();
#Modify exclude
$ipv6_range_template->exclude([$exl1]); #$exl1 is an Infoblox::DHCP::ExclusionRangeTemplate
object
```

extattrs( )

Use this method to set or retrieve the extensible attributes associated with a DHCP IPv6 range template object.

Parameter

Valid value is a hash reference containing the names of extensible attributes and their associated values (Infoblox::Grid::Extattr objects).

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get extattrs
my $ref_extattrs = $ipv6_range_template->extattrs();
#Modify extattrs
$ipv6_range_template->extattrs({ 'Site' => $extattr1, 'Administrator' => $extattr2 });
```

extensible_attributes( )

Use this method to set or retrieve the extensible attributes associated with a DHCP IPv6 range template object.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

For valid values for extensible attributes, see Infoblox::Grid::ExtensibleAttributeDef/Extensible Attribute Values.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get extensible attributes
my $ref_extensible_attributes = $ipv6_range_template->extensible_attributes();
#Modify extensible attributes
$ipv6_range_template->extensible_attributes({ 'Site' => 'Santa Clara', 'Administrator' => [ 'Peter',
```

```
    'Tom' ] });
```

member( )

Use this method to set or retrieve the grid member assigned to the DHCP IPv6 range template.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

The valid value is a defined Infoblox::DHCP::Member object. The default value is undefined.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get member
my $ref_member = $ipv6_range_template->member();
#Modify member
$ipv6_range_template->member($member1); #$member is an Infoblox::DHCP::Member object
```

name( )

Use this method to set or retrieve the name of the DHCP IPv6 range template.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

Name of the IPv6 range template in string format with a maximum of 64 bytes. The default value is undefined.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get name
my $name = $ipv6_range_template->name();
#Modify name
$ipv6_range_template->name("custom_range_template");
```

number_of_addresses( )

Use this method to set or retrieve the number of addresses in this DHCP IPv6 range template.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

The number of addresses in this DHCP IPv6 range template.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get number_of_addresses
my $number_of_addresses = $ipv6_range_template-> number_of_addresses();
#Modify number_of_addresses
$ipv6_range_template->number_of_addresses("100");
```

offset( )

Use this method to set or retrieve the address offset of this DHCP IPv6 range template.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

The address offset of this DHCP IPv6 range template. The offset is calculated from the start address of the IPv6 network template that includes the DHCP range template. For example, in the IPv6 network 2010:0:0:10::/64, if the offset is set to 10 and number_of_addresses is set to 100, then the start address of the DHCP IPv6 range created from this range template is 2010:0:0::A and the end address is 2010:0:0::6D.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get offset
my $offset = $ipv6_range_template->offset();
#Modify offset
$ipv6_range_template->offset("10");
```

override_recycle_leases( )

The override_recycle_leases attribute controls whether the recycle_leases value in the DHCP IPv6 range template is used, instead of the grid default.

The override_recycle_leases attribute can be specified explicitly. It is also set implicitly when recycle_leases is set to a defined value.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

Set the parameter to "true" to override the grid-level setting for recycle_leases. Set the parameter to "false" to inherit the grid-level setting for recycle_leases.

The default value of this parameter is "false".

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Getting override_recycle_leases
my $override_recycle_leases=$ipv6_range_template->override_recycle_leases( );
#Modifying override_recycle_leases
$ipv6_range_template->override_recycle_leases("true");
```

recycle_leases( )

Use this method to set or retrieve the recycle_leases flag of the DHCP IPv6 range template.

If the flag is enabled, the leases are kept in the recycle bin until one week after they expire. When the flag is disabled, the leases are irrecoverably deleted.

Setting this method to a defined value implicitly sets the override_recycle_leases method to "true". Setting the parameter to undefined causes the appliance to use the grid default and automatically resets the override_recycle_leases attribute to "false".

Note that when recycle_leases is set to a defined value and override_recycle_leases is set to "false", the last operation takes precedence. Thus the sequence $object->recycle_leases("true"); $object->override_recycle_leases("false"); will set override_recycle_leases to "false", and the sequence $objct->override_recycle_leases("false"); $object->recycle_leases("true"); will result in override_recycle_leases="true".

Parameter

Specify "true" to set the recycle_leases flag or "false" to deactivate/unset it. The default value is undefined.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get recycle_leases
my $recycle_leases = $ipv6_range_template->recycle_leases();
#Modify recycle_leases
$ipv6_range_template->recycle_leases("false");
```

server_association_type( )

Use this method to set or retrieve the DHCP IPv6 range template server association type.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

Valid values are "NONE" and "MEMBER".

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get server_association_type
my $server_association_type = $ipv6_range_template->server_association_type();
#Modify server_association_type
$ipv6_range_template->server_association_type("MEMBER");
```

SAMPLE CODE

The following sample code demonstrates the different functions that can be applied to a DHCP range template object such as add, modify, and remove. Also, this sample includes error handling for the operations.

#Preparation prior to a DHCP IPv6RangeTemplate object insertion

```
#PROGRAM STARTS: Include all the modules that will be used
use strict;
use Infoblox;

#Create a session to the Infoblox appliance
my $session = Infoblox::Session->new(
               master   => "192.168.1.2",      #appliance host ip
               username => "admin",       #appliance user login
               password => "infoblox"     #appliance password
);
unless ($session) {
       die("Construct session failed: ",
            Infoblox::status_code() . ":" . Infoblox::status_detail());
}
print "Session created successfully\n";

#Create a DHCP Member object
my $memberdhcp = Infoblox::DHCP::Member->new(
    name     => "infoblox.localdomain",
    ipv4addr => "192.168.1.2"
);
unless($memberdhcp) {
       die("Construct member failed: ",
            Infoblox::status_code() . ":" . Infoblox::status_detail());
}
print "DHCP Member object created successfully\n";

#Construct exclusion range template object
my $exclusion_range_template = Infoblox::DHCP::ExclusionRangeTemplate->new(
    offset              => "15",
    number_of_addresses => "50",
    comment             => "Excluded range"
);
unless($exclusion_range_template){
       die("Construct exclusion range template object failed: ",
                Infoblox::status_code() . ":" . Infoblox::status_detail());
       }
 print "Exclusion range template object created successfully\n";
```

#Create a DHCP IPv6 range template object

```
my $ipv6_range_template = Infoblox::DHCP::IPv6RangeTemplate->new(
        name                  => "custom_range_template",
        offset                => "20",
        number_of_addresses   => "100",
        comment               => "add range",
```

```
        exclude                 => [$exclusion_range_template],
        member                  => $memberdhcp,
        recycle_leases          => "false",
        override_recycle_leases => "true",
        server_association_type => 'MEMBER',
);
unless($ipv6_range_template) {
       die("Construct DHCP IPv6 range template object failed: ",
            Infoblox::status_code() . ":" . Infoblox::status_detail());
}
print "DHCP IPv6 range template object created successfully\n";

#Add the DHCP IPv6 range template object into the Infoblox appliance through a session
$session->add($ipv6_range_template)
    or die("Add DHCP IPv6 range template object failed: ",
            $session->status_code() . ":" . $session->status_detail());

print "DHCP IPv6 range template object added to the network successfully\n";
```

#Get and modify a DHCP IPv6 range template

```
#Get a DHCP IPv6 range template through the session
my @retrieved_objs = $session->get(
    object     => "Infoblox::DHCP::IPv6RangeTemplate",
    name       => "custom_range_template",
);
my $object = $retrieved_objs[0];
unless ($object) {
       die("Get DHCP IPv6 range template failed: ",
            $session->status_code() . ":" . $session->status_detail());
}
print "Get DHCP IPv6 range template object found at least 1 matching entry\n";

#Modify one of the attributes of the specified DHCP IPv6 range template
$object->comment("modified comment");

#Apply the changes
$session->modify($object)
    or die("Modify DHCP IPv6 range template failed: ",
            $session->status_code() . ":" . $session->status_detail());

print "DHCP IPv6 range template object modified successfully \n";
```

#Remove a DHCP IPv6 range template

```
#Get a DHCP IPv6 range template through the session
my @retrieved_objs = $session->get(
    object     => "Infoblox::DHCP::IPv6RangeTemplate",
    name       => "custom_range_template",
);

my $object = $retrieved_objs[0];
unless ($object) {
       die("Get DHCP IPv6 range template failed: ",
            $session->status_code() . ":" . $session->status_detail());
}
print "Get DHCP IPv6 range template object found at least 1 matching entry\n";

#Submit the object for removal
$session->remove($object)
    or die("Remove DHCP IPv6 range template failed: ",
            $session->status_code() . ":" . $session->status_detail());

print "DHCP IPv6 range template object removed successfully \n";

####PROGRAM ENDS####
```

AUTHOR

Infoblox Inc. http://www.infoblox.com/

SEE ALSO

Infoblox::Session, Infoblox::DHCP::IPv6NetworkTemplate, Infoblox::DHCP::RangeTemplate, Infoblox::DHCP::ExclusionRangeTemplate, Infoblox::DHCP::Template, Infoblox::DHCP::Member

COPYRIGHT

Copyright (c) 2017 Infoblox Inc.

Infoblox::DHCP::IPv6SharedNetwork - DHCP IPv6 Shared Network object

- NAME
- DESCRIPTION
- CONSTRUCTOR
- SESSION METHODS
  - Infoblox::Session->add( )
  - Infoblox::Session->get( )
  - Infoblox::Session->modify( )
  - Infoblox::Session->remove( )
  - Infoblox::Session->search( )
- METHODS
  - comment( )
  - ddns_domainname( )
  - ddns_enable_option_fqdn( )
  - ddns_generate_hostname( )
  - ddns_ttl( )
  - disable( )
  - domain_name( )
  - domain_name_servers( )
  - enable_ddns( )
  - extattrs( )
  - extensible_attributes( )
  - name( )
  - networks( )
  - network_view( )
  - options( )
  - override_ddns_domainname( )
  - override_ddns_enable_option_fqdn( )
  - override_ddns_generate_hostname( )
  - override_ddns_ttl( )
  - override_domain_name( )
  - override_domain_name_servers( )
  - override_enable_ddns( )
  - override_options( )
  - override_preferred_lifetime( )
  - override_update_dns_on_lease_renewal( )
  - override_valid_lifetime( )
  - preferred_lifetime( )
  - update_dns_on_lease_renewal( )
  - valid_lifetime( )
- SAMPLE CODE
- AUTHOR
- SEE ALSO
- COPYRIGHT

NAME

Infoblox::DHCP::IPv6SharedNetwork - DHCP IPv6 Shared Network object

---

DESCRIPTION

You can create a shared network when two subnets share a particular network segment. Before creating a shared network, you must first create the subnetworks. For example, you must first create the networks 2001:10:20:: and 2001:10:30:: before designating them as a shared network.

CONSTRUCTOR

```
  my $sharednetwork = Infoblox::DHCP::IPv6SharedNetwork->new(
         name                              => $string,                    #Required
         networks                          => [ $Network1,$Network2,...], #Required
         comment                           => $string | undef,            #Optional / Default is
undefined
         ddns_domainname                   => $string | undef,            #Optional / Default is
undefined
         ddns_enable_option_fqdn           => "true" | "false",           #Optional / Default is "false"
         ddns_generate_hostname            => "true" | "false",           #Optional / Default is "false"
         ddns_ttl                          => $num,                       #Optional / Default is 0
         disable                           => "true" | "false",           #Optional / Default is "false"
         domain_name                       => $string | undef,            #Optional / Default is
```

```
undefined
        domain_name_servers                 => "$ipv6addr,$ipv6addr,...",  #Optional / Default is
undefined
        enable_ddns                         => "true" | "false" | undef,   #Optional / Default is
undefined
        extattrs                            => { $string => $extattr, ... },      #Optional / Default is
undefined
        extensible_attributes               => { $string => $string | $num, $string => [ $string | $num,
... ], ... }, #Optional / Default is undefined.
        network_view                        => $NetworkView,               #Optional / Default is
"default" network view
        options                             => [$Option1,$Option2,...],    #Optional / Default is empty
list
        override_ddns_domainname            => "true" | "false",           #Optional / Default is "false"
        override_ddns_enable_option_fqdn    => "true" | "false",           #Optional / Default is "false"
        override_ddns_generate_hostname     => "true" | "false",           #Optional / Default is "false"
        override_ddns_ttl                   => "true" | "false",           #Optional / Default is "false"
        override_domain_name                => "true" | "false",           #Optional / Default is "false"
        override_domain_name_servers        => "true" | "false",           #Optional / Default is "false"
        override_enable_ddns                => "true" | "false",           #Optional / Default is "false"
        override_options                    => "true" | "false",           #Optional / Default is "false"
        override_preferred_lifetime         => "true" | "false",           #Optional / Default is "false"
        override_update_dns_on_lease_renewal=> "true" | "false",           #Optional / Default is "false"
        override_valid_lifetime             => "true" | "false",           #Optional / Default is "false"
        preferred_lifetime                  => $num,                       #Optional / Default is 27000
        update_dns_on_lease_renewal         => "true" | "false" | undef,   #Optional / Default is
undefined
        valid_lifetime                      => $num,                       #Optional / Default is 43200
    );
```

You cannot set both extattrs and extensible_attributes attributes at the same time.

SESSION METHODS

This section describes all the methods in the Infoblox::Session module that you can apply to a DHCP IPv6 shared network object.

Infoblox::Session->add( )

Use this method to add an IPv6 shared network object to the Infoblox appliance. See Infoblox::Session->add() for parameters and return values.

Example

```
#Construct a network object
my $network1 = Infoblox::DHCP::IPv6Network->new(
        network => "2001:10:20:30::/64",
);

#Construct a shared network object
  my $sharednetwork = Infoblox::DHCP::IPv6SharedNetwork->new(
        name     => "myname",
        networks => [$network1],
);

# Submit for adding a shared network
my $response = $session->add( $sharednetwork );
```

Infoblox::Session->get( )

Use this method to retrieve all the matching shared network objects from the Infoblox appliance. See Infoblox::Session->get() for parameters and return values.

Key References

```
Apply the following attributes to get a specific DHCP shared network object:

  name           - Required.    The name of the shared network in string format.
  network_view   - Optional.    The network view in which the shared network is located. By default,
all network views are searched.
  extattrs       - Optional. A hash reference containing extensible attributes.
  extensible_attributes - Optional. A hash reference containing extensible attributes.
```

Example

```
  my  @result_array = $session->get(
       object       => "Infoblox::DHCP::IPv6SharedNetwork" ,
       name         => "myname",
       network_view => "default"
```

```
);
```

Infoblox::Session->modify( )

Use this method to modify an IPv6 shared network object in the Infoblox appliance. See Infoblox::Session->modify() for parameters and return values.

Example

```
#Use this method to modify the enable_ddns method.
$object->enable_ddns("true");
# Submit for modification
my $response = $session->modify( $object );
```

Infoblox::Session->remove( )

Use this method to remove a shared network object from the Infoblox appliance. See Infoblox::Session->remove() for parameters and return values.

To remove a specific object, first use `get()` or `search()` to retrieve the object, and then submit it for removal.

Example

```
# Get the object under the same name
my  @result_array = $session->get(
      object       => "Infoblox::DHCP::IPv6SharedNetwork" ,
      name         => "myname",
      network_view => "default"
 );

# Find the desired object on the retrieved list
my $desired_network = $result_array[0];

# Submit for removal
my $response = $session->remove( $desired_network );
```

Infoblox::Session->search( )

Use this method to search for a DHCP shared network object in the Infoblox appliance. See Infoblox::Session->search() for parameters and return values.

Key References

```
Apply the following attributes to search for a DHCP shared network object:

  name          - Required. The name of the shared network in string format (regular expression).
  network_view  - Optional. The network view in which the shared network is located. By default, all
network views are searched.
  extattrs      - Optional. A hash reference containing extensible attributes.
  extensible_attributes - Optional. A hash reference containing extensible attributes.
```

For more information about searching extensible attributes, see Infoblox::Grid::ExtensibleAttributeDef/Searching Extensible Attributes.

Example

```
# Search for all DHCP shared networks in the "default" network view
    my @retrieved_objs = $session->search(
      object    => "Infoblox::DHCP::IPv6SharedNetwork",
      name      => "my.*",
      network_view => "default"
    );

# Search for all IPv6 shared networks with the extensible attribute 'Site'
    my @retrieved_objs = $session->search(
      object => "Infoblox::DHCP::IPv6SharedNetwork",
      extensible_attributes => { 'Site' => 'Santa Clara' }
    );
```

METHODS

This section describes all the methods that you can use to configure and retrieve the attribute values of DHCP a shared network object

comment( )

Use this method to set or retrieve a descriptive comment.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

Desired comment in string format with a maximum of 256 bytes.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get comment
my $comment = $sharednetwork->comment();
#Modify comment
$sharednetwork->comment("Modified shared network");
```

ddns_domainname( )

Use this method to set or retrieve the ddns_domainname value of a DHCP IPv6 shared network object.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Setting this method to a defined value implicitly sets the override_ddns_domainname method to "true". Setting the parameter to undefined causes the appliance to use the grid default and automatically resets the override_ddns_domainname attribute to "false".

Note that when ddns_domainname is set to a defined value and override_ddns_domainname is set to "false", the last operation takes precedence. Thus the sequence $object->ddns_domainname("testdomain"); $object->override_ddns_domainname("false"); will set override_ddns_domainname to "false", and the sequence $object->override_ddns_domainname("false"); $object->ddns_domainname("testdomain"); will result in override_ddns_domainname="true".

Parameter

The dynamic DNS domain name in string format.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get ddns_domainname
my $ddns_domainname = $sharednetwork->ddns_domainname();
#Modify ddns_domainname
$sharednetwork->ddns_domainname("test_domain.com");
```

ddns_enable_option_fqdn( )

Use this method to set or retrieve the ddns_enable_option_fqdn flag of a shared network object. This method controls whether the FQDN option sent by the client is to be used, or if the server can automatically generate the FQDN. This setting overrides the member-level settings.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Setting this method to a defined value implicitly sets the override_ddns_enable_option_fqdn method to "true". Setting the parameter to undefined causes the appliance to use the member default and automatically resets the override_ddns_enable_option_fqdn attribute to "false".

Note that when ddns_enable_option_fqdn is set to a defined value and override_ddns_enable_option_fqdn is set to "false", the last operation takes precedence. Thus the sequence $object->ddns_enable_option_fqdn("true"); $object->override_ddns_enable_option_fqdn("false"); will set override_ddns_enable_option_fqdn to "false", and the sequence $object->override_ddns_enable_option_fqdn("false"); $object->ddns_enable_option_fqdn("true"); will result in override_ddns_enable_option_fqdn="true".

Parameter

Specify "true" to generate the host name or "false" to deactivate/unset it. The default value for this field is undefined, which indicates that this attribute is inherited from the member-level setting.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
# Get ddns_enable_option_fqdn
my $ddns_enable_option_fqdn = $sharednetwork->ddns_enable_option_fqdn()
# Modify ddns_enable_option_fqdn
$sharednetwork->ddns_enable_option_fqdn("true");
#Un-override
$sharednetwork->ddns_enable_option_fqdn(undef);
```

ddns_generate_hostname( )

Use this method to set or retrieve the ddns_generate_hostname flag of a shared network object. Enable this method to allow only the DHCP server to generate host name. This setting overrides the member-level settings.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Setting this method to a defined value implicitly sets the override_ddns_generate_hostname method to "true". Setting the parameter to undefined causes the appliance to use the upper-level setting and automatically resets the override_ddns_generate_hostname attribute to "false".

Note that when ddns_generate_hostname is set to a defined value and override_ddns_generate_hostname is set to "false", the last operation takes precedence. Thus the sequence $object->ddns_generate_hostname("true"); $object->override_ddns_generate_hostname("false"); will set override_ddns_generate_hostname to "false", and the sequence $object->override_ddns_generate_hostname("false"); $object->ddns_generate_hostname("true"); will result in override_ddns_generate_hostname="true".

Parameter

Specify "true" to generate the host name or "false" to deactivate/unset it. The default value for this field is undefined, which indicates that this attribute is inherited from the member-level setting.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
# Get ddns_generate_hostname
my $ddns_generate_hostname = $sharednetwork->ddns_generate_hostname()
# Modify ddns_generate_hostname
$sharednetwork->ddns_generate_hostname("true");
#Un-override
$sharednetwork->ddns_generate_hostname(undef);
```

ddns_ttl( )

Use this method to set or retrieve the DNS update Time to Live (TTL) value of an IPv6 shared network object.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Setting this method to a defined value implicitly sets the override_ddns_ttl method to "true". Setting the parameter to undefined causes the appliance to use the grid default and automatically resets the override_ddns_ttl attribute to "false".

Note that when ddns_ttl is set to a defined value and override_dddns_ttl is set to "false", the last operation takes precedence. Thus the sequence $object->ddns_ttl(12); $object->override_ddns_ttl("false"); will set override_ddns_ttl to "false", and the sequence $object->override_ddns_ttl("false"); $object->ddns_ttl(12); will result in override_ddns_ttl="true".

Parameter

A 32-bit integer (range from 0 to 4294967295) that represents the duration,in seconds, that the update is cached. Zero indicates that the update should not be cached. The default value is zero.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get DNS Update TTL
my $dns_ttl = $sharednetwork->ddns_ttl();
#Modify DNS Update TTL
$sharednetwork->ddns_ttl(1200);
```

disable( )

Use this method to set or retrieve the disable flag of an IPv6 shared network object.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

Specify "true" to set the disable flag and disable the shared network or "false" to unset the flag and enable the IPv6 shared network. The default value for this field is "false".

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
# Get disable
my $disable = $sharednetwork->disable()
# Modify disable
$sharednetwork->disable("true");
```

domain_name( )

Use this method to set or retrieve the domain_name value of a DHCP IPv6 shared network object.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Setting this method to a defined value implicitly sets the override_domain_name method to "true". Setting the parameter to undefined causes the appliance to use the grid default and automatically resets the override_domain_name attribute to "false".

Note that when domain_name is set to a defined value and override_domain_name is set to "false", the last operation takes precedence. Thus the sequence $object->domain_name("testdomain"); $object->override_domain_name("false"); will set override_domain_name to "false", and the sequence $object->override_domain_name("false"); $object->domain_name("testdomain"); will result in override_domain_name="true".

Parameter

The name of the client's domain for resolving names in DNS in string format.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get domain_name
my $domain_name = $sharednetwork->domain_name();
#Modify domain_name
$sharednetwork->domain_name("test_domain.com");
```

domain_name_servers( )

Use this method to set or retrieve the domain_name_servers value of a DHCP IPv6 shared network object.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Setting this method to a defined value implicitly sets the override_domain_name_servers method to "true". Setting the parameter to undefined causes the appliance to use the grid default and automatically resets the override_domain_name_servers attribute to "false".

Note that when domain_name_servers is set to a defined value and override_domain_name_servers is set to "false", the last operation takes precedence. Thus the sequence $object->domain_name_servers("2001::1,2001::2"); $object->override_domain_name_servers("false"); will set override_domain_name_servers to "false", and the sequence $object->override_domain_name_servers("false"); $object->domain_name_servers("2001::1,2001::2"); will result in override_domain_name_servers="true".

Parameter

A comma separated list of the domain name server addresses in IPv6 address format. Spaces around the commas are not allowed.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get domain_name_servers
my $domain_name_servers = $sharednetwork->domain_name_servers();
#Modify domain_name_servers
$sharednetwork->domain_name_servers("2001::1,2001::2");
```

enable_ddns( )

Use this method to set or retrieve the dynamic DNS updates flag of a DHCP IPv6 shared network object. The DHCP server can send DDNS updates to DNS servers in the same grid and to external DNS servers. This setting overrides the member-level settings.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Setting this method to a defined value implicitly sets the override_enable_ddns method to "true". Setting the parameter to undefined causes the appliance to use the upper-level default and automatically resets the override_enable_ddns attribute to "false".

Note that when enable_ddns is set to a defined value and override_enable_ddns is set to "false", the last operation takes precedence. Thus the sequence $object->enable_ddns("true"); $object->override_enable_ddns("false"); will set override_enable_ddns to "false", and the sequence $object->override_enable_ddns("false"); $object->enable_ddns("true"); will result in override_enable_ddns="true".

Parameter

Specify "true" to set the dynamic DNS updates flag or "false" to deactivate/unset it. The default value for this field is undefined, which indicates that this attribute is inherited from the member-level setting.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
# Get enable_ddns
my $enable_ddns = $sharednetwork->enable_ddns()
# Modify enable_ddns
$sharednetwork->enable_ddns("true");
# Un-override enable_ddns
$sharednetwork->enable_ddns(undef);
```

extattrs( )

Use this method to set or retrieve the extensible attributes associated with an IPv6 shared network object.

Parameter

Valid value is a hash reference containing the names of extensible attributes and their associated values (Infoblox::Grid::Extattr objects).

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get extattrs
my $ref_extattrs = $sharednetwork->extattrs();
#Modify extattrs
$sharednetwork->extattrs({ 'Site' => $extattr1, 'Administrator' => $extattr2 });
```

extensible_attributes( )

Use this method to set or retrieve the extensible attributes associated with an IPv6 shared network object.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

For valid values for extensible attributes, see Infoblox::Grid::ExtensibleAttributeDef/Extensible Attribute Values.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get extensible attributes
my $ref_extensible_attributes = $shared_network->extensible_attributes();
#Modify extensible attributes
$shared_network->extensible_attributes({ 'Site' => 'Santa Clara', 'Administrator' => [ 'Peter',
'Tom' ] });
```

name( )

Use this method to retrieve the name of an IPv6 shared network object.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

The name of the shared network in string format. A name can have a maximum of 32 characters.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get name
my $sharednetwork = $sharednetwork->name();
```

networks( )

Use this method to retrieve the DHCP IPv6 networks associated with an IPv6 shared network object.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

The valid value is an array reference that contains Infoblox::DHCP::IPv6Network objects.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get networks
my $sharednetwork = $sharednetwork->networks();
#Modify networks
my $network1 = Infoblox::DHCP::IPv6Network->new(
                                    "network" => "2001:10:20:30::/64",
                                );
my $network2 = Infoblox::DHCP::IPv6Network->new(
                                    "network" => "2001:10:20:40::/64",
                                );
my $sharednetwork = $sharednetwork->networks([$network1, $network2]);
```

network_view( )

Use this method to set or retrieve the network view of the DHCP IPv6 shared network.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

The default value is the "default" network view, which means the DHCP shared network is in the default network view.

Parameter

Valid value is a defined Infoblox::DHCP::View object. The default network view value is the system-defined default network view object.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get network view
my $nview = $sharednetwork->network_view();
#Modify network view, with an Infoblox::DHCP::View object
$sharednetwork->network_view($nview);
```

options( )

Use this method to set or retrieve the options value of DHCP IPv6 shared network. DHCP options provide network configuration settings and various services available on the network for DHCP. This setting overrides the member-level setting.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Setting this method to a defined value implicitly sets the override_options method to "true". Setting the parameter to undefined causes the appliance to use the upper-level default and automatically resets the override_options attribute to "false".

Note that when the options method is set to a defined value and override_options is set to "false", the last operation takes precedence. Thus the sequence $object->options([$opt]); $object->override_options("false"); will set override_options to "false", and the sequence $object->override_options("false"); $object->options([$opt]); will result in override_options="true".

Parameter

The valid value is an array reference that contains Infoblox::DHCP::Option objects.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get options
my $options = $sharednetwork->options();
#Modify options
my $option1 = Infoblox::DHCP::Option->new(
    name   => "dhcp6.rapid-commit",
    value  => "true"
);
$sharednetwork->options([$option1]);
```

override_ddns_domainname( )

The override_ddns_domainname attribute controls whether the ddns_domainname value in the DHCP IPv6 shared network is used, instead of the grid default.

The override_ddns_domainname attribute can be specified explicitly. It is also set implicitly when ddns_domainname is set to a defined value.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

Set the parameter to "true" to override the grid-level setting for ddns_domainname. Set the parameter to "false" to inherit the grid-level setting for ddns_domainname.

The default value of this parameter is "false".

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Getting override_ddns_domainname
my $override_ddns_domainname=$sharednetwork->override_ddns_domainname( );
#Modifying override_ddns_domainname
$sharednetwork->override_ddns_domainname("true");
```

override_ddns_enable_option_fqdn( )

The override_ddns_enable_option_fqdn attribute controls whether the enable_ddns value in the DHCP IPv6 shared network is used, instead of the grid default.

The override_ddns_enable_option_fqdn attribute can be specified explicitly. It is also set implicitly when enable_ddns is set to a defined value.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

Set the parameter to "true" to override the grid-level setting for enable_ddns. Set the parameter to "false" to inherit the grid-level setting for enable_ddns.

The default value of this parameter is "false".

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Getting override_ddns_enable_option_fqdn
my $override_ddns_enable_option_fqdn=$sharednetwork->override_ddns_enable_option_fqdn( );
#Modifying override_ddns_enable_option_fqdn
$sharednetwork->override_ddns_enable_option_fqdn("true");
```

override_ddns_generate_hostname( )

The override_ddns_generate_hostname attribute controls whether the enable_ddns value in the DHCP IPv6 shared network is used, instead of the grid default.

The override_ddns_generate_hostname attribute can be specified explicitly. It is also set implicitly when enable_ddns is set to a defined value.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

Set the parameter to "true" to override the grid-level setting for enable_ddns. Set the parameter to "false" to inherit the grid-level setting for enable_ddns.

The default value of this parameter is "false".

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Getting override_ddns_generate_hostname
my $override_ddns_generate_hostname=$sharednetwork->override_ddns_generate_hostname( );
#Modifying override_ddns_generate_hostname
$sharednetwork->override_ddns_generate_hostname("true");
```

override_ddns_ttl( )

The override_ddns_ttl attribute controls whether the ddns_ttl value in the DHCP shared network object is used, instead of the grid default.

The override_ddns_ttl attribute can be specified explicitly. It is also set implicitly when ddns_ttl is set to a defined value.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

Set the parameter to "true" to override the grid-level setting for ddns_ttl. Set the parameter to "false" to inherit the grid-level setting for ddns_ttl.

The default value of this parameter is "false".

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Getting override_ddns_ttl
my $override_ddns_ttl=$sharednetwork->override_ddns_ttl( );
#Modifying override_ddns_ttl
$sharednetwork->override_ddns_ttl("true");
```

override_domain_name( )

The override_domain_name attribute controls whether the domain_name value in the DHCP IPv6 shared network is used, instead of the grid default.

The override_domain_name attribute can be specified explicitly. It is also set implicitly when domain_name is set to a defined value.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

Set the parameter to "true" to override the grid-level setting for domain_name. Set the parameter to "false" to inherit the grid-level setting for domain_name.

The default value of this parameter is "false".

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Getting override_domain_name
my $override_domain_name=$sharednetwork->override_domain_name( );
#Modifying override_domain_name
$sharednetwork->override_domain_name("true");
```

override_domain_name_servers( )

The override_domain_name_servers attribute controls whether the domain_name_servers value in the DHCP IPv6 shared network is used, instead of the grid default.

The override_domain_name_servers attribute can be specified explicitly. It is also set implicitly when domain_name_servers is set to a defined value.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

Set the parameter to "true" to override the grid-level setting for domain_name_servers. Set the parameter to "false" to inherit the grid-level setting for domain_name_servers.

The default value of this parameter is "false".

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Getting override_domain_name_servers
my $override_domain_name_servers=$sharednetwork->override_domain_name_servers( );
#Modifying override_domain_name_servers
$sharednetwork->override_domain_name_servers("true");
```

override_enable_ddns( )

The override_enable_ddns attribute controls whether the enable_ddns value in the DHCP IPv6 shared network is used, instead of the grid default.

The override_enable_ddns attribute can be specified explicitly. It is also set implicitly when enable_ddns is set to a defined value.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

Set the parameter to "true" to override the grid-level setting for enable_ddns. Set the parameter to "false" to inherit the grid-level setting for enable_ddns.

The default value of this parameter is "false".

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Getting override_enable_ddns
my $override_enable_ddns=$sharednetwork->override_enable_ddns( );
#Modifying override_enable_ddns
$sharednetwork->override_enable_ddns("true");
```

override_options( )

The override_options attribute controls whether the options value in the DHCP IPv6 shared network is used, instead of the grid default.

The override_options attribute can be specified explicitly. It is also set implicitly when options is set to a defined value.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

Set the parameter to "true" to override the grid-level setting for options. Set the parameter to "false" to inherit the grid-level setting for options.

The default value of this parameter is "false".

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Getting override_options
my $override_options=$sharednetwork->override_options( );
#Modifying override_options
$sharednetwork->override_options("true");
```

override_preferred_lifetime( )

The override_preferred_lifetime attribute controls whether the preferred_lifetime value in the DHCP IPv6 shared network is used, instead of the grid default.

The override_preferred_lifetime attribute can be specified explicitly. It is also set implicitly when preferred_lifetime is set to a defined value.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

Set the parameter to "true" to override the grid-level setting for options. Set the parameter to "false" to inherit the grid-level setting for options.

The default value of this parameter is "false".

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Getting override_preferred_lifetime
my $override_preferred_lifetime=$sharednetwork->override_preferred_lifetime( );
#Modifying override_preferred_lifetime
$sharednetwork->override_preferred_lifetime("true");
```

override_update_dns_on_lease_renewal( )

The override_update_dns_on_lease_renewal attribute controls whether the update_dns_on_lease_renewal value in the DHCP shared network object is used, instead of the grid default.

The override_update_dns_on_lease_renewal attribute can be specified explicitly. It is also set implicitly when update_dns_on_lease_renewal is set to a defined value.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

Set the parameter to "true" to override the grid-level setting for update_dns_on_lease_renewal. Set the parameter to "false" to inherit the grid-level setting for update_dns_on_lease_renewal.

The default value of this parameter is "false".

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Getting override_update_dns_on_lease_renewal
my $override_update_dns_on_lease_renewal=$sharednetwork->override_update_dns_on_lease_renewal( );
#Modifying override_update_dns_on_lease_renewal
$sharednetwork->override_update_dns_on_lease_renewal("true");
```

override_valid_lifetime( )

The override_valid_lifetime attribute controls whether the valid_lifetime value in the DHCP IPv6 shared network is used, instead of the grid default.

The override_valid_lifetime attribute can be specified explicitly. It is also set implicitly when valid_lifetime is set to a defined value.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

Set the parameter to "true" to override the grid-level setting for options. Set the parameter to "false" to inherit the grid-level setting for options.

The default value of this parameter is "false".

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Getting override_valid_lifetime
my $override_valid_lifetime=$sharednetwork->override_valid_lifetime( );
#Modifying override_valid_lifetime
$sharednetwork->override_valid_lifetime("true");
```

preferred_lifetime( )

Use this method to set or retrieve the preferred lifetime of the DHCP IPv6 shared network object.

Setting this method to a defined value implicitly sets the override_preferred_lifetime method to "true". Setting the parameter to undefined causes the appliance to use the upper-level default and automatically resets the override_preferred_lifetime attribute to "false".

Note that when preferred_lifetime is set to a defined value and override_preferred_lifetime is set to "false", the last operation takes precedence. Thus the sequence $object->preferred_lifetime(20000); $object->override_preferred_lifetime("false"); will set override_preferred_lifetime to "false", and the sequence $object->override_preferred_lifetime("false"); $object->preferred_lifetime(20000); will result in override_preferred_lifetime="true".

Parameter

Preferred lifetime value in seconds. The default value is 27000. Note that the preferred lifetime cannot be greater than the valid lifetime.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get preferred_lifetime
my $preferred_lifetime = $sharednetwork->preferred_lifetime();
#Modify preferred_lifetime
$sharednetwork->preferred_lifetime(12345);
```

update_dns_on_lease_renewal( )

The update_dns_on_lease_renewal attribute controls whether the DHCP server updates DNS when a DHCP lease is renewed.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Setting this method to a defined value implicitly sets the override_update_dns_on_lease_renewal method to "true". Setting the parameter to undefined causes the appliance to use the grid default and automatically resets the override_update_dns_on_lease_renewal attribute to "false".

Note that when update_dns_on_lease_renewal is set to a defined value and override_update_dns_on_lease_renewal is set to "false", the last operation takes precedence. Thus the sequence $object->update_dns_on_lease_renewal("true"); $object->override_update_dns_on_lease_renewal("false"); will set override_update_dns_on_lease_renewal to "false", and the sequence $object->override_update_dns_on_lease_renewal("false"); $object->update_dns_on_lease_renewal("true"); will result in override_update_dns_on_lease_renewal="true".

Parameter

Specify "true" to set the update_dns_on_lease_renewal flag or "false" to deactivate/unset it.

The default value of this parameter is "false".

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get update_dns_on_lease_renewal
my $update_dns_on_lease_renewal=$sharednetwork->update_dns_on_lease_renewal( );
#Modify update_dns_on_lease_renewal
$sharednetwork->update_dns_on_lease_renewal("true");
```

valid_lifetime( )

Use this method to set or retrieve the valid lifetime value of a DHCP IPv6 shared network object.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Setting this method to a defined value implicitly sets the override_valid_lifetime method to "true". Setting the parameter to undefined causes the appliance to use the grid default and automatically resets the override_valid_lifetime attribute to "false".

Note that when valid_lifetime is set to a defined value and override_valid_lifetime is set to "false", the last operation takes precedence. Thus the sequence $object->valid_lifetime(691200); $object->override_valid_lifetime("false"); will set override_valid_lifetime to "false", and the sequence $object->override_valid_lifetime("false"); $object->valid_lifetime(691200); will result in override_valid_lifetime="true".

Parameter

A 32-bit integer (valid range from 1 to 4294967295) that represents the lifetime, in seconds. The default value is 43200.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get valid lifetime
my $valid_lifetime = $sharednetwork->valid_lifetime();
#Modify valid lifetime
$sharednetwork->valid_lifetime(864000);
```

SAMPLE CODE

The following sample code demonstrates the different functions that can be applied to an object, such as add, search, modify, and remove. This sample also includes error handling for the operations.

#Preparation prior to a DHCP IPv6 shared network object insertion

```
#Create a session to the Infoblox device
my $session = Infoblox::Session->new(
    master   =>  "192.168.1.2",
    username => "admin",
    password => "infoblox"
);
unless ($session) {
```

```
    die("Construct session failed: ",
        Infoblox::status_code() . ":" . Infoblox::status_detail());
}
print "Session created successfully\n";

#Create a DHCP Network object>

my $network1 = Infoblox::DHCP::IPv6Network->new(
                                    "network" => "2001:10:20:30::/64",
                                );
my $network2 = Infoblox::DHCP::IPv6Network->new(
                                    "network" => "2001:10:20:40::/64",
                                );
my $response = $session->add($network1)
or die("Add \"2001:10:20:30::/64\"Network failed: ",
            $session->status_code() . ":" . $session->status_detail());

print "Network \"2001:10:20:30::/64\"added successfully\n"; my $response = $session->add($network2)
       or die("Add  \"2001:10:20:30::/64\" Network failed: ",
            $session->status_code() . ":" . $session->status_detail());

print "Network  \"2001:10:20:40::/64\",added successfully\n";
```

#Create a DHCP IPv6 Shared Network object

```
my $sharednetwork1 = Infoblox::DHCP::IPv6SharedNetwork->new(
                                    "name"                                => "myname",
                                    "comment"                             => "add shared network",
                                    "ddns_domainname"                     => 'ddns.com',
                                    "ddns_ttl"                            => 123,
                                    "networks"                            => [ $network1 , $network2 ],
                                    "disable"                             => "false",
                                    "domain_name"                         => 'bar.com',
                                    "domain_name_servers"                 => '2001::1,2001::2',
                                    "enable_ddns"                         => 'false',
                                    "ddns_generate_hostname"              => 'false',
                                    "override_ddns_domainname"            => 'true',
                                    "override_ddns_ttl"                   => 'true',
                                    "override_domain_name"                => 'true',
                                    "override_domain_name_servers"        => 'true',
                                    "override_enable_ddns"                => 'true',
                                    "override_valid_lifetime"             => 'true',
                                    "override_preferred_lifetime"         => 'true',
                                    "override_update_dns_on_lease_renewal" => 'true',
                                    "preferred_lifetime"                  => 1234,
                                    "update_dns_on_lease_renewal"         => 'true',
                                    "valid_lifetime"                      => 12345,
                                    );

my $response = $session->add( $sharednetwork1 )
or die("Add IPv6 Shared Network failed: ",
            $session->status_code() . ":" . $session->status_detail());

print "IPv6 Shared Network added successfully\n";
```

#Search for a specific IPv6 Shared Network

```
#Search the network
my @retrieved_objs = $session->search(
    object  => "Infoblox::DHCP::IPv6SharedNetwork",
    "name"  => "my.*"
);
my $object = $retrieved_objs[0];
unless ($object) {
    die("Search IPv6 Shared Network failed: ",
        $session->status_code() . ":" . $session->status_detail());
}

print "Search IPv6 Shared Network found at least 1 matching entry\n";
```

#Get and modify a IPv6 Shared Network attribute

```
#Get IPv6 Shared Network through the session
my  @retrieved_objs = $session->get(
          object =>"Infoblox::DHCP::IPv6SharedNetwork" ,
          "name" => "myname"
```

```
        );

  my $object = $retrieved_objs[0]; unless ($object) {
       die("Get IPv6 Shared Network object failed: ",
           $session->status_code() . ":" . $session->status_detail());
   }

  print "Get IPv6 Shared Network found at least 1 matching entry\n";

  #Modify one of the attributes of the specified Network attribute>
  $object->disable("true");
  #Apply the changes
  my $res = $session->modify($object) ;
  if ( !$res)
  {
       print "Modify IPv6 Shared Network attribute failed: ".$session->status_code() . ":" . $session-
>status_detail();

  }

  print "IPv6 Shared Network object modified successfully \n";
```

#Remove a IPv6 Shared Network Object

```
  #Get Network object through the session
  my @retrieved_objs = $session->get(
            object  => "Infoblox::DHCP::IPv6SharedNetwork" ,
            name    => "myname"
     );
  my $object = $retrieved_objs[0];    unless ($object) {
     die("Get IPv6 Shared Network object failed: ",
         $session->status_code() . ":" . $session->status_detail());
  }

  print "Get IPv6 Shared Network object found at least 1 matching entry\n";

  #Submit the object for removal
  $session->remove($object)
       or die("Remove IPv6 Shared Network failed: ",
           $session->status_code() . ":" . $session->status_detail());

  print "IPv6 Shared Network removed successfully \n";

  #Submit the network object for removal
  my $response = $session->remove($network1)
  or die("Remove \"2001:10:20:30::/64\"Network failed: ",
              $session->status_code() . ":" . $session->status_detail());

  print "Network \"2001:10:20:30::/64\"removed successfully\n";

  #Submit the network object for removal
  my $response = $session->remove($network2)
         or die("Remove  \"11.0.0.0\"Network failed: ",
              $session->status_code() . ":" . $session->status_detail());

  print "Network \"2001:10:20:40::/64\"removed successfully\n";

       ####PROGRAM ENDS####
```

AUTHOR

Infoblox Inc. http://www.infoblox.com/

SEE ALSO

Infoblox::Session, Infoblox::DHCP::Network, Infoblox::DHCP::View, Infoblox::DHCP::Option, Infoblox::Session->get(), Infoblox::Session->search(), Infoblox::Session->add(), Infoblox::Session->remove(), Infoblox::Session->modify()

COPYRIGHT

Copyright (c) 2017 Infoblox Inc.

Infoblox::DHCP::Lease - DHCP Lease object.

- NAME
- DESCRIPTION
- SESSION METHODS
  - Infoblox::Session->get( )
  - Infoblox::Session->remove( )
  - Infoblox::Session->search( )
- METHODS
  - billing_class( )
  - binding_state( )
  - client_hostname( )
  - cltt( )
  - discovered_data( )
  - discovered_duid( )
  - discovered_name( )
  - discoverer( )
  - ends( )
  - fingerprint( )
  - first_discovered( )
  - hardware( )
  - ip_address( )
  - ipv4addr( )
  - ipv6_duid( )
  - ipv6_iaid( )
  - ipv6_preferred_lifetime( )
  - ipv6_prefix_bits( )
  - is_invalid_mac( )
  - last_discovered( )
  - mac( )
  - ms_ad_user_data( )
  - network( )
  - network_component_description( )
  - network_component_ip( )
  - network_component_name( )
  - network_component_port_description( )
  - network_component_port_name( )
  - network_component_port_number( )
  - network_component_type( )
  - network_view( )
  - netbios( )
  - next_binding_state( )
  - on_commit( )
  - on_expiry( )
  - on_release( )
  - option( )
  - os( )
  - port_duplex( )
  - port_link_status( )
  - port_speed( )
  - port_status( )
  - port_vlan_description( )
  - port_vlan_name( )
  - port_vlan_number( )
  - protocol( )
  - remote_id( )
  - starts( )
  - tsfp( )
  - tstp( )
  - uid( )
  - username( )
  - served_by( )
  - variable( )
  - v_cluster( )
  - v_datacenter( )
  - v_host( )
  - v_name( )
  - v_netadapter( )
  - v_switch( )
  - v_type( )
- SAMPLE CODE
- AUTHOR

- SEE ALSO
- COPYRIGHT

NAME

Infoblox::DHCP::Lease - DHCP Lease object.

DESCRIPTION

A DHCP lease provides information about the lease of an IP address to a DHCP client, such as the time when the Infoblox appliance issued or freed an IP address, the MAC address and host name of the client that received the IP address, and the grid member that supplied the lease. It allows the Infoblox appliance to store and correlate DHCP lease information over the lifetime of a lease.

SESSION METHODS

This section describes all the methods in the Infoblox::Session module that can be applied to a DHCP Lease object.

Infoblox::Session->get( )

Use this method to retrieve all the matching DHCP Lease objects from the Infoblox appliance. See Infoblox::Session->get() for parameters and return values.

Key References

```
 Apply the following attributes to get a specific DHCP Lease object:

 ip_address                          - Optional. IPv4 or IPv6 address of the DHCP lease. Use this,
in place of ipv4addr, even for IPv4 addresses.
 ipv4addr                            - Optional. IPv4 address (32 bits) of the DHCP lease. This
attribute is being deprecated in favor of the ip_address attribute.
 ipv6_duid                           - Optional. IPv6 DUID of the DHCP lease. (For IPv6 leases
only)
 protocol                            - Optional. Specify whether to get IPv4, IPv6, or both leases.
Allowed values are: 'IPV4', 'IPV6', and 'BOTH'.
 start_addr                          - Optional. Start IPv4 or IPv6 address of the DHCP range.
 end_addr                            - Optional. End IPv4 or IPv6 address of the DHCP range.
 network_view                        - Optional. Name of the network view that contains the lease.
 mac                                 - Optional. MAC address of the DHCP lease. (For IPv4 leases
only)
 username                            - Optional. User name associated with the DHCP lease.
 client_hostname                     - Optional. Client hostname of a DHCP lease.
 fingerprint                         - Optional. DHCP fingerprint description.
 remote_id                           - Optional. Relay agent remote ID, a sub-option of DHCP option
82. (For IPv4 leases only)

 discovered_duid                     - Optional. Discovered DHCPv6 UID.
 discovered_name                     - Optional. Discovered name.
 discoverer                          - Optional. Discoverer name.
 first_discovered                    - Optional. Time of the first discovery.
 last_discovered                     - Optional. Time of the last discovery.
 netbios                             - Optional. Discovered NetBIOS name.
 network_component_description       - Optional. Discovered network component description.
 network_component_ip                - Optional. Discovered network component IP address.
 network_component_name              - Optional. Discovered network component name.
 network_component_port_description  - Optional. Discovered network component port description.
 network_component_port_name         - Optional. Discovered network component port name.
 network_component_port_number       - Optional. Discovered network component port number.
 network_component_type              - Optional. Discovered network component type.
 os                                  - Optional. Discovered OS.
 port_duplex                         - Optional. Discovered port duplex settings.
 port_link_status                    - Optional. Discovered port link status.
 port_speed                          - Optional. Discovered port speed settings.
 port_status                         - Optional. Discovered port status settings.
 port_vlan_description               - Optional. Discovered port VLAN description.
 port_vlan_name                      - Optional. Discovered port VLAN name.
 port_vlan_number                    - Optional. Discovered port VLAN number.
 v_cluster                           - Optional. Discovered VMware cluster name. (For IPv4 leases
only)
 v_datacenter                        - Optional. Discovered VMware datacenter name. (For IPv4
leases only)
 v_host                              - Optional. Discovered VMware server name. (For IPv4 leases
only)
 v_name                              - Optional. Discovered VMware host or virtual machine name.
(For IPv4 leases only)
 v_netadapter                        - Optional. Discovered VMware physical network adapter name.
(For IPv4 leases only)
 v_switch                            - Optional. Discovered VMware virtual switch name. (For IPv4
leases only)
 v_type                              - Optional. Discovered VMware entity type (possible values:
"VirtualMachine", "HostSystem", "VirtualCenter").
```

Example

```
#Get a DHCP lease by IP address.
```

```
my @retrieved_objs = $session->get(
                            object   => "Infoblox::DHCP::Lease",
                            ip_address => "10.0.0.10"
                            mac      => "22:22:22:22:22:22",
                            username => "johndoe"
                            );
my @retrieved_objs = $session->get(
                            object   => "Infoblox::DHCP::Lease",
                            ip_address => "2001::1"
                            username => "johndoe"
                            );

#Get DHCP leases in a DHCP range.
my @retrieved_objs = $session->get(
                            object     => "Infoblox::DHCP::Lease" ,
                            start_addr => "10.0.0.1",
                            end_addr   => "10.0.0.10"
                            );
my @retrieved_objs = $session->get(
                            object     => "Infoblox::DHCP::Lease" ,
                            start_addr => "2001::1",
                            end_addr   => "2001::5"
                            );

#Get DHCP leases by v_type:
my @retrieved_objs = $session->get(
  object    => 'Infoblox::DHCP::Lease',
  v_type    => 'VirtualMachine');
```

Infoblox::Session->remove( )

Use this method to remove a DHCP Lease object from the Infoblox appliance. See Infoblox::Session->remove() for parameters and return values.

To remove a specific object, first use `get()` or `search()` to retrieve the specific object, and then submit this object for removal.

Example

```
#Remove a DHCP IPv4 Lease object.
my $object = $session->get(
    object   => "Infoblox::DHCP::Lease",
    ip_address => "10.0.0.10"
    mac      => "22:22:22:22:22:22",
    username => "johndoe"
);
my $response = $session->remove($object);

#Remove a DHCP IPv6 Lease object.
my $object = $session->get(
    object   => "Infoblox::DHCP::Lease",
    ip_address => "2001::1"
    username => "johndoe"
);
my $response = $session->remove($object);
```

Infoblox::Session->search( )

Use this method to search for DHCP Lease objects in the Infoblox appliance. See Infoblox::Session->search() for parameters and return values.

Key References

```
  Apply the following attributes to search for a DHCP Lease object:

  ip_address                          - Optional. IPv4 or IPv6 address of the DHCP lease. Use this,
in place of ipv4addr, even for IPv4 addresses (regular expression).
  ipv4addr                            - Optional. IPv4 address (32 bits) of the DHCP lease. This
attribute is being deprecated in favor of the ip_address attribute (regular exression).
  ipv6_duid                           - Optional. IPv6 DUID of the DHCP lease (regular expression).
(For IPv6 leases only)
  protocol                            - Optional. Specify whether to get IPv4, IPv6, or both
leases. Allowed values are: 'IPV4', 'IPV6', and 'BOTH'.
  mac                                 - Optional. MAC address of the DHCP lease (regular
expression). (For IPv4 leases only)
  network_view                        - Optional. Name of the network view that contains the lease.
  username                            - Optional. User name associated with the DHCP lease (regular
expression).
  start_addr                          - Optional. Start IPv4 or IPv6 address of the DHCP range.
  end_addr                            - Optional. End IPv4 or IPv6 address of the DHCP range.
```

```
  client_hostname                       - Optional. Client hostname of a DHCP lease (regular
expression).
  fingerprint                           - Optional. DHCP fingerprint description (regular
expression).
  remote_id                             - Optional. Relay agent remote ID, a sub-option of DHCP
option 82 (regular expression).

  discovered_duid                       - Optional. Discovered DHCPv6 UID (exact search only).
  discovered_name                       - Optional. Discovered name(regex).
  discoverer                            - Optional. Discoverer name(regex).
  first_discovered                      - Optional. Time of the first discovery (in ISO format, exact
search only).
  last_discovered                       - Optional. Time of the last discovery (in ISO format, exact
search only).
  netbios                               - Optional. Discovered NetBIOS name(regex).
  network_component_description         - Optional. Discovered network component description(regex).
  network_component_ip                  - Optional. Discovered network component IP address(regex).
  network_component_name                - Optional. Discovered network component name(regex).
  network_component_port_description    - Optional. Discovered network component port
description(regex).
  network_component_port_name           - Optional. Discovered network component port name(regex).
  network_component_port_number         - Optional. Discovered network component port number (exact
search only).
  network_component_type                - Optional. Discovered network component type(regex).
  os                                    - Optional. Discovered OS (regex).
  port_duplex                           - Optional. Discovered port duplex settings (exact search
only).
  port_link_status                      - Optional. Discovered port link status (exact search only).
  port_speed                            - Optional. Discovered port speed settings (exact search
only).
  port_status                           - Optional. Discovered port status settings (exact search
only).
  port_vlan_description                 - Optional. Discovered port VLAN description(regex).
  port_vlan_name                        - Optional. Discovered port VLAN name(regex).
  port_vlan_number                      - Optional. Discovered port VLAN number(exact search only).
  v_cluster                             - Optional. Discovered VMware cluster name (regular
expression). (For IPv4 leases only)
  v_datacenter                          - Optional. Discovered VMware datacenter name (regular
expression). (For IPv4 leases only)
  v_host                                - Optional. Discovered VMware server name (regular
expression). (For IPv4 leases only)
  v_name                                - Optional. Discovered VMware host or virtual machine name
(regular expression). (For IPv4 leases only)
  v_netadapter                          - Optional. Discovered VMware physical network adapter name
(regular expression). (For IPv4 leases only)
  v_switch                              - Optional. Discovered VMware virtual switch name (regular
expression). (For IPv4 leases only)
  v_type                                - Optional. Discovered VMware entity type (possible values:
"VirtualMachine", "HostSystem", "VirtualCenter"). (For IPv4 leases only)
```

Example

```
#Search a DHCP lease.
my @result_array = $session->search(
                    object   => "Infoblox::DHCP::Lease",
                    ip_address => "^10\.*",
                    mac      => "22:.*",
                    username => "john.*"
                    );
my @result_array = $session->search(
                    object   => "Infoblox::DHCP::Lease",
                    ip_address => "^ff::.*",
                    username => "john.*"
                    );

# search a DHCP lease by v_datacenter:
my @retrieved_objs = $session->search(
   object       => 'Infoblox::DHCP::Lease',
   v_datacenter => 'test.*');
```

METHODS

This section describes all the methods that can be used to configure and retrieve the attribute value of a DHCP Lease object.

billing_class( )

Use this method to retrieve the billing_class of a DHCP Lease object. This method specifies the class to which this lease is currently billed. This method is for IPv4 leases only.

Parameter

N/A

Returns

The method returns the attribute value.

Example

```
#Get billing_class
my $billing_class = $lease->billing_class();
```

binding_state( )

Use this method to retrieve the binding state for the current lease. The lease state can be one of the following:

```
 Free      : The lease is available for clients to use.
 Active    : The lease is currently in use by a DHCP client.
 Expired   : The lease was in use, but the DHCP client never renewed it, so it is no longer valid.
 Released  : The DHCP client returned the lease to the appliance.
 Abandoned: The Infoblox appliance cannot lease this IP address because the appliance received a response
when it pinged the address.
```

Parameter

N/A

Returns

The method returns the attribute value.

Example

```
#Get binding_state
my $binding_state = $lease->binding_state();
```

client_hostname( )

Use this method to retrieve the client_hostname of a DHCP Lease object. This method specifies the host name that the DHCP client sends to the Infoblox appliance using DHCP option 12.

Parameter

N/A

Returns

The method returns the attribute value.

Example

```
#Get client_hostname
my $client_hostname = $lease->client_hostname();
```

cltt( )

Use this method to retrieve the CLTT (Client Last Transaction Time) value of a DHCP Lease object. This method specifies the time of the last transaction with the DHCP client for this lease.

Parameter

N/A

Returns

The method returns the attribute value.

Example

```
#Get cltt
my $cltt = $lease->cltt();
```

discovered_data( )

Use this method to retrieve the discovered data of a DHCP Lease object. This is a read-only attribute.

Omit the parameter to retrieve the attribute value.

Parameter

None

Returns

The valid return value is an Infoblox::Grid::Discovery::Data object.

Example

```
#Get discovered_data value
my $discovered_data = $object->discovered_data();
```

discovered_duid( )

Use this method to retrieve the discovered DHCPv6 UID (DUID) of a DHCP Lease object.

Parameter

none

Returns

The method returns the discovered name.

Example

```
#Get discovered_duid
my $discovered_duid = $bindptr->discovered_duid();
```

discovered_name( )

Use this method to retrieve the discovered name of a DHCP Lease object.

Parameter

none

Returns

The method returns the discovered name.

Example

```
#Get discovered_name
my $discovered_name = $lease->discovered_name();
```

discoverer( )

Use this method to retrieve the discoverer of a DHCP Lease object.

Parameter

none

Returns

The method returns the discoverer value.

Example

```
#Get discoverer
my $discoverer = $lease->discoverer();
```

ends( )

Use this method to retrieve the end time value of a DHCP Lease object. This method specifies the day, date, and time that a lease ended.

Parameter

N/A

Returns

The method returns the attribute value.

Example

```
#Get ends
my $ends = $lease->ends();
```

fingerprint( )

Use this method to retrieve the descripiton of the DHCP fingerprint of a DHCP Lease object. This is a read-only attribute.

Parameter

none

Returns

The method returns the attribute value.

Example

```
#Get fingerprint
my $fingerprint = $lease->fingerprint();
```

first_discovered( )

Use this method to retrieve the date and time when the DHCP Lease object was first discovered.

Parameter

none

Returns

The method returns the date and time when the DHCP Lease object was first discovered.

Example

```
#Get first_discovered
my $first_discovered = $lease->first_discovered();
```

hardware( )

Use this method to retrieve the hardware type of a DHCP Lease object. This method specifies the MAC address of the network interface on which the lease will be used. This method is for IPv4 leases only.

Parameter

N/A

Returns

The method returns the attribute value.

Example

```
#Get hardware
my $hardware = $lease->hardware();
```

ip_address( )

Use this method to retrieve the IPv4 or IPv6 address that the Infoblox appliance leased to the DHCP client. Use this method, in place of the `ipv4addr()` method, even for IPv4 addresses.

Parameter

N/A

Returns

The method returns the attribute value.

Example

```
#Get ip_address
my $ip_address = $lease->ip_address();
```

ipv4addr( )

Use this method to retrieve the IP address that the Infoblox appliance leased to the DHCP client. This method is being deprecated in favor of the `ip_address()` method. This method is for IPv4 leases only.

Parameter

N/A

Returns

The method returns the attribute value.

Example

```
#Get ipv4addr
my $ipv4addr = $lease->ipv4addr();
```

ipv6_duid( )

Use this method to retrieve the DUID value of an IPv6 address that the Infoblox appliance leased to the DHCP client. This method is for IPv6 leases only.

Parameter

N/A

Returns

The method returns the attribute value.

Example

```
#Get ipv6_duid
my $ipv6_duid = $lease->ipv6_duid();
```

ipv6_iaid( )

Use this method to retrieve the Interface ID value of an IPv6 address that the Infoblox appliance leased to the DHCP client. This method is for IPv6 leases only.

Parameter

N/A

Returns

The method returns the attribute value.

Example

```
#Get ipv6_iaid
my $ipv6_iaid = $lease->ipv6_iaid();
```

ipv6_preferred_lifetime( )

Use this method to retrieve the preferred lifetime value of an IPv6 address that the Infoblox appliance leased to the DHCP client. This method is for IPv6 leases only.

Parameter

N/A

Returns

The method returns the attribute value.

Example

```
#Get ipv6_preferred_lifetime
my $ipv6_preferred_lifetime = $lease->ipv6_preferred_lifetime();
```

ipv6_prefix_bits( )

Use this method to retrieve the prefix bits value of an IPv6 address that the Infoblox appliance leased to the DHCP client. This method is for IPv6 leases only.

Parameter

N/A

Returns

The method returns the attribute value.

Example

```
#Get ipv6_prefix_bits
my $ipv6_prefix_bits = $lease->ipv6_prefix_bits();
```

is_invalid_mac( )

Use this method to retrieve the flag used to indicate whether an invalid MAC address is used. This is a read-only attribute.

Parameter

None

Returns

The method returns the attribute value.

Example

```
#Get is_invalid_mac value
my $is_invalid_mac = $lease->is_invalid_mac();
```

last_discovered( )

Use this method to retrieve the time when this object was last discovered.

The default value for this field is empty.

Parameter

N/A

Returns

The method returns the attribute value.

Example

```
#Get last_discovered
my $last_discovered = $lease->last_discovered();
```

mac( )

Use this method to retrieve the MAC address of the DHCP client that leased the IP address. This method is for IPv4 leases only.

Parameter

N/A

Returns

The method returns the attribute value.

Example

```
#Get mac
my $mac = $lease->mac();
```

ms_ad_user_data( )

Use this method to retrieve Microsoft Active Directory users related information. This is a read-only attribute.

Parameter

None

Returns

The valid return value is an Infoblox::Grid::MSServer::AdUser::Data object.

Example

```
#Get ms_ad_user_data
my $ms_ad_user_data = $lease->ms_ad_user_data();
```

network( )

Use this method to find the network associated with this lease. This property is read-only.

Parameter

N/A

Returns

The method returns the network associated with the lease in the string of the form "network/netmask".

Example

```
#Get network property of the lease
my $network = $lease->network();
```

network_component_description( )

Use this method to retrieve the description of the network device that is connected to the DHCP Lease object.

Parameter

none

Returns

The method returns the network device description.

Example

```
#Get network_component_description
my $network_component_description = $lease->network_component_description();
```

network_component_ip( )

Use this method to retrieve the IP address of the network device that is connected to the DHCP Lease object.

Parameter

none

Returns

The method returns the network device IP address.

Example

```
#Get network_component_ip
my $network_component_ip = $lease->network_component_ip();
```

network_component_name( )

Use this method to retrieve the name of the network device that is connected to the DHCP Lease object.

Parameter

none

Returns

The method returns the network device name.

Example

```
#Get network_component_name
my $network_component_name = $lease->network_component_name();
```

network_component_port_description( )

Use this method to retrieve the description of the network device port that is connected to the DHCP Lease object.

Parameter

none

Returns

The method returns the description of the network device port.

Example

```
#Get network_component_port_description
my $network_component_port_description = $lease->network_component_port_description();
```

network_component_port_name( )

Use this method to retrieve the name of the network device port that is connected to the DHCP Lease object.

Parameter

none

Returns

The method returns the name of the port on the network device.

Example

```
#Get network_component_port_name
my $network_component_port_name = $lease->network_component_port_name();
```

network_component_port_number( )

Use this method to retrieve the number of the network device port that is connected to the DHCP Lease object.

Parameter

none

Returns

The method returns the network device port number.

Example

```
#Get network_component_port_number
my $network_component_port_number = $lease->network_component_port_number();
```

network_component_type( )

Use this method to retrieve the type of the network device that is connected to the DHCP Lease object.

Parameter

none

Returns

The method returns the network device type.

Example

```
#Get network_component_type
my $network_component_type = $lease->network_component_type();
```

network_view( )

Use this method to retrieve the network view of the DHCP lease.

Parameter

N/A

Returns

This method returns the network view of the DHCP lease as Infoblox::DHCP::View object.

Example

```
#Get network view
my $nview = $lease->network_view();
```

netbios( )

Use this method to retrieve the name in the NetBIOS reply that responded to a NetBIOS query.

Returns

The method returns the attribute value.

Example

```
#Get netbios
```

```
my $netbios = $lease->netbios();
```

next_binding_state( )

Use this method to retrieve the subsequent binding state when the current lease expires. This method is for IPv4 leases only.

```
 Free      : The lease is available for clients to use.
 Active    : The lease is currently in use by a DHCP client.
 Expired   : The lease was in use, but the DHCP client never renewed it, so it is no longer valid.
 Released  : The DHCP client returned the lease to the appliance.
 Abandoned: The Infoblox appliance cannot lease this IP address because the appliance received a response
when it pinged the address.
```

Parameter

N/A

Returns

The method returns the attribute value.

Example

```
#Get next_binding_state
my $next_binding_state = $lease->next_binding_state();
```

on_commit( )

Use this method to retrieve the list of commands to be executed upon the granting of the lease.

Parameter

N/A

Returns

The method returns the attribute value.

Example

```
#Get on_commit
my $on_commit = $lease->on_commit();
```

on_expiry( )

Use this method to retrieve the list of commands to be executed upon the expiration of the lease.

Parameter

N/A

Returns

The method returns the attribute value.

Example

```
#Get on_expiry
my $on_expiry = $lease->on_expiry();
```

on_release( )

Use this method to retrieve the list of commands to be executed upon the release of the lease.

Parameter

N/A

Returns

The method returns the attribute value.

Example

```
#Get on_release
my $on_release = $lease->on_release();
```

option( )

Use this method to retrieve the option value of a DHCP Lease object. This method specifies the agent circuit ID and remote ID sent by a DHCP relay agent in DHCP option 82. This method is for IPv4 leases only.

Parameter

N/A

Returns

The method returns the attribute value.

Example

```
#Get option
my $option = $lease->option();
```

os( )

Use this method to retrieve the operating system associated with this DHCP lease.

Returns

The method returns the attribute value.

Example

```
#Get os
my $os = $lease->os();
```

port_duplex( )

Use this method to retrieve the duplex setting of the network device port that is connected to the DHCP Lease object.

Parameter

none

Returns

The method returns the duplex setting of the network device port. The returend value, if any, is one of the following:

```
Full
Half
```

Example

```
#Get port_duplex
my $port_duplex = $lease->port_duplex();
```

port_link_status( )

The method returns the link status of the network device port. The returned value, if any, is one of the following:

Parameter

none

Returns

The method returns the port link status. Returned values, if any, are one of the following:

```
Connected
Not Connected
Unknown
```

Example

```
#Get port_link_status
my $port_link_status = $lease->port_link_status();
```

port_speed( )

Use this method to retrieve the speed of the network device port that is connected to the DHCP Lease object.

Parameter

none

Returns

The method returns the speed value of the network device port. The returned value, if any, is one of the following:

```
10M
100M
1G
10G
100G
Unknown
```

Example

```
#Get port_speed
my $port_speed = $lease->port_speed();
```

port_status( )

Use this method to retrieve the status of the network device port that is connected to the DHCP Lease object.

Parameter

none

Returns

The method returns the status of the network device port. The returned value, if any, is one of the following:

```
Up
Down
Unknown
```

Example

```
#Get port_status
my $port_status = $lease->port_status();
```

port_vlan_description( )

Use this method to retrieve the VLAN description of the network device port that is connected to the DHCP Lease object.

Parameter

none

Returns

The method returns the VLAN description of the network device port.

Example

```
#Get port_vlan_description
my $port_vlan_description = $lease->port_vlan_description();
```

port_vlan_name( )

Use this method to retrieve the VLAN name of the network device port that is connected to the DHCP Lease object.

Parameter

none

Returns

The method returns the VLAN name of the network device port.

Example

```
#Get port_vlan_name
my $port_vlan_name = $lease->port_vlan_name();
```

port_vlan_number( )

Use this method to retrieve the VLAN number of the network device port that is connected to the DHCP Lease object.

Parameter

none

Returns

The method returns the VLAN number of the network device port.

Example

```
#Get port_vlan_number
my $port_vlan_number = $lease->port_vlan_number();
```

protocol( )

Use this method to determine whether the IP address the Infoblox appliance leased an IPv4 or IPv6 address to the client.

Parameter

N/A

Returns

The method returns the attribute value ('IPV4' or 'IPV6').

Example

```
#Get protocol
my $protocol = $lease->protocol();
```

remote_id( )

Use this method to retrieve an agent remote ID (option 82 sub-option). This is a read-only attribute.

Remote ID can be presented either in the ASCII form (e.g. "abcd") or in the form of colon separated HEX values (e.g. 1:2:ab:cd). HEX representation is only used when sub-option value contains unprintable characters. If a remode ID sub-option value is an ASCII string, it's always enclosed in quotes to prevent ambiguous values (e.g. "10:20" - ASCII 5 byte string; 10:20 - HEX 2 byte value). Remote ID can be in ASCII form (e.g. ""abcd"") or in colon-separated HEX form (e.g. "1:2:ab:cd").

```
 ASCII: ASCII representation is used if the remote ID sub-option contains only printable ASCII charaters
(ASCII characters in range "x20-0x7E").
        - The backslash symbol (\) is used as an escape symbol to escape the quote symbol (") in an ASCII
string.
        - Double backslashes (\\) are used to represent the backslash symbol (\) in an ASCII string.
 HEX  : HEX representation is used only when the remote ID sub-option value contains unprintable characters
and is normalized as follows:
        - starting zero is removed from digits: 1, a - Valid, 01, 0a - Invalid;
        - lowercase characters are used for symbols: fa - Valid, FA - Invalid.
```

NIOS does not support the convertion between HEX and ASCII formats. Searches are performed using the exact same format and value as the sub-option is represented.

Query examples assume the following leases are stored in the database:

```
 Number  : option field                                                                  : extracted remote_id field
 Lease01 : agent.remote-id="00152654358700" agent.circuit-id="BX1-PORT-003" : "00152654358700"
 Lease02 : agent.remote-id="Dhcp Relay 10" agent.circuit-id="Port008"       : "Dhcp Relay 10"
 Lease03 : agent.remote-id="00:01:02"                                                : "00:01:02"
 Lease04 : agent.remote-id=0:1:2                                                     : 0:1:2
 Lease05 : agent.remote-id=02:03                                                     : 2:3
 Lease06 : agent.remote-id=10:20                                                     : 10:20
 Lease07 : agent.circuit-id="no-remote-id"                                           :
```

Expected results:

```
 Query                    : Returned leases                                          : Comments
 remote_id=01:02          : None                                                     : EXACT query. No results
are expected
 remote_id="Dhcp Relay 10" : Lease02                                                 : EXACT query for an
ASCII value.
 remote_id=0:1:2          : Lease04                                                  : EXACT query for a HEX
value.
 remote_id=00:01:02       : None                                                     : EXACT query for a HEX
value. No results are expected as the search value is not normalized to the same format used in the
database.
 remote_id~=10            : Lease02, Lease06                                         : REGEX query.
 remote_id~=^".*1         : Lease01, Lease02, Lease03                                : REGEX query. Only ASCII
values are expected due to the starting quote (") in the search value.
 remote_id~=^[^"]*2       : Lease04, Lease05, Lease06                                : REGEX query. Only HEX
values are expected as the starting quote (") is excluded from the search value.
 remote_id=""             : None                                                     : EXACT query. No results
are expected as no leases that contain an empty remote ID value exist in the database.
 remote_id~=""            : Lease01, Lease02, Lease03, Lease04, Lease05, Lease06 : REGEX query. This query
is expected to match any lease that contains remote ID set to any value.
```

NOTE: EXACT queries (without the ~ symbol) are perfomed using the get method. REGEX queries (with the ~ symbol) are perfomed using the search method.

NOTE: Lease07 is not expected to be returned when searching for the remote ID sub-option.

Parameter

None

Returns

The method returns attribute value.

Example

```
#Get remote_id
my $remote_id = $lease->remote_id();
```

starts( )

Use this method to retrieve the start time value of a DHCP Lease object. This method specifies the day, date, and time that the lease started.

Parameter

N/A

Returns

The method returns the attribute value.

Example

```
#Get starts
my $starts = $lease->starts();
```

tsfp( )

Use this method to retrieve the TSFP (Time Sent From Partner) value of a DHCP Lease object. This method specifies the time that the current lease state ends, from the point of view of a remote DHCP failover peer. This method is for IPv4 leases only.

Parameter

N/A

Returns

The method returns the attribute value.

Example

```
#Get tsfp
my $tsfp = $lease->tsfp();
```

tstp( )

Use this method to retrieve the TSTP (Time Sent To Partner) value of a DHCP Lease object. This method specifies the time that the current lease state ends, from the point of view of a local DHCP failover peer. This method is for IPv4 leases only.

Parameter

N/A

Returns

The method returns the attribute value.

Example

```
#Get tstp
my $tstp = $lease->tstp();
```

uid( )

Use this method to retrieve the UID (User ID) value of a DHCP Lease object. This method specifies the client identifier that the DHCP client sends the Infoblox appliance (in DHCP option 61) when it acquires the lease. Not all DHCP clients send a UID. This method is for IPv4 leases only.

Parameter

N/A

Returns

The method returns the attribute value.

Example

```
#Get uid
my $uid = $lease->uid();
```

username( )

Use this method to retrieve the user name that the server has associated with a DHCP Lease object.

Parameter

N/A

Returns

The method returns the attribute value.

Example

```
#Get username
my $username = $lease->username();
```

served_by( )

Use this method to retrieve the IP address of the server that hands out an active lease to a client.

Parameter

N/A

Returns

The method returns the attribute value.

Example

```
#Get served_by
my $served_by = $lease->served_by();
```

variable( )

Use this method to retrieve the variable value of a DHCP Lease object. This method keeps all variables related to the DDNS update of the DHCP lease. The variables can be one of the following:

```
 ddns-text        : The ddns-text variable is used to record the value of the client's TXT identification
record when the
                   interim DDNS update style has been used to update the DNS service for a particular
lease.

 ddns-fwd-name    : The ddns-fwd-name variable records the value of the name used when the client's A
record was updated, if a DDNS
                   update was successfully completed by the server. The server may have used this name
when it updated the
                   client's PTR record as well.

 ddns-client-fqdn : If the server is configured to use the interim DDNS update style, and is also
configured to allow clients
                   to update their own FQDNs, and the client did in fact update its own FQDN, then the
ddns-client-fqdn
                   variable records the name that the client indicated it used. This is also the name that
the server
                   used to update the client's PTR record in this case.

 ddns-rev-name    : If the server successfully updates the client's PTR record, this variable will record
the name that the
                   DHCP server used for the PTR record. The name to which the PTR record points will be
either the
                   ddns-fwd-name or the ddns-client-fqdn.
```

Parameter

N/A

Returns

The method returns the attribute value.

Example

```
#Get variable
```

```
my $variable = $lease->variable();
```

v_cluster( )

Use this method to retrieve the name of the VMware cluster associated with the DHCP Lease object. This method is read-only. This method is for IPv4 leases only.

Parameter

none

Returns

This method returns a string that contains the VMware cluster name. The default value is an empty string.

Example

```
#Get v_cluster
my $v_cluster = $lease->v_cluster();
```

v_datacenter( )

Use this method to retrieve the name of the VMware datacenter associated with the DHCP Lease object. This method is read-only. This method is for IPv4 leases only.

Parameter

none

Returns

This method returns a string that contains the VMware datacenter name. The default value is an empty string.

Example

```
#Get v_datacenter
my $v_datacenter = $lease->v_datacenter();
```

v_host( )

Use this method to retrieve the name of the VMware host associated with the DHCP Lease object. This method is read-only. This method is for IPv4 leases only.

Parameter

none

Returns

This method returns a string that contains the VMware host name. The default value is an empty string.

Example

```
#Get v_host
my $v_host = $lease->v_host();
```

v_name( )

Use this method to retrieve the name of the VMware entity associated with the DHCP Lease object. This method is read-only. This method is for IPv4 leases only.

Parameter

none

Returns

This method returns a string that contains the VMware entity name. The default value is an empty string.

Example

```
#Get v_name
my $v_name = $lease->v_name();
```

v_netadapter( )

Use this method to retrieve the name of the physical network adapter through which the VMware entity is connected to the appliance. This

method is read-only. This method is for IPv4 leases ohly.

Parameter

none

Returns

This method returns a string that contains the network adapter name. The default value is an empty string.

Example

```
#Get v_netadapter
my $v_netadapter = $lease->v_netadapter();
```

v_switch( )

Use this method to retrieve the name of the virtual switch through which the VMware entity is connected to the appliance. This method is read-only. This method is for IPv4 leases only.

Parameter

none

Returns

This method returns a string that contains the virtual switch name. The default value is an empty string.

Example

```
#Get v_switch
my $v_switch = $lease->v_switch();
```

v_type( )

Use this method to retrieve the type of VMware entity associated with the DHCP Lease object. This method is read-only. This method is for IPv4 leases only.

Parameter

none

Returns

This method returns a string that contains the VMware entity name. The default value is an empty string. The returned value, if any, can be one of the following:

```
VirtualMachine
HostSystem
VirtualCenter
```

Example

```
#Get v_type
my $v_type = $lease->v_type();
```

SAMPLE CODE

The following sample code demonstrates the different functions that can be applied to a DHCP Lease object, such as get, search, and remove. Also, this sample code includes error handling for the operations.

#Preparation prior to DHCP Lease operations

```
use strict;
use Infoblox;

#Create a session to the Infoblox appliance
my $host_ip   = "192.168.1.2";
my $host_name = "infoblox.localdomain";
my $session = Infoblox::Session->new(
    master   => $host_ip,       #appliance host ip
    username => "admin",        #appliance user login
    password => "infoblox"      #appliance password
);
unless ($session) {
   die("Construct session failed: ",
       Infoblox::status_code() . ":" . Infoblox::status_detail());
```

```
}
print "Session created successfully.\n";

#Create a member object
 my $member = Infoblox::DHCP::Member->new(
    name     => $host_name,
    ipv4addr => $host_ip,
);

unless($member) {
     die("Construct member failed: ",
           Infoblox::status_code() . ":" . Infoblox::status_detail());
}
print "Member object created successfully\n";

#Create a network object
my $network = Infoblox::DHCP::Network->new(
                            network  => "10.0.0.0/8",
                            comment  => "add network",
                            members  => [$member]
                            );
unless($network){
       die("Construct network object fail: ",
               Infoblox::status_code(). ":" .Infoblox::status_detail());
       }

print"Network object created successfully.\n";

#Verify if the network exists
my $object = $session->get(
               object  => "Infoblox::DHCP::Network",
               network => "10.0.0.0/8"
               );
unless($object){
       print"Network does not exist on server, safe to add the network.\n";
       $session->add($network)
               or die("Add network failed: ",
                       $session->status_code(). ":" .$session->status_detail());
       }

print"Network added successfully.\n";

#Create a dhcp range object
my $dhcp_range = Infoblox::DHCP::Range->new(
    network    => "10.0.0.0/8",
    start_addr => "10.0.0.1",
    end_addr   => "10.0.0.10",
    member     => $member,
);

unless($dhcp_range){
       die("Construct dhcp range object failed: ",
               Infoblox::status_code(). ":" .Infoblox::status_detail());
       }

print "dhcp range object created successfully.\n";

#Add the dhcp range to the Infoblox appliance
$session->add($dhcp_range)
               or die("Add dhcp range failed: ",
                       $session->status_code(). ":" .$session->status_detail());

print "dhcp range added successfully.\n";

my $result = $session->import_data(
       "type" => "leases",
       "path"=> "./API_unit_test_sample_code/import_dhcpd.lease",
       "format" => "ISC",
       "lease_precedence" => "replace-previous"
      );

unless($result){
        die("Import lease failed: ",
          $session->status_code() . ":" . $session->status_detail());
 }

print "Import dhcp lease successfull.\n";
```

#Search for a DHCP Lease object

```
my @retrieved_objs= $session->search(
                    object   => "Infoblox::DHCP::Lease" ,
                    ipv4addr => '^10\..*'
                    );
my $desired_lease = $retrieved_objs[0];
unless($desired_lease){
        die("search lease failed: ",
          $session->status_code() . ":" . $session->status_detail());
 }

print "Search dhcp lease object found at least 1 matching entry\n";
```

#Get a DHCP Lease object

```
#Get a DHCP lease by IP address.
my @retrieved_objs = $session->get(
                         object   => "Infoblox::DHCP::Lease",
                         ipv4addr => "10.0.0.10"
                         );
my $desired_lease = $retrieved_objs[0];
unless($desired_lease){
        die("get lease failed: ",
          $session->status_code() . ":" . $session->status_detail());
 }

print "Get dhcp lease object found at least 1 matching entry\n";
```

#Remove a DHCP Lease object

```
#Submit the object for removal
$session->remove($desired_lease)
    or die("Remove dhcp lease failed: ",
        $session->status_code() . ":" . $session->status_detail());
print "dhcp lease object removed successfully \n";

####PROGRAM ENDS####
```

AUTHOR

Infoblox Inc. http://www.infoblox.com/

SEE ALSO

Infoblox::Session, Infoblox::Session->add(), Infoblox::Session->get(), Infoblox::Session->remove(), Infoblox::Session->search(), Infoblox::DHCP::Network, Infoblox::DHCP::View, Infoblox::DHCP::Range, Infoblox::DHCP::IPv6Network, Infoblox::DHCP::IPv6Range, Infoblox::Grid::MSServer::AdUser::Data, Infoblox::Grid::Discovery::Data

---

COPYRIGHT

Copyright (c) 2017 Infoblox Inc.

Infoblox::DHCP::MAC - DHCP MAC object.

- NAME
- DESCRIPTION
- CONSTRUCTOR
- SESSION METHODS
  - Infoblox::Session->add( )
  - Infoblox::Session->get( )
  - Infoblox::Session->modify( )
  - Infoblox::Session->remove( )
  - Infoblox::Session->search( )
- METHODS
  - authentication_time( )
  - comment( )
  - expired( )
  - expiration_time( )
  - extattrs( )
  - extensible_attributes( )
  - fingerprint( )
  - guest_custom_field1 ( )
  - guest_custom_field2 ( )
  - guest_custom_field3 ( )
  - guest_custom_field4 ( )
  - guest_email ( )
  - guest_first_name ( )
  - guest_last_name ( )
  - guest_middle_name ( )
  - guest_phone ( )
  - filter( )
  - mac( )
  - is_registered_user( )
  - reserved_for_infoblox( )
  - username( )
- SAMPLE CODE
- AUTHOR
- SEE ALSO
- COPYRIGHT

NAME

Infoblox::DHCP::MAC - DHCP MAC object.

DESCRIPTION

A MAC Address is a unique hardware identifier. The Infoblox device can filter address requests by the MAC address of a requesting hosts. The filter instructs the Infoblox device either to grant or deny an address request if the requesting host matches the filter. Also, it is possible configure specific MAC addresses within a filter to expire after a certain amount of time has passed.

CONSTRUCTOR

```
 my $mac = Infoblox::DHCP::MAC->new(
       filter                => $string,        #Required
       mac                   => $mac_addr,      #Required
       authentication_time   => $num,           #Optional / Default is empty
       comment               => $string,        #Optional / Default is empty
       expiration_time       => $num,           #Optional / Default is 0 for never expire
       extattrs              => { $string => $extattr, ... },      #Optional / Default is undefined
       extensible_attributes => { $string => $string | $num, $string => [ $string | $num, ... ], ... },
#Optional / Default is undefined
       guest_custom_field1   => $string,        #Optional / Default is empty
       guest_custom_field2   => $string,        #Optional / Default is empty
       guest_custom_field3   => $string,        #Optional / Default is empty
       guest_custom_field4   => $string,        #Optional / Default is empty
       guest_email           => $string,        #Optional / Default is empty
       guest_first_name      => $string,        #Optional / Default is empty
       guest_last_name       => $string,        #Optional / Default is empty
       guest_middle_name     => $string,        #Optional / Default is empty
       guest_phone           => $string,        #Optional / Default is empty
       is_registered_user    => "true" | "false", #Optional / Default is "true"
       reserved_for_infoblox => $string,        #Optional / Default is "false"
```

```
        username              => $string,        #Optional / Default is empty
);
```

You cannot set both extattrs and extensible_attributes attributes at the same time.

SESSION METHODS

This section describes all the methods in Infoblox::Session module that you can apply to a DHCP MAC object.

Infoblox::Session->add( )

Use this method to add a DHCP MAC object to the Infoblox appliance. See Infoblox::Session->add() for parameters and return values.

Example

```
#Construct a DHCP MAC object
my $mac = Infoblox::DHCP::MAC->new(
            "mac"                  => "00:00:00:00:00:03",
            "filter"               => "new",
            "comment"              => "dhcp mac object",
            "expiration_time"      => "0",
            "username"             => "infoblox_user",
            "authentication_time"  => "0",
            "reserved_for_infoblox" => "For Infoblox Use only",
);
#Submit for addition
my $response = $session->add($mac)
```

Infoblox::Session->get( )

Use this method to retrieve all the matching objects from the Infoblox appliance. See Infoblox::Session->get() for parameters and return values.

Key References

```
Apply the following attributes to get a specific DHCP MAC object:

 mac    - Optional. MAC address of the DHCP MAC object.
 filter - Optional. MAC filter name.
 comment  - Optional. A comment in string format.
 username - Optional. Username of the DHCP MAC object.
 fingerprint - Optional. DHCP fingerprint description.
 extattrs - Optional. A hash reference containing extensible attributes.
 extensible_attributes - Optional. A hash reference containing extensible attributes.
```

Example

```
#Get DHCP MAC object through the session
my @retrieved_objs = $session->get(
    object => "Infoblox::DHCP::MAC",
    mac    => "00:00:00:00:00:03",
    filter => "new",
);

# get all objects with a given extensible attribute
my @retrieved_objs = $session->get(
    object               => "Infoblox::DHCP::MAC",
    extensible_attributes => { 'Site' => 'Santa Clara' }
);
```

Infoblox::Session->modify( )

Use this method to modify a DHCP MAC object in the Infoblox appliance. See Infoblox::Session->modify() for parameters and return values.

Example

```
#Use method to modify the expiration time.
$mac->expiration_time("1234567890");
#Submit modification
my $response = $session->modify( $mac );
```

Infoblox::Session->remove( )

Use this method to remove a DHCP MAC object from the Infoblox appliance. See Infoblox::Session->remove() for parameters and return values.

To remove a specific object, first use `get()` or `search()` to retrieve the specific DHCP MAC object, and then submit this object for removal.

Example

```
#Get DHCP MAC object through the session
my @retrieved_objs = $session->get(
    object     => "Infoblox::DHCP::MAC",
    mac        => "00:00:00:00:00:03",
    filter     => "new",
);
#Find the desired object from the retrieved list.
my $desired_mac = $retrieved_objs[0];
#Submit for removal
my $response = $session->remove( $desired_mac );
```

Infoblox::Session->search( )

Use this method to search for DHCP MAC objects in the Infoblox appliance. See Infoblox::Session->search() for parameters and return values.

Key References

```
Apply the following attributes to search for a specific DHCP MAC Address object:

 comment  - Optional. A comment in string format (regular expression).
 mac      - Optional. MAC address of the DHCP MAC object (regular expression).
 username - Optional. Username of the DHCP MAC object (regular expression).
 filter   - Optional. MAC address filter name (regular expression).
 fingerprint - Optional. DHCP fingerprint description (regular expression).
 extattrs - Optional. A hash reference containing extensible attributes.
 extensible_attributes - Optional. A hash reference containing extensible attributes.
```

For more information about searching extensible attributes, see Infoblox::Grid::ExtensibleAttributeDef/Searching Extensible Attributes.

Example

```
# search for all DHCP MAC objects that are specified in the "new" MAC filter.
my @retrieved_objs = $session->search(
    object => "Infoblox::DHCP::MAC",
    mac    => "00:00:00:00:00.*",
    filter => "new"
);

# search all DHCP MAC associations with the extensible attribute 'Site'
my @retrieved_objs = $session->search(
   object => "Infoblox::DHCP::MAC",
   extensible_attributes => { 'Site' => 'Santa Clara' });
```

METHODS

This section describes all the methods that you can use to set and retrieve the attribute values of a DHCP MAC object.

authentication_time( )

Use this method to set or retrieve the authentication time. Indicates the time when the MAC address was first authenticated.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

The absolute UNIX time (seconds since UNIX epoch) when the address was authenticated. The maximum value can extend upto 4294967295.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get authentication_time
my $authentication_time = $mac->authentication_time();
#Modify authentication_time
$mac->authentication_time("1234567890");
```

comment( )

Use this method to set or retrieve a descriptive comment.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

Desired comment in string format with a maximum of 256 bytes.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get comment
my $comment = $mac->comment();
#Modify comment
$mac->comment("Modified DHCP MAC comment");
```

expired( )

Use this method to retrieve the expired flag of the DHCP MAC object .

Parameter

None

Returns

The method returns the attribute value.

Example

```
#Get expired
my $expired = $mac->expired();
```

expiration_time( )

Use this method to set or retrieve the expiration time of the DHCP MAC object.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

By default, the MAC address never expires; otherwise, it is the absolute UNIX time (seconds since UNIX epoch) when the address expires. The maximum value can extend upto 4294967295.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get expiration_time
my $expiration_time = $mac->expiration_time();
#Modify expiration_time
$mac->expiration_time("1234567890");
```

extattrs( )

Use this method to set or retrieve the extensible attributes associated with a DHCP MAC object.

Parameter

Valid value is a hash reference containing the names of extensible attributes and their associated values (Infoblox::Grid::Extattr objects).

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get extattrs
my $ref_extattrs = $mac->extattrs();
#Modify extattrs
$mac->extattrs({ 'Site' => $extattr1, 'Administrator' => $extattr2 });
```

extensible_attributes( )

Use this method to set or retrieve the extensible attributes associated with a DHCP MAC object.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

For valid values for extensible attributes, see Infoblox::Grid::ExtensibleAttributeDef/Extensible Attribute Values.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get extensible attributes
my $ref_extensible_attributes = $mac->extensible_attributes();
#Modify extensible attributes
$mac->extensible_attributes({ 'Site' => 'Santa Clara', 'Administrator' => [ 'Peter', 'Tom' ] });
```

fingerprint( )

Use this method to retrieve the description of the DHCP fingerprint of a DHCP MAC object. This is a read-only attribute.

Parameter

none

Returns

The method returns the attribute value.

Example

```
#Get fingerprint
my $fingerprint = $mac->fingerprint();
```

guest_custom_field1 ( )

Use this method to set or retrieve the value in Custom Field 1 of the guest registration page of this DHCP MAC object.

Parameter

The valid value is a string. The default value is undefined. The maximum length is 1023 bytes.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Getting guest_custom_field1
my $guest_custom_field1 = $dhcp->guest_custom_field1();
#Modifying guest_custom_field1
$dhcp->guest_custom_field1('Custom value 1');
```

guest_custom_field2 ( )

Use this method to set or retrieve the value in Custom Field 2 of the guest registration page of this DHCP MAC object.

Parameter

The valid value is a string. The default value is undefined. The maximum length is 1023 bytes.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Getting guest_custom_field2
my $guest_custom_field2 = $dhcp->guest_custom_field2();
#Modifying guest_custom_field2
$dhcp->guest_custom_field2('Custom value 2');
```

guest_custom_field3 ( )

Use this method to set or retrieve the value in Custom Field 3 of the guest registration page of this DHCP MAC object.

Parameter

The valid value is a string. The default value is undefined. The maximum length is 1023 bytes.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Getting guest_custom_field3
my $guest_custom_field3 = $dhcp->guest_custom_field3();
#Modifying guest_custom_field3
$dhcp->guest_custom_field3('Custom value 3');
```

guest_custom_field4 ( )

Use this method to set or retrieve the value in Custom Field 4 of the guest registration page of this DHCP MAC object.

Parameter

The valid value is a string. The default value is undefined. The maximum length is 1023 bytes.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Getting guest_custom_field4
my $guest_custom_field4 = $dhcp->guest_custom_field4();
#Modifying guest_custom_field4
$dhcp->guest_custom_field4('Custom value 4');
```

guest_email ( )

Use this method to set or retrieve the email address in the Email field of the guest registration page of this DHCP MAC object.

Parameter

The valid value is a string. The default value is undefined. The maximum length is 128 bytes.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Getting guest_email
my $guest_email = $dhcp->guest_email();
#Modifying guest_email
$dhcp->guest_email('user@domain.com');
```

guest_first_name ( )

Use this method to set or retrieve the entry in the First Name field of the guest registration page of this DHCP MAC object.

Parameter

The valid value is a string. The default value is undefined. The maximum length is 1023 bytes.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Getting guest_first_name
my $guest_first_name = $dhcp->guest_first_name();
#Modifying guest_first_name
$dhcp->guest_first_name('John');
```

guest_last_name ( )

Use this method to set or retrieve the entry in the Last Name field of the guest registration page of this DHCP MAC object.

Parameter

The valid value is a string. The default value is undefined. The maximum length is 1023 bytes.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Getting guest_last_name
my $guest_last_name = $dhcp->guest_last_name();
#Modifying guest_last_name
$dhcp->guest_last_name('Doe');
```

guest_middle_name ( )

Use this method to set or retrieve the entry in the Middle Name field of the guest registration page of this DHCP MAC object.

Parameter

The valid value is a string. The default value is undefined. The maximum length is 1023 bytes.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Getting guest_middle_name
my $guest_middle_name = $dhcp->guest_middle_name();
#Modifying guest_middle_name
$dhcp->guest_middle_name('Robert');
```

guest_phone ( )

Use this method to set or retrieve the entry in the Phone field of the guest registration page of this DHCP MAC object.

Parameter

The valid value is a string. The default value is undefined. The maximum length is 1023 bytes.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Getting guest_phone
my $guest_phone = $dhcp->guest_phone();
#Modifying guest_phone
$dhcp->guest_phone('1-555-555-5555');
```

filter( )

Use this method to set or retrieve the filter value of the DHCP MAC object .

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

Name of the DHCP MAC filter. Maximum length upto 1024 bytes is supported.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get filter
my $filter = $mac->filter();
#Modify filter
$mac->filter("filter1");
```

mac( )

Use this method to set or retrieve the MAC address of the DHCP MAC object .

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

MAC Address of the DHCP MAC object. Format for the hexadecimal strings for MAC addresses and vendor prefixes with colons or dashes. Both of the following formats are acceptable: 11:11:11:11:11:11 and 11-11-11-11-11-11.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get mac
my $mac_value = $mac->mac();
#Modify mac
$mac->mac("00:00:00:11:11:11");
```

is_registered_user( )

Use this method to set or retrieve the is_registered_user flag of the DHCP MAC object. If this flag is "false", then this DHCP MAC object represents a "Guest" registration.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

Specify "true" to set the is_registered_user flag or "false" to deactivate/unset it. The default value for this field is "true".

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
# Get is_registered_user
my $is_registered_user = $mac->is_registered_user()
# Modify is_registered_user
$mac->is_registered_user("true");
```

reserved_for_infoblox( )

Set this flag to reserve the MAC address for an Infoblox appliance.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

Reserved to write comment related to this particular MAC address. Comment cannot exceed 1024 bytes long.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get reserved_for_infoblox
my $reserved_for_infoblox = $mac->reserved_for_infoblox();
#Modify reserved_for_infoblox
$mac->reserved_for_infoblox("reserved for Infoblox");
```

username( )

Use this method to set or retrieve the user name of the DHCP MAC object .

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

User name of the DHCP MAC object. Used for authenticated DHCP purposes. Maximum of 128 characters supported.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get username
my $username = $mac->username();
#Modify username
$mac->username("user_infoblox");
```

SAMPLE CODE

The following sample code demonstrates the different functions that can be applied to an object, such as add, search, modify, and remove. This sample also includes error handling for the operations.

#Preparation prior to a DHCP MAC object insertion

```
#PROGRAM STARTS: Include all the modules that will be used
use strict;
use Infoblox;

#Create a session to the Infoblox appliance

my $session = Infoblox::Session->new(
               master   => "192.168.1.2", #appliance host ip
               username => "admin",       #appliance user login
               password => "infoblox"     #appliance password
);

unless ($session) {
       die("Construct session failed: ",
               Infoblox::status_code() . ":" . Infoblox::status_detail());
}
print "Session created successfully\n";

#Create a DHCP MAC Filter object
my $macfilter = Infoblox::DHCP::Filter::MAC->new ( "name" => "mac_filter");

unless($macfilter) {
     die("Construct mac filter failed: ",
           Infoblox::status_code() . ":" . Infoblox::status_detail());
}
print "MAC Filter object created successfully\n";

#Verify if the DHCP MAC Filter exists in the Infoblox appliance
my $object = $session->get(object => "Infoblox::DHCP::Filter::MAC", name => "mac_filter");
unless ($object) {
   print "mac filter does not exist on server, safe to add the mac filter\n";
   $session->add($macfilter)
      or die
```

```
            ("Add mac filter failed: ",
             $session->status_code() . ":" . $session->status_detail());
}
print "DHCP MAC Filter added successfully\n";
```

#Create a DHCP MAC object

```
my $mac = Infoblox::DHCP::MAC->new(
          "mac"                  => "00:00:00:00:00:01",
          "filter"               => "mac_filter",
          "comment"              => "dhcp mac object",
          "expiration_time"      => "0",
          "username"             => "infoblox_user",
          "authentication_time"  => "0",
          "reserved_for_infoblox" => "For Infoblox Use only",
);

unless($mac) {
          die("Construct MAC object failed: ",
                Infoblox::status_code() . ":" . Infoblox::status_detail());
}
print "MAC Address object created successfully\n";

#Add the MAC Address object into the Infoblox appliance through a session
$session->add($mac)
     or die("Add MAC Address object failed: ",
          $session->status_code() . ":" . $session->status_detail());
print "DHCP MAC Address object added successfully\n";
```

#Search for a specific DHCP MAC Address object

```
#Search for all DHCP MAC address objects
my @retrieved_objs = $session->search(
    object => "Infoblox::DHCP::MAC",
    filter => "mac_filter" );
my $object = $retrieved_objs[0];

unless ($object) {
       die("Search for MAC Address object failed: ",
              $session->status_code() . ":" . $session->status_detail());
}
print "Search MAC Address object found at least 1 matching entry\n";
```

#Get and modify a MAC Address object

```
#Get MAC Address object through the session
my @retrieved_objs = $session->get(
    object => "Infoblox::DHCP::MAC",
    mac    => "00:00:00:00:00:01",
);
my $object = $retrieved_objs[0];
unless ($object) {
    die("Get MAC Address object failed: ",
        $session->status_code() . ":" . $session->status_detail());
}
print "Get MAC Address object found at least 1 matching entry\n";

#Modify one of the attributes of the obtained MAC Address object
$object->expiration_time("1234567890");

#Apply the changes
$session->modify($object)
    or die("Modify MAC Address object failed: ",
          $session->status_code() . ":" . $session->status_detail());
print "MAC Address object modified successfully \n";
```

#Remove a MAC Address object

```
#Get MAC Address object through the session
my @retrieved_objs = $session->get(
    object => "Infoblox::DHCP::MAC",
    mac    => "00:00:00:00:00:01",
    filter => "mac_filter",
);
my $object = $retrieved_objs[0];
unless ($object) {
    die("Get MAC Address object failed: ",
```

```
        $session->status_code() . ":" . $session->status_detail());
}
print "Get MAC Address object found at least 1 matching entry\n";

#Submit the object for removal
$session->remove($object)
    or die("Remove MAC Address object failed: ",
        $session->status_code() . ":" . $session->status_detail());
print "MAC Address object removed successfully \n";

#Submit the MAC Address Filter object for removal
$session->remove($macfilter)
    or die("Remove MAC Address Filter object failed: ",
        $session->status_code() . ":" . $session->status_detail());
print "MAC Address Filter object removed successfully \n";

####PROGRAM ENDS####
```

AUTHOR

Infoblox Inc. http://www.infoblox.com/

SEE ALSO

Infoblox::DHCP::Filter::MAC, Infoblox::DHCP::Filter::RelayAgent, Infoblox::DHCP::Lease,Infoblox::Session->add(), Infoblox::Session->get(), Infoblox::Session->modify(), Infoblox::Session->remove(), Infoblox::Session->search(), Infoblox::Session

---

COPYRIGHT

Copyright (c) 2017 Infoblox Inc.

Infoblox::DHCP::Member - DHCP Member object.

- NAME
- DESCRIPTION
- CONSTRUCTOR
- MODULE METHODS
  - Infoblox::DHCP::Network->members( )
  - Infoblox::DHCP::Range->member( )
- METHODS
  - ipv4addr( )
  - ipv6addr( )
  - name( )
- SAMPLE CODE
- AUTHOR
- SEE ALSO
- COPYRIGHT

NAME

Infoblox::DHCP::Member - DHCP Member object.

DESCRIPTION

A grid member is part of a grid which is a group of two or more Infoblox appliances that share sections of a common, distributed, built-in database. A grid member inherits its settings from the grid. A member is a single Infoblox appliance or an HA pair that provides DHCP services seamlessly across an entire network.

CONSTRUCTOR

```
my $member = Infoblox::DHCP::Member->new(
    ipv4addr => $ipv4addr, #Optional. At least one ipv4addr or ipv6addr is required.
    ipv6addr => $ipv6addr, #Optional. At least one ipv4addr or ipv6addr is required.
    name     => $fqdn,     #Optional
);
```

MODULE METHODS

This section describes all the functions that you can apply to an DHCP Member object.

Infoblox::DHCP::Network->members( )

Use this function to specify the primary member for a network. See Infoblox::DHCP::Network->members() for parameters and return values.

Example

```
my $member1 = Infoblox::DHCP::Member->new(
    name     => "localhost.localdomain",
    ipv4addr => "192.168.1.2",
);

unless($member1) {
     die("Construct member failed: ",
           Infoblox::status_code() . ":" . Infoblox::status_detail());
}
print "Member object created successfully\n";

#Create the Network object with this member
my $network = Infoblox::DHCP::Network->new(
    network => "10.0.0.0/255.0.0.0",
    members => [$member1],
);
```

Infoblox::DHCP::Range->member( )

Use this function to specify the primary member for a DHCP range. See Infoblox::DHCP::Range->members() for parameters and return values.

Example

```
my $member = Infoblox::DHCP::Member->new(
    name     => "localhost.localdomain",
    ipv4addr => "192.168.1.2",
);

unless($member) {
     die("Construct member failed: ",
           Infoblox::status_code() . ":" . Infoblox::status_detail());
}
print "Member object created successfully\n";

#Create the DHCP Range object with this member
my $dhcp_range = Infoblox::DHCP::Range->new(
    network    => "10.0.0.0/8",
    start_addr => "10.0.0.1",
    end_addr   => "10.0.0.10",
    member     => $member,
);
```

METHODS

This section describes all the methods that you can use to set and retrieve the attribute values of a DHCP Member object.

ipv4addr( )

Use this method to set or retrieve the IPv4 address of the member.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

IPv4 address (32 bits) of the member.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get ipv4addr
my $ipv4addr = $member->ipv4addr();
#Modify ipv4addr
$member->ipv4addr("192.168.1.3");
```

ipv6addr( )

Use this method to set or retrieve the IPv6 address of the member.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

IPv6 address of the member.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get ipv6addr
my $ipv6addr = $member->ipv6addr();
#Modify ipv6addr
$member->ipv6addr("fd46:e105:2eac:89b1::1");
```

name( )

Use this method to set or retrieve the name of the member.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

A name in FQDN (Fully Qualified Domain Name) format. The FQDN consists of the host name followed by the domain name (example: abc.com). A host name can have a maximum of 256 characters.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get name
my $name = $member->name();
#Modify name
$member->name("dhcp.infoblox.com");
```

SAMPLE CODE

The following sample code demonstrates the different functions that can be applied to an object, such as add, search, modify, and remove. This sample also includes error handling for the operations.

#Preparation prior to a DHCP Member object insertion

```
#PROGRAM STARTS: Include all the modules that will be used
use strict;
use Infoblox;

#Create a session to the Infoblox appliance

my $session = Infoblox::Session->new(
               master   => "192.168.1.2", #appliance host ip
               username => "admin",       #appliance user login
               password => "infoblox"     #appliance password
);

unless ($session) {
       die("Construct session failed: ",
               Infoblox::status_code() . ":" . Infoblox::status_detail());
}
print "Session created successfully\n";
```

#Create a DHCP Member object

```
my $member1 = Infoblox::DHCP::Member->new(
    name     => "infoblox.localdomain",
    ipv4addr => "192.168.1.2",
);

unless($member1) {
     die("Construct member failed: ",
           Infoblox::status_code() . ":" . Infoblox::status_detail());
}
print "Member object created successfully\n";

#Create the Network object with this member
my $network = Infoblox::DHCP::Network->new(
                    network => "10.0.0.0/255.0.0.0",
                    comment => "add network",
                    members => [ $member1 ]
);

unless($network) {
            die("Construct Network object failed: ",
                  Infoblox::status_code() . ":" . Infoblox::status_detail());
}
print "Network object created successfully\n";

#Add the Network object into the Infoblox appliance through a session
$session->add($network)
     or die("Add Network object failed: ",
           $session->status_code() . ":" . $session->status_detail());
print "DHCP member object added to the network successfully\n";

####PROGRAM ENDS####
```

AUTHOR

Infoblox Inc. http://www.infoblox.com/

SEE ALSO

Infoblox::DHCP::Network, Infoblox::Session->add(),Infoblox::Session

---

COPYRIGHT

Copyright (c) 2017 Infoblox Inc.

Infoblox::DHCP::MSOption - Microsoft Server DHCP Option object.

- NAME
- DESCRIPTION
- CONSTRUCTOR
- MODULE METHODS
  - Infoblox::Grid::MSServer::DHCP->ms_options( )
  - Infoblox::DHCP::Range->ms_options( )
  - Infoblox::DHCP::FixedAddr->ms_options( )
- METHODS
  - name( )
  - code( )
  - type( )
  - value( )
  - user_class( )
  - vendor_class( )
- AUTHOR
- SEE ALSO
- COPYRIGHT

NAME

Infoblox::DHCP::MSOption - Microsoft (r) Server DHCP Option object.

DESCRIPTION

DHCP options describe network configuration settings and various services available on the network. These options occur as variable-length fields at the end of DHCP messages.

You can add or change only DHCP options that are in the Microsoft server and NIOS predefined list of options. Options that are not common to both are read-only.

Following are the options that the Microsoft server and the NIOS predefined list of options have in common:

```
 code     name
 ============================================================
   2     'Time Offset'
   4     'Time Server'
   5     'Name Servers'
   7     'Log Servers'
   8     'Cookie Servers'
   9     'LPR Servers'
   10    'Impress Servers'
   11    'Resource Location Servers'
   12    'Host Name'
   13    'Boot File Size'
   14    'Merit Dump File'
   16    'Swap Server'
   17    'Root Path'
   18    'Extensions Path'
   19    'IP Layer Forwarding'
   20    'Nonlocal Source Routing'
   21    'Policy Filter Masks'
   22    'Max DG Reassembly Size'
   23    'Default IP Time-to-live'
   24    'Path MTU Aging Timeout'
   25    'Path MTU Plateau Table'
   26    'MTU Option'
   27    'All subnets are local'
   29    'Perform Mask Discovery'
   30    'Mask Supplier Option'
   31    'Perform Router Discovery'
   32    'Router Solicitation Address'
   33    'Static Route Option'
   34    'Trailer Encapsulation'
   35    'ARP Cache Timeout'
   36    'Ethernet Encapsulation'
   37    'TCP Default Time-to-live'
   38    'Keepalive Interval'
   39    'Keepalive Garbage'
   40    'NIS Domain Name'
   41    'NIS Servers'
   42    'NTP Servers'
   43    'Vendor Specific Info'
   44    'WINS/NBNS Servers'
   45    'NetBIOS over TCP/IP NBDD'
   46    'WINS/NBT Node Type'
   47    'NetBIOS Scope ID'
   48    'X Window System Font'
   49    'X Window System Display'
   64    'NIS+ Domain Name'
   65    'NIS+ Servers'
```

```
   66    'Boot Server Host Name'
   67    'Bootfile Name'
   68    'Mobile IP Home Agents'
   69    'Simple Mail Transport Protocol (SMTP) Servers'
   70    'Post Office Protocol (POP3) Servers'
   71    'Network News Transport Protocol (NNTP) Servers'
   72    'World Wide Web (WWW) Servers'
   73    'Finger Servers'
   74    'Internet Relay Chat (IRC) Servers'
   75    'StreetTalk Servers'
   76    'StreetTalk Directory Assistance (STDA) Servers'
```

CONSTRUCTOR

```
  my $option = Infoblox::DHCP::MSOption->new(
     code         => $int,                              #Required
     value        => $num | $string |"true"|"false",    #Required
     name         => $string,                           #Optional
     type         => $string,                           #Optional
     vendor_class => $string,                           #Optional / If not specified, it uses the default
Microsoft vendor class
     user_class   => $string,                           #Optional / If not specified, it uses the default
Microsoft user class
   );
```

MODULE METHODS

The following functions can be applied to a DHCP Option object.

Infoblox::Grid::MSServer::DHCP->ms_options( )

Use this function to specify DHCP options at the Microsoft server level. See Infoblox::Grid::MSServer::DHCP->ms_options() for parameters and return values.

Example

```
       #Construct a DHCP Option object
       my $opt1 = Infoblox::DHCP::MSOption->new(
                                              name         => 'Time Offset',
                                              code         => 2,
                                              type         => 'long',
                                              value        => 50,
                                              vendor_class => 'DHCP Standard Options',
                                              user_class   => 'Default User Class',
                                             );
       # Configure a DHCP option for a Microsoft DHCP server
       my $response = $MSServer_DHCP->ms_options([$opt1]);
```

Infoblox::DHCP::Range->ms_options( )

Use this function to specify Microsoft DHCP options at the range level. See Infoblox::DHCP::Range->ms_options() for parameters and return values.

Example

```
       my $opt1 = Infoblox::DHCP::MSOption->new(
                                              name         => 'LPR Servers',
                                              code         => 9,
                                              type         => 'array of ip address',
                                              value        => '10.4.1.0,10.4.1.1,10.4.1.2',
                                              vendor_class => 'DHCP Standard Options',
                                              user_class   => 'Default User Class',
                                             );
       # Configure a Microsoft DHCP option for a range
       my $response = $Range->ms_options([$opt1]);
```

Infoblox::DHCP::FixedAddr->ms_options( )

Use this function to specify DHCP options at the fixed address level. See Infoblox::DHCP::FixedAddr->ms_options() for parameters and return values.

Example

```
       my $opt1 = Infoblox::DHCP::MSOption->new(
                                              name         => 'Static Route Option',
                                              code         => 33,
```

```
                                                type          => 'array of ip address pair',
                                                value         => '10.5.1.0 10.5.1.1,10.5.1.2 10.5.1.3',
                                               );
    # Configure a DHCP option for a network
    my $response = $FixedAddr->ms_options([$opt1]);
```

METHODS

This section describes all the methods that can be used to configure and retrieve the attribute values of a DHCP option.

name( )

Use this method to set or retrieve the name of the DHCP option.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

Specify the name of the Microsoft DHCP option in string format.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get name
my $name = $option->name();
#Modify name
$option->name("Static Route Option");
```

code( )

Use this method to set or retrieve the code value of the DHCP option.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

Specify the code of the DHCP option. It must be an integer from 1 to 254.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get code
my $code = $option->code();
#Modify code
$option->code("12");
```

type( )

Use this method to set or retrieve the type value of the DHCP option.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

Specify the type of the DHCP option. It must be one of the following values:

```
'binary'
'byte'
'array of byte'
'array of ip address pair'
'ip address'
'array of ip address'
'long integer'
'array of long integer'
'long'
'array of long'
'array of string'
'array of word'
'encapsulated'
'string'
```

```
'word'
```

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get type
my $type = $option->type();
#Modify type
$option->type("string");
```

value( )

Use this method to set or retrieve the custom value of the DHCP option.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

Specify the data type of the DHCP option. If the type is set to 'array of...', the accepted type is a list of the specified types.

```
  "binary"                          - Binary data.

  "byte"                            - Specify a numeric value between 0 and 255.

  "encapsulated"                    - Encapsulated data.

  "ip address"                      - Specify an IPv4 address.

  "ip address pair"                 - Specify a pair of IPv4 addresses separated by a space.

  "long"                            - Specify a numeric value between 0 and 4294967295.

  "long integer"                    - Specify a numeric value between 0 and 1.84467441 * 10^19

  "string"                          - Specify either a value in ASCII text string or
                                      a list of hexadecimal characters (0-9, a-f, or A-F) without
spaces and separated by colons.

  "word"                            - Specify a numeric value between 0 and 65535.
```

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get value
my $value = $option->value();
#Modify value
$option->value("abc.com");
```

user_class( )

Use this method to set an option space name as the user class value of a DHCP option or to retrieve an option space name.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

Specify the defined option space name in string format. The default user class name is "Default User Class".

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get user_class
my $name = $option->user_class();
#Modify user_class
$option->user_class("Default User Class");
```

vendor_class( )

Use this method to set an option space name as the vendor class value of a DHCP option or to retrieve an option space name.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

Specify the defined option space name in string format. The default vendor space name is "DHCP Standard Options".

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get vendor_class
my $name = $option->vendor_class();
#Modify vendor_class
$option->vendor_class("DHCP Standard Options");
```

AUTHOR

Infoblox Inc. http://www.infoblox.com/

SEE ALSO

Infoblox::Session, Infoblox::Session->get(), Infoblox::Session->modify(),Infoblox::Grid::MSServer::DHCP, Infoblox::DHCP::Range, Infoblox::DHCP::FixedAddr

---

COPYRIGHT

Copyright (c) 2017 Infoblox Inc.

Infoblox::DHCP::MSServer - Microsoft DHCP Server object.

- NAME
- DESCRIPTION
- CONSTRUCTOR
- MODULE METHODS
  - Infoblox::DHCP::FixedAddr->member( ), Infoblox::DHCP::Network->members( ), Infoblox::DHCP::NetworkTemplate->members( ), Infoblox::DHCP::RangeTemplate->member( ), Infoblox::DHCP::Range->member( )
- METHODS
  - address( )
  - name( )
- SAMPLE CODE
- AUTHOR
- SEE ALSO
- COPYRIGHT

NAME

Infoblox::DHCP::MSServer - Microsoft (r) DHCP Server object.

DESCRIPTION

This object is used to reference a Microsoft DHCP server.

CONSTRUCTOR

```
my $member = Infoblox::DHCP::MSServer->new(
    address  => $address, #Required
    name     => $name,    #Optional
);
```

MODULE METHODS

This section describes all the functions that you can apply to a DHCP MSServer object.

Infoblox::DHCP::FixedAddr->member( ), Infoblox::DHCP::Network->members( ), Infoblox::DHCP::NetworkTemplate->members( ), Infoblox::DHCP::RangeTemplate->member( ), Infoblox::DHCP::Range->member( )

Use this function to specify a member for any of the above objects. See their respective documentation pages for parameters and return values.

Example

```
my $member1 = Infoblox::DHCP::MSServer->new(
    address => "10.0.0.1",
);

#Create the Network object with this member
my $network = Infoblox::DHCP::Network->new(
    network => "10.0.0.0/255.0.0.0",
    members => [$member1],
);
```

METHODS

This section describes all the methods that you can use to set and retrieve the attribute values of a DHCP MSServer object.

address( )

Use this method to set or retrieve the address of the member.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

The Microsoft DNS server address in IPV4 format.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get address
my $address = $member->address();
#Modify address
$member->address("192.168.1.3");
```

name( )

Use this method to set or retrieve the name of the member.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

The Microsoft DNS server name in FQDN format.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get name
my $name = $member->name();
#Modify name
$member->name("msserver.com");
```

SAMPLE CODE

The following sample code demonstrates the different functions that can be applied to an object, such as add, search, modify, and remove. This sample also includes error handling for the operations.

#Preparation prior to a DHCP MSServer object insertion

```
#PROGRAM STARTS: Include all the modules that will be used
use strict;
use Infoblox;

#Create a session to the Infoblox appliance

my $session = Infoblox::Session->new(
               master   => "192.168.1.2", #appliance host ip
               username => "admin",       #appliance user login
               password => "infoblox"     #appliance password
);

unless ($session) {
       die("Construct session failed: ",
               Infoblox::status_code() . ":" . Infoblox::status_detail());
}
print "Session created successfully\n";

#Create a grid member object

my $grid_member = Infoblox::Grid::Member->new(
    name          => "hostname.com",
    ipv4addr      => "3.0.0.2",
    mask          => "255.0.0.0",
    gateway       => "3.0.0.1",
    comment       => "Script Generated"
    );
unless ($grid_member) {
       die("Construct grid member object failed: ",
               Infoblox::status_code() . ":" . Infoblox::status_detail());
}
print "Grid member object created successfully\n";

#Adding the member object to appliance through session
```

```
my $response = $session->add( $grid_member );

unless ($response) {
    die("Add grid member failed: ",
            $session->status_code() . ":" . $session->status_detail());
}
print"Grid member added successfully \n";

my $msserver = Infoblox::Grid::MSServer->new(
                                            address                  => '10.0.0.1',
                                            comment                  => 'basic member',
                                            disable                  => 'true',
                                            extensible_attributes    => { Site => 'Local'},
                                            login                    => 'user',
                                            password                 => 'password',
                                            managing_member          => 'hostname.com',
                                            read_only                => 'false',
                                            synchronization_interval => 5,
                                            logging_mode             => 'minimum',
                                           );

unless ($msserver) {
       die("Construct grid MS DNS server object failed: ",
               Infoblox::status_code() . ":" . Infoblox::status_detail());
}
print "Grid MS DNS server object created successfully\n";

#Adding the member object to appliance through session
$response = $session->add( $msserver );

unless ($response) {
    die("Add grid MS DNS server failed: ",
            $session->status_code() . ":" . $session->status_detail());
}
print"Grid MS DNS server added successfully \n";
```

#Create a DHCP MSServer object

```
my $member1 = Infoblox::DHCP::MSServer->new(
    address => "10.0.0.1",
);

unless($member1) {
     die("Construct member failed: ",
            Infoblox::status_code() . ":" . Infoblox::status_detail());
}
print "Member object created successfully\n";

#Create the Network object with this member
my $network = Infoblox::DHCP::Network->new(
                    network => "10.0.0.0/255.0.0.0",
                    comment => "add network",
                    members => [ $member1 ]
);

unless($network) {
             die("Construct Network object failed: ",
                   Infoblox::status_code() . ":" . Infoblox::status_detail());
}
print "Network object created successfully\n";

#Add the Network object into the Infoblox appliance through a session
$session->add($network)
     or die("Add Network object failed: ",
            $session->status_code() . ":" . $session->status_detail());
print "DHCP member object added to the network successfully\n";

####PROGRAM ENDS####
```

AUTHOR

Infoblox Inc. http://www.infoblox.com/

SEE ALSO

Infoblox::DHCP::Network, Infoblox::Session->add(),Infoblox::Session

---

COPYRIGHT

Copyright (c) 2017 Infoblox Inc.

Infoblox::DHCP::MSSuperscope - Microsoft Server Superscope object.

- NAME
- DESCRIPTION
- CONSTRUCTOR
- SESSION METHODS
  - Infoblox::Session->add( )
  - Infoblox::Session->get( )
  - Infoblox::Session->modify( )
  - Infoblox::Session->remove( )
  - Infoblox::Session->search( )
- METHODS
  - comment( )
  - disable( )
  - extattrs( )
  - extensible_attributes( )
  - name( )
  - network_view( )
  - ranges( )
  - range_high_water_mark( )
  - range_low_water_mark( )
- SAMPLE CODE
- AUTHOR
- SEE ALSO
- COPYRIGHT

NAME

Infoblox::DHCP::MSSuperscope - Microsoft (r) Server Superscope object.

DESCRIPTION

A superscope is a feature of Microsoft DHCP servers. You can use a superscope to group and activate multiple ranges via a single object.

CONSTRUCTOR

```
 my $superscope = Infoblox::DHCP::Superscope->new(
                  name                          => $string,                              #Required
                  ranges                        => [ $dhcprange1,$dhcprange2,...],       #Required
                  comment                       => $string,                              #Optional
                  extattrs                      => { $string => $extattr, ... },         #Optional / Default
is undefined
                  extensible_attributes         => { $string => $string | $num, $string => [ $string |
$num, ... ], ... },
                                                                                         #Optional / Default
is undefined
                  network_view                  => $NetworkView,                         #Optional / Default
is the default network view
                  disable                       => "true" | "false",                     #Optional / Default
is "false"
            )
```

You cannot set both extattrs and extensible_attributes attributes at the same time.

SESSION METHODS

This section describes all the methods in an Infoblox::Session module that you can apply to a superscope object.

Infoblox::Session->add( )

Use this method to add a superscope object to the Infoblox appliance. See Infoblox::Session->add() for parameters and return values.

Example

```
       #Construct an object
       my $superscope = Infoblox::DHCP::MSSuperscope->new(
                                                          name                  => 'test superscope',
                                                          comment               => 'test comment',
                                                          disable               => 'true',
                                                          network_view          => $view,
```

```
                                                    ranges                => [ $dhcp_range ],
                                                  );

    # Submit for addition
    my $response = $session->add( $superscope );
```

Infoblox::Session->get( )

Use this method to retrieve all the matching objects from the Infoblox appliance. See Infoblox::Session->get() for parameters and return values.

Key References

```
    Apply the following attributes to get a specific superscope object:

    name                  - Optional. Name value for the superscope.
    comment               - Optional. Comment value for the superscope.
    network_view          - Optional. Network view in which the superscope is located. By default, all
   network views are searched.
    extattrs              - Optional. A hash reference containing extensible attributes.
    extensible_attributes  - Optional. A hash reference containing extensible attributes.
```

Examples

```
    my @retrieved_objs = $session->get(
                         object     => "Infoblox::DHCP::MSSuperscope",
                         name       => "test superscope" );

    my @retrieved_objs = $session->get(
                         object       => "Infoblox::DHCP::MSSuperscope",
                         name         => "other superscope",
                         network_view => "external" );

    my @retrieved_objs = $session->get(
                         object               => "Infoblox::DHCP::MSSuperscope",
                         extensible_attributes => { 'Site' => 'Santa Clara' });
```

Infoblox::Session->modify( )

Use this method to modify a superscope object in the Infoblox appliance. See Infoblox::Session->modify() for parameters and return values.

Example

```
    # Use this method to modify the comment.
    $superscope->comment("this is a modified comment");
    # Submit modification
    my $response = $session->modify( $superscope );
```

Infoblox::Session->remove( )

Use this method to remove a superscope object from the Infoblox appliance. See Infoblox::Session->remove() for parameters and return values.

To remove a specific object, first use `get()` or `search()` to retrieve the superscope object, and then submit this object for removal.

Example

```
    # Get the objects with the same name
    my @retrieved_objs = $session->get(
        object       => "Infoblox::DHCP::MSSuperscope",
        name         => "some superscope",
        newtork_view => "default" );
    # Find the desired object from the retrieved list.
    my $desired_superscope = $retrieved_objs[0];
    # Submit for removal
    my $response = $session->remove( $desired_superscope );
```

Infoblox::Session->search( )

Use this method to search for superscope objects in the Infoblox appliance. See Infoblox::Session->search() for parameters and return values.

Key References

```
    Apply the following attributes to search for a superscope object:

     name                  - Optional. Name value for the superscope (regular expression).
```

```
  comment               - Optional. Comment value for the superscope (regular expression).
  network_view          - Optional. Network view in which the superscope is located. By default, all
network views are searched.
  extattrs              - Optional. A hash reference containing extensible attributes.
  extensible_attributes  - Optional. A hash reference containing extensible attributes.
```

For more information about searching extensible attributes, see Infoblox::Grid::ExtensibleAttributeDef/Searching Extensible Attributes.

Examples

```
# search for superscope objects with comments ending with 'comment'
my @retrieved_objs = $session->search(
    object     => "Infoblox::DHCP::MSSuperscope",
    comment    => ".*comment" );
```

METHODS

This section describes all the methods that can be used to configure and retrieve the attribute values of a superscope.

comment( )

Use this method to set or retrieve a descriptive comment.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

Desired comment in string format with a maximum of 256 bytes. The default value is undefined.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get comment
my $comment = $superscope->comment();
#Modify comment
$superscope->comment("Modifying the Superscope comment");
```

disable( )

Use this method to set or retrieve the disable flag of a superscope.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

Specify "true" to set the disable flag or "false" to deactivate/unset it. The default value for this field is false.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get disable
my $disable = $superscope->disable();
#Modify disable
$superscope->disable("false");
```

extattrs( )

Use this method to set or retrieve the extensible attributes associated with a superscope object.

Parameter

Valid value is a hash reference containing the names of extensible attributes and their associated values (Infoblox::Grid::Extattr objects).

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get extattrs
my $ref_extattrs = $superscope->extattrs();
#Modify extattrs
$superscope->extattrs({ 'Site' => $extattr1, 'Administrator' => $extattr2 });
```

extensible_attributes( )

Use this method to set or retrieve the extensible attributes that are associated with a superscope object.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

For valid values for extensible attributes, see Infoblox::Grid::ExtensibleAttributeDef/Extensible Attribute Values.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get extensible attributes
my $ref_extensible_attributes = $superscope->extensible_attributes();
#Modify extensible attributes
$superscope->extensible_attributes({ 'Site' => 'Santa Clara', 'Administrator' => [ 'Peter', 'Tom' ]
});
```

name( )

Use this method to set or retrieve the name of the superscope.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

Specify the name of the superscope in string format. The string must not be longer than 256 characters.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get name
my $name = $superscope->name();
#Modify name
$superscope->name("superscope name");
```

network_view( )

Use this method to set or retrieve the network view of the superscope.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

The valid value is a defined Infoblox::DHCP::View object. The default network view value is the system-defined default network view object.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get network view
my $nview = $superscope->network_view();
#Modify network view, with an Infoblox::DHCP::View object
$superscope->network_view($nview);
```

ranges( )

Use this method to set or retrieve the DHCP ranges that are associated with this superscope.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

Valid value is an array reference that contains defined Infoblox::DHCP::Range objects. The default value is empty.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get ranges
my $ref_options = $superscope->ranges();
#Modify ranges
$superscope->ranges([$range]); #$range is an Infoblox::DHCP::Range object
```

range_high_water_mark( )

Use this method to retrieve the percent above "Normal" when the DHCP utilization status becomes "High".

Parameter

None

Returns

The method returns the attribute value.

Example

```
#Get high-water mark
my $range_high_water_mark = $superscope->range_high_water_mark();
```

range_low_water_mark( )

Use this method to retrieve the percent below "Normal" when the DHCP utilization status becomes "Low".

Parameter

None

Returns

The method returns the attribute value.

Example

```
#Get low-water mark
my $range_low_water_mark = $superscope->range_low_water_mark();
```

SAMPLE CODE

The following sample code demonstrates the different functions that can be applied to a superscope object such as add, modify, and remove. Also, this sample includes error handling for the operations.

#Preparation prior to a DHCP superscope object insertion

```
#PROGRAM STARTS: Include all the modules that will be used
use strict;
use Infoblox;

#Create a session to the Infoblox appliance
my $session = Infoblox::Session->new(
               master   => "192.168.1.2", #appliance host ip
               username => "admin",       #appliance user login
               password => "infoblox"     #appliance password
);
unless ($session) {
       die("Construct session failed: ",
```

```
            Infoblox::status_code() . ":" . Infoblox::status_detail());
}
print "Session created successfully\n";
```

#Create supporting objects, $host_name is the grid member's fqdn

```
my $ms_member = Infoblox::Grid::MSServer->new(
                                              address                  => '10.0.0.1',
                                              comment                  => 'basic member',
                                              disable                  => 'true',
                                              extensible_attributes    => { Site => 'Somewhere'},
                                              login                    => 'user',
                                              password                 => 'password',
                                              managing_member          => $host_name,
                                              read_only                => 'false',
                                              synchronization_interval => 5,
                                              logging_mode             => 'minimum',
                                             );

unless($ms_member) {
       die("Construct MS Member object failed: ",
            Infoblox::status_code() . ":" . Infoblox::status_detail());
}
print "MS Member object created successfully\n";

unless($session->add($ms_member)) {
       die("Add MS Member object failed: ",
            $session->status_code() . ":" . $session->status_detail());
}
print "MS Member object added successfully\n";

my $ms_dhcp = Infoblox::DHCP::MSServer->new(address => '10.0.0.1');

unless($ms_dhcp) {
       die("Construct MS DHCP Member object failed: ",
            Infoblox::status_code() . ":" . Infoblox::status_detail());
}
print "MS DHCP Member object created successfully\n";

my $network1 = Infoblox::DHCP::Network->new(
                                            network   => "10.0.0.0/255.255.0.0",
                                            comment   => "add network",
                                            members   => [$ms_dhcp],
                                           );
unless($network1) {
       die("Construct DHCP Network object failed: ",
            Infoblox::status_code() . ":" . Infoblox::status_detail());
}
print "DHCP Network object created successfully\n";

unless($session->add($network1)) {
       die("Add DHCP Network object failed: ",
            $session->status_code() . ":" . $session->status_detail());
}
print "DHCP Network object added successfully\n";

my $dhcp_range1 = Infoblox::DHCP::Range->new(
                                             end_addr                 => "10.0.0.10",
                                             network                  => "10.0.0.0/16",
                                             start_addr               => "10.0.0.1",
                                             disable                  => "true",
                                             member                   => $ms_dhcp,
                                            );

unless($dhcp_range1) {
       die("Construct DHCP Range object failed: ",
            Infoblox::status_code() . ":" . Infoblox::status_detail());
}
print "DHCP Range object created successfully\n";

unless($session->add($dhcp_range1)) {
       die("Add DHCP Range object failed: ",
            $session->status_code() . ":" . $session->status_detail());
}
print "DHCP Range object added successfully\n";
```

```
my $network2 = Infoblox::DHCP::Network->new(
                                          network   => "10.1.0.0/255.255.0.0",
                                          comment   => "add network",
                                          members   => [$ms_dhcp],
                                         );
unless($network2) {
       die("Construct DHCP Network object failed: ",
            Infoblox::status_code() . ":" . Infoblox::status_detail());
}
print "DHCP Network object created successfully\n";

unless($session->add($network2)) {
       die("Add DHCP Network object failed: ",
            $session->status_code() . ":" . $session->status_detail());
}
print "DHCP Network object added successfully\n";

my $dhcp_range2 = Infoblox::DHCP::Range->new(
                                          end_addr                => "10.1.0.30",
                                          network                 => "10.1.0.0/16",
                                          start_addr              => "10.1.0.21",
                                          disable                 => "true",
                                          member                  => $ms_dhcp,
                                         );

unless($dhcp_range2) {
       die("Construct DHCP Range object failed: ",
            Infoblox::status_code() . ":" . Infoblox::status_detail());
}
print "DHCP Range object created successfully\n";

unless($session->add($dhcp_range2)) {
       die("Add DHCP Range object failed: ",
            $session->status_code() . ":" . $session->status_detail());
}
print "DHCP Range object added successfully\n";
```

#Create a DHCP Superscope object

```
my $superscope = Infoblox::DHCP::MSSuperscope->new(
                                                name                    => 'test superscope',
                                                comment                 => 'test comment',
                                                disable                 => 'true',
                                                ranges                  => [ $dhcp_range1, $dhcp_range2],
                                                );
unless($superscope) {
       die("Construct DHCP Superscope object failed: ",
            Infoblox::status_code() . ":" . Infoblox::status_detail());
}
print "DHCP Superscope object created successfully\n";

unless($session->add($superscope)) {
       die("Add DHCP Superscope object failed: ",
            $session->status_code() . ":" . $session->status_detail());
}
print "DHCP Superscope object added successfully\n";
```

#Get the DHCP Superscope object and change its ranges

```
#Get the DHCP Superscope object
my @retrieved_objs = $session->get(
    object => "Infoblox::DHCP::MSSuperscope",
    name   => "test superscope"
);
my $object = $retrieved_objs[0];
unless ($object) {
       die("Get DHCP Superscope object failed: ",
            $session->status_code() . ":" . $session->status_detail());
}
print "Get DHCP Superscope object found at least 1 matching entry\n";

#Apply the changes
$session->modify($object)
    or die("Modify DHCP Superscope object failed: ",
            $session->status_code() . ":" . $session->status_detail());
```

```
print "DHCP Superscope object with new range assignments updated to Infoblox appliance successfully\n";
```

#Remove DHCP Superscope and support objects

```
$session->remove($object)
    or die("Remove DHCP Superscope object failed: ",
            $session->status_code() . ":" . $session->status_detail());

$session->remove($dhcp_range2)
    or die("Remove DHCP Range object failed: ",
            $session->status_code() . ":" . $session->status_detail());

$session->remove($dhcp_range1)
    or die("Remove DHCP Range object failed: ",
            $session->status_code() . ":" . $session->status_detail());

$session->remove($network2)
    or die("Remove DHCP Network object failed: ",
            $session->status_code() . ":" . $session->status_detail());

$session->remove($network1)
    or die("Remove DHCP Network object failed: ",
            $session->status_code() . ":" . $session->status_detail());

$session->remove($ms_member)
    or die("Remove Grid MSServer object failed: ",
            $session->status_code() . ":" . $session->status_detail());

print "All objects removed from the Infoblox appliance successfully\n";

####PROGRAM ENDS####
```

AUTHOR

Infoblox Inc. http://www.infoblox.com/

SEE ALSO

Infoblox::Session, Infoblox::Session->get(), Infoblox::Session->modify(),Infoblox::DHCP::Range,Infoblox::DHCP::MSOption

---

COPYRIGHT

Copyright (c) 2017 Infoblox Inc.

Infoblox::DHCP::Network - DHCP Network object

- NAME
- DESCRIPTION
- CONSTRUCTOR
- SESSION METHODS
  - Infoblox::Session->add( )
  - Infoblox::Session->get( )
  - Infoblox::Session->modify( )
  - Infoblox::Session->remove( )
  - Infoblox::Session->search( )
- METHODS
  - authority( )
  - auto_create_reversezone( )
  - bootfile( )
  - bootserver( )
  - comment( )
  - cloud_info( )
  - ddns_domainname( )
  - ddns_generate_hostname( )
  - ddns_server_always_updates( )
  - ddns_ttl( )
  - ddns_update_fixed_addresses( )
  - ddns_use_option81( )
  - deny_bootp( )
  - disable( )
  - discovered_bridge_domain( )
  - discovered_tenant( )
  - discovery_basic_poll_setting( )
  - discovery_exclusion_range( )
  - discovery_member( )
  - discover_now_status( )
  - discovery_blackout_setting( )
  - discovery_engine_type( )
  - email_list( )
  - enable_ddns( )
  - enable_dhcp_thresholds( )
  - enable_email_warnings( )
  - enable_discovery( )
  - enable_ifmap_publishing( )
  - enable_immediate_discovery( )
  - enable_snmp_warnings( )
  - endpoint_sources( )
  - extattrs( )
  - extensible_attributes( )
  - high_water_mark( )
  - high_water_mark_reset( )
  - ignore_id( )
  - ignore_mac_addresses( )
  - ignore_dhcp_option_list_request( )
  - ipam_email_addresses( )
  - ipam_trap_settings( )
  - ipam_threshold_settings( )
  - lease_scavenge_time( )
  - logic_filters( )
  - low_water_mark( )
  - low_water_mark_reset( )
  - members( )
  - mgm_private( )
  - mgm_private_overridable( )
  - ms_ad_user_data( )
  - network( )
  - network_container( )
  - network_view( )
  - nextserver( )
  - next_available_ip( )
  - next_available_network( )
  - options( )
  - override_ddns_domainname( )
  - override_ddns_ttl( )
  - override_enable_ifmap_publishing( )
  - override_blackout_setting( )
  - override_ignore_id( )

- override_ipam_email_addresses( )
- override_ipam_threshold_settings( )
- override_ipam_trap_settings( )
- override_lease_scavenge_time( )
- override_logic_filters( )
- override_mgm_private( )
- override_subscribe_settings( )
- override_update_dns_on_lease_renewal( )
- port_control_blackout_setting
- pxe_lease_time( )
- recycle_leases( )
- resize( )
- restart_if_needed( )
- rir( )
- rir_organization( )
- rir_registration_action( )
- rir_registration_status( )
- last_rir_registration_update_sent( )
- last_rir_registration_update_status( )
- same_port_control_discovery_blackout( )
- send_rir_request( )
- subscribe_settings( )
- delete_reason( )
- template( )
- zone_associations( )
- unmanaged( )
- update_dns_on_lease_renewal( )

- SAMPLE CODE
- AUTHOR
- SEE ALSO
- COPYRIGHT

NAME

Infoblox::DHCP::Network - DHCP Network object

DESCRIPTION

When DHCP services are configured on an appliance, a network must be defined that it serves. After a network is created, you can then create all the subnetworks individually or create a parent network that encompasses all the subnetworks.

CONSTRUCTOR

```
  my $network = Infoblox::DHCP::Network->new(
                network                         => $ipv4_subnet/prefix,             #Required, when
creating a network
                ipv4addr                        => $ipv4addr,                       #Use this parameter
instead of "network", when using a template with 'allow_any_netmask' set to false to create a network.
                authority                       => "true" | "false"| undef,         #Optional / Default
is undefined
                auto_create_reversezone         => "true" | "false"| undef,         #Optional / Default
is undefined
                bootfile                        => $filename | undef,               #Optional / Default
is undefined
                bootserver                      => $ipv4addr | $fqdn | undef,       #Optional / Default
is undefined
                comment                         => $string| undef,                  #Optional / Default
is undefined
                ddns_domainname                 => $string| undef,                  #Optional / Default
is undefined
                ddns_generate_hostname          => "true" | "false" | undef,        #Optional / Default
is undefined
                ddns_server_always_updates      => "true" | "false",                #Optional / Default
is "true"
                ddns_ttl                        => $num,                            #Optional / Default
is "0"
                ddns_use_option81               => "true" | "false" | undef,        #Optional / Default
is undefined
                ddns_update_fixed_addresses     => "true" | "false" | undef,        #Optional / Default
is undefined
                deny_bootp                      => "true" | "false" | undef,        #Optional / Default
is undefined
                disable                         => "true" | "false",                #Optional / Default
is "false"
                email_list                      => [ "admin\@test.com",...] | undef,  #Optional / Default
```

```
  is undefined
                  enable_ddns                     => "true" | "false" | undef,          #Optional / Default
  is undefined
                  enable_dhcp_thresholds          => "true" | "false",                  #Optional / Default
  is "false"
                  enable_email_warnings           => "true" | "false",                  #Optional / Default
  is "false"
                  enable_snmp_warnings            => "true" | "false",                  #Optional / Default
  is "false"
                  extattrs                        => { $string => $extattr, ... },      #Optional / Default
  is undefined
                  extensible_attributes           => { $string => $string | $num, $string => [ $string |
  $num, ... ], ... } #Required if 'rir_organization' attribute is defined
  #Otherwise is optional
                  high_water_mark                 => $number,                           #Optional / Default
  is "95"
                  high_water_mark_reset           => $number,                           #Optional / Default
  is "95"
                  ignore_dhcp_option_list_request => "true" | "false" | undef,          #Optional / Default
  is undefined
                  ignore_id                       => 'NONE' | 'CLIENT' | 'MACADDR',     #Optional / Default
  is 'NONE'
                  ignore_mac_addresses            => [ "mac_addr", ... ] | undef        #Optional / Default
  is undefined
                  lease_scavenge_time             => $number,                           #Optional / Default
  is "-1"
                  low_water_mark                  => $number,                           #Optional / Default
  is "0"
                  low_water_mark_reset            => $number,                           #Optional / Default
  is "0"
                  members                         => [ $Member1,$Member2,...],          #Optional / Default
  is empty
                  mgm_private                     => "true" | "false",                  #Optional / Default
  is "false"
                  network_container               => $ipv4_subnet/prefix,               #Optional / Default
  is "/"
                  network_view                    => $NetworkView,                      #Optional / Default
  is "default" network view
                  nextserver                      => $ipv4addr | $fqdn | undef,         #Optional / Default
  is undefined
                  options                         => [$Option1,$Option2,...],           #Optional / Default
  is empty list
                  override_ddns_domainname        => "true" | "false",                  #Optional / Default
  value is "false"
                  override_ddns_ttl               => "true" | "false",                  #Optional / Default
  value is "false"
                  override_enable_ifmap_publishing     => "true" | "false",             #Optional / Default
  value is "false"
                  override_lease_scavenge_time    => "true" | "false",                  #Optional / Default
  value is "false"
                  override_mgm_private            => "true" | "false",                  #Optional / Default
  value is "false"
                  override_update_dns_on_lease_renewal => "true" | "false",             #Optional / Default
  value is "false"
                  override_ignore_id              => "true" | "false",                  #Optional / Default
  value is "false"
                  pxe_lease_time                  => $num | undef,                      #Optional / Default
  is undefined
                  recycle_leases                  => "true" | "false" | undef,          #Optional / Default
  is undefined
                  rir_organization                => $rir_org,                          #Optional / An
  Infoblox::Grid::RIR::Organization object
                  rir_registration_action         => 'NONE' | 'CREATE' | 'MODIFY' | 'DELETE',  # Optional /
  write-only
                  rir_registration_status         => 'NOT_REGISTERED' | 'REGISTERED',   #Optional / Default
  is 'NOT_REGISTERED'
                  send_rir_request                => 'true' | 'false',                  #Optional / write-
  only
                  delete_reason                   => $string,                           #Optional / write-
  only
                                                                                        #Required only if
  'rir_registration_action' is specified as 'DELETE'
                  template                        => $string | undef,                   #Optional / Default
  is undefined
                  update_dns_on_lease_renewal     => "true" | "false",                  #Optional / Default
  value is "false"
                  zone_associations               => [$zone1, $zone2, ... ] | undef,    #Optional / Default
  is an empty list
                  discovery_blackout_setting      => $blackout,                         #Optional
                  port_control_blackout_setting   => $blackout,                         #Optional
                  override_blackout_setting       => "true" | "false",                  #Optional / Default
  value is "false"
                  same_port_control_discovery_blackout => "true" | "false",             #Optional / Default
```

```
value is "false"
                ipam_trap_settings              => $ipam_trap,                          #Optional / An
Infoblox::Grid::SNMP::IPAMTrap object
                override_ipam_trap_settings     => "true" | "false",                    #Optional / Default
value is "false"
                ipam_threshold_settings         => $ipam_threshold,                     #Optional / An
Infoblox::Grid::SNMP::IPAMThreshold object
                override_ipam_threshold_settings => "true" | "false",                   #Optional / Default
value is "false"
                ipam_email_addresses            => [$email1, $email2, ...],             #Optional / Default
is an empty list
                override_ipam_email_addresses   => "true" | "false",                    #Optional / Default
value is "false"
                logic_filters                => [ $filter_name | $filter_obj, ...],     #Optional / Default
is undefined
                override_logic_filters      => "true' | 'false",                        #Optional / Default
value is "false"
                subscribe_settings          => $subscribe_settings,                     #Optional / Default
is undefined
                override_subscribe_settings => 'true' | 'false',                        #Optional / Default
is 'false'
                discovery_engine_type       => 'NETMRI' | 'NETWORK_INSIGHT' | 'VDISCOVERY' | 'UNKNOWN' |
'NONE', #Optional / Default is 'NONE'
                discovered_bridge_domain    => $string,                                 #Optional / Default
is undefined
                discovered_tenant           => $string,                                 #Optional / Default
is undefined
              );
```

You cannot set both extattrs and extensible_attributes attributes at the same time.

SESSION METHODS

This section describes all the methods in an Infoblox::Session module that can apply to a DHCP Network object.

Infoblox::Session->add( )

Use this method to add a network object to the Infoblox appliance. See Infoblox::Session->add() for parameters and return values. Notice the syntax difference between simple adding a network and adding a network using a template.

Example

```
#Adding a network.
#Construct an object
my $network = Infoblox::DHCP::Network->new(
        network => "20.0.0.0/8",
);

# Submit for adding a network
my $response = $session->add( $network );

#Adding a network using template
#Construct an object
my $network = Infoblox::DHCP::Network->new(
        ipv4addr => '20.0.0.0',
        template => "mytemplate",
);

# Submit for adding a network
my $response = $session->add( $network );
```

Infoblox::Session->get( )

Use this method to retrieve all the matching network objects from the Infoblox appliance. See Infoblox::Session->get() for parameters and return values.

'discover_now_status' will not be returned by default, use 'return_methods' to request it if needed.

Key References

```
  Apply the following attributes to get a specific DHCP Network object:

    network                 - Optional. The IP address and netmask of a network in CIDR format.
    network_container       - Optional. The network container, in CIDR format, to which this network
belongs.
    network_view            - Optional. The network view in which the network is located. By default,
all network views are searched (except when contain_address is specified -- see below).
    extattrs                - Optional. A hash reference containing extensible attributes.
    extensible_attributes   - Optional. A hash reference containing extensible attributes.
    comment                 - Optional. A comment in string format.
    contains_address        - Optional. An IPv4 address in string format. If specified, all other
```

```
search attributes are ignored, except for network_view. When a value for this attribute is provided,
network_view defaults to the system-defined default network view, unless specified. The returned
network is the smallest network enclosing this IPv4 address.
  member                  - Optional. The member or Microsoft server serving DHCP for the network.
The valid value is an Infoblox::DHCP::Member or Infoblox::DHCP::MSServer object.
  discovered_bridge_domain - Optional. A bridge domain name in string format.
  discovered_tenant        - Optional. A tenant name in string format.
```

Examples

```
my  @result_array = $session->get(
     object         => "Infoblox::DHCP::Network" ,
     network        => "20.0.0.0/8",
     network_view => "default"
);

my  @result_array = $session->get(
     object                => "Infoblox::DHCP::Network" ,
     extensible_attributes => { 'Site' => 'Santa Clara' }
);

my  @result_array = $session->get(
     object  => "Infoblox::DHCP::Network" ,
     comment => "my-comment",
);

my  @result_array = $session->get(
     object  => "Infoblox::DHCP::Network" ,
     network_container => "10.0.0.0/16",
);

# Returns the smallest network enclosing 10.0.0.1 in the
# system-defined default network view.
my  @result_array = $session->get(
     object  => "Infoblox::DHCP::Network" ,
     contains_address => "10.0.0.1",
);

# Returns the smallest network enclosing 10.0.0.1 in the
# specified network view ($network_view_object).
my  @result_array = $session->get(
     object  => "Infoblox::DHCP::Network" ,
     contains_address => "10.0.0.1",
     network_view => $network_view_object,
);

# get a DHCP Network by member
my $member = Infoblox::DHCP::Member->new(
     name => "ib-10-0-0-1.infoblox.com",
     ipv4addr => "10.0.0.1",
);

my  @result_array = $session->get(
     object => "Infoblox::DHCP::Network",
     member => $member,
);

my @result_array = $session->get(
     object  => "Infoblox::DHCP::Network" ,
     discovered_bridge_domain => "domain1",
);

my @result_array = $session->get(
     object  => "Infoblox::DHCP::Network" ,
     discovered_tenant => "tenant1",
);
```

Infoblox::Session->modify( )

Use this method to modify a network object in the Infoblox appliance. See Infoblox::Session->modify() for parameters and return values.

Example

```
#Use method to modify enable_ddns method.
$object->enable_ddns("true");
# Submit modification
my $response = $session->modify( $object );
```

Infoblox::Session->remove( )

Use this method to remove an object from the Infoblox appliance. See Infoblox::Session->remove() for parameters and return values.

To remove a specific object, first use `get()` or `search()` to retrieve the specific object, and then submit this object for removal.

Key References

```
remove_options - Optional. A hash reference containing the following attributes:

+-------------------------------------------------------------------------------------------+
| reason             - Optional. The reason for removing the network in string format with  |
|                      a maximum of 256 bytes.                                              |
| send_rir_request   - Optional. Boolean flag that indicates whether the appliance should  |
|                      send an API or email request for Regional Internet Registry (RIR).  |
|                      Used when 'rir_organization' attribute has been defined.             |
+-------------------------------------------------------------------------------------------+
```

Example

```
 # Get the object under the same name
my  @result_array = $session->get(

       object        => "Infoblox::DHCP::Network" ,
       network       => "20.0.0.0/8",
       network_view  => "default"
         );

# find the desired object from the retrieved list.
my $desired_network = $result_array[0];

# Submit for removal
my $response = $session->remove(
    $desired_network,
    remove_options => {'reason' => 'Moved to B3', send_rir_request => 'true'} );
```

Infoblox::Session->search( )

Use this method to search for a DHCP Network object in the Infoblox appliance. See Infoblox::Session->search() for parameters and return values.

'discover_now_status' will not be returned by default, use 'return_methods' to request it if needed.

Key References

```
 Apply the following attributes to search for a DHCP Network object:

  network                  - Optional. The IP address and netmask of a network in CIDR format.
  network_container        - Optional. The network container, in CIDR format, to which this network
belongs.
  network_view             - Optional. The network view in which the network is located. By default,
all network views are searched.
  extattrs                 - Optional. A hash reference containing extensible attributes.
  extensible_attributes    - Optional. A hash reference containing extensible attributes.
  comment                  - Optional. A comment in string format (regular expression).
  contains_address         - Optional. An IPv4 address in string format. If specified, all other
search attributes are ignored, except for network view. When a value for this attribute is provided,
network view defaults to the system-defined default network view, unless specified. The returned
network is the smallest network enclosing this IPv4 address.
  member                   - Optional. The member or Microsoft server serving DHCP for the network.
The valid value is an Infoblox::DHCP::Member or Infoblox::DHCP::MSServer object.
  discovered_bridge_domain - Optional. A bridge domain name in string format (regular expression).
  discovered_tenant        - Optional. A tenant name in string format (regular expression).
```

For more information about searching extensible attributes, see Infoblox::Grid::ExtensibleAttributeDef/Searching Extensible Attributes.

Examples

```
# search for all DHCP Network objects
    my @retrieved_objs = $session->search(
      object       => "Infoblox::DHCP::Network",
      network      => '20\.0\.0\..*',
      network_view => "default"
    );

# search network by comment
    my @retrieved_objs = $session->search(
    object    => "Infoblox::DHCP::Network",
    comment   => "my-comment",
     );

# search network by network container
```

```
 my @result_array = $session->search(
      object  => "Infoblox::DHCP::Network" ,
      network_container => "10.0.0.0/16",
 );

# search for all DHCP Network objects defining "Santa Clara" for "Site" extensible attribute
    my @retrieved_objs = $session->search(
      object               => "Infoblox::DHCP::Network",
      extensible_attributes => { 'Site' => 'Santa Clara' }
    );

# Returns the smallest network enclosing 10.0.0.1 in the
# system-defined default network view.
    my @result_array = $session->search(
       object  => "Infoblox::DHCP::Network" ,
       contains_address => "10.0.0.1",
    );

# Returns the smallest network enclosing 10.0.0.1 in the
# specified network view ($network_view_object).
    my @result_array = $session->search(
       object  => "Infoblox::DHCP::Network" ,
       contains_address => "10.0.0.1",
       network_view => $network_view_object,
    );

 # search for all DHCP Network objects by member
 my $member = Infoblox::DHCP::Member->new(
      name => "ib-10-0-0-1.infoblox.com",
      ipv4addr => "10.0.0.1",
 );

 my  @result_array = $session->search(
      object => "Infoblox::DHCP::Network",
      member => $member,
 );

 # search network by discovered bridge domain
 my @result_array = $session->search(
      object  => "Infoblox::DHCP::Network" ,
      discovered_bridge_domain => "domain.*",
 );

 # search network by discovered tenant
 my @result_array = $session->search(
      object  => "Infoblox::DHCP::Network" ,
      discovered_tenant => "tenant.*",
 );
```

METHODS

This section describes all the methods that can be used to configure and retrieve the attribute values of a DHCP Network object

authority( )

Use this method to set or retrieve the authority of a DHCP network object. This setting overrides the upper-level settings.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

Specify "true" to set the authority or "false" to deactivate/unset it. The default value for this field is undefined, which indicates that this attribute inherit the upper-level setting.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
 # Get authority
 my $authority = $network->authority()
 # Modify authority
 $network->authority("true");
```

auto_create_reversezone( )

Use this method to set the auto_create_reversezone flag of a new DHCP network object. You can set this flag only when creating a new network object.

Include the specified parameter to set the attribute value.

Parameter

Specify "true" to set the auto_create_reversezone flag or "false" to deactivate/unset it. The default value for this field is "false".

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

Example

```
# Modify auto_create_reversezone
$network_container->auto_create_reversezone("true");
```

bootfile( )

Use this method to set or retrieve a bootfile attribute of a DHCP network object. You can configure the DHCP server to support clients that use boot file name option in their DHCPREQUEST messages. This setting overrides the upper-level settings. The default value is undefined, which indicates that this attribute inherits the upper-level setting.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

This option is overridden independently from Infoblox::DHCP::Network->bootserver() and Infoblox::DHCP::Network->nextserver().

Parameter

The name, in string format, of the file that the client must download. The default value is undefined.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
# Get bootfile
my $bootfile = $network->bootfile();
# Modify bootfile
$network->bootfile("bootfile1");
#Un-override bootfile
$network->bootfile(undef);
```

bootserver( )

Use this method to set or retrieve a bootserver attribute of a DHCP network object. You can specify the name and/or IP address of the boot server that host needs to boot. This setting overrides the upper-level settings. The default value is undefined, which indicates that this attribute inherits the upper-level setting.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

This option is overridden independently from Infoblox::DHCP::Network->bootfile() and Infoblox::DHCP::Network->nextserver().

Parameter

The boot server IP address or name in FQDN (Fully Qualified Domain Name) format. The FQDN consists of the host name followed by the domain name (example: abc.com). A boot server name can have a maximum of 256 bytes. The default value is undefined.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
# Get bootserver
my $bootserver = $network->bootserver();
# Modify bootserver
$network->bootserver("abc.domain.com");
#Un-override bootserver
$network->bootserver(undef);
```

comment( )

Use this method to set or retrieve a descriptive comment of a DHCP network object.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

Enter a descriptive comment for the network in string format with a maximum of 256 bytes.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
# Get comment
my $comment = $network->comment();
# Modify comment
$network->comment("add a network");
```

cloud_info( )

Use this method to set or retrieve cloud API related information for the Infoblox::DHCP::Network object.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

The valid value is a Infoblox::Grid::CloudAPI::Info object.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
# Get cloud_info
my $cloud_info = $network_view->cloud_info();
# Modify cloud_info
$network_view->cloud_info($cloud_info_object);
```

ddns_domainname( )

Use this method to set or retrieve the ddns_domainname value of a DHCP Network object.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Setting this method to a defined value implicitly sets the override_ddns_domainname method to "true". Setting the parameter to undefined causes the appliance to use the grid default and automatically resets the override_ddns_domainname attribute to "false".

Note that when ddns_domainname is set to a defined value and override_dddns_domainname is set to "false", the last operation takes precedence. Thus the sequence $object->ddns_domainname("testdomain"); $object->override_ddns_domainname("false"); will set override_ddns_domainname to "false", and the sequence $object->override_ddns_domainname("false"); $object->ddns_domainname("testdomain"); will result in override_ddns_domainname="true".

Parameter

The dynamic DNS domain name in string format.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get ddns_domainname
my $ddns_domainname = $network->ddns_domainname();
#Modify ddns_domainname
$network->ddns_domainname("test_domain.com");
```

ddns_generate_hostname( )

Use this method to set or retrieve the ddns_generate_hostname flag of a DHCP network object. Enable this method to allow only the DHCP server for generating hostname. This setting overrides the upper-level settings.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

Specify "true" to generate hostname or "false" to deactivate/unset it.The default value for this field is undefined, which indicates that this attribute inherit the member level setting.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
# Get ddns_generate_hostname
my $ddns_generate_hostname = $network->ddns_generate_hostname()
# Modify ddns_generate_hostname
$network->ddns_generate_hostname("true");
#Un-override
$network->ddns_generate_hostname(undef);
```

ddns_server_always_updates( )

Use this method to set or retrieve the ddns_server_always_updates flag of a DHCP network object. Enable this method to allow only the DHCP server to update DNS,regardless of the requests from the DHCP clients.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

Specify "true" to set the flag that enables the DHCP server to always updates DNS or "false" to deactivate/unset it.The default value for this field is "true".

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
# Get ddns_server_always_updates
my $ddns_server_always_updates = $network->ddns_server_always_updates()
# Modify ddns_server_always_updates
$network->ddns_server_always_updates("true");
```

ddns_ttl( )

Use this method to set or retrieve the DNS update Time to Live (TTL) value of a DHCP network object.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Setting this method to a defined value implicitly sets the override_ddns_ttl method to "true". Setting the parameter to undefined causes the appliance to use the grid default and automatically resets the override_ddns_ttl attribute to "false".

Note that when ddns_ttl is set to a defined value and override_dddns_ttl is set to "false", the last operation takes precedence. Thus the sequence $object->ddns_ttl(12); $object->override_ddns_ttl("false"); will set override_ddns_ttl to "false", and the sequence $object->override_ddns_ttl("false"); $object->ddns_ttl(12); will result in override_ddns_ttl="true".

Parameter

A 32-bit integer (range from 0 to 4294967295) that represents the duration, in seconds, that the update is cached. Zero indicates that the update is not cached. The default value is 0.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get DNS Update ttl
my $dns_ttl = $network->ddns_ttl();
#Modify DNS Update ttl
```

```
$network->ddns_ttl(1200);
```

ddns_update_fixed_addresses( )

Use this method to set or retrieve the ddns_update_fixed_addresses flag of a DHCP network object. By default, the DHCP server does not update DNS when it allocates a fixed address to a client. You can configure the DHCP server to update the A and PTR records of a client with a fixed address. When this feature is enabled and the DHCP server adds A and PTR records for a fixed address, the DHCP server never discards the records. This setting overrides the upper-level settings.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

Specify "true" to enable the appliance to update DNS when it assigns a fixed address to a client or "false" to disable it. The default value for this field is undefined, which indicates that this attribute inherit the member level setting.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
# Get ddns_update_fixed_addresses
my $ddns_update_fixed_addresses= $network->ddns_update_fixed_addresses()
# Modify ddns_update_fixed_addresses
$network->ddns_update_fixed_addresses("true");
#Un-override
$network->ddns_update_fixed_addresses(undef);
```

ddns_use_option81( )

Use this method to set or retrieve the ddns_use_option81 flag of a DHCP network object. Enable this method to define the settings for option 81 at the network level. This setting overrides the member level settings.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

Specify "true" to Configure Support for Option 81 or "false" to deactivate/unset it. The default value for this field is undefined, which indicates that this attribute inherit the member level setting.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
# Get ddns_use_option81
my $ddns_use_option81 = $network->ddns_use_option81()
# Modify ddns_use_option81
$network->ddns_use_option81("true");
# un-override ddns_use_option81
$network->ddns_use_option81(undef);
```

deny_bootp( )

Use this method to set or retrieve the deny_bootp flag of a DHCP Network object.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

Specify "true" to set the deny_bootp flag or "false" to deactivate/unset it. The default value is undefined, which indicates that this attribute inherit the upper-level setting.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get deny_bootp
my $deny_bootp   $network  deny bootp
```

```
                =          ->          ();
#Modify deny_bootp
$network->deny_bootp("true");
```

disable( )

Use this method to set or retrieve the disable flag of a DHCP network object.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

Specify "true" to set the disable flag or "false" to deactivate/unset it. The default value for this field is "false".

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
# Get disable
my $disable = $network->disable()
# Modify disable
$network->disable("true");
```

discovered_bridge_domain( )

Use this method to set or retrieve the discovered bridge domain name of a DHCP Network object.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

The desired bridge domain name in a string format.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get bridge domain
my $bridge_domain = $network->discovered_bridge_domain();
#Modify bridge domain
$network->discovered_bridge_domain("Domain");
```

discovered_tenant( )

Use this method to set or retrieve the discovered tenant name of a DHCP Network object.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

The desired tenant name in a string format.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get tenant
my $tenant = $network->discovered_tenant();
#Modify tenant
$network->discovered_tenant("Tenant");
```

discovery_basic_poll_setting( )

Use this method to set or retrieve the discovery_basic_poll_setting value of a DHCP Network object.

Include the specified parameter to set the attribute value.

Parameter

The valid value is object of the Infoblox::Grid::Discovery::BasicPollSettings manpage.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get discovery_basic_poll_setting
my $discovery_basic_poll_setting = $network->discovery_basic_poll_setting();
#Modify discovery_basic_poll_setting
$network->discovery_basic_poll_setting($poll_setting);
```

discovery_exclusion_range( )

Use this method to set or retrieve the discovery_exclusion_range value of a DHCP Network object.

Parameter

The valid value is an array of the Infoblox::DHCP::ExclusionRange manpage objects.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get discovery_exclusion_range
my $discovery_exclusion_range = $network->discovery_exclusion_range();
#Modify discovery_exclusion_range
$network->discovery_exclusion_range( [ $exclusion1, $exclusion2 ]);
```

discovery_member( )

Use this method to set or retrieve the discovery_member value of a DHCP Network object.

Parameter

The discovery member name in string format.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get discovery_member
my $discovery_member = $network->discovery_member();
#Modify discovery_member
$network->discovery_member("test.localdomain");
```

discover_now_status( )

Use this method to retrieve the current discovery status. This attribute is read-only.

'discover_now_status' will not be returned by default, use 'return_methods' to request it if needed.

Parameter

None.

Returns

The method returns the attribute value that can be 'COMPLETE', 'FAILED', 'NONE', 'PENDING' or 'RUNNING'.

Example

```
#Get discover_now_status
my $discover_now_status = $network->discover_now_status();
```

discovery_blackout_setting( )

Use this method to set or retrieve the discovery blackout setting.

Setting this method to a defined value implicitly sets the override_blackout_setting method to "true". If port_control_blackout_setting is undefined, setting the parameter to undefined causes the appliance to use the grid default and automatically resets the override_blackout_setting attribute to "false".

Note that when discovery_blackout_setting is set to a defined value and override_blackout_setting is set to "false", the last operation takes precedence. Thus the sequence $network->discovery_blackout_setting($blackout); $network->override_blackout_setting("false"); will set override_blackout_setting to "false", and the sequence $network->override_blackout_setting("false"); $network->discovery_blackout_setting($blackout); will result in override_blackout_setting="true".

Parameter

Valid value is an Infoblox::Grid::Discovery::Properties::BlackoutSetting object.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get discovery_blackout_setting
my $discovery_blackout_setting = $network->discovery_blackout_setting();
#Modify discovery_blackout_setting
$network->discovery_blackout_setting($blackout);
```

discovery_engine_type( )

Use this method to retrieve the DHCP Network discovery engine type. This attribute is read-only.

Parameter

None.

Returns

The method returns the attribute value that can be 'NETMRI', 'NETWORK_INSIGHT', 'VDISCOVERY', 'UNKNOWN' or 'NONE'.

Example

```
#Get discovery_engine_type
my $discovery_engine_type = $network->discovery_engine_type();
```

email_list( )

Use this method to set or retrieve the e-mail lists to which the appliance sends threshold e-mail messages.

If you enter e-mail addresses in the email_list, then the appliance sends DHCP threshold alarms to the e-mail addresses.

Parameter

The valid value is an array reference that contains string objects.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get email_list
my @email_list = $network->email_list();
#Modify email_list
$network->email_list( [ "admin\@test.com" ] );
```

enable_ddns( )

Use this method to set or retrieve the dynamic DNS updates flag of a DHCP network object. The DHCP server can send DDNS updates to DNS servers in the same grid and to external DNS servers. This setting overrides the member level settings.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

Specify "true" to set the dynamic DNS updates flag or "false" to deactivate/unset it. The default value for this field is undefined, which indicates that this attribute inherit the member level setting.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
# Get enable_ddns
my $enable_ddns = $network->enable_ddns()
# Modify enable_ddns
$network->enable_ddns("true");
# Un-override enable_ddns
$network->enable_ddns(undef);
```

enable_dhcp_thresholds( )

Use this method to set or retrieve DHCP thresholds at the network level. These thresholds are the high and low watermarks. If a DHCP range usage goes above or below these percentage values, the Infoblox appliance generates a syslog message and sends an SNMP notification.

Enable this flag to set DHCP thresholds values.

Parameter

Specify "true" to set the enable_dhcp_thresholds flag or "false" to deactivate/unset it. The default value is undefined.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get enable_dhcp_thresholds
my $enable_dhcp_thresholds = $network->enable_dhcp_thresholds();
#Modify enable_dhcp_thresholds
$network->enable_dhcp_thresholds("false");
```

enable_email_warnings( )

Use this method to send DHCP threshold warnings through e-mail.

Enable this flag if you want the appliance to send DHCP threshold warnings to the "email_list" provided when DHCP threshold is enabled.

Parameter

Specify "true" to set the enable_email_warnings flag or "false" to deactivate/unset it. The default value is undefined.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get enable_email_warnings
my $enable_email_warnings = $network->enable_email_warnings();
#Modify enable_email_warnings
$network->enable_email_warnings("false");
```

enable_discovery( )

Use this method to set or retrieve the enable_discovery value of a DHCP Network object. This setting overrides the member level settings.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Setting this method to a defined value implicitly sets the override_enable_discovery method to "true". Setting the parameter to undefined causes the appliance to use the default from the upper level and automatically resets the override_enable_discovery attribute to "false".

Enable this flag if you want to set discovery for this object.

Parameter

Specify "true" to set the enable_discovery flag or "false" to deactivate/unset it. The default value is undefined.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get enable_discovery
my $enable_discovery = $network->enable_discovery();
#Modify enable_discovery
$network->enable_discovery("false");
```

enable_ifmap_publishing( )

Use this method to set or retrieve ifmap publishing the option of a DHCP Network object. This setting overrides the member level settings.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

Specify "true" to set ifmap publishing option or "false" to deactivate/unset it. The default value 'false', which indicates that this attribute inherits the member level setting.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
# Get enable ifmap publishing
my $enable_ifmap_publishing = $network->enable_ifmap_publishing()
# Modify enable_ifmap_publishing
$network->enable_ifmap_publishing("true");
# Un-override enable_ifmap_publishing
$network->enable_ifmap_publishing(undef);
```

enable_immediate_discovery( )

Use this method to set or retrieve enable_immediate_discovery value of a DHCP Network object.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

Specify "true" to set the enable_immediate_discovery flag or "false" to deactivate/unset it. The default value is undefined.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get enable_immediate_discovery
my $enable_immediate_discovery = $network->enable_immediate_discovery();
#Modify enable_immediate_discovery
$network->enable_immediate_discovery("false");
```

enable_snmp_warnings( )

Use this method to send DHCP threshold warnings through SNMP.

If the flag is enabled, then the appliance sends DHCP threshold warnings through SNMP messages when DHCP threshold is enabled.

Parameter

Specify "true" to set the enable_snmp_warnings flag or "false" to deactivate/unset it. The default value is undefined.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get enable_snmp_warnings
my $enable_snmp_warnings = $network->enable_snmp_warnings();
#Modify enable_snmp_warnings
$network->enable_snmp_warnings("false");
```

endpoint_sources( )

Use this method to retrieve the list of endpoints that provide object data. This is a read-only attribute.

Omit the parameter to retrieve the attribute value.

Parameter

None

Returns

The valid return value is an array of Infoblox::CiscoISE::Endpoint objects.

Example

```
#Get endpoint_sources value
my $endpoint_sources = $object->endpoint_sources();
```

extattrs( )

Use this method to set or retrieve the extensible attributes associated with a DHCP network object.

Parameter

Valid value is a hash reference containing the names of extensible attributes and their associated values (Infoblox::Grid::Extattr objects).

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get extattrs
my $ref_extattrs = $network->extattrs();
#Modify extattrs
$network->extattrs({ 'Site' => $extattr1, 'Administrator' => $extattr2 });
```

extensible_attributes( )

Use this method to set or retrieve the extensible attributes associated with a DHCP network object

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

For valid values for extensible attributes, see Infoblox::Grid::ExtensibleAttributeDef/Extensible Attribute Values.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get extensible attributes
my $ref_extensible_attributes = $network->extensible_attributes();
#Modify extensible attributes
$network->extensible_attributes({ 'Site' => 'Santa Clara', 'Administrator' => [ 'Peter', 'Tom' ] });
```

high_water_mark( )

Use this method to set the percentage value for DHCP range usage above which the Infoblox appliance generates a syslog message and sends a warning.

When the percent of allocated addresses in a DHCP range exceeds the high watermark, the appliance generates a syslog message and sends a warning. Default value for this option is 95.

Parameter

The valid value is in string format.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get high_water_mark
my $high_water_mark = $network->high_water_mark();
#Modify high_water_mark
$network->high_water_mark("80");
```

high_water_mark_reset( )

Use this method to set or retrieve the high_water_mark_reset value. If the percentage of allocated addresses drops below this value, a corresponding SNMP trap is reset.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

A number that specifies the percentage of allocated addresses. The range is from 1 to 100. The high watermark reset value must be lower than the high watermark value. The default value is 95.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get high_water_mark_reset
my $high_water_mark_reset = $network->high_water_mark_reset();
#Modify high_water_mark_reset
$network->high_water_mark_reset(70);
```

ignore_id( )

Use this method to set or retrieve the ignore_id type of a DHCP network object. Indicates whether the appliance will ignore DHCP client IDs or MAC addresses. This setting overrides the upper-level settings.

Parameter

Valid values are 'NONE', 'CLIENT', or 'MACADDR'. The default is 'NONE'.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
# Get attribute value
$value = $network->ignore_id();
# Modify attribute value
$network->ignore_id('CLIENT');
#Un-override ignore_id
$network->ignore_id(undef);
```

ignore_mac_addresses( )

Use this method to set or retrieve the ignore_mac_addresses list of a DHCP network object. A list of MAC addresses the appliance will ignore. This setting overrides the upper-level settings.

If you enter MAC addresses in the ignore_mac_addresses, then the appliance will ignore them. If you enter empty array, then the appliance will ignore any MAC address.

Parameter

The valid value is an array reference that contains string objects.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
# Get attribute value
$value = $network->ignore_mac_addresses();
# Modify attribute value
$network->ignore_mac_addresses(["11::11::11::11::11::11"]);
```

ignore_dhcp_option_list_request( )

Use this method to set or retrieve the ignore_dhcp_option_list_request flag of a DHCP Network object. If this flag is set to false, the appliance returns all the defined DHCP options to the client. This setting overrides the upper-level settings.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

Specify "true" to set the ignore_dhcp_option_list_request flag or "false" to deactivate/unset it. The default value is undefined, which indicates that this attribute inherits the upper-level setting.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
# Get ignore_dhcp_option_list_request
my $ignore_dhcp_option_list_request = $network->ignore_dhcp_option_list_request();
# Modify ignore_dhcp_option_list_request
$network->ignore_dhcp_option_list_request("true");
#Do not override ignore_dhcp_option_list_request
$network->ignore_dhcp_option_list_request(undef);
```

ipam_email_addresses( )

Use this method to set or retrieve a list of IPAM Utilization email addresses.

Setting this method to a defined value implicitly sets the override_ipam_email_addresses method to "true". Setting the parameter to undefined causes the appliance to use the default from the upper level and automatically resets the override_ipam_email_addresses attribute to "false".

Note that when ipam_email_addresses is set to a defined value and override_ipam_email_addresses is set to "false", the last operation takes precedence. Thus the sequence $object->ipam_email_addresses(['admin@infoblox.com']); $object->override_ipam_email_addresses("false"); will set override_ipam_email_addresses to "false", and the sequence $object->override_ipam_email_addresses("false"); $object->ipam_email_addresses(['admin@infoblox.com']); will result in override_ipam_email_addresses="true".

Parameter

Valid value is an array reference that contains email addresses in string format.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get ipam_email_addresses
my $ipam_email_addresses = $network->ipam_email_addresses();
#Modify ipam_email_addresses
$network->ipam_email_addresses($ipam_email_addresses);
```

ipam_trap_settings( )

Use this method to set or retrieve IPAM Utilization SNMP Trap settings.

Setting this method to a defined value implicitly sets the override_ipam_trap_settings method to "true". Setting the parameter to undefined causes the appliance to use the default from the upper level and automatically resets the override_ipam_trap_settings attribute to "false".

Note that when ipam_trap_settings is set to a defined value and override_ipam_trap_settings is set to "false", the last operation takes precedence. Thus the sequence $object->ipam_trap_settings($ipam_trap); $object->override_ipam_trap_settings("false"); will set override_ipam_trap_settings to "false", and the sequence $object->override_ipam_trap_settings("false"); $object->ipam_trap_settings($ipam_trap); will result in override_ipam_trap_settings="true".

Parameter

Valid value is an Infoblox::Grid::SNMP::IPAMTrap object.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get ipam_trap_settings
my $ipam_trap_settings = $network->ipam_trap_settings();
#Modify ipam_trap_settings
$network->ipam_trap_settings($ipam_trap_settings);
```

ipam_threshold_settings( )

Use this method to set or retrieve IPAM Utilization SNMP Threshold settings.

Setting this method to a defined value implicitly sets the override_ipam_threshold_settings method to "true". Setting the parameter to undefined causes the appliance to use the default from the upper level and automatically resets the override_ipam_threshold_settings attribute to "false".

Note that when ipam_threshold_settings is set to a defined value and override_ipam_threshold_settings is set to "false", the last operation takes precedence. Thus the sequence $object->ipam_threshold_settings($ipam_threshold_settings); $object->override_ipam_trap_settings("false"); will set override_ipam_threshold_settings to "false", and the sequence $object->override_ipam_threshold_settings("false"); $object->ipam_threshold_settings($ipam_threshold_settings); will result in override_ipam_threshold_settings="true".

Parameter

Valid value is an Infoblox::Grid::SNMP::IPAMThreashold object.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get ipam_threshold_settings
my $ipam_threshold_settings = $network->ipam_threshold_settings();
#Modify ipam_threshold_settings
$network->ipam_threshold_settings($ipam_threshold_settings);
```

lease_scavenge_time( )

Use this method to set or retrieve the lease_scavenge_time value of a DHCP Network object.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Setting this method to a defined value implicitly sets the override_lease_scavenge_time method to "true". Setting the parameter to undefined causes the appliance to use the default from the upper level and automatically resets the override_lease_scavenge_time attribute to "false".

Note that when lease_scavenge_time is set to a defined value and override_lease_scavenge_time is set to "false", the last operation takes precedence. Thus the sequence $object->lease_scavenge_time(604800); $object->override_lease_scavenge_time("false"); will set override_lease_scavenge_time to "false", and the sequence $object->override_lease_scavenge_time("false"); $object->lease_scavenge_time(604800); will result in override_lease_scavenge_time="true".

Parameter

An integer that specifies the period of time, in seconds, that free and backup leases remain in the database before they are automatically deleted. To disable lease scavenging, set the parameter to -1. The minimum positive value must be greater than 86400 seconds (1 day). The default value of this parameter is -1, which means that lease scavenging is disabled.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get lease_scavenge_time
my $lease_scavenge = $network->lease_scavenge_time();
#Modify lease_scavenge_time
$network->lease_scavenge_time(30 * 24 * 60 * 60);
```

logic_filters( )

Use this method to set or retrieve all the logic (ordered) filters associated with the DHCP Network.

Filters are used to screen address assignments in a DHCP Network if the addresses are to be assigned to specific hosts.

Setting this method to a defined value implicitly sets the override_logic_filters method to "true". Setting the parameter to undefined causes the appliance to use the grid default and automatically resets the override_logic_filters attribute to "false".

Note that when logic_filters is set to a defined value and override_dlogic_filters is set to "false", the last operation takes precedence. Thus the sequence $object->logic_filters($filters); $object->override_logic_filters("false"); will set override_logic_filters to "false", and the sequence $object->override_logic_filters("false"); $object->logic_filters($filters); will result in override_logic_filters="true".

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

The valid value is an array reference that contains either the Infoblox::DHCP::Filter::MAC, Infoblox::DHCP::Filter::NAC, Infoblox::DHCP::Filter::Option objects or the names of the corresponding filters. The filters in the list must satisfy the following conditions:

Filter name must be globally unique.
Filters without expressions must be at the end of the list.
The filters should not be disabled.

The default value of the parameter is undefined.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get logic_filters
my $ref_logic_filters = $network->logic_filters();
#Modify logic_filters
$network->logic_filters(["filter_1","filter_2",$exl1]); #$exl1 is an Infoblox::DHCP::Filter::MAC,
Infoblox::DHCP::Filter::NAC or Infoblox::DHCP::Filter::Option object.
```

low_water_mark( )

Use this method to set the percentage value for DHCP range usage below which the Infoblox appliance generates a syslog message and sends a warning.

When the percent of allocated addresses in a DHCP range goes below the low watermark, the appliance generates a syslog message and sends a warning. Default value for this option is 0.

Parameter

The valid value is in string format.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get low_water_mark
my $low_water_mark = $network->low_water_mark();
#Modify low_water_mark
$network->low_water_mark("10");
```

low_water_mark_reset( )

Use this method to set or retrieve the low_water_mark_reset value. If the percentage of allocated addresses exceeds this value, a

corresponding SNMP trap is reset.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

A number that specifies the percentage of allocated addresses. The range is from 1 to 100. The low watermark reset value must be higher than the low watermark value. The default low watermark reset number is 0.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get low_water_mark_reset
my $low_water_mark_reset = $network->low_water_mark_reset();
#Modify low_water_mark_reset
$network->low_water_mark_reset(10);
```

members( )

Use this method to set or retrieve the member or Microsoft (r) server that serves DHCP for a network object.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

The valid value is an array reference that contains Infoblox::DHCP::Member or Infoblox::DHCP::MSServer objects.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
my $member1 = Infoblox::DHCP::Member->new(
    name     => "infoblox.localdomain",
    ipv4addr => "192.168.1.2",
);

#Create the Network object with this member
my $network = Infoblox::DHCP::Network->new(
    network => "10.0.0.0/8",
    members => [$member1],
);
```

mgm_private( )

Use this method to set or retrieve the flag that controls whether the object should be synced to the MGM or not.

Setting this method to a defined value implicitly sets the override_blackout_setting method to "true". If discovery_blackout_setting is undefined, setting the parameter to undefined causes the appliance to use the grid default and automatically resets the override_blackout_setting attribute to "false".

Note that when mgm_private is set to a defined value and override_mgm_private is set to "false", the last operation takes precedence. Thus the sequence $network->mgm_private("true"); $network->override_mgm_private("false"); will set override_mgm_private to "false", and the sequence $network->override_mgm_private("false"); $network->mgm_private("true"); will result in override_mgm_private="true".

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

Specify 'true' to enable synchronization of the managed Grid with the MGM or 'false' to forbid it. The default value is 'false'.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Note that this member is loaded on on-demand basis, so to retrieve the value the object must have been retrieved from the session first.

Example

```
#Get mgm_private value
my $value = $object->mgm_private();

#Modify mgm_private value
$object->mgm_private('true');
```

mgm_private_overridable( )

Use this method to retrieve the flag that indicates whether the overriding of 'mgm_private' flag is allowed or not. This is a read-only attribute.

Parameter

None

Returns

The method returns the attribute value.

Note that this member is loaded on on-demand basis, so to retrieve the value the object must have been retrieved from the session first.

Example

```
#Get mgm_private_overridable value
my $value = $object->mgm_private_overridable();
```

ms_ad_user_data( )

Use this method to retrieve Microsoft Active Directory users related information. This is a read-only attribute.

Parameter

None

Returns

The valid return value is an Infoblox::Grid::MSServer::AdUser::Data object.

Example

```
#Get ms_ad_user_data
my $ms_ad_user_data = $network->ms_ad_user_data();
```

network( )

Use this method to retrieve the network attribute of a DHCP network object.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

An IPv4 address is a 32-bit number in dotted decimal notation. It consists of four 8-bit groups of decimal digits separated by decimal points (example: 192.168.1.2).

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get network
my $network = $network->network();
```

network_container( )

Use this method to set or retrieve the network container of a DHCP network object.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

Specify the IP address and netmask of a network in CIDR format.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get network container
my $network_container = $network->network_container();
#Modify network container
$network->network_container("50.0.0.0/8");
```

network_view( )

Use this method to set or retrieve the network view of the DHCP network.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

The default value is the "default" network view, which means the network is in the default network view.

Parameter

Valid value is a defined Infoblox::DHCP::View object. The default network view value is the system-defined default network view object.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get network view
my $nview = $network->network_view();
#Modify network view, with an Infoblox::DHCP::View object
$network->network_view($nview);
```

nextserver( )

Use this method to set or retrieve the nextserver attribute of a DHCP network object. You can specify the name and/or IP address of the next server that the host needs to boot. This setting overrides the upper-level settings. The default value is undefined, which indicates that this attribute inherits the upper-level setting.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

This option is overridden independently from Infoblox::DHCP::Network->bootfile() and Infoblox::DHCP::Network->bootserver().

Parameter

The next server IP address or name in FQDN (Fully Qualified Domain Name) format. The FQDN consists of the host name followed by the domain name (example: abc.com). A next server name can have a maximum of 256 bytes. The default value is undefined.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
# Get nextserver
my $nextserver = $network->nextserver();
# Modify nextserver
$network->nextserver("blue.domain.com");
#Un-override nextserver
$network->nextserver(undef);
```

next_available_ip( )

Use this method to retrieve the next available IP `address(es)` in the network that was retrieved. Note that to use this method, the object MUST be retrieved through a get or search call, or created with an add call.

Parameter

The method accepts the following optional parameters:

`requested_num - The number of IP addresses requested. It must be a number between 1 and 20.`

```
  excluded - A reference to an array of addresses to be excluded from the results.
```

Returns

This method returns the next available IP address(es) in the network. If requested_num is not specified, a string with the next available address will be returned; otherwise, the method will return a reference to an array of addresses.

Within a DHCP Network object, the next available IP address(es) are the first addresses in this network that match all of the following rules:

```
   * Are not part of the passed 'excluded' parameter (if specified).
   * There is no A, PTR, fixed address or host address record matching this IP address
   * There is no bulk host record with this IP address
   * There is no unmanaged record matching this IP address
   * The IP address is not the network or broadcast address of this network
   * The IP address is not in any DHCP range within this network (including any exclusion range)
```

Example

```
# Retrieve a particular network
my @retrieved_objs = $session->get(
                          object   => "Infoblox::DHCP::Network",
                          network  => "10.0.0.0/8"
);

$dhcp_network = $retrieved_objs[0];

# Print the next available IP address in the network
print $dhcp_network->next_available_ip() . "\n";

# This will retrieve the next 10 available addresses and print the first one.
my $addressesref = $dhcp_network->next_available_ip(requested_num => 10);
print @$addressesref[0] . "\n";

# This will retrieve the next 10 available addresses and exclude two.
my $addressesref = $dhcp_network->next_available_ip(requested_num => 10, excluded => ["10.0.0.1",
"10.0.0.2"]);
```

next_available_network( )

Use this method to retrieve the next available network(s) in the network. Note that to use this method, the object MUST be retrieved through a get or search call, or created with an add call.

Parameter

The method accepts the following parameters:

```
  cidr - (required) The CIDR of the requested networks. It must be a value between 1 and 32,
inclusive.
  requested_num - (optional) The number of networks requested. It must be a value between 1 and 20,
inclusive.
  excluded - (optional) A reference to an array of networks to be excluded from the results.
```

Returns

This method returns the next available network(s) in the network. If requested_num is not specified, a string containing the next available network will be returned; otherwise, the method will return a reference to an array of networks.

A network cannot be returned as the next available network if it cannot be inserted; such as when any of the following conditions exist:

- It's parent network has a DHCP configuration assigned and therefore cannot be converted to a network container.

- It overlaps with an existing network or network container, regardless of whether the existing network has a greater or smaller CIDR than the requested CIDR.

- Its network address or broadcast address is used by a fixed address or a host configured for DHCP.

Example

```
# Retrieve a particular network
my @retrieved_objs = $session->get(
                          object   => "Infoblox::DHCP::Network",
                          network  => "10.0.0.0/8"
);

$network = $retrieved_objs[0];

# Print the next available IPv6 network in the network container
print $network->next_available_network(cidr => 12) . "\n";
```

```
# This will retrieve the next 10 available networks and print the first one
my $addressesref = $network->next_available_network(requested_num => 10, cidr => 12);
print @$addressesref[0] . "\n";

# This will retrieve the next 10 available networks and exclude two.
my $addressesref = $network->next_available_network(cidr => 12, requested_num => 10, excluded =>
['10.0.0.0/12']);
```

options( )

Use this method to set or retrieve the options value of a member DHCP server. DHCP options provide network configuration settings and various services available on the network for DHCP.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

The valid value is an array reference that contains Infoblox::DHCP::Option objects.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get options
my $options = $network->options();
#Modify options
my $option1 = Infoblox::DHCP::Option->new(
    name => "lease-time",
    value => 50
);

$network->options([$option1]);
```

override_ddns_domainname( )

The override_ddns_domainname attribute controls whether the ddns_domainname value in the DHCP Network is used, instead of the grid default.

The override_ddns_domainname attribute can be specified explicitly. It is also set implicitly when ddns_domainname is set to a defined value.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

Set the parameter to "true" to override the grid-level setting for ddns_domainname. Set the parameter to "false" to inherit the grid-level setting for ddns_domainname.

The default value of this parameter is "false".

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Getting override_ddns_domainname
my $override_ddns_domainname=$network->override_ddns_domainname( );
#Modifying override_ddns_domainname
$network->override_ddns_domainname("true");
```

override_ddns_ttl( )

The override_ddns_ttl attribute controls whether the ddns_ttl value in the DHCP Network is used, instead of the grid default.

The override_ddns_ttl attribute can be specified explicitly. It is also set implicitly when ddns_ttl is set to a defined value.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

Set the parameter to "true" to override the grid-level setting for ddns_ttl. Set the parameter to "false" to inherit the grid-level

setting for ddns_ttl.

The default value of this parameter is "false".

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Getting override_ddns_ttl
my $override_ddns_ttl=$network->override_ddns_ttl( );
#Modifying override_ddns_ttl
$network->override_ddns_ttl("true");
```

override_enable_ifmap_publishing( )

The override_enable_ifmap_publishing controls whether the enable_ifmap_publishing in the DHCP Network is used, instead of the grid default.

The override_enable_ifmap_publishing can be specified explicitly. It is also set implicitly when enable_ifmap_publishing is set to a defined value.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

Set the parameter to "true" to override the grid-level setting for enable_ifmap_publishing. Set the parameter to "false" to inherit the grid-level setting for enable_ifmap_publishing.

The default value of this parameter is "false".

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Getting override_enable_ifmap_publishing
my $override_enable_ifmap_publishing=$network->override_enable_ifmap_publishing( );
#Modifying override_enable_ifmap_publishing
$network->override_enable_ifmap_publishing("true");
```

override_blackout_setting( )

The override_blackout_setting attribute controls whether the discovery_blackout_setting and port_control_blackout_setting values in the object are used, instead of the grid default.

The override_blackout_setting attribute can be specified explicitly. It is also set implicitly when discovery_blackout_setting or port_control_blackout_setting is set to a defined value.

Parameter

Specify "true" to set the override_blackout_setting flag or "false" to deactivate/unset it. The default value is "false".

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
# Get attribute value
my $value = $network->override_blackout_setting();
# Modify attribute value
$network->override_blackout_setting('true');
```

override_ignore_id( )

The override_ignore_id controls whether the ignore_id in the DHCP network is used, instead of the upper-level default.

The override_ignore_id can be specified explicitly. It is also set implicitly when ignore_id is set to a defined value.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

Set the parameter to "true" to override the upper-level setting for ignore_id and ignore_mac_addresses. Set the parameter to "false" to inherit the upper-level setting for ignore_id and ignore_mac_addresses.

The default value of this parameter is "false".

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get attrubute value
my $value=$network->override_ignore_id();
#Modifying attribute value
$network->override_ignore_id('true');
```

override_ipam_email_addresses( )

The override_ipam_email_addresses controls whether the ipam_email_addresses in the DHCP network is used, instead of the upper-level default.

The override_ipam_email_addresses can be specified explicitly. It is also set implicitly when ipam_email_addresses is set to a defined value.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

Set the parameter to "true" to override the upper-level setting for ipam_email_addresses. Set the parameter to "false" to inherit the upper-level setting for ipam_email_addresses.

The default value of this parameter is "false".

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get attrubute value
my $value=$network->override_ipam_email_addresses();
#Modifying attribute value
$network->override_ipam_email_addresses('true');
```

override_ipam_threshold_settings( )

The override_ipam_threshold_settings controls whether the ipam_threshold_settings in the DHCP network is used, instead of the upper-level default.

The override_ipam_threshold_settings can be specified explicitly. It is also set implicitly when ipam_threshold_settings is set to a defined value.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

Set the parameter to "true" to override the upper-level setting for ipam_threshold_settings. Set the parameter to "false" to inherit the upper-level setting for ipam_threshold_settings.

The default value of this parameter is "false".

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get attrubute value
my $value=$network->override_ipam_threshold_settings();
#Modifying attribute value
```

```
$network->override_ipam_threshold_settings('true');
```

override_ipam_trap_settings( )

The override_ipam_trap_settings controls whether the ipam_trap_settings in the DHCP network is used, instead of the upper-level default.

The override_ipam_trap_settings can be specified explicitly. It is also set implicitly when ipam_trap_settings is set to a defined value.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

Set the parameter to "true" to override the upper-level setting for ipam_trap_settings. Set the parameter to "false" to inherit the upper-level setting for ipam_trap_settings.

The default value of this parameter is "false".

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get attrubute value
my $value=$network->override_ipam_trap_settings();
#Modifying attribute value
$network->override_ipam_trap_settings('true');
```

override_lease_scavenge_time( )

The override_lease_scavenge_time attribute controls whether the lease_scavenge_time value in the DHCP network is used, instead of the upper-level default.

The override_lease_scavenge_time attribute can be specified explicitly. It is also set implicitly when lease_scavenge_time is set to a defined value.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

Set the parameter to "true" to override the upper-level setting for lease_scavenge_time. Set the parameter to "false" to inherit the setting for lease_scavenge_time from the upper level.

The default value of this parameter is "false".

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Getting override_lease_scavenge_time
my $override_lease_scavenge_time=$network->override_lease_scavenge_time( );
#Modifying override_lease_scavenge_time
$network->override_lease_scavenge_time("true");
```

override_logic_filters( )

The override_logic_filters attribute controls whether the logic_filters value of the DHCP Network is used, instead of the Grid default.

The override_logic_filters attribute can be specified explicitly. It is also set implicitly when logic_filters is set to a defined value.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

Set the parameter to "true" to override the Grid-level setting for logic_filters. Set the parameter to "false" to inherit the Grid-level setting for logic_filters.

The default value of this parameter is "false".

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Getting override_logic_filters
my $override_logic_filters=$network->override_logic_filters( );
#Modifying override_logic_filters
$network->override_logic_filters("true");
```

override_mgm_private( )

The override_mgm_private attribute controls whether the mgm_private value in the object is used, instead of the parent-level default.

The override_mgm_private attribute can be specified explicitly. It is also set implicitly when mgm_private is set to a defined value.

Note that setting override_mgm_private to 'false' sets the original 'mgm_private' value to 'false'.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

Set the parameter to 'true' to override the parent-level setting for mgm_private. Set the parameter to 'false' to inherit the parent-level setting for mgm_private.

The default value of this parameter is 'false'.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Note that this member is loaded on on-demand basis, so to retrieve the value the object must have been retrieved from the session first.

Example

```
#Get override_mgm_private value
my $value = $object->override_mgm_private( );

#Modify override_mgm_private value
$object->override_mgm_private('true');
```

override_subscribe_settings( )

The override_subscribe_settings attribute controls whether the subscribe_settings method values of the member are used, instead of the Grid default.

The override_subscribe_settings attribute can be specified explicitly. It is also set implicitly when subscribe_settings is set to a defined value.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

Set the parameter to "true" to override the Grid-level setting for subscribe_settings. Set the parameter to "false" to inherit the Grid-level setting for subscribe_settings.

The default value of this parameter is "false".

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get override_subscribe_settings value
my $override_subscribe_settings = $object->override_subscribe_settings( );
#Modify override_subscribe_settings value
$object->override_subscribe_settings("true");
```

override_update_dns_on_lease_renewal( )

The override_update_dns_on_lease_renewal attribute controls whether the update_dns_on_lease_renewal value in the DHCP Network is used, instead of the grid default.

The override_update_dns_on_lease_renewal attribute can be specified explicitly. It is also set implicitly when

update_dns_on_lease_renewal is set to a defined value.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

Set the parameter to "true" to override the grid-level setting for update_dns_on_lease_renewal. Set the parameter to "false" to inherit the grid-level setting for update_dns_on_lease_renewal.

The default value of this parameter is "false".

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

#Getting override_update_dns_on_lease_renewal my $override_update_dns_on_lease_renewal=$network->override_update_dns_on_lease_renewal( ); #Modifying override_update_dns_on_lease_renewal $network->override_update_dns_on_lease_renewal("true");

port_control_blackout_setting

Use this method to set or retrieve the port control blackout setting.

Setting this method to a defined value implicitly sets the override_blackout_setting method to "true". If discovery_blackout_setting is undefined, setting the parameter to undefined causes the appliance to use the grid default and automatically resets the override_blackout_setting attribute to "false".

Note that when port_control_blackout_setting is set to a defined value and override_blackout_setting is set to "false", the last operation takes precedence. Thus the sequence $network->port_control_blackout_setting($blackout); $network->override_blackout_setting("false"); will set override_blackout_setting to "false", and the sequence $network->override_blackout_setting("false"); $network->port_control_blackout_setting($blackout); will result in override_blackout_setting="true".

Parameter

Valid value is an Infoblox::Grid::Discovery::Properties::BlackoutSetting object.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get port_control_blackout_setting
my $port_control_blackout_setting = $network->port_control_blackout_setting();
#Modify port_control_blackout_setting
$network->port_control_blackout_setting($blackout);
```

pxe_lease_time( )

Use this method to set or retrieve the PXE lease time value of a DHCP Network object. Some hosts use PXE (Preboot Execution Environment) to boot remotely from a server. To better manage your IP resources, set a different lease time for PXE boot requests. You can configure the DHCP server to allocate an IP address with a shorter lease time to hosts that send PXE boot requests, so IP addresses are not leased longer than necessary. This setting overrides the upper-level settings. The default value is undefined, which indicates that this attribute inherits the upper-level setting.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

A 32-bit integer (range from 0 to 4294967295) that represents the duration, in seconds, that the update is cached. Zero indicates that the update is not cached. The default value is undefined.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get PXE Lease Time
my $pxe_lease_time = $network->pxe_lease_time();
#Modify PXE Lease Time
```

```
$network->pxe_lease_time(1100);
#Un-override PXE Lease Time
$network->pxe_lease_time(undef);
```

recycle_leases( )

Use this method to set or retrieve the recycle_leases flag at the network level.

If the flag is enabled, the leases are kept in the recycle bin until one week after expiration. When the flag is disabled, the leases are irrecoverably deleted.

Parameter

Specify "true" to enable the recycle_leases flag or "false" to disable it. The default value is undefined.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get recycle_leases
my $recycle_leases = $network->recycle_leases();
#Modify recycle_leases
$network->recycle_leases("false");
```

resize( )

Use this method to resize a network. Note that to use this method, the object MUST be retrieved through a get or search call, or it must have been created with an add call.

Parameter

prefix - Required. The bitmask prefix length after resizing.

auto_create_reversezone - Optional. Boolean flag indicating whether or not to automatically create the reverse zone.

reason - Optional. The reason for resizing the network in string format with a maximum of 256 bytes. Used when 'rir_organization' attribute has been defined.

send_rir_request - Optional. Boolean flag that indicates whether the appliance should send an API or email request for Regional Internet Registry (RIR). Used when 'rir_organization' attribute has been defined.

Returns

The method returns true when the operation succeeds, and returns false when the operation fails. When an operation fails, the error code and error message are reported in the Session object's `status_code()` and `status_detail()` attributes.

Example

```
#Get Network through the session
my @retrieved_objs = $session->get(
        object =>"Infoblox::DHCP::Network" ,
        network => "20.0.0.0/8"
    );
my $network = $retrieved_objs[0];

# Expand a /24 network to a /16 network, inserting
# a corresponding reverse zone in the process.
my $response = $network->resize(
  prefix        => "16"
  auto_create_reversezone => "true"
) or die("Resize Network failed: ",
     $session->status_code() . ":" . $session->status_detail());
```

restart_if_needed( )

Use this method to set the flag that indicates whether DHCP service should be restarted on a member that is serving DHCP for the changed object referenced in a cloud API request.

Parameter

Specify true to restart the DHCP service.

Returns

None

Example

```
# Modify
$network->restart_if_needed('true');
```

rir( )

Use this method to retrieve the name of the Regional Internet Registry (RIR) that allocated this address space. This is a read-only attribute.

Parameter

None

Returns

The method returns the attribute value that can be 'RIPE' or 'NONE'.

Example

```
#Get rir
my $rir = $network->rir();
```

rir_organization( )

Use this method to set or retrieve the Regional Internet Registry (RIR) organization object.

Parameter

An Infoblox::Grid::RIR::Organization object.

You must enable support for RIR/SWIP at the Grid level before you can set attribute values. See Infoblox::Grid->enable_rir_swip() method.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get rir_organization
my $rir_organization = $network->rir_organization();
#Modify rir_organization
$network->rir_organization($rir_org);
```

rir_registration_action( )

Use this method to set the registration action of Regional Internet Registry (RIR). This is a write-only attribute.

Parameter

Valid values are 'NONE', 'CREATE', 'MODIFY' or 'DELETE'.

Returns

The method returns true when the modification succeeds, and returns false when the operation fails.

Example

```
#Set rir_registration_action
$network->rir_registration_action('CREATE');
```

rir_registration_status( )

Use this method to set or retrieve the registration status of Regional Internet Registry (RIR).

Parameter

The registration status of Regional Internet Registry (RIR), which can be 'REGISTERED' or 'NOT_REGISTERED'.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get rir_registration_status
my $rir_registration_status = $network->rir_registration_status();
#Modify rir_registration_status
$network->rir_registration_status('NOT_REGISTERED');
```

last_rir_registration_update_sent( )

Use this method to retrieve the last registration update timestamp of Regional Internet Registry (RIR). This is a read-only attribute.

Parameter

None

Returns

The method returns the attribute value.

Example

```
#Get last_rir_registration_update_sent
my $last_rir_registration_update_sent = $network->last_rir_registration_update_sent();
```

last_rir_registration_update_status( )

Use this method to retrieve the last registration update status of Regional Internet Registry (RIR). This is a read-only attribute.

Parameter

None

Returns

The method returns the attribute value.

Example

```
#Get last_rir_registration_update_status
my $last_rir_registration_update_status = $network->last_rir_registration_update_status();
```

same_port_control_discovery_blackout( )

Use this method to set or retrieve the flag that enables using the same discovery blackout for port control blackout.

Parameter

Specify 'true' to use same discovery blackout for port control blackout or 'false' to disable it. The default value is 'false'.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
# Get attribute value
my $value = $network->same_port_control_discovery_blackout();
# Modify attribute value
$network->same_port_control_discovery_blackout('true');
```

send_rir_request( )

Use this method to set the flag that indicates whether the appliance should send an API or email request for Regional Internet Registry (RIR). This is a write-only attribute.

Parameter

Specify "true" to set the send_rir_request flag or "false" to unset it.

Returns

The method returns true when the modification succeeds, and returns false when the operation fails.

Example

```
#Set send_rir_request
$network->send_rir_request('true');
```

subscribe_settings( )

Use this method to set or retrieve the Cisco ISE subscribe settings.

Setting this method to a defined value implicitly sets the override_subscribe_settings method to "true". Setting the parameter to undefined causes the appliance to use the Cisco ISE endpoint default and automatically resets the override_subscribe_settings method to "false".

Note that when subscribe_settings is set to a defined value and override_subscribe_settings is set to "false", the last operation takes precedence. Thus the sequence $object->subscribe_settings($value); $object->override_subscribe_settings("false"); will set override_subscribe_settings to "false", and the sequence $object->override_subscribe_settings("false"); $object->subscribe_settings($value); will result in override_subscribe_settings="true".

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

The valid value is an Infoblox::CiscoISE::SubscribeSetting object.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get subscribe_settings value
my $subscribe_settings = $object->subscribe_settings();
#Modify subscribe_settings value
$object->subscribe_settings($settings);
```

delete_reason( )

Use this method to specify the reason for deleting the Regional Internet Registry (RIR) registration request. This is a write-only attribute. This field is required only if 'rir_registration_action' is specified as 'DELETE'.

Parameter

The reason for deleting the RIR registration request in string format with a maximum of 256 bytes.

Returns

The method returns true when the modification succeeds, and returns false when the operation fails.

Example

```
#Set delete_reason
$network->delete_reason('invalid');
```

template( )

Use this method to set or retrieve the network template name of a DHCP Network object.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

The valid name in string format.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get template
my $template = $network->template();
#Modify template
$network->template("Template_Name");
```

zone_associations( )

Use this method to set or retrieve the associated zone information. Use the is_default boolean field of the Infoblox::DNS::Zone object to determine or specify the default zone in zone associations of a DHCP network. If the value is 'true', the associated zone is the default zone. Otherwise, it is not the default zone. The default value is 'false'.

Parameter

The valid value is an array reference that contains Infoblox::DNS::Zone object(s). The default value is an empty list. If the network has a parent, it is possible to use the zone associations of the parent by setting the parameter to undef.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails. If you did not specify a parameter, the method returns the attribute value.

Examples

```
#Getting zone associations
my $zone_associations = $network->zone_associations();
#Modifying zone_associations, setting 'test1.com' as the default zone
my $zone1 = Infoblox::DNS::Zone->new(
   name      => "test1.com",
   is_default => "true",
);
my $zone2 = Infoblox::DNS::Zone->new(
   name      => "test2.com",
   is_default => "false",
);
$network->zone_associations([$zone1, $zone2]);
#Clear zone_assocations
$network->zone_associations([]);
##Remove the override for zone_associations
$network->zone_associations(undef)
```

unmanaged( )

Use this method to set or retrieve the network status, which indicates whether the network is unmanaged or not.

Parameter

Specify true to mark the network as unmanaged or false as managed.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
# Get network status
$unmanaged = $network->unmanaged();
# Modify network status
$network->unmanaged('true');
```

update_dns_on_lease_renewal( )

The update_dns_on_lease_renewal attribute controls whether the DHCP server updates DNS when a DHCP lease is renewed.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Setting this method to a defined value implicitly sets the override_update_dns_on_lease_renewal method to "true". Setting the parameter to undefined causes the appliance to use the grid default and automatically resets the override_update_dns_on_lease_renewal attribute to "false".

Note that when update_dns_on_lease_renewal is set to a defined value and override_update_dns_on_lease_renewal is set to "false", the last operation takes precedence. Thus the sequence $object->update_dns_on_lease_renewal("true"); $object->override_update_dns_on_lease_renewal("false"); will set override_update_dns_on_lease_renewal to "false", and the sequence $object->override_update_dns_on_lease_renewal("false"); $object->update_dns_on_lease_renewal("true"); will result in override_update_dns_on_lease_renewal="true".

Parameter

Specify "true" to set the update_dns_on_lease_renewal flag or "false" to deactivate/unset it.

The default value of this parameter is "false".

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

#Get update_dns_on_lease_renewal my $update_dns_on_lease_renewal=$network->update_dns_on_lease_renewal( ); #Modify update_dns_on_lease_renewal $network->update_dns_on_lease_renewal("true");

SAMPLE CODE

The following sample code demonstrates the different functions that can be applied to an object such as add, search, modify, and remove. This sample also includes error handling for the operations.

#Preparation prior to a DHCP Network insertion

```
#PROGRAM STARTS: Include all the modules that will be used
use strict;
use Infoblox;

#Create a session to the Infoblox appliance

my $session = Infoblox::Session->new(
master   => "192.168.1.2",
username => "admin",
password => "infoblox"
);
unless ($session) {
die("Construct session failed: ",
   Infoblox::status_code() . ":" . Infoblox::status_detail());
}
print "Session created successfully\n";
```

#Create a DHCP Network object

```
my $memberdhcp = Infoblox::DHCP::Member->new(
        name     => "infoblox.localdomain",
        ipv4addr => "192.168.1.2"
       );

my  $network = Infoblox::DHCP::Network->new(
        network => "10.0.0.0/8",
        comment => "add network",
        members => [$memberdhcp],
         );

my $response = $session->add($network)
   or die("Add Network failed: ",
         $session->status_code() . ":" . $session->status_detail());

print "Network added successfully\n";

#Search for a specific Network

#Search the network
my @retrieved_objs = $session->search(
object  => "Infoblox::DHCP::Network",
network => '10\.0\.0\..*'
);
my $object = $retrieved_objs[0];
unless ($object) {
die("Search Network failed: ",
    $session->status_code() . ":" . $session->status_detail());
}
print "Search Network found at least 1 matching entry\n";
```

#Get and modify a Network attribute

```
#Get Network through the session
my @retrieved_objs = $session->get(
       object =>"Infoblox::DHCP::Network" ,
       network => "10.0.0.0"
    );

my $object = $retrieved_objs[0];
```

```
unless ($object) {
die("Get Network object failed: ",
    $session->status_code() . ":" . $session->status_detail());
}
print "Get Network found at least 1 matching entry\n";
```

#Modify attributes of the specified Network attribute

```
$object->enable_ddns("true"); #Apply the changes

#Un-override bootfile
$object->bootfile(undef);

#Un-override bootserver
$object->bootserver(undef);

#Un-override nextserver
$object->nextserver(undef);

$session->modify($object)
or die("Modify Network attribute failed: ",
       $session->status_code() . ":" . $session->status_detail());

print "Network object modified successfully \n";
```

#Resize a network

```
#Get Network through the session
my @retrieved_objs = $session->get(
       object =>"Infoblox::DHCP::Network" ,
       network => "10.0.0.0/8"
    );
my $object = $retrieved_objs[0];

#Resize
my $response = $object->resize(
  prefix        => "16"
) or die("Resize Network failed: ",
     $session->status_code() . ":" . $session->status_detail());
```

#Remove a Network Object

```
  #Get Network object through the session
  my @retrieved_objs = $session->get(
       object  => "Infoblox::DHCP::Network" ,
       network => "10.0.0.0"
    );
```

my $object = $retrieved_objs[0];

unless ($object) { die("Get Network object failed: ", $session->status_code() . ":" . $session->status_detail()); }

```
print "Get Network object found at least 1 matching entry\n";
```

#Submit the object for removal

```
$session->remove($object)
or die("Remove Network failed: ",
    $session->status_code() . ":" . $session->status_detail());

print "Network removed successfully \n";

####PROGRAM ENDS####
```

AUTHOR

Infoblox Inc. http://www.infoblox.com/

SEE ALSO

Infoblox::Session, Infoblox::DHCP::View, Infoblox::DHCP::Member, Infoblox::DHCP::Option, Infoblox::Session->get(), Infoblox::Session->search(), Infoblox::Session->add(), Infoblox::Session->remove(), Infoblox::Session->modify(), Infoblox::Grid::MSServer::AdUser::Data

---

COPYRIGHT

Copyright (c) 2017 Infoblox Inc.

Infoblox::DHCP::NetworkContainer - DHCP Network Container object

- NAME
- DESCRIPTION
- CONSTRUCTOR
- SESSION METHODS
  - Infoblox::Session->add( )
  - Infoblox::Session->get( )
  - Infoblox::Session->modify( )
  - Infoblox::Session->remove( )
  - Infoblox::Session->search( )
- METHODS
  - authority( )
  - auto_create_reversezone( )
  - bootfile( )
  - bootserver( )
  - comment( )
  - cloud_info( )
  - ddns_domainname( )
  - ddns_generate_hostname( )
  - ddns_server_always_updates( )
  - ddns_ttl( )
  - ddns_update_fixed_addresses( )
  - ddns_use_option81( )
  - deny_bootp( )
  - discovery_basic_poll_setting( )
  - discovery_member( )
  - discover_now_status( )
  - discovery_blackout_setting( )
  - discovery_engine_type( )
  - email_list( )
  - enable_ddns( )
  - enable_dhcp_thresholds( )
  - enable_email_warnings( )
  - enable_discovery( )
  - enable_immediate_discovery( )
  - enable_snmp_warnings( )
  - endpoint_sources( )
  - extattrs( )
  - extensible_attributes( )
  - high_water_mark( )
  - high_water_mark_reset( )
  - ignore_id( )
  - ignore_mac_addresses( )
  - ignore_dhcp_option_list_request( )
  - ipam_email_addresses( )
  - ipam_trap_settings( )
  - ipam_threshold_settings( )
  - mgm_private( )
  - mgm_private_overridable( )
  - ms_ad_user_data( )
  - lease_scavenge_time( )
  - logic_filters( )
  - low_water_mark( )
  - low_water_mark_reset( )
  - network( )
  - override_blackout_setting( )
  - nextserver( )
  - options( )
  - override_ddns_domainname( )
  - override_ddns_ttl( )
  - override_ignore_id( )
  - override_ipam_email_addresses( )
  - override_ipam_threshold_settings( )
  - override_ipam_trap_settings( )
  - override_logic_filters( )
  - override_lease_scavenge_time( )
  - override_mgm_private( )
  - override_subscribe_settings( )
  - override_update_dns_on_lease_renewal( )
  - port_control_blackout_setting
  - pxe_lease_time( )
  - recycle_leases( )

- remove_subnets( )
- network_container( )
- network_view( )
- next_available_network( )
- resize( )
- restart_if_needed( )
- rir( )
- rir_organization( )
- rir_registration_action( )
- rir_registration_status( )
- last_rir_registration_update_sent( )
- last_rir_registration_update_status( )
- same_port_control_discovery_blackout( )
- send_rir_request( )
- subscribe_settings( )
- delete_reason( )
- unmanaged( )
- update_dns_on_lease_renewal( )
- zone_associations( )

- SAMPLE CODE
- AUTHOR
- SEE ALSO
- COPYRIGHT

NAME

Infoblox::DHCP::NetworkContainer - DHCP Network Container object

---

DESCRIPTION

A network can contain child networks. The network that contains the child network is called a network container.

CONSTRUCTOR

```
  my $network = Infoblox::DHCP::NetworkContainer->new(
                network                          => $ipv4_subnet/prefix,               #Required, when
creating a network container
                auto_create_reversezone          => "true" | "false",                  #Optional / Default
is "false"
                comment                          => $string| undef,                    #Optional / Default
is undefined
                remove_subnets                   => "true" | "false",                  #Optional / Default
is undefined
                rir_organization                 => $rir_org,                          #Optional / An
Infoblox::Grid::RIR::Organization object
                rir_registration_action          => 'NONE' | 'CREATE' | 'MODIFY' | 'DELETE',  # Optional /
write-only
                rir_registration_status          => 'NOT_REGISTERED' | 'REGISTERED',   #Optional / Default
is 'NOT_REGISTERED'
                send_rir_request                 => 'true' | 'false',                  #Optional / write-
only
                delete_reason                    => $string,                           #Optional / write-
only
                                                                                       #Required only if
'rir_registration_action' is specified as 'DELETE'
                mgm_private                      => "true" | "false",                  #Optional / Default
is "false"
                network_container                => $ipv4_subnet/prefix,               #Optional / Default
is "/"
                network_view                     => $NetworkView,                      #Optional / Default
is "default" network view
                zone_associations                => [$zone1, $zone2, ... ] | undef,    #Optional / Default
is an empty list
                extattrs                         => { $string => $extattr, ... },      #Optional / Default
is undefined
                extensible_attributes            => { $string => $string | $num, $string => [ $string |
$num, ... ], ... } #Required if 'rir_organization' attribute is defined. Otherwise is optional
                discovery_blackout_setting       => $blackout,                         #Optional
                port_control_blackout_setting    => $blackout,                         #Optional
                override_blackout_setting        => "true" | "false",                  #Optional / Default
value is "false"
                same_port_control_discovery_blackout  => "true" | "false",             #Optional / Default
value is "false"
                ipam_trap_settings                => $ipam_trap,                       #Optional / An
Infoblox::Grid::SNMP::IPAMTrap object
                override_ipam_trap_settings      => "true" | "false",                  #Optional / Default
```

```
value is "false"
                ipam_threshold_settings          => $ipam_threshold,                #Optional / An
Infoblox::Grid::SNMP::IPAMThreshold object
                override_ipam_threshold_settings => "true" | "false",               #Optional / Default
value is "false"
                ipam_email_addresses             => [$email1, $email2, ...],        #Optional / Default
is an empty list
                override_ipam_email_addresses    => "true" | "false",               #Optional / Default
value is "false"

                authority                        => "true" | "false"| undef,        #Optional / Default
is undefined
                bootfile                         => $filename | undef,              #Optional / Default
is undefined
                bootserver                       => $ipv4addr | $fqdn | undef,      #Optional / Default
is undefined
                ddns_domainname                  => $string| undef,                 #Optional / Default
is undefined
                ddns_generate_hostname           => "true" | "false" | undef,       #Optional / Default
is undefined
                ddns_server_always_updates       => "true" | "false",               #Optional / Default
is "true"
                ddns_ttl                         => $num,                           #Optional / Default
is "0"
                ddns_use_option81                => "true" | "false" | undef,       #Optional / Default
is undefined
                ddns_update_fixed_addresses      => "true" | "false" | undef,       #Optional / Default
is undefined
                deny_bootp                       => "true" | "false" | undef,       #Optional / Default
is undefined
                email_list                       => [ "admin\@test.com",...] | undef,  #Optional / Default
is undefined
                enable_ddns                      => "true" | "false" | undef,       #Optional / Default
is undefined
                enable_dhcp_thresholds           => "true" | "false",               #Optional / Default
is "false"
                enable_email_warnings            => "true" | "false",               #Optional / Default
is "false"
                enable_snmp_warnings             => "true" | "false",               #Optional / Default
is "false"
                high_water_mark                  => $number,                        #Optional / Default
is "95"
                high_water_mark_reset            => $number,                        #Optional / Default
is "95"
                ignore_dhcp_option_list_request  => "true" | "false" | undef,       #Optional / Default
is undefined
                ignore_id                        => 'NONE' | 'CLIENT' | 'MACADDR',  #Optional / Default
is 'NONE'
                ignore_mac_addresses             => [ "mac_addr", ... ] | undef     #Optional / Default
is undefined
                lease_scavenge_time              => $number,                        #Optional / Default
is "-1"
                low_water_mark                   => $number,                        #Optional / Default
is "0"
                low_water_mark_reset             => $number,                        #Optional / Default
is "0"
                nextserver                       => $ipv4addr | $fqdn | undef,      #Optional / Default
is undefined
                options                          => [$Option1,$Option2,...],        #Optional / Default
is empty list
                override_ddns_domainname         => "true" | "false",               #Optional / Default
value is "false"
                override_ddns_ttl                => "true" | "false",               #Optional / Default
value is "false"
                override_lease_scavenge_time     => "true" | "false",               #Optional / Default
value is "false"
                override_mgm_private             => "true" | "false",               #Optional / Default
value is "false"
                override_update_dns_on_lease_renewal => "true" | "false",           #Optional / Default
value is "false"
                override_ignore_id               => "true" | "false",               #Optional / Default
value is "false"
                pxe_lease_time                   => $num | undef,                   #Optional / Default
is undefined
                recycle_leases                   => "true" | "false" | undef,       #Optional / Default
is undefined
                update_dns_on_lease_renewal      => "true" | "false",               #Optional / Default
value is "false"
                logic_filters                    => [ $filter_name | $filter_obj, ...], #Optional /
Default is undefined
                override_logic_filters           => "true' | 'false",                #Optional /
Default value is "false"
                subscribe_settings          => $subscribe_settings,                 #Optional /
Default is undefined
```

```
                override_subscribe_settings => 'true' | 'false',                        #Optional /
Default is 'false'
                discovery_engine_type       => 'NETMRI' | 'NETWORK_INSIGHT' | 'VDISCOVERY' | 'UNKNOWN' |
'NONE', #Optional / Default is 'NONE'
                                );
```

You cannot set both extattrs and extensible_attributes at the same time.

SESSION METHODS

This section describes all the methods in an Infoblox::Session module that you can apply to a DHCP Network Container object.

Infoblox::Session->add( )

Use this method to add a DHCP Network Container object to the Infoblox appliance. See Infoblox::Session->add() for parameters and return values.

Example

```
#Adding a network container.
#Construct an object
my $network_container = Infoblox::DHCP::NetworkContainer->new(
      network => "20.0.0.0/8",
);

# Submit for adding a network container
my $response = $session->add( $network_container );
```

Infoblox::Session->get( )

Use this method to retrieve all the matching DHCP Network Container objects from the Infoblox appliance. See Infoblox::Session->get() for parameters and return values.

'discover_now_status' will not be returned by default, use 'return_methods' to request it if needed.

Key References

```
 Apply the following attributes to get a specific DHCP Network Container object:

  network               - Optional. The IP address and netmask of a network in CIDR format.
  network_container     - Optional. The network container, in CIDR format, to which this network
container belongs.
  network_view          - Optional. The network view in which the network is located. By default, all
network views are searched.
  extattrs              - Optional. A hash reference containing extensible attributes.
  extensible_attributes - Optional. A hash reference containing extensible attributes.
  comment               - Optional. A comment in string format.
```

Examples

```
 my  @result_array = $session->get(
      object       => "Infoblox::DHCP::NetworkContainer",
      network      => "20.0.0.0/8",
      network_view  => "default"
 );

 my  @result_array = $session->get(
      object               => "Infoblox::DHCP::NetworkContainer",
      extensible_attributes => { 'Site' => 'Santa Clara' }
 );

 my  @result_array = $session->get(
      object  => "Infoblox::DHCP::NetworkContainer",
      comment => "my-comment",
 );
```

Infoblox::Session->modify( )

Use this method to modify a DHCP Network Container object in the Infoblox appliance. See Infoblox::Session->modify() for parameters and return values.

Example

```
#Use method to modify comment.
$network_container->comment("modified comment");
# Submit modification
my $response = $session->modify( $network_container );
```

Infoblox::Session->remove( )

Use this method to remove an object from the Infoblox appliance. See Infoblox::Session->remove() for parameters and return values.

To remove a specifc object, first use `get()` or `search()` to retrieve the specific object, and then submit this object for removal.

Key References

```
remove_options - Optional. A hash reference containing the following attributes:

+-------------------------------------------------------------------------------------------+
| reason             - Optional. The reason for removing the network container in string    |
|                      format with a maximum of 256 bytes.                                  |
|                      Used when 'rir_organization' attribute has been defined.             |
| send_rir_request   - Optional. Boolean flag that indicates whether the appliance should  |
|                      send an API or email request for Regional Internet Registry (RIR).   |
|                      Used when 'rir_organization' attribute has been defined.             |
+-------------------------------------------------------------------------------------------+
```

Example

```
 # Get the object under the same name
my  @result_array = $session->get(

       object       => "Infoblox::DHCP::NetworkContainer" ,
       network      => "20.0.0.0/8",
       network_view  => "default"
         );

 # Find the desired object in the retrieved list.
 my $network_container = $result_array[0];

 # Submit for removal
 my $response = $session->remove(
     $network_container,
     remove_options => {'reason' => 'Moved to B3', send_rir_request => 'true'} );
```

Infoblox::Session->search( )

Use this method to search for a DHCP Network Container object in the Infoblox appliance. See Infoblox::Session->search() for parameters and return values.

'discover_now_status' will not be returned by default, use 'return_methods' to request it if needed.

Key References

```
 Apply the following attributes to search for a DHCP Network Container object:

  network              - Optional. The IP address and netmask of a network in CIDR format (regular
expression).
  network_container    - Optional. The network container, in CIDR format, to which this network
container belongs.
  network_view         - Optional. The network view in which the network is located. By default, all
network views are searched.
  extattrs             - Optional. A hash reference containing extensible attributes.
  extensible_attributes - Optional. A hash reference containing extensible attributes.
  comment              - Optional. A comment in string format (regular expression).
```

For more information about searching extensible attributes, see Infoblox::Grid::ExtensibleAttributeDef/Searching Extensible Attributes.

Examples

```
 # search for all DHCP Network Container objects
     my @retrieved_objs = $session->search(
       object       => "Infoblox::DHCP::NetworkContainer",
       network      => '20\.0\.0\..*',
       network_view => "default"
     );

 # search network by comment
     my @retrieved_objs = $session->search(
       object    => "Infoblox::DHCP::NetworkContainer",
       comment   => "my-comment",
     );

 # search for all DHCP Network Container objects defining "Santa Clara" for "Site" extensible
attribute
     my @retrieved_objs = $session->search(
       object               => "Infoblox::DHCP::NetworkContainer",
       extensible_attributes => { 'Site' => 'Santa Clara' }
     );
```

METHODS

This section describes all the methods that can be used to configure and retrieve the attribute values of a DHCP Network Container object.

authority( )

Use this method to set or retrieve the authority of a DHCP Network Container object. This setting overrides the upper-level settings.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

Specify "true" to set the authority or "false" to deactivate/unset it. The default value for this field is undefined, which indicates that this attribute inherit the upper-level setting.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
# Get authority
my $authority = $network_container->authority()
# Modify authority
$network_container->authority("true");
```

auto_create_reversezone( )

Use this method to set the auto_create_reversezone flag of a DHCP Network Container object.

Include the specified parameter to set the attribute value. This is a write-only attribute and cannot be retrieved after it is set.

Parameter

Specify "true" to set the auto_create_reversezone flag or "false" to deactivate/unset it. The default value for this field is "false".

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

Example

```
# Modify auto_create_reversezone
$network_container->auto_create_reversezone("true");
```

bootfile( )

Use this method to set or retrieve a bootfile attribute of a DHCP Network Container object. You can configure the DHCP server to support clients that use boot file name option in their DHCPREQUEST messages. This setting overrides the upper-level settings. The default value is undefined, which indicates that this attribute inherits the upper-level setting.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

This option is overridden independently from Infoblox::DHCP::NetworkContainer->bootserver() and Infoblox::DHCP::NetworkContainer->nextserver().

Parameter

The name, in string format, of the file that the client must download. The default value is undefined.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
# Get bootfile
my $bootfile = $network_container->bootfile();
# Modify bootfile
$network_container->bootfile("bootfile1");
#Un-override bootfile
$network_container->bootfile(undef);
```

bootserver( )

Use this method to set or retrieve a bootserver attribute of a DHCP Network Container object. You can specify the name and/or IP address of the boot server that host needs to boot. This setting overrides the upper-level settings. The default value is undefined, which indicates that this attribute inherits the upper-level setting.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

This option is overridden independently from Infoblox::DHCP::NetworkContainer->bootfile() and Infoblox::DHCP::NetworkContainer->nextserver().

Parameter

The boot server IP address or name in FQDN (Fully Qualified Domain Name) format. The FQDN consists of the host name followed by the domain name (example: abc.com). A boot server name can have a maximum of 256 bytes. The default value is undefined.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
# Get bootserver
my $bootserver = $network_container->bootserver();
# Modify bootserver
$network_container->bootserver("abc.domain.com");
#Un-override bootserver
$network_container->bootserver(undef);
```

comment( )

Use this method to set or retrieve a descriptive comment for a DHCP Network Container object.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

Enter a descriptive comment for the network container in string format with a maximum of 256 bytes.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
# Get comment
my $comment = $network_container->comment();
# Modify comment
$network_container->comment("add a network");
```

cloud_info( )

Use this method to set or retrieve cloud API related information for the Infoblox::DHCP::NetworkContainer object.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

The valid value is a Infoblox::Grid::CloudAPI::Info object.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
# Get cloud_info
my $cloud_info = $network_view->cloud_info();
# Modify cloud_info
$network_view->cloud_info($cloud_info_object);
```

ddns_domainname( )

Use this method to set or retrieve the ddns_domainname value of a DHCP Network Container object.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Setting this method to a defined value implicitly sets the override_ddns_domainname method to "true". Setting the parameter to undefined causes the appliance to use the Grid default and automatically resets the override_ddns_domainname attribute to "false".

Note that when ddns_domainname is set to a defined value and override_dddns_domainname is set to "false", the last operation takes precedence. Thus the sequence $object->ddns_domainname("testdomain"); $object->override_ddns_domainname("false"); will set override_ddns_domainname to "false", and the sequence $object->override_ddns_domainname("false"); $object->ddns_domainname("testdomain"); will result in override_ddns_domainname="true".

Parameter

The dynamic DNS domain name in string format.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get ddns_domainname
my $ddns_domainname = $network_container->ddns_domainname();
#Modify ddns_domainname
$network_container->ddns_domainname("test_domain.com");
```

ddns_generate_hostname( )

Use this method to set or retrieve the ddns_generate_hostname flag of a DHCP Network Container object. Enable this method to allow only the DHCP server for generating hostname. This setting overrides the upper-level settings.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

Specify "true" to generate hostname or "false" to deactivate/unset it.The default value for this field is undefined, which indicates that this attribute inherit the member-level setting.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
# Get ddns_generate_hostname
my $ddns_generate_hostname = $network_container->ddns_generate_hostname()
# Modify ddns_generate_hostname
$network_container->ddns_generate_hostname("true");
#Un-override
$network_container->ddns_generate_hostname(undef);
```

ddns_server_always_updates( )

Use this method to set or retrieve the ddns_server_always_updates flag of a DHCP Network Container object. Enable this method to allow only the DHCP server to update DNS regardless of the requests from the DHCP clients.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

Specify "true" to set the flag that enables the DHCP server to always updates DNS or "false" to deactivate/unset it.The default value for this field is "true".

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
# Get ddns_server_always_updates
my $ddns_server_always_updates = $network_container->ddns_server_always_updates()
# Modify ddns_server_always_updates
```

```
$network_container->ddns_server_always_updates("true");
```

ddns_ttl( )

Use this method to set or retrieve the DNS update Time to Live (TTL) value of a DHCP Network Container object.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Setting this method to a defined value implicitly sets the override_ddns_ttl method to "true". Setting the parameter to undefined causes the appliance to use the Grid default and automatically resets the override_ddns_ttl attribute to "false".

Note that when ddns_ttl is set to a defined value and override_dddns_ttl is set to "false", the last operation takes precedence. Thus the sequence $object->ddns_ttl(12); $object->override_ddns_ttl("false"); will set override_ddns_ttl to "false", and the sequence $object->override_ddns_ttl("false"); $object->ddns_ttl(12); will result in override_ddns_ttl="true".

Parameter

A 32-bit integer (range from 0 to 4294967295) that represents the duration, in seconds, the update is cached. Zero indicates that the update is not cached. The default value is 0.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get DNS Update ttl
my $dns_ttl = $network_container->ddns_ttl();
#Modify DNS Update ttl
$network_container->ddns_ttl(1200);
```

ddns_update_fixed_addresses( )

Use this method to set or retrieve the ddns_update_fixed_addresses flag of a DHCP Network Container object. By default, the DHCP server does not update DNS when it allocates a fixed address to a client. You can configure the DHCP server to update the A and PTR records of a client with a fixed address. When this feature is enabled and the DHCP server adds A and PTR records for a fixed address, the DHCP server never discards the records. This setting overrides the upper-level settings.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

Specify "true" to enable the appliance to update DNS when it assigns a fixed address to a client or "false" to disable it. The default value for this field is undefined, which indicates that this attribute inherit the member-level setting.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
# Get ddns_update_fixed_addresses
my $ddns_update_fixed_addresses= $network_container->ddns_update_fixed_addresses()
# Modify ddns_update_fixed_addresses
$network_container->ddns_update_fixed_addresses("true");
#Un-override
$network_container->ddns_update_fixed_addresses(undef);
```

ddns_use_option81( )

Use this method to set or retrieve the ddns_use_option81 flag of a DHCP Network Container object. Enable this method to define the settings for option 81 at the network level. This setting overrides the member-level settings.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

Specify "true" to Configure Support for Option 81 or "false" to deactivate/unset it. The default value for this field is undefined, which indicates that this attribute inherit the member-level setting.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
# Get ddns_use_option81
my $ddns_use_option81 = $network_container->ddns_use_option81()
# Modify ddns_use_option81
$network_container->ddns_use_option81("true");
# un-override ddns_use_option81
$network_container->ddns_use_option81(undef);
```

deny_bootp( )

Use this method to set or retrieve the deny_bootp flag of a DHCP Network Container object.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

Specify "true" to set the deny_bootp flag or "false" to deactivate/unset it. The default value is undefined, which indicates that this attribute inherit the upper-level setting.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get deny_bootp
my $deny_bootp = $network_container->deny_bootp();
#Modify deny_bootp
$network_container->deny_bootp("true");
```

discovery_basic_poll_setting( )

Use this method to set or retrieve the discovery_basic_poll_setting value of a DHCP Network Container object.

Include the specified parameter to set the attribute value.

Parameter

The valid value is object of the Infoblox::Grid::Discovery::BasicPollSettings manpage.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get discovery_basic_poll_setting
my $discovery_basic_poll_setting = $network->discovery_basic_poll_setting();
#Modify discovery_basic_poll_setting
$network->discovery_basic_poll_setting($poll_setting);
```

discovery_member( )

Use this method to set or retrieve the discovery_member value of a DHCP Network Container object.

Parameter

The discovery member name in string format.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get discovery_member
my $discovery_member = $network->discovery_member();
#Modify discovery_member
$network->discovery_member("test.localdomain");
```

discover_now_status( )

Use this method to retrieve the current discovery status. This attribute is read-only.

'discover_now_status' will not be returned by default, use 'return_methods' to request it if needed.

Parameter

None.

Returns

The method returns the attribute value that can be 'COMPLETE', 'FAILED', 'NONE', 'PENDING' or 'RUNNING'.

Example

```
#Get discover_now_status
my $discover_now_status = $network->discover_now_status();
```

discovery_blackout_setting( )

Use this method to set or retrieve the discovery blackout setting.

Setting this method to a defined value implicitly sets the override_blackout_setting method to "true". If port_control_blackout_setting is undefined, setting the parameter to undefined causes the appliance to use the Grid default and automatically resets the override_blackout_setting attribute to "false".

Note that when discovery_blackout_setting is set to a defined value and override_blackout_setting is set to "false", the last operation takes precedence. Thus the sequence $network->discovery_blackout_setting($blackout); $network->override_blackout_setting("false"); will set override_blackout_setting to "false", and the sequence $network->override_blackout_setting("false"); $network->discovery_blackout_setting($blackout); will result in override_blackout_setting="true".

Parameter

Valid value is an Infoblox::Grid::Discovery::Properties::BlackoutSetting object.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get discovery_blackout_setting
my $discovery_blackout_setting = $network->discovery_blackout_setting();
#Modify discovery_blackout_setting
$network->discovery_blackout_setting($blackout);
```

discovery_engine_type( )

Use this method to retrieve the DHCP Network Container container discovery engine type. This attribute is read-only.

Parameter

None.

Returns

The method returns the attribute value that can be 'NETMRI', 'NETWORK_INSIGHT', 'VDISCOVERY', 'UNKNOWN' or 'NONE'.

Example

```
#Get discovery_engine_type
my $discovery_engine_type = $network_container->discovery_engine_type();
```

email_list( )

Use this method to set or retrieve the e-mail lists to which the appliance sends threshold e-mail messages.

If you enter e-mail addresses in the email_list, then the appliance sends DHCP threshold alerts to the e-mail addresses.

Parameter

The valid value is an array reference that contains string objects.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get email_list
my @email_list = $network_container->email_list();
#Modify email_list
$network_container->email_list( [ "admin\@test.com" ] );
```

enable_ddns( )

Use this method to set or retrieve the dynamic DNS updates flag of a DHCP Network Container object. The DHCP server can send DDNS updates to DNS servers in the same grid and to external DNS servers. This setting overrides the member-level settings.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

Specify "true" to set the dynamic DNS updates flag or "false" to deactivate/unset it. The default value for this field is undefined, which indicates that this attribute inherit the member-level setting.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
# Get enable_ddns
my $enable_ddns = $network_container->enable_ddns()
# Modify enable_ddns
$network_container->enable_ddns("true");
# Un-override enable_ddns
$network_container->enable_ddns(undef);
```

enable_dhcp_thresholds( )

Use this method to set or retrieve DHCP thresholds at the network container level. These thresholds are the high and low watermarks. If a DHCP Network Container usage goes above or below these percentage values, the Infoblox appliance generates a syslog message and sends an SNMP notification.

Enable this flag to set DHCP thresholds values.

Parameter

Specify "true" to set the enable_dhcp_thresholds flag or "false" to deactivate/unset it. The default value is undefined.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get enable_dhcp_thresholds
my $enable_dhcp_thresholds = $network_container->enable_dhcp_thresholds();
#Modify enable_dhcp_thresholds
$network_container->enable_dhcp_thresholds("false");
```

enable_email_warnings( )

Use this method to send DHCP threshold warnings through e-mail.

Enable this flag if you want the appliance to send DHCP threshold warnings to the "email_list" provided when DHCP threshold is enabled.

Parameter

Specify "true" to set the enable_email_warnings flag or "false" to deactivate/unset it. The default value is undefined.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get enable_email_warnings
my $enable_email_warnings = $network_container->enable_email_warnings();
#Modify enable_email_warnings
$network_container->enable_email_warnings("false");
```

enable_discovery( )

Use this method to set or retrieve the enable_discovery value of a DHCP Network Container object. This setting overrides the member-level settings.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Setting this method to a defined value implicitly sets the override_enable_discovery method to "true". Setting the parameter to undefined causes the appliance to use the default from the upper level and automatically resets the override_enable_discovery attribute to "false".

Enable this flag if you want to set discovery for this object.

Parameter

Specify "true" to set the enable_discovery flag or "false" to deactivate/unset it. The default value is undefined.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get enable_discovery
my $enable_discovery = $network->enable_discovery();
#Modify enable_discovery
$network->enable_discovery("false");
```

enable_immediate_discovery( )

Use this method to set or retrieve enable_immediate_discovery value of a DHCP Network Container object.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

Specify "true" to set the enable_immediate_discovery flag or "false" to deactivate/unset it. The default value is undefined.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get enable_immediate_discovery
my $enable_immediate_discovery = $network->enable_immediate_discovery();
#Modify enable_immediate_discovery
$network->enable_immediate_discovery("false");
```

enable_snmp_warnings( )

Use this method to send DHCP threshold warnings through SNMP.

If the flag is enabled, then the appliance sends DHCP threshold warnings through SNMP messages when DHCP threshold is enabled.

Parameter

Specify "true" to set the enable_snmp_warnings flag or "false" to deactivate/unset it. The default value is undefined.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get enable_snmp_warnings
my $enable_snmp_warnings = $network_container->enable_snmp_warnings();
```

```
#Modify enable_snmp_warnings
$network_container->enable_snmp_warnings("false");
```

endpoint_sources( )

Use this method to retrieve the list of endpoints that provide object data. This is a read-only attribute.

Omit the parameter to retrieve the attribute value.

Parameter

None

Returns

The valid return value is an array of Infoblox::CiscoISE::Endpoint objects.

Example

```
#Get endpoint_sources value
my $endpoint_sources = $object->endpoint_sources();
```

extattrs( )

Use this method to set or retrieve the extensible attributes associated with a DHCP Network Container object.

Parameter

Valid value is a hash reference containing the names of extensible attributes and their associated values (Infoblox::Grid::Extattr objects).

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get extattrs
my $ref_extattrs = $network_container->extattrs();
#Modify extattrs
$network_container->extattrs({ 'Site' => $extattr1, 'Administrator' => $extattr2 });
```

extensible_attributes( )

Use this method to set or retrieve the extensible attributes associated with a DHCP Network Container object

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

For valid values for extensible attributes, see Infoblox::Grid::ExtensibleAttributeDef/Extensible Attribute Values.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get extensible attributes
my $ref_extensible_attributes = $network_container->extensible_attributes();
#Modify extensible attributes
$network_container->extensible_attributes({ 'Site' => 'Santa Clara', 'Administrator' => [ 'Peter',
'Tom' ] });
```

high_water_mark( )

Use this method to set the percentage value for DHCP Network Container usage above which the Infoblox appliance generates a syslog message and sends a warning.

When the percentage of allocated addresses in a DHCP Network Container exceeds the high watermark, the appliance generates a syslog message and sends a warning. Default value for this option is 95.

Parameter

The valid value is in sting format.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get high_water_mark
my $high_water_mark = $network_container->high_water_mark();
#Modify high_water_mark
$network_container->high_water_mark("80");
```

high_water_mark_reset( )

Use this method to set or retrieve the high_water_mark_reset value. If the percentage of allocated addresses drops below this value, a corresponding SNMP trap is reset.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

A number that specifies the percentage of allocated addresses. The range is from 1 to 100. The high watermark reset value must be lower than the high watermark value. The default value is 95.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get high_water_mark_reset
my $high_water_mark_reset = $network_container->high_water_mark_reset();
#Modify high_water_mark_reset
$network_container->high_water_mark_reset(70);
```

ignore_id( )

Use this method to set or retrieve the ignore_id type of a DHCP Network Container object. Indicates whether the appliance will ignore DHCP client IDs or MAC addresses. This setting overrides the upper-level settings.

Parameter

Valid values are 'NONE', 'CLIENT', or 'MACADDR'. The default is 'NONE'.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
# Get attribute value
$value = $network_container->ignore_id();
# Modify attribute value
$network_container->ignore_id('CLIENT');
#Un-override ignore_id
$network_container->ignore_id(undef);
```

ignore_mac_addresses( )

Use this method to set or retrieve the ignore_mac_addresses list of a DHCP Network Container object. A list of MAC addresses the appliance will ignore. This setting overrides the upper-level settings.

If you enter MAC addresses in the ignore_mac_addresses, then the appliance will ignore them. If you enter empty array, then the appliance will ignore any MAC address.

Parameter

The valid value is an array reference that contains string objects.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
# Get attribute value
$value = $network_container->ignore_mac_addresses();
# Modify attribute value
$network_container->ignore_mac_addresses(["11::11::11::11::11::11"]);
```

ignore_dhcp_option_list_request( )

Use this method to set or retrieve the ignore_dhcp_option_list_request flag of a DHCP Network Container object. If this flag is set to false, the appliance returns all the defined DHCP options to the client. This setting overrides the upper-level settings.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

Specify "true" to set the ignore_dhcp_option_list_request flag or "false" to deactivate/unset it. The default value is undefined, which indicates that this attribute inherits the upper-level setting.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
# Get ignore_dhcp_option_list_request
my $ignore_dhcp_option_list_request = $network_container->ignore_dhcp_option_list_request();
# Modify ignore_dhcp_option_list_request
$network_container->ignore_dhcp_option_list_request("true");
#Do not override ignore_dhcp_option_list_request
$network_container->ignore_dhcp_option_list_request(undef);
```

ipam_email_addresses( )

Use this method to set or retrieve a list of IPAM Utilization email addresses.

Setting this method to a defined value implicitly sets the override_ipam_email_addresses method to "true". Setting the parameter to undefined causes the appliance to use the default from the upper level and automatically resets the override_ipam_email_addresses attribute to "false".

Note that when ipam_email_addresses is set to a defined value and override_ipam_email_addresses is set to "false", the last operation takes precedence. Thus the sequence $object->ipam_email_addresses(['admin@infoblox.com']); $object->override_ipam_email_addresses("false"); will set override_ipam_email_addresses to "false", and the sequence $object->override_ipam_email_addresses("false"); $object->ipam_email_addresses(['admin@infoblox.com']); will result in override_ipam_email_addresses="true".

Parameter

Valid value is an array reference that contains email addresses in string format.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get ipam_email_addresses
my $ipam_email_addresses = $network_container->ipam_email_addresses();
#Modify ipam_email_addresses
$network_container->ipam_email_addresses($ipam_email_addresses);
```

ipam_trap_settings( )

Use this method to set or retrieve IPAM Utilization SNMP Trap settings.

Setting this method to a defined value implicitly sets the override_ipam_trap_settings method to "true". Setting the parameter to undefined causes the appliance to use the default from the upper level and automatically resets the override_ipam_trap_settings attribute to "false".

Note that when ipam_trap_settings is set to a defined value and override_ipam_trap_settings is set to "false", the last operation takes precedence. Thus the sequence $object->ipam_trap_settings($ipam_trap); $object->override_ipam_trap_settings("false"); will set

override_ipam_trap_settings to "false", and the sequence $object->override_ipam_trap_settings("false"); $object->ipam_trap_settings($ipam_trap); will result in override_ipam_trap_settings="true".

Parameter

Valid value is an Infoblox::Grid::SNMP::IPAMTrap object.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get ipam_trap_settings
my $ipam_trap_settings = $network_container->ipam_trap_settings();
#Modify ipam_trap_settings
$network_container->ipam_trap_settings($ipam_trap_settings);
```

ipam_threshold_settings( )

Use this method to set or retrieve IPAM Utilization SNMP Threshold settings.

Setting this method to a defined value implicitly sets the override_ipam_threshold_settings method to "true". Setting the parameter to undefined causes the appliance to use the default from the upper level and automatically resets the override_ipam_threshold_settings attribute to "false".

Note that when ipam_threshold_settings is set to a defined value and override_ipam_threshold_settings is set to "false", the last operation takes precedence. Thus the sequence $object->ipam_threshold_settings($ipam_threshold_settings); $object->override_ipam_trap_settings("false"); will set override_ipam_threshold_settings to "false", and the sequence $object->override_ipam_threshold_settings("false"); $object->ipam_threshold_settings($ipam_threshold_settings); will result in override_ipam_threshold_settings="true".

Parameter

Valid value is an Infoblox::Grid::SNMP::IPAMThreashold object.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get ipam_threshold_settings
my $ipam_threshold_settings = $network_container->ipam_threshold_settings();
#Modify ipam_threshold_settings
$network_container->ipam_threshold_settings($ipam_threshold_settings);
```

mgm_private( )

Use this method to set or retrieve the flag that controls whether the object should be synced to the MGM or not.

Setting this method to a defined value implicitly sets the override_blackout_setting method to "true". If discovery_blackout_setting is undefined, setting the parameter to undefined causes the appliance to use the grid default and automatically resets the override_blackout_setting attribute to "false".

Note that when mgm_private is set to a defined value and override_mgm_private is set to "false", the last operation takes precedence. Thus the sequence $network->mgm_private("true"); $network->override_mgm_private("false"); will set override_mgm_private to "false", and the sequence $network->override_mgm_private("false"); $network->mgm_private("true"); will result in override_mgm_private="true".

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

Specify 'true' to enable synchronization of the managed Grid with the MGM or 'false' to forbid it. The default value is 'false'.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Note that this member is loaded on on-demand basis, so to retrieve the value the object must have been retrieved from the session first.

Example

```
#Get mgm_private value
my $value = $object->mgm_private();

#Modify mgm_private value
$object->mgm_private('true');
```

mgm_private_overridable( )

Use this method to retrieve the flag that indicates whether the overriding of 'mgm_private' flag is allowed or not. This is a read-only attribute.

Parameter

None

Returns

The method returns the attribute value.

Note that this member is loaded on on-demand basis, so to retrieve the value the object must have been retrieved from the session first.

Example

```
#Get mgm_private_overridable value
my $value = $object->mgm_private_overridable();
```

ms_ad_user_data( )

Use this method to retrieve Microsoft Active Directory users related information. This is a read-only attribute.

Parameter

None

Returns

The valid return value is an Infoblox::Grid::MSServer::AdUser::Data object.

Example

```
#Get ms_ad_user_data
my $ms_ad_user_data = $network_container->ms_ad_user_data();
```

lease_scavenge_time( )

Use this method to set or retrieve the lease_scavenge_time value of a DHCP Network Container object.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Setting this method to a defined value implicitly sets the override_lease_scavenge_time method to "true". Setting the parameter to undefined causes the appliance to use the default from the upper level and automatically resets the override_lease_scavenge_time attribute to "false".

Note that when lease_scavenge_time is set to a defined value and override_lease_scavenge_time is set to "false", the last operation takes precedence. Thus the sequence $object->lease_scavenge_time(604800); $object->override_lease_scavenge_time("false"); will set override_lease_scavenge_time to "false", and the sequence $object->override_lease_scavenge_time("false"); $object->lease_scavenge_time(604800); will result in override_lease_scavenge_time="true".

Parameter

An integer that specifies the period of time, in seconds, that free and backup leases remain in the database before they are automatically deleted. To disable lease scavenging, set the parameter to -1. The minimum positive value must be greater than 86400 seconds (1 day). The default value of this parameter is -1, which means that lease scavenging is disabled.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get lease_scavenge_time
my $lease_scavenge = $network_container->lease_scavenge_time();
#Modify lease_scavenge_time
```

```
$network_container->lease_scavenge_time(30 * 24 * 60 * 60);
```

logic_filters( )

Use this method to set or retrieve all the logic (ordered) filters associated with the DHCP Network Container.

Filters are used to screen address assignments in a DHCP Network Container if the addresses are to be assigned to specific hosts.

Setting this method to a defined value implicitly sets the override_logic_filters method to "true". Setting the parameter to undefined causes the appliance to use the Grid default and automatically resets the override_logic_filters attribute to "false".

Note that when logic_filters is set to a defined value and override_dlogic_filters is set to "false", the last operation takes precedence. Thus the sequence $object->logic_filters($filters); $object->override_logic_filters("false"); will set override_logic_filters to "false", and the sequence $object->override_logic_filters("false"); $object->logic_filters($filters); will result in override_logic_filters="true".

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

The valid value is an array reference that contains either the Infoblox::DHCP::Filter::MAC, Infoblox::DHCP::Filter::NAC, Infoblox::DHCP::Filter::Option objects or the names of the corresponding filters. The filters in the list must satisfy the following conditions:

Filter name must be globally unique.
Filters without expressions must be at the end of the list.
The filters should not be disabled.

The default value of the parameter is undefined.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get logic_filters
my $ref_logic_filters = $network_container->logic_filters();
#Modify logic_filters
$network_container->logic_filters(["filter_1","filter_2",$exl1]); #$exl1 is an
Infoblox::DHCP::Filter::MAC, Infoblox::DHCP::Filter::NAC or Infoblox::DHCP::Filter::Option object.
```

low_water_mark( )

Use this method to set the percentage value for DHCP Network Container usage below which the Infoblox appliance generates a syslog message and sends a warning.

When the percentage of allocated addresses in a DHCP Network Container goes below the low watermark, the appliance generates a syslog message and sends a warning. Default value for this option is 0.

Parameter

The valid value is in string format.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get low_water_mark
my $low_water_mark = $network_container->low_water_mark();
#Modify low_water_mark
$network_container->low_water_mark("10");
```

low_water_mark_reset( )

Use this method to set or retrieve the low_water_mark_reset value. If the percentage of allocated addresses exceeds this value, a corresponding SNMP trap is reset.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

A number that specifies the percentage of allocated addresses. The range is from 1 to 100. The low watermark reset value must be

higher than the low watermark value. The default low watermark reset number is 0.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get low_water_mark_reset
my $low_water_mark_reset = $network_container->low_water_mark_reset();
#Modify low_water_mark_reset
$network_container->low_water_mark_reset(10);
```

network( )

Use this method to retrieve the network attribute of a DHCP Network Container object.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

Specify the address and netmask of the network in CIDR format (example: 10.0.0.0/8).

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get network
my $network = $network_container->network();
```

override_blackout_setting( )

The override_blackout_setting attribute controls whether the discovery_blackout_setting and port_control_blackout_setting values in the object are used, instead of the Grid default.

The override_blackout_setting attribute can be specified explicitly. It is also set implicitly when discovery_blackout_setting or port_control_blackout_setting is set to a defined value.

Parameter

Specify "true" to set the override_blackout_setting flag or "false" to deactivate/unset it. The default value is "false".

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
# Get attribute value
my $value = $network->override_blackout_setting();
# Modify attribute value
$network->override_blackout_setting('true');
```

nextserver( )

Use this method to set or retrieve the nextserver attribute of a DHCP Network Container object. You can specify the name and/or IP address of the next server that the host needs to boot. This setting overrides the upper-level settings. The default value is undefined, which indicates that this attribute inherits the upper-level setting.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

This option is overridden independently from Infoblox::DHCP::NetworkContainer->bootfile() and Infoblox::DHCP::NetworkContainer->bootserver().

Parameter

The next server IP address or name in FQDN (Fully Qualified Domain Name) format. The FQDN consists of the host name followed by the domain name (example: abc.com). A next server name can have a maximum of 256 bytes. The default value is undefined.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
# Get nextserver
my $nextserver = $network_container->nextserver();
# Modify nextserver
$network_container->nextserver("blue.domain.com");
#Un-override nextserver
$network_container->nextserver(undef);
```

options( )

Use this method to set or retrieve the options value of a member DHCP server. DHCP options provide network configuration settings and various services available on the network for DHCP.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

The valid value is an array reference that contains Infoblox::DHCP::Option objects.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get options
my $options = $network_container->options();
#Modify options
my $option1 = Infoblox::DHCP::Option->new(
    name => "lease-time",
    value => 50
);

$network_container->options([$option1]);
```

override_ddns_domainname( )

The override_ddns_domainname attribute controls whether the ddns_domainname value in the DHCP Network Container is used, instead of the Grid default.

The override_ddns_domainname attribute can be specified explicitly. It is also set implicitly when ddns_domainname is set to a defined value.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

Set the parameter to "true" to override the upper-level setting for ddns_domainname. Set the parameter to "false" to inherit the upper-level setting for ddns_domainname.

The default value of this parameter is "false".

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Getting override_ddns_domainname
my $override_ddns_domainname=$network_container->override_ddns_domainname( );
#Modifying override_ddns_domainname
$network_container->override_ddns_domainname("true");
```

override_ddns_ttl( )

The override_ddns_ttl attribute controls whether the ddns_ttl value in the DHCP Network Container is used, instead of the Grid default.

The override_ddns_ttl attribute can be specified explicitly. It is also set implicitly when ddns_ttl is set to a defined value.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

Set the parameter to "true" to override the upper-level setting for ddns_ttl. Set the parameter to "false" to inherit the upper-level setting for ddns_ttl.

The default value of this parameter is "false".

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Getting override_ddns_ttl
my $override_ddns_ttl=$network_container->override_ddns_ttl( );
#Modifying override_ddns_ttl
$network_container->override_ddns_ttl("true");
```

override_ignore_id( )

The override_ignore_id controls whether the ignore_id in the DHCP Network Container is used, instead of the upper-level default.

The override_ignore_id can be specified explicitly. It is also set implicitly when ignore_id is set to a defined value.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

Set the parameter to "true" to override the upper-level setting for ignore_id and ignore_mac_addresses. Set the parameter to "false" to inherit the upper-level setting for ignore_id and ignore_mac_addresses.

The default value of this parameter is "false".

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get attrubute value
my $value=$network_container->override_ignore_id();
#Modifying attribute value
$network_container->override_ignore_id('true');
```

override_ipam_email_addresses( )

The override_ipam_email_addresses controls whether the ipam_email_addresses in the DHCP network container is used, instead of the upper-level default.

The override_ipam_email_addresses can be specified explicitly. It is also set implicitly when ipam_email_addresses is set to a defined value.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

Set the parameter to "true" to override the upper-level setting for ipam_email_addresses. Set the parameter to "false" to inherit the upper-level setting for ipam_email_addresses.

The default value of this parameter is "false".

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get attrubute value
my $value=$network_container->override_ipam_email_addresses();
#Modifying attribute value
$network_container->override_ipam_email_addresses('true');
```

override_ipam_threshold_settings( )

The override_ipam_threshold_settings controls whether the ipam_threshold_settings in the DHCP network container is used, instead of the upper-level default.

The override_ipam_threshold_settings can be specified explicitly. It is also set implicitly when ipam_threshold_settings is set to a defined value.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

Set the parameter to "true" to override the upper-level setting for ipam_threshold_settings. Set the parameter to "false" to inherit the upper-level setting for ipam_threshold_settings.

The default value of this parameter is "false".

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get attrubute value
my $value=$network_container->override_ipam_threshold_settings();
#Modifying attribute value
$network_container->override_ipam_threshold_settings('true');
```

override_ipam_trap_settings( )

The override_ipam_trap_settings controls whether the ipam_trap_settings in the DHCP network container is used, instead of the upper-level default.

The override_ipam_trap_settings can be specified explicitly. It is also set implicitly when ipam_trap_settings is set to a defined value.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

Set the parameter to "true" to override the upper-level setting for ipam_trap_settings. Set the parameter to "false" to inherit the upper-level setting for ipam_trap_settings.

The default value of this parameter is "false".

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get attrubute value
my $value=$network_container->override_ipam_trap_settings();
#Modifying attribute value
$network_container->override_ipam_trap_settings('true');
```

override_logic_filters( )

The override_logic_filters attribute controls whether the logic_filters value of the DHCP Network Container is used, instead of the Grid default.

The override_logic_filters attribute can be specified explicitly. It is also set implicitly when logic_filters is set to a defined value.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

Set the parameter to "true" to override the Grid-level setting for logic_filters. Set the parameter to "false" to inherit the Grid-level setting for logic_filters.

The default value of this parameter is "false".

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Getting override_logic_filters
my $override_logic_filters=$network_container->override_logic_filters( );
#Modifying override_logic_filters
$network_container->override_logic_filters("true");
```

override_lease_scavenge_time( )

The override_lease_scavenge_time attribute controls whether the lease_scavenge_time value in the DHCP Network Container is used, instead of the upper-level default.

The override_lease_scavenge_time attribute can be specified explicitly. It is also set implicitly when lease_scavenge_time is set to a defined value.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

Set the parameter to "true" to override the upper-level setting for lease_scavenge_time. Set the parameter to "false" to inherit the setting for lease_scavenge_time from the upper level.

The default value of this parameter is "false".

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Getting override_lease_scavenge_time
my $override_lease_scavenge_time=$network_container->override_lease_scavenge_time( );
#Modifying override_lease_scavenge_time
$network_container->override_lease_scavenge_time("true");
```

override_mgm_private( )

The override_mgm_private attribute controls whether the mgm_private value in the object is used, instead of the parent-level default.

The override_mgm_private attribute can be specified explicitly. It is also set implicitly when mgm_private is set to a defined value.

Note that setting override_mgm_private to 'false' sets the original 'mgm_private' value to 'false'.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

Set the parameter to 'true' to override the parent-level setting for mgm_private. Set the parameter to 'false' to inherit the parent-level setting for mgm_private.

The default value of this parameter is 'false'.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Note that this member is loaded on on-demand basis, so to retrieve the value the object must have been retrieved from the session first.

Example

```
#Get override_mgm_private value
my $value = $object->override_mgm_private( );

#Modify override_mgm_private value
$object->override_mgm_private('true');
```

override_subscribe_settings( )

The override_subscribe_settings attribute controls whether the subscribe_settings method values of the member are used, instead of the Grid default.

The override_subscribe_settings attribute can be specified explicitly. It is also set implicitly when subscribe_settings is set to a defined value.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

Set the parameter to "true" to override the Grid-level setting for subscribe_settings. Set the parameter to "false" to inherit the Grid-level setting for subscribe_settings.

The default value of this parameter is "false".

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get override_subscribe_settings value
my $override_subscribe_settings = $object->override_subscribe_settings( );
#Modify override_subscribe_settings value
$object->override_subscribe_settings("true");
```

override_update_dns_on_lease_renewal( )

The override_update_dns_on_lease_renewal attribute controls whether the update_dns_on_lease_renewal value in the DHCP Network is used, instead of the Grid default.

The override_update_dns_on_lease_renewal attribute can be specified explicitly. It is also set implicitly when update_dns_on_lease_renewal is set to a defined value.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

Set the parameter to "true" to override the upper-level setting for update_dns_on_lease_renewal. Set the parameter to "false" to inherit the upper-level setting for update_dns_on_lease_renewal.

The default value of this parameter is "false".

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Getting override_update_dns_on_lease_renewal
my $override_update_dns_on_lease_renewal=$network->override_update_dns_on_lease_renewal( );
#Modifying override_update_dns_on_lease_renewal
$network->override_update_dns_on_lease_renewal("true");
```

port_control_blackout_setting

Use this method to set or retrieve the port control blackout setting.

Setting this method to a defined value implicitly sets the override_blackout_setting method to "true". If discovery_blackout_setting is undefined, setting the parameter to undefined causes the appliance to use the Grid default and automatically resets the override_blackout_setting attribute to "false".

Note that when port_control_blackout_setting is set to a defined value and override_blackout_setting is set to "false", the last operation takes precedence. Thus the sequence $network->port_control_blackout_setting($blackout); $network->override_blackout_setting("false"); will set override_blackout_setting to "false", and the sequence $network->override_blackout_setting("false"); $network->port_control_blackout_setting($blackout); will result in override_blackout_setting="true".

Parameter

Valid value is an Infoblox::Grid::Discovery::Properties::BlackoutSetting object.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get port_control_blackout_setting
my $port_control_blackout_setting = $network->port_control_blackout_setting();
#Modify port_control_blackout_setting
$network->port_control_blackout_setting($blackout);
```

pxe_lease_time( )

Use this method to set or retrieve the PXE lease time value of a DHCP Network Container object. Some hosts use PXE (Preboot Execution Environment) to boot remotely from a server. To better manage your IP resources, set a different lease time for PXE boot requests. You can configure the DHCP server to allocate an IP address with a shorter lease time to hosts that send PXE boot requests, so IP addresses are not leased longer than necessary. This setting overrides the upper-level settings. The default value is undefined, which indicates that this attribute inherits the upper-level setting.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

A 32-bit integer (range from 0 to 4294967295) that represents the duration, in seconds, that the update is cached. Zero indicates that the update is not cached. The default value is undefined.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get PXE Lease Time
my $pxe_lease_time = $network_container->pxe_lease_time();
#Modify PXE Lease Time
$network_container->pxe_lease_time(1100);
#Un-override PXE Lease Time
$network_container->pxe_lease_time(undef);
```

recycle_leases( )

Use this method to set or retrieve the recycle_leases flag at the network level.

If the flag is enabled, the leases are kept in the recycle bin until one week after expiration. When the flag is disabled, the leases are irrecoverably deleted.

Parameter

Specify "true" to enable the recycle_leases flag or "false" to disable it. The default value is undefined.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get recycle_leases
my $recycle_leases = $network->recycle_leases();
#Modify recycle_leases
$network->recycle_leases("false");
```

remove_subnets( )

Use this method to set or retrieve the remove_subnets flag of a DHCP Network Container object.

You can remove the network, but keep all subnets under it if this flag is set to false. This flag is only intended as an option when removing DHCP Network Container objects. It is not retrieved or used for any other operation.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

Specify "false" to keep the subnets or "true" to remove them. The default value is undefined, which is to remove all subnets.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Set remove_subnets before submitting for removal
$network_container->remove_subnets("false");
#Remove the container keeping the subnets
$session->remove($network_container);
```

network_container( )

Use this method to set or retrieve the network container of a DHCP Network Container object.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

Specify the IP address and netmask of a network in CIDR format.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get network container
my $network_container = $network->network_container();
#Modify network container
$network->network_container("50.0.0.0/8");
```

network_view( )

Use this method to set or retrieve the network view of the DHCP network container.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

The default value is the "default" network view, which means the network is in the default network view.

Parameter

Valid value is a defined Infoblox::DHCP::View object. The default network view value is the system-defined default network view object.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get network view
my $nview = $network_container->network_view();
#Modify network view, with an Infoblox::DHCP::View object
$network_container->network_view($nview);
```

next_available_network( )

Use this method to retrieve the next available `network(s)` in the network container. Note that to use this method, the object MUST be retrieved through a get or search call, or created with an add call.

Parameter

The method accepts the following parameters:

```
  cidr - (required) The CIDR of the requested networks. It must be a value between 1 and 32,
inclusive.
  requested_num - (optional) The number of networks requested. It must be a value between 1 and 20,
inclusive.
  excluded - (optional) aA reference to an array of networks to be excluded from the results.
```

Returns

This method returns the next available `network(s)` in the network container. If requested_num is not specified, a string with the next available network will be returned; otherwise, the method will return a reference to an array of networks.

A network cannot be returned as the next available network if it cannot be inserted; such as when any of the following conditions exist:

- It's parent network has a DHCP configuration assigned and therefore cannot be converted to a network container.

- It overlaps with an existing network or network container, regardless of whether the existing network has a greater or smaller CIDR than the requested CIDR.

- Its network address or broadcast address is used by a fixed address or a host configured for DHCP.

Example

```
# Retrieve a particular network
my @retrieved_objs = $session->get(
                          object   => "Infoblox::DHCP::NetworkContainer",
                          network  => "10.0.0.0/8"
);

$network = $retrieved_objs[0];

# Print the next available IPv6 network in the network container
print $network->next_available_network(cidr => 12) . "\n";

# This will retrieve the next 10 available networks and print the first one.
my $addressesref = $network->next_available_network(requested_num => 10, cidr => 12);
print @$addressesref[0] . "\n";

# This will retrieve the next 10 available networks and exclude two.
my $addressesref = $network->next_available_network(cidr => 12, requested_num => 10, excluded =>
['10.0.0.0/12']);
```

resize( )

Use this method to resize a network container. Note that to use this method, the object MUST be retrieved through a get or search call, or it must have been created with an add call.

Parameter

prefix - Required. The bitmask prefix length after resizing.

auto_create_reversezone - Optional. Boolean flag indicating whether or not to automatically create the reverse zone.

reason - Optional. The reason for resizing the network container in string format with a maximum of 256 bytes. Used when 'rir_organization' attribute has been defined.

send_rir_request - Optional. Boolean flag that indicates whether the appliance should send an API or email request for Regional Internet Registry (RIR). Used when 'rir_organization' attribute has been defined.

Returns

The method returns true when the operation succeeds, and returns false when the operation fails. When an operation fails, the error code and error message are reported in the Session object's `status_code()` and `status_detail()` attributes.

Example

```
#Get Network Container through the session
my @retrieved_objs = $session->get(
           object =>"Infoblox::DHCP::NetworkContainer" ,
           network => "20.0.0.0/8"
        );
my $network_container = $retrieved_objs[0];

# Expand a /24 network container to a /16 network container, inserting
# a corresponding reverese zone in the process.
my $response = $network_container->resize(
      prefix        => "16"
      auto_create_reversezone => "true"
) or die("Resize Network Container failed: ",
         $session->status_code() . ":" . $session->status_detail());
```

restart_if_needed( )

Use this method to set the flag that indicates whether DHCP service should be restarted on a member that is serving DHCP for the changed object referenced in a cloud API request.

Parameter

Specify true to restart the DHCP service.

Returns

None

Example

```
# Modify
$network->restart_if_needed('true');
```

rir( )

Use this method to retrieve the name of the Regional Internet Registry (RIR) that allocated this address space. This is a read-only attribute.

Parameter

None

Returns

The method returns the attribute value that can be 'RIPE' or 'NONE'.

Example

```
#Get rir
my $rir = $network_container->rir();
```

rir_organization( )

Use this method to set or retrieve the Regional Internet Registry (RIR) organization object.

Parameter

An Infoblox::Grid::RIR::Organization object.

You must enable support for RIR/SWIP at the Grid level before you can set attribute values. See Infoblox::Grid->enable_rir_swip() method.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get rir_organization
my $rir_organization = $network_container->rir_organization();
#Modify rir_organization
$network_container->rir_organization($rir_org);
```

rir_registration_action( )

Use this method to set the registration action of Regional Internet Registry (RIR). This is a write-only attribute.

Parameter

Valid values are 'NONE', 'CREATE', 'MODIFY' or 'DELETE'.

Returns

The method returns true when the modification succeeds, and returns false when the operation fails.

Example

```
#Set rir_registration_action
$network_container->rir_registration_action('CREATE');
```

rir_registration_status( )

Use this method to set or retrieve the registration status of Regional Internet Registry (RIR).

Parameter

The registration status of Regional Internet Registry (RIR), which can be 'REGISTERED' or 'NOT_REGISTERED'.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get rir_registration_status
my $rir_registration_status = $network_container->rir_registration_status();
#Modify rir_registration_status
$network_container->rir_registration_status('NOT_REGISTERED');
```

last_rir_registration_update_sent( )

Use this method to retrieve the last registration update timestamp of Regional Internet Registry (RIR). This is a read-only attribute.

Parameter

None

Returns

The method returns the attribute value.

Example

```
#Get last_rir_registration_update_sent
my $last_rir_registration_update_sent = $network_container->last_rir_registration_update_sent();
```

last_rir_registration_update_status( )

Use this method to retrieve the last registration update status of Regional Internet Registry (RIR). This is a read-only attribute.

Parameter

None

Returns

The method returns the attribute value.

Example

```
#Get last_rir_registration_update_status
my $last_rir_registration_update_status = $network_container->last_rir_registration_update_status();
```

same_port_control_discovery_blackout( )

Use this method to set or retrieve the flag that enables using the same discovery blackout for port control blackout.

Parameter

Specify 'true' to use same discovery blackout for port control blackout or 'false' to disable it. The default value is 'false'.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
# Get attribute value
my $value = $network_container->same_port_control_discovery_blackout();
# Modify attribute value
$network_container->same_port_control_discovery_blackout('true');
```

send_rir_request( )

Use this method to set the flag that indicates whether the appliance should send an API or email request for Regional Internet Registry (RIR). This is a write-only attribute.

Parameter

Specify "true" to set the send_rir_request flag or "false" to unset it.

Returns

The method returns true when the modification succeeds, and returns false when the operation fails.

Example

```
#Set send_rir_request
$network_container->send_rir_request('true');
```

subscribe_settings( )

Use this method to set or retrieve the Cisco ISE subscribe settings.

Setting this method to a defined value implicitly sets the override_subscribe_settings method to "true". Setting the parameter to undefined causes the appliance to use the Cisco ISE endpoint default and automatically resets the override_subscribe_settings method to "false".

Note that when subscribe_settings is set to a defined value and override_subscribe_settings is set to "false", the last operation takes precedence. Thus the sequence $object->subscribe_settings($value); $object->override_subscribe_settings("false"); will set override_subscribe_settings to "false", and the sequence $object->override_subscribe_settings("false"); $object->subscribe_settings($value); will result in override_subscribe_settings="true".

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

The valid value is an Infoblox::CiscoISE::SubscribeSetting object.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get subscribe_settings value
my $subscribe_settings = $object->subscribe_settings();
#Modify subscribe_settings value
$object->subscribe_settings($settings);
```

delete_reason( )

Use this method to specify the reason for deleting the Regional Internet Registry (RIR) registration request. This is a write-only attribute. This field is required only if 'rir_registration_action' is specified as 'DELETE'.

Parameter

The reason for deleting the RIR registration request in string format with a maximum of 256 bytes.

Returns

The method returns true when the modification succeeds, and returns false when the operation fails.

Example

```
#Set delete_reason
$network_container->delete_reason('invalid');
```

unmanaged( )

Use this method to set or retrieve the network status, which indicates whether the network is unmanaged or not.

Parameter

Specify true to mark the network as unmanaged or false as managed.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
# Get network status
$unmanaged = $network_container->unmanaged();
# Modify network status
$network_container->unmanaged('true');
```

update_dns_on_lease_renewal( )

The update_dns_on_lease_renewal attribute controls whether the DHCP server updates DNS when a DHCP lease is renewed.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Setting this method to a defined value implicitly sets the override_update_dns_on_lease_renewal method to "true". Setting the parameter to undefined causes the appliance to use the Grid default and automatically resets the override_update_dns_on_lease_renewal attribute to "false".

Note that when update_dns_on_lease_renewal is set to a defined value and override_update_dns_on_lease_renewal is set to "false", the last operation takes precedence. Thus the sequence $object->update_dns_on_lease_renewal("true"); $object->override_update_dns_on_lease_renewal("false"); will set override_update_dns_on_lease_renewal to "false", and the sequence $object->override_update_dns_on_lease_renewal("false"); $object->update_dns_on_lease_renewal("true"); will result in override_update_dns_on_lease_renewal="true".

Parameter

Specify "true" to set the update_dns_on_lease_renewal flag or "false" to deactivate/unset it.

The default value of this parameter is "false".

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get update_dns_on_lease_renewal
my $update_dns_on_lease_renewal=$network_container->update_dns_on_lease_renewal( );
#Modify update_dns_on_lease_renewal
$network_container->update_dns_on_lease_renewal("true");
```

zone_associations( )

Use this method to set or retrieve the associated zone information.

Parameter

The valid value is an array reference that contains Infoblox::DNS::Zone object(s). The default value is an empty list. If the network container has a parent, it is possible to use the zone associations of the parent by setting the parameter to undef.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails. If you did not specify a parameter, the method returns the attribute value.

Examples

```
#Getting zone associations
my $zone_associations = $network_container->zone_associations();
my $zone1 = Infoblox::DNS::Zone->new(
      name     => "test1.com",
);
my $zone2 = Infoblox::DNS::Zone->new(
      name     => "test2.com",
   );
#Modifying zone_associations
$network_container->zone_associations([$zone1, $zone2]);
#Clear zone_assocations
$network_container->zone_associations([]);
##Remove the override for zone_associations
$network_container->zone_associations(undef)
```

SAMPLE CODE

The following sample code demonstrates the different functions that can be applied to an object such as add, search, modify, and remove. This sample also includes error handling for the operations.

#Preparation prior to a DHCP Network Container object insertion

```
#PROGRAM STARTS: Include all the modules that will be used
use strict;
use Infoblox;

#Create a session to the Infoblox appliance

my $session = Infoblox::Session->new(
    master   => "192.168.1.2",
    username => "admin",
```

```
    password => "infoblox"
);
unless ($session) {
   die("Construct session failed: ",
       Infoblox::status_code() . ":" . Infoblox::status_detail());
}
print "Session created successfully\n";
```

#Create a DHCP Network Container object

```
my $network_container = Infoblox::DHCP::NetworkContainer->new(
            network => "10.0.0.0/8",
            comment => "add network container",
             );

my $response = $session->add($network_container)
       or die("Add Network Container failed: ",
             $session->status_code() . ":" . $session->status_detail());

print "Network Container added successfully\n";

#Search for a specific Network Container

#Search the network container
my @retrieved_objs = $session->search(
    object  => "Infoblox::DHCP::NetworkContainer",
    network => '10\.0\.0\..*'
);
my $object = $retrieved_objs[0];
unless ($object) {
    die("Search Network Container failed: ",
        $session->status_code() . ":" . $session->status_detail());
}
print "Search Network Container found at least 1 matching entry\n";
```

#Get and modify a Network Container attribute

```
#Get Network Container through the session
my @retrieved_objs = $session->get(
           object =>"Infoblox::DHCP::NetworkContainer" ,
           network => "10.0.0.0/8"
        );
```

```
my $object = $retrieved_objs[0];
```

```
unless ($object) {
    die("Get Network Container object failed: ",
        $session->status_code() . ":" . $session->status_detail());
}
print "Get Network Container found at least 1 matching entry\n";
```

#Modify one of the attributes of the specified Network Container object

```
$object->comment("modified comment"); #Apply the changes

$session->modify($object)
    or die("Modify Network Container attribute failed: ",
           $session->status_code() . ":" . $session->status_detail());

print "Network Container object modified successfully \n";
```

#Resize a network container

```
#Get Network through the session
my @retrieved_objs = $session->get(
    object =>"Infoblox::DHCP::NetworkContainer" ,
    network => "10.0.0.0/8"
);
my $object = $retrieved_objs[0];

#Resize
my $response = $object->resize(
    prefix => "16"
) or die("Resize Network Container failed: ",
    $session->status_code() . ":" . $session->status_detail());
```

#Remove a Network Container Object

```
 #Get Network Container object through the session
 my @retrieved_objs = $session->get(
           object  => "Infoblox::DHCP::NetworkContainer" ,
           network => "10.0.0.0"
       );

   my $object = $retrieved_objs[0];

   unless ($object) {
    die("Get Network Container object failed: ",
        $session->status_code() . ":" . $session->status_detail());
    }

print "Get Network Container object found at least 1 matching entry\n";
```

#Submit the object for removal

```
$session->remove($object)
    or die("Remove Network Container failed: ",
        $session->status_code() . ":" . $session->status_detail());

print "Network Container removed successfully \n";

####PROGRAM ENDS####
```

AUTHOR

Infoblox Inc. http://www.infoblox.com/

SEE ALSO

Infoblox::Session, Infoblox::DHCP::View, Infoblox::DHCP::Network, Infoblox::Session->get(), Infoblox::Session->search(), Infoblox::Session->add(), Infoblox::Session->remove(), Infoblox::Session->modify(), Infoblox::Grid::MSServer::AdUser::Data, Infoblox::DHCP::Filter::MAC, Infoblox::DHCP::Filter::NAC, Infoblox::DHCP::Filter::Option.

COPYRIGHT

Copyright (c) 2017 Infoblox Inc.

Infoblox::DHCP::NetworkTemplate - DHCP Network Template object

- NAME
- DESCRIPTION
- CONSTRUCTOR
- SESSION METHODS
  - Infoblox::Session->add( )
  - Infoblox::Session->get( )
  - Infoblox::Session->search( )
  - Infoblox::Session->modify( )
  - Infoblox::Session->remove( )
- METHODS
  - allow_any_netmask( )
  - authority( )
  - auto_create_reversezone( )
  - bootfile( )
  - bootserver( )
  - comment( )
  - ddns_domainname( )
  - ddns_generate_hostname( )
  - ddns_update_fixed_addresses( )
  - ddns_server_always_updates( )
  - ddns_ttl( )
  - ddns_use_option81( )
  - deny_bootp( )
  - email_list( )
  - enable_ddns( )
  - enable_dhcp_thresholds( )
  - enable_email_warnings( )
  - enable_snmp_warnings( )
  - extattrs( )
  - extensible_attributes( )
  - fixed_address_templates( )
  - high_water_mark( )
  - high_water_mark_reset( )
  - ignore_dhcp_option_list_request( )
  - ipam_email_addresses( )
  - ipam_trap_settings( )
  - ipam_threshold_settings( )
  - lease_scavenge_time( )
  - logic_filters( )
  - low_water_mark( )
  - low_water_mark_reset( )
  - members( )
  - name( )
  - netmask( )
  - nextserver( )
  - options( )
  - override_ddns_domainname( )
  - override_ddns_ttl( )
  - override_ipam_email_addresses( )
  - override_ipam_threshold_settings( )
  - override_ipam_trap_settings( )
  - override_lease_scavenge_time( )
  - override_logic_filters( )
  - override_update_dns_on_lease_renewal( )
  - pxe_lease_time( )
  - range_templates( )
  - update_dns_on_lease_renewal( )
  - recycle_leases( )
  - rir( )
  - rir_organization( )
  - rir_registration_action( )
  - rir_registration_status( )
  - send_rir_request( )
- SAMPLE CODE
- AUTHOR
- SEE ALSO
- COPYRIGHT

NAME

Infoblox::DHCP::NetworkTemplate - DHCP Network Template object

DESCRIPTION

You can use a template to create networks in a quick and consistent way. Networks created from a network template inherit all the properties defined in the network template, except for the comment and netmask if they are defined in the network.

CONSTRUCTOR

```
 my $network_template = Infoblox::DHCP::NetworkTemplate->new(
        name                            => $string,
#Required
        allow_any_netmask               => "true" | "false",
#Optional / Default is "false"
        authority                       => "true" | "false",
#Optional / Default is "false"
        auto_create_reversezone         => "true" | "false"
#Optional / Default is "false"
        bootfile                        => $filename | undef,
#Optional / Default is undefined
        bootserver                      => $ipv4addr | $fqdn | undef,
#Optional / Default is undefined
        comment                         => $string,
#Optional / Default is empty
        ddns_domainname                 => $string | undef,
#Optional / Default is undefined
        ddns_generate_hostname          => "true" | "false" | undef,
#Optional / Default is undefined
        ddns_update_fixed_addresses     => "true" | "false" | undef,
#Optional / Default is undefined
        ddns_server_always_updates      => "true" | "false",
#Optional / Default is "true"
        ddns_ttl                        => $num,
#Optional / Default is 0
        ddns_use_option81               => "true" | "false" | undef,
#Optional / Default is undefined
        deny_bootp                      => "true" | "false" | undef,
#Optional / Default undefined
        email_list                      => [ "admin\@test.com",...],
#Optional / Default is undefined
        enable_ddns                     => "true" | "false" | undef,
#Optional / Default is undefined
        enable_dhcp_thresholds          => "true" | "false",
#Optional / Default is undefined
        enable_email_warnings           => "true" | "false",
#Optional / Default is undefined
        enable_snmp_warnings            => "true" | "false",
#Optional / Default is undefined
        extattrs                        => { $string => $extattr, ... },
#Optional / Default is undefined
        extensible_attributes           => { $string => $string | $num, $string => [ $string | $num, ... ],
... }, #Required if 'rir_organization' attribute is defined

#Otherwise is optional
        fixed_address_templates         => [$FixedAddrTemplate1,$FixedAddrTemplate2,...]
#Optional / Default is empty
        high_water_mark                 => $number,
#Optional / Default is "95"
        high_water_mark_reset           => $number,
#Optional / Default is "95"
        ignore_dhcp_option_list_request => "true" | "false" | undef,
#Optional / Default is undefined
        lease_scavenge_time             => $number,
#Optional / Default is -1
        low_water_mark                  => $number,
#Optional / Default is "0"
        low_water_mark_reset            => $number,
#Optional / Default is "0"
        members                         => [$Member1,$Member2,...],
#Optional / Default is empty
        netmask                         => $netmask,
#Optional / Default is /8
        nextserver                      => $ipv4addr | $fqdn | undef,
#Optional / Default is undefined
        options                         => [$Option1,$Option2,...]
#Optional / Default is empty
        override_ddns_ttl               => "true" | "false",
#Optional / Default value is "false"
        override_ddns_domainname        => "true" | "false",
#Optional / Default value is "false"
        override_lease_scavenge_time    => "true" | "false",
```

```
#Optional / Default value is "false"
        override_update_dns_on_lease_renewal  => "true" | "false",
#Optional / Default value is "false"
        pxe_lease_time                => $num | undef,
#Optional / Default is undefined
        range_templates               => [$RangeTemplate1,$RangeTemplate2,...],
#Optional / Default is empty
        recycle_leases                => "true" | "false" | undef,
#Optional / Default is undefined
        rir_organization              => $rir_org,
#Optional / An Infoblox::Grid::RIR::Organization object
        rir_registration_action       => 'NONE' | 'CREATE',
#Optional / Default is 'NONE'
        rir_registration_status       => 'NOT_REGISTERED' | 'REGISTERED',
#Optional / Default is 'NOT_REGISTERED'
        send_rir_request              => 'true' | 'false',
#Optional / Default is 'false'
        update_dns_on_lease_renewal   => "true" | "false",
#Optional / Default value is "false"
        ipam_trap_settings             => $ipam_trap,
#Optional / An Infoblox::Grid::SNMP::IPAMTrap object
        override_ipam_trap_settings   => "true" | "false",
#Optional / Default value is "false"
        ipam_threshold_settings        => $ipam_threshold,
#Optional / An Infoblox::Grid::SNMP::IPAMThreshold object
        override_ipam_threshold_settings => "true" | "false",
#Optional / Default value is "false"
        ipam_email_addresses          => [$email1, $email2, ...],
#Optional / Default is an empty list
        override_ipam_email_addresses => "true" | "false",
#Optional / Default value is "false"
        logic_filters                 => [ $filter_name | $filter_obj, ...], # Optional / Default is
undefined
        override_logic_filters        => "true" | 'false",                  # Optional / Default value
is "false"
      );
```

You cannot set both extattrs and extensible_attributes attributes at the same time.

SESSION METHODS

This section describes all the methods in an Infoblox::Session module that you can apply to a DHCP Network Template object.

Infoblox::Session->add( )

Use this method to add a network template object to the Infoblox appliance. See Infoblox::Session->add() for parameters and return values.

Example

```
#Construct an object
my $network_template = Infoblox::DHCP::NetworkTemplate->new(
       name             => "custom_network_template",
       allow_any_netmask => "true",
);

# Submit for adding network template
my $response = $session->add( $network_template );
```

Infoblox::Session->get( )

Use this method to retrieve the matching network template objects from the Infoblox appliance. See Infoblox::Session->get() for parameters and return values.

Key References

```
Apply the following attributes to get a specific DHCP Network template object:

 name - Required. The name of network template.
 extattrs     - Optional. A hash reference containing extensible attributes.
```

Example

```
 my  @result_array = $session->get(
      object  => "Infoblox::DHCP::NetworkTemplate" ,
      name    => "custom_network_template"
 );
```

Infoblox::Session->search( )

Use this method to retrieve all the matching network template objects from the Infoblox appliance. See Infoblox::Session->search() for parameters and return values.

Key References

```
Apply the following attributes to search for a specific DHCP Network Template object:

 name    - Optional. The name of the network template (regular
           expression).
 comment - Optional. The comment of the network template (regular
           expression).
 extattrs     - Optional. A hash reference containing extensible attributes.
 extensible_attributes - Optional. A hash reference containing extensible attributes.
```

For more information about searching extensible attributes, see Infoblox::Grid::ExtensibleAttributeDef/Searching Extensible Attributes.

Example

```
my  @result_array = $session->search(
     object  => "Infoblox::DHCP::NetworkTemplate" ,
     name    => "custom_network_template"
 comment => ".*",
);

# search all Network Templates with a given extensible attribute 'Site'
my @retrieved_objs = $session->search(
  object => "Infoblox::DHCP::NetworkTemplate",
  extensible_attributes => { 'Site' => 'Santa Clara' });
```

Infoblox::Session->modify( )

Use this method to modify a network template object in the Infoblox appliance. See Infoblox::Session->modify() for parameters and return values.

Example

```
#Use method to modify enable_ddns method.
$object->enable_ddns("true");
# Submit modification
my $response = $session->modify( $object );
```

Infoblox::Session->remove( )

Use this method to remove an object from the Infoblox appliance. See Infoblox::Session->remove() for parameters and return values.

To remove a specifc object, first use `get()` or `search()` to retrieve the specific object, and then submit this object for removal.

Example

```
# Get the object under the same name
my  @result_array = $session->get(

      object => "Infoblox::DHCP::NetworkTemplate" ,
      name   => "custom_network_template"
            );

# find the desired object from retrieved list.
my $desired_network_template = $result_array[0];

# Submit for removal
my $response = $session->remove( $desired_network_template );
```

METHODS

This section describes all the methods that can be used to configure and retrieve the attribute values of a DHCP Network template object

allow_any_netmask( )

Use this method to set or retrieve the allow_any_netmask parameter of a DHCP network template object. If you set this parameter to "true", then the template allows any netmask. A user must specify a netmask when creating a network using this template. If you set this parameter to false, you must specify the "netmask" property for the network template object.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

Specify "true" to set the allow_any_netmask parameter or specify "false" if you want to specify a particular netmask. The default value is false.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
# Get allow_any_netmask
my $allow_any_netmask = $network_template->allow_any_netmask()
# Modify allow_any_netmask
$network_template->allow_any_netmask("true");
```

authority( )

Use this method to set or retrieve the authority flag of a DHCP network template object. This flag specifies if a DHCP server is authoritative for the network.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

Specify "true" to set the authority flag or "false" to deactivate/unset it. The default value is undefined which indicates that this attribute inherits the upper-level setting.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
# Get authority
my $authority = $network_template->authority()
# Modify authority
$network_template->authority("true");
```

auto_create_reversezone( )

Use this method to set or retrieve the auto_create_reversezone flag of a DHCP network template object. This flag indicates whether the appliance must automatically create the corresponding reverse zone.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

Specify "true" to create a reverse zone automatically or "false" to disable the automatic creation of a reverse zone.The default value for this field is false.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
# Get auto_create_reversezone
my $auto_create_reversezone = $network_template->auto_create_reversezone();
# Modify auto_create_reversezone
$network_template->auto_create_reversezone("true");
```

bootfile( )

Use this method to set or retrieve the bootfile attribute of a DHCP network template object. You can configure the DHCP server to support clients that use the boot file name option in their DHCPREQUEST messages. The default value is undefined which indicates that this attribute inherits the upper-level setting.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

This option is overridden independently from Infoblox::DHCP::NetworkTemplate->bootserver() and Infoblox::DHCP::NetworkTemplate->nextserver().

Parameter

The name, in string format, of the file that the client must download. The default value is undefined.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
# Get bootfile
my $bootfile = $network_template->bootfile();
# Modify bootfile
$network_template->bootfile("bootfile1");
#Un-override bootfile
$network_template->bootfile(undef);
```

bootserver( )

Use this method to set or retrieve the boot server attribute of a DHCP network template object. You can specify the name of the boot server on which the boot file is stored. The default value is undefined which indicates that this attribute inherits the upper-level setting.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

This option is overridden independently from Infoblox::DHCP::NetworkTemplate->bootfile() and Infoblox::DHCP::NetworkTemplate->nextserver().

Parameter

The next server IP address or name in FQDN (Fully Qualified Domain Name) format. The FQDN consists of the host name followed by the domain name (example: abc.com). A next server name can have a maximum of 256 bytes. The default value is undefined.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
# Get bootserver
my $bootserver = $network_template->bootserver();
# Modify bootserver
$network_template->bootserver("abc.domain.com");
#Un-override bootserver
$network_template->bootserver(undef);
```

comment( )

Use this method to set or retrieve a descriptive comment about a DHCP network template object.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

Enter a descriptive comment for the network template in string format with a maximum of 256 bytes.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
# Get comment
my $comment = $network_template->comment();
# Modify comment
$network_template->comment("add a network");
```

ddns_domainname( )

Use this method to set or retrieve the ddns_domainname value of a DHCP Network template object.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Setting this method to a defined value implicitly sets the override_ddns_domainname method to "true". Setting the parameter to undefined causes the appliance to use the grid default and automatically resets the override_ddns_domainname attribute to "false".

Note that when ddns_domainname is set to a defined value and override_dddns_domainname is set to "false", the last operation takes precedence. Thus the sequence $object->ddns_domainname("testdomain"); $object->override_ddns_domainname("false"); will set override_ddns_domainname to "false", and the sequence $object->override_ddns_domainname("false"); $object->ddns_domainname("testdomain"); will result in override_ddns_domainname="true".

Parameter

The dynamic DNS domain name in string format.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get ddns_domainname
my $ddns_domainname = $network_template->ddns_domainname();
#Modify ddns_domainname
$network_template->ddns_domainname("test_domain.com");
```

ddns_generate_hostname( )

Use this method to set or retrieve the ddns_generate_hostname flag of a DHCP network template object. Set this flag to allow only the DHCP server to generate a host name. This setting overrides the upper-level settings.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

Specify "true" to generate hostname or "false" to deactivate/unset it. The default value is undefined which indicates that this attribute inherits the upper-level setting.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
# Get ddns_generate_hostname
my $ddns_generate_hostname = $network_template->ddns_generate_hostname()
# Modify ddns_generate_hostname
$network_template->ddns_generate_hostname("true");
#Un-override
$network_template->ddns_generate_hostname(undef);
```

ddns_update_fixed_addresses( )

Use this method to set or retrieve the ddns_update_fixed_addresses flag of a DHCP network template object. By default, the DHCP server does not update DNS when it allocates a fixed address to a client. You can configure the DHCP server to update the A and PTR records of a client with a fixed address. When this feature is enabled and the DHCP server adds A and PTR records for a fixed address, the DHCP server never discards the records. This setting overrides the upper-level settings.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

Specify "true" to update DNS when it allocates a fixed address to a client or "false" to deactivate/unset it. The default value is undefined which indicates that this attribute inherits the upper-level setting.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
# Get ddns_update_fixed_addresses
my $ddns_update_fixed_addresses= $network_template->ddns_update_fixed_addresses()
# Modify ddns_update_fixed_addresses
$network_template->ddns_update_fixed_addresses("true");
#Un-override
$network_template->ddns_update_fixed_addresses(undef);
```

ddns_server_always_updates( )

Use this method to set or retrieve the ddns_server_always_updates flag of a DHCP network template object. Set this flag to allow the DHCP server only to update DNS, regardless of the requests from DHCP clients.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

Specify "true" to set this flag and allow the DHCP server only to update DNS or "false" to deactivate/unset the flag. The default value is true.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
# Get ddns_server_always_updates
my $ddns_server_always_updates = $network_template->ddns_server_always_updates()
# Modify ddns_server_always_updates
$network_template->ddns_server_always_updates("true");
```

ddns_ttl( )

Use this method to set or retrieve the DNS update Time to Live (TTL) value of a DHCP network template object. The TTL value indicates the number of seconds the Infoblox appliance caches a DDNS update.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Setting this method to a defined value implicitly sets the override_ddns_ttl method to "true". Setting the parameter to undefined causes the appliance to use the grid default and automatically resets the override_ddns_ttl attribute to "false".

Note that when ddns_ttl is set to a defined value and override_dddns_ttl is set to "false", the last operation takes precedence. Thus the sequence $object->ddns_ttl(12); $object->override_ddns_ttl("false"); will set override_ddns_ttl to "false", and the sequence $object->override_ddns_ttl("false"); $object->ddns_ttl(12); will result in override_ddns_ttl="true".

Parameter

A 32-bit integer (range from 0 to 4294967295) that represents the duration, in seconds, that the update is cached. Zero indicates that the update should not be cached. The default value is 0.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get DNS Update ttl
my $dns_ttl = $network_template->ddns_ttl();
#Modify DNS Update ttl
$network_template->ddns_ttl(1200);
```

ddns_use_option81( )

Use this method to set or retrieve the ddns_use_option81 flag of a DHCP network template object. Enable this method to define the settings for option 81 at the network template level. This setting overrides the upper-level settings.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

Specify "true" to enable support for option 81 or "false" to disable support. The default value is undefined which indicates that this attribute inherits the upper-level setting.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
# Get ddns_use_option81
my $ddns_use_option81 = $network_template->ddns_use_option81()
# Modify ddns_use_option81
```

```
$network_template->ddns_use_option81("true");
# un-override ddns_use_option81
$network_template->ddns_use_option81(undef);
```

deny_bootp( )

Use this method to set or retrieve the deny_bootp flag of a DHCP Network object.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

Specify "true" to set the deny_bootp flag or "false" to deactivate/unset it. The default value is undefined which indicates that this attribute inherits the upper-level setting.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get deny_bootp
my $deny_bootp = $network->deny_bootp();
#Modify deny_bootp
$network->deny_bootp("true");
```

email_list( )

Use this method to set or retrieve the e-mail lists to which the appliance sends threshold e-mail messages.

If you enter e-mail addresses in the email_list, then the appliance sends DHCP threshold alarms to the e-mail addresses.

Parameter

The valid value is an array reference that contains string objects.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get email_list
my @email_list = $network_template->email_list();
#Modify email_list
$network_template->email_list( [ "admin\@test.com" ] );
```

enable_ddns( )

Use this method to set or retrieve the dynamic DNS updates flag of a DHCP network template object. The DHCP server can send DDNS updates to DNS servers in the same grid and to external DNS servers. This setting overrides the upper-level settings.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

Specify "true" to set the dynamic DNS updates flag or "false" to deactivate/unset it. The default value is undefined which indicates that this attribute inherits the upper-level setting.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
# Get enable_ddns
my $enable_ddns = $network_template->enable_ddns()
# Modify enable_ddns
$network_template->enable_ddns("true");
# Un-override enable_ddns
$network_template->enable_ddns(undef);
```

enable_dhcp_thresholds( )

Use this method to set or retrieve the DHCP thresholds of a network template object. These thresholds are the high and low watermarks. If DHCP range usage goes above or below these percentage values, the Infoblox appliance generates a syslog message and sends an SNMP notification.

Enable this flag to set DHCP threshold values.

Parameter

Specify "true" to set the enable_dhcp_thresholds flag or "false" to deactivate/unset it. The default value is undefined.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get enable_dhcp_thresholds
my $enable_dhcp_thresholds = $network_template->enable_dhcp_thresholds();
#Modify enable_dhcp_thresholds
$network_template->enable_dhcp_thresholds("false");
```

enable_email_warnings( )

Use this method to send DHCP thresholds warnings through e-mail.

Enable this flag if you want the appliance to send DHCP threshold warnings to the "email_list" provided when DHCP threshold is enabled.

Parameter

+Specify "true" to set the enable_email_warnings flag or "false" to deactivate/unset it. The default value is undefined.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get enable_email_warnings
my $enable_email_warnings = $network_template->enable_email_warnings();
#Modify enable_email_warnings
$network_template->enable_email_warnings("false");
```

enable_snmp_warnings( )

Use this method to send DHCP threshold warnings through SNMP.

If the flag is enabled, then the appliance sends DHCP threshold warnings through SNMP messages when DHCP threshold is enabled.

Parameter

Specify "true" to set the enable_snmp_warnings flag or "false" to deactivate/unset it. The default value is undefined.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get enable_snmp_warnings
my $enable_snmp_warnings = $network_template->enable_snmp_warnings();
#Modify enable_snmp_warnings
$network_template->enable_snmp_warnings("false");
```

extattrs( )

Use this method to set or retrieve the extensible attributes associated with a Network Template object.

Parameter

Valid value is a hash reference containing the names of extensible attributes and their associated values (Infoblox::Grid::Extattr

objects).

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get extattrs
my $ref_extattrs = $network_template->extattrs();
#Modify extattrs
$network_template->extattrs({ 'Site' => $extattr1, 'Administrator' => $extattr2 });
```

extensible_attributes( )

Use this method to set or retrieve the extensible attributes associated with a Network Template object.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

For valid values for extensible attributes, see Infoblox::Grid::ExtensibleAttributeDef/Extensible Attribute Values.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get extensible attributes
my $ref_extensible_attributes = $network_template->extensible_attributes();
#Modify extensible attributes
$network_template->extensible_attributes({ 'Site' => 'Santa Clara', 'Administrator' => [ 'Peter',
'Tom' ] });
```

fixed_address_templates( )

Use this method to set or retrieve the fixed address template assigned to this network template object. When you create a network based on a network template object with a fixed address template, the fixed addresses are created based on the associated fixed address template.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

The valid value is an array reference of Infoblox::DHCP::Template objects.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
# Get fixed_address_templates
my $fixed_address_templates = $network_template->fixed_address_templates();
# Modify fixed_address_templates
my $tmpl = Infoblox::DHCP::Template->new(
   name => "custom_fixed_address_template"
);
$network_template->fixed_address_templates([$tmpl]);
```

high_water_mark( )

Use this method to set the percentage value for DHCP range usage above which the Infoblox appliance generates a syslog message and sends a warning.

When the percent of allocated addresses in a DHCP range exceeds the high_water_mark, the appliance generates a syslog message and sends a warning.

Parameter

The valid value is in string format.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get high_water_mark
my $high_water_mark = $network_template->high_water_mark();
#Modify high_water_mark
$network_template->high_water_mark("80");
```

high_water_mark_reset( )

Use this method to set or retrieve the high_water_mark_reset value. If the percentage of allocated addresses drops below this value, the corresponding SNMP trap is reset.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

A number that specifies the percentage of allocated addresses. The range is from 1 to 100. The high watermark reset value must be lower than the high water mark value. The default value is 95.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get high_water_mark_reset
my $high_water_mark_reset = $network_template->high_water_mark_reset();
#Modify high_water_mark_reset
$network_template->high_water_mark_reset(70);
```

ignore_dhcp_option_list_request( )

Use this method to set or retrieve the ignore_dhcp_option_list_request flag of a DHCP network template object. Set this flag to ignore option lists requested by clients and return all defined options at the network template level.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

Specify "true" to set the ignore_dhcp_option_list_request flag or "false" to deactivate/unset it. The default value is empty.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
# Get ignore_dhcp_option_list_request
my $ignore_dhcp_option_list_request = $network_template->ignore_dhcp_option_list_request();
# Modify ignore_dhcp_option_list_request
$network_template->ignore_dhcp_option_list_request("false");
# Un-override ignore_dhcp_option_list_request
$network_template->ignore_dhcp_option_list_request(undef);
```

ipam_email_addresses( )

Use this method to set or retrieve a list of IPAM Utilization email addresses.

Setting this method to a defined value implicitly sets the override_ipam_email_addresses method to "true". Setting the parameter to undefined causes the appliance to use the default from the upper level and automatically resets the override_ipam_email_addresses attribute to "false".

Note that when ipam_email_addresses is set to a defined value and override_ipam_email_addresses is set to "false", the last operation takes precedence. Thus the sequence $object->ipam_email_addresses(['admin@infoblox.com']); $object->override_ipam_email_addresses("false"); will set override_ipam_email_addresses to "false", and the sequence $object->override_ipam_email_addresses("false"); $object->ipam_email_addresses(['admin@infoblox.com']); will result in override_ipam_email_addresses="true".

Parameter

Valid value is an array reference that contains email addresses in string format.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get ipam_email_addresses
my $ipam_email_addresses = $network_template->ipam_email_addresses();
#Modify ipam_email_addresses
$network_template->ipam_email_addresses($ipam_email_addresses);
```

ipam_trap_settings( )

Use this method to set or retrieve IPAM Utilization SNMP Trap settings.

Setting this method to a defined value implicitly sets the override_ipam_trap_settings method to "true". Setting the parameter to undefined causes the appliance to use the default from the upper level and automatically resets the override_ipam_trap_settings attribute to "false".

Note that when ipam_trap_settings is set to a defined value and override_ipam_trap_settings is set to "false", the last operation takes precedence. Thus the sequence $object->ipam_trap_settings($ipam_trap); $object->override_ipam_trap_settings("false"); will set override_ipam_trap_settings to "false", and the sequence $object->override_ipam_trap_settings("false"); $object->ipam_trap_settings($ipam_trap); will result in override_ipam_trap_settings="true".

Parameter

Valid value is an Infoblox::Grid::SNMP::IPAMTrap object.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get ipam_trap_settings
my $ipam_trap_settings = $network_template->ipam_trap_settings();
#Modify ipam_trap_settings
$network_template->ipam_trap_settings($ipam_trap_settings);
```

ipam_threshold_settings( )

Use this method to set or retrieve IPAM Utilization SNMP Threshold settings.

Setting this method to a defined value implicitly sets the override_ipam_threshold_settings method to "true". Setting the parameter to undefined causes the appliance to use the default from the upper level and automatically resets the override_ipam_threshold_settings attribute to "false".

Note that when ipam_threshold_settings is set to a defined value and override_ipam_threshold_settings is set to "false", the last operation takes precedence. Thus the sequence $object->ipam_threshold_settings($ipam_threshold_settings); $object->override_ipam_trap_settings("false"); will set override_ipam_threshold_settings to "false", and the sequence $object->override_ipam_threshold_settings("false"); $object->ipam_threshold_settings($ipam_threshold_settings); will result in override_ipam_threshold_settings="true".

Parameter

Valid value is an Infoblox::Grid::SNMP::IPAMThreashold object.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get ipam_threshold_settings
my $ipam_threshold_settings = $network_template->ipam_threshold_settings();
#Modify ipam_threshold_settings
$network_template->ipam_threshold_settings($ipam_threshold_settings);
```

lease_scavenge_time( )

Use this method to set or retrieve the lease_scavenge_time value of a Network Template object.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Setting this method to a defined value implicitly sets the override_lease_scavenge_time method to "true". Setting the parameter to undefined causes the appliance to use the default from the upper level and automatically resets the override_lease_scavenge_time attribute to "false".

Note that when lease_scavenge_time is set to a defined value and override_lease_scavenge_time is set to "false", the last operation takes precedence. Thus the sequence $object->lease_scavenge_time(604800); $object->override_lease_scavenge_time("false"); will set override_lease_scavenge_time to "false", and the sequence $object->override_lease_scavenge_time("false"); $object->lease_scavenge_time(604800); will result in override_lease_scavenge_time="true".

Parameter

An integer that specifies the period of time, in seconds, that free and backup leases remain in the database before they are automatically deleted. To disable lease scavenging, set the parameter to -1. The minimum positive value must be greater than 86400 seconds (1 day). The default value of this parameter is -1, which means that lease scavenging is disabled.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get lease_scavenge_time
my $lease_scavenge = $network_template->lease_scavenge_time();
#Modify lease_scavenge_time
$network_template->lease_scavenge_time(30 * 24 * 60 * 60);
```

logic_filters( )

Use this method to set or retrieve all the logic (ordered) filters associated with the DHCP Network Template.

Filters are used to screen address assignments in the DHCP Network Template if the addresses are to be assigned to specific hosts.

Setting this method to a defined value implicitly sets the override_logic_filters method to "true". Setting the parameter to undefined causes the appliance to use the grid default and automatically resets the override_logic_filters attribute to "false".

Note that when logic_filters is set to a defined value and override_dlogic_filters is set to "false", the last operation takes precedence. Thus the sequence $object->logic_filters($filters); $object->override_logic_filters("false"); will set override_logic_filters to "false", and the sequence $object->override_logic_filters("false"); $object->logic_filters($filters); will result in override_logic_filters="true".

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

The valid value is an array reference that contains either the Infoblox::DHCP::Filter::MAC, Infoblox::DHCP::Filter::NAC, Infoblox::DHCP::Filter::Option objects or the names of the corresponding filters. The filters in the list must satisfy the following conditions:

Filter name must be globally unique.
Filters without expressions must be at the end of the list.
The filters should not be disabled.

The default value of the parameter is undefined.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get logic_filters
my $ref_logic_filters = $network_template->logic_filters();
#Modify logic_filters
$network_template->logic_filters(["filter_1","filter_2",$exl1]); #$exl1 is an
Infoblox::DHCP::Filter::MAC, Infoblox::DHCP::Filter::NAC or Infoblox::DHCP::Filter::Option object.
```

low_water_mark( )

Use this method to set the percentage value for DHCP range usage below which the Infoblox appliance generates a syslog message and

sends a warning.

When the percent of allocated addresses in a DHCP range goes below the low_water_mark, the appliance generates a syslog message and sends a warning.

Parameter

The valid value is in string format.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get low_water_mark
my $low_water_mark = $network_template->low_water_mark();
#Modify low_water_mark
$network_template->low_water_mark("10");
```

low_water_mark_reset( )

Use this method to set or retrieve the low_water_mark_reset value. If the percentage of allocated addresses exceeds this value, a corresponding SNMP trap is reset.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

A number that specifies the percentage of allocated addresses. The range is from 1 to 100. The low watermark reset value must be higher than the low watermark value. The default low watermark reset number is 0.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get low_water_mark_reset
my $low_water_mark_reset = $network_template->low_water_mark_reset();
#Modify low_water_mark_reset
$network_template->low_water_mark_reset(10);
```

members( )

Use this method to set or retrieve the member or Microsoft (r) server that serves DHCP for the network created from the network template object.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

The valid value is an array reference that contains Infoblox::DHCP::Member or Infoblox::DHCP::MSServer objects.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
my $member1 = Infoblox::DHCP::Member->new(
    name     => "infoblox.localdomain",
    ipv4addr => "192.168.1.2",
);

# Get members
my $members = $network_template->members();
# Modify members
$network_template->members([$member1]);
```

name( )

Use this method to set or retrieve the description of the name of the DHCP network template.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

Name of the network template in string format with a maximum of 64 bytes.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get name
my $name = $network_template->name();
#Modify name
$network_template->name("custom_network_template");
```

netmask( )

Use this method to set or retrieve the netmask attribute of a DHCP network template object. If "allow_any_netmask" is set to "false", then this attribute must be specified.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

A valid subnet mask. The default value is "255.0.0.0".

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get netmask
my $netmask = $network_template->netmask();
#Set netmask
$nework_template->allow_any_netmask( "false" );
my $netmask = $network_template->netmask("255.0.0.0");
```

nextserver( )

Use this method to set or retrieve the nextserver attribute of a DHCP network template object. You can specify the name and/or IP address of the next server to which the host needs to connect in order to boot. This setting overrides the upper-level settings. The default value is undefined which indicates that this attribute inherits the upper-level setting.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

This option is overridden independently from Infoblox::DHCP::NetworkTemplate->bootfile() and Infoblox::DHCP::NetworkTemplate->bootserver().

Parameter

The next server IP address or name in FQDN (Fully Qualified Domain Name) format. The FQDN consists of the host name followed by the domain name (example: abc.com). A next server name can have a maximum of 256 bytes. The default value is undefined.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
# Get nextserver
my $nextserver = $network_template->nextserver();
# Modify nextserver
$network_template->nextserver("blue.domain.com");
#Un-override nextserver
$network_template->nextserver(undef);
```

options( )

Use this method to set or retrieve the options value of a network template object. DHCP options provide network configuration settings and various services available on the network for DHCP. This setting overrides the upper-level setting. The default value is undefined which indicates that this attribute inherit the upper-level setting.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

The valid value is an array reference that contains Infoblox::DHCP::Option objects.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get options
my $options = $network_template->options();
#Modify options
my $option1 = Infoblox::DHCP::Option->new(
    name  => "lease-time",
    value => 50
);

$network_template->options([$option1]);
```

override_ddns_domainname( )

The override_ddns_domainname attribute controls whether the ddns_domainname value in the DHCP network template object is used, instead of the grid default.

The override_ddns_domainname attribute can be specified explicitly. It is also set implicitly when ddns_domainname is set to a defined value.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

Set the parameter to "true" to override the grid-level setting for ddns_domainname. Set the parameter to "false" to inherit the grid-level setting for ddns_domainname.

The default value of this parameter is "false".

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Getting override_ddns_domainname
my $override_ddns_domainname=$network_template->override_ddns_domainname( );
#Modifying override_ddns_domainname
$network_template->override_ddns_domainname("true");
```

override_ddns_ttl( )

The override_ddns_ttl attribute controls whether the ddns_ttl value in the DHCP network template object is used, instead of the grid default.

The override_ddns_ttl attribute can be specified explicitly. It is also set implicitly when ddns_ttl is set to a defined value.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

Set the parameter to "true" to override the grid-level setting for ddns_ttl. Set the parameter to "false" to inherit the grid-level setting for ddns_ttl.

The default value of this parameter is "false".

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Getting override_ddns_ttl
my $override_ddns_ttl=$network_template->override_ddns_ttl( );
#Modifying override_ddns_ttl
$network_template->override_ddns_ttl("true");
```

override_ipam_email_addresses( )

The override_ipam_email_addresses controls whether the ipam_email_addresses in the DHCP network template is used, instead of the upper-level default.

The override_ipam_email_addresses can be specified explicitly. It is also set implicitly when ipam_email_addresses is set to a defined value.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

Set the parameter to "true" to override the upper-level setting for ipam_email_addresses. Set the parameter to "false" to inherit the upper-level setting for ipam_email_addresses.

The default value of this parameter is "false".

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get attrubute value
my $value=$network_template->override_ipam_email_addresses();
#Modifying attribute value
$network_template->override_ipam_email_addresses('true');
```

override_ipam_threshold_settings( )

The override_ipam_threshold_settings controls whether the ipam_threshold_settings in the DHCP network template is used, instead of the upper-level default.

The override_ipam_threshold_settings can be specified explicitly. It is also set implicitly when ipam_threshold_settings is set to a defined value.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

Set the parameter to "true" to override the upper-level setting for ipam_threshold_settings. Set the parameter to "false" to inherit the upper-level setting for ipam_threshold_settings.

The default value of this parameter is "false".

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get attrubute value
my $value=$network_template->override_ipam_threshold_settings();
#Modifying attribute value
$network_template->override_ipam_threshold_settings('true');
```

override_ipam_trap_settings( )

The override_ipam_trap_settings controls whether the ipam_trap_settings in the DHCP network template is used, instead of the upper-level default.

The override_ipam_trap_settings can be specified explicitly. It is also set implicitly when ipam_trap_settings is set to a defined value.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

Set the parameter to "true" to override the upper-level setting for ipam_trap_settings. Set the parameter to "false" to inherit the upper-level setting for ipam_trap_settings.

The default value of this parameter is "false".

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get attrubute value
my $value=$network_template->override_ipam_trap_settings();
#Modifying attribute value
$network_template->override_ipam_trap_settings('true');
```

override_lease_scavenge_time( )

The override_lease_scavenge_time attribute controls whether the lease_scavenge_time value in the network template is used, instead of the upper-level default.

The override_lease_scavenge_time attribute can be specified explicitly. It is also set implicitly when lease_scavenge_time is set to a defined value.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

Set the parameter to "true" to override the upper-level setting for lease_scavenge_time. Set the parameter to "false" to inherit the setting for lease_scavenge_time from the upper level.

The default value of this parameter is "false".

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Getting override_lease_scavenge_time
my $override_lease_scavenge_time=$network_template->override_lease_scavenge_time( );
#Modifying override_lease_scavenge_time
$network_template->override_lease_scavenge_time("true");
```

override_logic_filters( )

The override_logic_filters attribute controls whether the logic_filters value of the DHCP Network Template is used, instead of the Grid default.

The override_logic_filters attribute can be specified explicitly. It is also set implicitly when logic_filters is set to a defined value.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

Set the parameter to "true" to override the Grid-level setting for logic_filters. Set the parameter to "false" to inherit the Grid-level setting for logic_filters.

The default value of this parameter is "false".

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Getting override_logic_filters
my $override_logic_filters=$network_template->override_logic_filters( );
#Modifying override_logic_filters
$network_template->override_logic_filters("true");
```

override_update_dns_on_lease_renewal( )

The override_update_dns_on_lease_renewal attribute controls whether the update_dns_on_lease_renewal value in the DHCP network template object is used, instead of the grid default.

The override_update_dns_on_lease_renewal attribute can be specified explicitly. It is also set implicitly when update_dns_on_lease_renewal is set to a defined value.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

Set the parameter to "true" to override the grid-level setting for update_dns_on_lease_renewal. Set the parameter to "false" to inherit the grid-level setting for update_dns_on_lease_renewal.

The default value of this parameter is "false".

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Getting override_update_dns_on_lease_renewal
my $override_update_dns_on_lease_renewal=$network_template->override_update_dns_on_lease_renewal(
);
#Modifying override_update_dns_on_lease_renewal
$network_template->override_update_dns_on_lease_renewal("true");
```

pxe_lease_time( )

Use this method to set or retrieve the PXE lease time value of a DHCP network template object. Some hosts use PXE (Preboot Execution Environment) to boot remotely from a server. To better manage your IP resources, set a different lease time for PXE boot requests. You can configure the DHCP server to allocate an IP address with a shorter lease time to hosts that send PXE boot requests, so IP addresses are not leased longer than necessary. This setting overrides the upper-level settings. The default value is undefined, which indicates that this attribute inherits the upper-level setting.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

A 32-bit integer (range from 0 to 4294967295) that represents the duration, in seconds, that the update is cached. Zero indicates that the update should not be cached. The default value is undefined

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get PXE Lease Time
my $pxe_lease_time = $network_template->pxe_lease_time();
#Modify PXE Lease Time
$network_template->pxe_lease_time(1100);
#Un-override PXE Lease Time
$network_template->pxe_lease_time(undef);
```

range_templates( )

Use this method to set or retrieve the IP address range template assigned to this network template object. When you create a network based on a network template object with a range template, the IP address ranges are created based on the associated IP address range template.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

The valid value is an array reference of Infoblox::DHCP::Template objects.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get range_templates
my $range_templates = $network_template->range_templates();
#Modify range_templates
my $tmpl = Infoblox::DHCP::Template->new(
   name => "custom_range_template"
);
$network_template->range_templates([ $tmpl ]);
```

update_dns_on_lease_renewal( )

The update_dns_on_lease_renewal attribute controls whether the DHCP server updates DNS when a DHCP lease is renewed.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Setting this method to a defined value implicitly sets the override_update_dns_on_lease_renewal method to "true". Setting the parameter to undefined causes the appliance to use the grid default and automatically resets the override_update_dns_on_lease_renewal attribute to "false".

Note that when update_dns_on_lease_renewal is set to a defined value and override_update_dns_on_lease_renewal is set to "false", the last operation takes precedence. Thus the sequence $object->update_dns_on_lease_renewal("true"); $object->override_update_dns_on_lease_renewal("false"); will set override_update_dns_on_lease_renewal to "false", and the sequence $object->override_update_dns_on_lease_renewal("false"); $object->update_dns_on_lease_renewal("true"); will result in override_update_dns_on_lease_renewal="true".

Parameter

Specify "true" to set the update_dns_on_lease_renewal flag or "false" to deactivate/unset it.

The default value of this parameter is "false".

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get update_dns_on_lease_renewal
my $update_dns_on_lease_renewal=$network_template->update_dns_on_lease_renewal( );
#Modify update_dns_on_lease_renewal
$network_template->update_dns_on_lease_renewal("true");
```

recycle_leases( )

Use this method to set or retrieve the recycle_leases flag at the network template level.

If the flag is enabled, the leases are kept in the recycle bin until one week after expiration. When the flag is disabled, the leases are irrecoverably deleted.

Parameter

Specify "true" to enable the recycle_leases flag or "false" to disable it. The default value is undefined, which indicates that this attribute inherit the upper-level setting.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get recycle_leases
my $recycle_leases = $network_template->recycle_leases();
#Modify recycle_leases
$network_template->recycle_leases("false");
```

rir( )

Use this method to retrieve the name of the Regional Internet Registry (RIR) that allocated this address space. This is a read-only attribute.

Parameter

None

Returns

The method returns the attribute value that can be 'RIPE' or 'NONE'.

Example

```
#Get rir
my $rir = $network_template->rir();
```

rir_organization( )

Use this method to set or retrieve the Regional Internet Registry (RIR) organization object.

Parameter

An Infoblox::Grid::RIR::Organization object.

You must enable support for RIR/SWIP at the Grid level before you can set attribute values. See Infoblox::Grid->enable_rir_swip() method.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get rir_organization
my $rir_organization = $network_template->rir_organization();
#Modify rir_organization
$network_template->rir_organization($rir_org);
```

rir_registration_action( )

Use this method to set or retrieve the registration action of Regional Internet Registry (RIR).

Parameter

Valid values are 'NONE', 'CREATE'.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get rir_registration_action
my $rir_registration_action = $network_template->rir_registration_action();
#Modify rir_registration_action
$network_template->rir_registration_action('CREATE');
```

rir_registration_status( )

Use this method to set or retrieve the registration status of Regional Internet Registry (RIR).

Parameter

The registration status of Regional Internet Registry (RIR), which can be 'REGISTERED' or 'NOT_REGISTERED'.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get rir_registration_status
my $rir_registration_status = $network_template->rir_registration_status();
#Modify rir_registration_status
$network_template->rir_registration_status('NOT_REGISTERED');
```

send_rir_request( )

Use this method to set or retrieve the flag that indicates whether the appliance should send an API or email request for Regional Internet Registry (RIR).

Parameter

Specify "true" to set the send_rir_request flag or "false" to unset it.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get send_rir_request
my $send_rir_request = $network_template->send_rir_request();
#Modify send_rir_request
$network_template->send_rir_request('true');
```

SAMPLE CODE

The following sample code demonstrates the different functions that can be applied to an object such as add, search, modify, and remove. Also, this sample code includes error handling for the operations.

#Preparation prior to a DHCP network template insertion

```
#PROGRAM STARTS: Include all the modules that will be used

use strict;
use Infoblox;

#Create a session to the Infoblox appliance
my $host_ip = "192.168.1.2";
my $host_name = "infoblox.localdomain";

my $session = Infoblox::Session->new(
    master   => $host_ip,
    username => "admin",
    password => "infoblox"
);
unless ($session) {
   die("Construct session failed: ",
       Infoblox::status_code() . ":" . Infoblox::status_detail());
}
print "Session created successfully\n";

my $memberdhcp = Infoblox::DHCP::Member->new(
            name     => $host_name,
            ipv4addr => $host_ip
           );
```

#Add network template object

```
#Create a DHCP Network template object

my $network_template = Infoblox::DHCP::NetworkTemplate->new(
                               name                    => "custom_network_template",
                               netmask                 => "255.0.0.0",
                               comment                 => "add network template",
                               members                 => [ $memberdhcp ],
                               authority               => "true",
                               enable_ddns             => "true",
                               pxe_lease_time          => "3600",
                               bootserver              => "someserver.com",
                               nextserver              => "3.3.3.3",
                               bootfile                => "bootfile.com",
                               auto_create_reversezone => "true",
                               );

my $response = $session->add($network_template)
       or die("Add Network template failed: ",
             $session->status_code() . ":" . $session->status_detail());

print "Network template added successfully\n";
```

#Get and modify a network template attribute

```
#Get Network template through the session
my  @retrieved_objs = $session->get(
```

```
            object => "Infoblox::DHCP::NetworkTemplate",
            name   => "custom_network_template"
        );

 my $object = $retrieved_objs[0];

 unless ($object) {
     die("Get Network template object failed: ",
         $session->status_code() . ":" . $session->status_detail());
 }
 print "Get Network template found at least 1 matching entry\n";
```

#Search a network template attribute

```
 #Search Network template through the session
 my  @retrieved_objs = $session->search(
            object => "Infoblox::DHCP::NetworkTemplate",
            name   => "custom_network_template"
        );

 my $object = $retrieved_objs[0];

 unless ($object) {
     die("Search Network template object failed: ",
         $session->status_code() . ":" . $session->status_detail());
 }
 print "Search Network template found at least 1 matching entry\n";

 #Modify attributes of the specified Network Template

 $object->enable_ddns("true"); #Apply the changes

 #Un-override bootfile
 $object->bootfile(undef);

 #Un-override bootserver
 $object->bootserver(undef);

 #Un-override nextserver
 $object->nextserver(undef);

 $session->modify($object)
     or die("Modify Network template attribute failed: ",
            $session->status_code() . ":" . $session->status_detail());

 print "Network template object modified successfully \n";
```

#Remove a network template object

```
 #Get Network Template object through the session

  my @retrieved_objs = $session->get(
            object  => "Infoblox::DHCP::NetworkTemplate",
            name    => "custom_network_template"
       );

    my $object = $retrieved_objs[0];

    unless ($object) {
     die("Get Network Template object failed: ",
         $session->status_code() . ":" . $session->status_detail());
     }

 print "Get Network Template object found at least 1 matching entry\n";

 #Submit the object for removal

 $session->remove($object)
     or die("Remove Network Template failed: ",
         $session->status_code() . ":" . $session->status_detail());

 print "Network Template removed successfully \n";

 ####PROGRAM ENDS####
```

AUTHOR

Infoblox Inc. http://www.infoblox.com/

SEE ALSO

Infoblox::Session, Infoblox::DHCP::Member, Infoblox::DHCP::Option, Infoblox::DHCP::Network, Infoblox::DHCP::RangeTemplate, Infoblox::DHCP::FixedAddrTemplate, Infoblox::DHCP::Template, Infoblox::Session->get(), Infoblox::Session->search(), Infoblox::Session->add(), Infoblox::Session->remove(), Infoblox::Session->modify()

COPYRIGHT

Copyright (c) 2017 Infoblox Inc.

Infoblox::DHCP::NetworkUser - a DHCP Network User object.

- NAME
- DESCRIPTION
- CONSTRUCTOR
- SESSION METHODS
  - Infoblox::Session->add( )
  - Infoblox::Session->get( )
  - Infoblox::Session->modify( )
  - Infoblox::Session->remove( )
  - Infoblox::Session->search( )
- METHODS
  - address( )
  - address_object( )
  - data_source( )
  - data_source_ip( )
  - domainname( )
  - guid( )
  - last_seen_time( )
  - last_updated_time( )
  - first_seen_time( )
  - logon_id( )
  - logout_time( )
  - name( )
  - network( )
  - network_view( )
  - user_status( )
- AUTHOR
- SEE ALSO
- COPYRIGHT

NAME

Infoblox::DHCP::NetworkUser - a DHCP Network User object.

DESCRIPTION

The DHCP Network User provides information about Active Directory users such as user's session at a specific IP address, domain, logon and logoff timestamps.

CONSTRUCTOR

```
my $network_user = Infoblox::DHCP::NetworkUser->new(
   address           => $ipaddr,     # Required
   domainame         => $str,        # Required
   first_seen_time   => $epoch,      # Required
   name              => $str,        # Required
   network_view      => $view_name,  # Required
   logon_id          => $str,        # Required
   guid              => $str,        # Required
   logout_time       => $epoch,      # Optional
   last_seen_time    => $epoch,      # Optional
);
```

SESSION METHODS

This section describes all the methods in a Session module that you can apply to a DHCP Network User object.

Infoblox::Session->add( )

Use this method to add an object to the Infoblox appliance. See Infoblox::Session->add() for parameters and return values.

Example

```
#construct an object
my $network_user = Infoblox::DHCP::NetworkUser->new(
   address      => '10.1.1.1',
   domainame    => 'AD-7',
   first_seen_time   => 1234567,
```

```
    name         => 'user1',
    network_view => $dhcp_view,
    logon_id     => '10',
    guid         => '15',
);

#submit for addition
my $response = $session->add($network_user);
```

Infoblox::Session->get( )

Use this method to retrieve all the matching objects from the Infoblox appliance. See Infoblox::Session->get() for parameters and return values.

Key Reference

```
Apply the following attribute to get a specific object:

 address      - Optional. A DHCP Network User address.
 domainname   - Optional. A DHCP Network User domain name.
 guid         - Optional. A DHCP Network User group identifier.
 logon_id     - Optional. A DHCP Network User logon identifier.
 name         - Optional. A DHCP Network User name.
 network_view - Optional. A DHCP View object or a string which contains a DHCP View name.
 parent       - Optional. A DHCP or DNS object associated with the network user. Note that object
should be retrieved from the server.
 user_status  - Optional. A DHCP Network User status.
```

Example

```
my @retrieved_objs = $session->get(
    object => 'Infoblox::DHCP::NetworkUser',
    name   => 'user.*',
);

my @retrieved_objs = $session->get(
    object => 'Infoblox::DHCP::NetworkUser',
    parent => $dhcp_range,
);
```

Infoblox::Session->modify( )

Use this method to modify an object in the Infoblox appliance. See Infoblox::Session->modify() for parameters and return values.

Example

```
#Use method to modify the comment
$network_user->name('user2');
#Submit modification
my $response = $session->modify($network_user);
```

Infoblox::Session->remove( )

Use this method to remove an object from the Infoblox appliance. See Infoblox::Session->remove() for parameters and return values.

To remove a specific object, first use `get()` or `search()` to retrieve the specific object, and then submit this object for removal.

Example

```
#Get the objects with the same name
my @retrieved_objs = $session->get(
    object => 'Infoblox::DHCP::NetworkUser',
    name   => 'user1',
);
#Find the desired object from the retrieved list.
my $desired_user = $retrieved_objs[0];
#Submit for removal
my $response = $session->remove($desired_user);
```

Infoblox::Session->search( )

Use this method to search for objects in the Infoblox appliance. See Infoblox::Session->search() for parameters and return values.

Key Reference

```
Apply the following attribute to get a specific object:

 address      - Optional. A DHCP Network User address.
 domainname   - Optional. A DHCP Network User domain name.
```

```
  guid         - Optional. A DHCP Network User group identifier.
  logon_id     - Optional. A DHCP Network User logon identifier.
  name         - Optional. A DHCP Network User name.
  network_view - Optional. A DHCP View object or a string which contains a DHCP View name.
  parent       - Optional. A DHCP or DNS object associated with the network user. Note that object
should be retrieved from the server.
  user_status  - Optional. A DHCP Network User status.
```

Example

```
my @retrieved_objs = $session->search(
    object => 'Infoblox::DHCP::NetworkUser',
    name   => 'user.*',
);
```

METHODS

address( )

Use this method to set or retrieve the DHCP Network User address.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

The valid value is an IPv4 or IPv6 address.

Returns

If you specified a parameter, the method returns 'true' when the modification succeeds, and returns 'false' when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get address
my $address = $network_user->address();
#Modify address
$network_user->address('10.10.10.10');
```

address_object( )

Use this method to retrieve the DHCP Network User address IPAM object. This is a read-only attribute.

Omit the parameter to retrieve the attribute value.

Parameter

None

Returns

The valid return value is an Infoblox::IPAM::Address object.

Example

```
#Get address_object
my $address_object = $network_user->address_object();
```

data_source( )

Use this method to retrieve the DHCP Network User data source. This is a read-only attribute.

Omit the parameter to retrieve the attribute value.

Parameter

None

Returns

The method returns the attribute value. The valid return values are 'API', 'MS_SERVER' and 'CISCO_ISE'.

Example

```
#Get data_source
my $data_source = $network_user->data_source();
```

data_source_ip( )

Use this method to retrieve the DHCP Network User endpoint data source IP address or FQDN. This is a read-only attribute.

Omit the parameter to retrieve the attribute value.

Parameter

None

Returns

The method returns the attribute value.

Example

```
#Get data_source_ip
my $data_source_ip = $network_user->data_source_ip();
```

domainname( )

Use this method to set or retrieve the DHCP Network User domain name.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

The valid value is a desired domain name in string format.

Returns

If you specified a parameter, the method returns 'true' when the modification succeeds, and returns 'false' when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get domainname
my $domainname = $network_user->domainname();
#Modify domainname
$network_user->domainname('DOMAIN');
```

guid( )

Use this method to set or retrieve the DHCP Network User group identifier.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

The valid value is a desired group identifier in string format.

Returns

If you specified a parameter, the method returns 'true' when the modification succeeds, and returns 'false' when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get guid
my $guid = $network_user->guid();
#Modify guid
$network_user->guid('10');
```

last_seen_time( )

Use this method to set or retrieve the DHCP Network User last active timestamp.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

The valid value is the number of seconds that have elapsed since January 1st, 1970 UTC.

Returns

If you specified a parameter, the method returns 'true' when the modification succeeds, and returns 'false' when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get last_seen_time
my $last_seen_time = $network_user->last_seen_time();
#Modify last_seen_time
$network_user->last_seen_time(1234567);
```

last_updated_time( )

Use this method to retrieve the DHCP Network User last updated timestamp. This is a read-only attribute.

Parameter

None

Returns

The method returns the attribute value.

Example

```
#Get last_updated_time
my $last_updated_time = $network_user->last_updated_time();
```

first_seen_time( )

Use this method to set or retrieve the DHCP Network User first seen timestamp.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

The valid value is the number of seconds that have elapsed since January 1st, 1970 UTC.

Returns

If you specified a parameter, the method returns 'true' when the modification succeeds, and returns 'false' when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get first_seen_time
my $first_seen_time = $network_user->first_seen_time();
#Modify first_seen_time
$network_user->first_seen_time(1234567);
```

logon_id( )

Use this method to set or retrieve the DHCP Network User logon identifier.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

The valid value is a desired logon identifier in string format.

Returns

If you specified a parameter, the method returns 'true' when the modification succeeds, and returns 'false' when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get logon_id
my $logon_id = $network_user->logon_id();
#Modify logon_id
$network_user->logon_id('10');
```

logout_time( )

Use this method to set or retrieve the DHCP Network User logout timestamp.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

The valid value is the number of seconds that have elapsed since January 1st, 1970 UTC.

Returns

If you specified a parameter, the method returns 'true' when the modification succeeds, and returns 'false' when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get logout_time
my $logout_time = $network_user->logout_time();
#Modify logout_time
$network_user->logout_time(1234567);
```

name( )

Use this method to set or retrieve the DHCP Network User name.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

The valid value is a desired name in string format.

Returns

If you specified a parameter, the method returns 'true' when the modification succeeds, and returns 'false' when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get name
my $name = $network_user->name();
#Modify name
$network_user->name('user1');
```

network( )

Use this method to retrieve the DHCP Network User associated with the DHCP Network object. This is a read-only attribute.

Omit the parameter to retrieve the attribute value.

Parameter

None

Returns

The valid return values are Infoblox::DHCP::Network and Infoblox::DHCP::IPv6Network objects.

Example

```
#Get network
my $network = $network_user->network();
```

network_view( )

Use this method to set or retrieve the DHCP Network User associated with the DHCP View object.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

The valid value is a desired DHCP View name in string format.

Returns

If you specified a parameter, the method returns 'true' when the modification succeeds, and returns 'false' when the operation

fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get network_view
my $network_view = $network_user->network_view();
#Modify network_view
$network_user->network_view('internal');
```

user_status( )

Use this method to set or retrieve the DHCP Network User status. This is a read-only attribute.

Omit the parameter to retrieve the attribute value.

Parameter

None

Returns

The valid return values are 'ACTIVE', 'LOGOUT' and 'TIMEOUT'.

Example

```
#Get user_status
my $user_status = $network_user->user_status();
```

AUTHOR

Infoblox Inc. http://www.infoblox.com/

SEE ALSO

Infoblox::Session, Infoblox::Session->add(), Infoblox::Session->get(), Infoblox::Session->modify(), Infoblox::Session->remove(), Infoblox::Session->search(), Infoblox::IPAM::Address, Infoblox::DHCP::Network, Infoblox::DHCP::IPv6Network, Infoblox::DHCP::View

---

COPYRIGHT

Copyright (c) 2017 Infoblox Inc.

Infoblox::DHCP::Option - DHCP Option object.

- NAME
- DESCRIPTION
- CONSTRUCTOR
- MODULE METHODS
  - Infoblox::Grid::DHCP->options( )
  - Infoblox::Grid::Member::DHCP->options( )
  - Infoblox::DHCP::Network->options( )
  - Infoblox::DHCP::NetworkTemplate->options( )
  - Infoblox::DHCP::SharedNetwork->options( )
  - Infoblox::DHCP::FixedAddr->options( )
  - Infoblox::DHCP::FixedAddrTemplate->options( )
  - Infoblox::DHCP::Range->options( )
  - Infoblox::DHCP::RangeTemplate->options( )
  - Infoblox::DHCP::Filter::Option->option_list( )
- METHODS
  - name( )
  - num( )
  - value( )
  - vendor_class( )
- SAMPLE CODE
- AUTHOR
- SEE ALSO
- COPYRIGHT

NAME

Infoblox::DHCP::Option - DHCP Option object.

DESCRIPTION

An Option sets the value of a DHCP option that has been defined in an option space. DHCP options describe network configuration settings and various services available on the network. These options occur as variable-length fields at the end of DHCP messages.

CONSTRUCTOR

```
 my $option = Infoblox::DHCP::Option->new(
     value        => $num | $string |"true"|"false", #Required
     name         => $string,                        #Optional - One of name or num must be specified /
Default is undefined
     num          => $num,                           #Optional - One of name or num must be specified /
Default is undefined
     vendor_class => $string,                        #Optional / Default Option Space name is "DHCP"
 );
```

Note that in a future NIOS version 'num' will become mandatory and 'name' will become optional.

MODULE METHODS

The following functions can be applied to a DHCP Option object.

Infoblox::Grid::DHCP->options( )

Use this function to specify DHCP options at the grid level on the Infoblox appliance. See Infoblox::Grid::DHCP->options() for parameters and return values.

Example

```
       #Construct a DHCP Option object
       my $option1 = Infoblox::DHCP::Option->new(
                     name  => "lease-time",
                     value => "50"
       );
       # Configure a DHCP option for a grid
       my $response = $Grid_DHCP->options([$option1]);
```

Infoblox::Grid::Member::DHCP->options( )

Use this function to specify DHCP options at the grid member level on the Infoblox appliance. See Infoblox::Grid::Member::DHCP-

>options() for parameters and return values.

Example

```
#Construct a DHCP Option object
my $option2 = Infoblox::DHCP::Option->new(
              name  => "domain-name",
              value => "domainname.com"
);
# Configure a DHCP option for a grid member
my $response = $Grid_Member_DHCP->options([$option2]);
```

Infoblox::DHCP::Network->options( )

Use this function to specify DHCP options at the network level on the Infoblox appliance. See Infoblox::DHCP::Network->options() for parameters and return values.

Example

```
#Construct a DHCP Option object
my $option3 = Infoblox::DHCP::Option->new(
              name  => "broadcast",
              value => "255.0.0.0"
);
# Configure a DHCP option for a network
my $response = $Network_DHCP->options([$option3]);
```

Infoblox::DHCP::NetworkTemplate->options( )

Use this function to specify DHCP options at the network template level on the Infoblox appliance. See Infoblox::DHCP::NetworkTemplate->options() for parameters and return values.

Example

```
#Construct a DHCP Option object
my $option4 = Infoblox::DHCP::Option->new(
              name  => "broadcast",
              value => "2"
);
#Construct another DHCP Option object
my $option5 = Infoblox::DHCP::Option->new(
              name  => "routers",
              value => "1,2"
);
# Configure a DHCP options for a network template
my $response = $Network_Template_DHCP->options([$option4,$option5]);
```

Infoblox::DHCP::SharedNetwork->options( )

Use this function to specify DHCP options at the shared network level on the Infoblox appliance. See Infoblox::DHCP::SharedNetwork->options() for parameters and return values.

Example

```
#Construct a DHCP Option object
my $option6 = Infoblox::DHCP::Option->new(
              name  => "subnet-mask",
              value => "255.255.0.0"
);
# Configure a DHCP option for a shared network
my $response = $Shared_Network_DHCP->options([$option6]);
```

Infoblox::DHCP::FixedAddr->options( )

Use this function to specify DHCP options at the fixed address level on the Infoblox appliance. See Infoblox::DHCP::FixedAddr->options() for parameters and return values.

Example

```
#Construct a DHCP Option object
my $option7 = Infoblox::DHCP::Option->new(
              name  => "host-name",
              value => "host1.com"
);
# Configure a DHCP option for a fixed address
my $response = $Fixed_Address_DHCP->options([$option7]);
```

Infoblox::DHCP::FixedAddrTemplate->options( )

Use this function to specify DHCP options at the fixed address template level on the Infoblox appliance. See Infoblox::DHCP::FixedAddrTemplate->options() for parameters and return values.

Example

```
#Construct a DHCP Option object
my $option8 = Infoblox::DHCP::Option->new(
              name  => "broadcast",
              value => "2"
);
#Construct another DHCP Option object
my $option9 = Infoblox::DHCP::Option->new(
              name  => "routers",
              value => "1,2"
);
# Configure a DHCP options for a fixed address template
my $response = $Fixed_Address_Template_DHCP->options([$option8,$option9]);
```

Infoblox::DHCP::Range->options( )

Use this function to specify DHCP options at the DHCP range level on the Infoblox appliance. See Infoblox::DHCP::Range->options() for parameters and return values.

Example

```
#Construct a DHCP Option object
my $option10 = Infoblox::DHCP::Option->new(
              name  => "swap-server",
              value => "1.1.1.1"
);
# Configure a DHCP option for a DHCP range
my $response = $DHCP_Range->options([$option10]);
```

Infoblox::DHCP::RangeTemplate->options( )

Use this function to specify DHCP options at the range template level on the Infoblox appliance. See Infoblox::DHCP::RangeTemplate->options() for parameters and return values.

Example

```
#Construct a DHCP Option object
my $option11 = Infoblox::DHCP::Option->new(
              name  => "broadcast",
              value => "2"
);
#Construct another DHCP Option object
my $option12 = Infoblox::DHCP::Option->new(
              name  => "routers",
              value => "1,2"
);
# Configure a DHCP options for a range template
my $response = $Range_Template_DHCP->options([$option11,$option12]);
```

Infoblox::DHCP::Filter::Option->option_list( )

Use this function to specify DHCP options at the filter level on the Infoblox appliance. See Infoblox::DHCP::Filter::Option->option_list() for parameters and return values.

Example

```
#Construct a DHCP Option object
my $option13 = Infoblox::DHCP::Option->new(
              name  => "pop-server",
              value => "10.10.10.10"
);
# Configure a DHCP Option for a filter
my $response = $Filter_Option->option_list([$option13]);
```

METHODS

This section describes all the methods that can be used to configure and retrieve the attribute values of a DHCP option.

name( )

Use this method to set or retrieve the name of the DHCP option.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

Specify the name of the DHCP option in string format. Valid values are

```
  "subnet-mask",                              - DHCP Option   1              - ip-address
  "time-offset",                              - DHCP Option   2              - 32-bit signed integer
  "routers",                                  - DHCP Option   3              - array of ip-address |
array of offsets (for templates)
  "time-servers",                             - DHCP Option   4              - array of ip-address
  "ien116-name-servers",                      - DHCP Option   5              - array of ip-address
  "domain-name-servers",                      - DHCP Option   6              - array of ip-address
  "log-servers",                              - DHCP Option   7              - array of ip-address
  "cookie-servers",                           - DHCP Option   8              - array of ip-address
  "lpr-servers",                              - DHCP Option   9              - array of ip-address
  "impress-servers",                          - DHCP Option  10              - array of ip-address
  "resource-location-servers",                - DHCP Option  11              - array of ip-address
  "host-name",                                - DHCP Option  12              - string
  "boot-size",                                - DHCP Option  13              - 16-bit unsigned integer
  "merit-dump",                               - DHCP Option  14              - text
  "domain-name",                              - DHCP Option  15              - text
  "swap-server",                              - DHCP Option  16              - ip-address
  "root-path",                                - DHCP Option  17              - text
  "extensions-path",                          - DHCP Option  18              - text
  "ip-forwarding",                            - DHCP Option  19              - boolean
  "non-local-source-routing",                 - DHCP Option  20              - boolean
  "policy-filter",                            - DHCP Option  21              - array of ip-address |
ip-address
  "max-dgram-reassembly",                     - DHCP Option  22              - 16-bit unsigned integer
  "default-ip-ttl",                           - DHCP Option  23              - 8-bit unsigned integer
  "path-mtu-aging-timeout",                   - DHCP Option  24              - 32-bit unsigned integer
  "path-mtu-plateau-table",                   - DHCP Option  25              - array of 16-bit
unsigned integer
  "interface-mtu",                            - DHCP Option  26              - 16-bit unsigned integer
  "all-subnets-local",                        - DHCP Option  27              - boolean
  "broadcast",                                - DHCP Option  28              - ip-address | offset
(for templates)
  "perform-mask-discovery",                   - DHCP Option  29              - boolean
  "mask-supplier",                            - DHCP Option  30              - boolean
  "router-discovery",                         - DHCP Option  31              - boolean
  "router-solicitation-address",              - DHCP Option  32              - ip-address
  "static-routes",                            - DHCP Option  33              - array of ip-address |
ip-address
  "trailer-encapsulation",                    - DHCP Option  34              - boolean
  "arp-cache-timeout",                        - DHCP Option  35              - 32-bit unsigned integer
  "ieee802-3-encapsulation",                  - DHCP Option  36              - boolean
  "default-tcp-ttl",                          - DHCP Option  37              - 8-bit unsigned integer
  "tcp-keepalive-interval",                   - DHCP Option  38              - 32-bit unsigned integer
  "tcp-keepalive-garbage",                    - DHCP Option  39              - boolean
  "nis-domain",                               - DHCP Option  40              - text
  "nis-servers",                              - DHCP Option  41              - array of ip-address
  "ntp-servers",                              - DHCP Option  42              - array of ip-address
  "vendor-encapsulated-options",              - DHCP Option  43              - string
  "netbios-name-servers",                     - DHCP Option  44              - array of ip-address
  "netbios-dd-server",                        - DHCP Option  45              - array of ip-address
  "netbios-node-type",                        - DHCP Option  46              - 8-bit unsigned
integer(1,2,4,8)
  "netbios-scope",                            - DHCP Option  47              - text
  "font-servers",                             - DHCP Option  48              - array of ip-address
  "x-display-manager",                        - DHCP Option  49              - array of ip-address
  "dhcp-requested-address",                   - DHCP Option  50              - ip-address
  "lease-time",                               - DHCP Option  51              - 32-bit unsigned integer
  "dhcp-option-overload",                     - DHCP Option  52              - 8-bit unsigned integer
  "dhcp-message-type",                        - DHCP Option  53              - 8-bit unsigned integer
  "dhcp-server-identifier",                   - DHCP Option  54              - ip-address
  "dhcp-parameter-request-list",              - DHCP Option  55              - array of 8-bit unsigned
integer
  "dhcp-message",                             - DHCP Option  56              - text
  "dhcp-max-message-size",                    - DHCP Option  57              - 16-bit unsigned integer
  "dhcp-renewal-time",                        - DHCP Option  58              - 32-bit unsigned integer
  "dhcp-rebinding-time",                      - DHCP Option  59              - 32-bit unsigned integer
  "vendor-class-identifier",                  - DHCP Option  60              - string
  "dhcp-client-identifier",                   - DHCP Option  61              - string
  "nwip-domain",                              - DHCP Option  62              - string
  "nwip-suboptions",                          - DHCP Option  63              - string
  "nisplus-domain",                           - DHCP Option  64              - text
  "nisplus-servers",                          - DHCP Option  65              - array of ip-address
  "tftp-server-name",                         - DHCP Option  66              - text
  "bootfile-name",                            - DHCP Option  67              - text
  "mobile-ip-home-agent",                     - DHCP Option  68              - array of ip-address
  "smtp-server",                              - DHCP Option  69              - array of ip-address
  "pop-server",                               - DHCP Option  70              - array of ip-address
  "nntp-server",                              - DHCP Option  71              - array of ip-address
  "www-server",                               - DHCP Option  72              - array of ip-address
  "finger-server",                            - DHCP Option  73              - array of ip-address
  "irc-server",                               - DHCP Option  74              - array of ip-address
  "streettalk-server",                        - DHCP Option  75              - array of ip-address
  "streettalk-directory-assistance-server",   - DHCP Option  76              - array of ip-address
  "user-class",                               - DHCP Option  77              - text
  "slp-directory-agent",                      - DHCP Option  78              - boolean | array of ip-
address
  "slp-service-scope",                        - DHCP Option  79              - boolean | text
```

```
    "option-80",                          - DHCP Option  80             - custom defined
    "fqdn",                               - DHCP Option  81             - string
    "relay-agent-information",            - DHCP Option  82             - string
    "option-83",                          - DHCP Option  83             - custom defined
    "option-84",                          - DHCP Option  84             - custom defined
    "nds-servers",                        - DHCP Option  85             - array of ip-address
    "nds-tree-name",                      - DHCP Option  86             - string
    "nds-context",                        - DHCP Option  87             - string
    "option-88",                          - DHCP Option  88             - custom defined
    "bcms-controller-address",            - DHCP Option  89             - array of ip-address
    "option-90",                          - DHCP Option  90             - custom defined
    "client-last-transaction-time",       - DHCP Option  91             - 32-bit unsigned integer
    "associated-ip",                      - DHCP Option  92             - array of ip-address
    "option-93",                          - DHCP Option  93             - custom defined
    "option-94",                          - DHCP Option  94             - custom defined
    "option-95",                          - DHCP Option  95             - custom defined
    "option-96",                          - DHCP Option  96             - custom defined
    "option-97",                          - DHCP Option  97             - custom defined
    "uap-servers",                        - DHCP Option  98             - text
    "option-99",                          - DHCP Option  99             - custom defined
    "option-100",                         - DHCP Option 100             - custom defined
    "option-101",                         - DHCP Option 101             - custom defined
    "option-102",                         - DHCP Option 102             - custom defined
    "option-103",                         - DHCP Option 103             - custom defined
    "option-104",                         - DHCP Option 104             - custom defined
    "option-105",                         - DHCP Option 105             - custom defined
    "option-106",                         - DHCP Option 106             - custom defined
    "option-107",                         - DHCP Option 107             - custom defined
    "option-108",                         - DHCP Option 108             - custom defined
    "option-109",                         - DHCP Option 109             - custom defined
    "option-110",                         - DHCP Option 110             - custom defined
    "option-111",                         - DHCP Option 111             - custom defined
    "netinfo-server-address",             - DHCP Option 112             - array of ip-address
    "netinfo-server-tag",                 - DHCP Option 113             - text
    "default-url",                        - DHCP Option 114             - text
    "option-115",                         - DHCP Option 115             - custom defined
    "option-116",                         - DHCP Option 116             - custom defined
    "option-117",                         - DHCP Option 117             - custom defined
    "subnet-selection",                   - DHCP Option 118             - string
    "domain-search",                      - DHCP Option 119             - domain-list
    "option-120",                         - DHCP Option 120             - custom defined
    "option-121",                         - DHCP Option 121             - custom defined
    "option-122",                         - DHCP Option 122             - custom defined
    "option-123",                         - DHCP Option 123             - custom defined
    "vivco",                              - DHCP Option 124             - string
    "vivso",                              - DHCP Option 125             - string
    "option-126",                         - DHCP Option 126             - custom defined
    "option-127",                         - DHCP Option 127             - custom defined
    "option-128",                         - DHCP Option 128             - custom defined
    "option-129",                         - DHCP Option 129             - custom defined
    "option-130",                         - DHCP Option 130             - custom defined
    "option-131",                         - DHCP Option 131             - custom defined
    "option-132",                         - DHCP Option 132             - custom defined
    "option-133",                         - DHCP Option 133             - custom defined
    "option-134",                         - DHCP Option 134             - custom defined
    "option-135",                         - DHCP Option 135             - custom defined
    "option-136",                         - DHCP Option 136             - custom defined
    "option-137",                         - DHCP Option 137             - custom defined
    "option-138",                         - DHCP Option 138             - custom defined
    "option-139",                         - DHCP Option 139             - custom defined
    "option-140",                         - DHCP Option 140             - custom defined
    "option-141",                         - DHCP Option 141             - custom defined
    "option-142",                         - DHCP Option 142             - custom defined
    "option-143",                         - DHCP Option 143             - custom defined
    "option-144",                         - DHCP Option 144             - custom defined
    "option-145",                         - DHCP Option 145             - custom defined
    "option-146",                         - DHCP Option 146             - custom defined
    "option-147",                         - DHCP Option 147             - custom defined
    "option-148",                         - DHCP Option 148             - custom defined
    "option-149",                         - DHCP Option 149             - custom defined
    "option-150",                         - DHCP Option 150             - custom defined
    "option-151",                         - DHCP Option 151             - custom defined
    "option-152",                         - DHCP Option 152             - custom defined
    "option-153",                         - DHCP Option 153             - custom defined
    "option-154",                         - DHCP Option 154             - custom defined
    "option-155",                         - DHCP Option 155             - custom defined
    "option-156",                         - DHCP Option 156             - custom defined
    "option-157",                         - DHCP Option 157             - custom defined
    "option-158",                         - DHCP Option 158             - custom defined
    "option-159",                         - DHCP Option 159             - custom defined
    "option-160",                         - DHCP Option 160             - custom defined
    "option-161",                         - DHCP Option 161             - custom defined
    "option-162",                         - DHCP Option 162             - custom defined
    "option-163",                         - DHCP Option 163             - custom defined
    "option-164",                         - DHCP Option 164             - custom defined
    "option-165",                         - DHCP Option 165             - custom defined
    "option-166",                         - DHCP Option 166             - custom defined
    "option-167",                         - DHCP Option 167             - custom defined
    "option-168",                         - DHCP Option 168             - custom defined
    "option-169",                         - DHCP Option 169             - custom defined
    "option-170",                         - DHCP Option 170             - custom defined
    "option-171",                         - DHCP Option 171             - custom defined
    "option-172",                         - DHCP Option 172             - custom defined
    "option-173",                         - DHCP Option 173             - custom defined
    "option-174",                         - DHCP Option 174             - custom defined
    "option-175",                         - DHCP Option 175             - custom defined
    "option-176",                         - DHCP Option 176             - custom defined
```

```
    "option-177",                   - DHCP Option 177               - custom defined
    "option-178",                   - DHCP Option 178               - custom defined
    "option-179",                   - DHCP Option 179               - custom defined
    "option-180",                   - DHCP Option 180               - custom defined
    "option-181",                   - DHCP Option 181               - custom defined
    "option-182",                   - DHCP Option 182               - custom defined
    "option-183",                   - DHCP Option 183               - custom defined
    "option-184",                   - DHCP Option 184               - custom defined
    "option-185",                   - DHCP Option 185               - custom defined
    "option-186",                   - DHCP Option 186               - custom defined
    "option-187",                   - DHCP Option 187               - custom defined
    "option-188",                   - DHCP Option 188               - custom defined
    "option-189",                   - DHCP Option 189               - custom defined
    "option-190",                   - DHCP Option 190               - custom defined
    "option-191",                   - DHCP Option 191               - custom defined
    "option-192",                   - DHCP Option 192               - custom defined
    "option-193",                   - DHCP Option 193               - custom defined
    "option-194",                   - DHCP Option 194               - custom defined
    "option-195",                   - DHCP Option 195               - custom defined
    "option-196",                   - DHCP Option 196               - custom defined
    "option-197",                   - DHCP Option 197               - custom defined
    "option-198",                   - DHCP Option 198               - custom defined
    "option-199",                   - DHCP Option 199               - custom defined
    "option-200",                   - DHCP Option 200               - custom defined
    "option-201",                   - DHCP Option 201               - custom defined
    "option-202",                   - DHCP Option 202               - custom defined
    "option-203",                   - DHCP Option 203               - custom defined
    "option-204",                   - DHCP Option 204               - custom defined
    "option-205",                   - DHCP Option 205               - custom defined
    "option-206",                   - DHCP Option 206               - custom defined
    "option-207",                   - DHCP Option 207               - custom defined
    "option-208",                   - DHCP Option 208               - custom defined
    "option-209",                   - DHCP Option 209               - custom defined
    "option-210",                   - DHCP Option 210               - custom defined
    "option-211",                   - DHCP Option 211               - custom defined
    "option-212",                   - DHCP Option 212               - custom defined
    "option-213",                   - DHCP Option 213               - custom defined
    "option-214",                   - DHCP Option 214               - custom defined
    "option-215",                   - DHCP Option 215               - custom defined
    "option-216",                   - DHCP Option 216               - custom defined
    "option-217",                   - DHCP Option 217               - custom defined
    "option-218",                   - DHCP Option 218               - custom defined
    "option-219",                   - DHCP Option 219               - custom defined
    "option-220",                   - DHCP Option 220               - custom defined
    "option-221",                   - DHCP Option 221               - custom defined
    "option-222",                   - DHCP Option 222               - custom defined
    "option-223",                   - DHCP Option 223               - custom defined
    "option-224",                   - DHCP Option 224               - custom defined
    "option-225",                   - DHCP Option 225               - custom defined
    "option-226",                   - DHCP Option 226               - custom defined
    "option-227",                   - DHCP Option 227               - custom defined
    "option-228",                   - DHCP Option 228               - custom defined
    "option-229",                   - DHCP Option 229               - custom defined
    "option-230",                   - DHCP Option 230               - custom defined
    "option-231",                   - DHCP Option 231               - custom defined
    "option-232",                   - DHCP Option 232               - custom defined
    "option-233",                   - DHCP Option 233               - custom defined
    "option-234",                   - DHCP Option 234               - custom defined
    "option-235",                   - DHCP Option 235               - custom defined
    "option-236",                   - DHCP Option 236               - custom defined
    "option-237",                   - DHCP Option 237               - custom defined
    "option-238",                   - DHCP Option 238               - custom defined
    "option-239",                   - DHCP Option 239               - custom defined
    "option-240",                   - DHCP Option 240               - custom defined
    "option-241",                   - DHCP Option 241               - custom defined
    "option-242",                   - DHCP Option 242               - custom defined
    "option-243",                   - DHCP Option 243               - custom defined
    "option-244",                   - DHCP Option 244               - custom defined
    "option-245",                   - DHCP Option 245               - custom defined
    "option-246",                   - DHCP Option 246               - custom defined
    "option-247",                   - DHCP Option 247               - custom defined
    "option-248",                   - DHCP Option 248               - custom defined
    "option-249",                   - DHCP Option 249               - custom defined
    "option-250",                   - DHCP Option 250               - custom defined
    "option-251",                   - DHCP Option 251               - custom defined
    "option-252",                   - DHCP Option 252               - custom defined
    "option-253",                   - DHCP Option 253               - custom defined
    "option-254",                   - DHCP Option 254               - custom defined
```

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get name
my $name = $option->name();
#Modify name
$option->name("host-name");
```

num( )

Use this method to set or retrieve the numeric value of the DHCP option.

Note that specifying "num" without "name" is supported only for predefined options and custom options with names that haven't been modified.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

Specify the number of the DHCP option. It must be an integer ranging from 1 to 254.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get num
my $num = $option->num();
#Modify num
$option->num("12");
```

value( )

Use this method to set or retrieve the custom value of the DHCP option.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

Specify the data type of the DHCP option. Valid values are

```
  "boolean"                          - Specify "true" or "false" for boolean values.

  "ip-address"                       - Specify an IPv4 address. An IPv4 address is a 32-bit number
in dotted decimal notation.
                                       It consists of four 8-bit groups of decimal digits separated
                                       by decimal points (example: "192.168.1.2").

  "array of ip-address"              - Specify a list of IPv4 addresses, each separated by a comma.
                                       (example: "192.168.1.2, 10.11.12.13, 79.2.3.1")

  "string"                           - Specify a value in ASCII text string or
                                       a list of hexadecimal characters (0-9, a-f, or A-F) without
spaces and separated by colons.

  "text"                             - Specify a value in ASCII text string.

  "8-bit unsigned integer"           - Specify a numeric value between 0 and 255.

  "array of 8-bit unsigned integer"  - Specify a list of 8-bit unsigned integers, each separated by
a comma.

  "16-bit unsigned integer"          - Specify a numeric value between 0 and 65535.

  "array of 16-bit unsigned integer" - Specify a list of 16-bit unsigned integers, each separated by
a comma.

  "32-bit unsigned integer"          - Specify a numeric value between 0 and 4294967295.

  "array of 32-bit unsigned integer" - Specify a list of 32-bit unsigned integers, each separated by
a comma.

  "8-bit signed integer"             - Specify a numeric value between -128 and 127.

  "array of 8-bit signed integer"    - Specify a list of 8-bit signed integers, each separated by a
comma.

  "16-bit signed integer"            - Specify a numeric value between -32768 and 32767.

  "array of 16-bit signed integer"   - Specify a list of 16-bit signed integers, each separated by a
comma.

  "32-bit signed integer"            - Specify a numeric value between -2147483648 and 2147483647.

  "array of 32-bit signed integer"   - Specify a list of 32-bit signed integers, each separated by a
comma.

  "domain-list"                      - Specify a list of domain names, each double-quoted and
separated by a comma.
                                       (example: '"domain1.com","domain2.com","domain3.com"')
```

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get value
my $value = $option->value();
#Modify value
$option->value("abc.com");
```

vendor_class( )

Use this method to set an option space name as the vendor class value of a DHCP option or to retrieve an option space name.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

Specify the defined option space name in string format. The default option space name is "DHCP".

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get vendor_class
my $name = $option->vendor_class();
#Modify vendor_class
$option->vendor_class("DHCP");
```

SAMPLE CODE

The following sample code demonstrates the different functions that can be applied to a DHCP Option object such as add, modify, and remove. Also, this sample includes error handling for the operations.

#Preparation prior to a DHCP Option object insertion

```
#PROGRAM STARTS: Include all the modules that will be used
use strict;
use Infoblox;

#Create a session to the Infoblox appliance
my $session = Infoblox::Session->new(
               master   => "192.168.1.2", #appliance host ip
               username => "admin",       #appliance user login
               password => "infoblox"     #appliance password
);
unless ($session) {
       die("Construct session failed: ",
            Infoblox::status_code() . ":" . Infoblox::status_detail());
}
print "Session created successfully\n";
```

#Create a DHCP Option object

```
my $dhcp_option = Infoblox::DHCP::Option->new(
    name         => "subnet-mask",
    num          => "1",
    value        => "255.0.0.0",
    vendor_class => "DHCP"
);
unless($dhcp_option) {
       die("Construct DHCP option object failed: ",
            Infoblox::status_code() . ":" . Infoblox::status_detail());
}
print "DHCP option object created successfully\n";
```

#Get the Grid DHCP object and add a DHCP option to it

```
#Get the Grid DHCP object
my @retrieved_objs = $session->get(
    object => "Infoblox::Grid::DHCP",
```

```
    grid   => "Infoblox"
);
my $object = $retrieved_objs[0];
unless ($object) {
       die("Get Grid DHCP object failed: ",
            $session->status_code() . ":" . $session->status_detail());
}
print "Get Grid DHCP object found at least 1 matching entry\n";

#Apply the changes to the Grid DHCP object
$object->options([$dhcp_option]);

#Apply the changes
$session->modify($object)
    or die("Modify Grid DHCP object failed: ",
            $session->status_code() . ":" . $session->status_detail());
print "Grid DHCP object with DHCP option updated to Infoblox appliance successfully\n";
```

#Modify DHCP option

```
#Modify an existing option object

#Modifying the name of the DHCP option object.
$dhcp_option->name("host-name");

#Modifying the value of the DHCP option object.
$dhcp_option->value("infoblox.com");

#Modifying the number of the DHCP option object.
$dhcp_option->num("12");

#Apply changes to the Grid DHCP object.
$object->options([$dhcp_option]);

#Update Grid DHCP object through the Infoblox session.
$session->modify($object)
            or die("modify Grid DHCP object failed: ",
                $session->status_code() . ":" . $session->status_detail());
       print "Grid DHCP object with modified DHCP option updated to Infoblox appliance successfully\n";
```

#Remove DHCP option

```
#Apply changes to the Grid DHCP object.
$object->options([]);

#Update Grid DHCP object through the Infoblox session.
$session->modify($object)
            or die("modify Grid DHCP object failed: ",
                $session->status_code() . ":" . $session->status_detail());
       print "Removed DHCP option from Grid DHCP object successfully\n";

####PROGRAM ENDS####
```

AUTHOR

Infoblox Inc. http://www.infoblox.com/

SEE ALSO

Infoblox::Session, Infoblox::Session->get(), Infoblox::Session->modify(),Infoblox::Grid::DHCP, Infoblox::Grid::Member::DHCP, Infoblox::DHCP::Network, Infoblox::DHCP::NetworkTemplate, Infoblox::DHCP::SharedNetwork, Infoblox::DHCP::FixedAddr, Infoblox::DHCP::FixedAddrTemplate, Infoblox::DHCP::Range, Infoblox::DHCP::RangeTemplate, Infoblox::DHCP::Filter::Option

---

COPYRIGHT

Copyright (c) 2017 Infoblox Inc.

Infoblox::DHCP::Option60MatchRule - DHCP Option60MatchRule object.

- NAME
- DESCRIPTION
- CONSTRUCTOR
- MODULE METHODS
  - Infoblox::Grid::DHCP->option60_match_rules( )
  - Infoblox::Grid::Member::DHCP->option60_match_rules( )
- METHODS
  - match_value( )
  - option_space( )
  - is_substring( )
  - substring_offset( )
  - substring_length( )
- SAMPLE CODE
- AUTHOR
- SEE ALSO
- COPYRIGHT

NAME

Infoblox::DHCP::Option60MatchRule - DHCP Option60MatchRule object.

DESCRIPTION

An Option60MatchRule object allows specifying match rules for Option 60 globally.

CONSTRUCTOR

```
my $option60_match_rule = Infoblox::DHCP::Option60MatchRule->new(
                            match_value      => $string,          #Required
                            option_space     => $string,          #Required
                            is_substring     => "true" | "false", #Optional / Default is undefined
                            substring_offset => $num,             #Optional / Default is undefined
                            substring_length => $num              #Optional / Default is undefined
                          );
```

MODULE METHODS

The following functions are available to be applied to a DHCP Option60MatchRule object.

Infoblox::Grid::DHCP->option60_match_rules( )

Use this function to specify match rule for option60 at grid level. See Infoblox::Grid::DHCP->option60_match_rules() for parameters and return values.

Example

```
#Construct a DHCP Option60MatchRule object
my $option60_match_rule = Infoblox::DHCP::Option60MatchRule->new(
                                        "option_space" => "infospace",
                                        "match_value" => "Nortel-i2004-A",
                                        "is_substring" => "true",
                                        "substring_offset" => "8",
                                        "substring_length" => "14"
                                      );

# Configure DHCP Option60MatchRule on the Grid DHCP object
my $response = $grid_dhcp->option60_match_rules([$option60_match_rule]);
```

Infoblox::Grid::Member::DHCP->option60_match_rules( )

Use this function to specify match rule for option60 at member level. See Infoblox::Grid::Member::DHCP->option60_match_rules() for parameters and return values.

Example

```
#Construct a DHCP Option60MatchRule object
my $option60_match_rule = Infoblox::DHCP::Option60MatchRule->new(
                                        "option_space" => "infospace",
```

```
                                        "match_value" => "Nortel-i2004-A",
                                        "is_substring" => "true",
                                        "substring_offset" => "10",
                                        "substring_length" => "24"
                                      );

    # Configure DHCP Option60MatchRule on the Grid Member DHCP object
    my $response = $grid_member_dhcp->option60_match_rules([$option60_match_rule]);
```

METHODS

This section describes all the methods that can be used to configure and retrieve the attribute values of a DHCP Option60MatchRule object.

match_value( )

Use this method to set or retrieve the match value of a DHCP Option60MatchRule object.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

Desired match value in string format.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
    # Get match_value
    my $match_value = $option60_match_rule->match_value();
    # Modify match_value
    $option60_match_rule->match_value("SUNW");
```

option_space( )

Use this method to set or retrieve the option space of a DHCP Option60MatchRule object.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

Valid value is an string that contains defined Infoblox::DHCP::OptionSpace object. See Infoblox::DHCP::OptionSpace for parameters and return values.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
    # Get option_space
    my $match_value = $option60_match_rule->option_space();
    # Modify option_space
    $option60_match_rule->option_space("new_option");
```

is_substring( )

Use this method to set or retrieve the is_substring flag of a DHCP Option60MatchRule object.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

Specify "true" to set the is_substring flag or "false" to deactivate/unset it. The default value for this field is undefined.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
# Get is_substring
my $is_substring = $option60_match_rule->is_substring();
# Modify is_substring
$option60_match_rule->is_substring("true");
```

substring_offset( )

Use this method to set or retrieve the substring_offset of a DHCP Option60MatchRule object.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

The substring_offset for the DHCP Option60MatchRule object (range from 0 to 65535). The default value is undefined.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
# Get substring_offset
my $substring_offset = $option60_match_rule->substring_offset();
# Modify substring_offset
$option60_match_rule->substring_offset("10");
```

substring_length( )

Use this method to set or retrieve the substring_length of a DHCP Option60MatchRule object.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

The substring_length for the DHCP Option60MatchRule object (range from 0 to 65535). The default value is undefined.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
# Get substring_length
my $substring_length = $option60_match_rule->substring_length();
# Modify substring_length
$option60_match_rule->substring_length("23");
```

SAMPLE CODE

The following sample code demonstrates the different functions that can be applied to an object such as add, and remove. Also, this sample includes error handling for the operations.

#Preparation prior to a DHCP Option60MatchRule insertion

```
#PROGRAM STARTS: Include all the modules that will be used
use strict;
use Infoblox;

#Create a session to the Infoblox appliance

my $session = Infoblox::Session->new(
    master   => "192.168.1.2",
    username => "admin",
    password => "infoblox"
);
unless ($session) {
   die("Construct session failed: ",
       Infoblox::status_code() . ":" . Infoblox::status_detail());
}
print "Session created successfully\n";

my $option space1   Infoblox::DHCP::OptionSpace  new
```

```
                    =                                   ->   (
                                                        "name" => "infospace"
                                                        );
my $response = $session->add( $option_space1);
print "Add option space operation success \n" if ($response );
print "Add option space operation failed  \n" if ( not $response );

#Create a DHCP OptionDefinition object.>
my $option60_match_rule = Infoblox::DHCP::Option60MatchRule->new(
                                                        "option_space" => "infospace",
                                                        "match_value" => "Nortel-i2004-A",
                                                        "is_substring" => "true",
                                                        "substring_offset" => "0",
                                                        "substring_length" => "14"
                                                        );
unless ($option60_match_rule) {
     die("Construct DHCP Option60MatchRule failed: ",
       Infoblox::status_code() . ":" . Infoblox::status_detail());
}

#Get on Grid DHCP object to add Option60MatchRule object>
my @result = $session->get(
    object => "Infoblox::Grid::DHCP",
    grid   => "Infoblox"
    )
    or die("get DHCP failed: ",
                $session->status_code(). ":" .$session->status_detail());
print "get grid DHCP Objcet successful.\n";

unless (scalar(@result) == 0) {
my $griddhcp = $result[0];
if ($griddhcp) {

 $griddhcp->option60_match_rules([$option60_match_rule]);

  #Applying the changes to appliance through session.
  $session->modify($griddhcp)
      or die("Modify DHCP failed: ",
         $session->status_code(). ":" .$session->status_detail());
  print"Modified Grid DHCP Object successfully.\n";
    }
    } else {
      print "No Grid DHCP object found.";
   }

#To remove Option60MatchRule object in Grid DHCP object>
my @result = $session->get(
    object => "Infoblox::Grid::DHCP",
    grid   => "Infoblox"
    )
    or die("get DHCP failed: ",
                $session->status_code(). ":" .$session->status_detail());
print "get grid DHCP Objcet successful.\n";

unless (scalar(@result) == 0) {
my $griddhcp = $result[0];
if ($griddhcp) {

$griddhcp->option60_match_rules(undef);

  #Applying the changes to appliance through session.
  $session->modify($griddhcp)
      or die("Modify DHCP failed: ",
         $session->status_code(). ":" .$session->status_detail());
  print"Modified Grid DHCP Object successfully to remove Option60MatchRule object.\n";
  }
    } else {
      print "No Grid DHCP object found.";
   }

#Get OptionSpace object through the session
my @retrieved_objs = $session->get(
    object => "Infoblox::DHCP::OptionSpace",
    name   => "infospace"
);
my $object = $retrieved_objs[0];
unless  $object
```

```
        (       ) {
    die("Get OptionSpace object failed: ",
        $session->status_code() . ":" . $session->status_detail());
}
print "Get OptionSpace object found at least 1 matching entry\n";

#Submit the object for removal
$session->remove($object)
    or die("Remove OptionSpace object failed: ",
        $session->status_code() . ":" . $session->status_detail());
print "OptionSpace object removed successfully \n";

####PROGRAM ENDS####
```

AUTHOR

Infoblox Inc. http://www.infoblox.com/

SEE ALSO

Infoblox::Session,Infoblox::Grid::DHCP, Infoblox::Grid::Member::DHCP,Infoblox::DHCP::OptionSpace, Infoblox::Session->get(), Infoblox::Session->add(), Infoblox::Session->remove(), Infoblox::Session->modify()

COPYRIGHT

Copyright (c) 2017 Infoblox Inc.

Infoblox::DHCP::OptionDefinition - DHCP option definition object

- NAME
- DESCRIPTION
- CONSTRUCTOR
- SESSION METHODS
  - Infoblox::Session->add( )
  - Infoblox::Session->get( )
  - Infoblox::Session->modify( )
  - Infoblox::Session->search( )
  - Infoblox::Session->remove( )
- METHODS
  - name( )
  - code( )
  - space( )
  - type( )
- SAMPLE CODE
- AUTHOR
- SEE ALSO
- COPYRIGHT

NAME

Infoblox::DHCP::OptionDefinition - DHCP option definition object

---

DESCRIPTION

An OptionDefinition defines a DHCP option within a specific option space. A custom option can be defined in the predefined DHCP option space or in the user-defined vendor option space. To define an option, add the option definition to the required option space.

CONSTRUCTOR

```
my $option_definition1 = Infoblox::DHCP::OptionDefinition->new(
                                name  => $string, #Required
                                code  => $num,    #Required
                                space => $string, #Optional / Default is DHCP
                                type  => $string, #Optional / Default is string
                               );
```

SESSION METHODS

This section describes all the methods in an Infoblox::Session module that can be applied to a DHCP option definition object.

Infoblox::Session->add( )

Use this method to add a option definition object to the Infoblox appliance. See Infoblox::Session->add() for parameters and return values.

Example

```
#Create a DHCP OptionDefinition object.
my $option_definition1 = Infoblox::DHCP::OptionDefinition->new(
                                    space => "SUNW",
                                    name  => "JumpStart-server",
                                    code  => "6",
                                    type  => "text"
                                    );
# Submit for adding
my $response = $session->add( $option_definition1 );
```

Infoblox::Session->get( )

Use this method to retrieve all the matching DHCP option definition objects from the Infoblox appliance. See Infoblox::Session->get() for parameters and return values.

Key References

```
Apply the following attributes to get a specific DHCP option definition object:

 name - A name for the option definition in string format (regular expression).
```

```
  code - Option number. Either a name or code is required.
  space- Optional. The option space in which the option is defined. The default is the DHCP option
space.
```

Example

```
#Getting option by name
my  @result_array = $session->get(
                        object   => "Infoblox::DHCP::OptionDefinition",
                        space    => "SUNW",
                        name     => "JumpStart-server",
                        );

#Getting option by code
my @result_array = $session->get(
                        object   => "Infoblox::DHCP::OptionDefinition",
                        space    => "DHCP",
                        code     => 176,
                        );
```

Infoblox::Session->modify( )

Use this method to modify a DHCP option definition object in the Infoblox appliance. See Infoblox::Session->modify() for parameters and return values.

Example

```
#Use method to modify code
$object->code(20);
# Submit modification
my $response = $session->modify($object);
```

Infoblox::Session->search( )

Use this method to retrieve all the matching DHCP option definition objects from the Infoblox appliance. See Infoblox::Session->search() for parameters and return values.

Key References

```
Apply the following attributes to get a specific DHCP option definition object:

  name - A name for the option definition in string format (regular expression).
  code - Option number. Either a name or code is required.
  space- Optional. The option space in which the option is defined. The default is the DHCP option
space.
```

Example

```
#Getting option by name
my  @result_array = $session->search(
                        object   => "Infoblox::DHCP::OptionDefinition",
                        space    => "SUNW",
                        name     => "JumpStart-server",
                        );

#Getting option by code
my @result_array = $session->search(
                        object   => "Infoblox::DHCP::OptionDefinition",
                        space    => "DHCP",
                        code     => 176,
                        );
```

Infoblox::Session->remove( )

Use this method to remove a DHCP option definition object from the Infoblox appliance. See Infoblox::Session->remove() for parameters and return values.

To remove a specifc object, first use `get()` to retrieve the specific object, and then submit this object for removal.

Example

```
my  @result_array = $session->get(
                        object   => "Infoblox::DHCP::OptionDefinition",
                        space    => "SUNW",
                        name     => "JumpStart-server",
                        );

# find the desired object from retrieved list.
```

```
my $desired_obj = $result_array[0];

# Submit for removal
my $response = $session->remove($desired_obj);
```

METHODS

This section describes all the methods that can be used to configure and retrieve the attribute values of a DHCP option definition object.

name( )

Use this method to set or retrieve the name of a DHCP option definition object.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

Desired name in string format with a maximum of 256 bytes.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
# Get name
my $option_definitionname = $option_definition1 ->name();
# Modify name
$option_definition1 ->name("Start-server");
```

code( )

Use this method to set or retrieve the code of a DHCP option definition object. An option code number is used to identify the DHCP option.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

Number for the option code. Value must be between 1 and 254. Values 1-79, 81-82, 85-87, 89, 91-92, 98, 112-114, 118-119, 124-125 are predefined. These option codes have corresponding predefined Types. Therefore, you cannot change the predefined Types.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
# Get code
my $option_definitionname = $option_definition1 ->code();
# Modify code
$option_definition1 ->code(14);
```

space( )

Use this method to set or retrieve the space of a DHCP option definition object.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

Specify the space name in string format.

Returns

If you did not specify a parameter, the method returns the attribute value.

Example

```
# Get space
my $option_space = $option_definition1 ->space();
# Modify space
$option_definition1 ->space("SUNW");
```

---

type( )

Use this method to set or retrieve the data type of the grid DHCP option.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

The accepted types are the following:

```
  "boolean"                 - For "true" or "false" boolean values.

  "ip-address"              - For a single IPv4 address. An IPv4 address is a 32-bit number in dotted
decimal notation.
                              It consists of four 8-bit groups of decimal digits separated
                              by decimal points (example: "192.168.1.2").

  "array of ip-address"     - For a list of IPv4 addresses, each separated by a comma.
                              (example: "192.168.1.2, 10.11.12.13, 79.2.3.1")

  "string"                  - For value in ASCII text string or
                              a list of hexadecimal characters (0-9, a-f, or A-F) without spaces and
separated by colons.

  "text"                    - For value in ASCII text string.

  "domain-name"             - For domain name in ASCII text format

  "domain-list"             - For a list of domain names, each separated by a comma.

  "8-bit unsigned integer"  - For numeric value between 0 and 255.

  "16-bit unsigned integer" - For numeric value between 0 and 65535.

  "32-bit unsigned integer" - For numeric value between 0 and 4294967295.

  "8-bit signed integer"    - For numeric value between -128 and 127.

  "16-bit signed integer"   - For numeric value between -32768 and 32767.

  "32-bit signed integer"   - For numeric value between -2147483648 and 2147483647.

  "array of 8-bit unsigned integer"  - For a list of numeric values between 0 and 255, each separated
by a comma.

  "array of 16-bit unsigned integer" - For a list of numeric values between 0 and 65535, each
separated by a comma.

  "array of 32-bit unsigned integer" - For a list of numeric values between 0 and 4294967295, each
separated by a comma.

  "array of 8-bit integer"           - For a list of numeric values between -128 and 127, each
separated by a comma.

  "array of 16-bit integer"          - For a list of numeric values between -32768 and 32767, each
separated by a comma.

  "array of 32-bit integer"          - For a list of numeric values between -2147483648 and
2147483647, each separated by a comma.
```

The following types are valid only for `Microsoft(r)` server options

```
  "binary"

  "encapsulated"

  "array of string"

  "array of 64-bit unsigned integer" - For a list of numeric values between 0 and 1.8446744 * 10^19

  "64-bit unsigned integer"          - For a list of numeric values between 0 and 1.8446744 * 10^19,
each separated by a comma.
```

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
# Get type
my $option_definitiontype = $option_definition1 ->type();
# Modify type
$option_definition1 ->type("string");
```

SAMPLE CODE

The following sample code demonstrates the different functions that can be applied to an object such as add, modify, and remove. Also, this sample includes error handling for the operations.

#Preparation prior to a DHCP Network insertion

```
#PROGRAM STARTS: Include all the modules that will be used
use strict;
use Infoblox;

#Create a session to the Infoblox appliance

my $session = Infoblox::Session->new(
    master   => "192.168.1.2",
    username => "admin",
    password => "infoblox"
);
unless ($session) {
   die("Construct session failed: ",
       Infoblox::status_code() . ":" . Infoblox::status_detail());
}
print "Session created successfully\n";

my $option_space1 = Infoblox::DHCP::OptionSpace->new(
                                                "name" => "SUNW"
                                                );
my $response = $session->add( $option_space1);
print "Add option space operation success \n" if ($response );
print "Add option space operation failed  \n" if ( not $response );
```

#Create a DHCP OptionDefinition object.

```
my $option_definition1 = Infoblox::DHCP::OptionDefinition->new(
                                space => "SUNW",
                                name  => "JumpStart-server",
                                code  => "6",
                                type  => "text"
                              );
        unless ($option_definition1) {
        die("Construct DHCP OptionDefinition failed: ",
       Infoblox::status_code() . ":" . Infoblox::status_detail());
  }
 print "DHCP OptionDefinition object created successfully\n";

 #Add the DHCP OptionDefinition object to Infoblox appliance through a session
       my $response = $session->add( $option_definition1 )
       or die("Add a DHCP  OptionDefinition object failed: ",
             $session->status_code() . ":" . $session->status_detail());

print "Add a DHCP  OptionDefinition object successfully\n";
```

#Get and modify a DHCP OptionDefinition object attribute

```
#Get Network through the session
my  @retrieved_objs = $session->get(
              object   => "Infoblox::DHCP::OptionDefinition",
              space  => "SUNW",
              name   => "JumpStart-server",
            );

my $object = $retrieved_objs[0];

unless ($object) {
    die("Get DHCP OptionDefinition object failed: ",
        $session->status_code() . ":" . $session->status_detail());
}
print "Get DHCP OptionDefinition object found at least 1 matching entry\n";

#Modify one of the attributes of the specified Network attribute

 $object->code(20);    #Apply the changes

$session->modify($object)
    or die("Modify DHCP OptionDefinition attribute failed: ",
           $session->status_code() . ":" . $session->status_detail());
```

```
print "DHCP OptionDefinition object modified successfully \n";
```

#Remove a DHCP OptionDefinition Object

```
 #Get DHCP OptionDefinition object through the session
 my  @retrieved_objs = $session->get(
                  object =>"Infoblox::DHCP::OptionDefinition",
                  space  => "SUNW",
                  name   => "JumpStart-server",
            );

   my $object = $retrieved_objs[0];

   unless ($object) {
    die("Get DHCP OptionDefinition object failed: ",
        $session->status_code() . ":" . $session->status_detail());
    }

print "Get DHCP OptionDefinition object found at least 1 matching entry\n";
```

#Submit the object for removal

```
$session->remove($object)
    or die("Remove Network failed: ",
        $session->status_code() . ":" . $session->status_detail());

print "DHCP OptionDefinition removed successfully \n";

#Remove option space.

my $response = $session->remove( object => "Infoblox::DHCP::OptionSpace" ,
                                 name   => "SUNW",
                               );
print "Remove option space success \n" if ($response );
print "Remove option space failed  \n" if ( not $response );

####PROGRAM ENDS####
```

AUTHOR

Infoblox Inc. http://www.infoblox.com/

SEE ALSO

Infoblox::Session, Infoblox::DHCP::OptionSpace, Infoblox::DHCP::Option, Infoblox::Session->get(), Infoblox::Session->add(), Infoblox::Session->remove(), Infoblox::Session->modify()

COPYRIGHT

Copyright (c) 2017 Infoblox Inc.

Infoblox::DHCP::OptionSpace - DHCP OptionSpace object.

- NAME
- DESCRIPTION
- CONSTRUCTOR
- SESSION METHODS
  - Infoblox::Session->add( )
  - Infoblox::Session->get( )
  - Infoblox::Session->modify( )
  - Infoblox::Session->remove( )
  - Infoblox::Session->search( )
- METHODS
  - name( )
  - comment( )
  - space_type
- SAMPLE CODE
- AUTHOR
- SEE ALSO
- COPYRIGHT

NAME

Infoblox::DHCP::OptionSpace - DHCP OptionSpace object.

DESCRIPTION

An Option Space defines a namespace in which vendor options can be defined. To define a specific vendor option space, add an option space to DHCP.

CONSTRUCTOR

```
my $option_space = Infoblox::DHCP::OptionSpace->new(
      name             => $string,                          #Required
      comment          => $string,                          #Optional, default is undefined
);
```

SESSION METHODS

This section describes all the methods in Infoblox::Session module that can be applied to a DHCP OptionSpace object.

Infoblox::Session->add( )

Use this method to add a DHCP OptionSpace object to the Infoblox appliance. See Infoblox::Session->add() for parameters and return values.

Example

```
#Construct a DHCP OptionSpace object
my $option_space = Infoblox::DHCP::OptionSpace->new(
      name => "infospace",
      comment => "some comment"
);
#Submit for addition
my $response = $session->add($option_space)
```

Infoblox::Session->get( )

Use this method to retrieve all the matching objects from the Infoblox appliance. See Infoblox::Session->get() for parameters and return values.

Key References

```
Apply the following attributes to get a specific DHCP OptionSpace object:

 name     - Required. Option space name in string format.
 comment  - Optional. Option space comment.
```

Example

```
#Get DHCP OptionSpace object through the session
```

```
my @retrieved_objs = $session->get(
    object => "Infoblox::DHCP::OptionSpace",
    name   => "infospace",
);
```

Infoblox::Session->modify( )

Use this method to modify a DHCP OptionSpace object in the Infoblox appliance. See Infoblox::Session->modify() for parameters and return values.

Example

```
#Use method to modify the name.
$option_space->name("info_space");
#Submit modification
my $response = $session->modify( $option_space );
```

Infoblox::Session->remove( )

Use this method to remove a DHCP OptionSpace object from the Infoblox appliance. See Infoblox::Session->remove() for parameters and return values.

To remove a specific object, first use `get()` to retrieve the specific DHCP OptionSpace object, and then submit this object for removal.

Example

```
#Get DHCP OptionSpace object through the session
my @retrieved_objs = $session->get(
    object => "Infoblox::DHCP::OptionSpace",
    name   => "info_space",
);
#Find the desired object from the retrieved list.
my $desired_option_space = $retrieved_objs[0];
#Submit for removal
my $response = $session->remove( $desired_option_space );
```

Infoblox::Session->search( )

Use this method to retrieve all the matching objects from the Infoblox appliance. See Infoblox::Session->search() for parameters and return values.

Key References

```
Apply the following attributes to get a specific DHCP OptionSpace object:

 name     - Optional. Option space name in string format (regular expression).
 comment  - Optional. Option space comment (regular expression).
```

Example

```
#Get DHCP OptionSpace object through the session
my @retrieved_objs = $session->search(
    object => "Infoblox::DHCP::OptionSpace",
    comment => ".*comm.*"
);
```

METHODS

This section describes all the methods that can used to set and retrieve the attribute values of a DHCP OptionSpace object.

name( )

Use this method to set or retrieve name of option space object.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

Desired name in string format with a maximum of 256 bytes.

Returns

If parameter is specified, the method returns true when the modification succeeds, and returns false when the operation fails.

If parameter is not specified, the method returns the attribute value.

Example

```
#Get name
my $name = $option_space->name();
#Modify name
$option_space->name("info_space");
```

comment( )

Use this method to set or retrieve a comment about the option space object.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

Desired comment in string format with a maximum of 256 bytes.

Returns

If the parameter is specified, the method returns true when the modification succeeds, and returns false when the operation fails.

If the parameter is not specified, the method returns the attribute value.

Example

```
#Get comment
my $comment = $option_space->comment();
#Modify comment
$option_space->comment("my comment");
```

space_type

Use this method to retrieve the type of this option space.

Parameter

None.

Returns

The method returns the attribute value, which can be either 'predefined_dhcp' or 'vendor_space'.

Example

```
#Get space_type
my $space_type = $option_space->space_type();
```

SAMPLE CODE

The following sample code demonstrates the different functions that can be applied to an OptionSpace object, such as get, add, modify and remove. In addition, the sample includes error handling for the operations.

#Preparation prior to a DHCP OptionSpace object insertion

```
#PROGRAM STARTS: Include all the modules that will be used
use strict;
use Infoblox;

#Create a session to the Infoblox appliance

my $session = Infoblox::Session->new(
               master   => "192.168.1.2", #appliance host ip
               username => "admin",       #appliance user login
               password => "infoblox"     #appliance password
);

unless ($session) {
       die("Construct session failed: ",
               Infoblox::status_code() . ":" . Infoblox::status_detail());
}
print "Session created successfully\n";
```

#Create an OptionSpace object

```
my $option_space = Infoblox::DHCP::OptionSpace->new (
       "name" => "infospace"
);
```

```
unless($option_space) {
     die("Construct option_space filter failed: ",
           Infoblox::status_code() . ":" . Infoblox::status_detail());
}
print "OptionSpace object created successfully\n";

#Verify if the DHCP OptionSpace exists in the Infoblox appliance
my $object = $session->get(
       object => "Infoblox::DHCP::OptionSpace",
       name   => "infospace"
);
unless ($object) {
   print "option space does not exist on server, safe to add the option_space\n";
   $session->add($option_space)
      or die("Add option space failed: ",
             $session->status_code() . ":" . $session->status_detail());
}
print "DHCP OptionSpace added successfully\n";
```

#Get and Modify an OptionSpace object

```
#Get OptionSpace object through the session
my @retrieved_objs = $session->get(
    object => "Infoblox::DHCP::OptionSpace",
    name   => "infospace"
);
my $object = $retrieved_objs[0];
unless ($object) {
    die("Get OptionSpace object failed: ",
        $session->status_code() . ":" . $session->status_detail());
}
print "Get OptionSpace object found at least 1 matching entry\n";

#Modify name of the obtained OptionSpace object
$object->name("info_space");

#Apply the changes
$session->modify($object)
    or die("Modify OptionSpace object failed: ",
           $session->status_code() . ":" . $session->status_detail());
print "OptionSpace object modified successfully \n";
```

#Remove an OptionSpace object

```
#Get OptionSpace object through the session
my @retrieved_objs = $session->get(
    object => "Infoblox::DHCP::OptionSpace",
    name   => "info_space"
);
my $object = $retrieved_objs[0];
unless ($object) {
    die("Get OptionSpace object failed: ",
        $session->status_code() . ":" . $session->status_detail());
}
print "Get OptionSpace object found at least 1 matching entry\n";

#Submit the object for removal
$session->remove($object)
    or die("Remove OptionSpace object failed: ",
        $session->status_code() . ":" . $session->status_detail());
print "OptionSpace object removed successfully \n";

####PROGRAM ENDS####
```

AUTHOR

Infoblox Inc. http://www.infoblox.com/

SEE ALSO

Infoblox::DHCP::OptionDefinition,Infoblox::Session->get(), Infoblox::Session->modify(), Infoblox::Session->remove(), Infoblox::Session

---

COPYRIGHT

Infoblox::DHCP::OptionSpace - DHCP OptionSpace object.

Copyright (c) 2017 Infoblox Inc.

Infoblox::DHCP::OrderedRanges - Ordered Ranges object.

- NAME
- DESCRIPTION
- CONSTRUCTOR
- SESSION METHODS
  - Infoblox::Session->add( )
  - Infoblox::Session->get( )
  - Infoblox::Session->modify( )
- METHODS
  - network( )
  - ranges( )
- SAMPLE CODE
- AUTHOR
- SEE ALSO
- COPYRIGHT

NAME

Infoblox::DHCP::OrderedRanges - Ordered Ranges object.

DESCRIPTION

An object type to set or retrieve an ordered list of DHCP ranges in a network.

CONSTRUCTOR

```
my $ordered_ranges = Infoblox::DHCP::OrderedRanges->new(
                     network                  => $network,                    #Required
                     ranges                   => [$range1, $range2, ...],     #Required
);
```

SESSION METHODS

This section describes all the methods in the Infoblox::Session module that you can apply to an Ordered Ranges object.

Infoblox::Session->add( )

Use this method to add an object to the Infoblox appliance. See Infoblox::Session->add() for parameters and return values.

No object is added in this method, but the ranges for the specified network assume the specified order.

Example

```
#Construct an object
my $ordered_ranges = Infoblox::DHCP::OrderedRanges->new(

network => $network,
                                                        ranges  => [$range1, $range2]
                                                       );
#Submit for addition
my $response = $session->add( $ordered_ranges );
```

Infoblox::Session->get( )

Use this method to retrieve all the matching objects from the Infoblox appliance. See Infoblox::Session->get() for parameters and return values.

Key References

```
Apply the following attributes to get a specific Ordered Ranges object:

network - Required. An Infoblox::DHCP::Network object.
```

Examples

```
my @retrieved_objs = $session->get(
    object   => "Infoblox::DHCP::OrderedRanges",
    network  => $network
);
```

Infoblox::Session->modify( )

Use this method to modify an object in the Infoblox appliance. See Infoblox::Session->modify() for parameters and return values.

No object is modified in this method, but the ranges for the specified network assume the specified order.

Example

```
#Use this method to modify the comment.
$ordered_ranges->ranges([$range4, $range2, $range3, $range1]);
#Submit modification
my $response = $session->modify( $ordered_ranges );
```

METHODS

Use the following methods to access the attributes of an Infoblox::DHCP::OrderedRanges object. Specify a parameter to set the attribute's value, or omit the parameter to get the attribute's value.

network( )

Use this method to set or retrieve the parent network object.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

An Infoblox::DHCP::Network object.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Getting network
my $network = $ordered_ranges->network();
#Modifying network
$ordered_ranges->network($nw1);
```

ranges( )

Use this method to set or retrieve an ordered list of DHCP ranges object references.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

A list of Infoblox::DHCP::Range objects.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Getting ranges
my $ranges_ref = $ordered_ranges->ranges();
#Modifying ranges
$ordered_ranges->ranges([$range1, $range2, @range3]);
```

SAMPLE CODE

The following sample code demonstrates the different functions that can be applied to an object, such as add, search, modify, and remove. This sample also includes error handling for the operations.

#Preparation prior to the range reordering

```
#PROGRAM STARTS: Include all the modules that will be used
use strict;
use Infoblox;

#Create a session to the Infoblox appliance
```

```
my $host_ip =  "192.168.1.2";
my $session = Infoblox::Session->new(
    master   => $host_ip,
    username => "admin",
    password => "infoblox"
    );
unless($session){
        die("Constructor for session failed: ",
              Infoblox::status_code(). ":" . Infoblox::status_detail());
}
print "Session created successfully.\n";

my $network = Infoblox::DHCP::Network->new(
                                          network   => "10.0.0.0/255.255.0.0",
                                          comment   => "add network",
                                         );

$session->add($network)
    or die("Add network failed: ",
              $session->status_code(). ":" .$session->status_detail());

print"Network added successfully.\n";

my $range = Infoblox::DHCP::Range->new(
                                       start_addr              => "10.0.0.1",
                                       end_addr                => "10.0.0.10",
                                       network                 => "10.0.0.0/16",
                                       comment                 => "range 1",
                                      );

$session->add($range)
    or die("Add range failed: ",
              $session->status_code(). ":" .$session->status_detail());

print"Range added successfully.\n";

$range = Infoblox::DHCP::Range->new(
                                     start_addr              => "10.0.0.21",
                                     end_addr                => "10.0.0.30",
                                     network                 => "10.0.0.0/16",
                                     comment                 => "range 2",
                                   );

$session->add($range)
    or die("Add range failed: ",
              $session->status_code(). ":" .$session->status_detail());

print"Range added successfully.\n";

$range = Infoblox::DHCP::Range->new(
                                     start_addr              => "10.0.0.31",
                                     end_addr                => "10.0.0.40",
                                     network                 => "10.0.0.0/16",
                                     comment                 => "range 3",
                                   );

$session->add($range)
    or die("Add range failed: ",
              $session->status_code(). ":" .$session->status_detail());

print"Range added successfully.\n";
```

#Get the current range order.

```
my $tnetwork = Infoblox::DHCP::Network->new(
                                           network   => "10.0.0.0/255.255.0.0",
                                          );
my $or = $session->get(
     object  => "Infoblox::DHCP::OrderedRanges",
     network => $tnetwork
);

unless($or){
       die("Get ordered ranges failed: ",
                $session->status_code() . ":" . $session->status_detail());
```

```
        }

print "Get ordered ranges object found at least 1 matching entry\n";

my ($r1, $r2, $r3) = @{$or->ranges()};

print "The current order of the ranges is : " . $r1->comment() . "," .
                $r2->comment() . "," . $r3->comment() . "\n";
```

#Modify the range order.

```
$or->ranges([$r3, $r2, $r1]);

#Apply the change
$session->modify($or)
      or die("Modify ordered ranges failed: ",
                $session->status_code(). ":" .$session->status_detail());

print "Ordered ranges modified successfully.\n";

$or = $session->get(
    object  => "Infoblox::DHCP::OrderedRanges",
    network => $tnetwork
);

unless($or){
      die("Get ordered ranges failed: ",
                $session->status_code() . ":" . $session->status_detail());
      }

print "Get ordered ranges object found at least 1 matching entry\n";

($r1, $r2, $r3) = @{$or->ranges()};

print "The current order of the ranges is : " . $r1->comment() . "," .
                $r2->comment() . "," . $r3->comment() . "\n";

####PROGRAM ENDS####
```

AUTHOR

Infoblox Inc. http://www.infoblox.com

SEE ALSO

Infoblox::DHCP::Network, Infoblox::DHCP::Range, Infoblox::Session, Infoblox::Session->add(), Infoblox::Session->get(), Infoblox::Session->modify()

COPYRIGHT

Copyright (c) 2017 Infoblox Inc.

Infoblox::DHCP::Range - DHCP Range object.

- NAME
- DESCRIPTION
- CONSTRUCTOR
- SESSION METHODS
  - Infoblox::Session->add( )
  - Infoblox::Session->get( )
  - Infoblox::Session->modify( )
  - Infoblox::Session->remove( )
  - Infoblox::Session->search( )
- METHODS
  - always_update_dns( )
  - bootfile( )
  - bootserver( )
  - comment( )
  - cloud_info( )
  - ddns_domainname( )
  - ddns_generate_hostname( )
  - deny_all_clients( )
  - deny_bootp( )
  - disable( )
  - discovery_basic_poll_setting( )
  - discovery_member( )
  - discover_now_status( )
  - discovery_blackout_setting( )
  - enable_discovery( )
  - enable_immediate_discovery( )
  - email_list( )
  - enable_ddns( )
  - enable_dhcp_thresholds( )
  - enable_ifmap_publishing( )
  - enable_email_warnings( )
  - enable_known_clients_option( )
  - enable_snmp_warnings( )
  - enable_unknown_clients_option( )
  - end_addr( )
  - endpoint_sources( )
  - exclude( )
  - extattrs( )
  - extensible_attributes( )
  - failover_assoc( )
  - filters( )
  - ignore_id( )
  - ignore_mac_addresses( )
  - is_split_scope( )
  - known_clients_option( )
  - lease_scavenge_time( )
  - name( )
  - next_available_ip( )
  - ignore_dhcp_option_list_request( )
  - member( )
  - ms_ad_user_data( )
  - ms_options( )
  - network( )
  - network_view( )
  - nextserver( )
  - options( )
  - logic_filters( )
  - override_ddns_domainname( )
  - override_enable_ifmap_publishing( )
  - override_blackout_setting( )
  - override_ignore_id( )
  - override_lease_scavenge_time( )
  - override_logic_filters( )
  - override_subscribe_settings( )
  - override_update_dns_on_lease_renewal( )
  - port_control_blackout_setting
  - pxe_lease_time( )
  - range_high_water_mark( )
  - range_high_water_mark_reset( )
  - range_low_water_mark( )
  - range_low_water_mark_reset( )

- restart_if_needed( )
- recycle_leases( )
- same_port_control_discovery_blackout( )
- server_association_type( )
- split_member
- split_scope_exclusion_percent
- start_addr( )
- subscribe_settings( )
- template( )
- unknown_clients_option( )
- update_dns_on_lease_renewal( )

- SAMPLE CODE
- AUTHOR
- SEE ALSO
- COPYRIGHT

NAME

Infoblox::DHCP::Range - DHCP Range object.

DESCRIPTION

A DHCP range defines the specified range of IP addresses in a network. A DHCP range should be added for a network so the Infoblox appliance can assign IP addresses within that specified range to DHCP clients. If the client is on a network that is assigned a DHCP range, the device distributes an available IP address from that range to the DHCP client, or to a DHCP relay agent if the request came through an agent. The DHCP range should also be assigned with a device. If devices are in a grid, the particular member serving DHCP for the DHCP range must be specified. If the server is an independent device, this device must be specified as the member that serves the DHCP range.

CONSTRUCTOR

```
 my $dhcp_range = Infoblox::DHCP::Range->new(

     end_addr                        => $ipv4addr,                                  #Required
     network                         => $network,                                   #Optional
     network_view                    => $NetworkView,                               #Optional / Default is
system-defined default network view
     start_addr                      => $ipv4addr,                                  #Required
     always_update_dns               => "true" | "false",                           #Optional / Default is
"false"
     bootfile                        => $filename | undef,                          #Optional / Default is
undefined
     bootserver                      => $ipv4addr | $fqdn | undef,                 #Optional / Default is
undefined
     comment                         => $string,                                    #Optional / Default is
undefined
     ddns_domainname                 => $string | undef,                            #Optional / Default is
undefined
     ddns_generate_hostname          => "true" | "false",                           #Optional / Default is
undefined
     deny_all_clients                => "true" | "false",                           #Optional / Default is
"false"
     deny_bootp                      => "true" | "false" | undef,                   #Optional / Default is
undefined
     disable                         => "true" | "false",                           #Optional / Default is
"false"
     email_list                      => [ "admin\@test.com",...] | undef,           #Optional / Default is
undefined
     enable_ddns                     => "true" | "false",                           #Optional / Default is
undefined
     enable_dhcp_thresholds          => "true" | "false",                           #Optional / Default is
"false"
     enable_email_warnings           => "true" | "false",                           #Optional / Default is
"false"
     enable_ifmap_publishing         => "true" | "false",                           #Optional / Default is
undefined
     enable_known_clients_option     => "true" | "false",                           #Optional / Default is
"false"
     enable_snmp_warnings            => "true" | "false",                           #Optional / Default is
"false"
     enable_unknown_clients_option   => "true" | "false",                           #Optional / Default is
"false"
     exclude                         => [$ExclusionRange1,$ExclusionRange2,...],    #Optional / Default is
empty
     extattrs                        => { $string => $extattr, ... },               #Optional / Default is
undefined
     extensible_attributes           => { $string => $string | $num, $string => [ $string | $num, ... ],
... } #Optional / Default is undefined
```

```
     failover_assoc                  => $string,                                  #Optional / Default is
undefined
     filters                         => [$MAC, $RelayAgent, $Option...],          #Optional / Default is
empty
     ignore_dhcp_option_list_request => "true" | "false" | undef,                 #Optional / Default is
undefined
     ignore_id                       => 'NONE' | 'CLIENT' | 'MACADDR',            #Optional / Default is
'NONE'
     ignore_mac_addresses            => [ "mac_addr", ... ] | undef               #Optional / Default is
undefined
     known_clients_option            => "Allow" | "Deny",                         #Optional / Default is
undefined
     lease_scavenge_time             => $number,                                  #Optional / Default is
-1
     member                          => $Member,                                  #Optional / Default is
undefined
     ms_options                      => [$MSOption1,$MSOption2,...],              #Optional / Default is
empty
     name                            => $string,                                  #Optional / Default is
undefined
     nextserver                      => $ipv4addr | $fqdn | undef,                #Optional / Default is
undefined
     options                         => [$Option1,$Option2,...],                  #Optional / Default is
empty
     logic_filters                   => [ $filter_name | $filter_obj, ...],       #Optional / Default is
undefined
     override_logic_filters          => "true' | 'false",                         #Optional / Default
value is "false"
     override_ddns_domainname        => "true" | "false",                         #Optional / Default
value is "false"
     override_enable_ifmap_publishing => "true" | "false",                        #Optional / Default
value is "false"
     override_lease_scavenge_time    => "true" | "false",                         #Optional / Default
value is "false"
     override_update_dns_on_lease_renewal  => "true" | "false",                   #Optional / Default
value is "false"
     override_ignore_id              => "true" | "false",                         #Optional / Default
value is "false"
     pxe_lease_time                  => $num,                                     #Optional / Default is
undefined
     range_high_water_mark           => $num,                                     #Optional / Default is
95
     range_high_water_mark_reset     => $num,                                     #Optional / Default is
95
     range_low_water_mark            => $num,                                     #Optional / Default is 0
     range_low_water_mark_reset      => $num,                                     #Optional / Default is 0
     recycle_leases                  => "true" | "false",                         #Optional / Default is
undefined
     server_association_type         => "NONE" | "MEMBER" | "FAILOVER" | "MS_SERVER" #Optional / Default is
undefined
     split_member                    => $member,                                  #Optional / Default is
undefined
     split_scope_exclusion_percent   => $num                                      #Optional / Default is
undefined
     template                        => $string,                                  #Optional / Default is
undefined
     unknown_clients_option          => "Allow" | "Deny",                         #Optional / Default is
undefined
     update_dns_on_lease_renewal     => "true" | "false",                         #Optional / Default
value is "false"
     discovery_blackout_setting      => $blackout,                                #Optional
     port_control_blackout_setting   => $blackout,                                #Optional
     override_blackout_setting       => "true" | "false",                         #Optional / Default
value is "false"
     same_port_control_discovery_blackout => "true" | "false",                    #Optional / Default
value is "false"
     subscribe_settings          => $subscribe_settings,                          #Optional / Default is
undefined
     override_subscribe_settings => 'true' | 'false',                             #Optional / Default is
'false'
      );
```

You cannot set both extattrs and extensible_attributes attributes at the same time.

SESSION METHODS

This section describes all the methods in an Infoblox::Session module that you can apply to a DHCP range object.

Infoblox::Session->add( )

Use this method to add a DHCP range object to the Infoblox appliance. See Infoblox::Session->add() for parameters and return values.

Example

```
#Construct an object
my $dhcp_range = Infoblox::DHCP::Range->new(
        end_addr                => "10.0.0.10",
        network                 => "10.0.0.0/8",
        start_addr              => "10.0.0.1",
        bootfile                => "bootfile1.com",
        bootserver              => "someserver.com",
        comment                 => "add range",
        ddns_generate_hostname  => "true",
        deny_all_clients        => "true",
        deny_bootp              => "true",
        disable                 => "true",
        enable_ddns             => "true",
        exclude                 => [$exclusion_range],
        filters                 => [$filter],
        member                  => $memberdhcp,
        network                 => $nview,
        nextserver              => "3.3.3.3",
        options                 => [$option1],
        pxe_lease_time          => "3600",
        recycle_leases          => "true",
);

# Submit for addition
my $response = $session->add( $dhcp_range );
```

Infoblox::Session->get( )

Use this method to retrieve all the matching objects from the Infoblox appliance. See Infoblox::Session->get() for parameters and return values.

'discover_now_status' will not be returned by default, use 'return_methods' to request it if needed.

Key References

```
Apply the following attributes to get a specific DHCP range object:

start_addr             - Optional. Starting IP address of DHCP range within the network.
network_view           - Optional. Network view in which the DHCP range is located. By default, all
network views are searched.
extattrs               - Optional. A hash reference containing extensible attributes.
extensible_attributes  - Optional. A hash reference containing extensible attributes.
failover_assoc         - Optional. The name of the associated failover peer.
member                 - Optional. The member or Microsoft server serving the range. The valid value
is an Infoblox::DHCP::Member, Infoblox::DHCP::MSServer, Infoblox::Grid::Member::DHCP or
Infoblox::Grid::MSServer::DHCP object.
server_association_type - Optional. The server association type for the range.
```

Examples

```
my @retrieved_objs = $session->get(
                     object     => "Infoblox::DHCP::Range",
                     start_addr => "10.0.0.1" );

my @retrieved_objs = $session->get(
                     object       => "Infoblox::DHCP::Range",
                     start_addr   => "10.0.0.1",
                     network_view => "default" );

my @retrieved_objs = $session->get(
                     object               => "Infoblox::DHCP::Range",
                     extensible_attributes => { 'Site' => 'Santa Clara' });
```

Infoblox::Session->modify( )

Use this method to modify a DHCP range object in the Infoblox appliance. See Infoblox::Session->modify() for parameters and return values.

Example

```
# Use method to modify the comment.
$dhcp_range->comment("this is a modified comment");
# Submit modification
my $response = $session->modify( $dhcp_range );
```

Infoblox::Session->remove( )

Use this method to remove a DHCP range object from the Infoblox appliance. See Infoblox::Session->remove() for parameters and return values.

To remove a specific object, first use `get()` or `search()` to retrieve the specific DHCP range object, and then submit this object for removal.

Example

```
# Get the objects with the same name
my @retrieved_objs = $session->get(
    object       => "Infoblox::DHCP::Range",
    start_addr   => "10.0.0.1",
    newtork_view => "default" );
# Find the desired object from the retrieved list.
my $desired_dhcp_range = $retrieved_objs[0];
# Submit for removal
my $response = $session->remove( $desired_dhcp_range );
```

Infoblox::Session->search( )

Use this method to search for DHCP range objects in the Infoblox appliance. See Infoblox::Session->search() for parameters and return values.

'discover_now_status' will not be returned by default, use 'return_methods' to request it if needed.

Key References

```
 Apply the following attributes to search for a DHCP range object:

  network      - Optional. Network in CIDR format (regular expression).
  network_view - Optional. Network view in which the DHCP range is located. By default, all network
views are searched.
  start_addr   - Optional. IPv4 address in dotted decimal notation (regular expression).
  end_addr     - Optional. IPv4 address in dotted decimal notation (regular expression).
  comment      - Optional. Comment in string format (regular expression).
  extattrs     - Optional. A hash reference containing extensible attributes.
  extensible_attributes - Optional. A hash reference containing extensible attributes.
  failover_assoc        - Optional. The name of the associated failover peer.
  member                - Optional. The member or Microsoft server serving the range. The valid value
is an Infoblox::DHCP::Member, Infoblox::DHCP::MSServer, Infoblox::Grid::Member::DHCP or
Infoblox::Grid::MSServer::DHCP object.
  server_association_type - Optional. The server association type for the range.
```

For more information about searching extensible attributes, see Infoblox::Grid::ExtensibleAttributeDef/Searching Extensible Attributes.

Examples

```
# search for DHCP range object with start address having last octet as 1
my @retrieved_objs = $session->search(
    object       => "Infoblox::DHCP::Range",
    network      => "10.0.0.0/.",
    network_view => "default",
    start_addr   => ".*1",
    comment      => ".*comment" );

# search for DHCP range objects defining "Santa Clara" for "Site" extensible attribute
my @retrieved_objs = $session->search(
    object               => "Infoblox::DHCP::Range",
    extensible_attributes => { 'Site' => 'Santa Clara' });
```

METHODS

This section describes all the methods that you can use to configure and retrieve the attribute values of a DHCP range.

always_update_dns( )

Use this method to set or retrieve the always_update_dns flag of a DHCP range.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

This flag only applies if the range is associated with a Microsoft (r) server. If it is set to "true", the DHCP server will always update DNS, regardless of the requests from the DHCP clients. If this flag is set to false, clients will be allowed to update DNS when they request it.

Parameter

Specify "true" to set the always_update_dns flag or "false" to deactivate/unset it. The default value is "false".

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get always_update_dns
my $always_update_dns = $dhcp_range->always_update_dns();
#Modify always_update_dns
$dhcp_range->always_update_dns("false");
```

bootfile( )

Use this method to set or retrieve a bootfile name which is the name of the file the client must download.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

This option is overridden independently from Infoblox::DHCP::Range->bootserver() and Infoblox::DHCP::Range->nextserver().

Parameter

The boot file name in string format with a maximum of 128 bytes. The default value is undefined.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get bootfile
my $bootfile = $dhcp_range->bootfile();
#Modify bootfile
$dhcp_range->bootfile("bootfile2.com");
#Un-override bootfile
$dhcp_range->bootfile(undef);
```

bootserver( )

Use this method to set or retrieve a bootserver name which is the name of the server where the boot file is stored.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

This option is overridden independently from Infoblox::DHCP::Range->bootfile() and Infoblox::DHCP::Range->nextserver().

Parameter

The boot server IP address or name in FQDN (Fully Qualified Domain Name) format. The FQDN consists of the host name followed by the domain name (example: abc.com). A boot server name can have a maximum of 256 bytes. The default value is undefined.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get bootserver
my $bootserver = $dhcp_range->bootserver();
#Modify bootserver
$dhcp_range->bootserver("anotherserver.com");
#Un-override bootserver
$dhcp_range->bootserver(undef);
```

comment( )

Use this method to set or retrieve a descriptive comment.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

Desired comment in string format with a maximum of 256 bytes. The default value is undefined.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get comment
my $comment = $dhcp_range->comment();
#Modify comment
$dhcp_range->comment("Modifying the DHCP range comment");
```

cloud_info( )

Use this method to set or retrieve cloud API related information for the Infoblox::DHCP::Range object.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

The valid value is a Infoblox::Grid::CloudAPI::Info object.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
# Get cloud_info
my $cloud_info = $dhcp_range->cloud_info();
# Modify cloud_info
$dhcp_range->cloud_info($cloud_info_object);
```

ddns_domainname( )

Use this method to set or retrieve the ddns_domainname value of a DHCP Range object.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Setting this method to a defined value implicitly sets the override_ddns_domainname method to "true". Setting the parameter to undefined causes the appliance to use the grid default and automatically resets the override_ddns_domainname attribute to "false".

Note that when ddns_domainname is set to a defined value and override_dddns_domainname is set to "false", the last operation takes precedence. Thus the sequence $object->ddns_domainname("testdomain"); $object->override_ddns_domainname("false"); will set override_ddns_domainname to "false", and the sequence $object->override_ddns_domainname("false"); $object->ddns_domainname("testdomain"); will result in override_ddns_domainname="true".

Parameter

The dynamic DNS domain name in string format.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get ddns_domainname
my $ddns_domainname = $dhcp_range->ddns_domainname();
#Modify ddns_domainname
$dhcp_range->ddns_domainname("test_domain.com");
```

ddns_generate_hostname( )

Use this method to set or retrieve the ddns_generate_hostname flag of a DHCP range.

Enable this method to allow only the DHCP server to generate a hostname.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

Specify "true" to set the ddns_generate_hostname flag or "false" to deactivate/unset it. The default value is undefined.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get ddns_generate_hostname
my $ddns_generate_hostname = $dhcp_range->ddns_generate_hostname();
#Modify ddns_generate_hostname
$dhcp_range->ddns_generate_hostname("false");
```

deny_all_clients( )

Use this method to set or retrieve the deny_all_clients flag of a DHCP range.

Enable this method to deny all lease requests for the DHCP range.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

Specify "true" to set the deny_all_clients flag or "false" to deactivate/unset it. The default value for this field is false.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get deny_all_clients
my $deny_all_clients = $dhcp_range->deny_all_clients();
#Modify deny_all_clients
$dhcp_range->deny_all_clients("false");
```

deny_bootp( )

Use this method to set or retrieve the deny_bootp flag of a DHCP range.

Enable this method to deny all BOOTP requests.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

Specify "true" to set the deny_bootp flag or "false" to deactivate/unset it. The default value is false which indicates that this attribute inherit the upper-level setting.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get deny_bootp
my $deny_bootp = $dhcp_range->deny_bootp();
#Modify deny_bootp
$dhcp_range->deny_bootp("false");
```

disable( )

Use this method to set or retrieve the disable flag of a DHCP range.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

Specify "true" to set the disable flag or "false" to deactivate/unset it. The default value for this field is false.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get disable
my $disable = $dhcp_range->disable();
#Modify disable
$dhcp_range->disable("false");
```

discovery_basic_poll_setting( )

Use this method to set or retrieve the discovery_basic_poll_setting value of a DHCP Range.

Include the specified parameter to set the attribute value.

Parameter

The valid value is object of the Infoblox::Grid::Discovery::BasicPollSettings manpage.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get discovery_basic_poll_setting
my $discovery_basic_poll_setting = $dhcp_range->discovery_basic_poll_setting();
#Modify discovery_basic_poll_setting
$dhcp_range->discovery_basic_poll_setting($poll_setting);
```

discovery_member( )

Use this method to set or retrieve the discovery_member value of a DHCP Range.

Parameter

The discovery member name in string format.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get discovery_member
my $discovery_member = $dhcp_range->discovery_member();
#Modify discovery_member
$dhcp_range->discovery_member("test.localdomain");
```

discover_now_status( )

Use this method to retrieve the current discovery status. This attribute is read-only.

'discover_now_status' will not be returned by default, use 'return_methods' to request it if needed.

Parameter

None.

Returns

The method returns the attribute value that can be 'COMPLETE', 'FAILED', 'NONE', 'PENDING' or 'RUNNING'.

Example

```
#Get discover_now_status
my $discover_now_status = $dhcp_range->discover_now_status();
```

discovery_blackout_setting( )

Use this method to set or retrieve the discovery blackout setting.

Setting this method to a defined value implicitly sets the override_blackout_setting method to "true". If port_control_blackout_setting is undefined, setting the parameter to undefined causes the appliance to use the grid default and automatically resets the override_blackout_setting attribute to "false".

Note that when discovery_blackout_setting is set to a defined value and override_blackout_setting is set to "false", the last operation takes precedence. Thus the sequence $network->discovery_blackout_setting($blackout); $network->override_blackout_setting("false");

will set override_blackout_setting to "false", and the sequence $network->override_blackout_setting("false"); $network->discovery_blackout_setting($blackout); will result in override_blackout_setting="true".

Parameter

Valid value is an Infoblox::Grid::Discovery::Properties::BlackoutSetting object.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get discovery_blackout_setting
my $discovery_blackout_setting = $dhcp_range->discovery_blackout_setting();
#Modify discovery_blackout_setting
$dhcp_range->discovery_blackout_setting($blackout);
```

enable_discovery( )

Use this method to set or retrieve the enable_discovery value of a DHCP Range. This setting overrides the member level settings.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Setting this method to a defined value implicitly sets the override_enable_discovery method to "true". Setting the parameter to undefined causes the appliance to use the default from the upper level and automatically resets the override_enable_discovery attribute to "false".

Enable this flag if you want to set discovery for this object.

Parameter

Specify "true" to set the enable_discovery flag or "false" to deactivate/unset it. The default value is undefined.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get enable_discovery
my $enable_discovery = $dhcp_range->enable_discovery();
#Modify enable_discovery
$dhcp_range->enable_discovery("false");
```

enable_immediate_discovery( )

Use this method to set or retrieve enable_immediate_discovery value of a DHCP Range.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

Specify "true" to set the enable_immediate_discovery flag or "false" to deactivate/unset it. The default value is undefined.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get enable_immediate_discovery
my $enable_immediate_discovery = $dhcp_range->enable_immediate_discovery();
#Modify enable_immediate_discovery
$dhcp_range->enable_immediate_discovery("false");
```

email_list( )

Use this method to set or retrieve the email list to which the appliance sends notifications when DHCP address usage for the grid crosses a threshold.

If you enter email addresses in the email_list, then the appliance sends DHCP threshold alarms to the email addresses.

Parameter

The valid value is an array reference that contains string objects.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get email_list
my @email_list = $range->email_list();
#Modify email_list
$range->email_list( [ "admin\@test.com" ] );
```

enable_ddns( )

Use this method to set or retrieve the enable_ddns flag of a DHCP range.

Enable this method to allow dynamic DNS updates.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

Specify "true" to set the enable_ddns flag or "false" to deactivate/unset it. The default value is undefined.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get enable_ddns
my $enable_ddns = $dhcp_range->enable_ddns();
#Modify enable_ddns
$dhcp_range->enable_ddns("false");
```

enable_dhcp_thresholds( )

Use this method to set or retrieve DHCP thresholds at the DHCP address range level. These thresholds are the high and low watermarks. If DHCP range usage goes above or below these percentage values, the Infoblox appliance generates a syslog message and sends an SNMP trap, if configured.

Enable this flag to set DHCP threshold values.

Parameter

Specify "true" to set the enable_dhcp_thresholds flag or "false" to deactivate/unset it. The default value is undefined.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get enable_dhcp_thresholds
my $enable_dhcp_thresholds = $range->enable_dhcp_thresholds();
#Modify enable_dhcp_thresholds
$range->enable_dhcp_thresholds("false");
```

enable_ifmap_publishing( )

Use this method to set or retrieve the ifmap publishing option of a DHCP Range object. This setting overrides the member level settings.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

Specify "true" to set ifmap publishing option or "false" to deactivate/unset it. The default value is 'false', which indicates that this attribute inherits the member level setting.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
# Get enable_ifmap_publishing
my $enable_ifmap_publishing = $dhcp_range->enable_ifmap_publishing()
# Modify enable_ifmap_publishing
$dhcp_range->enable_ifmap_publishing("true");
# Un-override enable_ifmap_publishing
$dhcp_range->enable_ifmap_publishing(undef);
```

enable_email_warnings( )

Use this method to enable the appliance to send email notifications when DHCP address usage crosses a high or low watermark threshold.

Enable this flag if you want the appliance to send DHCP threshold warnings to the "email_list" provided when DHCP threshold is enabled.

Parameter

Specify "true" to set the enable_email_warnings flag or "false" to deactivate/unset it. The default value is undefined.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get enable_email_warnings
my $enable_email_warnings = $range->enable_email_warnings();
#Modify enable_email_warnings
$range->enable_email_warnings("false");
```

enable_known_clients_option( )

Use this method to set or retrieve the enable_known_clients_option flag of a DHCP range.

You must set the enable_known_clients_option flag to true in order to use the known_clients_option attribute.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

Specify "true" to set the enable_known_clients_option flag or "false" to deactivate/unset it. The default value is "false".

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get enable_known_clients_option
my $enable_known_clients_option = $dhcp_range->enable_known_clients_option();
#Modify enable_known_clients_option
$dhcp_range->enable_known_clients_option("false");
```

enable_snmp_warnings( )

Use this method to enable the appliance to send an SNMP trap when DHCP address usage crosses a watermark threshold.

If the flag is enabled, then the appliance sends SNMP traps when DHCP address usage crosses a watermark threshold when DHCP threshold is enabled.

Parameter

Specify "true" to set the enable_snmp_warnings flag or "false" to deactivate/unset it. The default value is undefined.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get enable_snmp_warnings
my $enable_snmp_warnings = $range->enable_snmp_warnings();
#Modify enable_snmp_warnings
$range->enable_snmp_warnings("false");
```

enable_unknown_clients_option( )

Use this method to set or retrieve the enable_unknown_clients_option flag of a DHCP range.

You must set the enable_unknown_clients_option flag to true in order to use the unknown_clients_option attribute.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

Specify "true" to set the enable_unknown_clients_option flag or "false" to deactivate/unset it. The default value is "false".

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get enable_unknown_clients_option
my $enable_unknown_clients_option = $dhcp_range->enable_unknown_clients_option();
#Modify enable_unknown_clients_option
$dhcp_range->enable_unknown_clients_option("false");
```

end_addr( )

Use this method to set or retrieve the end address of the DHCP range.

An end address is the last IP address in the DHCP range available for the clients.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

Specify the end address of the DHCP range in IPv4 address format. An IPv4 address is a 32-bit number in dotted decimal notation. It consists of four 8-bit groups of decimal digits separated by decimal points (example: 192.168.1.2).

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get end_addr
my $end_addr = $dhcp_range->end_addr();
#Modify end_addr
$dhcp_range->end_addr("10.0.0.20");
```

endpoint_sources( )

Use this method to retrieve the list of endpoints that provide object data. This is a read-only attribute.

Omit the parameter to retrieve the attribute value.

Parameter

None

Returns

The valid return value is an array of Infoblox::CiscoISE::Endpoint objects.

Example

```
#Get endpoint_sources value
my $endpoint_sources = $object->endpoint_sources();
```

exclude( )

Use this method to set or retrieve the exclusion range of IP addresses in the DHCP range which should not be assigned to clients.

If static addresses in the middle of an DHCP range are assigned to certain hosts, those addresses can be excluded from the DHCP range using this method.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

Valid value is an array reference that contains defined Infoblox::DHCP::ExclusionRange objects. The default value is empty.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get exclude
my $ref_exclude = $dhcp_range->exclude();
#Modify exclude
$dhcp_range->exclude([$exl1]); #$exl1 is an Infoblox::DHCP::ExclusionRange object
```

extattrs( )

Use this method to set or retrieve the extensible attributes associated with a DHCP range object.

Parameter

Valid value is a hash reference containing the names of extensible attributes and their associated values (Infoblox::Grid::Extattr objects).

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get extattrs
my $ref_extattrs = $dhcp_range->extattrs();
#Modify extattrs
$dhcp_range->extattrs({ 'Site' => $extattr1, 'Administrator' => $extattr2 });
```

extensible_attributes( )

Use this method to set or retrieve the extensible attributes associated with a DHCP range object

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

For valid values for extensible attributes, see Infoblox::Grid::ExtensibleAttributeDef/Extensible Attribute Values.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get extensible attributes
my $ref_extensible_attributes = $dhcp_range->extensible_attributes();
#Modify extensible attributes
$dhcp_range->extensible_attributes({ 'Site' => 'Santa Clara', 'Administrator' => [ 'Peter', 'Tom' ]
});
```

failover_assoc( )

Use this method to set or retrieve the failover peer association name of the DHCP range.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

If the attribute is set, the server_association_type attribute will also be set to "FAILOVER" if the name is defined, or to "NONE" if the member is also undefined.

Parameter

Specify the failover peer association name in string format. The default value is undefined.

Use an empty string ('') to remove a failover peer association from the DHCP range. Using undef is deprecated and may be changed in the future.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get failover_assoc
my $failover_assoc = $dhcp_range->failover_assoc();
#Modify failover_assoc
$dhcp_range->failover_assoc("peer2");
#Remove failover_assoc (using undef for removal is deprecated)
$dhcp_range->failover_assoc("");
```

filters( )

Use this method to set or retrieve the filters of the DHCP range.

Filters are used to screen the address assignments if the addresses in the DHCP range needs to be assigned to specific hosts.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

Valid value is an array reference that contains defined Infoblox::DHCP::FilterRule::MAC, Infoblox::DHCP::FilterRule::NAC, Infoblox::DHCP::FilterRule::RelayAgent, Infoblox::DHCP::FilterRule::Option, Infoblox::DHCP::FilterRule::Fingerprint objects. The default value is empty.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get filters
my $ref_filters = $dhcp_range->filters();
#Modify filters
$dhcp_range->filters([$exl1]); #$exl1 is an Infoblox::DHCP::FilterRule::MAC,
Infoblox::DHCP::FilterRule::NAC, Infoblox::DHCP::FilterRule::RelayAgent,
Infoblox::DHCP::FilterRule::Option or Infoblox::DHCP::FilterRule::Fingerprint object.
```

ignore_id( )

Use this method to set or retrieve the ignore_id type of a DHCP range object. Indicates whether the appliance will ignore DHCP client IDs or MAC addresses. This setting overrides the upper-level settings.

Parameter

Valid values are 'NONE', 'CLIENT', or 'MACADDR'. The default is 'NONE'.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
# Get attribute value
$value = $dhcp_range->ignore_id();
# Modify attribute value
$dhcp_range->ignore_id('CLIENT');
#Un-override ignore_id
$dhcp_range->ignore_id(undef);
```

ignore_mac_addresses( )

Use this method to set or retrieve the ignore_mac_addresses list of a DHCP range object. A list of MAC addresses the appliance will ignore. This setting overrides the upper-level settings.

If you enter MAC addresses in the ignore_mac_addresses, then the appliance will ignore them. If you enter empty array, then the

appliance will ignore any MAC address.

Parameter

The valid value is an array reference that contains string objects.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
# Get attribute value
$value = $dhcp_range->ignore_mac_addresses();
# Modify attribute value
$dhcp_range->ignore_mac_addresses(["11::11::11::11::11::11"]);
```

is_split_scope( )

Use this method to retrieve the is_split_scope flag of the DHCP range. This flag is "true" when the range is part of a split scope.

Parameter

None

Returns

The method returns the attribute value.

Example

```
#Get is_split_scope
my $is_split_scope = $dhcp_range->is_split_scope();
```

known_clients_option( )

Use this method to set or retrieve the known_clients_option option of a DHCP range.

Set this option to "Allow" or "Deny" to allow or prevent the allocation of an IP address from this pool to any client that has a host declaration.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

Specify "Allow" or "Deny" to set the known_clients_option flag value. The default value is undefined.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get known_clients_option
my $known_clients_option = $dhcp_range->known_clients_option();
#Modify known_clients_option
$dhcp_range->known_clients_option("Deny");
```

lease_scavenge_time( )

Use this method to set or retrieve the lease_scavenge_time value of a DHCP Range object.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Setting this method to a defined value implicitly sets the override_lease_scavenge_time method to "true". Setting the parameter to undefined causes the appliance to use the default from the upper level and automatically resets the override_lease_scavenge_time attribute to "false".

Note that when lease_scavenge_time is set to a defined value and override_lease_scavenge_time is set to "false", the last operation takes precedence. Thus the sequence $object->lease_scavenge_time(604800); $object->override_lease_scavenge_time("false"); will set override_lease_scavenge_time to "false", and the sequence $object->override_lease_scavenge_time("false"); $object->lease_scavenge_time(604800); will result in override_lease_scavenge_time="true".

Parameter

An integer that specifies the period of time, in seconds, that free and backup leases remain in the database before they are automatically deleted. To disable lease scavenging, set the parameter to -1. The minimum positive value must be greater than 86400 seconds (1 day). The default value of this parameter is -1, which means that lease scavenging is disabled.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get lease_scavenge_time
my $lease_scavenge = $dhcp_range->lease_scavenge_time();
#Modify lease_scavenge_time
$dhcp_range->lease_scavenge_time(30 * 24 * 60 * 60);
```

name( )

Use this method to set or retrieve a descriptive name for the range.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

The desired name in string format with a maximum of 256 bytes. The default value is undefined.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get name
my $name = $dhcp_range->name();
#Modify name
$dhcp_range->name("external");
```

next_available_ip( )

Use this method to get the next available IP `address(es)` in the DHCP range object that was retrieved. Note that to use this method, the object MUST be retrieved via a get or search call, or created with an add call.

Parameter

The method accepts the following optional parameters:

```
  requested_num - The number of IP addresses requested. It must be a value between 1 and 20,
inclusive.
  excluded - A reference to an array of addresses to be excluded from the results.
```

Returns

This method returns the next available IP `address(es)` in the range. If requested_num is not specified, a string with the next available address will be returned; otherwise, the method will return a reference to an array of addresses.

Within a DHCP Range object, the next available IP `address(es)` are the first addresses in this range that match all of the following rules:

```
   * Are not part of the passed 'excluded' parameter (if specified).
   * There is no fixed address or host address record matching this IP address
   * There is no unmanaged record matching this IP address
   * The IP address is not contained in any excluded range of this DHCP range
   * There is no active lease matching this IP address (Deleted leases in the recycle bin are not
considered.)
```

Example

```
# Retrieve a particular range
my @retrieved_objs = $session->get(
                        object     => "Infoblox::DHCP::Range",
                        start_addr => "10.0.0.20"
);

$dhcp_range = $retrieved_objs[0];

# Print the next available IP address in the range
print $dhcp_range->next_available_ip() . "\n";
```

```
# This will retrieve the next 10 available addresses and print the first one.
my $addressesref = $dhcp_range->next_available_ip(requested_num => 10);
print @$addressesref[0] . "\n";

# This will retrieve the next 10 available addresses and exclude a couple.
my $addressesref = $dhcp_range->next_available_ip(requested_num => 10, excluded => ["10.0.0.21",
"10.0.0.22"]);
```

ignore_dhcp_option_list_request( )

Use this method to set or retrieve the ignore_dhcp_option_list_request flag of a DHCP shared network object. If this flag is set to false all the defined DHCP options will be returned to the client. This setting overrides the upper-level settings.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

Specify "true" to set the ignore_dhcp_option_list_request flag or "false" to deactivate/unset it. The default value is undefined which indicates that this attribute inherit the upper-level setting..

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
# Get ignore_dhcp_option_list_request
my $ignore_dhcp_option_list_request = $dhcp_range->ignore_dhcp_option_list_request();
# Un-override ignore_dhcp_option_list_request
$dhcp_range->ignore_dhcp_option_list_request(undef);
# Modify ignore_dhcp_option_list_request
$dhcp_range->ignore_dhcp_option_list_request("true");
```

member( )

Use this method to set or retrieve the member or Microsoft server that serves the DHCP range.

The member or Microsoft server that serves DHCP for this IP address range.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

If the attribute is set, the server_association_type attribute will also be set to "MEMBER" if setting member to an Infoblox::DHCP::Member object, to "MS_SERVER" if setting member to an Infoblox::DHCP::MSServer object, or to "NONE" if failover_assoc is also empty.

Parameter

The valid value is a defined Infoblox::DHCP::Member or Infoblox::DHCP::MSServer object. The default value is empty.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get member
my $ref_member = $dhcp_range->member();
#Modify member
$dhcp_range->member($member1); #$member is an Infoblox::DHCP::Member object
```

ms_ad_user_data( )

Use this method to retrieve Microsoft Active Directory users related information. This is a read-only attribute.

Parameter

None

Returns

The valid return value is an Infoblox::Grid::MSServer::AdUser::Data object.

Example

```
#Get ms_ad_user_data
my $ms_ad_user_data = $dhcp_range->ms_ad_user_data();
```

ms_options( )

Use this method to set or retrieve the Microsoft server custom DHCP options that are associated with this DHCP range.

Note: You can set only options that are included in both the Microsoft server and NIOS predefined option definitions. Options that are not common to both are read only and cannot be modified.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

The valid value is an array reference that contains defined Infoblox::DHCP::MSOption objects. The default value is empty.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get options
my $ref_options = $dhcp_range->ms_options();
#Modify options
$dhcp_range->ms_options([$option12]); #$option12 is an Infoblox::DHCP::MSOption object
```

network( )

Use this method to set or retrieve the DHCP network of the address range.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

Specify the address and netmask of network in CIDR format (example: 10.0.0.0/8).

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get network
my $network = $dhcp_range->network();
```

network_view( )

Use this method to set or retrieve the network view of the DHCP range.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

The default value is the "default" network view, which means the DHCP range is in the default network view.

Parameter

Valid value is a defined Infoblox::DHCP::View object. The default network view value is the system-defined default network view object.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get network view
my $nview = $dhcp_range->network_view();
#Modify network view, with an Infoblox::DHCP::View object
$dhcp_range->network_view($nview);
```

nextserver( )

Use this method to set or retrieve the nextserver IP address which is the boot file server where the boot file is stored.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

This option is overridden independently from Infoblox::DHCP::Range->bootfile() and Infoblox::DHCP::Range->bootserver().

Parameter

The next server IP address or name in FQDN (Fully Qualified Domain Name) format. The FQDN consists of the host name followed by the domain name (example: abc.com). A next server name can have a maximum of 256 bytes. The default value is undefined.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get nextserver
my $nextserver = $dhcp_range->nextserver();
#Modify nextserver
$dhcp_range->nextserver("4.4.4.4");
#Un-override nextserver
$dhcp_range->nextserver(undef);
```

options( )

Use this method to set or retrieve the options of the DHCP range.

DHCP options describe network configuration settings and various services available on the network. Setting the DHCP options for the DHCP range overrides the options set at the network level.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

Valid value is an array reference that contains defined Infoblox::DHCP::Option objects. The default value is empty.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get options
my $ref_options = $dhcp_range->options();
#Modify options
$dhcp_range->options([$option12]); #$option12 is an Infoblox::DHCP::Option object
```

logic_filters( )

Use this method to set or retrieve all the logic (ordered) filters associated with the DHCP Range.

Filters are used to screen address assignments in a DHCP Range if the addresses are to be assigned to specific hosts.

Setting this method to a defined value implicitly sets the override_logic_filters method to "true". Setting the parameter to undefined causes the appliance to use the grid default and automatically resets the override_logic_filters attribute to "false".

Note that when logic_filters is set to a defined value and override_dlogic_filters is set to "false", the last operation takes precedence. Thus the sequence $object->logic_filters($filters); $object->override_logic_filters("false"); will set override_logic_filters to "false", and the sequence $object->override_logic_filters("false"); $object->logic_filters($filters); will result in override_logic_filters="true".

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

The valid value is an array reference that contains either the Infoblox::DHCP::Filter::MAC, Infoblox::DHCP::Filter::NAC, Infoblox::DHCP::Filter::Option objects or the names of the corresponding filters. The filters in the list must satisfy the following conditions:

Filter name must be globally unique.
Filters without expressions must be at the end of the list.
The filters should not be disabled.

The default value of the parameter is undefined.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get logic_filters
my $ref_logic_filters = $dhcp_range->logic_filters();
#Modify logic_filters
$dhcp_range->logic_filters(["filter_1","filter_2",$exl1]); #$exl1 is an Infoblox::DHCP::Filter::MAC,
Infoblox::DHCP::Filter::NAC or Infoblox::DHCP::Filter::Option object.
```

override_ddns_domainname( )

The override_ddns_domainname attribute controls whether the ddns_domainname value in the DHCP range is used, instead of the grid default.

The override_ddns_domainname attribute can be specified explicitly. It is also set implicitly when ddns_domainname is set to a defined value.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

Set the parameter to "true" to override the grid-level setting for ddns_domainname. Set the parameter to "false" to inherit the grid-level setting for ddns_domainname.

The default value of this parameter is "false".

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Getting override_ddns_domainname
my $override_ddns_domainname=$dhcp_range->override_ddns_domainname( );
#Modifying override_ddns_domainname
$dhcp_range->override_ddns_domainname("true");
```

override_enable_ifmap_publishing( )

The override_enable_ifmap_publishing controls whether the enable_ifmap_publishing in the DHCP Range is used, instead of the grid default.

The override_enable_ifmap_publishing can be specified explicitly. It is also set implicitly when enable_ifmap_publishing is set to a defined value.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

Set the parameter to "true" to override the grid-level setting for enable_ifmap_publishing. Set the parameter to "false" to inherit the grid-level setting for enable_ifmap_publishing.

The default value of this parameter is "false".

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Getting override_enable_ifmap_publishing
my $override_enable_ifmap_publishing=$dhcp_range->override_enable_ifmap_publishing( );
#Modifying override_enable_ifmap_publishing
$dhcp_range->override_enable_ifmap_publishing("true");
```

override_blackout_setting( )

The override_blackout_setting attribute controls whether the discovery_blackout_setting and port_control_blackout_setting values in the object are used, instead of the grid default.

The override_blackout_setting attribute can be specified explicitly. It is also set implicitly when discovery_blackout_setting or port_control_blackout_setting is set to a defined value.

Parameter

Specify "true" to set the override_blackout_setting flag or "false" to deactivate/unset it. The default value is "false".

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
# Get attribute value
my $value = $dhcp_range->override_blackout_setting();
# Modify attribute value
$dhcp_range->override_blackout_setting('true');
```

override_ignore_id( )

The override_ignore_id controls whether the ignore_id in the DHCP Range is used, instead of the upper-level default.

The override_ignore_id can be specified explicitly. It is also set implicitly when ignore_id is set to a defined value.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

Set the parameter to "true" to override the upper-level setting for ignore_id and ignore_mac_addresses. Set the parameter to "false" to inherit the upper-level setting for ignore_id and ignore_mac_addresses.

The default value of this parameter is "false".

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get attrubute value
my $value=$dhcp_range->override_ignore_id();
#Modifying attribute value
$dhcp_range->override_ignore_id('true');
```

override_lease_scavenge_time( )

The override_lease_scavenge_time attribute controls whether the lease_scavenge_time value in the DHCP range is used, instead of the upper-level default.

The override_lease_scavenge_time attribute can be specified explicitly. It is also set implicitly when lease_scavenge_time is set to a defined value.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

Set the parameter to "true" to override the upper-level setting for lease_scavenge_time. Set the parameter to "false" to inherit the setting for lease_scavenge_time from the upper level.

The default value of this parameter is "false".

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Getting override_lease_scavenge_time
my $override_lease_scavenge_time=$dhcp_range->override_lease_scavenge_time( );
#Modifying override_lease_scavenge_time
$dhcp_range->override_lease_scavenge_time("true");
```

override_logic_filters( )

The override_logic_filters attribute controls whether the logic_filters value in the DHCP Range is used, instead of the Grid default.

The override_logic_filters attribute can be specified explicitly. It is also set implicitly when logic_filters is set to a defined value.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

Set the parameter to "true" to override the Grid-level setting for logic_filters. Set the parameter to "false" to inherit the Grid-level setting for logic_filters.

The default value of this parameter is "false".

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Getting override_logic_filters
my $override_logic_filters=$dhcp_range->override_logic_filters( );
#Modifying override_logic_filters
$dhcp_range->override_logic_filters("true");
```

override_subscribe_settings( )

The override_subscribe_settings attribute controls whether the subscribe_settings method values of the member are used, instead of the Grid default.

The override_subscribe_settings attribute can be specified explicitly. It is also set implicitly when subscribe_settings is set to a defined value.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

Set the parameter to "true" to override the Grid-level setting for subscribe_settings. Set the parameter to "false" to inherit the Grid-level setting for subscribe_settings.

The default value of this parameter is "false".

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get override_subscribe_settings value
my $override_subscribe_settings = $object->override_subscribe_settings( );
#Modify override_subscribe_settings value
$object->override_subscribe_settings("true");
```

override_update_dns_on_lease_renewal( )

The override_update_dns_on_lease_renewal attribute controls whether the update_dns_on_lease_renewal value in the DHCP range is used, instead of the grid default.

The override_update_dns_on_lease_renewal attribute can be specified explicitly. It is also set implicitly when update_dns_on_lease_renewal is set to a defined value.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

Set the parameter to "true" to override the grid-level setting for update_dns_on_lease_renewal. Set the parameter to "false" to inherit the grid-level setting for update_dns_on_lease_renewal.

The default value of this parameter is "false".

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Getting override_update_dns_on_lease_renewal
my $override_update_dns_on_lease_renewal=$dhcp_range->override_update_dns_on_lease_renewal( );
#Modifying override_update_dns_on_lease_renewal
$dhcp_range->override_update_dns_on_lease_renewal("true");
```

port_control_blackout_setting

Use this method to set or retrieve the port control blackout setting.

Setting this method to a defined value implicitly sets the override_blackout_setting method to "true". If discovery_blackout_setting is undefined, setting the parameter to undefined causes the appliance to use the grid default and automatically resets the override_blackout_setting attribute to "false".

Note that when port_control_blackout_setting is set to a defined value and override_blackout_setting is set to "false", the last operation takes precedence. Thus the sequence $network->port_control_blackout_setting($blackout); $network->override_blackout_setting("false"); will set override_blackout_setting to "false", and the sequence $network->override_blackout_setting("false"); $network->port_control_blackout_setting($blackout); will result in override_blackout_setting="true".

Parameter

Valid value is an Infoblox::Grid::Discovery::Properties::BlackoutSetting object.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get port_control_blackout_setting
my $port_control_blackout_setting = $dhcp_range->port_control_blackout_setting();
#Modify port_control_blackout_setting
$dhcp_range->port_control_blackout_setting($blackout);
```

pxe_lease_time( )

Use this method to set or retrieve PXE Lease Time value of the DHCP range.

The Preboot Execution Environment Lease Time value is used by some hosts to boot remotely from a server.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

A 32-bit integer (range from 0 to 4294967295) represents the duration in seconds that the record is cached. Zero indicates that the record should not be cached. The default value is undefined.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get pxe_lease_time
my $pxe_lease_time = $dhcp_range->pxe_lease_time();
#Modify pxe_lease_time
$dhcp_range->pxe_lease_time("7200");
```

range_high_water_mark( )

Use this method to set or retrieve the range_high_water_mark value. If the percentage of allocated addresses exceeds this value, an SNMP trap is sent, unless this is a range served by a Microsoft server.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

A number that specifies the percentage of allocated addresses. The range is from 1 to 100. The high watermark value must be greater than the low watermark value. The default value is 95.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get range_high_water_mark
my $range_high_water_mark = $range->range_high_water_mark();
#Modify range_high_water_mark
$range->range_high_water_mark(80);
```

range_high_water_mark_reset( )

Use this method to set or retrieve the range_high_water_mark_reset value. If the percentage of allocated addresses drops below this value, a corresponding SNMP trap is reset.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

A number that specifies the percentage of allocated addresses. The range is from 1 to 100. The high watermark reset value must be lower than the high watermark value. The default value is 85.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get range_high_water_mark_reset
my $range_high_water_mark_reset = $range->range_high_water_mark_reset();
#Modify range_high_water_mark_reset
$range->range_high_water_mark_reset(70);
```

range_low_water_mark( )

Use this method to set or retrieve the range_low_water_mark value. If the percentage of allocated addresses drops below this watermark, an SNMP trap is sent, unless this is a range served by a Microsoft server.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

A number that specifies the percentage of allocated addresses. The range is from 1 to 100. The low watermark value must be lower than the high watermark value. The default low watermark number is 0.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get range_low_water_mark
my $range_low_water_mark = $range->range_low_water_mark();
#Modify range_low_water_mark
$range->range_low_water_mark(10);
```

range_low_water_mark_reset( )

Use this method to set or retrieve the range_low_water_mark_reset value. If the percentage of allocated addresses exceeds this value, a corresponding SNMP trap is reset.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

A number that specifies the percentage of allocated addresses. The range is from 1 to 100. The low watermark reset value must be higher than the low watermark value. The default low watermark reset number is 0.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get range_low_water_mark_reset
```

```
my $range_low_water_mark_reset = $range->range_low_water_mark_reset();
#Modify range_low_water_mark_reset
$range->range_low_water_mark_reset(10);
```

restart_if_needed( )

Use this method to set the flag that indicates whether DHCP service should be restarted on a member that is serving DHCP for the changed object referenced in a cloud API request.

Parameter

Specify true to restart the DHCP service.

Returns

None

Example

```
# Modify
$network->restart_if_needed('true');
```

recycle_leases( )

Use this method to set or retrieve the recycle_leases flag of the DHCP range.

If the flag is enabled, the leases are kept in recycle bin until one week after expiration. When the flag is disabled, the leases are irrecoverably deleted.

Parameter

Specify "true" to set the recycle_leases flag or "false" to deactivate/unset it. The default value is undefined.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get recycle_leases
my $recycle_leases = $dhcp_range->recycle_leases();
#Modify recycle_leases
$dhcp_range->recycle_leases("false");
```

same_port_control_discovery_blackout( )

Use this method to set or retrieve the flag that enables using the same discovery blackout for port control blackout.

Parameter

Specify 'true' to use same discovery blackout for port control blackout or 'false' to disable it. The default value is 'false'.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
# Get attribute value
my $value = $dhcp_range->same_port_control_discovery_blackout();
# Modify attribute value
$dhcp_range->same_port_control_discovery_blackout('true');
```

server_association_type( )

Use this method to set or retrieve the DHCP range server association type.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

The server_association_type attribute can be specified explicitly. It is also set implicitly when member or failover_assoc are set.

Parameter

Valid values are "NONE", "FAILOVER", "MS_SERVER" and "MEMBER".

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get server_association_type
my $server_association_type = $dhcp_range->server_association_type();
#Modify server_association_type
$dhcp_range->server_association_type("MEMBER");
```

split_member

Use this method to set the address or FQDN of the Microsoft server that is serving the split scope. Note that this method applies only when the range is first added.

Include the specified parameter to set the attribute value.

Parameter

The valid value is a defined Infoblox::DHCP::MSServer object. The default value is empty.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

Example

```
#Set split_member prior to the range insertion
$dhcp_range->split_member($member); #$member is an Infoblox::DHCP::Member object
```

split_scope_exclusion_percent

Use this method to set the percentage for the split scope exclusion. For example, if this is set to 40, then 60% of the range will be assigned to the Microsoft server specified in split_member. Note this method has an effect only when the range is first added.

Include the specified parameter to set the attribute value.

Parameter

An unsigned integer between 1 and 99. The default is undefined.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

Example

```
#Set split_scope_exclusion_percent prior to the range insertion
$dhcp_range->split_scope_exclusion_percent(35);
```

start_addr( )

Use this method to set or retrieve the start address of the DHCP range.

A start address is the first IP address in the DHCP range available for the clients.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

Specify the start address of the DHCP range in IPv4 address format. An IPv4 address is a 32-bit number in dotted decimal notation. It consists of four 8-bit groups of decimal digits separated by decimal points (example: 192.168.1.2).

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get start_addr
my $start_addr = $dhcp_range->start_addr();
#Modify start_addr
$dhcp_range->start_addr("10.0.0.2");
```

subscribe_settings( )

Use this method to set or retrieve the Cisco ISE subscribe settings.

Setting this method to a defined value implicitly sets the override_subscribe_settings method to "true". Setting the parameter to undefined causes the appliance to use the Cisco ISE endpoint default and automatically resets the override_subscribe_settings method to "false".

Note that when subscribe_settings is set to a defined value and override_subscribe_settings is set to "false", the last operation takes precedence. Thus the sequence $object->subscribe_settings($value); $object->override_subscribe_settings("false"); will set override_subscribe_settings to "false", and the sequence $object->override_subscribe_settings("false"); $object->subscribe_settings($value); will result in override_subscribe_settings="true".

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

The valid value is an Infoblox::CiscoISE::SubscribeSetting object.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get subscribe_settings value
my $subscribe_settings = $object->subscribe_settings();
#Modify subscribe_settings value
$object->subscribe_settings($settings);
```

template( )

Use this method to set or retrieve the DHCP range template name for a DHCP Range object.

Template is the name of a DHCP range template which will be used to create the DHCP range.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

The valid name in sting format.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

The template parameter can be used to only add a new DHCP range and cannot be used during object modify or returned during object get.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get template
my $template = $dhcp_range->template();
#Modify template
$dhcp_range->template("Template_Name");
```

unknown_clients_option( )

Use this method to set or retrieve the unknown_clients_option option of a DHCP range.

Set this option to "Allow" or "Deny" to allow or prevent the allocation of an IP address from this pool to any client that has no host declaration.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

Specify "Allow" or "Deny" to set the unknown_clients_option flag value. The default value is undefined.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get unknown_clients_option
my $unknown_clients_option = $dhcp_range->unknown_clients_option();
#Modify unknown_clients_option
$dhcp_range->unknown_clients_option("Deny");
```

update_dns_on_lease_renewal( )

The update_dns_on_lease_renewal attribute controls whether the DHCP server updates DNS when a DHCP lease is renewed.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Setting this method to a defined value implicitly sets the override_update_dns_on_lease_renewal method to "true". Setting the parameter to undefined causes the appliance to use the grid default and automatically resets the override_update_dns_on_lease_renewal attribute to "false".

Note that when update_dns_on_lease_renewal is set to a defined value and override_update_dns_on_lease_renewal is set to "false", the last operation takes precedence. Thus the sequence $object->update_dns_on_lease_renewal("true"); $object->override_update_dns_on_lease_renewal("false"); will set override_update_dns_on_lease_renewal to "false", and the sequence $object->override_update_dns_on_lease_renewal("false"); $object->update_dns_on_lease_renewal("true"); will result in override_update_dns_on_lease_renewal="true".

Parameter

Specify "true" to set the update_dns_on_lease_renewal flag or "false" to deactivate/unset it.

The default value of this parameter is "false".

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get update_dns_on_lease_renewal
my $update_dns_on_lease_renewal=$dhcp_range->update_dns_on_lease_renewal( );
#Modify update_dns_on_lease_renewal
$dhcp_range->update_dns_on_lease_renewal("true");
```

SAMPLE CODE

The following sample code demonstrates the different functions that can be applied to a DHCP range object, such as add, search, modify, and remove. This sample also includes error handling for the operations.

#Preparation prior to a DHCP Range object insertion

```
#PROGRAM STARTS: Include all the modules that will be used
use strict;
use Infoblox;

#Create a session to the Infoblox appliance
my $session = Infoblox::Session->new(
               master   => "192.168.1.2", #appliance host ip
               username => "admin",       #appliance user login
               password => "infoblox"     #appliance password
);
unless ($session) {
       die("Construct session failed: ",
            Infoblox::status_code() . ":" . Infoblox::status_detail());
}
print "Session created successfully\n";

#Create a DHCP Member object
my $memberdhcp = Infoblox::DHCP::Member->new(
    name     => "infoblox.localdomain",
    ipv4addr => "192.168.1.2"
);
unless($memberdhcp) {
       die("Construct member failed: ",
            Infoblox::status_code() . ":" . Infoblox::status_detail());
}
print "DHCP Member object created successfully\n";

#Create the Network object with the member
```

```
my $network = Infoblox::DHCP::Network->new(
    network => "10.0.0.0/8",
    comment => "add network",
    members => [ $memberdhcp ]
);
unless($network) {
       die("Construct Network object failed: ",
            Infoblox::status_code() . ":" . Infoblox::status_detail());
}
print "Network object created successfully\n";

#Add the Network object into the Infoblox appliance through a session
$session->add($network)
    or die("Add Network object failed: ",
            $session->status_code() . ":" . $session->status_detail());
print "Network object added to Infoblox appliance successfully\n";

#Create the Mac Filter object
my $mac_filter = Infoblox::DHCP::Filter::MAC->new(
    "name"    => "filter1",
    "comment" => "5 minutes"
);

unless($mac_filter) {
       die("Construct Mac Filter object failed: ",
            Infoblox::status_code() . ":" . Infoblox::status_detail());
}
print "Mac Filter object created successfully\n";

#Add the Mac Filter object into the Infoblox appliance through a session
$session->add($mac_filter)
    or die("Add Mac Filter object failed: ",
            $session->status_code() . ":" . $session->status_detail());
print "Mac Filter object added to Infoblox appliance successfully\n";

#Construct Custom Option object
my $option1 = Infoblox::DHCP::Option->new(
    type  => "1",
    value => "255.0.0.0", );

#Construct DHCP Mac Filter rule object
my $filter = Infoblox::DHCP::FilterRule::MAC->new(
    filter_name => "filter1",
    permission  => "grant"
);

#Construct exclusion range object
my $exclusion_range = Infoblox::DHCP::ExclusionRange->new(
    start_address => "10.0.0.5",
    end_address   => "10.0.0.7",
    comment       => "Excluded range"
);
```

#Create a DHCP Range object

```
my $dhcp_range = Infoblox::DHCP::Range->new(
        end_addr                => "10.0.0.10",
        network                 => "10.0.0.0/8",
        start_addr              => "10.0.0.1",
        bootfile                => "bootfile1.com",
        bootserver              => "someserver.com",
        comment                 => "add range",
        ddns_generate_hostname  => "true",
        deny_all_clients        => "true",
        deny_bootp              => "true",
        disable                 => "true",
        enable_ddns             => "true",
        exclude                 => [$exclusion_range],
        filters                 => [$filter],
        member                  => $memberdhcp,
        nextserver              => "3.3.3.3",
        options                 => [$option1],
        pxe_lease_time          => "3600",
        recycle_leases          => "true",
);
unless($dhcp_range) {
```

```
        die("Construct DHCP Range object failed: ",
            Infoblox::status_code() . ":" . Infoblox::status_detail());
}
print "DHCP Range object created successfully\n";

#Add the DHCP Range object into the Infoblox appliance through a session
$session->add($dhcp_range)
    or die("Add DHCP Range object failed: ",
            $session->status_code() . ":" . $session->status_detail());
print "DHCP Range object added to the network successfully\n";
```

#Create a DHCP Range object using a template

```
my $dhcp_range_template = Infoblox::DHCP::RangeTemplate->new(
        name                    => "custom_range_template",
        offset                  => "10",
        number_of_addresses     => "10",
);
unless($dhcp_range_template) {
        die("Construct DHCP Range template object failed: ",
            Infoblox::status_code() . ":" . Infoblox::status_detail());
}
print "DHCP Range template object created successfully\n";

#Add the DHCP Range template object into the Infoblox appliance through a session
$session->add($dhcp_range_template)
    or die("Add DHCP Range template object failed: ",
            $session->status_code() . ":" . $session->status_detail());
print "DHCP Range template object added to the network successfully\n";

my $dhcp_range = Infoblox::DHCP::Range->new(
        start_addr              => "10.0.0.11",
        end_addr                => "10.0.0.15",
        network                 => "10.0.0.0/8",
        template                => "custom_range_template"
);
unless($dhcp_range) {
        die("Construct DHCP Range object failed: ",
            Infoblox::status_code() . ":" . Infoblox::status_detail());
}
print "DHCP Range object created successfully\n";

#Add the DHCP Range object into the Infoblox appliance through a session
$session->add($dhcp_range)
    or die("Add DHCP Range object failed: ",
            $session->status_code() . ":" . $session->status_detail());
print "DHCP Range object added to the network successfully\n";
```

#Get and modify a DHCP Range

```
#Get a DHCP Range through the session
my @retrieved_objs = $session->get(
    object     => "Infoblox::DHCP::Range",
    start_addr => "10.0.0.1",
);
my $object = $retrieved_objs[0];
unless ($object) {
        die("Get DHCP Range failed: ",
            $session->status_code() . ":" . $session->status_detail());
}
print "Get DHCP Range object found at least 1 matching entry\n";

#Modify one of the attributes of the specified DHCP Range
$object->comment ("modified comment");

#Un-override bootfile
$object->bootfile(undef);

#Un-override bootserver
$object->bootserver(undef);

#Un-override nextserver
$object->nextserver(undef);

#Apply the changes
$session->modify($object)
    or die("Modify DHCP Range failed: ",
```

```
            $session->status_code() . ":" . $session->status_detail());
print "DHCP Range object modified successfully \n";
```

#Search a DHCP Range

```
# search for DHCP range object with start address having last octet as 1
my @retrieved_objs = $session->search(
    object     => "Infoblox::DHCP::Range",
    network    => "10.0.0.0/.",
    start_addr => ".*1",
    comment    => ".*comment"
);

my $object = $retrieved_objs[0];
unless ($object) {
    die("Search DHCP range failed: ",
        $session->status_code() . ":" . $session->status_detail());
}
print "Search DHCP range object found at least 1 matching entry\n";
```

#Remove a DHCP Range

```
#Get a DHCP Range through the session
my @retrieved_objs = $session->get(
    object     => "Infoblox::DHCP::Range",
    start_addr => "10.0.0.1",
    end_addr   => "10.0.0.10",
);

my $object = $retrieved_objs[0];
unless ($object) {
       die("Get DHCP Range failed: ",
            $session->status_code() . ":" . $session->status_detail());
}
print "Get DHCP Range object found at least 1 matching entry\n";

#submit the object for removal
$session->remove($object)
    or die("Remove DHCP Range failed: ",
            $session->status_code() . ":" . $session->status_detail());
print "DHCP Range object removed successfully \n";

#submit the Range Template object for removal
$session->remove($dhcp_range_template)
    or die("Remove DHCP Range Tempate failed: ",
            $session->status_code() . ":" . $session->status_detail());
print "DHCP Range Templateobject removed successfully \n";

####PROGRAM ENDS####
```

AUTHOR

Infoblox Inc. http://www.infoblox.com/

SEE ALSO

Infoblox::Session, Infoblox::Session->add(), Infoblox::Session->get(), Infoblox::Session->modify(), Infoblox::Session->remove(), Infoblox::Session->search(),Infoblox::DHCP::ViewInfoblox::DHCP::ExclusionRange, Infoblox::DHCP::FilterRule::MAC, Infoblox::DHCP::FilterRule::NAC, Infoblox::DHCP::FilterRule::RelayAgent, Infoblox::DHCP::FilterRule::Option, Infoblox::DHCP::Member, Infoblox::DHCP::Option, the Infoblox::DHCP::RangeTemplate manpage, Infoblox::Grid::MSServer::AdUser::Data

COPYRIGHT

Copyright (c) 2017 Infoblox Inc.

Infoblox::DHCP::RangeTemplate - DHCP RangeTemplate object.

- NAME
- DESCRIPTION
- CONSTRUCTOR
- MODULE METHODS
  - Infoblox::DHCP::NetworkTemplate->range_templates( )
- SESSION METHODS
  - Infoblox::Session->add( )
  - Infoblox::Session->get( )
  - Infoblox::Session->search( )
  - Infoblox::Session->modify( )
  - Infoblox::Session->remove( )
- METHODS
  - bootfile( )
  - bootserver( )
  - comment( )
  - ddns_domainname( )
  - ddns_generate_hostname( )
  - deny_all_clients( )
  - deny_bootp( )
  - enable_ddns( )
  - enable_known_clients_option( )
  - enable_unknown_clients_option( )
  - exclude( )
  - extattrs( )
  - extensible_attributes( )
  - failover_assoc( )
  - filters( )
  - known_clients_option( )
  - lease_scavenge_time( )
  - logic_filters( )
  - member( )
  - name( )
  - nextserver( )
  - number_of_addresses( )
  - offset( )
  - options( )
  - override_ddns_domainname( )
  - override_lease_scavenge_time( )
  - override_logic_filters( )
  - override_update_dns_on_lease_renewal( )
  - pxe_lease_time( )
  - range_high_water_mark( )
  - range_high_water_mark_reset( )
  - range_low_water_mark( )
  - range_low_water_mark_reset( )
  - server_association_type( )
  - recycle_leases( )
  - unknown_clients_option( )
  - update_dns_on_lease_renewal( )
- SAMPLE CODE
- AUTHOR
- SEE ALSO
- COPYRIGHT

NAME

Infoblox::DHCP::RangeTemplate - DHCP RangeTemplate object.

DESCRIPTION

A DHCP range template works as a 'cookie cutter' and can be used to create real DHCP range object in a quick and consistent way. The DHCP range object created from a DHCP range template will inherit most properties defined in range template object. So, most of the DHCP range template properties are the same as the DHCP range object properties.

CONSTRUCTOR

```
my $dhcp_range_template = Infoblox::DHCP::RangeTemplate->new(
```

```
     name                                => $string,
# Required
     number_of_addresses                 => $num,
# Required
     offset                              => $num,
# Required
     bootfile                            => $filename,
# Optional / Default is undefined
     bootserver                          => $ipv4addr | $fqdn | undef,
# Optional / Default is undefined
     comment                             => $string,
# Optional / Default is undefined
     ddns_domainname                     => $string | undef,
# Optional / Default is undefined
     ddns_generate_hostname              => "true" | "false",
# Optional / Default is undefined
     deny_all_clients                    => "true" | "false",
# Optional / Default is "false"
     deny_bootp                          => "true" | "false" | undef,
# Optional / Default is undefined
     enable_ddns                         => "true" | "false",
# Optional / Default is undefined
     enable_known_clients_option         => "true" | "false",
# Optional / Default is "false"
     enable_unknown_clients_option       => "true" | "false",
# Optional / Default is "false"
     exclude                             => [$ExclusionRange1,$ExclusionRange2,...],
# Optional / Default is empty
     extattrs                            => { $string => $extattr, ... },
# Optional / Default is undefined
     extensible_attributes               => { $string => $string | $num, $string => [ $string | $num, ...
], ... } # Optional / Default is undefined
     failover_assoc                      => $string,
# Optional / Default is undefined
     filters                             => [$MAC,$RelayAgent,$Option,...],
# Optional / Default is undefined
     known_clients_option                => "Allow" | "Deny",
# Optional / Default is undefined
     lease_scavenge_time                 => $number,
# Optional / Default is -1
     member                              => $Member,
# Optional / Default is undefined
     nextserver                          => $ipv4addr | $fqdn | undef,
# Optional / Default is undefined
     options                             => [$Option1,$Option2,...],
# Optional / Default is empty
     logic_filters                       => [ $filter_name | $filter_obj, ...],
# Optional / Default is undefined
     override_logic_filters              => "true' | 'false",
# Optional / Default value is "false"
     override_ddns_domainname            => "true" | "false",
# Optional / Default value is "false"
     override_lease_scavenge_time        => "true" | "false",
# Optional / Default value is "false"
     override_update_dns_on_lease_renewal => "true" | "false",
# Optional / Default value is "false"
     pxe_lease_time                      => $num,
# Optional / Default is undefined
     range_high_water_mark               => $num,
# Optional / Default is 95
     range_high_water_mark_reset         => $num,
# Optional / Default is 95
     range_low_water_mark                => $num,
# Optional / Default is 0
     range_low_water_mark_reset          => $num,
# Optional / Default is 0
     recycle_leases                      => "true" | "false",
# Optional / Default is undefined
     server_association_type             => "NONE" | "MEMBER" | "FAILOVER" | "MS_SERVER"
# Optional / Default is undefined
     unknown_clients_option              => "Allow" | "Deny",
# Optional / Default is undefined
     update_dns_on_lease_renewal         => "true" | "false",
# Optional / Default value is "false"
      );
```

You cannot set both extattrs and extensible_attributes attributes at the same time.

MODULE METHODS

The following functions are available to be applied to a DHCP Range Template object.

Infoblox::DHCP::NetworkTemplate->range_templates( )

Use this function to specify a range template at network template on the Infoblox appliance. See Infoblox::DHCP::NetworkTemplate->range_templates() for parameters and return values.

Example

```
#Construct a range template object
my $dhcp_range_template = Infoblox::DHCP::RangeTemplate->new(
            name                    => "custom_range_template",
            number_of_addresses     => "10",
            offset                  => "10",
    );

my $template = Infoblox::DHCP::Template->new(
            name                    => "custom_range_template",
    );

#Configure range template in network template object
my $responce = $network_template->range_templates([$template]);
```

SESSION METHODS

This section describes all the methods in an Infoblox::Session module that can be applied to a DHCP range template object.

Infoblox::Session->add( )

Use this method to add a DHCP range template object to the Infoblox appliance. See Infoblox::Session->add() for parameters and return values.

Example

```
#Construct an object
my $dhcp_range_template = Infoblox::DHCP::RangeTemplate->new(
        name                    => "custom_range_template",
        number_of_addresses     => "10",
        offset                  => "10",
        bootfile                => "bootfile1.com",
        bootserver              => "someserver.com",
        comment                 => "add range",
        ddns_generate_hostname  => "true",
        deny_all_clients        => "true",
        deny_bootp              => "true",
        enable_ddns             => "true",
        exclude                 => [$exclusion_range_template],
        filters                 => [$filter],
        member                  => $memberdhcp,
        nextserver              => "3.3.3.3",
        options                 => [$option1],
        pxe_lease_time          => "3600",
        recycle_leases          => "true",
);

# Submit for addition
my $response = $session->add( $dhcp_range_template );
```

Infoblox::Session->get( )

Use this method to retrieve all the matching objects from the Infoblox appliance. See Infoblox::Session->get() for parameters and return values.

Key References

```
Apply the following attributes to get a specific DHCP range template object:

name - Optional. The name of the DHCP range template.
extattrs     - Optional. A hash reference containing extensible attributes.
extensible_attributes - Optional. A hash reference containing extensible attributes.
server_association_type - Optional. The server association type for the range.
```

Examples

```
my @retrieved_objs = $session->get(
                        object => "Infoblox::DHCP::RangeTemplate",
                        name   => "custom_range_template"
                        );
```

```
my @retrieved_objs = $session->get(
                      object               => "Infoblox::DHCP::RangeTemplate",
                      extensible_attributes => { 'Site' => 'Santa Clara' }
                     );
```

Infoblox::Session->search( )

Use this method to retrieve all the matching objects from the Infoblox appliance. See Infoblox::Session->search() for parameters and return values.

Key References

```
Apply the following attributes to search for a specific DHCP Range Template object:

name    - Optional. The name of the DHCP range template (regular expression).
comment - Optional. The comment of the DHCP range template (regular expression).
extattrs     - Optional. A hash reference containing extensible attributes.
extensible_attributes - Optional. A hash reference containing extensible attributes.
server_association_type - Optional. The server association type for the range.
```

For more information about searching extensible attributes, see Infoblox::Grid::ExtensibleAttributeDef/Searching Extensible Attributes.

Examples

```
my @retrieved_objs = $session->search(
                      object  => "Infoblox::DHCP::RangeTemplate",
                      name    => "custom_range_template"
           comment => ".*",
       );

my @retrieved_objs = $session->search(
                      object               => "Infoblox::DHCP::RangeTemplate",
                      extensible_attributes => { 'Site' => 'Santa Clara' }
       );
```

Infoblox::Session->modify( )

Use this method to modify a DHCP range template object in the Infoblox appliance. See Infoblox::Session->modify() for parameters and return values.

Example

```
# Use method to modify the comment.
$dhcp_range_template->comment("this is a modified comment");
# Submit modification
my $response = $session->modify( $dhcp_range_template );
```

Infoblox::Session->remove( )

Use this method to remove a DHCP range template object from the Infoblox appliance. See Infoblox::Session->remove() for parameters and return values.

To remove a specific object, first use `get()` or `search()` to retrieve the specific DHCP range template object, and then submit this object for removal.

Example

```
# Get the objects with the same name
my @retrieved_objs = $session->get(
    object => "Infoblox::DHCP::RangeTemplate",
    name   => "custom_range_template"
    );

# Find the desired object from the retrieved list.
my $desired_dhcp_range_template = $retrieved_objs[0];
# Submit for removal
my $response = $session->remove( $desired_dhcp_range_template );
```

METHODS

This section describes all the methods that can be used to configure and retrieve the attribute values of a DHCP range template object.

bootfile( )

Use this method to set or retrieve a bootfile name which is the name of the file the client must download.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

This option is overridden independently from Infoblox::DHCP::RangeTemplate->bootserver() and Infoblox::DHCP::RangeTemplate->nextserver().

Parameter

The boot file name in string format with a maximum of 256 bytes. The default value is undefined.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get bootfile
my $bootfile = $dhcp_range_template->bootfile();
#Modify bootfile
$dhcp_range_template->bootfile("bootfile2.com");
#Un-override bootfile
$dhcp_range_template->bootfile(undef);
```

bootserver( )

Use this method to set or retrieve a bootserver name which is the name of the server where the boot file is stored.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

This option is overridden independently from Infoblox::DHCP::RangeTemplate->bootfile() and Infoblox::DHCP::RangeTemplate->nextserver().

Parameter

The boot server IP address or name in FQDN (Fully Qualified Domain Name) format. The FQDN consists of the host name followed by the domain name (example: abc.com). A boot server name can have a maximum of 256 bytes. The default value is undefined.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get bootserver
my $bootserver = $dhcp_range_template->bootserver();
#Modify bootserver
$dhcp_range_template->bootserver("anotherserver.com");
#Un-override bootserver
$dhcp_range_template->bootserver(undef);
```

comment( )

Use this method to set or retrieve a descriptive comment of a DHCP range template.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

Desired comment in string format with a maximum of 256 bytes.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get comment
my $comment = $dhcp_range_template->comment();
#Modify comment
$dhcp_range_template->comment("Modifying the DHCP range template comment");
```

ddns_domainname( )

Use this method to set or retrieve the ddns_domainname value of a DHCP Range template object.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Setting this method to a defined value implicitly sets the override_ddns_domainname method to "true". Setting the parameter to undefined causes the appliance to use the Grid default and automatically resets the override_ddns_domainname attribute to "false".

Note that when ddns_domainname is set to a defined value and override_dddns_domainname is set to "false", the last operation takes precedence. Thus the sequence $object->ddns_domainname("testdomain"); $object->override_ddns_domainname("false"); will set override_ddns_domainname to "false", and the sequence $object->override_ddns_domainname("false"); $object->ddns_domainname("testdomain"); will result in override_ddns_domainname="true".

Parameter

The dynamic DNS domain name in string format.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get ddns_domainname
my $ddns_domainname = $dhcp_range_template->ddns_domainname();
#Modify ddns_domainname
$dhcp_range_template->ddns_domainname("test_domain.com");
```

ddns_generate_hostname( )

Use this method to set or retrieve the ddns_generate_hostname flag of a DHCP range template.

Enable this method to allow only the DHCP server to generate a hostname.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

Specify "true" to set the ddns_generate_hostname flag or "false" to deactivate/unset it. The default value is undefined.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get ddns_generate_hostname
my $ddns_generate_hostname = $dhcp_range_template->ddns_generate_hostname();
#Modify ddns_generate_hostname
$dhcp_range_template->ddns_generate_hostname("false");
```

deny_all_clients( )

Use this method to set or retrieve the deny_all_clients flag of a DHCP range template.

Enable this method to deny all lease requests for the DHCP range created from template.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

Specify "true" to set the deny_all_clients flag or "false" to deactivate/unset it. The default value for this field is false.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get deny_all_clients
my $deny_all_clients = $dhcp_range_template->deny_all_clients();
#Modify deny_all_clients
$dhcp_range_template->deny_all_clients("false");
```

deny_bootp( )

Use this method to set or retrieve the deny_bootp flag of a DHCP range template.

Enable this method to deny all BOOTP requests for DHCP range created from this template.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

Specify "true" to set the deny_bootp flag or "false" to deactivate/unset it. The default value is false which indicates that this attribute inherit the upper-level setting.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get deny_bootp
my $deny_bootp = $dhcp_range_template->deny_bootp();
#Modify deny_bootp
$dhcp_range_template->deny_bootp("false");
```

enable_ddns( )

Use this method to set or retrieve the enable_ddns flag of a DHCP range template.

Enable this method to allow dynamic DNS updates.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

Specify "true" to set the enable_ddns flag or "false" to deactivate/unset it. The default value is undefined.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get enable_ddns
my $enable_ddns = $dhcp_range_template->enable_ddns();
#Modify enable_ddns
$dhcp_range_template->enable_ddns("false");
```

enable_known_clients_option( )

Use this method to set or retrieve the enable_known_clients_option flag of a DHCP range.

You must set the enable_known_clients_option flag to true in order to use the known_clients_option attribute.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

Specify "true" to set the enable_known_clients_option flag or "false" to deactivate/unset it. The default value is "false".

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get enable_known_clients_option
my $enable_known_clients_option = $dhcp_range->enable_known_clients_option();
#Modify enable_known_clients_option
$dhcp_range->enable_known_clients_option("false");
```

enable_unknown_clients_option( )

Use this method to set or retrieve the enable_unknown_clients_option flag of a DHCP range.

You must set the enable_unknown_clients_option flag to true in order to use the unknown_clients_option attribute.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

Specify "true" to set the enable_unknown_clients_option flag or "false" to deactivate/unset it. The default value is "false".

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get enable_unknown_clients_option
my $enable_unknown_clients_option = $dhcp_range->enable_unknown_clients_option();
#Modify enable_unknown_clients_option
$dhcp_range->enable_unknown_clients_option("false");
```

exclude( )

Use this method to set or retrieve the exclusion range template in the DHCP range template.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

Valid value is an array reference that contains defined Infoblox::DHCP::ExclusionRangeTemplate objects.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
    #Get exclude
    my $ref_exclude = $dhcp_range_template->exclude();
    #Modify exclude
    $dhcp_range_template->exclude([$exl1]); #$exl1 is an Infoblox::DHCP::ExclusionRangeTemplate
object
```

extattrs( )

Use this method to set or retrieve the extensible attributes associated with a DHCP Range Template object.

Parameter

Valid value is a hash reference containing the names of extensible attributes and their associated values (Infoblox::Grid::Extattr objects).

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get extattrs
my $ref_extattrs = $dhcp_range_template->extattrs();
#Modify extattrs
$dhcp_range_template->extattrs({ 'Site' => $extattr1, 'Administrator' => $extattr2 });
```

extensible_attributes( )

Use this method to set or retrieve the extensible attributes associated with a DHCP Range Template object.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

For valid values for extensible attributes, see Infoblox::Grid::ExtensibleAttributeDef/Extensible Attribute Values.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get extensible attributes
my $ref_extensible_attributes = $dhcp_range_template->extensible_attributes();
#Modify extensible attributes
$dhcp_range_template->extensible_attributes({ 'Site' => 'Santa Clara', 'Administrator' => [ 'Peter',
'Tom' ] });
```

failover_assoc( )

Use this method to set or retrieve the failover peer association name of the DHCP range template.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

If the attribute is set, the server_association_type attribute will also be set to "FAILOVER" if the name is defined, or to "NONE" if the member is also undefined.

Parameter

Specify the failover peer association name in string format. The default value is undefined.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get failover_assoc
my $failover_assoc = $dhcp_range_template->failover_assoc();
#Modify failover_assoc
$dhcp_range_template->failover_assoc("peer2");
```

filters( )

Use this method to set or retrieve the filters of the DHCP range template.

Filters are used to screen the address assignments if the addresses in the DHCP range needs to be assigned to specific hosts.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

The valid value is an array reference that contains defined Infoblox::DHCP::FilterRule::MAC, Infoblox::DHCP::FilterRule::NAC, Infoblox::DHCP::FilterRule::RelayAgent, Infoblox::DHCP::FilterRule::Fingerprint and Infoblox::DHCP::FilterRule::UserClass objects. The default value is undefined.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get filters
my $ref_filters = $dhcp_range_template->filters();
#Modify filters
$dhcp_range_template->filters([$filter1]); #$filter1 is an Infoblox::DHCP::FilterRule::MAC,
Infoblox::DHCP::FilterRule::MAC,  Infoblox::DHCP::FilterRule::RelayAgent,
Infoblox::DHCP::FilterRule::Fingerprint or Infoblox::DHCP::FilterRule::UserClass object.
```

known_clients_option( )

Use this method to set or retrieve the known_clients_option option of a DHCP range.

Set this option to "Allow" or "Deny" to allow or prevent the allocation of an IP address from this pool to any client that has a host declaration.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

Specify "Allow" or "Deny" to set the known_clients_option flag value. The default value is undefined.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get known_clients_option
my $known_clients_option = $dhcp_range->known_clients_option();
#Modify known_clients_option
$dhcp_range->known_clients_option("Deny");
```

lease_scavenge_time( )

Use this method to set or retrieve the lease_scavenge_time value of a DHCP Range Template object.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Setting this method to a defined value implicitly sets the override_lease_scavenge_time method to "true". Setting the parameter to undefined causes the appliance to use the default from the upper level and automatically resets the override_lease_scavenge_time attribute to "false".

Note that when lease_scavenge_time is set to a defined value and override_lease_scavenge_time is set to "false", the last operation takes precedence. Thus the sequence $object->lease_scavenge_time(604800); $object->override_lease_scavenge_time("false"); will set override_lease_scavenge_time to "false", and the sequence $object->override_lease_scavenge_time("false"); $object->lease_scavenge_time(604800); will result in override_lease_scavenge_time="true".

Parameter

An integer that specifies the period of time, in seconds, that free and backup leases remain in the database before they are automatically deleted. To disable lease scavenging, set the parameter to -1. The minimum positive value must be greater than 86400 seconds (1 day). The default value of this parameter is -1, which means that lease scavenging is disabled.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get lease_scavenge_time
my $lease_scavenge = $dhcp_range_template->lease_scavenge_time();
#Modify lease_scavenge_time
$dhcp_range_template->lease_scavenge_time(30 * 24 * 60 * 60);
```

logic_filters( )

Use this method to set or retrieve all the logic (ordered) filters associated with the DHCP Range Template object.

Filters are used to screen address assignments in a DHCP Range Template if the addresses are to be assigned to specific hosts.

Setting this method to a defined value implicitly sets the override_logic_filters method to "true". Setting the parameter to undefined causes the appliance to use the Grid default and automatically resets the override_logic_filters attribute to "false".

Note that when logic_filters is set to a defined value and override_dlogic_filters is set to "false", the last operation takes precedence. Thus the sequence $object->logic_filters($filters); $object->override_logic_filters("false"); will set override_logic_filters to "false", and the sequence $object->override_logic_filters("false"); $object->logic_filters($filters); will result in override_logic_filters="true".

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

The valid value is an array reference that contains either the Infoblox::DHCP::Filter::MAC, Infoblox::DHCP::Filter::NAC, Infoblox::DHCP::Filter::Option objects or the names of the corresponding filters. The filters in the list must satisfy the following conditions:

Filter name must be globally unique.
Filters without expressions must be at the end of the list.
The filters should not be disabled.

The default value of the parameter is undefined.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get logic_filters
my $ref_logic_filters = $dhcp_range_template->logic_filters();
#Modify logic_filters
$dhcp_range_template->logic_filters(["filter_1","filter_2",$exl1]); #$exl1 is an
Infoblox::DHCP::Filter::MAC, Infoblox::DHCP::Filter::NAC or Infoblox::DHCP::Filter::Option object.
```

member( )

Use this method to set or retrieve the member or Microsoft (r) server of the DHCP range template.

The member or Microsoft server that serves DHCP for this IP address range template.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

If the attribute is set, the server_association_type attribute will also be set to "MEMBER" if you set the member to an Infoblox::DHCP::Member object, to "MS_SERVER" if you set the member to an Infoblox::DHCP::MSServer object, or to "NONE" if failover_assoc is also empty.

Parameter

The valid value is a defined Infoblox::DHCP::Member or Infoblox::DHCP::MSServer object. The default value is undefined.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get member
my $ref_member = $dhcp_range_template->member();
#Modify member
$dhcp_range_template->member($member1); #$member is an Infoblox::DHCP::Member object
```

name( )

Use this method to set or retrieve the description of the name of the DHCP range template.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

Name of the fixed address in string format with a maximum of 64 bytes. The default value is undefined.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get name
my $name = $dhcp_range_template->name();
#Modify name
$dhcp_range_template->name("custom_range_template");
```

nextserver( )

Use this method to set or retrieve the nextserver IP address which is the boot file server where the boot file is stored.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

This option is overridden independently from Infoblox::DHCP::RangeTemplate->bootfile() and Infoblox::DHCP::RangeTemplate->bootserver().

Parameter

The next server IP address or name in FQDN (Fully Qualified Domain Name) format. The FQDN consists of the host name followed by the domain name (example: abc.com). A next server name can have a maximum of 256 bytes. The default value is undefined.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get nextserver
my $nextserver = $dhcp_range_template->nextserver();
#Modify nextserver
$dhcp_range_template->nextserver("4.4.4.4");
#Un-override nextserver
$dhcp_range_template->nextserver(undef);
```

number_of_addresses( )

Use this method to set or retrieve the number of addresses in this DHCP range template

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

The number of addresses in this DHCP range template.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get number_of_addresses
my $number_of_addresses = $dhcp_range_template-> number_of_addresses();
#Modify number_of_addresses
$dhcp_range_template->number_of_addresses("10");
```

offset( )

Use this method to set or retrieve the address offset of this DHCP range template.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

The address offset of this DHCP range template. The offset is calculated from the start address of network template to which the DHCP range template is assigned to. For example, if offset is set to 10 and number_of_addresses is set to 100, and network is set to 10.0.0.0/8, then start address of DHCP range created from this range template will be 10.0.0.10 and end address will be 10.0.0.100.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get offset
my $offset = $dhcp_range_template->offset();
#Modify offset
$dhcp_range_template->offset("10");
```

options( )

Use this method to set or retrieve the options of the DHCP range template.

DHCP options describe network configuration settings and various services available on the network.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

Valid value is an array reference that contains defined Infoblox::DHCP::Option objects. The default value is empty.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get options
my $ref_options = $dhcp_range_template->options();
#Modify options
$dhcp_range_template->options([$option12]); #$option12 is an Infoblox::DHCP::Option object
```

override_ddns_domainname( )

The override_ddns_domainname attribute controls whether the ddns_domainname value in the DHCP range template is used, instead of the Grid default.

The override_ddns_domainname attribute can be specified explicitly. It is also set implicitly when ddns_domainname is set to a defined value.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

Set the parameter to "true" to override the grid-level setting for ddns_domainname. Set the parameter to "false" to inherit the grid-level setting for ddns_domainname.

The default value of this parameter is "false".

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Getting override_ddns_domainname
my $override_ddns_domainname=$dhcp_range_template->override_ddns_domainname( );
#Modifying override_ddns_domainname
$dhcp_range_template->override_ddns_domainname("true");
```

override_lease_scavenge_time( )

The override_lease_scavenge_time attribute controls whether the lease_scavenge_time value in the DHCP range template is used, instead of the upper-level default.

The override_lease_scavenge_time attribute can be specified explicitly. It is also set implicitly when lease_scavenge_time is set to a defined value.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

Set the parameter to "true" to override the upper-level setting for lease_scavenge_time. Set the parameter to "false" to inherit the setting for lease_scavenge_time from the upper level.

The default value of this parameter is "false".

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Getting override_lease_scavenge_time
my $override_lease_scavenge_time=$dhcp_range_template->override_lease_scavenge_time( );
#Modifying override_lease_scavenge_time
$dhcp_range_template->override_lease_scavenge_time("true");
```

override_logic_filters( )

The override_logic_filters attribute controls whether the logic_filters value in the DHCP Range Template is used, instead of the Grid default.

The override_logic_filters attribute can be specified explicitly. It is also set implicitly when logic_filters is set to a defined value.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

Set the parameter to "true" to override the Grid-level setting for logic_filters. Set the parameter to "false" to inherit the Grid-level setting for logic_filters.

The default value of this parameter is "false".

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Getting override_logic_filters
my $override_logic_filters=$dhcp_range_template->override_logic_filters( );
#Modifying override_logic_filters
$dhcp_range_template->override_logic_filters("true");
```

override_update_dns_on_lease_renewal( )

The override_update_dns_on_lease_renewal attribute controls whether the update_dns_on_lease_renewal value in the DHCP range template is used, instead of the Grid default.

The override_update_dns_on_lease_renewal attribute can be specified explicitly. It is also set implicitly when update_dns_on_lease_renewal is set to a defined value.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

Set the parameter to "true" to override the grid-level setting for update_dns_on_lease_renewal. Set the parameter to "false" to inherit the grid-level setting for update_dns_on_lease_renewal.

The default value of this parameter is "false".

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
    #Getting override_update_dns_on_lease_renewal
    my $override_update_dns_on_lease_renewal=$dhcp_range_template-
>override_update_dns_on_lease_renewal( );
    #Modifying override_update_dns_on_lease_renewal
    $dhcp_range_template->override_update_dns_on_lease_renewal("true");
```

pxe_lease_time( )

Use this method to set or retrieve the PXE lease time value of the DHCP range template.

The Preboot Execution Environment lease time value is used by some hosts to boot remotely from a server.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

A 32-bit integer (range from 0 to 4294967295) represents the duration in seconds that the record is cached. Zero indicates that the record should not be cached. The default value is undefined.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get pxe_lease_time
my $pxe_lease_time = $dhcp_range_template->pxe_lease_time();
#Modify pxe_lease_time
$dhcp_range_template->pxe_lease_time("7200");
```

range_high_water_mark( )

Use this method to set or retrieve the range_high_water_mark value. If the percentage of allocated addresses exceeds this value, an

SNMP trap is sent, unless this is a range served by a Microsoft server.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

A number that specifies the percentage of allocated addresses. The range is from 1 to 100. The high watermark value must be greater than the low watermark value. The default value is 95.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get range_high_water_mark
my $dhcp_range_template_high_water_mark = $dhcp_range_template->range_high_water_mark();
#Modify range_high_water_mark
$dhcp_range_template->range_high_water_mark(80);
```

range_high_water_mark_reset( )

Use this method to set or retrieve the range_high_water_mark_reset value. If the percentage of allocated addresses drops below this value, a corresponding SNMP trap is reset.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

A number that specifies the percentage of allocated addresses. The range is from 1 to 100. The high watermark reset value must be lower than the high watermark value. The default value is 95.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get range_high_water_mark_reset
my $dhcp_range_template_high_water_mark_reset = $dhcp_range_template->range_high_water_mark_reset();
#Modify range_high_water_mark_reset
$dhcp_range_template->range_high_water_mark_reset(70);
```

range_low_water_mark( )

Use this method to set or retrieve the range_low_water_mark value. If the percentage of allocated addresses drops below this watermark, an SNMP trap is sent, unless this is a range served by a Microsoft server.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

A number that specifies the percentage of allocated addresses. The range is from 1 to 100. The low watermark value must be lower than the high watermark value. The default low watermark value is 0.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get range_low_water_mark
my $dhcp_range_template_low_water_mark = $dhcp_range_template->range_low_water_mark();
#Modify range_low_water_mark
$dhcp_range_template->range_low_water_mark(10);
```

range_low_water_mark_reset( )

Use this method to set or retrieve the range_low_water_mark_reset value. If the percentage of allocated addresses exceeds this value, a corresponding SNMP trap is reset.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

A number that specifies the percentage of allocated addresses. The range is from 1 to 100. The low watermark reset value must be higher than the low watermark value. The default low watermark reset number is 0.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get range_low_water_mark_reset
my $dhcp_range_template_low_water_mark_reset = $dhcp_range_template->range_low_water_mark_reset();
#Modify range_low_water_mark_reset
$dhcp_range_template->range_low_water_mark_reset(10);
```

server_association_type( )

Use this method to set or retrieve the DHCP range template server association type.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

The server_association_type attribute can be specified explicitly. It is also set implicitly when member or failover_assoc are set.

Parameter

Valid values are "NONE", "FAILOVER", "MS_SERVER" and "MEMBER".

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get server_association_type
my $server_association_type = $dhcp_range_template->server_association_type();
#Modify server_association_type
$dhcp_range_template->server_association_type("MEMBER");
```

recycle_leases( )

Use this method to set or retrieve the recycle_leases flag of the DHCP range template.

If the flag is enabled, the leases are kept in recycle bin until one week after expiration. When the flag is disabled, the leases are irrecoverably deleted.

Parameter

Specify "true" to set the recycle_leases flag or "false" to deactivate/unset it. The default value is undefined.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get recycle_leases
my $recycle_leases = $dhcp_range_template->recycle_leases();
#Modify recycle_leases
$dhcp_range_template->recycle_leases("false");
```

unknown_clients_option( )

Use this method to set or retrieve the unknown_clients_option option of a DHCP range.

Set this option to "Allow" or "Deny" to allow or prevent the allocation of an IP address from this pool to any client that has no host declaration.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

Specify "Allow" or "Deny" to set the unknown_clients_option flag value. The default value is undefined.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get unknown_clients_option
my $unknown_clients_option = $dhcp_range->unknown_clients_option();
#Modify unknown_clients_option
$dhcp_range->unknown_clients_option("Deny");
```

update_dns_on_lease_renewal( )

The update_dns_on_lease_renewal attribute controls whether the DHCP server updates DNS when a DHCP lease is renewed.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Setting this method to a defined value implicitly sets the override_update_dns_on_lease_renewal method to "true". Setting the parameter to undefined causes the appliance to use the Grid default and automatically resets the override_update_dns_on_lease_renewal attribute to "false".

Note that when update_dns_on_lease_renewal is set to a defined value and override_update_dns_on_lease_renewal is set to "false", the last operation takes precedence. Thus the sequence $object->update_dns_on_lease_renewal("true"); $object->override_update_dns_on_lease_renewal("false"); will set override_update_dns_on_lease_renewal to "false", and the sequence $object->override_update_dns_on_lease_renewal("false"); $object->update_dns_on_lease_renewal("true"); will result in override_update_dns_on_lease_renewal="true".

Parameter

Specify "true" to set the update_dns_on_lease_renewal flag or "false" to deactivate/unset it.

The default value of this parameter is "false".

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get update_dns_on_lease_renewal
my $update_dns_on_lease_renewal=$dhcp_range_template->update_dns_on_lease_renewal( );
#Modify update_dns_on_lease_renewal
$dhcp_range_template->update_dns_on_lease_renewal("true");
```

SAMPLE CODE

The following sample code demonstrates the different functions that can be applied to a DHCP range template object such as add, modify, and remove. Also, this sample includes error handling for the operations.

#Preparation prior to a DHCP RangeTemplate object insertion

```
#PROGRAM STARTS: Include all the modules that will be used
use strict;
use Infoblox;

#Create a session to the Infoblox appliance
my $host_ip = "192.168.1.2";
my $session = Infoblox::Session->new(
               master   => $host_ip,      #appliance host ip
               username => "admin",       #appliance user login
               password => "infoblox"     #appliance password
);
unless ($session) {
       die("Construct session failed: ",
            Infoblox::status_code() . ":" . Infoblox::status_detail());
}
print "Session created successfully\n";

#Create a DHCP Member object
my $memberdhcp = Infoblox::DHCP::Member->new(
    name     => "infoblox.localdomain",
    ipv4addr => $host_ip
);
```

```
unless($memberdhcp) {
       die("Construct member failed: ",
            Infoblox::status_code() . ":" . Infoblox::status_detail());
}
print "DHCP Member object created successfully\n";

#Create the Mac Filter object
my $mac_filter = Infoblox::DHCP::Filter::MAC->new(
    name    => "filter1",
    comment => "5 minutes"
);

unless($mac_filter) {
       die("Construct Mac Filter object failed: ",
            Infoblox::status_code() . ":" . Infoblox::status_detail());
}
print "Mac Filter object created successfully\n";

#Add the Mac Filter object into the Infoblox appliance through a session
$session->add($mac_filter)
    or die("Add Mac Filter object failed: ",
            $session->status_code() . ":" . $session->status_detail());

print "Mac Filter object added to Infoblox appliance successfully\n";

#Construct DHCP Mac Filter rule object
my $filter = Infoblox::DHCP::FilterRule::MAC->new(
    filter_name => "filter1",
    permission  => "grant"
);
unless($filter){
   die("Construct filter object failed: ",
        Infoblox::status_code() . ":" . Infoblox::status_detail());
   }
 print "Filter object created successfully\n";

#Construct exclusion range template object
my $exclusion_range_template = Infoblox::DHCP::ExclusionRangeTemplate->new(
    offset              => "15",
    number_of_addresses => "5",
    comment             => "Excluded range"
);
unless($exclusion_range_template){
       die("Construct exclusion range template object failed: ",
                Infoblox::status_code() . ":" . Infoblox::status_detail());
       }
 print "Exclusion range template object created successfully\n";
```

#Create a DHCP RangeTemplate object

```
my $dhcp_range_template = Infoblox::DHCP::RangeTemplate->new(
        name                    => "custom_range_template",
        offset                  => "10",
        number_of_addresses     => "10",
        bootfile                => "bootfile1.com",
        bootserver              => "someserver.com",
        comment                 => "add range",
        ddns_generate_hostname  => "true",
        deny_all_clients        => "true",
        deny_bootp              => "true",
        enable_ddns             => "true",
        exclude                 => [$exclusion_range_template],
        filters                 => [$filter],
        member                  => $memberdhcp,
        nextserver              => "3.3.3.3",
        pxe_lease_time          => "3600",
        recycle_leases          => "true",
);
unless($dhcp_range_template) {
       die("Construct DHCP RangeTemplate object failed: ",
            Infoblox::status_code() . ":" . Infoblox::status_detail());
}
print "DHCP RangeTemplate object created successfully\n";

#Add the DHCP RangeTemplate object into the Infoblox appliance through a session
$session->add($dhcp_range_template)
```

```
    or die("Add DHCP RangeTemplate object failed: ",
            $session->status_code() . ":" . $session->status_detail());

print "DHCP RangeTemplate object added to the network successfully\n";
```

#Search a DHCP RangeTemplate

```
#Search a DHCP RangeTemplate through the session
my @retrieved_objs = $session->search(
    object     => "Infoblox::DHCP::RangeTemplate",
    name       => "custom_range_template",
);
my $object = $retrieved_objs[0];
unless ($object) {
       die("Search DHCP RangeTemplate failed: ",
            $session->status_code() . ":" . $session->status_detail());
}
print "Search DHCP RangeTemplate object found at least 1 matching entry\n";
```

#Get and modify a DHCP RangeTemplate

```
#Get a DHCP RangeTemplate through the session
my @retrieved_objs = $session->get(
    object     => "Infoblox::DHCP::RangeTemplate",
    name       => "custom_range_template",
);
my $object = $retrieved_objs[0];
unless ($object) {
       die("Get DHCP RangeTemplate failed: ",
            $session->status_code() . ":" . $session->status_detail());
}
print "Get DHCP RangeTemplate object found at least 1 matching entry\n";

#Modify one of the attributes of the specified DHCP RangeTemplate
$object->comment("modified comment");

#Un-override bootfile
$object->bootfile(undef);

#Un-override bootserver
$object->bootserver(undef);

#Un-override nextserver
$object->nextserver(undef);

#Apply the changes
$session->modify($object)
    or die("Modify DHCP Range Template failed: ",
            $session->status_code() . ":" . $session->status_detail());

print "DHCP RangeTemplate object modified successfully \n";
```

#Remove a DHCP RangeTemplate

```
#Get a DHCP RangeTemplate through the session
my @retrieved_objs = $session->get(
    object     => "Infoblox::DHCP::RangeTemplate",
    name       => "custom_range_template",
);

my $object = $retrieved_objs[0];
unless ($object) {
       die("Get DHCP Range Template failed: ",
            $session->status_code() . ":" . $session->status_detail());
}
print "Get DHCP RangeTemplate object found at least 1 matching entry\n";

#submit the object for removal
$session->remove($object)
    or die("Remove DHCP Range Template failed: ",
            $session->status_code() . ":" . $session->status_detail());

print "DHCP Range Template object removed successfully \n";

####PROGRAM ENDS####
```

AUTHOR

Infoblox Inc. http://www.infoblox.com/

SEE ALSO

Infoblox::Session, Infoblox::DHCP::NetworkTemplate, Infoblox::DHCP::ExclusionRangeTemplate, Infoblox::DHCP::Template, Infoblox::DHCP::FilterRule::MAC, Infoblox::DHCP::FilterRule::NAC, Infoblox::DHCP::FilterRule::RelayAgent, Infoblox::DHCP::FilterRule::UserClass, Infoblox::DHCP::Member, Infoblox::DHCP::Option

---

COPYRIGHT

Copyright (c) 2017 Infoblox Inc.

Infoblox::DHCP::RoamingHost - DHCP Roaming Host object.

- NAME
- DESCRIPTION
- CONSTRUCTOR
- SESSION METHODS
  - Infoblox::Session->add( )
  - Infoblox::Session->get( )
  - Infoblox::Session->modify( )
  - Infoblox::Session->remove( )
  - Infoblox::Session->search( )
- METHODS
  - address_type( )
  - bootfile( )
  - bootserver( )
  - client_identifier_prepend_zero( )
  - comment( )
  - ddns_domainname( )
  - ddns_hostname( )
  - deny_bootp( )
  - dhcp_client_identifier( )
  - disable( )
  - discovered_name( )
  - discoverer( )
  - enable_ddns( )
  - extattrs( )
  - extensible_attributes( )
  - first_discovered( )
  - force_roaming_hostname( )
  - ignore_dhcp_option_list_request( )
  - ipv6_client_hostname( )
  - ipv6_ddns_domainname( )
  - ipv6_ddns_hostname( )
  - ipv6_domain_name( )
  - ipv6_domain_name_servers( )
  - ipv6_duid( )
  - ipv6_enable_ddns( )
  - ipv6_force_roaming_hostname( )
  - ipv6_match_option( )
  - ipv6_options( )
  - ipv6_template( )
  - last_discovered( )
  - mac( )
  - match_client( )
  - name( )
  - netbios( )
  - network_component_description( )
  - network_component_ip( )
  - network_component_name( )
  - network_component_port_description( )
  - network_component_port_name( )
  - network_component_port_number( )
  - network_component_type( )
  - network_view( )
  - nextserver( )
  - options( )
  - os( )
  - override_ddns_domainname( )
  - override_ipv6_ddns_domainname( )
  - override_ipv6_domain_name( )
  - override_ipv6_domain_name_servers( )
  - override_ipv6_enable_ddns( )
  - override_ipv6_options( )
  - override_preferred_lifetime( )
  - override_valid_lifetime( )
  - port_duplex( )
  - port_link_status( )
  - port_speed( )
  - port_status( )
  - port_vlan_description( )
  - port_vlan_name( )
  - port_vlan_number( )
  - preferred_lifetime( )

- pxe_lease_time( )
- template( )
- valid_lifetime( )

- SAMPLE CODE
- AUTHOR
- SEE ALSO
- COPYRIGHT

NAME

Infoblox::DHCP::RoamingHost - DHCP Roaming Host object.

DESCRIPTION

A Roaming Host is a specific host that a DHCP server always assigns when a lease request comes from a particular MAC address of the client.

CONSTRUCTOR

```
my $roaming_host = Infoblox::DHCP::RoamingHost -> new(
      name                          => $string,                        #Required
      address_type                  => "ipv4" | "ipv6" | "both",       #Optional / Default is "ipv4"
      bootfile                      => $filename,                      #Optional / Default is empty
      bootserver                    => $ipv4addr,                      #Optional / Default is empty
      client_identifier_prepend_zero  => "true" | "false",             #Optional / Default is "false"
      comment                       => $string,                        #Optional / Default is empty
      ddns_domainname               => $string,                        #Optional / Default is empty
      ddns_hostname                 => $string,                        #Optional / Default is empty
      deny_bootp                    => "true" | "false",               #Optional / Default is "false"
      dhcp_client_identifier        => $string,                        #Optional / Default is empty
      disable                       => "true" | "false",               #Optional / Default is "false"
      enable_ddns                   => "true" | "false",               #Optional / Default is "false"
      extattrs                      => { $string => $extattr, ... },      #Optional / Default is
undefined
      extensible_attributes         => { $string => $string | $num, $string => [ $string | $num, ... ],
... }, #Optional / Default is undefined
      force_roaming_hostname        => "true" | "false",               #Optional / Default is "false"
      ignore_dhcp_option_list_request => "true" | "false" | undef,     #Optional / Default is "false"
      ipv6_ddns_domainname          => $string,                        #Optional / Default is undefined
      ipv6_ddns_hostname            => $string,                        #Optional / Default is empty
      ipv6_domain_name_servers      => $string,                        #Optional / Default is undefined
      ipv6_domain_name              => $string,                        #Optional / Default is undefined
      ipv6_duid                     => $duid,                          #Optional / Default is undefined
      ipv6_enable_ddns              => "true" | "false" | undef,       #Optional / Default is undefined
      ipv6_force_roaming_hostname   => "true" | "false",               #Optional / Default is "false"
      ipv6_match_option             => "duid" | undef,                 #Optional / Default is undefined
      ipv6_options                  => [$option1, $option2,...],       #Optional / Default is undefined
      ipv6_template                 => $string,                        #Optional / Default is undefined
      last_discovered               => $string,                        #Optional / Default is empty
      mac                           => $mac_addr,                      #Optional / Default is empty
      match_client                  => "MAC" | "CLIENT_IDENTIFIER",    #Optional / Default is "MAC"
      network_view                  => $NetworkView,                   #Optional / Default is "default"
network view
      nextserver                    => $ipv4addr,                      #Optional / Default is empty
      options                       => [$option1, $option2, ...],      #Optional / Default is undefined
      override_ddns_domainname      => "true" | "false",               #Optional / Default value is "false"
      override_ipv6_ddns_domainname   => "true" | "false",             #Optional / Default is "false"
      override_ipv6_domain_name_servers => "true" | "false",           #Optional / Default is "false"
      override_ipv6_domain_name     => "true" | "false",               #Optional / Default is "false"
      override_ipv6_enable_ddns     => "true" | "false",               #Optional / Default is "false"
      override_ipv6_options         => "true" | "false",               #Optional / Default is "false"
      override_preferred_lifetime   => "true" | "false",               #Optional / Default is "false"
      override_valid_lifetime       => "true" | "false",               #Optional / Default is "false"
      preferred_lifetime            => $num,                           #Optional / Default is 27000
      pxe_lease_time                => $time,                          #Optional / Default is empty
      template                      => $string,                        #Optional / Default is undefined
      valid_lifetime                => $num,                           #Optional / Default is 43200
);
```

You cannot set both extattrs and extensible_attributes attributes at the same time.

SESSION METHODS

This section describes all the methods in Infoblox::Session module that you can apply to a DHCP Roaming Host object.

Infoblox::Session->add( )

Use this method to add a DHCP Roaming Host object to the Infoblox appliance. See Infoblox::Session->add() for parameters and return values.

Example

```
#Construct a DHCP Roaming Host object
my $roaming_host = Infoblox::DHCP::RoamingHost->new(
        "name"                          => "JohnSmith",
        "bootfile"                      => "bootfile",
        "bootserver"                    => "bootserver.com",
        "client_identifier_prepend_zero"    => "true",
        "comment"                       => "add roaming host using uid",
        "ddns_domainname"               => "ddns.domain.com",
        "ddns_hostname"                 => "ddns.host.domain.com",
        "deny_bootp"                    => "false",
        "dhcp_client_identifier"        => "foo",
        "disable"                       => "false",
        "enable_ddns"                   => "false",
        "match_client"                  => "MAC",
        "network_view"                  => $nview,
        "nextserver"                    => "3.3.3.3",
        "options"                       => [ $option1,  $option2, $option3,  $option4, $option5,
$option6 ],
        "pxe_lease_time"                => "3600",
);

#Submit for addition
my $response = $session->add($roaming_host)
```

Infoblox::Session->get( )

Use this method to retrieve all the matching objects from the Infoblox appliance. See Infoblox::Session->get() for parameters and return values.

Key References

Apply the following attributes to get a specific DHCP Roaming Host object:

```
  name                    - Optional. The roaming host name.
  mac                     - Optional. MAC address of the roaming host.
  dhcp_client_identifier  - Optional, DHCP option 61 client identifier.
  network_view            - Optional. Network view in which the roaming host is located. By default,
all network views are searched.
  extattrs                - Optional. A hash reference containing extensible attributes.
  extensible_attributes   - Optional. A hash reference containing extensible attributes.
```

Examples

```
#Get DHCP Roaming Host object through the session
my @retrieved_objs = $session->get(
    object => "Infoblox::DHCP::RoamingHost",
    mac    => "11:11:11:11:11.11",
);
my @retrieved_objs = $session->get(
    object => "Infoblox::DHCP::RoamingHost",
    dhcp_client_identifier => "foo",
);
my @retrieved_objs = $session->get(
    object       => "Infoblox::DHCP::RoamingHost",
    name     => "JohnSmith",
    network_view => "default",
);

my @retrieved_objs = $session->get(
   object               => "Infoblox::DHCP::RoamingHost",
   extensible_attributes => { 'Site' => 'Santa Clara' });
```

Infoblox::Session->modify( )

Use this method to modify a DHCP Roaming Host object in the Infoblox appliance. See Infoblox::Session->modify() for parameters and return values.

Example

```
#Use method to modify the pxe_lease_time
$roaming_host->pxe_lease_time("1234");
#Submit modification
my $response = $session->modify( $roaming_host );
```

Infoblox::Session->remove( )

Use this method to remove a DHCP Roaming Host object from the Infoblox appliance. See Infoblox::Session->remove() for parameters and return values.

To remove a specific object, first use `get()` or `search()` to retrieve the specific DHCP Roaming Host, and then submit this object for removal.

Example

```
#Get DHCP Roaming Host object through the session
my @retrieved_objs = $session->get(
    object       => "Infoblox::DHCP::RoamingHost",
    mac          => "11:11:11:11:11:11",
    network_view => "default"
);
#Find the desired object from the retrieved list.
my $desired_roaming_host = $retrieved_objs[0];
#Submit for removal
my $response = $session->remove( $desired_roaming_host );
```

Infoblox::Session->search( )

Use this method to search for DHCP Roaming Host object and matching Host object with MAC address in the Infoblox appliance. See Infoblox::Session->search() for parameters and return values.

Key References

Apply the following attributes to search for a specific DHCP Roaming Host object:

```
  name           - Optional. The roaming host name (regular expression).
  mac            - Optional. MAC address of the roaming host (regular expression).
  network_view   - Optional. Network view in which the roaming host is located. By default, all
network views are searched.
  extattrs       - Optional. A hash reference containing extensible attributes.
  extensible_attributes - Optional. A hash reference containing extensible attributes.
```

For more information about searching extensible attributes, see Infoblox::Grid::ExtensibleAttributeDef/Searching Extensible Attributes.

Examples

```
 # Search for all DHCP Roaming Host objects that are specified in the network "10.0.0.0/8" in the
"default" network view.
 my @retrieved_objs = $session->search(
     object       => "Infoblox::DHCP::RoamingHost",
     mac          => "11:11:11:11:11.*",
     network_view => "default"
 );

 # Search for all DHCP Roaming Host objects with "Santa Clara" for the "Site" extensible attribute
 my @retrieved_objs = $session->search(
     object               => "Infoblox::DHCP::RoamingHost",
     extensible_attributes => { 'Site' => 'Santa Clara' }
 );
```

METHODS

This section describes all the methods that you can use to set and retrieve the attribute values of a DHCP Roaming Host object.

address_type( )

Use this method to set or retrieve the address_type field.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

The possible values for address_type are the following: ipv4 - Roaming hosts are set up for IPv4 addresses only. ipv6 - Roaming hosts are set up for IPv6 addresses only. both - Both types of addresses are set up.

The default value of the parameter is "ipv4".

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get address_type
my $address_type = $roaminghost->address_type();
#Modify address_type
$roaminghost->address_type("both");
```

bootfile( )

Use this method to set or retrieve the bootfile value of a DHCP Roaming Host object.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

The name of the file the client must download.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get bootfile
my $bootfile = $roaming_host->bootfile();
#Modify bootfile
$roaming_host->bootfile("boot_file");
```

bootserver( )

Use this method to set or retrieve the bootserver of a DHCP Roaming Host object.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

The name of the server on which the boot file is stored.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get bootserver
my $bootserver = $roaming_host->bootserver();
#Modify bootserver
$roaming_host->bootserver("bootserver2.com");
```

client_identifier_prepend_zero( )

Use this method to prepend "\000" to the dhcp-client-identifier of a roaming host.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

When you set the client_identifier_prepend_zero parameter to "true", it prefixes a 0 (zero) to the DHCP client identifier whenever option 61 is used to assign roaming hosts. The following is an excerpt from "man dhcp-options":

Please be aware that some DHCP clients, when configured with client identifiers that are ASCII text, will prepend a zero to the ASCII text. So you may need to write: option dhcp-client-identifier "\000foo"; rather than: option dhcp-client-identifier "foo";

Returns

If you specified a parameter, the method returns true when the modification succeeds and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get client_identifier_prepend_zero
my $client_identifier_prepend_zero = $roaming_host->client_identifier_prepend_zero();
#Modify dhcp_client_identifier
$roaming_host->client_identifier_prepend_zero("true");
```

comment( )

Use this method to set or retrieve the descriptive comment of a DHCP Roaming Host object.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

Desired comment in string format with a maximum of 256 bytes.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get comment
my $comment = $roaming_host->comment();
#Modify comment
$roaming_host->comment("Modified DHCP Roaming Host object comment");
```

ddns_domainname( )

Use this method to set or retrieve the DDNS domain name value of a DHCP Roaming Host object.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Setting this method to a defined value implicitly sets the override_ddns_domainname method to "true". Setting the parameter to undefined causes the appliance to use the grid default and automatically resets the override_ddns_domainname attribute to "false".

Note that when ddns_domainname is set to a defined value and override_dddns_domainname is set to "false", the last operation takes precedence. Thus the sequence $object->ddns_domainname("testdomain"); $object->override_ddns_domainname("false"); will set override_ddns_domainname to "false", and the sequence $object->override_ddns_domainname("false"); $object->ddns_domainname("testdomain"); will result in override_ddns_domainname="true".

Parameter

Specify the DDNS domain name.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get the DDNS domain name
my $ddns_domainname = $roaming_host->ddns_domainname();
#Modify the DDNS domain name
$roaming_host->ddns_domainname("ddns.domain.com");
```

ddns_hostname( )

Use this method to set or retrieve the DDNS host name value of a DHCP Roaming Host object.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

Specify the DDNS host name.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get the DDNS host name
my $ddns_hostname = $roaming_host->ddns_hostname();
#Modify the DDNS host name
$roaming_host->ddns_hostname("host.ddns.domain.com");
```

deny_bootp( )

Use this method to set or retrieve the deny_bootp flag of a DHCP Roaming Host object.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

Specify "true" to set the deny_bootp flag or "false" to deactivate/unset it. The default value is "false".

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get deny_bootp
my $deny_bootp = $roaming_host->deny_bootp();
#Modify deny_bootp
$roaming_host->deny_bootp("true");
```

dhcp_client_identifier( )

Use this method to set or retrieve the dhcp-client-identifier value of a DHCP Roaming Host object.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

Use the dhcp_client_identifier or uid to assign roaming hosts if the DHCP clients send option 61. A DHCP client identifier can be a string or a MAC address. Set the "match_client" field to "CLIENT_IDENTIFIER" for roaming hosts to use the dhcp_client_identifier instead of the mac_address.

Returns

If you specified a parameter, the method returns true when the modification succeeds and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get dhcp_client_identifier
my $uid = $roaming_host->dhcp_client_identifier();
#Modify dhcp_client_identifier
$roaming_host->dhcp_client_identifier("foo");
or
$roaming_host->dhcp_client_identifier("1:11::11::11::11::11::12");
```

disable( )

Use this method to set or retrieve the disable flag of a DHCP Roaming Host object.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

Specify "true" to set the disable flag or "false" to deactivate/unset it. The default value is "false".

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get disable
my $disable = $roaming_host->disable();
#Modify disable
$roaming_host->disable("true");
```

discovered_name( )

Use this method to retrieve the discovered name of a DHCP Roaming Host object.

Parameter

none

Returns

The method returns the discovered name.

Example

```
#Get discovered_name
my $discovered_name = $roaming_host->discovered_name();
```

discoverer( )

Use this method to retrieve the discoverer of a DHCP Roaming Host object.

Parameter

none

Returns

The method returns the discoverer value.

Example

```
#Get discoverer
my $discoverer = $roaming_host->discoverer();
```

enable_ddns( )

Use this method to set or retrieve the enable_ddns flag of a DHCP Roaming Host object.

If enable_ddns value is specified as "true", then the A and PTR records corresponding to the Roaming Host object will be placed on the DNS server.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

Specify "true" to set the enable_ddns flag or "false" to deactivate/unset it. The default value is "false".

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get enable_ddns
my $enable_ddns = $roaming_host->enable_ddns();
#Modify enable_ddns
$roaming_host->enable_ddns("true");
```

extattrs( )

Use this method to set or retrieve the extensible attributes associated with a DHCP Roaming Host object.

Parameter

Valid value is a hash reference containing the names of extensible attributes and their associated values (Infoblox::Grid::Extattr objects).

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get extattrs
```

```
my $ref_extattrs = $roaming_host->extattrs();
#Modify extattrs
$roaming_host->extattrs({ 'Site' => $extattr1, 'Administrator' => $extattr2 });
```

extensible_attributes( )

Use this method to set or retrieve the extensible attributes associated with a DHCP Roaming Host object

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

For valid values for extensible attributes, see Infoblox::Grid::ExtensibleAttributeDef/Extensible Attribute Values.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get extensible attributes
my $ref_extensible_attributes = $roaming_host->extensible_attributes();
#Modify extensible attributes
$roaming_host->extensible_attributes({ 'Site' => 'Santa Clara', 'Administrator' => [ 'Peter', 'Tom'
] });
```

first_discovered( )

Use this method to retrieve the date and time that the DHCP Roaming Host object was first discovered.

Parameter

none

Returns

The method returns the date and time that the DHCP Roaming Host object was first discovered.

Example

```
#Get first_discovered
my $first_discovered = $roaming_host->first_discovered();
```

force_roaming_hostname( )

Use this method to set or retrieve the force_roaming_hostname flag of a DHCP Roaming Host object.

If the force_roaming_hostname value is "true", then the roaming host name is used as the ddns_hostname.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

Specify "true" to set the force_roaming_hostname flag or "false" to deactivate/unset it. The default value is "false".

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get force_roaming_hostname
my $force_roaming_hostname = $roaming_host->force_roaming_hostname();
#Modify force_roaming_hostname
$roaming_host->force_roaming_hostname("true");
```

ignore_dhcp_option_list_request( )

Use this method to set or retrieve the ignore_dhcp_option_list_request flag of a DHCP roaming host object. If this flag is set to false all the defined DHCP options will be returned to the client.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

Specify "true" to set the ignore_dhcp_option_list_request flag or "false" to deactivate/unset it. The default value is false which indicates that this attribute inherit the member level setting..

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
# Get ignore_dhcp_option_list_request
my $ignore_dhcp_option_list_request = $roaming_host->ignore_dhcp_option_list_request();
# Un-override ignore_dhcp_option_list_request
$roaming_host->ignore_dhcp_option_list_request(undef);
# Modify ignore_dhcp_option_list_request
$roaming_host->ignore_dhcp_option_list_request("true");
```

ipv6_client_hostname( )

Use this method to retrieve the ipv6_client_hostname of a DHCP Roaming Host object. This method specifies the host name that the DHCP client sends to the Infoblox appliance using DHCP option 12. The method is read-only.

Parameter

N/A

Returns

The method returns the attribute value.

Example

```
#Get ipv6_client_hostname
my $ipv6_client_hostname = $roaming_host->ipv6_client_hostname();
```

ipv6_ddns_domainname( )

Use this method to set or retrieve the IPv6 DDNS domain name value for the roaming host.

Setting this method to a defined value implicitly sets the override_ipv6_ddns_domainname method to "true". Setting the parameter to undefined causes the appliance to use the upper-level default and automatically resets the override_ipv6_ddns_domainname attribute to "false".

Note that when ipv6_ddns_domainname is set to a defined value and override_ipv6_ddns_domainname is set to "false", the last operation takes precedence. Thus the sequence $object->ipv6_ddns_domainname("testdomain"); $object->override_ipv6_ddns_domainname("false"); will set override_ipv6_ddns_domainname to "false", and the sequence $object->override_ipv6_ddns_domainname("false"); $object->ipv6_ddns_domainname("testdomain"); will result in override_ipv6_ddns_domainname="true".

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

The DDNS domain name in FQDN (Fully Qualified Domain Name) format.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get ipv6_ddns_domainname
my $ipv6_ddns_domainname = $roaming_host->ipv6_ddns_domainname();
#Modify ipv6_ddns_domainname
$roaming_host->ipv6_ddns_domainname("domain.com");
```

ipv6_ddns_hostname( )

Use this method to set or retrieve the IPv6 DDNS host name value of a DHCP Roaming Host object.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

Specify the IPv6 DDNS host name.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get the IPv6 DDNS host name
my $ipv6_ddns_hostname = $roaming_host->ipv6_ddns_hostname();
#Modify the IPv6 DDNS host name
$roaming_host->ipv6_ddns_hostname("host.ddns.domain.com");
```

ipv6_domain_name( )

Use this method to set or retrieve the ipv6_domain_name value of a DHCP Roaming Host object.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Setting this method to a defined value implicitly sets the override_ipv6_domain_name method to "true". Setting the parameter to undefined causes the appliance to use the upper level default and automatically resets the override_ipv6_domain_name attribute to "false".

Note that when ipv6_domain_name is set to a defined value and override_ipv6_domain_name is set to "false", the last operation takes precedence. Thus the sequence $object->ipv6_domain_name("testdomain"); $object->override_ipv6_domain_name("false"); will set override_ipv6_domain_name to "false", and the sequence $object->override_ipv6_domain_name("false"); $object->ipv6_domain_name("testdomain"); will result in override_ipv6_domain_name="true".

Parameter

The name, in string format, of the client's domain for resolving names in DNS.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get ipv6_domain_name
my $ipv6_domain_name = $roaming_host->ipv6_domain_name();
#Modify ipv6_domain_name
$roaming_host->ipv6_domain_name("test_domain.com");
```

ipv6_domain_name_servers( )

Use this method to set or retrieve the ipv6_domain_name_servers value of a DHCP Roaming Host object.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Setting this method to a defined value implicitly sets the override_ipv6_domain_name_servers method to "true". Setting the parameter to undefined causes the appliance to use the upper-level default and automatically resets the override_ipv6_domain_name_servers attribute to "false".

Note that when ipv6_domain_name_servers is set to a defined value and override_ipv6_domain_name_servers is set to "false", the last operation takes precedence. Thus the sequence $object->ipv6_domain_name_servers("2001::1,2001::2"); $object->override_ipv6_domain_name_servers("false"); will set override_ipv6_domain_name_servers to "false", and the sequence $object->override_ipv6_domain_name_servers("false"); $object->ipv6_domain_name_servers("2001::1,2001::2"); will result in override_ipv6_domain_name_servers="true".

Parameter

The IPv6 addresses of the DNS servers.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get ipv6_domain_name_servers
my $ipv6_domain_name_servers = $roaming_host->ipv6_domain_name_servers();
#Modify ipv6_domain_name_servers
$roaming_host->ipv6_domain_name_servers("2001::1,2001::2");
```

ipv6_duid( )

Use this method to set or retrieve the DHCPv6 Unique Identifier (DUID) of the Roaming Host object.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

The DUID of the matching client in hexadecimal format.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get ipv6_duid
my $ipv6_duid = $roaming_host->ipv6_duid();
#Modify ipv6_duid
$roaming_host->ipv6_duid("1c2edfaa");
```

ipv6_enable_ddns( )

Use this method to set or retrieve the ipv6_enable_ddns flag for the roaming host. Set this flag to enable DHCPv6 dynamic DNS updates.

Setting this method to a defined value implicitly sets the override_ipv6_enable_ddns method to "true". Setting the parameter to undefined causes the appliance to use the upper-level default and automatically resets the override_ipv6_enable_ddns attribute to "false".

Note that when ipv6_enable_ddns is set to a defined value and override_ipv6_enable_ddns is set to "false", the last operation takes precedence. Thus the sequence $object->ipv6_enable_ddns("false"); $object->override_ipv6_enable_ddns("false"); will set override_ipv6_enable_ddns to "false", and the sequence $object->override_ipv6_enable_ddns("false"); $object->ipv6_enable_ddns("false"); will result in override_ipv6_enable_ddns="true".

Parameter

Specify "true" to enable this feature or "false" to disable it. The default value is "false".

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get ipv6_enable_ddns
my $ipv6_enable_ddns = $roaming_host->ipv6_enable_ddns();
#Modify ipv6_enable_ddns
$roaming_host->ipv6_enable_ddns("true");
```

ipv6_force_roaming_hostname( )

Use this method to set or retrieve the ipv6_force_roaming_hostname flag of a DHCP Roaming Host object.

If the ipv6_force_roaming_hostname value is "true", then the roaming host name is used as the ddns_hostname. This flag applies to IPv6 roaming hosts.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

Specify "true" to set the ipv6_force_roaming_hostname flag or "false" to deactivate/unset it. The default value is "false".

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get ipv6_force_roaming_hostname
my $ipv6_force_roaming_hostname = $roaming_host->ipv6_force_roaming_hostname();
#Modify ipv6_force_roaming_hostname
$roaming_host->ipv6_force_roaming_hostname("true");
```

ipv6_match_option( )

Use this method to set or retrieve the ipv6_match_option value of a DHCP Roaming Host object.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

This field specifies the identification method for an IPv6 or mixed IPv4/IPv6 roaming host. Currently, the only supported value for this field is "duid", which corresponds to identification by DHCPv6 unique ID. The default value of this parameter is undefined.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get ipv6_match_option
my $ipv6_match_option = $roaming_host->ipv6_match_option();
#Modify ipv6_match_option
$roaming_host->ipv6_match_option("duid");
```

ipv6_options( )

Use this method to set or retrieve the IPv6 options of a DHCP Roaming Host object.

Setting this method to a defined value implicitly sets the override_ipv6_options method to "true". Setting the parameter to undefined causes the appliance to use the upper-level default and automatically resets the override_ipv6_options attribute to "false".

Note that when ipv6_options is set to a defined value and override_ipv6_options is set to "false", the last operation takes precedence. Thus the sequence $object->ipv6_options([$option1]); $object->override_ipv6_options("false"); will set override_ipv6_options to "false", and the sequence $object->override_ipv6_options("false"); $object->ipv6_options([$option1]); will result in override_ipv6_options="true".

Parameter

The valid value is an array reference that contains Infoblox::DHCP::Option object(s).

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get ipv6_options
my $ipv6_options = $roaming_host->ipv6_options();
#Modify ipv6_options
my $option1 = Infoblox::DHCP::Option->new(
    name   => "dhcp6.info-refresh-time",
    value  => "1000"
);
$roaming_host->ipv6_options([$option1]);
```

ipv6_template( )

Use this method to set or retrieve the fixed address ipv6_template name for a DHCP Roaming Host object.

Specify the IPv6 fixed address template that is used to create the DHCP roaming host.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

The IPv6 fixed address template name in sting format.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

The ipv6_template parameter can be used only to add a new DHCP roaming host. It cannot be used to modify an object or return an object during a get operation.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get ipv6_template
my $ipv6_template = $roaming_host->ipv6_template();
#Modify ipv6_template
$roaming_host->ipv6_template("Template_Name");
```

last_discovered( )

Use this method to retrieve the time this object was last seen by a discovery job.

The default value for this field is empty.

Parameter

N/A

Returns

The method returns the attribute value.

Example

```
#Get last_discovered
my $last_discovered = $roaming_host->last_discovered();
```

mac( )

Use this method to set or retrieve the MAC address value of a DHCP Roaming Host object.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

MAC Address of the DHCP roaming host object. Format for the hexadecimal strings for MAC addresses and vendor prefixes with colons or dashes. Both of the following formats are acceptable: 11:11:11:11:11:11 and 11-11-11-11-11-11.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get mac
my $mac = $roaming_host->mac();
#Modify mac
$roaming_host->mac("11::11::11::11::11::12");
```

match_client( )

Use this method to set or retrieve the match_client value of a DHCP Roaming Host object.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

match_client "MAC": The fixed IP address is leased to the matching MAC address.
match_client "CLIENT_IDENTIFIER": The fixed IP address is leased to the matching DHCP client identifier
Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get match_client
my $match_client = $roaming_host->match_client();
#Modify match_client
$roaming_host->match_client("CLIENT_IDENTIFIER");
```

name( )

Use this method to set or retrieve the roaming host name associated with the DHCP Roaming Host object.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

NOTE: To enable using roaming hosts, you must allow roaming hosts at the grid level by setting $griddhcp->enable_roaming_hosts("true");

Parameter

Desired roaming host name in string format with a maximum of 64 bytes.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get roaming host name
my $roaming_host_name = $roaming_host->name();
#Modify roaming host name
$roaming_host->name("JimmyJaz");
```

netbios( )

Use this method to retrieve the name in the NetBIOS reply that responded to a NetBIOS query.

Returns

The method returns the attribute value.

Example

```
#Get netbios
my $netbios = $roaming_host->netbios();
```

network_component_description( )

Use this method to retrieve the description of the network device that is connected to the DHCP Roaming Host object.

Parameter

none

Returns

The method returns the network device description.

Example

```
#Get network_component_description
my $network_component_description = $roaming_host->network_component_description();
```

network_component_ip( )

Use this method to retrieve the IP address of the network device that is connected to the DHCP Roaming Host object.

Parameter

none

Returns

The method returns the IP address of the network device.

Example

```
#Get network_component_ip
my $network_component_ip = $roaming_host->network_component_ip();
```

network_component_name( )

Use this method to retrieve the name of the network device that is connected to the DHCP Roaming Host object.

Parameter

none

Returns

The method returns the network device name.

Example

```
#Get network_component_name
my $network_component_name = $roaming_host->network_component_name();
```

network_component_port_description( )

Use this method to retrieve the description of the network device port that is connected to the DHCP Roaming Host object.

Parameter

none

Returns

The method returns the description of the port on the network device.

Example

```
#Get network_component_port_description
my $network_component_port_description = $roaming_host->network_component_port_description();
```

network_component_port_name( )

Use this method to retrieve the name of the network device port that is connected to the DHCP Roaming Host object.

Parameter

none

Returns

The method returns the name of the port on the network device.

Example

```
#Get network_component_port_name
my $network_component_port_name = $roaming_host->network_component_port_name();
```

network_component_port_number( )

Use this method to retrieve the number of the network device port that is connected to the DHCP Roaming Host object.

Parameter

none

Returns

The method returns the network device port number.

Example

```
#Get network_component_port_number
my $network_component_port_number = $roaming_host->network_component_port_number();
```

network_component_type( )

Use this method to retrieve the type of the network device that is connected to DHCP Roaming Host object.

Parameter

none

Returns

The method returns the network device type.

Example

```
#Get network_component_type
my $network_component_type = $roaming_host->network_component_type();
```

network_view( )

Use this method to set or retrieve the network view of the DHCP roaming host

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

The default value is the "default" network view, which means the roaming host is in the default network view.

Parameter

Valid value is a defined Infoblox::DHCP::View object. The default network view value is the system-defined default network view object.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get network view
my $nview = $roaming_host->network_view();
#Modify network view, with an Infoblox::DHCP::View object
$roaming_host->network_view($nview);
```

nextserver( )

Use this method to set or retrieve the nextserver value of a DHCP Roaming Host object.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

IP address of the boot file server on which the boot file is stored.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get nextserver
my $nextserver = $roaming_host->nextserver();
#Modify nextserver
$roaming_host->nextserver("3.3.3.4");
```

options( )

Use this method to set or retrieve the options of a DHCP Roaming Host object.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

See Infoblox::DHCP::Option for parameters and methods available for this object.

Parameter

Options describe network configuration settings and various services available on the network. These options occur as variable-length fields at the end of DHCP messages.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get options
my $options = $roaming_host->options();
#Modify options
$roaming_host->options([$option1, $option2]);
```

os( )

Use this method to retrieve the operating system associated with this roaming host.

Returns

The method returns the attribute value.

Example

```
#Get os
my $os = $roaming_host->os();
```

override_ddns_domainname( )

The override_ddns_domainname attribute controls whether the ddns_domainname value in the DHCP Roaming Host is used, instead of the grid default.

The override_ddns_domainname attribute can be specified explicitly. It is also set implicitly when ddns_domainname is set to a defined value.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

Set the parameter to "true" to override the grid-level setting for ddns_domainname. Set the parameter to "false" to inherit the grid-level setting for ddns_domainname.

The default value of this parameter is "false".

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Getting override_ddns_domainname
my $override_ddns_domainname=$roaming_host->override_ddns_domainname( );
#Modifying override_ddns_domainname
$roaming_host->override_ddns_domainname("true");
```

override_ipv6_ddns_domainname( )

The override_ipv6_ddns_domainname attribute controls whether the ddns_domainname value in the DHCP Roaming Host object is used, instead of the upper-level default.

The override_ipv6_ddns_domainname attribute can be specified explicitly. It is also set implicitly when ipv6_ddns_domainname is set to a defined value.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

Set the parameter to "true" to override the upper-level setting for ddns_domainname. Set the parameter to "false" to inherit the upper-level setting for ipv6_ddns_domainname.

The default value of this parameter is "false".

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Getting override_ipv6_ddns_domainname
my $override_ipv6_ddns_domainname=$roaming_host->override_ipv6_ddns_domainname( );
#Modifying override_ipv6_ddns_domainname
$roaming_host->override_ipv6_ddns_domainname("true");
```

override_ipv6_domain_name( )

Use this method to set or retrieve the override_ipv6_domain_name value of a DHCP Roaming Host object.

The override_ipv6_domain_name attribute can be specified explicitly. It is also set implicitly when ipv6_domain_name is set to a defined value.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

Set the parameter to "true" to override the upper-level setting for ipv6_domain_name. Set the parameter to "false" to inherit the upper-level setting for ipv6_domain_name.

The default value of this parameter is "false".

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get override_ipv6_domain_name
my $override_ipv6_domain_name = $roaming_host->override_ipv6_domain_name();
#Modify override_ipv6_domain_name
$roaming_host->override_ipv6_domain_name("test_domain.com");
```

override_ipv6_domain_name_servers( )

Use this method to set or retrieve the override_ipv6_domain_name_servers value of a DHCP Roaming Host object.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

Set the parameter to "true" to override the upper-level setting for ipv6_domain_name_servers. Set the parameter to "false" to inherit the upper-level setting for ipv6_domain_name_servers.

The default value of this parameter is "false".

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get override_ipv6_domain_name_servers
my $override_ipv6_domain_name_servers = $roaming_host->override_ipv6_domain_name_servers();
#Modify override_ipv6_domain_name_servers
$roaming_host->override_ipv6_domain_name_servers("true");
```

override_ipv6_enable_ddns( )

The override_ipv6_enable_ddns attribute controls whether the ipv6_enable_ddns value in the DHCP Roaming Host object is used, instead of the upper-level default.

The override_ipv6_enable_ddns attribute can be specified explicitly. It is also set implicitly when ipv6_enable_ddns is set to a defined value.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

Set the parameter to "true" to override the upper-level setting for ipv6_enable_ddns. Set the parameter to "false" to inherit the upper-level setting for ipv6_enable_ddns.

The default value of this parameter is "false".

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Getting override_ipv6_enable_ddns
my $override_ipv6_enable_ddns=$roaming_host->override_ipv6_enable_ddns( );
#Modifying override_ipv6_enable_ddns
$roaming_host->override_ipv6_enable_ddns("true");
```

override_ipv6_options( )

The override_ipv6_options attribute controls whether the ipv6_options value in the DHCP Roaming Host object is used, instead of the upper-level default.

The override_ipv6_options attribute can be specified explicitly. It is also set implicitly when ipv6_options is set to a defined value.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

Set the parameter to "true" to override the upper-level setting for ipv6_options. Set the parameter to "false" to inherit the upper-level setting for ipv6_options.

The default value of this parameter is "false".

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Getting override_ipv6_options
my $override_ipv6_options=$roaming_host->override_ipv6_options( );
#Modifying override_ipv6_options
$roaming_host->override_ipv6_options("true");
```

override_preferred_lifetime( )

The override_preferred_lifetime attribute controls whether the preferred_lifetime value in the DHCP Roaming Host object is used, instead of the upper-level default.

The override_preferred_lifetime attribute can be specified explicitly. It is also set implicitly when preferred_lifetime is set to a defined value.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

Set the parameter to "true" to override the grid-level setting for preferred_lifetime. Set the parameter to "false" to inherit the upper-level setting for preferred_lifetime.

The default value of this parameter is "false".

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Getting override_preferred_lifetime
my $override_preferred_lifetime=$roaming_host->override_preferred_lifetime( );
#Modifying override_preferred_lifetime
$roaming_host->override_preferred_lifetime("true");
```

override_valid_lifetime( )

The override_valid_lifetime attribute controls whether the valid_lifetime value in the DHCP Roaming Host object is used, instead of the upper-level default.

The override_valid_lifetime attribute can be specified explicitly. It is also set implicitly when valid_lifetime is set to a defined value.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

Set the parameter to "true" to override the grid-level setting for valid_lifetime. Set the parameter to "false" to inherit the upper-level setting for valid_lifetime.

The default value of this parameter is "false".

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Getting override_valid_lifetime
my $override_valid_lifetime=$roaming_host->override_valid_lifetime( );
#Modifying override_valid_lifetime
```

```
$roaming_host->override_valid_lifetime("true");
```

port_duplex( )

Use this method to retrieve the duplex setting of the network device port that is connected to the DHCP Roaming Host object.

Parameter

none

Returns

The method returns the port duplex setting. Returned values, if any, are one of the following:

Full
Half
Example

```
#Get port_duplex
my $port_duplex = $roaming_host->port_duplex();
```

port_link_status( )

Use this method to retrieve the link status of the network device port that is connected to the DHCP Roaming Host object.

Parameter

none

Returns

The method returns the network device port link status. Returned values, if any, are one of the following: Connected Not Connected Unknown

Example

```
#Get port_link_status
my $port_link_status = $roaming_host->port_link_status();
```

port_speed( )

Use this method to retrieve the speed of the network device port that is connected to the DHCP Roaming Host object.

Parameter

none

Returns

The method returns the network device port speed. Returned values, if any, are one of the following:

10 M
100 M
1G
10G
100G
Unknown
Example

```
#Get port_speed
my $port_speed = $roaming_host->port_speed();
```

port_status( )

Use this method to retrieve the status of the network device port that is connected to the DHCP Roaming Host object.

Parameter

none

Returns

The method returns the network device port status. Returned values, if any, are one of the following:

Up
Down
Unknown

Example

```
#Get port_status
my $port_status = $roaming_host->port_status();
```

port_vlan_description( )

Use this method to retrieve the VLAN description of the network device port that is connected to the DHCP Roaming Host object.

Parameter

none

Returns

The method returns the VLAN description of the network device port.

Example

```
#Get port_vlan_description
my $port_vlan_description = $roaming_host->port_vlan_description();
```

port_vlan_name( )

Use this method to retrieve the VLAN name of the network device port that is connected to the DHCP Roaming Host object.

Parameter

none

Returns

The method returns the network device port VLAN name.

Example

```
#Get port_vlan_name
my $port_vlan_name = $roaming_host->port_vlan_name();
```

port_vlan_number( )

Use this method to retrieve the VLAN number of the network device port that is connected to the DHCP Roaming Host object.

Parameter

none

Returns

The method returns the port VLAN number.

Example

```
#Get port_vlan_number
my $port_vlan_number = $roaming_host->port_vlan_number();
```

preferred_lifetime( )

Use this method to set or retrieve the preferred lifetime of the roaming host.

Setting this method to a defined value implicitly sets the override_preferred_lifetime method to "true". Setting the parameter to undefined causes the appliance to use the upper-level default and automatically resets the override_preferred_lifetime attribute to "false".

Note that when preferred_lifetime is set to a defined value and override_preferred_lifetime is set to "false", the last operation takes precedence. Thus the sequence $object->preferred_lifetime(20000); $object->override_preferred_lifetime("false"); will set override_preferred_lifetime to "false", and the sequence $object->override_preferred_lifetime("false"); $object->preferred_lifetime(20000); will result in override_preferred_lifetime="true".

Parameter

Preferred lifetime value in seconds. The default value is 27000. Note that the preferred lifetime cannot be greater than the valid lifetime.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get preferred_lifetime
my $preferred_lifetime = $roaming_host->preferred_lifetime();
#Modify preferred_lifetime
$roaming_host->preferred_lifetime(12345);
```

pxe_lease_time( )

Use this method to set or retrieve the pxe_lease_time value of a DHCP Roaming Host object.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

Enter appropriate values in the Days, Hours, Mins and Secs fields. Users can specify the duration of time it takes a host to connect to a boot server, such as a TFTP server, and download the file it needs to boot.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get pxe_lease_time
my $pxe_lease_time = $roaming_host->pxe_lease_time();
#Modify pxe_lease_time
$roaming_host->pxe_lease_time("369");
```

template( )

Use this method to set the fixed address template name for a DHCP Roaming Host object.

Template is the name of a DHCP Fixed Address template which will be used to create the DHCP Roaming Host.

Parameter

The valid name in string format.

Returns

The method returns true when the modification succeeds, and returns false when the operation fails.

The template parameter can be used to only add a new DHCP Roaming Host and cannot be used during object modify or returned during object get.

Example

```
#Add RoamingHost with template
my $roaming_host = Infoblox::DHCP::RoamingHost->new(
        "name"   => "JohnSmith",
        "template"   => "Template_Name",
);
#Submit for addition
my $response = $session->add($roaming_host)
```

valid_lifetime( )

Use this method to set or retrieve the preferred lifetime of the roaming host.

Setting this method to a defined value implicitly sets the override_valid_lifetime method to "true". Setting the parameter to undefined causes the appliance to use the upper-level default and automatically resets the override_valid_lifetime attribute to "false".

Note that when valid_lifetime is set to a defined value and override_valid_lifetime is set to "false", the last operation takes precedence. Thus the sequence $object->valid_lifetime(20000); $object->override_valid_lifetime("false"); will set override_valid_lifetime to "false", and the sequence $object->override_valid_lifetime("false"); $object->valid_lifetime(20000); will result in override_valid_lifetime="true".

Parameter

Valid lifetime value in seconds. The default value is 43200.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get valid_lifetime
my $valid_lifetime = $roaming_host->valid_lifetime();
#Modify valid_lifetime
$roaming_host->valid_lifetime(12345);
```

SAMPLE CODE

The following sample code demonstrates the different functions that can be applied to an object such as add, search, modify, and remove. This sample also includes error handling for the operations.

```
#Preparation prior to a DHCP Fixed Address object insertion

#PROGRAM STARTS: Include all the modules that will be used
use strict;
use Infoblox;

#Create a session to the Infoblox appliance

my $session = Infoblox::Session->new(
               master   => "192.168.1.2", #appliance host ip
               username => "admin",       #appliance user login
               password => "infoblox"     #appliance password
);

unless ($session) {
       die("Construct session failed: ",
               Infoblox::status_code() . ":" . Infoblox::status_detail());
}
print "Session created successfully\n";
```

#Modify Grid DHCP object to enable support of the Roaming Hosts

```
my $grid_dhcp=$session->get(
                            object => "Infoblox::Grid::DHCP",
                            name=>"Infoblox"
                            );
unless ($grid_dhcp) {
    die("Get Grid DHCP object failed: ",
        $session->status_code() . ":" . $session->status_detail());
}
print "Get Grid DHCP object found at least 1 matching entry\n";

my $save_grid_rh_status=$grid_dhcp->enable_roaming_hosts();
$grid_dhcp->enable_roaming_hosts("true");
unless($session->modify($grid_dhcp)) {
    die("Modifying Grid DHCP object failed: ",
        $session->status_code() . ":" . $session->status_detail());
}
print "Roaming Hosts enabled at Grid level successfuly\n";

#Adding DHCP options
my $option1 = Infoblox::DHCP::Option->new(
                                          "name"  => "lease-time",
                                          "value" => "50"
                                         );

unless($option1) {
     die("Construct option1 object failed: ",
           Infoblox::status_code() . ":" . Infoblox::status_detail());
}
print "option1 object created successfully\n";

my $option2 = Infoblox::DHCP::Option->new(
                                          "type"      => "routers",
                                          "ipv4addrs" => ["10.0.0.1","10.0.0.255"],
                                         );

unless($option2) {
     die("Construct option2 object failed: ",
           Infoblox::status_code() . ":" . Infoblox::status_detail());
}
print "option2 object created successfully\n";
```

#Create a DHCP Roaming Host object

```
#Create a DHCP Roaming Host object
my $roaming_host = Infoblox::DHCP::RoamingHost->new(
        "name"        => "JohnSmith",
        "mac"             => "00:00:00:11:11:11",
        "comment"         => "add roaming host",
        "enable_ddns"     => "true",
        "pxe_lease_time" => "3600",
        "bootserver"      => "someserver.com",
        "nextserver"      => "3.3.3.3",
        "bootfile"        => "bootfile.com",
        "options"         => [ $option1 , $option2 ],
        );
unless($roaming_host) {
     die("Construct roaming host failed: ",
           Infoblox::status_code() . ":" . Infoblox::status_detail());
}
print "DHCP Roaming Host object created successfully\n";

#Verify if the DHCP Roaming Host exists in the Infoblox appliance
my $object = $session->get(object => "Infoblox::DHCP::RoamingHost", mac => "00:00:00:11:11:11");
unless ($object) {
   print "roaming host does not exist on server, safe to add the roaming host\n";
   $session->add($roaming_host)
      or die("Add roaming host failed: ",
             $session->status_code() . ":" . $session->status_detail());
}
print "DHCP Roaming Host added successfully\n";
```

#Search for a specific DHCP Roaming Host object

```
#Search for all DHCP Roaming Host objects with a given mac address
my @retrieved_objs = $session->search(
    object  => "Infoblox::DHCP::RoamingHost",
    mac     => "00:00:00:11:11.*",
     );
my $object = $retrieved_objs[0];

unless ($object) {
       die("Search for Roaming Host object failed: ",
              $session->status_code() . ":" . $session->status_detail());
}
print "Search Roaming Host object found at least 1 matching entry\n";
```

#Get and modify a DHCP Roaming Host object

```
#Get DHCP Roaming Host object through the session
my @retrieved_objs = $session->get(
    object => "Infoblox::DHCP::RoamingHost",
    mac    => "00:00:00:11:11:11",
);
my $object = $retrieved_objs[0];
unless ($object) {
    die("Get Roaming Host object failed: ",
        $session->status_code() . ":" . $session->status_detail());
}
print "Get Roaming Host object found at least 1 matching entry\n";

#Modify one of the attributes of the obtained Roaming Host object
$object->pxe_lease_time("1234");

#Apply the changes
$session->modify($object)
    or die("Modify Roaming Host object failed: ",
           $session->status_code() . ":" . $session->status_detail());
print "Roaming Host object modified successfully \n";
```

#Remove a Roaming Host object

```
#Get Roaming Host object through the session
my @retrieved_objs = $session->get(
    object => "Infoblox::DHCP::RoamingHost",
    mac    => "00:00:00:11:11:11",
);
my $object = $retrieved_objs[0];
unless ($object) {
```

```
    die("Get Roaming Host object failed: ",
        $session->status_code() . ":" . $session->status_detail());
}
print "Get Roaming Host object found at least 1 matching entry\n";

#Submit the object for removal
$session->remove($object)
    or die("Remove Roaming Host object failed: ",
        $session->status_code() . ":" . $session->status_detail());
print "Roaming Host object removed successfully \n";

#Restore Grid DHCP
$grid_dhcp->enable_roaming_hosts($save_grid_rh_status);
print "Grid DHCP setting restored successfully\n" if($session->modify($grid_dhcp));

####PROGRAM ENDS####
```

AUTHOR

Infoblox Inc. http://www.infoblox.com/

SEE ALSO

Infoblox::Session->add(), Infoblox::Session->get(), Infoblox::Session->modify(), Infoblox::Session->remove(), Infoblox::Session->search(), Infoblox::Session, Infoblox::Grid::MSServer::AdUser::Data

---

COPYRIGHT

Copyright (c) 2017 Infoblox Inc.

Infoblox::DHCP::SharedNetwork - DHCP Shared Network object

- NAME
- DESCRIPTION
- CONSTRUCTOR
- SESSION METHODS
  - Infoblox::Session->add( )
  - Infoblox::Session->get( )
  - Infoblox::Session->modify( )
  - Infoblox::Session->remove( )
  - Infoblox::Session->search( )
- METHODS
  - authority( )
  - bootfile( )
  - bootserver( )
  - comment( )
  - ddns_generate_hostname( )
  - ddns_server_always_updates( )
  - ddns_update_fixed_addresses( )
  - ddns_ttl( )
  - ddns_use_option81( )
  - deny_bootp( )
  - disable( )
  - enable_ddns( )
  - extattrs( )
  - extensible_attributes( )
  - ignore_id( )
  - ignore_mac_addresses( )
  - ignore_dhcp_option_list_request( )
  - lease_scavenge_time( )
  - ms_ad_user_data( )
  - logic_filters( )
  - name( )
  - networks( )
  - network_view( )
  - nextserver( )
  - options( )
  - override_ddns_ttl( )
  - override_ignore_id( )
  - override_lease_scavenge_time( )
  - override_logic_filters( )
  - override_update_dns_on_lease_renewal( )
  - pxe_lease_time( )
  - update_dns_on_lease_renewal( )
- SAMPLE CODE
- AUTHOR
- SEE ALSO
- COPYRIGHT

NAME

Infoblox::DHCP::SharedNetwork - DHCP Shared Network object

DESCRIPTION

You can create a shared network when two subnets share a particular network segment. Before creating a shared network, you must first create the subnetworks. For example, you must first create the networks 10.32.1.0 and 10.30.0.0 before designating them as a shared network.

CONSTRUCTOR

```
  my $sharednetwork = Infoblox::DHCP::SharedNetwork->new(
                  name                              => $string,                          #Required
                  networks                          => [ $Network1,$Network2,...],       #Required
                  authority                         => "true" | "false" | undef,         #Optional / Default
is undefined
                  bootfile                          => $filename | undef,                #Optional / Default
is undefined
                  bootserver                        => $ipv4addr | $fqdn | undef,
#Optional / Default is undefined
                                                       $string   undef                   #Optional / Default
```

```
                comment                          =>         |      ,
is undefined
                ddns_generate_hostname           => "true" | "false" | undef,          #Optional / Default
is undefined
                ddns_server_always_updates       => "true" | "false",                  #Optional / Default
is "true"
                ddns_update_fixed_addresses      => "true" | "false" | undef,          #Optional / Default
is undefined
                ddns_ttl                         => $num,                              #Optional / Default
is zero
                ddns_use_option81                => "true" | "false" | undef,          #Optional / Default
is undefined
                deny_bootp                       => "true" | "false" | undef,          #Optional / Default
is undefined
                disable                          => "true" | "false",                  #Optional / Default
is "false"
                enable_ddns                      => "true" | "false" | undef,          #Optional / Default
is undefined
                extattrs                         => { $string => $extattr, ... },      #Optional / Default
is undefined
                extensible_attributes            => { $string => $string | $num, $string => [ $string |
$num, ... ], ... }, #Optional / Default is undefined
                ignore_dhcp_option_list_request  => "true" | "false" | undef,          #Optional / Default
is "false"
                ignore_id                        => 'NONE' | 'CLIENT' | 'MACADDR',     #Optional / Default
is 'NONE'
                ignore_mac_addresses             => [ "mac_addr", ... ] | undef        #Optional / Default
is undefined
                lease_scavenge_time              => $number,                           #Optional / Default
is -1
                network_view                     => $NetworkView,                      #Optional / Default
is "default" network view
                nextserver                       => $ipv4addr | $fqdn | undef,
#Optional / Default is undefined
                options                          => [$Option1,$Option2,...],           #Optional / Default
is empty list
                override_ddns_ttl                => "true" | "false",                  #Optional / Default
value is "false"
                override_lease_scavenge_time     => "true" | "false",                  #Optional / Default
value is "false"
                override_update_dns_on_lease_renewal  => "true" | "false",             #Optional / Default
value is "false"
                override_ignore_id               => "true" | "false",                  #Optional / Default
value is "false"
                pxe_lease_time                   => $num | undef,                      #Optional / Default
is undefined
                update_dns_on_lease_renewal      => "true" | "false",                  #Optional / Default
value is "false"
                logic_filters                    => [ $filter_name | $filter_obj, ...], # Optional /
Default is undefined
                override_logic_filters           => "true' | 'false",                    # Optional /
Default value is "false"
              );
```

You cannot set both extattrs and extensible_attributes attributes at the same time.

SESSION METHODS

This section describes all the methods in Infoblox::Session module that you can apply to a DHCP shared network object.

Infoblox::Session->add( )

Use this method to add a shared network object to the Infoblox appliance. See Infoblox::Session->add() for parameters and return values.

Example

```
#Construct an object for network
my $network1 = Infoblox::DHCP::Network->new(
        network => "20.0.0.0/8",
);

#Construct an object shared network
  my $sharednetwork = Infoblox::DHCP::SharedNetwork->new(
        name     => "myname",
        networks => [$network1],
);

# Submit for adding sharednetwork
my $response = $session->add( $sharednetwork );
```

Infoblox::Session->get( )

Use this method to retrieve all the matching shared network objects from the Infoblox appliance. See Infoblox::Session->get() for parameters and return values.

Key References

```
Apply the following attributes to get a specific DHCP shared network object:

  name           - Required.    The name of the shared network in string format.
  network        - Optional.    The network with which the shared network associated.
  network_view   - Optional.    The network view in which the shared network is located. By default,
all network views are searched.
  extattrs       - Optional. A hash reference containing extensible attributes.
  extensible_attributes - Optional. A hash reference containing extensible attributes.
```

Example

```
my  @result_array = $session->get(
      object       => "Infoblox::DHCP::SharedNetwork" ,
      name         => "myname",
      network_view => "default"
);

my  @result_array = $session->get(
      object       => "Infoblox::DHCP::SharedNetwork" ,
      network      => "1.0.0.0/24",
      network_view => "default"
);
```

Infoblox::Session->modify( )

Use this method to modify a shared network object in the Infoblox appliance. See Infoblox::Session->modify() for parameters and return values.

Example

```
#Use method to modify enable_ddns method.
$object->enable_ddns("true");
# Submit modification
my $response = $session->modify( $object );
```

Infoblox::Session->remove( )

Use this method to remove a shared network object from the Infoblox appliance. See Infoblox::Session->remove() for parameters and return values.

To remove a specifc object, first use `get()` or `search()` to retrieve the specific object, and then submit this object for removal.

Example

```
# Get the object under the same name
my  @result_array = $session->get(
      object       => "Infoblox::DHCP::SharedNetwork" ,
      name         => "myname",
      network_view => "default"
 );

# find the desired object from retrieved list.
my $desired_network = $result_array[0];

# Submit for removal
my $response = $session->remove( $desired_network );
```

Infoblox::Session->search( )

Use this method to search for DHCP shared network object in the Infoblox appliance. See Infoblox::Session->search() for parameters and return values.

Key References

```
Apply the following attributes to search for a DHCP shared network object:

  name           - Required. The name of the shared network in string format (regular expression).
  network        - Optional. The network with which the shared network associated.
  network_view   - Optional. The network view in which the shared network is located. By default, all
network views are searched.
  extattrs       - Optional. A hash reference containing extensible attributes.
  extensible_attributes - Optional. A hash reference containing extensible attributes.
```

For more information about searching extensible attributes, see Infoblox::Grid::ExtensibleAttributeDef/Searching Extensible Attributes.

Example

```
# search for all DHCP shared network objects in the "default" network view
    my @retrieved_objs = $session->search(
      object    => "Infoblox::DHCP::SharedNetwork",
      name      => "my.*",
      network_view => "default"
    );

# search all Shared Networks Templates with a given extensible attribute 'Site'
    my @retrieved_objs = $session->search(
      object => "Infoblox::DHCP::SharedNetwork",
      extensible_attributes => { 'Site' => 'Santa Clara' }
    );
```

METHODS

This section describes all the methods that you can use to configure and retrieve the attribute values of DHCP a shared network object

authority( )

Use this method to set or retrieve the authority flag of a shared network object. This setting overrides the member-level settings.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

Specify "true" to set the authority flag or "false" to deactivate/unset it. The default value for this field is undefined, which indicates that this attribute is inherited from the member-level setting.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
# Get authority
my $authority = $sharednetwork->authority()
# Modify authority
$sharednetwork->authority("true");
```

bootfile( )

Use this method to set or retrieve a bootfile attribute of a shared network object. You can configure the DHCP server to support clients that use the boot file name option in their DHCPREQUEST messages. This setting overrides the member-level settings. The default value is undefined, which indicates that this attribute is inherited from the member.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

This option is overridden independently from Infoblox::DHCP::SharedNetwork->bootserver() and Infoblox::DHCP::SharedNetwork->nextserver().

Parameter

The name of the file that the client must download. The file name must be in string format and can have a maximum of 128 characters. The default value is undefined.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
# Get bootfile
my $bootfile = $sharednetwork->bootfile();
# Modify bootfile
$sharednetwork->bootfile("bootfile1");
#Un-override bootfile
$sharednetwork->bootfile(undef);
```

---

bootserver( )

Use this method to set or retrieve the bootserver attribute of a shared network object. You can specify the name and/or IP address of the boot server that the host must access in order to boot. This setting overrides the member-level settings. The default value is undefined, which indicates that this attribute is inherited from the member.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

This option is overridden independently from Infoblox::DHCP::SharedNetwork->bootfile() and Infoblox::DHCP::SharedNetwork->nextserver().

Parameter

The boot server IP address or name in FQDN (Fully Qualified Domain Name) format. The FQDN consists of the host name followed by the domain name (example: abc.com). A boot server name can have a maximum of 256 bytes. The default value is undefined.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
# Get bootserver
my $bootserver = $sharednetwork->bootserver();
# Modify bootserver
$sharednetwork->bootserver("abc.domain.com");
#Un-override bootserver
$sharednetwork->bootserver(undef);
```

comment( )

Use this method to set or retrieve a descriptive comment for a shared network object.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

Enter a descriptive comment for the network in string format with a maximum of 256 bytes.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
# Get comment
my $comment = $sharednetwork->comment();
# Modify comment
$sharednetwork->comment("add a shared network");
```

ddns_generate_hostname( )

Use this method to set or retrieve the ddns_generate_hostname flag of a shared network object. Enable this method to allow only the DHCP server to generate host name. This setting overrides the member-level settings.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

Specify "true" to generate the host name or "false" to deactivate/unset it. The default value for this field is undefined, which indicates that this attribute is inherited from the member-level setting.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
# Get ddns_generate_hostname
my $ddns_generate_hostname = $sharednetwork->ddns_generate_hostname()
# Modify ddns_generate_hostname
$sharednetwork->ddns_generate_hostname("true");
#Un-override
```

```
$sharednetwork->ddns_generate_hostname(undef);
```

ddns_server_always_updates( )

Use this method to set or retrieve the ddns_server_always_updates flag of a shared network object. Enable this method to allow only the DHCP server to update DNS,regardless of the requests from the DHCP clients.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

Specify "true" to allow only the DHCP server to update DNS or "false" to deactivate/unset it. The default value for this field is "true".

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
# Get ddns_server_always_updates
my $ddns_server_always_updates = $sharednetwork->ddns_server_always_updates()
# Modify ddns_server_always_updates
$sharednetwork->ddns_server_always_updates("true");
```

ddns_update_fixed_addresses( )

Use this method to set or retrieve the ddns_update_fixed_addresses flag of a shared network object. By default, the DHCP server does not update DNS when it allocates a fixed address to a client. You can configure the DHCP server to update the A and PTR record of clients with fixed addresses. When you enable this feature and the DHCP server adds A and PTR records for a fixed address, the DHCP server never discards the records. This setting overrides the member-level settings.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

Specify "true" to update DNS when the device allocates a fixed address to a client or "false" to deactivate/unset it. The default value for this field is undefined.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
# Get ddns_update_fixed_addresses
my $ddns_update_fixed_addresses= $sharednetwork->ddns_update_fixed_addresses()
# Modify ddns_update_fixed_addresses
$sharednetwork->ddns_update_fixed_addresses("true");
#Un-override
$sharednetwork->ddns_update_fixed_addresses(undef);
```

ddns_ttl( )

Use this method to set or retrieve the DNS update Time to Live (TTL) value of a shared network object.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Setting this method to a defined value implicitly sets the override_ddns_ttl method to "true". Setting the parameter to undefined causes the appliance to use the grid default and automatically resets the override_ddns_ttl attribute to "false".

Note that when ddns_ttl is set to a defined value and override_dddns_ttl is set to "false", the last operation takes precedence. Thus the sequence $object->ddns_ttl(12); $object->override_ddns_ttl("false"); will set override_ddns_ttl to "false", and the sequence $object->override_ddns_ttl("false"); $object->ddns_ttl(12); will result in override_ddns_ttl="true".

Parameter

A 32-bit integer (range from 0 to 4294967295) that represents the duration,in seconds, that the update is cached. Zero indicates that the update should not be cached. The default value is zero.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get DNS Update ttl
my $dns_ttl = $sharednetwork->ddns_ttl();
#Modify DNS Update ttl
$sharednetwork->ddns_ttl(1200);
```

ddns_use_option81( )

Use this method to set or retrieve the ddns_use_option81 flag of a shared network object. Set this flag to define the settings for option 81 at the network level. This setting overrides the member-level settings.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

Specify "true" to configure support for option 81 or "false" to deactivate/unset it. The default value for this field is undefined.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
# Get ddns_use_option81
my $ddns_use_option81 = $sharednetwork->ddns_use_option81()
# Modify ddns_use_option81
$sharednetwork->ddns_use_option81("true");
# un-override ddns_use_option81
$sharednetwork->ddns_use_option81(undef);
```

deny_bootp( )

Use this method to set or retrieve the deny_bootp flag of a DHCP Shared Network object.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

Specify "true" to set the deny_bootp flag or "false" to deactivate/unset it. The default value is undefined, which indicates that this attribute is inherited from the member-level setting.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get deny_bootp
my $deny_bootp = $sharednetwork->deny_bootp();
#Modify deny_bootp
$sharednetwork->deny_bootp("true");
```

disable( )

Use this method to set or retrieve the disable flag of a shared network object.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

Specify "true" to set the disable flag and disable the shared network or "false" to unset the flag and enable the shared network. The default value for this field is "false".

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
# Get disable
```

```
my $disable = $sharednetwork->disable()
# Modify disable
$sharednetwork->disable("true");
```

enable_ddns( )

Use this method to set or retrieve the dynamic DNS updates flag of a DHCP shared network object. The DHCP server can send DDNS updates to DNS servers in the same grid and to external DNS servers. This setting overrides the member-level settings.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

Specify "true" to set the dynamic DNS updates flag or "false" to deactivate/unset it. The default value for this field is undefined, which indicates that this attribute is inherited from the member-level setting.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
# Get enable_ddns
my $enable_ddns = $sharednetwork->enable_ddns()
# Modify enable_ddns
$sharednetwork->enable_ddns("true");
# Un-override enable_ddns
$sharednetwork->enable_ddns(undef);
```

extattrs( )

Use this method to set or retrieve the extensible attributes associated with a Shared Network object.

Parameter

Valid value is a hash reference containing the names of extensible attributes and their associated values (Infoblox::Grid::Extattr objects).

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get extattrs
my $ref_extattrs = $sharednetwork->extattrs();
#Modify extattrs
$sharednetwork->extattrs({ 'Site' => $extattr1, 'Administrator' => $extattr2 });
```

extensible_attributes( )

Use this method to set or retrieve the extensible attributes associated with a Shared Network object.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

For valid values for extensible attributes, see Infoblox::Grid::ExtensibleAttributeDef/Extensible Attribute Values.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get extensible attributes
my $ref_extensible_attributes = $shared_network->extensible_attributes();
#Modify extensible attributes
$shared_network->extensible_attributes({ 'Site' => 'Santa Clara', 'Administrator' => [ 'Peter',
'Tom' ] });
```

ignore_id( )

Use this method to set or retrieve the ignore_id type of a DHCP shared network object. Indicates whether the appliance will ignore DHCP client IDs or MAC addresses. This setting overrides the upper-level settings.

Parameter

Valid values are 'NONE', 'CLIENT', or 'MACADDR'. The default is 'NONE'.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
# Get attribute value
$value = $shared_network->ignore_id();
# Modify attribute value
$shared_network->ignore_id('CLIENT');
#Un-override ignore_id
$shared_network->ignore_id(undef);
```

ignore_mac_addresses( )

Use this method to set or retrieve the ignore_mac_addresses list of a DHCP shared network object. A list of MAC addresses the appliance will ignore. This setting overrides the upper-level settings.

If you enter MAC addresses in the ignore_mac_addresses, then the appliance will ignore them. If you enter empty array, then the appliance will ignore any MAC address.

Parameter

The valid value is an array reference that contains string objects.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
# Get attribute value
$value = $shared_network->ignore_mac_addresses();
# Modify attribute value
$shared_network->ignore_mac_addresses(["11::11::11::11::11::11"]);
```

ignore_dhcp_option_list_request( )

Use this method to set or retrieve the ignore_dhcp_option_list_request flag of a DHCP shared network object. If this flag is set to true all the defined DHCP options will be returned to the client. This setting overrides the member level settings.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

Specify "true" to set the ignore_dhcp_option_list_request flag or "false" to deactivate/unset it. The default value is false which indicates that this attribute inherit the member level setting..

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
# Get ignore_dhcp_option_list_request
my $ignore_dhcp_option_list_request = $sharednetwork->ignore_dhcp_option_list_request();
# Un-override ignore_dhcp_option_list_request
$sharednetwork->ignore_dhcp_option_list_request(undef);
# Modify ignore_dhcp_option_list_request
$sharednetwork->ignore_dhcp_option_list_request("true");
```

lease_scavenge_time( )

Use this method to set or retrieve the lease_scavenge_time value of a Shared Network object.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Setting this method to a defined value implicitly sets the override_lease_scavenge_time method to "true". Setting the parameter to undefined causes the appliance to use the default from the upper level and automatically resets the override_lease_scavenge_time attribute to "false".

Note that when lease_scavenge_time is set to a defined value and override_lease_scavenge_time is set to "false", the last operation takes precedence. Thus the sequence $object->lease_scavenge_time(604800); $object->override_lease_scavenge_time("false"); will set override_lease_scavenge_time to "false", and the sequence $object->override_lease_scavenge_time("false"); $object->lease_scavenge_time(604800); will result in override_lease_scavenge_time="true".

Parameter

An integer that specifies the period of time, in seconds, that free and backup leases remain in the database before they are automatically deleted. To disable lease scavenging, set the parameter to -1. The minimum positive value must be greater than 86400 seconds (1 day). The default value of this parameter is -1, which means that lease scavenging is disabled.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get lease_scavenge_time
my $lease_scavenge = $sharednetwork->lease_scavenge_time();
#Modify lease_scavenge_time
$sharednetwork->lease_scavenge_time(30 * 24 * 60 * 60);
```

ms_ad_user_data( )

Use this method to retrieve Microsoft Active Directory users related information. This is a read-only attribute.

Parameter

None

Returns

The valid return value is an Infoblox::Grid::MSServer::AdUser::Data object.

Example

```
#Get ms_ad_user_data
my $ms_ad_user_data = $sharednetwork->ms_ad_user_data();
```

logic_filters( )

Use this method to set or retrieve all the logic (ordered) filters associated with the DHCP Shared Network.

Filters are used to screen address assignments in a DHCP Shared Network if the addresses are to be assigned to specific hosts.

Setting this method to a defined value implicitly sets the override_logic_filters method to "true". Setting the parameter to undefined causes the appliance to use the Grid default and automatically resets the override_logic_filters attribute to "false".

Note that when logic_filters is set to a defined value and override_dlogic_filters is set to "false", the last operation takes precedence. Thus the sequence $object->logic_filters($filters); $object->override_logic_filters("false"); will set override_logic_filters to "false", and the sequence $object->override_logic_filters("false"); $object->logic_filters($filters); will result in override_logic_filters="true".

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

The valid value is an array reference that contains either the Infoblox::DHCP::Filter::MAC, Infoblox::DHCP::Filter::NAC, Infoblox::DHCP::Filter::Option objects or the names of the corresponding filters. The filters in the list must satisfy the following conditions:

Filter name must be globally unique.
Filters without expressions must be at the end of the list.
The filters should not be disabled.

The default value of the parameter is undefined.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get logic_filters
my $ref_logic_filters = $sharednetwork->logic_filters();
#Modify logic_filters
$sharednetwork->logic_filters(["filter_1","filter_2",$exl1]); #$exl1 is an
Infoblox::DHCP::Filter::MAC, Infoblox::DHCP::Filter::NAC or Infoblox::DHCP::Filter::Option object.
```

name( )

Use this method to retrieve the name of a shared network object.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

The name of the shared network in string format. A name can have a maximum of 32 characters.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get name
my $sharednetwork = $sharednetwork->name();
```

networks( )

Use this method to retrieve the DHCP networks attribute of a shared network object.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

The valid value is an array reference that contains Infoblox::DHCP::Network objects.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
my $network1 = Infoblox::DHCP::Network->new(
                                    "network" => "20.0.0.0/8",
                                );
my $network2 = Infoblox::DHCP::Network->new(
                                    "network" => "30.0.0.0/8",
                                );

#Get networks
my $sharednetwork = $sharednetwork->networks();
#Modify networks
my $sharednetwork = $sharednetwork->networks([$network1, $network2]);
```

network_view( )

Use this method to set or retrieve the network view of the DHCP shared network.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

The default value is the "default" network view, which means the DHCP shared network is in the default network view.

Parameter

Valid value is a defined Infoblox::DHCP::View object. The default network view value is the system-defined default network view object.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get network view
my $nview = $sharednetwork->network_view();
#Modify network view, with an Infoblox::DHCP::View object
$sharednetwork->network_view($nview);
```

nextserver( )

Use this method to set or retrieve the nextserver attribute of a shared network object. You can specify the name and/or IP address of the next server that the host needs to access in order to boot. This setting overrides the member-level settings. The default value is undefined which indicates that this attribute is inherited from the member-level setting.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

This option is overridden independently from Infoblox::DHCP::SharedNetwork->bootfile() and Infoblox::DHCP::SharedNetwork->bootserver().

Parameter

The next server IP address or name in FQDN (Fully Qualified Domain Name) format. The FQDN consists of the host name followed by the domain name (example: abc.com). A next server name can have a maximum of 256 bytes. The default value is undefined.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
# Get nextserver
my $nextserver = $sharednetwork->nextserver();
# Modify nextserver
$sharednetwork->nextserver("blue.domain.com");
#Un-override nextserver
$sharednetwork->nextserver(undef);
```

options( )

Use this method to set or retrieve the options value of a member DHCP server. DHCP options provide network configuration settings and various services available on the network for DHCP. This setting overrides the member-level setting.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

The valid value is an array reference that contains Infoblox::DHCP::Option objects.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get options
my $options = $sharednetwork->options();
#Modify options
my $option1 = Infoblox::DHCP::Option->new(
    name  => "lease-time",
    value => 50
);

$sharednetwork->options([$option1]);
```

override_ddns_ttl( )

The override_ddns_ttl attribute controls whether the ddns_ttl value in the DHCP shared network object is used, instead of the grid default.

The override_ddns_ttl attribute can be specified explicitly. It is also set implicitly when ddns_ttl is set to a defined value.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

Set the parameter to "true" to override the grid-level setting for ddns_ttl. Set the parameter to "false" to inherit the grid-level

setting for ddns_ttl.

The default value of this parameter is "false".

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Getting override_ddns_ttl
my $override_ddns_ttl=$sharednetwork->override_ddns_ttl( );
#Modifying override_ddns_ttl
$sharednetwork->override_ddns_ttl("true");
```

override_ignore_id( )

The override_ignore_id controls whether the ignore_id in the DHCP shared network is used, instead of the upper-level default.

The override_ignore_id can be specified explicitly. It is also set implicitly when ignore_id is set to a defined value.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

Set the parameter to "true" to override the upper-level setting for ignore_id and ignore_mac_addresses. Set the parameter to "false" to inherit the upper-level setting for ignore_id and ignore_mac_addresses.

The default value of this parameter is "false".

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get attrubute value
my $value=$sharednetwork->override_ignore_id();
#Modifying attribute value
$sharednetwork->override_ignore_id('true');
```

override_lease_scavenge_time( )

The override_lease_scavenge_time attribute controls whether the lease_scavenge_time value in the shared network is used, instead of the upper-level default.

The override_lease_scavenge_time attribute can be specified explicitly. It is also set implicitly when lease_scavenge_time is set to a defined value.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

Set the parameter to "true" to override the upper-level setting for lease_scavenge_time. Set the parameter to "false" to inherit the setting for lease_scavenge_time from the upper level.

The default value of this parameter is "false".

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Getting override_lease_scavenge_time
my $override_lease_scavenge_time=$sharednetwork->override_lease_scavenge_time( );
#Modifying override_lease_scavenge_time
$sharednetwork->override_lease_scavenge_time("true");
```

override_logic_filters( )

The override_logic_filters attribute controls whether the logic_filters value of the DHCP Shared Network is used, instead of the Grid default.

The override_logic_filters attribute can be specified explicitly. It is also set implicitly when logic_filters is set to a defined value.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

Set the parameter to "true" to override the Grid-level setting for logic_filters. Set the parameter to "false" to inherit the Grid-level setting for logic_filters.

The default value of this parameter is "false".

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Getting override_logic_filters
my $override_logic_filters=$sharednetwork->override_logic_filters( );
#Modifying override_logic_filters
$sharednetwork->override_logic_filters("true");
```

override_update_dns_on_lease_renewal( )

The override_update_dns_on_lease_renewal attribute controls whether the update_dns_on_lease_renewal value in the DHCP shared network object is used, instead of the grid default.

The override_update_dns_on_lease_renewal attribute can be specified explicitly. It is also set implicitly when update_dns_on_lease_renewal is set to a defined value.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

Set the parameter to "true" to override the grid-level setting for update_dns_on_lease_renewal. Set the parameter to "false" to inherit the grid-level setting for update_dns_on_lease_renewal.

The default value of this parameter is "false".

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Getting override_update_dns_on_lease_renewal
my $override_update_dns_on_lease_renewal=$sharednetwork->override_update_dns_on_lease_renewal( );
#Modifying override_update_dns_on_lease_renewal
$sharednetwork->override_update_dns_on_lease_renewal("true");
```

pxe_lease_time( )

Use this method to set or retrieve the PXE lease time value of a shared network object. Some hosts use PXE (Preboot Execution Environment) to boot remotely from a server. To better manage your IP resources, set a different lease time for PXE boot requests. You can configure the DHCP server to allocate an IP address with a shorter lease time to hosts that send PXE boot requests, so IP addresses are not leased longer than necessary. This setting overrides the member-level settings. The default value is undefined, which indicates that this attribute is inherited from the member-level setting.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

A 32-bit integer (range from 0 to 4294967295) that represents the duration, in seconds, that the update is cached. Zero indicates that the update is not cached. The default value is undefined

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get PXE Lease Time
```

```
my $pxe_lease_time = $sharednetwork->pxe_lease_time();
#Modify PXE Lease Time
$sharednetwork->pxe_lease_time(1100);
#Un-override PXE Lease Time
$sharednetwork->pxe_lease_time(undef);
```

update_dns_on_lease_renewal( )

The update_dns_on_lease_renewal attribute controls whether the DHCP server updates DNS when a DHCP lease is renewed.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Setting this method to a defined value implicitly sets the override_update_dns_on_lease_renewal method to "true". Setting the parameter to undefined causes the appliance to use the grid default and automatically resets the override_update_dns_on_lease_renewal attribute to "false".

Note that when update_dns_on_lease_renewal is set to a defined value and override_update_dns_on_lease_renewal is set to "false", the last operation takes precedence. Thus the sequence $object->update_dns_on_lease_renewal("true"); $object->override_update_dns_on_lease_renewal("false"); will set override_update_dns_on_lease_renewal to "false", and the sequence $object->override_update_dns_on_lease_renewal("false"); $object->update_dns_on_lease_renewal("true"); will result in override_update_dns_on_lease_renewal="true".

Parameter

Specify "true" to set the update_dns_on_lease_renewal flag or "false" to deactivate/unset it.

The default value of this parameter is "false".

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get update_dns_on_lease_renewal
my $update_dns_on_lease_renewal=$sharednetwork->update_dns_on_lease_renewal( );
#Modify update_dns_on_lease_renewal
$sharednetwork->update_dns_on_lease_renewal("true");
```

SAMPLE CODE

The following sample code demonstrates the different functions that can be applied to an object, such as add, search, modify, and remove. This sample also includes error handling for the operations.

#Preparation prior to a DHCP shared network object insertion

```
#PROGRAM STARTS: Include all the modules that will be used
use strict;
use Infoblox;

#Create a session to the Infoblox appliance

my $session = Infoblox::Session->new(
    master   => "192.168.1.2",
    username => "admin",
    password => "infoblox"
);
unless ($session) {
   die("Construct session failed: ",
       Infoblox::status_code() . ":" . Infoblox::status_detail());
}
print "Session created successfully\n";
```

#Create a DHCP network object

```
my $network1 = Infoblox::DHCP::Network->new(
                                    "network" => "10.0.0.0/8",
                                );
my $network2 = Infoblox::DHCP::Network->new(
                                    "network" => "11.0.0.0/8",
                                );

my $response = $session->add($network1)
or die("Add \"10.0.0.0/8\"Network failed: ",
             $session->status_code() . ":" . $session->status_detail());
```

```
print "Network \"10.0.0.0/8\"added successfully\n";

my $response = $session->add($network2)
       or die("Add  \"11.0.0.0\"Network failed: ",
             $session->status_code() . ":" . $session->status_detail());

print "Network  \"11.0.0.0/8\",added successfully\n";
```

#Create a DHCP shared network object

```
my $sharednetwork1 = Infoblox::DHCP::SharedNetwork->new(
                                        "name"                          => "myname",
                                        "comment"                       => "add shared network",
                                        "networks"                      => [ $network1 , $network2 ],
                                        "disable"                       => "false",
                                        "authority"                     => "true",
                                        "enable_ddns"                   => "true",
                                        "ddns_use_option81"             => "true",
                                        "ddns_generate_hostname"        => "true",
                                        "ddns_server_always_updates"    => "true",
                                        "ddns_update_fixed_addresses"   => "true",
                                        "pxe_lease_time"                => "3600",
                                        "bootserver"                    => "someserver.com",
                                        "nextserver"                    => "3.3.3.3",
                                        "bootfile"                      => "bootfile.com",

                                        );

my $response = $session->add( $sharednetwork1 )
or die("Add Shared Network failed: ",
             $session->status_code() . ":" . $session->status_detail());

print "Shared Network added successfully\n";
```

#Search for a specific shared network object

```
#Search the network
my @retrieved_objs = $session->search(
    object  => "Infoblox::DHCP::SharedNetwork",
    "name"  => "my.*"
);
my $object = $retrieved_objs[0];
unless ($object) {
    die("Search Shared Network failed: ",
        $session->status_code() . ":" . $session->status_detail());
}
print "Search Shared Network found at least 1 matching entry\n";
```

#Get and modify a shared network attribute

```
#GetShared Network through the session
my  @retrieved_objs = $session->get(
           object =>"Infoblox::DHCP::SharedNetwork" ,
           "name" => "myname"
        );

my $object = $retrieved_objs[0];

unless ($object) {
    die("Get Shared Network object failed: ",
        $session->status_code() . ":" . $session->status_detail());
}
print "Get Shared Network found at least 1 matching entry\n";

#Modify attributes of the specified Network
$object->disable("true"); #Apply the changes
my $res = $session->modify($object) ;
if ( !$res)
{
    print "Modify Shared Network attribute failed: ".$session->status_code() . ":" . $session-
>status_detail();
}
else {
print "Shared Network object modified successfully \n";
}

#Un-override bootfile
$object->bootfile(undef);
```

```
#Un-override bootserver
$object->bootserver(undef);

#Un-override nextserver
$object->nextserver(undef);
```

#Remove a shared network object

```
 #Get Network object through the session
 my @retrieved_objs = $session->get(
           object  => "Infoblox::DHCP::SharedNetwork" ,
           name    => "myname"
    );

   my $object = $retrieved_objs[0];

   unless ($object) {
    die("Get Shared Network object failed: ",
        $session->status_code() . ":" . $session->status_detail());
    }

print "Get Shared Network object found at least 1 matching entry\n";

#Submit the shared network object for removal

$session->remove($object)
    or die("Remove Shared Network failed: ",
        $session->status_code() . ":" . $session->status_detail());

print "Shared Network removed successfully \n";

#Submit the network object for removal
my $response = $session->remove($network1)
or die("Remove \"10.0.0.0/8\"Network failed: ",
             $session->status_code() . ":" . $session->status_detail());

print "Network \"10.0.0.0/8\"remove successfully\n";

#Submit the network object for removal
my $response = $session->remove($network2)
       or die("Remove  \"11.0.0.0\"Network failed: ",
             $session->status_code() . ":" . $session->status_detail());

print "Network \"11.0.0.0/8\"remove successfully\

####PROGRAM ENDS####
```

AUTHOR

Infoblox Inc. http://www.infoblox.com/

SEE ALSO

Infoblox::Session, Infoblox::DHCP::Network, Infoblox::DHCP::View, Infoblox::DHCP::Option, Infoblox::Session->get(), Infoblox::Session->search(), Infoblox::Session->add(), Infoblox::Session->remove(), Infoblox::Session->modify(), Infoblox::Grid::MSServer::AdUser::Data

---

COPYRIGHT

Copyright (c) 2017 Infoblox Inc.

Infoblox::DHCP::SplitNetwork - Split network object.

- NAME
- DESCRIPTION
- CONSTRUCTOR
- SESSION METHODS
  - Infoblox::Session->add( )
- METHODS
  - add_all_subnetworks( )
  - auto_create_reversezone( )
  - network( )
  - prefix( )
  - prefix_collector_ipv6_network_addr( )
- SAMPLE CODE
- AUTHOR
- SEE ALSO
- COPYRIGHT

NAME

Infoblox::DHCP::SplitNetwork - Split network object.

DESCRIPTION

Once a network is created for DHCP, smaller subnetworks can be configured with larger netmasks. A larger netmask defines a larger number of network addresses and a smaller number of host addresses. Using Infoblox split network feature, the parent network can be split into multiple smaller networks without configuring each of the newly-created networks.

CONSTRUCTOR

```
my $split_network = Infoblox::DHCP::SplitNetwork->new(
            network                 => $Network|$IPv6Network,    #Required
            prefix                  => $num,                    #Required
            add_all_subnetworks     => "true" | "false",        #Optional / Default is "false"
            auto_create_reversezone => "true" | "false",        #Optional / Default is "false"
            prefix_collector_ipv6_network_addr => $string,      #Optional / Default is undefined
);
```

SESSION METHODS

This section describes all the methods in the Infoblox::Session module that can be applied to a split network object.

Infoblox::Session->add( )

Use this method to add an object to the Infoblox appliance. See Infoblox::Session->add() for parameters and return values.

Example

```
    #Construct an object
    my $network = Infoblox::DHCP::Network->new(
                              network => "10.0.0.0/8",
                              comment => "add network",
                              );

    my $split_network = Infoblox::DHCP::SplitNetwork->new (
                        network                 => $network,
                        prefix                  => 12,
                        add_all_subnetworks     => "true",
                        );
    #Submit for addition
    my $response = $session->add( $split_network );
```

METHODS

This section describes all the methods that can be used to configure and retrieve the attribute value of a split network object.

add_all_subnetworks( )

Use this method to set the add_all_subnetworks flag. If this flag is set, then all possible subnetworks will be added otherwise only networks with fixed addresses will be added.

Include the specific parameter to set the attribute value.

Parameter

Specify "true" to set add_all_subnetworks flag or "false" to deactivate/unset it. The default value is "false".

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

Example

```
#Set add_all_subnetworks
$split_network->add_all_subnetworks("true");
```

auto_create_reversezone( )

Use this method to set the auto_create_reversezone flag. When set, this flag will automatically create reverse-mapping zones for the subnets.

Include the specified parameter to set the attribute value.

Parameter

Specify "true" to set auto_create_reversezone flag or "false" to deactivate/unset it. The default value is "false".

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

Example

```
#Set auto_create_reversezone
$split_network->auto_create_reversezone("true");
```

network( )

Use this method to specify the network to be split.

Include the specified parameter to set the attribute value.

Parameter

The valid values are Infoblox::DHCP::Network or Infoblox::DHCP::IPv6Network objects.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

Example

```
#Specify network
my $network = Infoblox::DHCP::Network->new(
                    network => "10.0.0.0/8",
                    comment => "add network",
                    );

$split_network->network($network);

#Specify ipv6 network
my $network = Infoblox::DHCP::IPv6Network->new(
                    network => "10::/48",
                    comment => "add network",
                    );

$split_network->network($network);
```

prefix( )

Use this method to specify the appropriate subnet mask for each subnet created after splitting the network.

Include the specified parameter to set the attribute value.

Parameter

Number with the subnet mask value. This value must be 1 greater than the subnet mask of the network to be split. The maximum value is 31 for IPv4 network or 127 for IPv6 network.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

Example

```
#Specify prefix
$split_network->prefix(16);
```

prefix_collector_ipv6_network_addr( )

Use this method to specify the network that contains the DHCP IPv6FixedAddress and IPv6Range objects whose address_type is 'PREFIX' after a network is split.

Include the specified parameter to set the attribute value.

Parameter

The valid value is an IPv6 network address in string format and without any prefix bits appended.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

Example

```
#Set prefix_collector_ipv6_network_addr( )
$split_network->prefix_collector_ipv6_network_addr("2001::");
```

SAMPLE CODE

The following sample code demonstrates the split network operation. Also this sample code includes error handling for the operations.

#Preparation prior to splitting a network

```
use strict;
use Infoblox;

#Create a session to the Infoblox appliance
my $session = Infoblox::Session->new(
    master   => "192.168.1.2",
    username => "admin",
    password => "infoblox"
);
unless ($session) {
   die("Construct session failed: ",
       Infoblox::status_code() . ":" . Infoblox::status_detail());
}
print "Session created successfully.\n";

#Create network prior to splitting the network
my $network = Infoblox::DHCP::Network->new(
                            network => "20.0.0.0/8",
                            comment => "add network",
                            );
unless($network){
       die("Construct network object fail: ",
               Infoblox::status_code(). ":" .Infoblox::status_detail());
       }

print"Network object created successfully.\n";

#Verify if the network exists
my $object = $session->get(
               object  => "Infoblox::DHCP::Network",
               network => "20.0.0.0/8"
               );
unless($object){
       print"Network does not exist on server, safe to add the network.\n";
       $session->add($network)
               or die("Add network failed: ",
                       $session->status_code(). ":" .$session->status_detail());
       }
```

```
print"Network added successfully.\n";
```

#Split the network

```
my $split_network = Infoblox::DHCP::SplitNetwork->new (
                        network                 => $network,
                        prefix                  => 16,
                        add_all_subnetworks     => "true",
                        auto_create_reversezone => "true",
                      );
unless($split_network){
       die("Construct split network object fail: ",
                Infoblox::status_code(). ":" .Infoblox::status_detail());
       }

print"Split network object created successfully.\n";

#Split the network
$session->add( $split_network )
         or die("Split network fail:",
                    $session->status_code(). ":" .$session->status_detail());

print"Splited network successfully.\n";

####PROGRAM ENDS####
```

AUTHOR

Infoblox Inc. http://www.infoblox.com/

SEE ALSO

Infoblox::Session,Infoblox::Session->add(),Infoblox::DHCP::Network

---

COPYRIGHT

Copyright (c) 2017 Infoblox Inc.

Infoblox::DHCP::Statistics - Manages DHCP statistics.

- NAME
- DESCRIPTION
- CONSTRUCTOR
- SESSION METHODS
  - Infoblox::Session->get( )
- METHODS
  - available_hosts( )
  - dynamic_hosts( )
  - dhcp_utilization_status( )
  - static_hosts( )
  - usage( )
- SAMPLE CODE
- AUTHOR
- SEE ALSO
- COPYRIGHT

NAME

Infoblox::DHCP::Statistics - Manages DHCP statistics.

DESCRIPTION

The DHCP statistics object is used to view the DHCP statistics of the network in an Infoblox appliance.

CONSTRUCTOR

The Infoblox::DHCP::Statistics object is automatically generated upon the successful creation of the Infoblox::DHCP::Network object. It does not require manual construction.

SESSION METHODS

This section describes all the methods in an Infoblox::Session that can be applied to a DHCP statistics object.

Infoblox::Session->get( )

Use this method to retrieve the existing objects from an Infoblox appliance. See Infoblox::Session->get() for parameters and return values.

Key Reference

```
 Apply the following attributes to get the DHCP statistics:

 statistics_object - Required. An object returned via Infoblox::Session->get() or Infoblox::Session-
>search(). Supported objects are Infoblox::DHCP::Network, Infoblox::DHCP::Range,
Infoblox::DHCP::SharedNetwork, and Infoblox::DHCP::MSSuperscope.
```

Example

```
# API can return more than 1 answer for DHCP::Statistics
# for each member of the network
my @network_stats = $session->get(
        object            => "Infoblox::DHCP::Statistics",
        statistics_object => $network);
```

METHODS

This section describes all the methods that can be used to retrieve the attribute values of a DHCP statistics object.

available_hosts( )

Use this method to retrieve the number of available IP addresses in the given network.

Parameter

none

Returns

The method returns the attribute value.

Example

```
#Get available_hosts
my $available_hosts = $network_stats->available_hosts();
```

dynamic_hosts( )

Use this method to retrieve number of hosts which are assigned dynamic IP address.

Parameter

none

Returns

The method returns the attribute value.

Example

```
#Get dynamic_hosts
my $dynamic_hosts = $network_stats->dynamic_hosts();
```

dhcp_utilization_status( )

Use this method to retrieve the DHCP utilization status.

Parameter

none

Returns

The method returns the attribute value. The returned value is one of "FULL", "HIGH", "LOW" or "NORMAL".

Example

```
#Get member
my $status = $network_stats->dhcp_utilization_status();
```

static_hosts( )

Use this method to retrieve the number of hosts with fixed addresses.

Parameter

none

Returns

The method returns the attribute value.

Example

```
#Get static_hosts
my $static_hosts = $network_stats->static_hosts();
```

usage( )

Use this method to retrieve the percentage used of DHCP addresses in the network.

Parameter

none

Returns

The method returns the attribute value.

Example

```
#Get usage
my $usage = $network_stats->usage();
```

SAMPLE CODE

The following sample code demonstrates the get session method on a DHCP statistics object. Also, this sample code also includes error handling for the operations.

```
#PROGRAM STARTS: Include all the modules that will be used
use strict;
use Infoblox;
```

#Preparation prior to getting DHCP statistics

```
#Create a session to the Infoblox appliance
my $session = Infoblox::Session->new(
    master   => "192.168.1.2",
    username => "admin",
    password => "infoblox"
);
unless ($session) {
   die("Construct session failed: ",
       Infoblox::status_code() . ":" . Infoblox::status_detail());
}
print "Session created successfully\n";

my $member = Infoblox::DHCP::Member->new(
    name     => "infoblox.localdomain",
    ipv4addr => "192.168.1.2",
);

unless($member) {
     die("Construct member failed: ",
           Infoblox::status_code() . ":" . Infoblox::status_detail());
}
print "Member object created successfully\n";

#Create a Network View object
my $network_view = Infoblox::DHCP::View->new(
                    name => "my_network_view",
);
unless($network_view) {
             die("Construct Network View object failed: ",
                   Infoblox::status_code() . ":" . Infoblox::status_detail());
}
print "Network View object created successfully\n";

#Add the Network View object into the Infoblox appliance through a session
$session->add($network_view)
     or die("Add Network View object failed: ",
           $session->status_code() . ":" . $session->status_detail());
print "Network View added successfully\n";

#Create the Network object with this member
my $network = Infoblox::DHCP::Network->new(
                    network => "1.0.0.0/255.0.0.0",
                    members => [ $member ],
                    network_view => $network_view,
);
unless($network) {
             die("Construct Network object failed: ",
                   Infoblox::status_code() . ":" . Infoblox::status_detail());
}
print "Network object created successfully\n";

#Add the Network object into the Infoblox appliance through a session
$session->add($network)
     or die("Add Network object failed: ",
           $session->status_code() . ":" . $session->status_detail());
print "Network added successfully\n";

#Create the DHCP Range object with this member
my $dhcp_range = Infoblox::DHCP::Range->new(
    network    => "1.0.0.0/8",
    start_addr => "1.0.0.1",
    end_addr   => "1.0.0.10",
    member     => $member,
    network_view => $network_view,
);
unless($dhcp_range) {
     die("Construct DHCP Range failed: ",
           Infoblox::status_code() . ":" . Infoblox::status_detail());
```

```
}
print "DHCP range object created successfully\n";

$session->add($dhcp_range)
     or die("Add DHCP range object failed: ",
            $session->status_code() . ":" . $session->status_detail());
print "DHCP range added successfully\n";

#Create the Fixed Address object
my $fixed_address = Infoblox::DHCP::FixedAddr->new(
                                                    "network" => "1.0.0.0/8",
                                                    "ipv4addr" => "1.0.0.2",
                                                    "mac" => "22:22:22:22:22:22",
                                                    "network_view" => $network_view,
                                                    );
unless($fixed_address) {
     die("Construct DHCP Fixed address failed: ",
            Infoblox::status_code() . ":" . Infoblox::status_detail());
}
print "DHCP Fixed address object created successfully\n";

$session->add($fixed_address)
     or die("Add DHCP Fixed address object failed: ",
            $session->status_code() . ":" . $session->status_detail());
print "DHCP Fixed address added successfully\n";
```

#Get the DHCP statistics

```
 my  @result_array = $session->get(
     object  => "Infoblox::DHCP::Network",
     network => "1.0.0.0/8",
     network_view => "my_network_view");

 my $network = $result_array[0];
 unless ($network) {
     die("Get Network failed: ",
         $session->status_code() . ":" . $session->status_detail());
 }

 @result_array = $session->get(
                               object  => "Infoblox::DHCP::Statistics",
                               statistics_object  => $network,
                              );

 my $network_stats = $result_array[0];
 unless ($network_stats) {
     die("Get Network statistics failed: ",
         $session->status_code() . ":" . $session->status_detail());
 }

 print "Get Network Statistics successful\n";
```

#Clean up # Remove the Network Statistics object that was just retrieved $session->remove($network_stats);

```
 # Retrieve and remove the Network View object
 my $network_view = $session->get(
         object => "Infoblox::DHCP::View",
         name => "my_network_view",
 );
 unless ($network_view) {
      die("Get Network View failed: ",
           $session->status_code() . ":" . $session->status_detail());
 }
 $session->remove($network_view);

####PROGRAM ENDS####
```

AUTHOR

Infoblox Inc. http://www.infoblox.com/

SEE ALSO

Infoblox::Session, Infoblox::DHCP::Network,Infoblox::DHCP::View,Infoblox::DHCP::FixedAddr,Infoblox::DHCP::Range,Infoblox::Session-

>add(),Infoblox::Session->get()

COPYRIGHT

Copyright (c) 2017 Infoblox Inc.

Infoblox::DHCP::Template - DHCP Template object.

- NAME
- DESCRIPTION
- CONSTRUCTOR
- MODULE METHODS
  - Infoblox::DHCP::NetworkTemplate->range_templates( )
  - Infoblox::DHCP::NetworkTemplate->fixed_address_templates( )
- METHODS
  - name( )
  - count( )
  - offset( )
- SAMPLE CODE
- AUTHOR
- SEE ALSO
- COPYRIGHT

NAME

Infoblox::DHCP::Template - DHCP Template object.

DESCRIPTION

The DHCP associates Infoblox::DHCP::RangeTemplate->offset(), Infoblox::DHCP::RangeTemplate->number_of_addresses() and Infoblox::DHCP::FixedAddrTemplate->offset(), Infoblox::DHCP::FixedAddrTemplate->number_of_addresses() with Infoblox::DHCP::NetworkTemplate. When optional fields offset/count are set in this object which is passed to Infoblox::DHCP::NetworkTemplate->fixed_address_templates() or Infoblox::DHCP::NetworkTemplate->range_templates() they are used instead of offset/number_of_addresses values in Infoblox::DHCP::RangeTemplate and Infoblox::DHCP::FixedAddrTemplate when creating Infoblox::DHCP::Network based on a template.

CONSTRUCTOR

```
my $template = Infoblox::DHCP::Template->new(
      name                  => $string,       #Required
      count                 => $num,          #Optional
      offset                => $num,          #Optional
);
```

MODULE METHODS

The following functions are available to be applied to a DHCP template object.

Infoblox::DHCP::NetworkTemplate->range_templates( )

Use this function to specify range templates for the DHCP network template object. See Infoblox::DHCP::NetworkTemplate->range_templates() for parameters and return values.

Example

```
        #Construct and add a DHCP Range Template object
        my $range_template = Infoblox::DHCP::RangeTemplate->new(
                name                  => "custom_range_template",
                offset                => "20", # this won't be used if specified in
    Infoblox::DHCP::Template
                number_of_addresses   => "10", # this won't be used if specified in
    Infoblox::DHCP::Template
        );
        $session->add($range_template);

        #Construct a DHCP Template object
        my $template = Infoblox::DHCP::Template->new(
                offset                => "10", # this will be used instead if specified
                count                 => "5", # this will be used instead if specified
                name                  => "custom_range_template"
        );

        # Configure DHCP Template on the DHCP Network template object
        $network_template->range_templates([$template]);
```

Infoblox::DHCP::NetworkTemplate->fixed_address_templates( )

Use this function to specify fixed address templates for the DHCP network template object. See Infoblox::DHCP::NetworkTemplate->fixed_address_templates() for parameters and return values.

Example

```
#Construct and add a DHCP Fixed Address Template object
my $fixed_address_template = Infoblox::DHCP::FixedAddrTemplate->new(
        name                => "custom_fixed_address_template",
        offset              => "20", # this won't be used if specified in
Infoblox::DHCP::Template
        number_of_addresses  => "10", # this won't be used if specified in
Infoblox::DHCP::Template
);
$session->add($fixed_address_template);

#Construct a DHCP Template object
my $template = Infoblox::DHCP::Template->new(
        offset              => "10", # this will be used instead if specified
        count               => "5", # this will be used instead if specified
        name                => "custom_fixed_address_template"
);

# Configure DHCP Template on the DHCP Network template object
$network_template->fixed_address_templates([$template]);
```

METHODS

This section describes all the methods that can be used to configure and retrieve the attribute values of a DHCP range template object.

name( )

Use this method to set or retrieve name of Infoblox::DHCP::RangeTemplate or Infoblox::DHCP::FixedAddrTemplate object assigned to this template object.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

The name of range template or fixed address template object.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get name
my $name = $template->name();
#Modify name
$template->name("custom_range_or_fixed_address_template_name");
```

count( )

Use this method to set or retrieve the number of addresses of the DHCP range or fixed address template. This value will be used instead of Infoblox::DHCP::RangeTemplate->number_of_addresses() or Infoblox::DHCP::FixedAddrTemplate->number_of_addresses() if specified.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

The number of addresses in this template.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get count
my $count = $template->count();
#Modify count
$template->count("10");
```

offset( )

Use this method to set or retrieve the address offset of the DHCP range or fixed address template. This value will be used instead of Infoblox::DHCP::RangeTemplate->offset() or Infoblox::DHCP::FixedAddrTemplate->offset() if specified.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

The address offset of this template.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get offset
my $offset = $template->offset();
#Modify offset
$template->offset("10");
```

SAMPLE CODE

The following sample code demonstrates the different functions that can be applied to a DHCP template object such as add, modify, and remove. Also, this sample includes error handling for the operations.

#Preparation prior to a DHCP Range object insertion

```
#PROGRAM STARTS: Include all the modules that will be used
use strict;
use Infoblox;
#Create a session to the Infoblox appliance
my $session = Infoblox::Session->new(
   master   => "192.168.1.2",
   username => "admin",
   password => "infoblox"
);

unless ($session) {
       die("Construct session failed: ",
            Infoblox::status_code() . ":" . Infoblox::status_detail());
}
print "Session created successfully\n";
```

#Create the DHCP Range template object

```
my $dhcp_range_template = Infoblox::DHCP::RangeTemplate->new(
        name                    => "custom_range_template",
        offset                  => "10",
        number_of_addresses     => "10",
);
unless($dhcp_range_template) {
       die("Construct DHCP Range template object failed: ",
            Infoblox::status_code() . ":" . Infoblox::status_detail());
}
print "DHCP Range template object created successfully\n";

#Add the DHCP Range template object into the Infoblox appliance through a session
$session->add($dhcp_range_template)
    or die("Add DHCP Range template object failed: ",
            $session->status_code() . ":" . $session->status_detail());
print "DHCP Range template object added to Infoblox appliance successfully\n";
```

#Create the Fixed Address template object

```
my $dhcp_fixed_address_template = Infoblox::DHCP::FixedAddrTemplate->new(
        name                    => "custom_fixed_address_template",
        offset                  => "10",
        number_of_addresses     => "10",
);
unless($dhcp_fixed_address_template) {
       die("Construct DHCP Fixed Address template object failed: ",
            Infoblox::status_code() . ":" . Infoblox::status_detail());
```

```
}
print "DHCP Fixed Address template object created successfully\n";

#Add the DHCP Fixed Address Template  object into the Infoblox appliance through a session
$session->add($dhcp_fixed_address_template)
    or die("Add DHCP Fixed Address template object failed: ",
           $session->status_code() . ":" . $session->status_detail());
print "DHCP Fixed Address template object added to Infoblox appliance successfully\n";
```

#Create DHCP template objects

```
#Construct a DHCP Template object
my $tmpl1 = Infoblox::DHCP::Template->new(
         offset               => "10", # this will be used instead if specified
         count                => "5", # this will be used instead if specified
         name                 => "custom_range_template"
);

unless($tmpl1) {
       die("Construct DHCP template object failed: ",
            Infoblox::status_code() . ":" . Infoblox::status_detail());
}
print "DHCP template object created successfully\n";

#Construct a DHCP Template objects
my $tmpl2 = Infoblox::DHCP::Template->new(
         offset               => "20", # this will be used instead if specified
         count                => "3", # this will be used instead if specified
         name                 => "custom_fixed_address_template"
);

unless($tmpl2) {
       die("Construct DHCP template object failed: ",
            Infoblox::status_code() . ":" . Infoblox::status_detail());
}
print "DHCP template object created successfully\n";
```

#Add DHCP template to DHCP Network Template and insert it

```
my $network_template = Infoblox::DHCP::NetworkTemplate->new(
        name                   => "network_template",
        netmask                => "255.0.0.0",
        range_templates        => [ $tmpl1 ],
        fixed_address_templates => [ $tmpl2 ],
);
unless($network_template) {
       die("Construct DHCP Network template object failed: ",
            Infoblox::status_code() . ":" . Infoblox::status_detail());
}
print "DHCP Network template object created successfully\n";

#Add the DHCP Network template object into the Infoblox appliance through a session
$session->add($network_template)
    or die("Add Network template object failed: ",
           $session->status_code() . ":" . $session->status_detail());
print "DHCP Network template object added to Infoblox appliance successfully\n";

####PROGRAM ENDS####
```

AUTHOR

Infoblox Inc. http://www.infoblox.com/

SEE ALSO

Infoblox::Session, Infoblox::DHCP::RangeTemplate, Infoblox::DHCP::FixedAddrTemplate, Infoblox::DHCP::NetworkTemplate, Infoblox::DHCP::Network

COPYRIGHT

Copyright (c) 2017 Infoblox Inc.

Infoblox::DHCP::View - DHCP Network View object

- NAME
- DESCRIPTION
- CONSTRUCTOR
- SESSION METHODS
  - Infoblox::Session->add( )
  - Infoblox::Session->get( )
  - Infoblox::Session->modify( )
  - Infoblox::Session->remove( )
  - Infoblox::Session->search( )
- METHODS
  - associated_dns_views( )
  - cloud_info( )
  - comment( )
  - ddns_zone_primaries( )
  - extensible_attributes( )
  - external_ddns_primaries( )
  - mgm_private( )
  - modify_vrf_assignment()
  - internal_ddns_zones( )
  - internal_updates_view( )
  - ms_ad_user_data( )
  - name( )
- SAMPLE CODE
- AUTHOR
- SEE ALSO
- COPYRIGHT

NAME

Infoblox::DHCP::View - DHCP Network View object

DESCRIPTION

Network view provides the ability to manage multiple views of IP address spaces from NIOS. These address spaces could be overlapping or non-overlapping. When the network view is not specified during a write operation, the objects are automatically associated with the system-defined default network view.

A network view is a single routing domain, with its own networks and shared networks. You can manage the networks in one network view independently of the other network views.

CONSTRUCTOR

```
  my $network = Infoblox::DHCP::View->new(
     name                        => $string,
#Required
     comment                     => $string,
#Optional / Default is undefined
     ddns_zone_primaries         => [$ddns1, $ddns2, ... ],
#Optional / Default is undefined
     mgm_private                 => 'true' | 'false',
#Optional / Default is 'false'
     extensible_attributes       => { $string => $string|$num, $string => [ $string|$num, ...], ... },
#Optional / Default is undefined
     external_ddns_primaries     => [ $Nameserver1,$Nameserver2,...],
#Optional / Default is undefined
     internal_ddns_primaries     => [ $ForwardZone1,$ForwardZone2,...],
#Optional / Default is undefined
   );
```

SESSION METHODS

This section describes all the methods in an Infoblox::Session module that can apply to a DHCP Network View object.

Infoblox::Session->add( )

Use this method to add a network view object to the Infoblox appliance. See Infoblox::Session->add() for parameters and return values.

Example

```
#Construct an object
my $network_view = Infoblox::DHCP::View->new(
       name => "netview",
);

# Submit for adding network view
my $response = $session->add( $network_view );
```

Infoblox::Session->get( )

Use this method to retrieve all the matching network view objects from the Infoblox appliance. See Infoblox::Session->get() for parameters and return values.

Key References

```
Apply the following attributes to get a specific DHCP Network View object:

 name                  - Required. A name in string format.
 extensible_attributes - Optional. A hash reference containing extensible attributes.
```

Example

```
 my  @result_array = $session->get(
      object        => "Infoblox::DHCP::View" ,
      name          => "netview"
 );

# get all DHCP view objects with a given extensible attribute
my @retrieved_objs = $session->get(
   object => "Infoblox::DHCP::View",
   extensible_attributes => { 'Site' => 'Santa Clara' });
```

Infoblox::Session->modify( )

Use this method to modify a network object in the Infoblox appliance. See Infoblox::Session->modify() for parameters and return values.

Example

```
#Use method to modify comment method.
$object->comment("My modified comment");
# Submit modification
my $response = $session->modify( $object );
```

Infoblox::Session->remove( )

Use this method to remove an object from the Infoblox appliance. See Infoblox::Session->remove() for parameters and return values.

To remove a specifc object, first use `get()` or `search()` to retrieve the specific object, and then submit this object for removal.

Example

```
# Get the object under the same name
my  @result_array = $session->get(
      object        => "Infoblox::DHCP::View" ,
      name          => "netview"
 );

# find the desired object from the retrieved list.
my $desired_network_view = $result_array[0];

# Submit for removal
my $response = $session->remove( $desired_network_view );
```

Infoblox::Session->search( )

Use this method to search for a DHCP Network View object in the Infoblox appliance. See Infoblox::Session->search() for parameters and return values.

Key References

```
Apply the following attributes to search for a DHCP Network View object:

 name                  - Required. A name in string format (regular expression).
 extensible_attributes - Optional. A hash reference containing extensible attributes.
```

For more information about searching extensible attributes, see Infoblox::Grid::ExtensibleAttributeDef/Searching Extensible Attributes.

Example

```
# search for DHCP Network View object with name 'netview'
    my @retrieved_objs = $session->search(
      object       => "Infoblox::DHCP::Network",
      name         => "^netview\$"
    );

# search for DHCP views with the extensible attribute 'Site'
my @retrieved_objs = $session->search(
   object => "Infoblox::DHCP::View",
   extensible_attributes => { 'Site' => 'Santa Clara' });
```

METHODS

This section describes all the methods that can be used to configure and retrieve the attribute values of a DHCP Network View object

associated_dns_views( )

Use this method to retrieve the list of DNS views associated with this network view. This list is read-only and cannot be set.

Parameter

None

Returns

Returns an array reference that contains Infoblox::DNS:View object(s).

Example

```
# Get associated_dns_views
my $associated_dns_views = $network_view->associated_dns_views()
```

cloud_info( )

Use this method to set or retrieve cloud API related information for the Infoblox::DHCP::View object.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

The valid value is a Infoblox::Grid::CloudAPI::Info object.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
# Get cloud_info
my $cloud_info = $network_view->cloud_info();
# Modify cloud_info
$network_view->cloud_info($cloud_info_object);
```

comment( )

Use this method to set or retrieve a descriptive comment of a DHCP Network View object.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

Enter a descriptive comment for the network view in string format with a maximum of 256 bytes.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
# Get comment
my $comment = $network_view->comment();
# Modify comment
```

```
$network_view->comment("add a network view");
```

ddns_zone_primaries( )

Use this method to set or retrieve an ordered list of zone primaries that will receive DDNS updates.

Parameter

A valid value is a reference to an array of Infoblox::DHCP::DDNS objects.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
# Get the attribute value
$value = $network_view->ddns_zone_primaries();
# Modify the attribute value
$network_view->ddns_zone_primaries([$ddns]);
```

extensible_attributes( )

Use this method to set or retrieve the extensible attributes associated with a DHCP view object.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

For valid values for extensible attributes, see Infoblox::Grid::ExtensibleAttributeDef/Extensible Attribute Values.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get extensible attributes
my $ref_extensible_attributes = $firstview->extensible_attributes();
#Modify extensible attributes
$firstview->extensible_attributes({ 'Site' => 'Santa Clara', 'Administrator' => [ 'Peter', 'Tom' ]
});
```

external_ddns_primaries( )

Use this method to set or retrieve the network view-level external_ddns_primaries value. This specifies the external zone that is updated dynamically. The DHCP server can send dynamic updates to a specified external name server.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

A valid value is an array reference that contains Infoblox::DNS::Nameserver object(s).

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
# Get external_ddns_primaries
my $external_ddns_primaries = $network_view->external_ddns_primaries();

# Modify external_ddns_primaries
my $nameserver1 = Infoblox::DNS::Nameserver->new(
        ddns_zone => "ns1.del.com",
        ipv4addr  => "11.11.11.11",
    );
my $nameserver2 = Infoblox::DNS::Nameserver->new(
         ddns_zone => "3.2.1.in-addr.arpa",
         ipv4addr  => "22.22.22.22",
    );
```

```
# Nameserver with TSIG security
my $nameserver3 = Infoblox::DNS::Nameserver->new(
        ddns_zone     => 'grid_level.local',
        ipv4addr      => '10.102.30.122',
        TSIGalgorithm => 'HMAC-SHA256',
        TSIGname      => 'kk',
        TSIGkey       => '+NPBBVYjoNC32hfj8vepUw==',
);

# Nameserver with GSS-TSIG security
my $nameserver4 = Infoblox::DNS::Nameserver->new(
        ipv4addr               => '10.102.30.122',
        gss_tsig_domain        => 'AD-31.LOCAL',
        ddns_zone              => 'grid1_level.local',
        gss_tsig_dns_principal => 'DNS/w8s232-29.ad-31.local@AD-31.LOCAL'
);

$network_view->external_ddns_primaries([$nameserver1, $nameserver2, $nameserver3, $nameserver4]);

# Clear external_ddns_primaries list
$network_view->external_ddns_primaries([]);
```

mgm_private( )

Use this method to set or retrieve the flag that controls whether the object should be synced to the MGM or not.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

Specify 'true' to enable synchronization of the managed Grid with the MGM or 'false' to forbid it. The default value is 'false'.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get mgm_private value
my $value = $object->mgm_private();

#Modify mgm_private value
$object->mgm_private('true');
```

modify_vrf_assignment()

This method is used to assign (unassign) a list of the virtual networks (VRFs) to the network view.

Parameters

VRF membership modification mode: ASSIGN or UNASSIGN.

A valid list (array reference) that contains Infoblox::Grid::Member::Discovery::VRF object(s).

Returns

Returns true on success operation, otherwise false.

Example

```
# Get Network View
my $default_view = $session->get(object => 'Infoblox::DHCP::View', name => 'default');

# Get Infoblox::Grid::Member::Discovery::VRF objects
my $vrf1 = $session->get(object => 'Infoblox::Grid::Member::Discovery::VRF', name =>
'VirtualNetworkMember1');
my $vrf2 = $session->get(object => 'Infoblox::Grid::Member::Discovery::VRF', name =>
'VirtualNetworkMember2');

# Assign a list of the VRFs to the Network View
my $assign_result = $default_view->modify_vrf_assignment(
       vrfs => [ $vrf1, $vrf2 ],
       mode => 'ASSIGN'
);

# Unassign VRF from Network View
```

```
my $unassign_result = $default_view->modify_vrf_assignment(
        vrfs => [ $vrf2 ],
        mode => 'UNASSIGN'
);

# Check of VRF assignment via Infoblox::Grid::Member::Discovery::VRF object
$vrf = $session->get(object => 'Infoblox::Grid::Member::Discovery::VRF', name =>
'VirtualNetworkMember1');

# Get current network view name associated with VRF
print $vrf->network_view;
```

internal_ddns_zones( )

Use this method to set or retrieve forward mapping zones which are being served in the grid. Only an authoritative zone can be assigned to this list. All other types of zones will generate an error.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

A valid value is an array reference that contains Infoblox::DNS::Zone object(s).

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
# Get internal_ddns_zones
my $internal_ddns_zones = $network_view->internal_ddns_zones();

# Modify internal_ddns_zones
my $nameserver1 = Infoblox::DNS::Nameserver->new(
        name     => "ns1.del.com",
        ipv4addr => "11.11.11.11",
    );
my $nameserver2 = Infoblox::DNS::Nameserver->new(
        name     => "test",
        ipv6addr => "2006::b671",
    );
my $zone1 = Infoblox::DNS::Zone->new(
        name     => "test1.com",
        primary  => [$nameserver1],
        secondaries => [$nameserver2],
    );
my $zone2 = Infoblox::DNS::Zone->new(
        name     => "test2.com",
    );
$network_view->internal_ddns_zones([$zone1, $zone2]);

# Clear internal_ddns_zones list
$network_view->internal_ddns_zones([]);
```

internal_updates_view( )

Use this method to specify or retrieve the internal updates DNS view value of a network view. When you enable the appliance to send updates to grid members, you must specify which DNS view receives the updates.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

A valid value is an Infoblox::DNS::View object for internal dynamic updates.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
# Get internal_updates_view
my $internal_updates_view = $network_view->internal_updates_view();
# Modify internal_updates_view
```

```
my $otherview = Infoblox::DNS::View->new(
        name => "other",
    );
$network_view->internal_updates_view($otherview);
```

ms_ad_user_data( )

Use this method to retrieve Microsoft Active Directory users related information. This is a read-only attribute.

Parameter

None

Returns

The valid return value is an Infoblox::Grid::MSServer::AdUser::Data object.

Example

```
#Get ms_ad_user_data
my $ms_ad_user_data = $view->ms_ad_user_data();
```

name( )

Use this method to set or retrieve the name of a DHCP Network View object.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

The name of the network view is in string format with a maximum of 64 bytes.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
# Get name
my $name = $network_view->name()
# Modify name
$network_view->name("netview_renamed");
```

SAMPLE CODE

The following sample code demonstrates the different functions that can be applied to an object such as add, search, modify, and remove. This sample also includes error handling for the operations.

#Preparation prior to a DHCP Network View insertion

```
#PROGRAM STARTS: Include all the modules that will be used
use strict;
use Infoblox;

#Create a session to the Infoblox appliance

my $session = Infoblox::Session->new(
    master   => "192.168.1.2",
    username => "admin",
    password => "infoblox"
);
unless ($session) {
   die("Construct session failed: ",
       Infoblox::status_code() . ":" . Infoblox::status_detail());
}
print "Session created successfully\n";
```

#Create a DHCP Network View object

```
my $network_view = Infoblox::DHCP::View->new(
           name     => "netview",
          );

my $response = $session->add($network_view)
       or die("Add Network View failed: ",
```

```
            $session->status_code() . ":" . $session->status_detail());

print "Network View added successfully\n";
```

#Search for a specific Network View

```
#Search the network view
my @retrieved_objs = $session->search(
    object  => "Infoblox::DHCP::View",
    name => '^netview$',
);
my $object = $retrieved_objs[0];
unless ($object) {
    die("Search Network View failed: ",
        $session->status_code() . ":" . $session->status_detail());
}
print "Search Network found at least 1 matching entry\n";
```

#Get and modify a Network View attribute

```
#Get Network View through the session
my  @retrieved_objs = $session->get(
           object => "Infoblox::DHCP::View" ,
           name   => "netview",
        );

my $object = $retrieved_objs[0];

unless ($object) {
    die("Get Network View object failed: ",
        $session->status_code() . ":" . $session->status_detail());
}
print "Get Network View found at least 1 matching entry\n";

#Modify one of the attributes of the specified Network View attribute
$object->comment("updated comment"); #Apply the changes

$session->modify($object)
    or die("Modify Network View attribute failed: ",
           $session->status_code() . ":" . $session->status_detail());

print "Network View object modified successfully \n";
```

#Remove a Network View Object

```
 #Get Network View object through the session
 my @retrieved_objs = $session->get(
           object  => "Infoblox::DHCP::View" ,
           name    => "netview",
       );

   my $object = $retrieved_objs[0];

   unless ($object) {
    die("Get Network View object failed: ",
        $session->status_code() . ":" . $session->status_detail());
    }

print "Get Network View object found at least 1 matching entry\n";

#Submit the object for removal
$session->remove($object)
    or die("Remove Network View failed: ",
        $session->status_code() . ":" . $session->status_detail());

print "Network View removed successfully \n";

####PROGRAM ENDS####
```

AUTHOR

Infoblox Inc. http://www.infoblox.com/

SEE ALSO

Infoblox::Session, Infoblox::DNS::View, Infoblox::DHCP::Network, Infoblox::DNS::Zone, Infoblox::DNS::Nameserver, Infoblox::Session->get(), Infoblox::Session->search(), Infoblox::Session->add(), Infoblox::Session->remove(), Infoblox::Session->modify(), Infoblox::DHCP::DDNS, Infoblox::Grid::MSServer::AdUser::Data

---

COPYRIGHT

Copyright (c) 2017 Infoblox Inc.

Infoblox::DNS::AllRecords - A synthetic object used to return all record object types belonging to a zone.

- NAME
- DESCRIPTION
- SESSION METHODS
  - Infoblox::Session->search( )
- METHODS
  - comment( )
  - creator( )
  - ddns_principal( )
  - ddns_protected( )
  - disable( )
  - dtc_obscured( )
  - last_queried( )
  - name( )
  - reclaimable( )
  - record( )
  - ttl( )
  - type( )
  - view( )
  - zone( )
- SAMPLE CODE
- AUTHOR
- SEE ALSO
- COPYRIGHT

NAME

Infoblox::DNS::AllRecords - A synthetic object used to return all record object types belonging to a zone.

DESCRIPTION

By searching on this object, all the record objects associated with a specific zone can be retrieved

---

SESSION METHODS

This section describes all the methods in Infoblox::Session module that you can apply to a DNS AllRecords object.

Infoblox::Session->search( )

Use this method to search for record objects in the Infoblox appliance. See Infoblox::Session->search() for parameters and return values.

Key References

```
 Apply the following attributes to search for a DNS AllRecords object:

  zone           - Required. A zone name in FQDN format.
  creator        - Optional. The creator of the record. The valid values are 'SYSTEM', 'DYNAMIC' and
'STATIC'.
  ddns_principal - Optional. The GSS-TSIG principal that owns the record.
  view           - Optional. The DNS view in which the records are located. By default, the default
DNS views is searched.
  name           - Optional. The name of the matched record. (regular expression).
  reclaimable    - Optional. The flag that indicates whether the record is reclaimable or not.
  type           - Optional. The type of record to search for. By default, all record types are
searched. Following is the list of valid values for this search field:

                 'All A Records',
                 'All AAAA Records',
                 'All Alias Records',
                 'All Bulk Host',
                 'All CNAME Records',
                 'All DNAME Records',
                 'All DNSKEY Records',
                 'All DS Records',
                 'All DTCLBDN Records',
                 'All Host Address',
                 'All Host Alias',
                 'All Host Records',
                 'All IPV6 Host Address',
                 'All LBDN Records',
                 'All MX Records',
                 'All NAPTR Records',
                 'All NS Records',
                 'All NSEC Records',
                 'All NSEC3 Parameter Records',
```

```
                    'All NSEC3 Records',
                    'All PTR Records',
                    'All RSIG Records',
                    'All Records',
                    'All SEC Records',
                    'All SOA Records',
                    'All SRV Records',
                    'All TLSA Records',
                    'All Shared A Records',
                    'All Shared AAAA Records',
                    'All Shared MX Records',
                    'All Shared SRV Records',
                    'All Shared TXT Records',
                    'All TXT Records',
```

Example

```
# search for all records in the "domain.com" zone in the default DNS view
my @retrieved_objs = $session->search(
    object => "Infoblox::DNS::AllRecords",
    zone   => 'domain.com',
);

# search for all TXT records that match the 'rec' name in the "domain.com" zone in the default DNS
view
my @retrieved_objs = $session->search(
    object => "Infoblox::DNS::AllRecords",
    zone   => 'domain.com',
    type   => 'All TXT Records',
    name   => 'rec',
);
```

METHODS

This section describes all the methods that you can use to retrieve the attribute values of an AllRecords object. All methods are read-only and cannot be set.

comment( )

Use this method to retrieve the descriptive comment of the record associated with the AllRecords object.

Returns

The method returns the attribute value.

Example

```
#Get comment
my $comment = $allrecord->comment();
```

creator( )

Use this method to retrieve the resource record creator.

Returns

The method returns the attribute value.

Example

```
#Get creator
my $creator = $allrecord->creator();
```

ddns_principal( )

Use this method to retrieve the GSS-TSIG principal that owns the record.

Returns

The method returns the attribute value.

Example

```
#Get ddns_principal
my $ddns_principal = $allrecord->ddns_principal();
```

ddns_protected( )

Use this method to retrieve the flag that indicates whether this record is protected from DDNS updates or not.

Returns

The method returns the attribute value.

Example

```
#Get ddns_protected
my $ddns_protected = $allrecord->ddns_protected();
```

disable( )

Use this method to retrieve the disable flag of of the record associated with the AllRecords object (if present).

Returns

The method returns the attribute value.

Example

```
#Get disable
my $disable = $allrecord->disable();
```

dtc_obscured( )

Use this method to retrieve the LBDN object that obscures this record. This is a read-only attribute.

Parameter

None

Returns

The method returns attribute value.

Example

```
#Get dtc_obscured
my $dtc_obscured = $allrecord->dtc_obscured();
```

last_queried( )

Use this method to retrieve the time the associated record was last queried on.

Returns

The method returns the attribute value, the number of seconds that have elapsed since January 1st, 1970 UTC.

Example

```
#Get last_queried
my $last_queried = $allrecord->last_queried();
```

name( )

Use this method to retrieve the name of of the record associated with the AllRecords object. Note that this value could differ from the value of the associated record's name field.

Returns

The method returns the attribute value.

Example

```
#Get name
my $name = $allrecord->name();
```

reclaimable( )

Use this method to retrieve the flag that indicates whether the record is reclaimable or not.

Parameter

None

Returns

The method returns the attribute value.

Example

```
#Get reclaimable
my $reclaimable = $object->reclaimable();
```

record( )

Use this method to retrieve the record object associated with the AllRecords object.

This object is considered a 'partial' object. See Infoblox::Session->fill_partial_object() for more information on how it can be converted to a 'full' API object.

If the AllRecords object refers to a record not currently supported in the Perl API, the member value will be undefined.

Returns

The method returns the attribute value.

Example

```
#Get record
my $record = $allrecord->record();
```

ttl( )

Use this method to retrieve the TTL value of of the record associated with the AllRecords object (if present).

Returns

The method returns the attribute value.

Example

```
#Get ttl
my $ttl = $allrecord->ttl();
```

type( )

Use this method to retrieve the type of the record associated with the AllRecords object. This is a descriptive string that identifies the record to which this AllRecords object refers. (For example, 'A record', 'MX record', ...)

Returns

The method returns the attribute value.

Example

```
#Get type
my $type = $allrecord->type();
```

view( )

Use this method to retrieve the DNS view name of the record associated with the AllRecords object.

Returns

The method returns the attribute value.

Example

```
#Get view
my $view = $allrecord->view();
```

zone( )

Use this method to retrieve the zone name of the record associated with the AllRecords object.

Returns

The method returns the attribute value.

Example

```
#Get zone
my $zone = $allrecord->zone();
```

SAMPLE CODE

```
 In cases where a large number of matching records exist on the appliance, only a limited number will be
returned
 at a time. If all of the matching records are required, please consider using the paging support via
Infoblox::Cursor.

 #Code sample fragment Starts:

    $cursor=$session->new_cursor(
                                 fetch_size => 100,
                                 object => "Infoblox::DNS::AllRecords",
                                 zone   => 'domain.com',
                                 view   => 'default',
                                );

    while ($current_record = $cursor->fetch()) {
        # $current_record will contain a matching AllRecords object
    }

    # All matching AllRecords will be returned
    # when the cycle ends.

 If you want full record objects for some of the above, you can add the following code

 #Code sample fragment Starts:

     if( ... some criteria ...) {
         push @temp, $current_record->record();
     }

 ####Code Fragment Ends####

 inside the while() cycle and afterwards call

 #Code sample fragment Starts:

 @full_record_objects=$session->fill_partial_object(\@temp);

 ####Code Fragment Ends####

 to fill the wanted objects.
```

AUTHOR

Infoblox Inc. http://www.infoblox.com/

SEE ALSO

Infoblox::Session, Infoblox::Session->search()/>

COPYRIGHT

Copyright (c) 2017 Infoblox Inc.

Infoblox::DNS::AllRPZRecords - A synthetic object used to return record object types that belong to a Response Policy Zone.

- NAME
- DESCRIPTION
- SESSION METHODS
  - Infoblox::Session->search( )
- METHODS
  - alert_type( )
  - comment( )
  - expiration_time( )
  - disable( )
  - last_updated( )
  - name( )
  - record( )
  - ttl( )
  - type( )
  - view( )
  - zone( )
  - rpz_rule( )
- SAMPLE CODE
- AUTHOR
- SEE ALSO
- COPYRIGHT

NAME

Infoblox::DNS::AllRPZRecords - A synthetic object used to return record object types that belong to a Response Policy Zone.

DESCRIPTION

By searching on this object, all record objects associated with a specific zone can be retrieved.

SESSION METHODS

This section describes all the methods in Infoblox::Session module that you can apply to a DNS AllRPZRecords object.

Infoblox::Session->search( )

Use this method to search for record objects in the Infoblox appliance. See Infoblox::Session->search() for parameters and return values.

Key References

```
 Apply the following attributes to search for a DNS AllRPZRecords object:

  zone       - Required. A Response Policy Zone name in FQDN format.
  view       - Optional. The DNS view in which the records are located. By default, the default DNS
view is searched.
  name       - Optional. The name of the matched record. (regular expression).
  type       - Optional. The type of record to search for. By default, all record types are searched.
Following is the list of valid values for this search field:

                'All Records',
                'All A Records',
                'All AAAA Records',
                'All CNAME Records',
                'All IP Address Record',
                'All MX Record',
                'All NAPTR Record',
                'All PTR Record',
                'All SRV Record',
                'All TXT Record',
                'All A IP Address Record',
                'All AAAA IP Address Record'

  rpz_rule  - Optional. The Response Policy Zone rule type of record to search for. By default, all
rule types are searched. Following is the list of valid values for this search field:

                'PASSTHRUIPADDR',
                'PASSTHRUDOMAIN',
                'BLOCKNXDOMAINIPADDR',
                'BLOCKNXDOMAINDOMAIN',
                'BLOCKNODATAIPADDR',
                'BLOCKNODATADOMAIN',
                'SUBSTITUTE_CNAME',
                'SUBSTITUTE_A_RECORD',
                'SUBSTITUTE_AAAA_RECORD',
```

```
                    'SUBSTITUTE_MX_RECORD',
                    'SUBSTITUTE_PTR_RECORD',
                    'SUBSTITUTE_NAPTR_RECORD',
                    'SUBSTITUTE_SRV_RECORD',
                    'SUBSTITUTE_TXT_RECORD',
                    'SUBSTITUTE_IPV4ADDRESS_RECORD',
                    'SUBSTITUTE_IPV6ADDRESS_RECORD'
                    'SUBSTITUTE_IPADDR_CNAME'
```

Example

```
# search for all RPZ records in the "rpz.com" zone in the default DNS view
my @retrieved_objs = $session->search(
    object => "Infoblox::DNS::AllRPZRecords",
    zone   => 'rpz.com',
);

# search for all TXT records that match the 'rec' name in the "rpz.com" zone in the default DNS view
my @retrieved_objs = $session->search(
    object => "Infoblox::DNS::AllRPZRecords",
    zone   => 'rpz.com',
    type   => 'All TXT Records',
    name   => 'rec',
);

# search for all PASSTHRUDOMAIN Response Policy Rule type in the "rpz.com" zone in the default DNS
view
my @retrieved_objs = $session->search(
    object => "Infoblox::DNS::AllRPZRecords",
    zone   => 'rpz.com',
    rpz_rule   => 'PASSTHRUDOMAIN',
);
```

METHODS

This section describes all the methods that you can use to retrieve the attribute values of an AllRPZRecords object. All methods are read-only and cannot be set.

alert_type( )

Use this method to retrieve the alert type of the record associated with the AllRPZRecords object.

Returns

The method returns the attribute value.

Example

```
# Get alert_type
my $alert_type = $allrpzrecord->alert_type();
```

comment( )

Use this method to retrieve the descriptive comment of the record associated with the AllRPZRecords object.

Returns

The method returns the attribute value.

Example

```
#Get comment
my $comment = $allrpzrecord->comment();
```

expiration_time( )

Use this method to retrieve expiration time of the record associated with the AllRPZRecords object.

Returns

The method returns the attribute value.

Example

```
#Get expiration_time
my $expiration_time = $allrpzrecord->expiration_time();
```

disable( )

Use this method to retrieve the disable flag of the record associated with the AllRPZRecords object (if present).

Returns

The method returns the attribute value.

Example

```
#Get disable
my $disable = $allrpzrecord->disable();
```

last_updated( )

Use this method to retrieve the time the record associated with the AllRPZRecords object was last updated at.

Returns

The method returns the attribute value.

Example

```
#Get last_updated
my $last_updated = $allrpzrecord->last_updated();
```

name( )

Use this method to retrieve the name of of the record associated with the AllRPZRecords object. Note that this value could differ from the value of the associated record's name field.

Returns

The method returns the attribute value.

Example

```
#Get name
my $name = $allrpzrecord->name();
```

record( )

Use this method to retrieve the record object associated with the AllRPZRecords object.

This object is considered a 'partial' object. See Infoblox::Session->fill_partial_object() for more information on how it can be converted to a 'full' API object.

If the AllRPZRecords object refers to a record not currently supported in the Perl API, the member value will be undefined.

Returns

The method returns the attribute value.

Example

```
#Get record
my $record = $allrpzrecord->record();
```

ttl( )

Use this method to retrieve the TTL value of the record associated with the AllRPZRecords object (if present).

Returns

The method returns the attribute value.

Example

```
#Get ttl
my $ttl = $allrpzrecord->ttl();
```

type( )

Use this method to retrieve the type of record associated with the AllRPZRecords object. This is a descriptive string that identifies the record to which this AllRPZRecords object refers. (For example, 'A record', 'MX record', ...)

Returns

The method returns the attribute value.

Example

```
#Get type
my $type = $allrpzrecord->type();
```

view( )

Use this method to retrieve the DNS view name of the record associated with the AllRPZRecords object.

Returns

The method returns the attribute value.

Example

```
#Get view
my $view = $allrpzrecord->view();
```

zone( )

Use this method to retrieve the Response Policy Zone name of the record associated with the AllRPZRecords object.

Returns

The method returns the attribute value.

Example

```
#Get zone
my $zone = $allrpzrecord->zone();
```

rpz_rule( )

Use this method to retrieve the Response Policy Zone rule type of the record associated with the AllRPZRecords object.

Returns

The method returns the attribute value.

Example

```
#Get rule type
my $rpz_rule = $allrpzrecord->rpz_rule();
```

SAMPLE CODE

```
 In cases where a large number of matching records exist on the appliance, only a limited number will be
returned
 at a time. If all of the matching records are required, please consider using the paging support via
Infoblox::Cursor.

 #Code sample fragment Starts:

    $cursor=$session->new_cursor(
                                 fetch_size => 100,
                                 object => "Infoblox::DNS::AllRPZRecords",
                                 zone   => 'rpz.com',
                                 view   => 'default',
                                );

    while ($current_record = $cursor->fetch()) {
        # $current_record will contain a matching AllRPZRecords object
    }

    # All matching AllRPZRecords will be returned
    # when the cycle ends.

 If you want full record objects for some of the above, you can add the following code

 #Code sample fragment Starts:

     if( ... some criteria ...) {
         push @temp, $current_record->record();
     }

 ####Code Fragment Ends####
```

```
inside the while() cycle and afterwards call
#Code sample fragment Starts:
@full_record_objects=$session->fill_partial_object(\@temp);
####Code Fragment Ends####
to fill the wanted objects.
```

AUTHOR

Infoblox Inc. http://www.infoblox.com/

SEE ALSO

Infoblox::Session, Infoblox::Session->search()/>

COPYRIGHT

Copyright (c) 2017 Infoblox Inc.

Infoblox::DNS::BulkHost - Create a group of hosts.

- NAME
- DESCRIPTION
- CONSTRUCTOR
- SESSION METHODS
  - Infoblox::Session->add( )
  - Infoblox::Session->get( )
  - Infoblox::Session->modify( )
  - Infoblox::Session->remove( )
  - Infoblox::Session->search( )
- METHODS
  - cloud_info( )
  - comment( )
  - disable( )
  - dns_prefix( )
  - end_addr( )
  - extattrs( )
  - last_queried( )
  - name_template( )
  - prefix( )
  - reverse( )
  - start_addr( )
  - ttl( )
  - use_name_template( )
  - views( )
  - zone( )
- SAMPLE CODE
- AUTHOR
- SEE ALSO
- COPYRIGHT

NAME

Infoblox::DNS::BulkHost - Create a group of hosts.

DESCRIPTION

If you need to add a large number of hosts, you can have the Infoblox appliance add them as a group and automatically assign host names based on a range of IP addresses and name format applied to it. This group of hosts is referred to as a BulkHost. The Infoblox appliance uses the name space bulk-xx-xx-xx-xx for BulkHosts, so this name should not be used for CNAMEs and host aliases because doing so causes conflicts. Before adding a BulkHost, make sure that no CNAMEs or host aliases uses this name.

Note that you must specify only one view for the attribute "views".

CONSTRUCTOR

```
my $bulkhost = Infoblox::DNS::BulkHost->new(
    end_addr           => $ipv4addr,              #Required
    prefix             => $string,                #Required
    start_addr         => $ipv4addr,              #Required
    zone               => $fqdn,                  #Required
    comment            => $string,                #Optional / Default is empty
    disable            => "true" | "false",       #Optional / Default "false"
    name_template      => $string | undef,        #Optional / Default is undefined
    reverse            => "true" | "false",       #Optional / Default "false"
    ttl                => $num | undef,           #Optional / Default is undefined
    use_name_template  => "true" | "false",       #Optional / Default is "false"
    views              => [$View1]                        #Optional / Default is "default" view
    extattrs           => { $string => $extattr, ... },      #Optional / Default is undefined
);
```

SESSION METHODS

This section describes all the methods in Infoblox::Session module that you can apply to a DNS BulkHost record object.

Infoblox::Session->add( )

Use this method to add a BulkHost record object to the Infoblox appliance. See Infoblox::Session->add() for parameters and return

values.

Example

```
#Construct an object
my $bulk_host = Infoblox::DNS::BulkHost->new(
    zone               => "domain.com",
    comment            => "add bulk host",
    prefix             => "prefix",
    start_addr         => "1.0.0.1",
    end_addr           => "1.0.0.5",
    reverse            => "true",
    use_name_template  => "true",
    name_template      => "Three Octets"
);
#Submit for addition
my $response = $session->add($bulk_host)
```

Infoblox::Session->get( )

Use this method to retrieve all the matching objects from the Infoblox appliance. See Infoblox::Session->get() for parameters and return values.

Key References

```
Apply the following attributes to get a specific DNS BulkHost object:

 zone - Optional. A zone name in string format.
 prefix     - Optional. Prefix of the bulk host.
 start_addr - Optional. Start address of the bulk host.
 end_addr   - Optional. End address of the bulk host.
 comment    - Optional. A comment in string format.
 view - Optional. The DNS view in which the bulk host record is located. The default is all DNS
views.
 ttl - Optional. Time to Live (TTL) value in integer format.
 name_template - Optional. The bulk host name template in string format.
 disable - Optional. Disable flag of a DNS BulkHost record in boolean format.
 use_name_template - Optional. Use name template flag of a DNS BulkHost record in boolean format.
 reverse - Optional. Reverse flag of a DNS BulkHost record in boolean format.
 extattrs     - Optional. A hash reference containing extensible attributes.
```

Example

```
#Get BulkHost object through the session
my @retrieved_objs = $session->get(
    object => "Infoblox::DNS::BulkHost",
    zone   => "domain.com",
    prefix => "prefix",
    start_addr => "1.0.0.2",
    end_addr => "1.0.0.6".
    view => "default",
    ttl => 3600,
    name_template => "Three Octets",
    disable => "false",
    use_name_template => "true",
    reverse => "true"
);
```

Infoblox::Session->modify( )

Use this method to modify a BulkHost record object in the Infoblox appliance. See Infoblox::Session->modify() for parameters and return values.

Example

```
#Use method to modify the prefix.
$bulk_host->prefix("test");
#Submit modification
my $response = $session->modify( $bulk_host );
```

Infoblox::Session->remove( )

Use this method to remove a BulkHost record object from the Infoblox appliance. See Infoblox::Session->remove() for parameters and return values.

To remove a specific object, first use `get()` or `search()` to retrieve the specific BulkHost record object, and then submit this object for removal.

Example

```
#Get BulkHost object through the session
my @retrieved_objs = $session->get(
    object     => "Infoblox::DNS::BulkHost",
    zone       => "domain.com",
    prefix     => "test",
    start_addr => "1.0.0.1",
    end_addr   => "1.0.0.5"
);
#Find the desired object from the retrieved list.
my $desired_bulk = $retrieved_objs[0];
#Submit for removal
my $response = $session->remove( $desired_bulk );
```

Infoblox::Session->search( )

Use this method to search for Bulk Host objects in the Infoblox appliance. See Infoblox::Session->search() for parameters and return values.

Key References

```
Apply the following attributes to search for a specific Bulk Host object:

 zone       - Optional. Zone name in FQDN format (regular expression).
 prefix     - Optional. Prefix of the bulk host (regular expression).
 start_addr - Optional. Start address of the bulk host.
 end_addr   - Optional. End address of the bulk host.
 comment    - Optional. A comment in string format (regular expression).
 view       - Optional. The DNS view in which the bulk host record is located (regular expression).
The default is all DNS views.

 ttl - Optional. Time to Live (TTL) value in integer format.
 name_template - Optional. The bulk host name template in string format.
                 This is B<not> a regular expression, so do not specify this
                 value to ".*" if you want to obtain BulkHost records with all
                 name templates. Just leave this field unspecified.
 disable - Optional. Disable flag of a DNS BulkHost record in boolean format.
 use_name_template - Optional. Use name template flag of a DNS BulkHost record in boolean format.
 reverse - Optional. Reverse flag of a DNS BulkHost record in boolean format.
 extattrs     - Optional. A hash reference containing extensible attributes.
```

Example

```
# search for all Bulk Host objects that start with "prefix"
my @retrieved_objs = $session->search(
    object => "Infoblox::DNS::BulkHost",
    prefix => "prefix.*"
);
```

METHODS

This section describes all the methods that you can use to set and retrieve the attribute values of a BulkHost object.

cloud_info( )

Use this method to retrieve cloud API related information for the Infoblox::DNS::BulkHost object.

Parameter

None

Returns

The method returns the attribute value.

Example

```
# Get cloud_info
my $cloud_info = $object->cloud_info();
```

comment( )

Use this method to set or retrieve the descriptive comment.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

Desired comment in string format with a maximum of 256 bytes.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get comment
my $comment = $bulkhost->comment();
#Modify comment
$bulkhost->comment("Modifying the BulkHost comment");
```

disable( )

Use this method to set or retrieve the disable flag of a DNS BulkHost record.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

The default value for this field is false. The DNS BulkHost record is enabled.

Parameter

Specify "true" to set the disable flag or "false" to deactivate/unset it.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get disable
my $disable = $bulkhost->disable();
#Modify disable
$bulkhost->disable("true");
```

dns_prefix( )

Use this method to retrieve the prefix, in punycode format, for the bulk host. This is a read-only attribute.

Parameter

None

Returns

The method returns the attribute value.

Example

```
# Get attribute value
my $value = $bulkhost->dns_prefix();
```

end_addr( )

Use this method to set or retrieve the last IP address in the address range of the BulkHost.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

An IP address is a 32-bit number in dotted decimal notation. It consists of four 8-bit groups of decimal digits separated by decimal points (example: 192.168.1.2).

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get end_addr
my $end_addr = $bulkhost->end_addr();
#Modify end_addr
$bulkhost->end_addr("1.0.0.6");
```

extattrs( )

Use this method to set or retrieve the extensible attributes associated with a DNS BulkHost object.

Parameter

Valid value is a hash reference containing the names of extensible attributes and their associated values (Infoblox::Grid::Extattr objects).

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get extattrs
my $ref_extattrs = $bulkhost->extattrs();
#Modify extattrs
$bulkhost->extattrs({ 'Site' => $extattr1, 'Administrator' => $extattr2 });
```

last_queried( )

Use this method to retrieve the time when the associated record was last queried. This is a read-only attribute.

Parameter

None

Returns

The method returns the attribute value. The number of seconds that have elapsed since January 1st, 1970 UTC.

Example

```
#Get last_queried
my $last_queried = $bulkhost->last_queried();
```

name_template( )

Use this method to set or retrieve the bulk host name template.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

The default value is undefined which indicates the grid level default bulk host name format is used.

Specify a bulk host name format value to override the name format at the grid level.

Parameter

The name of a bulk host name format.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get name_template
my $name_template = $bulkhost->name_template();
#Modify name_template
$bulkhost->name_template("Three Octets");
```

prefix( )

Use this method to set or retrieve the prefix for the BulkHost. The prefix is the name (or a series of characters) inserted at the beginning of each host name.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

The attribute value can be in unicode format.

Parameter

Desired text with the prefix for the BulkHost. The sum of the bulk host prefix length and suffix length must not exceed 63

characters. If it does, the appliance displays an error message indicating the number of characters that you must remove to make a valid prefix.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get prefix
my $prefix = $bulkhost->prefix();
#Modify prefix
$bulkhost->prefix("test");
```

reverse( )

Use this method to set or retrieve the reverse flag of the BulkHost record.

If the flag is enabled, reverse mapping `address(es)` are added automatically if a reverse mapping zone exists.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

Specify "true" to set the flag or "false" to deactivate/unset it.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get reverse
my $reverse = $bulkhost->reverse();
#Modify reverse
$bulkhost->reverse("true");
```

start_addr( )

Use this method to set or retrieve the first IP address in the address range of the BulkHost.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

An IP address is a 32-bit number in dotted decimal notation. It consists of four 8-bit groups of decimal digits separated by decimal points (example: 192.168.1.2).

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get start_addr
my $start_addr = $bulkhost->start_addr();
#Modify start_addr
$bulkhost->start_addr("1.0.0.2");
```

ttl( )

Use this method to set or retrieve the Time to Live (TTL) value.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

The default value is undefined which indicates that the record inherits the TTL value of the zone.

Specify a TTL value to override the TTL value at the zone level.

Parameter

A 32-bit integer (range from 0 to 4294967295) that represents the duration in seconds that the record is cached. Zero indicates that the record should not be cached.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get ttl
my $ttl = $bulkhost->ttl();
#Modify ttl
$bulkhost->ttl(3600);
```

use_name_template( )

Use this method to set or retrieve the use_name_template flag value. This setting overrides the grid level bulk host name format.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

Specify "true" to set the flag or "false" to deactivate/unset it.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get use_name_template
my $use_name_template = $bulkhost->use_name_template();
#Modify use_name_template
$bulkhost->use_name_template("true");
```

views( )

Use this method to set or retrieve the view of a BulkHost.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

The default value is the "default" view, which means the BulkHost object is located under the default view.

Parameter

Array reference of defined Infoblox::DNS::View objects.

Note that the array size must be 1.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get views
my $views = $bulkhost->views();
#Modify views
$bulkhost->views([$view1]);
```

zone( )

Use this method to set or retrieve the zone name.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

A zone name in FQDN (Fully Qualified Domain Name) format. The FQDN consists of the host_name followed by the domain name (example: abc.com). A host_name can have a maximum of 256 characters.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get zone
my $zone = $bulkhost->zone();
#Modify zone
$bulkhost->zone("test.com");
```

SAMPLE CODE

The following sample code demonstrates the different functions that can be applied to an object, such as add, search, modify, and remove. This sample also includes error handling for the operations.

#Preparation prior to a Bulk Host object insertion

```
#PROGRAM STARTS: Include all the modules that will be used
use strict;
use Infoblox;

#Create a session to the Infoblox appliance

my $session = Infoblox::Session->new(
               master   => "192.168.1.2", #appliance host ip
               username => "admin",       #appliance user login
               password => "infoblox"     #appliance password
);

unless ($session) {
       die("Construct session failed: ",
               Infoblox::status_code() . ":" . Infoblox::status_detail());
}
print "Session created successfully\n";

#Create the zone prior to a BulkHost insertion
my $zone = Infoblox::DNS::Zone->new(
    name    => "domain.com",
    email   => "admin\@domain.com",
    comment => "add a zone domain.com",
    );
unless ($zone) {
   die("Construct zone failed: ",
       Infoblox::status_code() . ":" . Infoblox::status_detail());
}
print "Zone object created successfully\n";

#Verify if the zone exists
my $object = $session->get(object => "Infoblox::DNS::Zone", name => "domain.com");
unless ($object) {
   print "Zone does not exist on server, safe to add the zone\n";
   $session->add($zone)
      or die("Add zone failed: ",
             $session->status_code() . ":" . $session->status_detail());
}
print "Zone added successfully\n";
```

#Create a BulkHost object

```
my $bulk_host = Infoblox::DNS::BulkHost->new(
    zone               => "domain.com",
    comment            => "add bulk host",
    prefix             => "prefix",
    start_addr         => "1.0.0.1",
    end_addr           => "1.0.0.5",
    use_name_template  => "true",
    name_template      => "Three Octets",
    reverse            => "true"
    );
unless($bulk_host) {
     die("Construct BulkHost failed: ",
           Infoblox::status_code() . ":" . Infoblox::status_detail());
}
print "BulkHost object created successfully\n";

#Add the BulkHost object to Infoblox Appliance through a session
$session->add($bulk_host)
```

```
    or die("Add BulkHost object failed: ",
           $session->status_code() . ":" . $session->status_detail());
print "Bulk Host object added to server successfully\n";
```

#Search for a specific DNS BulkHost record

```
#Search for all BulkHost objects in the zone "domain.com"
my @retrieved_objs = $session->search(
    object => "Infoblox::DNS::BulkHost",
    zone   => "domain.com"
);
my $object = $retrieved_objs[0];

unless ($object) {
       die("Search Bulk Host object failed: ",
              $session->status_code() . ":" . $session->status_detail());
}
print "Search Bulk Host object found at least 1 matching entry\n";

#Search for all BulkHost objects that start with prefix as "prefix"
my @retrieved_objs = $session->search(
    object => "Infoblox::DNS::BulkHost",
    prefix => "prefix.*"
);
my $object = $retrieved_objs[0];

unless ($object) {
       die("Search Bulk Host object failed: ",
              $session->status_code() . ":" . $session->status_detail());
}
print "Search Bulk Host object found at least 1 matching entry\n";

#Search for all BulkHost objects with specific start_addr
my @retrieved_objs = $session->search(
    object => "Infoblox::DNS::BulkHost",
    start_addr => "1.0.0.1",
);
my $object = $retrieved_objs[0];

unless ($object) {
       die("Search Bulk Host object failed: ",
              $session->status_code() . ":" . $session->status_detail());
}
print "Search Bulk Host object found at least 1 matching entry\n";

#Search for all BulkHost objects with specific end_addr
my @retrieved_objs = $session->search(
    object => "Infoblox::DNS::BulkHost",
    end_addr => "1.0.0.5",
);
my $object = $retrieved_objs[0];

unless ($object) {
       die("Search Bulk Host object failed: ",
              $session->status_code() . ":" . $session->status_detail());
}
print "Search Bulk Host object found at least 1 matching entry\n";
```

#Get and modify a BulkHost object

```
#Get BulkHost object through the session
my @retrieved_objs = $session->get(
    object => "Infoblox::DNS::BulkHost",
    zone   => "domain.com",
);
my $object = $retrieved_objs[0];
unless ($object) {
    die("Get BulkHost object failed: ",
        $session->status_code() . ":" . $session->status_detail());
}
print "Get BulkHost object found at least 1 matching entry\n";

#Modify one of the attributes of the specified BulkHost object
$object->prefix("test");

#Apply the changes
$session->modify($object)
```

```
    or die("Modify BulkHost object failed: ",
           $session->status_code() . ":" . $session->status_detail());
print "BulkHost object modified successfully \n";
```

#Remove a BulkHost object

```
#Get BulkHost object through the session
my @retrieved_objs = $session->get(
    object     => "Infoblox::DNS::BulkHost",
    zone       => "domain.com",
    prefix     => "test",
    start_addr => "1.0.0.1",
    end_addr   => "1.0.0.5"
);
my $object = $retrieved_objs[0];
unless ($object) {
    die("Get BulkHost object failed: ",
        $session->status_code() . ":" . $session->status_detail());
}
print "Get BulkHost object found at least 1 matching entry\n";

#Submit the object for removal
$session->remove($object)
    or die("Remove  BulkHost object failed: ",
        $session->status_code() . ":" . $session->status_detail());
print "BulkHost object removed successfully \n";

####PROGRAM ENDS####
```

AUTHOR

Infoblox Inc. http://www.infoblox.com/

SEE ALSO

Infoblox::DNS::View, Infoblox::DNS::Zone, Infoblox::Session->add(), Infoblox::Session->get(), Infoblox::Session->modify(), Infoblox::Session->remove(), Infoblox::Session->search(),Infoblox::Session

---

COPYRIGHT

Copyright (c) 2017 Infoblox Inc.

Infoblox::DNS::DDNS::PrincipalCluster - A DDNS Principal Cluster object.

- NAME
- DESCRIPTION
- CONSTRUCTOR
- SESSION METHODS
  - Infoblox::Session->add( )
  - Infoblox::Session->get( )
  - Infoblox::Session->modify( )
  - Infoblox::Session->remove( )
  - Infoblox::Session->search( )
- MODULE METHODS
  - Infoblox::DNS::DDNS::PrincipalCluster::Group->clusters( )
- METHODS
  - comment( )
  - group( )
  - name( )
  - principals( )
- AUTHOR
- SEE ALSO
- COPYRIGHT

NAME

Infoblox::DNS::DDNS::PrincipalCluster - A DDNS Principal Cluster object.

DESCRIPTION

The DDNS Principal Cluster object represents a set of principals such that any principal in a DDNS Principal Cluster can update records created by any other principal in the same cluster.

CONSTRUCTOR

```
my $cluster = Infoblox::DNS::DDNS::PrincipalCluster->new(
    name       => $string,                    # Required
    group      => $string,                    # Required
    comment    => $string,                    # Optional / Default is undefined
    principals => [$string1, $string2, ...], # Optional / Default is undefined
);
```

SESSION METHODS

This section describes all the methods in a Session module that you can apply to a DDNS Principal Cluster object.

Infoblox::Session->add( )

Use this method to add an object to the Infoblox appliance. See Infoblox::Session->add() for parameters and return values.

Example

```
#create DDNS principal cluster group

my $cluster_group = Infoblox::DNS::DDNS::PrincipalCluster::Group->new(
    name => 'cluster_group',
);

#submit for addition
my $response = $session->add($cluster_group);

#construct an object
my $object = Infoblox::DNS::DDNS::PrincipalCluster->new(
   name  => 'cluster1',
   group => 'cluster_group',
);

#submit for addition
my $response = $session->add($object);
```

Infoblox::Session->get( )

Use this method to retrieve all the matching objects from the Infoblox appliance. See Infoblox::Session->get() for parameters and return values.

Key Reference

```
Apply the following attribute to get a specific object:

 name    - Optional. A DDNS Principal cluster name in a string format.
 comment - Optional. A DDNS Principal cluster comment.
 group   - Optional. A DDNS Principal cluster group name.
```

Example

```
my @retrieved_objs = $session->get(
    object => 'Infoblox::DNS::DDNS::PrincipalCluster',
    name   => 'cluster1',
    group  => 'cluster_group',
);
```

Infoblox::Session->modify( )

Use this method to modify an object in the Infoblox appliance. See Infoblox::Session->modify() for parameters and return values.

Example

```
#Use method to modify the comment.
$object->comment('this is a modified comment');
#Submit modification
my $response = $session->modify($object);
```

Infoblox::Session->remove( )

Use this method to remove an object from the Infoblox appliance. See Infoblox::Session->remove() for parameters and return values.

To remove a specific object, first use `get()` or `search()` to retrieve the specific object, and then submit this object for removal.

Example

```
#Get the objects with the same name
my @retrieved_objs = $session->get(
    object => 'Infoblox::DNS::DDNS::PrincipalCluster',
    name   => 'cluster1',
);
#Find the desired object from the retrieved list.
my $desired_cluster = $retrieved_objs[0];
#Submit for removal
my $response = $session->remove($desired_cluster);
```

Infoblox::Session->search( )

Use this method to search for objects in the Infoblox appliance. See Infoblox::Session->search() for parameters and return values.

Key Reference

```
Apply the following attribute to get a specific object:

 name    - Optional. A DDNS Principal Cluster name in a string format.
 comment - Optional. A DDNS Principal Cluster comment.
 group   - Optional. A DDNS Principal cluster group name.
```

Example

```
my @retrieved_objs = $session->get(
    object => 'Infoblox::DNS::DDNS::PrincipalCluster',
    name   => 'group11',
);
```

MODULE METHODS

Infoblox::DNS::DDNS::PrincipalCluster::Group->clusters( )

Use this method to retrieve a list of DDNS Principal Clusters. See Infoblox::DNS::DDNS::PrincipalCluster::Group->clusters() for parameters and return values.

Example

```
#get clusters value
```

```
my $clusters = $object->clusters();
```

METHODS

This section describes all the methods that you can use to set or retrieve the attribute values of the object.

comment( )

Use this method to set or retrieve a descriptive comment.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

The valid value is a comment in string format (UTF-8) with a maximum of 256 bytes.

Returns

If you specified a parameter, the method returns 'true' when the modification succeeds, and returns 'false' when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#get comment value
my $comment = $object->comment();

#modify comment value
$object->comment('desired comment');
```

group( )

Use this method to set or retrieve the DDNS Principal Cluster group the cluster associated with.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

The valid value is a DDNS Principal Cluster group name in string format (UTF-8) with a maximum of 256 bytes.

Returns

If you specified a parameter, the method returns 'true' when the modification succeeds, and returns 'false' when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#get group value
my $group = $object->group();

#modify group value
$object->group('desired_group');
```

name( )

Use this method to set or retrieve the DDNS Principal Cluster name.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

Desired name in string format with a maximum of 256 bytes.

Returns

If you specified a parameter, the method returns 'true' when the modification succeeds, and returns 'false' when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#get name value
my $name = $object->name();
```

```
#modify name value
$object->name('foo');
```

principals( )

Use this method to set or retrieve a list of equivalent principals.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

The valid value is an array of desired principal names in string format.

Returns

If you specified a parameter, the method returns 'true' when the modification succeeds, and returns 'false' when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#get principals value
my $principals = $object->principals();

#modify principals value
$object->principals(['foo', 'bar']);
```

AUTHOR

Infoblox Inc. http://www.infoblox.com/

SEE ALSO

Infoblox::Session, Infoblox::Session->add(), Infoblox::Session->get(), Infoblox::Session->modify(), Infoblox::Session->remove(), Infoblox::Session->search(), Infoblox::DNS::DDNS::PrincipalCluster::Group, Infoblox::DNS::DDNS::PrincipalCluster::Group->clusters().

COPYRIGHT

Copyright (c) 2017 Infoblox Inc.

Infoblox::DNS::DDNS::PrincipalCluster::Group - A DDNS Principal Cluster Group object.

- NAME
- DESCRIPTION
- CONSTRUCTOR
- SESSION METHODS
  - Infoblox::Session->add( )
  - Infoblox::Session->get( )
  - Infoblox::Session->modify( )
  - Infoblox::Session->remove( )
  - Infoblox::Session->search( )
- MODULE METHODS
  - Infoblox::Grid::DNS->ddns_principal_group( )
  - Infoblox::DNS::Zone->ddns_principal_group( )
  - Infoblox::DNS::View->ddns_principal_group( )
- METHODS
  - clusters( )
  - comment( )
  - name( )
- AUTHOR
- SEE ALSO
- COPYRIGHT

NAME

Infoblox::DNS::DDNS::PrincipalCluster::Group - A DDNS Principal Cluster Group object.

DESCRIPTION

The DDNS Principal Cluster Group object represents a set of DDNS Principal Clusters. A single group can be active at any time.

CONSTRUCTOR

```
my $cluster = Infoblox::DNS::DDNS::PrincipalCluster::Group->new(
    name     => $string,                    # Required
    comment  => $string,                    # Optional / Default is undefined
);
```

SESSION METHODS

This section describes all the methods in a Session module that you can apply to a DDNS Principal Cluster Group object.

Infoblox::Session->add( )

Use this method to add an object to the Infoblox appliance. See Infoblox::Session->add() for parameters and return values.

Example

```
#construct an object
my $object = Infoblox::DNS::DDNS::PrincipalCluster::Group->new(
   name  => 'cluster1',
);

#submit for addition
my $response = $session->add($object);
```

Infoblox::Session->get( )

Use this method to retrieve all the matching objects from the Infoblox appliance. See Infoblox::Session->get() for parameters and return values.

Key Reference

```
Apply the following attribute to get a specific object:

 name    - Optional. A DDNS Principal Cluster Group name in a string format.
 comment - Optional. A DDNS Principal Cluster Group comment.
```

Example

```
my @retrieved_objs = $session->get(
    object => 'Infoblox::DNS::DDNS::PrincipalCluster::Group',
    name   => 'cluster1',
);
```

Infoblox::Session->modify( )

Use this method to modify an object in the Infoblox appliance. See Infoblox::Session->modify() for parameters and return values.

Example

```
#Use method to modify the comment.
$object->comment('this is a modified comment');
#Submit modification
my $response = $session->modify($object);
```

Infoblox::Session->remove( )

Use this method to remove an object from the Infoblox appliance. See Infoblox::Session->remove() for parameters and return values.

To remove a specific object, first use `get()` or `search()` to retrieve the specific object, and then submit this object for removal.

Example

```
#Get the objects with the same name
my @retrieved_objs = $session->get(
    object => 'Infoblox::DNS::DDNS::PrincipalCluster::Group',
    name   => 'cluster1',
);
#Find the desired object from the retrieved list.
my $desired_cluster = $retrieved_objs[0];
#Submit for removal
my $response = $session->remove($desired_cluster);
```

Infoblox::Session->search( )

Use this method to search for objects in the Infoblox appliance. See Infoblox::Session->search() for parameters and return values.

Key Reference

```
Apply the following attribute to get a specific object:

 name    - Optional. A DDNS Principal Cluster Group name in a string format.
 comment - Optional. A DDNS Principal Cluster Group comment.
```

Example

```
my @retrieved_objs = $session->get(
    object => 'Infoblox::DNS::DDNS::PrincipalCluster::Group',
    name   => 'group1',
);
```

MODULE METHODS

Infoblox::Grid::DNS->ddns_principal_group( )

Use this method to set or retrieve the DDNS Principal Cluster Group. See Infoblox::Grid::DNS->ddns_principal_group() for parameters and return values.

Example

```
#get ddns_principal_group value
my $ddns_principal_group = $object->ddns_principal_group();

#modify ddns_principal_group value
$object->ddns_principal_group($group1);
```

Infoblox::DNS::Zone->ddns_principal_group( )

Use this method to set or retrieve the DDNS Principal Cluster Group. See Infoblox::DNS::Zone->ddns_principal_group() for parameters and return values.

Example

```
#get ddns_principal_group value
my $ddns_principal_group = $object->ddns_principal_group();
```

```
#modify ddns_principal_group value
$object->ddns_principal_group($group1);
```

Infoblox::DNS::View->ddns_principal_group( )

Use this method to set or retrieve the DDNS Principal Cluster Group. See Infoblox::DNS::View->ddns_principal_group() for parameters and return values.

Example

```
#get ddns_principal_group value
my $ddns_principal_group = $object->ddns_principal_group();

#modify ddns_principal_group value
$object->ddns_principal_group($group1);
```

METHODS

This section describes all the methods that you can use to set or retrieve the attribute values of the object.

clusters( )

Use this method to retrieve a list of equivalent DDNS principal clusters. This is a read-only attribute.

Omit the parameter to retrieve the attribute value.

Parameter

None

Returns

The valid return value is array of an Infoblox::DNS::DDNS::PrincipalCluster objects.

Example

```
#get clusters value
my $clusters = $object->clusters();
```

comment( )

Use this method to set or retrieve a descriptive comment.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

The valid value is a comment in string format (UTF-8) with a maximum of 256 bytes.

Returns

If you specified a parameter, the method returns 'true' when the modification succeeds, and returns 'false' when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#get comment value
my $comment = $object->comment();

#modify comment value
$object->comment('desired comment');
```

name( )

Use this method to set or retrieve the DDNS Principal Cluster Group name.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

Desired name in string format with a maximum of 256 bytes.

Returns

If you specified a parameter, the method returns 'true' when the modification succeeds, and returns 'false' when the operation

fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#get name value
my $name = $object->name();

#modify name value
$object->name('foo');
```

AUTHOR

Infoblox Inc. http://www.infoblox.com/

SEE ALSO

Infoblox::Session, Infoblox::Session->add(), Infoblox::Session->get(), Infoblox::Session->modify(), Infoblox::Session->remove(), Infoblox::Session->search(), Infoblox::Grid::DNS, Infoblox::Grid::DNS->ddns_principal_group(), Infoblox::DNS::Zone, Infoblox::DNS::Zone->ddns_principal_group(), Infoblox::DNS::View, Infoblox::DNS::View->ddns_principal_group(),

---

COPYRIGHT

Copyright (c) 2017 Infoblox Inc.

Infoblox::DNS::DNSSecKey - DNSSEC key object.

- NAME
- DESCRIPTION
- SESSION METHODS
- METHODS
  - algorithm( )
  - next_event_date( )
  - public_key( )
  - status( )
  - tag( )
  - type( )
- AUTHOR
- SEE ALSO
- COPYRIGHT

NAME

Infoblox::DNS::DNSSecKey - DNSSEC key object.

DESCRIPTION

The DNSSEC object contains information about keys available in authoritative zones.

SESSION METHODS

The object does not support any session methods.

METHODS

This section describes all the methods that you can use to set or retrieve the attribute values of the object.

algorithm( )

Use this method to retrieve the public-key encryption algorithm. This is a read-only attribute.

Parameter

None

Returns

The return value is the public-key encryption algorithm in string format.

Example

```
#Get algorithm value
my $value = $dnssec_key->algorithm();
```

next_event_date( )

Use this method to retrieve the next event date. The event date can be a date for the rollover of an ACTIVE key or a PUBLISHED key, or the removal of a ROLLED key. This is a read-only attribute.

Parameter

None

Returns

The date of next event in ISO 8601 extended format (e.g., 2008-10-21T10:56:00Z or 2008-10-21T10:56:00.001Z).

Example

```
#Get next_event_date value
my $value = $dnssec_key->next_event_date();
```

public_key( )

Use this method to retrieve the Base-64 encoding of the public key. This is a read-only attribute.

Parameter

None

Returns

The return value is a Base-64 encoded public key in string format.

Example

```
#Get public_key value
my $value = $dnssec_key->public_key();
```

status( )

Use this method to retrieve key status for the authoritative zone. This is a read-only attribute.

Parameter

None

Returns

The valid return values are "ACTIVE", "PUBLISHED" and "ROLLED".

Example

```
#Get status value
my $value = $dnssec_key->status();
```

tag( )

Use this method to retrieve the key tag for the authoritative zone. This is a read-only attribute.

Parameter

None

Returns

The return value is an integer between 0 and 65535.

Example

```
#Get tag value
my $value = $dnssec_key->tag();
```

type( )

Use this method to retrieve the type of key. This is a read-only attribute.

Parameter

None

Returns

The valid return values are "KSK" and "ZSK".

Example

```
#Get type value
my $value = $dnssec_key->type();
```

AUTHOR

Infoblox Inc. http://www.infoblox.com/

SEE ALSO

ISO 8601 extended format

COPYRIGHT

Copyright (c) 2017 Infoblox Inc.

Infoblox::DNS::DNSSecKey - DNSSEC key object.

Infoblox::DNS::DNSSecKeyAlgorithm - DNSSEC key algorithm object.

- NAME
- DESCRIPTION
- CONSTRUCTOR
- MODULE METHODS
  - Infoblox::Grid::DNS->dnssec_ksk_algorithms( )
  - Infoblox::Grid::DNS->dnssec_zsk_algorithms( )
  - Infoblox::Grid::Member::DNS->dnssec_ksk_algorithms( )
  - Infoblox::Grid::Member::DNS->dnssec_zsk_algorithms( )
- SESSION METHODS
- METHODS
  - algorithm( )
  - size( )
- AUTHOR
- SEE ALSO
- COPYRIGHT

NAME

Infoblox::DNS::DNSSecKeyAlgorithm - DNSSEC key algorithm object.

DESCRIPTION

The DNSSEC key algorithm structure provides for configuring algorithms for Key-signing and Zone-signing keys. Multiple algorithms can be used at the same time per Grid or per zone.

CONSTRUCTOR

```
 #Constructor for an DNSSEC key algorithm object
  my $algorithm = Infoblox::DNS::DNSSecKeyAlgorithm->new(
     algorithm => "RSAMD5" | "DSA" | "RSASHA1" | "RSASHA256" | "RSASHA512" ,  #Optional / Default is
undefined
     size      => $num,                                                       #Optional / Default is
undefined
   );
```

MODULE METHODS

The following functions are available to be applied to an DNSSecKeyAlgorithm object.

Infoblox::Grid::DNS->dnssec_ksk_algorithms( )

Use this function to specify a DNSSEC key signing key algorithm at the grid level on the Infoblox appliance. See Infoblox::Grid::DNS->dnssec_ksk_algorithms() for parameters and return values.

Example

```
#Create a DNSSEC key algorithm object.
my $algorithm = Infoblox::DNS::DNSSecKeyAlgorithm->new(
    algorithm => 'RSASHA256',
    size      => 512,
);

#Configure DNSSEC key signing key algorithms on the Infoblox grid object
my $response = $grid->dnssec_ksk_algorithms([$algorithm]);
```

Infoblox::Grid::DNS->dnssec_zsk_algorithms( )

Use this function to specify a DNSSEC zone signing key algorithm at the grid level on the Infoblox appliance. See Infoblox::Grid::DNS->dnssec_zsk_algorithms() for parameters and return values.

Example

```
#Create a DNSSEC key algorithm object.
my $algorithm = Infoblox::DNS::DNSSecKeyAlgorithm->new(
    algorithm => 'RSASHA512',
    size      => 1024,
);

#Configure DNSSEC zone signing key algorithms on the Infoblox grid object
```

```
my $response = $grid->dnssec_zsk_algorithms([$algorithm]);
```

Infoblox::Grid::Member::DNS->dnssec_ksk_algorithms( )

Use this function to specify a DNSSEC key signing key at the member level on the Infoblox appliance. See Infoblox::Grid::Member::DNS->dnssec_ksk_algorithms() for parameters and return values.

Example

```
#Create a DNSSEC key algorithm object.
my $algorithm = Infoblox::DNS::DNSSecKeyAlgorithm->new(
    algorithm => 'RSASHA512',
    size      => 1024,
);

#Configure DNSSEC key signing key algorithms on the Infoblox member object
my $response = $member->dnssec_ksk_algorithms([$algorithm]);
```

Infoblox::Grid::Member::DNS->dnssec_zsk_algorithms( )

Use this function to specify a DNSSEC zone signing key at the member level on the Infoblox appliance. See Infoblox::Grid::Member::DNS->dnssec_zsk_algorithms() for parameters and return values.

Example

```
#Create a DNSSEC key algorithm object.
my $algorithm = Infoblox::DNS::DNSSecKeyAlgorithm->new(
    algorithm => 'RSASHA512',
    size      => 1024,
);

#Configure DNSSEC zone signing key algorithms on the Infoblox member object
my $response = $member->dnssec_zsk_algorithms([$algorithm]);
```

SESSION METHODS

The object does not support any session methods.

METHODS

this section describes all the methods that you can use to set or retrieve the attribute values of the object.

algorithm( )

Use this method to set or retrieve signing key algorithm.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

Valid value is 'RSAMD5', 'DSA', 'RSASHA1', 'RSASHA256' or 'RSASHA512'.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get algorithm value
my $algorithm = $dnssec_key_algorithm->algorithm();
#Modify algorithm value
$dnssec_key_algorithm->algorithm('RSASHA512');
```

size( )

Use this method to set or retrieve signing key size, in bits.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

The DNSSEC key-signing key possible size depends on the algorithm. Use the following table to determine the possible values for key sizes:

```
  Algorithm    Key size     Additional restrictions
NSEC
  RSA/MD5      512-4096
  DSA/SHA1     512-1024     Key size must be divisible by 64
  RSA/SHA1     512-4096
  RSA256/SHA2  512-4096
  RSA512/SHA2  1024-4096
NSEC3
  DSA/SHA1     512-1024     Key size must be divisible by 64
  RSA/SHA1     512-4096
  RSA256/SHA2  512-4096
  RSA512/SHA2  1024-4096
```

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get size value
my $size = $dnssec_key_algorithm->size();
#Modify size value
$dnssec_key_algorithm->size(1024);
```

AUTHOR

Infoblox Inc. http://www.infoblox.com/

SEE ALSO

Infoblox::Grid::DNS, Infoblox::Grid::Member::DNS, Infoblox::Grid::DNS->dnssec_ksk_algorithms(), Infoblox::Grid::DNS->dnssec_zsk_algorithms(), Infoblox::Grid::Member::DNS->dnssec_ksk_algorithms(), Infoblox::Grid::Member::DNS->dnssec_zsk_algorithms()

COPYRIGHT

Copyright (c) 2017 Infoblox Inc.

Infoblox::DNS::DnssecTrustedKey - DNSSEC trusted key object.

- NAME
- DESCRIPTION
- CONSTRUCTOR
- MODULE METHODS
  - Infoblox::Grid::DNS->dnssec_trusted_keys( )
  - Infoblox::Grid::Member::DNS->dnssec_trusted_keys( )
  - Infoblox::DNS::View->dnssec_trusted_keys( )
- METHODS
  - algorithm( )
  - fqdn( )
  - key( )
  - secure_entry_point()
- SAMPLE CODE
- AUTHOR
- SEE ALSO
- COPYRIGHT

NAME

Infoblox::DNS::DnssecTrustedKey - DNSSEC trusted key object.

DESCRIPTION

For recursively derived signed data, the DNS server can perform validation. In order to be secure, this validation relies on a set of trust anchors. These trust anchors and their associated trusted keys can be specified via the Infoblox::DNS::DnssecTrustedKey objects.

DNSSEC is based on DNS specifications RFC 4033, RFC 4034, and RFC 4035, which provide authentication protocol enhancements to secure DNS transactions.

CONSTRUCTOR

```
 my $dnssec_trusted_key=Infoblox::DNS::DnssecTrustedKey->new(
                    algorithm             =>
"RSAMD5"|"DSA"|"RSASHA1"|"NSEC3DSA"|"NSEC3RSASHA1"|"RSASHA256"|"RSASHA512"|"NSEC3RSASHA256"|"NSEC3RSASHA512"|1|3|5|6|7,
  #Required
                    fqdn                  => $string,                                  #Required
                    key                   => $string,                                  #Required
                    secure_entry_point    => "true"|"false"                            #Optional / Default is "true"
                        );
```

MODULE METHODS

This section describes all the functions that you can apply to a DnssecTrustedKey object.

Infoblox::Grid::DNS->dnssec_trusted_keys( )

Use this function to specify a list of DNSSEC keys for a grid object.

Example

```
      ## Construct the trusted key object. When trying to implement this example replace with the actual public key
     of the signed zone.
      my $dnssec_key = Infoblox::DNS::DnssecTrustedKey->new(
          algorithm => "RSAMD5",
          fqdn => "example.com",
          key  => "Zx1jVJl7C58GT/sc7Q3ucA==",
          secure_entry_point => "false"
      );
      # Configure dnssec_trusted_keys on the grid DNS object
      my $response = $Grid_DNS->dnssec_trusted_keys([$dnssec_key]);
```

Infoblox::Grid::Member::DNS->dnssec_trusted_keys( )

Use this function to specify a list of DNSSEC keys for a grid member object.

Example

```
      ## Construct the trusted key object
      my $dnssec_key = Infoblox::DNS::DnssecTrustedKey->new(
          algorithm => "RSAMD5",
          fqdn => "example.com",
          key  => "Zx1jVJl7C58GT/sc7Q3ucA==",
          secure_entry_point => "false"
      );
      # Configure dnssec_trusted_keys on the grid DNS object
      my $response = $Grid Member DNS->dnssec trusted keys([$dnssec key]);
```

Infoblox::DNS::View->dnssec_trusted_keys( )

Use this function to specify a list of DNSSEC keys for a DNS View object.

Example

```
## Construct the trusted key object
my $dnssec_key = Infoblox::DNS::DnssecTrustedKey->new(
    algorithm => "RSAMD5",
    fqdn => "example.com",
    key  => "Zx1jVJl7C58GT/sc7Q3ucA==",
    secure_entry_point => "false"
);
# Configure dnssec_trusted_keys on the grid DNS object
my $response = $DNS_View->dnssec_trusted_keys([$dnssec_key]);
```

METHODS

algorithm( )

Use this method to set or retrieve a DNSSEC encryption algorithm.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

The public-key encryption algorithm. This can be either an integer or mnemonic. The following table shows the correlation between the integers and mnemonics for the algorithms supported by the appliance:

Value	Algorithm	Mnemonic
1	RSA/MD5	RSAMD5
3	DSA	DSA
5	RSA/SHA-1	RSASHA1
6	NSEC3DSA	NSEC3DSA
7	NSEC3RSA/SHA-1	NSEC3RSASHA1

For algorithms not listed above, only the mnemonic representation is supported.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
# Getting algorithm
my $algorithm = $dnssec_key->algorithm();
# Modifying algorithm
$dnssec_key->algorithm("DSA");
```

fqdn( )

Use this method to set or retrieve the FQDN of the zone to which the key belongs.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

FQDN of the zone to which the trusted_key pertains, in DNS label order.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
# Getting fqdn
my $zone = $dnssec_key->fqdn();
# Modifying algorithm
$dnssec_key->fqdn("zone.example.com");
```

key( )

Use this method to add or retrieve the DNSSEC public key for the zone.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

Base-64 encoding of the key.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
# Getting key
my $key = $dnssec_key->key();
# Modifying key
$dnssec_key->key("AQPdWbrGbVv1eDhNgRhpJMPonJfA3reyEo82ekwRnjbX7+uBxB11BqL7LAB7/C+eb0vCtI53FwMhkkNkTmA6bI8B");
```

secure_entry_point()

Use this method to set or retrieve the secure entry point value for the field.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

Set to "true" to enable the secure entry point, set to "false" to disable it. Default value of this parameter is "true".

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
# Getting secure_entry_point
my $sep = $dnssec_key->secure_entry_point();
# Modifying secure_entry_point
$dnssec_key->secure_entry_point("false");
```

SAMPLE CODE

The following sample code demonstrates the different functions that can be applied to an object, such as add and modify. This sample also includes error handling for the operations.

##Preparation prior to a DnssecTrustedKey object insertion

```
#PROGRAM STARTS: Include all the modules that will be used
use strict;
use Infoblox;

#Create a session to the Infoblox  appliance
my $session = Infoblox::Session->new(
    master   => "192.168.1.2",
    username => "admin",
    password => "infoblox"
);
unless ($session) {
   die("Construct session failed: ",
       Infoblox::status_code() . ":" . Infoblox::status_detail());
}
print "Session created successfully\n";
```

#Create a DnssecTrustedKey object

```
 my $dnssec_key = Infoblox::DNS::DnssecTrustedKey->new(
    key  => "ZX1jVJl7C58GT/sc7Q3ucA==",
    fqdn => "myzone.example.com",
    algorithm => "RSASHA1"
  );

unless($dnssec_key) {
     die("Construct DnssecTrustedKey failed: ",
           Infoblox::status_code() . ":" . Infoblox::status_detail());
}
print "DnssecTrustedKey object created successfully\n";
```

#Get the member DNS object and add a DnssecTrustedKey object to it

```
my @result_array = $session->get(
    object => "Infoblox::Grid::Member::DNS",
    name   => "infoblox.localdomain"
);

unless (scalar(@result_array) == 0) {
    my $memberdns = $result_array[0];

    if ($memberdns) {

        #Apply changes to the Member object.
```

```
                $memberdns->dnssec_trusted_keys([$dnssec_key])
            or die("modify member failed: ",
                   Infoblox::status_code() . ":" . Infoblox::status_detail());
        print "DnssecTrustedKey added to Member DNS object successfully\n";

  #Update member DNS object through the Infoblox session.
        $session->modify($memberdns)
            or die("modify session failed: ",
                   $session->status_code() . ":" . $session->status_detail());
        print "Member DNS object with DnssecTrustedKey updated to Infoblox device successfully\n";

         #Modify existing DnssecTrustedKey object

        #Modifying the value of the specified object.
        $dnssec_key->fqdn("another.example.com");
        print "Modify DnssecTrustedKey fqdn value\n";

        #Apply changes to the member object.
        $memberdns->transfer_keys([$dnssec_key])
            or die("modify member failed: ",
                   Infoblox::status_code() . ":" . Infoblox::status_detail());
        print "DnssecTrustedKey fqdn updated to Member DNS object successfully\n";

        #Update member DNS object through the Infoblox session.
        $session->modify($memberdns)
            or die("modify session failed: ",
                $session->status_code() . ":" . $session->status_detail());
                print "Member DNS object with DnssecTrustedKey fqdn updated to Infoblox device successfully\n";
             }
         } else {
             print "No member found with the specified name.";
         }

 ####PROGRAM ENDS####
```

AUTHOR

Infoblox Inc. http://www.infoblox.com/

SEE ALSO

Infoblox::Grid::Member::DNS, Infoblox::DNS::View, Infoblox::Grid::DNS

COPYRIGHT

Copyright (c) 2017 Infoblox Inc.

Infoblox::DNS::FireEyeAlertMap - FireEye Alert Map object

- NAME
- DESCRIPTION
- CONSTRUCTOR
- MODULE METHODS
  - Infoblox::DNS::FireEye::RuleMapping->fireeye_alert_mapping( )
- METHODS
  - alert_type( )
  - rpz_rule( )
  - lifetime( )
- AUTHOR
- SEE ALSO
- COPYRIGHT

NAME

Infoblox::DNS::FireEyeAlertMap - FireEye Alert Map object

DESCRIPTION

The alert map object is used to configure FireEye alerts to Response Policy Zone rules.

CONSTRUCTOR

```
 my $alertmap = Infoblox::DNS::FireEye::AlertMap->new(
     alert_type => "INFECTION_MATCH" | "WEB_INFECTION" | "MALWARE_OBJECT" | "DOMAIN_MATCH" |
"MALWARE_CALLBACK", # Required
     rpz_rule   => "PASSTHRU" | "NXDOMAIN" | "NODATA" | "SUBSTITUTE" | "NONE", # Required
     lifetime   => $uint, # Required
 );
```

MODULE METHODS

The following functions are available to apply to a FireEye Alert Map object.

Infoblox::DNS::FireEye::RuleMapping->fireeye_alert_mapping( )

Use this function to specify the alert map for this rule mapping. See Infoblox::DNS::FireEye::RuleMapping->fireeye_alert_mapping() for parameters, return values and code samples.

METHODS

This section describes all the methods that you can use to set and retrieve the attribute values of a DNS Member object.

alert_type( )

Use this method to set or retrieve the alert type.

Parameter

The valid value is one of "INFECTION_MATCH", "WEB_INFECTION", "MALWARE_OBJECT", "DOMAIN_MATCH" or "MALWARE_CALLBACK".

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get alert_type
my $alert_type = $am->alert_type();
#Modify alert_type
$am->alert_type("INFECTION_MATCH");
```

rpz_rule( )

Use this method to set or retrieve the rpz rule.

Parameter

The valid value is one of "PASSTHRU", "NXDOMAIN", "NODATA", "SUBSTITUTE" or "NONE".

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get rpz_rule
my $rpz_rule = $am->rpz_rule();
#Modify rpz_rule
$am->rpz_rule("PASSTHRU");
```

lifetime( )

Use this method to set or retrieve the expiration lifetime for this alert type.

Parameter

The valid value is an unsigned integer, which specifies the amount of seconds this alert type will live for. 0 means the alert will never expire.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get lifetime
my $lifetime = $am->lifetime();
#Modify lifetime
$am->lifetime(1234);
```

AUTHOR

Infoblox Inc. http://www.infoblox.com/

SEE ALSO

Infoblox::DNS::Zone, Infoblox::DNS::Zone

COPYRIGHT

Copyright (c) 2017 Infoblox Inc.

Infoblox::DNS::FireEyeRuleMapping - FireEye Rule Mapping object

- NAME
- DESCRIPTION
- CONSTRUCTOR
- MODULE METHODS
  - Infoblox::DNS::Zone->fireeye_rule_mapping( )
  - apt_override( )
  - fireeye_alert_mapping( )
  - substituted_domain_name( )
- AUTHOR
- SEE ALSO
- COPYRIGHT

NAME

Infoblox::DNS::FireEyeRuleMapping - FireEye Rule Mapping object

---

DESCRIPTION

The rule map object is used to configure rule mapping for FireEye devices and Response Policy Zones.

CONSTRUCTOR

```
my $rulemapping = Infoblox::DNS::FireEye::Rulemapping->new(
    fireeye_alert_mapping => [$am1, am2, ...], #Optional
    apt_override => "PASSTHRU" | "NXDOMAIN" | "NODATA" | "SUBSTITUTE" | "NOOVERRIDE", #Optional
    substituted_domain_name => $fqdn, #Optional
);
```

MODULE METHODS

The following functions are available to apply to a FireEye Alert Map object.

Infoblox::DNS::Zone->fireeye_rule_mapping( )

Use this function to specify the alert map for this rule mapping. See Infoblox::DNS::Zone->fireeye_rule_mapping() for parameters and return values.

Example

```
my $am1 = Infoblox::DNS::FireEye::AlertMap->new(
                                                alert_type => 'INFECTION_MATCH',
                                                rpz_rule => 'PASSTHRU',
                                                lifetime => 0,
                                               );

unless($am1) {
     die("Construct member failed: ",
           Infoblox::status_code() . ":" . Infoblox::status_detail());
}

my $am2 = Infoblox::DNS::FireEye::AlertMap->new(
                                                alert_type => 'WEB_INFECTION',
                                                rpz_rule => 'NONE',
                                                lifetime => 1234,
                                               );

unless($am2) {
     die("Construct member failed: ",
           Infoblox::status_code() . ":" . Infoblox::status_detail());
}

my $rm = Infoblox::DNS::FireEye::RuleMapping->new(
                                                  fireeye_alert_mapping => [$am1, $am2],
                                                  apt_override => 'SUBSTITUTE',
                                                  substituted_domain_name => 'somedomain.com'
                                                 );
$rpz_zone->rpz_type('FIREEYE');
$rpz_zone->fireeye_rule_mapping($rm);
```

```
my $response = $session->modify($rpz_zone)
unless($response) {
     die("Modify zone failed: ",
           session->status_code() . ":" . session->status_detail());
}
print "Zone modified successfully\n";
```

apt_override( )

Use this method to set or retrieve the APT override.

Parameter

The valid value is one of "PASSTHRU", "NXDOMAIN", "NODATA", "SUBSTITUTE" or "NOOVERRIDE".

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get apt_override
my $apt_override = $rm->apt_override();
#Modify apt_override
$rm->apt_override("NOOVERRIDE");
```

fireeye_alert_mapping( )

Use this method to set or retrieve the FireEye alert mapping.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Returns

An array of the Infoblox::DNS::FireEye::AlertMap manpage objects.

Example

```
#Get fireeye_alert_mapping
my $fireeye_alert_mapping = $zone->fireeye_alert_mapping();
#Modify fireeye_alert_mapping
$rm->fireeye_alert_mapping([$am1, $am2]);
```

substituted_domain_name( )

Use this method to set or retrieve the domain name to be substituted, this is applicable only when apt_override is set to "SUBSTITUTE".

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

The domain name to use.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get substituted_domain_name
my $substituted_domain_name = $rm->substituted_domain_name();
#Modify substituted_domain_name
$rm->substituted_domain_name("somedomain.com");
```

AUTHOR

Infoblox Inc. http://www.infoblox.com/

SEE ALSO

Infoblox::DNS::Zone, Infoblox::DNS::Zone

COPYRIGHT

Copyright (c) 2017 Infoblox Inc.

Infoblox::DNS::GlueRecordAddr - Infoblox DNS Glue Record Object.

- NAME
- DESCRIPTION
- CONSTRUCTOR
- MODULE METHODS
  - Infoblox::Grid::Member::DNS->glue_record_addresses( )
- METHODS
  - attach_empty_recursive_view( )
  - glue_record_address( )
  - view( )
- SAMPLE CODE
- AUTHOR
- SEE ALSO
- COPYRIGHT

NAME

Infoblox::DNS::GlueRecordAddr - Infoblox DNS Glue Record Object.

DESCRIPTION

The GlueRecordAddr allows you to specify which interface address the appliance uses to generate the glue record for each view.

CONSTRUCTOR

The Infoblox::DNS::GlueRecordAddr object is automatically generated upon the successful creation of the Infoblox::DNS::View object. It does not require manual construction.

MODULE METHODS

This section describes all the functions that you can apply to a GlueRecordAddr object.

Infoblox::Grid::Member::DNS->glue_record_addresses( )

Use this function to specify a glue record address for a view at the grid member level. See Infoblox::Grid::Member::DNS->glue_record_addresses() for parameters and return values.

Example

```
# Retrieve the list of GlueRecordAddr objects from Member DNS object.
my @retrieved_objs = $session->get(
    object => "Infoblox::Grid::Member::DNS",
    name   => "infoblox.localdomain" );

#select the desired GlueRecordAddr for the view
my $Grid_Member_DNS = $retrieved_objs[0];
my $ref_all_glue_record_addr = $Grid_Member_DNS->glue_record_addresses();
my $glue_record_addr = @$ref_all_glue_record_addr[0];

# Modify Glue Record Address for the view to use NAT address.
$glue_record_addr->glue_record_address("NAT");

#Save modification to Member DNS object.
my $response = $Grid_Member_DNS->glue_record_addresses($ref_all_glue_record_addr);
```

METHODS

This section describes all the methods that you can use to configure and retrieve the attribute values of a GlueRecordAddr object.

attach_empty_recursive_view( )

Use this method to set or retrieve the attach_empty_recursive_view flag of glue record object. If the attach_empty_recursive_view flag is set to "true", then empty view with recursion enabled will be written into the conf file.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

Specify "true" to set the attach_empty_recursive_view flag or "false" to deactivate/unset it.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get attach_empty_recursive_view.
my $attach_empty_recursive_view = $glue_record_addr->attach_empty_recursive_view();
#Modify attach_empty_recursive_view.
$glue_record_addr->attach_empty_recursive_view("true");
```

glue_record_address( )

Use this method to set or retrieve the address the appliance uses to generate the glue record.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

String which specifies the interface address to use. Valid values are "interface", "NAT", or specify an IPv4 or IPv6 address.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get glue record address.
my $glue_record_address = $glue_record_addr->glue_record_address();
#Modify glue record address.
$glue_record_addr->glue_record_address("NAT");
```

view( )

Use this method to retrieve the view of this Glue Record object

Parameter

None.

Returns

The method returns the attribute value.

Example

```
#Get view
my $glue_record_view = $glue_record_addr->view();
```

SAMPLE CODE

The following sample code demonstrates the different functions that can be applied to an object, such as get and modify. This sample also includes error handling for the operations.

#Preparation prior to a DNS GlueRecordAddr object modification

```
#PROGRAM STARTS: Include all the modules that will be used
use strict;
use Infoblox;

#Create a session to the Infoblox appliance

my $session = Infoblox::Session->new(
           master   => "192.168.1.2",
           username => "admin",
           password => "infoblox"
);

unless ($session) {
   die("Construct session failed: ",
        Infoblox::status_code() . ":" . Infoblox::status_detail());
}
print "Session created successfully\n";
```

```
# Retrieve the list of GlueRecordAddr objects from Member DNS object.
my @retrieved_objs = $session->get(
    object => "Infoblox::Grid::Member::DNS",
    name   => "infoblox.localdomain" );

my $Grid_Member_DNS = $retrieved_objs[0];
unless ($Grid_Member_DNS) {
   die("Retrieve Grid Member DNS object failed: ",
       $session->status_code() . ":" . $session->status_detail());
}
print "Grid Member DNS object retrieve successfully\n";
```

#Modify the Glue Record address

```
my $ref_all_glue_record_addr = $Grid_Member_DNS->glue_record_addresses();

# Select the desired GlueRecordAddr for the view
my $glue_record_addr = @$ref_all_glue_record_addr[0];
unless ($glue_record_addr) {
   die("Glue Record Address not found on Grid Member DNS object: ",
       $session->status_code() . ":" . $session->status_detail());
}
print "Glue Record Address found on Grid Member DNS object\n";

# Modify Glue Record Address for the view to use a custom defined address.
$glue_record_addr->glue_record_address("interface")
    or die("modify glue record failed: ",
        Infoblox::status_code() . ":" . Infoblox::status_detail());
print "Glue Record Address modified successfully\n";

# Save modification to Member DNS object.
my $response = $Grid_Member_DNS->glue_record_addresses($ref_all_glue_record_addr);
unless($response) {
    die("Modify glue record on Member DNS failed: ",
        Infoblox::status_code() . ":" . Infoblox::status_detail());
}
print "Glue Record Address updated to Grid Member DNS object successfully\n";

#submit changes to Infoblox appliance
$session->modify($Grid_Member_DNS)
   or die("Modify Member DNS object on Infoblox appliance failed: ",
        Infoblox::status_code() . ":" . Infoblox::status_detail());
print "Grid Member DNS modified on Infoblox appliance successfully\n";

####PROGRAM ENDS####
```

AUTHOR

Infoblox Inc. http://www.infoblox.com/

SEE ALSO

Infoblox::Session->get(), Infoblox::Session->modify(),Infoblox::Grid::Member::DNS->glue_record_addresses()

COPYRIGHT

Copyright (c) 2017 Infoblox Inc.

Infoblox::DNS::Host - DNS Host record object.

- NAME
- DESCRIPTION
- CONSTRUCTOR
- SESSION METHODS
  - Infoblox::Session->add( )
  - Infoblox::Session->get( )
  - Infoblox::Session->modify( )
  - Infoblox::Session->remove( )
  - Infoblox::Session->search( )
- METHODS
  - aliases( )
  - allow_telnet( )
  - cloud_info( )
  - comment( )
  - configure_for_dns( )
  - cli_credentials( )
  - ddns_protected( )
  - device_description( )
  - device_location( )
  - device_type( )
  - device_vendor( )
  - disable( )
  - dns_aliases( )
  - dns_name( )
  - extattrs( )
  - extensible_attributes( )
  - ipv4addrs( )
  - ipv6addrs( )
  - last_queried( )
  - ms_ad_user_data( )
  - name( )
  - network_view( )
  - override_cli_credentials( )
  - rrset_order( )
  - ttl( )
  - views( )
  - zone( )
- SAMPLE CODE
- AUTHOR
- SEE ALSO
- COPYRIGHT

NAME

Infoblox::DNS::Host - DNS Host record object.

DESCRIPTION

A host record defines attributes for a node, such as the name-to-address and address-to-name mapping. This alleviates having to specify an A record and a PTR record separately for the same node. A host can also define aliases and DHCP fixed address nodes. The zone must be created first before adding a host record for the zone.

Note that you must specify only one view for the attribute "views".

CONSTRUCTOR

```
 my $host = Infoblox::DNS::Host->new(
     ipv4addrs               => [$ipv4addr, $ipv4addr, $FixedAddr, ...], #Required
     name                    => $fqdn,                                   #Required
     aliases                 => [$fqdn, $fqdn, ...],                     #Optional / Default is empty
     comment                 => $string,                                 #Optional / Default is undefined
     configure_for_dns       => "true" | "false",                        #Optional / Default "true"
     disable                 => "true" | "false",                                #Optional / Default
"false"
     extattrs                => { $string => $extattr, ... },            #Optional / Default is undefined
     extensible_attributes   => { $string => $string | $num, $string => [ $string | $num, ... ], ... }
#Optional / Default is undefined
     ipv6addrs               => [$ipv6addr, $ipv6addr, ...],             #Optional / Default is empty
```

```
     network_view             => $NetworkView,                          #Optional / Default is "default"
network view
     rrset_order              => "cyclic" | "random" | "fixed",         #Optional / Default "cyclic"
     ttl                      => $num | undef,                              #Optional / Default is
undefined
     views                    => [$View]                                #Optional / Default is "default"
view
     cli_credentials          => [$cli1, ...],                          #Optional
     override_cli_credentials => "true" | "false",                      #Optional / Default is "false"
     device_type              => $string,                               #Optional
     device_vendor            => $string,                               #Optional
     device_location          => $string,                               #Optional
     device_description       => $string,                               #Optional
     allow_telnet             => "true" | "false",                      #Optional / Default is "false"
     ddns_protected           => "true" | "false",                      #Optional / Default is "false"
 );
```

You cannot set both extattrs and extensible_attributes attributes at the same time.

SESSION METHODS

This section describes all the methods in the Infoblox::Session module that you can apply to a DNS host record object.

Infoblox::Session->add( )

Use this method to add a host record object to the Infoblox appliance. See Infoblox::Session->add() for parameters and return values.

Example

```
#Construct an object
my $host = Infoblox::DNS::Host->new(
    name         => "host1.test.com",
    aliases      => ["one.foo.com", "two.foo.com"],
    comment      => "add a host host1.test.com",
    disable      => "true",
    ipv4addrs    => ["4.4.4.4"],
    ipv6addrs    => ["2001::1:2:3:4"],
    newtork_view => $nview,
    ttl          => 3600,
    views        => [$view1],
);

# Submit for addition
my $response = $session->add( $host );
```

Infoblox::Session->get( )

Use this method to retrieve all the matching objects from the Infoblox appliance. See Infoblox::Session->get() for parameters and return values.

Key References

```
Apply the following attributes to get a specific DNS host object:

  name          - Optional. A host name in string format.
  network_view  - Optional. The network view in which the host record is located. By default, all
network views are searched.
  view          - Optional. The DNS view in which the host record is located. By default, all DNS
views are searched. But if you omit this attribute and specify a zone, the appliance searches the
'default' view only.
  zone          - Optional. A zone name in FQDN format.
  comment       - Optional. A comment in string format (regular expression)
  mac           - Optional. A MAC address present in any FixedAddr object in the ipv4addrs member.
  ipv4addr      - Optional. An IP address in string format (regular expression).
  ipv6addr      - Optional. An IPv6 address in string format (regular expression).
  alias         - Optional. An alias of the host in string format (regular expression).
  extattrs      - Optional. A hash reference containing extensible attributes.
  extensible_attributes - Optional. A hash reference containing extensible attributes.
```

Examples

```
my @retrieved_objs = $session->get(
                     object       => "Infoblox::DNS::Host",
                     name         => "host1.test.com",
                     network_view => "default",
                     view         => "default" );

my @retrieved_objs = $session->get(
```

```
                        object                => "Infoblox::DNS::Host",
                        extensible_attributes => { 'Site' => 'Santa Clara' });

    # get all DNS host objects in zone "test.com" of default view
    my @retrieved_objs = $session->get(
                        object       => "Infoblox::DNS::Host",
                        zone         => "test.com",
                        network_view => "default",
                        view         => "default" );
```

Infoblox::Session->modify( )

Use this method to modify a host record object in the Infoblox appliance. See Infoblox::Session->modify() for parameters and return values.

Example

```
    # Use method to modify the comment.
    $host->comment("this is a modified comment");
    # Submit modification
    my $response = $session->modify( $host );
```

Infoblox::Session->remove( )

Use this method to remove a host record object from the Infoblox appliance. See Infoblox::Session->remove() for parameters and return values.

To remove a specific object, first use `get()` or `search()` to retrieve the specific host record object, and then submit this object for removal.

Example

```
    # Get the objects with the same name
    my @retrieved_objs = $session->get(
        object => "Infoblox::DNS::Host",
        name   => "host1.test.com",
        view   => "default" );
    # Find the desired object from the retrieved list.
    my $desired_host = $retrieved_objs[0];
    # Submit for removal
    my $response = $session->remove( $desired_host );
```

Infoblox::Session->search( )

Use this method to search for DNS host record objects in the Infoblox appliance. See Infoblox::Session->search() for parameters and return values.

Key References

```
    Apply the following attributes to search for a DNS host record object:

      name          - Optional. A host name in string format (regular expression).
      network_view  - Optional. The network view in which the host record is located. By default, all
    network views are searched.
      view          - Optional. The DNS view in which the host record is located. By default, all DNS
    views are searched. But if you omit this attribute and specify a zone, the appliance searches the
    'default' view only.
      zone          - Optional. A zone name in FQDN format.
      comment       - Optional. A comment in string format (regular expression).
      mac           - Optional. A MAC address present in any FixedAddr object in the ipv4addrs member
    (regular expression).
      ipv4addr      - Optional. An IP address in string format (regular expression).
      ipv6addr      - Optional. An IPv6 address in string format (regular expression).
      alias         - Optional. An alias of the host in string format (regular expression).
      extattrs      - Optional. A hash reference containing extensible attributes.
      extensible_attributes - Optional. A hash reference containing extensible attributes.
```

For more information about searching extensible attributes, see Infoblox::Grid::ExtensibleAttributeDef/Searching Extensible Attributes.

Examples

```
    # search for all DNS host objects that match ".test.com" in the default DNS view
    my @retrieved_objs = $session->search(
        object  => "Infoblox::DNS::Host",
        name    => '.*\.test\.com',
        view    => "default",
        comment => ".*modified comment" );

    # search for DNS host objects defining "Santa Clara" for "Site" extensible attribute
```

```
my @retrieved_objs = $session->search(
    object                => "Infoblox::DNS::Host",
    extensible_attributes => { 'Site' => 'Santa Clara' });

# search for all DNS host objects in zone "test.com" of default view
my @retrieved_objs = $session->search(
    object       => "Infoblox::DNS::Host",
    zone         => "test.com",
    network_view => "default",
    view         => "default" );
```

METHODS

This section describes all the methods that you can use to configure and retrieve the attribute values of a host record.

aliases( )

Use this method to set or retrieve the aliases of the host.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

The attribute value can be in unicode format.

Parameter

Array reference with the aliases for the host. The aliases of the host should be in Fully Qualified Domain Name (FQDN) format. The default value is undefined.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Getting aliases
my $ref_aliases = $host->aliases();
#Modifying aliases
$host->aliases(["one1.foo.com", "two2.foo.com"]);
```

allow_telnet( )

Use this method to set or retrieve the flag that enables copying SSH credential to TELNET.

Parameter

Specify 'true' to copy SSH credential to TELNET or 'false' to disable it. The default value is 'false'.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
# Get attribute value
my $value = $host->allow_telnet();
# Modify attribute value
$host->allow_telnet('true');
```

cloud_info( )

Use this method to retrieve cloud API related information for the Infoblox::DNS::Host object.

Parameter

None

Returns

The method returns the attribute value.

Example

```
# Get cloud_info
```

```
my $cloud_info = $object->cloud_info();
```

comment( )

Use this method to set or retrieve a descriptive comment.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

Desired comment in string format with a maximum of 256 bytes. The default value is undefined.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get comment
my $comment = $host->comment();
#Modify comment
$host->comment("Modifying the DNS host comment");
```

configure_for_dns( )

Use this method to set or retrieve the configure_for_dns flag of a DNS host.

When configure_for_dns is false the host will not have parent zone information.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

The default value for this field is true.

Parameter

Specify "true" to set the configure_for_dns flag or "false" to deactivate/unset it. The default value is "true".

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get configure_for_dns
my $configure_for_dns = $host->configure_for_dns();
#Modify configure_for_dns
$host->configure_for_dns("false");
```

cli_credentials( )

Use this method to set or retrieve the discovery CLI credentials.

Setting this method to a defined value implicitly sets the override_cli_credentials method to "true". Setting the parameter to undefined causes the appliance to use the grid default and automatically resets the override_cli_credentials attribute to "false".

Note that when cli_credentials is set to a defined value and override_cli_credentials is set to "false", the last operation takes precedence. Thus the sequence $host->cli_credentials([$cli]); $host->override_cli_credentials("false"); will set override_cli_credentials to "false", and the sequence $host->override_cli_credentials("false"); $host->cli_credentials([$cli]); will result in override_cli_credentials="true".

Parameter

Valid value is an array reference that contains Infoblox::Grid::Discovery::CLICredential objects.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get cli_credentials
my $cli_credentials = $host->cli_credentials();
#Modify cli_credentials
$host->cli_credentials([$cli1, $cli2]);
```

ddns_protected( )

Use this method to set or retrieve the flag that indicates whether DDNS updates for this record are allowed or not.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

Specify 'true' to protect record from DDNS updates and 'false' to allow DDNS updates for the specified record.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get ddns_protected value
my $ddns_protected = $object->ddns_protected();

#Modify ddns_protected value
$object->ddns_protected('true');
```

device_description( )

Use this method to set or retrieve the description of the discovery device.

Parameter

The device description in string format.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get device_description
my $device_description = $host->device_description();
#Modify device_description
$host->device_description('example');
```

device_location( )

Use this method to set or retrieve the location of the discovery device.

Parameter

The device location in string format.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get device_location
my $device_location = $host->device_location();
#Modify device_location
$host->device_location('example');
```

device_type( )

Use this method to set or retrieve the type of the discovery device.

Parameter

The device type in string format.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get device_type
my $device_type = $host->device_type();
#Modify device_type
$host->device_type('example');
```

device_vendor( )

Use this method to set or retrieve the vendor name of the discovery device.

Parameter

The device vendor name in string format.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get device_vendor
my $device_vendor = $host->device_vendor();
#Modify device_vendor
$host->device_vendor('example');
```

disable( )

Use this method to set or retrieve the disable flag of a DNS host.

The host can be disabled instead of removed. You can avoid removing and re-adding a host when a network appliance is repaired or relocated. When the changes to the physical appliance are complete, the host can be simply re-enabled.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

The default value for this field is false.

Parameter

Specify "true" to set the disable flag or "false" to deactivate/unset it. The default value is "false".

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get disable
my $disable = $host->disable();
#Modify disable
$host->disable("false");
```

dns_aliases( )

Use this method to retrieve the aliases, in punycode format, of the host. This is a read-only attribute.

Parameter

None

Returns

The method returns the attribute value.

Example

```
# Get attribute value
my $value = $host->dns_aliases();
```

dns_name( )

Use this method to retrieve the host name in punycode format. This is a read-only attribute.

Parameter

None

Returns

The method returns the attribute value.

Example

```
# Get attribute value
my $value = $host->dns_name();
```

extattrs( )

Use this method to set or retrieve the extensible attributes associated with a DNS Host object.

Parameter

Valid value is a hash reference containing the names of extensible attributes and their associated values (Infoblox::Grid::Extattr objects).

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get extattrs
my $ref_extattrs = $host->extattrs();
#Modify extattrs
$host->extattrs({ 'Site' => $extattr1, 'Administrator' => $extattr2 });
```

extensible_attributes( )

Use this method to set or retrieve the extensible attributes associated with a DNS Host object.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

For valid values for extensible attributes, see Infoblox::Grid::ExtensibleAttributeDef/Extensible Attribute Values.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get extensible attributes
my $ref_extensible_attributes = $host->extensible_attributes();
#Modify extensible attributes
$host->extensible_attributes({ 'Site' => 'Santa Clara', 'Administrator' => [ 'Peter', 'Tom' ] });
```

ipv4addrs( )

Use this method to set or retrieve the IPv4 addresses of the host.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

Valid value is an array reference that contains defined Infoblox::DHCP::FixedAddr objects or IPv4 addresses.

An IPv4 address is a 32-bit number in dotted decimal notation. It consists of four 8-bit groups of decimal digits separated by decimal points (example: 192.168.1.2).

If the Infoblox library is loaded with the :hostaddress option, the valid value is an array reference that contains Infoblox::DHCP::HostAddr objects.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Gett ipv4addrs
my $ref_ipv4addr = $host->ipv4addrs();
#Modify ipv4addrs
$host->ipv4addrs([$fixedaddr, "2.2.2.2"]); #$fixedaddr is an Infoblox::DHCP::FixedAddr object
```

ipv6addrs( )

Use this method to set or retrieve the IPv6 addresses of the host.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

Valid value is an array reference that contains IPv6 addresses in string format.

An IPv6 address is a string consisting of eight groups of four hexadecimal digits, where each group is separated by a colon (:) (example: 2001:0db8:85a3:0000:0000:8a2e:0370:7334). Various shortcuts exist to shorten the string representation of an IPv6 address, such as omitting the leading zeros of each group and replacing one or any number of consecutive groups of 0 value with two colons(::). i.e. The previous example can be written as 2001:db8:85a3::8a2e:370:7334.

When creating an Infoblox::DNS::Host object, setting this method to [] is not supported. If the Infoblox::DNS::Host object does not have associated IPv6 addresses, simply do not pass the ipv6addrs attribute to the constructor. If [] is set and the Host object is submitted for addition, the following error will be reported: "A host record requires at least one IP address." Setting the method to [] is supported for modify requests.

If the Infoblox library is loaded with the :hostaddress option, the valid value is an array reference that contains Infoblox::DHCP::IPv6HostAddr objects.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Gett ipv6addrs
my $ref_ipv6addr = $host->ipv6addrs();
#Modify ipv6addrs
$host->ipv6addrs(["2001::1", "2002::1:2:3:4"]);
```

last_queried( )

Use this method to retrieve the time when the associated record was last queried. This is a read-only attribute.

Parameter

None

Returns

The method returns the attribute value. The number of seconds that have elapsed since January 1st, 1970 UTC.

Example

```
#Get last_queried
my $last_queried = $host->last_queried();
```

ms_ad_user_data( )

Use this method to retrieve Microsoft Active Directory users related information. This is a read-only attribute.

Parameter

None

Returns

The valid return value is an Infoblox::Grid::MSServer::AdUser::Data object.

Example

```
#Get ms_ad_user_data
my $ms_ad_user_data = $host->ms_ad_user_data();
```

name( )

Use this method to set or retrieve the host name.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

The attribute value can be in unicode format.

Parameter

Host name in FQDN (Fully Qualified Domain Name) format. The FQDN consists of the host name followed by the domain name (example: abc.com). A host name can have a maximum of 256 bytes.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get name
my $name = $host->name();
#Modify name
$host-> name("host2.test.com");
```

network_view( )

Use this method to set or retrieve the network view of the DNS host.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

The default value is the "default" network view, which means the DNS host is in the default network view.

Parameter

Valid value is a defined Infoblox::DHCP::View object. The default network view value is the system-defined default network view object.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get network view
my $nview = $host->network_view();
#Modify network view, with an Infoblox::DHCP::View object
$host->network_view($nview);
```

override_cli_credentials( )

The override_cli_credentials attribute controls whether the cli_credentials value in the object is used, instead of the grid default.

The override_cli_credentials attribute can be specified explicitly. It is also set implicitly when cli_credentials is set to a defined value.

Parameter

Specify "true" to set the override_cli_credentials flag or "false" to deactivate/unset it. The default value is "false".

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
# Get attribute value
my $value = $host->override_cli_credentials();
# Modify attribute value
$host->override_cli_credentials('true');
```

rrset_order( )

Use this method to set or retrieve the rrset_order value. The value of this parameters specifies the order in which resource record sets are

returned.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

The possible values of the parameter are "cyclic", "random" and "fixed". The default value of this parameter is "cyclic".

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
# Get rrset_order
my $order = $host->rrset_order()
# Set fixed RR set order
$host->rrset_order("fixed");
```

ttl( )

Use this method to set or retrieve the Time to Live (TTL) value.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

The default value is undefined which indicates that the record inherits the TTL value of the zone.

Specify a TTL value to override the TTL value at the zone level.

Parameter

A 32-bit integer (range from 0 to 4294967295) that represents the duration in seconds that the record is cached. Zero indicates that the record should not be cached. The default value is undefined.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get ttl
my $ttl = $host->ttl();
#Modify ttl
$host->ttl(1800);
#Un-override ttl
$host->ttl(undef);
```

views( )

Use this method to set or retrieve the view of the DNS host.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

The default value is the "default" view, which means the DNS host is located under the default view.

Parameter

Array reference of defined Infoblox::DNS::View objects. The default view value is "default".

Note that the array size must be 1.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get views
my $ref_views = $host->views();
#Modify views, list of Infoblox::DNS::View objects
$host->views([$view1]);
```

zone( )

Use this method to retrieve the zone name of a DNS host object. This method is read-only and cannot be set.

Parameter

None

Returns

Returns the attribute value.

Example

```
# Get zone
my $zone = $host->zone();
```

SAMPLE CODE

The following sample code demonstrates the different functions that can be applied to a Host record object, such as add, search, modify, and remove. This sample also includes error handling for the operations.

#Preparation prior to a DNS host record insertion

```
#PROGRAM STARTS: Include all the modules that will be used

use strict;
use Infoblox;

#Create a session to the Infoblox Appliance
my $session = Infoblox::Session->new(
    master   => "192.168.1.2", #appliance host ip
    username => "admin",     #appliance user login
    password => "infoblox"   #appliance password
);

unless ($session) {
   die("Construct session failed: ",
       Infoblox::status_code() . ":" . Infoblox::status_detail());
}

#Create the zone prior to an host insertion
my $zone = Infoblox::DNS::Zone->new(name => "test.com");

unless ($zone) {
   die("Construct zone failed: ",
       Infoblox::status_code() . ":" . Infoblox::status_detail());
}

print "Zone object created successfully\n";

#Verify if the zone exists
my $object = $session->get(object => "Infoblox::DNS::Zone",
                           name => "test.com");
unless ($object) {
   print "Zone does not exist on server, safe to add the zone\n";
   $session->add($zone)
      or die("Add zone failed: ",
             $session->status_code() . ":" . $session->status_detail());
}
print "Zone added successfully\n";
```

#Create a DNS host record

```
#Construct a DNS host object

my $host = Infoblox::DNS::Host->new(
    name         => "host1.test.com",
    aliases      => ["one.foo.com", "two.foo.com"],
    comment      => "add a host host1.test.com",
    disable      => "true",
    ipv4addrs    => ["4.4.4.4"],
    ipv6addrs    => ["2001::4:4:4:4"],
    ttl          => 3600,
);

unless ($host) {
   die("Construct DNS host record failed: ",
```

```
        Infoblox::status_code() . ":" . Infoblox::status_detail());
}
print "DNS host object created successfully\n";

#Add the DNS host record object to Infoblox appliance through a session
$session->add($host)

    or die("Add host record failed: ",
           $session->status_code() . ":" . $session->status_detail());
     print "DNS host object added to server successfully\n";
```

#Search for a specific DNS host record

```
#Search host record host1.test.com
my @retrieved_objs = $session->search(
    object => "Infoblox::DNS::Host",
    name   => 'host1\.test\.com'
);
my $object = $retrieved_objs[0];
unless ($object) {
    die("Search host record failed: ",
        $session->status_code() . ":" . $session->status_detail());
}
print "Search DNS host object found at least 1 matching entry\n";

#Search all host records that start with "add" in the comment
my @retrieved_objs = $session->search(
    object  => "Infoblox::DNS::Host",
    comment => "add.*",
);
my $object = $retrieved_objs[0];
unless ($object) {
    die("Search host record failed: ",
        $session->status_code() . ":" . $session->status_detail());
}
print "Search DNS host object using regexp in comment field found at least 1 matching entry\n";

#Search all host records that start with "host1" in the zone
my @retrieved_objs = $session->search(
    object => "Infoblox::DNS::Host",
    name   => 'host1.*\.com',
 );
my $object = $retrieved_objs[0];
unless ($object) {
    die("Search host record failed: ",
        $session->status_code() . ":" . $session->status_detail());
}
print "Search DNS host object using regexp in name field found at least 1 matching entry\n";
```

#Get and modify a DNS host record

```
#Get host record through the session
my @retrieved_objs = $session->get(
    object => "Infoblox::DNS::Host",
    name   => "host1.test.com"
);
my $object = $retrieved_objs[0];

unless ($object) {
    die("Get host record failed: ",
        $session->status_code() . ":" . $session->status_detail());
}

#Modify one of the attributes of the specified host record
$object->comment("This is the modified comment");

#Applying the changes
$session->modify($object)
    or die("Modify host record failed: ",
           $session->status_code() . ":" . $session->status_detail());
print "DNS host object modified successfully \n";
```

#Remove a DNS host record.

```
#Get host record through the session
my @retrieved_objs = $session->get(
    object => "Infoblox::DNS::Host",
```

```
    name   => "host1.test.com"
);
my $object = $retrieved_objs[0];
unless ($object) {
    die("Get host record failed: ",
        $session->status_code() . ":" . $session->status_detail());
}

print "Get DNS host object successfull\n";

#Submit the object for removal

$session->remove($object)
    or die("Remove host record failed: ",
           $session->status_code() . ":" . $session->status_detail());
print "DNS host object removed successfully \n";

####PROGRAM ENDS####
```

AUTHOR

Infoblox Inc. http://www.infoblox.com/

SEE ALSO

Infoblox::DNS::View, Infoblox::DHCP::View, Infoblox::DNS::Zone, Infoblox::DHCP::FixedAddr,Infoblox::Session->add(), Infoblox::Session->get(), Infoblox::Session->modify(), Infoblox::Session->remove(), Infoblox::Session->search(), Infoblox::Session, Infoblox::Grid::MSServer::AdUser::Data

COPYRIGHT

Copyright (c) 2017 Infoblox Inc.

Infoblox::DNS::Member - DNS Member object

- NAME
- DESCRIPTION
- CONSTRUCTOR
- MODULE METHODS
  - Infoblox::DNS::Zone->multiple_primaries( )
  - Infoblox::DNS::Zone->secondary( )
  - Infoblox::Grid::DNS::Nsgroup->multiple_primaries( )
  - Infoblox::Grid::DNS::Nsgroup->secondary( )
- METHODS
  - forward_to( )
  - forwarders_only( )
  - override_forwarders( )
  - grid_replicate( )
  - ipv4addr( )
  - ipv6addr( )
  - lead( )
  - ms_parent_delegated( )
  - name( )
  - override_preferred_primaries( )
  - preferred_primaries( )
  - stealth( )
- SAMPLE CODE
- AUTHOR
- SEE ALSO
- COPYRIGHT

NAME

Infoblox::DNS::Member - DNS Member object

DESCRIPTION

A grid member is part of a grid which is a group of two or more Infoblox appliances that share sections of a common, distributed, built-in database. A grid member inherits its settings from the grid. A member is a single Infoblox appliance or an HA pair that provides DNS services seamlessly across an entire network.

CONSTRUCTOR

```
my $member = Infoblox::DNS::Member->new(
    ipv4addr       => $ipv4addr,        #Required if $ipv6addr is not present
    ipv6addr       => $ipv6addr,        #Required if $ipv4addr is not present
    grid_replicate => "true" | "false", #Optional / Default is "true"
    lead           => "true" | "false", #Optional / Default is "false"
    name           => $fqdn,            #Optional / Default is  hostname of the matching ipv4addr
    stealth        => "true" | "false"  #Optional / Default is "false"
    preferred_primaries          => [$pp1, $pp2, ...], #Optional / Default is undefined
    override_preferred_primaries => "true" | "false"   #Optional / Default is "false"
    forward_to          => [$ns1, $ns2, ...],  #Optional
    forwarders_only     => "true" | "false"    #Optional / Default is "false"
    override_forwarders => "true" | "false"    #Optional / Default is "false"
);
```

MODULE METHODS

The following functions are available to apply to an DNS Member object.

Infoblox::DNS::Zone->multiple_primaries( )

Use this function to specify primary members to a zone. See Infoblox::DNS::Zone->multiple_primaries() for parameters and return values.

Example

```
my $member1 = Infoblox::DNS::Member->new(
    name           => "localhost.localdomain",
    ipv4addr       => "192.168.1.2",
    stealth        => "false",
```

```
    lead           => "false",
    grid_replicate => "false"
);

unless($member1) {
     die("Construct member failed: ",
           Infoblox::status_code() . ":" . Infoblox::status_detail());
}
print "Member object created successfully\n";

#Create the Zone object with this member
my $firstzone = Infoblox::DNS::Zone->new(
    name        => "infoblox.com",
    multiple_primaries => [$member1],
);

my $response = $session->add($firstzone)
unless($response) {
     die("Add zone failed: ",
           session->status_code() . ":" . session->status_detail());
}
print "DNS member object added to the zone successfully\n";
```

Infoblox::DNS::Zone->secondary( )

Use this function to specify secondary member to a zone. See Infoblox::DNS::Zone->secondary() for parameters and return values.

Example

```
my $member1 = Infoblox::DNS::Member->new(
    name           => "localhost.localdomain",
    ipv4addr       => "192.168.1.2",
    stealth        => "false",
    lead           => "false",
    grid_replicate => "false"
);

unless($member1) {
     die("Construct member failed: ",
           Infoblox::status_code() . ":" . Infoblox::status_detail());
}
print "Member object created successfully\n";

my $member2 = Infoblox::DNS::Member->new(
    name           => "localhost1.localdomain",
    ipv4addr       => "192.168.1.3",
    stealth        => "false",
    lead           => "false",
    grid_replicate => "false"
);

unless($member2) {
     die("Construct member failed: ",
           Infoblox::status_code() . ":" . Infoblox::status_detail());
}
print "Member object created successfully\n";

#Create the Zone object with this member
my $firstzone = Infoblox::DNS::Zone->new(
    name        => "infoblox.com",
    multiple_primaries => [$member1],
    secondaries => [ $member2 ] ,
);

my $response = $session->add($firstzone)
unless($response) {
     die("Add zone failed: ",
           session->status_code() . ":" . session->status_detail());
}

print "DNS member object added to the zone successfully\n";
```

Infoblox::Grid::DNS::Nsgroup->multiple_primaries( )

Use this function to specify primary members to a NS group. See Infoblox::Grid::DNS::Nsgroup->multiple_primaries() for parameters and

return values.

Example

```
my $member1 = Infoblox::DNS::Member->new(
    name           => "localhost.localdomain",
    ipv4addr       => "192.168.1.2",
    stealth        => "false",
    lead           => "false",
    grid_replicate => "false"
);

unless($member1) {
     die("Construct member failed: ",
           Infoblox::status_code() . ":" . Infoblox::status_detail());
}
print "Member object created successfully\n";

my $nsgroup1 = Infoblox::Grid::DNS::Nsgroup->new (
   name      => "ns_group1",
   multiple_primaries => [$member1]
);
```

Infoblox::Grid::DNS::Nsgroup->secondary( )

Use this function to specify secondary member to a NS group. See Infoblox::Grid::DNS::Nsgroup->secondary() for parameters and return values.

Example

```
my $member1 = Infoblox::DNS::Member->new(
    name           => "localhost.localdomain",
    ipv4addr       => "192.168.1.2",
    stealth        => "false",
    lead           => "false",
    grid_replicate => "false"
);

unless($member1) {
     die("Construct member failed: ",
           Infoblox::status_code() . ":" . Infoblox::status_detail());
}
print "Member object created successfully\n";

my $member2 = Infoblox::DNS::Member->new(
    name           => "localhost1.localdomain",
    ipv4addr       => "192.168.1.3",
    stealth        => "false",
    lead           => "false",
    grid_replicate => "false"
);

unless($member2) {
     die("Construct member failed: ",
           Infoblox::status_code() . ":" . Infoblox::status_detail());
}
print "Member object created successfully\n";

my $nsgroup1 = Infoblox::Grid::DNS::Nsgroup->new (
   name        => "ns_group1",
   multiple_primaries => [$member1],
   secondaries => [ $member2 ],
);
```

METHODS

This section describes all the methods that you can use to set and retrieve the attribute values of a DNS Member object.

forward_to( )

Use this method to set or retrieve a list of DNS forwarders. You can use this attribute only for DNS members in forward-mapping zones.

Parameter

The valid value is an array reference that contains Infoblox::DNS::Nameserver objects.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get forward_to
my $forward_to = $member->forward_to();
#Modify forward_to
my $nameserver = Infoblox::DNS::Nameserver->new(
        name     => "ns1.forwd.com",
        ipv4addr => "11.11.11.11",
   );

$zone->forward_to([$nameserver]);
```

forwarders_only( )

Use this method to set the forwarders_only flag. You can use this attribute only for DNS members in forward-mapping zones.

Parameter

Specify "true" to use only DNS forwarders or "false" to disable it. The default value is 'false'.

The server writes 'forward only' into the DNS configuration file for the DNS member.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get forwarders_only
my $forwarders_only = $member->forwarders_only();
#Modify forwarders_only
$member->forwarders_only("true");
```

override_forwarders( )

Use this method to set the override_forwarders flag. You can use this attribute only for DNS members in forward-mapping zones.

Parameter

Specify "true" to override DNS forwarders or "false" to use zone default DNS forwarders. The default value is 'false'.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get override_forwarders
my $override_forwarders = $member->override_forwarders();
#Modify override_forwarders
$member->override_forwarders("true");
```

grid_replicate( )

Use this method to set or retrieve the grid_replicate flag, which indicates whether grid members that are primary and secondary servers use database replication or zone transfers for zone updates.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

Specify "true" to use ID grid replication or "false" to use DNS zone transfers.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get grid replicate
my $grid_replicate = $member->grid_replicate();
#Modify grid replicate
$member->grid_replicate("true");
```

ipv4addr( )

Use this method to set or retrieve the IPv4 address of the member.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

IPv4 address (32 bits) of the member.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get ipv4Addr
my $ipv4Addr = $member->ipv4Addr();
#Modify ipv4Addr
$member->ipv4Addr("192.168.1.3");
```

ipv6addr( )

Use this method to set or retrieve the IPv6 address if available. Required only when ipv4 is not specified.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

An IPv6 address is a 128-bit number in colon hexadecimal notation. It consists of eight groups of four hexadecimal digits separated by colons (example: 12ab:0000:0000:0123:4567:89ab:0000:cdef).

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
# Getting ipv6addr
my $ipv6addr = $member->ipv6addr();
# Modifying ipv6addr
$member->ipv6addr("12ab::345");
```

lead( )

Use this method to set or retrieve the "lead" flag.

When a primary server is external to a grid whose members are secondary servers, you can designate one member as a lead secondary. The primary server sends zone transfers to the lead secondary, which distributes the zone data to the other secondary servers in the grid using zone transfers (not the ID grid data replication mechanism).

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

Specify "true" to set lead flag or "false" to deactivate/unset it.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get lead
```

```
my $lead = $member->lead();
#Modify lead
$member->lead("true");
```

ms_parent_delegated( )

Use this method to retrieve the ms_parent_delegated flag. If the zone is a delegation and the primary server of the parent zone is a Microsoft (r) server, this flag is set to "true" if the FQDN and IP address in the delegation zone's NS record matches the FQDN and IP address of its authoritative name server.

Parameter

None.

Returns

The method returns the attribute value.

Example

```
#Get ms_parent_delegated
my $ms_parent_delegated = $member->ms_parent_delegated();
```

name( )

Use this method to set or retrieve the name of the member.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

A name in FQDN (Fully Qualified Domain Name) format. The FQDN consists of the host name followed by the domain name (example: host.domain.com). A host name can have a maximum of 256 characters.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get name
my $name = $member->name();
#Modify name
$member->name("dns.infoblox.com");
```

override_preferred_primaries( )

Use this method to set or retrieve the override_preferred_primaries attribute that controls whether the preferred_primaries method values of the member are used.

Parameter

Set the parameter to "true" to use preferred_primaries method values. Set the parameter to "false", and a primary preference list is determined automatically. The default value is "false".

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
# Get the attribute value
my $value = $member->override_preferred_primaries();
# Modify the attribute value
$member->override_preferred_primaries("false");
```

preferred_primaries( )

Use this method to set or retrieve a primary preference list for the member.

Parameter

A reference to an array of Grid member names only or Infoblox::DNS::Nameserver objects.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
# Get the attribute value
$value = $member->preferred_primaries();
# Modify the attribute value
$member->preferred_primaries(['test.com', 'example.com']);
$member->preferred_primaries([$nameserver1, $nameserver2]);
```

stealth( )

Use this method to set or retrieve the "stealth" flag, which indicates whether this member is in stealth mode. When a member is in stealth mode, its NS record is hidden from DNS queries; it does not respond to queries from resolvers and other name servers.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

Specify "true" to set the stealth flag or "false" to deactivate/unset it.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get stealth
my $stealth = $member->stealth();
#Modify stealth
$member->stealth("true");
```

SAMPLE CODE

The following sample code demonstrates the different functions that can be applied to an object, such as add, search, modify, and remove. This sample also includes error handling for the operations.

#Preparation prior to a DNS Member object insertion

```
#PROGRAM STARTS: Include all the modules that will be used
use strict;
use Infoblox;

#Create a session to the Infoblox appliance

my $host_ip   = "192.168.1.2";
my $host_name = "infoblox.localdomain";

my $session = Infoblox::Session->new(
               master   => $host_ip, #appliance host ip
               username => "admin",       #appliance user login
               password => "infoblox"     #appliance password
);

unless ($session) {
       die("Construct session failed: ",
               Infoblox::status_code() . ":" . Infoblox::status_detail());
}
print "Session created successfully\n";
```

#Create a DNS Member object

```
my $member1 = Infoblox::DNS::Member->new(
    name           => $host_name,
    ipv4addr       => $host_ip,
    stealth        => "false",
    lead           => "false",
    grid_replicate => "false"
);

unless($member1) {
```

```
      die("Construct member failed: ",
            Infoblox::status_code() . ":" . Infoblox::status_detail());
}
print "Member object created successfully\n";

my $default_view = Infoblox::DNS::View->new(name => "default");

#Create the Zone object with this member
my $firstzone = Infoblox::DNS::Zone->new(
    name         => "infoblox.com",
    views        => [$default_view],
    email        => "admin\@infoblox.com",
    comment      => "add a zone infoblox.com",
    multiple_primaries => [$member1],
);

unless($firstzone) {
            die("Construct Zone object failed: ",
                  Infoblox::status_code() . ":" . Infoblox::status_detail());
}
print "Zone object created successfully\n";

#Add the Zone object into the Infoblox appliance through a session
$session->add($firstzone)
     or die("Add Zone object failed: ",
           $session->status_code() . ":" . $session->status_detail());
print "DNS member object added to the zone successfully\n";

#Modify a DNS member

my @result = $session->search(object => "Infoblox::DNS::Zone",
                              name   => "infoblox.com");

 unless (scalar(@result) == 0) {
     $firstzone = $result[0];

     if ($firstzone) {

          #modify grid replicate value of member
          $member1->grid_replicate("true");

          #update zone
          $firstzone->multiple_primaries([$member1]);

          #update session
          $session->modify($firstzone)
                 or die("Modify zone object failed: ",
                        $session->status_code() . ":" . $session->status_detail());
          print "DNS member object updated and added to the zone successfully\n";
                }
            } else {
                print "No zone found with the specified name";
            }

####PROGRAM ENDS####
```

AUTHOR

Infoblox Inc. http://www.infoblox.com/

SEE ALSO

Infoblox::DNS::TSIGKey, Infoblox::Session->add(),Infoblox::Session->modify(),Infoblox::Session->search(),Infoblox::Session, Infoblox::DNS::Nameserver

COPYRIGHT

Copyright (c) 2017 Infoblox Inc.

Infoblox::DNS::Member::SoaMname - a per-master SOA MNAME information.

- NAME
- DESCRIPTION
- CONSTRUCTOR
- SESSION METHODS
- METHODS
  - dns_mname( )
  - grid_member( )
  - mname( )
  - ms_server( )
- AUTHOR
- SEE ALSO
- COPYRIGHT

NAME

Infoblox::DNS::Member::SoaMname - a per-master SOA MNAME information.

DESCRIPTION

The object contains a per-master SOA MNAME Information.

CONSTRUCTOR

```
my $soa_mname = Infoblox::DNS::Member::SoaMname->new(
   grid_member => $string,       # Required if ms_server is not specified
   ms_server   => $string,       # Required if grid_member is not specified
   mname       => $string,       # Optional
);
```

You cannot set both grid_member and ms_server attributes at the same time.

SESSION METHODS

The object doesn't support any session methods.

METHODS

This section describes all the methods that you can use to configure and retrieve the attribute values of the object.

dns_mname( )

Use this method to retrieve the master's SOA MNAME in punycode format. This is a read-only attribute.

Parameter

None

Returns

The method returns the attribute value.

Example

```
# Get attribute value
my $value = $soa_mname->dns_mname();
```

grid_member( )

Use this method to set or retrieve the primary server name for the zone.

Parameter

The primary server name in string format.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
# Get the attribute value
my $attr = $soa_mname->grid_member();
# Modify the attribute value
$soa_mname->grid_member('member.com');
```

mname( )

Use this method to set or retrieve the master's SOA MNAME.

Parameter

The master's SOA MNAME in string format.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
# Get the attribute value
my $attr = $soa_mname->mname();
# Modify the attribute value
$soa_mname->mname('dummy-mname.com');
```

ms_server( )

Use this method to set or retrieve the IPv4 address or FQDN of the Microsoft (r) Server for the zone.

Parameter

Valid value is an IPv4 address (32 bits) or FQDN of the Microsoft (r) Server.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
# Get the attribute value
my $attr = $soa_mname->ms_server();
# Modify the attribute value
$soa_mname->ms_server('10.0.0.1');
```

AUTHOR

Infoblox Inc. http://www.infoblox.com/

SEE ALSO

Infoblox::DNS::Zone

COPYRIGHT

Copyright (c) 2017 Infoblox Inc.

Infoblox::DNS::Member::SoaSerial - a per-master SOA serial information.

- NAME
- DESCRIPTION
- CONSTRUCTOR
- SESSION METHODS
- METHODS
  - grid_member( )
  - serial( )
  - ms_server
- AUTHOR
- SEE ALSO
- COPYRIGHT

NAME

Infoblox::DNS::Member::SoaSerial - a per-master SOA serial information.

DESCRIPTION

The object contains a per-master SOA serial information.

CONSTRUCTOR

The object cannot be created because all object attributes are read-only.

SESSION METHODS

The object doesn't support any session methods.

METHODS

This section describes all the methods that you can use to configure and retrieve the attribute values of the object.

grid_member( )

Use this method to retrieve the primary server name for the zone. This is a read-only attribute.

Parameter

None.

Returns

The method returns the attribute value.

Example

```
# Get the attribute value
my $attr = $soa_serial->grid_member();
```

serial( )

Use this method to retrieve the SOA serial number. This is a read-only attribute.

Parameter

None.

Returns

The method returns the attribute value.

Example

```
# Get the attribute value
my $attr = $soa_serial->serial();
```

ms_server

Use this method to retrieve the IPv4 address or FQDN of the Microsoft (r) Server for the zone. This is a read-only attribute.

Parameter

None.

Returns

The method returns the attribute value.

Example

```
# Get the attribute value
my $attr = $soa_serial->ms_server();
```

AUTHOR

Infoblox Inc. http://www.infoblox.com/

SEE ALSO

Infoblox::DNS::Zone

COPYRIGHT

Copyright (c) 2017 Infoblox Inc.

Infoblox::DNS::MSServer - Microsoft DNS Server object

- NAME
- DESCRIPTION
- CONSTRUCTOR
- MODULE METHODS
  - Infoblox::DNS::Zone->multiple_primaries( )
  - Infoblox::DNS::Zone->secondaries( )
  - Infoblox::DNS::Zone->stub_members( )
- METHODS
  - address( )
  - ipv4addr( )
  - ms_parent_delegated( )
  - name( )
  - stealth( )
- AUTHOR
- SEE ALSO
- COPYRIGHT

NAME

Infoblox::DNS::MSServer - Microsoft (r) DNS Server object

DESCRIPTION

A Microsoft DNS Server object is a Windows-based server that is running the DNS service and is synchronized with an Infoblox grid. As such, the Microsoft DNS server shares its DNS data with the Infoblox DNS servers in a grid.

CONSTRUCTOR

```
my $msserver = Infoblox::DNS::MSServer->new(
    address        => $string,           #Required
    ipv4addr       => $string,           #Required
    name           => $string,           #Required
    stealth        => "true" | "false",  #Optional
);
```

MODULE METHODS

The following functions can be applied to a Microsoft DNS Server object.

Infoblox::DNS::Zone->multiple_primaries( )

Use any of these functions to assign a Microsoft DNS server as the primary server of a zone. See Infoblox::DNS::Zone->multiple_primaries() for parameters and return values.

Example

```
my $msserver1 = Infoblox::DNS::MSServer->new(
    address        => "10.0.0.1",
    ipv4addr       => "10.0.0.1",
    name           => "hostname.com",
);

unless($msserver1) {
     die("Construct msserver failed: ",
           Infoblox::status_code() . ":" . Infoblox::status_detail());
}
print "Msserver object created successfully\n";

#Create the Zone object with this Microsoft DNS server
my $firstzone = Infoblox::DNS::Zone->new(
    name        => "infoblox.com",
    multiple_primaries => [$msserver1],
);

my $response = $session->add($firstzone)
unless($response) {
     die("Add zone failed: ",
```

```
            session->status_code() . ":" . session->status_detail());
}
print "DNS MS Server object added to the zone successfully\n";
```

Infoblox::DNS::Zone->secondaries( )

Use this function to specify an Microsoft DNS server as the secondary server to a zone. See Infoblox::DNS::Zone->secondaries() for parameters and return values.

Example

```
my $msserver1 = Infoblox::DNS::MSServer->new(
    address        => "10.0.0.1",
    ipv4addr       => "10.0.0.1",
    name           => "hostname.com",
);

unless($msserver1) {
     die("Construct msserver failed: ",
           Infoblox::status_code() . ":" . Infoblox::status_detail());
}
print "Msserver object created successfully\n";

#Create the Zone object with this member
my $firstzone = Infoblox::DNS::Zone->new(
    name        => "infoblox.com",
    multiple_primaries => [$member1],
    secondaries => [ $msserver2 ] ,
);

my $response = $session->add($firstzone)
unless($response) {
     die("Add zone failed: ",
           session->status_code() . ":" . session->status_detail());
}

print "DNS MS server member object added to the zone successfully\n";
```

Infoblox::DNS::Zone->stub_members( )

Use this function to assign a Microsoft DNS server to a stub zone. See Infoblox::DNS::Zone->stub_members() for parameters and return values.

Example

```
my $msserver1 = Infoblox::DNS::MSServer->new(
    address        => "10.0.0.1",
    ipv4addr       => "10.0.0.1",
    name           => "hostname.com",
);

unless($msserver1) {
     die("Construct msserver failed: ",
           Infoblox::status_code() . ":" . Infoblox::status_detail());
}
print "Msserver object created successfully\n";

#Create the Zone object with this member
my $firstzone = Infoblox::DNS::Zone->new(
    name        => "infoblox.com",
    stub_members => [ $msserver2 ] ,
);

my $response = $session->add($firstzone)
unless($response) {
     die("Add zone failed: ",
           session->status_code() . ":" . session->status_detail());
}

print "DNS MS server member object added to the zone successfully\n";
```

METHODS

This section describes all the methods that you can use to set and retrieve the attribute values of a DNS Member object.

address( )

Use this method to set or retrieve the IPv4 address or FQDN of the member.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

IPv4 address (32 bits) or FQDN of the member.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get address
my $address = $member->address();
#Modify address
$member->address("192.168.1.3");
```

ipv4addr( )

Use this method to set or retrieve the member IPv4 address that is published in the glue A record when setting this Microsoft DNS server as a name server.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

IPv4 address (32 bits) of the member.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get ipv4addr
my $ipv4addr = $member->ipv4addr();
#Modify ipv4addr
$member->ipv4addr("192.168.1.3");
```

ms_parent_delegated( )

Use this method to retrieve the ms_parent_delegated flag. If the zone is a delegation and the primary server of the parent zone is a Microsoft server, this flag is set to "true" if the FQDN and IP address in the delegation zone's NS record matches the FQDN and IP address of its authoritative name server.

Parameter

None.

Returns

The method returns the attribute value.

Example

```
#Get ms_parent_delegated
my $ms_parent_delegated = $member->ms_parent_delegated();
```

name( )

Use this method to set or retrieve the member FQDN that is used when setting this Microsoft DNS server as a name server.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

The member's FQDN.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get name
my $name = $member->name();
#Modify name
$member->name("hostname.com");
```

stealth( )

Use this method to set or retrieve the "stealth" flag, which indicates whether this name server is in stealth or normal mode.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

Specify "true" to set the stealth flag or "false" to deactivate/unset it.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get stealth
my $stealth = $member->stealth();
#Modify stealth
$member->stealth("true");
```

AUTHOR

Infoblox Inc. http://www.infoblox.com/

SEE ALSO

Infoblox::DNS::TSIGKey, Infoblox::Session->add(),Infoblox::Session->modify(),Infoblox::Session->search(),Infoblox::Session

---

COPYRIGHT

Copyright (c) 2017 Infoblox Inc.

Infoblox::DNS::Nameserver - DNS Nameserver object

- NAME
- DESCRIPTION
- CONSTRUCTOR
- MODULE METHODS
  - Infoblox::DHCP::View->external_ddns_primaries( )
  - Infoblox::DNS::Zone->delegate_to( )
  - Infoblox::DNS::Zone->forward_to( )
  - Infoblox::DNS::Zone->secondaries( )
  - Infoblox::DNS::Zone->multiple_primaries( )
  - Infoblox::Grid::Member->prefer_resolver( )
  - Infoblox::Grid::Member->alternate_resolver( )
  - Infoblox::Grid::DNS::Nsgroup->multiple_primaries( )
  - Infoblox::Grid::DNS::Nsgroup->secondaries( )
- METHODS
  - ddns_zone( )
  - gss_tsig_domain( )
  - gss_tsig_dns_principal( )
  - ipv4addr( )
  - ipv6addr( )
  - ms_parent_delegated( )
  - name( )
  - stealth( )
  - TSIGalgorithm( )
  - TSIGkey( )
  - TSIGname( )
- SAMPLE CODE
- AUTHOR
- SEE ALSO
- COPYRIGHT

NAME

Infoblox::DNS::Nameserver - DNS Nameserver object

DESCRIPTION

Manipulates a Nameserver object that contains information about the domain name space.

CONSTRUCTOR

```
 my $nameserver = Infoblox::DNS::Nameserver->new(
     ipv4addr  => $ipv4addr,            #Required if $ipv6addr is not present
     ipv6addr  => $ipv6addr,            #Required if $ipv4addr is not present
     name      => $fqdn,                #Required if this name server is not used to represent an external
DDNS server
     ddns_zone => $string,              #Required if this name server is used to represent an external DDNS
server
     gss_tsig_domain  => $string,       #Optional / Default is undefined
     gss_tsig_dns_principal => $string, #Optional / Default is undefined
     stealth   => "true" | "false",     #Optional / Default is "false"
     TSIGkey   => $string | ":2xCOMPAT" #Optional / Default is undefined
     TSIGname  => $string,              #Optional / Default is undefined
     TSIGalgorithm => "HMAC-MD5" | "HMAC-SHA256" ,  #Optional / Default is "HMAC-MD5"
 );
```

MODULE METHODS

The following functions are available to apply to a nameserver object.

Infoblox::DHCP::View->external_ddns_primaries( )

Use this function to specify a nameserver to send DDNS updates to at the network view level.

Example

```
      #Construct an object
        my $nameserver = Infoblox::DNS::Nameserver->new(
          ipv4addr  => "1.2.3.4",
```

```
    TSIGname  => "tsig_name",
    TSIGkey   => "ZX1jVJl7C58GT/sc7Q3uca==",
    ddns_zone => "zone1"
);

#Construct an object
  my $nameserver1 = Infoblox::DNS::Nameserver->new(
    ipv4addr  => "1.2.3.5",
    TSIGname  => "tsig_name",
    TSIGkey   => "ZX1jVJl7C58GT/sc7Q3uca==",
    ddns_zone => "zone2"
);

# Configure Nameserver
my $response = $View->external_ddns_primaries([$nameserver,$nameserver1]);
```

Infoblox::DNS::Zone->delegate_to( )

Use this function to specify a nameserver at the zone level.

Example

```
#Construct an object
  my $nameserver = Infoblox::DNS::Nameserver->new(
    name      => "ns1.test.com",
    ipv4addr  => "4.4.7.7",
);

# Configure Nameserver
my $response = $Zone->delegate_to([$nameserver]);
```

Infoblox::DNS::Zone->forward_to( )

Use this function to specify a nameserver at the zone level.

Example

```
#Construct an object
  my $nameserver = Infoblox::DNS::Nameserver->new(
    name      => "ns2.test.com",
    ipv4addr  => "4.4.6.6",
);

# Configure Nameserver
my $response = $Zone->forward_to([$nameserver]);
```

Infoblox::DNS::Zone->secondaries( )

Use this function to specify a nameserver at the zone level.

Example

```
#Construct an object
  my $nameserver = Infoblox::DNS::Nameserver->new(
    name      => "ns3.test.com",
    ipv4addr  => "4.5.6.4",
    stealth   => "false",
    TSIGname  => "tsig_name",
    TSIGkey   => "ZX1jVJl7C58GT/sc7Q3uca==",
);

#Construct an object
  my $nameserver1 = Infoblox::DNS::Nameserver->new(
    name      => "ns4.test.com",
    ipv6addr  => "2009::546",
    stealth   => "false",
    TSIGname  => "tsig_name",
    TSIGkey   => "ZX1jVJl7C58GT/sc7Q3uca==",
);

# Configure Nameserver
my $response = $Zone->secondaries([$nameserver,$nameserver1]);
```

Infoblox::DNS::Zone->multiple_primaries( )

Use this function to specify nameservers at the zone level.

Example

```
#Construct an object
  my $nameserver = Infoblox::DNS::Nameserver->new(
    name      => "ns4.test.com",
    ipv4addr  => "4.4.5.6",
    stealth   => "false",
    TSIGname  => "tsig_name",
    TSIGkey   => "ZX1jVJl7C58GT/sc7Q3uca==",
);

#Construct an object
  my $nameserver1 = Infoblox::DNS::Nameserver->new(
    name      => "nsv6.test.com",
    ipv6addr  => "2004::223",
    stealth   => "false",
    TSIGname  => "tsig_name",
    TSIGkey   => "ZX1jVJl7C58GT/sc7Q3uca==",
);

# Configure Nameserver
my $response = $Zone->multiple_primaries([$nameserver,$nameserver1]);
```

Infoblox::Grid::Member->prefer_resolver( )

Use this function to specify a nameserver at the grid member level.

Example

```
#Construct an object
  my $nameserver = Infoblox::DNS::Nameserver->new(
    ipv4addr  => "10.7.5.6",
);

# Configure Nameserver
my $response = $grid_member_object->prefer_resolver([$nameserver]);
```

Infoblox::Grid::Member->alternate_resolver( )

Use this function to specify a nameserver at the grid member level.

Example

```
#Construct an object
  my $nameserver = Infoblox::DNS::Nameserver->new(
    ipv4addr  => "10.10.5.6",
);

# Configure Nameserver
my $response = $grid_member_object->alternate_resolver([$nameserver]);
```

Infoblox::Grid::DNS::Nsgroup->multiple_primaries( )

Use this function to specify nameservers at the grid DNS level.

Example

```
#Construct an object
  my $nameserver = Infoblox::DNS::Nameserver->new(
    name      => "ns5.test.com",
    ipv4addr  => "10.10.5.6",
    stealth   => "false",
    TSIGname  => "tsig_name",
    TSIGkey   => "Zx1JVjl7C58gT/sc7Q3uca==",
);

# Configure Nameserver
my $response = $grid_member_object->multiple_primaries([$nameserver]);
```

Infoblox::Grid::DNS::Nsgroup->secondaries( )

Use this function to specify a nameserver at the grid DNS level.

Example

```
#Construct an object
  my $nameserver = Infoblox::DNS::Nameserver->new(
```

```
        name      => "ns5.test.com",
        ipv4addr  => "10.10.5.6",
        stealth   => "false",
        TSIGname  => "tsig_name",
        TSIGkey   => "Zx1JVjl7C58gT/sc7Q3uca==",
    );

    # Configure Nameserver
    my $response = $grid_member_object->secondaries([$nameserver]);
```

METHODS

This section describes all the methods that you can use to configure and retrieve the attribute values of a Nameserver object.

ddns_zone( )

Use this method to set or retrieve the zone name served by the name server.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

Desired ddns_zone for the name server in string format.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
    # Getting ddns_zone
    my $ddns_zone = $nameserver->ddns_zone();
    # Modifying ddns_zone
    $nameserver->ddns_zone("zone2");
```

gss_tsig_domain( )

Use this method to set or retrieve the domain in which GSS-TSIG for dynamic updates is enabled.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

Desired domain name in string format with a maximum of 1023 bytes.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
    # Getting gss_tsig_domain
    my $gss_tsig_domain = $nameserver->gss_tsig_domain();
    # Modifying gss_tsig_domain
    $nameserver->gss_tsig_domain("mydomain");
```

gss_tsig_dns_principal( )

Use this method to set or retrieve the principal name in which GSS-TSIG for dynamic updates is enabled.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

Desired principal name in string format with a maximum of 1023 bytes.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
# Getting gss_tsig_dns_principal
my $gss_tsig_dns_principal = $nameserver->gss_tsig_dns_principal();
# Modifying gss_tsig_dns_principal
$nameserver->gss_tsig_dns_principal("local1/a.localhost@LOCAL");
```

ipv4addr( )

Use this method to set or retrieve the IPv4 address. Required only when ipv6 is not specified.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

An IPv4 address is a 32-bit number in dotted decimal notation. It consists of four 8-bit groups of decimal digits separated by decimal points (example: 192.168.1.2).

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
# Getting ipv4addr
my $ipv4addr = $nameserver->ipv4addr();
# Modifying ipv4addr
$nameserver->ipv4addr("10.0.0.3");
```

ipv6addr( )

Use this method to set or retrieve the IPv6 address. Required only when ipv4 is not specified.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

An IPv6 address is a 128-bit number in colon hexadecimal notation. It consists of eight groups of four hexadecimal digits separated by colons (example: 12ab:0000:0000:0123:4567:89ab:0000:cdef).

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
# Getting ipv6addr
my $ipv6addr = $nameserver->ipv6addr();
# Modifying ipv6addr
$nameserver->ipv6addr("12ab::345");
```

ms_parent_delegated( )

Use this method to retrieve the ms_parent_delegated flag. If the zone is a delegation and the primary server of the parent zone is a Microsoft (r) server, this flag is set to "true" if the FQDN and IP address in the delegation zone's NS record matches the FQDN and IP address of its authoritative name server.

Parameter

None.

Returns

The method returns the attribute value.

Example

```
#Get ms_parent_delegated
my $ms_parent_delegated = $member->ms_parent_delegated();
```

name( )

Use this method to set or retrieve a resolvable domain name for the server.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

Desired name for the name server in FQDN format.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails. If you did not specify a parameter, the method returns the attribute value.

Example

```
# Getting name
my $name = $nameserver->name();
# Modifying name
$nameserver->name("ns2.domain.com");
```

stealth( )

Use this method to make the name server stealth or not. When the attribute stealth is "true", the name server is in stealth mode. This means that its NS record is not published among the zone data, and it does not respond to queries from resolvers and other name servers.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

Valid values are "true" or "false".

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
# Getting stealth
my $stealth = $nameserver->stealth();
# Modifying stealth
$nameserver->stealth("true");
```

TSIGalgorithm( )

Use this method to set or retrieve the algorithm of the TSIG key you want to use.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

Desired algorithm of the TSIG key. Valid values are "HMAC-SHA256" and "HMAC-MD5", which is the default.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
# Getting TSIGalgorithm
my $TSIGalgorithm = $nameserver->TSIGalgorithm();
# Modifying TSIGalgorithm
$nameserver->TSIGalgorithm("HMAC-SHA256");
```

TSIGkey( )

Use this method to set or retrieve the TSIG key. This key must also be present on the external primary server. You can generate a TSIG key, or you can obtain the TSIG key name and key from the external name server either by accessing the appliance yourself or by requesting the appliance administrator to deliver them to you through some out-of-band mechanism. Then type or copy-and-paste that name and key into the appropriate fields.

OR

Use DNS One 2.x TSIG: If you want to use TSIG authentication and the external primary name server is an Infoblox appliance running DNS One 2.x code. The local appliance generates the required TSIG key for authenticating DNS messages to and from appliances running DNS One 2.x code.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

Desired TSIGkey of the name server in string format. Valid values are ":2xCOMPAT" or a TSIG key string.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
# Getting TSIGkey
my $TSIGkey = $nameserver->TSIGkey();
# Modifying TSIGname
$nameserver->TSIGkey("ZX1jVJl7C58GT/sc7Q3uc2==");
```

TSIGname( )

Use this method to set or retrieve the name of the TSIG key you want to use.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

Desired name of the TSIG key in string format.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
# Getting TSIGname
my $TSIGname = $nameserver->TSIGname();
# Modifying TSIGname
$nameserver->TSIGname("tsig2_name");
```

SAMPLE CODE

The following sample code demonstrates the different functions that can be applied to an object such as add, modify. This sample also includes error handling for the operations.

##Preparation prior to a DNS nameserver object insertion

```
#PROGRAM STARTS: Include all the modules that will be used
use strict;
use Infoblox;

#Create a session to the Infoblox appliance
my $session = Infoblox::Session->new(
    master   => "192.168.1.2",
    username => "admin",
    password => "infoblox"
);
unless ($session) {
   die("Construct session failed: ",
       Infoblox::status_code() . ":" . Infoblox::status_detail());
}
print "Session created successfully\n";
```

#Create a nameserver object

```
 my $nameserver1 = Infoblox::DNS::Nameserver->new(
    name     => "ns1.domain.com",
    ipv4addr => "10.4.5.6",
    stealth  => "false",
    TSIGname => "tsig_name",
    TSIGkey  => "ZX1jVJl7C58GT/sc7Q3ucA==",
 );
 unless ($nameserver1){
     die("Construct Nameserver failed: ",
           Infoblox::status_code() . ":" . Infoblox::status_detail());
  }
```

```
print "Nameserver object created successfully\n";
```

 # Create a zone and use $nameserver1

```
#create a zone and use $nameserver1
my $zone = Infoblox::DNS::Zone->new(
   name    => "domain.com",
   multiple_primaries => [$nameserver1],
 );

#put zone into session
$session->add($zone)
  or die("Zone creation failed: ",
           Infoblox::status_code() . ":" . Infoblox::status_detail());

print " Zone creation successful\n";
```

#Get and modify the name server

```
#get Zone object from session
my @result = $session->get(
    object => "Infoblox::DNS::Zone",
    name   => "domain.com"
);

unless (scalar(@result) == 0) {
    my $zone = $result[0];

    if ($zone) {

        #modify ipv4addr value
        $nameserver1->ipv4addr("5.5.5.5");
        $zone->multiple_primaries([$nameserver1]);

        #update session
        $session->modify($zone)
            or die("modify zone domain.com failed: ",
               $session->status_code(), $session->status_detail());
           }
     } else {
         print "No zone found.";
     }
    ####PROGRAM ENDS####
```

AUTHOR

Infoblox Inc. http://www.infoblox.com/

SEE ALSO

Infoblox::Session, Infoblox::Session->add(), Infoblox::Session->get(), Infoblox::Session->modify(),Infoblox::DHCP::View,Infoblox::DNS::Zone, Infoblox::Grid::DHCP, Infoblox::Grid::Member, Infoblox::Grid::DNS::Nsgroup

---

COPYRIGHT

Copyright (c) 2017 Infoblox Inc.

Infoblox::DNS::Nameserver::Address - NS Name Server object.

- NAME
- DESCRIPTION
- CONSTRUCTOR
- MODULE METHODS
  - Infoblox::DNS::Record::NS->nameservers( )
- METHODS
  - address( )
  - auto_create_ptr( )
- AUTHOR
- SEE ALSO
- COPYRIGHT

NAME

Infoblox::DNS::Nameserver::Address - NS Name Server object.

DESCRIPTION

The NS Name Server object provides IP address information for the name server associated with an NS record.

CONSTRUCTOR

```
my $rootns = Infoblox::DNS::Nameserver::Address->new(
             address => $address,                  #Required
             auto_create_ptr => "true" | "false"   #Optional / Default "true"
);
```

MODULE METHODS

You can apply the following functions to an NS Name Server object:

Infoblox::DNS::Record::NS->nameservers( )

Use this function to specify the IP addresses associated with the NS record. See Infoblox::DNS::Record::NS->nameservers() for parameters and return values.

Example

```
#Construct an NS Name Server object
my $ns1 = Infoblox::DNS::Nameserver::Address->new(
              address  => "10.0.0.1"
);

# Configure a name server address on the NS record object
$ns_record->nameservers([$ns1]);
```

METHODS

This section describes all the methods you can use to configure and retrieve the attribute values of an NS Name Server object.

address( )

Use this method to set or retrieve the IPv4 address of the name server.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

The IPv4 address (32 bits) of the name server.

Returns

If you specify a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you do not specify a parameter, the method returns the attribute value.

Example

```
#Get the address
my $addr = $ns->address();
#Modify the address
$ns->address("10.0.0.1");
```

auto_create_ptr( )

Use this method to set or retrieve the auto_create_ptr flag of the name server. When this flag is set to true, PTR records are automatically created for NS records created in a reverse zone.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

Valid values are "true" or "false".

Returns

If you specify a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you do not specify a parameter, the method returns the attribute value.

Example

```
#Get auto_create_ptr
my $aptr = $ns->auto_create_ptr();
#Modify auto_create_ptr
$rootns->auto_create_ptr("false");
```

AUTHOR

Infoblox Inc. http://www.infoblox.com/

SEE ALSO

Infoblox::Session, Infoblox::DNS::Record::NS

COPYRIGHT

Copyright (c) 2017 Infoblox Inc.

Infoblox::DNS::OrderedResponsePolicyZones - Ordered Response Policy Zones object.

- NAME
- DESCRIPTION
- CONSTRUCTOR
- SESSION METHODS
  - Infoblox::Session->add( )
  - Infoblox::Session->get( )
  - Infoblox::Session->modify( )
- METHODS
  - view( )
  - rp_zones( )
- SAMPLE CODE
- AUTHOR
- SEE ALSO
- COPYRIGHT

NAME

Infoblox::DNS::OrderedResponsePolicyZones - Ordered Response Policy Zones object.

DESCRIPTION

An object type to set or retrieve an ordered list of Response Policy Zones in a DNS view.

Server will reject zones that are disabled or zones without primary name server assigned.

CONSTRUCTOR

```
 my $ordered_rpz = Infoblox::DNS::OrderedResponsePolicyZones->new(
                        view                    => $view,
#Required
                        rp_zones                => [$response_policy_zone1, $response_policy_zone2, ...],
#Required
 );
```

SESSION METHODS

This section describes all the methods in the Infoblox::Session module that you can apply to an Ordered Response Policy Zones object.

Infoblox::Session->add( )

Use this method to add an object to the Infoblox appliance. See Infoblox::Session->add() for parameters and return values.

No object is added in this method, but the Response Policy Zones for the specified DNS View assume the specified order.

Example

```
       #Construct an object
       my $ordered_rpz = Infoblox::DNS::OrderedResponsePolicyZones->new(
                                                                  view => $view,
                                                                  rp_zones  => [$response_policy_zone1,
      $response_policy_zone2]
                                                                 );
       #Submit for addition
       my $response = $session->add( $ordered_rpz );
```

Infoblox::Session->get( )

Use this method to retrieve all the matching objects from the Infoblox appliance. See Infoblox::Session->get() for parameters and return values.

Key References

```
       Apply the following attributes to get a specific Response Policy Zones object:

       view - Required. An Infoblox::DNS::View object.
```

Examples

```
       my @retrieved_objs = $session->get(
```

```
        object   => "Infoblox::DNS::OrderedResponsePolicyZones",
        view  => $view
    );
```

Infoblox::Session->modify( )

Use this method to modify an object in the Infoblox appliance. See Infoblox::Session->modify() for parameters and return values.

No object is modified in this method, but the Response Policy Zones for the specified DNS View assume the specified order.

Example

```
#Use this method to modify the order of Response Policy Zones.
$ordered_rpz->rp_zones([$rpz4, $rpz2, $rpz3, $rpz1]);
#Submit modification
my $response = $session->modify( $ordered_rpz );
```

METHODS

Use the following methods to access the attributes of an Infoblox::DNS::OrderedResponsePolicyZones object. Specify a parameter to set the attribute's value, or omit the parameter to get the attribute's value.

view( )

Use this method to set or retrieve the DNS View object.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

An Infoblox::DNS::View object.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Getting view
my $view = $ordered_rpz->view();
#Modifying view
$ordered_rpz->view($view1);
```

rp_zones( )

Use this method to set or retrieve an ordered list of Response Policy Zones object references.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

An array reference of defined Infoblox::DNS::Zone objects.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Getting zones list reference
my $zone_list_ref = $ordered_rpz->rp_zones();
#Getting zones list
my @zone_list = @{$oz->rp_zones()};
#Get Zone from list
my $zone = $zone_list[0];
#Modifying zones
$ordered_rpz->rp_zones([$rpz1, $rpz2, @rpz3]);
```

SAMPLE CODE

The following sample code demonstrates the different functions that can be applied to an object, such as add, search, modify, and remove. This sample also includes error handling for the operations.

#Preparation prior to the Response Policy Zone reordering

```
#PROGRAM STARTS: Include all the modules that will be used
use strict;
use Infoblox;

#Create a session to the Infoblox appliance
my $host_ip =  "192.168.1.2";
my $session = Infoblox::Session->new(
    master   => $host_ip,
    username => "admin",
    password => "infoblox"
    );
unless($session){
        die("Constructor for session failed: ",
               Infoblox::status_code(). ":" . Infoblox::status_detail());
}
print "Session created successfully.\n";

my $memberns1 = Infoblox::DNS::Member->new(
   name     => $host_name,
   ipv4addr => $node_ip,
);

my $rpz_zone = Infoblox::DNS::Zone->new (
   name     => 'rpz.com',
   comment  => "this is a demo zone 1",
   disable  => "false",
   rpz_policy => "GIVEN",
   primary => $memberns1,
);

$session->add($rpz_zone)
   or die("Add zone failed: ",
              $session->status_code(). ":" .$session->status_detail());

print"Response policy zone added successfully.\n";

$rpz_zone = Infoblox::DNS::Zone->new (
   name     => 'rpz1.com',
   comment  => "this is a demo zone 2",
   disable  => "false",
   rpz_policy => "GIVEN",
   primary => $memberns1,
);

$session->add($rpz_zone)
   or die("Add zone failed: ",
              $session->status_code(). ":" .$session->status_detail());

print"Response policy zone added successfully.\n";

$rpz_zone = Infoblox::DNS::Zone->new (
   name     => 'rpz2.com',
   comment  => "this is a demo zone 3",
   disable  => "false",
   rpz_policy => "GIVEN",
   primary => $memberns1,
);

$session->add($rpz_zone)
   or die("Add zone failed: ",
              $session->status_code(). ":" .$session->status_detail());

print"Response policy zone added successfully.\n";
```

#Get the current Response Policy Zones order.

```
my $orpz = $session->get(
    object  => "Infoblox::DNS::OrderedResponsePolicyZones",
    view => "default",
);

unless($orpz){
      die("Get Response Policy Zones failed: ",
               $session->status_code() . ":" . $session->status_detail());
```

```
      }

print "Get Response Policy Zones object found at least 1 matching entry\n";

my ($z1, $z2, $z3) = @{$orpz->rp_zones()};

print "The current order of the Response Policy Zones is : " . $z1->comment() . "," .
              $z2->comment() . "," . $z3->comment() . "\n";
```

#Modify the Response Policy Zones order.

```
$orpz->rp_zones([$z3, $z2, $z1]);

#Apply the change
$session->modify($orpz)
     or die("Modify Response Policy Zones failed: ",
              $session->status_code(). ":" .$session->status_detail());

print "Response Policy Zones modified successfully.\n";

$orpz = $session->get(
   object  => "Infoblox::DNS::OrderedResponsePolicyZones",
   view => "default",
);

unless($orpz){
     die("Get Response Policy Zones failed: ",
              $session->status_code() . ":" . $session->status_detail());
     }

print "Get Response Policy Zones object found at least 1 matching entry\n";

($z1, $z2, $z3) = @{$orpz->rp_zones()};

print "The current order of the Response Policy Zones is : " . $z1->comment() . "," .
              $z2->comment() . "," . $z3->comment() . "\n";

####PROGRAM ENDS####
```

AUTHOR

Infoblox Inc. http://www.infoblox.com

SEE ALSO

Infoblox::DNS::View, Infoblox::DNS::Zone, Infoblox::Session, Infoblox::Session->add(), Infoblox::Session->get(), Infoblox::Session->modify()

COPYRIGHT

Copyright (c) 2017 Infoblox Inc.

Infoblox::DNS::Record::A - DNS A record object.

- NAME
- DESCRIPTION
- CONSTRUCTOR
- SESSION METHODS
  - Infoblox::Session->add( )
  - Infoblox::Session->get( )
  - Infoblox::Session->modify( )
  - Infoblox::Session->remove( )
  - Infoblox::Session->search( )
- METHODS
  - comment( )
  - cloud_info( )
  - creator( )
  - creation_time( )
  - ddns_principal( )
  - ddns_protected( )
  - disable( )
  - discovered_data( )
  - discovered_name( )
  - discoverer( )
  - dns_name( )
  - first_discovered( )
  - forbid_reclamation( )
  - extattrs( )
  - extensible_attributes( )
  - ipv4addr( )
  - last_discovered( )
  - last_queried( )
  - mac_address( )
  - ms_ad_user_data( )
  - name( )
  - netbios( )
  - network_component_description( )
  - network_component_ip( )
  - network_component_name( )
  - network_component_port_description( )
  - network_component_port_name( )
  - network_component_port_number( )
  - network_component_type( )
  - os( )
  - port_duplex( )
  - port_link_status( )
  - port_speed( )
  - port_status( )
  - port_vlan_description( )
  - port_vlan_name( )
  - port_vlan_number( )
  - reclaimable( )
  - ttl( )
  - views( )
  - v_cluster( )
  - v_datacenter( )
  - v_host( )
  - v_name( )
  - v_netadapter( )
  - v_switch( )
  - v_type( )
  - zone( )
- SAMPLE CODE
- AUTHOR
- SEE ALSO
- COPYRIGHT

NAME

Infoblox::DNS::Record::A - DNS A record object.

DESCRIPTION

An A (address) record maps a domain name to an IPv4 address. To define a specific name-to-address mapping, add an A record to a previously defined authoritative forward-mapping zone.

Note that you must specify only one view for the attribute "views".

CONSTRUCTOR

```
 my $bind_a = Infoblox::DNS::Record::A->new (
     ipv4addr              => $ipv4addr,                                                                  #
Required
     name                  => $fqdn,                                                                      #
Required
     comment               => $string,                                                                    #
Optional / Default is empty
     disable               => "true" | "false"                                                            #
Optional / Default "false"
     extattrs              => { $string => $extattr, ... },                                               #
Optional / Default is undefined
     extensible_attributes => { $string => $string | $num, $string => [ $string | $num, ... ], ... },   #
Optional / Default is undefined
     last_discovered       => $string,                                                                    #
Optional / Default is empty
     ttl                   => $num | undef,                                                               #
Optional / Default is undefined
     views                 => [$View1],                                                                   #
Optional / Default is "default" view
     creator               => "DYNAMIC" | "STATIC",                                                       #
Optional / Default is "STATIC"
     ddns_protected        => "true" | "false",                                                           #
Optional / Default is "false"
     ddns_principal        => $string,                                                                    #
Optional / Default is undefined
     forbid_reclamation    => "true" | "false",                                                           #
Optional / Default is "false"
 );
```

You cannot set both extattrs and extensible_attributes attributes at the same time.

SESSION METHODS

This section describes all the methods in Infoblox::Session module that you can apply to a DNS A object.

Infoblox::Session->add( )

Use this method to add an object to the Infoblox appliance. See Infoblox::Session->add() for parameters and return values.

Example

```
#Construct an object
my $bind_a = Infoblox::DNS::Record::A->new (
    ipv4addr => "1.1.1.1",
    name     => "bind_a.domain.com",
    comment  => "this is a demo bind_a record",
    disable  => "true",
    ttl      => "33",
    views    => [$view1]  );
# Submit for addition
my $response = $session->add( $bind_a );
```

Infoblox::Session->get( )

Use this method to retrieve all the matching objects from the Infoblox appliance. See Infoblox::Session->get() for parameters and return values.

Key References

```
Apply the following attributes to get a specific DNS A object:

  name                  - Optional. A host name in string format.
  view                  - Optional. The DNS view in which the A record is located. By default, all
DNS views are searched. But if you omit this attribute and specify a zone, the appliance searches the
'default' view only.
  zone                  - Optional. A zone name in FQDN format.
  ipv4addr              - Optional. The address of the host record.
  reclaimable           - Optional. The flag that indicates whether the record is reclaimable or not.
  extattrs              - Optional. A hash reference containing extensible attributes.
  extensible_attributes - Optional. A hash reference containing extensible attributes.
  creator               - Optional. Record creator.
  ddns_principal        - Optional. GSS-TSIG principal that owns the record.
```

```
  v_cluster             - Optional. Discovered VMware cluster name.
  v_datacenter          - Optional. Discovered VMware datacenter name.
  v_host                - Optional. Discovered VMware server name.
  v_name                - Optional. Discovered VMware host or virtual machine name.
  v_netadapter          - Optional. Discovered VMware physical network adapter name.
  v_switch              - Optional. Discovered VMware virtual switch name.
  v_type                - Optional. Discovered VMware entity type (possible values: "VirtualMachine",
"HostSystem", "VirtualCenter").
```

Example

```
my @retrieved_objs = $session->get(
    object => "Infoblox::DNS::Record::A",
    name   => "bind_a.domain.com",
    view   => "default" );

#get all DNS A recods with the extensible attribute 'Site'
my @retrieved_objs = $session->get(
   object => "Infoblox::DNS::Record::A",
   extensible_attributes => { 'Site' => 'Santa Clara' });

# get DNS A records by v_type:
my @retrieved_objs = $session->get(
   object    => 'Infoblox::DNS::Record::A',
   v_type    => 'VirtualMachine');
```

Infoblox::Session->modify( )

Use this method to modify an object in the Infoblox appliance. See Infoblox::Session->modify() for parameters and return values.

Example

```
# Use method to modify the comment.
$bind_a->comment("this is a modified comment");
# Submit modification
my $response = $session->modify( $bind_a );
```

Infoblox::Session->remove( )

Use this method to submit to remove an object from the Infoblox appliance. See Infoblox::Session->remove() for parameters and return values.

To remove a specifc object, first use `get()` or `search()` to retrieve the specific object, and then submit this object for removal.

Key References

```
remove_options - Optional. A hash reference containing the following attributes:

+-------------------------------------------------------------------------------------------+
| remove_associated_ptr - Optional. Boolean flag that indicates whether the associated PTR  |
|                         record should be removed while deleting the specified A record.   |
+-------------------------------------------------------------------------------------------+
```

Example

```
# Get the objects with the same name
my @retrieved_objs = $session->get(
    object => "Infoblox::DNS::Record::A",
    name   => "bind_a.domain.com",
    view   => "default" );
# Find the desired object from the retrieved list.
my $desired_a = $retrieved_objs[0];
# Submit for removal
my $response = $session->remove(
    $desired_a,
    remove_options => {'remove_associated_ptr' => 'true'} );
```

Infoblox::Session->search( )

Use this method to search for DNS A record objects in the Infoblox appliance. See Infoblox::Session->search() for para meters and return values.

Key References

```
Apply the following attributes to search for a DNS A record object:

  name                  - Optional. A host name in string format (regular expression).
  view                  - Optional. The DNS view in which the A record is located. By default, all
DNS views are searched. But if you omit this attribute and specify a zone, the appliance searches the
'default' view only.
  zone                  - Optional. A zone name in FQDN format.
```

```
  comment               - Optional. A comment in string format (regular expression).
  ipv4addr              - Optional. The address of the host record (regular expression).
  reclaimable           - Optional. The flag that indicates whether the record is reclaimable or not.
  extattrs              - Optional. A hash reference containing extensible attributes.
  extensible_attributes - Optional. A hash reference containing extensible attributes.
  creator               - Optional. Record creator.
  ddns_principal        - Optional. GSS-TSIG principal that owns the record.
  v_cluster             - Optional. Discovered VMware cluster name (regular expression).
  v_datacenter          - Optional. Discovered VMware datacenter name (regular expression).
  v_host                - Optional. Discovered VMware server name (regular expression).
  v_name                - Optional. Discovered VMware host or virtual machine name (regular
expression).
  v_netadapter          - Optional. Discovered VMware physical network adapter name (regular
expression).
  v_switch              - Optional. Discovered VMware virtual switch name (regular expression).
  v_type                - Optional. Discovered VMware entity type (possible values: "VirtualMachine",
"HostSystem", "VirtualCenter").
```

For more information about searching extensible attributes, see Infoblox::Grid::ExtensibleAttributeDef/Searching Extensible Attributes.

Example

```
# search for all DNS A objects that match "domain.com" in the default DNS view
my @retrieved_objs = $session->search(
    object => "Infoblox::DNS::Record::A",
    name   => 'domain\.com',
    view   => "default" );

# search for all DNS A records in the "domain.com" zone of the default view
my @retrieved_objs = $session->search(
    object => "Infoblox::DNS::Record::A",
    zone   => "domain.com",
    view   => "default" );

#get all DNS A recods with the extensible attribute 'Site'
my @retrieved_objs = $session->search(
  object => "Infoblox::DNS::Record::A",
  extensible_attributes => { 'Site' => 'Santa Clara' });

# search DNS A records by v_datacenter:
my @retrieved_objs = $session->search(
   object       => "Infoblox::DNS::Record::A",
   v_datacenter => "test.*");
```

METHODS

This section describes all the methods that you can use to configure and retrieve the attribute values of an A record

comment( )

Use this method to set or retrieve the descriptive comment.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

Desired comment in string format with a maximum of 256 bytes.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get comment
my $comment = $bind_a->comment();
#Modify comment
$bind_a->comment("Modifying the DNS A comment");
```

cloud_info( )

Use this method to retrieve cloud API related information for the Infoblox::DNS::Record::A object.

Parameter

None

Returns

The method returns the attribute value.

Example

```
# Get cloud_info
my $cloud_info = $object->cloud_info();
```

creator( )

Use this method to set or retrieve the record creator.

Note that changing creator from or to 'SYSTEM' value is not allowed.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Paramter

The valid values are 'STATIC' and 'DYNAMIC'. The default value is 'STATIC'.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get creator value
my $creator = $object->creator();

#Modify creator value
$object->creator("DYNAMIC");
```

creation_time( )

Use this method to retrieve the creation time for the record. This is a read-only attribute.

Parameter

None

Returns

The valid return value is a number of seconds that have elapsed since January 1st, 1970 UTC.

Example

```
#Get creation_time value
my $creation_time = $object->creation_time();
```

ddns_principal( )

Use this method to set or retrive the GSS-TSIG principal that owns this record.

Note that you cannot set ddns_principal for 'STATIC' and 'SYSTEM' records.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Paramter

The GSS-TSIG principal FQDN (Fully Qualified Domain Name) format. The FQDN consists of the hostname followed by the domain name (example: abc.com). A hostname can have a maximum of 256 characters.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get ddns_principal value
my $ddns_principal = $object->ddns_principal();

#Modify ddns_principal value
$object->ddns_principal('foo.com');
```

ddns_protected( )

Use this method to set or retrieve the flag that indicates whether DDNS updates for this record are allowed or not.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

Specify 'true' to protect record from DDNS updates and 'false' to allow DDNS updates for the specified record.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get ddns_protected value
my $ddns_protected = $object->ddns_protected();

#Modify ddns_protected value
$object->ddns_protected('true');
```

disable( )

Use this method to set or retrieve the disable flag of a DNS record.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

The default value for this field is false. The DNS record is enabled.

Parameter

Specify "true" to set the disable flag or "false" to deactivate/unset it.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get disable
my $disable = $bind_a->disable();
#Modify disable
$bind_a->disable("true");
```

discovered_data( )

Use this method to retrieve the discovered data of an A Record object. This is a read-only attribute.

Omit the parameter to retrieve the attribute value.

Parameter

None

Returns

The valid return value is an Infoblox::Grid::Discovery::Data object.

Example

```
#Get discovered_data value
my $discovered_data = $object->discovered_data();
```

discovered_name( )

Use this method to retrieve the discovered name of an A Record object.

Parameter

none

Returns

The method returns the discovered name.

Example

```
#Get discovered_name
my $discovered_name = $bind_a->discovered_name();
```

discoverer( )

Use this method to retrieve the discoverer of an A Record object.

Parameter

none

Returns

The method returns the discoverer value.

Example

```
#Get discoverer
my $discoverer = $bind_a->discoverer();
```

dns_name( )

Use this method to retrieve the host name in punycode format. This is a read-only attribute.

Parameter

None

Returns

The method returns the attribute value.

Example

```
# Get attribute value
my $value = $bind_a->dns_name();
```

first_discovered( )

Use this method to retrieve the date and time that the A Record object was first discovered.

Parameter

none

Returns

The method returns the date and time when the A Record object was first discovered.

Example

```
#Get first_discovered
my $first_discovered = $bind_a->first_discovered();
```

forbid_reclamation( )

Use this method to set or retrieve the flag that indicates whether the reclamation is allowed for the record or not.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

Specify 'true' to forbid reclamation for the record and 'false' to allow it. The default value is 'false'.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get forbid_reclamation
my $forbid_reclamation = $object->forbid_reclamation();
#Modify forbid_reclamation
```

```
$object->forbid_reclamation('true');
```

extattrs( )

Use this method to set or retrieve the extensible attributes associated with a DNS A record object.

Parameter

Valid value is a hash reference containing the names of extensible attributes and their associated values (Infoblox::Grid::Extattr objects).

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get extattrs
my $ref_extattrs = $bind_a->extattrs();
#Modify extattrs
$bind_a->extattrs({ 'Site' => $extattr1, 'Administrator' => $extattr2 });
```

extensible_attributes( )

Use this method to set or retrieve the extensible attributes associated with a DNS A record.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

For valid values for extensible attributes, see Infoblox::Grid::ExtensibleAttributeDef/Extensible Attribute Values.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get extensible attributes
my $ref_extensible_attributes = $bind_a->extensible_attributes();
#Modify extensible attributes
$bind_a->extensible_attributes({ 'Site' => 'Santa Clara', 'Administrator' => [ 'Peter', 'Tom' ] });
```

ipv4addr( )

Use this method to set or retrieve the the IPv4 address.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

An IPv4 address is a 32-bit number in dotted decimal notation. It consists of four 8-bit groups of decimal digits separated by decimal points (example: 192. 168.1.2).

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get ipv4addr
my $ipv4addr = $bind_a->ipv4addr();
#Modify ipv4addr
$bind_a->ipv4addr("2.2.2.2");
```

last_discovered( )

Use this method to retrieve the time this object was last seen by a discovery job.

The default value for this field is empty.

Parameter

N/A

Returns

The method returns the attribute value.

Example

```
#Get last_discovered
my $last_discovered = $bind_a->last_discovered();
```

last_queried( )

Use this method to retrieve the time when the associated record was last queried. This is a read-only attribute.

Parameter

None

Returns

The method returns the attribute value. The number of seconds that have elapsed since January 1st, 1970 UTC.

Example

```
#Get last_queried
my $last_queried = $bind_a->last_queried();
```

mac_address( )

Use this method to retrieve the discovered MAC address of this object.

The default value for this field is empty.

Parameter

N/A

Returns

The method returns the attribute value.

Example

```
#Get mac_address
my $mac_address = $bind_a->mac_address();
```

ms_ad_user_data( )

Use this method to retrieve Microsoft Active Directory users related information. This is a read-only attribute.

Parameter

None

Returns

The valid return value is an Infoblox::Grid::MSServer::AdUser::Data object.

Example

```
#Get ms_ad_user_data
my $ms_ad_user_data = $bind_a->ms_ad_user_data();
```

name( )

Use this method to set or retrieve the host name.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

The attribute value can be in unicode format.

Parameter

Hostname in FQDN (Fully Qualified Domain Name) format. The FQDN consists of the hostname followed by the domain name (example: abc.com). A hostname can have a maximum of 256 bytes.

Infoblox also supports wildcard A records. A wildcard A record maps all the hostnames in a domain to a single IP address.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get name
my $name = $bind_a->name();
#Modify name
$bind_a->name("bind_a.domain.com");
```

netbios( )

Use this method to retrieve the name in the NetBIOS reply that responded to a NetBIOS query.

Returns

The method returns the attribute value.

Example

```
#Get netbios
my $netbios = $bind_a->netbios();
```

network_component_description( )

Use this method to retrieve the description of the network device that is connected to the A Record object.

Parameter

none

Returns

The method returns the network device description.

Example

```
#Get network_component_description
my $network_component_description = $bind_a->network_component_description();
```

network_component_ip( )

Use this method to retrieve the IP address of the network device that is connected to the A Record object.

Parameter

none

Returns

The method returns the network device IP address.

Example

```
#Get network_component_ip
my $network_component_ip = $bind_a->network_component_ip();
```

network_component_name( )

Use this method to retrieve the name of the network device that is connected to the A Record object.

Parameter

none

Returns

The method returns the network device name.

Example

```
#Get network_component_name
my $network_component_name = $bind_a->network_component_name();
```

network_component_port_description( )

Use this method to retrieve the description of the network device port that is connected to the A Record object.

Parameter

none

Returns

The method returns the network device port description.

Example

```
#Get network_component_port_description
my $network_component_port_description = $bind_a->network_component_port_description();
```

network_component_port_name( )

Use this method to retrieve the name of the network device port that is connected to the A Record object.

Parameter

none

Returns

The method returns the name of the port on the network device.

Example

```
#Get network_component_port_name
my $network_component_port_name = $bind_a->network_component_port_name();
```

network_component_port_number( )

Use this method to retrieve the number of the network device port that is connected to the A Record object.

Parameter

none

Returns

The method returns the network device port number.

Example

```
#Get network_component_port_number
my $network_component_port_number = $bind_a->network_component_port_number();
```

network_component_type( )

Use this method to retrieve the type of the network device that is connected to the A Record object.

Parameter

none

Returns

The method returns the network device type.

Example

```
#Get network_component_type
my $network_component_type = $bind_a->network_component_type();
```

os( )

Use this method to retrieve the operating system associated with this A record.

Returns

The method returns the attribute value.

Example

```
#Get os
my $os = $bind_a->os();
```

port_duplex( )

Use this method to retrieve the duplex setting of the network device port that is connected to the A Record object.

Parameter

none

Returns

The method returns the network device port duplex setting. Returned values, if any, are one of the following:

```
Full
Half
```

Example

```
#Get port_duplex
my $port_duplex = $bind_a->port_duplex();
```

port_link_status( )

Use this method to retrieve the link status of the network device port that is connected to the A Record object.

Parameter

none

Returns

The method returns the port link status. Returned values, if any, are one of the following:

```
Connected
Not Connected
Unknown
```

Example

```
#Get port_link_status
my $port_link_status = $bind_a->port_link_status();
```

port_speed( )

Use this method to retrieve the speed of the network device port that is connected to the A Record object.

Parameter

none

Returns

The method returns the network device port speed value. Returned values, if any, are one of the following:

```
10M
100M
1G
10G
100G
Unknown
```

Example

```
#Get port_speed
my $port_speed = $bind_a->port_speed();
```

port_status( )

Use this method to retrieve the status of the network device port that is connected to the A Record object.

Parameter

none

Returns

The method returns the network device port status. Returned values, if any, are one of the following:

```
Up
Down
Unknown
```

Example

```
#Get port_status
my $port_status = $bind_a->port_status();
```

port_vlan_description( )

Use this method to retrieve the VLAN description of the network device port that is connected to the A Record object.

Parameter

none

Returns

The method returns the VLAN description of the network device port.

Example

```
#Get port_vlan_description
my $port_vlan_description = $bind_a->port_vlan_description();
```

port_vlan_name( )

Use this method to retrieve the VLAN name of the network device port that is connected to the A Record object.

Parameter

none

Returns

The method returns the VLAN name of the network device port.

Example

```
#Get port_vlan_name
my $port_vlan_name = $bind_a->port_vlan_name();
```

port_vlan_number( )

Use this method to retrieve the VLAN number of the network device port that is connected to the A Record object.

Parameter

none

Returns

The method returns the network device port VLAN number.

Example

```
#Get port_vlan_number
my $port_vlan_number = $bind_a->port_vlan_number();
```

reclaimable( )

Use this method to retrieve the flag that indicates whether the record is reclaimable or not.

Parameter

None

Returns

The method returns the attribute value.

Example

```
#Get reclaimable
my $reclaimable = $object->reclaimable();
```

ttl( )

Use this method to set or retrieve the Time to Live (TTL) value.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

The default value is undefined which indicates that the record inherits the TTL value of the zone.

Specify a TTL value to override the TTL value at the zone level.

Parameter

A 32-bit integer (range from 0 to 4294967295) that represents the duration in seconds that the record is cached. Zero indicates that the record should not be cached.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get ttl
my $ttl = $bind_a->ttl();
#Modify ttl
$bind_a->ttl(1800);
#Un-override ttl
$bind_a->ttl(undef);
```

views( )

Use this method to set or retrieve the view of the A record.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

The default value is the "default" view, which means the A record is located under the default view.

Parameter

Array reference of defined Infoblox::DNS::View objects.

Note that the array size must be 1.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get views
my $ref_views = $bind_a->views();
#Modify views, list of Infoblox::DNS::View objects
$bind_a->views([$view1]);
```

v_cluster( )

Use this method to retrieve the name of the VMware cluster associated with the A Record object. This method is read-only.

Parameter

none

Returns

This method returns a string that contains the VMware cluster name. The default value is an empty string.

Example

```
#Get v_cluster
my $v_cluster = $bind_a->v_cluster();
```

v_datacenter( )

Use this method to retrieve the name of the VMware datacenter associated with the A Record object. This method is read-only.

Parameter

none

Returns

This method returns a string that contains the VMware datacenter name. The default value is an empty string.

Example

```
#Get v_datacenter
my $v_datacenter = $bind_a->v_datacenter();
```

v_host( )

Use this method to retrieve the name of the VMware host associated with the A Record object. This method is read-only.

Parameter

none

Returns

This method returns a string that contains the VMware host name. The default value is an empty string.

Example

```
#Get v_host
my $v_host = $bind_a->v_host();
```

v_name( )

Use this method to retrieve the name of the VMware entity associated with the A Record object. This method is read-only.

Parameter

none

Returns

This method returns a string that contains the VMware entity name. The default value is an empty string.

Example

```
#Get v_name
my $v_name = $bind_a->v_name();
```

v_netadapter( )

Use this method to retrieve the name of the physical network adapter through which the VMware entity is connected to the appliance. This method is read-only.

Parameter

none

Returns

This method returns a string that contains the network adapter name. The default value is an empty string.

Example

```
#Get v_netadapter
my $v_netadapter = $bind_a->v_netadapter();
```

v_switch( )

Use this method to retrieve the name of the virtual switch through which the VMware entity is connected to the appliance. This method is read-only.

Parameter

none

Returns

This method returns a string that contains the virtual switch name. The default value is an empty string.

Example

```
#Get v_switch
```

```
my $v_switch = $bind_a->v_switch();
```

v_type( )

Use this method to retrieve the type of VMware entity associated with the A Record object. This method is read-only.

Parameter

none

Returns

This method returns a string that contains the VMware entity type. The default value is an empty string. The returned value, if any, can be one of the following:

```
VirtualMachine
HostSystem
VirtualCenter
```

Example

```
#Get v_type
my $v_type = $bind_a->v_type();
```

zone( )

Use this method to retrieve the zone name of a DNS A record. This method is read-only and cannot be set.

Parameter

None

Returns

Returns the attribute value.

Example

```
# Get zone
my $zone = $bind_a->zone();
```

SAMPLE CODE

The following sample code demonstrates the different functions that can be applied to an object, such as add, search, modify, and remove. This sample also includes error handling for the operations.

#Preparation prior to a DNS A record insertion

```
#PROGRAM STARTS: Include all the modules that will be used
use strict;
use Infoblox;

#Create a session to the Infoblox Appliance
my $session = Infoblox::Session->new(
    master   => "192.168.1.2",
    username => "admin",
    password => "infoblox"
);
unless ($session) {
   die("Construct session failed: ",
       $session->status_code() . ":" . $session->status_detail());
}
print "Session created successfully\n";

#Create the zone prior to an A record insertion
my $zone = Infoblox::DNS::Zone->new(name => "domain.com");
unless ($zone) {
   die("Construct zone failed: ",
       Infoblox::status_code() . ":" . Infoblox::status_detail());
}
print "Zone object created successfully\n";

#Verify if the zone exists
my $object = $session->get(object => "Infoblox::DNS::Zone", name => "domain.com");
unless ($object) {
   print "Zone does not exist on server, safe to add the zone\n";
   $session->add($zone)
```

```
        or die("Add zone failed: ",
               $session->status_code() . ":" . $session->status_detail());
}
print "Zone added successfully\n";
```

#Create a DNS A record

```
#Construct a DNS A object
my $bind_a = Infoblox::DNS::Record::A->new(
    name     => "bind_a.domain.com",
    comment  => " this is a demo bind_a record ",
    ipv4addr => "1.1.1.1"
);
unless ($bind_a) {
   die("Construct DNS record A failed: ",
       Infoblox::status_code() . ":" . Infoblox::status_detail());
}
print "DNS A object created successfully\n";

#Add the DNS A record object to Infoblox Appliance through a session
$session->add($bind_a)
    or die("Add record A failed: ",
           $session->status_code() . ":" . $session->status_detail());
print "DNS A object added to server successfully\n";
```

#Search for a specific DNS A record

```
#Search all A records that match "domain.com"
my @retrieved_objs = $session->search(
    object => "Infoblox::DNS::Record::A",
    name   => 'domain\.com'
);
my $object = $retrieved_objs[0];
unless ($object) {
    die("Search record A failed: ",
        $session->status_code() . ":" . $session->status_detail());
}
print "Search DNS A object found at least 1 matching entry\n";

#Search all A records that start with "bind" and end with ".domain.com"
my @retrieved_objs = $session->search(
    object => "Infoblox::DNS::Record::A",
    name   => '^bind.*\.domain\.com\$'
);
my $object = $retrieved_objs[0];
unless ($object) {
    die("Search record A failed: ",
        $session->status_code() . ":" . $session->status_detail());
}
print "Search DNS A object using regexp found at least 1 matching entry\n";
```

#Get and modify a DNS A record

```
#Get A record through the session
my @retrieved_objs = $session->get(
    object => "Infoblox::DNS::Record::A",
    name   => "bind_a.domain.com"
);
my $object = $retrieved_objs[0];
unless ($object) {
    die("Get record A failed: ",
        $session->status_code() . ":" . $session->status_detail());
}
print "Get DNS A object found at least 1 matching entry\n";

#Modify one of the attributes of the specified A record
$object->ipv4addr("3.3.3.3");

#Apply the changes
$session->modify($object)
    or die("Modify record A failed: ",
           $session->status_code() . ":" . $session->status_detail());
print "DNS A object modified successfully \n";
```

#Remove a DNS A record

```
#Get A record through the session
```

```
my @retrieved_objs = $session->get(
    object => "Infoblox::DNS::Record::A",
    name   => "bind_a.domain.com"
);
my $object = $retrieved_objs[0];
unless ($object) {
    die("Get record A failed: ",
        $session->status_code() . ":" . $session->status_detail());
}
print "Get DNS A object found at least 1 matching entry\n";

#Submit the object for removal
$session->remove($object)
    or die("Remove record A failed: ",
        $session->status_code() . ":" . $session->status_detail());
print "DNS A object removed successfully \n";

####PROGRAM ENDS####
```

AUTHOR

Infoblox Inc. http://www.infoblox.com/

SEE ALSO

Infoblox::DNS::View, Infoblox::DNS::Zone, Infoblox::Session, Infoblox::Session->get(), Infoblox::Session->search(), Infoblox::Session->add(), Infoblox::Session->remove(), Infoblox::Session->modify(), Infoblox::Grid::MSServer::AdUser::Data, Infoblox::Grid::Discovery::Data

---

COPYRIGHT

Copyright (c) 2017 Infoblox Inc.

Infoblox::DNS::Record::AAAA - DNS AAAA record object.

- NAME
- DESCRIPTION
- CONSTRUCTOR
- SESSION METHODS
  - Infoblox::Session->add( )
  - Infoblox::Session->get( )
  - Infoblox::Session->modify( )
  - Infoblox::Session->remove( )
  - Infoblox::Session->search( )
- METHODS
  - comment( )
  - cloud_info( )
  - creator( )
  - creation_time( )
  - ddns_principal( )
  - ddns_protected( )
  - disable( )
  - discovered_data( )
  - discovered_duid( )
  - discovered_name( )
  - discoverer( )
  - dns_name( )
  - extattrs( )
  - extensible_attributes( )
  - first_discovered( )
  - forbid_reclamation( )
  - ipv6addr( )
  - last_discovered( )
  - last_queried( )
  - ms_ad_user_data( )
  - name( )
  - netbios( )
  - network_component_description( )
  - network_component_ip( )
  - network_component_name( )
  - network_component_port_description( )
  - network_component_port_name( )
  - network_component_port_number( )
  - network_component_type( )
  - os( )
  - port_duplex( )
  - port_link_status( )
  - port_speed( )
  - port_status( )
  - port_vlan_description( )
  - port_vlan_name( )
  - port_vlan_number ( )
  - reclaimable( )
  - ttl( )
  - views( )
  - zone( )
- SAMPLE CODE
- AUTHOR
- SEE ALSO
- COPYRIGHT

NAME

Infoblox::DNS::Record::AAAA - DNS AAAA record object.

DESCRIPTION

An AAAA record, also known as a quad A address record, maps a domain name to an IPv6 address. To define a specific name-to-address mapping, add an AAAA record to a previously defined authoritative forward-mapping zone.

Note that you must specify only one view for the attribute "views".

CONSTRUCTOR

```
 my $bindaaaa = Infoblox::DNS::Record::AAAA->new (
     ipv6addr              => $ipv6addr,                                                  #
Required
     name                  => $fqdn,                                                      #
Required
     comment               => $string,                                                    #
Optional / Default is empty
     extattrs              => { $string => $extattr, ... },                               #
Optional / Default is undefined
     extensible_attributes => { $string => $string | $num, $string => [ $string | $num, ... ], ... }, #
Optional / Default is undefined
     disable               => "true" | "false"                                            #
Optional / Default "false"
     ttl                   => $num | undef,                                               #
Optional / Default is undefined
     views                 => [$View1],                                                   #
Optional / Default is "default" view
     creator               => "DYNAMIC" | "STATIC",                                       #
Optional / Default is "STATIC"
     ddns_protected        => "true" | "false",                                           #
Optional / Default is "false"
     ddns_principal        => $string,                                                    #
Optional / Default is undefined
     forbid_reclamation    => "true" | "false",                                           #
Optional / Default is "false"
 );
```

You cannot set both extattrs and extensible_attributes attributes at the same time.

SESSION METHODS

This section describes all the methods in Infoblox::Session module that you can apply to a DNS AAAA record object.

Infoblox::Session->add( )

Use this method to add a AAAA record object to the Infoblox appliance. See Infoblox::Session->add() for parameters and return values.

Example

```
#Construct an object
my $bindaaaa = Infoblox::DNS::Record::AAAA->new(
    name     => "aaaa1.domain.com",
    comment  => "add a DNS AAAA aaaa1.domain.com",
    ipv6addr => "aaaa:1111:bbbb:2222:cccc:3333:dddd:4444",
    disable  => "true",
    ttl      => "33",
    views    => [$view1]  );
# Submit for addition
my $response = $session->add( $bindaaaa );
```

Infoblox::Session->get( )

Use this method to retrieve all the matching objects from the Infoblox appliance. See Infoblox::Session->get() for parameters and return values.

Key References

```
 Apply the following attributes to get a specific DNS AAAA object:

 name                              - Optional. A host name in string format.
 view                              - Optional. The DNS view in which the host record is located. By
default, all DNS views are searched. But if you omit this attribute and specify a zone, the appliance
searches the 'default' view only.
 zone                              - Optional. A zone name in FQDN format.
 ipv6addr                          - Optional. The address of the AAAA record.
 creator                           - Optional. Record creator.
 ddns_principal                    - Optional. GSS-TSIG principal that owns the record.
 reclaimable                       - Optional. The flag that indicates whether the record is
reclaimable or not.
 extattrs                          - Optional. A hash reference containing extensible attributes.
 extensible_attributes             - Optional. A hash reference containing extensible attributes.

 discovered_duid                   - Optional. Discovered DHCPv6 UID.
 discovered_name                   - Optional. Discovered name.
 discoverer                        - Optional. Discoverer name.
 first_discovered                  - Optional. Time of the first discovery.
 last_discovered                   - Optional. Time of the last discovery.
 netbios                           - Optional. Discovered NetBIOS name.
 network_component_description     - Optional. Discovered network component description.
 network_component_ip              - Optional. Discovered network component IP address.
 network_component_name            - Optional. Discovered network component name.
 network_component_port_description - Optional. Discovered network component port description.
```

```
network_component_port_name        - Optional. Discovered network component port name.
network_component_port_number      - Optional. Discovered network component port number.
network_component_type             - Optional. Discovered network component type.
os                                 - Optional. Discovered OS.
port_duplex                        - Optional. Discovered port duplex settings.
port_link_status                   - Optional. Discovered port link status.
port_speed                         - Optional. Discovered port speed settings.
port_status                        - Optional. Discovered port status settings.
port_vlan_description              - Optional. Discovered port VLAN description.
port_vlan_name                     - Optional. Discovered port VLAN name.
port_vlan_number                   - Optional. Discovered port VLAN number.
```

Example

```
my @retrieved_objs = $session->get(
                      object => "Infoblox::DNS::Record::AAAA",
                      name   => "aaaa1.domain.com",
                      view   => "default" );

# get all the DNS AAAA records in the "domain.com" zone of the default view
my @retrieved_objs = $session->get(
                      object => "Infoblox::DNS::Record::AAAA",
                      zone   => "domain.com",
                      view   => "default" );

#get all DNS AAAA recods with the extensible attribute 'Site'
my @retrieved_objs = $session->get(
   object => "Infoblox::DNS::Record::AAAA",
   extensible_attributes => { 'Site' => 'Santa Clara' });
```

Infoblox::Session->modify( )

Use this method to modify a AAAA record object in the Infoblox appliance. See Infoblox::Session->modify() for parameters and return values.

Example

```
# Use method to modify the comment.
$bindaaaa->comment("this is a modified comment");
# Submit modification
my $response = $session->modify( $bindaaaa );
```

Infoblox::Session->remove( )

Use this method to remove a AAAA record object from the Infoblox appliance. See Infoblox::Session->remove() for parameters and return values.

To remove a specifc object, first use `get()` or `search()` to retrieve the specific AAAA record object, and then submit this object for removal.

Key References

```
remove_options - Optional. A hash reference containing the following attributes:

+-------------------------------------------------------------------------------------------+
| remove_associated_ptr - Optional. Boolean flag that indicates whether the associated PTR  |
|                         record should be removed while deleting the specified AAAA record.|
+-------------------------------------------------------------------------------------------+
```

Example

```
# Get the objects with the same name
my @retrieved_objs = $session->get(
    object => "Infoblox::DNS::Record::AAAA",
    name   => "aaaa1.domain.com",
    view   => "default" );
# Find the desired object from the retrieved list.
my $desired_aaaa = $retrieved_objs[0];
# Submit for removal
my $response = $session->remove(
    $desired_aaaa,
    remove_options => {'remove_associated_ptr' => 'true'} );
```

Infoblox::Session->search( )

Use this method to search for DNS AAAA record objects in the Infoblox appliance. See Infoblox::Session->search() for parameters and return values.

Key References

```
 Apply the following attributes, to search for a DNS AAAA record object:

  name                                  - Optional. A host name in string format (regular
expression).
  view                                  - Optional. The DNS view in which the AAAA record is located.
By default, all DNS views are searched. But if you omit this attribute and specify a zone, the
appliance searches the 'default' view only.
  zone                                  - Optional. A zone name in FQDN format.
  comment                               - Optional. Comment in string format (regular expression).
  creator                               - Optional. Record creator.
  ddns_principal                        - Optional. GSS-TSIG principal that owns the record.
  reclaimable                           - Optional. The flag that indicates whether the record is
reclaimable or not.
  ipv6addr                              - Optional. The address of the AAAA record (regular
expression).

  discovered_duid                       - Optional. Discovered DHCPv6 UID (exact search only).
  discovered_name                       - Optional. Discovered name(regex).
  discoverer                            - Optional. Discoverer name(regex).
  first_discovered                      - Optional. Time of the first discovery (in ISO format, exact
search only).
  last_discovered                       - Optional. Time of the last discovery (in ISO format, exact
search only).
  netbios                               - Optional. Discovered NetBIOS name(regex).
  network_component_description         - Optional. Discovered network component description(regex).
  network_component_ip                  - Optional. Discovered network component IP address(regex).
  network_component_name                - Optional. Discovered network component name(regex).
  network_component_port_description    - Optional. Discovered network component port
description(regex).
  network_component_port_name           - Optional. Discovered network component port name(regex).
  network_component_port_number         - Optional. Discovered network component port number (exact
search only).
  network_component_type                - Optional. Discovered network component type(regex).
  os                                    - Optional. Discovered OS (regex).
  port_duplex                           - Optional. Discovered port duplex settings (exact search
only).
  port_link_status                      - Optional. Discovered port link status (exact search only).
  port_speed                            - Optional. Discovered port speed settings (exact search
only).
  port_status                           - Optional. Discovered port status settings (exact search
only).
  port_vlan_description                 - Optional. Discovered port VLAN description(regex).
  port_vlan_name                        - Optional. Discovered port VLAN name(regex).
  port_vlan_number                      - Optional. Discovered port VLAN number(exact search only).

  extattrs                              - Optional. A hash reference containing extensible
attributes.
  extensible_attributes                 - Optional. A hash reference containing extensible
attributes.
```

For more information about searching extensible attributes, see Infoblox::Grid::ExtensibleAttributeDef/Searching Extensible Attributes.

Example

```
# search for all DNS AAAA objects that match "domain.com" in the default DNS view
my @retrieved_objs = $session->search(
    object  => "Infoblox::DNS::Record::AAAA",
    name    => 'domain\.com',
    view    => "default",
    comment => ".*modified comment" );

# search for all DNS AAAA records in zone "domain.com" of default view
my @retrieved_objs = $session->search(
                     object => "Infoblox::DNS::Record::AAAA",
                     zone   => "domain.com",
                     view   => "default" );

# search all DNS AAAA recods with the extensible attribute 'Site'
my @retrieved_objs = $session->search(
   object => "Infoblox::DNS::Record::AAAA",
   extensible_attributes => { 'Site' => 'Santa Clara' });
```

METHODS

This section describes all the methods that you can use to configure and retrieve the attribute values of an AAAA record

comment( )

Use this method to set or retrieve the descriptive comment.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

Desired comment in string format with a maximum of 256 bytes.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get comment
my $comment = $bindaaaa->comment();
#Modify comment
$bindaaaa->comment("Modifying the DNS AAAA comment");
```

cloud_info( )

Use this method to retrieve cloud API related information for the Infoblox::DNS::Record::AAAA object.

Parameter

None

Returns

The method returns the attribute value.

Example

```
# Get cloud_info
my $cloud_info = $object->cloud_info();
```

creator( )

Use this method to set or retrieve the record creator.

Note that changing creator from or to 'SYSTEM' value is not allowed.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Paramter

The valid values are 'STATIC' and 'DYNAMIC'. The default value is 'STATIC'.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get creator value
my $creator = $object->creator();

#Modify creator value
$object->creator("DYNAMIC");
```

creation_time( )

Use this method to retrieve the creation time for the record. This is a read-only attribute.

Parameter

None

Returns

The valid return value is a number of seconds that have elapsed since January 1st, 1970 UTC.

Example

```
#Get creation_time value
my $creation_time = $object->creation_time();
```

ddns_principal( )

Use this method to set or retrive the GSS-TSIG principal that owns this record.

Note that you cannot set ddns_principal for 'STATIC' and 'SYSTEM' records.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Paramter

The GSS-TSIG principal FQDN (Fully Qualified Domain Name) format. The FQDN consists of the hostname followed by the domain name (example: abc.com). A hostname can have a maximum of 256 characters.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get ddns_principal value
my $ddns_principal = $object->ddns_principal();

#Modify ddns_principal value
$object->ddns_principal('foo.com');
```

ddns_protected( )

Use this method to set or retrieve the flag that indicates whether DDNS updates for this record are allowed or not.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

Specify 'true' to protect record from DDNS updates and 'false' to allow DDNS updates for the specified record.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get ddns_protected value
my $ddns_protected = $object->ddns_protected();

#Modify ddns_protected value
$object->ddns_protected('true');
```

disable( )

Use this method to set or retrieve the disable flag of a DNS record.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

The default value for this field is false. The DNS record is enabled.

Parameter

Specify "true" to set the disable flag or "false" to deactivate/unset it.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get disable
my $disable = $bindaaaa->disable();
#Modify disable
$bindaaaa->disable("true");
```

discovered_data( )

Use this method to retrieve the discovered data of an AAAA Record object. This is a read-only attribute.

Omit the parameter to retrieve the attribute value.

Parameter

None

Returns

The valid return value is an Infoblox::Grid::Discovery::Data object.

Example

```
#Get discovered_data value
my $discovered_data = $object->discovered_data();
```

discovered_duid( )

Use this method to retrieve the discovered DHCPv6 UID (DUID) of an AAAA record object.

Parameter

none

Returns

The method returns the discovered name.

Example

```
#Get discovered_duid
my $discovered_duid = $bind_aaaa->discovered_duid();
```

discovered_name( )

Use this method to retrieve the discovered name of an AAAA Record object.

Parameter

none

Returns

The method returns the discovered name.

Example

```
#Get discovered_name
my $discovered_name = $bindaaa->discovered_name();
```

discoverer( )

Use this method to retrieve the discoverer of an AAAA Record object.

Parameter

none

Returns

The method returns the discoverer value.

Example

```
#Get discoverer
my $discoverer = $bindaaa->discoverer();
```

dns_name( )

Use this method to retrieve the host name in punycode format. This is a read-only attribute.

Parameter

None

Returns

The method returns the attribute value.

Example

```
# Get attribute value
my $value = $bindaaaa->dns_name();
```

extattrs( )

Use this method to set or retrieve the extensible attributes associated with a DNS AAAA record object.

Parameter

Valid value is a hash reference containing the names of extensible attributes and their associated values (Infoblox::Grid::Extattr objects).

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get extattrs
my $ref_extattrs = $bindaaaa->extattrs();
#Modify extattrs
$bindaaaa->extattrs({ 'Site' => $extattr1, 'Administrator' => $extattr2 });
```

extensible_attributes( )

Use this method to set or retrieve the extensible attributes associated with a DNS AAAA record.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

For valid values for extensible attributes, see Infoblox::Grid::ExtensibleAttributeDef/Extensible Attribute Values.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get extensible attributes
my $ref_extensible_attributes = $bindaaaa->extensible_attributes();
#Modify extensible attributes
$bindaaaa->extensible_attributes({ 'Site' => 'Santa Clara', 'Administrator' => [ 'Peter', 'Tom' ]
});
```

first_discovered( )

Use this method to retrieve the date and time that the AAAA Record object was first discovered.

Parameter

none

Returns

The method returns the date and time when the AAAA Record object was first discovered.

Example

```
#Get first_discovered
my $first_discovered = $bindaaa->first_discovered();
```

forbid_reclamation( )

Use this method to set or retrieve the flag that indicates whether the reclamation is allowed for the record or not.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

Specify 'true' to forbid reclamation for the record and 'false' to allow it. The default value is 'false'.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get forbid_reclamation
my $forbid_reclamation = $object->forbid_reclamation();
#Modify forbid_reclamation
$object->forbid_reclamation('true');
```

ipv6addr( )

Use this method to set or retrieve the the IPv6 address.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

An IPv6 address is a 128-bit number in colon hexadecimal notation. It consists of eight 16-bit groups of hexadecimal digits separated by colons (example: 12ab:0000:0000:0123:4567:89ab:0000:cdef).

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get ipv6addr
my ipv6addr = $bindaaaa->ipv6addr();
#Modify ipv6addr
$bindaaaa->ipv6addr("aaaa:1111:bbbb:2222:cccc:3333:dddd:5555");
```

last_discovered( )

Use this method to retrieve the time this object was last seen by a discovery job.

This field is blank by default.

Parameter

N/A

Returns

The method returns the attribute value.

Example

```
#Get last_discovered
my $last_discovered = $bindaaa->last_discovered();
```

last_queried( )

Use this method to retrieve the time when the associated record was last queried. This is a read-only attribute.

Parameter

None

Returns

The method returns the attribute value. The number of seconds that have elapsed since January 1st, 1970 UTC.

Example

```
#Get last_queried
my $last_queried = $bindaaaa->last_queried();
```

ms_ad_user_data( )

Use this method to retrieve Microsoft Active Directory users related information. This is a read-only attribute.

Parameter

None

Returns

The valid return value is an Infoblox::Grid::MSServer::AdUser::Data object.

Example

```
#Get ms_ad_user_data
my $ms_ad_user_data = $bindaaaa->ms_ad_user_data();
```

name( )

Use this method to set or retrieve the host name.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

The attribute value can be in unicode format.

Parameter

Hostname in FQDN (Fully Qualified Domain Name) format. The FQDN consists of the hostname followed by the domain name (example: abc.com). A hostname can have a maximum of 256 bytes.

Infoblox also supports wildcard AAAA records. A wildcard AAAA record maps all the hostnames in a domain to a single IP address.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get name
my $name = $bindaaaa->name();
#Modify name
$bindaaaa-> name("aaaa2.domain.com");
```

netbios( )

Use this method to retrieve the name in the NetBIOS reply that responded to a NetBIOS query.

Returns

The method returns the attribute value.

Example

```
#Get NetBIOS
my $netbios = $bindaaa->netbios();
```

network_component_description( )

Use this method to retrieve the description of the network device that is connected to the AAAA Record object.

Parameter

none

Returns

The method returns the network device description.

Example

```
#Get network_component_description
my $network_component_description = $bindaaa->network_component_description();
```

network_component_ip( )

Use this method to retrieve the IP address of the network device that is connected to the AAAA Record object.

Parameter

none

Returns

The method returns the network device IP address.

Example

```
#Get network_component_ip
```

```
my $network_component_ip = $bindaaa->network_component_ip();
```

network_component_name( )

Use this method to retrieve the name of the network device that is connected to the AAAA Record object.

Parameter

none

Returns

The method returns the network device name.

Example

```
#Get network_component_name
my $network_component_name = $bindaaa->network_component_name();
```

network_component_port_description( )

Use this method to retrieve the description of the network device port that is connected to the AAAA Record object.

Parameter

none

Returns

The method returns the network device port description.

Example

```
#Get network_component_port_description
my $network_component_port_description = $bindaaa->network_component_port_description();
```

network_component_port_name( )

Use this method to retrieve the name of the network device port that is connected to the AAAA Record object.

Parameter

none

Returns

The method returns the name of the port on the network device.

Example

```
#Get network_component_port_name
my $network_component_port_name = $bindaaa->network_component_port_name();
```

network_component_port_number( )

Use this method to retrieve the number of the network device port that is connected to the AAAA Record object.

Parameter

none

Returns

The method returns the network device port number.

Example

```
#Get network_component_port_number
my $network_component_port_number = $bindaaa->network_component_port_number();
```

network_component_type( )

Use this method to retrieve the type of the network device that is connected to the AAAA Record object.

Parameter

none

Returns

The method returns the network device type.

Example

```
#Get network_component_type
my $network_component_type = $bindaaa->network_component_type();
```

os( )

Use this method to retrieve the operating system associated with this A record.

Returns

The method returns the attribute value.

Example

```
#Get OS
my $os = $bindaaa->os();
```

port_duplex( )

Use this method to retrieve the duplex setting of the network device port that is connected to the AAAA Record object.

Parameter

none

Returns

The method returns the network device port duplex setting. The returned value, if any, is one of the following:

```
Full
Half
```

Example

```
#Get port_duplex
my $port_duplex = $bindaaa->port_duplex();
```

port_link_status( )

Use this method to retrieve the link status of the network device port that is connected to the AAAA Record object.

Parameter

none

Returns

The method returns the port link status. The returned value, if any, is one of the following:

```
Connected
Not Connected
Unknown
```

Example

```
#Get port_link_status
my $port_link_status = $bindaaa->port_link_status();
```

port_speed( )

Use this method to retrieve the speed of the network device port that is connected to the AAAA Record object.

Parameter

none

Returns

The method returns the network device port speed value. The returned value, if any, is one of the following:

```
10M
100M
1G
10G
100G
```

```
Unknown
```

Example

```
#Get port_speed
my $port_speed = $bindaaa->port_speed();
```

port_status( )

Use this method to retrieve the status of the network device port that is connected to the AAAA Record object.

Parameter

none

Returns

The method returns the network device port status. The returned value, if any, is one of the following:

```
Up
Down
Unknown
```

Example

```
#Get port_status
my $port_status = $bindaaa->port_status();
```

port_vlan_description( )

Use this method to retrieve the VLAN description of the network device port that is connected to the AAAA Record object.

Parameter

none

Returns

The method returns the VLAN description of the network device port.

Example

```
#Get port_vlan_description
my $port_vlan_description = $bindaaa->port_vlan_description();
```

port_vlan_name( )

Use this method to retrieve the VLAN name of the network device port that is connected to the AAAA Record object.

Parameter

none

Returns

The method returns the VLAN name of the network device port.

Example

```
#Get port_vlan_name
my $port_vlan_name = $bindaaa->port_vlan_name();
```

port_vlan_number ( )

Use this method to retrieve the VLAN number of the network device port that is connected to the AAAA Record object.

Parameter

none

Returns

The method returns the network device port VLAN number.

Example

```
#Get port_vlan_number
my $port_vlan_number  = $bindaaa->port_vlan_number ();
```

reclaimable( )

Use this method to retrieve the flag that indicates whether the record is reclaimable or not.

Parameter

None

Returns

The method returns the attribute value.

Example

```
#Get reclaimable
my $reclaimable = $object->reclaimable();
```

ttl( )

Use this method to set or retrieve the Time to Live (TTL) value.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

The default value is undefined which indicates that the record inherits the TTL value of the zone.

Specify a TTL value to override the TTL value at the zone level.

Parameter

A 32-bit integer (range from 0 to 4294967295) that represents the duration in seconds that the record is cached. Zero indicates that the record should not be cached.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get ttl
my $ttl = $bindaaaa->ttl();
#Modify ttl
$bindaaaa->ttl(1800);
#Un-override ttl
$bindaaaa->ttl(undef);
```

views( )

Use this method to set or retrieve the view of the AAAA record.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

The default value is the "default" view, which means the AAAA record is located under the default view.

Parameter

Array reference of defined Infoblox::DNS::View objects.

Note that the array size must be 1.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get views
my $ref_views = $bindaaaa->views();
#Modify views, list of Infoblox::DNS::View objects
$bindaaaa->views([$view1]);
```

zone( )

Use this method to retrieve the zone name of a DNS AAAA record. This method is read-only and cannot be set.

Parameter

None

Returns

Returns the attribute value.

Example

```
# Get zone
my $zone = $bindaaaa->zone();
```

SAMPLE CODE

The following sample code demonstrates the different functions that can be applied to a DNAME record object, such as add, search, modify, and remove. This sample also includes error handling for the operations.

#Preparation prior to a DNS AAAA record insertion

```
#PROGRAM STARTS: Include all the modules that will be used

use strict;
use Infoblox;

#Create a session to the Infoblox Appliance
my $session = Infoblox::Session->new(
    master   => "192.168.1.2", #appliance host ip
    username => "admin",     #appliance user login
    password => "infoblox"   #appliance password
);

unless ($session) {
   die("Construct session failed: ",
       Infoblox::status_code() . ":" . Infoblox::status_detail());
}

#Create the zone prior to an AAAA record insertion
my $zone = Infoblox::DNS::Zone->new(name => "domain.com");

unless ($zone) {
   die("Construct zone failed: ",
       Infoblox::status_code() . ":" . Infoblox::status_detail());
}

print "Zone object created successfully\n";

#Verify if the zone exists
my $object = $session->get(object => "Infoblox::DNS::Zone",
                           name => "domain.com");
unless ($object) {
   print "Zone does not exist on server, safe to add the zone\n";
   $session->add($zone)
      or die("Add zone failed: ",
             $session->status_code() . ":" . $session->status_detail());
}
print "Zone added successfully\n";
```

#Create a DNS AAAA record

```
#Construct a DNS AAAA object

my $bindaaaa = Infoblox::DNS::Record::AAAA->new(
    name     => "aaaa1.domain.com",
    comment  => "add a DNS AAAA aaaa1.domain.com",
    ipv6addr => "aaaa:1111:bbbb:2222:cccc:3333:dddd:4444"
);

unless ($bindaaaa) {
   die("Construct DNS record AAAA failed: ",
       Infoblox::status_code() . ":" . Infoblox::status_detail());
}
print "DNS AAAA object created successfully\n";

#Add the DNS AAAA record object to Infoblox Appliance through a session
$session->add($bindaaaa)

    or die("Add record AAAA failed: ",
```

```
            $session->status_code() . ":" . $session->status_detail());
     print "DNS AAAA object added to server successfully\n";
```

#Search for a specific DNS AAAA record

```
#Search all AAAA records that match "domain.com"
my @retrieved_objs = $session->search(
    object => "Infoblox::DNS::Record::AAAA",
    name   => 'domain\.com'
);
my $object = $retrieved_objs[0];
unless ($object) {
    die("Search record AAAA failed: ",
        $session->status_code() . ":" . $session->status_detail());
}
print "Search DNS AAAA object found at least 1 matching entry\n";

#Search all AAAA records that start with "add" in the comment
my @retrieved_objs = $session->search(
    object    => "Infoblox::DNS::Record::AAAA",
    comment   => "add.*",
);
my $object = $retrieved_objs[0];
unless ($object) {
    die("Search record AAAA failed: ",
        $session->status_code() . ":" . $session->status_detail());
}
print "Search DNS AAAA object found at least 1 matching entry\n";

#Search all AAAA records that start with "aaaa1" and end with ".com"
my @retrieved_objs = $session->search(
    object => "Infoblox::DNS::Record::AAAA",
    name   => 'aaaa1.*\.com',
 );
my $object = $retrieved_objs[0];
unless ($object) {
    die("Search record AAAA failed: ",
        $session->status_code() . ":" . $session->status_detail());
}
print "Search DNS AAAA object using regexp found at least 1 matching entry\n";
```

#Get and modify a DNS AAAA record

```
#Get AAAA record through the session
my @retrieved_objs = $session->get(
    object => "Infoblox::DNS::Record::AAAA",
    name   => "aaaa1.domain.com"
);
my $object = $retrieved_objs[0];

unless ($object) {
    die("Get record AAAA failed: ",
        $session->status_code() . ":" . $session->status_detail());
}

#Modify one of the attributes of the specified AAAA record
$object->ipv6addr("aaaa:1111:bbbb:2222:cccc:3333:dddd:eeee");

#Applying the changes
$session->modify($object)

    or die("Modify record AAAA failed: ",
           $session->status_code() . ":" . $session->status_detail());
     print "DNS AAAA object modified successfully \n";
```

#Remove a DNS AAAA record.

```
#Get AAAA record through the session
my @retrieved_objs = $session->get(
    object   => "Infoblox::DNS::Record::AAAA",
    name     => "aaaa1.domain.com"
);
my $object = $retrieved_objs[0];
unless ($object) {
    die("Get record AAAA failed: ",
        $session->status_code() . ":" . $session->status_detail());
}
```

```
print "Get DNS AAAA object found at least 1 matching entry\n";

#Submit the object for removal

$session->remove($object)
    or die("Remove record AAAA failed: ",
           $session->status_code() . ":" . $session->status_detail());
print "DNS AAAA object removed successfully \n";

####PROGRAM ENDS####
```

AUTHOR

Infoblox Inc. http://www.infoblox.com/

SEE ALSO

Infoblox::DNS::View, Infoblox::DNS::Zone, Infoblox::Session, Infoblox::Session->get(), Infoblox::Session->search(), Infoblox::Session->add(), Infoblox::Session->remove(), Infoblox::Session->modify(), Infoblox::Grid::MSServer::AdUser::Data

---

COPYRIGHT

Copyright (c) 2017 Infoblox Inc.

Infoblox::DNS::Record::Alias - DNS Alias record object.

- NAME
- DESCRIPTION
- CONSTRUCTOR
- SESSION METHODS
  - Infoblox::Session->add( )
  - Infoblox::Session->get( )
  - Infoblox::Session->modify( )
  - Infoblox::Session->remove( )
  - Infoblox::Session->search( )
- METHODS
  - comment( )
  - cloud_info( )
  - creator( )
  - disable( )
  - dns_name( )
  - dns_target_name( )
  - extattrs( )
  - extensible_attributes( )
  - name( )
  - target_name( )
  - target_type( )
  - ttl( )
  - views( )
  - zone( )
- SAMPLE CODE
- AUTHOR
- SEE ALSO
- COPYRIGHT

NAME

Infoblox::DNS::Record::Alias - DNS Alias record object.

DESCRIPTION

The Alias DNS resource record (RR) allows you to create typed aliases for standard DNS resource records which are resolved dynamically by an authoritative server. Unlike CNAME Alias can be created in the zone apex.

Note that you must specify only one view for the attribute 'views'.

CONSTRUCTOR

```
 my $alias = Infoblox::DNS::Record::Alias->new(
     name                 => $fqdn,                                                    # Required
     target_name          => $fqdn,                                                    # Required
     target_type          => 'A' | 'AAAA' | 'MX' | 'NAPTR' | 'PTR' | 'SPF' | 'SRV' | 'TXT',  # Required
     comment              => $string,                                                  # Optional /
Default is empty
     disable              => 'true' | 'false'                                          # Optional /
Default 'false'
     extattrs             => {$string => $extattr, ...},                               # Optional /
Default is undefined
     extensible_attributes => {$string => $string|$num, $string => [$string|$num, ...], ...}, # Optional /
Default is undefined
     ttl                  => $num | undef,                                             # Optional /
Default is undefined
     views                => [$View1],                                                 # Optional /
Default is 'default' view
     creator              => 'STATIC',                                                 # Optional /
Default is 'STATIC'
 );
```

You cannot set both extattrs and extensible_attributes attributes at the same time.

SESSION METHODS

This section describes all the methods in Infoblox::Session module that you can apply to the object.

Infoblox::Session->add( )

Use this method to add an object to the Infoblox appliance. See Infoblox::Session->add() for parameters and return values.

Example

```
#Construct the object
my $object = Infoblox::DNS::Record::Alias->new(
    name               => 'alias.domain.com',
    target_name        => 'target.com',
    target_type        => 'A',
    comment            => 'add Alias alias.domain.com',
    disable            => 'false',
    ttl                => '360',
    views              => [$view1],
);

#Submit for addition
my $response = $session->add($object);
```

Infoblox::Session->get( )

Use this method to retrieve all the matching objects from the Infoblox appliance. See Infoblox::Session->get() for parameters and return values.

Key References

```
Apply the following attributes to get a specific DNS Alias object:

  name                  - Optional. A host name in string format.
  view                  - Optional. The DNS view in which the Alias record is located. By default,
all DNS views are searched. But if you omit this attribute and specify a zone, the appliance searches
the 'default' view only.
  zone                  - Optional. A zone name in FQDN format.
  target_name           - Optional. A target name.
  target_type           - Optional. A target type.
  creator               - Optional. Record creator.
  extattrs              - Optional. A hash reference containing extensible attributes.
  extensible_attributes - Optional. A hash reference containing extensible attributes.
```

Example

```
my @retrieved_objs = $session->get(
    object => 'Infoblox::DNS::Record::Alias',
    name   => 'alias.domain.com',
    view   => 'default' );

#Get all DNS Alias records from the 'domain.com' zone of the default view
my @retrieved_objs = $session->get(
    object => 'Infoblox::DNS::Record::Alias',
    zone   => 'domain.com',
    view   => 'default' );

#Get all DNS Alias records with a given extensible attribute
my @retrieved_objs = $session->get(
    object => 'Infoblox::DNS::Record::Alias',
    extensible_attributes => { 'Site' => 'Santa Clara' });
```

Infoblox::Session->modify( )

Use this method to modify an object in the Infoblox appliance. See Infoblox::Session->modify() for parameters and return values.

Example

```
#Use the following method to modify the comment.
$object->comment('This is a modified comment');

#Submit modification
my $response = $session->modify($object);
```

Infoblox::Session->remove( )

Use this method to remove an object from the Infoblox appliance. See Infoblox::Session->remove() for parameters and return values.

To remove a specifc object, first use `get()` or `search()` to retrieve the specific object, and then submit this object for removal.

Example

```
#Get the objects with the same name
my @retrieved_objs = $session->get(
```

```
    object => 'Infoblox::DNS::Record::Alias',
    name   => 'alias.domain.com',
    view   => 'default' );

#Find the desired object from the retrieved list.
my $desired_object = $retrieved_objs[0];

#Submit for removal
my $response = $session->remove($desired_object);
```

Infoblox::Session->search( )

Use this method to search for objects in the Infoblox appliance. See Infoblox::Session->search() for parameters and return values.

Key References

```
 Apply the following attributes to search for a DNS Alias record object:

  name                  - Optional. A name in FQDN (Fully Qualified Domain Name) format (regular
expression).
  target_name           - Optional. A target name (regular expression).
  target_type           - Optional. A target type.
  view                  - Optional. The DNS view in which the Alias record is located. By default,
all DNS views are searched. But if you omit this attribute and specify a zone, the appliance searches
the 'default' view only.
  zone                  - Optional. A zone name in FQDN format.
  comment               - Optional. A comment in string format (regular expression).
  creator               - Optional. Record creator.
  extattrs              - Optional. A hash reference containing extensible attributes.
  extensible_attributes - Optional. A hash reference containing extensible attributes.
```

For more information about searching extensible attributes, see Infoblox::Grid::ExtensibleAttributeDef/Searching Extensible Attributes.

Example

```
#Search for all DNS Alias objects that match 'domain.com' in the default DNS view
my @retrieved_objs = $session->search(
    object => 'Infoblox::DNS::Record::Alias',
    name   => 'domain\.com',
    view   => 'default' );

#Search for all DNS Alias records in the 'domain.com' zone of the default view
my @retrieved_objs = $session->search(
    object => 'Infoblox::DNS::Record::Alias',
    zone   => 'domain.com',
    view   => 'default' );

#Search for all DNS Alias records with a given extensible attribute 'Site'
my @retrieved_objs = $session->search(
   object => 'Infoblox::DNS::Record::Alias',
   extensible_attributes => { 'Site' => 'Santa Clara' });
```

METHODS

This section describes all the methods that you can use to configure and retrieve the attribute values of an Alias record.

comment( )

Use this method to set or retrieve the descriptive comment.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

Desired comment in string format with a maximum of 256 bytes.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get comment value
my $value = $object->comment();

#Modify comment value
$object->comment('Modifying the comment');
```

cloud_info( )

Use this method to retrieve cloud API related information for the Infoblox::DNS::Record::Alias object.

Parameter

None

Returns

The method returns the attribute value.

Example

```
# Get cloud_info value
my $value = $object->cloud_info();
```

creator( )

Use this method to set or retrieve the record creator.

Note that changing creator from or to 'SYSTEM' value is not allowed.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Paramter

The valid values are 'STATIC' and 'DYNAMIC'. The default value is 'STATIC'.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get creator value
my $value = $object->creator();

#Modify creator value
$object->creator('DYNAMIC');
```

disable( )

Use this method to set or retrieve the disable flag for a DNS record.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

The default value for this field is false. The DNS record is enabled.

Parameter

Specify 'true' to set the disable flag or 'false' to deactivate/unset it.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get disable value
my $value = $object->disable();

#Modify disable value
$object->disable('true');
```

dns_name( )

Use this method to retrieve the name in punycode format. This is a read-only attribute.

Parameter

None

Returns

The method returns the attribute value.

Example

```
# Get dns_name value
my $value = $object->dns_name();
```

dns_target_name( )

Use this method to retrieve the target name in punycode format. This is a read-only attribute.

Parameter

None

Returns

The method returns the attribute value.

Example

```
# Get dns_target_name value
my $value = $object->dns_target_name();
```

extattrs( )

Use this method to set or retrieve the extensible attributes associated with a DNS Alias record object.

Parameter

Valid value is a hash reference containing the names of extensible attributes and their associated values (Infoblox::Grid::Extattr objects).

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get extattrs value
my $value = $object->extattrs();

#Modify extattrs value
$object->extattrs({'Site' => $extattr1, 'Administrator' => $extattr2});
```

extensible_attributes( )

Use this method to set or retrieve the extensible attributes associated with a DNS Alias record.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

For valid values for extensible attributes, see Infoblox::Grid::ExtensibleAttributeDef/Extensible Attribute Values.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get extensible_attributes value
my $value = $object->extensible_attributes();

#Modify extensible_attributes value
$object->extensible_attributes({'Site' => 'Santa Clara', 'Administrator' => ['Peter', 'Tom']});
```

name( )

Use this method to set or retrieve the name.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

The attribute value can be in unicode format.

Parameter

The name of the DNS Alias object in Fully Qualified Domain Name (FQDN) format. The name can have a maximum of 256 bytes.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get name value
my $value = $object->name();

#Modify name value
$object->name('new.domain.com');
```

target_name( )

Use this method to set or retrieve the target name of the Alias.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

The attribute value can be in unicode format.

Parameter

Target name in FQDN (Fully Qualified Domain Name) format. The FQDN consists of the hostname followed by the domain name (example: abc.com). A hostname can have a maximum of 256 bytes.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get target_name
my $target_name = $alias->target_name();
#Modify target_name
$alias->target_name("alias.domain.com");
```

target_type( )

Use this method to set or retrieve the alias target type.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Paramter

The valid values are 'A', 'AAAA', 'MX', 'NAPTR', 'PTR', 'SPF', 'SRV' and 'TXT'.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get target_type value
my $value = $object->target_type();

#Modify target_type value
$object->target_type('A');
```

ttl( )

Use this method to set or retrieve the Time to Live (TTL) value.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

The default value is undefined which indicates that the record inherits the TTL value of the zone.

Specify a TTL value to override the TTL value at the zone level.

Parameter

A 32-bit integer (range from 0 to 4294967295) that represents the duration in seconds to cache the record. Zero indicates that the record should not be cached.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get ttl value
my $value = $object->ttl();

#Modify ttl value
$object->ttl(1800);

#Un-override ttl
$object->ttl(undef);
```

views( )

Use this method to set or retrieve the view of a Alias record.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

The default value is the 'default' view, which means that the Alias record is located under the default view.

Parameter

Array reference of defined Infoblox::DNS::View objects.

Note that the array size must be 1.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get views value
my $value = $object->views();

#Modify views value
$object->views([$view1]);
```

zone( )

Use this method to retrieve the zone name of a DNS Alias record. This is a read-only attribute.

Parameter

None

Returns

The method returns the attribute value.

Example

```
#Get zone value
my $zone = $object->zone();
```

SAMPLE CODE

The following sample code demonstrates the different functions that can be applied on an object, such as add, search, modify, and remove. This sample also includes error handling for the operations.

#Preparation prior to a DNS Alias record insertion

```
#PROGRAM STARTS: Include all the modules that will be used
use strict;
use Infoblox;
```

```
#Create a session on the Infoblox appliance

my $session = Infoblox::Session->new(
               master   => "192.168.1.2", #appliance host ip
               username => "admin",       #appliance user login
               password => "infoblox"     #appliance password
               );

unless ($session) {
       die("Construct session failed: ",
               Infoblox::status_code() . ":" . Infoblox::status_detail());
}
print "Session created successfully\n";

#Create the zone prior to a Alias record insertion
my $zone = Infoblox::DNS::Zone->new(name => "domain.com");
unless ($zone) {
       die("Construct zone failed: ",
               Infoblox::status_code() . ":" . Infoblox::status_detail());
}
print "Zone object created successfully\n";

#Verify if the zone exists
my $object = $session->get(object => "Infoblox::DNS::Zone", name => "domain.com");
unless ($object) {
       print "Zone does not exist on the server, safe to add the zone\n";
       $session->add($zone)
               or die("Add zone failed: ",
                               $session->status_code() . ":" . $session->status_detail());
}
print "Zone added successfully\n";
```

#Create a DNS Alias record

```
my $alias = Infoblox::DNS::Record::Alias->new(
               name              => "alias.domain.com",
               target_name       => "target.com",
               target_type       => "A",
               comment           => "add Alias alias.domain.com",
               );

unless ($alias) {
       die("Construct DNS record Alias failed: ",
               Infoblox::status_code() . ":" . Infoblox::status_detail());
}

#Add the DNS Alias record object to the Infoblox appliance through a session
$session->add($alias)
       or die("Add record Alias failed: ",
                       $session->status_code() . ":" . $session->status_detail());
print "DNS Alias object is added to the server successfully\n";
```

#Search for a specific DNS Alias record

```
#Search all Alias records in the zone that matches "domain.com"
my @retrieved_objs = $session->search(
               object => "Infoblox::DNS::Record::Alias",
               name   => 'domain\.com'
               );
my $object = $retrieved_objs[0];

unless ($object) {
       die("Search record Alias failed: ",
               $session->status_code() . ":" . $session->status_detail());
}
print "Search a DNS Alias object found at least 1 matching entry\n";

#Search all Alias records with names that start with "alias" and end with "domain.com"
my @retrieved_objs = $session->search(
               object => "Infoblox::DNS::Record::Alias",
               name   => '^alias.*\.domain\.com\$'
               );
my $object = $retrieved_objs[0];

unless ($object) {
       die("Search record Alias failed: ",
               $session->status_code() . ":" . $session->status_detail());
```

```
}
print "Search a DNS Alias object using regexp found at least 1 matching entry\n";
```

#Get and modify a DNS Alias record

```
#Get an Alias record through the session
my @retrieved_objs = $session->get(
               object => "Infoblox::DNS::Record::Alias",
               name   => "alias.domain.com"
               );
my $object = $retrieved_objs[0];

unless ($object) {
       die("Get record Alias failed: ",
               $session->status_code() . ":" . $session->status_detail());
}
print "Get a DNS Alias object found at least 1 matching entry\n";

#Modify one of the attributes of the specified Alias record
$object->comment("this is new comment string");

#Apply the changes
$session->modify($object)
       or die("Modify record Alias failed: ",
               $session->status_code() . ":" . $session->status_detail());
print "DNS Alias object modified successfully \n";
```

#Remove a DNS Alias record

```
#Get Alias record through the session
my @retrieved_objs = $session->get(
    object => "Infoblox::DNS::Record::Alias",
    name   => "alias.domain.com"
);
my $object = $retrieved_objs[0];
unless ($object) {
    die("Get record Alias failed: ",
        $session->status_code() . ":" . $session->status_detail());
}
print "Get DNS Alias object found at least 1 matching entry\n";

#Submit the object for removal
$session->remove($object)
       or die("Remove record Alias failed: ",
               $session->status_code() . ":" . $session->status_detail());
print "DNS Alias object removed successfully \n";

####PROGRAM ENDS####
```

AUTHOR

Infoblox Inc. http://www.infoblox.com/

SEE ALSO

Infoblox::Session->add(), Infoblox::Session->get(), Infoblox::Session->modify(), Infoblox::Session->remove(), Infoblox::Session->search(), Infoblox::Session, Infoblox::DNS::View, Infoblox::DNS::Zone

---

COPYRIGHT

Copyright (c) 2017 Infoblox Inc.

Infoblox::DNS::Record::CNAME - DNS CNAME record object.

- NAME
- DESCRIPTION
- CONSTRUCTOR
- SESSION METHODS
  - Infoblox::Session->add( )
  - Infoblox::Session->get( )
  - Infoblox::Session->modify( )
  - Infoblox::Session->remove( )
  - Infoblox::Session->search( )
- METHODS
  - canonical( )
  - comment( )
  - cloud_info( )
  - creator( )
  - creation_time( )
  - ddns_principal( )
  - ddns_protected( )
  - disable( )
  - dns_name( )
  - dns_canonical( )
  - extattrs( )
  - extensible_attributes( )
  - forbid_reclamation( )
  - last_queried( )
  - name( )
  - reclaimable( )
  - ttl( )
  - views( )
  - zone( )
- SAMPLE CODE
- AUTHOR
- SEE ALSO
- COPYRIGHT

NAME

Infoblox::DNS::Record::CNAME - DNS CNAME record object.

DESCRIPTION

A CNAME record maps an alias to a canonical name. You can use CNAME records in both forward- and IPv4 reverse-mapping zones to serve two different purposes. (At this time, you cannot use CNAME records with IPv6 reverse-mapping zones.)

Note that you must specify only one view for the attribute "views".

CONSTRUCTOR

```
  my $bind_cname =  Infoblox::DNS::Record::CNAME->new (
      canonical             => $fqdn,                                                                  #
Required
      name                  => $fqdn,                                                                  #
Required
      comment               => $string,                                                                #
Optional / Default is empty
      disable               => "true"|"false"                                                          #
Optional / Default "false"
      extattrs              => { $string => $extattr, ... },                                           #
Optional / Default is undefined
      extensible_attributes => { $string => $string|$num, $string => [ $string|$num, ...], ... },     #
Optional / Default is undefined
      ttl                   => $num|undef,                                                             #
Optional / Default is undefined
      views                 => [$View1],                                                               #
Optional / Default is "default" view
      creator               => "DYNAMIC" | "STATIC",                                                   #
Optional / Default is "STATIC"
      ddns_protected        => "true" | "false",                                                       #
Optional / Default is "false"
      ddns_principal        => $string,                                                                #
Optional / Default is undefined
                                                                                                       #
```

```
    forbid_reclamation     => "true" | "false",
Optional / Default is "false"
  );
```

You cannot set both extattrs and extensible_attributes attributes at the same time.

SESSION METHODS

This section describes all the methods in Infoblox::Session module that you can apply to a CNAME record object.

Infoblox::Session->add( )

Use this method to add an object to the Infoblox appliance. See Infoblox::Session->add() for parameters and return values.

Example

```
#Construct an object
my $bind_cname = Infoblox::DNS::Record::CNAME->new (
    name      => "bind_cname.domain.com",
    canonical => "cname.domain.com",
    comment   => "this is a demo bind_cname record",
    disable   => "true",
    ttl       => "33",
    views     => [$view1]  );
# Submit for addition
my $response = $session->add( $bind_cname );
```

Infoblox::Session->get( )

Use this method to retrieve all the matching objects from the Infoblox appliance. See Infoblox::Session->get() for parameters and return values.

Key References

```
Apply the following attributes to get a specific DNS CNAME object:

  name                 - Optional. A host name in string format.
  view                 - Optional. The DNS view in which the CNAME record is located. By default,
all DNS views are searched. But if you omit this attribute and specify a zone, the appliance searches
the 'default' view only.
  zone                 - Optional. A zone name in FQDN format.
  canonical            - Optional. A canonical name.
  creator              - Optional. Record creator.
  ddns_principal       - Optional. GSS-TSIG principal that owns the record.
  reclaimable          - Optional. The flag that indicates whether the record is reclaimable or not.
  extattrs             - Optional. A hash reference containing extensible attributes.
  extensible_attributes - Optional. A hash reference containing extensible attributes.
```

Example

```
my @retrieved_objs = $session->get(
    object => "Infoblox::DNS::Record::CNAME",
    name   => "bind_cname.domain.com",
    view   => "default" );

# get all DNS CNAME record in zone "domain.com" of default view
my @retrieved_objs = $session->get(
    object => "Infoblox::DNS::Record::CNAME",
    zone   => "domain.com",
    view   => "default" );

# get DNS CNAME record with canonical name "cname.domain.com" of default view
my @retrieved_objs = $session->get(
    object => "Infoblox::DNS::Record::CNAME",
    canonical => "cname.domain.com",
    view   => "default" );

# get all DNS CNAME record with a given extensible attribute
my @retrieved_objs = $session->get(
    object => "Infoblox::DNS::Record::CNAME",
    extensible_attributes => { 'Site' => 'Santa Clara' });
```

Infoblox::Session->modify( )

Use this method to modify an object in the Infoblox appliance. See Infoblox::Session->modify() for parameters and return values.

Example

```
# Use method to modify the comment.
$bind_cname->comment("this is a modified comment");
# Submit modification
my $response = $session->modify( $bind_cname );
```

Infoblox::Session->remove( )

Use this method to remove an object from the Infoblox appliance. See Infoblox::Session->remove() for parameters and return values.

To remove a specific object, first use `get()` or `search()` to retrieve the specific object, and then submit this object for removal.

Example

```
#Get the objects with the same name
my @retrieved_objs = $session->get(
  object => "Infoblox::DNS::Record::CNAME",
  name   => "bind_cname.domain.com",
  view   => "default");
#Find the desired object from the retrieved list.
my $desired_cname = $retrieved_objs[0];
# Submit for removal
my $response = $session->remove( $desired_cname );
```

Infoblox::Session->search( )

Use this method to search for DNS CNAME record objects in the Infoblox appliance. See Infoblox::Session->search() for parameters and return values.

Key References

```
 Apply the following attributes to search for a specific DNS CNAME object.

  name                 - Optional. A host name in string format (regular expression).
  view                 - Optional. The DNS view in which the CNAME record is located. By default,
all DNS views are searched. But if you omit this attribute and specify a zone, the appliance searches
the 'default' view only.
  zone                 - Optional. A zone name in FQDN format.
  canonical            - Optional. A canonical name (regular expression).
  comment              - Optional. A comment in string format (regular expression).
  creator              - Optional. Record creator.
  ddns_principal       - Optional. GSS-TSIG principal that owns the record.
  reclaimable          - Optional. The flag that indicates whether the record is reclaimable or not.
  extattrs             - Optional. A hash reference containing extensible attributes.
  extensible_attributes - Optional. A hash reference containing extensible attributes.
```

For more information about searching extensible attributes, see Infoblox::Grid::ExtensibleAttributeDef/Searching Extensible Attributes.

Example

```
# search for all DNS CNAME objects that match "domain.com" in the default DNS view
my @retrieved_objs = $session->search(
    object  => "Infoblox::DNS::Record::CNAME",
    name    => 'domain\.com',
    view    => "default",
    comment => "this is a modified comment" );

# search for all DNS CNAME record in zone "domain.com" of default view
my @retrieved_objs = $session->search(
    object => "Infoblox::DNS::Record::CNAME",
    zone   => "domain.com",
    view   => "default" );

# search DNS CNAME record with canonical name "cname.domain.com" of default view
my @retrieved_objs = $session->search(
    object => "Infoblox::DNS::Record::CNAME",
    canonical => "cname.domain.com",
    view   => "default" );

# search all DNS CNAME recods with the extensible attribute 'Site'
my @retrieved_objs = $session->search(
   object => "Infoblox::DNS::Record::CNAME",
   extensible_attributes => { 'Site' => 'Santa Clara' });
```

METHODS

This section describes all the methods that you can use to configure and retrieve the attribute values of a CNAME record.

canonical( )

Use this method to set or retrieve the canonical (or official) name of the host.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

The attribute value can be in unicode format.

Parameter

Canonical name in FQDN (Fully Qualified Domain Name) format. The FQDN consists of the hostname followed by the domain name (example: abc.com). A hostname can have a maximum of 256 bytes.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get canonical
my $canonical = $bind_cname->canonical();
#Modify canonical name
$bind_cname->canonical("bind_cname.domain.com");
```

comment( )

Use this method to set or retrieve the descriptive comment.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

Desired comment in string format with a maximum of 256 bytes.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get comment
my $comment = $bind_cname->comment();
#Modify comment
$bind_cname->comment("Modifying the DNS CNAME comment");
```

cloud_info( )

Use this method to retrieve cloud API related information for the Infoblox::DNS::Record::CNAME object.

Parameter

None

Returns

The method returns the attribute value.

Example

```
# Get cloud_info
my $cloud_info = $object->cloud_info();
```

creator( )

Use this method to set or retrieve the record creator.

Note that changing creator from or to 'SYSTEM' value is not allowed.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Paramter

The valid values are 'STATIC' and 'DYNAMIC'. The default value is 'STATIC'.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get creator value
my $creator = $object->creator();

#Modify creator value
$object->creator("DYNAMIC");
```

creation_time( )

Use this method to retrieve the creation time for the record. This is a read-only attribute.

Parameter

None

Returns

The valid return value is a number of seconds that have elapsed since January 1st, 1970 UTC.

Example

```
#Get creation_time value
my $creation_time = $object->creation_time();
```

ddns_principal( )

Use this method to set or retrive the GSS-TSIG principal that owns this record.

Note that you cannot set ddns_principal for 'STATIC' and 'SYSTEM' records.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Paramter

The GSS-TSIG principal FQDN (Fully Qualified Domain Name) format. The FQDN consists of the hostname followed by the domain name (example: abc.com). A hostname can have a maximum of 256 characters.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get ddns_principal value
my $ddns_principal = $object->ddns_principal();

#Modify ddns_principal value
$object->ddns_principal('foo.com');
```

ddns_protected( )

Use this method to set or retrieve the flag that indicates whether DDNS updates for this record are allowed or not.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

Specify 'true' to protect record from DDNS updates and 'false' to allow DDNS updates for the specified record.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get ddns_protected value
my $ddns_protected = $object->ddns_protected();

#Modify ddns_protected value
$object->ddns_protected('true');
```

disable( )

Use this method to set or retrieve the disable flag of a DNS record.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

The default value for this field is false. The DNS record is enabled.

Parameter

Specify "true" to set the disable flag or "false" to deactivate/unset it.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get disable
my $disable = $bind_cname->disable();
#Modify disable
$bind_cname->disable("true");
```

dns_name( )

Use this method to retrieve the host name in punycode format. This is a read-only attribute.

Parameter

None

Returns

The method returns the attribute value.

Example

```
# Get attribute value
my $value = $bind_cname->dns_name();
```

dns_canonical( )

Use this method to retrieve the canonical (or official) name, in punycode format, of the host. This is a read-only attribute.

Parameter

None

Returns

The method returns the attribute value.

Example

```
# Get attribute value
my $value = $bind_cname->dns_canonical();
```

extattrs( )

Use this method to set or retrieve the extensible attributes associated with a DNS CNAME record object.

Parameter

Valid value is a hash reference containing the names of extensible attributes and their associated values (Infoblox::Grid::Extattr objects).

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get extattrs
my $ref_extattrs = $bind_cname->extattrs();
```

```
#Modify extattrs
$bind_cname->extattrs({ 'Site' => $extattr1, 'Administrator' => $extattr2 });
```

extensible_attributes( )

Use this method to set or retrieve the extensible attributes associated with a DNS CNAME record.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

For valid values for extensible attributes, see Infoblox::Grid::ExtensibleAttributeDef/Extensible Attribute Values.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get extensible attributes
my $ref_extensible_attributes = $bind_cname->extensible_attributes();
#Modify extensible attributes
$bind_cname->extensible_attributes({ 'Site' => 'Santa Clara', 'Administrator' => [ 'Peter', 'Tom' ]
});
```

forbid_reclamation( )

Use this method to set or retrieve the flag that indicates whether the reclamation is allowed for the record or not.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

Specify 'true' to forbid reclamation for the record and 'false' to allow it. The default value is 'false'.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get forbid_reclamation
my $forbid_reclamation = $object->forbid_reclamation();
#Modify forbid_reclamation
$object->forbid_reclamation('true');
```

last_queried( )

Use this method to retrieve the time when the associated record was last queried. This is a read-only attribute.

Parameter

None

Returns

The method returns the attribute value. The number of seconds that have elapsed since January 1st, 1970 UTC.

Example

```
#Get last_queried
my $last_queried = $bind_cname->last_queried();
```

name( )

Use this method to set or retrieve the host name.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

The attribute value can be in unicode format.

Parameter

A name in FQDN (Fully Qualified Domain Name) format. The FQDN consists of the host_name followed by the domain name

(example: abc.com). A host_name can have a maximum of 256 bytes.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get name
my $name = $bind_cname->name();
#Modify name
$bind_cname->name("bind_cname2.domain.com");
```

reclaimable( )

Use this method to retrieve the flag that indicates whether the record is reclaimable or not.

Parameter

None

Returns

The method returns the attribute value.

Example

```
#Get reclaimable
my $reclaimable = $object->reclaimable();
```

ttl( )

Use this method to set or retrieve the Time to Live (TTL) value.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

The default value is undefined which indicates that the record inherits the TTL value of the zone.

Specify a TTL value to override the TTL value at the zone level.

Parameter

A 32-bit integer (range from 0 to 4294967295) that represents the duration, in seconds, that the record is cached. Zero indicates that the record should not be cached.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get ttl
my $ttl = $bind_cname->ttl();
#Modify ttl
$bind_cname->ttl(1800);
#Un-override ttl
$bind_cname->ttl(undef);
```

views( )

Use this method to set or retrieve the view of the CNAME record.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

The default value is the "default" view, which means the CNAME record is located under the default view.

Parameter

Array reference of defined Infoblox::DNS::View objects.

Note that the array size must be 1.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get views
my $ref_views = $bind_cname->views();
#Modify views, list of Infoblox::DNS::View objects
$bind_cname->views([$view1]);
```

zone( )

Use this method to retrieve the zone name of a DNS CNAME record. This method is read-only and cannot be set.

Parameter

None

Returns

Returns the attribute value.

Example

```
# Get zone
my $zone = $bind_cname->zone();
```

SAMPLE CODE

The following sample code demonstrates the different functions that can be applied to an object, such as add, search, modify, and remove. This sample code also includes error handling for the operations.

#Preparation prior to a DNS CNAME record insertion

```
#PROGRAM STARTS: Include all the modules that will be used
use strict;
use Infoblox;

#Create a session to the Infoblox appliance
my $session = Infoblox::Session->new(
    master   => "192.168.1.2",
    username => "admin",
    password => "infoblox"
);
unless ($session) {
   die("Construct session failed: ",
       Infoblox::status_code() . ":" . Infoblox::status_detail());
}
print "Session created successfully\n";

#Create the zone prior to a CNAME record insertion
my $zone = Infoblox::DNS::Zone->new(name => "domain.com");
unless ($zone) {
   die("Construct zone failed: ",
       Infoblox::status_code() . ":" . Infoblox::status_detail());
}
print "Zone object created successfully\n";

#Verify if the zone exists
my $object = $session->get(object => "Infoblox::DNS::Zone", name => "domain.com");
unless ($object) {
   print "Zone does not exist on server, safe to add the zone\n";
   $session->add($zone)
      or die("Add zone failed: ",
             $session->status_code() . ":" . $session->status_detail());
}
print "Zone added successfully\n";
```

#Create a DNS CNAME record

```
my $view1 = Infoblox::DNS::View->new(
    name      => "default" );

#Construct a DNS CNAME object
my $bind_cname = Infoblox::DNS::Record::CNAME->new(
    name      => "bind_cname.domain.com",
    canonical => "cname.domain.com",
```

```
    comment   => "this is a demo bind_cname record",
    ttl       => "33",
    views     => [$view1]);
unless ($bind_cname) {
   die("Construct DNS record CNAME failed: ",
       Infoblox::status_code() . ":" . Infoblox::status_detail());
}
print "DNS CNAME object created successfully\n";

#Add the DNS CNAME record object to the Infoblox appliance through a session
$session->add($bind_cname)
    or die("Add record cname failed: ",
            $session->status_code() . ":" . $session->status_detail());
print "DNS CNAME object added to server successfully\n";
```

#Search for a specific DNS CNAME record

```
#Search all CNAME records that match "domain.com"
my @retrieved_objs = $session->search(
    object => "Infoblox::DNS::Record::CNAME",
    name   => 'domain\.com'
);
my $object = $retrieved_objs[0];
unless ($object) {
    die("Search record CNAME failed: ",
        $session->status_code() . ":" . $session->status_detail());
}
print "Search DNS CNAME object found at least 1 matching entry\n";

#Search all CNAME records that start with "bind" and end with ".domain.com"
my @retrieved_objs = $session->search(
    object => "Infoblox::DNS::Record::CNAME",
    name   => '^bind.*\.domain\.com\$'
);
my $object = $retrieved_objs[0];
unless ($object) {
    die("Search record CNAME failed: ",
        $session->status_code() . ":" . $session->status_detail());
}
print "Search DNS CNAME object using regexp found at least 1 matching entry\n";
```

#Get and modify a DNS CNAME record

```
#Get CNAME record through the session
my @retrieved_objs = $session->get(
    object => "Infoblox::DNS::Record::CNAME",
    name   => "bind_cname.domain.com",
    view   => "default"
);
my $object = $retrieved_objs[0];
unless ($object) {
    die("Get record CNAME failed: ",
        $session->status_code() . ":" . $session->status_detail());
}
print "Get DNS CNAME object found at least 1 matching entry\n";

#Modify one of the attributes of the specified CNAME record
$object->disable("true");

#Apply the changes
$session->modify($object)
    or die("Modify record CNAME failed: ",
            $session->status_code() . ":" . $session->status_detail());
print "DNS CNAME object modified successfully \n";
```

#Remove a DNS CNAME record

```
#Get CNAME record through the session
my @retrieved_objs = $session->get(
    object => "Infoblox::DNS::Record::CNAME",
    name   => "bind_cname.domain.com",
    view   => "default"
);
$object = $retrieved_objs[0];
unless ($object) {
    die("Get record CNAME failed: ",
        $session->status_code() . ":" . $session->status_detail());
```

```
}
print "Get DNS CNAME object found at least 1 matching entry\n";

#Submit the object for removal
$session->remove( $object )
  or die("Remove record CNAME failed: ",
         $session->status_code() . ":" . $session->status_detail());
print "DNS CNAME object removed successfully \n";

####PROGRAM ENDS####
```

AUTHOR

Infoblox Inc. http://www.infoblox.com/

SEE ALSO

Infoblox::DNS::View, Infoblox::DNS::Zone, Infoblox::Session, Infoblox::Session->get(), Infoblox::Session->search(), Infoblox::Session->add(), Infoblox::Session->remove(), Infoblox::Session->modify()

COPYRIGHT

Copyright (c) 2017 Infoblox Inc.

Infoblox::DNS::Record::DHCID - DNS DHCID record object.

- NAME
- DESCRIPTION
- SESSION METHODS
  - Infoblox::Session->get( )
  - Infoblox::Session->remove( )
  - Infoblox::Session->search( )
- METHODS
  - creator( )
  - creation_time( )
  - dhcid( )
  - dns_name( )
  - name( )
  - ttl( )
  - views( )
  - zone( )
- AUTHOR
- SEE ALSO
- COPYRIGHT

NAME

Infoblox::DNS::Record::DHCID - DNS DHCID record object.

DESCRIPTION

The DHCID DNS resource record (RR) is used to associate the DNS domain names and DHCP clients using these domain names.

Note that insert/update operations are not allowed for this object.

SESSION METHODS

This section describes all the methods in Infoblox::Session module that you can apply to the object.

Infoblox::Session->get( )

Use this method to retrieve all the matching objects from the Infoblox appliance. See Infoblox::Session->get() for parameters and return values.

Key References

```
Apply the following attributes to get a specific DNS DHCID object:

  name    - Optional. Host name in string format.
  view    - Optional. The DNS view in which the DHCID record is located. By default, all DNS views
are searched. But if you omit this attribute and specify a zone, the appliance searches the 'default'
view only.
  dhcid   - Optional. Base-64 encoded string that contains client information.
  zone    - Optional. A zone name in FQDN format.
  creator - Optional. Record creator.
```

Example

```
# get all DNS DHCID records in the 'domain.com' zone of the default view
my @retrieved_objs = $session->get(
    object => 'Infoblox::DNS::Record::DHCID',
    zone   => 'domain.com',
    view   => 'default' );
```

Infoblox::Session->remove( )

Use this method to remove an object from the Infoblox appliance. See Infoblox::Session->remove() for parameters and return values.

To remove a specifc object, first use `get()` or `search()` to retrieve the specific object, and then submit this object for removal.

Example

```
#Get the objects with the same name
my @retrieved_objs = $session->get(
    object => 'Infoblox::DNS::Record::DHCID',
    name   => 'dhcid.domain.com',
    view   => 'default' );
```

```
# find the desired object from the retrieved list.
my $desired_object = $retrieved_objs[0];

# Submit for removal
my $response = $session->remove($desired_object);
```

Infoblox::Session->search( )

Use this method to search for objects in the Infoblox appliance. See Infoblox::Session->search() for parameters and return values.

Key References

```
Apply the following attributes to search for a DNS DHCID record object:

  name    - Optional. A name in FQDN (Fully Qualified Domain Name) format (regular expression).
  view    - Optional. The DNS view in which the DHCID record is located. By default, all DNS views
are searched. But if you omit this attribute and specify a zone, the appliance searches the 'default'
view only.
  dhcid   - Optional. Base-64 encoded string that contains client information (regexp).
  zone    - Optional. A zone name in FQDN format.
  creator - Optional. Record creator.
```

Example

```
# search for all DNS DHCID objects that match 'domain.com' in the default DNS view
my @retrieved_objs = $session->search(
    object => 'Infoblox::DNS::Record::DHCID',
    name   => 'domain\.com',
    view   => 'default' );
```

METHODS

This section describes all the methods that you can use to configure and retrieve the attribute values of a DHCID record.

Note that this is a read-only object.

creator( )

Use this method to retrieve the record creator. This is a read-only attribute.

Paramter

None

Returns

Valid return values are 'STATIC', 'DYNAMIC', and 'SYSTEM'.

Example

```
#Get creator value
my $value = $object->creator();
```

creation_time( )

Use this method to retrieve the creation time for the record. This is a read-only attribute.

Parameter

None

Returns

Valid return value is the number of seconds that has elapsed since January 1st, 1970 UTC.

Example

```
#Get creation_time value
my $value = $object->creation_time();
```

dhcid( )

Use this method to retrieve the Base64 encoded DHCP client information. This is a read-only attribute.

Parameter

None

Returns

The method returns the attribute value.

Example

```
# Get dhcid value
my $value = $object->dhcid();
```

dns_name( )

Use this method to retrieve the name in punycode format. This is a read-only attribute.

Parameter

None

Returns

The method returns the attribute value.

Example

```
# Get dns_name value
my $value = $object->dns_name();
```

name( )

Use this method to retrieve the name. This is a read-only attribute.

The attribute value can be in unicode format.

Parameter

None

Returns

The method returns the attribute value.

Example

```
#Get name value
my $value = $object->name();
```

ttl( )

Use this method to retrieve the Time to Live (TTL) value. This is a read-only attribute.

Parameter

None

Returns

The method returns the attribute value, a 32-bit integer (range from 0 to 4294967295).

Example

```
#Get ttl value
my $value = $object->ttl();
```

views( )

Use this method to retrieve the view of a DHCID record. This is a read-only attribute.

Parameter

None

Returns

The method returns the attribute value: the one-element array that contains Infoblox::DNS::View object.

Example

```
#Get views value
my $value = $object->views();
```

```
#Modify views values
$object->views([$view1]);
```

zone( )

Use this method to retrieve the zone name of a DNS DHCID record. This is a read-only attribute.

Parameter

None

Returns

The method returns the attribute value.

Example

```
#Get zone value
my $zone = $object->zone();
```

AUTHOR

Infoblox Inc. http://www.infoblox.com/

SEE ALSO

Infoblox::Session->get(), Infoblox::Session->remove(), Infoblox::Session->search(), Infoblox::Session, Infoblox::DNS::View, Infoblox::DNS::Zone

---

COPYRIGHT

Copyright (c) 2017 Infoblox Inc.

Infoblox::DNS::Record::DNAME - DNAME record object.

- NAME
- DESCRIPTION
- CONSTRUCTOR
- SESSION METHODS
  - Infoblox::Session->add( )
  - Infoblox::Session->get( )
  - Infoblox::Session->modify( )
  - Infoblox::Session->remove( )
  - Infoblox::Session->search( )
- METHODS
  - comment( )
  - cloud_info( )
  - creator( )
  - creation_time( )
  - ddns_principal( )
  - ddns_protected( )
  - disable( )
  - dns_name( )
  - dns_target( )
  - extattrs( )
  - extensible_attributes( )
  - forbid_reclamation( )
  - last_queried( )
  - name( )
  - reclaimable( )
  - target( )
  - ttl( )
  - views( )
  - zone( )
- SAMPLE CODE
- AUTHOR
- SEE ALSO
- COPYRIGHT

NAME

Infoblox::DNS::Record::DNAME - DNAME record object.

DESCRIPTION

A DNAME record maps all the names in one domain to those in another domain, essentially substituting one domain name suffix with the other.

Note that you must specify only one view for the attribute "views".

CONSTRUCTOR

```
 my $bind_dname = Infoblox::DNS::Record::DNAME->new (
     name                  => $fqdn,                                                    # Required
     target                => $fqdn,                                                    # Required
     comment               => $string,                                                  # Optional
/ Defauilt is empty
     disable               => "true" | "false",                                         # Optional
/ Default "false"
     extattrs              => { $string => $extattr, ... },                             # Optional
/ Default is undefined
     extensible_attributes => { $string => $string|$num, $string => [ $string|$num, ...], ... }, # Optional
/ Default is undefined
     ttl                   => $num | undef,                                             # Optional
/ Default is undefined
     views                 => [$View1],                                                 # Optional
/ Default is "default" view
     creator               => "DYNAMIC" | "STATIC",                                     # Optional
/ Default is "STATIC"
     ddns_protected        => "true" | "false",                                         # Optional
/ Default is "false"
     ddns_principal        => $string,                                                  # Optional
/ Default is undefined
     forbid_reclamation    => "true" | "false",                                         # Optional
/ Default is "false"
```

```
);
```

You cannot set both extattrs and extensible_attributes attributes at the same time.

SESSION METHODS

This section describes all the methods in Infoblox::Session module that you can apply to a DNS DNAME object.

Infoblox::Session->add( )

Use this method to add a DNAME record object to the Infoblox appliance. See Infoblox::Session->add() for parameters and return values.

Example

```
#Construct object
my $bind_dname = Infoblox::DNS::Record::DNAME->new (
    name    => "bind_dname.domain.com",
    comment => "add dname bind_dname.domain.com",
    target  => "target.org",
    disable => "true",
    ttl     => "33",
    views   => [$view1]
    );

# Submit for addition
my $response = $session->add( $bind_dname );
```

Infoblox::Session->get( )

Use this method to retrieve existing DNAME record objects from the Infoblox appliance. See Infoblox::Session->get() for parameters and return values.

Key Reference

```
Apply the following attributes to get a specific DNS DNAME object:

  name                - Optional. A domain name in string format (regular expression).
  view                - Optional. The DNS view in which the DNAME record is located. By default,
all DNS views are searched. But if you omit this attribute and specify a zone, the appliance searches
the 'default' view only.
  zone                - Optional. A zone name in FQDN format.
  creator             - Optional. Record creator.
  ddns_principal      - Optional. GSS-TSIG principal that owns the record.
  reclaimable         - Optional. The flag that indicates whether the record is reclaimable or not.
  extattrs            - Optional. A hash reference containing extensible attributes.
  extensible_attributes - Optional. A hash reference containing extensible attributes.
```

Example

```
my @retrieved_objs = $session->get(
    object => "Infoblox::DNS::Record::DNAME",
    name   => "bind_dname.domain.com",
    view   => "default"
    );

# get all DNS DNAME record in zone "domain.com" of default view
my @retrieved_objs = $session->get(
    object => "Infoblox::DNS::Record::DNAME",
    name   => "domain.com",
    view   => "default"
    );

# get all DNS DNAME record with a given extensible attribute
my @retrieved_objs = $session->get(
    object => "Infoblox::DNS::Record::DNAME",
    extensible_attributes => { 'Site' => 'Santa Clara' });
```

Infoblox::Session->modify( )

Use this method to modify DNAME record objects in the Infoblox appliance. See Infoblox::Session->modify() for parameters and return values.

Example

```
# Use method to modify the comment.
$bind_dname->comment("this is a modified comment.");
# Submit modification
my $response = $session->modify( $bind_dname );
```

Infoblox::Session->remove( )

Use this method to remove a DNAME record object from the Infoblox appliance. See Infoblox::Session->remove() for parameters and return values.

To remove a specific object, first use `get()` or `search()` to retrieve the specific object, and then submit this object for removal.

Example

```
# Get the objects with the same name
my @retrieved_objs = $session->get(
    object => "Infoblox::DNS::Record::DNAME",
    name   => "bind_dname.domain.com",
    view   => "default" );
# Find the desired object from the retrieved list.
my $desired_dname = $retrieved_objs[0];
# Submit for removal
my $response = $session->remove( $desired_dname );
```

Infoblox::Session->search( )

Use this method to search for existing DNAME record objects in the Infoblox appliance. See Infoblox::Session->search() for parameters and return values.

Key Reference

```
Apply the following attributes to search for a DNS DNAME record object:

  name                  - Optional. A domain name in string format (regular expression).
  view                  - Optional. The DNS view in which the DNAME record is located. By default,
all DNS views are searched. But if you omit this attribute and specify a zone, the appliance searches
the 'default' view only.
  comment               - Optional. A comment in string format (regular expression).
  creator               - Optional. Record creator.
  ddns_principal        - Optional. GSS-TSIG principal that owns the record.
  reclaimable           - Optional. The flag that indicates whether the record is reclaimable or not.
  extattrs              - Optional. A hash reference containing extensible attributes.
  extensible_attributes - Optional. A hash reference containing extensible attributes.
```

For more information about searching extensible attributes, see Infoblox::Grid::ExtensibleAttributeDef/Searching Extensible Attributes.

Example

```
# search for all DNS DNAME objects that match "domain.com" in the default DNS view
my @retrieved_objs = $session->search(
    object => "Infoblox::DNS::Record::DNAME",
    name   => '.*\.domain\.com',
    view   => "default"
    );

# search for all DNS DNAME records in the "domain.com" zone of the default view
my @retrieved_objs = $session->search(
    object => "Infoblox::DNS::Record::DNAME",
    zone   => "domain.com",
    view   => "default"
    );

# search all DNS DNAME recods with the extensible attribute 'Site'
my @retrieved_objs = $session->search(
   object => "Infoblox::DNS::Record::DNAME",
   extensible_attributes => { 'Site' => 'Santa Clara' });
```

METHODS

This section describes all the methods that you can use to configure and retrieve the attribute values of a DNAME record.

comment( )

Use this method to set or retrieve the comment value.

Enter identifying text for this record, such as a meaningful note or reminder.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

Desired comment in string format with a maximum of 256 bytes.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get comment
my $comment = $binddname->comment();
#Modify comment
$binddname->comment("Modifying the DNAME comment");
```

cloud_info( )

Use this method to retrieve cloud API related information for the Infoblox::DNS::Record::DNAME object.

Parameter

None

Returns

The method returns the attribute value.

Example

```
# Get cloud_info
my $cloud_info = $object->cloud_info();
```

creator( )

Use this method to set or retrieve the record creator.

Note that changing creator from or to 'SYSTEM' value is not allowed.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Paramter

The valid values are 'STATIC' and 'DYNAMIC'. The default value is 'STATIC'.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get creator value
my $creator = $object->creator();

#Modify creator value
$object->creator("DYNAMIC");
```

creation_time( )

Use this method to retrieve the creation time for the record. This is a read-only attribute.

Parameter

None

Returns

The valid return value is a number of seconds that have elapsed since January 1st, 1970 UTC.

Example

```
#Get creation_time value
my $creation_time = $object->creation_time();
```

ddns_principal( )

Use this method to set or retrive the GSS-TSIG principal that owns this record.

Note that you cannot set ddns_principal for 'STATIC' and 'SYSTEM' records.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Paramter

The GSS-TSIG principal FQDN (Fully Qualified Domain Name) format. The FQDN consists of the hostname followed by the domain name (example: abc.com). A hostname can have a maximum of 256 characters.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get ddns_principal value
my $ddns_principal = $object->ddns_principal();

#Modify ddns_principal value
$object->ddns_principal('foo.com');
```

ddns_protected( )

Use this method to set or retrieve the flag that indicates whether DDNS updates for this record are allowed or not.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

Specify 'true' to protect record from DDNS updates and 'false' to allow DDNS updates for the specified record.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get ddns_protected value
my $ddns_protected = $object->ddns_protected();

#Modify ddns_protected value
$object->ddns_protected('true');
```

disable( )

Use this method to set or retrieve the disable value.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

The default value for this field is "false", therefore the DNS record is enabled.

Parameter

Specify "true" to set the disable flag or "false" to deactivate/unset it.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get disable
my $disable = $binddname->disable();
#Modify disable
$binddname->disable("true");
```

dns_name( )

Use this method to retrieve the host name in punycode format. This is a read-only attribute.

Parameter

None

Returns

The method returns the attribute value.

Example

```
# Get attribute value
my $value = $binddname->dns_name();
```

dns_target( )

Use this method to retrieve the target in punycode format. This is a read-only attribute.

Parameter

None

Returns

The method returns the attribute value.

Example

```
# Get attribute value
my $value = $binddname->dns_target();
```

extattrs( )

Use this method to set or retrieve the extensible attributes associated with a DNS DNAME record object.

Parameter

Valid value is a hash reference containing the names of extensible attributes and their associated values (Infoblox::Grid::Extattr objects).

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get extattrs
my $ref_extattrs = $binddname->extattrs();
#Modify extattrs
$binddname->extattrs({ 'Site' => $extattr1, 'Administrator' => $extattr2 });
```

extensible_attributes( )

Use this method to set or retrieve the extensible attributes associated with a DNS DNAME record.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

For valid values for extensible attributes, see Infoblox::Grid::ExtensibleAttributeDef/Extensible Attribute Values.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get extensible attributes
my $ref_extensible_attributes = $binddname->extensible_attributes();
#Modify extensible attributes
$binddname->extensible_attributes({ 'Site' => 'Santa Clara', 'Administrator' => [ 'Peter', 'Tom' ]
});
```

forbid_reclamation( )

Use this method to set or retrieve the flag that indicates whether the reclamation is allowed for the record or not.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

Specify 'true' to forbid reclamation for the record and 'false' to allow it. The default value is 'false'.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get forbid_reclamation
my $forbid_reclamation = $object->forbid_reclamation();
#Modify forbid_reclamation
$object->forbid_reclamation('true');
```

last_queried( )

Use this method to retrieve the time when the associated record was last queried. This is a read-only attribute.

Parameter

None

Returns

The method returns the attribute value. The number of seconds that have elapsed since January 1st, 1970 UTC.

Example

```
#Get last_queried
my $last_queried = $binddname->last_queried();
```

name( )

Use this method to set or retrieve the name value.

Enter the name of a sub domain. If you are adding a DNAME record for the entire zone, leave this field empty. This field is for adding a DNAME record for a sub domain within the selected zone.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

The attribute value can be in unicode format.

Parameter

Hostname in FQDN (Fully Qualified Domain Name) format. The FQDN consists of the hostname followed by the domain name (example: abc.com). A hostname can have a maximum of 256 bytes.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get name
my $name = $binddname->name();
#Modify name
$binddname->name("bind_dname1.domain.com");
```

reclaimable( )

Use this method to retrieve the flag that indicates whether the record is reclaimable or not.

Parameter

None

Returns

The method returns the attribute value.

Example

```
#Get reclaimable
my $reclaimable = $object->reclaimable();
```

target( )

Use this method to set or retrieve the target value.

Enter the domain name to which you want to map all the domain names specified in the Domain Name field.

Include the specified parameter to set the attribute value. Must use some parameter to add a DNAME record.

The attribute value can be in unicode format.

Parameter

Text with the target (domain-name) for the DNS DNAME record.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get name
my $target = $binddname->target();
#Modify name
$binddname->target("modified_target.org");
```

ttl( )

Use this method to set or retrieve the Time to Live (TTL) value.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

The default value is undefined; therefore the record inherits the TTL value from the zone level.

With a specified TTL value, the method overrides the zone values with the specified TTL value.

Parameter

A 32-bit integer (range from 0 to 4294967295) that represents the duration in seconds that the record is cached. Zero indicates that the record should not be cached.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get ttl
my $ttl = $binddname->ttl();
#Modify ttl
$binddname->ttl(1800);
#Un-override ttl
$binddname->ttl(undef);
```

views( )

Use this method to set or retrieve the views value.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

The default value is the default view, therefore the DNAME record is referenced under the default view.

Parameter

Array reference of defined Infoblox::DNS::View objects.

Note that the array size must be 1.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get views
my $ref_views = $binddname->views();
#Modify views, list of Infoblox::DNS::View objects
$binddname->views([$view1]);
```

zone( )

Use this method to retrieve the zone name of a DNS DNAME record. This method is read-only and cannot be set.

Parameter

None

Returns

Returns the attribute value.

Example

```
# Get zone
my $zone = $binddname->zone();
```

SAMPLE CODE

The following sample code demonstrates different operations that can be applied to a DNAME record object, such as create, search, modify, and remove an object. This sample code also includes error handling for the operations.

#Preparation prior to a DNAME record insertion

```
#PROGRAM STARTS: Include all the modules that will be used
use strict;
use Infoblox;

#Create a session to the Infoblox appliance
my $session = Infoblox::Session->new(
    master   => "192.168.1.2",
    username => "admin",
    password => "infoblox"
);
unless ($session) {
   die("Construct session failed: ",
       Infoblox::status_code() . ":" . Infoblox::status_detail());
}
print "Session created successfully\n";

#Create the zone prior to a DNAME record insertion
my $zone = Infoblox::DNS::Zone->new(name => "domain.com");
unless ($zone) {
   die("Construct zone failed: ",
       Infoblox::status_code() . ":" . Infoblox::status_detail());
}
print "Zone object created successfully\n";

#Verify if the zone exists
my $object = $session->get(object => "Infoblox::DNS::Zone", name => "domain.com");
unless ($object) {
   print "Zone does not exist on server, safe to add the zone\n";
   $session->add($zone)
      or die("Add zone failed: ",
             $session->status_code() . ":" . $session->status_detail());
}
print "Zone added successfully\n";
```

#Create a DNAME record

```
#Construct a DNAME object
my $bind_dname = Infoblox::DNS::Record::DNAME->new(
    name     => "bind_dname.domain.com",
    target   => "target.org",
    comment  => "this is a demo bind_dname record",
);
unless ($bind_dname) {
   die("Construct DNS record DNAME failed: ",
       Infoblox::status_code() . ":" . Infoblox::status_detail());
}
print "DNS DNAME object created successfully\n";
```

```
#Add the DNS DNAME record object to Infoblox appliance through a session
$session->add($bind_dname)
    or die("Add record DNAME failed: ",
            $session->status_code() . ":" . $session->status_detail());
print "DNS DNAME object added to server successfully\n";
```

#Search for a specific DNS DNAME record

```
#Search all DNAME records in the zone
my @retrieved_objs = $session->search(
    object => "Infoblox::DNS::Record::DNAME",
    name   => 'domain\.com'
);
my $object = $retrieved_objs[0];
unless ($object) {
    die("Search record DNAME failed: ",
        $session->status_code() . ":" . $session->status_detail());
}
print "Search DNS DNAME object found at least 1 matching entry\n";

#Search all DNAME records that start with "bind" and end with "domain.com"
my @retrieved_objs = $session->search(
    object => "Infoblox::DNS::Record::DNAME",
    name   => '^bind.*\.domain.com\$'
);
my $object = $retrieved_objs[0];
unless ($object) {
    die("Search record DNAME failed: ",
        $session->status_code() . ":" . $session->status_detail());
}
print "Search DNS DNAME object using regexp found at least 1 matching entry\n";
```

#Get and modify a DNS DNAME record

```
#Get DNAME record through the session
my @retrieved_objs = $session->get(
    object => "Infoblox::DNS::Record::DNAME",
    name   => "bind_dname.domain.com",
    target => "target.org"
);
my $object = $retrieved_objs[0];
unless ($object) {
    die("Get record DNAME failed: ",
        $session->status_code() . ":" . $session->status_detail());
}
print "Get DNS DNAME object found at least 1 matching entry\n";

#Modify one of the attributes of the specified DNAME record
$object->comment("This is a modified comment.");

#Apply the changes
$session->modify($object)
    or die("Modify record DNAME failed: ",
            $session->status_code() . ":" . $session->status_detail());
print "DNS DNAME object modified successfully \n";
```

#Remove a DNS DNAME record

```
#Get DNAME record through the session
my @retrieved_objs = $session->get(
    object => "Infoblox::DNS::Record::DNAME",
    name   => "bind_dname.domain.com"
);
my $object = $retrieved_objs[0];
unless ($object) {
    die("Get record DNAME failed: ",
        $session->status_code() . ":" . $session->status_detail());
}
print "Get DNS DNAME object found at least 1 matching entry\n";

#Submit the object for removal
$session->remove($object)
    or die("Remove record DNAME failed: ",
        $session->status_code() . ":" . $session->status_detail());
print "DNS DNAME object removed successfully \n";

####PROGRAM ENDS####
```

AUTHOR

Infoblox Inc. http://www.infoblox.com/

SEE ALSO

Infoblox::DNS::View, Infoblox::DNS::Zone, Infoblox::Session, Infoblox::Session->get(), Infoblox::Session->search(), Infoblox::Session->add(), Infoblox::Session->remove(), Infoblox::Session->modify()

COPYRIGHT

Copyright (c) 2017 Infoblox Inc.

Infoblox::DNS::Record::DNSKEY - DNSKEY Record object.

- NAME
- DESCRIPTION
- CONSTRUCTOR
- SESSION METHODS
  - Infoblox::Session->get( )
  - Infoblox::Session->search( )
- METHODS
  - algorithm( )
  - creator( )
  - dns_name( )
  - flags( )
  - key_tag( )
  - name( )
  - public_key( )
  - ttl( )
  - view( )
  - zone( )
- SAMPLE CODE
- AUTHOR
- SEE ALSO
- COPYRIGHT

NAME

Infoblox::DNS::Record::DNSKEY - DNSKEY Record object.

DESCRIPTION

The DNSKEY resource record stores public keys for the DNSSEC authentication process. The DNSKEY records are generated automatically when the corresponding authoritative zone is signed. The the DNSKEY resource record object is read-only.

The DNSKEY resource record is defined in RFC 4034.

CONSTRUCTOR

The Infoblox::DNS::Record::DNSKEY object is a read-only object and does not require manual construction.

SESSION METHODS

This section describes all the methods in an Infoblox::Session that can be applied to a DNSKEY Record object.

Infoblox::Session->get( )

Use this method to retrieve the existing objects from an Infoblox appliance. See Infoblox::Session->get() for parameters and return values.

Key Reference

```
 Apply the following attributes to get the DNSKEY Record object(s):

     algorithm - Optional. The public key encryption algorithm. The allowed values are
'1','3','5','6','7','RSAMD5','RSASHA1','DSA','NSEC3RSASHA1','NSEC3DSA','RSASHA256','RSASHA512','NSEC3RSASHA256'
 and 'NSEC3RSASHA512'.
     flags     - Optional. Flags field to identify the type of the key.
     key_tag   - Optional. Key tag to identify the public key.
     name      - Optional. FQDN of the DNSKEY record.
     public_key- Optional. Public key in the base-64 encoding.
     ttl       - Optional. TTL of the DNSKEY record.
     view      - Optional. The DNS view in which the record is located. By default, all DNS views are
searched. But if you omit this attribute and specify a zone, the appliance searches the 'default' view only.
     zone      - Optional. A zone name in FQDN format.
```

Example

```
# get all the DNSKEY records for the zone 'domain.com' in the default view.
my @retrieved_objs = $session->get(
                     object => "Infoblox::DNS::Record::DNSKEY",
                     name   => "domain.com",
                     view   => "default" );

# get all the DNSKEY records in the "domain.com" zone of the default view
# with algorithm field having the value "RSASHA1" and "flags" having value 257
my @retrieved_objs = $session->get(
                     object    => "Infoblox::DNS::Record::DNSKEY",
                     name      => "domain.com",
                     algorithm => "RSASHA1",
                     flags     => 257,
                     view      => "default" );
```

Infoblox::Session->search( )

Use this method to retrieve the existing objects from an Infoblox appliance. See Infoblox::Session->search() for parameters and return values.

Key Reference

```
Apply the following attributes to search the DNSKEY Record object(s):

    algorithm - Optional. The public key encryption algorithm. The allowed values are
'1','3','5','6','7','RSAMD5','RSASHA1','DSA','NSEC3RSASHA1','NSEC3DSA','RSASHA256','RSASHA512','NSEC3RSASHA256'
 and 'NSEC3RSASHA512'.
    flags     - Optional. Flags field to identify the type of the key.
    key_tag   - Optional. Key tag to identify the public key.
    name      - Optional. FQDN of the DNSKEY record (regular expression).
    public_key- Optional. Public key in the base-64 encoding (regular expression).
    ttl       - Optional. TTL of the DNSKEY record.
    view      - Optional. The DNS view in which the record is located. By default, all DNS views are
searched. But if you omit this attribute and specify a zone, the appliance searches the 'default' view only.
    zone      - Optional. A zone name in FQDN format.
```

Example

```
# search the DNSKEY records in the default view by zone name.
my @retrieved_objs = $session->search(
                     object => "Infoblox::DNS::Record::DNSKEY",
                     name   => ".*[.]com",
                     view   => "default" );

# search for all DNSKEY records in the "domain.com" zone of the default view
# Note that the 'default' view is assumed implicitly here
my @retrieved_objs = $session->search(
    object => "Infoblox::DNS::Record::DNSKEY",
    zone   => "domain.com",
   );
```

METHODS

This section describes all the methods that can be used to retrieve the attribute values of a DNSKEY Record object.

algorithm( )

Use this method to retrieve the public key encryption algorithm of a DNSKEY Record object.

Parameter

none

Returns

The method returns the public key encryption algorithm attribute value. Returned values are one of the following:

RSAMD5
DSA
RSASHA1
NSEC3DSA
NSEC3RSASHA1
RSASHA256
RSASHA512

Example

```
#Get the public key encryption algorithm.
my $algorithm = $dnskey->algorithm();
```

creator( )

Use this method to retrieve the DNSKEY record creator. This is a read-only attribute.

Parameter

None

Returns

The method returns the attribute value.

Example

```
# Get attribute value
my $value = $dnskey->creator();
```

dns_name( )

Use this method to retrieve the FQDN, in punycode format, of the zone that owns the DNSKEY record. This is a read-only attribute.

Parameter

None

Returns

The method returns the attribute value.

Example

```
# Get attribute value
my $value = $dnskey->dns_name();
```

flags( )

Use this method to retrieve the flags of a DNSKEY Record object.

Parameter

none

Returns

The method returns the flags attribute value. The flags field is a 16-bit unsigned integer. Currently, only two bits of this value are used--the least significant bit and bit 7. The other bits are reserved for future use and must be zero. If bit 7 is set to 1, the key is a DNS zone key. Otherwise, the key is not a zone key and cannot be used to verify zone data. The least significant bit indicates "secure entry point property". If it is not zero, the key is a key signing key (KSK type). Otherwise, the key type is ZSK.

Example

```
#Getting flags
my $flags = $dnskey->flags();
```

key_tag( )

Use this method to retrieve the key tag identifying the public key of a DNSKEY Record object.

Parameter

none

Returns

The method returns the key tag of the public key value. A key tag is a 16-bit integer key fingerprint. It is used to distinguish one key from another when multiple keys are associated with the same zone.

Example

```
#Get the key tag of the public key
my $key_tag = $dnskey->key_tag();
```

name( )

Use this method to retrieve the FQDN of the zone that owns the DNSKEY record.

The attribute value can be in unicode format.

Parameter

none

Returns

The method returns the FQDN of the zone that contains the given DNSKEY record.

Example

```
#Get the FQDN of the DNSKEY record
my $name = $dnskey->name();
```

public_key( )

Use this method to retrieve the public key. The format of the returned value depend on the key algorithm.

Parameter

none

Returns

The method returns the public key. The exact format of the returned value depends on the algorithm of the key.

Example

```
#Get public key.
my $public_key = $dnskey->public_key();
```

ttl( )

Use this method to retrieve the Time to Live (TTL) value of a DNSKEY Record object.

Parameter

none

Returns

The method returns the TTL attribute value. The returned parameter is a 32-bit integer (range from 0 to 4294967295) that represents the duration, in seconds, that the record is cached. Zero indicates that the record should not be cached.

Example

```
#Get the TTL value
my $ttl = $dnskey->ttl();
```

view( )

Use this method to retrieve the DNS View object that contains the DNSKEY Record object.

Parameter

none

Returns

The method returns the Infoblox::DNS::View object that contains the DNSKEY record.

Example

```
#Get the view
my $view = $dnskey->view();
```

zone( )

Use this method to retrieve the zone name of a DNSKEY record.

Parameter

none

Returns

The method returns the name of the zone that contains the given DNSKEY record. The returned value is a string in FQDN format.

Example

```
# Get zone
my $zone = $dnskey->zone();
```

SAMPLE CODE

The following sample code demonstrates the session methods on a DNSKEY Record object.

```
#PROGRAM STARTS: Include all the modules that will be used
use strict;
use Infoblox;

#Create a session to the Infoblox device
       my $session = Infoblox::Session->new(
           master   => "192.168.1.2",
           username => "admin",
           password => "infoblox"
       );
       unless ($session) {
          die("Construct session failed: ",
              $session->status_code() . ":" . $session->status_detail());
       }
       print "Session created successfully\n";

       #Enable DNSSEC in the default view

       my $default_view=$session->get(
                                      object=> "Infoblox::DNS::View",
                                      name => "default"
                                     );
       unless($default_view) {
           die("Getting the default view failed:",
```

```
        Infoblox::status_code() . ":" . Infoblox::status_detail());
}
print "Got the default view successfully\n";

$default_view->dnssec_enabled("true")
    or die("Changing the dnssec_enabled in the default view failed:",
           Infoblox::status_code() . ":" . Infoblox::status_detail());
$session->modify($default_view)
    or die("Changing the dnssec_enabled in the default view failed:",
           Infoblox::status_code() . ":" . Infoblox::status_detail());

#Create the zone

print "Creating Member primary server for the zone\n";
my $primary=Infoblox::DNS::Member->new(
                                        ipv4addr => "192.168.1.2",
                                        name => "infoblox.localdomain"
                                      );
unless($primary) {
   die("Unable to create primary server object: ",
        Infoblox::status_code() . ":" . Infoblox::status_detail());
}

my $zone = Infoblox::DNS::Zone->new(
                                     name => "domain.com",
                                     primary => $primary
                                   );
unless ($zone) {
   die("Construct zone failed: ",
        Infoblox::status_code() . ":" . Infoblox::status_detail());
}
print "Zone object created successfully\n";

#Verify if the zone exists
my $object = $session->get(object => "Infoblox::DNS::Zone", name => "domain.com");
unless ($object) {
   print "Zone does not exist on server, safe to add the zone\n";
   $session->add($zone)
      or die("Add zone failed: ",
             $session->status_code() . ":" . $session->status_detail());
}
print "Zone added successfully\n";

#Retrieving zone back from the server in order to sign it
$zone = $session->get(object => "Infoblox::DNS::Zone", name => "domain.com");
unless($zone) {
   die("Retrieving zone back failed: ",
         Infoblox::status_code( ). ":". Infoblox::status_detail( ));
}
print "Zone retrieved for signing successfully.\n";

$zone->dnssec_ksk_algorithm("NSEC3RSASHA1") &&
$zone->dnssec_zsk_algorithm("NSEC3RSASHA1") &&
$zone->dnssec_ksk_size(640) &&
$zone->dnssec_zsk_size(640)
  or die("Changing the zone DNSSEC setting failed: ",
         Infoblox::status_code() . ":" . Infoblox::status_detail());
$session->modify($zone)
  or die("Modifying dnssec values in zone failed: ",
         Infoblox::status_code() . ":" . Infoblox::status_detail());
print "Zone modified successfully\n";

#Signing the zone
$zone->dnssec_signed("true")
   or die("Signing of the zone failed: ",
           Infoblox::status_code() . ":" . Infoblox::status_detail());
       print "Zone signed successfully\n";
```

#Getting the DNSKEY record

```
my $ksk_dnskey=$session->get(
                             object => "Infoblox::DNS::Record::DNSKEY",
                             flags  => 257,
                             name   => "domain.com",
                             view   => "default"
                            );
     unless($ksk_dnskey) {
 die("Getting ksk_dnskey failed: ",
     Infoblox::status_code() . ":" . Infoblox::status_detail());
     }
     print "Got Key-signing key DNSKEY record successfully\n";
     print "Key value: ".$ksk_dnskey->public_key()."\n";
```

```
my $zsk_dnskey=$session->get(
                                object => "Infoblox::DNS::Record::DNSKEY",
                                flags  => 256,
                                name   => "domain.com",
                                view   => "default"
                               );
unless($zsk_dnskey) {
   die("Getting zsk_dnskey failed: ",
       Infoblox::status_code() . ":" . Infoblox::status_detail());
}
print "Got Zone-signing key DNSKEY record successfully\n";
print "Key value: ".$zsk_dnskey->public_key()."\n";
```

#Searching for DNSKEY object

```
my @retrieved_objs=$session->search(
                                      object => "Infoblox::DNS::Record::DNSKEY",
                                      name   => ".*com",
                                      view   => "default"
                                    );
     unless(@retrieved_objs>0) {
   die("Searching for DNSKEY objects failed: ",
       Infoblox::status_code() . ":" . Infoblox::status_detail());
     }

#Removing the created zone and cleaning up the view
$session->remove($zone)
  or die("Unable to remove the zone: ",
         Infoblox::status_code() . ":" . Infoblox::status_detail());
print "Zone removed successfully\n";

$default_view->dnssec_enabled("false")
&& $default_view->override_dnssec("false")
&& $session->modify($default_view)
 or die("Restoring dnssec_enabled value in the default view failed: ",
        Infoblox::status_code() . ":" . Infoblox::status_detail());

####PROGRAM ENDS####
```

AUTHOR

Infoblox Inc. http://www.infoblox.com/

SEE ALSO

Infoblox::Session, Infoblox::Session->get(), Infoblox::Session->search(), Infoblox::DNS::Record::DS, Infoblox::DNS::Record::RRSIG, Infoblox::DNS::View, Infoblox::DNS::Zone

---

COPYRIGHT

Copyright (c) 2017 Infoblox Inc.

Infoblox::DNS::Record::DS - DS Record object.

- NAME
- DESCRIPTION
- CONSTRUCTOR
- SESSION METHODS
  - Infoblox::Session->get( )
  - Infoblox::Session->remove( )
  - Infoblox::Session->search( )
- METHODS
  - algorithm( )
  - cloud_info( )
  - creator( )
  - digest( )
  - digest_type( )
  - dns_name( )
  - key_tag( )
  - last_queried( )
  - name( )
  - ttl( )
  - view( )
  - zone( )
- SAMPLE CODE
- AUTHOR
- SEE ALSO
- COPYRIGHT

NAME

Infoblox::DNS::Record::DS - DS Record object.

DESCRIPTION

The DS key record is a part of the DNS security extension records. It is used to authorize the DNSKEY records of the child zone and thus to establish the DNSSEC chain of trust.

The DS resource record is defined in RFC 4034.

The DS resource records are automatically generated upon the signing of the child zone of an authoritative zone residing on the appliance. You can also upload the external DS records via Infoblox::Session->import_data(). It is not possible to add, remove or modify automatically generated DS records. However, the records imported with Infoblox::Session->import_data() method can be deleted.

CONSTRUCTOR

The Infoblox::DNS::Record::DS object is a read-only object and does not require manual construction.

SESSION METHODS

This section describes all the methods in an Infoblox::Session that can be applied to an DS Record object.

Infoblox::Session->get( )

Use this method to retrieve the existing objects from an Infoblox appliance. See Infoblox::Session->get() for parameters and return values.

Key Reference

```
 Apply the following attributes to get the DS Record object(s):

     algorithm       - Optional. The encryption algorithm of the corresponding DNSKEY record.
                       The allowed values are
'1','3','5','6','7','RSAMD5','RSASHA1','DSA','NSEC3RSASHA1','NSEC3DSA','RSASHA256','RSASHA512','NSEC3RSASHA256'
 and 'NSEC3RSASHA512'.
     digest          - Optional. The digest of the DNSKEY RR.
     digest_type     - Optional. Identifies the algorithm used to construct the digest. Allowed values are
'SHA1', 'SHA256', 1, 2.
     key_tag         - Optional. Key-tag to identify the public key.
     name            - Optional. FQDN of the DS record.
     ttl             - Optional. TTL value of the record.
     view            - Optional. The DNS view in which the record is located. By default, all DNS views are
searched. But if you omit this attribute and specify a zone, the appliance searches the 'default' view only.
     zone            - Optional. A zone name in FQDN format.
```

Example

```
 # get all the DS records for the zone 'domain.com' in the default DNS view
 my @retrieved_objs = $session->get(
                      object => "Infoblox::DNS::Record::DS",
                      name   => "domain.com",
                      view   => "default" );
```

```
# get all the DS records in the "domain.com" zone of the default DNS view
# with digest type "SHA1" and TTL of 1 day:
my @retrieved_objs = $session->get(
                     object     => "Infoblox::DNS::Record::DS",
                     name       => "domain.com",
                     digest_type => "SHA1",
                     ttl        =>  86400,
                     view       => "default" );
```

Infoblox::Session->remove( )

Use this method to submit an object for removal from the Infoblox appliance. See Infoblox::Session->remove() for parameters and return values.

To remove a specific object, first use `get()` or `search()` to retrieve the object, and then submit it for removal.

Example

```
# Get the objects with the same name
my @retrieved_objs = $session->get(
    object => "Infoblox::DNS::Record::DS",
    name   => "domain.com",
    view   => "default" );
# Find the desired object from the retrieved list.
my $desired_ds = $retrieved_objs[0];
# Submit for removal
my $response = $session->remove( $desired_ds );
```

Infoblox::Session->search( )

Use this method to retrieve the existing objects from an Infoblox appliance. See Infoblox::Session->search() for parameters and return values.

Key Reference

```
Apply the following attributes to search the DS Record object(s):
    algorithm       - Optional. The encryption algorithm of the corresponding DNSKEY record.
                      The allowed values are
'1','3','5','6','7','RSAMD5','RSASHA1','DSA','NSEC3RSASHA1','NSEC3DSA','RSASHA256','RSASHA512','NSEC3RSASHA256'
and 'NSEC3RSASHA512'.
    digest          - Optional. Digest of the DNSKEY RR (regular expression).
    digest_type     - Optional. Identifies the algorithm used to construct the digest. Allowed values are
'SHA1', 'SHA256', 1, 2.
    key_tag         - Optional. Key-tag to identify the public key.
    name            - Optional. FQDN of the DS record (regular expression).
    ttl             - Optional. TTL value of the record.
    view            - Optional. The DNS view in which the record is located. By default, all DNS views are
searched. But if you omit this attribute and specify a zone, the appliance searches the 'default' view only.
    zone            - Optional. A zone name in FQDN format.
```

Example

```
# search the DS records in the default DNS view by zone name.
my @retrieved_objs = $session->search(
                     object => "Infoblox::DNS::Record::DS",
                     name   => ".*[.]com",
                     view   => "default" );

# search for all DS records in the "domain.com" zone of the default DNS view
# Note that the 'default' view is assumed implicitly here
my @retrieved_objs = $session->search(
    object => "Infoblox::DNS::Record::DS",
    zone   => "domain.com",
   );
```

METHODS

This section describes all the methods that can be used to retrieve the attribute values of a DS Record object.

algorithm( )

Use this method to retrieve the public key encryption algorithm of the DNSKEY record of the given DS record.

Parameter

none

Returns

The method returns the public key encryption algorithm attribute value. Returned values are one of the following:

RSAMD5
DSA
RSASHA1
NSEC3DSA
NSEC3RSASHA1

RSASHA256
RSASHA512
Example

```
#Get the public key encryption algorithm.
my $algorithm = $ds_record->algorithm();
```

cloud_info( )

Use this method to retrieve cloud API related information for the Infoblox::DNS::Record::DS object.

Parameter

None

Returns

The method returns the attribute value.

Example

```
# Get cloud_info
my $cloud_info = $object->cloud_info();
```

creator( )

Use this method to retrieve the DS record creator. This is a read-only attribute.

Parameter

None

Returns

The method returns the attribute value.

Example

```
# Get attribute value
my $value = $object->creator();
```

digest( )

Use this method to retrieve the digest of the DNSKEY resource record that is stored in a DS Record object.

Parameter

none

Returns

The method returns the digest of the corresponding DNSKEY RR attribute value.

Example

```
#Get the digest of the DNSKEY RR
my $digest = $ds_record->digest();
```

digest_type( )

Use this method to retrieve the algorithm that is used to construct the digest stored in the digest field.

Parameter

none

Returns

The method returns the algorithm that is used to construct the digest attribute value.

Example

```
#Get the algorithm used to construct the digest.
my $digest_type = $ds_record->digest_type();
```

dns_name( )

Use this method to retrieve the FQDN, in punycode format, of the DNSKEY record referred by the DS Record object. This is a read-only attribute.

Parameter

None

Returns

The method returns the attribute value.

Example

```
# Get attribute value
my $value = $ds_record->dns_name();
```

key_tag( )

Use this method to retrieve the key tag of the DNSKEY RR that is referred by the given DS Record object.

Parameter

none

Returns

The method returns the key tag attribute value. The key tag is used to identify the public keys that belong to the different DS records. The returned value is an unsigned 16 bit integer.

Example

```
#Get key-tag
my $key_tag = $ds_record->key_tag();
```

last_queried( )

Use this method to retrieve the time when the associated record was last queried. This is a read-only attribute.

Parameter

None

Returns

The method returns the attribute value. The number of seconds that have elapsed since January 1st, 1970 UTC.

Example

```
#Get last_queried
my $last_queried = $ds_record->last_queried();
```

name( )

Use this method to retrieve the FQDN of the DNSKEY record referred by the DS Record object.

The attribute value can be in unicode format.

Parameter

none

Returns

The method returns the FQDN of the DNSKEY record that corresponds to the DS record.

Example

```
#Get  FQDN of the DS record
my $name = $ds_record->name();
```

ttl( )

Use this method to retrieve the Time to Live (TTL) value of a DS Record object.

Parameter

none

Returns

The method returns the TTL attribute value. The returned parameter is a 32-bit integer (range from 0 to 4294967295) that represents the duration, in seconds, that the record is cached. Zero indicates that the record should not be cached.

Example

```
#Get  TTL
my $ttl = $ds_record->ttl();
```

view( )

Use this method to retrieve the DNS view object that contains the DS Record object.

Parameter

none

Returns

The method returns the Infoblox::DNS::View object that contains the DS record.

Example

```
#Get  Ref to the view object
my $view = $ds_record->view();
```

zone( )

Use this method to retrieve the zone name of a DS record.

Parameter

none

Returns

Returns the zone name in FQDN format.

Example

```
# Get zone
my $zone = $ds_record->zone();
```

SAMPLE CODE

The following sample code demonstrates the session methods on a DS Record object.

```
#PROGRAM STARTS: Include all the modules that will be used
use strict;
use Infoblox;

#Create a session to the Infoblox device
       my $session = Infoblox::Session->new(
           master   => "192.168.1.2",
           username => "admin",
           password => "infoblox"
       );
       unless ($session) {
          die("Construct session failed: ",
              $session->status_code() . ":" . $session->status_detail());
       }
       print "Session created successfully\n";

       #Enable DNSSEC in the default view

       my $default_view=$session->get(
                                    object=> "Infoblox::DNS::View",
                                    name => "default"
                                   );
       unless($default_view) {
           die("Getting the default view failed:",
              Infoblox::status_code() . ":" . Infoblox::status_detail());
       }
       print "Got the default view successfully\n";

       $default_view->dnssec_enabled("true")
           or die("Changing the dnssec_enabled in the default view failed:",
                    Infoblox::status_code() . ":" . Infoblox::status_detail());
       $session->modify($default_view)
           or die("Changing the dnssec_enabled in the default view failed:",
                    Infoblox::status_code() . ":" . Infoblox::status_detail());

       #Create the zone

       print "Creating Member primary server for the zone\n";
       my $primary=Infoblox::DNS::Member->new(
                                           ipv4addr => "192.168.1.2",
                                           name => "infoblox.localdomain"
                                          );
       unless($primary) {
          die("Unable to create primary server object: ",
              Infoblox::status_code() . ":" . Infoblox::status_detail());
       }

       my $zone = Infoblox::DNS::Zone->new(
```

```
                                        name => "domain.com",
                                        primary => $primary
                                      );
unless ($zone) {
   die("Construct zone failed: ",
       Infoblox::status_code() . ":" . Infoblox::status_detail());
}
print "Zone object created successfully\n";

#Verify if the zone exists
my $object = $session->get(object => "Infoblox::DNS::Zone", name => "domain.com");
unless ($object) {
   print "Zone does not exist on server, safe to add the zone\n";
   $session->add($zone)
      or die("Add zone failed: ",
             $session->status_code() . ":" . $session->status_detail());
}
print "Zone added successfully\n";

#Retrieving zone back from the server in order to sign it
$zone = $session->get(object => "Infoblox::DNS::Zone", name => "domain.com");
unless($zone) {
   die("Retrieving zone back failed: ",
        Infoblox::status_code( ). ":". Infoblox::status_detail( ));
}
print "Zone retrieved for signing successfully.\n";

$zone->dnssec_ksk_algorithm("NSEC3RSASHA1") &&
$zone->dnssec_zsk_algorithm("NSEC3RSASHA1") &&
$zone->dnssec_ksk_size(640) &&
$zone->dnssec_zsk_size(640)
  or die("Changing the zone DNSSEC setting failed: ",
         Infoblox::status_code() . ":" . Infoblox::status_detail());
$session->modify($zone)
  or die("Modifying dnssec values in zone failed: ",
         Infoblox::status_code() . ":" . Infoblox::status_detail());
print "Zone modified successfully\n";

#Signing the zone
$zone->dnssec_signed("true")
   or die("Signing of the zone failed: ",
          Infoblox::status_code() . ":" . Infoblox::status_detail());
       print "Zone signed successfully\n";

# In order to have DS records in the parent zone we need to have a signed subzone
my $subzone = Infoblox::DNS::Zone->new(
                                        name => "subzone.domain.com",
                                        primary => $primary
                                      );
unless ($subzone) {
   die("Construct subzone failed: ",
       Infoblox::status_code() . ":" . Infoblox::status_detail());
}
print "Zone object created successfully\n";

#Verify if the subzone exists
my $object = $session->get(object => "Infoblox::DNS::Zone", name => "subzone.domain.com");
unless ($object) {
   print "Subzone does not exist on server, safe to add the subzone\n";
   $session->add($subzone)
      or die("Add subzone failed: ",
             $session->status_code() . ":" . $session->status_detail());
}
print "Subzone added successfully\n";

#Retrieving subzone back from the server in order to sign it
$subzone = $session->get(object => "Infoblox::DNS::Zone", name => "subzone.domain.com");
unless($subzone) {
   die("Retrieving subzone back failed: ",
        Infoblox::status_code( ). ":". Infoblox::status_detail( ));
}
print "Subzone retrieved for signing successfully.\n";

$subzone->dnssec_ksk_algorithm("NSEC3DSA") &&
$subzone->dnssec_zsk_algorithm("NSEC3DSA") &&
$subzone->dnssec_ksk_size(640) &&
$subzone->dnssec_zsk_size(640)
  or die("Changing the subzone DNSSEC setting failed: ",
         Infoblox::status_code() . ":" . Infoblox::status_detail());
$session->modify($subzone)
  or die("Modifying dnssec values in subzone failed: ",
         Infoblox::status_code() . ":" . Infoblox::status_detail());
```

```
print "Subzone modified successfully\n";

#Signing the subzone
$subzone->dnssec_signed("true")
   or die("Signing of the subzone failed: ",
            Infoblox::status_code() . ":" . Infoblox::status_detail());
        print "Subzone signed successfully\n";

#Getting the DNSKEY record for subzone
my $ksk_dnskey=$session->get(
                             object => "Infoblox::DNS::Record::DNSKEY",
                             flags  => 257,
                             name   => "subzone.domain.com",
                             view   => "default"
                            );
     unless($ksk_dnskey) {
  die("Getting ksk_dnskey failed: ",
      Infoblox::status_code() . ":" . Infoblox::status_detail());
      }
      print "Got Key-signing key DNSKEY record successfully\n";
```

#Retrieving the DS record corresponding to the given DNSKEY

```
my $ds_record=$session->get(
                             object => "Infoblox::DNS::Record::DS",
                             name   => "subzone.domain.com",
                             key_tag => $ksk_dnskey->key_tag(),
                             view   => "default"
                            );
unless($ds_record) {
   die("Getting DS record corresponding to ksk_dnskey failed: ",
       Infoblox::status_code() . ":" . Infoblox::status_detail());
}
print "Got the DS record corresponding to public key with key_tag ".$ksk_dnskey->key_tag()." successfully\n";
print "Digest value: ".$ds_record->digest()."\n";
```

#Searching for DS object

```
my @retrieved_objs=$session->search(
                                    object => "Infoblox::DNS::Record::DS",
                                    name   => ".*com",
                                    view   => "default"
                                   );
     unless(@retrieved_objs>0) {
   die("Searching for DS objects failed: ",
       Infoblox::status_code() . ":" . Infoblox::status_detail());
     }
     print scalar(@retrieved_objs)." DS objects successfully found by search\n";
```

#Removing the created zones and cleaning up the view

```
$session->remove($subzone)
  or die("Unable to remove the subzone: ",
           Infoblox::status_code() . ":" . Infoblox::status_detail());
print "Subzone removed successfully\n";

$session->remove($zone)
  or die("Unable to remove the zone: ",
           Infoblox::status_code() . ":" . Infoblox::status_detail());
print "Zone removed successfully\n";

$default_view->dnssec_enabled("false")
&& $default_view->override_dnssec("false")
&& $session->modify($default_view)
 or die("Restoring dnssec_enabled value in the default view failed: ",
        Infoblox::status_code() . ":" . Infoblox::status_detail());

####PROGRAM ENDS####
```

AUTHOR

Infoblox Inc. http://www.infoblox.com/

SEE ALSO

Infoblox::Session, Infoblox::Session->get(), Infoblox::Session->search(), Infoblox::Session->remove(), Infoblox::DNS::Record::DNSKEY, Infoblox::DNS::View, Infoblox::DNS::Zone

---

COPYRIGHT

Copyright (c) 2017 Infoblox Inc.

Infoblox::DNS::Record::DTCLBDN - A DTC Load Balanced Domain Name Record object.

- NAME
- DESCRIPTION
- CONSTRUCTOR
- SESSION METHODS
  - Infoblox::Session->get( )
  - Infoblox::Session->search( )
- METHODS
  - comment( )
  - disable( )
  - extattrs( )
  - extensible_attributes( )
  - last_queried( )
  - lbdn( )
  - name( )
  - pattern( )
  - view( )
  - zone( )
- AUTHOR
- SEE ALSO
- COPYRIGHT

NAME

Infoblox::DNS::Record::DTCLBDN - A DTC Load Balanced Domain Name Record object.

DESCRIPTION

A DTC Load Balanced Domain Name Record object.

CONSTRUCTOR

A DTC Load Balanced Domain Name Record is automatically created when the Load Balanced Domain Name obscures the existing A or AAAA record.

SESSION METHODS

This section describes all the methods in an Infoblox::Session module that you can apply to a DTC Load Balanced Domain Name Record object object.

Infoblox::Session->get( )

Use this method to retrieve all the matching objects from the Infoblox appliance. See Infoblox::Session->get() for parameters and return values.

Key Reference

```
 Apply the following attribute to get a specific object:

  comment              - Optional. Descriptive comments.
  extattrs             - Optional. A hash reference containing extensible attributes.
  extensible_atributes - Optional. Extensible attributes associated with a Grid Member object
  name                 - Optional. Host name in string format.
  view                 - Optional. The DNS view in which the DTC Load Balanced Domain Name Record is
located. By default, all DNS views are searched. But if you omit this attribute and specify a zone,
the appliance searches the 'default' view only.
  zone                 - Optional. The DNS zone in which the DTC Load Balanced Domain Name Record is
located.
```

Example

```
my @retrieved_objs = $session->get(
    object => "Infoblox::DNS::Record::DTCLBDN",
    name   => "dtclbdn.domain.com",
    view   => "default" );

# get all DTC Load Balanced Domain Name Records in the "domain.com" zone of the default view
my @retrieved_objs = $session->get(
    object => "Infoblox::DNS::Record::DTCLBDN",
    zone   => "domain.com",
    view   => "default" );
```

Infoblox::Session->search( )

Use this method to search for objects in the Infoblox appliance. See Infoblox::Session->search() for parameters and return values.

Key Reference

```
 Apply the following attribute to get a specific object:

  comment              - Optional. Descriptive comments.
  extattrs             - Optional. A hash reference containing extensible attributes.
  extensible_atributes - Optional. Extensible attributes associated with a Grid Member object
  name                 - Optional. Host name in string format.
  view                 - Optional. The DNS view in which the DTC Load Balanced Domain Name Record is
located. By default, all DNS views are searched. But if you omit this attribute and specify a zone,
the appliance searches the 'default' view only.
  zone                 - Optional. The DNS zone in which the DTC Load Balanced Domain Name Record is
located.
```

Example

```
my @retrieved_objs = $session->get(
    object => "Infoblox::DNS::Record::DTCLBDN",
    name   => "dtclbdn.domain.com",
    view   => "default" );

# get all DTC Load Balanced Domain Name Records in the "domain.com" zone of the default view
my @retrieved_objs = $session->get(
    object => "Infoblox::DNS::Record::DTCLBDN",
    zone   => "domain.com",
    view   => "default" );
```

METHODS

This section describes all the methods that you can use to set or retrieve the attribute values of the object.

comment( )

Use this method to retrieve a user comment. This is a read-only attribute.

Parameter

None

Returns

The method returns attribute value.

Example

```
#get comment value
my $comment = $dtclbdn->comment();
```

disable( )

Use this method to retrieve the disabled state for a DTC Load Balanced Doamin Name. This is a read-ony attribute.

Parameter

None

Returns

The method returns attribute value.

Example

```
#get disable value
my $disable = $dtclbdn->disable();
```

extattrs( )

Use this method to set or retrieve the extensible attributes associated with a DTC Load Balanced Domain Name object. This is a read-only attribute.

Parameter

None

Returns

Use this method to set or retrieve the extensible attributes associated with a DTC Load Balanced Domain Name object. This is a read-only attribute.

Example

```
#get extattrs value
my $extattrs = $dtclbdn->extattrs();
```

extensible_attributes( )

Use this method to set or retrieve the extensible attributes associated with a DTC Load Balancing Domain Name object. This is a read-only attribute.

Parameter

None

Returns

The method returns attribute value.

Example

```
#get extensible_attributes value
my $ea = $dtclbdn->extensible_attributes();
```

last_queried( )

Use this method to retrieve the time when the associated record was last queried. This is a read-only attribute. This is a read-only attribute.

Parameter

None

Returns

The method returns the attribute value. The number of seconds that have elapsed since January 1st, 1970 UTC.

Example

```
#Get last_queried
my $last_queried = $dtclbdn->last_queried();
```

lbdn( )

Use this method to retrieve a DTC Load Balanced Domain Name object. This is a read-only attribute.

Parameter

None

Returns

The method returns attribute value that can be an Infoblox::DTC::LBDN object.

Example

```
#get lbdn value
my $lbdn = $dtclbdn->lbdn();
```

name( )

Use this method to retrieve the name of a DTC Load Balanced Domain Name object. This is a read-only attribute.

Parameter

None

Returns

The method returns attribute value.

Example

```
#get name value
```

```
my $name = $dtclbdn->name();
```

pattern( )

Use this method to retrieve a DNS FQDN pattern that generated this record. This is a read-only attribute.

Parameter

None

Returns

The method returns attribute value.

Example

```
#get pattern value
my $pattern = $dtclbdn->pattern();
```

view( )

Use this method to retrieve the view name of a DTC Load Balanced Domain Name Record. This is a read-only attribute.

Parameter

None

Returns

The method returns attribute value.

Example

```
#get view value
my $view = $dtclbdn->view();
```

zone( )

Use this method to retrieve the zone name of a DTC Load Balanced Domain Name Record. This is a read-only attribute.

Parameter

None

Returns

The method returns attribute value.

Example

```
#get zone value
my $zone = $dtclbdn->zone();
```

AUTHOR

Infoblox Inc. http://www.infoblox.com/

SEE ALSO

Infoblox::Session, Infoblox::DNS::View, Infoblox::DNS::Zone, Infoblox::DTC::LBDN

COPYRIGHT

Copyright (c) 2017 Infoblox Inc.

Infoblox::DNS::Record::MX - DNS MX record object

- NAME
- DESCRIPTION
- CONSTRUCTOR
- SESSION METHODS
  - Infoblox::Session->add( )
  - Infoblox::Session->get( )
  - Infoblox::Session->modify( )
  - Infoblox::Session->remove( )
  - Infoblox::Session->search( )
- METHODS
  - comment( )
  - cloud_info( )
  - creator( )
  - creation_time( )
  - ddns_principal( )
  - ddns_protected( )
  - disable( )
  - dns_exchanger( )
  - dns_name( )
  - exchanger( )
  - extattrs( )
  - extensible_attributes( )
  - forbid_reclamation( )
  - last_queried( )
  - name( )
  - pref( )
  - reclaimable( )
  - ttl( )
  - views( )
  - zone( )
- SAMPLE CODE
- AUTHOR
- SEE ALSO
- COPYRIGHT

NAME

Infoblox::DNS::Record::MX - DNS MX record object

DESCRIPTION

An MX (mail exchanger) record maps a domain name to a mail exchanger. A mail exchanger is a server that either delivers or forwards mail. You can specify one or more mail exchangers for a zone, as well as the priority for using each mail exchanger. A standard MX record applies to a particular domain or subdomain. A wildcard MX record applies to a domain and all its subdomains.

Note that you must specify only one view for the attribute "views".

CONSTRUCTOR

```
my $mx = Infoblox::DNS::Record::MX->new(
    exchanger             => $fqdn,                                                    # Required
    name                  => $fqdn,                                                    # Required
    pref                  => $num,                                                     # Required
    comment               => $string,                                                  # Optional
/ Default is empty
    disable               => "true" | "false",                                         # Optional
/ Default "false"
    extattrs              => { $string => $extattr, ... },                             # Optional
/ Default is undefined
    extensible_attributes => { $string => $string|$num, $string => [ $string|$num, ...], ... }, # Optional
/ Default is undefined
    ttl                   => $num | undef,                                             # Optional
/ Default is undefined
    views                 => [$View1]                                                  # Optional
/ Default is "default" view
    creator               => "DYNAMIC" | "STATIC",                                     # Optional
/ Default is "STATIC"
    ddns_protected        => "true" | "false",                                         # Optional
/ Default is "false"
    ddns_principal        => $string,                                                  # Optional
```

```
/ Default is undefined
     forbid_reclamation    => "true" | "false",                                    # Optional
/ Default is "false"
 );
```

You cannot set both extattrs and extensible_attributes attributes at the same time.

SESSION METHODS

This section describes all the methods in Infoblox::Session module that you can apply to a DNS MX object.

Infoblox::Session->add( )

Use this method to add an object to the Infoblox appliance. See Infoblox::Session->add() for parameters and return values.

Example

```
#Construct an object
my $bindmx = Infoblox::DNS::Record::MX->new(
    name      => "bind_mx.domain.com",
    comment   => "add mx bind_mx.domain.com",
    pref      => 1,
    exchanger => "exchanger.domain.com",
);
# Submit for addition
my $response = $session->add( $bindmx );
```

Infoblox::Session->get( )

Use this method to retrieve all the matching objects from the Infoblox appliance. See Infoblox::Session->get() for parameters and return values.

Key References

```
Apply the following attributes to get a specific DNS MX object:

  name                  - Optional.  A domain name in string format
  view                  - Optional.  The DNS view in which the MX record is located. By default, all
DNS views are searched. But if you omit this attribute and specify a zone, the appliance searches the
'default' view only.
  zone                  - Optional. A zone name in FQDN format.
  exchanger             - Optional. The exchanger of the MX record.
  pref                  - Optional. The Preference of the MX record.
  creator               - Optional. Record creator.
  ddns_principal        - Optional. GSS-TSIG principal that owns the record.
  reclaimable           - Optional. The flag that indicates whether the record is reclaimable or not.
  extattrs              - Optional. A hash reference containing extensible attributes.
  extensible_attributes - Optional. A hash reference containing extensible attributes.
```

Example

```
 my @retrieved_objs = $session->get(
    object    => "Infoblox::DNS::Record::MX",
    name      => "bind_mx.domain.com",
    view      => "default" );

 # get all DNS MX records in the "domain.com" zone of the default view
 my @retrieved_objs = $session->get(
    object    => "Infoblox::DNS::Record::MX",
    zone      => "domain.com",
    view      => "default" );

# get all DNS MX records with a given extensible attribute
my @retrieved_objs = $session->get(
    object => "Infoblox::DNS::Record::MX",
    extensible_attributes => { 'Site' => 'Santa Clara' });
```

Infoblox::Session->modify( )

Use this method to modify an object in the Infoblox appliance. See Infoblox::Session->modify() for parameters and return values.

Example

```
#Use method to modify the comment.
$bindmx->comment("this is a modified comment");
# Submit modification
my $response = $session->modify( $bindmx );
```

Infoblox::Session->remove( )

Use this method to remove an object from the Infoblox appliance. See Infoblox::Session->remove() for parameters and return values.

To remove a specifc object, first use `get()` or `search()` to retrieve the specific object, and then submit this object for removal.

Example

```
# Get the object under the same name
my @retrieved_objs = $session->get(
    name      => "bind_mx.domain.com",
    comment   => "add mx bind_mx.domain.com",
    pref      => 1,
    exchanger => "exchanger.domain.com",
    view      => "default" );
# find the desired object from retrieved list.
my $desired_mx = $retrieved_objs[0];
# Submit for removal
my $response = $session->remove( $desired _mx );
```

Infoblox::Session->search( )

Use this method to search for DNS MX record objects in the Infoblox appliance. See Infoblox::Session->search() for parameters and return values.

Key References

```
Apply the following attributes to search for a DNS MX record object:

  name                  - Optional. A domain name in string format (regular expression).
  view                  - Optional. The DNS view in which the MX record is located. By default, all
DNS views are searched. But if you omit this attribute and specify a zone, the appliance searches the
'default' view only.
  zone                  - Optional. A zone name in FQDN format.
  comment               - Optional. A comment in string format (regular expression).
  exchanger             - Optional. The exchanger of the MX record (regular expression).
  pref                  - Optional. The Preference of the MX record (regular expression).
  creator               - Optional. Record creator.
  ddns_principal        - Optional. GSS-TSIG principal that owns the record.
  reclaimable           - Optional. The flag that indicates whether the record is reclaimable or not.
  extattrs              - Optional. A hash reference containing extensible attributes.
  extensible_attributes - Optional. A hash reference containing extensible attributes.
```

For more information about searching extensible attributes, see Infoblox::Grid::ExtensibleAttributeDef/Searching Extensible Attributes.

Example

```
# search for all DNS MX objects that match "domain.com" in the default DNS view
    my @retrieved_objs = $session->search(
    object => "Infoblox::DNS::Record::MX",
    name   => 'domain\.com',
    view   => "default" );

# search for all DNS MX objects in the domain.com zone of the default DNS view
    my @retrieved_objs = $session->search(
    object => "Infoblox::DNS::Record::MX",
    zone   => "domain.com",
    view   => "default" );

# search all DNS MX records with a given extensible attribute 'Site'
my @retrieved_objs = $session->search(
   object => "Infoblox::DNS::Record::MX",
   extensible_attributes => { 'Site' => 'Santa Clara' });
```

METHODS

This section describes all the methods that you can use to configure and retrieve the attribute values of an MX record

comment( )

Use this method to add or retrieve a descriptive comment.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

Enter a descriptive comment for this record in string format with a maximum of 256 bytes.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
# Getting comment
my $comment = $bindmx->comment();
# Modifying comment
$bindmx->comment("add mx2.domain.com");
```

cloud_info( )

Use this method to retrieve cloud API related information for the Infoblox::DNS::Record::MX object.

Parameter

None

Returns

The method returns the attribute value.

Example

```
# Get cloud_info
my $cloud_info = $object->cloud_info();
```

creator( )

Use this method to set or retrieve the record creator.

Note that changing creator from or to 'SYSTEM' value is not allowed.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Paramter

The valid values are 'STATIC' and 'DYNAMIC'. The default value is 'STATIC'.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get creator value
my $creator = $object->creator();

#Modify creator value
$object->creator("DYNAMIC");
```

creation_time( )

Use this method to retrieve the creation time for the record. This is a read-only attribute.

Parameter

None

Returns

The valid return value is a number of seconds that have elapsed since January 1st, 1970 UTC.

Example

```
#Get creation_time value
my $creation_time = $object->creation_time();
```

ddns_principal( )

Use this method to set or retrive the GSS-TSIG principal that owns this record.

Note that you cannot set ddns_principal for 'STATIC' and 'SYSTEM' records.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Paramter

The GSS-TSIG principal FQDN (Fully Qualified Domain Name) format. The FQDN consists of the hostname followed by the domain name (example: abc.com). A hostname can have a maximum of 256 characters.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get ddns_principal value
my $ddns_principal = $object->ddns_principal();

#Modify ddns_principal value
$object->ddns_principal('foo.com');
```

ddns_protected( )

Use this method to set or retrieve the flag that indicates whether DDNS updates for this record are allowed or not.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

Specify 'true' to protect record from DDNS updates and 'false' to allow DDNS updates for the specified record.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get ddns_protected value
my $ddns_protected = $object->ddns_protected();

#Modify ddns_protected value
$object->ddns_protected('true');
```

disable( )

Use this method to set or retrieve the disable flag of a DNS record.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

The default value for this field is false. The DNS record is enabled.

Parameter

Specify "true" to set the disable flag or "false" to deactivate/unset it.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
# Getting disable
my $disable = $bindmx->disable()
# Modifying disable
$bindmx->disable("true");
```

dns_exchanger( )

Use this method to retrieve the exchanger in punycode format. This is a read-only attribute.

Parameter

None

Returns

The method returns the attribute value.

Example

```
# Get attribute value
my $value = $bindmx->dns_exchanger();
```

dns_name( )

Use this method to retrieve the domain name in punycode format. This is a read-only attribute.

Parameter

None

Returns

The method returns the attribute value.

Example

```
# Get attribute value
my $value = $bindmx->dns_name();
```

exchanger( )

Use this method to set or retrieve the exchanger.

The name of the mail exchanger in Fully Qualified Domain Name (FQDN) format.

The attribute value can be in unicode format.

Parameter

Text with the name of the mail exchanger for the DNS MX record.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails. If you did not specify a parameter, the method returns the attribute value.

Example

```
# Getting exchanger
my $exchanger = $bindmx->exchanger();
# Modifying exchanger
$bindmx->exchanger("exchanger2.domain.com");
```

extattrs( )

Use this method to set or retrieve the extensible attributes associated with a DNS MX record object.

Parameter

Valid value is a hash reference containing the names of extensible attributes and their associated values (Infoblox::Grid::Extattr objects).

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get extattrs
my $ref_extattrs = $bindmx->extattrs();
#Modify extattrs
$bindmx->extattrs({ 'Site' => $extattr1, 'Administrator' => $extattr2 });
```

extensible_attributes( )

Use this method to set or retrieve the extensible attributes associated with a DNS MX record.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

For valid values for extensible attributes, see Infoblox::Grid::ExtensibleAttributeDef/Extensible Attribute Values.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get extensible attributes
my $ref_extensible_attributes = $bindmx->extensible_attributes();
#Modify extensible attributes
$bindmx->extensible_attributes({ 'Site' => 'Santa Clara', 'Administrator' => [ 'Peter', 'Tom' ] });
```

forbid_reclamation( )

Use this method to set or retrieve the flag that indicates whether the reclamation is allowed for the record or not.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

Specify 'true' to forbid reclamation for the record and 'false' to allow it. The default value is 'false'.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get forbid_reclamation
my $forbid_reclamation = $object->forbid_reclamation();
#Modify forbid_reclamation
$object->forbid_reclamation('true');
```

last_queried( )

Use this method to retrieve the time when the associated record was last queried. This is a read-only attribute.

Parameter

None

Returns

The method returns the attribute value. The number of seconds that have elapsed since January 1st, 1970 UTC.

Example

```
#Get last_queried
my $last_queried = $bindmx->last_queried();
```

name( )

Use this method to set or retrieve the domain name.

The name of the DNS MX object in Fully Qualified Domain Name (FQDN) format. If you want to define an MX record for a domain whose name matches the zone in which you define the MX record, leave this field empty. If you want to define an MX record for a subdomain, enter that subdomain name here. If you want to define an MX record for a domain and all its subdomains, enter an asterisk (*) here to create a wildcard MX record.

The attribute value can be in unicode format.

Parameter

Text with the name for the DNS MX record.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Getting name
```

```
my $name = $bindmx->name();
# Modifying name
$bindmx->name("mx2.domain.com");
```

pref( )

Use this method to set or retrieve the Preference value.

The preference value of the DNS MX object. Preference value of MX record for the zone range from 0 to 65535. Low values are more preferred.

Parameter

Enter an unsigned integer between 0-65535. The priority determines the order in which a client attempts to contact the target mail exchanger. The highest priority is 0 and is queried first.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
# Getting pref
my $pref = $bindmx->pref();
# Modifying pref
$bindmx->pref(3);
```

reclaimable( )

Use this method to retrieve the flag that indicates whether the record is reclaimable or not.

Parameter

None

Returns

The method returns the attribute value.

Example

```
#Get reclaimable
my $reclaimable = $object->reclaimable();
```

ttl( )

Use this method to configure or retrieve the Time to Live (TTL) value.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

The default value is undefined which indicates that the record inherits the TTL value of the zone.

Specify a TTL value to override the TTL value at the zone level.

Parameter

A 32-bit integer (range from 0 to 4294967295) that represents the duration in seconds that the record is cached. Zero indicates that the record should not be cached.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get ttl
my $ttl = $bindmx->ttl();
#Modify ttl
$bindmx->ttl(1800);
#Un-override ttl
$bindmx->ttl(undef);
```

views( )

Use this method to specify or retrieve the view of the MX record.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

The default value is the 'default' view, which means the MX record is located under the default view.

Parameter

Array reference of defined Infoblox::DNS::View objects.

Note that the array size must be 1.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get views
my $ref_views = $bindmx->views();
#Modify views, list of Infoblox::DNS::View objects
$bindmx->views([$view1]);
```

zone( )

Use this method to retrieve the zone name of a DNS MX record. This method is read-only and cannot be set.

Parameter

None

Returns

Returns the attribute value.

Example

```
# Get zone
my $zone = $bindmx->zone();
```

SAMPLE CODE

The following sample code demonstrates the different functions that can be applied to an object, such as add, search, modify, and remove. This sample also includes error handling for the operations.

#Preparation prior to a DNS MX record insertion

```
#PROGRAM STARTS: Include all the modules that will be used
use strict;
use Infoblox;

#Create a session to the Infoblox appliance
my $session = Infoblox::Session->new(
    master   => "192.168.1.2",
    username => "admin",
    password => "infoblox"
);
unless ($session) {
   die("Construct session failed: ",
       Infoblox::status_code() . ":" . Infoblox::status_detail());
}
print "Session created successfully\n";

#Create the zone prior to MX  record insertion
my $zone = Infoblox::DNS::Zone->new(name => "domain.com");
unless ($zone) {
   die("Construct zone failed: ",
     Infoblox::status_code() . ":" . Infoblox::status_detail());
}
print "Zone object created successfully\n";

#Verify if the zone exists
my $object = $session->get(object => "Infoblox::DNS::Zone", name => "domain.com");
unless ($object) {
   print "Zone does not exist on server, safe to add the zone\n";
   $session  add $zone
```

```
            ->   (     )
      or die("Add zone failed: ",
             $session->status_code() . ":" . $session->status_detail());
}
print "Zone added successfully\n";
```

#Create a DNS MX record

```
 #Construct a DNS MX object
  my $bindmx = Infoblox::DNS::Record::MX->new(
    name      => "bind_mx.domain.com",
    comment   => "add mx bind_mx.domain.com",
    pref      => 1,
    exchanger => "exchanger.domain.com",
  );

unless ($bindmx) {
   die("Construct DNS record MX failed: ",
       Infoblox::status_code() . ":" . Infoblox::status_detail());
}
print "DNS MX object created successfully\n"; #Add the DNS MX record object to Infoblox appliance through
a session
$session->add($bindmx)
    or die("Add record MX failed: ",
           $session->status_code() . ":" . $session->status_detail());
print "DNS MX object added to server successfully\n";
```

#Search for a specific DNS MX record

```
#Search all MX records that match "domain.com"
my @retrieved_objs = $session->search(
    object => "Infoblox::DNS::Record::MX",
    name   => 'domain\.com'
);
my $object = $retrieved_objs[0];
unless ($object) {
    die("Search record MX failed: ",
        $session->status_code() . ":" . $session->status_detail());
}
print "Search DNS MX object found at least 1 matching entry\n";

#Search all MX records that start with "bind" and end with "domain.com"
my @retrieved_objs = $session->search(
    object => "Infoblox::DNS::Record::MX",
    name   => '^bind.*\.domain\.com\$'
);
my $object = $retrieved_objs[0];
unless ($object) {
    die("Search record MX failed: ",
        $session->status_code() . ":" . $session->status_detail());
}
print "Search DNS MX object using regexp found at least 1 matching entry\n";
```

#Get and modify a DNS MX record

```
#Get MX record through the session
my  @retrieved_objs = $session->get(
    object    => "Infoblox::DNS::Record::MX",
    name      => "bind_mx.domain.com",
    pref      => 1,
    exchanger => "exchanger.domain.com",
);

my $object = $retrieved_objs[0];
unless ($object) {
    die("Get record MX failed: ",
        $session->status_code() . ":" . $session->status_detail());
}
print "Get DNS MX object found at least 1 matching entry\n";

#Modify one of the attributes of the specified MX record
$object->exchanger("exc.domain.com");

#Apply the changes
$session->modify($object)
    or die("Modify record MX failed: ",
           $session->status_code() . ":" . $session->status_detail());
```

```
print "DNS MX object modified successfully \n";
```

#Remove a DNS MX record

```
 #Get MX record through the session
 my @retrieved_objs = $session->get(
     object => "Infoblox::DNS::Record::MX",
     name   => "bind_mx.domain.com"
 );
  my $object = $retrieved_objs[0];
  unless ($object) {
   die("Get record MX failed: ",
       $session->status_code() . ":" . $session->status_detail());
 }
  print "Get DNS MX object found at least 1 matching entry\n";

#Submit the object for removal
$session->remove($object)
    or die("Remove record MX failed: ",
        $session->status_code() . ":" . $session->status_detail());
print "DNS MX object removed successfully \n";

####PROGRAM ENDS####
```

AUTHOR

Infoblox Inc. http://www.infoblox.com/

SEE ALSO

Infoblox::DNS::View, Infoblox::DNS::Zone, Infoblox::Session, Infoblox::Session->get(), Infoblox::Session->search(), Infoblox::Session->add(), Infoblox::Session->remove(), Infoblox::Session->modify()

COPYRIGHT

Copyright (c) 2017 Infoblox Inc.

Infoblox::DNS::Record::NAPTR - NAPTR record object.

- NAME
- DESCRIPTION
- CONSTRUCTOR
- SESSION METHODS
  - Infoblox::Session->add( )
  - Infoblox::Session->get( )
  - Infoblox::Session->modify( )
  - Infoblox::Session->remove( )
  - Infoblox::Session->search( )
- METHODS
  - comment( )
  - cloud_info( )
  - creator( )
  - creation_time( )
  - ddns_principal( )
  - ddns_protected( )
  - disable( )
  - dns_name( )
  - dns_replacement( )
  - extattrs( )
  - extensible_attributes( )
  - flags( )
  - forbid_reclamation( )
  - last_queried( )
  - name( )
  - order( )
  - preference( )
  - reclaimable( )
  - regexp( )
  - replacement( )
  - services( )
  - ttl( )
  - views( )
  - zone( )
- SAMPLE CODE
- AUTHOR
- SEE ALSO
- COPYRIGHT

NAME

Infoblox::DNS::Record::NAPTR - NAPTR record object.

DESCRIPTION

A DNS NAPTR object represents a Naming Authority Pointer (NAPTR) resource record. This resource record specifies a regular expression-based rewrite rule that, when applied to an existing string, produces a new domain name or URI.

CONSTRUCTOR

```
my $naptr = Infoblox::DNS::Record::NAPTR->new(
    name                => $fqdn,                                                    # Required
    order               => $num,                                                     # Required
    preference          => $num,                                                     # Required
    flags               => $string,                                                  # Optional
    services            => $string,                                                  # Optional
    regexp              => $string,                                                  # Optional
    replacement         => $string                                                   # Required
    comment             => $string,                                                  # Optional
/ Default is empty
    disable             => "true" | "false",                                         # Optional
/ Default is "false"
    extattrs            => { $string => $extattr, ... },                             # Optional
/ Default is undefined
    extensible_attributes => { $string => $string|$num, $string => [ $string|$num, ...], ... }, # Optional
/ Default is undefined
    ttl                 => $num | undef,                                             # Optional
/ Default is undefined
    views               => [$View1],                                                 # Optional
```

```
/ Default is "default" view
     creator               => "DYNAMIC" | "STATIC",                          # Optional
/ Default is "STATIC"
     ddns_protected        => "true" | "false",                              # Optional
/ Default is "false"
     ddns_principal        => $string,                                       # Optional
/ Default is undefined
     forbid_reclamation    => "true" | "false",                              # Optional
/ Default is "false"
 );
```

Note that you must specify only one DNS view for the attribute "views".

You cannot set both extattrs and extensible_attributes attributes at the same time.

SESSION METHODS

This section describes all the methods in an Infoblox::Session module that can be applied to an NAPTR record object.

Infoblox::Session->add( )

Use this method to add an object to the Infoblox appliance. See Infoblox::Session->add() for parameters and return values.

Example

```
#Construct an object
my $naptr = Infoblox::DNS::Record::NAPTR->new(
     name            => 'naptr.domain.com',
     order           => 100,
     preference      => 10,
     flags           => 'U',
     services        => 'http+E2U',
     regexp          => '!http://my[.](.*)!1!i',
     replacement     => 'domain2.com',
     comment         => 'Sample NAPTR record',
     disable         => 'false',
     ttl             => 84600,
     views           => [$view1],
);
# Submit for addition
my $response = $session->add( $naptr );
```

Infoblox::Session->get( )

Use this method to retrieve the existing objects from an Infoblox appliance. See Infoblox::Session->get() for parameters and return values.

Key Reference

```
Apply the following attributes to get specific NAPTR record objects:

     comment               - Optional. Descriptive comment in string format
     creator               - Optional. Record creator.
     ddns_principal        - Optional. GSS-TSIG principal that owns the record.
     reclaimable           - Optional. The flag that indicates whether the record is reclaimable or
not.
     extattrs              - Optional. A hash reference containing extensible attributes.
     extensible_attributes - Optional. A hash reference containing extensible attributes.
     flags                 - Optional. Flags in string format
     name                  - Optional. FQDN of the record in string format
     order                 - Optional. Order parameter (unsigned integer)
     preference            - Optional. Preference parameter (unsigned integer)
     replacement           - Optional. Replacement field in string format
     services              - Optional. Services field in string format
     view                  - Optional. The DNS view in which the NAPTR record is located. By default,
all DNS views are searched. But if you omit this attribute and specify a zone, the appliance searches
the 'default' view only.
     zone                  - Optional. A zone name in FQDN format.
```

Example

```
my @retrieved_objs = $session->get(
     object => "Infoblox::DNS::Record::NAPTR",
     name   => "naptr1.domain.com",
     view   => "default" );

#get all DNS A records with the extensible attribute 'Site'
my @retrieved_objs = $session->get(
   object => "Infoblox::DNS::Record::NAPTR",
   extensible_attributes => { 'Site' => 'Santa Clara' });
```

Infoblox::Session->modify( )

Use this method to modify an object in the Infoblox appliance. See Infoblox::Session->modify() for parameters and return values.

Example

```
# Use this method to modify the comment.
$naptr->comment("this is a modified comment");
# Submit the modification
my $response = $session->modify( $naptr );
```

Infoblox::Session->remove( )

Use this method to submit the removal of an object from the Infoblox appliance. See Infoblox::Session->remove() for parameters and return values.

To remove a specific object, first use `get()` or `search()` to retrieve the object, and then submit it for removal.

Example

```
# Get the objects with the same name
my @retrieved_objs = $session->get(
    object => "Infoblox::DNS::Record::NAPTR",
    name   => "naptr1.domain.com",
    view   => "default" );
# Find the desired object in the retrieved list.
my $desired_rec = $retrieved_objs[0];
# Submit for removal
my $response = $session->remove( $desired_rec );
```

Infoblox::Session->search( )

Use this method to retrieve the existing objects from an Infoblox appliance. See Infoblox::Session->search() for parameters and return values.

Key Reference

```
 Apply the following attributes to search the NAPTR record objects:

     comment               - Optional. Descriptive comment (regular expression)
     creator               - Optional. Record creator.
     ddns_principal        - Optional. GSS-TSIG principal that owns the record.
     reclaimable           - Optional. The flag that indicates whether the record is reclaimable or
not.
     extattrs              - Optional. A hash reference containing extensible attributes.
     extensible_attributes - Optional. A hash reference containing extensible attributes.
     flags                 - Optional. Flags in string format (regular expression)
     name                  - Optional. FQDN of the record in string format (regular expression)
     order                 - Optional. Order parameter (unsigned integer, exact search)
     preference            - Optional. Preference parameter (unsigned integer, exact search)
     replacement           - Optional. Replacement field in string format (regular expression)
     services              - Optional. Services field in string format (regular expression)
     view                  - Optional. The DNS view in which the NAPTR record is located. By default,
all DNS views are searched. But if you omit this attribute and specify a zone, the appliance searches
the 'default' view only. Only exact search is supported.
     zone                  - Optional. A zone name in FQDN format (exact search)
```

For more information about searching extensible attributes, see Infoblox::Grid::ExtensibleAttributeDef/Searching Extensible Attributes.

Example

```
# search for all NAPTR objects that match "domain.com" in the default DNS view
my @retrieved_objs = $session->search(
    object => "Infoblox::DNS::Record::NAPTR",
    name   => '.*domain\.com',
    view   => "default" );

# search for all NAPTR records in the "domain.com" zone of the default DNS view
my @retrieved_objs = $session->search(
    object => "Infoblox::DNS::Record::NAPTR",
    zone   => "domain.com",
    view   => "default" );

#get all NAPTR records with the extensible attribute 'Site'
my @retrieved_objs = $session->search(
  object => "Infoblox::DNS::Record::NAPTR",
  extensible_attributes => { 'Site' => 'Santa Clara' });
```

METHODS

This section describes all the methods that can be used to retrieve the attribute values of an NAPTR record object.

comment( )

Use this method to set or retrieve the descriptive comment of an NAPTR record object.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

Desired comment in string format with a maximum of 256 bytes.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get the descriptive comment
my $comment = $naptr->comment();
#Modify the comment
$naptr->comment("This is a new comment");
```

cloud_info( )

Use this method to retrieve cloud API related information for the Infoblox::DNS::Record::NAPTR object.

Parameter

None

Returns

The method returns the attribute value.

Example

```
# Get cloud_info
my $cloud_info = $object->cloud_info();
```

creator( )

Use this method to set or retrieve the record creator.

Note that changing creator from or to 'SYSTEM' value is not allowed.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Paramter

The valid values are 'STATIC' and 'DYNAMIC'. The default value is 'STATIC'.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get creator value
my $creator = $object->creator();

#Modify creator value
$object->creator("DYNAMIC");
```

creation_time( )

Use this method to retrieve the creation time for the record. This is a read-only attribute.

Parameter

None

Returns

The valid return value is a number of seconds that have elapsed since January 1st, 1970 UTC.

Example

```
#Get creation_time value
my $creation_time = $object->creation_time();
```

ddns_principal( )

Use this method to set or retrive the GSS-TSIG principal that owns this record.

Note that you cannot set ddns_principal for 'STATIC' and 'SYSTEM' records.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Paramter

The GSS-TSIG principal FQDN (Fully Qualified Domain Name) format. The FQDN consists of the hostname followed by the domain name (example: abc.com). A hostname can have a maximum of 256 characters.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get ddns_principal value
my $ddns_principal = $object->ddns_principal();

#Modify ddns_principal value
$object->ddns_principal('foo.com');
```

ddns_protected( )

Use this method to set or retrieve the flag that indicates whether DDNS updates for this record are allowed or not.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

Specify 'true' to protect record from DDNS updates and 'false' to allow DDNS updates for the specified record.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get ddns_protected value
my $ddns_protected = $object->ddns_protected();

#Modify ddns_protected value
$object->ddns_protected('true');
```

disable( )

Use this method to set or retrieve the disable flag of an NAPTR record.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

The default value for this field is false, which indicates that the NAPTR record is enabled.

Parameter

Specify "true" to set the disable flag or "false" to deactivate/unset it.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get the disable field
my $disable = $naptr->disable();
```

dns_name( )

Use this method to retrieve the FQDN, in punycode format, of the NAPTR record. This is a read-only attribute.

Parameter

None

Returns

The method returns the attribute value.

Example

```
# Get attribute value
my $value = $naptr->dns_name();
```

dns_replacement( )

Use this method to retrieve the replacement field, in punycode format, of an NAPTR record object. This is a read-only attribute.

Parameter

None

Returns

The method returns the attribute value.

Example

```
# Get attribute value
my $value = $naptr->dns_replacement();
```

extattrs( )

Use this method to set or retrieve the extensible attributes associated with a NAPTR record object.

Parameter

Valid value is a hash reference containing the names of extensible attributes and their associated values (Infoblox::Grid::Extattr objects).

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get extattrs
my $ref_extattrs = $naptr->extattrs();
#Modify extattrs
$naptr->extattrs({ 'Site' => $extattr1, 'Administrator' => $extattr2 });
```

extensible_attributes( )

Use this method to set or retrieve the extensible attributes associated with an NAPTR record.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

For valid values for extensible attributes, see Infoblox::Grid::ExtensibleAttributeDef/Extensible Attribute Values.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get  extensible attributes
```

```
my $ref_extensible_attributes = $naptr->extensible_attributes();
#Modify extensible attributes
$naptr->extensible_attributes({ 'Site' => 'Santa Clara', 'Administrator' => [ 'Peter', 'Tom' ] });
```

flags( )

Use this method to retrieve the flags to control the interpretation of the fields of an NAPTR record object.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

Currently supported values for the flags field are "U", "S", "P" and "A".

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get the flags field
my $flags = $naptr->flags();
#Modify flags
$naptr->flags("U");
```

forbid_reclamation( )

Use this method to set or retrieve the flag that indicates whether the reclamation is allowed for the record or not.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

Specify 'true' to forbid reclamation for the record and 'false' to allow it. The default value is 'false'.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get forbid_reclamation
my $forbid_reclamation = $object->forbid_reclamation();
#Modify forbid_reclamation
$object->forbid_reclamation('true');
```

last_queried( )

Use this method to retrieve the time when the associated record was last queried. This is a read-only attribute.

Parameter

None

Returns

The method returns the attribute value. The number of seconds that have elapsed since January 1st, 1970 UTC.

Example

```
#Get last_queried
my $last_queried = $naptr->last_queried();
```

name( )

Use this method to retrieve the FQDN (Fully Qualified Domain Name) of the NAPTR record.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

The attribute value can be in unicode format.

Parameter

The name parameter is a string in FQDN format. The FQDN consists of the record name followed by the domain name (example:

rec.abc.com). A record name can have a maximum of 256 bytes.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get the FQDN of the record
my $name = $naptr->name();
#Modify the FQDN of the record
$naptr->name("rec2.domain.com");
```

order( )

Use this method to retrieve the order parameter of an NAPTR record. This parameter specifies the order in which the NAPTR rules are applied when multiple rules are present.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

The order parameter is a 16-bit unsigned integer.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get the order parameter
my $order = $naptr->order();
#Modify the order
$naptr->order(200);
```

preference( )

Use this method to retrieve the preference field of an NAPTR record. The preference field determines the order NAPTR records are processed when multiple records with the same order parameter are present.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

The preference parameter is a 16-bit integer.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get the preference attribute value
my $preference = $naptr->preference();
#Modify preference
$naptr->preference(300);
```

reclaimable( )

Use this method to retrieve the flag that indicates whether the record is reclaimable or not.

Parameter

None

Returns

The method returns the attribute value.

Example

```
#Get reclaimable
my $reclaimable = $object->reclaimable();
```

regexp( )

Use this method to specify the regular expression-based rewriting rule of an NAPTR record. This should be a POSIX compliant regular expression, including the substitution rule and flags. Refer to RFC 2915 for the field syntax details.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

The valid parameter is a POSIX compliant regular expression or substitution expression.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get regexp
my $regexp = $naptr->regexp();
#Modify regexp
$naptr->regexp("!http://(.*)!1!i");
```

replacement( )

Use this method to retrieve the replacement field of an NAPTR record object. For nonterminal NAPTR records, this field specifies the next domain name to look up.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

The attribute value can be in unicode format.

Parameter

The replacement field is in FQDN format.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get the replacement field
my $replacement = $naptr->replacement();
#Modify the replacement field
$naptr->replacement("__tcp__.domain.com");
```

services( )

Use this field to specify services. The services field contains protocol and service identifiers, for example "http+E2U" or "SIPS+D2T".

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

The services field in string format.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get services
my $services = $naptr->services();
#Modify services
$naptr->services("http+E2U");
```

ttl( )

Use this method to set or retrieve the Time to Live (TTL) value.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

The default value is undefined, which indicates that the record inherits the TTL value of the zone.

Specify a TTL value to override the TTL value at the zone level.

Parameter

A 32-bit integer (from 0 to 4294967295) that represents the duration, in seconds, that the record is cached. Zero indicates that the record should not be cached.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get the TTL of the record
my $ttl = $naptr->ttl();
#Modify TTL
$naptr->ttl(900);
#Use zone ttl settings
$naptr->ttl(undef);
```

views( )

Use this method to set or retrieve the DNS view of the NAPTR record.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

The default value is the "default" view, which means the NAPTR record is located in the default DNS view.

Parameter

Array reference of defined Infoblox::DNS::View objects.

Note that the array size must be 1.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get views
my $ref_views = $naptr->views();
#Modify views, list of Infoblox::DNS::View objects
$naptr->views([$view1]);
```

zone( )

Use this method to retrieve the zone name of an NAPTR record. This method is read-only and cannot be set.

Parameter

None

Returns

Returns the attribute value.

Example

```
# Get zone
my $zone = $naptr->zone();
```

SAMPLE CODE

The following sample code demonstrates the session methods on an NAPTR record object.

#Preparation prior to a NAPTR record insertion

```
#PROGRAM STARTS: Include all the modules that will be used
use strict;
use Infoblox;
```

#Create a session to the Infoblox appliance

```
my $session = Infoblox::Session->new(
    master   => $host_ip,
    username => "admin",
    password => "infoblox"
);
unless ($session) {
   die("Construct session failed: ",
       $session->status_code() . ":" . $session->status_detail());
}
print "Session created successfully\n";

#Create the zone prior to an A record insertion
my $zone = Infoblox::DNS::Zone->new(name => "domain.com");
unless ($zone) {
   die("Construct zone failed: ",
       Infoblox::status_code() . ":" . Infoblox::status_detail());
}
print "Zone object created successfully\n";

#Verify if the zone exists
my $object = $session->get(object => "Infoblox::DNS::Zone", name => "domain.com");
unless ($object) {
   print "Zone does not exist on server, safe to add the zone\n";
   $session->add($zone)
      or die("Add zone failed: ",
             $session->status_code() . ":" . $session->status_detail());
}
print "Zone added successfully\n";
```

#Create a DNS NAPTR record

```
#Construct a DNS NAPTR object
my $bind_naptr = Infoblox::DNS::Record::NAPTR->new(
    name     => "bind_naptr.domain.com",
    comment  => " this is a demo bind_naptr record ",
    services => "http+E2U",
    flags    => "U",
    regexp   => "!http://(.*)!1!i",
    preference => 100,
    order    => 10,
    replacement => '_http._tcp.domain.com',
);
unless ($bind_naptr) {
   die("Construct DNS NAPTR record failed: ",
       Infoblox::status_code() . ":" . Infoblox::status_detail());
}
print "DNS NAPTR record object created successfully\n";

#Add the DNS NAPTR record object to the Infoblox appliance through a session
$session->add($bind_naptr)
    or die("Adding NAPTR record failed: ",
           $session->status_code() . ":" . $session->status_detail());
print "DNS NAPTR object added to server successfully\n";
```

#Search for a specific DNS NAPTR record

```
#Search all NAPTR records in the zone
my @retrieved_objs = $session->search(
    object => "Infoblox::DNS::Record::NAPTR",
    name   => '.*\.domain\.com'
);
my $object = $retrieved_objs[0];
unless ($object) {
    die("Search for NAPTR record failed: ",
        $session->status_code() . ":" . $session->status_detail());
}
print "Search DNS NAPTR record object found at least 1 matching entry\n";

#Search for all NAPTR records that start with "bind" in the zone
my @retrieved_objs = $session->search(
    object => "Infoblox::DNS::Record::NAPTR",
```

```
    name   => 'bind.*\.domain\.com'
);
my $object = $retrieved_objs[0];
unless ($object) {
    die("Search for NAPTR record failed: ",
        $session->status_code() . ":" . $session->status_detail());
}
print "Searching DNS NAPTR object using regexp found at least 1 matching entry\n";
```

#Get and modify a DNS NAPTR record

```
#Get a record through the session
my @retrieved_objs = $session->get(
    object => "Infoblox::DNS::Record::NAPTR",
    name   => "bind_naptr.domain.com"
);
my $object = $retrieved_objs[0];
unless ($object) {
    die("Get NAPTR record failed: ",
        $session->status_code() . ":" . $session->status_detail());
}
print "Get DNS NAPTR object found at least 1 matching entry\n";

#Modify some attributes of the specified NAPTR record
$object->preference(200);
$object->order(100);

#Apply the changes
$session->modify($object)
    or die("Modify NAPTR record failed: ",
           $session->status_code() . ":" . $session->status_detail());
print "DNS NAPTR object modified successfully \n";
```

#Remove a DNS NAPTR record

```
#Get NAPTR record through the session
my @retrieved_objs = $session->get(
    object => "Infoblox::DNS::Record::NAPTR",
    name   => "bind_naptr.domain.com"
);
my $object = $retrieved_objs[0];
unless ($object) {
    die("Get NAPTR record failed: ",
        $session->status_code() . ":" . $session->status_detail());
}
print "Get NAPTR DNS object found at least 1 matching entry\n";

#Submit the object for removal
$session->remove($object)
    or die("Remove NAPTR record failed: ",
        $session->status_code() . ":" . $session->status_detail());
print "DNS NAPTR object removed successfully \n";

#Remove the zone

#Get Zone object through the session
@retrieved_objs = $session->get(
    object => "Infoblox::DNS::Zone",
    name   => "domain.com"
);
$object = $retrieved_objs[0];
unless ($object) {
    die("Get zone failed: ",
        $session->status_code() . ":" . $session->status_detail());
}
print "Get zone object found at least 1 matching entry\n";

#Submit the object for removal
$session->remove($object)
    or die("Remove zone object failed: ",
        $session->status_code() . ":" . $session->status_detail());
print "Zone object removed successfully \n";

####PROGRAM ENDS####
```

AUTHOR

Infoblox Inc. http://www.infoblox.com/

SEE ALSO

Infoblox::Session, Infoblox::Session->add(), Infoblox::Session->get(),Infoblox::Session->modify(), Infoblox::Session->remove(), Infoblox::Session->search(), Infoblox::DNS::Zone

---

COPYRIGHT

Copyright (c) 2017 Infoblox Inc.

Infoblox::DNS::Record::NS - DNS NS record object

- NAME
- DESCRIPTION
- CONSTRUCTOR
- SESSION METHODS
  - Infoblox::Session->add( )
  - Infoblox::Session->get( )
  - Infoblox::Session->modify( )
  - Infoblox::Session->remove( )
  - Infoblox::Session->search( )
- METHODS
  - cloud_info( )
  - creator( )
  - dns_name( )
  - ms_delegation_name( )
  - last_queried( )
  - name( )
  - nameserver( )
  - addresses( )
  - views( )
- SAMPLE CODE
- AUTHOR
- SEE ALSO
- COPYRIGHT

NAME

Infoblox::DNS::Record::NS - DNS NS record object

DESCRIPTION

An NS record identifies an authoritative DNS server for a domain. Each authoritative DNS server must have an NS record. The appliance automatically creates an NS record when you assign a grid member as the primary server for a zone. You can manually create NS records for other zones.

Note that you must specify only one DNS view for the attribute "views".

CONSTRUCTOR

```
my $ns = Infoblox::DNS::Record::NS->new(
    name                => $name,                   #Required
    nameserver          => $nameserver,             #Required
    addresses           => [$ns1, $ns2, ...],       #Required
    ms_delegation_name  => $name,                   #Optional
    views               => [$View1]                 #Optional / Default is "default" view
);
```

SESSION METHODS

This section describes all the methods in Infoblox::Session module that you can apply to a DNS NS Record object.

Infoblox::Session->add( )

Use this method to add an object to the Infoblox appliance. See Infoblox::Session->add() for parameters and return values.

Example

```
#Construct an object
my $nsrecord = Infoblox::DNS::Record::NS->new(
    nameserver => "ns1.server.isp.org",
    name       => "domain.com",
    addresses  => [$ns1, $ns2],
);
# Submit for addition
my $response = $session->add( $nsrecord );
```

Infoblox::Session->get( )

Use this method to retrieve all the matching objects from the Infoblox appliance. See Infoblox::Session->get() for parameters and return values.

Key References

```
Apply the following attributes to get a specific DNS NS Record object:

 name        - Optional.  A domain name in FQDN (Fully Qualified Domain Name) format.
 nameserver  - Optional.  A domain name in FQDN (Fully Qualified Domain Name) format.
 view        - Optional.  The DNS view in which the NS record is located. By default, all DNS views
are searched.
```

Example

```
 my @retrieved_objs = $session->get(
    object    => "Infoblox::DNS::Record::NS",
    name      => "domain.com",
    view      => "default" );
```

Infoblox::Session->modify( )

Use this method to modify an object in the Infoblox appliance. See Infoblox::Session->modify() for parameters and return values.

Example

```
#Use method to modify the addresses.
$nsrecord->addresses([$ns3]);
# Submit modification
my $response = $session->modify( $nsrecord );
```

Infoblox::Session->remove( )

Use this method to remove an object from the Infoblox appliance. See Infoblox::Session->remove() for parameters and return values.

To remove a specific object, first use `get()` or `search()` to retrieve the specific object, and then submit this object for removal.

Example

```
# Get the object under the same name
my @retrieved_objs = $session->get(
    object    => "Infoblox::DNS::Record::NS",
    name      => "domain.com",
    view      => "default" );
# Find the desired object from the retrieved list.
my $desired_ns = $retrieved_objs[0];
# Submit for removal
my $response = $session->remove( $desired_ns );
```

Infoblox::Session->search( )

Use this method to search for DNS NS Record objects in the Infoblox appliance. See Infoblox::Session->search() for parameters and return values.

Key References

```
Apply the following attributes to search for a DNS NS Record object:

  name          - Optional. A domain name in FQDN (Fully Qualified Domain Name) format (regular
expression).
  nameserver    - Optional. A domain name in FQDN (Fully Qualified Domain Name) format (regular
expression).
  view          - Optional. The DNS view in which the NS record is located. By default, all DNS views
are searched.
```

Example

```
# search for all DNS NS objects that match "domain.com" in the default DNS view
    my @retrieved_objs = $session->search(
    object => "Infoblox::DNS::Record::NS",
    name   => 'domain\.com',
    view   => "default" );
```

METHODS

This section describes all the methods that you can use to configure and retrieve the attribute values of an NS record.

cloud_info( )

Use this method to retrieve cloud API related information for the Infoblox::DNS::Record::NS object.

Parameter

None

Returns

The method returns the attribute value.

Example

```
# Get cloud_info
my $cloud_info = $object->cloud_info();
```

creator( )

Use this method to retrieve the NS record creator. This is a read-only attribute.

Parameter

None

Returns

The method returns the attribute value.

Example

```
# Get attribute value
my $value = $object->creator();
```

dns_name( )

Use this method to retrieve the domain name, in punycode format, of the zone to be redirected. This is a read-only attribute.

Parameter

None

Returns

The method returns the attribute value.

Example

```
# Get attribute value
my $value = $nsrecord->dns_name();
```

ms_delegation_name( )

Use this method to set or retrieve the ms_delegation_name field of the NS record. This method can be used if the primary server of the zone is a Microsoft (r) server. If this NS record is for a subzone zone, it will be the subzone name.

Parameter

A subzone name.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Getting ms_delegation_name
my $ms_delegation_name = $nsrecord->ms_delegation_name();
# Modifying ms_delegation_name (the full FQDN of the zone is "sub.test.com")
$nsrecord->ms_delegation_name("sub");
```

last_queried( )

Use this method to retrieve the time when the associated record was last queried. This is a read-only attribute.

Parameter

None

Returns

The method returns the attribute value. The number of seconds that have elapsed since January 1st, 1970 UTC.

Example

```
#Get last_queried
my $last_queried = $nsrecord->last_queried();
```

name( )

Use this method to set or retrieve the domain name of the zone to be redirected.

The attribute value can be in unicode format.

Parameter

A domain name in FQDN format.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Getting name
my $name = $nsrecord->name();
# Modifying name
$nsrecord->name("domain.com");
```

nameserver( )

Use this method to set or retrieve the domain name of an authoritative server for the redirected zone.

Parameter

A domain name in FQDN format.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Getting nameserver
my $nameserver = $nsrecord->nameserver();
# Modifying nameserver
$nsrecord->nameserver("ns1.server.isp.org");
```

addresses( )

Use this method to set or retrieve the IP addresses of the name servers associated with the NS record.

Parameter

The valid value is an array reference that contains Infoblox::DNS::Nameserver::Address objects.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Getting addresses
my $nslist = $nsrecord->addresses();
# Modifying addresses
$nsrecord->addresses([$ns1, $ns2]);
```

views( )

Use this method to specify or retrieve the DNS view of the NS record.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

The default value is the 'default' view, which means the NS record is associated with a zone in the system-defined default view.

Parameter

Array reference of defined Infoblox::DNS::View objects.

Note that the array size must be 1.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get views
my $ref_views = $nsrecord->views();
#Modify views, list of Infoblox::DNS::View objects
$nsrecord->views([$view1]);
```

SAMPLE CODE

The following sample code demonstrates the different functions that can be applied to an object, such as add, search, modify, and remove. This sample also includes error handling for the operations.

#Preparation prior to a DNS NS record insertion

```
#PROGRAM STARTS: Include all the modules that will be used
use strict;
use Infoblox;

#Create a session to the Infoblox appliance
my $session = Infoblox::Session->new(
    master   => "192.168.1.2",
    username => "admin",
    password => "infoblox"
);
unless ($session) {
   die("Construct session failed: ",
       Infoblox::status_code() . ":" . Infoblox::status_detail());
}
print "Session created successfully\n";

#Create the zone prior to NS record insertion
my $zone = Infoblox::DNS::Zone->new(name => "domain.com");
unless ($zone) {
   die("Construct zone failed: ",
     Infoblox::status_code() . ":" . Infoblox::status_detail());
}
print "Zone object created successfully\n";

#Verify if the zone exists
my $object = $session->get(object => "Infoblox::DNS::Zone", name => "domain.com");
unless ($object) {
   print "Zone does not exist on server, safe to add the zone\n";
   $session->add($zone)
      or die("Add zone failed: ",
             $session->status_code() . ":" . $session->status_detail());
}
print "Zone added successfully\n";
```

#Create a DNS NS record

```
my $ns1 = Infoblox::DNS::Nameserver::Address->new(
                                                  address  => '10.0.0.1',
                                            auto_create_ptr => 'false'
                                            );

my $ns2 = Infoblox::DNS::Nameserver::Address->new(
                                            address         => '10.0.0.2',
                                            );

 #Construct a DNS NS object
  my $nsrecord = Infoblox::DNS::Record::NS->new(
```

```
    nameserver => "ns1.server.isp.org",
    name       => "domain.com",
    addresses  => [$ns1, $ns2],
  );

 unless ($nsrecord) {
   die("Construct DNS record NS failed: ",
       Infoblox::status_code() . ":" . Infoblox::status_detail());
 }
 print "DNS NS object created successfully\n"; #Add the DNS NS record object to Infoblox appliance through
a session
 $session->add($nsrecord)
    or die("Add record NS failed: ",
           $session->status_code() . ":" . $session->status_detail());
 print "DNS NS object added to server successfully\n";
```

#Search for a specific DNS NS record

```
 #Search all NS records that match "domain.com"
 my @retrieved_objs = $session->search(
    object     => "Infoblox::DNS::Record::NS",
    nameserver => "ns1.server.isp.org",
    name       => 'domain\.com'
 );
 my $object = $retrieved_objs[0];
 unless ($object) {
    die("Search record NS failed: ",
        $session->status_code() . ":" . $session->status_detail());
 }
 print "Search DNS NS object found at least 1 matching entry\n";
```

#Get and modify a DNS NS record

```
 #Get NS record through the session
 my  @retrieved_objs = $session->get(
    object     => "Infoblox::DNS::Record::NS",
    nameserver => "ns1.server.isp.org",
    name       => "domain.com",
 );

 my $object = $retrieved_objs[0];
 unless ($object) {
    die("Get record NS failed: ",
        $session->status_code() . ":" . $session->status_detail());
 }
 print "Get DNS NS object found at least 1 matching entry\n";

 #Modify one of the attributes of the specified NS record
 $object->addresses([$ns1]);

 #Apply the changes
 $session->modify($object)
    or die("Modify record NS failed: ",
           $session->status_code() . ":" . $session->status_detail());
 print "DNS NS object modified successfully \n";
```

#Remove a DNS NS record

```
  #Get NS record through the session
  my @retrieved_objs = $session->get(
     object     => "Infoblox::DNS::Record::NS",
     nameserver => "ns1.server.isp.org",
     name       => "domain.com"
   );
    my $object = $retrieved_objs[0];
    unless ($object) {
     die("Get record NS failed: ",
         $session->status_code() . ":" . $session->status_detail());
   }
    print "Get DNS NS object found at least 1 matching entry\n";

 #Submit the object for removal
 $session->remove($object)
    or die("Remove record NS failed: ",
        $session->status_code() . ":" . $session->status_detail());
 print "DNS NS object removed successfully \n";
```

```
####PROGRAM ENDS####
```

AUTHOR

Infoblox Inc. http://www.infoblox.com/

SEE ALSO

Infoblox::DNS::View, Infoblox::DNS::Zone, Infoblox::Session, Infoblox::Session->get(), Infoblox::Session->search(), Infoblox::Session->add(), Infoblox::Session->remove(), Infoblox::Session->modify()

COPYRIGHT

Copyright (c) 2017 Infoblox Inc.

Infoblox::DNS::Record::NSEC - NSEC Record object

- NAME
- DESCRIPTION
- CONSTRUCTOR
- SESSION METHODS
  - Infoblox::Session->get( )
  - Infoblox::Session->search( )
- METHODS
  - cloud_info( )
  - creator( )
  - dns_name( )
  - dns_next_domain_name( )
  - name( )
  - next_domain_name( )
  - ttl( )
  - types( )
  - view( )
  - zone( )
- SAMPLE CODE
- AUTHOR
- SEE ALSO
- COPYRIGHT

NAME

Infoblox::DNS::Record::NSEC - NSEC Record object

DESCRIPTION

The NSEC resource record is one of the resource records included in the DNS security extension mechanism (DNSSEC). This record is used to provide authenticated denial of existence of a resource record in responce to a resolver query.

The NSEC record is defined in RFC 4034.

The NSEC records are automatically generated upon the signing of an authoritative zone. The NSEC Record object is read-only.

CONSTRUCTOR

The Infoblox::DNS::Record::NSEC object is a read-only object and does not require manual construction.

SESSION METHODS

This section describes all the methods in an Infoblox::Session that can be applied to an NSEC Record object.

Infoblox::Session->get( )

Use this method to retrieve the existing objects from an Infoblox appliance. See Infoblox::Session->get() for parameters and return values.

Key Reference

```
 Apply the following attributes to get the NSEC Record object(s):

     name             - Optional. FQDN of the NSEC record (.e., FQDN of the owning RRset).
     next_domain_name - Optional. Next owner name (in the canonical ordering of the zone) that has
authoritative data
     ttl               - Optional. TTL value for the record
     types            - Optional. Identifies the RRset types that exists at the NSEC RR's owner name
     view             - Optional. The DNS view in which the record is located. By default, all DNS
views are searched. But if you omit this attribute and specify a zone, the appliance searches the
'default' view only.
     zone             - Optional. A zone name in FQDN format.
```

Example

```
   #get an NSEC record by name, DNS view name and record type
   my $nsec=$session->get(
                          object => "Infoblox::DNS::Record::NSEC",
                          name   => "a_record.test.com",
                          types  => "A",
                          view   => "default"
```

```
                              );
```

Infoblox::Session->search( )

Use this method to retrieve the existing objects from an Infoblox appliance. See Infoblox::Session->search() for parameters and return values.

Key Reference

```
 Apply the following attributes to search the NSEC Record object(s):

     name            - Optional. FQDN of the NSEC record. Regular expression.
     next_domain_name - Optional. Next owner name (in the canonical ordering of the zone) that has
authoritative data (regular expression).
     ttl              - Optional. TTL value for the record.
     types           - Optional. Identifies the RRset types that exist at the NSEC RR's owner name
(regular expression)
     view            - Optional. The DNS view in which the record is located. By default, all DNS
views are searched. But if you omit this attribute and specify a zone, the appliance searches the
'default' view only.
     zone            - Optional. A zone name in FQDN format.
```

Example

```
   #search for all NSEC records that match the regular expression for name and record type
   my @retrieved_objs=$session->search(
                          object => "Infoblox::DNS::Record::NSEC",
                          name   => "rec.*[.]test[.]com",
                          types  => "A|MX",
                        );

# search for all NSEC records in the "domain.com" zone of the default DNS view
# Note that the 'default' view is assumed implicitly here
my @retrieved_objs = $session->search(
    object => "Infoblox::DNS::Record::NSEC",
    zone   => "domain.com",
   );
```

METHODS

This section describes all the methods that can be used to retrieve the attribute values of an NSEC Record object.

cloud_info( )

Use this method to retrieve cloud API related information for the Infoblox::DNS::Record::NSEC object.

Parameter

None

Returns

The method returns the attribute value.

Example

```
# Get cloud_info
my $cloud_info = $object->cloud_info();
```

creator( )

Use this method to retrieve the NSEC record creator. This is a read-only attribute.

Parameter

None

Returns

The method returns the attribute value.

Example

```
# Get attribute value
my $value = $object->creator();
```

dns_name( )

Use this method to retrieve the FQDN, in punycode format, of the NSEC record. This is a read-only attribute.

Parameter

None

Returns

The method returns the attribute value.

Example

```
# Get attribute value
my $value = $nsec->dns_name();
```

dns_next_domain_name( )

Use this method to retrieve the next owner name in punycode format. This is a read-only attribute.

Parameter

None

Returns

The method returns the attribute value.

Example

```
# Get attribute value
my $value = $nsec->dns_next_domain_name();
```

name( )

Use this method to retrieve the FQDN of the NSEC record (which is the same as the name of the owning record).

The attribute value can be in unicode format.

Parameter

none

Returns

The method returns the FQDN of the NSEC record attribute value.

Example

```
#Get the FQDN of the NSEC record
my $name = $nsec->name();
```

next_domain_name( )

Use this method to retrieve the next owner name, in the canonical order of the zone, that has authoritative data.

The attribute value can be in unicode format.

Parameter

none

Returns

The method returns the next owner name (in the canonical ordering of the zone) that has authoritative data. The returned data is a string in FQDN format.

Example

```
#Get  next owner name (in the canonical ordering of the zone) that has authoritative data
my $next_domain_name = $nsec->next_domain_name();
```

ttl( )

Use this method to retrieve the Time to Live (TTL) value of an NSEC record object.

Parameter

none

Returns

The method returns the TTL attribute value. The returned parameter is a 32-bit integer (range from 0 to 4294967295) that represents the duration, in seconds, that the record is cached. Zero indicates that the record should not be cached.

Example

```
#Get TTL
my $ttl = $nsec->ttl();
```

types( )

Use this method to retrieve the types field of an NSEC record. This field identifies the RRSet types that exist at the original owner name of the NSEC RR.

Parameter

none

Returns

The method returns the RRSet types that exist at the owner name of the NSEC RR attribute value. The returned value is a string containing RR type mnemonics separated by a space; for example, "NS SOA RRSIG DNSKEY".

Example

```
#Get the types field value
my $types = $nsec->types();
```

view( )

Use this method to retrieve the DNS view object that contains the NSEC Record object.

Parameter

none

Returns

The method returns the Infoblox::DNS::View object that contains the NSEC record.

Example

```
#Get the view
my $view = $nsec->view();
```

zone( )

Use this method to retrieve the zone name of an NSEC record.

Parameter

none

Returns

Returns the zone name in FQDN format.

Example

```
# Get zone
my $zone = $nsec->zone();
```

SAMPLE CODE

The following sample code demonstrates the session methods on an NSEC record object.

```
#PROGRAM STARTS: Include all the modules that will be used
use strict;
use Infoblox;

#Create a session to the Infoblox appliance
       my $session = Infoblox::Session->new(
           master   => "192.168.1.2",
           username => "admin",
           password => "infoblox"
       );
```

```
unless ($session) {
   die("Construct session failed: ",
       $session->status_code() . ":" . $session->status_detail());
}
print "Session created successfully\n";

#Enable DNSSEC in the default view

my $default_view=$session->get(
                                object=> "Infoblox::DNS::View",
                                name => "default"
                               );
unless($default_view) {
    die("Getting the default view failed:",
       Infoblox::status_code() . ":" . Infoblox::status_detail());
}
print "Got the default view successfully\n";

$default_view->dnssec_enabled("true")
    or die("Changing the dnssec_enabled in the default view failed:",
            Infoblox::status_code() . ":" . Infoblox::status_detail());
$session->modify($default_view)
    or die("Changing the dnssec_enabled in the default view failed:",
            Infoblox::status_code() . ":" . Infoblox::status_detail());
```

#Create a signed zone and populate it with an A record to demonstrate NSEC record

```
print "Creating Member primary server for the zone\n";
my $primary=Infoblox::DNS::Member->new(
                                       ipv4addr => "192.168.1.2",
                                       name => "infoblox.localdomain",
                                      );
unless($primary) {
   die("Unable to create primary server object: ",
       Infoblox::status_code() . ":" . Infoblox::status_detail());
}

my $zone = Infoblox::DNS::Zone->new(
                                    name => "domain.com",
                                    primary => $primary
                                   );
unless ($zone) {
   die("Construct zone failed: ",
       Infoblox::status_code() . ":" . Infoblox::status_detail());
}
print "Zone object created successfully\n";

#Verify if the zone exists
my $object = $session->get(object => "Infoblox::DNS::Zone", name => "domain.com");
unless ($object) {
   print "Zone does not exist on server, safe to add the zone\n";
   $session->add($zone)
      or die("Add zone failed: ",
             $session->status_code() . ":" . $session->status_detail());
}
print "Zone added successfully\n";

#Adding A record to the zone to demonstrate the corresponding NSEC record
my $a_record=Infoblox::DNS::Record::A->new(
                                           ipv4addr => "10.9.8.7",
                                           name => "recorda.domain.com"
                                          );
unless($a_record) {
   die("Creating A record failed : ",
      Infoblox::status_code() . ":" . Infoblox::status_detail());
}

$object = $session->get( object => "Infoblox::DNS::Record::A", name => "recorda.domain.com" );
unless ($object) {
   print "A record does not exist on the server, safe to add the A record\n";
   $session->add($a_record)
     or die("Adding A record failed: ",
            Infoblox::status_code() . ":" . Infoblox::status_detail());
}
print "A record added to the zone successfully\n";
```

```
        #Retrieving zone back from the server in order to sign it
        $zone = $session->get(object => "Infoblox::DNS::Zone", name => "domain.com");
        unless($zone) {
           die("Retrieving zone back failed: ",
                Infoblox::status_code( ). ":". Infoblox::status_detail( ));
        }
        print "Zone retrieved for signing successfully.\n";

        $zone->dnssec_ksk_algorithm("RSASHA1") &&
        $zone->dnssec_zsk_algorithm("RSASHA1") &&
        $zone->dnssec_ksk_size(640) &&
        $zone->dnssec_zsk_size(640)
          or die("Changing the zone DNSSEC setting failed: ",
                 Infoblox::status_code() . ":" . Infoblox::status_detail());
        $session->modify($zone)
          or die("Modifying dnssec values in zone failed: ",
                 Infoblox::status_code() . ":" . Infoblox::status_detail());
        print "Zone modified successfully\n";

        #Signing the zone
        $zone->dnssec_signed("true")
           or die("Signing of the zone failed: ",
                  Infoblox::status_code() . ":" . Infoblox::status_detail());
              print "Zone signed successfully\n";
```

#Getting the NSEC record corresponding to the A record

```
       my $nsec_record_a=$session->get(
                                  object => "Infoblox::DNS::Record::NSEC",
                                  name   => "recorda.domain.com",
                                  view   => "default"
                                 );
            unless($nsec_record_a) {
         die("Getting NSEC corresponding to the A record failed: ",
            Infoblox::status_code() . ":" . Infoblox::status_detail());
            }
            print "Got NSEC record successfully, next_domain_name value: ".$nsec_record_a-
>next_domain_name()."\n";
```

#Searching for NSEC objects using regular expressions

```
      my @retrieved_objs=$session->search(
                                       object => "Infoblox::DNS::Record::NSEC",
                                       name   => ".*domain[.]com",
                                       types  => "A|MX|SOA",
                                       view   => "default"
                                      );
          unless(@retrieved_objs>0) {
         die("Searching for NSEC objects failed: ",
            Infoblox::status_code() . ":" . Infoblox::status_detail());
          }
          print "Search for the NSEC objects successful, ".scalar(@retrieved_objs)." objects found\n";
```

#Removing the created zone and cleaning up the view

```
      $session->remove($zone)
        or die("Unable to remove the zone: ",
               Infoblox::status_code() . ":" . Infoblox::status_detail());
      print "Zone removed successfully\n";

      $default_view->dnssec_enabled("false")
      && $default_view->override_dnssec("false")
      && $session->modify($default_view)
       or die("Restoring dnssec_enabled value in the default view failed: ",
              Infoblox::status_code() . ":" . Infoblox::status_detail());

 ####PROGRAM ENDS####
```

AUTHOR

Infoblox Inc. http://www.infoblox.com/

SEE ALSO

Infoblox::Session, Infoblox::Session->get(), Infoblox::Session->search(), Infoblox::DNS::Record::NSEC3, Infoblox::DNS::Record::NSEC3PARAM, Infoblox::DNS::View, Infoblox::DNS::Zone

---

COPYRIGHT

Copyright (c) 2017 Infoblox Inc.

Infoblox::DNS::Record::NSEC3 - NSEC3 Record object.

- NAME
- DESCRIPTION
- CONSTRUCTOR
- SESSION METHODS
  - Infoblox::Session->get( )
  - Infoblox::Session->search( )
- METHODS
  - cloud_info( )
  - creator( )
  - flags( )
  - hash_algorithm( )
  - iterations( )
  - name( )
  - next_hash_owner( )
  - salt( )
  - ttl( )
  - types( )
  - view( )
  - zone( )
- SAMPLE CODE
- AUTHOR
- SEE ALSO
- COPYRIGHT

NAME

Infoblox::DNS::Record::NSEC3 - NSEC3 Record object.

DESCRIPTION

The NSEC3 resource record is a part of DNS security extension (DNSSEC) authentication mechanism. It is used for authenticated denial of existence of DNS Resource record sets in the DNSSEC signed authoritative zones. The functionality that NSEC3 records provide is similar to the functionality of NSEC records, but NSEC3 includes cryptographic protection against zone enumeration.

The NSEC3 resource record is desribed in RFC 5155.

All the NSEC3 records are automatically generated during signing of the corresponding zone.

CONSTRUCTOR

The Infoblox::DNS::Record::NSEC3 object is a read-only object and does not require manual construction.

SESSION METHODS

This section describes all the methods in an Infoblox::Session that can be applied to an NSEC3 Record object.

Infoblox::Session->get( )

Use this method to retrieve the existing objects from an Infoblox appliance. See Infoblox::Session->get() for parameters and return values.

Key Reference

```
 Apply the following attributes to get the NSEC3 Record object(s):

     flags           - Optional. Integer containing 8 one-bit flags that can be used to indicate
different processing.
     hash_algorithm  - Optional. Integer representing the cryptographic hash algorithm used to
construct the hash-value.
     iterations      - Optional. Number of additional times the hash function has been performed.
     name            - Optional. FQDN of the NSEC3 record (contains zone name and the hashed record
name).
     next_hash_owner - Optional. Next hashed owner name in hash order.
     salt            - Optional. Salt value.
     ttl             - Optional. TTL value.
     types           - Optional. Identifies the RRSet types that exist at the original owner name of
the NSEC3 RR.
     view            - Optional. The DNS view in which the record is located. By default, all DNS
views are searched. But if you omit this attribute and specify a zone, the appliance searches the
'default' view only.
     zone            - Optional. A zone name in FQDN format.
```

Example

```
# get all the NSEC3 records corresponding to 'A' records in the default DNS view.
my @retrieved_objs = $session->get(
                     object => "Infoblox::DNS::Record::NSEC3",
                     types  => "A",
                     view   => "default" );
```

Infoblox::Session->search( )

Use this method to retrieve the existing objects from an Infoblox appliance. See Infoblox::Session->search() for parameters and return values.

Key Reference

```
Apply the following attributes to search the NSEC3 Record object(s):

     flags           - Optional. Integer containing 8 one-bit flags that can be used to indicate
different processing.
     hash_algorithm  - Optional. Integer representing the cryptographic hash algorithm used to
construct the hash-value.
     iterations      - Optional. Number of additional times the hash function has been performed.
     name            - Optional. FQDN of the NSEC3 record (contains zone name and the hashed record
name). Regular expression
     next_hash_owner - Optional. Next hashed owner name in hash order (regular expression).
     salt            - Optional. Salt value (regular expression).
     ttl             - Optional. TTL value.
     types           - Optional. Identifies the RRSet types that exist at the original owner name of
the NSEC3 RR (regular expression).
     view            - Optional. The DNS view in which the record is located. By default, all DNS
views are searched. But if you omit this attribute and specify a zone, the appliance searches the
'default' view only.
     zone            - Optional. A zone name in FQDN format.
```

Example

```
# get all the NSEC3 records in the zone test.com.
my @retrieved_objs = $session->get(
                     object => "Infoblox::DNS::Record::NSEC3",
                     name   => ".*test[.]com",
                     );

# search for all NSEC3 records in the "domain.com" zone of the default DNS view
# Note that the 'default' view is assumed implicitly here
my @retrieved_objs = $session->search(
    object => "Infoblox::DNS::Record::NSEC3",
    zone   => "domain.com",
   );
```

METHODS

This section describes all the methods that can be used to retrieve the attribute values of an NSEC3 Record object.

cloud_info( )

Use this method to retrieve cloud API related information for the Infoblox::DNS::Record::NSEC3 object.

Parameter

None

Returns

The method returns the attribute value.

Example

```
# Get cloud_info
my $cloud_info = $object->cloud_info();
```

creator( )

Use this method to retrieve the NSEC3 record creator. This is a read-only attribute.

Parameter

None

Returns

The method returns the attribute value.

Example

```
# Get attribute value
my $value = $object->creator();
```

flags( )

Use this method to retrieve the flags field of an NSEC3 Record object.

Parameter

none

Returns

The method returns the integer that contains 8 one-bit flags that can be used to indicate different processing. Currently only the least significant bit of the flags field is meaningful. It represents the opt-out flag, which is used to determine the processing of non-secure delegations.

Example

```
#Get the flags field.
my $flags = $nsec3->flags();
```

hash_algorithm( )

Use this method to retrieve the cryptographic hash algorithm used to construct the hash value of an NSEC3 Record object.

Parameter

none

Returns

The method returns an integer that corresponds to the cryptographic hash algorithm used to construct the hash value.

Example

```
#Get hash algorithm
my $hash_algorithm = $nsec3->hash_algorithm();
```

iterations( )

Use this method to retrieve the number of iterations of the hash function performed to obtain the hashed name.

Parameter

none

Returns

The method returns the number of additional times the hash function has been performed to compute the hash value. This is an integer in the range 0 through 65535, inclusive.

Example

```
#Get the iterations parameter
my $iterations = $nsec3->iterations();
```

name( )

Use this method to retrieve the name of NSEC3 record in FQDN format. The name of NSEC3 record consists of the hashed owner name prepended to the name of the zone, containing NSEC3 record.

Parameter

none

Returns

The method returns the name of NSEC3 record.

Example

```
#Get the NSEC3 record name
my $name = $nsec3->name();
```

next_hash_owner( )

Use this method to retrieve the next hashed owner name. This is a hash of an owner name that immediately follows the owner name of the given NSEC3 record in the hash order.

Parameter

none

Returns

The method returns the next hashed owner name. It is a string in base-32 encoding.

Example

```
#Get the next hashed owner name
my $next_hash_owner = $nsec3->next_hash_owner();
```

salt( )

Use this method to retrieve the salt value of a NSEC3 record. The salt is prepended to the original owner name before hashing and on each of the hash iterations in order to defend against pre-calculated dictionary attacks.

Parameter

none

Returns

The method returns a salt value. The returned value is a string containing a sequence of hexadecimal digits representing the salt or '-' if the salt is not used.

Example

```
#Get the salt
my $salt = $nsec3->salt();
```

ttl( )

Use this method to retrieve the Time to Live (TTL) value of a NSEC3 Record object.

Parameter

none

Returns

The method returns the TTLattribute value. The returned parameter is a 32-bit integer (range from 0 to 4294967295) that represents the duration in seconds that the record is cached. Zero indicates that the record should not be cached.

Example

```
#Get TTL
my $ttl = $nsec3->ttl();
```

types( )

Use this method to retrieve the types field of an NSEC3 record. Types field identifies the RRSet types that exist at the original owner name of the NSEC3 RR.

Parameter

none

Returns

The method returns the RRSet types that exist at the original owner name of the NSEC3 RR attribute value. The returned value is a string containing space separated RR types mnemonics, e.g. "NS SOA RRSIG DNSKEY NSEC3PARAM".

Example

```
#Get the types field value
my $types = $nsec3->types();
```

view( )

Use this method to retrieve the DNS view object that contains the NSEC3 Record object.

Parameter

none

Returns

The method returns the Infoblox::DNS::View object that contains the NSEC3 record.

Example

```
#Get the DNS view
my $view = $nsec3->view();
```

zone( )

Use this method to retrieve the zone name of a NSEC3 record.

Parameter

none

Returns

Returns the zone name of the zone that contains the NSEC3 record. The zone name is in FQDN format.

Example

```
# Get zone
my $zone = $nsec3->zone();
```

SAMPLE CODE

The following sample code demonstrates the session methods on a NSEC3 Record object.

```
#PROGRAM STARTS: Include all the modules that will be used
use strict;
use Infoblox;

#Create a session to the Infoblox device
       my $session = Infoblox::Session->new(
           master   => "192.168.1.2",
           username => "admin",
           password => "infoblox"
       );
       unless ($session) {
          die("Construct session failed: ",
              $session->status_code() . ":" . $session->status_detail());
       }
       print "Session created successfully\n";

       #Enable DNSSEC in the default view

       my $default_view=$session->get(
                                      object=> "Infoblox::DNS::View",
                                      name => "default"
                                     );
       unless($default_view) {
           die("Getting the default view failed:",
              Infoblox::status_code() . ":" . Infoblox::status_detail());
       }
       print "Got the default view successfully\n";

       $default_view->dnssec_enabled("true")
           or die("Changing the dnssec_enabled in the default view failed:",
                    Infoblox::status_code() . ":" . Infoblox::status_detail());
       $session->modify($default_view)
           or die("Changing the dnssec_enabled in the default view failed:",
                    Infoblox::status_code() . ":" . Infoblox::status_detail());
```

#Create a signed zone and populate it with an A record

```
       print "Creating Member primary server for the zone\n";
       my $primary=Infoblox::DNS::Member->new(
                                              ipv4addr => "192.168.1.2",
                                              name => "infoblox.localdomain",
```

```
                                        );
    unless($primary) {
       die("Unable to create primary server object: ",
           Infoblox::status_code() . ":" . Infoblox::status_detail());
    }

    my $zone = Infoblox::DNS::Zone->new(
                                        name => "domain.com",
                                        primary => $primary
                                      );
    unless ($zone) {
       die("Construct zone failed: ",
           Infoblox::status_code() . ":" . Infoblox::status_detail());
    }
    print "Zone object created successfully\n";

    #Verify if the zone exists
    my $object = $session->get(object => "Infoblox::DNS::Zone", name => "domain.com");
    unless ($object) {
       print "Zone does not exist on server, safe to add the zone\n";
       $session->add($zone)
          or die("Add zone failed: ",
                 $session->status_code() . ":" . $session->status_detail());
    }
    print "Zone added successfully\n";

    #Adding A record to the zone to demonstrate the corresponding NSEC3 record
    my $a_record=Infoblox::DNS::Record::A->new(
                                               ipv4addr => "10.9.8.7",
                                               name => "recorda.domain.com"
                                              );
    unless($a_record) {
       die("Creating A record failed : ",
          Infoblox::status_code() . ":" . Infoblox::status_detail());
    }

    $object = $session->get( object => "Infoblox::DNS::Record::A", name => "recorda.domain.com" );
    unless ($object) {
       print "A record does not exist on the server, safe to add the A record\n";
       $session->add($a_record)
         or die("Adding A record failed: ",
                Infoblox::status_code() . ":" . Infoblox::status_detail());
    }
    print "A record added to the zone successfully\n";

    #Retrieving zone back from the server in order to sign it
    $zone = $session->get(object => "Infoblox::DNS::Zone", name => "domain.com");
    unless($zone) {
       die("Retrieving zone back failed: ",
            Infoblox::status_code( ). ":". Infoblox::status_detail( ));
    }
    print "Zone retrieved for signing successfully.\n";

    $zone->dnssec_ksk_algorithm("NSEC3RSASHA1") &&
    $zone->dnssec_zsk_algorithm("NSEC3RSASHA1") &&
    $zone->dnssec_ksk_size(640) &&
    $zone->dnssec_zsk_size(640)
      or die("Changing the zone DNSSEC setting failed: ",
             Infoblox::status_code() . ":" . Infoblox::status_detail());
    $session->modify($zone)
      or die("Modifying dnssec values in zone failed: ",
             Infoblox::status_code() . ":" . Infoblox::status_detail());
    print "Zone modified successfully\n";

    #Signing the zone
    $zone->dnssec_signed("true")
       or die("Signing of the zone failed: ",
              Infoblox::status_code() . ":" . Infoblox::status_detail());
           print "Zone signed successfully\n";
```

#Getting the NSEC3 record corresponding to the A record

```
    my $nsec3_record_a=$session->get(
                                object => "Infoblox::DNS::Record::NSEC3",
                                types     => "A RRSIG",
                                view   => "default"
```

```
                                    );
        unless($nsec3_record_a) {
    die("Getting NSEC3 corresponding to the A record failed: ",
        Infoblox::status_code() . ":" . Infoblox::status_detail());
        }
        print "Got NSEC3 record successfully, name value: ".$nsec3_record_a->name()."\n";
```

#Searching for NSEC3 objects using regular expressions

```
    my @retrieved_objs=$session->search(
                                        object => "Infoblox::DNS::Record::NSEC3",
                                        name   => ".*domain[.]com",
                                        types  => "A|MX|SOA",
                                        view   => "default"
                                      );
      unless(@retrieved_objs>0) {
    die("Searching for NSEC3 objects failed: ",
        Infoblox::status_code() . ":" . Infoblox::status_detail());
      }
      print "Search for the NSEC3 objects successful, ".scalar(@retrieved_objs)." objects found\n";
```

#Removing the created zone and cleaning up the view

```
    $session->remove($zone)
      or die("Unable to remove the zone: ",
             Infoblox::status_code() . ":" . Infoblox::status_detail());
    print "Zone removed successfully\n";

    $default_view->dnssec_enabled("false")
    && $default_view->override_dnssec("false")
    && $session->modify($default_view)
     or die("Restoring dnssec_enabled value in the default view failed: ",
            Infoblox::status_code() . ":" . Infoblox::status_detail());
```

####PROGRAM ENDS####

AUTHOR

Infoblox Inc. http://www.infoblox.com/

SEE ALSO

Infoblox::Session, Infoblox::Session->get(), Infoblox::Session->search(), Infoblox::DNS::Record::NSEC, Infoblox::DNS::Record::NSEC3PARAM, Infoblox::DNS::View, Infoblox::DNS::Zone

---

COPYRIGHT

Copyright (c) 2017 Infoblox Inc.

Infoblox::DNS::NSEC3PARAM - NSEC3PARAM Record object

- NAME
- DESCRIPTION
- CONSTRUCTOR
- SESSION METHODS
  - Infoblox::Session->get( )
  - Infoblox::Session->search( )
- METHODS
  - cloud_info( )
  - creator( )
  - dns_name( )
  - flags( )
  - hash_algorithm( )
  - iterations( )
  - name( )
  - salt( )
  - ttl( )
  - view( )
  - zone( )
- SAMPLE CODE
- AUTHOR
- SEE ALSO
- COPYRIGHT

NAME

Infoblox::DNS::NSEC3PARAM - NSEC3PARAM Record object

DESCRIPTION

The NSEC3PARAM resource record is one of the resource records of the DNS security extension (DNSSEC) authentication mechanism. The NSEC3PARAM RR contains the NSEC3 parameters (hash algorithm, flags, iterations, and salt) needed by authoritative servers to calculate hashed owner names for NSEC3 records. The NSEC3PARAM record is present at the apex of the DNSSEC-signed authoritative zone.

The NSEC3PARAM resource record is desribed in RFC 5155.

The NSEC3PARAM record is generated automatically upon the signing of the corresponding zone. It is a read-only object.

CONSTRUCTOR

The Infoblox::DNS::NSEC3PARAM object is a read-only object and does not require manual construction.

SESSION METHODS

This section describes all the methods in an Infoblox::Session that can be applied to an NSEC3PARAM Record object.

Infoblox::Session->get( )

Use this method to retrieve the existing objects from an Infoblox appliance. See Infoblox::Session->get() for parameters and return values.

Key Reference

```
 Apply the following attributes to get the NSEC3PARAM Record object(s):

     flags           - Optional. Integer containing 8 one-bit flags that can be used to indicate
different processing.
     hash_algorithm  - Optional. Integer representing the cryptographic hash algorithm.
     iterations      - Optional. Number of additional times the hash function has been performed.
     name            - Optional. FQDN of the NSEC3PARAM record owner (zone name).
     salt            - Optional. Salt value.
     ttl             - Optional. TTL of the record.
     view            - Optional. The DNS view in which the record is located. By default, all DNS
views are searched. But if you omit this attribute and specify a zone, the appliance searches the
'default' view only.
     zone            - Optional. A zone name in FQDN format.
```

Example

```
#get the NSEC3PARAM record for the zone 'test.com'
my $nsec3param=$session->get(
                          object => "Infoblox::DNS::Record::NSEC3PARAM",
```

```
                                name   => "test.com"
                              );

#get all the NSEC3PARAM records for the default DNS view
my @retrieved_objs=$session->get(
                                    object => "Infoblox::DNS::Record::NSEC3PARAM",
                                    view   => "default"
                                  );
```

Infoblox::Session->search( )

Use this method to retrieve the existing objects from an Infoblox appliance. See Infoblox::Session->search() for parameters and return values.

Key Reference

```
Apply the following attributes to search the NSEC3PARAM Record object(s):

     flags           - Optional. Integer containing 8 one-bit flags that can be used to indicate
different processing.
     hash_algorithm  - Optional. Integer representing the cryptographic hash algorithm.
     iterations      - Optional. Number of additional times the hash function has been performed.
     name            - Optional. FQDN of the NSEC3PARAM record owner (zone name). Regular expression.
     salt            - Optional. Salt value (regular expression).
     ttl             - Optional. TTL of the record.
     view            - Optional. The DNS view in which the record is located. By default, all DNS
views are searched. But if you omit this attribute and specify a zone, the appliance searches the
'default' view only.
     zone            - Optional. A zone name in FQDN format.
```

Example

```
# search for all NSEC3PARAM records in the "domain.com" zone of the default DNS view
# Note that the 'default' DNS view is assumed implicitly here
my @retrieved_objs = $session->search(
    object => "Infoblox::DNS::Record::NSEC3PARAM",
    zone   => "domain.com",
   );
```

METHODS

This section describes all the methods that can be used to retrieve the attribute values of an NSEC3PARAM Record object.

cloud_info( )

Use this method to retrieve cloud API related information for the Infoblox::DNS::NSEC3PARAM object.

Parameter

None

Returns

The method returns the attribute value.

Example

```
# Get cloud_info
my $cloud_info = $object->cloud_info();
```

creator( )

Use this method to retrieve the NSEC3PARAM record creator. This is a read-only attribute.

Parameter

None

Returns

The method returns the attribute value.

Example

```
# Get attribute value
my $value = $object->creator();
```

dns_name( )

Use this method to retrieve the owner name, in punycode format, of the NSEC3PARAM record. This is a read-only attribute.

Parameter

None

Returns

The method returns the attribute value.

Example

```
# Get attribute value
my $value = $nsec3param->dns_name();
```

flags( )

Use this method to retrieve the flags field of an NSEC3PARAM Record object.

Parameter

none

Returns

The method returns the integer that contains 8 one-bit flags that can be used to indicate different processing. Currently only the least significant bit of the flags field is meaningful. It represents the opt-out flag, which is used to determine the processing of non-secure delegations.

Example

```
#Get the flags field.
my $flags = $nsec3param->flags();
```

hash_algorithm( )

Use this method to retrieve the cryptographic hash algorithm used to construct the hash value of the corresponding NSEC3 Record objects.

Parameter

none

Returns

The method returns an integer that corresponds to the cryptographic hash algorithm that is used to construct the hash value.

Example

```
#Get hash algorithm
my $hash_algorithm = $nsec3param->hash_algorithm();
```

iterations( )

Use this method to retrieve the number of iterations of the hash function performed to obtain the hashed name.

Parameter

none

Returns

The method returns the number of additional times the hash function has been performed to compute the hash value. This is an integer in the range 0 through 65535, inclusive.

Example

```
#Get the iterations parameter
my $iterations = $nsec3param->iterations();
```

name( )

Use this method to retrieve the owner name of the NSEC3PARAM record in FQDN format. The name value of the NSEC3PARAM record is the same as the name of the zone that contains this record.

The attribute value can be in unicode format.

Parameter

none

Returns

The method returns the name of the NSEC3PARAM record. This name is a zone name in FQDN format.

Example

```
#Get the NSEC3PARAM name field
my $name = $nsec3param->name();
```

salt( )

Use this method to retrieve the salt value of an NSEC3PARAM record. The salt is used to calculate the hashed name of NSEC3 resource records to defend against pre-calculated dictionary attacks.

Parameter

none

Returns

The method returns a salt value. The returned value is a string containing a sequence of hexadecimal digits representing the salt or '-' if the salt is not used.

Example

```
#Get the salt
my $salt = $nsec3param->salt();
```

ttl( )

Use this method to retrieve the Time to Live (TTL) value of an NSEC3PARAM Record object.

Parameter

none

Returns

The method returns the TTL attribute value. The returned parameter is a 32-bit integer (range from 0 to 4294967295) that represents the duration, in seconds, that the record is cached. Zero indicates that the record should not be cached.

Example

```
#Get TTL
my $ttl = $nsec3param->ttl();
```

view( )

Use this method to retrieve the DNS view object that contains the NSEC3PARAM Record object.

Parameter

none

Returns

The method returns the Infoblox::DNS::View object that contains the NSEC3PARAM record.

Example

```
#Get the view
my $view = $nsec3param->view();
```

zone( )

Use this method to retrieve the zone name of an NSEC3PARAM record.

Parameter

none

Returns

Returns the name of the zone that contains the given NSEC3PARAM record.

Example

```
# Get zone
my $zone = $nsec3param->zone();
```

SAMPLE CODE

The following sample code demonstrates the session methods on an NSEC3PARAM Record object.

```
#PROGRAM STARTS: Include all the modules that will be used
use strict;
use Infoblox;

#Create a session to the Infoblox appliance
       my $session = Infoblox::Session->new(
           master   => $host_ip,
           username => "admin",
           password => "infoblox"
       );
       unless ($session) {
          die("Construct session failed: ",
              $session->status_code() . ":" . $session->status_detail());
       }
       print "Session created successfully\n";

       #Enable DNSSEC in the default view

       my $default_view=$session->get(
                                       object=> "Infoblox::DNS::View",
                                       name => "default"
                                      );
       unless($default_view) {
           die("Getting the default view failed:",
              Infoblox::status_code() . ":" . Infoblox::status_detail());
       }
       print "Got the default view successfully\n";

       $default_view->dnssec_enabled("true")
           or die("Changing the dnssec_enabled in the default view failed:",
                    Infoblox::status_code() . ":" . Infoblox::status_detail());
       $session->modify($default_view)
           or die("Changing the dnssec_enabled in the default view failed:",
                    Infoblox::status_code() . ":" . Infoblox::status_detail());
```

#Create a DNSSEC signed zone

```
       print "Creating Member primary server for the zone\n";
       my $primary=Infoblox::DNS::Member->new(
                                           ipv4addr => $node_ip,
                                           name => $host_name
                                         );
       unless($primary) {
          die("Unable to create primary server object: ",
              Infoblox::status_code() . ":" . Infoblox::status_detail());
       }

       my $zone = Infoblox::DNS::Zone->new(
                                           name => "domain.com",
                                           primary => $primary
                                          );
       unless ($zone) {
          die("Construct zone failed: ",
              Infoblox::status_code() . ":" . Infoblox::status_detail());
       }
       print "Zone object created successfully\n";

       #Verify if the zone exists
       my $object = $session->get(object => "Infoblox::DNS::Zone", name => "domain.com");
       unless ($object) {
          print "Zone does not exist on server, safe to add the zone\n";
          $session->add($zone)
             or die("Add zone failed: ",
                    $session->status_code() . ":" . $session->status_detail());
       }
       print "Zone added successfully\n";
```

```
        #Retrieving zone back from the server in order to sign it
        $zone = $session->get(object => "Infoblox::DNS::Zone", name => "domain.com");
        unless($zone) {
           die("Retrieving zone back failed: ",
                Infoblox::status_code( ). ":". Infoblox::status_detail( ));
        }
        print "Zone retrieved for signing successfully.\n";

        $zone->dnssec_ksk_algorithm("NSEC3RSASHA1") &&
        $zone->dnssec_zsk_algorithm("NSEC3RSASHA1") &&
        $zone->dnssec_ksk_size(640) &&
        $zone->dnssec_zsk_size(640)
          or die("Changing the zone DNSSEC setting failed: ",
                 Infoblox::status_code() . ":" . Infoblox::status_detail());
        $session->modify($zone)
          or die("Modifying dnssec values in zone failed: ",
                 Infoblox::status_code() . ":" . Infoblox::status_detail());
        print "Zone modified successfully\n";

        #Signing the zone
        $zone->dnssec_signed("true")
           or die("Signing of the zone failed: ",
                  Infoblox::status_code() . ":" . Infoblox::status_detail());
               print "Zone signed successfully\n";
```

#Getting the NSEC3PARAM record by zone name

```
       my $nsec3param=$session->get(
                                   object => "Infoblox::DNS::Record::NSEC3PARAM",
                                   name   => "domain.com",
                                   view   => "default"
                                  );
            unless($nsec3param) {
         die("Getting NSEC3PARAM record failed: ",
            Infoblox::status_code() . ":" . Infoblox::status_detail());
            }
            print "Got NSEC3PARAM record successfully.\n";
```

#Searching for NSEC3PARAM objects using regular expressions

```
      my @retrieved_objs=$session->search(
                                          object => "Infoblox::DNS::Record::NSEC3PARAM",
                                          name   => ".*com",
                                          salt   => ".*",
                                          iterations => 100,
                                          view   => "default"
                                         );
           unless(@retrieved_objs>0) {
         die("Searching for NSEC3PARAM objects failed: ",
            Infoblox::status_code() . ":" . Infoblox::status_detail());
           }
           print "Search for the NSEC3PARAM objects successful, ".scalar(@retrieved_objs)." objects
found\n";
```

#Removing the created zone and cleaning up the view

```
      $session->remove($zone)
        or die("Unable to remove the zone: ",
               Infoblox::status_code() . ":" . Infoblox::status_detail());
      print "Zone removed successfully\n";

      $default_view->dnssec_enabled("false")
      && $default_view->override_dnssec("false")
      && $session->modify($default_view)
       or die("Restoring dnssec_enabled value in the default view failed: ",
              Infoblox::status_code() . ":" . Infoblox::status_detail());

 ####PROGRAM ENDS####
```

AUTHOR

Infoblox Inc. http://www.infoblox.com/

SEE ALSO

Infoblox::Session, Infoblox::Session->get(), Infoblox::Session->search(), Infoblox::DNS::Record::NSEC3, Infoblox::DNS::View, Infoblox::DNS::Zone

---

COPYRIGHT

Copyright (c) 2017 Infoblox Inc.

Infoblox::DNS::Record::PTR - DNS PTR record object.

- NAME
- DESCRIPTION
- CONSTRUCTOR
- SESSION METHODS
  - Infoblox::Session->add( )
  - Infoblox::Session->get( )
  - Infoblox::Session->modify( )
  - Infoblox::Session->remove( )
  - Infoblox::Session->search( )
- METHODS
  - comment( )
  - cloud_info( )
  - creator( )
  - creation_time( )
  - ddns_principal( )
  - ddns_protected( )
  - disable( )
  - discovered_data( )
  - discovered_duid( )
  - discovered_name( )
  - discoverer( )
  - dns_name( )
  - dns_ptrdname( )
  - first_discovered( )
  - extattrs( )
  - extensible_attributes( )
  - forbid_reclamation( )
  - ipv4addr( )
  - ipv6addr( )
  - last_discovered( )
  - last_queried( )
  - mac_address( )
  - ms_ad_user_data( )
  - name( )
  - netbios( )
  - network_component_description( )
  - network_component_ip( )
  - network_component_name( )
  - network_component_port_description( )
  - network_component_port_name( )
  - network_component_port_number( )
  - network_component_type( )
  - os( )
  - port_duplex( )
  - port_link_status( )
  - port_speed( )
  - port_status( )
  - port_vlan_description( )
  - port_vlan_name( )
  - port_vlan_number( )
  - ptrdname( )
  - reclaimable( )
  - ttl( )
  - views( )
  - v_cluster( )
  - v_datacenter( )
  - v_host( )
  - v_name( )
  - v_netadapter( )
  - v_switch( )
  - v_type( )
  - zone( )
- SAMPLE CODE
- AUTHOR
- SEE ALSO
- COPYRIGHT

NAME

Infoblox::DNS::Record::PTR - DNS PTR record object.

DESCRIPTION

In a forward-mapping zone, a PTR (pointer) record maps a domain name to another domain name. In a reverse-mapping zone, a PTR (pointer) record maps an address to a domain name. To define a specific address-to-name mapping, add a PTR record to a previously defined authoritative reverse-mapping zone.

To create a PTR record, you need to specify a domain name and host name.

Note that you must specify only one view for the attribute "views".

CONSTRUCTOR

```
 #Constructor for an IPv4 PTR object
 my $bind_ptr = Infoblox::DNS::Record::PTR->new (
        ipv4addr              => $ipv4addr,                                                  #
Required
        ptrdname              => $fqdn,                                                      #
Required
        comment               => $string,                                                    #
Optional / Default is empty
        disable               => "true" | "false"                                            #
Optional / Default "false"
        extattrs              => { $string => $extattr, ... },                               #
Optional / Default is undefined
        extensible_attributes => { $string => $string|$num, $string => [ $string|$num, ...], ... }, #
Optional / Default is undefined
        last_discovered       => $string,                                                    #
Optional / Default is empty
        name                  => $fqdn,                                                      #
Optional
        ttl                   => $num | undef,                                               #
Optional / Default is undefined
        views                 => [$View1],                                                   #
Optional / Default is "default" view
        creator               => "DYNAMIC" | "STATIC",                                       #
Optional / Default is "STATIC"
        ddns_protected        => "true" | "false",                                           #
Optional / Default is "false"
        ddns_principal        => $string,                                                    #
Optional / Default is undefined
        forbid_reclamation    => "true" | "false",                                           #
Optional / Default is "false"
     );

 #Constructor for an IPv6 PTR object
 my $bind_ptr = Infoblox::DNS::Record::PTR->new (
        ipv6addr              => $ipv6addr,                                                  #
Required
        ptrdname              => $fqdn,                                                      #
Required
        comment               => $string,                                                    #
Optional / Default is empty
        disable               => "true" | "false"                                            #
Optional / Default "false"
        extattrs              => { $string => $extattr, ... },                               #
Optional / Default is undefined
        extensible_attributes => { $string => $string|$num, $string => [ $string|$num, ...], ... }, #
Optional / Default is undefined
        name                  => $fqdn,                                                      #
Optional
        ttl                   => $nun | undef,                                               #
Optional / Default is undefined
        views                 => [$View1],                                                   #
Optional / Default is "default" view
        creator               => "DYNAMIC" | "STATIC",                                       #
Optional / Default is "STATIC"
        ddns_protected        => "true" | "false",                                           #
Optional / Default is "false"
        ddns_principal        => $string,                                                    #
Optional / Default is undefined
        forbid_reclamation    => "true" | "false",                                           #
Optional / Default is "false"
     );

 #Constructor for an PTR object in Forward Mapping Zone
 #Both ipv4addr and ipv6addr must be undefined
 my $bind_ptr = Infoblox::DNS::Record::PTR->new (
        name                  => $fqdn,                                                      #
Required
                                 $fqdn                                                       #
```

```
        ptrdname              =>      ,
Required
        comment               => $string,                                          #
Optional / Default is empty
        disable               => "true" | "false"                                  #
Optional / Default "false"
        extattrs              => { $string => $extattr, ... },                     #
Optional / Default is undefined
        extensible_attributes => { $string => $string|$num, $string => [ $string|$num, ...], ... }, #
Optional / Default is undefined
        ttl                   => $nun | undef,                                     #
Optional / Default is undefined
        views                 => [$View1],                                         #
Optional / Default is "default" view
        creator               => "DYNAMIC" | "STATIC",                             #
Optional / Default is "STATIC"
        ddns_protected        => "true" | "false",                                 #
Optional / Default is "false"
        ddns_principal        => $string,                                          #
Optional / Default is undefined
        forbid_reclamation    => "true" | "false",                                 #
Optional / Default is "false"
    );
```

You cannot set both extattrs and extensible_attributes attributes at the same time.

SESSION METHODS

This section describes all the methods in Infoblox::Session that you can apply to a DNS PTR record object.

Infoblox::Session->add( )

Use this method to add an object to the Infoblox appliance. See Infoblox::Session->add() for parameters and return values.

Example

```
#Constructor for an IPv4 PTR object
my $bind_ptr = Infoblox::DNS::Record::PTR->new (
    ipv4addr => "10.0.0.1",
    ptrdname => "bind_ptr.domain.com",
    comment => "this is a demo bind_ptr record",
    disable => "true",
    ttl => "33",
    views => [$view1]  );
 # Submit for addition
 my $response = $session->add( $bind_ptr );

#Constructor for an IPv6 PTR object
my $bind_ptr = Infoblox::DNS::Record::PTR->new(
    ptrdname     => "ip6host.this.com",
    comment  => " this is a demo ipv6 bind_ptr record ",
    disable => "true",
    ttl => "333",
    ipv6addr => "1234:5678:9abc:1000::1");
 # Submit for addition
 my $response = $session->add( $bind_ptr );
```

Infoblox::Session->get( )

Use this method to retrieve existing objects from the Infoblox appliance. See Infoblox::Session->get() for parameters and return values.

Key Reference

```
 Apply the following attributes to get a specific DNS PTR object:

 name | ipv4addr | ipv6addr | ptrdname - Optional. name/ptrdname is in FQDN format. ipv4addr is an
IPv4 address with a 32-bit number in dotted decimal notation. ipv6addr is the IPv6 (128 bits) address
of the DNS PTR record. ptrdname is the domain name of the DNS PTR record.
 view                                  - Optional. The DNS view in which the PTR record is located.
By default, all DNS views are searched. But if you omit this attribute and specify a zone, the
appliance searches the 'default' view only.
 zone                                  - Optional. A zone name in FQDN format.
 creator                               - Optional. Record creator.
 ddns_principal                        - Optional. GSS-TSIG principal that owns the record.
 reclaimable                           - Optional. The flag that indicates whether the record is
reclaimable or not.
 extattrs                              - Optional. A hash reference containing extensible attributes.
 extensible_attributes                 - Optional. A hash reference containing extensible attributes.
 discovered_duid                       - Optional. Discovered DHCPv6 UID.
 discovered_name                       - Optional. Discovered name.
 discoverer                            - Optional. Discoverer name.
```

```
        first_discovered                        - Optional. Time of the first discovery.
        last_discovered                         - Optional. Time of the last discovery.
        netbios                                 - Optional. Discovered NetBIOS name.
        network_component_description           - Optional. Discovered network component description.
        network_component_ip                    - Optional. Discovered network component IP address.
        network_component_name                  - Optional. Discovered network component name.
        network_component_port_description      - Optional. Discovered network component port description.
        network_component_port_name             - Optional. Discovered network component port name.
        network_component_port_number           - Optional. Discovered network component port number.
        network_component_type                  - Optional. Discovered network component type.
        os                                      - Optional. Discovered OS.
        port_duplex                             - Optional. Discovered port duplex settings.
        port_link_status                        - Optional. Discovered port link status.
        port_speed                              - Optional. Discovered port speed settings.
        port_status                             - Optional. Discovered port status settings.
        port_vlan_description                   - Optional. Discovered port VLAN description.
        port_vlan_name                          - Optional. Discovered port VLAN name.
        port_vlan_number                        - Optional. Discovered port VLAN number.
        v_cluster                               - Optional. Discovered VMware cluster name.
        v_datacenter                            - Optional. Discovered VMware datacenter name.
        v_host                                  - Optional. Discovered VMware server name.
        v_name                                  - Optional. Discovered VMware host or virtual machine name.
        v_netadapter                            - Optional. Discovered VMware physical network adapter name.
        v_switch                                - Optional. Discovered VMware virtual switch name.
        v_type                                  - Optional. Discovered VMware entity type (possible values:
      "VirtualMachine", "HostSystem", "VirtualCenter").
```

Example

```
#To get an IPv4 PTR object by name
my @retrieved_objs = $session->get(
    object => "Infoblox::DNS::Record::PTR",
    name   => "1.0.0.10.in-addr.arpa",
    view   => "default" );

#To get an IPv4 PTR object by ipv4addr
my @retrieved_objs = $session->get(
    object   => "Infoblox::DNS::Record::PTR",
    ipv4addr => "10.0.0.1",
    view     => "default" );

#To get an IPv4 PTR object by ptrdname
my @retrieved_objs = $session->get(
    object   => "Infoblox::DNS::Record::PTR",
    ptrdname => "bind_ptr.domain.com",
    view     => "default" );

#To get an IPv6 PTR object by name
my @retrieved_objs = $session->get(
    object => "Infoblox::DNS::Record::PTR",
    name   => "1.0.0.0.0.0.0.0.0.0.0.0.0.0.0.0.0.0.0.0.0.0.0.0.0.0.1.c.b.a.9.8.7.6.5.4.3.2.1.ip6.arpa",
    view   => "default" );

#To get an IPv6 PTR object by ipv6addr
my @retrieved_objs = $session->get(
    object   => "Infoblox::DNS::Record::PTR",
    ipv6addr => "1234:5678:9abc:1000::1",
    view     => "default" );

#To get an IPv6 PTR object by ptrdname
my @retrieved_objs = $session->get(
    object   => "Infoblox::DNS::Record::PTR",
    ptrdname => "ip6host.this.com",
    view     => "default" );

# get all PTR object in zone "10.10.10.in-addr.arpa" of default view
my @retrieved_objs = $session->get(
    object => "Infoblox::DNS::Record::PTR",
    zone   => "10.10.10.in-addr.arpa",
    view   => "default"
    );

# get all PTR objects with a given extensible attribute
my @retrieved_objs = $session->get(
    object => "Infoblox::DNS::Record::PTR",
    extensible_attributes => { 'Site' => 'Santa Clara' });

# get all PTR objects with a given v_type:
my @retrieved_objs = $session->get(
  object    => "Infoblox::DNS::Record::PTR",
  v_type    => "VirtualMachine");
```

Infoblox::Session->modify( )

Use this method to modify objects in the Infoblox appliance. See Infoblox::Session->modify() for parameters and return values.

Example

```
# Use method to modify the comment for IPv4 or IPv6 object.
$bind_ptr->comment("this is a modified comment for bind PTR");
# Submit modification
my $response = $session->modify( $bind_ptr );
```

Infoblox::Session->remove( )

Use this function to submit to remove an object from the Infoblox appliance. See Infoblox::Session->remove() for parameters and return values.

To remove a specific object, first use `get()` or `search()` to retrieve the specific object, and then submit this object for removal.

Example

#Remove an object with a unique name

```
# Get the object with the same name for IPv4 object
my @retrieved_objs = $session->get(
    object => "Infoblox::DNS::Record::PTR",
    name   => "1.0.0.10.in-addr.arpa",
    view   => "default" );
# find the desired object from retrieved list.
my $desired_ptr = $retrieved_objs[0];
# Submit for removal
my $response = $session->remove( $desired_ptr );

# Get the object with the same name for an IPv6 object
my @retrieved_objs = $session->get(
    object => "Infoblox::DNS::Record::PTR",
    name   => "1.0.0.0.0.0.0.0.0.0.0.0.0.0.0.0.0.0.0.0.0.0.0.0.0.0.1.c.b.a.9.8.7.6.5.4.3.2.1.ip6.arpa",
    view   => "default" );
# find the desired object from retrieved list.
my $desired_ptr = $retrieved_objs[0];
# Submit for removal
my $response = $session->remove( $desired_ptr );
```

Infoblox::Session->search( )

Use this method to search for existing DNS PTR record object in the Infoblox appliance. See Infoblox::Session->search() for parameters and return values.

Key Reference

```
 Apply the following attributes to search for a DNS PTR record object:

 name | ipv4addr | ipv6addr | ptrdname - Optional. The IPv4 or IPv6 address of the PTR record, or the
domain name in FQDN (Fully Qualified Domain Name) format (regular expression).
 view                                - Optional. The DNS view in which the PTR record is located.
By default, all DNS views are searched. But if you omit this attribute and specify a zone, the
appliance searches the 'default' view only.
 comment                             - Optional. A comment in string format (regular expression).
 creator                             - Optional. Record creator.
 ddns_principal                      - Optional. GSS-TSIG principal that owns the record.
 reclaimable                         - Optional. The flag that indicates whether the record is
reclaimable or not.
 extattrs                            - Optional. A hash reference containing extensible attributes.
 extensible_attributes               - Optional. A hash reference containing extensible attributes.
 discovered_duid                     - Optional. Discovered DHCPv6 UID (exact search only).
 discovered_name                     - Optional. Discovered name(regex).
 discoverer                          - Optional. Discoverer name(regex).
 first_discovered                    - Optional. Time of the first discovery (in the ISO format,
exact search only).
 last_discovered                     - Optional. Time of the last discovery (in the ISO format,
exact search only).
 netbios                             - Optional. Discovered NetBIOS name(regex).
 network_component_description       - Optional. Discovered network component description(regex).
 network_component_ip                - Optional. Discovered network component IP address(regex).
 network_component_name              - Optional. Discovered network component name(regex).
 network_component_port_description  - Optional. Discovered network component port
description(regex).
 network_component_port_name         - Optional. Discovered network component port name(regex).
 network_component_port_number       - Optional. Discovered network component port number (exact
search only).
 network_component_type              - Optional. Discovered network component type(regex).
 os                                  - Optional. Discovered OS (regex).
 port_duplex                         - Optional. Discovered port duplex settings (exact search
only).
 port_link_status                    - Optional. Discovered port link status (exact search only).
 port_speed                          - Optional. Discovered port speed settings (exact search
only).
 port_status                         - Optional. Discovered port status settings (exact search
```

```
only).
 port_vlan_description                  - Optional. Discovered port VLAN description(regex).
 port_vlan_name                         - Optional. Discovered port VLAN name(regex).
 port_vlan_number                       - Optional. Discovered port VLAN number(exact search only).
 v_cluster                              - Optional. Discovered VMware cluster name (regular
expression).
 v_datacenter                           - Optional. Discovered VMware datacenter name (regular
expression).
 v_host                                 - Optional. Discovered VMware server name (regular
expression).
 v_name                                 - Optional. Discovered VMware host or virtual machine name
(regular expression).
 v_netadapter                           - Optional. Discovered VMware physical network adapter name
(regular expression).
 v_switch                               - Optional. Discovered VMware virtual switch name (regular
expression).
 v_type                                 - Optional. Discovered VMware entity type (possible values:
"VirtualMachine", "HostSystem", "VirtualCenter").
```

For more information about searching extensible attributes, see Infoblox::Grid::ExtensibleAttributeDef/Searching Extensible Attributes.

Example

```
 # search for all DNS IPv4 PTR objects in the default DNS view that match 10.0.0.0/8 and that contain
"bind" in the comment field
 my @retrieved_objs = $session->search(
     object  => "Infoblox::DNS::Record::PTR",
     name    => "10.in-addr.arpa",
     comment => "bind*",
     view    => "default" );

 my @retrieved_objs = $session->search(
     object   => "Infoblox::DNS::Record::PTR",
     ipv4addr => '10\..*\..*\..*',
     comment  => "bind*",
     view     => "default" );

 my @retrieved_objs = $session->search(
     object   => "Infoblox::DNS::Record::PTR",
     ptrdname => "bind_ptr.domain.com",
     comment  => "bind*",
     view     => "default" );

 # search for all DNS IPv6 PTR objects under 1234:5678:9abc::/48 which have bind in comment field in
default view
 my @retrieved_objs = $session->search(
     object  => "Infoblox::DNS::Record::PTR",
     name    => "1.0.0.0.0.0.0.0.0.0.0.0.0.0.0.0.0.0.0.0.0.0.0.0.1.c.b.a.9.8.7.6.5.4.3.2.1.ip6.arpa",
     comment => "bind*",
     view    => "default" );

 my @retrieved_objs = $session->search(
     object   => "Infoblox::DNS::Record::PTR",
     ipv6addr => "1234:5678:9abc:1000::1",
     comment  => "bind*",
     view     => "default" );

 my @retrieved_objs = $session->search(
     object   => "Infoblox::DNS::Record::PTR",
     ptrdname => "ip6host.this.com",
     comment  => "bind*",
     view     => "default" );

 # search for all PTR objects in the "10.10.10.in-addr.arpa" zone of the default view
 my @retrieved_objs = $session->search(
     object => "Infoblox::DNS::Record::PTR",
     zone   => "10.10.10.in-addr.arpa",
     view   => "default"
     );

 # search all PTR records with the extensible attribute 'Site'
 my @retrieved_objs = $session->search(
    object => "Infoblox::DNS::Record::MX",
    extensible_attributes => { 'Site' => 'Santa Clara' });

 # search all PTR records by v_datacenter:
 my @retrieved_objs = $session->search(
    object       => "Infoblox::DNS::Record::PTR",
    v_datacenter => "test.*");
```

METHODS

This section describes all the methods that you can use to configure and retrieve the attribute value of a PTR record.

comment( )

Use this method to set or retrieve the comment.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

Desired comment in string format with a maximum of 256 bytes.

Returns

If you specified a parameter,the method returns true when the modification succeeds,and returns false when the operation fails.

Example

```
#Get comment
my $comment = $bindptr->comment();
#Modify comment
$bindptr->comment("Modifying the DNS PTR comment");
```

cloud_info( )

Use this method to retrieve cloud API related information for the Infoblox::DNS::Record::PTR object.

Parameter

None

Returns

The method returns the attribute value.

Example

```
# Get cloud_info
my $cloud_info = $object->cloud_info();
```

creator( )

Use this method to set or retrieve the record creator.

Note that changing creator from or to 'SYSTEM' value is not allowed.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Paramter

The valid values are 'STATIC' and 'DYNAMIC'. The default value is 'STATIC'.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get creator value
my $creator = $object->creator();

#Modify creator value
$object->creator("DYNAMIC");
```

creation_time( )

Use this method to retrieve the creation time for the record. This is a read-only attribute.

Parameter

None

Returns

The valid return value is a number of seconds that have elapsed since January 1st, 1970 UTC.

Example

```
#Get creation_time value
my $creation_time = $object->creation_time();
```

ddns_principal( )

Use this method to set or retrive the GSS-TSIG principal that owns this record.

Note that you cannot set ddns_principal for 'STATIC' and 'SYSTEM' records.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Paramter

The GSS-TSIG principal FQDN (Fully Qualified Domain Name) format. The FQDN consists of the hostname followed by the domain name (example: abc.com). A hostname can have a maximum of 256 characters.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get ddns_principal value
my $ddns_principal = $object->ddns_principal();

#Modify ddns_principal value
$object->ddns_principal('foo.com');
```

ddns_protected( )

Use this method to set or retrieve the flag that indicates whether DDNS updates for this record are allowed or not.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

Specify 'true' to protect record from DDNS updates and 'false' to allow DDNS updates for the specified record.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get ddns_protected value
my $ddns_protected = $object->ddns_protected();

#Modify ddns_protected value
$object->ddns_protected('true');
```

disable( )

Use this method to set or retrieve the disable flag of a DNS record.

Include the specific parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

The default value for this field is false, therefore the DNS record is enabled.

Parameter

Specify "true" to set the disable flag or "false" to deactivate/unset it.

Returns

If you specified a parameter,the method returns true when the modification succeeds,and returns false when the operation fails.

Example

```
#Get disable
my $disable = $bindptr->disable();
#Modify disable
```

```
$bindptr->disable("true");
```

discovered_data( )

Use this method to retrieve the discovered data of a PTR Record object. This is a read-only attribute.

Omit the parameter to retrieve the attribute value.

Parameter

None

Returns

The valid return value is an Infoblox::Grid::Discovery::Data object.

Example

```
#Get discovered_data value
my $discovered_data = $object->discovered_data();
```

discovered_duid( )

Use this method to retrieve the discovered DHCPv6 UID (DUID) of a PTR record object.

Parameter

none

Returns

The method returns the discovered name.

Example

```
#Get discovered_duid
my $discovered_duid = $bindptr->discovered_duid();
```

discovered_name( )

Use this method to retrieve the discovered name of a PTR record object.

Parameter

none

Returns

The method returns the discovered name.

Example

```
#Get discovered_name
my $discovered_name = $bindptr->discovered_name();
```

discoverer( )

Use this method to retrieve the discoverer of a PTR record object.

Parameter

none

Returns

The method returns the discoverer value.

Example

```
#Get discoverer
my $discoverer = $bindptr->discoverer();
```

dns_name( )

Use this method to retrieve the FQDN, in punycode format, of the PTR record. This is a read-only attribute.

Parameter

None

Returns

The method returns the attribute value.

Example

```
# Get attribute value
my $value = $bindptr->dns_name();
```

dns_ptrdname( )

Use this method to retrieve the PTR domain name in punycode format. This is a read-only attribute.

Parameter

None

Returns

The method returns the attribute value.

Example

```
# Get attribute value
my $value = $bindptr->ptrdname();
```

first_discovered( )

Use this method to retrieve the date and time that the PTR record object was first discovered.

Parameter

none

Returns

The method returns the date and time that the PTR record object was first discovered.

Example

```
#Get first_discovered
my $first_discovered = $bindptr->first_discovered();
```

extattrs( )

Use this method to set or retrieve the extensible attributes associated with a PTR record object.

Parameter

Valid value is a hash reference containing the names of extensible attributes and their associated values (Infoblox::Grid::Extattr objects).

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get extattrs
my $ref_extattrs = $bindptr->extattrs();
#Modify extattrs
$bindptr->extattrs({ 'Site' => $extattr1, 'Administrator' => $extattr2 });
```

extensible_attributes( )

Use this method to set or retrieve the extensible attributes associated with a PTR record.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

For valid values for extensible attributes, see Infoblox::Grid::ExtensibleAttributeDef/Extensible Attribute Values.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get extensible attributes
my $ref_extensible_attributes = $bindptr->extensible_attributes();
#Modify extensible attributes
$bindptr->extensible_attributes({ 'Site' => 'Santa Clara', 'Administrator' => [ 'Peter', 'Tom' ] });
```

forbid_reclamation( )

Use this method to set or retrieve the flag that indicates whether the reclamation is allowed for the record or not.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

Specify 'true' to forbid reclamation for the record and 'false' to allow it. The default value is 'false'.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get forbid_reclamation
my $forbid_reclamation = $object->forbid_reclamation();
#Modify forbid_reclamation
$object->forbid_reclamation('true');
```

ipv4addr( )

Use this method to set or retrieve the IPv4 address.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

An IPv4 address is a 32-bit number in dotted decimal notation. It consists of four 8-bit groups of decimal digits separated by decimal points (example: 192.168.1.2).

Returns

If you specified a parameter,the method returns true when the modification succeeds,and returns false when the operation fails.

Example

```
#Get ipv4addr
my $ipv4addr = $bindptr->ipv4addr();
#Modify ipv4addr
$bind_ptr->ipv4addr("10.0.0.2");
```

ipv6addr( )

Use this method to set or retrieve the IPv6 address.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

The IPv6 (128 bits) of the DNS PTR record (example: aaaa:bbbb:cccc:dddd:eeee:ffff:0000:1111).

Returns

If you specified a parameter,the method returns true when the modification succeeds,and returns false when the operation fails.

Example

```
#Get ipv6addr
my $ipv6addr = $bindptr->ipv6addr();
#Modify ipv6addr
$bind_ptr->ipv6addr("1234:5678:9abc:1000::2");
```

last_discovered( )

Use this method to retrieve the time this object was last seen by a discovery job.

The default value for this field is empty.

Parameter

N/A

Returns

The method returns the attribute value.

Example

```
#Get last_discovered
my $last_discovered = $bind_ptr->last_discovered();
```

last_queried( )

Use this method to retrieve the time when the associated record was last queried. This is a read-only attribute.

Parameter

None

Returns

The method returns the attribute value. The number of seconds that have elapsed since January 1st, 1970 UTC.

Example

```
#Get last_queried
my $last_queried = $bind_ptr->last_queried();
```

mac_address( )

Use this method to retrieve the discovered MAC address of this object.

The default value for this field is empty.

Parameter

N/A

Returns

The method returns the attribute value.

Example

```
#Get mac_address
my $mac_address = $bind_ptr->mac_address();
```

ms_ad_user_data( )

Use this method to retrieve Microsoft Active Directory users related information. This is a read-only attribute.

Parameter

None

Returns

The valid return value is an Infoblox::Grid::MSServer::AdUser::Data object.

Example

```
#Get ms_ad_user_data
my $ms_ad_user_data = $bind_ptr->ms_ad_user_data();
```

name( )

Use this method to set or retrieve the FQDN of the PTR record.

Include the specified parameter to set the attribute value if both ipv4addr and ipv6addr are undefined: PTR in Forward Mapping Zone. Otherwise use ipv4addr or ipv6addr methods.

Omit the parameter to retrieve the attribute value.

The attribute value can be in unicode format.

Parameter

FQDN of the PTR record

Returns

The method returns the attribute value.

Example

```
#Get name
my $name = $bindptr->name();
#Modify name
$bindptr->name('bindptr.domain.com');
```

netbios( )

Parameter

None

Use this method to retrieve the name in the NetBIOS reply that responded to a NetBIOS query.

Returns

The method returns the attribute value.

Example

```
#Get netbios
my $netbios = $bind_ptr->netbios();
```

network_component_description( )

Use this method to retrieve the description of the network device that is connected to the PTR record object.

Parameter

none

Returns

The method returns the description of the network device.

Example

```
#Get network_component_description
my $network_component_description = $bindptr->network_component_description();
```

network_component_ip( )

Use this method to retrieve the IP address of the network device that is connected to the PTR record object.

Parameter

none

Returns

The method returns the network device IP address.

Example

```
#Get network_component_ip
my $network_component_ip = $bindptr->network_component_ip();
```

network_component_name( )

Use this method to retrieve the name of the network device that is connected to the PTR record object.

Parameter

none

Returns

The method returns the network device name.

Example

```
#Get network_component_name
my $network_component_name = $bindptr->network_component_name();
```

network_component_port_description( )

Use this method to retrieve the description of the network device port that is connected to the PTR record object.

Parameter

none

Returns

The method returns the description of the network device port.

Example

```
#Get network_component_port_description
my $network_component_port_description = $bindptr->network_component_port_description();
```

network_component_port_name( )

Use this method to retrieve the name of the network device port that is connected to the PTR record object.

Parameter

none

Returns

The method returns the network device port name.

Example

```
#Get network_component_port_name
my $network_component_port_name = $bindptr->network_component_port_name();
```

network_component_port_number( )

Use this method to retrieve the number of the network device port that is connected to the PTR record object.

Parameter

none

Returns

The method returns the network device port number value.

Example

```
#Get network_component_port_number
my $network_component_port_number = $bindptr->network_component_port_number();
```

network_component_type( )

Use this method to retrieve the type of the network device that is connected to the PTR record object.

Parameter

none

Returns

The method returns the network device type.

Example

```
#Get network_component_type
my $network_component_type = $bindptr->network_component_type();
```

os( )

Use this method to retrieve the operating system associated with this PTR record.

Returns

The method returns the attribute value.

Example

```
#Get os
my $os = $bind_ptr->os();
#Modify os
$bind_ptr->os("os-name");
```

port_duplex( )

Use this method to retrieve the duplex setting of the network device port that is connected to the PTR record object.

Parameter

none

Returns

The method returns the network device port duplex setting. Returned values, if any, are one of the following:

```
Full
Half
```

Example

```
#Get port_duplex
my $port_duplex = $bindptr->port_duplex();
```

port_link_status( )

Use this method to retrieve the link status of the network device port that is connected to the PTR record object.

Parameter

none

Returns

The method returns the network device port link status. Returned values, if any, are one of the following:

```
Connected
Not Connected
Unknown
```

Example

```
#Get port_link_status
my $port_link_status = $bindptr->port_link_status();
```

port_speed( )

Use this method to retrieve the speed of the network device port that is connected to the PTR record object.

Parameter

none

Returns

The method returns the network device port speed value. Returned values, if any, are one of the following:

```
10M
100M
1G
10G
100G
Unknown
```

Example

```
#Get port_speed
my $port_speed = $bindptr->port_speed();
```

port_status( )

Use this method to retrieve the status of the network device port that is connected to the PTR record object.

Parameter

none

Returns

The method returns the network device port status. Returned values, if any, are one of the following:

```
Up
Down
Unknown
```

Example

```
#Get port_status
my $port_status = $bindptr->port_status();
```

port_vlan_description( )

Use this method to retrieve the VLAN description of the network device port that is connected to the PTR record object.

Parameter

none

Returns

The method returns the VLAN description of the network device port that is connected to the PTR record object.

Example

```
#Get port_vlan_description
my $port_vlan_description = $bindptr->port_vlan_description();
```

port_vlan_name( )

Use this method to retrieve the VLAN name of the network device port that is connected to the PTR record object.

Parameter

none

Returns

The method returns the VLAN name of the network device port.

Example

```
#Get port_vlan_name
my $port_vlan_name = $bindptr->port_vlan_name();
```

port_vlan_number( )

Use this method to retrieve the VLAN number of the network device port that is connected to the PTR record object.

Parameter

none

Returns

The method returns the port VLAN number.

Example

```
#Get port_vlan_number
my $port_vlan_number = $bindptr->port_vlan_number();
```

ptrdname( )

Use this method to set or retrieve the PTR domain name.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

The attribute value can be in unicode format.

Parameter

Text with the domain name for the DNS PTR record.

Returns

If you specified a parameter,the method returns true when the modification succeeds,and returns false when the operation fails.

Example

```
#Get ptrdname
my $ptrdname = $bindptr->ptrdname();
#Modify ptrdname
$bindptr->ptrdname("new domain name");
```

reclaimable( )

Use this method to retrieve the flag that indicates whether the record is reclaimable or not.

Parameter

None

Returns

The method returns the attribute value.

Example

```
#Get reclaimable
my $reclaimable = $object->reclaimable();
```

ttl( )

Use this method to configure or retrieve the Time to Live (TTL) value.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

The default value is undefined; therefore the record inherits the TTL value from the zone level.

With a specified TTL value, the method overrides the zone values with the specified TTL value.

Parameter

A 32-bit integer (range from 0 to 4294967295) that represents the duration ,in seconds, that the record is cached. Zero indicates that the record should not be cached.

Returns

If you specified a parameter,the method returns true when the modification succeeds,and returns false when the operation fails.

Example

```
#Get ttl
my $ttl = $bindptr->ttl();
#Modify ttl
$bindptr->ttl(1800);
#Un-override ttl
$bindptr->ttl(undef);
```

views( )

Use this method to specify or retrieve the view in which the PTR record is located.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

The default value is the default view therefore the PTR record is referenced under the default view.

Parameter

Array reference of defined Infoblox::DNS::View objects.

Note that the array size must be 1.

Returns

If you specified a parameter,the method returns true when the modification succeeds,and returns false when the operation fails.

Example

```
#Get views
```

```
my $ref_views = $bindptr->views();
#Modify views, list of Infoblox::DNS::View objects
$bindptr->views([$view1]);
```

v_cluster( )

Use this method to retrieve the name of the VMware cluster associated with the PTR Record object. This method is read-only.

Parameter

none

Returns

This method returns a string that contains the VMware cluster name. The default value is an empty string.

Example

```
#Get v_cluster
my $v_cluster = $bindptr->v_cluster();
```

v_datacenter( )

Use this method to retrieve the name of the VMware datacenter associated with the PTR Record object. This method is read-only.

Parameter

none

Returns

This method returns a string that contains the VMware datacenter name. The default value is an empty string.

Example

```
#Get v_datacenter
my $v_datacenter = $bindptr->v_datacenter();
```

v_host( )

Use this method to retrieve the name of the VMware host associated with the PTR Record object. This method is read-only.

Parameter

none

Returns

This method returns a string that contains the VMware host name. The default value is an empty string.

Example

```
#Get v_host
my $v_host = $bindptr->v_host();
```

v_name( )

Use this method to retrieve the name of the VMware entity associated with the PTR Record object. This method is read-only.

Parameter

none

Returns

This method returns a string that contains the VMware entity name. The default value is an empty string.

Example

```
#Get v_name
my $v_name = $bindptr->v_name();
```

v_netadapter( )

Use this method to retrieve the name of the physical network adapter through which the VMware entity is connected to the appliance. This method is read-only.

Parameter

none

Returns

This method returns a string that contains the network adapter name. The default value is an empty string.

Example

```
#Get v_netadapter
my $v_netadapter = $bindptr->v_netadapter();
```

v_switch( )

Use this method to retrieve the name of the virtual switch through which the VMware entity is connected to the appliance. This method is read-only.

Parameter

none

Returns

This method returns a string that contains the virtual switch name. The default value is an empty string.

Example

```
#Get v_switch
my $v_switch = $bindptr->v_switch();
```

v_type( )

Use this method to retrieve the type of VMware entity associated with the PTR Record object. This method is read-only.

Parameter

none

Returns

This method returns a string that contains the VMware entity name. The default value is an empty string. The returend value, if any, can be one of the following:

```
VirtualMachine
HostSystem
VirtualCenter
```

Example

```
#Get v_type
my $v_type = $bindptr->v_type();
```

zone( )

Use this method to retrieve the zone name of a DNS PTR record. This method is read-only and cannot be set.

Parameter

None

Returns

Returns the attribute value.

Example

```
# Get zone
my $zone = $bindptr->zone();
```

SAMPLE CODE

The following sample code demonstrates different operations that can be applied to an object, such as create, search, modify, and remove an object. This sample code also includes error handling for the operations.

#Preparation prior to a DNS IPv4 PTR record insertion

```
#PROGRAM STARTS: Include all the modules that will be used
use strict;
```

```
use Infoblox;

#Create a session to the Infoblox appliance
my $session = Infoblox::Session->new(
    master   => "192.168.1.2",
    username => "admin",
    password => "infoblox"
);
unless ($session) {
   die("Construct session failed: ",
       $session->status_code() . ":" . $session->status_detail());
}
print "Session created successfully\n";

#Create the zone prior to a PTR record insertion
my $zone = Infoblox::DNS::Zone->new(name => "10.0.0.0/8");
unless ($zone) {
   die("Construct zone failed: ",
       Infoblox::status_code() . ":" . Infoblox::status_detail());
}
print "Zone object created successfully\n";

#Verify if the zone exists
my $object = $session->get(object => "Infoblox::DNS::Zone", name => "10.in-addr.arpa");
unless ($object) {
   print "Zone does not exist on server, safe to add the zone\n";
   $session->add($zone)
      or die("Add zone failed: ",
             $session->status_code() . ":" . $session->status_detail());
}
print "Zone added successfully\n";
```

#Create a DNS IPv4 PTR record

```
my $bind_ptr = Infoblox::DNS::Record::PTR->new(
   ptrdname     => "bind_ptr.domain.com",
   comment  => " this is a demo bind_ptr record ",
   ipv4addr => "10.0.0.1"
 );
 unless ($bind_ptr) {
    die("Construct DNS record PTR failed: ",
        Infoblox::status_code() . ":" . Infoblox::status_detail());
 }
print "DNS PTR object created successfully\n";

#Add the DNS PTR record object to the Infoblox appliance through a session
$session->add($bind_ptr)
    or die("Add record PTR failed: ",
           $session->status_code() . ":" . $session->status_detail());
print "DNS PTR object added to server successfully\n";
```

#Search for a specific DNS IPv4 PTR record

```
#Search for all PTR records that match "10.in-addr.arpa"
my @retrieved_objs = $session->search(
    "object" => "Infoblox::DNS::Record::PTR",
    "name" => '10\.in-addr.arpa',
);
my $object = $retrieved_objs[0];
unless ($object) {
    die("Search record PTR failed: ",
        $session->status_code() . ":" . $session->status_detail());
}
print "Search DNS PTR object found at least 1 matching entry\n";

#Search all PTR records that have "bind" in the comment
my @retrieved_objs = $session->search(
    object => "Infoblox::DNS::Record::PTR",
    name   => '10\.*\.in-addr.arpa',
    comment => "bind*"
);
my $object = $retrieved_objs[0];
unless ($object) {
    die("Search record PTR failed: ",
        $session->status_code() . ":" . $session->status_detail());
}
```

```
print "Search DNS PTR object using regexp found at least 1 matching entry\n";
```

#Get and modify a DNS IPv4 PTR record

```
#Get a PTR record through the session
my @retrieved_objs = $session->get(
    object => "Infoblox::DNS::Record::PTR",
    name   => "1.0.0.10.in-addr.arpa",
);
my $object = $retrieved_objs[0];
unless ($object) {
    die("Get record PTR failed: ",
        $session->status_code() . ":" . $session->status_detail());
}
print "Get DNS PTR object found at least 1 matching entry\n";
#Modify one of the attributes of the specified PTR record
$object->comment ("modified comment");

#Apply the changes
$session->modify($object)
    or die("Modify record PTR failed: ",
           $session->status_code() . ":" . $session->status_detail());
print "DNS PTR object modified successfully \n";
```

#Remove a DNS IPv4 PTR record

```
#Get a PTR record through the session
my @retrieved_objs = $session->get(
    object => "Infoblox::DNS::Record::PTR",
    name   => "1.0.0.10.in-addr.arpa",
);

 my $object = $retrieved_objs[0];
  unless ($object) {
    die("Get record PTR failed: ",
         $session->status_code() . ":" . $session->status_detail());
  }
  print "Get DNS PTR object found at least 1 matching entry\n";

#submit the object for removal
$session->remove($object)
    or die("Remove record PTR failed: ",
           $session->status_code() . ":" . $session->status_detail());
print "DNS PTR object removed successfully \n";
```

#Preparation prior to a DNS IPv6 PTR record insertion

```
#PROGRAM STARTS: Include all the modules that will be used
use strict;
use Infoblox;

#Create a session to the Infoblox appliance
my $session = Infoblox::Session->new(
    master   => "192.168.1.2",
    username => "admin",
    password => "infoblox"
);
unless ($session) {
   die("Construct session failed: ",
       $session->status_code() . ":" . $session->status_detail());
}
print "Session created successfully\n";

#Create the zone prior to a PTR record insertion
my $zone = Infoblox::DNS::Zone->new(name => "1234:5678:9abc::/48");
unless ($zone) {
   die("Construct zone failed: ",
       Infoblox::status_code() . ":" . Infoblox::status_detail());
}
print "Zone object created successfully\n";

#Verify if the zone exists
my $object = $session->get(object => "Infoblox::DNS::Zone", name => "c.b.a.9.8.7.6.5.4.3.2.1.ip6.arpa");
unless ($object) {
   print "Zone does not exist on server, safe to add the zone\n";
   $session->add($zone)
      or die("Add zone failed: ",
```

```
            $session->status_code() . ":" . $session->status_detail());
}
print "Zone added successfully\n";
```

#Create a DNS IPv6 PTR record

```
my $bind_ptr = Infoblox::DNS::Record::PTR->new(
   ptrdname     => "ip6host.this.com",
   comment  => " this is a demo bind_ptr record ",
   ipv6addr => "1234:5678:9abc:1000::1"
 );
 unless ($bind_ptr) {
    die("Construct DNS record PTR failed: ",
        Infoblox::status_code() . ":" . Infoblox::status_detail());
 }
print "DNS PTR object created successfully\n";
#Add the DNS PTR record object to the Infoblox  appliance through a session
       $session->add($bind_ptr)
           or die("Add record PTR failed: ",
                  $session->status_code() . ":" . $session->status_detail());
       print "DNS PTR object added to server successfully\n";
```

#Search for a specific DNS IPv6 PTR record

```
#Search for a PTR record in the zone
my @retrieved_objs = $session->search(
    "object" => "Infoblox::DNS::Record::PTR",
    "name" => "1.0.0.0.0.0.0.0.0.0.0.0.0.0.0.0.0.0.0.0.0.0.0.0.0.1.c.b.a.9.8.7.6.5.4.3.2.1.ip6.arpa",
);
my $object = $retrieved_objs[0];
unless ($object) {
    die("Search record PTR failed: ",
        $session->status_code() . ":" . $session->status_detail());
}
print "Search DNS PTR object found at least 1 matching entry\n";
```

#Get and modify a DNS IPv6 PTR record

```
#Get PTR record through the session
my @retrieved_objs = $session->get(
    object => "Infoblox::DNS::Record::PTR",
    name   => "1.0.0.0.0.0.0.0.0.0.0.0.0.0.0.0.0.0.0.0.0.0.0.0.0.1.c.b.a.9.8.7.6.5.4.3.2.1.ip6.arpa",
);
my $object = $retrieved_objs[0];
unless ($object) {
    die("Get record PTR failed: ",
        $session->status_code() . ":" . $session->status_detail());
}
print "Get DNS PTR object found at least 1 matching entry\n";

#Modify one of the attributes of the specified PTR record
$object->comment ("modified comment");

#Apply the changes
$session->modify($object)
    or die("Modify record PTR failed: ",
            $session->status_code() . ":" . $session->status_detail());
print "DNS PTR object modified successfully \n";
```

#Remove a DNS IPv6 PTR record

```
#Get PTR record through the session
my @retrieved_objs = $session->get(
    object => "Infoblox::DNS::Record::PTR",
    name   => "1.0.0.0.0.0.0.0.0.0.0.0.0.0.0.0.0.0.0.0.0.0.0.0.0.1.c.b.a.9.8.7.6.5.4.3.2.1.ip6.arpa",
);
my $object = $retrieved_objs[0];
unless ($object) {
    die("Get record PTR failed: ",
         $session->status_code() . ":" . $session->status_detail());
}
print "Get DNS PTR object found at least 1 matching entry\n";

#Submit the object for removal
$session->remove($object)
    or die("Remove record PTR failed: ",
           $session->status_code() . ":" . $session->status_detail());
```

```
print "DNS PTR object removed successfully \n";

####PROGRAM ENDS####
```

AUTHOR

Infoblox Inc. http://www.infoblox.com/

SEE ALSO

Infoblox::DNS::View, Infoblox::DNS::Zone, Infoblox::Session, Infoblox::Session->get(), Infoblox::Session->search(), Infoblox::Session->add(), Infoblox::Session->remove(), Infoblox::Session->modify(), Infoblox::Grid::MSServer::AdUser::Data, Infoblox::Grid::Discovery::Data

COPYRIGHT

Copyright (c) 2017 Infoblox Inc.

Infoblox::DNS::Record::RRSIG - RRSIG Record object

- NAME
- DESCRIPTION
- CONSTRUCTOR
- SESSION METHODS
  - Infoblox::Session->get( )
  - Infoblox::Session->search( )
- METHODS
  - algorithm( )
  - cloud_info( )
  - creator( )
  - dns_name( )
  - dns_signer_name( )
  - expiration( )
  - inception( )
  - key_tag( )
  - labels( )
  - name( )
  - original_ttl( )
  - signature( )
  - signer_name( )
  - ttl( )
  - type_covered( )
  - view( )
  - zone( )
- SAMPLE CODE
- AUTHOR
- SEE ALSO
- COPYRIGHT

NAME

Infoblox::DNS::Record::RRSIG - RRSIG Record object

DESCRIPTION

RRSIG records are one of the resource records in DNSSEC. These records store digital signatures of resource record sets (RRsets). The digital signatures are used to authenticate the data that is in the signed RRsets.

RRSIG resource records are defined in RFC 4034.

RRSIG records are automatically generated upon the signing of an authoritative zone. The RRSIG record-object is read-only.

CONSTRUCTOR

The Infoblox::DNS::Record::RRSIG object is a read-only object and does not require manual construction.

SESSION METHODS

This section describes all the methods in an Infoblox::Session that can be applied to an RRSIG record object.

Infoblox::Session->get( )

Use this method to retrieve the existing objects from an Infoblox appliance. See Infoblox::Session->get() for parameters and return values.

Key Reference

```
    Apply the following attributes to get the RRSIG Record object(s):

     algorithm       - Optional. The public key encryption algorithm. The allowed values are
'1','3','5','6','7','RSAMD5','RSASHA1','DSA','NSEC3RSASHA1','NSEC3DSA','RSASHA256','RSASHA512','NSEC3RSASHA256'
 and 'NSEC3RSASHA512'.
     expiration      - Optional. Signature expiration time, in seconds, since 1 January 1970 00:00:00 UTC.
     name            - Optional. The name of the corresponding RRset.
     inception       - Optional. Signature inception time, in seconds, since 1 January 1970 00:00:00 UTC.
     key_tag         - Optional. Key tag identifying the corresponding public key in the DNSKEY record.
     labels          - Optional. Specifies the number of labels in the original RRSIG RR owner name.
     original_ttl    - Optional. TTL of the covered RRset as it appears in the authoritative zone.
     signature       - Optional. Cryptographic signature that covers the RRSIG RDATA.
     signer_name     - Optional. Domain name of the zone that contains the signed RRset.
     ttl             - Optional. TTL value of the record itself.
     type_covered    - Optional. Identifies the type of the RRset that is covered by this RRSIG record.
     view            - Optional. The DNS view in which the record is located. By default, all DNS views are
searched. But if you omit this attribute and specify a zone, the appliance searches the 'default' view only.
     zone            - Optional. A zone name in FQDN format.
```

Example

```
# get all the RRSIG records for the zone 'domain.com' in the default DNS view.
my @retrieved_objs = $session->get(
                     object => "Infoblox::DNS::Record::RRSIG",
                     signer_name   => "domain.com",
                     view   => "default" );

# get the RRSIG record that corresponds to the 'test.domain.com' A record of the default DNS view
my $retrieved_obj = $session->get(
                     object    => "Infoblox::DNS::Record::RRSIG",
                     name      => "test.domain.com",
                     type_covered => "A",
                     view      => "default" );
```

Infoblox::Session->search( )

Use this method to retrieve the existing objects from an Infoblox appliance. See Infoblox::Session->search() for parameters and return values.

Key Reference

```
 Apply the following attributes to search the RRSIG Record object(s):

     algorithm       - Optional. The public key encryption algorithm. The allowed values are
'1','3','5','6','7','RSAMD5','RSASHA1','DSA','NSEC3RSASHA1','NSEC3DSA','RSASHA256','RSASHA512','NSEC3RSASHA256'
 and 'NSEC3RSASHA512'.
     expiration      - Optional. Signature expiration time, specified as seconds since 1 January 1970
00:00:00 UTC. This method supports searching with limit parameters (see below).
     name            - Optional. The name of the corresponding RRset (regular expression).
     inception       - Optional. Signature inception time, specified as seconds since 1 January 1970 00:00:00
UTC. This method supports searching with limit parameters (see below).
     key_tag         - Optional. Key tag that identifies the corresponding public key in the DNSKEY record.
     labels          - Optional. Specifies the number of labels in the original RRSIG RR owner name.
     original_ttl    - Optional. TTL of the covered RRset as it appears in the authoritative zone.
     signature       - Optional. Cryptographic signature that covers the RRSIG RDATA (regular expression).
     signer_name     - Optional. Domain name of the zone that contains the signed RRset (regular expression).
     ttl             - Optional. TTL value of the record itself.
     type_covered    - Optional. Identifies the type of the RRset that is covered by this RRSIG record
(regular expression).
     view            - Optional. The DNS view in which the record is located. By default, all DNS views are
searched. But if you omit this attribute and specify a zone, the appliance searches the 'default' view only.
     zone            - Optional. A zone name in FQDN format.
```

See Infoblox::Session/search with limit parameters for more information on this functionality.

Example

```
# get all the RRSIG records for the zone matching the regular expression.
my @retrieved_objs = $session->search(
                     object => "Infoblox::DNS::Record::RRSIG",
                     signer_name   => ".*[.]com",);

# get all the RRSIG records with inception time between 12:00 September 1, 2009 and 12:00 September 10,
2009, expiring no later then January 1, 2010.
my @retrieved_objs = $session->search(
                     object           => "Infoblox::DNS::Record::RRSIG",
                     inception        => ">=< 1254398400,1255176000",
                     expiration       => "<= 1264982400");

# search for all RRSIG records in the "domain.com" zone of the default DNS view
# Note that the 'default' DNS view is assumed implicitly here
my @retrieved_objs = $session->search(
    object => "Infoblox::DNS::Record::RRSIG",
    zone   => "domain.com",
   );
```

METHODS

This section describes all the methods that can be used to retrieve the attribute values of an RRSIG Record object.

algorithm( )

Use this method to retrieve the value of the public key encryption algorithm used to create the RRSIG signature.

Parameter

none

Returns

The method returns the public key encryption algorithm. Returned values are one of the following:

RSAMD5
DSA
RSASHA1
NSEC3DSA
NSEC3RSASHA1
RSASHA256

RSASHA512
Example

```
#Get the public key encryption algorithm.
my $algorithm = $rrsig->algorithm();
```

cloud_info( )

Use this method to retrieve cloud API related information for the Infoblox::DNS::Record::RRSIG object.

Parameter

None

Returns

The method returns the attribute value.

Example

```
# Get cloud_info
my $cloud_info = $object->cloud_info();
```

creator( )

Use this method to retrieve the RRSIG record creator. This is a read-only attribute.

Parameter

None

Returns

The method returns the attribute value.

Example

```
# Get attribute value
my $value = $object->creator();
```

dns_name( )

Use this method to retrieve the FQDN, in punycode format, of the RRset covered by the RRSIG record. This is a read-only attribute.

Parameter

None

Returns

The method returns the attribute value.

Example

```
# Get attribute value
my $value = $rrsig->dns_name();
```

dns_signer_name( )

Use this method to retrieve the domain name, in punycode format, of the zone that contains the signed RRset. This is a read-only attribute.

Parameter

None

Returns

The method returns the attribute value.

Example

```
# Get attribute value
my $value = $bind_a->dns_signer_name();
```

expiration( )

Use this method to retrieve the expiration time of an RRSIG record.

Parameter

none

Returns

The method returns the RRSIG record expiration time. The returned value is in UNIX timestamp format; that is, it is measured in seconds since 00:00:00 January 1, 1970 UTC.

Example

```
#Get the RRSIG expiration time
my $expiration = $rrsig->expiration();
```

inception( )

Use this method to retrieve the RRSIG inception time. This is the time when the record was created.

Parameter

none

Returns

The method returns the RRSIG record inception time. The returned value is in UNIX timestamp format; that is, it is measured in seconds since 00:00:00 January 1, 1970 UTC.

Example

```
#Get the inception time of the RRSIG record
my $inception = $rrsig->inception();
```

key_tag( )

Use this method to retrieve the key tag of the public key that correspods to the RRSIG Record object.

Parameter

none

Returns

The method returns the key tag of the public key that corresponds to the private key used to encrypt RRSIG data. The key tag is a 16-bit integer key fingerprint that is used to distinguish one key from another when multiple keys are associated with the same zone.

Example

```
#Get the key tag of the public key
my $key_tag = $rrsig->key_tag();
```

labels( )

Use this method to retrieve the number of labels in the name of the RRset signed with the RRSIG object.

Parameter

none

Returns

The method returns the number of labels in the original RRSIG RR owner name. Usually this number is the same as the number of labels in the 'name' field.

Example

```
#Get the number of labels in the original RRSIG RR owner name.
my $labels = $rrsig->labels();
```

name( )

Use this method to retrieve the FQDN of the RRset covered by the RRSIG record.

The attribute value can be in unicode format.

Parameter

none

Returns

The method returns the name attribute of the RRSIG object. This is a name of the RRset covered by the RRSIG record.

Example

```
#Get the name attribute
my $fqdn = $rrsig->name();
```

original_ttl( )

Use this method to retrieve the TTL of the covered RRset as it appears in the authoritative zone of an RRSIG record object.

Parameter

none

Returns

The method returns the TTL of the covered RRset as it appears in the authoritative zone attribute value.

Example

```
#Get the TTL of the covered RRset
my $original_ttl = $rrsig->original_ttl();
```

signature( )

Use this method to retrieve the cryptographic signature that covers the RRSIG RDATA of an RRSIG Record object.

Parameter

none

Returns

The method returns the cryptographic signature that covers the RRSIG RDATA attribute value.

Example

```
#Get the signature that covers the RRSIG RDATA
my $signature = $rrsig->signature();
```

signer_name( )

Use this method to retrieve the domain name of the zone that contains the signed RRset.

The attribute value can be in unicode format.

Parameter

none

Returns

The method returns the domain name of the zone that contain the signed RRset in FQDN format.

Example

```
#Get the signer name
my $signer_name = $rrsig->signer_name();
```

ttl( )

Use this method to retrieve the Time to Live (TTL) value of an RRSIG Record object.

Parameter

none

Returns

The method returns the TTLattribute value. The returned parameter is a 32-bit integer (range from 0 to 4294967295) that represents the duration in seconds that the record is cached. Zero indicates that the record should not be cached.

Example

```
#Get the TTL of the RRSIG record
my $ttl = $rrsig->ttl();
```

type_covered( )

Use this method to retrieve the type of the RRset that is covered by this RRSIG record.

Parameter

none

Returns

The method returns the type of the RRset that is covered by this RRSIG record attribute value.

Example

```
#Get the type of the RRset that is covered by this RRSIG record
```

```
my $type_covered = $rrsig->type_covered();
```

view( )

Use this method to retrieve the DNS view object that contains the RRSIG Record object.

Parameter

none

Returns

The method returns the Infoblox::DNS::View object that contains the RRSIG record.

Example

```
#Get the DNS view
my $view = $rrsig->view();
```

zone( )

Use this method to retrieve the zone name of an RRSIG record.

Parameter

none

Returns

Returns the zone name of the zone that contains RRSIG record. The name is in FQDN format.

Example

```
# Get zone
my $zone = $rrsig->zone();
```

SAMPLE CODE

The following sample code demonstrates the session methods on an RRSIG record object.

```
#PROGRAM STARTS: Include all the modules that will be used
use strict;
use Infoblox;

#Create a session to the Infoblox appliance
       my $session = Infoblox::Session->new(
           master   => "192.168.1.2",
           username => "admin",
           password => "infoblox"
       );
       unless ($session) {
          die("Construct session failed: ",
              $session->status_code() . ":" . $session->status_detail());
       }
       print "Session created successfully\n";

       #Enable DNSSEC in the default view

       my $default_view=$session->get(
                                      object=> "Infoblox::DNS::View",
                                      name => "default"
                                     );
       unless($default_view) {
           die("Getting the default view failed:",
              Infoblox::status_code() . ":" . Infoblox::status_detail());
       }
       print "Got the default view successfully\n";

       $default_view->dnssec_enabled("true")
           or die("Changing the dnssec_enabled in the default view failed:",
                  Infoblox::status_code() . ":" . Infoblox::status_detail());
       $session->modify($default_view)
           or die("Changing the dnssec_enabled in the default view failed:",
                  Infoblox::status_code() . ":" . Infoblox::status_detail());
```

#Creating a signed zone and populationg it with an A record

```
       print "Creating Member primary server for the zone\n";
       my $primary=Infoblox::DNS::Member->new(
                                              ipv4addr => "192.168.1.2",
                                              name => "infoblox.localdomain"
                                             );
```

```
unless($primary) {
   die("Unable to create primary server object: ",
       Infoblox::status_code() . ":" . Infoblox::status_detail());
}

my $zone = Infoblox::DNS::Zone->new(
                                    name => "domain.com",
                                    primary => $primary
                                   );
unless ($zone) {
   die("Construct zone failed: ",
       Infoblox::status_code() . ":" . Infoblox::status_detail());
}
print "Zone object created successfully\n";

#Verify if the zone exists
my $object = $session->get(object => "Infoblox::DNS::Zone", name => "domain.com");
unless ($object) {
   print "Zone does not exist on server, safe to add the zone\n";
   $session->add($zone)
      or die("Add zone failed: ",
             $session->status_code() . ":" . $session->status_detail());
}
print "Zone added successfully\n";

#Adding A record to the zone to demonstrate the corresponding RRSIG record
my $a_record=Infoblox::DNS::Record::A->new(
                                           ipv4addr => "10.9.8.7",
                                           name => "recorda.domain.com"
                                          );
unless($a_record) {
   die("Creating A record failed : ",
      Infoblox::status_code() . ":" . Infoblox::status_detail());
}

$object = $session->get( object => "Infoblox::DNS::Record::A", name => "recorda.domain.com" );
unless ($object) {
   print "A record does not exist on the server, safe to add the A record\n";
   $session->add($a_record)
     or die("Adding A record failed: ",
            Infoblox::status_code() . ":" . Infoblox::status_detail());
}
print "A record added to the zone successfully\n";

#Retrieving zone back from the server in order to sign it
$zone = $session->get(object => "Infoblox::DNS::Zone", name => "domain.com");
unless($zone) {
   die("Retrieving zone back failed: ",
        Infoblox::status_code( ). ":". Infoblox::status_detail( ));
}
print "Zone retrieved for signing successfully.\n";

$zone->dnssec_ksk_algorithm("NSEC3RSASHA1") &&
$zone->dnssec_zsk_algorithm("NSEC3RSASHA1") &&
$zone->dnssec_ksk_size(640) &&
$zone->dnssec_zsk_size(640)
  or die("Changing the zone DNSSEC setting failed: ",
         Infoblox::status_code() . ":" . Infoblox::status_detail());
$session->modify($zone)
  or die("Modifying dnssec values in zone failed: ",
         Infoblox::status_code() . ":" . Infoblox::status_detail());
print "Zone modified successfully\n";

#Signing the zone
$zone->dnssec_signed("true")
   or die("Signing of the zone failed: ",
           Infoblox::status_code() . ":" . Infoblox::status_detail());
       print "Zone signed successfully\n";
```

#Getting the RRSIG record corresponding to A record

```
my $rrsig_record_a=$session->get(
                            object => "Infoblox::DNS::Record::RRSIG",
                            name   => "recorda.domain.com",
                            signer_name => "domain.com",
                            type_covered   => "A",
                            view   => "default"
                           );
     unless($rrsig_record_a) {
  die("Getting RRSIG corresponding to the A record failed: ",
      Infoblox::status_code() . ":" . Infoblox::status_detail());
      }
```

```
            print "Got RRSIG record successfully\n";
```

#Searching for RRSIG objects using regular expressions

```
      my @retrieved_objs=$session->search(
                                          object => "Infoblox::DNS::Record::RRSIG",
                                          name   => ".*com",
                                          type_covered => "A|MX",
                                          view   => "default"
                                         );
          unless(@retrieved_objs>0) {
        die("Searching for RRSIG objects failed: ",
            Infoblox::status_code() . ":" . Infoblox::status_detail());
          }
          print "Search for the RRSIG objects successful, ".scalar(@retrieved_objs)." objects found\n";

      #Searching for RRSIG objects created today using inception time interval search
      @retrieved_objs=$session->search(
                                          object => "Infoblox::DNS::Record::RRSIG",
                                          name   => ".*com",
                                          inception => ">=< " . (time() -86400) . "," . time(),
                                          view   => "default"
                                         );
          unless(@retrieved_objs>0) {
        die("Searching for RRSIG objects failed: ",
            Infoblox::status_code() . ":" . Infoblox::status_detail());
          }
          print "Search for the RRSIG objects successful, ".scalar(@retrieved_objs)." objects found\n";
```

#Removing the created zone and cleaning up the view

```
      $session->remove($zone)
        or die("Unable to remove the zone: ",
                Infoblox::status_code() . ":" . Infoblox::status_detail());
      print "Zone removed successfully\n";

      $default_view->dnssec_enabled("false")
      && $default_view->override_dnssec("false")
      && $session->modify($default_view)
       or die("Restoring dnssec_enabled value in the default view failed: ",
              Infoblox::status_code() . ":" . Infoblox::status_detail());

 ####PROGRAM ENDS####
```

AUTHOR

Infoblox Inc. http://www.infoblox.com/

SEE ALSO

Infoblox::Session, Infoblox::Session->get(), Infoblox::Session->search(), Infoblox::DNS::Record::DNSKEY, Infoblox::DNS::View, Infoblox::DNS::Zone

---

COPYRIGHT

Copyright (c) 2017 Infoblox Inc.

Infoblox::DNS::Record::SRV - DNS SRV record object

- NAME
- DESCRIPTION
- CONSTRUCTOR
- SESSION METHODS
  - Infoblox::Session->add( )
  - Infoblox::Session->get( )
  - Infoblox::Session->modify( )
  - Infoblox::Session->remove( )
  - Infoblox::Session->search( )
- METHODS
  - comment( )
  - cloud_info( )
  - creator( )
  - creation_time( )
  - ddns_principal( )
  - ddns_protected( )
  - disable( )
  - dns_name( )
  - dns_target( )
  - extattrs( )
  - extensible_attributes( )
  - forbid_reclamation( )
  - last_queried( )
  - name( )
  - port( )
  - priority( )
  - reclaimable( )
  - target( )
  - ttl( )
  - views( )
  - weight( )
  - zone( )
- SAMPLE CODE
- AUTHOR
- SEE ALSO
- COPYRIGHT

NAME

Infoblox::DNS::Record::SRV - DNS SRV record object

DESCRIPTION

A DNS SRV object represents an SRV resource record, which is also known as a service record. An SRV record provides information on available services.

Note that you must specify only one view for the attribute "views".

CONSTRUCTOR

```
 my $srv = Infoblox::DNS::Record::SRV->new(
     name                  => $fqdn,                                                  # Required
     port                  => $num,                                                   # Required
     priority              => $num,                                                   # Required
     target                => $fqdn,                                                  # Required
     weight                => $num,                                                   # Required
     comment               => $string,                                                # Optional
/ Default is empty
     disable               => "true" | "false"                                        # Optional
/ Default "false"
     extattrs              => { $string => $extattr, ... },                           # Optional
/ Default is undefined
     extensible_attributes => { $string => $string|$num, $string => [ $string|$num, ...], ... }, # Optional
/ Default is undefined
     ttl                   => $num | undef,                                           # Optional
/ Default is undefined
     views                 => [$View1],                                               # Optional
/ Default is "default" view
     creator               => "DYNAMIC" | "STATIC",                                   # Optional
/ Default is "STATIC"
```

```
    ddns_protected        => "true" | "false",                                  # Optional
/ Default is "false"
    ddns_principal        => $string,                                           # Optional
/ Default is undefined
    forbid_reclamation    => "true" | "false",                                  # Optional
/ Default is "false"
 );
```

You cannot set both extattrs and extensible_attributes attributes at the same time.

SESSION METHODS

This section describes all the methods in Infoblox::Session that you can apply to a DNS SRV record object.

Infoblox::Session->add( )

Use this method to add an object to the Infoblox appliance. See Infoblox::Session->add() for parameters and return values.

Example

```
#Construct object
my $bind_srv = Infoblox::DNS::Record::SRV->new(
    name     => "bind_srv.domain.com",
    comment  => "add bind srv bind_srv.domain.com",
    priority => 1,
    port     => 1,
    weight   => 1,
    target   => "old.target.test.org",
);
# Submit for addition
my $response = $session->add( $bind_srv );
```

Infoblox::Session->get( )

Use this method to retrieve the existing objects from the Infoblox appliance. See Infoblox::Session->get() for parameters and return values.

Key Reference

```
Apply the following attributes to get a specified DNS SRV object:

  name                  - Optional. A name in string format.
  view                  - Optional. The DNS view in which the SRV record is located. By default, all
DNS views are searched. But if you omit this attribute and specify a zone, the appliance searches the
'default' view only.
  zone                  - Optional. A zone name in FQDN format.
  weight                - Optional. The weight of the SRV record.
  port                  - Optional. The port of the SRV record.
  priority              - Optional. The priority of the SRV record.
  target                - Optional. The target of the SRV record.
  creator               - Optional. Record creator.
  ddns_principal        - Optional. GSS-TSIG principal that owns the record.
  reclaimable           - Optional. The flag that indicates whether the record is reclaimable or not.
  extattrs              - Optional. A hash reference containing extensible attributes.
  extensible_attributes - Optional. A hash reference containing extensible attributes.
```

Example

```
my @retrieved_objs = $session->get(
    object => "Infoblox::DNS::Record::SRV",
    name     => "bind_srv.domain.com",
    view   => "default" );

# get all DNS SRV record in zone "domain.com" of default view
my @retrieved_objs = $session->get(
    object => "Infoblox::DNS::Record::SRV",
    zone     => "domain.com",
    view   => "default" );

# get all DNS SRV records with a given extensible attribute
my @retrieved_objs = $session->get(
    object => "Infoblox::DNS::Record::SRV",
    extensible_attributes => { 'Site' => 'Santa Clara' });
```

Infoblox::Session->modify( )

Use this method to modify objects in the Infoblox appliance. See Infoblox::Session->modify() for parameters and return values.

Example

```
# Use method to modify the comment.
$bind_srv->comment("this is a modified comment");
# Submit modification
my $response = $session->modify( $bind_srv);
```

Infoblox::Session->remove( )

Use this method to remove an object from the Infoblox appliance. See Infoblox::Session->remove() for parameters and return values.

To remove a specific object, first use `get()` or `search()` to retrieve the specific object, and then submit this object for removal.

Example

```
# Get the object under the same name
my @retrieved_objs = $session->get(
    object => "Infoblox::DNS::Record::SRV",
    name     => "bind_srv.domain.com",
    view   => "default" );
my $desired_srv= $retrieved_objs[0];
# Submit for removal
my $response = $session->remove( $desired_srv );
```

Infoblox::Session->search( )

Use this method to search for existing DNS SRV record objects in the Infoblox appliance. See Infoblox::Session->search() for parameters and return values.

Key Reference

```
 Apply the following attributes to search for a DNS SRV record object:

   name                  - Required. A name in FQDN (Fully Qualified Domain Name) format (regular
expression).
   view                  - Optional. The DNS view in which the SRV record is located. By default, all
DNS views are searched. But if you omit this attribute and specify a zone, the appliance searches the
'default' view only.
   zone                  - Optional. A zone name in FQDN format.
   comment               - Optional . A comment in string format (regular expression).
   weight                - Optional. The weight of the SRV record (regular expression).
   port                  - Optional. The port of the SRV record (regular expression).
   priority              - Optional. The priority of the SRV record (regular expression).
   target                - Optional. The target of the SRV record (regular expression).
   creator               - Optional. Record creator.
   ddns_principal        - Optional. GSS-TSIG principal that owns the record.
   reclaimable           - Optional. The flag that indicates whether the record is reclaimable or
not.
   extattrs              - Optional. A hash reference containing extensible attributes.
   extensible_attributes - Optional. A hash reference containing extensible attributes.
```

For more information about searching extensible attributes, see Infoblox::Grid::ExtensibleAttributeDef/Searching Extensible Attributes.

Example

```
# search for all DNS SRV objects in the default DNS view that match "domain.com" and that contain
"bind" in the comment field
my @retrieved_objs = $session->search(
    object => "Infoblox::DNS::Record::SRV",
    name     => 'domain\.com',
    comment  => "bind",
    view     => "default" );

# search for all DNS SRV records in the "domain.com" zone of the default view
my @retrieved_objs = $session->search(
    object => "Infoblox::DNS::Record::SRV",
    zone     => "domain.com",
    view   => "default" );

# search all DNS SRV records with the extensible attribute 'Site'
my @retrieved_objs = $session->search(
   object => "Infoblox::DNS::Record::SRV",
   extensible_attributes => { 'Site' => 'Santa Clara' });
```

METHODS

This section describes all the methods that you can use to configure and retrieve the attribute values of an SRV record.

comment( )

Use this method to add or retrieve a descriptive comment.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

Desired comment in string format with a maximum of 256 bytes.

Returns

If you specified a parameter,the method returns true when the modification succeeds,and returns false when the operation fails.

Example

```
#Get comment
my $comment = $bindsrv->comment();
#Modify comment
$bindsrv->comment("Modifying the DNS SRV comment");
```

cloud_info( )

Use this method to retrieve cloud API related information for the Infoblox::DNS::Record::SRV object.

Parameter

None

Returns

The method returns the attribute value.

Example

```
# Get cloud_info
my $cloud_info = $object->cloud_info();
```

creator( )

Use this method to set or retrieve the record creator.

Note that changing creator from or to 'SYSTEM' value is not allowed.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Paramter

The valid values are 'STATIC' and 'DYNAMIC'. The default value is 'STATIC'.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get creator value
my $creator = $object->creator();

#Modify creator value
$object->creator("DYNAMIC");
```

creation_time( )

Use this method to retrieve the creation time for the record. This is a read-only attribute.

Parameter

None

Returns

The valid return value is a number of seconds that have elapsed since January 1st, 1970 UTC.

Example

```
#Get creation_time value
my $creation_time = $object->creation_time();
```

ddns_principal( )

Use this method to set or retrive the GSS-TSIG principal that owns this record.

Note that you cannot set ddns_principal for 'STATIC' and 'SYSTEM' records.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Paramter

The GSS-TSIG principal FQDN (Fully Qualified Domain Name) format. The FQDN consists of the hostname followed by the domain name (example: abc.com). A hostname can have a maximum of 256 characters.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get ddns_principal value
my $ddns_principal = $object->ddns_principal();

#Modify ddns_principal value
$object->ddns_principal('foo.com');
```

ddns_protected( )

Use this method to set or retrieve the flag that indicates whether DDNS updates for this record are allowed or not.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

Specify 'true' to protect record from DDNS updates and 'false' to allow DDNS updates for the specified record.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get ddns_protected value
my $ddns_protected = $object->ddns_protected();

#Modify ddns_protected value
$object->ddns_protected('true');
```

disable( )

Use this method to set or retrieve the disable flag.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

The default value for this field is false, therefore the DNS record is enabled.

Parameter

Specify "true" to set the disable flag or "false" to deactivate/unset it.

Returns

If you specified a parameter,the method returns true when the modification succeeds,and returns false when the operation fails.

Example

```
#Get disable
my $disable = $bindsrv->disable();
#Modify disable
$bindsrv->disable("true");
```

dns_name( )

Use this method to retrieve the name in punycode format. This is a read-only attribute.

Parameter

None

Returns

The method returns the attribute value.

Example

```
# Get attribute value
my $value = $bindsrv->dns_name();
```

dns_target( )

Use this method to retrieve the target in punycode format. This is a read-only attribute.

Parameter

None

Returns

The method returns the attribute value.

Example

```
# Get attribute value
my $value = $bindsrv->dns_target();
```

extattrs( )

Use this method to set or retrieve the extensible attributes associated with a DNS SRV record object.

Parameter

Valid value is a hash reference containing the names of extensible attributes and their associated values (Infoblox::Grid::Extattr objects).

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get extattrs
my $ref_extattrs = $bindsrv->extattrs();
#Modify extattrs
$bindsrv->extattrs({ 'Site' => $extattr1, 'Administrator' => $extattr2 });
```

extensible_attributes( )

Use this method to set or retrieve the extensible attributes associated with a DNS SRV record.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

For valid values for extensible attributes, see Infoblox::Grid::ExtensibleAttributeDef/Extensible Attribute Values.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get extensible attributes
my $ref_extensible_attributes = $bindsrv->extensible_attributes();
#Modify extensible attributes
$bindsrv->extensible_attributes({ 'Site' => 'Santa Clara', 'Administrator' => [ 'Peter', 'Tom' ] });
```

forbid_reclamation( )

Use this method to set or retrieve the flag that indicates whether the reclamation is allowed for the record or not.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

Specify 'true' to forbid reclamation for the record and 'false' to allow it. The default value is 'false'.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get forbid_reclamation
my $forbid_reclamation = $object->forbid_reclamation();
#Modify forbid_reclamation
$object->forbid_reclamation('true');
```

last_queried( )

Use this method to retrieve the time when the associated record was last queried. This is a read-only attribute.

Parameter

None

Returns

The method returns the attribute value. The number of seconds that have elapsed since January 1st, 1970 UTC.

Example

```
#Get last_queried
my $last_queried = $bindsrv->last_queried();
```

name( )

Use this method to set or retrieve the name.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

The attribute value can be in unicode format.

Parameter

The name of the DNS SRV record.

Returns

If you specified a parameter,the method returns true when the modification succeeds,and returns false when the operation fails.

Example

```
#Get name
my $name = $bindsrv->name();
#Modify name
$bindsrv->name("modified_bind_srv.domain.com");
```

port( )

Use this method to add or retrieve a port.

Parameter

The port number for the DNS SRV record (range from 0 to 65535).

Returns

If you specified a parameter,the method returns true when the modification succeeds,and returns false when the operation fails.

Example

```
#Get port
my $port = $bindsrv->port();
#Modify port
$bindport->port("2");
```

priority( )

Use this method to add or retrieve the priority.

Parameter

Number with the priority for the DNS SRV record (range from 0 to 65535).

Returns

If you specified a parameter,the method returns true when the modification succeeds,and returns false when the operation fails.

Example

```
#Get priority
my $priority = $bindsrv->priority();
#Modify priority
$bindsrv->priority("2");
```

reclaimable( )

Use this method to retrieve the flag that indicates whether the record is reclaimable or not.

Parameter

None

Returns

The method returns the attribute value.

Example

```
#Get reclaimable
my $reclaimable = $object->reclaimable();
```

target( )

Use this method to set or retrieve the target. The target must be in Fully Qualified Domain Name (FQDN) format.

The attribute value can be in unicode format.

Parameter

Text with the target for the DNS SRV record.

Returns

If you specified a parameter,the method returns true when the modification succeeds,and returns false when the operation fails.

Example

```
#Get target
my $target = $bindsrv->target();
#Modify target
$bindsrv->target ("new.target.test.org");
```

ttl( )

Use this method to configure or retrieve the Time to Live (TTL) value.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

The default value is undefined; therefore the record inherits the TTL value from the zone level.

With a specified TTL value, the method overrides the zone values with the specified TTL value.

Parameter

A 32-bit integer (range from 0 to 4294967295) that represents the duration in seconds that the record is cached. Zero indicates that the record should not be cached.

Returns

If you specified a parameter,the method returns true when the modification succeeds,and returns false when the operation fails.

Example

```
#Get ttl
my $ttl = $bindsrv->ttl();
```

```
#Modify ttl
$bindsrv->ttl(1800);
#Un-override ttl
$bindsrv->ttl(undef);
```

views( )

Use this method to specify or retrieve the views in which the SRV record is located.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

The default value is the default view, therefore the SRV record is located under the default view.

Parameter

Array reference of defined Infoblox::DNS::View objects.

Note that the array size must be 1.

Returns

If you specified a parameter,the method returns true when the modification succeeds,and returns false when the operation fails.

Example

```
#Get views
my $ref_views = $bindsrv->views();
#Modify views, list of Infoblox::DNS::View objects
$bindsrv->views([$view1]);
```

weight( )

Use this method to set or retrieve the weight.

Parameter

The weight of the DNS SRV record.

Returns

If you specified a parameter,the method returns true when the modification succeeds,and returns false when the operation fails.

Example

```
#Get weight
my $weight = $bindsrv->weight ();
#Modify weight
$bindsrv->weight ("3");
```

zone( )

Use this method to retrieve the zone name of the DNS SRV record. This method is read-only and cannot be set.

Parameter

None

Returns

Returns the attribute value.

Example

```
# Get zone
my $zone = $bindsrv->zone();
```

SAMPLE CODE

The following sample code demonstrates different operations that can be applied to an object, such as create, search, modify, and remove an object. This sample code also includes error handling for the operations.

#Preparation prior to a DNS SRV record insertion

```
#PROGRAM STARTS: Include all the modules that will be used
use strict;
use Infoblox;

#Create a session to the Infoblox appliance
```

```
my $session = Infoblox::Session->new(
    master   => "192.168.1.2",
    username => "admin",
    password => "infoblox"
);
unless ($session) {
   die("Construct session failed: ",
       $session->status_code() . ":" . $session->status_detail());
}
print "Session created successfully\n";

#Create the zone prior to a SRV record insertion
my $zone = Infoblox::DNS::Zone->new(name => "domain.com");
unless ($zone) {
   die("Construct zone failed: ",
       Infoblox::status_code() . ":" . Infoblox::status_detail());
}
print "Zone object created successfully\n";

#Verify if the zone exists
my $object = $session->get(object => "Infoblox::DNS::Zone", name => "domain.com");
unless ($object) {
   print "Zone does not exist on server, safe to add the zone\n";
   $session->add($zone)
      or die("Add zone failed: ",
             $session->status_code() . ":" . $session->status_detail());
}
print "Zone added successfully\n";
```

#Create a DNS SRV record

```
my $bind_srv = Infoblox::DNS::Record::SRV->new(
   name     => "bind_srv.domain.com",
   comment  => "add bind bind_srv.domain.com",
   priority => 1,
   port     => 1,
   weight   => 1,
   target   => "old.target.test.org",
 );
 unless ($bind_srv) {
    die("Construct DNS record SRV failed: ",
        Infoblox::status_code() . ":" . Infoblox::status_detail());
 }
print "DNS SRV object created successfully\n";

#Add the DNS SRV record object to Infoblox  appliance through a session
$session->add($bind_srv)
    or die("Add record SRV failed: ",
           $session->status_code() . ":" . $session->status_detail());
print "DNS SRV object added to server successfully\n";
```

#Search for a specific DNS SRV record

```
#Search for SRV records that match bind_srv.domain.com
my @retrieved_objs = $session->search(
    "object" => "Infoblox::DNS::Record::SRV",
    "name" => 'bind_srv\.domain\.com',
);
my $object = $retrieved_objs[0];
unless ($object) {
    die("Search record SRV failed: ",
        $session->status_code() . ":" . $session->status_detail());
}
print "Search DNS SRV object found at least 1 matching entry\n";

#Search all SRV records that match "domain.com" and that contain "bind" in the comment field
my @retrieved_objs = $session->search(
    object => "Infoblox::DNS::Record::SRV",
    name   => 'domain\.com',
    comment => "bind"
);
my $object = $retrieved_objs[0];
unless ($object) {
    die("Search record SRV failed: ",
       $session->status_code() . ":" . $session->status_detail());
}
```

```
print "Search DNS SRV object using regexp found at least 1 matching entry\n";
```

#Get and modify a DNS SRV record

```
#Get a SRV record through the session
my @retrieved_objs = $session->get(
    object => "Infoblox::DNS::Record::SRV",
    name   => "bind_srv.domain.com",
);
my $object = $retrieved_objs[0];
unless ($object) {
    die("Get record SRV failed: ",
        $session->status_code() . ":" . $session->status_detail());
}
print "Get DNS SRV object found at least 1 matching entry\n";

#Modify one of the attributes of the specified SRV record
$object->comment ("modified comment for SRV rec");

#Apply the changes
$session->modify($object)
    or die("Modify record SRV failed: ",
            $session->status_code() . ":" . $session->status_detail());
print "DNS SRV object modified successfully \n";
```

#Remove a DNS SRV record

```
#Get a SRV record through the session
my @retrieved_objs = $session->get(
    object => "Infoblox::DNS::Record::SRV",
    name   => "bind_srv.domain.com",
);
my $object = $retrieved_objs[0];
unless ($object) {
    die("Get record SRV failed: ",
        $session->status_code() . ":" . $session->status_detail());
}
print "Get DNS SRV object found at least 1 matching entry\n";

#submit the object for removal
$session->remove($object)
    or die("Remove record SRV failed: ",
            $session->status_code() . ":" . $session->status_detail());
print "DNS SRV object removed successfully \n";

####PROGRAM ENDS####
```

AUTHOR

Infoblox Inc. http://www.infoblox.com/

SEE ALSO

Infoblox::DNS::View, Infoblox::DNS::Zone, Infoblox::Session, Infoblox::Session->get(), Infoblox::Session->search(), Infoblox::Session->add(), Infoblox::Session->remove(), Infoblox::Session->modify()

COPYRIGHT

Copyright (c) 2017 Infoblox Inc.

Infoblox::DNS::Record::TLSA - DNS TLSA record object.

- NAME
- DESCRIPTION
- CONSTRUCTOR
- SESSION METHODS
  - Infoblox::Session->add( )
  - Infoblox::Session->get( )
  - Infoblox::Session->modify( )
  - Infoblox::Session->remove( )
  - Infoblox::Session->search( )
- METHODS
  - certificate_data( )
  - certificate_usage( )
  - comment( )
  - cloud_info( )
  - creator( )
  - disable( )
  - dns_name( )
  - extattrs( )
  - extensible_attributes( )
  - matched_type( )
  - name( )
  - selector( )
  - ttl( )
  - views( )
  - zone( )
- SAMPLE CODE
- AUTHOR
- SEE ALSO
- COPYRIGHT

NAME

Infoblox::DNS::Record::TLSA - DNS TLSA record object.

DESCRIPTION

The TLSA DNS resource record (RR) is used to associate a TLS server certificate or a public key with the domain name where the record is found, thus forming a 'TLSA certificate association'. For further details see RFC-6698.

Note that you must specify only one view for the attribute 'views'.

CONSTRUCTOR

```
 my $tlsa = Infoblox::DNS::Record::TLSA->new(
     name                  => $fqdn,                                                    # Required
     certificate_data      => $string,                                                  # Required
     certificate_usage     => $uint,                                                    # Required
     matched_type          => $uint,                                                    # Required
     selector              => $uint,                                                    # Required
     comment               => $string,                                                  # Optional /
Default is empty
     disable               => 'true' | 'false'                                          # Optional /
Default 'false'
     extattrs              => {$string => $extattr, ...},                               # Optional /
Default is undefined
     extensible_attributes => {$string => $string|$num, $string => [$string|$num, ...], ...}, # Optional /
Default is undefined
     ttl                   => $num | undef,                                             # Optional /
Default is undefined
     views                 => [$View1],                                                 # Optional /
Default is 'default' view
     creator               => 'DYNAMIC' | 'STATIC',                                     # Optional /
Default is 'STATIC'
 );
```

You cannot set both extattrs and extensible_attributes attributes at the same time.

SESSION METHODS

This section describes all the methods in Infoblox::Session module that you can apply to the object.

Infoblox::Session->add( )

Use this method to add an object to the Infoblox appliance. See Infoblox::Session->add() for parameters and return values.

Example

```
#Construct the object
my $object = Infoblox::DNS::Record::TLSA->new(
    name              => 'tlsa.domain.com',
    certificate_data  => '1234FFFFEEEE',
    certificate_usage => 0,
    matched_type      => 0,
    selector          => 0,
    comment           => 'add TLSA tlsa.domain.com',
    disable           => 'false',
    ttl               => '360',
    views             => [$view1],
);

#Submit for addition
my $response = $session->add($object);
```

Infoblox::Session->get( )

Use this method to retrieve all the matching objects from the Infoblox appliance. See Infoblox::Session->get() for parameters and return values.

Key References

```
Apply the following attributes to get a specific DNS TLSA object:

  name                - Optional. Host name in string format.
  view                - Optional. The DNS view in which the TLSA record is located. By default, all
DNS views are searched. But if you omit this attribute and specify a zone, the appliance searches the
'default' view only.
  certificate_data    - Optional. The certificate hash or actual data of the TLSA record in
hexadecimal string format.
  creator             - Optional. Record creator.
  extattrs            - Optional. A hash reference containing extensible attributes.
  extensible_attributes - Optional. A hash reference containing extensible attributes.
```

Example

```
my @retrieved_objs = $session->get(
    object => 'Infoblox::DNS::Record::TLSA',
    name   => 'tlsa.domain.com',
    view   => 'default' );

#Get all DNS TLSA records from the 'domain.com' zone of the default view
my @retrieved_objs = $session->get(
    object => 'Infoblox::DNS::Record::TLSA',
    zone   => 'domain.com',
    view   => 'default' );

#Get all DNS TLSA records with a given extensible attribute
my @retrieved_objs = $session->get(
    object => 'Infoblox::DNS::Record::TLSA',
    extensible_attributes => { 'Site' => 'Santa Clara' });
```

Infoblox::Session->modify( )

Use this method to modify an object in the Infoblox appliance. See Infoblox::Session->modify() for parameters and return values.

Example

```
#Use the following method to modify the comment.
$object->comment('This is a modified comment');

#Submit modification
my $response = $session->modify($object);
```

Infoblox::Session->remove( )

Use this method to remove an object from the Infoblox appliance. See Infoblox::Session->remove() for parameters and return values.

To remove a specifc object, first use `get()` or `search()` to retrieve the specific object, and then submit this object for removal.

Example

```
#Get the objects with the same name
my @retrieved_objs = $session->get(
    object => 'Infoblox::DNS::Record::TLSA',
    name   => 'tlsa.domain.com',
    view   => 'default' );

#Find the desired object from the retrieved list.
my $desired_object = $retrieved_objs[0];

#Submit for removal
my $response = $session->remove($desired_object);
```

Infoblox::Session->search( )

Use this method to search for objects in the Infoblox appliance. See Infoblox::Session->search() for parameters and return values.

Key References

```
Apply the following attributes to search for a DNS TLSA record object:

  name                  - Optional. A name in FQDN (Fully Qualified Domain Name) format (regular
expression).
  view                  - Optional. The DNS view in which the TLSA record is located. By default, all
DNS views are searched. But if you omit this attribute and specify a zone, the appliance searches the
'default' view only.
  zone                  - Optional. A zone name in FQDN format.
  comment               - Optional. A comment in string format (regular expression).
  certificate_data      - Optional. The certificate hash or actual data of the TLSA record in
hexadecimal string format (regular expression). In upper case only.
  creator               - Optional. Record creator.
  extattrs              - Optional. A hash reference containing extensible attributes.
  extensible_attributes - Optional. A hash reference containing extensible attributes.
```

For more information about searching extensible attributes, see Infoblox::Grid::ExtensibleAttributeDef/Searching Extensible Attributes.

Example

```
#Search for all DNS TLSA objects that match 'domain.com' in the default DNS view
my @retrieved_objs = $session->search(
    object => 'Infoblox::DNS::Record::TLSA',
    name   => 'domain\.com',
    view   => 'default' );

#Search for all DNS TLSA records in the 'domain.com' zone of the default view
my @retrieved_objs = $session->search(
    object => 'Infoblox::DNS::Record::TLSA',
    zone   => 'domain.com',
    view   => 'default' );

#Search for all DNS TLSA records with a given extensible attribute 'Site'
my @retrieved_objs = $session->search(
   object => 'Infoblox::DNS::Record::TLSA',
   extensible_attributes => { 'Site' => 'Santa Clara' });
```

METHODS

This section describes all the methods that you can use to configure and retrieve the attribute values of a TLSA record.

certificate_data( )

Use this method to set or retrieve the certificate association data.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

Hexadecimal dump of either the raw certificate data for matched_type 0, or the hash calculated on the raw data for matched_type 1 and 2 in string format.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get certificate_data value
my $value = $object->certificate_data();

#Modify certificate_data value
$object->certificate_data('DEA033FFEEAA3256FFADAAA3333111');
```

certificate_usage( )

Use this method to set or retrieve the value that specifies (as per RFE-6698) the provided certificate association data.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

The valid value is an unsigned integer between 0 and 255.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get certificate_usage value
my $value = $object->certificate_usage();

#Modify certificate_usage value
$object->certificate_usage(0);
```

comment( )

Use this method to set or retrieve the descriptive comment.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

Desired comment in string format with a maximum of 256 bytes.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get comment value
my $value = $object->comment();

#Modify comment value
$object->comment('Modifying the comment');
```

cloud_info( )

Use this method to retrieve cloud API related information for the Infoblox::DNS::Record::TLSA object.

Parameter

None

Returns

The method returns the attribute value.

Example

```
# Get cloud_info value
my $value = $object->cloud_info();
```

creator( )

Use this method to set or retrieve the record creator.

Note that changing creator from or to 'SYSTEM' value is not allowed.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Paramter

The valid values are 'STATIC' and 'DYNAMIC'. The default value is 'STATIC'.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get creator value
my $value = $object->creator();

#Modify creator value
$object->creator('DYNAMIC');
```

disable( )

Use this method to set or retrieve the disable flag for a DNS record.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

The default value for this field is false. The DNS record is enabled.

Parameter

Specify 'true' to set the disable flag or 'false' to deactivate/unset it.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get disable value
my $value = $object->disable();

#Modify disable value
$object->disable('true');
```

dns_name( )

Use this method to retrieve the name in punycode format. This is a read-only attribute.

Parameter

None

Returns

The method returns the attribute value.

Example

```
# Get dns_name value
my $value = $object->dns_name();
```

extattrs( )

Use this method to set or retrieve the extensible attributes associated with a DNS TLSA record object.

Parameter

Valid value is a hash reference containing the names of extensible attributes and their associated values (Infoblox::Grid::Extattr objects).

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get extattrs value
```

```
my $value = $object->extattrs();

#Modify extattrs value
$object->extattrs({'Site' => $extattr1, 'Administrator' => $extattr2});
```

extensible_attributes( )

Use this method to set or retrieve the extensible attributes associated with a DNS TLSA record.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

For valid values for extensible attributes, see Infoblox::Grid::ExtensibleAttributeDef/Extensible Attribute Values.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get extensible_attributes value
my $value = $object->extensible_attributes();

#Modify extensible_attributes value
$object->extensible_attributes({'Site' => 'Santa Clara', 'Administrator' => ['Peter', 'Tom']});
```

matched_type( )

Use this method to set or retrieve the value that specifies (as per RFE-6698) the format to display the certificate association data.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

The valid value is an unsigned integer between 0 and 255.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get matched_type value
my $value = $object->matched_type();

#Modify matched_type value
$object->matched_type(0);
```

name( )

Use this method to set or retrieve the name.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

The attribute value can be in unicode format.

Parameter

The name of the DNS TLSA object in Fully Qualified Domain Name (FQDN) format. The name can have a maximum of 256 bytes.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get name value
my $value = $object->name();

#Modify name value
$object->name('new.domain.com');
```

selector( )

Use this method to set or retrieve the value that specifies (as per RFE-6698) which part of the TLS certificate presented by the server will be matched against the association data.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

The valid value is an unsigned integer between 0 and 255.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get selector value
my $value = $object->selector();

#Modify selector value
$object->selector(0);
```

ttl( )

Use this method to set or retrieve the Time to Live (TTL) value.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

The default value is undefined which indicates that the record inherits the TTL value of the zone.

Specify a TTL value to override the TTL value at the zone level.

Parameter

A 32-bit integer (range from 0 to 4294967295) that represents the duration in seconds to cache the record. Zero indicates that the record should not be cached.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get ttl value
my $value = $object->ttl();

#Modify ttl value
$object->ttl(1800);

#Un-override ttl
$object->ttl(undef);
```

views( )

Use this method to set or retrieve the view of a TLSA record.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

The default value is the 'default' view, which means that the TLSA record is located under the default view.

Parameter

Array reference of defined Infoblox::DNS::View objects.

Note that the array size must be 1.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get views value
my $value = $object->views();

#Modify views value
$object->views([$view1]);
```

zone( )

Use this method to retrieve the zone name of a DNS TLSA record. This is a read-only attribute.

Parameter

None

Returns

The method returns the attribute value.

Example

```
#Get zone value
my $zone = $object->zone();
```

SAMPLE CODE

The following sample code demonstrates the different functions that can be applied on an object, such as add, search, modify, and remove. This sample also includes error handling for the operations.

#Preparation prior to a DNS TLSA record insertion

```
#PROGRAM STARTS: Include all the modules that will be used
use strict;
use Infoblox;

#Create a session on the Infoblox appliance

my $session = Infoblox::Session->new(
               master   => "192.168.1.2", #appliance host ip
               username => "admin",       #appliance user login
               password => "infoblox"     #appliance password
               );

unless ($session) {
       die("Construct session failed: ",
               Infoblox::status_code() . ":" . Infoblox::status_detail());
}
print "Session created successfully\n";

#Create the zone prior to a TLSA record insertion
my $zone = Infoblox::DNS::Zone->new(name => "domain.com");
unless ($zone) {
       die("Construct zone failed: ",
               Infoblox::status_code() . ":" . Infoblox::status_detail());
}
print "Zone object created successfully\n";

#Verify if the zone exists
my $object = $session->get(object => "Infoblox::DNS::Zone", name => "domain.com");
unless ($object) {
       print "Zone does not exist on the server, safe to add the zone\n";
       $session->add($zone)
               or die("Add zone failed: ",
                               $session->status_code() . ":" . $session->status_detail());
}
print "Zone added successfully\n";
```

#Create a DNS TLSA record

```
my $bind_tlsa = Infoblox::DNS::Record::TLSA->new(
               name              => "tlsa.domain.com",
               comment           => "add TLSA tlsa.domain.com",
               certificate_data  => '033FFEEAA3256FFADAAA3333',
               certificate_usage => 0,
               matched_type      => 0,
               selector          => 0,
               );
```

```
unless ($bind_tlsa) {
       die("Construct DNS record TLSA failed: ",
               Infoblox::status_code() . ":" . Infoblox::status_detail());
}

#Add the DNS TLSA record object to the Infoblox appliance through a session
$session->add($bind_tlsa)
       or die("Add record TLSA failed: ",
                       $session->status_code() . ":" . $session->status_detail());
print "DNS TLSA object is added to the server successfully\n";
```

#Search for a specific DNS TLSA record

```
#Search all TLSA records in the zone that matches "domain.com"
my @retrieved_objs = $session->search(
               object => "Infoblox::DNS::Record::TLSA",
               name   => 'domain\.com'
               );
my $object = $retrieved_objs[0];

unless ($object) {
       die("Search record TLSA failed: ",
               $session->status_code() . ":" . $session->status_detail());
}
print "Search a DNS TLSA object found at least 1 matching entry\n";

#Search all TLSA records with names that start with "tlsa" and end with "domain.com"
my @retrieved_objs = $session->search(
               object => "Infoblox::DNS::Record::TLSA",
               name   => '^tlsa.*\.domain\.com\$'
               );
my $object = $retrieved_objs[0];

unless ($object) {
       die("Search record TLSA failed: ",
               $session->status_code() . ":" . $session->status_detail());
}
print "Search a DNS TLSA object using regexp found at least 1 matching entry\n";
```

#Get and modify a DNS TLSA record

```
#Get a TLSA record through the session
my @retrieved_objs = $session->get(
               object => "Infoblox::DNS::Record::TLSA",
               name   => "tlsa.domain.com"
               );
my $object = $retrieved_objs[0];

unless ($object) {
       die("Get record TLSA failed: ",
               $session->status_code() . ":" . $session->status_detail());
}
print "Get a DNS TLSA object found at least 1 matching entry\n";

#Modify one of the attributes of the specified TLSA record
$object->comment("this is new comment string");

#Apply the changes
$session->modify($object)
       or die("Modify record TLSA failed: ",
               $session->status_code() . ":" . $session->status_detail());
print "DNS TLSA object modified successfully \n";
```

#Remove a DNS TLSA record

```
#Get TLSA record through the session
my @retrieved_objs = $session->get(
    object => "Infoblox::DNS::Record::TLSA",
    name   => "tlsa.domain.com"
);
my $object = $retrieved_objs[0];
unless ($object) {
    die("Get record TLSA failed: ",
        $session->status_code() . ":" . $session->status_detail());
}
print "Get DNS TLSA object found at least 1 matching entry\n";
```

```
#Submit the object for removal
$session->remove($object)
       or die("Remove record TLSA failed: ",
                $session->status_code() . ":" . $session->status_detail());
print "DNS TLSA object removed successfully \n";

####PROGRAM ENDS####
```

AUTHOR

Infoblox Inc. http://www.infoblox.com/

SEE ALSO

Infoblox::Session->add(), Infoblox::Session->get(), Infoblox::Session->modify(), Infoblox::Session->remove(), Infoblox::Session->search(), Infoblox::Session, Infoblox::DNS::View, Infoblox::DNS::Zone

---

COPYRIGHT

Copyright (c) 2017 Infoblox Inc.

Infoblox::DNS::Record::TXT - DNS TXT record object.

- NAME
- DESCRIPTION
- CONSTRUCTOR
- SESSION METHODS
  - Infoblox::Session->add( )
  - Infoblox::Session->get( )
  - Infoblox::Session->modify( )
  - Infoblox::Session->remove( )
  - Infoblox::Session->search( )
- METHODS
  - comment( )
  - cloud_info( )
  - creator( )
  - creation_time( )
  - ddns_principal( )
  - ddns_protected( )
  - disable( )
  - dns_name( )
  - extattrs( )
  - extensible_attributes( )
  - forbid_reclamation( )
  - last_queried( )
  - name( )
  - reclaimable( )
  - text( )
  - ttl( )
  - views( )
  - zone( )
- SAMPLE CODE
- AUTHOR
- SEE ALSO
- COPYRIGHT

NAME

Infoblox::DNS::Record::TXT - DNS TXT record object.

DESCRIPTION

A TXT (text) record contains supplemental information for a host. SPF (Sender Policy Framework) records are specialized TXT records that identify the servers that send mail from a domain.

Note that you must specify only one view for the attribute "views".

CONSTRUCTOR

```
 my $txt = Infoblox::DNS::Record::TXT->new(
     name                  => $fqdn,                                                    # Required
     text                  => $string                                                   # Required
     comment               => $string,                                                  # Optional
/ Default is empty
     disable               => "true" | "false"                                          # Optional
/ Default "false"
     extattrs              => { $string => $extattr, ... },                             # Optional
/ Default is undefined
     extensible_attributes => { $string => $string|$num, $string => [ $string|$num, ...], ... }, # Optional
/ Default is undefined
     ttl                   => $num | undef,                                             # Optional
/ Default is undefined
     views                 => [$View1],                                                 # Optional
/ Default is "default" view
     creator               => "DYNAMIC" | "STATIC",                                     # Optional
/ Default is "STATIC"
     ddns_protected        => "true" | "false",                                         # Optional
/ Default is "false"
     ddns_principal        => $string,                                                  # Optional
/ Default is undefined
     forbid_reclamation    => "true" | "false",                                         # Optional
/ Default is "false"
 );
```

You cannot set both extattrs and extensible_attributes attributes at the same time.

SESSION METHODS

This section describes all the methods in Infoblox::Session module that you can apply to a TXT record object.

Infoblox::Session->add( )

Use this method to add an object to the Infoblox appliance. See Infoblox::Session->add() for parameters and return values.

Example

```
#Construct the TXT object
my $bind_txt = Infoblox::DNS::Record::TXT->new(
    name    => "txt.domain.com",
    text    => "this is text string",
    comment => "add TXT txt.domain.com",
    disable => "false",
    ttl     => "360",
    views   => [$view1]  );
# Submit for addition
my $response = $session->add( $bind_txt );
```

Infoblox::Session->get( )

Use this method to retrieve all the matching objects from the Infoblox appliance. See Infoblox::Session->get() for parameters and return values.

Key References

```
Apply the following attributes to get a specific DNS TXT object:

  name                  - Optional. Host name in string format.
  view                  - Optional. The DNS view in which the TXT record is located. By default, all
DNS views are searched. But if you omit this attribute and specify a zone, the appliance searches the
'default' view only.
  text                  - Optional. The text of the TXT record.
  creator               - Optional. Record creator.
  ddns_principal        - Optional. GSS-TSIG principal that owns the record.
  reclaimable           - Optional. The flag that indicates whether the record is reclaimable or not.
  extattrs              - Optional. A hash reference containing extensible attributes.
  extensible_attributes - Optional. A hash reference containing extensible attributes.
```

Example

```
my @retrieved_objs = $session->get(
    object => "Infoblox::DNS::Record::TXT",
    name   => "txt.domain.com",
    view   => "default" );

# get all DNS TXT records in the "domain.com" zone of the default view
my @retrieved_objs = $session->get(
    object => "Infoblox::DNS::Record::TXT",
    zone   => "domain.com",
    view   => "default" );

# get all DNS TXT records with a given extensible attribute
my @retrieved_objs = $session->get(
    object => "Infoblox::DNS::Record::TXT",
    extensible_attributes => { 'Site' => 'Santa Clara' });
```

Infoblox::Session->modify( )

Use this method to modify an object in the Infoblox appliance. See Infoblox::Session->modify() for parameters and return values.

Example

```
# Use method to modify the comment.
$bind_txt->comment("This is a modified comment");
# Submit modification
my $response = $session->modify( $bind_txt );
```

Infoblox::Session->remove( )

Use this method to remove an object from the Infoblox appliance. See Infoblox::Session->remove() for parameters and return values.

To remove a specifc object, first use `get()` or `search()` to retrieve the specific object, and then submit this object for removal.

Example

```
# Get the objects with the same name
my @retrieved_objs = $session->get(
    object => "Infoblox::DNS::Record::TXT",
    name   => "txt.domain.com",
    view   => "default" );
# find the desired object from the retrieved list.
my $desired_txt = $retrieved_objs[0];
# Submit for removal
my $response = $session->remove( $ desired_txt );
```

Infoblox::Session->search( )

Use this method to search for DNS TXT record objects in the Infoblox appliance. See Infoblox::Session->search() for parameters and return values.

Key References

```
 Apply the following attributes to search for a DNS TXT record object:

  name                  - Optional. A name in FQDN (Fully Qualified Domain Name) format (regular
expression).
  view                  - Optional. The DNS view in which the TXT record is located. By default, all
DNS views are searched. But if you omit this attribute and specify a zone, the appliance searches the
'default' view only.
  zone                  - Optional. A zone name in FQDN format.
  comment               - Optional. A comment in string format (regular expression).
  text                  - Optional. The text of the TXT record (regular expression).
  creator               - Optional. Record creator.
  ddns_principal        - Optional. GSS-TSIG principal that owns the record.
  reclaimable           - Optional. The flag that indicates whether the record is reclaimable or not.
  extattrs              - Optional. A hash reference containing extensible attributes.
  extensible_attributes - Optional. A hash reference containing extensible attributes.
```

For more information about searching extensible attributes, see Infoblox::Grid::ExtensibleAttributeDef/Searching Extensible Attributes.

Example

```
# search for all DNS TXT objects that match "domain.com" in the default DNS view
my @retrieved_objs = $session->search(
    object => "Infoblox::DNS::Record::TXT",
    name   => 'domain\.com',
    view   => "default" );

# search for all DNS TXT records in the "domain.com" zone of the default view
my @retrieved_objs = $session->search(
    object => "Infoblox::DNS::Record::TXT",
    zone   => "domain.com",
    view   => "default" );

# search all DNS TXT records with a given extensible attribute 'Site'
my @retrieved_objs = $session->search(
   object => "Infoblox::DNS::Record::TXT",
   extensible_attributes => { 'Site' => 'Santa Clara' });
```

METHODS

This section describes all the methods that you can use to configure and retrieve the attribute values of a TXT record

comment( )

Use this method to set or retrieve the descriptive comment.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

Desired comment in string format with a maximum of 256 bytes.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get comment
```

```
my $comment = $bind_txt->comment();
#Modify comment
$bind_txt->comment("Modifying the DNS TXT comment");
```

cloud_info( )

Use this method to retrieve cloud API related information for the Infoblox::DNS::Record::TXT object.

Parameter

None

Returns

The method returns the attribute value.

Example

```
# Get cloud_info
my $cloud_info = $object->cloud_info();
```

creator( )

Use this method to set or retrieve the record creator.

Note that changing creator from or to 'SYSTEM' value is not allowed.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Paramter

The valid values are 'STATIC' and 'DYNAMIC'. The default value is 'STATIC'.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get creator value
my $creator = $object->creator();

#Modify creator value
$object->creator("DYNAMIC");
```

creation_time( )

Use this method to retrieve the creation time for the record. This is a read-only attribute.

Parameter

None

Returns

The valid return value is a number of seconds that have elapsed since January 1st, 1970 UTC.

Example

```
#Get creation_time value
my $creation_time = $object->creation_time();
```

ddns_principal( )

Use this method to set or retrive the GSS-TSIG principal that owns this record.

Note that you cannot set ddns_principal for 'STATIC' and 'SYSTEM' records.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Paramter

The GSS-TSIG principal FQDN (Fully Qualified Domain Name) format. The FQDN consists of the hostname followed by the domain name (example: abc.com). A hostname can have a maximum of 256 characters.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get ddns_principal value
my $ddns_principal = $object->ddns_principal();

#Modify ddns_principal value
$object->ddns_principal('foo.com');
```

ddns_protected( )

Use this method to set or retrieve the flag that indicates whether DDNS updates for this record are allowed or not.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

Specify 'true' to protect record from DDNS updates and 'false' to allow DDNS updates for the specified record.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get ddns_protected value
my $ddns_protected = $object->ddns_protected();

#Modify ddns_protected value
$object->ddns_protected('true');
```

disable( )

Use this method to set or retrieve the disable flag of a DNS record.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

The default value for this field is false. The DNS record is enabled.

Parameter

Specify "true" to set the disable flag or "false" to deactivate/unset it.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get disable
my $disable = $bind_txt->disable();
#Modify disable
$bind_txt->disable("true");
```

dns_name( )

Use this method to retrieve the name in punycode format. This is a read-only attribute.

Parameter

None

Returns

The method returns the attribute value.

Example

```
# Get attribute value
my $value = $bind_txt->dns_name();
```

extattrs( )

Use this method to set or retrieve the extensible attributes associated with a DNS TXT record object.

Parameter

Valid value is a hash reference containing the names of extensible attributes and their associated values (Infoblox::Grid::Extattr objects).

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get extattrs
my $ref_extattrs = $bind_txt->extattrs();
#Modify extattrs
$bind_txt->extattrs({ 'Site' => $extattr1, 'Administrator' => $extattr2 });
```

extensible_attributes( )

Use this method to set or retrieve the extensible attributes associated with a DNS TXT record.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

For valid values for extensible attributes, see Infoblox::Grid::ExtensibleAttributeDef/Extensible Attribute Values.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get extensible attributes
my $ref_extensible_attributes = $bindtxt->extensible_attributes();
#Modify extensible attributes
$bindtxt->extensible_attributes({ 'Site' => 'Santa Clara', 'Administrator' => [ 'Peter', 'Tom' ] });
```

forbid_reclamation( )

Use this method to set or retrieve the flag that indicates whether the reclamation is allowed for the record or not.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

Specify 'true' to forbid reclamation for the record and 'false' to allow it. The default value is 'false'.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get forbid_reclamation
my $forbid_reclamation = $object->forbid_reclamation();
#Modify forbid_reclamation
$object->forbid_reclamation('true');
```

last_queried( )

Use this method to retrieve the time when the associated record was last queried. This is a read-only attribute.

Parameter

None

Returns

The method returns the attribute value. The number of seconds that have elapsed since January 1st, 1970 UTC.

Example

```
#Get last_queried
my $last_queried = $bindtxt->last_queried();
```

name( )

Use this method to set or retrieve the name.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

The attribute value can be in unicode format.

Parameter

The name of the DNS TXT object in Fully Qualified Domain Name (FQDN) format. The name can have a maximum of 256 bytes.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get name
my $name = $bind_txt->name();
#Modify name
$bind_txt->name("new.domain.com");
```

reclaimable( )

Use this method to retrieve the flag that indicates whether the record is reclaimable or not.

Parameter

None

Returns

The method returns the attribute value.

Example

```
#Get reclaimable
my $reclaimable = $object->reclaimable();
```

text( )

Use this method to set or retrieve the text.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

Text that you want to associate with the record. It can contain up to 255 bytes.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get name
my $name = $bind_txt->text();
#Modify name
$bind_txt->text("this is new text string");
```

ttl( )

Use this method to set or retrieve the Time to Live (TTL) value.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

The default value is undefined which indicates that the record inherits the TTL value of the zone.

Specify a TTL value to override the TTL value at the zone level.

Parameter

A 32-bit integer (range from 0 to 4294967295) that represents the duration in seconds that the record is cached. Zero indicates that the record should not be cached.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get ttl
my $ttl = $bind_txt->ttl();
#Modify ttl
$bind_txt->ttl(1800);
#Un-override ttl
$bind_txt->ttl(undef);
```

views( )

Use this method to set or retrieve the view of a TXT record.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

The default value is the "default" view, which means the TXT record is located under the default view.

Parameter

Array reference of defined Infoblox::DNS::View objects.

Note that the array size must be 1.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get views
my $ref_views = $bind_txt->views();
#Modify views, list of Infoblox::DNS::View objects
$bind_txt->views([$view1]);
```

zone( )

Use this method to retrieve the zone name of a DNS TXT record. This method is read-only and cannot be set.

Parameter

None

Returns

Returns the attribute value.

Example

```
# Get zone
my $zone = $bind_txt->zone();
```

SAMPLE CODE

The following sample code demonstrates the different functions that can be applied to an object, such as add, search, modify, and remove. This sample also includes error handling for the operations.

#Preparation prior to a DNS TXT record insertion

```
#PROGRAM STARTS: Include all the modules that will be used
use strict;
use Infoblox;

#Create a session to the Infoblox appliance
```

```
my $session = Infoblox::Session->new(
               master   => "192.168.1.2", #appliance host ip
               username => "admin",       #appliance user login
               password => "infoblox"     #appliance password
               );

unless ($session) {
       die("Construct session failed: ",
               Infoblox::status_code() . ":" . Infoblox::status_detail());
}
print "Session created successfully\n";

#Create the zone prior to a TXT record insertion
my $zone = Infoblox::DNS::Zone->new(name => "domain.com");
unless ($zone) {
       die("Construct zone failed: ",
               Infoblox::status_code() . ":" . Infoblox::status_detail());
}
print "Zone object created successfully\n";

#Verify if the zone exists
my $object = $session->get(object => "Infoblox::DNS::Zone", name => "domain.com");
unless ($object) {
       print "Zone does not exist on server, safe to add the zone\n";
       $session->add($zone)
               or die("Add zone failed: ",
                               $session->status_code() . ":" . $session->status_detail());
}
print "Zone added successfully\n";
```

#Create a DNS TXT record

```
my $bind_txt = Infoblox::DNS::Record::TXT->new(
               name    => "txt.domain.com",
               comment => "add TXT txt.domain.com",
               text    => "this is text string",
               );

unless ($bind_txt) {
       die("Construct DNS record TXT failed: ",
               Infoblox::status_code() . ":" . Infoblox::status_detail());
}

#Add the DNS TXT record object to the Infoblox appliance through a session
$session->add($bind_txt)
       or die("Add record TXT failed: ",
                       $session->status_code() . ":" . $session->status_detail());
print "DNS TXT object added to server successfully\n";
```

#Search for a specific DNS TXT record

```
#Search all TXT records in the zone that matches "domain.com"
my @retrieved_objs = $session->search(
               object => "Infoblox::DNS::Record::TXT",
               name   => 'domain\.com'
               );
my $object = $retrieved_objs[0];

unless ($object) {
       die("Search record TXT failed: ",
               $session->status_code() . ":" . $session->status_detail());
}
print "Search DNS TXT object found at least 1 matching entry\n";

#Search all TXT records with names that start with "txt" and end with "domain.com"
my @retrieved_objs = $session->search(
               object => "Infoblox::DNS::Record::TXT",
               name   => '^txt.*\.domain\.com\$'
               );
my $object = $retrieved_objs[0];

unless ($object) {
       die("Search record TXT failed: ",
               $session->status_code() . ":" . $session->status_detail());
}
print "Search DNS TXT object using regexp found at least 1 matching entry\n";
```

#Get and modify a DNS TXT record

```
#Get a TXT record through the session
my @retrieved_objs = $session->get(
               object => "Infoblox::DNS::Record::TXT",
               name   => "txt.domain.com"
               );
my $object = $retrieved_objs[0];

unless ($object) {
       die("Get record TXT failed: ",
               $session->status_code() . ":" . $session->status_detail());
}
print "Get DNS TXT object found at least 1 matching entry\n";

#Modify one of the attributes of the specified TXT record
$object->text("this is new text string");

#Apply the changes
$session->modify($object)
       or die("Modify record TXT failed: ",
               $session->status_code() . ":" . $session->status_detail());
print "DNS TXT object modified successfully \n";
```

#Remove a DNS TXT record

```
#Get TXT record through the session
my @retrieved_objs = $session->get(
    object => "Infoblox::DNS::Record::TXT",
    name   => "txt.domain.com"
);
my $object = $retrieved_objs[0];
unless ($object) {
    die("Get record TXT failed: ",
        $session->status_code() . ":" . $session->status_detail());
}
print "Get DNS TXT object found at least 1 matching entry\n";

#Submit the object for removal
$session->remove($object)
       or die("Remove record TXT failed: ",
               $session->status_code() . ":" . $session->status_detail());
print "DNS TXT object removed successfully \n";

####PROGRAM ENDS####
```

AUTHOR

Infoblox Inc. http://www.infoblox.com/

SEE ALSO

Infoblox::Session->add(), Infoblox::Session->get(), Infoblox::Session->modify(), Infoblox::Session->remove(), Infoblox::Session->search(), Infoblox::Session, Infoblox::DNS::View, Infoblox::DNS::Zone

---

COPYRIGHT

Copyright (c) 2017 Infoblox Inc.

Infoblox::DNS::RootNameServer - Custom root name server object.

- NAME
- DESCRIPTION
- CONSTRUCTOR
- MODULE METHODS
  - Infoblox::DNS::View->custom_root_name_servers( )
  - Infoblox::Grid::Member::DNS->custom_root_name_servers( )
- METHODS
  - host_name( )
  - ipv4addr( )
  - ipv6addr( )
- SAMPLE CODE
- AUTHOR
- SEE ALSO
- COPYRIGHT

NAME

Infoblox::DNS::RootNameServer - Custom root name server object.

DESCRIPTION

The custom root name server provides host name and IP address information of the internal root name server. When the Infoblox appliance receives a recursive query for DNS data that it does not have, it uses the internal root name server specified instead of using the Internet root name server.

CONSTRUCTOR

```
my $rootns = Infoblox::DNS::RootNameServer->new(
             host_name => $fqdn,      #Required
             ipv4addr  => $ipv4addr   #Required if $ipv6addr is not present
             ipv6addr  => $ipv4addr   #Required if $ipv4addr is not present
);
```

MODULE METHODS

You can apply the following functions to a custom root name server object:

Infoblox::DNS::View->custom_root_name_servers( )

Use this function to specify custom root name servers at the view level on the Infoblox appliance. See Infoblox::DNS::View->custom_root_name_servers() for parameters and return values.

Example

```
#Construct a custom root name server object
my $rootns1 = Infoblox::DNS::RootNameServer->new(
             host_name => "rns1.test.com",
             ipv4addr  => "4.4.4.4"
);

#Construct a custom root name server object
my $rootns2 = Infoblox::DNS::RootNameServer->new(
             host_name => "rns3.test.com",
             ipv6addr  => "6666::"
);

# Configure custom root name server on the View object
my $response = $View->custom_root_name_servers([$rootns1, $rootns2]);
```

Infoblox::Grid::Member::DNS->custom_root_name_servers( )

Use this function to specify custom root name servers at the member level on the Infoblox appliance. See Infoblox::Grid::Member::DNS->custom_root_name_servers() for parameters and return values.

Example

```
#Construct a custom root name server object
my $rootns1 = Infoblox::DNS::RootNameServer->new(
```

```
                host_name => "rns1.test.com",
                ipv4addr  => "4.4.4.4"
);

#Construct a custom root name server object
my $rootns2 = Infoblox::DNS::RootNameServer->new(
                host_name => "rns3.test.com",
                ipv6addr  => "6666::"
);

# Configure custom root name server on the member DNS object
my $response = $Member_DNS->custom_root_name_servers([$rootns1, $rootns2]);
```

METHODS

This section describes all the methods you can use to configure and retrieve the attribute values of a custom root name server.

host_name( )

Use this method to set or retrieve the host name of the custom root name server.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

Specify the host name in the FQDN (Fully Qualified Domain Name) format. The FQDN consists of the host name followed by the domain name (example: abc.com). A host name can have a maximum of 256 characters.

Returns

If you specify a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you do not specify a parameter, the method returns the attribute value.

Example

```
#Get host name
my $host_name = $rootns->host_name();
#Modify host name
$rootns->host_name("qa.infoblox.com");
```

ipv4addr( )

Use this method to set or retrieve the IPv4 address of the custom root name server. It is required only when IPv6 is not specified.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

An IPv4 address is a 32-bit number in dotted decimal notation. It consists of four 8-bit groups of decimal digits separated by decimal points (example: 192. 168.1.2).

Returns

If you specify a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you do not specify a parameter, the method returns the attribute value.

Example

```
#Get ipv4addr
my $ipv4addr = $rootns->ipv4addr();
#Modify ipv4addr
$rootns->ipv4addr("2.2.2.2");
```

ipv6addr( )

Use this method to set or retrieve the IPv6 address of the custom root name server. It is required only when IPv4 is not specified.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

An IPv6 address is a 128-bit number in colon hexadecimal notation. It consists of eight groups of four hexadecimal digits separated by colons (example: 12ab:0000:0000:0123:4567:89ab:0000:cdef).

Returns

If you specify a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you do not specify a parameter, the method returns the attribute value.

Example

```
# Getting ipv6addr
my $ipv6addr = $rootns->ipv6addr();
# Modifying ipv6addr
$rootns->ipv6addr("12ab::345");
```

SAMPLE CODE

The following sample code demonstrates the different functions that can be applied to a custom root name server object such as add, modify, and remove. Also, this sample includes error handling for the operations.

#Preparation prior to a custom root name server object insertion

```
#PROGRAM STARTS: Include all the modules that will be used
use strict;
use Infoblox;

#Create a session to the Infoblox appliance
my $session = Infoblox::Session->new(
               master   => "192.168.1.2", #appliance host ip
               username => "admin",       #appliance user login
               password => "infoblox"     #appliance password
);
unless ($session) {
       die("Construct session failed: ",
            Infoblox::status_code() . ":" . Infoblox::status_detail());
}
print "Session created successfully\n";
```

#Create a custom root name server object

```
my $rootns = Infoblox::DNS::RootNameServer->new(
             host_name => "rns1.test.com",
             ipv4addr  => "4.4.4.4"
);
unless($rootns) {
       die("Construct custom root name server object failed: ",
            Infoblox::status_code() . ":" . Infoblox::status_detail());
}
print "Root name server object created successfully\n";
```

#Get the Member DNS object and add custom root name server to it

```
#Get the Member DNS object
my @retrieved_objs = $session->get(
    object => "Infoblox::Grid::Member::DNS",
    name   => "infoblox.localdomain"
);
my $object = $retrieved_objs[0];
unless ($object) {
       die("Get Member DNS object failed: ",
            $session->status_code() . ":" . $session->status_detail());
}
print "Get Member DNS object found at least 1 matching entry\n";

#Apply the changes to the Grid DNS object
$object->custom_root_name_servers([$rootns]);

#Apply the changes
$session->modify($object)
    or die("Modify Member DNS object failed: ",
            $session->status_code() . ":" . $session->status_detail());
print "Member DNS object with custom root name server updated to Infoblox appliance successfully\n";
```

#Modify custom root name server

```
#Modify existing custom root name server object

#Modifying the host name of the custom root name server object.
$rootns->host_name("eng.infoblox.com");

#Modifying the IPV4 address of the custom root name server object.
```

```
$rootns->ipv4addr("3.3.3.3");

#Apply changes to the Member DNS object.
$object->custom_root_name_servers([$rootns]);

#Update Member DNS object through the Infoblox session.
$session->modify($object)
            or die("modify Member DNS object failed: ",
                $session->status_code() . ":" . $session->status_detail());
       print "Member DNS object with modified custom root name server updated to Infoblox appliance
successfully\n";
```

#Remove custom root name server

```
#Apply changes to the Member DNS object to use default internet root name server.
$object->custom_root_name_servers(undef);

#Update Member DNS object through the Infoblox session.
$session->modify($object)
            or die("Remove custom root name server from Member DNS object failed: ",
                $session->status_code() . ":" . $session->status_detail());
       print "Removed custom root name server from Member DNS object successfully\n";

####PROGRAM ENDS####
```

AUTHOR

Infoblox Inc. http://www.infoblox.com/

SEE ALSO

Infoblox::Session, Infoblox::Grid::Member::DNS, Infoblox::DNS::View

---

COPYRIGHT

Copyright (c) 2017 Infoblox Inc.

Infoblox::DNS::RPZRecord::A - Response Policy Zone Rule object.

- NAME
- DESCRIPTION
- CONSTRUCTOR
- SESSION METHODS
  - Infoblox::Session->add( )
  - Infoblox::Session->get( )
  - Infoblox::Session->modify( )
  - Infoblox::Session->remove( )
  - Infoblox::Session->search( )
- METHODS
  - comment( )
  - disable( )
  - extattrs( )
  - extensible_attributes( )
  - ipv4addr( )
  - name( )
  - ttl( )
  - views( )
  - zone( )
  - rp_zone( )
- SAMPLE CODE
- AUTHOR
- SEE ALSO
- COPYRIGHT

NAME

Infoblox::DNS::RPZRecord::A - Response Policy Zone (RPZ) Substitute (A Record) Rule object.

DESCRIPTION

An RPZ Substitute (A Record) Rule maps a domain name to a substitute IPv4 address. To define a specific name-to-address mapping, add an Substitute (A Record) Rule to a previously defined Response Policy Zone.

This record represents the substitution rule for DNS A records.

Note that you must specify only one view for the attribute "views".

CONSTRUCTOR

```
 my $rpz_a = Infoblox::DNS::RPZRecord::A->new (
     ipv4addr => $ipv4addr,          #Required
     name     => $fqdn,              #Required
     rp_zone  => $rp_zone            #Required
     comment  => $string,            #Optional / Default is empty
     disable  => "true" | "false"    #Optional / Default "false"
     extattrs              => { $string => $extattr, ... },      #Optional / Default is undefined
     extensible_attributes => { $string => $string | $num, $string => [ $string | $num, ... ], ... },
#Optional / Default is undefined
     last_discovered  => $string,    #Optional / Default is empty
     ttl      => $num | undef,       #Optional / Default is undefined
     views    => [$View1],           #Optional / Default is "default" view
 );
```

You cannot set both extattrs and extensible_attributes attributes at the same time.

SESSION METHODS

This section describes all the methods in Infoblox::Session module that you can apply to a Substitute (A Record) Rule object.

Infoblox::Session->add( )

Use this method to add an object to the Infoblox appliance. See Infoblox::Session->add() for parameters and return values.

Example

```
#Construct an object
my $rpz_a = Infoblox::DNS::RPZRecord::A->new (
```

```
    ipv4addr => "1.1.1.1",
    name     => "bind_a.rpz.com",
    rp_zone  => $response_policy_zone1,
    comment  => "this is a demo rule",
    disable  => "true",
    ttl      => "33",
    views    => [$view1]  );
# Submit for addition
my $response = $session->add( $rpz_a );
```

Infoblox::Session->get( )

Use this method to retrieve all the matching objects from the Infoblox appliance. See Infoblox::Session->get() for parameters and return values.

Key References

```
Apply the following attributes to get a specific Substitute (A Record) Rule object:

 name - Optional. A host name in string format.
 view - Optional. The DNS view in which the Substitute (A Record) Rule is located. By default, all
DNS views are searched. But if you omit this attribute and specify a zone, the appliance searches the
'default' view only.
 zone - Optional. A zone name in FQDN format.
 ipv4addr - Optional. The address of the host record.
 extattrs     - Optional. A hash reference containing extensible attributes.
 extensible_attributes - Optional. A hash reference containing extensible attributes.
```

Example

```
my @retrieved_objs = $session->get(
    object => "Infoblox::DNS::RPZRecord::A",
    name   => "bind_a.rpz.com",
    view   => "default" );

#get all Substitute (A Record) Rule with the extensible attribute 'Site'
my @retrieved_objs = $session->get(
   object => "Infoblox::DNS::RPZRecord::A",
   extensible_attributes => { 'Site' => 'Santa Clara' });
```

Infoblox::Session->modify( )

Use this method to modify an object in the Infoblox appliance. See Infoblox::Session->modify() for parameters and return values.

Example

```
# Use method to modify the comment.
$rpz_a->comment("this is a modified comment");
# Submit modification
my $response = $session->modify( $rpz_a );
```

Infoblox::Session->remove( )

Use this method to submit the removal of an object from the Infoblox appliance. See Infoblox::Session->remove() for parameters and return values.

To remove a specifc object, first use `get()` or `search()` to retrieve the specific object, and then submit this object for removal.

Example

```
# Get the objects with the same name
my @retrieved_objs = $session->get(
    object => "Infoblox::DNS::RPZRecord::A",
    name   => "bind_a.rpz.com",
    view   => "default" );
# Find the desired object from the retrieved list.
my $desired_a = $retrieved_objs[0];
# Submit for removal
my $response = $session->remove( $desired_a );
```

Infoblox::Session->search( )

Use this method to search for Substitute (A Record) Rule objects in the Infoblox appliance. See Infoblox::Session->search() for parameters and return values.

Key References

```
Apply the following attributes to search for a Substitute (A Record) Rule object:
```

```
  name     - Optional. A host name in string format (regular expression).
  view     - Optional. The DNS view in which the Substitute (A Record) Rule is located. By default,
all DNS views are searched. But if you omit this attribute and specify a zone, the appliance searches
the 'default' view only.
  zone     - Optional. A zone name in FQDN format.
  comment  - Optional. A comment in string format (regular expression).
  ipv4addr - Optional. The address of the host record (regular expression).
  extattrs     - Optional. A hash reference containing extensible attributes.
  extensible_attributes - Optional. A hash reference containing extensible attributes.
```

For more information about searching extensible attributes, see Infoblox::Grid::ExtensibleAttributeDef/Searching Extensible Attributes.

Example

```
# search for all Substitute (A Record) Rule objects that match "rpz.com" in the default DNS view
my @retrieved_objs = $session->search(
    object => "Infoblox::DNS::RPZRecord::A",
    name   => 'rpz.com',
    view   => "default" );

# search for all Substitute (A Record) Rule in the "rpz.com" zone of the default view
my @retrieved_objs = $session->search(
    object => "Infoblox::DNS::RPZRecord::A",
    zone   => "rpz.com",
    view   => "default" );

#get all Substitute (A Record) Rules with the extensible attribute 'Site'
my @retrieved_objs = $session->search(
  object => "Infoblox::DNS::RPZRecord::A",
  extensible_attributes => { 'Site' => 'Santa Clara' });
```

METHODS

This section describes all the methods that you can use to configure and retrieve the attribute values of an DNS Substitute (A Record) Rule.

comment( )

Use this method to set or retrieve the descriptive comment.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

Desired comment in string format with a maximum of 256 bytes.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get comment
my $comment = $rpz_a->comment();
#Modify comment
$rpz_a->comment("Modifying the Substitute (A Record) Rule comment");
```

disable( )

Use this method to set or retrieve the disable flag of a DNS record.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

The default value for this field is false. The DNS record is enabled.

Parameter

Specify "true" to set the disable flag or "false" to deactivate/unset it.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get disable
my $disable = $rpz_a->disable();
#Modify disable
$rpz_a->disable("true");
```

extattrs( )

Use this method to set or retrieve the extensible attributes associated with a Substitute (A Record) Rule object.

Parameter

Valid value is a hash reference containing the names of extensible attributes and their associated values (Infoblox::Grid::Extattr objects).

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get extattrs
my $ref_extattrs = $rpz_a->extattrs();
#Modify extattrs
$rpz_a->extattrs({ 'Site' => $extattr1, 'Administrator' => $extattr2 });
```

extensible_attributes( )

Use this method to set or retrieve the extensible attributes associated with a Substitute (A Record) Rule.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

For valid values for extensible attributes, see Infoblox::Grid::ExtensibleAttributeDef/Extensible Attribute Values.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get extensible attributes
my $ref_extensible_attributes = $rpz_a->extensible_attributes();
#Modify extensible attributes
$rpz_a->extensible_attributes({ 'Site' => 'Santa Clara', 'Administrator' => [ 'Peter', 'Tom' ] });
```

ipv4addr( )

Use this method to set or retrieve the IPv4 address.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

An IPv4 address is a 32-bit number in dotted decimal notation. It consists of four 8-bit groups of decimal digits separated by decimal points (example: 192.168.1.2).

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get ipv4addr
my $ipv4addr = $rpz_a->ipv4addr();
#Modify ipv4addr
$rpz_a->ipv4addr("2.2.2.2");
```

name( )

Use this method to set or retrieve the host name.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

Hostname in FQDN (Fully Qualified Domain Name) format. The FQDN consists of the hostname followed by the domain name (example: abc.com). A hostname can have a maximum of 256 bytes.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get name
my $name = $rpz_a->name();
#Modify name
$rpz_a->name("bind_a.rpz.com");
```

ttl( )

Use this method to set or retrieve the Time to Live (TTL) value.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

The default value is undefined which indicates that the record inherits the TTL value of the zone.

Specify a TTL value to override the TTL value at the zone level.

Parameter

A 32-bit integer (range from 0 to 4294967295) that represents the duration in seconds that the record is cached. Zero indicates that the record should not be cached.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get ttl
my $ttl = $rpz_a->ttl();
#Modify ttl
$rpz_a->ttl(1800);
#Un-override ttl
$rpz_a->ttl(undef);
```

views( )

Use this method to set or retrieve the view of the Substitute (A Record) Rule.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

The default value is the "default" view, which means the Substitute (A Record) Rule is located under the default view.

Parameter

An array reference of defined Infoblox::DNS::View objects.

Note that the array size must be 1.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get views
my $ref_views = $rpz_a->views();
#Modify views, an array of Infoblox::DNS::View objects
$rpz_a->views([$view1]);
```

zone( )

Use this method to retrieve the zone name of a Substitute (A Record) Rule. This method is read-only and cannot be set.

Parameter

None

Returns

Returns the attribute value.

Example

```
# Get zone
my $zone = $rpz_a->zone();
```

rp_zone( )

Use this method to set or retrieve the zone object of a Substitute (A Record) Rule.

Parameter

An Infoblox::DNS::Zone object.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
# Get rp_zone
my $rp_zone = $rpz_a->rp_zone();
#Modify rp_zone, reference of Infoblox::DNS::Zone object
$rpz_a->rp_zone($response_policy_zone);
```

SAMPLE CODE

The following sample code demonstrates the different functions that can be applied to an object, such as add, search, modify, and remove. This sample also includes error handling for the operations.

#Preparation prior to a Substitute (A Record) Rule insertion

```
#PROGRAM STARTS: Include all the modules that will be used
use strict;
use Infoblox;

#Create a session to the Infoblox Appliance
my $session = Infoblox::Session->new(
    master   => "192.168.1.2",
    username => "admin",
    password => "infoblox"
);
unless ($session) {
   die("Construct session failed: ",
       $session->status_code() . ":" . $session->status_detail());
}
print "Session created successfully\n";

#Create the RPZ zone prior to an Substitute (A Record) Rule insertion
my $zone = Infoblox::DNS::Zone->new(name => "rpz.com",
                                    rpz_policy => "GIVEN");
unless ($zone) {
die("Construct zone failed: ",
    Infoblox::status_code() . ":" . Infoblox::status_detail());
}
print "Zone object created successfully\n";

#Verify if the zone exists
my $object = $session->get(object => "Infoblox::DNS::Zone", name => "rpz.com");
unless ($object) {
print "Zone does not exist on server, safe to add the zone\n";
$session->add($zone)
   or die("Add zone failed: ",
          $session->status_code() . ":" . $session->status_detail());
}
```

#Create a Substitute (A Record) Rule

```
#Construct a Substitute (A Record) Rule object
my $rpz_a = Infoblox::DNS::RPZRecord::A->new(
 name     => "bind_a.rpz.com",
 comment  => " this is a demo bind_a record ",
 ipv4addr => "6.6.6.6",
 rp_zone  => $zone,
);
unless ($rpz_a) {
die("Construct Substitute (A Record) Rule failed: ",
    Infoblox::status_code() . ":" . Infoblox::status_detail());
}
print "Substitute (A Record) Rule object created successfully\n";

#Add the Substitute (A Record) Rule object to Infoblox Appliance through a session
$session->add($rpz_a)
 or die("Add Substitute (A Record) Rule failed: ",
        $session->status_code() . ":" . $session->status_detail());
print "Substitute (A Record) Rule object added to server successfully\n";
```

#Search for a specific Substitute (A Record) Rule

```
#Search all Substitute (A Record) Rules that match "rpz.com"
my @retrieved_objs = $session->search(
 object => "Infoblox::DNS::RPZRecord::A",
 name   => 'rpz\.com'
);
my $object = $retrieved_objs[0];
unless ($object) {
 die("Search Substitute (A Record) Rule failed: ",
     $session->status_code() . ":" . $session->status_detail());
}
print "Search Substitute (A Record) Rule object found at least 1 matching entry\n";

#Search all Substitute (A Record) Rules that start with "bind" and end with ".rpz.com"
my @retrieved_objs = $session->search(
 object => "Infoblox::DNS::RPZRecord::A",
 name   => '^bind.*\.rpz\.com$'
);
my $object = $retrieved_objs[0];
unless ($object) {
 die("Search Substitute (A Record) Rule failed: ",
     $session->status_code() . ":" . $session->status_detail());
}
print "Search Substitute (A Record) Rule object using regexp found at least 1 matching entry\n";
```

#Get and modify a Substitute (A Record) Rule

```
#Get Substitute (A Record) Rule through the session
my @retrieved_objs = $session->get(
 object => "Infoblox::DNS::RPZRecord::A",
 name   => "bind_a.rpz.com"
);
my $object = $retrieved_objs[0];
unless ($object) {
 die("Get Substitute (A Record) Rule failed: ",
     $session->status_code() . ":" . $session->status_detail());
}
print "Get Substitute (A Record) Rule object found at least 1 matching entry\n";

#Modify one of the attributes of the specified Substitute (A Record) Rule
$object->ipv4addr("3.3.3.3");

#Apply the changes
$session->modify($object)
 or die("Modify Substitute (A Record) Rule failed: ",
        $session->status_code() . ":" . $session->status_detail());
print "Substitute (A Record) Rule object modified successfully \n";
```

#Remove a Substitute (A Record) Rule

```
#Get Substitute (A Record) Rule through the session
my @retrieved_objs = $session->get(
 object => "Infoblox::DNS::RPZRecord::A",
 name   => "bind_a.rpz.com"
);
```

```
my $object = $retrieved_objs[0];
unless ($object) {
 die("Get Substitute (A Record) Rule failed: ",
     $session->status_code() . ":" . $session->status_detail());
}
print "Get Substitute (A Record) Rule object found at least 1 matching entry\n";

#Submit the object for removal
$session->remove($object)
 or die("Remove Substitute (A Record) Rule failed: ",
     $session->status_code() . ":" . $session->status_detail());
print "Substitute (A Record) Rule object removed successfully \n";

####PROGRAM ENDS####
```

AUTHOR

Infoblox Inc. http://www.infoblox.com/

SEE ALSO

Infoblox::DNS::View, Infoblox::DNS::Zone, Infoblox::Session, Infoblox::Session->get(), Infoblox::Session->search(), Infoblox::Session->add(), Infoblox::Session->remove(), Infoblox::Session->modify()

COPYRIGHT

Copyright (c) 2017 Infoblox Inc.

Infoblox::DNS::RPZRecord::AAAA - Response Policy Zone Rule object.

- NAME
- DESCRIPTION
- CONSTRUCTOR
- SESSION METHODS
  - Infoblox::Session->add( )
  - Infoblox::Session->get( )
  - Infoblox::Session->modify( )
  - Infoblox::Session->remove( )
  - Infoblox::Session->search( )
- METHODS
  - comment( )
  - disable( )
  - extattrs( )
  - extensible_attributes( )
  - ipv6addr( )
  - name( )
  - ttl( )
  - views( )
  - zone( )
  - rp_zone( )
- SAMPLE CODE
- AUTHOR
- SEE ALSO
- COPYRIGHT

NAME

Infoblox::DNS::RPZRecord::AAAA - Response Policy Zone (RPZ) Substitute (AAAA Record) Rule object.

DESCRIPTION

An RPZ Substitute (AAAA Record) Rule, maps a domain name to a substitute IPv6 address. To define a specific name-to-address mapping, add an Substitute (AAAA Record) Rule to a previously defined Response Policy Zone.

This record represents the substitution rule for DNS AAAA records.

Note that you must specify only one view for the attribute "views".

CONSTRUCTOR

```
 my $rpz_aaaa = Infoblox::DNS::RPZRecord::AAAA->new (
     ipv6addr => $ipv6addr,          #Required
     name     => $fqdn,              #Required
     rp_zone  => $rp_zone,           #Required
     comment  => $string,            #Optional / Default is empty
     extattrs              => { $string => $extattr, ... },      #Optional / Default is undefined
     extensible_attributes => { $string => $string | $num, $string => [ $string | $num, ... ], ... },
#Optional / Default is undefined
     disable  => "true" | "false"    #Optional / Default "false"
     ttl      => $num | undef,       #Optional / Default is undefined
     views    => [$View1],           #Optional / Default is "default" view
 );
```

You cannot set both extattrs and extensible_attributes attributes at the same time.

SESSION METHODS

This section describes all the methods in Infoblox::Session module that you can apply to a Substitute (AAAA Record) Rule object.

Infoblox::Session->add( )

Use this method to add an Substitute (AAAA Record) Rule object to the Infoblox appliance. See Infoblox::Session->add() for parameters and return values.

Example

```
#Construct an object
my $rpz_aaaa = Infoblox::DNS::RPZRecord::AAAA->new(
```

```
    name     => "aaaa1.rpz.com",
    comment  => "add a Substitute (AAAA Record) Rule aaaa1.rpz.com",
    ipv6addr => "aaaa:1111:bbbb:2222:cccc:3333:dddd:4444",
    rp_zone  => $response_policy_zone,
    disable  => "true",
    ttl      => "33",
    views    => [$view1]  );
# Submit for addition
my $response = $session->add( $rpz_aaaa );
```

Infoblox::Session->get( )

Use this method to retrieve all the matching objects from the Infoblox appliance. See Infoblox::Session->get() for parameters and return values.

Key References

```
Apply the following attributes to get a specific Substitute (AAAA Record) Rule object:

name                                - Optional. A host name in string format.
view                                - Optional. The DNS view in which the host record is located.
By default, all DNS views are searched. But if you omit this attribute and specify a zone, the
appliance searches the 'default' view only.
zone                                - Optional. A zone name in FQDN format.
ipv6addr                            - Optional. The address of the Substitute (AAAA Record) Rule.
extattrs                            - Optional. A hash reference containing extensible attributes.
extensible_attributes               - Optional. A hash reference containing extensible attributes.
```

Example

```
my @retrieved_objs = $session->get(
                     object => "Infoblox::DNS::RPZRecord::AAAA",
                     name   => "aaaa1.rpz.com",
                     view   => "default" );

# get all the Substitute (AAAA Record) Rules in the "rpz.com" zone of the default view
my @retrieved_objs = $session->get(
                     object => "Infoblox::DNS::RPZRecord::AAAA",
                     zone   => "rpz.com",
                     view   => "default" );

#get all Substitute (AAAA Record) Rule recods with the extensible attribute 'Site'
my @retrieved_objs = $session->get(
   object => "Infoblox::DNS::RPZRecord::AAAA",
   extensible_attributes => { 'Site' => 'Santa Clara' });
```

Infoblox::Session->modify( )

Use this method to modify a Substitute (AAAA Record) Rule object in the Infoblox appliance. See Infoblox::Session->modify() for parameters and return values.

Example

```
# Use method to modify the comment.
$rpz_aaaa->comment("this is a modified comment");
# Submit modification
my $response = $session->modify( $rpz_aaaa );
```

Infoblox::Session->remove( )

Use this method to remove an Substitute (AAAA Record) Rule object from the Infoblox appliance. See Infoblox::Session->remove() for parameters and return values.

To remove a specifc object, first use `get()` or `search()` to retrieve the specific Substitute (AAAA Record) Rule object, and then submit this object for removal.

Example

```
# Get the objects with the same name
my @retrieved_objs = $session->get(
    object => "Infoblox::DNS::RPZRecord::AAAA",
    name   => "aaaa1.rpz.com",
    view   => "default" );
# Find the desired object from the retrieved list.
my $desired_aaaa = $retrieved_objs[0];
# Submit for removal
my $response = $session->remove( $desired_aaaa );
```

Infoblox::Session->search( )

Use this method to search for Substitute (AAAA Record) Rule objects in the Infoblox appliance. See Infoblox::Session->search() for parameters and return values.

Key References

```
 Apply the following attributes, to search for a Substitute (AAAA Record) Rule object:

  name                                  - Optional. A host name in string format (regular
expression).
  view                                  - Optional. The DNS view in which the Substitute (AAAA
Record) Rule is located. By default, all DNS views are searched. But if you omit this attribute and
specify a zone, the appliance searches the 'default' view only.
  zone                                  - Optional. A zone name in FQDN format.
  comment                               - Optional. Comment in string format (regular expression).
  ipv6addr                              - Optional. The address of the Substitute (AAAA Record) Rule
(regular expression).
  extattrs                              - Optional. A hash reference containing extensible
attributes.
  extensible_attributes                 - Optional. A hash reference containing extensible
attributes.
```

For more information about searching extensible attributes, see Infoblox::Grid::ExtensibleAttributeDef/Searching Extensible Attributes.

Example

```
# search for all Substitute (AAAA Record) Rule objects that match "rpz.com" in the default DNS view
my @retrieved_objs = $session->search(
    object  => "Infoblox::DNS::RPZRecord::AAAA",
    name    => 'rpz\.com',
    view    => "default",
    comment => ".*modified comment" );

# search for all Substitute (AAAA Record) Rules in zone "rpz.com" of default view
my @retrieved_objs = $session->search(
                     object => "Infoblox::DNS::RPZRecord::AAAA",
                     zone   => "rpz.com",
                     view   => "default" );

# search all Substitute (AAAA Record) Rule recods with the extensible attribute 'Site'
my @retrieved_objs = $session->search(
   object => "Infoblox::DNS::RPZRecord::AAAA",
   extensible_attributes => { 'Site' => 'Santa Clara' });
```

METHODS

This section describes all the methods that you can use to configure and retrieve the attribute values of an Substitute (AAAA Record) Rule.

comment( )

Use this method to set or retrieve the descriptive comment.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

Desired comment in string format with a maximum of 256 bytes.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get comment
my $comment = $rpz_aaaa->comment();
#Modify comment
$rpz_aaaa->comment("Modifying the Substitute (AAAA Record) Rule comment");
```

disable( )

Use this method to set or retrieve the disable flag of a DNS record.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

The default value for this field is false. The DNS record is enabled.

Parameter

Specify "true" to set the disable flag or "false" to deactivate/unset it.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get disable
my $disable = $rpz_aaaa->disable();
#Modify disable
$rpz_aaaa->disable("true");
```

extattrs( )

Use this method to set or retrieve the extensible attributes associated with a Substitute (AAAA Record) Rule object.

Parameter

Valid value is a hash reference containing the names of extensible attributes and their associated values (Infoblox::Grid::Extattr objects).

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get extattrs
my $ref_extattrs = $rpz_aaaa->extattrs();
#Modify extattrs
$rpz_aaaa->extattrs({ 'Site' => $extattr1, 'Administrator' => $extattr2 });
```

extensible_attributes( )

Use this method to set or retrieve the extensible attributes associated with a Substitute (AAAA Record) Rule.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

For valid values for extensible attributes, see Infoblox::Grid::ExtensibleAttributeDef/Extensible Attribute Values.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get extensible attributes
my $ref_extensible_attributes = $rpz_aaaa->extensible_attributes();
#Modify extensible attributes
$rpz_aaaa->extensible_attributes({ 'Site' => 'Santa Clara', 'Administrator' => [ 'Peter', 'Tom' ]
});
```

ipv6addr( )

Use this method to set or retrieve the IPv6 address.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

An IPv6 address is a 128-bit number in colon hexadecimal notation. It consists of eight 16-bit groups of hexadecimal digits separated by colons (example: 12ab:0000:0000:0123:4567:89ab:0000:cdef).

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get ipv6addr
my ipv6addr = $rpz_aaaa->ipv6addr();
#Modify ipv6addr
$rpz_aaaa->ipv6addr("aaaa:1111:bbbb:2222:cccc:3333:dddd:5555");
```

name( )

Use this method to set or retrieve the host name.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

Hostname in FQDN (Fully Qualified Domain Name) format. The FQDN consists of the hostname followed by the domain name (example: abc.com). A hostname can have a maximum of 256 bytes.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get name
my $name = $rpz_aaaa->name();
#Modify name
$rpz_aaaa-> name("aaaa2.rpz.com");
```

ttl( )

Use this method to set or retrieve the Time to Live (TTL) value.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

The default value is undefined which indicates that the record inherits the TTL value of the zone.

Specify a TTL value to override the TTL value at the zone level.

Parameter

A 32-bit integer (range from 0 to 4294967295) that represents the duration in seconds that the record is cached. Zero indicates that the record should not be cached.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get ttl
my $ttl = $rpz_aaaa->ttl();
#Modify ttl
$rpz_aaaa->ttl(1800);
#Un-override ttl
$rpz_aaaa->ttl(undef);
```

views( )

Use this method to set or retrieve the view of the Substitute (AAAA Record) Rule.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

The default value is the "default" view, which means the Substitute (AAAA Record) Rule is located under the default view.

Parameter

An array reference of defined Infoblox::DNS::View objects.

Note that the array size must be 1.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get views
my $ref_views = $rpz_aaaa->views();
#Modify views, an array of Infoblox::DNS::View objects
$rpz_aaaa->views([$view1]);
```

zone( )

Use this method to retrieve the zone name of a Substitute (AAAA Record) Rule. This method is read-only and cannot be set.

Parameter

None

Returns

Returns the attribute value.

Example

```
# Get zone
my $zone = $rpz_aaaa->zone();
```

rp_zone( )

Use this method to set or retrieve the zone object of a Substitute (AAAA Record) Rule.

Parameter

An Infoblox::DNS::Zone object.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
# Get rp_zone
my $rp_zone = $rpz_aaaa->rp_zone();
#Modify rp_zone, reference of Infoblox::DNS::Zone object
$rpz_aaaa->rp_zone($response_policy_zone);
```

SAMPLE CODE

The following sample code demonstrates the different functions that can be applied to a Substitute (AAAA Record) Rule object, such as add, search, modify, and remove. This sample also includes error handling for the operations.

#Preparation prior to a Substitute (AAAA Record) Rule insertion

```
#PROGRAM STARTS: Include all the modules that will be used

use strict;
use Infoblox;

#Create a session to the Infoblox Appliance
my $session = Infoblox::Session->new(
    master   => "192.168.1.2", #appliance host ip
    username => "admin",       #appliance user login
    password => "infoblox"     #appliance password
);

unless ($session) {
   die("Construct session failed: ",
       Infoblox::status_code() . ":" . Infoblox::status_detail());
}

#Create the Response Policy zone prior to an Substitute (AAAA Record) Rule insertion
my $zone = Infoblox::DNS::Zone->new(name => "rpz.com");

unless ($zone) {
```

```
    die("Construct zone failed: ",
        Infoblox::status_code() . ":" . Infoblox::status_detail());
}

print "Zone object created successfully\n";

#Verify if the zone exists
my $object = $session->get(object => "Infoblox::DNS::Zone",
                           name => "rpz.com");
unless ($object) {
   print "Zone does not exist on server, safe to add the zone\n";
   $session->add($zone)
      or die("Add zone failed: ",
             $session->status_code() . ":" . $session->status_detail());
}
print "Zone added successfully\n";

#Create the zone prior to a CNAME record insertion
my $zone = Infoblox::DNS::Zone->new(name => "rpz.com",
                                    rpz_policy => "GIVEN");
unless ($zone) {
die("Construct zone failed: ",
    Infoblox::status_code() . ":" . Infoblox::status_detail());
}
print "Zone object created successfully\n";

#Verify if the zone exists
my $object = $session->get(object => "Infoblox::DNS::Zone", name => "rpz.com");
unless ($object) {
print "Zone does not exist on server, safe to add the zone\n";
$session->add($zone)
   or die("Add zone failed: ",
          $session->status_code() . ":" . $session->status_detail());
}
```

#Create a Substitute (AAAA Record) Rule

```
#Construct a Substitute (AAAA Record) Rule object
my $rpz_aaaa = Infoblox::DNS::RPZRecord::AAAA->new(
 name     => "aaaa1.rpz.com",
 comment  => "add a Substitute (AAAA Record) Rule aaaa1.rpz.com",
 ipv6addr => "aaaa:1111:bbbb:2222:cccc:3333:dddd:4444",
 rp_zone  => $zone,
);

unless ($rpz_aaaa) {
die("Construct Substitute (AAAA Record) Rule failed: ",
    Infoblox::status_code() . ":" . Infoblox::status_detail());
}
print "Substitute (AAAA Record) Rule object created successfully\n";

#Add the Substitute (AAAA Record) Rule object to Infoblox Appliance through a session
$session->add($rpz_aaaa)

 or die("Add Substitute (AAAA Record) Rule failed: ",
        $session->status_code() . ":" . $session->status_detail());
  print "Substitute (AAAA Record) Rule object added to server successfully\n";
```

#Search for a specific Substitute (AAAA Record) Rule

```
#Search all Substitute (AAAA Record) Rules that match "rpz.com"
my @retrieved_objs = $session->search(
 object => "Infoblox::DNS::RPZRecord::AAAA",
 name   => 'rpz\.com'
);
my $object = $retrieved_objs[0];
unless ($object) {
 die("Search Substitute (AAAA Record) Rule failed: ",
     $session->status_code() . ":" . $session->status_detail());
}
print "Search Substitute (AAAA Record) Rule object found at least 1 matching entry\n";

#Search all Substitute (AAAA Record) Rules that start with "add" in the comment
my @retrieved_objs = $session->search(
 object    => "Infoblox::DNS::RPZRecord::AAAA",
 comment   => "add.*",
);
```

```
my $object = $retrieved_objs[0];
unless ($object) {
 die("Search Substitute (AAAA Record) Rule failed: ",
     $session->status_code() . ":" . $session->status_detail());
}
print "Search Substitute (AAAA Record) Rule object found at least 1 matching entry\n";

#Search all Substitute (AAAA Record) Rule that start with "aaaa1" and end with ".com"
my @retrieved_objs = $session->search(
 object => "Infoblox::DNS::RPZRecord::AAAA",
 name   => 'aaaa1.*\.com',
);
my $object = $retrieved_objs[0];
unless ($object) {
 die("Search Substitute (AAAA Record) Rule failed: ",
     $session->status_code() . ":" . $session->status_detail());
}
print "Search Substitute (AAAA Record) Rule object using regexp found at least 1 matching entry\n";
```

#Get and modify a Substitute (AAAA Record) Rule

```
#Get Substitute (AAAA Record) Rule through the session
my @retrieved_objs = $session->get(
 object => "Infoblox::DNS::RPZRecord::AAAA",
 name   => "aaaa1.rpz.com"
);
my $object = $retrieved_objs[0];

unless ($object) {
 die("Get Substitute (AAAA Record) Rule failed: ",
     $session->status_code() . ":" . $session->status_detail());
}

#Modify one of the attributes of the specified Substitute (AAAA Record) Rule
$object->ipv6addr("aaaa:1111:bbbb:2222:cccc:3333:dddd:eeee");

#Applying the changes
$session->modify($object)

 or die("Modify Substitute (AAAA Record) Rule failed: ",
        $session->status_code() . ":" . $session->status_detail());
  print "Substitute (AAAA Record) Rule object modified successfully \n";
```

#Remove a Substitute (AAAA Record) Rule.

```
#Get Substitute (AAAA Record) Rule through the session
my @retrieved_objs = $session->get(
 object   => "Infoblox::DNS::RPZRecord::AAAA",
 name     => "aaaa1.rpz.com"
);
my $object = $retrieved_objs[0];
unless ($object) {
 die("Get Substitute (AAAA Record) Rule failed: ",
     $session->status_code() . ":" . $session->status_detail());
}

print "Get Substitute (AAAA Record) Rule object found at least 1 matching entry\n";

#Submit the object for removal
$session->remove($object)
 or die("Remove Substitute (AAAA Record) Rule failed: ",
        $session->status_code() . ":" . $session->status_detail());
print "Substitute (AAAA Record) Rule object removed successfully \n";

####PROGRAM ENDS####
```

AUTHOR

Infoblox Inc. http://www.infoblox.com/

SEE ALSO

Infoblox::DNS::View, Infoblox::DNS::Zone, Infoblox::Session, Infoblox::Session->get(), Infoblox::Session->search(), Infoblox::Session->add(), Infoblox::Session->remove(), Infoblox::Session->modify()

COPYRIGHT

Copyright (c) 2017 Infoblox Inc.

Infoblox::DNS::RPZRecord::AAAAIpAddress - Response Policy Zone Rule object.

- NAME
- DESCRIPTION
- CONSTRUCTOR
- SESSION METHODS
  - Infoblox::Session->add( )
  - Infoblox::Session->get( )
  - Infoblox::Session->modify( )
  - Infoblox::Session->remove( )
  - Infoblox::Session->search( )
- METHODS
  - comment( )
  - disable( )
  - extattrs( )
  - extensible_attributes( )
  - ipv6addr( )
  - name( )
  - ttl( )
  - views( )
  - zone( )
  - rp_zone( )
- SAMPLE CODE
- AUTHOR
- SEE ALSO
- COPYRIGHT

NAME

Infoblox::DNS::RPZRecord::AAAAIpAddress - Response Policy Zone (RPZ) Substitute (IPv6 Address) Rule object.

DESCRIPTION

An RPZ Substitute (IPv6 Address) Rule maps an IP address represented by a host name to a substitute IPv6 address. To define a specific address-to-address mapping, add an RPZ Substitute (IPv6 Address) Rule to a previously defined Response Policy Zone.

This record represents the substitution rule for IP trigger policy. It matches IP addresses that would otherwise appear in AAAA record in the "answer" section of DNS response.

You should use this object to create IP address substitution rules instead usage Infoblox::DNS::RPZRecord::CNAME::IpAddress object.

Note that you must specify only one view for the attribute "views".

CONSTRUCTOR

```
 my $rpz_4aip = Infoblox::DNS::RPZRecord::AAAAIpAddress->new (
     ipv6addr => $ipv6addr,          #Required
     name     => $fqdn,              #Required
     rp_zone  => $rp_zone,           #Required
     comment  => $string,            #Optional / Default is empty
     extattrs              => { $string => $extattr, ... },      #Optional / Default is undefined
     extensible_attributes => { $string => $string | $num, $string => [ $string | $num, ... ], ... },
#Optional / Default is undefined
     disable  => "true" | "false"    #Optional / Default "false"
     ttl      => $num | undef,       #Optional / Default is undefined
     views    => [$View1],           #Optional / Default is "default" view
 );
```

You cannot set both extattrs and extensible_attributes attributes at the same time.

SESSION METHODS

This section describes all the methods in Infoblox::Session module that you can apply to a Substitute (IPv6 Address) Rule object.

Infoblox::Session->add( )

Use this method to add an RPZ Substitute (IPv6 Address) Rule object to the Infoblox appliance. See Infoblox::Session->add() for parameters and return values.

Example

```
#Construct an object
my $rpz_4aip = Infoblox::DNS::RPZRecord::AAAAIpAddress->new(
    name     => "2011::/16.rpz.com",
    comment  => "add a Substitute (IPv6 Address) Rule aaaa1.rpz.com",
    ipv6addr => "aaaa:1111:bbbb:2222:cccc:3333:dddd:4444",
    rp_zone  => $response_policy_zone,
    disable  => "true",
    ttl      => "33",
    views    => [$view1]  );
# Submit for addition
my $response = $session->add( $rpz_4aip );
```

Infoblox::Session->get( )

Use this method to retrieve all the matching objects from the Infoblox appliance. See Infoblox::Session->get() for parameters and return values.

Key References

```
Apply the following attributes to get a specific Substitute (IPv6 Address) Rule object:

name                                - Optional. A host name in string format.
view                                - Optional. The DNS view in which the host record is located.
By default, all DNS views are searched. But if you omit this attribute and specify a zone, the
appliance searches the 'default' view only.
zone                                - Optional. A zone name in FQDN format.
ipv6addr                            - Optional. The address of the RPZ Substitute (IPv6 Address)
Rule.
extattrs                            - Optional. A hash reference containing extensible attributes.
extensible_attributes               - Optional. A hash reference containing extensible attributes.
```

Example

```
my @retrieved_objs = $session->get(
                     object => "Infoblox::DNS::RPZRecord::AAAAIpAddress",
                     name   => "2001::1.rpz.com",
                     view   => "default" );

# get all the Substitute (IPv6 Address) Rules in the "rpz.com" zone of the default view
my @retrieved_objs = $session->get(
                     object => "Infoblox::DNS::RPZRecord::AAAAIpAddress",
                     zone   => "rpz.com",
                     view   => "default" );

#get all Substitute (IPv6 Address) Rule recods with the extensible attribute 'Site'
my @retrieved_objs = $session->get(
   object => "Infoblox::DNS::RPZRecord::AAAAIpAddress",
   extensible_attributes => { 'Site' => 'Santa Clara' });
```

Infoblox::Session->modify( )

Use this method to modify an RPZ Substitute (IPv6 Address) Rule object in the Infoblox appliance. See Infoblox::Session->modify() for parameters and return values.

Example

```
# Use method to modify the comment.
$rpz_4aip->comment("this is a modified comment");
# Submit modification
my $response = $session->modify( $rpz_4aip );
```

Infoblox::Session->remove( )

Use this method to remove an RPZ Substitute (IPv6 Address) Rule object from the Infoblox appliance. See Infoblox::Session->remove() for parameters and return values.

To remove a specifc object, first use `get()` or `search()` to retrieve the specific RPZ Substitute (IPv6 Address) Rule object, and then submit this object for removal.

Example

```
# Get the objects with the same name
my @retrieved_objs = $session->get(
    object => "Infoblox::DNS::RPZRecord::AAAAIpAddress",
    name   => "aaaa1.rpz.com",
    view   => "default" );
# Find the desired object from the retrieved list.
my $desired_aaaa = $retrieved_objs[0];
```

```
# Submit for removal
my $response = $session->remove( $desired_aaaa );
```

Infoblox::Session->search( )

Use this method to search for Substitute (IPv6 Address) Rule objects in the Infoblox appliance. See Infoblox::Session->search() for parameters and return values.

Key References

```
Apply the following attributes, to search for a Substitute (IPv6 Address) Rule object:

  name                                  - Optional. A host name in string format (regular
expression).
  view                                  - Optional. The DNS view in which the RPZ Substitute (IPv6
Address) Rule is located. By default, all DNS views are searched. But if you omit this attribute and
specify a zone, the appliance searches the 'default' view only.
  zone                                  - Optional. A zone name in FQDN format.
  comment                               - Optional. Comment in string format (regular expression).
  ipv6addr                              - Optional. The address of the RPZ Substitute (IPv6 Address)
Rule (regular expression).
  extattrs                              - Optional. A hash reference containing extensible
attributes.
  extensible_attributes                 - Optional. A hash reference containing extensible
attributes.
```

For more information about searching extensible attributes, see Infoblox::Grid::ExtensibleAttributeDef/Searching Extensible Attributes.

Example

```
# search for all Substitute (IPv6 Address) Rule objects that match "rpz.com" in the default DNS view
my @retrieved_objs = $session->search(
    object  => "Infoblox::DNS::RPZRecord::AAAAIpAddress",
    name    => 'rpz\.com',
    view    => "default",
    comment => ".*modified comment" );

# search for all Substitute (IPv6 Address) Rules in zone "rpz.com" of default view
my @retrieved_objs = $session->search(
                     object => "Infoblox::DNS::RPZRecord::AAAAIpAddress",
                     zone   => "rpz.com",
                     view   => "default" );

# search all Substitute (IPv6 Address) Rule recods with the extensible attribute 'Site'
my @retrieved_objs = $session->search(
   object => "Infoblox::DNS::RPZRecord::AAAAIpAddress",
   extensible_attributes => { 'Site' => 'Santa Clara' });
```

METHODS

This section describes all the methods that you can use to configure and retrieve the attribute values of an RPZ Substitute (IPv6 Address) Rule.

comment( )

Use this method to set or retrieve the descriptive comment.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

Desired comment in string format with a maximum of 256 bytes.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get comment
my $comment = $rpz_4aip->comment();
#Modify comment
$rpz_4aip->comment("Modifying the Substitute (IPv6 Address) Rule comment");
```

disable( )

Use this method to set or retrieve the disable flag of a DNS record.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

The default value for this field is false. The DNS record is enabled.

Parameter

Specify "true" to set the disable flag or "false" to deactivate/unset it.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get disable
my $disable = $rpz_4aip->disable();
#Modify disable
$rpz_4aip->disable("true");
```

extattrs( )

Use this method to set or retrieve the extensible attributes associated with a Substitute (IPv6 Address) Rule object.

Parameter

Valid value is a hash reference containing the names of extensible attributes and their associated values (Infoblox::Grid::Extattr objects).

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get extattrs
my $ref_extattrs = $rpz_4aip->extattrs();
#Modify extattrs
$rpz_4aip->extattrs({ 'Site' => $extattr1, 'Administrator' => $extattr2 });
```

extensible_attributes( )

Use this method to set or retrieve the extensible attributes associated with a Substitute (IPv6 Address) Rule.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

For valid values for extensible attributes, see Infoblox::Grid::ExtensibleAttributeDef/Extensible Attribute Values.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get extensible attributes
my $ref_extensible_attributes = $rpz_4aip->extensible_attributes();
#Modify extensible attributes
$rpz_4aip->extensible_attributes({ 'Site' => 'Santa Clara', 'Administrator' => [ 'Peter', 'Tom' ]
});
```

ipv6addr( )

Use this method to set or retrieve the the IPv6 address.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

An IPv6 address is a 128-bit number in colon hexadecimal notation. It consists of eight 16-bit groups of hexadecimal digits separated by colons (example: 12ab:0000:0000:0123:4567:89ab:0000:cdef).

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get ipv6addr
my ipv6addr = $rpz_4aip->ipv6addr();
#Modify ipv6addr
$rpz_4aip->ipv6addr("aaaa:1111:bbbb:2222:cccc:3333:dddd:5555");
```

name( )

Use this method to set or retrieve the host name.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

Hostname in FQDN (Fully Qualified Domain Name) format. The FQDN consists of the network address in CIDR notation or host ip address followed by the domain name (example: 2001::/16.rpz.com, 2001:2002::2203.rpz.com).

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get name
my $name = $rpz_4aip->name();
#Modify name
$rpz_4aip-> name("2001::/64.rpz.com");
```

ttl( )

Use this method to set or retrieve the Time to Live (TTL) value.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

The default value is undefined which indicates that the record inherits the TTL value of the zone.

Specify a TTL value to override the TTL value at the zone level.

Parameter

A 32-bit integer (range from 0 to 4294967295) that represents the duration in seconds that the record is cached. Zero indicates that the record should not be cached.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get ttl
my $ttl = $rpz_4aip->ttl();
#Modify ttl
$rpz_4aip->ttl(1800);
#Un-override ttl
$rpz_4aip->ttl(undef);
```

views( )

Use this method to set or retrieve the view of the RPZ Substitute (IPv6 Address) Rule.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

The default value is the "default" view, which means the RPZ Substitute (IPv6 Address) Rule is located under the default view.

Parameter

An array reference of defined Infoblox::DNS::View objects.

Note that the array size must be 1.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get views
my $ref_views = $rpz_4aip->views();
#Modify views, an array of Infoblox::DNS::View objects
$rpz_4aip->views([$view1]);
```

zone( )

Use this method to retrieve the zone name of a Substitute (IPv6 Address) Rule. This method is read-only and cannot be set.

Parameter

None

Returns

Returns the attribute value.

Example

```
# Get zone
my $zone = $rpz_4aip->zone();
```

rp_zone( )

Use this method to set or retrieve the zone object of a Substitute (IPv6 Address) Rule.

Parameter

An Infoblox::DNS::Zone object.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
# Get rp_zone
my $rp_zone = $rpz_4aip->rp_zone();
#Modify rp_zone, reference of Infoblox::DNS::Zone object
$rpz_4aip->rp_zone($response_policy_zone);
```

SAMPLE CODE

The following sample code demonstrates the different functions that can be applied to a Substitute (IPv6 Address) Rule object, such as add, search, modify, and remove. This sample also includes error handling for the operations.

#Preparation prior to a Substitute (IPv6 Address) Rule insertion

```
#PROGRAM STARTS: Include all the modules that will be used

use strict;
use Infoblox;

#Create a session to the Infoblox Appliance
my $session = Infoblox::Session->new(
    master   => "192.168.1.2", #appliance host ip
    username => "admin",       #appliance user login
    password => "infoblox"     #appliance password
);

unless ($session) {
   die("Construct session failed: ",
       Infoblox::status_code() . ":" . Infoblox::status_detail());
}

#Create the Response Policy zone prior to an RPZ Substitute (IPv6 Address) Rule insertion
```

```
my $zone = Infoblox::DNS::Zone->new(name => "rpz.com");

unless ($zone) {
   die("Construct zone failed: ",
       Infoblox::status_code() . ":" . Infoblox::status_detail());
}

print "Zone object created successfully\n";

#Verify if the zone exists
my $object = $session->get(object => "Infoblox::DNS::Zone",
                           name => "rpz.com");
unless ($object) {
   print "Zone does not exist on server, safe to add the zone\n";
   $session->add($zone)
      or die("Add zone failed: ",
             $session->status_code() . ":" . $session->status_detail());
}
print "Zone added successfully\n";

#Create the zone prior to a CNAME record insertion
my $zone = Infoblox::DNS::Zone->new(name => "rpz.com",
                                    rpz_policy => "GIVEN");
unless ($zone) {
die("Construct zone failed: ",
    Infoblox::status_code() . ":" . Infoblox::status_detail());
}
print "Zone object created successfully\n";

#Verify if the zone exists
my $object = $session->get(object => "Infoblox::DNS::Zone", name => "rpz.com");
unless ($object) {
print "Zone does not exist on server, safe to add the zone\n";
$session->add($zone)
   or die("Add zone failed: ",
          $session->status_code() . ":" . $session->status_detail());
}
```

#Create a Substitute (IPv6 Address) Rule

```
#Construct a Substitute (IPv6 Address) Rule object

my $rpz_4aip = Infoblox::DNS::RPZRecord::AAAAIpAddress->new(
 name     => "2001::/64.rpz.com",
 comment  => "add a Substitute (IPv6 Address) Rule aaaa1.rpz.com",
 ipv6addr => "aaaa:1111:bbbb:2222:cccc:3333:dddd:4444",
 rp_zone  => $zone,
);

unless ($rpz_4aip) {
die("Construct Substitute (IPv6 Address) Rule failed: ",
    Infoblox::status_code() . ":" . Infoblox::status_detail());
}
print "Substitute (IPv6 Address) Rule object created successfully\n";

#Add the Substitute (IPv6 Address) Rule object to Infoblox Appliance through a session
$session->add($rpz_4aip)

 or die("Add Substitute (IPv6 Address) Rule failed: ",
        $session->status_code() . ":" . $session->status_detail());
  print "Substitute (IPv6 Address) Rule object added to server successfully\n";
```

#Search for a specific Substitute (IPv6 Address) Rule

```
#Search all RPZ Substitute (IPv6 Address) Rules that match "rpz.com"
my @retrieved_objs = $session->search(
 object => "Infoblox::DNS::RPZRecord::AAAAIpAddress",
 name   => 'rpz\.com'
);
my $object = $retrieved_objs[0];
unless ($object) {
 die("Search Substitute (IPv6 Address) Rule failed: ",
     $session->status_code() . ":" . $session->status_detail());
}
print "Search Substitute (IPv6 Address) Rule object found at least 1 matching entry\n";

#Search all RPZ Substitute (IPv6 Address) Rules that start with "add" in the comment
```

```
my @retrieved_objs = $session->search(
 object    => "Infoblox::DNS::RPZRecord::AAAAIpAddress",
 comment   => "add.*",
);
my $object = $retrieved_objs[0];
unless ($object) {
 die("Search Substitute (IPv6 Address) Rule failed: ",
     $session->status_code() . ":" . $session->status_detail());
}
print "Search Substitute (IPv6 Address) Rule object found at least 1 matching entry\n";

#Search all RPZ Substitute (IPv6 Address) Rules that start with "aaaa1" and end with ".com"
my @retrieved_objs = $session->search(
 object => "Infoblox::DNS::RPZRecord::AAAAIpAddress",
 name   => '2001.*\.com',
);
my $object = $retrieved_objs[0];
unless ($object) {
 die("Search Substitute (IPv6 Address) Rule failed: ",
     $session->status_code() . ":" . $session->status_detail());
}
print "Search Substitute (IPv6 Address) Rule object using regexp found at least 1 matching entry\n";
```

#Get and modify a Substitute (IPv6 Address) Rule

```
#Get RPZ Substitute (IPv6 Address) Rule through the session
my @retrieved_objs = $session->get(
 object => "Infoblox::DNS::RPZRecord::AAAAIpAddress",
 name   => "2001::/64.rpz.com"
);
my $object = $retrieved_objs[0];

unless ($object) {
 die("Get Substitute (IPv6 Address) Rule failed: ",
     $session->status_code() . ":" . $session->status_detail());
}

#Modify one of the attributes of the specified RPZ Substitute (IPv6 Address) Rule
$object->ipv6addr("aaaa:1111:bbbb:2222:cccc:3333:dddd:eeee");

#Applying the changes
$session->modify($object)

 or die("Modify Substitute (IPv6 Address) Rule failed: ",
        $session->status_code() . ":" . $session->status_detail());
  print "Substitute (IPv6 Address) Rule object modified successfully \n";
```

#Remove a Substitute (IPv6 Address) Rule.

```
#Get RPZ Substitute (IPv6 Address) Rule through the session
my @retrieved_objs = $session->get(
 object   => "Infoblox::DNS::RPZRecord::AAAAIpAddress",
 name     => "2001::/64.rpz.com"
);
my $object = $retrieved_objs[0];
unless ($object) {
 die("Get Substitute (IPv6 Address) Rule failed: ",
     $session->status_code() . ":" . $session->status_detail());
}

print "Get Substitute (IPv6 Address) Rule object found at least 1 matching entry\n";

#Submit the object for removal
$session->remove($object)
 or die("Remove Substitute (IPv6 Address) Rule failed: ",
        $session->status_code() . ":" . $session->status_detail());
print "Substitute (IPv6 Address) Rule object removed successfully \n";

####PROGRAM ENDS####
```

AUTHOR

Infoblox Inc. http://www.infoblox.com/

SEE ALSO

Infoblox::DNS::View, Infoblox::DNS::Zone, Infoblox::Session, Infoblox::Session->get(), Infoblox::Session->search(), Infoblox::Session->add(), Infoblox::Session->remove(), Infoblox::Session->modify()

COPYRIGHT

Copyright (c) 2017 Infoblox Inc.

Infoblox::DNS::RPZRecord::AIpAddress - Response Policy Zone Rule object.

- NAME
- DESCRIPTION
- CONSTRUCTOR
- SESSION METHODS
  - Infoblox::Session->add( )
  - Infoblox::Session->get( )
  - Infoblox::Session->modify( )
  - Infoblox::Session->remove( )
  - Infoblox::Session->search( )
- METHODS
  - comment( )
  - disable( )
  - extattrs( )
  - extensible_attributes( )
  - ipv4addr( )
  - name( )
  - ttl( )
  - views( )
  - zone( )
  - rp_zone( )
- SAMPLE CODE
- AUTHOR
- SEE ALSO
- COPYRIGHT

NAME

Infoblox::DNS::RPZRecord::AIpAddress - Response Policy Zone (RPZ) Substitute (IPv4 Address) Rule object.

DESCRIPTION

An RPZ AIpAddress is an Substitute (IPv4 Address) Rule that maps an IP address represented by a host name to a substitute IPv4 address. To define a specific address-to-address mapping, add an Substitute (IPv4 Address) Rule to a previously defined Response Policy Zone.

This record represents the substitution rule for IP trigger policy. It matches IP addresses that would otherwise appear in A record in the "answer" section of DNS response.

You should use this object to create IP address substitution rules instead usage Infoblox::DNS::RPZRecord::CNAME::IpAddress object.

Note that you must specify only one view for the attribute "views".

CONSTRUCTOR

```
my $rpz_aip = Infoblox::DNS::RPZRecord::AIpAddress->new (
    ipv4addr => $ipv4addr,          #Required
    name     => $fqdn,              #Required
    rp_zone  => $rp_zone            #Required
    comment  => $string,            #Optional / Default is empty
    disable  => "true" | "false"    #Optional / Default "false"
    extattrs              => { $string => $extattr, ... },      #Optional / Default is undefined
    extensible_attributes => { $string => $string | $num, $string => [ $string | $num, ... ], ... },
#Optional / Default is undefined
    last_discovered  => $string,    #Optional / Default is empty
    ttl      => $num | undef,       #Optional / Default is undefined
    views    => [$View1],           #Optional / Default is "default" view
);
```

You cannot set both extattrs and extensible_attributes attributes at the same time.

SESSION METHODS

This section describes all the methods in Infoblox::Session module that you can apply to a Substitute (IPv4 Address) Rule object.

Infoblox::Session->add( )

Use this method to add an object to the Infoblox appliance. See Infoblox::Session->add() for parameters and return values.

Example

```
#Construct an object
my $rpz_aip = Infoblox::DNS::RPZRecord::AIpAddress->new (
    ipv4addr => "1.1.1.1",
    name     => "10.6.0.0/16.rpz.com",
    rp_zone  => response_policy_zone1,
    comment  => "this is a demo rule",
    disable  => "true",
    ttl      => "33",
    views    => [$view1]  );
# Submit for addition
my $response = $session->add( $rpz_aip );
```

Infoblox::Session->get( )

Use this method to retrieve all the matching objects from the Infoblox appliance. See Infoblox::Session->get() for parameters and return values.

Key References

```
Apply the following attributes to get a specific Substitute (IPv4 Address) Rule object:

  name - Optional. A host name in string format.
  view - Optional. The DNS view in which the RPZ Substitute (IPv4 Address) Rule is located. By
default, all DNS views are searched. But if you omit this attribute and specify a zone, the appliance
searches the 'default' view only.
  zone - Optional. A zone name in FQDN format.
  ipv4addr - Optional. The address of the host record.
  extattrs - Optional. A hash reference containing extensible attributes.
  extensible_attributes - Optional. A hash reference containing extensible attributes.
```

Example

```
my @retrieved_objs = $session->get(
    object => "Infoblox::DNS::RPZRecord::AIpAddress",
    name   => "1.2.3.4.rpz.com",
    view   => "default" );

#get all Substitute (IPv4 Address) Rule recods with the extensible attribute 'Site'
my @retrieved_objs = $session->get(
   object => "Infoblox::DNS::RPZRecord::AIpAddress",
   extensible_attributes => { 'Site' => 'Santa Clara' });
```

Infoblox::Session->modify( )

Use this method to modify an object in the Infoblox appliance. See Infoblox::Session->modify() for parameters and return values.

Example

```
# Use method to modify the comment.
$rpz_aip->comment("this is a modified comment");
# Submit modification
my $response = $session->modify( $rpz_aip );
```

Infoblox::Session->remove( )

Use this method to submit the removal of an object from the Infoblox appliance. See Infoblox::Session->remove() for parameters and return values.

To remove a specifc object, first use `get()` or `search()` to retrieve the specific object, and then submit this object for removal.

Example

```
# Get the objects with the same name
my @retrieved_objs = $session->get(
    object => "Infoblox::DNS::RPZRecord::AIpAddress",
    name   => "bind_a.rpz.com",
    view   => "default" );
# Find the desired object from the retrieved list.
my $desired_a = $retrieved_objs[0];
# Submit for removal
my $response = $session->remove( $desired_a );
```

Infoblox::Session->search( )

Use this method to search for Substitute (IPv4 Address) Rule objects in the Infoblox appliance. See Infoblox::Session->search() for para meters and return values.

Key References

```
 Apply the following attributes to search for a Substitute (IPv4 Address) Rule object:

  name     - Optional. A host name in string format (regular expression).
  view     - Optional. The DNS view in which the RPZ Substitute (IPv4 Address) Rule is located. By
default, all DNS views are searched. But if you omit this attribute and specify a zone, the appliance
searches the 'default' view only.
  zone     - Optional. A zone name in FQDN format.
  comment  - Optional. A comment in string format (regular expression).
  ipv4addr - Optional. The address of the host record (regular expression).
  extattrs - Optional. A hash reference containing extensible attributes.
  extensible_attributes - Optional. A hash reference containing extensible attributes.
```

For more information about searching extensible attributes, see Infoblox::Grid::ExtensibleAttributeDef/Searching Extensible Attributes.

Example

```
# search for all Substitute (IPv4 Address) Rule objects that match "rpz.com" in the default DNS view
my @retrieved_objs = $session->search(
    object => "Infoblox::DNS::RPZRecord::AIpAddress",
    name   => 'rpz\.com',
    view   => "default" );

# search for all Substitute (IPv4 Address) Rules in the "rpz.com" zone of the default view
my @retrieved_objs = $session->search(
    object => "Infoblox::DNS::RPZRecord::AIpAddress",
    zone   => "rpz.com",
    view   => "default" );

#get all Substitute (IPv4 Address) Rules with the extensible attribute 'Site'
my @retrieved_objs = $session->search(
  object => "Infoblox::DNS::RPZRecord::AIpAddress",
  extensible_attributes => { 'Site' => 'Santa Clara' });
```

METHODS

This section describes all the methods that you can use to configure and retrieve the attribute values of an RPZ Substitute (IPv4 Address) Rule.

comment( )

Use this method to set or retrieve the descriptive comment.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

Desired comment in string format with a maximum of 256 bytes.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get comment
my $comment = $rpz_aip->comment();
#Modify comment
$rpz_aip->comment("Modifying the Substitute (IPv4 Address) Rule comment");
```

disable( )

Use this method to set or retrieve the disable flag of a DNS record.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

The default value for this field is false. The DNS record is enabled.

Parameter

Specify "true" to set the disable flag or "false" to deactivate/unset it.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get disable
my $disable = $rpz_aip->disable();
#Modify disable
$rpz_aip->disable("true");
```

extattrs( )

Use this method to set or retrieve the extensible attributes associated with a Substitute (IPv4 Address) Rule object.

Parameter

Valid value is a hash reference containing the names of extensible attributes and their associated values (Infoblox::Grid::Extattr objects).

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get extattrs
my $ref_extattrs = $rpz_aip->extattrs();
#Modify extattrs
$rpz_aip->extattrs({ 'Site' => $extattr1, 'Administrator' => $extattr2 });
```

extensible_attributes( )

Use this method to set or retrieve the extensible attributes associated with a Substitute (IPv4 Address) Rule.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

For valid values for extensible attributes, see Infoblox::Grid::ExtensibleAttributeDef/Extensible Attribute Values.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get extensible attributes
my $ref_extensible_attributes = $rpz_aip->extensible_attributes();
#Modify extensible attributes
$rpz_aip->extensible_attributes({ 'Site' => 'Santa Clara', 'Administrator' => [ 'Peter', 'Tom' ] });
```

ipv4addr( )

Use this method to set or retrieve the the IPv4 address.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

An IPv4 address is a 32-bit number in dotted decimal notation. It consists of four 8-bit groups of decimal digits separated by decimal points (example: 192.168.1.2).

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get ipv4addr
my $ipv4addr = $rpz_aip->ipv4addr();
#Modify ipv4addr
$rpz_aip->ipv4addr("2.2.2.2");
```

name( )

Use this method to set or retrieve the host name.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

Hostname in FQDN (Fully Qualified Domain Name) format. The FQDN consists of the network address in CIDR notation or host ip address followed by the domain name (example: 10.0.0.0/8.rpz.com, 1.2.3.4.rpz.com).

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get name
my $name = $rpz_aip->name();
#Modify name
$rpz_aip->name("1.2.3.4.rpz.com");
```

ttl( )

Use this method to set or retrieve the Time to Live (TTL) value.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

The default value is undefined which indicates that the record inherits the TTL value of the zone.

Specify a TTL value to override the TTL value at the zone level.

Parameter

A 32-bit integer (range from 0 to 4294967295) that represents the duration in seconds that the record is cached. Zero indicates that the record should not be cached.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get ttl
my $ttl = $rpz_aip->ttl();
#Modify ttl
$rpz_aip->ttl(1800);
#Un-override ttl
$rpz_aip->ttl(undef);
```

views( )

Use this method to set or retrieve the view of the RPZ Substitute (IPv4 Address) Rule.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

The default value is the "default" view, which means the RPZ Substitute (IPv4 Address) Rule is in the default view.

Parameter

An array reference of defined Infoblox::DNS::View objects.

Note that the array size must be 1.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get views
my $ref_views = $rpz_aip->views();
#Modify views, an array of Infoblox::DNS::View objects
```

```
$rpz_aip->views([$view1]);
```

zone( )

Use this method to retrieve the zone name of a Substitute (IPv4 Address) Rule. This method is read-only and cannot be set.

Parameter

None

Returns

Returns the attribute value.

Example

```
# Get zone
my $zone = $rpz_aip->zone();
```

rp_zone( )

Use this method to set or retrieve the zone object of a Substitute (IPv4 Address) Rule.

Parameter

An Infoblox::DNS::Zone object.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
# Get rp_zone
my $rp_zone = $rpz_aipa->rp_zone();
#Modify rp_zone, reference of Infoblox::DNS::Zone object
$rpz_aipa->rp_zone($response_policy_zone);
```

SAMPLE CODE

The following sample code demonstrates the different functions that can be applied to an object, such as add, search, modify, and remove. This sample also includes error handling for the operations.

#Preparation prior to a Substitute (IPv4 Address) Rule insertion

```
#PROGRAM STARTS: Include all the modules that will be used
use strict;
use Infoblox;

#Create a session to the Infoblox Appliance
my $session = Infoblox::Session->new(
    master   => "192.168.1.2",
    username => "admin",
    password => "infoblox"
);
unless ($session) {
   die("Construct session failed: ",
       $session->status_code() . ":" . $session->status_detail());
}
print "Session created successfully\n";

#Create the RPZ zone prior to an RPZ Substitute (IPv4 Address) Rule insertion
my $zone = Infoblox::DNS::Zone->new(name => "rpz.com",
                                    rpz_policy => "GIVEN");
unless ($zone) {
die("Construct zone failed: ",
    Infoblox::status_code() . ":" . Infoblox::status_detail());
}
print "Zone object created successfully\n";

#Verify if the zone exists
my $object = $session->get(object => "Infoblox::DNS::Zone", name => "rpz.com");
unless ($object) {
print "Zone does not exist on server, safe to add the zone\n";
$session->add($zone)
```

```
    or die("Add zone failed: ",
           $session->status_code() . ":" . $session->status_detail());
}
```

#Create a Substitute (IPv4 Address) Rule

```
my $view1 = Infoblox::DNS::View->new(
 name      => "default" );

#Construct a Substitute (IPv4 Address) Rule object
my $ip_address = Infoblox::DNS::RPZRecord::AIpAddress->new(
 name      => "10.6.0.0/16.rpz.com",
 ipv4addr => "111.222.11.22",
 comment   => "this is a demo A ip address record",
 ttl       => "33",
 views     => [$view1],
 rp_zone   => $zone);
unless ($ip_address) {
die("Construct DNS record CNAME failed: ",
    Infoblox::status_code() . ":" . Infoblox::status_detail());
}
print "DNS RPZ CNAME object created successfully\n";

#Add the DNS RPZ CNAMEIpRecord record object to the Infoblox appliance through a session
$session->add($ip_address)
 or die("Add record cname failed: ",
        $session->status_code() . ":" . $session->status_detail());
print "DNS RPZ CNAME object added to server successfully\n";
```

#Search for a specific DNS RPZ CNAMEIpRecord record

```
#Search all CNAMEIpRecord records that match "rpz.com"
my @retrieved_objs = $session->search(
 object => "Infoblox::DNS::RPZRecord::AIpAddress",
 name   => 'rpz\.com'
);
my $object = $retrieved_objs[0];
unless ($object) {
 die("Search Substitute (IPv4 Address) Rule failed: ",
     $session->status_code() . ":" . $session->status_detail());
}
print "Search Substitute (IPv4 Address) Rule object found at least 1 matching entry\n";

#Search all Substitute (IPv4 Address) Rules that start with "bind" and end with ".rpz.com"
my @retrieved_objs = $session->search(
 object => "Infoblox::DNS::RPZRecord::AIpAddress",
 name   => '^10\.6.*\.rpz\.com'
);
my $object = $retrieved_objs[0];
unless ($object) {
 die("Search Substitute (IPv4 Address) Rule failed: ",
     $session->status_code() . ":" . $session->status_detail());
}
print "Search Substitute (IPv4 Address) Rule object using regexp found at least 1 matching entry\n";
```

#Get and modify a Substitute (IPv4 Address) Rule

```
#Get Substitute (IPv4 Address) Rule through the session
my @retrieved_objs = $session->get(
 object => "Infoblox::DNS::RPZRecord::AIpAddress",
 name   => "10.6.0.0/16.rpz.com",
 view   => "default"
);
my $object = $retrieved_objs[0];
unless ($object) {
 die("Get Substitute (IPv4 Address) Rule failed: ",
     $session->status_code() . ":" . $session->status_detail());
}
print "Get Substitute (IPv4 Address) Rule object found at least 1 matching entry\n";

#Modify one of the attributes of the specified Substitute (IPv4 Address) Rule
$object->disable("true");

#Apply the changes
$session->modify($object)
 or die("Modify Substitute (IPv4 Address) Rule failed: ",
        $session->status_code() . ":" . $session->status_detail());
```

```
print "Substitute (IPv4 Address) Rule object modified successfully \n";
```

#Remove a Substitute (IPv4 Address) Rule

```
#Get Substitute (IPv4 Address) Rule through the session
my @retrieved_objs = $session->get(
 object => "Infoblox::DNS::RPZRecord::AIpAddress",
 name   => "10.6.0.0/16.rpz.com",
 view   => "default"
);
$object = $retrieved_objs[0];
unless ($object) {
 die("Get Substitute (IPv4 Address) Rule failed: ",
     $session->status_code() . ":" . $session->status_detail());
}
print "Get Substitute (IPv4 Address) Rule object found at least 1 matching entry\n";

#Submit the object for removal
$session->remove( $object )
or die("Remove Substitute (IPv4 Address) Rule failed: ",
     $session->status_code() . ":" . $session->status_detail());
print "Substitute (IPv4 Address) Rule object removed successfully \n";

####PROGRAM ENDS####
```

AUTHOR

Infoblox Inc. http://www.infoblox.com/

SEE ALSO

Infoblox::DNS::View, Infoblox::DNS::Zone, Infoblox::Session, Infoblox::Session->get(), Infoblox::Session->search(), Infoblox::Session->add(), Infoblox::Session->remove(), Infoblox::Session->modify()

COPYRIGHT

Copyright (c) 2017 Infoblox Inc.

Infoblox::DNS::RPZRecord::CNAME - DNS Response Policy Zone CNAME record object.

- NAME
- DESCRIPTION
- CONSTRUCTOR
- SESSION METHODS
  - Infoblox::Session->add( )
  - Infoblox::Session->get( )
  - Infoblox::Session->modify( )
  - Infoblox::Session->remove( )
  - Infoblox::Session->search( )
- METHODS
  - canonical( )
  - comment( )
  - disable( )
  - extattrs( )
  - extensible_attributes( )
  - name( )
  - ttl( )
  - views( )
  - zone( )
  - rp_zone( )
- SAMPLE CODE
- AUTHOR
- SEE ALSO
- COPYRIGHT

NAME

Infoblox::DNS::RPZRecord::CNAME - DNS Response Policy Zone (RPZ) CNAME record object.

DESCRIPTION

An RPZ CNAME record represents different RPZ rules, depending on the value of the canonical name. The intention of this object is to support QNAME Trigger policy. The QNAME policy trigger applies to requested domain names (QNAME). This record represents Passthru Domain Name Rule, Block Domain Name (No Such Domain) Rule, Block Domain Name (No Data) Rule and Substitute (Domain Name) Rule.

If canonical name is empty, it is a Block Domain Name (No Such Domain) Rule.

If canonical name is asterisk, it is a Block Domain Name (No Data) Rule.

If canonical name is the same as record name, it is a Passthru Domain Name Rule. If name of object starts with wildcard you must specify special value 'infoblox-passthru' in canonical name in order to create Wildcard Passthru Domain Name Rule, for more details please see the Infoblox Administrator Guide.

If canonical name is not Block Domain Name (No Such Domain) Rule, Block Domain Name (No Data) Rule, or Passthru Domain Name Rule, it is a substitution rule.

Note that you must specify only one view for the attribute "views".

CONSTRUCTOR

```
  my $bind_cname =  Infoblox::DNS::RPZRecord::CNAME->new (
      canonical => $fqdn,                  #Required
      name      => $fqdn,                  #Required
      rp_zone   => $rp_zone,               #Required
      comment   => $string,                #Optional / Default is empty
      disable   => "true"|"false"          #Optional / Default "false"
      extattrs              => { $string => $extattr, ... },      #Optional / Default is undefined
      extensible_attributes => { $string => $string|$num, $string => [ $string|$num, ...], ... }, #Optional
/ Default is undefined
      ttl       => $num|undef,             #Optional / Default is undefined
      views     => [$View1],               #Optional / Default is "default" view
   );
```

You cannot set both extattrs and extensible_attributes attributes at the same time.

SESSION METHODS

This section describes all the methods in Infoblox::Session module that you can apply to a RPZ CNAME record object.

Infoblox::Session->add( )

Use this method to add an object to the Infoblox appliance. See Infoblox::Session->add() for parameters and return values.

Example

```
#Construct an object
my $bind_cname = Infoblox::DNS::RPZRecord::CNAME->new (
    name      => "bind_cname.rpz.com",
    canonical => "cname.domain.com",
    rp_zone   => response_policy_zone1,
    comment   => "this is a demo bind_cname record",
    disable   => "true",
    ttl       => "33",
    views     => [$view1]  );
# Submit for addition
my $response = $session->add( $bind_cname );
```

Infoblox::Session->get( )

Use this method to retrieve all the matching objects from the Infoblox appliance. See Infoblox::Session->get() for parameters and return values.

Key References

```
Apply the following attributes to get a specific DNS RPZ CNAME object:

 name - Optional. A host name in string format.
 view - Optional. The DNS view in which the RPZ CNAME record is located. By default, all DNS views
are searched. But if you omit this attribute and specify a zone, the appliance searches the 'default'
view only.
 zone - Optional. A zone name in FQDN format.
 canonical - Optional. A canonical name.
 extattrs  - Optional. A hash reference containing extensible attributes.
 extensible_attributes - Optional. A hash reference containing extensible attributes.
```

Example

```
my @retrieved_objs = $session->get(
    object => "Infoblox::DNS::RPZRecord::CNAME",
    name   => "bind_cname.rpz.com",
    view   => "default" );

# get all DNS RPZ CNAME record in zone "rpz.com" of default view
my @retrieved_objs = $session->get(
    object => "Infoblox::DNS::RPZRecord::CNAME",
    zone   => "rpz.com",
    view   => "default" );

# get DNS RPZ CNAME record with canonical name "cname.rpz.com" of default view
my @retrieved_objs = $session->get(
    object => "Infoblox::DNS::RPZRecord::CNAME",
    canonical => "cname.domain.com",
    view   => "default" );

# get all DNS RPZ CNAME record with a given extensible attribute
my @retrieved_objs = $session->get(
    object => "Infoblox::DNS::RPZRecord::CNAME",
    extensible_attributes => { 'Site' => 'Santa Clara' });
```

Infoblox::Session->modify( )

Use this method to modify an object in the Infoblox appliance. See Infoblox::Session->modify() for parameters and return values.

Example

```
# Use method to modify the comment.
$bind_cname->comment("this is a modified comment");
# Submit modification
my $response = $session->modify( $bind_cname );
```

Infoblox::Session->remove( )

Use this method to remove an object from the Infoblox appliance. See Infoblox::Session->remove() for parameters and return values.

To remove a specific object, first use `get()` or `search()` to retrieve the specific object, and then submit this object for removal.

Example

```
#Get the objects with the same name
my @retrieved_objs = $session->get(
  object => "Infoblox::DNS::RPZRecord::CNAME",
  name   => "bind_cname.rpz.com",
  view   => "default");
#Find the desired object from the retrieved list.
my $desired_cname = $retrieved_objs[0];
# Submit for removal
my $response = $session->remove( $desired_cname );
```

Infoblox::Session->search( )

Use this method to search for DNS RPZ CNAME record objects in the Infoblox appliance. See Infoblox::Session->search() for parameters and return values.

Key References

```
Apply the following attributes to search for a specific DNS RPZ CNAME object.

  name    - Optional. A host name in string format (regular expression).
  view    - Optional. The DNS view in which the RPZ CNAME record is located. By default, all DNS
views are searched. But if you omit this attribute and specify a zone, the appliance searches the
'default' view only.
  zone    - Optional. A zone name in FQDN format.
  canonical - Optional. A canonical name (regular expression).
  comment - Optional. A comment in string format (regular expression).
  extattrs     - Optional. A hash reference containing extensible attributes.
  extensible_attributes - Optional. A hash reference containing extensible attributes.
```

For more information about searching extensible attributes, see Infoblox::Grid::ExtensibleAttributeDef/Searching Extensible Attributes.

Example

```
# search for all DNS RPZ CNAME objects that match "rpz.com" in the default DNS view
my @retrieved_objs = $session->search(
    object  => "Infoblox::DNS::RPZRecord::CNAME",
    name    => 'rpz\.com',
    view    => "default",
    comment => "this is a modified comment" );

# search for all DNS RPZ CNAME record in zone "rpz.com" of default view
my @retrieved_objs = $session->search(
    object => "Infoblox::DNS::RPZRecord::CNAME",
    zone   => "rpz\.com",
    view   => "default" );

# search DNS RPZ CNAME record with canonical name "cname.rpz.com" of default view
my @retrieved_objs = $session->search(
    object => "Infoblox::DNS::RPZRecord::CNAME",
    canonical => "cname\.domain\.com",
    view   => "default" );

# search all DNS RPZ CNAME recods with the extensible attribute 'Site'
my @retrieved_objs = $session->search(
   object => "Infoblox::DNS::RPZRecord::CNAME",
   extensible_attributes => { 'Site' => 'Santa Clara' });
```

METHODS

This section describes all the methods that you can use to configure and retrieve the attribute values of a RPZ CNAME record.

canonical( )

Use this method to set or retrieve the canonical name.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

Canonical name can be empty, asterisk or in FQDN (Fully Qualified Domain Name) format. The FQDN consists of the hostname followed by the domain name (example: abc.com). A hostname can have a maximum of 256 bytes. If name of object starts with '*.' you must specify special value 'infoblox-passthru' in order to create Passthru Domain Name Rule.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get canonical
my $canonical = $bind_cname->canonical();
#Modify canonical name
$bind_cname->canonical("bind_cname.domain.com");
```

comment( )

Use this method to set or retrieve the descriptive comment.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

Desired comment in string format with a maximum of 256 bytes.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get comment
my $comment = $bind_cname->comment();
#Modify comment
$bind_cname->comment("Modifying the DNS RPZ CNAME comment");
```

disable( )

Use this method to set or retrieve the disable flag of a DNS record.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

The default value for this field is false. The DNS record is enabled.

Parameter

Specify "true" to set the disable flag or "false" to deactivate/unset it.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get disable
my $disable = $bind_cname->disable();
#Modify disable
$bind_cname->disable("true");
```

extattrs( )

Use this method to set or retrieve the extensible attributes associated with a DNS RPZ CNAME record object.

Parameter

Valid value is a hash reference containing the names of extensible attributes and their associated values (Infoblox::Grid::Extattr objects).

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get extattrs
my $ref_extattrs = $bind_cname->extattrs();
#Modify extattrs
```

```
$bind_cname->extattrs({ 'Site' => $extattr1, 'Administrator' => $extattr2 });
```

extensible_attributes( )

Use this method to set or retrieve the extensible attributes associated with a DNS RPZ CNAME record.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

For valid values for extensible attributes, see Infoblox::Grid::ExtensibleAttributeDef/Extensible Attribute Values.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get extensible attributes
my $ref_extensible_attributes = $bind_cname->extensible_attributes();
#Modify extensible attributes
$bind_cname->extensible_attributes({ 'Site' => 'Santa Clara', 'Administrator' => [ 'Peter', 'Tom' ]
});
```

name( )

Use this method to set or retrieve the host name.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

A name in FQDN (Fully Qualified Domain Name) format. The FQDN consists of the host_name followed by the domain name (example: abc.com). A host_name can have a maximum of 256 bytes.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get name
my $name = $bind_cname->name();
#Modify name
$bind_cname->name("bind_cname2.rpz.com");
```

ttl( )

Use this method to set or retrieve the Time to Live (TTL) value.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

The default value is undefined which indicates that the record inherits the TTL value of the zone.

Specify a TTL value to override the TTL value at the zone level.

Parameter

A 32-bit integer (range from 0 to 4294967295) that represents the duration, in seconds, that the record is cached. Zero indicates that the record should not be cached.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get ttl
my $ttl = $bind_cname->ttl();
#Modify ttl
$bind_cname->ttl(1800);
#Un-override ttl
$bind_cname->ttl(undef);
```

views( )

Use this method to set or retrieve the view of the RPZ CNAME record.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

The default value is the "default" view, which means the RPZ CNAME record is located under the default view.

Parameter

An array reference of defined Infoblox::DNS::View objects.

Note that the array size must be 1.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get views
my $ref_views = $bind_cname->views();
#Modify views, an array of Infoblox::DNS::View objects
$bind_cname->views([$view1]);
```

zone( )

Use this method to retrieve the zone name of a DNS RPZ CNAME record. This method is read-only and cannot be set.

Parameter

None

Returns

Returns the attribute value.

Example

```
# Get zone
my $zone = $bind_cname->zone();
```

rp_zone( )

Use this method to set or retrieve the zone object of a DNS RPZ CNAME record.

Parameter

An Infoblox::DNS::Zone object.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
# Get rp_zone
my $rp_zone = $rpz_cname->rp_zone();
#Modify rp_zone, reference of Infoblox::DNS::Zone object
$rpz_cname->rp_zone($response_policy_zone);
```

SAMPLE CODE

The following sample code demonstrates the different functions that can be applied to an object, such as add, search, modify, and remove. This sample code also includes error handling for the operations.

#Preparation prior to a DNS RPZ CNAME record insertion

```
#PROGRAM STARTS: Include all the modules that will be used
use strict;
use Infoblox;

#Create a session to the Infoblox appliance
```

```
my $session = Infoblox::Session->new(
    master   => "192.168.1.2",
    username => "admin",
    password => "infoblox"
);
unless ($session) {
   die("Construct session failed: ",
       Infoblox::status_code() . ":" . Infoblox::status_detail());
}
print "Session created successfully\n";

#Create the zone prior to a RPZ CNAME record insertion
my $zone = Infoblox::DNS::Zone->new(name => "rpz.com",
                                    rpz_policy => "GIVEN");
unless ($zone) {
die("Construct zone failed: ",
    Infoblox::status_code() . ":" . Infoblox::status_detail());
}
print "Zone object created successfully\n";

#Verify if the zone exists
my $object = $session->get(object => "Infoblox::DNS::Zone", name => "rpz.com");
unless ($object) {
print "Zone does not exist on server, safe to add the zone\n";
$session->add($zone)
   or die("Add zone failed: ",
          $session->status_code() . ":" . $session->status_detail());
}
```

#Create a DNS RPZ CNAME record

```
my $view1 = Infoblox::DNS::View->new(
 name      => "default" );

#Construct a DNS RPZ CNAME object
my $bind_cname = Infoblox::DNS::RPZRecord::CNAME->new(
 name      => "bind_cname.rpz.com",
 canonical => "cname.domain.com",
 comment   => "this is a demo bind_cname record",
 ttl       => "33",
 views     => [$view1],
 rp_zone   => $zone);
unless ($bind_cname) {
die("Construct DNS record RPZ CNAME failed: ",
    Infoblox::status_code() . ":" . Infoblox::status_detail());
}
print "DNS RPZ CNAME object created successfully\n";

#Add the DNS RPZ CNAME record object to the Infoblox appliance through a session
$session->add($bind_cname)
 or die("Add record cname failed: ",
        $session->status_code() . ":" . $session->status_detail());
print "DNS RPZ CNAME object added to server successfully\n";
```

#Search for a specific DNS RPZ CNAME record

```
#Search all RPZ CNAME records that match "rpz.com"
my @retrieved_objs = $session->search(
 object => "Infoblox::DNS::RPZRecord::CNAME",
 name   => 'rpz\.com'
);
my $object = $retrieved_objs[0];
unless ($object) {
 die("Search record RPZ CNAME failed: ",
     $session->status_code() . ":" . $session->status_detail());
}
print "Search DNS RPZ CNAME object found at least 1 matching entry\n";

#Search all RPZ CNAME records that start with "bind" and end with ".rpz.com"
my @retrieved_objs = $session->search(
 object => "Infoblox::DNS::RPZRecord::CNAME",
 name   => '^bind.*\.rpz\.com'
);
my $object = $retrieved_objs[0];
unless ($object) {
 die("Search record RPZ CNAME failed: ",
```

```
      $session->status_code() . ":" . $session->status_detail());
}
print "Search DNS RPZ CNAME object using regexp found at least 1 matching entry\n";
```

#Get and modify a DNS RPZ CNAME record

```
#Get CNAME record through the session
my @retrieved_objs = $session->get(
 object => "Infoblox::DNS::RPZRecord::CNAME",
 name   => "bind_cname.rpz.com",
 view   => "default"
);
my $object = $retrieved_objs[0];
unless ($object) {
 die("Get record RPZ CNAME failed: ",
      $session->status_code() . ":" . $session->status_detail());
}
print "Get DNS RPZ CNAME object found at least 1 matching entry\n";

#Modify one of the attributes of the specified RPZ CNAME record
$object->disable("true");

#Apply the changes
$session->modify($object)
 or die("Modify record RPZ CNAME failed: ",
        $session->status_code() . ":" . $session->status_detail());
print "DNS RPZ CNAME object modified successfully \n";
```

#Remove a DNS RPZ CNAME record

```
#Get CNAME record through the session
my @retrieved_objs = $session->get(
 object => "Infoblox::DNS::RPZRecord::CNAME",
 name   => "bind_cname.rpz.com",
 view   => "default"
);
$object = $retrieved_objs[0];
unless ($object) {
 die("Get record RPZ CNAME failed: ",
      $session->status_code() . ":" . $session->status_detail());
}
print "Get DNS RPZ CNAME object found at least 1 matching entry\n";

#Submit the object for removal
$session->remove( $object )
or die("Remove record RPZ CNAME failed: ",
      $session->status_code() . ":" . $session->status_detail());
print "DNS RPZ CNAME object removed successfully \n";

####PROGRAM ENDS####
```

AUTHOR

Infoblox Inc. http://www.infoblox.com/

SEE ALSO

Infoblox::DNS::View, Infoblox::DNS::Zone, Infoblox::Session, Infoblox::Session->get(), Infoblox::Session->search(), Infoblox::Session->add(), Infoblox::Session->remove(), Infoblox::Session->modify()

COPYRIGHT

Copyright (c) 2017 Infoblox Inc.

Infoblox::DNS::RPZRecord::CNAME::ClientIpAddress - DNS Response Policy Zone CNAMEClientIpAddress record object.

- NAME
- DESCRIPTION
- CONSTRUCTOR
- SESSION METHODS
  - Infoblox::Session->add( )
  - Infoblox::Session->get( )
  - Infoblox::Session->modify( )
  - Infoblox::Session->remove( )
  - Infoblox::Session->search( )
- METHODS
  - canonical( )
  - comment( )
  - disable( )
  - extattrs( )
  - extensible_attributes( )
  - name( )
  - ttl( )
  - views( )
  - zone( )
  - rp_zone( )
  - is_ipv4( )
- SAMPLE CODE
- AUTHOR
- SEE ALSO
- COPYRIGHT

NAME

Infoblox::DNS::RPZRecord::CNAME::ClientIpAddress - DNS Response Policy Zone (RPZ) CNAMEClientIpAddress record object.

DESCRIPTION

A DNS RPZ CNAMEClientIpAddress record represents different RPZ rules, depending on the value of the canonical name. This record represents Passthru IP Address Rule, Block IP Address (No Such Domain) Rule, Block IP Address (No Data) Rule.

This record represents the IP trigger policy. It matches IP addresses that would otherwise appear in A and AAAA records in the "answer" section of a DNS response.

If canonical name is empty, it is a Block IP Address (No Such Domain) Rule.

If canonical name is an asterisk, it is a Block IP Address (No Data) Rule.

If canonical name is equal to 'rpz-passthru', it is a Passthru IP Address Rule.

You cannot create Substitute (IPv4/IPv6 Address) Rule for this record see Infoblox::DNS::RPZRecord::AIpAddress or Infoblox::DNS::RPZRecord::AAAAIpAddress for details.

Note that you must specify only one view for the attribute "views".

CONSTRUCTOR

```
  my $bind_cname_client_ip =  Infoblox::DNS::RPZRecord::CNAME::ClientIpAddress->new (
      canonical => '' | '*' | 'rpz-passthru',#Required
      name      => $fqdn,                    #Required
      rp_zone   => $rp_zone,                 #Required
      comment   => $string,                  #Optional / Default is empty
      disable   => "true"|"false"            #Optional / Default "false"
      extattrs              => { $string => $extattr, ... },      #Optional / Default is undefined
      extensible_attributes => { $string => $string|$num, $string => [ $string|$num, ...], ... }, #Optional
/ Default is undefined
      ttl       => $num|undef,               #Optional / Default is undefined
      views     => [$View1],                 #Optional / Default is "default" view
   );
```

You cannot set both extattrs and extensible_attributes attributes at the same time.

SESSION METHODS

This section describes all the methods in Infoblox::Session module that you can apply to a DNS RPZ CNAMEClientIpAddress record object.

Infoblox::Session->add( )

Use this method to add an object to the Infoblox appliance. See Infoblox::Session->add() for parameters and return values.

Example

```
#Construct an IPv4 object
my $bind_cname_client_ip = Infoblox::DNS::RPZRecord::CNAME::ClientIpAddress->new (
    name      => "10.6.0.0/16.rpz.com",
    canonical => "rpz-passthru",
    rp_zone   => response_policy_zone1,
    comment   => "this is a demo record",
    disable   => "true",
    ttl       => "33",
    views     => [$view1]  );
# Submit for addition
my $response = $session->add( $bind_cname_client_ip );

#Construct an IPv6 object
my $bind_cname_client_ip = Infoblox::DNS::RPZRecord::CNAME::ClientIpAddress->new (
    name      => "2001::/64.rpz.com",
    canonical => "*",
    rp_zone   => response_policy_zone1,
    comment   => "this is a demo record",
    disable   => "true",
    ttl       => "33",
    views     => [$view1]  );
# Submit for addition
my $response = $session->add( $bind_cname_client_ip );
```

Infoblox::Session->get( )

Use this method to retrieve all the matching objects from the Infoblox appliance. See Infoblox::Session->get() for parameters and return values.

Key References

```
Apply the following attributes to get a specific DNS RPZ CNAMEClientIpAddress record object:

 name - Optional. A host name in string format.
 view - Optional. The DNS view in which the RPZ CNAMEClientIpAddress record is located. By default,
all DNS views are searched. But if you omit this attribute and specify a zone, the appliance searches
the 'default' view only.
 zone - Optional. A zone name in FQDN format.
 canonical - Optional. A canonical name.
 extattrs     - Optional. A hash reference containing extensible attributes.
 extensible_attributes - Optional. A hash reference containing extensible attributes.
```

Example

```
my @retrieved_objs = $session->get(
    object => "Infoblox::DNS::RPZRecord::CNAME::ClientIpAddress",
    name   => "1.2.3.4.rpz.com",
    view   => "default" );

#Get all DNS RPZ CNAMEClientIpAddress records in zone "rpz.com" of the default view
my @retrieved_objs = $session->get(
    object => "Infoblox::DNS::RPZRecord::CNAME::ClientIpAddress",
    zone   => "rpz.com",
    view   => "default" );

# Get all DNS RPZ CNAMEClientIpAddress passthru records of the default view
my @retrieved_objs = $session->get(
    object => "Infoblox::DNS::RPZRecord::CNAME::ClientIpAddress",
    canonical => "rpz-passthru",
    view   => "default" );

# Get all DNS RPZ CNAMEClientIpAddress records that contain a given extensible attribute
my @retrieved_objs = $session->get(
    object => "Infoblox::DNS::RPZRecord::CNAME::ClientIpAddress",
    extensible_attributes => { 'Site' => 'Santa Clara' });
```

Infoblox::Session->modify( )

Use this method to modify an object in the Infoblox appliance. See Infoblox::Session->modify() for parameters and return values.

Example

```
# Use this method to modify the comment
$bind_cname_client_ip->comment("this is a modified comment");
# Submit modification
my $response = $session->modify( $bind_cname_client_ip );
```

Infoblox::Session->remove( )

Use this method to remove an object from the Infoblox appliance. See Infoblox::Session->remove() for parameters and return values.

To remove a specific object, first use `get()` or `search()` to retrieve the specific object, and then submit this object for removal.

Example

```
#Get the objects with the same name
my @retrieved_objs = $session->get(
  object => "Infoblox::DNS::RPZRecord::CNAME::ClientIpAddress",
  name   => "1.2.3.4.rpz.com",
  view   => "default");
#Find the desired object from the retrieved list
my $desired_cname = $retrieved_objs[0];
#Submit for removal
my $response = $session->remove( $desired_cname );
```

Infoblox::Session->search( )

Use this method to search for DNS RPZ CNAMEClientIpAddress record objects in the Infoblox appliance. See Infoblox::Session->search() for parameters and return values.

Key References

```
Apply the following attributes to search for a specific DNS RPZ CNAME object.

  name     - Optional. A host name in string format (regular expression).
  view     - Optional. The DNS view in which the RPZ CNAMEClientIpAddress record is located. By
default, all DNS views are searched. But if you omit this attribute and specify a zone, the appliance
searches the 'default' view only.
  zone     - Optional. A zone name in FQDN format.
  canonical - Optional. A canonical name.
  comment - Optional. A comment in string format (regular expression).
  extattrs      - Optional. A hash reference containing extensible attributes.
  extensible_attributes - Optional. A hash reference containing extensible attributes.
```

For more information about searching extensible attributes, see Infoblox::Grid::ExtensibleAttributeDef/Searching Extensible Attributes.

Example

```
# search for all DNS RPZ CNAMEClientIpAddress records in zone "rpz.com" of the default view
my @retrieved_objs = $session->search(
    object  => "Infoblox::DNS::RPZRecord::CNAME::ClientIpAddress",
    name    => 'rpz\.com',
    view    => "default",
    comment => "this is a modified comment" );

# search for all DNS RPZ CNAMEClientIpAddress records in zone "rpz.com" of default view
my @retrieved_objs = $session->search(
    object => "Infoblox::DNS::RPZRecord::CNAME::ClientIpAddress",
    zone   => "rpz\.com",
    view   => "default" );

# # search DNS RPZ CNAMEClientIpAddress passthru record of the default view
my @retrieved_objs = $session->search(
    object => "Infoblox::DNS::RPZRecord::CNAME::ClientIpAddress",
    canonical => "rpz-passthru",
    view   => "default" );

# search all DNS RPZ CNAMEClientIpAddress records with the extensible attribute 'Site'
my @retrieved_objs = $session->search(
   object => "Infoblox::DNS::RPZRecord::CNAME::ClientIpAddress",
   extensible_attributes => { 'Site' => 'Santa Clara' });
```

METHODS

This section describes all the methods that you can use to configure and retrieve the attribute values of a DNS RPZ CNAMEClientIpAddress record.

canonical( )

Use this method to set or retrieve the canonical name.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

The valid values are '*', '' and 'rpz-passthru'.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get canonical
my $canonical = $bind_cname_client_ip->canonical();
#Modify canonical name
$bind_cname_client_ip->canonical("rpz-passthru");
```

comment( )

Use this method to set or retrieve the descriptive comment.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

Desired comment in string format with a maximum of 256 bytes.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get comment
my $comment = $bind_cname_client_ip->comment();
#Modify comment
$bind_cname_client_ip->comment("Modifying the DNS RPZ CNAMEClientIpAddress record comment");
```

disable( )

Use this method to set or retrieve the disable flag of a DNS RPZ CNAMEClientIpAddress record.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

The default value for this field is false. The DNS RPZ CNAMEClientIpAddress record is enabled.

Parameter

Specify "true" to set the disable flag or "false" to deactivate/unset it.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get disable
my $disable = $bind_cname_client_ip->disable();
#Modify disable
$bind_cname_client_ip->disable("true");
```

extattrs( )

Use this method to set or retrieve the extensible attributes associated with a DNS RPZ CNAMEClientIpAddress record object.

Parameter

Valid value is a hash reference containing the names of extensible attributes and their associated values (Infoblox::Grid::Extattr objects).

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get extattrs
my $ref_extattrs = $bind_cname_client_ip->extattrs();
#Modify extattrs
$bind_cname_client_ip->extattrs({ 'Site' => $extattr1, 'Administrator' => $extattr2 });
```

extensible_attributes( )

Use this method to set or retrieve the extensible attributes associated with a DNS RPZ CNAMEClientIpAddress record.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

For valid values for extensible attributes, see Infoblox::Grid::ExtensibleAttributeDef/Extensible Attribute Values.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get extensible attributes
my $ref_extensible_attributes = $bind_cname_client_ip->extensible_attributes();
#Modify extensible attributes
$bind_cname_client_ip->extensible_attributes({ 'Site' => 'Santa Clara', 'Administrator' => [
'Peter', 'Tom' ] });
```

name( )

Use this method to set or retrieve the host name.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

A name in FQDN (Fully Qualified Domain Name) format. The FQDN consists of the network address in CIDR notation or host IP address followed by the domain name (example: 10.0.0.0/8.rpz.com, 1.2.3.4.rpz.com).

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get name
my $name = $bind_cname_client_ip->name();
#Modify name
$bind_cname_client_ip->name("4.3.2.1.rpz.com");
```

ttl( )

Use this method to set or retrieve the Time to Live (TTL) value.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

The default value is undefined which indicates that the record inherits the TTL value of the zone.

Specify a TTL value to override the TTL value at the zone level.

Parameter

A 32-bit integer (range from 0 to 4294967295) that represents the duration, in seconds, that the record is cached. Zero indicates that the record should not be cached.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get ttl
my $ttl = $bind_cname_client_ip->ttl();
#Modify ttl
$bind_cname_client_ip->ttl(1800);
#Un-override ttl
$bind_cname_client_ip->ttl(undef);
```

views( )

Use this method to set or retrieve the view of the DNS RPZ CNAMEClientIpAddress record.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

The default value is the "default" view, which means the DNS RPZ CNAMEClientIpAddress record is located in the default view.

Parameter

An array reference of defined Infoblox::DNS::View objects.

Note that the array size must be 1.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get views
my $ref_views = $bind_cname_client_ip->views();
#Modify views, an array of Infoblox::DNS::View objects
$bind_cname_client_ip->views([$view1]);
```

zone( )

Use this method to retrieve the zone name of a DNS RPZ CNAMEClientIpAddress record. This method is read-only and cannot be set.

Parameter

None

Returns

Returns the attribute value.

Example

```
# Get zone
my $zone = $bind_cname_client_ip->zone();
```

rp_zone( )

Use this method to set or retrieve the zone object of a DNS RPZ CNAMEClientIpAddress record.

Parameter

An Infoblox::DNS::Zone object.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
# Get rp_zone
my $rp_zone = $bind_cname_client_ip->rp_zone();
#Modify rp_zone, reference of Infoblox::DNS::Zone object
$bind_cname_client_ip->rp_zone($response_policy_zone);
```

is_ipv4( )

Use this method to set or retrieve whether the record is an IPv4 record. This method is read-only and cannot be set.

Parameter

None

Returns

Use this method to retrieve whether the record is an IPv4 record. If the return value is "true", it is an IPv4 record. Ohterwise, it is an IPv6 record.

Example

```
#Get is_ipv4
my $ip_ipv4 = $bind_cname_client_ip->is_ipv4();
```

SAMPLE CODE

The following sample code demonstrates the different functions that can be applied to an object, such as add, search, modify, and remove. This sample code also includes error handling for the operations.

#Preparation prior to a DNS RPZ CNAMEClientIpAddress record insertion

```
#PROGRAM STARTS: Include all the modules that will be used
use strict;
use Infoblox;

#Create a session to the Infoblox appliance
my $session = Infoblox::Session->new(
    master   => "192.168.1.2",
    username => "admin",
    password => "infoblox"
);

unless($session){
        die("Constructor for session failed: ",
               Infoblox::status_code(). ":" . Infoblox::status_detail());
}
print "Session created successfully.\n";

#Create the zone prior to a CNAMEClientIpAddress record insertion
my $zone = Infoblox::DNS::Zone->new(name => "rpz.com",
                                    rpz_policy => "GIVEN");
unless ($zone) {
die("Construct zone failed: ",
    Infoblox::status_code() . ":" . Infoblox::status_detail());
}
print "Zone object created successfully\n";

#Verify if the zone exists
my $object = $session->get(object => "Infoblox::DNS::Zone", name => "rpz.com");
unless ($object) {
print "Zone does not exist on server, safe to add the zone\n";
$session->add($zone)
   or die("Add zone failed: ",
          $session->status_code() . ":" . $session->status_detail());
}
print "Zone added successfully\n";
if (defined($self)){
    unshift @{$self->{cleanup_objs}}, $zone;
}

#B<#Create a DNS RPZ CNAMEClientIpAddress record>

my $view1 = Infoblox::DNS::View->new(
 name      => "default" );

#Construct a DNS RPZ CNAMEClientIpAddress object
my $client_ip_address = Infoblox::DNS::RPZRecord::CNAME::ClientIpAddress->new(
 name      => "10.6.0.0/16.rpz.com",
 canonical => "rpz-passthru",
 comment   => "this is a demo client IP Address record",
 ttl       => "33",
 views     => [$view1],
 rp_zone   => $zone);
unless ($client_ip_address) {
```

```
die("Construct DNS record CNAME failed: ",
    Infoblox::status_code() . ":" . Infoblox::status_detail());
}
print "DNS RPZ CNAME object created successfully\n";

#Add the DNS RPZ CNAME record object to the Infoblox appliance through a session
$session->add($client_ip_address)
 or die("Add record cname failed: ",
        $session->status_code() . ":" . $session->status_detail());
print "DNS RPZ CNAME object added to server successfully\n";

#B<#Search for a specific DNS RPZ CNAME record>

#Search all CNAME records that match "rpz.com"
my @retrieved_objs = $session->search(
 object => "Infoblox::DNS::RPZRecord::CNAME::ClientIpAddress",
 name   => 'rpz\.com'
);
my $object = $retrieved_objs[0];
unless ($object) {
 die("Search record CNAMEClientIpAddress failed: ",
     $session->status_code() . ":" . $session->status_detail());
}
print "Search DNS RPZ CNAMEClientIpAddress object found at least 1 matching entry\n";

#Search all CNAMEClientIpAddress records that start with "bind" and end with ".rpz.com"
my @retrieved_objs = $session->search(
 object => "Infoblox::DNS::RPZRecord::CNAME::ClientIpAddress",
 name   => '^10\.6.*\.rpz\.com'
);
my $object = $retrieved_objs[0];
unless ($object) {
 die("Search record CNAMEClientIpAddress failed: ",
     $session->status_code() . ":" . $session->status_detail());
}
print "Search DNS RPZ CNAMEClientIpAddress object using regexp found at least 1 matching entry\n";

#B<#Get and modify a DNS RPZ CNAMEClientIpAddress record>

#Get CNAMEClientIpAddress record through the session
my @retrieved_objs = $session->get(
 object => "Infoblox::DNS::RPZRecord::CNAME::ClientIpAddress",
 name   => "10.6.0.0/16.rpz.com",
 view   => "default"
);
my $object = $retrieved_objs[0];
unless ($object) {
 die("Get record CNAMEClientIpAddress failed: ",
     $session->status_code() . ":" . $session->status_detail());
}
print "Get DNS RPZ CNAMEClientIpAddress object found at least 1 matching entry\n";

#Modify one of the attributes of the specified CNAMEClientIpAddress record
$object->disable("true");

#Apply the changes
$session->modify($object)
 or die("Modify record CNAMEClientIpAddress failed: ",
        $session->status_code() . ":" . $session->status_detail());
print "DNS RPZ CNAMEClientIpAddress object modified successfully \n";

#B<#Remove a DNS RPZ CNAMEClientIpAddress record>

#Get CNAMEClientIpAddress record through the session
my @retrieved_objs = $session->get(
 object => "Infoblox::DNS::RPZRecord::CNAME::ClientIpAddress",
 name   => "10.6.0.0/16.rpz.com",
 view   => "default"
);
$object = $retrieved_objs[0];
unless ($object) {
 die("Get record CNAMEClientIpAddress failed: ",
     $session->status_code() . ":" . $session->status_detail());
}
print "Get DNS RPZ CNAMEClientIpAddress object found at least 1 matching entry\n";

#Submit the object for removal
$session->remove( $object )
```

```
or die("Remove record CNAMEClientIpAddress failed: ",
     $session->status_code() . ":" . $session->status_detail());
print "DNS RPZ CNAMEClientIpAddress object removed successfully \n";

return 1;
```

####PROGRAM ENDS####

AUTHOR

Infoblox Inc. http://www.infoblox.com/

SEE ALSO

Infoblox::DNS::View, Infoblox::DNS::Zone, Infoblox::Session, Infoblox::Session->get(), Infoblox::Session->search(), Infoblox::Session->add(), Infoblox::Session->remove(), Infoblox::Session->modify()

COPYRIGHT

Copyright (c) 2017 Infoblox Inc.

Infoblox::DNS::RPZRecord::CNAME::ClientIpAddressDN - DNS Substitute Domain Name rule object.

- NAME
- DESCRIPTION
- CONSTRUCTOR
- SESSION METHODS
  - Infoblox::Session->add( )
  - Infoblox::Session->get( )
  - Infoblox::Session->modify( )
  - Infoblox::Session->remove( )
  - Infoblox::Session->search( )
- METHODS
  - canonical( )
  - comment( )
  - disable( )
  - extattrs( )
  - extensible_attributes( )
  - name( )
  - ttl( )
  - views( )
  - zone( )
  - rp_zone( )
  - is_ipv4( )
- SAMPLE CODE
- AUTHOR
- SEE ALSO
- COPYRIGHT

NAME

Infoblox::DNS::RPZRecord::CNAME::ClientIpAddressDN - DNS Substitute Domain Name (Based on Client IP Address) rule object.

DESCRIPTION

A DNS Substitute Domain Name (Based on Client IP Address) rule represents different Response Policy Zone (RPZ) rules, depending on the value of the canonical name.

This rule represents Substitute (Domain Name) Rule.

Note that you must specify only one view for the attribute "views".

CONSTRUCTOR

```
  my $bind_cname_dn =  Infoblox::DNS::RPZRecord::CNAME::ClientIpAddressDN->new (
      canonical => $fqdn,                 #Required
      name      => $fqdn,                 #Required
      rp_zone   => $rp_zone,              #Required
      comment   => $string,               #Optional / Default is empty
      disable   => "true"|"false"         #Optional / Default "false"
      extattrs              => { $string => $extattr, ... },      #Optional / Default is undefined
      extensible_attributes => { $string => $string|$num, $string => [ $string|$num, ...], ... }, #Optional
/ Default is undefined
      ttl       => $num|undef,            #Optional / Default is undefined
      views     => [$View1],              #Optional / Default is "default" view
   );
```

You cannot set both extattrs and extensible_attributes attributes at the same time.

SESSION METHODS

This section describes all the methods in Infoblox::Session module that you can apply to a DNS Substitute Domain Name (Based on Client IP Address) rule object.

Infoblox::Session->add( )

Use this method to add an object to the Infoblox appliance. See Infoblox::Session->add() for parameters and return values.

Example

```
#Construct a DNS Substitute Domain Name (Based on Client IP Address) rule object
my $bind_cname_dn = Infoblox::DNS::RPZRecord::CNAME::ClientIpAddressDN->new (
    name      => "10.6.0.0/16.rpz.com",
    canonical => "domain.com",
    rp_zone   => response_policy_zone1,
    comment   => "this is a demo rule",
    disable   => "true",
    ttl       => "33",
    views     => [$view1]  );
# Submit for addition
my $response = $session->add( $bind_cname_dn );
```

Infoblox::Session->get( )

Use this method to retrieve all the matching objects from the Infoblox appliance. See Infoblox::Session->get() for parameters and return values.

Key References

```
 Apply the following attributes to get a specific DNS Substitute Domain Name (Based on Client IP
Address) rule object:

  name - Optional. A host name in string format.
  view - Optional. The DNS view in which the Substitute Domain Name (Based on Client IP Address) rule
is located. By default, all DNS views are searched. But if you omit this attribute and specify a
zone, the appliance searches the 'default' view only.
  zone - Optional. A zone name in FQDN format.
  canonical - Optional. A canonical name.
  extattrs      - Optional. A hash reference containing extensible attributes.
  extensible_attributes - Optional. A hash reference containing extensible attributes.
```

Example

```
# get DNS Substitute Domain Name (Based on Client IP Address) rule with name "1.2.3.4.rpz.com" of
default view
my @retrieved_objs = $session->get(
    object => "Infoblox::DNS::RPZRecord::CNAME::ClientIpAddressDN",
    name   => "1.2.3.4.rpz.com",
    view   => "default" );

# get all DNS Substitute Domain Name (Based on Client IP Address) rules in zone "rpz.com" of default
view
my @retrieved_objs = $session->get(
    object => "Infoblox::DNS::RPZRecord::CNAME::ClientIpAddressDN",
    zone   => "rpz.com",
    view   => "default" );

# get DNS Substitute Domain Name (Based on Client IP Address) rule with canonical name "domain.com"
of default view
my @retrieved_objs = $session->get(
    object => "Infoblox::DNS::RPZRecord::CNAME::ClientIpAddressDN",
    canonical => "domain\.com",
    view   => "default" );

# get all DNS Substitute Domain Name (Based on Client IP Address) rules with a given extensible
attribute
my @retrieved_objs = $session->get(
    object => "Infoblox::DNS::RPZRecord::CNAME::ClientIpAddressDN",
    extensible_attributes => { 'Site' => 'Santa Clara' });
```

Infoblox::Session->modify( )

Use this method to modify an object in the Infoblox appliance. See Infoblox::Session->modify() for parameters and return values.

Example

```
# Use method to modify the comment.
$bind_cname_dn->comment("this is a modified comment");
# Submit modification
my $response = $session->modify( $bind_cname_dn );
```

Infoblox::Session->remove( )

Use this method to remove an object from the Infoblox appliance. See Infoblox::Session->remove() for parameters and return values.

To remove a specific object, first use `get()` or `search()` to retrieve the specific object, and then submit this object for removal.

Example

```
#Get the objects with the same name
```

```
my @retrieved_objs = $session->get(
  object => "Infoblox::DNS::RPZRecord::CNAME::ClientIpAddressDN",
  name   => "1.2.3.4.rpz.com",
  view   => "default");
#Find the desired object from the retrieved list.
my $desired_cname = $retrieved_objs[0];
# Submit for removal
my $response = $session->remove( $desired_cname );
```

Infoblox::Session->search( )

Use this method to search for DNS Substitute Domain Name (Based on Client IP Address) rule objects in the Infoblox appliance. See Infoblox::Session->search() for parameters and return values.

Key References

```
 Apply the following attributes to search for a specific DNS Substitute Domain Name (Based on Client
IP Address) rule object.

  name    - Optional. A host name in string format (regular expression).
  view    - Optional. The DNS view in which the Substitute Domain Name (Based on Client IP Address)
rule is located. By default, all DNS views are searched. But if you omit this attribute and specify a
zone, the appliance searches the 'default' view only.
  zone    - Optional. A zone name in FQDN format.
  canonical - Optional. A canonical name (regular expression).
  comment - Optional. A comment in string format (regular expression).
  extattrs     - Optional. A hash reference containing extensible attributes.
  extensible_attributes - Optional. A hash reference containing extensible attributes.
```

For more information about searching extensible attributes, see Infoblox::Grid::ExtensibleAttributeDef/Searching Extensible Attributes.

Example

```
 # search for all DNS Substitute Domain Name (Based on Client IP Address) rules that match "rpz.com"
in the default DNS view
 my @retrieved_objs = $session->search(
     object  => "Infoblox::DNS::RPZRecord::CNAME::ClientIpAddressDN",
     name    => 'rpz\.com',
     view    => "default" );

 # search for all DNS Substitute Domain Name (Based on Client IP Address) rules in zone "rpz.com" of
default view
 my @retrieved_objs = $session->search(
     object => "Infoblox::DNS::RPZRecord::CNAME::ClientIpAddressDN",
     zone   => "rpz\.com",
     view   => "default" );

 # search for all DNS Substitute Domain Name (Based on Client IP Address) rules with canonical name
that match "domain.com" of default view
 my @retrieved_objs = $session->search(
     object => "Infoblox::DNS::RPZRecord::CNAME::ClientIpAddressDN",
     canonical => "domain\.com",
     view   => "default" );

 # search all DNS Substitute Domain Name (Based on Client IP Address) rules with the extensible
attribute 'Site'
 my @retrieved_objs = $session->search(
    object => "Infoblox::DNS::RPZRecord::CNAME::ClientIpAddressDN",
    extensible_attributes => { 'Site' => 'Santa Clara' });
```

METHODS

This section describes all the methods that you can use to configure and retrieve the attribute values of a DNS Substitute Domain Name (Based on Client IP Address) rule.

canonical( )

Use this method to set or retrieve the canonical name.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

Canonical name in FQDN (Fully Qualified Domain Name) format. The FQDN consists of the hostname followed by the domain name (example: abc.com)

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get canonical
my $canonical = $bind_cname_dn->canonical();
#Modify canonical name
$bind_cname_dn->canonical("domain.name");
```

comment( )

Use this method to set or retrieve the descriptive comment.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

Desired comment in string format with a maximum of 256 bytes.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get comment
my $comment = $bind_cname_dn->comment();
#Modify comment
$bind_cname_dn->comment("Modifying the DNS Substitute Domain Name (Based on Client IP Address) rule
comment");
```

disable( )

Use this method to set or retrieve the disable flag of a DNS Substitute Domain Name (Based on Client IP Address) rule.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

The default value for this field is false. The DNS Substitute Domain Name (Based on Client IP Address) rule is enabled.

Parameter

Specify "true" to set the disable flag or "false" to deactivate/unset it.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get disable
my $disable = $bind_cname_dn->disable();
#Modify disable
$bind_cname_dn->disable("true");
```

extattrs( )

Use this method to set or retrieve the extensible attributes associated with a DNS Substitute Domain Name (Based on Client IP Address) rule object.

Parameter

Valid value is a hash reference containing the names of extensible attributes and their associated values (Infoblox::Grid::Extattr objects).

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get extattrs
```

```
my $ref_extattrs = $bind_cname_dn->extattrs();
#Modify extattrs
$bind_cname_dn->extattrs({ 'Site' => $extattr1, 'Administrator' => $extattr2 });
```

extensible_attributes( )

Use this method to set or retrieve the extensible attributes associated with a DNS Substitute Domain Name (Based on Client IP Address) rule.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

For valid values for extensible attributes, see Infoblox::Grid::ExtensibleAttributeDef/Extensible Attribute Values.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get extensible attributes
my $ref_extensible_attributes = $bind_cname_dn->extensible_attributes();
#Modify extensible attributes
$bind_cname_dn->extensible_attributes({ 'Site' => 'Santa Clara', 'Administrator' => [ 'Peter', 'Tom'
] });
```

name( )

Use this method to set or retrieve the host name.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

A name in FQDN (Fully Qualified Domain Name) format. The FQDN consists of the network address in CIDR notation or host IP address followed by the domain name (example: 10.0.0.0/8.rpz.com, 1.2.3.4.rpz.com).

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get name
my $name = $bind_cname_dn->name();
#Modify name
$bind_cname_dn->name("4.3.2.1.rpz.com");
```

ttl( )

Use this method to set or retrieve the Time to Live (TTL) value.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

The default value is undefined which indicates that the rule inherits the TTL value of the zone.

Specify a TTL value to override the TTL value at the zone level.

Parameter

A 32-bit integer (range from 0 to 4294967295) that represents the duration, in seconds, that the rule is cached. Zero indicates that the rule should not be cached.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get ttl
my $ttl = $bind_cname_dn->ttl();
#Modify ttl
```

```
$bind_cname_dn->ttl(1800);
#Un-override ttl
$bind_cname_dn->ttl(undef);
```

views( )

Use this method to set or retrieve the view of the DNS Substitute Domain Name (Based on Client IP Address) rule.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

The default value is the "default" view, which means the DNS Substitute Domain Name (Based on Client IP Address) rule is located in the default view.

Parameter

An array reference of defined Infoblox::DNS::View objects.

Note that the array size must be 1.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get views
my $ref_views = $bind_cname_dn->views();
#Modify views, an array of Infoblox::DNS::View objects
$bind_cname_dn->views([$view1]);
```

zone( )

Use this method to retrieve the zone name of a DNS Substitute Domain Name (Based on Client IP Address) rule. This method is read-only and cannot be set.

Parameter

None

Returns

Returns the attribute value.

Example

```
# Get zone
my $zone = $bind_cname_dn->zone();
```

rp_zone( )

Use this method to set or retrieve the zone object of a DNS Substitute Domain Name (Based on Client IP Address) rule.

Parameter

An Infoblox::DNS::Zone object.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
# Get rp_zone
my $rp_zone = $bind_cname_dn->rp_zone();
#Modify rp_zone, reference of Infoblox::DNS::Zone object
$bind_cname_dn->rp_zone($response_policy_zone);
```

is_ipv4( )

Use this method to set or retrieve whether the DNS Substitute Domain Name (Based on Client IP Address) rule is an IPv4 rule. This method is read-only and cannot be set.

Parameter

None

Returns

Use this method to retrieve whether the DNS Substitute Domain Name (Based on Client IP Address) rule is an IPv4 rule. If the return value is "true", it is an IPv4 rule. Ohterwise, it is an IPv6 rule.

Example

```
#Get is_ipv4
my $ip_ipv4 = $bind_cname_dn->is_ipv4();
```

SAMPLE CODE

The following sample code demonstrates the different functions that can be applied to an object, such as add, search, modify, and remove. This sample code also includes error handling for the operations.

#Preparation prior to a DNS Substitute Domain Name (Based on Client IP Address) rule insertion

```
#PROGRAM STARTS: Include all the modules that will be used
use strict;
use Infoblox;

#Create a session to the Infoblox appliance
my $session = Infoblox::Session->new(
    master   => "192.168.1.2",
    username => "admin",
    password => "infoblox"
);
unless ($session) {
   die("Construct session failed: ",
       Infoblox::status_code() . ":" . Infoblox::status_detail());
}
print "Session created successfully\n";

#Create the zone prior to a DNS Substitute Domain Name (Based on Client IP Address) rule insertion
my $zone = Infoblox::DNS::Zone->new(name => "rpz.com",
                                    rpz_policy => "GIVEN");
unless ($zone) {
die("Construct zone failed: ",
    Infoblox::status_code() . ":" . Infoblox::status_detail());
}
print "Zone object created successfully\n";

#Verify if the zone exists
my $object = $session->get(object => "Infoblox::DNS::Zone", name => "rpz.com");
unless ($object) {
print "Zone does not exist on server, safe to add the zone\n";
$session->add($zone)
   or die("Add zone failed: ",
          $session->status_code() . ":" . $session->status_detail());
}
print "Zone added successfully\n";
```

#Create a DNS Substitute Domain Name (Based on Client IP Address) rule

```
my $view1 = Infoblox::DNS::View->new(
 name      => "default" );

#Construct a DNS Substitute Domain Name (Based on Client IP Address) rule object
my $domain_name = Infoblox::DNS::RPZRecord::CNAME::ClientIpAddressDN->new(
 name      => "10.6.0.0/16.rpz.com",
 canonical => "test.com",
 comment   => "this is a demo DNS Substitute Domain Name (Based on Client IP Address) rule",
 ttl       => "33",
 views     => [$view1],
 rp_zone   => $zone);
unless ($domain_name) {
die("Construct DNS Substitute Domain Name (Based on Client IP Address) rule failed: ",
    Infoblox::status_code() . ":" . Infoblox::status_detail());
}
print "DNS Substitute Domain Name (Based on Client IP Address) rule object created successfully\n";

#Add the DNS Substitute Domain Name (Based on Client IP Address) rule object to the Infoblox appliance
through a session
$session->add($domain_name)
 or die("Add DNS Substitute Domain Name (Based on Client IP Address) rule failed: ",
```

```
        $session->status_code() . ":" . $session->status_detail());
 print "DNS Substitute Domain Name (Based on Client IP Address) rule object added to server
successfully\n";
```

#Search for a specific DNS Substitute Domain Name (Based on Client IP Address) rule

```
 #Search all DNS Substitute Domain Name (Based on Client IP Address) rules that match "rpz.com"
 my @retrieved_objs = $session->search(
  object => "Infoblox::DNS::RPZRecord::CNAME::ClientIpAddressDN",
  name   => 'rpz\.com'
 );
 my $object = $retrieved_objs[0];
 unless ($object) {
  die("Search DNS Substitute Domain Name (Based on Client IP Address) rule failed: ",
      $session->status_code() . ":" . $session->status_detail());
 }
 print "Search DNS Substitute Domain Name (Based on Client IP Address) rule object found at least 1
matching entry\n";

 #Search all DNS Substitute Domain Name (Based on Client IP Address) rules that start with "bind" and end
with ".rpz.com"
 my @retrieved_objs = $session->search(
  object => "Infoblox::DNS::RPZRecord::CNAME::ClientIpAddressDN",
  name   => '^10\.6.*\.rpz\.com'
 );
 my $object = $retrieved_objs[0];
 unless ($object) {
  die("Search DNS Substitute Domain Name (Based on Client IP Address) rule failed: ",
      $session->status_code() . ":" . $session->status_detail());
 }
 print "Search DNS Substitute Domain Name (Based on Client IP Address) rule object using regexp found at
least 1 matching entry\n";
```

#Get and modify a DNS Substitute Domain Name (Based on Client IP Address) rule

```
 #Get DNS Substitute Domain Name (Based on Client IP Address) rule through the session
 my @retrieved_objs = $session->get(
  object => "Infoblox::DNS::RPZRecord::CNAME::ClientIpAddressDN",
  name   => "10.6.0.0/16.rpz.com",
  view   => "default"
 );
 my $object = $retrieved_objs[0];
 unless ($object) {
  die("Get DNS Substitute Domain Name (Based on Client IP Address) rule failed: ",
      $session->status_code() . ":" . $session->status_detail());
 }
 print "Get DNS Substitute Domain Name (Based on Client IP Address) rule object found at least 1 matching
entry\n";

 #Modify one of the attributes of the specified DNS Substitute Domain Name (Based on Client IP Address)
rule
 $object->disable("true");

 #Apply the changes
 $session->modify($object)
  or die("Modify DNS Substitute Domain Name (Based on Client IP Address) rule failed: ",
         $session->status_code() . ":" . $session->status_detail());
 print "DNS Substitute Domain Name (Based on Client IP Address) rule object modified successfully \n";
```

#Remove a DNS Substitute Domain Name (Based on Client IP Address) rule

```
 #Get DNS Substitute Domain Name (Based on Client IP Address) rule through the session
 my @retrieved_objs = $session->get(
  object => "Infoblox::DNS::RPZRecord::CNAME::ClientIpAddressDN",
  name   => "10.6.0.0/16.rpz.com",
  view   => "default"
 );
 $object = $retrieved_objs[0];
 unless ($object) {
  die("Get DNS Substitute Domain Name (Based on Client IP Address) rule failed: ",
      $session->status_code() . ":" . $session->status_detail());
 }
 print "Get DNS Substitute Domain Name (Based on Client IP Address) rule object found at least 1 matching
entry\n";

 #Submit the object for removal
 $session->remove( $object )
```

```
or die("Remove DNS Substitute Domain Name (Based on Client IP Address) rule failed: ",
    $session->status_code() . ":" . $session->status_detail());
print "DNS Substitute Domain Name (Based on Client IP Address) rule object removed successfully \n";

####PROGRAM ENDS####
```

AUTHOR

Infoblox Inc. http://www.infoblox.com/

SEE ALSO

Infoblox::DNS::View, Infoblox::DNS::Zone, Infoblox::Session, Infoblox::Session->get(), Infoblox::Session->search(), Infoblox::Session->add(), Infoblox::Session->remove(), Infoblox::Session->modify()

COPYRIGHT

Copyright (c) 2017 Infoblox Inc.

Infoblox::DNS::RPZRecord::CNAME::IpAddress - DNS Response Policy Zone CNAMEIpAddress record object.

- NAME
- DESCRIPTION
- CONSTRUCTOR
- SESSION METHODS
  - Infoblox::Session->add( )
  - Infoblox::Session->get( )
  - Infoblox::Session->modify( )
  - Infoblox::Session->remove( )
  - Infoblox::Session->search( )
- METHODS
  - canonical( )
  - comment( )
  - disable( )
  - extattrs( )
  - extensible_attributes( )
  - name( )
  - ttl( )
  - views( )
  - zone( )
  - rp_zone( )
  - is_ipv4( )
- SAMPLE CODE
- AUTHOR
- SEE ALSO
- COPYRIGHT

NAME

Infoblox::DNS::RPZRecord::CNAME::IpAddress - DNS Response Policy Zone (RPZ) CNAMEIpAddress record object.

DESCRIPTION

A DNS RPZ CNAMEIpAddress record represents different RPZ rules, depending on the value of the canonical name. This record represents Passthru IP Address Rule, Block IP Address (No Such Domain) Rule, Block IP Address (No Data) Rule.

This record represents IP trigger policy. It matches IP addresses that would otherwise appear in A and AAAA records in the "answer" section of DNS response.

If canonical name is empty, it is a Block IP Address (No Such Domain) Rule.

If canonical name is an asterisk, it is a Block IP Address (No Data) Rule.

If canonical name is the same as host name, it is a Passthru IP Address Rule.

You cannot create Substitute (IPv4/IPv6 Address) Rule for this record see Infoblox::DNS::RPZRecord::AIpAddress or Infoblox::DNS::RPZRecord::AAAAIpAddress for details.

Note that you must specify only one view for the attribute "views".

CONSTRUCTOR

```
  my $bind_cnameip =  Infoblox::DNS::RPZRecord::CNAME::IpAddress->new (
      canonical => $ip_address,           #Required
      name      => $fqdn,                 #Required
      rp_zone   => $rp_zone,              #Required
      comment   => $string,               #Optional / Default is empty
      disable   => "true"|"false"         #Optional / Default "false"
      extattrs              => { $string => $extattr, ... },      #Optional / Default is undefined
      extensible_attributes => { $string => $string|$num, $string => [ $string|$num, ...], ... }, #Optional
/ Default is undefined
      ttl       => $num|undef,            #Optional / Default is undefined
      views     => [$View1],              #Optional / Default is "default" view
   );
```

You cannot set both extattrs and extensible_attributes attributes at the same time.

SESSION METHODS

This section describes all the methods in Infoblox::Session module that you can apply to a DNS RPZ CNAMEIpAddress record object.

Infoblox::Session->add( )

Use this method to add an object to the Infoblox appliance. See Infoblox::Session->add() for parameters and return values.

Example

```
#Construct an IPv4 object
my $bind_cnameip = Infoblox::DNS::RPZRecord::CNAME::IpAddress->new (
    name      => "10.6.0.0/16.rpz.com",
    canonical => "10.6.0.0/16",
    rp_zone   => response_policy_zone1,
    comment   => "this is a demo record",
    disable   => "true",
    ttl       => "33",
    views     => [$view1]  );
# Submit for addition
my $response = $session->add( $bind_cnameip );

#Construct an IPv6 object
my $bind_cnameip = Infoblox::DNS::RPZRecord::CNAME::IpAddress->new (
    name      => "2001::/64.rpz.com",
    canonical => "*",
    rp_zone   => response_policy_zone1,
    comment   => "this is a demo record",
    disable   => "true",
    ttl       => "33",
    views     => [$view1]  );
# Submit for addition
my $response = $session->add( $bind_cnameip );
```

Infoblox::Session->get( )

Use this method to retrieve all the matching objects from the Infoblox appliance. See Infoblox::Session->get() for parameters and return values.

Key References

```
Apply the following attributes to get a specific DNS RPZ CNAMEIpAddress record object:

 name - Optional. A host name in string format.
 view - Optional. The DNS view in which the RPZ CNAMEIpAddress record is located. By default, all
DNS views are searched. But if you omit this attribute and specify a zone, the appliance searches the
'default' view only.
 zone - Optional. A zone name in FQDN format.
 canonical - Optional. A canonical name.
 extattrs     - Optional. A hash reference containing extensible attributes.
 extensible_attributes - Optional. A hash reference containing extensible attributes.
```

Example

```
my @retrieved_objs = $session->get(
    object => "Infoblox::DNS::RPZRecord::CNAME::IpAddress",
    name   => "1.2.3.4.rpz.com",
    view   => "default" );

# get all DNS RPZ CNAMEIpAddress record in zone "rpz.com" of default view
my @retrieved_objs = $session->get(
    object => "Infoblox::DNS::RPZRecord::CNAME::IpAddress",
    zone   => "rpz.com",
    view   => "default" );

# get DNS RPZ CNAMEIpAddress record with canonical name "cname.rpz.com" of default view
my @retrieved_objs = $session->get(
    object => "Infoblox::DNS::RPZRecord::CNAME::IpAddress",
    canonical => "10.6.0.0/16",
    view   => "default" );

# get all DNS RPZ CNAMEIpAddress record with a given extensible attribute
my @retrieved_objs = $session->get(
    object => "Infoblox::DNS::RPZRecord::CNAME::IpAddress",
    extensible_attributes => { 'Site' => 'Santa Clara' });
```

Infoblox::Session->modify( )

Use this method to modify an object in the Infoblox appliance. See Infoblox::Session->modify() for parameters and return values.

Example

```
# Use method to modify the comment.
$bind_cnameip->comment("this is a modified comment");
# Submit modification
my $response = $session->modify( $bind_cnameip );
```

Infoblox::Session->remove( )

Use this method to remove an object from the Infoblox appliance. See Infoblox::Session->remove() for parameters and return values.

To remove a specific object, first use `get()` or `search()` to retrieve the specific object, and then submit this object for removal.

Example

```
#Get the objects with the same name
my @retrieved_objs = $session->get(
  object => "Infoblox::DNS::RPZRecord::CNAME::IpAddress",
  name   => "1.2.3.4.rpz.com",
  view   => "default");
#Find the desired object from the retrieved list.
my $desired_cname = $retrieved_objs[0];
# Submit for removal
my $response = $session->remove( $desired_cname );
```

Infoblox::Session->search( )

Use this method to search for DNS RPZ CNAMEIpAddress record objects in the Infoblox appliance. See Infoblox::Session->search() for parameters and return values.

Key References

```
Apply the following attributes to search for a specific DNS RPZ CNAME object.

  name     - Optional. A host name in string format (regular expression).
  view     - Optional. The DNS view in which the RPZ CNAMEIpAddress record is located. By default, all
DNS views are searched. But if you omit this attribute and specify a zone, the appliance searches the
'default' view only.
  zone     - Optional. A zone name in FQDN format.
  canonical - Optional. A canonical name (regular expression).
  comment - Optional. A comment in string format (regular expression).
  extattrs      - Optional. A hash reference containing extensible attributes.
  extensible_attributes - Optional. A hash reference containing extensible attributes.
```

For more information about searching extensible attributes, see Infoblox::Grid::ExtensibleAttributeDef/Searching Extensible Attributes.

Example

```
# search for all DNS RPZ CNAME objects that match "rpz.com" in the default DNS view
my @retrieved_objs = $session->search(
    object  => "Infoblox::DNS::RPZRecord::CNAME::IpAddress",
    name    => 'rpz\.com',
    view    => "default",
    comment => "this is a modified comment" );

# search for all DNS RPZ CNAMEIpAddress records in zone "rpz.com" of default view
my @retrieved_objs = $session->search(
    object => "Infoblox::DNS::RPZRecord::CNAME::IpAddress",
    zone   => "rpz\.com",
    view   => "default" );

# search DNS RPZ CNAMEIpAddress record with canonical name "cname.rpz.com" of default view
my @retrieved_objs = $session->search(
    object => "Infoblox::DNS::RPZRecord::CNAME::IpAddress",
    canonical => "cname\.rpz\.com",
    view   => "default" );

# search all DNS RPZ CNAMEIpAddress records with the extensible attribute 'Site'
my @retrieved_objs = $session->search(
   object => "Infoblox::DNS::RPZRecord::CNAME::IpAddress",
   extensible_attributes => { 'Site' => 'Santa Clara' });
```

METHODS

This section describes all the methods that you can use to configure and retrieve the attribute values of a DNS RPZ CNAMEIpAddress record.

canonical( )

Use this method to set or retrieve the canonical name.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

Canonical name can be network, host address, asterisk or empty.

You cannot specify canonical name value that is not Block IP Address (No Such Domain) Rule, Block IP Address (No Data) Rule, or Passthru IP Address Rules. You should use Infoblox::DNS::RPZRecord::CNAME::IpAddress or Infoblox::DNS::RPZRecord::CNAME::IpAddress objects for Substitute (IPv46 Address) Rule and Substitute (IPv6 Address) Ruless.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get canonical
my $canonical = $bind_cnameip->canonical();
#Modify canonical name
$bind_cnameip->canonical("10.6.1.2");
```

comment( )

Use this method to set or retrieve the descriptive comment.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

Desired comment in string format with a maximum of 256 bytes.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get comment
my $comment = $bind_cnameip->comment();
#Modify comment
$bind_cnameip->comment("Modifying the DNS RPZ CNAMEIpAddress record comment");
```

disable( )

Use this method to set or retrieve the disable flag of a DNS RPZ CNAMEIpAddress record.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

The default value for this field is false. The DNS RPZ CNAMEIpAddress record is enabled.

Parameter

Specify "true" to set the disable flag or "false" to deactivate/unset it.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get disable
my $disable = $bind_cnameip->disable();
#Modify disable
$bind_cnameip->disable("true");
```

extattrs( )

Use this method to set or retrieve the extensible attributes associated with a DNS RPZ CNAMEIpAddress record object.

Parameter

Valid value is a hash reference containing the names of extensible attributes and their associated values (Infoblox::Grid::Extattr objects).

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get extattrs
my $ref_extattrs = $bind_cnameip->extattrs();
#Modify extattrs
$bind_cnameip->extattrs({ 'Site' => $extattr1, 'Administrator' => $extattr2 });
```

extensible_attributes( )

Use this method to set or retrieve the extensible attributes associated with a DNS RPZ CNAMEIpAddress record.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

For valid values for extensible attributes, see Infoblox::Grid::ExtensibleAttributeDef/Extensible Attribute Values.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get extensible attributes
my $ref_extensible_attributes = $bind_cnameip->extensible_attributes();
#Modify extensible attributes
$bind_cnameip->extensible_attributes({ 'Site' => 'Santa Clara', 'Administrator' => [ 'Peter', 'Tom'
] });
```

name( )

Use this method to set or retrieve the host name.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

A name in FQDN (Fully Qualified Domain Name) format. The FQDN consists of the network address in CIDR notation or host IP address followed by the domain name (example: 10.0.0.0/8.rpz.com, 1.2.3.4.rpz.com).

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get name
my $name = $bind_cnameip->name();
#Modify name
$bind_cnameip->name("4.3.2.1.rpz.com");
```

ttl( )

Use this method to set or retrieve the Time to Live (TTL) value.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

The default value is undefined which indicates that the record inherits the TTL value of the zone.

Specify a TTL value to override the TTL value at the zone level.

Parameter

A 32-bit integer (range from 0 to 4294967295) that represents the duration, in seconds, that the record is cached. Zero indicates

that the record should not be cached.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get ttl
my $ttl = $bind_cnameip->ttl();
#Modify ttl
$bind_cnameip->ttl(1800);
#Un-override ttl
$bind_cnameip->ttl(undef);
```

views( )

Use this method to set or retrieve the view of the DNS RPZ CNAMEIpAddress record.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

The default value is the "default" view, which means the DNS RPZ CNAMEIpAddress record is located in the default view.

Parameter

An array reference of defined Infoblox::DNS::View objects.

Note that the array size must be 1.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get views
my $ref_views = $bind_cnameip->views();
#Modify views, an array of Infoblox::DNS::View objects
$bind_cnameip->views([$view1]);
```

zone( )

Use this method to retrieve the zone name of a DNS RPZ CNAMEIpAddress record. This method is read-only and cannot be set.

Parameter

None

Returns

Returns the attribute value.

Example

```
# Get zone
my $zone = $bind_cnameip->zone();
```

rp_zone( )

Use this method to set or retrieve the zone object of a DNS RPZ CNAMEIpAddress record.

Parameter

An Infoblox::DNS::Zone object.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
# Get rp_zone
my $rp_zone = $bind_cnameip->rp_zone();
```

```
#Modify rp_zone, reference of Infoblox::DNS::Zone object
$bind_cnameip->rp_zone($response_policy_zone);
```

is_ipv4( )

Use this method to set or retrieve whether the record is an IPv4 record. This method is read-only and cannot be set.

Parameter

None

Returns

Use this method to retrieve whether the record is an IPv4 record. If the return value is "true", it is an IPv4 record. Ohterwise, it is an IPv6 record.

Example

```
#Get is_ipv4
my $ip_ipv4 = $bind_cnameip->is_ipv4();
```

SAMPLE CODE

The following sample code demonstrates the different functions that can be applied to an object, such as add, search, modify, and remove. This sample code also includes error handling for the operations.

#Preparation prior to a DNS RPZ CNAMEIpAddress record insertion

```
#PROGRAM STARTS: Include all the modules that will be used
use strict;
use Infoblox;

#Create a session to the Infoblox appliance
my $session = Infoblox::Session->new(
    master   => "192.168.1.2",
    username => "admin",
    password => "infoblox"
);
unless ($session) {
   die("Construct session failed: ",
       Infoblox::status_code() . ":" . Infoblox::status_detail());
}
print "Session created successfully\n";

#Create the zone prior to a DNS RPZ CNAMEIpAddress record insertion
my $zone = Infoblox::DNS::Zone->new(name => "rpz.com",
                                    rpz_policy => "GIVEN");
unless ($zone) {
die("Construct zone failed: ",
    Infoblox::status_code() . ":" . Infoblox::status_detail());
}
print "Zone object created successfully\n";

#Verify if the zone exists
my $object = $session->get(object => "Infoblox::DNS::Zone", name => "rpz.com");
unless ($object) {
print "Zone does not exist on server, safe to add the zone\n";
$session->add($zone)
   or die("Add zone failed: ",
          $session->status_code() . ":" . $session->status_detail());
}
print "Zone added successfully\n";
```

#Create a DNS RPZ CNAMEIpAddress record

```
my $view1 = Infoblox::DNS::View->new(
 name      => "default" );

#Construct a DNS RPZ CNAMEIpAddress record object
my $ip_address = Infoblox::DNS::RPZRecord::CNAME::IpAddress->new(
 name      => "10.6.0.0/16.rpz.com",
 canonical => "10.6.0.0/16",
 comment   => "this is a demo ip address record",
 ttl       => "33",
 views     => [$view1],
 rp_zone   => $zone);
```

```
unless ($ip_address) {
die("Construct DNS RPZ CNAMEIpAddress record failed: ",
    Infoblox::status_code() . ":" . Infoblox::status_detail());
}
print "DNS RPZ CNAMEIpAddress record object created successfully\n";

#Add the DNS RPZ CNAMEIpAddress record object to the Infoblox appliance through a session
$session->add($ip_address)
 or die("Add DNS RPZ CNAMEIpAddress record failed: ",
        $session->status_code() . ":" . $session->status_detail());
print "DNS RPZ CNAMEIpAddress record object added to server successfully\n";
```

#Search for a specific DNS RPZ CNAMEIpAddress record

```
#Search all RPZ CNAMEIpAddress records that match "rpz.com"
my @retrieved_objs = $session->search(
 object => "Infoblox::DNS::RPZRecord::CNAME::IpAddress",
 name   => 'rpz\.com'
);
my $object = $retrieved_objs[0];
unless ($object) {
 die("Search DNS RPZ CNAMEIpAddress record failed: ",
     $session->status_code() . ":" . $session->status_detail());
}
print "Search DNS RPZ CNAMEIpAddress record object found at least 1 matching entry\n";

#Search all DNS RPZ CNAMEIpAddress records that start with "bind" and end with ".rpz.com"
my @retrieved_objs = $session->search(
 object => "Infoblox::DNS::RPZRecord::CNAME::IpAddress",
 name   => '^10\.6.*\.rpz\.com'
);
my $object = $retrieved_objs[0];
unless ($object) {
 die("Search DNS RPZ CNAMEIpAddress record failed: ",
     $session->status_code() . ":" . $session->status_detail());
}
print "Search DNS RPZ CNAMEIpAddress record object using regexp found at least 1 matching entry\n";
```

#Get and modify a DNS RPZ CNAMEIpAddress record

```
#Get DNS RPZ CNAMEIpAddress record through the session
my @retrieved_objs = $session->get(
 object => "Infoblox::DNS::RPZRecord::CNAME::IpAddress",
 name   => "10.6.0.0/16.rpz.com",
 view   => "default"
);
my $object = $retrieved_objs[0];
unless ($object) {
 die("Get DNS RPZ CNAMEIpAddress record failed: ",
     $session->status_code() . ":" . $session->status_detail());
}
print "Get DNS RPZ CNAMEIpAddress record object found at least 1 matching entry\n";

#Modify one of the attributes of the specified DNS RPZ CNAMEIpAddress record
$object->disable("true");

#Apply the changes
$session->modify($object)
 or die("Modify DNS RPZ CNAMEIpAddress record failed: ",
        $session->status_code() . ":" . $session->status_detail());
print "DNS RPZ CNAMEIpAddress record object modified successfully \n";
```

#Remove a DNS RPZ CNAMEIpAddress record

```
#Get DNS RPZ CNAMEIpAddress record through the session
my @retrieved_objs = $session->get(
 object => "Infoblox::DNS::RPZRecord::CNAME::IpAddress",
 name   => "10.6.0.0/16.rpz.com",
 view   => "default"
);
$object = $retrieved_objs[0];
unless ($object) {
 die("Get DNS RPZ CNAMEIpAddress record failed: ",
     $session->status_code() . ":" . $session->status_detail());
}
print "Get DNS RPZ CNAMEIpAddress record object found at least 1 matching entry\n";
```

```
#Submit the object for removal
$session->remove( $object )
or die("Remove DNS RPZ CNAMEIpAddress record failed: ",
     $session->status_code() . ":" . $session->status_detail());
print "DNS RPZ CNAMEIpAddress record object removed successfully \n";

####PROGRAM ENDS####
```

AUTHOR

Infoblox Inc. http://www.infoblox.com/

SEE ALSO

Infoblox::DNS::View, Infoblox::DNS::Zone, Infoblox::Session, Infoblox::Session->get(), Infoblox::Session->search(), Infoblox::Session->add(), Infoblox::Session->remove(), Infoblox::Session->modify()

COPYRIGHT

Copyright (c) 2017 Infoblox Inc.

Infoblox::DNS::RPZRecord::CNAME::IpAddressDN - DNS Substitute Domain Name rule object.

- NAME
- DESCRIPTION
- CONSTRUCTOR
- SESSION METHODS
  - Infoblox::Session->add( )
  - Infoblox::Session->get( )
  - Infoblox::Session->modify( )
  - Infoblox::Session->remove( )
  - Infoblox::Session->search( )
- METHODS
  - canonical( )
  - comment( )
  - disable( )
  - extattrs( )
  - extensible_attributes( )
  - name( )
  - ttl( )
  - views( )
  - zone( )
  - rp_zone( )
  - is_ipv4( )
- SAMPLE CODE
- AUTHOR
- SEE ALSO
- COPYRIGHT

NAME

Infoblox::DNS::RPZRecord::CNAME::IpAddressDN - DNS Substitute Domain Name (Based on IP Address) rule object.

DESCRIPTION

A DNS Substitute Domain Name (Based on IP Address) rule represents different Response Policy Zone (RPZ) rules, depending on the value of the canonical name.

This rule represents Substitute (Domain Name) Rule.

Note that you must specify only one view for the attribute "views".

CONSTRUCTOR

```
  my $bind_cname_dn =  Infoblox::DNS::RPZRecord::CNAME::IpAddressDN->new (
      canonical => $fqdn,                  #Required
      name      => $fqdn,                  #Required
      rp_zone   => $rp_zone,               #Required
      comment   => $string,                #Optional / Default is empty
      disable   => "true"|"false"          #Optional / Default "false"
      extattrs              => { $string => $extattr, ... },      #Optional / Default is undefined
      extensible_attributes => { $string => $string|$num, $string => [ $string|$num, ...], ... }, #Optional
/ Default is undefined
      ttl       => $num|undef,             #Optional / Default is undefined
      views     => [$View1],               #Optional / Default is "default" view
   );
```

You cannot set both extattrs and extensible_attributes attributes at the same time.

SESSION METHODS

This section describes all the methods in Infoblox::Session module that you can apply to a DNS Substitute Domain Name (Based on IP Address) rule object.

Infoblox::Session->add( )

Use this method to add an object to the Infoblox appliance. See Infoblox::Session->add() for parameters and return values.

Example

```
      #Construct a DNS Substitute Domain Name (Based on IP Address) rule object
```

```
my $bind_cname_dn = Infoblox::DNS::RPZRecord::CNAME::IpAddressDN->new (
    name      => "10.6.0.0/16.rpz.com",
    canonical => "domain.com",
    rp_zone   => response_policy_zone1,
    comment   => "this is a demo rule",
    disable   => "true",
    ttl       => "33",
    views     => [$view1]  );
# Submit for addition
my $response = $session->add( $bind_cname_dn );
```

Infoblox::Session->get( )

Use this method to retrieve all the matching objects from the Infoblox appliance. See Infoblox::Session->get() for parameters and return values.

Key References

```
 Apply the following attributes to get a specific DNS Substitute Domain Name (Based on IP Address)
rule object:

  name - Optional. A host name in string format.
  view - Optional. The DNS view in which the Substitute Domain Name (Based on IP Address) rule is
located. By default, all DNS views are searched. But if you omit this attribute and specify a zone,
the appliance searches the 'default' view only.
  zone - Optional. A zone name in FQDN format.
  canonical - Optional. A canonical name.
  extattrs     - Optional. A hash reference containing extensible attributes.
  extensible_attributes - Optional. A hash reference containing extensible attributes.
```

Example

```
# get DNS Substitute Domain Name (Based on IP Address) rule with name "1.2.3.4.rpz.com" of default
view
my @retrieved_objs = $session->get(
    object => "Infoblox::DNS::RPZRecord::CNAME::IpAddressDN",
    name   => "1.2.3.4.rpz.com",
    view   => "default" );

# get all DNS Substitute Domain Name (Based on IP Address) rules in zone "rpz.com" of default view
my @retrieved_objs = $session->get(
    object => "Infoblox::DNS::RPZRecord::CNAME::IpAddressDN",
    zone   => "rpz.com",
    view   => "default" );

# get DNS Substitute Domain Name (Based on IP Address) rule with canonical name "domain.com" of
default view
my @retrieved_objs = $session->get(
    object => "Infoblox::DNS::RPZRecord::CNAME::IpAddressDN",
    canonical => "domain\.com",
    view   => "default" );

# get all DNS Substitute Domain Name (Based on IP Address) rules with a given extensible attribute
my @retrieved_objs = $session->get(
    object => "Infoblox::DNS::RPZRecord::CNAME::IpAddressDN",
    extensible_attributes => { 'Site' => 'Santa Clara' });
```

Infoblox::Session->modify( )

Use this method to modify an object in the Infoblox appliance. See Infoblox::Session->modify() for parameters and return values.

Example

```
# Use method to modify the comment.
$bind_cname_dn->comment("this is a modified comment");
# Submit modification
my $response = $session->modify( $bind_cname_dn );
```

Infoblox::Session->remove( )

Use this method to remove an object from the Infoblox appliance. See Infoblox::Session->remove() for parameters and return values.

To remove a specific object, first use `get()` or `search()` to retrieve the specific object, and then submit this object for removal.

Example

```
#Get the objects with the same name
my @retrieved_objs = $session->get(
  object => "Infoblox::DNS::RPZRecord::CNAME::IpAddressDN",
```

```
    name    => "1.2.3.4.rpz.com",
    view    => "default");
 #Find the desired object from the retrieved list.
 my $desired_cname = $retrieved_objs[0];
 # Submit for removal
 my $response = $session->remove( $desired_cname );
```

Infoblox::Session->search( )

Use this method to search for DNS Substitute Domain Name (Based on IP Address) rule objects in the Infoblox appliance. See Infoblox::Session->search() for parameters and return values.

Key References

```
 Apply the following attributes to search for a specific DNS Substitute Domain Name (Based on IP
Address) rule object.

  name    - Optional. A host name in string format (regular expression).
  view    - Optional. The DNS view in which the Substitute Domain Name (Based on IP Address) rule is
located. By default, all DNS views are searched. But if you omit this attribute and specify a zone,
the appliance searches the 'default' view only.
  zone    - Optional. A zone name in FQDN format.
  canonical - Optional. A canonical name (regular expression).
  comment - Optional. A comment in string format (regular expression).
  extattrs     - Optional. A hash reference containing extensible attributes.
  extensible_attributes - Optional. A hash reference containing extensible attributes.
```

For more information about searching extensible attributes, see Infoblox::Grid::ExtensibleAttributeDef/Searching Extensible Attributes.

Example

```
 # search for all DNS Substitute Domain Name (Based on IP Address) rules that match "rpz.com" in the
default DNS view
 my @retrieved_objs = $session->search(
     object  => "Infoblox::DNS::RPZRecord::CNAME::IpAddressDN",
     name    => 'rpz\.com',
     view    => "default" );

 # search for all DNS Substitute Domain Name (Based on IP Address) rules in zone "rpz.com" of default
view
 my @retrieved_objs = $session->search(
     object => "Infoblox::DNS::RPZRecord::CNAME::IpAddressDN",
     zone   => "rpz\.com",
     view   => "default" );

 # search for all DNS Substitute Domain Name (Based on IP Address) rules with canonical name that
match "domain.com" of default view
 my @retrieved_objs = $session->search(
     object => "Infoblox::DNS::RPZRecord::CNAME::IpAddressDN",
     canonical => "domain\.com",
     view   => "default" );

 # search all DNS Substitute Domain Name (Based on IP Address) rules with the extensible attribute
'Site'
 my @retrieved_objs = $session->search(
    object => "Infoblox::DNS::RPZRecord::CNAME::IpAddressDN",
    extensible_attributes => { 'Site' => 'Santa Clara' });
```

METHODS

This section describes all the methods that you can use to configure and retrieve the attribute values of a DNS Substitute Domain Name (Based on IP Address) rule.

canonical( )

Use this method to set or retrieve the canonical name.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

Canonical name in FQDN (Fully Qualified Domain Name) format. The FQDN consists of the hostname followed by the domain name (example: abc.com)

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get canonical
my $canonical = $bind_cname_dn->canonical();
#Modify canonical name
$bind_cname_dn->canonical("domain.name");
```

comment( )

Use this method to set or retrieve the descriptive comment.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

Desired comment in string format with a maximum of 256 bytes.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get comment
my $comment = $bind_cname_dn->comment();
#Modify comment
$bind_cname_dn->comment("Modifying the DNS Substitute Domain Name (Based on IP Address) rule
comment");
```

disable( )

Use this method to set or retrieve the disable flag of a DNS Substitute Domain Name (Based on IP Address) rule.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

The default value for this field is false. The DNS Substitute Domain Name (Based on IP Address) rule is enabled.

Parameter

Specify "true" to set the disable flag or "false" to deactivate/unset it.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get disable
my $disable = $bind_cname_dn->disable();
#Modify disable
$bind_cname_dn->disable("true");
```

extattrs( )

Use this method to set or retrieve the extensible attributes associated with a DNS Substitute Domain Name (Based on IP Address) rule object.

Parameter

Valid value is a hash reference containing the names of extensible attributes and their associated values (Infoblox::Grid::Extattr objects).

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get extattrs
my $ref_extattrs = $bind_cname_dn->extattrs();
#Modify extattrs
$bind_cname_dn->extattrs({ 'Site' => $extattr1, 'Administrator' => $extattr2 });
```

extensible_attributes( )

Use this method to set or retrieve the extensible attributes associated with a DNS Substitute Domain Name (Based on IP Address) rule.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

For valid values for extensible attributes, see Infoblox::Grid::ExtensibleAttributeDef/Extensible Attribute Values.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get extensible attributes
my $ref_extensible_attributes = $bind_cname_dn->extensible_attributes();
#Modify extensible attributes
$bind_cname_dn->extensible_attributes({ 'Site' => 'Santa Clara', 'Administrator' => [ 'Peter', 'Tom'
] });
```

name( )

Use this method to set or retrieve the host name.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

A name in FQDN (Fully Qualified Domain Name) format. The FQDN consists of the network address in CIDR notation or host IP address followed by the domain name (example: 10.0.0.0/8.rpz.com, 1.2.3.4.rpz.com).

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get name
my $name = $bind_cname_dn->name();
#Modify name
$bind_cname_dn->name("4.3.2.1.rpz.com");
```

ttl( )

Use this method to set or retrieve the Time to Live (TTL) value.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

The default value is undefined which indicates that the rule inherits the TTL value of the zone.

Specify a TTL value to override the TTL value at the zone level.

Parameter

A 32-bit integer (range from 0 to 4294967295) that represents the duration, in seconds, that the rule is cached. Zero indicates that the rule should not be cached.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get ttl
my $ttl = $bind_cname_dn->ttl();
#Modify ttl
$bind_cname_dn->ttl(1800);
#Un-override ttl
$bind_cname_dn->ttl(undef);
```

views( )

Use this method to set or retrieve the view of the DNS Substitute Domain Name (Based on IP Address) rule.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

The default value is the "default" view, which means the DNS Substitute Domain Name (Based on IP Address) rule is located in the default view.

Parameter

An array reference of defined Infoblox::DNS::View objects.

Note that the array size must be 1.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get views
my $ref_views = $bind_cname_dn->views();
#Modify views, an array of Infoblox::DNS::View objects
$bind_cname_dn->views([$view1]);
```

zone( )

Use this method to retrieve the zone name of a DNS Substitute Domain Name (Based on IP Address) rule. This method is read-only and cannot be set.

Parameter

None

Returns

Returns the attribute value.

Example

```
# Get zone
my $zone = $bind_cname_dn->zone();
```

rp_zone( )

Use this method to set or retrieve the zone object of a DNS Substitute Domain Name (Based on IP Address) rule.

Parameter

An Infoblox::DNS::Zone object.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
# Get rp_zone
my $rp_zone = $bind_cname_dn->rp_zone();
#Modify rp_zone, reference of Infoblox::DNS::Zone object
$bind_cname_dn->rp_zone($response_policy_zone);
```

is_ipv4( )

Use this method to set or retrieve whether the DNS Substitute Domain Name (Based on IP Address) rule is an IPv4 rule. This method is read-only and cannot be set.

Parameter

None

Returns

Use this method to retrieve whether the DNS Substitute Domain Name (Based on IP Address) rule is an IPv4 rule. If the return value is "true", it is an IPv4 rule. Ohterwise, it is an IPv6 rule.

Example

```
#Get is_ipv4
my $ip_ipv4 = $bind_cname_dn->is_ipv4();
```

SAMPLE CODE

The following sample code demonstrates the different functions that can be applied to an object, such as add, search, modify, and remove. This sample code also includes error handling for the operations.

#Preparation prior to a DNS Substitute Domain Name (Based on IP Address) rule insertion

```
#PROGRAM STARTS: Include all the modules that will be used
use strict;
use Infoblox;

#Create a session to the Infoblox appliance
my $session = Infoblox::Session->new(
    master   => "192.168.1.2",
    username => "admin",
    password => "infoblox"
);
unless ($session) {
   die("Construct session failed: ",
       Infoblox::status_code() . ":" . Infoblox::status_detail());
}
print "Session created successfully\n";

#Create the zone prior to a DNS Substitute Domain Name (Based on IP Address) rule insertion
my $zone = Infoblox::DNS::Zone->new(name => "rpz.com",
                                    rpz_policy => "GIVEN");
unless ($zone) {
die("Construct zone failed: ",
    Infoblox::status_code() . ":" . Infoblox::status_detail());
}
print "Zone object created successfully\n";

#Verify if the zone exists
my $object = $session->get(object => "Infoblox::DNS::Zone", name => "rpz.com");
unless ($object) {
print "Zone does not exist on server, safe to add the zone\n";
$session->add($zone)
   or die("Add zone failed: ",
          $session->status_code() . ":" . $session->status_detail());
}
print "Zone added successfully\n";
```

#Create a DNS Substitute Domain Name (Based on IP Address) rule

```
my $view1 = Infoblox::DNS::View->new(
 name      => "default" );

#Construct a DNS Substitute Domain Name (Based on IP Address) rule object
my $domain_name = Infoblox::DNS::RPZRecord::CNAME::IpAddressDN->new(
 name      => "10.6.0.0/16.rpz.com",
 canonical => "test.com",
 comment   => "this is a demo DNS Substitute Domain Name (Based on IP Address) rule",
 ttl       => "33",
 views     => [$view1],
 rp_zone   => $zone);
unless ($domain_name) {
die("Construct DNS Substitute Domain Name (Based on IP Address) rule failed: ",
    Infoblox::status_code() . ":" . Infoblox::status_detail());
}
print "DNS Substitute Domain Name (Based on IP Address) rule object created successfully\n";

#Add the DNS Substitute Domain Name (Based on IP Address) rule object to the Infoblox appliance through a
session
$session->add($domain_name)
 or die("Add DNS Substitute Domain Name (Based on IP Address) rule failed: ",
        $session->status_code() . ":" . $session->status_detail());
print "DNS Substitute Domain Name (Based on IP Address) rule object added to server successfully\n";
```

#Search for a specific DNS Substitute Domain Name (Based on IP Address) rule

```
 #Search all DNS Substitute Domain Name (Based on IP Address) rules that match "rpz.com"
 my @retrieved_objs = $session->search(
  object => "Infoblox::DNS::RPZRecord::CNAME::IpAddressDN",
  name   => 'rpz\.com'
 );
 my $object = $retrieved_objs[0];
 unless ($object) {
  die("Search DNS Substitute Domain Name (Based on IP Address) rule failed: ",
      $session->status_code() . ":" . $session->status_detail());
 }
 print "Search DNS Substitute Domain Name (Based on IP Address) rule object found at least 1 matching
entry\n";

 #Search all DNS Substitute Domain Name (Based on IP Address) rules that start with "bind" and end with
".rpz.com"
 my @retrieved_objs = $session->search(
  object => "Infoblox::DNS::RPZRecord::CNAME::IpAddressDN",
  name   => '^10\.6.*\.rpz\.com'
 );
 my $object = $retrieved_objs[0];
 unless ($object) {
  die("Search DNS Substitute Domain Name (Based on IP Address) rule failed: ",
      $session->status_code() . ":" . $session->status_detail());
 }
 print "Search DNS Substitute Domain Name (Based on IP Address) rule object using regexp found at least 1
matching entry\n";
```

#Get and modify a DNS Substitute Domain Name (Based on IP Address) rule

```
 #Get DNS Substitute Domain Name (Based on IP Address) rule through the session
 my @retrieved_objs = $session->get(
  object => "Infoblox::DNS::RPZRecord::CNAME::IpAddressDN",
  name   => "10.6.0.0/16.rpz.com",
  view   => "default"
 );
 my $object = $retrieved_objs[0];
 unless ($object) {
  die("Get DNS Substitute Domain Name (Based on IP Address) rule failed: ",
      $session->status_code() . ":" . $session->status_detail());
 }
 print "Get DNS Substitute Domain Name (Based on IP Address) rule object found at least 1 matching
entry\n";

 #Modify one of the attributes of the specified DNS Substitute Domain Name (Based on IP Address) rule
 $object->disable("true");

 #Apply the changes
 $session->modify($object)
  or die("Modify DNS Substitute Domain Name (Based on IP Address) rule failed: ",
         $session->status_code() . ":" . $session->status_detail());
 print "DNS Substitute Domain Name (Based on IP Address) rule object modified successfully \n";
```

#Remove a DNS Substitute Domain Name (Based on IP Address) rule

```
 #Get DNS Substitute Domain Name (Based on IP Address) rule through the session
 my @retrieved_objs = $session->get(
  object => "Infoblox::DNS::RPZRecord::CNAME::IpAddressDN",
  name   => "10.6.0.0/16.rpz.com",
  view   => "default"
 );
 $object = $retrieved_objs[0];
 unless ($object) {
  die("Get DNS Substitute Domain Name (Based on IP Address) rule failed: ",
      $session->status_code() . ":" . $session->status_detail());
 }
 print "Get DNS Substitute Domain Name (Based on IP Address) rule object found at least 1 matching
entry\n";

 #Submit the object for removal
 $session->remove( $object )
 or die("Remove DNS Substitute Domain Name (Based on IP Address) rule failed: ",
      $session->status_code() . ":" . $session->status_detail());
 print "DNS Substitute Domain Name (Based on IP Address) rule object removed successfully \n";

 ####PROGRAM ENDS####
```

Infoblox::DNS::RPZRecord::CNAME::IpAddressDN - DNS Substitute Domain Name rule object.

AUTHOR

Infoblox Inc. http://www.infoblox.com/

SEE ALSO

Infoblox::DNS::View, Infoblox::DNS::Zone, Infoblox::Session, Infoblox::Session->get(), Infoblox::Session->search(), Infoblox::Session->add(), Infoblox::Session->remove(), Infoblox::Session->modify()

COPYRIGHT

Copyright (c) 2017 Infoblox Inc.

Infoblox::DNS::RPZRecord::MX - Response Policy Zone Rule object

- NAME
- DESCRIPTION
- CONSTRUCTOR
- SESSION METHODS
  - Infoblox::Session->add( )
  - Infoblox::Session->get( )
  - Infoblox::Session->modify( )
  - Infoblox::Session->remove( )
  - Infoblox::Session->search( )
- METHODS
  - comment( )
  - disable( )
  - exchanger( )
  - extattrs( )
  - extensible_attributes( )
  - name( )
  - pref( )
  - ttl( )
  - views( )
  - zone( )
  - rp_zone( )
- SAMPLE CODE
- AUTHOR
- SEE ALSO
- COPYRIGHT

NAME

Infoblox::DNS::RPZRecord::MX - Response Policy Zone (RPZ) Substitute (MX Record) Rule object

---

DESCRIPTION

An RPZ Substitute (MX Record) Rule maps a domain name to a mail exchanger. A mail exchanger is a server that either delivers or forwards mail.

This record represents the substitution rule for DNS MX records.

Note that you must specify only one view for the attribute "views".

CONSTRUCTOR

```
 my $mx = Infoblox::DNS::RPZRecord::MX->new(
     exchanger => $fqdn,                                                                  #Required
     name      => $fqdn,                                                                  #Required
     pref      => $num,                                                                   #Required
     rp_zone   => $rp_zone,                                                               #Required
     comment   => $string,                                                                #Optional
/ Default is empty
     disable   => "true" | "false",                                                       #Optional
/ Default "false"
     extattrs              => { $string => $extattr, ... },      #Optional / Default is undefined
     extensible_attributes => { $string => $string|$num, $string => [ $string|$num, ...], ... }, #Optional
/ Default is undefined
     ttl       => $num | undef,                                                           #Optional
/ Default is undefined
     views     => [$View1]                                                                #Optional
/ Default is "default" view
 );
```

You cannot set both extattrs and extensible_attributes attributes at the same time.

SESSION METHODS

This section describes all the methods in Infoblox::Session module that you can apply to an Substitute (MX Record) Rule object.

Infoblox::Session->add( )

Use this method to add an object to the Infoblox appliance. See Infoblox::Session->add() for parameters and return values.

Example

```
#Construct an object
my $rpzmx = Infoblox::DNS::RPZRecord::MX->new(
    name      => "bind_mx.rpz.com",
    comment   => "add mx bind_mx.rpz.com",
    pref      => 1,
    exchanger => "exchanger.rpz.com",
    rp_zone   => $response_policy_zone,
);
# Submit for addition
my $response = $session->add( $rpzmx );
```

Infoblox::Session->get( )

Use this method to retrieve all the matching objects from the Infoblox appliance. See Infoblox::Session->get() for parameters and return values.

Key References

```
Apply the following attributes to get a specific Substitute (MX Record) Rule object:

  name      - Optional.  A domain name in string format
  view      - Optional.  The DNS view in which the Substitute (MX Record) Rule is located. By
default, all DNS views are searched. But if you omit this attribute and specify a zone, the appliance
searches the 'default' view only.
  zone      - Optional. A zone name in FQDN format.
  exchanger - Optional. The exchanger of the Substitute (MX Record) Rule.
  pref      - Optional. The Preference of the Substitute (MX Record) Rule.
  extattrs  - Optional. A hash reference containing extensible attributes.
  extensible_attributes - Optional. A hash reference containing extensible attributes.
```

Example

```
 my @retrieved_objs = $session->get(
    object    => "Infoblox::DNS::RPZRecord::MX",
    name      => "bind_mx.rpz.com",
    view      => "default" );

 # get all Substitute (MX Record) Rules in the "rpz.com" zone of the default view
 my @retrieved_objs = $session->get(
    object    => "Infoblox::DNS::RPZRecord::MX",
    zone      => "rpz.com",
    view      => "default" );

# get all Substitute (MX Record) Rules with a given extensible attribute
my @retrieved_objs = $session->get(
    object => "Infoblox::DNS::RPZRecord::MX",
    extensible_attributes => { 'Site' => 'Santa Clara' });
```

Infoblox::Session->modify( )

Use this method to modify an object in the Infoblox appliance. See Infoblox::Session->modify() for parameters and return values.

Example

```
#Use method to modify the comment.
$rpzmx->comment("this is a modified comment");
# Submit modification
my $response = $session->modify( $rpzmx );
```

Infoblox::Session->remove( )

Use this method to remove an object from the Infoblox appliance. See Infoblox::Session->remove() for parameters and return values.

To remove a specifc object, first use `get()` or `search()` to retrieve the specific object, and then submit this object for removal.

Example

```
# Get the object under the same name
my @retrieved_objs = $session->get(
    name      => "bind_mx.rpz.com",
    comment   => "add mx bind_mx.rpz.com",
    pref      => 1,
    exchanger => "exchanger.rpz.com",
    view      => "default" );
# find the desired object from retrieved list.
my $desired_mx = $retrieved_objs[0];
# Submit for removal
```

```
my $response = $session->remove( $desired_mx );
```

Infoblox::Session->search( )

Use this method to search for Substitute (MX Record) Rule objects in the Infoblox appliance. See Infoblox::Session->search() for parameters and return values.

Key References

```
Apply the following attributes to search for an Substitute (MX Record) Rule object:

  name      - Optional. A domain name in string format (regular expression).
  view      - Optional. The DNS view in which the Substitute (MX Record) Rule is located. By default,
all DNS views are searched. But if you omit this attribute and specify a zone, the appliance searches
the 'default' view only.
  zone      - Optional. A zone name in FQDN format.
  comment   - Optional. A comment in string format (regular expression).
  exchanger - Optional. The exchanger of the Substitute (MX Record) Rule (regular expression).
  pref      - Optional. The Preference of the Substitute (MX Record) Rule (regular expression).
  extattrs  - Optional. A hash reference containing extensible attributes.
  extensible_attributes - Optional. A hash reference containing extensible attributes.
```

For more information about searching extensible attributes, see Infoblox::Grid::ExtensibleAttributeDef/Searching Extensible Attributes.

Example

```
# search for all Substitute (MX Record) Rule objects that match "rpz.com" in the default DNS view
    my @retrieved_objs = $session->search(
    object => "Infoblox::DNS::RPZRecord::MX",
    name   => 'rpz\.com',
    view   => "default" );

# search for all Substitute (MX Record) Rule objects in the rpz.com zone of the default DNS view
    my @retrieved_objs = $session->search(
    object => "Infoblox::DNS::RPZRecord::MX",
    zone   => "rpz.com",
    view   => "default" );

# search all Substitute (MX Record) Rules with a given extensible attribute 'Site'
my @retrieved_objs = $session->search(
   object => "Infoblox::DNS::RPZRecord::MX",
   extensible_attributes => { 'Site' => 'Santa Clara' });
```

METHODS

This section describes all the methods that you can use to configure and retrieve the attribute values of an Substitute (MX Record) Rule.

comment( )

Use this method to add or retrieve a descriptive comment.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

Enter a descriptive comment for this record in string format with a maximum of 256 bytes.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
# Getting comment
my $comment = $rpzmx->comment();
# Modifying comment
$rpzmx->comment("add mx2.rpz.com");
```

disable( )

Use this method to set or retrieve the disable flag of a DNS record.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

The default value for this field is false. The DNS record is enabled.

Parameter

Specify "true" to set the disable flag or "false" to deactivate/unset it.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
# Getting disable
my $disable = $rpzmx->disable()
# Modifying disable
$rpzmx->disable("true");
```

exchanger( )

Use this method to set or retrieve the exchanger.

The name of the mail exchanger in Fully Qualified domain name (FQDN) format.

Parameter

Text with the name of the mail exchanger for the Substitute (MX Record) Rule.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails. If you did not specify a parameter, the method returns the attribute value.

Example

```
# Getting exchanger
my $exchanger = $rpzmx->exchanger();
# Modifying exchanger
$rpzmx->exchanger("exchanger2.rpz.com");
```

extattrs( )

Use this method to set or retrieve the extensible attributes associated with a Substitute (MX Record) Rule object.

Parameter

Valid value is a hash reference containing the names of extensible attributes and their associated values (Infoblox::Grid::Extattr objects).

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get extattrs
my $ref_extattrs = $rpzmx->extattrs();
#Modify extattrs
$rpzmx->extattrs({ 'Site' => $extattr1, 'Administrator' => $extattr2 });
```

extensible_attributes( )

Use this method to set or retrieve the extensible attributes associated with an Substitute (MX Record) Rule.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

For valid values for extensible attributes, see Infoblox::Grid::ExtensibleAttributeDef/Extensible Attribute Values.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get extensible attributes
my $ref_extensible_attributes = $rpzmx->extensible_attributes();
#Modify extensible attributes
$rpzmx->extensible_attributes({ 'Site' => 'Santa Clara', 'Administrator' => [ 'Peter', 'Tom' ] });
```

name( )

Use this method to set or retrieve the domain name.

The name of the Substitute (MX Record) Rule object in Fully Qualified domain name (FQDN) format.

Parameter

Text with the name for the Substitute (MX Record) Rule.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Getting name
my $name = $rpzmx->name();
# Modifying name
$rpzmx->name("mx2.rpz.com");
```

pref( )

Use this method to set or retrieve the Preference value.

The preference value of the Substitute (MX Record) Rule object. The preference value of the RPZ Substitute (MX Record) Rule for the zone range from 0 to 65535. Lower values are more preferred.

Parameter

Enter an unsigned integer between 0-65535. The priority determines the order in which a client attempts to contact the target mail exchanger. The highest priority is 0 and is queried first.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
# Getting pref
my $pref = $rpzmx->pref();
# Modifying pref
$rpzmx->pref(3);
```

ttl( )

Use this method to configure or retrieve the Time to Live (TTL) value.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

The default value is undefined which indicates that the record inherits the TTL value of the zone.

Specify a TTL value to override the TTL value at the zone level.

Parameter

A 32-bit integer (range from 0 to 4294967295) that represents the duration in seconds that the record is cached. Zero indicates that the record should not be cached.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get ttl
my $ttl = $rpzmx->ttl();
#Modify ttl
```

```
$rpzmx->ttl(1800);
#Un-override ttl
$rpzmx->ttl(undef);
```

views( )

Use this method to specify or retrieve the view of the Substitute (MX Record) Rule.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

The default value is the 'default' view, which means the RPZ MX record is located in the default view.

Parameter

An array reference of defined Infoblox::DNS::View objects.

Note that the array size must be 1.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get views
my $ref_views = $rpzmx->views();
#Modify views, an array of Infoblox::DNS::View objects
$rpzmx->views([$view1]);
```

zone( )

Use this method to retrieve the zone name of an Substitute (MX Record) Rule. This method is read-only and cannot be set.

Parameter

None

Returns

Returns the attribute value.

Example

```
# Get zone
my $zone = $rpzmx->zone();
```

rp_zone( )

Use this method to set or retrieve the zone object of a DNS RPZ Substitute (MX Record) Rule.

Parameter

An Infoblox::DNS::Zone object.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
# Get rp_zone
my $rp_zone = $rpz_mx->rp_zone();
#Modify rp_zone, reference of Infoblox::DNS::Zone object
$rpz_mx->rp_zone($response_policy_zone);
```

SAMPLE CODE

The following sample code demonstrates the different functions that can be applied to an object, such as add, search, modify, and remove. This sample also includes error handling for the operations.

#Preparation prior to an Substitute (MX Record) Rule insertion

```
#PROGRAM STARTS: Include all the modules that will be used
use strict;
```

```
use Infoblox;

#Create a session to the Infoblox appliance
my $session = Infoblox::Session->new(
    master   => "192.168.1.2",
    username => "admin",
    password => "infoblox"
);
unless ($session) {
   die("Construct session failed: ",
       Infoblox::status_code() . ":" . Infoblox::status_detail());
}
print "Session created successfully\n";

#Create the zone prior to Substitute (MX Record) Rule insertion
my $zone = Infoblox::DNS::Zone->new(name => "rpz.com");
unless ($zone) {
   die("Construct zone failed: ",
     Infoblox::status_code() . ":" . Infoblox::status_detail());
}
print "Zone object created successfully\n";

#Verify if the zone exists
my $object = $session->get(object => "Infoblox::DNS::Zone", name => "rpz.com");
unless ($object) {
   print "Zone does not exist on server, safe to add the zone\n";
   $session->add($zone)
      or die("Add zone failed: ",
             $session->status_code() . ":" . $session->status_detail());
}
print "Zone added successfully\n";
```

#Create an Substitute (MX Record) Rule

```
#Construct an Substitute (MX Record) Rule object
my $rpzmx = Infoblox::DNS::RPZRecord::MX->new(
 name      => "bind_mx.rpz.com",
 comment   => "add mx bind_mx.rpz.com",
 pref      => 1,
 exchanger => "exchanger.rpz.com",
 rp_zone   => $zone,
);

unless ($rpzmx) {
die("Construct DNS record MX failed: ",
    Infoblox::status_code() . ":" . Infoblox::status_detail());
}
print "Substitute (MX Record) Rule object created successfully\n"; #Add the Substitute (MX Record) Rule
object to Infoblox appliance through a session
$session->add($rpzmx)
 or die("Add record MX failed: ",
        $session->status_code() . ":" . $session->status_detail());
print "Substitute (MX Record) Rule object added to server successfully\n";
```

#Search for a specific Substitute (MX Record) Rule

```
#Search all Substitute (MX Record) Rules that match "rpz.com"
my @retrieved_objs = $session->search(
 object => "Infoblox::DNS::RPZRecord::MX",
 name   => 'rpz\.com'
);
my $object = $retrieved_objs[0];
unless ($object) {
 die("Search record MX failed: ",
     $session->status_code() . ":" . $session->status_detail());
}
print "Search Substitute (MX Record) Rule object found at least 1 matching entry\n";

#Search all Substitute (MX Record) Rules that start with "bind" and end with "rpz.com"
my @retrieved_objs = $session->search(
 object => "Infoblox::DNS::RPZRecord::MX",
 name   => '^bind.*\.rpz\.com$'
);
my $object = $retrieved_objs[0];
unless ($object) {
 die("Search record MX failed: ",
     $session->status_code() . ":" . $session->status_detail());
```

```
}
print "Search Substitute (MX Record) Rule object using regexp found at least 1 matching entry\n";
```

#Get and modify an Substitute (MX Record) Rule record

```
#Get Substitute (MX Record) Rule through the session
my  @retrieved_objs = $session->get(
 object    => "Infoblox::DNS::RPZRecord::MX",
 name      => "bind_mx.rpz.com",
 pref      => 1,
 exchanger => "exchanger.rpz.com",
);

my $object = $retrieved_objs[0];
unless ($object) {
 die("Get record MX failed: ",
     $session->status_code() . ":" . $session->status_detail());
}
print "Get Substitute (MX Record) Rule object found at least 1 matching entry\n";

#Modify one of the attributes of the specified Substitute (MX Record) Rule
$object->exchanger("exc.rpz.com");

#Apply the changes
$session->modify($object)
 or die("Modify record MX failed: ",
        $session->status_code() . ":" . $session->status_detail());
print "Substitute (MX Record) Rule object modified successfully \n";
```

#Remove an Substitute (MX Record) Rule record

```
#Get Substitute (MX Record) Rule through the session
my @retrieved_objs = $session->get(
  object => "Infoblox::DNS::RPZRecord::MX",
  name   => "bind_mx.rpz.com"
);
my $object = $retrieved_objs[0];
unless ($object) {
 die("Get record MX failed: ",
     $session->status_code() . ":" . $session->status_detail());
}
print "Get Substitute (MX Record) Rule object found at least 1 matching entry\n";

#Submit the object for removal
$session->remove($object)
 or die("Remove record MX failed: ",
     $session->status_code() . ":" . $session->status_detail());
print "Substitute (MX Record) Rule object removed successfully \n";

####PROGRAM ENDS####
```

AUTHOR

Infoblox Inc. http://www.infoblox.com/

SEE ALSO

Infoblox::DNS::View, Infoblox::DNS::Zone, Infoblox::Session, Infoblox::Session->get(), Infoblox::Session->search(), Infoblox::Session->add(), Infoblox::Session->remove(), Infoblox::Session->modify()

COPYRIGHT

Copyright (c) 2017 Infoblox Inc.

Infoblox::DNS::RPZRecord::NAPTR - Response Policy Zone Rule object.

- NAME
- DESCRIPTION
- CONSTRUCTOR
- SESSION METHODS
  - Infoblox::Session->add( )
  - Infoblox::Session->get( )
  - Infoblox::Session->modify( )
  - Infoblox::Session->remove( )
  - Infoblox::Session->search( )
- METHODS
  - comment( )
  - disable( )
  - extattrs( )
  - extensible_attributes( )
  - flags( )
  - name( )
  - order( )
  - preference( )
  - regexp( )
  - replacement( )
  - services( )
  - ttl( )
  - views( )
  - zone( )
  - rp_zone( )
- SAMPLE CODE
- AUTHOR
- SEE ALSO
- COPYRIGHT

NAME

Infoblox::DNS::RPZRecord::NAPTR - Response Policy Zone (RPZ) Substitute (NAPTR Record) Rule object.

DESCRIPTION

An RPZ Substitute (NAPTR Record) Rule object represents the substitution rule for DNS Naming Authority Pointer (NAPTR) records. This rule specifies a regular expression-based rewrite rule that, when applied to an existing string, produces a new domain name or URI.

Note that you must specify only one view for the attribute "views".

CONSTRUCTOR

```
 my $rpz_naptr = Infoblox::DNS::RPZRecord::NAPTR->new(
     name            => $fqdn,             #Required
     order           => $num,              #Required
     preference      => $num,              #Required
     rp_zone         => $rp_zone,          #Required
     flags           => $string,           #Optional
     services        => $string,           #Optional
     regexp          => $string,           #Optional
     replacement     => $string            #Required
     comment         => $string,           #Optional / Default is empty
     disable         => "true" | "false", #Optional / Default is "false"
     extattrs              => { $string => $extattr, ... },      #Optional / Default is undefined
     extensible_attributes => { $string => $string|$num, $string => [ $string|$num, ...], ... }, #Optional
/ Default is undefined
     ttl             => $num | undef,      #Optional / Default is undefined
     views           => [$View1],          #Optional / Default is "default" view
 );
```

Note that you must specify only one DNS view for the attribute "views".

You cannot set both extattrs and extensible_attributes attributes at the same time.

SESSION METHODS

This section describes all the methods in an Infoblox::Session module that can be applied to an Substitute (NAPTR Record) Rule object.

Infoblox::Session->add( )

Use this method to add an object to the Infoblox appliance. See Infoblox::Session->add() for parameters and return values.

Example

```
#Construct an object
my $rpz_naptr = Infoblox::DNS::RPZRecord::NAPTR->new(
    name            => 'naptr.rpz.com',
    order           => 100,
    preference      => 10,
    flags           => 'U',
    services        => 'http+E2U',
    regexp          => '!http://my[.](.*)!1!i',
    replacement     => 'domain.com',
    comment         => 'Sample Substitute (NAPTR Record) Rule',
    disable         => 'false',
    ttl             => 84600,
    views           => [$view1],
    rp_zone   => $response_policy_zone,
);
# Submit for addition
my $response = $session->add( $rpz_naptr );
```

Infoblox::Session->get( )

Use this method to retrieve the existing objects from an Infoblox appliance. See Infoblox::Session->get() for parameters and return values.

Key Reference

```
Apply the following attributes to get specific Substitute (NAPTR Record) Rule objects:

    comment         - Optional. Descriptive comment in string format
    extattrs        - Optional. A hash reference containing extensible attributes.
    extensible_attributes - Optional. A hash reference containing extensible attributes.
    flags           - Optional. Flags in string format
    name            - Optional. FQDN of the record in string format
    order           - Optional. Order parameter (unsigned integer)
    preference      - Optional. Preference parameter (unsigned integer)
    replacement     - Optional. Replacement field in string format
    services        - Optional. Services field in string format
    view            - Optional. The DNS view in which the Substitute (NAPTR Record) Rule is located.
By default, all DNS views are searched. But if you omit this attribute and specify a zone, the
appliance searches the 'default' view only.
    zone            - Optional. A zone name in FQDN format.
```

Example

```
my @retrieved_objs = $session->get(
    object => "Infoblox::DNS::RPZRecord::NAPTR",
    name   => "naptr1.rpz.com",
    view   => "default" );

#get all DNS A records with the extensible attribute 'Site'
my @retrieved_objs = $session->get(
   object => "Infoblox::DNS::RPZRecord::NAPTR",
   extensible_attributes => { 'Site' => 'Santa Clara' });
```

Infoblox::Session->modify( )

Use this method to modify an object in the Infoblox appliance. See Infoblox::Session->modify() for parameters and return values.

Example

```
# Use this method to modify the comment.
$rpz_naptr->comment("this is a modified comment");
# Submit the modification
my $response = $session->modify( $rpz_naptr );
```

Infoblox::Session->remove( )

Use this method to submit the removal of an object from the Infoblox appliance. See Infoblox::Session->remove() for parameters and return values.

To remove a specific object, first use `get()` or `search()` to retrieve the object, and then submit it for removal.

Example

```
# Get the objects with the same name
my @retrieved_objs = $session->get(
    object => "Infoblox::DNS::RPZRecord::NAPTR",
    name   => "naptr1.rpz.com",
    view   => "default" );
# Find the desired object in the retrieved list.
my $desired_rec = $retrieved_objs[0];
# Submit for removal
my $response = $session->remove( $desired_rec );
```

Infoblox::Session->search( )

Use this method to retrieve the existing objects from an Infoblox appliance. See Infoblox::Session->search() for parameters and return values.

Key Reference

```
Apply the following attributes to search the Substitute (NAPTR Record) Rule objects:

    comment          - Optional. Descriptive comment (regular expression)
    extattrs         - Optional. A hash reference containing extensible attributes.
    extensible_attributes - Optional. A hash reference containing extensible attributes.
    flags            - Optional. Flags in string format (regular expression)
    name             - Optional. FQDN of the record in string format (regular expression)
    order            - Optional. Order parameter (unsigned integer, exact search)
    preference       - Optional. Preference parameter (unsigned integer, exact search)
    replacement      - Optional. Replacement field in string format (regular expression)
    services         - Optional. Services field in string format (regular expression)
    view             - Optional. The DNS view in which the Substitute (NAPTR Record) Rule is located.
By default, all DNS views are searched. But if you omit this attribute and specify a zone, the
appliance searches the 'default' view only. Only exact search is supported.
    zone             - Optional. A zone name in FQDN format (exact search)
```

For more information about searching extensible attributes, see Infoblox::Grid::ExtensibleAttributeDef/Searching Extensible Attributes.

Example

```
# search for all Substitute (NAPTR Record) Rule objects that match "rpz.com" in the default DNS view
my @retrieved_objs = $session->search(
    object => "Infoblox::DNS::RPZRecord::NAPTR",
    name   => '.*rpz\.com',
    view   => "default" );

# search for all Substitute (NAPTR Record) Rules in the "rpz.com" zone of the default DNS view
my @retrieved_objs = $session->search(
    object => "Infoblox::DNS::RPZRecord::NAPTR",
    zone   => "rpz.com",
    view   => "default" );

#get all Substitute (NAPTR Record) Rules with the extensible attribute 'Site'
my @retrieved_objs = $session->search(
  object => "Infoblox::DNS::RPZRecord::NAPTR",
  extensible_attributes => { 'Site' => 'Santa Clara' });
```

METHODS

This section describes all the methods that can be used to retrieve the attribute values of an Substitute (NAPTR Record) Rule object.

comment( )

Use this method to set or retrieve the descriptive comment of an Substitute (NAPTR Record) Rule object.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

Desired comment in string format with a maximum of 256 bytes.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get the descriptive comment
my $comment = $rpz_naptr->comment();
```

```
#Modify the comment
$rpz_naptr->comment("This is a new comment");
```

disable( )

Use this method to set or retrieve the disable flag of an Substitute (NAPTR Record) Rule.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

The default value for this field is false, which indicates that the Substitute (NAPTR Record) Rule is enabled.

Parameter

Specify "true" to set the disable flag or "false" to deactivate/unset it.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get the disable field
my $disable = $rpz_naptr->disable();
```

extattrs( )

Use this method to set or retrieve the extensible attributes associated with a Substitute (NAPTR Record) Rule object.

Parameter

Valid value is a hash reference containing the names of extensible attributes and their associated values (Infoblox::Grid::Extattr objects).

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get extattrs
my $ref_extattrs = $rpz_naptr->extattrs();
#Modify extattrs
$rpz_naptr->extattrs({ 'Site' => $extattr1, 'Administrator' => $extattr2 });
```

extensible_attributes( )

Use this method to set or retrieve the extensible attributes associated with an Substitute (NAPTR Record) Rule.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

For valid values for extensible attributes, see Infoblox::Grid::ExtensibleAttributeDef/Extensible Attribute Values.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get  extensible attributes
my $ref_extensible_attributes = $rpz_naptr->extensible_attributes();
#Modify extensible attributes
$rpz_naptr->extensible_attributes({ 'Site' => 'Santa Clara', 'Administrator' => [ 'Peter', 'Tom' ]
});
```

flags( )

Use this method to retrieve the flags that control the interpretation of the fields of an Substitute (NAPTR Record) Rule object.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

Currently supported values for the flags field are "U", "S", "P" and "A".

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get the flags field
my $flags = $rpz_naptr->flags();
#Modify flags
$rpz_naptr->flags("U");
```

name( )

Use this method to retrieve the FQDN (Fully Qualified Domain Name) of the Substitute (NAPTR Record) Rule.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

The name parameter is a string in FQDN format. The FQDN consists of the record name followed by the domain name (example: rec.abc.com). A record name can have a maximum of 256 bytes.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get the FQDN of the record
my $name = $rpz_naptr->name();
#Modify the FQDN of the record
$rpz_naptr->name("rec2.rpz.com");
```

order( )

Use this method to retrieve the order parameter of an Substitute (NAPTR Record) Rule. This parameter specifies the order in which the Substitute (NAPTR Record) Rules are applied when multiple rules are present.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

The order parameter is a 16-bit unsigned integer.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get the order parameter
my $order = $rpz_naptr->order();
#Modify the order
$rpz_naptr->order(200);
```

preference( )

Use this method to retrieve the preference field of an Substitute (NAPTR Record) Rule. The preference field determines the order Substitute (NAPTR Record) Rules are processed when multiple records with the same order parameter are present.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

The preference parameter is a 16-bit integer.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get the preference attribute value
my $preference = $rpz_naptr->preference();
#Modify preference
$rpz_naptr->preference(300);
```

regexp( )

Use this method to specify the regular expression-based rewriting rule of an Substitute (NAPTR Record) Rule. This should be a POSIX compliant regular expression, including the substitution rule and flags. Refer to RFC 2915 for the field syntax details.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

The valid parameter is a POSIX compliant regular expression or substitution expression.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get regexp
my $regexp = $rpz_naptr->regexp();
#Modify regexp
$rpz_naptr->regexp("!http://(.*)!1!i");
```

replacement( )

Use this method to retrieve the replacement field of an Substitute (NAPTR Record) Rule object. For nonterminal Substitute (NAPTR Record) Rules, this field specifies the next domain name to look up.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

The replacement field is in FQDN format.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get the replacement field
my $replacement = $rpz_naptr->replacement();
#Modify the replacement field
$rpz_naptr->replacement("__tcp__.rpz.com");
```

services( )

Use this field to specify services. The services field contains protocol and service identifiers, for example "http+E2U" or "SIPS+D2T".

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

The services field in string format.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get services
```

```
my $services = $rpz_naptr->services();
#Modify services
$rpz_naptr->services("http+E2U");
```

ttl( )

Use this method to set or retrieve the Time to Live (TTL) value.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

The default value is undefined, which indicates that the record inherits the TTL value of the zone.

Specify a TTL value to override the TTL value at the zone level.

Parameter

A 32-bit integer (from 0 to 4294967295) that represents the duration, in seconds, that the record is cached. Zero indicates that the record should not be cached.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get the TTL of the record
my $ttl = $rpz_naptr->ttl();
#Modify TTL
$rpz_naptr->ttl(900);
#Use zone ttl settings
$rpz_naptr->ttl(undef);
```

views( )

Use this method to set or retrieve the DNS view of the Substitute (NAPTR Record) Rule.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

The default value is the "default" view, which means the Substitute (NAPTR Record) Rule is located in the default DNS view.

Parameter

An array reference of defined Infoblox::DNS::View objects.

Note that the array size must be 1.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get views
my $ref_views = $rpz_naptr->views();
#Modify views, an array of Infoblox::DNS::View objects
$rpz_naptr->views([$view1]);
```

zone( )

Use this method to retrieve the zone name of an Substitute (NAPTR Record) Rule. This method is read-only and cannot be set.

Parameter

None

Returns

Returns the attribute value.

Example

```
# Get zone
my $zone = $rpz_naptr->zone();
```

rp_zone( )

Use this method to set or retrieve the zone object of a Substitute (NAPTR Record) Rule.

Parameter

An Infoblox::DNS::Zone object.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
# Get rp_zone
my $rp_zone = $rpz_naptr->rp_zone();
#Modify rp_zone, reference of Infoblox::DNS::Zone object
$rpz_naptr->rp_zone($response_policy_zone);
```

SAMPLE CODE

The following sample code demonstrates the session methods on an Substitute (NAPTR Record) Rule object.

#Preparation prior to an Substitute (NAPTR Record) Rule insertion

```
#PROGRAM STARTS: Include all the modules that will be used
use strict;
use Infoblox;
```

#Create a session to the Infoblox appliance

```
my $session = Infoblox::Session->new(
    master   => $host_ip,
    username => "admin",
    password => "infoblox"
);
unless ($session) {
   die("Construct session failed: ",
       $session->status_code() . ":" . $session->status_detail());
}
print "Session created successfully\n";

#Create the zone prior to an A record insertion
my $zone = Infoblox::DNS::Zone->new(name => "rpz.com");
unless ($zone) {
   die("Construct zone failed: ",
       Infoblox::status_code() . ":" . Infoblox::status_detail());
}
print "Zone object created successfully\n";

#Verify if the zone exists
my $object = $session->get(object => "Infoblox::DNS::Zone", name => "rpz.com");
unless ($object) {
   print "Zone does not exist on server, safe to add the zone\n";
   $session->add($zone)
      or die("Add zone failed: ",
             $session->status_code() . ":" . $session->status_detail());
}
print "Zone added successfully\n";
```

#Create a Substitute (NAPTR Record) Rule

```
#Construct a Substitute (NAPTR Record) Rule object
my $bind_naptr = Infoblox::DNS::RPZRecord::NAPTR->new(
    name     => "bind_naptr.rpz.com",
    comment  => " this is a demo Substitute (NAPTR Record) Rule ",
    services => "http+E2U",
    flags    => "U",
    regexp   => "!http://(.*)!1!i",
    preference => 100,
    order    => 10,
    replacement => '_http._tcp.rpz.com',
);
unless ($bind_naptr) {
   die("Construct Substitute (NAPTR Record) Rule failed: ",
       Infoblox::status code           Infoblox::status detail
```

```
                                () . ":" .                          ());
}
print "Substitute (NAPTR Record) Rule object created successfully\n";

#Add the Substitute (NAPTR Record) Rule object to the Infoblox appliance through a session
$session->add($bind_naptr)
    or die("Adding Substitute (NAPTR Record) Rule failed: ",
           $session->status_code() . ":" . $session->status_detail());
print "Substitute (NAPTR Record) Rule object added to server successfully\n";
```

#Search for a specific Substitute (NAPTR Record) Rule

```
#Search all Substitute (NAPTR Record) Rules in the zone
my @retrieved_objs = $session->search(
    object => "Infoblox::DNS::RPZRecord::NAPTR",
    name   => '.*\.rpz\.com'
);
my $object = $retrieved_objs[0];
unless ($object) {
    die("Search for Substitute (NAPTR Record) Rule failed: ",
        $session->status_code() . ":" . $session->status_detail());
}
print "Search Substitute (NAPTR Record) Rule object found at least 1 matching entry\n";

#Search for all Substitute (NAPTR Record) Rules that start with "bind" in the zone
my @retrieved_objs = $session->search(
    object => "Infoblox::DNS::RPZRecord::NAPTR",
    name   => 'bind.*\.rpz\.com'
);
my $object = $retrieved_objs[0];
unless ($object) {
    die("Search for Substitute (NAPTR Record) Rule failed: ",
        $session->status_code() . ":" . $session->status_detail());
}
print "Searching Substitute (NAPTR Record) Rule object using regexp found at least 1 matching entry\n";
```

#Get and modify a Substitute (NAPTR Record) Rule

```
#Get a record through the session
my @retrieved_objs = $session->get(
    object => "Infoblox::DNS::RPZRecord::NAPTR",
    name   => "bind_naptr.rpz.com"
);
my $object = $retrieved_objs[0];
unless ($object) {
    die("Get Substitute (NAPTR Record) Rule failed: ",
        $session->status_code() . ":" . $session->status_detail());
}
print "Get Substitute (NAPTR Record) Rule object found at least 1 matching entry\n";

#Modify some attributes of the specified Substitute (NAPTR Record) Rule
$object->preference(200);
$object->order(100);

#Apply the changes
$session->modify($object)
    or die("Modify Substitute (NAPTR Record) Rule failed: ",
           $session->status_code() . ":" . $session->status_detail());
print "Substitute (NAPTR Record) Rule object modified successfully \n";
```

#Remove a Substitute (NAPTR Record) Rule

```
#Get Substitute (NAPTR Record) Rule through the session
my @retrieved_objs = $session->get(
    object => "Infoblox::DNS::RPZRecord::NAPTR",
    name   => "bind_naptr.rpz.com"
);
my $object = $retrieved_objs[0];
unless ($object) {
    die("Get Substitute (NAPTR Record) Rule failed: ",
        $session->status_code() . ":" . $session->status_detail());
}
print "Get Substitute (NAPTR Record) Rule object found at least 1 matching entry\n";

#Submit the object for removal
$session->remove($object)
    or die("Remove Substitute (NAPTR Record) Rule failed: ",
```

```
        $session->status_code() . ":" . $session->status_detail());
 print "Substitute (NAPTR Record) Rule object removed successfully \n";

#Remove the zone

#Get Zone object through the session
@retrieved_objs = $session->get(
    object => "Infoblox::DNS::Zone",
    name   => "rpz.com"
);
$object = $retrieved_objs[0];
unless ($object) {
    die("Get zone failed: ",
        $session->status_code() . ":" . $session->status_detail());
}
print "Get zone object found at least 1 matching entry\n";

#Submit the object for removal
$session->remove($object)
    or die("Remove zone object failed: ",
        $session->status_code() . ":" . $session->status_detail());
print "Zone object removed successfully \n";

####PROGRAM ENDS####
```

AUTHOR

Infoblox Inc. http://www.infoblox.com/

SEE ALSO

Infoblox::Session, Infoblox::Session->add(), Infoblox::Session->get(),Infoblox::Session->modify(), Infoblox::Session->remove(), Infoblox::Session->search(), Infoblox::DNS::Zone

COPYRIGHT

Copyright (c) 2017 Infoblox Inc.

Infoblox::DNS::RPZRecord::PTR - Response Policy Zone Rule object.

- NAME
- DESCRIPTION
- CONSTRUCTOR
- SESSION METHODS
  - Infoblox::Session->add( )
  - Infoblox::Session->get( )
  - Infoblox::Session->modify( )
  - Infoblox::Session->remove( )
  - Infoblox::Session->search( )
- METHODS
  - comment( )
  - disable( )
  - extattrs( )
  - extensible_attributes( )
  - ipv4addr( )
  - ipv6addr( )
  - name( )
  - ptrdname( )
  - ttl( )
  - views( )
  - zone( )
  - rp_zone( )
- SAMPLE CODE
- AUTHOR
- SEE ALSO
- COPYRIGHT

NAME

Infoblox::DNS::RPZRecord::PTR - Response Policy Zone (RPZ) Substitute (PTR Record) Rule object.

DESCRIPTION

An RPZ Substitute (PTR Record) Rule object represents a Pointer (PTR) resource record. To define a specific address-to-name mapping, add an RPZ Substitute (PTR Record) Rule to a previously defined Response Policy Zone.

This record represents the substitution rule for DNS PTR records.

Note that you must specify only one view for the attribute "views".

CONSTRUCTOR

```
 #Constructor for an IPv4 Substitute (PTR Record) Rule object
 my $bind_ptr = Infoblox::DNS::RPZRecord::PTR->new (
        ipv4addr => $ipv4addr,              #Required
        ptrdname => $fqdn,                  #Required
        rp_zone  => $rp_zone,               #Required
        comment  => $string,                #Optional / Default is empty
        disable  => "true" | "false"        #Optional / Default "false"
        extattrs              => { $string => $extattr, ... },      #Optional / Default is undefined
        extensible_attributes => { $string => $string|$num, $string => [ $string|$num, ...], ... },
#Optional / Default is undefined
        last_discovered  => $string,        #Optional / Default is empty
        name     => $fqdn,                  #Optional
        ttl      => $num | undef,           #Optional / Default is undefined
        views    => [$View1],               #Optional / Default is "default" view
    );

 #Constructor for an IPv6 Substitute (PTR Record) Rule object
 my $bind_ptr = Infoblox::DNS::RPZRecord::PTR->new (
        ipv6addr => $ipv6addr,              #Required
        ptrdname => $fqdn,                  #Required
        rp_zone  => $rp_zone,               #Required
        comment  => $string,                #Optional / Default is empty
        disable  => "true" | "false"        #Optional / Default "false"
        extensible_attributes => { $string => $string|$num, $string => [ $string|$num, ...], ... },
#Optional / Default is undefined
        name     => $fqdn,                  #Optional
        ttl      => $nun | undef,           #Optional / Default is undefined
```

```
    views     => [$View1],              #Optional / Default is "default" view
);
```

You cannot set both extattrs and extensible_attributes attributes at the same time.

SESSION METHODS

This section describes all the methods in Infoblox::Session that you can apply to a Substitute (PTR Record) Rule object.

Infoblox::Session->add( )

Use this method to add an object to the Infoblox appliance. See Infoblox::Session->add() for parameters and return values.

Example

```
#Constructor for an IPv4 Substitute (PTR Record) Rule object
my $bind_ptr = Infoblox::DNS::RPZRecord::PTR->new (
    ipv4addr => "10.0.0.1",
    ptrdname => "bind_ptr.domain.com",
    comment => "this is a demo RPZ Substitute (PTR Record) Rule",
    disable => "true",
    ttl => "33",
    views => [$view1],
    rp_zone => $response_policy_zone  );
 # Submit for addition
 my $response = $session->add( $bind_ptr );

#Constructor for an IPv6 Substitute (PTR Record) Rule object
my $bind_ptr = Infoblox::DNS::RPZRecord::PTR->new(
    ptrdname     => "ip6host.this.com",
    comment  => " this is a demo ipv6 RPZ Substitute (PTR Record) Rule ",
    disable => "true",
    ttl => "333",
    ipv6addr => "1234:5678:9abc:1000::1",
    rp_zone => $response_policy_zone  );
 # Submit for addition
 my $response = $session->add( $bind_ptr );
```

Infoblox::Session->get( )

Use this method to retrieve existing objects from the Infoblox appliance. See Infoblox::Session->get() for parameters and return values.

Key Reference

```
 Apply the following attributes to get a specific Substitute (PTR Record) Rule object:

 name | ipv4addr | ipv6addr | ptrdname - Optional. name/ptrdname is in FQDN format. ipv4addr is an
IPv4 address with a 32-bit number in dotted decimal notation. ipv6addr is the IPv6 (128 bits) address
of the Substitute (PTR Record) Rule. ptrdname is the domain name of the Substitute (PTR Record) Rule.
 view - Optional. The DNS view in which the RPZ Substitute (PTR Record) Rule is located. By default,
all DNS views are searched. But if you omit this attribute and specify a zone, the appliance searches
the 'default' view only.
 zone - Optional. A zone name in FQDN format.
 extattrs     - Optional. A hash reference containing extensible attributes.
 extensible_attributes - Optional. A hash reference containing extensible attributes.
```

Example

```
#To get an IPv4 Substitute (PTR Record) Rule object by name
my @retrieved_objs = $session->get(
    object => "Infoblox::DNS::RPZRecord::PTR",
    name   => "1.0.0.10.in-addr.arpa.rpz.com",
    view   => "default" );

#To get an IPv4 Substitute (PTR Record) Rule object by ipv4addr
my @retrieved_objs = $session->get(
    object   => "Infoblox::DNS::RPZRecord::PTR",
    ipv4addr => "10.0.0.1",
    view     => "default" );

#To get an IPv4 Substitute (PTR Record) Rule object by ptrdname
my @retrieved_objs = $session->get(
    object   => "Infoblox::DNS::RPZRecord::PTR",
    ptrdname => "bind_ptr.domain.com",
    view     => "default" );

#To get an IPv6 Substitute (PTR Record) Rule object by name
my @retrieved_objs = $session->get(
```

```
    object => "Infoblox::DNS::RPZRecord::PTR",
    name   => "1.0.0.0.0.0.0.0.0.0.0.0.0.0.0.0.0.0.0.0.0.0.0.0.0.0.0.0.1.c.b.a.9.8.7.6.5.4.3.2.1.ip6.arpa.rpz.com",
    view   => "default" );

#To get an IPv6 Substitute (PTR Record) Rule object by ipv6addr
my @retrieved_objs = $session->get(
    object   => "Infoblox::DNS::RPZRecord::PTR",
    ipv6addr => "1234:5678:9abc:1000::1",
    view     => "default" );

#To get an IPv6 Substitute (PTR Record) Rule object by ptrdname
my @retrieved_objs = $session->get(
    object   => "Infoblox::DNS::RPZRecord::PTR",
    ptrdname => "ip6host.this.com",
    view     => "default" );

# get all Substitute (PTR Record) Rule objects with a given extensible attribute
my @retrieved_objs = $session->get(
    object => "Infoblox::DNS::RPZRecord::PTR",
    extensible_attributes => { 'Site' => 'Santa Clara' });
```

Infoblox::Session->modify( )

Use this method to modify objects in the Infoblox appliance. See Infoblox::Session->modify() for parameters and return values.

Example

```
# Use method to modify the comment for IPv4 or IPv6 object.
$bind_ptr->comment("this is a modified comment");
# Submit modification
my $response = $session->modify( $bind_ptr );
```

Infoblox::Session->remove( )

Use this function to submit the remove of an object from the Infoblox appliance. See Infoblox::Session->remove() for parameters and return values.

To remove a specific object, first use `get()` or `search()` to retrieve the specific object, and then submit this object for removal.

Example

#Remove an object with a unique name

```
# Get the object with the same name for IPv4 object
my @retrieved_objs = $session->get(
    object => "Infoblox::DNS::RPZRecord::PTR",
    name   => "1.0.0.10.in-addr.arpa.rpz.com",
    view   => "default" );
# find the desired object from retrieved list.
my $desired_ptr = $retrieved_objs[0];
# Submit for removal
my $response = $session->remove( $desired_ptr );

# Get the object with the same name for an IPv6 object
my @retrieved_objs = $session->get(
    object => "Infoblox::DNS::RPZRecord::PTR",
    name   => "1.0.0.0.0.0.0.0.0.0.0.0.0.0.0.0.0.0.0.0.0.0.0.0.0.0.0.0.1.c.b.a.9.8.7.6.5.4.3.2.1.ip6.arpa.rpz.com",
    view   => "default" );
# find the desired object from retrieved list.
my $desired_ptr = $retrieved_objs[0];
# Submit for removal
my $response = $session->remove( $desired_ptr );
```

Infoblox::Session->search( )

Use this method to search for existing Substitute (PTR Record) Rule objects in the Infoblox appliance. See Infoblox::Session->search() for parameters and return values.

Key Reference

```
 Apply the following attributes to search for a Substitute (PTR Record) Rule object:

 name | ipv4addr | ipv6addr | ptrdname - Optional. The IPv4 or IPv6 address of the RPZ Substitute
(PTR Record) Rule, or the domain name in FQDN (Fully Qualified Domain Name) format (regular
expression).
 view                                - Optional. The DNS view in which the RPZ Substitute (PTR
Record) Rule is located. By default, all DNS views are searched. But if you omit this attribute and
specify a zone, the appliance searches the 'default' view only.
 comment                             - Optional. A comment in string format (regular expression).
```

```
extattrs                              - Optional. A hash reference containing extensible attributes.
extensible_attributes                 - Optional. A hash reference containing extensible attributes.
```

For more information about searching extensible attributes, see Infoblox::Grid::ExtensibleAttributeDef/Searching Extensible Attributes.

Example

```
# search for all IPv4 Substitute (PTR Record) Rule objects in the default DNS view that match
10.0.0.0/8 and that contain "bind" in the comment field
my @retrieved_objs = $session->search(
    object  => "Infoblox::DNS::RPZRecord::PTR",
    name    => "10.in-addr.arpa.rpz.com",
    comment => "bind*",
    view    => "default" );

my @retrieved_objs = $session->search(
    object   => "Infoblox::DNS::RPZRecord::PTR",
    ipv4addr => '10\..*\..*\..*',
    comment  => "bind*",
    view     => "default" );

my @retrieved_objs = $session->search(
    object   => "Infoblox::DNS::RPZRecord::PTR",
    ptrdname => "bind_ptr.domain.com",
    comment  => "bind*",
    view     => "default" );

# search for all IPv6 Substitute (PTR Record) Rule objects under 1234:5678:9abc::/48 which have bind
in comment field in default view
my @retrieved_objs = $session->search(
    object  => "Infoblox::DNS::RPZRecord::PTR",
    name    => "1.0.0.0.0.0.0.0.0.0.0.0.0.0.0.0.0.0.0.0.0.0.0.0.0.0.0.0.1.c.b.a.9.8.7.6.5.4.3.2.1.ip6.arpa.rpz.com",
    comment => "bind*",
    view    => "default" );

my @retrieved_objs = $session->search(
    object   => "Infoblox::DNS::RPZRecord::PTR",
    ipv6addr => "1234:5678:9abc:1000::1",
    comment  => "bind*",
    view     => "default" );

my @retrieved_objs = $session->search(
    object   => "Infoblox::DNS::RPZRecord::PTR",
    ptrdname => "ip6host.this.com",
    comment  => "bind*",
    view     => "default" );

# search for all Substitute (PTR Record) Rule objects in the "10.10.10.in-addr.arpa" zone of the
default view
my @retrieved_objs = $session->search(
    object => "Infoblox::DNS::RPZRecord::PTR",
    zone   => "10.10.10.in-addr.arpa",
    view   => "default"
    );

# search all RPZ Substitute (PTR Record) Rules with the extensible attribute 'Site'
my @retrieved_objs = $session->search(
   object => "Infoblox::DNS::RPZRecord::MX",
   extensible_attributes => { 'Site' => 'Santa Clara' });
```

METHODS

This section describes all the methods that you can use to configure and retrieve the attribute value of an RPZ Substitute (PTR Record) Rule.

comment( )

Use this method to set or retrieve the comment.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

Desired comment in string format with a maximum of 256 bytes.

Returns

If you specified a parameter,the method returns true when the modification succeeds,and returns false when the operation fails.

Example

```
#Get comment
my $comment = $rpz_ptr->comment();
#Modify comment
$rpz_ptr->comment("Modifying the Substitute (PTR Record) Rule comment");
```

disable( )

Use this method to set or retrieve the disable flag of a DNS record.

Include the specific parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

The default value for this field is false, therefore the DNS record is enabled.

Parameter

Specify "true" to set the disable flag or "false" to deactivate/unset it.

Returns

If you specified a parameter,the method returns true when the modification succeeds,and returns false when the operation fails.

Example

```
#Get disable
my $disable = $rpz_ptr->disable();
#Modify disable
$rpz_ptr->disable("true");
```

extattrs( )

Use this method to set or retrieve the extensible attributes associated with a RPZ Substitute (PTR Record) Rule object.

Parameter

Valid value is a hash reference containing the names of extensible attributes and their associated values (Infoblox::Grid::Extattr objects).

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get extattrs
my $ref_extattrs = $rpz_ptr->extattrs();
#Modify extattrs
$rpz_ptr->extattrs({ 'Site' => $extattr1, 'Administrator' => $extattr2 });
```

extensible_attributes( )

Use this method to set or retrieve the extensible attributes associated with an RPZ Substitute (PTR Record) Rule.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

For valid values for extensible attributes, see Infoblox::Grid::ExtensibleAttributeDef/Extensible Attribute Values.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get extensible attributes
my $ref_extensible_attributes = $rpz_ptr->extensible_attributes();
#Modify extensible attributes
$rpz_ptr->extensible_attributes({ 'Site' => 'Santa Clara', 'Administrator' => [ 'Peter', 'Tom' ] });
```

ipv4addr( )

Use this method to set or retrieve the IPv4 address.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

An IPv4 address is a 32-bit number in dotted decimal notation. It consists of four 8-bit groups of decimal digits separated by decimal points (example: 192.168.1.2).

Returns

If you specified a parameter,the method returns true when the modification succeeds,and returns false when the operation fails.

Example

```
#Get ipv4addr
my $ipv4addr = $rpz_ptr->ipv4addr();
#Modify ipv4addr
$bind_ptr->ipv4addr("10.0.0.2");
```

ipv6addr( )

Use this method to set or retrieve the IPv6 address.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

The IPv6 (128 bits) of the Substitute (PTR Record) Rule (example: aaaa:bbbb:cccc:dddd:eeee:ffff:0000:1111).

Returns

If you specified a parameter,the method returns true when the modification succeeds,and returns false when the operation fails.

Example

```
#Get ipv6addr
my $ipv6addr = $rpz_ptr->ipv6addr();
#Modify ipv6addr
$bind_ptr->ipv6addr("1234:5678:9abc:1000::2");
```

name( )

Use this method to set or retrieve the FQDN of the RPZ Substitute (PTR Record) Rule.

Omit the parameter to retrieve the attribute value.

Parameter

FQDN of the RPZ Substitute (PTR Record) Rule

Returns

The method returns the attribute value.

Example

```
#Get name
my $name = $rpz_ptr->name();
#Modify name
$rpz_ptr->name('12.0.0.1.in-addr.arpa.rpz.com');
```

ptrdname( )

Use this method to set or retrieve the Substitute (PTR Record) Rule domain name.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

Text with the domain name for the Substitute (PTR Record) Rule.

Returns

If you specified a parameter,the method returns true when the modification succeeds,and returns false when the operation fails.

Example

```
#Get ptrdname
my $ptrdname = $rpz_ptr->ptrdname();
#Modify ptrdname
$rpz_ptr->ptrdname("new domain name");
```

ttl( )

Use this method to configure or retrieve the Time to Live (TTL) value.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

The default value is undefined; therefore the record inherits the TTL value from the zone level.

With a specified TTL value, the method overrides the zone values with the specified TTL value.

Parameter

A 32-bit integer (range from 0 to 4294967295) that represents the duration ,in seconds, that the record is cached. Zero indicates that the record should not be cached.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

Example

```
#Get ttl
my $ttl = $rpz_ptr->ttl();
#Modify ttl
$rpz_ptr->ttl(1800);
#Un-override ttl
$rpz_ptr->ttl(undef);
```

views( )

Use this method to specify or retrieve the view in which the RPZ Substitute (PTR Record) Rule is located.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

The default value is the "default" view, which means the RPZ Substitute (PTR Record) Rule is located in the default DNS view.

Parameter

An array reference of defined Infoblox::DNS::View objects.

Note that the array size must be 1.

Returns

If you specified a parameter,the method returns true when the modification succeeds,and returns false when the operation fails.

Example

```
#Get views
my $ref_views = $rpz_ptr->views();
#Modify views, an array of Infoblox::DNS::View objects
$rpz_ptr->views([$view1]);
```

zone( )

Use this method to retrieve the zone name of a Substitute (PTR Record) Rule. This method is read-only and cannot be set.

Parameter

None

Returns

Returns the attribute value.

Example

```
# Get zone
my $zone = $rpz_ptr->zone();
```

rp_zone( )

Use this method to set or retrieve the zone object of a Substitute (PTR Record) Rule.

Parameter

An Infoblox::DNS::Zone object.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
# Get rp_zone
my $rp_zone = $rpz_ptr->rp_zone();
#Modify rp_zone, reference of Infoblox::DNS::Zone object
$rpz_ptr->rp_zone($response_policy_zone);
```

SAMPLE CODE

The following sample code demonstrates different operations that can be applied to an object, such as create, search, modify, and remove an object. This sample code also includes error handling for the operations.

#Preparation prior to a DNS IPv4 RPZ Substitute (PTR Record) Rule insertion

```
#PROGRAM STARTS: Include all the modules that will be used
use strict;
use Infoblox;

#Create a session to the Infoblox appliance
my $session = Infoblox::Session->new(
    master   => "192.168.1.2",
    username => "admin",
    password => "infoblox"
);
unless ($session) {
   die("Construct session failed: ",
       $session->status_code() . ":" . $session->status_detail());
}
print "Session created successfully\n";

#Create the zone prior to an RPZ Substitute (PTR Record) Rule insertion
my $zone = Infoblox::DNS::Zone->new(name => "10.0.0.0/8");
unless ($zone) {
   die("Construct zone failed: ",
       Infoblox::status_code() . ":" . Infoblox::status_detail());
}
print "Zone object created successfully\n";

#Verify if the zone exists
my $object = $session->get(object => "Infoblox::DNS::Zone", name => "10.in-addr.arpa");
unless ($object) {
   print "Zone does not exist on server, safe to add the zone\n";
   $session->add($zone)
      or die("Add zone failed: ",
             $session->status_code() . ":" . $session->status_detail());
}
print "Zone added successfully\n";
```

#Create a DNS IPv4 RPZ Substitute (PTR Record) Rule

```
my $bind_ptr = Infoblox::DNS::RPZRecord::PTR->new(
   ptrdname     => "bind_ptr.domain.com",
   comment  => " this is a demo RPZ Substitute (PTR Record) Rule ",
   ipv4addr => "10.0.0.1"
 );
 unless ($bind_ptr) {
    die("Construct Substitute (PTR Record) Rule failed: ",
        Infoblox::status_code() . ":" . Infoblox::status_detail());
 }
print "Substitute (PTR Record) Rule object created successfully\n";

#Add the Substitute (PTR Record) Rule object to the Infoblox appliance through a session
$session->add($bind_ptr)
    or die("Add Substitute (PTR Record) Rule failed: ",
           $session->status_code() . ":" . $session->status_detail());
print "Substitute (PTR Record) Rule object added to server successfully\n";
```

#Search for a specific DNS IPv4 RPZ Substitute (PTR Record) Rule

```
#Search for all RPZ Substitute (PTR Record) Rules that match "10.in-addr.arpa"
my @retrieved_objs = $session->search(
    "object" => "Infoblox::DNS::RPZRecord::PTR",
    "name" => '10\.in-addr.arpa',
);
my $object = $retrieved_objs[0];
unless ($object) {
    die("Search Substitute (PTR Record) Rule failed: ",
        $session->status_code() . ":" . $session->status_detail());
}
print "Search Substitute (PTR Record) Rule object found at least 1 matching entry\n";

#Search all RPZ Substitute (PTR Record) Rules that have "bind" in the comment
my @retrieved_objs = $session->search(
    object => "Infoblox::DNS::RPZRecord::PTR",
    name   => '10\.*\.in-addr.arpa',
    comment => "bind*"
);
my $object = $retrieved_objs[0];
unless ($object) {
    die("Search Substitute (PTR Record) Rule failed: ",
        $session->status_code() . ":" . $session->status_detail());
}
print "Search Substitute (PTR Record) Rule object using regexp found at least 1 matching entry\n";
```

#Get and modify a DNS IPv4 RPZ Substitute (PTR Record) Rule

```
#Get an RPZ Substitute (PTR Record) Rule through the session
my @retrieved_objs = $session->get(
    object => "Infoblox::DNS::RPZRecord::PTR",
    name   => "1.0.0.10.in-addr.arpa.rpz.com",
);
my $object = $retrieved_objs[0];
unless ($object) {
    die("Get Substitute (PTR Record) Rule failed: ",
        $session->status_code() . ":" . $session->status_detail());
}
print "Get Substitute (PTR Record) Rule object found at least 1 matching entry\n";
#Modify one of the attributes of the specified RPZ Substitute (PTR Record) Rule
$object->comment ("modified comment");

#Apply the changes
$session->modify($object)
    or die("Modify Substitute (PTR Record) Rule failed: ",
           $session->status_code() . ":" . $session->status_detail());
print "Substitute (PTR Record) Rule object modified successfully \n";
```

#Remove a DNS IPv4 RPZ Substitute (PTR Record) Rule

```
#Get an RPZ Substitute (PTR Record) Rule through the session
my @retrieved_objs = $session->get(
    object => "Infoblox::DNS::RPZRecord::PTR",
    name   => "1.0.0.10.in-addr.arpa.rpz.com",
);

 my $object = $retrieved_objs[0];
  unless ($object) {
    die("Get Substitute (PTR Record) Rule failed: ",
         $session->status_code() . ":" . $session->status_detail());
  }
  print "Get Substitute (PTR Record) Rule object found at least 1 matching entry\n";

#submit the object for removal
$session->remove($object)
    or die("Remove Substitute (PTR Record) Rule failed: ",
           $session->status_code() . ":" . $session->status_detail());
print "Substitute (PTR Record) Rule object removed successfully \n";
```

#Preparation prior to a DNS IPv6 RPZ Substitute (PTR Record) Rule insertion

```
#PROGRAM STARTS: Include all the modules that will be used
use strict;
use Infoblox;

#Create a session to the Infoblox appliance
```

```
my $session = Infoblox::Session->new(
    master   => "192.168.1.2",
    username => "admin",
    password => "infoblox"
);
unless ($session) {
   die("Construct session failed: ",
       $session->status_code() . ":" . $session->status_detail());
}
print "Session created successfully\n";

#Create the zone prior to a Substitute (PTR Record) Rule insertion
my $zone = Infoblox::DNS::Zone->new(name => "rpz.com",
                                    rpz_policy => "GIVEN");
unless ($zone) {
die("Construct zone failed: ",
    Infoblox::status_code() . ":" . Infoblox::status_detail());
}
print "Zone object created successfully\n";

#Verify if the zone exists
my $object = $session->get(object => "Infoblox::DNS::Zone", name => "rpz.com");
unless ($object) {
print "Zone does not exist on server, safe to add the zone\n";
$session->add($zone)
   or die("Add zone failed: ",
          $session->status_code() . ":" . $session->status_detail());
}
```

#Create a DNS IPv6 RPZ Substitute (PTR Record) Rule

```
my $bind_ptr = Infoblox::DNS::RPZRecord::PTR->new(
    ptrdname     => "ip6host.this.com",
    comment  => " this is a demo RPZ Substitute (PTR Record) Rule ",
    ipv6addr => "1234:5678:9abc:1000::1",
    rp_zone  => $zone,
);
unless ($bind_ptr) {
 die("Construct DNS Substitute (PTR Record) Rule failed: ",
     Infoblox::status_code() . ":" . Infoblox::status_detail());
}
print "Substitute (PTR Record) Rule object created successfully\n";
#Add the Substitute (PTR Record) Rule object to the Infoblox  appliance through a session
$session->add($bind_ptr)
    or die("Add Substitute (PTR Record) Rule failed: ",
           $session->status_code() . ":" . $session->status_detail());
print "Substitute (PTR Record) Rule object added to server successfully\n";
```

#Search for a specific DNS IPv6 RPZ Substitute (PTR Record) Rule

```
#Search for an RPZ Substitute (PTR Record) Rule in the zone
my @retrieved_objs = $session->search(
 "object" => "Infoblox::DNS::RPZRecord::PTR",
 "name" => "1.0.0.0.0.0.0.0.0.0.0.0.0.0.0.0.0.0.0.0.0.0.0.0.0.0.0.0.1.c.b.a.9.8.7.6.5.4.3.2.1.ip6.arpa.rpz.com",
);
my $object = $retrieved_objs[0];
unless ($object) {
 die("Search Substitute (PTR Record) Rule failed: ",
     $session->status_code() . ":" . $session->status_detail());
}
print "Search Substitute (PTR Record) Rule object found at least 1 matching entry\n";
```

#Get and modify a DNS IPv6 RPZ Substitute (PTR Record) Rule

```
#Get RPZ Substitute (PTR Record) Rule through the session
my @retrieved_objs = $session->get(
 object => "Infoblox::DNS::RPZRecord::PTR",
 name   => "1.0.0.0.0.0.0.0.0.0.0.0.0.0.0.0.0.0.0.0.0.0.0.0.0.0.0.0.1.c.b.a.9.8.7.6.5.4.3.2.1.ip6.arpa.rpz.com",
);
my $object = $retrieved_objs[0];
unless ($object) {
 die("Get Substitute (PTR Record) Rule failed: ",
     $session->status_code() . ":" . $session->status_detail());
}
print "Get Substitute (PTR Record) Rule object found at least 1 matching entry\n";
```

```
#Modify one of the attributes of the specified RPZ Substitute (PTR Record) Rule
$object->comment ("modified comment");

#Apply the changes
$session->modify($object)
 or die("Modify Substitute (PTR Record) Rule failed: ",
         $session->status_code() . ":" . $session->status_detail());
print "Substitute (PTR Record) Rule object modified successfully \n";
```

#Remove a DNS IPv6 RPZ Substitute (PTR Record) Rule

```
#Get RPZ Substitute (PTR Record) Rule through the session
my @retrieved_objs = $session->get(
 object => "Infoblox::DNS::RPZRecord::PTR",
 name   => "1.0.0.0.0.0.0.0.0.0.0.0.0.0.0.0.0.0.0.0.0.0.0.0.0.0.0.0.1.c.b.a.9.8.7.6.5.4.3.2.1.ip6.arpa.rpz.com",
);
my $object = $retrieved_objs[0];
unless ($object) {
 die("Get Substitute (PTR Record) Rule failed: ",
      $session->status_code() . ":" . $session->status_detail());
}
print "Get Substitute (PTR Record) Rule object found at least 1 matching entry\n";

#Submit the object for removal
$session->remove($object)
 or die("Remove Substitute (PTR Record) Rule failed: ",
         $session->status_code() . ":" . $session->status_detail());
print "Substitute (PTR Record) Rule object removed successfully \n";

####PROGRAM ENDS####
```

AUTHOR

Infoblox Inc. http://www.infoblox.com/

SEE ALSO

Infoblox::DNS::View, Infoblox::DNS::Zone, Infoblox::Session, Infoblox::Session->get(), Infoblox::Session->search(), Infoblox::Session->add(), Infoblox::Session->remove(), Infoblox::Session->modify()

COPYRIGHT

Copyright (c) 2017 Infoblox Inc.

Infoblox::DNS::RPZRecord::SRV - Response Policy Zone Rule object

- NAME
- DESCRIPTION
- CONSTRUCTOR
- SESSION METHODS
  - Infoblox::Session->add( )
  - Infoblox::Session->get( )
  - Infoblox::Session->modify( )
  - Infoblox::Session->remove( )
  - Infoblox::Session->search( )
- METHODS
  - comment( )
  - disable( )
  - extattrs( )
  - extensible_attributes( )
  - name( )
  - port( )
  - priority( )
  - target( )
  - ttl( )
  - views( )
  - weight( )
  - zone( )
  - rp_zone( )
- SAMPLE CODE
- AUTHOR
- SEE ALSO
- COPYRIGHT

NAME

Infoblox::DNS::RPZRecord::SRV - Response Policy Zone (RPZ) Substitute (SRV Record) Rule object

---

DESCRIPTION

An RPZ Substitute (SRV Record) Rule object represents the substitution rule for DNS SRV records.

Note that you must specify only one view for the attribute "views".

CONSTRUCTOR

```
my $srv = Infoblox::DNS::RPZRecord::SRV->new(
    name      => $fqdn,                  #Required
    port      => $num,                   #Required
    priority  => $num,                   #Required
    target    => $fqdn,                  #Required
    weight    => $num,                   #Required
    rp_zone   => $rp_zone,               #Required
    comment   => $string,                #Optional / Default is empty
    disable   => "true" | "false"        #Optional / Default "false"
    extattrs             => { $string => $extattr, ... },      #Optional / Default is undefined
    extensible_attributes => { $string => $string|$num, $string => [ $string|$num, ...], ... }, #Optional
/ Default is undefined
    ttl       => $num | undef,           #Optional / Default is undefined
    views     => [$View1],               #Optional / Default is "default" view
);
```

You cannot set both extattrs and extensible_attributes attributes at the same time.

SESSION METHODS

This section describes all the methods in Infoblox::Session that you can apply to a Substitute (SRV Record) Rule object.

Infoblox::Session->add( )

Use this method to add an object to the Infoblox appliance. See Infoblox::Session->add() for parameters and return values.

Example

```
#Construct object
my $bind_srv = Infoblox::DNS::RPZRecord::SRV->new(
    name     => "bind_srv.rpz.com",
    comment  => "add bind srv bind_srv.rpz.com",
    priority => 1,
    port     => 1,
    weight   => 1,
    target   => "old.target.test.org",
    rp_zone  => $response_policy_zone,
);
# Submit for addition
my $response = $session->add( $bind_srv );
```

Infoblox::Session->get( )

Use this method to retrieve the existing objects from the Infoblox appliance. See Infoblox::Session->get() for parameters and return values.

Key Reference

```
Apply the following attributes to get a specified Substitute (SRV Record) Rule object:

  name     - Optional. A name in string format.
  view     - Optional. The DNS view in which the RPZ Substitute (SRV Record) Rule is located. By
default, all DNS views are searched. But if you omit this attribute and specify a zone, the appliance
searches the 'default' view only.
  zone     - Optional. A zone name in FQDN format.
  weight   - Optional. The weight of the RPZ Substitute (SRV Record) Rule.
  port     - Optional. The port of the RPZ Substitute (SRV Record) Rule.
  priority - Optional. The priority of the RPZ Substitute (SRV Record) Rule.
  target   - Optional. The target of the RPZ Substitute (SRV Record) Rule.
  extattrs - Optional. A hash reference containing extensible attributes.
  extensible_attributes - Optional. A hash reference containing extensible attributes.
```

Example

```
my @retrieved_objs = $session->get(
    object => "Infoblox::DNS::RPZRecord::SRV",
    name     => "bind_srv.rpz.com",
    view   => "default" );

# get all Substitute (SRV Record) Rule in zone "rpz.com" of default view
my @retrieved_objs = $session->get(
    object => "Infoblox::DNS::RPZRecord::SRV",
    zone     => "rpz.com",
    view   => "default" );

# get all Substitute (SRV Record) Rules with a given extensible attribute
my @retrieved_objs = $session->get(
    object => "Infoblox::DNS::RPZRecord::SRV",
    extensible_attributes => { 'Site' => 'Santa Clara' });
```

Infoblox::Session->modify( )

Use this method to modify objects in the Infoblox appliance. See Infoblox::Session->modify() for parameters and return values.

Example

```
# Use method to modify the comment.
$bind_srv->comment("this is a modified comment");
# Submit modification
my $response = $session->modify( $bind_srv);
```

Infoblox::Session->remove( )

Use this method to remove an object from the Infoblox appliance. See Infoblox::Session->remove() for parameters and return values.

To remove a specific object, first use `get()` or `search()` to retrieve the specific object, and then submit this object for removal.

Example

```
# Get the object under the same name
my @retrieved_objs = $session->get(
    object => "Infoblox::DNS::RPZRecord::SRV",
    name     => "bind_srv.rpz.com",
    view   => "default" );
my $desired_srv= $retrieved_objs[0];
# Submit for removal
my $response = $session->remove( $desired_srv );
```

Infoblox::Session->search( )

Use this method to search for existing Substitute (SRV Record) Rule objects in the Infoblox appliance. See Infoblox::Session->search() for parameters and return values.

Key Reference

```
 Apply the following attributes to search for a Substitute (SRV Record) Rule object:

   name     - Required. A name in FQDN (Fully Qualified Domain Name) format (regular expression).
   view     - Optional. The DNS view in which the RPZ Substitute (SRV Record) Rule is located. By
default, all DNS views are searched. But if you omit this attribute and specify a zone, the appliance
searches the 'default' view only.
   zone     - Optional. A zone name in FQDN format.
   comment - Optional . A comment in string format (regular expression).
   weight   - Optional. The weight of the RPZ Substitute (SRV Record) Rule (regular expression).
   port     - Optional. The port of the RPZ Substitute (SRV Record) Rule (regular expression).
   priority - Optional. The priority of the RPZ Substitute (SRV Record) Rule (regular expression).
   target   - Optional. The target of the RPZ Substitute (SRV Record) Rule (regular expression).
   extattrs      - Optional. A hash reference containing extensible attributes.
   extensible_attributes - Optional. A hash reference containing extensible attributes.
```

For more information about searching extensible attributes, see Infoblox::Grid::ExtensibleAttributeDef/Searching Extensible Attributes.

Example

```
 # search for all Substitute (SRV Record) Rule objects in the default DNS view that match "rpz.com"
and that contain "bind" in the comment field
 my @retrieved_objs = $session->search(
     object => "Infoblox::DNS::RPZRecord::SRV",
     name     => 'rpz\.com',
     comment  => "bind",
     view     => "default" );

 # search for all Substitute (SRV Record) Rules in the "rpz.com" zone of the default view
 my @retrieved_objs = $session->search(
     object => "Infoblox::DNS::RPZRecord::SRV",
     zone     => "rpz.com",
     view   => "default" );

 # search all Substitute (SRV Record) Rules with the extensible attribute 'Site'
 my @retrieved_objs = $session->search(
    object => "Infoblox::DNS::RPZRecord::SRV",
    extensible_attributes => { 'Site' => 'Santa Clara' });
```

METHODS

This section describes all the methods that you can use to configure and retrieve the attribute values of an RPZ Substitute (SRV Record) Rule.

comment( )

Use this method to add or retrieve a descriptive comment.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

Desired comment in string format with a maximum of 256 bytes.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

Example

```
 #Get comment
 my $comment = $rpz_srv->comment();
 #Modify comment
 $rpz_srv->comment("Modifying the Substitute (SRV Record) Rule comment");
```

disable( )

Use this method to set or retrieve the disable flag.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

The default value for this field is false, therefore the DNS record is enabled.

Parameter

Specify "true" to set the disable flag or "false" to deactivate/unset it.

Returns

If you specified a parameter,the method returns true when the modification succeeds,and returns false when the operation fails.

Example

```
#Get disable
my $disable = $rpz_srv->disable();
#Modify disable
$rpz_srv->disable("true");
```

extattrs( )

Use this method to set or retrieve the extensible attributes associated with a Substitute (SRV Record) Rule object.

Parameter

Valid value is a hash reference containing the names of extensible attributes and their associated values (Infoblox::Grid::Extattr objects).

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get extattrs
my $ref_extattrs = $rpz_srv->extattrs();
#Modify extattrs
$rpz_srv->extattrs({ 'Site' => $extattr1, 'Administrator' => $extattr2 });
```

extensible_attributes( )

Use this method to set or retrieve the extensible attributes associated with a Substitute (SRV Record) Rule.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

For valid values for extensible attributes, see Infoblox::Grid::ExtensibleAttributeDef/Extensible Attribute Values.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get extensible attributes
my $ref_extensible_attributes = $rpz_srv->extensible_attributes();
#Modify extensible attributes
$rpz_srv->extensible_attributes({ 'Site' => 'Santa Clara', 'Administrator' => [ 'Peter', 'Tom' ] });
```

name( )

Use this method to set or retrieve the name.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

The name of the Substitute (SRV Record) Rule.

Returns

If you specified a parameter,the method returns true when the modification succeeds,and returns false when the operation fails.

Example

```
#Get name
my $name = $rpz_srv->name();
```

```
#Modify name
$rpz_srv->name("modified_bind_srv.rpz.com");
```

port( )

Use this method to add or retrieve a port.

Parameter

The port number for the Substitute (SRV Record) Rule (range from 0 to 65535).

Returns

If you specified a parameter,the method returns true when the modification succeeds,and returns false when the operation fails.

Example

```
#Get port
my $port = $rpz_srv->port();
#Modify port
$bindport->port("2");
```

priority( )

Use this method to add or retrieve the priority.

Parameter

Number with the priority for the Substitute (SRV Record) Rule (range from 0 to 65535).

Returns

If you specified a parameter,the method returns true when the modification succeeds,and returns false when the operation fails.

Example

```
#Get priority
my $priority = $rpz_srv->priority();
#Modify priority
$rpz_srv->priority("2");
```

target( )

Use this method to set or retrieve the target. The target must be in Fully Qualified Domain Name (FQDN) format.

Parameter

Text with the target for the Substitute (SRV Record) Rule.

Returns

If you specified a parameter,the method returns true when the modification succeeds,and returns false when the operation fails.

Example

```
#Get target
my $target = $rpz_srv->target();
#Modify target
$rpz_srv->target ("new.target.test.org");
```

ttl( )

Use this method to configure or retrieve the Time to Live (TTL) value.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

The default value is undefined; therefore the record inherits the TTL value from the zone level.

With a specified TTL value, the method overrides the zone values with the specified TTL value.

Parameter

A 32-bit integer (range from 0 to 4294967295) that represents the duration in seconds that the record is cached. Zero indicates that the record should not be cached.

Returns

If you specified a parameter,the method returns true when the modification succeeds,and returns false when the operation fails.

Example

```
#Get ttl
my $ttl = $rpz_srv->ttl();
#Modify ttl
$rpz_srv->ttl(1800);
#Un-override ttl
$rpz_srv->ttl(undef);
```

views( )

Use this method to specify or retrieve the views in which the RPZ Substitute (SRV Record) Rule is located.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

The default value is the default view, therefore the RPZ Substitute (SRV Record) Rule is located under the default view.

Parameter

An array reference of defined Infoblox::DNS::View objects.

Note that the array size must be 1.

Returns

If you specified a parameter,the method returns true when the modification succeeds,and returns false when the operation fails.

Example

```
#Get views
my $ref_views = $rpz_srv->views();
#Modify views, an array of Infoblox::DNS::View objects
$rpz_srv->views([$view1]);
```

weight( )

Use this method to set or retrieve the weight.

Parameter

The weight of the Substitute (SRV Record) Rule.

Returns

If you specified a parameter,the method returns true when the modification succeeds,and returns false when the operation fails.

Example

```
#Get weight
my $weight = $rpz_srv->weight ();
#Modify weight
$rpz_srv->weight ("3");
```

zone( )

Use this method to retrieve the zone name of the Substitute (SRV Record) Rule. This method is read-only and cannot be set.

Parameter

None

Returns

Returns the attribute value.

Example

```
# Get zone
my $zone = $rpz_srv->zone();
```

rp_zone( )

Use this method to set or retrieve the zone object of a Substitute (SRV Record) Rule.

Parameter

An Infoblox::DNS::Zone object.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
# Get rp_zone
my $rp_zone = $rpz_srv->rp_zone();
#Modify rp_zone, reference of Infoblox::DNS::Zone object
$rpz_srv->rp_zone($response_policy_zone);
```

SAMPLE CODE

The following sample code demonstrates different operations that can be applied to an object, such as create, search, modify, and remove an object. This sample code also includes error handling for the operations.

#Preparation prior to a Substitute (SRV Record) Rule insertion

```
#PROGRAM STARTS: Include all the modules that will be used
use strict;
use Infoblox;

#Create a session to the Infoblox appliance
my $session = Infoblox::Session->new(
    master   => "192.168.1.2",
    username => "admin",
    password => "infoblox"
);
unless ($session) {
   die("Construct session failed: ",
       $session->status_code() . ":" . $session->status_detail());
}
print "Session created successfully\n";

#Create the zone prior to an RPZ Substitute (SRV Record) Rule insertion
my $zone = Infoblox::DNS::Zone->new(name => "rpz.com",
                                    rpz_policy => "GIVEN");
unless ($zone) {
   die("Construct zone failed: ",
       Infoblox::status_code() . ":" . Infoblox::status_detail());
}
print "Zone object created successfully\n";

#Verify if the zone exists
my $object = $session->get(object => "Infoblox::DNS::Zone", name => "rpz.com");
unless ($object) {
   print "Zone does not exist on server, safe to add the zone\n";
   $session->add($zone)
      or die("Add zone failed: ",
             $session->status_code() . ":" . $session->status_detail());
}
print "Zone added successfully\n";
```

#Create a Substitute (SRV Record) Rule

```
my $bind_srv = Infoblox::DNS::RPZRecord::SRV->new(
name     => "bind_srv.rpz.com",
comment  => "add bind bind_srv.rpz.com",
priority => 1,
port     => 1,
weight   => 1,
target   => "old.target.test.org",
rp_zone  => $zone,
);
unless ($bind_srv) {
 die("Construct DNS record SRV failed: ",
     Infoblox::status_code() . ":" . Infoblox::status_detail());
}
print "Substitute (SRV Record) Rule object created successfully\n";

#Add the Substitute (SRV Record) Rule object to Infoblox  appliance through a session
$session->add($bind_srv)
 or die("Add record SRV failed: ",
        $session->status_code() . ":" . $session->status_detail());
print "Substitute (SRV Record) Rule object added to server successfully\n";
```

#Search for a specific Substitute (SRV Record) Rule

```
#Search for RPZ Substitute (SRV Record) Rules that match bind_srv.rpz.com
my @retrieved_objs = $session->search(
 "object" => "Infoblox::DNS::RPZRecord::SRV",
 "name" => 'bind_srv\.rpz\.com',
);
my $object = $retrieved_objs[0];
unless ($object) {
 die("Search record SRV failed: ",
     $session->status_code() . ":" . $session->status_detail());
}
print "Search Substitute (SRV Record) Rule object found at least 1 matching entry\n";

#Search all RPZ Substitute (SRV Record) Rules that match "rpz.com" and that contain "bind" in the comment
field
my @retrieved_objs = $session->search(
 object => "Infoblox::DNS::RPZRecord::SRV",
 name   => 'rpz\.com',
 comment => "bind"
);
my $object = $retrieved_objs[0];
unless ($object) {
 die("Search record SRV failed: ",
    $session->status_code() . ":" . $session->status_detail());
}
print "Search Substitute (SRV Record) Rule object using regexp found at least 1 matching entry\n";
```

#Get and modify a Substitute (SRV Record) Rule

```
#Get an RPZ Substitute (SRV Record) Rule through the session
my @retrieved_objs = $session->get(
 object => "Infoblox::DNS::RPZRecord::SRV",
 name   => "bind_srv.rpz.com",
);
my $object = $retrieved_objs[0];
unless ($object) {
 die("Get record SRV failed: ",
     $session->status_code() . ":" . $session->status_detail());
}
print "Get Substitute (SRV Record) Rule object found at least 1 matching entry\n";

#Modify one of the attributes of the specified RPZ Substitute (SRV Record) Rule
$object->comment ("modified comment for SRV rec");

#Apply the changes
$session->modify($object)
 or die("Modify record SRV failed: ",
        $session->status_code() . ":" . $session->status_detail());
print "Substitute (SRV Record) Rule object modified successfully \n";
```

#Remove a Substitute (SRV Record) Rule

```
#Get a RPZ Substitute (SRV Record) Rule through the session
my @retrieved_objs = $session->get(
 object => "Infoblox::DNS::RPZRecord::SRV",
 name   => "bind_srv.rpz.com",
);
my $object = $retrieved_objs[0];
unless ($object) {
 die("Get record SRV failed: ",
     $session->status_code() . ":" . $session->status_detail());
}
print "Get Substitute (SRV Record) Rule object found at least 1 matching entry\n";

#submit the object for removal
$session->remove($object)
 or die("Remove record SRV failed: ",
        $session->status_code() . ":" . $session->status_detail());
print "Substitute (SRV Record) Rule object removed successfully \n";

####PROGRAM ENDS####
```

AUTHOR

Infoblox Inc. http://www.infoblox.com/

SEE ALSO

Infoblox::DNS::View, Infoblox::DNS::Zone, Infoblox::Session, Infoblox::Session->get(), Infoblox::Session->search(), Infoblox::Session->add(), Infoblox::Session->remove(), Infoblox::Session->modify()

COPYRIGHT

Copyright (c) 2017 Infoblox Inc.

Infoblox::DNS::RPZRecord::TXT - Response Policy Zone Rule object.

- NAME
- DESCRIPTION
- CONSTRUCTOR
- SESSION METHODS
  - Infoblox::Session->add( )
  - Infoblox::Session->get( )
  - Infoblox::Session->modify( )
  - Infoblox::Session->remove( )
  - Infoblox::Session->search( )
- METHODS
  - comment( )
  - disable( )
  - extattrs( )
  - extensible_attributes( )
  - name( )
  - text( )
  - ttl( )
  - views( )
  - zone( )
  - rp_zone( )
- SAMPLE CODE
- AUTHOR
- SEE ALSO
- COPYRIGHT

NAME

Infoblox::DNS::RPZRecord::TXT - Response Policy Zone (RPZ) Substitute (TXT Record) Rule object.

DESCRIPTION

An RPZ Substitute (TXT Record) Rule object represents the substitution rule for DNS TXT records.

Note that you must specify only one view for the attribute "views".

CONSTRUCTOR

```
 my $txt = Infoblox::DNS::RPZRecord::TXT->new(
     name      => $fqdn,                  #Required
     text      => $string                 #Required
     rp_zone   => $rp_zone,               #Required
     comment   => $string,                #Optional / Default is empty
     disable   => "true" | "false"        #Optional / Default "false"
     extattrs               => { $string => $extattr, ... },      #Optional / Default is undefined
     extensible_attributes => { $string => $string|$num, $string => [ $string|$num, ...], ... }, #Optional
/ Default is undefined
     ttl       => $num | undef,           #Optional / Default is undefined
     views     => [$View1],               #Optional / Default is "default" view
 );
```

You cannot set both extattrs and extensible_attributes attributes at the same time.

SESSION METHODS

This section describes all the methods in Infoblox::Session module that you can apply to an RPZ Substitute (TXT Record) Rule object.

Infoblox::Session->add( )

Use this method to add an object to the Infoblox appliance. See Infoblox::Session->add() for parameters and return values.

Example

```
       #Construct the RPZ TXT object
       my $rpz_txt = Infoblox::DNS::RPZRecord::TXT->new(
           name    => "txt.rpz.com",
           text    => "this is text string",
           comment => "add TXT txt.rpz.com",
           rp_zone  => $response_policy_zone,
```

```
        disable => "false",
        ttl     => "360",
        views   => [$view1]  );
    # Submit for addition
    my $response = $session->add( $rpz_txt );
```

Infoblox::Session->get( )

Use this method to retrieve all the matching objects from the Infoblox appliance. See Infoblox::Session->get() for parameters and return values.

Key References

```
    Apply the following attributes to get a specific Substitute (TXT Record) Rule object:

     name - Optional. Host name in string format.
     view - Optional. The DNS view in which the RPZ Substitute (TXT Record) Rule is located. By default,
    all DNS views are searched. But if you omit this attribute and specify a zone, the appliance searches
    the 'default' view only.
     text - Optional. The text of the RPZ Substitute (TXT Record) Rule.
     extattrs      - Optional. A hash reference containing extensible attributes.
     extensible_attributes - Optional. A hash reference containing extensible attributes.
```

Example

```
    my @retrieved_objs = $session->get(
        object => "Infoblox::DNS::RPZRecord::TXT",
        name   => "txt.rpz.com",
        view   => "default" );

    # get all Substitute (TXT Record) Rules in the "rpz.com" zone of the default view
    my @retrieved_objs = $session->get(
        object => "Infoblox::DNS::RPZRecord::TXT",
        zone   => "rpz.com",
        view   => "default" );

    # get all Substitute (TXT Record) Rules with a given extensible attribute
    my @retrieved_objs = $session->get(
        object => "Infoblox::DNS::RPZRecord::TXT",
        extensible_attributes => { 'Site' => 'Santa Clara' });
```

Infoblox::Session->modify( )

Use this method to modify an object in the Infoblox appliance. See Infoblox::Session->modify() for parameters and return values.

Example

```
    # Use method to modify the comment.
    $rpz_txt->comment("This is a modified comment");
    # Submit modification
    my $response = $session->modify( $rpz_txt );
```

Infoblox::Session->remove( )

Use this method to remove an object from the Infoblox appliance. See Infoblox::Session->remove() for parameters and return values.

To remove a specifc object, first use `get()` or `search()` to retrieve the specific object, and then submit this object for removal.

Example

```
    # Get the objects with the same name
    my @retrieved_objs = $session->get(
        object => "Infoblox::DNS::RPZRecord::TXT",
        name   => "txt.rpz.com",
        view   => "default" );
    # find the desired object from the retrieved list.
    my $desired_txt = $retrieved_objs[0];
    # Submit for removal
    my $response = $session->remove( $ desired_txt );
```

Infoblox::Session->search( )

Use this method to search for Substitute (TXT Record) Rule objects in the Infoblox appliance. See Infoblox::Session->search() for parameters and return values.

Key References

```
    Apply the following attributes to search for a Substitute (TXT Record) Rule object:
```

```
  name    - Optional. A name in FQDN (Fully Qualified Domain Name) format (regular expression).
  view    - Optional. The DNS view in which the RPZ Substitute (TXT Record) Rule is located. By
default, all DNS views are searched. But if you omit this attribute and specify a zone, the appliance
searches the 'default' view only.
  zone    - Optional. A zone name in FQDN format.
  comment - Optional. A comment in string format (regular expression).
  text    - Optional. The text of the RPZ Substitute (TXT Record) Rule (regular expression).
  extattrs     - Optional. A hash reference containing extensible attributes.
  extensible_attributes - Optional. A hash reference containing extensible attributes.
```

For more information about searching extensible attributes, see Infoblox::Grid::ExtensibleAttributeDef/Searching Extensible Attributes.

Example

```
# search for all Substitute (TXT Record) Rule objects that match "rpz.com" in the default DNS view
my @retrieved_objs = $session->search(
    object => "Infoblox::DNS::RPZRecord::TXT",
    name   => 'rpz\.com',
    view   => "default" );

# search for all Substitute (TXT Record) Rules in the "rpz.com" zone of the default view
my @retrieved_objs = $session->search(
    object => "Infoblox::DNS::RPZRecord::TXT",
    zone   => "rpz.com",
    view   => "default" );

# search all Substitute (TXT Record) Rules with a given extensible attribute 'Site'
my @retrieved_objs = $session->search(
   object => "Infoblox::DNS::RPZRecord::TXT",
   extensible_attributes => { 'Site' => 'Santa Clara' });
```

METHODS

This section describes all the methods that you can use to configure and retrieve the attribute values of an RPZ Substitute (TXT Record) Rule.

comment( )

Use this method to set or retrieve the descriptive comment.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

Desired comment in string format with a maximum of 256 bytes.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get comment
my $comment = $rpz_txt->comment();
#Modify comment
$rpz_txt->comment("Modifying the Substitute (TXT Record) Rule comment");
```

disable( )

Use this method to set or retrieve the disable flag of a DNS record.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

The default value for this field is false. The DNS record is enabled.

Parameter

Specify "true" to set the disable flag or "false" to deactivate/unset it.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get disable
my $disable = $rpz_txt->disable();
#Modify disable
$rpz_txt->disable("true");
```

extattrs( )

Use this method to set or retrieve the extensible attributes associated with a Substitute (TXT Record) Rule object.

Parameter

Valid value is a hash reference containing the names of extensible attributes and their associated values (Infoblox::Grid::Extattr objects).

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get extattrs
my $ref_extattrs = $rpz_txt->extattrs();
#Modify extattrs
$rpz_txt->extattrs({ 'Site' => $extattr1, 'Administrator' => $extattr2 });
```

extensible_attributes( )

Use this method to set or retrieve the extensible attributes associated with a Substitute (TXT Record) Rule.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

For valid values for extensible attributes, see Infoblox::Grid::ExtensibleAttributeDef/Extensible Attribute Values.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get extensible attributes
my $ref_extensible_attributes = $bindtxt->extensible_attributes();
#Modify extensible attributes
$bindtxt->extensible_attributes({ 'Site' => 'Santa Clara', 'Administrator' => [ 'Peter', 'Tom' ] });
```

name( )

Use this method to set or retrieve the name.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

The name of the Substitute (TXT Record) Rule object in Fully Qualified Domain Name (FQDN) format. The name can have a maximum of 256 bytes.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get name
my $name = $rpz_txt->name();
#Modify name
$rpz_txt->name("new.rpz.com");
```

text( )

Use this method to set or retrieve the text.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

Text that you want to associate with the record. It can contain up to 255 bytes.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get name
my $name = $rpz_txt->text();
#Modify name
$rpz_txt->text("this is new text string");
```

ttl( )

Use this method to set or retrieve the Time to Live (TTL) value.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

The default value is undefined which indicates that the record inherits the TTL value of the zone.

Specify a TTL value to override the TTL value at the zone level.

Parameter

A 32-bit integer (range from 0 to 4294967295) that represents the duration in seconds that the record is cached. Zero indicates that the record should not be cached.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get ttl
my $ttl = $rpz_txt->ttl();
#Modify ttl
$rpz_txt->ttl(1800);
#Un-override ttl
$rpz_txt->ttl(undef);
```

views( )

Use this method to set or retrieve the view of an RPZ Substitute (TXT Record) Rule.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

The default value is the "default" view, which means the RPZ Substitute (TXT Record) Rule is in the default view.

Parameter

An array reference of defined Infoblox::DNS::View objects.

Note that the array size must be 1.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get views
my $ref_views = $rpz_txt->views();
#Modify views, an array of Infoblox::DNS::View objects
$rpz_txt->views([$view1]);
```

zone( )

Use this method to retrieve the zone name of a Substitute (TXT Record) Rule. This method is read-only and cannot be set.

Parameter

None

Returns

Returns the attribute value.

Example

```
# Get zone
my $zone = $rpz_txt->zone();
```

rp_zone( )

Use this method to set or retrieve the zone object of a DNS RPZ SRV record.

Parameter

An Infoblox::DNS::Zone object.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
# Get rp_zone
my $rp_zone = $rpz_txt->rp_zone();
#Modify rp_zone, reference of Infoblox::DNS::Zone object
$rpz_txt->rp_zone($response_policy_zone);
```

SAMPLE CODE

The following sample code demonstrates the different functions that can be applied to an object, such as add, search, modify, and remove. This sample also includes error handling for the operations.

#Preparation prior to a Substitute (TXT Record) Rule insertion

```
#PROGRAM STARTS: Include all the modules that will be used
use strict;
use Infoblox;

#Create a session to the Infoblox appliance

my $session = Infoblox::Session->new(
               master   => "192.168.1.2", #appliance host ip
               username => "admin",       #appliance user login
               password => "infoblox"     #appliance password
               );

unless ($session) {
       die("Construct session failed: ",
               Infoblox::status_code() . ":" . Infoblox::status_detail());
}
print "Session created successfully\n";

#Create the zone prior to an RPZ Substitute (TXT Record) Rule insertion
my $zone = Infoblox::DNS::Zone->new(name => "rpz.com",
                                    rpz_policy => "GIVEN");
unless ($zone) {
       die("Construct zone failed: ",
               Infoblox::status_code() . ":" . Infoblox::status_detail());
}
print "Zone object created successfully\n";

#Verify if the zone exists
my $object = $session->get(object => "Infoblox::DNS::Zone", name => "rpz.com");
unless ($object) {
       print "Zone does not exist on server, safe to add the zone\n";
       $session->add($zone)
               or die("Add zone failed: ",
                               $session->status_code() . ":" . $session->status_detail());
}
print "Zone added successfully\n";
```

#Create a Substitute (TXT Record) Rule

```
my $rpz_txt = Infoblox::DNS::RPZRecord::TXT->new(
            name    => "txt.rpz.com",
            comment => "add TXT txt.rpz.com",
            text    => "this is text string",
            rp_zone => $zone,
            );

unless ($rpz_txt) {
    die("Construct DNS record TXT failed: ",
            Infoblox::status_code() . ":" . Infoblox::status_detail());
}

#Add the Substitute (TXT Record) Rule object to the Infoblox appliance through a session
$session->add($rpz_txt)
    or die("Add record TXT failed: ",
                    $session->status_code() . ":" . $session->status_detail());
print "Substitute (TXT Record) Rule object added to server successfully\n";
```

#Search for a specific Substitute (TXT Record) Rule

```
#Search all RPZ Substitute (TXT Record) Rules in the zone that matches "rpz.com"
my @retrieved_objs = $session->search(
            object => "Infoblox::DNS::RPZRecord::TXT",
            name   => 'rpz\.com'
            );
my $object = $retrieved_objs[0];

unless ($object) {
    die("Search record TXT failed: ",
            $session->status_code() . ":" . $session->status_detail());
}
print "Search Substitute (TXT Record) Rule object found at least 1 matching entry\n";

#Search all RPZ Substitute (TXT Record) Rules with names that start with "txt" and end with "rpz.com"
my @retrieved_objs = $session->search(
            object => "Infoblox::DNS::RPZRecord::TXT",
            name   => '^txt.*\.rpz\.com$'
            );
my $object = $retrieved_objs[0];

unless ($object) {
    die("Search record TXT failed: ",
            $session->status_code() . ":" . $session->status_detail());
}
print "Search Substitute (TXT Record) Rule object using regexp found at least 1 matching entry\n";
```

#Get and modify a Substitute (TXT Record) Rule

```
#Get an RPZ Substitute (TXT Record) Rule through the session
my @retrieved_objs = $session->get(
            object => "Infoblox::DNS::RPZRecord::TXT",
            name   => "txt.rpz.com"
            );
my $object = $retrieved_objs[0];

unless ($object) {
    die("Get record TXT failed: ",
            $session->status_code() . ":" . $session->status_detail());
}
print "Get Substitute (TXT Record) Rule object found at least 1 matching entry\n";

#Modify one of the attributes of the specified RPZ Substitute (TXT Record) Rule
$object->text("this is new text string");

#Apply the changes
$session->modify($object)
    or die("Modify record TXT failed: ",
            $session->status_code() . ":" . $session->status_detail());
print "Substitute (TXT Record) Rule object modified successfully \n";
```

#Remove a Substitute (TXT Record) Rule

```
#Get RPZ Substitute (TXT Record) Rule through the session
my @retrieved_objs = $session->get(
```

```
 object => "Infoblox::DNS::RPZRecord::TXT",
 name   => "txt.rpz.com"
);
my $object = $retrieved_objs[0];
unless ($object) {
 die("Get record TXT failed: ",
     $session->status_code() . ":" . $session->status_detail());
}
print "Get Substitute (TXT Record) Rule object found at least 1 matching entry\n";

#Submit the object for removal
$session->remove($object)
    or die("Remove record TXT failed: ",
            $session->status_code() . ":" . $session->status_detail());
print "Substitute (TXT Record) Rule object removed successfully \n";

####PROGRAM ENDS####
```

AUTHOR

Infoblox Inc. http://www.infoblox.com/

SEE ALSO

Infoblox::Session->add(), Infoblox::Session->get(), Infoblox::Session->modify(), Infoblox::Session->remove(), Infoblox::Session->search(), Infoblox::Session, Infoblox::DNS::View, Infoblox::DNS::Zone

---

COPYRIGHT

Copyright (c) 2017 Infoblox Inc.

Infoblox::DNS::Ruleset - Ruleset object

- NAME
- DESCRIPTION
- CONSTRUCTOR
- SESSION METHODS
  - Infoblox::Session->add( )
  - Infoblox::Session->get( )
  - Infoblox::Session->modify( )
  - Infoblox::Session->remove( )
  - Infoblox::Session->search( )
- METHODS
  - name( )
  - type( )
  - comment( )
  - disabled( )
  - nxdomain_rules( )
- SAMPLE CODE
- AUTHOR
- SEE ALSO
- COPYRIGHT

NAME

Infoblox::DNS::Ruleset - Ruleset object

DESCRIPTION

Represents a Ruleset object, which is a collection of rules that is used to match domain names.

CONSTRUCTOR

See Infoblox::DNS::Ruleset::Rule for details.

```
 my $rule1 = Infoblox::DNS::Ruleset::NxdomainRule->new(
     pattern => $string,                                          #Optional
     action => $string,                                           #Optional
 );
 my $rule2 = Infoblox::DNS::Ruleset::NxdomainRule->new(
     pattern => $string,                                          #Optional
     action => $string,                                           #Optional
 );
 my $ruleset = Infoblox::DNS::Ruleset->new (
     name => $string,                                             #Required
     type => <"NXDOMAIN" | "BLACKLIST">                           #Required
     comment => $string,                                          #Optional
     disabled => "true" | "false",                                #Optional
     nxdomain_rules => [$rule1, $rule2]                           #Optional, valid only when
type is "NXDOMAIN"
 );
```

SESSION METHODS

This section describes all the methods in the Infoblox::Session module that you can apply to a Ruleset object.

Infoblox::Session->add( )

Use this method to add an object to the Infoblox appliance. See Infoblox::Session->add() for parameters and return values.

Example

```
#Create Rule 1 object
my $rule1 = Infoblox::DNS::Ruleset::NxdomainRule->new(
    pattern => "www.a.zone.com",
    action => 'PASS' );

#Create Rule 2 object
my $rule2 = Infoblox::DNS::Ruleset::NxdomainRule->new(
    pattern=> "www.b.zone.com",
    action => 'REDIRECT' );
```

```
my $ruleset = Infoblox::DNS::Ruleset->new (
   name => $ruleset_name,
   type => "NXDOMAIN",
   comment=> "This is a demo ruleset",
   disabled=> "false",
   nxdomain_rules=> [$rule1, $rule2]);

 #Submit for addition
 my $response = $session->add( $nsg1 );
```

Infoblox::Session->get( )

Use this method to retrieve all the matching objects from the Infoblox appliance. See Infoblox::Session->get() for parameters and return values.

Key References

```
Apply the following attributes to get a specific Ruleset object:

name - Required. The name of the ruleset in string format.
disabled - Optional. Indicates if the ruleset is disabled. The valid value is either "true" or
"false".
comment - Optional. The comment that was entered for the ruleset.
```

Examples

```
my @retrieved_objs = $session->get(
    object => "Infoblox::DNS::Ruleset",
    name   => "Ruleset 1",
    );

my @retrieved_objs = $session->get(
    object => "Infoblox::DNS::Ruleset",
    disabled   => "false",
    );

my @retrieved_objs = $session->get(
    object => "Infoblox::DNS::Ruleset",
    comment   => "Ruleset comment",
    );
```

Infoblox::Session->modify( )

Use this method to modify an object in the Infoblox appliance. See Infoblox::Session->modify() for parameters and return values.

Example

```
#Modify the comment
$ruleset->comment("this is a modified comment");
#Submit the modification
my $response = $session->modify( $ruleset );
```

Infoblox::Session->remove( )

Use this method to remove an object from the Infoblox appliance. See Infoblox::Session->remove() for parameters and return values.

To remove a specific object, first use `get()` or `search()` to retrieve the object, and then submit it for removal.

Example

#Get the objects with the same name my @retrieved_objs = $session->get(object => "Infoblox::DNS::Ruleset", name => "Ruleset 1",); #Find the desired object on the retrieved list my $ruleset= $retrieved_objs[0]; #Submit the object for removal my $response = $session->remove($ruleset);

Infoblox::Session->search( )

Use this method to search for Ruleset objects in the Infoblox appliance. See Infoblox::Session->search() for parameters and return values.

Key References

Apply the following attributes to search for a Ruleset object:

```
name - Required. The ruleset name in string format (regular expression).
disabled - Optional. Indicates if the ruleset is disabled. The valid value is either "true" or
"false".
comment - Optional. The comment that was entered for the ruleset (regular expression).
```

Examples

```
# Search for all Ruleset objects
my @retrieved_objs = $session->search(
    object => "Infoblox::DNS::Ruleset",
    name   => ".*",
    );
```

Search for all Ruleset objects with "false" as the value for the "disabled" attribute my @retrieved_objs = $session->get(object => "Infoblox::DNS::Ruleset", disabled => "false");

Search for all Ruleset objects with "Ruleset Comm" as the value for the "comment" attribute my @retrieved_objs = $session->get(object => "Infoblox::DNS::Ruleset", comment => "Ruleset Comm");

METHODS

This section describes all the methods that you can use to configure and retrieve the attribute values of a Ruleset object.

name( )

Use this method to set or retrieve the name of this Ruleset object.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

The ruleset name in string format, with a maximum of 256 characters.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get name
my $name = $ruleset->name();
#Modify name
$ruleset->name("rulset");
```

type( )

Use this method to set or retrieve the type of this Ruleset object.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

The ruleset type. Supported values are "NXDOMAIN" or "BLACKLIST".

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get type
my $type = $ruleset->type();
#Modify type
$ruleset->type("NXDOMAIN");
```

comment( )

Use this method to set or retrieve a descriptive comment about the Ruleset object.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

Comment in string format, with a maximum of 256 characters.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get comment
my $comment = $ruleset->comment();
#Modify comment
$ruleset->comment("This is Default Ruleset");
```

disabled( )

Use this method to set or retrieve the flag that indicates if the Ruleset object is disabled.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

Specify "true" to set the disabled flag or "false" to deactivate/unset it. The default value is "false".

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get disabled
my $disabled = $ruleset->disabled();
#Modify disabled
$ruleset->disabled("true");
```

nxdomain_rules( )

Use this method to set or retrieve a list of Infoblox::DNS::Ruleset::NxdomainRule objects assigned to this Ruleset object. Rules can be set only when the ruleset type is set to "NXDOMAIN".

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

The valid value is an array reference that contains a list of Infoblox::DNS::Ruleset::NxdomainRule objects.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Getting rules
my $rules = $ruleset->nxdomain_rules( );
#Modifying rules
$ruleset->nxdomain_rules([$rule1]);
```

SAMPLE CODE

The following sample code demonstrates the different functions that can be applied to an object, such as modify and remove. This sample also includes error handling for the operations.

#Preparation prior to a Ruleset object insertion

```
#PROGRAM STARTS: Include all the modules that will be used
use strict;
use Infoblox;

#Create a session to the Infoblox appliance
my $session = Infoblox::Session->new(
    master   => "192.168.1.2",
    username => "admin",
    password => "infoblox"
);
unless ($session) {
   die("Construct session failed: ",
       Infoblox::status_code() . ":" . Infoblox::status_detail());
}
```

```
print "Session created successfully\n";
```

#Create a Ruleset object

```
my $rule1 = Infoblox::DNS::Ruleset::NxdomainRule->new(
    pattern => "www.a.zone.com",
    action => 'PASS' );
unless ($rule1) {
   die("Construct Rule 1 failed: ",
       Infoblox::status_code() . ":" . Infoblox::status_detail());
}

my $rule2 = Infoblox::DNS::Ruleset::NxdomainRule->new(
    pattern=> "www.b.zone.com",
    action => 'REDIRECT' );
unless ($rule2) {
   die("Construct Rule 2 failed: ",
       Infoblox::status_code() . ":" . Infoblox::status_detail());
}

my $ruleset = Infoblox::DNS::Ruleset->new (
    name => "Ruleset 1",
    type => "NXDOMAIN",
    comment=> "This is demo ruleset",
    disabled=> "false",
    nxdomain_rules=> [$rule1, $rule2]);

unless ($ruleset) {
   die("Construct Ruleset failed: ",
       Infoblox::status_code() . ":" . Infoblox::status_detail());
}
print "Ruleset object created successfully\n";
```

#Add the Ruleset object

```
my $resp = $session->add($ruleset);
unless ($resp == 1) {
   die("Add Ruleset failed: ",
       Infoblox::status_code() . ":" . Infoblox::status_detail());
}
```

#Modify the Ruleset object

```
my @result_array = $session->get(
       object => "Infoblox::DNS::Ruleset",
       name => "Ruleset 1");
if( defined @result_array  and scalar( @result_array ) > 0 ){
       my $ruleset = $result_array[0];
       $ruleset->name("Ruleset 2");

       my $response = $session->modify( $ruleset );
       print "Modify the Ruleset object success.\n" if ( $response );
        }
```

#Remove a Ruleset object

```
my @result_array = $session->get( "object" => "Infoblox::DNS::Ruleset" ,  "name" => "Ruleset 2" );

if( defined @result_array  and scalar( @result_array ) > 0 ){
       my $ruleset = $result_array[0];
       if( ref( $ruleset ) eq "Infoblox::DNS::Ruleset" )
       {

               my $response = $session->remove($ruleset);
               print "Remove Ruleset success.\n" if ( $response );

       }

       unless ($ruleset) {
               die("Remove Ruleset failed: ",
                       $session->status_code() . ":" . $session->status_detail());
       }
        }

####PROGRAM ENDS####
```

AUTHOR

Infoblox Inc. http://www.infoblox.com/

SEE ALSO

Infoblox::Session, Infoblox::Session->get(),Infoblox::Session->remove(),Infoblox::Session->modify(),Infoblox::DNS,Infoblox::DNS::Ruleset::NxdomainRule

---

COPYRIGHT

Copyright (c) 2017 Infoblox Inc.

Infoblox::DNS::Ruleset::NxdomainRule - Rule of Ruleset object

- NAME
- DESCRIPTION
- CONSTRUCTOR
- MODULE METHODS
  - Infoblox::DNS::Ruleset->nxdomain_rules( )
- METHODS
  - pattern( )
  - action( )
- AUTHOR
- SEE ALSO
- COPYRIGHT

NAME

Infoblox::DNS::Ruleset::NxdomainRule - Rule of Ruleset object

DESCRIPTION

Represents a rule that is used to match domain names.

CONSTRUCTOR

```
my $rule = Infoblox::DNS::Ruleset::NxdomainRule->new(
             pattern => $string,                    #Required
             action => $string,                     #Required
);
```

MODULE METHODS

The following functions can be applied to an Infoblox::DNS::Ruleset::NxdomainRule object.

Infoblox::DNS::Ruleset->nxdomain_rules( )

Use this method to add a list of Infoblox::DNS::Ruleset::NxdomainRule objects to the Ruleset object of an Infoblox appliance. See Infoblox::DNS::Ruleset->`nxdomain_rules()` for parameters and return values.

Example

#Construct a rule my $rule1 = Infoblox::DNS::Ruleset::NxdomainRule->new(pattern => "www.a.zone.com", action => "PASS");

Configure a rule for the Ruleset object $ruleset->nxdomain_rules([$rule1]);

METHODS

This section describes all the methods you can use to configure and retrieve the attribute values of a Rule object.

pattern( )

Use this method to specify the pattern that is used to match domain names.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

The pattern in string format, with a maximum of 1024 characters.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get pattern
my $pattern = $ruleset_item->pattern();
#Modify pattern
$ruleset_item->pattern("^*$");
```

action( )

Use this method to specify the action to perform when a domain name matches the pattern defined in this Infoblox::DNS::Ruleset::NxdomainRule object.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

The valid value is "PASS", "REDIRECT" or "MODIFY".

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get action
my $action = $ruleset_item->action();
#Modify action
$ruleset_item->action("PASS");
```

AUTHOR

Infoblox Inc. http://www.infoblox.com/

SEE ALSO

Infoblox::Session, Infoblox::DNS::Ruleset

COPYRIGHT

Copyright (c) 2017 Infoblox Inc.

Infoblox::DNS::SharedRecord::A - DNS shared A record object.

- NAME
- DESCRIPTION
- CONSTRUCTOR
- SESSION METHODS
  - Infoblox::Session->add( )
  - Infoblox::Session->get( )
  - Infoblox::Session->modify( )
  - Infoblox::Session->remove( )
  - Infoblox::Session->search( )
- METHODS
  - comment( )
  - disable( )
  - dns_name( )
  - extattrs( )
  - extensible_attributes( )
  - ipv4addr( )
  - name( )
  - shared_record_group( )
  - ttl( )
- SAMPLE CODE
- AUTHOR
- SEE ALSO
- COPYRIGHT

NAME

Infoblox::DNS::SharedRecord::A - DNS shared A record object.

DESCRIPTION

A shared A (address) record is similar to a regular A record. It maps a domain name to an IPv4 address. The difference is that a shared A record should be added into a shared record group. If this shared record group is associated with other zones, then this shared A record will be shared into these zones. For more information about how shared record groups and shared records work, please refer to Infoblox Administrator Guide.

CONSTRUCTOR

```
 my $shared_a = Infoblox::DNS::SharedRecord::A->new (
     ipv4addr                => $ipv4addr,          #Required
     name                    => $string,            #Required
     shared_record_group     => $string,            #Required
     comment                 => $string,            #Optional / Default is empty
     disable                 => "true" | "false"    #Optional / Default "false"
     extattrs                => { $string => $extattr, ... },      #Optional / Default is undefined
     extensible_attributes => { $string => $string|$num, $string => [ $string|$num, ...], ... }, #Optional
/ Default is undefined
     ttl                     => $num | undef,       #Optional / Default is undefined
 );
```

You cannot set both extattrs and extensible_attributes attributes at the same time.

SESSION METHODS

This section describes all the methods in Infoblox::Session module that you can apply to a DNS shared A object.

Infoblox::Session->add( )

Use this method to add an object to the Infoblox appliance. See Infoblox::Session->add() for parameters and return values.

Example

```
       #Construct an object
       my $shared_a = Infoblox::DNS::SharedRecord::A->new (
           ipv4addr            => "1.1.1.1",
           name                => "shared_a",
           comment             => "this is a demo shared A record",
           disable             => "true",
```

```
    ttl                 => "33",
    shared_record_group => "test_srg"  );
# Submit for addition
my $response = $session->add( $shared_a );
```

Infoblox::Session->get( )

Use this method to retrieve all the matching objects from the Infoblox appliance. See Infoblox::Session->get() for parameters and return values.

Key References

```
Apply the following attributes to get a specific DNS shared A object:

 name - Required. A host name in string format.
 extattrs     - Optional. A hash reference containing extensible attributes.
 extensible_attributes - Optional. A hash reference containing extensible attributes.
```

Example

```
my @retrieved_objs = $session->get(
    object => "Infoblox::DNS::SharedRecord::A",
    name   => "shared_a");

# get all A shared records with a given extensible attribute
my @retrieved_objs = $session->get(
    object => "Infoblox::DNS::SharedRecord::A",
    extensible_attributes => { 'Site' => 'Santa Clara' });
```

Infoblox::Session->modify( )

Use this method to modify an object in the Infoblox appliance. See Infoblox::Session->modify() for parameters and return values.

Example

```
# Use method to modify the comment.
$shared_a->comment("this is a modified comment");
# Submit modification
my $response = $session->modify( $shared_a );
```

Infoblox::Session->remove( )

Use this method to submit to remove an object from the Infoblox appliance. See Infoblox::Session->remove() for parameters and return values.

To remove a specifc object, first use `get()` or `search()` to retrieve the specific object, and then submit this object for removal.

Example

```
# Get the objects with the same name
my @retrieved_objs = $session->get(
    object => "Infoblox::DNS::SharedRecord::A",
    name   => "shared_a");
# Find the desired object from the retrieved list.
my $desired_a = $retrieved_objs[0];
# Submit for removal
my $response = $session->remove( $desired_a );
```

Infoblox::Session->search( )

Use this method to search for DNS shared A record objects in the Infoblox appliance. See Infoblox::Session->search() for para meters and return values.

Key References

```
Apply the following attributes to search for a DNS shared A record object:

 name - Required. Reverse host name in string format (regular expression).
 comment - Optional. A comment in string format (regular expression).
 extattrs     - Optional. A hash reference containing extensible attributes.
 extensible_attributes - Optional. A hash reference containing extensible attributes.
```

For more information about searching extensible attributes, see Infoblox::Grid::ExtensibleAttributeDef/Searching Extensible Attributes.

Example

```
# search for all DNS A objects under domain.com in default view
my @retrieved_objs = $session->search(
```

```
    object => "Infoblox::DNS::SharedRecord::A",
    name   => "shared");

# search all DNS shared A records with the extensible attribute 'Site'
my @retrieved_objs = $session->search(
   object => "Infoblox::DNS::SharedRecord::A",
   extensible_attributes => { 'Site' => 'Santa Clara' });
```

METHODS

This section describes all the methods that you can use to configure and retrieve the attribute values of a DNS shared A record object.

comment( )

Use this method to set or retrieve the descriptive comment of a DNS shared A record.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

Desired comment in string format with a maximum of 256 bytes.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get comment
my $comment = $shared_a->comment();
#Modify comment
$shared_a->comment("Modifying the DNS A comment");
```

disable( )

Use this method to set or retrieve the disable flag of a DNS shared A record.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

The default value for this field is false. The DNS shared A record is enabled.

Parameter

Specify "true" to set the disable flag or "false" to deactivate/unset it.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get disable
my $disable = $shared_a->disable();
#Modify disable
$shared_a->disable("true");
```

dns_name( )

Use this method to retrieve the host name in punycode format. This is a read-only attribute.

Parameter

None

Returns

The method returns the attribute value.

Example

```
# Get attribute value
my $value = $shared_a->dns_name();
```

extattrs( )

Use this method to set or retrieve the extensible attributes associated with a DNS shared A record object.

Parameter

Valid value is a hash reference containing the names of extensible attributes and their associated values (Infoblox::Grid::Extattr objects).

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get extattrs
my $ref_extattrs = $shared_a->extattrs();
#Modify extattrs
$shared_a->extattrs({ 'Site' => $extattr1, 'Administrator' => $extattr2 });
```

extensible_attributes( )

Use this method to set or retrieve the extensible attributes associated with a DNS shared A record.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

For valid values for extensible attributes, see Infoblox::Grid::ExtensibleAttributeDef/Extensible Attribute Values.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get extensible attributes
my $ref_extensible_attributes = $shared_a->extensible_attributes();
#Modify extensible attributes
$shared_a->extensible_attributes({ 'Site' => 'Santa Clara', 'Administrator' => [ 'Peter', 'Tom' ]
});
```

ipv4addr( )

Use this method to set or retrieve the IPv4 address of a DNS shared A record.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

An IPv4 address is a 32-bit number in dotted decimal notation. It consists of four 8-bit groups of decimal digits separated by decimal points (example: 192. 168.1.2).

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get ipv4addr
my $ipv4addr = $shared_a->ipv4addr();
#Modify ipv4addr
$shared_a->ipv4addr("2.2.2.2");
```

name( )

Use this method to set or retrieve the host name of a DNS shared A record.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

The attribute value can be in unicode format.

Parameter

Hostname of this shared A record. This is not the FQDN name of the host name. It should only be the hostname portion of FQDN (Fully Qualified Domain Name) name. For example, if this shared A record 'shared_a' will be shared in zone test.com, then the name should be 'shared_a'. System will append 'test.com' when this shared A record is populated and shared in zone test.com.

A hostname can have a maximum of 256 bytes.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get name
my $name = $shared_a->name();
#Modify name
$shared_a->name("shared_a");
```

shared_record_group( )

Use this method to set or retrieve the shared record group of a DNS shared A record.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

The name of defined Infoblox::DNS::SRG objects. The shared A record will be defined under this shared record group and shared among zones associated with this shared record group.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get shared record group
my $srg = $shared_a->shared_record_group();
#Modify shared record group
$shared_a->shared_record_group("test_srg");
```

ttl( )

Use this method to set or retrieve the Time to Live (TTL) value of a DNS shared A record.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

The default value is undefined which indicates that the record inherits the TTL value of the zone.

Specify a TTL value to override the TTL value at the zone level.

Parameter

A 32-bit integer (range from 0 to 4294967295) that represents the duration in seconds that the record is cached. Zero indicates that the record should not be cached.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get ttl
my $ttl = $shared_a->ttl();
#Modify ttl
$shared_a->ttl(1800);
#Un-override ttl
$shared_a->ttl(undef);
```

SAMPLE CODE

The following sample code demonstrates the different functions that can be applied to an object such as add, search, modify, and remove.

This sample also includes error handling for the operations.

#Preparation prior to a DNS shared A record insertion

```
#PROGRAM STARTS: Include all the modules that will be used
use strict;
use Infoblox;

#Create a session to the Infoblox Appliance
my $session = Infoblox::Session->new(
    master   => "192.168.1.2",
    username => "admin",
    password => "infoblox"
);
unless ($session) {
   die("Construct session failed: ",
       $session->status_code() . ":" . $session->status_detail());
}
print "Session created successfully\n";

#Create the shared record group prior to an shared A record insertion
my $srg = Infoblox::DNS::SRG->new(name => "test_srg");
unless ($srg) {
   die("Construct srg failed: ",
       Infoblox::status_code() . ":" . Infoblox::status_detail());
}
print "SRG object created successfully\n";

#Verify if the SRG exists
my $object = $session->get(object => "Infoblox::DNS::SRG", name => "test_srg");
unless ($object) {
   print "SRG does not exist on server, safe to add the SRG\n";
   $session->add($SRG)
      or die("Add SRG failed: ",
             $session->status_code() . ":" . $session->status_detail());
}
print "SRG added successfully\n";
```

#Create a DNS shared A record

```
#Construct a DNS A object
my $shared_a = Infoblox::DNS::SharedRecord::A->new(
    name     => "shared_a",
    comment  => " this is a demo shared_a record ",
    ipv4addr => "1.1.1.1",
    shared_record_group => "test_srg"
);
unless ($shared_a) {
   die("Construct DNS record A failed: ",
       Infoblox::status_code() . ":" . Infoblox::status_detail());
}
print "DNS A object created successfully\n";

#Add the DNS shared A record object to Infoblox Appliance through a session
$session->add($shared_a)
    or die("Add record A failed: ",
           $session->status_code() . ":" . $session->status_detail());
print "DNS A object added to server successfully\n";
```

#Search for a specific DNS shared A record

```
#Search all shared A records
my @retrieved_objs = $session->search(
    object => "Infoblox::DNS::SharedRecord::A",
    name   => ".*"
);
my $object = $retrieved_objs[0];
unless ($object) {
    die("Search shared record A failed: ",
        $session->status_code() . ":" . $session->status_detail());
}
print "Search DNS shared A object found at least 1 matching entry\n";
```

#Get and modify a DNS shared A record

```
#Get shared A record through the session
my @retrieved_objs = $session->get(
```

```
    object => "Infoblox::DNS::SharedRecord::A",
    name   => "shared_a"
);
my $object = $retrieved_objs[0];
unless ($object) {
    die("Get shared record A failed: ",
        $session->status_code() . ":" . $session->status_detail());
}
print "Get DNS shared A object found at least 1 matching entry\n";

#Modify one of the attributes of the specified shared A record
$object->ipv4addr("3.3.3.3");

#Apply the changes
$session->modify($object)
    or die("Modify record A failed: ",
           $session->status_code() . ":" . $session->status_detail());
print "DNS A object modified successfully \n";
```

#Remove a DNS shared A record

```
#Get shared A record through the session
my @retrieved_objs = $session->get(
    object => "Infoblox::DNS::SharedRecord::A",
    name   => "shared_a"
);
my $object = $retrieved_objs[0];
unless ($object) {
    die("Get shared record A failed: ",
        $session->status_code() . ":" . $session->status_detail());
}
print "Get DNS shared A object found at least 1 matching entry\n";

#Submit the object for removal
$session->remove($object)
    or die("Remove shared record A failed: ",
        $session->status_code() . ":" . $session->status_detail());
print "DNS shared A object removed successfully \n";

####PROGRAM ENDS####
```

AUTHOR

Infoblox Inc. http://www.infoblox.com/

SEE ALSO

Infoblox::DNS::View, Infoblox::DNS::Zone, Infoblox::DNS::SRG, Infoblox::Session, Infoblox::Session->get(), Infoblox::Session->search(), Infoblox::Session->add(), Infoblox::Session->remove(), Infoblox::Session->modify()

COPYRIGHT

Copyright (c) 2017 Infoblox Inc.

Infoblox::DNS::SharedRecord::AAAA - DNS AAAA shared record object.

- NAME
- DESCRIPTION
- CONSTRUCTOR
- SESSION METHODS
  - Infoblox::Session->add( )
  - Infoblox::Session->get( )
  - Infoblox::Session->modify( )
  - Infoblox::Session->remove( )
  - Infoblox::Session->search( )
- METHODS
  - comment( )
  - disable( )
  - dns_name( )
  - extattrs( )
  - extensible_attributes( )
  - ipv6addr( )
  - name( )
  - shared_record_group( )
  - ttl( )
- SAMPLE CODE
- AUTHOR
- SEE ALSO
- COPYRIGHT

NAME

Infoblox::DNS::SharedRecord::AAAA - DNS AAAA shared record object.

DESCRIPTION

A shared AAAA record is similar to a regular AAAA record. It maps a domain name to an IPv6 address. The difference is that a shared AAAA record should be added into a shared record group. If this shared record group is associated with other zones, then this shared AAAA record will be shared into these zones. For more information about how shared record groups and shared records work, please refer to Infoblox Administrator Guide.

CONSTRUCTOR

```
 my $srg_aaaa = Infoblox::DNS::SharedRecord::AAAA->new (
     ipv6addr                => $ipv6addr,          #Required
     name                    => $fqdn,              #Required
     shared_record_group     => $string,            #Required
     comment                 => $string,            #Optional / Default is empty
     disable                 => "true" | "false"    #Optional / Default "false"
     extattrs                => { $string => $extattr, ... },      #Optional / Default is undefined
     extensible_attributes => { $string => $string|$num, $string => [ $string|$num, ...], ... }, #Optional
/ Default is undefined
     ttl                     => $num | undef,       #Optional / Default is undefined
 );
```

You cannot set both extattrs and extensible_attributes attributes at the same time.

SESSION METHODS

This section describes all the methods in Infoblox::Session module that you can apply to a DNS AAAA shared record object.

Infoblox::Session->add( )

Use this method to add a AAAA shared record object to the Infoblox appliance. See Infoblox::Session->add() for parameters and return values.

Example

```
       #Construct an object
       my $srg_aaaa = Infoblox::DNS::SharedRecord::AAAA->new(
           name     => "aaaa1",
           comment  => "add a shared DNS AAAA aaaa1",
           ipv6addr => "aaaa:1111:bbbb:2222:cccc:3333:dddd:4444",
```

```
    disable  => "true",
    ttl      => "33",
    shared_record_group => "test_srg"  );
# Submit for addition
my $response = $session->add( $srg_aaaa );
```

Infoblox::Session->get( )

Use this method to retrieve all the matching objects from the Infoblox appliance. See Infoblox::Session->get() for parameters and return values.

Key References

```
Apply the following attributes to get a specific DNS AAAA object:

 name - Required. A host name in string format.
 extattrs     - Optional. A hash reference containing extensible attributes.
 extensible_attributes - Optional. A hash reference containing extensible attributes.
```

Example

```
my @retrieved_objs = $session->get(
                     object => "Infoblox::DNS::SharedRecord::AAAA",
                     name   => "aaaa1");

# get all shared AAAA records with a given extensible attribute
my @retrieved_objs = $session->get(
    object => "Infoblox::DNS::SharedRecord::AAAA",
    extensible_attributes => { 'Site' => 'Santa Clara' });
```

Infoblox::Session->modify( )

Use this method to modify a AAAA shared record object in the Infoblox appliance. See Infoblox::Session->modify() for parameters and return values.

Example

```
# Use method to modify the comment.
$srg_aaaa->comment("this is a modified comment");
# Submit modification
my $response = $session->modify( $srg_aaaa );
```

Infoblox::Session->remove( )

Use this method to remove a AAAA shared record object from the Infoblox appliance. See Infoblox::Session->remove() for parameters and return values.

To remove a specifc object, first use `get()` or `search()` to retrieve the specific AAAA shared record object, and then submit this object for removal.

Example

```
# Get the objects with the same name
my @retrieved_objs = $session->get(
    object => "Infoblox::DNS::SharedRecord::AAAA",
    name   => "aaaa1");
# Find the desired object from the retrieved list.
my $desired_aaaa = $retrieved_objs[0];
# Submit for removal
my $response = $session->remove( $desired_aaaa );
```

Infoblox::Session->search( )

Use this method to search for DNS AAAA shared record objects in the Infoblox appliance. See Infoblox::Session->search() for parameters and return values.

Key References

```
Apply the following attributes, to search for a DNS AAAA shared record object:

 name    - Required. Reverse host name in string format (regular expression).
 comment - Optional. Comment in string format (regular expression).
 extattrs     - Optional. A hash reference containing extensible attributes.
 extensible_attributes - Optional. A hash reference containing extensible attributes.
```

For more information about searching extensible attributes, see Infoblox::Grid::ExtensibleAttributeDef/Searching Extensible Attributes.

Example

```
# search for all DNS AAAA objects
my @retrieved_objs = $session->search(
    object  => "Infoblox::DNS::SharedRecord::AAAA",
    name    => ".*",
    comment => ".*modified comment" );

# search all DNS shared AAAA records with the extensible attribute 'Site'
my @retrieved_objs = $session->search(
   object => "Infoblox::DNS::SharedRecord::AAAA",
   extensible_attributes => { 'Site' => 'Santa Clara' });
```

METHODS

This section describes all the methods that you can use to configure and retrieve the attribute values of a DNS shared AAAA record object.

comment( )

Use this method to set or retrieve the descriptive comment of a DNS shared AAAA record.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

Desired comment in string format with a maximum of 256 bytes.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get comment
my $comment = $srg_aaaa->comment();
#Modify comment
$srg_aaaa->comment("Modifying the DNS AAAA comment");
```

disable( )

Use this method to set or retrieve the disable flag of a DNS shared AAAA record.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

The default value for this field is false. The DNS shared AAAA record is enabled.

Parameter

Specify "true" to set the disable flag or "false" to deactivate/unset it.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get disable
my $disable = $srg_aaaa->disable();
#Modify disable
$srg_aaaa->disable("true");
```

dns_name( )

Use this method to retrieve the host name in punycode format. This is a read-only attribute.

Parameter

None

Returns

The method returns the attribute value.

Example

```
# Get attribute value
my $value = $srg_aaaa->dns_name();
```

extattrs( )

Use this method to set or retrieve the extensible attributes associated with a DNS shared AAAA record object.

Parameter

Valid value is a hash reference containing the names of extensible attributes and their associated values (Infoblox::Grid::Extattr objects).

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get extattrs
my $ref_extattrs = $srg_aaaa->extattrs();
#Modify extattrs
$srg_aaaa->extattrs({ 'Site' => $extattr1, 'Administrator' => $extattr2 });
```

extensible_attributes( )

Use this method to set or retrieve the extensible attributes associated with a DNS shared AAAA record.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

For valid values for extensible attributes, see Infoblox::Grid::ExtensibleAttributeDef/Extensible Attribute Values.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get extensible attributes
my $ref_extensible_attributes = $srg_aaaa->extensible_attributes();
#Modify extensible attributes
$srg_aaaa->extensible_attributes({ 'Site' => 'Santa Clara', 'Administrator' => [ 'Peter', 'Tom' ]
});
```

ipv6addr( )

Use this method to set or retrieve the IPv6 address of a DNS shared AAAA record.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

An IPv6 address is a 128-bit number in colon hexadecimal notation. It consists of eight 16-bit groups of hexadecimal digits separated by colons (example: 12ab:0000:0000:0123:4567:89ab:0000:cdef).

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get ipv6addr
my ipv6addr = $srg_aaaa->ipv6addr();
#Modify ipv6addr
$srg_aaaa->ipv6addr("aaaa:1111:bbbb:2222:cccc:3333:dddd:5555");
```

name( )

Use this method to set or retrieve the host name of a DNS shared AAAA record.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

The attribute value can be in unicode format.

Parameter

Hostname of this shared AAAA record. This is not the FQDN name of the host name. It should only be the hostname portion of FQDN (Fully Qualified Domain Name) name. For example, if this shared AAAA record 'shared_aaaa' will be shared in zone test.com, then the name should be 'shared_a'. System will append 'test.com' when this shared AAAA record is populated and shared in zone test.com.

A hostname can have a maximum of 256 bytes.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get name
my $name = $shared_aaaaa->name();
#Modify name
$shared_aaaaa->name("shared_aaaa");
```

shared_record_group( )

Use this method to set or retrieve the shared record group of a DNS shared AAAA record.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

The name of defined Infoblox::DNS::SRG objects. The shared AAAA record will be defined under this shared record group and shared among zones associated with this shared record group.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get shared record group
my $srg = $shared_aaaa->shared_record_group();
#Modify shared record group
$shared_aaaa->shared_record_group("test_srg");
```

ttl( )

Use this method to set or retrieve the Time to Live (TTL) value of a DNS shared AAAA record.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

The default value is undefined which indicates that the record inherits the TTL value of the zone.

Specify a TTL value to override the TTL value at the zone level.

Parameter

A 32-bit integer (range from 0 to 4294967295) that represents the duration in seconds that the record is cached. Zero indicates that the record should not be cached.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get ttl
my $ttl = $srg_aaaa->ttl();
#Modify ttl
$srg_aaaa->ttl(1800);
#Un-override ttl
$srg_aaaa->ttl(undef);
```

SAMPLE CODE

The following sample code demonstrates the different functions that can be applied to a shared AAAA record object such as add, search, modify, and remove. This sample also includes error handling for the operations.

#Preparation prior to a DNS shared AAAA shared record insertion

```
#PROGRAM STARTS: Include all the modules that will be used

use strict;
use Infoblox;

#Create a session to the Infoblox Appliance
my $session = Infoblox::Session->new(
    master   => "192.168.1.2", #appliance host ip
    username => "admin",     #appliance user login
    password => "infoblox"   #appliance password
);

unless ($session) {
   die("Construct session failed: ",
       Infoblox::status_code() . ":" . Infoblox::status_detail());
}

#Create the shared record group prior to an shared AAAA record insertion
my $srg = Infoblox::DNS::SRG->new(name => "test_srg");
unless ($srg) {
   die("Construct srg failed: ",
       Infoblox::status_code() . ":" . Infoblox::status_detail());
}
print "SRG object created successfully\n";

#Verify if the SRG exists
my $object = $session->get(object => "Infoblox::DNS::SRg", name => "test_srg");
unless ($object) {
   print "SRG does not exist on server, safe to add the SRG\n";
   $session->add($SRG)
      or die("Add SRG failed: ",
             $session->status_code() . ":" . $session->status_detail());
}
print "SRG added successfully\n";
```

#Create a DNS AAAA shared record

```
#Construct a DNS AAAA object

my $srg_aaaa = Infoblox::DNS::SharedRecord::AAAA->new(
    name     => "aaaa1",
    comment  => "add a DNS shared AAAA aaaa1",
    ipv6addr => "aaaa:1111:bbbb:2222:cccc:3333:dddd:4444",
    shared_record_group => "test_srg"
);

unless ($srg_aaaa) {
   die("Construct DNS shared record AAAA failed: ",
       Infoblox::status_code() . ":" . Infoblox::status_detail());
}
print "DNS shared AAAA object created successfully\n";

#Add the DNS AAAA shared record object to Infoblox Appliance through a session
$session->add($srg_aaaa)

    or die("Add record AAAA failed: ",
           $session->status_code() . ":" . $session->status_detail());
     print "DNS AAAA object added to server successfully\n";
```

#Search for a specific DNS AAAA shared record

```
#Search all AAAA shared records in the zone
my @retrieved_objs = $session->search(
    object => "Infoblox::DNS::SharedRecord::AAAA",
    name   => ".*"
);
my $object = $retrieved_objs[0];
unless ($object) {
    die("Search shared record AAAA failed: ",
        $session->status_code() . ":" . $session->status_detail());
```

```
}
print "Search DNS shared AAAA object found at least 1 matching entry\n";

#Search all AAAA shared records that start with "add" in the comment
my @retrieved_objs = $session->search(
    object    => "Infoblox::DNS::SharedRecord::AAAA",
    comment   => "add.*",
);
my $object = $retrieved_objs[0];
unless ($object) {
    die("Search shared record AAAA failed: ",
        $session->status_code() . ":" . $session->status_detail());
}
print "Search DNS shared AAAA object found at least 1 matching entry\n";

#Search all shared AAAA shared records that start with "aaaa1"
my @retrieved_objs = $session->search(
    object => "Infoblox::DNS::SharedRecord::AAAA",
    name   => "^aaaa1",
 );
my $object = $retrieved_objs[0];
unless ($object) {
    die("Search shared record AAAA failed: ",
        $session->status_code() . ":" . $session->status_detail());
}
print "Search DNS shared AAAA object using regexp found at least 1 matching entry\n";
```

#Get and modify a DNS AAAA shared record

```
#Get AAAA shared record through the session
my @retrieved_objs = $session->get(
    object => "Infoblox::DNS::SharedRecord::AAAA",
    name   => "aaaa1"
);
my $object = $retrieved_objs[0];

unless ($object) {
    die("Get shared record AAAA failed: ",
        $session->status_code() . ":" . $session->status_detail());
}

#Modify one of the attributes of the specified AAAA shared record
$object->ipv6addr("aaaa:1111:bbbb:2222:cccc:3333:dddd:eeee");

#Applying the changes
$session->modify($object)

    or die("Modify shared record AAAA failed: ",
           $session->status_code() . ":" . $session->status_detail());
     print "DNS shared AAAA object modified successfully \n";
```

#Remove a DNS AAAA shared record.

```
#Get AAAA shared record through the session
my @retrieved_objs = $session->get(
    object   => "Infoblox::DNS::SharedRecord::AAAA",
    name     => "aaaa1"
);
my $object = $retrieved_objs[0];
unless ($object) {
    die("Get record AAAA failed: ",
        $session->status_code() . ":" . $session->status_detail());
}

print "Get DNS shared AAAA object found at least 1 matching entry\n";

#Submit the object for removal

$session->remove($object)
    or die("Remove shared record AAAA failed: ",
           $session->status_code() . ":" . $session->status_detail());
print "DNS shared AAAA object removed successfully \n";

####PROGRAM ENDS####
```

AUTHOR

Infoblox::DNS::SharedRecord::AAAA - DNS AAAA shared record object.

Infoblox Inc. http://www.infoblox.com/

SEE ALSO

Infoblox::DNS::View, Infoblox::DNS::Zone, Infoblox::DNS::SRG, Infoblox::Session, Infoblox::Session->get(), Infoblox::Session->search(), Infoblox::Session->add(), Infoblox::Session->remove(), Infoblox::Session->modify()

COPYRIGHT

Copyright (c) 2017 Infoblox Inc.

Infoblox::DNS::SharedRecord::CNAME - DNS shared CNAME record object.

- NAME
- DESCRIPTION
- CONSTRUCTOR
- SESSION METHODS
  - Infoblox::Session->add( )
  - Infoblox::Session->get( )
  - Infoblox::Session->modify( )
  - Infoblox::Session->remove( )
  - Infoblox::Session->search( )
- METHODS
  - canonical( )
  - comment( )
  - disable( )
  - dns_canonical( )
  - dns_name( )
  - extattrs( )
  - extensible_attributes( )
  - name( )
  - shared_record_group( )
  - ttl( )
- AUTHOR
- SEE ALSO
- COPYRIGHT

NAME

Infoblox::DNS::SharedRecord::CNAME - DNS shared CNAME record object.

DESCRIPTION

A shared CNAME (canonical name) record is similar to a regular CNAME record. It maps a domain name to a different domain name. The difference is that a shared CNAME record should be added into a shared record group. If this shared record group is associated with other zones, then this shared CNAME record will be shared into these zones. For more information about how shared record groups and shared records work, please refer to Infoblox Administrator Guide.

CONSTRUCTOR

```
 my $shared_cname = Infoblox::DNS::SharedRecord::CNAME->new (
     name                => $string,                                                    # Required
     canonical           => $string,                                                    # Required
     shared_record_group => $string,                                                    # Required
     comment             => $string,                                                    # Optional
/ Default is undefined
     disable             => "true" | "false"                                            # Optional
/ Default "false"
     extattrs            => { $string => $extattr, ... },                               # Optional
/ Default is undefined
     extensible_attributes => { $string => $string|$num, $string => [ $string|$num, ...], ... }, # Optional
/ Default is undefined
     ttl                 => $num | undef,                                               # Optional
/ Default is undefined
 );
```

You cannot set both extattrs and extensible_attributes attributes at the same time.

SESSION METHODS

This section describes all the methods in Infoblox::Session module that you can apply to a DNS shared CNAME object.

Infoblox::Session->add( )

Use this method to add an object to the Infoblox appliance. See Infoblox::Session->add() for parameters and return values.

Example

```
      #Construct an object
      my $shared_cname = Infoblox::DNS::SharedRecord::CNAME->new(
          canonical           => "shared_cname_canonical",
```

```
    name                => "shared_cname",
    comment             => "this is a demo shared CNAME record",
    disable             => "true",
    ttl                 => "33",
    shared_record_group => "test_srg"
);

# Submit for addition
my $response = $session->add($shared_cname);
```

Infoblox::Session->get( )

Use this method to retrieve all the matching objects from the Infoblox appliance. See Infoblox::Session->get() for parameters and return values.

Key References

```
Apply the following attributes to get a specific DNS shared CNAME object:

 name                - Optional. A host name in string format.
 comment             - Optional. A desired comment in string format.
 canonical           - Optional. A host canonical name in string format.
 extattrs            - Optional. An Infoblox::Grid::Extattr object.
 extensible_attributes - Optional. A hash reference containing extensible attributes.
```

Example

```
my @retrieved_objs = $session->get(
    object => "Infoblox::DNS::SharedRecord::CNAME",
    name   => "shared_cname",
);

# get all CNAME shared records with a given extensible attribute
my @retrieved_objs = $session->get(
    object               => "Infoblox::DNS::SharedRecord::CNAME",
    extensible_attributes => {'Site' => 'Santa Clara'},
);
```

Infoblox::Session->modify( )

Use this method to modify an object in the Infoblox appliance. See Infoblox::Session->modify() for parameters and return values.

Example

```
# Use method to modify the comment.
$shared_cname->comment("this is a modified comment");
# Submit modification
my $response = $session->modify($shared_cname);
```

Infoblox::Session->remove( )

Use this method to remove an object from the Infoblox appliance. See Infoblox::Session->remove() for parameters and return values.

To remove a specific object, first use `get()` or `search()` to retrieve the specific object, and then submit this object for removal.

Example

```
# Get the objects with the same name
my @retrieved_objs = $session->get(
    object => "Infoblox::DNS::SharedRecord::CNAME",
    name   => "shared_cname"
);

# Find the desired object from the retrieved list.
my $desired_cname = $retrieved_objs[0];

# Submit for removal
my $response = $session->remove($desired_cname);
```

Infoblox::Session->search( )

Use this method to search for DNS shared CNAME record objects in the Infoblox appliance. See Infoblox::Session->search() for parameters and return values.

Key References

```
Apply the following attributes to search for a DNS shared CNAME record object:
```

```
  name                  - Optional. Host name in string format (regular expression).
  canonical             - Optional. Host canonical name in string format (regular expression).
  comment               - Optional. A comment in string format (regular expression).
  extattrs              - Optional. An Infoblox::Grid::Extattr object.
  extensible_attributes - Optional. A hash reference containing extensible attributes.
```

For more information about searching extensible attributes, see Infoblox::Grid::ExtensibleAttributeDef/Searching Extensible Attributes.

Example

```
 # search for all DNS CNAME objects with name which matches 'shared'with name which matches 'shared'
expression in default view
 my @retrieved_objs = $session->search(
     object => "Infoblox::DNS::SharedRecord::CNAME",
     name   => "shared",
 );

 # search all DNS shared CNAME records with the extensible attribute 'Site'
 my @retrieved_objs = $session->search(
    object                => "Infoblox::DNS::SharedRecord::CNAME",
    extensible_attributes => {'Site' => 'Santa Clara'},
 );
```

METHODS

This section describes all the methods that you can use to configure and retrieve the attribute values of a DNS shared CNAME record object.

canonical( )

Use this method to set or retrieve the canonical name of a DNS shared CNAME record.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

The canonical name of the host for this shared CNAME record.

Note that this is not the FQDN canonical name of the host name. It is the host name portion of the FQDN name. For example, if the shared CNAME record 'shared_cname' is shared in zone test.com, then the name should be 'shared_cname'. The appliance will append 'test.com' when the shared CNAME record is populated and shared in zone test.com.

A host name can have a maximum of 256 characters.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
 #get canonical value
 my $canonical = $shared_cname->canonical();

 #modify canonical value
 $shared_cname->canonical('new_canonical');
```

comment( )

Use this method to set or retrieve the descriptive comment of a DNS shared CNAME record.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

Desired comment in string format with a maximum of 256 bytes.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
 #Get comment
 my $comment   $shared cname  comment
```

```
                =               ->       ();
#Modify comment
$shared_cname->comment("Modifying the DNS CNAME comment");
```

disable( )

Use this method to set or retrieve the disable flag of a DNS shared CNAME record.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

The default value for this field is false, which means the DNS shared CNAME record is enabled.

Parameter

Specify "true" to set the disable flag or "false" to deactivate/unset it.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get disable
my $disable = $shared_cname->disable();
#Modify disable
$shared_cname->disable("true");
```

dns_canonical( )

Use this method to retrieve the host canonical name in punycode format. This is a read-only attribute.

Parameter

None

Returns

The method returns the attribute value.

Example

```
#get dns_canonical value
my $dns_canonical = $shared_cname->dns_canonical();
```

dns_name( )

Use this method to retrieve the host name in punycode format. This is a read-only attribute.

Parameter

None

Returns

The method returns the attribute value.

Example

```
# Get attribute value
my $value = $shared_cname->dns_name();
```

extattrs( )

Use this method to set or retrieve the extensible attributes associated with a DNS shared CNAME record object.

Parameter

Valid value is a hash reference containing the names of extensible attributes and their associated values (Infoblox::Grid::Extattr objects).

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get extattrs
my $ref_extattrs = $shared_cname->extattrs();
#Modify extattrs
$shared_cname->extattrs({ 'Site' => $extattr1, 'Administrator' => $extattr2 });
```

extensible_attributes( )

Use this method to set or retrieve the extensible attributes associated with a DNS shared CNAME record.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

For valid values for extensible attributes, see Infoblox::Grid::ExtensibleAttributeDef/Extensible Attribute Values.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get extensible attributes
my $ref_extensible_attributes = $shared_cname->extensible_attributes();
#Modify extensible attributes
$shared_cname->extensible_attributes({ 'Site' => 'Santa Clara', 'Administrator' => [ 'Peter', 'Tom'
] });
```

name( )

Use this method to set or retrieve the host name of a DNS shared CNAME record.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

The attribute value can be in unicode format.

Parameter

The host name of the shared CNAME record. Note that this is not the FQDN name of the host name. It is the host name portion of the FQDN name. For example, if this shared CNAME record 'shared_cname' is shared in zone test.com, then th

A host name can have a maximum of 256 characters.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get name
my $name = $shared_cname->name();
#Modify name
$shared_cname->name("shared_cname");
```

shared_record_group( )

Use this method to set or retrieve the shared record group of a DNS shared CNAME record.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

The name of defined Infoblox::DNS::SRG objects. The shared CNAME record will be defined under this shared record group and shared among zones associated with this shared record group.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get shared record group
```

```
my $srg = $shared_cname->shared_record_group();
#Modify shared record group
$shared_cname->shared_record_group("test_srg");
```

ttl( )

Use this method to set or retrieve the Time to Live (TTL) value of a DNS shared CNAME record.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

The default value is undefined which indicates that the record inherits the TTL value of the zone.

Specify a TTL value to override the TTL value at the zone level.

Parameter

A 32-bit integer (range from 0 to 4294967295) that represents the duration in seconds that the record is cached. Zero indicates that the record should not be cached.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get ttl
my $ttl = $shared_cname->ttl();
#Modify ttl
$shared_cname->ttl(1800);
#Un-override ttl
$shared_cname->ttl(undef);
```

AUTHOR

Infoblox Inc. http://www.infoblox.com/

SEE ALSO

Infoblox::DNS::View, Infoblox::DNS::Zone, Infoblox::DNS::SRG, Infoblox::Session, Infoblox::Session->get(), Infoblox::Session->search(), Infoblox::Session->add(), Infoblox::Session->remove(), Infoblox::Session->modify()

COPYRIGHT

Copyright (c) 2017 Infoblox Inc.

Infoblox::DNS::SharedRecord::MX - DNS shared MX record object

- NAME
- DESCRIPTION
- CONSTRUCTOR
- SESSION METHODS
  - Infoblox::Session->add( )
  - Infoblox::Session->get( )
  - Infoblox::Session->modify( )
  - Infoblox::Session->remove( )
  - Infoblox::Session->search( )
- METHODS
  - comment( )
  - disable( )
  - dns_exchanger( )
  - dns_name( )
  - exchanger( )
  - extattrs( )
  - extensible_attributes( )
  - name( )
  - pref( )
  - shared_record_group( )
  - ttl( )
- SAMPLE CODE
- AUTHOR
- SEE ALSO
- COPYRIGHT

NAME

Infoblox::DNS::SharedRecord::MX - DNS shared MX record object

---

DESCRIPTION

A shared MX (mail exchanger) record is similiar to a regular MX record, which maps a domain name to a mail exchanger. The difference is that a shared MX record should be added into a shared record group. If this shared record group is associated with other zones, then this shared MX record will be shared into these zones. For more information about how shared record groups and shared records work, please refer to Infoblox Administrator Guide.

CONSTRUCTOR

```
 my $mx = Infoblox::DNS::SharedRecord::MX->new(
     exchanger               => $fqdn,               #Required
     name                    => $fqdn,               #Required
     pref                    => $num,                #Required
     shared_record_group     => $string,             #Required
     comment                 => $string,             #Optional / Default is empty
     disable                 => "true" | "false",    #Optional / Default "false"
     extattrs                => { $string => $extattr, ... },      #Optional / Default is undefined
     extensible_attributes => { $string => $string|$num, $string => [ $string|$num, ...], ... }, #Optional
/ Default is undefined
     ttl                     => $num | undef         #Optional / Default is undefined
 );
```

You cannot set both extattrs and extensible_attributes attributes at the same time.

SESSION METHODS

This section describes all the methods in Infoblox::Session module that you can apply to a DNS shared MX object.

Infoblox::Session->add( )

Use this method to add an object to the Infoblox appliance. See Infoblox::Session->add() for parameters and return values.

Example

```
       #Construct an object
       my $srg_mx = Infoblox::DNS::SharedRecord::MX->new(
           name      => "bind_mx",
```

```
    comment   => "add mx bind_mx",
    pref      => 1,
    exchanger => "exchanger",
);
# Submit for addition
my $response = $session->add( $srg_mx );
```

Infoblox::Session->get( )

Use this method to retrieve all the matching objects from the Infoblox appliance. See Infoblox::Session->get() for parameters and return values.

Key References

```
Apply the following attributes to get a specific DNS shared MX object:

 name - Required.  A domain name in string format
 extattrs     - Optional. A hash reference containing extensible attributes.
 extensible_attributes - Optional. A hash reference containing extensible attributes.
```

Example

```
 my @retrieved_objs = $session->get(
    object    => "Infoblox::DNS::SharedRecord::MX",
    name      => "bind_mx");

# get all shared MX records with a given extensible attribute
my @retrieved_objs = $session->get(
    object => "Infoblox::DNS::SharedRecord::MX",
    extensible_attributes => { 'Site' => 'Santa Clara' });
```

Infoblox::Session->modify( )

Use this method to modify an object in the Infoblox appliance. See Infoblox::Session->modify() for parameters and return values.

Example

```
#Use method to modify the comment.
$srg_mx->comment("this is a modified comment");
# Submit modification
my $response = $session->modify( $srg_mx );
```

Infoblox::Session->remove( )

Use this method to remove an object from the Infoblox appliance. See Infoblox::Session->remove() for parameters and return values.

To remove a specifc object, first use `get()` or `search()` to retrieve the specific object, and then submit this object for removal.

Example

```
# Get the object under the same name
my @retrieved_objs = $session->get(
    name      => "bind_mx",
    comment   => "add mx bind_mx",
    pref      => 1,
    exchanger => "exchanger");
# find the desired object from retrieved list.
my $desired_mx = $retrieved_objs[0];
# Submit for removal
my $response = $session->remove( $desired_mx );
```

Infoblox::Session->search( )

Use this method to search for DNS shared MX record objects in the Infoblox appliance. See Infoblox::Session->search() for parameters and return values.

Key References

```
Apply the following attributes to search for a DNS shared MX record object:

 name - Required. Reverse domain name in string format (regular expression).
 comment - Optional. A comment in string format (regular expression).
 extattrs     - Optional. A hash reference containing extensible attributes.
 extensible_attributes - Optional. A hash reference containing extensible attributes.
```

For more information about searching extensible attributes, see Infoblox::Grid::ExtensibleAttributeDef/Searching Extensible Attributes.

Example

```
# search for all DNS shared MX objects
    my @retrieved_objs = $session->search(
    object => "Infoblox::DNS::SharedRecord::MX",
    name   => ".*");

# search all DNS shared MX records with the extensible attribute 'Site'
my @retrieved_objs = $session->search(
   object => "Infoblox::DNS::SharedRecord::MX",
   extensible_attributes => { 'Site' => 'Santa Clara' });
```

METHODS

This section describes all the methods that you can use to configure and retrieve the attribute values of a DNS shared MX record object.

comment( )

Use this method to add or retrieve a descriptive comment of a DNS shared MX record.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

Enter a descriptive comment for this record in string format with a maximum of 256 bytes.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
# Getting comment
my $comment = $srg_mx->comment();
# Modifying comment
$srg_mx->comment("add mx2");
```

disable( )

Use this method to set or retrieve the disable flag of a DNS shared MX record.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

The default value for this field is false. The DNS shared MX record is enabled.

Parameter

Specify "true" to set the disable flag or "false" to deactivate/unset it.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
# Getting disable
my $disable = $srg_mx->disable()
# Modifying disable
$srg_mx->disable("true");
```

dns_exchanger( )

Use this method to retrieve the exchanger in punycode format. This is a read-only attribute.

Parameter

None

Returns

The method returns the attribute value.

Example

```
# Get attribute value
```

```
my $value = $srg_mx->dns_exchanger();
```

dns_name( )

Use this method to retrieve the domain name in punycode format. This is a read-only attribute.

Parameter

None

Returns

The method returns the attribute value.

Example

```
# Get attribute value
my $value = $srg_mx->dns_name();
```

exchanger( )

Use this method to set or retrieve the exchanger of a DNS shared MX record.

The name of the mail exchanger in Fully Qualified Domain Name (FQDN) format.

The attribute value can be in unicode format.

Parameter

Text with the name of the mail exchanger for the DNS shared MX record.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails. If you did not specify a parameter, the method returns the attribute value.

Example

```
# Getting exchanger
my $exchanger = $srg_mx->exchanger();
# Modifying exchanger
$srg_mx->exchanger("exchanger2");
```

extattrs( )

Use this method to set or retrieve the extensible attributes associated with a DNS shared MX record object.

Parameter

Valid value is a hash reference containing the names of extensible attributes and their associated values (Infoblox::Grid::Extattr objects).

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get extattrs
my $ref_extattrs = $srg_mx->extattrs();
#Modify extattrs
$srg_mx->extattrs({ 'Site' => $extattr1, 'Administrator' => $extattr2 });
```

extensible_attributes( )

Use this method to set or retrieve the extensible attributes associated with a DNS shared MX record.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

For valid values for extensible attributes, see Infoblox::Grid::ExtensibleAttributeDef/Extensible Attribute Values.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get extensible attributes
my $ref_extensible_attributes = $srg_mx->extensible_attributes();
#Modify extensible attributes
$srg_mx->extensible_attributes({ 'Site' => 'Santa Clara', 'Administrator' => [ 'Peter', 'Tom' ] });
```

name( )

Use this method to set or retrieve the domain name of a DNS shared MX record.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

The attribute value can be in unicode format.

Parameter

Text with the name for the DNS shared MX record.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Getting name
my $name = $srg_mx->name();
# Modifying name
$srg_mx->name("mx2");
```

pref( )

Use this method to set or retrieve the Preference value of a DNS shared MX record.

The preference value of the DNS shared MX object. Preference value of shared MX record for the zone range from 0 to 65535. Low values are more preferred.

Parameter

Enter an unsigned integer between 0-65535. The priority determines the order in which a client attempts to contact the target mail exchanger. The highest priority is 0 and is queried first.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
# Getting pref
my $pref = $srg_mx->pref();
# Modifying pref
$srg_mx->pref(3);
```

shared_record_group( )

Use this method to set or retrieve the shared record group of a DNS shared MX record.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

The name of defined Infoblox::DNS::SRG objects. The shared MX record will be defined under this shared record group and shared among zones associated with this shared record group.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get shared record group
```

```
my $srg = $shared_mx->shared_record_group();
#Modify shared record group
$shared_mx->shared_record_group("test_srg");
```

ttl( )

Use this method to configure or retrieve the Time to Live (TTL) value of a DNS shared MX record.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

The default value is undefined which indicates that the record inherits the TTL value of the zone.

Specify a TTL value to override the TTL value at the zone level.

Parameter

A 32-bit integer (range from 0 to 4294967295) that represents the duration in seconds that the record is cached. Zero indicates that the record should not be cached.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get ttl
my $ttl = $srg_mx->ttl();
#Modify ttl
$srg_mx->ttl(1800);
#Un-override ttl
$srg_mx->ttl(undef);
```

SAMPLE CODE

The following sample code demonstrates the different functions that can be applied to an object such as add, search, modify, and remove. This sample also includes error handling for the operations.

#Preparation prior to a DNS shared MX record insertion

```
#PROGRAM STARTS: Include all the modules that will be used
use strict;
use Infoblox;

#Create a session to the Infoblox appliance
my $session = Infoblox::Session->new(
    master   => "192.168.1.2",
    username => "admin",
    password => "infoblox"
);
unless ($session) {
   die("Construct session failed: ",
       Infoblox::status_code() . ":" . Infoblox::status_detail());
}
print "Session created successfully\n";

#Create the shared record group prior to an shared MX record insertion
my $srg = Infoblox::DNS::SRG->new(name => "test_srg");
unless ($srg) {
   die("Construct srg failed: ",
       Infoblox::status_code() . ":" . Infoblox::status_detail());
}
print "SRG object created successfully\n";

#Verify if the SRG exists
my $object = $session->get(object => "Infoblox::DNS::SRG", name => "test_srg");
unless ($object) {
   print "SRG does not exist on server, safe to add the SRG\n";
   $session->add($SRG)
      or die("Add SRG failed: ",
             $session->status_code() . ":" . $session->status_detail());
}
print "SRG added successfully\n";
```

#Create a DNS shared MX record

```
#Construct a DNS shared MX object
my $srg_mx = Infoblox::DNS::SharedRecord::MX->new(
    name      => "bind_mx",
    comment   => "add mx bind_mx",
    pref      => 1,
    exchanger => "exchanger",
    shared_record_group => "test_srg"
);

unless ($srg_mx) {
   die("Construct DNS shared record MX failed: ",
       Infoblox::status_code() . ":" . Infoblox::status_detail());
}
print "DNS shared MX object created successfully\n"; #Add the DNS shared MX record object to Infoblox
appliance through a session
$session->add($srg_mx)
    or die("Add shared record MX failed: ",
           $session->status_code() . ":" . $session->status_detail());
print "DNS shared MX object added to server successfully\n";
```

#Search for a specific DNS shared MX record

```
#Search all shared MX records in the zone
my @retrieved_objs = $session->search(
    object => "Infoblox::DNS::SharedRecord::MX",
    name   => ".*"
);
my $object = $retrieved_objs[0];
unless ($object) {
    die("Search shared record MX failed: ",
        $session->status_code() . ":" . $session->status_detail());
}
print "Search DNS shared MX object found at least 1 matching entry\n";

#Search all shared MX records that start with "bind" in the zone
my @retrieved_objs = $session->search(
    object => "Infoblox::DNS::SharedRecord::MX",
    name   => "bind.*"
);
my $object = $retrieved_objs[0];
unless ($object) {
    die("Search shared record MX failed: ",
        $session->status_code() . ":" . $session->status_detail());
}
print "Search DNS shared MX object using regexp found at least 1 matching entry\n";
```

#Get and modify a DNS shared MX record

```
#Get shared MX record through the session
my  @retrieved_objs = $session->get(
    object    => "Infoblox::DNS::SharedRecord::MX",
    name      => "bind_mx",
    pref      => 1,
    exchanger => "exchanger",
);

my $object = $retrieved_objs[0];
unless ($object) {
    die("Get shared record MX failed: ",
        $session->status_code() . ":" . $session->status_detail());
}
print "Get DNS shared MX object found at least 1 matching entry\n";

#Modify one of the attributes of the specified shared MX record
$object->exchanger("exc");

#Apply the changes
$session->modify($object)
    or die("Modify shared record MX failed: ",
           $session->status_code() . ":" . $session->status_detail());
print "DNS shared MX object modified successfully \n";
```

#Remove a DNS shared MX record

```
 #Get shared MX record through the session
 my @retrieved_objs = $session->get(
     object => "Infoblox::DNS::SharedRecord::MX",
```

```
      name   => "bind_mx"
  );
   my $object = $retrieved_objs[0];
   unless ($object) {
    die("Get shared record MX failed: ",
        $session->status_code() . ":" . $session->status_detail());
  }
  print "Get DNS shared MX object found at least 1 matching entry\n";

#Submit the object for removal
$session->remove($object)
    or die("Remove shared record MX failed: ",
        $session->status_code() . ":" . $session->status_detail());
print "DNS shared MX object removed successfully \n";

####PROGRAM ENDS####
```

AUTHOR

Infoblox Inc. http://www.infoblox.com/

SEE ALSO

Infoblox::DNS::View, Infoblox::DNS::Zone, Infoblox::DNS::SRG, Infoblox::Session, Infoblox::Session->get(), Infoblox::Session->search(), Infoblox::Session->add(), Infoblox::Session->remove(), Infoblox::Session->modify()

COPYRIGHT

Copyright (c) 2017 Infoblox Inc.

Infoblox::DNS::SharedRecord::SRV - DNS shared SRV record object

- NAME
- DESCRIPTION
- CONSTRUCTOR
- SESSION METHODS
  - Infoblox::Session->add( )
  - Infoblox::Session->get( )
  - Infoblox::Session->modify( )
  - Infoblox::Session->remove( )
  - Infoblox::Session->search( )
- METHODS
  - comment( )
  - disable( )
  - dns_name( )
  - dns_target( )
  - extattrs( )
  - extensible_attributes( )
  - name( )
  - port( )
  - priority( )
  - target( )
  - shared_record_group( )
  - ttl( )
  - weight( )
- SAMPLE CODE
- AUTHOR
- SEE ALSO
- COPYRIGHT

NAME

Infoblox::DNS::SharedRecord::SRV - DNS shared SRV record object

---

DESCRIPTION

A shared SRV record is similar to a regular SRV record, which is also known as a service record. A SRV record provides information on available services. The difference is that shared SRV records should be added into a shared record group. If this shared record group is associated with other zones, then this shared SRV record will be shared into these zones. For more information about how shared record groups and shared records work, please refer to Infoblox Administrator Guide.

CONSTRUCTOR

```
my $srv = Infoblox::DNS::SharedRecord::SRV->new(
    name                    => $string,              #Required
    port                    => $num,                 #Required
    priority                => $num,                 #Required
    shared_record_group     => $string,              #Required
    target                  => $fqdn,                #Required
    weight                  => $num,                 #Required
    comment                 => $string,              #Optional / Default is empty
    disable                 => "true" | "false"      #Optional / Default "false"
    extattrs                => { $string => $extattr, ... },      #Optional / Default is undefined
    extensible_attributes => { $string => $string|$num, $string => [ $string|$num, ...], ... }, #Optional
/ Default is undefined
    ttl                     => $num | undef,         #Optional / Default is undefined
);
```

You cannot set both extattrs and extensible_attributes attributes at the same time.

SESSION METHODS

This section describes all the methods in Infoblox::Session that you can apply to a DNS shared SRV record object.

Infoblox::Session->add( )

Use this method to add an object to the Infoblox appliance. See Infoblox::Session->add() for parameters and return values.

Example

```
#Construct object
my $srg_srv = Infoblox::DNS::SharedRecord::SRV->new(
    name     => "srg_srv",
    comment  => "add shared srv srg_srv",
    priority => 1,
    port     => 1,
    weight   => 1,
    target   => "old.target.test.org",
    shared_record_group => "test_srg"
);
# Submit for addition
my $response = $session->add( $srg_srv );
```

Infoblox::Session->get( )

Use this method to retrieve the existing objects from the Infoblox appliance. See Infoblox::Session->get() for parameters and return values.

Key Reference

```
Apply the following attributes to get a specified DNS shared SRV object:

 name - Required. A name in string format.
 extattrs     - Optional. A hash reference containing extensible attributes.
 extensible_attributes - Optional. A hash reference containing extensible attributes.
```

Example

```
my @retrieved_objs = $session->get(
    object => "Infoblox::DNS::SharedRecord::SRV",
    name     => "srg_srv");

# get all shared SRV records with a given extensible attribute
my @retrieved_objs = $session->get(
    object => "Infoblox::DNS::SharedRecord::SRV",
    extensible_attributes => { 'Site' => 'Santa Clara' });
```

Infoblox::Session->modify( )

Use this method to modify objects in the Infoblox appliance. See Infoblox::Session->modify() for parameters and return values.

Example

```
# Use method to modify the comment.
$srg_srv->comment("this is a modified comment");
# Submit modification
my $response = $session->modify( $srg_srv);
```

Infoblox::Session->remove( )

Use this method to remove an object from the Infoblox appliance. See Infoblox::Session->remove() for parameters and return values.

To remove a specific object, first use `get()` or `search()` to retrieve the specific object, and then submit this object for removal.

Example

```
# Get the object under the same name
my @retrieved_objs = $session->get(
    object => "Infoblox::DNS::SharedRecord::SRV",
    name     => "srg_srv");
my $desired_srv= $retrieved_objs[0];
# Submit for removal
my $response = $session->remove( $desired_srv );
```

Infoblox::Session->search( )

Use this method to search for existing DNS shared SRV record objects in the Infoblox appliance. See Infoblox::Session->search() for parameters and return values.

Key Reference

```
Apply the following attributes, to search for a DNS shared SRV record object:

  name    - Required. Reverse domain name of the shared SRV record (regular expression).
  comment - Optional . A comment in string format (regular expression).
  extattrs     - Optional. A hash reference containing extensible attributes.
  extensible_attributes - Optional. A hash reference containing extensible attributes.
```

For more information about searching extensible attributes, see Infoblox::Grid::ExtensibleAttributeDef/Searching Extensible Attributes.

Example

```
# search for all DNS shared SRV object with comment containing 'shared'.
my @retrieved_objs = $session->search(
    object => "Infoblox::DNS::SharedRecord::SRV",
    name     => ".*",
    comment  => "shared");

# search all DNS shared SRV records with the extensible attribute 'Site'
my @retrieved_objs = $session->search(
   object => "Infoblox::DNS::SharedRecord::SRV",
   extensible_attributes => { 'Site' => 'Santa Clara' });
```

METHODS

This section describes all the methods that you can use to configure and retrieve the attribute values of a DNS shared SRV record object.

comment( )

Use this method to add or retrieve a descriptive comment of a DNS shared SRV record.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

Desired comment in string format with a maximum of 256 bytes.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

Example

```
#Get comment
my $comment = $sharedsrv->comment();
#Modify comment
$sharedsrv->comment("Modifying the DNS shared SRV comment");
```

disable( )

Use this method to set or retrieve the disable flag of a DNS shared SRV record.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

The default value for this field is false, therefore the DNS shared SRV record is enabled.

Parameter

Specify "true" to set the disable flag or "false" to deactivate/unset it.

Returns

If you specified a parameter,the method returns true when the modification succeeds,and returns false when the operation fails.

Example

```
#Get disable
my $disable = $sharedsrv->disable();
#Modify disable
$sharedsrv->disable("true");
```

dns_name( )

Use this method to retrieve the name, in punycode format, of a DNS shared SRV record. This is a read-only attribute.

Parameter

None

Returns

The method returns the attribute value.

Example

```
# Get attribute value
my $value = $sharedsrv->dns_name();
```

dns_target( )

Use this method to retrieve the target, in punycode format, of a DNS shared SRV record. This is a read-only attribute.

Parameter

None

Returns

The method returns the attribute value.

Example

```
# Get attribute value
my $value = $sharedsrv->dns_target();
```

extattrs( )

Use this method to set or retrieve the extensible attributes associated with a DNS shared SRV record object.

Parameter

Valid value is a hash reference containing the names of extensible attributes and their associated values (Infoblox::Grid::Extattr objects).

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get extattrs
my $ref_extattrs = $sharedsrv->extattrs();
#Modify extattrs
$sharedsrv->extattrs({ 'Site' => $extattr1, 'Administrator' => $extattr2 });
```

extensible_attributes( )

Use this method to set or retrieve the extensible attributes associated with a DNS shared SRV record.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

For valid values for extensible attributes, see Infoblox::Grid::ExtensibleAttributeDef/Extensible Attribute Values.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get extensible attributes
my $ref_extensible_attributes = $sharedsrv->extensible_attributes();
#Modify extensible attributes
$sharedsrv->extensible_attributes({ 'Site' => 'Santa Clara', 'Administrator' => [ 'Peter', 'Tom' ]
});
```

name( )

Use this method to set or retrieve the name of a DNS shared SRV record.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

The attribute value can be in unicode format.

Parameter

The name of the DNS shared SRV record.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

Example

```
#Get name
my $name = $sharedsrv->name();
#Modify name
$sharedsrv->name("modified_srg_srv");
```

port( )

Use this method to add or retrieve a port of a DNS shared SRV record.

Parameter

The port number for the DNS shared SRV record (range from 0 to 65535).

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

Example

```
#Get port
my $port = $sharedsrv->port();
#Modify port
$sharedport->port("2");
```

priority( )

Use this method to add or retrieve the priority of a DNS shared SRV record.

Parameter

Number with the priority for the DNS shared SRV record (range from 0 to 65535).

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

Example

```
#Get priority
my $priority = $sharedsrv->priority();
#Modify priority
$sharedsrv->priority("2");
```

target( )

Use this method to set or retrieve the target of a DNS shared SRV record. The target must be in Fully Qualified Domain Name (FQDN) format.

The attribute value can be in unicode format.

Parameter

Text with the target for the DNS shared SRV record.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

Example

```
#Get target
my $target = $sharedsrv->target();
#Modify target
$sharedsrv->target ("new.target.test.org");
```

shared_record_group( )

Use this method to set or retrieve the shared record group of a DNS shared SRV record.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

The name of defined Infoblox::DNS::SRG objects. The shared SRV record will be defined under this shared record group and shared among zones associated with this shared record group.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get shared record group
my $srg = $shared_srv->shared_record_group();
#Modify shared record group
$shared_srv->shared_record_group("test_srg");
```

ttl( )

Use this method to configure or retrieve the Time to Live (TTL) value of a DNS shared SRV record.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

The default value is undefined; therefore the record inherits the TTL value from the zone level.

With a specified TTL value, the method overrides the zone values with the specified TTL value.

Parameter

A 32-bit integer (range from 0 to 4294967295) that represents the duration in seconds that the record is cached. Zero indicates that the record should not be cached.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

Example

```
#Get ttl
my $ttl = $sharedsrv->ttl();
#Modify ttl
$sharedsrv->ttl(1800);
#Un-override ttl
$sharedsrv->ttl(undef);
```

weight( )

Use this method to set or retrieve the weight of a DNS shared SRV record.

Parameter

The weight of the DNS shared SRV record.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

Example

```
#Get weight
my $weight = $sharedsrv->weight ();
#Modify weight
$sharedsrv->weight ("3");
```

SAMPLE CODE

The following sample code demonstrates different operations that can be applied to an object such as create, search, modify, and remove an object. This sample code also includes error handling for the operations.

#Preparation prior to a DNS shared SRV record insertion

```
#PROGRAM STARTS: Include all the modules that will be used
use strict;
use Infoblox;

#Create a session to the Infoblox appliance
my $session = Infoblox::Session->new(
    master   => "192.168.1.2",
    username => "admin",
```

```
    password => "infoblox"
);
unless ($session) {
   die("Construct session failed: ",
       $session->status_code() . ":" . $session->status_detail());
}
print "Session created successfully\n";

#Create the shared record group prior to an shared SRV record insertion
my $srg = Infoblox::DNS::SRG->new(name => "test_srg");
unless ($srg) {
   die("Construct srg failed: ",
       Infoblox::status_code() . ":" . Infoblox::status_detail());
}
print "SRG object created successfully\n";

#Verify if the SRG exists
my $object = $session->get(object => "Infoblox::DNS::SRG", name => "test_srg");
unless ($object) {
   print "SRG does not exist on server, safe to add the SRG\n";
   $session->add($SRG)
      or die("Add SRG failed: ",
             $session->status_code() . ":" . $session->status_detail());
}
print "SRG added successfully\n";
```

#Create a DNS shared SRV record

```
my $srg_srv = Infoblox::DNS::SharedRecord::SRV->new(
   name     => "srg_srv",
   comment  => "add shared srg_srv",
   priority => 1,
   port     => 1,
   weight   => 1,
   target   => "old.target.test.org",
   shared_record_group => "test_srg",
 );
 unless ($srg_srv) {
    die("Construct DNS shared record SRV failed: ",
        Infoblox::status_code() . ":" . Infoblox::status_detail());
 }
print "DNS shared SRV object created successfully\n";

#Add the DNS shared SRV record object to Infoblox  appliance through a session
$session->add($srg_srv)
    or die("Add shared record SRV failed: ",
           $session->status_code() . ":" . $session->status_detail());
print "DNS shared SRV object added to server successfully\n";
```

#Search for a specific DNS shared SRV record

```
#Search for SRV record
my @retrieved_objs = $session->search(
    "object" => "Infoblox::DNS::SharedRecord::SRV",
    "name" => "srg_srv",
);
my $object = $retrieved_objs[0];
unless ($object) {
    die("Search shared record SRV failed: ",
        $session->status_code() . ":" . $session->status_detail());
}
print "Search DNS shared SRV object found at least 1 matching entry\n";

#Search all shared SRV records that  has "shared" in the comment
my @retrieved_objs = $session->search(
    object => "Infoblox::DNS::SharedRecord::SRV",
    name   => ".*",
    comment => "shared"
);
my $object = $retrieved_objs[0];
unless ($object) {
    die("Search record SRV failed: ",
       $session->status_code() . ":" . $session->status_detail());
}
print "Search DNS shared SRV object using regexp found at least 1 matching entry\n";
```

#Get and modify a DNS shared SRV record

```
#Get a SRV shared record through the session
my @retrieved_objs = $session->get(
    object => "Infoblox::DNS::SharedRecord::SRV",
    name   => "srg_srv",
);
my $object = $retrieved_objs[0];
unless ($object) {
    die("Get shared record SRV failed: ",
        $session->status_code() . ":" . $session->status_detail());
}
print "Get DNS shared SRV object found at least 1 matching entry\n";

#Modify one of the attributes of the specified SRV record
$object->comment ("modified comment for shared SRV rec");

#Apply the changes
$session->modify($object)
    or die("Modify shared record SRV failed: ",
            $session->status_code() . ":" . $session->status_detail());
print "DNS shared SRV object modified successfully \n";
```

#Remove a DNS shared SRV record

```
#Get a SRV record through the session
my @retrieved_objs = $session->get(
    object => "Infoblox::DNS::SharedRecord::SRV",
    name   => "srg_srv",
);
my $object = $retrieved_objs[0];
unless ($object) {
    die("Get shared record SRV failed: ",
        $session->status_code() . ":" . $session->status_detail());
}
print "Get DNS shared SRV object found at least 1 matching entry\n";

#submit the object for removal
$session->remove($object)
    or die("Remove shared record SRV failed: ",
            $session->status_code() . ":" . $session->status_detail());
print "DNS shared SRV object removed successfully \n";

####PROGRAM ENDS####
```

AUTHOR

Infoblox Inc. http://www.infoblox.com/

SEE ALSO

Infoblox::DNS::View, Infoblox::DNS::Zone, Infoblox::DNS::SRG, Infoblox::Session, Infoblox::Session->get(), Infoblox::Session->search(), Infoblox::Session->add(), Infoblox::Session->remove(), Infoblox::Session->modify()

COPYRIGHT

Copyright (c) 2017 Infoblox Inc.

Infoblox::DNS::SharedRecord::TXT - DNS shared TXT record object.

- NAME
- DESCRIPTION
- CONSTRUCTOR
- SESSION METHODS
  - Infoblox::Session->add( )
  - Infoblox::Session->get( )
  - Infoblox::Session->modify( )
  - Infoblox::Session->remove( )
  - Infoblox::Session->search( )
- METHODS
  - comment( )
  - disable( )
  - dns_name( )
  - extattrs( )
  - extensible_attributes( )
  - name( )
  - text( )
  - shared_record_group( )
  - ttl( )
- SAMPLE CODE
- AUTHOR
- SEE ALSO
- COPYRIGHT

NAME

Infoblox::DNS::SharedRecord::TXT - DNS shared TXT record object.

DESCRIPTION

A shared TXT record is similar to a regular TXT record. It contains supplemental information for a host. SPF (Sender Policy Framework) records are specialized TXT records that identify the servers that send mail from a domain. The difference is that shared TXT records should be added into a shared record group. If this shared record group is associated with other zones, then this shared TXT record will be shared into these zones. For more information about how shared record groups and shared records work, please refer to Infoblox Administrator Guide.

CONSTRUCTOR

```
 my $txt = Infoblox::DNS::SharedRecord::TXT->new(
     name                    => $fqdn,             #Required
     text                    => $string            #Required
     shared_record_group     => $string,           #Required
     comment                 => $string,           #Optional / Default is empty
     disable                 => "true" | "false"   #Optional / Default "false"
     extattrs                => { $string => $extattr, ... },      #Optional / Default is undefined
     extensible_attributes => { $string => $string|$num, $string => [ $string|$num, ...], ... }, #Optional
/ Default is undefined
     ttl                     => $num | undef,      #Optional / Default is undefined
 );
```

You cannot set both extattrs and extensible_attributes attributes at the same time.

SESSION METHODS

This section describes all the methods in Infoblox::Session module that you can apply to a TXT shard record object.

Infoblox::Session->add( )

Use this method to add an object to the Infoblox appliance. See Infoblox::Session->add() for parameters and return values.

Example

```
       #Construct the TXT object
       my $srg_txt = Infoblox::DNS::SharedRecord::TXT->new(
           name    => "txt",
           text    => "this is text string",
           comment => "add TXT txt",
```

```
    disable => "false",
    ttl     => "360",
    shared_record_group => "test_srg" );
# Submit for addition
my $response = $session->add( $srg_txt );
```

Infoblox::Session->get( )

Use this method to retrieve all the matching objects from the Infoblox appliance. See Infoblox::Session->get() for parameters and return values.

Key References

```
Apply the following attributes to get a specific DNS shared TXT object:

 name - Required. Host name in string format.
 extattrs     - Optional. A hash reference containing extensible attributes.
 extensible_attributes - Optional. A hash reference containing extensible attributes.
```

Example

```
my @retrieved_objs = $session->get(
    object => "Infoblox::DNS::SharedRecord::TXT",
    name   => "txt");

# get all shared TXT records with a given extensible attribute
my @retrieved_objs = $session->get(
    object => "Infoblox::DNS::SharedRecord::TXT",
    extensible_attributes => { 'Site' => 'Santa Clara' });
```

Infoblox::Session->modify( )

Use this method to modify an object in the Infoblox appliance. See Infoblox::Session->modify() for parameters and return values.

Example

```
# Use method to modify the comment.
$srg_txt->comment("This is a modified comment");
# Submit modification
my $response = $session->modify( $srg_txt );
```

Infoblox::Session->remove( )

Use this method to remove an object from the Infoblox appliance. See Infoblox::Session->remove() for parameters and return values.

To remove a specifc object, first use `get()` or `search()` to retrieve the specific object, and then submit this object for removal.

Example

```
# Get the objects with the same name
my @retrieved_objs = $session->get(
    object => "Infoblox::DNS::SharedRecord::TXT",
    name   => "txt");

# find the desired object from the retrieved list.
my $desired_txt = $retrieved_objs[0];
# Submit for removal
my $response = $session->remove( $ desired_txt );
```

Infoblox::Session->search( )

Use this method to search for DNS shared TXT record objects in the Infoblox appliance. See Infoblox::Session->search() for parameters and return values.

Key References

```
Apply the following attributes to search for a DNS shared TXT record object:

 name    - Required. Reverse domain name of the TXT record (regular expression).
 comment - Optional. A comment in string format (regular expression).
 extattrs     - Optional. A hash reference containing extensible attributes.
 extensible_attributes - Optional. A hash reference containing extensible attributes.
```

For more information about searching extensible attributes, see Infoblox::Grid::ExtensibleAttributeDef/Searching Extensible Attributes.

Example

```
# search for all DNS shared TXT objects.
```

```
my @retrieved_objs = $session->search(
    object => "Infoblox::DNS::SharedRecord::TXT",
    name   => ".*");

# search all DNS shared TXT records with the extensible attribute 'Site'
my @retrieved_objs = $session->search(
   object => "Infoblox::DNS::SharedRecord::TXT",
   extensible_attributes => { 'Site' => 'Santa Clara' });
```

METHODS

This section describes all the methods that you can use to configure and retrieve the attribute values of a DNS shared TXT record object.

comment( )

Use this method to set or retrieve the descriptive comment of a DNS shared TXT record.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

Desired comment in string format with a maximum of 256 bytes.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get comment
my $comment = $srg_txt->comment();
#Modify comment
$srg_txt->comment("Modifying the DNS shared TXT comment");
```

disable( )

Use this method to set or retrieve the disable flag of a DNS shared TXT record.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

The default value for this field is false. The DNS shared TXT record is enabled.

Parameter

Specify "true" to set the disable flag or "false" to deactivate/unset it.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get disable
my $disable = $srg_txt->disable();
#Modify disable
$srg_txt->disable("true");
```

dns_name( )

Use this method to retrieve the name, in punycode format, of a DNS shared TXT record. This is a read-only attribute.

Parameter

None

Returns

The method returns the attribute value.

Example

```
# Get attribute value
my $value = $srg_txt->dns_name();
```

extattrs( )

Use this method to set or retrieve the extensible attributes associated with a DNS shared TXT record object.

Parameter

Valid value is a hash reference containing the names of extensible attributes and their associated values (Infoblox::Grid::Extattr objects).

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get extattrs
my $ref_extattrs = $srg_txt->extattrs();
#Modify extattrs
$srg_txt->extattrs({ 'Site' => $extattr1, 'Administrator' => $extattr2 });
```

extensible_attributes( )

Use this method to set or retrieve the extensible attributes associated with a DNS shared TXT record.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

For valid values for extensible attributes, see Infoblox::Grid::ExtensibleAttributeDef/Extensible Attribute Values.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get extensible attributes
my $ref_extensible_attributes = $srg_txt->extensible_attributes();
#Modify extensible attributes
$srg_txt->extensible_attributes({ 'Site' => 'Santa Clara', 'Administrator' => [ 'Peter', 'Tom' ] });
```

name( )

Use this method to set or retrieve the name of a DNS shared TXT record.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

The attribute value can be in unicode format.

Parameter

The name of the DNS shared TXT object. The name can have a maximum of 256 bytes.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get name
my $name = $srg_txt->name();
#Modify name
$srg_txt->name("new");
```

text( )

Use this method to set or retrieve the text of a DNS shared TXT record.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

Text that you want to associate with the record. It can contain up to 255 bytes.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get name
my $name = $srg_txt->text();
#Modify name
$srg_txt->text("this is new text string");
```

shared_record_group( )

Use this method to set or retrieve the shared record group of a DNS shared TXT record.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

The name of defined Infoblox::DNS::SRG objects. The shared TXT record will be defined under this shared record group and shared among zones associated with this shared record group.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get shared record group
my $srg = $shared_txt->shared_record_group();
#Modify shared record group
$shared_txt->shared_record_group("test_srg");
```

ttl( )

Use this method to set or retrieve the Time to Live (TTL) value of a DNS shared TXT record.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

The default value is undefined which indicates that the record inherits the TTL value of the zone.

Specify a TTL value to override the TTL value at the zone level.

Parameter

A 32-bit integer (range from 0 to 4294967295) that represents the duration in seconds that the record is cached. Zero indicates that the record should not be cached.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get ttl
my $ttl = $srg_txt->ttl();
#Modify ttl
$srg_txt->ttl(1800);
#Un-override ttl
$srg_txt->ttl(undef);
```

SAMPLE CODE

The following sample code demonstrates the different functions that can be applied to an object such as add, search, modify, and remove. This sample also includes error handling for the operations.

#Preparation prior to a DNS shared TXT record insertion

```
#PROGRAM STARTS: Include all the modules that will be used
use strict;
```

```
use Infoblox;

#Create a session to the Infoblox appliance

my $session = Infoblox::Session->new(
               master   => "192.168.1.2", #appliance host ip
               username => "admin",       #appliance user login
               password => "infoblox"     #appliance password
               );

unless ($session) {
       die("Construct session failed: ",
               Infoblox::status_code() . ":" . Infoblox::status_detail());
}
print "Session created successfully\n";

#Create the shared record group prior to an shared TXT record insertion
my $srg = Infoblox::DNS::SRG->new(name => "test_srg");
unless ($srg) {
   die("Construct srg failed: ",
       Infoblox::status_code() . ":" . Infoblox::status_detail());
}
print "SRG object created successfully\n";

#Verify if the SRG exists
my $object = $session->get(object => "Infoblox::DNS::SRG", name => "test_srg");
unless ($object) {
   print "SRG does not exist on server, safe to add the SRG\n";
   $session->add($SRG)
      or die("Add SRG failed: ",
             $session->status_code() . ":" . $session->status_detail());
}
print "SRG added successfully\n";
```

#Create a DNS shared TXT record

```
my $srg_txt = Infoblox::DNS::SharedRecord::TXT->new(
               name    => "txt",
               comment => "add TXT txt",
               text    => "this is text string",
               shared_record_group => "test_srg",
               );

unless ($srg_txt) {
       die("Construct DNS shared record TXT failed: ",
               Infoblox::status_code() . ":" . Infoblox::status_detail());
}

#Add the DNS shared TXT record object to the Infoblox appliance through a session
$session->add($srg_txt)
       or die("Add shared record TXT failed: ",
                      $session->status_code() . ":" . $session->status_detail());
print "DNS shared TXT object added to server successfully\n";
```

#Search for a specific DNS shared TXT record

```
#Search all shared TXT records in the zone
my @retrieved_objs = $session->search(
               object => "Infoblox::DNS::SharedRecord::TXT",
               name   => ".*"
               );
my $object = $retrieved_objs[0];

unless ($object) {
       die("Search shared record TXT failed: ",
               $session->status_code() . ":" . $session->status_detail());
}
print "Search DNS shared TXT object found at least 1 matching entry\n";

#Search all TXT records whose name starts with "txt".
my @retrieved_objs = $session->search(
               object => "Infoblox::DNS::SharedRecord::TXT",
               name   => "txt.*"
               );
my $object = $retrieved_objs[0];

unless ($object) {
```

```
        die("Search shared record TXT failed: ",
                $session->status_code() . ":" . $session->status_detail());
}
print "Search DNS shared TXT object using regexp found at least 1 matching entry\n";
```

#Get and modify a DNS shared TXT record

```
#Get a shared TXT record through the session
my @retrieved_objs = $session->get(
                object => "Infoblox::DNS::SharedRecord::TXT",
                name   => "txt"
                );
my $object = $retrieved_objs[0];

unless ($object) {
        die("Get record TXT failed: ",
                $session->status_code() . ":" . $session->status_detail());
}
print "Get DNS shared TXT object found at least 1 matching entry\n";

#Modify one of the attributes of the specified TXT record
$object->text("this is new text string");

#Apply the changes
$session->modify($object)
        or die("Modify shared record TXT failed: ",
                $session->status_code() . ":" . $session->status_detail());
print "DNS shared TXT object modified successfully \n";
```

#Remove a DNS shared TXT record

```
#Get TXT record through the session
my @retrieved_objs = $session->get(
    object => "Infoblox::DNS::SharedRecord::TXT",
    name   => "txt"
);
my $object = $retrieved_objs[0];
unless ($object) {
    die("Get shared record TXT failed: ",
        $session->status_code() . ":" . $session->status_detail());
}
print "Get DNS shared TXT object found at least 1 matching entry\n";

#Submit the object for removal
$session->remove($object)
        or die("Remove shared record TXT failed: ",
                $session->status_code() . ":" . $session->status_detail());
print "DNS shared TXT object removed successfully \n";

####PROGRAM ENDS####
```

AUTHOR

Infoblox Inc. http://www.infoblox.com/

SEE ALSO

Infoblox::DNS::View, Infoblox::DNS::Zone, Infoblox::DNS::SRG, Infoblox::Session, Infoblox::Session->get(), Infoblox::Session->search(), Infoblox::Session->add(), Infoblox::Session->remove(), Infoblox::Session->modify()

COPYRIGHT

Copyright (c) 2017 Infoblox Inc.

Infoblox::DNS::Sortlist - DNS Sortlist Object.

- NAME
- DESCRIPTION
- CONSTRUCTOR
- MODULE METHODS
  - Infoblox::Grid::DNS->sortlist( )
  - Infoblox::Grid::Member::DNS->sortlist( )
- METHODS
  - match_list( )
  - source_ipv4addr( )
  - source_ipv6addr( )
  - comment( )
- SAMPLE CODE
- AUTHOR
- SEE ALSO
- COPYRIGHT

NAME

Infoblox::DNS::Sortlist - DNS Sortlist Object.

DESCRIPTION

A sortlist sorts the order of addresses in responses made to DNS queries.

CONSTRUCTOR

```
 my $sort_list = Infoblox::DNS::Sortlist->new(
     source_ipv4addr => $ipv4addr | $ipv4_subnet/prefix
#Either source_ipv4addr or source_ipv6addr is required
     source_ipv6addr => $ipv6addr | $ipv6_subnet/prefix
#Either source_ipv4addr or source_ipv6addr is required
     match_list      => [ $ipv4addr | $ipv4_subnet/prefix | $ipv6addr |  $ipv6_subnet/prefix ...] | undef
#Optional / Default is undefined
     comment         => $string| undef,                    #Optional / Default is undefined
 );
```

MODULE METHODS

The following functions are available to apply to an Sortlist object.

Infoblox::Grid::DNS->sortlist( )

Use this function to specify a sortlist at the grid level on the Infoblox appliance. See Infoblox::Grid::DNS->sortlist() for parameters and return values.

Example

```
# Construct the object
my $sortlist = Infoblox::DNS::Sortlist->new(
    match_list      => ["10.20.1.0/24", "10.20.2.0/24"],
    source_ipv4addr => "1.2.3.4"
);
# Configure sortlist on the grid DNS object
my $response = $Grid_DNS->sortlist([$sortlist]);
```

Infoblox::Grid::Member::DNS->sortlist( )

Use this function to specify a sortlist at the grid member level. See Infoblox::Grid::Member::DNS->sortlist() for parameters and return values.

Example

```
# Construct the object
my $sortlist = Infoblox::DNS::Sortlist->new(
    match_list      => ["10.20.1.0/24", "10.20.2.0/24"],
    source_ipv4addr => "1.2.3.4"
);
# Configure sortlist on a Grid Member DNS object
```

```
my $response = $Grid_Member_DNS->sortlist([$sortlist]);
```

METHODS

This section describes all the methods that you can use to configure and retrieve the attribute values of a sortlist object.

match_list( )

Use this method to set or retrieve the match list of a sortlist.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

Array reference which contains IP addresses and/or networks.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get match list.
my $ref_matchlist = $sort_list->match_list();
#Modify match list.
$sort_list->match_list(["10.0.0.10", "20.0.0.0/24", "2001:db8:1234::/48", "2001:db8::1428:57ab"]);
```

source_ipv4addr( )

Use this method to set or retrieve the source IPv4 address of a sortlist object.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

Address or network of querier which will be preferred by the resolver as the result of a query. To specify "ANY" querier, please use "0.0.0.0/0".

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get ipv4addrs
my $source_ipv4addr = $sort_list->source_ipv4addr();
#Modify ipv4addrs
$sort_list->source_ipv4addr("10.20.1.0/24");
```

source_ipv6addr( )

Use this method to set or retrieve the source IPv6 address of a sortlist object.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

Address or network of querier which will be preferred by the resolver as the result of a query.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get ipv6addrs
my $source_ipv6addr = $sort_list->source_ipv6addr();
#Modify ipv6addrs
$sort_list->source_ipv6addr("2001:25d8:44:55::/64");
```

comment( )

Use this method to set or retrieve a descriptive comment of a sortlist object.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

Enter a descriptive comment for the sortlist in string format with a maximum of 256 bytes.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
# Get comment
my $comment = $sort_list->comment();
# Modify comment
$sort_list->comment("test comment");
```

SAMPLE CODE

The following sample code demonstrates the different functions that can be applied to an object, such as add, modify. This sample also includes error handling for the operations.

#Preparation prior to a DNS sortlist object insertion

```
#PROGRAM STARTS: Include all the modules that will be used
use strict;
use Infoblox;

#Create a session to the Infoblox appliance

my $session = Infoblox::Session->new(
           master   => "192.168.1.2",
           username => "admin",
           password => "infoblox"
);

unless ($session) {
   die("Construc session failed: ",
       Infoblox::status_code() . ":" . Infoblox::status_detail());
}
print "Session created successfully\n";
```

#Create a Sortlist object

```
my $sort1 = Infoblox::DNS::Sortlist->new(
    match_list      => ["10.20.1.0/24", "10.20.2.0/24"],
    source_ipv4addr => "1.2.3.4"
);

unless($sort1) {
     die("Construct sort list failed: ",
           Infoblox::status_code() . ":" . Infoblox::status_detail());
}
print "First sort list object (IPv4) created successfully\n";
```

#Create another sortlist to demonstrate IPv6

```
my $sort2 = Infoblox::DNS::Sortlist->new(
    match_list      => ["2001:db8:1234::/48", "2001:db8:345::/64", "192.168.1.1"],
    source_ipv6addr => "2001:db8::1428:57ab"
);

unless($sort2) {
     die("Construct sort list failed: ",
           Infoblox::status_code() . ":" . Infoblox::status_detail());
}
print "Second sort list object (IPv6) created successfully\n";
```

#Get the Member DNS object to add sortlist object to it

```
my @result_array = $session->get(
    object => "Infoblox::Grid::Member::DNS",
```

```
    name    => $host_name
);

unless (scalar(@result_array) == 0) {
    my $memberdns = $result_array[0];

    if ($memberdns) {

        #Apply changes to the Member object.
        $memberdns->sortlist([$sort1, $sort2])
            or die("modify member failed: ",
                   $session->status_code() . ":" . $session->status_detail());
        print "sort lists added to Member DNS object successfully\n";

        #Update member DNS object through the Infoblox session.
        $session->modify($memberdns)
            or die("modify session failed: ",
                   $session->status_code() . ":" . $session->status_detail());
        print "Member DNS object with sortlists updated to Infoblox device successfully\n";

        #Modify existing sortlist objects

        #Modifying the value of the specified object.
        $sort1->source_ipv4addr("10.20.1.0/24");
        print "Modify first sortlist value\n";

        #Modifying the value of the specified object.
        $sort1->match_list(["10.0.0.10", "20.0.0.0/24", "30.0.0.30", "40.10.0.0/24"]);

        #Modifying the value of the specified object.
        $sort2->source_ipv6addr("2001:db8::1428:0/112");
        print "Modify second sortlist value\n";

        #Apply changes to the member object.
        $memberdns->sortlist([$sort1, $sort2])
            or die("modify member failed: ",
                   Infoblox::status_code() . ":" . Infoblox::status_detail());
        print "sort list updated to Member DNS object successfully\n";

        #Update member DNS object through the Infoblox session.
        $session->modify($memberdns)
            or die("modify session failed: ",
                $session->status_code() . ":" . $session->status_detail());
                print "Member DNS object with sortlist updated to Infoblox device successfully\n";
            }
         } else {
             print "No member found with the specified name.";
         }

 ####PROGRAM ENDS####
```

AUTHOR

Infoblox Inc. http://www.infoblox.com/

SEE ALSO

Infoblox::Session,Infoblox::Session->get(), Infoblox::Session->modify(),Infoblox::Grid::Member::DNS->sortlist(), Infoblox::Grid::DNS->sortlist()

COPYRIGHT

Copyright (c) 2017 Infoblox Inc.

Infoblox::DNS::SRG - Create a shared record group.

- NAME
- DESCRIPTION
- CONSTRUCTOR
- SESSION METHODS
  - Infoblox::Session->add( )
  - Infoblox::Session->get( )
  - Infoblox::Session->modify( )
  - Infoblox::Session->remove( )
  - Infoblox::Session->search( )
- METHODS
  - comment( )
  - extattrs( )
  - extensible_attributes( )
  - name( )
  - override_record_name_policy( )
  - record_name_policy( )
  - zone_association( )
- SAMPLE CODE
- AUTHOR
- SEE ALSO
- COPYRIGHT

NAME

Infoblox::DNS::SRG - Create a shared record group.

DESCRIPTION

A shared record group (SRG) is created to contain DNS shared records and share them between different zones. For example, if a group of DNS records needs to be in three different zones, you can include the records in a shared record group and assign the group to the three zones. For more information about shared record groups and shared records, please refer to Infoblox Administrator Guide.

CONSTRUCTOR

```
 my $srg = Infoblox::DNS::SRG->new(
     name                          => $string,               #Required
     comment                       => $string,               #Optional / Default is undefined
     extattrs                      => { $string => $extattr, ... },      #Optional / Default is undefined
     extensible_attributes => { $string => $string|$num, $string => [ $string|$num, ...], ... }, #Optional
/ Default is undefined
     override_record_name_policy => "true" | "false",      #Optional / Default is "false"
     record_name_policy            => $string,               #Optional / Required when
override_record_name_policy is set to "true". Default is undefined
     zone_association              => [$Zone1, $Zone2, ...]  #Optional / Default is undefined
 );
```

You cannot set both extattrs and extensible_attributes attributes at the same time.

SESSION METHODS

This section describes all the methods in Infoblox::Session module that you can apply to a DNS SRG object.

Infoblox::Session->add( )

Use this method to add an SRG object to the Infoblox appliance. See Infoblox::Session->add() for parameters and return values.

Example

```
#Construct an object
my $srg = Infoblox::DNS::SRG->new (
    name     => "test_srg",
    comment  => "this is a demo SRG object",
    );
# Submit for addition
my $response = $session->add( $srg );
```

Infoblox::Session->get( )

Use this method to retrieve all the matching objects from the Infoblox appliance. See Infoblox::Session->get() for parameters and return values.

Key References

```
Apply the following attributes to get a specific DNS SRG object:

 name - Required. A SRG name in string format.
 extattrs     - Optional. A hash reference containing extensible attributes.
 extensible_attributes - Optional. A hash reference containing extensible attributes.
```

Example

```
my @retrieved_objs = $session->get(
    object => "Infoblox::DNS::SRG",
    name   => "test_srg");

# get all shared record group objects with a given extensible attribute
my @retrieved_objs = $session->get(
    object => "Infoblox::DNS::SRG",
    extensible_attributes => { 'Site' => 'Santa Clara' });
```

Infoblox::Session->modify( )

Use this method to modify an object in the Infoblox appliance. See Infoblox::Session->modify() for parameters and return values.

Example

```
# Use method to modify the comment.
$srg->comment("this is a modified comment");
# Submit modification
my $response = $session->modify( $srg );
```

Infoblox::Session->remove( )

Use this method to submit to remove an object from the Infoblox appliance. See Infoblox::Session->remove() for parameters and return values.

To remove a specific object, first use `get()` or `search()` to retrieve the specific object, and then submit this object for removal.

Example

```
# Get the objects with the same name
my @retrieved_objs = $session->get(
    object => "Infoblox::DNS::SRG",
    name   => "test_srg");
# Find the desired object from the retrieved list.
my $desired_srg = $retrieved_objs[0];
# Submit for removal
my $response = $session->remove( $desired_srg );
```

Infoblox::Session->search( )

Use this method to search for DNS SRG record objects in the Infoblox appliance. See Infoblox::Session->search() for parameters and return values.

Key References

```
Apply the following attributes to search for a DNS SRG record object:

 name - Required. A SRG name in string format (regular expression).
 comment - Optional. A comment in string format (regular expression).
 extattrs     - Optional. A hash reference containing extensible attributes.
 extensible_attributes - Optional. A hash reference containing extensible attributes.
```

For more information about searching extensible attributes, see Infoblox::Grid::ExtensibleAttributeDef/Searching Extensible Attributes.

Example

```
# search for all DNS SRG objects with name containing 'test'
my @retrieved_objs = $session->search(
    object => "Infoblox::DNS::SRG",
    name   => ".*test.*");

# search all DNS SRG objects with the extensible attribute 'Site'
my @retrieved_objs = $session->search(
   object => "Infoblox::DNS::SRG",
   extensible_attributes => { 'Site' => 'Santa Clara' });
```

METHODS

This section describes all the methods that you can use to configure and retrieve the attribute values of a DNS SRG object.

comment( )

Use this method to set or retrieve the descriptive comment of a DNS SRG object.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

Desired comment in string format with a maximum of 256 bytes.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get comment
my $comment = $srg->comment();
#Modify comment
$srg->comment("Modifying the DNS SRG comment");
```

extattrs( )

Use this method to set or retrieve the extensible attributes associated with a DNS SRG object.

Parameter

Valid value is a hash reference containing the names of extensible attributes and their associated values (Infoblox::Grid::Extattr objects).

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get extattrs
my $ref_extattrs = $srg->extattrs();
#Modify extattrs
$srg->extattrs({ 'Site' => $extattr1, 'Administrator' => $extattr2 });
```

extensible_attributes( )

Use this method to set or retrieve the extensible attributes associated with a DNS SRG object.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

For valid values for extensible attributes, see Infoblox::Grid::ExtensibleAttributeDef/Extensible Attribute Values.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get extensible attributes
my $ref_extensible_attributes = $srg->extensible_attributes();
#Modify extensible attributes
$srg->extensible_attributes({ 'Site' => 'Santa Clara', 'Administrator' => [ 'Peter', 'Tom' ] });
```

name( )

Use this method to set or retrieve the shared record group name of a DNS SRG object.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

The name of shared record group.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get name
my $name = $srg->name();
#Modify name
$srg->name("test_srg");
```

override_record_name_policy( )

Use this method to set or retrieve the override record name policy flag of a DNS SRG object.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

The default value for this field is false, which means this SRG will use default record name policy setting defined at grid level.

Parameter

Specify "true" to set the override_record_name_policy flag or "false" to deactivate/unset it.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get override_record_name_policy
my $override = $srg->doverride_record_name_policy();
#Modify override_record_name_policy
$srg->override_record_name_policy("true");
```

record_name_policy( )

Use this method to set or retrieve the record name policy of a DNS SRG object.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

The name of the record name policy defined at the grid level. See Infoblox::Grid::DNS for more information about record name policies.

This property is required only if override_record_name_policy is set to "true", which means SRG object will use its own record name policy.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get record name policy
my $policy = $srg->record_name_policy();
#Modify record name policy
$srg->record_name_policy("Allow Any");
```

zone_association( )

Use this method to set or retrieve the zone association of a DNS SRG object. All zone objects in a association list are associated with the SRG object, which means that all shared records in this SRG will be shared among these zones. For more information about SRG and zone association, please refer to Infoblox Administrator's Guide.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

Array reference of defined Infoblox::DNS::Zone objects.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get zone association
my $zone_associations = $srg->zone_association();
#Modify zone associations, list of Infoblox::DNS::Zone objects
$srg->zone_association([$zone1,$zone2,$zone3]);
```

SAMPLE CODE

The following sample code demonstrates the different functions that can be applied to an object such as add, search, modify, and remove. This sample also includes error handling for the operations.

#Preparation prior to a DNS SRG object insertion

```
#PROGRAM STARTS: Include all the modules that will be used
use strict;
use Infoblox;

#Create a session to the Infoblox Appliance
my $session = Infoblox::Session->new(
    master   => "192.168.1.2",
    username => "admin",
    password => "infoblox"
);
unless ($session) {
   die("Construct session failed: ",
       $session->status_code() . ":" . $session->status_detail());
}
print "Session created successfully\n";

#Create the zone prior to an SRG object insertion
my $zone = Infoblox::DNS::Zone->new(name => "domain.com");
unless ($zone) {
   die("Construct zone failed: ",
       Infoblox::status_code() . ":" . Infoblox::status_detail());
}
print "Zone object created successfully\n";

#Verify if the zone exists
my $object = $session->get(object => "Infoblox::DNS::Zone", name => "domain.com");
unless ($object) {
   print "Zone does not exist on server, safe to add the zone\n";
   $session->add($zone)
      or die("Add zone failed: ",
             $session->status_code() . ":" . $session->status_detail());
}
print "Zone added successfully\n";
```

#Create a DNS SRG object

```
#Construct a DNS SRG object
my $srg = Infoblox::DNS::SRG->new(
    name     => "test_srg",
    comment  => "this is a demo srg object",
    zone_association=> [ $zone ],
    override_record_name_policy => "true",
    record_name_policy=>"Allow Underscore"
);
unless ($srg) {
   die("Construct DNS SRG failed: ",
       Infoblox::status_code() . ":" . Infoblox::status_detail());
}
print "DNS SRG object created successfully\n";

#Add the DNS SRG record object to Infoblox Appliance through a session
$session->add($srg)
    or die("Add SRG object failed: ",
           $session->status_code() . ":" . $session->status_detail());
```

```
print "DNS SRG object added to server successfully\n";
```

#Search for a specific DNS SRG record

```
#Search all SRG records with name containing 'test'
my @retrieved_objs = $session->search(
    object => "Infoblox::DNS::SRG",
    name   => ".*test.*"
);
my $object = $retrieved_objs[0];
unless ($object) {
    die("Search SRG failed: ",
        $session->status_code() . ":" . $session->status_detail());
}
print "Search DNS SRG object found at least 1 matching entry\n";
```

#Get and modify a DNS SRG record

```
#Get SRG object through the session
my @retrieved_objs = $session->get(
    object => "Infoblox::DNS::SRG",
    name   => "test_srg"
);
my $object = $retrieved_objs[0];
unless ($object) {
    die("Get SRG failed: ",
        $session->status_code() . ":" . $session->status_detail());
}
print "Get DNS SRG object found at least 1 matching entry\n";

#Modify one of the attributes of the specified SRG object
$object->comment("Updated comment.");

#Apply the changes
$session->modify($object)
    or die("Modify SRG object failed: ",
           $session->status_code() . ":" . $session->status_detail());
print "DNS SRG object modified successfully \n";
```

#Remove a DNS SRG object

```
#Get SRG object through the session
my @retrieved_objs = $session->get(
    object => "Infoblox::DNS::SRG",
    name   => "test_srg"
);
my $object = $retrieved_objs[0];
unless ($object) {
    die("Get SRG object failed: ",
        $session->status_code() . ":" . $session->status_detail());
}
print "Get SRG object found at least 1 matching entry\n";

#Submit the object for removal
$session->remove($object)
    or die("Remove SRG object failed: ",
        $session->status_code() . ":" . $session->status_detail());
print "DNS SRG object removed successfully \n";

####PROGRAM ENDS####
```

AUTHOR

Infoblox Inc. http://www.infoblox.com/

SEE ALSO

Infoblox::DNS::Zone, Infoblox::Grid::DNS, Infoblox::Grid::RecordNamePolicy, Infoblox::Session, Infoblox::Session->get(), Infoblox::Session->search(), Infoblox::Session->add(), Infoblox::Session->remove(), Infoblox::Session->modify()

COPYRIGHT

Copyright (c) 2017 Infoblox Inc.

Infoblox::DNS::TSIGKey - DNS TSIGKey object

- NAME
- DESCRIPTION
- CONSTRUCTOR
- MODULE METHODS
  - Infoblox::Grid::DNS->allow_query( )
  - Infoblox::Grid::Member::DNS->allow_query( )
  - Infoblox::DNS::Zone->allow_query( )
  - Infoblox::Grid::DNS->allow_transfer( )
  - Infoblox::Grid::Member::DNS->allow_transfer( )
  - Infoblox::DNS::Zone->allow_transfer( )
  - Infoblox::Grid::DNS->allow_update( )
  - Infoblox::Grid::Member::DNS->allow_update( )
  - Infoblox::DNS::Zone->allow_update( )
  - Infoblox::Grid::DNS->recursive_query_list( )
  - Infoblox::Grid::Member::DNS->recursive_query_list( )
- METHODS
  - algorithm( )
  - key( )
  - name( )
- SAMPLE CODE
- AUTHOR
- SEE ALSO
- COPYRIGHT

NAME

Infoblox::DNS::TSIGKey - DNS TSIGKey object

DESCRIPTION

On a device you configure as a secondary serer for a zone, you can associate a TSIG key for each primary server to which the secondary server requests zone transfers. On the device you configure as a primary server for a zone, you can set a TSIG key at the grid, member, or zone level. When the secondary server requests zone transfers, it must send a specific key in its requests to the primary server. When the primary server responds to the requests, it can have a set of TSIG keys from which it can draw when responding. As long as the primary server can find the same TSIG key that the secondary sends it, it can verify the authenticity of the requests it receives and authenticate the responses it sends.

TSIG is based on DNS specifications RFC 2845, TSIG and RFC 2930, TKEY which provides authentication protocol enhancements to secure Server-Server transactions.

CONSTRUCTOR

```
my $tsig_key = Infoblox::DNS::TSIGKey->new(
   key  => $string | ":2xCOMPAT",  #Optional / Default is empty
   name => $string,                #Optional / Default is empty
 );
```

MODULE METHODS

This section describes all the functions that you can apply to a TSIGKey object.

Infoblox::Grid::DNS->allow_query( )

Use this function to specify a TSIG key for a Grid DNS object. This key is used to authenticate clients from which DNS queries are allowed.

Example

```
## Construct the object
my $tsig_key = Infoblox::DNS::TSIGKey->new(
    key  => "Zx1jVJl7C58GT/sc7Q3ucA==",
    name => "test_key"
);
#Adding a TSIG key to the allow_query list of the Grid DNS object
#Retrieving the list
my $allow_query_ref= $Grid_DNS->allow_query();
push @$allow_query_ref, $tsig_key;
#Modifying the object;
```

```
my $response = $Grid_DNS->allow_query($allow_query_ref);
```

Infoblox::Grid::Member::DNS->allow_query( )

Use this function to specify a TSIG key for a Grid Member DNS object. This key is used to authenticate clients from which DNS queries are allowed.

Example

```
##Construct the object
my $tsig_key = Infoblox::DNS::TSIGKey->new(
    key  => "ZX1jVjl7C58GT/sc7Q3ucA==",
    name => "test_key"
);
#Adding a TSIG key to the allow_query list of the Grid Member DNS object
#Retrieving the list
my $allow_query_ref= $Grid_Member_DNS->allow_query();
push @$allow_query_ref, $tsig_key;
#Modifying the object;
my $response = $Grid_Member_DNS->allow_query($allow_query_ref);
```

Infoblox::DNS::Zone->allow_query( )

Use this function to specify a TSIG key for a Zone object. This key is used to authenticate clients from which DNS queries are allowed.

Example

```
## Construct the object
my $tsig_key = Infoblox::DNS::TSIGKey->new(
    key  => "ZX1jVJl7c58GT/sc7Q3ucA==",
    name => "test_key"
);
#Adding a TSIG key to the allow_query list of a zone
#Retrieving the list
my $allow_query_ref= $zone->allow_query();
push @$allow_query_ref, $tsig_key;
#Modifying the object;
my $response = $zone->allow_query($allow_query_ref);
```

Infoblox::Grid::DNS->allow_transfer( )

Use this function to specify a TSIG key for a Grid DNS object. This key is used during zone transfers from the primary server to the secondary servers.

Example

```
## Construct the object
my $tsig_key = Infoblox::DNS::TSIGKey->new(
    key  => "Zx1jVJl7C58GT/sc7Q3ucA==",
    name => "test_key"
);
#Adding a TSIG key to the allow_transfer list of the Grid DNS object
#Retrieving the list
my $allow_transfer_ref= $Grid_DNS->allow_transfer();
push @$allow_transfer_ref, $tsig_key;
#Modifying the object;
my $response = $Grid_DNS->allow_transfer($allow_transfer_ref);
```

Infoblox::Grid::Member::DNS->allow_transfer( )

Use this function to specify a TSIGKey for a grid member object. This key is used during zone transfers from the primary server to the secondary servers.

Example

```
## Construct the object
my $tsig_key = Infoblox::DNS::TSIGKey->new(
    key  => "ZX1jVjl7C58GT/sc7Q3ucA==",
    name => "test_key"
);
#Adding a TSIG key to the allow_transfer list of the Grid Member DNS object
#Retrieving the list
my $allow_transfer_ref= $Grid_Member_DNS->allow_transfer();
push @$allow_transfer_ref, $tsig_key;
#Modifying the object;
my $response = $Grid_Member_DNS->allow_transfer($allow_transfer_ref);
```

Infoblox::DNS::Zone->allow_transfer( )

Use this function to specify a TSIGKey for a Zone object. This key is used during zone transfers from the primary server to the secondary servers.

Example

```
## Construct the object
 my $tsig_key = Infoblox::DNS::TSIGKey->new(
   key  => "ZX1jVJl7c58GT/sc7Q3ucA==",
   name => "test_key"
 );
 #Adding a TSIG key to the allow_transfer list of a zone
 #Retrieving the list
 my $allow_transfer_ref= $zone->allow_transfer();
 push @$allow_transfer_ref, $tsig_key;
 #Modifying the object;
 my $response = $zone->allow_transfer($allow_transfer_ref);
```

Infoblox::Grid::DNS->allow_update( )

Use this function to specify a TSIGKey for a grid DNS object. This key is used during dynamic DNS updates.

Example

```
## Construct the object
 my $tsig_key = Infoblox::DNS::TSIGKey->new(
   key  => "ZX1jVJl7C58gT/sc7Q3ucA==",
   name => "test_key"
 );
 #Adding a TSIG key to the allow_update list of the Grid DNS object
 #Retrieving the list
 my $allow_update_ref= $Grid_DNS->allow_update();
 push @$allow_update_ref, $tsig_key;
 #Modifying the object;
 my $response = $Grid_DNS->allow_update($allow_transfer_ref);
```

Infoblox::Grid::Member::DNS->allow_update( )

Use this function to specify a TSIGKey for a grid member object. This key is used during dynamic DNS updates.

Example

```
## Construct the object
my $tsig_key = Infoblox::DNS::TSIGKey->new(
    key  => "ZX1jVJl7C58Gt/sc7Q3ucA==",
    name => "test_key"
);
# Configure a TSIG key for a Grid Member DNS object
#Retrieving the list
my $allow_update_ref= $Grid_Member_DNS->allow_update();
push @$allow_update_ref, $tsig_key;
#Modifying the object;
my $response = $Grid_Member_DNS->allow_update($allow_transfer_ref);
```

Infoblox::DNS::Zone->allow_update( )

Use this function to specify a TSIGKey for a Zone object. This key is used during dynamic DNS updates.

Example

```
## Construct the object
 my $tsig_key = Infoblox::DNS::TSIGKey->new(
   key  => "ZX1jVJl7C58GT/sc7q3ucA==",
   name => "test_key"
 );
 # Configure TSIGKey on a Zone
 #Retrieving the list
 my $allow_update_ref= $zone->allow_update();
 push @$allow_update_ref, $tsig_key;
 #Modifying the object;
 my $response = $zone->allow_update($allow_transfer_ref);
```

Infoblox::Grid::DNS->recursive_query_list( )

Use this function to specify a TSIG key for a Grid DNS object. This key is used to authenticate clients from which the grid is allowed to receive recursive queries.

Example

```
## Construct the object
 my $tsig_key = Infoblox::DNS::TSIGKey->new(
   key  => "ZX1jVJl7C58gT/sc7Q3ucA==",
   name => "test_key"
 );
 #Adding a TSIG key to the recursive_query_list list of the Grid DNS object
 #Retrieving the list
 my $recursive_query_list_ref= $Grid_DNS->recursive_query_list();
 push @$recursive_query_list_ref, $tsig_key;
 #Modifying the object;
 my $response = $Grid_DNS->recursive_query_list($allow_transfer_ref);
```

Infoblox::Grid::Member::DNS->recursive_query_list( )

Use this function to specify a TSIG key for a Grid Member DNS object. This key is used to authenticate clients from which recursive queries are allowed.

Example

Example

```
## Construct the object
my $tsig_key = Infoblox::DNS::TSIGKey->new(
    key  => "ZX1jVJl7C58Gt/sc7Q3ucA==",
    name => "test_key"
);

# Configure a TSIG key for a Grid Member DNS object
#Retrieving the list
my $recursive_query_list_ref= $Grid_Member_DNS->recursive_query_list();
push @$recursive_query_list_ref, $tsig_key;
#Modifying the object;
my $response = $Grid_Member_DNS->recursive_query_list($allow_transfer_ref);
```

METHODS

This section describes all the methods that you can use to configure and retrieve the attribute values of a TSIGKey object.

algorithm( )

Use this method to set or retrieve the algorithm of the TSIG key you want to use.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

Desired algorithm of the TSIG key. Valid values are "HMAC-SHA256" and "HMAC-MD5", which is the default.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
# Getting Algorithm
my $alg = $algorithmserver->algorithm();
# Modifying algorithm
$tsig_key->algorithm("HMAC-SHA256");
```

key( )

Use this method to add or retrieve a TSIGKey. To use an existing TSIG key, type or paste the key in the Key field. The TSIG key string is a text string with a key generated by an encryption algorithm

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

For TSIG zone transfers, valid values for a key are `":2xCOMPAT"` or a TSIG key string. `":2xCOMPAT"` means you are using Infoblox DNSone 2.x TSIG. If the key is `":2xCOMPAT"`, the name can only be `"tsig_xfer"`. For TSIG dynamic updates, the valid value a for key is a TSIG key string.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
# Getting key
my $key = $tsig_key->key();
# Modifying key
$tsig_key->key("ZX1jVJl7C58GT/sc7Q3ucB==");
```

name( )

Use this method to set or retrieve the name of the TSIGKey.

Enter a meaningful name for the key, such as a zone name or the name of the remote name server with which the local server authenticates zone transfer requests and replies. This name must match the name of the TSIG key on the other name server that use it to authenticate zone transfers with the local server.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

Text with the TSIG key name.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
# Getting name
my $name = $tsig_key->name();
# Modifying name
$tsig_key->name("tsig_key1");
```

SAMPLE CODE

The following sample code demonstrates the different functions that can be applied to an object, such as add and modify. This sample also includes error handling for the operations.

##Preparation prior to a DNS TSIGKey object insertion

```
#PROGRAM STARTS: Include all the modules that will be used
use strict;
use Infoblox;

#Create a session to the Infoblox device
my $session = Infoblox::Session->new(
    master   => "192.168.1.2",
    username => "admin",
    password => "infoblox"
);
unless ($session) {
   die("Construct session failed: ",
       Infoblox::status_code() . ":" . Infoblox::status_detail());
}
print "Session created successfully\n";
```

#Create a TSIGKey object

```
 my $tsig_key = Infoblox::DNS::TSIGKey->new(
    key  => "ZX1jVJl7C58GT/sc7Q3ucA==",
    name => "test_key"
  );

unless($tsig_key) {
     die("Construct TSIGKey failed: ",
           Infoblox::status_code() . ":" . Infoblox::status_detail());
}
print "TSIGKey object created successfully\n";

#Get the member DNS object and add a TSIGKey object to it

my @result_array = $session->get(
```

```
    object => "Infoblox::Grid::Member::DNS",
    name   => "infoblox.localdomain"
);

unless (scalar(@result_array) == 0) {
    my $memberdns = $result_array[0];
    if ($memberdns) {

       #Add a TSIG key to the Grid Member DNS object allow_transfer list.
       my $allow_transfer_ref=$memberdns->allow_transfer();
       push @$allow_transfer_ref, $tsig_key;
       $memberdns->allow_transfer($allow_transfer_ref)
            or die("modify member failed: ",
                   Infoblox::status_code() . ":" . Infoblox::status_detail());
        print "TSIGKey added to Member DNS object successfully\n";

    #Update the Grid Member DNS object through the Infoblox session.
        $session->modify($memberdns)
            or die("modify session failed: ",
                   $session->status_code() . ":" . $session->status_detail());
        print "Member DNS object with TSIGKey updated to Infoblox device successfully\n";

        #Modify existing TSIGKey object
        #Modifying the value of the specified object.

        $tsig_key->name("another_test_key");
        print "Modify TSIG name value\n";

        #Replace the key in the allow_transfer list
        my $allow_transfer_ref2=$memberdns->allow_transfer();

        if(ref($allow_transfer_ref2) eq 'ARRAY'){
            map { if((ref($_) eq 'Infoblox::DNS::TSIGKey')&&($_->name() eq 'test_key')){ $_=$tsig_key;} }
@$allow_transfer_ref2;
        }

        $memberdns->allow_transfer($allow_transfer_ref2)
            or die("modify member failed: ",
                   Infoblox::status_code() . ":" . Infoblox::status_detail());
        print "TSIG name updated to Member DNS object successfully\n";

       #Update the Grid Member DNS object through the Infoblox session.
        $session->modify($memberdns)
            or die("modify session failed: ",
                $session->status_code() . ":" . $session->status_detail());
                print "Member DNS object with TSIG name updated to Infoblox device successfully\n";
             }
         } else {
             print "No member found with the specified name.";
         }

 ####PROGRAM ENDS####
```

AUTHOR

Infoblox Inc. http://www.infoblox.com/

SEE ALSO

Infoblox::Session,Infoblox::Session->get(), Infoblox::Session->modify(),Infoblox::Grid::Member::DNS, Infoblox::DNS::Zone, Infoblox::Grid::DNS

COPYRIGHT

Copyright (c) 2017 Infoblox Inc.

Infoblox::DNS::View - DNS View object

- NAME
- DESCRIPTION
- CONSTRUCTOR
- SESSION METHODS
  - Infoblox::Session->add( )
  - Infoblox::Session->get( )
  - Infoblox::Session->modify( )
  - Infoblox::Session->remove( )
  - Infoblox::Session->search( )
- METHODS
  - blacklist_action( )
  - blacklist_log_query( )
  - blacklist_redirect_addresses( )
  - blacklist_redirect_ttl( )
  - blacklist_rulesets( )
  - cloud_info( )
  - comment( )
  - custom_root_name_servers( )
  - ddns_force_creation_timestamp_update( )
  - ddns_restrict_patterns_list( )
  - ddns_restrict_patterns( )
  - ddns_restrict_static( )
  - ddns_restrict_protected( )
  - ddns_restrict_secure( )
  - ddns_principal_tracking( )
  - ddns_principal_group( )
  - disable( )
  - dns64_groups( )
  - dnssec_enabled( )
  - dnssec_expired_signatures_enabled( )
  - dnssec_validation_enabled( )
  - dnssec_blacklist_enabled()
  - dnssec_negative_trust_anchors( )
  - dnssec_dns64_enabled()
  - dnssec_nxdomain_enabled()
  - dnssec_rpz_enabled()
  - dnssec_trusted_keys( )
  - enable_blacklist( )
  - enable_dns64( )
  - enable_match_recursive_only( )
  - enable_fixed_rrset_order_fqdns( )
  - extattrs( )
  - extensible_attributes( )
  - filter_aaaa( )
  - filter_aaaa_list( )
  - fixed_rrset_order_fqdns( )
  - forwarders( )
  - forward_only( )
  - match_clients( )
  - match_destinations( )
  - max_cache_ttl( )
  - max_ncache_ttl( )
  - lame_ttl( )
  - name( )
  - network_view( )
  - notify_delay( )
  - override_blacklist( )
  - override_ddns_force_creation_timestamp_update( )
  - override_ddns_patterns_restriction( )
  - override_ddns_restrict_static( )
  - override_ddns_restrict_protected( )
  - override_ddns_principal_security( )
  - override_dns64( )
  - override_dnssec( )
  - override_filter_aaaa( )
  - override_fixed_rrset_order_fqdns( )
  - override_forwarders( )
  - override_lame_ttl( )
  - override_max_cache_ttl( )
  - override_max_ncache_ttl( )
  - scavenging_settings( )

- override_scavenging_settings( )
- recursion( )
- rpz_drop_ip_rule_enabled( )
- rpz_drop_ip_rule_min_prefix_length_ipv4( )
- rpz_drop_ip_rule_min_prefix_length_ipv6( )
- rpz_qname_wait_recurse( )
- response_rate_limiting( )
- use_root_name_servers( )
- override_nxdomain_redirect( )
- nxdomain_redirect( )
- nxdomain_redirect_addresses( )
- nxdomain_redirect_addresses_ipv6( )
- nxdomain_redirect_ttl( )
- nxdomain_log_query( )
- nxdomain_rulesets( )
- override_rpz_drop_ip_rule( )
- override_rpz_qname_wait_recurse( )
- override_response_rate_limiting( )
- override_sortlist( )
- sortlist( )

- SAMPLE CODE
- AUTHOR
- SEE ALSO
- COPYRIGHT

NAME

Infoblox::DNS::View - DNS View object

DESCRIPTION

DNS views provide the ability to serve one version of DNS data to one set of clients and another version to another set of clients. With DNS views, the Infoblox appliance can provide a different answer to the same DNS query, depending on the source of the query. When the DNS view is not specified, all objects are automatically associated with the default DNS view.

CONSTRUCTOR

```
 my $view = Infoblox::DNS::View->new(
     name                      => $string,
#Required
     blacklist_action          => "REDIRECT" | "REFUSE",
#Optional / Default is "REFUSE"
     blacklist_log_query       => "true" | "false",
#Optional / Default is "false"
     blacklist_redirect_addresses => [$ipv4addr, ... ],
#Optional / Default is undefined
     blacklist_redirect_ttl    => $num,
#Optional / Default is 60
     blacklist_rulesets        => [ $rulesetname1, $rulesetname2, ...],
#Optional / Default is undefined
     comment                   => $string,
#Optional / Default is empty
     custom_root_name_servers => [$RootNameServer1, $RootNameServer2, ...],
#Optional / Default is undefined
     disable                   => "true" | "false",
#Optional / Default "false"
     dns64_groups              => [$group1, $group2, ... ],
#Optional / Default is undefined
     dnssec_enabled            => "true" | "false" | undef,
#Optional / Default is undefined
     dnssec_expired_signatures_enabled => "true" | "false",
#Optional / Default is "false"
     dnssec_validation_enabled         => "true" | "false",
#Optional / Default is "true"
     dnssec_blacklist_enabled => "true" | "false",
#Optional / Default is "false"
     dnssec_negative_trust_anchors => $strig_array
#Optional / Default value is undefined
     dnssec_dns64_enabled      => "true" | "false",
#Optional / Default is "false"
     dnssec_nxdomain_enabled   => "true" | "false",
#Optional / Default is "false"
     dnssec_rpz_enabled        => "true" | "false",
#Optional / Default is "false"
     dnssec_trusted_keys               => [$key1, $key2, ...]
#Optional / Default is undefined
```

```
     enable_blacklist          => "true" | "false" | undef,
#Optional / Default is undefined
     enable_dns64              => "true" | "false" | undef,
#Optional / Default is undefined
     enable_match_recursive_only => "true" | "false"
#Optional / Default is "false"
     enable_fixed_rrset_order_fqdns => "true" | "false" | undef,
#Optional / Default is undefined
     extattrs                  => { $string => $extattr, ... },
#Optional / Default is undefined
     extensible_attributes => { $string => $string|$num, $string => [ $string|$num, ...], ... },
#Optional / Default is undefined
     filter_aaaa               => "YES" | "NO" | "BREAK_DNSSEC",
#Optional / Default is "NO"
     filter_aaaa_list          => [$ipv4addr | $ipv4_subnet/prefix | ...] | $nacl,
#Optional / Default is undefined
     fixed_rrset_order_fqdns   => [$fqdn1, $fqdn2],
#Optional / Default is undefined
     forwarders                => [ $ipv4addr, ...],
#Optional / Default is undefined
     forward_only              => "true" | "false" | undef,
#Optional / Default is undefined
     match_clients             => [$ipv4addr | $ipv4_subnet/prefix | $ipv6addr |
                                   $ipv6_subnet/prefix | "any" | $TSIGKey ...] | $nacl,
#Optional / Default is undefined
     match_destinations        => [$ipv4addr | $ipv4_subnet/prefix | $ipv6addr |
                                   $ipv6_subnet/prefix | "any" | $TSIGKey ...] | $nacl,
#Optional / Default is undefined
     max_cache_ttl             => $uint,
#Optional / Default is 604800 seconds
     max_ncache_ttl            => $uint,
#Optional / Default is 10800 seconds
     network_view              => $Object,
#Optional / Default is "default" network view
     notify_delay              => $num,
#Optional / Default is undefined
     override_dnssec           => "true"| "false",
#Optional / Default is "false"
     override_blacklist        => "true" | "false",
#Optional / Default is "false"
     override_dns64            => "true" | "false",
#Optional / Default is "false"
     override_filter_aaaa      => "true" | "false",
#Optional / Default is "false"
     override_fixed_rrset_order_fqdns => "true", "false",
#Optional / Default is "false"
     override_forwarders       => "true" | "false",
#Optional / Default is "false"
     override_max_cache_ttl    => "true" | "false",
#Optional / Default is "false"
     override_max_ncache_ttl   => "true" | "false",
#Optional / Default is "false"
     override_nxdomain_redirect => "true"| "false",
#Optional / Default is "false"
     override_rpz_qname_wait_recurse => "true" | "false",
#Optional / Default is "false"
     override_sortlist         => "true"| "false",
#Optional / Default is "false"
     recursion                 => "true" | "false",
#Optional / Default "false"
     rpz_qname_wait_recurse    => "true" | "false",
#Optional / Default is undefined
     scavenging_settings      => $scavenging_settings,
#Optional
     sortlist                  => [ $sortlist1, $sortlist2, ...],
#Optional / Default is undefined
     use_root_name_servers     => "true" | "false",
#Optional / Default "false"
     nxdomain_redirect         => "true" | "false",
#Optional / Default is "false"
     nxdomain_redirect_addresses => [$ipv4addr, ...],
#Optional / Default is undefined
     nxdomain_redirect_addresses_ipv6 => [$ipv6addr, ...],
#Optional / Default is undefined
     nxdomain_redirect_ttl     => $num
#Optional / Default is 60
     nxdomain_log_query        => "true" | "false",
#Optional / Default is "false"
     nxdomain_rulesets         => [$reulesetname1, $reulesetname2, ...]
#Optional / Default is undefined
     lame_ttl                  => $uint,
#Optional / Default is 600
     override_lame_ttl         => "true" | "false",
#Optional / Default is "false"
```

```
     response_rate_limiting   => $response_rate_limiting
#Optional / Default is undefined
     ddns_restrict_patterns_list        => [$string, $string, ...],
#Optional / Default is undefined
     ddns_restrict_patterns             => "true" | "false",
#Optional / Default is 'false'
     ddns_restrict_static               => "true" | "false",
#Optional / Default is 'false'
     ddns_restrict_protected            => "true" | "false",
#Optional / Default is 'false'
     ddns_restrict_secure               => "true" | "false",
#Optional / Default is 'false'
     ddns_principal_tracking            => "true" | "false",
#Optional / Default is 'false'
     ddns_principal_group               => $principal_group,
#Optional / Default is undefined
     ddns_force_creation_timestamp_update     => "true" | "false",
#Optional / Default is 'false'
     override_ddns_patterns_restriction => "true" | "false",
#Optional / Default is 'false'
     override_ddns_restrict_static      => "true" | "false",
#Optional / Default is 'false'
     override_ddns_restrict_protected   => "true" | "false",
#Optional / Default is 'false'
     override_ddns_principal_security   => "true" | "false",
#Optional / Default is 'false'
     override_ddns_force_creation_timestamp_update => "true" | "false",
#Optional / Default is 'false'
     override_scavenging_settings                  => "true" | "false",
#Optional / Default is 'false'
     override_rpz_drop_ip_rule                     => "true" | "false" | undef,
#Optional / Default is "false"
     rpz_drop_ip_rule_enabled                      => "true" | "false" | undef,
#Optional / Default is "false"
     rpz_drop_ip_rule_min_prefix_length_ipv4       => $uint | undef,
#Optional / Default is 29
     rpz_drop_ip_rule_min_prefix_length_ipv6       => $uint | undef,
#Optional / Default is 112
 );
```

You cannot set both extattrs and extensible_attributes attributes at the same time.

SESSION METHODS

This section describes all the methods in Infoblox::Session that you can apply to a DNS view object.

Infoblox::Session->add( )

Use this method to add a DNS view object to the Infoblox appliance. See Infoblox::Session->add() for parameters and return values.

If you have applied an access control list to an object, you must restart services after you update the access control list for the changes to take effect. See Infoblox::Session->restart() method.

Example

```
#create member
my $member1 = Infoblox::DNS::Member->new(
    name     => "member.com",
    ipv4addr => "192.168.1.2",
    lead     => "false",
    stealth  => "false"
);

#create keys
my $key1 = Infoblox::DNS::TSIGKey->new(
    name => "tsig_xfer",
    key  => ":2xCOMPAT"
);

my $tsig_key2 = $session->gen_tsig_key();

my $key2 = Infoblox::DNS::TSIGKey->new(
    name => "key2",
    key  => $tsig_key2
);

#create view object using members
my $firstview = Infoblox::DNS::View->new(
    name              => "internal",
```

```
    match_clients     => ["1.1.1.1", $key1 , "10.0.0.0/8", $key2],
    recursion         => "true",
    disable           => "false",
    comment           => "new Internal view"
    )
    or die(qq(constructor for view internal failed: ),
    join(":", Infoblox::status_code(), Infoblox::status_detail()));

#Add a  View through session
$session->add($firstview);
unless ($firstview) {
         die("add view internal failed: ",
              Infoblox::status_code() . ":" . Infoblox::status_detail());
}
```

Infoblox::Session->get( )

Use this method to retrieve an existing DNS view object from the Infoblox appliance. See Infoblox::Session->get() for parameters and return values.

Key Reference

Apply the following attributes to get a specific DNS view object:

```
name - Optional. A name in string format.
extattrs     - Optional. A hash reference containing extensible attributes.
extensible_attributes - Optional. A hash reference containing extensible attributes.
```

Example

```
#get view object from session
my @result = $session->get(
    object => "Infoblox::DNS::View",
    name   => "internal"
);

my $firstview = $result[0];

# get all DNS view objects with a given extensible attribute
my @retrieved_objs = $session->get(
    object => "Infoblox::DNS::View",
    extensible_attributes => { 'Site' => 'Santa Clara' });
```

Infoblox::Session->modify( )

Use this method to modify a DNS view object in the Infoblox appliance. See Infoblox::Session->modify() for parameters and return values.

If you have applied an access control list to an object, you must restart services after you update the access control list for the changes to take effect. See Infoblox::Session->restart() method.

Example

```
# Use method to modify the comment for view object.
$firstview->comment("new comment internal view");
# Submit modification
my $response = $session->modify($firstview);
```

Infoblox::Session->remove( )

Use this function to submit to remove a DNS view object from the Infoblox appliance. See Infoblox::Session->remove() for parameters and return values.

To remove a specific object, first use `get()` or `search()` to retrieve the specific object, and then submit this object for removal.

Example

#Remove an object with a unique name

```
my @result = $session->search(object => "Infoblox::DNS::View", name => "^internal\$");
my $firstview = $result[0];
#Submit for removal
my $response = $session->remove($firstview);
```

Infoblox::Session->search( )

Use this method to search for an existing DNS view object in the Infoblox appliance. See Infoblox::Session->search() for parameters and

return values.

Key Reference

Apply the following attributes to search for a DNS view object:

```
name    - Optional. A name in string format (regular expression).
extattrs     - Optional. A hash reference containing extensible attributes.
extensible_attributes - Optional. A hash reference containing extensible attributes.
```

For more information about searching extensible attributes, see Infoblox::Grid::ExtensibleAttributeDef/Searching Extensible Attributes.

Example

```
#Search for a DNS view by name
my @result = $session->search(object => "Infoblox::DNS::View", name => "^internal\$");

# search for DNS views with the extensible attribute 'Site'
my @retrieved_objs = $session->search(
   object => "Infoblox::DNS::View",
   extensible_attributes => { 'Site' => 'Santa Clara' });
```

METHODS

This section describes all the methods that you can use to configure and retrieve the attribute value of a DNS view object.

blacklist_action( )

Use this method to specify the action to perform when a domain name matches the pattern defined in a rule that is specified by the blacklist_ruleset method.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

Valid values are "REDIRECT" or "REFUSE". The default value is "REFUSE".

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

Examples

```
#Get blacklist_action
my $blacklist_action = $firstview->blacklist_action();
#Modify blacklist_action
$firstview->blacklist_action("REDIRECT");
```

blacklist_log_query( )

Use this method to set or retrieve the flag that indicates whether blacklist redirection queries are logged.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

Specify "true" to enable logging, or "false" to disable it. The default value is "false".

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get blacklist_log_query
my $blacklist_log_query = $firstview->nxdomain_log_query( );
#Modify blacklist_log_query
$firstview->blacklist_log_query("true");
```

blacklist_redirect_addresses( )

Use this method to specify the IP addresses the appliance includes in the response it sends in place of a blacklisted IP address.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

The valid value is an array of IPv4 addresses. The default value is undefined.

Setting this parameter to ["10.0.0.10", "10.0.0.20"] or [], implicitly sets the override_blacklist_redirect attribute to "true". Setting this parameter to undefined causes the appliance to use the member-level blacklist redirection settings and automatically sets the override_blacklist_redirect attribute to "false" if "enable_blacklist_redirect" is also undefined. But if "enable_blacklist_redirect" contains a value, then the override_blacklist_redirect attribute is set to "true".

Note that when blacklist_redirect_addresses contains a valid array reference and override_blacklist_redirect is set to "false", the last operation takes precedence. Thus the sequence $firstview->blacklist_redirect_addresses(["10.0.0.10", "10.0.0.20"]); $firstview->override_blacklist_redirect("false"); sets override_blacklist_redirect to "false", and the sequence $firstview->override_blacklist_redirect("false"); $firstview->blacklist_redirect_addresses(["10.0.0.10", "10.0.0.20"]); sets override_blacklist_redirect to "true".

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get blacklist_redirect_addresses
my $blacklist_redirect_addresses = $firstview->blacklist_redirect_addresses( );
#Modify blacklist_redirect_addresses
$firstview->blacklist_redirect_addresses(["10.0.0.10", "10.0.0.20"]);
```

blacklist_redirect_ttl( )

Use this method to set or retrieve the TTL value of the synthetic DNS responses that result from blacklist redirection.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

A 32-bit integer, from 0 to 4294967295, that represents the TTL in seconds. The default value is 60.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get blacklist_redirect_ttl
my $blacklist_redirect_ttl = $firstview->blacklist_redirect_ttl( );
#Modify blacklist_redirect_ttl
$firstview->blacklist_redirect_ttl(60);
```

blacklist_rulesets( )

Use this method to specify or retrieve Infoblox::DNS::Ruleset object names assigned at the grid level for blacklist redirection.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

The valid value is an array reference that contains the names of the Infoblox::DNS::Ruleset objects assigned to the grid. The names must be in string format.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get blacklist_rulesets
my $blacklist_rulesets = $firstview->blacklist_rulesets();
#Modify blacklist_rulesets
$firstview->blacklist_rulesets(["ruleset1", "ruleset2"]);
```

cloud_info( )

Use this method to retrieve cloud API related information for the Infoblox::DNS::View object.

Parameter

None

Returns

The method returns the attribute value.

Example

```
# Get cloud_info
my $cloud_info = $object->cloud_info();
```

comment( )

Use this method to set or retrieve the comment.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

Desired comment in string format with a maximum of 256 bytes.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

Example

```
#Get comment
my $comment = $firstview->comment();
#Modify comment
$firstview->comment("modifying the internal view comment");
```

custom_root_name_servers( )

Use this method to set or retrieve the list of customized root name servers. You can either select and use Internet root name servers or specify custom root name servers by providing a host name and IP address to which the Infoblox appliance can send queries.Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

Valid value is an array reference that contains Infoblox::DNS::RootNameServer object(s).

Returns

If you specify a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get custom_root_name_servers
my $rootNS_list = $firstview->custom_root_name_servers();
#Modify the root nameserver list
#Creating RootNameServer objects.
my $rootns1 = Infoblox::DNS::RootNameServer->new(
    ipv4addr  => "4.4.4.4",
    host_name => "rns1.test.com",
    );

my $rootns2 = Infoblox::DNS::RootNameServer->new(
    ipv4addr  => "5.5.5.5",
    host_name => "rns2.test.com",
    );

$firstview->custom_root_name_servers([$rootns1, $rootns2]);
```

ddns_force_creation_timestamp_update( )

Use this method to set or retrieve the flag that indicates whether the creation timestamp of resource records (RR) should be updated when DDNS update happens even if there is no change to the RR.

Setting this parameter to defined value, implicitly sets the override_ddns_force_creation_timestamp_update attribute to "true". Setting this parameter to undefined causes the appliance to use the Grid-level ddns_force_creation_timestamp_update flag and automatically sets the override_ddns_force_creation_timestamp_update attribute to "false".

Note that when ddns_force_creation_timestamp_update contains a value and override_ddns_force_creation_timestamp_update is set to

"false", the last operation takes precedence. Thus the sequence $object->ddns_force_creation_timestamp_update("true"); $object->override_ddns_force_creation_timestamp_update("false"); sets override_ddns_force_creation_timestamp_update to "false", and the sequence $object->override_ddns_force_creation_timestamp_update("false"); $object->ddns_force_creation_timestamp_update("true"); sets override_ddns_force_creation_timestamp_update to "true".

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

Specify 'true' to update the creation timestamp of RR on every DDNS update, and 'false' to update the creation timestamp only when RR changes during a DDNS update. The default is 'false'.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get ddns_force_creation_timestamp_update value
$value = $object->ddns_force_creation_timestamp_update();

#modify ddns_force_creation_timestamp_update value
$object->ddns_force_creation_timestamp_update('true');
```

ddns_restrict_patterns_list( )

Use this method to set or retrieve an unordered list of restriction patterns for an option of to restrict DDNS updates based on FQDN patterns.

Setting this parameter to any list, implicitly sets the override_ddns_patterns_restriction attribute to "true". Setting this parameter to undefined causes the appliance to use the Grid-level DDNS update restrictions and automatically sets the override_ddns_patterns_restriction attribute to "false" if "ddns_restrict_patterns" is also undefined. But if "ddns_restrict_patterns" contains a value, then the override_ddns_patterns_restriction attribute is set to "true".

Note that when ddns_restrict_patterns_list contains a valid array reference and override_ddns_patterns_restriction is set to "false", the last operation takes precedence. Thus the sequence $firstview->ddns_restrict_patterns_list([$pattern1, $pattern2]); $firstview->override_ddns_patterns_restriction("false"); sets override_ddns_patterns_restriction to "false", and the sequence $firstview->override_ddns_patterns_restriction("false"); $firstview->ddns_restrict_patterns_list([$pattern1, $pattern2]); sets override_ddns_patterns_restriction to "true".

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

The valid value is an array of FQDNs in a string format.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#get ddns_restrict_patterns_list value
$ddns_restrict_patterns_list = $firstview->ddns_restrict_patterns_list();

#modify ddns_restrict_patterns_list value
$firstview->ddns_restrict_patterns_list(['foo.com', 'bar.com']);
```

ddns_restrict_patterns( )

Use this method to set or retrieve the flag which indicates whether an option to restrict DDNS update request based on FQDN patterns is enabled or disabled.

Setting this parameter to defined value, implicitly sets the override_ddns_patterns_restriction attribute to "true". Setting this parameter to undefined causes the appliance to use the Grid-level DDNS update restrictions and automatically sets the override_ddns_patterns_restriction attribute to "false" if "ddns_restrict_patterns_list" is also undefined. But if "ddns_restrict_patterns_list" contains a value, then the override_ddns_patterns_restriction attribute is set to "true".

Note that when ddns_restrict_patterns contains a value and override_ddns_patterns_restriction is set to "false", the last operation takes precedence. Thus the sequence $firstview->ddns_restrict_patterns("true"); $firstview->override_ddns_patterns_restriction("false"); sets override_ddns_patterns_restriction to "false", and the sequence $firstview->override_ddns_patterns_restriction("false"); $firstview->ddns_restrict_patterns("true"); sets override_ddns_patterns_restriction to "true".

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

Specify 'true' to enable DDNS update request restriction base on FQDN patterns and 'false to disable it. The default value is 'false'.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#get ddns_restrict_patterns value
$ddns_restrict_patterns = $firstview->ddns_restrict_patterns();

#modify ddns_restrict_patterns value
$firstview->ddns_restrict_patterns('true');
```

ddns_restrict_static( )

Use this method to set or retrieve the flag which indicates whether an option to restrict DDNS update request to resource records which are marked as 'STATIC' is enabled or disabled.

Setting this parameter to defined value, implicitly sets the override_ddns_restrict_static attribute to "true". Setting this parameter to undefined causes the appliance to use the Grid-level ddns_restrict_static flag and automatically sets the override_ddns_restrict_static attribute to "false".

Note that when ddns_restrict_static contains a value and override_ddns_restrict_static is set to "false", the last operation takes precedence. Thus the sequence $firstview->ddns_restrict_static("true"); $firstview->override_ddns_restrict_static("false"); sets override_ddns_restrict_static to "false", and the sequence $firstview->override_ddns_restrict_static("false"); $firstview->ddns_restrict_static("true"); sets override_ddns_restrict_static to "true".

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

Specify 'true' to enable DDNS update requests to resource records which are marked as 'STATIC' and 'false' to disable it. The default value is 'false'.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#get ddns_restrict_static value
$ddns_restrict_static = $firstview->ddns_restrict_static();

#modify ddns_restrict_static value
$firstview->ddns_restrict_static('true');
```

ddns_restrict_protected( )

Use this method to set or retrieve the flag which indicates whether an option to restrict DDNS update request to protected resource records is enabled or disabled.

Setting this parameter to defined value, implicitly sets the override_ddns_restrict_protected attribute to "true". Setting this parameter to undefined causes the appliance to use the Grid-level ddns_restrict_protected flag and automatically sets the override_ddns_restrict_protected attribute to "false".

Note that when ddns_restrict_protected contains a value and override_ddns_restrict_protected is set to "false", the last operation takes precedence. Thus the sequence $firstview->ddns_restrict_protected("true"); $firstview->override_ddns_restrict_protected("false"); sets override_ddns_restrict_protected to "false", and the sequence $firstview->override_ddns_restrict_protected("false"); $firstview->ddns_restrict_protected("true"); sets override_ddns_restrict_protected to "true".

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

Specify 'true' to enable DDNS update requests to protected resource records and 'false' to disable it. The default value is 'false'.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#get ddns_restrict_protected value
$ddns_restrict_protected = $firstview->ddns_restrict_protected();

#modify ddns_restrict_protected value
$firstview->ddns_restrict_protected('true');
```

ddns_restrict_secure( )

Use this method to set or retrieve the flag which indicates whether DDNS update request for principal other than target resource record's principal is restricted.

Setting this parameter to a defined value, implicitly sets the override_ddns_principal_security attribute to "true". Setting this parameter to undefined causes the appliance to use the Grid-level DDNS principal security setting and automatically sets the override_ddns_principal_security attribute to "false" if "ddns_principal_tracking" and "ddns_principal_group" are also undefined. But if "ddns_principal_tracking" or "ddns_principal_group" contains a value, then the override_ddns_principal_security attribute is set to "true".

Note that when ddns_restrict_secure contains a value and override_ddns_principal_security is set to "false", the last operation takes precedence. Thus the sequence $firstview->ddns_restrict_secure("true"); $firstview->override_ddns_principal_security("false"); sets override_ddns_principal_security to "false", and the sequence $firstview->override_ddns_principal_security("false"); $firstview->ddns_restrict_secure("true"); sets override_ddns_principal_security to "true".

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

Specify 'true' to enable DDNS update requests restriction for principal other than target resource record's principal and 'false' to disable it. The default value is 'false'.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#get ddns_restrict_secure value
$ddns_restrict_secure = $firstview->ddns_restrict_secure();

#modify ddns_restrict_secure value
$firstview->ddns_restrict_secure('true');
```

ddns_principal_tracking( )

Use this method to set or retrieve the flag which indicates whether the DDNS principal track is enabled or disabled.

DDNS principal tracking includes putting principal name into resource record affected by update.

Setting this parameter to a defined value, implicitly sets the override_ddns_principal_security attribute to "true". Setting this parameter to undefined causes the appliance to use the Grid-level DDNS principal security setting and automatically sets the override_ddns_principal_security attribute to "false" if "ddns_restrict_secure" and "ddns_principal_group" are also undefined. But if "ddns_restrict_secure" or "ddns_principal_group" contains a value, then the override_ddns_principal_security attribute is set to "true".

Note that when ddns_principal_tracking contains a value and override_ddns_principal_security is set to "false", the last operation takes precedence. Thus the sequence $firstview->ddns_principal_tracking("true"); $firstview->override_ddns_principal_security("false"); sets override_ddns_principal_security to "false", and the sequence $firstview->override_ddns_principal_security("false"); $firstview->ddns_principal_tracking("true"); sets override_ddns_principal_security to "true".

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

Specify 'true' to enable DDNS principal tracking and 'false' to disable it. The default value is 'false'.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#get ddns_principal_tracking value
$ddns_principal_tracking = $firstview->ddns_principal_tracking();

#modify ddns_principal_tracking value
```

```
$firstview->ddns_principal_tracking('true');
```

ddns_principal_group( )

Use this method to set or retrieve the DDNS principal group.

Setting this parameter to a defined value, implicitly sets the override_ddns_principal_security attribute to "true". Setting this parameter to undefined causes the appliance to use the Grid-level DDNS principal security setting and automatically sets the override_ddns_principal_security attribute to "false" if "ddns_restrict_secure" and "ddns_principal_tracking" are also undefined. But if "ddns_restrict_secure" or "ddns_principal_tracking" contains a value, then the override_ddns_principal_security attribute is set to "true".

Note that when ddns_principal_group contains a value and override_ddns_principal_security is set to "false", the last operation takes precedence. Thus the sequence $firstview->ddns_principal_group($principal_group); $firstview->override_ddns_principal_security("false"); sets override_ddns_principal_security to "false", and the sequence $firstview->override_ddns_principal_security("false"); $firstview->ddns_principal_group($principal_group); sets override_ddns_principal_security to "true".

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

The valid value is an Infoblox::DNS::DDNS::PrincipalCluster::Group object.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#get ddns_principal_group value
$ddns_principal_group = $firstview->ddns_principal_group();

#modify ddns_principal_group value
$firstview->ddns_principal_group($principal_group);
```

disable( )

Use this method to set or retrieve the disable flag of a DNS view object.

Include the specific parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

The default value for this field is false; therefore, the DNS view is enabled.

Parameter

Specify "true" to set the disable flag or "false" to deactivate/unset it.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

Example

```
#Get disable
my $disable = $firstview->disable();
#Modify disable
$firstview->disable("true");
```

dns64_groups( )

Use this method to set or retrieve the list of DNS64 synthesis groups associated with this DNS view.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Setting this method to a defined value implicitly sets the override_dns64 method to "true". Setting the parameter to undefined causes the appliance to use the grid DNS default and automatically resets the override_dns64 method to "false".

Note that when dns64_groups is set to a defined value and override_dns64_groups is set to "false", the last operation takes precedence. Thus the sequence $object->dns64_groups([$group1]); $object->override_dns64("false"); will set override_dns64 to "false", and the sequence $object->override_dns64([$group1]); $object->dns64_groups("testdomain"); will result in override_dns64="true".

Parameter

Valid value is an array reference that contains Infoblox::Grid::DNS::DNS64Group objects.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get the list of DNS64 synthesis groups
my $dns64_groups_list = $firstview->dns64_groups();

#Modify it
my $group = $session->get(
                        object => "Infoblox::Grid::DNS::DNS64Group"),
                        name => "groupname",
                        prefix => "groupprefix",
                      );
$firstview->dns64_groups([$group]);
```

dnssec_enabled( )

Use this method to enable/disable the DNS security extension.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

Set the parameter to "true" to enable the DNS security extension. Set it to "false" to disable the feature.

Setting this parameter to "true"/"false" implicitly sets the override_dnssec attribute to "true". Setting the parameter to undefined causes the appliance to use the member-level DNS security settings and automatically resets the override_dnssec attribute to "false".

Note that in the case when dnssec_enabled is set to "true"/"false" and override_dnssec is set to "false" simultaneously, the last operation takes precedence. Thus the sequence $firstview->dnssec_enabled("false"); $firstview->override_dnssec("false"); will set override_dnssec to "false", and the sequence $firstview->override_dnssec("false"); $firstview->dnssec_enabled("false"); will result in override_dnssec="true".

The default value of the parameter is undefined, which means the DNS view inherits the member-level settings.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get dnssec_enabled
my $dnssec_enabled = $firstview->dnssec_enabled();
#Modify dnssec_enabled
$firstview->dnssec_enabled("true");
#Set the value to undefined to inherit the member-level settings.
$firstview->dnssec_enabled(undef);
```

dnssec_expired_signatures_enabled( )

Use this method to control whether DNS accepts expired signatures.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

Set the parameter to "true" to accept expired signatures. Set it to "false" to reject expired signatures.

The default value of the parameter is "false".

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get dnssec_expired_signatures_enabled
my $dnssec_expired_signatures_enabled = $firstview->dnssec_expired_signatures_enabled();
#Modify dnssec_expired_signatures_enabled
$firstview->dnssec_expired_signatures_enabled("true");
```

dnssec_validation_enabled( )

Use this method to enable/disable DNS security validation.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

Set the parameter to "true" to enable the validation. Set the parameter to "false" to disable the validation.

The default value of the parameter is "true".

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get dnssec_validation_enabled
my $dnssec_validation_enabled = $firstview->dnssec_validation_enabled();
#Modify dnssec_validation_enabled
$firstview->dnssec_validation_enabled("false");
```

dnssec_blacklist_enabled()

Use this method to enable and disable blacklist rules for DNSSEC-enabled clients.

Parameter

Set the parameter to "true" to apply blacklist rules for DNSSEC-enabled clients. Set to "false" to disable them.

The default value of the parameter is "false".

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get dnssec_blacklist_enabled
my $dnssec_blacklist_enabled = $firstview->dnssec_blacklist_enabled();
#Modify dnssec_blacklist_enabled
$firstview->dnssec_blacklist_enabled("true");
```

dnssec_negative_trust_anchors( )

Use this method to set or retrieve the list of zone for which the appliance does not perform DNSSEC validation.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

The valid value is an array of strings that contains the list of zone for which the appliance does not perform DNSSEC validation.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get dnssec_negative_trust_anchors value
my $value = $view->dnssec_negative_trust_anchors();
#Modify dnssec_negative_trust_anchors value
$view->dnssec_negative_trust_anchors($zones);
```

dnssec_dns64_enabled()

Use this method to enable and disable DNS64 groups for DNSSEC-enabled clients.

Parameter

Set the parameter to "true" to apply DNS64 groups for DNSSEC-enabled clients. Set to "false" to disable them.

The default value of the parameter is "false".

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get dnssec_dns64_enabled
my $dnssec_dns64_enabled = $grid_dns->dnssec_dns64_enabled();
#Modify dnssec_dns64_enabled
$grid_dns->dnssec_dns64_enabled("true");
```

dnssec_nxdomain_enabled()

Use this method to enable and disable NXDOMAIN rules for DNSSEC-enabled clients.

Parameter

Set the parameter to "true" to apply NXDOMAIN rules for DNSSEC-enabled clients. Set to "false" to disable them.

The default value of the parameter is "false".

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get dnssec_nxdomain_enabled
my $dnssec_nxdomain_enabled = $firstview->dnssec_nxdomain_enabled();
#Modify dnssec_nxdomain_enabled
$firstview->dnssec_nxdomain_enabled("true");
```

dnssec_rpz_enabled()

Use this method to enable and disable RPZ policies for DNSSEC-enabled clients.

Parameter

Set the parameter to "true" to apply RPZ policies for DNSSEC-enabled clients. Set to "false" to disable them.

The default value of the parameter is "false".

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get dnssec_rpz_enabled
my $dnssec_rpz_enabled = $firstview->dnssec_rpz_enabled();
#Modify dnssec_rpz_enabled
$firstview->dnssec_rpz_enabled("true");
```

dnssec_trusted_keys( )

Use this method to specify the list of trusted keys for the DNS security extension.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

Array reference that contains a list of Infoblox::DNS::DnssecTrustedKey objects.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns a reference to a list of the trusted keys.

Example

```
#Get dnssec_trusted_keys
```

```
my $dnssec_trusted_keys = $firstview->dnssec_trusted_keys();
#Modify dnssec_trusted_keys
my $key1=Infoblox::DNS::DnssecTrustedKey->new(
   fqdn => "myzone.com",
   algorithm => "RSAMD5",
   key       => "c2ltcGxlIGtleQo="
   );
$firstview->dnssec_trusted_keys([$key1]);
```

enable_blacklist( )

Use this method to enable or disable a blacklist in a DNS view.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

Specify "true" to set the enable_blacklist flag or "false" to deactivate/unset it. The default is "false".

Setting this parameter to "true" or "false" implicitly sets the override_blacklist attribute to "true". Setting the parameter to undefined causes the appliance to use the member-level blacklist settings and automatically sets the override_blacklist attribute to "false" if "blacklist_redirect_addresses" is also undefined.

Note that when enable_blacklist is set to "true"/"false" and override_blacklist is set to "false", the last operation takes precedence. Thus the sequence $firstview->enable_blacklist("false"); $firstview->override_blacklist("false"); sets override_blacklist to "false", and the sequence $firstview->override_blacklist("false"); $firstview->enable_blacklist("false"); sets override_blacklist to "true".

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Getting enable_blacklist
my $enable_blacklist = $firstview->enable_blacklist( );
#Modifying enable_blacklist
$firstview->enable_blacklist("true");
```

enable_dns64( )

Use this method to enable/disable DNS64 support for this view.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Setting this method to a defined value implicitly sets the override_dns64 method to "true". Setting the parameter to undefined causes the appliance to use the grid DNS default and automatically resets the override_dns64 method to "false".

Note that when enable_dns64 is set to a defined value and override_enable_dns64 is set to "false", the last operation takes precedence. Thus the sequence $object->enable_dns64("true"); $object->override_dns64("false"); will set override_dns64 to "false", and the sequence $object->override_dns64("true"); $object->enable_dns64("testdomain"); will result in override_dns64="true".

Parameter

Specify "true" to set the enable_dns64 flag or "false" to deactivate and unset it. The default value is "false".

Returns

When you specify a parameter and the modification succeeds, the method returns "true". When the operation fails, the method returns "false".

When you do not specify a parameter, the method returns the attribute value.

Example

```
#Get the enable_dns64 flag
my $enable_dns64 = $firstview->enable_dns64();
#Modify the enable_dns64 flag
$firstview->enable_dns64("true");
```

enable_match_recursive_only( )

Use this method to enable or disable the match-recursive-only setting for a DNS View on the member.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

Specify 'true' to enable 'match-recursive-only' setting or 'false' to disable it.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#get enable_match_recursive_only value
my $enable_match_recursive_only = $firstview->enable_match_recursive_only();

#modify enable_match_recursive_only value
$firstview->enable_match_recursive_only('true');
```

enable_fixed_rrset_order_fqdns( )

Use this method to enable or disable fixed RRset order for specified FQDNs.

Setting this method to a defined value implicitly sets the override_fixed_rrset_order_fqdns method to "true".

Note that when enable_fixed_rrset_order_fqdns is set to a defined value and override_fixed_rrset_order_fqdns is set to "false", the last operation takes precedence. Thus the sequence $object->enable_fixed_rrset_order_fqdns("true"); $object->override_fixed_rrset_order_fqdns("false"); will set override_fixed_rrset_order_fqdns to "false", and the sequence $object->override_fixed_rrset_order_fqdns("false"); $object->enable_fixed_rrset_order_fqdns("true"); will result in override_fixed_rrset_order_fqdns="true".

The enable_fixed_rrset_order_fqdns attribute can be specified explicitly. It is also set implicitly when fixed_rrset_order_fqdns is set to a defined value.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

Specify "true" to enable fixed RRset order for specified FQDNs or "false" to disable it.

Returns

When you specify a parameter and the modification succeeds, the method returns "true". When the operation fails, the method returns "false".

When you do not specify a parameter, the method returns the attribute value.

Example

```
#Get enable_fixed_rrset_order_fqdns flag
my $enable_fixed_rrset_order_fqdns = $firstview->enable_fixed_rrset_order_fqdns();
#Modify enable_fixed_rrset_order_fqdns flag
$firstview->enable_fixed_rrset_order_fqdns("true");
```

extattrs( )

Use this method to set or retrieve the extensible attributes associated with a DNS view object.

Parameter

Valid value is a hash reference containing the names of extensible attributes and their associated values (Infoblox::Grid::Extattr objects).

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get extattrs
my $ref_extattrs = $firstview->extattrs();
#Modify extattrs
$firstview->extattrs({ 'Site' => $extattr1, 'Administrator' => $extattr2 });
```

extensible_attributes( )

Use this method to set or retrieve the extensible attributes associated with a DNS view object.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

For valid values for extensible attributes, see Infoblox::Grid::ExtensibleAttributeDef/Extensible Attribute Values.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get extensible attributes
my $ref_extensible_attributes = $firstview->extensible_attributes();
#Modify extensible attributes
$firstview->extensible_attributes({ 'Site' => 'Santa Clara', 'Administrator' => [ 'Peter', 'Tom' ]
});
```

filter_aaaa( )

Use this method to set or retrieve the type of AAAA filtering for this DNS view object.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Specifying values in this parameter, such as "YES", implicitly sets the override_filter_aaaa attribute to "true". Setting this parameter and the "filter_aaaa" attribute to undefined causes the appliance to use the grid-level filter_aaaa settings and automatically sets the override_filter_aaaa attribute to "false". But if the "filter_aaaa" attribute contains a value, then the override_filter_aaaa attribute is set to "true".

Note that when filter_aaaa contains a valid value and override_filter_aaaa is set to "false", the last operation takes precedence. Thus the sequence $firstview->filter_aaaa("YES"); $firstview->override_filter_aaaa("false"); sets override_filter_aaaa to "false", and the sequence $firstview->override_filter_aaaa("false"); $firstview->filter_aaaa("true"); sets override_filter_aaaa to "true".

Parameter

Valid values are "YES", "NO" and "BREAK_DNSSEC". The default value is "NO".

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get the filter aaaa type
my $filter_aaaa = $firstview->filter_aaaa();
#Modify the filtering
$firstview->filter_aaaa("YES");
```

filter_aaaa_list( )

Use this method to set or retrieve the list of IPv4 addresses and networks from which queries are received. AAAA filtering is applied to these addresses. This setting overrides the grid filter_aaaa_list if override_filter_aaaa is set to "true".

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Specifying values in this parameter, such as ["10.0.0.10", "10.0.0.20"] or [], implicitly sets the override_filter_aaaa attribute to "true". Setting this parameter and the "filter_aaaa" attribute to undefined causes the appliance to use the grid-level filter_aaaa settings and automatically sets the override_filter_aaaa attribute to "false". But if the "filter_aaaa" attribute contains a value, then the override_filter_aaaa attribute is set to "true".

Note that when filter_aaaa_list contains a valid array reference and override_filter_aaaa is set to "false", the last operation takes precedence. Thus the sequence $firstview->filter_aaaa_list(["10.0.0.10", "10.0.0.20"]); $firstview->override_filter_aaaa("false"); sets override_filter_aaaa to "false", and the sequence $firstview->override_filter_aaaa("false"); $firstview->filter_aaaa_list(["10.0.0.10", "10.0.0.20"]); sets override_filter_aaaa to "true".

Parameter

A valid value is an Infoblox::Grid::NamedACL object or an array reference that contains IPv4 IP addresses and networks. To return AAAA records for queries received from an IPv4 client, add "!" as the prefix to the IP addresses and networks. Specify "any" to allow AAAA records to be returned for any query received by an IPv4 client.

Returns

When you specify a parameter and the modification succeeds, the method returns "true". When the operation fails, the method returns "false".

When you do not specify a parameter, the method returns the attribute value.

Example

```
#Getting filter_aaaa_list
my $filter_aaaa_list = $firstview->filter_aaaa_list( );
#Modifying filter_aaaa_list
$firstview->filter_aaaa_list(["any"]);
#Modifying filter_aaaa_list
$firstview->filter_aaaa_list(["10.0.0.10", "20.0.0.0/24", "!"."30.0.0.30", "!"."40.10.0.0/24"]);
#Use Grid::DNS level filter_aaaa_list
$firstview->filter_aaaa_list(undef);
$firstview->enable_filter_aaaa(undef);
#Use Infoblox::Grid::NamedACL object
$firstview->filter_aaaa_list($nacl);
```

fixed_rrset_order_fqdns( )

Use this method to set or retrieve configuration for fixed RRSET order for specified FQDNs.

Setting this method to a defined value implicitly sets the enable_fixed_rrset_order_fqdns and override_fixed_rrset_order_fqdns methods to "true". Setting the parameter to undefined causes the appliance to disable fixed RRset order of FQDNs and automatically resets the enable_fixed_rrset_order_fqdns method to "false".

Note that when fixed_rrset_order_fqdns is set to a defined value and enable_fixed_rrset_order_fqdns is set to "false", the last operation takes precedence. Thus the sequence $object->fixed_rrset_order_fqdns($fqdns); $object->enable_fixed_rrset_order_fqdns("false"); will set enable_fixed_rrset_order_fqdns to "false", and the sequence $object->enable_fixed_rrset_order_fqdns("false"); $object->fixed_rrset_order_fqdns($fqdns); will result in enable_fixed_rrset_order_fqdns="true".

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

The valid value is an array reference that contains Infoblox::Grid::DNS::FixedRRSetOrderFQDN objects.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get the fixed_rrset_order_fqdns list
my $fixed_rrset_order_fqdns = $firstview->fixed_rrset_order_fqdns();
#Modify the fixed_rrset_order_fqdns list
$firstview->fixed_rrset_order_fqdns([$fqdn1, $fqdn2]);
```

forwarders( )

Use this method to specify forwarders for the DNS view. A forwarder is essentially a name server to which other name servers first send all of their off-site queries. The forwarder builds up a cache of information, avoiding the need for the other name servers to send queries off-site. This setting overrides the grid-level setting.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

Valid value is an array reference that contains Infoblox::DNS::Member, Infoblox::DNS::Nameserver object(s), or IP addresses.

Specifying values in this parameter, for example ["10.0.0.10", "10.0.0.20"] or [], implicitly sets the override_forwarders attribute to "true". Setting this parameter and the "forward_only" attribute to undefined causes the appliance to use the member-level forwarders setting and automatically sets the override_forwarders attribute to "false". But if the "forward_only" attribute contains a value, then the override_forwarders attribute is set to "true".

Note that when forwarders_redirect_addresses contains a valid array reference and override_forwarders is set to "false", the last operation takes precedence. Thus the sequence $firstview->forwarders(["10.0.0.10", "10.0.0.20"]); $firstview->override_forwarders("false"); sets override_forwarders to "false", and the sequence $firstview->override_forwarders("false"); $firstview->forwarders(["10.0.0.10", "10.0.0.20"]); sets override_forwarders to "true".

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Getting forwarders
```

```
$firstview->forwarders( );
#Modifying forwarders
$firstview->forwarders( [ "10.0.0.10" ] );
```

forward_only( )

Use this method to configure this view to send queries to forwarders only. When the value is "true", the queries are sent to forwarders only, and not to other internal or Internet root servers.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

Specify "true" to use forwarders only or "false" to use forwarders and root servers. The default is "false"

Setting this parameter to "true" or "false" implicitly sets the override_forwarders attribute to "true". Setting the parameter to undefined causes the appliance to use the member-level forwarders settings and automatically sets the override_forwarders attribute to "false" if "forwarders" is also undefined.

Note that when forward_only is set to "true"/"false" and override_forwarders is set to "false", the last operation takes precedence. Thus the sequence $firstview->forward_only("false"); $firstview->override_forwarders("false"); sets override_forwarders to "false", and the sequence $firstview->override_forwarders("false"); $firstview->forward_only("false"); sets override_forwarders to "true".

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get forward_only
$firstview->forward_only( );
#Modify forward_only
$firstview->forward_only("true");
```

match_clients( )

Use this method to set or retrieve the match_clients list. This list specifies IP addresses, networks and/or TSIG keys of the clients that are allowed or denied access to the DNS view.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

Valid value is an Infoblox::Grid::NamedACL object or an array reference that contains IPv4/IPv6 addresses or networks. To deny access to IP addresses and/or networks, add "!" as a prefix to IP addresses and/or networks. Specify "any" to allow access to any client. To allow access to a client authorized by a TSIG key, use Infoblox::DNS::TSIGKey objects.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

Example

```
#Get match_clients
my $clients = $firstview->match_clients();
#Modify match_clients
$firstview->match_clients(["any"]);

#Specifying TSIG keys
my $tsig_key3 = $session->gen_tsig_key();
my $key3 = Infoblox::DNS::TSIGKey->new(
    name => "key3",
    key  => $tsig_key3
);
#Modify match_clients
$firstview->match_clients(["10.0.0.10" , $key3, "20.0.0.0/24"
,"2001::123a","2003::222/64","!"."30.0.0.30" ,
"!"."40.10.0.0/24","!"."2004::125b","!"."2006::34bc"]);
#Use Infoblox::Grid::NamedACL
$firstview->match_clients($nacl);
```

match_destinations( )

Use this method to set or retrieve the match_destinations list. This list specifies IP addresses, networks and/or TSIG keys of the clients that are allowed or denied access to the DNS view.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

Valid value is an Infoblox::Grid::NamedACL object or an array reference that contains IPv4/IPv6 addresses or networks. To deny access to IP addresses and/or networks, add "!" as a prefix to IP addresses and/or networks. Specify "any" to allow access to any client. To allow access to a client authorized by a TSIG key, use Infoblox::DNS::TSIGKey objects.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

Example

```
#Get match_destinations
my $clients = $firstview->match_destinations();
#Modify match_destinations
$firstview->match_destinations(["any"]);

#Specifying TSIG keys
my $tsig_key3 = $session->gen_tsig_key();
my $key3 = Infoblox::DNS::TSIGKey->new(
    name => "key3",
    key  => $tsig_key3
);
#Modify match_destinations
$firstview->match_destinations(["10.0.0.10" , $key3, "20.0.0.0/24"
,"2001::123a","2003::222/64","!"."30.0.0.30" ,
"!"."40.10.0.0/24","!"."2004::125b","!"."2006::34bc"]);
#Use Infoblox::Grid::NamedACL
$firstview->match_destinations($nacl);
```

max_cache_ttl( )

Use this method to set or retrieve the maximum time (in seconds) for which the server will cache positive answers.

Parameter

The maximum time is specified in seconds. The default value is 604800.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get max_cache_ttl
my $max_cache_ttl = $firstview->max_cache_ttl();
#Modify max_cache_ttl
$firstview->max_cache_ttl(400);
```

max_ncache_ttl( )

Use this method to set or retrieve the maximum time (in seconds) for which the server will cache negative (NXDOMAIN) responses.

Parameter

The maximum time is specified in seconds. The default value is 10800. The maximum allowed value is 604800.

Setting this parameter implicitle sets the override_max_ncache_ttl attribute to "true". Setting the parameter to undefined causes the appliance to use the Grid-level settings and automatically sets the override_max_ncache_ttl attribute to "false".

Note that when max_ncache_ttl is set and override_max_ncache_ttl is set to "false", the last operation takes precedence. Thus the sequence $firstview->max_ncache_ttl(10800); $firstview->override_max_ncache_ttl("false"); sets override_max_ncache_ttl to "false", and the sequence $firstview->override_max_ncache_ttl("false"); $firstview->max_ncache_ttl(30600); sets override_max_ncache_ttl to "true".

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get max_ncache_ttl
```

```
my $max_ncache_ttl = $firstview->max_ncache_ttl();
#Modify max_ncache_ttl
$firstview->max_ncache_ttl(400);
```

lame_ttl( )

Use this method to set or retrieve the number of seconds to cache lame delegations or lame servers.

Parameter

The number of seconds to cache lame delegations or lame servers. The default value is 600.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get lame_ttl
my $lame_ttl = $firstview->lame_ttl();
#Modify lame_ttl
$firstview->lame_ttl(700);
```

name( )

Use this method to configure or retrieve the name of a DNS view.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

Name of the DNS view in string format. It has a maximum of 64 bytes.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

Example

```
#Get name
my $name = $firstview->name();
#Modify name
$firstview->name("external");
```

network_view( )

Use this method to set or retrieve the network view object associated with this DNS view object.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

The value should be a reference of defined Infoblox::DHCP::View objects.

Returns

If you specify a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you do not specify a parameter, the method returns the attribute value.

Example

```
#Get network_view
my $useRootNS = $firstview->network_view( );
#Modify network_view
$firstview->use_root_name_servers($network_view);
```

notify_delay( )

Use this method to specify with how many seconds of delay the notify messages are sent to secondaries.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

Valid values are unsigned integer between 5 and 86400, inclusive. Default value is undefined.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Getting notify_delay
my $notify_delay = $firstview->notify_delay( );
#Modifying notify_delay
$firstview->notify_delay(5);
```

override_blacklist( )

The "override_blacklist" attribute controls whether the "enable_blacklist", "blacklist_redirect_addresses", "blacklist_redirect_ttl", "blacklist_log_query", and "blacklist_rulesets" values in the database are used.

The "override_blacklist" attribute can be specified explicitly. It is also set implicitly as follows: "enable_blacklist" => "true"|"false" or "blacklist_redirect_addresses" => ["10.0.0.10", "10.0.0.20"]|[] results in "override_blacklist" => "true", "enable_blacklist" => undef and "blacklist_redirect_addresses" => undef results in "override_blacklist" => "false". If "override_blacklist" is specified explicitly as "true", and "enable_blacklist" => undef or "blacklist_redirect_addresses" => undef, then the explicit value takes precedence. Therefore, "override_blacklist"=>"true", "enable_blacklist"=>undef, and "blacklist_redirect_addresses"=>undef remain unchanged in the database.

Specifying the "blacklist_redirect_ttl", "blacklist_log_query", or "blacklist_rulesets" attributes does not affect the "override_blacklist" value. It is implicitly set only by "enable_blacklist" and "blacklist_redirect_addresses".

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

Set the parameter to "true" to override the member-level settings of the blacklist redirect attributes. Set the parameter to "false" to inherit the member-level settings. The default is "false".

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get override_blacklist
my $override_blacklist=$firstview->override_blacklist( );
#Modify override_blacklist
$firstview->override_blacklist("true");
```

override_ddns_force_creation_timestamp_update( )

The override_ddns_force_creation_timestamp_update attribute controls whether the ddns_force_creation_timestamp_update method value of the DNS View is used, instead of the Grid default.

The override_ddns_force_creation_timestamp_update attribute can be specified explicitly. It is also set implicitly when ddns_force_creation_timestamp_update is set to a defined value.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

Set the parameter to "true" to override the Grid-level setting for ddns_force_creation_timestamp_update. Set the parameter to "false" to inherit the Grid-level setting for ddns_force_creation_timestamp_update.

The default value of this parameter is "false".

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get override_ddns_force_creation_timestamp_update value
my $value = $object->override_ddns_force_creation_timestamp_update();

#Modify override_ddns_force_creation_timestamp_update value
$object->override_ddns_force_creation_timestamp_update("true");
```

override_ddns_patterns_restriction( )

The override_ddns_patterns_restriction attribute controls whether the ddns_restrict_patterns_list and ddns_restrict_patterns methods values of the DNS View are used, instead of the Grid default.

The override_ddns_patterns_restriction attribute can be specified explicitly. It is also set implicitly when ddns_restrict_patterns_list or ddns_restrict_patterns is set to a defined value.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

Set the parameter to "true" to override the Grid-level setting for ddns_restrict_patterns_list and ddns_restrict_patterns. Set the parameter to "false" to inherit the Grid-level setting for ddns_restrict_patterns_list and ddns_restrict_patterns.

The default value of this parameter is "false".

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get override_ddns_patterns_restriction
my $override_ddns_patterns_restriction = $firstview->override_ddns_patterns_restriction();
#Modify override_ddns_patterns_restriction
$firstview->override_ddns_patterns_restriction("true");
```

override_ddns_restrict_static( )

The override_ddns_restrict_static attribute controls whether the ddns_restrict_static method value of the DNS View is used, instead of the Grid default.

The override_ddns_restrict_static attribute can be specified explicitly. It is also set implicitly when ddns_restrict_static is set to a defined value.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

Set the parameter to "true" to override the Grid-level setting for ddns_restrict_static. Set the parameter to "false" to inherit the Grid-level setting for ddns_restrict_static.

The default value of this parameter is "false".

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get override_ddns_restrict_static
my $override_ddns_restrict_static = $firstview->override_ddns_restrict_static();
#Modify override_ddns_restrict_static
$firstview->override_ddns_restrict_static("true");
```

override_ddns_restrict_protected( )

The override_ddns_restrict_protected attribute controls whether the ddns_restrict_protected method value of the DNS View is used, instead of the Grid default.

The override_ddns_restrict_protected attribute can be specified explicitly. It is also set implicitly when ddns_restrict_protected is set to a defined value.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

Set the parameter to "true" to override the Grid-level setting for ddns_restrict_protected. Set the parameter to "false" to inherit the Grid-level setting for ddns_restrict_protected.

The default value of this parameter is "false".

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get override_ddns_restrict_protected
my $override_ddns_restrict_protected = $firstview->override_ddns_restrict_protected();
#Modify override_ddns_restrict_protected
$firstview->override_ddns_restrict_protected("true");
```

override_ddns_principal_security( )

The override_ddns_principal_security attribute controls whether the ddns_restrict_secure, ddns_principal_tracking and ddns_principal_group methods values of the DNS View are used, instead of the Grid default.

The override_ddns_principal_security attribute can be specified explicitly. It is also set implicitly when ddns_restrict_secure, ddns_principal_tracking or ddns_principal_group is set to a defined value.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

Set the parameter to "true" to override the Grid-level setting for ddns_restrict_secure, ddns_principal_tracking and ddns_principal_group. Set the parameter to "false" to inherit the Grid-level setting for ddns_restrict_secure, ddns_principal_tracking and ddns_principal_group.

The default value of this parameter is "false".

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get override_ddns_principal_security
my $override_ddns_principal_security = $firstview->override_ddns_principal_security();
#Modify override_ddns_principal_security
$firstview->override_ddns_principal_security("true");
```

override_dns64( )

The override_dns64 attribute controls whether the DNS64 method values in this DNS view are used, instead of the grid default.

The override_dns64 attribute can be specified explicitly. It is also set implicitly when enable_dns64 or dns64_groups are set to a defined value.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

Set the parameter to "true" to override the grid-level setting for enable_dns64 and dns64_groups. Set the parameter to "false" to inherit the grid-level setting for enable_dns64 and dns64_groups.

The default value of this parameter is "false".

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Getting override_dns64
my $override_dns64=$firstview->override_dns64( );
#Modifying override_dns64
$firstview->override_dns64("true");
```

override_dnssec( )

The override_dnssec attribute controls whether the dnssec_enabled, dnssec_validation_enabled, dnssec_expired_signatures_enabled, and dnssec_trusted_keys values in the database are used.

The override_dnssec attribute can be specified explicitly. It is also established implicitly as follows: "dnssec_enabled" => "true"|"false" results in "override_dnssec" => "true" "dnssec_enabled" => undef results in "override_dnssec" => "false" If the override_dnssec attribute

is specified explicitly as "true", while dnssec_enabled=>undef, the explicit value takes precedence, and override_dnssec=>"true" while dnssec_enabled remains unchanged in the DB.

Supplying the dnssec_validation_enabled, dnssec_expired_signatures_enabled, or dnssec_trusted_keys attributes does not affect the override_dnssec value. It is implicitly affected only by dnssec_enabled.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

Set the parameter to "true" to override the member-level setting of the DNSSEC attributes. Set the parameter to "false" to inherit the grid-member level settings of DNSSec.

The default value of this parameter is "false".

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Getting override_dnssec
my $override_dnssec=$firstview->override_dnssec( );
#Modifying override_dnssec
$firstview->override_dnssec("true");
```

override_filter_aaaa( )

The override_filter_aaaa attribute controls whether the AAAA filter method values of the view are used, instead of the grid default.

The override_filter_aaaa attribute can be specified explicitly. It is also set implicitly when filter_aaaa or filter_aaaa_list are set to a defined value.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

Set the parameter to "true" to override the grid-level setting for filter_aaaa and filter_aaaa_list. Set the parameter to "false" to inherit the grid-level setting for filter_aaaa and filter_aaaa_list.

The default value of this parameter is "false".

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Getting override_filter_aaaa
my $override_filter_aaaa=$firstview->override_filter_aaaa( );
#Modifying override_filter_aaaa
$firstview->override_filter_aaaa("true");
```

override_fixed_rrset_order_fqdns( )

The override_fixed_rrset_order_fqdns controls whether the enable_fixed_rrset_order_fqdns and fixed_rrset_order_fqdns values in the database are used.

The override_fixed_rrset_order_fqdns can be specified explicitly. It is also set implicitly as follows: enable_fixed_rrset_order_fqdns => "true" | "false" or fixed_rrset_order_fqdns => $fqdns results in override_fixed_rrset_order_fqdns => "true";

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

Set the parameter to "true" to override the Grid-leval fixed RRset order settings. Set the parameter to "false" to inherit the Grid-level settings. The default is "false".

Returns

When you specify a parameter and the modification succeeds, the method returns "true". When the operation fails, the method returns "false".

When you do not specify a parameter, the method returns the attribute value.

Example

```
#Get override_fixed_rrset_order_fqdns flag
my $override_fixed_rrset_order_fqdns = $firstview->override_fixed_rrset_order_fqdns();
#Modify override_fixed_rrset_order_fqdns flag
$firstview->override_fixed_rrset_order_fqdns("true");
```

override_forwarders( )

The "override_forwarders" attribute controls whether the "forward_only" and "forwarders" values in the database are used.

The "override_forwarders" attribute can be specified explicitly. It is also set implicitly as follows: "forward_only" => "true"|"false" or "forwarders" => ["10.0.0.10", "10.0.0.20"]|[] results in "override_forwarders" => "true"; "forward_only" => undef and "forwarders" => undef result in "override_forwarders" => "false". If "override_forwarders" is specified explicitly as "true", and "forward_only" => undef or "forwarders" => undef, then the explicit value takes precedence. Therefore, "override_forwarders"=>"true", "forward_only"=>undef, and "forwarders"=>undef remain unchanged in the database.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

Set the parameter to "true" to override the member-level settings of the forwarders attributes. Set the parameter to "false" to inherit the member-level settings. The default is "false".

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get override_forwarders
my $override_forwarders=$firstview->override_forwarders( );
#Modify override_forwarders
$firstview->override_forwarders("true");
```

override_lame_ttl( )

Use this method to set or retrieve the override_lame_ttl attribute that controls whether the lame_ttl method values of the view are used, instead of the Grid default

Parameter

Set the parameter to "true" to override the grid-level setting for lame_ttl. Set the parameter to "false" to inherit the grid-level setting for lame_ttl. The default value is 'false'.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get override_lame_ttl
my $override_lame_ttl = $firstview->override_lame_ttl();
#Modify override_lame_ttl
$firstview->override_lame_ttl('true');
```

override_max_cache_ttl( )

The override_max_cache_ttl attribute controls whether the max_cache_ttl method values of the member are used, instead of the grid default.

The override_max_cache_ttl attribute can be specified explicitly. It is also set implicitly when max_cache_ttl is set to a defined value.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

Set the parameter to "true" to override the grid-level setting for max_cache_ttl. Set the parameter to "false" to inherit the grid-level setting for max_cache_ttl.

The default value of this parameter is "false".

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Getting override_max_cache_ttl
my $override_max_cache_ttl = $firstview->override_max_cache_ttl( );
#Modifying override_max_cache_ttl
$firstview->override_max_cache_ttl("true");
```

override_max_ncache_ttl( )

The override_max_ncache_ttl attribute controls whether the max_ncache_ttl method values of the member are used, instead of the Grid default.

The override_max_ncache_ttl attribute can be specified explicitly. It is also set implicitly when max_ncache_ttl is set to a defined value.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

Set the parameter to "true" to override the grid-level setting for max_ncache_ttl. Set the parameter to "false" to inherit the Grid-level setting for max_ncache_ttl.

The default value of this parameter is "false".

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Getting override_max_ncache_ttl
my $override_max_ncache_ttl = $firstview->override_max_ncache_ttl( );
#Modifying override_max_ncache_ttl
$firstview->override_max_ncache_ttl("true");
```

scavenging_settings( )

Use this method to set or retrieve the view-level scavenging settings.

Setting this parameter implicitle sets the override_scavenging_settings attribute to "true". Setting the parameter to undefined causes the appliance to use the Grid-level settings and automatically sets the override_scavenging_settings attribute to "false".

Note that when scavenging_settings is set and override_scavenging_settings is set to "false", the last operation takes precedence. Thus the sequence $object->scavenging_settings($recl_settings); $object->override_scavenging_settings("false"); sets override_scavenging_settings to "false", and the sequence $object->override_scavenging_settings("false"); $object->scavenging_settings($recl_settings); sets override_scavenging_settings to "true".

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

The valid value is an Infoblox::Grid::DNS::ScavengingSetting object.

Parameter

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get scavenging_settings value
my $scavenging_settings = $object->scavenging_settings();

#Modify scavenging_settings value
$object->scavenging_settings($scavenging_settings);
```

override_scavenging_settings( )

The override_scavenging_settings attribute controls whether the scavenging_settings method values of the DNS View are used, instead of the Grid default.

The override_scavenging_settings attribute can be specified explicitly. It is also set implicitly when scavenging_settings is set to a defined

value.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

Set the parameter to "true" to override the Grid-level setting for scavenging_settings. Set the parameter to "false" to inherit the Grid-level setting for scavenging_settings.

The default value of this parameter is "false".

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get override_scavenging_settings value
my $override_scavenging_settings = $object->override_scavenging_settings( );

#Modify override_scavenging_settings value
$object->override_scavenging_settings("true");
```

recursion( )

Use this method to enable/disable recursion.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

Specify "true" to set the recursion flag or "false" to deactivate/unset it.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

Example

```
#Get recursion
my $recursion = $firstview->recursion();
#Modify recursion
$firstview->recursion("false");
```

rpz_drop_ip_rule_enabled( )

Use this method to set or retreive the flag that indicates whether the DNS server ignores RPZ-IP triggers with large prefix lengths or not.

Setting this method to a defined value implicitly sets the override_rpz_drop_ip_rule method to "true". Setting the parameter to undefined if rpz_drop_ip_rule_min_prefix_length_ipv4 and rpz_drop_ip_rule_min_prefix_length_ipv6 are undefined causes the appliance to use the Grid default and automatically resets the override_rpz_drop_ip_rule attribute to "false".

Note that when rpz_drop_ip_rule_enabled is set to a defined value and override_rpz_drop_ip_rule is set to "false", the last operation takes precedence. Thus the sequence $object->rpz_drop_ip_rule_enabled("true"); $object->override_rpz_drop_ip_rule("false"); will set override_rpz_drop_ip_rule to "false", and the sequence $object->override_rpz_drop_ip_rule("false"); $object->rpz_drop_ip_rule_enabled("true"); will result in override_rpz_drop_ip_rule="true".

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

Specify 'true' to enable ignoring of RPZ-IP triggers with large prefix lengths or 'false' to disable it. The default value is 'false'.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
# Get attribute value
my $value = $firstview->rpz_drop_ip_rule_enabled();
# Modify attribute value
$firstview->rpz_drop_ip_rule_enabled("true");
```

rpz_drop_ip_rule_min_prefix_length_ipv4( )

Use this method to set or retreive minimum prefix length for IPv4 RPZ-IP triggers.

Setting this method to a defined value implicitly sets the override_rpz_drop_ip_rule method to "true". Setting the parameter to undefined if rpz_drop_ip_rule_enabled and rpz_drop_ip_rule_min_prefix_length_ipv6 are undefined causes the appliance to use the Grid default and automatically resets the override_rpz_drop_ip_rule attribute to "false".

Note that when rpz_drop_ip_rule_min_prefix_length_ipv4 is set to a defined value and override_rpz_drop_ip_rule is set to "false", the last operation takes precedence. Thus the sequence $object->rpz_drop_ip_rule_min_prefix_length_ipv4("20"); $object->override_rpz_drop_ip_rule("false"); will set override_rpz_drop_ip_rule to "false", and the sequence $object->override_rpz_drop_ip_rule("false"); $object->rpz_drop_ip_rule_min_prefix_length_ipv4("20"); will result in override_rpz_drop_ip_rule="true".

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

The valid value is an unsigned integer between 1 and 31 that represents prefix length (in bits) for IPv4 RPZ-IP triggers.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
# Get attribute value
my $value = $firstview->rpz_drop_ip_rule_min_prefix_length_ipv4();
# Modify attribute value
$firstview->rpz_drop_ip_rule_min_prefix_length_ipv4("16");
```

rpz_drop_ip_rule_min_prefix_length_ipv6( )

Use this method to set or retreive minimum prefix length for IPv6 RPZ-IP triggers.

Setting this method to a defined value implicitly sets the override_rpz_drop_ip_rule method to "true". Setting the parameter to undefined if rpz_drop_ip_rule_enabled and rpz_drop_ip_rule_min_prefix_length_ipv4 are undefined causes the appliance to use the Grid default and automatically resets the override_rpz_drop_ip_rule attribute to "false".

Note that when rpz_drop_ip_rule_min_prefix_length_ipv6 is set to a defined value and override_rpz_drop_ip_rule is set to "false", the last operation takes precedence. Thus the sequence $object->rpz_drop_ip_rule_min_prefix_length_ipv6("20"); $object->override_rpz_drop_ip_rule("false"); will set override_rpz_drop_ip_rule to "false", and the sequence $object->override_rpz_drop_ip_rule("false"); $object->rpz_drop_ip_rule_min_prefix_length_ipv6("20"); will result in override_rpz_drop_ip_rule="true".

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

The valid value is an unsigned integer between 1 and 127 that represents prefix length (in bits) for IPv6 RPZ-IP triggers.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
# Get attribute value
my $value = $firstview->rpz_drop_ip_rule_min_prefix_length_ipv6();
# Modify attribute value
$firstview->rpz_drop_ip_rule_min_prefix_length_ipv6("120");
```

rpz_qname_wait_recurse( )

Use this method to enable or disable recursive RPZ lookups.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

Specify "true" to enable recursive RPZ lookups or "false" to disable them. The default value is "false".

Note that if both rpz_qname_wait_recurse and override_rpz_qname_wait_recurse are set to some values, the last operation takes precedence. Thus the sequence $firstview->rpz_qname_wait_recurse("true"); $firstview->override_rpz_qname_wait_recurse("false"); results in inheriting rpz_qname_wait_recurse value from the Grid. And the sequence

$firstview->override_rpz_qname_wait_recurse("false"); $firstview->rpz_qname_wait_recurse("true"); sets override_rpz_qname_wait_recurse to "true".

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
# Get attribute value
my $value = $firstview->rpz_qname_wait_recurse();
# Modify attribute value
$firstview->rpz_qname_wait_recurse("true");
```

response_rate_limiting( )

Use this method to set or retrieve response rate limiting settings for the DNS view.

Setting this method to a defined value implicitly sets the override_response_rate_limiting method to "true". Setting the parameter to undefined causes the appliance to use the Member DNS default and automatically resets the override_response_rate_limiting method to "false".

Note that when response_rate_limiting is set to a defined value and override_response_rate_limiting is set to "false", the last operation takes precedence. Thus the sequence $object->response_rate_limiting($response_rate_limiting); $object->override_response_rate_limiting("false"); will set override_response_rate_limiting to "false", and the sequence $object->override_response_rate_limiting("false"); $object->response_rate_limiting($response_rate_limiting); will result in override_response_rate_limiting="true".

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

The valid value is an Infoblox::Grid::DNS::ResponseRateLimiting object.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get response_rate_limiting
my $response_rate_limiting = $firstview->response_rate_limiting();
#Modify response_rate_limiting
$firstview->response_rate_limiting($response_rate_limiting);
```

use_root_name_servers( )

Use this method to enable/disable custom or Internet root name servers. This setting overrides grid root name server settings.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

Specify "true" to enable custom root name servers. Specify "false" to use Internet root name servers.

Returns

If you specify a parameter, the method returns true when the modification succeeds, and returns false when the operation fails. If you do not specify a parameter, the method returns the attribute value.

Example

```
#Get use_root_name_servers
my $useRootNS = $firstview->use_root_name_servers( );
#Modify use_root_name_servers
$firstview->use_root_name_servers("false");
```

override_nxdomain_redirect( )

The "override_nxdomain_redirect" attribute controls whether the "nxdomain_redirect", "nxdomain_redirect_addresses", "nxdomain_redirect_addresses_ipv6", "nxdomain_redirect_ttl", "nxdomain_log_query", and "nxdomain_rulesets" values for this DNS view are used.

The "override_nxdomain_redirect" attribute can be specified explicitly. It is also set implicitly as follows: "nxdomain_redirect" =>

"true"|"false" or "nxdomain_redirect_addresses" => ["10.0.0.10", "10.0.0.20"]|[] or "nxdomain_redirect_addresses" => ["2001:db8::1", "2002:ffff::1"]|[] result in "override_nxdomain_redirect" => "true", "nxdomain_redirect" => undef and "nxdomain_redirect_addresses" => undef and "nxdomain_redirect_addresses_ipv6" => undef result in "override_nxdomain_redirect" => "false". If "override_nxdomain_redirect" is specified explicitly as "true", and "nxdomain_redirect" => undef, "nxdomain_redirect_addresses" => undef and "nxdomain_redirect_addresses_ipv6" => undef, then the explicit value takes precedence; and "override_nxdomain_redirect"=>"true", "nxdomain_redirect", "nxdomain_redirect_addresses", and "nxdomain_redirect_addresses" remain unchanged in the database.

Specifying the "nxdomain_redirect_ttl", "nxdomain_log_query", or "nxdomain_rulesets" attributes does not affect the "override_nxdomain_redirect" value. It is implicitly set only by "nxdomain_redirect".

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

Set the parameter to "true" to override the member-level settings of the NXDOMAIN redirect attributes. Set the parameter to "false" to inherit the member-level settings. The default is "false".

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get override_nxdomain_redirect
my $override_nxdomain_redirect=$firstview->override_nxdomain_redirect( );
#Modify override_nxdomain_redirect
$firstview->override_nxdomain_redirect("true");
```

nxdomain_redirect( )

Use this method to enable or disable NXDOMAIN redirection in a DNS view.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

Specify "true" to set the nxdomain_redirect flag or "false" to deactivate/unset it. The default is "false".

Setting this parameter to "true" or "false" implicitly sets the override_nxdomain_redirect attribute to "true". Setting the parameter to undefined causes the appliance to use the grid-level NXDOMAIN redirection settings and automatically sets the override_nxdomain_redirect attribute to "false" if nxdomain_redirect_addresses or nxdomain_redirect_addresses_ipv6 is also undefined. But if nxdomain_redirect_addresses or nxdomain_redirect_addresses_ipv6 contains a value, then the override_nxdomain_redirect attribute is set to true.

Note that when nxdomain_redirect is set to "true"/"false" and override_nxdomain_redirect is set to "false", the last operation takes precedence. Thus the sequence $firstview->nxdomain_redirect("false"); $firstview->override_nxdomain_redirect("false"); sets override_nxdomain_redirect to "false", and the sequence $firstview->override_nxdomain_redirect("false"); $firstview->nxdomain_redirect("false"); sets override_nxdomain_redirect to "true".

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Getting nxdomain_redirect
my $nxdomain_redirect = $firstview->nxdomain_redirect( );
#Modifying nxdomain_redirect
$firstview->nxdomain_redirect("true");
```

nxdomain_redirect_addresses( )

Use this method to specify the IP addresses the appliance includes in the response it sends in place of an NXDOMAIN response.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

The valid value is an array of IPv4 addresses. The default value is undefined.

Setting this parameter to a defined value implicitly sets the override_nxdomain_redirect attribute to "true". Setting the parameter to undefined causes the appliance to use the grid-level NXDOMAIN redirection settings and automatically sets the override_nxdomain_redirect attribute to "false" if nxdomain_redirect and nxdomain_redirect_addresses_ipv6 is also undefined. But

if nxdomain_redirect or nxdomain_redirect_addresses_ipv6 contains a value, then the override_nxdomain_redirect attribute is set to true.

Note that when nxdomain_redirect_addresses contains a valid array reference and override_nxdomain_redirect is set to "false", the last operation takes precedence. Thus the sequence $firstview->nxdomain_redirect_addresses(["10.0.0.10", "10.0.0.20"]); $firstview->override_nxdomain_redirect("false"); sets override_nxdomain_redirect to "false", and the sequence $firstview->override_nxdomain_redirect("false"); $firstview->nxdomain_redirect_addresses(["10.0.0.10", "10.0.0.20"]); sets override_nxdomain_redirect to "true".

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get nxdomain_redirect_addresses
my $nxdomain_redirect_addresses = $firstview->nxdomain_redirect_addresses( );
#Modify nxdomain_redirect_addresses
$firstview->nxdomain_redirect_addresses(["10.0.0.10", "10.0.0.20"]);
```

nxdomain_redirect_addresses_ipv6( )

Use this method to specify the IP addresses the appliance includes in the response it sends in place of an NXDOMAIN response.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

The valid value is an array of IPv6 addresses. The default value is undefined.

Setting this parameter to a defined value implicitly sets the override_nxdomain_redirect attribute to "true". Setting the parameter to undefined causes the appliance to use the grid-level NXDOMAIN redirection settings and automatically sets the override_nxdomain_redirect attribute to "false" if nxdomain_redirect and nxdomain_redirect_addresses is also undefined. But if nxdomain_redirect or nxdomain_redirect_addresses contains a value, then the override_nxdomain_redirect attribute is set to true.

Note that when nxdomain_redirect_addresses_ipv6 contains a valid array reference and override_nxdomain_redirect is set to "false", the last operation takes precedence. Thus the sequence $firstview->nxdomain_redirect_addresses_ipv6(["2001:db8::1", "2002:ffff::1"]); $firstview->override_nxdomain_redirect("false"); sets override_nxdomain_redirect to "false", and the sequence $firstview->override_nxdomain_redirect("false"); $firstview->nxdomain_redirect_addresses_ipv6(["2001:db8::1", "2002:ffff::1"]); sets override_nxdomain_redirect to "true".

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get nxdomain_redirect_addresses_ipv6
my $nxdomain_redirect_addresses_ipv6 = $firstview->nxdomain_redirect_addresses_ipv6( );
#Modify nxdomain_redirect_addresses_ipv6
$firstview->nxdomain_redirect_addresses_ipv6(["2001:db8::1", "2002:ffff::1"]);
```

nxdomain_redirect_ttl( )

Use this method to set or retrieve the TTL value of the synthetic DNS responses that result from NXDOMAIN redirection.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

A 32-bit integer, from 0 to 4294967295, that represents the TTL in seconds. The default value is 60.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get nxdomain_redirect_ttl
my $nxdomain_redirect_ttl = $firstview->nxdomain_redirect_ttl( );
#Modify nxdomain_redirect_ttl
$firstview->nxdomain_redirect_ttl(60);
```

nxdomain_log_query( )

Use this method to set or retrieve the flag that indicates whether NXDOMAIN redirection queries are logged.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

Specify "true" to enable logging, or "false" to disable it. The default value is "false".

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get nxdomain_log_query
my $nxdomain_log_query = $firstview->nxdomain_log_query( );
#Modify nxdomain_log_query
$firstview->nxdomain_log_query("true");
```

nxdomain_rulesets( )

Use this method to specify or retrieve Infoblox::DNS::Ruleset object names assigned at the grid level for NXDOMAIN redirection.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

The valid value is an array reference that contains the names of the Infoblox::DNS::Ruleset objects assigned to the grid. The names must be in string format.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get nxdomain_rulesets
my $nxdomain_rulesets = $firstview->nxdomain_rulesets();
#Modify nxdomain_rulesets
$firstview->nxdomain_rulesets(["ruleset1", "ruleset2"]);
```

override_rpz_drop_ip_rule( )

The override_rpz_drop_ip_rule attribute controls whether the rpz_drop_ip_rule_enabled, rpz_drop_ip_rule_min_prefix_length_ipv4 and rpz_drop_ip_rule_min_prefix_length_ipv6 method values of the member are used, instead of the Grid default.

The override_rpz_drop_ip_rule attribute can be specified explicitly. It is also set implicitly when one of rpz_drop_ip_rule_enabled, rpz_drop_ip_rule_min_prefix_length_ipv4 or rpz_drop_ip_rule_min_prefix_length_ipv6 methods is set to a defined value.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

Set the parameter to "true" to override the Grid-level setting for rpz_drop_ip_rule_enabled, rpz_drop_ip_rule_min_prefix_length_ipv4 and rpz_drop_ip_rule_min_prefix_length_ipv6 methods. Set the parameter to "false" to inherit the Grid-level setting for rpz_drop_ip_rule_enabled, rpz_drop_ip_rule_min_prefix_length_ipv4 and rpz_drop_ip_rule_min_prefix_length_ipv6 methods.

The default value of this parameter is "false".

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Getting override_rpz_drop_ip_rule
my $override_rpz_drop_ip_rule = $firstview->override_rpz_drop_ip_rule( );
#Modifying override_rpz_drop_ip_rule
$firstview->override_rpz_drop_ip_rule("true");
```

override_rpz_qname_wait_recurse( )

The override_rpz_qname_wait_recurse attribute controls whether the rpz_qname_wait_recurse method value for the member is used, instead of the Grid default.

The "override_rpz_qname_wait_recurse" attribute can be specified explicitly. It is also established implicitly as follows: "rpz_qname_wait_recurse" => "true"|"false" results in "override_rpz_qname_wait_recurse" => "true"; "rpz_qname_wait_recurse" => undef results in "override_rpz_qname_wait_recurse" => "false"; If "override_rpz_qname_wait_recurse" is specified explicitly as "true", while "rpz_qname_wait_recurse"=>undef, the explicit value takes precedence, and "override_rpz_qname_wait_recurse"=>"true" while "rpz_qname_wait_recurse" remains unchanged in the database.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

Set the parameter to "true" to override the Grid-level setting for rpz_qname_wait_recurse. Set the parameter to "false" to inherit the Grid-level setting for rpz_qname_wait_recurse value.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Getting override_rpz_qname_wait_recurse
my $override_rpz_qname_wait_recurse = $firstview->override_rpz_qname_wait_recurse();
#Modyfying override_rpz_qname_wait_recurse
$firstview->override_rpz_qname_wait_recurse("true");
```

override_response_rate_limiting( )

The override_response_rate_limiting attribute controls whether the response_rate_limiting method values of the member are used, instead of the Grid Member default.

The override_response_rate_limiting attribute can be specified explicitly. It is also set implicitly when response_rate_limiting is set to a defined value.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

Set the parameter to "true" to override the member-level setting for response_rate_limiting. Set the parameter to "false" to inherit the member-level setting for response_rate_limiting.

The default value of this parameter is "false".

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get override_response_rate_limiting
my $override_response_rate_limiting = $firstview->override_response_rate_limiting( );
#Modify override_response_rate_limiting
$firstview->override_response_rate_limiting("true");
```

override_sortlist( )

The override_sortlist attribute controls whether the sort list value in this DNS view is used, instead of the member default.

The override_sortlist attribute can be specified explicitly. It is also set implicitly when the sort list is set to a defined value.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

Set the parameter to "true" to override the member-level setting for the sort list. Set the parameter to "false" to inherit the member-level setting for the sort list.

The default value of this parameter is "false".

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Getting override_sortlist
my $override_sortlist=$firstview->override_sortlist( );
#Modifying override_sortlist
$firstview->override_sortlist("true");
```

sortlist( )

Use this method to create a sort list.

A sort list determines the order of addresses in responses made to DNS queries.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Setting this method to a defined value implicitly sets the override_sortlist method to "true". Setting the parameter to undefined causes the appliance to use the member default and automatically resets the override_sortlist attribute to "false".

Note that when the sort list is set to a defined value and override_sortlist is set to "false", the last operation takes precedence. Thus the sequence $object->sortlist([$s1,$s2]); $object->override_sortlist("false"); will set override_sortlist to "false", and the sequence $object->override_sortlist("false"); $object->sortlist([$s1,$s2]); will result in override_sortlist="true".

Parameter

Valid value is an array reference that contains Infoblox::DNS::Sortlist object(s).

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
my $sort1 = Infoblox::DNS::Sortlist->new (
                        "source_ipv4addr" => "1.2.3.4",
                        "match_list"      => [ "10.20.1.0/24" , "10.20.2.0/24"]
                );
$firstview->sortlist( [ $sort1 ] );
```

SAMPLE CODE

The following sample code demonstrates different operations that can be applied to an object, such as create, search, modify, and remove. This sample code also includes error handling for the operations.

#Preparation prior to a DNS view insertion

```
 use strict;
 use Infoblox;

#Create a session to the Infoblox appliance
my $session = Infoblox::Session->new(
    master   => "192.168.1.2",
    username => "admin",
    password => "infoblox"
);
unless ($session) {
   die("Construct session failed: ",
       Infoblox::status_code() . ":" . Infoblox::status_detail());
}
print "Session created successfully\n";
```

#Create a DNS view

```
#create member
my $member1 = Infoblox::DNS::Member->new(
    name     => "member.com",
    ipv4addr => "192.168.1.2",
    lead     => "false",
    stealth  => "false"
);

#create keys
```

```
my $key1 = Infoblox::DNS::TSIGKey->new(
    name => "tsig_xfer",
    key  => ":2xCOMPAT"
);

my $tsig_key2 = $session->gen_tsig_key();

my $key2 = Infoblox::DNS::TSIGKey->new(
    name => "key2",
    key  => $tsig_key2
);

#create View object using members
my $firstview = Infoblox::DNS::View->new(
    name              => "internal",
    match_clients     => ["1.1.1.1", $key1, "10.0.0.0/8", $key2],
    recursion         => "true",
    disable           => "false",
    comment           => "new Internal view"
    );
 unless ($firstview) {
          die("constructor for view internal failed: ",
               Infoblox::status_code() . ":" . Infoblox::status_detail());
     }

#put View into session
$session->add($firstview);
unless ($firstview) {
      die("add view internal failed: ",
      $session->status_code() . ":" . $session->status_detail());
}
```

#Search for a specific view

```
#Searching the internal view
my @result = $session->search(object => "Infoblox::DNS::View", name => "^internal\$");
my $object = $result[0];
unless ($object) {
          die("Search view failed: ",
               $session->status_code() . ":" . $session->status_detail());
     }
print "Search view object found at least 1 matching entry\n";
```

#Get and modify a DNS view

```
my @result = $session->get(
    object => "Infoblox::DNS::View",
    name   => "internal"
);

my $firstview = $result[0];
unless ($firstview) {
         die("Get view failed: ",
           $session->status_code() . ":" . $session->status_detail());
}
print "Get view found at least 1 matching entry\n";
$firstview->comment("new comment internal view");

#Apply the changes
$session->modify($firstview);
unless ($firstview) {
          die("Modify view failed: ",
               $session->status_code() . ":" . $session->status_detail());
     }
print "View object modified successfully \n";
```

#Remove a view

```
my @result = $session->get(
    object => "Infoblox::DNS::View",
    name   => "internal"
);

my $firstview = $result[0];
unless ($firstview) {
         die("Get view failed: ",
```

```
            $session->status_code() . ":" . $session->status_detail());
}
print "Get view found at least 1 matching entry\n";

#submit the object for removal
$session->remove($firstview);
unless ($firstview) {
          die("Remove view failed: ",
            $session->status_code() . ":" . $session->status_detail());
}
print "View object removed successfully \n";

####PROGRAM ENDS####
```

AUTHOR

Infoblox Inc. http://www.infoblox.com/

SEE ALSO

Infoblox::DNS::View, Infoblox::DHCP::View, Infoblox::DNS::Zone, Infoblox::DNS::DnssecTrustedKey, Infoblox::DNS::Ruleset, Infoblox::Session, Infoblox::Session->get(), Infoblox::Session->search(), Infoblox::Session->add(), Infoblox::Session->remove(), Infoblox::Session->modify(), Infoblox::DNS::Sortlist, Infoblox::Grid::NamedACL, Infoblox::Grid::DNS::FixedRRSetOrderFQDN, Infoblox::Grid::DNS::ResponseRateLimiting

COPYRIGHT

Copyright (c) 2017 Infoblox Inc.

Infoblox::DNS::Zone - DNS Zone object

- NAME
- DESCRIPTION
- CONSTRUCTOR
- SESSION METHODS
  - Infoblox::Session->add( )
  - Infoblox::Session->get( )
  - Infoblox::Session->modify( )
  - Infoblox::Session->remove( )
  - Infoblox::Session->search( )
- METHODS
  - ad_servers( )
  - allow_gss_tsig_for_underscore_zone( )
  - allow_gss_tsig_zone_updates( )
  - allow_ptr_creation_in_parent( )
  - allow_query( )
  - allow_transfer( )
  - allow_update( )
  - allow_update_forwarding( )
  - bind_check_names_policy( )
  - comment( )
  - cloud_info( )
  - convert_a_host_ptr_to_bulk_host( )
  - copy_xfer_to_notify( )
  - create_ptr_for_bulk_hosts( )
  - create_ptr_for_hosts( )
  - create_underscore_zones( )
  - ddns_force_creation_timestamp_update( )
  - ddns_restrict_patterns_list( )
  - ddns_restrict_patterns( )
  - ddns_restrict_static( )
  - ddns_restrict_protected( )
  - ddns_restrict_secure( )
  - ddns_principal_tracking( )
  - ddns_principal_group( )
  - delegate_to( )
  - delegated_ttl( )
  - delegation_ns_group( )
  - disable( )
  - disable_forwarding( )
  - dnssec_enable_ksk_auto_rollover( )
  - dnssec_keys( )
  - dnssec_ksk_email_notification_enabled( )
  - dnssec_ksk_rollover_date( )
  - dnssec_ksk_rollover_notification_config( )
  - dnssec_ksk_snmp_notification_enabled( )
  - dns_email( )
  - dns_integrity_check_enable( )
  - dns_integrity_frequency( )
  - dns_integrity_member( )
  - dns_integrity_verbose_logging( )
  - dns_name( )
  - dns_soa_mname( )
  - dnssec_ksk_algorithms( )
  - dnssec_ksk_rollover_interval()
  - dnssec_nsec3_iterations( )
  - dnssec_nsec3_salt_min_length( )
  - dnssec_nsec3_salt_max_length( )
  - dnssec_resign( )
  - dnssec_signature_expiration( )
  - dnssec_signed( )
  - dnssec_zsk_algorithms( )
  - dnssec_zsk_rollover_date( )
  - dnssec_zsk_rollover_interval()
  - dnssec_zsk_rollover_mechanism( )
  - do_host_abstraction( )
  - enable_ad_servers( )
  - email( )
  - execute_dns_parent_check( )
  - extattrs( )
  - extensible_attributes( )
  - fireeye_rule_mapping( )

- forward_external_ns_group( )
- forward_ns_group( )
- forward_only( )
- forward_to( )
- host_name_restriction_policy( )
- import_from( )
- initiate_ksk_rollover( )
- initiate_zsk_rollover( )
- is_default( )
- is_multimaster( )
- last_queried( )
- rpz_drop_ip_rule_enabled( )
- rpz_drop_ip_rule_min_prefix_length_ipv4( )
- rpz_drop_ip_rule_min_prefix_length_ipv6( )
- rpz_policy( )
- restart_if_needed( )
- rpz_type( )
- rpz_severity( )
- substitute_name( )
- rpz_priority( )
- rpz_last_updated_time( )
- rpz_severity( )
- is_feed_zone( )
- locked( )
- members( )
- member_soa_serials( )
- member_soa_mnames( )
- ms_dc_ns_record_creation( )
- name( )
- network_associations( )
- next_secure_type( )
- notify_delay( )
- `ms_ad_integrated()`
- `ms_managed()`
- `ms_read_only()`
- `ms_ddns_mode()`
- `ms_allow_transfer()`
- ms_sync_disable( )
- ns_group( )
- override_copy_xfer_to_notify( )
- override_ddns_force_creation_timestamp_update( )
- override_ddns_patterns_restriction( )
- override_ddns_restrict_static( )
- override_ddns_restrict_protected( )
- override_ddns_principal_security( )
- override_delegated_ttl( )
- override_grid_zone_timer( )
- override_grid_email( )
- override_scavenging_settings( )
- override_rpz_drop_ip_rule( )
- override_serial_number( )
- prefix( )
- multiple_primaries( )
- primary_shared_with_ms_parent_delegation( )
- scavenging_settings( )
- remove_sub_zones( )
- secondaries( )
- resource_record_monitored_since( )
- soa_default_ttl( )
- soa_expire( )
- soa_mname( )
- soa_negative_ttl( )
- soa_refresh( )
- soa_retry( )
- soa_serial_number( )
- shared_record_groups( )
- stub_external_ns_group( )
- stub_from( )
- stub_members( )
- stub_ns_group( )
- views( )
- zone( )
- zone_monitored_since( )

- SAMPLE CODE

- AUTHOR
- SEE ALSO
- COPYRIGHT

NAME

Infoblox::DNS::Zone - DNS Zone object

DESCRIPTION

A zone is a portion of the domain name space for which an Infoblox appliance or another name server is authoritative (for example, has the start of authority [SOA] record). A zone can also be delegated or forwarded. Zones are the primary objects used to manage DNS data and DNS services.

Note that you must specify only one view for the attribute "views".

CONSTRUCTOR

```
 my $zone = Infoblox::DNS::Zone->new(
     name                                    => $fqdn | $ipv4_subnet/prefix | $ipv6_subnet/prefix,
# Required
     ad_servers                              => [ $ipv4addr | $ipv6addr ],
# Optional / Default is empty list
     allow_gss_tsig_for_underscore_zone      => "true" | "false",
# Optional / Default is "false"
     allow_gss_tsig_zone_updates             => "true" | "false",
# Optional / Default is "false"
     allow_query                             => [ $ipv4addr | $ipv4_subnet/prefix | $ipv6addr |
                                                  $ipv6_subnet/prefix | $TSIGKey ,... ] | $nacl |
undef,                  # Optional / Default is undefined
     allow_transfer                          => [ $ipv4addr | $ipv4_subnet/prefix | $ipv6addr |
                                                  $ipv6_subnet/prefix | $TSIGKey ,... ] | $nacl |
undef,                  # Optional / Default is undefined
     allow_update                            => [ $ipv4addr | $ipv4_subnet/prefix | $ipv6addr |
                                                  $ipv6_subnet/prefix | $TSIGKey ,... ] | $nacl |
undef,                  # Optional / Default is undefined
     allow_update_forwarding                 => [ $ipv4addr | $ipv4_subnet/prefix | $ipv6addr |
                                                  $ipv6_subnet/prefix | $TSIGKey ,... ] | $nacl |
undef,                  # Optional / Default is undefined
     bind_check_names_policy                 => [ "fail" | "warn" | undef],
# Optional / Default is undefined
     comment                                 => $string,
# Optional / Default is undefined
     create_underscore_zones                 => "true" | "false",
# Optional / Default is "false"
     delegate_to                             => [ $Nameserver1, $Nameserver2 ... ],
# Optional / Default is undefined
     delegated_ttl                           => $num,
# Optional / Default is undefined
     delegation_ns_group                     => $string | $delegation_ns_group
# Optional / Default is undefined
     disable                                 => "true" | "false",
# Optional / Default is "false"
     disable_forwarding                      => "true" | "false",
# Optional / Default is "false"
     dnssec_enable_ksk_auto_rollover         => "true" | "false",
# Optional / Default is "false"
     dnssec_keys                             => $keys,
# Optional / Default is empty list
     dnssec_zsk_algorithms                   => [$algorithm1, $algorithm2, ...],
# Optional / Default is undefined
     dnssec_ksk_email_notifiaction_enabled   => "true" | "false",
# Optional / Default is false
     dnssec_ksk_rollover_date                => $date,
# Optional / Default is undefined
     dnssec_ksk_rollover_interval            => $num,
# Optional / Default is undefined
     dnssec_ksk_rollover_notification_config => "NONE" | "ALL" | "REQUIRE_MANUAL_INTERVENTION",
# Optional / Default is undefined
     dnssec_ksk_email_notification_enabled   => "true" | "false",
# Optional / Default is "false"
     dnssec_ksk_snmp_notification_enabled    => "true" | "false",
# Optional / Default is "false"
     dnssec_nsec3_iterations                 => $num,
# Optional / Default is undefined
     dnssec_nsec3_salt_min_length            => $num,
# Optional / Default is undefined
                                                $num
```

```
      dnssec_nsec3_salt_max_length                => ,
# Optional / Default is undefined
      dnssec_signature_expiration                 => $num,
# Optional / Default is undefined
      dnssec_signed                               => "true" | "false",
# Optional / Default is undefined
      dnssec_zsk_algorithms                       => [$algorithm1, $algorithm2, ...],
# Optional / Default is undefined
      dnssec_zsk_rollover_date                    => $date,
# Optional / Default is undefined
      dnssec_zsk_rollover_mechanism               => "DOUBLE_SIGN" | "PRE_PUBLISH",
# Optional / Default is undefined
      dnssec_zsk_rollover_interval                => $num,
# Optional / Default is undefined
      next_secure_type                            => "NSEC" | "NSEC3",
# Optional / Default is undefined
      enable_ad_servers                           => "true" | "false",
# Optional / Default is "false"
      allow_ptr_creation_in_parent                => "true" | "false",
# Optional / Default is "false"
      email                                       => $email,
# Optional / Default is undefined
      extattrs                                    => { $string => $extattr, ... },
# Optional / Default is undefined
      extensible_attributes                       => { $string => $string | $num, $string => [ $string |
$num, ... ], ... }    # Optional / Default is undefined
      fireeye_rule_mapping                        => $rulemap,
# Optional / Default is undefined
      forward_only                                => "true" | "false",
# Optional / Default is "false"
      forward_to                                  => [ $Member1, $Member2, $Nameserver1,
$Nameserver2,...],                    # Optional / Default is undefined
      forward_ns_group                            => $string | $forward_ns_group
# Optional / Default is undefined
      forward_external_ns_group                   => $string
# Optional / Default is undefined
      host_name_restriction_policy                => $string,
# Optional / Default is undefined
      import_from                                 => $ipv4addr,
# Optional / Default is undefined
      is_default                                  => "true" | "false",
# Optional / Default is "false"
      is_multimaster                              => "true" | "false",
# Optional / Default is "false"
      locked                                      => "true" | "false",
# Optional / Default is "false"
      members                                     => [ $Member1, $Member2,...],
# Optional / Default is undefined
      member_soa_mnames                           => [$soa_mname1, $soa_mname2, ...],
# Optional / Default is undefined
      member_soa_serials                          => [$soa_serial1, $soa_serial2, ...],
# Optional / Default is undefined
      ms_ad_integrated                            => "true" | "false"
# Optional / Default is "false"
      ms_allow_transfer                           => "Any" | "NSOnly" | "None" | [ $ipv4addr1, ... ]
# Optional / Default is "NSOnly"
      ms_ddns_mode                                => "Secure" | "None" | "Any"
# Optional / Default is undefined
      ms_sync_disable                             => "true" | "false"
# Optional / Default is "false"
      notify_delay                                => $num,
# Optional / Default is undefined
      multiple_primaries                          => [$Member1, ...] | [$Nameserver1, ...] | [$MSServer1,
...],                 # Optional / Default is undefined
      ns_group                                    => $string | $ns_group
# Optional / Default is undefined
      override_delegated_ttl                      => "true" | "false",
# Optional / Default is "false"
      override_grid_zone_timer                    => "true" | "false",
# Optional / Default is "false"
      override_grid_email                         => "true" | "false",
# Optional / Default is "false"
      override_serial_number                      => "true" | undef,
# Optional / Default is undef
      prefix                                      => $string,
# Optional / Default is undefined
      scavenging_settings                         => $scavenging_settings,
# Optional / Default is undefined
      remove_sub_zones                            => "true" | "false",
# Optional / Default is undefined
      rpz_type                                    => "LOCAL" | "FIREEYE" | "FEED",
# Optional / Default is undefined
      rpz_severity                                => "CRITICAL" | "MAJOR" | "WARNING" | "INFORMATIONAL",
```

```
  # Optional / Default is undefined
      secondaries                                => [ $Member1, $Member2, $Nameserver1, $MSServer2, ...],
  # Optional / Default is empty
      soa_default_ttl                            => $num,
  # Optional / Default is undefined
      soa_expire                                 => $num,
  # Optional / Default is undefined
      soa_mname                                  => $fqdn,
  # Optional / Default is undefined
      soa_negative_ttl                           => $num,
  # Optional / Default is undefined
      soa_retry                                  => $num,
  # Optional / Default is undefined
      soa_refresh                                => $num,
  # Optional / Default is undefined
      soa_serial_number                          => $num,
  # Optional / Default is 1
      shared_record_groups                       => [ $shared_record_group1_name, ... ],
  # Optional / Default is undefined
      stub_from                                  => [ $Nameserver1, $Nameserver2, ... ],
  # Optional / Default is undefined
      stub_members                               => [ $Member1, $Member2, $MSServer1, $MSServer2, ...],
  # Optional / Default is undefined
      stub_ns_group                              => $string | $stub_ns_group
  # Optional / Default is undefined
      stub_external_ns_group                     => $string
  # Optional / Default is undefined
      views                                      => [$View1],
  # Optional / Default is "default" view
      rpz_policy                                 => "GIVEN" | "PASSTHRU" | "NXDOMIAN" | "NODATA" |
  "SUBSTITUTE" | "DISABLED", # Optional / Default is undefined
      substitute_name                            => $string,
  # Optional / Default is empty
      copy_xfer_to_notify                        => "true" | "false",
  # Optional / Default is "false"
      override_copy_xfer_to_notify               => "true" | "false",
  # Optional / Default is "false"
      dns_integrity_check_enable                 => "true" | "false",
  # Optional / Default is "false"
      dns_integrity_frequency                    => $uint,
  # Optional / Default is undefined
      dns_integrity_member                       => $hostname,
  # Optional / Default is undefined
      dns_integrity_verbose_logging              => "true" | "false",
  # Optional / Default is undefined
      ddns_restrict_patterns_list                => [$string, $string, ...],
  # Optional / Default is undefined
      ddns_restrict_patterns                     => "true" | "false",
  # Optional / Default is 'false'
      ddns_restrict_static                       => "true" | "false",
  # Optional / Default is 'false'
      ddns_restrict_protected                    => "true" | "false",
  # Optional / Default is 'false'
      ddns_restrict_secure                       => "true" | "false",
  # Optional / Default is 'false'
      ddns_principal_tracking                    => "true" | "false",
  # Optional / Default is 'false'
      ddns_principal_group                       => $principal_group,
  # Optional / Default is undefined,
      ddns_force_creation_timestamp_update       => "true" | "false",
  # Optional / Default is 'false'
      override_ddns_patterns_restriction         => "true" | "false",
  # Optional / Default is 'false'
      override_ddns_restrict_static              => "true" | "false",
  # Optional / Default is 'false'
      override_ddns_restrict_protected           => "true" | "false",
  # Optional / Default is 'false'
      override_ddns_principal_security           => "true" | "false",
  # Optional / Default is 'false'
      override_ddns_force_creation_timestamp_update => "true" | "false",
  # Optional / Default is 'false'
      override_scavenging_settings               => "true" | "false",
  # Optional / Default is 'false'
      override_rpz_drop_ip_rule                  => "true" | "false" | undef,
  # Optional / Default is "false"
      rpz_drop_ip_rule_enabled                   => "true" | "false" | undef,
  # Optional / Default is "false"
      rpz_drop_ip_rule_min_prefix_length_ipv4    => $uint | undef,
  # Optional / Default is 29
      rpz_drop_ip_rule_min_prefix_length_ipv6    => $uint | undef,
  # Optional / Default is 112
      ms_dc_ns_record_creation                   => $msdcnsrecordcreation,
  # Optional / Default is empty list
   );
```

You cannot set both extattrs and extensible_attributes attributes at the same time.

SESSION METHODS

This section describes all the methods in the Infoblox::Session module that you can apply to a DNS Zone object.

Infoblox::Session->add( )

Use this method to add an object to the Infoblox appliance. See Infoblox::Session->add() for parameters and return values.

If you have applied an access control list to an object, you must restart services after you update the access control list for the changes to take effect. See Infoblox::Session->restart() method.

Example

```
#Construct an object
my $zone = Infoblox::DNS::Zone->new (
    name     => "domain.com",
    comment  => "this is a demo zone",
    disable  => "false",
    views    => [$view1]
    );
#Submit for addition
my $response = $session->add( $zone );
```

Infoblox::Session->get( )

Use this method to retrieve all the matching objects from the Infoblox appliance. See Infoblox::Session->get() for parameters and return values.

Key References

```
Apply the following attributes to get a specific DNS Zone object:

 name - Required. A zone name in string format.
 view - Optional. The DNS view in which the zone is located. The default is all DNS views.
 extattrs      - Optional. A hash reference containing extensible attributes.
 extensible_attributes - Optional. A hash reference that contains extensible attributes.
 zone - Optional. The parent zone name in string format.
```

Examples

```
my @retrieved_objs = $session->get(
    object => "Infoblox::DNS::Zone",
    name   => "domain.com",
    view   => "default" );

my @retrieved_objs = $session->get(
    object => "Infoblox::DNS::Zone",
    extensible_attributes => { 'Site' => 'Santa Clara' });
```

Infoblox::Session->modify( )

Use this method to modify an object in the Infoblox appliance. See Infoblox::Session->modify() for parameters and return values.

If you have applied an access control list to an object, you must restart services after you update the access control list for the changes to take effect. See Infoblox::Session->restart() method.

Example

```
#Use method to modify the comment.
$zone->comment("this is a modified comment");
#Submit modification
my $response = $session->modify( $zone );
```

Infoblox::Session->remove( )

Use this method to remove an object from the Infoblox appliance. See Infoblox::Session->remove() for parameters and return values.

To remove a specific object, first use `get()` or `search()` to retrieve the specific object, and then submit this object for removal.

Example

```
#Get the objects with the same name
my @retrieved_objs = $session->get(
    object => "Infoblox::DNS::Zone",
```

```
    name   => "domain.com",
    view   => "default" );
#Find the desired object from the retrieved list.
my $desired_zone = $retrieved_objs[0];
#Submit for removal
my $response = $session->remove( $desired_zone );
```

Infoblox::Session->search( )

Use this method to search for DNS Zone objects in the Infoblox appliance. See Infoblox::Session->search() for parameters and return values.

Key References

```
Apply the following attributes to search for a DNS Zone object:

 name - Required. A host name in string format (regular expression).
 view - Optional. The DNS view in which the zone is located. The default is all DNS views.
 extattrs     - Optional. A hash reference containing extensible attributes.
 extensible_attributes - Optional. A hash reference containing extensible attributes.
 comment - Optional. A comment in string format (regular expression).
 primary_association_type - Optional. "Microsoft" is the only allowed value. If it is specified, the
search returns only zones with primary servers that are Microsoft (r) DNS servers.
 dnssec_ksk_rollover_date - Optional. Value is the number of seconds that have elapsed since January
1st, 1970 UTC for the next KSK rollover.
 dnssec_zsk_rollover_date - Optional. Value is the number of seconds that have elapsed since January
1st, 1970 UTC for the next ZSK rollover.
 zone - Optional. The parent zone name in string format (regular expression).
```

For more information about searching extensible attributes, see Infoblox::Grid::ExtensibleAttributeDef/Searching Extensible Attributes.

Examples

```
# search for all DNS zone objects under com in default view
my @retrieved_objs = $session->search(
    object => "Infoblox::DNS::Zone",
    name   => ".*\\.com",
    view   => "default" );

# search for all DNS zone objects defining "Santa Clara" for "Site" extensible attribute
my @retrieved_objs = $session->get(
    object => "Infoblox::DNS::Zone",
    extensible_attributes => { 'Site' => 'Santa Clara' });
```

METHODS

This section describes all the methods that you can use to configure and retrieve the attribute values of a zone.

ad_servers( )

Use this method to set or retrieve the list of AD domain controllers that are allowed to send GSS-TSIG unsigned updates.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

The valid value is an array reference that contains IPv4 or IPv6 addresses.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
# Get ad_servers
my $ad_servers = $zone->ad_servers();
# Modify ad_servers
$zone->ad_servers(["1.2.3.4", "dead::beef"]);
```

allow_gss_tsig_for_underscore_zone( )

Use this method to set or retrieve the flag that allows DHCP clients to perform GSS-TSIG signed updates for underscore zones.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

Specify "true" to set the disable flag or "false" to deactivate/unset it. The default value is "false".

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
# Get allow_gss_tsig_for_underscore_zone.
my $allow_gss_tsig_for_underscore_zone = $zone->allow_gss_tsig_for_underscore_zone();
# Modify allow_gss_tsig_for_underscore_zone.
$zone->allow_gss_tsig_for_underscore_zone("true");
```

allow_gss_tsig_zone_updates( )

Use this method to enable/disable the zone to accept GSS-TSIG updates. This setting overrides the grid and member GSS-TSIG zone update settings.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

Specify "true" to set the disable flag or "false" to deactivate/unset it. The default value is "false".

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
# Get allow_gss_tsig_zone_updates.
my $allow_gss_tsig_zone_updates = $zone->allow_gss_tsig_zone_updates();
# Modify allow_gss_tsig_zone_updates.
$zone->allow_gss_tsig_zone_updates("true");
```

allow_ptr_creation_in_parent( )

This flag controls whether automatic generation of RFC 2317 CNAMEs for delegated reverse zones overwrite existing PTR records. The default behavior is to overwrite all the existing records in the range, this corresponds to "allow_ptr_creation_in_parent" set to "false". However, when this flag is set to "true" the existing PTR records are not overwritten.

Parameter

Specify "true" to set the flag or "false" to deactivate/unset it. The default value is "false".

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
# Get allow_ptr_creation_in_parent.
my $allow_ptr_creation_in_parent = $zone->allow_ptr_creation_in_parent();
# Modify allow_ptr_creation_in_parent.
$zone->allow_ptr_creation_in_parent("true");
```

allow_query( )

Use this method to set or retrieve the allow_query flag of a DNS zone.

You can allow or reject queries from specified IP addresses and networks. You can also specify TSIG keys for the allowed hosts. This setting overrides the member-level query settings.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

The valid value is an Infoblox::Grid::NamedACL object or an array reference that contains IPv4/IPv6 addresses, networks, and/or Infoblox::DNS::TSIGKey objects. To deny queries from certain IP addresses and networks, add "!" as a prefix to the IP addresses and networks. Specify "any" to allow any query, or undefine to use the Grid::Member::DNS level allow_query list. The default value is undefined.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Getting allow_query
my $allow_query = $zone->allow_query();
#Modifying allow_query
$zone->allow_query(["any"]);

#Specifying TSIG keys
my $tsig_key1 = $session->gen_tsig_key();
my $query_key = Infoblox::DNS::TSIGKey->new(
   name => "query_key1",
   key  => $tsig_key1
);
#Modifying allow_query
$zone->allow_query( [ "10.0.0.10" , "20.0.0.0/24","2001::123", $key1,
"2004::444/64","!"."30.0.0.30" , "!"."40.10.0.0/24","!"."2005::222/64"] );
##Un-override allow_query
$zone->allow_query(undef)
#Use Infoblox::Grid::NamedACL
$zone->allow_query($nacl);
```

allow_transfer( )

Use this method to set or retrieve the allow_transfer list of a DNS zone.

You can allow or reject zone transfers from specified IP addresses and networks. This setting overrides the member-level zone transfer settings. You can also provide Transaction Signature (TSIG) keys for clients that are allowed to do zone transfers.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

The valid value is an Infoblox::Grid::NamedACL object or an array reference that contains IPv4/IPv6 addresses and networks. It is also possible to specify the TSIG keys of clients that are allowed to perform zone transfers. TSIG keys are used to authenticate zone transfer requests and replies. The same key name and key value must be on the primary and secondary name servers for TSIG-authenticated zone transfers to occur. To deny zone transfers from certain IP addresses and networks, add "!" as a prefix to the IP addresses and networks. Specify "any" to allow any transfer, or undefine to use the Grid::Member::DNS level allow_transfer list. The default value is undefined.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Getting allow_transfer
my $allow_transfer = $zone->allow_transfer( );
#Modifying allow_transfer
$zone->allow_transfer(["any"]);

#Specifying TSIG keys
my $tsig_key1 = $session->gen_tsig_key();
my $key1 = Infoblox::DNS::TSIGKey->new(
   name => "key1",
   key  => $tsig_key1
);
my $key2 = Infoblox::DNS::TSIGKey->new(
   name => "tsig_xfer",
   key  => ":2xCOMPAT"
   );
#Modifying allow_transfer
$zone->allow_transfer(["10.0.0.10", "20.0.0.0/24","2001::1234", $key1, "2001::333/64", $key2,
"!"."30.0.0.30", "!"."40.10.0.0/24","!"."2008::543","!"."2007::456/64"]);
#Use Grid::Member::DNS level allow_transfer list
$zone->allow_transfer(undef)
#Use Infoblox::Grid::NamedACL
$zone->allow_transfer($nacl);
```

allow_update( )

Use this method to set or retrieve the allow_update flag of a DNS zone.

You can allow or deny dynamic updates from specified IP addresses and networks. This setting overrides member-level update settings. You can also provide TSIG keys for clients that are allowed to do zone updates.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

The valid value is an Infoblox::Grid::NamedACL object or an array reference that contains IPv4/IPv6 addresses, networks and/or Infoblox::DNS::TSIGKey objects. To deny updates from certain IP addresses and networks, add "!" as a prefix to the IP addresses and networks. Specify "any" to allow any update, or undefine to use the Grid::Member::DNS allow_update list. Note: You should also undefine the update_keys list to use Grid::Member::DNS settings. The default value is undefined.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Getting allow_update
my $allow_update = $zone->allow_update();
#Modifying allow_update
$zone->allow_update(["any"]);

#Creating a TSIGKey object.
my $update_key1 = $session->gen_tsig_key();
my $up_key1 = Infoblox::DNS::TSIGKey->new(
   name => "update_key",
   key  => $update_key1
   );

#Modifying allow_update
$zone->allow_update(["10.0.0.10", $up_key1, "20.0.0.0/24","2006::987","2007::777/64",
"!"."30.0.0.30", "!"."40.10.0.0/24"]);

#Use Grid::Member::DNS level allow_update list
$zone->allow_update(undef)
#Use Infoblox:Grid::NamedACL
$zone->allow_update($nacl);
```

allow_update_forwarding( )

Use this method to set or retrieve the allow_update_forwarding flag of a DNS zone.

You can allow or deny dynamic updates forwarded from specified IP addresses and networks. This setting overrides member-level update settings. You can also provide TSIG keys for clients that are allowed to do zone updates.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

The valid value is an Infoblox::Grid::NamedACL object or an array reference that contains IPv4/IPv6 addresses, networks and/or Infoblox::DNS::TSIGKey objects. To deny updates from certain IP addresses and networks, add "!" as a prefix to the IP addresses and networks. Specify "any" to allow any update, or undefine to use the Grid::Member::DNS allow_update list. Note: You should also undefine the update_keys list to use Grid::Member::DNS settings. The default value is undefined.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Getting allow_update_forwarding
my $allow_update_forwarding = $zone->allow_update_forwarding();
#Modifying allow_update_forwarding
$zone->allow_update_forwarding(["any"]);

#Creating a TSIGKey object.
my $update_key1 = $session->gen_tsig_key();
my $up_key1 = Infoblox::DNS::TSIGKey->new(
   name => "update_key",
   key  => $update_key1
   );

#Modifying allow_update_forwarding
```

```
  $zone->allow_update_forwarding(["10.0.0.10", $up_key1,
"20.0.0.0/24","2009::654/64","!"."30.0.0.30", "!"."40.10.0.0/24","!"."2007::432/64"]);

  ##Un-override allow_update_forwarding
  $zone->allow_update_forwarding(undef);
  #Use Infoblox::Grid::NamedACL
  $zone->allow_update_forwarding($nacl);
```

bind_check_names_policy( )

Use this method to set or retrieve the BIND check names policy, which indicates the action the appliance takes when it encounters host names that do not comply with the Strict Hostname Checking policy. This method applies only if the host name restriction policy is set to "Strict Hostname Checking".

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

The allowed values of the parameter are "fail", "warn" or undefined. This indicates the desired BIND behavior in the case of illegal hostnames. The default value is undefined, which means the appliance ignores illegal hostnames.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get bind_check_names_policy value
my $names_policy = $zone->bind_check_names_policy();
#Modify bind_check_names_policy
$zone->bind_check_names_policy("warn");
```

comment( )

Use this method to set or retrieve a descriptive comment about the zone.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

Desired comment in string format with a maximum of 256 bytes.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get comment
my $comment = $zone->comment();
#Modify comment
$zone->comment("Modifying the DNS Zone comment");
```

cloud_info( )

Use this method to retrieve cloud API related information for the Infoblox::DNS::Zone object.

Parameter

None

Returns

The method returns the attribute value.

Example

```
# Get cloud_info
my $cloud_info = $object->cloud_info();
```

convert_a_host_ptr_to_bulk_host( )

Use this method to convert all A, HOST and PTR records that have sequential IP addresses to bulk host records.

The zone object must be retrieved from the server before calling this method.

The method returns true when the modification succeeds, and returns false when the operation fails.

Parameter

The parameter 'create_reverse_mapping' toggles if the method creates records in the matching reverse-mapping zone.

The parameter is set to 'True' if records are created in the reverse-mapping zone. Otherwise, it is set to 'False'.

Returns

Example

```
my %args;
$args{'create_reverse_mapping'} = 'true';
$zone->convert_a_host_ptr_to_bulk_host(\%args);
```

copy_xfer_to_notify( )

Use this method to enable/disable copying of the allowed IPs from zone transfer list into also-notify statement in named.conf.

Parameter

Set the parameter to "true" to enable copying of the allowed IPs from zone transfer list into also-notify statement in named.conf. Set the parameter to "false" to disable it.

The default value of the parameter is "false".

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
# Get attribute value
my $value = $zone->copy_xfer_to_notify();
# Modify attribute value
$zone->copy_xfer_to_notify('true');
```

create_ptr_for_bulk_hosts( )

Use this method to set the create_ptr_for_bulk_hosts property used in zone import operations. This is a write-only attribute.

Set this flag to indicate that Infoblox PTR records will be automatically created for Bulk Host records if necessary when the zone data is imported.

Parameter

Specify "true" to trigger host creation or "false" to disable it. The default value is "false".

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

Example

```
#Modify create_ptr_for_bulk_hosts
$zone->create_ptr_for_bulk_hosts("true");
```

create_ptr_for_hosts( )

Use this method to set the create_ptr_for_hosts property used in zone import operations. This is a write-only attribute.

Set this flag to indicate that Infoblox PTR records will be automatically created for Host records if necessary when the zone data is imported.

Parameter

Specify "true" to trigger host creation or "false" to disable it. The default value is "false".

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

Example

```
#Modify create_ptr_for_bulk_hosts
$zone->create_ptr_for_bulk_hosts("true");
```

create_underscore_zones( )

Use this method to set or retrieve the create_underscore_zones flag of a DNS zone.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

Specify "true" to set the create_underscore_zones flag or "false" to deactivate/unset it. The default value is "false".

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get create_underscore_zones
my $create_underscore_zones = $zone->create_underscore_zones();
#Modify create_underscore_zones
$zone->create_underscore_zones("true");
```

ddns_force_creation_timestamp_update( )

Use this method to set or retrieve the flag that indicates whether the creation timestamp of resource records (RR) should be updated when DDNS update happens even if there is no change to the RR.

Setting this parameter to defined value, implicitly sets the override_ddns_force_creation_timestamp_update attribute to "true". Setting this parameter to undefined causes the appliance to use the Grid-level ddns_force_creation_timestamp_update flag and automatically sets the override_ddns_force_creation_timestamp_update attribute to "false".

Note that when ddns_force_creation_timestamp_update contains a value and override_ddns_force_creation_timestamp_update is set to "false", the last operation takes precedence. Thus the sequence $object->ddns_force_creation_timestamp_update("true"); $object->override_ddns_force_creation_timestamp_update("false"); sets override_ddns_force_creation_timestamp_update to "false", and the sequence $object->override_ddns_force_creation_timestamp_update("false"); $object->ddns_force_creation_timestamp_update("true"); sets override_ddns_force_creation_timestamp_update to "true".

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

Specify 'true' to update the creation timestamp of RR on every DDNS update, and 'false' to update the creation timestamp only when RR changes during a DDNS update. The default is 'false'.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get ddns_force_creation_timestamp_update value
$value = $object->ddns_force_creation_timestamp_update();

#modify ddns_force_creation_timestamp_update value
$object->ddns_force_creation_timestamp_update('true');
```

ddns_restrict_patterns_list( )

Use this method to set or retrieve an unordered list of restriction patterns for an option of to restrict DDNS updates based on FQDN patterns.

Setting this parameter to any list, implicitly sets the override_ddns_patterns_restriction attribute to "true". Setting this parameter to undefined causes the appliance to use the Grid-level DDNS update restrictions and automatically sets the override_ddns_patterns_restriction attribute to "false" if "ddns_restrict_patterns" is also undefined. But if "ddns_restrict_patterns" contains a value, then the override_ddns_patterns_restriction attribute is set to "true".

Note that when ddns_restrict_patterns_list contains a valid array reference and override_ddns_patterns_restriction is set to "false", the last operation takes precedence. Thus the sequence $zone->ddns_restrict_patterns_list([$pattern1, $pattern2]); $zone->override_ddns_patterns_restriction("false"); sets override_ddns_patterns_restriction to "false", and the sequence $zone->override_ddns_patterns_restriction("false"); $zone->ddns_restrict_patterns_list([$pattern1, $pattern2]); sets override_ddns_patterns_restriction to "true".

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

The valid value is an array of FQDNs in a string format.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#get ddns_restrict_patterns_list value
$ddns_restrict_patterns_list = $zone->ddns_restrict_patterns_list();

#modify ddns_restrict_patterns_list value
$zone->ddns_restrict_patterns_list(['foo.com', 'bar.com']);
```

ddns_restrict_patterns( )

Use this method to set or retrieve the flag which indicates whether an option to restrict DDNS update request based on FQDN patterns is enabled or disabled.

Setting this parameter to defined value, implicitly sets the override_ddns_patterns_restriction attribute to "true". Setting this parameter to undefined causes the appliance to use the Grid-level DDNS update restrictions and automatically sets the override_ddns_patterns_restriction attribute to "false" if "ddns_restrict_patterns_list" is also undefined. But if "ddns_restrict_patterns_list" contains a value, then the override_ddns_patterns_restriction attribute is set to "true".

Note that when ddns_restrict_patterns contains a value and override_ddns_patterns_restriction is set to "false", the last operation takes precedence. Thus the sequence $zone->ddns_restrict_patterns("true"); $zone->override_ddns_patterns_restriction("false"); sets override_ddns_patterns_restriction to "false", and the sequence $zone->override_ddns_patterns_restriction("false"); $zone->ddns_restrict_patterns("true"); sets override_ddns_patterns_restriction to "true".

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

Specify 'true' to enable DDNS update request restriction base on FQDN patterns and 'false to disable it. The default value is 'false'.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#get ddns_restrict_patterns value
$ddns_restrict_patterns = $zone->ddns_restrict_patterns();

#modify ddns_restrict_patterns value
$zone->ddns_restrict_patterns('true');
```

ddns_restrict_static( )

Use this method to set or retrieve the flag which indicates whether an option to restrict DDNS update request to resource records which are marked as 'STATIC' is enabled or disabled.

Setting this parameter to defined value, implicitly sets the override_ddns_restrict_static attribute to "true". Setting this parameter to undefined causes the appliance to use the Grid-level ddns_restrict_static flag and automatically sets the override_ddns_restrict_static attribute to "false".

Note that when ddns_restrict_static contains a value and override_ddns_restrict_static is set to "false", the last operation takes precedence. Thus the sequence $zone->ddns_restrict_static("true"); $zone->override_ddns_restrict_static("false"); sets override_ddns_restrict_static to "false", and the sequence $zone->override_ddns_restrict_static("false"); $zone->ddns_restrict_static("true"); sets override_ddns_restrict_static to "true".

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

Specify 'true' to enable DDNS update requests to resource records which are marked as 'STATIC' and 'false' to disable it. The default value is 'false'.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#get ddns_restrict_static value
$ddns_restrict_static = $zone->ddns_restrict_static();

#modify ddns_restrict_static value
$zone->ddns_restrict_static('true');
```

ddns_restrict_protected( )

Use this method to set or retrieve the flag which indicates whether an option to restrict DDNS update request to protected resource records is enabled or disabled.

Setting this parameter to defined value, implicitly sets the override_ddns_restrict_protected attribute to "true". Setting this parameter to undefined causes the appliance to use the Grid-level ddns_restrict_protected flag and automatically sets the override_ddns_restrict_protected attribute to "false".

Note that when ddns_restrict_protected contains a value and override_ddns_restrict_protected is set to "false", the last operation takes precedence. Thus the sequence $zone->ddns_restrict_protected("true"); $zone->override_ddns_restrict_protected("false"); sets override_ddns_restrict_protected to "false", and the sequence $zone->override_ddns_restrict_protected("false"); $zone->ddns_restrict_protected("true"); sets override_ddns_restrict_protected to "true".

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

Specify 'true' to enable DDNS update requests to protected resource records and 'false' to disable it. The default value is 'false'.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#get ddns_restrict_protected value
$ddns_restrict_protected = $zone->ddns_restrict_protected();

#modify ddns_restrict_protected value
$zone->ddns_restrict_protected('true');
```

ddns_restrict_secure( )

Use this method to set or retrieve the flag which indicates whether DDNS update request for principal other than target resource record's principal is restricted.

Setting this parameter to a defined value, implicitly sets the override_ddns_principal_security attribute to "true". Setting this parameter to undefined causes the appliance to use the Grid-level DDNS principal security setting and automatically sets the override_ddns_principal_security attribute to "false" if "ddns_principal_tracking" and "ddns_principal_group" are also undefined. But if "ddns_principal_tracking" or "ddns_principal_group" contains a value, then the override_ddns_principal_security attribute is set to "true".

Note that when ddns_restrict_secure contains a value and override_ddns_principal_security is set to "false", the last operation takes precedence. Thus the sequence $zone->ddns_restrict_secure("true"); $zone->override_ddns_principal_security("false"); sets override_ddns_principal_security to "false", and the sequence $zone->override_ddns_principal_security("false"); $zone->ddns_restrict_secure("true"); sets override_ddns_principal_security to "true".

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

Specify 'true' to enable DDNS update requests restriction for principal other than target resource record's principal and 'false' to disable it. The default value is 'false'.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#get ddns_restrict_secure value
$ddns_restrict_secure = $zone->ddns_restrict_secure();
```

```
#modify ddns_restrict_secure value
$zone->ddns_restrict_secure('true');
```

ddns_principal_tracking( )

Use this method to set or retrieve the flag which indicates whether the DDNS principal track is enabled or disabled.

DDNS principal tracking includes putting principal name into resource record affected by update.

Setting this parameter to a defined value, implicitly sets the override_ddns_principal_security attribute to "true". Setting this parameter to undefined causes the appliance to use the Grid-level DDNS principal security setting and automatically sets the override_ddns_principal_security attribute to "false" if "ddns_restrict_secure" and "ddns_principal_group" are also undefined. But if "ddns_restrict_secure" or "ddns_principal_group" contains a value, then the override_ddns_principal_security attribute is set to "true".

Note that when ddns_principal_tracking contains a value and override_ddns_principal_security is set to "false", the last operation takes precedence. Thus the sequence $zone->ddns_principal_tracking("true"); $zone->override_ddns_principal_security("false"); sets override_ddns_principal_security to "false", and the sequence $zone->override_ddns_principal_security("false"); $zone->ddns_principal_tracking("true"); sets override_ddns_principal_security to "true".

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

Specify 'true' to enable DDNS principal tracking and 'false' to disable it. The default value is 'false'.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#get ddns_principal_tracking value
$ddns_principal_tracking = $zone->ddns_principal_tracking();

#modify ddns_principal_tracking value
$zone->ddns_principal_tracking('true');
```

ddns_principal_group( )

Use this method to set or retrieve the DDNS principal group.

Setting this parameter to a defined value, implicitly sets the override_ddns_principal_security attribute to "true". Setting this parameter to undefined causes the appliance to use the Grid-level DDNS principal security setting and automatically sets the override_ddns_principal_security attribute to "false" if "ddns_restrict_secure" and "ddns_principal_tracking" are also undefined. But if "ddns_restrict_secure" or "ddns_principal_tracking" contains a value, then the override_ddns_principal_security attribute is set to "true".

Note that when ddns_principal_group contains a value and override_ddns_principal_security is set to "false", the last operation takes precedence. Thus the sequence $zone->ddns_principal_group($principal_group); $zone->override_ddns_principal_security("false"); sets override_ddns_principal_security to "false", and the sequence $zone->override_ddns_principal_security("false"); $zone->ddns_principal_group($principal_group); sets override_ddns_principal_security to "true".

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

The valid value is an Infoblox::DNS::DDNS::PrincipalCluster::Group object.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#get ddns_principal_group value
$ddns_principal_group = $zone->ddns_principal_group();

#modify ddns_principal_group value
$zone->ddns_principal_group($principal_group);
```

delegate_to( )

Use this method to set or retrieve the delegate_to flag of a DNS zone.

This provides information for the remote name server that maintains data for the delegated zone. The Infoblox appliance redirects queries

for data for the delegated zone to this remote name server.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

The valid value is an array reference that contains Infoblox::DNS::Nameserver objects.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get delegate_to
my $delegate_to = $zone->delegate_to();
#Modify delegate_to
my $nameserver1 = Infoblox::DNS::Nameserver->new(
       name     => "ns1.del.com",
       ipv4addr => "11.11.11.11",
   );
my $nameserver2 = Infoblox::DNS::Nameserver->new(
       name     => "test",
       ipv6addr => "2006::b671",
   );
$zone->delegate_to([$nameserver1, $nameserver2]);
```

delegated_ttl( )

Use this method to set or retrieve the delegated_ttl value of a delegated DNS zone.

You can specify the Time to Live (TTL) values of auto-generated NS and glue records for a delegated zone. This value is the number of seconds that data is cached.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

A 32-bit integer (range from 0 to 4294967295) that represents the delegated_ttl in seconds. The default value is undefined.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails. When delegated_ttl is modified, override_delegated_ttl will be set to 'true' implicitly.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get delegated_ttl
my $delegated_ttl = $zone->delegated_ttl();
#Modify delegated_ttl
$zone->delegated_ttl("3600");
```

delegation_ns_group( )

Use this method to set or retrieve the delegation name servers group for the Delegation DNS Zone object.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

The valid values are string that contains name of the group or Infoblox::Grid::DNS::Nsgroup::DelegationMember.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Construct external nameserver
my $ns1 = Infoblox::DNS::Nameserver->new(
    name     => 'ns1.domain0.extra',
    ipv4addr => '5.5.5.5',
```

```
);

#Create delegation nameservers group
my $delegation_group = Infoblox::Grid::DNS::DelegationMember->new(
    name               => 'delegation_ns_group_A',
    delegation_servers => [$member],
);

#Submit created delegation nameserver group to session
$session->add($delegation_group);

#Get delegation_ns_group
my $delegation_ns_group = $zone->delegation_ns_group();

#Modify delegation_ns_group
$zone->delegation_ns_group("delegation_ns_group_A");
```

disable( )

Use this method to set or retrieve the disable flag of a DNS zone.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

Specify "true" to set the disable flag or "false" to deactivate/unset it. The default value is "false".

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get disable
my $disable = $zone->disable();
#Modify disable
$zone->disable("true");
```

disable_forwarding( )

Use this method to set or retrieve the disable_forwarding flag of a DNS zone.

Set this flag to indicate that the name servers hosting the zone should not forward queries that end with the domain name of the zone to any configured forwarders.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

Specify "true" to disable forwarding or "false" to enable it. The default value is "false".

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get disable_forwarding
my $disable_forwarding = $zone->disable_forwarding();
#Modify disable_forwarding
$zone->disable_forwarding("true");
```

dnssec_enable_ksk_auto_rollover( )

Use this method to enable automatic rollover for Key-Signing key.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

Specify "true" to enable automatic rollover for Key-signing key or "false" to disable it. The default value is "false".

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get dnssec_enable_ksk_auto_rollover value
my $enable_ksk_auto_rollover = $zone->dnssec_enable_ksk_auto_rollover();
#Modify dnssec_enable_ksk_auto_rollover value
$zone->dnssec_enable_ksk_auto_rollover("true");
```

dnssec_keys( )

Use this method to set or retrieve the list of public keys.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

The valid value is a list of Infoblox::DNS::DNSSecKey objects.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get dnssec_keys value
my $keys = $zone->dnssec_keys();
#Modify dnssec_keys value
$zone->dnssec_keys($keys);
```

dnssec_ksk_email_notification_enabled( )

Use this method to enable or disable email notifications for KSK related events.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

Specify "true" to enable email notification or "false" to disable it. The default value is "false".

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get dnssec_ksk_email_notification_enabled value
my $value = $zone->dnssec_ksk_email_notification_enabled();
#Modify dnssec_ksk_email_notification_enabled value
$zone->dnssec_ksk_email_notification_enabled("true");
```

dnssec_ksk_rollover_date( )

Use this method to set or retrieve the time for the next KSK rollover.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

The valid value is the number of seconds that have elapsed since January 1st, 1970 UTC.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get dnssec_ksk_rollover_date value
my $value = $zone->dnssec_ksk_rollover_date_value();
#Modify dnssec_ksk_rollover_date value
$zone->dnssec_ksk_rollover_date_value($date);
```

dnssec_ksk_rollover_notification_config( )

Use this method to specify the type of events users will be notified.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

The valid values are "NONE", "ALL" and "REQUIRE_MANUAL_INTERVENTION". Default value is undefined.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get dnssec_ksk_rollover_notification_config value
my $value = $zone->dnssec_ksk_rollover_notification_config();
#Modify dnssec_ksk_rollover_notification_config value
$zone->dnssec_ksk_rollover_notification_config("ALL");
```

dnssec_ksk_snmp_notification_enabled( )

Use this method to enable or disable SNMP notifications for KSK related events.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

Specify "true" to enable SNMP notification or "false" to disable it. The default value is "false".

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get dnssec_ksk_snmp_notification_enabled value
my $value = $zone->dnssec_ksk_snmp_notification_enabled();
#Modify dnssec_ksk_snmp_notification_enabled value
$zone->dnssec_ksk_snmp_notification_enabled("true");
```

dns_email( )

Use this method to retrieve the email address (in punycode format) for the administrator who manages DNS zones. This is a read-only attribute.

Parameter

None

Returns

The method returns the attribute value.

Example

```
# Get attribute value
my $value = $zone->dns_email();
```

dns_integrity_check_enable( )

Use this method to set or retrieve the flag used to indicate whether this authoritative zone is monitored for discrepancies or not.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

Specify "true" to enable authoritative zone monitoring for discrepancies, or "false" do disable it. The default value is "false".

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get dns_integrity_check_enable value
my $value = $zone->dns_integrity_check_enable();
#Modify dns_integrity_check_enable value
$zone->dns_integrity_check_enable("true");
```

dns_integrity_frequency( )

Use this method to set or retrieve the frequency (in seconds) for monitoring authoritative zones.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

The valid values are unsigned integers between 900 and 65535. The default value is undefined.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get dns_integrity_frequency value
my $value = $zone->dns_integrity_frequency();
#Modify dns_integrity_frequency value
$zone->dns_integrity_frequency(1000);
```

dns_integrity_member( )

Use this method to set or retrieve the authoritative zone member on which DNS integrity check is performed.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

The valid value is authoritative zone member name in string format. The default value is undefined.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get dns_integrity_member value
my $value = $zone->dns_integrity_member();
#Modify dns_integrity_member value
$zone->dns_integrity_member("test.localdomain");
```

dns_integrity_verbose_logging( )

Use this method to specify whether we should put more details in the authoritative zone syslog for troubleshooting or not.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

Specify "true" to put more details in syslog, or "false" to put less details in syslog. Default value is "false".

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get dns_integrity_verbose_logging value
my $value = $zone->dns_integrity_verbose_logging();
#Modify dns_integrity_verbose_logging value
$zone->dns_integrity_verbose_logging("true");
```

dns_name( )

Use this method to retrieve the name, in punycode format, of a DNS zone. This is a read-only attribute.

Parameter

None

Returns

The method returns the attribute value.

Example

```
# Get attribute value
my $value = $zone->dns_name();
```

dns_soa_mname( )

Use this method to retrieve the primary server name, in punycode format, of a DNS zone. This is a read-only attribute.

Parameter

None

Returns

The method returns the attribute value.

Example

```
# Get attribute value
my $value = $zone->dns_soa_mname();
```

dnssec_ksk_algorithms( )

Use this method to set or retrieve the DNSSEC key-signing key algorithms of the zone.

This property determines which algorithms to use when generating the keys for signing the DNSKEY records.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

Currently there are two major versions of the DNSSEC zone encryption mechanism. The difference between the two is in their implementation of the Next Secure (NSEC) records. The older version, which is referred to as NSEC, is based on RFCs 4033-4035. The newer version, which is referred to as NSEC3, is described in RFC 5155.

The valid value is a list containing Infoblox::DNS::DNSSecKeyAlgorithm objects.

The dnssec_ksk_algorithms parameter inherits the grid setting only when it and all the following DNSSEC parameters are undefined: dnssec_ksk_rollover_interval, dnssec_signature_expiration, dnssec_zsk_algorithms, dnssec_zsk_rollover_interval.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
# Getting algorithm
my $algorithms = $zone->dnssec_ksk_algorithms();

# Modifying algorithm
$gzone->dnssec_ksk_algorithms([$alg1, $alg2]);

#Inheriting the grid DNSSEC settings
$zone->dnssec_ksk_algorithms(undef);
$zone->dnssec_ksk_rollover_interval(undef);
$zone->dnssec_signature_expiration(undef
$zone->dnssec_zsk_algorithms(undef);
$zone->dnssec_zsk_rollover_interval(undef);
```

dnssec_ksk_rollover_interval()

Use this method to set or retrieve the rollover interval of the KSK keys.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

A positive integer that specifies the number of seconds between KSK rollovers. Normally, this parameter should be larger than the ZSK rollover interval. The DNSSEC key-signing key rollover interval (in seconds) must be between 1 day and seconds to Jan 2, 2038.

The dnssec_ksk_rollover_interval parameter inherits the grid setting only when it and all the following DNSSEC parameters are undefined: dnssec_ksk_algorithms, dnssec_signature_expiration, dnssec_zsk_algorithms, dnssec_zsk_rollover_interval. When at least one of the parameters is specified, then the dnssec_ksk_rollover_interval parameter uses the value 31536000 seconds (1 year), unless a different value is explicitly specified.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
# Getting dnssec_ksk_rollover_interval
my $ksk_rollover = $zone->dnssec_ksk_rollover_interval();

# Modifying dnssec_ksk_rollover_interval
$zone->dnssec_ksk_rollover_interval(86400);

#Use the grid setting
$zone->dnssec_ksk_algorithms(undef);
$zone->dnssec_ksk_rollover_interval(undef);
$zone->dnssec_signature_expiration(undef);
$zone->dnssec_zsk_algorithms(undef);
$zone->dnssec_zsk_rollover_interval(undef);
```

dnssec_nsec3_iterations( )

Use this method to define the number of iterations for NSEC3 hashing.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

The valid value are integers between 0 and 65535. Default value is undefined.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get dnssec_nsec3_iterations value
my $iteration = $zone->dnssec_nsec3_iterations();
#Modify dnssec_nsec3_iterations value
$zone->dnssec_nsec3_iterations();
```

dnssec_nsec3_salt_min_length( )

Use this method to define the minimum length for the NSEC3 salt.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

The valid value are integers between 0 and 65535. Default value is undefined.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get dnssec_nsec3_salt_min_length value
my $salt_min_length = $zone->dnssec_nsec3_salt_min_length();
#Modify dnssec_nsec3_salt_min_length value
$zone->dnssec_nsec3_salt_min_length();
```

dnssec_nsec3_salt_max_length( )

Use this method to define the maximum legnth for the NSEC3 salt.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

The valid value are integers between 0 and 65535. Default value is undefined.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get dnssec_nsec3_salt_max_length value
my $salt_max_length = $zone->dnssec_nsec3_salt_max_length();
#Modify dnssec_nsec3_salt_max_length value
$zone->dnssec_nsec3_salt_max_length();
```

dnssec_resign( )

Use this method to initiate the rollover of the key-signing key and the rollover of the zone-signing key for the DNSSEC signed zone.

Parameter

None

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

Example

```
#Do KSK/ZSK rollover
unless($zone->dnssec_resign()){ warn "KSK/ZSK rollover failed!";}
```

dnssec_signature_expiration( )

Use this method to set or retrieve the expiration time for DNSSEC signatures.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

The signature expiration time in seconds. The minimum value is 86400 (1 day), the maximum value is the number of seconds from the time you set it to January 2038.

The dnssec_signature_expiration parameter inherits the grid setting only when it and all the following DNSSEC parameters are undefined: dnssec_ksk_algorithms, dnssec_ksk_rollover_interval, dnssec_zsk_algorithms, dnssec_zsk_rollover_interval. When at least one of the parameters is specified, then the dnssec_signature_expiration parameter uses the value 2592000 seconds (30 days), unless a different value is explicitly specified.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
# Getting dnssec_signature_expiration
my $signature_expiration = $zone->dnssec_signature_expiration();

# Modifying dnssec_signature_expiration
$zone->dnssec_signature_expiration(31536000);

#Use the grid setting
$zone->dnssec_ksk_algorithms(undef);
$zone->dnssec_ksk_rollover_interval(undef);
$zone->dnssec_signature_expiration(undef);
$zone->dnssec_zsk_algorithms(undef);
$zone->dnssec_zsk_rollover_interval(undef);
```

dnssec_signed( )

Use this method to check if the zone is DNSSEC signed and to sign/unsign the zone.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value. Note that for this method it is not necessary to use the call $session->`modify()` on the zone object for the setting to take effect.

Parameter

Set the parameter to "true" to sign the zone, set the parameter to "false" to unsign the zone. The default value of this parameter is "false".

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns "true" if the zone is signed and "false" if it is unsigned.

Example

```
# Checking whether the zone is signed
my $is_signed = $zone->dnssec_signed();

# Signing the zone
$zone->dnssec_signed("true");

# Unsigning the zone
$zone->dnssec_signed("false");
```

dnssec_zsk_algorithms( )

Use this method to set or retrieve the DNSSEC zone-signing key algorithms of the zone.

This property determines which algorithms to use when generating the keys for signing the DNSKEY records.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

Currently there are two major versions of the DNSSEC zone encryption mechanism. The difference between the two is in their implementation of the Next Secure (NSEC) records. The older version, which is referred to as NSEC, is based on RFCs 4033-4035. The newer version, which is referred to as NSEC3, is described in RFC 5155.

The valid value is a list containing Infoblox::DNS::DNSSecKeyAlgorithm objects.

The dnssec_zsk_algorithms parameter inherits the grid setting only when it and all the following DNSSEC parameters are undefined: dnssec_ksk_rollover_interval, dnssec_signature_expiration, dnssec_ksk_algorithms, dnssec_zsk_rollover_interval.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
# Getting algorithm
my $algorithms = $zone->dnssec_zsk_algorithms();

# Modifying algorithm
$gzone->dnssec_zsk_algorithms([$alg1, $alg2]);

#Inheriting the grid DNSSEC settings
$zone->dnssec_ksk_algorithms(undef);
$zone->dnssec_ksk_rollover_interval(undef);
$zone->dnssec_signature_expiration(undef
$zone->dnssec_zsk_algorithms(undef);
$zone->dnssec_zsk_rollover_interval(undef);
```

dnssec_zsk_rollover_date( )

Use this method to set or retrieve the time for the next ZSK rollover.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

The valid value is the number of seconds that have elapsed since January 1st, 1970 UTC.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get dnssec_zsk_rollover_date value
my $value = $zone->dnssec_zsk_rollover_date_value();
#Modify dnssec_zsk_rollover_date value
$zone->dnssec_zsk_rollover_date_value($date);
```

dnssec_zsk_rollover_interval()

Use this method to set or retrieve the zone-signing key rollover interval.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

A positive integer that specifies the number of seconds between ZSK rollovers. Normally, this parameter should be smaller than the KSK rollover interval. The DNSSEC zone-signing key over interval (in seconds) must be between 1 day and the number of seconds from present to Jan 2038.

The dnssec_zsk_rollover_interval parameter inherits the grid setting only when it and all the following DNSSEC parameters are undefined: dnssec_ksk_algorithms, dnssec_ksk_rollover_interval, dnssec_signature_expiration, dnssec_zsk_algorithms. When at least one of the parameters is specified, then the dnssec_zsk_rollover_interval parameter uses the value 2592000 seconds (30 days), unless a different value is explicitly specified.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
# Getting dnssec_zsk_rollover_interval
my $zsk_rollover = $zone->dnssec_zsk_rollover_interval();

# Modifying dnssec_ksk_rollover_interval
$zone->dnssec_zsk_rollover_interval(86400);

#Use the grid setting
$zone->dnssec_ksk_algorithms(undef);
$zone->dnssec_ksk_rollover_interval(undef);
$zone->dnssec_signature_expiration(undef);
$zone->dnssec_zsk_algorithms(undef);
$zone->dnssec_zsk_rollover_interval(undef);
```

dnssec_zsk_rollover_mechanism( )

Use this method to set or retrive the ZSK rollover mechanism type.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

The valid values are "DOUBLE_SING" and "PRE_PUBLISH". Default value is undefined.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get dnssec_zsk_rollover_mechanism value
my $value = $zone->dnssec_zsk_rollover_mechanism();
#Modify dnssec_zsk_rollover_mechanism value
$zone->dnssec_zsk_rollover_mechanism("DOUBLE_SIGN");
```

do_host_abstraction( )

Use this method to set the do_host_abstraction property used in zone import operations. This is a write-only attribute.

Set this flag to indicate that Infoblox host records will be automatically created from A records when the zone data is imported.

Parameter

Specify "true" to trigger host creation or "false" to disable it. The default value is "false".

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

Example

```
#Modify do_host_abstraction
$zone->do_host_abstraction("true");
```

enable_ad_servers( )

Use this method to set or retrieve the flag that allows GSS-TSIG unsigned updates from AD domain controllers.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

Specify "true" to set the flag or "false" to deactivate/unset it. The default value is "false".

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
# Get enable_ad_servers.
my $enable_ad_servers = $zone->enable_ad_servers();
# Modify enable_ad_servers.
$zone->enable_ad_servers("true");
```

email( )

Use this method to set or retrieve the e-mail address of an administrator of a DNS zone.

This setting overrides grid-level e-mail settings.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

The attribute value can be in unicode format.

Parameter

The e-mail address of an administrator.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get email
my $email = $zone->email();
#Modify email
$zone->email("admin\@infoblox.com");
```

execute_dns_parent_check( )

Use this method to perform an one-time DNS integrity check for the selected zone.

Parameter

None

Returns

Returns "true" value upon the successful completion, and returns fail string otherwise.

Example

```
#perform one-time check for zone
$zone->execute_dns_parent_check();
```

extattrs( )

Use this method to set or retrieve the extensible attributes associated with a DNS zone object.

Parameter

Valid value is a hash reference containing the names of extensible attributes and their associated values (Infoblox::Grid::Extattr objects).

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get extattrs
my $ref_extattrs = $zone->extattrs();
#Modify extattrs
$zone->extattrs({ 'Site' => $extattr1, 'Administrator' => $extattr2 });
```

extensible_attributes( )

Use this method to set or retrieve the extensible attributes associated with a DNS zone.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

For valid values for extensible attributes, see Infoblox::Grid::ExtensibleAttributeDef/Extensible Attribute Values.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get extensible attributes
my $ref_extensible_attributes = $zone->extensible_attributes();
#Modify extensible attributes
$zone->extensible_attributes({ 'Site' => 'Santa Clara', 'Administrator' => [ 'Peter', 'Tom' ] });
```

fireeye_rule_mapping( )

Use this method to set or retrieve the FireEye rule mapping.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Returns

An the Infoblox::DNS::FireEye::RuleMapping manpage object.

Example

```
#Get fireeye_rule_mapping
my $fireeye_rule_mapping = $zone->fireeye_rule_mapping();
#Modify fireeye_rule_mapping
$zone->fireeye_rule_mapping($rm);
```

forward_external_ns_group( )

Use this method to set or retrieve the external nameservers group name for forward DNS zone.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

The valid value is a desired external name servers group name in a string format.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Construct external nameserver
my $ns1 = Infoblox::DNS::Nameserver->new(
    name     => 'ns1.domain0.extra',
    ipv4addr => '5.5.5.5',
);

#Construct external nameservers group
my $ext = Infoblox::DNS::Nsgroup::ForwardStubServer->new(
    name             => 'extsrvg1',
    external_servers => [$ns1],
);

#Submit created forward_external nameservers group to session
$session->add($ext);

#Get forward_external_ns_group value
my $forward_external_ns_group = $zone->forward_external_ns_group();

#Modify forward_external_ns_group value
$zone->forward_external_ns_group('extsrvg1');
```

forward_ns_group( )

Use this method to set or retrieve the name servers group for the Forward DNS Zone object.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

The valid values are string that contains name of the group or Infoblox::Grid::DNS::Nsgroup::ForwardingMember.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Construct external nameserver
my $ns1 = Infoblox::DNS::Nameserver->new(
    name     => 'ns1.domain0.extra',
    ipv4addr => '5.5.5.5',
);

#Construct dns member
my $member = Infoblox::DNS::Member->new(
    ipv4addr   => '192.168.1.2',
    forward_to => [$ns1],
);

#Create forward nameservers group
my $forward_group = Infoblox::Grid::DNS::ForwardingMember->new(
    name               => 'forward_ns_group_A',
    forwarding_servers => [$member],
);

#Submit created forward nameserver group to session
$session->add($forward_group);

#Get forward_ns_group
my $forward_ns_group = $zone->forward_ns_group();

#Modify forward_ns_group
$zone->forward_ns_group("forward_ns_group_A");
```

forward_only( )

Use this method to set or retrieve the forward_only flag of a DNS forward zone. When you set this flag, the appliance sends queries to forwarders only, and not to other internal or Internet root servers.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

Specify "true" to send queries to forwarders only. Specify "false" to disable it. Default is "false."

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Getting forward_only
my $forward_only = $fwd_zone->forward_only( );
#Modifying forward_only
$fwd_zone->forward_only("true");
```

forward_to( )

Use this method to set or retrieve the forward_to flag of a DNS zone.

This method provides information for the remote name server to which you want the Infoblox appliance to forward queries for a specified domain name.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

The valid value is an array reference that contains Infoblox::DNS::Member objects for grid secondary servers and Infoblox::DNS::Nameserver objects for external secondary servers. The default value is undefined.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get forward_to
my $forward_to = $zone->forward_to();
#Modify forward_to
my $member = Infoblox::DNS::Member->new(
        name     => "hostname.com",
        ipv4addr => "3.0.0.2",
   );

my $nameserver = Infoblox::DNS::Nameserver->new(
        name     => "ns1.forwd.com",
        ipv4addr => "11.11.11.11",
   );

$zone->forward_to([$member, $nameserver]);
```

host_name_restriction_policy( )

Use this method to set or retrieve the host_name_restriction_policy flag of a DNS zone.

This setting overrides the member-level host name restriction policy settings.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

String with the name of predefined record policies or user-defined record policies. The default value is undefined, which is to follow the Record Name policy specified in Infoblox::Grid::Member::DNS level.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get host name restriction policy
my $host_name_restriction_policy = $zone->host_name_restriction_policy();
#Modify host name restriction policy
$zone->host_name_restriction_policy("Allow Any");
```

import_from( )

Use this method to set or retrieve the import_from flag of a DNS zone.

This method specifies the IP address of the Infoblox appliance from which zone data is imported. This is only valid for authoritative zones. Setting this address to '255.255.255.255' and do_host_abstraction to 'true' will create Host records from A records in this zone without importing zone data.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

An IPv4 address is a 32-bit number with the basic format representation of `X.X.X.X` where x refers to decimal digits (8 bits), for example: `128.6.4.7`. The default value is undefined.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Modify import_from
$zone->import_from("192.168.1.2");
```

initiate_ksk_rollover( )

Use this method to initiate the rollover of the key-signing key for the DNSSEC signed zone.

Parameter

None

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

Example

```
#Do KSK rollover
unless($zone->initiate_ksk_rollover()){ warn "KSK rollover failed!";}
```

initiate_zsk_rollover( )

Use this method to initiate the rollover of the Zone-signing key for the DNSSEC signed zone.

Parameter

None

Returns

Returns true value upon the successful completion, and returns false value otherwise.

Example

```
#Do ZSK rollover
unless($zone->initiate_zsk_rollover()){ warn "ZSK rollover failed!";}
```

is_default( )

Use this method to set or retrieve the is_default flag of a DNS zone.

This flag is used to determine the default zone for the zone associations of a DHCP network. This is set or returned only for zone objects set in the zone-associations method of the DHCP network.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

Specify "true" to set the is_default flag or "false" to deactivate/unset it. The default value is "false".

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get is_default
my $is_default = $zone->is_default();
```

```
#Modify is_default
$zone->is_default("true");
```

is_multimaster( )

Use this method to set or retrieve the is_multimaster flag.

Parameter

Specify "true" to set the is_multimaster flag or "false" to deactivate/unset it. The default value is "false".

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
# Get the attribute value
my $attr = $zone->is_multimaster();
# Modify the attribute value
$zone->is_multimaster('true');
```

last_queried( )

Use this method to retrieve the time the zone was last queried on.

Returns

The method returns the attribute value, the number of seconds that have elapsed since January 1st, 1970 UTC.

Example

```
#Get last_queried
my $last_queried = $zone->last_queried();
```

rpz_drop_ip_rule_enabled( )

Use this method to set or retreive the flag that indicates whether the DNS server ignores RPZ-IP triggers with large prefix lengths or not.

Setting this method to a defined value implicitly sets the override_rpz_drop_ip_rule method to "true". Setting the parameter to undefined if rpz_drop_ip_rule_min_prefix_length_ipv4 and rpz_drop_ip_rule_min_prefix_length_ipv6 are undefined causes the appliance to use the Grid default and automatically resets the override_rpz_drop_ip_rule attribute to "false".

Note that when rpz_drop_ip_rule_enabled is set to a defined value and override_rpz_drop_ip_rule is set to "false", the last operation takes precedence. Thus the sequence $object->rpz_drop_ip_rule_enabled("true"); $object->override_rpz_drop_ip_rule("false"); will set override_rpz_drop_ip_rule to "false", and the sequence $object->override_rpz_drop_ip_rule("false"); $object->rpz_drop_ip_rule_enabled("true"); will result in override_rpz_drop_ip_rule="true".

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

Specify 'true' to enable ignoring of RPZ-IP triggers with large prefix lengths or 'false' to disable it. The default value is 'false'.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
# Get attribute value
my $value = $zone->rpz_drop_ip_rule_enabled();
# Modify attribute value
$zone->rpz_drop_ip_rule_enabled("true");
```

rpz_drop_ip_rule_min_prefix_length_ipv4( )

Use this method to set or retreive minimum prefix length for IPv4 RPZ-IP triggers.

Setting this method to a defined value implicitly sets the override_rpz_drop_ip_rule method to "true". Setting the parameter to undefined if rpz_drop_ip_rule_enabled and rpz_drop_ip_rule_min_prefix_length_ipv6 are undefined causes the appliance to use the Grid default and automatically resets the override_rpz_drop_ip_rule attribute to "false".

Note that when rpz_drop_ip_rule_min_prefix_length_ipv4 is set to a defined value and override_rpz_drop_ip_rule is set to "false", the last

operation takes precedence. Thus the sequence $object->rpz_drop_ip_rule_min_prefix_length_ipv4("20"); $object->override_rpz_drop_ip_rule("false"); will set override_rpz_drop_ip_rule to "false", and the sequence $object->override_rpz_drop_ip_rule("false"); $object->rpz_drop_ip_rule_min_prefix_length_ipv4("20"); will result in override_rpz_drop_ip_rule="true".

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

The valid value is an unsigned integer between 1 and 31 that represents prefix length (in bits) for IPv4 RPZ-IP triggers.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
# Get attribute value
my $value = $zone->rpz_drop_ip_rule_min_prefix_length_ipv4();
# Modify attribute value
$zone->rpz_drop_ip_rule_min_prefix_length_ipv4("16");
```

rpz_drop_ip_rule_min_prefix_length_ipv6( )

Use this method to set or retreive minimum prefix length for IPv6 RPZ-IP triggers.

Setting this method to a defined value implicitly sets the override_rpz_drop_ip_rule method to "true". Setting the parameter to undefined if rpz_drop_ip_rule_enabled and rpz_drop_ip_rule_min_prefix_length_ipv4 are undefined causes the appliance to use the Grid default and automatically resets the override_rpz_drop_ip_rule attribute to "false".

Note that when rpz_drop_ip_rule_min_prefix_length_ipv6 is set to a defined value and override_rpz_drop_ip_rule is set to "false", the last operation takes precedence. Thus the sequence $object->rpz_drop_ip_rule_min_prefix_length_ipv6("20"); $object->override_rpz_drop_ip_rule("false"); will set override_rpz_drop_ip_rule to "false", and the sequence $object->override_rpz_drop_ip_rule("false"); $object->rpz_drop_ip_rule_min_prefix_length_ipv6("20"); will result in override_rpz_drop_ip_rule="true".

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

The valid value is an unsigned integer between 1 and 127 that represents prefix length (in bits) for IPv6 RPZ-IP triggers.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
# Get attribute value
my $value = $zone->rpz_drop_ip_rule_min_prefix_length_ipv6();
# Modify attribute value
$zone->rpz_drop_ip_rule_min_prefix_length_ipv6("120");
```

rpz_policy( )

Use this method to set or retrieve the override policy for the Response Policy Zone.

This property determines the generic override rule for all records in this Response Policy Zone.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

The valid value is one of "GIVEN", "PASSTHRU", "NXDOMAIN", "NODATA", "SUBSTITUTE", "DISABLED". The default value is undefined.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
# Getting rpz_policy
```

```
my $rpz_policy = $zone->rpz_policy();

# Modifying rpz_policy
$zone->rpz_policy("PASSTHRU");
```

restart_if_needed( )

Use this method to set the flag that indicates whether DHCP service should be restarted on a member that is serving DHCP for the changed object referenced in a cloud API request.

Parameter

Specify true to restart the DHCP service.

Returns

None

Example

```
# Modify
$zone->restart_if_needed('true');
```

rpz_type( )

Use this method to set or retrieve the type of the Response Policy Zone.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

The valid value is one of "LOCAL", "FIREEYE", "FEED". The default value is undefined.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
# Getting rpz_type
my $rpz_type = $zone->rpz_type();

# Modifying rpz_type
$zone->rpz_type("LOCAL");
```

rpz_severity( )

Use this method to set or retrieve the type of severity for Response Policy Zone.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

The valid value is CRITICAL, MAJOR, WARNING, or INFORMATIONAL. The default value is undefined.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
# Getting rpz_severity
my $rpz_severity = $zone->rpz_severity();

# Modifying rpz_type
$zone->rpz_severity("MAJOR");
```

substitute_name( )

Use this method to retrieve the substitution name when the RPZ policy is set to "SUBSTITUTE".

You must specify a parameter if rpz_policy is set to "SUBSTITUTE".

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
# Getting algorithm
my $substitution_name = $zone->substitute_name();

# Modifying algorithm
$zone->substitute_name("something");
```

rpz_priority( )

Use this method to retrieve the priority of zones among all Response Policy Zones.

This attribute is readonly and you can use an Infoblox::DNS::OrderedResponsePolicyZones object to reorder Response Policy Zones in a DNS view.

Returns

This method returns the attribute value in integer format. "1" indicates the highest priority.

Example

```
#Get rpz_priority
my $rpz_priority = $zone->rpz_priority();
```

rpz_last_updated_time( )

Use this method to retrieve the date and time the Response Policy Zone was last updated.

Returns

This method returns the attribute value in number of seconds that have elapsed since January 1st, 1970 UTC.

Example

```
#Get rpz_last_updated_time
my $last_updated_time = $zone->rpz_last_updated_time();
```

rpz_severity( )

Use this method to set or retrieve the severity of this response policy zone.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

The valid value is one of the following: CRITICAL, MAJOR, WARNING or INFORMATIONAL.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get rpz_severity
my $severity = $zone->rpz_severity();

# Modify rpz_severity
$zone->rpz_severity("MAJOR");
```

is_feed_zone( )

Use this method to retrieve the boolena value that indicates whether the Response Policy Zone is a "feed" zone or not.

Returns

This method returns the attribute value. "True" means the response policy zone is a feed zone that has an assigned external primary.

Example

```
#Get is_feed_zone
my $is_feed_zone = $zone->is_feed_zone();
```

locked( )

Use this method to set or retrieve the locked flag of a DNS zone.

If you enable this flag other administrators cannot make changes that would "collide". This is for administration purposes only. The zone will continue to serve DNS data even when it is locked.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

Specify "true" to set the locked flag or "false" to deactivate/unset it. The default value is "false".

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get locked
my $locked = $zone->locked();
#Modify locked
$zone->locked("true");
```

members( )

Use this method to set or retrieve the members flag of a DNS forward zone.

This method specifies the grid member that serves the forward zone.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

The valid value is an array reference that contains Infoblox::DNS::Member object(s). The default value is undefined.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get members
my $members = $zone->members();
#Modify members
my $memberns1 = Infoblox::DNS::Member->new(
      name => "member1.forwd.com",
  );

my $memberns2 = Infoblox::DNS::Member->new(
      name => "member2.forwd.com",
  );

$zone->members([$memberns1,$memberns2]);
```

member_soa_serials( )

Use this method to retrieve per-member SOA serial information. This is a read-only attribute.

Parameter

None.

Returns

The method returns the attribute value that contains Infoblox::DNS::Member::SoaSerial object(s).

Example

```
# Get the attribute value
my $member_soa_serials = $zone->member_soa_serials();
```

member_soa_mnames( )

Use this method to set or retrieve per-member SOA MNAME information.

Parameter

The valid value is an array reference that contains Infoblox::DNS::Member::SoaMname objects.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
# Get the attribute value
my $member_soa_mnames = $zone->member_soa_mnames();
# Modify the attribute value
$zone->member_soa_mnames([$soa_mname]);
```

ms_dc_ns_record_creation( )

Use this method to set or retrieve the list of Active Directory Domain Controllers which are allowed to create NS Record for the DNS Zone.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameters

The valid value is an array of Infoblox::Grid::MSServer::DCNSRecordCreation objects.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get ms_dc_ns_record_creation
my $ms_dc_ns_record_creation = $zone->ms_dc_ns_record_creation();

#Modify ms_dc_ns_record_creation
$zone->ms_dc_ns_record_creation([$msdcns1, $msdcns1]);
```

name( )

Use this method to set or retrieve the name of a DNS zone.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

The attribute value can be in unicode format.

Parameter

Zone name. For a reverse zone, this is in "address/cidr" format. For other zones, this is in FQDN (Fully Qualified Domain Name) format. A zone name can have a maximum of 256 characters.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get name
my $name = $zone->name();
```

network_associations( )

Use this method to retrieve the associated network/network container information. This is a read-only method.

Parameter

No parameter need to be specified

Returns

This method returns an array reference that contains Infoblox::DHCP::Network, Infoblox::DHCP::NetworkContainer, Infoblox::DHCP::IPv6Network, and Infoblox::DHCP::IPv6NetworkContainer objects.

Examples

```
#Getting network associations
my $network_associations = $zone->network_associations();
```

next_secure_type( )

Use this method to set or retrieve resource record type.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

Valid value is 'NSEC' or 'NSEC3'.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get the next_secure_type value
my $next_secure_type = $zone->next_secure_type();
#Modify the next_secure_type value
$zone->next_secure_type('NSEC3');
```

notify_delay( )

Use this method to specify with how many seconds of delay the notify messages are sent to secondaries.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

Valid values are unsigned integer between 5 and 86400, inclusive. Default value is undefined.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Getting notify_delay
my $notify_delay = $zone->notify_delay( );
#Modifying notify_delay
$zone->notify_delay(5);
```

ms_ad_integrated()

Use this method to set or retrieve the Active Directory-integrated flag of a zone. This attribute is valid only when `ms_managed()` is "Stub", "Auth_primary", or "Auth_both".

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

Specify "true" to set the Active Directory-integrated flag or "false" to unset it. The default value is "false".

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get ms_ad_integrated
my $ms_ad_integrated = $zone->ms_ad_integrated();
#Modify ms_ad_integrated
$zone->ms_ad_integrated("true");
```

ms_managed()

Use this method to retrieve the flag that indicates whether the zone is assigned to a Microsoft DNS server. This flag returns the

authoritative name server type of the Microsoft DNS server. The value can be:

"None" if the zone is not assigned to any Microsoft DNS server
"Stub" if the zone is assigned to a Microsoft DNS server as a stub zone
"Auth_primary" if only the primary server of the zone is a Microsoft DNS server
"Auth_secondary" if only the secondary server of the zone is a Microsoft DNS server
"Auth_both" if both the primary and secondary servers of the zone are Microsoft DNS servers

Returns

The method returns the attribute value.

Example

```
#Get ms_managed
my $ms_managed = $zone->ms_managed();
```

ms_read_only()

Use this method to retrieve the flag that indicates if a grid member manages the zone served by a Microsoft DNS server in read-only mode. This flag returns "true" when a grid member manages the zone in read-only mode. Otherwise, it returns "false".

When the zone has the `ms_read_only()` flag set to "true", no change can be made to this zone.

Returns

The method returns the attribute value.

Example

```
#Get ms_read_only
my $ms_read_only = $zone->ms_read_only();
```

ms_ddns_mode()

Use this method to set or retrieve the flag that indicates whether an Active Directory-integrated zone with a Microsoft DNS server as primary allows dynamic updates. This flag returns "Secure" if the zone allows secure updates only, "None" if the zone forbids dynamic updates, or "Any" if the zone accepts both secure and nonsecure updates. This flag is valid only if `ms_managed()` is either "Auth_primary" or "Auth_both". If the flag `ms_ad_integrated()` is false, the value "Secure" is not allowed.

Parameter

Specify "Secure", "None", or "Any" to control the dynamic updates that an Active Directory-integrated zone is allowed to accept. The default value is undefined.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get ms_ddns_mode
my $ms_ddns_mode = $zone->ms_ddns_mode();
#Modify ms_ddns_mode
$zone->ms_ddns_mode("None");
```

ms_allow_transfer()

Use this method to set or retrieve the list of DNS clients that are allowed to perform zone transfers from a Microsoft DNS server.

This setting applies only to zones with Microsoft DNS servers that are either primary or secondary servers. This setting does not inherit any value from the grid or from any member that defines an `allow_transfer()` value. This setting does not apply to any grid member. Use the `allow_transfer()` attribute to control allowed clients on grid members.

Valid values are:

An array that contains a list of IPv4 addresses
The keyword "Any" to allow any client
The keyword "NSOnly" to allow only the nameservers listed in the zone
The keyword "None" to deny all zone transfer requests

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

Specify one of the allowed values to identify the DNS clients that are allowed to perform zone transfers from a Microsoft DNS

server. The default is "NSOnly".

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get ms_allow_transfer
my $ms_allow_transfer = $zone->ms_allow_transfer();
#Modify ms_allow_transfer
$zone->ms_allow_transfer("Any");
```

ms_sync_disable( )

Use this method to disable Microsoft Server synchronization for this zone.

Parameter

Set the parameter to "true" to disable Microsoft Server synchronization for this zone. Set the parameter to "false" to enable Microsoft Server synchronization for this zone.

If you did not specify a parameter, the method returns the attribute value.

Return

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get ms_sync_disable value
my $ms_sync_disable = $zone->ms_sync_disable();

#Set ms_sync_disable value
$zone->ms_sync_disable("true");
```

ns_group( )

Use this method to set or retrieve the name servers group for the DNS Authoritative and Response Policy Zone objects.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

The valid values are string that contains name of the group or Infoblox::Grid::DNS::Nsgroup.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get ns_group
my $ns_group = $zone->ns_group();

#Modify ns_group
$zone->ns_group("ns_group_A");
```

override_copy_xfer_to_notify( )

The override_copy_xfer_to_notify attribute controls whether the copy_xfer_to_notify method values of the DNS zone are used, instead of the member default.

The override_copy_xfer_to_notify attribute can be specified explicitly. It is also set implicitly when copy_xfer_to_notify is set to a defined value.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

Set the parameter to "true" to override the member-level setting for copy_xfer_to_notify. Set the parameter to "false" to inherit the member-level setting for copy_xfer_to_notify.

The default value of this parameter is "false".

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Getting override_copy_xfer_to_notify
my $override_copy_xfer_to_notify = $zone->override_copy_xfer_to_notify( );
#Modifying override_copy_xfer_to_notify
$zone->override_copy_xfer_to_notify("true");
```

override_ddns_force_creation_timestamp_update( )

The override_ddns_force_creation_timestamp_update attribute controls whether the ddns_force_creation_timestamp_update method value of the DNS View is used, instead of the Grid default.

The override_ddns_force_creation_timestamp_update attribute can be specified explicitly. It is also set implicitly when ddns_force_creation_timestamp_update is set to a defined value.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

Set the parameter to "true" to override the Grid-level setting for ddns_force_creation_timestamp_update. Set the parameter to "false" to inherit the Grid-level setting for ddns_force_creation_timestamp_update.

The default value of this parameter is "false".

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get override_ddns_force_creation_timestamp_update value
my $value = $object->override_ddns_force_creation_timestamp_update();

#Modify override_ddns_force_creation_timestamp_update value
$object->override_ddns_force_creation_timestamp_update("true");
```

override_ddns_patterns_restriction( )

The override_ddns_patterns_restriction attribute controls whether the ddns_restrict_patterns_list and ddns_restrict_patterns methods values of the DNS Zone are used, instead of the Grid default.

The override_ddns_patterns_restriction attribute can be specified explicitly. It is also set implicitly when ddns_restrict_patterns_list or ddns_restrict_patterns is set to a defined value.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

Set the parameter to "true" to override the Grid-level setting for ddns_restrict_patterns_list and ddns_restrict_patterns. Set the parameter to "false" to inherit the Grid-level setting for ddns_restrict_patterns_list and ddns_restrict_patterns.

The default value of this parameter is "false".

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get override_ddns_patterns_restriction
my $override_ddns_patterns_restriction = $zone->override_ddns_patterns_restriction();
#Modify override_ddns_patterns_restriction
$zone->override_ddns_patterns_restriction("true");
```

override_ddns_restrict_static( )

The override_ddns_restrict_static attribute controls whether the ddns_restrict_static method value of the DNS Zone is used, instead of the

Grid default.

The override_ddns_restrict_static attribute can be specified explicitly. It is also set implicitly when ddns_restrict_static is set to a defined value.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

Set the parameter to "true" to override the Grid-level setting for ddns_restrict_static. Set the parameter to "false" to inherit the Grid-level setting for ddns_restrict_static.

The default value of this parameter is "false".

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get override_ddns_restrict_static
my $override_ddns_restrict_static = $zone->override_ddns_restrict_static();
#Modify override_ddns_restrict_static
$zone->override_ddns_restrict_static("true");
```

override_ddns_restrict_protected( )

The override_ddns_restrict_protected attribute controls whether the ddns_restrict_protected method value of the DNS Zone is used, instead of the Grid default.

The override_ddns_restrict_protected attribute can be specified explicitly. It is also set implicitly when ddns_restrict_protected is set to a defined value.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

Set the parameter to "true" to override the Grid-level setting for ddns_restrict_protected. Set the parameter to "false" to inherit the Grid-level setting for ddns_restrict_protected.

The default value of this parameter is "false".

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get override_ddns_restrict_protected
my $override_ddns_restrict_protected = $zone->override_ddns_restrict_protected();
#Modify override_ddns_restrict_protected
$zone->override_ddns_restrict_protected("true");
```

override_ddns_principal_security( )

The override_ddns_principal_security attribute controls whether the ddns_restrict_secure, ddns_principal_tracking and ddns_principal_group methods values of the DNS Zone are used, instead of the Grid default.

The override_ddns_principal_security attribute can be specified explicitly. It is also set implicitly when ddns_restrict_secure, ddns_principal_tracking or ddns_principal_group is set to a defined value.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

Set the parameter to "true" to override the Grid-level setting for ddns_restrict_secure, ddns_principal_tracking and ddns_principal_group. Set the parameter to "false" to inherit the Grid-level setting for ddns_restrict_secure, ddns_principal_tracking and ddns_principal_group.

The default value of this parameter is "false".

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get override_ddns_principal_security
my $override_ddns_principal_security = $zone->override_ddns_principal_security();
#Modify override_ddns_principal_security
$zone->override_ddns_principal_security("true");
```

override_delegated_ttl( )

Use this method to set or retrieve the override_delegated_ttl flag of a DNS zone.

This flag indicates whether to override the TTL values of auto-generated NS and GLUE records for a delegated zone. If you set it to "true", then you must specify the 'delegated_ttl' value.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

Specify "true" to override the TTL values or "false" to have these records inherit the TTL values from their parent zone. The default value is "false".

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get override_delegated_ttl
my $override_delegated_ttl = $zone->override_delegated_ttl();
#Modify override_delegated_ttl
$zone->override_delegated_ttl("true");
```

override_grid_zone_timer( )

Use this method to set or retrieve the override_grid_zone_timer flag of a DNS zone.

This flag indicates whether to override the grid's zone timer settings. If you set it to "true", then you must specify the following properties: soa_refresh, soa_retry, soa_expire, soa_default_ttl, soa_negative_ttl.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

Specify "true" to override grid zone timer settings or "false" to use the grid zone timer settings. The default value is "false".

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get override_grid_zone_timer
my $override_grid_zone_timer = $zone->override_grid_zone_timer();
#Modify override_grid_zone_timer
$zone->override_grid_zone_timer("true");
```

override_grid_email( )

Use this method to set or retrieve the override_grid_email flag of a DNS zone.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

Specify "true" to override grid e-mail settings or "false" to use the grid e-mail settings. The default value is "false".

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get override_grid_email
my $override_grid_email = $zone->override_grid_email();
#Modify override_grid_email
$zone->override_grid_email("false");
```

override_scavenging_settings( )

The override_scavenging_settings attribute controls whether the scavenging_settings method values of the DNS Zone are used, instead of the Grid default.

The override_scavenging_settings attribute can be specified explicitly. It is also set implicitly when scavenging_settings is set to a defined value.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

Set the parameter to "true" to override the Grid-level setting for scavenging_settings. Set the parameter to "false" to inherit the Grid-level setting for scavenging_settings.

The default value of this parameter is "false".

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get override_scavenging_settings value
my $override_scavenging_settings = $object->override_scavenging_settings( );

#Modify override_scavenging_settings value
$object->override_scavenging_settings("true");
```

override_rpz_drop_ip_rule( )

The override_rpz_drop_ip_rule attribute controls whether the rpz_drop_ip_rule_enabled, rpz_drop_ip_rule_min_prefix_length_ipv4 and rpz_drop_ip_rule_min_prefix_length_ipv6 method values of the member are used, instead of the Grid default.

The override_rpz_drop_ip_rule attribute can be specified explicitly. It is also set implicitly when one of rpz_drop_ip_rule_enabled, rpz_drop_ip_rule_min_prefix_length_ipv4 or rpz_drop_ip_rule_min_prefix_length_ipv6 methods is set to a defined value.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

Set the parameter to "true" to override the Grid-level setting for rpz_drop_ip_rule_enabled, rpz_drop_ip_rule_min_prefix_length_ipv4 and rpz_drop_ip_rule_min_prefix_length_ipv6 methods. Set the parameter to "false" to inherit the Grid-level setting for rpz_drop_ip_rule_enabled, rpz_drop_ip_rule_min_prefix_length_ipv4 and rpz_drop_ip_rule_min_prefix_length_ipv6 methods.

The default value of this parameter is "false".

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Getting override_rpz_drop_ip_rule
my $override_rpz_drop_ip_rule = $zone->override_rpz_drop_ip_rule( );
#Modifying override_rpz_drop_ip_rule
$zone->override_rpz_drop_ip_rule("true");
```

override_serial_number( )

Use this method to set the override_serial_number flag of a DNS zone.

If set this flag to 'true', soa_serial_number can be set. This value behaves somewhat unusually in that the server always resets it to undef in the response, even if it is true in the request.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

Specify "true" to override serial number. The default value is undef

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get override_serial_number
my $override_serial_number = $zone->override_serial_number();
#Modify override_serial_number
$zone->override_serial_number("true");
```

prefix( )

Use this method to set or retrieve the prefix flag of a DNS zone.

Use this method only when the netmask is greater than 24 bits; that is, for a mask between 25 and 31 bits. Enter a prefix, such as the name of the allocated address block. The prefix can be alphanumeric characters; for example: 128/26 , 128-189 , or sub-B . This is only valid for authoritative reverse zones.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

This prefix can be anything such as the address range (examples: 0-127, 0/127) or descriptions (examples: first-network, customer1).A prefix can have a maximum of 63 bytes. The default value is undefined.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get prefix
my $prefix = $zone->prefix();
#Modify prefix
$zone->prefix("first-network");
```

multiple_primaries( )

Use this method to set or retrieve the primary flag of a DNS zone.

This method specifies Grid primary servers, external primary servers, or Microsoft DNS servers that are primary servers for the zone. Although a zone typically has just one primary name server, you can specify up to ten independent servers for a single zone.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

The valid value is an array reference that contains Infoblox::DNS::Member objects for Grid primary servers, or Infoblox::DNS::Nameserver objects for external primary servers, or Infoblox::DNS::MSServer objects for Microsoft DNS servers. The default value is undefined.

To set two and more Infoblox::DNS::Member objects you should set the is_multimaster flag to "true".

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get multiple_primaries
my $multiple_primaries = $zone->multiple_primaries();
#Modify multiple_primaries
my $memberns1 = Infoblox::DNS::Member->new(
       name     => "member1.server.com",
       ipv4addr => "1.1.1.1",
   );

$zone->multiple_primaries([$memberns1]);
```

primary_shared_with_ms_parent_delegation( )

Use this method to retrieve the primary_shared_with_ms_parent_delegation flag. If the zone is a subzone and the primary server of the parent zone is a Microsoft server, it will be set to "true" if the FQDN and IP address in the delegation zone's NS record matches the FQDN and IP address of the authoritative name server of the zone.

Parameter

None.

Returns

The method returns the attribute value.

Example

```
#Get primary_shared_with_ms_parent_delegation
my $primary_shared_with_ms_parent_delegation = $member->primary_shared_with_ms_parent_delegation();
```

scavenging_settings( )

Use this method to set or retrieve the zone-level scavenging settings.

Setting this parameter implicitle sets the override_scavenging_settings attribute to "true". Setting the parameter to undefined causes the appliance to use the Grid-level settings and automatically sets the override_scavenging_settings attribute to "false".

Note that when scavenging_settings is set and override_scavenging_settings is set to "false", the last operation takes precedence. Thus the sequence $object->scavenging_settings($recl_settings); $object->override_scavenging_settings("false"); sets override_scavenging_settings to "false", and the sequence $object->override_scavenging_settings("false"); $object->scavenging_settings($recl_settings); sets override_scavenging_settings to "true".

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

The valid value is an Infoblox::Grid::DNS::ScavengingSetting object.

Parameter

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get scavenging_settings value
my $scavenging_settings = $object->scavenging_settings();

#Modify scavenging_settings value
$object->scavenging_settings($scavenging_settings);
```

remove_sub_zones( )

Use this method to set or retrieve the remove_sub_zones flag of a DNS zone.

You can remove the sub-zones and resource records of the selected zone.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

Specify "true" to remove sub zones or "false" to not to remove sub zones. The default value is undefined, which is to remove all subzones.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get remove_sub_zones
my $remove_sub_zones = $zone->remove_sub_zones();
#Modify remove_sub_zones
$zone->remove_sub_zones("true");
```

secondaries( )

Use this method to set or retrieve the secondaries flag of a DNS zone.

You can specify grid secondary servers, external secondary servers, or Microsoft DNS servers that are secondary servers for the zone.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

The valid value is an array reference that contains Infoblox::DNS::Member objects for grid secondary servers, Infoblox::DNS::Nameserver objects for external secondary servers, and Infoblox::DNS::MSServer objects for Microsoft DNS servers. The default value is empty.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get secondaries
my $secondaries = $zone->secondaries();
#Modify secondaries
my $memberns1 = Infoblox::DNS::Member->new(
        name     => "member1.server.com",
        ipv4addr => "1.1.1.1",
    );

my $nameserver1 = Infoblox::DNS::Nameserver->new(
        name     => "ns1.exter.com",
        ipv4addr => "11.11.11.11",
    );

$zone->secondaries([$memberns1, $nameserver1]);
```

resource_record_monitored_since( )

Use this method to retrieve the time data collection for "DNS Resource Records Last Queried" was turned on for the zone.

Returns

The method returns the attribute value, the number of seconds that have elapsed since January 1st, 1970 UTC.

Example

```
#Get resource_record_monitored_since
my $resource_record_monitored_since = $zone->resource_record_monitored_since();
```

soa_default_ttl( )

Use this method to set or retrieve the soa_default_ttl flag of a DNS zone.

You can set the Time to Live (TTL) value of the SOA record of a zone. This value is the number of seconds that data is cached.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

A 32-bit integer (range from 0 to 4294967295) that represents the soa_default_ttl in seconds. The default value is undefined.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get soa_default_ttl
my $soa_default_ttl = $zone->soa_default_ttl();
#Modify soa_default_ttl
$zone->soa_default_ttl("3600");
```

soa_expire( )

Use this method to set or retrieve the soa_expire flag of a DNS zone.

This setting defines the amount of time, in seconds, after which the secondary server stops giving out answers about the zone because the

zone data is too old to be useful. The default is one week.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

A 32-bit integer (range from 0 to 4294967295) that represents the soa_expire in seconds. The default value is undefined.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get soa_expire
my $soa_expire = $zone->soa_expire();
#Modify soa_expire
$zone->soa_expire("3600");
```

soa_mname( )

Use this method to set or retrieve the soa_mname flag of a DNS zone.

The Infoblox appliance allows you to change the name of the primary server on the SOA record that is automatically created when you initially configure a zone. Use this method to change the name of the primary server on the SOA record. For example, you may want to hide the primary server for a zone. If your device is named dns1.zone.tld, and for security reasons, you want to show a secondary server called dns2.zone.tld as the primary server. To do so, you would go to dns1.zone.tld zone (being the true primary) and change the primary server on the SOA to dns2.zone.tld to hide the true identity of the real primary server.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

The attribute value can be in unicode format.

Parameter

Primary server name in FQDN (Fully Qualified Domain Name) format. The default value is undefined.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get soa_mname
my $soa_mname = $zone->soa_mname();
#Modify soa_mname
$zone->soa_mname("soamname.com");
```

soa_negative_ttl( )

Use this method to set or retrieve the soa_negative_ttl flag of a DNS zone.

The negative Time to Live (TTL) value of the SOA of the zone indicates how long a secondary server can cache data for "Does Not Respond" responses.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

A 32-bit integer (range from 0 to 4294967295) that represents the soa_negative_ttl in seconds. The default value is undefined.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get soa_negative_ttl
my $soa_negative_ttl = $zone->soa_negative_ttl();
#Modify soa_negative_ttl
$zone->soa_negative_ttl("3600");
```

soa_refresh( )

Use this method to set or retrieve the soa_refresh flag of a DNS zone.

This indicates the interval at which a secondary server sends a message to the primary server for a zone to check that its data is current, and retrieve fresh data if it is not.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

A 32-bit integer (range from 0 to 4294967295) that represents the soa_refresh in seconds. The default value is undefined.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get soa_refresh
my $soa_refresh = $zone->soa_refresh();
#Modify soa_refresh
$zone->soa_refresh("1000");
```

soa_retry( )

Use this method to set or retrieve the soa_retry flag of a DNS zone.

This indicates how long a secondary server must wait before attempting to recontact the primary server after a connection failure between the two servers occurs.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

A 32-bit integer (range from 0 to 4294967295) that represents the soa_retry in seconds. The default value is undefined.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get soa_retry
my $soa_retry = $zone->soa_retry();
#Modify soa_retry
$zone->soa_retry("1000");
```

soa_serial_number( )

Use this method to set or retrieve the soa_serial_number flag of a DNS zone.

The serial number in the SOA record incrementally changes every time the record is modified. The Infoblox appliance allows you to change the serial number (in the SOA record) for the primary server so it is higher than the secondary server, thereby ensuring zone transfers come from the primary server (as they should).

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

A 32-bit integer (range from 0 to 4294967295) that represents the soa serial number. The default value is 1.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get soa_serial_number
my $soa_serial_number = $zone->soa_serial_number();
#Modify soa_serial_number
$zone->soa_serial_number("222222");
```

shared_record_groups( )

Use this method to set or retrieve the associated shared record groups of a DNS zone.

If a shared record group is associated with a zone, then all shared records in a shared record group will be shared in the zone. For more information about shared record groups and shared records, please refer to Infoblox Administrator Guide.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

Array reference containing the name of shared record groups associated with this zone. The name of shared record group is in string format. The default value is undefined.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get shared record groups
my $srgs = $zone->shared_record_groups();
#Modify shared record groups. Assume "test_srg" and "another_srg" are the name
#of already defined shared record groups.
$zone->shared_record_groups(["test_srg", "another_srg"]);
```

stub_external_ns_group( )

Use this method to set or retrieve the external nameservers group name for stub DNS zone.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

The valid value is a desired external name servers group name in a string format.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Construct external nameserver
my $ns1 = Infoblox::DNS::Nameserver->new(
    name     => 'ns1.domain0.extra',
    ipv4addr => '5.5.5.5',
);

#Construct external nameservers group
my $ext = Infoblox::DNS::Nsgroup::ForwardStubServer->new(
    name             => 'extsrvg1',
    external_servers => [$ns1],
);

#Submit created stub_external nameservers group to session
$session->add($ext);

#Get stub_external_ns_group value
my $stub_external_ns_group = $zone->stub_external_ns_group();

#Modify stub_external_ns_group value
$zone->stub_external_ns_group('extsrvg1');
```

stub_from( )

Use this method to set or retrieve the stub_from servers of a DNS zone.

This method specifies the primary servers (masters) of the stub zone.

User can also specify grid member server IP/name pair in this list of primary servers.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

The valid value is an array reference that contains Infoblox::DNS::Nameserver object(s). The default value is undefined.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get stub_from
my $stub_from = $zone->stub_from();
#Modify stub_from
my $nameserver1 = Infoblox::DNS::Nameserver->new(
        name     => "ns1.stub.com",
        ipv4addr => "11.11.11.11",
    );
my $nameserver2 = Infoblox::DNS::Nameserver->new(
        name     => "test",
        ipv4addr => "2006::b671",
    );
#Specify a grid member server IP and name
my $nameserver3 = Infoblox::DNS::Nameserver->new(
        name     => "ns.member.com",
        ipv4addr => "192.168.1.2",
    );
$zone->stub_from([$nameserver1, $nameserver2, $nameserver3]);
```

stub_members( )

Use this method to set or retrieve the stub_members flag of a DNS zone.

This method specifies the grid member or Microsoft DNS servers of the stub zone.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

The valid value is an array reference that contains Infoblox::DNS::Member `object(s)` for grid DNS servers and Infoblox::DNS::MSServer `object(s)` for Microsoft DNS Servers. The default value is undefined.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get stub_members
my $stub_members = $zone->stub_members();
#Modify stub_members
my $memberns1 = Infoblox::DNS::Member->new(
        name     => "member1.stub.com",
        ipv4addr => "3.3.3.3",
    );

my $memberns2 = Infoblox::DNS::Member->new(
        name     => "member2.stub.com",
        ipv4addr => "4.4.4.4",
    );

$zone->stub_members([$memberns1,$memberns2 ]);
```

stub_ns_group( )

Use this method to set or retrieve the name servers group for the Stub DNS Zone object.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

The valid values are string that contains name of the group or Infoblox::Grid::DNS::Nsgroup::StubMember.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Create stub nameservers group
my $stub_group = Infoblox::Grid::DNS::StubMember->new(
    name         => 'stub_ns_group_A',
    stub_members => ['infoblox.localdomain'],
);

#Submit created stub nameserver group to session
$session->add($stub_group);

#Get stub_ns_group
my $stub_ns_group = $zone->stub_ns_group();

#Modify stub_ns_group
$zone->stub_ns_group('stub_ns_group_A');
```

views( )

Use this method to set or retrieve the views flag of a DNS zone.

You can list the views in which the zone is located.

When the `ms_managed()` flag is not None, the method is limited to retrieving the list of DNS views only. Setting the DNS view of a zone served by a Microsoft DNS server is done from the Microsoft server.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

The valid value is an array reference that contains the Infoblox::DNS::View object. The default value is the "default" view.

Note that the array size must be 1.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get views
my $views = $zone->views();
#Modify views
my $internal_view = Infoblox::DNS::View->new(
                "name"    => "internal",
                "comment" => "insert internal",
                "disable" => "false",
                );

$zone->views([$internal_view]);
```

zone( )

Use this method to retrieve the parent zone value of a DNS zone. A parent zone can be of any type, except that a delegated child zone can not have a parent zone of delegated type. The zone name can be empty.

Parameter

None

Returns

The method returns the attribute value.

Example

```
#Get zone
my $zone1 = $zone->zone();
```

zone_monitored_since( )

Use this method to retrieve the time data collection for "DNS Zones Last Queried" was turned on for the zone.

Returns

The method returns the attribute value, the number of seconds that have elapsed since January 1st, 1970 UTC.

Example

```
#Get zone_monitored_since
my $zone_monitored_since = $zone->zone_monitored_since();
```

SAMPLE CODE

The following sample code demonstrates the different functions that can be applied to an object, such as add, search, modify, and remove. This sample also includes error handling for the operations.

#Preparation prior to a zone object insertion

```
#PROGRAM STARTS: Include all the modules that will be used
use strict;
use Infoblox;

#Create a session to the Infoblox appliance
my $host_ip =  "192.168.1.2";
my $session = Infoblox::Session->new(
    master   => $host_ip,
    username => "admin",
    password => "infoblox"
    );
unless($session){
        die("Constructor for session failed: ",
               Infoblox::status_code(). ":" . Infoblox::status_detail());
}
print "Session created successfully.\n";
```

#Create a DNS zone.

```
my $memberns1 = Infoblox::DNS::Member->new(
    name     => "infoblox.localdomain",
    ipv4addr => $host_ip,
    lead     => "false",
    stealth  => "false"
);

my $nameserver1 = Infoblox::DNS::Nameserver->new(
    name     => "ns1.test.com",
    ipv4addr => "1.1.1.1",
    stealth  => "false",
    TSIGname => "tsig_name",
    TSIGkey  => "ZX1jVJl7C58GT/sc7Q3ucA==",
);

my $nameserver2 = Infoblox::DNS::Nameserver->new(
    name     => "ns2.test.com",
    ipv4addr => "2.2.2.2",
    stealth  => "false",
    TSIGname => "tsig_name2",
    TSIGkey  => "qwqwqwqwqwqwqwqwqwqA==",
);

my $default_view = Infoblox::DNS::View->new(name => "default");

#To add zone test.com in the default view
my $firstzone = Infoblox::DNS::Zone->new(
    name        => "test.com",
    views       => [$default_view],
    email       => "admin\@infoblox.com",
    comment     => "add a zone test.com",
    multiple_primaries => [$memberns1],
    secondaries => [$nameserver1, $nameserver2],
    );

unless($firstzone){
       die("Construct test.com zone object failed: ",
               Infoblox::status_code(). ":" .Infoblox::status_detail());
       }
print "test.com zone object created successfully.\n";

$session->add($firstzone)
    or die("Add zone for test.com failed: ",
               $session->status_code(). ":" .$session->status_detail());
```

```
print"Zone test.com added successfully.\n";

#Create a reverse zone.
my $revzone1 = Infoblox::DNS::Zone->new(
    name    => "10.1.4.0/24",
    email   => "admin\@infoblox.com",
    comment => "add a zone 4.168.192.in-addr.arpa",
    multiple_primaries => [$memberns1],
    );

unless($revzone1){
     die("Constructor for zone 4.1.10.in-addr.arpa failed: ",
                Infoblox::status_code(). ":" .Infoblox::status_detail());
       }
print "4.1.10.in-addr.arpa zone object created successfully.\n";

$session->add($revzone1)
    or die("Add zone for 4.1.10.in-addr.arpa failed: ",
                $session->status_code(). ":" .$session->status_detail());

print"Zone 4.1.10.in-addr.arpa added successfully.\n";

#Create a reverse zone with a prefix.
my $revzone2 = Infoblox::DNS::Zone->new(
    name    => "10.168.192.in-addr.arpa",
    email   => "admin\@infoblox.com",
    comment => "add a zone 0-63.10.168.192.in-addr.arpa",
    prefix  => "0-63",
    );

unless($revzone2){
        die("Constructor for zone 0-63.10.168.192.in-addr.arpa failed: ",
               Infoblox::status_code(). ":" .Infoblox::status_detail());
       }
print "0-63.10.168.192.in-addr.arpa zone object created successfully.\n";

$session->add($revzone2)
    or die("Add zone for 0-63.10.168.192.in-addr.arpa failed: ",
              $session->status_code(). ":" .$session->status_detail());

print"Zone 0-63.10.168.192.in-addr.arpa added successfully.\n";
```

#Search for a specific DNS zone

```
#Searches zones that ends with com
my @search_result = $session->search(
    object => "Infoblox::DNS::Zone",
    name   => ".*\\.com"
);

my $search_obj = $search_result[0];
unless($search_obj){
       die("Search zone failed: ",
                $session->status_code() . ":" . $session->status_detail());
       }

print "Search zone object found at least 1 matching entry\n";
```

#Get and modify a DNS zone

```
#Get zone test.com through session
my @result_array = $session->get(
    object => "Infoblox::DNS::Zone",
    name   => "test.com",
    view   => "default"
);
my $for_obj = $result_array[0];
unless($for_obj){
       die("Get zone test.com failed: ",
       $session->status_code() . ":" . $session->status_detail());
       }
print "Get test.com zone object found at least 1 matching entry\n";

#Modify one of the attributes of the specified zone
$for_obj->comment("this is a modified comment.");

#Apply the change
```

```
    $session->modify($result_array[0])
        or die("Modify zone for test.com failed: ",
               $session->status_code(). ":" .$session->status_detail());

 print "test.com zone object modified successfully.\n";

#Get and modify zone 4.1.10.in-addr.arpa.
my @result = $session->get(
    object => "Infoblox::DNS::Zone",
    name   => "4.1.10.in-addr.arpa"
);

my $rev_object = $result[0];
unless ($rev_object) {
       die("Get zone 4.1.10.in-addr.arpa failed: ",
                $session->status_code() . ":" . $session->status_detail());
       }
print "Get 4.1.10.in-addr.arpa zone object found at least 1 matching entry\n";

#Modify one of the attributes of the specified zoe
$rev_object->secondaries([$nameserver2]);

#Apply the changes
$session->modify($rev_object)
            or die("Modify reverse zone 4.1.10.in-addr.arpa failed: ",
                        $session->status_code(). ":" .$session->status_detail());

print "4.1.10.in-addr.arpa zone object modified successfully.\n";
```

#Remove a DNS zone

```
#Remove zone test.com from a view.
$session->remove(
    object => "Infoblox::DNS::Zone",
    name   => "test.com",
    views  => [$default_view]
    )
    or die("Remove zone test.com failed: ",
             $session->status_code(). ":" .$session->status_detail());

print "Remove zone test.com successfull\n";

#Remove zone 4.1.10.in-addr.arpa from a view.
$session->remove(
    object => "Infoblox::DNS::Zone",
    name   => "4.1.10.in-addr.arpa",
    views  => [$default_view]
    )
    or die("Remove zone 4.1.10.in-addr.arpa failed: ",
             $session->status_code(). ":" .$session->status_detail());

print "Remove zone 4.1.10.in-addr.arpa successfull\n";

#Remove zone with prefix from a view.
$session->remove(
    object => "Infoblox::DNS::Zone",
    name    => "10.168.192.in-addr.arpa",
    prefix  => "0-63",
   )
   or die("Remove zone 0-63.10.168.192.in-addr.arpa failed: ",
             $session->status_code(). ":" .$session->status_detail());

print "Remove zone 0-63.10.168.192.in-addr.arpa successfull\n";

####PROGRAM ENDS####
```

AUTHOR

Infoblox Inc. http://www.infoblox.com/

SEE ALSO

Infoblox::DNS::Member, Infoblox::DNS::Nameserver, Infoblox::DNS::SRG, Infoblox::DNS::TSIGKey, Infoblox::DNS::View, Infoblox::Session, Infoblox::Session->add(), Infoblox::Session->get(), Infoblox::Session->modify(), Infoblox::Session->remove(), Infoblox::Session->search(), Infoblox::DNS::Member::SoaSerial, Infoblox::DNS::Member::Mname,

Infoblox::DNS::DNSSecKeyAlgorithm, Infoblox::Grid::MSServer::DCNSRecordCreation, Infoblox::Grid::DNS::Nsgroup, Infoblox::Grid::DNS::Nsgroup, Infoblox::Grid::DNS::Nsgroup,

---

COPYRIGHT

Copyright (c) 2017 Infoblox Inc.

Infoblox::DNS::Zone::Discrepancy - DNS Zone discrepancy object.

- NAME
- DESCRIPTION
- SESSION METHODS
  - Infoblox::Session->get( )
  - Infoblox::Session->search( )
- METHODS
  - description( )
  - severity( )
  - timestamp( )
  - view( )
  - zone( )
- AUTHOR
- SEE ALSO
- COPYRIGHT

NAME

Infoblox::DNS::Zone::Discrepancy - DNS Zone discrepancy object.

DESCRIPTION

The DNS zone discrepancy object represents discrepancy information for a specified zone.

SESSION METHODS

This section describes all the methods in the Infoblox::Session module that you can apply to a DNS Zone Discrepancy object.

Infoblox::Session->get( )

Use this method to retrieve all the matching objects from the Infoblox appliance. See Infoblox::Session->get() for parameters and return values.

Key References

```
Apply the following attributes to get a specific DNS Zone Discrepancy object:

  view                  - Optional. The DNS view in which the zone checking takes place. The default
is all DNS views.
  severity              - Optional. The severity for discrepancy. The valid values are 'CRITICAL',
'SEVERE', 'WARNING', 'INFORMATIONAL', 'NORMAL'. The default is any severity.
  zone                  - Optional. The parent zone name in string format.
```

Examples

```
my @retrieved_objs = $session->get(
    object => "Infoblox::DNS::Zone::Discrepancy",
    view   => "default" );
```

Infoblox::Session->search( )

Use this method to search for DNS Zone Discrepancy objects from the Infoblox appliance. See Infoblox::Session->search() for parameters and return values.

Key References

```
Apply the following attributes to get a specific DNS Zone Discrepancy object:

  view                  - Optional. The DNS view in which the zone checking takes place. The default
is all DNS views.
  severity              - Optional. The severity for discrepancy. The valid values are 'CRITICAL',
'SEVERE', 'WARNING', 'INFORMATIONAL', 'NORMAL'. The default is any severity.
  zone                  - Optional. The parent zone name in string format.
```

Examples

```
my @retrieved_objs = $session->search(
    object => "Infoblox::DNS::Zone::Discrepancy",
    view   => "default"
    zone   => "*.com");
```

METHODS

This section describes all the methods that you can use to configure and retrieve the attribute values of a zone.

description( )

Use this method to retrieve the description for discrepancy. This is a read-only attribute.

Parameter

None

Returns

The description in string format with a maximum of 256 bytes.

Example

```
#Get description value
my $value = $discrepancy->description();
```

severity( )

Use this method to retrieve the severity for discrepancy. This is a read-only attribute.

Parameter

None

Returns

The method returns attribute value. The valid returns values are 'CRITICAL', 'SEVERE', 'WARNING', 'INFORMATIONAL' and 'NORMAL'.

Example

```
#Get zone discrepancy severity value
my $severity = $discrepancy->severity();
```

timestamp( )

Use this method to retrieve the timestamp when results were obtained. This is a read-only attribute.

Parameter

None

Returns

The return value is a timestamp in UNIX format that is measured in seconds since 00:00:00 January 1, 1970 UTC.

Example

```
#Get zone discrepancy timestamp value
my $timestamp = $discrepancy->timestamp();
```

view( )

Use this method to retrieve the DNS view name of the record associated with the zone discrepancy object. This is a read-only attribute.

Parameter

None

Returns

The method returns attribute value.

Example

```
#Get view value
my $value = $discrepancy->view();
```

zone( )

Use this method to retrieve the zone name. This is a read-only attribute.

Parameter

None

Returns

A zone name in FQDN (Fully Qualified Domain Name) format. The FQDN consists of the host_name followed by the domain name (example: abc.com). A host_name can have a maximum of 256 characters.

Example

```
#Get zone value
my $zone = $discrepancy->zone();
```

AUTHOR

Infoblox Inc. http://www.infoblox.com/

SEE ALSO

Infoblox::Session->get(), Infoblox::Session->search(), Infoblox::Session.

COPYRIGHT

Copyright (c) 2017 Infoblox Inc.

Infoblox::DTC - A DTC functions.

- NAME
- DESCRIPTION
- CONSTRUCTOR
- SESSION METHODS
- METHODS
  - query( )
  - import_maxminddb( )
  - add_certificate( )
- SEE ALSO
- AUTHOR
- COPYRIGHT

NAME

Infoblox::DTC - A DTC functions.

DESCRIPTION

The object provides access to DTC functions.

CONSTRUCTOR

```
my $dtc = Infoblox::DTC->new(
    'session' => $session,       # Required
);
```

SESSION METHODS

The object does not support any session methods.

METHODS

query( )

Use this method to perform a DTC query on a given member.

Parameter

Valid value is an Infoblox::DTC::Query::Parameters object.

Returns

The method returns an Infoblox::DTC::Query::Response object when operation succeeds, and returns 'false' when the operation fails.

Example

```
#perform DTC query
my $response = $dtc->query($params);
```

import_maxminddb( )

Use this method to upload the MaxMind database.

Key Reference

```
Apply the following attribute to upload a MaxMind Database:

    path - Required. The path to a tar archive that contains the MaxMind database.
```

Returns

The method returns "true" when an upload of a MaxMind Database has been performed successfully, and returns "false" when the operation fails.

Example

```
#upload MaxMind DB
my $res = $dtc->import_maxminddb(path => $path_to_maxminddb);
```

add_certificate( )

Use this method to upload a DTC certificate.

Key Reference

```
Apply the following attribute to upload a DTC certificate:
   path - Required. The path to the DTC certificate file.
```

Returns

The method returns 'true' when the upload of a DTC certificate has been performed successfully, and returns 'false' when the operation fails.

Example

```
#add certificate with a path given
my $res = $dtc->add_certificate(path => $path_to_cert);
```

SEE ALSO

Infoblox::DTC::Query::Parameters.

AUTHOR

Infoblox Inc. http://www.infoblox.com/

COPYRIGHT

Copyright (c) 2017 Infoblox Inc.

Infoblox::DTC::AllRecords - A synthetic object used to return all DTC record object types belonging to a DTC server.

- NAME
- DESCRIPTION
- SESSION METHODS
  - Infoblox::Session->search( )
- METHODS
  - comment( )
  - disable( )
  - dtc_server( )
  - record( )
  - ttl( )
  - type( )
- SAMPLE CODE
- AUTHOR
- SEE ALSO
- COPYRIGHT

NAME

Infoblox::DTC::AllRecords - A synthetic object used to return all DTC record object types belonging to a DTC server.

DESCRIPTION

By searching on this object, all the DTC record objects associated with a specific DTC server can be retrieved.

SESSION METHODS

This section describes all the methods in Infoblox::Session module that you can apply to a DTC AllRecords object.

Infoblox::Session->search( )

Use this method to search for record objects in the Infoblox appliance. See Infoblox::Session->search() for parameters and return values.

Key References

```
Apply the following attributes to search for a DTC AllRecords object:

 dtc_server - Required. A DTC Server object.
 type       - Optional. The type of record to search for. By default, all record types are searched.
Following is the list of valid values for this search field:

      'All Records',
      'All DTC A Records'
      'All DTC AAAA Records'
      'All DTC CNAME Records'
      'All DTC NAPTR Records'
```

Example

```
# search for all DTC records associated with DTC Server 'server-1'

my $dtc_server = Infoblox::DTC::Server->new(
    name => 'server-1',
    host => '10.0.0.1',
);

my @retrieved_objs = $session->search(
    object     => "Infoblox::DTC::AllRecords",
    dtc_server => $dtc_server,
);

# search for all DTC NAPTR records that are associated with DTC Server 'server-1'

my $dtc_server = Infoblox::DTC::Server->new(
    name => 'server-1',
    host => '10.0.0.1',
);

my @retrieved_objs = $session->search(
    object     => "Infoblox::DTC::AllRecords",
    type       => 'All DTC NAPTR Records',
```

```
        dtc_server => $dtc_server,
);
```

METHODS

This section describes all the methods that you can use to retrieve the attribute values of a DTC AllRecords object. All methods are read-only and cannot be set.

comment( )

Use this method to retrieve the descriptive comment of the record associated with the DTC AllRecords object.

Returns

The method returns the attribute value.

Example

```
#Get comment
my $comment = $alldtcrecord->comment();
```

disable( )

Use this method to retrieve the disable flag of the record associated with the DTC AllRecords object (if present).

Returns

The method returns the attribute value.

Example

```
#Get disable
my $disable = $alldtcrecord->disable();
```

dtc_server( )

Use this method to retrieve the DTC Server with which the DTC record is associated.

Returns

The method returns an Infoblox::DTC::Server partial object.

Example

```
#Get dtc_server
my $dtc_server = $alldtcrecord->dtc_server();
```

record( )

Use this method to retrieve the record object associated with the DTC AllRecords object.

This object is considered a 'partial' object. See Infoblox::Session->fill_partial_object() for more information on how it can be converted to a 'full' API object.

If the DTC AllRecords object refers to a record that is not currently supported in the Perl API, the member value will be undefined.

Returns

The method returns the attribute value.

Example

```
#Get record
my $record = $alldtcrecord->record();
```

ttl( )

Use this method to retrieve the TTL value of the record associated with the DTC AllRecords object (if present).

Returns

The method returns the attribute value.

Example

```
#Get ttl
```

```
my $ttl = $alldtcrecord->ttl();
```

type( )

Use this method to retrieve the type of the record associated with the AllRecords object. This is a descriptive string that identifies the record to which this DTC AllRecords object refers. (For example, 'DTC NAPTR record', ...)

Returns

The method returns the attribute value.

Example

```
#Get type
my $type = $alldtcrecord->type();
```

SAMPLE CODE

For examples see Infoblox::DNS::AllRecords SAMPLE CODE section.

AUTHOR

Infoblox Inc. http://www.infoblox.com/

SEE ALSO

Infoblox::Session, Infoblox::Session->search(), Infoblox::DTC::Server, Infoblox::DTC::Record::NAPTR, Infoblox::DTC::Record::A, Infoblox::DTC::Record::AAAA, Infoblox::DTC::Record::CNAME

COPYRIGHT

Copyright (c) 2017 Infoblox Inc.

Infoblox::DTC::Certificate - a DTC Certificate object.

- NAME
- DESCRIPTION
- CONSTRUCTOR
- SESSION METHODS
  - Infoblox::Session->get( )
  - Infoblox::Session->search( )
- MODULE METHODS
  - Infoblox::DTC->add_certificate( )
- METHODS
  - certificate( )
  - in_use( )
- AUTHOR
- SEE ALSO
- COPYRIGHT

NAME

Infoblox::DTC::Certificate - a DTC Certificate object.

DESCRIPTION

A DTC Certificate object.

CONSTRUCTOR

SESSION METHODS

This section describes all the methods in a Session module that you can apply to a DTC Certificate object.

Infoblox::Session->get( )

Use this method to retrieve all the matching objects from the Infoblox appliance. See Infoblox::Session->get() for parameters and return values.

Key Reference

```
Apply the following attribute to get a specific object:

  distinguished_name - Optional. A X509Certificate Distinguished name (DN).
  issuer             - Optional. A X509Certificate issuer.
  serial             - Optional. A X509Certificate serial number.
  valid_not_after    - Optional. A X509Certificate expiry date.
  valid_not_before   - Optional. A X509Certificate validity start date.
```

Example

```
my @retrieved_objs = $session->get(
    object => 'Infoblox::DTC::Certificate',
    serial => 'd80b0e8c47df5082',
);
```

Infoblox::Session->search( )

Use this method to search for objects in the Infoblox appliance. See Infoblox::Session->search() for parameters and return values.

Key Reference

```
Apply the following attribute to get a specific object:

  distinguished_name - Optional. A X509Certificate Distinguished name (DN).
  issuer             - Optional. A X509Certificate issuer.
  serial             - Optional. A X509Certificate serial number.
  valid_not_after    - Optional. A X509Certificate expiry date.
  valid_not_before   - Optional. A X509Certificate validity start date.
```

Example

```
my @retrieved_objs = $session->search(
    object => 'Infoblox::DTC::Certificate',
```

```
        serial => 'd80b0e8c47df5082',
);
```

MODULE METHODS

Infoblox::DTC->add_certificate( )

Use this method to upload a DTC certificate.

Key Reference

```
Apply the following attribute to upload a DTC certificate:

   path - Required. The path to the file that contains the DTC certificate.
```

Returns

The method returns 'true' when the upload of a MaxMind Database has been performed successfully, and returns 'false' when the operation fails.

Example

```
#add certificate with a path given
my $res = $dtc->add_certificate(path => $path_to_cert);
```

METHODS

This section describes all the methods that you can use to set or retrieve the attribute values of the object.

certificate( )

Use this method to retrieve an X509Certificate object. This is a read-only attribute.

Parameter

None

Returns

The valid return value is an Infoblox::Grid::X509Certificate object.

Example

```
#get certificate value
my $certificate = $dtc_certificate->certificate();
```

in_use( )

Use this method to retrieve the certificate status. This is a read-only attribute.

Parameter

None

Returns

The method returns the attribute value.

Example

```
#get in_use value
my $in_use = $dtc_certificate->in_use();
```

AUTHOR

Infoblox Inc. http://www.infoblox.com/

SEE ALSO

Infoblox::Session, Infoblox::Session->get(), Infoblox::Session->search(), Infoblox::DTC->add_certificate()

COPYRIGHT

Infoblox::DTC::Certificate - a DTC Certificate object.

Copyright (c) 2017 Infoblox Inc.

Infoblox::DTC::Health - A DTC health information object.

- NAME
- DESCRIPTION
- SESSION METHODS
- METHODS
  - availability( )
  - description( )
  - enabled_state( )
- AUTHOR
- SEE ALSO
- COPYRIGHT

NAME

Infoblox::DTC::Health - A DTC health information object.

DESCRIPTION

The DTC health object provides information about the health status for a specified object.

SESSION METHODS

The object does not support any session methods. It is retrieved as a part of the following objects: Infoblox::DTC::Pool, Infoblox::DTC::Server, Infoblox::DTC::LBDN.

METHODS

This section describes all the methods that you can use to set or retrieve the attribute values of the object.

availability( )

Use this method to retrieve the color status for availability. This is a read-only attribute.

Parameter

None

Returns

The valid return values are 'NONE', 'GREEN', 'YELLOW', 'RED', 'BLUE' and 'GRAY'.

Example

```
#get availability value
my $availability = $health->availability();
```

description( )

Use this method to retrieve the textual description of the object's status. This is a read-only attribute.

Parameter

None

Returns

The method returns the attribute value.

Example

```
#get description value
my $description = $health->description();
```

enabled_state( )

Use this method to retrieve the enabled state of the object. This is a read-only attribute.

Parameter

None

Returns

The valid return values are 'NONE', 'ENABLED', 'DISABLED' and 'DISABLED_BY_PARENT'.

Example

```
#get enabled_state value
my $enabled_state = $health->enabled_state();
```

AUTHOR

Infoblox Inc. http://www.infoblox.com/

SEE ALSO

Infoblox::DTC::Pool, Infoblox::DTC::Server, Infoblox::DTC::LBDN.

COPYRIGHT

Copyright (c) 2017 Infoblox Inc.

Infoblox::DTC::LBDN - A DTC Load Balanced Domain Name object.

- NAME
- DESCRIPTION
- CONSTRUCTOR
- SESSION METHODS
  - Infoblox::Session->add( )
  - Infoblox::Session->get( )
  - Infoblox::Session->modify( )
  - Infoblox::Session->remove( )
  - Infoblox::Session->search( )
- METHODS
  - auth_zones( )
  - comment( )
  - disable( )
  - extattrs( )
  - extensible_attributes( )
  - health( )
  - lb_method( )
  - name( )
  - override_ttl( )
  - patterns( )
  - persistence( )
  - pools( )
  - priority( )
  - topology( )
  - ttl( )
  - types( )
- AUTHOR
- SEE ALSO
- COPYRIGHT

NAME

Infoblox::DTC::LBDN - A DTC Load Balanced Domain Name object.

DESCRIPTION

A DTC Load Balanced Domain Name object.

CONSTRUCTOR

```
 my $lbdn = Infoblox::DTC::LBDN->new(
    name                 => $string,
#Required
    lb_method            => 'GLOBAL_AVAILABILITY' | 'RATIO' | 'ROUND_ROBIN' | 'TOPOLOGY',
#Required
    patterns             => [$pattern1, $pattern2, ...],
#Optional / Default is undefined
    disable              => 'true' | 'false',
#Optional / Default is 'false'
    comment              => $string,
#Optional / Default is undefined
    persistence          => $uint,
#Optional / Default is undefined
    topology             => $topology_obj,
#Optional / Default is undefined
    ttl                  => $uint,
#Optional / Default is undefined
    types                => ['A', 'AAAA', 'NAPTR', 'CNAME'],
#Optional / Default is ['A', 'AAAA']
    override_ttl         => 'true' | 'false',
#Optional / Default is 'false'
    pools                => [$pool_link_obj1, $pool_link_obj2, ...],
#Optional / Default is undefined
    priority             => $uint,
#Optional / Default is 1
    auth_zones           => [$auth_zone1, $auth_zone2, ...],
#Optional / Default is undefined
    extattrs             => { $string => $extattr, ... },
#Optional / Default is undefined
    extensible_attributes => { $string => $string | $num, $string => [ $string | $num, ... ], ... }
#Optional / Default is undefined
```

```
);

You cannot set both extattrs and extensible_attributes attributes at the same time.
```

SESSION METHODS

This section describes all the methods in a Session module that you can apply to a DTC LBDN object.

Infoblox::Session->add( )

Use this method to add an object to the Infoblox appliance. See Infoblox::Session->add() for parameters and return values.

Example

```
#construct an object
my $lbdn = Infoblox::DTC::LBDN->new(
   name      => 'load_bal',
   lb_method => 'GLOBAL_AVAILABILITY',
);

#submit for addition
my $response = $session->add($lbdn);
```

Infoblox::Session->get( )

Use this method to retrieve all the matching objects from the Infoblox appliance. See Infoblox::Session->get() for parameters and return values.

Key Reference

```
Apply the following attribute to get a specific object:

 name                 - Optional. A DTC LBDN name in a string format.
 comment              - Optional. A DTC LBDN comment.
 fqdn                 - Optional. A fully qualified domain name (FQDN).
 extattrs             - Optional. A hash reference containing extensible attributes.
 extensible_attributes - Optional. A hash reference that contains extensible attributes.
 status_member        - Optional. A member that provides the object health status. The default is
Grid.
```

Example

```
my @retrieved_objs = $session->get(
    object => 'Infoblox::DTC::LBDN',
    name   => 'load_bal',
    fqdn   => 'lb.com'
);

my @retrieved_objs = $session->get(
    object => 'Infoblox::DTC::LBDN',
    extensible_attributes => { 'Site' => 'Santa Clara' }
);
```

Infoblox::Session->modify( )

Use this method to modify an object in the Infoblox appliance. See Infoblox::Session->modify() for parameters and return values.

Example

```
#Use method to modify the comment.
$lbdn->comment('this is a modified comment');
#Submit modification
my $response = $session->modify($lbdn);
```

Infoblox::Session->remove( )

Use this method to remove an object from the Infoblox appliance. See Infoblox::Session->remove() for parameters and return values.

To remove a specific object, first use `get()` or `search()` to retrieve the specific object, and then submit this object for removal.

Example

```
#Get the objects with the same name
my @retrieved_objs = $session->get(
    object => 'Infoblox::DTC::LBDN',
    name   => 'load_bal',
);
```

```
#Find the desired object from the retrieved list.
my $desired_lbdn = $retrieved_objs[0];
#Submit for removal
my $response = $session->remove($desired_lbdn);
```

Infoblox::Session->search( )

Use this method to search for objects in the Infoblox appliance. See Infoblox::Session->search() for parameters and return values.

Key Reference

```
Apply the following attribute to get a specific object:

 name                  - Optional. A DTC LBDN name in a string format.
 comment               - Optional. A DTC LBDN comment.
 fqdn                  - Optional. A fully qualified domain name (FQDN).
 extattrs              - Optional. A hash reference containing extensible attributes.
 extensible_attributes - Optional. A hash reference that contains extensible attributes.
 status_member         - Optional. A member that provides the object health status. The default is
Grid.
```

Example

```
my @retrieved_objs = $session->search(
    object => 'Infoblox::DTC::LBDN',
    name   => 'load_b.*',
    fqdn   => 'lb.*'
);

my @retrieved_objs = $session->search(
    object => 'Infoblox::DTC::LBDN',
    extensible_attributes => { 'Site' => 'Santa Clara' }
);
```

METHODS

This section describes all the methods that you can use to set or retrieve the attribute values of the object.

auth_zones( )

Use this method to set or retrieve a list of authoritative zones the DTC LBDN is linked to.

Note that two different DTC LBDNs cannot be linked to the same zone if their patterns overlap.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

The valid value is an array containing Infoblox::DNS::Zone objects.

Returns

If you specified a parameter, the method returns 'true' when the modification succeeds, and returns 'false' when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#get auth_zones value
my @auth_zones = $lbdn->auth_zones();

#modify auth_zones value
$lbdn->auth_zones([$zone1, $zone2]);
```

comment( )

Use this method to set or retrieve a descriptive comment.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

Desired comment in string format (UTF-8) with a maximum of 256 bytes.

Returns

If you specified a parameter, the method returns 'true' when the modification succeeds, and returns 'false' when the operation

fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#get comment value
my $comment = $lbdn->comment();

#modify comment value
$lbdn->comment('desired comment');
```

disable( )

Use this method to enable or disable a DTC LBDN object.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

Specify 'true' to disable an DTC LBDN object or 'false' to enable it.

Returns

If you specified a parameter, the method returns 'true' when the modification succeeds, and returns 'false' when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#get disable value
my $disable = $lbdn->disable();

#modify disable value
$lbdn->disable('true');
```

extattrs( )

Use this method to set or retrieve the extensible attributes associated with a DTC LBDN object.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

Valid value is a hash reference containing the names of extensible attributes and their associated values (Infoblox::Grid::Extattr objects).

Returns

If you specified a parameter, the method returns 'true' when the modification succeeds, and returns 'false' when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#get extattrs value
my $ref_extattrs = $lbdn->extattrs();

#Modify extattrs
$lbdn->extattrs({ 'Site' => $extattr1, 'Administrator' => $extattr2 });
```

extensible_attributes( )

Use this method to set or retrieve the extensible attributes associated with a DTC LBDN object.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

For valid values for extensible attributes, see Infoblox::Grid::ExtensibleAttributeDef/Extensible Attribute Values.

Returns

If you specified a parameter, the method returns 'true' when the modification succeeds, and returns 'false' when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get extensible attributes
my $ref_extensible_attributes = $lbdn->extensible_attributes();

#Modify extensible attributes
$lbdn->extensible_attributes({'Site' => 'Santa Clara', 'Administrator' => ['Peter', 'Tom']});
```

health( )

Use this method to retrieve DTC LBDN health information. This is a read-only attribute.

Parameter

None

Returns

The valid return value is an Infoblox::DTC::Health object.

Example

```
#get health value
my $health = $lbdn->health();
```

lb_method( )

Use this method to set or retrieve a DTC LBDN load balancing method.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

The valid values are 'GLOBAL_AVAILABILITY', 'RATIO', 'ROUND_ROBIN' and 'TOPOLOGY'.

Returns

If you specified a parameter, the method returns 'true' when the modification succeeds, and returns 'false' when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#get lb_method value
my $lb_method = $lbdn->lb_method();

#modify lb_method value
$lbdn->lb_method('RATIO');
```

name( )

Use this method to set or retrieve the DTC LBDN name.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

The valid value is a desired name in a string format.

Returns

If you specified a parameter, the method returns 'true' when the modification succeeds, and returns 'false' when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#get name value
my $name = $lbdn->name();

#modify name value
$lbdn->name('load_bal1');
```

override_ttl( )

The override_ttl attribute controls whether the TTL value in the object is used, instead of the Grid default.

The override_ttl attribute can be specified explicitly. It is also set implicitly when ttl is set to a defined value.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

Specify 'true' to set override_ttl flag or 'false' to disable it. Default value is 'false'.

Returns

If you specified a parameter, the method returns 'true' when the modification succeeds, and returns 'false' when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#get override_ttl value
my $override_ttl = $lbdn->override_ttl();

#modify override_ttl value
$lbdn->override_ttl('true');
```

patterns( )

Use this method to set or retrieve an array of DNS FQDN patterns. The patterns are used to match DNS query names processed by the DTC LBDN.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

The valid value is an array of FQDN patterns in string format.

Returns

If you specified a parameter, the method returns 'true' when the modification succeeds, and returns 'false' when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#get patterns value
my @patterns = $lbdn->patterns();

#modify patterns value
$lbdn->patterns(['???'.domain.com', '*.test.com', 'test.domain.com']);
```

persistence( )

Use this method to set or retrieve the maximum time (in seconds) for which client specific DTC LBDN responses are cached.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

The valid value is an unsigned integer. Zero specifies no caching.

Returns

If you specified a parameter, the method returns 'true' when the modification succeeds, and returns 'false' when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#get persistence value
my $persistence = $lbdn->persistence();

#modify persistence value
$lbdn->persistence(5);
```

pools( )

Use this method to set or retrieve the list of pools the DTC LBDN is linked to.

Note that a DTC LBDN linked to a zone must contain at least one enabled pool if lb_method is set to anything other than 'TOPOLOGY'.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

The valid value is an array of Infoblox::DTC::Pool::Link objects.

Returns

If you specified a parameter, the method returns 'true' when the modification succeeds, and returns 'false' when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#get pools value
my @pools = $lbdn->pools();

#modify pools value
$lbdn->pools([$pool_link1, $pool_link2]);
```

priority( )

Use this method to set or retrieve the LBDN pattern match priority for "overlapping" DTC LBDN objects.

LBDNs are "overlapping" if they are simultaneously assigned to a zone and have patterns that can match the same FQDN. The matching LBDN with highest priority (lowest ordinal) will be used.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

The valid value is an unsigned integer between 1 and 3. The default is 3.

Returns

If you specified a parameter, the method returns 'true' when the modification succeeds, and returns 'false' when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get priority value
my $priority = $object->priority();

#Modify priority value
$object->priority(3);
```

topology( )

Use this method to set or retrieve topology rules for the 'TOPOLOGY' load balancing method.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

The valid value is an Infoblox::DTC::Topology object.

Returns

If you specified a parameter, the method returns 'true' when the modification succeeds, and returns 'false' when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#get topology value
my $topology = $lbdn->topology();

#modify topology value
$lbdn->topology($topology);
```

ttl( )

Use this method to set a TTL value.

Setting this method to a defined value implicitly sets the override_ttl method to 'true'. Setting the parameter to undefined causes the appliance to use the Grid default and automatically resets the override_ttl attribute to 'false'.

Note that when ttl is set to a defined value and override_ttl is set to 'false', the last operation takes precedence. Thus the sequence $object->ttl(10); $object->override_ttl('false'); will set override_ttl to 'false', and the sequence $object->override_ttl('false'); $object->ttl(10); will result in override_ttl='true'.

Parameter

The valid value is an unsigned integer.

Returns

If you specified a parameter, the method returns 'true' when the modification succeeds, and returns 'false' when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#get ttl value
my $ttl = $lbdn->ttl();

#modify ttl value
$lbdn->ttl(10);
```

types( )

Use this method to set or retrieve the list of record types supported by LBDN.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

The valid valid is an array reference that contains any of the following values: 'A', 'AAA', 'NAPTR', 'CNAME'.

Returns

If you specified a parameter, the method returns 'true' when the modification succeeds, and returns 'false' when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#get types value
my $types = $lbdn->types();

#modify types value
$lbdn->types(['NAPTR']);
```

AUTHOR

Infoblox Inc. http://www.infoblox.com/

SEE ALSO

Infoblox::Session, Infoblox::Session->add(), Infoblox::Session->get(), Infoblox::Session->modify(), Infoblox::Session->remove(), Infoblox::Session->search(), Infoblox::Grid::Extattr, Infoblox::Grid::ExtensibleAttributeDef/Extensible Attribute Values, Infoblox::DTC::Health, Infoblox::DTC::Pool::Link, Infoblox::DTC::Topology.

COPYRIGHT

Copyright (c) 2017 Infoblox Inc.

Infoblox::DTC::Monitor - A DTC Health Monitor object.

- NAME
- DESCRIPTION
- SESSION METHODS
  - Infoblox::Session->get( )
  - Infoblox::Session->search( )
- METHODS
  - comment( )
  - interval( )
  - monitor( )
  - name( )
  - port( )
  - retry_up( )
  - retry_down( )
  - timeout( )
  - type( )
- AUTHOR
- SEE ALSO
- COPYRIGHT

NAME

Infoblox::DTC::Monitor - A DTC Health Monitor object.

DESCRIPTION

A DTC Health Monitor object.

SESSION METHODS

This section describes all the methods in a Session module that you can apply to a DTC Health Monitor object.

Infoblox::Session->get( )

Use this method to retrieve all the matching objects from the Infoblox appliance. See Infoblox::Session->get() for parameters and return values.

Key Reference

```
Apply the following attribute to get a specific object:

  comment - Optional. A DTC Health Monitor comment.
  name    - Optional. A DTC Health Monitor name.
```

Example

```
my @retrieved_objs = $session->get(
    object => 'Infoblox::DTC::Monitor',
    name   => 'tcp_monitor',
);
```

Infoblox::Session->search( )

Use this method to search for objects in the Infoblox appliance. See Infoblox::Session->search() for parameters and return values.

Key Reference

```
Apply the following attribute to get a specific object:

  comment - Optional. A DTC Health Monitor comment.
  name    - Required. A DTC Health Monitor name.
```

Example

```
my @retrieved_objs = $session->search(
    object => 'Infoblox::DTC::Monitor',
    name   => '.*_monitor',
);
```

METHODS

This section describes all the methods that you can use to set or retrieve the attribute values of the object.

comment( )

Use this method to retrieve a comment. This is a read-only attribute.

Parameter

None

Returns

The method returns the attribute value.

Example

```
#get comment value
my $comment = $monitor->comment();
```

interval( )

Use this method to retrieve the interval value for the request being sent. This is a read-only attribute.

Parameter

None

Returns

The valid return value is an unsigned integer.

Example

```
#get interval value
my $interval = $monitor->interval();
```

monitor( )

Use this method to retrieve an exact DTC Health Monitor object. This is a read-only attribute.

Parameter

None

Returns

The valid return value can be one of the following objects: Infoblox::DTC::Monitor::TCP, Infoblox::DTC::Monitor::ICMP, Infoblox::DTC::Monitor::SIP, Infoblox::DTC::Monitor::HTTP, Infoblox::DTC::Monitor::PDP, Infoblox::DTC::Monitor::SNMP.

Example

```
#get monitor value
my $monitor = $monitor->monitor();
```

name( )

Use this method to retrieve an object name. This is a read-only attribute.

Parameter

None

Returns

The method returns the attribute value.

Example

```
#get name value
my $name = $monitor->name();
```

port( )

Use this method to retrieve a DTC Health Monitor port value. This is a read-only attribute.

Note that port value for the ICMP Health Monitor is 0.

Parameter

None

Returns

The valid return value is an unsigned integer between 0 and 65535.

Example

```
#get port value
my $port = $monitor->port();
```

retry_up( )

Use this method to retrieve the number of times the server appears online after it was offline so it is treated as alive. This is a read-only attribute.

Parameter

None

Returns

The valid return value is an unsigned integer.

Example

```
#get retry_up value
my $retry_up = $monitor->retry_up();
```

retry_down( )

Use this method to retrieve the number of times the server appears online after it was offline so it is treated as alive. This is a read-only attribute.

Parameter

None

Returns

The valid return value is an unsigned integer.

Example

```
#get retry_down value
my $retry_down = $monitor->retry_down();
```

timeout( )

Use this method to retrieve the value of a request timeout. This is a read-only attribute.

Parameter

None

Returns

The valid return value is an unsigned integer.

Example

```
#get timeout value
my $timeout = $monitor->timeout();
```

type( )

Use this method to retrieve the transport type. This is a read-only attribute.

Parameter

None

Returns

The valid return value is 'HTTP', 'HTTPS', 'SIP', 'PDP', 'SNMP', 'TCP' or 'ICMP'.

Example

```
#get type value
my $type = $monitor->type();
```

AUTHOR

Infoblox Inc. http://www.infoblox.com/

SEE ALSO

Infoblox::Session, Infoblox::Session->get(), Infoblox::Session->search(), Infoblox::DTC::Monitor::TCP, Infoblox::DTC::Monitor::ICMP, Infoblox::DTC::Monitor::SIP, Infoblox::DTC::Monitor::HTTP, Infoblox::DTC::Monitor::PDP, Infoblox::DTC::Monitor::SNMP.

COPYRIGHT

Copyright (c) 2017 Infoblox Inc.

Infoblox::DTC::Monitor::HTTP - An DTC HTTP Health Monitor object.

- NAME
- DESCRIPTION
- CONSTRUCTOR
- SESSION METHODS
  - Infoblox::Session->add( )
  - Infoblox::Session->get( )
  - Infoblox::Session->modify( )
  - Infoblox::Session->remove( )
  - Infoblox::Session->search( )
- METHODS
  - ciphers( )
  - client_cert( )
  - comment( )
  - content_check( )
  - content_check_input( )
  - content_check_regex( )
  - content_check_op( )
  - content_extract_group( )
  - content_extract_type( )
  - content_extract_value( )
  - enable_sni( )
  - extattrs( )
  - extensible_attributes( )
  - interval( )
  - name( )
  - port( )
  - request( )
  - result( )
  - result_code( )
  - retry_down( )
  - retry_up( )
  - secure( )
  - timeout( )
  - validate_cert( )
- AUTHOR
- SEE ALSO
- COPYRIGHT

NAME

Infoblox::DTC::Monitor::HTTP - An DTC HTTP Health Monitor object.

DESCRIPTION

The DTC HTTP Health Monitor validates the health of a HTTP service by first sending a specific HTTP message to a server and then examining the response received from the server. The validation is successful if the received response matches the expected message.

CONSTRUCTOR

```
 my $http_monitor = Infoblox::DTC::Monitor::HTTP->new(
    name                 => $string,
#Required
    ciphers              => [$cipher1, $cipher2, ...],
#Optional / Default is undefined
    client_cert          => $certificate
#Optional / Default is undefined
    comment              => $string,
#Optional / Default is undefined
    content_check        => 'NONE' | 'MATCH' | 'EXTRACT',
#Optional / Default is 'NONE'
    content_check_input  => 'HEADERS' | 'ALL' | 'BODY',
#Optional / Default is 'ALL'
    content_check_op     => 'EQ' | 'NEQ' | 'LEQ' | 'GEQ',
#Optional / Default is undefined
    content_check_regex  => $string,
#Optional / Default is undefined
    content_extract_group => $uint,
#Optional / Default is 0
    content_extract_type  => 'STRING' | 'INTEGER',
#Optional / Default is 'STRING'
```

```
    content_extract_value => $string,
#Optional / Default is undefined
    enable_sni            => 'true' | 'false',
#Optional / Default is 'false'
    extattrs              => { $string => $extattr, ... },
#Optional / Default is undefined
    extensible_attributes => { $string => $string | $num, $string => [ $string | $num, ... ], ... }
#Optional / Default is undefined
    interval              => $uint,
#Optional / Default is undefined
    port                  => $uint,
#Optional / Default is undefined
    request               => $string,
#Optional / Default is undefined
    result                => 'ANY' | 'CODE_IS' | 'CODE_IS_NOT',
#Optional / Default is undefined
    result_code           => $uint,
#Optional / Default is undefined
    retry_down            => $uint,
#Optional / Default is undefined
    retry_up              => $uint,
#Optional / Default is undefined
    secure                => 'true' | 'false',
#Optional / Default is 'false'
    timeout               => $uint,
#Optional / Default is undefined
    validate_cert         => 'true' | 'false',
#Optional / Default is 'true'
 );
```

You cannot set both extattrs and extensible_attributes attributes at the same time.

SESSION METHODS

This section describes all the methods in a Session module that you can apply to a DTC HTTP Monitor object.

Infoblox::Session->add( )

Use this method to add an object to the Infoblox appliance. See Infoblox::Session->add() for parameters and return values.

Example

```
#construct an object
my $http_monitor = Infoblox::DTC::Monitor::HTTP->new(
   name  => 'http_monitor1',
);

#submit for addition
my $response = $session->add($http_monitor);
```

Infoblox::Session->get( )

Use this method to retrieve all the matching objects from the Infoblox appliance. See Infoblox::Session->get() for parameters and return values.

Key Reference

```
Apply the following attribute to get a specific object:

 name                  - Optional. A DTC HTTP Monitor name in a string format.
 comment               - Optional. A DTC HTTP Monitor comment.
 extattrs              - Optional. A hash reference containing extensible attributes.
 extensible_attributes - Optional. A hash reference that contains extensible attributes.
```

Example

```
my @retrieved_objs = $session->get(
    object => 'Infoblox::DTC::Monitor::HTTP',
    name   => 'http_monitor1',
);

my @retrieved_objs = $session->get(
    object => 'Infoblox::DTC::Monitor::HTTP',
    extensible_attributes => { 'Site' => 'Santa Clara' }
);
```

Infoblox::Session->modify( )

Use this method to modify an object in the Infoblox appliance. See Infoblox::Session->modify() for parameters and return values.

Example

```
#Use method to modify the comment.
$http_monitor->comment('this is a modified comment');
#Submit modification
my $response = $session->modify($http_monitor);
```

Infoblox::Session->remove( )

Use this method to remove an object from the Infoblox appliance. See Infoblox::Session->remove() for parameters and return values.

To remove a specific object, first use `get()` or `search()` to retrieve the specific object, and then submit this object for removal.

Example

```
#Get the objects with the same name
my @retrieved_objs = $session->get(
    object => 'Infoblox::DTC::Monitor::HTTP',
    name   => 'http_monitor1',
);
#Find the desired object from the retrieved list.
my $desired_http_monitor = $retrieved_objs[0];
#Submit for removal
my $response = $session->remove($desired_http_monitor);
```

Infoblox::Session->search( )

Use this method to search for objects in the Infoblox appliance. See Infoblox::Session->search() for parameters and return values.

Key Reference

```
Apply the following attribute to get a specific object:

 name                  - Optional. A DTC HTTP Monitor name in a string format.
 comment               - Optional. A DTC HTTP Monitor Descriptive comment.
 extattrs              - Optional. A hash reference containing extensible attributes.
 extensible_attributes - Optional. A hash reference that contains extensible attributes.
```

Example

```
my @retrieved_objs = $session->search(
    object => 'Infoblox::DTC::Monitor::HTTP',
    name   => 'http_monitor1',
);

my @retrieved_objs = $session->search(
    object => 'Infoblox::DTC::Monitor::HTTP',
    extensible_attributes => { 'Site' => 'Santa Clara' }
);
```

METHODS

This section describes all the methods that you can use to set or retrieve the attribute values of the object.

ciphers( )

Use this method to set or retrieve a list of ciphers for a secure HTTP connection.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

The valid value is an array of ciphers in a string format. Ciphers use an OpenSSL syntax. Empty array is equivalent to 'ALL' in the OpenSSL notation.

Returns

If you specified a parameter, the method returns 'true' when the modification succeeds, and returns 'false' when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#get ciphers value
```

```
my @ciphers = $http_monitor->ciphers();

#modify ciphers value
$http_monitor->ciphers(['RC4-MD5', 'KRB5-RC4-MD5']);
```

client_cert( )

Use this method to set or retrieve a client certificate, supplied in a secure HTTP mode if present.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

The valid value is an Infoblox::DTC::Certificate object.

Returns

If you specified a parameter, the method returns 'true' when the modification succeeds, and returns 'false' when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#get client_cert value
my $cert = $http_monitor->client_cert();

#modify client_cert value
$http_monitor->client_cert($cert);
```

comment( )

Use this method to set or retrieve a DTC HTTP Health Monitor object comment.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

The valid value is a comment in string format (UTF-8) with a maximum of 256 bytes.

Returns

If you specified a parameter, the method returns 'true' when the modification succeeds, and returns 'false' when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#get comment value
my $comment = $http_monitor->comment();

#modify comment value
$http_monitor->comment('desired comment');
```

content_check( )

Use this method to set or retrieve content check type.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

The valid values are 'NONE', 'MATCH' and 'EXTRACT'. The default is 'NONE'.

Returns

If you specified a parameter, the method returns 'true' when the modification succeeds, and returns 'false' when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#get content_check value
my $content_check = $http_monitor->content_check();

#modify content_check value
$http_monitor->content_check('EXTRACT');
```

content_check_input( )

Use this method to set or retrieve a portion of reponse to use as input for content check.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

The valid values are 'HEADERS', 'ALL' and 'BODY'. The default is 'ALL'.

Returns

If you specified a parameter, the method returns 'true' when the modification succeeds, and returns 'false' when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#get content_check_input value
my $content_check_input = $http_monitor->content_check_input();

#modify content_check_input value
$http_monitor->content_check_input('BODY');
```

content_check_regex( )

Use this method to set or retrieve a content check regular expression.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

The valid value is a desired regular expression in a string format.

Returns

If you specified a parameter, the method returns 'true' when the modification succeeds, and returns 'false' when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#get content_check_regex value
my $content_check_regex = $http_monitor->content_check_regex();

#modify content_check_regex value
$http_monitor->content_check_regex('SQL Error');
```

content_check_op( )

Use this method to set or retrieve a content check success criteria operator.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

The valid values are 'EQ' and 'NEQ' for 'MATCH' content check, and 'EQ', 'NEQ', 'LEQ' and 'GEQ' for 'EXTRACT' content check type.

Returns

If you specified a parameter, the method returns 'true' when the modification succeeds, and returns 'false' when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#get content_check_op value
my $content_check_op = $http_monitor->content_check_op();

#modify content_check_op value
$http_monitor->content_check_op('EQ');
```

content_extract_group( )

Use this method to set or retrieve a content extraction sub-expression to extract.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

The valid value is an unsigned integer between 0 and 8. The default is 0.

Returns

If you specified a parameter, the method returns 'true' when the modification succeeds, and returns 'false' when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#get content_extract_group value
my $content_extract_group = $http_monitor->content_extract_group();

#modify content_extract_group value
$http_monitor->content_extract_group(3);
```

content_extract_type( )

Use this method to set or retrieve a content extraction expected type for the extracted data.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

The valid values are 'INTEGER' and 'STRING'. The default is 'STRING'.

Returns

If you specified a parameter, the method returns 'true' when the modification succeeds, and returns 'false' when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#get content_extract_type value
my $content_extract_type = $http_monitor->content_extract_type();

#modify content_extract_type value
$http_monitor->content_extract_type('INTEGER');
```

content_extract_value( )

Use this method to set or retrieve a content extraction value to compare with extracted result.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

The valid value is a desired extraction value in string format. The default is undefined.

Returns

If you specified a parameter, the method returns 'true' when the modification succeeds, and returns 'false' when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#get content_extract_value value
my $content_extract_value = $http_monitor->content_extract_value();

#modify content_extract_value value
$http_monitor->content_extract_value('1');
```

enable_sni( )

Use this method to set or retrieve the flag that indicates whether the Server Name Indication (SNI) for HTTPS monitor is enabled.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

Specify 'true' to enable SNI for HTTPS monitor and 'false' to disable it. The default is 'false'.

Returns

If you specified a parameter, the method returns 'true' when the modification succeeds, and returns 'false' when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#get enable_sni value
my $enable_sni = $http_monitor->enable_sni();

#modify enable_sni value
$http_monitor->enable_sni('true');
```

extattrs( )

Use this method to set or retrieve the extensible attributes associated with a DTC HTTP Health Monitor object.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

Valid value is a hash reference containing the names of extensible attributes and their associated values (Infoblox::Grid::Extattr objects).

Returns

If you specified a parameter, the method returns 'true' when the modification succeeds, and returns 'false' when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#get extattrs value
my $ref_extattrs = $http_monitor->extattrs();

#Modify extattrs
$http_monitor->extattrs({ 'Site' => $extattr1, 'Administrator' => $extattr2 });
```

extensible_attributes( )

Use this method to set or retrieve the extensible attributes associated with a DTC HTTP Health Monitor object.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

For valid values for extensible attributes, see Infoblox::Grid::ExtensibleAttributeDef/Extensible Attribute Values.

Returns

If you specified a parameter, the method returns 'true' when the modification succeeds, and returns 'false' when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get extensible attributes
my $ref_extensible_attributes = $http_monitor->extensible_attributes();

#Modify extensible attributes
$http_monitor->extensible_attributes({'Site' => 'Santa Clara', 'Administrator' => ['Peter',
'Tom']});
```

interval( )

Use this method to set or retrieve the interval for the HTTP health check.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

The valid value is an unsigned integer.

Returns

If you specified a parameter, the method returns 'true' when the modification succeeds, and returns 'false' when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#get interval value
my $interval = $http_monitor->interval();

#modify interval value
$http_monitor->interval(10);
```

name( )

Use this method to set or retrieve a DTC HTTP Health Monitor name.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

The valid value is a desired name in a string format.

Returns

If you specified a parameter, the method returns 'true' when the modification succeeds, and returns 'false' when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#get name value
my $name = $http_monitor->name();

#modify name value
$http_monitor->name('http_monitor1');
```

port( )

Use this method to set or retrieve the port value for HTTP or HTTPS requests.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

The valid value is an unsigned integer between 1 and 65535.

Returns

If you specified a parameter, the method returns 'true' when the modification succeeds, and returns 'false' when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#get port value
my $port = $http_monitor->port();

#modify port value
$http_monitor->port(8080);
```

request( )

Use this method to set or retrieve an HTTP request to send.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

The valid value is a string up to 1024 characters.

Returns

If you specified a parameter, the method returns 'true' when the modification succeeds, and returns 'false' when the operation

fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#get request value
my $request = $http_monitor->request();

#modify request value
$http_monitor->request('GET /');
```

result( )

Use this method to set or retrieve the type of expected result.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

The valid values are 'ANY', 'CODE_IS' and 'CODE_IS_NOT'.

Returns

If you specified a parameter, the method returns 'true' when the modification succeeds, and returns 'false' when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#get result value
my $result = $http_monitor->result();

#modify result value
$http_monitor->result('CODE_IS');
```

result_code( )

Use this method to set or retrieve the expected return code.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

The valid value is an unsigned integer between 0 and 999.

Returns

If you specified a parameter, the method returns 'true' when the modification succeeds, and returns 'false' when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#get result_code value
my $result_code = $http_monitor->result_code();

#modify result_code value
$http_monitor->result_code(300);
```

retry_down( )

Use this method to set or retrieve the number of times the server appears offline after it was online so it is treated as as dead.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

The valid value is an unsigned integer between 1 and 10.

Returns

If you specified a parameter, the method returns 'true' when the modification succeeds, and returns 'false' when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#get retry_down value
my $retry_down = $http_monitor->retry_down();

#modify retry_down value
$http_monitor->retry_down(3);
```

retry_up( )

Use this method to set or retrieve the number of times the server appears online after it was offline so it is treated as alive.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

The valid value is an unsigned integer between 1 and 10.

Returns

If you specified a parameter, the method returns 'true' when the modification succeeds, and returns 'false' when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#get retry_up value
my $retry_up = $http_monitor->retry_up();

#modify retry_up value
$http_monitor->retry_up(3);
```

secure( )

Use this method to disable or enable a secure connection.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

Specify 'true' to disable a secure connection or 'false' to enable it. The default value is 'false'.

Returns

If you specified a parameter, the method returns 'true' when the modification succeeds, and returns 'false' when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#get secure value
my $secure = $http_monitor->secure();

#modify secure value
$http_monitor->secure('true');
```

timeout( )

Use this method to set or retrieve the value of a timeout for HTTP health check.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

The valid value is an unsigned integer between 1 and 15.

Returns

If you specified a parameter, the method returns 'true' when the modification succeeds, and returns 'false' when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#get timeout value
my $timeout = $http_monitor->timeout();
```

```
#modify timeout value
$http_monitor->timeout(7);
```

validate_cert( )

Use this method to set or retrieve the flag that indicates whether the validation of the remote server's certificate is enabled.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

Specify 'true' to enable remote server's certificate validation and 'false' to disable it. The default is 'true'.

Returns

If you specified a parameter, the method returns 'true' when the modification succeeds, and returns 'false' when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#get validate_cert value
my $validate_cert = $http_monitor->validate_cert();

#modify validate_cert value
$http_monitor->validate_cert('false');
```

AUTHOR

Infoblox Inc. http://www.infoblox.com/

SEE ALSO

Infoblox::Session, Infoblox::Session->add(), Infoblox::Session->get(), Infoblox::Session->modify(), Infoblox::Session->remove(), Infoblox::Session->search(), Infoblox::Grid::Extattr, Infoblox::Grid::ExtensibleAttributeDef/Extensible Attribute Values, Infoblox::DTC::Certificate.

COPYRIGHT

Copyright (c) 2017 Infoblox Inc.

Infoblox::DTC::Monitor::ICMP - A DTC ICMP Health Monitor object.

- NAME
- DESCRIPTION
- CONSTRUCTOR
- SESSION METHODS
  - Infoblox::Session->add( )
  - Infoblox::Session->get( )
  - Infoblox::Session->modify( )
  - Infoblox::Session->remove( )
  - Infoblox::Session->search( )
- METHODS
  - comment( )
  - extattrs( )
  - extensible_attributes( )
  - interval( )
  - name( )
  - retry_down( )
  - retry_up( )
  - timeout( )
- AUTHOR
- SEE ALSO
- COPYRIGHT

NAME

Infoblox::DTC::Monitor::ICMP - A DTC ICMP Health Monitor object.

DESCRIPTION

A DTC ICMP Health Monitor object.

CONSTRUCTOR

```
 my $icmp_monitor = Infoblox::DTC::Monitor::ICMP->new(
    name                  => $string,
#Required
    comment               => $string,
#Optional / Default is undefined
    interval              => $uint,
#Optional / Default is undefined
    retry_down            => $uint,
#Optional / Default is undefined
    retry_up              => $uint,
#Optional / Default is undefined
    timeout               => $uint,
#Optional / Default is undefined
    extattrs              => { $string => $extattr, ... },
#Optional / Default is undefined
    extensible_attributes => { $string => $string | $num, $string => [ $string | $num, ... ], ... }
#Optional / Default is undefined
 );
```

You cannot set both extattrs and extensible_attributes attributes at the same time.

SESSION METHODS

This section describes all the methods in a Session module that you can apply to a DTC ICMP Monitor object

Infoblox::Session->add( )

Use this method to add an object to the Infoblox appliance. See Infoblox::Session->add() for parameters and return values.

Example

```
#construct an object
my $icmp_monitor = Infoblox::DTC::Monitor::ICMP->new(
   name  => 'icmp_monitor1',
);

#submit for addition
```

```
my $response = $session->add($icmp_monitor);
```

Infoblox::Session->get( )

Use this method to retrieve all the matching objects from the Infoblox appliance. See Infoblox::Session->get() for parameters and return values.

Key Reference

```
Apply the following attribute to get a specific object:

 name                  - Optional. A DTC ICMP Monitor name in a string format.
 comment               - Optional. A DTC ICMP Monitor comment.
 extattrs              - Optional. A hash reference containing extensible attributes.
 extensible_attributes - Optional. A hash reference that contains extensible attributes.
```

Example

```
my @retrieved_objs = $session->get(
    object => 'Infoblox::DTC::Monitor::ICMP',
    name   => 'icmp_monitor1',
);

my @retrieved_objs = $session->get(
    object => 'Infoblox::DTC::Monitor::ICMP',
    extensible_attributes => { 'Site' => 'Santa Clara' }
);
```

Infoblox::Session->modify( )

Use this method to modify an object in the Infoblox appliance. See Infoblox::Session->modify() for parameters and return values.

Example

```
#Use method to modify the comment.
$icmp_monitor->comment('this is a modified comment');
#Submit modification
my $response = $session->modify($icmp_monitor);
```

Infoblox::Session->remove( )

Use this method to remove an object from the Infoblox appliance. See Infoblox::Session->remove() for parameters and return values.

To remove a specific object, first use `get()` or `search()` to retrieve the specific object, and then submit this object for removal.

Example

```
#Get the objects with the same name
my @retrieved_objs = $session->get(
    object => 'Infoblox::DTC::Monitor::ICMP',
    name   => 'icmp_monitor1',
);
#Find the desired object from the retrieved list.
my $desired_icmp_monitor = $retrieved_objs[0];
#Submit for removal
my $response = $session->remove($desired_icmp_monitor);
```

Infoblox::Session->search( )

Use this method to search for objects in the Infoblox appliance. See Infoblox::Session->search() for parameters and return values.

Key Reference

```
Apply the following attribute to get a specific object:

 name                  - Optional. A DTC ICMP Monitor name in a string format.
 comment               - Optional. A DTC ICMP Monitor comment.
 extattrs              - Optional. A hash reference containing extensible attributes.
 extensible_attributes - Optional. A hash reference that contains extensible attributes.
```

Example

```
my @retrieved_objs = $session->search(
    object => 'Infoblox::DTC::Monitor::ICMP',
    name   => 'icmp_monitor1',
);

my @retrieved_objs = $session->search(
```

```
        object => 'Infoblox::DTC::Monitor::ICMP',
        extensible_attributes => { 'Site' => 'Santa Clara' }
    );
```

METHODS

This section describes all the methods that you can use to set or retrieve the attribute values of the object.

comment( )

Use this method to set or retrieve a DTC ICMP Health Monitor object comment.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

The valid value is a comment in string format (UTF-8) with a maximum of 256 bytes.

Returns

If you specified a parameter, the method returns 'true' when the modification succeeds, and returns 'false' when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#get comment value
my $comment = $icmp_monitor->comment();

#modify comment value
$icmp_monitor->comment('desired comment');
```

extattrs( )

Use this method to set or retrieve the extensible attributes associated with a DTC ICMP Health Monitor object.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

Valid value is a hash reference containing the names of extensible attributes and their associated values (Infoblox::Grid::Extattr objects).

Returns

If you specified a parameter, the method returns 'true' when the modification succeeds, and returns 'false' when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#get extattrs value
my $ref_extattrs = $icmp_monitor->extattrs();

#Modify extattrs
$icmp_monitor->extattrs({ 'Site' => $extattr1, 'Administrator' => $extattr2 });
```

extensible_attributes( )

Use this method to set or retrieve the extensible attributes associated with a DTC ICMP Health Monitor object.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

For valid values for extensible attributes, see Infoblox::Grid::ExtensibleAttributeDef/Extensible Attribute Values.

Returns

If you specified a parameter, the method returns 'true' when the modification succeeds, and returns 'false' when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get extensible attributes
my $ref_extensible_attributes = $icmp_monitor->extensible_attributes();

#Modify extensible attributes
$icmp_monitor->extensible_attributes({'Site' => 'Santa Clara', 'Administrator' => ['Peter',
'Tom']});
```

interval( )

Use this method to set or retrieve the time interval for the ICMP health check.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

The valid value is an unsigned integer.

Returns

If you specified a parameter, the method returns 'true' when the modification succeeds, and returns 'false' when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#get interval value
my $interval = $icmp_monitor->interval();

#modify interval value
$icmp_monitor->interval(10);
```

name( )

Use this method to set or retrieve a DTC ICMP Health Monitor name.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

The valid value is a desired name in a string format.

Returns

If you specified a parameter, the method returns 'true' when the modification succeeds, and returns 'false' when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#get name value
my $name = $icmp_monitor->name();

#modify name value
$icmp_monitor->name('icmp_monitor1');
```

retry_down( )

Use this method to set or retrieve the number of times the server appears offline after it was online so it is treated as dead.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

The valid value is an unsigned integer between 1 and 10.

Returns

If you specified a parameter, the method returns 'true' when the modification succeeds, and returns 'false' when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#get retry_down value
my $retry_down = $icmp_monitor->retry_down();
```

```
#modify retry_down value
$icmp_monitor->retry_down(3);
```

retry_up( )

Use this method to set or retrieve the number of times the server appears online after it was offline so it is treated as alive.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

The valid value is an unsigned integer between 1 and 10.

Returns

If you specified a parameter, the method returns 'true' when the modification succeeds, and returns 'false' when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#get retry_up value
my $retry_up = $icmp_monitor->retry_up();

#modify retry_up value
$icmp_monitor->retry_up(3);
```

timeout( )

Use this method to set or retrieve the value of a timeout for ICMP health check.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

The valid value is an unsigned integer between 1 and 15.

Returns

If you specified a parameter, the method returns 'true' when the modification succeeds, and returns 'false' when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#get timeout value
my $timeout = $icmp_monitor->timeout();

#modify timeout value
$icmp_monitor->timeout(7);
```

AUTHOR

Infoblox Inc. http://www.infoblox.com/

SEE ALSO

Infoblox::Session, Infoblox::Session->add(), Infoblox::Session->get(), Infoblox::Session->modify(), Infoblox::Session->remove(), Infoblox::Session->search(), Infoblox::Grid::Extattr, Infoblox::Grid::ExtensibleAttributeDef/Extensible Attribute Values.

COPYRIGHT

Copyright (c) 2017 Infoblox Inc.

Infoblox::DTC::Monitor::PDP - A DTC PDP Health Monitor object.

- NAME
- DESCRIPTION
- CONSTRUCTOR
- SESSION METHODS
  - Infoblox::Session->add( )
  - Infoblox::Session->get( )
  - Infoblox::Session->modify( )
  - Infoblox::Session->remove( )
  - Infoblox::Session->search( )
- METHODS
  - comment( )
  - extattrs( )
  - extensible_attributes( )
  - interval( )
  - name( )
  - port( )
  - retry_down( )
  - retry_up( )
  - timeout( )
- AUTHOR
- SEE ALSO
- COPYRIGHT

NAME

Infoblox::DTC::Monitor::PDP - A DTC PDP Health Monitor object.

DESCRIPTION

A DTC PDP Health Monitor object.

CONSTRUCTOR

```
 my $pdp_monitor = Infoblox::DTC::Monitor::PDP->new(
    name                  => $string,
#Required
    comment               => $string,
#Optional / Default is undefined
    interval              => $uint,
#Optional / Default is undefined
    port                  => $uint,
#Optional / Default is undefined
    retry_down            => $uint,
#Optional / Default is undefined
    retry_up              => $uint,
#Optional / Default is undefined
    timeout               => $uint,
#Optional / Default is undefined
    extattrs              => { $string => $extattr, ... },
#Optional / Default is undefined
    extensible_attributes => { $string => $string | $num, $string => [ $string | $num, ... ], ... }
#Optional / Default is undefined
 );
```

You cannot set both extattrs and extensible_attributes attributes at the same time.

SESSION METHODS

This section describes all the methods in a Session module that you can apply to a DTC PDP Monitor object.

Infoblox::Session->add( )

Use this method to add an object to the Infoblox appliance. See Infoblox::Session->add() for parameters and return values.

Example

```
#construct an object
my $pdp_monitor = Infoblox::DTC::Monitor::PDP->new(
   name  => 'pdp_monitor1',
```

```
);

#submit for addition
my $response = $session->add($pdp_monitor);
```

Infoblox::Session->get( )

Use this method to retrieve all the matching objects from the Infoblox appliance. See Infoblox::Session->get() for parameters and return values.

Key Reference

```
Apply the following attribute to get a specific object:

 name                  - Optional. A DTC PDP Monitor name in a string format.
 comment               - Optional. A DTC PDP Monitor comment.
 extattrs              - Optional. A hash reference containing extensible attributes.
 extensible_attributes - Optional. A hash reference that contains extensible attributes.
```

Example

```
my @retrieved_objs = $session->get(
    object => 'Infoblox::DTC::Monitor::PDP',
    name   => 'pdp_monitor1',
);

my @retrieved_objs = $session->get(
    object => 'Infoblox::DTC::Monitor::PDP',
    extensible_attributes => { 'Site' => 'Santa Clara' }
);
```

Infoblox::Session->modify( )

Use this method to modify an object in the Infoblox appliance. See Infoblox::Session->modify() for parameters and return values.

Example

```
#Use method to modify the comment.
$pdp_monitor->comment('this is a modified comment');
#Submit modification
my $response = $session->modify($pdp_monitor);
```

Infoblox::Session->remove( )

Use this method to remove an object from the Infoblox appliance. See Infoblox::Session->remove() for parameters and return values.

To remove a specific object, first use `get()` or `search()` to retrieve the specific object, and then submit this object for removal.

Example

```
#Get the objects with the same name
my @retrieved_objs = $session->get(
    object => 'Infoblox::DTC::Monitor::PDP',
    name   => 'pdp_monitor1',
);
#Find the desired object from the retrieved list.
my $desired_pdp_monitor = $retrieved_objs[0];
#Submit for removal
my $response = $session->remove($desired_pdp_monitor);
```

Infoblox::Session->search( )

Use this method to search for objects in the Infoblox appliance. See Infoblox::Session->search() for parameters and return values.

Key Reference

```
Apply the following attribute to get a specific object:

 name                  - Optional. A DTC PDP Monitor name in a string format.
 comment               - Optional. A DTC PDP Monitor comment.
 extattrs              - Optional. A hash reference containing extensible attributes.
 extensible_attributes - Optional. A hash reference that contains extensible attributes.
```

Example

```
my @retrieved_objs = $session->search(
    object => 'Infoblox::DTC::Monitor::PDP',
    name   => 'pdp_monitor1',
```

```
);

my @retrieved_objs = $session->search(
    object => 'Infoblox::DTC::Monitor::PDP',
    extensible_attributes => { 'Site' => 'Santa Clara' }
);
```

METHODS

This section describes all the methods that you can use to set or retrieve the attribute values of the object.

comment( )

Use this method to set or retrieve a DTC PDP Health Monitor object comment.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

The valid value is a comment in string format (UTF-8) with a maximum of 256 bytes.

Returns

If you specified a parameter, the method returns 'true' when the modification succeeds, and returns 'false' when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#get comment value
my $comment = $pdp_monitor->comment();

#modify comment value
$pdp_monitor->comment('desired comment');
```

extattrs( )

Use this method to set or retrieve the extensible attributes associated with a DTC PDP Health Monitor object.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

Valid value is a hash reference containing the names of extensible attributes and their associated values (Infoblox::Grid::Extattr objects).

Returns

If you specified a parameter, the method returns 'true' when the modification succeeds, and returns 'false' when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#get extattrs value
my $ref_extattrs = $pdp_monitor->extattrs();

#Modify extattrs
$pdp_monitor->extattrs({ 'Site' => $extattr1, 'Administrator' => $extattr2 });
```

extensible_attributes( )

Use this method to set or retrieve the extensible attributes associated with a DTC PDP Health Monitor object.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

For valid values for extensible attributes, see Infoblox::Grid::ExtensibleAttributeDef/Extensible Attribute Values.

Returns

If you specified a parameter, the method returns 'true' when the modification succeeds, and returns 'false' when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get extensible attributes
my $ref_extensible_attributes = $pdp_monitor->extensible_attributes();

#Modify extensible attributes
$pdp_monitor->extensible_attributes({'Site' => 'Santa Clara', 'Administrator' => ['Peter', 'Tom']});
```

interval( )

Use this method to set or retrieve the time interval for the PDP health check.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

The valid value is an unsigned integer.

Returns

If you specified a parameter, the method returns 'true' when the modification succeeds, and returns 'false' when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#get interval value
my $interval = $pdp_monitor->interval();

#modify interval value
$pdp_monitor->interval(10);
```

name( )

Use this method to set or retrieve a DTC PDP Health Monitor name.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

The valid value is a desired name in a string format.

Returns

If you specified a parameter, the method returns 'true' when the modification succeeds, and returns 'false' when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#get name value
my $name = $pdp_monitor->name();

#modify name value
$pdp_monitor->name('pdp_monitor1');
```

port( )

Use this method to set or retrieve the port value for PDP requests.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

The valid value is an unsigned integer between 1 and 65535.

Returns

If you specified a parameter, the method returns 'true' when the modification succeeds, and returns 'false' when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#get port value
my $port = $pdp_monitor->port();

#modify port value
$pdp_monitor->port(6030);
```

retry_down( )

Use this method to set or retrieve the number of times the server appears offline after it was online so it is treated as dead.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

The valid value is an unsigned integer between 1 and 10.

Returns

If you specified a parameter, the method returns 'true' when the modification succeeds, and returns 'false' when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#get retry_down value
my $retry_down = $pdp_monitor->retry_down();

#modify retry_down value
$pdp_monitor->retry_down(3);
```

retry_up( )

Use this method to set or retrieve the number of times the server appears online after it was offline so it is treated as alive.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

The valid value is an unsigned integer between 1 and 10.

Returns

If you specified a parameter, the method returns 'true' when the modification succeeds, and returns 'false' when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#get retry_up value
my $retry_up = $pdp_monitor->retry_up();

#modify retry_up value
$pdp_monitor->retry_up(3);
```

timeout( )

Use this method to set or retrieve the value of a timeout for PDP health check.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

The valid value is an unsigned integer between 1 and 15.

Returns

If you specified a parameter, the method returns 'true' when the modification succeeds, and returns 'false' when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#get timeout value
my $timeout = $pdp_monitor->timeout();

#modify timeout value
```

```
$pdp_monitor->timeout(7);
```

AUTHOR

Infoblox Inc. http://www.infoblox.com/

SEE ALSO

Infoblox::Session, Infoblox::Session->add(), Infoblox::Session->get(), Infoblox::Session->modify(), Infoblox::Session->remove(), Infoblox::Session->search(), Infoblox::Grid::Extattr, Infoblox::Grid::ExtensibleAttributeDef/Extensible Attribute Values.

COPYRIGHT

Copyright (c) 2017 Infoblox Inc.

Infoblox::DTC::Monitor::SIP - A DTC SIP Health Monitor object.

- NAME
- DESCRIPTION
- CONSTRUCTOR
- SESSION METHODS
  - Infoblox::Session->add( )
  - Infoblox::Session->get( )
  - Infoblox::Session->modify( )
  - Infoblox::Session->remove( )
  - Infoblox::Session->search( )
- METHODS
  - ciphers( )
  - client_cert( )
  - comment( )
  - extattrs( )
  - extensible_attributes( )
  - interval( )
  - name( )
  - port( )
  - request( )
  - result( )
  - result_code( )
  - retry_down( )
  - retry_up( )
  - timeout( )
  - transport( )
  - validate_cert( )
- AUTHOR
- SEE ALSO
- COPYRIGHT

NAME

Infoblox::DTC::Monitor::SIP - A DTC SIP Health Monitor object.

DESCRIPTION

The DTC SIP Health Monitor determines the health of a SIP servers such as SIP Proxies and Session Border Controllers, and SIP gateways by issuing SIP options to a server and examining the response provided by the server. The service is considered available if the received response matches the expected response.

CONSTRUCTOR

```
 my $sip_monitor = Infoblox::DTC::Monitor::SIP->new(
    name                => $string,
#Required
    ciphers             => [$cipher1, $cipher2, ...],
#Optional / Default is undefined
    client_cert         => $certificate
#Optional / Default is undefined
    comment             => $string,
#Optional / Default is undefined
    interval            => $uint,
#Optional / Default is undefined
    port                => $uint,
#Optional / Default is undefined
    request             => $string,
#Optional / Default is undefined
    result              => 'ANY' | 'CODE_IS' | 'CODE_IS_NOT',
#Optional / Default is undefined
    result_code         => $uint,
#Optional / Default is undefined
    retry_down          => $uint,
#Optional / Default is undefined
    retry_up            => $uint,
#Optional / Default is undefined
    timeout             => $uint,
#Optional / Default is undefined
    transport           => 'TCP' | 'UDP' | 'TLS' | 'SIPS',
#Optional / Default is undefined
    validate_cert       => 'true' | 'false',
#Optional / Default is 'false'
```

```
    extattrs              => { $string => $extattr, ... },
#Optional / Default is undefined
    extensible_attributes => { $string => $string | $num, $string => [ $string | $num, ... ], ... }
#Optional / Default is undefined
 );
```

You cannot set both extattrs and extensible_attributes attributes at the same time.

SESSION METHODS

This section describes all the methods in a Session module that you can apply to a DTC SIP Monitor object

Infoblox::Session->add( )

Use this method to add an object to the Infoblox appliance. See Infoblox::Session->add() for parameters and return values.

Example

```
#construct an object
my $sip_monitor = Infoblox::DTC::Monitor::SIP->new(
   name  => 'sip_monitor1',
);

#submit for addition
my $response = $session->add($sip_monitor);
```

Infoblox::Session->get( )

Use this method to retrieve all the matching objects from the Infoblox appliance. See Infoblox::Session->get() for parameters and return values.

Key Reference

```
Apply the following attribute to get a specific object:

 name                  - Optional. A DTC SIP Monitor name in a string format.
 comment               - Optional. A DTC SIP Monitor comment.
 extattrs              - Optional. A hash reference containing extensible attributes.
 extensible_attributes - Optional. A hash reference that contains extensible attributes.
```

Example

```
my @retrieved_objs = $session->get(
    object => 'Infoblox::DTC::Monitor::SIP',
    name   => 'sip_monitor1',
);

my @retrieved_objs = $session->get(
    object => 'Infoblox::DTC::Monitor::SIP',
    extensible_attributes => { 'Site' => 'Santa Clara' }
);
```

Infoblox::Session->modify( )

Use this method to modify an object in the Infoblox appliance. See Infoblox::Session->modify() for parameters and return values.

Example

```
#Use method to modify the comment.
$sip_monitor->comment('this is a modified comment');
#Submit modification
my $response = $session->modify($sip_monitor);
```

Infoblox::Session->remove( )

Use this method to remove an object from the Infoblox appliance. See Infoblox::Session->remove() for parameters and return values.

To remove a specific object, first use `get()` or `search()` to retrieve the specific object, and then submit this object for removal.

Example

```
#Get the objects with the same name
my @retrieved_objs = $session->get(
    object => 'Infoblox::DTC::Monitor::SIP',
    name   => 'sip_monitor1',
);
#Find the desired object from the retrieved list.
```

```
my $desired_sip_monitor = $retrieved_objs[0];
#Submit for removal
my $response = $session->remove($desired_sip_monitor);
```

Infoblox::Session->search( )

Use this method to search for objects in the Infoblox appliance. See Infoblox::Session->search() for parameters and return values.

Key Reference

```
Apply the following attribute to get a specific object:

 name                 - Optional. A DTC SIP Monitor name in a string format.
 comment              - Optional. A DTC SIP Monitor Descriptive comment.
 extattrs             - Optional. A hash reference containing extensible attributes.
 extensible_attributes - Optional. A hash reference that contains extensible attributes.
```

Example

```
my @retrieved_objs = $session->search(
    object => 'Infoblox::DTC::Monitor::SIP',
    name   => 'sip_monitor1',
);

my @retrieved_objs = $session->search(
    object => 'Infoblox::DTC::Monitor::SIP',
    extensible_attributes => { 'Site' => 'Santa Clara' }
);
```

METHODS

This section describes all the methods that you can use to set or retrieve the attribute values of the object.

ciphers( )

Use this method to set or retrieve a list of ciphers for a secure TLS or SIPS connection.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

The valid value is an array of ciphers in a string format. Ciphers use an OpenSSL syntax. Empty array is equivalent to 'ALL' in an OpenSSL notation.

Returns

If you specified a parameter, the method returns 'true' when the modification succeeds, and returns 'false' when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#get ciphers value
my @ciphers = $sip_monitor->ciphers();

#modify ciphers value
$sip_monitor->ciphers(['RC4-MD5', 'KRB5-RC4-MD5']);
```

client_cert( )

Use this method to set or retrieve a client certificate, supplied in a secure TLS or SIPS mode if present.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

The valid value is an Infoblox::DTC::Certificate object.

Returns

If you specified a parameter, the method returns 'true' when the modification succeeds, and returns 'false' when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#get client_cert value
my $cert = $sip_monitor->client_cert();

#modify client_cert value
$sip_monitor->client_cert($cert);
```

comment( )

Use this method to set or retrieve the comment for a DTC SIP Health Monitor object.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

The valid value is a comment in string format (UTF-8) with a maximum of 256 bytes.

Returns

If you specified a parameter, the method returns 'true' when the modification succeeds, and returns 'false' when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#get comment value
my $comment = $sip_monitor->comment();

#modify comment value
$sip_monitor->comment('desired comment');
```

extattrs( )

Use this method to set or retrieve the extensible attributes associated with a DTC SIP Health Monitor object.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

Valid value is a hash reference containing the names of extensible attributes and their associated values (Infoblox::Grid::Extattr objects).

Returns

If you specified a parameter, the method returns 'true' when the modification succeeds, and returns 'false' when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#get extattrs value
my $ref_extattrs = $sip_monitor->extattrs();

#Modify extattrs
$sip_monitor->extattrs({ 'Site' => $extattr1, 'Administrator' => $extattr2 });
```

extensible_attributes( )

Use this method to set or retrieve the extensible attributes associated with a DTC SIP Health Monitor object.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

For valid values for extensible attributes, see Infoblox::Grid::ExtensibleAttributeDef/Extensible Attribute Values.

Returns

If you specified a parameter, the method returns 'true' when the modification succeeds, and returns 'false' when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get extensible attributes
my $ref_extensible_attributes = $sip_monitor->extensible_attributes();
```

```
#Modify extensible attributes
$sip_monitor->extensible_attributes({'Site' => 'Santa Clara', 'Administrator' => ['Peter', 'Tom']});
```

interval( )

Use this method to set or retrieve the time interval for the SIP health check.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

The valid value is an unsigned integer.

Returns

If you specified a parameter, the method returns 'true' when the modification succeeds, and returns 'false' when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#get interval value
my $interval = $sip_monitor->interval();

#modify interval value
$sip_monitor->interval(10);
```

name( )

Use this method to set or retrieve a DTC SIP Health Monitor name.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

The valid value is a desired name in a string format.

Returns

If you specified a parameter, the method returns 'true' when the modification succeeds, and returns 'false' when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#get name value
my $name = $sip_monitor->name();

#modify name value
$sip_monitor->name('sip_monitor1');
```

port( )

Use this method to set or retrieve the port value for SIP or SIPS requests.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

The valid value is an unsigned integer between 1 and 65535.

Returns

If you specified a parameter, the method returns 'true' when the modification succeeds, and returns 'false' when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#get port value
my $port = $sip_monitor->port();

#modify port value
$sip_monitor->port(8080);
```

request( )

Use this method to set or retrieve an SIP request to send.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

The valid value is a string of 1024 characters maximum.

Returns

If you specified a parameter, the method returns 'true' when the modification succeeds, and returns 'false' when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#get request value
my $request = $sip_monitor->request();

#modify request value
$sip_monitor->request('GET /');
```

result( )

Use this method to set or retrieve the type of expected result.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

The valid values are 'ANY', 'CODE_IS' and 'CODE_IS_NOT'.

Returns

If you specified a parameter, the method returns 'true' when the modification succeeds, and returns 'false' when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#get result value
my $result = $sip_monitor->result();

#modify result value
$sip_monitor->result('CODE_IS');
```

result_code( )

Use this method to set or retrieve the expected return code.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

The valid value is an unsigned integer between 0 and 999.

Returns

If you specified a parameter, the method returns 'true' when the modification succeeds, and returns 'false' when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#get result_code value
my $result_code = $sip_monitor->result_code();

#modify result_code value
$sip_monitor->result_code(300);
```

retry_down( )

Use this method to set or retrieve the number of times the server appears offline after it was online so it is treated as dead.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

The valid value is an unsigned integer between 1 and 10.

Returns

If you specified a parameter, the method returns 'true' when the modification succeeds, and returns 'false' when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#get retry_down value
my $retry_down = $sip_monitor->retry_down();

#modify retry_down value
$sip_monitor->retry_down(3);
```

retry_up( )

Use this method to set or retrieve the number of times the server appears online after it was offline so it is treated as alive.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

The valid value is an unsigned integer between 1 and 10.

Returns

If you specified a parameter, the method returns 'true' when the modification succeeds, and returns 'false' when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#get retry_up value
my $retry_up = $sip_monitor->retry_up();

#modify retry_up value
$sip_monitor->retry_up(3);
```

timeout( )

Use this method to set or retrieve the value of a timeout for SIP health check.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

The valid value is an unsigned integer between 1 and 15.

Returns

If you specified a parameter, the method returns 'true' when the modification succeeds, and returns 'false' when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#get timeout value
my $timeout = $sip_monitor->timeout();

#modify timeout value
$sip_monitor->timeout(7);
```

transport( )

Use this method of set or retrieve the type of transport layer protocol being used for SIP health check.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

The valid values are 'TCP', 'UDP', 'SIPS' and 'TLS'.

Returns

If you specified a parameter, the method returns 'true' when the modification succeeds, and returns 'false' when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#get transport value
my $transport = $sip_monitor->transport();

#modify transport value
$sip_monitor->transport('UDP');
```

validate_cert( )

Use this method to set or retrieve the flag that indicates whether the validation of the remote server's certificate is enabled.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

Specify 'true' to enable remote server's certificate validation and 'false' to disable it. The default is 'true'.

Returns

If you specified a parameter, the method returns 'true' when the modification succeeds, and returns 'false' when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#get validate_cert value
my $validate_cert = $http_monitor->validate_cert();

#modify validate_cert value
$http_monitor->validate_cert('false');
```

AUTHOR

Infoblox Inc. http://www.infoblox.com/

SEE ALSO

Infoblox::Session, Infoblox::Session->add(), Infoblox::Session->get(), Infoblox::Session->modify(), Infoblox::Session->remove(), Infoblox::Session->search(), Infoblox::Grid::Extattr, Infoblox::Grid::ExtensibleAttributeDef/Extensible Attribute Values, Infoblox::DTC::Certificate.

COPYRIGHT

Copyright (c) 2017 Infoblox Inc.

Infoblox::DTC::Monitor::SNMP - A DTC SNMP Health Monitor object.

- NAME
- DESCRIPTION
- CONSTRUCTOR
- SESSION METHODS
  - Infoblox::Session->add( )
  - Infoblox::Session->get( )
  - Infoblox::Session->modify( )
  - Infoblox::Session->remove( )
  - Infoblox::Session->search( )
- METHODS
  - comment( )
  - community( )
  - context( )
  - engine_id( )
  - extattrs( )
  - extensible_attributes( )
  - interval( )
  - name( )
  - oids( )
  - port( )
  - retry_down( )
  - retry_up( )
  - timeout( )
  - user( )
  - version( )
- AUTHOR
- SEE ALSO
- COPYRIGHT

NAME

Infoblox::DTC::Monitor::SNMP - A DTC SNMP Health Monitor object.

DESCRIPTION

The DTC SNMP Health Monitor determines the health of SNMP servers such as SNMP Proxies and Session Border Controllers, and SNMP gateways by issuing SNMP options to a server and examining the response provided by the server. The service is considered available if the received response matches the expected response.

CONSTRUCTOR

```
 my $snmp_monitor = Infoblox::DTC::Monitor::SNMP->new(
    name                => $string,
#Required
    comment             => $string,
#Optional / Default is undefined
    community           => $string,
#Optional / Default is 'public'
    interval            => $uint,
#Optional / Default is 5
    oids                => [$oid1, $oid2, ...],
#Optional / Default is empty list
    port                => $uint,
#Optional / Default is 161
    retry_down          => $uint,
#Optional / Default is 1
    retry_up            => $uint,
#Optional / Default is 1
    timeout             => $uint,
#Optional / Default is 15
    version             => 'V1' | 'V2C' | 'V3',
#Optional / Default is 'V2C'
    context             => $string,
#Optional / Default is undefined
    engine_id           => $string,
#Optional / Default is undefined
    user                => $string,
#Optional / Default is undefined
    extattrs            => { $string => $extattr, ... },
#Optional / Default is undefined
    extensible_attributes => { $string => $string | $num, $string => [ $string | $num, ... ], ... }
```

```
#Optional / Default is undefined
 );
```

You cannot set both extattrs and extensible_attributes attributes at the same time.

SESSION METHODS

This section describes all the methods in a Session module that you can apply to a DTC SNMP Monitor object

Infoblox::Session->add( )

Use this method to add an object to the Infoblox appliance. See Infoblox::Session->add() for parameters and return values.

Example

```
#construct an object
my $snmp_monitor = Infoblox::DTC::Monitor::SNMP->new(
   name  => 'snmp_monitor1',
);

#submit for addition
my $response = $session->add($snmp_monitor);
```

Infoblox::Session->get( )

Use this method to retrieve all the matching objects from the Infoblox appliance. See Infoblox::Session->get() for parameters and return values.

Key Reference

```
Apply the following attribute to get a specific object:

 name                  - Optional. A DTC SNMP Monitor name in string format.
 comment               - Optional. A DTC SNMP Monitor descriptive comment.
 engine_id             - Optional. A DTC SNMPv3 Monitor engine identifier.
 context               - Optional. A DTC SNMPv3 Monitor context.
 extattrs              - Optional. A hash reference containing extensible attributes.
 extensible_attributes - Optional. A hash reference that contains extensible attributes.
```

Example

```
my @retrieved_objs = $session->get(
    object => 'Infoblox::DTC::Monitor::SNMP',
    name   => 'snmp_monitor1',
);

my @retrieved_objs = $session->get(
    object => 'Infoblox::DTC::Monitor::SNMP',
    extensible_attributes => { 'Site' => 'Santa Clara' }
);
```

Infoblox::Session->modify( )

Use this method to modify an object in the Infoblox appliance. See Infoblox::Session->modify() for parameters and return values.

Example

```
#Use method to modify the comment.
$snmp_monitor->comment('this is a modified comment');
#Submit modification
my $response = $session->modify($snmp_monitor);
```

Infoblox::Session->remove( )

Use this method to remove an object from the Infoblox appliance. See Infoblox::Session->remove() for parameters and return values.

To remove a specific object, first use `get()` or `search()` to retrieve the specific object, and then submit this object for removal.

Example

```
#Get the objects with the same name
my @retrieved_objs = $session->get(
    object => 'Infoblox::DTC::Monitor::SNMP',
    name   => 'snmp_monitor1',
);
#Find the desired object from the retrieved list.
my $desired_snmp_monitor = $retrieved_objs[0];
```

```
#Submit for removal
my $response = $session->remove($desired_snmp_monitor);
```

Infoblox::Session->search( )

Use this method to search for objects in the Infoblox appliance. See Infoblox::Session->search() for parameters and return values.

Key Reference

```
Apply the following attribute to get a specific object:

 name                 - Optional. A DTC SNMP Monitor name in string format (regexp).
 comment              - Optional. A DTC SNMP Monitor descriptive comment (regexp).
 engine_id            - Optional. A DTC SNMPv3 Monitor engine identifier (regexp).
 context              - Optional. A DTC SNMPv3 Monitor context (regexp).
 extattrs             - Optional. A hash reference containing extensible attributes.
 extensible_attributes - Optional. A hash reference that contains extensible attributes.
```

Example

```
my @retrieved_objs = $session->search(
    object => 'Infoblox::DTC::Monitor::SNMP',
    name   => 'snmp_monitor1',
);

my @retrieved_objs = $session->search(
    object => 'Infoblox::DTC::Monitor::SNMP',
    extensible_attributes => { 'Site' => 'Santa Clara' }
);
```

METHODS

This section describes all the methods that you can use to set or retrieve the attribute values of the object.

comment( )

Use this method to set or retrieve the comment for a DTC SNMP Health Monitor object.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

The valid value is a comment in string format (UTF-8) with a maximum of 256 bytes.

Returns

If you specified a parameter, the method returns 'true' when the modification succeeds, and returns 'false' when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#get comment value
my $comment = $snmp_monitor->comment();

#modify comment value
$snmp_monitor->comment('desired comment');
```

community( )

Use this method to set or retrieve the SNMP community string for a SNMP authentication.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

The valid value is a comment in string format (UTF-8) with a maximum of 256 bytes.

Returns

If you specified a parameter, the method returns 'true' when the modification succeeds, and returns 'false' when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#get community value
my $community = $snmp_monitor->community();

#modify community value
$snmp_monitor->community('desired community');
```

context( )

Use this method to set or retrive the SNMPv3 context.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

The valid value is a context in string format (UTF-8) with a maximum of 1023 characters.

Returns

If you specified a parameter, the method returns 'true' when the modification succeeds, and returns 'false' when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#get context value
my $context = $snmp_monitor->context();

#modify context value
$snmp_monitor->context('desired_context');
```

engine_id( )

Use this method to set or retrive the SNMPv3 engine identifier.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

The valid value is an engine identifier in string format (UTF-8) with a maximum of 1023 characters.

Returns

If you specified a parameter, the method returns 'true' when the modification succeeds, and returns 'false' when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#get engine identifiervalue
my $engine identifier= $snmp_monitor->engine_id();

#modify engine identifiervalue
$snmp_monitor->engine_id('desired_engine_id');
```

extattrs( )

Use this method to set or retrieve the extensible attributes associated with a DTC SNMP Health Monitor object.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

Valid value is a hash reference containing the names of extensible attributes and their associated values (Infoblox::Grid::Extattr objects).

Returns

If you specified a parameter, the method returns 'true' when the modification succeeds, and returns 'false' when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#get extattrs value
my $ref_extattrs = $snmp_monitor->extattrs();
```

```
#Modify extattrs
$snmp_monitor->extattrs({ 'Site' => $extattr1, 'Administrator' => $extattr2 });
```

extensible_attributes( )

Use this method to set or retrieve the extensible attributes associated with a DTC SNMP Health Monitor object.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

For valid values for extensible attributes, see Infoblox::Grid::ExtensibleAttributeDef/Extensible Attribute Values.

Returns

If you specified a parameter, the method returns 'true' when the modification succeeds, and returns 'false' when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get extensible attributes
my $ref_extensible_attributes = $snmp_monitor->extensible_attributes();

#Modify extensible attributes
$snmp_monitor->extensible_attributes({'Site' => 'Santa Clara', 'Administrator' => ['Peter',
'Tom']});
```

interval( )

Use this method to set or retrieve the time interval for the SNMP health check.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

The valid value is an unsigned integer.

Returns

If you specified a parameter, the method returns 'true' when the modification succeeds, and returns 'false' when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#get interval value
my $interval = $snmp_monitor->interval();

#modify interval value
$snmp_monitor->interval(10);
```

name( )

Use this method to set or retrieve a DTC SNMP Health Monitor name.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

The valid value is a desired name in string format.

Returns

If you specified a parameter, the method returns 'true' when the modification succeeds, and returns 'false' when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#get name value
my $name = $snmp_monitor->name();

#modify name value
$snmp_monitor->name('snmp_monitor1');
```

oids( )

Use this method to set or retrieve the list of OIDs for SNMP monitoring.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

The valid value is an array of an Infoblox::DTC::Monitor::SNMP::OID objects.

Returns

If you specified a parameter, the method returns 'true' when the modification succeeds, and returns 'false' when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#get oids value
my $oids = $snmp_monitor->oids();

#modify oids value
$snmp_monitor->oids([$oid1, $oid2]);
```

port( )

Use this method to set or retrieve the port value for SNMP or SNMPS requests.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

The valid value is an unsigned integer between 1 and 65535.

Returns

If you specified a parameter, the method returns 'true' when the modification succeeds, and returns 'false' when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#get port value
my $port = $snmp_monitor->port();

#modify port value
$snmp_monitor->port(8080);
```

retry_down( )

Use this method to set or retrieve the number of times the server appears offline after it was online so it is treated as unavailable.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

The valid value is an unsigned integer between 1 and 10.

Returns

If you specified a parameter, the method returns 'true' when the modification succeeds, and returns 'false' when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#get retry_down value
my $retry_down = $snmp_monitor->retry_down();

#modify retry_down value
$snmp_monitor->retry_down(3);
```

retry_up( )

Use this method to set or retrieve the number of times the server appears online after it was offline so it is treated as alive.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

The valid value is an unsigned integer between 1 and 10.

Returns

If you specified a parameter, the method returns 'true' when the modification succeeds, and returns 'false' when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#get retry_up value
my $retry_up = $snmp_monitor->retry_up();

#modify retry_up value
$snmp_monitor->retry_up(3);
```

timeout( )

Use this method to set or retrieve the value of a timeout for SNMP health check.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

The valid value is an unsigned integer between 1 and 15.

Returns

If you specified a parameter, the method returns 'true' when the modification succeeds, and returns 'false' when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#get timeout value
my $timeout = $snmp_monitor->timeout();

#modify timeout value
$snmp_monitor->timeout(7);
```

user( )

Use this method to set or retrieve the SNMPv3 user setting.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

The valid value is an Infoblox::Grid::SNMP::User object name.

Returns

If you specified a parameter, the method returns 'true' when the modification succeeds, and returns 'false' when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#get user value
my $user = $snmp_monitor->user();

#modify user value
$snmp_monitor->user('user1');
```

version( )

Use this method to set or retrieve the SNMP protocol version for the SNMP health check.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

The valid values are 'V1', 'V2C' and 'V3'.

Returns

If you specified a parameter, the method returns 'true' when the modification succeeds, and returns 'false' when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#get version value
my $version = $snmp_monitor->version();

#modify version value
$snmp_monitor->version('V1');
```

AUTHOR

Infoblox Inc. http://www.infoblox.com/

SEE ALSO

Infoblox::Session, Infoblox::Session->add(), Infoblox::Session->get(), Infoblox::Session->modify(), Infoblox::Session->remove(), Infoblox::Session->search(), Infoblox::Grid::Extattr, Infoblox::Grid::ExtensibleAttributeDef/Extensible Attribute Values, Infoblox::DTC::Monitor::SNMP::OID, Infoblox::Grid::SNMP::User

---

COPYRIGHT

Copyright (c) 2017 Infoblox Inc.

Infoblox::DTC::Monitor::SNMP::OID - a DTC SNMP Monitor OID object.

- NAME
- DESCRIPTION
- CONSTRUCTOR
- SESSION METHODS
- MODULE METHODS
  - Infoblox::DTC::Monitor::SNMP->oids( )
- METHODS
  - comment( )
  - condition( )
  - first( )
  - last( )
  - oid( )
  - type( )
- AUTHOR
- SEE ALSO
- COPYRIGHT

NAME

Infoblox::DTC::Monitor::SNMP::OID - a DTC SNMP Monitor OID object.

DESCRIPTION

SNMP is stateless request-response protocol for monitoring and managing devices on network. SNMP agent (software running on monitored server) exposes management data in form of variables. Each variable associated with a uniq id called OID.

CONSTRUCTOR

```
my $snmp_oid = Infoblox::DTC::Monitor::SNMP::OID->new(
    oid       => $string,                                    # Required
    comment   => $string,                                    # Optional / Default is undefined
    type      => 'STRING' | 'INTEGER',                       # Optional / Default is 'STRING'
    condition => 'ANY' | 'EXACT' | 'LEQ' | 'GEQ' | 'RANGE', # Optional / Default is 'ANY'
    first     => $string,                                    # Optional / Default is undefined
    last      => $string,                                    # Optional / Default is undefined
);
```

SESSION METHODS

The object does not support any session methods.

MODULE METHODS

The following functions can be applied to a DTC SNMP Monitor OID object.

Infoblox::DTC::Monitor::SNMP->oids( )

Use this method to set or retrieve the list of OIDs for SNMP monitoring. See Infoblox::DTC::Monitor::SNMP->oids() for parameters and return values.

Example

```
#Get snmp_oid value
my $snmp_oid = $snmp_monitor->snmp_oid();
#Modify bind_check_snmp_oid
$snmp_monitor->snmp_oid([$snmp_oids]);
```

METHODS

comment( )

Use this method to set or retrieve the comment for a DTC SNMP Health Monitor OID object.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

The valid value is a comment in string format (UTF-8) with a maximum of 256 bytes.

Returns

If you specified a parameter, the method returns 'true' when the modification succeeds, and returns 'false' when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#get comment value
my $comment = $snmp_oid->comment();

#modify comment value
$snmp_oid->comment('desired comment');
```

condition( )

Use this method to set or retrieve the condition for validating the result of a SNMP health check.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

The following conditions can be applied to result check:

'ANY' accepts any response.

'EXACT' accepts result equal to 'first'

'LEQ' accepts result which is less than 'first'

'GEQ' accepts result which is greater than 'first'

'RANGE' accepts result value of which is between 'first' and 'last'

The default value is 'ANY'.

Returns

If you specified a parameter, the method returns 'true' when the modification succeeds, and returns 'false' when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#get condition value
my $condition = $snmp_oid->condition();

#modify condition value
$snmp_oid->condition('RANGE');
```

first( )

Use this method to set or retrieve the condition's first term to match against the SNMP health check result.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

The valid value is a string or integer both in string format (UTF-8) with a maximum of 256 bytes.

Returns

If you specified a parameter, the method returns 'true' when the modification succeeds, and returns 'false' when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#get first value
my $first = $snmp_oid->first();

#modify first value
$snmp_oid->first('first_value');
```

last( )

Use this method to set or retrieve the condition's second term to match against the SNMP health check result with 'RANGE' condition.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

The valid value is a string or integer both in string format (UTF-8) with a maximum of 256 bytes.

Returns

If you specified a parameter, the method returns 'true' when the modification succeeds, and returns 'false' when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#get last value
my $last = $snmp_oid->last();

#modify last value
$snmp_oid->last('1000');
```

oid( )

Use this method to set or retrieve the SNMP OID value for DTC SNMP monitor health checks.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

The valid value is an OID in string format (UTF-8) with a maximum of 256 bytes.

Returns

If you specified a parameter, the method returns 'true' when the modification succeeds, and returns 'false' when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#get oid value
my $oid = $snmp_oid->oid();

#modify oid value
$snmp_oid->oid('sampleoid.1.1.1.1.2);
```

type( )

Use this method to set or retrieve the condition's value type for DTC SNMP monitor health check results.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

The valid values are 'STRING' and 'INTEGER'.

Returns

If you specified a parameter, the method returns 'true' when the modification succeeds, and returns 'false' when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#get type value
my $type = $snmp_oid->type();

#modify type value
$snmp_oid->type('INTEGER');
```

AUTHOR

Infoblox::DTC::Monitor::SNMP::OID - a DTC SNMP Monitor OID object.

Infoblox Inc. http://www.infoblox.com/

SEE ALSO

Infoblox::DTC::Monitor::SNMP, Infoblox::DTC::Monitor::SNMP->oids()

COPYRIGHT

Copyright (c) 2017 Infoblox Inc.

Infoblox::DTC::Monitor::TCP - A DTC TCP Health Monitor object.

- NAME
- DESCRIPTION
- CONSTRUCTOR
- SESSION METHODS
  - Infoblox::Session->add( )
  - Infoblox::Session->get( )
  - Infoblox::Session->modify( )
  - Infoblox::Session->remove( )
  - Infoblox::Session->search( )
- METHODS
  - comment( )
  - extattrs( )
  - extensible_attributes( )
  - interval( )
  - name( )
  - port( )
  - retry_down( )
  - retry_up( )
  - timeout( )
- AUTHOR
- SEE ALSO
- COPYRIGHT

NAME

Infoblox::DTC::Monitor::TCP - A DTC TCP Health Monitor object.

DESCRIPTION

A DTC TCP Health Monitor object.

CONSTRUCTOR

```
 my $tcp_monitor = Infoblox::DTC::Monitor::TCP->new(
    name                  => $string,
#Required
    comment               => $string,
#Optional / Default is undefined
    interval              => $uint,
#Optional / Default is undefined
    port                  => $uint,
#Required
    retry_down            => $uint,
#Optional / Default is undefined
    retry_up              => $uint,
#Optional / Default is undefined
    timeout               => $uint,
#Optional / Default is undefined
    extattrs              => { $string => $extattr, ... },
#Optional / Default is undefined
    extensible_attributes => { $string => $string | $num, $string => [ $string | $num, ... ], ... }
#Optional / Default is undefined
 );
```

You cannot set both extattrs and extensible_attributes attributes at the same time.

SESSION METHODS

This section describes all the methods in a Session module that you can apply to a DTC TCP Monitor object.

Infoblox::Session->add( )

Use this method to add an object to the Infoblox appliance. See Infoblox::Session->add() for parameters and return values.

Example

```
#construct an object
my $tcp_monitor = Infoblox::DTC::Monitor::TCP->new(
   name  => 'tcp_monitor1',
```

```
    port  => 15,
);

#submit for addition
my $response = $session->add($tcp_monitor);
```

Infoblox::Session->get( )

Use this method to retrieve all the matching objects from the Infoblox appliance. See Infoblox::Session->get() for parameters and return values.

Key Reference

```
Apply the following attribute to get a specific object:

 name                - Optional. A DTC TCP Monitor name in a string format.
 comment             - Optional. A DTC TCP Monitor comment.
 extattrs            - Optional. A hash reference containing extensible attributes.
 extensible_attributes - Optional. A hash reference that contains extensible attributes.
```

Example

```
my @retrieved_objs = $session->get(
    object => 'Infoblox::DTC::Monitor::TCP',
    name   => 'tcp_monitor1',
);

my @retrieved_objs = $session->get(
    object => 'Infoblox::DTC::Monitor::TCP',
    extensible_attributes => { 'Site' => 'Santa Clara' }
);
```

Infoblox::Session->modify( )

Use this method to modify an object in the Infoblox appliance. See Infoblox::Session->modify() for parameters and return values.

Example

```
#Use method to modify the comment.
$tcp_monitor->comment('this is a modified comment');
#Submit modification
my $response = $session->modify($tcp_monitor);
```

Infoblox::Session->remove( )

Use this method to remove an object from the Infoblox appliance. See Infoblox::Session->remove() for parameters and return values.

To remove a specific object, first use `get()` or `search()` to retrieve the specific object, and then submit this object for removal.

Example

```
#Get the objects with the same name
my @retrieved_objs = $session->get(
    object => 'Infoblox::DTC::Monitor::TCP',
    name   => 'tcp_monitor1',
);
#Find the desired object from the retrieved list.
my $desired_tcp_monitor = $retrieved_objs[0];
#Submit for removal
my $response = $session->remove($desired_tcp_monitor);
```

Infoblox::Session->search( )

Use this method to search for objects in the Infoblox appliance. See Infoblox::Session->search() for parameters and return values.

Key Reference

```
Apply the following attribute to get a specific object:

 name                - Optional. A DTC TCP Monitor name in a string format.
 comment             - Optional. A DTC TCP Monitor comment.
 extattrs            - Optional. A hash reference containing extensible attributes.
 extensible_attributes - Optional. A hash reference that contains extensible attributes.
```

Example

```
my @retrieved_objs = $session->search(
    object => 'Infoblox::DTC::Monitor::TCP',
```

```
    name   => 'tcp_monitor1',
);

my @retrieved_objs = $session->search(
    object => 'Infoblox::DTC::Monitor::TCP',
    extensible_attributes => { 'Site' => 'Santa Clara' }
);
```

METHODS

This section describes all the methods that you can use to set or retrieve the attribute values of the object.

comment( )

Use this method to set or retrieve a DTC TCP Health Monitor object comment.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

The valid value is a comment in string format (UTF-8) with a maximum of 256 bytes.

Returns

If you specified a parameter, the method returns 'true' when the modification succeeds, and returns 'false' when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#get comment value
my $comment = $tcp_monitor->comment();

#modify comment value
$tcp_monitor->comment('desired comment');
```

extattrs( )

Use this method to set or retrieve the extensible attributes associated with a DTC TCP Health Monitor object.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

Valid value is a hash reference containing the names of extensible attributes and their associated values (Infoblox::Grid::Extattr objects).

Returns

If you specified a parameter, the method returns 'true' when the modification succeeds, and returns 'false' when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#get extattrs value
my $ref_extattrs = $tcp_monitor->extattrs();

#Modify extattrs
$tcp_monitor->extattrs({ 'Site' => $extattr1, 'Administrator' => $extattr2 });
```

extensible_attributes( )

Use this method to set or retrieve the extensible attributes associated with a DTC TCP Health Monitor object.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

For valid values for extensible attributes, see Infoblox::Grid::ExtensibleAttributeDef/Extensible Attribute Values.

Returns

If you specified a parameter, the method returns 'true' when the modification succeeds, and returns 'false' when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get extensible attributes
my $ref_extensible_attributes = $tcp_monitor->extensible_attributes();

#Modify extensible attributes
$tcp_monitor->extensible_attributes({'Site' => 'Santa Clara', 'Administrator' => ['Peter', 'Tom']});
```

interval( )

Use this method to set or retrieve the time interval for the TCP health check.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

The valid value is an unsigned integer.

Returns

If you specified a parameter, the method returns 'true' when the modification succeeds, and returns 'false' when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#get interval value
my $interval = $tcp_monitor->interval();

#modify interval value
$tcp_monitor->interval(10);
```

name( )

Use this method to set or retrieve a DTC TCP Health Monitor name.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

The valid value is a desired name in a string format.

Returns

If you specified a parameter, the method returns 'true' when the modification succeeds, and returns 'false' when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#get name value
my $name = $tcp_monitor->name();

#modify name value
$tcp_monitor->name('tcp_monitor1');
```

port( )

Use this method to set or retrieve the port value for TCP requests.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

The valid value is an unsigned integer between 1 and 65535.

Returns

If you specified a parameter, the method returns 'true' when the modification succeeds, and returns 'false' when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#get port value
my $port = $tcp_monitor->port();

#modify port value
$tcp_monitor->port(6030);
```

retry_down( )

Use this method to set or retrieve the number of times the server appears offline after it was online so it is treated as dead.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

The valid value is an unsigned integer between 1 and 10.

Returns

If you specified a parameter, the method returns 'true' when the modification succeeds, and returns 'false' when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#get retry_down value
my $retry_down = $tcp_monitor->retry_down();

#modify retry_down value
$tcp_monitor->retry_down(3);
```

retry_up( )

Use this method to set or retrieve the number of times the server appears online after it was offline so it is treated as alive.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

The valid value is an unsigned integer between 1 and 10.

Returns

If you specified a parameter, the method returns 'true' when the modification succeeds, and returns 'false' when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#get retry_up value
my $retry_up = $tcp_monitor->retry_up();

#modify retry_up value
$tcp_monitor->retry_up(3);
```

timeout( )

Use this method to set or retrieve the value of a timeout for TCP health check.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

The valid value is an unsigned integer between 1 and 15.

Returns

If you specified a parameter, the method returns 'true' when the modification succeeds, and returns 'false' when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#get timeout value
my $timeout = $tcp_monitor->timeout();

#modify timeout value
```

```
$tcp_monitor->timeout(7);
```

AUTHOR

Infoblox Inc. http://www.infoblox.com/

SEE ALSO

Infoblox::Session, Infoblox::Session->add(), Infoblox::Session->get(), Infoblox::Session->modify(), Infoblox::Session->remove(), Infoblox::Session->search(), Infoblox::Grid::Extattr, Infoblox::Grid::ExtensibleAttributeDef/Extensible Attribute Values.

COPYRIGHT

Copyright (c) 2017 Infoblox Inc.

Infoblox::DTC::Object - An object for all managed objects on DNS traffic control devices.

- NAME
- DESCRIPTION
- CONSTRUCTOR
- SESSION METHODS
  - Infoblox::Session->get( )
  - Infoblox::Session->search( )
- METHODS
  - abstract_type( )
  - comment( )
  - display_type( )
  - extattrs( )
  - extensible_attributes( )
  - ipv4_address_list( )
  - ipv6_address_list( )
  - name( )
  - object( )
  - status( )
  - status_time( )
- AUTHOR
- SEE ALSO
- COPYRIGHT

NAME

Infoblox::DTC::Object - An object for all managed objects on DNS traffic control devices.

DESCRIPTION

This is a read-only object for all managed objects on DNS traffic control devices.

CONSTRUCTOR

SESSION METHODS

This section describes all the methods in an Infoblox::Session module that you can apply to an object.

Infoblox::Session->get( )

Use this method to retrieve all the matching objects from the Infoblox appliance. See Infoblox::Session->get() for parameters and return values.

Key References

```
Apply the following attributes to get a specific object:

 comment       - Optional. The comment.
 name          - Optional. The object name.
 type          - Optional. The object type. The valid values are 'LBDN', 'POOL' and 'SERVER'.
 status_member - Optional. Optional. The member that provides the health status for the object. The
default is Grid.
```

Example

```
my @retrieved_objs = $session->get(
   object => 'Infoblox::DTC::Object',
   name   => 'dtc_device',
);
```

Infoblox::Session->search( )

Use this method to search for objects in the Infoblox appliance. See Infoblox::Session->search() for parameters and return values.

Key References

```
Apply the following attributes to search a specific object:

 comment       - Optional. The comment.
 name          - Optional. The object name.
 type          - Optional. The object type. The valid values are 'LBDN', 'POOL' and 'SERVER'.
 status_member - Optional. The member that provides the health status for the object. The default is
```

Grid.

Example

```
my @retrieved_objs = $session->search(
   object => 'Infoblox::DTC::Object',
   name   => 'dtc_device',
);
```

METHODS

This section describes all the methods that you can use to retrieve the attribute values of an object.

abstract_type( )

Use this method to retrieve an abstract object type. This method does not support any argument because the attribute is read-only and is set by the server.

Parameter

None

Returns

The method returns the attribute value.

Example

```
#Get attribute value
my $value = $object->abstract_type();
```

comment( )

Use this method to retrieve a descriptive comment. This method does not support any argument because the attribute is read-only and is set by the server.

Parameter

None

Returns

The method returns the attribute value.

Example

```
#Get attribute value
my $value = $object->comment();
```

display_type( )

Use this method to retrieve an object type. This method does not support any argument because the attribute is read-only and is set by the server.

Parameter

None

Returns

The method returns the attribute value.

Example

```
#Get attribute value
my $value = $object->display_type();
```

extattrs( )

Use this method to retrieve the extensible attributes associated with an object object. This attribute is read-only and cannot be set.

Parameter

None.

Returns

The method returns the attribute value.

Example

```
#Get extattrs
my $ref_extattrs = $object->extattrs();
```

extensible_attributes( )

Use this method to retrieve the extensible attributes associated with an object. This method does not support any argument because the attribute is read-only and is set by the server.

Parameter

None

Returns

The method returns the attribute value.

Example

```
#Get attribute value
my $value = $object->extensible_attributes();
```

ipv4_address_list( )

Use this method to retrieve the IPv4 addresses for display. This method does not support any argument because the attribute is read-only and is set by the server.

Parameter

None

Returns

The method returns the attribute value.

Example

```
#Get attribute value
my $value = $object->ipv4_address_list();
```

ipv6_address_list( )

Use this method to retrieve the IPv6 addresses for display. This method does not support any argument because the attribute is read-only and is set by the server.

Parameter

None

Returns

The method returns the attribute value.

Example

```
#Get attribute value
my $value = $object->ipv6_address_list();
```

name( )

Use this method to retrieve an object name. This method does not support any argument because the attribute is read-only and is set by the server.

Parameter

None

Returns

The method returns the attribute value.

Example

```
#Get attribute value
my $value = $object->name();
```

object( )

Use this method to retrieve an object. This method does not support any argument because the attribute is read-only and is set by the server.

Parameter

None

Returns

The valid return value is an Infoblox::DTC::Pool, Infoblox::DTC::Server or Infoblox::DTC::LBDN object.

Example

```
#Get attribute value
my $value = $object->object();
```

status( )

Use this method to retrieve the current status of an object. This method does not support any argument because the attribute is read-only and is set by the server.

Parameter

None

Returns

The method returns the attribute value that can be 'BLUE', 'GRAY', 'GREEN', 'NONE', 'RED' or 'YELLOW'.

```
+---------------+--------------------------------+
|  GREEN        |  Working, OK                   |
+---------------+--------------------------------+
|  YELLOW       |  Warning                       |
+---------------+--------------------------------+
|  RED          |  Failed, Error                 |
+---------------+--------------------------------+
|  BLUE         |  Transitional, Connecting      |
+---------------+--------------------------------+
|  GRAY         |  Disabled                      |
+---------------+--------------------------------+
|  NONE (BLACK) |  Inactive, Offline, Unknown    |
+---------------+--------------------------------+
```

Example

```
#Get attribute value
my $value = $object->status();
```

status_time( )

Use this method to retrieve the timestamp of the object's current status. This is a read-only attribute.

Parameter

None

Returns

The method returns the attribute value.

Example

```
#Get attribute value
my $value = $object->status_time();
```

AUTHOR

Infoblox Inc. http://www.infoblox.com/

SEE ALSO

Infoblox::Session, Infoblox::DTC::Pool, Infoblox::DTC::Server, Infoblox::DTC::LBDN

COPYRIGHT

Infoblox::DTC::Object - An object for all managed objects on DNS traffic control devices.

Copyright (c) 2017 Infoblox Inc.

Infoblox::DTC::Pool - a DTC Pool object.

- NAME
- DESCRIPTION
- CONSTRUCTOR
- SESSION METHODS
  - Infoblox::Session->add( )
  - Infoblox::Session->get( )
  - Infoblox::Session->modify( )
  - Infoblox::Session->remove( )
  - Infoblox::Session->search( )
- METHODS
  - availability( )
  - comment( )
  - dynamic_ratio_alternate( )
  - dynamic_ratio_preferred( )
  - disable( )
  - extattrs( )
  - extensible_attributes( )
  - health( )
  - lb_alternate_method( )
  - lb_alternate_topology( )
  - lb_preferred_method( )
  - lb_preferred_topology( )
  - monitors( )
  - name( )
  - override_ttl( )
  - quorum( )
  - servers( )
  - ttl( )
- AUTHOR
- SEE ALSO
- COPYRIGHT

NAME

Infoblox::DTC::Pool - a DTC Pool object.

DESCRIPTION

A DTC Pool object.

CONSTRUCTOR

```
 my $pool = Infoblox::DTC::Pool->new(
    name                    => $string,,
#Required
    lb_preferred_method     => 'ALL_AVAILABLE' | 'GLOBAL_AVAILABILITY' | 'RATIO' |
                               'ROUND_ROBIN' | 'TOPOLOGY' | 'DYNAMIC_RATIO',
#Required
    disable                 => 'true' | 'false',
#Optional / Default is 'false'
    comment                 => $string,
#Optional / Default is undefined
    lb_preferred_topology   => $topology,
#Optional / Default is undefined
    lb_alternate_method     => 'ALL_AVAILABLE' | 'GLOBAL_AVAILABILITY' | 'RATIO' |
                               'ROUND_ROBIN' | 'TOPOLOGY' | 'DYNAMIC_RATIO',
#Optional / Default is undefined
    lb_alternate_topology   => $topology,
#Optional / Default is undefined
    availability            => 'ANY' | 'QUORUM' | 'ALL',
#Optional / Default is undefined
    quorum                  => $uint,
#Optional / Default is undefined
    ttl                     => $uint,
#Optional / Default is undefined
    override_ttl            => 'true' | 'false',
#Optional / Default is 'false'
    monitors                => [$monitor1, $monitor2, ...],
#Optional / Default is empty list
    servers                 => [$server1, $server2, ...],
#Optional / Default is empty list
```

```
    dynamic_ratio_preferred => $ratio_setting,
#Optional / Default is undefined
    dynamic_ratio_alternate => $ratio_setting,
#Optional / Default is undefined
    extattrs                => { $string => $extattr, ... },
#Optional / Default is undefined
    extensible_attributes   => { $string => $string | $num, $string => [ $string | $num, ... ], ... }
#Optional / Default is undefined
 );
```

You cannot set both extattrs and extensible_attributes attributes at the same time.

SESSION METHODS

This section describes all the methods in a Session module that you can apply to a DTC Pool object

Infoblox::Session->add( )

Use this method to add an object to the Infoblox appliance. See Infoblox::Session->add() for parameters and return values.

Example

```
#construct an object
my $pool = Infoblox::DTC::Pool->new(
   name  => 'pool1',
   lb_preferred_method => 'RATIO',
);

#submit for addition
my $response = $session->add($pool);
```

Infoblox::Session->get( )

Use this method to retrieve all the matching objects from the Infoblox appliance. See Infoblox::Session->get() for parameters and return values.

Key Reference

```
Apply the following attribute to get a specific object:

 name                - Optional. A DTC Pool name in a string format.
 comment             - Optional. A DTC Pool comment.
 extattrs            - Optional. A hash reference containing extensible attributes.
 extensible_attributes - Optional. A hash reference that contains extensible attributes.
 status_member       - Optional. A member which provides object health status, default is Grid.
```

Example

```
my @retrieved_objs = $session->get(
    object => 'Infoblox::DTC::Pool',
    name   => 'pool1',
);

my @retrieved_objs = $session->get(
    object => 'Infoblox::DTC::Pool',
    extensible_attributes => { 'Site' => 'Santa Clara' }
);
```

Infoblox::Session->modify( )

Use this method to modify an object in the Infoblox appliance. See Infoblox::Session->modify() for parameters and return values.

Example

```
#Use method to modify the comment.
$pool->comment('this is a modified comment');
#Submit modification
my $response = $session->modify($pool);
```

Infoblox::Session->remove( )

Use this method to remove an object from the Infoblox appliance. See Infoblox::Session->remove() for parameters and return values.

To remove a specific object, first use `get()` or `search()` to retrieve the specific object, and then submit this object for removal.

Example

```
#Get the objects with the same name
my @retrieved_objs = $session->get(
    object => 'Infoblox::DTC::Pool',
    name   => 'pool1',
);
#Find the desired object from the retrieved list.
my $desired_pool = $retrieved_objs[0];
#Submit for removal
my $response = $session->remove($desired_pool);
```

Infoblox::Session->search( )

Use this method to search for objects in the Infoblox appliance. See Infoblox::Session->search() for parameters and return values.

Key Reference

```
Apply the following attribute to get a specific object:

 name                 - Optional. A DTC Pool name in a string format.
 comment              - Optional. A DTC Pool comment.
 extattrs             - Optional. A hash reference containing extensible attributes.
 extensible_attributes - Optional. A hash reference that contains extensible attributes.
 status_member        - Optional. A member which provides object health status, default is Grid.
```

Example

```
my @retrieved_objs = $session->get(
    object => 'Infoblox::DTC::Pool',
    name   => 'pool1',
);

my @retrieved_objs = $session->get(
    object => 'Infoblox::DTC::Pool',
    extensible_attributes => { 'Site' => 'Santa Clara' }
);
```

METHODS

This section describes all the methods that you can use to set or retrieve the attribute values of the object.

availability( )

Use this method to set or retrieve an DTC Pool resources availability status.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

The valid values are 'ALL', 'ANY' and 'QUORUM'.

Returns

If you specified a parameter, the method returns 'true' when the modification succeeds, and returns 'false' when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#get availability value
my $availability = $pool->availability();

#modify availability value
$pool->availability('ANY');
```

comment( )

Use this method to set or retrieve a descriptive comment.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

The valid value is a comment in string format (UTF-8) with a maximum of 256 bytes.

Returns

If you specified a parameter, the method returns 'true' when the modification succeeds, and returns 'false' when the operation

fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#get comment value
my $comment = $pool->comment();

#modify comment value
$pool->comment('desired comment');
```

dynamic_ratio_alternate( )

Use this method to set or retrieve the alternate dynamic ratio load balancing settings.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

The valid value is an Infoblox::DTC::Pool::DynamicRatioSetting object.

Returns

If you specified a parameter, the method returns 'true' when the modification succeeds, and returns 'false' when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#get dynamic_ratio_alternate value
my $dynamic_ratio_alternate = $pool->dynamic_ratio_alternate();

#Construct dynamic_ratio_alternate value
my $dynamic_ratio = Infoblox::DTC::Pool::DynamicRatioSetting->new(
    method                => 'ROUND_TRIP_DELAY',
    invert_monitor_metric => 'true',
);

#modify dynamic_ratio_alternate value
$pool->dynamic_ratio_alternate($dynamic_ratio);
```

dynamic_ratio_preferred( )

Use this method to set or retrieve the preferred dynamic ratio load balancing settings.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

The valid value is an Infoblox::DTC::Pool::DynamicRatioSetting object.

Returns

If you specified a parameter, the method returns 'true' when the modification succeeds, and returns 'false' when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#get dynamic_ratio_preferred value
my $dynamic_ratio_preferred = $pool->dynamic_ratio_preferred();

#Construct dynamic_ratio_preferred value
my $dynamic_ratio = Infoblox::DTC::Pool::DynamicRatioSetting->new(
    method                => 'ROUND_TRIP_DELAY',
    invert_monitor_metric => 'true',
);

#modify dynamic_ratio_preferred value
$pool->dynamic_ratio_preferred($dynamic_ratio);
```

disable( )

Use this method to enable or disable a DTC Pool.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

Specify 'true' to disable a DTC Pool or 'false' to enable it. The default value is 'false'.

Returns

If you specified a parameter, the method returns 'true' when the modification succeeds, and returns 'false' when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#get disable value
my $disable = $pool->disable();

#modify disable value
$pool->disable('true');
```

extattrs( )

Use this method to set or retrieve the extensible attributes associated with a DTC Pool object.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

Valid value is a hash reference containing the names of extensible attributes and their associated values (Infoblox::Grid::Extattr objects).

Returns

If you specified a parameter, the method returns 'true' when the modification succeeds, and returns 'false' when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#get extattrs value
my $ref_extattrs = $pool->extattrs();

#Modify extattrs
$pool->extattrs({ 'Site' => $extattr1, 'Administrator' => $extattr2 });
```

extensible_attributes( )

Use this method to set or retrieve the extensible attributes associated with a DTC Pool object.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

For valid values for extensible attributes, see Infoblox::Grid::ExtensibleAttributeDef/Extensible Attribute Values.

Returns

If you specified a parameter, the method returns 'true' when the modification succeeds, and returns 'false' when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get extensible attributes
my $ref_extensible_attributes = $pool->extensible_attributes();

#Modify extensible attributes
$pool->extensible_attributes({'Site' => 'Santa Clara', 'Administrator' => ['Peter', 'Tom']});
```

health( )

Use this method to retrieve a DTC Pool health information. This is a read-only attribute.

Parameter

None

Returns

The valid return value is an Infoblox::DTC::Health object.

Example

```
#get health value
my $health = $pool->health();
```

lb_alternate_method( )

Use this method to set or retrieve the alternate load balancing method for the DTC Pool object.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

The valid values are 'ALL_AVAILABLE', 'DYNAMIC_RATIO', 'NONE', 'GLOBAL_AVAILABILITY', 'RATIO', 'ROUND_ROBIN' and 'TOPOLOGY'.

Returns

If you specified a parameter, the method returns 'true' when the modification succeeds, and returns 'false' when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#get lb_alternate_method value
my $lb_alternate_method = $pool->lb_alternate_method();

#modify lb_alternate_method value
$pool->lb_alternate_method('RATIO');
```

lb_alternate_topology( )

Use this method to set or retrieve topology rules for the alternate 'TOPOLOGY' load balancing method for the DTC Pool object.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Not that this field will be retrieved from the appliance only if alternate load balancing method is 'TOPOLOGY'.

Parameter

The valid value is an Infoblox::DTC::Topology object.

Returns

If you specified a parameter, the method returns 'true' when the modification succeeds, and returns 'false' when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#get lb_alternate_topology value
my $lb_alternate_topology = $pool->lb_alternate_topology();

#modify lb_alternate_topology value
$pool->lb_alternate_topology($lb_alternate_topology);
```

lb_preferred_method( )

Use this method to set or retrieve the preferred load balancing method for the DTC Pool object.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

The valid values are 'ALL_AVAILABLE', 'DYNAMIC_RATIO', 'GLOBAL_AVAILABILITY', 'RATIO', 'ROUND_ROBIN' and 'TOPOLOGY'.

Returns

If you specified a parameter, the method returns 'true' when the modification succeeds, and returns 'false' when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#get lb_preferred_method value
my $lb_preferred_method = $pool->lb_preferred_method();

#modify lb_preferred_method value
$pool->lb_preferred_method('RATIO');
```

lb_preferred_topology( )

Use this method to set or retrieve topology rules for the preferred 'TOPOLOGY' load balancing method for the DTC Pool object.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

The valid value is an Infoblox::DTC::Topology object.

Returns

If you specified a parameter, the method returns 'true' when the modification succeeds, and returns 'false' when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Not that this field will be retrieved from the appliance only if preferred load balancing method is 'TOPOLOGY'.

Example

```
#get lb_preferred_topology value
my $lb_preferred_topology = $pool->lb_preferred_topology();

#modify lb_preferred_topology value
$pool->lb_preferred_topology($lb_preferred_topology);
```

monitors( )

Use this method to set or retrieve health monitors related to a DTC Pool object.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

The valid value is an array which contains following objects: Infoblox::DTC::Monitor::TCP, Infoblox::DTC::Monitor::ICMP, Infoblox::DTC::Monitor::SIP, Infoblox::DTC::Monitor::HTTP, Infoblox::DTC::Monitor::PDP.

Returns

If you specified a parameter, the method returns 'true' when the modification succeeds, and returns 'false' when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#get monitors value
my @monitors = $pool->monitors();

#modify monitors value
$pool->monitors([$monitor1, $monitor2]);
```

name( )

Use this method to set or retrieve a DTC Pool name.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

The valid value is a desired name in a string format.

Returns

If you specified a parameter, the method returns 'true' when the modification succeeds, and returns 'false' when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#get name value
my $name = $pool->name();

#modify name value
$pool->name('pool1');
```

override_ttl( )

The override_ttl attribute controls whether the TTL value in the object is used, instead of the Grid default.

The override_ttl attribute can be specified explicitly. It is also set implicitly when ttl is set to a defined value.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

Specify 'true' to set override_ttl flag or 'false' to disable it. Default value is 'false'.

Returns

If you specified a parameter, the method returns 'true' when the modification succeeds, and returns 'false' when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#get override_ttl value
my $override_ttl = $pool->override_ttl();

#modify override_ttl value
$pool->override_ttl('true');
```

quorum( )

Use this method to set or retrieve the number of monitors that must report the resource as 'up' for 'QUORUM' availability mode to be available.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

The valid value is an unsigned integer.

Returns

If you specified a parameter, the method returns 'true' when the modification succeeds, and returns 'false' when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#get quorum value
my $quorum = $pool->quorum();

#modify quorum value
$pool->quorum(20);
```

servers( )

Use this method to set or retrieve a list of DTC Servers linked to a DTC Pool.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

The valid value is an array of Infoblox::DTC::Server::Link objects.

Returns

If you specified a parameter, the method returns 'true' when the modification succeeds, and returns 'false' when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#get servers value
my @servers = $pool->servers();

#modify servers value
$pool->servers([$server1, $server2]);
```

ttl( )

Use this method to set a TTL value.

Setting this method to a defined value implicitly sets the override_ttl method to 'true'. Setting the parameter to undefined causes the appliance to use the Grid default and automatically resets the override_ttl attribute to 'false'.

Note that when ttl is set to a defined value and override_ttl is set to 'false', the last operation takes precedence. Thus the sequence $object->ttl(10); $object->override_ttl('false'); will set override_ttl to 'false', and the sequence $object->override_ttl('false'); $object->ttl(10); will result in override_ttl='true'.

Parameter

The valid value is an unsigned integer.

Returns

If you specified a parameter, the method returns 'true' when the modification succeeds, and returns 'false' when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#get ttl value
my $ttl = $pool->ttl();

#modify ttl value
$pool->ttl(10);
```

AUTHOR

Infoblox Inc. http://www.infoblox.com/

SEE ALSO

Infoblox::Session, Infoblox::Session->add(), Infoblox::Session->get(), Infoblox::Session->modify(), Infoblox::Session->remove(), Infoblox::Session->search(), Infoblox::Grid::Extattr, Infoblox::Grid::ExtensibleAttributeDef/Extensible Attribute Values, Infoblox::DTC::Health, Infoblox::DTC::Pool::DynamicRatioSetting, Infoblox::DTC::Pool::Link, Infoblox::DTC::Server::Link, Infoblox::DTC::Topology, Infoblox::DTC::Monitor::TCP, Infoblox::DTC::Monitor::ICMP, Infoblox::DTC::Mo nitor::SIP, Infoblox::DTC::Monitor::HTTP, Infoblox::DTC::Monitor::PDP.

COPYRIGHT

Copyright (c) 2017 Infoblox Inc.

Infoblox::DTC::Pool::DynamicRatioSetting - An DTC Pool dynamic ratio setting object.

- NAME
- DESCRIPTION
- CONSTRUCTOR
- SESSION METHODS
- MODULE METHODS
  - Infoblox::DTC::Pool->dynamic_ratio_preferred( )
  - Infoblox::DTC::Pool->dynamic_ratio_alternate( )
- METHODS
  - method( )
  - monitor( )
  - monitor_metric( )
  - monitor_weighing( )
  - invert_monitor_metric( )
- AUTHOR
- SEE ALSO
- COPYRIGHT

NAME

Infoblox::DTC::Pool::DynamicRatioSetting - An DTC Pool dynamic ratio setting object.

DESCRIPTION

The DTC Pool dynamic ratio setting object provides settings for dynamic ratio load balancing if you later select preferred or alternate as the load balancing method.

CONSTRUCTOR

```
my $object = Infoblox::DTC::Pool::DynamicRatioSetting->new(
    method                => 'MONITOR' | 'ROUND_TRIP_DELAY', # Optional
    monitor               => $dtc_monitor,                   # Optional
    monitor_metric        => $string,                        # Optional
    monitor_weighing      => 'RATIO' | 'PRIORITY',           # Optional
    invert_monitor_metric => 'true' | 'false',               # Optional
);
```

SESSION METHODS

The object does not support any session methods.

MODULE METHODS

The following functions can be applied to the object.

Infoblox::DTC::Pool->dynamic_ratio_preferred( )

Use this method to set or retrieve the preferred dynamic ratio load balancing settings. See Infoblox::DTC::Pool->dynamic_ratio_preferred() for parameters and return values.

Example

```
#get dynamic_ratio_preferred value
my $dynamic_ratio_preferred = $pool->dynamic_ratio_preferred();

#Construct dynamic_ratio_preferred value
my $dynamic_ratio = Infoblox::DTC::Pool::DynamicRatioSetting->new(
    method                => 'ROUND_TRIP_DELAY',
    invert_monitor_metric => 'true',
);

#modify dynamic_ratio_preferred value
$pool->dynamic_ratio_preferred($dynamic_ratio);
```

Infoblox::DTC::Pool->dynamic_ratio_alternate( )

Use this method to set or retrieve the alternate dynamic ratio load balancing settings. See Infoblox::DTC::Pool->dynamic_ratio_alternate() for parameters and return values.

Example

```
#get dynamic_ratio_alternate value
my $dynamic_ratio_alternate = $pool->dynamic_ratio_alternate();

#modify dynamic_ratio_alternate value
$pool->dynamic_ratio_alternate($dynamic_ratio);
```

METHODS

This section describes all the methods that you can use to set or retrieve the attribute values of the object.

method( )

Use this method to set or retrieve the dynamic ratio load balancing method.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

The valid values are 'MONITOR' and 'ROUND_TRIP_DELAY'.

Returns

If you specified a parameter, the method returns 'true' when the modification succeeds, and returns 'false' when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get method value
my $method = $object->method();

#Modify method value
$object->method('MONITOR');
```

monitor( )

Use this method to set or retrieve the DTC monitor that will be used for the dynamic ratio load balancing.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

The valid value is an Infoblox::DTC::Monitor::TCP, Infoblox::DTC::Monitor::ICMP, Infoblox::DTC::Monitor::SIP, Infoblox::DTC::Monitor::HTTP, or Infoblox::DTC::Monitor::PDP objects.

Note that you must retrieve the object from the server first, or set the name of the monitor.

Returns

If you specified a parameter, the method returns 'true' when the modification succeeds, and returns 'false' when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get monitor value
my $monitor = $object->monitor();

#Construct Infoblox::DTC::Monitor::SNMP object (should be an existing object)
my $snmp_monitor = Infoblox::DTC::Monitor::SNMP->new(name => 'snmp1');

#Modify monitor value
$object->monitor($snmp_monitor);
```

monitor_metric( )

Use this method to set or retrieve the DTC monitor metric that will be used for dynamic weighing.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

The valid value is a desired metric in string format.

Returns

If you specified a parameter, the method returns 'true' when the modification succeeds, and returns 'false' when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get monitor_metric value
my $monitor_metric = $object->monitor_metric();

#Modify monitor_metric value
$object->monitor_metric('222');
```

monitor_weighing( )

Use this method to set or retrieve the weighing settings for the dynamic ratio load balancing servers.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

The valid values are 'PRIORITY' and 'RATIO'. 'PRIORITY' means that all clients will be forwarded to the least loaded server. 'RATIO' means that distribution will be calculated based on dynamic weights.

Returns

If you specified a parameter, the method returns 'true' when the modification succeeds, and returns 'false' when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get monitor_weighing value
my $monitor_weighing = $object->monitor_weighing();

#Modify monitor_weighing value
$object->monitor_weighing('RATIO');
```

invert_monitor_metric( )

Use this method to set or retrieve the flag that indicates whether the inverted value of the monitor metric is used.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Specify 'true' to force the server to use the inverted monitor metric and 'false' to use the direct metric value. The default is 'false'.

Returns

If you specified a parameter, the method returns 'true' when the modification succeeds, and returns 'false' when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get invert_monitor_metric value
my $invert_monitor_metric = $object->invert_monitor_metric();

#Modify invert_monitor_metric value
$object->invert_monitor_metric('true');
```

AUTHOR

Infoblox Inc. http://www.infoblox.com/

SEE ALSO

Infoblox::DTC::Pool, Infoblox::DTC::Pool->dynamic_ratio_preferred(), Infoblox::DTC::Pool->dynamic_ratio_alternate(), Infoblox::DTC::Monitor::TCP, Infoblox::DTC::Monitor::ICMP, Infoblox::DTC::Monitor::SIP, Infoblox::DTC::Monitor::HTTP, Infoblox::DTC::Monitor::PDP

COPYRIGHT

Infoblox::DTC::Pool::DynamicRatioSetting - An DTC Pool dynamic ratio setting object.

Copyright (c) 2017 Infoblox Inc.

Infoblox::DTC::Pool::Link - An DTC Pool Link object.

- NAME
- DESCRIPTION
- CONSTRUCTOR
- SESSION METHODS
- METHODS
  - dtc_pool( )
  - ratio( )
- AUTHOR
- SEE ALSO
- COPYRIGHT

NAME

Infoblox::DTC::Pool::Link - An DTC Pool Link object.

DESCRIPTION

An DTC Pool Link object.

CONSTRUCTOR

```
my $pool_link = Infoblox::DTC::Pool::Link->new(
    dtc_pool => $pool, #Optional / Default is undefined
    ratio    => $uint, #Optional / Default is undefined
);
```

SESSION METHODS

The object does not support any session methods.

METHODS

This section describes all the methods that you can use to set or retrieve the attribute values of the object.

dtc_pool( )

Use this method to set or retrieve the associated DTC Pool.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

The valid value is an Infoblox::DTC::Pool object.

Returns

If you specified a parameter, the method returns 'true' when the modification succeeds, and returns 'false' when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#get dtc_pool value
my $pool = $pool_link->dtc_pool();

#modify dtc_pool value
$pool_link->dtc_pool($pool);
```

ratio( )

Use this method to set or retrieve the weight of a DTC Pool.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

The valid value is an unsigned integer.

Returns

If you specified a parameter, the method returns 'true' when the modification succeeds, and returns 'false' when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#get ratio value
my $ratio = $pool_link->ratio();

#modify ratio value
$pool_link->ratio(70);
```

AUTHOR

Infoblox Inc. http://www.infoblox.com/

SEE ALSO

Infoblox::DTC::Pool.

COPYRIGHT

Copyright (c) 2017 Infoblox Inc.

Infoblox::DTC::Query::Parameters - A DTC query call parameters object.

- NAME
- DESCRIPTION
- CONSTRUCTOR
- SESSION METHODS
- MODULE METHODS
- Infoblox::DTC->query( )
- METHODS
  - address( )
  - lbdn( )
  - member( )
  - qname( )
  - type( )
- AUTHOR
- SEE ALSO
- COPYRIGHT

NAME

Infoblox::DTC::Query::Parameters - A DTC query call parameters object.

DESCRIPTION

A DTC query call parameters object.

CONSTRUCTOR

```
my $params = Infoblox::DTC::Query::Parameters->new(
    address => $string,                          # Required
    qname   => $string,                          # Required
    type    => 'A' | 'AAAA' | 'NAPTR' | 'CNAME', # Required
    lbdn    => $lbdn,                            # Required
    member  => $string,                          # Required
);
```

SESSION METHODS

The object does not support any session methods. It's used as a parameters for a DTC query call.

MODULE METHODS

Infoblox::DTC->query( )

Use this method to perform a DTC query on a given member.

Parameter

Valid value is an Infoblox::DTC::Query::Parameters object.

Returns

The method returns an Infoblox::DTC::Query::Response when operation succeeds, and returns 'false' when the operation fails.

Example

```
#perform DTC query
my $response = $dtc->query($params);
```

METHODS

This section describes all the methods that you can use to set or retrieve the attribute values of the object.

address( )

Use this method to set or retrieve an ostensible IPv4 or IPv6 source address.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

The valid value is a string that contains an IPv4 or IPv6 address.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#get address value
my $address = $params->address();

#modify address value
$params->address('192.168.1.0');
```

lbdn( )

Use this method to set or retrieve a specific LBDN object for querying.

Note that LBDN objects must be first retrieved using the `get()` or `search()` operation.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

The valid value is an Infoblox::DTC::LBDN object.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#get lbdn value
my $lbdn = $params->lbdn();

#modify lbdn value
$params->lbdn($lbdn);
```

member( )

Use this method to set or retrieve a member name for querying.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

The valid value is a string that contains the member name.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#get member value
my $member = $params->member();

#modify member value
$params->member('localhost.localdomain');
```

qname( )

Use this method to set or retrieve a fully-qualified DNS query name.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

The valid value is a string, which contains a query name in FQDN format.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#get qname value
my $qname = $params->qname();

#modify qname value
$params->qname($fqdn);
```

type( )

Use this method to set or retrieve the type of desired results.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

The valid values are 'A', 'AAAA', 'CNAME' and 'NAPTR'.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#get type value
my $type = $params->type();

#modify type value
$params->type('AAAA');
```

AUTHOR

Infoblox Inc. http://www.infoblox.com/

SEE ALSO

Infoblox::DTC->query()

COPYRIGHT

Copyright (c) 2017 Infoblox Inc.

Infoblox::DTC::Query::Response - A DTC query call response object.

- NAME
- DESCRIPTION
- SESSION METHODS
- MODULE METHODS
- Infoblox::DTC->query( )
- METHODS
  - records( )
- AUTHOR
- SEE ALSO
- COPYRIGHT

NAME

Infoblox::DTC::Query::Response - A DTC query call response object.

DESCRIPTION

A DTC query call response object.

SESSION METHODS

The object does not support any session methods. It is retrieved as a result of a DTC query call.

MODULE METHODS

Infoblox::DTC->query( )

Use this method to perform a DTC query on a member.

Parameter

The valid value is an Infoblox::DTC::Query::Parameters object.

Returns

The method returns an Infoblox::DTC::Query::Response when operation succeeds, and returns 'false' when the operation fails.

Example

```
#perform DTC query
my $response = $dtc->query($params);
```

METHODS

This section describes all the methods that you can use to set or retrieve the attribute values of the object.

records( )

Use this method to retrieve an array of query call records. This is a read-only attribute.

Parameter

None

Returns

The valid return value is an array of Infoblox::DTC::Query::Result objects.

Example

```
#get records value
my @records = $response->records();
```

AUTHOR

Infoblox Inc. http://www.infoblox.com/

SEE ALSO

Infoblox::DTC->query()

COPYRIGHT

Copyright (c) 2017 Infoblox Inc.

Infoblox::DTC::Query::Result - A DTC Query result object.

- NAME
- DESCRIPTION
- SESSION METHODS
- METHODS
  - override_ttl( )
  - ttl( )
  - rdata( )
  - type( )
- AUTHOR
- SEE ALSO
- COPYRIGHT

NAME

Infoblox::DTC::Query::Result - A DTC Query result object.

DESCRIPTION

A DTC Query function call result object.

SESSION METHODS

The object does not support any session methods. It is retrieved as a part of the DTC Query response object.

METHODS

override_ttl( )

Use this method to retrieve the attribute, which controls whether the TTL value of the object is used, instead of the Grid default. This is a read-only attribute.

Parameter

None

Returns

The method returns the attribute value, which can be 'true' or 'false'.

Example

```
#get override_ttl value
my $override_ttl = $result->override_ttl();
```

ttl( )

Use this method to retrieve a TTL value. This is a read-only attribute.

Parameter

None

Returns

The valid return value is an unsigned integer.

Example

```
#get ttl value
my $ttl = $result->ttl();
```

rdata( )

Use this method to retrieve DNS record data. This is a read-only attribute.

Parameter

None

Returns

The method returns the attribute value.

Example

```
#get rdata value
my $rdata = $result->rdata();
```

type( )

Use this method to retrieve the DNS record type. This is a read-only attribute.

Parameter

None

Returns

The valid return values are 'A', 'AAAA', 'CNAME' and 'NAPTR'.

Example

```
#get type value
my $type = $result->type();
```

AUTHOR

Infoblox Inc. http://www.infoblox.com/

SEE ALSO

Infoblox::DTC->query(), Infoblox::DTC::Response.

COPYRIGHT

Copyright (c) 2017 Infoblox Inc.

Infoblox::DTC::Record::A - A DTC A record object.

- NAME
- DESCRIPTION
- CONSTRUCTOR
- SESSION METHODS
  - Infoblox::Session->add( )
  - Infoblox::Session->get( )
  - Infoblox::Session->modify( )
  - Infoblox::Session->remove( )
  - Infoblox::Session->search( )
- METHODS
  - auto_created( )
  - comment( )
  - disable( )
  - dtc_server( )
  - ipv4addr( )
  - ttl( )
- AUTHOR
- SEE ALSO
- COPYRIGHT

NAME

Infoblox::DTC::Record::A - A DTC A record object.

DESCRIPTION

A DTC A object represents a DNS Traffic Control Address (DTC A) resource record. DTC A record maps a load balancing domain name to an IPv4 address.

CONSTRUCTOR

```
my $object = Infoblox::DTC::Record::A->new(
    'dtc_server'  => $dtc_server,      #Required
    'ipv4addr'    => $ipv4addr,        #Required
    'comment'     => $string,          #Optional
    'disable'     => 'true' | 'false', #Optional / Default is 'false'
    'ttl'         => $uint | undef,    #Optional / Default is undefined
);
```

SESSION METHODS

This section describes all the methods in an Infoblox::Session module that can be applied to a DTC A record object.

Infoblox::Session->add( )

Use this method to add an object to the Infoblox appliance. See Infoblox::Session->add() for parameters and return values.

Example

```
#Construct an object
my $object = Infoblox::DTC::Record::A->new(
    'comment'     => 'Sample DTC A record',
    'disable'     => 'false',
    'dtc_server'  => $dtc_server,
    'ipv4addr'    => '10.0.0.1',
    'ttl'         => 1024,
);

# Submit for addition
my $response = $session->add($object);
```

Infoblox::Session->get( )

Use this method to retrieve the existing objects from an Infoblox appliance. See Infoblox::Session->get() for parameters and return values.

Key Reference

```
Apply the following attributes to get specific DTC A record objects:

    dtc_server - Required. DTC Server object the DTC A record is associated with (exact).
    comment    - Optional. Descriptive comment in string format (regexp).
    ipv4addr   - Optional. DTC A record IPv4 address in string format (regexp).
```

Example

```
my @retrieved_objs = $session->get(
    object     => 'Infoblox::DTC::Record::A',
    dtc_server => $dtc_server,
);

my @retrieved_objs = $session->get(
   object     => 'Infoblox::DTC::Record::A',
   dtc_server => $dtc_server,
   ipv4addr   => '10.0.0.1',
);
```

Infoblox::Session->modify( )

Use this method to modify an object in the Infoblox appliance. See Infoblox::Session->modify() for parameters and return values.

Example

```
# Use this method to modify the comment.
$object->comment('this is a modified comment');

# Submit the modification
my $response = $session->modify($object);
```

Infoblox::Session->remove( )

Use this method to submit the removal of an object from the Infoblox appliance. See Infoblox::Session->remove() for parameters and return values.

To remove a specific object, first use `get()` or `search()` to retrieve the object, and then submit it for removal.

Example

```
# Get the objects associated with DTC Server $dtc_server
my @retrieved_objs = $session->get(
    object     => 'Infoblox::DTC::Record::A',
    dtc_server => $dtc_server,
);

# Find the desired object in the retrieved list.
my $desired_rec = $retrieved_objs[0];

# Submit for removal
my $response = $session->remove($desired_rec);
```

Infoblox::Session->search( )

Use this method to retrieve the existing objects from an Infoblox appliance. See Infoblox::Session->search() for parameters and return values.

Key Reference

```
Apply the following attributes to search the DTC A record objects:

    dtc_server - Required. DTC Server object the DTC A record is associated with.
    comment    - Optional. Descriptive comment in string format.
    ipv4addr   - Optional. DTC A record address.
```

Example

```
# Get the objects associated with DTC Server $dtc_server
my @retrieved_objs = $session->search(
    object     => 'Infoblox::DTC::Record::A',
    dtc_server => $dtc_server,
);

# Get the objects associated with DTC Server $dtc_server with comment that starts from 'A'
my @retrieved_objs = $session->get(
    object     => 'Infoblox::DTC::Record::A',
    dtc_server => $dtc_server,
    comment    => '^A.*',
```

```
);
```

METHODS

This section describes all the methods that can be used to retrieve the attribute values of a DTC A record object.

auto_created( )

Use this method to retrieve the flag that indicates whether this record was automatically created by NIOS or not. This is a read-only attribute.

Parameter

None

Returns

The method returns the attribute value.

Example

```
#Get auto_created value
my $auto_created = $object->auto_created();
```

comment( )

Use this method to set or retrieve the descriptive comment of a DTC A record object.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

Desired comment in string format with a maximum of 256 characters.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get comment value
my $comment = $object->comment();

#Modify comment value
$object->comment('This is a new comment');
```

disable( )

Use this method to set or retrieve the disable flag of a DTC A record.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

Specify 'true' to set the disable flag or 'false' to deactivate/unset it.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get disable value
my $disable = $object->disable();

#Modify disable value
$object->disable('true');
```

dtc_server( )

Use this method to set or retrieve the DTC Server the DTC A record is associated with.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Note that once the DTC record was created, the DTC Server value cannot be changed.

Parameter

The valid value is an Infoblox::DTC::Server object.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get the dtc_server field
my $dtc_server = $object->dtc_server();

#Modify dtc_server value
$object->dtc_server($server1);
```

ipv4addr( )

Use this method to set or retrieve the IPv4 address that is mapped to a domain name.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

The valid value is desired IPv4 address in string format.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get ipv4addr value
my $ipv4addr = $object->ipv4addr();

#Modify ipv4addr value
$object->ipv4addr('10.0.0.10');
```

ttl( )

Use this method to set or retrieve the Time to Live (TTL) value.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

The default value is undefined which indicates that the record inherits the TTL value of the DTC server pool.

Specify a TTL value to override the TTL value at the DTC server pool level.

Parameter

The valid value is an unsigned integer between 0 and 4294967295 that represents the duration (in seconds) that the record is cached. Zero indicates that the record should not be cached.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get ttl value
my $ttl = $object->ttl();

#Modify ttl value
$object->ttl(1800);

#inherit ttl value
$object->ttl(undef);
```

AUTHOR

Infoblox Inc. http://www.infoblox.com/

SEE ALSO

Infoblox::Session, Infoblox::Session->add(), Infoblox::Session->get(), Infoblox::Session->modify(), Infoblox::Session->remove(), Infoblox::Session->search(), Infoblox::DTC::Server

---

COPYRIGHT

Copyright (c) 2017 Infoblox Inc.

Infoblox::DTC::Record::AAAA - A DTC AAAA record object.

- NAME
- DESCRIPTION
- CONSTRUCTOR
- SESSION METHODS
  - Infoblox::Session->add( )
  - Infoblox::Session->get( )
  - Infoblox::Session->modify( )
  - Infoblox::Session->remove( )
  - Infoblox::Session->search( )
- METHODS
  - auto_created( )
  - comment( )
  - disable( )
  - dtc_server( )
  - ipv6addr( )
  - ttl( )
- AUTHOR
- SEE ALSO
- COPYRIGHT

NAME

Infoblox::DTC::Record::AAAA - A DTC AAAA record object.

DESCRIPTION

A DTC AAAA object represents a DNS Traffic Control IPv6 Address (DTC AAAA) resource record. DTC AAAA record maps a load balancing domain name to an IPv6 address.

CONSTRUCTOR

```
my $object = Infoblox::DTC::Record::AAAA->new(
    'dtc_server'  => $dtc_server,      #Required
    'ipv6addr'    => $ipv6addr,        #Required
    'comment'     => $string,          #Optional
    'disable'     => 'true' | 'false', #Optional / Default is 'false'
    'ttl'         => $uint | undef,    #Optional / Default is undefined
);
```

SESSION METHODS

This section describes all the methods in an Infoblox::Session module that can be applied to a DTC AAAA record object.

Infoblox::Session->add( )

Use this method to add an object to the Infoblox appliance. See Infoblox::Session->add() for parameters and return values.

Example

```
#Construct an object
my $object = Infoblox::DTC::Record::AAAA->new(
    'comment'     => 'Sample DTC AAAA record',
    'disable'     => 'false',
    'dtc_server'  => $dtc_server,
    'ipv6addr'    => '2001:db8::1',
    'ttl'         => 1024,
);

# Submit for addition
my $response = $session->add($object);
```

Infoblox::Session->get( )

Use this method to retrieve the existing objects from an Infoblox appliance. See Infoblox::Session->get() for parameters and return values.

Key Reference

```
Apply the following attributes to get specific DTC AAAA record objects:

    dtc_server - Required. DTC Server object the DTC AAAA record is associated with.
    comment    - Optional. Descriptive comment in string format.
    ipv6addr   - Optional. DTC AAAA record IPv6 address in string format.
```

Example

```
my @retrieved_objs = $session->get(
    object     => 'Infoblox::DTC::Record::AAAA',
    dtc_server => $dtc_server,
);

my @retrieved_objs = $session->get(
   object     => 'Infoblox::DTC::Record::AAAA',
   dtc_server => $dtc_server,
   ipv6addr   => '2001:db8::1',
);
```

Infoblox::Session->modify( )

Use this method to modify an object in the Infoblox appliance. See Infoblox::Session->modify() for parameters and return values.

Example

```
# Use this method to modify the comment.
$object->comment('this is a modified comment');

# Submit the modification
my $response = $session->modify($object);
```

Infoblox::Session->remove( )

Use this method to submit the removal of an object from the Infoblox appliance. See Infoblox::Session->remove() for parameters and return values.

To remove a specific object, first use `get()` or `search()` to retrieve the object, and then submit it for removal.

Example

```
# Get the objects associated with DTC Server $dtc_server
my @retrieved_objs = $session->get(
    object     => 'Infoblox::DTC::Record::AAAA',
    dtc_server => $dtc_server,
);

# Find the desired object in the retrieved list.
my $desired_rec = $retrieved_objs[0];

# Submit for removal
my $response = $session->remove($desired_rec);
```

Infoblox::Session->search( )

Use this method to retrieve the existing objects from an Infoblox appliance. See Infoblox::Session->search() for parameters and return values.

Key Reference

```
Apply the following attributes to search the DTC AAAA record objects:

    dtc_server - Required. DTC Server object the DTC AAAA record is associated with (exact).
    comment    - Optional. Descriptive comment in string format (regexp).
    ipv6addr   - Optional. DTC AAAA record address (regexp).
```

Example

```
# Get the objects associated with DTC Server $dtc_server
my @retrieved_objs = $session->search(
    object     => 'Infoblox::DTC::Record::AAAA',
    dtc_server => $dtc_server,
);

# Get the objects associated with DTC Server $dtc_server with comment that starts from 'A'
my @retrieved_objs = $session->get(
    object     => 'Infoblox::DTC::Record::AAAA',
    dtc_server => $dtc_server,
    comment    => '^A.*',
```

```
);
```

METHODS

This section describes all the methods that can be used to retrieve the attribute values of a DTC AAAA record object.

auto_created( )

Use this method to retrieve the flag that indicates whether this record was automatically created by NIOS or not. This is a read-only attribute.

Parameter

None

Returns

The method returns the attribute value.

Example

```
#Get auto_created value
my $auto_created = $object->auto_created();
```

comment( )

Use this method to set or retrieve the descriptive comment of a DTC AAAA record object.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

Desired comment in string format with a maximum of 256 characters.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get comment value
my $comment = $object->comment();

#Modify comment value
$object->comment('This is a new comment');
```

disable( )

Use this method to set or retrieve the disable flag of a DTC AAAA record.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

Specify 'true' to set the disable flag or 'false' to deactivate/unset it.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get disable value
my $disable = $object->disable();

#Modify disable value
$object->disable('true');
```

dtc_server( )

Use this method to set or retrieve the DTC Server the DTC AAAA record is associated with.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Note that once the DTC record was created, the DTC Server value cannot be changed.

Parameter

The valid value is an Infoblox::DTC::Server object.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get the dtc_server field
my $dtc_server = $object->dtc_server();

#Modify dtc_server value
$object->dtc_server($server1);
```

ipv6addr( )

Use this method to set or retrieve the IPv6 address that is mapped to a domain name.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

The valid value is desired IPv6 address in string format.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get ipv6addr value
my $ipv6addr = $object->ipv6addr();

#Modify ipv6addr value
$object->ipv6addr('2001:db8::1');
```

ttl( )

Use this method to set or retrieve the Time to Live (TTL) value.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

The default value is undefined which indicates that the record inherits the TTL value of the DTC server pool.

Specify a TTL value to override the TTL value at the DTC server pool level.

Parameter

The valid value is an unsigned integer between 0 and 4294967295 that represents the duration (in seconds) that the record is cached. Zero indicates that the record should not be cached.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get ttl value
my $ttl = $object->ttl();

#Modify ttl value
$object->ttl(1800);

#inherit ttl value
$object->ttl(undef);
```

AUTHOR

Infoblox::DTC::Record::AAAA - A DTC AAAA record object.

Infoblox Inc. http://www.infoblox.com/

SEE ALSO

Infoblox::Session, Infoblox::Session->add(), Infoblox::Session->get(), Infoblox::Session->modify(), Infoblox::Session->remove(), Infoblox::Session->search(), Infoblox::DTC::Server

---

COPYRIGHT

Copyright (c) 2017 Infoblox Inc.

Infoblox::DTC::Record::CNAME - A DTC CNAME record object.

- NAME
- DESCRIPTION
- CONSTRUCTOR
- SESSION METHODS
  - Infoblox::Session->add( )
  - Infoblox::Session->get( )
  - Infoblox::Session->modify( )
  - Infoblox::Session->remove( )
  - Infoblox::Session->search( )
- METHODS
  - auto_created( )
  - comment( )
  - disable( )
  - dtc_server( )
  - canonical( )
  - dns_canonical( )
  - ttl( )
- AUTHOR
- SEE ALSO
- COPYRIGHT

NAME

Infoblox::DTC::Record::CNAME - A DTC CNAME record object.

DESCRIPTION

A DTC CNAME object represents a DNS Traffic Control Canonical Name (DTC CNAME) resource record. DTC CNAME record maps domain name alias to its canonical domain name.

CONSTRUCTOR

```
my $object = Infoblox::DTC::Record::CNAME->new(
    'dtc_server' => $dtc_server,       #Required
    'canonical'  => $fqdn,             #Required
    'comment'    => $string,           #Optional
    'disable'    => 'true' | 'false', #Optional / Default is 'false'
    'ttl'        => $uint | undef,     #Optional / Default is undefined
);
```

SESSION METHODS

This section describes all the methods in an Infoblox::Session module that can be applied to a DTC CNAME record object.

Infoblox::Session->add( )

Use this method to add an object to the Infoblox appliance. See Infoblox::Session->add() for parameters and return values.

Example

```
#Construct an object
my $object = Infoblox::DTC::Record::CNAME->new(
    'comment'    => 'Sample DTC CNAME record',
    'disable'    => 'false',
    'dtc_server' => $dtc_server,
    'canonical'  => 'dtc.localdomain',
    'ttl'        => 1024,
);

# Submit for addition
my $response = $session->add($object);
```

Infoblox::Session->get( )

Use this method to retrieve the existing objects from an Infoblox appliance. See Infoblox::Session->get() for parameters and return values.

Key Reference

```
Apply the following attributes to get specific DTC CNAME record objects:

    dtc_server - Required. DTC Server object the DTC CNAME record is associated with.
    comment    - Optional. Descriptive comment in string format.
    canonical  - Optional. DTC CNAME record canonical name in string format.
```

Example

```
my @retrieved_objs = $session->get(
    object     => 'Infoblox::DTC::Record::CNAME',
    dtc_server => $dtc_server,
);

my @retrieved_objs = $session->get(
   object     => 'Infoblox::DTC::Record::CNAME',
   dtc_server => $dtc_server,
   canonical  => 'dtc.localdomain',
);
```

Infoblox::Session->modify( )

Use this method to modify an object in the Infoblox appliance. See Infoblox::Session->modify() for parameters and return values.

Example

```
# Use this method to modify the comment.
$object->comment('this is a modified comment');

# Submit the modification
my $response = $session->modify($object);
```

Infoblox::Session->remove( )

Use this method to submit the removal of an object from the Infoblox appliance. See Infoblox::Session->remove() for parameters and return values.

To remove a specific object, first use `get()` or `search()` to retrieve the object, and then submit it for removal.

Example

```
# Get the objects associated with DTC Server $dtc_server
my @retrieved_objs = $session->get(
    object     => 'Infoblox::DTC::Record::CNAME',
    dtc_server => $dtc_server,
);

# Find the desired object in the retrieved list.
my $desired_rec = $retrieved_objs[0];

# Submit for removal
my $response = $session->remove($desired_rec);
```

Infoblox::Session->search( )

Use this method to retrieve the existing objects from an Infoblox appliance. See Infoblox::Session->search() for parameters and return values.

Key Reference

```
Apply the following attributes to search the DTC CNAME record objects:

    dtc_server - Required. DTC Server object the DTC CNAME record is associated with (exact).
    comment    - Optional. Descriptive comment in string format (regexp).
    canonical  - Optional. DTC CNAME canonical name (regexp).
```

Example

```
# Get the objects associated with DTC Server $dtc_server
my @retrieved_objs = $session->search(
    object     => 'Infoblox::DTC::Record::CNAME',
    dtc_server => $dtc_server,
);

# Get the objects associated with DTC Server $dtc_server with comment that starts from 'A'
my @retrieved_objs = $session->get(
    object     => 'Infoblox::DTC::Record::CNAME',
    dtc_server => $dtc_server,
```

```
        comment     => '^A.*',
);
```

METHODS

This section describes all the methods that can be used to retrieve the attribute values of a DTC CNAME record object.

auto_created( )

Use this method to retrieve the flag that indicates whether this record was automatically created by NIOS or not. This is a read-only attribute.

Parameter

None

Returns

The method returns the attribute value.

Example

```
#Get auto_created value
my $auto_created = $object->auto_created();
```

comment( )

Use this method to set or retrieve the descriptive comment of a DTC CNAME record object.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

Desired comment in string format with a maximum of 256 characters.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get comment value
my $comment = $object->comment();

#Modify comment value
$object->comment('This is a new comment');
```

disable( )

Use this method to set or retrieve the disable flag of a DTC CNAME record.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

Specify 'true' to set the disable flag or 'false' to deactivate/unset it.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get disable value
my $disable = $object->disable();

#Modify disable value
$object->disable('true');
```

dtc_server( )

Use this method to set or retrieve the DTC Server the DTC CNAME record associated with.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Note that once the DTC record was created, the DTC Server value cannot be changed.

Parameter

The valid value is an Infoblox::DTC::Server object.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get the dtc_server field
my $dtc_server = $object->dtc_server();

#Modify dtc_server value
$object->dtc_server($server1);
```

canonical( )

Use this method to set or retrieve the canonical name value.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

The valid value is desired canonical name in string format. Note that canonical name should be in Fully-Qualified Domain Name (FQDN) format.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get canonical value
my $canonical = $object->canonical();

#Modify canonical value
$object->canonical('dtc.localdomain');
```

dns_canonical( )

Use this method to retrieve the canonical name as served by DNS protocol. This is a read-only attribute.

Omit the parameter to retrieve the attribute value.

Parameter

None

Returns

The method returns the attribute value.

Example

```
#Get dns_canonical value
my $dns_canonical = $object->dns_canonical();
```

ttl( )

Use this method to set or retrieve the Time to Live (TTL) value.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

The default value is undefined which indicates that the record inherits the TTL value of the DTC server pool.

Specify a TTL value to override the TTL value at the DTC server pool level.

Parameter

The valid value is an unsigned integer between 0 and 4294967295 that represents the duration (in seconds) that the record is

cached. Zero indicates that the record should not be cached.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get ttl value
my $ttl = $object->ttl();

#Modify ttl value
$object->ttl(1800);

#inherit ttl value
$object->ttl(undef);
```

AUTHOR

Infoblox Inc. http://www.infoblox.com/

SEE ALSO

Infoblox::Session, Infoblox::Session->add(), Infoblox::Session->get(), Infoblox::Session->modify(), Infoblox::Session->remove(), Infoblox::Session->search(), Infoblox::DTC::Server

---

COPYRIGHT

Copyright (c) 2017 Infoblox Inc.

Infoblox::DTC::Record::NAPTR - A DTC NAPTR record object.

- NAME
- DESCRIPTION
- CONSTRUCTOR
- SESSION METHODS
  - Infoblox::Session->add( )
  - Infoblox::Session->get( )
  - Infoblox::Session->modify( )
  - Infoblox::Session->remove( )
  - Infoblox::Session->search( )
- METHODS
  - comment( )
  - disable( )
  - dtc_server( )
  - flags( )
  - order( )
  - preference( )
  - regexp( )
  - replacement( )
  - services( )
  - ttl( )
- AUTHOR
- SEE ALSO
- COPYRIGHT

NAME

Infoblox::DTC::Record::NAPTR - A DTC NAPTR record object.

DESCRIPTION

A DTC NAPTR object represents a DNS Traffic Control Naming Authority Pointer (DTC NAPTR) resource record. This resource record specifies a regular expression-based rewrite rule that, when applied to an existing string, produces a new domain name or URI.

CONSTRUCTOR

```
my $dtc_naptr = Infoblox::DTC::Record::NAPTR->new(
    'dtc_server'  => $dtc_server,      #Required
    'order'       => $num,             #Required
    'preference'  => $num,             #Required
    'replacement' => $string           #Required
    'comment'     => $string,          #Optional
    'disable'     => 'true' | 'false', #Optional / Default is 'false'
    'flags'       => $string,          #Optional
    'regexp'      => $string,          #Optional
    'services'    => $string,          #Optional
    'ttl'         => $uint | undef,    #Optional / Default is undefined
);
```

SESSION METHODS

This section describes all the methods in an Infoblox::Session module that can be applied to a DTC NAPTR record object.

Infoblox::Session->add( )

Use this method to add an object to the Infoblox appliance. See Infoblox::Session->add() for parameters and return values.

Example

```
#Construct an object
my $dtc_naptr = Infoblox::DTC::Record::NAPTR->new(
    'comment'     => 'Sample DTC NAPTR record',
    'disable'     => 'false',
    'dtc_server'  => $dtc_server,
    'flags'       => 'U',
    'order'       => 100,
    'preference'  => 10,
    'regexp'      => '!http://my[.](.*)!1!i',
```

```
    'replacement' => 'domain2.com',
    'services'    => 'http+E2U',
    'ttl'         => 1024,
);

# Submit for addition
my $response = $session->add($dtc_naptr);
```

Infoblox::Session->get( )

Use this method to retrieve the existing objects from an Infoblox appliance. See Infoblox::Session->get() for parameters and return values.

Key Reference

```
Apply the following attributes to get specific DTC NAPTR record objects:

    dtc_server      - Required. DTC Server object the DTC NAPTR record is associated with.
    comment         - Optional. Descriptive comment in string format.
    flags           - Optional. Flags in string format.
    order           - Optional. Unsigned integer that represent DTC NAPTR order.
    preference      - Optional. Unsigned integer that represent DTC NAPTR preference.
    replacement     - Optional. Replacement field in string format.
    services        - Optional. Services field in string format.
```

Example

```
my @retrieved_objs = $session->get(
    object     => 'Infoblox::DTC::Record::NAPTR',
    dtc_server => $dtc_server,
);

my @retrieved_objs = $session->get(
   object     => 'Infoblox::DTC::Record::NAPTR',
   dtc_server => $dtc_server,
   order      => 1,
);
```

Infoblox::Session->modify( )

Use this method to modify an object in the Infoblox appliance. See Infoblox::Session->modify() for parameters and return values.

Example

```
# Use this method to modify the comment.
$dtc_naptr->comment('this is a modified comment');

# Submit the modification
my $response = $session->modify($dtc_naptr);
```

Infoblox::Session->remove( )

Use this method to submit the removal of an object from the Infoblox appliance. See Infoblox::Session->remove() for parameters and return values.

To remove a specific object, first use `get()` or `search()` to retrieve the object, and then submit it for removal.

Example

```
# Get the objects associated with DTC Server $dtc_server
my @retrieved_objs = $session->get(
    object     => 'Infoblox::DTC::Record::NAPTR',
    dtc_server => $dtc_server,
);

# Find the desired object in the retrieved list.
my $desired_rec = $retrieved_objs[0];

# Submit for removal
my $response = $session->remove($desired_rec);
```

Infoblox::Session->search( )

Use this method to retrieve the existing objects from an Infoblox appliance. See Infoblox::Session->search() for parameters and return values.

Key Reference

```
Apply the following attributes to search the DTC NAPTR record objects:

    dtc_server      - Required. DTC Server object the DTC NAPTR record is associated with (exact).
    comment         - Optional. Descriptive comment in string format (regexp).
    flags           - Optional. Flags in string format (regexp).
    order           - Optional. Unsigned integer that represent DTC NAPTR order (exact).
    preference      - Optional. Unsigned integer that represent DTC NAPTR preference (exact).
    replacement     - Optional. Replacement field in string format (regexp).
    services        - Optional. Services field in string format (regexp).
```

Example

```
# Get the objects associated with DTC Server $dtc_server
my @retrieved_objs = $session->search(
    object     => 'Infoblox::DTC::Record::NAPTR',
    dtc_server => $dtc_server,
);

# Get the objects associated with DTC Server $dtc_server with comment that starts from 'NAPTR'
my @retrieved_objs = $session->get(
    object     => 'Infoblox::DTC::Record::NAPTR',
    dtc_server => $dtc_server,
    comment    => '^NAPTR.*',
);
```

METHODS

This section describes all the methods that can be used to retrieve the attribute values of a DTC NAPTR record object.

comment( )

Use this method to set or retrieve the descriptive comment of a DTC NAPTR record object.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

Desired comment in string format with a maximum of 256 characters.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get comment value
my $comment = $dtc_naptr->comment();
#Modify comment value
$dtc_naptr->comment('This is a new comment');
```

disable( )

Use this method to set or retrieve the disable flag of a DTC NAPTR record.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

Specify 'true' to set the disable flag or 'false' to deactivate/unset it.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get disable value
my $disable = $dtc_naptr->disable();
#Modify disable value
$dtc_naptr->disable('true');
```

dtc_server( )

Use this method to set or retrieve the DTC Server the DTC NAPTR record associated with.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Note that once the DTC record has been created the DTC Server value cannot be changed.

Parameter

The valid value is an Infoblox::DTC::Server object.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get the dtc_server field
my $dtc_server = $dtc_naptr->dtc_server();

#Modify dtc_server value
$dtc_naptr->dtc_server($server1);
```

flags( )

Use this method to retrieve the flags that control the interpretation of the fields for a DTC NAPTR record object.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

Currently supported values for the flags field are 'U', 'S', 'P' and 'A'.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get flags value
my $flags = $dtc_naptr->flags();
#Modify flags value
$dtc_naptr->flags('U');
```

order( )

Use this method to retrieve the order parameter of a DTC NAPTR record. This parameter specifies the order in which the DTC NAPTR rules are applied when multiple rules are present.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

The order parameter is a 16-bit unsigned integer.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get order value
my $order = $dtc_naptr->order();
#Modify order value
$dtc_naptr->order(200);
```

preference( )

Use this method to retrieve the preference field of a DTC NAPTR record. The preference field determines the order DTC NAPTR records are processed when multiple records with the same order parameter are present.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

The preference parameter is a 16-bit integer.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get preference value
my $preference = $dtc_naptr->preference();
#Modify preference value
$dtc_naptr->preference(300);
```

regexp( )

Use this method to specify the regular expression-based rewriting rule of a DTC NAPTR record. This should be a POSIX compliant regular expression, including the substitution rule and flags. Refer to RFC 2915 for the field syntax details.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

The valid parameter is a POSIX compliant regular expression or substitution expression.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get regexp value
my $regexp = $dtc_naptr->regexp();
#Modify regexp value
$dtc_naptr->regexp('!http://(.*)!1!i');
```

replacement( )

Use this method to retrieve the replacement field of a DTC NAPTR record object. For non-terminal DTC NAPTR records, this field specifies the next domain name to look up.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

The attribute value can be in unicode format.

Parameter

The desired replacement value in a Fully-Qualified Domain Name (FQDN) format.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get replacement value
my $replacement = $dtc_naptr->replacement();
#Modify replacement value
$dtc_naptr->replacement('__tcp__.domain.com');
```

services( )

Use this field to specify services. The services field contains protocol and service identifiers, for example 'http+E2U' or 'SIPS+D2T'.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

The services field in string format.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get services value
my $services = $dtc_naptr->services();
#Modify services value
$dtc_naptr->services('http+E2U');
```

ttl( )

Use this method to set or retrieve the Time to Live (TTL) value.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

The default value is undefined which indicates that the record inherits the TTL value of the DTC server pool.

Specify a TTL value to override the TTL value at the DTC server pool level.

Parameter

The valid value is an unsigned integer between 0 and 4294967295 that represents the duration (in seconds) that the record is cached. Zero indicates that the record should not be cached.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get ttl value
my $ttl = $dtc_naptr->ttl();

#Modify ttl value
$dtc_naptr->ttl(1800);

#inherit ttl value
$dtc_naptr->ttl(undef);
```

AUTHOR

Infoblox Inc. http://www.infoblox.com/

SEE ALSO

Infoblox::Session, Infoblox::Session->add(), Infoblox::Session->get(), Infoblox::Session->modify(), Infoblox::Session->remove(), Infoblox::Session->search(), Infoblox::DTC::Server

---

COPYRIGHT

Copyright (c) 2017 Infoblox Inc.

Infoblox::DTC::Server - A DTC Server object.

- NAME
- DESCRIPTION
- CONSTRUCTOR
- SESSION METHODS
  - Infoblox::Session->add( )
  - Infoblox::Session->get( )
  - Infoblox::Session->modify( )
  - Infoblox::Session->remove( )
  - Infoblox::Session->search( )
- METHODS
  - auto_create_host_record( )
  - comment( )
  - disable( )
  - extattrs( )
  - extensible_attributes( )
  - health( )
  - host( )
  - monitors( )
  - name( )
  - sni_hostname( )
  - override_sni_hostname( )
- AUTHOR
- SEE ALSO
- COPYRIGHT

NAME

Infoblox::DTC::Server - A DTC Server object.

DESCRIPTION

A DTC Server object.

CONSTRUCTOR

```
 my $server = Infoblox::DTC::Server->new(
    name                    => $string,
#Required
    host                    => $string,
#Reqiured
    auto_create_host_record => 'true' | 'false',
#Optional Default is 'false'
    comment                 => $string,
#Optional / Default is undefined
    disable                 => 'true' | 'false',
#Optional / Default is 'false'
    sni_hostname            => $fqdn,
#Optional / Default is undefined
    override_sni_hostname   => 'true' | 'false',
#Optional / Default is 'false'
    monitors                => [$dtc_server_monitor1, $dtc_server_monitor2, ...],
#Optional / Default is empty list
    extattrs                => { $string => $extattr, ... },
#Optional / Default is undefined
    extensible_attributes   => { $string => $string | $num, $string => [ $string | $num, ... ], ... }
#Optional / Default is undefined
 );
```

You cannot set both extattrs and extensible_attributes attributes at the same time.

SESSION METHODS

This section describes all the methods in a Session module that you can apply to a DTC Server object.

Infoblox::Session->add( )

Use this method to add an object to the Infoblox appliance. See Infoblox::Session->add() for parameters and return values.

Example

```
#construct an object
my $server = Infoblox::DTC::Server->new(
   name  => 'server1',
   host  => '192.168.1.2',
);

#submit for addition
my $response = $session->add($server);
```

Infoblox::Session->get( )

Use this method to retrieve all the matching objects from the Infoblox appliance. See Infoblox::Session->get() for parameters and return values.

Key Reference

```
Apply the following attribute to get a specific object:

 name                  - Optional. A DTC Server name in a string format.
 host                  - Optional. A DTC Server host address or FQDN.
 comment               - Optional. A DTC Server comment.
 extattrs              - Optional. A hash reference containing extensible attributes.
 extensible_attributes - Optional. A hash reference that contains extensible attributes.
 pool                  - Optional. A DTC Pool which is linked with a DTC Server.
 status_member         - Optional. A member that provides the object health status. The default is
Grid.
 sni_hostname          - Optional. A hostname for Server Network Indication (SNI) to be used with
the HTTPS monitor.
```

Example

```
my @retrieved_objs = $session->get(
    object => 'Infoblox::DTC::Server',
    name   => 'server1',
    host   => '192.168.1.2'
);

my @retrieved_objs = $session->get(
    object => 'Infoblox::DTC::Server',
    extensible_attributes => { 'Site' => 'Santa Clara' }
);
```

Infoblox::Session->modify( )

Use this method to modify an object in the Infoblox appliance. See Infoblox::Session->modify() for parameters and return values.

Example

```
#Use method to modify the comment.
$server->comment('this is a modified comment');
#Submit modification
my $response = $session->modify($server);
```

Infoblox::Session->remove( )

Use this method to remove an object from the Infoblox appliance. See Infoblox::Session->remove() for parameters and return values.

To remove a specific object, first use `get()` or `search()` to retrieve the specific object, and then submit this object for removal.

Example

```
#Get the objects with the same name
my @retrieved_objs = $session->get(
    object => 'Infoblox::DTC::Server',
    name   => 'server1',
);
#Find the desired object from the retrieved list.
my $desired_server = $retrieved_objs[0];
#Submit for removal
my $response = $session->remove($desired_server);
```

Infoblox::Session->search( )

Use this method to search for objects in the Infoblox appliance. See Infoblox::Session->search() for parameters and return values.

Key Reference

```
Apply the following attribute to get a specific object:
```

```
  name                  - Optional. A DTC Server name in a string format.
  host                  - Optional. A DTC Server host address or FQDN.
  comment               - Optional. A DTC Server comment.
  extattrs              - Optional. A hash reference containing extensible attributes.
  extensible_attributes - Optional. A hash reference that contains extensible attributes.
  pool                  - Optional. A DTC Pool which is linked with a DTC Server.
  status_member         - Optional. A member that provides the object health status. The default is
Grid.
  sni_hostname          - Optional. A hostname for Server Network Indication (SNI) to be used with
the HTTPS monitor.
```

Example

```
my @retrieved_objs = $session->get(
    object => 'Infoblox::DTC::Server',
    name   => 'server1',
    host   => '192.168.1.2'
);

my @retrieved_objs = $session->get(
    object => 'Infoblox::DTC::Server',
    extensible_attributes => { 'Site' => 'Santa Clara' }
);
```

METHODS

This section describes all the methods that you can use to set or retrieve the attribute values of the object.

auto_create_host_record( )

Use this method to set or retrieve the flag that indicates whether the generation of resource records that represents the host is allowed or not.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

Specify 'true' to allow host record generation and 'false' to forbid it. The default value is 'true'.

Returns

If you specified a parameter, the method returns 'true' when the modification succeeds, and returns 'false' when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#get auto_create_host_record value
my $auto_create_host_record = $server->auto_create_host_record();

#modify auto_create_host_record value
$server->auto_create_host_record('false');
```

comment( )

Use this method to set or retrieve a descriptive comment.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

The valid value is a comment in string format (UTF-8) with a maximum of 256 bytes.

Returns

If you specified a parameter, the method returns 'true' when the modification succeeds, and returns 'false' when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#get comment value
my $comment = $server->comment();

#modify comment value
$server->comment('desired comment');
```

disable( )

Use this method to enable or disable a DTC Server.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

Specify 'true' to disable a DTC Server or 'false' to enable it. The default value is 'false'.

Returns

If you specified a parameter, the method returns 'true' when the modification succeeds, and returns 'false' when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#get disable value
my $disable = $server->disable();

#modify disable value
$server->disable('true');
```

extattrs( )

Use this method to set or retrieve the extensible attributes associated with a DTC Server object.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

Valid value is a hash reference containing the names of extensible attributes and their associated values (Infoblox::Grid::Extattr objects).

Returns

If you specified a parameter, the method returns 'true' when the modification succeeds, and returns 'false' when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#get extattrs value
my $ref_extattrs = $server->extattrs();

#Modify extattrs
$server->extattrs({ 'Site' => $extattr1, 'Administrator' => $extattr2 });
```

extensible_attributes( )

Use this method to set or retrieve the extensible attributes associated with a DTC Server object.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

For valid values for extensible attributes, see Infoblox::Grid::ExtensibleAttributeDef/Extensible Attribute Values.

Returns

If you specified a parameter, the method returns 'true' when the modification succeeds, and returns 'false' when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get extensible attributes
my $ref_extensible_attributes = $server->extensible_attributes();

#Modify extensible attributes
$server->extensible_attributes({'Site' => 'Santa Clara', 'Administrator' => ['Peter', 'Tom']});
```

health( )

Use this method to retrieve a DTC Server health information. This is a read-only attribute.

Parameter

None

Returns

The valid return value is an Infoblox::DTC::Health object.

Example

```
#get health value
my $health = $server->health();
```

host( )

Use this method to set or retrieve an address or FQDN of a DTC Server.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

The valid value is a string containing an address or FQDN of a DTC server.

Returns

If you specified a parameter, the method returns 'true' when the modification succeeds, and returns 'false' when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#get host value
my $host = $server->host();

#modify host value
$server->host('192.168.1.2');
```

monitors( )

Use this method to set or retrieve the list of DTC health monitors for the DTC Server.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameters

The valid value is an array reference that contains Infoblox::DTC::Server::Monitor objects.

Returns

If you specified a parameter, the method returns 'true' when the modification succeeds, and returns 'false' when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#get monitors value
my $monitors = $server->monitors();

#retrieve TCP monitor
my $tcp_monitor = $session->get(
    object => 'Infoblox::DTC::Monitor::TCP',
    name   => 'tcp_monitor',
);

#construct DTC server monitor
my $monitor = Infoblox::DTC::Server::Monitor->new(
    monitor => $tcp_monitor,
    host    => '192.168.1.10',
);

#modify monitors value
$server->monitors([$monitor]);
```

name( )

Use this method to set or retrieve the DTC Server name.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

Desired name in string format with a maximum of 256 bytes.

Returns

If you specified a parameter, the method returns 'true' when the modification succeeds, and returns 'false' when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#get name value
my $name = $server->name();

#modify name value
$server->name('server_name');
```

sni_hostname( )

Use this method to set or retrieve the hostname for Server Network Indication (SNI) to be used with the HTTPS monitor.

Setting this method to a defined value implicitly sets the override_sni_hostname method to 'true'. Setting the parameter to undefined causes the HTTPS monitor to use DTC server's hostname with SNI option and automatically resets the override_sni_hostname method to 'false'.

Note that when sni_hostname is set to a defined value and override_sni_hostname is set to 'false', the last operation takes precedence. Thus the sequence $object->sni_hostname('sni-host.infoblox.localdomain'); $object->override_sni_hostname('false'); will set override_sni_hostname to 'false', and the sequence $object->override_sni_hostname('false'); $object->sni_hostname('sni-host.infoblox.localdomain'); will result in override_sni_hostname='true'.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

The valid value is the FQDN for the desired host in string format.

Returns

If you specified a parameter, the method returns 'true' when the modification succeeds, and returns 'false' when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#get sni_hostname value
my $sni_hostname = $server->sni_hostname();

#modify sni_hostname value
$server->sni_hostname('sni-host.infoblox.localdomain');
```

override_sni_hostname( )

The override_sni_hostname attribute controls whether the sni_hostname method values of the DTC server are used, instead of the DTC server name.

The override_sni_hostname attribute can be specified explicitly. It is also set implicitly when sni_hostname is set to a defined value.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

Set the parameter to 'true' to allow usage of the sni_hostname. Set the parameter to 'false' to use the DTC server name with the SNI option.

The default value of this parameter is 'false'.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get override_sni_hostname value
my $override_sni_hostname = $server->override_sni_hostname();

#Modify override_sni_hostname value
$server->override_sni_hostname('true');
```

AUTHOR

Infoblox Inc. http://www.infoblox.com/

SEE ALSO

Infoblox::Session, Infoblox::Session->add(), Infoblox::Session->get(), Infoblox::Session->modify(), Infoblox::Session->remove(), Infoblox::Session->search(), Infoblox::Grid::Extattr, Infoblox::Grid::ExtensibleAttributeDef/Extensible Attribute Values, Infoblox::DTC::Health.

COPYRIGHT

Copyright (c) 2017 Infoblox Inc.

Infoblox::DTC::Server::Link - An DTC Server Link object.

- NAME
- DESCRIPTION
- CONSTRUCTOR
- SESSION METHODS
- METHODS
  - server( )
  - ratio( )
- AUTHOR
- SEE ALSO
- COPYRIGHT

NAME

Infoblox::DTC::Server::Link - An DTC Server Link object.

DESCRIPTION

An DTC Pool Link object.

CONSTRUCTOR

```
my $server_link = Infoblox::DTC::Server::Link->new(
    server   => $server, #Optional / Default is undefined
    ratio    => $uint,   #Optional / Default is undefined
);
```

SESSION METHODS

The object does not support any session methods.

METHODS

This section describes all the methods that you can use to set or retrieve the attribute values of the object.

server( )

Use this method to set or retrieve a DTC Server to link with.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

The valid value is an Infoblox::DTC::Server object.

Returns

If you specified a parameter, the method returns 'true' when the modification succeeds, and returns 'false' when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#get dtc_server value
my $server = $server_link->server();

#modify dtc_server value
$server_link->dtc_server($server);
```

ratio( )

Use this method to set or retrieve the weight of a DTC Server.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

The valid value is an unsigned integer.

Returns

If you specified a parameter, the method returns 'true' when the modification succeeds, and returns 'false' when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#get ratio value
my $ratio = $server_link->ratio();

#modify ratio value
$server_link->ratio(70);
```

AUTHOR

Infoblox Inc. http://www.infoblox.com/

SEE ALSO

Infoblox::DTC::Server.

COPYRIGHT

Copyright (c) 2017 Infoblox Inc.

Infoblox::DTC::Server::Monitor - A DTC Server Monitor object.

- NAME
- DESCRIPTION
- CONSTRUCTOR
- SESSION METHODS
- MODULE METHODS
  - Infoblox::DTC::Server->monitors( )
- METHODS
  - host( )
  - monitor( )
- AUTHOR
- SEE ALSO
- COPYRIGHT

NAME

Infoblox::DTC::Server::Monitor - A DTC Server Monitor object.

DESCRIPTION

The DTC Server Monitor object is used to configure DTC Health Monitor for the particular DTC Server.

CONSTRUCTOR

```
 my $server_monitor = Infoblox::DTC::Server::Monitor->new(
     monitor => $tcp_monitor | $icmp_monitor | $snmp_monitor | $sip_monitor | $pdp_monitor | $http_monitor,
#Required
     host    => $fqdn | $ipaddr,
#Optional / Default is undefined
```

SESSION METHODS

The object does not support any session methods.

MODULE METHODS

This section describes all the functions that you can apply to an DTC Server Monitor object.

Infoblox::DTC::Server->monitors( )

Use this method to set or retrieve the list of DTC health monitors for the DTC Server. See Infoblox::DTC::Server->monitors() for parameters and return values.

Example

```
#get monitors value
my $monitors = $server->monitors();

#modify monitors value

#retrieve TCP monitor
my $tcp_monitor = $session->get(
    object => 'Infoblox::DTC::Monitor::TCP',
    name   => 'tcp_monitor',
);

#construct DTC server monitor
my $monitor = Infoblox::DTC::Server::Monitor->new(
    monitor => $tcp_monitor,
    host    => '192.168.1.10',
);

#modify monitors value
$server->monitors([$monitor]);
```

METHODS

This section describes all the methods that you can use to set or retrieve the attribute values of the object.

host( )

Use this method to set or retrieve the address that is a subject of the DTC health monitoring.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

The valid value is an IP address or FQDN in string format.

Returns

If you specified a parameter, the method returns 'true' when the modification succeeds, and returns 'false' when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get host value
my $value = $object->host();

#Modify host value
$object->host('1.1.1.1');
```

monitor( )

Use this method to set or retrieve the DTC Health Monitor used to monitor the DTC Server.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

The valid value is one of following objects: Infoblox::DTC::Monitor::HTTP, Infoblox::DTC::Monitor::SIP, Infoblox::DTC::Monitor::PDP, Infoblox::DTC::Monitor::SNMP, Infoblox::DTC::Monitor::TCP or Infoblox::DTC::Monitor::ICMP.

Note that yo must first retrieve the Monitor object from the session.

Returns

If you specified a parameter, the method returns 'true' when the modification succeeds, and returns 'false' when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get monitor value
my $value = $object->monitor();

#Retrieve TCP monitor
my $tcp_monitor = $session->get(
    object => 'Infoblox::DTC::Monitor::TCP',
    name   => 'tcp_monitor',
);

#Modify monitor value
$object->monitor($tcp_monitor);
```

AUTHOR

Infoblox Inc. http://www.infoblox.com/

SEE ALSO

Infoblox::DTC::Server, Infoblox::DTC::Server->monitors(), Infoblox::DTC::Monitor::HTTP, Infoblox::DTC::Monitor::SIP, Infoblox::DTC::Monitor::PDP, Infoblox::DTC::Monitor::SNMP, Infoblox::DTC::Monitor::TCP, Infoblox::DTC::Monitor::ICMP.

COPYRIGHT

Copyright (c) 2017 Infoblox Inc.

Infoblox::DTC::Topology - A DTC Topology object.

- NAME
- DESCRIPTION
- CONSTRUCTOR
- SESSION METHODS
  - Infoblox::Session->add( )
  - Infoblox::Session->get( )
  - Infoblox::Session->modify( )
  - Infoblox::Session->remove( )
  - Infoblox::Session->search( )
- METHODS
  - comment( )
  - extattrs( )
  - extensible_attributes( )
  - name( )
  - rules( )
- AUTHOR
- SEE ALSO
- COPYRIGHT

NAME

Infoblox::DTC::Topology - A DTC Topology object.

DESCRIPTION

A DTC Topology is a named list of ordered DTC topology rules. These rules cannot be shared. Topology rules map client IPs to pools or resources.

CONSTRUCTOR

```
 my $topology = Infoblox::DTC::Topology->new(
    name                  => $string,
#Required
    comment               => $string,
#Optional / Default is undefined
    rules                 => [$rule1, $rule2, ...],
#Optional / Default is undefined
    extattrs              => { $string => $extattr, ... },
#Optional / Default is undefined
    extensible_attributes => { $string => $string | $num, $string => [ $string | $num, ... ], ... }
#Optional / Default is undefined
 );
```

You cannot set both extattrs and extensible_attributes attributes at the same time.

SESSION METHODS

This section describes all the methods in a Session module that you can apply to a Topology object.

Infoblox::Session->add( )

Use this method to add an object to the Infoblox appliance. See Infoblox::Session->add() for parameters and return values.

Example

```
#construct an object
my $topology = Infoblox::DTC::Topology->new(
   name  => 'topology',
   rules => [$rule1, $rule2, ...],
);

#submit for addition
my $response = $session->add($topology);
```

Infoblox::Session->get( )

Use this method to retrieve all the matching objects from the Infoblox appliance. See Infoblox::Session->get() for parameters and return values.

Key Reference

```
Apply the following attribute to get a specific object:

 name                  - Optional. A DTC Topology name in a string format.
 comment               - Optional. A DTC Topology comment.
 extattrs              - Optional. A hash reference containing extensible attributes.
 extensible_attributes - Optional. A hash reference that contains extensible attributes.
```

Example

```
my @retrieved_objs = $session->get(
    object => 'Infoblox::DTC::Topology',
    name   => 'topology',
);

my @retrieved_objs = $session->get(
    object => 'Infoblox::DTC::Topology',
    extensible_attributes => { 'Site' => 'Santa Clara' }
);
```

Infoblox::Session->modify( )

Use this method to modify an object in the Infoblox appliance. See Infoblox::Session->modify() for parameters and return values.

Example

```
#Use method to modify the comment
$topology->comment('this is a modified comment');
#Submit modification
my $response = $session->modify($topology);
```

Infoblox::Session->remove( )

Use this method to remove an object from the Infoblox appliance. See Infoblox::Session->remove() for parameters and return values.

To remove a specific object, first use `get()` or `search()` to retrieve the specific object, and then submit this object for removal.

Example

```
#Get the objects with the same name
my @retrieved_objs = $session->get(
    object => 'Infoblox::DTC::Topology',
    name   => 'topology',
);
#Find the desired object from the retrieved list.
my $desired_topology = $retrieved_objs[0];
#Submit for removal
my $response = $session->remove($desired_topology);
```

Infoblox::Session->search( )

Use this method to search for objects in the Infoblox appliance. See Infoblox::Session->search() for parameters and return values.

Key Reference

```
Apply the following attribute to get a specific object:

 name                  - Optional. A DTC Topology name in a string format.
 comment               - Optional. A DTC Topology comment.
 extattrs              - Optional. A hash reference containing extensible attributes.
 extensible_attributes - Optional. A hash reference that contains extensible attributes.
```

Example

```
my @retrieved_objs = $session->search(
    object => 'Infoblox::DTC::Topology',
    name   => 'top.*',
);

my @retrieved_objs = $session->search(
    object => 'Infoblox::DTC::Topology',
    extensible_attributes => { 'Site' => 'Santa Clara' }
);
```

METHODS

This section describes all the methods that you can use to set or retrieve the attribute values of the object.

comment( )

Use this method to set or retrieve a descriptive comment.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

Desired comment in a string format (UTF-8) with a maximum of 256 bytes.

Returns

If you specified a parameter, the method returns 'true' when the modification succeeds, and returns 'false' when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#get comment value
my $comment = $topology->comment();

#modify comment value
$topology->comment('desired comment');
```

extattrs( )

Use this method to set or retrieve the extensible attributes associated with a DTC Topology object.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

Valid value is a hash reference containing the names of extensible attributes and their associated values (Infoblox::Grid::Extattr objects).

Returns

If you specified a parameter, the method returns 'true' when the modification succeeds, and returns 'false' when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#get extattrs value
my $ref_extattrs = $topology->extattrs();

#Modify extattrs
$topology->extattrs({ 'Site' => $extattr1, 'Administrator' => $extattr2 });
```

extensible_attributes( )

Use this method to set or retrieve the extensible attributes associated with a DTC Topology object.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

For valid values for extensible attributes, see Infoblox::Grid::ExtensibleAttributeDef/Extensible Attribute Values.

Returns

If you specified a parameter, the method returns 'true' when the modification succeeds, and returns 'false' when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get extensible attributes
my $ref_extensible_attributes = $topology->extensible_attributes();

#Modify extensible attributes
$topology->extensible_attributes({'Site' => 'Santa Clara', 'Administrator' => ['Peter', 'Tom']});
```

name( )

Use this method to set or retrieve topology name.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

The valid value is a topology list name in a string format.

Returns

If you specified a parameter, the method returns 'true' when the modification succeeds, and returns 'false' when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#get name value
my $name = $topology->name();

#modify name value
$topology->name('topology1');
```

rules( )

Use this method to set or retrieve a list of topology rules.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

The valid value is an Infoblox::DTC::Topology::Rule object.

Returns

If you specified a parameter, the method returns 'true' when the modification succeeds, and returns 'false' when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#get rules value
my $rules = $topology->rules();

#modify rules value
$topology->rules([$rule1, $rule2]);
```

AUTHOR

Infoblox Inc. http://www.infoblox.com/

SEE ALSO

Infoblox::Session, Infoblox::Session->add(), Infoblox::Session->get(), Infoblox::Session->modify(), Infoblox::Session->remove(), Infoblox::Session->search(), Infoblox::Grid::Extattr, Infoblox::Grid::ExtensibleAttributeDef/Extensible Attribute Values, Infoblox::DTC::Topology::Rule.

COPYRIGHT

Copyright (c) 2017 Infoblox Inc.

Infoblox::DTC::Topology::Label - A DTC Topology label object.

- NAME
- DESCRIPTION
- SESSION METHODS
  - Infoblox::Session->get( )
  - Infoblox::Session->search( )
- METHODS
  - field( )
  - label( )
- AUTHOR
- SEE ALSO
- COPYRIGHT

NAME

Infoblox::DTC::Topology::Label - A DTC Topology label object.

DESCRIPTION

A DTC Topology label object, which represents a label in a MaxMind database.

SESSION METHODS

This section describes all the methods in a Session module that you can apply to a Topology label object

Infoblox::Session->get( )

Use this method to retrieve all the matching objects from the Infoblox appliance. See Infoblox::Session->get() for parameters and return values.

Key Reference

```
Apply the following attribute to get a specific object:

  field - Optional. The field in the MaxMind database the label was obtained from.
  label - Optional. The label value.
```

Example

```
my @retrieved_objs = $session->get(
    object => 'Infoblox::DTC::Topology::Label',
    field  => 'COUNTRY',
    label  => 'United States',
);
```

Infoblox::Session->search( )

Use this method to search for objects in the Infoblox appliance. See Infoblox::Session->search() for parameters and return values.

Key Reference

```
Apply the following attribute to get a specific object:

  field - Optional. The field in the MaxMind database from which the label was obtained.
  label - Optional. The label value.
```

Example

```
my @retrieved_objs = $session->search(
    object => 'Infoblox::DTC::Topology::Label',
    field  => 'COUNTRY',
);
```

METHODS

This section describes all the methods that you can use to set or retrieve the attribute values of the object.

field( )

Use this method to retrieve the name of the field in the MaxMind database from which the label was obtained. This is a read-only

attribute.

Parameter

None

Returns

The method returns the attribute value.

Example

```
#get field value
my $field = $label->field();
```

label( )

Use this method to retrieve the label name. This is a read-only attribute.

Parameter

None

Returns

The method returns the attribute value.

Example

```
#get label value
my $label = $label->label();
```

AUTHOR

Infoblox Inc. http://www.infoblox.com/

SEE ALSO

Infoblox::Session, Infoblox::Session->get(), Infoblox::Session->search().

COPYRIGHT

Copyright (c) 2017 Infoblox Inc.

Infoblox::DTC::Topology::Rule - A DTC Topology Rule object.

- NAME
- DESCRIPTION
- CONSTRUCTOR
- SESSION METHODS
  - Infoblox::Session->get( )
  - Infoblox::Session->modify( )
  - Infoblox::Session->search( )
- METHODS
  - destination_link( )
  - destination_type( )
  - sources( )
  - topology( )
  - valid( )
- AUTHOR
- SEE ALSO
- COPYRIGHT

NAME

Infoblox::DTC::Topology::Rule - A DTC Topology Rule object.

DESCRIPTION

Topology Rule describes the behavior of a load balancer used for a matching condition contained in the rules for a 'TOPOLOGY' load balancing method.

CONSTRUCTOR

```
my $rule = Infoblox::DTC::Topology::Rule->new(
   sources          => [$source1, $source2, ...], #Optional / default is undefined
   destination_type => 'POOL' | 'SERVER',         #Optional / default is undefined
   destination_link => $link_object               #Optional / default is undefined
);
```

SESSION METHODS

This section describes all the methods in a Session module that you can apply to a Topology Rule object. Note that to add a rule to the Infoblox appliance, you must add it to the existing Topology object. To remove a rule from the Infoblox appliance, you must remove it from the existing Topology.

Infoblox::Session->get( )

Use this method to retrieve all the matching objects from the Infoblox appliance. See Infoblox::Session->get() for parameters and return values.

Key Reference

```
Apply the following attribute to get a specific object:

 topology         - Optional. A DTC Topology the rule belongs to.
 destination_type - Optional. The type of the destination link.
 destination_link - Optional. The destination link value.
 valid            - Optional. The flag that indicates whether a rule is valid or not.
```

Example

```
my @retrieved_objs = $session->get(
    object => 'Infoblox::DTC::Topology::Rule',
    destination_type => 'POOL',
);
```

Infoblox::Session->modify( )

Use this method to modify an object in the Infoblox appliance. See Infoblox::Session->modify() for parameters and return values.

Example

```
#Use method to modify the comment
```

```
$rule->comment('modified_rule1');
#Submit modification
my $response = $session->modify($rule);
```

Infoblox::Session->search( )

Use this method to search for objects in the Infoblox appliance. See Infoblox::Session->search() for parameters and return values.

Key Reference

```
Apply the following attribute to get a specific object:

 topology         - Optional. A DTC Topology the rule belongs to.
 destination_type - Optional. The type of the destination link.
 destination_link - Optional. The destination link value.
 valid            - Optional. The flag that indicates whether a rule is valid or not.
```

Example

```
my @retrieved_objs = $session->search(
    object => 'Infoblox::DTC::Topology::Rule',
    valid  => 'true',
);
```

METHODS

This section describes all the methods that you can use to set or retrieve the attribute values of the object.

destination_link( )

Use this method to set or retrieve a DTC Topology Rule destination link.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

The valid value is an Infoblox::DTC::Pool or Infoblox::DTC::Server object.

Returns

If you specified a parameter, the method returns 'true' when the modification succeeds, and returns 'false' when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#get destination_link value
my $destination_link = $rule->destination_link();

#modify destination_link value
$rule->destination_link($pool_link);
```

destination_type( )

Use this method to set or retrieve the destination type for a DTC Topology rule.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

The valid values are 'POOL' and 'SERVER'.

Returns

If you specified a parameter, the method returns 'true' when the modification succeeds, and returns 'false' when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#get destination_type value
my $destination_type = $rule->destination_type();

#modify destination_type value
$rule->destination_type('SERVER');
```

sources( )

Use this method to set or retrieve conditions for matching sources.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

The valid value is an array of Infoblox::DTC::Topology::Rule::Source objects.

Returns

If you specified a parameter, the method returns 'true' when the modification succeeds, and returns 'false' when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#get sources value
my @sources = $rule->sources();

#modify sources value
$rule->sources([$source1, $source2]);
```

topology( )

Use this method to retrieve the DTC Topoloyg to which a rule belongs. This is a read-only attribute.

Parameter

None

Returns

The valid return value is an partial Infoblox::DTC::Topology object.

Example

```
#get sources value
my $topology = $rule->topology();
```

valid( )

Use this method to retrieve the status of GeoIP labels. This is a read-only attribute.

Parameter

None

Returns

The method returns 'true' if GeoIP labels exist or 'false' if they do not.

Example

```
#get sources value
my $valid = $rule->valid();
```

AUTHOR

Infoblox Inc. http://www.infoblox.com/

SEE ALSO

Infoblox::Session, Infoblox::Session->get(), Infoblox::Session->modify(), Infoblox::Session->search(), Infoblox::DTC::Pool, Infoblox::DTC::Server, Infoblox::DTC::Topology::Rule::Source, Infoblox::DTC::Topology.

COPYRIGHT

Copyright (c) 2017 Infoblox Inc.

Infoblox::DTC::Topology::Rule::Source - A DTC Topology Rule Source object.

- NAME
- DESCRIPTION
- CONSTRUCTOR
- SESSION METHODS
- METHODS
  - source_op( )
  - source_type( )
  - source_value( )
- AUTHOR
- SEE ALSO
- COPYRIGHT

NAME

Infoblox::DTC::Topology::Rule::Source - A DTC Topology Rule Source object.

DESCRIPTION

A DTC Topology Rule Source object.

CONSTRUCTOR

```
 my $rule_source = Infoblox::DTC::Topology::Rule::Source->new(
    source_type  => 'SUBNET' | 'CONTINENT' | 'COUNTRY' | 'SUBDIVISION' |
                    'CITY' | 'EA0' | 'EA1' | 'EA2' | 'EA3',                    #Required
    source_value => $string,                                                    #Required
    source_op    => 'IS' | 'IS_NOT',                                            #Optional / Default is
'IS'
 );
```

SESSION METHODS

The object does not support any session methods. It's retrieved as a part of an Infoblox::DTC::Topology::Rule, and used as a matching conditions for a DTC load balancer.

METHODS

This section describes all the methods that you can use to set or retrieve the attribute values of the object.

source_op( )

Use this method to set or retrieve the operation used to match the value.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

The valid values are 'IS' and 'IS_NOT'.

Returns

If you specified a parameter, the method returns 'true' when the modification succeeds, and returns 'false' when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#get source_op value
my $source_op = $source->source_op();

#modify source_op value
$source->source_op('IS');
```

source_type( )

Use this method to set or retrieve the source type.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

The valid values are 'SUBNET', 'CONTINENT', 'COUNTRY', 'CITY', 'SUBDIVISION', 'EA0', 'EA1', 'EA2' and 'EA3'.

Returns

If you specified a parameter, the method returns 'true' when the modification succeeds, and returns 'false' when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#get source_type value
my $source_type = $source->source_type();

#modify source_type value
$source->source_type('COUNTRY');
```

source_value( )

Use this method to set or retrieve the source value.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

The valid value is a string that contains the desired source value.

Returns

If you specified a parameter, the method returns 'true' when the modification succeeds, and returns 'false' when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#get source_value value
my $source_value = $source->source_value();

#modify source_value value
$source->source_value('United States');
```

AUTHOR

Infoblox Inc. http://www.infoblox.com/

SEE ALSO

Infoblox::DTC::Topology::Rule

COPYRIGHT

Copyright (c) 2017 Infoblox Inc.

Infoblox::DXL::Endpoint - A Data Exchange Layer endpoint object.

- NAME
- DESCRIPTION
- CONSTRUCTOR
- SESSION METHODS
  - Infoblox::Session->add( )
  - Infoblox::Session->get( )
  - Infoblox::Session->modify( )
  - Infoblox::Session->remove( )
  - Infoblox::Session->search( )
- METHODS
  - brokers( )
  - client_certificate( )
  - client_certificate_subject( )
  - client_certificate_valid_from( )
  - client_certificate_valid_to( )
  - comment( )
  - disable( )
  - extattrs( )
  - extensible_attributes( )
  - name( )
  - outbound_members( )
  - outbound_member_type( )
  - log_level( )
  - template_instance( )
  - vendor_identifier( )
  - wapi_user_name( )
  - wapi_user_password( )
- AUTHOR
- SEE ALSO
- COPYRIGHT

NAME

Infoblox::DXL::Endpoint - A Data Exchange Layer (DXL) endpoint object.

DESCRIPTION

The DXL endpoint object provides configuration for third party DXL server integration.

CONSTRUCTOR

```
 my $endpoint = Infoblox::DXL::Endpoint->new(
     name                  => $string,                                                                  #
Required
     outbound_member_type  => 'GM' | 'MEMBER',                                                          #
Required
     brokers               => [$endpoint_dxl_broker1, ...],                                             #
Required
     client_certificate    => $path,                                                                    #
Required
     comment               => $string,                                                                  #
Optional / Default is undefined
     disable               => 'true' | 'false',                                                         #
Optional / Default is 'false'
     extattrs              => { $string => $extattr, ... },                                             #
Optional / Default is undefined
     extensible_attributes => {$string => $string | $num, $string => [ $string | $num, ... ], ... } #
Optional / Default is undefined
     log_level             => 'WARNING' | 'DEBUG' | 'INFO' | 'ERROR',                                   #
Optional / Default is 'WARNING'
     outbound_members      => [$string, $string, ...],                                                  #
Optional / Default is empty list
     template_instance     => $template_instance                                                        #
Optional / Default is undefined
     vendor_identifier     => $string,                                                                  #
Optional / Default is undefined
     wapi_user_name        => $string,                                                                  #
Optional / Default is undefined
     wapi_user_password    => $string,                                                                  #
Optional / Default is undefined
 );
```

You cannot set both extattrs and extensible_attributes attributes at the same time.

SESSION METHODS

This section describes all the methods in an Infoblox::Session module that you can apply to a DXL endpoint object.

Infoblox::Session->add( )

Use this method to add the object to the Infoblox appliance. See Infoblox::Session->add() for parameters and return values.

Example

```
#Construct Infoblox::DXL::Endpoint::Broker object
my $broker = Infoblox::DXL::Endpoint::Broker->new(
    host_name => 'dxl-br.localdomain',
);

#Construct an object
my $object = Infoblox::DXL::Endpoint->new(
    name                 => 'dxl_ep',
    outbound_member_type => 'GM',
    brokers              => [$broker],
    client_certificate   => '/path/to/certificate.pem',
);

#Submit for addition
my $response = $session->add($object);
```

Infoblox::Session->get( )

Use this method to retrieve all the matching objects from the Infoblox appliance. See Infoblox::Session->get() for parameters and return values.

Key References

```
Apply the following attributes to get a specific DXL endpoint object:

 comment                  - Optional. The DXL endpoint comment in string format.
 name                     - Optional. The DXL endpoint name in string format.
 vendor_identifier        - Optional. The DXL endpoint vendor identifier in string format.
 outbound_member_type     - Optional. The DXL endpoint outbound member type.
 extattrs                 - Optional. A hash reference containing Infoblox::Grid::Extattr object.
 extensible_attributes    - Optional. A hash reference containing extensible attributes.
```

Examples

```
my @retrieved_objs = $session->get(
    object => 'Infoblox::DXL::Endpoint',
    name   => 'dxl_ep',
);

my @retrieved_objs = $session->get(
    object               => 'Infoblox::DXL::Endpoint',
    extensible_attributes => {'Site' => 'Santa Clara'},
);
```

Infoblox::Session->modify( )

Use this method to modify the object in the Infoblox appliance. See Infoblox::Session->modify() for parameters and return values.

Example

```
#Modify comment value
$object->comment('this is a modified comment');

#Submit modification
my $response = $session->modify($object);
```

Infoblox::Session->remove( )

Use this method to remove the object from the Infoblox appliance. See Infoblox::Session->remove() for parameters and return values.

To remove a specific object, first use `get()` or `search()` to retrieve the specific DXL endpoint object, and then submit this object for removal.

Example

```
#Get the objects
my @retrieved_objs = $session->get(
    object => 'Infoblox::DXL::Endpoint',
    name   => 'dxl_ep',
);

#Find the desired object from the retrieved list
my $desired_object = $retrieved_objs[0];

#Submit for removal
my $response = $session->remove($desired_object);
```

Infoblox::Session->search( )

Use this method to search for objects in the Infoblox appliance. See Infoblox::Session->search() for parameters and return values.

Key References

```
Apply the following attributes to search for the DXL endpoint object:

 comment                    - Optional. The DXL endpoint comment in string format (regexp).
 name                       - Optional. The DXL endpoint name in string format (regexp).
 vendor_identifier          - Optional. The DXL endpoint vendor identifier in string format
(regexp).
 outbound_member_type       - Optional. The DXL endpoint outbound member type.
 extattrs                   - Optional. A hash reference containing Infoblox::Grid::Extattr object.
 extensible_attributes      - Optional. A hash reference containing extensible attributes.
```

For more information about searching extensible attributes, see Infoblox::Grid::ExtensibleAttributeDef/Searching Extensible Attributes.

Examples

```
# search for objects
my @retrieved_objs = $session->search(
    object            => 'Infoblox::DXL::Endpoint',
    name              => 'dxl_.*',
    vendor_identifier => 'Carbon',
);

# search for objects defining 'Santa Clara' for 'Site' extensible attribute
my @retrieved_objs = $session->search(
    object                => 'Infoblox::DHCP::Range',
    extensible_attributes => {'Site' => 'Santa Clara'},
);
```

METHODS

This section describes all the methods that you can use to configure and retrieve the attribute values of a DXL endpoint.

brokers( )

Use this method to set or retrieve the list of DXL brokers for the DXL endpoint.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

The valid value is an array reference that contains Infoblox::DXL::Endpoint::Broker objects.

Returns

If you have specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get brokers value
my $brokers = $object->brokers();

#Construct Infoblox::DXL::Endpoint::Broker object
my $broker = Infoblox::DXL::Endpoint::Broker->new(
    host_name => 'dxl-br.localdomain',
);

#Modify brokers value
$object->brokers([$broker]);
```

client_certificate( )

Use this method to set the path to a client certificate file. This is a write-only attribute.

Include the specified parameter to set the attribute value.

Parameter

The valid value is a path to a certificate in string format.

Returns

If you have specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

Example

```
#Modify client_certificate value
$object->client_certificate('/path/to/cert.pem');
```

client_certificate_subject( )

Use this method to retrieve the client certificate subject. This is a read-only attrbiute.

Omit the parameter to retrieve the attribute value.

Parameter

None

Returns

The method returns the attribute value.

Example

```
#Get client_certificate_subject value
my $client_certificate_subject = $object->client_certificate_subject();
```

client_certificate_valid_from( )

Use this method to retrieve the time from which the client certificate is valid. This is a read-only attrbiute.

Omit the parameter to retrieve the attribute value.

Parameter

None

Returns

The method returns the attribute value. The number of seconds that have elapsed since January 1st, 1970 UTC.

Example

```
#Get client_certificate_valid_from value
my $client_certificate_valid_from = $object->client_certificate_valid_from();
```

client_certificate_valid_to( )

Use this method to retrieve the expiration time for the client certificate. This is a read-only attrbiute.

Omit the parameter to retrieve the attribute value.

Parameter

None

Returns

The method returns the attribute value. The number of seconds that have elapsed since January 1st, 1970 UTC.

Example

```
#Get client_certificate_valid_to value
my $client_certificate_valid_to = $object->client_certificate_valid_to();
```

comment( )

Use this method to set or retrieve the DXL endpoint comment.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

The valid value is a desired comment in string format.

Returns

If you have specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get comment value
my $comment = $object->comment();

#Modify comment value
$object->comment('modified comment');
```

disable( )

Use this method to set or retrieve the flag that indicates whether the DXL endpoint is enabled or disabled.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

Specify 'true' to disable the DXL endpoint or 'false' to enable it. The default value is 'true'.

Returns

If you have specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get disable value
my $disable = $object->disable();

#Modify disable value
$object->disable('false');
```

extattrs( )

Use this method to set or retrieve the extensible attributes associated with a DXL endpoint object.

Parameter

Valid value is a hash reference that contains the names of extensible attributes and their associated values (Infoblox::Grid::Extattr objects).

Returns

If you have specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get extattrs value
my $extattrs = $object->extattrs();

#Modify extattrs value
$object->extattrs({'Site' => $extattr1, 'Administrator' => $extattr2});
```

extensible_attributes( )

Use this method to set or retrieve the extensible attributes associated with a DXL endpoint object.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

For valid values for extensible attributes, see Infoblox::Grid::ExtensibleAttributeDef/Extensible Attribute Values.

Returns

If you have specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get extensible attributes value
my $extensible_attributes = $object->extensible_attributes();
#Modify extensible attributes
$object->extensible_attributes({'Site' => 'Santa Clara', 'Administrator' => ['Peter', 'Tom']});
```

name( )

Use this method to set or retrieve the DXL endpoint name.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

The valid value is a desired name in string format.

Returns

If you have specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get name value
my $name = $object->name();

#Modify name value
$object->name('modified_name');
```

outbound_members( )

Use this method to set or retrieve the list of DXL endpoint members that are responsible for event generation.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

The valid value is an array that contains Grid member names in string format.

Returns

If you have specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get outbound_members value
my $outbound_members = $object->outbound_members();

#Modify outbound_members value
$object->outbound_members(['member1', 'member2']);
```

outbound_member_type( )

Use this method to set or retrieve the DXL endpoint member that is responsible for event generation.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

The valid values are 'GM' and 'MEMBER'.

Returns

If you have specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get outbound_member_type value
my $outbound_member_type = $object->outbound_member_type();

#Modify outbound_member_type value
$object->outbound_member_type('MEMBER');
```

log_level( )

Use this method to set or retrieve the DXL endpoint logging level.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

The valid values are 'DEBUG', 'ERROR', 'INFO', and 'WARNING'.

Returns

If you have specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get log_level value
my $value = $object->log_level();

#Modify log_level value
$object->log_level('INFO');
```

template_instance( )

Use this method to set or retrieve the DXL endpoint template instance.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Note that you cannot change the 'parameters' value of the template instance object for a DXL endpoint.

Parameter

The valid value is an Infoblox::Notification::REST::TemplateInstance object.

Returns

If you have specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get template_instance value
my $value = $object->template_instance();

#Modify template_instance value
$object->template_instance($instance);
```

vendor_identifier( )

Use this method to set or retrieve the DXL endpoint vendor identifier.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

The valid value is a desired vendor identifier in string format.

Returns

If you have specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get vendor_identifier value
my $vendor_identifier = $object->vendor_identifier();

#Modify vendor_identifier value
$object->vendor_identifier('CarbonBlack');
```

wapi_user_name( )

Use this method to set or retrieve the user name for WAPI integration.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

The valid value is a desired user name in string format.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get wapi_user_name value
my $value = $object->wapi_user_name();

#Modify wapi_user_name value
$object->wapi_user_name('admin');
```

wapi_user_password( )

Use this method to set the user password for WAPI integration. This is a write-only attribute.

Include the specified parameter to set the attribute value.

Parameter

The valid value is a desired user password in string format.

Returns

The method returns true when the modification succeeds, and returns false when the operation fails.

Example

```
#Modify wapi_user_password value
$object->wapi_user_password('infoblox');
```

AUTHOR

Infoblox Inc. http://www.infoblox.com/

SEE ALSO

Infoblox::Session, Infoblox::Session->add(), Infoblox::Session->get(), Infoblox::Session->modify(), Infoblox::Session->remove(), Infoblox::Session->search(), Infoblox::Grid::Extattr, Infoblox::Grid::ExtensibleAttributeDef/Searching Extensible Attributes, Infoblox::Notification::REST::TemplateInstance, Infoblox::DXL::Endpoint::Broker.

COPYRIGHT

Copyright (c) 2017 Infoblox Inc.

Infoblox::DXL::Endpoint::Broker - A Data Exchange Layer endpoint broker object.

- NAME
- DESCRIPTION
- CONSTRUCTOR
- SESSION METHODS
- MODULE METHODS
  - Infoblox::DXL::Endpoint->brokers( )
- METHODS
  - address( )
  - host_name( )
  - port( )
  - unique_id( )
- AUTHOR
- SEE ALSO
- COPYRIGHT

NAME

Infoblox::DXL::Endpoint::Broker - A Data Exchange Layer (DXL) endpoint broker object.

DESCRIPTION

DXL brokers are installed on managed systems for routing messages between connected clients. The network of brokers tracks active consumers and dynamically adjusts the message routing as needed. When a client requests a service or when an update is broadcasted, brokers relay these messages.

The DXL endpoint broker is used to configure DXL broker for a particular endpoint.

CONSTRUCTOR

```
my $object = Infoblox::DXL::Endpoint::Broker->new(
    host_name => $fqdn,    #Required
    address   => $ipaddr, #Optional / Default is undefined
    port      => $uint,    #Optional / Default is 8883
    unique_id => $string, #Optional / Default is undefined
);
```

SESSION METHODS

The object does not support any session methods.

MODULE METHODS

This section describes all the functions that you can apply to a DXL Broker object.

Infoblox::DXL::Endpoint->brokers( )

Use this method to set or retrieve the list of DXL brokers for the DXL endpoint. See Infoblox::DXL::Endpoint->brokers() for parameters and return values.

Example

```
#Get brokers value
my $brokers = $object->brokers();

#Construct Infoblox::DXL::Endpoint::Broker object
my $broker = Infoblox::DXL::Endpoint::Broker->new(
    host_name => 'dxl-br.localdomain',
);

#Modify brokers value
$object->brokers([$broker]);
```

METHODS

This section describes all the methods that you can use to set or retrieve the attribute values of the object.

address( )

Use this method to set or retrieve the DXL broker IP address.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

The valid value is an IP address in string format.

Returns

If you specified a parameter, the method returns 'true' when the modification succeeds, and returns 'false' when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get address value
my $value = $object->address();

#Modify address value
$object->address('1.1.1.1');
```

host_name( )

Use this method to set or retrieve the host name for the DXL broker.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

The valid value is a desired FQDN in string format.

Returns

If you specified a parameter, the method returns 'true' when the modification succeeds, and returns 'false' when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get host_name value
my $value = $object->host_name();

#Modify host_name value
$object->host_name('dxl-broker.localdomain');
```

port( )

Use this method to set or retrieve the port number for the DXL broker communication.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

The valid value is an unsigned integer between 1 and 65535. The default value is 8883.

Returns

If you specified a parameter, the method returns 'true' when the modification succeeds, and returns 'false' when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get port value
my $value = $object->port();

#Modify port value
$object->port('1312');
```

unique_id( )

Use this method to set or retrieve the unique ID for the DXL broker.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

The valid value is a desired ID in string format.

Returns

If you specified a parameter, the method returns 'true' when the modification succeeds, and returns 'false' when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get unique_id value
my $value = $object->unique_id();

#Modify unique_id value
$object->unique_id('5d73b77f-8c4b-4ae0-b437-febd12facfd4');
```

AUTHOR

Infoblox Inc. http://www.infoblox.com/

SEE ALSO

Infoblox::DXL::Endpoint, Infoblox::DXL::Endpoint->brokers().

COPYRIGHT

Copyright (c) 2017 Infoblox Inc.

Infoblox::Grid - Manages the Grid settings.

- NAME
- DESCRIPTION
- CONSTRUCTOR
- SESSION METHODS
  - Infoblox::Session->get( )
  - Infoblox::Session->search( )
  - Infoblox::Session->modify( )
- METHODS
  - admin_access_items( )
  - allow_recursive_deletion( )
  - admin_access_remote_console( )
  - audit_log_format( )
  - alternate_resolver( )
  - consent_banner_setting( )
  - copy_audit_to_syslog( )
  - descendants_action( )
  - deny_mgm_snapshots( )
  - dscp( )
  - enable_login_banner( )
  - enable_lom( )
  - enable_gui_api_for_lan_vip( )
  - enable_member_redirect( )
  - enable_ntp( )
  - enable_recycle_bin( )
  - enable_rir_swip( )
  - enable_snmpv3_query( )
  - enable_snmpv3_traps( )
  - enable_security_access( )
  - external_syslog_server_enable( )
  - http_proxy_server_setting( )
  - informational_banner_setting( )
  - is_grid_visualization_visible( )
  - lcd_input( )
  - login_banner_text( )
  - lom_users( )
  - mgm_strict_delegate_mode( )
  - ms_default_ip_site_link( )
  - ms_enable_invalid_mac( )
  - ms_enable_dhcp_monitoring( )
  - ms_enable_dns_monitoring( )
  - ms_enable_dns_reports_sync( )
  - ms_enable_network_users( )
  - ms_enable_user_sync( )
  - ms_ldap_timeout( )
  - ms_log_destination( )
  - ms_max_connection( )
  - ms_rpc_timeout( )
  - ms_user_default_timeout( )
  - name( )
  - nat_group_list( )
  - notification_email_addr( )
  - ntp_server( )
  - ntp_access_list( )
  - ntp_service_type( )
  - ntp_authentication_key( )
  - ntp_kod( )
  - objects_changes_tracking_setting( )
  - password_chars_to_change( )
  - password_expire_days( )
  - password_expire_enable( )
  - password_force_reset( )
  - password_min_length( )
  - password_num_lower_char( )
  - password_num_numeric_char( )
  - password_num_symbol_char( )
  - password_num_upper_char( )
  - password_reminder_days( )
  - prefer_resolver( )
  - query_comm_string ( )
  - remote_console_access( )
  - restart_banner_setting( )

- restart_status( )
- rpz_hit_rate_interval( )
- rpz_hit_rate_min_query( )
- rpz_hit_rate_max_query( )
- search_domains( )
- secret( )
- service_status( )
- support_access( )
- support_access_info( )
- scheduled_backup( )
- session_timeout( )
- snmp_admin( )
- snmpv3_query_users( )
- syslog_backup_servers( )
- syslog_server( )
- syslog_size( )
- threshold_traps( )
- token_usage_delay( )
- trap_comm_string( )
- trap_notifications( )
- trap_receiver( )
- updates_download_member_config( )
- vpn_port( )

- SAMPLE CODE
- AUTHOR
- SEE ALSO
- COPYRIGHT

NAME

Infoblox::Grid - Manages the Grid settings.

DESCRIPTION

This class manages the database that is distributed among all grid members. The grid object is not intended to be instantiated, it should only be used to modify information.

A grid is a group of two or more Infoblox appliances that share sections of a common, distributed, built-in database and also, are connected together to provide a single point of administration and service configuration in a secure, highly available environment.

CONSTRUCTOR

The Infoblox::Grid object is automatically generated upon the successful installation of the Infoblox appliance. It does not require manual construction.

SESSION METHODS

This section describes all the methods in Infoblox::Session that you can apply to a grid object.

Infoblox::Session->get( )

Use this method to retrieve the existing objects from the Infoblox appliance. See Infoblox::Session->get() for parameters and return values.

Example

```
my $grid = $session->get(
    object => "Infoblox::Grid");
```

Infoblox::Session->search( )

Use this method to search for the existing objects from the Infoblox appliance. See Infoblox::Session->search() for parameters and return values.

Example

```
my $grid = $session->search(
    object => "Infoblox::Grid");
```

Infoblox::Session->modify( )

Use this method to modify objects in the Infoblox appliance. See Infoblox::Session->modify() for parameters and return values.

If you have applied an access control list to an object, you must restart services after you update the access control list for the changes to take effect. See Infoblox::Session->restart() method.

Example

```
# Use method to modify the name
$grid->name("Infoblox2");
# Submit modification
my $response = $session->modify( $grid );
```

METHODS

This section describes all the methods that you can use to configure and retrieve the attribute values of a grid.

admin_access_items( )

Use this method to set or retrieve the security access items.

Parameter

Valid value is an Infoblox::Grid::NamedACL object or an array reference that contains IPv4/IPv6 addresses or networks.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get admin_access_items
my $admin_access_items = $grid->admin_access_items();
#Modify admin_access_items
$grid->admin_access_items(["10.10.10.1", "10.10.2.0/24", "2001::1"]);
#Use Infoblox::Grid::NamedACL
$grid->admin_access_items($nacl);
```

allow_recursive_deletion( )

Use this method to set or retrieve the allow_recursive_deletion property.

Parameter

Valid values are "NOBODY", "SUPERUSERS" and "ALL". Default value is "ALL".

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get allow_recursive_deletion
my $allow_recursive_deletion = $grid->allow_recursive_deletion();
#Modify allow_recursive_deletion
$grid->allow_recursive_deletion("SUPERUSERS");
```

admin_access_remote_console( )

Use this method to set or retrieve the flag that enables or disables access control for remote console access.

Parameter

Valid values are "true" or "false".

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get admin_access_remote_console
my $admin_access_remote_console = $obj->admin_access_remote_console();
```

```
#Modify admin_access_remote_console
$obj->admin_access_remote_console("false");
```

audit_log_format( )

Use this method to set or retrieve the audit log format.

Parameter

Valid values are "detailed" or "brief". Default value is "detailed".

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get audit_log_format
my $audit_log_format = $grid->audit_log_format();
#Modify audit_log_format
$grid->audit_log_format("brief");
```

alternate_resolver( )

Use this method to set or retrieve the alternate resolver.

Parameter

The valid values are Infoblox::DNS::Member object or Infoblox::DNS::Nameserver object.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

Example

```
#Get alternate_resolver
my $alternate = $grid->alternate_resolver();
#Modify alternate_resolver
$grid->alternate_resolver("4.4.4.4");
```

consent_banner_setting( )

Use this method to set or retrieve the Grid consent banner settings.

Parameter

An Infoblox::Grid::ConsentBannerSetting object.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

Example

```
#Enable the consent banner
my $banner = Infoblox::Grid::ConsentBannerSetting->new(enable => 'true', message => 'Consent
message');
$grid->consent_banner_setting($banner);
```

copy_audit_to_syslog( )

Use this method to set or retrieve the flag that enables the appliance to copy audit log messages to syslog.

Parameter

Valid values are "true" or "false".

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get copy_audit_to_syslog
my $copy_audit_to_syslog = $obj->copy_audit_to_syslog();
#Modify copy_audit_to_syslog
$obj->copy_audit_to_syslog("true");
```

descendants_action( )

Use this method to set or retrieve the default actions for extensbile attributes that exist on descendants.

Parameter

Valid value is an Infoblox::Grid::ExtensibleAttributeDef::Descendants object.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get attribute value
my $descendants_action = $grid->descendants_action();
#Modify attribute value
$grid->descendants_action($descendants);
```

deny_mgm_snapshots( )

Use this method to set or retrieve the flag that controls whether the subgrid should send snapshots to MGM or not.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Paramter

Specify 'true' to allow sending snapshots to MGM by subgrid and 'false' to forbid it.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get deny_mgm_snapshots value
my $value = $object->deny_mgm_snapshots();

#Modify deny_mgm_snapshots value
$object->deny_mgm_snapshots('true');
```

dscp( )

Use this method to set or retrieve the DSCP value.

Parameter

Valid values are integers between 0 and 63 inclusive. The default value is 0.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get dscp
my $dscp = $grid->dscp();
#Modify dscp
$grid->dscp(12);
```

enable_login_banner( )

Use this method to enable or disable the login banner.

Parameter

Specify "true" to enable the login banner or "false" to disable it. The default value is "true".

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get enable_login_banner
my $enable_login_banner = $grid->enable_login_banner();
#Modify enable_login_banner
$grid->enable_login_banner("false");
```

enable_lom( )

Use this method to set or retrieve the flag that controls whether the LOM functionality is enabled.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

Specify "true" to set the enable_lom flag or "false" to deactivate/unset it. The default value is "false".

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get enable_lom from the grid
my $lom = $grid->enable_lom();
#Modify enable_lom
$grid->enable_lom("true");
```

enable_gui_api_for_lan_vip( )

Use this method to enable or disable GUI and API access on the LAN/VIP port, in addition to the MGMT port when it is configured.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Paramter

Specify 'true' to enable GUI and API access on the LAN/VIP port or 'false' to disable it.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get enable_gui_api_for_lan_vip
my $enable_gui_api_for_lan_vip = $grid->enable_gui_api_for_lan_vip();

#Modify enable_gui_api_for_lan_vip
$grid->enable_gui_api_for_lan_vip('true');
```

enable_member_redirect( )

Use this method to set or retrieve the flag that specifies if GUI redirection is enabled for members.

Parameter

Valid values are "true" or "false". Default value is "true".

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get enable_member_redirect
my $enable_member_redirect = $obj->enable_member_redirect();
#Modify enable_member_redirect
```

```
$obj->enable_member_redirect("true");
```

enable_ntp( )

Use this method to enable/disable NTP (Network Time Protocol).

Parameter

Specify "true" to enable NTP or "false" to disable it. Default value is "false".

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

Example

```
#Get ntp
my $ntp = $grid->enable_ntp();
#Modify NTP
$grid->enable_ntp ("true");
```

enable_recycle_bin( )

Use this method to enable/disable the recycle bin.

When the recycle bin is enabled, deleted DNS data (Zone,View) and DHCP data (Network,Network container,Shared network,DHCP range,Fixed address,DHCP MAC filter) is moved to the recycle bin and can be restored later.

Parameter

Specify "true" to enable the recycle bin or "false" to disable it. Default value is "true".

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

Example

```
#Get enable_recycle_bin
my $recycle_bin = $grid->enable_recycle_bin();
#Modify enable_recycle_bin
$grid->enable_recycle_bin ("false");
```

enable_rir_swip( )

Use this method to enable or diasble support for RIR/SWIP.

The following specific extensible attribute definitions with the 'RIPE' namespace will be added when support for RIR/SWIP is enabled for the first time:

```
 +-------------------------------------------------------------------------------------------------------------------------------+
 | RIPE Network Name      - The name of the network that contains the RIR assignment, in string format.
|
 |                          Required for Infoblox::DHCP::Network, Infoblox::DHCP::IPv6Network,
Infoblox::DHCP::NetworkContainer,       |
 |                          Infoblox::DHCP::IPv6NetworkContainer, Infoblox::DHCP::NetworkTemplate,
Infoblox::DHCP::IPv6NetworkTemplate  |
 |                          objects if rir_organization member is defined. Otherwise this is optional RIR
attribute for these objects.  |
 | RIPE Description       - The short description related to the object in string format.
|
 |                          Required for Infoblox::DHCP::Network, Infoblox::DHCP::IPv6Network,
Infoblox::DHCP::NetworkContainer,       |
 |                          Infoblox::DHCP::IPv6NetworkContainer, Infoblox::DHCP::NetworkTemplate,
Infoblox::DHCP::IPv6NetworkTemplate  |
 |                          objects if rir_organization member is defined. Otherwise this is optional RIR
attribute for these objects.  |
 |                          Optional for Infoblox::Grid::RIR::Organization object.
|
 | RIPE Country           - The country name with the two-letter ISO 3166 country code in string format,
e.g. 'Afghanistan (AF)'.      |
 |                          Valid values you can see in the next table.
|
 |                          Required for Infoblox::DHCP::Network, Infoblox::DHCP::IPv6Network,
Infoblox::DHCP::NetworkContainer,       |
 |                          Infoblox::DHCP::IPv6NetworkContainer, Infoblox::DHCP::NetworkTemplate,
Infoblox::DHCP::IPv6NetworkTemplate  |
 |                          objects if rir_organization member is defined. Otherwise this is optional RIR
attribute for these objects.  |
 |                          Required for Infoblox::Grid::RIR::Organization object.
|
 | RIPE Admin Contact     - The admin contact in string format.
|
```

| | Required for Infoblox::DHCP::Network, Infoblox::DHCP::IPv6Network, Infoblox::DHCP::NetworkContainer, | | Infoblox::DHCP::IPv6NetworkContainer, Infoblox::DHCP::NetworkTemplate, Infoblox::DHCP::IPv6NetworkTemplate | | objects if rir_organization member is defined. Otherwise this is optional RIR attribute for these objects. | | Required for Infoblox::Grid::RIR::Organization object.

| RIPE Technical Contact - The technical contact in string format.

| | Required for Infoblox::DHCP::Network, Infoblox::DHCP::IPv6Network, Infoblox::DHCP::NetworkContainer, | | Infoblox::DHCP::IPv6NetworkContainer, Infoblox::DHCP::NetworkTemplate, Infoblox::DHCP::IPv6NetworkTemplate | | objects if rir_organization member is defined. Otherwise this is optional RIR attribute for these objects. | | Required for Infoblox::Grid::RIR::Organization object.

| RIPE IPv4 Status - The status of the address range represented by inetnum object that can be 'ALLOCATED PA', | | 'ALLOCATED PI', 'ALLOCATED UNSPECIFIED', 'LIR-PARTITIONED PA', 'LIR-PARTITIONED PI', 'SUB-ALLOCATED PA', | | 'ASSIGNED PA', 'ASSIGNED PI', 'ASSIGNED ANYCAST', 'EARLY-REGISTRATION' or 'NOT-SET'. | | Required for Infoblox::DHCP::Network, Infoblox::DHCP::NetworkContainer and Infoblox::DHCP::NetworkTemplate | | objects if rir_organization member is defined. Otherwise this is optional RIR attribute for these objects. |

| RIPE IPv6 Status - The status of the address range represented by inet6num object that can be 'ALLOCATED-BY-RIR', | | 'ALLOCATED-BY-LIR', 'ASSIGNED', 'ASSIGNED ANYCAST' or 'ASSIGNED PI'.

| | Required for Infoblox::DHCP::IPv6Network, Infoblox::DHCP::IPv6NetworkContainer and | | Infoblox::DHCP::IPv6NetworkTemplate objects if rir_organization member is defined. Otherwise this is | | optional RIR attribute for these objects.

| RIPE Remarks - Some freeform remarks in string format.

| | Required for Infoblox::DHCP::Network, Infoblox::DHCP::IPv6Network, Infoblox::DHCP::NetworkContainer, | | Infoblox::DHCP::IPv6NetworkContainer, Infoblox::DHCP::NetworkTemplate, Infoblox::DHCP::IPv6NetworkTemplate | | objects if rir_organization member is defined. Otherwise this is optional RIR attribute for these objects. | | Optional for Infoblox::Grid::RIR::Organization object.

| RIPE Notify - The Email address to which notifications must be sent for any updates in string format. | | Optional for Infoblox::DHCP::Network, Infoblox::DHCP::IPv6Network, Infoblox::DHCP::NetworkContainer, | | Infoblox::DHCP::IPv6NetworkContainer, Infoblox::DHCP::NetworkTemplate, Infoblox::DHCP::IPv6NetworkTemplate, | | Infoblox::Grid::RIR::Organization objects.

| RIPE Lower Level Maintainer

| | - The identifier of a registerd maintainer object used for hierarchical authorization, in string format. | | Optional for Infoblox::DHCP::Network, Infoblox::DHCP::IPv6Network, Infoblox::DHCP::NetworkContainer, | | Infoblox::DHCP::IPv6NetworkContainer, Infoblox::DHCP::NetworkTemplate, Infoblox::DHCP::IPv6NetworkTemplate | | objects.

| RIPE Reverse Domain Maintainer

| | - The identifier of a registered maintainer object used for reverse domain authorization, in string format. | | Optional for Infoblox::DHCP::Network, Infoblox::DHCP::IPv6Network, Infoblox::DHCP::NetworkContainer, | | Infoblox::DHCP::IPv6NetworkContainer, Infoblox::DHCP::NetworkTemplate, Infoblox::DHCP::IPv6NetworkTemplate | | objects.

| RIPE Routes Maintainer - The identifier of a registered maintainer object that contols the authorization of the creation of route | | and route6 objects, in string format.

| | Optional for Infoblox::DHCP::Network, Infoblox::DHCP::IPv6Network, Infoblox::DHCP::NetworkContainer, | | Infoblox::DHCP::IPv6NetworkContainer, Infoblox::DHCP::NetworkTemplate, Infoblox::DHCP::IPv6NetworkTemplate | | objects.

| RIPE Computer Security Incident Response Team

| | - The point to an irt object representing a Computer Security Incident Response Team (CSIRT) that handles | | security incidents for the address space specified by the inetnum or inet6num object in string format. | | Optional for Infoblox::DHCP::Network, Infoblox::DHCP::IPv6Network, Infoblox::DHCP::NetworkContainer, | | Infoblox::DHCP::IPv6NetworkContainer, Infoblox::DHCP::NetworkTemplate, Infoblox::DHCP::IPv6NetworkTemplate | | objects.

RIPE Registry Source	The registry where the object is registered. This can be 'RIPE' or "TEST". Required for Infoblox::DHCP::Network, Infoblox::DHCP::IPv6Network, Infoblox::DHCP::NetworkContainer, Infoblox::DHCP::IPv6NetworkContainer, Infoblox::DHCP::NetworkTemplate, Infoblox::DHCP::IPv6NetworkTemplate objects if rir_organization member is defined. Otherwise this is optional RIR attribute for these objects. Required for Infoblox::Grid::RIR::Organization object.
RIPE Organization Type	The organization type. This can be 'IANA','RIR','NIR','LIR','WHITEPAGES','DIRECT ASSIGNMENT', or 'OTHER'. Optional for Infoblox::Grid::RIR::Organization object.
RIPE Address	The organization address in string format. Optional for Infoblox::Grid::RIR::Organization object.
RIPE Phone Number	The organization phone number in string format. Optional for Infoblox::Grid::RIR::Organization object.
RIPE Fax Number	The organization fax number in string format. Optional for Infoblox::Grid::RIR::Organization object.
RIPE Email	The organization email address in string format. Required for Infoblox::Grid::RIR::Organization object.
RIPE Abuse Mailbox	The email address to which abuse complaints should be sent in string format. Optional for Infoblox::Grid::RIR::Organization object.
RIPE Reference Notify	The email address to be notified when a reference to the organization object is added or removed, in string format. Optional for Infoblox::Grid::RIR::Organization object.

The following values are valid for 'RIPE Country' attribute:

Code	Country	Value
AF	Afghanistan	Afghanistan (AF)
AX	Aland Islands	Aland Islands (AX)
AL	Albania	Albania (AL)
DZ	Algeria	Algeria (DZ)
AS	American Samoa	American Samoa (AS)
AD	Andorra	Andorra (AD)
AO	Angola	Angola (AO)
AI	Anguilla	Anguilla (AI)
AQ	Antarctica	Antarctica (AQ)
AG	Antigua and Barbuda	Antigua and Barbuda (AG)
AR	Argentina	Argentina (AR)
AM	Armenia	Armenia (AM)
AW	Aruba	Aruba (AW)
AU	Australia	Australia (AU)
AT	Austria	Austria (AT)
AZ	Azerbaijan	Azerbaijan (AZ)
BS	Bahamas	Bahamas (BS)
BH	Bahrain	Bahrain (BH)
BD	Bangladesh	Bangladesh (BD)
BB	Barbados	Barbados (BB)
BY	Belarus	Belarus (BY)
BE	Belgium	Belgium (BE)

BZ	Belize	Belize (BZ)
BJ	Benin	Benin (BJ)
BM	Bermuda	Bermuda (BM)
BT	Bhutan	Bhutan (BT)
BO	Bolivia, Plurinational State of	Bolivia, Plurinational State of (BO)
BQ	Bonaire, Sint Eustatius and Saba	Bonaire, Sint Eustatius and Saba (BQ)
BA	Bosnia and Herzegovina	Bosnia and Herzegovina (BA)
BW	Botswana	Botswana (BW)
BV	Bouvet Island	Bouvet Island (BV)
BR	Brazil	Brazil (BR)
IO	British Indian Ocean Territory	British Indian Ocean Territory (IO)
BN	Brunei Darussalam	Brunei Darussalam (BN)
BG	Bulgaria	Bulgaria (BG)
BF	Burkina Faso	Burkina Faso (BF)
BI	Burundi	Burundi (BI)
KH	Cambodia	Cambodia (KH)
CM	Cameroon	Cameroon (CM)
CA	Canada	Canada (CA)
CV	Cape Verde	Cape Verde (CV)
KY	Cayman Islands	Cayman Islands (KY)
CF	Central African Republic	Central African Republic (CF)
TD	Chad	Chad (TD)
CL	Chile	Chile (CL)
CN	China	China (CN)
CX	Christmas Island	Christmas Island (CX)
CC	Cocos (Keeling) Islands	Cocos (Keeling) Islands (CC)
CO	Colombia	Colombia (CO)
KM	Comoros	Comoros (KM)
CG	Congo	Congo (CG)
CD	Congo, The Democratic Republic of the	Congo, The Democratic Republic of the (CD)
CK	Cook Islands	Cook Islands (CK)
CR	Costa Rica	Costa Rica (CR)
CI	Côte d'Ivoire	Côte d'Ivoire (CI)
HR	Croatia	Croatia (HR)
CU	Cuba	Cuba (CU)
CW	Curaçao	Curaçao (CW)
CY	Cyprus	Cyprus (CY)
CZ	Czech Republic	Czech Republic (CZ)
DK	Denmark	Denmark (DK)
DJ	Djibouti	Djibouti (DJ)
DM	Dominica	Dominica (DM)
DO	Dominican Republic	Dominican Republic (DO)
EC	Ecuador	Ecuador (EC)
EG	Egypt	Egypt (EG)
SV	El Salvador	El Salvador (SV)
GQ	Equatorial Guinea	Equatorial Guinea (GQ)
ER	Eritrea	Eritrea (ER)
EE	Estonia	Estonia (EE)
ET	Ethiopia	Ethiopia (ET)

FK	Falkland Islands (Malvinas)	Falkland Islands (Malvinas) (FK)
FO	Faroe Islands	Faroe Islands (FO)
FJ	Fiji	Fiji (FJ)
FI	Finland	Finland (FI)
FR	France	France (FR)
GF	French Guiana	French Guiana (GF)
PF	French Polynesia	French Polynesia (PF)
TF	French Southern Territories	French Southern Territories (TF)
GA	Gabon	Gabon (GA)
GM	Gambia	Gambia (GM)
GE	Georgia	Georgia (GE)
DE	Germany	Germany (DE)
GH	Ghana	Ghana (GH)
GI	Gibraltar	Gibraltar (GI)
GR	Greece	Greece (GR)
GL	Greenland	Greenland (GL)
GD	Grenada	Grenada (GD)
GP	Guadeloupe	Guadeloupe (GP)
GU	Guam	Guam (GU)
GT	Guatemala	Guatemala (GT)
GG	Guernsey	Guernsey (GG)
GN	Guinea	Guinea (GN)
GW	Guinea-Bissau	Guinea-Bissau (GW)
GY	Guyana	Guyana (GY)
HT	Haiti	Haiti (HT)
HM	Heard Island and McDonald Islands	Heard Island and McDonald Islands (HM)
VA	Holy See (Vatican City State)	Holy See (Vatican City State) (VA)
HN	Honduras	Honduras (HN)
HK	Hong Kong	Hong Kong (HK)
HU	Hungary	Hungary (HU)
IS	Iceland	Iceland (IS)
IN	India	India (IN)
ID	Indonesia	Indonesia (ID)
IR	Iran, Islamic Republic of	Iran, Islamic Republic of (IR)
IQ	Iraq	Iraq (IQ)
IE	Ireland	Ireland (IE)
IM	Isle of Man	Isle of Man (IM)
IL	Israel	Israel (IL)
IT	Italy	Italy (IT)
JM	Jamaica	Jamaica (JM)
JP	Japan	Japan (JP)
JE	Jersey	Jersey (JE)
JO	Jordan	Jordan (JO)
KZ	Kazakhstan	Kazakhstan (KZ)
KE	Kenya	Kenya (KE)
KI	Kiribati	Kiribati (KI)
KP	Korea, Democratic People's Republic of	Korea, Democratic People's Republic of (KP)
KR	Korea, Republic of	Korea, Republic of (KR)

KW	Kuwait	Kuwait (KW)
KG	Kyrgyzstan	Kyrgyzstan (KG)
LA	Lao People's Democratic Republic	Lao People's Democratic Republic (LA)
LV	Latvia	Latvia (LV)
LB	Lebanon	Lebanon (LB)
LS	Lesotho	Lesotho (LS)
LR	Liberia	Liberia (LR)
LY	Libya	Libya (LY)
LI	Liechtenstein	Liechtenstein (LI)
LT	Lithuania	Lithuania (LT)
LU	Luxembourg	Luxembourg (LU)
MO	Macao	Macao (MO)
MK	Macedonia, The Former Yugoslav Republic of (MK)	Macedonia, The Former Yugoslav Republic of (MK)
MG	Madagascar	Madagascar (MG)
MW	Malawi	Malawi (MW)
MY	Malaysia	Malaysia (MY)
MV	Maldives	Maldives (MV)
ML	Mali	Mali (ML)
MT	Malta	Malta (MT)
MH	Marshall Islands	Marshall Islands (MH)
MQ	Martinique	Martinique (MQ)
MR	Mauritania	Mauritania (MR)
MU	Mauritius	Mauritius (MU)
YT	Mayotte	Mayotte (YT)
MX	Mexico	Mexico (MX)
FM	Micronesia, Federated States of	Micronesia, Federated States of (FM)
MD	Moldova, Republic of	Moldova, Republic of (MD)
MC	Monaco	Monaco (MC)
MN	Mongolia	Mongolia (MN)
ME	Montenegro	Montenegro (ME)
MS	Montserrat	Montserrat (MS)
MA	Morocco	Morocco (MA)
MZ	Mozambique	Mozambique (MZ)
MM	Myanmar	Myanmar (MM)
NA	Namibia	Namibia (NA)
NR	Nauru	Nauru (NR)
NP	Nepal	Nepal (NP)
NL	Netherlands	Netherlands (NL)
NC	New Caledonia	New Caledonia (NC)
NZ	New Zealand	New Zealand (NZ)
NI	Nicaragua	Nicaragua (NI)
NE	Niger	Niger (NE)
NG	Nigeria	Nigeria (NG)
NU	Niue	Niue (NU)
NF	Norfolk Island	Norfolk Island (NF)
MP	Northern Mariana Islands	Northern Mariana Islands (MP)
NO	Norway	Norway (NO)
OM	Oman	Oman (OM)
PK	Pakistan	Pakistan (PK)

PW	Palau	Palau (PW)
PS	Palestinian Territory, Occupied	Palestinian Territory, Occupied (PS)
PA	Panama	Panama (PA)
PG	Papua New Guinea	Papua New Guinea (PG)
PY	Paraguay	Paraguay (PY)
PE	Peru	Peru (PE)
PH	Philippines	Philippines (PH)
PN	Pitcairn	Pitcairn (PN)
PL	Poland	Poland (PL)
PT	Portugal	Portugal (PT)
PR	Puerto Rico	Puerto Rico (PR)
QA	Qatar	Qatar (QA)
RE	Réunion	Réunion (RE)
RO	Romania	Romania (RO)
RU	Russian Federation	Russian Federation (RU)
RW	Rwanda	Rwanda (RW)
BL	Saint Barthélemy	Saint Barthélemy (BL)
SH	Saint Helena, Ascension and Tristan da Cunha	Saint Helena, Ascension and Tristan da Cunha (SH)
KN	Saint Kitts and Nevis	Saint Kitts and Nevis (KN)
LC	Saint Lucia	Saint Lucia (LC)
MF	Saint Martin (French part)	Saint Martin (French part) (MF)
PM	Saint Pierre and Miquelon	Saint Pierre and Miquelon (PM)
VC	Saint Vincent and the Grenadines	Saint Vincent and the Grenadines (VC)
WS	Samoa	Samoa (WS)
SM	San Marino	San Marino (SM)
ST	Sao Tome and Principe	Sao Tome and Principe (ST)
SA	Saudi Arabia	Saudi Arabia (SA)
SN	Senegal	Senegal (SN)
RS	Serbia	Serbia (RS)
SC	Seychelles	Seychelles (SC)
SL	Sierra Leone	Sierra Leone (SL)
SG	Singapore	Singapore (SG)
SX	Sint Maarten (Dutch part)	Sint Maarten (Dutch part) (SX)
SK	Slovakia	Slovakia (SK)
SI	Slovenia	Slovenia (SI)
SB	Solomon Islands	Solomon Islands (SB)
SO	Somalia	Somalia (SO)
ZA	South Africa	South Africa (ZA)
GS	South Georgia and the South Sandwich Islands	South Georgia and the South Sandwich Islands (GS)
SS	South Sudan	South Sudan (SS)
ES	Spain	Spain (ES)
LK	Sri Lanka	Sri Lanka (LK)
SD	Sudan	Sudan (SD)
SR	Suriname	Suriname (SR)
SJ	Svalbard and Jan Mayen	Svalbard and Jan Mayen (SJ)
SZ	Swaziland	Swaziland (SZ)
SE	Sweden	Sweden (SE)
CH	Switzerland	Switzerland (CH)

```
|  SY  | Syrian Arab Republic                                | Syrian Arab Republic (SY)
|
|  TW  | Taiwan, Province of China                           | Taiwan, Province of China (TW)
|
|  TJ  | Tajikistan                                          | Tajikistan (TJ)
|
|  TZ  | Tanzania, United Republic of                        | Tanzania, United Republic of (TZ)
|
|  TH  | Thailand                                            | Thailand (TH)
|
|  TL  | Timor-Leste                                         | Timor-Leste (TL)
|
|  TG  | Togo                                                | Togo (TG)
|
|  TK  | Tokelau                                             | Tokelau (TK)
|
|  TO  | Tonga                                               | Tonga (TO)
|
|  TT  | Trinidad and Tobago                                 | Trinidad and Tobago (TT)
|
|  TN  | Tunisia                                             | Tunisia (TN)
|
|  TR  | Turkey                                              | Turkey (TR)
|
|  TM  | Turkmenistan                                        | Turkmenistan (TM)
|
|  TC  | Turks and Caicos Islands                            | Turks and Caicos Islands (TC)
|
|  TV  | Tuvalu                                              | Tuvalu (TV)
|
|  UG  | Uganda                                              | Uganda (UG)
|
|  UA  | Ukraine                                             | Ukraine (UA)
|
|  AE  | United Arab Emirates                                | United Arab Emirates (AE)
|
|  GB  | United Kingdom                                      | United Kingdom (GB)
|
|  US  | United States                                       | United States (US)
|
|  UM  | United States Minor Outlying Islands                | United States Minor Outlying Islands (UM)
|
|  UY  | Uruguay                                             | Uruguay (UY)
|
|  UZ  | Uzbekistan                                          | Uzbekistan (UZ)
|
|  VU  | Vanuatu                                             | Vanuatu (VU)
|
|  VE  | Venezuela, Bolivarian Republic of                   | Venezuela, Bolivarian Republic of (VE)
|
|  VN  | Viet Nam                                            | Viet Nam (VN)
|
|  VG  | Virgin Islands, British                             | Virgin Islands, British (VG)
|
|  VI  | Virgin Islands, U.S.                                | Virgin Islands, U.S. (VI)
|
|  WF  | Wallis and Futuna                                   | Wallis and Futuna (WF)
|
|  EH  | Western Sahara                                      | Western Sahara (EH)
|
|  YE  | Yemen                                               | Yemen (YE)
|
|  ZM  | Zambia                                              | Zambia (ZM)
|
|  ZW  | Zimbabwe                                            | Zimbabwe (ZW)
|
 +------+-----------------------------------------------------+-------------------------------------------
---------+
```

Parameter

Specify "true" to enable the RIR SWIP feature or "false" to disable it. Default value is "false".

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get enable_rir_swip
my $enable_rir_swip = $grid->enable_rir_swip();
#Modify enable_rir_swip
$grid->enable_rir_swip("true");
```

enable_snmpv3_query( )

Use this method to enable or disable SNMPv3 queries.

Parameter

Specify "true" to enable SNMPv3 queries or "false" to disable it.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

Example

```
#Get enable_snmpv3_query
my $enable_snmpv3_query = $grid->enable_snmpv3_query();
#Modify enable_snmpv3_query
$grid->enable_snmpv3_query("false");
```

enable_snmpv3_traps( )

Use this method to enable or disable SNMPv3 traps.

Parameter

Specify "true" to enable SNMPv3 traps or "false" to disable it. Default value is "false".

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

Example

```
#Get enable_snmpv3_traps
my $enable_snmpv3_traps = $grid->enable_snmpv3_traps();
#Modify enable_snmpv3_traps
$grid->enable_snmpv3_traps("true");
```

enable_security_access( )

Use this method to enable or disable the security access.

Parameter

Specify "true" to enable the security access or "false" to disable it. Default value is "false".

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get enable_security_access
my $enable_security_access = $grid->enable_security_access();
#Modify enable_security_access
$grid->enable_security_access("true");
```

external_syslog_server_enable( )

Use this method to set or retrieve the flag that indicates whether external syslog servers are enabled.

The external_syslog_server_enable attribute can be specified explicitly. It is also set implicitly when syslog_server is set to a defined value.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

Specify 'true' to enable external syslog servers and 'false' to disable them.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get external_syslog_server_enable value
my $value = $object->external_syslog_server_enable();

#Modify external_syslog_server_enable value
$object->external_syslog_server_enable('true');
```

http_proxy_server_setting( )

Use this method to set or retreive the Grid HTTP proxy server settings.

Parameter

The valid value is an Infoblox::Grid::HTTPProxyServerSetting object.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

Example

```
#Get http_proxy_server_setting value
my $http_proxy_server_setting = $grid->http_proxy_server_setting();

#Modify http_proxy_server_setting value
$grid->http_proxy_server_setting($http_proxy_setting);
```

informational_banner_setting( )

Use this method to set or retrieve the Grid informational banner settings.

Parameter

An Infoblox::Grid::InformationalBannerSetting object.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

Example

```
#Enable the informational banner
my $banner = Infoblox::Grid::InformationalBannerSetting->new(enable => 'true', message =>
'Informational message', color = > 'GREEN');
$grid->informational_banner_setting($banner);
```

is_grid_visualization_visible( )

Use this method to enable or disable graphical visualization of the Grid. The default value is 'false'.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

Specify 'true' to enable graphical visualization of the Grid or 'false' to disable it.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get is_grid_visualization_visible
my $is_grid_visualization_visible = $grid->is_grid_visualization_visible();

#Modify is_grid_visualization_visible
$grid->is_grid_visualization_visible('true');
```

lcd_input( )

Use this method to enable/disable LCD (Liquid Crystal Display) input. Default value is "true".

Parameter

Specify "true" to enable LCD input or "false" to disable it.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

Example

```
#Get lcd_input
```

```
my $lcd = $grid->lcd_input();
#Modify lcd_input
$grid->lcd_input("false");
```

login_banner_text( )

Use this method to set or retrieve the login banner text.

Parameter

The login banner text.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get login_banner_text
my $login_banner_text = $grid->login_banner_text();
#Modify login_banner_text
$grid->login_banner_text('Your login banner text');
```

lom_users( )

Use this method to set or retrieve the list of users for the LOM functionality.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

Valid value is an array reference that contains Infoblox::Grid::LOM::User objects.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get lom_users from the grid
my $users = $grid->lom_users();
#Modify the lom users
$grid->lom_users([$u1, $u2]);
```

mgm_strict_delegate_mode( )

Use this method to enable/disable strict delegate mode for a grid managed by the Master Grid.

Parameter

Specify "true" to enable strict delegate mode or "false" to disable it. Default value is "false".

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

Example

```
#Get mgm_strict_delegate_mode
my $delegated = $grid->mgm_strict_delegate_mode();
#Modify mgm_strict_delegate_mode
$grid->mgm_strict_delegate_mode ("true");
```

ms_default_ip_site_link( )

Use this method set or retrieve the default IP site link for sites created from NIOS for all Microsoft servers.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

Valid value is a string with the default IP site link name.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get ms_default_ip_site_link value
my $default_ip_site_link = $grid->ms_default_ip_site_link();
#Modify ms_default_ip_site_link value
$grid->ms_default_ip_site_link('Site1');
```

ms_enable_invalid_mac( )

Use this method to allow invalid MAC address synchronization for DHCP leases and fixed addresses.

Parameter

Specify "true" to allow invalid MAC address synchronization or "false" to forbid it. Default value is "false".

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get ms_enable_invalid_mac value
my $enable_invalid_mac = $grid->ms_enable_invalid_mac();
#Modify ms_enable_invalid_mac value
$grid->ms_enable_invalid_mac("true");
```

ms_enable_dhcp_monitoring( )

Use this method to enable or disable the monitoring and control of DHCP service on all Microsoft servers in the Grid.

Parameter

Specify "true" to enable monitoring and control of DHCP service on all Microsoft servers in the Grid and "false" to forbid it. Default value is "false".

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get ms_enable_dhcp_monitoring value
my $enable_dhcp_monitoring = $grid->ms_enable_dhcp_monitoring();
#Modify ms_enable_dhcp_monitoring value
$grid->ms_enable_dhcp_monitoring("true");
```

ms_enable_dns_monitoring( )

Use this method to enable or disable the monitoring and control of DNS service on all Microsoft servers in the Grid.

Parameter

Specify "true" to enable monitoring and control of DNS service on all Microsoft servers in the Grid and "false" to forbid it. Default value is "false".

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get ms_enable_dns_monitoring value
my $enable_dns_monitoring = $grid->ms_enable_dns_monitoring();
#Modify ms_enable_dns_monitoring value
$grid->ms_enable_dns_monitoring("true");
```

ms_enable_dns_reports_sync( )

Use this method to enable or disable the synchronization of DNS reporting data from all Microsoft servers in the Grid.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

Specify "true" to enable the synchronization of DNS reporting data from all Microsoft servers in the Grid. Specify "false" to disable the synchronization of DNS reporting data from all Microsoft servers in the Grid. Default value is "true".

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get ms_enable_dns_reports_sync
my $enable_dns_reports_sync = $grid->ms_enable_dns_reports_sync();
#Modify ms_enable_dns_reports_sync
$grid->ms_enable_dns_reports_sync("true");
```

ms_enable_network_users( )

Use this method to enable or disable Network Users creation.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

Specify "true" to enable network users creation for all Microsoft servers in the Grid and "false" to forbid it. Default value is "false".

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get ms_enable_network_users value
my $enable_network_users = $grid->ms_enable_network_users();
#Modify ms_enable_network_users value
$grid->ms_enable_network_users('true');
```

ms_enable_user_sync( )

Use this method to enable or disable Active Directory users synchronization for all Microsoft servers in the Grid.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

Specify "true" to enable Active Directory users synchronization for all Microsoft servers in the Grid and "false" to forbid it. Default value is "false".

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get ms_enable_user_sync value
my $enable_user_sync = $grid->ms_enable_user_sync();
#Modify ms_enable_user_sync value
$grid->ms_enable_user_sync('true');
```

ms_ldap_timeout( )

Use this method to set or retrieve an LDAP connection timeout interval in seconds for all Microsoft servers.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

Valid value is an unsigned integer. Default value is 10.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get ms_ldap_timeout value
my $ldap_timeout = $grid->ms_ldap_timeout();
#Modify ms_ldap_timeout value
$grid->ms_ldap_timeout(10);
```

ms_log_destination( )

Use this method to direct the logging of synchronization messages to the syslog or mslog.

Parameter

Valid values are "SYSLOG" and "MSLOG".

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get ms_log_destination value
my $log_destination = $grid->ms_log_destination();
#Modify ms_log_destination value
$grid->ms_log_destination("SYSLOG");
```

ms_max_connection( )

Use this method to set or retrieve the maximum number of connections to Microsoft servers.

Parameter

Valid values are integers between 2 and 40 inclusive.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get ms_max_connection value
my $max_connection = $grid->ms_max_connection();
#Modify ms_max_connection value
$grid->ms_max_connection(20);
```

ms_rpc_timeout( )

Use this method to set or retrieve timeout in seconds for RPC connections to all Microsoft servers.

Parameter

Valid values are integers between 0 and 65535 inclusive.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get ms_rpc_timeout value
my $rpc_timeout = $grid->ms_rpc_timeout();
#Modify ms_rpc_timeout value
$grid->ms_rpc_timeout(10);
```

ms_user_default_timeout( )

Use this method to set or retrieve the default timeout in seconds for Active Directory users synchronization for all Microsoft servers.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

The valid value is an unsigned integer.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get ms_user_default_timeout value
my $user_default_timeout = $grid->ms_user_default_timeout();
#Modify ms_user_default_timeout value
$grid->ms_user_default_timeout(10);
```

name( )

Use this method to set or retrieve the grid name.

Parameter

String with the name of the grid.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

Example

```
#Get name
my $name = $grid->name();
#Modify name
$grid->name("InfobloxGrid");
```

nat_group_list( )

Use this method to set or retrieve the NAT (Network Address Translation) group list.

Parameter

Array reference that contains each Infoblox::Grid::NatGroup object. The default value is empty.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

Example

```
#Add a new nat_group_list
my $nat1 = Infoblox::Grid::NatGroup->new( "name" => "natgroup" ,
                                          "comment" => "new NAT group" );

$grid->nat_group_list( [$nat1] );

#To undefined the nat_group_list
$grid->nat_group_list([ ]);
```

notification_email_addr( )

Use this method to set or retrieve the notification e-mail address.

Parameter

Text with the e-mail address of the administrator.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

Example

```
#Get notification_email_addr
my $email = $grid->notification_email_addr();
#Modify notification_email_addr
```

```
$grid->notification_email_addr("admin\@domain.com");
```

ntp_server( )

Use this method to set or retrieve the NTP (Network Time Protocol) server list. Only valid when enable_ntp is set to "true".

Parameter

Array reference that contains Infoblox::Grid::NTPServer objects.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

Example

```
#Add a new ntp_server
my $ntp_server1 = Infoblox::Grid::NTPServer->new(address => "1.1.1.1");
my $ntp_server2 = Infoblox::Grid::NTPServer->new(address => "2.2.2.2");
$grid->ntp_server([$ntp_server1, $ntp_server2]);
```

ntp_access_list( )

Use this method to set or retrieve the NTP (Network Time Protocol) access control list. Only valid when enable_ntp is set to "true".

Parameter

Valid value is an Infoblox::Grid::NamedACL object or an array reference that contains Infoblox::Grid::NTPAccess objects.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

Example

```
#Get ntp_access_list
my @ntp_access_list = $grid->ntp_access_list();
#Modify ntp_access_list
$grid->ntp_access_list([$ntpac1,$ntpac2]);
#To undefined ntp_access_list
$grid->ntp_access_list([ ]);
#Use Infoblox::Grid::NamedACL
$grid->ntp_access_list($nacl);
```

ntp_service_type( )

Use this method to set or retrieve the NTP service type for assigned Named ACL object.

The attribute value is ignored if 'ntp_access_list' attribute is not a Named ACL object.

Parameter

Valid values are "Time" or "Time and NTP control". The default value is "Time".

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get ntp_service_type
my $service = $grid->ntp_service_type();
#Modify ntp_service_type
$grid->ntp_service_type("Time and NTP control");
```

ntp_authentication_key( )

Use this method to set or retrieve the NTP (Network Time Protocol) authentication keys of the grid. Only valid when enable_ntp is set to "true".

Parameter

Valid value is an array reference that contains Infoblox::Grid::NTPKey objects.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

Example

```
#Add a new ntp_authentication_key
my $ntp_key1 = Infoblox::Grid::NTPKey->new(
            key_number => "1111",
            key_type   => "M",
            key_string => "aaaa"
        );

my $ntp_key2 = Infoblox::Grid::NTPKey->new(
            key_number => "2222",
            key_type   => "n",
            key_string => "d3e54352e5548080"
        );

$grid->ntp_authentication_key([$ntp_key1, $ntp_key2]);
#To undefined ntp_authentication_key
$grid->ntp_authentication_key(undef);
```

ntp_kod( )

Use this method to enable Kiss-o'-Death packets on the NTP server.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

Specify "true" to enable Kiss-o'-Death packets on the NTP server or "false" to disable them. The default value is "false".

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get the current value of the ntp_kod parameter
my $ntp_kod = $grid->ntp_kod();
#Set the ntp_kod parameter
$grid->ntp_kod("true");
```

objects_changes_tracking_setting( )

Use this method to set or retrieve the object changes tracking setting.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

The valid value is an Infoblox::Grid::ObjectsChangesTrackingSetting object.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get objects_changes_tracking_setting value
my $value = $object->objects_changes_tracking_setting();

#Construct objects_changes_tracking_setting value
my $objects_changes_tracking_setting = Infoblox::Grid::ObjectsChangesTrackingSetting->new(
    enable => 'true',
);

#Modify objects_changes_tracking_setting value
$object->objects_changes_tracking_setting($objects_changes_tracking_setting);
```

password_chars_to_change( )

Use this method to set or retrieve the minimum required character changes from the previous user password.

Parameter

Valid values are between 0 and 64. The default value is 0.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get password_chars_to_change
my $password_chars_to_change = $obj->password_chars_to_change();
#Modify password_chars_to_change
$obj->password_chars_to_change(6);
```

password_expire_days( )

Use this parameter to set the number of days that a password is valid. This parameter will be ignored if password_expiration_enable is not set to "true".

Parameter

The parameter is a number of days of password validity. The default value is 30.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get the current value of the password_expire_days
my $passw_expiration = $grid->password_expire_days();
#Set the password_expire_days parameter
$grid->password_expire_days(120);
```

password_expire_enable( )

Use this parameter to enable password expiration.

Parameter

Setting the parameter to "true" enables user passwords to expire. Setting the parameter to "false" enables user passwords to remain valid indefinitely. The default value is "false".

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get the current value of the password_expire_enable parameter
my $passw_expiration = $grid->password_expire_enable();
#Set the password_expire_enable parameter
$grid->password_expire_enable("true");
```

password_force_reset( )

Use this parameter to force users to reset their passwords when they first log in.

Parameter

Setting the parameter to "true" forces users to reset their passwords when they first log in. Setting the parameter to "false" allows the new user to keep the admin-defined password. The default value is "false".

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get the current value of the password_force_reset parameter
my $passw_force_reset = $grid->password_force_reset();
#Set the password_force_reset parameter
```

```
$grid->password_force_reset("true");
```

password_min_length( )

Use this method to set or retrieve the minimum user password length.

Parameter

Valid values are between 4 and 64.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get password_min_length
my $password_min_length = $obj->password_min_length();
#Modify password_min_length
$obj->password_min_length(6);
```

password_num_lower_char( )

Use this method to set or retrieve the required minimum number of lowercase characters in a user password.

Parameter

Valid values are between 0 and 64. The default value is 0. Note that the total of password_num_lower_char, password_num_upper_char, password_num_numeric_char and password_num_symbol_char must not exceed password_min_length.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get password_num_lower_char
my $password_num_lower_char = $obj->password_num_lower_char();
#Modify password_num_lower_char
$obj->password_num_lower_char(6);
```

password_num_numeric_char( )

Use this method to set or retrieve the required minimum number of numeric characters in a user password.

Parameter

Valid values are between 0 and 64. The default value is 0. Note that the total of password_num_lower_char, password_num_upper_char, password_num_numeric_char and password_num_symbol_char must not exceed password_min_length.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get password_num_numeric_char
my $password_num_numeric_char = $obj->password_num_numeric_char();
#Modify password_num_numeric_char
$obj->password_num_numeric_char(6);
```

password_num_symbol_char( )

Use this method to set or retrieve the required minimum number of symbol (non-alphanumeric) characters in a user password.

Parameter

Valid values are between 0 and 64. The default value is 0. Note that the total of password_num_lower_char, password_num_upper_char, password_num_numeric_char and password_num_symbol_char must not exceed password_min_length.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get password_num_symbol_char
my $password_num_symbol_char = $obj->password_num_symbol_char();
#Modify password_num_symbol_char
$obj->password_num_symbol_char(6);
```

password_num_upper_char( )

Use this method to set or retrieve the required minimum number of uppercase characters in a user password.

Parameter

Valid values are between 0 and 64. The default value is 0. Note that the total of password_num_lower_char, password_num_upper_char, password_num_numeric_char and password_num_symbol_char must not exceed password_min_length.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get password_num_upper_char
my $password_num_upper_char = $obj->password_num_upper_char();
#Modify password_num_upper_char
$obj->password_num_upper_char(6);
```

password_reminder_days( )

Use this method to set the number of days before a password expires that the appliance starts displaying password change reminders. This parameter will be ignored if password_expiration_enable is "false".

Parameter

The parameter is a number of days before password expiration when the user get a reminder to change password. The default value is 7. Note that password_reminder_days must be lower than password_expire_days.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get the current value of the password_reminder_days parameter
my $passw_reminder= $grid->password_reminder_days();
#Set the password_reminder_days parameter
$grid->password_reminder_days(7);
```

prefer_resolver( )

Use this method to set or retrieve the preferred resolver.

Parameter

The valid values are Infoblox::DNS::Member object or Infoblox::DNS::Nameserver object.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

Example

```
#Get prefer_resolver
my $prefer_resolver = $grid->prefer_resolver();
#Modify prefer_resolver
$grid->prefer_resolver("3.3.3.3");
```

query_comm_string ( )

Use this method to set or retrieve the SNMP (Simple Network Management Protocol) community string that management systems must send together with their queries to the Infoblox appliance. If this is set to undef, SNMP queries will be disabled.

Parameter

Text with the SNMP community string.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

Example

```
#Get query_comm_string
my $query_comm_string = $grid->query_comm_string();
#Modify query_comm_string
$grid->query_comm_string("private");
```

remote_console_access( )

Use this method to enable/disable remote console access.

Parameter

Specify "true" to enable remote console access or "false" to disable it.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

Example

```
#Get remote_console_access
my $remote_console_access = $grid->remote_console_access();
#Modify remote_console_access
$grid->remote_console_access("false");
```

restart_banner_setting( )

Use this method to set or retrieve Grid restart banner setting.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

The valid value is an Infoblox::Grid::RestartBannerSetting object.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get restart_banner_setting
$restart_banner_setting = $grid->restart_banner_setting();

#Modify restart_banner_setting
$grid->restart_banner_setting($banner);
```

restart_status( )

Use this method to retrieve the restart status of the Grid. This is a read-only attribute.

Omit the parameter to retrieve the attribute value.

Parameter

None

Returns

The valid return value is an Infoblox::Grid::ServiceRestart::Status object.

Example

```
#Get restart_status
$restart_status = $grid->restart_status();
```

rpz_hit_rate_interval( )

Use this method to set or retrieve the minimum time interval (in seconds) between changes in RPZ Hit Rate.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

The valid value is an unsigned integer greater than 1 that represents the number of seconds between changes in RPZ Hit Rate.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
# Get attribute value
my $value = $grid->rpz_hit_rate_interval();
# Modify attribute value
$grid->rpz_hit_rate_interval(20);
```

rpz_hit_rate_min_query( )

Use this method to set or retrieve the minimum number of queries between RPZ Hit Rate checks.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

The valid value is an unsigned integer between 0 and 1073741824 that represents the minimum number of queries between changes in RPZ Hit Rate.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
# Get attribute value
my $value = $grid->rpz_hit_rate_min_query();
# Modify attribute value
$grid->rpz_hit_rate_min_query(20);
```

rpz_hit_rate_max_query( )

Use this method to set or retrieve the maximum number of queries between RPZ Hit Rate checks.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

The valid value is an unsigned integer between 1 and 1073741824 that represents the maximum number of queries between changes in RPZ Hit Rate.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
# Get attribute value
my $value = $grid->rpz_hit_rate_max_query();
# Modify attribute value
$grid->rpz_hit_rate_max_query(20);
```

search_domains( )

Use this method to set or retrieve the Search Domain Group, which is a group of domain names that the Infoblox device can add to partial queries that do not specify a domain name. Note that you can set this parameter only when prefer_resolver or alternate_resolver is set.

Parameter

Valid value is a reference to an array of strings.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

Example

```
#Get search_domains
my $search_domains = $grid->search_domains();
#Modify search_domains
$grid->search_domains(["domain1.com", "domain2.com"]);
```

secret( )

Use this method to set the shared secret of the grid. This is a write-only attribute.

Parameter

Text with the shared secret of the grid.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

Example

```
#Modify secret
$grid->secret("shared secret");
```

service_status( )

Use this method to retrieve the overall service status of the Grid. This is a read-only attribute.

Parameter

None

Returns

The valid return values are 'FAILED', 'OFFLINE', 'INACTIVE', 'UNKNOWN', 'WARNING' and 'WORKING'.

Example

```
#Get service_status
$grid->service_status();
```

support_access( )

Use this method to enable/disable support access. Default value is "true".

Parameter

Specify "true" to enable support access or "false" to disable it.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

Example

```
#Get support_access
my $support_access = $grid->support_access();
#Modify support_access
$grid->support_access("false");
```

support_access_info( )

Use this method to retrieve the information string for support access.

Parameter

None.

Returns

The method returns the attribute value.

Example

```
#Get support_access_info
my $support_access_info = $grid->support_access_info();
```

scheduled_backup( )

Use this method to retrieve the scheduled backup configuration.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

Valid value is an Infoblox::Grid::ScheduledBackup object.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

Example

```
#Get scheduled_backup
my $scheduled_backup = $grid->scheduled_backup();
#Modify scheduled_backup
$grid->scheduled_backup($scheduled_backup);
```

session_timeout( )

Use this method to set or retrieve the session timeout.

Parameter

Number that specifies the session timeout for the grid (range is from 60 to 31536000 seconds).

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

Example

```
#Get session_timeout
my $session_timeout = $grid->session_timeout();
#Modify session_timeout
$grid->session_timeout("6000");
```

snmp_admin( )

Use this method to set or retrieve the SNMP admin object.

Parameter

Valid data type is an Infoblox::Grid::SNMP::Admin object.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

Example

```
#Add a SNMP Admin object
my $snmp = Infoblox::Grid::SNMP::Admin->new(
    sysContact  => "Somebody",
    sysLocation => "Infoblox Headquarter",
    sysName     => "noc1",
    sysDescr    => "SunOS noc1 5.6 Generic_105181-03 sun4u"
);

$grid->snmp_admin($snmp);
#To undefined SNMP Admin object
$grid->snmp_admin(undef);
```

snmpv3_query_users( )

Use this method to set or retrieve the SNMPv3 users from which the appliance accepts SNMP queries.

Parameter

Valid value is a reference to the list of the Infoblox::Grid::SNMP::User manpage objects.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

Example

```
#Get snmpv3_query_users field
my $snmpv3_users = $grid->snmpv3_query_users();
my $user = $$snmpv3_users[0];
#Set snmpv3_query_users field
$grid->snmpv3_query_users([$user1, $user2]);
```

syslog_backup_servers( )

Use this method to set or retrieve an array that contains syslog backup server settings for a Grid.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

Valid value is an array reference that contains Infoblox::Grid::SyslogBackupServer objects.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#get syslog_backup_servers value
my $servers = $grid->syslog_backup_servers();

#modify syslog_backup_servers value
$grid->syslog_backup_servers([$server1, $server2]);
```

syslog_server( )

Use this method to set or retrieve the list of syslog servers.

Setting this method to a defined value implicitly sets the external_syslog_server_enable method to 'true'. Setting the parameter to undefined sets the external_syslog_server_enable method to 'false'.

Note that when syslog_server is set to a defined value and external_syslog_server_enable is set to 'false', the last operation takes precedence. Thus the sequence $object->syslog_server([$syslog]); $object->external_syslog_server_enable('false'); will set external_syslog_server_enable to 'false', and the sequence $object->external_syslog_server_enable('false'); $object->syslog_server([$syslog]); will result in external_syslog_server_enable='true'.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

Array reference that contains Infoblox::Grid::SyslogServer objects.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

Example

```
my $syslog1 = Infoblox::Grid::SyslogServer->new(
    ipv4addr => "44.44.44.44",
    severity => "debug",
);

my $syslog2 = Infoblox::Grid::SyslogServer->new(
    ipv4addr => "55.55.55.55",
    severity => "debug",
);

$grid->syslog_server([$syslog1, $syslog2]);
```

syslog_size( )

Use this method to set or retrieve the grid syslog rotation size in megabytes.

Parameter

Valid value is between 10 and 300.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get syslog_size
my $syslog_size = $obj->syslog_size();
#Modify syslog_size
$obj->syslog_size(100);
```

threshold_traps( )

Use this method to retrieve or modify the list of threshold traps. Each element in the list is a reference to the Infoblox::Grid::SNMP::ThresholdTrap object. All the possible threshold traps are in the list. The user can only change the values for each trap or remove traps.

Parameter

Reference to a list of Infoblox::Grid::SNMP::ThresholdTrap objects. The default value is a reference to a list of all threshold traps.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get threshold traps
my $threshold_traps = $grid->threshold_traps();
#Modify threshold traps
my $threshold_traps = $grid->threshold_traps();
foreach(@$threshold_traps){if $_->trap_name() eq 'Rootfs'; $_->trap_reset=60;} #Change value for
'Rootfs' trap
$grid->threshold_traps($threshold_traps);
$session->modify($grid);
```

token_usage_delay( )

Use this method to set or retrieve the amount of time (in minutes) that the one-time security token for a pre-provisioned vNIOS member can stay unused before the appliance logs a warning message. The token_usage_delay has a life time of 60 minutes.

Parameter

The amount of time (in minutes) that the one-time security token for a pre-provisioned vNIOS member can stay unused before the appliance logs a warning message. Valid values must be between 1 and 60. The default value is 10.

Returns

If you specified a parameter,the method returns true when the modification succeeds,and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get token_usage_delay
my $delay = $grid->token_usage_delay();
#Modify token_usage_delay
$grid->token_usage_delay("30");
```

trap_comm_string( )

Use this method to set or retrieve the trap community string. If this is set to undef, SNMP traps will be disabled.

Parameter

Text with the SNMP trap community string.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

Example

```
#Get trap_comm_string
my $trap_comm_string = $grid->trap_comm_string();
#Modify trap_comm_string
$grid->trap_comm_string("public");
```

trap_notifications( )

Use this method to configure trap notifications. Each element in the list is a reference to the Infoblox::Grid::SNMP::TrapNotification object.

Parameter

Reference to a list of Infoblox::Grid::SNMP::TrapNotification objects. The default value is a reference to a list of all trap notifications.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get a reference to trap notifications array
my $trap_notifications = $grid->trap_notifications();
#Modify trap notifications
my $notifications = $grid->trap_notifications();
foreach(@$notifications){if $_->trap_type() eq 'CPU'; $_->enable_email="true";} #Change value for
'CPU' trap
$grid->trap_notifications($notifications);
$session->modify($grid);
```

trap_receiver( )

Use this method to set or retrieve a trap receiver.

Parameter

Array reference that contains IPv4 addresses (example: 192.168.1.2) and the Infoblox::Grid::SNMP::TrapReceiver manpage objects.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

Example

```
#Get trap_receiver
my $trap_receiver = $grid->trap_receiver();
#Modify trap receiver (SNMPv2)
$grid->trap_receiver(["10.0.0.10"]);
#SNMPv3 trap receivers
my $receiver1 = Infoblox::Grid::SNMP::TrapReceiver->new( ip_addr => "19.20.21.22", user => $user);
$grid->trap_receiver([$receiver1]);
```

updates_download_member_config( )

Use this method to set or retrieve the list of updates download member configurations.

Parameter

The valid value is array of Infoblox::Grid::UpdatesDownloadMemberConfig objects.

Returns

If you specified a parameter,the method returns true when the modification succeeds,and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get updates_download_member_config value
my $updates_download_member_config = $grid->updates_download_member_config();

#Modify updates_download_member_config value
$grid->updates_download_member_config([$config1, $config2]);
```

vpn_port( )

Use this method to set or retrieve the VPN port.

Parameter

Number of the Virtual Private Network (VPN) port of the grid. Value must be between 1024 and 63999.

Returns

If you specified a parameter,the method returns true when the modification succeeds,and returns false when the operation fails.

Example

```
#Get port
my $port = $grid->vpn_port();
#Modify vpn_port
$grid->vpn_port("2000");
```

SAMPLE CODE

The following sample code demonstrates different operations that can be applied to an object, such create, modify, and remove an object. This sample code also includes error handling for the operations.

```
#PROGRAM STARTS: Include all the modules that will be used
use strict;
use Infoblox;

#Create a session to the Infoblox device
my $session = Infoblox::Session->new(
    master   => "192.168.1.2",
    username => "admin",
    password => "infoblox"
);
unless ($session) {
   die("Construct session failed: ",
       Infoblox::status_code() . ":" . Infoblox::status_detail());
}
print "Session created successfully\n";
```

#Get and modify the Grid object.

```
#Get a Grid through the session
my $grid = $session->get(
    object => "Infoblox::Grid",
    name   => "Infoblox"
);
unless ($grid) {
    die("Get Grid failed: ",
        $session->status_code() . ":" . $session->status_detail());
}

#Modifying the value of the specified object.
my $nat1 = Infoblox::Grid::NatGroup->new( "name" => "natgroup" ,
                                          "comment" => "new NAT group" );

$grid->nat_group_list( [$nat1] );
$grid->enable_ntp ("true");
my $ntp_server1 = Infoblox::Grid::NTPServer->new(address => "2.2.2.2");
$grid->ntp_server([$ntp_server1]);

my $ntpac1 = Infoblox::Grid::NTPAccess->new(
                                           address => "2.2.2.2",
                                           service => "Time",
                                          );

my $ntpac2 = Infoblox::Grid::NTPAccess->new(
                                           address => "4.4.4.4",
                                           service => "Time and NTP control",
                                          );

$grid->ntp_access_list([$ntpac1,$ntpac2]);
my $ntp_key1 = Infoblox::Grid::NTPKey->new(
           key_number => "1111",
           key_type   => "M",
           key_string => "aaaa"
```

```
        );
my $ntp_key2 = Infoblox::Grid::NTPKey->new(
            key_number => "2222",
            key_type   => "n",
            key_string => "d3e54352e5548080"
        );
$grid->ntp_authentication_key([$ntp_key1, $ntp_key2]);

#Apply the changes
$session->modify($grid)
    or die("Modify Grid failed: ",
            $session->status_code() . ":" . $session->status_detail());
print "Grid object modified successfully \n";

####PROGRAM ENDS####
```

AUTHOR

Infoblox Inc. http://www.infoblox.com/

SEE ALSO

Infoblox::Session, Infoblox::Session->get(), Infoblox::Session->search(), Infoblox::Session->add(), Infoblox::Session->remove(), Infoblox::Session->modify(), Infoblox::Grid::NamedACL, Infoblox::Grid::ExtensibleAttributeDef::Descendants, Infoblox::Grid::SyslogBackupServer , Infoblox::DNS::View, Infoblox::DNS::Zone, Infoblox::Grid::LOM::User, Infoblox::Grid::ServiceRestart::Status, Infoblox::Grid::RestartBannerSetting

COPYRIGHT

Copyright (c) 2017 Infoblox Inc.

Infoblox::Grid::Admin::AdAuthServer - Active Directory Authentication Server object

- NAME
- DESCRIPTION
- CONSTRUCTOR
- SESSION METHODS
  - Infoblox::Grid::Admin::AdAuthService->ad_auth_server_list()
- METHODS
  - comment( )
  - disabled( )
  - encryption( )
  - name( )
  - port( )
  - mgmt_port( )
- AUTHOR
- SEE ALSO
- COPYRIGHT

NAME

Infoblox::Grid::Admin::AdAuthServer - Active Directory Authentication Server object

DESCRIPTION

Represents an AD authentication server (i.e., domain controller) that is used to authenticate administrators.

CONSTRUCTOR

```
my $ad_auth_server = Infoblox::Grid::Admin::AdAuthServer->new(
    name        => $fqdn | $ipv4addr,    # Required
    port        => $num,                 # Required
    comment     => $string,              # Optional / Default is empty
    disabled    => "true" | "false",     # Optional / Default is "false"
    encryption  => "SSL" | "None",       # Optional / Default is "None"
    mgmt_port   => "true" | "false",     # Optional / Default is "false"
);
```

SESSION METHODS

This section describes all the methods in an Infoblox::Session module that you can apply to an object.

Infoblox::Grid::Admin::AdAuthService->ad_auth_server_list()

Use this method to specify AD authentication servers that are used to authenticate administrators.

Example

```
# Construct an object
my $ad_auth_server1 = Infoblox::Grid::Admin::AdAuthServer->new(
    name        => "1.2.3.4",
    port        => 636,
    comment     => "domain controller in room 1"
    disabled    => "false",
    encryption  => "SSL",
    mgmt_port   => "false",
);

# Construct another object
my $ad_auth_server2 = Infoblox::Grid::Admin::AdAuthServer->new(
    name        => "foo.com",
    port        => 389,
    comment     => "domain controller in room 2"
    disabled    => "false",
    encryption  => "None",
    mgmt_port   => "false",
);

# Configure AdAuthServer
my $response = $ad_auth_service->ad_auth_server_list([$ad_auth_server1, $ad_auth_server2]);
```

METHODS

Use the following methods to access the attributes of an Infoblox::Grid::Admin::AdAuthServer object. Specify a parameter to set the attribute's value, or omit the parameter to get the attribute's value.

comment( )

Use this method to set or retrieve comments about the server.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

Comments in string format with a maximum of 256 bytes.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get comment
my $val = $obj->comment();
#Modify comment
$obj->comment("head office");
```

disabled( )

Use this method to disable the Active Directory authentication service.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

Set the parameter to "true" to disable the service. Set the parameter to "false" to enable the service. The default value is "false".

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get disabled
my $val = $obj->disabled();
#Modify disabled
$obj->disabled("true");
```

encryption( )

Use this method to set or retrieve the encryption flag.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

Specify "SSL" to use SSL encryption or "None" to use no encryption.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get encryption
my $val = $obj->encryption();
#Modify encryption
$obj->encryption("SSL");
```

name( )

Use this method to set or retrieve the FQDN or IPv4 address of the AD authentication server.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

The FQDN or IPv4 address of the AD authentication server in string format.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get name
my $val = $obj->name();
#Modify name
$obj->name("1.2.3.5");
```

port( )

Use this method to set or retrieve the port number on the AD server to which the appliance sends authentication requests.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

TCP port of the AD authentication server

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get port
my $val = $obj->port();
#Modify port
$obj->port(389);
```

mgmt_port( )

Use this method to set or retrieve the MGMT port flag of the AD server. Set this flag to "true" to restrict the appliance to connecting to the AD server through the management port only.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

```
Specify "true" to set the mgmt_port flag or "false" to deactivate/unset it. The default value is
"false".
```

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get mgmt_port
my $val = $obj->mgmt_port();
#Modify mgmt_port
$obj->mgmt_port('false');
```

AUTHOR

Infoblox Inc. http://www.infoblox.com

SEE ALSO

Infoblox::Grid::Admin::AdAuthService, Infoblox::Session, Infoblox::Session->add(), Infoblox::Session->get(), Infoblox::Session->modify(), Infoblox::Session->remove()

COPYRIGHT

Copyright (c) 2017 Infoblox Inc.

Infoblox::Grid::Admin::AdAuthService - Active Directory Authentication Service object

- NAME
- DESCRIPTION
- CONSTRUCTOR
- SESSION METHODS
  - Infoblox::Session->add( )
  - Infoblox::Session->get( )
  - Infoblox::Session->modify( )
  - Infoblox::Session->remove( )
- METHODS
  - ad_auth_server_list( )
  - ad_domain( )
  - additional_search_paths( )
  - comment( )
  - disabled( )
  - disable_default_search_path( )
  - name( )
  - nested_group_querying( )
  - timeout( )
- SAMPLE CODE
- AUTHOR
- SEE ALSO
- COPYRIGHT

NAME

Infoblox::Grid::Admin::AdAuthService - Active Directory Authentication Service object

---

DESCRIPTION

This object allows you to specify an Active Directory (AD) authentication method and the AD authentication servers that Infoblox uses to authenticate administrators.

CONSTRUCTOR

```
my $obj = Infoblox::Grid::Admin::AdAuthService->new(
    ad_auth_server_list  => [$ad_server1, $ad_server2, ...],      #Required
    ad_domain            => $string,                              #Required
    additional_search_paths => [$string, ...],                    #Optional / Default is undefined
    disable_default_search_path => "true" | "false",              #Optional / Default is "false"
    name                 => $string,                              #Required
    nested_group_querying => "true" | "false",                    #Optional / Default is "false"
    comment              => $string,                              #Optional / Default is empty
    disabled             => "true" | "false",                     #Optional / Default is "false"
    timeout              => $num,                                 #Optional / Default is 5
);
```

SESSION METHODS

This section describes all the methods in an Infoblox::Session module that you can apply to an object.

Infoblox::Session->add( )

Use this method to add an object to the Infoblox appliance. See Infoblox::Session->add() for parameters and return values.

Example

```
# Construct an object
my $ad_auth_server1 = Infoblox::Grid::Admin::AdAuthServer->new(
    name       => "1.2.3.4",
    port       => 636,
    comment    => "domain controller in room 1"
    disabled   => "false",
    encryption => "SSL",
    mgmt_port  => "false",
);

# Construct another object
```

```
my $ad_auth_server2 = Infoblox::Grid::Admin::AdAuthServer->new(
    name       => "foo.com",
    port       => 389,
    comment    => "domain controller in room 2"
    disabled   => "false",
    encryption => "None",
    mgmt_port  => "false",
);

# Construct an object
my $obj = Infoblox::Grid::Admin::AdAuthService->new(
   ad_auth_server_list  => [$ad_auth_server1, $ad_auth_server2],
   ad_domain            => "authdhcptest.infoblox.com",
   comment              => "test comment",
   disabled             => "false",
   name                 => "Test AD Auth",
   timeout              => 333,
   nested_group_querying => "true",
   disable_default_search_path => "false",
   additional_search_paths => ["OU=OU_1","OU=OU_2,OU=OU_3"],
);
# Submit for addition
my $response = $session->add( $obj );
```

Infoblox::Session->get( )

Use this method to retrieve all the matching objects from the Infoblox appliance. See Infoblox::Session->get() for parameters and return values.

Key References

Apply the following attributes to get a specific object:

name - Required. AD authentication service name

Example

```
my @retrieved_objs = $session->get(
  object => "Infoblox::Grid::Admin::AdAuthService",
    name => "Test AD Auth",
);
```

Infoblox::Session->modify( )

Use this method to modify an object in the Infoblox appliance. See Infoblox::Session->modify() for parameters and return values.

Example

```
# Modify the comment
$obj->comment("this is a modified comment");
# Submit modification
my $response = $session->modify( $obj);
```

Infoblox::Session->remove( )

Use this method to remove an object from the Infoblox appliance. See Infoblox::Session->remove() for parameters and return values.

To remove a specific object, first use `get()` to retrieve the specific object, and then submit this object for removal.

Example

```
#Get the objects with the same name
my @retrieved_objs = $session->get(
  object => "Infoblox::Grid::Admin::AdAuthService",
    name => "Test AD Auth",
);
#Find the desired object from the retrieved list.
my $desired_obj = $retrieved_objs[0];
# Submit for removal
my $response = $session->remove( $desired_obj );
```

METHODS

Use the following methods to access the attributes of an Infoblox::Grid::Admin::AdAuthService object. Specify a parameter to set the attribute's value, or omit the parameter to get the attribute's value.

ad_auth_server_list( )

Use this method to set or retrieve the AD authentication server list.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

Valid value is an array reference that contains a list of Infoblox::Grid::Admin::AdAuthServer objects that describe the AD servers to which the appliance connects.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get ad_auth_server_list
my $val = $obj->ad_auth_server_list();
#Modify ad_auth_server_list
$obj->ad_auth_server_list([ $adauthserver1 ]);
```

ad_domain( )

Use this method to set or retrieve the Active Directory Domain in which the user credentials are located.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

Valid Active Directory Domain name

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get ad_domain
my $val = $obj->ad_domain();
#Modify ad_domain
$obj->ad_domain("authdhcptest.infoblox.com");
```

additional_search_paths( )

Use this method to set or retrieve an unordered list of additional search paths for nested group querying.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

The valid value is an array of search paths in a string format.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#get additional_search_paths value
$additional_search_paths = $obj->additional_search_paths();

#modify additional_search_paths value
$obj->additional_search_paths(['OU=OU_1', 'OU=OU_2,OU=OU_3']);
```

comment( )

Use this method to set or retrieve comments about the service.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

Comments in string format with a maximum of 256 bytes.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get comment
my $val = $obj->comment();
#Modify comment
$obj->comment("head office");
```

disabled( )

Use this method to disable the Active Directory authentication service.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

Set the parameter to "true" to disable the service. Set the parameter to "false" to enable the service. The default value is "false".

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get disabled
my $val = $obj->disabled();
#Modify disabled
$obj->disabled("true");
```

disable_default_search_path( )

Use this method to set or retrieve the flag that indicates whether default search path is used or not.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Paramter

Specify 'true' to exclude default search path or 'false' otherwise. The default value is 'false'.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get disable_default_search_path value
my $val = $obj->disable_default_search_path();
#Modify disable_default_search_path value
$obj->disable_default_search_path("true");
```

name( )

Use this method to set or retrieve the name of the AD authentication service.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

The AD authentication service name.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get name
my $val = $obj->name();
#Modify name
$obj->name("AD service");
```

nested_group_querying( )

Use this method to set or retrieve the flag that indicates whether nested group querying is enabled or not.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Paramter

Specify 'true' to enable nested group querying or 'false' to disable it. The default value is 'false'.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get nested_group_querying value
my $val = $obj->nested_group_querying();
#Modify nested_group_querying value
$obj->nested_group_querying("true");
```

timeout( )

Use this method to set or retrieve the timeout value, which is the number of seconds that the appliance waits for a response from the AD server.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

The number of seconds that the Infoblox appliance waits to connect to the server. The default value is 5.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get timeout
my $val = $obj->timeout();
#Modify timeout
$obj->timeout(22);
```

SAMPLE CODE

The following sample code demonstrates the different functions that can be applied to an Active Directory Authentication Service object, such as add, get, modify, and remove. This sample also includes error handling for the operations.

#Preparation prior to an AD Auth Service object insertion

```
use strict;
use Infoblox;

#Create a session to the Infoblox appliance
my $session = Infoblox::Session->new(
               master   => "192.168.1.2", #appliance host ip
               username => "admin",       #appliance user login
               password => "infoblox"     #appliance password
);
unless ($session) {
       die("Construct session failed: ",
            Infoblox::status_code() . ":" . Infoblox::status_detail());
}
print "Session created successfully\n";
```

#Create an AD Auth Service object

```
# Construct an object
```

```
my $ad_auth_server1 = Infoblox::Grid::Admin::AdAuthServer->new(
    name       => "1.2.3.4",
    port       => 636,
    comment    => "domain controller in room 1",
    disabled   => "false",
    encryption => "None",
    mgmt_port  => "false",
);

# Construct another object
my $ad_auth_server2 = Infoblox::Grid::Admin::AdAuthServer->new(
    name       => "foo.com",
    port       => 389,
    comment    => "domain controller in room 2",
    disabled   => "false",
    encryption => "SSL",
    mgmt_port  => "false",
);

#Create an AD Auth Service object
my $ad_service = Infoblox::Grid::Admin::AdAuthService->new(
    ad_auth_server_list => [ $ad_auth_server1, $ad_auth_server2 ],
    ad_domain           => "authdhcptest.infoblox.com",
    comment             => "test comment",
    disabled            => "false",
    name                => "Test AD Auth",
    timeout             => 333,
);
unless($ad_service) {
       die("Construct AD Auth Service object failed: ",
            Infoblox::status_code() . ":" . Infoblox::status_detail());
}
print "AD Auth Service object created successfully\n";

#Add the AD Auth Service object into the Infoblox appliance through a session
$session->add($ad_service)
    or die("Add AD Auth Service object failed: ",
            $session->status_code() . ":" . $session->status_detail());
print "AD Auth Service object added successfully\n";
```

#Get and modify an AD Auth Service

```
#Get an AD Auth Service through the session
my @retrieved_objs = $session->get(
    object     => "Infoblox::Grid::Admin::AdAuthService",
    name => "Test AD Auth",
);
my $object = $retrieved_objs[0];
unless ($object) {
       die("Get AD Auth Service failed: ",
            $session->status_code() . ":" . $session->status_detail());
}
print "Get AD Auth Service object found at least 1 matching entry\n";

#Modify one of the attributes of the specified AD Auth Service
$object->comment ("modified comment");

#Apply the changes
$session->modify($object)
    or die("Modify AD Auth Service failed: ",
            $session->status_code() . ":" . $session->status_detail());
print "AD Auth Service object modified successfully \n";
```

#Remove an AD Auth Service

```
#Get an AD Auth Service through the session
my @retrieved_objs = $session->get(
    object     => "Infoblox::Grid::Admin::AdAuthService",
    name       => "Test AD Auth"
);

my $object = $retrieved_objs[0];
unless ($object) {
       die("Get AD Auth Service failed: ",
            $session->status_code() . ":" . $session->status_detail());
}
print "Get AD Auth Service object found at least 1 matching entry\n";
```

```
#submit the object for removal
$session->remove($object)
    or die("Remove AD Auth Service failed: ",
            $session->status_code() . ":" . $session->status_detail());
print "AD Auth Service object removed successfully \n";

####PROGRAM ENDS####
```

AUTHOR

Infoblox Inc. http://www.infoblox.com

SEE ALSO

Infoblox::Grid::Admin::AdAuthServer, Infoblox::Session, Infoblox::Session->add(), Infoblox::Session->get(), Infoblox::Session->modify(), Infoblox::Session->remove()

COPYRIGHT

Copyright (c) 2017 Infoblox Inc.

Infoblox::Grid::Admin::AuthPolicy - Authentication Policy object

- NAME
- DESCRIPTION
- SESSION METHODS
  - Infoblox::Session->get( )
  - Infoblox::Session->modify( )
- METHODS
  - admin_groups( )
  - auth_services( )
  - default_group( )
  - usage_type( )
- SAMPLE CODE
- AUTHOR
- SEE ALSO
- COPYRIGHT

NAME

Infoblox::Grid::Admin::AuthPolicy - Authentication Policy object

---

DESCRIPTION

This object represents the grid authentication policy.

SESSION METHODS

This section describes all the methods in the Infoblox::Session module that you can apply to the Grid Authentication Policy object.

Infoblox::Session->get( )

Use this method to retrieve the grid authentication policy from the Infoblox appliance. See Infoblox::Session->get() for parameters and return values.

Key Reference

No parameters are required.

Example

```
my $auth_policy = $session->get(
        object => "Infoblox::Grid::Admin::AuthPolicy",
    );
```

Infoblox::Session->modify( )

Use this method to modify the grid authentication policy on the Infoblox appliance. See Infoblox::Session->modify() for parameters and return values.

Example

```
#Changing the default group
$auth_policy->default_group($group);
#Submit modification
my $response = $session->modify( $auth_policy );
```

METHODS

This section describes all the methods that you can use to set or retrieve the attribute values of the grid authentication policy.

admin_groups( )

Use this method to set or retrieve the list of local admin groups that are mapped to remote administration groups. The default value is an empty list.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

The valid value is an array reference that contains Infoblox::Grid::Admin::Group objects.

Returns

If you specified a parameter, the method returns "true" when the modification succeeds, and returns "false" when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get the admin groups list
my $admin_groups = $auth_policy->admin_groups();

#Modify the admin_groups
$auth_policy->admin_groups([$admingroup1,$admingroup2]);
```

auth_services( )

Use this method to set or retrieve the list of authentication services that grid members use to authenticate admins. The default is a list that contains the LocalUserAuthService object.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

The valid value is an array reference that contains an ordered list of Infoblox::Grid::Admin::AdAuthService, Infoblox::Grid::Admin::LocalUserAuthService, Infoblox::Grid::Admin::TACACSPlusAuthService, Infoblox::LDAP::AuthService, Infoblox::Grid::Admin::CertificateAuthService, Infoblox::Grid::Admin::RadiusAuthService objects.

Returns

If you specified a parameter, the method returns "true" when the modification succeeds, and returns "false" when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get the services list
my $auth_services = $auth_policy->auth_services();
#Modify the auth_services
$auth_policy->auth_services([$radius1, $local, $ad2]);
```

default_group( )

Use this method to set or retrieve the default admin group that provides authentication in case no valid group is found. The default value is none (represented by undef).

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

The valid value is an Infoblox::Grid::Admin::Group object.

Returns

If you specified a parameter, the method returns "true" when the modification succeeds, and returns "false" when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get the default group
my $default_group = $auth_policy->default_group();

#Modify the default_group
$auth_policy->default_group($group2);
```

usage_type( )

Use this method to set or retrieve remote policies usage.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

Specify 'FULL' to define the list of remote users, their passwords and their group ownership, and specify 'AUTH_ONLY' to validate passwords of local users only. The default value is 'FULL'.

Returns

If you specified a parameter, the method returns "true" when the modification succeeds, and returns "false" when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get usage_type value
my $usage_type = $auth_policy->usage_type();

#Modify usage_type value
$auth_policy->usage_type('AUTH_ONLY');
```

SAMPLE CODE

The following sample code demonstrates the different functions that can be applied to an object, such as get and modify. This sample code also includes error handling for the operations.

#Preparation prior to getting and modifying an auth policy object

```
use strict;
use Infoblox;

#Creating a session to appliance.
my $session = Infoblox::Session->new(
    master   => "192.168.1.2",
    username => "admin",
    password => "infoblox"
    );
unless( $session ){
    die("Constructor for session failed:",
    Infoblox::status_code() . ":" . Infoblox::status_detail());
}
print" Session object created successfully \n";
```

#Retrieving the auth policy object

```
my $policy = $session->get(
                          object => 'Infoblox::Grid::Admin::AuthPolicy',
                         ) || die("Retrieval of the auth policy failed: ",
                Infoblox::status_code() . ":" . Infoblox::status_detail());
```

#Retrieving the local user auth service

```
my $local_auth = $session->get(
                               object => 'Infoblox::Grid::Admin::LocalUserAuthService',
                              ) || die("Retrieval of the local user auth service failed: ",
                Infoblox::status_code() . ":" . Infoblox::status_detail());
```

#Creation of a Radius auth server/service pair

```
my $ads = Infoblox::Grid::Admin::RadiusAuthServer->new(
                                                        fqdn_or_ip => '10.1.0.1',
                                                        shared_secret => 'secret',
                                                       ) ||
  die("Couldn't create the auth server: ",
      Infoblox::status_code() . ":" . Infoblox::status_detail());

my $service = Infoblox::Grid::Admin::RadiusAuthService->new(
                                                             acct_timeout => 1100,
                                                             auth_timeout => 1100,
                                                             name => 'some.name.com',
                                                             radius_servers => [$ads]
                                                            ) ||
  die("Couldn't create the auth service: ",
      Infoblox::status_code() . ":" . Infoblox::status_detail());

$session->add($service) || die("Couldn't add the auth service: ",
        Infoblox::status_code() . ":" . Infoblox::status_detail());
```

#Retrieving the auth service created above

```
my $radius_auth = $session->get(
```

```
                                object => 'Infoblox::Grid::Admin::RadiusAuthService',
                                name   => 'some.name.com',
                            ) || die("Retrieval of the radius auth service failed: ",
                        Infoblox::status_code() . ":" . Infoblox::status_detail());
```

#Modify the authpolicy

```
$policy->auth_services([$radius_auth,$local_auth]);

$session->modify($policy) || die("Policy modify failed: ",
        Infoblox::status_code() . ":" . Infoblox::status_detail());

####PROGRAM ENDS####
```

AUTHOR

Infoblox Inc. http://www.infoblox.com/

SEE ALSO

Infoblox::Grid::Admin::Group, Infoblox::Grid::Admin::AdAuthService, Infoblox::Grid::Admin::TACACSPlusAuthService, Infoblox::Grid::Admin::LocalUserAuthService, Infoblox::Grid::Admin::RadiusAuthService, Infoblox::LDAP::AuthService, Infoblox::Grid::Admin::CertificateAuthService, Infoblox::Session->get(), Infoblox::Session->get()

COPYRIGHT

Copyright (c) 2017 Infoblox Inc.

Infoblox::Grid::Admin::CertificateAuthService - a Certificate Authentication Service.

- NAME
- DESCRIPTION
- CONSTRUCTOR
- SESSION METHODS
  - Infoblox::Session->add( )
  - Infoblox::Session->get( )
  - Infoblox::Session->modify( )
  - Infoblox::Session->remove( )
  - Infoblox::Session->search( )
- METHODS
  - auto_populate_login( )
  - ca_certificates( )
  - comment( )
  - disabled( )
  - enable_password_request( )
  - enable_remote_lookup( )
  - max_retries( )
  - name( )
  - ocsp_check( )
  - ocsp_responders( )
  - recovery_interval( )
  - remote_lookup_password( )
  - remote_lookup_service( )
  - remote_lookup_username( )
  - response_timeout( )
  - trust_model( )
  - user_match_type( )
- SAMPLE CODE
- AUTHOR
- SEE ALSO
- COPYRIGHT

NAME

Infoblox::Grid::Admin::CertificateAuthService - a Certificate Authentication Service.

DESCRIPTION

An Infoblox::Grid::Admin::CertificateAuthService object represents a Certificate Authentication Service.

CONSTRUCTOR

```
 my $auth_service = Infoblox::Grid::Admin::CertificateAuthService->new(
     name                  => $string,                                  # Required
     ca_certificates       => [$certificate1, ...],                     # Required
     ocsp_responders       => [$responder1, ...],                       # Required unless ocsp_check is "DISABLED"
or "AIA_ONLY"
     disabled              => "true" | "false",                         # Optional / Default value is "false"
     comment               => $string,                                  # Optional / Default value is empty
     trust_model           => "DIRECT" | "DELEGATED",                   # Optional / Default value is "DIRECT"
     recovery_interval     => $uint,                                    # Optional / Default value is 30
     max_retries           => $uint,                                    # Optional / Default value is 0
     response_timeout      => $uint,                                    # Optional / Default value is 1000
     auto_populate_login   => "SERIAL_NUMBER" | "S_DN_CN" | "S_DN_EMAIL" | "SAN_UPN" | "SAN_EMAIL" |
"AD_SUBJECT_ISSUER" # Optional / Default value is "S_DN_CN"
     enable_password_request => "true" | "false",                       # Optional / Default value is "true"
     user_match_type       => "DIRECT_MATCH" | "AUTO_MATCH",            # Optional / Default value is "AUTO_MATCH"
     ocsp_check            => "MANUAL" | "AIA_ONLY" | "AIA_AND_MANUAL" | "DISABLED", # Optional / Default
value is "MANUAL"
     enable_remote_lookup  => "true" | "false",                         # Optional / Default value is "false"
     remote_lookup_service => $auth_server,                             # Optional / Default value is undefined
     remote_lookup_username => $string,                                 # Optional / Default value is undefined
     remote_lookup_password => $string,                                 # Optional / Default value is undefined
 );
```

SESSION METHODS

This section describes all the methods in an Infoblox::Session module that you can apply to a Certificate Authentication Service object.

Infoblox::Session->add( )

Use this method to add an object to the Infoblox appliance. See Infoblox::Session->add() for parameters and return values.

Example

```
#Construct an object
my $auth_service = Infoblox::Grid::Admin::CertificateAuthService->new(
    name            => 'cas',
    ca_certificates => [$ca1, $ca2],
    ocsp_responders => [$responder],

#Submit for addition
my $response = $session->add($auth_service);
```

Infoblox::Session->get( )

Use this method to retrieve all the matching objects from the Infoblox appliance. See Infoblox::Session->get() for parameters and return values.

Key References

```
Apply the following attributes to get a specific Certificate Authentication Service object:

name                   - Optional. The name of the Certificate Authentication Service in string
format.
disabled               - Optional. The boolean flag that shows whether the Certificate
Authentication Service is enabled or disabled.
user_match_type        - Optional. The Certificate Authentication Service user match type in enum
format.
ocsp_check             - Optional. The Certificate Authentication Service ocsp check in enum format.
remote_lookup_username - Optional. The Certificate Authentication Service remote lookup username in
string format.
```

Example

```
my @retrieved_objs = $session->get(
     object => 'Infoblox::Grid::Admin::CertificateAuthService',
     name   => 'ocsp',
);

my @retrieved_objs = $session->get(
     object   => 'Infoblox::Grid::Admin::CertificateAuthService',
     disabled => 'false',
);
```

Infoblox::Session->modify( )

Use this method to modify an object on the Infoblox appliance. See Infoblox::Session->modify() for parameters and return values.

Example

```
# Use this method to modify the comment of a Certificate Authentication Service object
$auth_service->comment('This is modified Certificate Authentication Service object');

# Submit modification
my $response = $session->modify($auth_service);
```

Infoblox::Session->remove( )

Use this method to remove an object from the Infoblox appliance. See Infoblox::Session->remove() for parameters and return values.

To remove a specific object, first use `get()` to retrieve the object, and then submit it for removal.

Example

```
#Get the objects with the specified name.
my @retrieved_objs = $session->get(
     object => 'Infoblox::Grid::Admin::CertificateAuthService',
     name   => 'ocsp',
);

#Find the desired object on the retrieved list.
my $desired_auth_service = $retrieved_objs[0];

# Submit for removal
my $response = $session->remove($desired_auth_service);
```

Infoblox::Session->search( )

Use this method to retrieve all the matching objects from the Infoblox appliance. See Infoblox::Session->search() for parameters and return values.

Key References

```
 Apply the following attributes to get a specific Certificate Authentication Service object:

 name                   - Optional. The name of the Certificate Authentication Service in string
format.
 disabled               - Optional. The boolean flag that shows whether the Certificate
Authentication Service is enabled or disabled.
 user_match_type        - Optional. The Certificate Authentication Service user match type in enum
format.
 ocsp_check             - Optional. The Certificate Authentication Service ocsp check in enum format.
 remote_lookup_username - Optional. The Certificate Authentication Service remote lookup username in
string format.
```

Example

```
my @retrieved_objs = $session->search(
     object => 'Infoblox::Grid::Admin::CertificateAuthService',
     name   => 'ocsp',
);

my @retrieved_objs = $session->search(
     object   => 'Infoblox::Grid::Admin::CertificateAuthService',
     disabled => 'false',
);
```

METHODS

This section describes all the methods that you can use to configure and retrieve the attribute values of a Certificate Authentication Service object.

auto_populate_login( )

Use this method to set or retrieve a parameter that specifies the client certificate value to auto-populate the NIOS login name field.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

The method returns either of these attribute values: "SERIAL_NUMBER", "S_DN_CN", "S_DN_EMAIL", "SAN_UPN", "SAN_EMAIL" or "AD_SUBJECT_ISSUER". The default value is "S_DN_CN".

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get auto_populate_login
my $auto_populate_login = $auth_service->auto_populate_login();

#Set auto_populate_login
$auth_service->auto_populate_login('SERIAL_NUMBER');
```

ca_certificates( )

Use this method to set or retrieve the list of CA certificates in the Certificate Authentication Service object.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

Valid value is an array reference that contains Infoblox::Grid::CACertificate object(s).

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get ca_certificates value
my $ca_certificates = $auth_service->ca_certificates();

#Modify ca_certificates

#Retrieve the existing certificate
my $certificate = $session->get(
    object => 'Infoblox::Grid::CACertificate',
    serial => 'e76227f314c0f3db',
);

#Modify ca_certificates value
$auth_service->ca_certificates([$certificate]);
```

comment( )

Use this method to set or retrieve a descriptive comment.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

Desired comment in string format with a maximum of 256 bytes.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get comment
my $comment = $auth_service->comment();

#Modify comment
$auth_service->comment("Modified comment");
```

disabled( )

Use this method to set or retrieve the disabled flag for the Certificate Authentication Service.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

Specify "true" to disable the Certificate Authentication Service or "false" to enable it. Default value is "false".

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value as "true" or "false".

Example

```
#Get disabled flag
my $disabled = $auth_service->disabled();

#Modify disabled flag
$auth_service->disabled("true");
```

enable_password_request( )

Use this method to set or retrieve the flag that shows whether the username/password authentication together with certificate based is enabled or disabled.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

Specify "true" to enable the username/password authentication together with certificate based or "false" to disable it. Default value is "true".

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value as "true" or "false".

Example

```
#Get enable_password_request flag
my $enable_password_request = $auth_service->enable_password_request();

#Modify enable_password_request flag
$auth_service->enable_password_request("false");
```

enable_remote_lookup( )

Use this method to set or retrieve the flag that shows whether the lookup for users group membership info on remote services is enabled or disabled.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

Specify "true" to enable the lookup for users group membership info on remote services or "false" to disable it. Default value is "false".

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value as "true" or "false".

Example

```
#Get enable_remote_lookup flag
my $enable_remote_lookup = $auth_service->enable_remote_lookup();

#Modify enable_remote_lookup flag
$auth_service->enable_remote_lookup("true");
```

max_retries( )

Use this method to set or retrieve the number of validation retries before the appliance moves on to the next OCSP responder.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

An unsigned integer. Default value is 0.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get max_retries
my $max_retries = $auth_service->max_retries();

#Modify max_retries
$auth_service->max_retries(1);
```

name( )

Use this method to set or retrieve the Certificate Authentication Service name.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

The Certificate Authentication Service name in string format. The default value is "Cert auth service".

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get name
my $name = $auth_service->name();
```

```
#Modify comment
$auth_service->name("New cert group");
```

ocsp_check( )

Use this method to set or retrieve the source of OCSP settings.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

The name of the parameter can be "MANUAL", "AIA_ONLY", "AIA_AND_MANUAL" or "DISABLED". The default value is "MANUAL".

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get ocsp_check
my $ocsp_check = $auth_service->ocsp_check();

#Set ocsp_check
$auth_service->ocsp_check('DISABLED');
```

ocsp_responders( )

Use this method to set or retrieve the list of OCSP responders in the Certificate Authentication Service object.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

The valid value is an array reference that contains Infoblox::OCSP::Responder object(s).

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get ocsp_responders
my $responder_list = $auth_service->ocsp_responders();

#Modify ocsp_responders
my $responder1 = Infoblox::OCSP::Responder->new(
     fqdn_or_ip  => 'domain.com',
     certificate => '/tmp/cert1.pem',
);

my $responder2 = Infoblox::OCSP::Responder->new(
     fqdn_or_ip  => '192.168.1.10',
     certificate => '/tmp/cert2.pem',
);

$auth_service->ocsp_responders([$responder1, $responder2]);
```

recovery_interval( )

Use this method to set or retrieve the specified period of time the appliance waits before retrying a responder that has been offline. The value must be between 1 and 600 seconds.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

An unsigned integer. Default value is 30.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get recovery_interval
my $recovery_interval = $auth_service->recovery_interval();

#Modify recovery_interval
$auth_service->recovery_interval(5);
```

remote_lookup_password( )

Use this method to set or retrieve the password for a service account.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

The password for service account in string format.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get remote_lookup_password
my $remote_lookup_password = $auth_service->remote_lookup_password();

#Set remote_lookup_password
$auth_service->remote_lookup_password('infoblox');
```

remote_lookup_service( )

Use this method to set or retrieve the service that will be used for remote lookup.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

Valid value is a reference that contains Infoblox::Grid::Admin::AdAuthServer object.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get remote_lookup_service
my $remote_lookup_service = $auth_service->remote_lookup_service();

#Modify remote_lookup_service
my $remote_lookup_service1 = Infoblox::Grid::Admin::AdAuthServer->new(
   name => "1.2.3.4",
   port => 15623,
   encryption => "SSL"
);

$auth_service->remote_lookup_service($remote_lookup_service1);
```

remote_lookup_username( )

Use this method to set or retrieve the username for a service account.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

The username for a service account in string format.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get remote_lookup_username
my $remote_lookup_username = $auth_service->remote_lookup_username();

#Set remote_lookup_username
$auth_service->remote_lookup_username('admin');
```

response_timeout( )

Use this method to set or retrieve validation timeout period in milliseconds, must be between 1000 and 60000.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

An unsigned integer. Default value is 1000.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get response_timeout
my $response_timeout = $auth_service->response_timeout();

#Modify response_timeout
$auth_service->response_timeout(7000);
```

trust_model( )

Use this method to set or retrieve a parameter that specifies how the appliance performs OCSP requests.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

The name of parameter is either "DIRECT" or "DELEGATED". Default value is "DIRECT".

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get trust_model
my $trust_model = $auth_service->trust_model();

#Set trust_model
$auth_service->trust_model('DELEGATED');
```

user_match_type( )

Use this method to set or retrieve a parameter that specifies how to search for a particular user: if the parameter is equal to "DIRECT_MATCH" the method searches for only local users with manually assigned certificates, else if the parameter is equal to "AUTO_MATCH" the method populates username from certificate and searches against effective Auth Policies.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

The valid value is "DIRECT_MATCH" or "AUTO_MATCH". The default value is "DIRECT_MATCH".

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get user_match_type
my $user_match_type = $auth_service->user_match_type();

#Set user_match_type
```

```
$auth_service->user_match_type('AUTO_MATCH');
```

SAMPLE CODE

The following sample code demonstrates the different functions that can be applied to an object such as add, get, modify and remove. This sample also includes error handling for the operations.

#Preparation prior to creating and modifying a Certificate Authentication Service object

```
use strict;
use Infoblox;

#refers to Infoblox Appliance IP address
my $host_ip = "192.168.1.2";

#Create a session to the Infoblox appliance

my $session = Infoblox::Session->new(
    master   => $host_ip,
    username => "admin",
    password => "infoblox"
);
unless ($session) {
       die("Construct session failed: ",
               Infoblox::status_code() . ":" . Infoblox::status_detail());
}
print "Session created successfully\n";
```

#Create an OCSP responder

```
my $responder = Infoblox::OCSP::Responder->new(
    address     => 'domain.com',
    certificate => '/tmp/cert.pem',
    disabled => 'true',
);

unless ($responder) {
       die("Construct OCSP responder object failed: ",
               Infoblox::status_code() . ":" . Infoblox::status_detail());
}
print "OCSP responder object created successfully\n";
```

#Create a Certificate Authentication Service object

```
my $auth_service = Infoblox::Grid::Admin::CertificateAuthService->new(
    name            => 'cas',
    ca_certificates => [$ca1, $ca2],
    ocsp_responders => [$responder],
    disabled        => 'true',
);

unless ($auth_service) {
       die("Construct Certificate Authentication Service object failed: ",
               Infoblox::status_code() . ":" . Infoblox::status_detail());
}
print "Certificate Authentication Service object created successfully\n";

#Add the Certificate Authentication Service object to an appliance through session
my $response = $session->add($auth_service);

unless ($response) {
    die("Add Certificate Authentication Service object failed: ",
           $session->status_code() . ":" . $session->status_detail());
}
print "Certificate Authentication Service object added successfully \n";
```

#Get and modify the Certificate Authentication Service object

```
#Get a Certificate Authentication Service object through session
my @retrieved_objs = $session->get(
    object   => "Infoblox::Grid::Admin::CertificateAuthService",
    name     => "cas",
    );
my $auth_service = $retrieved_objs[0];

unless ($auth_service) {
```

```
    die("Get a Certificate Authentication Service object failed: ",
           $session->status_code() . ":" . $session->status_detail());
}
print"Get a Certificate Authentication Service object successful \n";

# Modify one of the attributes of the selected Certificate Authentication Service object.

$auth_service->comment("new comment");

#Applying the changes
$session->modify($auth_service)
   or die("Modify Certificate Authentication Service object failed",
            $session->status_code() . ":" . $session->status_detail());
print "Certificate Authentication Service object modified successfully \n";
```

#Remove the Certificate Authentication Service object

```
#Get a Certificate Authentication Service object through session
my @retrieved_objs = $session->get(
    object => "Infoblox::Grid::Admin::CertificateAuthService",
    name   => "cas",
    );
my $desired_auth_service = $retrieved_objs[0];

unless ($desired_auth_service) {
    die("Get a Certificate Authentication Service object failed: ",
           $session->status_code() . ":" . $session->status_detail());
}
print "Get a Certificate Authentication Service object successful \n";

#Submit the object for removal
$session->remove($desired_auth_service)
    or die("Remove Certificate Authentication Service object failed",
           $session->status_code() . ":" . $session->status_detail());
print "Certificate Authentication Service object removed successfully \n";

####PROGRAM ENDS####
```

AUTHOR

Infoblox Inc. http://www.infoblox.com/

SEE ALSO

Infoblox::OCSP::Responder

COPYRIGHT

Copyright (c) 2017 Infoblox Inc.

Infoblox::Grid::Admin::Group - Admin Group object.

- NAME
- DESCRIPTION
- CONSTRUCTOR
- SESSION METHODS
  - Infoblox::Session->add( )
  - Infoblox::Session->get( )
  - Infoblox::Session->modify( )
  - Infoblox::Session->remove( )
  - Infoblox::Session->search( )
- METHODS
  - access_method( )
  - comment( )
  - disabled( )
  - enable_restricted_user_access( )
  - extattrs( )
  - extensible_attributes( )
  - email_addresses( )
  - name( )
  - superuser( )
  - roles( )
  - user_access( )
- SAMPLE CODE
- AUTHOR
- SEE ALSO
- COPYRIGHT

NAME

Infoblox::Grid::Admin::Group - Admin Group object.

DESCRIPTION

An Admin Group object creates and manages a local admin group on the Infoblox appliance. The privileges and properties that are set for the group apply to all the admin accounts that are assigned to the group.

CONSTRUCTOR

```
 my $group = Infoblox::Grid::Admin::Group->new(
     name                          => $string,
# Required
     access_method                 => ["GUI"|"API"|"TAXII",...],
# Optional / Default is ["GUI", "API", "TAXII"]
     comment                       => $string,
# Optional / Default is undefined
     disabled                      => "true" | "false",
# Optional / Default is "false"
     enable_restricted_user_access => "true" | "false",
# Optional / Default is "false"
     extattrs                      => { $string => $extattr, ... },
# Optional / Default is undefined
     extensible_attributes         => { $string => $string | $num, $string => [ $string | $num, ... ], ...
} # Optional / Default is undefined
     email_addresses               => [ 'admin@test.com',...],
# Optional / Default is undefined
     superuser                     => "true" | "false",
# Optional / Default is "false"
     roles                         => [$string, ...]
# Optional / Default is an empty array reference
     user_access                   => [$ipaddr, $netaddr, ...]
# Optional / Default is empty list
 );
```

You cannot set both extattrs and extensible_attributes attributes at the same time.

SESSION METHODS

This section describes all the methods in the Infoblox::Session module that you can apply to an Admin Group object.

Infoblox::Session->add( )

Use this method to add an object to the Infoblox appliance. See Infoblox::Session->add() for parameters and return values.

Example

```
#Construct the Group object
my $group = Infoblox::Grid::Admin::Group->new(
    name                => "testgroup",
    access_method       => ["GUI", "API"],
    comment             => "test group",
    disabled            => "true",
    superuser           => "false"
);
# Submit for addition
my $response = $session->add( $group );
```

Infoblox::Session->get( )

Use this method to retrieve all the matching objects from the Infoblox appliance. See Infoblox::Session->get() for parameters and return values.

Key References

```
Apply the following attributes to get a specific Group object:

 name      - Required. Group name in string format.
 extensible_attributes - Optional. A hash reference containing extensible attributes.
 extattrs  - Optional. A hash reference containing extensible attributes.
```

Example

```
my @retrieved_objs = $session->get(
    object     => "Infoblox::Grid::Admin::Group",
    name       => "testgroup" );
```

Infoblox::Session->modify( )

Use this method to modify an object in the Infoblox appliance. See Infoblox::Session->modify() for parameters and return values.

Example

```
# Use this method to modify the comment.
$group->comment("This is a modified comment");
# Submit modification
my $response = $session->modify( $group );
```

Infoblox::Session->remove( )

Use this method to remove an object from the Infoblox appliance. See Infoblox::Session->remove() for parameters and return values.

To remove a specific object, use `get()` or `search()` to retrieve the specific object first, and then submit this object for removal.

Example

```
# Get the group objects with the same starting name
my @retrieved_objs = $session->get(
    object     => "Infoblox::Grid::Admin::Group",
    name       => "testgroup" );
# find the desired object from the retrieved list.
my $desired_group = $retrieved_objs[0];
# Submit for removal
my $response = $session->remove( $desired_group );
```

Infoblox::Session->search( )

Use this method to search for Admin Group objects in the Infoblox appliance. See Infoblox::Session->search() for parameters and return values.

Key References

```
Apply the following attributes to search for a specific Group object:

 name                            - Optional. Group name in string format (regular expression).
 roles                           - Optional. An array of role names in string format.
 ca_certificate_issuer           - Optional. The name of the CA certificate issuer in string
format.
 client_certificate_serial_number - Optional. The serial number of the CA certificate in string
format. This is unique for a particular issuer.
 extensible_attributes           - Optional. A hash reference containing extensible attributes.
 extattrs                        - Optional. A hash reference containing extensible attributes.
```

For more information about searching extensible attributes, see Infoblox::Grid::ExtensibleAttributeDef/Searching Extensible Attributes.

Example

```
# search for all Group objects that start with "test"
my @retrieved_objs = $session->search(
    object => "Infoblox::Grid::Admin::Group",
    name   => "test.*" );

# search for all Group objects that start with "test"
my @retrieved_objs = $session->search(
    object => "Infoblox::Grid::Admin::Group",
    roles   => ["DHCP Admin", "AAA Admin"]);
```

METHODS

This section describes all the methods that you can use to set and retrieve the attribute values of a Group object.

access_method( )

Use this method to set or retrieve a group's access methods. Access methods specify whether a group can use the GUI and the API to access the appliance or to send Taxii messages to the appliance. Valid value is an array reference that contains the literal string "GUI", "TAXII" and/or "API". The default value is ["GUI", "API", "TAXII"].

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

Array reference of the literal string "GUI", "TAXII" and/or "API". It indicates whether a group has permission to access the appliance through the GUI and/or API and/or to send Taxii messages to the appliance.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get access methods
my $ref_access_method = $group->access_method();
#Modify access methods
$group->access_method(["GUI"]);
```

comment( )

Use this method to set or retrieve the comment.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

Pertinent information about the admin group, such as location or department. Comment in string format with a maximum of 256 bytes. The default value is undefined.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get comment
my $comment = $group->comment();
#Modify comment
$group->comment("This is the modified comment for admin group");
```

disabled( )

Use this method to set or retrieve the "disabled" flag.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

The default value for this field is false. The admin group is enabled.

Parameter

Specify "true" to set the disable flag or "false" to deactivate/unset it.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get disabled
my $disabled = $group->disabled();
#Modify disabled
$group->disabled("true");
```

enable_restricted_user_access( )

Use this method to set or retrieve the flag that indicates whether the restrictions will be applied on an admin connector level for a given user or not.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

Specify "true" to enable restricted access on admin connector level and "false" to disable restrictions. The default value is "false".

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get enable_restricted_user_access
my $enable_restricted_user_access = $group->enable_restricted_user_access();

#Modify enable_restricted_user_access
$group->enable_restricted_user_access("true");
```

extattrs( )

Use this method to set or retrieve the extensible attributes associated with a Group object.

Parameter

Valid value is a hash reference containing the names of extensible attributes and their associated values (Infoblox::Grid::Extattr objects).

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get extattrs
my $ref_extattrs = $group->extattrs();
#Modify extattrs
$group->extattrs({ 'Site' => $extattr1, 'Administrator' => $extattr2 });
```

extensible_attributes( )

Use this method to set or retrieve the extensible attributes associated with a Group object.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

For valid values for extensible attributes, see Infoblox::Grid::ExtensibleAttributeDef/Extensible Attribute Values.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get extensible attributes
my $ref_extensible_attributes = $group->extensible_attributes();
#Modify extensible attributes
$group->extensible_attributes({ 'Site' => 'Santa Clara', 'Administrator' => [ 'Peter', 'Tom' ] });
```

email_addresses( )

Use this method to set or retrieve the email addresses for the Admin Group.

Parameter

The valid value is an array reference that contains emails in string format.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get email_addresses
my @email_addresses = $rgroup->email_addresses();
#Modify addresses
$group->addresses( [ 'admin@test.com' ] );
```

name( )

Use this method to set or retrieve the name of an admin group.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

Text with the name of the admin group.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get name
my $name = $group->name();
```

superuser( )

Use this method to set or retrieve the "superuser" flag. A superuser group can perform all operations on the appliance, and can view and configure all types of data.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

The default value for this field is "false" which means that this is not a superuser admin group. The admins in the group have limited privileges.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get super user flag
my $disabled = $group->superuser();
#Modify super user
$group->superuser("true");
```

roles( )

Use this method to set or retrieve the role names associated with this admin group.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

The default value for this field is an empty array. A valid value is an array reference with literal strings containing valid role names. The corresponding role objects must exist in the database.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get roles
my $ref_roles = $group->roles();
#Modify roles
$group->roles(["role1", "role2"]);
#Remove roles
$group->roles([]);
```

user_access( )

Use this method to set or retrieve the access control items for a given group.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

Valid value is an Infoblox::Grid::NamedACL object or an array reference that contains IPv4/IPv6 addresses or networks.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get user_access
my $user_access = $group->user_access();

#Modify user_access
$group->user_access(['10.0.0.1', '2001:db8::/64']);
```

SAMPLE CODE

The following sample code demonstrates the different functions that can be applied to an object, such as add, search, modify, and remove. This sample also includes error handling for the operations.

#Preparation prior to an Admin Group object insertion

```
#PROGRAM STARTS: Include all the modules that will be used
use strict;
use Infoblox;

#Create a session to the Infoblox appliance

my $session = Infoblox::Session->new(
               master   => "192.168.1.2", #appliance host ip
               username => "admin",       #appliance user login
               password => "infoblox"     #appliance password
               );

unless ($session) {
       die("Construct session failed: ",
               Infoblox::status_code() . ":" . Infoblox::status_detail());
}
print "Session created successfully\n";

my $role = Infoblox::Grid::Admin::Role->new(
    name                 => "testrole",
    comment              => "test admin role",
);

unless ($role) {
       die("Construct role object failed: ",
```

```
                Infoblox::status_code() . ":" . Infoblox::status_detail());
}
print "Admin Role object created successfully\n";

#Add the Admin Role object to the Infoblox appliance through a session
$session->add($role)
       or die("Add Admin Role object failed: ",
                        $session->status_code() . ":" . $session->status_detail());
print "Admin Role object added to server successfully\n";
```

#Create an Admin Group object

```
my $group = Infoblox::Grid::Admin::Group->new(
    name                => "testgroup",
    comment             => "test admin group",
    disabled            => "false",
    superuser           => "false",
    access_method       => ["GUI", "API"],
    roles               => ["testrole"]
);

unless ($group) {
       die("Construct group object failed: ",
                Infoblox::status_code() . ":" . Infoblox::status_detail());
}
print "Admin Group object created successfully\n";

#Add the Admin Group object to the Infoblox appliance through a session
$session->add($group)
       or die("Add Admin Group object failed: ",
                        $session->status_code() . ":" . $session->status_detail());
print "Admin Group object added to server successfully\n";
```

#Search for an Admin Group

```
my @retrieved_objs = $session->search(
               object => "Infoblox::Grid::Admin::Group",
               name   => "test.*"
               );
my $object = $retrieved_objs[0];

unless ($object) {
       die("Search Admin Group object failed: ",
                $session->status_code() . ":" . $session->status_detail());
}
print "Search Admin Group object found at least 1 matching entry\n";
```

#Get and modify an Admin Group object

```
#Get the Admin Group object from Infoblox appliance through a session
my @retrieved_objs = $session->get(
    object => "Infoblox::Grid::Admin::Group",
    name   => "testgroup"
);
my $object = $retrieved_objs[0];

unless ($object) {
       die("Get Admin Group object failed: ",
                $session->status_code() . ":" . $session->status_detail());
}
print "Get Admin Group object found at least 1 matching entry\n";

#Modify the Admin Group object
$object->name("testgroup_renamed");
$object->access_method(["API"]);
$object->comment("modified test group");
$object->disabled("false");
$object->superuser("false"); # no real change

#Apply the changes.
$session->modify($object)
       or die("Modify Admin Group object failed: ",
                $session->status_code() . ":" . $session->status_detail());
print "Admin Group object modified successfully \n";
```

#Remove an Admin Group object

```
#Get the Admin Group object through the session
my @retrieved_objs = $session->get(
    object => "Infoblox::Grid::Admin::Group",
    name   => "testgroup_renamed"
);
my $object = $retrieved_objs[0];
unless ($object) {
    die("Get Admin Group object failed: ",
        $session->status_code() . ":" . $session->status_detail());
}
print "Get Admin Group object found at least 1 matching entry\n";

#Submit the object for removal
$session->remove($object)
       or die("Remove Admin Group object failed: ",
                $session->status_code() . ":" . $session->status_detail());
print "Admin Group object removed successfully \n";
```

#Clean up the Admin Role object

```
$session->remove($role)
       or die("Remove Admin Role object failed: ",
                $session->status_code() . ":" . $session->status_detail());
print "Admin Role object removed successfully \n";

####PROGRAM ENDS####
```

AUTHOR

Infoblox Inc. http://www.infoblox.com/

SEE ALSO

Infoblox::Grid::Admin::Role, Infoblox::Grid::Admin::User, Infoblox::Session->add(), Infoblox::Session->get(), Infoblox::Session->modify(), Infoblox::Session->remove(), Infoblox::Session->search(),Infoblox::Session

---

COPYRIGHT

Copyright (c) 2017 Infoblox Inc.

Infoblox::Grid::Admin::LocalUserAuthService - Local User Authentication Service object

- NAME
- DESCRIPTION
- MODULE METHODS
  - Infoblox::Grid::Admin::AuthPolicy->auth_services( )
- SESSION METHODS
  - Infoblox::Session->get( )
- METHODS
  - comment( )
  - disabled( )
  - name( )
- AUTHOR
- SEE ALSO
- COPYRIGHT

NAME

Infoblox::Grid::Admin::LocalUserAuthService - Local User Authentication Service object

DESCRIPTION

This object represents a local authentication service for authenticating users against the local database. All methods are read-only.

MODULE METHODS

The following functions can be applied to an Infoblox::Grid::Admin::LocalUserAuthService object.

Infoblox::Grid::Admin::AuthPolicy->auth_services( )

Use this function to set or retrieve a reference to an array of authentication services.

Example

```
#Retrieve the local user authentication service
my $local = $session->get(object => "Infoblox::Grid::Admin::LocalUserAuthService");

#Set it as the authentication policy
$auth_policy->auth_services([$local]);
```

SESSION METHODS

This section describes all the methods in the Infoblox::Session module that you can apply to a Local User Authentication Service object.

Infoblox::Session->get( )

Use this method to retrieve the local user authentication service from the Infoblox appliance. See Infoblox::Session->get() for parameters and return values.

Key Reference

No parameters are required.

Example

```
my $local_user_auth_service = $session->get(
    object => "Infoblox::Grid::Admin::LocalUserAuthService",
    );
```

METHODS

This section describes all the methods that you can use to retrieve the attribute values of a local user authentication service.

comment( )

Use this method to retrieve the descriptive comment.

Returns

The method returns the attribute value.

Example

```
#Get comment
my $comment = $local_user_auth_service->comment();
```

disabled( )

Use this method to retrieve the disabled flag for the local user authentication service.

Returns

The method returns the attribute value.

Example

```
#Get disabled
my $disabled = $local_user_auth_service->disabled();
```

name( )

Use this method to retrieve the name of the local user authentication service.

Returns

The method returns the attribute value.

Example

```
#Get name
my $name = $local_user_auth_service->name();
```

AUTHOR

Infoblox Inc. http://www.infoblox.com/

SEE ALSO

Infoblox::Grid::Admin::AuthPolicy/>, Infoblox::Session->get()

COPYRIGHT

Copyright (c) 2017 Infoblox Inc.

Infoblox::Grid::Admin::Permission - Admin Permission object.

- NAME
- DESCRIPTION
- CONSTRUCTOR
- SESSION METHODS
  - Infoblox::Session->add( )
  - Infoblox::Session->get( )
  - Infoblox::Session->modify( )
  - Infoblox::Session->remove( )
  - Infoblox::Session->search( )
- METHODS
  - admin_group( )
  - role( )
  - permission( )
  - resource_type( )
  - resource_object( )
- SAMPLE CODE
- AUTHOR
- SEE ALSO
- COPYRIGHT

NAME

Infoblox::Grid::Admin::Permission - Admin Permission object.

DESCRIPTION

An Admin Permission object is used to set the permissions for an admin to access various resources.

CONSTRUCTOR

```
 my $group = Infoblox::Grid::Admin::Permission->new(
     admin_group          => $string,                                    #exactly
one of admin_group and role Required
     role                 => $string,                                    #exactly
one of admin_group and role Required
     permission           => "read" | "write" | "deny",                  #Required
     resource_type        => $string,
#Optional, if resource_object is provided
     resource_object      => $Object,
#Optional, if resource_type is provided
 );
```

SESSION METHODS

This section describes all the methods in the Infoblox::Session module that you can apply to an Admin Permission object.

Infoblox::Session->add( )

Use this method to add an object to the Infoblox appliance. See Infoblox::Session->add() for parameters and return values.

Example

```
       #Construct the Group object
       my $permission = Infoblox::Grid::Admin::Permission->new(
                 admin_group => "test_group",
                 resource_type => "All Zones",
                 permission => "write",
       );
       # Submit for addition
       my $response = $session->add( $permission );
```

Infoblox::Session->get( )

Use this method to retrieve all the matching objects from the Infoblox appliance. See Infoblox::Session->get() for parameters and return values.

Key References

```
Apply the following attributes to get a specific Group object:

 admin_group      - Optional. Group name in string format.
 role             - Optional. Role name in string format.
 permission       - Optional. Permission in string format.
 resource_type    - Optional. Resource type in string format.
 resource_object  - Optional. Resource Object reference.

NOTE: You cannot specify both an admin_group and role.
```

Example

```
my @retrieved_objs = $session->get(
    object     => "Infoblox::Grid::Admin::Permission",
    admin_group   => "test_group",
    resource_type => "All Members");

or

my @retrieved_objs = $session->get(
    object     => "Infoblox::Grid::Admin::Permission",
    role          => "test_role",
    resource_type => "All Members");
```

Infoblox::Session->modify( )

Use this method to modify an object in the Infoblox appliance. See Infoblox::Session->modify() for parameters and return values.

Example

```
# Use this method to modify the permission.
$permission->permission("read");
# Submit modification
my $response = $session->modify( $permission );
```

Infoblox::Session->remove( )

Use this method to remove an object from the Infoblox appliance. See Infoblox::Session->remove() for parameters and return values.

To remove a specific object, use `get()` or `search()` to retrieve the specific object first, and then submit this object for removal.

Example

```
# Get the permission objects with the same group name
my @retrieved_objs = $session->get(
    object     => "Infoblox::Grid::Admin::Permission",
    admin_group   => "test_group",
    resource_type => "All Members");
# find the desired object from the retrieved list.
my $desired_perm = $retrieved_objs[0];
# Submit for removal
my $response = $session->remove( $desired_perm );

or

# Get the permission objects with the same role name
my @retrieved_objs = $session->get(
    object     => "Infoblox::Grid::Admin::Permission",
    role          => "test_role",
    resource_type => "All Members");
# find the desired object from the retrieved list.
my $desired_perm = $retrieved_objs[0];
# Submit for removal
my $response = $session->remove( $desired_perm );
```

Infoblox::Session->search( )

Use this method to search for Admin Permission objects in the Infoblox appliance. See Infoblox::Session->search() for parameters and return values.

Key References

```
Apply the following attributes to search for a specific Group object:

 admin_group          - Optional. Group name in string format.
 role                 - Optional. Role name in string format.
 permission           - Optional. Permission in string format.
 resource_object      - Optional. Resource Object reference.
 resource_type        - Optional. Resource Type in string format.

NOTE: You cannot specify both an admin group and role.
```

Example

```
# search for all permission objects of admin group "test_group"
my @retrieved_objs = $session->search(
    object => "Infoblox::Grid::Admin::Permission",
    admin_group   => "test_group" );

or

# search for all permission objects of admin role "test_role"
my @retrieved_objs = $session->search(
    object => "Infoblox::Grid::Admin::Permission",
    role   => "test_role" );
```

METHODS

This section describes all the methods that you can use to set and retrieve the attribute values of a Group object.

admin_group( )

Use this method to set or retrieve the admin group information.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

The admin group name for the specified object.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get admin group name
my $name = $permission->admin_group();
#Modify admin group
$permission->admin_group("test-group1");
```

role( )

Use this method to set or retrieve the role information.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

The admin role name for the specified object.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get admin role name
my $name = $permission->role();
#Modify admin role
$permission->role("test-role1");
```

permission( )

Use this method to set or retrieve the admin group permission.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

The admin group permission for the specified object. Permission values are "write", "read" and "deny".

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get permission
my $perm = $permission->permission();
#Modify permission
$permission->permission("deny");
```

resource_type( )

Use this method to set or retrieve the resource type. Resource type represents all resources of the specific type.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

The resource type for which permission is set. Sets permissions for all resources of the resource_type specified.

If resource_object is also provided, it sets the permission for all resources of the specified type under the resource_object.

The allowed values are:

```
"All Zones"
"All DNS Views"
"All DNS64 Synthesis Groups"
"All Members"
"All MS Server Members"
"All Microsoft Active Directory Domains"
"All Microsoft Superscopes"
"All Shared Record Groups"
"All Networks" (deprecated in favor of "All IPv4 Networks")
"All IPv4 Networks"
"All IPv6 Networks"
"All Network Views"
"All Shared Networks" (deprecated in favor of "All IPv4 DHCP Shared Networks")
"All IPv4 DHCP Shared Networks"
"All IPv6 DHCP Shared Networks"
"All Mac Address Filters"
"All Ranges" (deprecated in favor of "All IPv4 Ranges")
"All IPv4 DHCP Ranges" (deprecated in favor of "All IPv4 Ranges")
"All IPv4 Ranges"
"All IPv6 DHCP Ranges"  (deprecated in favor of "All IPv6 Ranges")
"All IPv6 Ranges"
"All Fixed Addresses/Reservations" (deprecated in favor of "All IPv4 DHCP Fixed
Addresses/Reservations")
"All IPv4 DHCP Fixed Addresses/Reservations"
"All IPv6 DHCP Fixed Addresses"
"All Roaming Hosts"
"All Rulesets"
"All Templates" (deprecated in favor of "All IPv4 DHCP Templates")
"All IPv4 DHCP Templates"
"All IPv6 DHCP Templates"
"All IPv4 Host Addresses"
"All IPv6 Host Addresses"
"All A Records"
"All AAAA Records"
"All Alias Records"
"All BULKHOST Records"
"All CNAME Records"
"All DNAME Records"
"All HOST Records"
"All MX Records"
"All NAPTR Records"
"All PTR Records"
"All SRV Records"
"All TLSA Records"
"All TXT Records"
"All Shared A Records"
"All Shared AAAA Records"
"All Shared MX Records"
"All Shared SRV Records"
"All Shared TXT Records"
"All Shared CNAME Records"
"DNS Scavenging"
"Grid DHCP properties"
"Grid DNS properties"
"Lease History Access" (deprecated in favor of "All IPv4 DHCP Lease History")
"All IPv4 DHCP Lease History"
"All IPv6 DHCP Lease History"
"Grid File Distribution Properties"
"Grid AAA Properties"
"All AAA External Services"
"Network Discovery"
"Restart"
"All HSM Groups"
"All Certificate Auth Services"
"All CA Certificates"
"All CA Certificates"
"All Response Policy Zones"
"All Response Policy Rules"
```

```
"All CSV Import Tasks"
"All Dashboard Tasks"
"All DHCP Fingerprints"
"All Named ACLs"
"All Kerberos Keys"
"All BFD Templates"
"Grid Security configuration"
"Port Control"
"All DTC LBDNs"
"All DTC LBDN Records"
"All DTC Pools"
"All DTC Servers"
"All DTC Monitors"
"All DTC Topologies"
"All DTC GeoIP"
"All DTC Certificates"
"Grid Reporting properties"
"Reporting dashboard"
"Reporting search"
"Allow adding A/AAAA records with empty hostname"
"Allow deleted objects information tracking"
"All Parental Control accounting AVPs"
"Parental Control Properties"
"All Parental Control Subscriber Sites"
"Scheduling"
```

The "Restart" resource_type is supported only when resource_object is an Infoblox::Grid::Member::DHCP or Infoblox::Grid::Member::DNS object.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get resource_type
my $resource_type = $permission->resource_type();
#Modify resource_type
$permission->resource_type("All Networks");
```

resource_object( )

Use this method to set or retrieve a resource object. Resource object is a reference to a single object.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

The resource object for which permission is set. Sets the permission for the specified resource object only.

If resource_type is also provided, it sets the permission for all resources of the specified type under the resource_object.

The value must be a reference of a defined object of any of the following types:

```
Infoblox::DHCP::Filter::MAC
Infoblox::DHCP::FixedAddr
Infoblox::DHCP::FixedAddrTemplate
Infoblox::DHCP::IPv6FixedAddr
Infoblox::DHCP::IPv6Network
Infoblox::DHCP::IPv6NetworkContainer
Infoblox::DHCP::IPv6Range
Infoblox::DHCP::IPv6SharedNetwork
Infoblox::DHCP::Network
Infoblox::DHCP::NetworkContainer
Infoblox::DHCP::NetworkTemplate
Infoblox::DHCP::Range
Infoblox::DHCP::RangeTemplate
Infoblox::DHCP::RoamingHost
Infoblox::DHCP::SharedNetwork
Infoblox::DHCP::View
Infoblox::DNS::BulkHost
Infoblox::DNS::Host
Infoblox::DNS::Record::A
Infoblox::DNS::Record::AAAA
Infoblox::DNS::Record::Alias
Infoblox::DNS::Record::CNAME
Infoblox::DNS::Record::DNAME
Infoblox::DNS::Record::MX
Infoblox::DNS::Record::PTR
Infoblox::DNS::Record::SRV
Infoblox::DNS::Record::TLSA
Infoblox::DNS::Record::TXT
Infoblox::DNS::Ruleset
Infoblox::DNS::SharedRecord::A
Infoblox::DNS::SharedRecord::AAAA
Infoblox::DNS::SharedRecord::MX
Infoblox::DNS::SharedRecord::SRV
Infoblox::DNS::SharedRecord::TXT
```

```
Infoblox::DNS::SharedRecord::CNAME
Infoblox::DNS::SRG
Infoblox::DNS::View
Infoblox::DNS::Zone
Infoblox::Grid::DNS::DNS64Group
Infoblox::Grid::FileDistributionDir
Infoblox::Grid::Member
Infoblox::Grid::Member::FileDistribution
Infoblox::Grid::MSServer
Infoblox::Grid::MSServer::AdSites::Domain
Infoblox::Grid::MSSuperscope
Infoblox::Grid::Member::ThreatProtection
Infoblox::Grid::Discovery::Device
```

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get resource_obejct
my $resource_object = $permission->resource_object();
#Modify resource_object
$object = Infoblox::DNS::Zone->new(
                name => "zone.com"
                );
$session->add($object);
$permission->resource_object($zone);
```

SAMPLE CODE

The following sample code demonstrates the different functions that can be applied to an object, such as add, search, modify, and remove. This sample also includes error handling for the operations.

#Preparation prior to an Admin Permission object insertion

```
#PROGRAM STARTS: Include all the modules that will be used
use strict;
use Infoblox;

#Create a session to the Infoblox appliance

my $session = Infoblox::Session->new(
               master   => "192.168.1.2", #appliance host ip
               username => "admin",       #appliance user login
               password => "infoblox"     #appliance password
               );

unless ($session) {
       die("Construct session failed: ",
               Infoblox::status_code() . ":" . Infoblox::status_detail());
}
print "Session created successfully\n";

#Create an admin group

my $group = Infoblox::Grid::Admin::Group->new(
    name                => "test_group",
    disabled            => "false",
    superuser           => "false"

);

unless ($group) {
       die("Construct user object failed: ",
               Infoblox::status_code() . ":" . Infoblox::status_detail());
}
print "Admin Group object created successfully\n";

#Add the Admin Group object to the Infoblox appliance through a session
$session->add($group)
       or die("Add Admin Group object failed: ",
                       $session->status_code() . ":" . $session->status_detail());
print "Admin Group object added to server successfully\n";

# Create an Admin Role

my $role = Infoblox::Grid::Admin::Role->new(
    name                => "test_role",
```

```
    comment              => "test admin role",
);

unless ($role) {
       die("Construct role object failed: ",
               Infoblox::status_code() . ":" . Infoblox::status_detail());
}
print "Admin Role object created successfully\n";

#Add the Admin Role object to the Infoblox appliance through a session
$session->add($role)
       or die("Add Admin Role object failed: ",
                       $session->status_code() . ":" . $session->status_detail());
print "Admin Role object added to server successfully\n";

#Create a zone
my $zone = Infoblox::DNS::Zone->new(
               name => "zone.com"
);

unless ($zone) {
       die("Construct user object failed: ",
               Infoblox::status_code() . ":" . Infoblox::status_detail());
}

#Add the Zone object to the Infoblox appliance through a session
$session->add($zone)
       or die("Add Zone object failed: ",
                       $session->status_code() . ":" . $session->status_detail());
print "Zone object added to server successfully\n";

#Create a shared record group (SRG)
my $srg = Infoblox::DNS::SRG->new(
               name => "mysrg"
);

unless ($srg) {
       die("Construct user object failed: ",
               Infoblox::status_code() . ":" . Infoblox::status_detail());
}

#Add the SRG object to the Infoblox appliance through a session
$session->add($srg)
       or die("Add SRG object failed: ",
                       $session->status_code() . ":" . $session->status_detail());
print "SRG object added to server successfully\n";
```

#Create an Admin Permission object based on Admin Group

```
#Create an admin permission object
my $group_permission = Infoblox::Grid::Admin::Permission->new(
   admin_group     => $group->name(),
   resource_object => $zone,  # This can be omitted if don't care about
                              # which zone
   resource_type => "All A Records",
   permission => "write");

unless ($group_permission) {
       die("Construct Admin Permission object failed: ",
               Infoblox::status_code() . ":" . Infoblox::status_detail());
}
print "Admin Permission object created successfully\n";

#Add the Admin Permission object to the Infoblox appliance through a session
$session->add($group_permission)
       or die("Add Admin Permission object failed: ",
                       $session->status_code() . ":" . $session->status_detail());
print "Admin Permission object added to server successfully\n";

#Create an admin permission object
my $scheduling_permission = Infoblox::Grid::Admin::Permission->new(
     admin_group => $group->name(),
     resource_type => "Scheduling",
     permission => "write");

unless ($scheduling_permission) {
       die("Construct Admin Permission object failed: ",
               Infoblox::status_code() . ":" . Infoblox::status_detail());
```

```
}
print "Admin Permission object created successfully\n";

#Add the Admin Permission object to the Infoblox appliance through a session
$session->add($scheduling_permission)
       or die("Add Admin Permission object failed: ",
                        $session->status_code() . ":" . $session->status_detail());
print "Admin Permission object added to server successfully\n";
```

#Create an Admin Permission object based on Admin Role

```
#Create an admin permission object
my $role_permission = Infoblox::Grid::Admin::Permission->new(
   role            => $role->name(),
   resource_object => $srg,  # This can be omitted if don't care about
                             # which shared record group
   resource_type => "All Shared A Records",
   permission => "write");

unless ($role_permission) {
       die("Construct user object failed: ",
               Infoblox::status_code() . ":" . Infoblox::status_detail());
}
print "Admin Permission object created successfully\n";

#Add the Admin Permission object to the Infoblox appliance through a session
$session->add($role_permission)
       or die("Add Admin Permission object failed: ",
                        $session->status_code() . ":" . $session->status_detail());
print "Admin Permission object added to server successfully\n";
```

#Search for an Admin Permission object based on Admin Group

```
my @retrieved_objs = $session->search(
               object => "Infoblox::Grid::Admin::Permission",
               admin_group   => "test_group"
               );
my $object = $retrieved_objs[0];

unless ($object) {
       die("Search Admin Permission object based on Admin Group failed: ",
               $session->status_code() . ":" . $session->status_detail());
}
print "Search Admin Permission object based on Admin Group found at least 1 matching entry\n";
```

#Search for an Admin Permission object based on Admin Role

```
my @retrieved_objs = $session->search(
               object => "Infoblox::Grid::Admin::Permission",
               role   => "test_role"
               );
my $object = $retrieved_objs[0];

unless ($object) {
       die("Search Admin Permission object based on Admin Role failed: ",
               $session->status_code() . ":" . $session->status_detail());
}
print "Search Admin Permission object based on Admin Role found at least 1 matching entry\n";
```

#Get and modify an Admin Permission object based on Admin Group

```
#Get the Admin Group object from the Infoblox appliance through a session
my @retrieved_objs = $session->get(
    object => "Infoblox::Grid::Admin::Permission",
    admin_group   => "test_group",
    resource_object => $zone,
    resource_type => "All A Records",
);
my $object = $retrieved_objs[0];

unless ($object) {
       die("Get Admin Permission object based on Admin Group failed: ",
               $session->status_code() . ":" . $session->status_detail());
}
print "Get Admin Permission object found at least 1 matching entry\n";

#Modify the permission of the Admin Permission object
```

```
$object->permission("read");

#Apply the changes.
$session->modify($object)
       or die("Modify Admin Permission object failed: ",
                $session->status_code() . ":" . $session->status_detail());
print "Admin Permission object modified successfully \n";
```

#Get Admin Permission object based on Admin Role

```
#Get the Admin Group object from the Infoblox appliance through a session
my @retrieved_objs = $session->get(
    object => "Infoblox::Grid::Admin::Permission",
    role             => "test_role",
    resource_object => $srg,
    resource_type => "All Shared A Records",
);
my $object = $retrieved_objs[0];

unless ($object) {
       die("Get Admin Permission object based on Admin Role failed: ",
                $session->status_code() . ":" . $session->status_detail());
}
print "Get Admin Permission object found at least 1 matching entry\n";
```

#Remove an Admin Permission object

```
#Get the Admin Permission object through the session
my @retrieved_objs = $session->get(
    object => "Infoblox::Grid::Admin::Permission",
    admin_group    => "test_group",
    resource_object => $zone,
    resource_type => "All A Records",
);
my $object = $retrieved_objs[0];
unless ($object) {
    die("Get Admin Permission object failed: ",
        $session->status_code() . ":" . $session->status_detail());
}
print "Get Admin Permission object found at least 1 matching entry\n";

#Submit the object for removal
$session->remove($object)
       or die("Remove Admin Permission object failed: ",
                $session->status_code() . ":" . $session->status_detail());
print "Admin Permission object removed successfully \n";

# We can remove the other permissions in the same way

#Get the Admin Permission object for Shared Resource Records through the session
@retrieved_objs = $session->get(
    object => "Infoblox::Grid::Admin::Permission",
    role    => "test_role",
    resource_object => $srg,    # Omit this if it was omitted during creation
    resource_type => "All Shared A Records",
);
$object = $retrieved_objs[0];
unless ($object) {
    die("Get Admin Permission object for Shared RR failed: ",
        $session->status_code() . ":" . $session->status_detail());
}
print "Get Admin Permission object found at least 1 matching entry\n";

#Submit the object for removal
$session->remove($object)
       or die("Remove Admin Permission object for Shared RR failed: ",
                $session->status_code() . ":" . $session->status_detail());
print "Admin Permission object for Shared RR removed successfully \n";
```

#Clean up

```
$session->remove($role)
       or die("Remove Admin Role object failed: ",
                $session->status_code() . ":" . $session->status_detail());
print "Admin Role object removed successfully \n";

$session->remove($group)
```

```
        or die("Remove Admin Group object failed: ",
                $session->status_code() . ":" . $session->status_detail());
print "Admin Group object removed successfully \n";

$session->remove($zone)
        or die("Remove Zone object failed: ",
                $session->status_code() . ":" . $session->status_detail());
print "Zone object removed successfully \n";

$session->remove($srg)
        or die("Remove Shared Record Group object failed: ",
                $session->status_code() . ":" . $session->status_detail());
print "Shared Record Group object removed successfully \n";

####PROGRAM ENDS####
```

AUTHOR

Infoblox Inc. http://www.infoblox.com/

SEE ALSO

Infoblox::Grid::Admin::Role, Infoblox::Grid::Admin::Group, Infoblox::Grid::Admin::User, Infoblox::Session->add(), Infoblox::Session->get(), Infoblox::Session->modify(), Infoblox::Session->remove(), Infoblox::Session->search(), Infoblox::Session

---

COPYRIGHT

Copyright (c) 2017 Infoblox Inc.

Infoblox::Grid::Admin::RadiusAuthServer - RADIUS Authentication Server object

- NAME
- DESCRIPTION
- CONSTRUCTOR
- SESSION METHODS
  - Infoblox::Grid::Admin::RadiusAuthService->radius_servers()
- METHODS
  - acct_enabled( )
  - acct_port( )
  - auth_port( )
  - auth_type( )
  - comment( )
  - disabled( )
  - fqdn_or_ip( )
  - shared_secret( )
  - use_mgmt_port( )
- AUTHOR
- SEE ALSO
- COPYRIGHT

NAME

Infoblox::Grid::Admin::RadiusAuthServer - RADIUS Authentication Server object

DESCRIPTION

Represents a RADIUS server that is used to authenticate administrators and DHCP clients.

CONSTRUCTOR

```
my $radius_auth_server = Infoblox::Grid::Admin::RadiusAuthServer->new(
    acct_enabled     => "true" | "false"      #Optional / Default is false
    acct_port        => $num,                 #Optional / Default is 1813
    auth_port        => $num,                 #Optional / Default is 1812
    auth_type        => "PAP" | "CHAP",       #Optional / Default is "PAP"
    comment          => $string,              #Optional / Default is empty
    disabled         => "true" | "false"      #Optional / Default is false
    fqdn_or_ip       => $fqdn | $ipv4addr     #Required
    shared_secret    => $string,              #Required
    use_mgmt_port    => "true" | "false"      #Optional / Default is false
);
```

SESSION METHODS

This section describes all the methods in an Infoblox::Session module that you can apply to an object.

Infoblox::Grid::Admin::RadiusAuthService->radius_servers()

Use this method to add a list of RADIUS Server objects to the RADIUS Service object on an Infoblox appliance. See Infoblox::Grid::Admin::RadiusAuthService for parameters and return values.

Example

```
# Construct an object
my $radius_auth_server1 = Infoblox::Grid::Admin::RadiusAuthServer->new(
                                                    fqdn_or_ip    => '10.0.1.1',
                                                    shared_secret => 'secret',
);

# Construct another object
my $radius_auth_server2 = Infoblox::Grid::Admin::RadiusAuthServer->new(
                                                    fqdn_or_ip    => '10.0.2.1',
                                                    shared_secret => 'secret',
);

# Configure RadiusAuthServer
my $response = $radius_auth_service->radius_servers([$radius_auth_server1, $radius_auth_server2]);
```

METHODS

Use the following methods to access the attributes of an Infoblox::Grid::Admin::RadiusAuthServer object. Specify a parameter to set the attribute's value, or omit the parameter to get the attribute's value.

acct_enabled( )

Use this method to set or retrieve the flag that indicates if the RADIUS server is enabled for accounting packets.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

Specify "true" to set the disable flag or "false" to deactivate/unset it. The default value is "false".

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get acct_enabled
my $acct_enabled = $radius_server->acct_enabled();
#Modify acct_enabled
$radius_server->acct_enabled("true");
```

acct_port( )

Use this method to set or retrieve the port number used by RADIUS server for accounting packets.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

An unsigned integer between 1 and 65535, inclusive.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Getting acct_port
my $acct_port = $radius_server->acct_port( );
#Modifying acct_port
$radius_server->acct_port("1813");
```

auth_port( )

Use this method to set or retrieve the port number used by the RADIUS server for accounting packets.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

An unsigned integer between 1 and 65535, inclusive.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Getting auth_port
my $auth_port = $radius_server->auth_port( );
#Modifying auth_port
$radius_server->auth_port("1812");
```

auth_type( )

Use this method to set or retrieve the authentication type used by the RADIUS client and server.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

The valid value is either "PAP" or "CHAP", the default is "PAP".

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Getting auth_type
my $auth_type = $radius_server->auth_type( );
#Modifying auth_type
$radius_server->auth_type("PAP");
```

comment( )

Use this method to set or retrieve a descriptive comment about the RADIUS server.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

The comment in string format, with a maximum of 256 characters.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get comment
my $comment = $radius_server->comment();
#Modify comment
$radius_server->comment("NAC Authentication RADIUS server");
```

disabled( )

Use this method to set or retrieve the disable flag of the RADIUS server.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

Specify "true" to set the disable flag or "false" to deactivate/unset it. The default value is "false".

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get disabled
my $disable = $radius_server->disabled();
#Modify disabled
$radius_server->disabled("true");
```

fqdn_or_ip( )

Use this method to set or retrieve the FQDN or IP address of the RADIUS server.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

A string in FQDN (Fully Qualified Domain Name) format, or an IP address.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get fqdn_or_ip
my $fqdn_or_ip = $radius_server->fqdn_or_ip();
#Modify fqdn_or_ip
$radius_server->fqdn_or_ip("10.0.0.1");
```

shared_secret( )

Use this method to set the shared secret used by the RADIUS client and RADIUS server. This is a write-only attribute.

Include the specified parameter to set the attribute value.

Parameter

The shared secret in string format, with a maximum of 128 characters.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

Example

```
#Modify shared_secret
$radius_server->shared_secret("secret");
```

use_mgmt_port( )

Use this method to set or retrieve the flag that indicates if the appliance connects to the RADIUS server through the MGMT network interface.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

Specify "true" to set the flag or "false" to deactivate/unset it. The default value is "false".

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get use_mgmt_port
my $use_mgmt_port = $radius_server->use_mgmt_port();
#Modify use_mgmt_port
$radius_server->use_mgmt_port("true");
```

AUTHOR

Infoblox Inc. http://www.infoblox.com

SEE ALSO

Infoblox::Grid::Admin::RadiusAuthService, Infoblox::Session, Infoblox::Session->add(), Infoblox::Session->get(), Infoblox::Session->modify(), Infoblox::Session->remove()

COPYRIGHT

Copyright (c) 2017 Infoblox Inc.

Infoblox::Grid::Admin::RadiusAuthService - RADIUS Authentication Service object

- NAME
- DESCRIPTION
- CONSTRUCTOR
- SESSION METHODS
  - Infoblox::Session->add( )
  - Infoblox::Session->get( )
  - Infoblox::Session->modify( )
  - Infoblox::Session->remove( )
- METHODS
  - acct_retries( )
  - acct_timeout( )
  - auth_retries( )
  - auth_timeout( )
  - cache_ttl( )
  - comment( )
  - disabled( )
  - enable_cache( )
  - name( )
  - selection_mode( )
  - radius_servers( )
  - recovery_interval( )
- SAMPLE CODE
- AUTHOR
- SEE ALSO
- COPYRIGHT

NAME

Infoblox::Grid::Admin::RadiusAuthService - RADIUS Authentication Service object

DESCRIPTION

Represents a group of RADIUS servers that are used to authenticate administrators and DHCP clients.

CONSTRUCTOR

```
my $radius_auth_server = Infoblox::Grid::Admin::RadiusAuthService->new(
    acct_retries          => $num,                                  #Optional / Default is 1000
    acct_timeout          => $num,                                  #Required
    auth_retries          => $num,                                  #Optional / Default is 5
    auth_timeout          => $num,                                  #Required
    cache_ttl             => $num,                                  #Optional / Default is 3600
    comment               => $string,                               #Optional / Default is empty
    disabled              => "true" | "false"                       #Optional / Default is false
    enable_cache          => "true" | "false"                       #Optional / Default is false
    name                  => $string,                               #Required
    radius_servers        => [$RadiusAuthServer1, ...],             #Required
    recovery_ttl          => $num,                                  #Optional / Default is 30
    selection_mode        => "HUNT_GROUP" | "ROUND_ROBIN",          #Optional / Default is "HUNT_GROUP"
);
```

SESSION METHODS

This section describes all the methods in the Infoblox::Session module that you can apply to a RADIUS Auth object.

Infoblox::Session->add( )

Use this method to add an object to the Infoblox appliance. See Infoblox::Session->add() for parameters and return values.

Example

```
#Construct an object
my $service = Infoblox::Grid::Admin::RadiusAuthService->new(
                                                             name           => 'some.name.com',
                                                             radius_servers => [$ads1, $ads2]
                                                            );
#Submit for addition
```

```
my $response = $session->add( $service );
```

Infoblox::Session->get( )

Use this method to retrieve all the matching objects from the Infoblox appliance. See Infoblox::Session->get() for parameters and return values.

Key References

```
Apply the following attributes to get a specific RADIUS Auth object:

 name - Required. RADIUS Authentication Service name.
```

Examples

```
my @retrieved_objs = $session->get(
    object => "Infoblox::Grid::Admin::RadiusAuthService",
    name   => "some.name.com"
);
```

Infoblox::Session->modify( )

Use this method to modify an object in the Infoblox appliance. See Infoblox::Session->modify() for parameters and return values.

Example

```
#Use this method to modify the comment.
$service->comment("this is a modified comment");
#Submit modification
my $response = $session->modify( $service );
```

Infoblox::Session->remove( )

Use this method to remove an object from the Infoblox appliance. See Infoblox::Session->remove() for parameters and return values.

To remove a specific object, first use `get()` or `search()` to retrieve the specific object, and then submit this object for removal.

Example

```
#Get the objects with the same name
my @retrieved_objs = $session->get(
    object => "Infoblox::Grid::Admin::RadiusAuthService",
    name   => "some.name.com"
);

#Find the desired object from the retrieved list.
my $desired_service = $retrieved_objs[0];
#Submit for removal
my $response = $session->remove( $desired_service );
```

METHODS

Use the following methods to access the attributes of an Infoblox::Grid::Admin::RadiusAuthService object. Specify a parameter to set the attribute's value, or omit the parameter to get the attribute's value.

acct_retries( )

Use this method to set or retrieve the value that specifies how many times the appliance must resend the RADIUS accounting packet if the appliance does not receive a response.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

An unsigned integer between 1 and 10, inclusive.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Getting acct_retries
my $acct_retries = $radius_service->acct_retries( );
#Modifying acct_retries
```

```
$radius_service->acct_retries("5");
```

acct_timeout( )

Use this method to set or retrieve the value, in milliseconds, that specifies how long the appliance must wait for authentication response packets.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

An unsigned integer between 1000 and 10000, inclusive.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Getting acct_timeout
my $acct_timeout = $radius_service->acct_timeout( );
#Modifying acct_timeout
$radius_service->acct_timeout("5000");
```

auth_retries( )

Use this method to set or retrieve the value of how many times to retry if there is no response for the RADIUS authentication packet.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

An unsigned integer between 1 and 10, inclusive.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Getting auth_retries
my $auth_retries = $radius_service->auth_retries( );
#Modifying auth_retries
$radius_service->auth_retries("5");
```

auth_timeout( )

Use this method to set or retrieve the value in milliseconds to wait for authentication response packets.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

An unsigned integer between 1000 and 10000, inclusive.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Getting auth_timeout
my $auth_timeout = $radius_service->auth_timeout( );
#Modifying auth_timeout
$radius_service->auth_timeout("5000");
```

cache_ttl( )

Use this method to set or retrieve the TTL for the authentication response in the authentication cache.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

An unsigned integer between 1 and 259200, inclusive.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Getting cache_ttl
my $cache_ttl = $radius_service->cache_ttl( );
#Modifying cache_ttl
$radius_service->cache_ttl("3600");
```

comment( )

Use this method to set or retrieve a descriptive comment about the RADIUS Authentication Service object.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

The comment in string format, with a maximum of 256 characters.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get comment
my $comment = $radius_service->comment( );
#Modify comment
$radius_service->comment("RADIUS Authentication Service for NAC authentication");
```

disabled( )

Use this method to set or retrieve the disable flag of the RADIUS Authentication Service object.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

Specify "true" to set the disable flag or "false" to deactivate/unset it. The default value is "false".

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get disable
my $disable = $radius_service->disabled( );
#Modify disabled
$radius_service->disabled("true");
```

enable_cache( )

Use this method to set or retrieve the flag that indicates if the authentication cache is enabled on the RADIUS Authentication Service object.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

Specify "true" to set the disable flag or "false" to deactivate/unset it. The default value is "false".

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get enable_cache
my $enable_cache = $radius_service->enable_cache( );
#Modify enable_cache
$radius_service->enable_cache("true");
```

name( )

Use this method to set or retrieve the name of the RADIUS Authentication Service object.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

The name in string format, with a maximum of 64 characters.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Getting name
my $name = $radius_service->name( );
#Modifying name
$radius_service->name("RADIUS Authentication Service");
```

selection_mode( )

Use this method to set or retrieve the process the appliance uses to select to which RADIUS server in the RADIUS Authentication Service it sends RADIUS requests.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

Valid values are "HUNT_GROUP" (ordered list) or "ROUND_ROBIN". The default is "HUNT_GROUP".

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Getting selection_mode
my $selection_mode = $radius_service->selection_mode( );
#Modifying selection_mode
$radius_service->selection_mode("HUNT_GROUP");
```

radius_servers( )

Use this method to set or retrieve a list of RADIUS servers assigned to this RADIUS Service.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

An array reference that contains a list of Infoblox::Grid::Admin::RadiusAuthServer objects.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Getting radius_servers
my $radius_servers = $radius_service->radius_servers( );
#Modifying radius_servers
$radius_service->radius_servers([$radius_server1]);
```

recovery_interval( )

Use this method to set or retrieve the time in seconds that the appliance waits before re-enabling a RADIUS server marked as DOWN.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

An unsigned integer between 1 and 600, inclusive.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Getting recovery_interval
my $recovery_interval = $radius_service->recovery_interval( );
#Modifying recovery_interval
$radius_service->recovery_interval("30");
```

SAMPLE CODE

The following sample code demonstrates the different functions that can be applied to an object, such as add, search, modify, and remove. This sample also includes error handling for the operations.

#Preparation prior to a Radius Auth Service object insertion

```
#PROGRAM STARTS: Include all the modules that will be used
use strict;
use Infoblox;

#Create a session to the Infoblox appliance
my $host_ip =  "192.168.1.2";
my $session = Infoblox::Session->new(
    master   => $host_ip,
    username => "admin",
    password => "infoblox"
    );
unless($session){
        die("Constructor for session failed: ",
               Infoblox::status_code(). ":" . Infoblox::status_detail());
}
print "Session created successfully.\n";

my $ads1 = Infoblox::Grid::Admin::RadiusAuthServer->new(
                                                        comment  => 'server 1',
                                                        fqdn_or_ip => '10.0.1.1',
                                                        shared_secret => 'secret1',
                                                       );

my $ads2 = Infoblox::Grid::Admin::RadiusAuthServer->new(
                                                        comment  => 'server 2',
                                                        fqdn_or_ip => 'domain.com',
                                                        shared_secret => 'secret2',
                                                      );
```

#Create the RADIUS Auth Service.

```
my $service = Infoblox::Grid::Admin::RadiusAuthService->new(
                                                            acct_retries   => 2000,
                                                            auth_retries   => 9,
                                                            cache_ttl      => 500,
                                                            disabled       => 'true',
                                                            enable_cache   => 'true',
                                                            recovery_ttl   => 50,
                                                            selection_mode => 'ROUND_ROBIN',
                                                            acct_timeout   => 1100,
                                                            auth_timeout   => 1200,
                                                            name           => 'some.name.com',
                                                            radius_servers => [$ads1, $ads2]
                                                           );

unless($service){
       die("Construct service object failed: ",
               Infoblox::status_code(). ":" .Infoblox::status_detail());
```

```
        }
print "test.com zone object created successfully.\n";

$session->add($service)
    or die("Add zone service failed: ",
                $session->status_code(). ":" .$session->status_detail());

print"Service added successfully.\n";
```

#Get a specific service

```
my @search_result = $session->get(
    object => "Infoblox::Grid::Admin::RadiusAuthService",
    name   => "some.name.com"
);

my $search_obj = $search_result[0];
unless($search_obj){
        die("Search service failed: ",
                 $session->status_code() . ":" . $session->status_detail());
        }

print "Search service object found at least 1 matching entry\n";
```

#Get and modify a RADIUS service

```
$search_obj->comment("this is a modified comment.");

#Apply the change
    $session->modify($search_obj)
        or die("Modify service failed: ",
                $session->status_code(). ":" .$session->status_detail());

 print "Service object modified successfully.\n";
```

#Remove a Service

```
my @search_result = $session->get(
    object => "Infoblox::Grid::Admin::RadiusAuthService",
    name   => "some.name.com"
);

my $search_obj = $search_result[0];
unless($search_obj){
        die("Search service failed: ",
                 $session->status_code() . ":" . $session->status_detail());
        }

print "Search service object found at least 1 matching entry\n";

$session->remove($search_obj)
    or die("Remove service failed: ",
              $session->status_code(). ":" .$session->status_detail());

print "Remove service successfull\n";

####PROGRAM ENDS####
```

AUTHOR

Infoblox Inc. http://www.infoblox.com

SEE ALSO

Infoblox::Grid::Admin::RadiusAuthServer, Infoblox::Session, Infoblox::Session->add(), Infoblox::Session->get(), Infoblox::Session->modify(), Infoblox::Session->remove()

COPYRIGHT

Copyright (c) 2017 Infoblox Inc.

Infoblox::Grid::Admin::Role - Admin Role object.

- NAME
- DESCRIPTION
- CONSTRUCTOR
- SESSION METHODS
  - Infoblox::Session->add( )
  - Infoblox::Session->get( )
  - Infoblox::Session->modify( )
  - Infoblox::Session->remove( )
  - Infoblox::Session->search( )
- METHODS
  - comment( )
  - disabled( )
  - extattrs( )
  - extensible_attributes( )
  - name( )
- SAMPLE CODE
- AUTHOR
- SEE ALSO
- COPYRIGHT

NAME

Infoblox::Grid::Admin::Role - Admin Role object.

DESCRIPTION

An Admin Role object creates and manages a local admin role on the Infoblox appliance. A Role object is used to aggregate a set of permissions (represented by Permission objects).

CONSTRUCTOR

```
 my $role = Infoblox::Grid::Admin::Role->new(
     name                  => $string,
#Required
     comment               => $string,
#Optional / Default is undefined
     disabled              => "true" | "false",
#Optional / Default is "false"
     extattrs              => { $string => $extattr, ... },
#Optional / Default is undefined
     extensible_attributes => { $string => $string | $num, $string => [ $string | $num, ... ], ... }
#Optional / Default is undefined
 );
```

You cannot set both extattrs and extensible_attributes attributes at the same time.

SESSION METHODS

This section describes all the methods in Infoblox::Session module that you can apply to an Admin Role object.

Infoblox::Session->add( )

Use this method to add an object to the Infoblox appliance. See Infoblox::Session->add() for parameters and return values.

Example

```
#Construct the Role object
my $role = Infoblox::Grid::Admin::Role->new(
    name                  => "testrole",
    comment               => "test role",
);
# Submit for addition
my $response = $session->add( $role );
```

Infoblox::Session->get( )

Use this method to retrieve all the matching objects from the Infoblox appliance. See Infoblox::Session->get() for parameters and return values.

Key References

```
Apply the following attributes to get a specific Role object:

 name      - Required. Role name in string format.
 extattrs  - Optional. A hash reference containing extensible attributes.
 extensible_attributes - Optional. A hash reference containing extensible attributes.
```

Example

```
my @retrieved_objs = $session->get(
    object     => "Infoblox::Grid::Admin::Role",
    name       => "testrole" );
```

Infoblox::Session->modify( )

Use this method to modify an object in the Infoblox appliance. See Infoblox::Session->modify() for parameters and return values.

Example

```
# Use this method to modify the comment.
$role->comment("This is a modified comment");
# Submit modification
my $response = $session->modify( $role );
```

Infoblox::Session->remove( )

Use this method to remove an object from the Infoblox appliance. See Infoblox::Session->remove() for parameters and return values.

To remove a specific object, use `get()` or `search()` to retrieve the specific object first, and then submit this object for removal.

Example

```
# Get the role objects with the same starting name
my @retrieved_objs = $session->get(
    object     => "Infoblox::Grid::Admin::Role",
    name       => "testrole" );
# find the desired object from the retrieved list.
my $desired_role = $retrieved_objs[0];
# Submit for removal
my $response = $session->remove( $desired_role );
```

Infoblox::Session->search( )

Use this method to search for Admin Role objects in the Infoblox appliance. See Infoblox::Session->search() for parameters and return values.

Key References

```
Apply the following attributes to search for a specific Role object:

 name      - Required. Role name in string format (regular expression).
 extattrs  - Optional. A hash reference containing extensible attributes.
 extensible_attributes - Optional. A hash reference containing extensible attributes.
```

For more information about searching extensible attributes, see Infoblox::Grid::ExtensibleAttributeDef/Searching Extensible Attributes.

Example

```
# search for all Role objects that start with "test"
my @retrieved_objs = $session->search(
    object => "Infoblox::Grid::Admin::Role",
    name   => "test.*" );
```

METHODS

This section describes all the methods that you can use to set and retrieve the attribute values of a Role object.

comment( )

Use this method to set or retrieve the comment.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

Pertinent information about the admin role, such as its purpose. Comment in string format with a maximum of 256 bytes. The

default value is undefined.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get comment
my $comment = $role->comment();
#Modify comment
$role->comment("This is the modified comment for admin role");
```

disabled( )

Use this method to set or retrieve the "disabled" flag.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

The default value for this field is false. The role is enabled.

Parameter

Specify "true" to set the disable flag or "false" to deactivate/unset it.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get disabled
my $disabled = $role->disabled();
#Modify disabled
$role->disabled("true");
```

extattrs( )

Use this method to set or retrieve the extensible attributes associated with a Role object.

Parameter

Valid value is a hash reference containing the names of extensible attributes and their associated values (Infoblox::Grid::Extattr objects).

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get extattrs
my $ref_extattrs = $role->extattrs();
#Modify extattrs
$role->extattrs({ 'Site' => $extattr1, 'Administrator' => $extattr2 });
```

extensible_attributes( )

Use this method to set or retrieve the extensible attributes associated with a Role object.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

For valid values for extensible attributes, see Infoblox::Grid::ExtensibleAttributeDef/Extensible Attribute Values.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get extensible attributes
my $ref_extensible_attributes = $role->extensible_attributes();
#Modify extensible attributes
$role->extensible_attributes({ 'Site' => 'Santa Clara', 'Administrator' => [ 'Peter', 'Tom' ] });
```

name( )

Use this method to set or retrieve the name of an admin role.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

Text with the name of the admin role with a maximum of 64 characters.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get name
my $name = $role->name();
```

SAMPLE CODE

The following sample code demonstrates the different functions that can be applied to an object, such as add, search, modify, and remove. This sample also includes error handling for the operations.

#Preparation prior to an Admin Role object insertion

```
#PROGRAM STARTS: Include all the modules that will be used
use strict;
use Infoblox;

#Create a session to the Infoblox appliance

my $session = Infoblox::Session->new(
               master   => "192.168.1.2", #appliance host ip
               username => "admin",       #appliance user login
               password => "infoblox"     #appliance password
               );

unless ($session) {
       die("Construct session failed: ",
               Infoblox::status_code() . ":" . Infoblox::status_detail());
}
print "Session created successfully\n";
```

#Create an Admin Role object

```
my $role = Infoblox::Grid::Admin::Role->new(
    name                => "testrole",
    comment             => "test admin role",
);

unless ($role) {
       die("Construct role object failed: ",
               Infoblox::status_code() . ":" . Infoblox::status_detail());
}
print "Admin Role object created successfully\n";

#Add the Admin Role object to the Infoblox appliance through a session
$session->add($role)
       or die("Add Admin Role object failed: ",
                       $session->status_code() . ":" . $session->status_detail());
print "Admin Role object added to server successfully\n";
```

#Search for an Admin Role

```
my @retrieved_objs = $session->search(
               object => "Infoblox::Grid::Admin::Role",
               name   => "test.*"
               );
```

```
my $object = $retrieved_objs[0];

unless ($object) {
       die("Search Admin Role object failed: ",
               $session->status_code() . ":" . $session->status_detail());
}
print "Search Admin Role object found at least 1 matching entry\n";
```

#Get and modify an Admin Role object

```
#Get the Admin Role object from Infoblox appliance through a session
my @retrieved_objs = $session->get(
    object => "Infoblox::Grid::Admin::Role",
    name   => "testrole"
);
my $object = $retrieved_objs[0];

unless ($object) {
       die("Get Admin Role object failed: ",
               $session->status_code() . ":" . $session->status_detail());
}
print "Get Admin Role object found at least 1 matching entry\n";

#Modify the Admin Role object
$object->name("testrole_renamed");
$object->comment("this is a new comment");

#Apply the changes.
$session->modify($object)
       or die("Modify Admin Role object failed: ",
               $session->status_code() . ":" . $session->status_detail());
print "Admin Role object modified successfully \n";

#Accessor methods
print "name: " . $object->name() . "\n";
print "comment: " . $object->comment() . "\n";
```

#Remove an Admin Role object

```
#Get the Admin Role object through the session
my @retrieved_objs = $session->get(
    object => "Infoblox::Grid::Admin::Role",
    name   => "testrole_renamed"
);
my $object = $retrieved_objs[0];
unless ($object) {
    die("Get Admin Role object failed: ",
        $session->status_code() . ":" . $session->status_detail());
}
print "Get Admin Role object found at least 1 matching entry\n";

#Submit the object for removal
$session->remove($object)
       or die("Remove Admin Role object failed: ",
               $session->status_code() . ":" . $session->status_detail());
print "Admin Role object removed successfully \n";

####PROGRAM ENDS####
```

AUTHOR

Infoblox Inc. http://www.infoblox.com/

SEE ALSO

Infoblox::Grid::Admin::Group, Infoblox::Grid::Admin::Permission, Infoblox::Grid::Admin::User, Infoblox::Session->add(), Infoblox::Session->get(), Infoblox::Session->modify(), Infoblox::Session->remove(), Infoblox::Session->search(),Infoblox::Session

---

COPYRIGHT

Copyright (c) 2017 Infoblox Inc.

Infoblox::Grid::Admin::TACACSPlusAuthServer - TACACS+ Authentication Server object

- NAME
- DESCRIPTION
- CONSTRUCTOR
- SESSION METHODS
  - Infoblox::Grid::Admin::TACACSPlusAuthService->tacacsplus_servers()
- METHODS
  - acct_enabled( )
  - auth_type( )
  - comment( )
  - disabled( )
  - fqdn_or_ip( )
  - port( )
  - shared_secret( )
  - use_mgmt_port( )
- AUTHOR
- SEE ALSO
- COPYRIGHT

NAME

Infoblox::Grid::Admin::TACACSPlusAuthServer - TACACS+ Authentication Server object

---

DESCRIPTION

Represents a TACACS+ server that is used to authenticate administrators.

CONSTRUCTOR

```
my $tacacs_auth_server = Infoblox::Grid::Admin::TACACSPlusAuthServer->new(
    fqdn_or_ip       => $fqdn | $ipv4addr         #Required
    shared_secret    => $string,                  #Required
    acct_enabled     => "true" | "false"          #Optional / Default is false
    auth_type        => "PAP" | "CHAP" | "ASCII", #Optional / Default is "CHAP"
    comment          => $string,                  #Optional / Default is empty
    disabled         => "true" | "false"          #Optional / Default is false
    use_mgmt_port    => "true" | "false"          #Optional / Default is false
    port             => $num,                     #Optional / Default is 49
);
```

SESSION METHODS

This section describes all the methods in an Infoblox::Session module that you can apply to an object.

Infoblox::Grid::Admin::TACACSPlusAuthService->tacacsplus_servers()

Use this method to add a list of TACACS+ Server objects to the TACACS+ Service object on an Infoblox appliance. See Infoblox::Grid::Admin::TACACSPlusAuthService for parameters and return values.

Example

```
# Construct an object
my $tacacs_auth_server1 = Infoblox::Grid::Admin::TACACSPlusAuthServer->new(
                                                    fqdn_or_ip    => '10.0.1.1',
                                                    shared_secret => 'secret',
);

# Construct another object
my $tacacs_auth_server2 = Infoblox::Grid::Admin::TACACSPlusAuthServer->new(
                                                    fqdn_or_ip    => '10.0.2.1',
                                                    shared_secret => 'secret',
);

# Configure TACACSPlusAuthServer
my $response = $tacacs_auth_service->tacacs_servers([$tacacs_auth_server1, $tacacs_auth_server2]);
```

METHODS

Use the following methods to access the attributes of an Infoblox::Grid::Admin::TACACSPlusAuthServer object. Specify a parameter to set the attribute's value, or omit the parameter to get the attribute's value.

acct_enabled( )

Use this method to set or retrieve the flag that indicates if the TACACS+ server is enabled for accounting packets.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

Specify "true" to set the disable flag or "false" to deactivate/unset it. The default value is "false".

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get acct_enabled
my $acct_enabled = $tacacs_server->acct_enabled();
#Modify acct_enabled
$tacacs_server->acct_enabled("true");
```

auth_type( )

Use this method to set or retrieve the authentication type used by the TACACS+ client and server.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

The valid value is either "PAP", "ASCII" or "CHAP". The default is "CHAP".

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get auth_type
my $auth_type = $tacacs_server->auth_type( );
#Modifying auth_type
$tacacs_server->auth_type("PAP");
```

comment( )

Use this method to set or retrieve a descriptive comment about the TACACS+ server.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

The comment in string format, with a maximum of 256 characters.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get comment
my $comment = $tacacs_server->comment();
#Modify comment
$tacacs_server->comment("NAC Authentication TACACS+ server");
```

disabled( )

Use this method to set or retrieve the disable flag of the TACACS+ server.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

Specify "true" to set the disable flag or "false" to deactivate/unset it. The default value is "false".

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get disabled
my $disable = $tacacs_server->disabled();
#Modify disabled
$tacacs_server->disabled("true");
```

fqdn_or_ip( )

Use this method to set or retrieve the FQDN or IP address of the TACACS+ server.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

A string in FQDN (Fully Qualified Domain Name) format, or an IP address.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get fqdn_or_ip
my $fqdn_or_ip = $tacacs_server->fqdn_or_ip();
#Modify fqdn_or_ip
$tacacs_server->fqdn_or_ip("10.0.0.1");
```

port( )

Use this method to set or retrieve the destination port number on the TACACS+ server.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

An unsigned integer between 1 and 65535, inclusive.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get port
my $port = $tacacs_server->port( );
#Modifying port
$tacacs_server->port("49");
```

shared_secret( )

Use this method to set the shared secret that the NIOS appliance and the TACACS+ server use to encrypt and decrypt their messages. This is a write-only attribute.

Include the specified parameter to set the attribute value.

Parameter

The shared secret in string format, with a maximum of 64 characters.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

Example

```
#Modify shared_secret
$tacacs_server->shared_secret("secret");
```

use_mgmt_port( )

Use this method to set or retrieve the flag that indicates if the appliance connects to the TACACS+ server through the MGMT network interface.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

Specify "true" to set the flag or "false" to deactivate/unset it. The default value is "false".

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get use_mgmt_port
my $use_mgmt_port = $tacacs_server_server->use_mgmt_port();
#Modify use_mgmt_port
$tacacs_server->use_mgmt_port("true");
```

AUTHOR

Infoblox Inc. http://www.infoblox.com

SEE ALSO

Infoblox::Grid::Admin::TACACSPlusAuthService, Infoblox::Session, Infoblox::Session->add(), Infoblox::Session->get(), Infoblox::Session->modify(), Infoblox::Session->remove()

COPYRIGHT

Copyright (c) 2017 Infoblox Inc.

Infoblox::Grid::Admin::TACACSPlusAuthService - TACACS+ Authentication Service object

- NAME
- DESCRIPTION
- CONSTRUCTOR
- SESSION METHODS
  - Infoblox::Session->add( )
  - Infoblox::Session->get( )
  - Infoblox::Session->modify( )
  - Infoblox::Session->remove( )
- METHODS
  - acct_retries( )
  - acct_timeout( )
  - auth_retries( )
  - auth_timeout( )
  - comment( )
  - disabled( )
  - name( )
  - tacacsplus_servers( )
- SAMPLE CODE
- AUTHOR
- SEE ALSO
- COPYRIGHT

NAME

Infoblox::Grid::Admin::TACACSPlusAuthService - TACACS+ Authentication Service object

---

DESCRIPTION

Represents a group of TACACS+ servers that are used to authenticate administrators.

CONSTRUCTOR

```
my $tacacs_auth_server = Infoblox::Grid::Admin::TACACSPlusAuthService->new(
    name                => $string,                             #Required
    tacacsplus_servers  => [$TACACSPlusAuthServer1, ...],       #Required
    acct_retries        => $num,                                #Optional / Default is 0
    acct_timeout        => $num,                                #Optional / Default is 1000
    auth_retries        => $num,                                #Optional / Default is 0
    auth_timeout        => $num,                                #Optional / Default is 5000
    comment             => $string,                             #Optional / Default is empty
    disabled            => "true" | "false"                     #Optional / Default is false
);
```

SESSION METHODS

This section describes all the methods in the Infoblox::Session module that you can apply to a TACACS+ Authentication Service object.

Infoblox::Session->add( )

Use this method to add an object to the Infoblox appliance. See Infoblox::Session->add() for parameters and return values.

Example

```
#Construct an object
my $service = Infoblox::Grid::Admin::TACACSPlusAuthService->new(
                                                    name           => 'some.name.com',
                                                    tacacsplus_servers => [$tcs1, $tcs2]
                                                   );
#Submit for addition
my $response = $session->add( $service );
```

Infoblox::Session->get( )

Use this method to retrieve all the matching objects from the Infoblox appliance. See Infoblox::Session->get() for parameters and return values.

Key References

```
Apply the following attributes to get a specific TACACS+ Authentication Service object:

 name - Required. TACACS+ Authentication Service name.
```

Examples

```
my @retrieved_objs = $session->get(
    object => "Infoblox::Grid::Admin::TACACSPlusAuthService",
    name   => "some.name.com"
);
```

Infoblox::Session->modify( )

Use this method to modify an object in the Infoblox appliance. See Infoblox::Session->modify() for parameters and return values.

Example

```
#Use this method to modify the comment.
$service->comment("this is a modified comment");
#Submit modification
my $response = $session->modify( $service );
```

Infoblox::Session->remove( )

Use this method to remove an object from the Infoblox appliance. See Infoblox::Session->remove() for parameters and return values.

To remove a specific object, first use `get()` or `search()` to retrieve the specific object, and then submit this object for removal.

Example

```
#Get the objects with the same name
my @retrieved_objs = $session->get(
    object => "Infoblox::Grid::Admin::TACACSPlusAuthService",
    name   => "some.name.com"
);

#Find the desired object on the retrieved list.
my $desired_service = $retrieved_objs[0];
#Submit for removal
my $response = $session->remove( $desired_service );
```

METHODS

Use the following methods to access the attributes of an Infoblox::Grid::Admin::TACACSPlusAuthService object. Specify a parameter to set the attribute's value, or omit the parameter to get the attribute's value.

acct_retries( )

Use this method to set or retrieve the value that specifies how many times the appliance must resend the TACACS+ accounting packet if the appliance does not receive a response.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

An unsigned integer between 0 and 5, inclusive.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Getting acct_retries
my $acct_retries = $tacacsplus_service->acct_retries( );
#Modifying acct_retries
$tacacsplus_service->acct_retries("5");
```

acct_timeout( )

Use this method to set or retrieve the value, in milliseconds, that specifies how long the appliance must wait for accounting response packets.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

An unsigned integer between 1000 and 30000, inclusive.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get acct_timeout
my $acct_timeout = $tacacsplus_service->acct_timeout( );
#Modify acct_timeout
$tacacsplus_service->acct_timeout("5000");
```

auth_retries( )

Use this method to set or retrieve the number of times NIOS retries authentication and authorization if there is no response or a server error occurrs.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

An unsigned integer between 0 and 5, inclusive.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get auth_retries
my $auth_retries = $tacacsplus_service->auth_retries( );
#Modify auth_retries
$tacacsplus_service->auth_retries("5");
```

auth_timeout( )

Use this method to set or retrieve the number of milliseconds NIOS waits to complete admin authentication and authorization by the TACACS+ server.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

An unsigned integer between 5000 and 60000, inclusive.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get auth_timeout
my $auth_timeout = $tacacsplus_service->auth_timeout( );
#Modify auth_timeout
$tacacsplus_service->auth_timeout("5000");
```

comment( )

Use this method to set or retrieve a descriptive comment about the TACACS+ Authentication Service object.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

The comment in string format, with a maximum of 256 characters.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get comment
my $comment = $tacacsplus_service->comment( );
#Modify comment
$tacacsplus_service->comment("TACACS+ Authentication Service for NAC authentication");
```

disabled( )

Use this method to set or retrieve the disable flag of the TACACS+ Authentication Service object.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

Specify "true" to set the disable flag or "false" to deactivate/unset it. The default value is "false".

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get disable
my $disable = $tacacsplus_service->disabled( );
#Modify disabled
$tacacsplus_service->disabled("true");
```

name( )

Use this method to set or retrieve the name of the TACACS+ Authentication Service object.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

The name in string format, with a maximum of 64 characters.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get name
my $name = $tacacsplus_service->name( );
#Modify name
$tacacsplus_service->name("TACACS+ Authentication Service");
```

tacacsplus_servers( )

Use this method to set or retrieve a list of TACACS+ servers assigned to this TACACS+ service.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

An array reference that contains a list of Infoblox::Grid::Admin::TACACSPlusAuthServer objects.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get tacacsplus_servers
my $tacacsplus_servers = $tacacsplus_service->tacacsplus_servers( );
#Modify tacacsplus_servers
$tacacsplus_service->tacacsplus_servers([$tacacsplus_server1]);
```

SAMPLE CODE

The following sample code demonstrates the different functions that can be applied to an object, such as add, search, modify, and remove. This sample also includes error handling for the operations.

#Preparation prior to a TACACS+ Authentication Service object insertion

```
#PROGRAM STARTS: Include all the modules that will be used
use strict;
use Infoblox;

#Create a session to the Infoblox appliance
my $host_ip =  "192.168.1.2";
my $session = Infoblox::Session->new(
    master   => $host_ip,
    username => "admin",
    password => "infoblox"
    );
unless($session){
        die("Constructor for session failed: ",
               Infoblox::status_code(). ":" . Infoblox::status_detail());
}
print "Session created successfully.\n";

my $tcs1 = Infoblox::Grid::Admin::TACACSPlusAuthServer->new(
                                                        comment  => 'server 1',
                                                        fqdn_or_ip => '10.0.1.1',
                                                        shared_secret => 'secret1',

auth_type => 'ASCII',
                                                      );

my $tcs2 = Infoblox::Grid::Admin::TACACSPlusAuthServer->new(
                                                        comment  => 'server 2',
                                                        fqdn_or_ip => 'domain.com',
                                                        shared_secret => 'secret2',

auth_type => 'ASCII',
                                                     );
```

#Create the TACACS+ Authentication Service.

```
my $service = Infoblox::Grid::Admin::TACACSPlusAuthService->new(
                                                            acct_retries   => 1,
                                                            auth_retries   => 1,
                                                            disabled       => 'true',
                                                            acct_timeout   => 1100,
                                                            auth_timeout   => 5000,
                                                            name           => 'some.name.com',
                                                            tacacsplus_servers => [$tcs1, $tcs2]
                                                          );

unless($service){
       die("Construct service object failed: ",
               Infoblox::status_code(). ":" .Infoblox::status_detail());
       }
print "Service object created successfully.\n";

$session->add($service)
    or die("Add TACACS+ service failed: ",
               $session->status_code(). ":" .$session->status_detail());

print"Service added successfully.\n";
```

#Get a specific service.

```
my @search_result = $session->get(
    object => "Infoblox::Grid::Admin::TACACSPlusAuthService",
    name   => "some.name.com"
);

my $search_obj = $search_result[0];
unless($search_obj){
       die("Search service failed: ",
                $session->status_code() . ":" . $session->status_detail());
       }
```

```
print "Search service object found at least 1 matching entry\n";
```

#Get and modify a TACACS+ service.

```
$search_obj->comment("this is a modified comment.");

#Apply the change
    $session->modify($search_obj)
        or die("Modify service failed: ",
               $session->status_code(). ":" .$session->status_detail());

 print "Service object modified successfully.\n";
```

#Remove a service.

```
my @search_result = $session->get(
    object => "Infoblox::Grid::Admin::TACACSPlusAuthService",
    name   => "some.name.com"
);

my $search_obj = $search_result[0];
unless($search_obj){
       die("Search service failed: ",
                $session->status_code() . ":" . $session->status_detail());
       }

print "Search service object found at least 1 matching entry\n";

$session->remove($search_obj)
    or die("Remove service failed: ",
            $session->status_code(). ":" .$session->status_detail());

print "Remove service successfull\n";

####PROGRAM ENDS####
```

AUTHOR

Infoblox Inc. http://www.infoblox.com

SEE ALSO

Infoblox::Grid::Admin::TACACSPlusAuthServer, Infoblox::Session, Infoblox::Session->add(), Infoblox::Session->get(), Infoblox::Session->modify(), Infoblox::Session->remove()

COPYRIGHT

Copyright (c) 2017 Infoblox Inc.

Infoblox::Grid::Admin::User - User Admin object.

- NAME
- DESCRIPTION
- CONSTRUCTOR
- SESSION METHODS
  - Infoblox::Session->add( )
  - Infoblox::Session->get( )
  - Infoblox::Session->modify( )
  - Infoblox::Session->remove( )
  - Infoblox::Session->search( )
- METHODS
  - admin_group( )
  - auth_type( )
  - ca_certificate_issuer( )
  - client_certificate_serial_number( )
  - comment( )
  - disabled( )
  - email( )
  - enable_certificate_authentication( )
  - extattrs( )
  - extensible_attributes( )
  - name( )
  - password( )
  - roles( )
  - time_zone( )
- SAMPLE CODE
- AUTHOR
- SEE ALSO
- COPYRIGHT

NAME

Infoblox::Grid::Admin::User - User Admin object.

DESCRIPTION

An admin account provides access to the Infoblox appliance. An admin account inherits the privileges and properties of the group to which it belongs.

CONSTRUCTOR

```
 my $user = Infoblox::Grid::Admin::User->new(
     admin_group                       => $string,
# Required
     auth_type                         => 'LOCAL' | 'REMOTE',
# Optional / Default is 'LOCAL'
     name                              => $string,
# Required
     password                          => $string,
# Required
     ca_certificate_issuer             => $string,
# Optional / Default is undefined
     client_certificate_serial_number  => $string,
# Optional / Default is undefined
     comment                           => $string,
# Optional / Default is undefined
     disabled                          => "true" | "false",
# Optional / Default is "false"
     enable_certificate_authentication => "true" | "false",
# Optional / Default is undefined
     extattrs                          => { $string => $extattr, ... },
# Optional / Default is undefined
     extensible_attributes             => { $string => $string | $num, $string => [ $string | $num, ... ],
... } # Optional / Default is undefined
     email                             => $email,
# Optional / Default is undefined
     time_zone                         => $TimeZone | undef
# Optional / Default value is undefined
 );
```

You cannot set both extattrs and extensible_attributes attributes at the same time.

SESSION METHODS

This section describes all the methods in the Infoblox::Session module that you can apply to an Admin User object.

Infoblox::Session->add( )

Use this method to add an object to the Infoblox appliance. See Infoblox::Session->add() for parameters and return values.

Example

```
#Construct the user object
my $user = Infoblox::Grid::Admin::User->new(
    admin_group => "admin-group",
    name        => "testuser",
    password    => "infoblox",
    comment     => "test user account"
    disabled    => "true",
    email       => "testuser\@test.com",
);
# Submit for addition
my $response = $session->add( $user );
```

Infoblox::Session->get( )

Use this method to retrieve all the matching objects from the Infoblox appliance. See Infoblox::Session->get() for parameters and return values.

Key References

```
Apply the following attributes to get a specific User object:

 name                            - Optional. Optional. The name of the user in string format.
 extattrs                        - Optional. A hash reference containing extensible attributes.
 extensible_attributes           - Optional. A hash reference containing extensible attributes.
 role                            - Optional. A role in string format or undef.
 ca_certificate_issuer           - Optional. The name of the CA certificate issuer in string
format.
 client_certificate_serial_number - Optional. The serial number of the client certificate in string
format.
```

When searching by role, admins assigned to the specified role, regardless of group, will be retrieved. If undef is passed as a get parameter for the role attribute, users that have no assigned roles, regardless of admin group, will be retrieved.

Note that name and role cannot be specified at the same time.

Example

```
my @retrieved_objs = $session->get(
    object      => "Infoblox::Grid::Admin::User",
    name        => "testuser" );
```

Infoblox::Session->modify( )

Use this method to modify an object in the Infoblox appliance. See Infoblox::Session->modify() for parameters and return values.

Example

```
# Use method to modify the comment.
$user->comment("This is a modified comment");
# Submit modification
my $response = $session->modify( $user );
```

Infoblox::Session->remove( )

Use this method to remove an object from the Infoblox appliance. See Infoblox::Session->remove() for parameters and return values.

To remove a specific object, use `get()` or `search()` to retrieve the specific object first, and then submit this object for removal.

Example

```
# Get the user objects with the same starting name
my @retrieved_objs = $session->get(
    object      => "Infoblox::Grid::Admin::User",
    name        => "testuser" );
# find the desired object from the retrieved list.
my $desired_user = $retrieved_objs[0];
# Submit for removal
my $response = $session->remove( $desired_user );
```

Infoblox::Session->search( )

Use this method to search for Admin User objects in the Infoblox appliance. See Infoblox::Session->search() for parameters and return values.

Key References

```
 Apply the following attributes to search for a specific User object:

  name                              - Optional. The name of the user in string format (regular
expression).
  admin_group                       - Optional. The name of the admin group in string format.
  extattrs                          - Optional. A hash reference containing extensible attributes.
  extensible_attributes             - Optional. A hash reference containing extensible attributes.
  role                              - Optional. A role in string format or undef.
  ca_certificate_issuer             - Optional. The name of the CA certificate issuer in string
format.
  client_certificate_serial_number - Optional. The serial number of the client certificate in string
format (regular expression).
```

When searching by role, the matching users will be users member of a group that has the specified role. If undef is passed as a search parameter for role, matching users will be ones belonging to groups with no roles assigned.

Note that name and role cannot be specified at the same time.

For more information about searching extensible attributes, see Infoblox::Grid::ExtensibleAttributeDef/Searching Extensible Attributes.

Example

```
# search for all User objects that start with "test"
my @retrieved_objs = $session->search(
    object      => "Infoblox::Grid::Admin::User",
    name        => "test.*",
    admin_group => "admin-group" );
```

METHODS

This section describes all the methods that you can use to set and retrieve the attribute values of a User object.

admin_group( )

Use this method to set or retrieve the admin_group.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

Admin group to which the user belongs. A user can belong to only one admin group at a time.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get admin group
my $admin_group = $user->admin_group();
#Modify name
$user->name("admin-group");
```

auth_type( )

Use this method to set or retrieve the user authentication type.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

The valid values are 'LOCAL' and 'REMOTE'. The default value is 'LOCAL'.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get auth_type value
my $auth_type = $user->auth_type();
#Modify auth_type value
$user->auth_type('REMOTE');
```

ca_certificate_issuer( )

Use this method to set or retrieve the name of the CA certificate issuer that is used for user lookup during authentication.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

The name of the CA certificate issuer that is used for user lookup during authentication in string format.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get ca_certificate_issuer
my $ca_certificate_issuer = $user->ca_certificate_issuer();

#Modify ca_certificate_issuer
$user->ca_certificate_issuer("issuer");
```

client_certificate_serial_number( )

Use this method to set or retrieve a parameter that is used in pair with the name of the CA certificate issuer.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

The parameter in string format that is used in pair with the name of the CA certificate issuer. Serial number can be either integer or hex. Serial number is unique for a particular issuer.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get client_certificate_serial_number
my $client_certificate_serial_number = $user->client_certificate_serial_number();

#Modify client_certificate_serial_number
$user->client_certificate_serial_number("11223344");
```

comment( )

Use this method to set or retrieve a comment.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

Pertinent information about the administrator, such as location or department. Comment in string format with a maximum of 256 bytes.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get comment
my $comment = $user->comment();
#Modify comment
$user->comment("This is the modified comment for admin user");
```

disabled( )

Use this method to set or retrieve the "disabled" flag.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

The default value for this field is false. The admin is enabled.

Parameter

Specify "true" to set the disable flag or "false" to deactivate/unset it. The default value is "false".

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get disabled
my $disabled = $user->disabled();
#Modify disabled
$user->disabled("true");
```

email( )

Use this method to set or retrieve the e-mail address of an administrator.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

The e-mail address of an administrator. The default value is undefined.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get email
my $email = $user->email();
#Modify email
$user->email("usertest\@infoblox.com");
```

enable_certificate_authentication( )

Use this method to set or retrieve the flag that indicates whether the user is allowed to log in only with the certificate. Regular username/password authentication will be disabled for this user.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

Specify 'true' to enable certificate-only user authentification and 'false' to disable it. The default value is 'false'.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get enable_certificate_authentication
my $value = $object->enable_certificate_authentication();

#Modify enable_certificate_authentication
$object->enable_certificate_authentication('true');
```

extattrs( )

Use this method to set or retrieve the extensible attributes associated with an User object.

Parameter

Valid value is a hash reference containing the names of extensible attributes and their associated values (Infoblox::Grid::Extattr objects).

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get extattrs
my $ref_extattrs = $user->extattrs();
#Modify extattrs
$user->extattrs({ 'Site' => $extattr1, 'Administrator' => $extattr2 });
```

extensible_attributes( )

Use this method to set or retrieve the extensible attributes associated with a User object.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

For valid values for extensible attributes, see Infoblox::Grid::ExtensibleAttributeDef/Extensible Attribute Values.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get extensible attributes
my $ref_extensible_attributes = $user->extensible_attributes();
#Modify extensible attributes
$user->extensible_attributes({ 'Site' => 'Santa Clara', 'Administrator' => [ 'Peter', 'Tom' ] });
```

name( )

Use this method to set or retrieve the name of an administrator.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

Text with the name of the admin.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get name
my $name = $user->name();
#Modify name
$user->name("test_user");
```

password( )

Use this method to set the password of an administrator. This is a write-only attribute.

Parameter

Password for the administrator to use when logging in.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

Example

```
#Modify password
$user->password("infobloxone");
```

roles( )

Use this method to retrieve the list of roles assigned to the admin's group.

Parameter

None

Returns

The method returns a reference to an array of strings of the role names.

Example

```
#Get the roles
my $roles = $user->roles();
```

time_zone( )

Use this method to set or retrieve the time zone of the administrator.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

The UTC string that represents the time zone. For example "(UTC - 6:00) Central Time (US and Canada)". Default value is undefined.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get time zone
my $time_zone = $user->time_zone();
#Modify time zone
$user->time_zone("(UTC - 6:00) Central Time (US and Canada)");
#Undef time zone
$user->time_zone(undef);
```

SAMPLE CODE

The following sample code demonstrates the different functions that can be applied to an object, such as add, search, modify, and remove. This sample also includes error handling for the operations.

#Preparation prior to an Admin User object insertion

```
#PROGRAM STARTS: Include all the modules that will be used
use strict;
use Infoblox;

#Create a session to the Infoblox appliance

my $session = Infoblox::Session->new(
              master   => "192.168.1.2", #appliance host ip
              username => "admin",       #appliance user login
              password => "infoblox"     #appliance password
              );

unless ($session) {
       die("Construct session failed: ",
              Infoblox::status_code() . ":" . Infoblox::status_detail());
}
print "Session created successfully\n";
```

#Create an Admin User object

```
my $user = Infoblox::Grid::Admin::User->new(
    name        => "testuser",
    password    => "infoblox",
    admin_group => "admin-group",
    email       => "testuser\@test.com",
    disabled    => "false",
```

```
    comment     => "test user account"
);

unless ($user) {
       die("Construct user object failed: ",
               Infoblox::status_code() . ":" . Infoblox::status_detail());
}
print "User object created successfully\n";

#Add the user object to the Infoblox appliance through a session
$session->add($user)
       or die("Add User object failed: ",
                       $session->status_code() . ":" . $session->status_detail());
print "Admin User object added to server successfully\n";
```

#Search for all users

```
my @retrieved_objs = $session->search(
               object      => "Infoblox::Grid::Admin::User",
               name        => "test.*",
               admin_group => "admin-group"
             );
my $object = $retrieved_objs[0];

unless ($object) {
       die("Search User object failed: ",
               $session->status_code() . ":" . $session->status_detail());
}
print "Search Admin User object found at least 1 matching entry\n";
```

#Get and modify a User object

```
#Get the user object from Infoblox appliance through a session
my @retrieved_objs = $session->get(
            object      => "Infoblox::Grid::Admin::User",
            name        => "testuser",
);
my $object = $retrieved_objs[0];

unless ($object) {
       die("Get User object failed: ",
               $session->status_code() . ":" . $session->status_detail());
}
print "Get Admin User object found at least 1 matching entry\n";

#Modify the password of the User object
$object->password("infoblox123");

#Apply the changes.
$session->modify($object)
       or die("Modify User object failed: ",
               $session->status_code() . ":" . $session->status_detail());
print "User object modified successfully \n";
```

#Remove the User object

```
#Get the User object through the session
my @retrieved_objs = $session->get(
            object      => "Infoblox::Grid::Admin::User",
            name        => "testuser",
);
my $object = $retrieved_objs[0];
unless ($object) {
    die("Get User object failed: ",
        $session->status_code() . ":" . $session->status_detail());
}
print "Get User object found at least 1 matching entry\n";

#Submit the object for removal
$session->remove($object)
       or die("Remove User object failed: ",
               $session->status_code() . ":" . $session->status_detail());
print "Admin User object removed successfully \n";

####PROGRAM ENDS####
```

Infoblox::Grid::Admin::User - User Admin object.

AUTHOR

Infoblox Inc. http://www.infoblox.com/

SEE ALSO

Infoblox::Grid::Admin::Group,Infoblox::Session->add(), Infoblox::Session->get(), Infoblox::Session->modify(), Infoblox::Session->remove(), Infoblox::Session->search(), Infoblox::Session

---

COPYRIGHT

Copyright (c) 2017 Infoblox Inc.

Infoblox::Grid::Admin::User::Profile - User Profile object.

- NAME
- DESCRIPTION
- SESSION METHODS
  - Infoblox::Session->get( )
- METHODS
  - admin_group( )
  - days_to_expire( )
  - email( )
  - global_search_on_ea( )
  - last_login( )
  - old_password( )
  - password( )
  - table_size( )
  - time_zone( )
  - user_type( )
- SAMPLE CODE
- AUTHOR
- SEE ALSO
- COPYRIGHT

NAME

Infoblox::Grid::Admin::User::Profile - User Profile object.

DESCRIPTION

Object that represents the user profile of the admin that is logged in.

SESSION METHODS

This section describes all the methods in the Infoblox::Session module that you can apply to a User Profile object.

Infoblox::Session->get( )

Use this method to retrieve an object from the Infoblox appliance. See Infoblox::Session->get() for parameters and return values.

Example

```
my $user_profile = $session->get(object => "Infoblox::Grid::Admin::User::Profile");
```

METHODS

This section describes all the methods that you can use to set and retrieve the attribute values of a User Profile object.

admin_group( )

Use this method to retrieve the Admin Group object to which the admin belongs. An admin user can belong to only one admin group at a time. This attribute is read-only and cannot be modified.

Parameter

None

Returns

The method returns the attribute value as Infoblox::Grid::Admin::Group object.

Example

```
#Get admin_group
my $admin_group = $user_profile->admin_group();
```

days_to_expire( )

Use this method to retrieve the number of days left before the admin's password expires. This attribute is read-only and cannot be modified.

Parameter

None

Returns

The method returns the attribute value. -1 means that the password doesn't expire.

Example

```
#Get days_to_expire
my $days_to_expire = $user_profile->days_to_expire();
```

email( )

Use this method to set or retrieve the email address of the admin.

Parameter

String with the email address of the admin.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get email
my $email = $user_profile->email();
#Modify email
$user_profile->email("usertest\@infoblox.com");
```

global_search_on_ea( )

Use this method to set or retrieve the flag that indicates whether extensible attribute values will be returned by global search or not.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

Specify 'true' to set global_search_on_ea flag or 'false' to unset it.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get global_search_on_ea
my $global_search_on_ea = $user_profile->global_search_on_ea();

#Modify global_search_on_ea
$user_profile->global_search_on_ea('true');
```

last_login( )

Use this method to retrieve the timestamp of when the admin last logged in. This attribute is read-only and cannot be modified.

Parameter

None

Returns

The method returns the timestamp.

Example

```
#Get last_login
my $last_login = $user_profile->last_login();
```

old_password( )

Use this method to specify the current password that will be replaced by a new password. To change a password in the database, you must provide both the current and new password values. This is a write-only attribute.

Parameter

Current password that the admin uses to log in.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

Example

```
#Modify password
$user_profile->old_password("infoblox");
```

password( )

Use this method to set the new password of the admin. To change a password in the database, you must provide both the current and new password values. This is a write-only attribute.

Parameter

New password of the admin.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

Example

```
#Modify password
$user_profile->password("infobloxone");
```

table_size( )

Use this method to set or retrieve the number of lines of data a table or a single list view can contain.

Parameter

The number of lines, which can be from 10 to 256.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get table_size
my $table_size = $user_profile->table_size();
#Modify table_size
$user_profile->table_size(20);
```

time_zone( )

Use this method to set or retrieve the time zone of the admin user.

Parameter

The UTC string that represents the time zone. For example "(UTC - 6:00) Central Time (US and Canada)".

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get time zone
my $time_zone = $user_profile->time_zone();
#Modify time zone
$user_profile->time_zone("(UTC - 6:00) Central Time (US and Canada)");
```

user_type( )

Use this method to retrieve the admin type. This attribute is read-only and cannot be modified.

Parameter

None

Returns

The method returns the attribute value as "LOCAL" or "REMOTE".

Example

```
#Get user_type
my $user_type = $user_profile->user_type();
```

SAMPLE CODE

The following sample code demonstrates the different functions that can be applied to an object. This sample also includes error handling for the operations.

#Preparation prior to a User Profile object insertion

```
#PROGRAM STARTS: Include all the modules that will be used
use strict;
use Infoblox;

#Create a session to the Infoblox appliance

my $session = Infoblox::Session->new(
               master   => "192.168.1.2", #appliance host ip
               username => "admin",       #appliance user login
               password => "infoblox"     #appliance password
               );

unless ($session) {
       die("Construct session failed: ",
               Infoblox::status_code() . ":" . Infoblox::status_detail());
}
print "Session created successfully\n";
```

#Get a User Profile object

```
#Get the user profile object from Infoblox appliance through a session
my $user_profile = $session->get(object => "Infoblox::Grid::Admin::User::Profile);


unless ($user_profile) {
       die("Get User Profile object failed: ",
               $session->status_code() . ":" . $session->status_detail());
}
####PROGRAM ENDS####
```

AUTHOR

Infoblox Inc. http://www.infoblox.com/

SEE ALSO

Infoblox::Grid::Admin::Group,Infoblox::Session->get(),Infoblox::Session

---

COPYRIGHT

Copyright (c) 2017 Infoblox Inc.

Infoblox::Grid::AllEndpoints - A class that represents all Grid endpoints.

- NAME
- DESCRIPTION
- SESSION METHODS
  - Infoblox::Session->get( )
  - Infoblox::Session->search( )
- METHODS
  - address( )
  - comment( )
  - disable( )
  - subscribing_member( )
  - type( )
  - version( )
- AUTHOR
- SEE ALSO
- COPYRIGHT

NAME

Infoblox::Grid::AllEndpoints - A class that represents all Grid endpoints.

DESCRIPTION

The Grid endpoint object provides information about all third party servers.

SESSION METHODS

This section describes all the methods in an Infoblox::Session module that you can apply to object.

Infoblox::Session->get( )

Use this method to retrieve all the matching objects from the Infoblox appliance. See Infoblox::Session->get() for parameters and return values.

Key References

```
Apply the following attributes to get a specific Grid endpoint object:

 address            - Optional. The FQDN or IP address of the Grid endpoint.
 comment            - Optional. The Grid endpoint comment in string format.
 subscribing_member - Optional. The Grid endpoint subscribing member name.
 type               - Optional. The Grid endpoint type.
```

Examples

```
my @retrieved_objs = $session->get(
    object  => 'Infoblox::Grid::AllEndpoints',
    address => '10.0.0.1',
);
```

Infoblox::Session->search( )

Use this method to search for objects in the Infoblox appliance. See Infoblox::Session->search() for parameters and return values.

Key References

```
Apply the following attributes to search for the object:

 address            - Optional. The FQDN or IP address of the Grid endpoint (regexp).
 comment            - Optional. The Grid endpoint comment in string format (regexp).
 subscribing_member - Optional. The Grid endpoint subscribing member name (exact).
 type               - Optional. The Grid endpoint type (exact).
```

Examples

```
# search for objects
my @retrieved_objs = $session->search(
    object       => 'Infoblox::Grid::AllEndpoints',
    address      => '10.*',
    comment      => '.*comment',
);
```

METHODS

This section describes all the methods that you can use to configure and retrieve the attribute values of a Grid endpoint.

address( )

Use this method to retrieve the fully-qualified domain name (FQDN) or IP address of the Grid endpoint. This is a read-only attribute.

Omit the parameter to retrieve the attribute value.

Parameter

None

Returns

The method returns the attribute value.

Example

```
#Get address value
my $address = $object->address();
```

comment( )

Use this method to retrieve the Grid endpoint comment. This is a read-only attribute.

Omit the parameter to retrieve the attribute value.

Parameter

None

Returns

The method returns the attribute value.

Example

```
#Get comment value
my $comment = $object->comment();
```

disable( )

Use this method to retrieve the flag that indicates whether the Grid endpoint is enabled or disabled. This is a read-only attribute.

Omit the parameter to retrieve the attribute value.

Parameter

None

Returns

The method returns the attribute value.

Example

```
#Get disable value
my $disable = $object->disable();
```

subscribing_member( )

Use this method to retrieve the Grid member host name serving the Grid endpoint. This is a read-only attribute.

Omit the parameter to retrieve the attribute value.

Parameter

None

Returns

The method returns the attribute value.

Example

```
#Get subscribing_member value
my $subscribing_member = $object->subscribing_member();
```

type( )

Use this method to retrieve the Grid endpoint type. This is a read-only attribute.

Omit the parameter to retrieve the attribute value.

Parameter

None

Returns

The method returns the attribute value.

Example

```
#Get type value
my $type = $object->type();
```

version( )

Use this method to retrieve the Grid endpoint version. This is a read-only attribute.

Omit the parameter to retrieve the attribute value.

Parameter

None

Returns

The method returns the attribute value.

Example

```
#Get version value
my $version = $object->version();
```

AUTHOR

Infoblox Inc. http://www.infoblox.com/

SEE ALSO

Infoblox::Session, Infoblox::Session->get(), Infoblox::Session->search(),

---

COPYRIGHT

Copyright (c) 2017 Infoblox Inc.

Infoblox::Grid::ApprovalWorkflow - An approval object.

- NAME
- DESCRIPTION
- CONSTRUCTOR
- SESSION METHODS
  - Infoblox::Session->add( )
  - Infoblox::Session->get( )
  - Infoblox::Session->modify( )
  - Infoblox::Session->remove( )
  - Infoblox::Session->search( )
- METHODS
  - approval_group( )
  - submitter_group( )
  - approval_notify_enabled( )
  - approval_notify_to( )
  - approved_notify_enabled( )
  - approved_notify_to( )
  - approver_comment_used( )
  - enable_notify_group( )
  - enable_notify_user( )
  - failed_notify_enabled( )
  - failed_notify_to( )
  - rejected_notify_enabled( )
  - rejected_notify_to( )
  - rescheduled_notify_enabled( )
  - rescheduled_notify_to( )
  - submitter_comment_used( )
  - succeeded_notify_enabled( )
  - succeeded_notify_to( )
  - ticket_number_used( )
- SAMPLE CODE
- AUTHOR
- SEE ALSO
- COPYRIGHT

NAME

Infoblox::Grid::ApprovalWorkflow - An approval object.

DESCRIPTION

The approval object.

CONSTRUCTOR

```
my $approval = Infoblox::Grid::ApprovalWorkflow->new(
    'submitter_group'           => $string,                                 # Required
    'approval_group'            => $string,                                 # Required
    'ticket_number_used'        => 'UNUSED' | 'OPTIONAL' | 'REQUIRED',      # Optional, Default is 'UNUSED'
    'submitter_comment_used'    => 'UNUSED' | 'OPTIONAL' | 'REQUIRED',      # Optional, Default is 'UNUSED'
    'approver_comment_used'     => 'UNUSED' | 'OPTIONAL' | 'REQUIRED',      # Optional, Default is 'UNUSED'
    'enable_notify_user'        => 'true' | 'false',                        # Optional, Default is 'false'
    'enable_notify_group'       => 'true' | 'false',                        # Optional, Default is 'true'
    'approval_notify_enable'    => 'true' | 'false',                        # Optional, Default is 'true'
    'approval_notify_to'        => 'SUBMITTER' | 'APPROVER' | 'BOTH',       # Optional, Default is 'BOTH'
    'approved_notify_enable'    => 'true' | 'false',                        # Optional, Default is 'true'
    'approved_notify_to'        => 'SUBMITTER' | 'APPROVER' | 'BOTH',       # Optional, Default is 'BOTH'
    'rejected_notify_enable'    => 'true' | 'false',                        # Optional, Default is 'true'
    'rejected_notify_to'        => 'SUBMITTER' | 'APPROVER' | 'BOTH',       # Optional, Default is 'BOTH'
    'succeeded_notify_enable'   => 'true' | 'false',                        # Optional, Default is 'true'
    'succeeded_notify_to'       => 'SUBMITTER' | 'APPROVER' | 'BOTH',       # Optional, Default is 'BOTH'
    'failed_notify_enable'      => 'true' | 'false',                        # Optional, Default is 'true'
    'failed_notify_to'          => 'SUBMITTER' | 'APPROVER' | 'BOTH',       # Optional, Default is 'BOTH'
    'rescheduled_notify_enable' => 'true' | 'false',                        # Optional, Default is 'true'
    'rescheduled_notify_to'     => 'SUBMITTER' | 'APPROVER' | 'BOTH',       # Optional, Default is 'BOTH'
);
```

SESSION METHODS

This section describes all the methods in an Infoblox::Session module that you can apply to an approval object.

Infoblox::Session->add( )

Use this method to add an object to the Infoblox appliance. See Infoblox::Session->add() for parameters and return values.

Example

```
#Construct an object
my $approval = Infoblox::Grid::ApprovalWorkflow->new(
   submitter_group => 'user-group',
   approval_group  => 'admin-group',
);

#Submit for addition
my $response = $session->add( $approval );
```

Infoblox::Session->get( )

Use this method to retrieve all the matching objects from the Infoblox appliance. See Infoblox::Session->get() for parameters and return values.

Key References

```
Apply the following attributes to get a specific approval object:

 submitter_group   - Optional. The submitter group name.
 approval_group    - Optional. The approval group name.
```

Example

```
my @retrieved_objs = $session->get(
   object          => 'Infoblox::Grid::ApprovalWorkflow',
   submitter_group => 'user-group',
);
```

Infoblox::Session->modify( )

Use this method to modify an object in the Infoblox appliance. See Infoblox::Session->modify() for parameters and return values.

Example

```
# Use this method to modify ticket_number_used attribute of an approval object.
$approval->ticket_number_used('REQUIRED');
# Submit modification
my $response = $session->modify( $approval );
```

Infoblox::Session->remove( )

Use this method to remove an object from the Infoblox appliance. See Infoblox::Session->remove() for parameters and return values.

To remove a specific object, first use `get()` or `search()` to retrieve the specific object, and then submit this object for removal.

Example

```
#Get the objects
my @retrieved_objs = $session->get(
   object          => 'Infoblox::Grid::ApprovalWorkflow',
   submitter_group => 'user-group',
);
#Find the desired object from the retrieved list.
my $desired_approval = $retrieved_objs[0];
# Submit for removal
my $response = $session->remove( $desired_approval );
```

Infoblox::Session->search( )

Use this method to search for approval objects in the Infoblox appliance. See Infoblox::Session->search() for parameters and return values.

Key References

```
Apply the following attributes to search a specific approval object:

 submitter_group   - Optional. The submitter group name.
 approval_group    - Optional. The approval group name.
```

Example

```
my @retrieved_objs = $session->search(
   object          => 'Infoblox::Grid::ApprovalWorkflow',
   submitter_group => 'user-group',
);
```

METHODS

This section describes all the methods that you can use to configure and retrieve the attribute values of an approval object.

approval_group( )

Use this method to set or retrieve an approval group name.

Parameter

The approval group name in string format.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get attribute value
my $value = $approval->approval_group();
#Modify attribute value
$approval->approval_group('admin-group');
```

submitter_group( )

Use this method to set or retrieve a submitter group name.

Parameter

The submitter group name in string format.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get attribute value
my $value = $approval->submitter_group();
#Modify attribute value
$approval->submitter_group('user_group');
```

approval_notify_enabled( )

Use this method to enable or disable sending approval task notifications.

Parameter

Specify "false" to disable this mode or "true" to enable it. The default value is 'true'.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get attribute value
my $value = $approval->approval_notify_enabled();
#Modify attribute value
$approval->approval_notify_enabled('false');
```

approval_notify_to( )

Use this method to set or retrieve to whom approval task notifications are sent.

Parameter

The valid values are 'APPROVER', 'BOTH' and 'SUBMITTER'. The default value is 'BOTH'.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get attribute value
my $value = $approval->approval_notify_to();
#Modify attribute value
$approval->approval_notify_to('APPROVER');
```

approved_notify_enabled( )

Use this method to enable or disable sending approved task notifications.

Parameter

Specify "false" to disable this mode or "true" to enable it. The default value is 'true'.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get attribute value
my $value = $approval->approved_notify_enabled();
#Modify attribute value
$approval->approved_notify_enabled('false');
```

approved_notify_to( )

Use this method to set or retrieve to whom approved task notifications are sent.

Parameter

Valid values are 'APPROVER', 'BOTH' and 'SUBMITTER'. The default value is 'BOTH'.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get attribute value
my $value = $approval->approved_notify_to();
#Modify attribute value
$approval->approved_notify_to('APPROVER');
```

approver_comment_used( )

Use this method to set or retrieve the option for approver to provide the comment field.

Parameter

Valid values are 'OPTIONAL', 'REQUIRED' and 'UNUSED'. The default value is 'UNUSED'.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get attribute value
my $value = $approval->approver_comment_used();
#Modify attribute value
$approval  approver comment used
```

```
                ->                          ('REQUIRED');
```

enable_notify_group( )

Use this method to enable or disable sending email notifications to admin group email addresses.

Parameter

Specify "false" to disable this mode or "true" to enable it. The default value is 'true'.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get attribute value
my $value = $approval->enable_notify_group();
#Modify attribute value
$approval->enable_notify_group('false');
```

enable_notify_user( )

Use this method to enable or disable sending email notifications to admin member email addresses.

Parameter

Specify "false" to disable this mode or "true" to enable it. The default value is 'false'.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get attribute value
my $value = $approval->enable_notify_user();
#Modify attribute value
$approval->enable_notify_user('true');
```

failed_notify_enabled( )

Use this method to enable or disable sending failed task notifications.

Parameter

Specify "false" to disable this mode or "true" to enable it. The default value is 'true'.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get attribute value
my $value = $approval->failed_notify_enabled();
#Modify attribute value
$approval->failed_notify_enabled('false');
```

failed_notify_to( )

Use this method to set or retrieve to whom failed task notifications are sent.

Parameter

Valid values are 'APPROVER', 'BOTH' and 'SUBMITTER'. The default value is 'BOTH'.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get attribute value
my $value = $approval->failed_notify_to();
#Modify attribute value
$approval->failed_notify_to('APPROVER');
```

rejected_notify_enabled( )

Use this method to enable or disable sending rejected task notifications.

Parameter

Specify "false" to disable this mode or "true" to enable it. The default value is 'false'.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get attribute value
my $value = $approval->rejected_notify_enabled();
#Modify attribute value
$approval->rejected_notify_enabled('false');
```

rejected_notify_to( )

Use this method to set or retrieve to whom rejected task notifications are sent.

Parameter

Valid values are 'APPROVER', 'BOTH' and 'SUBMITTER'. The default value is 'BOTH'.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get attribute value
my $value = $approval->rejected_notify_to();
#Modify attribute value
$approval->rejected_notify_to('APPROVER');
```

rescheduled_notify_enabled( )

Use this method to enable or disable sending rescheduled task notifications.

Parameter

Specify "false" to disable this mode or "true" to enable it. The default value is 'false'.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get attribute value
my $value = $approval->rescheduled_notify_enabled();
#Modify attribute value
$approval->rescheduled_notify_enabled('true');
```

rescheduled_notify_to( )

Use this method to set or retrieve to whom rescheduled task notifications are sent.

Parameter

Valid values are 'APPROVER', 'BOTH' and 'SUBMITTER'. The default value is 'BOTH'.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get attribute value
my $value = $approval->();
#Modify attribute value
$approval->('APPROVER');
```

submitter_comment_used( )

Use this method to set or retrieve the option for submitter to provide the comment field.

Parameter

Valid values are 'OPTIONAL', 'REQUIRED' and 'UNUSED'. The default value is 'UNUSED'.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get attribute value
my $value = $approval->submitter_comment_used();
#Modify attribute value
$approval->submitter_comment_used('OPTIONAL');
```

succeeded_notify_enabled( )

Use this method to enable or disable sending succeeded task notifications.

Parameter

Specify "false" to disable this mode or "true" to enable it. The default value is 'true'.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get attribute value
my $value = $approval->succeeded_notify_enabled();
#Modify attribute value
$approval->succeeded_notify_enabled('false');
```

succeeded_notify_to( )

Use this method to set or retrieve to whom succeeded task notifications are sent.

Parameter

Valid values are 'APPROVER', 'BOTH' and 'SUBMITTER'. The default value is 'BOTH'.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get attribute value
my $value = $approval->succeeded_notify_to();
#Modify attribute value
$approval->succeeded_notify_to('APPROVER');
```

ticket_number_used( )

Use this method to set or retrieve the option for submitter to provide the ticket number field.

Parameter

Valid values are 'OPTIONAL', 'REQUIRED' and 'UNUSED'. The default value is 'UNUSED'.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get attribute value
my $value = $approval->ticket_number_used();
#Modify attribute value
$approval->ticket_number_used('OPTIONAL');
```

SAMPLE CODE

The following sample code demonstrates the different functions that can be applied to an object such as add, get, modify, search and remove. This sample also includes error handling for the operations.

#Preparation prior to getting and modifying a approval object

```
use strict;
use Infoblox;

#refers to Infoblox Appliance IP address
my $host_ip = "192.168.1.2";

#Create a session to the Infoblox appliance

my $session = Infoblox::Session->new(
    master   => $host_ip,
    username => "admin",
    password => "infoblox"
);
unless ($session) {
       die("Construct session failed: ",
               Infoblox::status_code() . ":" . Infoblox::status_detail());
}
print "Session created successfully\n";
```

#Create a approval object

```
my $approval = Infoblox::Grid::ApprovalWorkflow->new(
   submitter_group => 'user-group',
   approval_group  => 'admin-group',
);

unless ($approval) {
       die("Construct approval object failed: ",
               Infoblox::status_code() . ":" . Infoblox::status_detail());
}
print "Approval object created successfully\n";

my $response = $session->add($approval);

unless ($response) {
    die("Add approval failed: ",
           $session->status_code() . ":" . $session->status_detail());
}
print"Approval added successfully \n";
```

#Search for a approval object

```
my @retrieved_objs = $session->search(
   object          => 'Infoblox::Grid::ApprovalWorkflow',
   submitter_group => 'user-group',
);

$approval = $retrieved_objs[0];

unless ($approval) {
    die("Get approval object failed: ",
           $session  status code           $session  status detail
```

```
                    ->           () . ":" .          ->             ());
}
print"Get approval successful \n";

$approval->comment("This is modified object");

$session->modify($approval)
   or die("Modify approval failed",
            $session->status_code() . ":" . $session->status_detail());
print "Approval modified successfully \n";
```

#Get and modify the approval object

```
@retrieved_objs = $session->get(
   object          => 'Infoblox::Grid::ApprovalWorkflow',
   submitter_group => 'user-group',
);

$approval = $retrieved_objs[0];

unless ($approval) {
    die("Get approval object failed: ",
           $session->status_code() . ":" . $session->status_detail());
}
print"Get approval successful \n";

$approval->comment("This is modified object");

$session->modify($approval)
   or die("Modify approval failed",
            $session->status_code() . ":" . $session->status_detail());
print "Approval modified successfully \n";
```

#Remove the approval object

```
@retrieved_objs = $session->get(
   object          => 'Infoblox::Grid::ApprovalWorkflow',
   submitter_group => 'user-group',
);

$approval = $retrieved_objs[0];

unless ($approval) {
    die("Get approval object failed: ",
           $session->status_code() . ":" . $session->status_detail());
}
print"Get approval successful \n";

$session->remove( $approval )
    or die("Remove approval failed",
           $session->status_code() . ":" . $session->status_detail());
print"Approval removed successfully \n";

####PROGRAM ENDS####
```

AUTHOR

Infoblox Inc. http://www.infoblox.com/

SEE ALSO

Infoblox::Session

COPYRIGHT

Copyright (c) 2017 Infoblox Inc.

Infoblox::Grid::BFD::Template - a Bidirectional Forwarding Detection template object.

- NAME
- DESCRIPTION
- CONSTRUCTOR
- SESSION METHODS
  - Infoblox::Session->add( )
  - Infoblox::Session->get( )
  - Infoblox::Session->modify( )
  - Infoblox::Session->remove( )
  - Infoblox::Session->search( )
- METHODS
  - authentication_key( )
  - authentication_key_id( )
  - authentication_type( )
  - detection_multiplier( )
  - is_default( )
  - name( )
  - min_rx_interval( )
  - min_tx_interval( )
- AUTHOR
- SEE ALSO
- COPYRIGHT

NAME

Infoblox::Grid::BFD::Template - a Bidirectional Forwarding Detection (BFD) template object.

DESCRIPTION

The BFD template is used to configure advanced BFD settings such as time intervals, authentication setting.

CONSTRUCTOR

```
my $bfd_template = Infoblox::Grid::BFD::Template->new(
    name                 => $string,                              #Required
    authentication_type  => 'NONE' | 'MD5' | 'METICULOUS-MD5' |
                            'SHA1' | 'METICULOUS-SHA1',           #Optional / Default is 'NONE'
    authentication_key   => $string,                              #Optional / Default is undefined
    authentication_key_id => $string,                             #Optional / Default is undefined
    detection_multiplier  => $uint,                               #Optional / Default is 3
    min_rx_interval       => $uint,                               #Optional / Default is 100
    min_tx_interval       => $uint,                               #Optional / Default is 100
);
```

SESSION METHODS

This section describes all the methods in an Infoblox::Session module that you can apply to a BFD template object.

Infoblox::Session->add( )

Use this method to add the object to the Infoblox appliance. See Infoblox::Session->add() for parameters and return values.

Example

```
    #Construct an object
    my $object = Infoblox::Grid::BFD::Template->new(
        name                 => 'bfd_template1',
        authentication_type  => 'MD5',
        authentication_key   => 'secret',
        authentication_key_id => 42,
    );

    #Submit for addition
    my $response = $session->add($object);
```

Infoblox::Session->get( )

Use this method to retrieve all the matching objects from the Infoblox appliance. See Infoblox::Session->get() for parameters and return

values.

Key References

```
Apply the following attributes to get a specific BFD template object:

 name - Optional. The BFD template name in string format.
```

Examples

```
my @retrieved_objs = $session->get(
    object => 'Infoblox::Grid::BFD::Template',
    name   => 'bfd_template1',
);
```

Infoblox::Session->modify( )

Use this method to modify the object in the Infoblox appliance. See Infoblox::Session->modify() for parameters and return values.

Example

```
#Modify min_rx value
$object->min_rx_interval(100);

#Submit modification
my $response = $session->modify($object);
```

Infoblox::Session->remove( )

Use this method to remove the object from the Infoblox appliance. See Infoblox::Session->remove() for parameters and return values.

To remove a specific object, first use `get()` or `search()` to retrieve the specific DHCP range object, and then submit this object for removal.

Example

```
#Get the objects
my @retrieved_objs = $session->get(
    object => 'Infoblox::Grid::BFD::Template',
    name   => 'bfd_template1',
);

#Find the desired object from the retrieved list
my $desired_object = $retrieved_objs[0];

#Submit for removal
my $response = $session->remove($desired_object);
```

Infoblox::Session->search( )

Use this method to search for objects in the Infoblox appliance. See Infoblox::Session->search() for parameters and return values.

Key References

```
Apply the following attributes to search for the BFD template object:

 name - Optional. The BFD template name in string format (regexp).
```

Examples

```
# search for objects
my @retrieved_objs = $session->search(
    object => 'Infoblox::Grid::BFD::Template',
    name   => '.*template',
);
```

METHODS

This section describes all the methods that you can use to configure and retrieve the attribute values of a BFD template object.

authentication_key( )

Use this method to set or retrieve the authentication key for BFD protocol message-digest authentication. This is a write-only attribute.

Parameter

The valid value is a authentication key in a string format.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

Example

```
#Modify authentication_key value
$object->authentication_key('secret');
```

authentication_key_id( )

Use this method to set or retrieve the authentication key identifier for BFD protocol authentication.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

The valid value is an unsigned integer between 1 and 255. The default is 1.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get authentication_key_id value
my $authentication_key_id = $object->authentication_key_id();

#Modify authentication_key_id value
$object->authentication_key_id(200);
```

authentication_type( )

Use this method to set or retrieve the authentication type for BFD protocol.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

The valid values are 'NONE', 'MD5', 'METICULOUS-MD5', 'SHA1', and 'METICULOUS-SHA1'. The default value is 'NONE'.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get authentication_type value
my $authentication_type = $object->authentication_type();

#Modify authentication_type value
$object->authentication_type('SHA1');
```

detection_multiplier( )

Use this method to set or retrieve the detection time multiplier value for BFD protocol.

The negotiated transmit interval, multiplied by this value, provides the detection time for the receiving system in asynchronous BFD mode.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

The valid value is an unsigned integer between 3 and 50. The default is 3.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get detection_multiplier value
```

```
my $detection_multiplier = $object->detection_multiplier();

#Modify detection_multiplier value
$object->detection_multiplier(20);
```

is_default( )

Use this method to retrieve the flag that indicates whether the BFD template is a Grid default or not. This is a read-only attribute.

Parameter

None

Returns

The method returns the attribute value.

Example

```
#Get is_default value
my $is_default = $object->is_default();
```

name( )

Use this method to set or retrieve the BFD template name.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

The valid value is a BFD template name in a string format.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get name value
my $name = $object->name();

#Modify name value
$object->name('template1');
```

min_rx_interval( )

Use this method to set or retrieve the minimum receive time (in seconds) for BFD protocol.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

The valid value is an unsigned integer between 50 and 9999. The default is 100.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get min_rx_interval value
my $min_rx_interval = $object->min_rx_interval();

#Modify min_rx_interval value
$object->min_rx_interval(200);
```

min_tx_interval( )

Use this method to set or retrieve the minimum transmission time (in seconds) for BFD protocol.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

The valid value is an unsigned integer between 50 and 9999. The default is 100.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get min_tx_interval value
my $min_tx_interval = $object->min_tx_interval();

#Modify min_tx_interval value
$object->min_tx_interval(200);
```

AUTHOR

Infoblox Inc. http://www.infoblox.com/

SEE ALSO

Infoblox::Session, Infoblox::Session->add(), Infoblox::Session->get(), Infoblox::Session->modify(), Infoblox::Session->remove(), Infoblox::Session->search(),

---

COPYRIGHT

Copyright (c) 2017 Infoblox Inc.

Infoblox::Grid::CACertificate - CA certificate description.

- NAME
- DESCRIPTION
- SESSION METHODS
  - Infoblox::Session->get( )
  - Infoblox::Session->remove( )
  - Infoblox::Session->search( )
- METHODS
  - distinguished_name( )
  - issuer( )
  - serial( )
  - used_by( )
- AUTHOR
- COPYRIGHT

NAME

Infoblox::Grid::CACertificate - CA certificate description.

DESCRIPTION

An Infoblox::Grid::CACertificate object represents a CA certificate description.

SESSION METHODS

This section describes all the methods in an Infoblox::Session module that you can apply to an Infoblox::Grid::CACertificate object.

Infoblox::Session->get( )

Use this method to retrieve all the matching objects from the Infoblox appliance. See Infoblox::Session->get() for parameters and return values.

Example

```
my @retrieved_objs = $session->get(
     object => 'Infoblox::Grid::CACertificate',
     serial => 'serial',
);

my @retrieved_objs = $session->get(
     object  => 'Infoblox::Grid::CACertificate',
     issuer => 'issuer',
);

my @retrieved_objs = $session->get(
     object   => 'Infoblox::Grid::CACertificate',
     distinguished_name => 'distinguished_name',
);
```

Infoblox::Session->remove( )

Use this method to remove an object from the Infoblox appliance. See Infoblox::Session->remove() for parameters and return values.

To remove a specific object, first use `get()` to retrieve the object, and then submit it for removal.

Example

```
#Get the objects with the specified serial.
my @retrieved_objs = $session->get(
     object => 'Infoblox::Grid::CACertificate',
     serial => 'serial',
);

#Find the desired object on the retrieved list.
my $desired_ca_cert = $retrieved_objs[0];

# Submit for removal
my $response = $session->remove($desired_ca_cert);
```

Infoblox::Session->search( )

Use this method to retrieve all the matching objects from the Infoblox appliance. See Infoblox::Session->search() for parameters and return values.

Example

```
my @retrieved_objs = $session->search(
     object => 'Infoblox::Grid::CACertificate',
     serial => 'serial',
);

my @retrieved_objs = $session->search(
     object  => 'Infoblox::Grid::CACertificate',
     issuer => 'issuer',
);

my @retrieved_objs = $session->search(
     object   => 'Infoblox::Grid::CACertificate',
     distinguished_name => 'distinguished_name',
);
```

METHODS

This section describes all the methods that you can use to configure and retrieve the attribute values of an Infoblox::Grid::CACertificate object.

distinguished_name( )

Use this method to retrieve certificate subject name. This attribute is read-only and cannot be set.

Parameter

None.

Returns

The method returns the attribute value.

Example

```
#Get distinguished_name
my $distinguished_name = $ca_cert->distinguished_name();
```

issuer( )

Use this method to retrieve issuer subject name. This attribute is read-only and cannot be set.

Parameter

None.

Returns

The method returns the attribute value.

Example

```
#Get issuer
my $issuer = $ca_cert->issuer();
```

serial( )

Use this method to retrieve Certificate serial number in hex format. This attribute is read-only and cannot be set.

Parameter

None.

Returns

The method returns the attribute value.

Example

```
#Get serial
my $serial = $ca_cert->serial();
```

used_by( )

Use this method to retrieve information about usages of the certificate. This attribute is read-only and cannot be set.

Parameter

None.

Returns

The method returns the attribute value. If the returned value is equal to SSL/TLS - all certificates in CA store can be used for HTTPS and if the returned value is equal to CAS - additionally certificate can be assigned to Certificate Authentication Group.

Example

```
#Get used_by
my $used_by = $ca_cert->used_by();
```

AUTHOR

Infoblox Inc. http://www.infoblox.com/

COPYRIGHT

Copyright (c) 2017 Infoblox Inc.

Infoblox::Grid::CloudAPI - Infoblox::Grid::CloudAPI object.

- NAME
- DESCRIPTION
- SESSION METHODS
  - Infoblox::Session->get( )
  - Infoblox::Session->modify( )
- METHODS
  - allow_api_admins( )
  - allowed_api_admins( )
  - enable_recycle_bin( )
- AUTHOR
- SEE ALSO
- COPYRIGHT

NAME

Infoblox::Grid::CloudAPI - Infoblox::Grid::CloudAPI object.

DESCRIPTION

This object represents the cloud Grid.

SESSION METHODS

This section describes all the methods in an Infoblox::Session module that you can apply to a Grid CloudAPI object.

Infoblox::Session->get( )

Use this method to retrieve all the matching objects from the Infoblox appliance. See Infoblox::Session->get() for parameters and return values.

Key Reference

```
Apply the following attribute to get a specific object:

  allow_api_admins   - Optional. The definition of administrators who can perform cloud API requests
on the Grid Master. The valid value is NONE, ALL or LIST.
  allowed_api_admins   - Optional. The list of administrators who can perform cloud API requests on
the Cloud Platform Appliance.
```

Example

```
my @retrieved_objs = $session->get(
    object => "Infoblox::Grid::CloudAPI"
);
```

Infoblox::Session->modify( )

Use this method to modify an object in the Infoblox appliance. See Infoblox::Session->modify() for parameters and return values.

Example

```
$cloud_api->allow_api_admins("ALL");
$session->modify($cloud_api);
```

METHODS

This section describes all the methods that you can use to set or retrieve the attribute values of the object.

allow_api_admins( )

Use this method to set or retrieve the definition of administrators who can perform cloud API requests on the Grid Master. The valid value is NONE (no administrator), ALL (all administrators) or LIST (administrators on the ACL).

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

The valid value is one of the following: NONE, ALL or LIST. The default is ALL.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
# Get allow_api_admins
$allow_api_admins = $cloud_api->allow_api_admins();
# Modify allow_api_admin
$cloud_api->allow_api_admins("ALL");
```

allowed_api_admins( )

Use this method to set or retrieve the list of administrators who can perform cloud API requests on the Grid Master.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

The valid value is an array containing Infoblox::Grid::CloudAPI::User objects.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
# Get allowed_api_admins
$allowed_api_admins = $cloud_api->allowed_api_admins();
# Modify allowed_api_admins
$cloud_api->allowed_api_admins([$cloud_user1, $cloud_user2]);
```

enable_recycle_bin( )

Use this method to enable or disable the recycle bin for deleted cloud objects on the Grid Master regardless of whether certificate is enabled or not.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

Specify "true" to enable recycle or "false" to disable it.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
# Get enable_recycle_bin
$enabled_recycle_bins = $cloud_api->enable_recycle_bin();
# Modify enable_recycle_bin
$cloud_api->enable_recycle_bin("true");
```

AUTHOR

Infoblox Inc. http://www.infoblox.com/

SEE ALSO

Infoblox::Session, Infoblox::Session->get(), Infoblox::Session->modify(), Infoblox::Grid::CloudAPI::User

---

COPYRIGHT

Copyright (c) 2017 Infoblox Inc.

Infoblox::Grid::CloudAPI::CloudStatistics - Infoblox::Grid::CloudAPI::CloudStatistics object.

- NAME
- DESCRIPTION
- CONSTRUCTOR
- SESSION METHODS
  - Infoblox::Session->get( )
- METHODS
  - allocated_ip_count( )
  - available_ip_count( )
  - fixed_ip_count( )
  - floating_ip_count( )
  - tenant_count( )
  - tenant_ip_count( )
  - tenant_vm_count( )
- AUTHOR
- SEE ALSO
- COPYRIGHT

NAME

Infoblox::Grid::CloudAPI::CloudStatistics - Infoblox::Grid::CloudAPI::CloudStatistics object.

DESCRIPTION

This object aggregates Cloud API statistics.

CONSTRUCTOR

SESSION METHODS

This section describes all the methods in an Infoblox::Session module that you can apply to a Grid member object.

Infoblox::Session->get( )

Use this method to retrieve the object from the Infoblox appliance. See Infoblox::Session->get() for parameters and return values.

```
=item B<Example>

 $session->get(object => "Infoblox::Grid::CloudAPI::CloudStatistics");
```

METHODS

This section describes all the methods that you can use to set or retrieve the attribute values of the object.

allocated_ip_count( )

Use this method to retrieve the total number of IPs allocated by tenants. This is a read-only attribute.

Parameter

None

Returns

The method returns attribute value.

Example

```
# Get allocated_ip_count
$allocated_ip_count = $cloud_statistic->allocated_ip_count();
```

available_ip_count( )

Use this method to retrieve the total number of IPs available to tenants. Only IP under networks within a delegation scope are counted. This is a read-only attribute.

Parameter

None

Returns

The method returns attribute value.

Example

```
# Get available_ip_count
$available_ip_count = $cloud_statistic->available_ip_count();
```

fixed_ip_count( )

Use this method to retrieve the total number of Fixed IPs currently in use by all tenants in the system. This is a read-only attribute.

Parameter

None

Returns

The method returns attribute value.

Example

```
# Get fixed_ip_count
$fixed_ip_count = $cloud_statistic->fixed_ip_count();
```

floating_ip_count( )

Use this method to retrieve the total number of Floating IPs currently in use by all tenants in the system. This is a read-only attribute.

Parameter

None

Returns

The method returns attribute value.

Example

```
# Get floating_ip_count
$floating_ip_count = $cloud_statistic->floating_ip_count();
```

tenant_count( )

Use this method to retrieve the total number of tenants currently in the system. This is a read-only attribute.

Parameter

None

Returns

The method returns attribute value.

Example

```
# Get tenant_count
$tenant_count = $cloud_statistic->tenant_count();
```

tenant_ip_count( )

Use this method to retrieve the total number of IPs currently in use by all tenants in the system. This is a read-only attribute.

Parameter

None

Returns

The method returns attribute value.

Example

```
# Get tenant_ip_count
$tenant_ip_count = $cloud_statistic->tenant_ip_count();
```

tenant_vm_count( )

Use this method to retrieve the total number of VMs currently in use by all tenants the system. This is a read-only attribute.

Parameter

None

Returns

The method returns attribute value.

Example

```
# Get tenant_vm_count
$tenant_vm_count = $cloud_statistic->tenant_vm_count();
```

AUTHOR

Infoblox Inc. http://www.infoblox.com/

SEE ALSO

Infoblox::Session->get()

COPYRIGHT

Copyright (c) 2017 Infoblox Inc.

Infoblox::Grid::CloudAPI::Info - Infoblox::Grid::CloudAPI::Info object.

- NAME
- DESCRIPTION
- SESSION METHODS
- METHODS
  - delegated_member( )
  - delegated_root( )
  - delegated_scope( )
  - owned_by_adaptor( )
  - tenant( )
  - usage( )
  - authority_type( )
- AUTHOR
- SEE ALSO
- COPYRIGHT

NAME

Infoblox::Grid::CloudAPI::Info - Infoblox::Grid::CloudAPI::Info object.

DESCRIPTION

This structure aggregates all information related to Cloud objects.

SESSION METHODS

The object does not support any session methods. It is retrieved as part of the objects supporting it.

METHODS

This section describes all the methods that you can use to set or retrieve the attribute values of the object.

delegated_member( )

Use this method to set or retrieve the reference to the Cloud Platform Appliance to which authority of an object is delegated. If the delegated_scope value is SUBTREE or RECLAIMING, this is a read-only attribute.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

None if the attribute is a read-only. Otherwise the valid value is an Infoblox::DHCP::Member object.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

If this is a read-only attribute, the method returns the attribute value.

Example

```
# Get delegated_member
$delegated_member = $cloud_api_info->delegated_member();
# Set delegated_member
$cloud_api_info->delegated_member($member);
```

delegated_root( )

Use this method to retrieve the root of the delegation if delegated_scope is SUBTREE or RECLAIMING. This is a read-only attribute.

Parameter

None

Returns

This method returns the attribute value.

Example

```
# Get delegated_root
$delegated_root = $cloud_api_info->delegated_root();
```

delegated_scope( )

Use this method to retrieve the delegation scope of the object. This is a read-only attribute.

Parameter

None

Returns

This method returns the attribute value.

Example

```
# Get delegated_scope
$delegated_scope = $cloud_api_info->delegated_scope();
```

owned_by_adaptor( )

Use this method to retrieve the information about whether the object was created by the cloud adapter or not.

Parameter

None

Returns

This method returns the attribute value.

Example

```
# Get owned_by_adaptor
$owned_by_adaptor = $cloud_api_info->owned_by_adaptor();
```

tenant( )

Use this method to retrieve the reference to the tenant object associated with the object, if any. This is a read-only attribute.

Parameter

None

Returns

This method returns the attribute value.

Example

```
# Get tenant
$tenant = $cloud_api_info->tenant();
```

usage( )

Use this method to retrieve the the cloud origin of the object. This is a read-only attribute.

Parameter

None

Returns

This method returns the attribute value.

Example

```
# Get usage
$usage = $cloud_api_info->usage();
```

authority_type( )

Use this method to retrieve the authority type of the object. This is a read-only attribute.

Parameter

None

Returns

This method returns the attribute value.

Example

```
# Get authority_type
$authority_type = $cloud_api_info->authority_type();
```

AUTHOR

Infoblox Inc. http://www.infoblox.com/

SEE ALSO

Infoblox::DHCP::Member, Infoblox::Grid::CloudAPI::Tenant

COPYRIGHT

Copyright (c) 2017 Infoblox Inc.

Infoblox::Grid::CloudAPI::Member - Infoblox::Grid::CloudAPI::Member object.

- NAME
- DESCRIPTION
- CONSTRUCTOR
- SESSION METHODS
  - Infoblox::Session->get( )
  - Infoblox::Session->modify( )
  - Infoblox::Session->search( )
- METHODS
  - allow_api_admins( )
  - allowed_api_admins( )
  - enable_service( )
  - extattrs( )
  - extensible_attributes( )
  - member( )
  - status( )
- AUTHOR
- SEE ALSO
- COPYRIGHT

NAME

Infoblox::Grid::CloudAPI::Member - Infoblox::Grid::CloudAPI::Member object.

DESCRIPTION

This object defines an entry in the ACL for cloud API users.

CONSTRUCTOR

SESSION METHODS

This section describes all the methods in an Infoblox::Session module that you can apply to a CloudAPI Member object.

Infoblox::Session->get( )

Use this method to retrieve all the matching objects from the Infoblox appliance. See Infoblox::Session->get() for parameters and return values.

Key Reference

```
Apply the following attribute to get a specific object:

  comment               - Optional. Descriptive comments.
  extattrs              - Optional. A hash reference containing extensible attributes.
  extensible_attributes - Optional. A hash reference containing extensible attributes.
  ipv4addr              - Optional. IPv4 Member address.
  ipv6addr              - Optional. IPv6 Member address.
  name                  - Optional. The host name of the parent member.
```

Example

```
$session->get(object => "Infoblox::Grid::CloudAPI::Member", allowed_api_admins => "ALL");
```

Infoblox::Session->modify( )

Use this method to modify an object in the Infoblox appliance. See Infoblox::Session->modify() for parameters and return values.

Example

```
$session->modify($cloud_api_member);
```

Infoblox::Session->search( )

Use this method to search for objects in the Infoblox appliance. See Infoblox::Session->search() for parameters and return values.

Key Reference

```
Apply the following attribute to search a specific object:
```

```
comment                - Optional. Descriptive comments (regular expression).
extattrs               - Optional. A hash reference containing extensible attributes.
extensible_attributes  - Optional. A hash reference containing extensible attributes.
ipv4addr               - Optional. IPv4 Member address (regular expression).
ipv6addr               - Optional. IPv6 Member address (regular expression).
name                   - Optional. The host name of the parent member.
```

Example

```
$session->search(object => "Infoblox::Grid::CloudAPI::Member", comment => "some text");
```

METHODS

This section describes all the methods that you can use to set or retrieve the attribute values of the object.

allow_api_admins( )

Use this method to set or retrieve the definition of administrators who can perform cloud API requests on the Grid Master. The valid value is NONE (no administrator), ALL (all administrators) or LIST (administrators on the ACL).

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

The valid value is one of the following: NONE, ALL or LIST. Default is ALL.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
# Get allow_api_admins
$allow_api_admins = $cloud_api->allow_api_admins();
# Modify allow_api_admins
$cloud_api->allow_api_admins("ALL");
```

allowed_api_admins( )

Use this method to set or retrieve the list of administrators who can perform cloud API requests on the Grid Master.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

The valid value is an array containing Infoblox::Grid::CloudAPI::User objects.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
# Get allowed_api_admins
$allowed_api_admins = $cloud_api->allowed_api_admins();
# Modify allowed_api_admins
$cloud_api->allowed_api_admins([$cloud_user1, $cloud_user2]);
```

enable_service( )

Use this method to enable or disable the Cloud API service on the member.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

Specify "true" to enable the Cloud API service or "false" to disable it. The default value is "false".

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
# Get enable_service
$enabled_service = $cloud_api->enable_service();
# Modify enable_service
$cloud_api->enable_service('true');
```

extattrs( )

Use this method to retrieve the extensible attributes associated with an Infoblox::Grid::CloudAPI::Member object.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

None

Returns

This method returns the attribute value.

Example

```
# Get extattrs
my $ref_extattrs = $cloud_api->extattrs();
# Modify extattrs
$cloud_api->extattrs({ 'Site' => $extattr1, 'Administrator' => $extattr2 });
```

extensible_attributes( )

Use this method to retrieve the extensible attributes associated with an Infoblox::Grid::CloudAPI::Member object

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

For valid values for extensible attributes, see Infoblox::Grid::ExtensibleAttributeDef/Extensible Attribute Values.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
# Get extensible attributes
my $ref_extensible_attributes = $cloud_api->extensible_attributes();
# Modify extensible attributes
$cloud_api->extensible_attributes({ 'Site' => 'Santa Clara', 'Administrator' => [ 'Peter', 'Tom' ]
});
```

member( )

Use this method to retrieve an Infoblox DHCP Member object. This is a read-only attribute.

Parameter

None

Returns

This method returns the attribute value.

Example

```
# Get member
my $member = $cloud_api->member();
```

status( )

Use this method to retrieve the status of the Cloud API service on a member. This is a read-only attribute.

Parameter

None

Returns

This method returns the attribute value.

Example

```
# Get status
my $status = $cloud_api->status();
```

AUTHOR

Infoblox Inc. http://www.infoblox.com/

SEE ALSO

COPYRIGHT

Copyright (c) 2017 Infoblox Inc.

Infoblox::Grid::CloudAPI::Tenant - Infoblox::Grid::CloudAPI::Tenant object.

- NAME
- DESCRIPTION
- CONSTRUCTOR
- SESSION METHODS
  - Infoblox::Session->get( )
  - Infoblox::Session->modify( )
  - Infoblox::Session->search( )
- METHODS
  - comment( )
  - first_seen( )
  - id( )
  - last_seen( )
  - name( )
  - network_count( )
  - vm_count( )
  - unmanaged( )
  - first_discovered_timestamp( )
  - last_discovered_timestamp( )
- AUTHOR
- SEE ALSO
- COPYRIGHT

NAME

Infoblox::Grid::CloudAPI::Tenant - Infoblox::Grid::CloudAPI::Tenant object.

DESCRIPTION

This object represents an administrative concept in Cloud Management Platforms that associates all the network elements (networks, zones, VMs, IPs, ...) that are related and constitutive of the tenant.

CONSTRUCTOR

SESSION METHODS

This section describes all the methods in an Infoblox::Session module that you can apply to a Grid member object.

Infoblox::Session->get( )

Use this method to retrieve all the matching objects from the Infoblox appliance. See Infoblox::Session->get() for parameters and return values.

Key Reference

```
Apply the following attribute to get a specific object:

  comment       - Optional. Descriptive comments.
  id            - Optional. Unique ID associated with the tenant. Set at creation time only.
  name          - Required. Name of the tenant.
  unmanaged     - Optional. Flag to indicate if the tenant is unmanaged. Valid value is 'true' or
'false'.
```

Example

```
$session->get(object => "Infoblox::Grid::CloudAPI::Tenant", comment => "comment");
```

Infoblox::Session->modify( )

Use this method to modify an object in the Infoblox appliance. See Infoblox::Session->modify() for parameters and return values.

Example

```
$session->modify($tenant_object);
```

Infoblox::Session->search( )

Use this method to search for objects in the Infoblox appliance. See Infoblox::Session->search() for parameters and return values.

Key Reference

```
Apply the following attribute to get a specific object:

   comment       - Optional. Descriptive comments (regular expression).
   id            - Optional. Unique ID associated with the tenant. Set at creation time only (regular
expression).
   name          - Required. Name of the tenant (regular expression).
   unmanaged     - Optional. Flag to indicate if the tenant is unmanaged. Valid value is 'true' or
'false'.
```

Example

```
$session->get(object => "Infoblox::Grid::CloudAPI::Tenant", comment => "comment");
```

METHODS

This section describes all the methods that you can use to set or retrieve the attribute values of the object.

comment( )

Use this method to set or retrieve a descriptive comment.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

The valid value is a comment in string format (UTF8) with a maximum of 256 bytes.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
# Get comment
$comment = $tenant->comment();
# Modify comment
$tenant->comment("New comment");
```

first_seen( )

Use this method to retrieve the date and time the first time the tenant was seen on the system. This is a read-only attribute.

Parameter

None

Returns

The valid return value is number of seconds that have elapsed since January 1st, 1970 UTC.

Example

```
# Get first_seen
$first_seen = $tenant->first_seen();
```

id( )

Use this method to retrieve a unique ID associated with the tenant. Set at creation time only. This is a read-only attribute.

Parameter

None

Returns

The method returns attribute value.

Example

```
# Get id
$id = $tenant->id();
```

last_seen( )

Use this method to retrieve the date and time the last time the tenant was seen on the system. This is a read-only attribute.

Parameter

None

Returns

The valid return value is number of seconds that have elapsed since January 1st, 1970 UTC.

Example

```
# Get last_seen
$last_seen = $tenant->last_seen();
```

name( )

Use this method to set or retrieve the name of the tenant.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

The valid value is a comment in string format (UTF8) with a maximum of 64 bytes.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
# Get name
$name = $tenant->name();
# Modify name
$tenant->name("New name");
```

network_count( )

Use this method to retrieve the number of networks associated with the tenant. This is a read-only attribute.

Parameter

None

Returns

The method returns attribute value.

Example

```
# Get network_count
$network_count = $tenant->network_count();
```

vm_count( )

Use this method to retrieve the number of VMs associated with the tenant. This is a read-only attribute.

Parameter

None

Returns

The method returns attribute value.

Example

```
# Get vm_count
$vm_count = $tenant->vm_count();
```

unmanaged( )

Use this method to retrieve the boolean flag used to indicate if the tenant is unmanaged or not. This is a read-only attribute

Parameter

None

Returns

The method returns attribute value. The valid return value is "true" or "false".

Example

```
# Get unmanaged
$unmanaged = $tenant->unmanaged();
```

first_discovered_timestamp( )

Use this method to retrieve the date and time the tenant was first seen on the system. This is a read-only attribute.

Parameter

None

Returns

The valid return value is the number of seconds that have elapsed since January 1st, 1970 UTC.

Example

```
# Get first_discovered_timestamp
$first_discovered_timestamp = $tenant->first_discovered_timestamp();
```

last_discovered_timestamp( )

Use this method to retrieve the date and time the tenant was last seen on the system. This is a read-only attribute.

Parameter

None

Returns

The valid return value is the number of seconds that have elapsed since January 1st, 1970 UTC.

Example

```
# Get last_discovered_timestamp
$last_discovered_timestamp = $tenant->last_discovered_timestamp();
```

AUTHOR

Infoblox Inc. http://www.infoblox.com/

SEE ALSO

Infoblox::Session->get(), Infoblox::Session->search(), Infoblox::Session->modify()

COPYRIGHT

Copyright (c) 2017 Infoblox Inc.

Infoblox::Grid::CloudAPI::User - Infoblox::Grid::CloudAPI::User object.

- NAME
- DESCRIPTION
- CONSTRUCTOR
- SESSION METHODS
- MODULE METHODS
  - Infoblox::Grid::CloudAPI->allowed_api_admins( )
- METHODS
  - is_remote( )
  - local_admin( )
  - remote_admin( )
- AUTHOR
- SEE ALSO
- COPYRIGHT

NAME

Infoblox::Grid::CloudAPI::User - Infoblox::Grid::CloudAPI::User object.

DESCRIPTION

This object defines an entry in ACL for Cloud API Users.

CONSTRUCTOR

SESSION METHODS

The object does not support any session methods. It is retrieved as a part of the Grid CloudAPI object.

MODULE METHODS

This section describes all the functions that you can apply to an CloudAPI User object.

Infoblox::Grid::CloudAPI->allowed_api_admins( )

Use this method to set or retrieve the list of administrators who can perform cloud API requests on the Grid Master. See Infoblox::Grid::CloudAPI->allowed_api_admins()

Example

```
my $cloud_user = Infoblox::Grid::CloudAPI::User->new(
    is_remote => "false",
    remote_admin => "",
    local_admin => $local_admin_user
);

unless($cloud_user) {
     die("Construct member failed: ",
           Infoblox::status_code() . ":" . Infoblox::status_detail());
}
print "User object created successfully\n";

# Get CloudAPI object
my $cloud_api = $session->get( object => "Infoblox::Grid::CloudAPI" );

# Modify CloudAPI object
my $cloud_api->allowed_api_admins([$cloud_user]);
```

METHODS

This section describes all the methods that you can use to set or retrieve the attribute values of the object.

is_remote( )

Use this method to set or retrieve the user remote_admin status.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

Specify "true" to set user status to remote_admin or "false" to disable it.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
# Get is_remote
$is_remote = $user->is_remote()
# Modify is_remote
$user->is_remote('true')
```

local_admin( )

Use this method to set or retrieve a local administrator allowed to perform Cloud Platform API requests. Required if remote is false, should be none otherwise.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

The valid value is an Infoblox::Grid::Admin::User object.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
# Get local_admin
$local_admin = $user->local_admin()
# Modify local_admin
$user->local_admin($admin_user)
```

remote_admin( )

Use this method to set or retrieve a username matching a remotely authenticated administrator allowed to perform Cloud Platform API requests. Required if remote is true, should be none otherwise.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

The valid value is a string containing a remotely authenticated administrator name.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
# Get remote_admin
$remote_admin = $user->remote_admin()
# Modify remote_admin
$user->remote_admin("remote_admin")
```

AUTHOR

Infoblox Inc. http://www.infoblox.com/

SEE ALSO

Infoblox::Grid::Admin::User

COPYRIGHT

Copyright (c) 2017 Infoblox Inc.

Infoblox::Grid::CloudAPI::VMAddress - Infoblox::Grid::CloudAPI::VMAddress object.

- NAME
- DESCRIPTION
- CONSTRUCTOR
- SESSION METHODS
  - Infoblox::Session->get( )
  - Infoblox::Session->search( )
  - address( )
  - associated_objects( )
  - dns_names( )
  - is_ipv4( )
  - mac_address( )
  - ms_ad_user_data( )
  - network( )
  - network_view( )
  - port_id( )
  - tenant( )
  - tenant_name( )
  - vm_id( )
  - vm_name( )
- AUTHOR
- SEE ALSO
- COPYRIGHT

NAME

Infoblox::Grid::CloudAPI::VMAddress - Infoblox::Grid::CloudAPI::VMAddress object.

DESCRIPTION

Class that represents Tenant.

CONSTRUCTOR

SESSION METHODS

This section describes all the methods in an Infoblox::Session module that you can apply to a Grid member object.

Infoblox::Session->get( )

Use this method to retrieve all the matching objects from the Infoblox appliance. See Infoblox::Session->get() for parameters and return values.

Key Reference

```
Apply the following attribute to get a specific object:

  address            - Optional. IP address of the interface.
  mac_address        - Optional. MAC address of the interface.
  tenant_name        - Optional. Name of the tenant associated with the VM.
  vm_id              - Optional. UUID of the VMirtual Machine.
  vm_name            - Optional. Name of the Virtual Machine.
```

Example

```
$session->get(object => "Infoblox::Grid::CloudAPI::VMAddress", vm_id => $vm_id);
```

Infoblox::Session->search( )

Use this method to search for objects in the Infoblox appliance. See Infoblox::Session->search() for parameters and return values.

Key Reference

```
Apply the following attribute to get a specific object:

  address            - Optional. IP address of the interface.
  mac_address        - Optional. MAC address of the interface.
  tenant_name        - Optional. Name of the tenant associated with the VM.
  vm_id              - Optional. UUID of the VMirtual Machine.
  vm_name            - Optional. Name of the Virtual Machine.
```

Example

```
$session->search(object => "Infoblox::Grid::CloudAPI::VMAddress", vm_name => "vm name");
```

address( )

Use this method to retrieve the IP address of the interface. This is a read-only attribute.

Parameter

None

Returns

The method returns attribute value.

Example

```
# Get address
$address = $vmaddress->address();
```

associated_objects( )

Use this method to retrieve the list of references to the object (Host, Fixed Address, RR, ...) that define this IP. This is a read-only attribute.

Parameter

None

Returns

The method returns attribute value.

Example

```
# Get associated_objects
@associated_objects = $vmaddress->associated_objects();
```

dns_names( )

Use this method to retrieve the list of all FQDN associated with the IP address. This is a read-only attribute.

Parameter

None

Returns

The method returns attribute value.

Example

```
# Get dns_names
@dns_names = $vmaddress->dns_names();
```

is_ipv4( )

Use this method to retrieve the address type whether the address is IPv4 or IPv6. This is a read-only attribute.

Parameter

None

Returns

The method returns attribute value.

Example

```
# Get is_ipv4
$is_ipv4 = $vmaddress->is_ipv4();
```

mac_address( )

Use this method to retrieve the MAC address of the interface. This is a read-only attribute.

Parameter

None

Returns

The method returns attribute value.

Example

```
# Get mac_address
$mac_address = $vmaddress->mac_address();
```

ms_ad_user_data( )

Use this method to retrieve Microsoft Active Directory users related information. This is a read-only attribute.

Parameter

None

Returns

The valid return value is an Infoblox::Grid::MSServer::AdUser::Data object.

Example

```
#Get ms_ad_user_data
my $ms_ad_user_data = $vmaddress->ms_ad_user_data();
```

network( )

Use this method to retrieve the network address / CIDR that contains the IP. This is a read-only attribute.

Parameter

None

Returns

The method returns attribute value.

Example

```
# Get network
$network = $vmaddress->network();
```

network_view( )

Use this method to retrieve the network view name that contains the IP. This is a read-only attribute.

Parameter

None

Returns

The method returns attribute value.

Example

```
# Get network_view
$network_view = $vmaddress->network_view();
```

port_id( )

Use this method to retrieve the port identifier of the interface. This is a read-only attribute.

Parameter

None

Returns

The method returns attribute value.

Example

```
# Get port_id
$port_id = $vmaddress->port_id();
```

tenant( )

Use this method to retrieve a Infoblox::Grid::CloudAPI::Tenant object. This is a read-only attribute.

Parameter

None

Returns

The method returns attribute value.

Example

```
# Get tenant
$tenant = $vmaddress->tenant();
```

tenant_name( )

Use this method to retrieve the tenant name. This is a read-only attribute.

Parameter

None

Returns

The method returns attribute value.

Example

```
# Get tenant_name
$tenant_name = $vmaddress->tenant_name();
```

vm_id( )

Use this method to retrieve the UUID of the VMirtual Machine. This is a read-only attribute.

Parameter

None

Returns

The method returns attribute value.

Example

```
# Get vm_id
$vm_id = $vmaddress->vm_id();
```

vm_name( )

Use this method to retrieve the name of the Virtual Machine. This is a read-only attribute.

Parameter

None

Returns

The method returns attribute value.

Example

```
# Get vm_name
$vm_name = $vmaddress->vm_name();
```

AUTHOR

Infoblox Inc. http://www.infoblox.com/

SEE ALSO

Infoblox::Session->get(), Infoblox::Session->search(), Infoblox::Grid::CloudAPI::Tenant, Infoblox::Grid::MSServer::AdUser::Data

COPYRIGHT

Copyright (c) 2017 Infoblox Inc.

Infoblox::Grid::ConsentBannerSetting - The consent banner setting.

- NAME
- DESCRIPTION
- CONSTRUCTOR
- MODULE METHODS
  - Infoblox::Grid->consent_banner_setting( )
- METHODS
  - enable( )
  - message( )
- AUTHOR
- SEE ALSO
- COPYRIGHT

NAME

Infoblox::Grid::ConsentBannerSetting - The consent banner setting.

DESCRIPTION

The consent banner setting object controls the banner displayed on the login page of Grid Manager.

CONSTRUCTOR

```
my $consent = Infoblox::Grid::ConsentBannerSetting->new(
    enable    => "true" | "false", #Mandatory
    message   => $string,          #Optional / Default is empty
);
```

MODULE METHODS

The following functions are available to apply to a consent banner setting object.

Infoblox::Grid->consent_banner_setting( )

Use this function to specify the consent banner settings for the Grid. See Infoblox::Grid->consent_banner_setting() for parameters and return values.

Example

```
 my $grid = $session->get(object => 'Infoblox::Grid');

 my $banner = Infoblox::Grid::ConsentBannerSetting->new(enable => 'true', message => 'Consent
message');
 $grid->consent_banner_setting($banner);
 my $response = $session->modify($grid);
 unless($response) {
      die("Modify grid failed: ",
            session->status_code() . ":" . session->status_detail());
 }
 print "Grid banner setting modified successfully\n";
```

METHODS

This section describes all the methods that you can use to set and retrieve the attribute values of a consent banner setting object.

enable( )

Use this method to set the enable flag.

Parameter

Specify "true" to enable the banner or "false" to disable it.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get enable
my $enable = $banner->enable();
#Modify enable
$banner->enable("true");
```

message( )

Use this method to set the message that will be displayed on the login page of Grid Manager.

Parameter

A string containing the message.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get message
my $message = $banner->message();
#Modify message
$banner->message("Test message");
```

AUTHOR

Infoblox Inc. http://www.infoblox.com/

SEE ALSO

Infoblox::Grid

COPYRIGHT

Copyright (c) 2017 Infoblox Inc.

Infoblox::Grid::CSVImportStatus - CSV import status object

- NAME
- DESCRIPTION
- MODULE METHODS
  - Infoblox::Session->get( )
- METHODS
  - admin( )
  - end_time( )
  - file_name( )
  - import_id( )
  - lines_failed( )
  - lines_processed( )
  - lines_total( )
  - lines_warning( )
  - start_time( )
  - status( )
- SAMPLE CODE
- AUTHOR
- SEE ALSO
- COPYRIGHT

NAME

Infoblox::Grid::CSVImportStatus - CSV import status object

DESCRIPTION

This object is used to represent the status of the current CSV import task. All methods are read-only.

MODULE METHODS

The following functions can be applied to a Grid::CSVImportStatus object.

Infoblox::Session->get( )

Use this method to retrieve matching objects from the Infoblox appliance. See Infoblox::Session->get() for parameters and return values.

Key Reference

```
Apply the following attributes to get a specific CSV import status object

 object    - Required. It must be set to "Infoblox::Grid::CSVImportStatus".
 import_id - Optional. ID of the import task that the status object is describing. If not specified,
             the status of the first import task is returned.
```

Example

```
my $import_id = $session->import_data(
                                      type => 'csv',
                                      path => "/tmp/zone.csv",
                                      separator   => 'semicolon',
                                     );

unless($import_id){
    die("Import failed: ",
        Infoblox::status_code(). ":" .Infoblox::status_detail());
}

my @retrieved_objs = $session->get(
    object    => "Infoblox::Grid::CSVImportStatus",
    import_id => $import_id,
    );
```

METHODS

This section describes all the methods that you can use to retrieve the status of a CSV import operation.

admin( )

Use this method to retrieve the name of the administrator that started the import.

Returns

The method returns the attribute value.

Example

```
#Get admin
my $admin = $service_status->admin();
```

end_time( )

Use this method to retrieve the time the CSV import ended.

Returns

The method returns the attribute value.

Example

```
#Get end_time
my $end_time = $service_status->end_time();
```

file_name( )

Use this method to retrieve the name of the file used for the CSV import.

Returns

The method returns the attribute value.

Example

```
#Get file_name
my $file_name = $service_status->file_name();
```

import_id( )

Use this method to retrieve the ID of the import task that the status object is describing.

Returns

The method returns the attribute value.

Example

```
#Get import_id
my $import_id = $service_status->import_id();
```

lines_failed( )

Use this method to retrieve the number of lines that had failures in the CSV file used for the current CSV import.

Returns

The method returns the attribute value.

Example

```
#Get lines_failed
my $lines_failed = $service_status->lines_failed();
```

lines_processed( )

Use this method to retrieve the number of lines processed in the CSV file used for the current CSV import.

Returns

The method returns the attribute value.

Example

```
#Get lines_processed
my $lines_processed = $service_status->lines_processed();
```

lines_total( )

Use this method to retrieve the total number of lines read from the CSV file during the current CSV import.

Returns

The method returns the attribute value.

Example

```
#Get lines_total
my $lines_total = $service_status->lines_total();
```

lines_warning( )

Use this method to retrieve the number of lines that had warnings in the CSV file used for the current CSV import.

Returns

The method returns the attribute value.

Example

```
#Get lines_warning
my $lines_warning = $service_status->lines_warning();
```

start_time( )

Use this method to retrieve the time the CSV import started.

Returns

The method returns the attribute value.

Example

```
#Get start_time
my $start_time = $service_status->start_time();
```

status( )

Use this method to retrieve the status of the current CSV import. The returned status is one of the following: 'Uploaded', 'Pending', 'Running', 'Failed', 'Stopped' or 'Completed'.

Returns

The method returns the attribute value.

Example

```
#Get status
my $status = $service_status->status();
```

SAMPLE CODE

The following sample code demonstrates how to export / import data by using CSV files.

```
#PROGRAM STARTS: Include all the modules that will be used
use strict;
use Infoblox;

#Create a session to the Infoblox appliance
my $host_ip =  "192.168.1.2";
my $session = Infoblox::Session->new(
                                     master   => $host_ip,
                                     username => "admin",
                                     password => "infoblox"
                                    );
unless($session){
    die("Constructor for session failed: ",
        Infoblox::status_code(). ":" . Infoblox::status_detail());
}
print "Session created successfully.\n";

#Create a DNS zone
my $memberns1 = Infoblox::DNS::Member->new(
                                           name     => "infoblox.localdomain",
                                           ipv4addr => $host_ip,
```

```
                                          lead     => "false",
                                          stealth  => "false"
                                         );

#Add the zone
my $firstzone = Infoblox::DNS::Zone->new(
                                         name        => "test.com",
                                         email       => "admin\@infoblox.com",
                                         comment     => "add a zone test.com",
                                         primary     => $memberns1,
                                        );

unless($firstzone){
    die("Construct test.com zone object failed: ",
        Infoblox::status_code(). ":" .Infoblox::status_detail());
}
print "test.com zone object created successfully.\n";

$session->add($firstzone)
  or die("Add zone for test.com failed: ",
         $session->status_code(). ":" .$session->status_detail());

print"Zone test.com added successfully.\n";

#Export the newly created zone
$session->export_data(
                      type        => 'csv',
                      path        => '/tmp/zone.csv',
                      object_type => 'Infoblox::DNS::Zone',
                      separator   => 'semicolon',

                      # search parameters
                      name        => '.*test.com',
                                          )
                         or die("Export zone for test.com failed: ",
                                $session->status_code(). ":" .$session->status_detail());

# Wait for the export to complete
sleep(5);

print "The zone was exported\n";

# Remove the zone from the database
$session->remove($firstzone)
  or die("Remove zone for test.com failed: ",
         $session->status_code(). ":" .$session->status_detail());

print "The zone was removed\n";

# Import the zone by using the CSV file that was exported

my $import_id = $session->import_data(
                                      type => 'csv',
                                      path => "/tmp/zone.csv",
                                      separator   => 'semicolon',
                                     );

unless($import_id){
    die("Import failed: ",
        Infoblox::status_code(). ":" .Infoblox::status_detail());
}

# Wait for the import to complete
sleep(5);

my $status = $session->get(
                           object  => 'Infoblox::Grid::CSVImportStatus',
                          );

unless($status){
    die("Failed to fetch the import status: ",
        Infoblox::status_code(). ":" .Infoblox::status_detail());
}

print "The zone was re-imported\n";

# The code should be cycling to sleep, fetch and wait until the import is finished.
unless($status->status() eq 'Completed'){
    die("The import took too long to complete: " . $status->status());
```

```
}

unless($status->lines_failed() == 0){
    die("There were some import failures: " . $status->lines_failed());
}

print "The zone re-import was successful\n";

# The zone was re-imported. If we import the file again without merging,
# an error occurs.

$import_id = $session->import_data(
                                    type => 'csv',
                                    path => "/tmp/zone.csv",
                                    separator   => 'semicolon',
                                  );

unless($import_id){
    die("Import failed: ",
        Infoblox::status_code(). ":" .Infoblox::status_detail());
}

# Wait for the import to complete
sleep(5);

print "The zone re-re-import was completed\n";

my $status = $session->get(
                            object  => 'Infoblox::Grid::CSVImportStatus',
                           );

unless($status){
    die("Failed to fetch the import status: ",
        Infoblox::status_code(). ":" .Infoblox::status_detail());
}

# The code should be cycling to sleep, fetch and wait until the import is finished.
unless($status->status() eq 'Completed'){
    die("The import took too long to complete: " . $status->status());
}

if($status->lines_failed() == 0){
    die("There were no import failures: " . $status->lines_failed());
}

print "The zone re-re-import correctly failed\n";

# Review the  failure
$session->export_data(
                      type        => 'csv_error_log',
                      path        => '/tmp/error_log.txt',
                      import_id   => $import_id,
                     )
  or die("Export error log failed: ",
         $session->status_code(). ":" .$session->status_detail());

# The failure is in this file
print "Error log for the failed import\n-------\n";
print `cat /tmp/error_log.txt`;

#Remove the zone
$session->remove(
                 object => "Infoblox::DNS::Zone",
                 name   => "test.com",
                )
  or die("Remove zone test.com failed: ",
         $session->status_code(). ":" .$session->status_detail());

print "The zone was removed\n";

####PROGRAM ENDS####
```

AUTHOR

Infoblox Inc. http://www.infoblox.com/

SEE ALSO

Infoblox::Session,Infoblox::Session->get()

COPYRIGHT

Copyright (c) 2017 Infoblox Inc.

Infoblox::Grid::Dashboard - A Grid Dashboard object.

- NAME
- DESCRIPTION
- SESSION METHODS
  - Infoblox::Session->get( )
  - Infoblox::Session->modify( )
- METHODS
  - atp_critical_event_warning_threshold( )
  - atp_critical_event_critical_threshold( )
  - atp_major_event_warning_threshold( )
  - atp_major_event_critical_threshold( )
  - atp_warning_event_warning_threshold( )
  - atp_warning_event_critical_threshold( )
  - rpz_blocked_hit_warning_threshold( )
  - rpz_blocked_hit_critical_threshold( )
  - rpz_substituted_hit_warning_threshold( )
  - rpz_substituted_hit_warning_threshold( )
  - rpz_passthru_event_warning_threshold( )
  - rpz_passthru_event_critical_threshold( )
  - analytics_tunneling_event_warning_threshold( )
  - analytics_tunneling_event_critical_threshold( )
- AUTHOR
- SEE ALSO
- COPYRIGHT

NAME

Infoblox::Grid::Dashboard - A Grid Dashboard object.

DESCRIPTION

The Grid Dashboard object provides configuration interface to a threshold values for warning and critical ATP, RPZ and Analytics events. These values are used to calculate ATP, RPZ and Analytics security statuses.

SESSION METHODS

This section describes all the methods in the Infoblox::Session module that you can apply to a Grid Dashboard object.

Infoblox::Session->get( )

Use this method to retrieve the matching Grid Dashboard object from the Infoblox appliance. See Infoblox::Session->get() for parameters and return values.

Example

```
my @retrieved_objs = $session->get(
    object => "Infoblox::Grid::Dashboard"
);
```

Infoblox::Session->modify( )

Use this method to modify a Grid Dashboard object in the Infoblox appliance. See Infoblox::Session->modify() for parameters and return values.

Example

```
#Modify atp_critical_event_critical_threshold value
$grid_dashboard->atp_critical_event_critical_threshold('10');
#Submit modification
my $response = $session->modify( $grid_dashboard );
```

METHODS

This section describes all the methods that you can use to configure and retrieve the attribute values of a Grid Dashboard object.

atp_critical_event_warning_threshold( )

Use this method to set or retrieve the Grid Dashboard warning threshold for ATP critical event.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

The valid value is an unsigned integer.

Returns

If you specified a parameter, the method returns 'true' when the modification succeeds, and returns 'false' when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get atp_critical_event_warning_threshold value
my $atp_critical_event_warning_threshold = $dashboard->atp_critical_event_warning_threshold();

#Modify atp_critical_event_warning_threshold value
$dashboard->atp_critical_event_warning_threshold(10);
```

atp_critical_event_critical_threshold( )

Use this method to set or retrieve the Grid Dashboard critical threshold for ATP critical event.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

The valid value is an unsigned integer.

Returns

If you specified a parameter, the method returns 'true' when the modification succeeds, and returns 'false' when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get atp_critical_event_critical_threshold value
my $atp_critical_event_critical_threshold = $dashboard->atp_critical_event_critical_threshold();

#Modify atp_critical_event_critical_threshold value
$dashboard->atp_critical_event_critical_threshold(10);
```

atp_major_event_warning_threshold( )

Use this method to set or retrieve the Grid Dashboard warning threshold for ATP major event.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

The valid value is an unsigned integer.

Returns

If you specified a parameter, the method returns 'true' when the modification succeeds, and returns 'false' when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get atp_major_event_warning_threshold value
my $atp_major_event_warning_threshold = $dashboard->atp_major_event_warning_threshold();

#Modify atp_major_event_warning_threshold value
$dashboard->atp_major_event_warning_threshold(10);
```

atp_major_event_critical_threshold( )

Use this method to set or retrieve the Grid Dashboard critical threshold for ATP major event.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

The valid value is an unsigned integer.

Returns

If you specified a parameter, the method returns 'true' when the modification succeeds, and returns 'false' when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get atp_major_event_critical_threshold value
my $atp_major_event_critical_threshold = $dashboard->atp_major_event_critical_threshold();

#Modify atp_major_event_critical_threshold value
$dashboard->atp_major_event_critical_threshold(10);
```

atp_warning_event_warning_threshold( )

Use this method to set or retrieve the Grid Dashboard warning threshold for ATP warning event.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

The valid value is an unsigned integer.

Returns

If you specified a parameter, the method returns 'true' when the modification succeeds, and returns 'false' when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get atp_warning_event_warning_threshold value
my $atp_warning_event_warning_threshold = $dashboard->atp_warning_event_warning_threshold();

#Modify atp_warning_event_warning_threshold value
$dashboard->atp_warning_event_warning_threshold(10);
```

atp_warning_event_critical_threshold( )

Use this method to set or retrieve the Grid Dashboard critical threshold for ATP warning event.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

The valid value is an unsigned integer.

Returns

If you specified a parameter, the method returns 'true' when the modification succeeds, and returns 'false' when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get atp_warning_event_critical_threshold value
my $atp_warning_event_critical_threshold = $dashboard->atp_warning_event_critical_threshold();

#Modify atp_warning_event_critical_threshold value
$dashboard->atp_warning_event_critical_threshold(10);
```

rpz_blocked_hit_warning_threshold( )

Use this method to set or retrieve the Grid Dashboard warning threshold for RPZ blocked hits.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

The valid value is an unsigned integer.

Returns

If you specified a parameter, the method returns 'true' when the modification succeeds, and returns 'false' when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get rpz_blocked_hit_warning_threshold value
my $rpz_blocked_hit_warning_threshold = $dashboard->rpz_blocked_hit_warning_threshold();

#Modify rpz_blocked_hit_warning_threshold value
$dashboard->rpz_blocked_hit_warning_threshold(10);
```

rpz_blocked_hit_critical_threshold( )

Use this method to set or retrieve the Grid Dashboard critical threshold for RPZ blocked hits.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

The valid value is an unsigned integer.

Returns

If you specified a parameter, the method returns 'true' when the modification succeeds, and returns 'false' when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get rpz_blocked_hit_critical_threshold value
my $rpz_blocked_hit_critical_threshold = $dashboard->rpz_blocked_hit_critical_threshold();

#Modify rpz_blocked_hit_critical_threshold value
$dashboard->rpz_blocked_hit_critical_threshold(10);
```

rpz_substituted_hit_warning_threshold( )

Use this method to set or retrieve the Grid Dashboard warning threshold for RPZ substituted hits.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

The valid value is an unsigned integer.

Returns

If you specified a parameter, the method returns 'true' when the modification succeeds, and returns 'false' when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get rpz_substituted_hit_warning_threshold value
my $rpz_substituted_hit_warning_threshold = $dashboard->rpz_substituted_hit_warning_threshold();

#Modify rpz_substituted_hit_warning_threshold value
$dashboard->rpz_substituted_hit_warning_threshold(10);
```

rpz_substituted_hit_warning_threshold( )

Use this method to set or retrieve the Grid Dashboard warning threshold for RPZ substituted hits.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

The valid value is an unsigned integer.

Returns

If you specified a parameter, the method returns 'true' when the modification succeeds, and returns 'false' when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get rpz_substituted_hit_warning_threshold value
my $rpz_substituted_hit_warning_threshold = $dashboard->rpz_substituted_hit_warning_threshold();

#Modify rpz_substituted_hit_warning_threshold value
$dashboard->rpz_substituted_hit_warning_threshold(10);
```

rpz_passthru_event_warning_threshold( )

Use this method to set or retrieve the Grid Dashboard warning threshold for RPZ passthru event.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

The valid value is an unsigned integer.

Returns

If you specified a parameter, the method returns 'true' when the modification succeeds, and returns 'false' when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get rpz_passthru_event_warning_threshold value
my $rpz_passthru_event_warning_threshold = $dashboard->rpz_passthru_event_warning_threshold();

#Modify rpz_passthru_event_warning_threshold value
$dashboard->rpz_passthru_event_warning_threshold(10);
```

rpz_passthru_event_critical_threshold( )

Use this method to set or retrieve the Grid Dashboard critical threshold for RPZ passthru event.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

The valid value is an unsigned integer.

Returns

If you specified a parameter, the method returns 'true' when the modification succeeds, and returns 'false' when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get rpz_passthru_event_critical_threshold value
my $rpz_passthru_event_critical_threshold = $dashboard->rpz_passthru_event_critical_threshold();

#Modify rpz_passthru_event_critical_threshold value
$dashboard->rpz_passthru_event_critical_threshold(10);
```

analytics_tunneling_event_warning_threshold( )

Use this method to set or retrieve the Grid Dashboard warning threshold for Analytics tunneling event.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

The valid value is an unsigned integer.

Returns

If you specified a parameter, the method returns 'true' when the modification succeeds, and returns 'false' when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get analytics_tunneling_event_warning_threshold value
```

```
 my $analytics_tunneling_event_warning_threshold = $dashboard-
>analytics_tunneling_event_warning_threshold();

 #Modify analytics_tunneling_event_warning_threshold value
 $dashboard->analytics_tunneling_event_warning_threshold(10);
```

analytics_tunneling_event_critical_threshold( )

Use this method to set or retrieve the Grid Dashboard critical threshold for Analytics tunneling event.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

The valid value is an unsigned integer.

Returns

If you specified a parameter, the method returns 'true' when the modification succeeds, and returns 'false' when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
 #Get analytics_tunneling_event_critical_threshold value
 my $analytics_tunneling_event_critical_threshold = $dashboard-
>analytics_tunneling_event_critical_threshold();

 #Modify analytics_tunneling_event_critical_threshold value
 $dashboard->analytics_tunneling_event_critical_threshold(10);
```

AUTHOR

Infoblox Inc. http://www.infoblox.com/

SEE ALSO

Infoblox::Session, Infoblox::Session->get(), Infoblox::Session->modify().

COPYRIGHT

Copyright (c) 2017 Infoblox Inc.

Infoblox::Grid::DataCollection::BlacklistedCluster - A Data Collection blacklisted cluster object.

- NAME
- DESCRIPTION
- CONSTRUCTOR
- SESSION METHODS
- MODULE METHODS
  - Infoblox::Grid::DataCollection::Cluster->blacklisted_clusters( )
- METHODS
  - comment( )
  - name( )
  - unique_id( )
- AUTHOR
- SEE ALSO
- COPYRIGHT

NAME

Infoblox::Grid::DataCollection::BlacklistedCluster - A Data Collection blacklisted cluster object.

DESCRIPTION

The Data Collection blacklisted cluster object is used to configure Data Collection cluster blacklist.

CONSTRUCTOR

```
my $blacklisted_cluster = Infoblox::Grid::DataCollection::BlacklistedCluster->new(
    unique_id => $string, #Required
    comment   => $string, #Optional / Default is undefined
    name      => $string, #Optional / Default is undefined
);
```

SESSION METHODS

The object does not support any session methods.

MODULE METHODS

The following functions are available to be applied to a Data Collection blacklisted cluster object.

Infoblox::Grid::DataCollection::Cluster->blacklisted_clusters( )

Use this method to set or retrieve the list of clusters which are prevented from activation. See Infoblox::Grid::DataCollection::Cluster->blacklisted_clusters() for parameters and return values.

Example

```
#Get blacklisted_clusters value
my $blacklisted_clusters = $object->blacklisted_clusters();

#Modify blacklisted_clusters value
$object->blacklisted_clusters([$cluster1, $cluster2]);
```

METHODS

This section describes all the methods that can be used to configure and retrieve the attribute values of a Data Collection blacklisted cluster object.

comment( )

Use this method to set or retrieve the blacklisted cluster comment.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

The valid value is a desired cluster comment in string format.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get comment value
my $comment = $object->comment();

#Modify comment value
$object->comment('cluster1 comment');
```

name( )

Use this method to retrieve the blacklisted cluster name. This is a read-only attribute.

Omit the parameter to retrieve the attribute value.

Parameter

None

Returns

The method returns the attribute value.

Example

```
#Get name value
my $name = $object->name();
```

unique_id( )

Use this method to set or retrieve the blacklisted cluster unique hexadecimal identifier.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

The valid value is a desired cluster unique hexadecimal identifier in string format.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get unique_id value
my $unique_id = $object->unique_id();

#Modify unique_id value
$object->unique_id('ABCD');
```

AUTHOR

Infoblox Inc. http://www.infoblox.com/

SEE ALSO

Infoblox::Grid::DataCollection::Cluster, Infoblox::Grid::DataCollection::Cluster->blacklisted_clusters()

COPYRIGHT

Copyright (c) 2017 Infoblox Inc.

Infoblox::Grid::DataCollection::Cluster - A data collection cluster object.

- NAME
- DESCRIPTION
- SESSION METHODS
  - Infoblox::Session->get( )
  - Infoblox::Session->modify( )
  - Infoblox::Session->search( )
- METHODS
  - blacklisted_clusters( )
  - comment( )
  - datacollector_vms( )
  - disable( )
  - enable_registration( )
  - last_activation_time( )
  - name( )
  - registration_time( )
  - unique_id( )
- AUTHOR
- SEE ALSO
- COPYRIGHT

NAME

Infoblox::Grid::DataCollection::Cluster - A data collection cluster object.

DESCRIPTION

The data collection cluster object provides information and configuration for data collection feature.

SESSION METHODS

This section describes all the methods in the Infoblox::Session module that you can apply to a data collection cluster object.

Infoblox::Session->get( )

Use this method to retrieve all the matching objects from the Infoblox appliance. See Infoblox::Session->get() for parameters and return values.

Key Reference

Apply the following attributes to get a specific data collection cluster object:

```
comment   - Optional. The data collection cluster comment.
name      - Optional. The data collection cluster name.
unique_id - Optional. The data collection cluster unique hexadecimal identifier.
```

Example

```
#Get cluster with name 'cluster1'
my $object = $session->get(
    object => 'Infoblox::Grid::DataCollection::Cluster',
    name   => 'cluster1',
);
```

Infoblox::Session->modify( )

Use this method to modify an object in the Infoblox appliance. See Infoblox::Session->modify() for parameters and return values.

Example

```
#Modify disable value
$object->disable('true');

#Submit modification
my $response = $session->modify($object);
```

Infoblox::Session->search( )

Use this method to retrieve all the matching objects from the Infoblox appliance. See Infoblox::Session->search() for parameters and return values.

Key Reference

Apply the following attributes to search for a data collection cluster object:

```
comment   - Optional. The data collection cluster comment (regexp);
name      - Optional. The data collection cluster name (regexp);
unique_id - Optional. The data collection cluster unique hexadecimal identifier (regexp).
```

Example

```
#Search for cluster that matches /cluster.*/ pattern
my @objects = $session->search(
    object => 'Infoblox::Grid::DataCollection::Cluster',
    name   => 'cluster.*',
);
```

METHODS

This section describes all the methods that can be used to configure and retrieve the attribute values of a data collection cluster object.

blacklisted_clusters( )

Use this method to set or retrieve the list of clusters that are prevented from activation.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

The valid value is an array of Infoblox::Grid::DataCollection::BlacklistedCluster objects.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get blacklisted_clusters value
my $blacklisted_clusters = $object->blacklisted_clusters();

#Modify blacklisted_clusters value
$object->blacklisted_clusters([$cluster1, $cluster2]);
```

comment( )

Use this method to set or retrieve the data collection cluster comment.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

The valid value is a desired comment in a string format.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get comment value
my $comment = $object->comment();

#Modify comment value
$object->comment('cluster comment');
```

datacollector_vms( )

Use this method to retrieve the list of data collector virtual machines. This is a read-only attribute.

Omit the parameter to retrieve the attribute value.

Parameter

None

Returns

The valid return value is an array of Infoblox::Grid::DataCollection::IPInfo objects.

Example

```
#Get datacollector_vms value
my $datacollector_vms = $object->datacollector_vms();
```

disable( )

Use this method to set or retrieve the flag that indicates whether the data collection cluster is enabled or disabled.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

Specify 'true' to enable the data collection cluster or 'false' to disable/deactivate it. The default value is 'false'.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get disable value
my $disable = $object->disable();

#Modify disable value
$object->disable('true');
```

enable_registration( )

Use this method to set or retrieve the flag that indicates whether the data collection cluster registration is enabled or disabled.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

Specify 'true' to enable the data collection cluster registration or 'false' to prevent it from registration. The default value is 'false'.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get enable_registration value
my $enable_registration = $object->enable_registration();

#Modify enable_registration value
$object->enable_registration('true');
```

last_activation_time( )

Use this method to retrieve the last activation time for the data collection cluster.

Omit the parameter to retrieve the attribute value.

Parameter

None

Returns

The method returns the attribute value, the number of seconds that have elapsed since January 1st, 1970 UTC.

Example

```
#Get last_activation_time value
my $last_activation_time = $object->last_activation_time();
```

name( )

Use this method to set or retrieve the name of the data collection cluster. This is a read-only attribute.

Omit the parameter to retrieve the attribute value.

Parameter

None

Returns

The method returns the attribute value.

Example

```
#Get name value
my $name = $object->name();
```

registration_time( )

Use this method to retrieve the time the data collection cluster was registered. This is a read-only attribute.

Omit the parameter to retrieve the attribute value.

Parameter

None

Returns

The method returns the attribute value, the number of seconds that have elapsed since January 1st, 1970 UTC.

Example

```
#Get registration_time value
my $registration_time = $object->registration_time();
```

unique_id( )

Use this method to set or retrieve the unique identifier for the data collection cluster.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

The valid value is a desired hexadecimal identifier in a string format.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get unique_id value
my $unique_id = $object->unique_id();

#Modify unique_id value
$object->unique_id('ABCD');
```

AUTHOR

Infoblox Inc. http://www.infoblox.com/

SEE ALSO

Infoblox::Grid::DataCollection::BlacklistedCluster, Infoblox::Grid::DataCollection::IPInfo,

---

COPYRIGHT

Copyright (c) 2017 Infoblox Inc.

Infoblox::Grid::DataCollection::IPInfo - A data collection IP information object.

- NAME
- DESCRIPTION
- SESSION METHODS
- MODULE METHODS
  - Infoblox::Grid::DataCollection::Cluster->datacollector_vms( )
- METHODS
  - ip_address( )
  - nat_enabled( )
  - nat_group( )
  - nat_ip( )
- AUTHOR
- SEE ALSO
- COPYRIGHT

NAME

Infoblox::Grid::DataCollection::IPInfo - A data collection IP information object.

DESCRIPTION

The Data Collection IP information object is used to configure the list of data collector virtual machines in the data collection cluster.

SESSION METHODS

The object does not support any session methods.

MODULE METHODS

The following functions are available to be applied to a data collection IP information object.

Infoblox::Grid::DataCollection::Cluster->datacollector_vms( )

Use this method to retrieve the list of data collector virtual machines. This is a read-only attribute. See Infoblox::Grid::DataCollection::Cluster->datacollector_vms() for parameters and return values.

Example

```
#Get datacollector_vms value
my $datacollector_vms = $object->datacollector_vms();
```

METHODS

This section describes all the methods that can be used to retrieve the attribute values of a data collection IP information.

Note that this is a read-only object.

ip_address( )

Use this method to retrieve the virtual machine IP address. This is a read-only attribute.

Omit the parameter to retrieve the attribute value.

Parameter

None

Returns

The method returns the attribute value.

Example

```
#Get ip_address value
my $ip_address = $object->ip_address();
```

nat_enabled( )

Use this method or retrieve the flag that indicates whether the NAT translation of the virtual machine IP address is enabled or disabled. This is a read-only attribute.

Omit the parameter to retrieve the attribute value.

Parameter

None

Returns

The method returns the attribute value.

Example

```
#Get enable_nat value
my $enable_nat = $object->enable_nat();
```

nat_group( )

Use this method to retrieve the virtual machine NAT group. This is a read-only attribute.

Omit the parameter to retrieve the attribute value.

Parameter

None

Returns

The method returns the attribute value.

Example

```
#Get nat_group value
my $nat_group = $object->nat_group();
```

nat_ip( )

Use this method to retrieve the virtual machine external IP address. This is a read-only attribute.

Omit the parameter to retrieve the attribute value.

Parameter

None

Returns

The method returns the attribute value.

Example

```
#Get nat_ip value
my $nat_ip = $object->nat_ip();
```

AUTHOR

Infoblox Inc. http://www.infoblox.com/

SEE ALSO

Infoblox::Grid::DataCollection::Cluster, Infoblox::Grid::DataCollection::Cluster->datacollector_vms()

COPYRIGHT

Copyright (c) 2017 Infoblox Inc.

Infoblox::Grid::DateTime - Grid date and time object.

- NAME
- DESCRIPTION
- CONSTRUCTOR
- SESSION METHODS
  - Infoblox::Session->get( )
  - Infoblox::Session->modify( )
- METHODS
  - date( )
  - time( )
- SAMPLE CODE
- AUTHOR
- SEE ALSO
- COPYRIGHT

NAME

Infoblox::Grid::DateTime - Grid date and time object.

DESCRIPTION

Grid DateTime object is used to set the date and the time for an Infoblox appliance.

CONSTRUCTOR

```
 #Constructor for an grid DateTime object
 my $date_time = Infoblox::Grid::DateTime->new (
        date     =>  $date            #Optional / Default is undefined and continue to ues current system
date
        time     =>  $time            #Optional / Default is undefined and continue to use current system
time
     );
```

SESSION METHODS

This section describes all the methods in Infoblox::Session that you can apply to a grid DateTime object.

Infoblox::Session->get( )

Use this method to retrieve existing objects from the Infoblox appliance. See Infoblox::Session->get() for parameters and return values.

Key Reference

```
Apply the following attributes to get a grid DateTime object:

none.
```

Example

```
my @result_array = $session->get( "object" => "Infoblox::Grid::DateTime");
```

Infoblox::Session->modify( )

Use this method to modify objects in the Infoblox appliance. See Infoblox::Session->modify() for parameters and return values.

Example

```
# Use this method to modify the date and time of Infoblox appliance.
$object->date( "11/04/2007" );
$object->time( "15:00:00" );
# Submit modification
my $response = $session->modify( $object );
```

METHODS

This section describes all the methods that you can use to configure and retrieve the attribute value of a date time.

date( )

Use this method to set or retrieve date for an Infoblox appliance.

Parameter

Desired date in mm/dd/yy format eg (11/04/2007).

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

Example

```
#Get date
my $date = $object->date();
#Modify ntp_enabled
$object->date( "11/04/2007" );
```

time( )

Use this method to set or retrieve the time for an Infoblox appliance. Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

Desired time in hh/mm/ss format eg. (11:59:59).

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

Example

```
#Get time
my $time = $object->time();
#Modify time
$object->time("15:00:00");
```

SAMPLE CODE

The following sample code demonstrates different operations that can be applied to an object such as create, modify, and remove an object. This sample code also includes error handling for the operations.

```
#PROGRAM STARTS: Include all the modules that will be used
use strict;
use Infoblox;

#Create a session to the Infoblox appliance
my $session = Infoblox::Session->new(
    master   => "192.168.1.2",
    username => "admin",
    password => "infoblox"
);
unless ($session) {
   die("Construct session failed: ",
       Infoblox::status_code() . ":" . Infoblox::status_detail());
}
print "Session created successfully\n";

my @result_array = $session->get( "object" => "Infoblox::Grid::DateTime");
if( defined @result_array  and scalar( @result_array ) > 0 )
 {
        my $object = $result_array[0];
        unless ($object) {
                die("Get on Date Time failed: ",
         $session->status_code() . ":" . $session->status_detail());
        }

$object->date( "11/07/2007" );
$object->time( "15:00:00" );
$session->modify($object)
   or die("Modify on Date Time failed: ",
            $session->status_code() . ":" . $session->status_detail());
print "Grid Date Time modified successfully \n";
}
else
{
        print "No  DateTime Structure found !!!\n";
```

```
}

####PROGRAM ENDS####
```

AUTHOR

Infoblox Inc. http://www.infoblox.com/

SEE ALSO

Infoblox::Session, Infoblox::Session->get(), Infoblox::Session->modify()

COPYRIGHT

Copyright (c) 2017 Infoblox Inc.

Infoblox::Grid::DBSnapshot - OneDB snapshot information object.

- NAME
- DESCRIPTION
- SESSION METHODS
  - Infoblox::Session->get( )
- METHODS
  - comment( )
  - timestamp( )
- AUTHOR
- SEE ALSO
- COPYRIGHT

NAME

Infoblox::Grid::DBSnapshot - OneDB snapshot information object.

DESCRIPTION

The object provides information and a descriptive comment about the last OneDB snapshot.

SESSION METHODS

This section describes all the methods in a Session module that you can apply to a DBSnapshot object. Note that the object supports only the get method since there is only one such object in the database.

Infoblox::Session->get( )

Use this method to retrieve all the matching objects from the Infoblox appliance. See Infoblox::Session->get() for parameters and return values.

Example

```
my @retrieved_objs = $session->get(
    object => 'Infoblox::Grid::DBSnapshot',
);
```

METHODS

This section describes all the methods that you can use to set or retrieve the attribute values of the object.

comment( )

Use this method to retrieve the descriptive comment.

Parameter

None

Returns

The method returns attribute value.

Example

```
#get comment value
my $value = $object->comment();
```

timestamp( )

Use this method to retrieve the time when the last OneDB snapshot was created. This is a read-only attribute.

Parameter

None

Returns

This method returns a attribute value. The number of seconds that have elapsed since January 1, 1970 UTC.

Example

```
#get timestamp value
my $value = $object->timestamp();
```

AUTHOR

Infoblox Inc. http://www.infoblox.com/

SEE ALSO

Infoblox::Session, Infoblox::Session->get()

COPYRIGHT

Copyright (c) 2017 Infoblox Inc.

Infoblox::Grid::DHCP - Grid DHCP services object.

- NAME
- DESCRIPTION
- CONSTRUCTOR
- SESSION METHODS
  - Infoblox::Session->get( )
  - Infoblox::Session->modify( )
- METHODS
  - authority( )
  - bootfile( )
  - bootserver( )
  - capture_hostname( )
  - ddns_generate_hostname( )
  - ddns_retry_interval( )
  - ddns_server_always_updates( )
  - ddns_ttl( )
  - ddns_domainname( )
  - ddns_update_fixed_addresses( )
  - ddns_use_option81( )
  - deny_bootp( )
  - disable_all_nac_filters( )
  - dns_update_style( )
  - format_log_option_82( )
  - email_list( )
  - enable_ddns( )
  - enable_dhcp_thresholds( )
  - enable_email_warnings( )
  - enable_fingerprint( )
  - enable_gss_tsig( )
  - enable_ifmap( )
  - enable_ifmap_publishing( )
  - enable_hostname_rewrite( )
  - enable_roaming_hosts( )
  - enable_snmp_warnings( )
  - grid( )
  - gss_tsig_keys( )
  - high_water_mark( )
  - high_water_mark_reset( )
  - hostname_rewrite_policy( )
  - ifmap_delete_option( )
  - ifmap_ip_mac_lifetime( )
  - ifmap_port( )
  - ifmap_protocol( )
  - ifmap_url( )
  - ifmap_server_ca_cert( )
  - ignore_id( )
  - ignore_mac_addresses( )
  - ignore_dhcp_option_list_request( )
  - immediate_fa_configuration( )
  - ipv6_capture_hostname( )
  - ipv6_ddns_enable_option_fqdn( )
  - ipv6_ddns_server_always_updates( )
  - ipv6_ddns_domainname( )
  - ipv6_ddns_ttl( )
  - ipv6_default_prefix( )
  - ipv6_dns_update_style( )
  - ipv6_domain_name( )
  - ipv6_domain_name_servers( )
  - ipv6_enable_ddns( )
  - ipv6_enable_gss_tsig( )
  - ipv6_enable_lease_scavenge( )
  - ipv6_remember_expired_client_association( )
  - ipv6_enable_retry_updates( )
  - ipv6_generate_hostname( )
  - ipv6_gss_tsig_keys( )
  - ipv6_kdc_server( )
  - ipv6_lease_scavenge_time( )
  - ipv6_microsoft_code_page( )
  - ipv6_options( )
  - ipv6_prefixes( )
  - ipv6_recycle_leases( )
  - ipv6_retry_updates_interval( )
  - ipv6_txt_record_handling( )
  - ipv6_update_dns_on_lease_renewal( )

- kdc_server( )
- lease_logging_member( )
- lease_per_client_settings( )
- lease_scavenge_time( )
- log_lease_events( )
- logic_filters( )
- low_water_mark( )
- low_water_mark_reset( )
- microsoft_code_page( )
- nextserver( )
- option60_match_rules( )
- options( )
- override_immediate_fa_configuration( )
- ping_count( )
- ping_timeout( )
- preferred_lifetime( )
- protocol_hostname_rewrite_policies( )
- pxe_lease_time( )
- recycle_leases( )
- restart_setting( )
- retry_ddns_updates( )
- syslog_facility( )
- txt_record_handling( )
- update_dns_on_lease_renewal( )
- valid_lifetime( )
- validate_ifmap_ca_cert( )

- SAMPLE CODE
- AUTHOR
- SEE ALSO
- COPYRIGHT

NAME

Infoblox::Grid::DHCP - Grid DHCP services object.

DESCRIPTION

When you configure an Infoblox appliance as a full-featured DHCP server, you can set operational parameters at the grid, member, network, DHCP range and individual IP address levels. This class supports grid-level DHCP services management and network configuration. It implements methods for configuring grid-level DHCP parameters, such as DHCP general properties, ping and DHCP lease settings, Bootstrap Protocol (BOOTP) properties, custom DHCP options, and DHCP logging.

CONSTRUCTOR

The Infoblox::Grid::DHCP object is automatically generated upon the successful installation of the Infoblox appliance. It does not require manual construction.

SESSION METHODS

This section describes all the methods in the Infoblox::Session module that you can apply to a grid DHCP object.

Infoblox::Session->get( )

Use this method to retrieve matching objects from the Infoblox appliance. See Infoblox::Session->get() for parameters and return values.

Key Reference

```
Apply the following attributes to get a specific grid DHCP object:

 grid - Required. A grid name in string format.
```

Example

```
my @retrieved_objs = $session->get(
    object => "Infoblox::Grid::DHCP",
    grid   => "Infoblox"
    );
```

Infoblox::Session->modify( )

Use this method to modify objects in the Infoblox appliance. See Infoblox::Session->modify() for parameters and return values.

Example

```
#Use method to modify authority.
$griddhcp->authority("true");
#Submit modification
my $response = $session->modify( $griddhcp );
```

METHODS

This section describes all the methods that you can use to configure and retrieve the attribute values of the DHCP properties of a grid.

authority( )

Use this method to set or retrieve the grid-level authority flag. This flag specifies whether a DHCP server is authoritative for a domain.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

Specify "true" to set the authority flag or "false" to deactivate/unset it. The default value is "false".

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get authority
my $authority = $griddhcp->authority();
#Modify authority
$griddhcp->authority("true");
```

bootfile( )

Some DHCP clients use BOOTP (bootstrap protocol) or include the boot file name option in their DHCPREQUEST messages. Use this method to set or retrieve a boot file name.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

The name of the file that the client must download.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get bootfile
my $bootfile = $griddhcp->bootfile();
#Modify bootfile
$griddhcp->bootfile("bootfile.txt");
```

bootserver( )

Use this method to set or retrieve the name of the server on which a boot file is stored.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

The boot server IP address or name in FQDN (Fully Qualified Domain Name) format. The FQDN consists of the host name followed by the domain name (example: abc.com). A boot server name can have a maximum of 256 bytes.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get bootserver
my $bootserver = $griddhcp->bootserver();
#Modify bootserver
$griddhcp->bootserver("hostname.com");
```

capture_hostname( )

Use this method to set or retrieve the grid-level capture_hostname flag. Set this flag to capture the hostname and lease time when assigning a fixed address.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

Specify "true" to enable this feature or "false" to disable it. The default value is "false".

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get capture_hostname
my $capture_hostname = $griddhcp->capture_hostname();
#Modify disable
$griddhcp->capture_hostname("true");
```

ddns_generate_hostname( )

Use this method to set or retrieve the grid-level ddns_generate_hostname flag. Set this flag to enable a DHCP server to generate a host name and update DNS with this host name when it receives a DHCP REQUEST message that does not include a host name.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

Specify "true" to enable this feature or "false" to disable it. The default value is "false".

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get ddns_generate_hostname
my $ddns_generate_hostname = $griddhcp->ddns_generate_hostname();
#Modify disable
$griddhcp->ddns_generate_hostname("true");
```

ddns_retry_interval( )

Use this method to set or retrieve the grid-level ddns_retry_interval value. Set the retry interval when the DHCP server makes repeated attempts to send DDNS updates to a DNS server.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

A 32-bit integer (range from 0 to 4294967295) that represents the retry interval in minutes. The default retry interval is five minutes.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get ddns_retry_interval
my $ddns_retry_interval = $griddhcp->ddns_retry_interval();
#Modify ddns_retry_interval
$griddhcp->ddns_retry_interval(10);
```

ddns_server_always_updates( )

Use this method to set or retrieve the grid-level ddns_server_always_updates flag. Set this flag to allow only the DHCP server to update DNS, regardless of requests from DHCP clients.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

Specify "true" to set the ddns_server_always_updates flag or "false" to deactivate/unset it. The default value is "false".

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get ddns_server_always_updates
my $ddns_server_always_updates = $griddhcp->ddns_server_always_updates();
#Modify ddns_server_always_updates
$griddhcp->ddns_server_always_updates("true");
```

ddns_ttl( )

Use this method to set or retrieve the grid-level ddns_ttl value. The DDNS TTL (Dynamic DNS Time To Live) value specifies the number of seconds a DNS record is cached.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

A 32-bit integer (range from 0 to 4294967295) that specifies the number of seconds a DNS record is cached. Default value is half of lease time.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get ddns_ttl
my $ddns_ttl = $griddhcp->ddns_ttl();
#Modify ddns_ttl
$griddhcp->ddns_ttl(1000);
```

ddns_domainname( )

Use this method to set or retrieve the grid-level DDNS domain name value.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

The DDNS domain name in FQDN (Fully Qualified Domain Name) format.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get ddns_domainname
my $ddns_domainname = $griddhcp->ddns_domainname();
#Modify ddns_domainname
$griddhcp->ddns_domainname("domain.com");
```

ddns_update_fixed_addresses( )

Use this method to set or retrieve the grid-level ddns_update_fixed_addresses flag. Set this flag to enable the DHCP server to update the A and PTR records of clients with fixed addresses.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

Specify "true" to enable the DHCP server to update the A and PTR records of clients with fixed addresses, or specify "false" to disable this feature. The default value is "false".

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get ddns_update_fixed_addresses
my $ddns_update_fixed_addresses = $griddhcp->ddns_update_fixed_addresses();
#Modify ddns_update_fixed_addresses
$griddhcp->ddns_update_fixed_addresses("true");
```

ddns_use_option81( )

Use this method to set or retrieve the grid-level ddns_use_option81 flag. Set this flag to enable support for option 81.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

Specify "true" to enable support for option 81 or "false" to disable it. The default value is "false".

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get ddns_use_option81
my $ddns_use_option81 = $griddhcp->ddns_use_option81();
#Modify ddns_use_option81
$griddhcp->ddns_use_option81("true");
```

deny_bootp( )

Use this method to set or retrieve the grid-level deny_bootp flag. Set this flag to enable a BOOTP server to deny BOOTP request.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

Specify "true" to enable this feature or "false" to disable it. The default value is "false".

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get deny_bootp
my $deny_bootp = $griddhcp->deny_bootp();
#Modify deny_bootp
$griddhcp->deny_bootp("true");
```

disable_all_nac_filters( )

Use this method to enable/disable the use of all NAC filters.

Parameter

Specify "true" to disable the use of all NAC filters or "false" to enable it. The default value is "false".

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get disable_all_nac_filters
my $disable_all_nac_filters = $griddhcp->disable_all_nac_filters();
#Modify disable_all_nac_filters
$griddhcp->disable_all_nac_filters("true");
```

dns_update_style( )

Use this method to set or retrieve the update style for dynamic DNS updates (DHCPv4).

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

Valid values are 'INTERIM' and 'STANDARD'.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get dns_update_style value
my $value = $object->dns_update_style();

#Modify dns_update_style value
$object->dns_update_style('STANDARD');
```

format_log_option_82( )

Use this method to set or retrieve the format option for Option 82 logging. The possible values are 'HEX' or 'TEXT'.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

Switch between HEX and human readable format of agent.remote-id/circuit-id in syslog for Option 82. Default is 'HEX'.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get format of option82 logging
my $option82_format = $griddhcp->format_log_option_82();
#Modify format of option82 logging
$griddhcp->format_log_option_82('TEXT');
```

email_list( )

Use this method to specify or retrieve the grid-level email_list value. Specify an e-mail address to which you want the Infoblox appliance to send e-mail notifications when the DHCP address usage for the grid crosses a threshold. You can create a list of several e-mail addresses.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

Array reference with the e-mail values.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get email_list
my $email_list = $griddhcp->email_list();
#Modify email_list
$griddhcp->email_list(["admin\@infoblox.com"]);
```

enable_ddns( )

Use this method to set or retrieve the grid-level enable_ddns flag. Set this flag to enable DHCP servers in a grid to send DDNS updates.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

Specify "true" to enable support for DDNS updates or "false" to disable it. The default value is "false".

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get enable_ddns
```

```
my $enable_ddns= $griddhcp->enable_ddns();
#Modify enable_ddns
$griddhcp->enable_ddns("true");
```

enable_dhcp_thresholds( )

Use this method to set or retrieve the grid-level enable_dhcp_thresholds flag. Thresholds represent the watermarks above or below which address usage in a network is unexpected and might warrant your attention.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

Specify "true" to enable DHCP thresholds or "false" to disable them. The default value is "false".

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get enable_dhcp_thresholds
my $enable_dhcp_thresholds = $griddhcp->enable_dhcp_thresholds();
#Modify enable_dhcp_thresholds
$griddhcp->enable_dhcp_thresholds("true");
```

enable_email_warnings( )

Use this method to set or retrieve the grid-level enable_email_warnings flag. If you enable this feature and DHCP threshold is enabled, the Infoblox appliance sends an e-mail notification to an administrator if DHCP address usage crosses a high or low watermark threshold.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

Specify "true" to set the enable_email_warnings flag or "false" to deactivate/unset it. The default value is "false".

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get enable_email_warnings
my $enable_email_warnings = $griddhcp->enable_email_warnings();
#Modify enable_email_warnings
$griddhcp->enable_email_warnings("true");
```

enable_fingerprint( )

Use this method to set or retrieve the Grid-level enable_fingerprint flag. If you enable this feature, the server will match a fingerprint for incoming lease requests.

Parameter

Specify "true" to set the enable_fingerprint flag or "false" to deactivate/unset it. The default value is "true".

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get enable_fingerprint
my $enable_fingerprint = $griddhcp->enable_fingerprint();
#Modify enable_fingerprint
$griddhcp->enable_fingerprint("false");
```

enable_gss_tsig( )

Use this method to retrieve the grid-level enable_gss_tsig flag. This method returns "true" only if both the gss_tsig_key and kdc_server are specified.

This is for DHCPv4 GSS-TSIG authentication.

Parameter

This is a read-only method. You cannot enter any parameter.

Returns

This method returns the enable_gss_tsig attribute value.

Example

```
#Get enable_gss_tsig
my $enable_gss_tsig = $griddhcp->enable_gss_tsig();
```

enable_ifmap( )

Use this method to enable or disable the IF-MAP DHCP client feature on this grid.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

Boolean to enable or disable IF-MAP client feature. The default value is "false".

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get IF-MAP enable value
my $ifmap_enabled = $griddhcp->enable_ifmap();
#Enable IF-MAP Client feature
$griddhcp->enable_ifmap("true");
```

enable_ifmap_publishing( )

Use this method to enable or disable the IF-MAP publishing feature on this grid.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

Boolean to enable or disable IF-MAP publishing feature. The default value is "true".

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get IF-MAP publishing enable value
my $ifmap_publishing_enabled = $griddhcp->enable_ifmap_publishing();
#Disable IF-MAP publishing feature
$griddhcp->enable_ifmap_publishing("false");
```

enable_hostname_rewrite( )

Use this method to set or retrieve the grid enable_hostname_rewrite flag.

Parameter

Boolean to enable or disable hostname rewrite for the grid. The default value is "false".

Cannot enable the hostname rewrite policy along with the Microsoft code page. See microsoft_code_page().

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get enable_hostname_rewrite
$status = $griddhcp->enable_hostname_rewrite();
#Modify enable_hostname_rewrite
$griddhcp->enable_hostname_rewrite("true");
```

enable_roaming_hosts( )

Use this method to set or retrieve the grid-level enable_roaming_hosts flag. Set this flag to enable DHCP servers in a grid to support roaming hosts.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

Specify "true" to set the enable_roaming_hosts flag or "false" to deactivate/unset it. The default value is "false".

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get enable_roaming_hosts
my $enable_roaming_hosts = $griddhcp->enable_roamin_hosts();
#Modify enable_roaming_hosts
$griddhcp->enable_roaming_hosts("true");
```

enable_snmp_warnings( )

Use this method to set or retrieve the grid-level enable_snmp_warnings flag. When DHCP address usage crosses a watermark threshold and DHCP threshold is enabled, the appliance sends an SNMP trap to trap receivers that you defined for the grid.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

Specify "true" to set the enable_snmp_warnings flag or "false" to deactivate/unset it. The default value is "false".

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get enable_snmp_warnings
my $enable_snmp_warnings = $griddhcp->enable_snmp_warnings();
#Modify enable_snmp_warnings
$griddhcp->enable_snmp_warnings("true");
```

grid( )

Use this method to retrieve the grid that serves DHCP. This specifies a group of Infoblox appliances that are connected together to provide a single point of device administration and service configuration in a secure, highly available environment.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

Text with the name of the grid that provides DHCP service. Default value is "Infoblox".

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#get grid
my $grid = $griddhcp->grid();
```

gss_tsig_keys( )

Use this method to set or retrieve the list of GSS-TSIG keys for a Grid DHCPv4 object.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

The valid value is an array reference that contains Infoblox::Grid::KerberosKey objects.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get the Kerberos keys list
my $kerberos_keys_list = $grid_dhcp->gss_tsig_keys();

#Modify the Kerberos keys list
my $kerberos_key1 = Infoblox::Cluster::KerberosKey->new(
       principal => 'host/appliance.ivan.test1@LADYBUG.LOCAL',
       );
my $kerberos_key2 = Infoblox::Cluster::KerberosKey->new(
       principal => 'host/appliance.ivan.test2@LADYBUG.LOCAL',
       );
$grid_dhcp->gss_tsig_keys([$kerberos_key1, $kerberos_key2]);
```

high_water_mark( )

Use this method to set or retrieve the grid-level high_water_mark value. If the percent of allocated addresses exceeds this watermark, the appliance makes a syslog entry and if enabled, sends an e-mail notification.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

Number with the percentage value. Range is from 1 to 100. The high watermark value must be greater than the low watermark value. Default value is 95.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get high_water_mark
my $high_water_mark = $griddhcp->high_water_mark();
#Modify high_water_mark
$griddhcp->high_water_mark(80);
```

high_water_mark_reset( )

Use this method to set or retrieve the grid-level high_water_mark_reset value. If the percentage of allocated addresses drops below this value, a corresponding SNMP trap is reset.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

Number with the percentage value. Range is from 1 to 100. The high watermark value must be greater than the low watermark value. Default value is 85.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get high_water_mark_reset
my $high_water_mark_reset = $griddhcp->high_water_mark_reset();
#Modify high_water_mark_reset
$griddhcp->high_water_mark_reset(80);
```

hostname_rewrite_policy( )

Use this method to retrieve the name of the default hostname rewrite policy, which is also in the protocol_hostname_rewrite_policies array. This attribute is read-only.

Parameter

None.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get hostname_rewrite_policy
my $name = $griddhcp->hostname_rewrite_policy();
```

ifmap_delete_option( )

Use this method to set or retrieve the IF-MAP delete option. The possible values are 'never', 'always', 'older_than'.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

The IF-MAP delete option. The default value is 'always'.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get IF-MAP delete option
my $delete_option = $griddhcp->ifmap_delete_option();
#Modify IF-MAP delete option
$griddhcp->ifmap_delete_option('never');
```

ifmap_ip_mac_lifetime( )

Use this method to set or retrieve the IF-MAP ip-mac (ip-duid) metadata lifetime. This method applies only if the IF-MAP delete option is set to "older_than".

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

The IF-MAP ip mac lifetime.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get IF-MAP ip mac lifetime
my $port = $griddhcp->ifmap_ip_mac_lifetime();
#Modify IF-MAP ip mac lifetime
$griddhcp->ifmap_ip_mac_lifetime(300);
```

ifmap_port( )

Use this method to set or retrieve the IF-MAP server port used by DHCP IF-MAP clients. This setting is optional and defaults to the port associated with the protocol specified in ifmap_url (eg: for HTTP, the port is 80; for HTTPS, the port is 443).

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

The IF-MAP server port number. The default value is undef.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get IF-MAP server port
my $port = $griddhcp->ifmap_port();
#Modify IF-MAP server port
$griddhcp->ifmap_port(1234);
```

ifmap_protocol( )

Use this method to set or retrieve the IF-MAP server protocol. This setting is required and must be one of 'ifmap_20' or 'ifmap_11'.

Parameter

The IF-MAP server protocol. The default value is 'ifmap_20'.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get IF-MAP server protocol
my $protocol = $griddhcp->ifmap_protocol();
#Modify IF-MAP server protocol
$griddhcp->ifmap_protocol('ifmap_11');
```

ifmap_url( )

Use this method to set or retrieve the IF-MAP server URL used by DHCP IF-MAP clients. This setting is mandatory if IF-MAP is enabled on this grid.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

String for the IF-MAP server URL. The default value is undef.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get IF-MAP server URL
my $url = $griddhcp->ifmap_url();
#Modify IF-MAP server URL
$griddhcp->ifmap_url("https://ifmap.infoblox.com");
```

ifmap_server_ca_cert( )

Use this method to set or retrieve the ifmap server CA certificate.

Parameter

An Infoblox::IFMap::CACertificate object.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute server_ca_cert.

Example

```
# Get server_ca_cert
my $server_ca_cert = $griddhcp->server_ca_cert( );

# Change it
$griddhcp->server_ca_cert($ca_cert);
```

ignore_id( )

Use this method to set or retrieve the grid-level ignore_id type. Indicates whether the appliance will ignore DHCP client IDs or MAC addresses.

Parameter

Valid values are 'NONE', 'CLIENT', or 'MACADDR'. The default is 'NONE'.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
# Get attribute value
$value = $griddhcp->ignore_id();
```

```
# Modify attribute value
$griddhcp->ignore_id('CLIENT');
#Un-override ignore_id
$griddhcp->ignore_id(undef);
```

ignore_mac_addresses( )

Use this method to set or retrieve the grid-level ignore_mac_addresses list. A list of MAC addresses the appliance will ignore.

If you enter MAC addresses in the ignore_mac_addresses, then the appliance will ignore them. If you enter empty array, then the appliance will ignore any MAC address.

Parameter

The valid value is an array reference that contains string objects.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
# Get attribute value
$value = $griddhcp->ignore_mac_addresses();
# Modify attribute value
$griddhcp->ignore_mac_addresses(["11::11::11::11::11::11"]);
```

ignore_dhcp_option_list_request( )

Use this method to set or retrieve the grid-level ignore_dhcp_option_list_request flag. If this flag is set to true all available DHCP options will be returned to the client.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

Specify "true" to set the ignore_dhcp_option_list_request flag or "false" to deactivate/unset it. The default value is "false".

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get ignore_dhcp_option_list_request
my $ignore_dhcp_option_list_request = $griddhcp->ignore_dhcp_option_list_request();
#Modify ignore_dhcp_option_list_request
$griddhcp->ignore_dhcp_option_list_request("true");
```

immediate_fa_configuration( )

Use this method to control whether fixed address configuration takes effect immediately without DHCP service restart.

Setting this method to a defined value implicitly sets the override_immediate_fa_configuration method to "true". Setting the parameter to undefined causes the appliance to use the **Grid|MS Server|...** default and automatically resets the override_immediate_fa_configuration attribute to "false".

Note that when immediate_fa_configuration is set to a defined value and override_immediate_fa_configuration is set to "false", the last operation takes precedence. Thus the sequence $object->immediate_fa_configuration(**VAL**); $object->override_immediate_fa_configuration("false"); will set override_immediate_fa_configuration to "false", and the sequence $object->override_immediate_fa_configuration("false"); $object->immediate_fa_configuration(**VAL**); will result in override_immediate_fa_configuration="true".

Parameter

The valid value is an unsigned integer. Default value is "false".

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get immediate_fa_configuration
my $immediate_fa_configuration = $griddhcp->immediate_fa_configuration();
```

```
#Modify disable
$griddhcp->immediate_fa_configuration("true");
```

ipv6_capture_hostname( )

Use this method to set or retrieve the grid-level ipv6_capture_hostname flag. Set this flag to capture the ipv6 hostname and lease time when assigning a fixed address.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

Specify "true" to enable this feature or "false" to disable it. The default value is "false".

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get ipv6_capture_hostname
my $ipv6_capture_hostname = $griddhcp->ipv6_capture_hostname();
#Modify disable
$griddhcp->ipv6_capture_hostname("true");
```

ipv6_ddns_enable_option_fqdn( )

Use this method to set or retrieve the grid-level ipv6_ddns_enable_option_fqdn flag. This method controls whether the FQDN option sent by the client is to be used, or if the server can automatically generate the FQDN.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

Specify "true" to enable this feature or "false" to disable it. The default value is "false".

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get ipv6_ddns_enable_option_fqdn
my $ipv6_ddns_enable_option_fqdn = $griddhcp->ipv6_ddns_enable_option_fqdn();
#Modify disable
$griddhcp->ipv6_ddns_enable_option_fqdn("true");
```

ipv6_ddns_server_always_updates( )

Use this method to set or retrieve the grid-level ipv6_ddns_server_always_updates flag. Set this flag to "false" to update DNS only if requested by the client.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

Specify "true" to enable this feature or "false" to disable it. The default value is "false".

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get ipv6_ddns_server_always_updates
my $ipv6_ddns_server_always_updates = $griddhcp->ipv6_ddns_server_always_updates();
#Modify ipv6_ddns_server_always_updates
$griddhcp->ipv6_ddns_server_always_updates("true");
```

ipv6_ddns_domainname( )

Use this method to set or retrieve the grid-level IPv6 DDNS domain name value.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

The DDNS domain name in FQDN (Fully Qualified Domain Name) format.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get ipv6_ddns_domainname
my $ipv6_ddns_domainname = $griddhcp->ipv6_ddns_domainname();
#Modify ipv6_ddns_domainname
$griddhcp->ipv6_ddns_domainname("domain.com");
```

ipv6_ddns_ttl( )

Use this method to set or retrieve the grid-level IPv6 DDNS TTL.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

The preferred DDNS TTL value in seconds. The default is undefined.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get ipv6_ddns_ttl
my $ipv6_ddns_ttl = $griddhcp->ipv6_ddns_ttl();
#Modify ipv6_ddns_ttl
$griddhcp->ipv6_ddns_ttl(12345);
```

ipv6_default_prefix( )

Use this method to set or retrieve the grid-level IPv6 default prefix.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

The desired IPv6 prefix. It must be one of those listed in the ipv6_prefixes method.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get ipv6_default_prefix
my $ipv6_default_prefix = $griddhcp->ipv6_default_prefix();
#Modify ipv6_default_prefix
$griddhcp->ipv6_default_prefix("2001::");
```

ipv6_dns_update_style( )

Use this method to set or retrieve the update style for dynamic DNS updates (DHCPv6).

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

Valid values are 'INTERIM' and 'STANDARD'.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get ipv6_dns_update_style value
```

```
my $value = $object->ipv6_dns_update_style();

#Modify ipv6_dns_update_style value
$object->ipv6_dns_update_style('STANDARD');
```

ipv6_domain_name( )

Use this method to set or retrieve the ipv6_domain_name value for a Grid DHCP object.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

The name of the client's domain for resolving names in DNS in string format.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get ipv6_domain_name
my $ipv6_domain_name = $griddhcp->ipv6_domain_name();
#Modify ipv6_domain_name
$griddhcp->ipv6_domain_name("test_domain.com");
```

ipv6_domain_name_servers( )

Use this method to set or retrieve the ipv6_domain_name_servers value of a Grid DHCP object.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

The comma-separated domain name server addresses in IPv6 address format.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get ipv6_domain_name_servers
my $ipv6_domain_name_servers = $griddhcp->ipv6_domain_name_servers();
#Modify ipv6_domain_name_servers
$griddhcp->ipv6_domain_name_servers("2001::1,2001::2");
```

ipv6_enable_ddns( )

Use this method to set or retrieve the grid-level ipv6_enable_ddns flag. Set this flag to enable DHCPv6 dynamic DNS updates.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

Specify "true" to enable this feature or "false" to disable it. The default value is "false".

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get ipv6_enable_ddns
my $ipv6_enable_ddns = $griddhcp->ipv6_enable_ddns();
#Modify ipv6_enable_ddns
$griddhcp->ipv6_enable_ddns("true");
```

ipv6_enable_gss_tsig( )

Use this method to retrieve the grid-level ipv6_enable_gss_tsig flag. This method returns "true" only if both the ipv6_gss_tsig_key and ipv6_kdc_server are specified.

This is for DHCPv6 GSS-TSIG authentication.

Parameter

This is a read-only method. You cannot enter any parameter.

Returns

This method returns the ipv6_enable_gss_tsig attribute value.

Example

```
#Get ipv6_enable_gss_tsig
my $ipv6_enable_gss_tsig = $griddhcp->ipv6_enable_gss_tsig();
```

ipv6_enable_lease_scavenge( )

Use this method to set or retrieve the flag that indicates whether the DHCPv6 lease scavenging is enabled or disabled.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

Specify 'true' to enable DHCPv6 lease scavenging and 'false' to disable it. The default value is false.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get ipv6_enable_lease_scavenge
my $ipv6_enable_lease_scavenge = $griddhcp->ipv6_enable_lease_scavenge();
#Modify ipv6_enable_lease_scavenge
$griddhcp->ipv6_enable_lease_scavenge("true");
```

ipv6_remember_expired_client_association( )

Use this method to set or retrieve the flag that indicates whether the DHCPv6 server can 'remember' the client for a configurable amount of time and lease the same IPv6 address to the client.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

Specify 'true' to enable the DHCPv6 server to 'remember' the client and 'false' to disable it. The default value is false.

Returns

If you specified a parameter, the method returns true when the modification succeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get ipv6_remember_expired_client_association
my $ipv6_remember_expired_client_association = $griddhcp->ipv6_remember_expired_client_association();
#Modify ipv6_remember_expired_client_association
$griddhcp->ipv6_remember_expired_client_association("true");
```

ipv6_enable_retry_updates( )

Use this method to set or retrieve the ipv6_enable_retry_updates flag. If this flag is set to "true", the DHCPv6 server retries failed dynamic DNS updates.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

Specify "true" to enable the DHCP server to retry failed DNS updates or "false" to disable it. The default value is "true".

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get ipv6_enable_retry_updates
my $ipv6_enable_retry_updates = $griddhcp->ipv6_enable_retry_updates();
#Modify ipv6_enable_retry_updates
```

```
$griddhcp->ipv6_enable_retry_updates("true");
```

ipv6_generate_hostname( )

Use this method to set or retrieve the grid-level ipv6_generate_hostname flag. Set this flag to generate the hostname if it is not sent by the client.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

Specify "true" to enable this feature or "false" to disable it. The default value is "false".

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get ipv6_generate_hostname
my $ipv6_generate_hostname = $griddhcp->ipv6_generate_hostname();
#Modify ipv6_generate_hostname
$griddhcp->ipv6_generate_hostname("true");
```

ipv6_gss_tsig_keys( )

Use this method to set or retrieve the list of GSS-TSIG keys for a Grid DHCPv6 object.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

The valid value is an array reference that contains Infoblox::Grid::KerberosKey objects.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get the Kerberos keys list
my $kerberos_keys_list = $grid_dhcp->ipv6_gss_tsig_keys();

#Modify the Kerberos keys list
my $kerberos_key1 = Infoblox::Cluster::KerberosKey->new(
       principal => 'host/appliance.ivan.test1@LADYBUG.LOCAL',
       );
my $kerberos_key2 = Infoblox::Cluster::KerberosKey->new(
       principal => 'host/appliance.ivan.test2@LADYBUG.LOCAL',
       );
$grid_dhcp->ipv6_gss_tsig_keys([$kerberos_key1, $kerberos_key2]);
```

ipv6_kdc_server( )

Use this method to specify or retrieve the IPv4 address or FQDN of the Kerberos server for DHCPv6 GSS-TSIG authentication.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

The Kerberos server IPv4 address or FQDN, with a maximum of 1023 bytes.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get ipv6_kdc_server
my $ipv6_kdc_server = $griddhcp->ipv6_kdc_server();
#Modify ipv6_kdc_server
$griddhcp->ipv6_kdc_server("10.32.0.8");
```

ipv6_lease_scavenge_time( )

Use this method to set or retrieve the Grid-level grace period in seconds to keep an expired lease before it gets deleted by the scavenging process.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

The valid value is an unsigned integer greater than or equal to 21600.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get ipv6_lease_scavenge_time
my $ipv6_lease_scavenge_time = $griddhcp->ipv6_lease_scavenge_time();
#Modify ipv6_lease_scavenge_time
$griddhcp->ipv6_lease_scavenge_time(22000);
```

ipv6_microsoft_code_page( )

Use this method to set or retrieve the grid-level Microsoft client DHCP IPv6 code page value. This value is the hostname translation code page for Microsoft DHCP IPv6 clients.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

Microsoft DHCP client code page in string format. Valid values are:

None

Arabic (1256)

Arabic (ISO-8859-6)

Baltic (1257)

Baltic (775)

Baltic (ISO-8859-4)

Central Europe (1250)

Cyrillic (1251)

Cyrillic (855)

Cyrillic (ISO-8859-5)

Greek (1253)

Greek (737)

Greek (ISO-8859-7)

Hebrew (1255)

Hebrew (862)

Hebrew (ISO-8859-8)

Japanese Shift-JIS (932)

Korean (949)

Latin 1 (ISO-8859-1)

Latin 2 (ISO-8859-2)

Latin 3 (ISO-8859-3)

Latin 9 (ISO-8859-15)

Latin I (1252)

Latin II (852)

Multilingual Latin I (850)

Russian (866)

Simplified Chinese GBK (936)

Thai (874)

Traditional Chinese Big5 (950)

Turkish (1254)

Turkish (857)

Turkish (ISO-8859-9)

US (437)

Vietnam (1258)

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get ipv6_microsoft_code_page
my $ipv6_microsoft_code_page = $griddhcp->ipv6_microsoft_code_page();
#Modify ipv6_microsoft_code_page
$griddhcp->ipv6_microsoft_code_page("Cyrillic (ISO-8859-5)");
```

ipv6_options( )

Use this method to set or retrieve the grid-level IPv6 options.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

The valid value is an array reference that contains Infoblox::DHCP::Option object(s).

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get ipv6_options
my $ipv6_options = $griddhcp->ipv6_options();
#Modify ipv6_options
my $option1 = Infoblox::DHCP::Option->new(
    name   => "dhcp6.rapid-commit",
    value  => "true"
);
$griddhcp->ipv6_options([$option1]);
```

ipv6_prefixes( )

Use this method to set or retrieve the grid-level list of IPv6 prefixes.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

A comma-separated list of IPv6 prefixes.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get ipv6_prefixes
my $ipv6_prefixes = $griddhcp->ipv6_prefixes();
#Modify ipv6_prefixes
$griddhcp->ipv6_prefixes("2001::,2002::");
```

ipv6_recycle_leases( )

Use this method to set or retrieve the ipv6_recycle_leases flag of a grid DHCP server. If you enable this feature, and then delete an IPv6 DHCP range, the appliance stores active leases from this range up to one week after the leases expire.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

Specify "true" to set the ipv6_recycle_leases flag or "false" to deactivate/unset it. The default value is "true".

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get ipv6_recycle_leases
my $ipv6_recycle_leases = $griddhcp->ipv6_recycle_leases();
#Modify ipv6_recycle_leases
$griddhcp->ipv6_recycle_leases("true");
```

ipv6_retry_updates_interval( )

Use this method to set or retrieve the ipv6_retry_updates_interval value. Set the retry interval when the member DHCPv6 server makes repeated attempts to send DDNS updates to a DNS server.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

A 32-bit integer (range from 0 to 4294967295) that represents the retry interval in minutes. The default retry interval is five minutes.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get ipv6_retry_updates_interval
my $ipv6_retry_updates_interval = $griddhcp->ipv6_retry_updates_interval();
#Modify ipv6_retry_updates_interval
$griddhcp->ipv6_retry_updates_interval(10);
```

ipv6_txt_record_handling( )

Use this method to set or retrieve the grid-level TXT record handling value. This value specifies how DHCP should treat TXT records when performing DNS updates.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

TXT record handling value in string format. Valid values are: "IGNORE_CONTENTS", "ISC", "ISC_TRANSITIONAL" and "MS". The default value is "ISC".

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get ipv6_txt_record_handling
my $ipv6_txt_record_handling = $griddhcp->ipv6_txt_record_handling();
#Modify ipv6_txt_record_handling
$griddhcp->ipv6_txt_record_handling("ISC_TRANSITIONAL");
```

ipv6_update_dns_on_lease_renewal( )

The ipv6_update_dns_on_lease_renewal attribute controls whether the DHCP server updates DNS when an IPv6 DHCP lease is renewed.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

Specify "true" to set the ipv6_update_dns_on_lease_renewal flag or "false" to deactivate/unset it.

The default value of this parameter is "false".

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get ipv6_update_dns_on_lease_renewal
my $ipv6_update_dns_on_lease_renewal=$griddhcp->ipv6_update_dns_on_lease_renewal( );
#Modify ipv6_update_dns_on_lease_renewal
$griddhcp->ipv6_update_dns_on_lease_renewal("true");
```

kdc_server( )

Use this method to specify or retrieve the IPv4 address or FQDN of the Kerberos server for DHCPv4 GSS-TSIG authentication.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

The Kerberos server IPv4 address or FQDN, with a maximum of 1023 bytes.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get kdc_server
my $kdc_server = $griddhcp->kdc_server();
#Modify  kdc_server
$griddhcp->kdc_server("10.32.0.7");
```

lease_logging_member( )

Use this method to specify or retrieve the grid member on which you want to store the DHCP lease history log. Infoblox recommends that you dedicate a member other than the master as a logging member. If possible, use this member solely for storing the DHCP lease history log. If you do not select a member, no logging can occur.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

The valid value is an array reference that contains Infoblox::DHCP::Member object(s).

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get lease_logging_member
my $lease_logging_member = $griddhcp->lease_logging_member();
#Modify lease_logging_member
my $dhcpmember = Infoblox::DHCP::Member->new(
    name     => "infoblox.localdomain",
    ipv4addr => "192.168.1.2"
);

$griddhcp->lease_logging_member($dhcpmember);
```

lease_per_client_settings( )

Use this method to set or retrieve the setting that defines how DHCP server will free or release client leases.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

The valid values are 'ONE_LEASE_PER_CLIENT', 'RELEASE_MATCHING_ID' and 'NEVER_RELEASE'.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get lease_per_client_settings value
my $value = $object->lease_per_client_settings();

#Modify lease_per_client_settings value
$object->lease_per_client_settings('NEVER_RELEASE');
```

lease_scavenge_time( )

Use this method to set or retrieve the grid-level lease_scavenge_time value.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

An integer that specifies the period of time, in seconds, that free and backup leases remain in the database before they are automatically deleted. To disable lease scavenging, set the parameter to -1. The minimum positive value must be greater than 86400 seconds (1 day). The default value of this parameter is -1, which means that lease scavenging is disabled.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get lease_scavenge_time
my $lease_scavenge = $griddhcp->lease_scavenge_time();
#Modify lease_scavenge_time
$griddhcp->lease_scavenge_time(30 * 24 * 60 * 60);
```

log_lease_events( )

Use this method to set or retrieve the grid-level log_lease_events flag. This value specifies whether grid members log lease events.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

Specify "true" to set the log_lease_events flag or "false" to deactivate/unset it. The default value is "false".

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get log_lease_events
my $log_lease_events = $griddhcp->log_lease_events();
#Modify log_lease_events
$griddhcp->log_lease_events("true");
```

logic_filters( )

Use this method to set or retrieve all the Grid-level logic (ordered) filters.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

The valid value is an array reference that contains either the Infoblox::DHCP::Filter::MAC, Infoblox::DHCP::Filter::NAC, Infoblox::DHCP::Filter::Option objects or the names of the corresponding filters. The filters in the list must satisfy the following conditions:

Filter name must be globally unique.
Filters without expressions must be at the end of the list.
The filters should not be disabled.

The default value of the parameter is undefined.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get logic_filters
my $ref_logic_filters = $griddhcp->logic_filters();
#Modify logic_filters
$griddhcp->logic_filters(["filter_1","filter_2",$exl1]); #$exl1 is an Infoblox::DHCP::Filter::MAC,
Infoblox::DHCP::Filter::NAC or Infoblox::DHCP::Filter::Option object.
```

low_water_mark( )

Use this method to set or retrieve the grid-level low_water_mark value. If the percent of allocated addresses drops below this watermark, the appliance makes a syslog entry and if enabled, sends an e-mail notification.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

Number with the percentage value. Range is from 0 to 100. The low watermark value must be lower than the high watermark value. The default low watermark number is 0.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get low_water_mark
my $low_water_mark = $griddhcp->low_water_mark();
#Modify low_water_mark
$griddhcp->low_water_mark(10);
```

low_water_mark_reset( )

Use this method to set or retrieve the grid-level low_water_mark_reset value. If the percentage of allocated addresses exceeds this value, a corresponding SNMP trap is reset.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

Number with the percentage value. Range is from 0 to 100. The low watermark reset value must be higher than the low watermark value. The default low watermark reset value is 0.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get low_water_mark_reset
my $low_water_mark_reset = $griddhcp->low_water_mark_reset();
#Modify low_water_mark_reset
$griddhcp->low_water_mark_reset(10);
```

microsoft_code_page( )

Use this method to set or retrieve the grid-level Microsoft client DHCP IPv4 code page value. This value is the hostname translation code page for Microsoft DHCP IPv4 clients.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

Microsoft DHCP client code page in string format. Valid values are the same as those for ipv6_microsoft_code_page. See ipv6_microsoft_code_page().

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get microsoft_code_page
my $microsoft_code_page = $griddhcp->microsoft_code_page();
#Modify microsoft_code_page
```

```
$griddhcp->microsoft_code_page("Cyrillic (ISO-8859-5)");
```

nextserver( )

Use this method to set or retrieve the grid-level "next server" value. This value is the IP address or name of the boot file server on which the boot file is stored.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

The boot file server IP address or name in FQDN format. The FQDN consists of the host name followed by the domain name (example: abc.com). A boot file server name can have a maximum of 256 bytes.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get nextserver
my $nextserver = $griddhcp->nextserver();
#Modify nextserver
$griddhcp->nextserver("1.1.1.1");
```

option60_match_rules( )

Use this method to set or retrieve grid-level DHCP option60 match rules. An Option60MatchRule object allows specifying match rules for Option 60 globally.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

The valid value is an array reference that contains Infoblox::DHCP::Option60MatchRule object(s).

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get option60_match_rules
my $option_matchrule = $griddhcp->option60_match_rules();
#Modify option60_match_rules
my $option60_match_rule = Infoblox::DHCP::Option60MatchRule->new(
                                    "option_space" => "infospace",
                                    "match_value" => "Nortel-i2004-A",
                                    "is_substring" => "true",
                                    "substring_offset" => "8",
                                    "substring_length" => "14"
                                  );

# Configure DHCP Option60MatchRule on the Grid DHCP object
$griddhcp->option60_match_rules([$option60_match_rule]);

#Remove Option60MatchRule object
$griddhcp->option60_match_rules([]);
```

options( )

Use this method to set or retrieve grid-level DHCP options. These options provide network configuration settings and various services available on the network.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

The valid value is an array reference that contains Infoblox::DHCP::Option object(s).

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get options
my $options = $griddhcp->options();
#Modify options
my $option1 = Infoblox::DHCP::Option->new(
    name   => "lease-time",
    value  => 50
);

my $option2 = Infoblox::DHCP::Option->new(
    name    => "routers",
    value   => "1.4.1.1,2.2.2.2",
);

my $option3 = Infoblox::DHCP::Option->new(
    name     => "nameservers",
    value    => "1.1.1.1, 2.2.2.2",
);

my $option4 = Infoblox::DHCP::Option->new(
    name  => "domain-name",
    value => "domainname.com",
);

my $option5 = Infoblox::DHCP::Option->new(
    name     => "broadcast",
    value    => "255.0.0.0",
);

$griddhcp->options([$option1, $option2, $option3, $option4, $option5]);
```

override_immediate_fa_configuration( )

The override_immediate_fa_configuration attribute controls whether the immediate_fa_configuration value in the object is used, instead of the Grid default.

The override_immediate_fa_configuration attribute can be specified explicitly. It is also set implicitly when immediate_fa_configuration is set to a defined value.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

Specify "true" to set override_ttl flag or "false" to disable it. Default value is "false".

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get override_immediate_fa_configuration
my $override_immediate_fa_configuration = $griddhcp->override_immediate_fa_configuration();
#Modify override_immediate_fa_configuration
$griddhcp->override_immediate_fa_configuration("true");
```

ping_count( )

Use this method to set or retrieve the grid-level ping_count value. This value specifies the number of pings that the Infoblox appliance sends to an IP address to verify that it is not in use.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

Number of pings. Range is from 0 to 10, where 0 disables pings. The default value is 1.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get ping_count
my $ping_count = $griddhcp->ping_count();
#Modify ping_count
$griddhcp->ping_count(4);
```

ping_timeout( )

Use this method to set or retrieve the grid-level ping_timeout value. This value indicates the number of seconds the appliance waits for a response to its ping.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

Ping timeout value in seconds. Valid values are 0.1, 0.5, 1, 2, 3, 4 and 5 seconds. The default value is 1.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get ping_timeout
my $ping_timeout = $griddhcp->ping_timeout();
#Modify ping_timeout
$griddhcp->ping_timeout(3);
```

preferred_lifetime( )

Use this method to set or retrieve the grid-level preferred lifetime.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

Preferred lifetime value in seconds. The default value is 27000.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get preferred_lifetime
my $preferred_lifetime = $griddhcp->preferred_lifetime();
#Modify preferred_lifetime
$griddhcp->preferred_lifetime(12345);
```

protocol_hostname_rewrite_policies( )

Use this method to set or retrieve a list of hostname rewrite policies. This attribute is mandatory if enable_hostname_rewrite is "true".

Parameter

Array reference with Infoblox::Grid::HostNameRewritePolicy objects.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get protocol_hostname_rewrite_policies
my $policies = $griddhcp->protocol_hostname_rewrite_policies();

#Modify protocol_hostname_rewrite_policies
push(@$policies, $policy2);
$griddhcp->protocol_hostname_rewrite_policies($policies);
```

pxe_lease_time( )

Use this method to set or retrieve the grid-level pxe_lease_time value. This value is the duration of time it takes a host to connect to a boot server, such as a TFTP server, and download the file it needs to boot.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

A 32-bit integer (range from 0 to 4294967295) that specifies the lease time in seconds. Default value is 43200 seconds.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get pxe_lease_time
my $pxe_lease_time = $griddhcp->pxe_lease_time();
#Modify pxe_lease_time
$griddhcp->pxe_lease_time(200);
```

recycle_leases( )

Use this method to set or retrieve the recycle_leases flag of a grid DHCP server. If you enable this feature, and then delete a DHCP range, the appliance stores active leases from this range up to one week after the leases expire.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

Specify "true" to set the recycle_leases flag or "false" to deactivate/unset it. The default value is "true".

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get recycle_leases
my $recycle_leases = $griddhcp->recycle_leases();
#Modify recycle_leases
$griddhcp->recycle_leases("true");
```

restart_setting( )

Use this method to set or retrive Grid DHCP service restart settings.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

The valid value is an Infoblox::Grid::ServiceRestart object.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get restart_setting
my $restart_setting = $grid_dhcp->restart_setting();
#Modify the retry time
$grid_dhcp->restart_setting($restart_setting);
```

retry_ddns_updates( )

Use this method to set or retrieve the grid-level retry_ddns_updates flag. This value indicates whether the grid DHCP servers make repeated attempts to send DDNS updates to a DNS server.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

Specify "true" to enable the DHCP servers to retry DDNS updates or "false" to disable this feature. The default value is "true".

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get retry_ddns_updates
my $retry_ddns_updates = $griddhcp->retry_ddns_updates();
#Modify retry_ddns_updates
```

```
$griddhcp->retry_ddns_updates("true");
```

syslog_facility( )

Use this method to set or retrieve the grid-level syslog_facility value. The syslog facility is the location on the syslog server to which you want to sort syslog messages.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

Valid values are: "daemon", "local0", "local1", "local2", "local3", "local4", "local5", "local6", "local7". The default value is "daemon".

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get syslog_facility
my $syslog_facility = $griddhcp->syslog_facility();
#Modify syslog_facility
$griddhcp->syslog_facility("local0");
```

txt_record_handling( )

Use this method to set or retrieve the grid-level TXT record handling value. This value specifies how DHCP should treat TXT records when performing DNS updates.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

TXT record handling value in string format. Valid values are: "IGNORE_CONTENTS", "ISC", "ISC_TRANSITIONAL" and "MS". The default value is "ISC".

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get txt_record_handling
my $txt_record_handling = $griddhcp->txt_record_handling();
#Modify txt_record_handling
$griddhcp->txt_record_handling("ISC_TRANSITIONAL");
```

update_dns_on_lease_renewal( )

The update_dns_on_lease_renewal attribute controls whether the DHCP server updates DNS when a DHCP lease is renewed.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

Specify "true" to set the update_dns_on_lease_renewal flag or "false" to deactivate/unset it.

The default value of this parameter is "false".

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get update_dns_on_lease_renewal
my $update_dns_on_lease_renewal=$griddhcp->update_dns_on_lease_renewal( );
#Modify update_dns_on_lease_renewal
$griddhcp->update_dns_on_lease_renewal("true");
```

valid_lifetime( )

Use this method to set or retrieve the valid lifetime value on the Grid level.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

Valid lifetime value in seconds. The default value is 43200.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get valid_lifetime
my $valid_lifetime = $roaming_host->valid_lifetime();
#Modify valid_lifetime
$roaming_host->valid_lifetime(12345);
```

validate_ifmap_ca_cert( )

Use this method to set or retrieve the validate CA certificate value.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

Specify "true" to set the validate_ifmap_ca_cert flag or "false" to deactivate/unset it.

The default value of this parameter is "true".

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get validate_ifmap_ca_cert
my $validate_ifmap_ca_cert = $griddhcp->validate_ifmap_ca_cert();
#Modify validate_ifmap_ca_cert
$griddhcp->validate_ifmap_ca_cert('false');
```

SAMPLE CODE

The following sample code demonstrates different operations that can be applied to a grid DHCP object, such as get and modify an object. This sample code also includes error handling for the operations.

#Preparation prior to get and modify operations on a grid DHCP object

```
#PROGRAM STARTS: Include all the modules that will be used
use strict;
use Infoblox;

#refers to Infoblox Appliance IP address
my $host_ip   = "192.168.1.2";
my $host_name = "infoblox.localdomain";

#Creating a session to appliance.
my $session = Infoblox::Session->new(
    master   => $host_ip,
    username => "admin",
    password => "infoblox"
    )
    or die("Constructor for session failed: ",
               Infoblox::status_code(). ":" .Infoblox::status_detail());

print"Session created successfully.\n";

#Creating a domain-name option object.
my $option = Infoblox::DHCP::Option->new(
                 name   => "domain-name",
                 value  => "domainname.com",
)
or die("Constructor for option failed: ",
               Infoblox::status_code(). ":" .Infoblox::status_detail());

print"dhcp option object created successfully.\n";

#Creating a Member object.
```

```
my $dhcpmember = Infoblox::DHCP::Member->new(
    name     => $host_name,
    ipv4addr => $host_ip
    )
    or die("Constructor for dhcpmember failed: ",
               Infoblox::status_code(). ":" .Infoblox::status_detail());

print"dhcp member object created successfully.\n";

#Creating a forward zone nameserver.
my $ns1 = Infoblox::DNS::Nameserver->new(
    ddns_zone => "sub.test.com",
    name      => "server.dns.com",
    ipv4addr  => "1.1.1.1",
    TSIGname  => "tsig_name",
    TSIGkey   => "ZX1jVJl7C58GT/sc7Q3ucA=="
    )
    or die("Constructor for ns1 failed: ",
               Infoblox::status_code(). ":" .Infoblox::status_detail());

print"forward zone nameserver object created successfully.\n";

#Creating a reverse zone nameserver.
my $ns2 = Infoblox::DNS::Nameserver->new(
    ddns_zone => "10.0.0.0/8",
    name      => "server.dns.com",
    ipv4addr  => "1.1.1.1"
    )
    or die("Constructor for ns2 failed: ",
               Infoblox::status_code(). ":" .Infoblox::status_detail());

print"reverse zone nameserver object created successfully.\n";
```

#Get and modify the grid DHCP object.

```
#Getting the DHCP object from appliance through session.
my @result = $session->get(
    object => "Infoblox::Grid::DHCP",
    grid   => "Infoblox"
    )
    or die("get DHCP failed: ",
               $session->status_code(). ":" .$session->status_detail());
print "get grid DHCP Objcet successful.\n";

unless (scalar(@result) == 0) {
    my $griddhcp = $result[0];
    if ($griddhcp) {

        #Modifying the value of the specified objects.
        $griddhcp->authority("false");
        $griddhcp->enable_ddns("true");
        $griddhcp->bootserver("boot.test.com");
        $griddhcp->ddns_ttl(21600);
        $griddhcp->nextserver("77.77.77.7");
        $griddhcp->pxe_lease_time(43200);
        $griddhcp->bootfile("bootfile");
        $griddhcp->ping_count(3);
        $griddhcp->ping_timeout(2);
        $griddhcp->email_list(["admin\@infoblox.com", "bkp_admin\@infoblox.com"]);
        $griddhcp->options([$option]);
        $griddhcp->lease_logging_member($dhcpmember);
        $griddhcp->microsoft_code_page("Cyrillic (ISO-8859-5)");

        #Applying the changes to appliance through session.
        $session->modify($griddhcp)
            or die("Modify DHCP failed: ",
               $session->status_code(). ":" .$session->status_detail());
                print"Modified Grid DHCP Object successfully.\n";
            }
         } else {
             print "No Grid DHCP object found.";
         }

####PROGRAM ENDS####
```

AUTHOR

Infoblox Inc. http://www.infoblox.com/

Infoblox::Grid::DHCP - Grid DHCP services object.

SEE ALSO

Infoblox::DHCP::Member,Infoblox::DHCP::Option,Infoblox::DNS::Nameserver,Infoblox::Grid::KerberosKey,Infoblox::Session,Infoblox::Session->get(),Infoblox::Session->modify()

COPYRIGHT

Copyright (c) 2017 Infoblox Inc.

Infoblox::Grid::Discovery - Grid discovery functions.

- NAME
- DESCRIPTION
- CONSTRUCTOR
- SESSION METHODS
- METHODS
  - clear_network_port_assignment( )
  - control_ip_address( )
  - control_switch_port( )
  - diagnostics( )
  - diagnostics_status( )
  - discovery_data_conversion( )
  - get_device_support_info( )
  - get_job_devices( )
  - get_job_process_details( )
  - provision_network_dhcp_relay( )
  - provision_network_port( )
  - start_discovery( )
- SEE ALSO
- AUTHOR
- COPYRIGHT

NAME

Infoblox::Grid::Discovery - Grid discovery functions.

DESCRIPTION

The object provides access to the Grid discovery and Network Insight functions.

CONSTRUCTOR

```
my $discovery = Infoblox::Grid::Discovery->new(
    'session' => $session,      # Required
);
```

SESSION METHODS

The object does not support any session methods.

METHODS

clear_network_port_assignment( )

Use this method to clear network provision information on given router or switch-router device interfaces for given networks.

Note that this function is always executed as part of scheduled task. Also the function goes through approval process if it is applicable.

Parameter

network_deprovision_info - Required. An array reference that contains Infoblox::Grid::Discovery::NetworkDeprovisionInfo objects.
scheduled_at - Required. Scheduled time in ISO 8601 extended format for date and time (e.g. 2008-10-21T10:56:00Z or 2008-10-21T10:56:00.001Z) or 'now' which will set scheduled date and time to the current date and time.

Returns

The method returns true when a scheduled task has been created successfully, and returns false when the operation fails.

Example

```
 my $res = $discovery->clear_network_port_assignment('scheduled_at' => $scheduled_time,
'network_deprovision_info' => [$deprovision_info1, $deprovision_info2]);
```

control_ip_address( )

Use this method to control multiple IP addresses.

Parameter

addresses - Required. An array reference that contains IP addresses. network_view - Optional. The network view name. exclude - Optional. The flag for exclude. Valid value is 'true' or 'false'.

Returns

The method returns true when the modification succeeds, and returns false when the operation fails.

Example

```
my $res = $discovery->control_ip_address('addresses' => ['10.0.0.1', '10.0.0.2']);
```

control_switch_port( )

Use this method to configure the interface of a given switch device.

Note that this function is always executed as part of scheduled task. Also the function goes through approval process if it is applicable. This function is used for both IPAM IP address object port configuration and device interface port control.

Parameter

port_config - Required. An array reference that contains Infoblox::Grid::Discovery::Port::Control objects. scheduled_at - Required. Scheduled time in ISO 8601 extended format for date and time (e.g. 2008-10-21T10:56:00Z or 2008-10-21T10:56:00.001Z) or 'now' which will set scheduled date and time to the current date and time.

Returns

The method returns "true" when a scheduled task has been created successfully, and returns "false" when the operation fails.

Example

```
my $res = $discovery->control_switch_port('scheduled_at' => $scheduled_time, 'port_configs' =>
[$config1, $config2]);
```

diagnostics( )

Use this method to execute discovery diagnostics for the IP address.

Parameter

address - Required. The IP address. community_string - Optional. The communitry string. force_test - Optional. The flag for force test. Valid value is 'true' or 'false'. discovery_member - Optional. The discovery member name. network_view - Optional. The network view name. debug_snmp - Optional. The flag for enable/disable SNMP debug (enabled by default). Valid value is 'true' or 'false'.

Returns

The method returns a discovery diagnostic status ID in string format when the operations succeeds, and returns false when teh operation fails.

Example

```
my $id = $discovery->diagnostics('address' => '10.0.0.1');
```

diagnostics_status( )

Use this method to retrieve the discovery diagnostic status.

Parameter

id - Required. The discovery diagnostic status ID. start - Optional. The start position of the text.

Returns

The method returns an Infoblox::Grid::Discovery::DiagnosticsStatus object when the operation succeeds, and returns false when the operation fails.

Example

```
my $object = $discovery->diagnostics_status('id' => $id);
```

discovery_data_conversion( )

Use this method to convert discovery data to specified objects.

Parameter

type - Required. The object type of the converted object. Valid value is one of the following object types: A_RECORD,

AAAA_RECORD, PTR_RECORD, HOST_RECORD, FIXED_ADDRESS, IPV6_FIXED_ADDRESS. addresses - Required. An array reference that contains Infoblox::IPAM::Address objects retrieved from the appliance. attributes - Optional. An Infoblox::Grid::Discovery::ConversionAttributes object. extensible_attributes - Optional. The list of extensible attributes associated with the created objects. Valid value is a hash reference containing the names of extensible attributes and their associated values (Infoblox::Grid::Extattr objects).

Returns

The method returns an array reference that contains Infoblox::Grid::Discovery::DiscoveryDataConversionResult objects if the operation succeeds.

Example

```
my $address = $discovery->get('object' => 'Infoblox::IPAM::Address', 'ip_address' => '10.0.0.31');
my $zone = $discovery->get('object' => 'Infoblox::DNS::Zone', 'name' => 'papi.com');
my $attrs = Infoblox::Grid::Discovery::ConversionAttributes->new(
    'configure_for_dns' => 'true',
    'configure_for_dhcp' => 'true',
    'comment' => 'convert',
    'disabled' => 'false',
    'zone' => $zone);
my $results = $discovery->discovery_data_conversion(
    'type' => 'HOST_RECORD',
    'attributes' => $attrs,
    'addresses' => [$address],
    'extensible_attributes' => {'Site' => Infoblox::Grid::Extattr->new('value' => 'test.com')});
```

get_device_support_info( )

Use this method to retrieve such information about device as device support statuses and device support info.

Parameter

device - Required. An Infoblox::Grid::Discovery::Device object.

Returns

The method returns an Infoblox::Grid::Discovery::DeviceSupportInfoResponse object.

Example

```
my $res = $discovery->get_device_support_info('device' => $device);
```

get_job_devices( )

Use this method to get the list of devices on wich the network insight job is processing.

This function should be called only if the task is related to network insight port control job.

Parameter

task - Required. An Infoblox::Grid::ScheduledTask object or a task ID.

Returns

The method returns an array reference that contains Infoblox::Grid::Discovery::Device object if operation succeeds, and returns "false" when the operation fails.

Example

```
# Get value by task_id
my $devices = $discovery->get_job_devices('task' => $task_id);

# Get value by scheduled_task object
my $devices = $discovery->get_job_devices('task' => $scheduled_task);
```

get_job_process_details( )

Use this method to get a network insight job processing logs related to a given task.

This function should be called only if the task is related to network insight port control job.

Parameter

device - Required. An Infoblox::Grid::Discovery::Device object. task - Required. An Infoblox::Grid::ScheduledTask object or a task ID. start_line - Optional. Valid value is unsigned integer representing starting line of a log stream. Default value is 0.

Returns

The method returns a Infoblox::Grid::Discovery::JobProcessDetails object if operation succeeds, and returns "false" if operation fails.

Example

```
 # Get job prcess details
 my $job_process_details = $discovery->get_job_process_details('device' => $device, 'task' =>
$scheduled_task, 'start_line' => 10);
```

provision_network_dhcp_relay( )

Use this method to provision only DHCP relay information for given devices.

Note that this function is always executed as part of scheduled task. Also the function goes through approval process if it is applicable.

Parameter

interfaces - Required. An array reference that contains Infoblox::Grid::Discovery::DeviceInterface objects. network - Required. The IP address and netmask of a network in CIDR format. scheduled_at - Required. Scheduled time in ISO 8601 extended format for date and time (e.g. 2008-10-21T10:56:00Z or 2008-10-21T10:56:00.001Z) or 'now' which will set scheduled date and time to the current date and time. enable_dhcp_relay - Optional. Enables DHCP relay. Valid value is 'true' and 'false'. Default value is 'false'. network_view - Optional. The network view name.

Returns

The method returns 'true' when a scheduled task has been created successfully, and returns 'false' when the operation fails.

Example

```
 my $res = $discovery->provision_network_dhcp_relay('scheduled_at' => 'now', 'interfaces' =>
\@interfaces, 'network' => $network);
```

provision_network_port( )

Use this method to provision the network with interface and assign default router ip address on a device.

Note that this function is always executed as part of scheduled task. Also the function goes through approval process if it is applicable.

Parameter

device - Optional. An Infoblox::Grid::Discovery::Device object. At least one of device or interface is required. interface - Optional. An Infoblox::Grid::Discovery::DeviceInterface object. At least one of device or interface is required. network - Required. The IP address and netmask of a network in CIDR format to operate on. router_ip - Required. Router IP address to operate on. scheduled_at - Required. Scheduled time in ISO 8601 extended format for date and time (e.g. 2008-10-21T10:56:00Z or 2008-10-21T10:56:00.001Z) or 'now' which will set scheduled date and time to the current date and time. enable_dhcp_relay - Optional. Enables DHCP relay. Valid value is 'true' and 'false'. Default value is 'false'. network_view - Optional. An Infoblox::DHCP::View object. vlan_info - Optional. An Infoblox::Grid::Discovery::VLANInfo object.

Returns

The method returns 'true' when a scheduled task has been created successfully, and returns 'false' when the operation fails.

Example

```
 my $res = $discovery->provision_network_port('scheduled_at' => '2008-10-21T10:56:00Z', 'device' =>
$device, 'network' => '10.0.0.0/8', router_ip => '10.0.0.1');
```

start_discovery( )

Use this method to start discovery on specified objects.

Parameter

objects - Required. An array reference that contains Infoblox::IPAM::Address, Infoblox::DHCP::Network, Infoblox::DHCP::IPv6Network, Infoblox::DHCP::NetworkContainer and Infoblox::DHCP::IPv6NetworkContainer objects that have been retrieved from the appliance.

Returns

The method returns true when the modification succeeds, and returns false when the operation fails.

Example

```
 my @networks = $discovery->get('object' => 'Infoblox::DHCP::Network');
 my $res = $discovery->start_discovery('objects' => \@networks);
```

Infoblox::Grid::Discovery - Grid discovery functions.

SEE ALSO

Infoblox::Grid::Discovery::DiagnosticsStatus, Infoblox::IPAM::Address, Infoblox::DHCP::Network, Infoblox::DHCP::IPv6Network, Infoblox::DHCP::NetworkContainer, Infoblox::DHCP::IPv6NetworkContainer, Infoblox::Grid::Discovery::NetworkDeprovisionInfo, Infoblox::Grid::Discovery::Port::Control, Infoblox::Grid::Discovery::Device, Infoblox::Grid::Discovery::JobProcessDetails, Infoblox::Grid::Discovery::DeviceInterface, Infoblox::Grid::Discovery::VLANInfo, Infoblox::Grid::Discovery::DiscoveryDataConversionResult, Infoblox::Grid::Discovery::ConversionAttributes, Infoblox::Grid::Discovery::DeviceSupportInfoResponse.

AUTHOR

Infoblox Inc. http://www.infoblox.com/

COPYRIGHT

Copyright (c) 2017 Infoblox Inc.

Infoblox::Grid::Discovery::AdvancedPollSettings - advanced polling settings.

- NAME
- DESCRIPTION
- CONSTRUCTOR
- SESSION METHODS
  - arp_aggregate_limit( )
  - arp_cache_refresh_interval( )
  - dhcp_router_as_seed( )
  - disable_discovery_outside_ipam( )
  - enable_purge_expired_endhost_data( )
  - ping_retries( )
  - ping_sweep_interval( )
  - ping_timeout( )
  - polling_authenticate_snmpv1_only( )
  - purge_expired_device_data( )
  - purge_expired_endhost_data( )
  - route_limit( )
  - tcp_scan_technique( )
- AUTHOR
- COPYRIGHT

NAME

Infoblox::Grid::Discovery::AdvancedPollSettings - advanced polling settings.

DESCRIPTION

The object provides information about advanced polling settings.

CONSTRUCTOR

```
my $object = Infoblox::Grid::Discovery::AdvancedPollSettings->new(
    'arp_aggregate_limit'               => $uint,                  # Optional
    'arp_cache_refresh_interval'        => $uint,                  # Optional
    'dhcp_router_as_seed'               => 'true' | 'false',       # Optional / Default is 'false'
    'disable_discovery_outside_ipam'    => 'true' | 'false',       # Optional / Default is 'false'
    'enable_purge_expired_endhost_data' => 'true' | 'false',       # Optional / Default is 'false'
    'ping_retries'                      => $uint,                  # Optional
    'ping_sweep_interval'               => $uint,                  # Optional
    'ping_timeout'                      => $uint,                  # Optional
    'polling_authenticate_snmpv1_only'  => 'true' | 'false',       # Optional / Default is 'false'
    'purge_expired_device_data'         => $uint,                  # Optional
    'purge_expired_endhost_data'        => $uint,                  # Optional
    'route_limit'                       => $uint,                  # Optional
    'tcp_scan_technique'                => 'CONNECT' | 'SYN',      # Optional
);
```

SESSION METHODS

The object does not support any session methods.

arp_aggregate_limit( )

Use this method to set or retrieve the ARP aggregate limit.

Parameter

Valid value is an unsigned integer.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
# Get attribute value
my $value = $object->arp_aggregate_limit();
# Modify attribute value
```

```
$object->arp_aggregate_limit(10);
```

arp_cache_refresh_interval( )

Use this method to set or retrieve the refresh interval for ARP cache in second.

Parameter

Valid value is an unsigned integer.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
# Get attribute value
my $value = $object->arp_cache_refresh_interval();
# Modify attribute value
$object->arp_cache_refresh_interval(5);
```

dhcp_router_as_seed( )

Use this method to enable or disable the flag that controls using DHCP routers as seed routers for discovery.

Parameter

Specify "true" to enable the flag or "false" to disable it. Default value is "false".

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
# Get attribute value
my $value = $object->dhcp_router_as_seed();
# Modify attribute value
$object->dhcp_router_as_seed('true');
```

disable_discovery_outside_ipam( )

Use this method to enable or disable the discovery of networks that are not in IPAM.

Parameter

Specify "true" to disable the discovery of networks that are not in IPAM or "false" to enable it. Default value is "false".

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
# Get attribute value
my $value = $object->disable_discovery_outside_ipam();
# Modify attribute value
$object->disable_discovery_outside_ipam('true');
```

enable_purge_expired_endhost_data( )

Use this method to enable or disable the purge of expired end host data.

Parameter

Specify "true" to enable the purge of expired end host data or "false" to disable it. Default value is "false".

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
# Get attribute value
my $value = $object->enable_purge_expired_endhost_data();
# Modify attribute value
$object->enable_purge_expired_endhost_data('true');
```

ping_retries( )

Use this method to set or retrieve ping retries.

Parameter

Valid value is an unsigned integer.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
# Get attribute value
my $value = $object->ping_retries();
# Modify attribute value
$object->ping_retries(3);
```

ping_sweep_interval( )

Use this method to set or retrieve the hourly wait interval between ping sweeps for individual discovery ranges.

Parameter

Valid value is an unsigned integer.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
# Get attribute value
my $value = $object->ping_sweep_interval();
# Modify attribute value
$object->ping_sweep_interval(1);
```

ping_timeout( )

Use this method to set or retrieve the ping timeout in seconds.

Parameter

Valid value is an unsigned integer.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
# Get attribute value
my $value = $object->ping_timeout();
# Modify attribute value
$object->ping_timeout(10);
```

polling_authenticate_snmpv1_only( )

Use this method to enable or disable polling and authenticating using SNMPv1 only.

Parameter

Specify "true" to enable polling and authenticating using SNMPv1 only or "false" to disable it. Default value is "false".

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
# Get attribute value
my $value = $object->polling_authenticate_snmpv1_only();
# Modify attribute value
$object->polling_authenticate_snmpv1_only('true');
```

purge_expired_device_data( )

Use this method to set or retrieve the number of days a device remains in database after it is no longer found in network.

Parameter

Valid value is an unsigned integer.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
# Get attribute value
my $value = $object->purge_expired_device_data();
# Modify attribute value
$object->purge_expired_device_data(5);
```

purge_expired_endhost_data( )

Use this method to set or retrieve the number of days an end host remains in database after it is no longer found in network.

Parameter

Valid value is an unsigned integer.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
# Get attribute value
my $value = $object->purge_expired_endhost_data();
# Modify attribute value
$object->purge_expired_endhost_data(5);
```

route_limit( )

Use this method to set or retrieve the route limit.

Parameter

Valid value is an unsigned integer.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
# Get attribute value
my $value = $object->route_limit();
# Modify attribute value
$object->route_limit(10);
```

tcp_scan_technique( )

Use this method to set or retrieve the TCP scan method.

Parameter

Valid value is 'CONNECT' or 'SYN'.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
# Get attribute value
my $value = $object->tcp_scan_technique();
# Modify attribute value
$object->tcp_scan_technique('SYN');
```

AUTHOR

Infoblox Inc. http://www.infoblox.com/

COPYRIGHT

Copyright (c) 2017 Infoblox Inc.

Infoblox::Grid::Discovery::AutoConversionSetting - Auto conversion setting.

- NAME
- DESCRIPTION
- CONSTRUCTOR
- SESSION METHODS
- METHODS
  - comment( )
  - condition( )
  - format( )
  - network_view( )
  - type( )
- AUTHOR
- COPYRIGHT

NAME

Infoblox::Grid::Discovery::AutoConversionSetting - Auto conversion setting.

DESCRIPTION

The object contains settings for automatic conversion of discovered data to managed objects.

CONSTRUCTOR

```
my $object = Infoblox::Grid::Discovery::AutoConversionSetting->new(
    'comment'           => $string,     # Optional
    'condition'         => $string,     # Optional
    'format'            => $string,     # Required
    'network_view'      => $string,     # Required
    'type'              => 'A_AND_PTR_RECORD' | 'HOST_RECORD' | 'FIXED_ADDRESS', # Required
);
```

SESSION METHODS

The object does not support any session methods.

METHODS

This section describes all the methods that you can use to set or retrieve the attribute values of the object.

comment( )

Use this method to set or retrieve the comment for the automatic conversion setting.

Parameter

The comment in string format.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
# Get attribute value
my $value = $object->comment();
# Modify attribute value
$object->comment('comment');
```

condition( )

Use this method to set or retrieve the condition for automatic conversion settings.

Parameter

The string contains the condition for automatic conversion settings.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
# Get attribute value
my $value = $object->condition();
# Modify attribute value
$object->condition('${discovered_name} == "unknown"');
```

format( )

Use this method to set or retrieve the format of the template for conversion.

Parameter

Template string used to generate host names.

Note that using double quotes may lead to interpolation of variables. Instead, use single quotes or encapsulated '$' characters.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
# Get attribute value
my $value = $object->format();
# Modify attribute value
$object->format('${discovered_name}.test.com');
```

network_view( )

Use this method to set or retrieve the name of the network view which contains discovered data for convert.

Parameter

The name of the network view.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
# Get attribute value
my $value = $object->network_view();
# Modify attribute value
$object->network_view('default');
```

type( )

Use this method to set or retrieve the type of the converted object.

Parameter

The object type used to define the converted object. Valid value is one of the following object types: 'A_AND_PTR_RECORD', 'HOST_RECORD', 'FIXED_ADDRESS'.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
# Get attribute value
my $value = $object->type();
# Modify attribute value
$object->type('FIXED_ADDRESS');
```

AUTHOR

Infoblox Inc. http://www.infoblox.com/

COPYRIGHT

Copyright (c) 2017 Infoblox Inc.

Infoblox::Grid::Discovery::BasicPollSettings - basic polling settings.

- NAME
- DESCRIPTION
- CONSTRUCTOR
- SESSION METHODS
- METHODS
  - auto_arp_refresh_before_switch_port_polling( )
  - cli_collection( )
  - complete_ping_sweep( )
  - device_profile( )
  - netbios_scanning( )
  - port_scanning( )
  - smart_subnet_ping_sweep( )
  - snmp_collection( )
  - switch_port_data_collection_polling( )
  - switch_port_data_collection_polling_interval( )
  - switch_port_data_collection_polling_schedule( )
- AUTHOR
- SEE ALSO
- COPYRIGHT

NAME

Infoblox::Grid::Discovery::BasicPollSettings - basic polling settings.

DESCRIPTION

The object provides information about basic polling settings.

CONSTRUCTOR

```
 my $object = Infoblox::Grid::Discovery::BasicPollSettings->new(
    'auto_arp_refresh_before_switch_port_polling'  => 'true' | 'false',                    # Optional /
Default value is 'false'
    'device_profile'                               => 'true' | 'false',                    # Optional /
Default value is 'false'
    'complete_ping_sweep'                          => 'true' | 'false',                    # Optional /
Default value is 'false'
    'netbios_scanning'                             => 'true' | 'false',                    # Optional /
Default value is 'false'
    'port_scanning'                                => 'true' | 'false',                    # Optional /
Default value is 'false'
    'smart_subnet_ping_sweep'                      => 'true' | 'false',                    # Optional /
Default value is 'false'
    'snmp_collection'                              => 'true' | 'false',                    # Optional /
Default value is 'false'
    'cli_collection'                               => 'true' | 'false',                    # Optional /
Default value is 'false'
    'switch_port_data_collection_polling'          => 'DISABLED' | 'PERIODIC' | 'SCHEDULED',    # Optional
    'switch_port_data_collection_polling_interval' => $uint,                               # Optional
    'switch_port_data_collection_polling_schedule' => $schedule,                           # Optional
 );
```

SESSION METHODS

The object does not support any session methods.

METHODS

This section describes all the methods that you can use to set or retrieve the attribute values of the object.

auto_arp_refresh_before_switch_port_polling( )

Use this method to enable or disable auto ARP refresh before switch port polling.

Parameter

Specify "true" to enable auto ARP refresh before switch port polling or "false" to disable it. Default value is "false".

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
# Get attribute value
my $value = $object->auto_arp_refresh_before_switch_port_polling();
# Modify attribute value
$object->auto_arp_refresh_before_switch_port_polling('true');
```

cli_collection( )

Use this method to disable or enable CLI collection.

Parameter

Specify "true" to enable CLI collection or "false" to disable it. Default value if "false".

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
# Get attribute value
my $value = $object->cli_collection();
# Modify attribute value
$object->cli_collection('true');
```

complete_ping_sweep( )

Use this method to enable or disable ping sweep.

Parameter

Specify "true" to enable ping sweep or "false" to disable it. Default value is "false".

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
# Get attribute value
my $value = $object->complete_ping_sweep();
# Modify attribute value
$object->complete_ping_sweep('true');
```

device_profile( )

Use this method to enable or disable DHCP fingerprinting.

Parameter

Specify "true" to enable DHCP fingerprinting or "false" to disable it. Default value is "false".

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
# Get attribute value
my $value = $object->device_profile();
# Modify attribute value
$object->device_profile('true');
```

netbios_scanning( )

Use this method to enable or disable NETBIOS scanning.

Parameter

Specify "true" to enable NETBIOS scanning or "false" to disable it. Default value is "false".

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
# Get attribute value
my $value = $object->netbios_scanning();
# Modify attribute value
$object->netbios_scanning('true');
```

port_scanning( )

Use this method to enable or disable port scanning.

Parameter

Specify "true" to enable port scanning or "false" to disable it. Default value is "false".

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
# Get attribute value
my $value = $object->port_scanning();
# Modify attribute value
$object->port_scanning('true');
```

smart_subnet_ping_sweep( )

Use this method to enable or disable smart subnet ping sweep.

Parameter

Specify "true" to enable smart subnet ping sweep or "false" to disable it. Default value is "false".

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
# Get attribute value
my $value = $object->smart_subnet_ping_sweep();
# Modify attribute value
$object->smart_subnet_ping_sweep('true');
```

snmp_collection( )

Use this method to enable or disable SNMP collection.

Parameter

Specify "true" to enable SNMP collection or "false" to disable it. Default value is "false".

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
# Get attribute value
my $value = $object->snmp_collection();
# Modify attribute value
$object->snmp_collection('true');
```

switch_port_data_collection_polling( )

Use this method to set or retrieve the polling mode.

Parameter

Valid value is 'DISABLED', 'PERIODIC' or 'SCHEDULED'.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
# Get attribute value
my $value = $object->switch_port_data_collection_polling();
# Modify attribute value
$object->switch_port_data_collection_polling('SCHEDULED');
```

switch_port_data_collection_polling_interval( )

Use this method to set or retrieve the polling interval.

Parameter

Valid value is an unsigned integer.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
# Get attribute value
my $value = $object->switch_port_data_collection_polling_interval();
# Modify attribute value
$object->switch_port_data_collection_polling_interval(100);
```

switch_port_data_collection_polling_schedule( )

Use this method to set or retrieve the polling schedule.

Parameter

Valid value is an Infoblox::Grid::ScheduleSetting object.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
# Get attribute value
my $value = $object->switch_port_data_collection_polling_schedule();
# Modify attribute value
$object->switch_port_data_collection_polling_schedule($schedule);
```

AUTHOR

Infoblox Inc. http://www.infoblox.com/

SEE ALSO

Infoblox::Grid::ScheduleSetting

COPYRIGHT

Copyright (c) 2017 Infoblox Inc.

NAME

- NAME
- DESCRIPTION
- CONSTRUCTOR
- SESSION METHODS
- METHODS
  - id( )
  - comment( )
  - password( )
  - type( )
  - user( )
- AUTHOR
- SEE ALSO
- COPYRIGHT

NAME

Infoblox::Grid::Discovery::CLICredential

DESCRIPTION

The object provides information about CLI credentials.

CONSTRUCTOR

```
my $object = Infoblox::Grid::Discovery::CLICredential->new(
    'user'     => $string,                                          # optional
    'password' => $string,                                          # optional
    'comment'  => $string,                                          # optional
    'type'     => 'ENABLE_SSH' | 'ENABLE_TELNET' | 'SSH' | 'TELNET',   # required
);
```

SESSION METHODS

The object does not support any session methods.

METHODS

This section describes all the methods that you can use to set or retrieve the attribute values of the object.

id( )

Use this method to retrieve the credential ID. This attribute is read-only.

Parameter

None.

Returns

The method returns the attribute value.

Example

```
# Get attribute value
my $value = $object->id();
```

comment( )

Use this method to set or retrieve the descriptive comment.

Parameter

Desired comment in string format.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
# Get attribute value
my $value = $object->comment();
# Modify attribute value
$object->comment('CLI credentials');
```

password( )

Use this method to set the CLI user password. This attribute is write-only.

Parameter

The CLI password in string format.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

Example

```
# Modify attribute value
$object->password('infoblox');
```

type( )

Use this method to set or retrieve the credential type.

Parameter

The valid value is 'ENABLE_SSH', 'ENABLE_TELNET', 'SSH' or 'TELNET'.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
# Get attribute value
my $value = $object->typ();
# Modify attribute value
$object->type('ENABLE_SSH');
```

user( )

Use this method to set or retrieve the CLI user name.

Parameter

The CLI user name in string format.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
# Get attribute value
my $value = $object->user();
# Modify attribute value
$object->user('admin');
```

AUTHOR

Infoblox Inc. http://www.infoblox.com/

SEE ALSO

Infoblox::Grid::Discovery::Properties

COPYRIGHT

Copyright (c) 2017 Infoblox Inc.

Infoblox::Grid::Discovery::ConversionAttributes - conversion attributes.

- NAME
- DESCRIPTION
- SESSION METHODS
- METHODS
  - configure_for_dhcp()
  - configure_for_dns()
  - disabled()
  - comment()
  - zone()
- AUTHOR
- SEE ALSO
- COPYRIGHT

NAME

Infoblox::Grid::Discovery::ConversionAttributes - conversion attributes.

DESCRIPTION

The object represents the list of attributes for discovery data conversion.

SESSION METHODS

The object does not support any session methods.

METHODS

configure_for_dhcp()

Use this method to set the configure_for_dhcp flag of a DHCP Host Address object.

Parameter

Specify "true" to set the configure_for_dhcp flag or "false" to deactivate/unset it. The default value is "false".

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Set value
my $value = $conversion_attributes->configure_for_dhcp('true');
```

configure_for_dns()

Use this method to set the configure_for_dns flag of a DNS host.

Parameter

Specify "true" to set the configure_for_dns flag or "false" to deactivate/unset it. The default value is "false".

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Set value
my $value = $conversion_attributes->configure_for_dns('true');
```

disabled()

Use this method to set or retrieve the flag that indicates whether the target object should be disabled or not.

Parameter

Specify "true" to set the disable flag or "false" to deactivate/unset it. The default value is "false".

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Set value
my $value = $conversion_attributes->disable('true');
```

comment()

Use this method to set or retrieve a descriptive comment.

Parameter

Desired comment in string format with a maximum of 256 bytes. The default value is undefined.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Set value
my $value = $conversion_attributes->comment('test comment');
```

zone()

Use this method to set the authoritative zone for DNS record.

Parameter

The valid value is an Infoblox::DNS::Zone object.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
my $zone = $discovery->get('object' => 'Infoblox::DNS::Zone', 'name' => 'papi.com');
#Set value
my $value = $conversion_attributes->zone($zone);
```

AUTHOR

Infoblox Inc. http://www.infoblox.com/

SEE ALSO

Infoblox::Grid::Discovery, Infoblox::DNS::Zone.

COPYRIGHT

Copyright (c) 2017 Infoblox Inc.

Infoblox::Grid::Discovery::Data - the discovered data.

- NAME
- DESCRIPTION
- CONSTRUCTOR
- SESSION METHODS
- METHODS
  - device_model( )
  - device_port_name( )
  - device_port_type( )
  - device_type( )
  - device_vendor( )
  - discovered_name( )
  - discoverer( )
  - duid( )
  - endpoint_groups( )
  - first_discovered( )
  - iprg_no( )
  - iprg_state( )
  - iprg_type( )
  - last_discovered( )
  - mac_address( )
  - mgmt_ip_address( )
  - netbios( )
  - network_component_description( )
  - network_component_ip( )
  - network_component_model( )
  - network_component_name( )
  - network_component_port_description( )
  - network_component_port_name( )
  - network_component_port_number( )
  - network_component_type( )
  - network_component_vendor( )
  - open_ports( )
  - os( )
  - port_duplex( )
  - port_link_status( )
  - port_speed( )
  - port_status( )
  - port_type( )
  - port_vlan_description( )
  - port_vlan_name( )
  - port_vlan_number( )
  - task_name( )
  - tenant( )
  - network_component_location( )
  - ap_name( )
  - ap_ip_address( )
  - ap_ssid( )
  - bridge_domain( )
  - network_component_contact( )
  - device_location( )
  - device_contact( )
  - v_cluster( )
  - v_datacenter( )
  - v_host( )
  - v_name( )
  - v_netadapter( )
  - v_os( )
  - v_switch( )
  - v_type( )
  - vmi_name( )
  - vmi_id( )
  - vmi_tenant_id( )
  - vlan_port_group( )
  - vswitch_name( )
  - vswitch_id( )
  - vswitch_type( )
  - vswitch_ipv6_enabled( )
  - vport_name( )
  - vport_mac_address( )
  - vport_link_status( )
  - vport_conf_speed( )

- vport_conf_mode( )
- vport_speed( )
- vport_mode( )
- vswitch_segment_type( )
- vswitch_segment_name( )
- vswitch_segment_id( )
- vswitch_segment_port_group( )
- vswitch_available_ports_count( )
- vswitch_tep_type( )
- vswitch_tep_ip ( )
- vswitch_tep_port_group( )
- vswitch_tep_vlan( )
- vswitch_tep_dhcp_server( )
- vswitch_tep_multicast( )
- vmhost_ip_address( )
- vmhost_name( )
- vmhost_mac_address ( )
- vmhost_subnet_cidr ( )
- vmhost_nic_names( )
- cisco_ise_endpoint_profile( )
- cisco_ise_security_group( )
- cisco_ise_session_state( )
- cisco_ise_ssid( )
- cmp_type( )
- vmi_ip_type( )
- vmi_private_address( )
- vmi_is_public_address( )

- SAMPLE CODE
- AUTHOR
- COPYRIGHT

NAME

Infoblox::Grid::Discovery::Data - the discovered data.

DESCRIPTION

The object provides information about the discovered data.

CONSTRUCTOR

```
my $object = Infoblox::Grid::Discovery::Data->new(
    'ap_name'                           => $string,
    'ap_ip_address'                     => $ipaddr,
    'ap_ssid'                           => $string,
    'bridge_domain                      => $string,
    'duid'                              => $string,
    'mac_address'                       => $string,
    'netbios'                           => $string,
    'os'                                => $string,
    'device_model'                      => $string,
    'device_port_name'                  => $string,
    'device_port_type'                  => $string,
    'device_type'                       => $string,
    'device_vendor'                     => $string,
    'discovered_name'                   => $string,
    'discoverer'                        => $string,
    'endpoint_groups'                   => $string,
    'first_discovered'                  => $datetime,
    'last_discovered'                   => $datetime,
    'mgmt_ip_address'                   => $ipaddr,
    'network_component_description'     => $string,
    'network_component_ip'              => $ipaddr,
    'network_component_model'           => $string,
    'network_component_name'            => $string,
    'network_component_port_description' => $string,
    'network_component_port_name'       => $string,
    'network_component_port_number'     => $uint,
    'network_component_type'            => $string,
    'network_component_vendor'          => $string,
    'open_ports'                        => $string,
    'port_duplex'                       => 'Full' | 'Half',
```

```
    'port_link_status'                  => 'Connected' | 'Not Connected' | 'Unknown',
    'port_speed'                        => '100G' | '100M' | '10G' | '10M' | '1G' | 'Unknown',
    'port_status'                       => 'Down' | 'Unknown' | 'Up',
    'port_type'                         => $string,
    'port_vlan_description'             => $string,
    'port_vlan_name'                    => $string,
    'port_vlan_number'                  => $uint,
    'task_name'                         => $string,
    'tenant'                            => $string,
    'network_component_location'        => $string,
    'network_component_contact'         => $string,
    'device_location'                   => $string,
    'device_contact'                    => $string,
    'v_netadapter'                      => $string,
    'v_cluster'                         => $string,
    'v_datacenter'                      => $string,
    'v_name'                            => $string,
    'v_type'                            => 'VirtualCenter' | 'HostSystem' | 'VirtualMachine',
    'v_host'                            => $string,
    'v_os'                              => $string,
    'v_switch'                          => $string,
);
```

SESSION METHODS

The object does not support any session methods.

METHODS

This section describes all the methods that you can use to set or retrieve the attribute values of the discovered data object.

device_model( )

Use this method to set or retrieve the model name of the device in the vendor terminology.

Parameter

The model name of the device in string format.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
# Get attribute value
my $value = $object->device_model();
# Modify attribute value
$object->device_model('model');
```

device_port_name( )

Use this method to set or retrieve the system name of the interface the IP associates with.

Parameter

The system name of the interface in string format.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
# Get attribute value
my $value = $object->device_port_name();
# Modify attribute value
$object->device_port_name('port');
```

device_port_type( )

Use this method to set or retrieve the hardware type of the interface the IP associates with.

Parameter

The hardware type of the interface in string format.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
# Get attribute value
my $value = $object->device_port_type();
# Modify attribute value
$object->device_port_type('type');
```

device_type( )

Use this method to set or retrieve the type of the device in the vendor terminology.

Parameter

The type of the device in string format.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
# Get attribute value
my $value = $object->device_type();
# Modify attribute value
$object->device_type('type');
```

device_vendor( )

Use this method to set or retrieve the vendor name of the device.

Parameter

The vendor name of the device in string format.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
# Get attribute value
my $value = $object->device_vendor();
# Modify attribute value
$object->device_vendor('vendor');
```

discovered_name( )

Use this method to set or retrieve the name of the IP as seen by the discovery station.

Parameter

The discovered name in string format.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
# Get attribute value
```

```
my $value = $object->discovered_name();
# Modify attribute value
$object->discovered_name('name');
```

discoverer( )

Use this method to set or retrieve the name of the discoverer.

Parameter

The name of the discoverer in string format.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
# Get attribute value
my $value = $object->discoverer();
# Modify attribute value
$object->discoverer('discoverer');
```

duid( )

Use this method to set or retrieve the discovered DHCPv6 UID (DUID).

Parameter

The discovered DHCPv6 UID (DUID) in string format.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
# Get attribute value
my $value = $object->duid();
# Modify attribute value
$object->duid('aa:aa');
```

endpoint_groups( )

Use this method to retrieve comma-separated list of discovered endpoint groups. This is a read-only attribute.

Parameter

None

Returns

The method returns the attribute value.

Example

```
# Get attribute value
my $value = $object->endpoint_groups();
```

first_discovered( )

Use this method to set or retrieve the date and time when the object was first discovered.

Parameter

The date and time in the ISO 8601 extended format (e.g. 2008-10-21T10:56:33Z or 2008-10-21T10:56:33.001Z).

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
# Get attribute value
my $value = $object->first_discovered();
# Modify attribute value
$object->first_discovered('2008-10-21T10:56:33Z');
```

iprg_no( )

Use this method to set or retrieve the port redundant group number.

Parameter

Valid value is an unsigned integer which describes port redundant group number.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
# Get attribute value
my $value = $object->iprg_no();
# Modify attribute value
$object->iprg_no(10);
```

iprg_state( )

Use this method to set or retrieve the status for the IP address within port redundant group.

Parameter

Valid value is "VIP", "ACTIVE", "NEGOTIATION" or "STANDBY".

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
# Get attribute value
my $value = $object->iprg_state();
# Modify attribute value
$object->iprg_state('ACTIVE');
```

iprg_type( )

Use this method to set or retrieve the port redundant group type.

Parameter

Valid value is "HSRP" or "VRRP".

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
# Get attribute value
my $value = $object->iprg_type();
# Modify attribute value
$object->iprg_type('HSRP');
```

last_discovered( )

Use this method to set or retrieve the date and time when the object was last discovered.

Parameter

The date and time in the ISO 8601 extended format (e.g. 2008-10-21T10:56:33Z or 2008-10-21T10:56:33.001Z).

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
# Get attribute value
my $value = $object->last_discovered();
# Modify attribute value
$object->last_discovered('2008-10-21T10:56:33Z');
```

mac_address( )

Use this method to set or retrieve the MAC address.

Parameter

The MAC address in string format.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
# Get attribute value
my $value = $object->mac_address();
# Modify attribute value
$object->mac_address('11:11:11:11:11:11');
```

mgmt_ip_address( )

Use this method to set or retrieve the management IP address of the device if the device has more than one IP.

Parameter

The management IP address of the device.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
# Get attribute value
my $value = $object->mgmt_ip_address();
# Modify attribute value
$object->mgmt_ip_address('10.0.0.1');
```

netbios( )

Use this method to set or retrieve the discovered NetBIOS name.

Parameter

The discovered NetBIOS name in string format.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
# Get attribute value
my $value = $object->netbios();
# Modify attribute value
$object->netbios('netbios');
```

network_component_description( )

Use this method to set or retrieve the descriptive string for the network component.

Parameter

The descriptive string for the network component.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
# Get attribute value
my $value = $object->network_component_description();
# Modify attribute value
$object->network_component_description('description');
```

network_component_ip( )

Use this method to set or retrieve the IP Address the network component.

Parameter

The IP Address the network component.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
# Get attribute value
my $value = $object->network_component_ip();
# Modify attribute value
$object->network_component_ip('10.0.0.2');
```

network_component_model( )

Use this method to set or retrieve the model name of the network component.

Parameter

The model name of the network component in string format.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
# Get attribute value
my $value = $object->network_component_model();
# Modify attribute value
$object->network_component_model('model');
```

network_component_name( )

Use this method to set or retrieve the name of the network component.

Parameter

The name of the network component in string format.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
# Get attribute value
my $value = $object->network_component_name();
# Modify attribute value
$object->network_component_name('name');
```

network_component_port_description( )

Use this method to set or retrieve the description of the port on the network component.

Parameter

The description of the port on the network component in string format.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
# Get attribute value
my $value = $object->network_component_port_description();
# Modify attribute value
$object->network_component_port_description('description');
```

network_component_port_name( )

Use this method to set or retrieve the port name on the network component on which the IP was discovered.

Parameter

The port name on the network component in string format.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
# Get attribute value
my $value = $object->network_component_port_name();
# Modify attribute value
$object->network_component_port_name('port');
```

network_component_port_number( )

Use this method to set or retrieve the port number on the network component on which the IP was discovered.

Parameter

The port number on the network component.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
# Get attribute value
my $value = $object->network_component_port_number();
# Modify attribute value
$object->network_component_port_number(10);
```

network_component_type( )

Use this method to set or retrieve the type of network component.

Parameter

The type of network component in string format.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
# Get attribute value
my $value = $object->network_component_type();
# Modify attribute value
$object->network_component_type('type');
```

network_component_vendor( )

Use this method to set or retrieve the vendor name of the network component to which the device is connected.

Parameter

The vendor name of the network component in string format.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
# Get attribute value
my $value = $object->network_component_vendor();
# Modify attribute value
$object->network_component_vendor('vendor');
```

open_ports( )

Use this method to set or retrieve the list of opened ports on the IP address, represented as: "TCP: 21,22,23 UDP: 137,139".

Parameter

The list of opened ports in string format.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
# Get attribute value
my $value = $object->open_ports();
# Modify attribute value
$object->open_ports('TCP: 21,22,23 UDP: 137,139');
```

os( )

Use this method to set or retrieve the name of the operation system by network discovery.

Parameter

The name of the operation system in string format.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
# Get attribute value
my $value = $object->os();
# Modify attribute value
$object->os('linux');
```

port_duplex( )

Use this method to set or retrieve duplex settings on the port on the network component.

Parameter

Valid value is 'Full' or 'Half'.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
# Get attribute value
my $value = $object->port_duplex();
# Modify attribute value
$object->port_duplex('Full');
```

port_link_status( )

Use this method to set or retrieve the link status of the port on the network component.

Parameter

Valid value is 'Connected', 'Not Connected' or 'Unknown'.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
# Get attribute value
my $value = $object->port_link_status();
# Modify attribute value
$object->port_link_status('Connected');
```

port_speed( )

Use this method to set or retrieve speed settings on the port on the network component.

Parameter

Valid value is '10M', '100M', '1G', '10G', '100G' or 'Unknown'.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
# Get attribute value
my $value = $object->port_speed();
# Modify attribute value
$object->port_speed('Unknown');
```

port_status( )

Use this method to set or retrieve the status of the port on the network component.

Parameter

Valid value is 'Up', 'Down' or 'Unknown'.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
# Get attribute value
my $value = $object->port_status();
# Modify attribute value
$object->port_status('Unknown');
```

port_type( )

Use this method to set or retrieve the type of the interface on the network component to which the device is connected.

Parameter

The type of the interface on the network component in string format.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
# Get attribute value
my $value = $object->port_type();
# Modify attribute value
$object->port_type('type');
```

port_vlan_description( )

Use this method to set or retrieve the description of the VLAN on the port on the network component.

Parameter

The description of the VLAN on the port on the network component in string format.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
# Get attribute value
my $value = $object->port_vlan_description();
# Modify attribute value
$object->port_vlan_description('description');
```

port_vlan_name( )

Use this method to set or retrieve the name of the VLAN on the port on the network component.

Parameter

The name of the VLAN on the port on the network component in string format.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
# Get attribute value
my $value = $object->port_vlan_name();
# Modify attribute value
$object->port_vlan_name('name');
```

port_vlan_number( )

Use this method to set or retrieve the number VLANs on the port on the network component.

Parameter

The number of VLANs on the port on the network component.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
# Get attribute value
my $value = $object->port_vlan_number();
# Modify attribute value
$object->port_vlan_number(10);
```

task_name( )

Use this method to retrieve the name of discovery task. This is a read-only attribute.

Parameter

None

Returns

The method returns the attribute value.

Example

```
# Get attribute value
my $value = $object->task_name();
```

tenant( )

Use this method to retrieve the discovered tenant. This is a read-only attribute.

Parameter

None

Returns

The method returns the attribute value.

Example

```
# Get attribute value
my $value = $object->tenant();
```

network_component_location( )

Use this method to retrieve location of the network component on which the IP address was discovered. This is a read-only attribute.

Parameter

None

Returns

The method returns the attribute value.

Example

```
# Get attribute value
my $value = $object->network_component_location();
```

ap_name( )

Use this method to retrieve discovered name of Wireless Access Point. This is a read-only attribute.

Parameter

None

Returns

The method returns the attribute value.

Example

```
# Get attribute value
my $value = $object->ap_name();
```

ap_ip_address( )

Use this method to retrieve discovered IP address of Wireless Access Point. This is a read-only attribute.

Parameter

None

Returns

The method returns the attribute value.

Example

```
# Get attribute value
my $value = $object->ap_ip_address();
```

ap_ssid( )

Use this method to retrieve service set identifier (SSID) associated with Wireless Access Point. This is a read-only attribute.

Parameter

None

Returns

The method returns the attribute value.

Example

```
# Get attribute value
my $value = $object->ap_ssid();
```

bridge_domain( )

Use this method to retrieve the discovered bridge domain. This is a read-only attribute.

Parameter

None

Returns

The method returns the attribute value.

Example

```
# Get attribute value
my $value = $object->bridge_domain();
```

network_component_contact( )

Use this method to retrieve contact information from the network component on which the IP address was discovered. This is a read-only attribute.

Parameter

None

Returns

The method returns the attribute value.

Example

```
# Get attribute value
my $value = $object->network_component_contact();
```

device_location( )

Location of device on which the IP address was discovered. This is a read-only attribute.

Parameter

None

Returns

The method returns the attribute value.

Example

```
# Get attribute value
my $value = $object->device_location();
```

device_contact( )

Contact information from device on which the IP address was discovered. This is a read-only attribute.

Parameter

None

Returns

The method returns the attribute value.

Example

```
# Get attribute value
my $value = $object->device_contact();
```

v_cluster( )

Use this method to set or retrieve the VMware cluster in the entity found.

Parameter

The VMware cluster in string format.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
# Get attribute value
my $value = $object->v_cluster();
# Modify attribute value
$object->v_cluster('cluster');
```

v_datacenter( )

Use this method to set or retrieve the VMware datacenter in the entity found.

Parameter

The VMware datacenter in string format.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
# Get attribute value
my $value = $object->v_datacenter();
# Modify attribute value
$object->v_datacenter('datacenter');
```

v_host( )

Use this method to set or retrieve the VMware host system in the entity found.

Parameter

The VMware host system in string format.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
# Get attribute value
my $value = $object->v_host();
# Modify attribute value
$object->v_host('host');
```

v_name( )

Use this method to set or retrieve the VMware entity name in the entity found.

Parameter

The VMware entity name in string format.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
# Get attribute value
my $value = $object->v_name();
# Modify attribute value
$object->v_name('name');
```

v_netadapter( )

Use this method to set or retrieve the VMware physical adapter in the entity found.

Parameter

The VMware physical adapter in string format.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
# Get attribute value
my $value = $object->v_netadapter();
# Modify attribute value
$object->v_netadapter('netadapter');
```

v_os( )

Use this method to set or retrieve the VMware Operating System in the entity found.

Parameter

The VMware Operating System in string format.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
# Get attribute value
my $value = $object->v_os();
# Modify attribute value
$object->v_os('linux');
```

v_switch( )

Use this method to set or retrieve the VMware virtual switch in the entity found.

Parameter

The VMware virtual switch in string format.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
# Get attribute value
my $value = $object->v_switch();
```

```
# Modify attribute value
$object->v_switch('switch');
```

v_type( )

Use this method to set or retrieve the type of encryption to use.

Parameter

Valid value is 'VirtualCenter', 'HostSystem' or 'VirtualMachine'.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
# Get attribute value
my $value = $object->v_type();
# Modify attribute value
$object->v_type('VirtualMachine');
```

vmi_name( )

Use this method to retrieve the name of the virtual machine. This is a read-only attribute.

Parameter

None

Returns

The method returns the attribute value.

Example

```
# Get attribute value
my $value = $object->vmi_name();
```

vmi_id( )

Use this method to retrieve the ID of the virtual machine. This is a read-only attribute.

Parameter

None

Returns

The method returns the attribute value.

Example

```
# Get attribute value
my $value = $object->vmi_id();
```

vmi_tenant_id( )

Use this method to retrieve the Tenant ID to which the virtual machine belongs. This is a read-only attribute

Parameter

None

Returns

The method returns the attribute value.

Example

```
# Get attribute value
my $value = $object->vmi_tenant_id();
```

vlan_port_group( )

Use this method to retrieve the port group to which the virtual machine belongs. This is a read-only attribute.

Parameter

None

Returns

The method returns the attribute value.

Example

```
# Get attribute value
my $value = $object->vlan_port_group();
```

vswitch_name( )

Use this method to retrieve the name of the virtual switch. This is a read-only attribute.

Parameter

None

Returns

The method returns the attribute value.

Example

```
# Get attribute value
my $value = $object->vswitch_name();
```

vswitch_id( )

Use this method to retrieve the ID of the virtual switch. This is a read-only attribute.

Parameter

None

Returns

The method returns the attribute value.

Example

```
# Get attribute value
my $value = $object->vswitch_id();
```

vswitch_type( )

Use this method to retrieve the type of the virtual switch: Standard or Distributed. 'Unknown' indicates the value is not avialable. This is a read-only attribute.

Parameter

None

Returns

The method returns the attribute value.

Example

```
# Get attribute value
my $value = $object->vswitch_type();
```

vswitch_ipv6_enabled( )

Use this method to retrieve the flag used to indicate if the virtual swich has IPv6 enabled or not. This is a read-only attribute.

Parameter

None

Returns

The method returns the attribute value.

Example

```
# Get attribute value
my $value = $object->vswitch_ipv6_enabled();
```

vport_name( )

Use this method to retrieve the name of the network adapter on the virtual switch to which the virtual machine is connected. This is a read-only attribute.

Parameter

None

Returns

The method returns the attribute value.

Example

```
# Get attribute value
my $value = $object->vport_name();
```

vport_mac_address( )

Use this method to retrieve the MAC address of the network adapter on the virtual switch to which the the virtual machine is connected. This is a read-only attribute.

Parameter

None

Returns

The method returns the attribute value.

Example

```
# Get attribute value
my $value = $object->vport_mac_address();
```

vport_link_status( )

Use this method to retrieve the status of the link of the network adapter on the virtual switch to which the virtual machine is connected. This is a read-only attribute.

Parameter

None

Returns

The method returns the attribute value.

Example

```
# Get attribute value
my $value = $object->vport_link_status();
```

vport_conf_speed( )

Use this method to retrieve the configured speed (in Kb) of the network adapter on the virtual switch to which the virtual machine is connected. Example: 1500Kb. This is a read-only attribute.

Parameter

None

Returns

The method returns the attribute value.

Example

```
# Get attribute value
my $value = $object->vport_conf_speed();
```

vport_conf_mode( )

Use this method to retrieve the configured mode of the network adapter on the virtual switch to which the virtual machine is connected. Valid value is Full-duplex or Half-duplex. 'Unknown' indicates the value is not available. This is a read-only attribute.

Parameter

None

Returns

The method returns the attribute value.

Example

```
# Get attribute value
my $value = $object->vport_conf_mode();
```

vport_speed( )

Use this method to retrieve the actual speed (in Kb) of the network adapter on the virtual switch to which the virtual machine is connected. Example: 1500Kb. This is a read-only attribute.

Parameter

None

Returns

The method returns the attribute value.

Example

```
# Get attribute value
my $value = $object->vport_speed();
```

vport_mode( )

Use this method to retrieve the actual mode of the network adapter on the virtual switch to which the virtual machine is connected. Valid value is Full-duplex or Half-duplex. 'Unknown' indicates the value is not available. This is a read-only attribute.

Parameter

None

Returns

The method returns the attribute value.

Example

```
# Get attribute value
my $value = $object->vport_mode();
```

vswitch_segment_type( )

Use this method to retrieve the type of the network segment to which the current virtual machine/vport is connected. This is a read-only attribute.

Parameter

None

Returns

The method returns the attribute value.

Example

```
# Get attribute value
my $value = $object->vswitch_segment_type();
```

vswitch_segment_name( )

Use this method to retrieve the name of the network segment to which the current virtual machine/vport is connected. This is a read-only attribute.

Parameter

None

Returns

The method returns the attribute value.

Example

```
# Get attribute value
my $value = $object->vswitch_segment_name();
```

vswitch_segment_id( )

Use this method to retrieve the ID of the network segment to which the current virtual machine/vport is connected. This is a read-only attribute.

Parameter

None

Returns

The method returns the attribute value.

Example

```
# Get attribute value
my $value = $object->vswitch_segment_id();
```

vswitch_segment_port_group( )

Use this method to retrieve the port group of the network segment to which the current virtual machine/vport is connected. This is a read-only attribute.

Parameter

None

Returns

The method returns the attribute value.

Example

```
# Get attribute value
my $value = $object->vswitch_segment_port_group();
```

vswitch_available_ports_count( )

Use this method to retrieve the number of available ports reported by the virtual switch to which the virtual machine/vport is connected. This is a read-only attribute.

Parameter

None

Returns

The method returns the attribute value.

Example

```
# Get attribute value
my $value = $object->vswitch_available_ports_count();
```

vswitch_tep_type( )

Use this method to retrieve the type of virtual tunnel endpoint (VTEP) in the virtual switch. This is a read-only attribute.

Parameter

None

Returns

The method returns the attribute value.

Example

```
# Get attribute value
my $value = $object->vswitch_tep_type();
```

vswitch_tep_ip ( )

Use this method to retrieve the IP address of the virtual tunnel endpoint (VTEP) in the virtual switch. This is a read-only attribute.

Parameter

None

Returns

The method returns the attribute value.

Example

```
# Get attribute value
my $value = $object->vswitch_tep_ip ();
```

vswitch_tep_port_group( )

Use this method to retrieve the port group of the virtual tunnel endpoint (VTEP) in the virtual switch. This is a read-only attribute.

Parameter

None

Returns

The method returns the attribute value.

Example

```
# Get attribute value
my $value = $object->vswitch_tep_port_group();
```

vswitch_tep_vlan( )

Use this method to retrieve the VLAN of the virtual tunnel endpoint (VTEP) in the virtual switch. This is a read-only attribute.

Parameter

None

Returns

The method returns the attribute value.

Example

```
# Get attribute value
my $value = $object->vswitch_tep_vlan();
```

vswitch_tep_dhcp_server( )

Use this method to retrieve the DHCP server of the virtual tunnel endpoint (VTEP) in the virtual switch. This is a read-only attribute.

Parameter

None

Returns

The method returns the attribute value.

Example

```
# Get attribute value
my $value = $object->vswitch_tep_dhcp_server();
```

vswitch_tep_multicast( )

Use this method to retrieve the multicast address of the virtual tunnel endpoint (VTEP) in the virtual swtich. This is a read-only attribute.

Parameter

None

Returns

The method returns the attribute value.

Example

```
# Get attribute value
my $value = $object->vswitch_tep_multicast();
```

vmhost_ip_address( )

Use this method to retrieve the IP address of the physical node on which the virtual machine is hosted. This is a read-only attribute.

Parameter

None

Returns

The method returns the attribute value.

Example

```
# Get attribute value
my $value = $object->vmhost_ip_address ();
```

vmhost_name( )

Use this method to retrieve the name of the physical node on which the virtual machine is hosted. This is a read-only attribute.

Parameter

None

Returns

The method returns the attribute value.

Example

```
# Get attribute value
my $value = $object->vmhost_name();
```

vmhost_mac_address ( )

Use this method to retrieve the MAC address of the physical node on which the virtual machine is hosted. This is a read-only attribute.

Parameter

None

Returns

The method returns the attribute value.

Example

```
# Get attribute value
my $value = $object->vmhost_mac_address ();
```

vmhost_subnet_cidr ( )

Use this method to retrieve the CIDR subnet of the physical node on which the virtual machine is hosted. This is a read-only attribute.

Parameter

None

Returns

The method returns the attribute value.

Example

```
# Get attribute value
my $value = $object->vmhost_subnet_cidr ();
```

vmhost_nic_names( )

Use this method to retrieve the list of all physical port names used by the virtual switch on the physical node on which the virtual machine is hosted. Represented as: "eth1,eth2,eth3". This is a read-only attribute.

Parameter

None

Returns

The method returns the attribute value.

Example

```
# Get attribute value
my $value = $object->vmhost_nic_names();
```

cisco_ise_endpoint_profile( )

Use this method to retrieve a name of the endpoint profile created in Cisco ISE. This is a read-only attribute.

Parameter

None

Returns

The method returns the attribute value.

Example

```
# Get attribute value
my $value = $object->cisco_ise_endpoint_profile();
```

cisco_ise_security_group( )

Use this method to retrieve the Cisco ISE security group. This is a read-only attribute.

Parameter

None

Returns

The method returns the attribute value.

Example

```
# Get attribute value
my $value = $object->cisco_ise_security_group();
```

cisco_ise_session_state( )

Use this method to retrieve the Cisco ISE session state. This is a read-only attribute.

Parameter

None

Returns

The method returns the attribute value.

Example

```
# Get attribute value
my $value = $object->cisco_ise_session_state();
```

cisco_ise_ssid( )

Use this method to retrieve the Cisco ISE SSID. This is a read-only attribute.

Parameter

None

Returns

The method returns the attribute value.

Example

```
# Get attribute value
my $value = $object->cisco_ise_ssid();
```

cmp_type( )

Use this method to retrieve the Cloud Management Platform type. This is a read-only attribute.

Parameter

None

Returns

The method returns the attribute value.

Example

```
# Get attribute value
my $value = $object->cmp_type();
```

vmi_ip_type( )

Use this method to retrieve the discovered IP address type. This is a read-only attribute.

Parameter

None

Returns

The method returns the attribute value.

Example

```
# Get attribute value
my $value = $object->vmi_ip_type();
```

vmi_private_address( )

Use this method to retrieve the private IP address of the virtual machine. This is a read-only attribute.

Parameter

None

Returns

The method returns the attribute value.

Example

```
# Get attribute value
my $value = $object->vmi_private_address();
```

vmi_is_public_address( )

Use this method to retrieve the flag used to indicate whether the IP address is a public address or not. This is a read-only attribute.

Parameter

None

Returns

The method returns the attribute value.

Example

```
# Get attribute value
my $value = $object->vmi_is_public_address();
```

SAMPLE CODE

The following sample code demonstrates the different functions that can be applied to a discovered data object.

#Preparation

```
use strict;
use Infoblox;

#Create a session to the Infoblox appliance
my $host_ip   = "192.168.1.2";
my $host_name = "infoblox.localdomain";
my $session = Infoblox::Session->new(
    master   => $host_ip,       #appliance host ip
    username => "admin",        #appliance user login
    password => "infoblox"      #appliance password
);
unless ($session) {
   die("Construct session failed: ".
        Infoblox::status_code() . ":" . Infoblox::status_detail());
}
print "Session created successfully.\n";
```

#Get an Infoblox::Grid::Discovery::Data object

```
my @retrieved_objs = $session->get('object' => 'Infoblox::DHCP::FixedAddr', 'ipv4addr' => '10.0.0.5');
my $fa = $retrieved_objs[0];
unless ($fa) {
   die("Get FixedAddr object failed: ".
       $session->status_code() . ":" . $session->status_detail());
}

my $data = $fa->discovered_data();
unless ($data) {
   die("FixedAddr object doesn't contain the discovered data");
}
```

#Search by discovered data

```
$data = Infoblox::Grid::Discovery::Data->new(
   'netbios' => 'Fixed*',
   'v_type'  => 'VirtualCenter',
);
unless ($data) {
   die("Construct Infoblox::Grid::Discovery::Data object failed: ".
        Infoblox::status_code() . ":" . Infoblox::status_detail());
}

@retrieved_objs = $session->get('object' => 'Infoblox::DHCP::FixedAddr', 'discovered_data' => $data);
$fa = $retrieved_objs[0];
unless ($fa) {
   die("Get FixedAddr object failed: ".
       $session->status_code() . ":" . $session->status_detail());
}

####PROGRAM ENDS####
```

AUTHOR

Infoblox Inc. http://www.infoblox.com/

COPYRIGHT

Copyright (c) 2017 Infoblox Inc.

Infoblox::Grid::Discovery::Device - the discovery device.

- NAME
- DESCRIPTION
- SESSION METHODS
  - Infoblox::Session->get( )
  - Infoblox::Session->modify( )
  - Infoblox::Session->search( )
- METHODS
  - address( )
  - address_object( )
  - cap_admin_status_ind( )
  - cap_admin_status_na_reason( )
  - cap_description_ind( )
  - cap_description_na_reason( )
  - cap_net_deprovisioning_ind( )
  - cap_net_deprovisioning_na_reason( )
  - cap_net_provisioning_ind( )
  - cap_net_provisioning_na_reason( )
  - cap_net_vlan_provisioning_ind( )
  - cap_net_vlan_provisioning_na_reason( )
  - cap_vlan_assignment_ind( )
  - cap_vlan_assignment_na_reason( )
  - cap_voice_vlan_ind( )
  - cap_voice_vlan_na_reason( )
  - chassis_serial_number( )
  - class( )
  - description( )
  - discover_now_status( )
  - extattrs( )
  - interfaces( )
  - location( )
  - model( )
  - ms_ad_user_data( )
  - name( )
  - neighbors( )
  - network( )
  - network_view( )
  - networks( )
  - os_version( )
  - port_stats( )
  - vendor( )
  - vlan_infos( )
- AUTHOR
- SEE ALSO
- COPYRIGHT

NAME

Infoblox::Grid::Discovery::Device - the discovery device.

DESCRIPTION

The object provides information about the discovery device.

SESSION METHODS

This section describes all the methods in Infoblox::Session that you can apply to the object.

Infoblox::Session->get( )

Use this method to retrieve all the matching objects from the Infoblox appliance. See Infoblox::Session->get() for parameters and return values.

Note that you should use 'return_methods' attribute to retrieve 'networks', 'interfaces', 'cap_admin_status_ind', 'cap_admin_status_na_reason', 'cap_description_ind', 'cap_description_na_reason', 'cap_net_deprovisioning_ind', 'cap_net_deprovisioning_na_reason', 'cap_net_provisioning_ind', 'cap_net_provisioning_na_reason', 'cap_net_vlan_provisioning_ind', 'cap_net_vlan_provisioning_na_reason', 'cap_vlan_assignment_ind', 'cap_vlan_assignment_na_reason', 'cap_voice_vlan_ind', 'cap_voice_vlan_na_reason' and 'neighbors' members of the device.

Key References

```
Apply the following attributes to get a specific object:

 address     - Optional. The IP address of the device.
 chassis_serial_number - Optional. The device chassis serial number.
 name        - Optional. The device name.
 model       - Optional. The device model.
 class       - Optional. The device class.
 location    - Optional. The device location.
 description - Optional. The device description.
 discovery_member - Optional. The name of the discovery member.
```

Example

```
# without networks', 'interfaces' and 'neighbors' members
my @retrieved_objs = $session->get(
    'object' => 'Infoblox::Grid::Discovery::Device',
    'name'   => 'device',
);

# with networks', 'interfaces' and 'neighbors' members
my @retrieved_objs = $session->get(
    'object' => 'Infoblox::Grid::Discovery::Device',
    'return_methods' => ['+networks', '+interfaces', '+neighbors'],
    'name'   => 'device',
);
```

Infoblox::Session->modify( )

Use this method to modify an object in the Infoblox appliance. See Infoblox::Session->modify() for parameters and return values.

Example

```
# Use this method to modify extensible attributes
$object->extattrs({ 'Site' => $extattr1, 'Administrator' => $extattr2 });
# Submit modification
my $response = $session->modify( $object );
```

Infoblox::Session->search( )

Use this method to search for objects in the Infoblox appliance. See Infoblox::Session->search() for parameters and return values.

Note that you should use 'return_methods' attribute to retrieve 'networks', 'interfaces', 'cap_admin_status_ind', 'cap_admin_status_na_reason', 'cap_description_ind', 'cap_description_na_reason', 'cap_net_deprovisioning_ind', 'cap_net_deprovisioning_na_reason', 'cap_net_provisioning_ind', 'cap_net_provisioning_na_reason', 'cap_net_vlan_provisioning_ind', 'cap_net_vlan_provisioning_na_reason', 'cap_vlan_assignment_ind', 'cap_vlan_assignment_na_reason', 'cap_voice_vlan_ind', 'cap_voice_vlan_na_reason' and 'neighbors' members of the device.

Key References

```
Apply the following attributes to search for a object:

 address     - Optional. The IP address of the device (regular expression).
 chassis_serial_number - Optional. The device chassis serial number (regular expression).
 name        - Optional. The device name (regular expression).
 model       - Optional. The device model (regular expression).
 class       - Optional. The device class (regular expression).
 location    - Optional. The device location (regular expression).
 description - Optional. The device description (regular expression).
 discovery_member - Optional. The host name of the discovery member.
```

Example

```
# without networks', 'interfaces' and 'neighbors' members
my @retrieved_objs = $session->search(
    'object' => 'Infoblox::Grid::Discovery::Device',
    'name'   => 'device',
);

# with networks', 'interfaces' and 'neighbors' members
my @retrieved_objs = $session->session(
    'object' => 'Infoblox::Grid::Discovery::Device',
    'return_methods' => ['+networks', '+interfaces', '+neighbors'],
    'name'   => 'device',
);
```

METHODS

This section describes all the methods that you can use to set or retrieve the attribute values of the object.

address( )

Use this method to retrieve the IP address of the device. This is a read-only attribute.

Parameter

None

Returns

The method returns the attribute value.

Example

```
# Get attribute value
my $value = $object->address();
```

address_object( )

Use this method to retrieve the IP address object of the device. This is a read-only attribute.

Parameter

None

Returns

The method returns the attribute value that can be an Infoblox::IPAM::Address object.

Example

```
# Get attribute value
my $value = $object->address_object();
```

cap_admin_status_ind( )

Use this method to retrieve the flag indicates whether or not modify the admin status of an interface in this device. This is a read-only attribute.

Parameter

None

Returns

The method returns the attribute value. Note that this method has to be explicitly requested via Infoblox::Session's return_methods member.

Example

```
# Get attribute value
my $value = $object->cap_admin_status_ind();
```

cap_admin_status_na_reason( )

Use this method to retrieve the reason if the edit admin status action is not available. This is a read-only attribute.

Parameter

None

Returns

The method returns the attribute value. Note that this method has to be explicitly requested via Infoblox::Session's return_methods member.

Example

```
# Get attribute value
my $value = $object->cap_admin_status_na_reason();
```

cap_description_ind( )

Use this method to retrieve the flag indicates whether or not modify the description of an interface in this device. This is a read-only attribute.

Parameter

None

Returns

The method returns the attribute value. Note that this method has to be explicitly requested via Infoblox::Session's return_methods member.

Example

```
# Get attribute value
my $value = $object->cap_description_ind();
```

cap_description_na_reason( )

Use this method to retrieve the reason if the description edit action is not available. This is a read-only attribute.

Parameter

None

Returns

The method returns the attribute value. Note that this method has to be explicitly requested via Infoblox::Session's return_methods member.

Example

```
# Get attribute value
my $value = $object->cap_description_na_reason();
```

cap_net_deprovisioning_ind( )

Use this method to retrieve the flag indicates whether or not deprovision a network from interfaces of this device. This is a read-only attribute.

Parameter

None

Returns

The method returns the attribute value. Note that this method has to be explicitly requested via Infoblox::Session's return_methods member.

Example

```
# Get attribute value
my $value = $object->cap_net_deprovisioning_ind();
```

cap_net_deprovisioning_na_reason( )

Use this method to retrieve the reason if the deprovision a network from interfaces of this device is not available. This is a read-only attribute.

Parameter

None

Returns

The method returns the attribute value. Note that this method has to be explicitly requested via Infoblox::Session's return_methods member.

Example

```
# Get attribute value
my $value = $object->cap_net_deprovisioning_na_reason();
```

cap_net_provisioning_ind( )

Use this method to retrieve the flag indicates whether or not modify the network associated to an interface of this device. This is a read-only attribute.

Parameter

None

Returns

The method returns the attribute value. Note that this method has to be explicitly requested via Infoblox::Session's return_methods member.

Example

```
# Get attribute value
my $value = $object->cap_net_provisioning_ind();
```

cap_net_provisioning_na_reason( )

Use this method to retrieve the reason if the network provisioning is not available. This is a read-only attribute.

Parameter

None

Returns

The method returns the attribute value. Note that this method has to be explicitly requested via Infoblox::Session's return_methods member.

Example

```
# Get attribute value
my $value = $object->cap_net_provisioning_na_reason();
```

cap_net_vlan_provisioning_ind( )

Use this method to retrieve the flag indicates whether or not create a VLAN and then provision a network in to this interface. This is a read-only attribute.

Parameter

None

Returns

The method returns the attribute value. Note that this method has to be explicitly requested via Infoblox::Session's return_methods member.

Example

```
# Get attribute value
my $value = $object->cap_net_vlan_provisioning_ind();
```

cap_net_vlan_provisioning_na_reason( )

Use this method to retrieve the reason if the network provisioning on VLAN is not available. This is a read-only attribute.

Parameter

None

Returns

The method returns the attribute value. Note that this method has to be explicitly requested via Infoblox::Session's return_methods member.

Example

```
# Get attribute value
my $value = $object->cap_net_vlan_provisioning_na_reason();
```

cap_vlan_assignment_ind( )

Use this method to retrieve the flag indicates whether or not modify the VLAN assignement of an interface in this device. This is a read-only attribute.

Parameter

None

Returns

The method returns the attribute value. Note that this method has to be explicitly requested via Infoblox::Session's return_methods member.

Example

```
# Get attribute value
my $value = $object->cap_vlan_assignment_ind();
```

cap_vlan_assignment_na_reason( )

Use this method to retrieve the reason if the vlan assignment action is not available. This is a read-only attribute.

Parameter

None

Returns

The method returns the attribute value. Note that this method has to be explicitly requested via Infoblox::Session's return_methods member.

Example

```
# Get attribute value
my $value = $object->cap_vlan_assignment_na_reason();
```

cap_voice_vlan_ind( )

Use this method to retrieve the flag indicates whether or not modify the voice vlan assignement of an interface in this device. This is a read-only attribute.

Parameter

None

Returns

The method returns the attribute value. Note that this method has to be explicitly requested via Infoblox::Session's return_methods member.

Example

```
# Get attribute value
my $value = $object->cap_voice_vlan_ind();
```

cap_voice_vlan_na_reason( )

Use this method to retrieve the reason if the voice vlan assignment action is not available. This is a read-only attribute.

Parameter

None

Returns

The method returns the attribute value. Note that this method has to be explicitly requested via Infoblox::Session's return_methods member.

Example

```
# Get attribute value
my $value = $object->cap_voice_vlan_na_reason();
```

chassis_serial_number( )

Use this method to retrieve the device chassis serial number. This is a read-only attribute.

Parameter

None

Returns

The method returns the attribute value.

Example

```
# Get attribute value
my $value = $object->chassis_serial_number();
```

class( )

Use this method to retrieve the device class. This is a read-only attribute.

Parameter

None

Returns

The method returns the attribute value.

Example

```
# Get attribute value
my $value = $object->class();
```

description( )

Use this method to retrieve the device description. This is a read-only attribute.

Parameter

None

Returns

The method returns the attribute value.

Example

```
# Get attribute value
my $value = $object->description();
```

discover_now_status( )

Use this method to retrieve the current discovery status. This attribute is read-only.

Parameter

None.

Returns

The method returns the attribute value that can be 'COMPLETE', 'FAILED', 'NONE', 'PENDING' or 'RUNNING'.

Example

```
#Get discover_now_status
my $discover_now_status = $object->discover_now_status();
```

extattrs( )

Use this method to set or retrieve the extensible attributes associated with the Device object.

Parameter

Valid value is a hash reference containing the names of extensible attributes and their associated values (Infoblox::Grid::Extattr objects).

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get extattrs
my $ref_extattrs = $object->extattrs();
#Modify extattrs
$object->extattrs({ 'Site' => $extattr1, 'Administrator' => $extattr2 });
```

interfaces( )

Use this method to retrieve a list of the device interfaces. This is a read-only attribute.

Parameter

None

Returns

The method returns the attribute value that can be an array reference that contains Infoblox::Grid::Discovery::DeviceInterface objects. Note that this method has to be explicitly requested via Infoblox::Session's return_methods member.

Example

```
# Get attribute value
my $value = $object->interfaces();
```

location( )

Use this method to retrieve the device location. This is a read-only attribute.

Parameter

None

Returns

The method returns the attribute value.

Example

```
# Get attribute value
my $value = $object->location();
```

model( )

Use this method to retrieve the device model name. This is a read-only attribute.

Parameter

None

Returns

The method returns the attribute value.

Example

```
# Get attribute value
my $value = $object->model();
```

ms_ad_user_data( )

Use this method to retrieve Microsoft Active Directory users related information. This is a read-only attribute.

Parameter

None

Returns

The valid return value is an Infoblox::Grid::MSServer::AdUser::Data object.

Example

```
#Get ms_ad_user_data
my $ms_ad_user_data = $object->ms_ad_user_data();
```

name( )

Use this method to retrieve the device name. This is a read-only attribute.

Parameter

None

Returns

The method returns the attribute value.

Example

```
# Get attribute value
```

```
my $value = $object->name();
```

neighbors( )

Use this method to retrieve a list of the device neighbors. This is a read-only attribute.

Parameter

None

Returns

The method returns the attribute value that can be an array reference that contains Infoblox::Grid::Discovery::DeviceNeighbor objects. Note that this method has to be explicitly requested via Infoblox::Session's return_methods member.

Example

```
# Get attribute value
my $value = $object->neighbors();
```

network( )

Use this method to retrieve the network object. This is a read-only attribute.

Parameter

None

Returns

The method returns the attribute value that can be an Infoblox::DHCP::Network or Infoblox::DHCP::IPv6Network object.

Example

```
# Get attribute value
my $value = $object->network();
```

network_view( )

Use this method to retrieve the network view name. This is a read-only attribute.

Parameter

None

Returns

The method returns the attribute value.

Example

```
# Get attribute value
my $value = $object->network_view();
```

networks( )

Use this method to retrieve a list of network objects. This is a read-only attribute.

Parameter

None

Returns

The method returns the attribute value that can be an array reference that contains Infoblox::DHCP::Network and Infoblox::DHCP::IPv6Network objects. Note that this method has to be explicitly requested via Infoblox::Session's return_methods member.

Example

```
# Get attribute value
my $value = $object->networks();
```

os_version( )

Use this method to retrieve the Operating System version running on the device. This is a read-only attribute.

Parameter

None

Returns

The method returns the attribute value.

Example

```
# Get attribute value
my $value = $object->os_version();
```

port_stats( )

Use this method to retrieve a list of device port statistics. This is a read-only attribute.

Parameter

None

Returns

The method returns the attribute value that can be an arrary reference that contains Infoblox::Grid::Discovery::Device::PortStatistics objects.

Example

```
# Get attribute value
my $value = $object->port_stats();
```

vendor( )

Use this method to retrieve the device vendor name. This is a read-only attribute.

Parameter

None

Returns

The method returns the attribute value.

Example

```
# Get attribute value
my $value = $object->vendor();
```

vlan_infos( )

Use this method to retrieve a list of VLAN information. This is a read-only attribute.

Parameter

None

Returns

The method returns the attribute value that can be an arrary reference that contains Infoblox::Grid::Discovery::VLANInfo objects.

Example

```
# Get attribute value
my $value = $object->vlan_infos();
```

AUTHOR

Infoblox Inc. http://www.infoblox.com/

SEE ALSO

Infoblox::Session, Infoblox::DHCP::Network, Infoblox::DHCP::IPv6Network, Infoblox::Grid::Discovery::DeviceNeighbor, Infoblox::Grid::Discovery::DeviceInterface, Infoblox::IPAM::Address, Infoblox::Grid::MSServer::AdUser::Data

---

COPYRIGHT

Infoblox::Grid::Discovery::Device - the discovery device.

Copyright (c) 2017 Infoblox Inc.

Infoblox::Grid::Discovery::Device::PortStatistics - the device port statistics object.

- NAME
- DESCRIPTION
- SESSION METHODS
- METHODS
  - admin_down_oper_down_count( )
  - admin_up_oper_down_count( )
  - admin_up_oper_up_count( )
  - interfaces_count( )
- AUTHOR
- SEE ALSO
- COPYRIGHT

NAME

Infoblox::Grid::Discovery::Device::PortStatistics - the device port statistics object.

DESCRIPTION

The object provides information about device port statistics.

SESSION METHODS

The object does not support any session methods.

METHODS

admin_down_oper_down_count( )

Use this method to retrieve the total number of interfaces which have administrative state "DOWN" and operating state "DOWN". This is a read-only attribute.

Parameter

None

Returns

The method returns the attribute value.

Example

```
# Get attribute value
my $value = $stats->admin_down_oper_down_count();
```

admin_up_oper_down_count( )

Use this method to retrieve the total number of interfaces which have administrative state "UP" and operating state "DOWN". This is a read-only attribute.

Parameter

None

Returns

The method returns the attribute value.

Example

```
# Get attribute value
my $value = $stats->admin_up_oper_down_count();
```

admin_up_oper_up_count( )

Use this method to retrieve the total number of interfaces which have administrative state "UP" and operating state "UP". This is a read-only attribute.

Parameter

v

None

Returns

The method returns the attribute value.

Example

```
# Get attribute value
my $value = $stats->admin_up_oper_up_count();
```

interfaces_count( )

Use this method to retrieve the total number of interfaces available on device. This is a read-only attribute.

Parameter

None

Returns

The method returns the attribute value.

Example

```
# Get attribute value
my $value = $stats->interfaces_count();
```

AUTHOR

Infoblox Inc. http://www.infoblox.com/

SEE ALSO

Infoblox::Grid::Discovery::Device

COPYRIGHT

Copyright (c) 2017 Infoblox Inc.

NAME

- NAME
- DESCRIPTION
- SESSION METHODS
  - Infoblox::Session->get( )
  - Infoblox::Session->search( )
- METHODS
  - class( )
  - component_name( )
  - description( )
  - model( )
  - device( )
  - serial( )
- AUTHOR
- SEE ALSO
- COPYRIGHT

NAME

Infoblox::Grid::Discovery::DeviceComponent

DESCRIPTION

The object provides information about the device component.

SESSION METHODS

Infoblox::Session->get( )

Use this method to retrieve all the matching objects from the Infoblox appliance. See Infoblox::Session->get() for parameters and return values.

Key References

```
Apply the following attributes to get a specific object:

   device      - Optional. The device object.
   description - Optional. The component description.
```

Example

```
my @retrieved_objs = $session->get(
    'object'      => 'Infoblox::Grid::Discovery::DeviceComponent',
    'description' => 'simple',
);
```

Infoblox::Session->search( )

Use this method to search for objects in the Infoblox appliance. See Infoblox::Session->search() for parameters and return values.

Key References

```
Apply the following attributes to search for an object:

   device      - Optional. The device object.
   description - Optional. The component description (regular expression).
```

Example

```
my @retrieved_objs = $session->search(
    'object'      => 'Infoblox::Grid::Discovery::DeviceComponent',
    'description' => '^simple',
);
```

METHODS

This section describes all the methods that you can use to set or retrieve the attribute values of the object.

class( )

Use this method to retrieve the general hardware type of the componenet. This attribute is read-only.

Parameter

None.

Returns

The method returns the attribute value.

Example

```
# Get attribute value
my $value = $object->class();
```

component_name( )

Use this method to retrieve the component name. This attribute is read-only.

Parameter

None.

Returns

The method returns the attribute value.

Example

```
# Get attribute value
my $value = $object->component_name();
```

description( )

Use this method to retrieve the component description. This attribute is read-only.

Parameter

None.

Returns

The method returns the attribute value.

Example

```
# Get attribute value
my $value = $object->description();
```

model( )

Use this method to retrieve the model name. This attribute is read-only.

Parameter

None.

Returns

The method returns the attribute value.

Example

```
# Get attribute value
my $value = $object->model();
```

device( )

Use this method to retrieve the device object. This attribute is read-only.

Parameter

None.

Returns

The method returns an Infoblox::Grid::Discovery::Device object.

Example

```
# Get attribute value
my $value = $object->device();
```

serial( )

Use this method to retrieve the component serial number. This attribute is read-only.

Parameter

None.

Returns

The method returns the attribute value.

Example

```
# Get attribute value
my $value = $object->serial();
```

AUTHOR

Infoblox Inc. http://www.infoblox.com/

SEE ALSO

Infoblox::Session, Infoblox::Grid::Discovery::Device

COPYRIGHT

Copyright (c) 2017 Infoblox Inc.

Infoblox::Grid::Discovery::DeviceDataCollectionStatus - the device data collection status.

- NAME
- DESCRIPTION
- SESSION METHODS
- MODULE METHODS
  - Infoblox::Grid::Discovery::DeviceSupportInfoResponse->device_data_collection_status( )
- METHODS
  - data_source()
  - end_time()
- AUTHOR
- SEE ALSO
- COPYRIGHT

NAME

Infoblox::Grid::Discovery::DeviceDataCollectionStatus - the device data collection status.

DESCRIPTION

The object provides information about the device data collection status. This is a read-only object.

SESSION METHODS

The object does not support any session methods.

MODULE METHODS

This section describes all the functions that you can apply to a Device Data Collection Status object.

Infoblox::Grid::Discovery::DeviceSupportInfoResponse->device_data_collection_status( )

Use this method to retrieve the Device data collection statuses. See Infoblox::Grid::Discovery::DeviceSupportInfoResponse->device_data_collection_status() for parameters and return values.

Example

```
#Get value
my $device_data_collection_status = $device_support_info_response->device_data_collection_status();
```

METHODS

This section describes all the methods that you can use to retrieve the attribute values of a device data collection status.

data_source()

Use this method to retrieve the source from which device support information is collected. This is a read-only attribute.

Parameter

None

Returns

The method returns the attribute value.

Example

```
#Get value
my $data_source = $device_data_collection_status->data_source();
```

end_time()

Use this method to retrieve the time when the most recent collection from the data sources was completed. This is a read-only attribute.

Parameter

None

Returns

The method returns the attribute value. The number of seconds that have elapsed since January 1st, 1970 UTC.

Example

```
#Get value
my $end_time = $device_data_collection_status->end_time();
```

AUTHOR

Infoblox Inc. http://www.infoblox.com/

SEE ALSO

Infoblox::Grid::Discovery::DeviceSupportInfoResponse, Infoblox::Grid::Discovery::DeviceSupportInfo.

COPYRIGHT

Copyright (c) 2017 Infoblox Inc.

Infoblox::Grid::Discovery::DeviceInterface - the discovery interface.

- NAME
- DESCRIPTION
- SESSION METHODS
  - Infoblox::Session->modify( )
- METHODS
  - admin_status( )
  - admin_status_task_info( )
  - cap_if_admin_status_ind( )
  - cap_if_admin_status_na_reason( )
  - cap_if_description_ind( )
  - cap_if_description_na_reason( )
  - cap_if_net_deprovisioning_ipv4_ind( )
  - cap_if_net_deprovisioning_ipv6_ind( )
  - cap_if_net_deprovisioning_ipv4_na_reason( )
  - cap_if_net_deprovisioning_ipv6_na_reason( )
  - cap_if_net_provisioning_ipv4_ind( )
  - cap_if_net_provisioning_ipv6_ind( )
  - cap_if_net_provisioning_ipv4_na_reason( )
  - cap_if_net_provisioning_ipv6_na_reason( )
  - cap_if_vlan_assignment_ind( )
  - cap_if_vlan_assignment_na_reason( )
  - cap_if_voice_vlan_ind( )
  - cap_if_voice_vlan_na_reason( )
  - description( )
  - description_task_info( )
  - device( )
  - duplex( )
  - discover_now_status( )
  - extattrs( )
  - ifaddr_infos( )
  - index( )
  - last_change( )
  - link_aggregation( )
  - mac( )
  - ms_ad_user_data( )
  - name( )
  - oper_status( )
  - port_fast( )
  - reserved_object( )
  - speed( )
  - trunk_status( )
  - type( )
  - vlan_infos( )
  - vlan_info_task_info( )
  - network_view( )
  - vrf_name( )
  - vrf_description( )
  - vrf_route_distinguisher( )
- AUTHOR
- SEE ALSO
- COPYRIGHT

NAME

Infoblox::Grid::Discovery::DeviceInterface - the discovery interface.

DESCRIPTION

The object provides information about the discovery interface.

SESSION METHODS

This section describes all the methods in Infoblox::Session that you can apply to the object.

Infoblox::Session->modify( )

Use this method to modify an object in the Infoblox appliance. See Infoblox::Session->modify() for parameters and return values.

Example

```
# Use this method to modify extensible attributes
$object->extattrs({ 'Site' => $extattr1, 'Administrator' => $extattr2 });
# Submit modification
my $response = $session->modify( $object );
```

METHODS

This section describes all the methods that you can use to set or retrieve the attribute values of the object.

admin_status( )

Use this method to retrieve the administrative state of the interface. This is a read-only attribute.

Parameter

None

Returns

The method returns the attribute value that can be 'UP' or 'DOWN'.

Example

```
# Get attribute value
my $value = $object->admin_status();
```

admin_status_task_info( )

Use this method to retrieve the configured admin status task information.

Parameter

None

Returns

The method returns the attribute value that can be an Infoblox::Grid::Discovery::Port::Config::AdminStatus object.

Example

```
# Get attribute value
my $value = $object->admin_status_task_info();
```

cap_if_admin_status_ind( )

Use this method to retrieve the flag indicates whether or not modify the admin status of this interface. This is a read-only attribute.

Parameter

None

Returns

The method returns the attribute value.

Example

```
# Get attribute value
my $value = $object->cap_if_admin_status_ind();
```

cap_if_admin_status_na_reason( )

Use this method to retrieve the reason if the edit admin status action is not available. This is a read-only attribute.

Parameter

None

Returns

The method returns the attribute value.

Example

```
# Get attribute value
```

```
my $value = $object->cap_if_admin_status_na_reason();
```

cap_if_description_ind( )

Use this method to retrieve the flag indicates whether or not modify the description of this interface. This is a read-only attribute.

Parameter

None

Returns

The method returns the attribute value.

Example

```
# Get attribute value
my $value = $object->cap_if_description_ind();
```

cap_if_description_na_reason( )

Use this method to retrieve the reason if the edit description action is not available. This is a read-only attribute.

Parameter

None

Returns

The method returns the attribute value.

Example

```
# Get attribute value
my $value = $object->cap_if_description_na_reason();
```

cap_if_net_deprovisioning_ipv4_ind( )

Use this method to retrieve the flag indicates whether or not deprovision the IPv4 network associated to this interface. This is a read-only attribute.

Parameter

None

Returns

The method returns the attribute value.

Example

```
# Get attribute value
my $value = $object->cap_if_net_deprovisioning_ipv4_ind();
```

cap_if_net_deprovisioning_ipv6_ind( )

Use this method to retrieve the flag indicates whether or not deprovision the IPv6 network associated to this interface. This is a read-only attribute.

Parameter

None

Returns

The method returns the attribute value.

Example

```
# Get attribute value
my $value = $object->cap_if_net_deprovisioning_ipv6_ind();
```

cap_if_net_deprovisioning_ipv4_na_reason( )

Use this method to retrieve the reason if the deprovision the IPv4 network from this interface is not available. This is a read-only attribute.

Parameter

None

Returns

The method returns the attribute value.

Example

```
# Get attribute value
my $value = $object->cap_if_net_deprovisioning_ipv4_na_reason();
```

cap_if_net_deprovisioning_ipv6_na_reason( )

Use this method to retrieve the reason if the deprovision the IPv6 network from this interface is not available. This is a read-only attribute.

Parameter

None

Returns

The method returns the attribute value.

Example

```
# Get attribute value
my $value = $object->cap_if_net_deprovisioning_ipv6_na_reason();
```

cap_if_net_provisioning_ipv4_ind( )

Use this method to retrieve the flag indicates whether or not modify the IPv4 network associated to this interface. This is a read-only attribute.

Parameter

None

Returns

The method returns the attribute value.

Example

```
# Get attribute value
my $value = $object->cap_if_net_provisioning_ipv4_ind();
```

cap_if_net_provisioning_ipv6_ind( )

Use this method to retrieve the flag indicates whether or not modify the IPv6 network associated to this interface. This is a read-only attribute.

Parameter

None

Returns

The method returns the attribute value.

Example

```
# Get attribute value
my $value = $object->cap_if_net_provisioning_ipv6_ind();
```

cap_if_net_provisioning_ipv4_na_reason( )

Use this method to retrieve the reason if the IPv4 network provisioning is not available. This is a read-only attribute.

Parameter

None

Returns

The method returns the attribute value.

Example

```
# Get attribute value
my $value = $object->cap_if_net_provisioning_ipv4_na_reason();
```

cap_if_net_provisioning_ipv6_na_reason( )

Use this method to retrieve the reason if the IPv6 network provisioning is not available. This is a read-only attribute.

Parameter

None

Returns

The method returns the attribute value.

Example

```
# Get attribute value
my $value = $object->cap_if_net_provisioning_ipv6_na_reason();
```

cap_if_vlan_assignment_ind( )

Use this method to retrieve the flag indicates whether or not modify the VLAN assignement for this interface. This is a read-only attribute.

Parameter

None

Returns

The method returns the attribute value.

Example

```
# Get attribute value
my $value = $object->cap_if_vlan_assignment_ind();
```

cap_if_vlan_assignment_na_reason( )

Use this method to retrieve the reason if the VLAN assignment action is not available. This is a read-only attribute.

Parameter

None

Returns

The method returns the attribute value.

Example

```
# Get attribute value
my $value = $object->cap_if_vlan_assignment_na_reason();
```

cap_if_voice_vlan_ind( )

Use this method to retrieve the flag indicates whether or not modify the voice VLAN assignement for this interface. This is a read-only attribute.

Parameter

None

Returns

The method returns the attribute value.

Example

```
# Get attribute value
my $value = $object->cap_if_vlan_assignment_ind();
```

cap_if_voice_vlan_na_reason( )

Use this method to retrieve the reason if the voice VLAN assignment action is not available. This is a read-only attribute.

Parameter

None

Returns

The method returns the attribute value.

Example

```
# Get attribute value
my $value = $object->cap_if_voice_vlan_na_reason();
```

description( )

Use this method to retrieve the description of the interface. This is a read-only attribute.

Parameter

None

Returns

The method returns the attribute value.

Example

```
# Get attribute value
my $value = $object->description();
```

description_task_info( )

Use this method to retrieve the configured description task information.

Parameter

None

Returns

The method returns the attribute value that can be an Infoblox::Grid::Discovery::Port::Config::Description object.

Example

```
# Get attribute value
my $value = $object->description_task_info();
```

device( )

Use this method to retrieve the device object. This is a read-only attribute.

Parameter

None

Returns

The method returns the attribute value that can be an Infoblox::Grid::Discovery::Device object.

Example

```
# Get attribute value
my $value = $object->device();
```

duplex( )

Use this method to retrieve the duplex state of the interface. This is a read-only attribute.

Parameter

None

Returns

The method returns the attribute value that can be 'FULL', 'HALF', 'UNKNOWN' or 'UNSUPPORTED'.

Example

```
# Get attribute value
my $value = $object->duplex();
```

discover_now_status( )

Use this method to retrieve the current discovery status. This attribute is read-only. 'discover_now_status' will not be returned by default, use 'return_methods' to request it if needed.

Parameter

None.

Returns

The method returns the attribute value that can be 'COMPLETE', 'FAILED', 'NONE', 'PENDING' or 'RUNNING'.

Example

```
#Get discover_now_status
my $discover_now_status = $object->discover_now_status();
```

extattrs( )

Use this method to set or retrieve the extensible attributes associated with the DeviceInterface object.

Parameter

Valid value is a hash reference containing the names of extensible attributes and their associated values (Infoblox::Grid::Extattr objects).

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get extattrs
my $ref_extattrs = $object->extattrs();
#Modify extattrs
$object->extattrs({ 'Site' => $extattr1, 'Administrator' => $extattr2 });
```

ifaddr_infos( )

Use this method to retrieve a list of IFaddr information associated with the interface. This is a read-only attribute.

Parameter

None

Returns

The method returns the attribute value that can be an array reference that contains Infoblox::Grid::Discovery::IFAddrInfo objects.

Example

```
# Get attribute value
my $value = $object->ifaddr_infos();
```

index( )

Use this method to retrieve the interface index number, as reported by SNMP. This is a read-only attribute.

Parameter

None

Returns

The method returns the attribute value.

Example

```
# Get attribute value
my $value = $object->index();
```

last_change( )

Use this method to retrieve the timestamp of the last detected interface property change. This is a read-only attribute.

Parameter

None

Returns

The method returns the attribute value.

Example

```
# Get attribute value
my $value = $object->last_change();
```

link_aggregation( )

Use this method to retrieve the flag that indicates whether this is a link aggregation interface. This is a read-only attribute.

Parameter

None

Returns

The method returns the attribute value that can be 'true' or 'false'.

Example

```
# Get attribute value
my $value = $object->link_aggregation();
```

mac( )

Use this method to retrieve the MAC address of the interface. This is a read-only attribute.

Parameter

None

Returns

The method returns the attribute value.

Example

```
# Get attribute value
my $value = $object->mac();
```

ms_ad_user_data( )

Use this method to retrieve Microsoft Active Directory users related information. This is a read-only attribute.

Parameter

None

Returns

The valid return value is an Infoblox::Grid::MSServer::AdUser::Data object.

Example

```
#Get ms_ad_user_data
my $ms_ad_user_data = $object->ms_ad_user_data();
```

name( )

Use this method to retrieve the interface system name. This is a read-only attribute.

Parameter

None

Returns

The method returns the attribute value.

Example

```
# Get attribute value
my $value = $object->name();
```

oper_status( )

Use this method to retrieve the operating state of the interface. This is a read-only attribute.

Parameter

None

Returns

The method returns the attribute value that can be 'UP' or 'DOWN'.

Example

```
# Get attribute value
my $value = $object->oper_status();
```

port_fast( )

Use this method to retrieve the port fast status of the interface. This is a read-only attribute.

Parameter

None

Returns

The method returns the attribute value that can be 'ENABLED' or 'DISABLED'.

Example

```
# Get attribute value
my $value = $object->port_fast();
```

reserved_object( )

Use this method to retrieve the object to which this port is reserved. This is a read-only attribute.

Parameter

None

Returns

The method returns the attribute value that can be Infoblox::DHCP::HostAddr, Infoblox::DHCP::IPv6HostAddr, Infoblox::DHCP::FixedAddr, Infoblox::DHCP::IPv6FixedAddr or Infoblox::Grid::Member object.

Example

```
# Get attribute value
my $value = $object->reserved_object();
```

speed( )

Use this method to retrieve the interface speed in bps. This is a read-only attribute.

Parameter

None

Returns

The method returns the attribute value.

Example

```
# Get attribute value
my $value = $object->speed();
```

trunk_status( )

Use this method to retrieve the flag that indicates if the interface is tagged as trunk or not. This is a read-only attribute.

Parameter

None

Returns

The method returns the attribute value that can be 'ON' or 'OFF'.

Example

```
# Get attribute value
my $value = $object->trunk_status();
```

type( )

Use this method to retrieve the interface type. This is a read-only attribute.

Parameter

None

Returns

The method returns the attribute value.

Example

```
# Get attribute value
my $value = $object->type();
```

vlan_infos( )

Use this method to retrieve a list of VLAN Information associated with this interface. This is a read-only attribute.

Parameter

None

Returns

The method returns the attribute value that can be an array reference that contains Infoblox::Grid::Discovery::VLANInfo objects.

Example

```
# Get attribute value
my $value = $object->vlan_infos();
```

vlan_info_task_info( )

Use this method to retrieve the configured VLAN task information.

Parameter

None

Returns

The method returns the attribute value that can be an Infoblox::Grid::Discovery::Port::Config::VlanInfo object.

Example

```
# Get attribute value
my $value = $object->vlan_info_task_info();
```

network_view( )

Use this method to retrieve the name of the network view associated with the interface. This is a read-only attribute.

Parameter

None

Returns

The method returns the attribute value.

Example

```
# Get attribute value
my $value = $object->network_view();
```

vrf_name( )

Use this method to retrieve the name of the VRF associated with the interface, if any. This is a read-only attribute.

Parameter

None

Returns

The method returns the attribute value.

Example

```
# Get attribute value
my $value = $object->vrf_name();
```

vrf_description( )

Use this method to retrieve the VRF description. This is a read-only attribute.

Parameter

None

Returns

The method returns the attribute value.

Example

```
# Get attribute value
my $value = $object->vrf_description();
```

vrf_route_distinguisher( )

Use this method to retrieve the route distinguisher associated with the VRF. This is a read-only attribute.

Parameter

None

Returns

The method returns the attribute value.

Example

```
# Get attribute value
my $value = $object->vrf_route_distinguisher();
```

AUTHOR

Infoblox Inc. http://www.infoblox.com/

SEE ALSO

Infoblox::Grid::Discovery::Device, Infoblox::Grid::Discovery::VLANInfo, Infoblox::Grid::Discovery::IFAddrInfo

---

COPYRIGHT

Copyright (c) 2017 Infoblox Inc.

Infoblox::Grid::Discovery::DeviceNeighbor - the neighbor associated with the device.

- NAME
- DESCRIPTION
- SESSION METHODS
- METHODS
  - address( )
  - address_object( )
  - device( )
  - interface( )
  - mac( )
  - name( )
  - vlan_infos( )
- AUTHOR
- SEE ALSO
- COPYRIGHT

NAME

Infoblox::Grid::Discovery::DeviceNeighbor - the neighbor associated with the device.

DESCRIPTION

The object provides information about the neighbor associated with the device.

SESSION METHODS

The object does not support any session methods.

METHODS

This section describes all the methods that you can use to set or retrieve the attribute values of the object.

address( )

Use this method to retrieve the neighbor IP address. This is a read-only attribute.

Parameter

None

Returns

The method returns the attribute value.

Example

```
# Get attribute value
my $value = $object->address();
```

address_object( )

Use this method to retrieve the IP Address object. This is a read-only attribute.

Parameter

None

Returns

The method returns the attribute value that can be an Infoblox::IPAM::Address object.

Example

```
# Get attribute value
my $value = $object->address_object();
```

device( )

Use this method to retrieve the device object. This is a read-only attribute.

Parameter

None

Returns

The method returns the attribute value that can be an Infoblox::Grid::Discovery::Device object.

Example

```
# Get attribute value
my $value = $object->device();
```

interface( )

Use this method to retrieve the interface object. This is a read-only attribute.

Parameter

None

Returns

The method returns the attribute value that can be an Infoblox::Grid::Discovery::DeviceInterface object.

Example

```
# Get attribute value
my $value = $object->interface();
```

mac( )

Use this method to retrieve the MAC address of the neighbor. This is a read-only attribute.

Parameter

None

Returns

The method returns the attribute value.

Example

```
# Get attribute value
my $value = $object->mac();
```

name( )

Use this method to retrieve the neighbor name. This is a read-only attribute.

Parameter

None

Returns

The method returns the attribute value.

Example

```
# Get attribute value
my $value = $object->name();
```

vlan_infos( )

Use this method to retrieve a list of VLAN Information associated with this neighbor. This is a read-only attribute.

Parameter

None

Returns

The method returns the attribute value that can be an array reference that contains Infoblox::Grid::Discovery::VLANInfo objects.

Example

```
# Get attribute value
my $value = $object->vlan_infos();
```

AUTHOR

Infoblox Inc. http://www.infoblox.com/

SEE ALSO

Infoblox::IPAM::Address, Infoblox::Grid::Discovery::Device, Infoblox::Grid::Discovery::DeviceInterface

---

COPYRIGHT

Copyright (c) 2017 Infoblox Inc.

Infoblox::Grid::Discovery::DeviceSupportInfo - the device support info.

- NAME
- DESCRIPTION
- SESSION METHODS
- MODULE METHODS
  - Infoblox::Grid::Discovery::DeviceSupportInfoResponse->device_support_info( )
- METHODS
  - available()
  - function()
  - supported()
  - value()
- AUTHOR
- SEE ALSO
- COPYRIGHT

NAME

Infoblox::Grid::Discovery::DeviceSupportInfo - the device support info.

DESCRIPTION

The object provides information about the device support info. This is a read-only object.

SESSION METHODS

The object does not support any session methods.

MODULE METHODS

This section describes all the functions that you can apply to a Device Support Info object.

Infoblox::Grid::Discovery::DeviceSupportInfoResponse->device_support_info( )

Use this method to retrieve the Device support infos. See Infoblox::Grid::Discovery::DeviceSupportInfoResponse->device_support_info() for parameters and return values.

Example

```
#Get value
my $device_support_info = $device_support_info_response->device_support_info();
```

METHODS

This section describes all the methods that you can use to retrieve the attribute values of a device support info.

available()

Use this method to determine whether the function is available for the device or not. This is a read-only attribute.

Parameter

None

Returns

The method returns the attribute value.

Example

```
#Get value
my $available = $device_support_info->available();
```

function()

Use this method to retrieve the function of the device. This is a read-only attribute.

Parameter

None

Returns

The method returns the attribute value.

Example

```
#Get value
my $function = $device_support_info->function();
```

supported()

Use this method to determine whether the function is supported by the device or not. This is a read-only attribute.

Parameter

None

Returns

The method returns the attribute value.

Example

```
#Get value
my $supported = $device_support_info->supported();
```

value()

Use this method to retrieve the discovery member indicator knowing that a given device supports the given type of data collection. This is a read-only attribute.

Parameter

None

Returns

The method returns the attribute value.

Example

```
#Get value
my $value = $device_support_info->value();
```

AUTHOR

Infoblox Inc. http://www.infoblox.com/

SEE ALSO

Infoblox::Grid::Discovery::DeviceSupportInfoResponse, Infoblox::Grid::Discovery::DeviceDataCollectionStatus.

COPYRIGHT

Copyright (c) 2017 Infoblox Inc.

Infoblox::Grid::Discovery::DeviceSupportInfoResponse - the device support info response.

- NAME
- DESCRIPTION
- SESSION METHODS
- METHODS
  - device_data_collection_status()
  - device_support_info()
- AUTHOR
- SEE ALSO
- COPYRIGHT

NAME

Infoblox::Grid::Discovery::DeviceSupportInfoResponse - the device support info response.

DESCRIPTION

The object is created as a result of Infoblox::Grid::Discovery->get_device_support_info() function call and cannot be added or removed manually.

SESSION METHODS

The object does not support any session methods.

METHODS

This section describes all the methods that you can use to retrieve the attribute values of a device support info response.

device_data_collection_status()

Use this method to retrieve the list of Discovery device data collection statuses. This is a read-only attribute.

Parameter

None

Returns

The method returns the attribute value, that is an array of Infoblox::Grid::Discovery::DeviceDataCollectionStatus objects.

Example

```
#Get value
my $device_data_collection_status = $device_support_info_response->device_data_collection_status();
```

device_support_info()

Use this method to retrieve the list of Discovery device support infos. This is a read-only attribute.

Parameter

None

Returns

The method returns the attribute value, that is an array of Infoblox::Grid::Discovery::DeviceSupportInfo objects.

Example

```
#Get value
my $device_support_info = $device_support_info_response->device_support_info();
```

AUTHOR

Infoblox Inc. http://www.infoblox.com/

SEE ALSO

Infoblox::Grid::Discovery, Infoblox::Grid::Discovery::DeviceDataCollectionStatus, Infoblox::Grid::Discovery::DeviceSupportInfo.

Infoblox::Grid::Discovery::DeviceSupportInfoResponse - the device support info response.

COPYRIGHT

Copyright (c) 2017 Infoblox Inc.

Infoblox::Grid::Discovery::DiagnosticsStatus - The discovery diagnostic status.

- NAME
- DESCRIPTION
- SESSION METHODS
  - Infoblox::Session->discovery_diagnostics_status( )
- METHODS
  - end( )
  - id( )
  - start( )
  - status( )
  - text( )
- AUTHOR
- SEE ALSO
- COPYRIGHT

NAME

Infoblox::Grid::Discovery::DiagnosticsStatus - The discovery diagnostic status.

DESCRIPTION

The object provides information about the discovery diagnostic status.

SESSION METHODS

This section describes all the methods in Infoblox::Session that you can apply to the object.

Infoblox::Session->discovery_diagnostics_status( )

Use this method to retrieve the discovery diagnostic status.

Key References

```
Apply the following attributes to get a specific object:

  id     - Required. The discovery diagnostic status ID.
  start  - Optional. The start position of the text.
```

Returns

The method returns an Infoblox::Grid::Discovery::DiagnosticsStatus object when the operation succeeds, and returns false when the operation fails.

Example

```
my $id = $session->discovery_diagnostics('address' => '10.0.0.1');
my $object = $session->discovery_diagnostics_status('id' => $id);
```

METHODS

This section describes all the methods that you can use to set or retrieve the attribute values of the object.

end( )

Use this method to retrieve the end position of text. This is a read-only attribute.

Parameter

None

Returns

The method returns the attribute value.

Example

```
# Get attribute value
my $value = $object->end();
```

id( )

Use this method to retrieve the discovery diagnostic status ID. This is a read-only attribute.

Parameter

None

Returns

The method returns the attribute value.

Example

```
# Get attribute value
my $value = $object->id();
```

start( )

Use this method to retrieve the start position of the text. This is a read-only attribute.

Parameter

None

Returns

The method returns the attribute value.

Example

```
# Get attribute value
my $value = $object->start();
```

status( )

Use this method to retrieve the status. This is a read-only attribute.

Parameter

None

Returns

The method returns the attribute value that can be 'COMPLETED', 'FAILED' or 'INPROGRESS'.

Example

```
# Get attribute value
my $value = $object->status();
```

text( )

Use this method to retrieve the text. This is a read-only attribute.

Parameter

None

Returns

The method returns the attribute value.

Example

```
# Get attribute value
my $value = $object->text();
```

AUTHOR

Infoblox Inc. http://www.infoblox.com/

SEE ALSO

Infoblox::Session

Infoblox::Grid::Discovery::DiagnosticsStatus - The discovery diagnostic status.

COPYRIGHT

Copyright (c) 2017 Infoblox Inc.

Infoblox::Grid::Discovery::DiagnosticTask - The discovery diagnostic task.

- NAME
- DESCRIPTION
- SESSION METHODS
  - Infoblox::Session->get( )
  - Infoblox::Session->search( )
- METHODS
  - community_string( )
  - debug_snmp( )
  - force_test( )
  - ip_address( )
  - network_view( )
  - start_time( )
  - task_id( )
- AUTHOR
- SEE ALSO
- COPYRIGHT

NAME

Infoblox::Grid::Discovery::DiagnosticTask - The discovery diagnostic task.

DESCRIPTION

The object provides information about the discovery diagnostic task.

SESSION METHODS

This section describes all the methods in an Infoblox::Session module that you can apply to a discovery diagnostic task object.

Infoblox::Session->get( )

Use this method to retrieve all the matching objects from the Infoblox appliance. See Infoblox::Session->get() for parameters and return values.

Key References

```
Apply the following attributes to get a specific discovery diagnostic task object:

 address                    - Optional. The IP address of the discovery diagnostic task.
 network_view               - Optional. The network view of the discovery diagnostic task.
 task_id                    - Optional. The id of the discovery diagnostic task.
```

Examples

```
my @retrieved_objs = $session->get(
    object     => 'Infoblox::Grid::Discovery::DiagnosticTask',
    ip_address => '10.0.0.1',
);

my @retrieved_objs = $session->get(
    object  => 'Infoblox::Grid::Discovery::DiagnosticTask',
    task_id => 'task_1',
);
```

Infoblox::Session->search( )

Use this method to search for the existing objects from the Infoblox appliance. See Infoblox::Session->search() for parameters and return values.

Key References

```
Apply the following attributes to get a specific discovery diagnostic task object:

 address                    - Optional. The IP address of the discovery diagnostic task (regular
expression).
 network_view               - Optional. The network view name of the discovery diagnostic task.
 task_id                    - Optional. The id of the discovery diagnostic task.
```

Example

```
my @retrieved_objs = $session->search(
    object     => 'Infoblox::Grid::Discovery::DiagnosticTask',
    ip_address => '10.*',
);

my @retrieved_objs = $session->search(
    object  => 'Infoblox::Grid::Discovery::DiagnosticTask',
    task_id => 'task_1',
);
```

METHODS

This section describes all the methods that you can use to configure and retrieve the attribute values of a discovery diagnostic task.

community_string( )

Use this method to retrieve the SNMP community string. This is a read-only attribute.

Returns

The method returns the attribute value.

Example

```
#Get attribute value
my $community_string = $object->community_string();
```

debug_snmp( )

Use this method to retrieve the flag for the SNMP debug. This is a read-only attribute.

Returns

The method returns the attribute value.

Example

```
#Get attribute value
my $debug_snmp = $object->debug_snmp();
```

force_test( )

Use this method to retrieve the flag for the force test. This is a read-only attribute.

Returns

The method returns the attribute value.

Example

```
#Get attribute value
my $force_test = $object->force_test();
```

ip_address( )

Use this method to retrieve the discovery diagnostic IP address. This is a read-only attribute.

Returns

The method returns the attribute value.

Example

```
#Get attribute value
my $ip_address = $object->ip_address();
```

network_view( )

Use this method to retrieve the network view for public IPs. This is a read-only attribute.

Returns

The method returns the attribute value.

Example

```
#Get attribute value
my $network_view = $object->network_view();
```

start_time( )

Use this method to retrieve the timestamp when the task was started. This is a read-only attribute.

Returns

The method returns the attribute value.

Example

```
#Get attribute value
my $start_time = $object->start_time();
```

task_id( )

Use this method to retrieve the unique identifier for the discovery diagnostic task. This is a read-only attribute.

Returns

The method returns the attribute value.

Example

```
#Get attribute value
my $task_id = $object->task_id();
```

AUTHOR

Infoblox Inc. http://www.infoblox.com/

SEE ALSO

Infoblox::Session

COPYRIGHT

Copyright (c) 2017 Infoblox Inc.

Infoblox::Grid::Discovery::DiscoveryDataConversionResult - discovery data conversion result.

- NAME
- DESCRIPTION
- SESSION METHODS
- METHODS
  - object()
  - address()
  - status()
  - message()
- AUTHOR
- SEE ALSO
- COPYRIGHT

NAME

Infoblox::Grid::Discovery::DiscoveryDataConversionResult - discovery data conversion result.

DESCRIPTION

The object represents the result of the discovery data conversion. This is a read-only object.

SESSION METHODS

The object does not support any session methods.

METHODS

object()

Use this method to retrieve the reference to the converted object.

Parameter

None

Returns

The method returns the attribute value.

Example

```
#Get value
my $object = $conversion_result->object();
```

address()

Use this method to retrieve the reference to the Infoblox::IPAM::Address object that has been converted.

Parameter

None

Returns

The method returns the attribute value.

Example

```
#Get value
my $address = $conversion_result->address();
```

status()

Use this method to retrieve the status of the conversion operation.

Parameter

None

Returns

The method returns the attribute value.

Example

```
#Get value
my $status = $conversion_result->status();
```

message()

Use this method to retrieve the detailed status of the conversion operation. This could contain either the error or success message.

Parameter

None

Returns

The method returns the attribute value.

Example

```
#Get value
my $message = $conversion_result->message();
```

AUTHOR

Infoblox Inc. http://www.infoblox.com/

SEE ALSO

Infoblox::Grid::Discovery, Infoblox::IPAM::Address.

COPYRIGHT

Copyright (c) 2017 Infoblox Inc.

Infoblox::Grid::Discovery::DSBundle - the device support bundle.

- NAME
- DESCRIPTION
- SESSION METHODS
  - Infoblox::Session->get( )
  - Infoblox::Session->search( )
- METHODS
  - author( )
  - integrated_ind( )
  - name( )
  - version( )
- AUTHOR
- SEE ALSO
- COPYRIGHT

NAME

Infoblox::Grid::Discovery::DSBundle - the device support bundle.

DESCRIPTION

The object provides information about the device support bundle.

SESSION METHODS

This section describes all the methods in Infoblox::Session that you can apply to the object.

Infoblox::Session->get( )

Use this method to retrieve all the matching objects from the Infoblox appliance. See Infoblox::Session->get() for parameters and return values.

Key References

```
Apply the following attributes to get a specific object:

 name  - Optional. The name of the device support bundle.
```

Example

```
my @retrieved_objs = $session->get(
    'object' => 'Infoblox::Grid::Discovery::DSBundle',
    'name'   => 'dsb',
);
```

Infoblox::Session->search( )

Use this method to search for objects in the Infoblox appliance. See Infoblox::Session->search() for parameters and return values.

Key References

```
Apply the following attributes to search for an object:

 name  - Optional. The name of the device support bundle (regular expression).
```

Example

```
my @retrieved_objs = $session->search(
    'object' => 'Infoblox::Grid::Discovery::DSBundle',
    'name'   => 'dsb',
);
```

METHODS

This section describes all the methods that you can use to set or retrieve the attribute values of the object.

author( )

Use this method to retrieve the author of the device support bundle. This is a read-only attribute.

Parameter

None

Returns

The method returns the attribute value.

Example

```
# Get attribute value
my $value = $object->author();
```

integrated_ind( )

Use this method to retrieve information about the integrated DSB type. This is a read-only attribute.

Parameter

None

Returns

The method returns "true" if DSB is integrated, "false" if DSB is imported.

Example

```
# Get attribute value
my $value = $object->integrated_ind();
```

name( )

Use this method to retrieve the name of the device support bundle. This is a read-only attribute.

Parameter

None

Returns

The method returns the attribute value.

Example

```
# Get attribute value
my $value = $object->name();
```

version( )

Use this method to retrieve the version of the device support bundle. This is a read-only attribute.

Parameter

None

Returns

The method returns the attribute value.

Example

```
# Get attribute value
my $value = $object->version();
```

AUTHOR

Infoblox Inc. http://www.infoblox.com/

SEE ALSO

Infoblox::Session

COPYRIGHT

Copyright (c) 2017 Infoblox Inc.

Infoblox::Grid::Discovery::DSBundle - the device support bundle.

Infoblox::Grid::Discovery::IFAddrInfo - IFAddr information associated with the discovery interface.

- NAME
- DESCRIPTION
- CONSTRUCTOR
- SESSION METHODS
- METHODS
  - address( )
  - address_object( )
  - network( )
  - network_str( )
- AUTHOR
- SEE ALSO
- COPYRIGHT

NAME

Infoblox::Grid::Discovery::IFAddrInfo - IFAddr information associated with the discovery interface.

DESCRIPTION

The object provides IFAddr information associated with the discovery interface.

CONSTRUCTOR

```
my $object = Infoblox::Grid::Discovery::IFAddrInfo->new(
    'address'        => $ipaddr,          # Required
    'address_object' => $addr_obj,        # Optional
    'network'        => $network,         # Optional
    'network_str'    => $address/cidr,    # Optional
);
```

SESSION METHODS

The object does not support any session methods.

METHODS

This section describes all the methods that you can use to set or retrieve the attribute values of the object.

address( )

Use this method to set or retrieve the IP address of the discovery interface.

Parameter

The IP address in string format.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
# Get attribute value
my $value = $object->address();
# Modify attribute value
$object->address('10.0.0.1');
```

address_object( )

Use this method to set or retrieve the IP address object of the discovery interface.

Parameter

Valid value is an Infoblox::IPAM::Address object.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
# Get attribute value
my $value = $object->address_object();
# Modify attribute value
$object->address_object($addr_obj);
```

network( )

Use this method to set or retrieve the network object of the discovery interface.

Parameter

Valid value is an Infoblox::DHCP::Network or Infoblox::DHCP::IPv6Network object.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
# Get attribute value
my $value = $object->network();
# Modify attribute value
$object->network($network);
```

network_str( )

Use this method to set or retrieve the network address.

Parameter

The network address in address/CIDR format.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
# Get attribute value
my $value = $object->network_str();
# Modify attribute value
$object->network_str('10.0.0.0/8');
```

AUTHOR

Infoblox Inc. http://www.infoblox.com/

SEE ALSO

Infoblox::DHCP::Network, Infoblox::DHCP::IPv6Network, Infoblox::IPAM::Address

---

COPYRIGHT

Copyright (c) 2017 Infoblox Inc.

Infoblox::Grid::Discovery::JobProcessDetails - job process details object.

- NAME
- DESCRIPTION
- SESSION METHODS
- METHODS
  - end_line( )
  - status( )
  - stream( )
- AUTHOR
- SEE ALSO
- COPYRIGHT

NAME

Infoblox::Grid::Discovery::JobProcessDetails - job process details object.

DESCRIPTION

The object provides discovery job processing logs related to a given task.

SESSION METHODS

The object does not support any session methods.

METHODS

end_line( )

Use this method to retrieve the ending line of the returned stream. This is a read-only attribute.

Parameter

None

Returns

The method returns the attribute value.

Example

```
#Get value
my $value = $process_details->end_line();
```

status( )

Use this method to retrieve the indicator of session status. This is a read-only attribute.

Parameter

None

Returns

The method returns the attribute value that can be "INPROGRESS", "COMPLETED" and "FAILED".

Example

```
#Get value
my $value = $process_details->status();
```

stream( )

Use this method to retrieve the job process log stream. This is a read-only attribute.

Parameter

None

Returns

The method returns the attribute value.

Example

```
#Get value
my $value = $process_details->stream();
```

AUTHOR

Infoblox Inc. http://www.infoblox.com/

SEE ALSO

Infoblox::Grid::Discovery

COPYRIGHT

Copyright (c) 2017 Infoblox Inc.

Infoblox::Discovery::NetworkDeprovisionInfo - Network deprovision information.

- NAME
- DESCRIPTION
- CONSTRUCTOR
- SESSION METHODS
- METHODS
  - interface( )
  - network( )
  - network_view( )
- AUTHOR
- COPYRIGHT

NAME

Infoblox::Discovery::NetworkDeprovisionInfo - Network deprovision information.

DESCRIPTION

The object provides information about network deprovision.

CONSTRUCTOR

```
my $object = Infoblox::Discovery::NetworkDeprovisionInfo->new(
    'interface'          => $interface,       # Required
    'network'            => $network,         # Required
    'network_view'       => $address/cidr,    # Optional, default value is the default network view
);
```

SESSION METHODS

The object does not support any session methods.

METHODS

interface( )

Use this method to set or retrieve the interface object.

Parameter

Valid value is an Infoblox::Grid::Discovery::DeviceInterface object.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get interface value
$value = $deprovision_info->interface();

#Set interface value
$deprovision_info->interface($interface);
```

network( )

Use this method to set or retrieve the network address.

Parameter

Valid value is a string representing the address and netmask of the network in CIDR format (example: 10.0.0.0/8).

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get attribute value
my $value = $deprovision_info->network();

#Set attribute value
$deprovision_info->network("10.0.0.0/8");
```

network_view( )

Use this method to set or retrieve the network view name.

Parameter

Valid value is a string representing network view name.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get attribute value
my $value = $deprovision_info->network_view();

#Set attribute value
$deprovision_info->network_view($view);
```

AUTHOR

Infoblox Inc. http://www.infoblox.com/

COPYRIGHT

Copyright (c) 2017 Infoblox Inc.

Infoblox::Grid::Discovery::Port - the discovery port structure.

- NAME
- DESCRIPTION
- CONSTRUCTOR
- SESSION METHODS
- METHODS
  - comment( )
  - port( )
  - type( )
- AUTHOR
- COPYRIGHT

NAME

Infoblox::Grid::Discovery::Port - the discovery port structure.

DESCRIPTION

The object provides information about the discovery port.

CONSTRUCTOR

```
my $object = Infoblox::Grid::Discovery::Port->new(
    'comment' => $string,            # Optional
    'port'    => $uint,              # Required
    'type'    => 'TCP' | 'UDP',      # Required
);
```

SESSION METHODS

The object does not support any session methods.

METHODS

This section describes all the methods that you can use to set or retrieve the attribute values of the object.

comment( )

Use this method to set or retrieve the comment.

Parameter

The comment in string format.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
# Get attribute value
my $value = $object->comment();
# Modify attribute value
$object->comment('comment');
```

port( )

Use this method to set or retrieve the port number.

Parameter

The port number.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
# Get attribute value
my $value = $object->port();
# Modify attribute value
$object->port(56);
```

type( )

Use this method to set or retrieve the port type.

Parameter

Valid value is 'TCP' or 'UDP'.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
# Get attribute value
my $value = $object->type();
# Modify attribute value
$object->type('UDP');
```

AUTHOR

Infoblox Inc. http://www.infoblox.com/

COPYRIGHT

Copyright (c) 2017 Infoblox Inc.

Infoblox::Grid::Discovery::Port::Config::AdminStatus - admin status of port configuration.

- NAME
- DESCRIPTION
- SESSION METHODS
- METHODS
  - status( )
  - task_details( )
- AUTHOR
- SEE ALSO
- COPYRIGHT

NAME

Infoblox::Grid::Discovery::Port::Config::AdminStatus - admin status of port configuration.

DESCRIPTION

The object provides information about the admin status of port configuration.

SESSION METHODS

The object does not support any session methods.

METHODS

This section describes all the methods that you can use to retrieve the attribute values of the object.

status( )

Use this method to retrieve the configured admin status. This attribute is read-only.

Parameter

None

Returns

The method returns the attribute value that can be 'UP' or 'DOWN'.

Example

```
# Get attribute value
my $value = $object->status();
```

task_details( )

Use this method to retrieve task details of the port control. This attribute is read-only.

Parameter

None

Returns

The method returns the attribute value that can be an Infoblox::Grid::Discovery::Port::Control::TaskDetails object.

Example

```
# Get attribute value
my $value = $object->task_details();
```

AUTHOR

Infoblox Inc. http://www.infoblox.com/

SEE ALSO

Infoblox::Grid::Discovery::Port::Control::TaskDetails, Infoblox::Grid::Discovery::DeviceInterface

Infoblox::Grid::Discovery::Port::Config::AdminStatus - admin status of port configuration.

COPYRIGHT

Copyright (c) 2017 Infoblox Inc.

Infoblox::Grid::Discovery::Port::Config::Description - port configuration description.

- NAME
- DESCRIPTION
- SESSION METHODS
- METHODS
  - description( )
  - task_details( )
- AUTHOR
- SEE ALSO
- COPYRIGHT

NAME

Infoblox::Grid::Discovery::Port::Config::Description - port configuration description.

DESCRIPTION

The object provides information about the port configuration description.

SESSION METHODS

The object does not support any session methods.

METHODS

This section describes all the methods that you can use to retrieve the attribute values of the object.

description( )

Use this method to retrieve the configured description. This attribute is read-only.

Parameter

None

Returns

The method returns the attribute value.

Example

```
# Get attribute value
my $value = $object->description();
```

task_details( )

Use this method to retrieve task details of the port control. This attribute is read-only.

Parameter

None

Returns

The method returns the attribute value that can be an Infoblox::Grid::Discovery::Port::Control::TaskDetails object.

Example

```
# Get attribute value
my $value = $object->task_details();
```

AUTHOR

Infoblox Inc. http://www.infoblox.com/

SEE ALSO

Infoblox::Grid::Discovery::Port::Control::TaskDetails, Infoblox::Grid::Discovery::DeviceInterface

Infoblox::Grid::Discovery::Port::Config::Description - port configuration description.

COPYRIGHT

Copyright (c) 2017 Infoblox Inc.

Infoblox::Grid::Discovery::Port::Config::VlanInfo - VLAN information of port configuration.

- NAME
- DESCRIPTION
- SESSION METHODS
- METHODS
  - data_vlan_info( )
  - task_details( )
  - voice_vlan_info( )
- AUTHOR
- SEE ALSO
- COPYRIGHT

NAME

Infoblox::Grid::Discovery::Port::Config::VlanInfo - VLAN information of port configuration.

DESCRIPTION

The object provides information about the port configuration VLAN.

SESSION METHODS

The object does not support any session methods.

METHODS

This section describes all the methods that you can use to retrieve the attribute values of the object.

data_vlan_info( )

Use this method to retrieve information about data VLAN. This attribute is read-only.

Parameter

None

Returns

The method returns the attribute value that can be an Infoblox::Grid::Discovery::VLANInfo object.

Example

```
# Get attribute value
my $value = $object->data_vlan_info();
```

task_details( )

Use this method to retrieve task details of the port control. This attribute is read-only.

Parameter

None

Returns

The method returns the attribute value that can be an Infoblox::Grid::Discovery::Port::Control::TaskDetails object.

Example

```
# Get attribute value
my $value = $object->task_details();
```

voice_vlan_info( )

Use this method to retrieve information about voice VLAN. This attribute is read-only.

Parameter

None

Returns

The method returns the attribute value that can be an Infoblox::Grid::Discovery::VLANInfo object.

Example

```
# Get attribute value
my $value = $object->voice_vlan_info();
```

AUTHOR

Infoblox Inc. http://www.infoblox.com/

SEE ALSO

Infoblox::Grid::Discovery::Port::Control::TaskDetails, Infoblox::Grid::Discovery::DeviceInterface, Infoblox::Grid::Discovery::VLANInfo

---

COPYRIGHT

Copyright (c) 2017 Infoblox Inc.

Infoblox::Grid::Discovery::Port::Control::Info - port control information.

- NAME
- DESCRIPTION
- CONSTRUCTOR
- SESSION METHODS
- METHODS
  - admin_status( )
  - data_vlan_info( )
  - description( )
  - device( )
  - interface( )
  - parent( )
  - voice_vlan_info( )
- AUTHOR
- SEE ALSO
- COPYRIGHT

NAME

Infoblox::Grid::Discovery::Port::Control::Info - port control information.

DESCRIPTION

The object provides information about port control.

CONSTRUCTOR

```
my $object = Infoblox::Grid::Discovery::Port::Control::Info->new(
    'device'            => $device,          # optional
    'interface'         => $interface,       # required
    'data_vlan_info'    => $vlan_info,       # optional
    'voice_vlan_info'   => $vlan_info,       # optional
    'admin_status'      => 'UP' | 'DOWN',    # optional
    'description'       => $string,          # optional
    'parent'            => $parent,          # optional
);
```

SESSION METHODS

The object does not support any session methods.

METHODS

This section describes all the methods that you can use to set or retrieve the attribute values of the object.

admin_status( )

Use this method to set or retrieve the administrative state of the interface.

Parameter

Valid value is 'UP' or 'DOWN'.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
# Get attribute value
my $value = $object->admin_status();
# Modify attribute value
$object->admin_status('UP');
```

data_vlan_info( )

Use this method to set or retrieve information about data VLAN of the interface.

Parameter

Valid value is an Infoblox::Grid::Discovery::VLANInfo object.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
# Get attribute value
my $value = $object->data_vlan_info();
# Modify attribute value
$object->data_vlan_info($vlan_info);
```

description( )

Use this method to set or retrieve the description of the interface.

Parameter

The description in string format.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
# Get attribute value
my $value = $object->description();
# Modify attribute value
$object->description('example');
```

device( )

Use this method to set or retrieve the device.

Parameter

Valid value is an Infoblox::Grid::Discovery::Device object.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
# Get attribute value
my $value = $object->device();
# Modify attribute value
$object->device($device);
```

interface( )

Use this method to set or retrieve the interface.

Parameter

Valid value is an Infoblox::Grid::Discovery::DeviceInterface object.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
# Get attribute value
my $value = $object->interface();
# Modify attribute value
$object  interface $interface
```

```
->          (          );
```

parent( )

Use this method to set or retrieve the parent to which port is configured.

Parameter

Valid value is Infoblox::Grid::Discovery::DeviceInterface, Infoblox::DNS::Host, Infoblox::DHCP::FixedAddr, Infoblox::DHCP::IPv6FixedAddr or Infoblox::Grid::Member object.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
# Get attribute value
my $value = $object->parent();
# Modify attribute value
$object->parent($parent);
```

voice_vlan_info( )

Use this method to set or retrieve information about voice VLAN of the interface.

Parameter

Valid value is an Infoblox::Grid::Discovery::VLANInfo object.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
# Get attribute value
my $value = $object->voice_vlan_info();
# Modify attribute value
$object->voice_vlan_info($vlan_info);
```

AUTHOR

Infoblox Inc. http://www.infoblox.com/

SEE ALSO

Infoblox::Grid::Discovery, Infoblox::Grid::Discovery::VLANInfo, Infoblox::Grid::Discovery::Device, Infoblox::Grid::Discovery::DeviceInterface

---

COPYRIGHT

Copyright (c) 2017 Infoblox Inc.

Infoblox::Grid::Discovery::Port::Control::TaskDetails - port control task details.

- NAME
- DESCRIPTION
- SESSION METHODS
- METHODS
  - id( )
  - is_synchronized( )
  - status( )
- AUTHOR
- SEE ALSO
- COPYRIGHT

NAME

Infoblox::Grid::Discovery::Port::Control::TaskDetails - port control task details.

DESCRIPTION

The object provides information about port control task details.

SESSION METHODS

The object does not support any session methods.

METHODS

This section describes all the methods that you can use to retrieve the attribute values of the object.

id( )

Use this method to retrieve the task ID. This attribute is read-only.

Parameter

None

Returns

The method returns the attribute value.

Example

```
# Get attribute value
my $value = $object->id();
```

is_synchronized( )

Use this method to retrieve the flag indicates if the task completed and matched with current discovered value. This attribute is read-only.

Parameter

None

Returns

The method returns the attribute value.

Example

```
# Get attribute value
my $value = $object->is_synchronized();
```

status( )

Use this method to retrieve the task status. This attribute is read-only.

Parameter

None

Returns

The method returns the attribute value.

Example

```
# Get attribute value
my $value = $object->status();
```

AUTHOR

Infoblox Inc. http://www.infoblox.com/

SEE ALSO

Infoblox::Grid::Discovery::Port::Config::AdminStatus, Infoblox::Grid::Discovery::Port::Config::Description, Infoblox::Grid::Discovery::Port::Config::VlanInfo

---

COPYRIGHT

Copyright (c) 2017 Infoblox Inc.

Infoblox::Grid::Discovery::Properties - Grid discovery properties.

- NAME
- DESCRIPTION
- CONSTRUCTOR
- SESSION METHODS
  - Infoblox::Session->get( )
  - Infoblox::Session->modify( )
- METHODS
  - advanced_polling_settings( )
  - auto_conversion_settings( )
  - basic_polling_settings( )
  - enable_auto_conversion( )
  - enable_auto_updates( )
  - cli_credentials( )
  - discovery_blackout_setting( )
  - ignore_conflict_duration( )
  - ports( )
  - port_control_blackout_setting( )
  - same_port_control_discovery_blackout( )
  - snmp3_credentials( )
  - snmp_credentials( )
  - test_credential( )
  - unmanaged_ips_limit( )
  - unmanaged_ips_timeout( )
  - vrf_mapping_rules( )
  - vrf_mapping_policy( )
- AUTHOR
- SEE ALSO
- COPYRIGHT

NAME

Infoblox::Grid::Discovery::Properties - Grid discovery properties.

DESCRIPTION

The object provides information about Grid discovery properties.

CONSTRUCTOR

```
my $object = Infoblox::Grid::Discovery::Properties->new(
    'advanced_polling_settings'      => $adv,                 # Required
    'auto_conversion_settings'       => [$auto_conversion1, ...], # Optional
    'basic_polling_settings'         => $basic,               # Required
    'enable_auto_conversion'         => 'true' | 'false'      # Optional
    'enable_auto_updates'            => 'true' | 'false'      # Optional
    'snmp3_credentials'              => [$snmp3_1, ...],      # Optional
    'snmp_credentials'               => [$snmp_1, ...],       # Optional
    'cli_credentials'                => [$cli1, ...],         # Optional
    'discovery_blackout_setting'     => $blackout,            # Optional
    'port_control_blackout_setting' => $blackout,             # Optional
    'ports'                          => [$port1, ...],        # Optional
    'ignore_conflict_duration'       => $uint,                # Optional
    'same_port_control_discovery_blackout' => 'true' | 'false' # Optional
    'unmanaged_ips_limit'            => $uint,                  # Optional
    'unmanaged_ips_timeout'          => $uint,                  # Optional
    'vrf_mapping_rules'              => [$vrf_rule_1, ...],     # Optional
    'vrf_mapping_policy'             => 'NONE' | 'RULE_BASED' | 'RULE_AND_INTERNAL_BASED', # Optional
);
```

SESSION METHODS

This section describes all the methods in Infoblox::Session that you can apply to a object.

Infoblox::Session->get( )

Use this method to retrieve the existing object from the Infoblox appliance. See Infoblox::Session->get() for parameters and return values.

Example

```
my $object = $session->get('object' => 'Infoblox::Grid::Discovery::Properties');
```

Infoblox::Session->modify( )

Use this method to modify object in the Infoblox appliance. See Infoblox::Session->modify() for parameters and return values.

Example

```
my $port = Infoblox::Grid::Discovery::Port->new(
    'port' => 40,
    'type' => 'UDP',
);

# Use method to modify ports
$object->ports([$port]);
# Submit modification
my $response = $session->modify( $object );
```

METHODS

This section describes all the methods that you can use to set or retrieve the attribute values of the object.

advanced_polling_settings( )

Use this method to set or retrieve discovery advanced poll settings.

Parameter

Valid value is an Infoblox::Grid::Discovery::AdvancedPollSettings object.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
# Get attribute value
my $value = $object->advanced_polling_settings();
# Modify attribute value
$object->advanced_polling_settings($adv);
```

auto_conversion_settings( )

Use this method to set or retrieve a list of automatic conversion settings.

Parameter

Valid value is an array reference that contains Infoblox::Grid::Discovery::AutoConversionSetting objects.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
# Get attribute value
my $value = $object->auto_conversion_settings();
# Modify attribute value
$object->auto_conversion_settings([$auto_conversion_settings1, $auto_conversion_settings2]);
```

basic_polling_settings( )

Use this method to set or retrieve discovery basic poll settings.

Parameter

Valid value is an Infoblox::Grid::Discovery::BasicPollSettings object.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
# Get attribute value
my $value = $object->basic_polling_settings();
# Modify attribute value
$object->basic_polling_settings($basic);
```

enable_auto_conversion( )

Use this method to set or retrieve the flag that enables automatic conversion of discovered data.

Parameter

Specify 'true' to enable automatic conversion of discovery data or 'false' to disable it. The default value is 'false'.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
# Get attribute value
my $value = $object->enable_auto_conversion();
# Modify attribute value
$object->enable_auto_conversion('true');
```

enable_auto_updates( )

Use this method to set or retrieve the flag that enables updating discovered data for managed objects.

Parameter

Specify 'true' to enable updating discovered data for managed objects or 'false' to disable it. The default value is 'true'.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
# Get attribute value
my $value = $object->enable_auto_updates();
# Modify attribute value
$object->enable_auto_updates('true');
```

cli_credentials( )

Use this method to set or retrieve the discovery CLI credentials.

Parameter

Valid value is an array reference that contains Infoblox::Grid::Discovery::CLICredential objects.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
# Get attribute value
my $value = $object->cli_credentials();
# Modify attribute value
$object->cli_credentials([$cli1, $cli2]);
```

discovery_blackout_setting( )

Use this method to set or retrieve the discovery blackout setting.

Parameter

Valid value is an Infoblox::Grid::Discovery::Properties::BlackoutSetting object.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
# Get attribute value
my $value = $object->discovery_blackout_setting();
# Modify attribute value
$object->discovery_blackout_setting($blackout);
```

ignore_conflict_duration( )

Use this method to set or retrieve the ignore discovery conflict duration.

Parameter

The ignore discovery conflict duration in seconds.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
# Get attribute value
my $value = $object->ignore_conflict_duration();
# Modify attribute value
$object->ignore_conflict_duration(3600);
```

ports( )

Use this method to set or retrieve ports to scan.

Parameter

Valid value is an array reference that contains Infoblox::Grid::Discovery::Port objects.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
# Get attribute value
my $value = $object->ports();
# Modify attribute value
$object->ports([$port1, $port2]);
```

port_control_blackout_setting( )

Use this method to set or retrieve the port control blackout setting.

Parameter

Valid value is an Infoblox::Grid::Discovery::Properties::BlackoutSetting object.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
# Get attribute value
my $value = $object->port_control_blackout_setting();
# Modify attribute value
$object->port_control_blackout_setting($blackout);
```

same_port_control_discovery_blackout( )

Use this method to set or retrieve the flag that enables using the same discovery blackout for port control blackout.

Parameter

Specify 'true' to use same discovery blackout for port control blackout or 'false' to disable it. The default value is 'false'.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
# Get attribute value
my $value = $object->same_port_control_discovery_blackout();
# Modify attribute value
$object->same_port_control_discovery_blackout('true');
```

snmp3_credentials( )

Use this method to set or retrieve a list of discovery SNMPv3 credentials.

Parameter

Valid value is an array reference that contains Infoblox::Grid::Discovery::SNMP3Credential objects.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
# Get attribute value
my $value = $object->snmp3_credentials();
# Modify attribute value
$object->snmp3_credentials([$snmp3_1, $snmp3_2]);
```

snmp_credentials( )

Use this method to set or retrieve a list of discovery SNMP v1 and v2 credentials>.

Parameter

Valid value is an array reference that contains Infoblox::Grid::Discovery::SNMPCredential objects.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
# Get attribute value
my $value = $object->snmp_credentials();
# Modify attribute value
$object->snmp_credentials([$snmp1, $snmp2]);
```

test_credential( )

Use this method to test credentials.

Note that this method does not perform automatic polling for the operational status. It returns a snapshot of the status at the time when you call the method, which may not be the final status. To get the latest status, the client script needs to provide polling logic for this method until the operation status is 'INPROGRESS' by calling the method with start parameter assigned to the end field in the previous call result. For more details, see the example below.

Parameter

```
id              - Optional. The test credential ID.
```

The following members are supported when id is defined:

```
start           - Optional. The start position of the text.
```

The following members are supported when id is not defined:

```
 view             - Optional. The network view name. The default value is the system-defined default
network view.

 type             - Optional if one of the following is defined: cli_credentials, snmp_credential, or
snmp3_credential. Otherwise, this member is required. The valid value is 'CLI', 'SNMP' or 'SNMP3'.

 device           - Required if address is not defined. The Infoblox::Grid::Discovery::Device object.

 member           - Optional. The discovery member name.

 address          - Required if device is not defined. The IP address of the device.

 cli_credentials  - Optional. At least one of the following must be defined: cli_credentials,
snmp_credential, or snmp3_credential. The array reference that contains
Infoblox::Grid::Discovery::CLICredential objects.

 snmp_credential  - Optional. At least one of the following must be defined: cli_credentials,
snmp_credential, or snmp3_credential. The Infoblox::Grid::Discovery::SNMPCredential object.

 snmp3_credential - Optional. At least one of the following must be defined: cli_credentials,
snmp_credential, or snmp3_credential. The Infoblox::Grid::Discovery::SNMP3Credential object.
```

Returns

The method returns an Infoblox::Grid::Discovery::TestCredential object when the operation succeeds, and returns false when the operation fails.

Example

```
# Get a test credential ID
my $result = $object->test_credential(
   'address'         => '10.0.0.1',
   'snmp_credential' => $snmp,
);

# Get detailed information by id
if (ref($result) eq 'Infoblox::Grid::Discovery::TestCredential') {
    $result = $object->test_credential('id' => $result->id());
}

# check for the 'COMPLETE' or 'FAILED' status of test credential
while (ref($result) eq 'Infoblox::Grid::Discovery::TestCredential' &&
       $result->status() eq 'INPROGRESS') {
    $result = $grid->test_credential('id' => $result->id(), 'start' => $result->end());
}
```

unmanaged_ips_limit( )

Use this method to set or retrieve the limit value for discovered unmanaged IP addresses. This value determines how often users are notified when new unmanaged IP addresses are discovered in a particular network.

Parameter

The valid value is an unsigned integer. The minimun value is 1.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
# Get attribute value
my $value = $object->unmanaged_ips_limit();
# Modify attribute value
$object->unmanaged_ips_limit(30);
```

unmanaged_ips_timeout( )

Use this method to set or retrieve the timeout value (in seconds) between notifications about new unmanaged IP addresses in a particular network.

Parameter

The valid value is an unsigned integer. The minimum value is 60 (one minute), and the maximum value is the number of seconds from the time you set it until January 2038.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
# Get attribute value
my $value = $object->unmanaged_ips_timeout();
# Modify attribute value
$object->unmanaged_ips_timeout(300);
```

vrf_mapping_rules( )

Use this method to set or retrieve a list of VRF mapping rules.

Parameter

Valid value is an array reference that contains Infoblox::Grid::Discovery::VRFMappingRule objects.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
# Get attribute value
my $value = $object->vrf_mapping_rules();
# Modify attribute value
$object->vrf_mapping_rules([$vrf_mapping_rule1, $vrf_mapping_rule2]);
```

vrf_mapping_policy( )

Use this method to set or retrieve the VRF mapping policy.

Parameter

The policy type used to define the behavior of the VRF mapping. Valid value is one of the following policy types: 'NONE', 'RULE_BASED', 'RULE_AND_INTERNAL_BASED'.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
# Get attribute value
my $value = $object->vrf_mapping_policy();
# Modify attribute value
$object->vrf_mapping_policy('RULE_AND_INTERNAL_BASED');
```

AUTHOR

Infoblox Inc. http://www.infoblox.com/

SEE ALSO

Infoblox::Session, Infoblox::Grid::Discovery::SNMPCredential, Infoblox::Grid::Discovery::SNMP3Credential, Infoblox::Grid::Discovery::Port, Infoblox::Grid::Discovery::CLICredential, Infoblox::Grid::Discovery::Properties::BlackoutSetting, Infoblox::Grid::Discovery::VRFMappingRule, Infoblox::Grid::Discovery::AutoConversionSetting

---

COPYRIGHT

Copyright (c) 2017 Infoblox Inc.

NAME

- NAME
- DESCRIPTION
- CONSTRUCTOR
- SESSION METHODS
- METHODS
  - blackout_duration( )
  - blackout_schedule( )
  - enable_blackout( )
- AUTHOR
- SEE ALSO
- COPYRIGHT

NAME

Infoblox::Grid::Discovery::Properties::BlackoutSetting

DESCRIPTION

The object provides information about the discovery or port control blackout settings.

CONSTRUCTOR

```
my $object = Infoblox::Grid::Discovery::Properties::BlackoutSetting->new(
    'enable_blackout'   => 'true' | 'false',      # optional
    'blackout_duration' => $uint,                 # optional
    'blackout_schedule' => $schedule,             # optional
);
```

SESSION METHODS

The object does not support any session methods.

METHODS

This section describes all the methods that you can use to set or retrieve the attribute values of the object.

blackout_duration( )

Use this method to set or retrieve the blackout duration.

Parameter

The blackout duration in minutes.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
# Get attribute value
my $value = $object->blackout_duration();
# Modify attribute value
$object->blackout_duration(10);
```

blackout_schedule( )

Use this method to set or retrieve the blackout schedule.

Parameter

Valid value is an Infoblox::Grid::ScheduleSetting object.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
# Get attribute value
my $value = $object->blackout_schedule();
# Modify attribute value
$object->blackout_schedule($schedule);
```

enable_blackout( )

Use this method to enable or disable the Discovery or port control blackout.

Parameter

Specify 'true' to enable the discovery or port control blackout or 'false' to disable it. The default value is 'false'.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
# Get attribute value
my $value = $object->enable_blackout();
# Modify attribute value
$object->enable_blackout('true');
```

AUTHOR

Infoblox Inc. http://www.infoblox.com/

SEE ALSO

Infoblox::Grid::Discovery::Properties

COPYRIGHT

Copyright (c) 2017 Infoblox Inc.

Infoblox::Grid::Discovery::SeedRouter - The seed router structure.

- NAME
- DESCRIPTION
- CONSTRUCTOR
- SESSION METHODS
- METHODS
  - address( )
  - network_view( )
  - comment( )
- AUTHOR
- COPYRIGHT

NAME

Infoblox::Grid::Discovery::SeedRouter - The seed router structure.

DESCRIPTION

The object provides information about the seed router.

CONSTRUCTOR

```
my $object = Infoblox::Grid::Discovery::SeedRouter->new(
    'address'      => $ipaddr,       # Required
    'network_view' => $string,       # Optional
    'comment'      => $string,       # Optional
);
```

SESSION METHODS

The object does not support any session methods.

METHODS

This section describes all the methods that you can use to set or retrieve the attribute values of the object.

address( )

Use this method to set or retrieve the IP address of the seed router.

Parameter

The IP address in string format.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
# Get attribute value
my $value = $object->address();
# Modify attribute value
$object->address('10.0.0.1');
```

network_view( )

Use this method to set or retrieve the network view associated with the seed router.

Parameter

The name of the network view

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
# Get attribute value
my $value = $object->network_view();
# Modify attribute value
$object->network_view('papi_network_view');
```

comment( )

Use this method to set or retrieve the description of the seed router.

Parameter

The description of the seed router in string format.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
# Get attribute value
my $value = $object->comment();
# Modify attribute value
$object->comment('comment');
```

AUTHOR

Infoblox Inc. http://www.infoblox.com/

COPYRIGHT

Copyright (c) 2017 Infoblox Inc.

Infoblox::Grid::Discovery::SNMP3Credential - SNMPv3 credential.

- NAME
- DESCRIPTION
- CONSTRUCTOR
- SESSION METHODS
- METHODS
  - authentication_password( )
  - authentication_protocol( )
  - comment( )
  - privacy_password( )
  - privacy_protocol( )
  - user( )
- AUTHOR
- COPYRIGHT

NAME

Infoblox::Grid::Discovery::SNMP3Credential - SNMPv3 credential.

DESCRIPTION

The object provides information about SNMPv3 credential.

CONSTRUCTOR

```
 my $object = Infoblox::Grid::Discovery::SNMP3Credential->new(
    'authentication_password' => $string,                          # Required if authentication_protocol is
MD5 or SHA
    'authentication_protocol' => 'MD5' | 'NONE' | 'SHA',           # Requred
    'comment'                 => $string,                          # Optional
    'privacy_password'        => $string,                          # Required if privacy_protocol is 'AES',
'DES' or '3DES'
    'privacy_protocol'        => 'AES' | 'DES' | '3DES' | 'NONE', # Required
    'user'                    => $string,                          # Required
 );
```

SESSION METHODS

The object does not support any session methods.

METHODS

This section describes all the methods that you can use to set or retrieve the attribute values of the object.

authentication_password( )

Use this method to set the authentication password if the authentication protocol is MD5 or SHA. This is a write-only attribute.

Parameter

The authentication password in string format.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

Example

```
 # Modify attribute value
 $object->authentication_password('infoblox');
```

authentication_protocol( )

Use this method to set or retrieve the authentication protocol.

Parameter

Valid value is 'MD5', 'NONE' or 'SHA'.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
# Get attribute value
my $value = $object->authentication_protocol();
# Modify attribute value
$object->authentication_protocol('MD5');
```

comment( )

Use this method to set or retrieve the comment on SNMPv3 user.

Parameter

The comment in string format.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
# Get attribute value
my $value = $object->comment();
# Modify attribute value
$object->comment('SNMPv3');
```

privacy_password( )

Use this method to set the privacy password if the privacy protocol is 'AES', 'DES' or '3DES'. This is a write-only attribute.

Parameter

The privacy password in string format.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

Example

```
# Modify attribute value
$object->privacy_password('infoblox');
```

privacy_protocol( )

Use this method to set or retrieve the privacy protocol.

Parameter

Valid value is 'AES', 'DES', '3DES' or 'NONE'.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
# Get attribute value
my $value = $object->privacy_protocol();
# Modify attribute value
$object->privacy_protocol('3DES');
```

user( )

Use this method to set or retrieve the SNMPv3 user name.

Parameter

Tthe SNMPv3 user name in string format.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
# Get attribute value
my $value = $object->user();
# Modify attribute value
$object->user('admin');
```

AUTHOR

Infoblox Inc. http://www.infoblox.com/

COPYRIGHT

Copyright (c) 2017 Infoblox Inc.

Infoblox::Grid::Discovery::SNMPCredential - SNMPv1 and SNMPv2 credential.

- NAME
- DESCRIPTION
- CONSTRUCTOR
- SESSION METHODS
- METHODS
  - comment( )
  - community_string( )
- AUTHOR
- COPYRIGHT

NAME

Infoblox::Grid::Discovery::SNMPCredential - SNMPv1 and SNMPv2 credential.

DESCRIPTION

The object provides information about SNMPv1 and SNMPv2 credential.

CONSTRUCTOR

```
my $object = Infoblox::Grid::Discovery::SNMPCredential->new(
    'comment'          => $string,    # Optional
    'community_string' => $string,    # Required
);
```

SESSION METHODS

The object does not support any session methods.

METHODS

This section describes all the methods that you can use to set or retrieve the attribute values of the object.

comment( )

Use this method to set or retrieve the comment on SNMPv1 and SNMPv2 user.

Parameter

The comment in string format.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
# Get attribute value
my $value = $object->comment();
# Modify attribute value
$object->comment('comment');
```

community_string( )

Use this method to set or retrieve the public community string.

Parameter

The public community string.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
# Get attribute value
my $value = $object->community_string();
# Modify attribute value
$object->community_string('string');
```

AUTHOR

Infoblox Inc. http://www.infoblox.com/

COPYRIGHT

Copyright (c) 2017 Infoblox Inc.

Infoblox::Grid::Discovery::Status - the discovery status of discovered data.

- NAME
- DESCRIPTION
- SESSION METHOD
  - Infoblox::Session->get( )
  - Infoblox::Session->search( )
- METHODS
  - address( )
  - cli_collection_enabled( )
  - existence_info( )
  - fingerprint_enabled( )
  - fingerprint_info( )
  - first_seen( )
  - last_action( )
  - last_seen( )
  - last_timestamp( )
  - name( )
  - network_view( )
  - reachable_info( )
  - snmp_collection_enabled( )
  - snmp_collection_info( )
  - snmp_credential_info( )
  - status( )
  - type( )
- AUTHOR
- SEE ALSO
- COPYRIGHT

NAME

Infoblox::Grid::Discovery::Status - the discovery status of discovered data.

DESCRIPTION

The object provides information about the discovery status of discovered data.

SESSION METHOD

This section describes all the methods in Infoblox::Session that you can apply to the object.

Infoblox::Session->get( )

Use this method to retrieve all the matching objects from the Infoblox appliance. See Infoblox::Session->get() for parameters and return values.

Key References

```
Apply the following attributes to get a specific object:

 address - Optional. The IP address of the device.
 name    - Optional. The device name.
 network_view - Optional. The network view name.
```

Example

```
my @retrieved_objs = $session->get(
    'object' => 'Infoblox::Grid::Discovery::Status',
    'address' => '10.0.0.1',
);
```

Infoblox::Session->search( )

Use this method to search for objects in the Infoblox appliance. See Infoblox::Session->search() for parameters and return values.

Key References

```
Apply the following attributes to search for a object:

 address - Optional. The IP address of the device (regular expression).
 name    - Optional. The device name (regular expression).
 network_view - Optional. The network view name.
```

Example

```
# search for all objects
my @retrieved_objs = $session->search(
    'object' => 'Infoblox::Grid::Discovery::Status',
    'address' => '10.0.0.1',
);
```

METHODS

This section describes all the methods that you can use to set or retrieve the attribute values of the object.

address( )

Use this method to retrieve the IP address of the device. This is a read-only attribute.

Parameter

None

Returns

The method returns the attribute value.

Example

```
# Get attribute value
my $value = $object->address();
```

cli_collection_enabled( )

Use this method to retrieve the flag that indicates if CLI collection is enabled. This is a read-only attribute.

Parameter

None

Returns

The method returns the attribute value that can be 'true' or 'false'.

Example

```
#Get attribute value
my $value = $status->cli_collection_enabled();
```

existence_info( )

Use this method to retrieve information about the device existence status. This is a read-only attribute.

Parameter

None

Returns

The method returns the attribute value that can be an Infoblox::Grid::Discovery::StatusInfo object.

Example

```
# Get attribute value
my $value = $object->existence_info();
```

fingerprint_enabled( )

Use this method to retrieve the flag that indicates if DHCP fingerprinting is enabled. This is a read-only attribute.

Parameter

None

Returns

The method returns the attribute value that can be 'true' or 'false'.

Example

```
# Get attribute value
my $value = $object->fingerprint_enabled();
```

fingerprint_info( )

Use this method to retrieve information about the device fingerptint status. This is a read-only attribute.

Parameter

None

Returns

The method returns the attribute value that can be an Infoblox::Grid::Discovery::StatusInfo object.

Example

```
# Get attribute value
my $value = $object->fingerprint_info();
```

first_seen( )

Use this method to retrieve the timestamp when the device was first discovered. This is a read-only attribute.

Parameter

None

Returns

The method returns the attribute value.

Example

```
# Get attribute value
my $value = $object->first_seen();
```

last_action( )

Use this method to retrieve the timestamp of the last detected interface property change. This is a read-only attribute.

Parameter

None

Returns

The method returns the attribute value.

Example

```
# Get attribute value
my $value = $object->last_action();
```

last_seen( )

Use this method to retrieve the timestamp when the device was last discovered. This is a read-only attribute.

Parameter

None

Returns

The method returns the attribute value.

Example

```
# Get attribute value
my $value = $object->last_seen();
```

last_timestamp( )

Use this method to retrieve the timestamp of the last action executed for the device. This is a read-only attribute.

Parameter

None

Returns

The method returns the attribute value.

Example

```
# Get attribute value
my $value = $object->last_timestamp();
```

name( )

Use this method to retrieve the device name. This is a read-only attribute.

Parameter

None

Returns

The method returns the attribute value.

Example

```
# Get attribute value
my $value = $object->name();
```

network_view( )

Use this method to retrieve the network view name. This is a read-only attribute.

Parameter

None

Returns

The method returns the attribute value.

Example

```
# Get attribute value
my $value = $object->network_view();
```

reachable_info( )

Use this method to retrieve information about the device reachable status. This is a read-only attribute.

Parameter

None

Returns

The method returns the attribute value that can be an Infoblox::Grid::Discovery::StatusInfo object.

Example

```
# Get attribute value
my $value = $object->reachable_info();
```

snmp_collection_enabled( )

Use this method to retrieve the flag that indicates whether SNMP collection is enabled for the device. This is a read-only attribute.

Parameter

None

Returns

The method returns the attribute value that can be 'true' or 'false'.

Example

```
# Get attribute value
my $value = $object->snmp_collection_enabled();
```

snmp_collection_info( )

Use this method to retrieve information about the device SNMP collection status. This is a read-only attribute.

Parameter

None

Returns

The method returns the attribute value that can be an Infoblox::Grid::Discovery::StatusInfo object.

Example

```
# Get attribute value
my $value = $object->snmp_collection_info();
```

snmp_credential_info( )

Use this method to retrieve information about the device SNMP credential status. This is a read-only attribute.

Parameter

None

Returns

The method returns the attribute value that can be an Infoblox::Grid::Discovery::StatusInfo object.

Example

```
# Get attribute value
my $value = $object->snmp_credential_info();
```

status( )

Use this method to retrieve the status of the device. This is a read-only attribute.

Parameter

None

Returns

The method returns the attribute value that can be 'OK', 'ERROR' or 'NON_REACHABLE'.

Example

```
# Get attribute value
my $value = $object->status();
```

type( )

Use this method to retrieve the device type. This is a read-only attribute.

Parameter

None

Returns

The method returns the attribute value.

Example

```
# Get attribute value
my $value = $object->type();
```

AUTHOR

Infoblox Inc. http://www.infoblox.com/

SEE ALSO

Infoblox::Session, Infoblox::Grid::Discovery::StatusInfo

Infoblox::Grid::Discovery::Status - the discovery status of discovered data.

COPYRIGHT

Copyright (c) 2017 Infoblox Inc.

Infoblox::Grid::Discovery::StatusInfo - the discovery status information.

- NAME
- DESCRIPTION
- CONSTRUCTOR
- SESSION METHODS
- METHODS
  - message( )
  - status( )
  - timestamp( )
- AUTHOR
- COPYRIGHT

NAME

Infoblox::Grid::Discovery::StatusInfo - the discovery status information.

DESCRIPTION

The object provides information about the discovery status.

CONSTRUCTOR

```
my $object = Infoblox::Grid::Discovery::StatusInfo->new(
    'message'   => $string,                                  # Required
    'status'    => 'OK' | 'ERROR' | 'NON_REACHABLE',         # Required
    'timestamp' => $epoch,                                   # Required
);
```

SESSION METHODS

The object does not support any session methods.

METHODS

This section describes all the methods that you can use to set or retrieve the attribute values of the object.

message( )

Use this method to set or retrieve the detailed message.

Parameter

The detailed message in string format.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
# Get attribute value
my $value = $object->message();
# Modify attribute value
$object->message('text');
```

status( )

Use this method to set or retrieve the status of the device.

Parameter

Valid value is 'OK', 'ERROR' or 'NON_REACHABLE'.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
# Get attribute value
my $value = $object->status();
# Modify attribute value
$object->status('OK');
```

timestamp( )

Use this method to set or retrieve the timestamp when status was generated.

Parameter

The date and time in the ISO 8601 extended format (e.g. 2008-10-21T10:56:33Z or 2008-10-21T10:56:33.001Z).

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
# Get attribute value
my $value = $object->timestamp();
# Modify attribute value
$object->timestamp('2008-10-21T10:56:33Z');
```

AUTHOR

Infoblox Inc. http://www.infoblox.com/

COPYRIGHT

Copyright (c) 2017 Infoblox Inc.

NAME

- NAME
- DESCRIPTION
- SESSION METHODS
- METHODS
  - end( )
  - id( )
  - status( )
  - text( )
- AUTHOR
- SEE ALSO
- COPYRIGHT

NAME

Infoblox::Grid::Discovery::TestCredential

DESCRIPTION

The object provides information about results of the test credential.

SESSION METHODS

The object does not support any session methods.

METHODS

This section describes all the methods that you can use to set or retrieve the attribute values of the object.

end( )

Use this method to retrieve the end position of the returned text. This attribute is read-only.

Parameter

None.

Returns

The method returns the attribute value.

Example

```
# Get attribute value
my $value = $object->end();
```

id( )

Use this method to retrieve the test credential ID. This attribute is read-only.

Parameter

None.

Returns

The method returns the attribute value.

Example

```
# Get attribute value
my $value = $object->id();
```

status( )

Use this method to retrieve the test credential status. This attribute is read-only.

Parameter

None.

Returns

The method returns the attribute value that can be 'COMPLETED', 'FAILED' or 'INPROGRESS'.

Example

```
# Get attribute value
my $value = $object->status();
```

text( )

Use this method to retrieve the portion of the text from start (inclusive) to end (exclusive). This attribute is read-only.

Parameter

None.

Returns

The method returns the attribute value.

Example

```
# Get attribute value
my $value = $object->text();
```

AUTHOR

Infoblox Inc. http://www.infoblox.com/

SEE ALSO

Infoblox::Grid::Discovery::Properties

COPYRIGHT

Copyright (c) 2017 Infoblox Inc.

Infoblox::Grid::Discovery::VLANInfo - VLAN information associated with the discovery interface.

- NAME
- DESCRIPTION
- CONSTRUCTOR
- SESSION METHODS
- METHODS
  - id( )
  - name( )
- AUTHOR
- COPYRIGHT

NAME

Infoblox::Grid::Discovery::VLANInfo - VLAN information associated with the discovery interface.

DESCRIPTION

The object provides VLAN information associated with the discovery interface.

CONSTRUCTOR

```
my $object = Infoblox::Grid::Discovery::VLANInfo->new(
    'id'   => $string,            # Optional
    'name' => $string,            # Optional
);
```

SESSION METHODS

The object does not support any session methods.

METHODS

This section describes all the methods that you can use to set or retrieve the attribute values of the object.

id( )

Use this method to set or retrieve the VLAN ID.

Parameter

The VLAN ID in string format.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
# Get attribute value
my $value = $object->id();
# Modify attribute value
$object->id('23');
```

name( )

Use this method to set or retrieve the VLAN name.

Parameter

The VLAN name in string format.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
# Get attribute value
my $value = $object->name();
# Modify attribute value
$object->name('vlan');
```

AUTHOR

Infoblox Inc. http://www.infoblox.com/

COPYRIGHT

Copyright (c) 2017 Infoblox Inc.

Infoblox::Grid::Discovery::VRFMappingRule - VRF mapping rule.

- NAME
- DESCRIPTION
- CONSTRUCTOR
- SESSION METHODS
- METHODS
  - network_view( )
  - criteria( )
  - comment( )
- AUTHOR
- COPYRIGHT

NAME

Infoblox::Grid::Discovery::VRFMappingRule - VRF mapping rule.

DESCRIPTION

The object provides mapping of regular expressions that represent the VRF name and network view.

CONSTRUCTOR

```
my $object = Infoblox::Grid::Discovery::VRFMappingRule->new(
    'network_view'      => $string,     # Required
    'criteria'          => $string,     # Required
    'comment'           => $string,     # Optional
);
```

SESSION METHODS

The object does not support any session methods.

METHODS

This section describes all the methods that you can use to set or retrieve the attribute values of the object.

network_view( )

Use this method to set or retrieve the name of the network view associated with the VRF mapping rule.

Parameter

The name of the network view associated with the VRF mapping rule.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
# Get attribute value
my $value = $object->network_view();
# Modify attribute value
$object->network_view('default');
```

criteria( )

Use this method to set or retrieve the criteria of the VRF mapping rule.

Parameter

Extended POSIX regular expression matching the VRF name. The regular expression string should contain the leading caret anchor ^ and the trailing dollar anchor $ symbols.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
# Get attribute value
my $value = $object->criteria();
# Modify attribute value
$object->criteria('.*+Red\d');
```

comment( )

Use this method to set or retrieve the comment for the VRF mapping rule.

Parameter

The comment in string format.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
# Get attribute value
my $value = $object->comment();
# Modify attribute value
$object->comment('comment');
```

AUTHOR

Infoblox Inc. http://www.infoblox.com/

COPYRIGHT

Copyright (c) 2017 Infoblox Inc.

Infoblox::Grid::DistributionSchedule - Information about the Grid's distribution schedule.

- NAME
- DESCRIPTION
- CONSTRUCTOR
- SESSION METHODS
  - Infoblox::Session->get( )
  - Infoblox::Session->modify( )
- METHODS
  - active( )
  - groups( )
  - start_time( )
  - time_zone( )
- AUTHOR
- SEE ALSO
- COPYRIGHT

NAME

Infoblox::Grid::DistributionSchedule - Information about the Grid's distribution schedule.

DESCRIPTION

The Distribution Schedule object is used to set and view the distribution schedule of an Infoblox Grid.

CONSTRUCTOR

The Infoblox::Grid::DistributionSchedule object is automatically generated. It does not require manual construction.

SESSION METHODS

This section describes all the methods in an Infoblox::Session that can be applied to a Distribution Schedule object.

Infoblox::Session->get( )

Use this method to retrieve the existing objects from an Infoblox appliance. See Infoblox::Session->get() for parameters and return values.

Example

```
my $schedule = $session->get(object => "Infoblox::Grid::DistributionSchedule");
```

Infoblox::Session->modify( )

Use this method to modify objects in the Infoblox appliance. See Infoblox::Session->modify() for parameters and return values.

Example

```
# Submit modification
my $response = $session->modify( $schedule );
```

METHODS

This section describes all the methods that can be used to set or retrieve the attribute values of a Distribution Schedule object.

active( )

Use this method to set or retrieve the "active" flag.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

Specify "true" to set the active flag or "false" to deactivate/unset it. The default value is "false".

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get active
my $active = $schedule->active();
#Modify active
$schedule->active("true");
```

groups( )

Use this method to set or retrieve the list of groups affected by this distribution schedule.

Parameter

Array reference that contains each Infoblox::Grid::Upgradegroup object.

Note that the members of this list must have their distribution_time parameter set in order for the modification to succeed. This means that it is possible to retrieve a distribution schedule but not be allowed to modify it without setting the distribution_time values for the groups.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

Example

```
#Get the group list
my $grouplist = $schedule->groups();
#Modify it
$grid->groups([$ug1, $ug2, $ug3 ]);
```

start_time( )

Use this method to set or retrieve the start time of the distribution.

Parameter

The start time of the distribution in ISO 8601 extended format (e.g. 2008-10-21T10:56:00Z or 2008-10-21T10:56:00.001Z).

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get start_time
my $start_time = $schedule->start_time();
#Modify start_time
$schedule->start_time("2020-12-21T11:22:33Z");
```

time_zone( )

Use this method to set or retrieve the time zone of the distribution schedule.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

The UTC string that represents the time zone. For example "(UTC - 6:00) Central Time (US and Canada)". Default value is undefined.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get time_zone
my $time_zone = $schedule->time_zone();
#Modify time_zone
$schedule->time_zone("(UTC - 6:00) Central Time (US and Canada)");
```

AUTHOR

Infoblox Inc. http://www.infoblox.com/

SEE ALSO

Infoblox::Session, Infoblox::Session->get()

COPYRIGHT

Copyright (c) 2017 Infoblox Inc.

Infoblox::Grid::DNS - DNS objects management and configuration

- NAME
- DESCRIPTION
- CONSTRUCTOR
- SESSION METHODS
  - Infoblox::Session->get( )
  - Infoblox::Session->modify( )
- METHODS
  - allow_bulkhost_ddns( )
  - allow_gss_tsig_zone_updates( )
  - allow_query( )
  - allow_recursive_query( )
  - allow_transfer( )
  - allow_update( )
  - anonymize_response_logging( )
  - attack_mitigation( )
  - auto_blackhole( )
  - bind_check_names_policy( )
  - bind_hostname_directive( )
  - blackhole_list( )
  - blacklist_action( )
  - blacklist_log_query( )
  - blacklist_redirect_addresses( )
  - blacklist_redirect_ttl( )
  - blacklist_rulesets( )
  - bulk_host_name_template( )
  - bulk_host_name_templates( )
  - capture_dns_queries_on_all_domains( )
  - client_subnet_domains( )
  - client_subnet_ipv4_prefix_length( )
  - client_subnet_ipv6_prefix_length( )
  - copy_xfer_to_notify( )
  - ddns_force_creation_timestamp_update( )
  - ddns_restrict_patterns_list( )
  - ddns_restrict_patterns( )
  - ddns_restrict_static( )
  - ddns_restrict_protected( )
  - ddns_restrict_secure( )
  - ddns_principal_tracking( )
  - ddns_principal_group( )
  - default_ttl( )
  - disable_edns( )
  - dns_cache_acceleration_ttl( )
  - dns_health_check_anycast_control( )
  - dns_health_check_domain_list( )
  - dns_health_check_interval( )
  - dns_health_check_recursion_flag( )
  - dns_health_check_retries( )
  - dns_health_check_timeout( )
  - dns_query_capture_file_time_limit( )
  - dns64_groups( )
  - dnssec_enabled( )
  - dnssec_enable_ksk_auto_rollover( )
  - dnssec_expired_signatures_enabled( )
  - dnssec_ksk_algorithms( )
  - dnssec_ksk_email_notification_enabled( )
  - dnssec_ksk_rollover_interval()
  - dnssec_ksk_rollover_notification_config( )
  - dnssec_ksk_snmp_notification_enabled( )
  - dnssec_negative_trust_anchors( )
  - dnssec_nsec3_iterations( )
  - dnssec_nsec3_salt_min_length( )
  - dnssec_nsec3_salt_max_length( )
  - dnssec_signature_expiration( )
  - dnssec_validation_enabled( )
  - dnssec_blacklist_enabled()
  - dnssec_dns64_enabled()
  - dnssec_nxdomain_enabled()
  - dnssec_rpz_enabled()
  - dnssec_trusted_keys( )
  - dnssec_zsk_algorithms( )
  - dnssec zsk rollover interval()

- dnssec_zsk_rollover_mechanism( )
- domains_to_capture_dns_queries( )
- dtc_dnssec_mode( )
- dtc_edns_prefer_client_subnet( )
- dtc_topology_ea_list( )
- email( )
- enable_blackhole( )
- enable_blacklist( )
- enable_capture_dns_queries( )
- enable_capture_dns_responses( )
- enable_client_subnet_recursive( )
- enable_client_subnet_forwarding( )
- enable_delete_associated_ptr( )
- enable_dns_health_check( )
- enable_dns64( )
- enable_dtc_dns_fall_through( )
- enable_excluded_domain_names( )
- enable_fixed_rrset_order_fqdns( )
- enable_ftc( )
- enable_gss_tsig( )
- enable_host_rrset_order( )
- enable_hsm_signing( )
- enable_query_rewrite( )
- excluded_domain_names( )
- expire_after( )
- file_transfer_setting( )
- filter_aaaa( )
- filter_aaaa_list( )
- forward_only( )
- forward_updates( )
- forwarders( )
- fixed_rrset_order_fqdns( )
- ftc_expired_record_ttl( )
- ftc_expired_record_timeout( )
- gss_tsig_keys( )
- host_name_restriction_policy( )
- lame_ttl( )
- logging_categories( )
- logging_facility( )
- max_cache_ttl( )
- max_ncache_ttl( )
- max_cached_lifetime( )
- member_secondary_notify( )
- negative_ttl( )
- next_secure_type( )
- notify_delay( )
- nxdomain_redirect( )
- nxdomain_redirect_addresses( )
- nxdomain_redirect_addresses_ipv6( )
- nxdomain_redirect_ttl( )
- nxdomain_log_query( )
- nxdomain_rulesets( )
- notify_source_port( )
- nsgroup_default( )
- nsgroups( )
- preserve_host_rrset_order_on_secondaries( )
- query_source_port( )
- query_rewrite_prefix( )
- query_rewrite_domain_names( )
- scavenging_settings( )
- record_name_policies( )
- recursive_query_list( )
- refresh_timer( )
- resolver_query_timeout( )
- response_rate_limiting( )
- restart_setting( )
- retry_timer( )
- rootNS( )
- serial_query_rate( )
- server_id_directive( )
- sortlist( )
- store_locally( )
- transfer_excluded_servers( )
- transfer_format( )

    - transfers_in( )
    - transfers_out( )
    - transfers_per_ns( )
    - rpz_disable_nsdname_nsip( )
    - rpz_drop_ip_rule_enabled( )
    - rpz_drop_ip_rule_min_prefix_length_ipv4( )
    - rpz_drop_ip_rule_min_prefix_length_ipv6( )
    - rpz_qname_wait_recurse( )
    - zone_deletion_double_confirm( )
- SAMPLE CODE
- AUTHOR
- SEE ALSO
- COPYRIGHT

NAME

Infoblox::Grid::DNS - DNS objects management and configuration

DESCRIPTION

This class supports DNS service management and configuration such as time-to-live (TTL) settings, zone transfers, queries, root name servers, dynamic updates, sort lists, Transaction Signatures (TSIG) for DNS and others, all at the grid level.

The service configurations of a grid are inherited by all members, zones, and networks unless you specifically override them for selected members, zones, and networks. For this reason, it is recommended that you configure services at the grid level before configuring member, zone and network services.

CONSTRUCTOR

The Infoblox::Grid::DNS object is automatically generated upon the successful installation of the Infoblox appliance. It does not require manual construction.

SESSION METHODS

This section describes all the methods in the Infoblox::Session module that you can apply to a grid DNS object.

Infoblox::Session->get( )

Use this method to retrieve the matching grid DNS object from the Infoblox appliance. See Infoblox::Session->get() for parameters and return values.

Key Reference

```
Apply the following attribute to get a specific grid DNS object:

 name - Required. A grid name in string format.
```

Example

```
my @retrieved_objs = $session->get(
    object => "Infoblox::Grid::DNS",
    name   => "Infoblox"
    );
```

Infoblox::Session->modify( )

Use this method to modify a grid DNS object in the Infoblox appliance. See Infoblox::Session->modify() for parameters and return values.

If you have applied an access control list to an object, you must restart services after you update the access control list for the changes to take effect. See Infoblox::Session->restart() method.

Example

```
#Use this method to modify the grid  DNS service.
$grid_dns->allow_recursive_query("true");
#Submit modification
my $response = $session->modify( $grid_dns );
```

METHODS

This section describes all the methods that you can use to configure and retrieve the attribute values of a grid DNS object.

allow_bulkhost_ddns( )

Use this method to set or retrieve the allow_bulkhost_ddns flag of a grid DNS object.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

Specify "true" to set the allow_bulkhost_ddns flag or "false" to deactivate/unset it. The default value is "false".

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get  the allow_bulkhost_ddns flag
my $allow_bulkhost_ddns = $grid_dns->allow_bulkhost_ddns();
#Modify allow bulkhost ddns flag
$grid_dns->allow_bulkhost_ddns("true");
```

allow_gss_tsig_zone_updates( )

Use this method to set or retrieve the allow_gss_tsig_zone_updates flag of a grid.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

Specify "true" to set the flag or "false" to deactivate/unset it. The default value is "false".

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get  the allow_gss_tsig_zone_updates flag
my $allow_gss_tsig_zone_updates = $grid_dns->allow_gss_tsig_zone_updates();
#Modify allow_gss_tsig_zone_updates flag
$grid_dns->allow_gss_tsig_zone_updates("true");
```

allow_query( )

Use this method to set or retrieve the list of allowed/denied queries for a Grid DNS object. The list specifies IP addresses and/or networks from which queries are allowed or denied. The appliance can also use TSIG keys to authenticate the queries.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

Valid value is an Infoblox::Grid::NamedACL object or an array reference that contains IPv4/IPv6 addresses, networks and/or Infoblox::DNS::TSIGKey objects. To deny queries, add "!" as a prefix to addresses and networks. Specify "any" to allow queries from any address. By default, queries are allowed from any address.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get allow query list
my $allow_query_list = $grid_dns->allow_query();
#Modify allow query list
$grid_dns->allow_query(["any"]);

#Specifying TSIG keys
my $tsig_key1 = $session->gen_tsig_key();
my $query_key = Infoblox::DNS::TSIGKey->new(
    name => "query_key1",
    key  => $tsig_key1
);
#Modify allow query list
```

```
 $grid_dns->allow_query(["10.0.0.10" , "20.0.0.0/24" ,"2001::123a","2003::222/64", "!"."30.0.0.30",
$query_key, "!"."40.10.0.0/24","!"."2004::125b","!"."2006::34bc"]);
 #Use Infoblox::Grid::NamedACL
 $grid_dns->allow_query($nacl);
```

allow_recursive_query( )

Use this method to set or retrieve the allow_recursive_query flag of a grid DNS object. If enabled and the Infoblox appliance receives a query for DNS data, it sends a recursive query to the configured internal root servers. If there are no internal root servers configured, the appliance then sends a recursive query to the Internet root servers.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

Specify "true" to set the allow_recursive_query flag or "false" to deactivate/unset it. The default value is "false".

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
 #Get allow recursive query flag
 my $allow_recursive_query = $grid_dns->allow_recursive_query();
 #Modify allow recursive query flag
 $grid_dns->allow_recursive_query("true");
```

allow_transfer( )

Use this method to set or retrieve the list of allowed/denied zone transfers for a Grid DNS object. The list specifies IP addresses and/or networks from which zone transfers are allowed or denied. It is also possible to specify the TSIG keys of clients that are allowed to perform zone transfers. TSIG keys are used to authenticate zone transfer requests and replies. The same key name and key value must be on the primary and secondary name servers for TSIG-authenticated zone transfers to occur.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

Valid value is an Infoblox::Grid::NamedACL object or an array reference that contains IPv4/IPv6 addresses, networks, and/or Infoblox::DNS::TSIGKey objects. To deny zone transfer, add "!" as a prefix to IP addresses and/or networks. Specify "any" to allow transfers from any IP address.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
 #Get allow transfer list
 my $allow_transfer_list = $grid_dns->allow_transfer();
 #Modify allow transfer list
 $grid_dns->allow_transfer(["any"]);

 #Specifying TSIG keys
 my $tsig_key1 = $session->gen_tsig_key();
 my $key1 = Infoblox::DNS::TSIGKey->new(
     name => "key1",
     key  => $tsig_key1
 );
 my $key2 = Infoblox::DNS::TSIGKey->new(
     name => "tsig_xfer",
     key  => ":2xCOMPAT"
     );
 #Modify allow transfer list
 $grid_dns->allow_transfer(["10.0.0.10", "20.0.0.0/24","2004::325f","2007::876c/64","!"."30.0.0.30",
$key1, $key2, "!"."40.10.0.0/24"]);
 #Use Infoblox::Grid::NamedACL
 $grid_dns->allow_transfer($nacl);
```

allow_update( )

Use this method to set or retrieve the list of allowed/denied dynamic updates for a grid DNS object. The list specifies from which IP addresses, networks or TSIG key authenticated hosts dynamic updates are allowed or denied.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

Valid value is an Infoblox::Grid::NamedACL object or an array reference that contains IPv4/IPv6 addresses, networks, and/or Infoblox::DNS::TSIGKey objects. To deny updates, add "!" as a prefix to IP addresses and/or networks. Specify "any" to allow updates from any IP address.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get allow update list
my $allow_update_list = $grid_dns->allow_update();
#Modify allow update list
$grid_dns->allow_update(["any"]);

#Creating a TSIG key object.
my $update_key1 = $session->gen_tsig_key();

my $up_key1 = Infoblox::DNS::TSIGKey->new(
    name => "update_key",
    key  => $update_key1
    );

#Modify allow update list
$grid_dns->allow_update(["10.0.0.10", "20.0.0.0/24","2008::129b","!"."30.0.0.30", $up_key1,
"!"."40.10.0.0/24","!"."2004::76bc/64"]);
#Use Infoblox::Grid::NamedACL
$grid_dns->allow_update($nacl);
```

anonymize_response_logging( )

Use this method to set or retrieve the flag that indicates whether the anonymization of captured DNS responses is enabled or disabled.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

Specify 'true' to enable anonymization of captured DNS response or 'false' to disable it. The default value is 'false'.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get anonymize_response_logging value
my $anonymize_response_logging = $object->anonymize_response_logging();

#Modify anonymize_response_logging value
$object->anonymize_response_logging('true');
```

attack_mitigation( )

Use this method to set or retrieve mitigation settings for DNS attacks.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

The valid value is an Infoblox::Grid::DNS::AttackMitigation object.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get attack_mitigation value
my $attack_mitigation   $grid_dns  attack_mitigation
```

```
                          =          ->                    ();
#Modify attack_mitigation
$grid_dns->attack_mitigation($attack_mitigation);
```

auto_blackhole( )

Use this method to set or retrieve the auto blackhole settings.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

The valid value is an Infoblox::Grid::DNS::AutoBlackHole object.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get auto_blackhole value
my $auto_blackhole = $grid_dns->auto_blackhole();
#Modify auto_blackhole
$grid_dns->auto_blackhole($auto_blackhole);
```

bind_check_names_policy( )

Use this method to set or retrieve the BIND check names policy, which indicates the action the appliance takes when it encounters host names that do not comply with the Strict Hostname Checking policy. This method applies only if the host name restriction policy is set to "Strict Hostname Checking".

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

The allowed values of the parameter are "fail", "warn" or undefined. This indicates the desired BIND behavior in the case of illegal hostnames. The default value is undefined, which means the appliance ignores illegal hostnames.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get bind_check_names_policy value
my $names_policy = $grid_dns->bind_check_names_policy();
#Modify bind_check_names_policy
$grid_dns->bind_check_names_policy("warn");
```

bind_hostname_directive( )

Use this method to set or retrieve the value of BIND 'hostname' directive.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

The valid values are "NONE" and "HOSTNAME". The default value is "NONE".

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get bind_hostname_directive
my $bind_hostname_directive = $grid_dns->bind_hostname_directive();
#Modify bind_hostname_directive
$grid_dns->bind_hostname_directive("HOSTNAME");
```

blackhole_list( )

Use this method to set or retrieve the blackhole list of a grid DNS object. This list specifies the IP addresses and networks from which the DNS queries are blocked.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

A valid value is an Infoblox::Grid::NamedACL object or an array reference that contains IPV4/IPv6 addresses and networks. To allow queries, add "!" as the prefix to the IP addresses and networks. Specify "any" to block DNS queries from any IP address.

Returns

When you specify a parameter and the modification succeeds, the method returns "true". When the operation fails, the method returns "false".

When you do not specify a parameter, the method returns the attribute value.

Example

```
#Get the blackhole list
my $blackhole_list = $grid_dns->blackhole_list();
#Modify the blackhole_list
$grid_dns->blackhole_list(["any"]);
#Modify the blackhole list
$grid_dns->blackhole_list(["10.0.0.10",
"20.0.0.0/24","2010::256/64","!"."2011::23cb/64","!"."30.0.0.30", "!"."40.10.0.0/24"]);
#Modify the blackhole list
$grid_dns->blackhole_list(undef);
#Use Infoblox::Grid::NamedACL
$grid_dns->blackhole_list($nacl);
```

blacklist_action( )

Use this method to specify the action to perform when a domain name matches the pattern defined in a rule that is specified by the blacklist_ruleset method.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

Valid values are "REDIRECT" or "REFUSE". The default value is "REFUSE".

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

Examples

```
#Get blacklist_action
my $blacklist_action = $grid_dns->blacklist_action();
#Modify blacklist_action
$grid_dns->blacklist_action("REDIRECT");
```

blacklist_log_query( )

Use this method to set or retrieve the flag that indicates whether blacklist redirection queries are logged.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

Specify "true" to enable logging, or "false" to disable it. The default value is "false".

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get blacklist_log_query
my $blacklist_log_query = $grid_dns->nxdomain_log_query( );
#Modify blacklist_log_query
$grid_dns->blacklist_log_query("true");
```

blacklist_redirect_addresses( )

Use this method to specify the IP addresses the appliance includes in the response it sends in place of a blacklisted IP address.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

The valid value is an array of IPv4 addresses. The default value is undefined.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get blacklist_redirect_addresses
my $blacklist_redirect_addresses = $grid_dns->blacklist_redirect_addresses( );
#Modify blacklist_redirect_addresses
$grid_dns->blacklist_redirect_addresses(["10.0.0.10", "10.0.0.20"]);
```

blacklist_redirect_ttl( )

Use this method to set or retrieve the TTL value of the synthetic DNS response that result from blacklist redirection.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

A 32-bit integer, from 0 to 4294967295, that represents the TTL in seconds. The default value is 60.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get blacklist_redirect_ttl
my $blacklist_redirect_ttl = $grid_dns->blacklist_redirect_ttl( );
#Modify blacklist_redirect_ttl
$grid_dns->blacklist_redirect_ttl(60);
```

blacklist_rulesets( )

Use this method to specify or retrieve Infoblox::DNS::Ruleset object names assigned at the grid level for blacklist redirection.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

The valid value is an array reference that contains the names of the Infoblox::DNS::Ruleset objects assigned to the grid. The names must be in string format.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get blacklist_rulesets
my $blacklist_rulesets = $grid_dns->blacklist_rulesets();
#Modify blacklist_rulesets
$grid_dns->blacklist_rulesets(["ruleset1", "ruleset2"]);
```

bulk_host_name_template( )

Use this method to set or retrieve the default bulk host name format of a grid DNS object.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

Name of a valid bulk host name format.

For instance, if the prefix of a bulk host is *info*, the IP address range is *213.19.32.133-213.19.32.134*, and the domain name is *infoblox.com*. If the bulk host name template "Three Octets" is set at the grid level, then the bulk host names will be *info-213-19-*

23-133.infoblox.com and *info-213-19-23-134.infoblox.com*.

If the bulk host name template specified is "CustomerTemplate"(*#3*#4), the bulk host names will be *info*023*133.infoblox.com* and *info-023-134.infoblox.com*.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get the bulk host name format
my $grid_template = $grid_dns->bulk_host_name_template();
#Modify bulk host name format
$grid_dns->bulk_host_name_template("Three Octets");
```

bulk_host_name_templates( )

Use this method to set or retrieve the list of bulk host name templates.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

Valid value is an array reference that contains Infoblox::Grid::DNS::BulkHostNameTemplate object(s).

There are four Infoblox predefined bulk host name templates.

Template Name Template Format

"Four Octets" -$1-$2-$3-$4

"Three Octets" -$2-$3-$4

"Two Octets" -$3-$4

"One Octet" -$4

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get bulk_host_name_templates
my $bulk_host_name_templates = $grid_dns->bulk_host_name_templates();

#Modify bulk_host_name_templates

#Constructing four out-of-box bulk host name templates
my $four_octets = Infoblox::Grid::DNS::BulkHostNameTemplate->new(
    template_name   => "Four Octets",
    template_format => '-$1-$2-$3-$4'
);
my $three_octets = Infoblox::Grid::DNS::BulkHostNameTemplate->new(
    template_name   => "Three Octets",
    template_format => '-$2-$3-$4'
);
my $two_octets = Infoblox::Grid::DNS::BulkHostNameTemplate->new(
    template_name   => "Two Octets",
    template_format => '-$3-$4'
);
my $one_octet = Infoblox::Grid::DNS::BulkHostNameTemplate->new(
    template_name   => "One Octet",
    template_format => '-$4'
);

#Creating a new bulk host name templates.
my $custom_template = Infoblox::Grid::DNS::BulkHostNameTemplate->new(
    template_name   => "Custom Template",
    template_format => '-#1-#2-#3-#4'
);

$grid_dns->bulk_host_name_templates([$custom_template, $four_octets, $three_octets, $two_octets,
$one_octet]);
```

capture_dns_queries_on_all_domains( )

Use this method to set or retrieve the flag that indicates whether the capturing of DNS queries for all domains is enabled or disabled.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

Specify 'true' to enable capturing of DNS queries for all domains or 'false' to disable it. The default value is 'false'.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get capture_dns_queries_on_all_domains value
my $capture_dns_queries_on_all_domains = $object->capture_dns_queries_on_all_domains();

#Modify capture_dns_queries_on_all_domains value
$object->capture_dns_queries_on_all_domains('true');
```

client_subnet_domains( )

Use this method to set or retrieve the list of zone domain names that are allowed or forbidden for EDNS client subnet (ECS) recursion.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

The valid value is an array of Infoblox::Grid::DNS::ClientSubnetDomain objects.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get client_subnet_domains value
my $value = $grid_dns->client_subnet_domains();

#Construct Infoblox::Grid::DNS::ClientSubnetDomain object
my $object = Infoblox::Grid::ClientSubnetDomain->new(
    domain     => 'domain.com',
    permission => 'DENY',
);

#Modify client_subnet_domains value
$grid_dns->client_subnet_domains([$object]);
```

client_subnet_ipv4_prefix_length( )

Use this method to set or retrieve the default IPv4 source prefix length used when sending queries with EDNS client subnet option.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

The valid value is an unsigned integer between 1 and 32.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get client_subnet_ipv4_prefix_length value
my $value = $object->client_subnet_ipv4_prefix_length();

#Modify client_subnet_ipv4_prefix_length value
$object->client_subnet_ipv4_prefix_length(30);
```

client_subnet_ipv6_prefix_length( )

Use this method to set or retrieve the default IPv6 source prefix length used when sending queries with EDNS client subnet option.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

The valid value is an unsigned integer between 1 and 128.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get client_subnet_ipv6_prefix_length value
my $value = $object->client_subnet_ipv6_prefix_length();

#Modify client_subnet_ipv6_prefix_length value
$object->client_subnet_ipv6_prefix_length(30);
```

copy_xfer_to_notify( )

Use this method to enable/disable copying of the allowed IPs from zone transfer list into also-notify statement in named.conf.

Parameter

Set the parameter to "true" to enable copying of the allowed IPs from zone transfer list into also-notify statement in named.conf. Set the parameter to "false" to disable it.

The default value of the parameter is "false".

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
# Get attribute value
my $value = $grid_dns->copy_xfer_to_notify();
# Modify attribute value
$grid_dns->copy_xfer_to_notify('true');
```

ddns_force_creation_timestamp_update( )

Use this method to set or retrieve the flag that indicates whether the creation timestamp of resource records (RR) should be updated when DDNS update happens even if there is no change to the RR.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

Specify 'true' to update the creation timestamp of RR on every DDNS update, and 'false' to update the creation timestamp only when RR changes during a DDNS update. The default is 'false'.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get ddns_force_creation_timestamp_update value
$value = $object->ddns_force_creation_timestamp_update();

#modify ddns_force_creation_timestamp_update value
$object->ddns_force_creation_timestamp_update('true');
```

ddns_restrict_patterns_list( )

Use this method to set or retrieve an unordered list of restriction patterns for an option of to restrict DDNS updates based on FQDN patterns.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

The valid value is an array of FQDNs in a string format.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#get ddns_restrict_patterns_list value
$ddns_restrict_patterns_list = $grid_dns->ddns_restrict_patterns_list();

#modify ddns_restrict_patterns_list value
$grid_dns->ddns_restrict_patterns_list(['foo.com', 'bar.com']);
```

ddns_restrict_patterns( )

Use this method to set or retrieve the flag which indicates whether an option to restrict DDNS update request based on FQDN patterns is enabled or disabled.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

Specify 'true' to enable DDNS update request restriction base on FQDN patterns and 'false to disable it. The default value is 'false'.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#get ddns_restrict_patterns value
$ddns_restrict_patterns = $grid_dns->ddns_restrict_patterns();

#modify ddns_restrict_patterns value
$grid_dns->ddns_restrict_patterns('true');
```

ddns_restrict_static( )

Use this method to set or retrieve the flag which indicates whether an option to restrict DDNS update request to resource records which are marked as 'STATIC' is enabled or disabled.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

Specify 'true' to enable DDNS update requests to resource records which are marked as 'STATIC' and 'false' to disable it. The default value is 'false'.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#get ddns_restrict_static value
$ddns_restrict_static = $grid_dns->ddns_restrict_static();

#modify ddns_restrict_static value
$grid_dns->ddns_restrict_static('true');
```

ddns_restrict_protected( )

Use this method to set or retrieve the flag which indicates whether an option to restrict DDNS update request to protected resource records is enabled or disabled.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

Specify 'true' to enable DDNS update requests to protected resource records and 'false' to disable it. The default value is 'false'.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#get ddns_restrict_protected value
$ddns_restrict_protected = $grid_dns->ddns_restrict_protected();

#modify ddns_restrict_protected value
$grid_dns->ddns_restrict_protected('true');
```

ddns_restrict_secure( )

Use this method to set or retrieve the flag which indicates whether DDNS update request for principal other than target resource record's principal is restricted.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

Specify 'true' to enable DDNS update requests restriction for principal other than target resource record's principal and 'false' to disable it. The default value is 'false'.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#get ddns_restrict_secure value
$ddns_restrict_secure = $grid_dns->ddns_restrict_secure();

#modify ddns_restrict_secure value
$grid_dns->ddns_restrict_secure('true');
```

ddns_principal_tracking( )

Use this method to set or retrieve the flag which indicates whether the DDNS principal track is enabled or disabled.

DDNS principal tracking includes putting them into resource record affected by update.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

Specify 'true' to enable DDNS principal tracking and 'false' to disable it. The default value is 'false'.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#get ddns_principal_tracking value
$ddns_principal_tracking = $grid_dns->ddns_principal_tracking();

#modify ddns_principal_tracking value
$grid_dns->ddns_principal_tracking('true');
```

ddns_principal_group( )

Use this method to set or retrieve the DDNS principal group.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

The valid value is an Infoblox::DNS::DDNS::PrincipalCluster::Group object.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#get ddns_principal_group value
$ddns_principal_group = $grid_dns->ddns_principal_group();

#modify ddns_principal_group value
$grid_dns->ddns_principal_group($principal_group);
```

default_ttl( )

Use this method to set or retrieve the default TTL value of a grid DNS object. This interval tells the secondary how long the data can be cached.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

Number for the TTL, in seconds. *Default* value is 1/2 of lease time or 28800 seconds.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get the default TTL
my $default_ttl = $grid_dns->default_ttl();
#Modify the default TTL
$grid_dns->default_ttl(28800);
```

disable_edns( )

Use this method to enable or disable EDNS0 support for queries that require recursive resolution.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

Specify "true" to disable EDNS0 or "false" to enable it. The default value is "false".

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get the double confirmation of zone deletion flag
my $disable_edns = $grid_dns->disable_edns();
#Modify disable_edns flag
$grid_dns->disable_edns("true");
```

dns_cache_acceleration_ttl( )

Use this method to set or retrieve the minimum TTL value, in seconds, that a DNS record must have in order for it to be cached by the DNS Cache Acceleration service.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

An integer from 1 to 65000 that represents the TTL in seconds. The default value is 1.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get dns_cache_acceleration_ttl
my $dns_cache_acceleration_ttl = $grid_dns->dns_cache_acceleration_ttl( );
#Modify dns_cache_acceleration_ttl
$grid_dns->dns_cache_acceleration_ttl(60);
```

dns_health_check_anycast_control( )

Use this method to set or retrieve the flag that indicates whether the anycast failure (BFD session down) is enabled on member failure or not.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

Specify 'true' to enable anycast failure on Member failure, and 'false' to disable it.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get dns_health_check_anycast_control value
my $value = $object->dns_health_check_anycast_control();

#Modify dns_health_check_anycast_control value
$object->dns_health_check_anycast_control('true');
```

dns_health_check_domain_list( )

Use this method to set or retrieve the list of domain names for the DNS health check.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

The valid value is an array reference that contains desired domain names in string format.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get dns_health_check_domain_list value
my $value = $object->dns_health_check_domain_list();

#Modify dns_health_check_domain_list value
$object->dns_health_check_domain_list(['foo.com', 'bar.net']);
```

dns_health_check_interval( )

Use this method to set or retrieve the time interval (in seconds) for DNS health check.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

The valid value is an unsigned integer between 10 and 21600 that represents number of seconds between two health checks.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get dns_health_check_interval value
my $value = $object->dns_health_check_interval();

#Modify dns_health_check_interval value
$object->dns_health_check_interval(2000);
```

dns_health_check_recursion_flag( )

Use this method to set or retrieve the flag that indicates whether the recursive DNS health check is enabled or not.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

Specify 'true' to enable recursive DNS health check queries, and 'false' to disable it.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get dns_health_check_recursion_flag value
my $value = $object->dns_health_check_recursion_flag();

#Modify dns_health_check_recursion_flag value
$object->dns_health_check_recursion_flag('true');
```

dns_health_check_retries( )

Use this method to set or retrieve the number of DNS health check retries.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

The valid value is an unsigned integer between 1 and 10.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get dns_health_check_retries value
my $value = $object->dns_health_check_retries();

#Modify dns_health_check_retries value
$object->dns_health_check_retries(2);
```

dns_health_check_timeout( )

Use this method to set or retrieve the DNS health check timeout interval (in seconds).

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

The valid value is an unsigned integer between 1 and 10.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get dns_health_check_timeout value
my $value = $object->dns_health_check_timeout();

#Modify dns_health_check_timeout value
$object->dns_health_check_timeout(10);
```

dns_query_capture_file_time_limit( )

Use this method to set or retrieve the time limit (in minutes) for the DNS query capture file.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

The valid value is unsigned integer between 1 and 10. The default value is 10.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get dns_query_capture_file_time_limit value
my $dns_query_capture_file_time_limit = $object->dns_query_capture_file_time_limit();

#Modify dns_query_capture_file_time_limit value
$object->dns_query_capture_file_time_limit('5');
```

dns64_groups( )

Use this method to set or retrieve the list of DNS64 synthesis groups.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

Valid value is an array reference that contains Infoblox::Grid::DNS::DNS64Group objects.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get the list of DNS64 synthesis groups
my $dns64_groups_list = $grid_dns->dns64_groups();

#Modify it
my $group = $session->get(
                        object => "Infoblox::Grid::DNS::DNS64Group"),
                        name => "groupname",
                        prefix => "groupprefix",
                      );
$grid_dns->dns64_groups([$group]);
```

dnssec_enabled( )

Use this method to enable/disable the DNS security extension.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

Set the parameter to "true" to enable the DNS security extension. Set the parameter to "false" to disable the feature.

The default value of the parameter is "false".

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get dnssec_enabled
my $dnssec_enabled = $grid_dns->dnssec_enabled();
#Modify dnssec_enabled
$grid_dns->dnssec_enabled("true");
```

dnssec_enable_ksk_auto_rollover( )

Use this method to enable automatic rollover for Key-signing key.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

Specify "true" to enable automatic rollover for Key-signing key or "false" to disable it. The default value is "false".

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get dnssec_enable_ksk_auto_rollover value
my $enable_ksk_auto_rollover = $grid_dns->dnssec_enable_ksk_auto_rollover();
#Modify dnssec_enable_ksk_auto_rollover value
$grid_dns->dnssec_enable_ksk_auto_rollover("true");
```

dnssec_expired_signatures_enabled( )

Use this method to control whether the DNS grid object accepts expired signatures.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

Set the parameter to "true" to accept expired signatures. Set the parameter to "false" to reject expired signatures.

The default value of the parameter is "false".

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get dnssec_expired_signatures_enabled
my $dnssec_expired_signatures_enabled = $grid_dns->dnssec_expired_signatures_enabled();
#Modify dnssec_expired_signatures_enabled
$grid_dns->dnssec_expired_signatures_enabled("true");
```

dnssec_ksk_algorithms( )

Use this method to set or retrieve the DNSSEC key-signing key algorithms of all the zones in the grid.

This property determines which algorithms to use when generating the keys for signing the DNSKEY records.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

Currently there are two major versions of the DNSSEC zone encryption mechanism. The difference between the two is in their implementation of the Next Secure (NSEC) records. The older version, which is referred to as NSEC, is based on RFCs 4033-4035. The newer version, which is referred to as NSEC3, is described in RFC 5155.

The valid value is a list containing Infoblox::DNS::DNSSecKeyAlgorithm objects.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
# Getting algorithm
my $algorithms = $grid_dns->dnssec_ksk_algorithms();

# Modifying algorithm
$grid_dns->dnssec_ksk_algorithm([$alg1, $alg2]);
```

dnssec_ksk_email_notification_enabled( )

Use this method to enable or disable email notifications for KSK related events.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

Specify "true" to enable email notification or "false" to disable it. The default value is "false".

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get dnssec_ksk_email_notification_enabled value
my $value = $grid_dns->dnssec_ksk_email_notification_enabled();
#Modify dnssec_ksk_email_notification_enabled value
$grid_dns->dnssec_ksk_email_notification_enabled("true");
```

dnssec_ksk_rollover_interval()

Use this method to set or retrieve the rollover interval of the KSK keys.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

A positive integer that specifies the number of seconds between KSK rollovers. Normally, this parameter should be larger than the ZSK rollover interval. The DNSSEC key-signing key rollover interval (in seconds) must be between 1 day and seconds to Jan 2, 2038.

The default value of the dnssec_ksk_rollover_interval parameter is 31536000 (1 year).

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
# Getting dnssec_ksk_rollover_interval
my $ksk_rollover = $grid_dns->dnssec_ksk_rollover_interval();

# Modifying dnssec_ksk_rollover_interval
$grid_dns->dnssec_ksk_rollover_interval(86400);
```

dnssec_ksk_rollover_notification_config( )

Use this method to determine the type of events users will be notified.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

The valid values are "NONE", "ALL" and "REQUIRE_MANUAL_INTERVENTION". Default value is undefined.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get dnssec_ksk_rollover_notification_config value
my $value = $grid_dns->dnssec_ksk_rollover_notification_config();
#Modify dnssec_ksk_rollover_notification_config value
$grid_dns->dnssec_ksk_rollover_notification_config("ALL");
```

dnssec_ksk_snmp_notification_enabled( )

Use this method to enable or disable SNMP notifiations for KSK related events.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

Specify "true" to enable SNMP notification or "false" to disable it. The default value is "false".

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get dnssec_ksk_snmp_notification_enabled value
my $value = $grid_dns->dnssec_ksk_snmp_notification_enabled();
#Modify dnssec_ksk_snmp_notification_enabled value
$grid_dns->dnssec_ksk_snmp_notification_enabled("true");
```

dnssec_negative_trust_anchors( )

Use this method to set or retrive the list of zones for which the appliance does not perform DNSSEC validation.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

The valid value is an array of strings that contains the list of zones for which the appliance does not perform DNSSEC validation.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get dnssec_negative_trust_anchors value
my $value = $grid_dns->dnssec_negative_trust_anchors();
#Modify dnssec_negative_trust_anchors value
$grid_dns->dnssec_negative_trust_anchors($zones);
```

dnssec_nsec3_iterations( )

Use this method to define the number of iterations for NSEC3 hashing.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

The valid value are integers between 0 and 65535. Default value is undefined.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get dnssec_nsec3_iterations value
my $iteration = $grid_dns->dnssec_nsec3_iterations();
#Modify dnssec_nsec3_iterations value
$grid_dns->dnssec_nsec3_iterations();
```

dnssec_nsec3_salt_min_length( )

Use this method to define the minimum length for the NSEC3 salt.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

The valid value are integers between 0 and 65535. Default value is undefined.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get dnssec_nsec3_salt_min_length value
my $salt_min_length = $grid_dns->dnssec_nsec3_salt_min_length();
#Modify dnssec_nsec3_salt_min_length value
$grid_dns->dnssec_nsec3_salt_min_length();
```

dnssec_nsec3_salt_max_length( )

Use this method to define the maximum length for the NSEC3 salt.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

The valid value are integers between 0 and 65535. Default value is undefined.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get dnssec_nsec3_salt_max_length value
my $salt_max_length = $grid_dns->dnssec_nsec3_salt_max_length();
#Modify dnssec_nsec3_salt_max_length value
$grid_dns->dnssec_nsec3_salt_max_length();
```

dnssec_signature_expiration( )

Use this method to set or retrieve the expiration time for DNSSEC signatures.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

The signature expiration time in seconds. The minimum value is 86400 (1 day), the maximum value is the number of seconds from the time you set it to January 2038.

The default value of dnssec_signature_expiration parameter is 2592000 seconds (i.e. 30 days).

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
# Getting dnssec_signature_expiration
my $signature_expiration = $grid_dns->dnssec_signature_expiration();

# Modifying dnssec_signature_expiration
$grid_dns->dnssec_signature_expiration(31536000);
```

dnssec_validation_enabled( )

Use this method to enable and disable DNS security validation (for the grid appliances working as clients of secure zones).

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

Set the parameter to "true" to enable the validation. Set to "false" to disable the validation.

The default value of the parameter is "true".

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get dnssec_validation_enabled
my $dnssec_validation_enabled = $grid_dns->dnssec_validation_enabled();
#Modify dnssec_validation_enabled
$grid_dns->dnssec_validation_enabled("false");
```

dnssec_blacklist_enabled()

Use this method to enable and disable blacklist rules for DNSSEC-enabled clients.

Parameter

Set the parameter to "true" to apply blacklist rules for DNSSEC-enabled clients. Set to "false" to disable them.

The default value of the parameter is "false".

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get dnssec_blacklist_enabled
my $dnssec_blacklist_enabled = $grid_dns->dnssec_blacklist_enabled();
#Modify dnssec_blacklist_enabled
$grid_dns->dnssec_blacklist_enabled("true");
```

dnssec_dns64_enabled()

Use this method to enable and disable DNS64 groups for DNSSEC-enabled clients.

Parameter

Set the parameter to "true" to apply DNS64 groups for DNSSEC-enabled clients. Set to "false" to disable them.

The default value of the parameter is "false".

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get dnssec_dns64_enabled
my $dnssec_dns64_enabled = $grid_dns->dnssec_dns64_enabled();
#Modify dnssec_dns64_enabled
$grid_dns->dnssec_dns64_enabled("true");
```

dnssec_nxdomain_enabled()

Use this method to enable and disable NXDOMAIN rules for DNSSEC-enabled clients.

Parameter

Set the parameter to "true" to apply NXDOMAIN rules for DNSSEC-enabled clients. Set to "false" to disable them.

The default value of the parameter is "false".

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get dnssec_nxdomain_enabled
my $dnssec_nxdomain_enabled = $grid_dns->dnssec_nxdomain_enabled();
#Modify dnssec_nxdomain_enabled
$grid_dns->dnssec_nxdomain_enabled("true");
```

dnssec_rpz_enabled()

Use this method to enable and disable RPZ policies for DNSSEC-enabled clients.

Parameter

Set the parameter to "true" to apply RPZ policies for DNSSEC-enabled clients. Set to "false" to disable them.

The default value of the parameter is "false".

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get dnssec_rpz_enabled
my $dnssec_rpz_enabled = $grid_dns->dnssec_rpz_enabled();
#Modify dnssec_rpz_enabled
$grid_dns->dnssec_rpz_enabled("true");
```

dnssec_trusted_keys( )

Use this method to specify the list of trusted keys for the DNSSEC feature.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

Array reference that contains a list of Infoblox::DNS::DnssecTrustedKey objects.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns a reference to a list of the trusted keys.

Example

```
#Get dnssec_trusted_keys
my $dnssec_trusted_keys = $grid_dns->dnssec_trusted_keys();
#Modify dnssec_trusted_keys
my $key1=Infoblox::DNS::DnssecTrustedKey->new(
   fqdn => "myzone.com",
   algorithm => "RSAMD5",
   key       => "c2ltcGxlIGtleQo="
   );
$grid_dns->dnssec_trusted_keys([$key1]);
```

dnssec_zsk_algorithms( )

Use this method to set or retrieve the DNSSEC zone-signing key algorithms of all the zones in the grid.

This property determines which algorithms to use when generating the keys for signing the DNSKEY records.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

Currently there are two major versions of the DNSSEC zone encryption mechanism. The difference between the two is in their implementation of the Next Secure (NSEC) records. The older version, which is referred to as NSEC, is based on RFCs 4033-4035. The newer version, which is referred to as NSEC3, is described in RFC 5155.

The valid value is a list containing Infoblox::DNS::DNSSecKeyAlgorithm objects.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
# Getting algorithms
my $algorithms = $grid_dns->dnssec_zsk_algorithms();

# Modifying algorithms
$grid_dns->dnssec_zsk_algorithm([$alg1, $alg2]);
```

dnssec_zsk_rollover_interval()

Use this method to set or retrieve the zone-signing key rollover interval.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

A positive integer that specifies the number of seconds between ZSK rollovers. Normally, this parameter should be smaller than the KSK rollover interval. The DNSSEC zone-signing key over interval (in seconds) must be between 1 day and the number of seconds from present to Jan 2038.

The default value of the dnssec_zsk_rollover_interval parameter is 2592000 seconds (30 days).

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
# Getting dnssec_zsk_rollover_interval
my $zsk_rollover = $grid_dns->dnssec_zsk_rollover_interval();

# Modifying dnssec_ksk_rollover_interval
$grid_dns->dnssec_zsk_rollover_interval(86400);
```

dnssec_zsk_rollover_mechanism( )

Use this method to set or retrieve the ZSK rollover mechanism type.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

The valid values are "DOUBLE_SIGN" and "PRE_PUBLISH". Default value is undefined.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get dnssec_zsk_rollover_mechanism value
my $value = $grid_dns->dnssec_zsk_rollover_mechanism();
#Modify dnssec_zsk_rollover_mechanism value
$grid_dns->dnssec_zsk_rollover_mechanism("DOUBLE_SIGN");
```

domains_to_capture_dns_queries( )

Use this method to set or retrieve the list of domains to capture DNS queries.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

The valid value is an array reference that contains desired domain names in a string format.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get domains_to_capture_dns_queries value
my $domains_to_capture_dns_queries = $object->domains_to_capture_dns_queries();

#Modify domains_to_capture_dns_queries value
$object->domains_to_capture_dns_queries(['domain1.com', 'domain2.com']);
```

dtc_dnssec_mode( )

Use this method to set or retrieve the DTC DNSSEC operation mode.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

The valid values are 'SIGNED' and 'UNSIGNED'. The default value is 'UNSIGNED'.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get dtc_dnssec_mode value
my $dtc dnssec mode   $object  dtc dnssec mode
```

```
                        =       ->              ();
```

```
#Modify dtc_dnssec_mode value
$object->dtc_dnssec_mode('UNSIGNED');
```

dtc_edns_prefer_client_subnet( )

Use this method to set or retrieve the flag that indicates whether the use of the edns-client-subnet option for DTC is enabled or disabled.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

Specify 'true' to enable use of the edns-client-subnet option for DTC or 'false' to disable it.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get dtc_edns_prefer_client_subnet value
my $dtc_edns_prefer_client_subnet = $object->dtc_edns_prefer_client_subnet();

#Modify dtc_edns_prefer_client_subnet value
$object->dtc_edns_prefer_client_subnet('true');
```

dtc_topology_ea_list( )

Use this method to set or retrieve the DTC topology extensible attribute definition list. When configuring a DTC topology, users may configure classification as either "Geographic" or "Extensible Attributes". Selecting "Extensible Attributes" will replace supported MaxMind database labels (Continent, Country, Subdivision, City) with the names of the selected EA types and provide values of those extensible attributes extracted from DHCP Network Container, Network and Range objects.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

The valid value is an Infoblox::Grid::ExtensibleAttributeDef object name.

Note that the number of extensible attribute definitions is limited to 4. You can select only string, enum, and integer extensible attribute definitions.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get dtc_topology_ea_list value
my $dtc_topology_ea_list = $object->dtc_topology_ea_list();

#Modify dtc_topology_ea_list value
$object->dtc_topology_ea_list(['Site', 'State']);
```

email( )

Use this method to set or retrieve the email address of a grid DNS object.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

String with email address.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get email address
my $email = $grid_dns->email();
```

```
#Modify email address
$grid_dns->email("admin\@infoblox.com");
```

enable_blackhole( )

Use this method to set or retrieve the enable_blackhole flag of a grid DNS object. When enabled, the Infoblox appliance does not respond to DNS queries from clients that are in the configured blackhole_list.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

Specify "true" to set the enable_blackhole flag or "false" to deactivate and unset it. The default value is "false".

Returns

When you specify a parameter and the modification succeeds, the method returns "true". When the operation fails, the method returns "false".

When you do not specify a parameter, the method returns the attribute value.

Example

```
#Get enable_blackhole flag
my $enable_blackhole = $grid_dns->enable_blackhole();
#Modify enable_blackhole recursive query flag
$grid_dns->enable_blackhole("true");
```

enable_blacklist( )

Use this method to enable or disable a blacklist globaly.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

Specify "true" to set the enable_blacklist flag or "false" to deactivate/unset it. The default is "false".

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Getting enable_blacklist
my $enable_blacklist = $grid_dns->enable_blacklist( );
#Modifying enable_blacklist
$grid_dns->enable_blacklist("true");
```

enable_capture_dns_queries( )

Use this method to set or retrieve the flag that indicates whether the capture of DNS queries is enabled or disabled.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

Specify 'true' to enable the capture of DNS queries or 'false' to disable it. The default value is 'false'.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get enable_capture_dns_queries value
my $enable_capture_dns_queries = $object->enable_capture_dns_queries();

#Modify enable_capture_dns_queries value
$object->enable_capture_dns_queries('true');
```

enable_capture_dns_responses( )

Use this method to set or retrieve the flag that indicates whether the capture of DNS responses is enabled or disabled.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

Specify 'true' to enable the capture of DNS responses or 'false' to disable it. The default value is 'false'.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get enable_capture_dns_responses value
my $enable_capture_dns_responses = $object->enable_capture_dns_responses();

#Modify enable_capture_dns_responses value
$object->enable_capture_dns_responses('true');
```

enable_client_subnet_recursive( )

Use this method to set or retrieve the flag that indicates whether adding EDNS client subnet options in recursive resolutions is enabled or not.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

Specify 'true' to enable adding EDNS client subnet options in recursive resolutions or 'false' to disable it. The default value is 'false'.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get enable_client_subnet_recursive value
my $value = $object->enable_client_subnet_recursive();

#Modify enable_client_subnet_recursive value
$object->enable_client_subnet_recursive('true');
```

enable_client_subnet_forwarding( )

Use this method to set or retrieve the flag that indicates whether forwarding EDNS client subnet options to upstream servers is enabled or not.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

Specify 'true' to enable forwarding of EDNS client subnet options to upstream servers or 'false' to disable it. The default value is 'false'.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get enable_client_subnet_forwarding value
my $value = $object->enable_client_subnet_forwarding();

#Modify enable_client_subnet_forwarding value
$object->enable_client_subnet_forwarding('true');
```

enable_delete_associated_ptr( )

Use this method to enable or disable the ability to automatically remove associated PTR records while deleting A or AAAA records.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

Specify "true" to enable the ability to automatically remove associated PTR records while deleting A or AAAA records or "false" to disable it. The default value is "false".

Returns

When you specify a parameter and the modification succeeds, the method returns "true". When the operation fails, the method returns "false".

When you do not specify a parameter, the method returns the attribute value.

Example

```
#Get enable_delete_associated_ptr flag
my $enable_delete_associated_ptr = $grid_dns->enable_delete_associated_ptr();
#Modify enable_delete_associated_ptr flag
$grid_dns->enable_delete_associated_ptr("true");
```

enable_dns_health_check( )

Use this method to set or retrieve the flag that indicates whether the DNS health check is enabled or not.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

Specify 'true' to enable DNS health check, and 'false' to disable it.

Returns

When you specify a parameter and the modification succeeds, the method returns "true". When the operation fails, the method returns "false".

When you do not specify a parameter, the method returns the attribute value.

Example

```
#Get enable_dns_health_check value
my $value = $object->enable_dns_health_check();

#Modify enable_dns_health_check value
$object->enable_dns_health_check('true');
```

enable_dns64( )

Use this method to enable/disable DNS64 support.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

Specify "true" to set the enable_dns64 flag or "false" to deactivate and unset it. The default value is "false".

Returns

When you specify a parameter and the modification succeeds, the method returns "true". When the operation fails, the method returns "false".

When you do not specify a parameter, the method returns the attribute value.

Example

```
#Get enable_dns64 flag
my $enable_dns64 = $grid_dns->enable_dns64();
#Modify enable_dns64 recursive query flag
$grid_dns->enable_dns64("true");
```

enable_dtc_dns_fall_through( )

Use this method to set or retrieve the flag that indicates whether the DNS traffic control (DTC) to DNS fall through is enabled or not.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

Specify 'true' to enable DTC to DNS fall through or 'false' to disable it.

Returns

When you specify a parameter and the modification succeeds, the method returns "true". When the operation fails, the method

returns "false".

When you do not specify a parameter, the method returns the attribute value.

Example

```
#Get enable_dtc_dns_fall_through value
my $value = $grid_dns->enable_dtc_dns_fall_through();

#Modify enable_dtc_dns_fall_through value
$grid_dns->enable_dtc_dns_fall_through('false');
```

enable_excluded_domain_names( )

Use this method to set or retrieve the flag that indicates whether the excluding domain names from captured DNS query and response is enabled or disabled.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

Specify 'true' to enable the excluding domain names from captured DNS query and response or 'false' to disable it. The default value is 'false'.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get enable_excluded_domain_names value
my $enable_excluded_domain_names = $object->enable_excluded_domain_names();

#Modify enable_excluded_domain_names value
$object->enable_excluded_domain_names('true');
```

enable_fixed_rrset_order_fqdns( )

Use this method to enable or disable fixed RRset order for specified FQDNs.

The enable_fixed_rrset_order_fqdns attribute can be specified explicitly. It is also set implicitly when fixed_rrset_order_fqdns is set to a defined value.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

Specify "true" to enable fixed RRset order for specified FQDNs or "false" to disable it.

Returns

When you specify a parameter and the modification succeeds, the method returns "true". When the operation fails, the method returns "false".

When you do not specify a parameter, the method returns the attribute value.

Example

```
#Get enable_fixed_rrset_order_fqdns flag
my $enable_fixed_rrset_order_fqdns = $grid_dns->enable_fixed_rrset_order_fqdns();
#Modify enable_fixed_rrset_order_fqdns flag
$grid_dns->enable_fixed_rrset_order_fqdns("true");
```

enable_ftc( )

Use this method to set or retrieve the flag that indicates whether the Fault Tolerant Caching (FTC) feature is enabled or not.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

Specify 'true' to enable the FTC feature or 'false' to disable it. The default value is 'false'.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get enable_ftc
my $enable_ftc = $grid_dns->enable_ftc();
#Modify enable_ftc
$grid_dns->enable_ftc('true');
```

enable_gss_tsig( )

Use this method to enable/disable the appliance to receive GSS-TSIG authenticated updates from DHCP clients. You must import the keytab file before you enable this feature. See Infoblox::Session->import_data().

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

Specify "true" to enable the the appliance to receive GSS-TSIG authenticated updates or "false" to disable it. The default value is "false".

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get enable_gss_tsig value
$my $enable_gss_tsig = $grid_dns->enable_gss_tsig();

#Modify enable_gss_tsig value

$session->import_data(
     type        => 'keytab',
     path        => '/infoblox/tests/datasets/gss-tsig-gs.keytab',
     service     => 'dns',
     virtual_ip  => '192.168.1.2');

$grid_dns->enable_gss_tsig('true');
```

enable_host_rrset_order( )

Use this method to set or retrieve the grid-level enable_host_rrset_order value.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

Specify "true" to set the enable_host_rrset_order flag or "false" to deactivate and unset it. The default value is "false".

Returns

When you specify a parameter and the modification succeeds, the method returns "true". When the operation fails, the method returns "false".

When you do not specify a parameter, the method returns the attribute value.

Example

```
# Get enable_host_rrset_order
my $host_rrset_order = $grid_dns->enable_host_rrset_order();
# Set enable_host_rrset_order
$grid_dns->enable_host_rrset_order("true");
```

enable_hsm_signing( )

Use this method to enable/disable the HSM for DNSSEC key generation and signing. To enable this feature, an HSM group must be configured with at least one enabled HSM.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

Specify "true" to enable HSM signing or "false" to disable it. The default value is "false".

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get enable_hsm_signing
$grid_dns->enable_hsm_signing( );
#Modify enable_hsm_signing
$grid_dns->enable_hsm_signing("true");
```

enable_query_rewrite( )

Use this method to enable or disable DNS query rewrite.

Parameter

Specify "true" to enable DNS query rewrite or "false" to disable it. The default value is "false".

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get enable_query_rewrite
$grid_dns->enable_query_rewrite( );
#Modify enable_query_rewrite
$grid_dns->enable_query_rewrite("true");
```

excluded_domain_names( )

Use this method to set or retrieve the list of domains that are excluded from DNS response and query capturing.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

The valid value is an array reference that contains desired domain names in a string format.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get excluded_domain_names value
my $excluded_domain_names = $object->excluded_domain_names();

#Modify excluded_domain_names value
$object->excluded_domain_names(['domain1.com', 'domain2.com']);
```

expire_after( )

Use this method to set or retrieve the expiration time of a grid DNS object. If the secondary DNS server fails to contact the primary server for the specified interval, the secondary server stops giving out answers about the zone because the zone data is too old to be useful.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

Number with the seconds for the expiration time. *Default* value is 2419200 seconds (4 weeks).

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get the expiration time
my $expire_after = $grid_dns->expire_after();
#Modify the expiration time
$grid_dns->expire_after(2419200);
```

file_transfer_setting( )

Use this method to set or retrieve the DNS capture file transfer settings.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

The valid value is an Infoblox::Grid::DNS::FileTransferSetting object.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get file_transfer_setting value
my $file_transfer_setting = $object->file_transfer_setting();

#Modify file_transfer_setting value
$object->file_transfer_setting($file_transfer);
```

filter_aaaa( )

Use this method to set or retrieve the type of AAAA filtering for this grid DNS object.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

Valid values are "YES", "NO" and "BREAK_DNSSEC". The default value is "NO".

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get the filter aaaa type
my $filter_aaaa = $grid_dns->filter_aaaa();
#Modify the filtering
$grid_dns->filter_aaaa("YES");
```

filter_aaaa_list( )

Use this method to set or retrieve the list of IPv4 addresses and networks from which queries are received. AAAA filtering is applied to these addresses.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

A valid value is an Infoblox::Grid::NamedACL object or an array reference that contains IPv4 addresses and networks. To return AAAA records for queries received from an IPv4 client, add "!" as the prefix to the IP addresses and networks. Specify "any" to allow AAAA records to be returned for any query received by an IPv4 client.

Returns

When you specify a parameter and the modification succeeds, the method returns "true". When the operation fails, the method returns "false".

When you do not specify a parameter, the method returns the attribute value.

Example

```
#Get the filter_aaaa list
my $filter_aaaa_list = $grid_dns->filter_aaaa_list();
#Modify the filter_aaaa_list
$grid_dns->filter_aaaa_list(["any"]);
#Modify the filter_aaaa list
$grid_dns->filter_aaaa_list(["10.0.0.10", "20.0.0.0/24", "!"."30.0.0.30", "!"."40.10.0.0/24"]);
#Modify the filter_aaaa list
$grid_dns->filter_aaaa_list(undef);
#Use Infoblox::Grid::NamedACL
```

```
$grid_dns->filter_aaaa_list($nacl);
```

forward_only( )

Use this method to set or retrieve the forward only flag of a grid DNS object. On enabling, the Infoblox appliance will send queries to forwarders only, and not to internal or Internet root servers.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

Specify "true" to set the forward_only flag or "false" to deactivate/unset it. The default value is "false".

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get the forward only flag
my $forward_only = $grid_dns->forward_only();
#Modify the forward only
$grid_dns->forward_only("true");
```

forward_updates( )

Use this method to set or retrieve the forward_updates flag of a grid DNS object. This enables a secondary server to receive updates from the DHCP server, and then forward them to the primary DNS server.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

Specify "true" to set the forward_updates flag or "false" to deactivate/unset it. The default value is "false".

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get the forward updates flag
my $forward_updates = $grid_dns->forward_updates();
#Modify the forward updates flag
$grid_dns->forward_updates("true");
```

forwarders( )

Use this method to set or retrieve the forwarders list of a grid DNS object. A forwarder is essentially a name server to which other name servers first send all of their off-site queries. The forwarder builds up a cache of information, avoiding the necessity for the other name servers to send queries off-site.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

The valid value is an array reference that contains IP address(es).

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get the forwarders list
my $forwarders_list = $grid_dns->forwarders();
#Modify the forwarders list
$grid_dns->forwarders(["2.2.2.2", "3.3.3.3"]);
```

fixed_rrset_order_fqdns( )

Use this method to set or retrieve configuration for fixed RRSET order for specified FQDNs.

Setting this method to a defined value implicitly sets the enable_fixed_rrset_order_fqdns method to "true". Setting the parameter to undefined causes the appliance to disable fixed RRset order of FQDNs and automatically resets the enable_fixed_rrset_order_fqdns method to "false".

Note that when fixed_rrset_order_fqdns is set to a defined value and enable_fixed_rrset_order_fqdns is set to "false", the last operation takes precedence. Thus the sequence $object->fixed_rrset_order_fqdns($fqdns); $object->enable_fixed_rrset_order_fqdns("false"); will set enable_fixed_rrset_order_fqdns to "false", and the sequence $object->enable_fixed_rrset_order_fqdns("false"); $object->fixed_rrset_order_fqdns($fqdns); will result in enable_fixed_rrset_order_fqdns="true".

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

The valid value is an array reference that contains Infoblox::Grid::DNS::FixedRRSetOrderFQDN objects.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get the fixed_rrset_order_fqdns list
my $fixed_rrset_order_fqdns = $grid_dns->fixed_rrset_order_fqdns();
#Modify the fixed_rrset_order_fqdns list
$grid_dns->fixed_rrset_order_fqdns([$fqdn1, $fqdn2]);
```

ftc_expired_record_ttl( )

Use this method to set or retrieve the TTL value of the expired Fault Tolerant Caching (FTC) record in DNS responses.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

A 32-bit integer, from 0 to 4294967295, that represents the TTL in seconds. The default value is 5.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get ftc_expired_record_ttl
my $ftc_expired_record_ttl = $grid_dns->ftc_expired_record_ttl( );
#Modify ftc_expired_record_ttl
$grid_dns->ftc_expired_record_ttl(60);
```

ftc_expired_record_timeout( )

Use this method to set or retrieve the timeout value after which the expired Fault Tolerant Caching (FTC) record becomes stale and is no longer valid.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

A 32-bit integer, from 0 to 4294967295, that represents the timeout in seconds. The default value is 86400.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get ftc_expired_record_timeout
my $ftc_expired_record_timeout = $grid_dns->ftc_expired_record_timeout( );
#Modify ftc_expired_record_timeout
$grid_dns->ftc_expired_record_timeout(3600);
```

gss_tsig_keys( )

Use this method to set or retrieve the list of Kerberos keys for a Grid DNS object.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

The valid value is an array reference that contains Infoblox::Grid::KerberosKey objects.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get the Kerberos keys list
my $kerberos_keys_list = $grid_dns->gss_tsig_keys();
#Modify the Kerberos keys list
my $kerberos_key1 = Infoblox::Cluster::KerberosKey->new(
       principal => 'host/appliance.ivan.test1@LADYBUG.LOCAL',
       );
my $kerberos_key2 = Infoblox::Cluster::KerberosKey->new(
       principal => 'host/appliance.ivan.test2@LADYBUG.LOCAL',
       );
$grid_dns->gss_tsig_keys([$kerberos_key1, $kerberos_key2]);
```

host_name_restriction_policy( )

Use this method to set or retrieve the default host name restriction policy of a grid DNS object.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

String with the name of predefined record policies or user-defined record policies. The default value is the predefined record policy "Allow Underscore".

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get host name restriction policy
my $host_name_restriction_policy = $grid_dns->host_name_restriction_policy();
#Modify host name restriction policy
$grid_dns->host_name_restriction_policy("Allow Any");
```

lame_ttl( )

Use this method to set or retrieve the number of seconds to cache lame delegations or lame servers.

Parameter

The number of seconds to cache lame delegations or lame servers. The default value is 600.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get lame_ttl
my $lame_ttl = $grid_dns->lame_ttl();
#Modify lame_ttl
$grid_dns->lame_ttl(700);
```

logging_categories( )

Use this method to set or retrieve the logging categories. It is possible to specify several BIND logging message categories to be captured by syslog.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

Valid value is an array reference that contains one or more of the following values:

"general" - Records the BIND messages that are not specifically classified.

"config" - Records the parsing messages of the configuration file.

"dnssec" - Records the DNSSEC-signed responses.

"network" - Records the network operation messages.

"queries" - Records the query messages.

"query_rewrite" - Records rewritten queries.

"responses" - Records the response messages.

"security" - Records the approved and denied requests.

"xfer-in" - Records zone transfer messages from the remote name servers to the appliance.

"xfer-out" - Records zone transfer messages from the Infoblox appliance to remote name servers.

"update" - Records the dynamic update instances.

"resolver" - Records the DNS resolution instances, including recursive queries from resolvers.

"notify" - Records the asynchronous zone change notification messages.

"lame_servers" - Records bad delegation instances.

"database" - Records BIND internal database processes.

"client" - Records client requests.

"rpz" - Records RPZ messages.

"dtc_gslb" - Records the DTC GSLB activity.

"dtc_health" - Records the DTC health monitoring.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get the logging categories list
my $logging_categories_list = $grid_dns->logging_categories();
#Modify the logging categories list
$grid_dns->logging_categories(["general", "database", "queries"]);
```

logging_facility( )

Use this method to set or retrieve the logging facility of a grid DNS object.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

Text with the facility name. Valid values are: "daemon", "local0", "local1", "local2", "local3", "local4", "local5", "local6", and "local7".

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get the logging facility
my $logging_facility = $grid_dns->logging_facility();
#Modify the logging facility
$grid_dns->logging_facility("daemon");
```

max_cache_ttl( )

Use this method to set or retrieve the maximum time (in seconds) for which the server will cache positive answers.

Parameter

The maximum time is specified in seconds. The default value is 604800.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get max_cache_ttl
my $max_cache_ttl = $grid_dns->max_cache_ttl();
#Modify max_cache_ttl
$grid_dns->max_cache_ttl(400);
```

max_ncache_ttl( )

Use this method to set or retrieve the maximum time (in seconds) for which the server will cache negative (NXDOMAIN) responses.

Parameter

The maximum time is specified in seconds. The default value is 10800. The maximum allowed value is 604800.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get max_ncache_ttl
my $max_ncache_ttl = $grid_dns->max_ncache_ttl();
#Modify max_ncache_ttl
$grid_dns->max_ncache_ttl(400);
```

max_cached_lifetime( )

Use this method to set or retrieve maximum time in seconds a DNS response can be stored in the hardware acceleration cache.

Parameter

Valid values are unsigned integer between 60 and 86400, inclusive. Default value is 86400.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get max_cached_lifetime
my $max_cached_lifetime = $grid_dns->max_cached_lifetime();
#Modify max_cached_lifetime
$grid_dns->max_cached_lifetime(400);
```

member_secondary_notify( )

Use this method to set or retrieve the member_secondary_notify flag of a grid DNS object. When enabled, this feature allows grid members that are authoritative secondary servers to send notification messages to external name servers, if the grid member that is primary for a zone fails or loses connectivity.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

Specify "true" to set member_secondary_notify flag or "false" to deactivate/unset it. The default value is "true".

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get the member secondary notification flag
my $member_secondary_notify = $grid_dns->member_secondary_notify();
#Modify the member secondary notification flag
$grid_dns->member_secondary_notify("false");
```

negative_ttl( )

Use this method to set or retrieve the negative TTL value of a grid DNS object. This interval tells the secondary how long data can be cached for "Does Not Respond" responses.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

Number for the negative TTL, in seconds. *Default* value is 900 seconds.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get the negative ttl value
my $negative_ttl = $grid_dns->negative_ttl();
#Modify the negative ttl value
$grid_dns->negative_ttl(800);
```

next_secure_type( )

Use this method to set or retrieve resource record type.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

Valid value is 'NSEC' or 'NSEC3'.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get the next_secure_type value
my $next_secure_type = $grid_dns->next_secure_type();
#Modify the next_secure_type value
$grid_dns->next_secure_type('NSEC3');
```

notify_delay( )

Use this method to specify with how many seconds of delay the notify messages are sent to secondaries.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

Valid values are unsigned integer between 5 and 86400, inclusive. Default value is 5.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Getting notify_delay
my $notify_delay = $griddns->notify_delay( );
#Modifying notify_delay
$griddns->notify_delay(5);
```

nxdomain_redirect( )

Use this method to enable or disable NXDOMAIN redirection.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

Specify "true" to set the nxdomain_redirect flag or "false" to deactivate/unset it. The default value is "false".

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Getting nxdomain_redirect
my $nxdomain_redirect = $griddns->nxdomain_redirect( );
#Modifying nxdomain_redirect
$griddns->nxdomain_redirect("true");
```

nxdomain_redirect_addresses( )

Use this method to set the NXDOMAIN redirection addresses.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

The valid value is an array of IPv4 addresses. The default value is undefined.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get nxdomain_redirect_addresses
my $nxdomain_redirect_addresses = $griddns->nxdomain_redirect_addresses( );
#Modify nxdomain_redirect_addresses
$griddns->nxdomain_redirect_addresses(["10.0.0.10", "10.0.0.20"]);
```

nxdomain_redirect_addresses_ipv6( )

Use this method to set the NXDOMAIN redirection IPv6 addresses.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

The valid value is an array of IPv6 addresses. The default value is undefined.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get nxdomain_redirect_addresses_ipv6
my $nxdomain_redirect_addresses_ipv6 = $griddns->nxdomain_redirect_addresses_ipv6( );
#Modify nxdomain_redirect_addresses_ipv6
$griddns->nxdomain_redirect_addresses_ipv6(["2001:db8::1", "2002:ffff::1"]);
```

nxdomain_redirect_ttl( )

Use this method to set or retrieve the TTL value of synthetic DNS responses that result from NXDOMAIN redirection.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

A 32-bit integer, from 0 to 4294967295, that represents the TTL in seconds. The default value is 60.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get nxdomain_redirect_ttl
my $nxdomain_redirect_ttl = $griddns->nxdomain_redirect_ttl( );
#Modify nxdomain_redirect_ttl
$griddns->nxdomain_redirect_ttl(60);
```

nxdomain_log_query( )

Use this method to set or retrieve the flag that indicates whether NXDOMAIN redirection queries are logged.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

Specify "true" to enable logging, or "false" to deactivate it. The default value is "false".

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get nxdomain_log_query
my $nxdomain_log_query = $griddns->nxdomain_log_query( );
#Modify nxdomain_log_query
$griddns->nxdomain_log_query("true");
```

nxdomain_rulesets( )

Use this method to specify or retrieve Infoblox::DNS::Ruleset object names assigned at the grid level for NXDOMAIN redirection.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

The valid value is an array reference that contains the names of the Infoblox::DNS::Ruleset objects assigned to the grid. The names must be in string format. The default value is undefined.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get nxdomain_rulesets
my $nxdomain_rulesets = $griddns->nxdomain_rulesets();
#Modify nxdomain_rulesets
$griddns->nxdomain_rulesets(["ruleset1", "ruleset2"]);
```

notify_source_port( )

Use this method to set or retrieve the source port number for notify messages. When requesting zone transfers from the primary server, some secondary DNS servers use the source port number (the primary server used to send the notify message) as the destination port number in the zone transfer request.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

Source port number for notify messages. Valid values are between 1 and 63999. *Default* is picked by BIND.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get the notify source port value
```

```
my $notify_source_port = $grid_dns->notify_source_port();
#Modify the notify source port value
$grid_dns->notify_source_port(77);
```

nsgroup_default( )

Use this method to set or retrieve the default nameserver group.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

Text with the name of the default nameserver group. *Default* value is None.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get the default nameserver group
my $nsgroup_default = $grid_dns->nsgroup_default();
#Modify the default nameserver group
$grid_dns->nsgroup_default("ns_group_A");
```

nsgroups( )

Use this method to set or retrieve the list of nameserver groups. A name server group is a collection of one primary DNS server and one or more secondary DNS servers.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

Valid value is an array reference that contains Infoblox::Grid::DNS::Nsgroup and Infoblox::DNS::Member object(s).

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get the nameserver group
my $nsgroup = $grid_dns->nsgroups();
#Modify the nameserver group
#Creating Member objects.
my $member1 = Infoblox::DNS::Member->new(
    name     => "infoblox.localdomain",
    ipv4addr => "192.168.1.2",
    stealth  => "false"
    );

#Creating a nameserver object.
my $nameserver1 = Infoblox::DNS::Nameserver->new(
    name     => "ns1.test.extra",
    ipv4addr => "4.4.4.4",
    stealth  => "false",
    TSIGname => "tsig_name",
    TSIGkey  => "ZX1jVJl7C58GT/sc7Q3ucA=="
    );

#Creating a nameserver object.
my $nameserver2 = Infoblox::DNS::Nameserver->new(
    name     => "ns2.test.extra",
    ipv4addr => "5.5.5.5",
    stealth  => "false",
    TSIGname => "tsig_name2",
    TSIGkey  => "qwqwqwqwqwqwqwqwqwqA=="
    );

my $nsg1 = Infoblox::Grid::DNS::Nsgroup->new(
    name        => "ns_group_A",
    primary     => $member1,
    secondaries => [$nameserver1, $nameserver2]
```

```
    );

$grid_dns->nsgroups([$nsg1]);
```

preserve_host_rrset_order_on_secondaries( )

Use this method to set or retrieve the grid-level preserve_host_rrset_order_on_secondaries value.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

Specify "true" to set the preserve_host_rrset_order_on_secondaries flag or "false" to unset it. The default value is "false".

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
# Get preserve_host_rrset_order_on_secondaries
my $preserve_order = $griddns->preserve_host_rrset_order_on_secondaries();
# Set preserve_host_rrset_order_on_secondaries
$griddns->preserve_host_rrset_order_on_secondaries("true");
```

query_source_port( )

Use this method to set or retrieve the query source port of a grid DNS object. Specifying a source port number for recursive queries ensures that a firewall will allow the response.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

Source port number for recursive queries . Valid values are between 1 and 63999. *Default* is picked by BIND.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get the query source port
my $query_source_port = $grid_dns->query_source_port();
#Modify the query source port
$grid_dns->query_source_port(99);
```

query_rewrite_prefix( )

Use this method to set or retrieve the domain name prefix for DNS query rewrite.

Parameter

Valid value is the domain name prefix in string format. The default value is undefined.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get query_rewrite_prefix
my $query_rewrite_prefix = $grid_dns->query_rewrite_prefix();
#Modify query_rewrite_prefix
$grid_dns->query_rewrite_prefix("user-ersgcds");
```

query_rewrite_domain_names( )

Use this method to set or retrieve the list of domain names that trigger DNS query rewrite.

Parameter

Valid value is an array reference that contains domain names in string format. The default value is undefined.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get query_rewrite_domain_names
my $query_rewrite_domain_names = $grid_dns->query_rewrite_domain_names();
#Modify query_rewrite_domain_names
$grid_dns->query_rewrite_domain_names(["florida.south", "florida.west"]);
```

scavenging_settings( )

Use this method to set or retrieve the Grid-level scavenging settings.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

The valid value is an Infoblox::Grid::DNS::ScavengingSetting object.

Parameter

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get scavenging_settings value
my $scavenging_settings = $object->scavenging_settings();

#Modify scavenging_settings value
$object->scavenging_settings($scavenging_settings);
```

record_name_policies( )

Use this method to set or retrieve the list of record name policies.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

Valid value is an array reference that contains Infoblox::Grid::RecordNamePolicy object(s). By default there are three predefined policies.

Policy Name Regex Applied

"Allow Any" .+

"Allow Underscore" ^[-a-zA-Z0-9_.]+$

"Strict Hostname Checking" ^[a-zA-Z0-9]$|^[a-zA-Z0-9][-a-zA-Z0-9.]*[a-zA-Z0-9]$

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get record name policies
my $record_name_policies = $grid_dns->record_name_policies();
#Modify record_name_policies

#Creating record name policy objects.
my $recpolicy1 = Infoblox::Grid::RecordNamePolicy->new(
    regex => "*",
    name  => "Local Policy",
    );
my $recpolicy2 = Infoblox::Grid::RecordNamePolicy->new(
    regex => ".+",
    name  => "Allow Any",
    );
```

```
my $recpolicy3 = Infoblox::Grid::RecordNamePolicy->new(
    regex => "^[-a-zA-Z0-9_.]+\$",
    name  => "Allow Underscore",
    );
my $recpolicy4 = Infoblox::Grid::RecordNamePolicy->new(
    regex => "^[a-zA-Z0-9]\$|^[a-zA-Z0-9][-a-zA-Z0-9.]*[a-zA-Z0-9]\$",
    name  => "Strict Hostname Checking",
    );

$grid_dns->record_name_policies([$recpolicy1, $recpolicy2, $recpolicy3, $recpolicy4]);
```

recursive_query_list( )

Use this method to set or retrieve the recursive query list of a Grid DNS object. This list specifies from which IP addresses, networks and TSIG key authenticated clients recursive queries are allowed.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

Valid value is an Infoblox::Grid::NamedACL object or an array reference that contains IPv4/IPv6 addresses, networks, and/or Infoblox::DNS::TSIGKey objects. To deny queries, add "!" as a prefix to IP addresses and/or networks. Specify "any" to allow recursive queries from any IP address.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get the recursive query list
my $recursive_query_list = $grid_dns->recursive_query_list();
#Modify the recursive query list
$grid_dns->recursive_query_list(["any"]);

#Creating a TSIG key object
my $tsig_key2 = $session->gen_tsig_key();

my $key2 = Infoblox::DNS::TSIGKey->new(
    name => "rcl_key",
    key  => $tsig_key2
    );
#Modify the recursive query list
$grid_dns->recursive_query_list(["10.0.0.10", "20.0.0.0/24","2010::256/64", $key2,
"!"."2011::23cb/64","!"."30.0.0.30", "!"."40.10.0.0/24"]);
#Modify the recursive query list
$grid_dns->recursive_query_list(undef);
#Use Infoblox::Grid::NamedACL
$grid_dns->recursive_query_list($nacl);
```

refresh_timer( )

Use this method to set or retrieve the refresh time. This interval tells the secondary how often to send a message to the primary for a zone to check that its data is current, and retrieve fresh data if it is not.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

Number with the seconds for the refresh time. *Default* value is 10800 seconds (3 hours).

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get the refresh time
my $refresh_timer = $grid_dns->refresh_timer();
#Modify the refresh time
$grid_dns->refresh_timer(108);
```

resolver_query_timeout( )

Use this method to set or retrieve recursive query timeout.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

The valid value is an unsigned integer that contains the recursive query time (in seconds) for the resolver.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get resolver_query_timeout
my $resolver_query_timeout = $grid_dns->resolver_query_timeout();
#Modify resolver_query_timeout
$grid_dns->resolver_query_timeout(29);
```

response_rate_limiting( )

Use this method to set or retrieve response rate limiting settings.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

The valid value is an Infoblox::Grid::DNS::ResponseRateLimiting object.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get response_rate_limiting
my $response_rate_limiting = $grid_dns->response_rate_limiting();
#Modify response_rate_limiting
$grid_dns->response_rate_limiting($response_rate_limiting);
```

restart_setting( )

Use this method to set or retrive Grid DNS service restart settings.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

The valid value is an Infoblox::Grid::ServiceRestart object.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get restart_setting
my $restart_setting = $grid_dns->restart_setting();
#Modify the retry time
$grid_dns->restart_setting($restart_setting);
```

retry_timer( )

Use this method to set or retrieve the retry time. This interval tells the secondary how long to wait before attempting to recontact the primary after a connection failure occurs between the two servers.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

Number with the seconds for the retry time. *Default* value is 3600 seconds (1 hour).

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get the retry time
my $retry_timer = $grid_dns->retry_timer();
#Modify the retry time
$grid_dns->retry_timer(360);
```

rootNS( )

Use this method to set or retrieve the list of customized root nameserver(s). You can use Internet root name servers or specify host names and IP addresses of custom root name servers.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

Valid value is an array reference that contains Infoblox::Grid::RootNameServer object(s).

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get the root nameserver list
my $rootNS_list = $grid_dns->rootNS();
#Modify the root nameserver list
#Creating RootNameServer objects.
my $rootns1 = Infoblox::Grid::RootNameServer->new(
    ipv4addr  => "44.44.44.44",
    host_name => "test1.com",
    );

my $rootns2 = Infoblox::Grid::RootNameServer->new(
    ipv4addr  => "55.5.5.5",
    host_name => "test2.com",
    );

$grid_dns->rootNS([$rootns1, $rootns2]);
```

serial_query_rate( )

Use this method to set or retrieve the number of maximum concurrent SOA queries per second for the Grid.

Parameter

Valid values are unsigned integer between 20 and 1000, inclusive. Default value is 20.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
# Get attribute value
my $value = $grid_dns->serial_query_rate();
# Modify attribute value
$grid_dns->serial_query_rate(30);
```

server_id_directive( )

Use this method to set or retrieve the value for the BIND and UNBOUND 'server-id' directive.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

The valid values are "NONE" and "HOSTNAME". The default value is "NONE".

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get server_id_directive
my $server_id_directive = $grid_dns->server_id_directive();
#Modify server_id_directive
$grid_dns->server_id_directive("HOSTNAME");
```

sortlist( )

Use this method to set or retrieve the sort list of a grid DNS object. A sort list sorts the order of addresses in responses made to DNS queries.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

Valid value is an array reference that contains Infoblox::DNS::Sortlist object(s).

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get the sort list
my $sortlist = $grid_dns->sortlist();
#Modify the sort list
#Creating a SortList object.
my $sort1 = Infoblox::DNS::Sortlist->new(
    match_list      => ["10.20.1.0/24", "10.20.2.0/24"],
    source_ipv4addr => "1.2.3.4"
    );

$grid_dns->sortlist([$sort1]);
```

store_locally( )

Use this method to set or retrieve the flag that indicates whether the storing of query capture reports on the appliance is enabled or disabled.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

Specify 'true' to enable storing of query capture reports on the appliance or 'false' to disable it. The default value is 'false'.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get store_locally value
my $store_locally = $object->store_locally();

#Modify store_locally value
$object->store_locally('true');
```

transfer_excluded_servers( )

Use this method to set or retrieve the list of excluded servers for zone transfers of a grid DNS object. This list exclude the specified DNS servers during zone transfers.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

The valid value is an array reference that contains IP addresses of DNS servers.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get the transfer excluded server list
my $transfer_excluded_servers = $grid_dns->transfer_excluded_servers();
#Modify the transfer excluded server list
$grid_dns->transfer_excluded_servers(["10.0.0.10", "20.0.0.0"]);
```

transfer_format( )

Use this method to set or retrieve the zone transfer format. The zone transfer format determines the BIND format for a zone transfer. This provides tracking capabilities for single or multiple transfers and their associated servers.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

Valid value is one of the following:

"many_answers" - Includes as many records as the packet size allows.

"one_answer" - Includes one record per packet.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get the zone transfer format
my $transfer_format = $grid_dns->transfer_format();
#Modify the zone transfer format
$grid_dns->transfer_format("many_answers");
```

transfers_in( )

Use this method to set or retrieve the number of maximum concurrent transfers for the Grid.

Parameter

Valid values are unsigned integer between 10 and 100, inclusive. Default value is 10.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
# Get attribute value
my $value = $grid_dns->transfers_in();
# Modify attribute value
$grid_dns->transfers_in(20);
```

transfers_out( )

Use this method to set or retrieve the number of maximum outbound concurrent zone transfers for the Grid.

Parameter

Valid values are unsigned integers between 1 and 100, inclusive. The default value is 10.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
# Get attribute value
my $value = $grid_dns->transfers_out();
# Modify attribute value
```

```
$grid_dns->transfers_out(20);
```

transfers_per_ns( )

Use this method to set or retrieve the number of maximum concurrent transfers per member for the Grid.

Parameter

Valid values are unsigned integer between 2 and 100, inclusive. Default value is 2.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
# Get attribute value
my $value = $grid_dns->transfers_per_ns();
# Modify attribute value
$grid_dns->transfers_per_ns(3);
```

rpz_disable_nsdname_nsip( )

Use this method to set or retrieve the option to disable NSDNAME and NSIP resource records from RPZ feeds.

Parameter

Specify "true" to enable the filter (to filter out NSDNAME and NSIP resource records). Specify "false" to disable the filter. Valid values are "true" or "false". Default value is "false".

Returns

If you specify a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
# Get attribute value
my $value = $grid_dns->rpz_disable_nsdname_nsip();
# Modify attribute value
$grid_dns->rpz_disable_nsdname_nsip("true");
```

rpz_drop_ip_rule_enabled( )

Use this method to set or retreive the flag that indicates whether the DNS server ignores RPZ-IP triggers with large prefix lengths or not.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

Specify 'true' to enable ignoring of RPZ-IP triggers with large prefix lengths or 'false' to disable it. The default value is 'false'.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
# Get attribute value
my $value = $grid_dns->rpz_drop_ip_rule_enabled();
# Modify attribute value
$grid_dns->rpz_drop_ip_rule_enabled("true");
```

rpz_drop_ip_rule_min_prefix_length_ipv4( )

Use this method to set or retreive minimum prefix length for IPv4 RPZ-IP triggers.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

The valid value is an unsigned integer between 1 and 31 that represents prefix length (in bits) for IPv4 RPZ-IP triggers.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
# Get attribute value
my $value = $grid_dns->rpz_drop_ip_rule_min_prefix_length_ipv4();
# Modify attribute value
$grid_dns->rpz_drop_ip_rule_min_prefix_length_ipv4("16");
```

rpz_drop_ip_rule_min_prefix_length_ipv6( )

Use this method to set or retreive minimum prefix length for IPv6 RPZ-IP triggers.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

The valid value is an unsigned integer between 1 and 127 that represents prefix length (in bits) for IPv6 RPZ-IP triggers.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
# Get attribute value
my $value = $grid_dns->rpz_drop_ip_rule_min_prefix_length_ipv6();
# Modify attribute value
$grid_dns->rpz_drop_ip_rule_min_prefix_length_ipv6("120");
```

rpz_qname_wait_recurse( )

Use this method to enable or disable recursive RPZ lookups.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

Specify "true" to enable recursive RPZ lookups or "false" to disable them. The default value is "false".

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
# Get attribute value
my $value = $grid_dns->rpz_qname_wait_recurse();
# Modify attribute value
$grid_dns->rpz_qname_wait_recurse("true");
```

zone_deletion_double_confirm( )

Use this method to enable or disable double confirmation during zone deletion.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

Specify "true" to enable the zone_deletion_double_confirm flag or "false" to deactivate/unset it. The default value is "true".

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get the double confirmation of zone deletion flag
my $zone_deletion_double_confirm = $grid_dns->zone_deletion_double_confirm();
```

```
#Modify the double confirmation of zone deletion flag
$grid_dns->zone_deletion_double_confirm("false");
```

SAMPLE CODE

The following sample code demonstrates the different functions that can be applied to an object, such as get and modify. This sample code also includes error handling for the operations.

#Preparation prior to getting and modifying a grid DNS object

```
use strict;
use Infoblox;

#Creating a session to appliance.
my $session = Infoblox::Session->new(
    master   => "192.168.1.2",
    username => "admin",
    password => "infoblox"
    );
unless( $session ){
    die("Constructor for session failed:",
    Infoblox::status_code() . ":" . Infoblox::status_detail());
}
print" Session object created successfully \n";

#Creating RootNameServer objects.
my $rootns1 = Infoblox::Grid::RootNameServer->new(
    ipv4addr  => "44.44.44.44",
    host_name => "test1.com",
    );
unless( $rootns1 ){
    die("Constructor for rootnameserver1 failed:",
    Infoblox::status_code() . ":" . Infoblox::status_detail());
}
print" Rootnameserver1 object created successfully \n";

my $rootns2 = Infoblox::Grid::RootNameServer->new(
    ipv4addr  => "55.5.5.5",
    host_name => "test2.com",
    );
unless( $rootns2 ){
    die("Constructor for rootnameserver2 failed:",
    Infoblox::status_code() . ":" . Infoblox::status_detail());
}
print" Rootnameserver2 object created successfully \n";

#Creating Transfer Key objects.
my $key1 = Infoblox::DNS::TSIGKey->new(
    name => "tsig_xfer",
    key  => ":2xCOMPAT"
    );
unless( $key1 ){
    die("Constructor for transfer key \(key1\) failed:",
    Infoblox::status_code() . ":" . Infoblox::status_detail());
}
print" Transfer key object \(key1\) created successfully \n";

my $tsig_key1 = $session->gen_tsig_key();
unless( $tsig_key1 ){
    die("Generate tsig_key1 failed: ",
    $session->status_code() . ":" . $session->status_detail());
}

my $key2 = Infoblox::DNS::TSIGKey->new(
    name => "transfer_key",
    key  => $tsig_key1
    );
unless( $key2 ){
    die("Constructor for transfer key \(key2\) failed:",
    Infoblox::status_code() . ":" . Infoblox::status_detail());
}
print" Transfer key object \(key2\) created successfully \n";

#Creating Update Key object.
my $update_key1 = $session->gen_tsig_key();
```

```
unless( $update_key1 ){
    die("Generate update_key1 failed:",
    $session->status_code() . ":" . $session->status_detail());
}

my $up_key1 = Infoblox::DNS::TSIGKey->new(
    name => "update_key",
    key  => $update_key1
    );
unless( $up_key1 ){
    die("Constructor for update key failed: ",
    Infoblox::status_code() . ":" . Infoblox::status_detail());
}
print" Update key object created successfully \n";

#Creating a SortList object.
my $sort1 = Infoblox::DNS::Sortlist->new(
    match_list      => ["10.20.1.0/24", "10.20.2.0/24"],
    source_ipv4addr => "1.2.3.4"
    );
unless( $sort1 ){
    die("Constructor for sortlist failed: ",
    Infoblox::status_code() . ":" . Infoblox::status_detail());
}
print" Sortlist object created successfully \n";
```

#Get and modify the DNS object

```
#Getting the DNS object from appliance through session.
my @result = $session->get(
    object => "Infoblox::Grid::DNS",
    name   => "Infoblox"
    );
unless( @result ){
    die("Get grid DNS failed: ",
    $session->status_code() . ":" . $session->status_detail());
}
print" Get grid DNS object found at least 1 matching entry \n";

my $grid_dns = $result[0];
if ($grid_dns) {

#Modifying the value of the specified object
   $grid_dns->allow_query(["10.0.0.10", "20.0.0.0/24", "!" . "30.0.0.30", "!" . "40.10.0.0/24"]);
   $grid_dns->allow_recursive_query("true");
   $grid_dns->allow_transfer(["10.0.0.10", "20.0.0.0/24", $key1, "!"."30.0.0.30", "!"."40.10.0.0/24",
$key2]);
   $grid_dns->allow_update(["10.0.0.10", "20.0.0.0/24", "!"."30.0.0.30", "!"."40.10.0.0/24", $up_key1]);
   $grid_dns->default_ttl(28800);
   $grid_dns->expire_after(2419200);
   $grid_dns->forwarders(["2.2.2.2", "3.3.3.3"]);
   $grid_dns->logging_facility("daemon");
   $grid_dns->negative_ttl(900);
   $grid_dns->recursive_query_list(["10.0.0.10", "20.0.0.0/24", "!" . "30.0.0.30", "!" . "40.10.0.0/24"]);
   $grid_dns->refresh_timer(10800);
   $grid_dns->retry_timer(3600);
   $grid_dns->rootNS([$rootns1, $rootns2]);
   $grid_dns->sortlist([$sort1]);
   $grid_dns->transfer_excluded_servers(["10.0.0.10", "20.0.0.0"]);
   $grid_dns->transfer_format("many_answers");

 #Applying the changes
 my $response = $session->modify($grid_dns);
 unless( $response ){
    die("Modify grid DNS failed: ",
    $session->status_code() . ":" . $session->status_detail());
 }
 print" Modify grid DNS successful \n";
  }
  else {
   print "No grid DNS object found.";
  }

####PROGRAM ENDS####
```

AUTHOR

Infoblox Inc. http://www.infoblox.com/

SEE ALSO

Infoblox::DNS::Sortlist, Infoblox::DNS::TSIGKey, Infoblox::DNS::DnssecTrustedKey, Infoblox::Grid::DNS::Nsgroup, Infoblox::Grid::RootNameServer, Infoblox::DNS::Ruleset, Infoblox::Session->get(), Infoblox::Session->modify(), Infoblox::Session, Infoblox::Grid::NamedACL, Infoblox::DNS::DNSSecKeyAlgorithm, Infoblox::Grid::DNS::FixedRRSetOrderFQDN, Infoblox::Grid::DNS::ResponseRateLimiting, Infoblox::Grid::DNS::FileTransferSetting, Infoblox::Grid::DNS::ClientSubnetDomain

---

COPYRIGHT

Copyright (c) 2017 Infoblox Inc.

Infoblox::Grid::DNS::AllNsgroups - All NS Groups object.

- NAME
- DESCRIPTION
- SESSION METHODS
  - Infoblox::Session->get( )
  - Infoblox::Session->search( )
- METHODS
  - comment( )
  - name( )
  - type( )
- AUTHOR
- SEE ALSO
- COPYRIGHT

NAME

Infoblox::Grid::DNS::AllNsgroups - All NS Groups object.

DESCRIPTION

The All NS Groups object is a generic nameserver group object that provides information about all nameserver groups.

SESSION METHODS

This section describes all the methods in an Infoblox::Session module that you can apply to object.

Infoblox::Session->get( )

Use this method to retrieve all the matching objects from the Infoblox appliance. See Infoblox::Session->get() for parameters and return values.

Key References

```
Apply the following attributes to get a specific nameserver group object:

 name    - Optional. The nameserver group name in a string format.
 comment - Optional. The nameserver group comment in a string format.
 type    - Optional. The nameserver group type in a string format.
```

Examples

```
my @retrieved_objs = $session->get(
    object  => 'Infoblox::Grid::DNS::AllNsgroups',
    name    => 'ns_group_a',
);
```

Infoblox::Session->search( )

Use this method to search for objects in the Infoblox appliance. See Infoblox::Session->search() for parameters and return values.

Key References

```
Apply the following attributes to search for the object:

 name    - Optional. The nameserver group name in a string format (regexp).
 comment - Optional. The nameserver group comment in a string format (regexp).
 type    - Optional. The nameserver group type in a string format (exact).
```

Examples

```
# search for objects
my @retrieved_objs = $session->search(
    object       => 'Infoblox::Grid::DNS::AllNsgroups',
    name         => 'ns_.*',
    comment      => '.*comment',
);
```

METHODS

This section describes all the methods that you can use to retrieve the attribute values of a nameserver group.

comment( )

Use this method to retrieve the nameserver group descriptive comment. This is a read-only attribute.

Omit the parameter to retrieve the attribute value.

Parameter

None

Returns

The method returns the attribute value.

Example

```
#Get comment value
my $comment = $object->comment();
```

name( )

Use this method to retrieve the nameserver group name. This is a read-only attribute.

Omit the parameter to retrieve the attribute value.

Parameter

None

Returns

The method returns the attribute value.

Example

```
#Get name value
my $name = $object->name();
```

type( )

Use this method to retrieve the nameserver group type. This is a read-only attribute.

Omit the parameter to retrieve the attribute value.

Parameter

None

Returns

The valid return values are 'AUTH', 'FORWARDING_MEMBER', 'STUB_MEMBER', 'DELEGATION', 'FORWARD_STUB_SERVER'.

Example

```
#Get type value
my $type = $object->type();
```

AUTHOR

Infoblox Inc. http://www.infoblox.com/

SEE ALSO

Infoblox::Session, Infoblox::Session->get(), Infoblox::Session->search(),

---

COPYRIGHT

Copyright (c) 2017 Infoblox Inc.

Infoblox::Grid::DNS::AttackDetect - a DNS Attack Detect object.

- NAME
- DESCRIPTION
- CONSTRUCTOR
- SESSION METHODS
- MODULE METHODS
  - Infoblox::Grid::DNS::AttackMitigation->detect_chr( )
  - Infoblox::Grid::DNS::AttackMitigation->detect_nxdomain_responses( )
  - Infoblox::Grid::DNS::AttackMitigation->detect_udp_drop( )
- METHODS
  - enabled( )
  - high( )
  - interval_max( )
  - interval_min( )
  - interval_time( )
  - low( )
- AUTHOR
- SEE ALSO
- COPYRIGHT

NAME

Infoblox::Grid::DNS::AttackDetect - a DNS Attack Detect object.

DESCRIPTION

The DNS Attack Detect object provides information about DNS attack detection settings.

CONSTRUCTOR

```
my $attack_detect = Infoblox::Grid::DNS::AttackDetect->new(
    enable        => 'true' | 'false', # Optional
    high          => $uint,            # Optional
    interval_max  => $uint,            # Optional
    interval_min  => $uint,            # Optional
    interval_time => $uint,            # Optional
    low           => $uint,            # Optional
);
```

SESSION METHODS

The object does not support any session methods.

MODULE METHODS

Infoblox::Grid::DNS::AttackMitigation->detect_chr( )

Use this method to set or retrieve configuration for the Cache Hit Ratio (CHR) changes detection. See Infoblox::Grid::DNS::AttackMitigation->detect_chr() for parameters and return values.

Example

```
#Get detect_chr value
my $detect_chr = $attack_mitigation->detect_chr();
#Modify detect_chr
$attack_mitigation->detect_chr($detect_chr);
```

Infoblox::Grid::DNS::AttackMitigation->detect_nxdomain_responses( )

Use this method to set or retrieve configuration for the recursive queries NXDOMAIN responses detection. See Infoblox::Grid::DNS::AttackMitigation->detect_nxdomain_responses() for parameters and return values.

Example

```
#Get detect_nxdomain_responses value
my $detect_nxdomain_responses = $attack_mitigation->detect_nxdomain_responses();
#Modify detect_nxdomain_responses
$attack_mitigation->detect_nxdomain_responses($detect_nxdomain_responses);
```

Infoblox::Grid::DNS::AttackMitigation->detect_udp_drop( )

Use this method to set or retrieve configuration for the UDP packet drop rate detection. See Infoblox::Grid::DNS::AttackMitigation->detect_udp_drop() for parameters and return values.

Example

```
#Get detect_udp_drop value
my $detect_udp_drop = $attack_mitigation->detect_udp_drop();
#Modify detect_udp_drop
$attack_mitigation->detect_udp_drop($detect_udp_drop);
```

METHODS

enabled( )

Use this method to enable or disable DNS attack detection.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

Specify 'true' to enable DNS attack detection or 'false' to disable it. The default value is 'false'.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get enable value
my $enable = $attack_detect->enable();
#Modify enable
$attack_detect->enable('true');
```

high( )

Use this method to set or retrieve the high threshold value (in percentage) for starting DNS attack detection.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

The valid value is an unsigned integer between 0 and 100.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get high value
my $high = $attack_detect->high();
#Modify high
$attack_detect->high(90);
```

interval_max( )

Use this method to set or retrieve the maximum number of events that have occurred before processing DNS attack detection.

Note that detection processing occurs each time when interval_max events have occurred or at least interval_min events have occurred and at least interval_time seconds have elapsed.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

The valid value is an unsigned integer.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get interval_max value
my $interval_max = $attack_detect->interval_max();
#Modify interval_max
$attack_detect->interval_max(100000);
```

interval_min( )

Use this method to set or retrieve the minimum number of events that have occurred before processing DNS attack detection.

Note that detection processing occurs each time when interval_max events have occurred or at least interval_min events have occurred and at least interval_time seconds have elapsed.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

The valid value is an unsigned integer.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get interval_min value
my $interval_min = $attack_detect->interval_min();
#Modify interval_min
$attack_detect->interval_min(1000);
```

interval_time( )

Use this method to set or retrieve the time interval between detection processing.

Note that detection processing occurs each time when interval_max events have occurred or at least interval_min events have occurred and at least interval_time seconds have elapsed.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

The valid value is an unsigned integer.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get interval_time value
my $interval_time = $attack_detect->interval_time();
#Modify interval_time
$attack_detect->interval_time(10);
```

low( )

Use this method to set or retrieve the low threshold value (in percentage) for starting DNS attack detection.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

The valid value is an unsigned integer between 0 and 100.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get low value
my $low = $attack_detect->low();
#Modify low
$attack_detect->low(40);
```

AUTHOR

Infoblox Inc. http://www.infoblox.com/

SEE ALSO

the Infoblox::Grid::DNS::AttackMitigation manpage, Infoblox::Grid::DNS::AttackMitigation->detect_chr(), Infoblox::Grid::DNS::AttackMitigation->detect_nxdomain_responses(), Infoblox::Grid::DNS::AttackMitigation->detect_udp_drop()

COPYRIGHT

Copyright (c) 2017 Infoblox Inc.

Infoblox::Grid::DNS::AttackMitigation - a DNS Attack Mitigation object.

- NAME
- DESCRIPTION
- CONSTRUCTOR
- SESSION METHODS
- MODULE METHODS
  - Infoblox::Grid::DNS->attack_mitigation( )
  - Infoblox::Grid::Member::DNS->attack_mitigation( )
- METHODS
  - detect_chr( )
  - detect_chr_grace( )
  - detect_nxdomain_responses( )
  - detect_udp_drop( )
  - interval( )
  - mitigate_nxdomain_lru( )
- AUTHOR
- SEE ALSO
- COPYRIGHT

NAME

Infoblox::Grid::DNS::AttackMitigation - a DNS Attack Mitigation object.

DESCRIPTION

The DNS Attack Mitigation object provides information about mitigation configuration for DNS attacks.

CONSTRUCTOR

```
my $attack_mitigation = Infoblox::Grid::DNS::AttackMitigation->new(
    detect_chr                => $attack_detect,    # Optional
    detect_chr_grace          => $uint,             # Optional
    detect_nxdomain_responses => $attack_detect,    # Optional
    detect_udp_drop           => $attack_detect,    # Optional
    interval                  => $uint,             # Optional
    mitigate_nxdomain_lru     => 'true' | 'false', # Optional
);
```

SESSION METHODS

The object does not support any session methods.

MODULE METHODS

The following functions can be applied to a DNS Attack Mitigation object.

Infoblox::Grid::DNS->attack_mitigation( )

Use this method to set or retrieve mitigation settings for DNS attacks. See Infoblox::Grid::DNS->attack_mitigation() for parameters and return values.

Example

```
#Get attack_mitigation value
my $attack_mitigation = $grid_dns->attack_mitigation();
#Modify attack_mitigation
$grid_dns->attack_mitigation($attack_mitigation);
```

Infoblox::Grid::Member::DNS->attack_mitigation( )

Use this method to set or retrieve mitigation settings for DNS attacks on the member. See Infoblox::Grid::Member::DNS->attack_mitigation() for parameters and return values.

Example

```
#Get attack_mitigation value
my $attack_mitigation = $member_dns->attack_mitigation();
#Modify attack_mitigation
```

```
$member_dns->attack_mitigation($attack_mitigation);
```

METHODS

detect_chr( )

Use this method to set or retrieve configuration for the Cache Hit Ratio (CHR) changes detection.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

The valid value is an Infoblox::Grid::DNS::AttackDetect object.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get detect_chr value
my $detect_chr = $attack_mitigation->detect_chr();
#Modify detect_chr
$attack_mitigation->detect_chr($detect_chr);
```

detect_chr_grace( )

Use this method to set or retrieve the cache usage (in percentage) when Cache Hit Ratio (CHR) detection starts.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

The valid value is an unsigned integer between 0 and 100.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get detect_chr_grace value
my $detect_chr_grace = $attack_mitigation->detect_chr_grace();
#Modify detect_chr_grace
$attack_mitigation->detect_chr_grace(50);
```

detect_nxdomain_responses( )

Use this method to set or retrieve configuration for the recursive queries NXDOMAIN responses detection.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

The valid value is an Infoblox::Grid::DNS::AttackDetect object.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get detect_nxdomain_responses value
my $detect_nxdomain_responses = $attack_mitigation->detect_nxdomain_responses();
#Modify detect_nxdomain_responses
$attack_mitigation->detect_nxdomain_responses($detect_nxdomain_responses);
```

detect_udp_drop( )

Use this method to set or retrieve configuration for the UDP packet drop rate detection.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

The valid value is an Infoblox::Grid::DNS::AttackDetect object.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get detect_udp_drop value
my $detect_udp_drop = $attack_mitigation->detect_udp_drop();
#Modify detect_udp_drop
$attack_mitigation->detect_udp_drop($detect_udp_drop);
```

interval( )

Use this method to set or retrieve the minimum time interval (in seconds) between changes in attack status.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

The valid value is an unsigned integer.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get interval value
my $interval = $attack_mitigation->interval();
#Modify interval
$attack_mitigation->interval(50);
```

mitigate_nxdomain_lru( )

Use this method to enable or disable impact mitigation of NXDOMAIN responses in cache by tracking NXDOMAIN entries.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

Specify 'true' to enable impact mitigation of NXDOMAIN responses, or 'false' to disable it. The default value is 'false'.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get mitigate_nxdomain_lru value
my $mitigate_nxdomain_lru = $attack_mitigation->mitigate_nxdomain_lru();
#Modify mitigate_nxdomain_lru
$attack_mitigation->mitigate_nxdomain_lru('true');
```

AUTHOR

Infoblox Inc. http://www.infoblox.com/

SEE ALSO

Infoblox::Grid::DNS, Infoblox::Grid::DNS->attack_mitigation(), Infoblox::Grid::Member::DNS, Infoblox::Grid::Member::DNS->attack_mitigation(), Infoblox::Grid::DNS::AttackDetect,

---

COPYRIGHT

Infoblox::Grid::DNS::AttackMitigation - a DNS Attack Mitigation object.

Copyright (c) 2017 Infoblox Inc.

Infoblox::Grid::DNS::AutoBlackHole - a DNS Auto Blackhole object.

- NAME
- DESCRIPTION
- CONSTRUCTOR
- SESSION METHODS
- MODULE METHODS
  - Infoblox::Grid::DNS->auto_blackhole( )
  - Infoblox::Grid::Member::DNS->auto_blackhole( )
- METHODS
  - enable_fetches_per_server( )
  - enable_fetches_per_zone( )
  - enable_holddown( )
  - fetches_per_server( )
  - fetches_per_zone( )
  - fps_freq( )
  - holddown( )
  - holddown_threshold( )
  - holddown_timeout( )
- AUTHOR
- SEE ALSO
- COPYRIGHT

NAME

Infoblox::Grid::DNS::AutoBlackHole - a DNS Auto Blackhole object.

DESCRIPTION

The DNS Auto Blackhole object provides information about DNS auto blackhole configuration.

CONSTRUCTOR

```
my $auto_blackhole = Infoblox::Grid::DNS::AutoBlackHole->new(
        enable_fetches_per_server => 'true' | 'false', # Optional
        enable_fetches_per_zone   => 'true' | 'false', # Optional
        enable_holddown           => 'true' | 'false', # Optional
        fetches_per_server        => $uint,            # Optional
        fetches_per_zone          => $uint,            # Optional
        fps_freq                  => $uint,            # Optional
        holddown                  => $uint,            # Optional
        holddown_timeout          => $uint,            # Optional
        holddown_threshold        => $uint,            # Optional
);
```

SESSION METHODS

The object does not support any session methods.

MODULE METHODS

The following functions can be applied to a DNS Auto Blackhole object.

Infoblox::Grid::DNS->auto_blackhole( )

Use this method to set or retrieve the auto blackhole settings. See Infoblox::Grid::DNS->auto_blackhole() for parameters and return values.

Example

```
#Get auto_blackhole value
my $auto_blackhole = $grid_dns->auto_blackhole();
#Modify bind_check_auto_blackhole
$grid_dns->auto_blackhole($auto_blackhole);
```

Infoblox::Grid::Member::DNS->auto_blackhole( )

Use this method to set or retrieve the auto blackhole settings on the member. See Infoblox::Grid::Member::DNS->auto_blackhole() for

paparameters and return values.

Example

```
#Get auto_blackhole value
my $auto_blackhole = $member_dns->auto_blackhole();
#Modify auto_blackhole
$member_dns->auto_blackhole($auto_blackhole);
```

METHODS

enable_fetches_per_server( )

Use this method to enable or disable the maximum number of concurrent recursive queries sent to each upstream DNS server.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

Specify 'true' to enable the maximum number of concurrent recursive queries sent to each upstream DNS server or 'false" to disable it.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get enable_fetches_per_server value
my $enable_fetches_per_server = $auto_blackhole->enable_fetches_per_server();
#Modify enable_fetches_per_server
$auto_blackhole->enable_fetches_per_server('true');
```

enable_fetches_per_zone( )

Use this method to enable or disable the maximum number of concurrent recursive queries sent to each DNS zone.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

Specify 'true' to enable the maximum number of concurrent recursive queries sent to each DNS zone or 'false" to disable it.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get enable_fetches_per_zone value
my $enable_fetches_per_zone = $auto_blackhole->enable_fetches_per_zone();
#Modify enable_fetches_per_zone
$auto_blackhole->enable_fetches_per_zone('true');
```

enable_holddown( )

Use this method to enable or disable the holddown for non-responsive servers for a specified time interval.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

Specify 'true' to enable the holddown for non-responsive servers or 'false' to disable it.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get enable_holddown value
my $enable_holddown = $auto_blackhole->enable_holddown();
```

```
#Modify enable_holddown
$auto_blackhole->enable_holddown('true');
```

fetches_per_server( )

Use this method to set or retrieve the maximum number of concurrent recursive queries for each DNS server.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

The valid value is an unsigned integer.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get fetches_per_server value
my $fetches_per_server = $auto_blackhole->fetches_per_server();
#Modify fetches_per_server
$auto_blackhole->fetches_per_server(2000);
```

fetches_per_zone( )

Use this method to set or retrieve the maximum number of concurrent recursive queries for each DNS zone.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

The valid value is an unsigned integer.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get fetches_per_zone value
my $fetches_per_zone = $auto_blackhole->fetches_per_zone();
#Modify fetches_per_zone
$auto_blackhole->fetches_per_zone(2000);
```

fps_freq( )

Use this method to set or retrieve the number of recursive queries that determines the recalculation of the timeout ratio for each DNS server.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

The valid value is an unsigned integer.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get fps_freq value
my $fps_freq = $auto_blackhole->fps_freq();
#Modify fps_freq
$auto_blackhole->fps_freq(2000);
```

holddown( )

Use this method to set or retrieve the holddown duration for non-responsive servers.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

The valid value is an unsigned integer.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get holddown value
my $holddown = $auto_blackhole->holddown();
#Modify holddown
$auto_blackhole->holddown(100);
```

holddown_threshold( )

Use this method to set or retrieve the number of consecutive timeouts before holding down a non-responsive server.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

The valid value is an unsigned integer.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get holddown_threshold value
my $holddown_threshold = $auto_blackhole->holddown_threshold();
#Modify holddown_threshold
$auto_blackhole->holddown_threshold(100);
```

holddown_timeout( )

Use this method to set or retrieve the minimum time (in seconds) that needs to be passed before a timeout occurs. Note that only these timeouts are counted towards the number of consecutive timeouts.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

The valid value is an unsigned integer.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get holddown_timeout value
my $holddown_timeout = $auto_blackhole->holddown_timeout();
#Modify holddown_timeout
$auto_blackhole->holddown_timeout(100);
```

AUTHOR

Infoblox Inc. http://www.infoblox.com/

SEE ALSO

Infoblox::Grid::DNS, Infoblox::Grid::DNS->auto_blackhole(), Infoblox::Grid::Member::DNS, Infoblox::Grid::Member::DNS->auto_blackhole()

COPYRIGHT

Copyright (c) 2017 Infoblox Inc.

Infoblox::Grid::DNS::AutoBlackHole - a DNS Auto Blackhole object.

Infoblox::Grid::DNS::BulkHostNameTemplate - Manages the DNS bulk host name formats defined at the grid level.

- NAME
- DESCRIPTION
- CONSTRUCTOR
- MODULE METHODS
  - Infoblox::Grid::DNS->bulk_host_name_templates( )
- METHODS
  - template_name( )
  - template_format( )
- SAMPLE CODE
- AUTHOR
- SEE ALSO
- COPYRIGHT

NAME

Infoblox::Grid::DNS::BulkHostNameTemplate - Manages the DNS bulk host name formats defined at the grid level.

DESCRIPTION

Bulk host name template defines the format based on which the host names are generated from a bulk host..

CONSTRUCTOR

```
my $bulk_host_name_template = Infoblox::Grid::DNS::BulkHostNameTemplate->new(
    template_name   => $string,    #Required
    template_format => $string     #Required
);
```

MODULE METHODS

The following functions are available to apply to the bulk host name template object.

Infoblox::Grid::DNS->bulk_host_name_templates( )

Use this function to specify a bulk host name template at the grid level in the Infoblox appliance.

Note that we can't remove or modify default bulk host name templates we can only add and modify our custom templates to the current bulk host name templates array.

Example

```
#Create a BulkHostNameTemplate object.
my $custom_template = Infoblox::Grid::DNS::BulkHostNameTemplate->new(
   template_name   => "test",
   template_format => "-#1-#2-#3-#4",
);

#Retrieve bulk host name templates array
my $bhnt = $grid_dns->bull_host_name_templates();

#Configure bulk host name template on the Infoblox grid object
push @$bhnt, $custom_template;
my $response = $grid_dns->bulk_host_name_templates($bhnt);
```

METHODS

This section describes all the methods that can be used to configure and retrieve the attributes values of a bulk host name template object.

template_name( )

Use this method to set or retrieve the name of bulk host name template.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

String with the name of the bulk host name template.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get template_name
my $template_name = $bulk_host_name_template->template_name();
#Modify template_name
$template_name->template_name("new_name");
```

template_format( )

Use this method to set or retrieve the format of bulk host name template.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

The format of bulk host name template. It should follow certain rules (please use Administration Guide as reference).

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get template_format
my $template_format = $bulk_host_name_template->template_format();
#Modify template_format
$bulk_host_name_template>template_format("-#1-#2-#3-#4");
```

SAMPLE CODE

The following sample code demonstrates different operations that can be applied to a bulk host name template object such as create, modify, and remove. Also, this sample code includes error handling for the operations.

```
#PROGRAM STARTS: Include all the modules that will be used
use strict;
use Infoblox;

#Create a session to the Infoblox appliance
my $session = Infoblox::Session->new(
    master   => "192.168.1.2",
    username => "admin",
    password => "infoblox"
);
unless ($session) {
    die( "Construct session failed: ",
        Infoblox::status_code() . ":" . Infoblox::status_detail() );
}
print "Session created successfully\n";

#Create a new bulk host name template object

my $my_template = Infoblox::Grid::DNS::BulkHostNameTemplate->new(
    template_name   => "My Template",
    template_format => "#4",
);
unless ($my_template) {
    die(
        "Construct bulk host name template object my_template failed: ",
        Infoblox::status_code() . ":" . Infoblox::status_detail()
    );
}
print "Bulk host name template object my_template created successfully\n";

#Get the Member DNS object to add a bulk host name template object to it

my $grid_dns =
  $session->get( "object" => "Infoblox::Grid::DNS", "grid" => "Infoblox" );
```

```
unless ($grid_dns) {
    die( "Get Grid DNS failed: ",
        $session->status_code() . ":" . $session->status_detail() );
}

my $bhnt = $grid_dns->bulk_host_name_templates();

#Add new bulk host name template to Grid DNS bulk host name templates

unshift(@$bhnt, $my_template);
$grid_dns->bulk_host_name_templates($bhnt);

$session->modify($grid_dns)
  or die( "modify Grid DNS failed:",
    $session->status_code(), $session->status_detail() );
print "Modify grid DNS successful for bulk host name template object\n";

####PROGRAM ENDS####
```

AUTHOR

Infoblox Inc. http://www.infoblox.com/

SEE ALSO

Infoblox::Grid::DNS, Infoblox::Session, Infoblox::Session->get(), Infoblox::Session->modify()

COPYRIGHT

Copyright (c) 2017 Infoblox Inc.

Infoblox::Grid::DNS::ClientSubnetDomain - a DNS client subnet domain object.

- NAME
- DESCRIPTION
- CONSTRUCTOR
- SESSION METHODS
- MODULE METHODS
  - Infoblox::Grid::DNS->client_subnet_domains( )
- METHODS
  - domain( )
  - permission( )
- AUTHOR
- SEE ALSO
- COPYRIGHT

NAME

Infoblox::Grid::DNS::ClientSubnetDomain - a DNS client subnet domain object.

DESCRIPTION

The DNS client subnet domain represent the zone domain name that is allowed or forbidden for the EDNS client subnet (ECS) recursion.

CONSTRUCTOR

```
my $object = Infoblox::Grid::DNS::ClientSubnetDomain->new(
    'domain'     => $fqdn,            # Required
    'permission' => 'ALLOW' | 'DENY', # Optional / Default is 'ALLOW'
);
```

SESSION METHODS

The object does not support any session methods.

MODULE METHODS

The following functions can be applied to a DNS client subnet domain object.

Infoblox::Grid::DNS->client_subnet_domains( )

Use this method to set or retrieve the list of zone domain names that are allowed or forbidden for EDNS client subnet (ECS) recursion.

Example

```
#Get client_subnet_domains value
my $value = $grid_dns->client_subnet_domains();

#Construct Infoblox::Grid::DNS::ClientSubnetDomain object
my $object = Infoblox::Grid::ClientSubnetDomain->new(
    domain     => 'domain.com',
    permission => 'DENY',
);

#Modify client_subnet_domains value
$grid_dns->client_subnet_domains([$object]);
```

METHODS

This section describes all the methods that you can use to configure and retrieve the attribute values of the object.

domain( )

Use this method to set or retrieve the client subnet zone domain name.

Parameter

The valid value is a Fully-Qualified Domain Name (FQDN) that represents the desired zone domain name.

Returns

The method returns the attribute value.

Example

```
#Get domain value
my $value = $object->domain();

#Modify domain value
$object->domain('test-domain.com');
```

permission( )

Use this method to set or retrieve the client subnet domain ECS permission.

Parameter

The valid values are 'ALLOW' and 'DENY'. The default value is 'ALLOW'.

Returns

The method returns the attribute value.

Example

```
#Get permission value
my $value = $object->permission();

#Modify permission value
$object->permission('DENY');
```

AUTHOR

Infoblox Inc. http://www.infoblox.com/

SEE ALSO

Infoblox::Grid::DNS, Infoblox::Grid::DNS->client_subnet_domains(),

---

COPYRIGHT

Copyright (c) 2017 Infoblox Inc.

Infoblox::Grid::DNS::DNS64Group - DNS64 Synthesis Group object

- NAME
- DESCRIPTION
- CONSTRUCTOR
- SESSION METHODS
  - Infoblox::Session->add( )
  - Infoblox::Session->get( )
  - Infoblox::Session->modify( )
  - Infoblox::Session->remove( )
  - Infoblox::Session->search( )
- METHODS
  - comment( )
  - clients( )
  - disable( )
  - dnssec_dns64_enabled( )
  - exclude( )
  - extattrs( )
  - extensible_attributes( )
  - name( )
  - mapped( )
  - prefix( )
- SAMPLE CODE
- AUTHOR
- SEE ALSO
- COPYRIGHT

NAME

Infoblox::Grid::DNS::DNS64Group - DNS64 Synthesis Group object

---

DESCRIPTION

A DNS64 synthesis group is used to configure the DNS64 feature.

CONSTRUCTOR

```
 my $group = Infoblox::Grid::DNS::DNS64Group->new(
     name                  => $string,
#Required
     prefix                => $ipv6prefix,
#Required
     clients               => [ $ipv4addr | $ipv4_subnet/prefix | $ipv6addr | $ipv6_subnet/prefix ,... ] |
$nacl, #Optional / Default is an empty list
     comment               => $string,
#Optional
     disable               => "true" | "false"
#Optional / Default is "false"
     dnssec_dns64_enabled  => "true" | "false"
#Optional / Default is "false"
     exclude               => [ $ipv4addr | $ipv4_subnet/prefix | $ipv6addr | $ipv6_subnet/prefix ,... ] |
$nacl, #Optional / Default is an empty list
     extattrs              => { $string => $extattr, ... },
#Optional / Default is undefined
     extensible_attributes => { $string => $string|$num, $string => [ $string|$num, ...], ... },
#Optional / Default is undefined
     mapped                => [ $ipv4addr | $ipv4_subnet/prefix | $ipv6addr | $ipv6_subnet/prefix ,... ] |
$nacl, #Optional / Default is an empty list
 );
```

You cannot set both extattrs and extensible_attributes attributes at the same time.

SESSION METHODS

This section describes all the methods in the Infoblox::Session module that you can apply to a DNS64 synthesis group object.

Infoblox::Session->add( )

Use this method to add an object to the Infoblox appliance. See Infoblox::Session->add() for parameters and return values.

If you have applied an access control list to an object, you must restart services after you update the access control list for the changes to

take effect. See Infoblox::Session->restart() method.

Example

```
#Construct an object
my $group = Infoblox::Grid::DNS::DNS64Group->new(
    name        => "dns64group_A",
    prefix      => "64:FF9B::/96",
    );

#Submit for addition
my $response = $session->add( $group );
```

Infoblox::Session->get( )

Use this method to retrieve all the matching objects from the Infoblox appliance. See Infoblox::Session->get() for parameters and return values.

Key References

```
Apply the following attributes to get a specific DNS64 synthesis group object:

 name                - Optional. Group name in string format.
 prefix              - Optional. An IPv6 prefix.
 extattrs            - Optional. A hash reference containing extensible attributes.
 extensible_attributes - Optional. A hash reference containing extensible attributes.
 comment             - Optional. User-specified comment for the group.
```

Examples

```
my @retrieved_objs = $session->get(
    object => "Infoblox::Grid::DNS::DNS64Group",
    name   => "dns64group_A",
    );

my @retrieved_objs = $session->get(
    object => "Infoblox::Grid::DNS::DNS64Group",
    extensible_attributes => { 'Site' => 'Santa Clara' });
```

Infoblox::Session->modify( )

Use this method to modify an object in the Infoblox appliance. See Infoblox::Session->modify() for parameters and return values.

If you have applied an access control list to an object, you must restart services after you update the access control list for the changes to take effect. See Infoblox::Session->restart() method.

Example

```
#Use this method to modify the comment.
$group->comment("this is a modified comment");
#Submit modification
my $response = $session->modify( $group );
```

Infoblox::Session->remove( )

Use this method to remove an object from the Infoblox appliance. See Infoblox::Session->remove() for parameters and return values.

To remove a specific object, first use `get()` or `search()` to retrieve the specific object, and then submit this object for removal.

Example

```
#Get the objects with the same name
my @retrieved_objs = $session->get(
    object => "Infoblox::Grid::DNS::DNS64Group",
    name   => "dns64group_A",
     );
#Find the desired object on the retrieved list
my $desired_nsgroup= $retrieved_objs[0];
#Submit for removal
my $response = $session->remove( $desired_nsgroup );
```

Infoblox::Session->search( )

Use this method to search for DNS64 synthesis group objects in the Infoblox appliance. See Infoblox::Session->search() for parameters and return values.

Key References

```
Apply the following attributes to search for a DNS64 synthesis group object:

 name                  - Optional. Group name in string format. (regular expression)
 prefix                - Optional. An IPv6 prefix. (regular expression)
 extattrs              - Optional. A hash reference containing extensible attributes.
 extensible_attributes - Optional. A hash reference containing extensible attributes.
 comment               - Optional. User-specified comment for the group. (regular expression)
```

For more information about searching extensible attributes, see Infoblox::Grid::ExtensibleAttributeDef/Searching Extensible Attributes.

Examples

```
# Search for all DNS64 synthesis group objects
my @retrieved_objs = $session->search(
    object => "Infoblox::Grid::DNS::DNS64Group",
    name   => ".*",
    );

# Search for all DNS64 synthesis group objects with "Santa Clara" as the value for the "Site"
extensible attribute
my @retrieved_objs = $session->get(
    object => "Infoblox::Grid::DNS::DNS64Group",
    extensible_attributes => { 'Site' => 'Santa Clara' });
```

METHODS

This section describes all the methods that you can use to configure and retrieve the attribute values of a DNS64 synthesis group object.

comment( )

Use this method to set or retrieve the comment field of the DNS64 synthesis group object.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

Comment of the group in string format.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Getting comment
my $comment = $group->comment();
#Modifying comment
$group->comment("Some comment");
```

clients( )

Use this method to set or retrieve the clients ACL list of a DNS64 synthesis group object. This is the list of clients for which synthesized records are returned. The default is "any".

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

The valid value is an Infoblox::Grid::NamedACL object or an array reference that contains IPv4/IPv6 addresses and networks. To exclude addresses and networks, add "!" as a prefix to the IP addresses and networks. The default value is an empty list.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Getting clients
my $clients = $group->clients();
#Modifying clients
$group->clients(["2001::333/64", "!2001::1234", "20.0.0.0/8"]);
#Use Infoblox::Grid::NamedACL
$group->clients($nacl);
```

disable( )

Use this method to set or retrieve the disable flag of a DNS64 synthesis group object.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

Specify "true" to set the disable flag or "false" to deactivate/unset it. The default value is "false".

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get disable
my $disable = $fixed_addr->disable();
#Modify disable
$fixed_addr->disable("true");
```

dnssec_dns64_enabled( )

Use this method to set or retrieve the dnssec_dns64_enabled flag of a DNS64 synthesis group object.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

Specify "true" to set the dnssec_dns64_enabled flag or "false" to deactivate/unset it. The default value is "false".

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get dnssec_dns64_enabled
my $dnssec_dns64_enabled = $group->dnssec_dns64_enabled();
#Modify dnssec_dns64_enabled
$group->dnssec_dns64_enabled('true');
```

exclude( )

Use this method to set or retrieve the exclude ACL list of a DNS64 synthesis group object. AAAA records which match this ACL are treated as non-existent for the purposes of synthesis. By default, the ACL is empty and no records match.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

The valid value is an Infoblox::Grid::NamedACL object or an array reference that contains IPv4/IPv6 addresses and networks. To exclude addresses and networks, add "!" as a prefix to the IP addresses and networks. The default value is an empty list.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Getting exclude
my $exclude = $group->exclude();
#Modifying exclude
$group->exclude(["2001::333/64", "!2001::1234"]);
#Use Infoblox::Grid::NamedACL
$group->exclude($nacl);
```

extattrs( )

Use this method to set or retrieve the extensible attributes associated with a DNS64 synthesis group object.

Parameter

Valid value is a hash reference containing the names of extensible attributes and their associated values (Infoblox::Grid::Extattr objects).

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get extattrs
my $ref_extattrs = $group->extattrs();
#Modify extattrs
$group->extattrs({ 'Site' => $extattr1, 'Administrator' => $extattr2 });
```

extensible_attributes( )

Use this method to set or retrieve the extensible attributes associated with a DNS64 synthesis group object.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

For valid values for extensible attributes, see Infoblox::Grid::ExtensibleAttributeDef/Extensible Attribute Values.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get extensible attributes
my $ref_extensible_attributes = $group->extensible_attributes();
#Modify extensible attributes
$group->extensible_attributes({ 'Site' => 'Santa Clara', 'Administrator' => [ 'Peter', 'Tom' ] });
```

name( )

Use this method to set or retrieve the DNS64 synthesis group name.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

Name of the group in string format.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Getting name
my $name = $group->name();
#Modifying name
$group->name("ns_group_2");
```

mapped( )

Use this method to set or retrieve the mapped ACL list of a DNS64 synthesis group object. The appliance generates AAAA records only for A records that match this ACL. If the ACL is empty, as it is by default, then all records will match.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

The valid value is an Infoblox::Grid::NamedACL object or an array reference that contains IPv4/IPv6 addresses and networks. To deny addresses and networks, add "!" as a prefix to the IP addresses and networks. The default value is an empty list.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Getting mapped
my $mapped = $group->mapped();
#Modifying mapped
$group->mapped(["40.10.0.0/24", "!40.10.3.4]);
#Use Infoblox::Grid::NamedACL
$group->mapped($nacl);
```

prefix( )

Use this method to set or retrieve the DNS64 synthesis group prefix. The prefix is an IPv6 subnet prefix with a length of /32, /40, /48, /56, /64, or /96.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

Prefix of the group in string format.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Getting prefix
my $prefix = $group->prefix();
#Modifying prefix
$group->prefix("64:FF9B::/96");
```

SAMPLE CODE

The following sample code demonstrates the different functions that can be applied to an object, such as modify and remove. This sample also includes error handling for the operations.

#Preparation prior to the addition of a DNS64 synthesis group object

```
#PROGRAM STARTS: Include all the modules that will be used
use strict;
use Infoblox;

#Create a session to the Infoblox device
my $session = Infoblox::Session->new(
                                     master   => "192.168.1.2",
                                     username => "admin",
                                     password => "infoblox"
                                    );
unless ($session) {
    die("Construct session failed: ",
        Infoblox::status_code() . ":" . Infoblox::status_detail());
}
print "Session created successfully\n";
```

#Create a DNS64 synthesis group object and add it to the appliance

```
my $group = Infoblox::Grid::DNS::DNS64Group->new(
                                                 name => 'testgroup',
                                                 prefix => '64:FF9B::/96',
                                                 clients => ["2001::333/64", "!2001::1234", "20.0.0.0/8"],
                                                 comment => 'some comment',
                                                );

unless ($group) {
    die("Construct DNS64 synthesis group failed: ",
        Infoblox::status_code() . ":" . Infoblox::status_detail());
}
print "DNS64 synthesis group object created successfully\n";

my $response = $session->add( $group );
unless ($response) {
    die("Adding DNS64 synthesis group failed: ",
        Infoblox::status_code() . ":" . Infoblox::status_detail());
```

```
}
print "Add DNS64 synthesis group success.\n" if ( $response );
```

#Add the DNS64 synthesis group object to the grid and enable it

```
my @result_array = $session->get( "object" => "Infoblox::Grid::DNS" ,  "grid" => "Infoblox" );

if ( defined @result_array  and scalar( @result_array ) > 0 ) {
    my $object = $result_array[0];
    if ( ref( $object ) eq "Infoblox::Grid::DNS" ) {
        $object->dns64_groups( [ $group ] );
        $object->enable_dns64("true");

        $response = $session->modify( $object );
        print "Add DNS64 synthesis group to grid dns properties success.\n" if ( $response );
             }

    unless ($object) {
        die("Add DNS64 synthesis group to grid dns properties failed: ",
            $session->status_code() . ":" . $session->status_detail());
    }
     }
```

#Modify the DNS64 synthesis group object

```
@result_array = $session->get( "object" => "Infoblox::Grid::DNS::DNS64Group" ,  name => "testgroup" );

if ( defined @result_array  and scalar( @result_array ) > 0 ) {
    my $object = $result_array[0];
    if ( ref( $object ) eq "Infoblox::Grid::DNS::DNS64Group" ) {

        $object->clients(["2001::333/64", "!2001::1234"]);

        $response = $session->modify( $object );
        print "Modify DNS64 synthesis group success.\n" if ( $response );
             }

    unless ($object) {
        die("Modify Group failed: ",
            $session->status_code() . ":" . $session->status_detail());
    }
     }
```

#Remove a DNS64 synthesis group object

#First let's remove it from the grid configuration

```
@result_array = $session->get( "object" => "Infoblox::Grid::DNS" ,  "grid" => "Infoblox" );

if ( defined @result_array  and scalar( @result_array ) > 0 ) {
    my $object = $result_array[0];
    if ( ref( $object ) eq "Infoblox::Grid::DNS" ) {

        $object->dns64_groups( [ ] );
        $object->enable_dns64( "false" );

        $response = $session->modify( $object );
        print "Remove DNS64 synthesis group from the grid success.\n" if ( $response );
             }

    unless ($object) {
        die("Remove DNS64 synthesis group failed: ",
            $session->status_code() . ":" . $session->status_detail());
    }
     }
```

#And now remove the group

```
@result_array = $session->get( "object" => "Infoblox::Grid::DNS::DNS64Group" ,  name => "testgroup" );

if ( defined @result_array  and scalar( @result_array ) > 0 ) {
    my $object = $result_array[0];
    if ( ref( $object ) eq "Infoblox::Grid::DNS::DNS64Group" ) {

        $response = $session->remove( $object );
        print "Remove DNS64 synthesis group success.\n" if ( $response );
```

```
            }

    unless ($object) {
        die("Remove DNS64 synthesis group failed: ",
            $session->status_code() . ":" . $session->status_detail());
    }
     }

 ####PROGRAM ENDS####
```

AUTHOR

Infoblox Inc. http://www.infoblox.com/

SEE ALSO

Infoblox::Session,Infoblox::Session->get(),Infoblox::Session->modify(),Infoblox::Session->search(),Infoblox::Session->remove()Infoblox::Grid::DNS, Infoblox::Grid::Member::DNS, Infoblox::DNS::View, Infoblox::Grid::NamedACL

COPYRIGHT

Copyright (c) 2017 Infoblox Inc.

Infoblox::Grid::DNS::FileTransferSetting - A DNS traffic capture file transfer setting object.

- NAME
- DESCRIPTION
- CONSTRUCTOR
- SESSION METHODS
- MODULE METHODS
  - Infoblox::Grid::DNS->file_transfer_setting( )
  - Infoblox::Grid::Member::DNS->file_transfer_setting( )
- METHODS
  - directory( )
  - host( )
  - password( )
  - type( )
  - username( )
- AUTHOR
- SEE ALSO
- COPYRIGHT

NAME

Infoblox::Grid::DNS::FileTransferSetting - A DNS traffic capture file transfer setting object.

DESCRIPTION

The DNS traffic capture file transfer setting object provides information and configuration for captured DNS traffic transfer to a remote server.

CONSTRUCTOR

```
my $transfer_setting = Infoblox::Grid::DNS::FileTransferSetting->new(
    type      => 'NONE' | 'FTP' | 'SCP', # Required
    directory => $string,                # Optional / default is undefined
    host      => $string,                # Optional / default is undefined
    username  => $string,                # Optional / default is undefined
    password  => $string,                # Optional / default is undefined
);
```

SESSION METHODS

The object does not support any session methods.

MODULE METHODS

The following functions are available to be applied to a DNS traffic capture file transfer setting object.

Infoblox::Grid::DNS->file_transfer_setting( )

Use this method to set or retrieve the DNS capture file transfer settings. See Infoblox::Grid::DNS->file_transfer_setting() for parameters and return values.

Example

```
#Get override_enable_excluded_domain_names value
my $override_enable_excluded_domain_names = $object->override_enable_excluded_domain_names();

#Modify override_enable_excluded_domain_names value
$object->override_enable_excluded_domain_names("true");
```

Infoblox::Grid::Member::DNS->file_transfer_setting( )

Use this method to set or retrieve the DNS capture file transfer settings. See Infoblox::Grid::Member::DNS->file_transfer_setting() for parameters and return values.

Example

```
#Get override_enable_excluded_domain_names value
my $override_enable_excluded_domain_names = $object->override_enable_excluded_domain_names();
```

```
#Modify override_enable_excluded_domain_names value
$object->override_enable_excluded_domain_names("true");
```

METHODS

This section describes all the methods that can be used to configure and retrieve the attribute values of a DNS traffic capture file transfer setting object.

directory( )

Use this method to set or retrieve the directory to save the captured DNS queries and responses.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

The valid value is a desired path in a string format.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get directory value
my $directory = $object->directory();

#Modify directory value
$object->directory('/path/to/dir');
```

host( )

Use this method to set or retrieve the hostname for the DNS capture transfer destination server.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

The valid value is Fully-Qualified Domain Name (FQDN) of the DNS capture transfer destination server.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get host value
my $host = $object->host();

#Modify host value
$object->host('transfer.com');
```

password( )

Use this method to set the password to access the destination server directory. This is a write-only attribute.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

The valid value is a desired password in a string format.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Modify password value
$object->password('pass');
```

type( )

Use this method to set or retrieve the transfer protocol used to capture DNS queries and responses.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

The valid value are 'FTP', 'SCP' and 'NONE'.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get type value
my $type = $object->type();

#Modify type value
$object->type('SCP');
```

username( )

Use this method to set or retrieve the username to access the destination server directory.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

The valid value is a desired username in a string format.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get username value
my $username = $object->username();

#Modify username value
$object->username('admin');
```

AUTHOR

Infoblox Inc. http://www.infoblox.com/

SEE ALSO

Infoblox::Grid::DNS, Infoblox::Grid::DNS->file_transfer_setting(), Infoblox::Grid::Member::DNS, Infoblox::Grid::Member::DNS->file_transfer_setting()

COPYRIGHT

Copyright (c) 2017 Infoblox Inc.

Infoblox::Grid::DNS::FixedRRSetOrderFQDN - A fixed RRset order FQDN object.

- NAME
- DESCRIPTION
- CONSTRUCTOR
- SESSION METHODS
- METHODS
  - fqdn( )
  - record_type( )
- AUTHOR
- SEE ALSO
- COPYRIGHT

NAME

Infoblox::Grid::DNS::FixedRRSetOrderFQDN - A fixed RRset order FQDN object.

DESCRIPTION

A fixed RRset order FQDN contains information about the fixed RRset configuration items.

By default, when a client queries a domain name, the DNS caching appliance returns the A and AAAA records of the domain name in the cyclic order. However, this default behavior can be overridden if you have enabled and configured (at the Grid level) fixed RRset order for hosts that have multiple addresses. When you override the default behavior and preserve the fixed RRset order for cached DNS responses, the DNS caching appliance returns A and AAAA records associated with domain names in the order they were received from an upstream server. You can preserve the RRset order for the cached DNS responses and specify the fixed RRset order for A, AAAA, or both A and AAAA records at the Grid level and override at the member and DNS view levels.

Note that configuring fixed RRset order for specific FQDNs might significantly affect the performance of the DNS caching appliance.

CONSTRUCTOR

```
my $rrset_order = Infoblox::Grid::DNS::FixedRRSetOrderFQDN->new(
    fqdn        => $string,               #Required
    record_type => 'A' | 'AAAA' | 'BOTH', #Optional / Default is 'A'
);
```

SESSION METHODS

The object does not support any session methods. It's used for fixed RRset order configuration for Infoblox::Grid::DNS, Infoblox::Grid::Member::DNS, Infoblox::DNS::View objects.

METHODS

This section describes all the methods that you can use to set or retrieve the attribute values of the object.

fqdn( )

Use this method to set or retrieve the FQDN of the fixed RRset configuration item.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

The valid value is a string containing desired FQDN.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get fqdn value
my $fqdn = $order_fqdn->fqdn();
#Modify fqdn
$order_fqdn->fqdn('domain.com');
```

record_type( )

Use this method to set or retrieve the returned record type for the specified FQDN in the fixed RRset configuration.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

The valid values are 'A', 'AAAA' and 'BOTH'. The default value is 'A'.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get record_type value
my $record_type = $order_fqdn->record_type();
#Modify record_type
$order_fqdn->record_type('AAAA');
```

AUTHOR

Infoblox Inc. http://www.infoblox.com/

SEE ALSO

Infoblox::Grid::DNS, Infoblox::Grid::Member::DNS, Infoblox::DNS::View

---

COPYRIGHT

Copyright (c) 2017 Infoblox Inc.

Infoblox::Grid::DNS::Nsgroup - Creates a name server group.

- NAME
- DESCRIPTION
- CONSTRUCTOR
- SESSION METHODS
  - Infoblox::Session->add( )
  - Infoblox::Session->get( )
  - Infoblox::Session->modify( )
  - Infoblox::Session->remove( )
  - Infoblox::Session->search( )
- MODULE METHODS
  - Infoblox::Grid::DNS->nsgroups( )
- METHODS
  - name( )
  - comment( )
  - extensible_attributes( )
  - is_multimaster( )
  - multiple_primaries( )
  - secondaries( )
- SAMPLE CODE
- AUTHOR
- SEE ALSO
- COPYRIGHT

NAME

Infoblox::Grid::DNS::Nsgroup - Creates a name server group.

DESCRIPTION

A name server group is a collection of one and more primary DNS servers and one or more secondary DNS servers. Grouping a commonly used set of primary and secondary DNS servers together simplifies zone creation, allowing you to specify a single name server group instead of specifying multiple name servers individually.

CONSTRUCTOR

```
 my $nsg1 = Infoblox::Grid::DNS::Nsgroup->new(
     name         => $string,                                                          #Required
     comment      => $string,                                                          #Optional
     extattrs              => { $string => $extattr, ... },                            #Optional
/ Default is undefined
     extensible_attributes => { $string => $string|$num, $string => [ $string|$num, ...], ... },  #Optional
/ Default is undefined
     is_multimaster       => "true" | "false",                                        #Optional
/ Default is "false"
     multiple_primaries   => [$Member1, ...] | [$Nameserver1, ...] | [$MSServer1, ...],  #Optional
/ Default is undefined
     secondaries          => [$Member1, $Member2, $Nameserver1, $Nameserver2, ...]     #Optional
/ List of Infoblox::DNS::Member object(s) and/or Infoblox::DNS::Nameserver object(s).
 );
```

You cannot set both extattrs and extensible_attributes attributes at the same time.

SESSION METHODS

This section describes all the methods in the Infoblox::Session module that you can apply to an Nsgroup object.

Infoblox::Session->add( )

Use this method to add an object to the Infoblox appliance. See Infoblox::Session->add() for parameters and return values.

Example

```
#Creating Member objects.
my $member1 = Infoblox::DNS::Member->new(
    name     => "infoblox.localdomain",
    ipv4addr => "192.168.1.2",
    stealth  => "false"
    );
```

```
my $member2 = Infoblox::DNS::Member->new(
    name     => "member2.test.com",
    ipv4addr => "2.2.2.2",
    stealth  => "false"
    );

#Creating a nameserver object.
my $nameserver1 = Infoblox::DNS::Nameserver->new(
    name     => "ns1.test.extra",
    ipv4addr => "4.4.4.4",
    stealth  => "false",
    TSIGname => "tsig_name",
    TSIGkey  => "ZX1jVJl7C58GT/sc7Q3ucA=="
    );

#Creating a nameserver object.
my $nameserver2 = Infoblox::DNS::Nameserver->new(
    name     => "ns2.test.extra",
    ipv4addr => "5.5.5.5",
    stealth  => "false",
    TSIGname => "tsig_name2",
    TSIGkey  => "qwqwqwqwqwqwqwqwqwqA=="
    );

#Creating a nameserver object.
my $nameserver3 = Infoblox::DNS::Nameserver->new(
    name     => "ns3.test.extra",
    ipv6addr => "12ab::34a",
    stealth  => "false",
    TSIGname => "tsig_name2",
    TSIGkey  => "qwqwqwqwqwqwqwqwqwqA=="
    );

#Construct an object
my $nsg1 = Infoblox::Grid::DNS::Nsgroup->new(
    name        => "ns_group_A",
    multiple_primaries => [$member1],
    secondaries => [$member2, $nameserver1, $nameserver2, $nameserver3]
    );

#Submit for addition
my $response = $session->add( $nsg1 );
```

Infoblox::Session->get( )

Use this method to retrieve all the matching objects from the Infoblox appliance. See Infoblox::Session->get() for parameters and return values.

Key References

```
Apply the following attributes to get a specific Nsgroup object:

 name - Required. A name server group name in string format.
 extattrs      - Optional. A hash reference containing extensible attributes.
 extensible_attributes - Optional. A hash reference containing extensible attributes.
```

Examples

```
my @retrieved_objs = $session->get(
    object => "Infoblox::Grid::DNS::Nsgroup",
    name   => "ns_group_A",
    );

my @retrieved_objs = $session->get(
    object => "Infoblox::Grid::DNS::Nsgroup",
    extensible_attributes => { 'Site' => 'Santa Clara' });
```

Infoblox::Session->modify( )

Use this method to modify an object in the Infoblox appliance. See Infoblox::Session->modify() for parameters and return values.

Example

```
#Use this method to modify the comment.
$nsg1->comment("this is a modified comment");
#Submit modification
my $response = $session->modify( $nsg1 );
```

Infoblox::Session->remove( )

Use this method to remove an object from the Infoblox appliance. See Infoblox::Session->remove() for parameters and return values.

To remove a specific object, first use `get()` or `search()` to retrieve the specific object, and then submit this object for removal.

Example

```
#Get the objects with the same name
my @retrieved_objs = $session->get(
    object => "Infoblox::Grid::DNS::Nsgroup",
    name   => "ns_group_A",
     );
#Find the desired object from the retrieved list.
my $desired_nsgroup= $retrieved_objs[0];
#Submit for removal
my $response = $session->remove( $desired_nsgroup );
```

Infoblox::Session->search( )

Use this method to search for Nsgroup objects in the Infoblox appliance. See Infoblox::Session->search() for parameters and return values.

Key References

```
Apply the following attributes to search for an Nsgroup object:

 name - Required. A host name in string format (regular expression).
 extattrs     - Optional. A hash reference containing extensible attributes.
 extensible_attributes - Optional. A hash reference containing extensible attributes.
```

For more information about searching extensible attributes, see Infoblox::Grid::ExtensibleAttributeDef/Searching Extensible Attributes.

Examples

```
# Search for all Nsgroup objects
my @retrieved_objs = $session->search(
    object => "Infoblox::Grid::DNS::Nsgroup",
    name   => ".*",
    );

# search for all Nsgroup objects with "Santa Clara" as the value for the "Site" extensible attribute
my @retrieved_objs = $session->get(
    object => "Infoblox::Grid::DNS::Nsgroup",
    extensible_attributes => { 'Site' => 'Santa Clara' });
```

MODULE METHODS

This section describes all the methods that you can appy to an Nsgroup object.

Infoblox::Grid::DNS->nsgroups( )

Use this function to specify a name server group at the grid level on the Infoblox appliance. See Infoblox::Grid::DNS->nsgroups() for parameters and return values.

Example

```
#Creating Member objects.

my $member1 = Infoblox::DNS::Member->new(
    name     => "infoblox.localdomain",
    ipv4addr => "192.168.1.2",
    stealth  => "false"
    );

my $member2 = Infoblox::DNS::Member->new(
    name     => "member2.test.com",
    ipv4addr => "2.2.2.2",
    stealth  => "false"
    );

#Creating a nameserver object.
my $nameserver1 = Infoblox::DNS::Nameserver->new(
    name     => "ns1.test.extra",
    ipv4addr => "4.4.4.4",
```

```
    stealth  => "false",
    TSIGname => "tsig_name",
    TSIGkey  => "ZX1jVJl7C58GT/sc7Q3ucA=="
    );

#Creating a nameserver object.
my $nameserver2 = Infoblox::DNS::Nameserver->new(
    name     => "ns2.test.extra",
    ipv4addr => "5.5.5.5",
    stealth  => "false",
    TSIGname => "tsig_name2",
    TSIGkey  => "qwqwqwqwqwqwqwqwqwqA=="
    );

#Creating a nameserver object.
my $nameserver3 = Infoblox::DNS::Nameserver->new(
    name     => "ns3.test.extra",
    ipv6addr => "12ab::34a",
    stealth  => "false",
    TSIGname => "tsig_name2",
    TSIGkey  => "qwqwqwqwqwqwqwqwqwqA=="
    );

my $nsg1 = Infoblox::Grid::DNS::Nsgroup->new(
    name        => "ns_group_A",
    multiple_primaries => [$member1],
    secondaries => [$member2, $nameserver1, $nameserver2,$nameserver3]
    );

my @result_array = $session->get( "object" => "Infoblox::Grid::DNS" ,  "grid" => "Infoblox" );

 my $dns = $result_array[0];
 $dns->nsgroups([$nsg1]);
```

METHODS

This section describes all the methods that you can use to configure and retrieve the attribute values of an Nsgroup object

name( )

Use this method to set or retrieve the name server group, providing a meaningful reference for the set of servers.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

Name of the nameserver group in string format.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
 #Getting name
  my $name = $nsg1->name();
  #Modifying name
  $nsg1->name("ns_group_2");
```

comment( )

Use this method to set or retrieve the comment field of the Nsgroup object. You can enter additional information about the NS group in this field.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

Comment of the nameserver group in string format.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Getting comment
my $comment = $nsg1->comment();
#Modifying comment
$nsg1->comment("Some comment");
```

extensible_attributes( )

Use this method to set or retrieve the extensible attributes associated with an Nsgroup object.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

For valid values for extensible attributes, see Infoblox::Grid::ExtensibleAttributeDef/Extensible Attribute Values.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get extensible attributes
my $ref_extensible_attributes = $nsg1->extensible_attributes();
#Modify extensible attributes
$nsg1->extensible_attributes({ 'Site' => 'Santa Clara', 'Administrator' => [ 'Peter', 'Tom' ] });
```

is_multimaster( )

Use this method to set or retrieve the is_multimaster flag.

Parameter

Specify "true" to set the is_multimaster flag or "false" to deactivate/unset it. The default value is "false".

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
# Get the attribute value
my $attr = $nsg1->is_multimaster();
# Modify the attribute value
$nsg1->is_multimaster('true');
```

multiple_primaries( )

Use this method to set or retrieve Grid primary servers or external primary servers of the name server group.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

The valid value is an array reference that contains Infoblox::DNS::Member objects for Grid primary servers or Infoblox::DNS::Nameserver objects for external primary servers.

To set two and more Infoblox::DNS::Member objects you should set the is_multimaster flag to "true".

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#For grid primary:

my $member1 = Infoblox::DNS::Member->new(
    name     => "infoblox.localdomain",
    ipv4addr => "192.168.1.2",
```

```
    stealth  => "true"
);

$nsg1->multiple_primaries([$member1]);

#For external primary:

my $nameserver1 = Infoblox::DNS::Nameserver->new(
    name     => "ns1.domain.extra",
    ipv4addr => "5.5.5.5",
    stealth  => "false",
    TSIGname => "tsig_name",
    TSIGkey  => "ZX1jVJl7C58GT/sc7Q3ucA=="
);

my $nameserver2 = Infoblox::DNS::Nameserver->new(
    name     => "ns2.domain.extra",
    ipv4addr => "6.6.6.6",
    stealth  => "false",
    TSIGname => "tsig_name2",
    TSIGkey  => "qwqwqwqwqwqwqwqwqwqA=="
);

#Creating a nameserver object.
my $nameserver3 = Infoblox::DNS::Nameserver->new(
    name     => "ns3.test.extra",
    ipv6addr => "12ab::34a",
    stealth  => "false",
    TSIGname => "tsig_name2",
    TSIGkey  => "qwqwqwqwqwqwqwqwqwqA=="
    );

$nsg1->multiple_primaries([$nameserver1, $nameserver2,$nameserver3]);
```

secondaries( )

Use this method to set or retrieve the grid secondary server or external secondary servers of the nameserver group.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

The valid value is an array reference that contains Infoblox::DNS::Member objects for grid secondary servers and Infoblox::DNS::Nameserver objects for external secondary servers.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails. If you did not specify a parameter, the method returns the attribute value.

Example

```
my $member2 = Infoblox::DNS::Member->new(
    name     => "member2.domain.com",
    ipv4addr => "2.2.2.2",
    stealth  => "false"
);

my $nameserver1 = Infoblox::DNS::Nameserver->new(
    name     => "ns1.domain.extra",
    ipv4addr => "4.4.4.4",
    stealth  => "false",
    TSIGname => "tsig_name",
    TSIGkey  => "ZX1jVJl7C58GT/sc7Q3ucA=="
);

my $nameserver2 = Infoblox::DNS::Nameserver->new(
    name     => "ns2.domain.extra",
    ipv4addr => "5.5.5.5",
    stealth  => "false",
    TSIGname => "tsig_name2",
    TSIGkey  => "qwqwqwqwqwqwqwqwqwqA=="
);

#Creating a nameserver object.
my $nameserver3 = Infoblox::DNS::Nameserver->new(
```

```
            name     => "ns3.test.extra",
            ipv6addr => "12ab::34a",
            stealth  => "false",
            TSIGname => "tsig_name2",
            TSIGkey  => "qwqwqwqwqwqwqwqwqwqA=="
            );

      $nsg1->secondaries([$member2, $nameserver1, $nameserver2,$nameserver3]);
```

SAMPLE CODE

The following sample code demonstrates the different functions that can be applied to an object, such as modify and remove. This sample also includes error handling for the operations.

#Preparation prior to an Nsgroup object insertion

```
#PROGRAM STARTS: Include all the modules that will be used
use strict;
use Infoblox;

#Create a session to the Infoblox device
my $session = Infoblox::Session->new(
    master   => "192.168.1.2",
    username => "admin",
    password => "infoblox"
);
unless ($session) {
   die("Construct session failed: ",
       Infoblox::status_code() . ":" . Infoblox::status_detail());
}
print "Session created successfully\n";
```

#Create an Nsgroup object

```
#Creating a member object.
my $member1 = Infoblox::DNS::Member->new(
    name     => "infoblox.localdomain",
    ipv4addr => "192.168.1.2",
    stealth  => "false"
);

#Creating a nsgroup object.
my $nsg1 = Infoblox::Grid::DNS::Nsgroup->new(
    name    => "ns_group_1",
    multiple_primaries => [$member1]
);

my $nsg1_A = Infoblox::Grid::DNS::Nsgroup->new(
    name    => "ns_group_A",
    multiple_primaries => [$member1]
);

unless ($nsg1) {
   die("Construct Nsgroup failed: ",
       Infoblox::status_code() . ":" . Infoblox::status_detail());
}
print "Nsgroup object created successfully\n";
```

#Get and add the Nsgroup object

```
my @result_array = $session->get( "object" => "Infoblox::Grid::DNS" ,  "grid" => "Infoblox" );

if( defined @result_array  and scalar( @result_array ) > 0 )
{
       my $object = $result_array[0];
       if( ref( $object ) eq "Infoblox::Grid::DNS" )
       {

               $object->nsgroups( [ $nsg1 ,$nsg1_A] );

               my $response = $session->modify( $object );
               print "Add Nsgroup to grid dns properties success.\n" if ( $response );
                     }

 unless ($object) {
    die("Add Nsgroup to grid dns properties failed: ",
```

```
        $session->status_code() . ":" . $session->status_detail());
 }
  }
```

#Modify the Nsgroup object

```
my @result_array = $session->get( "object" => "Infoblox::Grid::DNS" ,  "grid" => "Infoblox" );

if( defined @result_array  and scalar( @result_array ) > 0 )
{
       my $object = $result_array[0];
       if( ref( $object ) eq "Infoblox::Grid::DNS" )
       {

          $nsg1_A->name("ns_group_modified");

         $object->nsgroups( [ $nsg1 ,$nsg1_A] );

               my $response = $session->modify( $object );
               print "Modify Nsgroup success.\n" if ( $response );
                        }

 unless ($object) {
    die("Modify NSgroup failed: ",
        $session->status_code() . ":" . $session->status_detail());
 }
  }
```

#Remove a Nsgroup object

```
my @result_array = $session->get( "object" => "Infoblox::Grid::DNS" ,  "grid" => "Infoblox" );

if( defined @result_array  and scalar( @result_array ) > 0 )
{
       my $object = $result_array[0];
       if( ref( $object ) eq "Infoblox::Grid::DNS" )
       {

               $object->nsgroups( [ ] );

               my $response = $session->modify( $object );
               print "Remove Nsgroup success.\n" if ( $response );
                        }

 unless ($object) {
    die("Remove Nsgroup failed: ",
        $session->status_code() . ":" . $session->status_detail());
 }
  }

####PROGRAM ENDS####
```

AUTHOR

Infoblox Inc. http://www.infoblox.com/

SEE ALSO

Infoblox::Session, Infoblox::Session->get(),Infoblox::Session->modify(),Infoblox::Grid::DNS,
Infoblox::DNS::Member,Infoblox::DNS::Nameserver

COPYRIGHT

Copyright (c) 2017 Infoblox Inc.

Infoblox::Grid::DNS::Nsgroup::DelegationMember - A delegation nameservers group.

- NAME
- DESCRIPTION
- CONSTRUCTOR
- SESSION METHODS
  - Infoblox::Session->add( )
  - Infoblox::Session->get( )
  - Infoblox::Session->modify( )
  - Infoblox::Session->remove( )
  - Infoblox::Session->search( )
- METHODS
  - comment( )
  - extattrs( )
  - extensible_attributes( )
  - delegation_servers( )
  - name( )
- SAMPLE CODE
- AUTHOR
- SEE ALSO
- COPYRIGHT

NAME

Infoblox::Grid::DNS::Nsgroup::DelegationMember - A delegation nameservers group.

DESCRIPTION

When you configure a name server group, you can now create a set of external name servers as a delegation name server group and assign it to delegated zones. Specifying a single delegation name server group instead of configuring multiple name servers individually for delegated zones can significantly reduce configuration efforts.

CONSTRUCTOR

```
 my $object = Infoblox::Grid::DNS::Nsgroup::DelegationMember->new(
     name                  => $string,
#Required
     delegation_servers    => [$nameserver1, $nameserver1, ...],
#Required
     comment               => $string,
#Optional / Default is undefined
     extattrs              => { $string => $extattr, ... },
#Optional / Default is undefined
     extensible_attributes => {$string => $string | $num, $string => [ $string | $num, ... ], ... }
#Optional / Default is undefined
 );
```

You cannot set both extattrs and extensible_attributes attributes at the same time.

SESSION METHODS

This section describes all the methods in an Infoblox::Session module that you can apply to a delegation members group object.

Infoblox::Session->add( )

Use this method to add the object to the Infoblox appliance. See Infoblox::Session->add() for parameters and return values.

Example

```
#Construct an object
my $object = Infoblox::Grid::DNS::Nsgroup::DelegationMember->new(
    name               => 'dlggroup',
    delegation_servers => [$nameserver1, $nameserver2],
);

#Submit for addition
my $response = $session->add($object);
```

Infoblox::Session->get( )

Use this method to retrieve all the matching objects from the Infoblox appliance. See Infoblox::Session->get() for parameters and return values.

Key Reference

```
Apply the following attributes to get a specific delegation member group object:

 comment               - Optional. The delegation member group comment in a string format.
 name                  - Optional. The delegation member group name in a string format.
 extattrs              - Optional. A hash reference containing extensible attributes.
 extensible_attributes - Optional. A hash reference containing extensible attributes.
```

For more information about searching extensible attributes, see Infoblox::Grid::ExtensibleAttributeDef/Searching Extensible Attributes.

Example

```
my @retrieved_objs = $session->get(
    object => 'Infoblox::Grid::DNS::Nsgroup::DelegationMember',
    name   => 'dlggroup',
);
```

Infoblox::Session->modify( )

Use this method to modify the object in the Infoblox appliance. See Infoblox::Session->modify() for parameters and return values.

Example

```
#Modify comment value
$object->comment('this is a modified comment');

#Submit modification
my $response = $session->modify($object);
```

Infoblox::Session->remove( )

Use this method to remove the object from the Infoblox appliance. See Infoblox::Session->remove() for parameters and return values.

To remove a specific object, first use `get()` or `search()` to retrieve the specific DHCP range object, and then submit this object for removal.

Example

```
#Get the objects
my @retrieved_objs = $session->get(
    object => 'Infoblox::Grid::DNS::Nsgroup::DelegationMember',
    name   => 'dlggroup',
);

#Find the desired object from the retrieved list
my $desired_object = $retrieved_objs[0];

#Submit for removal
my $response = $session->remove($desired_object);
```

Infoblox::Session->search( )

Use this method to search for objects in the Infoblox appliance. See Infoblox::Session->search() for parameters and return values.

Key References

```
Apply the following attributes to search for the delegation member group object:

 comment               - Optional. The delegation member group comment in a string format (regexp).
 name                  - Optional. The delegation member group name in a string format (regexp).
 extattrs              - Optional. A hash reference containing extensible attributes.
 extensible_attributes - Optional. A hash reference containing extensible attributes.
```

For more information about searching extensible attributes, see Infoblox::Grid::ExtensibleAttributeDef/Searching Extensible Attributes.

Example

```
# search for objects
my @retrieved_objs = $session->search(
    object  => 'Infoblox::Grid::DNS::Nsgroup::DelegationMember',
    name    => '^dlg.*',
    comment => '.*comment',
);
```

```
# search for objects defining 'Santa Clara' for 'Site' extensible attribute
my @retrieved_objs = $session->search(
    object               => 'Infoblox::Grid::DNS::Nsgroup::DelegationMember',
    extensible_attributes => {'Site' => 'Santa Clara'},
);
```

METHODS

This section describes all the methods that you can use to configure and retrieve the attribute values of a delegation member group.

comment( )

Use this method to set or retrieve the delegation member group comment.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

The valid value is a desired comment in a string format.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get comment value
my $comment = $object->comment();

#Modify comment value
$object->comment('dlg comment');
```

extattrs( )

Use this method to set or retrieve the extensible attributes associated with a delegation member group object.

Parameter

Valid value is a hash reference that contains the names of extensible attributes and their associated values (Infoblox::Grid::Extattr objects).

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get extattrs value
my $extattrs = $object->extattrs();

#Modify extattrs value
$object->extattrs({'Site' => $extattr1, 'Administrator' => $extattr2});
```

extensible_attributes( )

Use this method to set or retrieve the extensible attributes associated with a delegation member group object.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

For valid values for extensible attributes, see Infoblox::Grid::ExtensibleAttributeDef/Extensible Attribute Values.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get extensible attributes value
my $extensible_attributes = $object->extensible_attributes();
```

```
#Modify extensible attributes
$object->extensible_attributes({'Site' => 'Santa Clara', 'Administrator' => ['Peter', 'Tom']});
```

delegation_servers( )

Use this method to set or retrieve the list of delegation servers.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

The valid value is an array of Infoblox::DNS::Nameserver objects.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get delegation_servers value
my $delegation_servers = $object->delegation_servers();

#Modify delegation_servers value

#Create nameserver object
my $nameserver = Infoblox::DNS::Nameserver->new(
    name     => 'ns1.domain1.extra',
    ipv4addr => '5.5.5.5',
);

#Set delegation_servers value
$object->delegation_servers([$nameserver]);
```

name( )

Use this method to set or retrieve the delegation member group name.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

The valid value is a desired name in a string format.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get name value
my $name = $object->name();

#Modify name value
$object->name('dlgg1');
```

SAMPLE CODE

The following sample code demonstrates the different functions that can be applied to an object, such as modify and remove. This sample also includes error handling for the operations.

#Preparation prior to an Delegation Member Nsgroup object insertion

```
#PROGRAM STARTS: Include all the modules that will be used
use strict;
use Infoblox;

my ($session, $result);

#Create a session to the Infoblox device
$session = Infoblox::Session->new(
    master   => "192.168.1.2",
    username => "admin",
    password => "infoblox"
);
```

```
unless ($session) {
   die("Construct session failed: ",
       Infoblox::status_code() . ":" . Infoblox::status_detail());
}
print "Session created successfully\n";
```

#Create an Delegation Member Nsgroup object

```
#Creating an external nameserver
my $nameserver = Infoblox::DNS::Nameserver->new(
    name      => "ns1.domain0.external",
    ipv4addr => "5.5.5.5",
);

#Creating a delegation nsgroup object.
my $dlg_nsg1 = Infoblox::Grid::DNS::Nsgroup::DelegationMember->new(
    name               => "dlg_group_1",
    delegation_servers => [$nameserver],
);

unless ($dlg_nsg1) {
   die("Construct Delegation Member Nsgroup failed: ",
       Infoblox::status_code() . ":" . Infoblox::status_detail());
}

print "Delegation Member Nsgroup object constructed successfully\n";
```

#Add the Nsgroup object

```
$result = $session->add($dlg_nsg1);

unless ($result) {
    die("Add Delegation Member Nsgroup to session failed: ",
        $session->status_code() . ":" . $session->status_detail());
}

print "Delegation Member Nsgroup object created successfully\n";
```

#Search and Modify the Nsgroup object

```
my @result_array = $session->search(
    object => "Infoblox::Grid::DNS::Nsgroup::DelegationMember",
    name   => "dlg_.*",
);

my $object = $result_array[0];

unless ($object) {
   die("Search for Delegation Member Nsgroup failed: ",
       $session->status_code() . ":" . $session->status_detail());
}

$result = $object->comment("delegation member");

unless ($result) {
    die("Modify Delegation Member Nsgroup failed: ",
        Infoblox::status_code() . ":" . Infoblox::status_detail());
}

$result = $session->modify($object);

unless ($result) {
    die("Submit modification to the session failed: ",
        $session->status_code() . ":" . $session->status_detail());
}

print "Delegation Member Nsgroup object modified successfully\n";
```

#Remove a Nsgroup object

```
my @result_array = $session->get(
    object => "Infoblox::Grid::DNS::Nsgroup::DelegationMember",
    name   => "dlg_group_1",
);

my $object = $result_array[0];
```

```
unless ($object) {
   die("Get Delegation Member Nsgroup failed: ",
       $session->status_code() . ":" . $session->status_detail());
}

$result = $session->remove($object);

unless ($result) {
    die("Remove Nsgroup failed: ",
        $session->status_code() . ":" . $session->status_detail());
}

print "Delegation Member Nsgroup object removed successfully\n";

####PROGRAM ENDS####
```

AUTHOR

Infoblox Inc. http://www.infoblox.com/

SEE ALSO

Infoblox::Session, Infoblox::Session->add(), Infoblox::Session->get(), Infoblox::Session->modify(), Infoblox::Session->remove(), Infoblox::Session->search(), Infoblox::DNS::Nameserver, Infoblox::Grid::ExtensibleAttributeDef/Extensible Attribute Values, Infoblox::Grid::Extattr

---

COPYRIGHT

Copyright (c) 2017 Infoblox Inc.

Infoblox::Grid::DNS::Nsgroup::ForwardingMember - A forwarding member group.

- NAME
- DESCRIPTION
- CONSTRUCTOR
- SESSION METHODS
  - Infoblox::Session->add( )
  - Infoblox::Session->get( )
  - Infoblox::Session->modify( )
  - Infoblox::Session->remove( )
  - Infoblox::Session->search( )
- METHODS
  - comment( )
  - extattrs( )
  - extensible_attributes( )
  - forwarding_servers( )
  - name( )
- SAMPLE CODE
- AUTHOR
- SEE ALSO
- COPYRIGHT

NAME

Infoblox::Grid::DNS::Nsgroup::ForwardingMember - A forwarding member group.

DESCRIPTION

The forwarding member group provides forwarding servers configuration for forward zones.

CONSTRUCTOR

```
 my $object = Infoblox::Grid::DNS::Nsgroup::ForwardingMember->new(
     name                  => $string,
#Required
     forwarding_servers    => [$dnsmember1, $dnsmember2, ...],
#Required
     comment               => $string,
#Optional / Default is undefined
     extattrs              => { $string => $extattr, ... },
#Optional / Default is undefined
     extensible_attributes => {$string => $string | $num, $string => [ $string | $num, ... ], ... }
#Optional / Default is undefined
 );
```

You cannot set both extattrs and extensible_attributes attributes at the same time.

SESSION METHODS

This section describes all the methods in an Infoblox::Session module that you can apply to a forwarding members group object.

Infoblox::Session->add( )

Use this method to add the object to the Infoblox appliance. See Infoblox::Session->add() for parameters and return values.

Example

```
#Construct an object
my $object = Infoblox::Grid::DNS::Nsgroup::ForwardingMember->new(
    name             => 'fwdgroup',
    forwarding_servers => [$dnsmember1, $dnsmember2],
);

#Submit for addition
my $response = $session->add($object);
```

Infoblox::Session->get( )

Use this method to retrieve all the matching objects from the Infoblox appliance. See Infoblox::Session->get() for parameters and return values.

Key Reference

```
Apply the following attributes to get a specific forwarding member group object:

 comment               - Optional. The forwarding member group comment in a string format.
 name                  - Optional. The forwarding member group name in a string format.
 extattrs              - Optional. A hash reference containing extensible attributes.
 extensible_attributes - Optional. A hash reference containing extensible attributes.
```

For more information about searching extensible attributes, see Infoblox::Grid::ExtensibleAttributeDef/Searching Extensible Attributes.

Example

```
my @retrieved_objs = $session->get(
    object => 'Infoblox::Grid::DNS::Nsgroup::ForwardingMember',
    name   => 'fwdgroup',
);

# search for objects defining 'Santa Clara' for 'Site' extensible attribute
my @retrieved_objs = $session->search(
    object               => 'Infoblox::Grid::DNS::Nsgroup::ForwardingMember',
    extensible_attributes => {'Site' => 'Santa Clara'},
);
```

Infoblox::Session->modify( )

Use this method to modify the object in the Infoblox appliance. See Infoblox::Session->modify() for parameters and return values.

Example

```
#Modify comment value
$object->comment('this is a modified comment');

#Submit modification
my $response = $session->modify($object);
```

Infoblox::Session->remove( )

Use this method to remove the object from the Infoblox appliance. See Infoblox::Session->remove() for parameters and return values.

To remove a specific object, first use `get()` or `search()` to retrieve the specific DHCP range object, and then submit this object for removal.

Example

```
#Get the objects
my @retrieved_objs = $session->get(
    object => 'Infoblox::Grid::DNS::Nsgroup::ForwardingMember',
    name   => 'fwdgroup',
);

#Find the desired object from the retrieved list
my $desired_object = $retrieved_objs[0];

#Submit for removal
my $response = $session->remove($desired_object);
```

Infoblox::Session->search( )

Use this method to search for objects in the Infoblox appliance. See Infoblox::Session->search() for parameters and return values.

Key References

```
Apply the following attributes to search for the forwarding member group object:

 comment               - Optional. The forwarding member group comment in a string format (regexp).
 name                  - Optional. The forwarding member group name in a string format (regexp).
 extattrs              - Optional. A hash reference containing extensible attributes.
 extensible_attributes - Optional. A hash reference containing extensible attributes.
```

For more information about searching extensible attributes, see Infoblox::Grid::ExtensibleAttributeDef/Searching Extensible Attributes.

Examples

```
# search for objects
my @retrieved_objs = $session->search(
    object  => 'Infoblox::Grid::DNS::Nsgroup::ForwardingMember',
```

```
    name    => '^fwd.*',
    comment => '.*comment',
);

# search for objects defining 'Santa Clara' for 'Site' extensible attribute
my @retrieved_objs = $session->search(
    object               => 'Infoblox::Grid::DNS::Nsgroup::ForwardingMember',
    extensible_attributes => {'Site' => 'Santa Clara'},
);
```

METHODS

This section describes all the methods that you can use to configure and retrieve the attribute values of a forwarding member group.

comment( )

Use this method to set or retrieve the forwarding member group comment.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

The valid value is a desired comment in a string format.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get comment value
my $comment = $object->comment();

#Modify comment value
$object->comment('fwd comment');
```

extattrs( )

Use this method to set or retrieve the extensible attributes associated with a forwarding member group object.

Parameter

Valid value is a hash reference that contains the names of extensible attributes and their associated values (Infoblox::Grid::Extattr objects).

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get extattrs value
my $extattrs = $object->extattrs();

#Modify extattrs value
$object->extattrs({'Site' => $extattr1, 'Administrator' => $extattr2});
```

extensible_attributes( )

Use this method to set or retrieve the extensible attributes associated with a forwarding member group object.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

For valid values for extensible attributes, see Infoblox::Grid::ExtensibleAttributeDef/Extensible Attribute Values.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get extensible attributes value
my $extensible_attributes = $object->extensible_attributes();
#Modify extensible attributes
$object->extensible_attributes({'Site' => 'Santa Clara', 'Administrator' => ['Peter', 'Tom']});
```

forwarding_servers( )

Use this method to set or retrieve the list of forwarding servers.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

The valid value is an array of Infoblox::DNS::Member objects.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get forwarding_servers value
my $forwarding_servers = $object->forwarding_servers();

#Modify forwarding_servers value

#Create nameserver object
my $nameserver = Infoblox::DNS::Nameserver->new(
    name     => 'ns1.domain1.extra',
    ipv4addr => '5.5.5.5',
);

#Create dns member object
my $dnsmember = Infoblox::DNS::Member->new(
    ipv4addr   => '1.1.1.1',
    forward_to => [$nameserver],
);

#Set forwarding_servers value
$object->forwarding_servers([$dnsmember]);
```

name( )

Use this method to set or retrieve the forwarding member group name.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

The valid value is a desired name in a string format.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get name value
my $name = $object->name();

#Modify name value
$object->name('fwdg1');
```

SAMPLE CODE

The following sample code demonstrates the different functions that can be applied to an object, such as modify and remove. This sample also includes error handling for the operations.

#Preparation prior to an Forwarding Member Nsgroup object insertion

```
#PROGRAM STARTS: Include all the modules that will be used
use strict;
```

```
use Infoblox;

my ($session, $result);

#Create a session to the Infoblox device
$session = Infoblox::Session->new(
    master   => "192.168.1.2",
    username => "admin",
    password => "infoblox"
);
unless ($session) {
   die("Construct session failed: ",
       Infoblox::status_code() . ":" . Infoblox::status_detail());
}
print "Session created successfully\n";
```

#Create an Forwarding Member Nsgroup object

```
#Creating an external nameserver
my $nameserver = Infoblox::DNS::Nameserver->new(
    name     => "ns1.domain0.external",
    ipv4addr => "5.5.5.5",
);

#Creating a forwarding member object.
my $fwd_member = Infoblox::DNS::Member->new(
    name       => "infoblox.localdomain",
    ipv4addr   => "192.168.1.2",
    forward_to =>  [$nameserver],
);

#Creating a forwarding nsgroup object.
my $fwd_nsg1 = Infoblox::Grid::DNS::Nsgroup::ForwardingMember->new(
    name               => "fwd_group_1",
    forwarding_servers => [$fwd_member],
);

unless ($fwd_nsg1) {
   die("Construct Forwarding Member Nsgroup failed: ",
       Infoblox::status_code() . ":" . Infoblox::status_detail());
}

print "Forwarding Member Nsgroup object constructed successfully\n";
```

#Add the Nsgroup object

```
$result = $session->add($fwd_nsg1);

unless ($result) {
    die("Add Forwarding Member Nsgroup to session failed: ",
        $session->status_code() . ":" . $session->status_detail());
}

print "Forwarding Member Nsgroup object created successfully\n";
```

#Search and Modify the Nsgroup object

```
my @result_array = $session->search(
    object => "Infoblox::Grid::DNS::Nsgroup::ForwardingMember",
    name   => "fwd_.*",
);

my $object = $result_array[0];

unless ($object) {
   die("Search for Forwarding Member Nsgroup failed: ",
       $session->status_code() . ":" . $session->status_detail());
}

$result = $object->comment("forwarding member");

unless ($result) {
    die("Modify Forwarding Member Nsgroup failed: ",
        Infoblox::status_code() . ":" . Infoblox::status_detail());
}

$result = $session->modify($object);
```

```
unless ($result) {
    die("Submit modification to the session failed: ",
        $session->status_code() . ":" . $session->status_detail());
}

print "Forwarding Member Nsgroup object modified successfully\n";
```

#Remove a Nsgroup object

```
my @result_array = $session->get(
    object => "Infoblox::Grid::DNS::Nsgroup::ForwardingMember",
    name   => "fwd_group_1",
);

my $object = $result_array[0];

unless ($object) {
   die("Get Forwarding Member Nsgroup failed: ",
       $session->status_code() . ":" . $session->status_detail());
}

$result = $session->remove($object);

unless ($result) {
    die("Remove Nsgroup failed: ",
        $session->status_code() . ":" . $session->status_detail());
}

print "Forwarding Member Nsgroup object removed successfully\n";

####PROGRAM ENDS####
```

AUTHOR

Infoblox Inc. http://www.infoblox.com/

SEE ALSO

Infoblox::Session, Infoblox::Session->add(), Infoblox::Session->get(), Infoblox::Session->modify(), Infoblox::Session->remove(), Infoblox::Session->search(), Infoblox::DNS::Member, Infoblox::Grid::ExtensibleAttributeDef/Extensible Attribute Values, Infoblox::Grid::Extattr

---

COPYRIGHT

Copyright (c) 2017 Infoblox Inc.

Infoblox::Grid::DNS::Nsgroup::ForwardStubServer - A forward/stub external nameserver group.

- NAME
- DESCRIPTION
- CONSTRUCTOR
- SESSION METHODS
  - Infoblox::Session->add( )
  - Infoblox::Session->get( )
  - Infoblox::Session->modify( )
  - Infoblox::Session->remove( )
  - Infoblox::Session->search( )
- METHODS
  - comment( )
  - extattrs( )
  - extensible_attributes( )
  - external_servers( )
  - name( )
- SAMPLE CODE
- AUTHOR
- SEE ALSO
- COPYRIGHT

NAME

Infoblox::Grid::DNS::Nsgroup::ForwardStubServer - A forward/stub external nameserver group.

DESCRIPTION

The forward/stub external nameserver group provides forward/stub external servers configuration for forward zones.

CONSTRUCTOR

```
 my $object = Infoblox::Grid::DNS::Nsgroup::ForwardStubServer->new(
     name                  => $string,                                                  #
Required
     external_servers      => [$ns1, $ns2, ...],                                        #
Required
     comment               => $string,                                                  #
Optional / Default is undefined
     extattrs              => { $string => $extattr, ... },                             #
Optional / Default is undefined
     extensible_attributes => {$string => $string | $num, $string => [ $string | $num, ... ], ... } #
Optional / Default is undefined
 );
```

You cannot set both extattrs and extensible_attributes attributes at the same time.

SESSION METHODS

This section describes all the methods in an Infoblox::Session module that you can apply to a forward/stub external nameservers group object.

Infoblox::Session->add( )

Use this method to add the object to the Infoblox appliance. See Infoblox::Session->add() for parameters and return values.

Example

```
#Construct an object
my $object = Infoblox::Grid::DNS::Nsgroup::ForwardStubServer->new(
    name             => 'extsrvgroup',
    external_servers => [$ns1, $ns2],
);

#Submit for addition
my $response = $session->add($object);
```

Infoblox::Session->get( )

Use this method to retrieve all the matching objects from the Infoblox appliance. See Infoblox::Session->get() for parameters and return

values.

Key Reference

```
 Apply the following attributes to get a specific forward/stub external nameserver group object:

  comment               - Optional. The forward/stub external nameserver group comment in a string
format.
  name                  - Optional. The forward/stub external nameserver group name in a string
format.
  extattrs              - Optional. A hash reference containing extensible attributes.
  extensible_attributes - Optional. A hash reference containing extensible attributes.
```

For more information about searching extensible attributes, see Infoblox::Grid::ExtensibleAttributeDef/Searching Extensible Attributes.

Example

```
my @retrieved_objs = $session->get(
    object => 'Infoblox::Grid::DNS::Nsgroup::ForwardStubServer',
    name   => 'extsrvgroup',
);

my @retrieved_objs = $session->get(
    object               => 'Infoblox::Grid::DNS::Nsgroup::ForwardStubServer',
    extensible_attributes => {'Site' => 'Santa Clara'},
);
```

Infoblox::Session->modify( )

Use this method to modify the object in the Infoblox appliance. See Infoblox::Session->modify() for parameters and return values.

Example

```
#Modify comment value
$object->comment('this is a modified comment');

#Submit modification
my $response = $session->modify($object);
```

Infoblox::Session->remove( )

Use this method to remove the object from the Infoblox appliance. See Infoblox::Session->remove() for parameters and return values.

To remove a specific object, first use `get()` or `search()` to retrieve the specific DHCP range object, and then submit this object for removal.

Example

```
#Get the objects
my @retrieved_objs = $session->get(
    object => 'Infoblox::Grid::DNS::Nsgroup::ForwardStubServer',
    name   => 'extsrvgroup',
);

#Find the desired object from the retrieved list
my $desired_object = $retrieved_objs[0];

#Submit for removal
my $response = $session->remove($desired_object);
```

Infoblox::Session->search( )

Use this method to search for objects in the Infoblox appliance. See Infoblox::Session->search() for parameters and return values.

Key Reference

```
 Apply the following attributes to search for the forward/stub external nameserver group object:

  comment               - Optional. The forward/stub external nameserver group comment in a string
format (regexp).
  name                  - Optional. The forward/stub external nameserver group name in a string
format (regexp).
  extattrs              - Optional. A hash reference containing extensible attributes.
  extensible_attributes - Optional. A hash reference containing extensible attributes.
```

For more information about searching extensible attributes, see Infoblox::Grid::ExtensibleAttributeDef/Searching Extensible Attributes.

Example

```
# search for objects
my @retrieved_objs = $session->search(
    object  => 'Infoblox::Grid::DNS::Nsgroup::ForwardStubServer',
    name    => '^ext.*',
    comment => '.*comment',
);

# search for objects defining 'Santa Clara' for 'Site' extensible attribute
my @retrieved_objs = $session->search(
    object               => 'Infoblox::Grid::DNS::Nsgroup::ForwardStubServer',
    extensible_attributes => {'Site' => 'Santa Clara'},
);
```

METHODS

This section describes all the methods that you can use to configure and retrieve the attribute values of a forward/stub external nameserver group.

comment( )

Use this method to set or retrieve the forward/stub external nameserver group comment.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

The valid value is a desired comment in a string format.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get comment value
my $comment = $object->comment();

#Modify comment value
$object->comment('ext comment');
```

extattrs( )

Use this method to set or retrieve the extensible attributes associated with a forward/stub external nameserver group object.

Parameter

Valid value is a hash reference that contains the names of extensible attributes and their associated values (Infoblox::Grid::Extattr objects).

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get extattrs value
my $extattrs = $object->extattrs();

#Modify extattrs value
$object->extattrs({'Site' => $extattr1, 'Administrator' => $extattr2});
```

extensible_attributes( )

Use this method to set or retrieve the extensible attributes associated with a forward/stub external nameserver group object.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

For valid values for extensible attributes, see Infoblox::Grid::ExtensibleAttributeDef/Extensible Attribute Values.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get extensible attributes value
my $extensible_attributes = $object->extensible_attributes();
#Modify extensible attributes
$object->extensible_attributes({'Site' => 'Santa Clara', 'Administrator' => ['Peter', 'Tom']});
```

external_servers( )

Use this method to set or retrieve the list of forward/stub external nameservers.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

The valid value is an array of Infoblox::DNS::Nameserver objects.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get external_servers value
my $external_servers = $object->external_servers();

#Modify external_servers value

#Create nameserver object
my $nameserver = Infoblox::DNS::Nameserver->new(
    name      => 'ns1.domain1.extra',
    ipv4addr => '5.5.5.5',
);

#Set external_servers value
$object->external_servers([$nameserver]);
```

name( )

Use this method to set or retrieve the forward/stub external nameserver group name.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

The valid value is a desired name in a string format.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get name value
my $name = $object->name();

#Modify name value
$object->name('extsrvgroup');
```

SAMPLE CODE

The following sample code demonstrates the different functions that can be applied to an object, such as modify and remove. This sample also includes error handling for the operations.

#Preparation prior to an Forward Stub Server Nsgroup object insertion

```
#PROGRAM STARTS: Include all the modules that will be used
use strict;
use Infoblox;
```

```
my ($session, $result);

#Create a session to the Infoblox device
$session = Infoblox::Session->new(
    master   => "192.168.1.2",
    username => "admin",
    password => "infoblox"
);
unless ($session) {
   die("Construct session failed: ",
       Infoblox::status_code() . ":" . Infoblox::status_detail());
}
print "Session created successfully\n";
```

#Create an Forward Stub Server Nsgroup object

```
#Creating an external nameserver
my $nameserver = Infoblox::DNS::Nameserver->new(
    name     => "ns1.domain0.external",
    ipv4addr => "5.5.5.5",
);

#Creating a forwarding nsgroup object.
my $fwd_stub_nsg1 = Infoblox::Grid::DNS::Nsgroup::ForwardStubServer->new(
    name             => "fwd_stub_group_1",
    external_servers => [$nameserver],
);

unless ($fwd_stub_nsg1) {
   die("Construct Forward Stub Server Nsgroup failed: ",
       Infoblox::status_code() . ":" . Infoblox::status_detail());
}

print "Forward Stub Server Nsgroup object constructed successfully\n";
```

#Add the Nsgroup object

```
$result = $session->add($fwd_stub_nsg1);

unless ($result) {
    die("Add Forward Stub Server Nsgroup to session failed: ",
        $session->status_code() . ":" . $session->status_detail());
}

print "Forward Stub Server Nsgroup object created successfully\n";
```

#Search and Modify the Nsgroup object

```
my @result_array = $session->search(
    object => "Infoblox::Grid::DNS::Nsgroup::ForwardStubServer",
    name   => "fwd_stub_.*",
);

my $object = $result_array[0];

unless ($object) {
   die("Search for Forward Stub Server Nsgroup failed: ",
       $session->status_code() . ":" . $session->status_detail());
}

$result = $object->comment("forwarding member");

unless ($result) {
    die("Modify Forward Stub Server Nsgroup failed: ",
        Infoblox::status_code() . ":" . Infoblox::status_detail());
}

$result = $session->modify($object);

unless ($result) {
    die("Submit modification to the session failed: ",
        $session->status_code() . ":" . $session->status_detail());
}

print "Forward Stub Server Nsgroup object modified successfully\n";
```

#Remove a Nsgroup object

```
my @result_array = $session->get(
    object => "Infoblox::Grid::DNS::Nsgroup::ForwardStubServer",
    name   => "fwd_stub_group_1",
);

my $object = $result_array[0];

unless ($object) {
   die("Get Forward Stub Server Nsgroup failed: ",
       $session->status_code() . ":" . $session->status_detail());
}

$result = $session->remove($object);

unless ($result) {
    die("Remove Nsgroup failed: ",
        $session->status_code() . ":" . $session->status_detail());
}

print "Forward Stub Server Nsgroup object removed successfully\n";

####PROGRAM ENDS####
```

AUTHOR

Infoblox Inc. http://www.infoblox.com/

SEE ALSO

Infoblox::Session, Infoblox::Session->add(), Infoblox::Session->get(), Infoblox::Session->modify(), Infoblox::Session->remove(), Infoblox::Session->search(), Infoblox::DNS::Nameserver, Infoblox::Grid::ExtensibleAttributeDef/Extensible Attribute Values, Infoblox::Grid::Extattr

---

COPYRIGHT

Copyright (c) 2017 Infoblox Inc.

Infoblox::Grid::DNS::Nsgroup::StubMember - A stub member group.

- NAME
- DESCRIPTION
- CONSTRUCTOR
- SESSION METHODS
  - Infoblox::Session->add( )
  - Infoblox::Session->get( )
  - Infoblox::Session->modify( )
  - Infoblox::Session->remove( )
  - Infoblox::Session->search( )
- METHODS
  - comment( )
  - extattrs( )
  - extensible_attributes( )
  - stub_members( )
  - name( )
- SAMPLE CODE
- AUTHOR
- SEE ALSO
- COPYRIGHT

NAME

Infoblox::Grid::DNS::Nsgroup::StubMember - A stub member group.

DESCRIPTION

The stub member group provides stub servers configuration for stub zones.

CONSTRUCTOR

```
 my $object = Infoblox::Grid::DNS::Nsgroup::StubMember->new(
     name                  => $string,
#Required
     stub_members          => [$grid_member1, $grid_member2, ...],
#Required
     comment               => $string,
#Optional / Default is undefined
     extattrs              => { $string => $extattr, ... },
#Optional / Default is undefined
     extensible_attributes => {$string => $string | $num, $string => [ $string | $num, ... ], ... }
#Optional / Default is undefined
 );
```

You cannot set both extattrs and extensible_attributes attributes at the same time.

SESSION METHODS

This section describes all the methods in an Infoblox::Session module that you can apply to a stub members group object.

Infoblox::Session->add( )

Use this method to add the object to the Infoblox appliance. See Infoblox::Session->add() for parameters and return values.

Example

```
#Construct an object
my $object = Infoblox::Grid::DNS::Nsgroup::StubMember->new(
    name         => 'stubgroup',
    stub_members => ['member1.com', 'member2.com'],
);

#Submit for addition
my $response = $session->add($object);
```

Infoblox::Session->get( )

Use this method to retrieve all the matching objects from the Infoblox appliance. See Infoblox::Session->get() for parameters and return values.

Key Reference

```
Apply the following attributes to get a specific stub member group object:

 comment               - Optional. The stub member group comment in a string format.
 name                  - Optional. The stub member group name in a string format.
 extattrs              - Optional. A hash reference containing extensible attributes.
 extensible_attributes - Optional. A hash reference containing extensible attributes.
```

For more information about searching extensible attributes, see Infoblox::Grid::ExtensibleAttributeDef/Searching Extensible Attributes.

Example

```
my @retrieved_objs = $session->get(
    object => 'Infoblox::Grid::DNS::Nsgroup::StubMember',
    name   => 'stubgroup',
);

my @retrieved_objs = $session->get(
    object               => 'Infoblox::Grid::DNS::Nsgroup::StubMember',
    extensible_attributes => {'Site' => 'Santa Clara'},
);
```

Infoblox::Session->modify( )

Use this method to modify the object in the Infoblox appliance. See Infoblox::Session->modify() for parameters and return values.

Example

```
#Modify comment value
$object->comment('this is a modified comment');

#Submit modification
my $response = $session->modify($object);
```

Infoblox::Session->remove( )

Use this method to remove the object from the Infoblox appliance. See Infoblox::Session->remove() for parameters and return values.

To remove a specific object, first use `get()` or `search()` to retrieve the specific DHCP range object, and then submit this object for removal.

Example

```
#Get the objects
my @retrieved_objs = $session->get(
    object => 'Infoblox::Grid::DNS::Nsgroup::StubMember',
    name   => 'stubgroup',
);

#Find the desired object from the retrieved list
my $desired_object = $retrieved_objs[0];

#Submit for removal
my $response = $session->remove($desired_object);
```

Infoblox::Session->search( )

Use this method to search for objects in the Infoblox appliance. See Infoblox::Session->search() for parameters and return values.

Key Reference

```
Apply the following attributes to search for the stub member group object:

 comment               - Optional. The stub member group comment in a string format (regexp).
 name                  - Optional. The stub member group name in a string format (regexp).
 extattrs              - Optional. A hash reference containing extensible attributes.
 extensible_attributes - Optional. A hash reference containing extensible attributes.
```

For more information about searching extensible attributes, see Infoblox::Grid::ExtensibleAttributeDef/Searching Extensible Attributes.

Example

```
# search for objects
my @retrieved_objs = $session->search(
    object  => 'Infoblox::Grid::DNS::Nsgroup::StubMember',
    name    => '^stub.*',
```

```
    comment => '.*comment',
);

# search for objects defining 'Santa Clara' for 'Site' extensible attribute
my @retrieved_objs = $session->search(
    object               => 'Infoblox::Grid::DNS::Nsgroup::StubMember',
    extensible_attributes => {'Site' => 'Santa Clara'},
);
```

METHODS

This section describes all the methods that you can use to configure and retrieve the attribute values of a stub member group.

comment( )

Use this method to set or retrieve the stub member group comment.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

The valid value is a desired comment in a string format.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get comment value
my $comment = $object->comment();

#Modify comment value
$object->comment('stub comment');
```

extattrs( )

Use this method to set or retrieve the extensible attributes associated with a stub member group object.

Parameter

Valid value is a hash reference that contains the names of extensible attributes and their associated values (Infoblox::Grid::Extattr objects).

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get extattrs value
my $extattrs = $object->extattrs();

#Modify extattrs value
$object->extattrs({'Site' => $extattr1, 'Administrator' => $extattr2});
```

extensible_attributes( )

Use this method to set or retrieve the extensible attributes associated with a stub member group object.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

For valid values for extensible attributes, see Infoblox::Grid::ExtensibleAttributeDef/Extensible Attribute Values.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get extensible attributes value
my $extensible_attributes = $object->extensible_attributes();
#Modify extensible attributes
$object->extensible_attributes({'Site' => 'Santa Clara', 'Administrator' => ['Peter', 'Tom']});
```

stub_members( )

Use this method to set or retrieve the list of stub members.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

The valid value is an array of Grid Member host names in a string format.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get stub_members value
my $stub_members = $object->stub_members();

#Modify stub_members value

#Set stub_members value
$object->stub_members(['member1.com', 'member2.com']);
```

name( )

Use this method to set or retrieve the stub member group name.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

The valid value is a desired name in a string format.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get name value
my $name = $object->name();

#Modify name value
$object->name('stubg1');
```

SAMPLE CODE

The following sample code demonstrates the different functions that can be applied to an object, such as modify and remove. This sample also includes error handling for the operations.

#Preparation prior to an Stub Member Nsgroup object insertion

```
#PROGRAM STARTS: Include all the modules that will be used
use strict;
use Infoblox;

my ($session, $result);

#Create a session to the Infoblox device
$session = Infoblox::Session->new(
    master   => "192.168.1.2",
    username => "admin",
    password => "infoblox"
);
unless ($session) {
   die("Construct session failed: ",
       Infoblox::status_code() . ":" . Infoblox::status_detail());
}
```

```
print "Session created successfully\n";
```

#Create an Stub Member Nsgroup object

```
#Creating a forwarding nsgroup object.
my $stub_nsg1 = Infoblox::Grid::DNS::Nsgroup::StubMember->new(
    name         => "stub_group_1",
    stub_members => ["infoblox.localdomain"],
);

unless ($stub_nsg1) {
   die("Construct Stub Member Nsgroup failed: ",
       Infoblox::status_code() . ":" . Infoblox::status_detail());
}

print "Stub Member Nsgroup object constructed successfully\n";
```

#Add the Nsgroup object

```
$result = $session->add($stub_nsg1);

unless ($result) {
    die("Add Stub Member Nsgroup to session failed: ",
        $session->status_code() . ":" . $session->status_detail());
}

print "Stub Member Nsgroup object created successfully\n";
```

#Search and Modify the Nsgroup object

```
my @result_array = $session->search(
    object => "Infoblox::Grid::DNS::Nsgroup::StubMember",
    name   => "stub_.*",
);

my $object = $result_array[0];

unless ($object) {
   die("Search for Stub Member Nsgroup failed: ",
       $session->status_code() . ":" . $session->status_detail());
}

$result = $object->comment("stub member");

unless ($result) {
    die("Modify Stub Member Nsgroup failed: ",
        Infoblox::status_code() . ":" . Infoblox::status_detail());
}

$result = $session->modify($object);

unless ($result) {
    die("Submit modification to the session failed: ",
        $session->status_code() . ":" . $session->status_detail());
}

print "Stub Member Nsgroup object modified successfully\n";
```

#Remove a Nsgroup object

```
my @result_array = $session->get(
    object => "Infoblox::Grid::DNS::Nsgroup::StubMember",
    name   => "stub_group_1",
);

my $object = $result_array[0];

unless ($object) {
   die("Get Stub Member Nsgroup failed: ",
       $session->status_code() . ":" . $session->status_detail());
}

$result = $session->remove($object);

unless ($result) {
    die("Remove Nsgroup failed: ",
        $session->status_code() . ":" . $session->status_detail());
```

```
}

print "Stub Member Nsgroup object removed successfully\n";

####PROGRAM ENDS####
```

AUTHOR

Infoblox Inc. http://www.infoblox.com/

SEE ALSO

Infoblox::Session, Infoblox::Session->add(), Infoblox::Session->get(), Infoblox::Session->modify(), Infoblox::Session->remove(), Infoblox::Session->search(), Infoblox::Grid::ExtensibleAttributeDef/Extensible Attribute Values, Infoblox::Grid::Extattr

COPYRIGHT

Copyright (c) 2017 Infoblox Inc.

Infoblox::Grid::DNS::ResponseRateLimiting - a DNS Response Rate Limiting object.

- NAME
- DESCRIPTION
- CONSTRUCTOR
- SESSION METHODS
- MODULE METHODS
  - Infoblox::Grid::DNS->response_rate_limiting( )
  - Infoblox::Grid::Member::DNS->response_rate_limiting( )
  - Infoblox::DNS::View->response_rate_limiting( )
- METHODS
- enable_rrl( )
- log_only( )
- responses_per_second( )
- slip( )
- window( )
- AUTHOR
- SEE ALSO
- COPYRIGHT

NAME

Infoblox::Grid::DNS::ResponseRateLimiting - a DNS Response Rate Limiting object.

DESCRIPTION

The DNS Response Rate Limiting object provides information about DNS response rate limiting configuration.

CONSTRUCTOR

```
my $response_rate_limiting = Infoblox::Grid::DNS::ResponseRateLimiting->new(
        enable_rrl           => 'true' | 'false', # Optional
        log_only             => 'true' | 'false', # Optional
        responses_per_second => $uint,            # Optional
        window               => $uint,            # Optional
        slip                 => $uint,            # Optional
);
```

SESSION METHODS

The object does not support any session methods.

MODULE METHODS

The following functions can be applied to a DNS Response Rate Limiting object.

Infoblox::Grid::DNS->response_rate_limiting( )

+Use this method to set or retrieve DNS response rate limiting settings. See Infoblox::Grid::DNS->response_rate_limiting() for parameters and return values.

Example

```
#Get response_rate_limiting value
my $response_rate_limiting = $grid_dns->response_rate_limiting();
#Modify bind_check_response_rate_limiting
$grid_dns->response_rate_limiting($response_rate_limiting);
```

Infoblox::Grid::Member::DNS->response_rate_limiting( )

Use this method to set or retrieve DNS response rate limiting settings on the member. See Infoblox::Grid::Member::DNS->response_rate_limiting() for parameters and return values.

Example

```
#Get response_rate_limiting value
my $response_rate_limiting = $member_dns->response_rate_limiting();
#Modify response_rate_limiting
$member_dns->response_rate_limiting($response_rate_limiting);
```

Infoblox::DNS::View->response_rate_limiting( )

Use this method to set or retrieve DNS response rate limiting settings for the DNS view. See Infoblox::DNS::View->response_rate_limiting() for parameters and return values.

Example

```
#Get response_rate_limiting value
my $response_rate_limiting = $dns_view->response_rate_limiting();
#Modify response_rate_limiting
$dns_view->response_rate_limiting($response_rate_limiting);
```

METHODS

---

enable_rrl( )

Use this method to enabel or disable response rate limiting.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

Specify 'true' to enable response rate limiting or 'false' to disable it.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get enable_rrl value
my $enable_rrl = $response_rate_limiting->enable_rrl();
#Modify enable_rrl
$response_rate_limiting->enable_rrl('true');
```

---

log_only( )

Use this method to enable or disable logging for response rate limiting without dropping any requests.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

Specify 'true' to enable logging for response rate limiting without dropping any requests or 'false' to disable it.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get log_only value
my $log_only = $response_rate_limiting->log_only();
#Modify log_only
$response_rate_limiting->log_only('true');
```

---

responses_per_second( )

Use this method to set or retrieve the number of responses per client per second.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

The valid value is an unsigned integer.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get responses_per_second value
my $responses_per_second = $response_rate_limiting->responses_per_second();
#Modify responses_per_second
$response_rate_limiting->responses_per_second(50);
```

---

slip( )

Use this method to set or retrieve the response rate limiting slip.

Note that if a slip does not equal 0, instead of dropping the request, a truncated response is sent to each n-th rate-limited UDP request.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

The valid value is an unsigned integer.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get slip value
my $slip = $response_rate_limiting->slip();
#Modify slip
$response_rate_limiting->slip(50);
```

---

window( )

Use this method to set or retrieve the time interval in seconds over which responses are tracked.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

The valid value is an unsigned integer.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get window value
my $window = $response_rate_limiting->window();
#Modify window
$response_rate_limiting->window(10);
```

AUTHOR

Infoblox Inc. http://www.infoblox.com/

SEE ALSO

Infoblox::Grid::DNS, Infoblox::Grid::DNS->response_rate_limiting(), Infoblox::Grid::Member::DNS, Infoblox::Grid::Member::DNS->response_rate_limiting(), Infoblox::DNS::View->response_rate_limiting()

COPYRIGHT

Copyright (c) 2017 Infoblox Inc.

Infoblox::Grid::DNS::ScavengingSetting - A DNS scavenging settings object.

- NAME
- DESCRIPTION
- CONSTRUCTOR
- SESSION METHODS
- MODULE METHODS
  - Infoblox::Grid::DNS->scavenging_settings( )
  - Infoblox::DNS::View->scavenging_settings( )
  - Infoblox::DNS::Zone->scavenging_settings( )
- METHODS
  - ea_expression_list( )
  - enable_scavenging( )
  - enable_recurrent_scavenging( )
  - enable_rr_last_queried( )
  - enable_zone_last_queried( )
  - enable_auto_reclamation( )
  - reclaim_associated_records( )
  - expression_list( )
  - scavenging_schedule( )
- AUTHOR
- SEE ALSO
- COPYRIGHT

NAME

Infoblox::Grid::DNS::ScavengingSetting - A DNS scavenging settings object.

DESCRIPTION

The DNS scavenging settings object provides information about DNS records scavenging configuration such as the conditions under which records can be reclaimed and periodicity of the scavening operations.

CONSTRUCTOR

```
my $reclamation_setting = Infoblox::Grid::DNS::ScavengingSetting->new(
    ea_expression_list          => [$ea_expr1, $ea_expr2, ...], # Optional / Default is undefined
    enable_auto_reclamation     => 'true' | 'false',            # Optional / Default is 'false'
    enable_recurrent_scavenging => 'true' | 'false',            # Optional / Default is 'false'
    enable_rr_last_queried      => 'true' | 'false',            # Optional / Default is 'false'
    enable_scavenging           => 'true' | 'false',            # Optional / Default is 'false'
    enable_zone_last_queried    => 'true' | 'false',            # Optional / Default is 'false'
    expression_list             => [$expr1, $expr2, ...],       # Optional / Default is undefined
    reclaim_associated_records  => 'true' | 'false',            # Optional / Default is 'false'
    scavenging_schedule         => $sched_setting,              # Optional / Default is undefined
);
```

SESSION METHODS

The object does not support any session methods.

MODULE METHODS

Infoblox::Grid::DNS->scavenging_settings( )

Use this method to set or retrieve the Grid-level scavenging settings. See Infoblox::Grid::DNS->scavenging_settings() for parameters and return values.

Example

```
#Get scavenging_settings value
my $scavenging_settings = $object->scavenging_settings();

#Modify scavenging_settings value
$object->scavenging_settings($scavenging_settings);
```

Infoblox::DNS::View->scavenging_settings( )

Use this method to set or retrieve the DNS view-level scavenging settings. See Infoblox::DNS::View->scavenging_settings() for parameters and return values.

Example

```
#Get scavenging_settings value
my $scavenging_settings = $object->scavenging_settings();

#Modify scavenging_settings value
$object->scavenging_settings($scavenging_settings);
```

Infoblox::DNS::Zone->scavenging_settings( )

Use this method to set or retrieve the DNS zone-level scavenging settings. See Infoblox::DNS::Zone->scavenging_settings() for parameters and return values.

Example

```
#Get scavenging_settings value
my $scavenging_settings = $object->scavenging_settings();

#Modify scavenging_settings value
$object->scavenging_settings($scavenging_settings);
```

METHODS

This section describes all the methods that you can use to configure and retrieve the attribute values of a DNS scavenging settings object.

ea_expression_list( )

Use this method to set or retrieve the extensible attributes expression list.

The particular record is treated as reclaimable if the extensible attribute expression condition evaluates to 'true' and scavenging is not manually disabled on the given record. Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

The valid value is an array of Infoblox::Grid::EAExpressionOp objects.

The following is a table of allowed expression operand parameters:

op1_type	op1	op2_type	op2	op
FIELD	Infoblox::Grid::ExtensibleAttributeDef	STRING	$value	EQ, NOT_EQ
	Infoblox::Grid::ExtensibleAttributeDef	undef	undef	EXISTS, NOT_EXISTS
LIST	undef	undef	undef	AND, OR, ENDLIST

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get ea_expression_list value
my $ea_expression_list = $object->ea_expression_list();

#Construct expression (Site == LA OR Site == CA)

my $ead = Infoblox::Grid::ExtensibleAttributeDef->new(
    name => 'Site',
);

my $start_list = Infoblox::Grid::EAExpressionOp->new(
    op1_type => 'LIST',
    op       => 'OR',
);
```

```
my $ea_op1 = Infoblox::Grid::EAExpressionOp->new(
    op1_type => 'FIELD',
    op1      => $ead,
    op2_type => 'STRING',
    op2      => 'LA',
    op       => 'EQ',
);

my $ea_op2 = Infoblox::Grid::EAExpressionOp->new(
    op1_type => 'FIELD',
    op1      => $ead,
    op2_type => 'STRING',
    op2      => 'CA',
    op       => 'EQ',
);

my $end_list = Infoblox::Grid::EAExpressionOp->new(
    op1_type => 'LIST',
    op       => 'ENDLIST',
);

#Modify ea_expression_list value
$object->ea_expression_list([$start_list, $ea_op1, $ea_op2, $end_list]);
```

enable_scavenging( )

Use this method to set or retrieve the flag that indicates whether the resource record scavenging is enabled or not.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

Specify 'true' to enable the resource record reclamation or 'false' to disable it. The default value is 'false'.

Parameter

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get enable_scavenging value
my $enable_scavenging = $object->enable_scavenging();

#Modify enable_scavenging value
$object->enable_scavenging('true');
```

enable_recurrent_scavenging( )

Use this method to set or retrieve the flag that indicates whether the recurrent resource record scavenging is enabled or not.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

Specify 'true' to enable the recurrent resource record scavenging or 'false' to disable it. The default value is 'false'.

Parameter

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get enable_recurrent_scavenging value
my $enable_recurrent_scavenging = $object->enable_recurrent_scavenging();

#Modify enable_recurrent_scavenging value
$object->enable_recurrent_scavenging('true');
```

enable_rr_last_queried( )

Use this method to set or retrieve the flag that indicates whether the resource record last queried monitoring in affected zones is enabled or not.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

Specify 'true' to enable the resource record last queried monitoring or 'false' to disable it. The default value is 'false'.

Parameter

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get enable_rr_last_queried value
my $enable_rr_last_queried = $object->enable_rr_last_queried();

#Modify enable_rr_last_queried value
$object->enable_rr_last_queried('true');
```

enable_zone_last_queried( )

Use this method to set or retrieve the flag that indicates whether the last queried monitoring for affected zones is enabled or not.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

Specify 'true' to enable the last queried monitoring for affected zones or 'false' to disable it. The default value is 'false'.

Parameter

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get enable_zone_last_queried value
my $enable_zone_last_queried = $object->enable_zone_last_queried();

#Modify enable_zone_last_queried value
$object->enable_zone_last_queried('true');
```

enable_auto_reclamation( )

Use this method to set or retrieve the flag that indicates whether the automatic resource record reclamation is enabled or not.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

Specify 'true' to enable the automatic resource record reclamation or 'false' to disable it. The default value is 'false'.

Parameter

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get enable_auto_reclamation value
my $enable_auto_reclamation = $object->enable_auto_reclamation();

#Modify enable_auto_reclamation value
$object->enable_auto_reclamation('true');
```

reclaim_associated_records( )

Use this method to set or retrieve the flag that indicates whether the associated resource record reclamation is enabled or not.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

Specify 'true' to enable the associated resource record reclamation or 'false' to disable it. The default value is 'false'.

Parameter

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

G #Get reclaim_associated_records value my $reclaim_associated_records = $object->reclaim_associated_records();

```
#Modify reclaim_associated_records value
$object->reclaim_associated_records('true');
```

expression_list( )

Use this method to set or retrieve the expression list.

The particular record is treated as reclaimable if expression condition evaluates to 'true' for given record if reclamation hasn't been manually disabled on a given resource record.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

The valid value is an array of Infoblox::Grid::ExpressionOp objects.

The following is a table of allowed expression operand parameters:

```
  +----------+-------------------------------------------------------------+----------+-------------------
------------------------------+----------------------------------+
  | op1_type | op1                                                         | op2_type | op2
|  op                          |
  +----------+-------------------------------------------------------------+----------+-------------------
------------------------------+----------------------------------+
  |          | rtype    - the resource record type                         | STRING   | A, AAAA, CNAME,
DNAME, MX, NAPTR, PTR, SRV, TXT | EQ, NOT_EQ                     |
  |          | ctime    - the resource record creation time                | STRING   | $uint
| GT (in days)                 |
  |          | qtime    - the resource record last queried time            | STRING   | $uint
| GT (in days)                 |
  | FIELD    | stime    - the resource record last seen time               | STRING   | $uint
| GT (in days)                 |
  |          | rcreator - the resource record creator                      | STRING   | STATIC, DYNAMIC
| EQ                           |
  |          | assocr   - the resource record associated record            | undef    | undef
| EXISTS, NOT_EXISTS           |
  +----------+-------------------------------------------------------------+----------+-------------------
------------------------------+----------------------------------+
  | LIST     | undef                                                       | undef    | undef
| AND, OR, ENDLIST             |
  +----------+-------------------------------------------------------------+----------+-------------------
------------------------------+----------------------------------+
```

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get expression_list value
my $expression_list = $object->expression_list();

#Construct expression (rcreator == DYNAMIC AND ctime > 10)

my $start_list = Infoblox::Grid::ExpressionOp->new(
    op1_type => 'LIST',
    op       => 'AND',
);

my $op1 = Infoblox::Grid::ExpressionOp->new(
    op1_type => 'FIELD',
    op1      => 'rcreator',
    op2_type => 'STRING',
    op2      => 'DYNAMIC',
    op       => 'EQ',
);

my $op2 = Infoblox::Grid::ExpressionOp->new(
    op1_type => 'FIELD',
    op1      => 'ctime',
    op2_type => 'STRING',
    op2      => '10',
```

```
    op       => 'GT',
);

my $end_list = Infoblox::Grid::ExpressionOp->new(
    op1_type => 'LIST',
    op       => 'ENDLIST',
);

#Modify expression_list value
$object->expression_list([$start_list, $op1, $op2, $end_list]);
```

scavenging_schedule( )

Use this method to set or retrieve the scavenging schedule settings.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

The valid value is an Infoblox::Grid::ScheduleSetting object.

Parameter

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get scavenging_schedule value
my $scavenging_schedule = $object->scavenging_schedule();

#Modify scavenging_schedule value
$object->scavenging_schedule($schedule);
```

AUTHOR

Infoblox Inc. http://www.infoblox.com/

SEE ALSO

Infoblox::Grid::DNS, Infoblox::Grid::DNS->scavenging_settings(), Infoblox::DNS::View, Infoblox::DNS::View->scavenging_settings(), Infoblox::DNS::Zone, Infoblox::DNS::Zone->scavenging_settings(), Infoblox::Grid::ScheduleSetting, Infoblox::Grid::ExpressionOp, Infoblox::Grid::EAExpressionOp,

---

COPYRIGHT

Copyright (c) 2017 Infoblox Inc.

Infoblox::Grid::DNS::ScavengingTask - A DNS scavenging task object.

- NAME
- DESCRIPTION
- SESSION METHODS
  - Infoblox::Session->get( )
  - Infoblox::Session->search( )
- METHODS
  - action( )
  - associated_object( )
  - end_time( )
  - processed_records( )
  - reclaimable_records( )
  - reclaimed_records( )
  - start_time( )
  - status( )
- AUTHOR
- SEE ALSO
- COPYRIGHT

NAME

Infoblox::Grid::DNS::ScavengingTask - A DNS scavenging task object.

DESCRIPTION

The DNS scavenging task object provides information on scavenging process state. This is a read-only object.

The object is created as a result of Infoblox::Session->run_scavenging() function call and cannot be added or removed manually.

SESSION METHODS

This section describes all the methods in the Infoblox::Session module that you can apply to a DNS scavenging task object.

Infoblox::Session->get( )

Use this method to retrieve all the matching objects from the Infoblox appliance. See Infoblox::Session->get() for parameters and return values.

Key Reference

```
Apply the following attribute to get a specific object:
    associated_object - Optional. The DNS object associated with scavenging process.
    action            - Optional. The scavenging action.
    status            - Optional. The scavenging status.
```

Example

```
my @retrieved_objs = $session->get(
    object => 'Infoblox::Grid::DNS::ScavengingTask',
    action => 'ANALYZE',
);
```

Infoblox::Session->search( )

Use this method to search for objects in the Infoblox appliance. See Infoblox::Session->search() for parameters and return values.

Key Reference

```
Apply the following attribute to get a specific object:
    associated_object - Optional. The DNS object associated with scavenging process.
    action            - Optional. The scavenging action.
    status            - Optional. The scavenging status.
```

Example

```
my @retrieved_objs = $session->search(
    object => 'Infoblox::Grid::DNS::ScavengingTask',
    action => 'ANALYZE',
);
```

METHODS

This section describes all the methods that you can use to retrieve the attribute values of a DNS scavenging task object.

action( )

Use this method to retrieve the scavenging action. This is a read-only attribute.

Omit the parameter to retrieve the attribute value.

Parameter

None

Returns

The valid return values are 'ANALYZE', 'RECLAIM', 'ANALYZE_RECLAIM' and 'RESET'.

Example

```
#Get action value
my $action = $object->action();
```

associated_object( )

Use this method to retrieve the object associated with the scavenging task. This is a read-only attribute.

Omit the parameter to retrieve the attribute value.

Parameter

None

Returns

The valid return values are Infoblox::Grid::DNS, Infoblox::DNS::View and Infoblox::DNS::Zone.

Example

```
#Get associated_object value
my $associated_object = $object->associated_object();
```

end_time( )

Use this method to retrieve the end time for the scavenging task. This is a read-only attribute.

Omit the parameter to retrieve the attribute value.

Parameter

None

Returns

The method returns the attribute value, the number of seconds that have elapsed since January 1st, 1970 UTC.

Example

```
#Get end_time value
my $end_time = $object->end_time();
```

processed_records( )

Use this method to retrieve the number of resource records being processed during the scavenging process. This is a read-only attribute.

Omit the parameter to retrieve the attribute value.

Parameter

None

Returns

The method returns the attribute value.

Example

```
#Get processed_records value
```

```
my $processed_records = $object->processed_records();
```

reclaimable_records( )

Use this method to retrieve the number of scavenged resource records during the scavenging process. This is a read-only attribute.

Omit the parameter to retrieve the attribute value.

Parameter

None

Returns

The method returns the attribute value.

Example

```
#Get reclaimable_records value
my $reclaimable_records = $object->reclaimable_records();
```

reclaimed_records( )

Use this method to retrieve the number of reclaimed during the scavenging process resource records. This is a read-only attribute.

Omit the parameter to retrieve the attribute value.

Parameter

None

Returns

The method returns the attribute value.

Example

```
#Get reclaimed_records value
my $reclaimed_records = $object->reclaimed_records();
```

start_time( )

Use this method to retrieve the start time for the scavenging process. This is a read-only attribute.

Omit the parameter to retrieve the attribute value.

Parameter

None

Returns

The method returns the attribute value, the number of seconds that have elapsed since January 1st, 1970 UTC.

Example

```
#Get start_time value
my $start_time = $object->start_time();
```

status( )

Use this method to retrieve the scavenging process status. This is a read-only attribute.

Omit the parameter to retrieve the attribute value.

Parameter

None

Returns

The valid return values are 'CREATED', 'RUNNING'. 'COMPLETED' and 'ERROR'.

Example

```
#Get status value
my $status = $object->status();
```

AUTHOR

Infoblox Inc. http://www.infoblox.com/

SEE ALSO

Infoblox::Session, Infoblox::Session->get(), Infoblox::Session->search(), Infoblox::Session->run_scavenging(), Infoblox::Grid::DNS, Infoblox::DNS::View, Infoblox::DNS::Zone,

---

COPYRIGHT

Copyright (c) 2017 Infoblox Inc.

Infoblox::Grid::EAExpressionOp - A extensible attribute expression operand object.

- NAME
- DESCRIPTION
- CONSTRUCTOR
- SESSION METHODS
- MODULE METHODS
  - Infoblox::Grid::DNS::ReclamationSetting->ea_expression_list( )
- METHODS
  - op( )
  - op1( )
  - op1_type( )
  - op2( )
  - op2_type( )
- AUTHOR
- SEE ALSO
- COPYRIGHT

NAME

Infoblox::Grid::EAExpressionOp - A extensible attribute expression operand object.

DESCRIPTION

The extensible attribute expression operand object is used to build an extensible attribute expression lists. The allowed values for the expression operand object depend on the object they appear to be a part of.

CONSTRUCTOR

```
 my $expression = Infoblox::Grid::EAExpressionOp->new(
     op       => 'AND'| 'ENDLIST'| 'EQ'| 'EXISTS'| 'NOT_EQ'| 'NOT_EXISTS'| 'OR', # Required
     op1      => $ead,                                                          # Optional / Default is
undefined
     op2      => $string,                                                       # Optional / Default is
undefined
     op1_type => 'FIELD' | 'LIST' | 'STRING',                                   # Optional / Default is
undefined
     op2_type => 'FIELD' | 'LIST' | 'STRING',                                   # Optional / Default is
undefined
 );
```

SESSION METHODS

The object does not support any session methods.

MODULE METHODS

Infoblox::Grid::DNS::ReclamationSetting->ea_expression_list( )

Use this method to set or retrieve the ea_expression list. See Infoblox::Grid::DNS::ReclamationSetting->ea_expression_list() for parameters and return values.

Example

```
#Get ea_expression_list value
my $ea_expression_list = $object->ea_expression_list();

#Modify ea_expression_list value
$object->ea_expression_list([$start_list, $op1, $op2, $end_list]);
```

METHODS

op( )

Use this method to set or retrieve the operation name.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

The valid values are 'AND', 'ENDLIST', 'EQ', 'EXISTS', 'NOT_EQ', 'NOT_EXISTS' and 'OR'.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get op value
my $op = $object->op();

#Modify op value
$object->op('AND');
```

op1( )

Use this method to set or retrieve the first operand value.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

The valid value is an Infoblox::Grid::ExtensibleAttributeDef object.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get op1 value
my $op1 = $object->op1();

#Modify op1 value
$object->op1($ead);
```

op1_type( )

Use this method to set or retrieve the first operand type.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

The valid values are 'FIELD', 'LIST' and 'STRING'.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get op1_type value
my $op1_type = $object->op1_type();

#Modify op1_type value
$object->op1_type('LIST');
```

op2( )

Use this method to set or retrieve the second operand value.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

The valid value is a string that contains the second operand value.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get op2 value
my $op2 = $object->op2();

#Modify op2 value
$object->op2('OP2_VAL');
```

op2_type( )

Use this method to set or retrieve the second operand type.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

The valid values are 'FIELD', 'LIST' and 'STRING'.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get op2_type value
my $op2_type = $object->op2_type();

#Modify op2_type value
$object->op2_type('STRING');
```

AUTHOR

Infoblox Inc. http://www.infoblox.com/

SEE ALSO

Infoblox::Grid::DNS::ReclamationSetting, Infoblox::Grid::DNS::ReclamationSetting->ea_expression_list(), Infoblox::Grid::ExtensibleAttributeDef,

---

COPYRIGHT

Copyright (c) 2017 Infoblox Inc.

Infoblox::Grid::ExpressionOp - An expression operand object.

- NAME
- DESCRIPTION
- CONSTRUCTOR
- SESSION METHODS
- MODULE METHODS
  - Infoblox::Grid::DNS::ReclamationSetting->expression_list( )
- METHODS
  - op( )
  - op1( )
  - op1_type( )
  - op2( )
  - op2_type( )
- AUTHOR
- SEE ALSO
- COPYRIGHT

NAME

Infoblox::Grid::ExpressionOp - An expression operand object.

DESCRIPTION

The expression operand object is used to build expression lists. The allowed values for the expression operand object depend on the object they appear to be a part of.

CONSTRUCTOR

```
 my $expression = Infoblox::Grid::ExpressionOp->new(
     op       => 'AND'| 'OR'| 'GT'| 'GE'| 'LT'| 'LE'| 'EQ'| 'NOT_EQ'| 'ENDLIST'| 'EXISTS'| 'MATCH_IP'|
'MATCH_RANGE'| 'MATCH_CIDR', # Required
     op1      => $string,
# Optional / Default is undefined
     op2      => $string,
# Optional / Default is undefined
     op1_type => 'FIELD' | 'LIST' | 'STRING',
# Optional / Default is undefined
     op2_type => 'FIELD' | 'LIST' | 'STRING',
# Optional / Default is undefined
 );
```

SESSION METHODS

The object does not support any session methods.

MODULE METHODS

Infoblox::Grid::DNS::ReclamationSetting->expression_list( )

Use this method to set or retrieve the expression list. See Infoblox::Grid::DNS::ReclamationSetting->expression_list() for parameters and return values.

Example

```
#Get expression_list value
my $expression_list = $object->expression_list();

#Modify expression_list value
$object->expression_list([$start_list, $op1, $op2, $end_list]);
```

METHODS

op( )

Use this method to set or retrieve the operation name.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

The valid values are 'AND', 'ENDLIST', 'GT', 'LT', 'LE', 'GE', 'MATCH_IP', 'MATCH_RANGE', 'MATCH_CIDR', 'EQ', 'EXISTS', 'NOT_EQ', 'NOT_EXISTS' and 'OR'.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get op value
my $op = $object->op();

#Modify op value
$object->op('LE');
```

op1( )

Use this method to set or retrieve the first operand value.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

The valid value is a string that contains the first operand value.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get op1 value
my $op1 = $object->op1();

#Modify op1 value
$object->op1('OP1_VAL');
```

op1_type( )

Use this method to set or retrieve the first operand type.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

The valid values are 'FIELD', 'LIST' and 'STRING'.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get op1_type value
my $op1_type = $object->op1_type();

#Modify op1_type value
$object->op1_type('LIST');
```

op2( )

Use this method to set or retrieve the second operand value.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

The valid value is a string that contains the second operand value.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get op2 value
my $op2 = $object->op2();

#Modify op2 value
$object->op2('OP2_VAL');
```

op2_type( )

Use this method to set or retrieve the second operand type.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

The valid values are 'FIELD', 'LIST' and 'STRING'.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get op2_type value
my $op2_type = $object->op2_type();

#Modify op2_type value
$object->op2_type('STRING');
```

AUTHOR

Infoblox Inc. http://www.infoblox.com/

SEE ALSO

Infoblox::Grid::DNS::ReclamationSetting, Infoblox::Grid::DNS::ReclamationSetting->expression_list(),

---

COPYRIGHT

Copyright (c) 2017 Infoblox Inc.

Infoblox::Grid::Extattr - an extensible attribute representation for an attribute.

- NAME
- DESCRIPTION
- CONSTRUCTOR
- SESSION METHODS
- METHODS
  - descendants_action( )
  - inheritance_operation( )
  - inheritance_source( )
  - value( )
- AUTHOR
- SEE ALSO
- COPYRIGHT

NAME

Infoblox::Grid::Extattr - an extensible attribute representation for an attribute.

DESCRIPTION

An object that provides an extensible attribute representation for an attribute.

CONSTRUCTOR

```
my $extattr = Infoblox::Grid::Extattr->new(
   value                => $scalar | [$scalar1, $scalar2, ...],      # required
   inheritance_operation => 'DELETE' | 'INHERIT' | 'OVERRIDE',       # optional
   descendants_action    => $descendants,                            # optional
);
```

SESSION METHODS

The object does not support any session methods.

METHODS

This section describes all the methods that you can use to configure and retrieve the attribute values of the extensible attribute representation for an attribute.

descendants_action( )

Use this method to set the actions for extensible attributes that exist on descendants. This attribute is write-only and cannot be retrieved.

Parameter

Valid value is an Infoblox::Grid::ExtensibleAttributeDef::Descendants object.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

Example

```
#Modify attribute value
$extattr->descendants_action($descendants);
```

inheritance_operation( )

Use this method to set the operation that will be performed on an extensible attribute. This attribute is write-only and cannot be retrieved.

Parameter

Valid value is 'DELETE', 'INHERIT' or 'OVERRIDE'.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

Example

```
#Modify attribute value
$extattr->inheritance_operation('INHERIT');
```

inheritance_source( )

Use this method to retrieve the object from which the extensible attribute was inherited. This attribute is read-only and cannot be set

This object is considered a 'partial' object. See Infoblox::Session->fill_partial_object() for more information on how it can be converted to a 'full' API object.

Parameter

None.

Returns

The method returns the attribute value.

Example

```
#Get attribute value
my $value = $extattr->inheritance_source();
```

value( )

Use this method to set or retrieve the value of the extensible attribute representation for an attribute.

Parameter

For email, URL, and string types, the value is a string with a maximum of 256 characters.

For a list, the value is a reference to an array of strings with a maximum of 64 characters.

For an integer, the value is an integer from -2147483648 through 2147483647.

For a date, the value is the number of seconds that have elapsed since January 1st, 1970 UTC or a string in YYYY-MM-DD format. Any date entered will be converted to an epoch time corresponding to that day at 11:01am UTC.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get attribute value
my $value = $extattr->value();
#Modify attribute value
$extattr->value('www.infoblox.com');
```

AUTHOR

Infoblox Inc. http://www.infoblox.com/

SEE ALSO

Infoblox::Session, Infoblox::Grid::ExtensibleAttributeDef::Descendants

---

COPYRIGHT

Copyright (c) 2017 Infoblox Inc.

Infoblox::Grid::ExtensibleAttributeDef - Extensible Attribute Definition object.

- NAME
- DESCRIPTION
- Searching Extensible Attributes
  - String Searching
  - Integer Searching
  - Date Searching
- Extensible Attribute Values
- CONSTRUCTOR
- SESSION METHODS
  - Infoblox::Session->add( )
  - Infoblox::Session->get( )
  - Infoblox::Session->modify( )
  - Infoblox::Session->remove( )
  - Infoblox::Session->search( )
- METHODS
  - allowed_object_types( )
  - audit_log( )
  - comment( )
  - default_value( )
  - descendants_action( )
  - inheritable( )
  - list_values( )
  - max( )
  - min( )
  - multiple( )
  - cloud_api( )
  - grid_master( )
  - mgm_private( )
  - name( )
  - namespace( )
  - type( )
  - user_input( )
- SAMPLE CODE
- AUTHOR
- SEE ALSO
- COPYRIGHT

NAME

Infoblox::Grid::ExtensibleAttributeDef - Extensible Attribute Definition object.

DESCRIPTION

The Extensible Attribute Definition object is used to create or modify the definition of an extensible attribute. After the definition is created, the attribute can be defined on other Infoblox objects.

Searching Extensible Attributes

When searching for integer, string and date extensible attributes, the following constructs are supported, in addition to specifying the actual desired value.

String Searching

The string search can be modified by specifying the string in the following format:

```
 stringattr => { value => $wantedstring, ... options ... }
```

the following options are supported:

```
 case_insensitive => "true" | "false" # Optional. If "true", the string search will be case insensitive.
The default is "false".
```

Following is an example of a string search:

```
 @result = $session->search(object => 'Infoblox::DNS::Record::A',
                            extensible_attributes => { stringattr => { value => "some string",
case_insensitive => "true"});
```

Integer Searching

Integer searches support the following where X and Y are integers:

'> X' or '< X' - Matches if the extensible attribute value is GREATER or LESS than X respectively.
'>= X' or '<= X' - Matches if the extensible attribute value is GREATER THAN OR EQUALS or is LESS THAN OR EQUALS X respectively.
'!= X' or '/= X' - Matches if the extensible attribute value DOES NOT EQUAL X.
'>=< X,Y' - Matches if the extensible attribute value is GREATER THAN OR EQUALS X AND is LESS THAN OR EQUALS Y.

Following are examples of integer searches:

```
@result = $session->search(object => 'Infoblox::DNS::Record::A',
                           extensible_attributes => { integerattr => '<= 5'});

@result = $session->search(object => 'Infoblox::DNS::Record::A',
                           extensible_attributes => { integerattr => '!= 5'});

@result = $session->search(object => 'Infoblox::DNS::Record::A',
                           extensible_attributes => { integerattr => '>=< 5,10'});
```

Date Searching

Valid values for dates when searching are:

The number of seconds that have elapsed since January 1st, 1970 UTC.
A string matching 'YYYY-MM-DD'.
'today', 'yesterday', 'tomorrow'.
'this week', 'this month'.

Date searches support the following where X and Y are dates:

'> X' or '< X' - Matches if the extensible attribute value is GREATER or LESS than X respectively.
'>= X' or '<= X' - Matches if the extensible attribute value is GREATER THAN OR EQUALS or is LESS THAN OR EQUALS X respectively.
'>=< X,Y' - Matches if the extensible attribute value is GREATER THAN OR EQUALS X AND is LESS THAN OR EQUALS Y. This is supported only for epoch or YYYY-MM-DD strings.
'- n' or '+ n' where n is an integer - Matches if the extensible attribute value is within the past or next n days respectively, excluding today.

Following are examples of date searches:

```
@result = $session->search(object => 'Infoblox::DNS::Record::A',
                           extensible_attributes => { dateattr => '<= yesterday'});

@result = $session->search(object => 'Infoblox::DNS::Record::A',
                           extensible_attributes => { dateattr => 'this week'});

@result = $session->search(object => 'Infoblox::DNS::Record::A',
                           extensible_attributes => { dateattr => '>=< 2010-10-20,2010-11-01'});

@result = $session->search(object => 'Infoblox::DNS::Record::A',
                           extensible_attributes => { dateattr => '- 3'});
```

Extensible Attribute Values

The valid value for an extensible attribute is a hash reference containing the names of extensible attributes and their associated values.

The names must be those of existing Infoblox::Grid::ExtensibleAttributeDef objects in the database.

For email, URL, and string types, the value is a string with a maximum of 256 characters.

For a list, the value is a string with a maximum of 64 characters.

For an integer, the value is an integer from -2147483648 through 2147483647.

For a date, the value is the number of seconds that have elapsed since January 1st, 1970 UTC or a string of the format YYYY-MM-DD. Any date entered will be converted to an epoch time corresponding to that day at 11:01am UTC.

The default value for the member is undefined.

CONSTRUCTOR

```
my $ext_attr_def = Infoblox::Grid::ExtensibleAttributeDef->new(
```

```
    name                => $string,                                                    #Required
    type                => "string" | "integer" | "email" | "date" | "list" | "url",   #Optional
/ Default is "string"
    comment             => $string,                                                    #Optional
/ Default is undefined
    audit_log           => "true" | "false",                                           #Optional
/ Default is "false"
    inheritable         => "true" | "false",                                           #Optional
/ Default is "false"
    multiple            => "true" | "false",                                           #Optional
/ Default is "false"
    cloud_api           => "true" | "false",                                           #Optional
/ Default is "false"
    mgm_private         => "true" | "false",                                           #Optional
/ Default is "false"
    grid_master         => "true" | "false",                                           #Optional
/ Default is "false"
    default_value       => $string | $num,                                             #Optional
/ Default is undefined
    list_values         => [$Value1, $Value2, ...],                                    #Optional
/ Default is undefined (Mandatory if Extensible Attributes of type='list' is defined; optional otherwise)
    min                 => $num,                                                       #Optional
/ Default is undefined
    max                 => $num,                                                       #Optional
/ Default is undefined
    allowed_object_types=> [$string, $string, ...]                                     #Optional
/ Default is undefined
    user_input          => 'recommended' | 'required' | 'optional',                    #Optional
/ Default is 'optional'
    descendants_action  => $descendants,                                               #Optional
/ Default is undefined
 );
```

SESSION METHODS

This section describes all the methods in the Infoblox::Session module that you can apply to an Extensible Attribute Definition object.

Infoblox::Session->add( )

Use this method to add an object to the Infoblox appliance. See Infoblox::Session->add() for parameters and return values.

Example

```
# Construct an Extensible Attribute Definition object
my $ext_attr_def = Infoblox::Grid::ExtensibleAttributeDef->new(
    name                => "test_ext_attr",
    comment             => "test extensible attribute",
    type                => "integer"
);
# Submit for addition
my $response = $session->add( $ext_attr_def );
```

Infoblox::Session->get( )

Use this method to retrieve all the matching objects from the Infoblox appliance. See Infoblox::Session->get() for parameters and return values.

Key References

```
Apply the following attributes to get a specific Extensible Attribute Definition object:

 name      - Required. Extensible Attribute Definition name in string format.
 namespace - Optional. Extensible Attribute Definition namespace.
```

Example

```
my @retrieved_objs = $session->get(
    object     => "Infoblox::Grid::ExtensibleAttributeDef",
    name       => "test_ext_attr" );
```

Infoblox::Session->modify( )

Use this method to modify an object in the Infoblox appliance. See Infoblox::Session->modify() for parameters and return values.

Example

```
# Use this method to modify the comment.
$ext_attr_def->comment("This is a modified comment");
# Submit modification
```

```
my $response = $session->modify( $ext_attr_def );
```

Infoblox::Session->remove( )

Use this method to remove an object from the Infoblox appliance. See Infoblox::Session->remove() for parameters and return values.

To remove a specific object, use `get()` or `search()` to retrieve the specific object first, and then submit this object for removal.

Example

```
# Get the Extensible Attribute Definition object matching the given name
my @retrieved_objs = $session->get(
    object     => "Infoblox::Grid::ExtensibleAttributeDef",
    name       => "test_ext_attr" );
# Find the desired object from the retrieved list.
my $ext_attr_def = $retrieved_objs[0];
# Submit for removal
my $response = $session->remove( $ext_attr_def );
```

Infoblox::Session->search( )

Use this method to search for Extensible Attribute Definition objects in the Infoblox appliance. See Infoblox::Session->search() for parameters and return values.

Key References

```
Apply the following attributes to search for a specific Extensible Attribute Definition object:

 name      - Optional. Extensible Attribute Definition name (regular expression).
 comment   - Optional. Extensible Attribute Definition comment (regular expression).
 namespace - Optional. Extensible Attribute Definition namespace.
```

For more information about searching extensible attributes, see Infoblox::Grid::ExtensibleAttributeDef/Searching Extensible Attributes.

Example

```
# search for all Extensible Attribute Definition objects starting with "test"
my @retrieved_objs = $session->search(
    object => "Infoblox::Grid::ExtensibleAttributeDef",
    name   => "test.*" );
```

METHODS

This section describes all the methods that you can use to set and retrieve the attribute values of an Extensible Attribute Definition object.

allowed_object_types( )

Use this method to set or retrieve the object types on which the extensible attribute can be defined.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

Parameter is an array reference that contains the list of object types on which the extensible attribute can be defined. The following object types are supported here:

```
Infoblox::Grid::Member
Infoblox::Grid::Admin::User
Infoblox::Grid::Admin::Role
Infoblox::DHCP::FailOver
Infoblox::DHCP::Filter::Fingerprint
Infoblox::DHCP::Filter::MAC
Infoblox::DHCP::Filter::NAC
Infoblox::DHCP::Filter::Option
Infoblox::DHCP::Filter::RelayAgent
Infoblox::DHCP::Fingerprint
Infoblox::DHCP::FixedAddr
Infoblox::DHCP::FixedAddrTemplate
Infoblox::DHCP::IPv6FixedAddr
Infoblox::DHCP::IPv6FixedAddrTemplate
Infoblox::DHCP::IPv6Network
Infoblox::DHCP::IPv6Network
Infoblox::DHCP::IPv6NetworkContainer
Infoblox::DHCP::IPv6NetworkContainer
Infoblox::DHCP::IPv6NetworkTemplate
Infoblox::DHCP::IPv6Range
Infoblox::DHCP::IPv6RangeTemplate
Infoblox::DHCP::IPv6SharedNetwork
Infoblox::DHCP::MAC
Infoblox::DHCP::MSSuperscope
Infoblox::DHCP::Network
```

```
Infoblox::DHCP::NetworkContainer
Infoblox::DHCP::NetworkTemplate
Infoblox::DHCP::Range
Infoblox::DHCP::RangeTemplate
Infoblox::DHCP::RoamingHost
Infoblox::DHCP::SharedNetwork
Infoblox::DHCP::View
Infoblox::DNS::BulkHost
Infoblox::DNS::Host
Infoblox::DNS::RPZRecord::A
Infoblox::DNS::RPZRecord::AAAA
Infoblox::DNS::RPZRecord::AAAAIpAddress
Infoblox::DNS::RPZRecord::AIpAddress
Infoblox::DNS::RPZRecord::CNAME
Infoblox::DNS::RPZRecord::CNAME::ClientIpAddress
Infoblox::DNS::RPZRecord::CNAME::ClientIpAddressDN
Infoblox::DNS::RPZRecord::CNAME::IpAddress
Infoblox::DNS::RPZRecord::CNAME::IpAddressDN
Infoblox::DNS::RPZRecord::MX
Infoblox::DNS::RPZRecord::NAPTR
Infoblox::DNS::RPZRecord::PTR
Infoblox::DNS::RPZRecord::SRV
Infoblox::DNS::RPZRecord::TXT
Infoblox::DNS::Record::A
Infoblox::DNS::Record::AAAA
Infoblox::DNS::Record::Alias
Infoblox::DNS::Record::CNAME
Infoblox::DNS::Record::DNAME
Infoblox::DNS::Record::MX
Infoblox::DNS::Record::NS
Infoblox::DNS::Record::PTR
Infoblox::DNS::Record::SRV
Infoblox::DNS::Record::TLSA
Infoblox::DNS::Record::TXT
Infoblox::DNS::SRG
Infoblox::DNS::SharedRecord::A
Infoblox::DNS::SharedRecord::AAAA
Infoblox::DNS::SharedRecord::CNAME
Infoblox::DNS::SharedRecord::MX
Infoblox::DNS::SharedRecord::SRV
Infoblox::DNS::SharedRecord::TXT
Infoblox::DNS::View
Infoblox::DNS::Zone
Infoblox::DTC::LBDN
Infoblox::DTC::Monitor::HTTP
Infoblox::DTC::Monitor::ICMP
Infoblox::DTC::Monitor::PDP
Infoblox::DTC::Monitor::SIP
Infoblox::DTC::Monitor::SNMP
Infoblox::DTC::Monitor::TCP
Infoblox::DTC::Pool
Infoblox::DTC::Server
Infoblox::DTC::Topology
Infoblox::Grid::AllEndpoints
Infoblox::Grid::DNS
Infoblox::Grid::DNS::DNS64Group
Infoblox::Grid::DNS::Nsgroup
Infoblox::Grid::DNS::Nsgroup::DelegationMember
Infoblox::Grid::DNS::Nsgroup::ForwardStubServer
Infoblox::Grid::DNS::Nsgroup::ForwardingMember
Infoblox::Grid::Discovery::Device
Infoblox::Grid::Discovery::DeviceInterface
Infoblox::Grid::MSServer
Infoblox::Grid::Member::DNS
Infoblox::Grid::NamedACL
Infoblox::Grid::ParentalControl::SubscriberSite
Infoblox::Grid::RIR::Organization
Infoblox::Grid::SNMP::User
Infoblox::Grid::ServiceRestart::Group
```

If the parameter is undefined, the extensible attribute can be defined on all object types. The default value is undefined.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
# Get allowed object types
my $ref_allowed_object_types = $ext_attr_def->allowed_object_types();
# Modify allowed object types
$ext_attr_def->allowed_object_types(['Infoblox::DHCP::Range', 'Infoblox::DHCP::RangeTemplate']);
```

audit_log( )

Use this method to set or retrieve the "audit_log" flag.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

Specify "true" to set the audit_log flag or "false" to deactivate/unset it. When the flag is set the extensible attribute name and value assigned to an object are logged in the audit log when a change occurs on the object. The default value for this field is "false".

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
# Get audit_log flag
my $audit_log = $ext_attr_def->audit_log();
# Modify audit_log flag
$ext_attr_def->audit_log("true");
```

comment( )

Use this method to set or retrieve the comment.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

Comment in string format with a maximum of 256 characters. The default value is undefined.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get comment
my $comment = $ext_attr_def->comment();
#Modify comment
$ext_attr_def->comment("This is the modified comment for extensible attribute definition");
```

default_value( )

Use this method to set or retrieve the value used to pre-populate the attribute value in the GUI.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

Default value used to pre-populate the attribute value in the GUI. For email, URL, and string types, the value is a string with a maximum of 256 characters. For a list, the value is a string with a maximum of 64 characters. For an integer, the value is an integer from -2147483648 through 2147483647. For a date, the value is the number of seconds that have elapsed since January 1st, 1970 UTC. The default value is undefined.

Note that, for a date, the time value is automatically modified to 11:01:00 before insertion, so you may not get the same value that you've inserted upon retrieval. e.g. 1000000 (1970-01-12T13:46:40Z) will be returned as 990060 (1970-01-12T11:01:00Z).

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
# Get default value of an extensible attribute of type string
my $ext_attr_def = $ext_attr_def->default_value();
# Modify default value
$ext_attr_def->default_value("Router");
```

descendants_action( )

Use this method to set the actions for extensible attributes that exist on descendants. This attribute is write-only and cannot be retrieved.

Parameter

Valid value is an Infoblox::Grid::ExtensibleAttributeDef::Descendants object.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

Example

```
#Modify attribute value
$ext_attr_def->descendants_action($descendants);
```

inheritable( )

Use this method to set or retrieve the "inheritable" flag.

Parameter

Specify "true" to set the inheritable flag or "false" to deactivate/unset it. The default value is "false".

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
# Get inheritable flag
my $inheritable = $ext_attr_def->inheritable();
# Modify inheritable flag
$ext_attr_def->inheritable("true");
```

list_values( )

Use this method to set or retrieve the list of values that can be defined for the list type.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

Parameter is an array reference that contains Infoblox::Grid::ExtensibleAttributeDef::ListValue values associated with an extensible attribute object defined as the "list" type. This parameter is not used for other types. The default value for this field is undefined.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
# Get list of values
my $ref_list_values = $ext_attr_def->list_values();

# Add a new value to the list
my $ref_list_values = $ext_attr_def->list_values();
push @$ref_list_values, Infoblox::Grid::ExtensibleAttributeDef::ListValue->new(value => 'Santa
Clara');
$ext_attr_def->list_values($ref_list_values);

# Rename a value in the list
my $ref_list_values = $ext_attr_def->list_values();
foreach my $value (@$ref_list_values)
{
   if ($value->value() eq 'San Francisco')
   {
       $value->value('Los Angeles');
   }
}
$ext_attr_def->list_values($ref_list_values);
```

max( )

Use this method to set or retrieve the maximal length of string for string type or maximal value for integer type.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

For a string, the value is an integer from 1 through 256. For an integer, the value is an integer from -2147483648 through

2147483647. This parameter is not used for other types. Using the Infoblox::Session->modify() function you can only define a value greater than the current one. The default value is undefined.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
# Get max
my $ext_attr_def = $ext_attr_def->max();
# Modify max
$ext_attr_def->max(23);
```

min( )

Use this method to set or retrieve the minimal length of string for string type or minimal value for integer type.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

For a string, the value is an integer from 1 through 256. For an integer, the value is an integer from -2147483648 through 2147483647. This parameter is not used for other types. Using the Infoblox::Session->modify() function you can only define a value lesser than the current one. The default value is undefined.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
# Get min
my $ext_attr_def = $ext_attr_def->min();
# Modify min
$ext_attr_def->min(3);
```

multiple( )

Use this method to set or retrieve the "multiple" flag.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

Specify "true" to set the multiple flag or "false" to deactivate/unset it. When the flag is set multiple values can be assigned to an extensible attribute. Using the Infoblox::Session->modify() function you can change this flag from "false" to "true". The reverse operation is not allowed. The default value for this field is "false".

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
# Get multiple flag
my $audit_log = $ext_attr_def->multiple();
# Modify multiple flag
$ext_attr_def->multiple("true");
```

cloud_api( )

Use this method to set or retrieve the "cloud_api" flag.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

Specify "true" to set the cloud_api flag or "false" to deactivate or unset it. The default value is "false".

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
# Get cloud_api flag
my $cloud_api = $ext_attr_def->cloud_api();
# Modify cloud_api flag
$ext_attr_def->cloud_api("true");
```

grid_master( )

Use this method to set or retrieve the "grid_master" flag.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

Specify "true" to set the grid_master flag or "false" to deactivate or unset it. The default value is "false".

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
# Get grid_master flag
my $grid_master = $ext_attr_def->grid_master();
# Modify grid_master flag
$ext_attr_def->grid_master("true");
```

mgm_private( )

Use this method to set or retrieve the "mgm_private" flag.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

Specify "true" to set the mgm_private flag or "false" to deactivate or unset it. The default value is "false".

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
# Get mgm_private flag
my $mgm_private = $ext_attr_def->mgm_private();
# Modify mgm_private flag
$ext_attr_def->mgm_private("true");
```

name( )

Use this method to set or retrieve the name of an extensible attribute definition.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

Name of the extensible attribute with a maximum of 128 characters.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
# Get name
my $name = $ext_attr_def->name();
# Modify name
$ext_attr_def->name("Class");
```

namespace( )

Use this method to retrieve the extensible attribute definition namespace. This is a read-only attribute.

Specific extensible attribute definitions with 'RIPE' namespace will be added only when support for RIR/SWIP is enabled at the Grid level for the first time. See Infoblox::Grid->enable_rir_swip() method.

Parameter

None

Returns

The method returns the attribute value that can be 'default', 'MSADSITES', 'RIPE' or 'CLOUD'.

Example

```
# Get namespace
$namespace = $ext_attr_def->namespace();
```

type( )

Use this method to set or retrieve the type of an extensible attribute.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

Parameter is a string. The default value for this field is "string". Currenty the supported object types are 'string', 'integer', 'email', 'date', 'list' and 'url'. Modification of the type using the Infoblox::Session->modify() function is not allowed.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get type
my $type = $ext_attr_def->type();
```

user_input( )

Use this method to set or retrieve the flag that indicates whether the extensible attribute value is required, recommended or optional.

Parameter

Valid values are 'recommended', 'required' or 'optional'. The default value is 'optional'.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get user_input
my $user_input = $ext_attr_def->user_input();
#Modify user_input
$ext_attr_def->user_input('required');
```

SAMPLE CODE

The following sample code demonstrates the different functions that can be applied to an object, such as add, get, modify, and remove. This sample also includes error handling for the operations.

#Preparation prior to an Extensible Attribute Definition object insertion

```
#PROGRAM STARTS: Include all the modules that will be used
use strict;
use Infoblox;

#Create a session to the Infoblox appliance

my $session = Infoblox::Session->new(
                                        #appliance host ip
```

```
                master   => "192.168.1.2",
                username => "admin",        #appliance user login
                password => "infoblox"      #appliance password
                );

unless (($session) {
        die("Construct session failed: ",
                Infoblox::status_code() . ":" . Infoblox::status_detail());
}
print "Session created successfully\n";
```

#Create an Extensible Attribute Definition object

```
my $ext_attr_def = Infoblox::Grid::ExtensibleAttributeDef->new(
    name                => "test_extensible_atttribute",
    comment             => "test extensibleatttribute",
    type                => "integer",
    min                 => -50,
    max                 => 50,
);

unless (($ext_attr_def) {
        die("Construct extensible attribute definition object failed: ",
                Infoblox::status_code() . ":" . Infoblox::status_detail());
}
print "Extensible Attribute Definition object created successfully\n";

#Add this object to the Infoblox appliance through a session
$session->add($ext_attr_def)
        or die("Add Extensible Attribute Definition object failed: ",
                        $session->status_code() . ":" . $session->status_detail());
print "Extensible Attribute Definition object added to server successfully\n";
```

#Get and modify an Extensible Attribute Definition object

```
#Get the object from Infoblox appliance through a session
my @retrieved_objs = $session->get(
    object => "Infoblox::Grid::ExtensibleAttributeDef",
    name   => "test_extensible_atttribute"
);
my $object = $retrieved_objs[0];

unless (($object) {
        die("Get Extensible Attribute Definition object failed: ",
                $session->status_code() . ":" . $session->status_detail());
}
print "Get Extensible Attribute Definition object found at least 1 matching entry\n";

#Modify the object
$object->comment("this is a new comment");
$object->min(-100);
$object->max(100);

#Apply the changes.
$session->modify($object)
        or die("Modify Extensible Attribute definition object failed: ",
                $session->status_code() . ":" . $session->status_detail());
print "Extensible Attribute definition object modified successfully \n";

#Accessor methods
print "name: " . $object->name() . "\n";
print "comment: " . $object->comment() . "\n";
print "min: " . $object->min() . "\n";
print "max: " . $object->max() . "\n";
```

#Remove an Extensible Attribute Definition object

```
#Get the object through the session
my @retrieved_objs = $session->get(
    object => "Infoblox::Grid::ExtensibleAttributeDef",
    name   => "test_extensible_atttribute"
);
my $object = $retrieved_objs[0];
unless (($object) {
    die("Get Extensible Attribute Definition object failed: ",
        $session->status_code() . ":" . $session->status_detail());
}
```

```
print "Get Extensible Attribute Definition object found at least 1 matching entry\n";

#Submit the object for removal
$session->remove($object)
       or die("Remove Extensible Attribute Definition object failed: ",
                $session->status_code() . ":" . $session->status_detail());
print "Extensible Attribute Definition object removed successfully \n";

####PROGRAM ENDS####
```

AUTHOR

Infoblox Inc. http://www.infoblox.com/

SEE ALSO

Infoblox::Session->add(), Infoblox::Session->get(), Infoblox::Session->modify(), Infoblox::Session->remove(), Infoblox::Session->search(),Infoblox::Session,Infoblox::Grid::ExtensibleAttributeDef::Descendants

---

COPYRIGHT

Copyright (c) 2019 Infoblox Inc.

Infoblox::Grid::ExtensibleAttributeDef::Descendants - an action for the extensible attributes that exist on descendants.

- NAME
- DESCRIPTION
- CONSTRUCTOR
- SESSION METHODS
- METHODS
  - option_delete_ea( )
  - option_with_ea( )
  - option_without_ea( )
- AUTHOR
- COPYRIGHT

NAME

Infoblox::Grid::ExtensibleAttributeDef::Descendants - an action for the extensible attributes that exist on descendants.

DESCRIPTION

An object that provides the action for extensible attributes that exist on descendants.

CONSTRUCTOR

```
my $descendants = Infoblox::Grid::ExtensibleAttributeDef::Descendants->new(
    option_without_ea => "INHERIT" | "NOT_INHERIT",             # optional
    option_with_ea    => "RETAIN" | "CONVERT" | "INHERIT",      # optional
    option_delete_ea  => "RETAIN" | "REMOVE",                   # optional
);
```

SESSION METHODS

The object does not support any session methods.

METHODS

This section describes all the methods that you can use to configure and retrieve attribute values of the extensible attributes that exist on descendants based on the corresponding action.

option_delete_ea( )

Use this method to set or retrieve the action for objects that contain inheritable extensible attributes if a parent object is deleted.

Parameter

Valid value is "RETAIN" or "REMOVE".

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get attribute value
my $value = $descendants->option_delete_ea();
#Modify attribute value
$descendants->option_delete_ea('RETAIN');
```

option_with_ea( )

Use this method to set or retrieve the action for objects that contain inheritable extensible attributes.

Parameter

Valid value is "RETAIN", "CONVERT" or "INHERIT".

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get attribute value
my $value = $descendants->option_with_ea();
#Modify attribute value
$descendants->option_with_ea('INHERIT');
```

option_without_ea( )

Use this method to set or retrieve the action for objects that do not contain inheritable extensible attributes.

Parameter

Valid value is "INHERIT" or "NOT_INHERIT".

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get attribute value
my $value = $descendants->option_without_ea();
#Modify attribute value
$descendants->option_without_ea('NOT_INHERIT');
```

AUTHOR

Infoblox Inc. http://www.infoblox.com/

COPYRIGHT

Copyright (c) 2017 Infoblox Inc.

Infoblox::Grid::ExtensibleAttributeDef::ListValue - Extensible Attribute Definition List Value object.

- NAME
- DESCRIPTION
- CONSTRUCTOR
- METHODS
  - value( )
- AUTHOR
- SEE ALSO
- COPYRIGHT

NAME

Infoblox::Grid::ExtensibleAttributeDef::ListValue - Extensible Attribute Definition List Value object.

DESCRIPTION

The Extensible Attribute Definition List Value object is used to store values for an extensible attribute object defined as the "list" type.

CONSTRUCTOR

```
my $ext_attr_def = Infoblox::Grid::ExtensibleAttributeDef::ListValue->new(
    value                => $string,                                          #Required
);
```

METHODS

This section describes all the methods that you can use to set and retrieve the values of an Extensible Attribute Definition List Value object.

value( )

Use this method to set or retrieve the value.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

Parameter is a string.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get value
my $value = $ext_attr_def_value->value();
#Modify value
$ext_attr_def_value->value($value);
```

AUTHOR

Infoblox Inc. http://www.infoblox.com/

SEE ALSO

Infoblox::Grid::ExtensibleAttributeDef

COPYRIGHT

Copyright (c) 2017 Infoblox Inc.

Infoblox::Grid::FileDistribution - Grid FileDistribution object.

- NAME
- DESCRIPTION
- CONSTRUCTOR
- SESSION METHODS
  - Infoblox::Session->get( )
  - Infoblox::Session->modify( )
- METHODS
  - allow_uploads( )
  - backup_storage( )
  - current_usage( )
  - enable_anonymous_ftp( )
  - global_status( )
  - storage_limit( )
- SAMPLE CODE
- AUTHOR
- SEE ALSO
- COPYRIGHT

NAME

Infoblox::Grid::FileDistribution - Grid FileDistribution object.

DESCRIPTION

You can configure the file distribution properties of a grid. You can enable and disable TFTP, HTTP, and FTP services, and set their operational parameters.

CONSTRUCTOR

The Infoblox::Grid::FileDistribution object is automatically generated upon the successful installation of the Infoblox appliance. It does not require manual construction.

SESSION METHODS

This section describes all the methods in the Infoblox::Session module that you can apply to a grid member FileDistribution object.

Infoblox::Session->get( )

Use this method to retrieve all the matching objects from the Infoblox appliance. See Infoblox::Session->get() for parameters and return values.

Example

```
my $object   = $session->get(
   object    => "Infoblox::Grid::FileDistribution",
   );
```

Infoblox::Session->modify( )

Use this method to modify an object in the Infoblox appliance. See Infoblox::Session->modify() for parameters and return values.

Example

```
#Use method to modify the enable_tftp Property.
$object->enable_tftp("true");
#Submit modification
my $response = $session->modify( $object );
```

METHODS

This section describes all the methods that you can use to configure and retrieve the attribute values of the file distribution properties of a grid member.

allow_uploads( )

Use this method to enable or disable file uploads for the grid.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

Specify "true" to enable the file uploads or "false" to disable it. The default value is "false".

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Getting allow_uploads
$allow_uploads = $grid->allow_uploads( );
#Modifying allow_uploads
$grid->allow_uploads("true");
```

backup_storage( )

Use this method to control whether distributed files are included in the backup or not.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

Specify "true" to enable the backup of distributed files or "false" to disable it. The default value is "true".

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Getting backup_storage
$backup_storage = $grid->backup_storage( );
#Modifying backup_storage
$grid->backup_storage("true");
```

current_usage( )

Use this method to retrieve the current usage of the allocated space.

Parameter

The value is the percentage of the usage of the allocated space expressed in tenth of a %. Valid values are from 0 to 1000.

Returns

The method returns the attribute value.

Example

```
#Getting current_usage
$current_usage = $grid->current_usage( );
```

enable_anonymous_ftp( )

Use this method to enable or disable anonymous FTP access for the grid.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

Specify "true" to enable anonymous FTP access or "false" to disable it. The default value is "false".

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Getting enable_anonymous_ftp
$enable_anonymous_ftp = $grid->enable_anonymous_ftp( );
```

```
#Modifying enable_anonymous_ftp
$grid->enable_anonymous_ftp("true");
```

global_status( )

Use this method to retrieve the overall status of all grid members.

Parameter

The method returns the "worst" status among all grid members. Valid values are "FAILED", "INACTIVE", "UNKNOWN", "WARNING" or "WORKING".

Returns

The method returns the attribute value.

Example

```
#Getting global_status
$global_status = $grid->global_status( );
```

storage_limit( )

Use this method to set the maximum TFTP storage allowed on the grid.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

The maximum storage size expressed in Megabytes. Valid values are from 1 to 10000.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Getting storage_limit
$storage_limit = $grid->storage_limit( );
#Modifying storage_limit
$grid->storage_limit(2000);
```

SAMPLE CODE

The following sample code demonstrates the different functions that can be applied to an object, such as modify and get. This sample also includes error handling for the operations.

#Preparation prior to grid FileDistribution object operations

```
#PROGRAM STARTS: Include all the modules that will be used

use strict;
use Infoblox;

#Create a session to the Infoblox appliance
my $session = Infoblox::Session->new(
    master   => "192.168.1.2",
    username => "admin",
    password => "infoblox"
);
unless ($session) {
   die("Construct session failed: ",
       Infoblox::status_code() . ":" . Infoblox::status_detail());
}
print "Session created successfully\n";
```

#Get and modify a Grid FileDistribution object

```
# Get and modify the grid FileDistribution property object.
my $object = $session->get(
    object => "Infoblox::Grid::FileDistribution",
);

 unless ($object) {
      die("get grid FileDistribution failed: ",
```

```
        $session->status_code() . ":" . $session->status_detail());
    }
    print "grid FileDistribution get successful\n";

#Modifying the value of the specified object
$object->enable_anonymous_ftp("true");

#Apply the changes
$session->modify($object)
    or die("modify grid FileDistribution failed: ",
        $session->status_code() . ":" . $session->status_detail());

print "FileDistribution grid object modified successfully \n";

####PROGRAM ENDS####
```

AUTHOR

Infoblox Inc. http://www.infoblox.com/

SEE ALSO

Infoblox::Session, Infoblox::Session->get(), Infoblox::Session->modify()

COPYRIGHT

Copyright (c) 2017 Infoblox Inc.

Infoblox::Grid::FileDistributionDir - Grid FileDistributionDir object.

- NAME
- DESCRIPTION
- CONSTRUCTOR
- SESSION METHODS
  - Infoblox::Session->add( )
  - Infoblox::Session->get( )
  - Infoblox::Session->modify( )
  - Infoblox::Session->remove( )
  - Infoblox::Session->search( )
- METHODS
  - name( )
- SAMPLE CODE
- AUTHOR
- SEE ALSO
- COPYRIGHT

NAME

Infoblox::Grid::FileDistributionDir - Grid FileDistributionDir object.

DESCRIPTION

The FileDistributionDir object is used to add, get, modify, and remove directories that store files for a file distribution service.

CONSTRUCTOR

```
my $dir = Infoblox::Grid::FileDistributionDir->new(
    name        => $string,             #Required
);
```

SESSION METHODS

This section describes all the methods in the Infoblox::Session module that you can apply to a FileDistributionDir object.

Infoblox::Session->add( )

Use this method to add an object to the Infoblox appliance. See Infoblox::Session->add() for parameters and return values.

Example

```
#Construct the file distribution directory object
my $dir = Infoblox::Grid::FileDistributionDir->new(
    name        => "mydir",
);
# Submit for addition
my $response = $session->add( $dir );
```

Infoblox::Session->get( )

Use this method to retrieve all the matching objects from the Infoblox appliance. See Infoblox::Session->get() for parameters and return values.

Key References

```
Apply the following attributes to get a specific FileDistributionDir object:

 name          - Required. User name in string format.
```

Example

```
my @retrieved_objs = $session->get(
    object      => "Infoblox::Grid::FileDistributionDir",
    name        => "mydir" );
```

Infoblox::Session->modify( )

Use this method to modify an object in the Infoblox appliance. See Infoblox::Session->modify() for parameters and return values.

Example

```
# Use method to rename a file distribution directory.
$dir->name("mynewdir");
# Submit modification
my $response = $session->modify( $dir );
```

Infoblox::Session->remove( )

Use this method to remove an object from the Infoblox appliance. See Infoblox::Session->remove() for parameters and return values.

To remove a specific object, use `get()` or `search()` to retrieve the specific object first, and then submit this object for removal.

Example

```
# Get the user objects with the same starting name
my @retrieved_objs = $session->get(
    object      => "Infoblox::Grid::FileDistributionDir",
    name        => "mydir" );
# find the desired object from the retrieved list.
my $dir = $retrieved_objs[0];
# Submit for removal
my $response = $session->remove( $dir );
```

Infoblox::Session->search( )

Use this method to search for FileDistributionDir objects in the Infoblox appliance. See Infoblox::Session->search() for parameters and return values.

Key References

```
Apply the following attributes to search for a specific FileDistributionDir object:

 name          - Required. Directory name in string format. Only the exact match is supported (that
is, exactly the same as 'get' method).
```

Example

```
# search for all FileDistributionDir objects with name "mydir"
my @retrieved_objs = $session->search(
    object      => "Infoblox::Grid::FileDistributionDir",
    name        => "mydir");
```

METHODS

This section describes all the methods that you can use to set and retrieve the attribute values of a FileDistributionDir object.

name( )

Use this method to set or retrieve the name of a file distribution directory.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

Text with the name of the file distribution directory. Enter the complete path to the file distribution directory, including all existing intermediate directories.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get name of the file distribution directory
my $name = $file_distribution->name();
#Modify name of the file distribution directory (i.e. rename the directory)
$file_distribution->name("mynewdir");
#Create a new subdirectory 'subdir' under 'mynewdir' ('mynewdir' must exist)
$file_distribution->name("mynewdir/subdir");
```

SAMPLE CODE

The following sample code demonstrates the different functions that can be applied to an object, such as add, search, modify, and remove. This sample also includes error handling for the operations.

#Preparation prior to a FileDistributionDir object insertion

```
#PROGRAM STARTS: Include all the modules that will be used
use strict;
use Infoblox;

#Create a session to the Infoblox appliance

my $session = Infoblox::Session->new(
               master   => "192.168.1.2", #appliance host ip
               username => "admin",       #appliance user login
               password => "infoblox"     #appliance password
               );

unless ($session) {
       die("Construct session failed: ",
               Infoblox::status_code() . ":" . Infoblox::status_detail());
}
print "Session created successfully\n";
```

#Create a FileDistributionDir object

```
my $dir = Infoblox::Grid::FileDistributionDir->new(
    name        => "mydir",
);

unless ($dir) {
       die("Construct file distribution directory object failed: ",
               Infoblox::status_code() . ":" . Infoblox::status_detail());
}
print "FileDistributionDir object created successfully\n";

#Add the file distribution dir object to the Infoblox appliance through a session
$session->add($dir)
       or die("Add FileDistributionDir object failed: ",
               $session->status_code() . ":" . $session->status_detail());
print "FileDistributionDir object added to server successfully\n";
```

#Search for a file distribution directory

```
my @retrieved_objs = $session->search(
               object      => "Infoblox::Grid::FileDistributionDir",
               name        => "mydir"
              );
my $object = $retrieved_objs[0];

unless ($object) {
       die("Search FileDistributionDir object failed: ",
               $session->status_code() . ":" . $session->status_detail());
}
print "Search FileDistributionDir object found at least 1 matching entry\n";
```

#Get and modify a FileDistributionDir object

```
#Get the file distribution dir object from Infoblox appliance through a session
my @retrieved_objs = $session->get(
            object      => "Infoblox::Grid::FileDistributionDir",
            name        => "mydir",
);
my $object = $retrieved_objs[0];

unless ($object) {
       die("Get FileDistributionDir object failed: ",
               $session->status_code() . ":" . $session->status_detail());
}
print "Get FileDistributionDir object found at least 1 matching entry\n";

#Modify the name of the FileDistributionDir object (that is, rename the directory)
$object->name("mynewdir");

#Apply the changes.
$session->modify($object)
       or die("Modify User object failed: ",
               $session->status_code() . ":" . $session->status_detail());
print "FileDistributionDir object modified successfully \n";
```

#Remove a FileDistributionDir object

```
#Get a FileDistributionDir object through the session
my @retrieved_objs = $session->get(
            object      => "Infoblox::Grid::FileDistributionDir",
            name        => "mynewdir",
);
my $object = $retrieved_objs[0];
unless ($object) {
    die("Get FileDistributionDir object failed: ",
        $session->status_code() . ":" . $session->status_detail());
}
print "Get FileDistributionDir object found at least 1 matching entry\n";

#Submit an object for removal
$session->remove($object)
       or die("Remove FileDistributionDir object failed: ",
                $session->status_code() . ":" . $session->status_detail());
print "FileDistributionDir object removed successfully \n";

####PROGRAM ENDS####
```

AUTHOR

Infoblox Inc. http://www.infoblox.com/

SEE ALSO

Infoblox::Grid::Member::FileDistribution, Infoblox::Session->add(), Infoblox::Session->get(), Infoblox::Session->modify(), Infoblox::Session->remove(), Infoblox::Session->search(), Infoblox::Session

---

COPYRIGHT

Copyright (c) 2017 Infoblox Inc.

Infoblox::Grid::GlobalSmartFolder - Global Smart Folder object.

- NAME
- DESCRIPTION
- CONSTRUCTOR
- SESSION METHODS
  - Infoblox::Session->add( )
  - Infoblox::Session->get( )
  - Infoblox::Session->modify( )
  - Infoblox::Session->remove( )
  - Infoblox::Session->search( )
- METHODS
  - comment( )
  - group_bys( )
  - name( )
  - query_items( )
- SAMPLE CODE
- AUTHOR
- SEE ALSO
- COPYRIGHT

NAME

Infoblox::Grid::GlobalSmartFolder - Global Smart Folder object.

DESCRIPTION

A Global Smart Folder object is used for data organization.

CONSTRUCTOR

```
 my $group = Infoblox::Grid::GlobalSmartFolder->new(
     name                => $string,                                    #Required
     comment             => $string,                                    #Optional
/ Default is undefined
     group_bys           => [$GroupbyItem1, $GroupbyItem2],             #Optional
/ Default is undefined
     query_items         => [$QueryItem1, $QueryItem2],                 #Optional
/ Default is undefined
 );
```

SESSION METHODS

This section describes all the methods in the Infoblox::Session module that you can apply to a Global Smart Folder object.

Infoblox::Session->add( )

Use this method to add an object to the Infoblox appliance. See Infoblox::Session->add() for parameters and return values.

Example

```
#Construct the QueryItem in-memory object
my $query_item1 = Infoblox::Grid::SmartFolder::QueryItem->new(
    name                => "location",
    operator            => "eq",
    value               => "Santa Clara",
);

#Construct the GlobalSmartFolder object
my $group_by_state = Infoblox::Grid::SmartFolder::GroupBy->new(
                                                    enable_grouping => 'true',
                                                    value => "State",
                                                    value_type => "EXTATTR",
                                                   );

my $smart_folder = Infoblox::Grid::GlobalSmartFolder->new(
    name                => "smartfolder1",
    comment             => "global smart folder",
    group_bys           => [$group_by_state],
    query_items         => [$query_item1],
```

```
);
# Submit for addition
my $response = $session->add( $smart_folder );
```

Infoblox::Session->get( )

Use this method to retrieve all the matching objects from the Infoblox appliance. See Infoblox::Session->get() for parameters and return values.

Key References

```
Apply the following attributes to get a specific GlobalSmartFolder object:

 name      - Required. GlobalSmartFolder name in string format.
 comment   - Optional. Comment in string format.
```

Example

```
my @retrieved_objs = $session->get(
    object     => "Infoblox::Grid::GlobalSmartFolder",
    name       => "smartfolder1" );
```

Infoblox::Session->modify( )

Use this method to modify an object in the Infoblox appliance. See Infoblox::Session->modify() for parameters and return values.

Example

```
# Use this method to modify the comment.
$smart_folder->comment("This is a modified comment");
# Submit modification
my $response = $session->modify( $smart_folder );
```

Infoblox::Session->remove( )

Use this method to remove an object from the Infoblox appliance. See Infoblox::Session->remove() for parameters and return values.

To remove a specific object, use `get()` or `search()` to retrieve the specific object first, and then submit this object for removal.

Example

```
# Get the GlobalSmartFolder objects using a name
my @retrieved_objs = $session->get(
    object     => "Infoblox::Grid::GlobalSmartFolder",
    name       => "smartfolder1" );
# find the desired object from the retrieved list.
my $desired_smart_folder = $retrieved_objs[0];
# Submit for removal
my $response = $session->remove( $desired_smart_folder );
```

Infoblox::Session->search( )

Use this method to search for Global Smart Folder objects in the Infoblox appliance. See Infoblox::Session->search() for parameters and return values.

Key References

```
Apply the following attributes to search for a specific GlobalSmartFolder object:

 name      - Optional. GlobalSmartFolder name in string format (regular expression).
 comment   - Optional. Comment in string format (regular expression).
```

Example

```
# search for all GlobalSmartFolder objects that start with "smartfolder"
my @retrieved_objs = $session->search(
    object => "Infoblox::Grid::GlobalSmartFolder",
    name   => "smartfolder.*" );
```

METHODS

This section describes all the methods that you can use to set and retrieve the attribute values of a GlobalSmartFolder object.

comment( )

Use this method to set or retrieve the comment.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

Pertinent information about the global smart folder. Comment in string format with a maximum of 256 bytes. The default value is undefined.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get comment
my $comment = $smart_folder->comment();
#Modify comment
$smart_folder->comment("This is the modified comment for smartfolder1");
```

group_bys( )

Use this method to set or retrieve the group_bys value.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

The valid value is an array reference that contains Infoblox::Grid::SmartFolder::GroupBy object(s). The default value is undefined.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
my $group_by_state = Infoblox::Grid::SmartFolder::GroupBy->new(
                                                    enable_grouping => 'true',
                                                    value => "State",
                                                    value_type => "EXTATTR",
                                                   );
#Get group_by value
my $group_by = $smart_folder->group_bys();
#Modify group_bys value
$smart_folder->group_bys([$group_by_state]);
```

name( )

Use this method to set or retrieve the name of a global smart folder.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

Text with the name of the global smart folder.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get name
my $name = $smart_folder->name();
```

query_items( )

Use this method to set or retrieve the query items. If query items are not specified when you `add()` a smart folder, the server automatically assigns a default query item for the smart folder. This default query item is "type=Network/Zone/Range/Member". This ensures that objects are included in searches for these 4 types, by default. When you `modify()` a smart folder, the server keeps the query items provided by the client and does not add the default items.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

The valid value is an array reference that contains Infoblox::Grid::SmartFolder::QueryItem object(s). The default value is undefined.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get query items
my $query_items = $smart_folder->query_items();

#Modify query items
my $query_item1 = Infoblox::Grid::SmartFolder::QueryItem->new(
    name                => "location",
    operator            => "eq",
    value               => "Santa Clara",
);
$smart_folder->query_items([$query_items1]);
```

SAMPLE CODE

The following sample code demonstrates the different functions that can be applied to an object, such as add, search, modify, and remove. This sample also includes error handling for the operations.

#Preparation prior to an GlobalSmartFolder object insertion

```
#PROGRAM STARTS: Include all the modules that will be used
use strict;
use Infoblox;

#Create a session to the Infoblox appliance

my $session = Infoblox::Session->new(
              master   => "192.168.1.2", #appliance host ip
              username => "admin",       #appliance user login
              password => "infoblox"     #appliance password
              );

unless ($session) {
       die("Construct session failed: ",
              Infoblox::status_code() . ":" . Infoblox::status_detail());
}
print "Session created successfully\n";

# Add a network with extensible attributes

my %extensible_attributes=(
    'Region' => 'Santa Clara County',
    'Country' => 'USA',
    'State' => 'California',
    'Site' => 'Santa Clara',
    'Building' => 'HQ',
    'VLAN' => '1',
    );

my $network = Infoblox::DHCP::Network->new(
    network => "10.0.0.0/24",
    extensible_attributes => \%extensible_attributes,
    );
unless ($network) {
  die("Construct Network failed: ",
      Infoblox::status_code() . ":" . Infoblox::status_detail());
}
print "Network created successfully\n";

$session->add($network)
  or die("Add Network object failed: ",
      $session->status_code() . ":" . $session->status_detail());
print "Network object added to server successfully\n";
```

#Create a GlobalSmartFolder object

```
my $query_item = Infoblox::Grid::SmartFolder::QueryItem->new(
    name                    => "Country",
    is_extensible_attribute => "true",
    operator                => "eq",
    value                   => "USA",
    );

unless ($query_item) {
  die("Construct query item object failed: ",
      Infoblox::status_code() . ":" . Infoblox::status_detail());
}
print "Smart folder query item object created successfully\n";

my $group_by_country = Infoblox::Grid::SmartFolder::GroupBy->new(
                                                        enable_grouping => 'true',
                                                        value => "Country",
                                                        value_type => "EXTATTR",
                                                       );

my $smart_folder = Infoblox::Grid::GlobalSmartFolder->new(
    name                    => "my_folder_global",
    comment                 => "my global folder",
    query_items             => [$query_item],
    group_bys               => [$group_by_country],
    );

unless ($smart_folder) {
  die("Construct global smart folder object failed: ",
      Infoblox::status_code() . ":" . Infoblox::status_detail());
}
print "Smart folder object created successfully\n";

#Add the GlobalSmartFolder object to the Infoblox appliance through a session
$session->add($smart_folder)
  or die("Add GlobalSmartFolder object failed: ",
      $session->status_code() . ":" . $session->status_detail());
print "GlobalSmartFolder object added to server successfully\n";
```

#Search for a GlobalSmartFolder

```
my @retrieved_objs = $session->search(
    object => "Infoblox::Grid::GlobalSmartFolder",
    name   => "my_folder.*"
    );
my $object = $retrieved_objs[0];

unless ($object) {
  die("Search for a GlobalSmartFolder object failed: ",
      $session->status_code() . ":" . $session->status_detail());
}
print "Search for a GlobalSmartFolder object found at least 1 matching entry\n";
```

#Get and modify a GlobalSmartFolder object

```
#Get the GlobalSmartFolder object from Infoblox appliance through a session
my @retrieved_objs = $session->get(
    object => "Infoblox::Grid::GlobalSmartFolder",
    name   => "my_folder_global"
    );
my $object = $retrieved_objs[0];

unless ($object) {
  die("Get GlobalSmartFolder object failed: ",
      $session->status_code() . ":" . $session->status_detail());
}
print "Get GlobalSmartFolder object found at least 1 matching entry\n";

#Modify the GlobalSmartFolder object
$object->name("my_folder_global_renamed");
$object->comment("modified global smart folder");

#Apply the changes.
$session->modify($object)
  or die("Modify GlobalSmartFolder object failed: ",
      $session->status_code() . ":" . $session->status_detail());
print "GlobalSmartFolder object modified successfully \n";
```

#Remove a GlobalSmartFolder object

```
#Get the GlobalSmartFolder object through the session
my @retrieved_objs = $session->get(
    object => "Infoblox::Grid::GlobalSmartFolder",
    name   => "my_folder_global_renamed"
    );
my $object = $retrieved_objs[0];
unless ($object) {
  die("Get GlobalSmartFolder object failed: ",
      $session->status_code() . ":" . $session->status_detail());
}
print "Get GlobalSmartFolder object found at least 1 matching entry\n";

#Submit the object for removal
$session->remove($object)
  or die("Remove GlobalSmartFolder object failed: ",
      $session->status_code() . ":" . $session->status_detail());
print "GlobalSmartFolder object removed successfully \n";
```

#Cleanup

```
#Get the Network object through the session
my @retrieved_objs = $session->get(
    object  => "Infoblox::DHCP::Network",
    network => "10.0.0.0/24"
    );
my $object = $retrieved_objs[0];
unless ($object) {
  die("Get Network object failed: ",
      $session->status_code() . ":" . $session->status_detail());
}
print "Get Network object found at least 1 matching entry\n";

#Submit the object for removal
$session->remove($object)
  or die("Remove Network object failed: ",
      $session->status_code() . ":" . $session->status_detail());
print "Network object removed successfully \n";

####PROGRAM ENDS####
```

AUTHOR

Infoblox Inc. http://www.infoblox.com/

SEE ALSO

Infoblox::Grid::SmartFolderChildren, Infoblox::Grid::PersonalSmartFolder, Infoblox::Session->add(), Infoblox::Session->get(), Infoblox::Session->modify(), Infoblox::Session->remove(), Infoblox::Session->search(),Infoblox::Session

---

COPYRIGHT

Copyright (c) 2017 Infoblox Inc.

Infoblox::Grid::HostNameRewritePolicy - a structure for storing hostname rewrite policies.

- NAME
- DESCRIPTION
- CONSTRUCTOR
- MODULE METHODS
  - Infoblox::Grid::DHCP->protocol_hostname_rewrite_policies( )
- METHODS
  - is_grid_default( )
  - name( )
  - replacement_character( )
  - pre_defined( )
  - valid_characters( )
- AUTHOR
- SEE ALSO
- COPYRIGHT

NAME

Infoblox::Grid::HostNameRewritePolicy - a structure for storing hostname rewrite policies.

DESCRIPTION

An Infoblox::Grid::HostNameRewritePolicy object represents a hostname rewrite policy.

CONSTRUCTOR

```
my $policy = Infoblox::Grid::HostNameRewritePolicy->new(
     name                => $string,           # Required
     valid_characters    => $string,           # Required
     replacement_character => $string,         # Required
     is_grid_default     => "true" | "false",  # Optional / Default value is "false"
);
```

MODULE METHODS

The following functions can be applied to an Infoblox::Grid::HostNameRewritePolicy object.

Infoblox::Grid::DHCP->protocol_hostname_rewrite_policies( )

Use this function to add or retrieve hostname rewrite policies. See Infoblox::Grid::DHCP->protocol_hostname_rewrite_policies() for parameters and return values.

Example

```
#Get Grid DHCP object
my @retrieved_objs = $session->get(
       object => "Infoblox::Grid::DHCP",
       grid   => "Infoblox"
);
my $griddhcp = $retrieved_objs[0];

#Modify protocol_hostname_rewrite_policies
my $policies = $griddhcp->protocol_hostname_rewrite_policies();
push(@$policies, $policy);
$griddhcp->protocol_hostname_rewrite_policies($policies);
```

METHODS

This section describes all the methods that can be used to configure and retrieve the attribute values of an Infoblox::Grid::HostNameRewritePolicy object.

is_grid_default( )

Use this method to set or retrieve the grid's default hostname rewrite policy.

Parameter

Specify "true" to set a policy as the grid default hostname rewrite policy or "false" to disable it. Default value is "false".

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get is_grid_default
my $is_grid_default = $policy->is_grid_default();
#Modify is_grid_default
$policy->is_grid_default('true');
```

name( )

Use this method to set or retrieve the policy name.

Parameter

String with policy name.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get name
my $name = $policy->name();
#Modify name
$policy->name('new_policy');
```

replacement_character( )

Use this method to set or retrieve the replacement character.

Parameter

The replacement character.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get replacement_character
my $replacement_character = $policy->replacement_character();
#Modify name
$policy->replacement_character('-');
```

pre_defined( )

Use this method to retrieve a predefined hostname rewrite policy. This attribute is read-only.

Parameter

None.

Returns

The method returns the attribute value.

Example

```
#Get pre_defined
my $pre_defined = $policy->pre_defined();
```

valid_characters( )

Use this method to set or retrieve permitted characters for a hostname.

Parameter

String with permitted characters for a hostname.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get valid_characters
my $valid_characters = $policy->valid_characters();
#Modify valid_characters
$policy->valid_characters('A-Z0-9_');
```

AUTHOR

Infoblox Inc. http://www.infoblox.com/

SEE ALSO

Infoblox::Grid::DHCP, Infoblox::Grid::Member::DHCP

COPYRIGHT

Copyright (c) 2017 Infoblox Inc.

Infoblox::Grid::HSM::AllGroup - All HSM groups.

- NAME
- DESCRIPTION
- CONSTRUCTOR
- SESSION METHODS
  - Infoblox::Session->get( )
- METHODS
  - groups( )
- AUTHOR
- SEE ALSO
- COPYRIGHT

NAME

Infoblox::Grid::HSM::AllGroup - All HSM groups.

DESCRIPTION

This is a read-only object which allows reading of all hsm groups.

CONSTRUCTOR

The Infoblox::Grid::HSM::AllGroup object is a read-only object and does not require manual construction.

SESSION METHODS

This section describes all the methods in an Infoblox::Session module that you can apply to the Infoblox::Grid::HSM::AllGroup object.

Infoblox::Session->get( )

Use this method to retrieve all the matching objects from the Infoblox appliance. See Infoblox::Session->get() for parameters and return values.

Example

```
my $allgroup = $session->get(object => "Infoblox::Grid::HSM::AllGroup");
```

METHODS

This section describes all the methods that you can use to retrieve the attribute values of the Infoblox::Grid::HSM::AllGroup object.

groups( )

Use this method to retrieve the list of HSM groups. This attribute is read-only and cannot be modified.

Parameter

None

Returns

The method returns the attribute value as an array reference that contains Infoblox::Grid::HSM::Thales::Group and Infoblox::Grid::HSM::SafeNet::Group object(s).

Example

```
#Get groups
my $groups = $allgroup->groups();
```

AUTHOR

Infoblox Inc. http://www.infoblox.com/

SEE ALSO

Infoblox::Grid::HSM::SafeNet::Group, Infoblox::Grid::HSM::Thales::Group

COPYRIGHT

Copyright (c) 2017 Infoblox Inc.

Infoblox::Grid::HSM::SafeNet - SafeNet HSM Server object

- NAME
- DESCRIPTION
- CONSTRUCTOR
- MODULE METHODS
  - Infoblox::Grid::HSM::SafeNet::Group->hsm_servers( )
- METHODS
  - disabled( )
  - is_fips_compliant( )
  - name( )
  - partition_capacity( )
  - partition_id( )
  - partition_serial( )
  - server_certificate( )
  - status( )
- AUTHOR
- SEE ALSO
- COPYRIGHT

NAME

Infoblox::Grid::HSM::SafeNet - SafeNet HSM (Hardware Security Module) Server object

DESCRIPTION

An Infoblox::Grid::HSM::SafeNet object represents a server configured in the SafeNet HSM group.

CONSTRUCTOR

```
my $hsm_server = Infoblox::Grid::HSM::SafeNet->new(
    name                => $string,                 # Required
    partition_serial    => $uint,                   # Required
    server_certificate  => $string,                 # Required
    disabled            => "true" | "false",        # Optional / Default value is "false"
);
```

MODULE METHODS

The following functions can be applied to a SafeNet HSM Server object.

Infoblox::Grid::HSM::SafeNet::Group->hsm_servers( )

Use this function to add or retrieve SafeNet HSM servers in the SafeNet HSM group. See Infoblox::Grid::HSM::SafeNet::Group->hsm_servers() for parameters and return values.

Example

```
#Construct a SafeNet HSM Group
my $hsm_group = Infoblox::Grid::HSM::SafeNet::Group->new(
    name          => 'SafeNetGroup',
    passphrase    => 'PassPhrase',
    hsm_servers   => [],
    hsm_version   => "LunaSA_4",
);

# Configure the SafeNet HSM server in the SafeNet HSM Group object.
my $response = $hsm_group->hsm_servers([$hsm_server]);
```

METHODS

This section describes all the methods that can be used to configure and retrieve the attribute values of an Infoblox::Grid::HSM::SafeNet object.

disabled( )

Use this method to set or retrieve the disabled setting.

Parameter

Specify "true" to disable the SafeNet HSM Server object or "false" to enable it. Default value is "false".

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value as "true" or "false".

Example

```
#Get disabled flag
my $disabled = $hsm_server->disabled();

#Modify disabled flag
$hsm_server->disabled("true");
```

is_fips_compliant( )

Use this method to retrieve the FIPS compliant mode status. This attribute is read-only and cannot be modified.

Parameter

None

Returns

The method returns the attribute value as "true" or "false".

Example

```
#Get is_fips_compliant flag
my $is_fips_compliant = $hsm_server->is_fips_compliant();
```

name( )

Use this method to set or retrieve the IP address or the domain name for the HSM.

Parameter

String with the IP address or the domain name for the HSM.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get name
my $name = $hsm_server->name();

#Modify name
$hsm_server->name("SafeNet");
```

partition_capacity( )

Use this method to retrieve the partition capacity percentage used. This attribute is read-only and cannot be modified.

Parameter

None

Returns

The method returns the attribute value.

Example

```
#Get partition_capacity
my $partition_capacity = $hsm_server->partition_capacity();
```

partition_id( )

Use this method to retrieve the partition label on the HSM. This attribute is read-only and cannot be modified.

Parameter

None

Returns

The method returns the attribute value.

Example

```
#Get partition_id
my $partition_id = $hsm_server->partition_id();
```

partition_serial( )

Use this method to set or retrieve the serial number of the partition on the HSM.

Parameter

An unsigned integer.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get partition_serial
my $partition_serial = $hsm_server->partition_serial();

#Modify partition_serial
$hsm_server->partition_serial(12345);
```

server_certificate( )

Use this method to generate the server certificate of the SafeNet HSM server. This attribute is write-only and cannot be retrieved.

Parameter

String with the content of the server certificate or a reference to a file.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

Example

```
#Set server_certificate as string
$hsm_server->server_certificate($cert);

#Set server_certificate as reference to a file
$hsm_server->server_certificate('/tmp/safenet/cert.pem');
```

status( )

Use this method to retrieve the status of the SafeNet HSM server. This attribute is read-only and cannot be modified.

Parameter

None

Returns

The method returns the attribute value as "UP" or "DOWN".

Example

```
#Get status
my $status = $hsm_server->status();
```

AUTHOR

Infoblox Inc. http://www.infoblox.com/

SEE ALSO

Infoblox::Grid::HSM::SafeNet::Group

COPYRIGHT

Copyright (c) 2017 Infoblox Inc.

Infoblox::Grid::HSM::SafeNet::Group - SafeNet HSM Group object

- NAME
- DESCRIPTION
- CONSTRUCTOR
- SESSION METHODS
  - Infoblox::Session->add( )
  - Infoblox::Session->get( )
  - Infoblox::Session->modify( )
  - Infoblox::Session->remove( )
  - Infoblox::Session->search( )
- METHODS
  - comment( )
  - name( )
  - passphrase( )
  - hsm_servers( )
  - hsm_version( )
  - serial_number( )
  - status( )
- SAMPLE CODE
- AUTHOR
- SEE ALSO
- COPYRIGHT

NAME

Infoblox::Grid::HSM::SafeNet::Group - SafeNet HSM (Hardware Security Module) Group object

---

DESCRIPTION

Hardware Security Modules (HSMs) are used for secure private key storage, and key generation and zone-signing offloading.

CONSTRUCTOR

```
 my $hsm_group = Infoblox::Grid::HSM::SafeNet::Group->new(
     name                    => $string,                                  #Required
     passphrase              => $string,                                  #Required
     hsm_servers             => [$safenet1, ...],                         #Required
     hsm_version             => "LunaSA_4" | "LunaSA_5" | "LunaSA_6",     #Required
     comment                 => $string,                                  #Optional / Default
value is empty
 );
```

SESSION METHODS

This section describes all the methods in an Infoblox::Session module that you can apply to a SafeNet HSM Group object.

Infoblox::Session->add( )

Use this method to add an object to the Infoblox appliance. See Infoblox::Session->add() for parameters and return values.

Example

```
#Construct an object
my $hsm_group = Infoblox::Grid::HSM::SafeNet::Group->new(
    name                    => "SafeNetGroup",
    passphrase              => "PassPhrase",
    comment                 => "This is a test SafeNet HSM Group",
    hsm_servers             => [$safenet1],
    hsm_version             => "LunaSA_4",
    );

#Submit for addition
my $response = $session->add($hsm_group);
```

Infoblox::Session->get( )

Use this method to retrieve all the matching objects from the Infoblox appliance. See Infoblox::Session->get() for parameters and return values.

Example

```
my @retrieved_objs = $session->get(
    object => "Infoblox::Grid::HSM::SafeNet::Group",
    name   => "SafeNetGroup");
```

Infoblox::Session->modify( )

Use this method to modify an object in the Infoblox appliance. See Infoblox::Session->modify() for parameters and return values.

Example

```
# Use this method to modify the comment of a SafeNet HSM Group object.
$hsm_group->comment("This is modified SafeNet HSM Group object");

# Submit modification
my $response = $session->modify($hsm_group);
```

Infoblox::Session->remove( )

Use this method to remove an object from the Infoblox appliance. See Infoblox::Session->remove() for parameters and return values.

To remove a specific object, first use `get()` to retrieve the object, and then submit it for removal.

Example

```
#Get the objects with the specified name.
my @retrieved_objs = $session->get(
    object => "Infoblox::Grid::HSM::SafeNet::Group",
    name   => "SafeNetGroup");

#Find the desired object on the retrieved list.
my $desired_hsmgroup = $retrieved_objs[0];

# Submit for removal
my $response = $session->remove($desired_hsmgroup);
```

Infoblox::Session->search( )

Use this method to retrieve all the matching objects from the Infoblox appliance. See Infoblox::Session->search() for parameters and return values.

Example

```
my @retrieved_objs = $session->search(
    object => "Infoblox::Grid::HSM::SafeNet::Group",
    name   => "SafeNetGroup");
```

METHODS

This section describes all the methods that you can use to configure and retrieve the attribute values of an HSM Group object.

comment( )

Use this method to set or retrieve a descriptive comment.

Parameter

Desired comment in string format with a maximum of 256 bytes.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get comment
my $comment = $hsm_group->comment();

#Modify comment
$hsm_group->comment("Modified comment");
```

name( )

Use this method to set or retrieve the name of the SafeNet HSM Group object.

Parameter

Desired name for the SafeNet HSM Group object in string format.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get name
my $name = $hsm_group->name();

#Modify name
$hsm_group->name("SafeNetGroup2")
```

passphrase( )

Use this method to set the passphrase that is used to unlock the key store. This is a write-only attribute.

Parameter

The valid value is a string, which cannot include a tab and carriage return.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

Example

```
#Modify passphrase
$hsm_group->passphrase("pass1234");
```

hsm_servers( )

Use this method to set or retrieve the list of SafeNet servers in a SafeNet HSM Group object.

Parameter

The valid value is an array reference that contains Infoblox::Grid::HSM::SafeNet object(s).

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get SafeNet HSM servers list
my $hsm_servers = $hsm_group->hsm_servers();

#Modify SafeNet HSM servers list
my $safenet_server1 = Infoblox::Grid::HSM::SafeNet->new(
   name               => 'server1',
   partition_serial   => 123,
   server_certificate => '/tmp/safenet/cert1.pem',
);
my $safenet_server2 = Infoblox::Grid::HSM::SafeNet->new(
   name               => 'server2',
   partition_serial   => 456,
   server_certificate => '/tmp/safenet/cert2.pem',
);

$hsm_group->hsm_servers([$safenet_server1,$safenet_server2]);
```

hsm_version( )

Use this method to set or retrieve the HSM version of the SafeNet HSM Group.

Parameter

The value can be either "LunaSA_4", "LunaSA_5" or "LunaSA_6".

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value of "LunaSA_4", "LunaSA_5" or "LunaSA_6".

Example

```
#Get hsm_version
my $hsm_version = $hsm_group->hsm_version();

#Modify hsm_version
$hsm_version->hsm_version("LunaSA_4");
```

serial_number( )

Use this method to retrieve the serial number of the SafeNet HSM Group. This attribute is read-only and cannot be modified.

Parameter

None

Returns

The method returns the attribute value.

Example

```
#Get serial_number
my $serial_number = $hsm_group->serial_number();
```

status( )

Use this method to retrieve the status of the SafeNet HSM Group. This attribute is read-only and cannot be modified.

Parameter

None

Returns

The method returns the attribute value of "UP" or "DOWN".

Example

```
#Get status
my $status = $hsm_group->status();
```

SAMPLE CODE

The following sample code demonstrates the different functions that can be applied to an object such as add, get, modify and remove. This sample also includes error handling for the operations.

#Preparation prior to getting and modifying a grid member object

```
use strict;
use Infoblox;

#refers to Infoblox Appliance IP address
my $host_ip = "192.168.1.2";

#Create a session to the Infoblox appliance

my $session = Infoblox::Session->new(
    master   => $host_ip,
    username => "admin",
    password => "infoblox"
);
unless ($session) {
       die("Construct session failed: ",
               Infoblox::status_code() . ":" . Infoblox::status_detail());
}
print "Session created successfully\n";
```

#Create a SafeNet HSM Group object

```
my $safenet_server = Infoblox::Grid::HSM::SafeNet->new(
```

```
    name                => "20.0.0.1",
    partition_serial    => 1234,
    server_certificate  => '/tmp/safenet/cert.pem',
    );
unless (safenet_server) {
       die("Construct safenet server object failed: ",
               Infoblox::status_code() . ":" . Infoblox::status_detail());
}
print "SafeNet server object created successfully\n";

my $hsm_group = Infoblox::Grid::HSM::SafeNet::Group->new(
    name         => "SafeNet Group",
    passphrase   => 'pass',
    hsm_servers  => [safenet_server],
    hsm_version  => "LunaSA_4",
    );
unless ($hsm_group) {
       die("Construct SafeNet HSM Group object failed: ",
               Infoblox::status_code() . ":" . Infoblox::status_detail());
}
print "SafeNet HSM Group object created successfully\n";

#Add the SafeNet HSM object to appliance through session
my $response = $session->add($hsm_group);

unless ($response) {
    die("Add SafeNet HSM Group failed: ",
           $session->status_code() . ":" . $session->status_detail());
}
print "SafeNet HSM Group added successfully \n";
```

#Get and modify the SafeNet HSM Group object

```
#Get SafeNet HSM Group through session
my @retrieved_objs = $session->get(
    object   => "Infoblox::Grid::HSM::SafeNet::Group",
    name     => "SafeNet Group",
    );
my $hsm_group = $retrieved_objs[0];

unless ($hsm_group) {
    die("Get SafeNet HSM Group object failed: ",
           $session->status_code() . ":" . $session->status_detail());
}
print"Get SafeNet HSM Group successful \n";

# Modify one of the attributes of the selected SafeNet HSM Group.

$hsm group->comment("Modified hsm group object");

#Applying the changes
$session->modify($hsm_group)
   or die("Modify SafeNet HSM Group failed",
            $session->status_code() . ":" . $session->status_detail());
print "SafeNet HSM Group modified successfully \n";
```

#Remove the grid member object

```
#Get SafeNet HSM Group through session
my @retrieved_objs = $session->get(
    object => "Infoblox::Grid::HSM::SafeNet::Group",
    name   => "SafeNet Group",
    );
my $desired_hsm_group = $retrieved_objs[0];

unless ($desired_hsm_group) {
    die("Get SafeNet HSM Group object failed: ",
           $session->status_code() . ":" . $session->status_detail());
}
print "Get SafeNet HSM Group successful \n";

#Submit the object for removal
$session->remove($desired_hsm_group)
    or die("Remove SafeNet HSM Group failed",
           $session->status_code() . ":" . $session->status_detail());
print "SafeNet HSM Group removed successfully \n";
```

```
####PROGRAM ENDS####
```

AUTHOR

Infoblox Inc. http://www.infoblox.com/

SEE ALSO

Infoblox::Grid::HSM::SafeNet

COPYRIGHT

Copyright (c) 2017 Infoblox Inc.

Infoblox::Grid::HSM::Thales - Thales HSM Server object

- NAME
- DESCRIPTION
- CONSTRUCTOR
- MODULE METHODS
  - Infoblox::Grid::HSM::Thales::Group->thales_hsm( )
- METHODS
  - disabled( )
  - keyhash( )
  - remote_ip( )
  - remote_port( )
  - remote_esn( )
  - status( )
- AUTHOR
- SEE ALSO
- COPYRIGHT

NAME

Infoblox::Grid::HSM::Thales - Thales HSM (Hardware Security Module) Server object

---

DESCRIPTION

An Infoblox::Grid::HSM::Thales object represents a server configured in the HSM group.

CONSTRUCTOR

```
my $hsm_server = Infoblox::Grid::HSM::Thales->new(
    remote_ip       => $ipv4addr,                   # Required
    keyhash         => $string,                     # Required
    disabled        => "true" | "false",            # Optional / Default value is "false"
    remote_port     => $num,                        # Optional / Default value is 9004
);
```

MODULE METHODS

The following functions can be applied to an HSM Server object.

Infoblox::Grid::HSM::Thales::Group->thales_hsm( )

Use this function to add or retrieve HSM servers in the HSM group. See Infoblox::Grid::HSM::Thales::Group->thales_hsm() for parameters and return values.

Example

```
#Construct an HSM Group
my $hsm_group = Infoblox::Grid::HSM::Thales::Group->new(
    name                    => "HSM Group",
    key_server_ip           => "10.0.0.1",
    key_server_port         => 9004,
    comment                 => "A new HSM Group ",
    thales_hsm              => []
);

# Configure the HSM server in the HSM Group object.
my $response = $hsm_group->thales_hsm([$thales_server]);
```

METHODS

This section describes all the methods that can be used to configure and retrieve the attribute values of an Infoblox::Grid::HSM::Thales object.

disabled( )

Use this method to set or retrieve the disabled setting.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

Specify "true" to disable the HSM Server object or "false" to enable it. Default value is "false".

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get disabled flag
my $disabled = $thales_server->disabled();
#Modify disabled flag
$thales_server->disabled("true");
```

keyhash( )

Use this method to set or retrieve the key hash used to authenticate the HSM.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

Desired key hash in string format.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get key hash
my $key = $thales_server->keyhash();
#Modify key hash
$thales_server->keyhash("123hashxyz");
```

remote_ip( )

Use this method to set or retrieve the IP address of the Thales HSM server.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

The IPv4 address of the Thales HSM server.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get the Thales HSM server IP address
my $remote_ip = $thales_server->remote_ip();
#Modify the Thales HSM server IP address
$thales_server->remote_ip("20.0.0.9");
```

remote_port( )

Use this method to set or retrieve the remote port setting of the Thales HSM server.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

Valid values are between 1 and 65535.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get remote port
my $remote_port = $thales_server->remote_port();
#Modify remote port
$thales_server->remote_port(9012);
```

remote_esn( )

Use this method to retrieve the Electronic Serial Number (ESN) of the Thales HSM server.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Returns

The method returns the attribute value.

Example

```
#Get remote ESN
my $remote_esn= $thales_server->remote_esn();
```

status( )

Use this method to retrieve the status of the Thales HSM server.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Returns

The method returns the attribute value as "UP" or "DOWN".

Example

```
#Get status
my $status= $thales_server->status();
```

AUTHOR

Infoblox Inc. http://www.infoblox.com/

SEE ALSO

Infoblox::Grid::HSM::Thales::Group

COPYRIGHT

Copyright (c) 2017 Infoblox Inc.

Infoblox::Grid::HSM::Thales::Group - Thales HSM Group object

- NAME
- DESCRIPTION
- CONSTRUCTOR
- SESSION METHODS
  - Infoblox::Session->add( )
  - Infoblox::Session->get( )
  - Infoblox::Session->modify( )
  - Infoblox::Session->remove( )
- METHODS
  - card_name( )
  - comment( )
  - key_server_ip( )
  - key_server_port( )
  - name( )
  - pass_phrase( )
  - protection( )
  - thales_hsm( )
  - status( )
- SAMPLE CODE
- AUTHOR
- SEE ALSO
- COPYRIGHT

NAME

Infoblox::Grid::HSM::Thales::Group - Thales HSM Group object

DESCRIPTION

Hardware Security Modules (HSMs) are used for secure private key storage, and key generation and zone-signing offloading.

CONSTRUCTOR

```
 my $hsm_group = Infoblox::Grid::HSM::Thales::Group->new(
     key_server_ip           => $ipv4addr,                                   #Required
     name                    => $string,                                     #Required
     thales_hsm              => [$thales1, ...],                             #Required
     card_name               => $string,                                     #Required if
"protection" is "SOFTCARD"
     pass_phrase             => $string,                                     #Required if
"protection" is "SOFTCARD"
     comment                 => $string,                                     #Optional / Default
value is empty
     key_server_port         => $num,                                        #Optional / Default
value is 9004
     protection              => "MODULE" | "SOFTCARD" ,                      #Optional / Default
value is Module
 );
```

SESSION METHODS

This section describes all the methods in an Infoblox::Session module that you can apply to an HSM Group object.

Infoblox::Session->add( )

Use this method to add an object to the Infoblox appliance. See Infoblox::Session->add() for parameters and return values.

Example

```
#Construct an object
my $hsm_group = Infoblox::Grid::HSM::Thales::Group->new(
    name                    => "HSM Group",
    key_server_ip           => "10.0.0.1",
    key_server_port         => 9004,
    comment                 => "A new HSM Group ",
    thales_hsm              => [$thales1]
    );

#Submit for addition
```

```
my $response = $session->add($hsm_group);
```

Infoblox::Session->get( )

Use this method to retrieve all the matching objects from the Infoblox appliance. See Infoblox::Session->get() for parameters and return values.

Example

```
my @retrieved_objs = $session->get(
    object => "Infoblox::Grid::HSM::Thales::Group",
    key_server_ip   => "10.0.0.1");
```

Infoblox::Session->modify( )

Use this method to modify an object in the Infoblox appliance. See Infoblox::Session->modify() for parameters and return values.

Example

```
# Use this method to modify the comment of an HSM Group object.
$hsm_group->comment("This is modified HSM Group object");
# Submit modification
my $response = $session->modify($hsm_group );
```

Infoblox::Session->remove( )

Use this method to remove an object from the Infoblox appliance. See Infoblox::Session->remove() for parameters and return values.

To remove a specific object, first use `get()` to retrieve the object, and then submit it for removal.

Example

```
#Get the objects with the specified key_server_ip.
my @retrieved_objs = $session->get(
    object => "Infoblox::Grid::HSM::Thales::Group",
    key_server_ip   => "10.0.0.1");

#Find the desired object on the retrieved list.
my $desired_hsmgroup = $retrieved_objs[0];

# Submit for removal
my $response = $session->remove($desired_hsmgroup);
```

METHODS

This section describes all the methods that you can use to configure and retrieve the attribute values of an HSM Group object.

card_name( )

Use this method to set or retrieve the name of the softcard used by the HSM Group object.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

Desired name for the softcard in string format.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get softcard name
my $card_name = $hsm_group->card_name();
#Modify softcard name
$hsm_group->card_name("Softcard 1")
```

comment( )

Use this method to set or retrieve a descriptive comment.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

Desired comment in string format with a maximum of 256 bytes.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get comment
my $comment = $hsm_group->comment();
#Modify comment
$hsm_group->comment("Modified HSM Group");
```

key_server_ip( )

Use this method to set or retrieve the key file server IP address of an HSM Group object.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

The IPv4 address of the key file server.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get key server IP address
my $key_server_ip = $hsm_group->key_server_ip();
#Modify key server IP address
$hsm_group->key_server_ip("10.0.0.5");
```

key_server_port( )

Use this method to set or retrieve the port number used by the key file server configured in the HSM Group object.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

Valid value is an unsigned integer ranging from 0 to 65535.The default value is 9004.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get key server port
my $key_server_port = $hsm_group->key_server_port();
#Modify key server port
$hsm_group->key_server_port(9012);
```

name( )

Use this method to set or retrieve the name of the HSM Group object.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

Desired name for the HSM Group object in string format.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get name
my $name = $hsm_group->name();
#Modify name
$hsm_group->name("HSM GROUP 2")
```

pass_phrase( )

Use this method to set the passphrase that is used to unlock the key store. This is a write-only attribute.

Include the specified parameter to set the attribute value.

Parameter

The valid value is a string, which cannot include a tab and carriage return.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

Example

```
#Modify passphrase
$hsm_group->pass_phrase("pass1234");
```

protection( )

Use this method to set or retrieve the protection type of an HSM Group object.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

The valid values are "MODULE" and "SOFTCARD".The default value is "MODULE".

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get protection
my $protection = $hsm_group->protection();
#Modify protection
$grid_member->protection("SOFTCARD");
```

thales_hsm( )

Use this method to set or retrieve the list of Thales servers in an HSM Group object.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

The valid value is an array reference that contains Infoblox::Grid::HSM::Thales object(s).

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get Thales HSM list
my $thales_hsm = $hsm_group->thales_hsm();
#Modify Thales HSM list
my $thales_hsm1 = Infoblox::Grid::HSM::Thales->new(
    remote_ip              => "20.0.0.1",
    keyhash                => "a123",
);

my $thales_hsm2 = Infoblox::Grid::HSM::Thales->new(
    remote_ip              => "20.0.0.5",
    keyhash                => "b456",
```

```
);

$hsm_group->thales_server([$thales_hsm1,$thales_hsm2]);
```

status( )

Use this method to retrieve the status of the remote file server used in the HSM Group object.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Returns

The method returns the attribute value of "UP" or "DOWN" .

Example

```
#Get status
my $status = $hsm_group->status();
```

SAMPLE CODE

The following sample code demonstrates the different functions that can be applied to an object such as add, get, modify and remove. This sample also includes error handling for the operations.

#Preparation prior to getting and modifying a grid member object

```
use strict;
use Infoblox;

#refers to Infoblox Appliance IP address
my $host_ip = "192.168.1.2";

#Create a session to the Infoblox appliance

my $session = Infoblox::Session->new(
    master   => $host_ip,
    username => "admin",
    password => "infoblox"
);
unless ($session) {
       die("Construct session failed: ",
               Infoblox::status_code() . ":" . Infoblox::status_detail());
}
print "Session created successfully\n";
```

#Create an HSM Group object

```
my $thales_server = Infoblox::Grid::HSM::Thales->new(
    remote_ip     => "20.0.0.1",
    keyhash       => "1234hash",
    remote_port   => 9012,
    );
unless ($thales_server) {
       die("Construct thales server object failed: ",
               Infoblox::status_code() . ":" . Infoblox::status_detail());
}
print "Thales server object created successfully\n";

my $hsm_group = Infoblox::Grid::HSM::Thales::Group->new(
    key_server_ip     => "20.0.0.1",
    key_server_port   => 9007,
    );
unless ($hsm_group) {
       die("Construct HSM Group object failed: ",
               Infoblox::status_code() . ":" . Infoblox::status_detail());
}
print "HSM Group object created successfully\n";

#Add the HSM object to appliance through session
my $response = $session->add( $hsm_group );

unless ($response) {
    die("Add HSM Group failed: ",
           $session->status_code() . ":" . $session->status_detail());
}
print"HSM Group added successfully \n";
```

#Get and modify the HSM Group object

```
#Get HSM Group through session
my @retrieved_objs = $session->get(
    object => "Infoblox::Grid::HSM::Thales::Group",
    key_server_ip     => "20.0.0.1",
    );
my $hsm_group = $retrieved_objs[0];

unless ($hsm_group) {
    die("Get HSM Group object failed: ",
           $session->status_code() . ":" . $session->status_detail());
}
print"Get HSM Group successful \n";

# Modify one of the attributes of the selected HSM Group.

$hsm group->comment("Modified hsm group object");

#Applying the changes
$session->modify($hsm_group)
   or die("Modify HSM Group failed",
            $session->status_code() . ":" . $session->status_detail());
print"HSM Group modified successfully \n";
```

#Remove the grid member object

```
#Get HSM Group through session
my @retrieved_objs = $session->get(
    object => "Infoblox::Grid::HSM::Thales::Group",
    key_server_ip     => "20.0.0.1"
    );
my $desired_hsm_group = $retrieved_objs[0];

unless ($desired_hsm_group) {
    die("Get HSM Group object failed: ",
           $session->status_code() . ":" . $session->status_detail());
}
print"Get HSM Group successful \n";

#Submit the object for removal
$session->remove( $desired_hsm_group )
    or die("Remove HSM Group failed",
           $session->status_code() . ":" . $session->status_detail());
print"HSM Group removed successfully \n";

####PROGRAM ENDS####
```

AUTHOR

Infoblox Inc. http://www.infoblox.com/

SEE ALSO

Infoblox::Grid::HSM::Thales

COPYRIGHT

Copyright (c) 2017 Infoblox Inc.

Infoblox::Grid::HTTPProxyServerSetting - An Infoblox Grid HTTP Proxy Server Setting object.

- NAME
- DESCRIPTION
- CONSTRUCTOR
- SESSION METHODS
- MODULE METHODS
  - Infoblox::Grid->http_proxy_server_setting( )
- METHODS
  - address( )
  - certificate( )
  - comment( )
  - enable_content_inspection( )
  - enable_username_and_password( )
  - enable_proxy( )
  - password( )
  - port( )
  - username( )
  - verify_cname( )
- AUTHOR
- SEE ALSO
- COPYRIGHT

NAME

Infoblox::Grid::HTTPProxyServerSetting - An Infoblox Grid HTTP Proxy Server Setting object.

DESCRIPTION

The Grid HTTP Proxy Server Setting object provides information about HTTP Proxy Server configuration.

CONSTRUCTOR

```
my $http_proxy_setting = Infoblox::Grid::HTTPProxyServerSetting->new(
    address                     => $fqdn | $ipv4addr | $ipv6addr, # Required
    port                        => $uint,                         # Required
    enable_username_and_password => 'true' | 'false',             # Optional / Default is 'false'
    enable_proxy                => 'true' | 'false',              # Optional / Default is 'false'
    enable_content_inspection   => 'true' | 'false',              # Optional / Default is 'false'
    verify_cname                => 'true' | 'false',              # Optional / Default is 'false'
    comment                     => $string,                       # Optional / Default is undefined
    username                    => $string,                       # Optional / Default is undefined
    password                    => $string,                       # Optional / Default is undefined
    certificate                 => $string,                       # Optional / Default is undefined
);
```

SESSION METHODS

The object does not support any session methods.

MODULE METHODS

The following functions are available to be applied to a HTTP Proxy Server object.

Infoblox::Grid->http_proxy_server_setting( )

Use this method to set or retreive the Grid HTTP proxy server settings. See Infoblox::Grid->http_proxy_server_setting() for parameters andreturn values.

Example

```
#Get http_proxy_server_setting value
my $http_proxy_server_setting = $grid->http_proxy_server_setting();

#Modify http_proxy_server_setting value
$grid->http_proxy_server_setting($http_proxy_setting);
```

METHODS

This section describes all the methods that you can use to configure and retrieve the attribute values of a HTTP Proxy Server Setting object.

address( )

Use this method to set or retrieve the address of the HTTP Proxy Server.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

The valid value is a string that contains Fully-Qualified Domain Name (FQDN), IPv4 address or IPv6 address.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get address value
my $address = $obj->address();

#Modify address value
$obj->address('10.0.0.1');
```

certificate( )

Use this method to set the path to a CA Certificate used in a content inspection by a HTTP Proxy Server. This is a write-only attribute.

Include the specified parameter to set the attribute value.

Parameter

The valid value is a string which contains a path to certificate.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

Example

```
#Modify certificateibute value
$obj->certificate('/path/to/cert.pem');
```

comment( )

Use this method to set or retrieve the descriptive comment for HTTP Proxy Server configuration.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

The valid value is a desired comment in a string format.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get comment value
my $comment = $obj->attr();

#Modify comment value
$obj->comment('desired comment');
```

enable_content_inspection( )

Use this method to set or retrieve the flag that indicates whether the HTTPS packets content inspection by HTTP Proxy Server is enabled or disabled.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

Specify 'true' to enable HTTPS packets content inspection or 'false' to disable it. The default value is 'false'.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get enable_content_inspection value
my $enable_content_inspection = $obj->enable_content_inspection();

#Modify enable_content_inspection value
$obj->enable_content_inspection('true');
```

enable_username_and_password( )

Use this method to set or retrieve the flag that indicates whether the use of username and password for the connectivity of HTTP proxy server is enabled or not.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

Specify 'true' to use username and password for testing the connectivity of HTTP proxy server or 'false' to disallow it. The default value is 'false'.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get enable_username_and_password value
my $enable_username_and_password = $obj->enable_username_and_password();

#Modify enable_username_and_password value
$obj->enable_username_and_password('true');
```

enable_proxy( )

Use this method to set or retrieve the flag that indicates whether HTTP Proxy Server is enabled or disabled.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

Specify 'true' to enable HTTP Proxy Server or 'false' to disable it. The default value is 'false'.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get enable_proxy value
my $enable_proxy = $obj->enable_proxy();

#Modify enable_proxy value
$obj->enable_proxy('true');
```

password( )

Use this method to set the HTTP Proxy password. This is a write-only attribute.

Include the specified parameter to set the attribute value.

Parameter

The valid value is a desired password in a string format.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

Example

```
#Modify password value
$obj->password('pass');
```

port( )

Use this method to set or retrieve the HTTP Proxy Server port it listens to.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

The valid value is an unsigned integer between 0 and 65535.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get port value
my $port = $obj->port();

#Modify port value
$obj->port(10000);
```

username( )

Use this method to set or retrieve the HTTP Proxy user name.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

The valid value is a desired user name in a string format.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get username value
my $username = $obj->username();

#Modify username value
$obj->username('user1');
```

verify_cname( )

Use this method to set or retrieve the flag which indicates whether the CNAME record query verification is enabled or not.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

Specify 'true' to enable CNAME record query verification or 'false' to disable it. The default value is 'false'.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get verify_cname value
my $verify_cname = $obj->verify_cname();

#Modify verify_cname value
$obj->verify_cname('true');
```

AUTHOR

Infoblox Inc. http://www.infoblox.com/

SEE ALSO

Infoblox::Grid, Infoblox::Grid->http_proxy_server_setting()

COPYRIGHT

Copyright (c) 2017 Infoblox Inc.

Infoblox::Grid::InformationalBannerSetting - The informational banner setting.

- NAME
- DESCRIPTION
- CONSTRUCTOR
- MODULE METHODS
  - Infoblox::Grid->informational_banner_setting( )
- METHODS
  - enable( )
  - color( )
  - message( )
- AUTHOR
- SEE ALSO
- COPYRIGHT

NAME

Infoblox::Grid::InformationalBannerSetting - The informational banner setting.

DESCRIPTION

The informational banner setting object that controls the banner displayed on the login page of Grid Manager.

CONSTRUCTOR

```
 my $color = "BLACK" | "BLUE" | "BROWN" | "CYAN" | "GREEN" | "MAGENTA" | "ORANGE" | "PURPLE" | "RED" |
"YELLOW";

 my $informational = Infoblox::Grid::InformationalBannerSetting->new(
     enable    => "true" | "false", #Mandatory
     color     => $color,           #Mandatory
     message   => $string,          #Optional / Default is empty
 );
```

MODULE METHODS

The following functions are available to apply to an informational banner setting object.

Infoblox::Grid->informational_banner_setting( )

Use this function to specify the informational banner settings for the Grid. See Infoblox::Grid->informational_banner_setting() for parameters and return values.

Example

```
      my $grid = $session->get(object => 'Infoblox::Grid');

      my $banner = Infoblox::Grid::InformationalBannerSetting->new(enable => 'true', color => 'BLUE',
    message => 'Informational message');
      $grid->informational_banner_setting($banner);
      my $response = $session->modify($grid);
      unless($response) {
           die("Modify grid failed: ",
                 session->status_code() . ":" . session->status_detail());
      }
      print "Grid banner setting modified successfully\n";
```

METHODS

This section describes all the methods that you can use to set and retrieve the attribute values of an informational banner setting object.

enable( )

Use this method to set the enable flag.

Parameter

Specify "true" to enable the banner or "false" to disable it.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get enable
my $enable = $banner->enable();
#Modify enable
$banner->enable("true");
```

color( )

Use this method to set the color of the message that will be displayed on the login page of Grid Manager.

Parameter

A string containing the color. Banner colors can be one of the following: 'BLACK', 'BLUE', 'BROWN', 'CYAN', 'GREEN', 'MAGENTA', 'ORANGE', 'PURPLE', 'RED', 'YELLOW'.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get color
my $color = $banner->color();
#Modify color
$banner->color("RED");
```

message( )

Use this method to set the message that will be displayed on the login page of Grid Manager.

Parameter

A string containing the message.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get message
my $message = $banner->message();
#Modify message
$banner->message("Test message");
```

AUTHOR

Infoblox Inc. http://www.infoblox.com/

SEE ALSO

Infoblox::Grid

COPYRIGHT

Copyright (c) 2017 Infoblox Inc.

Infoblox::Grid::IPValidationInfo - IP Validation information

- NAME
- DESCRIPTION
- CONSTRUCTOR
- MODULE METHODS
  - Infoblox::Session->add( ) and Infoblox::Session->modify( )
- METHODS
  - address( )
  - parent( )
- SAMPLE CODE
- AUTHOR
- SEE ALSO
- COPYRIGHT

NAME

Infoblox::Grid::IPValidationInfo - IP Validation information

DESCRIPTION

When submitting add or modify requests that contain IPs returned by the next_available_ip functionality, this object can be passed to ensure that these IPs are unique and that concurrent additions and modifications containing the same IPs will fail.

CONSTRUCTOR

```
my $member = Infoblox::Grid::IPValidationInfo->new(
    address        => $ipaddr,         #Required
    parent         => $obj,            #Required
);
```

MODULE METHODS

The following functions are available for an IP validation info object.

Infoblox::Session->add( ) and Infoblox::Session->modify( )

The object can be passed in the ip_validation option. For usage information, see the sample code section.

METHODS

This section describes all the methods that you can use to set and retrieve the attribute values of an IP validation info object.

address( )

Use this method to set or retrieve the address.

Parameter

The valid value is an IPv4 or IPv6 address.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get address
my $address = $vip->address();
#Modify address
$vip->address('10.0.0.1');
```

parent( )

Use this method to set the parent object for the validation. This is the object from which the next available IP was retrieved, and it must be an object retrieved from the server.

Parameter

The valid value is an Infoblox::DHCP::Range, Infoblox::DHCP::IPv6Range, Infoblox::DHCP::Network, Infoblox::DHCP::IPv6Network

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get parent
my $net = $member->vip();
#Modify vip
$member->vip($net2);
```

SAMPLE CODE

The following sample code demonstrates the different functions that can be applied to an object, such as add, search, modify, and remove. This sample also includes error handling for the operations.

#Preparation prior to the insertion

```
#PROGRAM STARTS: Include all the modules that will be used
use strict;
use Infoblox;

#Create a session to the Infoblox appliance

my $host_ip   = "192.168.1.2";
my $host_name = "infoblox.localdomain";

my $session = Infoblox::Session->new(
              master   => $host_ip,      #appliance host ip
              username => "admin",       #appliance user login
              password => "infoblox"     #appliance password
);

unless ($session) {
       die("Construct session failed: ",
              Infoblox::status_code() . ":" . Infoblox::status_detail());
}
print "Session created successfully\n";
```

#Create a DNS Zone object

```
my $default_view = Infoblox::DNS::View->new(name => "default");

my $memberns1 = Infoblox::DNS::Member->new(
    name     => "infoblox.localdomain",
    ipv4addr => $host_ip,
    lead     => "false",
    stealth  => "false"
);

#To add zone test.com in the default view
my $firstzone = Infoblox::DNS::Zone->new(
    name         => "test.com",
    views        => [$default_view],
    email        => "admin\@infoblox.com",
    comment      => "add a zone test.com",
    primary      => $memberns1,
    );

unless($firstzone){
       die("Construct test.com zone object failed: ",
              Infoblox::status_code(). ":" .Infoblox::status_detail());
       }
print "test.com zone object created successfully.\n";

$session->add($firstzone)
    or die("Add zone for test.com failed: ",
              $session->status_code(). ":" .$session->status_detail());

print"Zone test.com added successfully.\n";
```

#Create a DHCP Network

```
my $memberdhcp = Infoblox::DHCP::Member->new(
            name     => "infoblox.localdomain",
            ipv4addr => $host_ip,
           );

my  $network = Infoblox::DHCP::Network->new(
            network => "10.0.0.0/8",
            comment => "add network",
            members => [$memberdhcp],
             );
unless($network){
       die("Construct test.com zone object failed: ",
               Infoblox::status_code(). ":" .Infoblox::status_detail());
       }
print "Network created successfully\n";

my $response = $session->add($network)
       or die("Add Network failed: ",
             $session->status_code() . ":" . $session->status_detail());

print "Network added successfully\n";
```

#Fetch a next available IP from the network

```
my $nip = $network->next_available_ip();
```

#Create a validation object for this IP

```
my $vip = Infoblox::Grid::IPValidationInfo->new(
   parent  => $network,
   address => $nip,
);
```

#Add a DNS A record with this IP and validation

```
#Construct a DNS A object
my $bind_a = Infoblox::DNS::Record::A->new(
    name     => "bind_a.test.com",
    comment  => " this is a demo bind_a record ",
    ipv4addr => $nip,
);
unless ($bind_a) {
   die("Construct DNS record A failed: ",
       Infoblox::status_code() . ":" . Infoblox::status_detail());
}
print "DNS A object created successfully\n";

#Add the DNS A record object to Infoblox Appliance through a session
$session->add($bind_a, ip_validation => [$vip])
    or die("Add record A failed: ",
            $session->status_code() . ":" . $session->status_detail());
print "DNS A object added to server successfully\n";
```

#Cleanup

```
$session->remove($bind_a)
    or die("Remove record A failed: ",
        $session->status_code() . ":" . $session->status_detail());

$session->remove($network)
    or die("Remove network failed: ",
        $session->status_code() . ":" . $session->status_detail());

$session->remove($firstzone)
    or die("Remove zone failed: ",
        $session->status_code() . ":" . $session->status_detail());

####PROGRAM ENDS####
```

AUTHOR

Infoblox Inc. http://www.infoblox.com/

SEE ALSO

Infoblox::Session

COPYRIGHT

Copyright (c) 2017 Infoblox Inc.

Infoblox::Grid::KerberosKey - Grid kerberos key object.

- NAME
- DESCRIPTION
- CONSTRUCTOR
- MODULE METHODS
  - Infoblox::Grid::DNS->gss_tsig_keys( )
  - Infoblox::Grid::DHCP->gss_tsig_keys( )
  - Infoblox::Grid::Member::DNS->gss_tsig_keys( )
  - Infoblox::Grid::Member::DHCP->gss_tsig_keys( )
- SESSION METHODS
  - Infoblox::Session->get( )
  - Infoblox::Session->remove( )
  - Infoblox::Session->search( )
- METHODS
  - enctype( )
  - domain( )
  - in_use( )
  - principal( )
  - version( )
  - upload_timestamp( )
- SAMPLE CODE
- AUTHOR
- SEE ALSO
- COPYRIGHT

NAME

Infoblox::Grid::KerberosKey - Grid kerberos key object.

DESCRIPTION

This object is used to manage kerberos keys for GSS-TSIG.

CONSTRUCTOR

```
#Constructor for an Grid KerberosKey object
 my $key = Infoblox::Grid::KerberosKey->new(
    principal => $string,  #Required.
    domain    => $string,  #Optional / Default is undefined
    version   => $num,     #Optional / Default is undefined
  );
```

MODULE METHODS

The following functions are available to be applied to an KerberosKey object.

Infoblox::Grid::DNS->gss_tsig_keys( )

Use this function to specify a Kerberos key at the grid level on the Infoblox appliance. See Infoblox::Grid::DNS->gss_tsig_keys() for parameters and return values.

Example

```
#Create a Kerberos object.
my $key = Infoblox::Grid::KerberosKey->new(
                  principal => 'host/appliance.ivan.test@LADYBUG.LOCAL',
                  domain => 'JACKIE.LOCAL',
                  version => '3',
);

#Configure kerberos key on the Infoblox grid object
my $response = $grid->gss_tsig_keys([$key]);
```

Infoblox::Grid::DHCP->gss_tsig_keys( )

Use this function to specify a Kerberos key at the grid level on the Infoblox appliance. See Infoblox::Grid::DHCP->gss_tsig_keys() for parameters and return values.

Example

```
#Create a Kerberos object.
my $key = Infoblox::Grid::KerberosKey->new(
                principal => 'host/appliance.ivan.test@LADYBUG.LOCAL',
                domain => 'JACKIE.LOCAL',
                version => '3',
              );

#Configure kerberos key on the Infoblox grid object
my $response = $grid->gss_tsig_keys([$key]);
```

Infoblox::Grid::Member::DNS->gss_tsig_keys( )

Use this function to specify a Kerberos key at the member level on the Infoblox appliance. See Infoblox::Grid::Member::DNS->gss_tsig_keys() for parameters and return values.

Example

```
#Create a KerberosKey object.
my $key = Infoblox::Grid::KerberosKey->new(
                principal => 'host/appliance.ivan.test@LADYBUG.LOCAL',
                domain => 'JACKIE.LOCAL',
                version => '3',
);

#Configure KerberosKey on the Infoblox member object
my $response = $member->gss_tsig_keys([$key]);
```

Infoblox::Grid::Member::DHCP->gss_tsig_keys( )

Use this function to specify a Kerberos key at the member level on the Infoblox appliance. See Infoblox::Grid::Member::DHCP->gss_tsig_keys() for parameters and return values.

Example

```
 #Create a KerberosKey object.
  my $key = Infoblox::Grid::KerberosKey->new(
                principal => 'host/appliance.ivan.test@LADYBUG.LOCAL',
                domain => 'JACKIE.LOCAL',
                version => '3',
              );

#Configure KerberosKey on the Infoblox member object
my $response = $member->gss_tsig_keys([$key]);
```

SESSION METHODS

This section describes all the methods in an Infoblox::Session module that you can apply to a Kerberos key object.

Infoblox::Session->get( )

Use this method to retrieve all the matching objects from the Infoblox appliance. See Infoblox::Session->get() for parameters and return values.

Key References

```
Apply the following attributes to get a specific Kerberos key object:

 principal - Required. The principal of the Kerberos key object you want to retrieve.
```

Example

```
my @retrieved_objs = $session->get(
    object    => "Infoblox::Grid::KerberosKey",
    principal => "my-principal");
```

Infoblox::Session->remove( )

Use this method to remove a Kerberos key object from the Infoblox appliance. See Infoblox::Session->remove() for parameters and return values.

To remove a specific object, first use `get()` or `search()` to retrieve the specific Kerberos key object, and then submit this object for removal.

Example

```
# Get the objects with the same principal
my @retrieved_objs = $session->get(
    object    => "Infoblox::Grid::KerberosKey",
    principal => "my-principal");
# Find the desired object from the retrieved list.
my $desired_kerberos_key = $retrieved_objs[0];
# Submit for removal
my $response = $session->remove( $desired_kerberos_key );
```

Infoblox::Session->search( )

Use this method to search for Kerberos key objects in the Infoblox appliance. See Infoblox::Session->search() for parameters and return values.

Key References

```
Apply the following attributes to search for a Kerberos key object:

 principal - Required. The principal of the Kerberos key object you want to retrieve (regular
expression is supported).
```

Example

```
# search for all grid member objects
my @retrieved_objs = $session->search(
    object => "Infoblox::Grid::KerberosKey",
    principal => ".*");
```

METHODS

This section describes all the methods that you can used to configure and retrieve the attribute value of a KerberosKey object.

enctype( )

Use this method to retrieve the encryption type for the KerberosKey object. This is a read-only attribute.

Parameter

None

Returns

The method returns the attribute value.

Example

```
#Get enctype value
my $enctype = $key->enctype();
```

domain( )

Use this method to set or retrieve the domain of the KerberosKey object.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

Desired domain in string format with a maximum of 1023 bytes.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get domain
my $domain = $key->domain();
#Modify domain
$key->domain("my-domain");
```

in_use( )

Use this method to retrieve the flag used to indicate whether the key has been assigned to a member. This is a read-only attribute.

Parameter

None

Returns

The return value is "true" if KerberosKey object has been assigned to a member and "false" if it hasn't.

Example

```
#Get in_use value
my $in_use = $key->in_use();
```

principal( )

Use this method to set or retrieve the principal string of the KerberosKey object.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

Desired principal in string format with a maximum of 1023 bytes.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get principal
my $string = $key->principal();
#Modify principal
$key->principal("my-principal");
```

version( )

Use this method to set or retrieve the version number of the KerberosKey object.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

Enter an unsigned integer between 0 and 4294967295.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get version
my $version = $key->version();
#Modify version
$key->version("5");
```

upload_timestamp( )

Use this method to retrieve the timestamp when the KerberosKey object was uploaded. This is a read-only attribute.

Parameter

None

Returns

The return value is the date and time in the ISO 8601 extended format (e.g. 2008-10-21T10:56:33Z or 2008-10-21T10:56:33.001Z).

Example

```
#Get upload_timestamp value
my $upload_timestamp = $key->upload_timestamp();
```

SAMPLE CODE

The following sample code demonstrates different operations that can be applied to an object such as create, get, and remove an object. Also this sample code also includes error handling for the operations.

```
#PROGRAM STARTS: Include all the modules that will be used
 use strict;
 use Infoblox;

 #Create a session to the Infoblox appliance
 my $session = Infoblox::Session->new(
    master   => "192.168.1.2",
    username => "admin",
    password => "infoblox"
 );
 unless ($session) {
   die("Construct session failed: ",
       Infoblox::status_code() . ":" . Infoblox::status_detail());
 }
 print "Session created successfully\n";
```

#import kerberos key object

```
my $result = $session->import_data(
    type       => "keytab",
    path       => "/infoblox/tests/datasets/dnsone.keytab",
    service => 'dhcp',
   );

unless($result) {
       die("import kerberos key object failed: ",
            Infoblox::status_code() . ":" . Infoblox::status_detail());
}
print "import kerberos key object successfully\n";
```

#retrieve kerberos key object

```
my @keys = $session->get(
   object    => "Infoblox::Grid::KerberosKey",
   principal => 'host/appliance.ivan.test@LADYBUG.LOCAL',
 );

 unless ($key) {
       die("Get KerberosKey object failed: ",
            $session->status_code() . ":" . $session->status_detail());
  }
  print "Get KerberosKey object found at least 1 matching entry\n";

my $grid_dhcp = $session->get(
   object  => 'Infoblox::Grid::DHCP',
   cluster => 'Infoblox',
   );
unless ($grid_dhcp) {
       die("Get Grid DHCP object failed: ",
            $session->status_code() . ":" . $session->status_detail());
}

$grid_dhcp->gss_tsig_keys([$key]);
$grid_dhcp->kdc_server('test.com');

#Applying the changes to appliance through session.
$session->modify($grid_dhcp)
     or die("modify Grid DHCP properties failed:" ,
      $session->status_code(), $session->status_detail());
print "Grid DHCP properties modified successfully for assigning KerberosKey object\n";
```

#Removing an KerberosKey object

```
# remove the reference to kerberos key
$grid_dhcp->gss_tsig_keys(undef);
$grid_dhcp->kdc_server(undef);

#Applying the changes to appliance through session.
$session->modify($grid_dhcp)
     or die("modify Grid DHCP properties failed:" ,
      $session->status_code(), $session->status_detail());
print "Grid DHCP properties modified successfully for unassigning KerberosKey object\n";
```

```
$session->remove(object => 'Infoblox::Grid::KerberosKey',
                 principal => 'host/appliance.ivan.test@LADYBUG.LOCAL')
     or die("remove KerberosKey object failed:" ,
      $session->status_code(), $session->status_detail());
print "remove KerberosKey object successfully\n";

####PROGRAM ENDS####
```

AUTHOR

Infoblox Inc. http://www.infoblox.com/

SEE ALSO

Infoblox::Session, Infoblox::Grid::DNS, Infoblox::Grid::DHCP, Infoblox::Grid::Member::DNS, Infoblox::Grid::Member::DHCP, Infoblox::Grid::DNS->gss_tsig_keys(), Infoblox::Grid::DHCP->gss_tsig_keys(), Infoblox::Grid::Member::DNS->gss_tsig_keys(), Infoblox::Grid::Member::DHCP->gss_tsig_keys(), Infoblox::Session->get(), Infoblox::Session->modify(),Infoblox::Session->remove()

COPYRIGHT

Copyright (c) 2017 Infoblox Inc.

Infoblox::Grid::LicenseGridWide - a Grid-wide license object.

- NAME
- DESCRIPTION
- CONSTRUCTOR
- SESSION METHODS
  - Infoblox::Session->get( )
  - Infoblox::Session->search( )
  - Infoblox::Session->remove( )
- METHODS
  - expiration( )
  - expiration_status( )
  - key( )
  - limit( )
  - limit_context( )
  - type( )
- AUTHOR
- SEE ALSO
- COPYRIGHT

NAME

Infoblox::Grid::LicenseGridWide - a Grid-wide license object.

DESCRIPTION

A Grid-wide license is assigned and locked to the Grid to activate the licensed feature on all eligible appliances. The Grid-wide license object is used to display infromation related to Grid-wide license (i.e. expiration date, limit value).

CONSTRUCTOR

The Infoblox::Grid::LicenseGridWide object is a read-only object and does not require manual construction.

SESSION METHODS

This section describes all the methods in the Infoblox::Session module that you can apply to a Grid-wide license object.

Infoblox::Session->get( )

Use this method to retrieve the existing objects from an Infoblox appliance. See Infoblox::Session->get() for parameters and return values.

Example

```
my $retrieved_objs = $session->get(object => "Infoblox::Grid::LicenseGridWide");
```

Infoblox::Session->search( )

Use this method to retrieve the existing objects from an Infoblox appliance. See Infoblox::Session->search() for parameters and return values.

Key Reference

```
Apply the following attributes to search the Grid-wide license object(s):

    "type"  - Optional. The license type.
    "limit" - Optional. The limitation of dynamic license.
    "key"   - Optional. The license string.
```

Example

```
my @retrieved_objs = $session->search(
    object => "Infoblox::Grid::LicenseGridWide",
    type   => "DNS"
);
```

Infoblox::Session->remove( )

Use this method to remove a Grid-wide license object from the Infoblox device. See Infoblox::Session->remove() for parameters and return values.

Example

```
#search for a specific object
my @retrieved_objs = $session->search(
    object => "Infoblox::Grid::LicenseGridWide",
    type   => "DNS"
);

my $object = $retrieved_objs[0];

#remove the object
my $response = $session->remove($object);
```

METHODS

This section describes all the methods that you can use to set or retrieve the attribute values of a Grid-wide license object.

expiration( )

Use this method to retrieve the Grid-wide license expiration timestamp. This is a read-only attribute.

Parameter

None

Returns

The method returns the number of seconds elapsed since January 1, 1970, in UTC format.

Example

```
#Get expiration value
my $expiration = $object->expiration();
```

expiration_status( )

Use this method to retrieve the Grid-wide license expiration status. This is a read-only attribute.

Parameter

None

Returns

The method returns the attribute value.

Example

```
#Get expiration_status value
my $expiration_status = $object->expiration_status();
```

key( )

Use this method to retrieve the license string for the Grid-wide license. This is a read-only attribute.

Parameter

None

Returns

The method returns the attribute value.

Example

```
#Get key value
my $value = $object->key();
```

limit( )

Use this method to retrieve the number of dynamic licenses that can be allocated from the Grid-wide license. This is a read-only attribute.

Parameter

None

Returns

The method returns the attribute value.

Example

```
#Get limit value
my $value = $object->limit();
```

limit_context( )

Use this method to retrieve the context of the license limit. This is a read-only attribute.

Parameter

None

Returns

The method returns the attribute value.

Example

```
#Get limit_context value
my $limit_context = $object->limit_context();
```

type( )

Use this method to retrieve the license type. This is a read-only attribute.

Parameter

None

Returns

The method returns the attribute value.

Example

```
#Get type value
my $value = $object->type();
```

AUTHOR

Infoblox Inc. http://www.infoblox.com/

SEE ALSO

Infoblox::Session,

COPYRIGHT

Copyright (c) 2017 Infoblox Inc.

Infoblox::Grid::LicensePool - License Pool object.

- NAME
- DESCRIPTION
- CONSTRUCTOR
- SESSION METHODS
  - Infoblox::Session->get( )
  - Infoblox::Session->search( )
- METHODS
  - assigned( )
  - expiration( )
  - expiration_status( )
  - installed( )
  - key( )
  - limit( )
  - limit_context( )
  - model( )
  - subpools( )
  - temp_assigned( )
  - type( )
- SAMPLE CODE
- AUTHOR
- SEE ALSO
- COPYRIGHT

NAME

Infoblox::Grid::LicensePool - License Pool object.

DESCRIPTION

A license pool is a container associated with the Grid and holds dynamic licenses for a specific feature. Licenses in the pool can be dynamically allocated to and deallocated from Grid members. When not in use, dynamic licenses are released back to the pool for future allocation. There is no expiration for dynamic licenses.

CONSTRUCTOR

The Infoblox::Grid::LicensePool object is a read-only object and does not require manual construction.

SESSION METHODS

This section describes all the methods in the Infoblox::Session module that you can apply to a License Pool object.

Infoblox::Session->get( )

Use this method to retrieve the existing objects from an Infoblox appliance. See Infoblox::Session->get() for parameters and return values.

Example

```
my $retrieved_objs = $session->get(object => "Infoblox::Grid::LicensePool");
```

Infoblox::Session->search( )

Use this method to retrieve the existing objects from an Infoblox appliance. See Infoblox::Session->search() for parameters and return values.

Key Reference

```
Apply the following attributes to search the License Pool object(s):

    "type"  - Optional. The license type.
    "limit" - Optional. The limitation of dynamic license.
    "model" - Optional. The supported vNIOS virtual applaince model.
    "key"   - Optional. The license string.
```

Example

```
my @retrieved_objs = $session->search(
    object => "Infoblox::Grid::LicensePool",
```

```
        type    => "DNS"
);
```

METHODS

This section describes all the methods that you can use to set or retrieve the attribute values of a License Pool object.

assigned( )

Use this method to retrieve the number of dynamic licenses allocated to vNIOS appliances. This is a read-only attribute.

Parameter

None

Returns

The method returns the attribute value.

Example

```
#Get assigned value
my $value = $object->assigned();
```

expiration( )

Use this method to retrieve the license pool expiration timestamp. This is a read-only attribute.

Parameter

None

Returns

The method returns the number of seconds elapsed since January 1, 1970, in UTC format.

Example

```
#Get expiration value
my $expiration = $object->expiration();
```

expiration_status( )

Use this method to retrieve the license pool expiration status. This is a read-only attribute.

Parameter

None

Returns

The method returns the attribute value.

Example

```
#Get expiration_status value
my $expiration_status = $object->expiration_status();
```

installed( )

Use this method to retrieve the total number of dynamic licenses allowed for this license pool. This is a read-only attribute.

Parameter

None

Returns

The method returns the attribute value.

Example

```
#Get installed value
my $value = $object->installed();
```

key( )

Use this method to retrieve the license string for the license pool. This is a read-only attribute.

Parameter

None

Returns

The method returns the attribute value.

Example

```
#Get key value
my $value = $object->key();
```

limit( )

Use this method to retrieve the total number of available dynamic licenses from the license pool. This is a read-only attribute.

Parameter

None

Returns

The method returns the attribute value.

Example

```
#Get limit value
my $value = $object->limit();
```

limit_context( )

Use this method to retrieve the context of the license limit. This is a read-only attribute.

Parameter

None

Returns

The method returns the attribute value.

Example

```
#Get limit_context value
my $limit_context = $object->limit_context();
```

model( )

Use this method to retrieve the supported vNIOS appliance model. This is a read-only attribute.

Parameter

None

Returns

The method returns the attribute value.

Example

```
#Get model value
my $value = $object->model();
```

subpools( )

Use this method to retrieve dynamic licenses in sub pools. This is a read-only attribute.

Parameter

None

Returns

The method returns the attribute value. An array of Infoblox::Grid::LicenseSubPool objects.

Example

```
#Get subpools value
my $value = $object->subpools();
```

temp_assigned( )

Use this method to retrieve the total number of temporary dynamic licenses allocated to vNIOS appliances. This is a read-only attribute.

Parameter

None

Returns

The method returns the attribute value.

Example

```
#Get temp_assigned value
my $value = $object->temp_assigned();
```

type( )

Use this method to retrieve the license type. This is a read-only attribute.

Parameter

None

Returns

The method returns the attribute value.

Example

```
#Get type value
my $value = $object->type();
```

SAMPLE CODE

The following sample code demonstrates different operations that can be applied to a License Pool object. This sample code also includes error handling for the operations.

#Preparation prior License Pool fetching

```
#PROGRAM STARTS: Include all the modules that will be used
use strict;
use Infoblox;

#Create a session to the Infoblox device
my $session = Infoblox::Session->new(
    master   => "192.168.1.2",
    username => "admin",
    password => "infoblox"
);

unless ($session) {
   die("Construct session failed: ",
       Infoblox::status_code() . ":" . Infoblox::status_detail());
}
print "Session created successfully\n";
```

#Fetching the License Pool objects

```
my $retrieved_objs=$session->get(object=> "Infoblox::Grid::LicensePool");

unless($retrieved_objs) {
    die("Getting the license pools failed:",
       Infoblox::status_code() . ":" . Infoblox::status_detail());
}
print "Got the license pools successfully\n";
####PROGRAM ENDS####
```

AUTHOR

Infoblox Inc. http://www.infoblox.com/

SEE ALSO

Infoblox::Session, Infoblox::Grid::LicenseSubPool

COPYRIGHT

Copyright (c) 2017 Infoblox Inc.

Infoblox::Grid::LicensePoolContainer - License Pool Container object.

- NAME
- DESCRIPTION
- CONSTRUCTOR
- SESSION METHODS
  - Infoblox::Session->get( )
- METHODS
  - lpc_uid( )
  - last_entitlement_update( )
  - update_licenses( )
  - allocate_licenses( )
  - download_pool_status( )
- SAMPLE CODE
- AUTHOR
- SEE ALSO
- COPYRIGHT

NAME

Infoblox::Grid::LicensePoolContainer - License Pool Container object.

DESCRIPTION

License Pool Container is the list of all license pools existing for a Grid.

CONSTRUCTOR

The Infoblox::Grid::LicensePoolContainer object is a read-only object and does not require manual construction.

SESSION METHODS

This section describes all the methods in the Infoblox::Session module that you can apply to a License Pool Container object.

Infoblox::Session->get( )

Use this method to retrieve the existing objects from an Infoblox appliance. See Infoblox::Session->get() for parameters and return values.

Example

```
my $retrieved_objs = $session->get(object => "Infoblox::Grid::LicensePoolContainer");
```

METHODS

This section describes all the methods that you can use to set or retrieve the attribute values of a License Pool Container object.

lpc_uid( )

Use this method to retrieve the world-wide unique ID for the license pool container. This attribute is read-only and cannot be set.

Parameter

None.

Returns

The method returns the attribute value.

Example

```
#Get the unique ID for the license pool container
my $lpc_uid = $license_pool_container->lpc_uid();
```

last_entitlement_update( )

Use this method to retrieve the timestamp of the last entitlement pool update. This attribute is read-only and cannot be set.

Parameter

None.

Returns

The method returns the attribute value.

Example

```
#Get the last entitlement pool update
my $last_entitlement_update = $license_pool_container->last_entitlement_update();
```

update_licenses( )

Use this method to upload new license file.

Parameter

"path" - Required. Path to the signed file with static/dynamic licenses to upload.

Returns

None.

Example

```
$license_pool_container->update_licenses(path => $path_to_license);
```

allocate_licenses( )

Use this method to allocate dynamic licenses to a node.

Parameter

"hwid" - Required. The hardware ID of a physical node to install dynamic licenses to.

"license_pools" - Required. The license pools from which you allocate dynamic licenses. An array containing Infoblox::Grid::LicensePool objects.

Returns

None.

Example

```
$license_pool_container->allocate_licenses(hwid => $hwid,
                                           license_pools => [$licence_pool1, $licence_pool2]);
```

download_pool_status( )

Use this method to to download signed file with static licenses/license pools.

Parameter

None.

Returns

"path" - The path to the file with licenses to download.

Example

```
$hash_with_path = $license_pool_container->download_pool_status();
```

SAMPLE CODE

The following sample code demonstrates different operations that can be applied to a License Pool object. This sample code also includes error handling for the operations.

#Preparation prior License Pool Container fetching

```
#PROGRAM STARTS: Include all the modules that will be used
use strict;
use Infoblox;

#Create a session to the Infoblox device
my $session = Infoblox::Session->new(
```

```
    master   => "192.168.1.2",
    username => "admin",
    password => "infoblox"
);

unless ($session) {
   die("Construct session failed: ",
       Infoblox::status_code() . ":" . Infoblox::status_detail());
}
print "Session created successfully\n";
```

#Fetching the License Pool Container objects

```
my $retrieved_objs=$session->get(object=> "Infoblox::Grid::LicensePoolContainer");

unless($retrieved_objs) {
    die("Getting the license pool container failed:",
       Infoblox::status_code() . ":" . Infoblox::status_detail());
}
print "Got the license pool container successfully\n";
####PROGRAM ENDS####
```

AUTHOR

Infoblox Inc. http://www.infoblox.com/

SEE ALSO

Infoblox::Session, Infoblox::Grid::LicensePool

COPYRIGHT

Copyright (c) 2017 Infoblox Inc.

Infoblox::Grid::LicenseSubPool - a license subpool object.

- NAME
- DESCRIPTION
- CONSTRUCTOR
- SESSION METHODS
- MODULE METHODS
  - Infoblox::Grid::LicensePool->subpools( )
- METHODS
  - expiration( )
  - installed( )
  - key( )
- AUTHOR
- SEE ALSO
- COPYRIGHT

NAME

Infoblox::Grid::LicenseSubPool - a license subpool object.

DESCRIPTION

The license subpool object shows the license installments that have the same expiry date.

CONSTRUCTOR

The Infoblox::Grid::LicenseSubPool object is a read-only object and does not require manual construction.

SESSION METHODS

The object does not support any session methods.

MODULE METHODS

Infoblox::Grid::LicensePool->subpools( )

Use this method to retrieve the license pool subpools. For parameters and return values see Infoblox::Grid::LicensePool->subpools.

Example

```
#Get subpools value
my $value = $object->subpools();
```

METHODS

This section describes all the methods that you can use to set or retrieve the attribute values of a license subpool object.

expiration( )

Use this method to retrieve the license subpool expiration timestamp. This is a read-only attribute.

Parameter

None

Returns

The method returns the number of seconds elapsed since January 1, 1970, in UTC format.

Example

```
#Get expiration value
my $expiration = $object->expiration();
```

installed( )

Use this method to retrive the total number of dynamic licenses allowed for this license sub pool. This is a read-only attribute.

Parameter

None

Returns

The method returns the attribute value.

Example

```
#Get installed value
my $value = $object->installed();
```

key( )

Use this method to retrieve the license string for the license subpool. This is a read-only attribute.

Parameter

None

Returns

The method returns the attribute value.

Example

```
#Get key value
my $value = $object->key();
```

AUTHOR

Infoblox Inc. http://www.infoblox.com/

SEE ALSO

Infoblox::Session, Infoblox::Grid::LicensePool, Infoblox::Grid::LicensePool->subpools,

---

COPYRIGHT

Copyright (c) 2017 Infoblox Inc.

Infoblox::Grid::LOM::User - User information for the Lights Out Management functionality.

- NAME
- DESCRIPTION
- CONSTRUCTOR
- MODULE METHODS
  - Infoblox::Grid::Member->bgp_as( )
- METHODS
  - disable( )
  - name( )
  - password( )
  - role( )
- AUTHOR
- SEE ALSO
- COPYRIGHT

NAME

Infoblox::Grid::LOM::User - User information for the Lights Out Management functionality.

DESCRIPTION

An Infoblox::Grid::LOM::User object represents a user for the LOM functionality.

CONSTRUCTOR

```
my $user    = Infoblox::Grid::LOM::User->new(
     name     => $string,              # Required
     password => $string,              # Required
     role     => "OPERATOR" | "USER", # Required
     disable  => "true" | "false",     # Required
);
```

MODULE METHODS

The following functions can be applied to a LOM User object.

Infoblox::Grid::Member->bgp_as( )

Use this function to specify BGP Autonomous Systems for the grid member. See Infoblox::Grid::Member->bgp_as() for parameters and return values.

Example

```
#Construct a LOM User
my $lomuser = Infoblox::Grid::LOM::User->new(
     name     => 'username',
     password => 'password',
     role     => 'OPERATOR',
     disable  => 'false,
);


# Configure the LOM user on the grid object
$grid->lom_users([$lomuser]);
```

METHODS

This section describes all the methods that can be used to configure and retrieve the attribute values of an Infoblox::Grid::LOM::User object.

disable( )

Use this method to set or retrieve the disable flag for the user.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

Specify "true" to set the disable flag or "false" to deactivate/unset it.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get the disable
my $disable = $lom_user->disable();
#Modify the disable
$lom_user->disable('false');
```

name( )

Use this method to set or retrieve the name of the user.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

The name of the user in string format.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get the name
my $name = $lom_user->name();
#Modify the name
$lom_user->name('username');
```

password( )

Use this method to set the password the user will use to authenticate. This is a write-only attribute.

Include the specified parameter to set the attribute value.

Parameter

The password for the user in string format.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

Example

```
#Modify the password
$lom_user->password('password');
```

role( )

Use this method to set or retrieve the role of the user.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

Valid values are "OPERATOR" and "USER".

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get the role
my $role = $lom_user->role();
#Modify the role
$lom_user->role('OPERATOR');
```

Infoblox::Grid::LOM::User - User information for the Lights Out Management functionality.

AUTHOR

Infoblox Inc. http://www.infoblox.com/

SEE ALSO

Infoblox::Grid::Member, Infoblox::Grid::Member::BGP::Neighbor

COPYRIGHT

Copyright (c) 2017 Infoblox Inc.

Infoblox::Grid::MaxMindDBInfo - an MaxMind Database information object.

- NAME
- DESCRIPTION
- SESSION METHODS
  - Infoblox::Session->get( )
- MODULE METHODS
  - Infoblox::DTC->import_maxminddb( )
  - import_maxminddb( )
- METHODS
  - binary_major_version( )
  - binary_minor_version( )
  - build_time( )
  - database_type( )
  - deployment_time( )
  - member( )
  - topology_type( )
- AUTHOR
- SEE ALSO
- COPYRIGHT

NAME

Infoblox::Grid::MaxMindDBInfo - an MaxMind Database information object.

DESCRIPTION

The object provides information about the MaxMind database.

SESSION METHODS

This section describes all the methods in a Session module that you can apply to a MaxMindDBInfo object. Note that the object supports only the get method since there is only one such object in the database.

Infoblox::Session->get( )

Use this method to retrieve all the matching objects from the Infoblox appliance. See Infoblox::Session->get() for parameters and return values.

Key Reference

```
Apply the following attribute to get a specific object:

  binary_major_version - Optional. The major binary version of MaxMind database.
  binary_minor_version - Optional. The minor binary version of MaxMind database.
  build_time           - Optional. The time at which the MaxMind database was built.
  database_type        - Optional. The structure of data records.
  deployment_time      - Optional. The time at which the current MaxMind database was deployed.
  topology_type        - Optional. The MaxMind database topology type.
```

Example

```
my @retrieved_objs = $session->get(
    object => 'Infoblox::DTC::Monitor::TCP',
    database_type => 'GeoLite2-Country',
);
```

MODULE METHODS

Infoblox::DTC->import_maxminddb( )

import_maxminddb( )

Use this method to upload MaxMind Database.

Key Reference

```
Apply the following attribute to upload a MaxMind Database:

    path - Required. The path to a tar archive that contains the MaxMind Database.
```

Returns

The method returns "true" when an upload of a MaxMind Database has been performed successfully, and returns "false" when the operation fails.

Example

```
#upload MaxMind DB
my $res = $dtc->import_maxminddb(path => $path_to_maxminddb);
```

METHODS

This section describes all the methods that you can use to set or retrieve the attribute values of the object.

binary_major_version( )

Use this method to retrieve the major version of the MaxMind database binary format. This is a read-only attribute.

Parameter

None

Returns

The method returns attribute value.

Example

```
#get binary_major_version value
my $binary_major_version = $maxminddbinfo->binary_major_version();
```

binary_minor_version( )

Use this method to retrieve the minor version of the MaxMind database binary format. This is a read-only attribute.

Parameter

None

Returns

The method returns the attribute value.

Example

```
#get binary_minor_version value
my $binary_minor_version = $maxminddbinfo->binary_minor_version();
```

build_time( )

Use this method to retrieve the time when the MaxMind database was built. This is a read-only attribute.

Parameter

None

Returns

The method returns attribute value.

Example

```
#get build_time value
my $build_time = $maxminddbinfo->build_time();
```

database_type( )

Use this method to retrieve the structure of MaxMind database records. This is a read-only attribute.

Parameter

None

Returns

The method returns the attribute value.

Example

```
#get database_type value
my $database_type = $maxminddbinfo->database_type();
```

deployment_time( )

Use this method to retrieve the time when the current MaxMind database was deployed. This is a read-only attribute.

Parameter

None

Returns

The method returns the attribute value.

Example

```
#get deployment_time value
my $deployment_time = $maxminddbinfo->deployment_time();
```

member( )

Use this method to retrieve the Grid member name on which the current MaxMind database was deployed. This is a read-only attribute.

Parameter

None

Returns

The method returns the attribute value.

Example

```
#Get member value
my $value = $object->member();
```

topology_type( )

Use this method to retrieve the MaxMind database topology type. This is a read-only attribute.

Parameter

None

Returns

The method returns the attribute value. The valid return values are 'GEOIP' and 'EA'.

Example

```
#get topology_type value
my $topology_type = $maxminddbinfo->topology_type();
```

AUTHOR

Infoblox Inc. http://www.infoblox.com/

SEE ALSO

Infoblox::Session, Infoblox::Session->get(), Infoblox::DTC->import_maxminddb()

COPYRIGHT

Copyright (c) 2017 Infoblox Inc.

Infoblox::Grid::Member - Grid Member object.

- NAME
- DESCRIPTION
- CONSTRUCTOR
- SESSION METHODS
  - Infoblox::Session->add( )
  - Infoblox::Session->get( )
  - Infoblox::Session->modify( )
  - Infoblox::Session->remove( )
  - Infoblox::Session->search( )
- METHODS
  - active_position( )
  - additional_ip_list( )
  - allow_syslog_clients( )
  - alternate_resolver( )
  - bgp_as( )
  - comment( )
  - config_addr_type( )
  - dscp( )
  - enable_ha( )
  - enable_lom( )
  - enable_member_redirect( )
  - enable_ntp( )
  - enable_snmpv3_query( )
  - enable_snmpv3_traps( )
  - enable_ro_api_access( )
  - enable_syslog_proxy( )
  - external_ntp_servers_enabled( )
  - exclude_grid_master_as_ntp_server( )
  - extattrs( )
  - extensible_attributes( )
  - gateway( )
  - ha_node2_port_duplex( )
  - ha_node2_port_speed( )
  - ha_port_duplex( )
  - ha_port_speed( )
  - ha_reserved_interface( )
  - paid_nios( )
  - hwplatform( )
  - hwid( )
  - is_dscp_capable( )
  - ipv4addr( )
  - ipv6addr( )
  - ipv6_cidr( )
  - ipv6_enable_auto_config( )
  - ipv6_gateway( )
  - ipv6_static_routes( )
  - ipv6_vlan_id( )
  - lan_reserved_interface( )
  - lan1_dscp( )
  - lan1_ipv6_dscp( )
  - lan2_dscp( )
  - lan2_gateway( )
  - lan2_ipv4addr( )
  - lan2_ipv6addr( )
  - lan2_ipv6_cidr( )
  - lan2_ipv6_dscp( )
  - lan2_ipv6_enable( )
  - lan2_ipv6_enable_auto_config( )
  - lan2_ipv6_gateway( )
  - lan2_ipv6_vlan_id( )
  - lan2_mask( )
  - lan2_node2_port_duplex( )
  - lan2_port( )
  - lan2_node2_port_speed( )
  - lan2_port_duplex( )
  - lan2_port_speed( )
  - lan2_reserved_interface( )
  - lan2_router_id( )
  - lan2_vlan_id( )
  - lan_node2_port_duplex( )
  - lan_node2_port_speed( )

- lan_port_duplex( )
- lan_port_speed( )
- lcd_input( )
- lcd_autodim( )
- lcd_brightness( )
- lcd_hwident( )
- lcd_version( )
- lom_gateway( )
- lom_ipv4addr( )
- lom_is_capable( )
- lom_mask( )
- lom_users( )
- mask( )
- master_candidate( )
- member_service_communication( )
- mgmt_dscp( )
- mgmt_gateway( )
- mgmt_lan( )
- mgmt_ipv6addr( )
- mgmt_ipv6_cidr( )
- mgmt_ipv6_dscp( )
- mgmt_ipv6_enable_auto_config( )
- mgmt_ipv6_gateway( )
- mgmt_mask( )
- mgmt_node2_port_duplex( )
- mgmt_node2_port_speed( )
- mgmt_port( )
- mgmt_port_duplex( )
- mgmt_port_speed( )
- mgmt_reserved_interface( )
- mmdb_ea_build_time( )
- mmdb_geoip_build_time( )
- name( )
- nat_enabled( )
- nat_group( )
- nat_ip_address( )
- nic_failover_enabled( )
- nic_failover_enable_primary( )
- node1_ha( )
- node1_ha_reserved_interface( )
- node1_paid_nios( )
- node1_hwplatform( )
- node1_hwid( )
- node1_lan( )
- node1_lan_ipv6addr( )
- node1_lan_reserved_interface( )
- node1_lan2_reserved_interface( )
- node1_lcd_autodim( )
- node1_lcd_brightness( )
- node1_lcd_hwident( )
- node1_lcd_version( )
- node1_lom_gateway( )
- node1_lom_is_capable( )
- node1_lom_ipv4addr( )
- node1_lom_mask( )
- node1_mgmt_dscp( )
- node1_mgmt_gateway( )
- node1_mgmt_ipv6addr( )
- node1_mgmt_ipv6_cidr( )
- node1_mgmt_ipv6_dscp( )
- node1_mgmt_ipv6_enable_auto_config( )
- node1_mgmt_ipv6_gateway( )
- node1_mgmt_mask( )
- node1_mgmt_lan( )
- node1_mgmt_port( )
- node1_mgmt_reserved_interface( )
- node1_nat( )
- node1_service_status( )
- vpn_mtu( )
- node1_vpn_on_mgmt( )
- node1_hwtype( )
- node2_ha( )
- node2_ha_reserved_interface( )
- node2_paid_nios( )

  - node2_hwplatform( )
  - node2_hwid( )
  - node2_lan( )
  - node2_lan_ipv6addr( )
  - node2_lan_reserved_interface( )
  - node2_lan2_reserved_interface( )
  - node2_lcd_autodim( )
  - node2_lcd_brightness( )
  - node2_lcd_hwident( )
  - node2_lcd_version( )
  - node2_lom_gateway( )
  - node2_lom_is_capable( )
  - node2_lom_ipv4addr( )
  - node2_lom_mask( )
  - node2_mgmt_dscp( )
  - node2_mgmt_gateway( )
  - node2_mgmt_mask( )
  - node2_mgmt_lan( )
  - node2_mgmt_port( )
  - node2_mgmt_reserved_interface( )
  - node2_mgmt_ipv6_dscp( )
  - node2_nat( )
  - node2_service_status( )
  - node2_vpn_on_mgmt( )
  - node2_hwtype( )
  - notification_email_addr( )
  - ntp_access_list( )
- ntp_kod( )
  - ntp_service_type( )
  - ntp_authentication_key( )
  - ntp_server( )
  - ospf_list( )
  - override_enable_lom( )
  - override_lom_users( )
  - override_dscp( )
  - override_enable_member_redirect( )
  - override_lan1_dscp( )
  - override_lan1_ipv6_dscp( )
  - override_lan2_dscp( )
  - override_lan2_ipv6_dscp( )
  - override_mgmt_dscp( )
  - override_mgmt_ipv6_dscp( )
  - override_node1_mgmt_dscp( )
  - override_node1_mgmt_ipv6_dscp( )
  - override_node2_mgmt_dscp( )
  - override_node2_mgmt_ipv6_dscp( )
  - override_ntp_kod( )
  - override_syslog_backup_servers( )
  - override_threshold_traps( )
  - override_trap_notifications( )
  - passive_ha_arp_enabled( )
  - platform( )
  - prefer_resolver( )
  - pre_provisioning( )
  - preserve_if_owns_delegation( )
  - query_comm_string( )
  - remote_console_access( )
  - router_ID( )
  - search_domains( )
  - service_status( )
  - service_type_configuration( )
  - snmp_admin( )
  - snmpv3_query_users( )
  - static_routes( )
  - support_access( )
  - support_access_info( )
  - syslog_proxy_tcp_port( )
  - syslog_proxy_udp_port( )
  - syslog_size( )
  - syslog_backup_servers( )
  - syslog_server( )
  - time_zone( )
  - threshold_traps( )
  - traffic_capture( )

- trap_comm_string( )
- trap_notifications( )
- trap_receiver( )
- upgrade_group( )
- use_v4_vrrp( )
- vpn_on_mgmt( )
- vlan_id( )
- create_token( )
- read_token( )

- SAMPLE CODE
- AUTHOR
- SEE ALSO
- COPYRIGHT

NAME

Infoblox::Grid::Member - Grid Member object.

DESCRIPTION

A grid member is a single Infoblox appliance or an HA pair of Infoblox appliances that belong to a grid.

CONSTRUCTOR

```
 my $grid_member = Infoblox::Grid::Member->new(
     gateway                 => $ipv4addr,                                   #Required
     ipv4addr                => $ipv4addr,                                   #Required
     mask                    => $netmask,                                    #Required
     name                    => $fqdn,                                       #Required
     additional_ip_list      => [$Interface1, $ipv4addr2, ...],              #Optional / Default
value is empty
     allow_syslog_clients    => [$ipv4addr | $ipv4_subnet/prefix, ...] | $nacl | undef, #Optional / Default
value is undefined
     bgp_as                  => [$AS1, ...],                                 #Optional / Default
value is empty
     alternate_resolver      => $Nameserver | $Member | $ipv4addr | undef,    #Optional / Default
value is undefined
     comment                 => $string,                                     #Optional / Default
value is empty
     config_addr_type        => "IPv4", "IPv6", "BOTH"                       #Optional / Default
value is "IPv4"
     dscp                    => $uint,
#Optional / Default value is 0
     enable_member_redirect  => "true" | "false",                            #Optional / Default
value is "false"
     enable_ha               => "true" | "false",                            #Optional / Default
value is "false"
     enable_lom              => "true" | "false",                            #Optional / Default
value is undefined
     enable_ntp              => "true" | "false",                            #Optional / Default
value is "false"
     enable_ro_api_access    => "true" | "false",                            #Optional / Default
value is "false"
     enable_syslog_proxy     => "true" | "false",                            #Optional / Default
value is "false"
     exclude_grid_master_as_ntp_server=> "true" | "false",                   #Optional / Default
value is "false"
     extattrs                => { $string => $extattr, ... },                #Optional / Default is
undefined
     extensible_attributes   => { $string => $string | $num, $string => [ $string | $num, ... ], ... }
#Optional / Default is undefined
     external_ntp_servers_enabled  => "true" | "false",                      #Optional / Default
value is "false"
     ha_node2_port_duplex    => "Half" | "Full" | undef,                     #Optional / Default
value is undefined
     ha_node2_port_speed     => 10 | 100 | 1000 | undef,                     #Optional / Default
value is undefined
     ha_port_duplex          => "Half" | "Full" | undef,                     #Optional / Default
value is undefined
     ha_port_speed           => 10 | 100 | 1000 | undef,                     #Optional / Default
value is undefined
     ha_reserved_interface   => $interface,                                  #Optional
     ipv6addr                => $ipv6addr,                                   #Optional / Default
value is undefined
     ipv6_cidr               => $cidr,                                       #Optional / Default
value is undefined
     ipv6_gateway            => $ipv6addr,                                   #Optional / Default
```

```
  value is undefined
      ipv6_enable_auto_config => "true" | "false",                    #Optional / Default
  value is "false"
      ipv6_static_routes      => [$Route1, $Route2, ...],             #Optional / Default
  value is empty
      ipv6_vlan_id            => $uint,                               #Optional / Default
  value is undefined
      lan1_dscp               => $uint,                               #Optional / Default
  value is undefined
      lan1_ipv6_dscp          => $uint,                               #Optional / Default
  value is undefined
      lan2_dscp               => $uint,                               #Optional / Default
  value is undefined
      lan2_gateway            => $ipv4addr,                           #Optional / Default
  value is undefined
      lan2_ipv4addr           => $ipv4addr,                           #Optional / Default
  value is undefined
      lan2_ipv6addr           => $ipv6addr,                           #Optional / Default
  value is undefined
      lan2_ipv6_cidr          => $cidr,                               #Optional / Default
  value is undefined
      lan2_ipv6_dscp          => $uint,                               #Optional / Default
  value is undefined
      lan2_ipv6_gateway       => $ipv6addr,                           #Optional / Default
  value is undefined
      lan2_ipv6_enable        => "true" | "false",                    #Optional / Default
  value is "false"
      lan2_ipv6_enable_auto_config => "true" | "false",               #Optional / Default
  value is "false"
      lan2_ipv6_vlan_id       => $uint,                               #Optional / Default
  value is undefined
      lan2_mask               => $netmask,                            #Optional / Default
  value is undefined
      lan2_node2_port_duplex  => "Half" | "Full" | undef,             #Optional / Default
  value is undefined
      lan2_node2_port_speed   => 10 | 100 | 1000 | undef,             #Optional / Default
  value is undefined
      lan2_port_duplex        => "Half" | "Full" | undef,             #Optional / Default
  value is undefined
      lan2_port_speed         => 10 | 100 | 1000 | undef,             #Optional / Default
  value is undefined
      lan2_port               => "true" | "false",                    #Optional / Default
  value is "false"
      lan2_reserved_interface => $interface,                          #Optional
      lan2_router_id          => $num,                                #Optional / Default
  value is undefined
      lan2_vlan_id            => $uint,                               #Optional / Default
  value is undefined
      lan_node2_port_duplex   => "Half" | "Full" | undef,             #Optional / Default
  value is undefined
      lan_node2_port_speed    => 10 | 100 | 1000 | undef,             #Optional / Default
  value is undefined
      lan_port_duplex         => "Half" | "Full" | undef,             #Optional / Default
  value is undefined
      lan_port_speed          => 10 | 100 | 1000 | undef,             #Optional / Default
  value is undefined
      lan_reserved_interface  => $interface,                          #Optional
      lcd_input               => "true" | "false" | undef,            #Optional / Default
  value is undefined
      lcd_autodim             => $num,                                #Optional / Default
  value is 30
      lcd_brightness          => $num,                                #Optional / Default
  value is 7
      lcd_hwident             => "true" | "false",                    #Optional / Default
  value is "false"
      lom_gateway             => $ipv4addr,
  #Optional / Default value is undefined
      lom_ipv4addr            => $ipv4addr,
  #Optional / Default value is undefined
      lom_mask                => $netmask,
  #Optional / Default value is undefined
      lom_users               => [$user1, $user2, ...],
  #Optional / Default value is undefined
      node1_lom_gateway       => $ipv4addr,
  #Optional / Default value is undefined
      node1_lom_ipv4addr      => $ipv4addr,
  #Optional / Default value is undefined
      node1_lom_mask          => $netmask,
  #Optional / Default value is undefined
      node2_lom_gateway       => $ipv4addr,
  #Optional / Default value is undefined
      node2_lom_ipv4addr      => $ipv4addr,
  #Optional / Default value is undefined
      node2_lom_mask          => $netmask,
```

```
#Optional / Default value is undefined
     master_candidate         => "true" | "false",                          #Optional / Default
value is undefined
     member_service_communication => [$c1, $c2, ...],                       #Optional / Default
value is undefined
     mgmt_dscp                => $uint,                                     #Optional / Default
value is undefined
     mgmt_gateway             => $ipv4addr,                                 #Optional / Default
value is undefined
     mgmt_lan                 => $ipv4addr,                                 #Optional / Default
value is undefined
     mgmt_ipv6addr            => $ipv6addr,                                 #Optional / Default
value is undefined
     mgmt_ipv6_cidr           => $cidr,                                     #Optional / Default
value is undefined
     mgmt_ipv6_dscp           => $uint,                                     #Optional / Default
value is undefined
     mgmt_ipv6_gateway        => $ipv6addr,                                 #Optional / Default
value is undefined
     mgmt_ipv6_enable_auto_config => "true" | "false",                      #Optional / Default
value is "false"
     mgmt_mask                => $netmask,                                  #Optional / Default
value is undefined
     mgmt_node2_port_duplex   => "Half" | "Full" | undef,                   #Optional / Default
value is undefined
     mgmt_node2_port_speed    => 10 | 100 | 1000 | undef,                   #Optional / Default
value is undefined
     mgmt_port_duplex         => "Half" | "Full" | undef,                   #Optional / Default
value is undefined
     mgmt_port_speed          => 10 | 100 | 1000 | undef,                   #Optional / Default
value is undefined
     mgmt_port                => "true" | "false",                          #Optional / Default
value is "false"
     mgmt_reserved_interface => $interface,                                 #Optional
     nat_enabled              => "true" | "false",                          #Optional / Default
value is "false"
     nat_group                => $string | undef,                           #Optional / Default
value is empty
     nat_ip_address           => $ipv4addr | undef,                         #Optional / Default
value is undefined

     nic_failover_enabled         => "true" | "false",                         #Optional / Default
value is "false"
     nic_failover_enable_primary => "true" | "false",                          #Optional / Default
value is "false"
     node1_ha                 => $ipv4addr,                                 #Optional / Default
value is undefined
     node1_ha_reserved_interface => $interface,                             #Optional
     node1_hwtype             => "IB-RSP" | ""                              #Optional / Default
value is ""
     node1_lan                => $ipv4addr,                                 #Optional / Default
value is undefined
     node1_lan_ipv6addr       => $ipv6addr,                                 #Optional / Default
value is undefined
     node1_lan_reserved_interface  => $interface,                           #Optional
     node1_lan2_reserved_interface => $interface,                           #Optional
     node1_lcd_autodim        => $num,                                      #Optional / Default
value is undefined
     node1_lcd_brightness     => $num,                                      #Optional / Default
value is undefined
     node1_lcd_hwident        => "true" | "false",                          #Optional / Default
value is undefined
     node1_mgmt_dscp          => $uint,                                     #Optional / Default
value is undefined
     node1_mgmt_gateway       => $ipv4addr,                                 #Optional / Default
value is undefined
     node1_mgmt_lan           => $ipv4addr,                                 #Optional / Default
value is undefined
     node1_mgmt_ipv6addr      => $ipv6addr,                                 #Optional / Default
value is undefined
     node1_mgmt_ipv6_cidr     => $cidr,                                     #Optional / Default
value is undefined
     node1_mgmt_ipv6_dscp     => $uint,                                     #Optional / Default
value is undefined
     node1_mgmt_ipv6_gateway => $ipv6addr,                                  #Optional / Default
value is undefined
     node1_mgmt_ipv6_enable_auto_config => "true" | "false",                #Optional / Default
value is "false"
     node1_mgmt_mask          => $netmask,                                  #Optional / Default
value is undefined
     node1_mgmt_port          => "true" | "false",                          #Optional / Default
value is undefined
     node1_mgmt_reserved_interface => $interface,                           #Optional
```

```
     node1_nat               => $ipv4addr | undef,                                  #Optional / Default
value is undefined
     node1_vpn_on_mgmt       => "true" | "false",                                   #Optional / Default
value is undefined
     node2_ha                => $ipv4addr,                                          #Optional / Default
value is undefined
     node2_ha_reserved_interface => $interface,                                     #Optional
     node2_hwtype            => ""                                                  #Optional / Default
value is ""
     node2_lan               => $ipv4addr,                                          #Optional / Default
value is undefined
     node2_lan_ipv6addr      => $ipv6addr,                                          #Optional / Default
value is undefined
     node2_lan_reserved_interface  => $interface,                                   #Optional
     node2_lan2_reserved_interface => $interface,                                   #Optional
     node2_mgmt_dscp         => $uint,                                              #Optional / Default
value is undefined
     node2_mgmt_ipv6addr     => $ipv6addr,                                          #Optional / Default
value is undefined
     node2_mgmt_ipv6_cidr    => $cidr,                                              #Optional / Default
value is undefined
     node2_mgmt_ipv6_dscp    => $uint,                                              #Optional / Default
value is undefined
     node2_mgmt_ipv6_gateway => $ipv6addr,                                          #Optional / Default
value is undefined
     node2_mgmt_ipv6_enable_auto_config => "true" | "false",                        #Optional / Default
value is "false"
     node2_mgmt_reserved_interface      => $interface,                              #Optional
     node2_lcd_autodim       => $num,                                               #Optional / Default
value is undefined
     node2_lcd_brightness    => $num,                                               #Optional / Default
value is undefined
     node2_lcd_hwident       => "true" | "false",                                   #Optional / Default
value is undefined
     node2_mgmt_gateway      => $ipv4addr,                                          #Optional / Default
value is undefined
     node2_mgmt_lan          => $ipv4addr,                                          #Optional / Default
value is undefined
     node2_mgmt_mask         => $netmask,                                           #Optional / Default
value is undefined
     node2_mgmt_port         => "true" | "false",                                   #Optional / Default
value is undefined
     node2_nat               => $ipv4addr | undef,                                  #Optional / Default
value is undefined
     node2_vpn_on_mgmt       => "true" | "false",                                   #Optional / Default
value is undefined
     notification_email_addr => $email,                                             #Optional / Default
value is undefined
     ntp_access_list         => [$ntpaccess1, $ntpaccess2, ... ] | $nacl | undef,  #Optional / Default
value is undefined
     ntp_service_type        => "Time" | "Time and NTP control",                    #Optional / Default
value is "Time"
     ntp_authentication_key  => [$NTPKey1, $NTPKey2, ...],                          #Optional / Default
value is undefined
     ntp_server              => [$ntp_server1, $ntp_server2] | undef,               #Optional / Default
value is undefined
     ntp_kod                 => "true" | "false",                                   #Optional / Default
value is undefined
     override_ntp_kod        => "true" | "false",                                   #Optional / Default
value is undefined
     ospf_list               => [$OSPF1, $OSPF2, ...],                              #Optional / Default
value is empty
     override_dscp           => "true" | "false",
#Optional / Default value is "false"
     override_enable_member_redirect  => "true" | "false",                          #Optional / Default
value is "false"
     override_lan1_dscp            => "true" | "false",                             #Optional / Default
value is "false"
     override_lan1_ipv6_dscp       => "true" | "false",                             #Optional / Default
value is "false"
     override_lan2_dscp            => "true" | "false",                             #Optional / Default
value is "false"
     override_lan2_ipv6_dscp       => "true" | "false",                             #Optional / Default
value is "false"
     override_mgmt_dscp            => "true" | "false",                             #Optional / Default
value is "false"
     override_mgmt_ipv6_dscp       => "true" | "false",                             #Optional / Default
value is "false"
     override_node1_mgmt_dscp      => "true" | "false",                             #Optional / Default
value is "false"
     override_node1_mgmt_ipv6_dscp => "true" | "false",                             #Optional / Default
value is "false"
     override_node2_mgmt_dscp      => "true" | "false",                             #Optional / Default
value is "false"
```

```
     override_node2_mgmt_ipv6_dscp => "true" | "false",                    #Optional / Default
value is "false"
     override_syslog_backup_servers => "true" | "false",                   #Optional / Default
value is "false"
     override_threshold_traps      => "true" | "false",                    #Optional / Default
value is "false"
     override_trap_notifications   => "true" | "false",                    #Optional / Default
value is "false"
     override_enable_lom      => "true" | "false",
#Optional / Default value is undefined
     override_lom_users       => "true" | "false",                         #Optional / Default
value is undefined
     passive_ha_arp_enabled   => "true" | "false",                         #Optional / Default
value is "false"
     platform                 => "Infoblox" | "Cisco" | "Riverbed" | "VNIOS",      #Optional / Default
value is "Infoblox"
     prefer_resolver          => $Nameserver | $Member | $ipv4addr | undef,        #Optional / Default
value is undefined
     query_comm_string        => $string | undef,                          #Optional / Default
value is undefined
     remote_console_access    => "true" | "false" | undef,                 #Optional / Default
value is undefined
     router_ID                => $num,                                     #Optional / Default
value is undefined
     service_type_configuration => "ALL_V4", "ALL_V6", "CUSTOM"            #Optional / Default
value depends from the member type.
     snmp_admin               => $Admin | undef,                           #Optional / Default
value is undefined
     static_routes            => [$Route1, $Route2, ...],                  #Optional / Default
value is empty
     support_access           => "true" | "false" | undef,                 #Optional / Default
value is undefined
     syslog_proxy_tcp_port    => $num,                                     #Optional / Default
value is 514
     syslog_proxy_udp_port    => $num,                                     #Optional / Default
value is 514
     syslog_server            => [$SyslogServer, ...] | undef,             #Optional / Default
value is undefined
     syslog_backup_servers    => [$SyslogBackupServer, ...] | undef,       #Optional / Default
value is undefined
     syslog_size              => $num,                                     #Optional / Default
value is 300
     time_zone                => $TimeZone | undef,                        #Optional / Default
value is undefined
     threshold_traps          => [$trap1, $trap2,...] | undef,             #Optional / Default
value is a list of all the traps
     trap_comm_string         => $string | undef,                          #Optional / Default
value is undefined
     trap_notifications       => [$notification1, $notification2,...] | undef,     #Optional / Default
value is a list of all the notifications
     trap_receiver            => [$ipv4addr, ...] | undef,                 #Optional / Default
value is undefined
     upgrade_group            => $string,                                  #Optional / Default
value is undefined
     vlan_id                  => $uint,                                    #Optional / Default
value is undefined
     vpn_mtu                  => $num,                                     #Optional / Default
value is 1450
     vpn_on_mgmt              => "true" | "false",                         #Optional / Default
value is "false"
     use_v4_vrrp              => "true" | "false",                         #Optional / Default
value depends from the member type.
     pre_provisioning         => $pp,                                      #optional / Default
value is undef. $pp is an Infoblox::Grid:Member::PreProvisioning object
     preserve_if_owns_delegation => "true" | "false"                       #Optional / Default
value is "false"
      );
```

You cannot set both extattrs and extensible_attributes attributes at the same time.

SESSION METHODS

This section describes all the methods in an Infoblox::Session module that you can apply to a grid member object.

Infoblox::Session->add( )

Use this method to add an object to the Infoblox appliance. See Infoblox::Session->add() for parameters and return values.

If you have applied an access control list to an object, you must restart services after you update the access control list for the changes to take effect. See Infoblox::Session->restart() method.

Example

```
#Construct an object
my $grid_member = Infoblox::Grid::Member->new(
    name                    => "hostname.com",
    ipv4addr                => "3.0.0.2",
    mask                    => "255.0.0.0",
    gateway                 => "3.0.0.1",
    );
#Submit for addition
my $response = $session->add( $grid_member );
```

Infoblox::Session->get( )

Use this method to retrieve all the matching objects from the Infoblox appliance. See Infoblox::Session->get() for parameters and return values.

Key References

```
Apply the following attributes to get a specific grid member object:

 name - Optional. A name in FQDN (Fully Qualified Domain Name) format.
 ipv4addr  - Optional. The IPv4 address of the member.
 ipv6addr  - Optional. The IPv6 address of the member.
 extattrs  - Optional. A hash reference containing extensible attributes.
```

Example

```
my @retrieved_objs = $session->get(
    object => "Infoblox::Grid::Member",
    name   => "hostname.com");
```

Infoblox::Session->modify( )

Use this method to modify an object in the Infoblox appliance. See Infoblox::Session->modify() for parameters and return values.

If you have applied an access control list to an object, you must restart services after you update the access control list for the changes to take effect. See Infoblox::Session->restart() method.

Example

```
# Use this method to modify comment of grid member.
$grid_member->comment("This is modified object");
# Submit modification
my $response = $session->modify( $grid_member );
```

Infoblox::Session->remove( )

Use this method to remove an object from the Infoblox appliance. See Infoblox::Session->remove() for parameters and return values.

To remove a specific object, first use `get()` or `search()` to retrieve the specific object, and then submit this object for removal.

Example

```
#Get the objects with the same name
my @retrieved_objs = $session->get(
    object => "Infoblox::Grid::Member",
    name   => "hostname.com");
#Find the desired object from the retrieved list.
my $desired_gridmember = $retrieved_objs[0];
# Submit for removal
my $response = $session->remove( $desired_gridmember );
```

Infoblox::Session->search( )

Use this method to search for grid member objects in the Infoblox appliance. See Infoblox::Session->search() for parameters and return values.

Key References

```
Apply the following attributes to search for a grid member object:

 name    - Optional. A name in FQDN (Fully Qualified Domain Name) format (regular expression).
 comment - Optional. A comment in string format (regular expression).
 extattrs     - Optional. A hash reference containing extensible attributes.
 extensible_attributes   - Optional. A hash reference containing extensible attributes.
 ipv4addr  - Optional. The IPv4 address of the member.
 ipv6addr  - Optional. The IPv6 address of the member.
```

For more information about searching extensible attributes, see Infoblox::Grid::ExtensibleAttributeDef/Searching Extensible Attributes.

Example

```
# search for all grid member objects
my @retrieved_objs = $session->search(
    object => "Infoblox::Grid::Member",
    name   => ".*\\.com");
```

METHODS

This section describes all the methods that you can use to configure and retrieve the attribute values of a grid member.

active_position( )

Use this method to retrieve the active server of a grid member.

Parameter

None

Returns

The method returns the attribute value.

Example

```
#Get active server
my $active_server = $grid_member->active_position();
```

additional_ip_list( )

Use this method to set or retrieve the additional_ip_list of a grid member. This list contains additional interface information that can be used on the member level.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Any object in the list with interface set to "LAN_HA" and ipv6 parameters will be mapped to the toplevel ipv6 methods (ipv6addr, ipv6_cidr, ipv6_enable_auto_config and ipv6_gateway). If such an object is already in the list and is removed, the values of those methods will be cleared.

Parameter

Valid value is an array reference that contains Infoblox::Grid::Member::Interface `object(s)` or IPv4 addresses in string format.

Note that Infoblox::Grid::Member::Interface `object(s)` with interface set to "MGMT" are not supported.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get additional ip list
my $additional_ip_list = $grid_member->additional_ip_list();
#Modify additional ip list
my $interface_obj = Infoblox::Grid::Member::Interface->new(
        subnet_mask => "255.255.255.255",
        anycast     => "false",
        comment     => "advanced ip config",
        ipv4addr    => "4.3.2.1",
        interface   => "LOOPBACK",
);
$grid_member->additional_ip_list([$interface_obj]);
```

allow_syslog_clients( )

Use this method to set or retrieve the allow_syslog_clients list of a grid member. This list specifies the clients where the syslog messages are logged. This is required only when enable_syslog_proxy is set to "true".

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

Valid value is an Infoblox::Grid::NamedACL object or an array reference that contains IPv4/IPv6 addresses or networks. To undefine allow_syslog_clients, you must disable the enable_syslog_proxy flag.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get allow syslog clients
my $allow_syslog_clients = $grid_member->allow_syslog_clients();
#Modify allow syslog clients
$grid_member->allow_syslog_clients(["10.10.10.1", "10.10.2.0/24"]);
#Use Infoblox::Grid::NamedACL
$grid_member->allow_syslog_clients($nacl);
```

alternate_resolver( )

Use this method to set or retrieve the alternate_resolver of a grid member. The alternate resolver receives queries from the appliance if it does not receive a response from the preferred name server first.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

The valid values are Infoblox::DNS::Member object or Infoblox::DNS::Nameserver object.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get alternate resolver
my $alternate_resolver = $grid_member->alternate_resolver();
#Modify alternate resolver
my $member1 = Infoblox::DNS::Member->new(
    name           => $host_name,
    ipv4addr       => $host_ip,
    stealth        => "false",
    lead           => "false",
    grid_replicate => "false"
);
$grid_member->alternate_resolver($member1);
#Undef alternate resolver
$grid_member->alternate_resolver(undef);
```

bgp_as( )

Use this method to set or retrieve the BGP configuration for anycast, of a grid member.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

The valid value is an array reference that contains Infoblox::Grid::Member::BGP::AS object(s). Currently, only one Autonomous System (AS) object per grid member is supported.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get AS list
my $as = $grid_member->bgp_as();
#Modify AS list
my $as1 = Infoblox::Grid::Member::BGP::AS->new(
    as             => 269,
    keepalive      => 4,
    holddown       => 16,
    neighbors      => [ $neighbor ]
);
```

```
$grid_member->bgp_as([$as1]);
```

comment( )

Use this method to set or retrieve a descriptive comment.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

Desired comment in string format with a maximum of 256 bytes.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get comment
my $comment = $grid_member->comment();
#Modify comment
$grid_member->comment("Modified grid member");
```

config_addr_type( )

Use this method to set or retrieve the configuration type for the address.

Parameter

The valid value is one of the following: "IPv4", "IPv6" or "BOTH".

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get config_addr_type
my $config_addr_type = $grid_member->config_addr_type();
#Modify config_addr_type
$grid_member->config_addr_type("IPv4");
```

dscp( )

Use this method to set or retrieve the DSCP value.

Setting this method to a defined value implicitly sets the override_dscp method to "true". Setting the parameter to undefined causes the appliance to use the grid default and automatically resets the override_dscp attribute to "false".

Note that when dscp is set to a defined value and override_ddscp is set to "false", the last operation takes precedence. Thus the sequence $object->dscp(11); $object->override_dscp("false"); will set override_dscp to "false", and the sequence $object->override_dscp("false"); $object->dscp(11); will result in override_dscp="true".

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

Valid values are integers between 0 and 63 inclusive. The default value is 0.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get the DSCP value from the member
my $users = $grid_member->dscp();
#Modify the DSCP value
$grid_member->dscp(11);
```

enable_ha( )

Use this method to set or retrieve the HA state of a member. It indicates whether a member is a single appliance or an HA pair.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

Specify "true" to set the enable_ha flag or "false" to deactivate/unset it. The default value is "false". If set to "false", then all the "node1_xxxx" and "node2_xxxx" attributes but node1_service_status and node1_hwtype are disabled, and "get" does not return those attributes.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get enable_ha from the member
my $enable_ha = $grid_member->enable_ha();
#Modify enable_ha
$grid_member->enable_ha("true");
```

enable_lom( )

Use this method to set or retrieve the flag that controls whether the LOM functionality is enabled.

Setting this method to a defined value implicitly sets the override_enable_lom method to "true". Setting the parameter to undefined causes the appliance to use the grid default and automatically resets the override_enable_lom attribute to "false".

Note that when enable_lom is set to a defined value and override_denable_lom is set to "false", the last operation takes precedence. Thus the sequence $object->enable_lom("true"); $object->override_enable_lom("false"); will set override_enable_lom to "false", and the sequence $object->override_enable_lom("false"); $object->enable_lom("true"); will result in override_enable_lom="true".

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

Specify "true" to set the enable_lom flag or "false" to deactivate/unset it. The default value is "false".

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get enable_lom from the member
my $lom = $grid_member->enable_lom();
#Modify enable_lom
$grid_member->enable_lom("true");
```

enable_member_redirect( )

Use this method to set or retrieve the flag that controls whether the member will redirect GUI connections to the grid master.

Setting this method to a defined value implicitly sets the override_enable_member_redirect method to "true". Setting the parameter to undefined causes the appliance to use the grid default and automatically resets the override_enable_member_redirect attribute to "false".

Note that when enable_member_redirect is set to a defined value and override_denable_member_redirect is set to "false", the last operation takes precedence. Thus the sequence $object->enable_member_redirect("true"); $object->override_enable_member_redirect("false"); will set override_enable_member_redirect to "false", and the sequence $object->override_enable_member_redirect("false"); $object->enable_member_redirect("true"); will result in override_enable_member_redirect="true".

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

Specify "true" to set the enable_member_redirect flag or "false" to deactivate/unset it. The default value is "false".

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get enable_member_redirect from the member
my $redirect = $grid_member->enable_member_redirect();
#Modify type of member
$grid_member->enable_member_redirect("true");
```

enable_ntp( )

Use this method to set or retrieve the enable_ntp flag of a grid member. Network Time Protocol (NTP) is a protocol designed to synchronize the clocks of computers over a network. This method enables/disables NTP service on the grid member.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

Specify "true" to set the enable_ntp flag or "false" to deactivate/unset it. The default value is "false".

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get enable ntp flag
my $enable_ntp = $grid_member->enable_ntp();
#Modify enable ntp flag
$grid_member->enable_ntp("true");
```

enable_snmpv3_query( )

Use this method to enable or disable SNMPv3 queries.

Parameter

Specify "true" to enable SNMPv3 queries or "false" to disable it.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

Example

```
#Get enable_snmpv3_query
my $enable_snmpv3_query = $gridmember->enable_snmpv3_query();
#Modify enable_snmpv3_query
$gridmember->enable_snmpv3_query("false");
```

enable_snmpv3_traps( )

Use this method to enable or disable SNMPv3 traps.

Parameter

Specify "true" to enable SNMPv3 traps or "false" to disable it. Default value is "false".

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

Example

```
#Get enable_snmpv3_traps
my $enable_snmpv3_traps = $gridmember->enable_snmpv3_traps();
#Modify enable_snmpv3_traps
$gridmember->enable_snmpv3_traps("true");
```

enable_ro_api_access( )

Use this method to enable or disable read-only API access for the Grid Master candidate.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

Specify "true" to allow read-only API access or "false" to forbid it. The default value is "false".

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get enable syslog proxy flag
my $enable_ro_api_access = $grid_member->enable_ro_api_access();
#Modify enable syslog proxy flag
$grid_member->enable_ro_api_access("true");
```

enable_syslog_proxy( )

Use this method to set or retrieve the enable_syslog_proxy flag of a grid member. When enabled, the member receives syslog messages from specified devices, such as syslog servers and routers, and then forwards these messages to an external syslog server.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

Specify "true" to set the enable_syslog_proxy flag or "false" to deactivate/unset it. The default value is "false".

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get enable syslog proxy flag
my $enable_syslog_proxy = $grid_member->enable_syslog_proxy();
#Modify enable syslog proxy flag
$grid_member->enable_syslog_proxy("true");
```

external_ntp_servers_enabled( )

Use this method to set or retrieve the external_ntp_servers_enabled flag of a grid member.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

When this flag is set to "false", the member does not use the configured list of NTP (Network Time Protocol) servers.

When this flag is set to "true", the member uses the configured list of NTP servers. If no NTP servers are configured on the member, it uses the NTP servers configured on the grid.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get external_ntp_servers_enabled flag
my $external_ntp_servers_enabled= $grid_member->external_ntp_servers_enabled();
#Modify external_ntp_servers_enabled flag
$grid_member->external_ntp_servers_enabled("true");
```

exclude_grid_master_as_ntp_server( )

Use this method to set or retrieve the exclude_grid_master_as_ntp_server flag of a grid member.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

When this flag is set to "true", the member does not use the grid master as an NTP (Network Time Protocol) time source. The default value of this parameter is "false", which means that by default the Grid Master is used as one of the time sources for Grid Member.

Note that setting this flag to "true" may have a detrimental effect on the database integrity of the member.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get exclude_grid_master_as_ntp_server flag
my $exclude_grid_master_as_ntp_server= $grid_member->exclude_grid_master_as_ntp_server();
#Modify exclude_grid_master_as_ntp_server flag
$grid_member->exclude_grid_master_as_ntp_server("true");
```

extattrs( )

Use this method to set or retrieve the extensible attributes associated with a Grid Member object.

Parameter

Valid value is a hash reference containing the names of extensible attributes and their associated values (Infoblox::Grid::Extattr objects).

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get extattrs
my $ref_extattrs = $grid_member->extattrs();
#Modify extattrs
$grid_member->extattrs({ 'Site' => $extattr1, 'Administrator' => $extattr2 });
```

extensible_attributes( )

Use this method to set or retrieve the extensible attributes associated with a Grid Member object

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

For valid values for extensible attributes, see Infoblox::Grid::ExtensibleAttributeDef/Extensible Attribute Values.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get extensible attributes
my $ref_extensible_attributes = $member->extensible_attributes();
#Modify extensible attributes
$member->extensible_attributes({ 'Site' => 'Santa Clara', 'Administrator' => [ 'Peter', 'Tom' ] });
```

gateway( )

Use this method to set or retrieve the gateway IP address of a grid member.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

An IPv4 address is a 32-bit number in dotted decimal notation. It consists of four 8-bit groups of decimal digits separated by decimal points (example: 192.168.1.2).

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get gateway
my $gateway = $grid_member->gateway();
#Modify gateway
$grid_member->gateway("3.0.0.1");
```

ha_node2_port_duplex( )

Use this method to specify or retrieve the duplex setting of the HA port on node 2 of an HA grid member. Note: Use this method in combination with ha_node2_port_speed. If any of these attributes are undefined, the default values for duplex and speed settings are used.

Parameter

Valid values are "Half" or "Full". The default value is undefined.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get ha_node2_port_duplex
my $ha_node2_port_duplex = $obj->ha_node2_port_duplex();
#Modify ha_node2_port_duplex
$obj->ha_node2_port_duplex("Half");
```

ha_node2_port_speed( )

Use this method to set or retrieve the connection speed of the HA port on node #2 of an HA grid member. Note: Use this method in combination with ha_node2_port_duplex. If any of these attributes are undefined, the default values for duplex and speed settings are used.

Parameter

Valid values are 10, 100 or 1000. The default value is undefined.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get ha_node2_port_speed
my $ha_node2_port_speed = $obj->ha_node2_port_speed();
#Modify ha_node2_port_speed
$obj->ha_node2_port_speed(10);
```

ha_port_duplex( )

Use this method to specify or retrieve the duplex setting of the HA port on node 1 of an HA grid member. Note: Use this method in combination with ha_port_speed. If any of these attributes are undefined, the default values for duplex and speed settings are used.

Parameter

Valid values are "Half" or "Full". The default value is undefined.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get ha_port_duplex
my $ha_port_duplex = $obj->ha_port_duplex();
#Modify ha_port_duplex
$obj->ha_port_duplex("Half");
```

ha_port_speed( )

Use this method to set or retrieve the connection speed of the HA port on node 1 of an HA grid member. Note: Use this method in combination with ha_port_duplex. If any of these attributes are undefined, the default values for duplex and speed settings are used.

Parameter

Valid values are 10, 100 or 1000. The default value is undefined.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get ha_port_speed
my $ha_port_speed = $obj->ha_port_speed();
#Modify ha_port_speed
$obj->ha_port_speed(10);
```

ha_reserved_interface( )

Use this method to set or retrieve the discovery reserved interface of the HA port on node 1 of an HA grid member.

Parameter

Valid value is an Infoblox::Grid::Discovery::DeviceInterface object.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get attribute value
my $attr = $grid_member->ha_reserved_interface();
#Modify attribute value
$grid_member->ha_reserved_interface($interface);
```

paid_nios( )

Use this method to retrieve whether the grid member is Paid NIOS.

Parameter

None

Returns

The method returns the attribute value.

Example

```
#Get Paid NIOS status
my $paid_nios = $grid_member->paid_nios();
```

hwplatform( )

Use this method to retrieve the hardware platform of the member appliance.

Parameter

None

Returns

The method returns the attribute value.

Example

```
#Get hardware platform
my $hwplatform = $grid_member->hwplatform();
```

hwid( )

Use this method to retrieve the hardware ID of the member appliance.

Parameter

None

Returns

The method returns the attribute value.

Example

```
#Get hardware ID
my $hwid = $grid_member->hwid();
```

is_dscp_capable( )

Use this method to retrieve the flag that describes whether a Grid member supports DSCP.

Parameter

The method retruns "true" if the member supports DSCP, and "false" otherwise.

Returns

The method returns the attribute value.

Example

```
#Get is_dscp_capable
my $is_dscp_capable = $grid_member->is_dscp_capable();
```

ipv4addr( )

Use this method to set or retrieve the IPv4 address of a grid member.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

An IPv4 address is a 32-bit number in dotted decimal notation. It consists of four 8-bit groups of decimal digits separated by decimal points (example: 192.168.1.2).

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get ipv4addr
my $ipv4addr = $grid_member->ipv4addr();
#Modify ipv4addr
$grid_member->ipv4addr("3.0.0.3");
```

ipv6addr( )

Use this method to set or retrieve the LAN IPv6 address of the grid member.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Setting any of the ipv6 methods in the member (ipv6addr, ipv6_cidr, ipv6_enable_auto_config and ipv6_gateway) is equivalent to creating an Infoblox::Grid::Member::Interface object with those fields set and interface set to "LAN_HA", if there is already such an object it will be modified, if not it will be created. See the additional_ip_list section for more information.

Parameter

An IPv6 address is a 128-bit number represented as eight groups of four hexadecimal digits (i.e. two octets), each group separated by a `colon(:)` (example: 2001:0db8:85a3:0000:0000:8a2e:0370:7334). Leading zeroes and groups of consecutive zeros within the value can be omitted to produce a more compact representation (example: 2001::1, ::1).

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get LAN2 IPv6 address
my $ipv6addr = $grid_member->ipv6addr();
#Modify LAN2 IPv6 address
$grid_member->ipv6addr("2001::2");
```

ipv6_cidr( )

Use this method to set or retrieve the CIDR of the grid member. Required only when ipv6addr is specified.

See ipv6addr for more information on how this impacts additional_ip_list.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

Specify the netmask in CIDR format.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get LAN2 cidr
my $cidr = $grid_member->cidr();
#Modify LAN2 cidr
$grid_member->cidr("64");
```

ipv6_enable_auto_config( )

Use this method to enable/disable IPv6 auto-configuration of the grid member.

See ipv6addr for more information on how this impacts additional_ip_list.

When IPv6 auto-configuration is enabled, a gateway is not needed since it will be automatically discovered.

Parameter

Specify "true" to enable IPv6 auto-configuration or "false" to disable it. Default value is "false".

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

Example

```
#Get IPv6 auto-configuration
my $ipv6_autoconfig_enabled = $grid_member->ipv6_enable_auto_config();
#Modify IPv6 auto-configuration
$grid_member->ipv6_enable_auto_config("true");
```

ipv6_gateway( )

Use this method to set or retrieve the LAN IPv6 gateway of the grid member.

See ipv6addr for more information on how this impacts additional_ip_list.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

An IPv6 address is a 128-bit number represented as eight groups of four hexadecimal digits (i.e. two octets), each group separated by a `colon(:)` (example: 2001:0db8:85a3:0000:0000:8a2e:0370:7334). Leading zeroes and groups of consecutive zeros within the value can be omitted to produce a more compact representation (example: 2001::1, ::1).

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get LAN2 IPv6 gateway
my $ipv6_gateway = $grid_member->ipv6_gateway();
#Modify LAN2 IPv6 gateway
$grid_member->ipv6_gateway("2001::1");
```

ipv6_static_routes( )

Use this method to set or retrieve the IPv6 static routes list of a grid member. This list contains the static routes of a grid member.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

Valid value is an array reference that contains Infoblox::Grid::Member::IPv6StaticRoute object(s).

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get static routes list
my $ipv6_static_routes = $grid_member->ipv6_static_routes();
#Modify static routes list
my $route = Infoblox::Grid::Member::IPv6StaticRoute->new(
        "address" => "2002::5",
        "cidr"    => 64
        "gateway" => "2002::1",
);
$grid_member->ipv6_static_routes([$route]);
```

ipv6_vlan_id( )

Use this method to set or retrieve the IPv6 VLAN ID of a grid member.

Parameter

Valid values are unsigned integers between 1 and 4095, inclusive. The default value is undefined.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get attribute value
my $attr = $grid_member->ipv6_vlan_id();
#Modify attribute value
$grid_member->ipv6_vlan_id(10);
```

lan_reserved_interface( )

Use this method to set or retrieve the discovery reserved interface of the LAN port on a single grid member or on node 1 of an HA grid member.

Parameter

Valid value is an Infoblox::Grid::Discovery::DeviceInterface object.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get attribute value
my $attr = $grid_member->lan_reserved_interface();
#Modify attribute value
$grid_member->lan_reserved_interface($interface);
```

lan1_dscp( )

Use this method to set or retrieve the DSCP value of the IPv4 LAN1 port.

Setting this method to a defined value implicitly sets the override_lan1_dscp method to "true". Setting the parameter to undefined causes the appliance to use the grid default and automatically resets the override_lan1_dscp attribute to "false".

Note that when dscp is set to a defined value and override_lan1_dscp is set to "false", the last operation takes precedence. Thus the sequence $object->lan1_dscp(11); $object->override_lan1_dscp("false"); will set override_lan1_dscp to "false", and the sequence $object->override_lan1_dscp("false"); $object->lan1_dscp(11); will result in override_lan1_dscp="true".

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

Valid values are integers between 0 and 63, inclusive. The default value is undefined.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get attribute value
my $value = $grid_member->lan1_dscp();
#Modify attribute value
$grid_member->lan1_dscp(11);
```

lan1_ipv6_dscp( )

Use this method to set or retrieve the DSCP value of the IPv6 LAN1 port.

Setting this method to a defined value implicitly sets the override_lan1_ipv6_dscp method to "true". Setting the parameter to undefined causes the appliance to use the grid default and automatically resets the override_lan1_ipv6_dscp attribute to "false".

Note that when dscp is set to a defined value and override_lan1_ipv6_dscp is set to "false", the last operation takes precedence. Thus the sequence $object->lan1_ipv6_dscp(11); $object->override_lan1_ipv6_dscp("false"); will set override_lan1_ipv6_dscp to "false", and the sequence $object->override_lan1_ipv6_dscp("false"); $object->lan1_ipv6_dscp(11); will result in override_lan1_ipv6_dscp="true".

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

Valid values are integers between 0 and 63, inclusive. The default value is undefined.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get attribute value
my $value = $grid_member->lan1_ipv6_dscp();
#Modify attribute value
$grid_member->lan1_ipv6_dscp(11);
```

lan2_dscp( )

Use this method to set or retrieve the DSCP value of the IPv4 LAN2 port.

Setting this method to a defined value implicitly sets the override_lan1_dscp method to "true". Setting the parameter to undefined causes the appliance to use the grid default and automatically resets the override_lan1_dscp attribute to "false".

Note that when dscp is set to a defined value and override_lan2_dscp is set to "false", the last operation takes precedence. Thus the sequence $object->lan2_dscp(11); $object->override_lan2_dscp("false"); will set override_lan2_dscp to "false", and the sequence $object->override_lan2_dscp("false"); $object->lan2_dscp(11); will result in override_lan2_dscp="true".

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

Valid values are integers between 0 and 63, inclusive. The default value is undefined.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get attribute value
my $value = $grid_member->lan2_dscp();
#Modify attribute value
$grid_member->lan2_dscp(11);
```

lan2_gateway( )

Use this method to set or retrieve the gateway IP address of the LAN2 port. Note that to configure the LAN2 port, the lan2_port flag must

be set to 'true'.

lan2_ipv4addr, lan2_mask, lan2_gateway attributes are required if one of them is defined.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

An IPv4 address is a 32-bit number in dotted decimal notation. It consists of four 8-bit groups of decimal digits separated by decimal points (example: 192.168.1.2).

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get LAN2 gateway
my $lan2_gateway = $grid_member->lan2_gateway();
#Modify LAN2 gateway
$grid_member->lan2_gateway("4.0.0.1");
```

lan2_ipv4addr( )

Use this method to set or retrieve the LAN IP address of the LAN2 port. Note that to configure the LAN2 port, the lan2_port flag must be set to 'true'.

lan2_gateway, lan2_ipv4addr, lan2_mask attributes are required if one of them is defined.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

An IPv4 address is a 32-bit number in dotted decimal notation. It consists of four 8-bit groups of decimal digits separated by decimal points (example: 192.168.1.2).

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get LAN2 IPv4 address
my $lan2_ipv4addr = $grid_member->lan2_ipv4addr();
#Modify LAN2 IPv4 address
$grid_member->lan2_ipv4addr("4.0.0.2");
```

lan2_ipv6addr( )

Use this method to set or retrieve the LAN IPv6 address of the LAN2 port. Note that to configure the LAN2 port, the lan2_port flag must be set to 'true'.

lan2_ipv6addr, lan2_ipv6_cidr, lan2_ipv6_gateway attributes are required if one of them is defined.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

An IPv6 address is a 128-bit number represented as eight groups of four hexadecimal digits (i.e. two octets), each group separated by a `colon(:)` (example: 2001:0db8:85a3:0000:0000:8a2e:0370:7334). Leading zeroes and groups of consecutive zeros within the value can be omitted to produce a more compact representation (example: 2001::1, ::1).

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get LAN2 IPv6 address
my $lan2_ipv6addr = $grid_member->lan2_ipv6addr();
#Modify LAN2 IPv6 address
$grid_member->lan2_ipv6addr("2001::2");
```

lan2_ipv6_cidr( )

Use this method to set or retrieve the CIDR of the LAN2 port. Note that to configure the LAN2 port, the lan2_port flag must be set to 'true'.

lan2_ipv6addr, lan2_ipv6_cidr, lan2_ipv6_gateway attributes are required if one of them is defined.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

Specify the netmask in CIDR format.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get LAN2 cidr
my $lan2_cidr = $grid_member->lan2_cidr();
#Modify LAN2 cidr
$grid_member->lan2_cidr("64");
```

lan2_ipv6_dscp( )

Use this method to set or retrieve the DSCP value of the IPv6 LAN2 port.

Setting this method to a defined value implicitly sets the override_lan2_ipv6_dscp method to "true". Setting the parameter to undefined causes the appliance to use the grid default and automatically resets the override_lan2_ipv6_dscp attribute to "false".

Note that when dscp is set to a defined value and override_lan2_ipv6_dscp is set to "false", the last operation takes precedence. Thus the sequence $object->lan2_ipv6_dscp(11); $object->override_lan2_ipv6_dscp("false"); will set override_lan1_ipv6_dscp to "false", and the sequence $object->override_lan2_ipv6_dscp("false"); $object->lan2_ipv6_dscp(11); will result in override_lan2_ipv6_dscp="true".

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

Valid values are integers between 0 and 63, inclusive. The default value is undefined.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get attribute value
my $value = $grid_member->lan2_ipv6_dscp();
#Modify attribute value
$grid_member->lan2_ipv6_dscp(11);
```

lan2_ipv6_enable( )

Use this method to enable/disable IPv6 configuration of the LAN2 interface.

Parameter

Specify "true" to enable IPv6 configuration or "false" to disable it. Default value is "false".

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get attribute value
my $attr = $grid_member->lan2_ipv6_enable();
#Modify attribute value
$grid_member->lan2_ipv6_enable("true");
```

lan2_ipv6_enable_auto_config( )

Use this method to enable/disable IPv6 auto-configuration of the LAN2 interface.

When IPv6 auto-configuration is enabled, a gateway is not needed since it will be automatically discovered.

Parameter

Specify "true" to enable IPv6 auto-configuration or "false" to disable it. Default value is "false".

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get IPv6 auto-configuration
my $ipv6_autoconfig_enabled = $grid_member->lan2_ipv6_enable_auto_config();
#Modify IPv6 auto-configuration
$grid_member->lan2_ipv6_enable_auto_config("true");
```

lan2_ipv6_gateway( )

Use this method to set or retrieve the LAN IPv6 gateway of the LAN2 port. Note that to configure the LAN2 port, the lan2_port flag must be set to 'true'.

lan2_ipv6addr, lan2_ipv6_cidr, lan2_ipv6_gateway attributes are required if one of them is defined.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

An IPv6 address is a 128-bit number represented as eight groups of four hexadecimal digits (i.e. two octets), each group separated by a `colon(:)` (example: 2001:0db8:85a3:0000:0000:8a2e:0370:7334). Leading zeroes and groups of consecutive zeros within the value can be omitted to produce a more compact representation (example: 2001::1, ::1).

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get LAN2 IPv6 gateway
my $lan2_ipv6_gateway = $grid_member->lan2_ipv6_gateway();
#Modify LAN2 IPv6 gateway
$grid_member->lan2_ipv6_gateway("2001::1");
```

lan2_ipv6_vlan_id( )

Use this method to set or retrieve the VLAN ID of the IPv6 LAN2 port. Note that to configure the LAN2 port, the lan2_port flag must be set to 'true'.

Parameter

Valid values are unsigned integers between 1 and 4095, inclusive. The default value is undefined.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get attribute value
my $attr = $grid_member->lan2_ipv6_vlan_id();
#Modify attribute value
$grid_member->lan2_ipv6_vlan_id(10);
```

lan2_mask( )

Use this method to set or retrieve the netmask of the LAN2 port. Note that to configure the LAN2 port, the lan2_port flag must be set to 'true'.

lan2_ipv4addr, lan2_mask, lan2_gateway attributes are required if one of them is defined.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

Netmask address is a 32-bit number in dotted decimal notation. It consists of four 8-bit groups of decimal digits separated by decimal points (example: 255.255.0.0).

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get LAN2 mask
my $lan2_mask = $grid_member->lan2_mask();
#Modify LAN2 mask
$grid_member->lan2_mask("255.255.0.0");
```

lan2_node2_port_duplex( )

Use this method to specify or retrieve the duplex setting of the LAN2 port on node 2 of an HA grid member. Note: Use this method in combination with lan2_node2_port_speed. If any of these attributes are undefined, the default values for duplex and speed settings are used.

Parameter

Valid values are "Half" or "Full". The default value is undefined.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get lan2_node2_port_duplex
my $lan2_node2_port_duplex = $obj->lan2_node2_port_duplex();
#Modify lan2_node2_port_duplex
$obj->lan2_node2_port_duplex("Half");
```

lan2_port( )

Use this method to set or retrieve the LAN2 port flag of a single grid member or node 1 of an HA grid member.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

Specify "true" to set the lan2_port flag or "false" to deactivate/unset it. The default value is "false".

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get LAN2 port flag
my $lan2_port = $grid_member->lan2_port();
#Modify LAN2 port flag
$grid_member->lan2_port("true");
```

lan2_node2_port_speed( )

Use this method to set or retrieve the connection speed of the LAN2 port on node 2 of an HA grid member. Note: Use this method in combination with lan2_node2_port_duplex. If any of these attributes are undefined, the default values for duplex and speed settings are used.

Parameter

Valid values are 10, 100 or 1000. The default value is undefined.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get lan2_node2_port_speed
my $lan2_node2_port_speed = $obj->lan2_node2_port_speed();
#Modify lan2_node2_port_speed
$obj->lan2_node2_port_speed(10);
```

lan2_port_duplex( )

Use this method to specify or retrieve the duplex setting of the LAN2 port on a single grid member or on node 1 of an HA grid member. Note: Use this method in combination with lan2_port_speed. If any of these attributes are undefined, the default values for duplex and speed settings are used.

Parameter

Valid values are "Half" or "Full". The default value is undefined.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get lan2_port_duplex
my $lan2_port_duplex = $obj->lan2_port_duplex();
#Modify lan2_port_duplex
$obj->lan2_port_duplex("Half");
```

lan2_port_speed( )

Use this method to set or retrieve the connection speed of the LAN2 port on a single grid member or on node 1 of an HA grid member. Note: Use this method in combination with lan2_port_duplex. If any of these attributes are undefined, the default values for duplex and speed settings are used.

Parameter

Valid values are 10, 100 or 1000. The default value is undefined.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get lan2_port_speed
my $lan2_port_speed = $obj->lan2_port_speed();
#Modify lan2_port_speed
$obj->lan2_port_speed(10);
```

lan2_reserved_interface( )

Use this method to set or retrieve the discovery reserved interface of the LAN2 port on a single grid member or on node 1 of an HA grid member.

Parameter

Valid value is an Infoblox::Grid::Discovery::DeviceInterface object.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get attribute value
my $attr = $grid_member->lan2_reserved_interface();
#Modify attribute value
$grid_member->lan2_reserved_interface($interface);
```

lan2_router_id( )

Use this method to set or retrieve the LAN2 virtual router ID of a grid member. Required only when the member is an HA pair.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

A number with the LAN2 router identification. Valid values are from 1 through 255.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get LAN2 virtual router ID
my $lan2_router_id = $grid_member->lan2_router_id();
#Modify LAN2 virtual router ID
$grid_member->lan2_router_id("123");
```

lan2_vlan_id( )

Use this method to set or retrieve the VLAN ID of the IPv4 LAN2 port. Note that to configure the LAN2 port, the lan2_port flag must be set to 'true'.

Parameter

Valid values are unsigned integers between 1 and 4095, inclusive. The default value is undefined.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get attribute value
my $attr = $grid_member->lan2_vlan_id();
#Modify attribute value
$grid_member->lan2_vlan_id(10);
```

lan_node2_port_duplex( )

Use this method to specify or retrieve the duplex setting of the LAN port on node 2 of an HA grid member. Note: Use this method in combination with lan_node2_port_speed. If any of these attributes are undefined, the default values for duplex and speed settings are used.

Parameter

Valid values are "Half" or "Full". The default value is undefined.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get lan_node2_port_duplex
my $lan_node2_port_duplex = $obj->lan_node2_port_duplex();
#Modify lan_node2_port_duplex
$obj->lan_node2_port_duplex("Half");
```

lan_node2_port_speed( )

Use this method to set or retrieve the connection speed of the LAN port on node 2 of an HA grid member. Note: Use this method in combination with lan_node2_port_duplex. If any of these attributes are undefined, the default values for duplex and speed settings are used.

Parameter

Valid values are 10, 100 or 1000. The default value is undefined.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get lan_node2_port_speed
my $lan_node2_port_speed = $obj->lan_node2_port_speed();
#Modify lan_node2_port_speed
$obj->lan_node2_port_speed(10);
```

lan_port_duplex( )

Use this method to specify or retrieve the duplex setting of the LAN port on a single grid member or on node 1 of an HA grid member.. Note: Use this method in combination with lan_port_speed. If any of these attributes are undefined, the default values for duplex and speed settings are used.

Parameter

Valid values are "Half" or "Full". The default value is undefined.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get lan_port_duplex
my $lan_port_duplex = $obj->lan_port_duplex();
#Modify lan_port_duplex
$obj->lan_port_duplex("Half");
```

lan_port_speed( )

Use this method to set or retrieve the connection speed of the LAN port on a single grid member or on node 1 of an HA grid member.. Note: Use this method in combination with lan_port_duplex. If any of these attributes are undefined, the default values for duplex and speed settings are used.

Parameter

Valid values are 10, 100 or 1000. The default value is undefined.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get lan_port_speed
my $lan_port_speed = $obj->lan_port_speed();
#Modify lan_port_speed
$obj->lan_port_speed(10);
```

lcd_input( )

Use this method to set or retrieve the lcd_input flag of a grid member. This flag enables/disables the Liquid Crystal Display (LCD) input buttons on the front panel of the appliance.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

Specify "true" to set the lcd_input flag or "false" to deactivate/unset it. The default value is undefined.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get lcd input flag
```

```
my $lcd_input = $grid_member->lcd_input();
#Modify lcd input flag
$grid_member->lcd_input("true");
#Undef lcd input flag
$grid_member->lcd_input(undef);
```

lcd_autodim( )

Use this method to set or retrieve the number of seconds before the Liquid Crystal Display (LCD) on the front panel of the appliance automatically dims.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

The number of seconds before the LCD automatically dims. Valid values are between 5 and 3600. The default value is 30.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get lcd_autodim
my $lcd_autodim = $grid_member->lcd_autodim();
#Modify the autodim value
$grid_member->lcd_autodim(60);
```

lcd_brightness( )

Use this method to set or retrieve the brightness level of the LCD on the front panel of the appliance.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

Valid values are between 1 and 10. The default value is 7.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get lcd_brightness
my $lcd_brightness = $grid_member->lcd_brightness();
#Modify the brightness value
$grid_member->lcd_brightness(3);
```

lcd_hwident( )

Use this method to set or retrieve whether the grid member's hardware ID identification functionality is active.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

Specify "true" to set the lcd_hwident flag or "false" to deactivate/unset it. The default value is "false".

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get the lcd_hwident status
my $lcd_hwident = $grid_member->lcd_hwident();
#Modify the lcd_hwident status
$grid_member->lcd_hwident("true");
```

lcd_version( )

Use this method to retrieve the version of the Liquid Crystal Display (LCD) of the appliance.

Parameter

None.

Returns

The method returns the attribute value.

Example

```
#Get lcd_version
my $lcd_version = $grid_member->lcd_version();
```

lom_gateway( )

Use this method to set or retrieve the gateway IP address of a grid member for the LOM functionality.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

An IPv4 address is a 32-bit number in dotted decimal notation. It consists of four 8-bit groups of decimal digits separated by decimal points (example: 192.168.1.2).

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get lom_gateway
my $lom_gateway = $grid_member->lom_gateway();
#Modify lom_gateway
$grid_member->lom_gateway("3.0.0.1");
```

lom_ipv4addr( )

Use this method to set or retrieve the IPv4 address of a grid member for the LOM functionality.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

An IPv4 address is a 32-bit number in dotted decimal notation. It consists of four 8-bit groups of decimal digits separated by decimal points (example: 192.168.1.2).

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get lom_ipv4addr
my $lom_ipv4addr = $grid_member->lom_ipv4addr();
#Modify lom_ipv4addr
$grid_member->lom_ipv4addr("3.0.0.3");
```

lom_is_capable( )

Use this method to retrieve a flag describing if a grid member is capable of the LOM functionality.

Parameter

The method returns "true" if the member is capable of LOM operation, "false" otherwise.

Returns

The method returns the attribute value.

Example

```
#Get lom_is_capable
my $lom_is_capable = $grid_member->lom_is_capable();
```

lom_mask( )

Use this method to set or retrieve the netmask of a grid member for the LOM functionality.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

Netmask address is a 32-bit number in dotted decimal notation. It consists of four 8-bit groups of decimal digits separated by decimal points (example: 255.255.0.0).

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get lom_mask
my $lom_mask = $grid_member->lom_mask();
#Modify lom_mask
$grid_member->lom_mask("255.255.0.0");
```

lom_users( )

Use this method to set or retrieve the list of users for the LOM functionality.

Setting this method to a defined value implicitly sets the override_lom_users method to "true". Setting the parameter to undefined causes the appliance to use the grid default and automatically resets the override_lom_users attribute to "false".

Note that when lom_users is set to a defined value and override_dlom_users is set to "false", the last operation takes precedence. Thus the sequence $object->lom_users([$user1,$user2]); $object->override_lom_users("false"); will set override_lom_users to "false", and the sequence $object->override_lom_users("false"); $object->lom_users([$user1,$user2]); will result in override_lom_users="true".

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

Valid value is an array reference that contains Infoblox::Grid::LOM::User objects.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get lom_users from the member
my $users = $grid_member->lom_users();
#Modify the lom users
$grid_member->lom_users([$u1, $u2]);
```

mask( )

Use this method to set or retrieve the netmask of a grid member.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

Netmask address is a 32-bit number in dotted decimal notation. It consists of four 8-bit groups of decimal digits separated by decimal points (example: 255.255.0.0).

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get mask
my $mask = $grid_member->mask();
#Modify mask
$grid_member->mask("255.255.0.0");
```

master_candidate( )

Use this method to set or retrieve the master_candidate flag of a grid member. This flag enables the grid member to assume the role of grid master as a disaster recovery measure.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

Specify "true" to set the master_candidate flag or "false" to deactivate/unset it. The default value is "false".

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get master candidate flag
my $master_candidate = $grid_member->master_candidate();
#Modify master candidate flag
$grid_member->master_candidate("true");
```

member_service_communication( )

Use this method to set or retrieve the communication type for various services.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

The valid value is an array reference that contains Infoblox::Grid::Member::Communication objects.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
# Get the attribute value
my $attr = $grid_member->member_service_communication();
# Modify the attribute value
$grid_member->member_service_communication([$c1, $c2, $c3]);
```

mgmt_dscp( )

Use this method to set or retrieve the DSCP value of the IPv4 MGMT port.

Setting this method to a defined value implicitly sets the override_mgmt_dscp method to "true". Setting the parameter to undefined causes the appliance to use the grid default and automatically resets the override_mgmt_dscp attribute to "false".

Note that when dscp is set to a defined value and override_mgmt_dscp is set to "false", the last operation takes precedence. Thus the sequence $object->mgmt_dscp(11); $object->override_mgmt_dscp("false"); will set override_mgmt_dscp to "false", and the sequence $object->override_mgmt_dscp("false"); $object->mgmt_dscp(11); will result in override_mgmt_dscp="true".

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

Valid values are integers between 0 and 63, inclusive. The default value is undefined.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get attribute value
my $value = $grid_member->mgmt_dscp();
#Modify attribute value
$grid_member->mgmt_dscp(11);
```

mgmt_gateway( )

Use this method to set or retrieve the gateway IP address of the MGMT port.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

An IPv4 address is a 32-bit number in dotted decimal notation. It consists of four 8-bit groups of decimal digits separated by decimal points (example: 192.168.1.2).

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get mgmt gateway
my $mgmt_gateway = $grid_member->mgmt_gateway();
#Modify mgmt gateway
$grid_member->mgmt_gateway("4.0.0.1");
```

mgmt_lan( )

Use this method to set or retrieve the LAN IP address of the MGMT port.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

An IPv4 address is a 32-bit number in dotted decimal notation. It consists of four 8-bit groups of decimal digits separated by decimal points (example: 192.168.1.2).

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get mgmt lan
my $mgmt_lan = $grid_member->mgmt_lan();
#Modify mgmt lan
$grid_member->mgmt_lan("4.0.0.2");
```

mgmt_ipv6addr( )

Use this method to set or retrieve the LAN IPv6 address of the MGMT port.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

An IPv6 address is a 128-bit number represented as eight groups of four hexadecimal digits (i.e. two octets), each group separated by a `colon(:)` (example: 2001:0db8:85a3:0000:0000:8a2e:0370:7334). Leading zeroes and groups of consecutive zeros within the value can be omitted to produce a more compact representation (example: 2001::1, ::1).

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get MGMT IPv6 address
my $mgmt_ipv6addr = $grid_member->mgmt_ipv6addr();
#Modify MGMT IPv6 address
$grid_member->mgmt_ipv6addr("2001::2");
```

mgmt_ipv6_cidr( )

Use this method to set or retrieve the CIDR of the MGMT port. Required only when mgmt_ipv6addr is specified.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

Specify the netmask in CIDR format.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get MGMT cidr
my $mgmt_cidr = $grid_member->mgmt_cidr();
#Modify MGMT cidr
$grid_member->mgmt_cidr("64");
```

mgmt_ipv6_dscp( )

Use this method to set or retrieve the DSCP value of the IPv6 MGMT port.

Setting this method to a defined value implicitly sets the override_mgmt_ipv6_dscp method to "true". Setting the parameter to undefined causes the appliance to use the grid default and automatically resets the override_mgmt_ipv6_dscp attribute to "false".

Note that when dscp is set to a defined value and override_mgmt_ipv6_dscp is set to "false", the last operation takes precedence. Thus the sequence $object->mgmt_ipv6_dscp(11); $object->override_mgmt_ipv6_dscp("false"); will set override_mgmt_ipv6_dscp to "false", and the sequence $object->override_mgmt_ipv6_dscp("false"); $object->mgmt_ipv6_dscp(11); will result in override_mgmt_ipv6_dscp="true".

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

Valid values are integers between 0 and 63, inclusive. The default value is undefined.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get attribute value
my $value = $grid_member->mgmt_ipv6_dscp();
#Modify attribute value
$grid_member->mgmt_ipv6_dscp(11);
```

mgmt_ipv6_enable_auto_config( )

Use this method to enable/disable IPv6 auto-configuration of the MGMT interface.

When IPv6 auto-configuration is enabled, a gateway is not needed since it will be automatically discovered.

Parameter

Specify "true" to enable IPv6 auto-configuration or "false" to disable it. Default value is "false".

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

Example

```
#Get IPv6 auto-configuration
my $ipv6_autoconfig_enabled = $grid_member->mgmt_ipv6_enable_auto_config();
#Modify IPv6 auto-configuration
$grid_member->mgmt_ipv6_enable_auto_config("true");
```

mgmt_ipv6_gateway( )

Use this method to set or retrieve the LAN IPv6 gateway of the MGMT port.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

An IPv6 address is a 128-bit number represented as eight groups of four hexadecimal digits (i.e. two octets), each group separated by a `colon(:)` (example: 2001:0db8:85a3:0000:0000:8a2e:0370:7334). Leading zeroes and groups of consecutive zeros within the value can be omitted to produce a more compact representation (example: 2001::1, ::1).

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get MGMT IPv6 gateway
my $mgmt_ipv6_gateway = $grid_member->mgmt_ipv6_gateway();
#Modify MGMT IPv6 gateway
$grid_member->mgmt_ipv6_gateway("2001::1");
```

mgmt_mask( )

Use this method to set or retrieve the netmask of the MGMT port.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

Netmask address is a 32-bit number in dotted decimal notation. It consists of four 8-bit groups of decimal digits separated by decimal points (example: 255.255.0.0).

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get mgmt mask
my $mgmt_mask = $grid_member->mgmt_mask();
#Modify mgmt mask
$grid_member->mgmt_mask("255.255.0.0");
```

mgmt_node2_port_duplex( )

Use this method to specify the duplex setting of the MGMT port on node 2 of an HA pair. Note: Use this method in combination with mgmt_node2_port_speed. If any of these attributes are undefined, the default values for duplex and speed settings are used.

Parameter

Valid values are "Half" or "Full". The default value is undefined.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get mgmt_node2_port_duplex
my $mgmt_node2_port_duplex = $obj->mgmt_node2_port_duplex();
#Modify mgmt_node2_port_duplex
$obj->mgmt_node2_port_duplex("Half");
```

mgmt_node2_port_speed( )

Use this method to set the connection speed of the MGMT port on node 2 of an HA pair. Note: Use this method in combination with mgmt_node2_port_duplex. If any of these attributes are undefined, the default values for duplex and speed settings are used.

Parameter

Valid values are 10, 100 or 1000. The default value is undefined.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get mgmt_node2_port_speed
my $mgmt_node2_port_speed = $obj->mgmt_node2_port_speed();
#Modify mgmt_node2_port_speed
```

```
$obj->mgmt_node2_port_speed(10);
```

mgmt_port( )

Use this method to set or retrieve the MGMT port flag of a grid member. You can restrict administrative access to an Infoblox appliance by connecting the MGMT port to a subnet containing only management systems.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Note that node1_mgmt_port, node2_mgmt_port and mgmt_port share the value; setting any of them will have the same effect.

Parameter

Specify "true" to set the mgmt_port flag or "false" to deactivate/unset it. The default value is "false". To disable the mgmt_port flag, you must disable the "node1_mgmt_port" and "node2_mgmt_port" flags as well.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get MGMT port flag
my $mgmt_port = $grid_member->mgmt_port();
#Modify MGMT port flag
$grid_member->mgmt_port("true");
```

mgmt_port_duplex( )

Use this method to specify the duplex setting of the MGMT port on a single grid member or on node 1 of an HA grid member. Note: Use this method in combination with mgmt_port_speed. If any of these attributes are undefined, the default values for duplex and speed settings are used.

Parameter

Valid values are "Half" or "Full". The default value is undefined.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get mgmt_port_duplex
my $mgmt_port_duplex = $obj->mgmt_port_duplex();
#Modify mgmt_port_duplex
$obj->mgmt_port_duplex("Half");
```

mgmt_port_speed( )

Use this method to specify the connection speed of the MGMT port on a single grid member or on node 1 of an HA grid member. Note: Use this method in combination with mgmt_port_duplex. If any of these attributes are undefined, the default values for duplex and speed settings are used.

Parameter

Valid values are 10, 100 or 1000. The default value is undefined.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get mgmt_port_speed
my $mgmt_port_speed = $obj->mgmt_port_speed();
#Modify mgmt_port_speed
$obj->mgmt_port_speed(10);
```

mgmt_reserved_interface( )

Use this method to set or retrieve the discovery reserved interface of the MGMT port on a single grid member or on node 1 of an HA grid

member.

Parameter

Valid value is an Infoblox::Grid::Discovery::DeviceInterface object.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get attribute value
my $attr = $grid_member->mgmt_reserved_interface();
#Modify attribute value
$grid_member->mgmt_reserved_interface($interface);
```

mmdb_ea_build_time( )

Use this method to retrieve the extensible attributes MaxMind database build time. This is a read-only attribute.

Parameter

None

Returns

The method returns the attribute value. The number of seconds that have elapsed since January 1, 1970 UTC.

Example

```
#Get mmdb_ea_build_time value
my $value = $object->mmdb_ea_build_time();
```

mmdb_geoip_build_time( )

Use this method to retrieve the GeoIP MaxMind database build time. This is a read-only attribute.

Parameter

None

Returns

The method returns the attribute value. The number of seconds that have elapsed since January 1, 1970 UTC.

Example

```
#Get mmdb_ea_build_time value
my $value = $object->mmdb_ea_build_time();
```

name( )

Use this method to set or retrieve the host name of a grid member.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

The host name in FQDN (Fully Qualified Domain Name) format of a grid member. The FQDN consists of the host name followed by the domain name (example: abc.com). A host name can have a maximum of 256 bytes.

Note that name matching is not case-sensitive, and that names are inserted in lowercase.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get name
my $name = $grid_member->name();
#Modify name
$grid_member->name("hostname2.com");
```

nat_enabled( )

Use this method to set or retrieve the NAT compatibility setting.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

Specify "true" to enable the NAT compatibility setting or "false" to disable it.

This method will also be automatically set to "true" or "false" when "nat_group", "nat_ip_address", "node1_nat" and "node2_nat" change. If any of them is set, nat_enabled will be set to "true"; if all of them are undefined, it will be set to "false".

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get nat group
my $nat_group = $grid_member->nat_group();
#Modify nat group
$grid_member->nat_group("nat_group");
```

nat_group( )

Use this method to set or retrieve the NAT group of a grid member. Required only when NAT compatibility is enabled.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

Text with the NAT (Network Address Translation) group name with a maximum of 256 bytes. Default value is empty.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get nat group
my $nat_group = $grid_member->nat_group();
#Modify nat group
$grid_member->nat_group("nat_group");
```

nat_ip_address( )

Use this method to set or retrieve the NAT IP address of a grid member. Required only when NAT compatibility is enabled.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

An IPv4 address is a 32-bit number in dotted decimal notation. It consists of four 8-bit groups of decimal digits separated by decimal points (example: 192.168.1.2).

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get nat IP address
my $nat_ip_address = $grid_member->nat_ip_address();
#Modify nat ip address
$grid_member->nat_ip_address("5.5.5.5");
```

nic_failover_enabled( )

Use this method to set or retrieve the nic_failover_enabled flag of a grid member. This is required only when the member is a single appliance. To enable this flag, the lan2_port flag must also be set to 'true'.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

Specify "true" to set the nic_failover_enabled flag or "false" to deactivate/unset it. The default value is "false".

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get NIC failover enabled flag
my $nic_failover_enabled = $grid_member->nic_failover_enabled();
#Modify NIC failover enabled flag
$grid_member->nic_failover_enabled("true");
```

nic_failover_enable_primary( )

Use this method to set or retrieve the flag that indicates whether to prefer LAN1 when it's available or not.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

Specify "true" to nic_failover_enable_primary flag or "false" to deactivate/unset it. The default value is "false".

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get nic_failover_enable_primary value
my $value = $object->nic_failover_enable_primary();

#Modify nic_failover_enable_primary value
$object->nic_failover_enable_primary("true");
```

node1_ha( )

Use this method to set or retrieve the HA IP address of node 1. Required only when the member is an HA pair.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

An IPv4 address is a 32-bit number in dotted decimal notation. It consists of four 8-bit groups of decimal digits separated by decimal points (example: 192.168.1.2).

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get node1 ha IP address
my $node1_ha = $grid_member->node1_ha();
#Modify node1 ha IP address
$grid_member->node1_ha("3.0.0.4");
```

node1_ha_reserved_interface( )

Use this method to set or retrieve the discovery reserved interface of the HA port on node 1 of an HA grid member.

Parameter

Valid value is an Infoblox::Grid::Discovery::DeviceInterface object.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get attribute value
my $attr = $grid_member->node1_ha_reserved_interface();
#Modify attribute value
$grid_member->node1_ha_reserved_interface($interface);
```

node1_paid_nios( )

Use this method to retrieve whether the node 1 of an HA grid member is Paid NIOS.

Parameter

None

Returns

The method returns the attribute value.

Example

```
#Get Paid NIOS status
my $node1_paid_nios = $grid_member->node1_paid_nios();
```

node1_hwplatform( )

Use this method to retrieve the hardware platform of node 1 of an HA grid member.

Parameter

None

Returns

The method returns the attribute value.

Example

```
#Get hardware platform
my $node1_hwplatform = $grid_member->node1_hwplatform();
```

node1_hwid( )

Use this method to retrieve the hardware ID of node 1 of an HA grid member.

Parameter

None

Returns

The method returns the attribute value.

Example

```
#Get node1 hardware ID
my $node1_hwid = $grid_member->node1_hwid();
```

node1_lan( )

Use this method to set or retrieve the LAN IPv4 address of node 1 of an HA grid member. Required only when the member is an HA pair.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

An IPv4 address is a 32-bit number in dotted decimal notation. It consists of four 8-bit groups of decimal digits separated by decimal points (example: 192.168.1.2).

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get LAN IP address of node1
my $node1_lan = $grid_member->node1_lan();
#Modify LAN IP address of node1
$grid_member->node1_lan("3.0.0.9");
```

node1_lan_ipv6addr( )

Use this method to set or retrieve the LAN IPv6 address of node 1 of an HA grid member. Required only when the member is an HA pair.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

An IPv6 address is a 128-bit number represented as eight groups of four hexadecimal digits (i.e. two octets), each group separated by a `colon(:)` (example: 2001:0db8:85a3:0000:0000:8a2e:0370:7334). Leading zeroes and groups of consecutive zeros within the value can be omitted to produce a more compact representation (example: 2001::1, ::1).

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get LAN IPv6 address of node1
my $node1_lan_ipv6addr = $grid_member->node1_lan_ipv6addr();
#Modify LAN IPv6 address of node1
$grid_member->node1_lan_ipv6addr("2001::1");
```

node1_lan_reserved_interface( )

Use this method to set or retrieve the discovery reserved interface of the LAN port on node 1 of an HA grid member.

Parameter

Valid value is an Infoblox::Grid::Discovery::DeviceInterface object.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get attribute value
my $attr = $grid_member->node1_lan_reserved_interface();
#Modify attribute value
$grid_member->node1_lan_reserved_interface($interface);
```

node1_lan2_reserved_interface( )

Use this method to set or retrieve the discovery reserved interface of the LAN2 port on node 1 of an HA grid member.

Parameter

Valid value is an Infoblox::Grid::Discovery::DeviceInterface object.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get attribute value
my $attr = $grid_member->node1_lan2_reserved_interface();
#Modify attribute value
$grid_member->node1_lan2_reserved_interface($interface);
```

node1_lcd_autodim( )

Use this method to set or retrieve the number of seconds before the LCD on the front panel of node 1 of an HA grid member automatically dims.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

The number of seconds. Valid values are between 5 and 3600. The default value is 30.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get lcd_autodim on node 1
my $node1_lcd_autodim = $grid_member->node1_lcd_autodim();
#Modify the autodim value
$grid_member->node1_lcd_autodim(60);
```

node1_lcd_brightness( )

Use this method to set or retrieve the brightness level of the LCD on the front panel of node 1 of an HA grid member.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

Valid values are between 1 and 10. The default value is 7.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get lcd_brightness on node 1
my $node1_lcd_brightness = $grid_member->node1_lcd_brightness();
#Modify the brightness value
$grid_member->node1_lcd_brightness(3);
```

node1_lcd_hwident( )

Use this method to set or retrieve whether the node 1 of an HA grid member's hardware ID identification functionality is active.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

Specify "true" to set the lcd_hwident flag or "false" to deactivate/unset it. The default value is undefined.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get the lcd_hwident status
my $node1_lcd_hwident = $grid_member->node1_lcd_hwident();
#Modify the lcd_hwident status
$grid_member->node1_lcd_hwident("true");
```

node1_lcd_version( )

Use this method to retrieve the version of the Liquid Crystal Display (LCD) of the node 1 of an HA grid member.

Parameter

None.

Returns

The method returns the attribute value.

Example

```
#Get node1_lcd_version
my $node1_lcd_version = $grid_member->node1_lcd_version();
```

node1_lom_gateway( )

Use this method to set or retrieve the gateway IP address on node 1 of an HA grid member for the LOM functionality.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

An IPv4 address is a 32-bit number in dotted decimal notation. It consists of four 8-bit groups of decimal digits separated by decimal points (example: 192.168.1.2).

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get node1_lom_gateway
my $node1_lom_gateway = $grid_member->node1_lom_gateway();
#Modify node1_lom_gateway
$grid_member->node1_lom_gateway("3.0.0.1");
```

node1_lom_is_capable( )

Use this method to retrieve a flag describing if node 1 of an HA grid member is capable of the LOM functionality.

Parameter

The method returns "true" if the node is capable of LOM operation, "false" otherwise.

Returns

The method returns the attribute value.

Example

```
#Get node1_lom_is_capable
my $node1_lom_is_capable = $grid_member->node1_lom_is_capable();
```

node1_lom_ipv4addr( )

Use this method to set or retrieve the IPv4 address on node 1 of an HA grid member for the LOM functionality.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

An IPv4 address is a 32-bit number in dotted decimal notation. It consists of four 8-bit groups of decimal digits separated by decimal points (example: 192.168.1.2).

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get node1_lom_ipv4addr
my $node1_lom_ipv4addr = $grid_member->node1_lom_ipv4addr();
#Modify node1_lom_ipv4addr
$grid_member->node1_lom_ipv4addr("3.0.0.3");
```

node1_lom_mask( )

Use this method to set or retrieve the netmask on node 1 of an HA grid member for the LOM functionality.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

Netmask address is a 32-bit number in dotted decimal notation. It consists of four 8-bit groups of decimal digits separated by decimal points (example: 255.255.0.0).

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get node1_lom_mask
my $node1_lom_mask = $grid_member->node1_lom_mask();
#Modify node1_lom_mask
$grid_member->node1_lom_mask("255.255.0.0");
```

node1_mgmt_dscp( )

Use this method to set or retrieve the DSCP value of the IPv4 MGMT port on node 1 of an HA grid member.

Setting this method to a defined value implicitly sets the override_node1_mgmt_dscp method to "true". Setting the parameter to undefined causes the appliance to use the grid default and automatically resets the override_node1_mgmt_dscp attribute to "false".

Note that when dscp is set to a defined value and override_node1_mgmt_dscp is set to "false", the last operation takes precedence. Thus the sequence $object->node1_mgmt_dscp(11); $object->override_node1_mgmt_dscp("false"); will set override_node1_mgmt_dscp to "false", and the sequence $object->override_node1_mgmt_dscp("false"); $object->node1_mgmt_dscp(11); will result in override_node1_mgmt_dscp="true".

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

Valid values are integers between 0 and 63, inclusive. The default value is undefined.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get attribute value
my $value = $grid_member->node1_mgmt_dscp();
#Modify attribute value
$grid_member->node1_mgmt_dscp(11);
```

node1_mgmt_gateway( )

Use this method to set or retrieve the gateway IP address of the MGMT port on node 1 of an HA grid member. Required only when the MGMT port is enabled.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

An IPv4 address is a 32-bit number in dotted decimal notation. It consists of four 8-bit groups of decimal digits separated by decimal points (example: 192.168.1.2).

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get gateway of MGMT port of node1
my $node1_mgmt_gateway = $grid_member->node1_mgmt_gateway();
#Modify gateway of MGMT port of node1
$grid_member->node1_mgmt_gateway("6.0.0.1");
```

node1_mgmt_ipv6addr( )

Use this method to set or retrieve the LAN IPv6 address of the MGMT port on node 1 of an HA grid member.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

An IPv6 address is a 128-bit number represented as eight groups of four hexadecimal digits (i.e. two octets), each group separated by a `colon(:)` (example: 2001:0db8:85a3:0000:0000:8a2e:0370:7334). Leading zeroes and groups of consecutive zeros within the value can be omitted to produce a more compact representation (example: 2001::1, ::1).

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get MGMT IPv6 address of node 1
my $node1_mgmt_ipv6addr = $grid_member->node1_mgmt_ipv6addr();
#Modify MGMT IPv6 address of node 1
$grid_member->node1_mgmt_ipv6addr("2001::2");
```

node1_mgmt_ipv6_cidr( )

Use this method to set or retrieve the CIDR of the MGMT port on node 1 of an HA grid member. Required only when node1_mgmt_ipv6addr is specified.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

Specify the netmask in CIDR format.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get MGMT cidr of node 1
my $node1_mgmt_cidr = $grid_member->node1_mgmt_cidr();
#Modify MGMT cidr of node 1
$grid_member->node1_mgmt_cidr("64");
```

node1_mgmt_ipv6_dscp( )

Use this method to set or retrieve the DSCP value of the IPv6 MGMT port on node 1 of an HA grid member.

Setting this method to a defined value implicitly sets the override_node1_mgmt_ipv6_dscp method to "true". Setting the parameter to undefined causes the appliance to use the grid default and automatically resets the override_node1_mgmt_ipv6_dscp attribute to "false".

Note that when dscp is set to a defined value and override_node1_mgmt_ipv6_dscp is set to "false", the last operation takes precedence. Thus the sequence $object->node1_mgmt_ipv6_dscp(11); $object->override_node1_mgmt_ipv6_dscp("false"); will set override_node1_mgmt_ipv6_dscp to "false", and the sequence $object->override_node1_mgmt_ipv6_dscp("false"); $object->node1_mgmt_ipv6_dscp(11); will result in override_node1_mgmt_ipv6_dscp="true".

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

Valid values are integers between 0 and 63, inclusive. The default value is undefined.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get attribute value
my $value = $grid_member->node1_mgmt_ipv6_dscp();
#Modify attribute value
$grid_member->node1_mgmt_ipv6_dscp(11);
```

node1_mgmt_ipv6_enable_auto_config( )

Use this method to enable/disable IPv6 auto-configuration of the MGMT interface.

When IPv6 auto-configuration is enabled, a gateway is not needed since it will be automatically discovered.

Parameter

Specify "true" to enable IPv6 auto-configuration or "false" to disable it. Default value is "false".

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

Example

```
#Get IPv6 auto-configuration of node 1
my $ipv6_autoconfig_enabled = $grid_member->node1_mgmt_ipv6_enable_auto_config();
#Modify IPv6 auto-configuration of node 1
$grid_member->node1_mgmt_ipv6_enable_auto_config("true");
```

node1_mgmt_ipv6_gateway( )

Use this method to set or retrieve the LAN IPv6 gateway of the MGMT port on node 1 of an HA grid member.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

An IPv6 address is a 128-bit number represented as eight groups of four hexadecimal digits (i.e. two octets), each group separated by a `colon(:)` (example: 2001:0db8:85a3:0000:0000:8a2e:0370:7334). Leading zeroes and groups of consecutive zeros within the value can be omitted to produce a more compact representation (example: 2001::1, ::1).

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get MGMT IPv6 gateway of node 1
my $node1_mgmt_ipv6_gateway = $grid_member->node1_mgmt_ipv6_gateway();
#Modify MGMT IPv6 gateway of node 1
$grid_member->node1_mgmt_ipv6_gateway("2001::1");
```

node1_mgmt_mask( )

Use this method to set or retrieve the netmask of the MGMT port on node 1 of an HA grid member. Required only when the MGMT port is enabled.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

Netmask address is a 32-bit number in dotted decimal notation. It consists of four 8-bit groups of decimal digits separated by decimal points (example: 255.255.0.0).

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get netmask of MGMT port of node 1
my $node1_mgmt_mask = $grid_member->node1_mgmt_mask();
#Modify netmask of MGMT port of node 1
$grid_member->node1_mgmt_mask("255.255.0.0");
```

node1_mgmt_lan( )

Use this method to set or retrieve the LAN IP address of the MGMT port on node 1 of an HA grid member. Required only when the MGMT port is enabled.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

An IPv4 address is a 32-bit number in dotted decimal notation. It consists of four 8-bit groups of decimal digits separated by decimal points (example: 192.168.1.2).

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get LAN IP address of MGMT port on node 1
my $node1_mgmt_lan = $grid_member->node1_mgmt_lan();
#Modify LAN IP address of MGMT port on node 1
$grid_member->node1_mgmt_lan("6.0.0.5");
```

node1_mgmt_port( )

Use this method to set or retrieve the MGMT port flag of an HA grid member. Required only when the MGMT port is enabled.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Note that node1_mgmt_port, node2_mgmt_port and mgmt_port share the value; setting any of them will have the same effect.

Parameter

Specify "true" to set the node1_mgmt_port flag or "false" to deactivate/unset it. The default value is undefined. To disable node1_mgmt_port flag we also need to disable "mgmt_port" and "node2_mgmt_port" flags.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get node 1 MGMT port flag
my $node1_mgmt_port = $grid_member->node1_mgmt_port();
#Modify node 1 MGMT port flag
$grid_member->node1_mgmt_port("true");
```

node1_mgmt_reserved_interface( )

Use this method to set or retrieve the discovery reserved interface of the MGMT port on node 1 of an HA grid member.

Parameter

Valid value is an Infoblox::Grid::Discovery::DeviceInterface object.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get attribute value
my $attr = $grid_member->node1_mgmt_reserved_interface();
#Modify attribute value
$grid_member->node1_mgmt_reserved_interface($interface);
```

node1_nat( )

Use this method to set or retrieve the Network Address Translation (NAT) address of node 1 of an HA grid member. Required only when NAT compatibility is enabled.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

An IPv4 address is a 32-bit number in dotted decimal notation. It consists of four 8-bit groups of decimal digits separated by decimal points (example: 192.168.1.2).

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get Network Address Translation (NAT) address of node 1
my $node1_nat = $grid_member->node1_nat();
#Modify Network Address Translation (NAT) address of node 1
$grid_member->node1_nat("5.2.2.1");
```

node1_service_status( )

Use this method to retrieve the service status list of a grid member.

Returns

On retrieval, the member returns an array reference that contains Infoblox::Grid::Member::ServiceStatus objects.

Example

```
#Get the service status list
my $services_list = $grid_member->node1_service_status();
```

vpn_mtu( )

Use this method to set or retrieve the VPN maximum transmission unit (MTU).

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

Specify the value to set the vpn_mtu value.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get VPN MTU
my $vpn_mtu = $grid_member->vpn_mtu();
#Modify VPN MTU
$grid_member->vpn_mtu(600);
```

node1_vpn_on_mgmt( )

Use this method to set or retrieve the VPN on MGMT port flag of an HA grid member. Valid only when the MGMT port is enabled and the member is not a master candidate. This flag enables VPN services on the MGMT port of an HA grid member.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Note that node1_vpn_on_mgmt, node2_vpn_on_mgmt and vpn_on_mgmt share the value; setting any of them will have the same effect.

Parameter

Specify "true" to set the node1_vpn_on_mgmt flag or "false" to deactivate/unset it. The default value is undefined.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get node 1 VPN on MGMT port flag
my $node1_vpn_on_mgmt = $grid_member->node1_vpn_on_mgmt();
#Modify node 1 VPN on MGMT port flag
$grid_member->node1_vpn_on_mgmt("false");
```

node1_hwtype( )

Use this method to retrieve the hardware type of node 1 of an HA grid member.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

The actual hardware type is populated when the physical node joins the grid. The default value is "".

Returns

The method returns the attribute value.

Example

```
#Get node1 hardware type
```

```
my $node1_hwtype = $grid_member->node1_hwtype();
```

node2_ha( )

Use this method to set or retrieve the HA IP address of node 2 of an HA grid member.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

An IPv4 address is a 32-bit number in dotted decimal notation. It consists of four 8-bit groups of decimal digits separated by decimal points (example: 192.168.1.2).

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get node 2 HA IP address
my $node2_ha = $grid_member->node2_ha();
#Modify node 2 HA IP address
$grid_member->node2_ha("3.0.0.6");
```

node2_ha_reserved_interface( )

Use this method to set or retrieve the discovery reserved interface of the HA port on node 2 of an HA grid member.

Parameter

Valid value is an Infoblox::Grid::Discovery::DeviceInterface object.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get attribute value
my $attr = $grid_member->node2_ha_reserved_interface();
#Modify attribute value
$grid_member->node2_ha_reserved_interface($interface);
```

node2_paid_nios( )

Use this method to retrieve whether the node 2 of an HA grid member is Paid NIOS.

Parameter

None

Returns

The method returns the attribute value.

Example

```
#Get Paid NIOS status
my $node2_paid_nios = $grid_member->node2_paid_nios();
```

node2_hwplatform( )

Use this method to retrieve the hardware platform of node 2 of an HA grid member.

Parameter

None

Returns

The method returns the attribute value.

Example

```
#Get hardware platform
my $node2_hwplatform = $grid_member->node2_hwplatform();
```

node2_hwid( )

Use this method to retrieve the hardware ID of node 2 of an HA grid member.

Parameter

None

Returns

The method returns the attribute value.

Example

```
#Get node2 hardware ID
my $node2_hwid = $grid_member->node2_hwid();
```

node2_lan( )

Use this method to set or retrieve the LAN IP address of node 2 of an HA grid member.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

An IPv4 address is a 32-bit number in dotted decimal notation. It consists of four 8-bit groups of decimal digits separated by decimal points (example: 192.168.1.2).

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get LAN IP address of node 2
my $node2_lan = $grid_member->node2_lan();
#Modify LAN IP address of node 2
$grid_member->node2_lan("3.0.0.5");
```

node2_lan_ipv6addr( )

Use this method to set or retrieve the LAN IPv6 address of node 2 of an HA grid member. Required only when the member is an HA pair.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

An IPv6 address is a 128-bit number represented as eight groups of four hexadecimal digits (i.e. two octets), each group separated by a `colon(:)` (example: 2001:0db8:85a3:0000:0000:8a2e:0370:7334). Leading zeroes and groups of consecutive zeros within the value can be omitted to produce a more compact representation (example: 2001::1, ::1).

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get LAN IPv6 address of node2
my $node2_lan_ipv6addr = $grid_member->node2_lan_ipv6addr();
#Modify LAN IPv6 address of node2
$grid_member->node2_lan_ipv6addr("2001::1");
```

node2_lan_reserved_interface( )

Use this method to set or retrieve the discovery reserved interface of the LAN port on node 2 of an HA grid member.

Parameter

Valid value is an Infoblox::Grid::Discovery::DeviceInterface object.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get attribute value
my $attr = $grid_member->node2_lan_reserved_interface();
#Modify attribute value
$grid_member->node2_lan_reserved_interface($interface);
```

node2_lan2_reserved_interface( )

Use this method to set or retrieve the discovery reserved interface of the LAN2 port on node 2 of an HA grid member.

Parameter

Valid value is an Infoblox::Grid::Discovery::DeviceInterface object.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get attribute value
my $attr = $grid_member->node2_lan2_reserved_interface();
#Modify attribute value
$grid_member->node2_lan2_reserved_interface($interface);
```

node2_lcd_autodim( )

Use this method to set or retrieve the number of seconds before the LCD on the front panel of node 2 of an HA grid member automatically dims.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

The number of seconds. Valid values are between 5 and 3600. The default value is 30.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get lcd_autodim on node 2
my $node2_lcd_autodim = $grid_member->node2_lcd_autodim();
#Modify the autodim value
$grid_member->node2_lcd_autodim(60);
```

node2_lcd_brightness( )

Use this method to set or retrieve the brightness level of the LCD on the front panel of node 2 of an HA grid member.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

Valid values are between 1 and 10. The default value is 7.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get lcd_brightness on node 2
my $node2_lcd_brightness = $grid_member->node2_lcd_brightness();
#Modify the brightness value
$grid_member->node2_lcd_brightness(3);
```

node2_lcd_hwident( )

Use this method to set or retrieve whether the node 2 of an HA grid member's hardware ID identification functionality is active.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

Specify "true" to set the node2_lcd_hwident flag or "false" to deactivate/unset it. The default value is undefined.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get the lcd_hwident status
my $node2_lcd_hwident = $grid_member->node2_lcd_hwident();
#Modify the lcd_hwident status
$grid_member->node2_lcd_hwident("true");
```

node2_lcd_version( )

Use this method to retrieve the version of the Liquid Crystal Display (LCD) of the node 2 of an HA grid member.

Parameter

None.

Returns

The method returns the attribute value.

Example

```
#Get node2_lcd_version
my $node2_lcd_version = $grid_member->node2_lcd_version();
```

node2_lom_gateway( )

Use this method to set or retrieve the gateway IP address on node 2 of an HA grid member for the LOM functionality.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

An IPv4 address is a 32-bit number in dotted decimal notation. It consists of four 8-bit groups of decimal digits separated by decimal points (example: 192.168.1.1).

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get node2_lom_gateway
my $node2_lom_gateway = $grid_member->node2_lom_gateway();
#Modify node2_lom_gateway
$grid_member->node2_lom_gateway("3.0.0.1");
```

node2_lom_is_capable( )

Use this method to retrieve a flag describing if node 1 of an HA grid member is capable of the LOM functionality.

Parameter

The method returns "true" if the node is capable of LOM operation, "false" otherwise.

Returns

The method returns the attribute value.

Example

```
#Get node2 lom is capable
```

```
my $node2_lom_is_capable = $grid_member->node2_lom_is_capable();
```

node2_lom_ipv4addr( )

Use this method to set or retrieve the IPv4 address on node 2 of an HA grid member for the LOM functionality.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

An IPv4 address is a 32-bit number in dotted decimal notation. It consists of four 8-bit groups of decimal digits separated by decimal points (example: 192.168.1.2).

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get node2_lom_ipv4addr
my $node2_lom_ipv4addr = $grid_member->node2_lom_ipv4addr();
#Modify node2_lom_ipv4addr
$grid_member->node2_lom_ipv4addr("3.0.0.3");
```

node2_lom_mask( )

Use this method to set or retrieve the netmask on node 2 of an HA grid member for the LOM functionality.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

Netmask address is a 32-bit number in dotted decimal notation. It consists of four 8-bit groups of decimal digits separated by decimal points (example: 255.255.0.0).

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get node2_lom_mask
my $node2_lom_mask = $grid_member->node2_lom_mask();
#Modify node2_lom_mask
$grid_member->node2_lom_mask("255.255.0.0");
```

node2_mgmt_dscp( )

Use this method to set or retrieve the DSCP value of the IPv4 MGMT port on node 2 of an HA grid member.

Setting this method to a defined value implicitly sets the override_node2_mgmt_dscp method to "true". Setting the parameter to undefined causes the appliance to use the grid default and automatically resets the override_node2_mgmt_dscp attribute to "false".

Note that when dscp is set to a defined value and override_node2_mgmt_dscp is set to "false", the last operation takes precedence. Thus the sequence $object->node2_mgmt_dscp(11); $object->override_node2_mgmt_dscp("false"); will set override_node2_mgmt_dscp to "false", and the sequence $object->override_node2_mgmt_dscp("false"); $object->node2_mgmt_dscp(11); will result in override_node2_mgmt_dscp="true".

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

Valid values are integers between 0 and 63, inclusive. The default value is undefined.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get attribute value
my $value = $grid_member->node2_mgmt_dscp();
#Modify attribute value
```

```
$grid_member->node2_mgmt_dscp(11);
```

node2_mgmt_gateway( )

Use this method to set or retrieve the gateway IP address of the MGMT port on node 2 of an HA grid member. Required only when the MGMT port is enabled.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

An IPv4 address is a 32-bit number in dotted decimal notation. It consists of four 8-bit groups of decimal digits separated by decimal points (example: 192.168.1.2).

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get gateway of MGMT port of node 2
my $node2_mgmt_gateway = $grid_member->node2_mgmt_gateway();
#Modify gateway of MGMT port of node 2
$grid_member->node2_mgmt_gateway("7.0.0.1");
```

node2_mgmt_mask( )

Use this method to set or retrieve the netmask of the MGMT port on node 2 of an HA grid member. Required only when the MGMT port is enabled.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

Netmask address is a 32-bit number in dotted decimal notation. It consists of four 8-bit groups of decimal digits separated by decimal points (example: 255.255.0.0).

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get netmask of MGMT port of node 2
my $node2_mgmt_mask = $grid_member->node2_mgmt_mask();
#Modify netmask of MGMT port of node 2
$grid_member->node2_mgmt_mask("255.255.0.0");
```

node2_mgmt_lan( )

Use this method to set or retrieve the LAN IP address of the MGMT port on node 2 of an HA grid member. Required only when the MGMT port is enabled.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

An IPv4 address is a 32-bit number in dotted decimal notation. It consists of four 8-bit groups of decimal digits separated by decimal points (example: 192.168.1.2).

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get LAN IP address of MGMT port of node 2
my $node2_mgmt_lan = $grid_member->node2_mgmt_lan();
#Modify LAN IP address of MGMT port of node 2
$grid_member->node2_mgmt_lan("7.0.0.5");
```

node2_mgmt_port( )

Use this method to set or retrieve the MGMT port flag of an HA grid member. Required only when the MGMT port is enabled.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Note that node1_mgmt_port, node2_mgmt_port and mgmt_port share the value: setting any of them will have the same effect.

Parameter

Specify "true" to set the node2_mgmt_port flag or "false" to deactivate/unset it. The default value is undefined. To disable node2_mgmt_port flag, you must disable the "mgmt_port" and "node1_mgmt_port" flags as well.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get MGMT port flag of node 2
my $node2_mgmt_port = $grid_member->node2_mgmt_port();
#Modify MGMT port flag of node 2
$grid_member->node2_mgmt_port("true");
```

node2_mgmt_reserved_interface( )

Use this method to set or retrieve the discovery reserved interface of the MGMT port on node 2 of an HA grid member.

Parameter

Valid value is an Infoblox::Grid::Discovery::DeviceInterface object.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get attribute value
my $attr = $grid_member->node2_mgmt_reserved_interface();
#Modify attribute value
$grid_member->node2_mgmt_reserved_interface($interface);
```

node2_mgmt_ipv6_dscp( )

Use this method to set or retrieve the DSCP value of the IPv6 MGMT port on node 2 of an HA grid member.

Setting this method to a defined value implicitly sets the override_node2_mgmt_ipv6_dscp method to "true". Setting the parameter to undefined causes the appliance to use the grid default and automatically resets the override_node2_mgmt_ipv6_dscp attribute to "false".

Note that when dscp is set to a defined value and override_node2_mgmt_ipv6_dscp is set to "false", the last operation takes precedence. Thus the sequence $object->node2_mgmt_ipv6_dscp(11); $object->override_node2_mgmt_ipv6_dscp("false"); will set override_node2_mgmt_ipv6_dscp to "false", and the sequence $object->override_node2_mgmt_ipv6_dscp("false"); $object->node2_mgmt_ipv6_dscp(11); will result in override_node2_mgmt_ipv6_dscp="true".

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

Valid values are integers between 0 and 63, inclusive. The default value is undefined.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get attribute value
my $value = $grid_member->node2_mgmt_ipv6_dscp();
#Modify attribute value
$grid_member->node2_mgmt_ipv6_dscp(11);
```

node2_nat( )

Use this method to set or retrieve the Network Address Translation (NAT) address of node 2. Required only when NAT compatibility is enabled.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

An IPv4 address is a 32-bit number in dotted decimal notation. It consists of four 8-bit groups of decimal digits separated by decimal points (example: 192.168.1.2).

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get Network Address Translation (NAT) address of node2
my $node2_nat = $grid_member->node2_nat();
#Modify Network Address Translation (NAT) address of node2
$grid_member->node2_nat("5.4.4.6");
```

node2_service_status( )

Use this method to retrieve the service status list of a grid member.

Returns

On retrieval, the member returns an array reference that contains Infoblox::Grid::Member::ServiceStatus objects.

Example

```
#Get the service status list
my $services_list = $grid_member->node2_service_status();
```

node2_vpn_on_mgmt( )

Use this method to set or retrieve the VPN on MGMT port flag of an HA grid member. Valid only when the MGMT port is enabled and the member is not a master candidate. This flag enables VPN services on the MGMT port of an HA grid member.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Note that node1_vpn_on_mgmt, node2_vpn_on_mgmt and vpn_on_mgmt share the value: setting any of them will have the same effect.

Parameter

Specify "true" to set the node2_vpn_on_mgmt flag or "false" to deactivate/unset it. The default value is undefined.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get node 2 VPN on MGMT port flag
my $node2_vpn_on_mgmt = $grid_member->node2_vpn_on_mgmt();
#Modify node 2 VPN on MGMT port flag
$grid_member->node2_vpn_on_mgmt("false");
```

node2_hwtype( )

Use this method to retrieve the hardware type of node 2 of an HA grid member.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

The actual hardware type is populated when the physical node joins the grid. The default value is "".

Returns

The method returns the attribute value.

Example

```
#Get node2 hardware type
```

```
my $node2_hwtype = $grid_member->node2_hwtype();
```

notification_email_addr( )

Use this method to set or retrieve the notification email address of a grid member.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

The email address. Default value is undefined.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get notification email address
my $notification_email_addr = $grid_member->notification_email_addr();
#Modify notification email address
$grid_member->notification_email_addr("admin\@infoblox.com");
```

ntp_access_list( )

Use this method to set or retrieve the Network Time Protocols (NTP) access control list of a grid member. Valid only when the enable_ntp flag is set to "true".

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

Valid value is an Infoblox::Grid::NamedACL object or an array reference that contains Infoblox::Grid::NTPAccess objects .

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get NTP access control list
my $ntp_access_list = $grid_member->ntp_access_list();
#Modify NTP access control list
$grid_member->ntp_access_list([$ntpac1,$ntpac2]);
#Undef NTP access control list
$grid_member->ntp_access_list(undef);
#Use Infoblox::Grid::NamedACL
$grid_member->ntp_access_list($nacl);
```

---

ntp_kod( )

Use this method to enable Kiss-o'-Death packets on the NTP server.

Setting this method to a defined value implicitly sets the override_ntp_kod method to "true". Setting the parameter to undefined causes the appliance to use the Grid default and automatically resets the override_ntp_kod attribute to "false".

Note that when ntp_kod is set to a defined value and override_ntp_kod is set to "false", the last operation takes precedence. Thus the sequence $object->ntp_kod("true"); $object->override_ntp_kod("false"); will set override_ntp_kod to "false", and the sequence $object->override_ntp_kod("false"); $object->ntp_kod("true"); will result in override_ntp_kod="true".

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

Specify "true" to enable Kiss-o'-Death packets on the NTP server or "false" to disable them. The default value is "false".

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get the current value of the ntp_kod parameter
my $ntp_kod = $grid->ntp_kod();
#Set the ntp_kod parameter
$grid->ntp_kod("true");
```

ntp_service_type( )

Use this method to set or retrieve the NTP service type for assigned Named ACL object.

The attribute value is ignored if 'ntp_access_list' attribute is not a Named ACL object.

Parameter

Valid values are "Time" or "Time and NTP control". The default value is "Time".

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get ntp_service_type
my $service = $grid_member->ntp_service_type();
#Modify ntp_service_type
$grid_member->ntp_service_type("Time and NTP control");
```

ntp_authentication_key( )

Use this method to set or retrieve the Network Time Protocol (NTP) authentication `key(s)` of a grid member. Valid only when the enable_ntp flag is set to "true".

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

Valid value is an array reference that contains Infoblox::Grid::NTPKey object(s).

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get NTP authentication key
my $ntp_authentication_key = $grid_member->ntp_authentication_key();
#Modify NTP authentication key
my $ntp_key1 = Infoblox::Grid::NTPKey->new(
    key_number => "1111",
    key_type   => "M",
    key_string => "aaaa"
);
my $ntp_key2 = Infoblox::Grid::NTPKey->new(
    key_number => "2222",
    key_type   => "n",
    key_string => "d3e54352e5548080"
);
$grid_member->ntp_authentication_key([$ntp_key1, $ntp_key2]);
```

ntp_server( )

Use this method to set or retrieve the NTP server list.

Parameter

Array reference that contains Infoblox::Grid::NTPServer objects or undef.

Specifying a reference to an empty array while the "exclude_grid_master_as_ntp_server" attribute is set to "false" sets the Grid Master as the only NTP source for the given Member. Setting the value of this parameter to undef disables the feature.

NOTE: Currently, if you need to specify an external NTP server with authentication, the NTP keys that you use must be known to the Infoblox::Grid object. See the following example.

NOTE: Setting this to a list also sets external_ntp_servers_enabled to 'true'. Setting this to undef also sets external_ntp_servers_enabled to 'false'.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

Example

```
#Get the ntp_server list for the given member
my $ntp_server_list=$grid_member->ntp_server();

#Modify ntp_server property
my $ntp_server1 = Infoblox::Grid::NTPServer->new(address => "1.1.1.1");
my $ntp_server2 = Infoblox::Grid::NTPServer->new(address => "2.2.2.2");
$grid_member->ntp_server([$ntp_server1,$ntp_server2]);

#Setting grid master as the only NTP source
my $session=Infoblox::Session->new(
    master   => "127.0.0.1",
    username => "admin",
    password => "infoblox"
   );
die(Infoblox::status_code().":".Infoblox::status_detail()) unless $session;

my $grid_member=$session->get(
                              object => "Infoblox::Grid::Member",
                              name   => "infoblox.localdomain"
                             );
die($session->status_code()." : ".$session->status_detail()) unless (ref($grid_member) eq
"Infoblox::Grid::Member");

$grid_member->ntp_server([]);
$grid_member->exclude_grid_master_as_ntp_server("false");
$session->modify($grid_member) or die($session->status_code()." : ".$session->status_detail());

#Adding external NTP servers with authentication
my $session=Infoblox::Session->new(
    master   => "127.0.0.1",
    username => "admin",
    password => "infoblox"
   );
die(Infoblox::status_code().":".Infoblox::status_detail()) unless $session;
my $grid_member=$session->get(
                              object => "Infoblox::Grid::Member",
                              name   => "infoblox.localdomain"
                             );
die($session->status_code()." : ".$session->status_detail()) unless (ref($grid_member) eq
"Infoblox::Grid::Member");

my $grid=$session->get(
                        object => "Infoblox::Grid",
                        name   => "Infoblox"
                      );
die($session->status_code()." : ".$session->status_detail()) unless (ref($grid) eq
"Infoblox::Grid");

#It is assumed in the following that the grid has NTP enabled
die("Grid must have ntp enabled) unless ($grid->enable_ntp() eq "true");

my $ntp_key_1= Infoblox::Grid::NTPKey->new(
           key_number => "101",
           key_type   => "n",
           key_string => "d3e54352e5548080"
       );
$grid->ntp_authentication_key([$ntp_key_1]);
$session->modify($grid) or die(($session->status_code()." : ".$session->status_detail());

my $ntp_server=Infoblox::Grid::NTPServer->new(
                                              address => "1.1.1.1"
                                              authentication => "true"
                                              ntp_key=>$ntp_key_1
                                             );

$grid_member->ntp_server([$ntp_server]);
$session->modify($grid_member) or die(($session->status_code()." : ".$session->status_detail());
```

ospf_list( )

Use this method to set or retrieve the OSPF area configuration (for anycast) list of a grid member.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

Valid value is an array reference that contains Infoblox::Grid::Member::OSPF object(s).

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get OSPF list
my $ospf_list = $grid_member->ospf_list();
#Modify OSPF list
my $ospf1 = Infoblox::Grid::Member::OSPF->new(
        area_id             => "1",
        interface           => "LAN_HA",
        authentication_type => "none",
);

$grid_member->ospf_list([$ospf1]);
```

override_enable_lom( )

The override_enable_lom attribute controls whether the enable_lom value in the member is used, instead of the grid default.

The override_enable_lom attribute can be specified explicitly. It is also set implicitly when enable_lom is set to a defined value.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

Specify "true" to set the override_enable_lom flag or "false" to deactivate/unset it. The default value is "false".

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get override_enable_lom from the member
my $override = $grid_member->override_enable_lom();
#Modify override_enable_lom
$grid_member->override_enable_lom("true");
```

override_lom_users( )

The override_lom_users attribute controls whether the lom_users value in the member is used, instead of the grid default.

The override_lom_users attribute can be specified explicitly. It is also set implicitly when lom_users is set to a defined value.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

Specify "true" to set the override_lom_users flag or "false" to deactivate/unset it. The default value is "false".

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get override_lom_users from the member
my $override = $grid_member->override_lom_users();
#Modify lom_users
$grid_member->override_lom_users("true");
```

override_dscp( )

The override_dscp attribute contols whether the member DSCP value is used, instead of the Grid default.

The override_dscp attribute can be specified explicitly. It is also set implicitly when DSCP is set to a defined value.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

Specify "true" to set the override_dscp flag or "false" to deactivate/unset it. The default value is "false".

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get override_dscp from the member
my $override = $grid_member->override_dscp();
#Modify override_dscp
$grid_member->override_dscp("true");
```

override_enable_member_redirect( )

The override_enable_member_redirect attribute controls whether the enable_member_redirect value in the member is used, instead of the grid default.

The override_enable_member_redirect attribute can be specified explicitly. It is also set implicitly when enable_member_redirect is set to a defined value.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

Specify "true" to set the override_enable_member_redirect flag or "false" to deactivate/unset it. The default value is "false".

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get override_enable_member_redirect from the member
my $override = $grid_member->override_enable_member_redirect();
#Modify enable_member_redirect
$grid_member->override_enable_member_redirect("true");
```

override_lan1_dscp( )

The override_lan1_dscp attribute contols whether the DSCP value of the IPv4 LAN1 port is used, instead of the Grid default.

The override_lan1_dscp attribute can be specified explicitly. It is also set implicitly when DSCP is set to a defined value.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

Specify "true" to set the override_lan1_dscp flag or "false" to deactivate/unset it. The default value is "false".

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get override_lan1_dscp from the member
my $override = $grid_member->override_lan1_dscp();
#Modify override_lan1_dscp
$grid_member->override_lan1_dscp("true");
```

override_lan1_ipv6_dscp( )

The override_lan1_ipv6_dscp attribute contols whether the DSCP value of the IPv6 LAN1 port is used, instead of the Grid default.

The override_lan1_ipv6_dscp attribute can be specified explicitly. It is also set implicitly when DSCP is set to a defined value.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

Specify "true" to set the override_lan1_ipv6_dscp flag or "false" to deactivate/unset it. The default value is "false".

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get override_lan1_ipv6_dscp from the member
my $override = $grid_member->override_lan1_ipv6_dscp();
#Modify override_lan1_ipv6_dscp
$grid_member->override_lan1_ipv6_dscp("true");
```

override_lan2_dscp( )

The override_lan2_dscp attribute contols whether the DSCP value of the IPv4 LAN2 port is used, instead of the Grid default.

The override_lan2_dscp attribute can be specified explicitly. It is also set implicitly when DSCP is set to a defined value.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

Specify "true" to set the override_lan2_dscp flag or "false" to deactivate/unset it. The default value is "false".

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get override_lan2_dscp from the member
my $override = $grid_member->override_lan2_dscp();
#Modify override_lan2_dscp
$grid_member->override_lan2_dscp("true");
```

override_lan2_ipv6_dscp( )

The override_lan2_ipv6_dscp attribute contols whether the DSCP value of the IPv6 LAN2 port is used, instead of the Grid default.

The override_lan2_ipv6_dscp attribute can be specified explicitly. It is also set implicitly when DSCP is set to a defined value.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

Specify "true" to set the override_lan2_ipv6_dscp flag or "false" to deactivate/unset it. The default value is "false".

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get override_lan2_ipv6_dscp from the member
my $override = $grid_member->override_lan2_ipv6_dscp();
#Modify override_lan2_ipv6_dscp
$grid_member->override_lan2_ipv6_dscp("true");
```

override_mgmt_dscp( )

The override_mgmt_dscp attribute contols whether the DSCP value of the IPv4 MGMT port is used, instead of the Grid default.

The override_mgmt_dscp attribute can be specified explicitly. It is also set implicitly when DSCP is set to a defined value.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

Specify "true" to set the override_mgmt_dscp flag or "false" to deactivate/unset it. The default value is "false".

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get override_mgmt_dscp from the member
my $override = $grid_member->override_mgmt_dscp();
#Modify override_mgmt_dscp
$grid_member->override_mgmt_dscp("true");
```

override_mgmt_ipv6_dscp( )

The override_mgmt_ipv6_dscp attribute contols whether the DSCP value of the IPv6 MGMT port is used, instead of the Grid default.

The override_mgmt_ipv6_dscp attribute can be specified explicitly. It is also set implicitly when DSCP is set to a defined value.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

Specify "true" to set the override_mgmt_ipv6_dscp flag or "false" to deactivate/unset it. The default value is "false".

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get override_mgmt_ipv6_dscp from the member
my $override = $grid_member->override_mgmt_ipv6_dscp();
#Modify override_mgmt_ipv6_dscp
$grid_member->override_mgmt_ipv6_dscp("true");
```

override_node1_mgmt_dscp( )

The override_node1_mgmt_dscp attribute contols whether the DSCP value of the IPv4 MGMT port on node 1 of an HA grid member is used, instead of the Grid default.

The override_node1_mgmt_dscp attribute can be specified explicitly. It is also set implicitly when DSCP is set to a defined value.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

Specify "true" to set the override_node1_mgmt_dscp flag or "false" to deactivate/unset it. The default value is "false".

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get override_node1_mgmt_dscp from the member
my $override = $grid_member->override_node1_mgmt_dscp();
#Modify override_node1_mgmt_dscp
$grid_member->override_node1_mgmt_dscp("true");
```

override_node1_mgmt_ipv6_dscp( )

The override_node1_mgmt_ipv6_dscp attribute contols whether the DSCP value of the IPv6 MGMT port on node 1 of an HA grid member is used, instead of the Grid default.

The override_node1_mgmt_ipv6_dscp attribute can be specified explicitly. It is also set implicitly when DSCP is set to a defined value.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

Specify "true" to set the override_node1_mgmt_ipv6_dscp flag or "false" to deactivate/unset it. The default value is "false".

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get override_node1_mgmt_ipv6_dscp from the member
my $override = $grid_member->override_node1_mgmt_ipv6_dscp();
#Modify override_node1_mgmt_ipv6_dscp
$grid_member->override_node1_mgmt_ipv6_dscp("true");
```

override_node2_mgmt_dscp( )

The override_node2_mgmt_dscp attribute contols whether the DSCP value of the IPv4 MGMT port on node 2 of an HA grid member is used, instead of the Grid default.

The override_node2_mgmt_dscp attribute can be specified explicitly. It is also set implicitly when DSCP is set to a defined value.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

Specify "true" to set the override_node2_mgmt_dscp flag or "false" to deactivate/unset it. The default value is "false".

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get override_node2_mgmt_dscp from the member
my $override = $grid_member->override_node2_mgmt_dscp();
#Modify override_node2_mgmt_dscp
$grid_member->override_node2_mgmt_dscp("true");
```

override_node2_mgmt_ipv6_dscp( )

The override_node2_mgmt_ipv6_dscp attribute contols whether the DSCP value of the IPv6 MGMT port on node 2 of an HA grid member is used, instead of the Grid default.

The override_node2_mgmt_ipv6_dscp attribute can be specified explicitly. It is also set implicitly when DSCP is set to a defined value.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

Specify "true" to set the override_node2_mgmt_ipv6_dscp flag or "false" to deactivate/unset it. The default value is "false".

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get override_node2_mgmt_ipv6_dscp from the member
my $override = $grid_member->override_node2_mgmt_ipv6_dscp();
#Modify override_node2_mgmt_ipv6_dscp
$grid_member->override_node2_mgmt_ipv6_dscp("true");
```

override_ntp_kod( )

The override_ntp_kod attribute controls whether the ntp_kod value in the member is used, instead of the Grid default.

The override_ntp_kod attribute can be specified explicitly. It is also set implicitly when ntp_kod is set to a defined value.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

Specify "true" to set the override_ntp_kod flag or "false" to deactivate it. The default value is "false".

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get override_ntp_kod from the member
my $override = $grid_member->override_ntp_kod();
#Modify override_ntp_kod
$grid_member->override_ntp_kod("true");
```

override_syslog_backup_servers( )

The override_syslog_backup_servers attribute controls whether the syslog_backup_servers value for the Member is used, instead of the Grid default.

The override_syslog_backup_servers attribute can be specified explicitly. It is also set implicitly when syslog_backup_servers is set to a defined value.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

Specify 'true' to set the override_syslog_backup_servers flag or 'false' to deactivate/unset it. The default value is 'false'.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#get override_syslog_backup_servers of member
my $override_syslog_backup_servers = $grid_member->override_syslog_backup_servers();

#modify override_syslog_backup_servers
$grid_member->override_syslog_backup_servers('true');
```

override_threshold_traps( )

The override_threshold_traps attribute controls whether the threshold_traps value in the member is used, instead of the grid default.

The override_threshold_traps attribute can be specified explicitly. It is also set implicitly when threshold_traps is set to a defined value.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

Specify "true" to set the override_threshold_traps flag or "false" to deactivate/unset it. The default value is "false".

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get override_threshold_traps of member
my $override_threshold_traps = $grid_member->override_threshold_traps();
#Set override_threshold_traps
$grid_member->override_threshold_traps("true");
```

override_trap_notifications( )

The override_trap_notifications attribute controls whether the trap_notifications value in the member is used, instead of the grid default.

The override_trap_notifications attribute can be specified explicitly. It is also set implicitly when trap_notifications is set to a defined value.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

Specify "true" to set the override_trap_notifications flag or "false" to deactivate/unset it. The default value is "false".

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get override_trap_notifications of member
my $override_trap_notifications = $grid_member->override_trap_notifications();
#Set override_trap_notifications
$grid_member->override_trap_notifications("true");
```

passive_ha_arp_enabled( )

Use this method to set or retrieve the ARP protocol setting on the passive node of an HA pair. If you do not specify a value, the default value is "false". You can only set this value to "true" if the member is an HA pair.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

Specify "true" to set the passive_ha_arp_enabled flag or "false" to deactivate/unset it. The default value is "false".

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get member passive_ha_arp_enabled
my $passive_ha_arp_enabled = $grid_member->passive_ha_arp_enabled();
#Modify member passive_ha_arp_enabled
$grid_member->passive_ha_arp_enabled("true");
```

platform( )

Use this method to set or retrieve the platform type of a grid member. If you do not specify a value, the default value is "Infoblox".

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

Valid value is one of either "Infoblox", "Cisco", "Riverbed", or "VNIOS".

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get member platform
my $platform = $grid_member->platform();
#Modify node 2 hardware type
$grid_member->platform("Cisco");
```

prefer_resolver( )

Use this method to set or retrieve the preferred name server of the grid member. The grid member sends queries to this name server first.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

Valid values are Infoblox::DNS::Nameserver object and Infoblox::DNS::Member object.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get preferred resolver
```

```
my $prefer_resolver = $grid_member->prefer_resolver();
#Modify preferred resolver
my $member1 = Infoblox::DNS::Member->new(
    name           => $host_name,
    ipv4addr       => $host_ip,
    stealth        => "false",
    lead           => "false",
    grid_replicate => "false"
);
$grid_member->prefer_resolver($member1);
#Undef prefer resolver
$grid_member->prefer_resolver(undef);
```

pre_provisioning( )

Use this method to set or retrieve the pre-provisioned Grid member.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

Valid value is an Infoblox::Grid::Member::PreProvisioning object.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get PreProvisioning grid Member
my $pp = $grid_member->pre_provisioning();
#Modify PreProvisioning grid Member
my $hw = Infoblox::Grid::Member::PreProvisioning::Hardware->new(
   hwtype          => 'IB-4010',
   hwmodel         => 'Rev1',
);

my $pp = Infoblox::Grid::Member::PreProvisioning->new(
   hardware_info   => [$hw],
   license         => ['dns', 'dhcp'],
);
$grid_member->pre_provisioning($pp);
```

preserve_if_owns_delegation( )

Use this method to set or retrieve the preserve_if_owns_delegation attribute. If you do not specify a value, the default value is "false".

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

Specify "true" to set the preserve_if_owns_delegation flag or "false" to deactivate/unset it. The default value is "false".

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get member preserve_if_owns_delegation
my $preserve_if_owns_delegation  = $grid_member->preserve_if_owns_delegation ();
#Modify member preserve_if_owns_delegation
$grid_member->preserve_if_owns_delegation ("true");
```

query_comm_string( )

Use this method to set or retrieve the Simple Network Management Protocol (SNMP) query community string that the SNMP management system must send together with its queries to the grid member. An SNMP community string is a text string that acts as a password. It is used to authenticate messages that are sent between the management station (the SNMP manager) and the grid member.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

Text with the Simple Network Management Protocol (SNMP) query community string with a maximum of 50 bytes. Default value is undefined.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get query community string
my $query_comm_string = $grid_member->query_comm_string();
#Modify query community string
$grid_member->query_comm_string("test");
#Undef query communication string
$grid_member->query_comm_string(undef);
```

remote_console_access( )

Use this method to set or retrieve the remote_console support access flag of a grid member. When you enable remote console access, a client can access the device using a Secure Shell (SSH) connection.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

Specify "true" to set the remote_console_access flag or "false" to deactivate/unset it. The default value is undefined.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get remote console access flag
my $remote_console_access = $grid_member->remote_console_access();
#Modify remote console access flag
$grid_member->remote_console_access("true");
#Undef remote console access flag
$grid_member->remote_console_access(undef);
```

router_ID( )

Use this method to set or retrieve the virtual router ID of an HA grid member. Required only when the member is an HA pair.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

Number with the router identification. Default value is undefined. Valid values should be between 1 and 255

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get router ID
my $router_ID = $grid_member->router_ID();
#Modify router ID
$grid_member->router_ID("34");
```

search_domains( )

Use this method to set or retrieve the Search Domain Group, which is a group of domain names that the Infoblox device can add to partial queries that do not specify a domain name. Note that you can set this parameter only when prefer_resolver or alternate_resolver is set.

Parameter

Valid value is a reference to an array of strings.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

Example

```
#Get search_domains
my $search_domains = $grid_member->search_domains();
#Modify search_domains
$grid_member->search_domains(["domain1.com", "domain2.com"]);
```

service_status( )

Use this method to retrieve the service status list of a grid member.

Returns

On retrieval, the member returns an array reference that contains Infoblox::Grid::Member::ServiceStatus objects.

Example

```
#Get the service status list
my $services_list = $grid_member->service_status();
```

service_type_configuration( )

Use this method to set or retrieve the configuration option for the member service type.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

The valid value is one of the following: "ALL_V4", "ALL_V6" or "CUSTOM".

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
# Get the value of service_type_configuration
my $service_type_configuration = $grid_member->service_type_configuration();
# Modify service_type_configuration
$grid_member->service_type_configuration("ALL_V6");
```

snmp_admin( )

Use this method to set or retrieve the Simple Network Management Protocol (SNMP) admin object of a grid member.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

Valid value is an Infoblox::Grid::SNMP::Admin object.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get SNMP admin
my $snmp_admin = $grid_member->snmp_admin();
#Modify SNMP admin
my $snmp_admin_obj = Infoblox::Grid::SNMP::Admin->new(
    sysContact_node1  => "Somebody",
    sysDescr_node1    => "SunOS noc1 5.6 Generic_105181-03 sun4u",
    sysLocation_node1 => "Infoblox Headquarter",
    sysName_node1     => "noc1"
);
$grid_member->snmp_admin($snmp_admin_obj);
#Undef snmp admin
$grid_member->snmp_admin(undef);
```

snmpv3_query_users( )

Use this method to set or retrieve the SNMPv3 user setting at the grid level.

Parameter

Valid value is a reference to the list of the Infoblox::Grid::SNMP::User manpage objects.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

Example

```
#Get snmpv3_query_users field
my $snmpv3_users = $gridmember->snmpv3_query_users();
my $user = $$snmpv3_users[0];
#Set snmpv3_query_users field
$gridmember->snmpv3_query_users([$user1, $user2]);
```

static_routes( )

Use this method to set or retrieve the static routes list of a grid member. This list contains the static routes of a grid member.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

Valid value is an array reference that contains Infoblox::Grid::Member::StaticRoute object(s).

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get static routes list
my $static_routes = $grid_member->static_routes();
#Modify static routes list
my $route = Infoblox::Grid::Member::StaticRoute->new(
   "network" => "192.168.2.0/24",
   "gateway" => "192.168.1.1",
);
$grid_member->static_routes([$route]);
```

support_access( )

Use this method to set or retrieve the support access flag of a grid member. This flag enables a Secure Shell (SSH) daemon that only Infoblox Technical Support can access to perform troubleshooting at the grid member level.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

Specify "true" to set the remote_console_access flag or "false" to deactivate/unset it. The default value is undefined.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get support access flag
my $support_access = $grid_member->support_access();
#Modify support access flag
$grid_member->support_access("true");
#Undef support access
$grid_member->support_access(undef);
```

support_access_info( )

Use this method to retrieve the information string for support access.

Parameter

None.

Returns

The method returns the attribute value.

Example

```
#Get support_access_info
my $support_access_info = $grid_member->support_access_info();
```

syslog_proxy_tcp_port( )

Use this method to set or retrieve the Transmission Control Protocol (TCP) listening port for syslog proxy.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

TCP port number. Default value is 514. Valid values should be between 1 and 65535.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get syslog proxy tcp port
my $syslog_proxy_tcp_port = $grid_member->syslog_proxy_tcp_port();
#Modify syslog proxy tcp port
$grid_member->syslog_proxy_tcp_port(514);
```

syslog_proxy_udp_port( )

Use this method to set or retrieve the User Datagram Protocol (UDP) listening port for syslog proxy.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

UDP port number. Default value is 514. Valid values should be between 1 and 65535.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get syslog proxy udp port
my $syslog_proxy_udp_port = $grid_member->syslog_proxy_udp_port();
#Modify syslog proxy udp port
$grid_member->syslog_proxy_udp_port(514);
```

syslog_size( )

Use this method to set or retrieve the grid member syslog rotation size in megabytes.

Parameter

Valid value is between 10 and 300.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get syslog_size
my $syslog_size = $obj->syslog_size();
#Modify syslog_size
$obj->syslog_size(100);
```

syslog_backup_servers( )

Use this method to set or retrieve an array that contains syslog backup server settings for a Grid member.

Setting this method to a defined value implicitly sets the override_syslog_backup_servers method to 'true'. Setting the parameter to undefined causes the appliance to use the Grid default and automatically resets the override_syslog_backup_servers attribute to 'false'.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

Valid value is an array reference that contains Infoblox::Grid::SyslogBackupServer objects.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#get syslog_backup_servers value
my $servers = $grid_member->syslog_backup_servers();

#modify syslog_backup_servers value
$grid_member->syslog_backup_servers([$server1, $server2]);
```

syslog_server( )

Use this method to set or retrieve the syslog server of a grid member.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

Valid value is an array reference that contains Infoblox::Grid::SyslogServer object(s).

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get syslog server
my $syslog_server = $grid_member->syslog_server();
#Modify syslog server
my $syslog1 = Infoblox::Grid::SyslogServer->new(
    ipv4addr => "44.44.44.44",
    severity => "debug",
);
my $syslog2 = Infoblox::Grid::SyslogServer->new(
    ipv4addr => "55.55.55.55",
    severity => "debug",
);

$grid_member->syslog_server([$syslog1, $syslog2]);
#Undef syslog server
$grid_member->syslog_server(undef);
```

time_zone( )

Use this method to set or retrieve the time zone of the grid member.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

The UTC string that represents the time zone. For example "(UTC - 5:00) Eastern Time (US and Canada)". Default value is undefined.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get time zone
my $time_zone = $grid_member->time_zone();
#Modify time zone
```

```
$grid_member->time_zone("(UTC - 5:00) Eastern Time (US and Canada)");
#Undef time zone
$grid_member->time_zone(undef);
```

threshold_traps( )

Use this method to retrieve the reference to the list of threshold traps. Each element in the list is a reference to the Infoblox::Grid::SNMP::ThresholdTrap object. If the member inherits traps from the grid, the threshold_traps value is undefined.

Setting this method to a defined value implicitly sets the override_threshold_traps method to "true". Setting the parameter to undefined causes the appliance to use the grid default and automatically resets the override_threshold_traps attribute to "false".

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

List of Infoblox::Grid::SNMP::ThresholdTrap objects.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get threshold traps
my $threshold_traps = $member->threshold_traps();
#Modify threshold traps
my $threshold_traps = $member->threshold_traps();
foreach(@$threshold_traps){if $_->trap_name() eq 'Rootfs'; $_->trap_reset=60;} #Change value for
'Rootfs' trap
$grid->threshold_traps($threshold_traps);
$session->modify($member);
```

traffic_capture( )

Use this method to control traffic capture for the specified member. Note that when you use this method, the object MUST be retrieved through a get or search call, or created with an add call.

Parameter

The method accepts an Infoblox::Grid::Member::Capture::Control object as a parameter.

Returns

When passing a capture control object as a parameter, the method returns true if the specified traffic control operation succeeds. It returns false when the operation fails.

If invoked without a parameter, the method returns a Infoblox::Grid::Member::Capture::Status object that describes the current traffic capture status.

Please see the status and control objects for usage examples.

trap_comm_string( )

Use this method to set or retrieve the Simple Network Management Protocol (SNMP) trap community string of the grid member. An SNMP trap community string is a text string that acts as a password. It is used to authenticate messages that are sent between the grid member and the SNMP management systems.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

Text with the Simple Network Management Protocol (SNMP) trap community, with a maximum of 50 bytes. Default value is undefined. To undefine trap_comm_string we also need to undefine "trap_receiver" attribute.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get trap community string
my $trap_comm_string = $grid_member->trap_comm_string();
#Modify trap community string
```

```
$grid_member->trap_comm_string("trap_string");
```

trap_notifications( )

Use this method to configure trap notifications. Each element in the list is a reference to the Infoblox::Grid::SNMP::TrapNotification object. If the trap_notifications value is undefined the member inherits trap notifications from the grid.

Explicitly setting this method to a defined value implicitly sets the override_trap_notifications method to "true". Setting the parameter to undefined causes the appliance to use the grid default and automatically resets the override_trap_notifications attribute to "false".

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

Reference to a list of Infoblox::Grid::SNMP::TrapNotification objects. The default value is a reference to a list of all trap notifications.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get a reference to a trap notifications array
my $trap_notifications = $member->trap_notifications();
#Modify trap notifications
my $notifications = $member->trap_notifications();
foreach(@$notifications){if $_->trap_type() eq 'CPU'; $_->enable_email="true";} #Change value for
'CPU' trap
$member->trap_notifications($notifications);
$session->modify($member);
```

trap_receiver( )

Use this method to set or retrieve the Simple Network Management Protocol (SNMP) trap receivers of a grid member.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

Valid value is an array reference that contains IPv4 addresses and the Infoblox::Grid::SNMP::TrapReceiver manpage objects. Default value is undefined. To undefine trap_receiver, you must also undefine the "trap_comm_string" attribute.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get trap receiver
my $trap_receiver = $grid_member->trap_receiver();
#Modify trap receiver (SNMPv2)
$grid_member->trap_receiver(["10.0.0.10"]);
#SNMPv3 trap receivers
my $receiver1 = Infoblox::Grid::SNMP::TrapReceiver->new( ip_addr => "19.20.21.22", user => $user);
$gridmember->trap_receiver([$receiver1]);
```

upgrade_group( )

Use this method to set or retrieve the name of the upgrade group to which this Grid member belongs.

Parameter

Valid value is a string representing the upgrade group name.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get upgrade_group
my $upgrade group   $obj  upgrade group
```

```
                    =     ->              ();
#Modify upgrade_group
$obj->upgrade_group("group2");
```

use_v4_vrrp( )

Use this method to switch between VRRPv4 and VRRPv6.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

Specify "true" to use VRRPv4 or "false" to use VRRPv6.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get use_v4_vrrp
my $use_v4_vrrp = $obj->use_v4_vrrp();
#Modify use_v4_vrrp
$obj->use_v4_vrrp("false");
```

vpn_on_mgmt( )

Use this method to set or retrieve the VPN services on the MGMT port of a grid member that is a single appliance.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Note that node1_vpn_on_mgmt, node2_vpn_on_mgmt and vpn_on_mgmt share the value; setting any of them will have the same effect.

Parameter

Specify "true" to set the vpn_on_mgmt flag or "false" to deactivate/unset it. The default value is "false".

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get VPN on MGMT port flag
my $vpn_on_mgmt = $grid_member->vpn_on_mgmt();
#Modify VPN on MGMT port flag
$grid_member->vpn_on_mgmt("false");
```

vlan_id( )

Use this method to set or retrieve the VLAN ID of a grid member.

Parameter

Valid value is an unsigned integer between 1 and 4095 inclusive. The default value is undefined.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get attribute value
my $attr = $grid_member->vlan_id();
#Modify attribute value
$grid_member->vlan_id(10);
```

create_token( )

Use this function to create the authorization token string for each physical node of the pre-provisioned member.

Parameter

None

Returns

The method returns a reference to an array of Infoblox::Grid::Member::PNodeToken objects.

Example

```
my $tokens = $grid_member->create_token();
```

read_token( )

Use this function to read the authorization token string for each physical node of the pre-provisioned member.

Parameter

None

Returns

The method returns a reference to an array of Infoblox::Grid::Member::PNodeToken objects. If tokens were not created previously, or have already expired, an empty array will be returned.

Example

```
my $tokens = $grid_member->read_token();
```

SAMPLE CODE

The following sample code demonstrates the different functions that can be applied to an object such as add, get, modify, search and remove. This sample also includes error handling for the operations.

#Preparation prior to getting and modifying a grid member object

```
use strict;
use Infoblox;

#refers to Infoblox Appliance IP address
my $host_ip = "192.168.1.2";

#Create a session to the Infoblox appliance

my $session = Infoblox::Session->new(
    master   => $host_ip,
    username => "admin",
    password => "infoblox"
);
unless ($session) {
       die("Construct session failed: ",
               Infoblox::status_code() . ":" . Infoblox::status_detail());
}
print "Session created successfully\n";

my $sysserv1 = Infoblox::Grid::SyslogServer->new(
    ipv4addr => "7.7.7.7"
);
unless ($sysserv1) {
       die("Construct sysserv1 failed: ",
               Infoblox::status_code() . ":" . Infoblox::status_detail());
}
print "Sysserv1 created successfully\n";
```

#Create a grid member object

```
my $grid_member = Infoblox::Grid::Member->new(
    name          => "hostname.com",
    ipv4addr      => "3.0.0.2",
    mask          => "255.0.0.0",
    gateway       => "3.0.0.1",
    syslog_server => [$sysserv1],
    comment       => "Script Generated"
    );
unless ($grid_member) {
       die("Construct grid member object failed: ",
               Infoblox::status_code() . ":" . Infoblox::status_detail());
}
```

```
print "Grid member object created successfully\n";

#Adding the member object to appliance through session
my $response = $session->add( $grid_member );

unless ($response) {
    die("Add grid member failed: ",
            $session->status_code() . ":" . $session->status_detail());
}
print"Grid member added successfully \n";
```

#Search for a grid member object

```
 my @retrieved_objs = $session->search(
     object => "Infoblox::Grid::Member",
     name   => ".*\\.com");
 my $object = $retrieved_objs[0];

 unless ($object) {
        die("Search grid member object failed: ",
                $session->status_code() . ":" . $session->status_detail());
 }
 print "Search grid member object found at least 1 matching entry\n";
```

#Get and modify the member object

```
#Get grid member through session
my @retrieved_objs = $session->get(
    object => "Infoblox::Grid::Member",
    name   => "hostname.com"
    );
my $grid_member = $retrieved_objs[0];

unless ($grid_member) {
    die("Get grid member object failed: ",
            $session->status_code() . ":" . $session->status_detail());
}
print"Get grid member successful \n";

# Modify one of the attributes of selected grid member.

$grid_member->comment("Modified member object");

#Applying the changes
$session->modify($grid_member)
   or die("Modify grid member failed",
            $session->status_code() . ":" . $session->status_detail());
print"Grid member modified successfully \n";
```

#Remove the grid member object

```
#Get grid member through session
my @retrieved_objs = $session->get(
    object => "Infoblox::Grid::Member",
    name   => "hostname.com"
    );
my $desired_grid_member = $retrieved_objs[0];

unless ($desired_grid_member) {
    die("Get grid member object failed: ",
            $session->status_code() . ":" . $session->status_detail());
}
print"Get grid member successful \n";

#Submit the object for removal
$session->remove( $desired_grid_member )
    or die("Remove grid member failed",
            $session->status_code() . ":" . $session->status_detail());
print"Grid member removed successfully \n";

####PROGRAM ENDS####
```

AUTHOR

Infoblox Inc. http://www.infoblox.com/

SEE ALSO

Infoblox::Grid, Infoblox::Grid::NTPKey, Infoblox::Grid::SNMP::Admin, Infoblox::Grid::SyslogServer, Infoblox::Session, Infoblox::Grid::NamedACL, Infoblox::Grid::SyslogBackupServer Infoblox::Grid::Member::OSPF,Infoblox::Grid::LOM::User, Infoblox::DNS::Member, Infoblox::DNS::Nameserver,

---

COPYRIGHT

Copyright (c) 2017 Infoblox Inc.

Infoblox::Grid::Member::BGP::AS - BGP object.

- NAME
- DESCRIPTION
- CONSTRUCTOR
- MODULE METHODS
  - Infoblox::Grid::Member->bgp_as( )
- METHODS
  - as( )
  - keepalive( )
  - holddown( )
  - neighbors( )
- AUTHOR
- SEE ALSO
- COPYRIGHT

NAME

Infoblox::Grid::Member::BGP::AS - BGP (Border Gateway Protocol) Autonomous System (AS) object.

DESCRIPTION

An Infoblox::BGP::AS object represents a BGP Autonomous System configured at the grid member level.

CONSTRUCTOR

```
my $bgp_as = Infoblox::Grid::Member::BGP::AS->new(
    as              => $num,                          # Optional / Default value is undefined
    keepalive       => $num,                          # Optional / Default value is 4
    holddown        => $num,                          # Optional / Default value is 16
    neighbors       => [ $neighbor1, $neighbor2, ... ], # Optional / Default value is empty
);
```

MODULE METHODS

The following functions can be applied to a BGP AS object.

Infoblox::Grid::Member->bgp_as( )

Use this function to specify BGP Autonomous Systems for the grid member. See Infoblox::Grid::Member->bgp_as() for parameters and return values.

Example

```
#Construct a BGP neighbor
my $neighbor = Infoblox::Grid::Member::BGP::Neighbor->new(
    neighbor_ip          => "172.16.0.1",
    remote_as            => 1234,
    interface            => "LAN_HA",
);

#Construct a BGP AS
my $autonomous_system = Infoblox::Grid::Member::BGP::AS->new(
    as          => 269,
    keepalive       => 4,
    holddown        => 16,
    neighbors       => [ $neighbor ]
);

# Configure BGP on the Grid Member object
my $response = $Grid_Member->bgp_as([$as]);
```

METHODS

This section describes all the methods that can be used to configure and retrieve the attribute values of an Infoblox::BGP::AS object.

as( )

Use this method to set or retrieve the number of this AS.

A valid AS number is required to advertise to neighbors defined in this AS.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

An AS number is a 16-bit integer from 1 to 65535.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get AS number
my $as_number = $autonomous_system->as();
#Modify AS number
$autonomous_system->as(300);
```

keepalive( )

Use this method to set or retrieve the AS keepalive timer.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

The number of seconds for the keepalive timer. The valid value is from 1 to 21845 seconds.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get AS keepalive timer
my $keepalive = $autonomous_system->keepalive();
#Modify AS keepalive timer
$autonomous_system->keepalive(3600);
```

holddown( )

Use this method to set or retrieve the AS hold down timer.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

The number of seconds for the hold down timer. The valid value is from 3 to 65535 seconds.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get Autonomous System hold down timer
my $holddown = $autonomous_system->holddown();
#Modify Autonomous System hold down timer
$autonomous_system->holddown(3600);
```

neighbors( )

Use this method to set or retrieve the BGP neighbors for the Infoblox::BGP::AS object.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

The valid value is an array reference that contains Infoblox::Grid::Member::BGP::Neighbor object(s).

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get AS neighbor list
my $neighbors = $autonomous_system->neighbors();
#Modify AS neighbors
$autonomous_system->neighbors( [ $neighbor1, $neighbor2 ] );
```

AUTHOR

Infoblox Inc. http://www.infoblox.com/

SEE ALSO

Infoblox::Grid::Member, Infoblox::Grid::Member::BGP::Neighbor

---

COPYRIGHT

Copyright (c) 2017 Infoblox Inc.

Infoblox::Grid::Member::BGP::Neighbor - BGP neighbor object

- NAME
- DESCRIPTION
- CONSTRUCTOR
- MODULE METHODS
  - Infoblox::Grid::Member::BGP::AS->neighbors( )
- METHODS
  - enable_multihop( )
  - enable_bfd( )
  - interface( )
  - multihop_ttl( )
  - neighbor_ip( )
  - remote_as( )
  - authentication_mode( )
  - bfd_template( )
  - bgp_neighbor_pass( )
  - comment( )
- AUTHOR
- SEE ALSO
- COPYRIGHT

NAME

Infoblox::Grid::Member::BGP::Neighbor - BGP (Border Gateway Protocol) neighbor object

DESCRIPTION

An Infoblox::BGP::Neighbor object represents the BGP neighbor that is configured in an Autonomous System (AS). BGP neighbors are configured at the grid member level.

CONSTRUCTOR

```
my $neighbor = Infoblox::Grid::Member::BGP::Neighbor->new(
    neighbor_ip          => $ipv4addr | $ipv6addr, # Required
    remote_as            => $num,                  # Required
    bgp_neighbor_pass    => $string,               # Required if  authentication_mode set to "MD5"
    authentication_mode  => "NONE" | "MD5",        # Optional / default is "NONE"
    bfd_template         => $string,               # Optional / default is undefined
    comment              => $string,               # Optional / default is undefined
    enable_bfd           => "true" | "false",      # Optional / default is "false"
    enable_multihop      => "true" | "false",      # Optional / default is "false"
    interface            => $string,               # Optional / default is "LAN_HA"
    multihop_ttl         => $uint | undef,         # Optional / default is undefined
);
```

MODULE METHODS

The following functions can be applied to a BGP AS object.

Infoblox::Grid::Member::BGP::AS->neighbors( )

Use this function to specify BGP Autonomous Systems for the grid member. See Infoblox::Grid::Member::BGP::AS->neighbors() for parameters and return values.

Example

```
#Construct a BGP neighbor
my $neighbor = Infoblox::Grid::Member::BGP::Neighbor->new(
    neighbor_ip           => "172.16.0.1",
    remote_as             => 1234,
    authentication_mode   => "NONE",
    interface             => "LAN_HA",
);

#Construct a BGP AS
my $autonomous_system = Infoblox::Grid::Member::BGP::AS->new(
    as          => 269,
    keepalive       => 4,
```

```
    holddown        => 16,
    neighbors       => [ $neighbor ]
);

# Configure BGP on the Grid Member object
my $response = $Grid_Member->bgp_as([$as]);
```

METHODS

This section describes all the methods that can be used to configure and retrieve the attribute values of an Infoblox::BGP::AS object.

enable_multihop( )

Use this method to set or retrieve the flag that indicates whether the multihop option is enabled or not.

By default, BGP connects to external peers residing on directly connected networks. Usually the ttl in IP header (hop-limit IPv6) is set to 1 to avoid BGP packets crossing networks. EBGP multihop option allows connection to peers which have one or more non-BGP routers in between.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

Specify 'true' to enable multihop option and 'false' to disable it. The default value is 'false'.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get enable_multihop value
my $enable_multihop = $object->enable_multihop();
#Modify enable_multihop value
$object->enable_multihop("true");
```

enable_bfd( )

Use this method to set or retrieve the flag that indicates whether the Bidirectional Forwarding Detection (BFD) fallover is enabled or not.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

Specify 'true' to enable BFD fallover and false to disable it. The default value is 'false'.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get enable_bfd value
my $enable_bfd = $object->enable_bfd();
#Modify enable_bfd value
$object->enable_bfd('true');
```

interface( )

Use this method to set or retrieve the interface that sends BGP advertisement information.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

Specify the interface in string format. The only supported value is "LAN_HA".

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get interface
my $interface = $neighbor->interface();
#Modify interface
$neighbor->interface("LAN_HA");
```

multihop_ttl( )

Use this method to set or retrieve the TTL setting for multihop option.

By default, BGP connects to external peers residing on directly connected networks. Usually the ttl in IP header (hop-limit IPv6) is set to 1 to avoid BGP packets crossing networks. EBGP multihop option allows connection to peers which have one or more non-BGP routers in between.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

The valid value is an unsigned integer between 1 and 255. The default value is undefined.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get multihop_ttl value
my $multihop_ttl = $object->multihop_ttl();
#Modify multihop_ttl value
$object->multihop_ttl(200);
```

neighbor_ip( )

Use this method to set or retrieve the IP address of the BGP neighbor.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

The IPv4 or IPv6 address of the BGP neighbor.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get neighbor IP address
my $neighbor_ip = $neighbor->neighbor_ip();
#Modify neighbor IP address
$neighbor->neighbor_ip("192.168.1.2");
$neighbor->neighbor_ip("2001::1");
```

remote_as( )

Use this method to set or retrieve the remote AS number of the BGP neighbor.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

An AS number is a 16-bit number from 1 to 65535.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get neighbor remote AS number
my $remote_as = $neighbor->remote_as();
#Modify remote AS number
```

```
$neighbor->remote_as(269);
```

authentication_mode( )

Use this method to set or retrieve the attribute about whether BGP authentication has been set for each neighbor.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

Set parameter to "MD5" to enable BGP authentication for a definite neighbor.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get neighbor enable authentication
my $remote_as = $neighbor->authentication_mode();
#Modify enable authentication
$neighbor->authentication_mode("MD5");
```

bfd_template( )

Use this method to set or retrieve the Bidirectional Forwarding Detection (BFD) template name.

BFD template is used to configure advanced BFD settings such as timer intervals, authentication.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

Desired BFD template in string format. Default value is undefined.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get bfd_template value
my $bfd_template = $object->bfd_template();
#Modify bfd_template value
$object->bfd_template("template1");
```

bgp_neighbor_pass( )

Use this method to set the password for a BGP neighbor. This is required only if authentication_mode is set to "MD5". When the password is entered, the value is preserved even if authentication_mode is changed to "NONE". This is a write-only attribute.

Parameter

The bgp_neighbor_pass in string format.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

Example

```
#Modify authentication password per neighbor
$neighbor->bgp_neighbor_pass("authpassword");
```

comment( )

Use this method to set or retrieve a descriptive comment.

Parameter

Desired comment in string format with a maximum of 256 bytes.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get neighbor comment
my $comment = $neighbor->comment();
#Modify comment
$neighbor->comment("my comment");
```

AUTHOR

Infoblox Inc. http://www.infoblox.com/

SEE ALSO

L<Infoblox::Grid::Member::BGP::AS|Infoblox::Grid::Member::BGP::AS>

COPYRIGHT

Copyright (c) 2017 Infoblox Inc.

Infoblox::Grid::Member::CapacityReport - Capacity report for a specified member.

- NAME
- DESCRIPTION
- CONSTRUCTOR
- SESSION METHODS
  - Infoblox::Session->get( )
- METHODS
  - hardware_type( )
  - max_capacity( )
  - name( )
  - object_counts( )
  - percent_used( )
  - role( )
  - total_objects( )
- AUTHOR
- SEE ALSO
- COPYRIGHT

NAME

Infoblox::Grid::Member::CapacityReport - Capacity report for a specified member.

DESCRIPTION

This is a read-only object that provides the capacity report for the specified member.

CONSTRUCTOR

The Infoblox::Grid::Member::CapacityReport object is a read-only object and does not require manual construction.

SESSION METHODS

This section describes all the methods in an Infoblox::Session module that you can apply to the Infoblox::Grid::HSM::AllGroup object.

Infoblox::Session->get( )

Use this method to retrieve all the matching objects from the Infoblox appliance. See Infoblox::Session->get() for parameters and return values.

Key Reference

Apply the following attribute to get a specific capacity report object:

```
name - Required. A member name.
```

Example

```
my @retrieved_objs = $session->get(
    object => "Infoblox::Grid::Member::CapacityReport",
    name   => "infoblox.localdomain"
    );
```

METHODS

This section describes all the methods that you can use to retrieve the attribute values of the Infoblox::Grid::Member::CapacityReport object.

hardware_type( )

Use this method to retrieve the member's hardware type.

Parameter

None

Returns

The method returns the attribute value.

Example

```
#Get hardware_type
my $hardware_type = $report->hardware_type();
```

max_capacity( )

Use this method to retrieve the member's maximum object capacity.

Parameter

None

Returns

The method returns the attribute value.

Example

```
#Get max_capacity
my $max_capacity = $report->max_capacity();
```

name( )

Use this method to retrieve the member name.

Parameter

None

Returns

The method returns the attribute value.

Example

```
#Get name
my $name = $report->name();
```

object_counts( )

Use this method to retrieve the object counts for the member.

Parameter

None

Returns

The method returns the attribute value, which is a reference to a hash that contains keys to the object types and the corresponding values to the counts of each particular object type.

Example

```
#Get object_counts
my $object_counts = $report->object_counts();
#Get the zone count
my $zone_count = $object_counts->{'Zone'};
```

percent_used( )

Use this method to retrieve the percentage of the member capacity currently being used.

Parameter

None

Returns

The method returns the attribute value.

Example

```
#Get percent_used
my $percent_used = $report->percent_used();
```

role( )

Use this method to retrieve the member role.

Parameter

None

Returns

The method returns the attribute value.

Example

```
#Get role
my $role = $report->role();
```

total_objects( )

Use this method to retrieve the total number of objects on the member.

Parameter

None

Returns

The method returns the attribute value.

Example

```
#Get total_objects
my $total_objects = $report->total_objects();
```

AUTHOR

Infoblox Inc. http://www.infoblox.com/

SEE ALSO

Infoblox::Session, Infoblox::Session->get()

COPYRIGHT

Copyright (c) 2017 Infoblox Inc.

Infoblox::Grid::Member::CaptivePortal - Captive Portal object.

- NAME
- DESCRIPTION
- CONSTRUCTOR
- SESSION METHODS
  - Infoblox::Session->get( )
  - Infoblox::Session->modify( )
- METHODS
  - authn_server_group ( )
  - company_name ( )
  - enable_syslog_auth_failure ( )
  - enable_syslog_auth_success ( )
  - enable_user_type ( )
  - encryption ( )
  - files ( )
  - guest_custom_field1_name ( )
  - guest_custom_field1_required ( )
  - guest_custom_field2_name ( )
  - guest_custom_field2_required ( )
  - guest_custom_field3_name ( )
  - guest_custom_field3_required ( )
  - guest_custom_field4_name ( )
  - guest_custom_field4_required ( )
  - guest_email_required ( )
  - guest_first_name_required ( )
  - guest_last_name_required ( )
  - guest_middle_name_required ( )
  - guest_phone_required ( )
  - helpdesk_message ( )
  - listen_address ( )
  - name ( )
  - network_view ( )
  - port ( )
  - service_enabled ( )
  - syslog_auth_failure_level ( )
  - syslog_auth_success_level ( )
  - welcome_message ( )
- SAMPLE CODE
- AUTHOR
- SEE ALSO
- COPYRIGHT

NAME

Infoblox::Grid::Member::CaptivePortal - Captive Portal object.

DESCRIPTION

This object represents the captive portal configuration.

CONSTRUCTOR

The Infoblox::Grid::Member::CaptivePortal object is automatically generated upon the successful installation of the Infoblox appliance. It does not require manual construction.

SESSION METHODS

This section describes all the methods in the Infoblox::Session module that you can apply to a Captive Portal object.

Infoblox::Session->get( )

Use this method to retrieve matching objects from the Infoblox appliance. See Infoblox::Session->get() for parameters and return values.

Key Reference

```
Apply the following attribute to get a specific Captive Portal object:

 name - Required. A domain name in string format.
```

Example

```
my @retrieved objs   $session  get
```

```
                =          ->    (
object => "Infoblox::Grid::Member::CaptivePortal",
name   => "infoblox.localdomain"
);
```

Infoblox::Session->modify( )

Use this method to modify objects in the Infoblox appliance. See Infoblox::Session->modify() for parameters and return values.

Example

```
#Use this method to modify the welcome message.
$captive_portal->welcome_message("New welcome message");
#Submit modification
my $response = $session->modify( $captive_portal );
```

METHODS

This section describes all the methods that you can use to configure and retrieve the attribute values of the captive portal.

authn_server_group ( )

Use this method to set or retrieve the authentication server group assigned to this captive portal.

Parameter

The valid values are an Infoblox::Grid::Admin::AdAuthService, an Infoblox::Grid::Admin::RadiusAuthService or an Infoblox::LDAP::AuthService object. The default value is undefined.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Getting authn_server_group
my $authn_server_group = $captive_portal->authn_server_group();
#Modifying authn_server_group
$captive_portal->authn_server_group($adservice);
```

company_name ( )

Use this method to set or retrieve the company name that appears in the guest registration page.

Parameter

The valid value is a string. The default value is undefined.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Getting company_name
my $company_name = $captive_portal->company_company_name();
#Modifying company_name
$captive_portal->company_name('Company Name');
```

enable_syslog_auth_failure ( )

Use this method to set or retrieve the flag that controls whether authentication failures are logged to syslog.

Parameter

Specify "true" to enable logging or "false" to disable it. The default value is "false".

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Getting enable_syslog_auth_failure
```

```
my $enable_syslog_auth_failure = $captive_portal->enable_syslog_auth_failure();
#Modifying enable_syslog_auth_failure
$captive_portal->enable_syslog_auth_failure('true');
```

enable_syslog_auth_success ( )

Use this method to set or retrieve the flag that controls whether successful authentications are logged to syslog.

Parameter

Specify "true" to enable logging or "false" to disable it. The default value is "true".

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Getting enable_syslog_auth_success
my $enable_syslog_auth_success = $captive_portal->enable_syslog_auth_success();
#Modifying enable_syslog_auth_success
$captive_portal->enable_syslog_auth_success('false');
```

enable_user_type ( )

Use this method to set or retrieve the type of user to be enabled for the captive portal.

Parameter

Valid values are "Authenticated", "Guest" or "Both". The default value is "Authenticated".

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Getting enable_user_type
my $enable_user_type = $captive_portal->enable_user_type();
#Modifying enable_user_type
$captive_portal->enable_user_type('Both');
```

encryption ( )

Use this method to set or retrieve the encryption the captive portal uses.

Parameter

Valid values are 'None' or 'SSL'. The default value is 'None'.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Getting encryption
my $encryption = $captive_portal->encryption();
#Modifying encryption
$captive_portal->encryption('SSL');
```

files ( )

Use this method to set or retrieve the list of files associated with the captive portal.

Parameter

The valid value is an array reference that contains Infoblox::Grid::Member::CaptivePortal::File object(s). New files must be added using Infoblox::Session->import_data(). However, if a file is removed from this list, it is also removed from the captive portal.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Getting files
my $files = $captive_portal->files();
#Modifying files
$captive_portal->files([$logo]);
```

guest_custom_field1_name ( )

Use this method to set or retrieve the name of the custom field that you are adding to the guest registration page.

Parameter

The valid value is a string. The default value is undefined. The maximum length is 200 bytes.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Getting guest_custom_field1_name
my $guest_custom_field1_name = $captive_portal->guest_custom_field1_guest_custom_field1_name();
#Modifying guest_custom_field1_name
$captive_portal->guest_custom_field1_name('Extra field 1');
```

guest_custom_field1_required ( )

Use this method to set or retrieve the flag that controls whether the custom field is required.

Parameter

Specify "true" to set the flag or "false" to disable it. The default value is "false".

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Getting guest_custom_field1_required
my $guest_custom_field1_required = $captive_portal->guest_custom_field1_required();
#Modifying guest_custom_field1_required
$captive_portal->guest_custom_field1_required('true');
```

guest_custom_field2_name ( )

Use this method to set or retrieve the name of the custom field that you are adding to the guest registration page.

Parameter

The valid value is a string. The default value is undefined. The maximum length is 200 bytes.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Getting guest_custom_field2_name
my $guest_custom_field2_name = $captive_portal->guest_custom_field2_guest_custom_field2_name();
#Modifying guest_custom_field2_name
$captive_portal->guest_custom_field2_name('Extra field 2');
```

guest_custom_field2_required ( )

Use this method to set or retrieve the flag that controls whether the custom field is required.

Parameter

Specify "true" to set the flag or "false" to disable it. The default value is "false".

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Getting guest_custom_field2_required
my $guest_custom_field2_required = $captive_portal->guest_custom_field2_required();
#Modifying guest_custom_field2_required
$captive_portal->guest_custom_field2_required('true');
```

guest_custom_field3_name ( )

Use this method to set or retrieve the name of the custom field that you are adding to the guest registration page.

Parameter

The valid value is a string. The default value is undefined. The maximum length is 200 bytes.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Getting guest_custom_field3_name
my $guest_custom_field3_name = $captive_portal->guest_custom_field3_guest_custom_field3_name();
#Modifying guest_custom_field3_name
$captive_portal->guest_custom_field3_name('Extra field 3');
```

guest_custom_field3_required ( )

Use this method to set or retrieve the flag that controls whether the custom field is required.

Parameter

Specify "true" to set the flag or "false" to disable it. The default value is "false".

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Getting guest_custom_field3_required
my $guest_custom_field3_required = $captive_portal->guest_custom_field3_required();
#Modifying guest_custom_field3_required
$captive_portal->guest_custom_field3_required('true');
```

guest_custom_field4_name ( )

Use this method to set or retrieve the name of the custom field that you are adding to the guest registration page.

Parameter

The valid value is a string. The default value is undefined. The maximum length is 200 bytes.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Getting guest_custom_field4_name
my $guest_custom_field4_name = $captive_portal->guest_custom_field4_guest_custom_field4_name();
#Modifying guest_custom_field4_name
$captive_portal->guest_custom_field4_name('Extra field 4');
```

guest_custom_field4_required ( )

Use this method to set or retrieve the flag that controls whether the custom field is required.

Parameter

Specify "true" to set the flag or "false" to disable it. The default value is "false".

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Getting guest_custom_field4_required
my $guest_custom_field4_required = $captive_portal->guest_custom_field4_required();
#Modifying guest_custom_field4_required
$captive_portal->guest_custom_field4_required('true');
```

guest_email_required ( )

Use this method to set or retrieve the flag that controls whether the email address of the guest is required.

Parameter

Specify "true" to set this flag or "false" to disable it. The default value is "true".

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Getting guest_email_required
my $guest_email_required = $captive_portal->guest_email_required();
#Modifying guest_email_required
$captive_portal->guest_email_required('false');
```

guest_first_name_required ( )

Use this method to set or retrieve the flag that controls whether the first name of the guest is required.

Parameter

Specify "true" to enable or "false" to disable it. The default value is "true".

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Getting guest_first_name_required
my $guest_first_name_required = $captive_portal->guest_first_name_required();
#Modifying guest_first_name_required
$captive_portal->guest_first_name_required('false');
```

guest_last_name_required ( )

Use this method to set or retrieve the flag that controls whether the last name of the guest is required.

Parameter

Specify "true" to set the flag or "false" to disable it. The default value is "true".

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Getting guest_last_name_required
my $guest_last_name_required = $captive_portal->guest_last_name_required();
#Modifying guest_last_name_required
$captive_portal->guest_last_name_required('false');
```

guest_middle_name_required ( )

Use this method to set or retrieve the flag that controls whether the middle name of the guest is required.

Parameter

Specify "true" to set the flag or "false" to disable it. The default value is "false".

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Getting guest_middle_name_required
my $guest_middle_name_required = $captive_portal->guest_middle_name_required();
#Modifying guest_middle_name_required
$captive_portal->guest_middle_name_required('true');
```

guest_phone_required ( )

Use this method to set or retrieve the flag that controls whether the phone number of the guest is required.

Parameter

Specify "true" to set this flag or "false" to disable it. The default value is "true".

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Getting guest_phone_required
my $guest_phone_required = $captive_portal->guest_phone_required();
#Modifying guest_phone_required
$captive_portal->guest_phone_required('false');
```

helpdesk_message ( )

Use this method to set or retrieve the helpdesk message that appears in the guest registration page.

Parameter

The valid value is a string. The default value is undefined.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Getting helpdesk_message
my $helpdesk_message = $captive_portal->helpdesk_message();
#Modifying helpdesk_message
$captive_portal->helpdesk_message('In case of problems, please contact us at 555-555-5555');
```

listen_address ( )

Use this method to set or retrieve the IP address on which the captive portal listens.

Parameter

The valid values are 'VIP', 'LAN2' or an IPv4 address. The default value is 'VIP'.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Getting listen_address
my $listen_address = $captive_portal->listen_address();
#Modifying listen_address
$captive_portal->listen_address('VIP');
```

name ( )

Use this method to retrieve the hostname of the grid member that hosts the captive portal.

Returns

The method returns the attribute value.

Example

```
#Getting name
my $name = $captive_portal->name();
```

network_view ( )

Use this method to set or retrieve the network view of the captive portal.

Parameter

The valid value is a network view name. The default is the system default network view.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Getting network_view
my $network_view = $captive_portal->network_view();
#Modifying network_view
$captive_portal->network_view('otherview');
```

port ( )

Use this method to set or retrieve the TCP port used by the Captive Portal service. The port is required when the Captive Portal service is enabled.

Parameter

Valid values are between 1 and 63999. The default value is 4433. Please note that setting the port number to 80 or 443 might impact performance.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Getting the port
my $port = $captive_portal->port();
#Modifying the port
$captive_portal->port(4457);
```

service_enabled ( )

Use this method to set or retrieve the state of the captive portal service.

Parameter

Specify "true" to enable the service or "false" to disable it. The default value is "false".

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Getting service_enabled
my $service_enabled = $captive_portal->service_enabled();
#Modifying service_enabled
$captive_portal->service_enabled('false');
```

syslog_auth_failure_level ( )

Use this method to set or retrieve the syslog level at which authentication failures are logged.

Parameter

Valid values are "debug", "info", "notice", "warning", "err", "crit", "alert" or "emerg" . The default value is "info".

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Getting syslog_auth_failure_level
my $syslog_auth_failure_level = $captive_portal->syslog_auth_failure_level();
#Modifying syslog_auth_failure_level
$captive_portal->syslog_auth_failure_level('notice');
```

syslog_auth_success_level ( )

Use this method to set or retrieve the syslog level at which successful authentications are logged.

Parameter

Valid values are "debug", "info", "notice", "warning", "err", "crit", "alert" or "emerg" . The default value is "info".

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Getting syslog_auth_success_level
my $syslog_auth_success_level = $captive_portal->syslog_auth_success_level();
#Modifying syslog_auth_success_level
$captive_portal->syslog_auth_success_level('warning');
```

welcome_message ( )

Use this method to set or retrieve the welcome message that appears in the guest registration page.

Parameter

The valid value is a string. The default value is undefined.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Getting welcome_message
my $welcome_message = $captive_portal->welcome_message();
#Modifying welcome_message
$captive_portal->welcome_message('Welcome to our network!');
```

SAMPLE CODE

The following sample code demonstrates different operations that can be applied to a captive portal object, such as get and modify an object. This sample code also includes error handling for the operations.

#Preparation prior to operations on a captive portal object

```
#PROGRAM STARTS: Include all the modules that will be used
use strict;
use Infoblox;

#Create a session to the Infoblox appliance
my $session = Infoblox::Session->new(
    master   => "192.168.1.2",
    username => "admin",
    password => "infoblox"
);
unless ($session) {
   die("Construct session failed: ",
       Infoblox::status_code() . ":" . Infoblox::status_detail());
}
print "Session created successfully\n";

# Upload a new AUP to the captive portal, from now on
# the files() method will contain an Infoblox::Grid::Member::CaptivePortal::File
# object referencing this.
$session->import_data(
                      type     => 'captive_portal_file',
```

```
                        path    => 'aup.txt',
                        member  => '192.168.1.2',
                        cp_type => 'aup'
    )
    or die("file import failed: ",
            $session->status_code(), $session->status_detail());

    my $ads1 = Infoblox::Grid::Admin::RadiusAuthServer->new(
                                                            comment  => 'server 1',
                                                            fqdn_or_ip => '10.0.1.1',
                                                            shared_secret => 'secret1',
                                                           );

    my $ads2 = Infoblox::Grid::Admin::RadiusAuthServer->new(
                                                            comment  => 'server 2',
                                                            fqdn_or_ip => 'domain.com',
                                                            shared_secret => 'secret2',
                                                          );
```

#Create the RADIUS Auth Service.

```
    my $service = Infoblox::Grid::Admin::RadiusAuthService->new(
                                                                acct_retries   => 2000,
                                                                auth_retries   => 9,
                                                                cache_ttl      => 500,
                                                                disabled       => 'true',
                                                                enable_cache   => 'true',
                                                                recovery_ttl   => 50,
                                                                selection_mode => 'ROUND_ROBIN',
                                                                acct_timeout   => 1100,
                                                                auth_timeout   => 1200,
                                                                name           => 'some.name.com',
                                                                radius_servers => [$ads1, $ads2]
                                                              );

    unless($service){
           die("Construct service object failed: ",
                   Infoblox::status_code(). ":" .Infoblox::status_detail());
           }
    print "service object object created successfully.\n";

    $session->add($service)
        or die("Add service object failed: ",
                   $session->status_code(). ":" .$session->status_detail());

    print"Service object added successfully.\n";
```

#Get and modify the captive portal property object.

```
    #Get the captive portal object. Considering the id grid member named infoblox.localdomain was previously
created.
    my @object = $session->get(
        object => "Infoblox::Grid::Member::CaptivePortal",
        name   => "infoblox.localdomain"
        )
        or die("get captive portal failed: ",
             $session->status_code(), $session->status_detail());

    #Modify captive portal object.
    unless (scalar(@object) == 0) {
        my $object = $object[0];

        if ($object) {
            $object->authn_server_group($service);
            $object->guest_phone_required('false');
            $object->guest_custom_field1_name('Custom field 1');
            $object->guest_custom_field1_required('true');
            $object->guest_custom_field2_name('Custom field 2');
            $object->guest_custom_field2_required('true');

            # This will remove the AUP uploaded above
            $object->files([]);

            #Applying the changes
            $session->modify($object)
                or die("Modify captive portal failed: ",
                     $session->status_code(), $session->status_detail());

        }
```

```
    } else {
   print "No grid captive portal found.";
    }

####PROGRAM ENDS####
```

AUTHOR

Infoblox Inc. http://www.infoblox.com/

SEE ALSO

Infoblox::Grid::Member::CaptivePortal::File,Infoblox::Grid::Admin::AdAuthService,Infoblox::Grid::Admin::RadiusAuthService,Infoblox::Session,
Infoblox::Session->get(),Infoblox::Session->modify(),Infoblox::LDAP::AuthService

---

COPYRIGHT

Copyright (c) 2017 Infoblox Inc.

Infoblox::Grid::Member::CaptivePortal::File - Captive Portal File object

- NAME
- DESCRIPTION
- MODULE METHODS
  - Infoblox::Grid::Member::CaptivePortal->files( )
- METHODS
  - name( )
  - type( )
- AUTHOR
- SEE ALSO
- COPYRIGHT

NAME

Infoblox::Grid::Member::CaptivePortal::File - Captive Portal File object

DESCRIPTION

This object is used to represent files uploaded to the captive portal. All methods are read-only.

MODULE METHODS

The following functions can be applied to a CaptivePortal::File object.

Infoblox::Grid::Member::CaptivePortal->files( )

Use this function to retrieve a reference to an array of Infoblox::Grid::Member::CaptivePortal::File objects that represent the files uploaded to the captive portal.

Example

```
#Retrieve the captive portal object
my $member = $session->get(object => 'Infoblox::Grid::Member::CaptivePortal', name =>
'cportal.com');

# Access its files
my @files = @{$member->files()};
print $files[0]->name();
```

METHODS

This section describes all the methods that you can use to retrieve the attribute value of a file uploaded to a captive portal.

name( )

Use this method to retrieve the name of the uploaded file.

Returns

The method returns the attribute value.

Example

```
#Get name
my $name = $service_status->name();
```

type( )

Use this method to retrieve the type of the uploaded file.

Returns

The method returns the attribute value. Valid values are "Logo", "Footer", "Header", and "AUP"

Example

```
#Get type
my $type = $service_type->type();
```

AUTHOR

Infoblox Inc. http://www.infoblox.com/

SEE ALSO

Infoblox::Grid::Member::CaptivePortal/>

COPYRIGHT

Copyright (c) 2017 Infoblox Inc.

Infoblox::Grid::Member::Capture::Control - Traffic capture control object

- NAME
- DESCRIPTION
- CONSTRUCTOR
- MODULE METHODS
  - Infoblox::Grid::Member->traffic_capture( )
- METHODS
  - action( )
  - interface( )
  - seconds_to_run( )
- SAMPLE CODE
- AUTHOR
- SEE ALSO
- COPYRIGHT

NAME

Infoblox::Grid::Member::Capture::Control - Traffic capture control object

---

DESCRIPTION

You can control traffic capture for a grid member via the settings in this object.

CONSTRUCTOR

```
my $control = Infoblox::Grid::Member::Capture::Control->new(
              action         => "START" | "STOP",                          # Required
              interface      => "ALL" | "HA" | "LAN1" | "LAN2" | "MGMT",# Required
              seconds_to_run => $num,                                      # Required
);
```

MODULE METHODS

The following functions are available to be applied to a traffic capture control object.

Infoblox::Grid::Member->traffic_capture( )

Use this function to control traffic capture for the Grid Member. See Infoblox::Grid::Member->traffic_capture() for parameters and return values. Note that the member object must be retrieved from the appliance in order for the traffic capture method to be available.

METHODS

This section describes all the methods that you can use to control traffic capture for a grid member.

action( )

Use this method to select the traffic capture action.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

Valid values are "START" or "STOP".

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Setting the action
$control->action("START");
```

interface( )

Use this method to specify the interface on which you capture traffic.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

Valid values are "ALL", "HA", "LAN", "LAN2" or "MGMT".

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Choosing the interface
$control->interface("ALL");
```

seconds_to_run( )

Use this method to select the number of seconds for which you want to run the traffic capture.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

Valid value is an integer specifying the number of seconds.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Setting the capture interfal
$control->seconds_to_run(5);
```

SAMPLE CODE

The following sample code demonstrates the different functions that can be applied to a capture control object. This sample also includes error handling for the operations.

#Preparation prior to traffic capture object operations

```
#PROGRAM STARTS: Include all the modules that will be used

use strict;
use Infoblox;

#Create a session to the Infoblox appliance
my $session = Infoblox::Session->new(
    master   => "192.168.1.2",
    username => "admin",
    password => "infoblox"
);
unless ($session) {
   die("Construct session failed: ",
       Infoblox::status_code() . ":" . Infoblox::status_detail());
}
print "Session created successfully\n";
```

#Start traffic capture for a member

```
# Get the grid member object.
my $object = $session->get(
    object => "Infoblox::Grid::Member",
    name => "infoblox.localdomain",
);

unless ($object) {
      die("get grid member failed: ",
      $session->status_code() . ":" . $session->status_detail());
}
print "grid member get successful\n";
```

```
my $capture = Infoblox::Grid::Member::Capture::Control->new(
                                                            action => 'START',
                                                            interface => 'ALL',
                                                            seconds_to_run => 2,
                                                           );
unless ($object->traffic_capture($capture)) {
      die("traffic capture failed: ",
      $session->status_code() . ":" . $session->status_detail());
}

print "traffic capture started\n";

####PROGRAM ENDS####
```

AUTHOR

Infoblox Inc. http://www.infoblox.com/

SEE ALSO

Infoblox::Session, Infoblox::Grid::Member

COPYRIGHT

Copyright (c) 2017 Infoblox Inc.

Infoblox::Grid::Member::Capture::Status - Traffic capture status object

- NAME
- DESCRIPTION
- MODULE METHODS
  - Infoblox::Grid::Member->traffic_capture( )
- METHODS
  - file_exists( )
  - file_size( )
  - status( )
- SAMPLE CODE
- AUTHOR
- SEE ALSO
- COPYRIGHT

NAME

Infoblox::Grid::Member::Capture::Status - Traffic capture status object

---

DESCRIPTION

You can retrieve the traffic capture status for a grid member via this object.

MODULE METHODS

The following functions are available to be applied to a traffic capture status object.

Infoblox::Grid::Member->traffic_capture( )

Use this function to retrieve the traffic capture status for the Grid Member. See Infoblox::Grid::Member->traffic_capture() for parameters and return values. Note that the member object must be retrieved from the appliance in order for the traffic capture method to be available.

METHODS

This section describes all the methods that explain a grid member's traffic capture status. All members are read-only.

file_exists( )

Use this method to retrieve a traffic capture file if it exists for the member. If the file exists, it can be downloaded using the export_data method in Infoblox::Session.

Returns

The method returns the attribute value, which can be "true" or "false".

Example

```
#Get file_exists
$exists = capstatus->file_exists();
```

file_size( )

Use this method to retrieve the size of the traffic capture file for the member.

Returns

The method returns the attribute value, which is the file size in bytes.

Example

```
#Get the file_size
$size = capstatus->file_size();
```

status( )

Use this method to retrieve the status of the capture operation for the grid member.

Returns

The method returns the attribute value, which can be "RUNNING", "STOPPED" or "UNKNOWN".

Example

```
#Get the status
$status = capstatus->status();
```

SAMPLE CODE

The following sample code demonstrates the different functions that can be applied to a capture status object. This sample also includes error handling for the operations.

#Preparation prior to traffic capture object operations

```
#PROGRAM STARTS: Include all the modules that will be used

use strict;
use Infoblox;

#Create a session to the Infoblox appliance
my $session = Infoblox::Session->new(
    master   => "192.168.1.2",
    username => "admin",
    password => "infoblox"
);
unless ($session) {
   die("Construct session failed: ",
       Infoblox::status_code() . ":" . Infoblox::status_detail());
}
print "Session created successfully\n";
```

#Start traffic capture for a member

```
# Get the grid member object.
my $object = $session->get(
    object => "Infoblox::Grid::Member",
    name => "infoblox.localdomain",
);

unless ($object) {
      die("get grid member failed: ",
      $session->status_code() . ":" . $session->status_detail());
}
print "grid member get successful\n";

my $capture = Infoblox::Grid::Member::Capture::Status->new(
                                                            action => 'START',
                                                            interface => 'ALL',
                                                            seconds_to_run => 2,
                                                           );
unless ($object->traffic_capture($capture)) {
      die("traffic capture failed: ",
      $session->status_code() . ":" . $session->status_detail());
}

print "traffic capture started\n";

my $capstatus = $object->traffic_capture();

print "The capture is " . $capstatus->status() . "\n";

####PROGRAM ENDS####
```

AUTHOR

Infoblox Inc. http://www.infoblox.com/

SEE ALSO

Infoblox::Session, Infoblox::Grid::Member

COPYRIGHT

Copyright (c) 2017 Infoblox Inc.

Grid::Member::Communication - Grid::Member::Communication object.

- NAME
- DESCRIPTION
- MODULE METHODS
  - Infoblox::Grid::Member->member_service_communication( )
- METHODS
  - option( )
  - service( )
  - type( )
- AUTHOR
- SEE ALSO
- COPYRIGHT

NAME

Grid::Member::Communication - Grid::Member::Communication object.

DESCRIPTION

The structure stores information regarding service, transport type and preference.

MODULE METHODS

The following functions can be applied to a communication object.

Infoblox::Grid::Member->member_service_communication( )

Use this function to retrieve a reference to an array of Infoblox::Grid::Member::Communication objects that represents the communication settings for an Infoblox appliance.

Example

```
#Retrieve the member object
my $member = $session->get(object => 'Infoblox::Grid::Member', name => 'hostname.com');

# Access its communication settings
my @communication = @{$member->member_service_communication()};
print $communications[0]->type();
```

METHODS

This section describes all the methods that you can use to set or retrieve the attribute values of the object.

option( )

Use this method to retrieve the flag when the service is either forced to use one type of transport or have a preference for the transport type. This is a read-only attribute.

Parameter

None

Returns

The method returns the attribute value, which can be either "FORCE" or "PREFER".

Example

```
#Get option
my $option = $comm->option();
```

service( )

Use this method to set or retrieve services for which communication type can be configured.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

The valid value is one of the following: "GRID", "GRID_BACKUP", "NTP", "REPORTING", "REPORTING_BACKUP", "OCSP", "MAIL",

"AD".

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
# Get service
my $service = $comm->service();
# Modifying service
$comm->service("GRID");
```

type( )

Use this method to set or retrieve the transport type.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

The valid value is IPv4 or IPv6.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
# Get type
my $type = $comm->type();
# Modifying type
$comm->type("IPv4");
```

AUTHOR

Infoblox Inc. http://www.infoblox.com/

SEE ALSO

Infoblox::Grid::Member/>

COPYRIGHT

Copyright (c) 2017 Infoblox Inc.

Infoblox::Grid::Member::DHCP - DHCP member object.

- NAME
- DESCRIPTION
- CONSTRUCTOR
- SESSION METHODS
  - Infoblox::Session->get( )
  - Infoblox::Session->modify( )
- METHODS
  - auth_server_group( )
  - authority( )
  - authn_captive_portal
  - authn_captive_portal_authenticated_filter
  - authn_captive_portal_enabled
  - authn_captive_portal_guest_filter
  - auth_server_group_enabled
  - bootfile( )
  - bootserver( )
  - ddns_generate_hostname( )
  - ddns_retry_interval( )
  - ddns_server_always_updates( )
  - ddns_ttl( )
  - ddns_domainname( )
  - ddns_update_fixed_addresses( )
  - ddns_use_option81( )
  - ddns_zone_primaries( )
  - deny_bootp( )
  - dns_update_style( )
  - email_list( )
  - enable_ddns( )
  - enable_dhcp( )
  - enable_dhcpv6_service( )
  - enable_dhcp_on_lan2( )
  - enable_dhcp_on_ipv6_lan2( )
  - enable_dhcp_thresholds( )
  - enable_email_warnings( )
  - enable_fingerprint( )
  - enable_gss_tsig( )
  - enable_hostname_rewrite( )
  - enable_ifmap( )
  - enable_ifmap_publishing( )
  - enable_snmp_warnings( )
  - extattrs( )
  - extensible_attributes( )
  - gss_tsig_keys( )
  - high_water_mark( )
  - high_water_mark_reset( )
  - hostname_rewrite_policy( )
  - ifmap_client_credentials( )
  - ifmap_last_status_update( )
  - ifmap_password( )
  - ifmap_status( )
  - ifmap_status_details( )
  - ifmap_username( )
  - ignore_id( )
  - ignore_mac_addresses( )
  - ignore_dhcp_option_list_request( )
  - ipv6_ddns_enable_option_fqdn( )
  - ipv6_ddns_server_always_updates( )
  - ipv6_ddns_domainname( )
  - ipv6_ddns_hostname( )
  - ipv6_ddns_ttl( )
  - ipv6_dns_update_style( )
  - ipv6_domain_name( )
  - ipv6_domain_name_servers( )
  - ipv6_enable_ddns( )
  - ipv6_enable_gss_tsig( )
  - ipv6_enable_lease_scavenge( )
  - ipv6_remember_expired_client_association( )
  - ipv6_enable_retry_updates( )
  - ipv6_generate_hostname( )
  - ipv6_gss_tsig_keys( )
  - ipv6_kdc_server( )

- ipv6_lease_scavenge_time( )
- ipv6_microsoft_code_page( )
- ipv6_options( )
- ipv6_recycle_leases( )
- ipv6_retry_updates_interval( )
- ipv6_server_duid( )
- ipv6_update_dns_on_lease_renewal( )
- kdc_server( )
- ipv6_lease_scavenge_time( )
- lease_per_client_settings( )
- lease_scavenge_time( )
- log_lease_events( )
- logic_filters( )
- low_water_mark( )
- low_water_mark_reset( )
- override_dns_update_style( )
- override_enable_ifmap_publishing( )
- override_gss_tsig_keys( )
- override_ipv6_dns_update_style( )
- override_ipv6_gss_tsig_keys( )
- override_ipv6_lease_scavenge( )
- override_logic_filters( )
- name( )
- nextserver( )
- microsoft_code_page( )
- options( )
- option60_match_rules( )
- override_ddns_ttl( )
- override_enable_dhcpv6_service( )
- override_enable_fingerprint( )
- override_hostname_rewrite( )
- override_ignore_id( )
- override_ipv6_ddns_domainname( )
- override_ipv6_ddns_hostname( )
- override_ipv6_ddns_enable_option_fqdn( )
- override_ipv6_ddns_ttl( )
- override_ipv6_domain_name( )
- override_ipv6_domain_name_servers( )
- override_ipv6_enable_ddns( )
- override_ipv6_enable_gss_tsig( )
- override_ipv6_enable_retry_updates( )
- override_ipv6_generate_hostname( )
- override_ipv6_microsoft_code_page( )
- override_ipv6_options( )
- override_ipv6_recycle_leases( )
- override_ipv6_update_dns_on_lease_renewal( )
- override_lease_per_client_settings( )
- override_lease_scavenge_time( )
- override_log_lease_events( )
- override_preferred_lifetime( )
- override_update_dns_on_lease_renewal( )
- override_valid_lifetime( )
- ping_count( )
- ping_timeout( )
- preferred_lifetime( )
- pxe_lease_time( )
- recycle_leases( )
- retry_ddns_updates( )
- syslog_facility( )
- update_dns_on_lease_renewal( )
- valid_lifetime( )

- SAMPLE CODE
- AUTHOR
- SEE ALSO
- COPYRIGHT

NAME

Infoblox::Grid::Member::DHCP - DHCP member object.

DESCRIPTION

When you configure an Infoblox appliance to function as a DHCP server, you can set certain properties that control how the server

operates. You can also specify the configuration information the server includes in its DHCP messages. When a DHCP server assigns an IP address to a client, it can include configuration information the client needs to connect to the network and communicate with other hosts and appliances in the network.

Though you can set general DHCP properties at the grid level, you enable DHCP services at the member level only. Infoblox recommends that you configure the DHCP parameters before you enable DHCP on the Infoblox appliance.

CONSTRUCTOR

The Infoblox::Grid::Member::DHCP object is automatically generated upon the successful installation of the Infoblox appliance. It does not require manual construction.

SESSION METHODS

This section describes all the methods in the Infoblox::Session module that you can apply to a member DHCP object.

Infoblox::Session->get( )

Use this method to retrieve matching objects from the Infoblox appliance. See Infoblox::Session->get() for parameters and return values.

Key Reference

Apply the following attribute to get a specific member DHCP object:

```
name - Optional. A domain name in string format.
ipv4addr - Optional.   IPv4 address of the member
ipv6addr - Optional.   IPv6 address of the member
extattrs     - Optional. A hash reference containing extensible attributes.
extensible_attributes - Optional. A hash reference containing extensible attributes.
```

Example

```
my @retrieved_objs = $session->get(
    object => "Infoblox::Grid::Member::DHCP",
    name   => "infoblox.localdomain"
    );

# get all Grid::Member::DHCP objects with a given extensible attribute
my @retrieved_objs = $session->get(
    object => "Infoblox::Grid::Member::DHCP",
    extensible_attributes => { 'Site' => 'Santa Clara' });
```

Infoblox::Session->modify( )

Use this method to modify objects in the Infoblox appliance. See Infoblox::Session->modify() for parameters and return values.

Example

```
#Use method to modify dhcp service.
$member_dhcp->enable_dhcp("true");
#Submit modification
my $response = $session->modify( $member_dhcp );
```

METHODS

This section describes all the methods that you can use to configure and retrieve the attribute values of the DHCP properties of a grid member.

auth_server_group( )

Use this method to set or retrieve the Authentication Server Group object associated with this member.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

An Infoblox::Grid::Admin::RadiusAuthService object.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Getting auth_server_group

my $auth_server_group = $memberdhcp->auth_server_group( );
#Modifying auth_server_group
$auth_server_group->$memberdhcp($radius_service);
```

authority( )

Use this method to set or retrieve the authority flag of a grid member. This flag specifies if a DHCP server is authoritative for a domain.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

Specify "true" to set the authority flag or "false" to deactivate/unset it.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get authority
my $authority = $memberdhcp->authority();
#Modify authority
$memberdhcp->authority("true");
```

authn_captive_portal

The captive portal responsible for authentications for this DHCP member.

Parameter

The valid value is a name in FQDN format.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get authn_captive_portal
my $authn_captive_portal = $memberdhcp->authn_captive_portal();
#Modify authn_captive_portal
$memberdhcp->authn_captive_portal("cpmember.com");
```

authn_captive_portal_authenticated_filter

The MAC filter representing the authenticated range.

Parameter

The valid value is an Infoblox::DHCP::Filter::MAC object.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get authn_captive_portal_authenticated_filter
my $authn_captive_portal_authenticated_filter = $memberdhcp-
>authn_captive_portal_authenticated_filter();
#Modify authn_captive_portal_authenticated_filter
$memberdhcp->authn_captive_portal_authenticated_filter($macfilter);
```

authn_captive_portal_enabled

Use this method to set or retrieve the flag that controls if this DHCP member is enabled for captive portal authentication.

Parameter

Specify "true" to enable this feature or "false" to disable it. The default value is "false".

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get authn_captive_portal_enabled
my $authn_captive_portal_enabled = $memberdhcp->authn_captive_portal_enabled();
#Modify authn_captive_portal_enabled
$memberdhcp->authn_captive_portal_enabled("true");
```

authn_captive_portal_guest_filter

The MAC filter representing the guest range.

Parameter

The valid value is an Infoblox::DHCP::Filter::MAC object.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get authn_captive_portal_guest_filter
my $authn_captive_portal_guest_filter = $memberdhcp->authn_captive_portal_guest_filter();
#Modify authn_captive_portal_guest_filter
$memberdhcp->authn_captive_portal_guest_filter($macfilter);
```

auth_server_group_enabled

Use this method to set or retrieve the flag that controls if this DHCP member can send authentication requests to an authentication server group.

Parameter

Specify "true" to enable this feature or "false" to disable it. The default value is "false".

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get auth_server_group_enabled
my $auth_server_group_enabled = $memberdhcp->auth_server_group_enabled();
#Modify auth_server_group_enabled
$memberdhcp->auth_server_group_enabled("true");
```

bootfile( )

Use this method to set or retrieve the name of a file that DHCP clients need to boot. This setting overrides the grid level setting.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

This option is overridden independently from Infoblox::Grid::Member::DHCP->bootserver() and Infoblox::Grid::Member::DHCP->nextserver().

Parameter

The name of the file that the client must download. The default value is undefined.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get bootfile
my $bootfile = $memberdhcp->bootfile();
#Modify bootfile
$memberdhcp->bootfile("bootfile.txt");
#Un-override bootfile
$memberdhcp->bootfile(undef);
```

bootserver( )

Use this method to set or retrieve the name of the server on which a boot file is stored. This setting overrides the grid level setting.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

This option is overridden independently from Infoblox::Grid::Member::DHCP->bootfile() and Infoblox::Grid::Member::DHCP->nextserver().

Parameter

The boot server IP address or name in FQDN (Fully Qualified Domain Name) format. The FQDN consists of the host name followed by the domain name (example: abc.com). A boot server name can have a maximum of 256 bytes. The default value is undefined.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get bootserver
my $bootserver = $memberdhcp->bootserver();
#Modify bootserver
$memberdhcp->bootserver("hostname.com");
#Un-override bootserver
$memberdhcp->bootserver(undef);
```

ddns_generate_hostname( )

Use this method to set or retrieve the ddns_generate_hostname value. Set this value to enable/disable the ability of a member DHCP server to generate a host name and update DNS with this host name when it receives a DHCP REQUEST message that does not include a host name.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

Specify "true" to enable this feature or "false" to disable it.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get ddns_generate_hostname
my $ddns_generate_hostname = $memberdhcp->ddns_generate_hostname();
#Modify disable
$memberdhcp->ddns_generate_hostname("true");
```

ddns_retry_interval( )

Use this method to set or retrieve the ddns_retry_interval value. Set the retry interval when the member DHCP server makes repeated attempts to send DDNS updates to a DNS server.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

A 32-bit integer (range from 0 to 4294967295) that represents the retry interval in minutes. The default retry interval is five minutes.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get ddns_retry_interval
my $ddns_retry_interval = $memberdhcp->ddns_retry_interval();
#Modify ddns_retry_interval
$memberdhcp->ddns_retry_interval(10);
```

ddns_server_always_updates( )

Use this method to set or retrieve the ddns_server_always_updates value of a member DHCP server. Select this check box to allow only the DHCP server to update DNS,regardless of the requests from the DHCP clients. This setting overrides the grid level setting.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

Specify "true" to set ddns_server_always_updates flag or "false" to deactivate/unset it.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get ddns_server_always_updates
my $ddns_server_always_updates = $memberdhcp->ddns_server_always_updates();
#Modify ddns_server_always_updates
$memberdhcp->ddns_server_always_updates("true");
```

ddns_ttl( )

Use this method to set or retrieve the ddns_ttl value of a member DHCP server. The DDNS TTL (Dynamic DNS Time To Live) value specifies the number of seconds an IP address for the name is cached.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Setting this method to a defined value implicitly sets the override_ddns_ttl method to "true". Setting the parameter to undefined causes the appliance to use the Grid default and automatically resets the override_ddns_ttl attribute to "false".

Note that when ddns_ttl is set to a defined value and override_dddns_ttl is set to "false", the last operation takes precedence. Thus the sequence $object->ddns_ttl(12); $object->override_ddns_ttl("false"); will set override_ddns_ttl to "false", and the sequence $object->override_ddns_ttl("false"); $object->ddns_ttl(12); will result in override_ddns_ttl="true".

Parameter

A 32-bit integer (range from 0 to 4294967295) that specifies the number of seconds the IP address for the name is cached. Default value is half of lease time.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get ddns_ttl
my $ddns_ttl = $memberdhcp->ddns_ttl();
#Modify ddns_ttl
$memberdhcp->ddns_ttl(1000);
```

ddns_domainname( )

Use this method to set or retrieve the member DDNS domain name value.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Setting this method to a defined value implicitly sets the override_ddns_domainname method to "true". Setting the parameter to undefined causes the appliance to use the Grid default and automatically resets the override_ddns_domainname attribute to "false".

Note that when ddns_domainname is set to a defined value and override_ddns_domainname is set to "false", the last operation takes precedence. Thus the sequence $object->ddns_domainname("domain.com"); $object->override_ddns_domainname("false"); will set override_ddns_domainname to "false", and the sequence $object->override_ddns_domainname("false"); $object->ddns_domainname("domain.com"); will result in override_ddns_domainname="true".

Parameter

The DDNS domain name in FQDN (Fully Qualified Domain Name) format.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get ddns_domainname
my $ddns_domainname = $memberdhcp->ddns_domainname();
#Modify ddns_domainname
$memberdhcp->ddns_domainname("domain.com");
```

ddns_update_fixed_addresses( )

Use this method to set or retrieve the ddns_update_fixed_addresses value. Set this value to enable/disable the member DHCP server to update the A and PTR record of clients with a fixed address.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

Specify "true" to enable the member DHCP server to update the A and PTR record of clients with a fixed address or "false" to disable it.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get ddns_update_fixed_addresses
my $ddns_update_fixed_addresses = $memberdhcp->ddns_update_fixed_addresses();
#Modify ddns_update_fixed_addresses
$memberdhcp->ddns_update_fixed_addresses("true");
```

ddns_use_option81( )

Use this method to set or retrieve the ddns_use_option81 value of a member DHCP server. Set this value to enable/disable support for option 81.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

Specify "true" to enable support for option 81 or "false" to disable it.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get ddns_use_option81
my $ddns_use_option81 = $memberdhcp->ddns_use_option81();
#Modify ddns_use_option81
$memberdhcp->ddns_use_option81("true");
```

ddns_zone_primaries( )

Use this method to set or retrieve an ordered list of zone primaries that will receive DDNS updates.

Parameter

A valid value is a reference to an array of Infoblox::DHCP::DDNS objects.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
# Get the attribute value
$value = $memberdhcp->ddns_zone_primaries();
# Modify the attribute value
$memberdhcp->ddns_zone_primaries([$ddns]);
```

deny_bootp( )

Use this method to set or retrieve the deny_bootp value of a member DHCP server. Set this value to enable/disable deny_bootp and Grid level.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

Specify "true" to enable deny_bootp or "false" to disable it.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get deny_bootp
my $deny_bootp = $memberdhcp->deny_bootp();
#Modify deny_bootp
$memberdhcp->deny_bootp("true");
```

dns_update_style( )

Use this method to set or retrieve the update style for dynamic DNS updates (DHCPv4).

Setting this method to a defined value implicitly sets the override_dns_update_style method to 'true'. Setting the parameter to undefined causes the appliance to use the Grid default and automatically resets the override_dns_update_style attribute to 'false'.

Note that when dns_update_style is set to a defined value and override_dns_update_style is set to 'false', the last operation takes precedence. Thus the sequence $object->dns_update_style('INTERIM'); $object->override_dns_update_style('false'); will set override_dns_update_style to 'false', and the sequence $object->override_dns_update_style('false'); $object->dns_update_style('INTERIM'); will result in override_dns_update_style='true'.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

Valid values are 'INTERIM' and 'STANDARD'.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get dns_update_style value
my $value = $object->dns_update_style();

#Modify dns_update_style value
$object->dns_update_style('STANDARD');
```

email_list( )

Use this method to set or retrieve the email_list value of a member DHCP server.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

Array reference with the e-mail values.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get email_list
my $email_list = $memberdhcp->email_list();
#Modify email_list
$memberdhcp->email_list(["admin\@infoblox.com"]);
```

enable_ddns( )

Use this method to set or retrieve the enable_ddns value of a member DHCP server. Set this value to enable/disable the member DHCP server to send DDNS updates.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

Specify "true" to enable support for DDNS updates or "false" to disable it.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get enable_ddns
my $enable_ddns= $memberdhcp->enable_ddns();
#Modify enable_ddns
$memberdhcp->enable_ddns("true");
```

enable_dhcp( )

Use this method to set or retrieve the enable_dhcp value of a member. Set this value to enable/disable DHCP service.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

Specify "true" to enable DHCP or "false" to disable it.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get enable_dhcp
my $enable_dhcp= $memberdhcp->enable_dhcp();
#Modify enable_dhcp
$memberdhcp->enable_dhcp("true");
```

enable_dhcpv6_service( )

Use this method to set or retrieve the enable_dhcpv6_service flag. Set this flag to enable DHCPv6 service for the member.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Setting this method to a defined value implicitly sets the override_enable_dhcpv6_service method to "true". Setting the parameter to undefined causes the appliance to use the Grid default and automatically resets the override_enable_dhcpv6_service attribute to "false".

Note that when enable_dhcpv6_service is set to a defined value and override_denable_dhcpv6_service is set to "false", the last operation takes precedence. Thus the sequence $object->enable_dhcpv6_service("true"); $object->override_enable_dhcpv6_service("false"); will set override_enable_dhcpv6_service to "false", and the sequence $object->override_enable_dhcpv6_service("false"); $object->enable_dhcpv6_service("true"); will result in override_enable_dhcpv6_service="true".

Parameter

Specify "true" to enable this feature or "false" to disable it. The default value is "false".

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get enable_dhcpv6_service
my $enable_dhcpv6_service = $memberdhcp->enable_dhcpv6_service();
#Modify enable_dhcpv6_service
$memberdhcp->enable_dhcpv6_service("true");
```

enable_dhcp_on_lan2( )

Use this method to set or retrieve the enable_dhcp_on_lan2 value of a member. Set this value to enable/disable DHCP service on the LAN2 interface.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

Specify "true" to enable DHCP on the LAN2 interface or "false" to disable it.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get enable_dhcp_on_lan2
my $enable_dhcp_on_lan2 = $memberdhcp->enable_dhcp_on_lan2();
#Modify enable_dhcp_on_lan2
$memberdhcp->enable_dhcp_on_lan2("true");
```

enable_dhcp_on_ipv6_lan2( )

Use this method to set or retrieve the enable_dhcp_on_ipv6_lan2 value of a member. Set this value to enable/disable DHCP service on the IPv6 LAN2 interface.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

Specify "true" to enable DHCP on the IPv6 LAN2 interface or "false" to disable it.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get enable_dhcp_on_ipv6_lan2
my $enable_dhcp_on_ipv6_lan2 = $memberdhcp->enable_dhcp_on_ipv6_lan2();
#Modify enable_dhcp_on_ipv6_lan2
$memberdhcp->enable_dhcp_on_ipv6_lan2("true");
```

enable_dhcp_thresholds( )

Use this method to set or retrieve the enable_dhcp_thresholds value of a member DHCP server. Thresholds represents the watermarks above or below which address usage in a network is unexpected and might warrant your attention. This setting overrides the grid level setting.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

Specify "true" to enable DHCP thresholds or "false" to disable them.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get enable_dhcp_thresholds
my $enable_dhcp_thresholds = $memberdhcp->enable_dhcp_thresholds();
#Modify enable_dhcp_thresholds
$memberdhcp->enable_dhcp_thresholds("true");
```

enable_email_warnings( )

Use this method to set or retrieve the enable_email_warnings value of a member DHCP server. If we enable it and DHCP threshold is enabled, the Infoblox appliance sends an e-mail notification to an administrator if DHCP address usage crosses a high or low watermark threshold.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

Specify "true" to set enable_email_warnings flag or "false" to deactivate/unset it.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get enable_email_warnings
my $enable_email_warnings = $memberdhcp->enable_email_warnings();
#Modify enable_email_warnings
$memberdhcp->enable_email_warnings("true");
```

enable_fingerprint( )

Use this method to set or retrieve the member-level enable_fingerprint flag. If you enable this feature, the server will match a fingerprint for incoming lease requests.

Setting this method to a defined value implicitly sets the override_enable_fingerprint method to "true". Setting the parameter to undefined causes the appliance to use the Grid default and automatically resets the override_enable_fingerprint attribute to "false".

Parameter

Specify "true" to set the enable_fingerprint flag or "false" to deactivate/unset it. The default value is "true".

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get enable_fingerprint
my $enable_fingerprint = $memberdhcp->enable_fingerprint();
#Modify enable_fingerprint
$memberdhcp->enable_fingerprint("false");
```

enable_gss_tsig( )

Use this method to set or retrieve the member-level enable_gss_tsig flag.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

This is for DHCPv4 GSS-TSIG authentication.

Parameter

Specify "true" to enable GSS-TSIG authentication for DHCP IPv4 or "false" to disable it. The default value is undefined. Setting this flag overrides the enable_gss_tsig attribute of the grid.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get enable_gss_tsig
my $enable_gss_tsig = $memberdhcp->enable_gss_tsig();
#Modify enable_gss_tsig
$memberdhcp->enable_gss_tsig("true");
```

enable_hostname_rewrite( )

Use this method to set or retrieve the grid member's enable_hostname_rewrite flag.

Parameter

Boolean to enable or disable hostname rewrite for the member. The default value is "false".

Cannot enable the hostname rewrite policy along with the Microsoft code page. See microsoft_code_page().

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get enable_hostname_rewrite
$status = $memberdhcp->enable_hostname_rewrite();
#Modify enable_hostname_rewrite
$memberdhcp->enable_hostname_rewrite("true");
```

enable_ifmap( )

Use this method to enable or disable the IF-MAP DHCP client feature on this member. Enabling this feature requires a restart of DHCP services.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

Boolean to enable or disable the IF-MAP client. The default value is "false".

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get IF-MAP enable value
my $ifmap_enabled = $memberdhcp->enable_ifmap();
#Enable IF-MAP Client feature
$memberdhcp->enable_ifmap("true");
```

enable_ifmap_publishing( )

Use this method to set or retrieve the ifmap publishing option of a DHCP Member object. This setting overrides the member level settings.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

Specify "true" to set ifmap publishing option or "false" to deactivate/unset it. The default value is 'false', which indicates that this attribute inherits the member level setting.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
# Get enable_ifmap_publishing
my $enable_ifmap_publishing = $memberdhcp->enable_ifmap_publishing()
# Modify enable_ifmap_publishing
$memberdhcp->enable_ifmap_publishing("true");
# Un-override enable_ifmap_publishing
$memberdhcp->enable_ifmap_publishing(undef);
```

enable_snmp_warnings( )

Use this method to set or retrieve the enable_snmp_warnings value of member DHCP server. When DHCP threshold is enabled and DHCP address usage crosses a watermark threshold, the appliance sends an SNMP trap to the trap receiver that you defined for the grid member level.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

Specify "true" to set enable_snmp_warnings flag or "false" to deactivate/unset it.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get enable_snmp_warnings
my $enable_snmp_warnings = $memberdhcp->enable_snmp_warnings();
#Modify enable_snmp_warnings
$memberdhcp->enable_snmp_warnings("true");
```

extattrs( )

Use this method to retrieve the extensible attributes associated with a Grid Member DHCP object. This attribute is read-only and cannot be set.

Parameter

None.

Returns

The method returns the attribute value.

Example

```
#Get extattrs
my $ref_extattrs = $memberdhcp->extattrs();
```

extensible_attributes( )

Use this method to retrieve the extensible attributes associated with a Grid Member DHCP object.

Parameter

none

Returns

The method returns the attribute value.

Example

```
#Get extensible attributes
my $ref_extensible_attributes = $memberdhcp->extensible_attributes();
```

gss_tsig_keys( )

Use this method to set or retrieve the list of GSS-TSIG keys for a member DHCPv4 object.

Setting this method to a defined value implicitly sets the override_gss_tsig_keys method to "true". Setting the parameter to undefined causes the appliance to use the Grid default and automatically resets the override_gss_tsig_keys method to "false".

Note that when gss_tsig_keys is set to a defined value and override_gss_tsig_keys is set to "false", the last operation takes precedence. Thus the sequence $object->gss_tsig_keys($keys); $object->override_gss_tsig_keys("false"); will set override_gss_tsig_keys to "false", and the sequence $object->override_gss_tsig_keys("false"); $object->gss_tsig_keys($keys); will result in override_gss_tsig_keys="true".

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

The valid value is an array reference that contains Infoblox::Grid::KerberosKey objects.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get the Kerberos keys list
my $kerberos_keys_list = $member_dhcp->gss_tsig_keys();
```

```
#Modify the Kerberos keys list
my $kerberos_key1 = Infoblox::Cluster::KerberosKey->new(
       principal => 'host/appliance.ivan.test1@LADYBUG.LOCAL',
       );
my $kerberos_key2 = Infoblox::Cluster::KerberosKey->new(
       principal => 'host/appliance.ivan.test2@LADYBUG.LOCAL',
       );
$member_dhcp->gss_tsig_keys([$kerberos_key1, $kerberos_key2]);
```

high_water_mark( )

Use this method to set or retrieve the high_water_mark value of a member DHCP server. If the percent of allocated addresses exceeds this watermark, the appliance makes a syslog entry and if enabled, sends an e-mail notification.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

Number with the percentage value. Range is from 1 to 100. The high watermark value must be greater than the low watermark value. Default value is 95.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get high_water_mark
my $high_water_mark = $memberdhcp->high_water_mark();
#Modify high_water_mark
$memberdhcp->high_water_mark(80);
```

high_water_mark_reset( )

Use this method to set or retrieve the high_water_mark_reset value of a member DHCP server. If the percentage of allocated addresses drops below this value, a corresponding SNMP trap is reset.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

Number with the percentage value. Range is from 1 to 100. The high watermark value must be greater than the low watermark value. Default value is 85.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get high_water_mark_reset
my $high_water_mark_reset = $memberdhcp->high_water_mark_reset();
#Modify high_water_mark_reset
$memberdhcp->high_water_mark_reset(80);
```

hostname_rewrite_policy( )

Use this method to set or retrieve the name of the hostname rewrite policy that is in the protocol_hostname_rewrite_policies array of the Grid DHCP object. This attribute is mandatory if enable_hostname_rewrite is "true".

Parameter

The name, in string format, of the hostname rewrite policy.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get hostname_rewrite_policy
```

```
my $name = $memberdhcp->hostname_rewrite_policy();
#Modify hostname_rewrite_policy
$memberdhcp->hostname_rewrite_policy('new_policy');
```

ifmap_client_credentials( )

Use this method to set the IF-MAP client credentials used by the DHCP IF-MAP client on this member.

Include the specified parameter to set the attribute value.

Parameter

IF-MAP client credentials. The default value is undefined.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Create new IF-MAP client credentials
my $credential = Infoblox::IFMap::Client::Credentials->new(
                                                            authentication => 'basic',
                                                            username       => 'something',
                                                            password       => 'else',
                                                           );

#Modify IF-MAP client credentials
$memberdhcp->ifmap_client_credentials($credentials);
```

ifmap_last_status_update( )

Use this method to get the last status update of the IF-MAP member.

Parameter

None

Returns

The method returns the attribute value.

Example

```
#Get IF-MAP member last status update
$ifmap_last_status_update = $memberdhcp->ifmap_last_status_update();
```

ifmap_password( )

Use this method to set the IF-MAP client password used by the DHCP IF-MAP client on this member. The value can only be set or modified, but not retrieved. Note that use of this member has now been deprecated.

Include the specified parameter to set the attribute value.

Parameter

String for the IF-MAP client password. The default value is undef.

Returns

The method returns true when the modification succeeds, and returns false when the operation fails.

Example

```
#Modify IF-MAP client password
$memberdhcp->ifmap_password("secret");
```

ifmap_status( )

Use this method to get the IF-MAP member status.

Parameter

None

Returns

The method returns the attribute value.

Example

```
#Get IF-MAP member status
$ifmap_status = $memberdhcp->ifmap_status();
```

ifmap_status_details( )

Use this method to get the status details of the IF-MAP member.

Parameter

None

Returns

The method returns the attribute value.

Example

```
#Get IF-MAP member status details
$ifmap_status_details = $memberdhcp->ifmap_status_details();
```

ifmap_username( )

Use this method to set or retrieve the IF-MAP client username used by the DHCP IF-MAP client on this member. Note that use of this member has now been deprecated.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

String for the IF-MAP client username. The default value is undef.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get IF-MAP client username
my $username = $memberdhcp->ifmap_username();
#Modify IF-MAP client username
$memberdhcp->ifmap_username("ifmapclient");
```

ignore_id( )

Use this method to set or retrieve the ignore_id type of a grid member DHCP object. Indicates whether the appliance will ignore DHCP client IDs or MAC addresses. This setting overrides the upper-level settings.

Parameter

Valid values are 'NONE', 'CLIENT', or 'MACADDR'. The default is 'NONE'.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
# Get attribute value
$value = $memberdhcp->ignore_id();
# Modify attribute value
$memberdhcp->ignore_id('CLIENT');
#Un-override ignore_id
$memberdhcp->ignore_id(undef);
```

ignore_mac_addresses( )

Use this method to set or retrieve the ignore_mac_addresses list of a grid member DHCP object. A list of MAC addresses the appliance will ignore. This setting overrides the upper-level settings.

If you enter MAC addresses in the ignore_mac_addresses, then the appliance will ignore them. If you enter empty array, then the

appliance will ignore any MAC address.

Parameter

The valid value is an array reference that contains string objects.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
# Get attribute value
$value = $memberdhcp->ignore_mac_addresses();
# Modify attribute value
$memberdhcp->ignore_mac_addresses(["11::11::11::11::11::11"]);
```

ignore_dhcp_option_list_request( )

Use this method to set or retrieve the ignore_dhcp_option_list_request flag of a grid member DHCP. If this flag is set to true all available DHCP options will be returned to the client.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

Specify "true" to set the ignore_dhcp_option_list_request flag or "false" to deactivate/unset it.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get ignore_dhcp_option_list_request
my $ignore_dhcp_option_list_request = $memberdhcp->ignore_dhcp_option_list_request();
#Modify ignore_dhcp_option_list_request
$memberdhcp->ignore_dhcp_option_list_request("false");
#Do not override ignore_dhcp_option_list_request
$memberdhcp->ignore_dhcp_option_list_request(undef);
```

ipv6_ddns_enable_option_fqdn( )

Use this method to set or retrieve the ipv6_ddns_enable_option_fqdn value of a member DHCP server. This method controls whether the FQDN option sent by the client is to be used, or if the server can automatically generate the FQDN.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Setting this method to a defined value implicitly sets the override_ipv6_ddns_enable_option_fqdn method to "true". Setting the parameter to undefined causes the appliance to use the upper-level default and automatically resets the override_ipv6_ddns_enable_option_fqdn attribute to "false".

Note that when ipv6_ddns_enable_option_fqdn is set to a defined value and override_ipv6_ddns_enable_option_fqdn is set to "false", the last operation takes precedence. Thus the sequence $object->ipv6_ddns_enable_option_fqdn("true"); $object->override_ipv6_ddns_enable_option_fqdn("false"); will set override_ipv6_ddns_enable_option_fqdn to "false", and the sequence $object->override_ipv6_ddns_enable_option_fqdn("false"); $object->ipv6_ddns_enable_option_fqdn("true"); will result in override_ipv6_ddns_enable_option_fqdn="true".

Parameter

A 32-bit integer (range from 0 to 4294967295) that specifies the number of seconds the IP address for the name is cached. Default value is half of lease time.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get ipv6_ddns_enable_option_fqdn
my $ipv6_ddns_enable_option_fqdn = $memberdhcp->ipv6_ddns_enable_option_fqdn();
#Modify ipv6_ddns_enable_option_fqdn
$memberdhcp->ipv6_ddns_enable_option_fqdn("true");
```

ipv6_ddns_server_always_updates( )

Use this method to set or retrieve the ipv6_ddns_server_always_updates flag. Set this flag to "false" to update DNS only if requested by the client.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

Specify "true" to enable this feature or "false" to disable it. The default value is "false".

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get ipv6_ddns_server_always_updates
my $ipv6_ddns_server_always_updates = $memberdhcp->ipv6_ddns_server_always_updates();
#Modify ipv6_ddns_server_always_updates
$memberdhcp->ipv6_ddns_server_always_updates("true");
```

ipv6_ddns_domainname( )

Use this method to set or retrieve the member IPv6 DDNS domain name value.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Setting this method to a defined value implicitly sets the override_ipv6_ddns_domainname method to "true". Setting the parameter to undefined causes the appliance to use the Grid default and automatically resets the override_ipv6_ddns_domainname attribute to "false".

Note that when ipv6_ddns_domainname is set to a defined value and override_dipv6_ddns_domainname is set to "false", the last operation takes precedence. Thus the sequence $object->ipv6_ddns_domainname("domain.com"); $object->override_ipv6_ddns_domainname("false"); will set override_ipv6_ddns_domainname to "false", and the sequence $object->override_ipv6_ddns_domainname("false"); $object->ipv6_ddns_domainname("domain.com"); will result in override_ipv6_ddns_domainname="true".

Parameter

The DDNS domain name in FQDN (Fully Qualified Domain Name) format.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get ipv6_ddns_domainname
my $ipv6_ddns_domainname = $memberdhcp->ipv6_ddns_domainname();
#Modify ipv6_ddns_domainname
$memberdhcp->ipv6_ddns_domainname("domain.com");
```

ipv6_ddns_hostname( )

Use this method to set or retrieve the member IPv6 DDNS hostname value.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Setting this method to a defined value implicitly sets the override_ipv6_ddns_hostname method to "true". Setting the parameter to undefined causes the appliance to use the Grid default and automatically resets the override_ipv6_ddns_hostname attribute to "false".

Note that when ipv6_ddns_hostname is set to a defined value and override_dipv6_ddns_hostname is set to "false", the last operation takes precedence. Thus the sequence $object->ipv6_ddns_hostname("host.com"); $object->override_ipv6_ddns_hostname("false"); will set override_ipv6_ddns_hostname to "false", and the sequence $object->override_ipv6_ddns_hostname("false"); $object->ipv6_ddns_hostname("host.com"); will result in override_ipv6_ddns_hostname="true".

Parameter

The DDNS hostname in FQDN (Fully Qualified Domain Name) format.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get ipv6_ddns_hostname
my $ipv6_ddns_hostname = $memberdhcp->ipv6_ddns_hostname();
#Modify ipv6_ddns_hostname
$memberdhcp->ipv6_ddns_hostname("host.com");
```

ipv6_ddns_ttl( )

Use this method to set or retrieve the member IPv6 DDNS TTL value.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Setting this method to a defined value implicitly sets the override_ipv6_ddns_ttl method to "true". Setting the parameter to undefined causes the appliance to use the Grid default and automatically resets the override_ipv6_ddns_ttl attribute to "false".

Note that when ipv6_ddns_ttl is set to a defined value and override_dipv6_ddns_ttl is set to "false", the last operation takes precedence. Thus the sequence $object->ipv6_ddns_ttl(12345); $object->override_ipv6_ddns_ttl("false"); will set override_ipv6_ddns_ttl to "false", and the sequence $object->override_ipv6_ddns_ttl("false"); $object->ipv6_ddns_ttl(12345); will result in override_ipv6_ddns_ttl="true".

Parameter

The preferred DDNS TTL value in seconds. The default is undefined.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get ipv6_ddns_ttl
my $ipv6_ddns_ttl = $memberdhcp->ipv6_ddns_ttl();
#Modify ipv6_ddns_ttl
$memberdhcp->ipv6_ddns_ttl(12345);
```

ipv6_dns_update_style( )

Use this method to set or retrieve the update style for dynamic DNS updates (DHCPv6).

Setting this method to a defined value implicitly sets the override_ipv6_dns_update_style method to 'true'. Setting the parameter to undefined causes the appliance to use the Grid default and automatically resets the override_ipv6_dns_update_style attribute to 'false'.

Note that when ipv6_dns_update_style is set to a defined value and override_ipv6_dns_update_style is set to 'false', the last operation takes precedence. Thus the sequence $object->ipv6_dns_update_style('INTERIM'); $object->override_ipv6_dns_update_style('false'); will set override_ipv6_dns_update_style to 'false', and the sequence $object->override_ipv6_dns_update_style('false'); $object->ipv6_dns_update_style('INTERIM'); will result in override_ipv6_dns_update_style='true'.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

Valid values are 'INTERIM' and 'STANDARD'.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get ipv6_dns_update_style value
my $value = $object->ipv6_dns_update_style();

#Modify ipv6_dns_update_style value
$object->ipv6_dns_update_style('STANDARD');
```

ipv6_domain_name( )

Use this method to set or retrieve the ipv6_domain_name value of a Grid Member DHCP object.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Setting this method to a defined value implicitly sets the override_ipv6_domain_name method to "true". Setting the parameter to

undefined causes the appliance to use the upper level default and automatically resets the override_ipv6_domain_name attribute to "false".

Note that when ipv6_domain_name is set to a defined value and override_ipv6_domain_name is set to "false", the last operation takes precedence. Thus the sequence $object->ipv6_domain_name("testdomain"); $object->override_ipv6_domain_name("false"); will set override_ipv6_domain_name to "false", and the sequence $object->override_ipv6_domain_name("false"); $object->ipv6_domain_name("testdomain"); will result in override_ipv6_domain_name="true".

Parameter

The name of the client's domain for resolving names in DNS in string format.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get ipv6_domain_name
my $ipv6_domain_name = $memberdhcp->ipv6_domain_name();
#Modify ipv6_domain_name
$memberdhcp->ipv6_domain_name("test_domain.com");
```

ipv6_domain_name_servers( )

Use this method to set or retrieve the ipv6_domain_name_servers value of a Grid Member DHCP object.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Setting this method to a defined value implicitly sets the override_ipv6_domain_name_servers method to "true". Setting the parameter to undefined causes the appliance to use the upper-level default and automatically resets the override_ipv6_domain_name_servers attribute to "false".

Note that when ipv6_domain_name_servers is set to a defined value and override_ipv6_domain_name_servers is set to "false", the last operation takes precedence. Thus the sequence $object->ipv6_domain_name_servers("2001::1,2001::2"); $object->override_ipv6_domain_name_servers("false"); will set override_ipv6_domain_name_servers to "false", and the sequence $object->override_ipv6_domain_name_servers("false"); $object->ipv6_domain_name_servers("2001::1,2001::2"); will result in override_ipv6_domain_name_servers="true".

Parameter

The domain name server addresses in IPv6 address format.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get ipv6_domain_name_servers
my $ipv6_domain_name_servers = $memberdhcp->ipv6_domain_name_servers();
#Modify ipv6_domain_name_servers
$memberdhcp->ipv6_domain_name_servers("2001::1,2001::2");
```

ipv6_enable_ddns( )

Use this method to set or retrieve the ipv6_enable_ddns flag. Set this flag to enable DHCPv6 dynamic updates.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Setting this method to a defined value implicitly sets the override_ipv6_enable_ddns method to "true". Setting the parameter to undefined causes the appliance to use the Grid default and automatically resets the override_ipv6_enable_ddns attribute to "false".

Note that when ipv6_enable_ddns is set to a defined value and override_dipv6_enable_ddns is set to "false", the last operation takes precedence. Thus the sequence $object->ipv6_enable_ddns("true"); $object->override_ipv6_enable_ddns("false"); will set override_ipv6_enable_ddns to "false", and the sequence $object->override_ipv6_enable_ddns("false"); $object->ipv6_enable_ddns("true"); will result in override_ipv6_enable_ddns="true".

Parameter

Specify "true" to enable this feature or "false" to disable it. The default value is "false".

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get ipv6_enable_ddns
my $ipv6_enable_ddns = $memberdhcp->ipv6_enable_ddns();
#Modify ipv6_enable_ddns
$memberdhcp->ipv6_enable_ddns("true");
```

ipv6_enable_gss_tsig( )

Use this method to set or retrieve the member-level ipv6_enable_gss_tsig flag.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

This is for DHCPv6 GSS-TSIG authentication.

Setting this method to a defined value implicitly sets the override_ipv6_enable_gss_tsig method to "true". Setting the parameter to undefined causes the appliance to use the upper-level default and automatically resets the override_ipv6_enable_gss_tsig attribute to "false".

Note that when ipv6_enable_gss_tsig is set to a defined value and override_ipv6_enable_gss_tsig is set to "false", the last operation takes precedence. Thus the sequence $object->ipv6_enable_gss_tsig("true"); $object->override_ipv6_enable_gss_tsig("false"); will set override_ipv6_enable_gss_tsig to "false", and the sequence $object->override_ipv6_enable_gss_tsig("false"); $object->ipv6_enable_gss_tsig("true"); will result in override_ipv6_enable_gss_tsig="true".

Parameter

Specify "true" to enable GSS-TSIG authentication for DHCP IPv6 or "false" to disable it. The default value is undefined. Setting this flag overrides the ipv6_enable_gss_tsig attribute of the grid.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get ipv6_enable_gss_tsig
my $ipv6_enable_gss_tsig = $memberdhcp->ipv6_enable_gss_tsig();
#Modify ipv6_enable_gss_tsig
$memberdhcp->ipv6_enable_gss_tsig("true");
```

ipv6_enable_lease_scavenge( )

Use this method to set or retrieve the flag that indicates whether the DHCPv6 lease scavenging is enabled or disabled.

Setting this method to a defined value implicitly sets the override_ipv6_lease_scavenge method to "true". When you set the parameter to undefined and if ipv6_lease_scavenge_time or ipv6_remember_expired_client_association are defined, the appliance uses the default Grid value and automatically resets override_ipv6_lease_scavenge attribute to "false".

Note that when ipv6_enable_lease_scavenge is set to a defined value and override_ipv6_enable_lease_scavenge is set to "false", the last operation takes precedence. Thus, the sequence $object->ipv6_enable_lease_scavenge("true"); $object->override_ipv6_lease_scavenge("false"); sets override_ipv6_lease_scavenge to "false", and the sequence $object->override_ipv6_lease_scavenge("false"); $object->ipv6_enable_lease_scavenge("true"); results in override_ipv6_lease_scavenge="true".

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

Specify 'true' to enable DHCPv6 lease scavenging and 'false' to disable it. The default value is false.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get ipv6_enable_lease_scavenge
my $ipv6_enable_lease_scavenge = $memberdhcp->ipv6_enable_lease_scavenge();
#Modify ipv6_enable_lease_scavenge
$memberdhcp->ipv6_enable_lease_scavenge("true");
```

ipv6_remember_expired_client_association( )

Use this method to set or retrieve the flag that indicates whether the DHCPv6 expired lease binding is enabled or disabled.

Setting this method to a defined value implicitly sets the override_ipv6_lease_scavenge method to "true". When you set the parameter to undefined and if ipv6_enable_lease_scavenge or ipv6_lease_scavenge_time are defined, the appliance uses the default Grid value and automatically resets override_ipv6_lease_scavenge to "false".

Note that when ipv6_remember_expired_client_association is set to a defined value and override_ipv6_lease_scavenge is set to "false", the last operation takes precedence. Thus, the sequence $object->ipv6_remember_expired_client_association("true"); $object->override_ipv6_lease_scavenge("false"); sets override_ipv6_lease_scavenge to "false", and the sequence $object->override_ipv6_lease_scavenge("false"); $object->ipv6_remember_expired_client_association("true"); results in override_ipv6_lease_scavenge="true".

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

Specify 'true' to enable the DHCPv6 server to 'remember' the client and 'false' to disable it. The default value is false.

Returns

If you specified a parameter, the method returns true when the modification succeds, and return false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get ipv6_remember_expired_client_association
my $ipv6_remember_expired_client_association = $memberdhcp-
>ipv6_remember_expired_client_association();
#Modify ipv6_remember_expired_client_association
$memberdhcp->ipv6_remember_expired_client_association("true");
```

ipv6_enable_retry_updates( )

Use this method to set or retrieve the ipv6_enable_retry_updates flag. If this flag is set to "true", the DHCPv6 server retries failed dynamic DNS updates.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Setting this method to a defined value implicitly sets the override_ipv6_enable_retry_updates method to "true". Setting the parameter to undefined causes the appliance to use the Grid default and automatically resets the override_ipv6_enable_retry_updates attribute to "false".

Note that when ipv6_enable_retry_updates is set to a defined value and override_dipv6_enable_retry_updates is set to "false", the last operation takes precedence. Thus the sequence $object->ipv6_enable_retry_updates("true"); $object->override_ipv6_enable_retry_updates("false"); will set override_ipv6_enable_retry_updates to "false", and the sequence $object->override_ipv6_enable_retry_updates("false"); $object->ipv6_enable_retry_updates("true"); will result in override_ipv6_enable_retry_updates="true".

Parameter

Specify "true" to enable this feature or "false" to disable it. The default value is "true".

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get ipv6_enable_retry_updates
my $ipv6_enable_retry_updates = $memberdhcp->ipv6_enable_retry_updates();
#Modify ipv6_enable_retry_updates
$memberdhcp->ipv6_enable_retry_updates("true");
```

ipv6_generate_hostname( )

Use this method to set or retrieve the ipv6_generate_hostname flag. Set this flag to generate the hostname if it is not sent by the client.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Setting this method to a defined value implicitly sets the override_ipv6_generate_hostname method to "true". Setting the parameter to undefined causes the appliance to use the Grid default and automatically resets the override_ipv6_generate_hostname attribute to "false".

Note that when ipv6_generate_hostname is set to a defined value and override_dipv6_generate_hostname is set to "false", the last

operation takes precedence. Thus the sequence $object->ipv6_generate_hostname("true"); $object->override_ipv6_generate_hostname("false"); will set override_ipv6_generate_hostname to "false", and the sequence $object->override_ipv6_generate_hostname("false"); $object->ipv6_generate_hostname("true"); will result in override_ipv6_generate_hostname="true".

Parameter

Specify "true" to enable this feature or "false" to disable it. The default value is "false".

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get ipv6_generate_hostname
my $ipv6_generate_hostname = $memberdhcp->ipv6_generate_hostname();
#Modify ipv6_generate_hostname
$memberdhcp->ipv6_generate_hostname("true");
```

ipv6_gss_tsig_keys( )

Use this method to set or retrieve the list of GSS-TSIG keys for a member DHCPv6 object.

Setting this method to a defined value implicitly sets the override_ipv6_gss_tsig_keys method to "true". Setting the parameter to undefined causes the appliance to use the Grid default and automatically resets the override_ipv6_gss_tsig_keys method to "false".

Note that when ipv6_gss_tsig_keys is set to a defined value and override_ipv6_gss_tsig_keys is set to "false", the last operation takes precedence. Thus the sequence $object->ipv6_gss_tsig_keys($keys); $object->override_ipv6_gss_tsig_keys("false"); will set override_ipv6_gss_tsig_keys to "false", and the sequence $object->override_ipv6_gss_tsig_keys("false"); $object->ipv6_gss_tsig_keys($keys); will result in override_ipv6_gss_tsig_keys="true".

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

The valid value is an array reference that contains Infoblox::Grid::KerberosKey objects.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get the Kerberos keys list
my $kerberos_keys_list = $member_dhcp->ipv6_gss_tsig_keys();

#Modify the Kerberos keys list
my $kerberos_key1 = Infoblox::Cluster::KerberosKey->new(
       principal => 'host/appliance.ivan.test1@LADYBUG.LOCAL',
       );
my $kerberos_key2 = Infoblox::Cluster::KerberosKey->new(
       principal => 'host/appliance.ivan.test2@LADYBUG.LOCAL',
       );
$member_dhcp->ipv6_gss_tsig_keys([$kerberos_key1, $kerberos_key2]);
```

ipv6_kdc_server( )

Use this method to specify or retrieve the IPv4 address or FQDN of the Kerberos server for DHCPv6 GSS-TSIG authentication. This setting overrides the grid-level setting.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

The Kerberos server IPv4 address or hostname, with a maximum of 1023 bytes.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get ipv6_kdc_server
my $ipv6_kdc_server = $memberdhcp->ipv6_kdc_server();
#Modify ipv6_kdc_server
$memberdhcp->ipv6_kdc_server("10.32.0.8");
```

ipv6_lease_scavenge_time( )

Use this method to set or retrieve the Grid-level grace period in seconds to keep an expired lease before it gets deleted by the scavenging process.

Setting this method to a defined value implicitly sets the override_ipv6_lease_scavenge method to "true". When you set the parameter to undefined and if ipv6_enable_lease_scavenge or ipv6_remember_expired_client_association are defined, the appliance uses the default Grid value and automatically resets override_ipv6_lease_scavenge attribute to "false".

Note that when ipv6_lease_scavenge_time is set to a defined value and override_ipv6_enable_lease_scavenge is set to "false", the last operation takes precedence. Thus, the sequence $object->ipv6_lease_scavenge_time(32000); $object->override_ipv6_lease_scavenge("false"); sets override_ipv6_lease_scavenge to "false", and the sequence $object->override_ipv6_lease_scavenge("false"); $object->ipv6_lease_scavenge_time(32000); results in override_ipv6_lease_scavenge="true".

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

The valid value is an unsigned integer greater than or equal to 21600.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get ipv6_lease_scavenge_time
my $ipv6_lease_scavenge_time = $griddhcp->ipv6_lease_scavenge_time();
#Modify ipv6_lease_scavenge_time
```

ipv6_microsoft_code_page( )

Use this method to set or retrieve the Microsoft client DHCP IPv6 code page value of a grid member. This value is the hostname translation code page for Microsoft DHCP IPv6 clients and overrides the grid-level Microsoft DHCP IPv6 client code page.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Setting this method to a defined value implicitly sets the override_ipv6_microsoft_code_page method to "true". Setting the parameter to undefined causes the appliance to use the Grid default and automatically resets the override_ipv6_microsoft_code_page attribute to "false".

Note that when ipv6_microsoft_code_page is set to a defined value and override_dipv6_microsoft_code_page is set to "false", the last operation takes precedence. Thus the sequence $object->ipv6_microsoft_code_page("Cyrillic (ISO-5589-5)"); $object->override_ipv6_microsoft_code_page("false"); will set override_ipv6_microsoft_code_page to "false", and the sequence $object->override_ipv6_microsoft_code_page("false"); $object->ipv6_microsoft_code_page("Cyrillic (ISO-5589-5)"); will result in override_ipv6_microsoft_code_page="true".

Parameter

Microsoft DHCP client code page in string format. Valid values are:

None

Arabic (1256)

Arabic (ISO-8859-6)

Baltic (1257)

Baltic (775)

Baltic (ISO-8859-4)

Central Europe (1250)

Cyrillic (1251)

Cyrillic (855)

Cyrillic (ISO-8859-5)

Greek (1253)

Greek (737)

Greek (ISO-8859-7)

Hebrew (1255)

Hebrew (862)

Hebrew (ISO-8859-8)

Japanese Shift-JIS (932)

Korean (949)

Latin 1 (ISO-8859-1)

Latin 2 (ISO-8859-2)

Latin 3 (ISO-8859-3)

Latin 9 (ISO-8859-15)

Latin I (1252)

Latin II (852)

Multilingual Latin I (850)

Russian (866)

Simplified Chinese GBK (936)

Thai (874)

Traditional Chinese Big5 (950)

Turkish (1254)

Turkish (857)

Turkish (ISO-8859-9)

US (437)

Vietnam (1258)

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get ipv6_microsoft_code_page
my $ipv6_microsoft_code_page = $memberdhcp->ipv6_microsoft_code_page();
#Modify ipv6_microsoft_code_page
$memberdhcp->ipv6_microsoft_code_page("Cyrillic (ISO-8859-5)");
#Do not override ipv6_microsoft_code_page
$memberdhcp->ipv6_microsoft_code_page("None");
```

ipv6_options( )

Use this method to set or retrieve the ipv6 options.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Setting this method to a defined value implicitly sets the override_ipv6_options method to "true". Setting the parameter to undefined causes the appliance to use the Grid default and automatically resets the override_ipv6_options attribute to "false".

Note that when ipv6_options is set to a defined value and override_dipv6_options is set to "false", the last operation takes precedence. Thus the sequence $object->ipv6_options([$opt]); $object->override_ipv6_options("false"); will set override_ipv6_options to "false", and the sequence $object->override_ipv6_options("false"); $object->ipv6_options([$opt]); will result in override_ipv6_options="true".

Parameter

The valid value is an array reference that contains Infoblox::DHCP::Option object(s).

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get ipv6_options
my $ipv6_options = $memberdhcp->ipv6_options();
#Modify ipv6_options
my $option1 = Infoblox::DHCP::Option->new(
    name   => "dhcp6.rapid-commit",
    value  => "true"
);
$memberdhcp->ipv6_options([$option1);
```

ipv6_recycle_leases( )

Use this method to set or retrieve the ipv6_recycle_leases flag of the DHCP member.

If the flag is enabled, the leases are kept in recycle bin until one week after expiration. When the flag is disabled, the leases are irrecoverably deleted.

Setting this method to a defined value implicitly sets the override_ipv6_recycle_leases method to "true". Setting the parameter to undefined causes the appliance to use the Grid default and automatically resets the override_ipv6_recycle_leases attribute to "false".

Note that when ipv6_recycle_leases is set to a defined value and override_ipv6_recycle_leases is set to "false", the last operation takes precedence. Thus the sequence $object->ipv6_recycle_leases("true"); $object->override_ipv6_recycle_leases("false"); will set override_ipv6_recycle_leases to "false", and the sequence $objct->override_ipv6_recycle_leases("false"); $object->ipv6_recycle_leases("true"); will result in override_ipv6_recycle_leases="true".

Parameter

Specify "true" to set the ipv6_recycle_leases flag or "false" to deactivate/unset it. The default value is undefined.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get ipv6_recycle_leases
my $ipv6_recycle_leases = $memberdhcp->ipv6_recycle_leases();
#Modify ipv6_recycle_leases
$memberdhcp->ipv6_recycle_leases("false");
```

ipv6_retry_updates_interval( )

Use this method to set or retrieve the ipv6_retry_updates_interval value. Set the retry interval when the member DHCPv6 server makes repeated attempts to send DDNS updates to a DNS server.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

A 32-bit integer (range from 0 to 4294967295) that represents the retry interval in minutes. The default retry interval is five minutes.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get ipv6_retry_updates_interval
my $ipv6_retry_updates_interval = $memberdhcp->ipv6_retry_updates_interval();
#Modify ipv6_retry_updates_interval
$memberdhcp->ipv6_retry_updates_interval(10);
```

ipv6_server_duid( )

Use this method to set or retrieve the server DHCPv6 unique identifier (DUID) for the grid member.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

DUID of the grid member in string format.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get ipv6_server_duid
my $ipv6_server_duid = $memberdhcp->ipv6_server_duid();
#Modify ipv6_server_duid
$memberdhcp->ipv6_server_duid("false");
```

ipv6_update_dns_on_lease_renewal( )

Use this method to set or retrieve the ipv6_update_dns_on_lease_renewal flag. This attribute controls whether the DHCP server updates DNS when an IPv6 DHCP lease is renewed.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Setting this method to a defined value implicitly sets the override_ipv6_update_dns_on_lease_renewal method to "true". Setting the parameter to undefined causes the appliance to use the Grid default and automatically resets the override_ipv6_update_dns_on_lease_renewal attribute to "false".

Note that when ipv6_update_dns_on_lease_renewal is set to a defined value and override_dipv6_update_dns_on_lease_renewal is set to "false", the last operation takes precedence. Thus the sequence $object->ipv6_update_dns_on_lease_renewal("true"); $object->override_ipv6_update_dns_on_lease_renewal("false"); will set override_ipv6_update_dns_on_lease_renewal to "false", and the sequence $object->override_ipv6_update_dns_on_lease_renewal("false"); $object->ipv6_update_dns_on_lease_renewal("true"); will result in override_ipv6_update_dns_on_lease_renewal="true".

Parameter

Specify "true" to enable this feature or "false" to disable it. The default value is "false".

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get ipv6_update_dns_on_lease_renewal
my $ipv6_update_dns_on_lease_renewal = $memberdhcp->ipv6_update_dns_on_lease_renewal();
#Modify ipv6_update_dns_on_lease_renewal
$memberdhcp->ipv6_update_dns_on_lease_renewal("true");
```

kdc_server( )

Use this method to specify or retrieve the IPv4 address or FQDN of the Kerberos server for DHCPv4 GSS-TSIG authentication. This setting overrides the grid-level setting.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

The Kerberos server IPv4 address or FQDN, with a maximum of 1023 bytes.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get kdc_server
my $kdc_server = $memberdhcp->kdc_server();
#Modify  kdc_server
$memberdhcp->kdc_server("10.32.0.7");
```

ipv6_lease_scavenge_time( )

Use this method to set or retrieve the member-level grace period in seconds to keep an expired lease before it gets deleted by the scavenging process.

Setting this method to a defined value implicitly sets the override_ipv6_lease_scavenge method to "true". Setting the parameter to undefined unless ipv6_enable_lease_scavenge is defined causes the appliance to use the Grid default and automatically resets the override_ipv6_lease_scavenge attribute to "false".

Note that when ipv6_lease_scavenge_time is set to a defined value and override_dipv6_lease_scavenge_time is set to "false", the last operation takes precedence. Thus the sequence $object->ipv6_lease_scavenge_time("true"); $object->override_ipv6_lease_scavenge("false"); will set override_ipv6_lease_scavenge to "false", and the sequence $object->override_ipv6_lease_scavenge("false"); $object->ipv6_lease_scavenge_time("true"); will result in override_ipv6_lease_scavenge="true".

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

The valid value is an unsigned integer greater than or equal to 21600.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get ipv6_lease_scavenge_time
my $ipv6_lease_scavenge_time = $memberdhcp->ipv6_lease_scavenge_time();
#Modify ipv6_lease_scavenge_time
$memberdhcp->ipv6_lease_scavenge_time("22000");
```

lease_per_client_settings( )

Use this method to set or retrieve the Grid member setting that defines how DHCP server will free or release client leases.

Setting this method to a defined value implicitly sets the override_lease_per_client_settings method to "true". Setting the parameter to undefined causes the appliance to use the Grid default and automatically resets the override_lease_per_client_settings attribute to "false".

Note that when lease_per_client_settings is set to a defined value and override_lease_per_client_settings is set to "false", the last operation takes precedence. Thus the sequence $object->lease_per_client_settings("NEVER_RELEASE"); $object->override_lease_per_client_settings("false"); will set override_lease_per_client_settings to "false", and the sequence $object->override_lease_per_client_settings("false"); $object->lease_per_client_settings("NEVER_RELEASE"); will set override_lease_per_client_settings to "true".

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

The valid values are 'ONE_LEASE_PER_CLIENT', 'RELEASE_MATCHING_ID' and 'NEVER_RELEASE'.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get lease_per_client_settings value
my $lease_per_client_settings = $object->lease_per_client_settings();

#Modify lease_per_client_settings value
$object->lease_per_client_settings("NEVER_RELEASE");
```

lease_scavenge_time( )

Use this method to set or retrieve the member-level lease_scavenge_time value.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Setting this method to a defined value implicitly sets the override_lease_scavenge_time method to "true". Setting the parameter to undefined causes the appliance to use the Grid default and automatically resets the override_lease_scavenge_time attribute to "false".

Note that when lease_scavenge_time is set to a defined value and override_lease_scavenge_time is set to "false", the last operation takes precedence. Thus the sequence $object->lease_scavenge_time(604800); $object->override_lease_scavenge_time("false"); will set override_lease_scavenge_time to "false", and the sequence $object->override_lease_scavenge_time("false"); $object->lease_scavenge_time(604800); will result in override_lease_scavenge_time="true".

Parameter

An integer that specifies the period of time, in seconds, that free and backup leases remain in the database before they are automatically deleted. To disable lease scavenging, set the parameter to -1. The minimum positive value must be greater than 86400 seconds (1 day). The default value of this parameter is -1, which means that lease scavenging is disabled.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get lease_scavenge_time
my $lease_scavenge = $memberdhcp->lease_scavenge_time();
#Modify lease_scavenge_time
$memberdhcp->lease_scavenge_time(30 * 24 * 60 * 60);
```

log_lease_events( )

Use this method to set or retrieve the log_lease_events value of a member DHCP server. This value specifies whether the grid member logs lease events.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Setting this method to a defined value implicitly sets the override_log_lease_events method to "true". Setting the parameter to undefined causes the appliance to use the Grid default and automatically resets the override_log_lease_events attribute to "false".

Note that when log_lease_events is set to a defined value and override_dlog_lease_events is set to "false", the last operation takes precedence. Thus the sequence $object->log_lease_events("true"); $object->override_log_lease_events("false"); will set override_log_lease_events to "false", and the sequence $object->override_log_lease_events("false"); $object->log_lease_events("true"); will result in override_log_lease_events="true".

Parameter

Specify "true" to set log_lease_events flag or "false" to deactivate/unset it.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get log_lease_events
my $log_lease_events = $memberdhcp->log_lease_events();
#Modify log_lease_events
$memberdhcp->log_lease_events("true");
```

logic_filters( )

Use this method to set or retrieve all the logic (ordered) filters associated with the Member DHCP.

Filters are used to screen address assignments on a Member DHCP if the addresses are to be assigned to specific hosts.

Setting this method to a defined value implicitly sets the override_logic_filters method to "true". Setting the parameter to undefined causes the appliance to use the Grid default and automatically resets the override_logic_filters attribute to "false".

Note that when logic_filters is set to a defined value and override_dlogic_filters is set to "false", the last operation takes precedence. Thus the sequence $object->logic_filters($filters); $object->override_logic_filters("false"); will set override_logic_filters to "false", and the sequence $object->override_logic_filters("false"); $object->logic_filters($filters); will result in override_logic_filters="true".

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

The valid value is an array reference that contains either the Infoblox::DHCP::Filter::MAC, Infoblox::DHCP::Filter::NAC, Infoblox::DHCP::Filter::Option objects or the names of the corresponding filters. The filters in the list must satisfy the following conditions:

Filter name must be globally unique.
Filters without expressions must be at the end of the list.
The filters should not be disabled.

The default value of the parameter is undefined.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get logic_filters
my $ref_logic_filters = $memberdhcp->logic_filters();
#Modify logic_filters
$memberdhcp->logic_filters(["filter_1","filter_2",$exl1]); #$exl1 is an Infoblox::DHCP::Filter::MAC,
Infoblox::DHCP::Filter::NAC or Infoblox::DHCP::Filter::Option object.
```

low_water_mark( )

Use this method to set or retrieve the low_water_mark value of a member DHCP server. If the percent of allocated addresses drops below this watermark, the appliance makes a syslog entry and if enabled, sends an e-mail notification.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

Number with the percentage value. Range is from 0 to 100. The low watermark value must be lower than the high watermark value. The default low watermark number is 0.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get low_water_mark
my $low_water_mark = $memberdhcp->low_water_mark();
#Modify low_water_mark
$memberdhcp->low_water_mark(10);
```

low_water_mark_reset( )

Use this method to set or retrieve the low_water_mark_reset value of a member DHCP server. If the percentage of allocated addresses exceeds this value, a corresponding SNMP trap is reset.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

Number with the percentage value. Range is from 0 to 100. The low watermark reset value must be higher than the low watermark value. The default low watermark reset value is 0.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get low_water_mark_reset
my $low_water_mark_reset = $memberdhcp->low_water_mark_reset();
#Modify low_water_mark_reset
$memberdhcp->low_water_mark_reset(10);
```

override_dns_update_style( )

The override_dns_update_style controls whether the dns_update_style in the DHCP Member is used, instead of the Grid default.

The override_dns_update_style can be specified explicitly. It is also set implicitly when dns_update_style is set to a defined value.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

Set the parameter to 'true' to override the Grid-level setting for dns_update_style. Set the parameter to 'false' to inherit the Grid-level setting for dns_update_style.

The default value of this parameter is 'false'.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get override_dns_update_style value
my $value = $object->override_dns_update_style();

#Modify override_dns_update_style value
$object->override_dns_update_style('true');
```

override_enable_ifmap_publishing( )

The override_enable_ifmap_publishing controls whether the enable_ifmap_publishing in the DHCP Member is used, instead of the Grid default.

The override_enable_ifmap_publishing can be specified explicitly. It is also set implicitly when enable_ifmap_publishing is set to a defined value.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

Set the parameter to "true" to override the grid-level setting for enable_ifmap_publishing. Set the parameter to "false" to inherit the grid-level setting for enable_ifmap_publishing.

The default value of this parameter is "false".

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Getting override_enable_ifmap_publishing
my $override_enable_ifmap_publishing=$memberdhcp->override_enable_ifmap_publishing( );
#Modifying override_enable_ifmap_publishing
$memberdhcp->override_enable_ifmap_publishing("true");
```

override_gss_tsig_keys( )

The override_gss_tsig_keys attribute controls whether the gss_tsig_keys method values of the member are used, instead of the Grid default.

The override_gss_tsig_keys attribute can be specified explicitly. It is also set implicitly when gss_tsig_keys is set to a defined value.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

Set the parameter to "true" to override GSS-TSIG keys. Set the parameter to "false" to inherit the grid-level GSS-TSIG keys.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Getting override_gss_tsig_keys
my $override_gss_tsig_keys = $member_dhcp->override_gss_tsig_keys();
#Modyfying override_gss_tsig_keys
$member_dhcp->override_gss_tsig_keys("true");
```

override_ipv6_dns_update_style( )

The override_ipv6_dns_update_style controls whether the ipv6_dns_update_style is used instead of the Grid default in the DHCP member.

The override_ipv6_dns_update_style can be specified explicitly. It is also set implicitly when ipv6_dns_update_style is set to a defined value.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

Set the parameter to 'true' to override the Grid-level setting for ipv6_dns_update_style. Set the parameter to 'false' to inherit the Grid-level setting for ipv6_dns_update_style.

The default value of this parameter is 'false'.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get override_ipv6_dns_update_style value
my $value = $object->override_ipv6_dns_update_style();

#Modify override_ipv6_dns_update_style value
$object->override_ipv6_dns_update_style('true');
```

override_ipv6_gss_tsig_keys( )

The override_ipv6_gss_tsig_keys attribute controls whether the ipv6_gss_tsig_keys method values of the member are used, instead of the Grid default.

The override_ipv6_gss_tsig_keys attribute can be specified explicitly. It is also set implicitly when ipv6_gss_tsig_keys is set to a defined value.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

Set the parameter to "true" to override GSS-TSIG keys. Set the parameter to "false" to inherit the grid-level GSS-TSIG keys.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Getting override_ipv6_gss_tsig_keys
my $override_ipv6_gss_tsig_keys = $member_dhcp->override_ipv6_gss_tsig_keys();
#Modyfying override_ipv6_gss_tsig_keys
$member_dhcp->override_ipv6_gss_tsig_keys("true");
```

override_ipv6_lease_scavenge( )

The override_ipv6_lease_scavenge attribute controls whether the ipv6_enable_lease_scavenge, ipv6_lease_scavenge_time and ipv6_remember_expired_client_association methods values of the member are used, instead of the Grid default.

The override_ipv6_lease_scavenge attribute can be specified explicitly. It is also set implicitly when one of ipv6_enable_lease_scavenge, ipv6_lease_scavenge_time and ipv6_remember_expired_client_association is set to a defined value.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

Set the parameter to "true" to override settings for scavenging of DHCPv6 leases. Set the parameter to "false" to inherit settings for scavenging of DHCPv6 leases.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Getting override_ipv6_lease_scavenge
my $override_ipv6_lease_scavenge = $memberdhcp->override_ipv6_lease_scavenge();
#Modyfying override_ipv6_lease_scavenge
$memberdhcp->override_ipv6_lease_scavenge("true");
```

override_logic_filters( )

The override_logic_filters attribute controls whether the logic_filters value of the Member DHCP is used, instead of the Grid default.

The override_logic_filters attribute can be specified explicitly. It is also set implicitly when logic_filters is set to a defined value.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

Set the parameter to "true" to override the Grid-level setting for logic_filters. Set the parameter to "false" to inherit the Grid-level setting for logic_filters.

The default value of this parameter is "false".

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Getting override_logic_filters
my $override_logic_filters=$memberdhcp->override_logic_filters( );
#Modifying override_logic_filters
$memberdhcp->override_logic_filters("true");
```

name( )

Use this method to retrieve the grid name.

Parameter

None.

Returns

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get name
my $name = $memberdhcp->name();
```

nextserver( )

Use this method to set or retrieve the next server value of a member DHCP server. This value is the IP address or name of the boot file server on which the boot file is stored.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

This option is overridden independently from Infoblox::Grid::Member::DHCP->bootfile() and Infoblox::Grid::Member::DHCP->bootserver().

Parameter

The boot file server IP address or name in FQDN format. The FQDN consists of the host name followed by the domain name (example: abc.com). A boot file server name can have a maximum of 256 bytes.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get nextserver
my $nextserver = $memberdhcp->nextserver();
#Modify nextserver
$memberdhcp->nextserver("1.1.1.1");
#Un-override nextserver
$memberdhcp->nextserver(undef);
```

microsoft_code_page( )

Use this method to set or retrieve the Microsoft client DHCP IPv4 code page value of a grid member. This value is the hostname translation code page for Microsoft DHCP IPv4 clients and overrides the grid-level Microsoft DHCP IPv4 client code page.

Setting microsoft_code_page() to undef removes the override, enabling the appliance to use the grid-level microsoft_code_page setting.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

Microsoft DHCP client code page in string format. Valid values are the same as those for ipv6_microsoft_code_page. See ipv6_microsoft_code_page().

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get microsoft_code_page
my $microsoft_code_page = $memberdhcp->microsoft_code_page();
#Modify microsoft_code_page
$memberdhcp->microsoft_code_page("Cyrillic (ISO-8859-5)");
#Do not override microsoft_code_page
$memberdhcp->microsoft_code_page("None");
```

options( )

Use this method to set or retrieve the options value of a member DHCP server. DHCP options provide network configuration settings and various services available on the network for DHCP member. This setting overrides the grid level setting.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

The valid value is an array reference that contains Infoblox::DHCP::Option object(s).

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get options
my $options = $memberdhcp->options();
#Modify options
my $option1 = Infoblox::DHCP::Option->new(
    name  => "lease-time",
    value => 50
);

my $option2 = Infoblox::DHCP::Option->new(
    name  => "routers",
    value => "1.4.1.1, 2.2.2.2"
);

my $option3 = Infoblox::DHCP::Option->new(
    name  => "nameservers",
    value => "1.1.1.1, 2.2.2.2"
);

my $option4 = Infoblox::DHCP::Option->new(
    name  => "domain-name",
    value => "domainname.com",
);

my $option5 = Infoblox::DHCP::Option->new(
    name  => "broadcast",
    value => "255.0.0.0",
);

$memberdhcp->options([$option1, $option2, $option3, $option4, $option5]);
```

option60_match_rules( )

Use this method to set or retrieve the option60_match_rules value of a member DHCP server. An Option60MatchRule object allows specifying match rules for Option 60 globally. This setting overrides the grid level setting.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

The valid value is an array reference that contains Infoblox::DHCP::Option60MatchRule object(s).

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get option60_match_rules
my $options = $memberdhcp->option60_match_rules();
#Modify option60_match_rules
my $option60_match_rule = Infoblox::DHCP::Option60MatchRule->new(
                                  "option_space" => "infospace",
                                  "match_value" => "Nortel-i2004-A",
                                  "is_substring" => "true",
                                  "substring_offset" => "8",
                                  "substring_length" => "14"
                                );

$memberdhcp->option60_match_rules([$option60_match_rule]);
#Do not override option60_match_rules
$memberdhcp->option60_match_rules(undef);
```

override_ddns_ttl( )

The override_ddns_ttl attribute controls whether the ddns_ttl value in the member DHCP server is used, instead of the Grid default.

The override_ddns_ttl attribute can be specified explicitly. It is also set implicitly when ddns_ttl is set to a defined value.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

Set the parameter to "true" to override the grid-level setting for ddns_ttl. Set the parameter to "false" to inherit the grid-level setting for ddns_ttl.

The default value of this parameter is "false".

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Getting override_ddns_ttl
my $override_ddns_ttl=$memberdhcp->override_ddns_ttl( );
#Modifying override_ddns_ttl
$memberdhcp->override_ddns_ttl("true");
```

override_enable_dhcpv6_service( )

The override_enable_dhcpv6_service attribute controls whether the enable_dhcpv6_service value in the DHCP member is used, instead of the Grid default.

The override_enable_dhcpv6_service attribute can be specified explicitly. It is also set implicitly when enable_dhcpv6_service is set to a defined value.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

Set the parameter to "true" to override the grid-level setting for enable_dhcpv6_service. Set the parameter to "false" to inherit the grid-level setting for enable_dhcpv6_service.

The default value of this parameter is "false".

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Getting override_enable_dhcpv6_service
my $override_enable_dhcpv6_service=$memberdhcp->override_enable_dhcpv6_service( );
#Modifying override_enable_dhcpv6_service
$memberdhcp->override_enable_dhcpv6_service("true");
```

override_enable_fingerprint( )

Use this method to set or retrieve the override_enable_fingerprint flag.

Parameter

Boolean to enable or disable the override fibgerprint flag for the member. The default value is "false".

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get override_enable_fingerprint
my $status = $memberdhcp->override_enable_fingerprint();
#Modify override_enable_fingerprint
$memberdhcp->override_enable_fingerprint("true");
```

override_hostname_rewrite( )

Use this method to set or retrieve the override_hostname_rewrite flag.

Parameter

Boolean to enable or disable the override hostname rewrite policy flag for the member. The default value is "false".

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get override_hostname_rewrite
my $status = $memberdhcp->override_hostname_rewrite();
#Modify override_hostname_rewrite
$memberdhcp->override_hostname_rewrite("true");
```

override_ignore_id( )

The override_ignore_id controls whether the ignore_id in the DHCP member is used, instead of the grid-level default.

The override_ignore_id can be specified explicitly. It is also set implicitly when ignore_id is set to a defined value.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

Set the parameter to "true" to override the grid-level setting for ignore_id and ignore_mac_addresses. Set the parameter to "false" to inherit the grid-level setting for ignore_id and ignore_mac_addresses.

The default value of this parameter is "false".

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get attrubute value
my $value=$memberdhcp->override_ignore_id();
#Modifying attribute value
$memberdhcp->override_ignore_id('true');
```

override_ipv6_ddns_domainname( )

The override_ipv6_ddns_domainname attribute controls whether the ipv6_ddns_domainname value in the DHCP member is used, instead

of the Grid default.

The override_ipv6_ddns_domainname attribute can be specified explicitly. It is also set implicitly when ipv6_ddns_domainname is set to a defined value.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

Set the parameter to "true" to override the grid-level setting for ipv6_ddns_domainname. Set the parameter to "false" to inherit the grid-level setting for ipv6_ddns_domainname.

The default value of this parameter is "false".

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Getting override_ipv6_ddns_domainname
my $override_ipv6_ddns_domainname=$memberdhcp->override_ipv6_ddns_domainname( );
#Modifying override_ipv6_ddns_domainname
$memberdhcp->override_ipv6_ddns_domainname("true");
```

override_ipv6_ddns_hostname( )

The override_ipv6_ddns_hostname attribute controls whether the ipv6_ddns_hostname value in the DHCP member is used, instead of the Grid default.

The override_ipv6_ddns_hostname attribute can be specified explicitly. It is also set implicitly when ipv6_ddns_hostname is set to a defined value.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

Set the parameter to "true" to override the grid-level setting for ipv6_ddns_hostname. Set the parameter to "false" to inherit the grid-level setting for ipv6_ddns_hostname.

The default value of this parameter is "false".

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Getting override_ipv6_ddns_hostname
my $override_ipv6_ddns_hostname=$memberdhcp->override_ipv6_ddns_hostname( );
#Modifying override_ipv6_ddns_hostname
$memberdhcp->override_ipv6_ddns_hostname("true");
```

override_ipv6_ddns_enable_option_fqdn( )

Use this method to set or retrieve the override_ipv6_ddns_enable_option_fqdn value of a Grid Member DHCP object.

The override_ipv6_ddns_enable_option_fqdn attribute can be specified explicitly. It is also set implicitly when ipv6_ddns_enable_option_fqdn is set to a defined value.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

Set the parameter to "true" to override the upper-level setting for ipv6_ddns_enable_option_fqdn. Set the parameter to "false" to inherit the upper-level setting for ipv6_ddns_enable_option_fqdn.

The default value of this parameter is "false".

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get override_ipv6_ddns_enable_option_fqdn
my $override_ipv6_ddns_enable_option_fqdn = $memberdhcp->override_ipv6_ddns_enable_option_fqdn();
#Modify override_ipv6_ddns_enable_option_fqdn
$memberdhcp->override_ipv6_ddns_enable_option_fqdn("true");
```

override_ipv6_ddns_ttl( )

The override_ipv6_ddns_ttl attribute controls whether the ipv6_ddns_ttl value in the DHCP member is used, instead of the Grid default.

The override_ipv6_ddns_ttl attribute can be specified explicitly. It is also set implicitly when ipv6_ddns_ttl is set to a defined value.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

Set the parameter to "true" to override the grid-level setting for ipv6_ddns_ttl. Set the parameter to "false" to inherit the grid-level setting for ipv6_ddns_ttl.

The default value of this parameter is "false".

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Getting override_ipv6_ddns_ttl
my $override_ipv6_ddns_ttl=$memberdhcp->override_ipv6_ddns_ttl( );
#Modifying override_ipv6_ddns_ttl
$memberdhcp->override_ipv6_ddns_ttl("true");
```

override_ipv6_domain_name( )

Use this method to set or retrieve the override_ipv6_domain_name value of a Grid Member DHCP object.

The override_ipv6_domain_name attribute can be specified explicitly. It is also set implicitly when ipv6_domain_name is set to a defined value.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

Set the parameter to "true" to override the upper-level setting for ipv6_domain_name. Set the parameter to "false" to inherit the upper-level setting for ipv6_domain_name.

The default value of this parameter is "false".

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get override_ipv6_domain_name
my $override_ipv6_domain_name = $memberdhcp->override_ipv6_domain_name();
#Modify override_ipv6_domain_name
$memberdhcp->override_ipv6_domain_name("true");
```

override_ipv6_domain_name_servers( )

Use this method to set or retrieve the override_ipv6_domain_name_servers value of a Grid Member DHCP object.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

Set the parameter to "true" to override the upper-level setting for ipv6_domain_name_servers. Set the parameter to "false" to inherit the upper-level setting for ipv6_domain_name_servers.

The default value of this parameter is "false".

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get override_ipv6_domain_name_servers
my $override_ipv6_domain_name_servers = $memberdhcp->override_ipv6_domain_name_servers();
#Modify override_ipv6_domain_name_servers
$memberdhcp->override_ipv6_domain_name_servers("true");
```

override_ipv6_enable_ddns( )

The override_ipv6_enable_ddns attribute controls whether the ipv6_enable_ddns value in the DHCP member is used, instead of the Grid default.

The override_ipv6_enable_ddns attribute can be specified explicitly. It is also set implicitly when ipv6_enable_ddns is set to a defined value.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

Set the parameter to "true" to override the grid-level setting for ipv6_enable_ddns. Set the parameter to "false" to inherit the grid-level setting for ipv6_enable_ddns.

The default value of this parameter is "false".

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Getting override_ipv6_enable_ddns
my $override_ipv6_enable_ddns=$memberdhcp->override_ipv6_enable_ddns( );
#Modifying override_ipv6_enable_ddns
$memberdhcp->override_ipv6_enable_ddns("true");
```

override_ipv6_enable_gss_tsig( )

The override_ipv6_enable_gss_tsig attribute controls whether the ipv6_enable_gss_tsig value in the DHCP member is used, instead of the Grid default.

The override_ipv6_enable_gss_tsig attribute can be specified explicitly. It is also set implicitly when ipv6_enable_gss_tsig is set to a defined value.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

Set the parameter to "true" to override the grid-level setting for ipv6_enable_gss_tsig. Set the parameter to "false" to inherit the grid-level setting for ipv6_enable_gss_tsig.

The default value of this parameter is "false".

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Getting override_ipv6_enable_gss_tsig
my $override_ipv6_enable_gss_tsig=$memberdhcp->override_ipv6_enable_gss_tsig( );
#Modifying override_ipv6_enable_gss_tsig
$memberdhcp->override_ipv6_enable_gss_tsig("true");
```

override_ipv6_enable_retry_updates( )

The override_ipv6_enable_retry_updates attribute controls whether the ipv6_enable_retry_updates value of the DHCP member is used, instead of the Grid default.

The override_ipv6_enable_retry_updates attribute can be specified explicitly. It is also set implicitly when ipv6_enable_retry_updates is set to a defined value.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

Set the parameter to "true" to override the grid-level setting for ipv6_enable_retry_updates. Set the parameter to "false" to inherit the grid-level setting for ipv6_enable_retry_updates.

The default value of this parameter is "false".

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Getting override_ipv6_enable_retry_updates
my $override_ipv6_enable_retry_updates=$memberdhcp->override_ipv6_enable_retry_updates( );
#Modifying override_ipv6_enable_retry_updates
$memberdhcp->override_ipv6_enable_retry_updates("true");
```

override_ipv6_generate_hostname( )

The override_ipv6_generate_hostname attribute controls whether the ipv6_generate_hostname value in the DHCP member is used, instead of the Grid default.

The override_ipv6_generate_hostname attribute can be specified explicitly. It is also set implicitly when ipv6_generate_hostname is set to a defined value.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

Set the parameter to "true" to override the grid-level setting for ipv6_generate_hostname. Set the parameter to "false" to inherit the grid-level setting for ipv6_generate_hostname.

The default value of this parameter is "false".

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Getting override_ipv6_generate_hostname
my $override_ipv6_generate_hostname=$memberdhcp->override_ipv6_generate_hostname( );
#Modifying override_ipv6_generate_hostname
$memberdhcp->override_ipv6_generate_hostname("true");
```

override_ipv6_microsoft_code_page( )

The override_ipv6_microsoft_code_page attribute controls whether the ipv6_microsoft_code_page value in the DHCP member is used, instead of the Grid default.

The override_ipv6_microsoft_code_page attribute can be specified explicitly. It is also set implicitly when ipv6_microsoft_code_page is set to a defined value.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

Set the parameter to "true" to override the grid-level setting for ipv6_microsoft_code_page. Set the parameter to "false" to inherit the grid-level setting for ipv6_microsoft_code_page.

The default value of this parameter is "false".

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Getting override_ipv6_microsoft_code_page
my $override_ipv6_microsoft_code_page=$memberdhcp->override_ipv6_microsoft_code_page( );
```

```
#Modifying override_ipv6_microsoft_code_page
$memberdhcp->override_ipv6_microsoft_code_page("true");
```

override_ipv6_options( )

The override_ipv6_options attribute controls whether the ipv6_options value in the DHCP member is used, instead of the Grid default.

The override_ipv6_options attribute can be specified explicitly. It is also set implicitly when ipv6_options is set to a defined value.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

Set the parameter to "true" to override the grid-level setting for ipv6_options. Set the parameter to "false" to inherit the grid-level setting for ipv6_options.

The default value of this parameter is "false".

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Getting override_ipv6_options
my $override_ipv6_options=$memberdhcp->override_ipv6_options( );
#Modifying override_ipv6_options
$memberdhcp->override_ipv6_options("true");
```

override_ipv6_recycle_leases( )

The override_ipv6_recycle_leases attribute controls whether the ipv6_recycle_leases value in the DHCP member is used, instead of the Grid default.

The override_ipv6_recycle_leases attribute can be specified explicitly. It is also set implicitly when ipv6_recycle_leases is set to a defined value.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

Set the parameter to "true" to override the grid-level setting for ipv6_recycle_leases. Set the parameter to "false" to inherit the grid-level setting for ipv6_recycle_leases.

The default value of this parameter is "false".

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Getting override_ipv6_recycle_leases
my $override_ipv6_recycle_leases=$memberdhcp->override_ipv6_recycle_leases( );
#Modifying override_ipv6_recycle_leases
$memberdhcp->override_ipv6_recycle_leases("true");
```

override_ipv6_update_dns_on_lease_renewal( )

The override_ipv6_update_dns_on_lease_renewal attribute controls whether the ipv6_update_dns_on_lease_renewal value in the DHCP member is used, instead of the Grid default.

The override_ipv6_update_dns_on_lease_renewal attribute can be specified explicitly. It is also set implicitly when ipv6_update_dns_on_lease_renewal is set to a defined value.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

Set the parameter to "true" to override the grid-level setting for ipv6_update_dns_on_lease_renewal. Set the parameter to "false" to inherit the grid-level setting for ipv6_update_dns_on_lease_renewal.

The default value of this parameter is "false".

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
    #Getting override_ipv6_update_dns_on_lease_renewal
    my $override_ipv6_update_dns_on_lease_renewal=$memberdhcp-
>override_ipv6_update_dns_on_lease_renewal( );
    #Modifying override_ipv6_update_dns_on_lease_renewal
    $memberdhcp->override_ipv6_update_dns_on_lease_renewal("true");
```

override_lease_per_client_settings( )

Use this method to set or retrieve the override_lease_per_client_settings flag.

The override_lease_per_client_settings attribute can be specified explicitly. It is also set implicitly when lease_per_client_settings is set to a defined value.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

Set the parameter to "true" to override the grid-level setting for lease_per_client_settings. Set the parameter to "false" to inherit the grid-level setting for lease_per_client_settings.

The default value of this parameter is "false".

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get override_lease_per_client_settings value
my $value = $object->override_lease_per_client_settings();

#Modify override_lease_per_client_settings value
$object->override_lease_per_client_settings("true");
```

override_lease_scavenge_time( )

The override_lease_scavenge_time attribute controls whether the lease_scavenge_time value in the member DHCP server is used, instead of the Grid default.

The override_lease_scavenge_time attribute can be specified explicitly. It is also set implicitly when lease_scavenge_time is set to a defined value.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

Set the parameter to "true" to override the grid-level setting for lease_scavenge_time. Set the parameter to "false" to inherit the grid-level setting for lease_scavenge_time.

The default value of this parameter is "false".

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
    #Getting override_lease_scavenge_time
    my $override_lease_scavenge_time=$memberdhcp->override_lease_scavenge_time( );
    #Modifying override_lease_scavenge_time
    $memberdhcp->override_lease_scavenge_time("true");
```

override_log_lease_events( )

The override_log_lease_events attribute controls whether the log_lease_events value in the member DHCP server is used, instead of the Grid default.

The override_log_lease_events attribute can be specified explicitly. It is also set implicitly when log_lease_events is set to a defined value.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

Set the parameter to "true" to override the grid-level setting for log_lease_events. Set the parameter to "false" to inherit the grid-level setting for log_lease_events.

The default value of this parameter is "false".

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Getting override_log_lease_events
my $override_log_lease_events=$memberdhcp->override_log_lease_events( );
#Modifying override_log_lease_events
$memberdhcp->override_log_lease_events("true");
```

override_preferred_lifetime( )

The override_preferred_lifetime attribute controls whether the preferred_lifetime value in the DHCP member is used, instead of the Grid default.

The override_preferred_lifetime attribute can be specified explicitly. It is also set implicitly when preferred_lifetime is set to a defined value.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

Set the parameter to "true" to override the grid-level setting for preferred_lifetime. Set the parameter to "false" to inherit the grid-level setting for preferred_lifetime.

The default value of this parameter is "false".

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Getting override_preferred_lifetime
my $override_preferred_lifetime=$memberdhcp->override_preferred_lifetime( );
#Modifying override_preferred_lifetime
$memberdhcp->override_preferred_lifetime("true");
```

override_update_dns_on_lease_renewal( )

The override_update_dns_on_lease_renewal attribute controls whether the update_dns_on_lease_renewal value in the member DHCP server is used, instead of the Grid default.

The override_update_dns_on_lease_renewal attribute can be specified explicitly. It is also set implicitly when update_dns_on_lease_renewal is set to a defined value.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

Set the parameter to "true" to override the grid-level setting for update_dns_on_lease_renewal. Set the parameter to "false" to inherit the grid-level setting for update_dns_on_lease_renewal.

The default value of this parameter is "false".

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Getting override_update_dns_on_lease_renewal
my $override_update_dns_on_lease_renewal=$memberdhcp->override_update_dns_on_lease_renewal( );
#Modifying override_update_dns_on_lease_renewal
$memberdhcp->override_update_dns_on_lease_renewal("true");
```

override_valid_lifetime( )

The override_valid_lifetime attribute controls whether the valid_lifetime value in the Grid Member DHCP is used, instead of the upper-level default.

The override_valid_lifetime attribute can be specified explicitly. It is also set implicitly when valid_lifetime is set to a defined value.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

Set the parameter to "true" to override the grid-level setting for valid_lifetime. Set the parameter to "false" to inherit the upper-level setting for valid_lifetime.

The default value of this parameter is "false".

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Getting override_valid_lifetime
my $override_valid_lifetime=$memberdhcp->override_valid_lifetime( );
#Modifying override_valid_lifetime
$memberdhcp->override_valid_lifetime("true");
```

ping_count( )

Use this method to set or retrieve the ping_count value of a member DHCP server. This value specifies the number of pings that the Infoblox appliance sends to an IP address to verify that it is not in use.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

Number of pings. Range is from 0 to 10, where 0 disables pings. The default value is 1.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get ping_count
my $ping_count = $memberdhcp->ping_count();
#Modify ping_count
$memberdhcp->ping_count(4);
```

ping_timeout( )

Use this method to set or retrieve the ping_timeout value of a member DHCP server. This value indicates the number of seconds the appliance waits for a response to its ping.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

Ping timeout value in seconds. Valid values are 0.1, 0.5, 1, 2, 3, 4 and 5 seconds. The default value is 1.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get ping_timeout
my $ping_timeout = $memberdhcp->ping_timeout();
```

```
#Modify ping_timeout
$memberdhcp->ping_timeout(3);
```

preferred_lifetime( )

Use this method to set or retrieve preferred lifetime value.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Setting this method to a defined value implicitly sets the override_preferred_lifetime method to "true". Setting the parameter to undefined causes the appliance to use the Grid default and automatically resets the override_preferred_lifetime attribute to "false".

Note that when preferred_lifetime is set to a defined value and override_dpreferred_lifetime is set to "false", the last operation takes precedence. Thus the sequence $object->preferred_lifetime(12345); $object->override_preferred_lifetime("false"); will set override_preferred_lifetime to "false", and the sequence $object->override_preferred_lifetime("false"); $object->preferred_lifetime(12345); will result in override_preferred_lifetime="true".

Parameter

Preferred lifetime value in seconds. The default value is 27000.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get preferred_lifetime
my $preferred_lifetime = $memberdhcp->preferred_lifetime();
#Modify preferred_lifetime
$memberdhcp->preferred_lifetime(12345);
```

pxe_lease_time( )

Use this method to set or retrieve the pxe_lease_time value of a member DHCP server. You can specify the duration of time it takes a host to connect to a boot server, such as a TFTP server, and download the file it needs to boot.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

A 32-bit integer (range from 0 to 4294967295) that specifies the lease time in seconds. Default value is 43200 seconds.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get pxe_lease_time
my $pxe_lease_time = $memberdhcp->pxe_lease_time();
#Modify pxe_lease_time
$memberdhcp->pxe_lease_time(200);
```

recycle_leases( )

Use this method to set or retrieve the recycle_leases value of a member DHCP server. If you enable this check box, and then delete a DHCP range, the appliance stores active leases from this range up to one week after the leases expires.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

Specify "true" to set recycle_leases flag or "false" to deactivate/unset it.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get recycle_leases
my $recycle_leases = $memberdhcp->recycle_leases();
```

```
#Modify recycle_leases
$memberdhcp->recycle_leases("true");
```

retry_ddns_updates( )

Use this method to set or retrieve the retry_ddns_updates value of a member DHCP server. This value indicates whether the DHCP server makes repeated attempts to send DDNS updates to a DNS server.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

Specify "true" to enable the DHCP server to retry DDNS updates or "false" to disable this feature.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get retry_ddns_updates
my $retry_ddns_updates = $memberdhcp->retry_ddns_updates();
#Modify retry_ddns_updates
$memberdhcp->retry_ddns_updates("true");
```

syslog_facility( )

Use this method to set or retrieve the syslog_facility value of a member DHCP server. The syslog facility is the location on the syslog server to which you want to sort the syslog messages.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

Valid values are: `"daemon", "local0", "local1", "local2", "local3", "local4", "local5", "local6", "local7"`.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get syslog_facility
my $syslog_facility = $memberdhcp->syslog_facility();
#Modify syslog_facility
$memberdhcp->syslog_facility("daemon");
```

update_dns_on_lease_renewal( )

The update_dns_on_lease_renewal attribute controls whether the DHCP server updates DNS when a DHCP lease is renewed.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Setting this method to a defined value implicitly sets the override_update_dns_on_lease_renewal method to "true". Setting the parameter to undefined causes the appliance to use the Grid default and automatically resets the override_update_dns_on_lease_renewal attribute to "false".

Note that when update_dns_on_lease_renewal is set to a defined value and override_update_dns_on_lease_renewal is set to "false", the last operation takes precedence. Thus the sequence $object->update_dns_on_lease_renewal("true"); $object->override_update_dns_on_lease_renewal("false"); will set override_update_dns_on_lease_renewal to "false", and the sequence $object->override_update_dns_on_lease_renewal("false"); $object->update_dns_on_lease_renewal("true"); will result in override_update_dns_on_lease_renewal="true".

Parameter

Specify "true" to set the update_dns_on_lease_renewal flag or "false" to deactivate/unset it.

The default value of this parameter is "false".

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get update_dns_on_lease_renewal
my $update_dns_on_lease_renewal=$memberdhcp->update_dns_on_lease_renewal( );
#Modify update_dns_on_lease_renewal
$memberdhcp->update_dns_on_lease_renewal("true");
```

valid_lifetime( )

Use this method to set or retrieve the value of valid_lifetime for Grid Member DHCP.

Setting this method to a defined value implicitly sets the override_valid_lifetime method to "true". Setting the parameter to undefined causes the appliance to use the upper-level default and automatically resets the override_valid_lifetime attribute to "false".

Note that when valid_lifetime is set to a defined value and override_valid_lifetime is set to "false", the last operation takes precedence. Thus the sequence $object->valid_lifetime(20000); $object->override_valid_lifetime("false"); will set override_valid_lifetime to "false", and the sequence $object->override_valid_lifetime("false"); $object->valid_lifetime(20000); will result in override_valid_lifetime="true".

Parameter

Valid lifetime value in seconds. The default value is 43200.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get valid_lifetime
my $valid_lifetime = $memberdhcp->valid_lifetime();
#Modify valid_lifetime
$memberdhcp->valid_lifetime(12345);
```

SAMPLE CODE

The following sample code demonstrates different operations that can be applied to a member DHCP object, such as get and modify an object. This sample code also includes error handling for the operations.

#Preparation prior to operations on a member DHCP object

```
#PROGRAM STARTS: Include all the modules that will be used
use strict;
use Infoblox;

#Create a session to the Infoblox appliance
my $session = Infoblox::Session->new(
    master   => "192.168.1.2",
    username => "admin",
    password => "infoblox"
);
unless ($session) {
   die("Construct session failed: ",
       Infoblox::status_code() . ":" . Infoblox::status_detail());
}
print "Session created successfully\n";
```

#Get and modify the member DHCP property object.

```
#Get the member DHCP object. Considering the id grid member named infoblox.localdomain was previously
created.
my @object = $session->get(
    object => "Infoblox::Grid::Member::DHCP",
    name   => "infoblox.localdomain"
    )
    or die("get member DHCP failed: ",
         $session->status_code(), $session->status_detail());

#Modify member DHCP object.
my $option1 = Infoblox::DHCP::Option->new(
    name  => "lease-time",
    value => 50
);
```

```
my $option2 = Infoblox::DHCP::Option->new(
    name  => "domain-name",
    value => "domainname.com",
);

unless (scalar(@object) == 0) {
    my $object = $object[0];

    if ($object) {
        $object->enable_dhcp("true");
        $object->authority("true");
        $object->enable_ddns("true");
        $object->ddns_update_fixed_addresses("true");
        $object->ddns_use_option81("true");
        $object->ddns_server_always_updates("true");
        $object->ddns_generate_hostname("true");
        $object->pxe_lease_time(1000);
        $object->options([$option1, $option2]);
        $object->microsoft_code_page("Cyrillic (ISO-5589-5)");

        #Un-override bootfile
        $object->bootfile(undef);

        #Un-override bootserver
        $object->bootserver(undef);

        #Un-override nextserver
        $object->nextserver(undef);

        #Applying the changes
        $session->modify($object)
            or die("Modify member DHCP failed: ",
                $session->status_code(), $session->status_detail());

    }
     } else {
    print "No grid member DHCP found.";
     }

####PROGRAM ENDS####
```

AUTHOR

Infoblox Inc. http://www.infoblox.com/

SEE ALSO

Infoblox::Grid::Member::DHCP,Infoblox::DHCP::Option,Infoblox::Grid::KerberosKey,Infoblox::Session, Infoblox::Session->get(),Infoblox::Session->modify(), Infoblox::DHCP::Filter::MAC, Infoblox::DHCP::DDNS

COPYRIGHT

Copyright (c) 2017 Infoblox Inc.

Infoblox::Grid::Member::Discovery::ScanInterface - The scan interfaces structure.

- NAME
- DESCRIPTION
- CONSTRUCTOR
- MODULE METHODS
  - Infoblox::Grid::Member::DiscoveryProperties->scan_interfaces( )
- SESSION METHODS
- METHODS
  - scan_interface_type( )
  - network_view( )
  - vlan_id( )
- AUTHOR
- COPYRIGHT

NAME

Infoblox::Grid::Member::Discovery::ScanInterface - The scan interfaces structure.

DESCRIPTION

The object provides information about the scan interfaces.

CONSTRUCTOR

```
my $object = Infoblox::Grid::Member::Discovery::ScanInterface->new(
    'scan_interface_type' => (LAN1|LAN2|MGMT), # Required
    'network_view' => $string,         # Required
    'vlan_id' => $uint,                # Optional
);
```

MODULE METHODS

The following functions can be applied to a Discovery Properties object.

Infoblox::Grid::Member::DiscoveryProperties->scan_interfaces( )

Use this method to set or retrieve the scan interfaces configured for the Grid member discovery properties. See Infoblox::Grid::Member::DiscoveryProperties->scan_interfaces() for parameters and return values.

Example

```
#Get scan_interfaces value
my $scan_interfaces = $discovery_properties->scan_interfaces();
#Modify scan_interfaces
$discovery_properties->scan_interfaces($scan_interfaces);
```

SESSION METHODS

The object does not support any session methods.

METHODS

This section describes all the methods that you can use to set or retrieve the attribute values of the object.

scan_interface_type( )

Use this method to set or retrieve the probing interface type associated with the discovery probe member.

Parameter

The name of the probing interface (LAN1, LAN2, MGMT).

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

The method returns the value of scan_interface_type attribute.

Example

```
# Get attribute value
my $value = $object->scan_interface_type();
# Modify attribute value
my $result = $object->scan_interface_type('LAN1');
```

network_view( )

Use this method to set or retrieve the name of the network view associated with the interface.

Parameter

The name of the network view associated with the interface.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

The method returns the name of the network view associated with the interface.

Example

```
# Get attribute value
my $value = $object->network_view();
# Modify attribute value
my $result = $object->('papi_network_view');
```

vlan_id( )

Use this method to set or retrieve information for the VLAN tag of the probing network. This method is allowed only for the LAN1 and LAN2 interfaces.

Parameter

VLAN tag of the probing network.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
# Get attribute value
my $value = $object->vlan_id();
# Modify attribute value
$object->vlan_id(22);
```

AUTHOR

Infoblox Inc. http://www.infoblox.com/

COPYRIGHT

Copyright (c) 2017 Infoblox Inc.

Infoblox::Grid::Member::Discovery::SDNConfig - The SDN controller configuration structure.

- NAME
- DESCRIPTION
- CONSTRUCTOR
- MODULE METHODS
  - Infoblox::Grid::Member::DiscoveryProperties->sdn_configs( )
- SESSION METHODS
- METHODS
  - sdn_type( )
  - addresses( )
  - network_view( )
  - protocol( )
  - handle( )
  - password( )
  - username( )
  - api_key( )
  - on_prem( )
  - use_global_proxy( )
  - ca_certificate( )
  - comment( )
  - uuid( )
  - network_interface_type( )
  - network_interface_virtual_ip( )
- AUTHOR
- SEE ALSO
- COPYRIGHT

NAME

Infoblox::Grid::Member::Discovery::SDNConfig - The SDN controller configuration structure.

DESCRIPTION

The object provides information about the SDN controller configuration.

CONSTRUCTOR

```
my $object = Infoblox::Grid::Member::Discovery::SDNConfig->new(
    'sdn_type'                    => 'CISCO_APIC' | 'MERAKI', # Required
    'addresses'                   => ARRAY[$ipaddr | $fqdn],  # Required
    'network_view'                => $string,                 # Optional
    'protocol'                    => 'HTTP' | 'HTTPS',        # Optional
    'handle'                      => $string,                 # Required
    'password'                    => $string,                 # Optional
    'username'                    => $string,                 # Optional
    'api_key'                     => $string,                 # Optional
    'on_prem'                     => $bool,                   # Optional
    'use_global_proxy'            => $bool,                   # Optional
    'ca_certificate'              => $ca_cert,                # Optional
    'comment'                     => $string,                 # Optional
    'uuid'                        => $string,                 # Optional
    'network_interface_type'      => 'LAN1', 'LAN2', 'MGMT'   # Optional
    'network_interface_virtual_ip' => $vip                    # Optional
);
```

MODULE METHODS

The following functions can be applied to a SDN controller configuration structure.

Infoblox::Grid::Member::DiscoveryProperties->sdn_configs( )

Use this method to set or retrieve the SDN controller configurations configured for the Grid member discovery properties. See Infoblox::Grid::Member::DiscoveryProperties->sdn_configs() for parameters and return values.

Example

```
#Get sdn_configs value
my $sdn_configs = $discovery_properties->sdn_configs();
```

```
#Modify sdn_configs
$discovery_properties->sdn_configs([$sdn_config1, $sdn_config2]);
```

SESSION METHODS

The object does not support any session methods.

METHODS

This section describes all the methods that you can use to set or retrieve the attribute values of the object.

sdn_type( )

Use this method to set or retrieve SDN controller type.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

The valid values are 'CISCO_APIC' and 'MERAKI'.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
# Get attribute value
my $value = $object->sdn_type();
# Modify attribute value
my $result = $object->sdn_type('CISCO_APIC');
```

addresses( )

Use this method to set or retrieve SDN controller address or FQDN.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

The desired SDN address or Fully-Qualified Domain Name (FQDN) in a string format as array.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
# Get attribute value
my $value = $object->addresses();
# Modify attribute value
my $result = $object->addresses(['10.11.0.1']);
```

network_view( )

Use this method to set or retrieve the network view associated with SDN controller.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

The valid value is a network view name that is associated with a SDN controller in a string format.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
# Get attribute value
my $value = $object->network_view();
```

```
# Modify attribute value
my $result = $object->network_view('papi_network_view');
```

protocol( )

Use this method to set or retrieve the connection protocol for the SDN controller.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

The valid values are 'HTTP' and 'HTTPS'.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
# Get attribute value
my $value = $object->protocol();
# Modify attribute value
$object->protocol('HTTP');
```

handle( )

Use this method to set or retrieve SDN controller unique configuration handle.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

The valid value is a desired SDN controller unique configuration handle in a string format.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
# Get attribute value
my $value = $object->handle();
# Modify attribute value
$object->handle('handle');
```

password( )

Use this method to set SDN controller login password. This is a write-only attribute.

Include the specified parameter to set the attribute value.

Parameter

SDN controller login password.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

Example

```
# Modify attribute value
$object->password('password');
```

username( )

Use this method to set or retrieve SDN controller login name.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

The valid value is a desired SDN controller login name in a string format.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
# Get attribute value
my $value = $object->username();
# Modify attribute value
$object->username('username');
```

api_key( )

Use this method to set or retrieve SDN controller api key.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

The valid value is a desired SDN controller api key in a string format.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
# Get attribute value
my $value = $object->api_key();
# Modify attribute value
$object->api_key('api_key');
```

on_prem( )

Use this method to set or retrieve SDN controller on-prem value. True if controller is on-premises. Otherwise we consider it is in cloud.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

The valid value is a desired SDN controller on-prem in a boolean format.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
# Get attribute value
my $value = $object->on_prem();
# Modify attribute value
$object->on_prem(true);
```

use_global_proxy( )

Use this method to set or retrieve SDN controller use_global_proxy value. Use global proxy settings to access SDN controller.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

The valid value is a desired SDN controller use_global_proxy value in a boolean format.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
# Get attribute value
my $value = $object->use_global_proxy();
# Modify attribute value
```

```
$object->use_global_proxy(false);
```

ca_certificate( )

Use this method to set or retrieve the CA certificate.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

The valid value is an Infoblox::Grid::CACertificate object.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
# Get attribute value
my $value = $object->ca_certificate();
# Modify attribute value
my $result = $object->ca_certificate($ca_cert);
```

comment( )

Use this method to set or retrieve the descriptive comment for the SDN controller configuration.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

The valid value is a desired comment in a string format.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
# Get attribute value
my $value = $object->comment();
# Modify attribute value
$object->comment('SDN configuration');
```

uuid( )

Use this method to set or retrieve SDN controller configuration UUID.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

The valid value is a desired SDN controller uuid in a string format.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
# Get attribute value
my $value = $object->uuid();
# Modify attribute value
$object->uuid('ABCDEF');
```

network_interface_type( )

Use this method to set or retrieve SDN controller network interface type.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

The valid values are 'LAN1', 'LAN2', 'MGMT'.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
# Get attribute value
my $value = $object->network_interface_type();
# Modify attribute value
$object->network_interface_type('LAN1');
```

network_interface_virtual_ip( )

Use this method to set or retrieve SDN controller network interface virtual ip.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

The valid value is a desired SDN controller interface virtual ip in a string format in case network interface type is VLAN.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
# Get attribute value
my $value = $object->network_interface_virtual_ip();
# Modify attribute value
$object->network_interface_virtual_ip(.);
```

AUTHOR

Infoblox Inc. http://www.infoblox.com/

SEE ALSO

Infoblox::Grid::Member::DiscoveryProperties->sdn_configs(), Infoblox::Grid::Member::DiscoveryProperties, Infoblox::Grid::CACertificate

---

COPYRIGHT

Copyright (c) 2019 Infoblox Inc.

Infoblox::Grid::Member::Discovery::VRF - Object represents the relation between network view and the virtual network membership.

- NAME
- DESCRIPTION
- SESSION METHODS
  - Infoblox::Session->get()
  - Infoblox::Session->search()
- METHODS
  - name()
  - network_view()
  - route_distinguisher()
  - description()
  - device()
- AUTHOR
- SEE ALSO
- COPYRIGHT

NAME

Infoblox::Grid::Member::Discovery::VRF - Object represents the relation between network view and the virtual network membership.

DESCRIPTION

This object is used to represents relation between network view and the virtual network membership. Object is read-only.

SESSION METHODS

Infoblox::Session->get()

Use this method to retrieve all the matching virtual network objects from the Infoblox appliance. See Infoblox::Session->get() for parameters and return values.

Key References

Apply the following attributes to get a specific virtual network objects:

```
name                - Optional. A name in string format.
description         - Optional. A description in string format.
route_distinguisher - Optional. Route distinguisher associated with the VRF in sting format.
network_view        - Optional. Name of the network view associated with the VRF in sting format.
```

Returns

Returns Virtual network membership object.

Example

```
#Get VRF object with the name 'VRF1'
my $vrf_obj = $session->get(
    object  => "Infoblox::Grid::Member::Discovery::VRF" ,
    name    => 'VRF1'
);
```

Infoblox::Session->search()

Use this method to search for a virtual network objects in the Infoblox appliance. See Infoblox::Session->search() for parameters and return values.

Key References

```
Apply the following attributes to search for a virtual network object:

name                - Optional. A name in string format (regular expression).
description         - Optional. A description in string format (regular expression).
route_distinguisher - Optional. Route distinguisher associated with the VRF in sting format.
network_view        - Optional. Name of the network view associated with the VRF in sting format.
```

Returns

Returns Virtual network membership object(s).

Example

```
# Get all VRFs
my @all_vrfs = $session->search(
      object => 'Infoblox::Grid::Member::Discovery::VRF',
      name => '^.*$'
);

# Get vrf by Route distinguisher
my $vrf = $session->search(
      object => 'Infoblox::Grid::Member::Discovery::VRF',
      route_distinguisher => '123:500'
);
```

METHODS

This section describes all the methods that can be used to retrieve values from VRF object.

`name()`

Use this method to retrieve name associated with the VRF. This is a read-only attribute.

Parameter

None

Returns

The method returns the attribute value.

Example

```
#Get VRF name
my $vrf_name = $vrf->name();
```

`network_view()`

Use this method to retrieve name of network view associated with the VRF. This is a read-only attribute.

Parameter

None

Returns

The returned string contains name network view. This is a read-only attribute.

Example

```
#Get VRF network view name
my $vrf_nv = $vrf->network_view();
```

`route_distinguisher()`

Use this method to retrieve route distinguisher associated with the VRF. This is a read-only attribute.

Parameter

None

Returns

The method returns the attribute value.

Example

```
#Get VRF route distinguisher
my $vrf_rd = $vrf->route_distinguisher();
```

`description()`

Use this method to retrieve description associated with the VRF. This is a read-only attribute.

Parameter

None

Returns

The method returns the attribute value.

Example

```
#Get VRF description
my $vrf_desc = $vrf->description();
```

device()

Use this method to retrieve Infoblox::Grid::Discovery::Device object from which this VRF membership configuration was collected. This is a read-only attribute.

Parameter

None

Returns

The valid return value is an Infoblox::Grid::Discovery::Device partial object.

Example

```
#Get VRF device
my $vrf_device = $vrf->device();
```

AUTHOR

Infoblox Inc. http://www.infoblox.com/

SEE ALSO

Infoblox::Grid::Discovery::Device Infoblox::Session, Infoblox::Session->get(), Infoblox::Session->search()

COPYRIGHT

Copyright (c) 2017 Infoblox Inc.

Infoblox::Grid::Member::DiscoveryProperties - Grid member discovery properties.

- NAME
- DESCRIPTION
- CONSTRUCTOR
- SESSION METHODS
  - Infoblox::Session->get( )
  - Infoblox::Session->modify( )
  - Infoblox::Session->search( )
- METHODS
  - sdn_configs( )
  - default_seed_routers( )
  - enable_service( )
  - cli_credentials( )
  - member( )
  - override_snmp3_credentials( )
  - override_cli_credentials( )
  - override_snmp_credentials( )
  - role( )
  - seed_routers( )
  - snmp3_credentials( )
  - snmp_credentials( )
  - test_credential( )
  - scan_interfaces( )
- AUTHOR
- SEE ALSO
- COPYRIGHT

NAME

Infoblox::Grid::Member::DiscoveryProperties - Grid member discovery properties.

DESCRIPTION

The object provides information about Grid member discovery properties.

CONSTRUCTOR

```
my $object = Infoblox::Grid::Member::DiscoveryProperties->new(
    'member'                    => $string,                     # Optional
    'enable_service'            => 'true' | 'false',            # Optional
    'scan_interfaces'           => [$si1, ...],                 # Optional
    'role'                      => 'DNM' | 'DNP' | 'NONE',      # Optional
    'seed_routers'              => [$seed1, ...],               # Optional
    'snmp3_credentials'         => [$snmp3, ...],               # Optional
    'override_snmp3_credentials' => 'true' | 'false',           # Optional
    'snmp_credentials'          => [$snmp, ...],                # Optional
    'override_snmp_credentials'  => 'true' | 'false',           # Optional
    'cli_credentials'           => [$cli1, ...],                # Optional
    'override_cli_credentials'   => 'true' | 'false',           # Optional
    'sdn_configs'               => [$conf1, ...],               # Optional
);
```

SESSION METHODS

This section describes all the methods in Infoblox::Session that you can apply to the object.

Infoblox::Session->get( )

Use this method to retrieve all the matching objects from the Infoblox appliance. See Infoblox::Session->get() for parameters and return values.

Key References

```
Apply the following attributes to get a specific object:

 member - Optional. The member name.
```

Example

```
my @retrieved_objs = $session->get(
    'object' => 'Infoblox::Grid::Member::DiscoveryProperties',
    'member' => 'member.com',
);
```

Infoblox::Session->modify( )

Use this method to modify an object in the Infoblox appliance. See Infoblox::Session->modify() for parameters and return values.

Example

```
# Use this method to modify role.
$object->role("DNP");
# Submit modification
my $response = $session->modify( $object );
```

Infoblox::Session->search( )

Use this method to search for objects in the Infoblox appliance. See Infoblox::Session->search() for parameters and return values.

Key References

```
Apply the following attributes to search for a object:

 member - Optional. The member name.
```

Example

```
# search for all objects
my @retrieved_objs = $session->search(
    'object' => 'Infoblox::Grid::Member::DiscoveryProperties',
    'member' => 'member.com',
);
```

METHODS

This section describes all the methods that you can use to set or retrieve the attribute values of the object.

sdn_configs( )

Use this method to set or retrieve the list of SDN configurations for a given probe Member.

Parameter

Valid value is an array reference that contains Infoblox::Grid::Member::Discovery::SDNConfig objects.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
# Get attribute value
my $value = $object->sdn_configs();
# Construct SDN config
my $conf = Infoblox::Grid::Member::Discovery::SDNConfig->new(
    'sdn_type'       => 'CISCO_APIC',
    'addresses'      => ['apic.aci.com'],
    'network_view'   => 'default',
    'protocol'       => 'HTTPS',
    'handle'         => 'ACIConf1',
    'password'       => 'infoblox',
    'username'       => 'admin',
    'ca_certificate' => $cert,
    'comment'        => 'test_sdn_config',
);
$object->sdn_configs([$conf]);
```

default_seed_routers( )

Use this method to retrieve a list of default seed routers. This is a read-only attribute.

Parameter

None

Returns

The method returns the attribute value.

Example

```
# Get attribute value
my $value = $object->default_seed_routers();
```

enable_service( )

Use this method to enable or disable the discovery service.

Parameter

Specify "true" to enable the discovery service or "false" to disable it. Default value is "false".

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
# Get attribute value
my $value = $object->enable_service();
# Modify attribute value
$object->enable_service('true');
```

cli_credentials( )

Use this method to set or retrieve the Discovery CLI credentials.

Setting this method to a defined value implicitly sets the override_cli_credentials method to "true". Setting the parameter to undefined causes the appliance to use the grid default and automatically resets the override_cli_credentials attribute to "false".

Note that when cli_credentials is set to a defined value and override_cli_credentials is set to "false", the last operation takes precedence. Thus the sequence $object->cli_credentials([$snmp]); $object->override_cli_credentials("false"); will set override_cli_credentials to "false", and the sequence $object->override_cli_credentials("false"); $object->cli_credentials([$snmp]); will result in override_cli_credentials="true".

Parameter

Valid value is an array reference that contains Infoblox::Grid::Discovery::CLICredential objects.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
# Get attribute value
my $value = $object->cli_credentials();
# Modify attribute value
$object->cli_credentials([$cli1, $cli2]);
```

member( )

Use this method to set or retrieve the member name.

Parameter

The member name in string format.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
# Get attribute value
my $value = $object->member();
```

```
# Modify attribute value
$object->member('infoblox.com');
```

override_snmp3_credentials( )

The override_snmp3_credentials attribute controls whether the snmp3_credentials value in the member is used, instead of the grid default.

The override_snmp3_credentials attribute can be specified explicitly. It is also set implicitly when snmp3_credentials is set to a defined value.

Parameter

Specify "true" to set the override_snmp3_credentials flag or "false" to deactivate/unset it. The default value is "false".

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
# Get attribute value
my $value = $object->override_snmp3_credentials();
# Modify attribute value
$object->override_snmp3_credentials('true');
```

override_cli_credentials( )

The override_cli_credentials attribute controls whether the cli_credentials value in the member is used, instead of the grid default.

The override_cli_credentials attribute can be specified explicitly. It is also set implicitly when cli_credentials is set to a defined value.

Parameter

Specify "true" to set the override_cli_credentials flag or "false" to deactivate/unset it. The default value is "false".

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
# Get attribute value
my $value = $object->override_cli_credentials();
# Modify attribute value
$object->override_cli_credentials('true');
```

override_snmp_credentials( )

The override_snmp_credentials attribute controls whether the snmp_credentials value in the member is used, instead of the grid default.

The override_snmp_credentials attribute can be specified explicitly. It is also set implicitly when snmp_credentials is set to a defined value.

Parameter

Specify "true" to set the override_snmp_credentials flag or "false" to deactivate/unset it. The default value is "false".

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
# Get attribute value
my $value = $object->override_snmp_credentials();
# Modify attribute value
$object->override_snmp_credentials('true');
```

role( )

Use this method to set or retrieve the role of the discovery member.

Parameter

Valid value is 'DNM', 'DNP' or 'NONE'.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
# Get attribute value
my $value = $object->role();
# Modify attribute value
$object->role('NONE');
```

seed_routers( )

Use this method to set or retrieve seed routers.

Parameter

Valid value is an array reference that contains Infoblox::Grid::Discovery::SeedRouter objects.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
# Get attribute value
my $value = $object->seed_routers();
# Modify attribute value
$object->seed_routers([$seed1, $seed2]);
```

snmp3_credentials( )

Use this method to set or retrieve discovery SNMPv3 credentials.

Setting this method to a defined value implicitly sets the override_snmp3_credentials method to "true". Setting the parameter to undefined causes the appliance to use the grid default and automatically resets the override_snmp3_credentials attribute to "false".

Note that when snmp3_credentials is set to a defined value and override_snmp3_credentials is set to "false", the last operation takes precedence. Thus the sequence $object->snmp3_credentials([$snmp3]); $object->override_snmp3_credentials("false"); will set override_snmp3_credentials to "false", and the sequence $object->override_snmp3_credentials("false"); $object->snmp3_credentials([$snmp3]); will result in override_snmp3_credentials="true".

Parameter

Valid value is an array reference that contains Infoblox::Grid::Discovery::SNMP3Credential objects.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
# Get attribute value
my $value = $object->snmp3_credentials();
# Modify attribute value
$object->snmp3_credentials([$snmp3]);
```

snmp_credentials( )

Use this method to set or retrieve SNMPv1 and SNMPv2 credentials.

Setting this method to a defined value implicitly sets the override_snmp_credentials method to "true". Setting the parameter to undefined causes the appliance to use the grid default and automatically resets the override_snmp_credentials attribute to "false".

Note that when snmp_credentials is set to a defined value and override_snmp_credentials is set to "false", the last operation takes precedence. Thus the sequence $object->snmp_credentials([$snmp]); $object->override_snmp_credentials("false"); will set override_snmp_credentials to "false", and the sequence $object->override_snmp_credentials("false"); $object->snmp_credentials([$snmp]); will result in override_snmp_credentials="true".

Parameter

Valid value is an array reference that contains Infoblox::Grid::Discovery::SNMPCredential objects.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
# Get attribute value
my $value = $object->snmp_credentials();
# Modify attribute value
$object->snmp_credentials([$snmp]);
```

test_credential( )

Use this method to test credentials.

Parameter

```
id               - Optional. The test credential ID.
```

The following members are supported when id is defined:

```
start            - Optional. The start position of the text.
```

The following members are supported when id is not defined:

```
view             - Optional. The network view name. The default value is the system-defined default
network view.

type             - Optional if one of the following is defined: cli_credentials, snmp_credential or
snmp3_credential. Otherwise, this member is required. The valid value is 'CLI', 'SNMP' or 'SNMP3'.

device           - Required if address is not defined. The Infoblox::Grid::Discovery::Device object.

member           - Optional. The discovery member name.

address          - Required if device is not defined. The IP address of the device.

cli_credentials  - Optional. At least one of the following must be defined: cli_credentials,
snmp_credential, or  snmp3_credential. The array reference that contains
Infoblox::Grid::Discovery::CLICredential objects.

snmp_credential  - Optional. At least one of the following must be defined: cli_credentials,
snmp_credential, or  snmp3_credential. The Infoblox::Grid::Discovery::SNMPCredential object.

snmp3_credential - Optional. At least one of the following must be defined: cli_credentials,
snmp_credential, or snmp3_credential. The Infoblox::Grid::Discovery::SNMP3Credential object.
```

Returns

The method returns an Infoblox::Grid::Discovery::TestCredential object when the operation succeeds, and returns false when the operation fails.

Example

```
# Get a test credential ID
my $result = $object->test_credential(
   'address'         => '10.0.0.1',
   'snmp_credential' => $snmp,
);

# Get detailed information by id
if (ref($result) eq 'Infoblox::Grid::Discovery::TestCredential') {
    $result = $object->test_credential('id' => $result->id());
}
```

scan_interfaces( )

Use this method to set or retrieve the scan interface configuration for a given probe member.

Parameter

Valid value is an array reference that contains Infoblox::Grid::Member::Discovery::ScanInterface objects.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
# Get attribute value
my $value = $object->scan_interfaces();
# Modify attribute value
$object->scan_interfaces([$interface1, $interface2]);
```

AUTHOR

Infoblox Inc. http://www.infoblox.com/

SEE ALSO

Infoblox::Session, Infoblox::Grid::Member::Discovery::SDNConfig, Infoblox::Grid::Discovery::SNMPCredential, Infoblox::Grid::Discovery::SNMP3Credential, Infoblox::Grid::Discovery::SeedRouter, Infoblox::Grid::Discovery::TestCredential, Infoblox::Grid::Discovery::CLICredential, Infoblox::Grid::Member::Discovery::ScanInterface

---

COPYRIGHT

Copyright (c) 2017 Infoblox Inc.

Infoblox::Grid::Member::DNS - Grid Member DNS object.

- NAME
- DESCRIPTION
- CONSTRUCTOR
- SESSION METHODS
  - Infoblox::Session->get( )
  - Infoblox::Session->modify( )
- METHODS
  - anonymize_response_logging( )
  - allow_gss_tsig_zone_updates( )
  - allow_query( )
  - allow_recursive_query( )
  - allow_transfer( )
  - allow_update( )
  - attack_mitigation( )
  - auto_blackhole( )
  - auto_create_a_and_ptr_for_lan2( )
  - auto_create_a_and_ipv6ptr_for_lan2( )
  - auto_sort_views( )
  - blackhole_list( )
  - blacklist_action( )
  - blacklist_log_query( )
  - blacklist_redirect_addresses( )
  - blacklist_redirect_ttl( )
  - blacklist_rulesets( )
  - bind_check_names_policy( )
  - bind_hostname_directive( )
  - bind_hostname_directive_fqdn( )
  - capture_dns_queries_on_all_domains( )
  - copy_xfer_to_notify( )
  - custom_root_name_servers( )
  - disable_edns( )
  - dns_cache_acceleration_ttl( )
  - dns_cache_acceleration_status( )
  - dns_health_check_anycast_control( )
  - dns_health_check_domain_list( )
  - dns_health_check_interval( )
  - dns_health_check_recursion_flag( )
  - dns_health_check_retries( )
  - dns_health_check_timeout( )
  - dns_ip_list( )
  - dns64_groups( )
  - dnssec_enabled( )
  - dnssec_expired_signatures_enabled( )
  - dnssec_validation_enabled( )
  - dnssec_blacklist_enabled()
  - dnssec_negative_trust_anchors( )
  - dnssec_dns64_enabled()
  - dnssec_nxdomain_enabled()
  - dnssec_rpz_enabled()
  - dnssec_trusted_keys( )
  - dns_notify_transfer_source_interface( )
  - dns_query_capture_file_time_limit( )
  - dns_query_source_interface( )
  - dns_view_address_settings( )
  - domains_to_capture_dns_queries( )
  - dtc_edns_prefer_client_subnet( )
  - dtc_health_source_interface( )
  - enable_blackhole( )
  - enable_blacklist( )
  - enable_capture_dns_queries( )
  - enable_capture_dns_responses( )
  - enable_dns( )
  - enable_dns_cache_acceleration( )
  - enable_dns_health_check( )
  - enable_dns64( )
  - enable_excluded_domain_names( )
  - enable_fixed_rrset_order_fqdns( )
  - enable_ftc( )
  - enable_gss_tsig( )
  - enable_query_rewrite( )
  - excluded_domain_names( )

- extattrs( )
- extensible_attributes( )
- facility( )
- file_transfer_setting( )
- filter_aaaa( )
- filter_aaaa_list( )
- fixed_rrset_order_fqdns( )
- forwarders( )
- forward_only( )
- forward_updates( )
- ftc_expired_record_ttl( )
- ftc_expired_record_timeout( )
- glue_record_addresses( )
- gss_tsig_keys( )
- host_name_restriction_policy( )
- ipv6_glue_record_addresses( )
- is_unbound_capable( )
- lame_ttl( )
- logging_categories( )
- max_cache_ttl( )
- max_ncache_ttl( )
- max_cached_lifetime( )
- minimal_resp( )
- name( )
- notify_delay( )
- notify_source_port( )
- override_attack_mitigation( )
- override_auto_blackhole( )
- override_bind_hostname_directive( )
- override_server_id_directive( )
- override_capture_dns_queries_on_all_domains( )
- override_copy_xfer_to_notify( )
- override_disable_edns( )
- override_dns64( )
- override_dns_health_check( )
- override_filter_aaaa( )
- override_dns_cache_acceleration_ttl( )
- override_ftc( )
- override_gss_tsig_keys( )
- override_enable_capture_dns( )
- override_enable_excluded_domain_names( )
- override_enable_gss_tsig( )
- override_max_cache_ttl( )
- override_max_ncache_ttl( )
- override_max_cached_lifetime( )
- override_nxdomain_redirect( )
- override_lame_ttl( )
- override_rpz_qname_wait_recurse( )
- override_transfers_in( )
- override_transfers_out( )
- override_transfers_per_ns( )
- override_serial_query_rate( )
- override_query_rewrite( )
- nxdomain_redirect( )
- nxdomain_redirect_addresses( )
- nxdomain_redirect_addresses_ipv6( )
- nxdomain_redirect_ttl( )
- nxdomain_log_query( )
- nxdomain_rulesets( )
- override_blacklist( )
- override_dnssec( )
- override_dtc_edns_prefer_client_subnet( )
- override_forwarders( )
- override_fixed_rrset_order_fqdns( )
- query_source_port( )
- recursive_client_limit( )
- recursive_query_list( )
- resolver_query_timeout( )
- response_rate_limiting( )
- recursive_resolver( )
- serial_query_rate( )
- server_id_directive( )
- server_id_directive_string( )
- skip_in_grid_rpz_queries( )

- sortlist( )
- store_locally( )
- transfer_excluded_servers( )
- transfer_format( )
- transfers_in( )
- transfers_out( )
- transfers_per_ns( )
- unbound_logging_level( )
- use_lan_ipv6_port( )
- use_lan2_port( )
- use_lan2_ipv6_port( )
- use_mgmt_port( )
- use_mgmt_ipv6_port( )
- use_root_name_servers( )
- rpz_disable_nsdname_nsip( )
- rpz_drop_ip_rule_enabled( )
- rpz_drop_ip_rule_min_prefix_length_ipv4( )
- rpz_drop_ip_rule_min_prefix_length_ipv6( )
- rpz_qname_wait_recurse( )
- override_resolver_query_timeout( )
- override_response_rate_limiting( )
- override_rpz_disable_nsdname_nsip( )
- override_rpz_drop_ip_rule( )
- use_lan_port( )
- views( )

- SAMPLE CODE
- AUTHOR
- SEE ALSO
- COPYRIGHT

NAME

Infoblox::Grid::Member::DNS - Grid Member DNS object.

DESCRIPTION

You can configure the DNS properties of a grid member, including enabling/disabling DNS services and other DNS service parameters. The service configurations of a grid are inherited by all members. For this reason, it is recommended that you configure services at the grid level before configuring member services. When you configure DNS services at the grid level, all grid members inherit the grid-level configuration settings, unless you specifically override them for selected members.

CONSTRUCTOR

The Infoblox::Grid::Member::DNS object is automatically generated upon the successful installation of the Infoblox appliance. It does not require manual construction.

SESSION METHODS

This section describes all the methods in the Infoblox::Session module that you can apply to a grid member DNS object.

Infoblox::Session->get( )

Use this method to retrieve all the matching objects from the Infoblox appliance. See Infoblox::Session->get() for parameters and return values.

Key Reference

Apply the following attributes to get a specific grid member DNS object:

```
name - Optional.   Host name in string format
ipv4addr - Optional.   IPv4 address of the member
ipv6addr - Optional.   IPv6 address of the member
extattrs     - Optional. A hash reference containing extensible attributes.
extensible_attributes - Optional. A hash reference containing extensible attributes.
```

Example

```
my $object   = $session->get(
   object    => "Infoblox::Grid::Member::DNS",
   name      => "infoblox.localdomain",
    );

# get all Grid::Member::DNS objects with a given extensible attribute
```

```
my @retrieved_objs = $session->get(
    object => "Infoblox::Grid::Member::DNS",
    extensible_attributes => { 'Site' => 'Santa Clara' });
```

Infoblox::Session->modify( )

Use this method to modify an object in the Infoblox appliance. See Infoblox::Session->modify() for parameters and return values.

If you have applied an access control list to an object, you must restart services after you update the access control list for the changes to take effect. See Infoblox::Session->restart() method.

Example

```
#Use this method to modify the enable_dns  property.
$object->enable_dns("false");
#Submit modification
my $response = $session->modify( $object );
```

METHODS

This section describes all the methods that you can use to configure and retrieve the attribute values of the DNS properties of a grid member.

anonymize_response_logging( )

Use this method to set or retrieve the flag that indicates whether the anonymization of captured DNS responses is enabled or disabled.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

Specify 'true' to enable anonymization of captured DNS responses or 'false' to disable it. The default value is 'false'.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get anonymize_response_logging value
my $anonymize_response_logging = $object->anonymize_response_logging();

#Modify anonymize_response_logging value
$object->anonymize_response_logging('true');
```

allow_gss_tsig_zone_updates( )

Use this method to enable/disable GSS-TSIG zone updates for this member. This setting overrides the grid GSS-TSIG zone update settings.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

Specify "true" to allow GSS-TSIG zone updates for this member or "false" to disallow them. The default value is "false".

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails. If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get allow_gss_tsig_zone_updates
my $allow_gss_tsig_zone_updates = $memberdns->allow_gss_tsig_zone_updates( );
#Modify allow_gss_tsig_zone_updates
$memberdns->allow_gss_tsig_zone_updates("true");
```

allow_query( )

Use this method to add or retrieve an allow_query list.

Select "allow query" to permit queries from the specified IPv4 or IPv6 addresses and networks. The list specifies IP `address(es)` and/or `network(s)` from which queries are allowed or denied. The appliance can also use Transaction Signature (TSIG) keys to authenticate the

queries. This setting overrides the grid query settings.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

Valid value is an Infoblox::Grid::NamedACL object or an array reference that contains IPv4/IPv6 addresses, networks and/or Infoblox::DNS::TSIGKey object(s). To deny queries, add "!" as a prefix to `address(es)` and network(s). Specify "any" to allow queries from any address. By default, queries are allowed from any address. Undefine the value to use the Grid::DNS level allow_query list.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails. If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get allow_query
$memberdns->allow_query();
#Modify allow_query
$memberdns->allow_query( ["any"] );

#Specifying TSIG keys
my $tsig_key1 = $session->gen_tsig_key();
my $query_key = Infoblox::DNS::TSIGKey->new(
  name => "query_key1",
  key  => $tsig_key1
);
#Modify allow_query
$memberdns->allow_query( [ "10.0.0.10" , "20.0.0.0/24"
,"2001::123a","2003::222/64","!"."30.0.0.30" , "!"."40.10.0.0/24","!"."2004::125b","!"."2006::34bc"]
);

#Modify allow_query to use Grid level settings
$memberdns->allow_query(undef);
#Use Infoblox::Grid::NamedACL
$memberdns->allow_query($nacl);
```

allow_recursive_query( )

Use this method to enable/disable responses to recursive queries. This setting overrides grid recursive query settings.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

Specify "true" to allow responses to recursive queries or "false" to reject recursive queries. The default value is "false". Undefine to use the Grid::DNS level allow_recursive_query setting. Note: You should also undefine the recursive_query_list list to use the Grid::DNS level settings.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails. If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get allow_recursive_query
my $allow_recursive_query = $memberdns->allow_recursive_query( );
#Modify allow_recursive_query
$memberdns->allow_recursive_query("true");
```

allow_transfer( )

Use this method to allow or reject zone transfers from specified IPv4 or IPv6 addresses and networks or to allow transfers from hosts authenticated by Transaction signature (TSIG) key. This setting overrides the grid zone transfer settings.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

Valid value is an Infoblox::Grid::NamedACL object or an array reference that contains IPv4/IPv6 addresses, networks and/or Infoblox::DNS::TSIGKey object(s). To reject zone transfers, add '!' as a prefix to IP addresses and networks. Specify "any" to allow any transfer, or undefine to use the Grid::DNS level allow_transfer list.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Getting allow_transfer
my $memberdns = $object->allow_transfer( );
#Modifying allow_transfer
$memberdns->allow_transfer(["any"]);

#Specifying TSIG keys
my $tsig_key1 = $session->gen_tsig_key();
my $key1 = Infoblox::DNS::TSIGKey->new(
  name => "key1",
  key  => $tsig_key1
);
my $key2 = Infoblox::DNS::TSIGKey->new(
  name => "tsig_xfer",
  key  => ":2xCOMPAT"
);
#Modifying allow_transfer
$memberdns->allow_transfer(["10.0.0.10", $key1, "20.0.0.0/24", $key2,
"2004::325f","2007::876c/64","!"."30.0.0.30", "!"."40.10.0.0/24"]);

#Use Grid::DNS level allow_transfer list
$memberdns->allow_transfer(undef);

#Use Infoblox::Grid::NamedACL
$memberdns->allow_transfer($nacl);
```

allow_update( )

Use this method to allow or reject dynamic updates from specified IPv4 or IPv6 addresses, networks or from host authenticated by TSIG key. This setting overrides grid update settings.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

Valid value is an Infoblox::Grid::NamedACL object or an array reference that contains IPv4/IPv6 addresses, networks and/or Infoblox::DNS::TSIGKey object(s). To reject updates, add '!' as a prefix to IP addresses and networks. Specify "any" to allow any update, or undefine to use the Grid::DNS level allow_update list.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Getting allow_update
$memberdns->allow_update();
#Modifying allow_update
$memberdns->allow_update(["any"]);

#Creating TSIG Key object.
my $update_key1 = $session->gen_tsig_key();
my $up_key1 = Infoblox::DNS::TSIGKey->new(
  name => "update_key",
  key  => $update_key1
  );
#Modifying allow_update
$memberdns->allow_update([ $up_key1, "10.0.0.10", "20.0.0.0/24","2008::129b","!"."30.0.0.30",
"!"."40.10.0.0/24","!"."2004::76bc/64"]);

#Use Grid::DNS level allow_update list
$memberdns->allow_update(undef);
$memberdns->update_keys(undef);
#Use Infoblox::Grid::NamedACL
$memberdns->allow_update($nacl);
```

attack_mitigation( )

Use this method to set or retrieve DNS attack mitigation settings for the member.

Setting this method to a defined value implicitly sets the override_attack_mitigation method to "true". Setting the parameter to undefined causes the appliance to use the Grid DNS default and automatically resets the override_attack_mitigation method to "false".

Note that when attack_mitigation is set to a defined value and override_attack_mitigation is set to "false", the last operation takes

precedence. Thus the sequence $object->attack_mitigation($attack_mitigation); $object->override_attack_mitigation("false"); will set override_attack_mitigation to "false", and the sequence $object->override_attack_mitigation("false"); $object->attack_mitigation($attack_mitigation); will result in override_attack_mitigation="true".

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

The valid value is an Infoblox::Grid::DNS::AttackMitigation object.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get attack_mitigation value
my $attack_mitigation = $memberdns->attack_mitigation();
#Modify attack_mitigation
$memberdns->attack_mitigation($attack_mitigation);
```

auto_blackhole( )

Use this method to set or retrieve the auto blackhole settings for the member.

Setting this method to a defined value implicitly sets the override_auto_blackhole method to "true". Setting the parameter to undefined causes the appliance to use the Grid DNS default and automatically resets the override_auto_blackhole method to "false".

Note that when auto_blackhole is set to a defined value and override_auto_blackhole is set to "false", the last operation takes precedence. Thus the sequence $object->auto_blackhole($auto_blackhole); $object->override_auto_blackhole("false"); will set override_auto_blackhole to "false", and the sequence $object->override_auto_blackhole("false"); $object->auto_blackhole($auto_blackhole); will result in override_auto_blackhole="true".

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

The valid value is an Infoblox::Grid::DNS::AutoBlackHole object.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get auto_blackhole value
my $auto_blackhole = $memberdns->auto_blackhole();
#Modify auto_blackhole
$memberdns->auto_blackhole($auto_blackhole);
```

auto_create_a_and_ptr_for_lan2( )

Use this method to enable/disable the auto-generation of A and PTR records for the LAN2 IP address, if DNS service is enabled on LAN2.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

Specify "true" to enable the auto-generation of A and PTR records for the LAN2 IP address or "false" to disable it. Default is "true".

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Getting auto_create_a_and_ptr_for_lan2
my $auto_create_a_and_ptr_for_lan2 = $memberdns->auto_create_a_and_ptr_for_lan2( );
#Modifying auto_create_a_and_ptr_for_lan2
$memberdns->auto_create_a_and_ptr_for_lan2("true");
```

auto_create_a_and_ipv6ptr_for_lan2( )

Use this method to enable/disable the auto-generation of AAAA and IPv6 PTR records for the LAN2 IP address, if DNS service is enabled on LAN2.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

Specify "true" to enable the auto-generation of AAAA and IPv6 PTR records for the LAN2 IP address or "false" to disable it. Default is "true".

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Getting auto_create_a_and_ipv6ptr_for_lan2
my $auto_create_a_and_ipv6ptr_for_lan2 = $memberdns->auto_create_a_and_ipv6ptr_for_lan2( );
#Modifying auto_create_a_and_ipv6ptr_for_lan2
$memberdns->auto_create_a_and_ipv6ptr_for_lan2("true");
```

auto_sort_views( )

Use this method to enable/disable a grid member to automatically sort DNS views. The order of the DNS views determines the order in which the appliance checks the match lists.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

Specify "true" to enable the automatic sorting of DNS views or "false" to disable it. Default is "false"

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Getting auto_sort_views
$memberdns->auto_sort_views( );
#Modifying auto_sort_views
$memberdns->auto_sort_views("true");
```

blackhole_list( )

Use this method to set or retrieve the list of IPv4 or IPv6 addresses and networks from which DNS queries are blocked. This setting overrides the grid blackhole_list.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

A valid value is an Infoblox::Grid::NamedACL object or an array reference that contains IPv4/IPv6 addresses and networks. To allow queries, add '!' as the prefix to IP addresses and networks. Specify "any" to block queries or undefine the usage of Grid::DNS level blackhole_list. Note: You should also undefine the enable_blackhole list to use Grid::DNS level settings.

Returns

When you specify a parameter and the modification succeeds, the method returns "true". When the operation fails, the method returns "false".

When you do not specify a parameter, the method returns the attribute value.

Example

```
    #Getting blackhole_list
    my $blackhole_list = $memberdns->blackhole_list( );
    #Modifying blackhole_list
    $memberdns->blackhole_list(["any"]);
    #Modifying blackhole_list
    $memberdns->blackhole_list(["10.0.0.10",
"20.0.0.0/24","2010::256/64","!"."2011::23cb/64","!"."30.0.0.30", "!"."40.10.0.0/24"]);
    #Use Grid::DNS level blackhole_list
    $memberdns->blackhole_list(undef);
    $memberdns->enable_blackhole(undef);
```

```
#Use Infoblox::Grid::NamedACL
$memberdns->blackhole_list($nacl);
```

blacklist_action( )

Use this method to specify the action to perform when a domain name matches the pattern defined in a rule that is specified by the blacklist_ruleset method.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

Valid values are "REDIRECT" or "REFUSE". The default value is "REFUSE".

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

Examples

```
#Get blacklist_action
my $blacklist_action = $memberdns->blacklist_action();
#Modify blacklist_action
$memberdns->blacklist_action("REDIRECT");
```

blacklist_log_query( )

Use this method to set or retrieve the flag that indicates whether blacklist redirection queries are logged.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

Specify "true" to enable logging, or "false" to disable it. The default value is "false".

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get blacklist_log_query
my $blacklist_log_query = $memberdns->nxdomain_log_query( );
#Modify blacklist_log_query
$memberdns->blacklist_log_query("true");
```

blacklist_redirect_addresses( )

Use this method to specify the IP addresses the appliance includes in the response it sends in place of a blacklisted IP address.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

The valid value is an array of IPv4 addresses. The default value is undefined.

Setting this parameter to ["10.0.0.10", "10.0.0.20"] or [], implicitly sets the override_blacklist_redirect attribute to "true". Setting this parameter to undefined causes the appliance to use the grid-level blacklist redirection settings and automatically sets the override_blacklist_redirect attribute to "false" if "enable_blacklist_redirect" is also undefined. But if "enable_blacklist_redirect" contains a value, then the override_blacklist_redirect attribute is set to "true".

Note that when blacklist_redirect_addresses contains a valid array reference and override_blacklist_redirect is set to "false", the last operation takes precedence. Thus the sequence $memberdns->blacklist_redirect_addresses(["10.0.0.10", "10.0.0.20"]); $memberdns->override_blacklist_redirect("false"); sets override_blacklist_redirect to "false", and the sequence $memberdns->override_blacklist_redirect("false"); $memberdns->blacklist_redirect_addresses(["10.0.0.10", "10.0.0.20"]); sets override_blacklist_redirect to "true".

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get blacklist_redirect_addresses
```

```
my $blacklist_redirect_addresses = $memberdns->blacklist_redirect_addresses( );
#Modify blacklist_redirect_addresses
$memberdns->blacklist_redirect_addresses(["10.0.0.10", "10.0.0.20"]);
```

blacklist_redirect_ttl( )

Use this method to set or retrieve the TTL value of the synthetic DNS responses that result from blacklist redirection.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

A 32-bit integer, from 0 to 4294967295, that represents the TTL in seconds. The default value is 60.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get blacklist_redirect_ttl
my $blacklist_redirect_ttl = $memberdns->blacklist_redirect_ttl( );
#Modify blacklist_redirect_ttl
$memberdns->blacklist_redirect_ttl(60);
```

blacklist_rulesets( )

Use this method to specify or retrieve Infoblox::DNS::Ruleset object names assigned at the grid level for blacklist redirection.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

The valid value is an array reference that contains the names of the Infoblox::DNS::Ruleset objects assigned to the grid. The names must be in string format.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get blacklist_rulesets
my $blacklist_rulesets = $memberdns->blacklist_rulesets();
#Modify blacklist_rulesets
$memberdns->blacklist_rulesets(["ruleset1", "ruleset2"]);
```

bind_check_names_policy( )

Use this method to set or retrieve the BIND check names policy, which indicates the action the appliance takes when it encounters host names that do not comply with the Strict Hostname Checking policy. This method applies only if the host name restriction policy is set to "Strict Hostname Checking".

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

The allowed values of the parameter are "fail", "warn" or undefined. This indicates the desired BIND behavior in the case of illegal hostnames. The default value is undefined, which means the appliance ignores illegal hostnames.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get bind_check_names_policy value
my $names_policy = $memberdns->bind_check_names_policy();
#Modify bind_check_names_policy
$memberdns->bind_check_names_policy("warn");
```

bind_hostname_directive( )

Use this method to set or retrieve the value of BIND 'hostname' directive.

Setting this method to a defined value implicitly sets the override_bind_hostname_directive method to "true". Setting the parameter to undefined causes the appliance to use the Grid DNS default and automatically resets the override_bind_hostname_directive method to "false".

Note that when bind_hostname_directive is set to a defined value and override_bind_hostname_directive is set to "false", the last operation takes precedence. Thus the sequence $object->bind_hostname_directive("HOSTNAME"); $object->override_bind_hostname_directive("false"); will set override_bind_hostname_directive to "false", and the sequence $object->override_bind_hostname_directive("false"); $object->bind_hostname_directive("true"); will result in override_bind_hostname_directive="true".

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

The valid values are "NONE", "HOSTNAME" and "USER_DEFINED". The default value is "NONE".

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get bind_hostname_directive
my $bind_hostname_directive = $memberdns->bind_hostname_directive();
#Modify bind_hostname_directive
$memberdns->bind_hostname_directive("HOSTNAME");
```

bind_hostname_directive_fqdn( )

Use this method to set or retrieve the value of BIND user-defined hostname for 'hostname' directive.

Note that to enable user-defined 'hostname' the bind_hostname_directive should be set to "USER_DEFINED".

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

The valid value is Fully-Qualified Domain Name (FQDN) in a string format. The default value is undefined.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get bind_hostname_directive_fqdn
my $bind_hostname_directive_fqdn = $memberdns->bind_hostname_directive_fqdn();
#Modify bind_hostname_directive_fqdn
$memberdns->bind_hostname_directive_fqdn("foo-bar.com");
```

capture_dns_queries_on_all_domains( )

Use this method to set or retrieve the flag that indicates whether the capturing of DNS queries for all domains is enabled or disabled.

Setting this method to a defined value implicitly sets the override_capture_dns_queries_on_all_domains method to "true". Setting the parameter to undefined causes the appliance to use the Grid DNS default and automatically resets the override_capture_dns_queries_on_all_domains method to "false".

Note that when capture_dns_queries_on_all_domains is set to a defined value and override_capture_dns_queries_on_all_domains is set to "false", the last operation takes precedence. Thus the sequence $object->capture_dns_queries_on_all_domains("true"); $object->override_capture_dns_queries_on_all_domains("false"); will set override_capture_dns_queries_on_all_domains to "false", and the sequence $object->override_capture_dns_queries_on_all_domains("false"); $object->capture_dns_queries_on_all_domains("true"); will result in override_capture_dns_queries_on_all_domains="true".

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

Specify 'true' to enable capturing of DNS queries for all domains or 'false' to disable it. The default value is 'false'.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get capture_dns_queries_on_all_domains value
my $capture_dns_queries_on_all_domains = $object->capture_dns_queries_on_all_domains();

#Modify capture_dns_queries_on_all_domains value
$object->capture_dns_queries_on_all_domains('true');
```

copy_xfer_to_notify( )

Use this method to enable/disable copying of the allowed IPs from zone transfer list into also-notify statement in named.conf.

Parameter

Set the parameter to "true" to enable copying of the allowed IPs from zone transfer list into also-notify statement in named.conf. Set the parameter to "false" to disable it.

The default value of the parameter is "false".

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
# Get attribute value
my $value = $memberdns->copy_xfer_to_notify();
# Modify attribute value
$memberdns->copy_xfer_to_notify('true');
```

custom_root_name_servers( )

Use this method to set or retrieve the list of customized root nameserver(s). You can either select and use Internet root name servers or specify custom root name servers by providing a host name and IP address to which the Infoblox appliance can send queries. Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

Valid value is an array reference that contains Infoblox::DNS::RootNameServer object(s).

Returns

If you specify a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get the root nameserver list
my $rootNS_list = $memberdns->custom_root_name_servers();
#Modify the root nameserver list
#Creating RootNameServer objects.
my $rootns1 = Infoblox::DNS::RootNameServer->new(
    ipv4addr  => "4.4.4.4",
    host_name => "rns1.test.com",
    );

my $rootns2 = Infoblox::DNS::RootNameServer->new(
    ipv4addr  => "5.5.5.5",
    host_name => "rns2.test.com",
    );

$memberdns->custom_root_name_servers([$rootns1, $rootns2]);
```

disable_edns( )

Use this method to disable or enable EDNS0 support for queries that require recursive resolution on Grid members.

Setting this method to a defined value implicitly sets the override_disable_edns method to "true". Setting the parameter to undefined causes the appliance to use the grid DNS default and automatically resets the override_disable_edns method to "false".

Note that when disable_edns is set to a defined value and override_disable_edns is set to "false", the last operation takes precedence.

Thus the sequence $object->disable_edns("true"); $object->override_disable_edns("false"); will set override_disable_edns to "false", and the sequence $object->override_disable_edns("false"); $object->disable_edns("true"); will result in override_disable_edns="true".

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

Specify "true" to disable EDNS0 or "false" to enable it. The default value is "false".

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get disable_edns flag
my $disable_edns = $memberdns->disable_edns();
#Modify disable_edns flag
$memberdns->disable_edns("true");
```

dns_cache_acceleration_ttl( )

Use this method to set or retrieve the minimum TTL value, in seconds, that a DNS record must have in order for it to be cached by the DNS Cache Acceleration service.

Setting this method to a defined value implicitly sets the override_dns_cache_acceleration_ttl method to "true". Setting the parameter to undefined causes the appliance to use the grid DNS default and automatically resets the override_dns_cache_acceleration_ttl method to "false".

Note that when dns_cache_acceleration_ttl is set to a defined value and override_dns_cache_acceleration_ttl is set to "false", the last operation takes precedence. Thus the sequence $object->dns_cache_acceleration_ttl(60); $object->override_dns_cache_acceleration_ttl("false"); will set override_dns_cache_acceleration_ttl to "false", and the sequence $object->override_dns_cache_acceleration_ttl("false"); $object->dns_cache_acceleration_ttl(30); will result in override_dns_cache_acceleration_ttl="true".

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

An integer from 1 to 65000 that represents the TTL in seconds. The default value is 1.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get dns_cache_acceleration_ttl
my $dns_cache_acceleration_ttl = $member_dns->dns_cache_acceleration_ttl( );
#Modify dns_cache_acceleration_ttl
$member_dns->dns_cache_acceleration_ttl(60);
```

dns_cache_acceleration_status( )

Use this method to retrieve the DNS cache acceleration status.

Parameter

None.

Returns

The method returns the attribute value, which is one of the following: "Failed", "Inactive", "Offline", "Unknown", "Warning", or "Working".

Example

```
#Get dns_cache_acceleration_status
my $dns_cache_acceleration_status = $member_dns->dns_cache_acceleration_status( );
```

dns_health_check_anycast_control( )

Use this method to set or retrieve the flag that indicates whether the anycast failure (BFD session down) is enabled on Member failure or not.

Setting this parameter to defined value, implicitly sets the override_dns_health_check attribute to "true". Setting this parameter to undefined causes the appliance to use the Grid-level DNS health check settings and automatically sets the override_dns_health_check attribute to "false" if following fields are undefined:

```
dns_health_check_domain_list
dns_health_check_recursion_flag
dns_health_check_interval
dns_health_check_retries
dns_health_check_timeout
enable_dns_health_check
```

But if any of the fields above contains a value, then the override_dns_health_check attribute is set to "true".

Note that when dns_health_check_anycast_control contains a value and override_dns_health_check is set to "false", the last operation takes precedence. Thus the sequence $zone->dns_health_check_anycast_control("true"); $zone->override_dns_health_check("false"); sets override_dns_health_check to "false", and the sequence $zone->override_dns_health_check("false"); $zone->dns_health_check_anycast_control("true"); sets override_dns_health_check to "true".

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

Specify 'true' to enable anycast failure on Member failure, and 'false' to disable it.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get dns_health_check_anycast_control value
my $value = $object->dns_health_check_anycast_control();

#Modify dns_health_check_anycast_control value
$object->dns_health_check_anycast_control('true');
```

dns_health_check_domain_list( )

Use this method to set or retrieve the list of domain names for the DNS health check.

Setting this parameter to defined value, implicitly sets the override_dns_health_check attribute to "true". Setting this parameter to undefined causes the appliance to use the Grid-level DNS health check settings and automatically sets the override_dns_health_check attribute to "false" if following fields are undefined:

```
dns_health_check_recursion_flag
dns_health_check_anycast_control
dns_health_check_interval
dns_health_check_retries
dns_health_check_timeout
enable_dns_health_check
```

But if any of the fields above contains a value, then the override_dns_health_check attribute is set to "true".

Note that when dns_health_check_anycast_control contains a value and override_dns_health_check is set to "false", the last operation takes precedence. Thus the sequence $zone->dns_health_check_domain_list(['domain1.com', 'domain2.com']); $zone->override_dns_health_check("false"); sets override_dns_health_check to "false", and the sequence $zone->override_dns_health_check("false"); $zone->dns_health_check_domain_list(['domain1.com', 'domain2.com']); sets override_dns_health_check to "true".

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

The valid value is an array reference that contains desired domain names in string format.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get dns_health_check_domain_list value
my $value = $object->dns_health_check_domain_list();

#Modify dns_health_check_domain_list value
$object->dns_health_check_domain_list(['foo.com', 'bar.net']);
```

dns_health_check_interval( )

Use this method to set or retrieve the time interval (in seconds) for DNS health check.

Setting this parameter to defined value, implicitly sets the override_dns_health_check attribute to "true". Setting this parameter to undefined causes the appliance to use the Grid-level DNS health check settings and automatically sets the override_dns_health_check attribute to "false" if following fields are undefined:

```
dns_health_check_domain_list
dns_health_check_recursion_flag
dns_health_check_anycast_control
dns_health_check_retries
dns_health_check_timeout
enable_dns_health_check
```

But if any of the fields above contains a value, then the override_dns_health_check attribute is set to "true".

Note that when dns_health_check_interval contains a value and override_dns_health_check is set to "false", the last operation takes precedence. Thus the sequence $zone->dns_health_check_interval("500"); $zone->override_dns_health_check("false"); sets override_dns_health_check to "false", and the sequence $zone->override_dns_health_check("false"); $zone->dns_health_check_interval("500"); sets override_dns_health_check to "true".

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

The valid value is an unsigned integer between 10 and 21600 that represents number of seconds between two health checks.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get dns_health_check_interval value
my $value = $object->dns_health_check_interval();

#Modify dns_health_check_interval value
$object->dns_health_check_interval(2000);
```

dns_health_check_recursion_flag( )

Use this method to set or retrieve the flag that indicates whether the recursive DNS health check is enabled or not.

Setting this parameter to defined value, implicitly sets the override_dns_health_check attribute to "true". Setting this parameter to undefined causes the appliance to use the Grid-level DNS health check settings and automatically sets the override_dns_health_check attribute to "false" if following fields are undefined:

```
dns_health_check_domain_list
dns_health_check_anycast_control
dns_health_check_interval
dns_health_check_retries
dns_health_check_timeout
enable_dns_health_check
```

But if any of the fields above contains a value, then the override_dns_health_check attribute is set to "true".

Note that when dns_health_check_recursion_flag contains a value and override_dns_health_check is set to "false", the last operation takes precedence. Thus the sequence $zone->dns_health_check_recursion_flag("true"); $zone->override_dns_health_check("false"); sets override_dns_health_check to "false", and the sequence $zone->override_dns_health_check("false"); $zone->dns_health_check_recursion_flag("true"); sets override_dns_health_check to "true".

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

Specify 'true' to enable recursive DNS health check queries, and 'false' to disable it.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get dns_health_check_recursion_flag value
my $value = $object->dns_health_check_recursion_flag();
```

```
#Modify dns_health_check_recursion_flag value
$object->dns_health_check_recursion_flag('true');
```

dns_health_check_retries( )

Use this method to set or retrieve the number of DNS health check retries.

Setting this parameter to defined value, implicitly sets the override_dns_health_check attribute to "true". Setting this parameter to undefined causes the appliance to use the Grid-level DNS health check settings and automatically sets the override_dns_health_check attribute to "false" if following fields are undefined:

```
dns_health_check_domain_list
dns_health_check_recursion_flag
dns_health_check_anycast_control
dns_health_check_interval
dns_health_check_timeout
enable_dns_health_check
```

But if any of the fields above contains a value, then the override_dns_health_check attribute is set to "true".

Note that when dns_health_check_retries contains a value and override_dns_health_check is set to "false", the last operation takes precedence. Thus the sequence $zone->dns_health_check_retries("4"); $zone->override_dns_health_check("false"); sets override_dns_health_check to "false", and the sequence $zone->override_dns_health_check("false"); $zone->dns_health_check_retries("4"); sets override_dns_health_check to "true".

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

The valid value is an unsigned integer between 1 and 10.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get dns_health_check_retries value
my $value = $object->dns_health_check_retries();

#Modify dns_health_check_retries value
$object->dns_health_check_retries(2);
```

dns_health_check_timeout( )

Use this method to set or retrieve the DNS health check timeout interval (in seconds).

Setting this parameter to defined value, implicitly sets the override_dns_health_check attribute to "true". Setting this parameter to undefined causes the appliance to use the Grid-level DNS health check settings and automatically sets the override_dns_health_check attribute to "false" if following fields are undefined:

```
dns_health_check_domain_list
dns_health_check_recursion_flag
dns_health_check_anycast_control
dns_health_check_interval
dns_health_check_retries
enable_dns_health_check
```

But if any of the fields above contains a value, then the override_dns_health_check attribute is set to "true".

Note that when dns_health_check_timeout contains a value and override_dns_health_check is set to "false", the last operation takes precedence. Thus the sequence $zone->dns_health_check_timeout("4"); $zone->override_dns_health_check("false"); sets override_dns_health_check to "false", and the sequence $zone->override_dns_health_check("false"); $zone->dns_health_check_timeout("4"); sets override_dns_health_check to "true".

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

The valid value is an unsigned integer between 1 and 10.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get dns_health_check_timeout value
my $value = $object->dns_health_check_timeout();

#Modify dns_health_check_timeout value
$object->dns_health_check_timeout(10);
```

dns_ip_list( )

Use this method to add or retrieve a list of additional IP addresses on which DNS is enabled for a grid member.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

Valid value is an Infoblox::Grid::Member::DNS::IP object.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get dns_ip_list value
my $value = $object->dns_ip_list();

#Modify dns_ip_list

#Construct Infoblox::Grid::Member::DNS::IP object

my $ip = Infoblox::Grid::Member::DNS::IP->new(
    member_virtual_ip => '10.0.0.10',
);

$object->dns_ip_list([$ip]);
```

dns64_groups( )

Use this method to set or retrieve the list of DNS64 synthesis groups associated with this member.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Setting this method to a defined value implicitly sets the override_dns64 method to "true". Setting the parameter to undefined causes the appliance to use the grid DNS default and automatically resets the override_dns64 method to "false".

Note that when dns64_groups is set to a defined value and override_dns64_groups is set to "false", the last operation takes precedence. Thus the sequence $object->dns64_groups([$group1]); $object->override_dns64("false"); will set override_dns64 to "false", and the sequence $object->override_dns64("false"); $object->dns64_groups([$group1]); will result in override_dns64="true".

Parameter

Valid value is an array reference that contains Infoblox::Grid::DNS::DNS64Group objects.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get the list of DNS64 synthesis groups
my $dns64_groups_list = $memberdns->dns64_groups();

#Modify it
my $group = $session->get(
                          object => "Infoblox::Grid::DNS::DNS64Group"),
                          name => "groupname",
                          prefix => "groupprefix",
                        );
$memberdns->dns64_groups([$group]);
```

dnssec_enabled( )

Use this method to enable/disable the DNSSEC feature.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

Set the parameter to "true" to enable the DNSSEC feature. Set the parameter to "false" to disable the feature.

Setting this parameter to "true" or "false" implicitly sets the override_dnssec attribute to "true". Setting the parameter to undefined causes the appliance to use the grid-level DNSSEC settings and automatically sets the override_dnssec attribute to "false".

Note that in the case when dnssec_enabled is set to "true"/"false" and override_dnssec is set to "false" simultaneously, the last operation takes precedence. Thus the sequence $grid_member->dnssec_enabled("false"); $grid_member->override_dnssec("false"); will set override_dnssec to "false", and the sequence $grid_member->override_dnssec("false"); $grid_member->dnssec_enabled("false"); will result in override_dnssec="true".

The default value of the parameter is "false".

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get dnssec_enabled
my $dnssec_enabled = $memberdns->dnssec_enabled();
#Modify dnssec_enabled
$memberdns->dnssec_enabled("true");
#Set the value to undefined to inherit grid level settings.
$memberdns->dnssec_enabled(undef);
```

dnssec_expired_signatures_enabled( )

Use this method to control whether the DNS member accepts expired signatures.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

Set the parameter to "true" to accept expired signatures. Set it to "false" to reject expired signatures.

The default value of the parameter is "false".

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get dnssec_expired_signatures_enabled
my $dnssec_expired_signatures_enabled = $memberdns->dnssec_expired_signatures_enabled();
#Modify dnssec_expired_signatures_enabled
$memberdns->dnssec_expired_signatures_enabled("true");
```

dnssec_validation_enabled( )

Use this method to enable or disable DNS security validation.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

Set the parameter to "true" to enable the validation. Set it to "false" to disable the validation.

The default value of the parameter is "true".

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get dnssec_validation_enabled
my $dnssec_validation_enabled = $memberdns->dnssec_validation_enabled();
#Modify dnssec_validation_enabled
$memberdns->dnssec_validation_enabled("false");
```

dnssec_blacklist_enabled()

Use this method to enable and disable blacklist rules for DNSSEC-enabled clients.

Parameter

Set the parameter to "true" to apply blacklist rules for DNSSEC-enabled clients. Set to "false" to disable them.

The default value of the parameter is "false".

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get dnssec_blacklist_enabled
my $dnssec_blacklist_enabled = $memberdns->dnssec_blacklist_enabled();
#Modify dnssec_blacklist_enabled
$memberdns->dnssec_blacklist_enabled("true");
```

dnssec_negative_trust_anchors( )

Use this method to set or retrieve the list of zones for which the appliance does not perform DNSSEC validation.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

The valid value is an array of strings that contains the lsit of zones for which the appliance does not perform DNSSEC validation.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get dnssec_negative_trust_anchors value
my $value = $member_dns->dnssec_negative_trust_anchors();
#Modify dnssec_negative_trust_anchors value
$member_dns->dnssec_negative_trust_anchors($zones);
```

dnssec_dns64_enabled()

Use this method to enable and disable DNS64 groups for DNSSEC-enabled clients.

Parameter

Set the parameter to "true" to apply DNS64 groups for DNSSEC-enabled clients. Set to "false" to disable them.

The default value of the parameter is "false".

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get dnssec_dns64_enabled
my $dnssec_dns64_enabled = $member_dns->dnssec_dns64_enabled();
#Modify dnssec_dns64_enabled
$member_dns->dnssec_dns64_enabled("true");
```

dnssec_nxdomain_enabled()

Use this method to enable and disable NXDOMAIN rules for DNSSEC-enabled clients.

Parameter

Set the parameter to "true" to apply NXDOMAIN rules for DNSSEC-enabled clients. Set to "false" to disable them.

The default value of the parameter is "false".

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get dnssec_nxdomain_enabled
my $dnssec_nxdomain_enabled = $memberdns->dnssec_nxdomain_enabled();
#Modify dnssec_nxdomain_enabled
$memberdns->dnssec_nxdomain_enabled("true");
```

dnssec_rpz_enabled()

Use this method to enable and disable RPZ policies for DNSSEC-enabled clients.

Parameter

Set the parameter to "true" to apply RPZ policies for DNSSEC-enabled clients. Set to "false" to disable them.

The default value of the parameter is "false".

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get dnssec_rpz_enabled
my $dnssec_rpz_enabled = $memberdns->dnssec_rpz_enabled();
#Modify dnssec_rpz_enabled
$memberdns->dnssec_rpz_enabled("true");
```

dnssec_trusted_keys( )

Use this method to specify the list of trusted keys for the DNSSEC feature.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

Array reference that contains a list of Infoblox::DNS::DnssecTrustedKey objects.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns a reference to a list of the trusted keys.

Example

```
#Get dnssec_trusted_keys
my $dnssec_trusted_keys = $memberdns->dnssec_trusted_keys();
#Modify dnssec_trusted_keys
my $key1=Infoblox::DNS::DnssecTrustedKey->new(
   fqdn => "myzone.com",
   algorithm => "RSAMD5",
   key       => "c2ltcGxlIGtleQo="
   );
$memberdns->dnssec_trusted_keys([$key1]);
```

dns_notify_transfer_source_interface( )

Use this method to specify which IP address is used as the source for DDNS notify and transfer operations.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

Valid values are "VIP", "MGMT", "LAN2", "Any" or an IPV4 address. Default is "VIP".

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Getting dns_source_interface
my $dns_source_interface = $memberdns->dns_source_interface( );
#Modifying dns_source_interface
$memberdns->dns_source_interface("VIP");
```

dns_query_capture_file_time_limit( )

Use this method to set or retrieve the DNS query capture file time (in minutes) limit.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

The valid value is unsigned integer between 1 and 10. The default value is 10.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get dns_query_capture_file_time_limit value
my $dns_query_capture_file_time_limit = $object->dns_query_capture_file_time_limit();

#Modify dns_query_capture_file_time_limit value
$object->dns_query_capture_file_time_limit('5');
```

dns_query_source_interface( )

Use this method to specify which IP address is used as the source for DDNS query operations.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

Valid values are "VIP", "MGMT", "LAN2", "Any" or an IPV4 address. Default is "VIP".

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Getting dns_source_interface
my $dns_source_interface = $memberdns->dns_source_interface( );
#Modifying dns_source_interface
$memberdns->dns_source_interface("VIP");
```

dns_view_address_settings( )

Use this method to set or retrieve the list of address settings for Grid member views.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

The valid value is an array of Infoblox::Grid::Member::DNS::ViewAddressSetting objects.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get dns_view_address_settings value
my $value = $object->dns_view_address_settings();

#Construct view address setting
my $view_address_setting = Infoblox::Grid::Member::DNS::ViewAddressSetting->new('view_name' =>
'view1');
```

```
#Modify dns_view_address_settings value
$object->dns_view_address_settings([$view_address_setting]);
```

domains_to_capture_dns_queries( )

Use this method to set or retrieve the list of domains to capture DNS queries.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

The valid value is an array reference that contains desired domain names in a string format.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get domains_to_capture_dns_queries value
my $domains_to_capture_dns_queries = $object->domains_to_capture_dns_queries();

#Modify domains_to_capture_dns_queries value
$object->domains_to_capture_dns_queries(['domain1.com', 'domain2.com']);
```

dtc_edns_prefer_client_subnet( )

Use this method to set or retrieve the flag that indicates whether the use of the edns-client-subnet option for DTC is enabled or disabled.

Setting this method to a defined value implicitly sets the override_dtc_edns_prefer_client_subnet method to "true". Setting the parameter to undefined causes the appliance to use the Grid DNS default and automatically resets the override_dtc_edns_prefer_client_subnet method to "false".

Note that when dtc_edns_prefer_client_subnet is set to a defined value and override_dtc_edns_prefer_client_subnet is set to "false", the last operation takes precedence. Thus the sequence $object->dtc_edns_prefer_client_subnet("true"); $object->override_dtc_edns_prefer_client_subnet("false"); will set override_dtc_edns_prefer_client_subnet to "false", and the sequence $object->override_dtc_edns_prefer_client_subnet("false"); $object->dtc_edns_prefer_client_subnet("true"); will result in override_dtc_edns_prefer_client_subnet="true".

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

Specify 'true' to enable use of the edns-client-subnet option for DTC or 'false' to disable it.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get dtc_edns_prefer_client_subnet value
my $dtc_edns_prefer_client_subnet = $object->dtc_edns_prefer_client_subnet();

#Modify dtc_edns_prefer_client_subnet value
$object->dtc_edns_prefer_client_subnet('true');
```

dtc_health_source_interface( )

Use this method to set or retrieve the source address for the health checks.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

The valid value is one of the following: VIP, MGMT, LAN2, ANY or IPv4 / IPv6 address.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Getting dtc_health_source_interface
my $dtc_source_interface = $memberdns->dtc_health_source_interface();
#Modifying dtc_health_source_interface
$memberdns->dtc_health_source_interface('VIP');
```

enable_blackhole( )

Use this method to enable or disable the blocking of DNS queries. This setting overrides the grid enable_blackhole settings.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

Specify "true" to block DNS queries or "false" to disable blocking DNS queries. The default value is "false".

Returns

When you specify a parameter and the modification succeeds, the method returns "true". When the operation fails, the method returns "false".

When you do not specify a parameter, the method returns the attribute value.

Example

```
#Get enable_blackhole
my $enable_blackhole = $object->enable_blackhole( );
#Modify enable_blackhole
$memberdns->enable_blackhole("true");
```

enable_blacklist( )

Use this method to enable or disable a blacklist on the grid member.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

Specify "true" to set the enable_blacklist flag or "false" to deactivate/unset it. The default is "false".

Setting this parameter to "true" or "false" implicitly sets the override_blacklist attribute to "true". Setting the parameter to undefined causes the appliance to use the grid-level blacklist settings and automatically sets the override_blacklist attribute to "false" if "blacklist_redirect_addresses" is also undefined.

Note that when enable_blacklist is set to "true"/"false" and override_blacklist is set to "false", the last operation takes precedence. Thus the sequence $memberdns->enable_blacklist("false"); $memberdns->override_blacklist("false"); sets override_blacklist to "false", and the sequence $memberdns->override_blacklist("false"); $memberdns->enable_blacklist("false"); sets override_blacklist to "true".

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Getting enable_blacklist
my $enable_blacklist = $memberdns->enable_blacklist( );
#Modifying enable_blacklist
$memberdns->enable_blacklist("true");
```

enable_capture_dns_queries( )

Use this method to set or retrieve the flag that indicates whether the capture of DNS queries is enabled or disabled.

Setting this method to a defined value implicitly sets the override_enable_capture_dns method to "true". Setting the parameter to undefined unless enable_capture_dns_responses is defined causes the appliance to use the Grid DNS default and automatically resets the override_enable_capture_dns method to "false".

Note that when enable_capture_dns_queries is set to a defined value and override_enable_capture_dns is set to "false", the last operation takes precedence. Thus the sequence $object->enable_capture_dns_queries("true"); $object->override_enable_capture_dns("false"); will set override_enable_capture_dns to "false", and the sequence $object->override_enable_capture_dns("false"); $object->enable_capture_dns_queries("true"); will result in override_enable_capture_dns="true".

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

Specify 'true' to enable the capture of DNS queries or 'false' to disable it. The default value is 'false'.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get enable_capture_dns_queries value
my $enable_capture_dns_queries = $object->enable_capture_dns_queries();

#Modify enable_capture_dns_queries value
$object->enable_capture_dns_queries('true');
```

enable_capture_dns_responses( )

Use this method to set or retrieve the flag that indicates whether the capture of DNS responses is enabled or disabled.

Setting this method to a defined value implicitly sets the override_enable_capture_dns method to "true". Setting the parameter to undefined unless enable_capture_dns_queries is defined causes the appliance to use the Grid DNS default and automatically resets the override_enable_capture_dns method to "false".

Note that when enable_capture_dns_responses is set to a defined value and override_enable_capture_dns is set to "false", the last operation takes precedence. Thus the sequence $object->enable_capture_dns_responses("true"); $object->override_enable_capture_dns("false"); will set override_enable_capture_dns to "false", and the sequence $object->override_enable_capture_dns("false"); $object->enable_capture_dns_responses("true"); will result in override_enable_capture_dns="true".

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

Specify 'true' to enable the capture of DNS responses or 'false' to disable it. The default value is 'false'.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get enable_capture_dns_responses value
my $enable_capture_dns_responses = $object->enable_capture_dns_responses();

#Modify enable_capture_dns_responses value
$object->enable_capture_dns_responses('true');
```

enable_dns( )

Use this method to enable/disable the DNS service of a member.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

Specify "true" to enable DNS service or "false" to disable it. The default value is "true".

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Getting enable_dns
$memberdns->enable_dns( );
#Modifying enable_dns
$memberdns->enable_dns("true");
```

enable_dns_cache_acceleration( )

Use this method to enable/disable the DNS Cache Acceleration service of a member.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

Specify "true" to enable the DNS Cache Acceleration service or "false" to disable it. The default value is "false".

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Getting enable_dns_cache_acceleration
$memberdns->enable_dns_cache_acceleration( );
#Modifying enable_dns_cache_acceleration
$memberdns->enable_dns_cache_acceleration("true");
```

enable_dns_health_check( )

Use this method to set or retrieve the flag that indicates whether the DNS health check is enabled or not.

Setting this parameter to defined value, implicitly sets the override_dns_health_check attribute to "true". Setting this parameter to undefined causes the appliance to use the Grid-level DNS health check settings and automatically sets the override_dns_health_check attribute to "false" if following fields are undefined:

```
dns_health_check_domain_list
dns_health_check_recursion_flag
dns_health_check_anycast_control
dns_health_check_interval
dns_health_check_retries
dns_health_check_timeout
```

But if any of the fields above contains a value, then the override_dns_health_check attribute is set to "true".

Note that when enable_dns_health_check contains a value and override_dns_health_check is set to "false", the last operation takes precedence. Thus the sequence $zone->enable_dns_health_check("true"); $zone->override_dns_health_check("false"); sets override_dns_health_check to "false", and the sequence $zone->override_dns_health_check("false"); $zone->enable_dns_health_check("true"); sets override_dns_health_check to "true".

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

Specify 'true' to enable DNS health check, and 'false' to disable it.

Returns

When you specify a parameter and the modification succeeds, the method returns "true". When the operation fails, the method returns "false".

When you do not specify a parameter, the method returns the attribute value.

Example

```
#Get enable_dns_health_check value
my $value = $object->enable_dns_health_check();

#Modify enable_dns_health_check value
$object->enable_dns_health_check('true');
```

enable_dns64( )

Use this method to enable/disable DNS64 support for this member.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Setting this method to a defined value implicitly sets the override_dns64 method to "true". Setting the parameter to undefined causes the appliance to use the grid DNS default and automatically resets the override_dns64 method to "false".

Note that when enable_dns64 is set to a defined value and override_enable_dns64 is set to "false", the last operation takes precedence. Thus the sequence $object->enable_dns64("true"); $object->override_dns64("false"); will set override_dns64 to "false", and the sequence $object->override_dns64("true"); $object->enable_dns64("testdomain"); will result in override_dns64="true".

Parameter

Specify "true" to set the enable_dns64 flag or "false" to deactivate and unset it. The default value is "false".

Returns

When you specify a parameter and the modification succeeds, the method returns "true". When the operation fails, the method

returns "false".

When you do not specify a parameter, the method returns the attribute value.

Example

```
#Get the enable_dns64 flag
my $enable_dns64 = $memberdns->enable_dns64();
#Modify the enable_dns64 flag
$memberdns->enable_dns64("true");
```

enable_excluded_domain_names( )

Use this method to set or retrieve the flag that indicates whether the excluding domain names captured DNS query and response is enabled or disabled..

Setting this method to a defined value implicitly sets the override_enable_excluded_domain_names method to "true". Setting the parameter to undefined causes the appliance to use the Grid DNS default and automatically resets the override_enable_excluded_domain_names method to "false".

Note that when enable_excluded_domain_names is set to a defined value and override_enable_excluded_domain_names is set to "false", the last operation takes precedence. Thus the sequence $object->enable_excluded_domain_names("true"); $object->override_enable_excluded_domain_names("false"); will set override_enable_excluded_domain_names to "false", and the sequence $object->override_enable_excluded_domain_names("false"); $object->enable_excluded_domain_names("true"); will result in override_enable_excluded_domain_names="true".

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

Specify 'true' to enable the excluding domain names from captured DNS query and response or 'false' to disable it. The default value is 'false'.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get enable_excluded_domain_names value
my $enable_excluded_domain_names = $object->enable_excluded_domain_names();

#Modify enable_excluded_domain_names value
$object->enable_excluded_domain_names('true');
```

enable_fixed_rrset_order_fqdns( )

Use this method to enable or disable fixed RRset order for specified FQDNs.

Setting this method to a defined value implicitly sets the override_fixed_rrset_order_fqdns method to "true".

Note that when enable_fixed_rrset_order_fqdns is set to a defined value and override_fixed_rrset_order_fqdns is set to "false", the last operation takes precedence. Thus the sequence $object->enable_fixed_rrset_order_fqdns("true"); $object->override_fixed_rrset_order_fqdns("false"); will set override_fixed_rrset_order_fqdns to "false", and the sequence $object->override_fixed_rrset_order_fqdns("false"); $object->enable_fixed_rrset_order_fqdns("true"); will result in override_fixed_rrset_order_fqdns="true".

The enable_fixed_rrset_order_fqdns attribute can be specified explicitly. It is also set implicitly when fixed_rrset_order_fqdns is set to a defined value.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

Specify "true" to enable fixed RRset order for specified FQDNs or "false" to disable it.

Returns

When you specify a parameter and the modification succeeds, the method returns "true". When the operation fails, the method returns "false".

When you do not specify a parameter, the method returns the attribute value.

Example

```
#Get enable_fixed_rrset_order_fqdns flag
```

```
my $enable_fixed_rrset_order_fqdns = $member_dns->enable_fixed_rrset_order_fqdns();
#Modify enable_fixed_rrset_order_fqdns flag
$member_dns->enable_fixed_rrset_order_fqdns("true");
```

enable_ftc( )

Use this method to set or retrieve the flag that indicates whether the Fault Tolerant Caching (FTC) feature is enabled or not.

Setting this method to a defined value implicitly sets the override_ftc method to "true". Setting the parameter to undefined if ftc_expired_record_ttl and ftc_expired_record_timeout are also undefined causes the appliance to use the Grid DNS default and automatically resets the override_ftc method to "false".

Note that when enable_ftc is set to a defined value and override_ftc is set to "false", the last operation takes precedence. Thus the sequence $object->enable_ftc("true"); $object->override_ftc("false"); will set override_ftc to "false", and the sequence $object->override_ftc("false"); $object->enable_ftc("true"); will result in override_ftc="true".

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

Specify 'true' to enable FTC or 'false' to disable it. The default value is 'false'.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get enable_ftc
my $enable_ftc = $memberdns->enable_ftc();
#Modify enable_ftc
$memberdns->enable_ftc('true');
```

enable_gss_tsig( )

Use this method to enable/disable the appliance to receive GSS-TSIG authenticated updates from DHCP clients. You must import the keytab file before you enable this feature. See Infoblox::Session->import_data().

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

Specify "true" to enable the the appliance to receive GSS-TSIG authenticated updates or "false" to disable it. The default value is "false".

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Getting enable_gss_tsig
$my $enable_gss_tsig = $memberdns->enable_gss_tsig( );
#Modifying enable_gss_tsig
$session->import_data(
     type        => 'keytab',
     path        => '/infoblox/tests/datasets/gss-tsig-gs.keytab',
     service     => 'dns',
     virtual_ip  => '192.168.1.2');
$memberdns->enable_gss_tsig("true");
```

enable_query_rewrite( )

Use this method to enable or disable DNS query rewrite for this member.

Setting this method to a defined value implicitly sets the override_query_rewrite method to "true". Setting the parameter to undefined causes the appliance to use the grid-level enable_query_rewrite value and automatically resets the override_query_rewrite method to "false".

Note that when enable_query_rewrite is set to a defined value and override_query_rewrite is set to "false", the last operation takes precedence. Thus the sequence $object->enable_query_rewrite("true"); $object->override_query_rewrite("false"); will set override_query_rewrite to "false", and the sequence $object->override_query_rewrite("true"); $object->enable_query_rewrite("testdomain"); will result in override_query_rewrite="true".

Parameter
Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get enable_query_rewrite
$memberdns->enable_query_rewrite( );
#Modify enable_query_rewrite
$memberdns->enable_query_rewrite("true");
```

excluded_domain_names( )

Use this method to set or retrieve the list of domains that are excluded from DNS response and query capturing.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

The valid value is an array reference that contains desired domain names in a string format.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get excluded_domain_names value
my $excluded_domain_names = $object->excluded_domain_names();

#Modify excluded_domain_names value
$object->excluded_domain_names(['domain1.com', 'domain2.com']);
```

extattrs( )

Use this method to set or retrieve the extensible attributes associated with a Grid Member DNS object.

Parameter

Valid value is a hash reference containing the names of extensible attributes and their associated values (Infoblox::Grid::Extattr objects).

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get extattrs
my $ref_extattrs = $memberdns->extattrs();
#Modify extattrs
$memberdns->extattrs({ 'Site' => $extattr1, 'Administrator' => $extattr2 });
```

extensible_attributes( )

Use this method to set or retrieve the extensible attributes associated with a Grid Member DNS object.

This method is read-only. Use the Infoblox::Grid::Member object to set or modify the grid member extensible attributes.

Returns

The method returns the attribute value. Attempting to modify the extensible attributes produces an error.

Example

```
#Get extensible attributes
my $ref_extensible_attributes = $memberdns->extensible_attributes();
```

facility( )

Use this method to specify a syslog facility. This is the location on the syslog server to which you want to sort the DNS logging messages.

This setting overrides the grid logging facility settings.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

Values are: "daemon", "local0", "local1", "local2", "local3", "local4", "local5", "local6", and "local7".

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get facility
$memberdns->facility( );
#Modify facility
$memberdns->facility("local0");
```

file_transfer_setting( )

Use this method to set or retrieve the DNS capture file transfer settings.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

The valid value is an Infoblox::Grid::DNS::FileTransferSetting object.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get file_transfer_setting value
my $file_transfer_setting = $object->file_transfer_setting();

#Modify file_transfer_setting value
$object->file_transfer_setting($file_transfer);
```

filter_aaaa( )

Use this method to set or retrieve the type of AAAA filtering for this member DNS object.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Specifying values in this parameter, such as "YES", implicitly sets the override_filter_aaaa attribute to "true". Setting this parameter and the "filter_aaaa" attribute to undefined causes the appliance to use the grid-level filter_aaaa settings and automatically sets the override_filter_aaaa attribute to "false". But if the "filter_aaaa" attribute contains a value, then the override_filter_aaaa attribute is set to "true".

Note that when filter_aaaa contains a valid value and override_filter_aaaa is set to "false", the last operation takes precedence. Thus the sequence $memberdns->filter_aaaa("YES"); $memberdns->override_filter_aaaa("false"); sets override_filter_aaaa to "false", and the sequence $memberdns->override_filter_aaaa("false"); $memberdns->filter_aaaa("true"); sets override_filter_aaaa to "true".

Parameter

Valid values are "YES", "NO" and "BREAK_DNSSEC". The default value is "NO".

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get the filter aaaa type
my $filter_aaaa = $memberdns->filter_aaaa();
#Modify the filtering
$memberdns->filter_aaaa("YES");
```

filter_aaaa_list( )

Use this method to set or retrieve the list of IPv4 addresses and networks from which queries are received. AAAA filtering is applied to these addresses. This setting overrides the grid filter_aaaa_list if override_filter_aaaa is set to "true".

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Specifying values in this parameter, such as ["10.0.0.10", "10.0.0.20"] or [], implicitly sets the override_filter_aaaa attribute to "true". Setting this parameter and the "filter_aaaa" attribute to undefined causes the appliance to use the grid-level filter_aaaa settings and automatically sets the override_filter_aaaa attribute to "false". But if the "filter_aaaa" attribute contains a value, then the override_filter_aaaa attribute is set to "true".

Note that when filter_aaaa_list contains a valid array reference and override_filter_aaaa is set to "false", the last operation takes precedence. Thus the sequence $memberdns->filter_aaaa_list(["10.0.0.10", "10.0.0.20"]); $memberdns->override_filter_aaaa("false"); sets override_filter_aaaa to "false", and the sequence $memberdns->override_filter_aaaa("false"); $memberdns->filter_aaaa_list(["10.0.0.10", "10.0.0.20"]); sets override_filter_aaaa to "true".

Parameter

A valid value is an Infoblox::Grid::NamedACL object or an array reference that contains IPv4 addresses and networks. To return AAAA records for queries received from an IPv4 client, add "!" as the prefix to the IP addresses and networks. Specify "any" to allow AAAA records to be returned for any query received by an IPv4 client.

Returns

When you specify a parameter and the modification succeeds, the method returns "true". When the operation fails, the method returns "false".

When you do not specify a parameter, the method returns the attribute value.

Example

```
#Getting filter_aaaa_list
my $filter_aaaa_list = $memberdns->filter_aaaa_list( );
#Modifying filter_aaaa_list
$memberdns->filter_aaaa_list(["any"]);
#Modifying filter_aaaa_list
$memberdns->filter_aaaa_list(["10.0.0.10", "20.0.0.0/24", "!"."30.0.0.30", "!"."40.10.0.0/24"]);
#Use Grid::DNS level filter_aaaa_list
$memberdns->filter_aaaa_list(undef);
$memberdns->enable_filter_aaaa(undef);
#Use Infoblox::Grid::NamedACL
$memberdns->filter_aaaa_list($nacl);
```

fixed_rrset_order_fqdns( )

Use this method to set or retrieve configuration for fixed RRSET order for specified FQDNs.

Setting this method to a defined value implicitly sets the enable_fixed_rrset_order_fqdns and override_fixed_rrset_order_fqdns methods to "true". Setting the parameter to undefined causes the appliance to disable fixed RRset order of FQDNs and automatically resets the enable_fixed_rrset_order_fqdns method to "false".

Note that when fixed_rrset_order_fqdns is set to a defined value and enable_fixed_rrset_order_fqdns is set to "false", the last operation takes precedence. Thus the sequence $object->fixed_rrset_order_fqdns($fqdns); $object->enable_fixed_rrset_order_fqdns("false"); will set enable_fixed_rrset_order_fqdns to "false", and the sequence $object->enable_fixed_rrset_order_fqdns("false"); $object->fixed_rrset_order_fqdns($fqdns); will result in enable_fixed_rrset_order_fqdns="true".

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

The valid value is an array reference that contains Infoblox::Grid::DNS::FixedRRSetOrderFQDN objects.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get the fixed_rrset_order_fqdns list
my $fixed_rrset_order_fqdns = $member_dns->fixed_rrset_order_fqdns();
#Modify the fixed_rrset_order_fqdns list
$member_dns->fixed_rrset_order_fqdns([$fqdn1, $fqdn2]);
```

forwarders( )

Use this method to specify forwarders for the member. A forwarder is essentially a name server to which other name servers first send all

of their off-site queries. The forwarder builds up a cache of information, avoiding the need for the other name servers to send queries off-site. This setting overrides the grid-level setting.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

Valid value is an array reference that contains Infoblox::DNS::Member, Infoblox::DNS::Nameserver object(s), or IP addresses.

Specifying values in this parameter, such as ["10.0.0.10", "10.0.0.20"] or [], implicitly sets the override_forwarders attribute to "true". Setting this parameter and the "forward_only" attribute to undefined causes the appliance to use the grid-level forwarders settings and automatically sets the override_forwarders attribute to "false". But if the "forward_only" attribute contains a value, then the override_forwarders attribute is set to "true".

Note that when forwarders_redirect_addresses contains a valid array reference and override_forwarders is set to "false", the last operation takes precedence. Thus the sequence $memberdns->forwarders(["10.0.0.10", "10.0.0.20"]); $memberdns->override_forwarders("false"); sets override_forwarders to "false", and the sequence $memberdns->override_forwarders("false"); $memberdns->forwarders(["10.0.0.10", "10.0.0.20"]); sets override_forwarders to "true".

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Getting forwarders
$memberdns->forwarders( );
#Modifying forwarders
$memberdns->forwarders( [ "10.0.0.10" ] );
```

forward_only( )

Use this method to configure this member to send queries to forwarders only. When the value is "true", the member sends queries to forwarders only, and not to other internal or Internet root servers.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

Specify "true" to enable forward only or "false" to desable it. Default is "false"

Setting this parameter to "true" or "false" implicitly sets the override_forwarders attribute to "true". Setting the parameter to undefined causes the appliance to use the grid-level forwarders settings and automatically sets the override_forwarders attribute to "false", if "forwarders" is also undefined.

Note that when forward_only is set to "true"/"false" and override_forwarders is set to "false", the last operation takes precedence. Thus the sequence $memberdns->forward_only("false"); $memberdns->override_forwarders("false"); sets override_forwarders to "false", and the sequence $memberdns->override_forwarders("false"); $memberdns->forward_only("false"); sets override_forwarders to "true".

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Getting forward_only
$memberdns->forward_only( );
#Modifying forward_only
$memberdns->forward_only("true");
```

forward_updates( )

Use this method to allow/disallow secondary servers to forward updates to the DNS server. This setting overrides grid update settings.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

Specify "true" to enable forward updates or "false" desable it. Default is "false"

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Getting forward_updates
$memberdns->forward_updates( );
#Modifying forward_only
$memberdns->forward_updates("true");
```

ftc_expired_record_ttl( )

Use this method to set or retrieve the TTL value of the expired Fault Tolerant Caching (FTC) record in the DNS responses.

Setting this method to a defined value implicitly sets the override_ftc method to "true". Setting the parameter to undefined if enable_ftc and ftc_expired_record_timeout are also undefined causes the appliance to use the Grid DNS default and automatically resets the override_ftc method to "false".

Note that when ftc_expired_record_ttl is set to a defined value and override_ftc is set to "false", the last operation takes precedence. Thus the sequence $object->ftc_expired_record_ttl("true"); $object->override_ftc("false"); will set override_ftc to "false", and the sequence $object->override_ftc("false"); $object->ftc_expired_record_ttl("true"); will result in override_ftc="true".

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

A 32-bit integer, from 0 to 4294967295, that represents the TTL in seconds. The default value is 5.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get ftc_expired_record_ttl
my $ftc_expired_record_ttl = $memberdns->ftc_expired_record_ttl( );
#Modify ftc_expired_record_ttl
$memberdns->ftc_expired_record_ttl(60);
```

ftc_expired_record_timeout( )

Use this method to set or retrieve the timeout value after which the expired Fault Tolerant Caching (FTC) record becomes stale and is no longer valid.

Setting this method to a defined value implicitly sets the override_ftc method to "true". Setting the parameter to undefined if ftc_expired_record_ttl and enable_ftc are also undefined causes the appliance to use the Grid DNS default and automatically resets the override_ftc method to "false".

Note that when ftc_expired_record_timeout is set to a defined value and override_ftc is set to "false", the last operation takes precedence. Thus the sequence $object->ftc_expired_record_timeout("true"); $object->override_ftc("false"); will set override_ftc to "false", and the sequence $object->override_ftc("false"); $object->ftc_expired_record_timeout("true"); will result in override_ftc="true".

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

A 32-bit integer, from 0 to 4294967295, that represents the timeout in seconds. The default value is 86400.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get ftc_expired_record_timeout
my $ftc_expired_record_timeout = $memberdns->ftc_expired_record_timeout( );
#Modify ftc_expired_record_timeout
$memberdns->ftc_expired_record_timeout(3600);
```

glue_record_addresses( )

Use this method to specify which IP address the appliance uses when it generates the glue record. It can use the IP address of the appliance, the NAT IP address, or an IP address that you define.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

Valid value is an array reference that contains Infoblox::DNS::GlueRecordAddr object(s).

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
my $glue_record_addr1 = Infoblox::DNS::GlueRecordAddr->new(
    view                => "default",  #can only be one of existing views
    glue_record_address => "NAT",
);
$memberdns->glue_record_addresses([$glue_record_addr1]);
```

gss_tsig_keys( )

Use this method to set or retrieve the list of GSS-TSIG keys for a member DNS object.

Setting this method to a defined value implicitly sets the override_gss_tsig_keys method to "true". Setting the parameter to undefined causes the appliance to use the grid default and automatically resets the override_gss_tsig_keys method to "false".

Note that when gss_tsig_keys is set to a defined value and override_gss_tsig_keys is set to "false", the last operation takes precedence. Thus the sequence $object->gss_tsig_keys($keys); $object->override_gss_tsig_keys("false"); will set override_gss_tsig_keys to "false", and the sequence $object->override_gss_tsig_keys("false"); $object->gss_tsig_keys($keys); will result in override_gss_tsig_keys="true".

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

The valid value is an array reference that contains Infoblox::Grid::KerberosKey objects.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get the Kerberos keys list
my $kerberos_keys_list = $memberdns->gss_tsig_keys();
#Modify the Kerberos keys list
my $kerberos_key1 = Infoblox::Cluster::KerberosKey->new(
       principal => 'host/appliance.ivan.test1@LADYBUG.LOCAL',
       );
my $kerberos_key2 = Infoblox::Cluster::KerberosKey->new(
       principal => 'host/appliance.ivan.test2@LADYBUG.LOCAL',
       );
$memberdns->gss_tsig_keys([$kerberos_key1, $kerberos_key2]);
```

host_name_restriction_policy( )

Use this method to set or retrieve the member-level host name restriction policy that overrides the grid-level settings.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

Valid value is the name of record name policy defined at the grid level via Infoblox::Grid::RecordNamePolicy

There are three predefined policies:

Policy Name Regex Applied

"Allow Any" .+

"Allow Underscore" ^[-a-zA-Z0-9_.]+$

"Strict Hostname Checking" ^[a-zA-Z0-9]$|^[a-zA-Z0-9][-a-zA-Z0-9.]*[a-zA-Z0-9]$

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Getting host_name_restriction_policy
$memberdns->host_name_restriction_policy( );
#Modifying host_name_restriction_policy
$memberdns->host_name_restriction_policy("Allow Any");
#Un-override host_name_restriction_policy
$memberdns->host_name_restriction_policy(undef);
```

ipv6_glue_record_addresses( )

Use this method to specify which IPv6 address the appliance uses when it generates the glue record. It can use the IPv6 address of the appliance, the NAT IPv6 address, or an IPv6 address that you define.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

Valid value is an array of Infoblox::DNS::GlueRecordAddr objects.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
my $v6_glue_record_addr1 = Infoblox::DNS::GlueRecordAddr->new(
    view                => "default",  #can only be one of existing views
    glue_record_address => "2001:db8::1",
);

$memberdns->ipv6_glue_record_addresses([$v6_glue_record_addr1]);
```

is_unbound_capable( )

Use this method to retrieve the flag that indicates whether Member DNS supports unbound recursive resolver or not. This is a read-only attribute.

Omit the parameter to retrieve the attribute value.

Parameter

None

Returns

The method returns the attribute value.

Example

```
#Get is_unbound_capable
my $is_unbound_capable = $memberdns->is_unbound_capable();
```

lame_ttl( )

Use this method to set or retrieve the number of seconds to cache lame delegations or lame servers.

Parameter

The number of seconds to cache lame delegations or lame servers. The default value is 600.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get lame_ttl
my $lame_ttl = $memberdns->lame_ttl();
#Modify lame_ttl
$memberdns->lame_ttl(700);
```

logging_categories( )

Use this method to set logging categories for this member. This setting overrides grid logging category settings.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

Valid value is an array reference that contains one or more of the following values:

"general": Records the BIND messages that are not specifically classified.

"config": Records the configuration file parsing messages.

"dnssec": Records the DNSSEC-signed responses.

"network": Records the network operation messages.

"queries": Records the query messages.

"responses": Records the response messages.

"security": Records the approved and denied requests.

"xfer-in": Records zone transfer messages from the remote name servers to the appliance.

"xfer-out": Records zone transfer messages from the Infoblox appliance to remote name servers.

"update": Records the dynamic update instances.

"resolver": Records the DNS resolution instances, including recursive queries from resolvers.

"notify": Records the asynchronous zone change notification messages.

"lame_servers": Records bad delegation instances.

"database": Records BIND's internal database processes.

"client": Records client requests.

"rpz": Records RPZ messages.

"dtc_gslb": Records DTC GSLB activity.

"dtc_health": Records DTC health monitoring.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Getting logging_categories
$memberdns->logging_categories( );
#Modifying logging_categories
$memberdns->logging_categories( [ "general", "database" , "queries" ] );
```

max_cache_ttl( )

Use this method to set or retrieve the maximum time (in seconds) for which the server will cache positive answers.

Parameter

The maximum time is specified in seconds. The default value is 604800.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get max_cache_ttl
my $max_cache_ttl = $memberdns->max_cache_ttl();
#Modify max_cache_ttl
$memberdns->max_cache_ttl(400);
```

max_ncache_ttl( )

Use this method to set or retrieve the maximum time (in seconds) for which the server will cache negative (NXDOMAIN) responses. This method applies for the defined member configuration.

Parameter

The maximum time is specified in seconds. The default value is 10800. The maximum allowed value is 604800.

Setting this parameter implicitle sets the override_max_ncache_ttl attribute to "true". Setting the parameter to undefined causes the appliance to use the Grid-level settings and automatically sets the override_max_ncache_ttl attribute to "false".

Note that when max_ncache_ttl is set and override_max_ncache_ttl is set to "false", the last operation takes precedence. Thus the sequence $firstview->max_ncache_ttl(10800); $firstview->override_max_ncache_ttl("false"); sets override_max_ncache_ttl to "false", and the sequence $firstview->override_max_ncache_ttl("false"); $firstview->max_ncache_ttl(30600); sets override_max_ncache_ttl to "true".

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get max_ncache_ttl
my $max_ncache_ttl = $memberdns->max_ncache_ttl();
#Modify max_ncache_ttl
$memberdns->max_ncache_ttl(400);
```

max_cached_lifetime( )

Use this method to set or retrieve maximum time in seconds a DNS response can be stored in the hardware acceleration cache.

Parameter

Valid values are unsigned integer between 60 and 86400, inclusive. Default value is 86400.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get max_cached_lifetime
my $max_cached_lifetime = $memberdns->max_cached_lifetime();
#Modify max_cached_lifetime
$memberdns->max_cached_lifetime(400);
```

minimal_resp( )

Use this method to enable/disable the ability to return a minimal amount of data in response to a query. This capability speeds up the DNS services provided by the appliance.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

Specify "true" to enable minimal responses or "false" to desable it. Default is "true"

Returns

If you specified a parameter, the method returns 1 when the modification succeeds, and returns other values when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Getting minimal_resp
$memberdns->minimal_resp( );
#Modifying minimal_resp
$memberdns->minimal_resp("true");
```

name( )

Use this method to retrieve the grid name.

Parameter

None.

Returns

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get name
my $name = $memberdns->name();
```

notify_delay( )

Use this method to specify with how many seconds of delay the notify messages are sent to secondaries.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

Valid values are unsigned integer between 5 and 86400, inclusive. Default value is undefined.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Getting notify_delay
my $notify_delay = $memberdns->notify_delay( );
#Modifying notify_delay
$memberdns->notify_delay(5);
```

notify_source_port( )

Use this method to specify the source port for notify messages. When requesting zone transfers from the primary server, some secondary DNS servers use the source port number (the primary server used to send the notify message) as the destination port number in the zone transfer request. This setting overrides grid static source port settings.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

Source port number for notify messages. Valid values are between 1 and 63999. The default is picked by BIND.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Getting notify_source_port
$memberdns->notify_source_port( );
#Modifying notify_source_port
$memberdns->notify_source_port("2");
```

override_attack_mitigation( )

The override_attack_mitigation attribute controls whether the attack_mitigation method values of the member are used, instead of the Grid default.

The override_attack_mitigation attribute can be specified explicitly. It is also set implicitly when attack_mitigation is set to a defined value.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

Set the parameter to "true" to override the Grid-level setting for attack_mitigation. Set the parameter to "false" to inherit the Grid-level setting for attack_mitigation.

The default value of this parameter is "false".

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Getting override_attack_mitigation
my $override_attack_mitigation = $memberdns->override_attack_mitigation( );
#Modifying override_attack_mitigation
$memberdns->override_attack_mitigation("true");
```

override_auto_blackhole( )

The override_auto_blackhole attribute controls whether the auto_blackhole method values of the member are used, instead of the Grid default.

The override_auto_blackhole attribute can be specified explicitly. It is also set implicitly when auto_blackhole is set to a defined value.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

Set the parameter to "true" to override the Grid-level setting for auto_blackhole. Set the parameter to "false" to inherit the Grid-level setting for auto_blackhole.

The default value of this parameter is "false".

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Getting override_auto_blackhole
my $override_auto_blackhole = $memberdns->override_auto_blackhole( );
#Modifying override_auto_blackhole
$memberdns->override_auto_blackhole("true");
```

override_bind_hostname_directive( )

The override_bind_hostname_directive attribute controls whether the bind_hostname_directive method values of the member are used, instead of the Grid default.

The override_bind_hostname_directive attribute can be specified explicitly. It is also set implicitly when bind_hostname_directive is set to a defined value.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

Set the parameter to "true" to override the Grid-level setting for bind_hostname_directive. Set the parameter to "false" to inherit the Grid-level setting for bind_hostname_directive.

The default value of this parameter is "false".

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Getting override_bind_hostname_directive
my $override_bind_hostname_directive = $memberdns->override_bind_hostname_directive( );
#Modifying override_bind_hostname_directive
$memberdns->override_bind_hostname_directive("true");
```

override_server_id_directive( )

The override_server_id_directive attribute controls whether the server_id_directive method values of the member are used, instead of the Grid default.

The override_server_id_directive attribute can be specified explicitly. It is also set implicitly when server_id_directive is set to a defined

value.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

Set the parameter to "true" to override the Grid-level setting for server_id_directive. Set the parameter to "false" to inherit the Grid-level setting for server_id_directive.

The default value of this parameter is "false".

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Getting override_server_id_directive
my $override_server_id_directive = $memberdns->override_server_id_directive( );
#Modifying override_server_id_directive
$memberdns->override_server_id_directive("true");
```

override_capture_dns_queries_on_all_domains( )

The override_capture_dns_queries_on_all_domains attribute controls whether the capture_dns_queries_on_all_domains method values of the member are used, instead of the Grid default.

The override_capture_dns_queries_on_all_domains attribute can be specified explicitly. It is also set implicitly when capture_dns_queries_on_all_domains is set to a defined value.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

Set the parameter to "true" to override the Grid-level setting for capture_dns_queries_on_all_domains. Set the parameter to "false" to inherit the Grid-level setting for capture_dns_queries_on_all_domains.

The default value of this parameter is "false".

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
 #Get override_capture_dns_queries_on_all_domains value
 my $override_capture_dns_queries_on_all_domains = $object-
>override_capture_dns_queries_on_all_domains();

 #Modify override_capture_dns_queries_on_all_domains value
 $object->override_capture_dns_queries_on_all_domains("true");
```

override_copy_xfer_to_notify( )

The override_copy_xfer_to_notify attribute controls whether the copy_xfer_to_notify method values of the member are used, instead of the grid default.

The override_copy_xfer_to_notify attribute can be specified explicitly. It is also set implicitly when copy_xfer_to_notify is set to a defined value.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

Set the parameter to "true" to override the grid-level setting for copy_xfer_to_notify. Set the parameter to "false" to inherit the grid-level setting for copy_xfer_to_notify.

The default value of this parameter is "false".

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Getting override_copy_xfer_to_notify
my $override_copy_xfer_to_notify = $memberdns->override_copy_xfer_to_notify( );
#Modifying override_copy_xfer_to_notify
$memberdns->override_copy_xfer_to_notify("true");
```

override_disable_edns( )

The override_disable_edns attribute controls whether the disable_edns method values of the member are used, instead of the Grid default.

The override_disable_edns attribute can be specified explicitly. It is also set implicitly when disable_edns is set to a defined value.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

Set the parameter to "true" to override the grid-level setting for disable_edns. Set the parameter to "false" to inherit the grid-level setting for disable_edns. The default value is "false".

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get override_disable_edns
my $override_disable_edns = $memberdns->override_disable_edns();
#Modify override_disable_edns
$memberdns->override_disable_edns("true");
```

override_dns64( )

The override_dns64 attribute controls whether the DNS64 method values of the member are used, instead of the grid default.

The override_dns64 attribute can be specified explicitly. It is also set implicitly when enable_dns64 or dns64_groups are set to a defined value.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

Set the parameter to "true" to override the grid-level setting for enable_dns64 and dns64_groups. Set the parameter to "false" to inherit the grid-level setting for enable_dns64 and dns64_groups.

The default value of this parameter is "false".

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Getting override_dns64
my $override_dns64=$memberdns->override_dns64( );
#Modifying override_dns64
$memberdns->override_dns64("true");
```

override_dns_health_check( )

The override_dns_health_check attribute controls whether the DNS health check settings Member are used, instead of the Grid default.

The override_dns_health_check attribute can be specified explicitly. It is also set implicitly when following fields are set to a defined value:

```
dns_health_check_domain_list
dns_health_check_recursion_flag
dns_health_check_anycast_control
dns_health_check_interval
dns_health_check_retries
dns_health_check_timeout
enable_dns_health_check
```

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

Specify 'true' to override the Grid-level DNS health check setting or 'false' to inherit the Grid-level DNS health check setting.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get override_dns_health_check value
my $value = $object->override_dns_health_check();

#Modify override_dns_health_check value
$object->override_dns_health_check('true');
```

override_filter_aaaa( )

The override_filter_aaaa attribute controls whether the AAAA filter method values of the member are used, instead of the grid default.

The override_filter_aaaa attribute can be specified explicitly. It is also set implicitly when filter_aaaa or filter_aaaa_list are set to a defined value.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

Set the parameter to "true" to override the grid-level setting for filter_aaaa and filter_aaaa_list. Set the parameter to "false" to inherit the grid-level setting for filter_aaaa and filter_aaaa_list.

The default value of this parameter is "false".

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Getting override_filter_aaaa
my $override_filter_aaaa=$memberdns->override_filter_aaaa( );
#Modifying override_filter_aaaa
$memberdns->override_filter_aaaa("true");
```

override_dns_cache_acceleration_ttl( )

The override_dns_cache_acceleration_ttl attribute controls whether the dns_cache_acceleration_ttl method values of the member are used, instead of the grid default.

The override_dns_cache_acceleration_ttl attribute can be specified explicitly. It is also set implicitly when dns_cache_acceleration_ttl is set to a defined value.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

Set the parameter to "true" to override the grid-level setting for dns_cache_acceleration_ttl. Set the parameter to "false" to inherit the grid-level setting for dns_cache_acceleration_ttl.

The default value of this parameter is "false".

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Getting override_dns_cache_acceleration_ttl
my $override_dns_cache_acceleration_ttl=$memberdns->override_dns_cache_acceleration_ttl( );
#Modifying override_dns_cache_acceleration_ttl
$memberdns->override_dns_cache_acceleration_ttl("true");
```

override_ftc( )

The override_ftc attribute controls whether the values of the Fault Tolerant Caching (FTC) feature are used for the member, instead of the

Grid default.

The override_ftc attribute can be specified explicitly. It is also set implicitly when the following fields are set to a defined value:

```
enable_ftc
ftc_expired_record_ttl
ftc_expired_record_timeout
```

Parameter

Set the parameter to "true" to override the Grid-level setting for FTC feature. Set the parameter to "false" to inherit the Grid-level setting for FTC. The default value is "false".

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get override_ftc
my $override_ftc = $memberdns->override_ftc();
#Modify override_ftc
$memberdns->override_ftc('true');
```

override_gss_tsig_keys( )

The override_gss_tsig_keys attribute controls whether the gss_tsig_keys method values for the member are used, instead of the Grid default.

The override_gss_tsig_keys attribute can be specified explicitly. It is also set implicitly when gss_tsig_keys is set to a defined value.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

Set the parameter to "true" to override GSS-TSIG keys. Set the parameter to "false" to inherit the grid-level GSS-TSIG keys.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Getting override_gss_tsig_keys
my $override_gss_tsig_keys = $memberdns->override_gss_tsig_keys();
#Modyfying override_gss_tsig_keys
$memberdns->override_gss_tsig_keys("true");
```

override_enable_capture_dns( )

The override_enable_capture_dns attribute controls whether the enable_capture_dns_queries and enable_capture_dns_responses method values of the member are used, instead of the Grid default.

The override_enable_capture_dns attribute can be specified explicitly. It is also set implicitly when enable_capture_dns_queries or enable_capture_dns_responses is set to a defined value.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

Set the parameter to "true" to override the Grid-level setting for enable_capture_dns_queries and enable_capture_dns_responses. Set the parameter to "false" to inherit the Grid-level setting for enable_capture_dns_queries and enable_capture_dns_responses.

The default value of this parameter is "false".

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get override_enable_capture_dns value
my $override_enable_capture_dns = $object->override_enable_capture_dns();
```

```
#Modify override_enable_capture_dns value
$object->override_enable_capture_dns("true");
```

override_enable_excluded_domain_names( )

The override_enable_excluded_domain_names attribute controls whether the capture_dns_queries_on_all_domains method values of the member are used, instead of the Grid default.

The override_enable_excluded_domain_names attribute can be specified explicitly. It is also set implicitly when capture_dns_queries_on_all_domains is set to a defined value.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

Set the parameter to "true" to override the Grid-level setting for capture_dns_queries_on_all_domains. Set the parameter to "false" to inherit the Grid-level setting for capture_dns_queries_on_all_domains.

The default value of this parameter is "false".

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get override_enable_excluded_domain_names value
my $override_enable_excluded_domain_names = $object->override_enable_excluded_domain_names();

#Modify override_enable_excluded_domain_names value
$object->override_enable_excluded_domain_names("true");
```

override_enable_gss_tsig( )

The override_enable_gss_tsig attribute controls whether the enable_gss_tsig method value for the member is used, instead of the Grid default.

The override_enable_gss_tsig attribute can be specified explicitly. It is also set implicitly when enable_gss_tsig is set to a defined value.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

Set the parameter to "true" to override the grid-level setting for enable_gss_tsig. Set the parameter to "false" to inherit the grid-level setting for enable_gss_tsig value.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Getting override_enable_gss_tsig
my $override_enable_gss_tsig = $memberdns->override_enable_gss_tsig();
#Modyfying override_enable_gss_tsig
$memberdns->override_enable_gss_tsig("true");
```

override_max_cache_ttl( )

The override_max_cache_ttl attribute controls whether the max_cache_ttl method values of the member are used, instead of the grid default.

The override_max_cache_ttl attribute can be specified explicitly. It is also set implicitly when max_cache_ttl is set to a defined value.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

Set the parameter to "true" to override the grid-level setting for max_cache_ttl. Set the parameter to "false" to inherit the grid-level setting for max_cache_ttl.

The default value of this parameter is "false".

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Getting override_max_cache_ttl
my $override_max_cache_ttl = $memberdns->override_max_cache_ttl( );
#Modifying override_max_cache_ttl
$memberdns->override_max_cache_ttl("true");
```

override_max_ncache_ttl( )

The override_max_ncache_ttl attribute controls whether the max_ncache_ttl method values of the member are used, instead of the Grid default.

The override_max_ncache_ttl attribute can be specified explicitly. It is also set implicitly when max_ncache_ttl is set to a defined value.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

Set the parameter to "true" to override the grid-level setting for max_ncache_ttl. Set the parameter to "false" to inherit the Grid-level setting for max_ncache_ttl.

The default value of this parameter is "false".

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Getting override_max_ncache_ttl
my $override_max_ncache_ttl = $memberdns->override_max_ncache_ttl( );
#Modifying override_max_ncache_ttl
$memberdns->override_max_ncache_ttl("true");
```

override_max_cached_lifetime( )

The override_max_cached_lifetime attribute controls whether the max_cached_lifetime method values of the member are used, instead of the grid default.

The override_max_cached_lifetime attribute can be specified explicitly. It is also set implicitly when max_cached_lifetime is set to a defined value.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

Set the parameter to "true" to override the grid-level setting for max_cached_lifetime. Set the parameter to "false" to inherit the grid-level setting for max_cached_lifetime.

The default value of this parameter is "false".

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Getting override_max_cached_lifetime
my $override_max_cached_lifetime = $memberdns->override_max_cached_lifetime( );
#Modifying override_max_cached_lifetime
$memberdns->override_max_cached_lifetime("true");
```

override_nxdomain_redirect( )

The "override_nxdomain_redirect" attribute controls whether the "nxdomain_redirect", "nxdomain_redirect_addresses", "nxdomain_redirect_addresses_ipv6", "nxdomain_redirect_ttl", "nxdomain_log_query", and "nxdomain_rulesets" values for this member are used. Note that these values can be overridden at the DNS view level as well.

The "override_nxdomain_redirect" attribute can be specified explicitly. It is also set implicitly as follows: "nxdomain_redirect" => "true"|"false" or "nxdomain_redirect_addresses" => ["10.0.0.10", "10.0.0.20"]|[] or "nxdomain_redirect_addresses" => ["2001:db8::1",

"2002:ffff::1"]|[] result in "override_nxdomain_redirect" => "true", "nxdomain_redirect" => undef and "nxdomain_redirect_addresses" => undef and "nxdomain_redirect_addresses_ipv6" => undef result in "override_nxdomain_redirect" => "false". If "override_nxdomain_redirect" is specified explicitly as "true", and "nxdomain_redirect" => undef, "nxdomain_redirect_addresses" => undef and "nxdomain_redirect_addresses_ipv6" => undef, then the explicit value takes precedence; and "override_nxdomain_redirect"=>"true", "nxdomain_redirect", "nxdomain_redirect_addresses", and "nxdomain_redirect_addresses" remain unchanged in the database.

Specifying the "nxdomain_redirect_ttl", "nxdomain_log_query", or "nxdomain_rulesets" attributes does not affect the "override_nxdomain_redirect" value.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

Set the parameter to "true" to override the grid-level setting of the NXDOMAIN redirection attributes. Set the parameter to "false" to inherit the grid-level settings.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get override_nxdomain_redirect
my $override_nxdomain_redirect=$memberdns->override_nxdomain_redirect( );
#Modify override_nxdomain_redirect
$memberdns->override_nxdomain_redirect("true");
```

override_lame_ttl( )

Use this method to set or retrieve the override_lame_ttl attribute that controls whether the lame_ttl method values of the member are used, instead of the Grid default

Parameter

Set the parameter to "true" to override the grid-level setting for lame_ttl. Set the parameter to "false" to inherit the grid-level setting for lame_ttl. The default value is 'false'.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get override_lame_ttl
my $override_lame_ttl = $memberdns->override_lame_ttl();
#Modify override_lame_ttl
$memberdns->override_lame_ttl('true');
```

override_rpz_qname_wait_recurse( )

The override_rpz_qname_wait_recurse attribute controls whether the rpz_qname_wait_recurse method value for the member is used, instead of the Grid default.

The "override_rpz_qname_wait_recurse" attribute can be specified explicitly. It is also established implicitly as follows: "rpz_qname_wait_recurse" => "true"|"false" results in "override_rpz_qname_wait_recurse" => "true"; "rpz_qname_wait_recurse" => undef results in "override_rpz_qname_wait_recurse" => "false"; If "override_rpz_qname_wait_recurse" is specified explicitly as "true", while "rpz_qname_wait_recurse"=>undef, the explicit value takes precedence, and "override_rpz_qname_wait_recurse"=>"true" while "rpz_qname_wait_recurse" remains unchanged in the database.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

Set the parameter to "true" to override the Grid-level setting for rpz_qname_wait_recurse. Set the parameter to "false" to inherit the Grid-level setting for rpz_qname_wait_recurse value.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Getting override_rpz_qname_wait_recurse
my $override_rpz_qname_wait_recurse = $memberdns->override_rpz_qname_wait_recurse();
#Modyfying override_rpz_qname_wait_recurse
$memberdns->override_rpz_qname_wait_recurse("true");
```

override_transfers_in( )

Use this method to set or retrieve the override_transfers_in attribute that controls whether the transfers_in method values of the member are used, instead of the Grid default.

Parameter

Set the parameter to "true" to override the grid-level setting for transfers_in. Set the parameter to "false" to inherit the grid-level setting for transfers_in. The default value is 'false'.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get override_transfers_in
my $override_transfers_in = $memberdns->override_transfers_in();
#Modify override_transfers_in
$memberdns->override_transfers_in('true');
```

override_transfers_out( )

Use this method to set or retrieve the override_transfers_out attribute that controls whether the transfers_out method values of the member are used, instead of the Grid default.

Parameter

Set the parameter to "true" to override the grid-level setting for transfers_out. Set the parameter to "false" to inherit the grid-level setting for transfers_out. The default value is '

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get override_transfers_out
my $override_transfers_out = $memberdns->override_transfers_out();
#Modify override_transfers_out
$memberdns->override_transfers_out('true');
```

override_transfers_per_ns( )

Use this method to set or retrieve the transfers_per_ns attribute that controls whether the transfers_per_ns method values of the member are used, instead of the Grid default.

Parameter

Set the parameter to "true" to override the grid-level setting for transfers_per_ns. Set the parameter to "false" to inherit the grid-level setting for transfers_per_ns. The default value is 'false'.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get override_transfers_per_ns
my $override_transfers_per_ns = $memberdns->override_transfers_per_ns();
#Modify override_transfers_per_ns
$memberdns->override_transfers_per_ns('true');
```

override_serial_query_rate( )

Use this method to set or retrieve the override_serial_query_rate attribute that controls whether the serial_query_rate method values of the member are used, instead of the Grid default.

Parameter

Set the parameter to "true" to override the grid-level setting for serial_query_rate. Set the parameter to "false" to inherit the grid-level setting for serial_query_rate. The default value is 'false'.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get override_serial_query_rate
my $override_serial_query_rate = $memberdns->override_serial_query_rate();
#Modify override_serial_query_rate
$memberdns->override_serial_query_rate('true');
```

override_query_rewrite( )

Use this method to set or retrieve the override_query_rewrite that controls whether the enable_query_rewrite method values of the member are used, instead of the grid-level enable_query_rewrite value.

Parameter

Set the parameter to "true" to override the grid-level DNS query rewrite settings. Set the parameter to "false" to inherit the grid-level DNS query rewrite settings. The default is "false".

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get override_query_rewrite
my $override_query_rewrite = $memberdns->override_query_rewrite();
#Modify override_query_rewrite
$memberdns->override_query_rewrite("true");
```

nxdomain_redirect( )

Use this method to enable or disable NXDOMAIN redirection.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

Specify "true" to set the nxdomain_redirect flag or "false" to deactivate/unset it. The default is "false".

Setting this parameter to "true" or "false" implicitly sets the override_nxdomain_redirect attribute to "true". Setting the parameter to undefined causes the appliance to use the grid-level NXDOMAIN redirection settings and automatically sets the override_nxdomain_redirect attribute to "false" if nxdomain_redirect_addresses or nxdomain_redirect_addresses_ipv6 is also undefined. But if nxdomain_redirect_addresses or nxdomain_redirect_addresses_ipv6 contains a value, then the override_nxdomain_redirect attribute is set to true.

Note that when nxdomain_redirect is set to "true"/"false" and override_nxdomain_redirect is set to "false", the last operation takes precedence. Thus the sequence $grid_member->nxdomain_redirect("false"); $grid_member->override_nxdomain_redirect("false"); sets override_nxdomain_redirect to "false", and the sequence $grid_member->override_nxdomain_redirect("false"); $grid_member->nxdomain_redirect("false"); sets override_nxdomain_redirect to "true".

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Getting nxdomain_redirect
my $nxdomain_redirect = $grid_member->nxdomain_redirect( );
#Modifying nxdomain_redirect
$grid_member->nxdomain_redirect("true");
```

nxdomain_redirect_addresses( )

Use this method to set the NXDOMAIN redirection addresses.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

The valid value is an array of IPv4 addresses. The default value is undefined.

Setting this parameter to a defined value implicitly sets the override_nxdomain_redirect attribute to "true". Setting the parameter to undefined causes the appliance to use the grid-level NXDOMAIN redirection settings and automatically sets the override_nxdomain_redirect attribute to "false" if nxdomain_redirect and nxdomain_redirect_addresses_ipv6 is also undefined. But if nxdomain_redirect or nxdomain_redirect_addresses_ipv6 contains a value, then the override_nxdomain_redirect attribute is set to true.

Note that when nxdomain_redirect_addresses contains a valid array reference and override_nxdomain_redirect is set to "false", the last operation takes precedence. Thus the sequence $grid_member->nxdomain_redirect_addresses(["10.0.0.10", "10.0.0.20"]); $grid_member->override_nxdomain_redirect("false"); sets override_nxdomain_redirect to "false", and the sequence $grid_member->override_nxdomain_redirect("false"); $grid_member->nxdomain_redirect_addresses(["10.0.0.10", "10.0.0.20"]); sets override_nxdomain_redirect to "true".

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get nxdomain_redirect_addresses
my $nxdomain_redirect_addresses = $grid_member->nxdomain_redirect_addresses( );
#Modify nxdomain_redirect_addresses
$grid_member->nxdomain_redirect_addresses(["10.0.0.10", "10.0.0.20"]);
```

nxdomain_redirect_addresses_ipv6( )

Use this method to set the NXDOMAIN redirection IPv6 addresses.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

The valid value is an array of IPv6 addresses. The default value is undefined.

Setting this parameter to a defined value implicitly sets the override_nxdomain_redirect attribute to "true". Setting the parameter to undefined causes the appliance to use the grid-level NXDOMAIN redirection settings and automatically sets the override_nxdomain_redirect attribute to "false" if nxdomain_redirect and nxdomain_redirect_addresses is also undefined. But if nxdomain_redirect or nxdomain_redirect_addresses contains a value, then the override_nxdomain_redirect attribute is set to true.

Note that when nxdomain_redirect_addresses_ipv6 contains a valid array reference and override_nxdomain_redirect is set to "false", the last operation takes precedence. Thus the sequence $grid_member->nxdomain_redirect_addresses_ipv6(["2001:db8::1", "2002:ffff::1"]); $grid_member->override_nxdomain_redirect("false"); sets override_nxdomain_redirect to "false", and the sequence $grid_member->override_nxdomain_redirect("false"); $grid_member->nxdomain_redirect_addresses_ipv6(["2001:db8::1", "2002:ffff::1"]); sets override_nxdomain_redirect to "true".

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get nxdomain_redirect_addresses_ipv6
my $nxdomain_redirect_addresses_ipv6 = $grid_member->nxdomain_redirect_addresses_ipv6( );
#Modify nxdomain_redirect_addresses_ipv6
$grid_member->nxdomain_redirect_addresses_ipv6(["2001:db8::1", "2002:ffff::1"]);
```

nxdomain_redirect_ttl( )

Use this method to set or retrieve the TTL value of synthetic DNS responses that result from NXDOMAIN redirection.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

A 32-bit integer, from 0 to 4294967295, that represents the TTL in seconds. The default value is 60.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get nxdomain_redirect_ttl
my $nxdomain_redirect_ttl = $grid_member->nxdomain_redirect_ttl( );
#Modify nxdomain_redirect_ttl
$grid_member->nxdomain_redirect_ttl(60);
```

nxdomain_log_query( )

Use this method to set or retrieve the flag that indicates whether NXDOMAIN redirection queries are logged.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

Specify "true" to enable logging, or "false" to deactivate it. The default value is "false".

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get nxdomain_log_query
my $nxdomain_log_query = $grid_member->nxdomain_log_query( );
#Modify nxdomain_log_query
$grid_member->nxdomain_log_query("true");
```

nxdomain_rulesets( )

Use this method to specify or retrieve Infoblox::DNS::Ruleset object names assigned at the grid level for NXDOMAIN redirection.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

The valid value is an array reference that contains the names of the Infoblox::DNS::Ruleset objects assigned to the grid. The names must be in string format. The default value is undefined.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get nxdomain_rulesets
my $nxdomain_rulesets = $grid_member->nxdomain_rulesets();
#Modify nxdomain_rulesets
$grid_member->nxdomain_rulesets(["ruleset1", "ruleset2"]);
```

override_blacklist( )

The "override_blacklist" attribute controls whether the "enable_blacklist", "blacklist_redirect_addresses", "blacklist_redirect_ttl", "blacklist_log_query", and "blacklist_rulesets" values in the database are used.

The "override_blacklist" attribute can be specified explicitly. It is also set implicitly as follows: "enable_blacklist" => "true"|"false" or "blacklist_redirect_addresses" => ["10.0.0.10", "10.0.0.20"]|[] results in "override_blacklist" => "true"; "enable_blacklist" => undef and "blacklist_redirect_addresses" => undef result in "override_blacklist" => "false". If "override_blacklist" is specified explicitly as "true", and "enable_blacklist" => undef or "blacklist_redirect_addresses" => undef, then the explicit value takes precedence. Therefore, "override_blacklist"=>"true", "enable_blacklist"=>undef, and "blacklist_redirect_addresses"=>undef remain unchanged in the database.

Specifying the "blacklist_redirect_ttl", "blacklist_log_query", or "blacklist_rulesets" attributes does not affect the "override_blacklist" value. It is implicitly set only by "enable_blacklist" and "blacklist_redirect_addresses".

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

Set the parameter to "true" to override the grid-level settings of the blacklist redirect attributes. Set the parameter to "false" to inherit the grid-level settings. The default is "false".

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get override_blacklist
my $override_blacklist=$memberdns->override_blacklist( );
#Modify override_blacklist
$memberdns->override_blacklist("true");
```

override_dnssec( )

The "override_dnssec" attribute controls whether the "dnssec_enabled", "dnssec_validation_enabled", "dnssec_expired_signatures_enabled", and "dnssec_trusted_keys" values in the database are used. Note that these values can be overridden at the DNS view level as well.

The "override_dnssec" attribute can be specified explicitly. It is also established implicitly as follows: "dnssec_enabled" => "true"|"false" results in "override_dnssec" => "true" "dnssec_enabled" => undef results in "override_dnssec" => "false" If "override_dnssec" is specified explicitly as "true", while "dnssec_enabled"=>undef, the explicit value takes precedence, and "override_dnssec"=>"true" while "dnssec_enabled" remains unchanged in the database.

Supplying the "dnssec_validation_enabled", "dnssec_expired_signatures_enabled", or "dnssec_trusted_keys" attributes does not affect the "override_dnssec" value. It is implicitly affected only by "dnssec_enabled".

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

Set the parameter to "true" to override the grid-level setting of DNSSEC attributes. Set the parameter to "false" to inherit the grid-level settings.

The default value of this parameter is "false".

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Getting override_dnssec
my $override_dnssec=$memberdns->override_dnssec( );
#Modifying override_dnssec
$memberdns->override_dnssec("true");
```

override_dtc_edns_prefer_client_subnet( )

The override_dtc_edns_prefer_client_subnet attribute controls whether the dtc_edns_prefer_client_subnet method values of the member are used, instead of the Grid default.

The override_dtc_edns_prefer_client_subnet attribute can be specified explicitly. It is also set implicitly when dtc_edns_prefer_client_subnet is set to a defined value.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

Set the parameter to "true" to override the grid-level setting for dtc_edns_prefer_client_subnet. Set the parameter to "false" to inherit the Grid-level setting for dtc_edns_prefer_client_subnet. The default value is "false".

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get override_dtc_edns_prefer_client_subnet
my $override_dtc_edns_prefer_client_subnet = $memberdns->override_dtc_edns_prefer_client_subnet();
#Modify override_dtc_edns_prefer_client_subnet
$memberdns->override_dtc_edns_prefer_client_subnet("true");
```

override_forwarders( )

The "override_forwarders" attribute controls whether the "forward_only" and "forwarders" values in the database are used.

The "override_forwarders" attribute can be specified explicitly. It is also set implicitly as follows: "forward_only" => "true"|"false" or "forwarders" => ["10.0.0.10", "10.0.0.20"]|[] results in "override_forwarders" => "true"; "forward_only" => undef and "forwarders" => undef result in "override_forwarders" => "false". If "override_forwarders" is specified explicitly as "true", and "forward_only" => undef or "forwarders" => undef, then the explicit value takes precedence. Therefore, "override_forwarders"=>"true", "forward_only"=>undef, and "forwarders"=>undef remain unchanged in the database.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

Set the parameter to "true" to override the grid-level settings of the forwarders attributes. Set the parameter to "false" to inherit the grid-level settings. The default is "false".

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get override_forwarders
my $override_forwarders=$memberdns->override_forwarders( );
#Modify override_forwarders
$memberdns->override_forwarders("true");
```

override_fixed_rrset_order_fqdns( )

The override_fixed_rrset_order_fqdns controls whether the enable_fixed_rrset_order_fqdns and fixed_rrset_order_fqdns values in the database are used. Note that these values can be overridden at the DNS View level as well.

The override_fixed_rrset_order_fqdns can be specified explicitly. It is also set implicitly as follows: enable_fixed_rrset_order_fqdns => "true" | "false" or fixed_rrset_order_fqdns => $fqdns results in override_fixed_rrset_order_fqdns => "true";

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

Set the parameter to "true" to override the Grid-level fixed RRset order settings. Set the parameter to "false" to inherit the Grid-level settings. The default is "false".

Returns

When you specify a parameter and the modification succeeds, the method returns "true". When the operation fails, the method returns "false".

When you do not specify a parameter, the method returns the attribute value.

Example

```
#Get override_fixed_rrset_order_fqdns flag
my $override_fixed_rrset_order_fqdns = $member_dns->override_fixed_rrset_order_fqdns();
#Modify override_fixed_rrset_order_fqdns flag
$member_dns->override_fixed_rrset_order_fqdns("true");
```

query_source_port( )

Use this method to specify the source port for queries. Specifying a source port number for recursive queries ensures that a firewall will allow the response. This setting overrides grid static source port settings.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

Source port number for queries. Valid values are between 1 and 63999. The default is picked by BIND.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Getting query_source_port
$memberdns->query_source_port( );
#Modifying notify source port
```

```
$memberdns->query_source_port("2");
```

recursive_client_limit( )

Use this method to set a limit on the number of concurrent recursive clients.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

Maximum number of concurrent recursive clients. Range is from 0 to 4294967295

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Getting recursive_client_limit
$memberdns->recursive_client_limit( );
#Modifying recursive_client_limit
$memberdns->recursive_client_limit("10");
```

recursive_query_list( )

Use this method to set or retrieve the list of IPv4 or IPv6 addresses, networks or hosts authenticated by Transaction signature (TSIG) key from which recursive queries are allowed or denied. This setting overrides grid recursive query list settings.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

Valid value is an Infoblox::Grid::NamedACL object or an array reference that contains IPv4/IPv6 addresses, networks and/or Infoblox::DNS::TSIGKey objects. To deny queries, add '!' as a prefix to IP addresses and networks. Specify "any" to allow any recursive query, or undefine to use the Grid::DNS level recursive_query_list list. Note: You should also undefine the allow_recursive_query list to use Grid::DNS level settings.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Getting recursive_query_list
$memberdns->recursive_query_list( );
#Modifying recursive_query_list
$memberdns->recursive_query_list(["any"]);

#Creating TSIG key object
my $tsig_key2 = $session->gen_tsig_key();
my $key2 = Infoblox::DNS::TSIGKey->new(
   name => "rcl_key",
   key  => $tsig_key2
);
#Modifying recursive_query_list
$memberdns->recursive_query_list(["10.0.0.10", $key2,
"20.0.0.0/24","2010::256/64","!"."2011::23cb/64","!"."30.0.0.30", "!"."40.10.0.0/24"]);

#Use Grid::DNS level recursive_query_list
$memberdns->recursive_query_list(undef);
$memberdns->allow_recursive_query(undef);

#Use Infoblox::Grid::NamedACL
$memberdns->recursive_query_list($nacl);
```

resolver_query_timeout( )

Use this method to set or retrieve recursive query timeout for the member.

Setting this method to a defined value implicitly sets the override_resolver_query_timeout method to "true". Setting the parameter to undefined causes the appliance to use the Grid DNS default and automatically resets the override_resolver_query_timeout method to "false".

Note that when resolver_query_timeout is set to a defined value and override_resolver_query_timeout is set to "false", the last operation

takes precedence. Thus the sequence $object->resolver_query_timeout(29); $object->override_resolver_query_timeout("false"); will set override_resolver_query_timeout to "false", and the sequence $object->override_resolver_query_timeout("false"); $object->resolver_query_timeout(29); will result in override_resolver_query_timeout="true".

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

The valid value is an unsigned integer that contains the recursive query time (in seconds) for the resolver.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get resolver_query_timeout
my $resolver_query_timeout = $memberdns->resolver_query_timeout();
#Modify resolver_query_timeout
$memberdns->resolver_query_timeout(29);
```

response_rate_limiting( )

Use this method to set or retrieve response rate limiting settings for the member.

Setting this method to a defined value implicitly sets the override_response_rate_limiting method to "true". Setting the parameter to undefined causes the appliance to use the Grid DNS default and automatically resets the override_response_rate_limiting method to "false".

Note that when response_rate_limiting is set to a defined value and override_response_rate_limiting is set to "false", the last operation takes precedence. Thus the sequence $object->response_rate_limiting($response_rate_limiting); $object->override_response_rate_limiting("false"); will set override_response_rate_limiting to "false", and the sequence $object->override_response_rate_limiting("false"); $object->response_rate_limiting($response_rate_limiting); will result in override_response_rate_limiting="true".

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

The valid value is an Infoblox::Grid::DNS::ResponseRateLimiting object.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get response_rate_limiting
my $response_rate_limiting = $memberdns->response_rate_limiting();
#Modify response_rate_limiting
$memberdns->response_rate_limiting($response_rate_limiting);
```

recursive_resolver( )

Use this method to set or retrieve the recursive resolver for Member DNS.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

The valid value is 'BIND' or 'UNBOUND'. The default value is 'BIND'.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get recursive_resolver
my $recursive_resolver = $memberdns->recursive_resolver();

#Modify recursive_resolver
$memberdns->recursive_resolver('UNBOUND');
```

serial_query_rate( )

Use this method to set or retrieve the number of maximum concurrent SOA queries per second for the Member.

Parameter

Valid values are unsigned integer between 20 and 1000, inclusive. Default value is 20.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
# Get attribute value
my $value = $memberdns->serial_query_rate();
# Modify attribute value
$memberdns->serial_query_rate(30);
```

server_id_directive( )

Use this method to set or retrieve the value for the BIND and UNBOUND 'server-id' directive.

Setting this method to a defined value implicitly sets the override_server_id_directive method to "true". Setting the parameter to undefined causes the appliance to use the Grid DNS default and automatically resets the override_server_id_directive method to "false".

Note that when server_id_directive is set to a defined value and override_server_id_directive is set to "false", the last operation takes precedence. Thus the sequence $object->server_id_directive("HOSTNAME"); $object->override_server_id_directive("false"); will set override_server_id_directive to "false", and the sequence $object->override_server_id_directive("false"); $object->server_id_directive("true"); will result in override_server_id_directive="true".

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

The valid values are "NONE", "HOSTNAME" and "USER_DEFINED". The default value is "NONE".

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get server_id_directive
my $server_id_directive = $memberdns->server_id_directive();
#Modify server_id_directive
$memberdns->server_id_directive("HOSTNAME");
```

server_id_directive_string( )

Use this method to set or retrieve the value for the BIND and UNBOUND user-defined server ID for 'server-id' directive.

Note that to enable user-defined 'server-id' the server_id_directive should be set to "USER_DEFINED".

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

The valid value is a desired server ID in a string format. The default value is undefined.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get server_id_directive_string
my $server_id_directive_string = $memberdns->server_id_directive_string();
#Modify server_id_directive_string
$memberdns->server_id_directive_string("test-server-id-1");
```

skip_in_grid_rpz_queries( )

Use this method to set or retrieve the flag that indicates whether the RPZ rules are applied to queries originating from this member received by other members of the Grid or not.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

Specify 'true' to allow other members to recieve RPZ queries by other members and 'false' to forbid it.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get skip_in_grid_rpz_queries value
my $value = $object->skip_in_grid_rpz_queries();

#Modify skip_in_grid_rpz_queries value
$memberdns->skip_in_grid_rpz_queries('false');
```

sortlist( )

Use this method to create a sort list.

A sort list determines the order of addresses in responses made to DNS queries. This setting overrides grid sort list settings.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

Valid value is an array reference that contains Infoblox::DNS::Sortlist object(s).

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
my $sort1 = Infoblox::DNS::Sortlist->new (
                        "source_ipv4addr" => "1.2.3.4",
                        "match_list"      => [ "10.20.1.0/24" , "10.20.2.0/24"]
                );
$memberdns_object->sortlist( [ $sort1 ] );
```

store_locally( )

Use this method to set or retrieve the flag that indicates whether the storing of query capture reports on the appliance is enabled or disabled.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

Specify 'true' to enable storing of query capture reports on the appliance or 'false' to disable it. The default value is 'false'.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get store_locally value
my $store_locally = $object->store_locally();

#Modify store_locally value
$object->store_locally('true');
```

transfer_excluded_servers( )

Use this method to exclude specified DNS servers during zone transfers. This setting overrides grid zone transfer settings.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

The valid value is an array reference that contains IP addresses of the DNS server.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#This means that it will set zone transfer format to Many answers except these servers.
$memberdns->transfer_format("many_answers");
$memberdns->transfer_excluded_servers(["10.0.0.10", "20.0.0.0"]);
```

transfer_format( )

Use this method to set or retrieve the BIND format for a zone transfer. This provides tracking capabilities for single or multiple transfers and their associated servers. This setting overrides grid zone transfer format settings.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

Valid value is one of the following values:

`"Many Answers" (Secondaries run BIND 8/9)`: includes as many records as the packet size allows.

`"Many Answers" Except for these servers`: includes as many records as the packet size allows.

`"One Answer" (Secondaries run BIND 4)`: includes one record per packet.

`"One Answer" Except for these servers`: includes one record per packet.

If you specified servers in the transfer_excluded_servers method, the Many Answers or One Answer zone format (depending on transfer_format method setting) will allow zone transfers, except from the specified servers.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Getting transfer_format
$memberdns->transfer_format( );
#Modifying transfer_format
$memberdns->transfer_format("many_answers");
```

transfers_in( )

Use this method to set or retrieve the number of maximum concurrent transfers for the Member.

Parameter

Valid values are unsigned integer between 10 and 100, inclusive. Default value is 10.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
# Get attribute value
my $value = $memberdns->transfers_in();
# Modify attribute value
$memberdns->transfers_in(20);
```

transfers_out( )

Use this method to set or retrieve the number of maximum outbound concurrent zone transfers for the member.

Parameter

Valid values are unsigned integers between 1 and 100, inclusive. The default value is 10.

Specifying value in this parameter sets the override_transfers_out attribute to "true". Setting this parameter attribute to undefined causes a member to use the grid-level transfers_out value and automatically sets the override_transfers_out attribute to "false".

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
# Get attribute value
my $value = $memberdns->transfers_out();
# Modify attribute value
$memberdns->transfers_out(20);
```

transfers_per_ns( )

Use this method to set or retrieve the number of maximum concurrent transfers per member for the Member.

Parameter

Valid values are unsigned integer between 2 and 100, inclusive. Default value is 2.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
# Get attribute value
my $value = $memberdns->transfers_per_ns();
# Modify attribute value
$memberdns->transfers_per_ns(3);
```

unbound_logging_level( )

Use this method to set or retrieve the logging level for the unbound recursive resolver.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

The valid value is 'ERRORS_ONLY', 'OPERATIONS', 'DETAILED_OPERATIONS', 'QUERY', 'ALGORITHM' or 'CACHE_MISSES'.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get unbound_logging_level
my $unbound_logging_level = $memberdns->unbound_logging_level();

#Modify unbound_logging_level
$memberdns->unbound_logging_level('ALGORITHM');
```

use_lan_ipv6_port( )

Use this method to enable/disable DNS service on the IPv6 LAN port.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

Specify "true" to enable DNS service on the IPv6 LAN port or "false" to disable it. Default is "false"

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Getting use_lan_ipv6_port
$memberdns->use_lan_ipv6_port( );
#Modifying use_lan_ipv6_port
$memberdns->use_lan_ipv6_port("true");
```

use_lan2_port( )

Use this method to enable/disable DNS service on the LAN2 port.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

Specify "true" to enable DNS service on the LAN2 port or "false" to disable it. Default is "false"

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Getting use_lan2_port
$memberdns->use_lan2_port( );
#Modifying use_lan2_port
$memberdns->use_lan2_port("true");
```

use_lan2_ipv6_port( )

Use this method to enable/disable DNS service on the IPv6 LAN2 port.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

Specify "true" to enable DNS service on the IPv6 LAN2 port or "false" to disable it. Default is "false"

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Getting use_lan2_ipv6_port
$memberdns->use_lan2_ipv6_port( );
#Modifying use_lan2_ipv6_port
$memberdns->use_lan2_ipv6_port("true");
```

use_mgmt_port( )

Use this method to enable/disable DNS services on the MGMT port.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

Specify "true" to enable DNS services on the MGMT port or "false" to disable it. Default is "false"

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Getting use_mgmt_port
$memberdns->use_mgmt_port( );
#Modifying enable_dns
$memberdns->use_mgmt_port("true");
```

use_mgmt_ipv6_port( )

Use this method to enable/disable DNS services on the IPv6 MGMT port.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

Specify "true" to enable DNS services on the IPv6 MGMT port or "false" to disable it. Default is "false"

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Getting use_mgmt_ipv6_port
$memberdns->use_mgmt_ipv6_port( );
#Modifying enable_dns
$memberdns->use_mgmt_ipv6_port("true");
```

use_root_name_servers( )

Use this method to enable/disable custom or Internet root name servers. This setting overrides the grid root name server settings. Specify "False" to indicate that the member overrides the grid root server setting and uses Internet root name servers. Specify "True" to indicate that the member overrides the grid root server setting and uses custom root name servers. Specify "undef" to indicate that the member does not override the grid root server settings.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

Specify "true" to allow custom root name servers. Specify "false" to use Internet root name servers. Set it to undef to use the grid root name server settings.

Returns

If you specify a parameter, the method returns true when the modification succeeds, and returns false when the operation fails. If you do not specify a parameter, the method returns the attribute value.

Example

```
#Get use_root_name_servers
my $useRootNS = $memberdns->use_root_name_servers( );
#Modify use_root_name_servers
$memberdns->use_root_name_servers("false");
#Use grid root server settings
$memberdns->use_root_name_servers(undef);
```

rpz_disable_nsdname_nsip( )

Use this method to disable NSDNAME and NSIP resource records from RPZ feeds at member level.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

Specify "true" to enable the filter (to filter out NSDNAME and NSIP resource records). Specify "false" to disable the filter. The default value is "false".

Note that if both rpz_disable_nsdname_nsip and override_rpz_disable_nsdname_nsip are set to some value, the last operation takes precedence. Thus the sequence $grid_member->rpz_disable_nsdname_nsip("true"); $grid_member->override_rpz_disable_nsdname_nsip("false"); results in inheriting rpz_disable_nsdname_nsip value from the Grid. And the sequence $grid_member->override_rpz_disable_nsdname_nsip("false"); $grid_member->rpz_disable_nsdname_nsip("true"); sets override_rpz_disable_nsdname_nsip to "true".

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Getting rpz_disable_nsdname_nsip
$my $rpz_filter = $memberdns->rpz_disable_nsdname_nsip( );
#Modifying rpz_disable_nsdname_nsip
$memberdns->rpz_disable_nsdname_nsip("true");
```

rpz_drop_ip_rule_enabled( )

Use this method to set or retreive the flag that indicates whether the DNS server ignores RPZ-IP triggers with large prefix lengths or not.

Setting this method to a defined value implicitly sets the override_rpz_drop_ip_rule method to "true". Setting the parameter to undefined if rpz_drop_ip_rule_min_prefix_length_ipv4 and rpz_drop_ip_rule_min_prefix_length_ipv6 are undefined causes the appliance to use the Grid default and automatically resets the override_rpz_drop_ip_rule attribute to "false".

Note that when rpz_drop_ip_rule_enabled is set to a defined value and override_rpz_drop_ip_rule is set to "false", the last operation takes precedence. Thus the sequence $object->rpz_drop_ip_rule_enabled("true"); $object->override_rpz_drop_ip_rule("false"); will set override_rpz_drop_ip_rule to "false", and the sequence $object->override_rpz_drop_ip_rule("false"); $object->rpz_drop_ip_rule_enabled("true"); will result in override_rpz_drop_ip_rule="true".

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

Specify 'true' to enable ignoring of RPZ-IP triggers with large prefix lengths or 'false' to disable it. The default value is 'false'.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
# Get attribute value
my $value = $memberdns->rpz_drop_ip_rule_enabled();
# Modify attribute value
$memberdns->rpz_drop_ip_rule_enabled("true");
```

rpz_drop_ip_rule_min_prefix_length_ipv4( )

Use this method to set or retreive minimum prefix length for IPv4 RPZ-IP triggers.

Setting this method to a defined value implicitly sets the override_rpz_drop_ip_rule method to "true". Setting the parameter to undefined if rpz_drop_ip_rule_enabled and rpz_drop_ip_rule_min_prefix_length_ipv6 are undefined causes the appliance to use the Grid default and automatically resets the override_rpz_drop_ip_rule attribute to "false".

Note that when rpz_drop_ip_rule_min_prefix_length_ipv4 is set to a defined value and override_rpz_drop_ip_rule is set to "false", the last operation takes precedence. Thus the sequence $object->rpz_drop_ip_rule_min_prefix_length_ipv4("20"); $object->override_rpz_drop_ip_rule("false"); will set override_rpz_drop_ip_rule to "false", and the sequence $object->override_rpz_drop_ip_rule("false"); $object->rpz_drop_ip_rule_min_prefix_length_ipv4("20"); will result in override_rpz_drop_ip_rule="true".

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

The valid value is an unsigned integer between 1 and 31 that represents prefix length (in bits) for IPv4 RPZ-IP triggers.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
# Get attribute value
my $value = $member_dns->rpz_drop_ip_rule_min_prefix_length_ipv4();
# Modify attribute value
$member_dns->rpz_drop_ip_rule_min_prefix_length_ipv4("16");
```

rpz_drop_ip_rule_min_prefix_length_ipv6( )

Use this method to set or retreive minimum prefix length for IPv6 RPZ-IP triggers.

Setting this method to a defined value implicitly sets the override_rpz_drop_ip_rule method to "true". Setting the parameter to undefined if rpz_drop_ip_rule_enabled and rpz_drop_ip_rule_min_prefix_length_ipv4 are undefined causes the appliance to use the Grid default and automatically resets the override_rpz_drop_ip_rule attribute to "false".

Note that when rpz_drop_ip_rule_min_prefix_length_ipv6 is set to a defined value and override_rpz_drop_ip_rule is set to "false", the last operation takes precedence. Thus the sequence $object->rpz_drop_ip_rule_min_prefix_length_ipv6("20"); $object->override_rpz_drop_ip_rule("false"); will set override_rpz_drop_ip_rule to "false", and the sequence $object->override_rpz_drop_ip_rule("false"); $object->rpz_drop_ip_rule_min_prefix_length_ipv6("20"); will result in override_rpz_drop_ip_rule="true".

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

The valid value is an unsigned integer between 1 and 127 that represents prefix length (in bits) for IPv6 RPZ-IP triggers.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
# Get attribute value
my $value = $member_dns->rpz_drop_ip_rule_min_prefix_length_ipv6();
# Modify attribute value
$member_dns->rpz_drop_ip_rule_min_prefix_length_ipv6("120");
```

rpz_qname_wait_recurse( )

Use this method to enable or disable recursive RPZ lookups.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

Specify "true" to enable recursive RPZ lookups or "false" to disable them. The default value is "false".

Note that if both rpz_qname_wait_recurse and override_rpz_qname_wait_recurse are set to some values, the last operation takes precedence. Thus the sequence $grid_member->rpz_qname_wait_recurse("true"); $grid_member->override_rpz_qname_wait_recurse("false"); results in inheriting rpz_qname_wait_recurse value from the Grid. And the sequence $grid_member->override_rpz_qname_wait_recurse("false"); $grid_member->rpz_qname_wait_recurse("true"); sets override_rpz_qname_wait_recurse to "true".

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
# Get attribute value
my $value = $grid_member->rpz_qname_wait_recurse();
# Modify attribute value
$grid_member->rpz_qname_wait_recurse("true");
```

override_resolver_query_timeout( )

The override_resolver_query_timeout attribute controls whether the resolver_query_timeout method values of the member are used, instead of the Grid default.

The override_resolver_query_timeout attribute can be specified explicitly. It is also set implicitly when resolver_query_timeout is set to a defined value.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

Set the parameter to "true" to override the Grid-level setting for resolver_query_timeout. Set the parameter to "false" to inherit the Grid-level setting for resolver_query_timeout.

The default value of this parameter is "false".

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Getting override_resolver_query_timeout
my $override_resolver_query_timeout = $memberdns->override_resolver_query_timeout( );
#Modifying override_resolver_query_timeout
$memberdns->override_resolver_query_timeout("true");
```

override_response_rate_limiting( )

The override_response_rate_limiting attribute controls whether the response_rate_limiting method values of the member are used, instead of the Grid default.

The override_response_rate_limiting attribute can be specified explicitly. It is also set implicitly when response_rate_limiting is set to a defined value.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

Set the parameter to "true" to override the Grid-level setting for response_rate_limiting. Set the parameter to "false" to inherit the Grid-level setting for response_rate_limiting.

The default value of this parameter is "false".

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Getting override_response_rate_limiting
my $override_response_rate_limiting = $memberdns->override_response_rate_limiting( );
#Modifying override_response_rate_limiting
$memberdns->override_response_rate_limiting("true");
```

override_rpz_disable_nsdname_nsip( )

The override_rpz_disable_nsdname_nsip attribute controls whether the rpz_disable_nsdname_nsip method value for the member is used, instead of the Grid default.

The "override_rpz_disable_nsdname_nsip" attribute can be specified explicitly. It is also established implicitly as follows: "rpz_disable_nsdname_nsip" => "true"|"false" results in "override_rpz_disable_nsdname_nsip" => "true" "rpz_disable_nsdname_nsip" => undef results in "override_rpz_disable_nsdname_nsip" => "false" If "override_rpz_disable_nsdname_nsip" is specified explicitly as "true", while "rpz_disable_nsdname_nsip"=>undef, the explicit value takes precedence, and "override_rpz_disable_nsdname_nsip"=>"true" while "rpz_disable_nsdname_nsip" remains unchanged in the database.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

Set the parameter to "true" to override the Grid-level setting for rpz_disable_nsdname_nsip. Set the parameter to "false" to inherit the grid-level setting for rpz_disable_nsdname_nsip value.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Getting override_rpz_disable_nsdname_nsip
my $override_rpz_filter = $memberdns->override_rpz_disable_nsdname_nsip();
#Modyfying override_rpz_disable_nsdname_nsip
$memberdns->override_disable_nsdname_nsip("true");
```

override_rpz_drop_ip_rule( )

The override_rpz_drop_ip_rule attribute controls whether the rpz_drop_ip_rule_enabled, rpz_drop_ip_rule_min_prefix_length_ipv4 and rpz_drop_ip_rule_min_prefix_length_ipv6 method values of the member are used, instead of the Grid default.

The override_rpz_drop_ip_rule attribute can be specified explicitly. It is also set implicitly when one of rpz_drop_ip_rule_enabled, rpz_drop_ip_rule_min_prefix_length_ipv4 or rpz_drop_ip_rule_min_prefix_length_ipv6 methods is set to a defined value.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

Set the parameter to "true" to override the Grid-level setting for rpz_drop_ip_rule_enabled, rpz_drop_ip_rule_min_prefix_length_ipv4 and rpz_drop_ip_rule_min_prefix_length_ipv6 methods. Set the parameter to "false" to inherit the Grid-level setting for rpz_drop_ip_rule_enabled, rpz_drop_ip_rule_min_prefix_length_ipv4 and rpz_drop_ip_rule_min_prefix_length_ipv6 methods.

The default value of this parameter is "false".

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Getting override_rpz_drop_ip_rule
my $override_rpz_drop_ip_rule = $memberdns->override_rpz_drop_ip_rule( );
#Modifying override_rpz_drop_ip_rule
$memberdns->override_rpz_drop_ip_rule("true");
```

use_lan_port( )

Use this method to set or retrieve the status of the use of DNS services on the IPv4 LAN1 port.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

Specify true to enable DNS services on the IPv4 LAN1 port of false to disable it.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Getting use_lan_port
my $use_lan_port = $memberdns->use_lan_port();
#Modyfying use_lan_port
$memberdns->use_lan_port("true");
```

views( )

Use this method to specify or retrieve the view of this member.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value. The default value is the "default" view, which means the DNS member object is located under the default view.

Parameter

Valid value is an array reference that contains the names of the DNS views associated with the member. The names must be in string format.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get views
my $ref_views = $object->views();
#Modify DNS views, list of DNS view names in string format
$object->views(["view1","view2","view3"]);
```

SAMPLE CODE

The following sample code demonstrates the different functions that can be applied to an object,such as modify and get. This sample also includes error handling for the operations.

#Preparation prior to member DNS object operations

```
#PROGRAM STARTS: Include all the modules that will be used

use strict;
use Infoblox;

#Create a session to the Infoblox appliance
my $session = Infoblox::Session->new(
    master   => "192.168.1.2",
```

```
    username => "admin",
    password => "infoblox"
);
unless ($session) {
   die("Construct session failed: ",
       Infoblox::status_code() . ":" . Infoblox::status_detail());
}
print "Session created successfully\n";

# Get and modify the member DNS property object. Considering the id grid member named infoblox.localdomain
was previously created.
my $object = $session->get(
    object => "Infoblox::Grid::Member::DNS",
    name   => "infoblox.localdomain"
);

 unless ($object) {
      die("get member DNS failed: ",
      $session->status_code() . ":" . $session->status_detail());
  }
 print "member DNS get  successfull\n";

#Modifying the value of the specified object
$object->enable_dns("true");
$object->forward_only("false");
$object->minimal_resp("true");
$object->use_mgmt_port("true");
$object->notify_source_port(53);
$object->query_source_port(53);

 #Apply the changes
 $session->modify($object)
  or die("modify member DNS failed: ",
      $session->status_code() . ":" . $session->status_detail());

print "DNS member object modified successfully \n";

####PROGRAM ENDS####
```

AUTHOR

Infoblox Inc. http://www.infoblox.com/

SEE ALSO

Infoblox::DNS::View, Infoblox::Session, Infoblox::Grid::Member::DNS, Infoblox::DNS::DnssecTrustedKey, Infoblox::DNS::Ruleset, Infoblox::Session->get(), Infoblox::Session->modify(), Infoblox::Grid::NamedACL, Infoblox::Grid::DNS::AutoBlackHole, Infoblox::Grid::DNS::AttackMitigation, Infoblox::Grid::DNS::ResponseRateLimiting, Infoblox::Grid::Member::DNS::ViewAddressSetting, Infoblox::Grid::DNS::FileTransferSetting

COPYRIGHT

Copyright (c) 2017 Infoblox Inc.

Infoblox::Grid::Member::DNS::IP - a Grid member DNS IP.

- NAME
- DESCRIPTION
- CONSTRUCTOR
- SESSION METHODS
- MODULE METHODS
  - Infoblox::Grid::Member::DNS->dns_ip_list( )
- METHODS
  - member_virtual_ip( )
- AUTHOR
- SEE ALSO
- COPYRIGHT

NAME

Infoblox::Grid::Member::DNS::IP - a Grid member DNS IP.

DESCRIPTION

The DNS IP object is used to configure additional DNS-enabled interfaces for Grid member.

CONSTRUCTOR

```
my $object = Infoblox::Grid::Member::DNS::IP->new(
    member_virtual_ip => $Interface | $ipv4addr | $ipv6addr, #Required
);
```

SESSION METHODS

The object does not have any session methods.

MODULE METHODS

This section describes all the functions that you can apply to DNS IP object.

Infoblox::Grid::Member::DNS->dns_ip_list( )

Use this method to add or retrieve a list of additional IP addresses on which DNS is enabled for a grid member. See Infoblox::Grid::Member::DNS->dns_ip_list() for parameters and return values.

Example

```
#Get dns_ip_list value
my $value = $object->dns_ip_list();

#Modify dns_ip_list

#Construct Infoblox::Grid::Member::DNS::IP object

my $ip = Infoblox::Grid::Member::DNS::IP->new(
    member_virtual_ip => '10.0.0.10',
);

$object->dns_ip_list([$ip]);
```

METHODS

This section describes all the methods that you can use to configure and retrieve the attribute values of a DNS IP object.

member_virtual_ip( )

Use this method to set or retrieve the member's additional interface or interface IP address.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

The valid value is an Infoblox::Grid::Member::Interface object or IPv4 address or IPv6 address.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get member_virtual_ip value
my $member_virtual_ip = $object->member_virtual_ip();

#Modify member_virtual_ip value
$object->member_virtual_ip('10.0.0.10');
```

AUTHOR

Infoblox Inc. http://www.infoblox.com/

SEE ALSO

Infoblox::Grid::Member::DNS, Infoblox::Grid::Member::DNS->dns_ip_list(), Infoblox::Grid::Member::Interface,

---

COPYRIGHT

Copyright (c) 2016 Infoblox Inc.

IInfoblox::Grid::Member::DNS::ViewAddressSetting - The Grid member DNS view address settings.

- NAME
- DESCRIPTION
- CONSTRUCTOR
- MODULE METHODS
  - Infoblox::Grid::Member::DNS->dns_view_address_settings( )
- METHODS
  - enable_notify_source_port( )
  - enable_query_source_port( )
  - notify_delay( )
  - notify_source_port( )
  - notify_transfer_source_interface( )
  - query_source_interface( )
  - override_notify_delay( )
  - override_source_ports( )
  - query_source_port( )
  - view_name( )
- AUTHOR
- SEE ALSO
- COPYRIGHT

NAME

IInfoblox::Grid::Member::DNS::ViewAddressSetting - The Grid member DNS view address settings.

DESCRIPTION

The object provides information about the settings for the query source address and port, zone transfer request address, notify address and port as well as notify delay.

CONSTRUCTOR

```
 my $object = IInfoblox::Grid::Member::DNS::ViewAddressSetting->new(
     enable_notify_source_port        => 'true' | 'false',                          #Optional / Default is
'false'
     enable_query_source_port         => 'true' | 'false',                          #Optional / Default is
'false'
     notify_delay                     => $uint,                                     #Optional / Default is
5
     notify_source_port               => $uint,                                     #Optional / Default is
undefined
     notify_transfer_source_interface => 'VIP' | 'MGMT' | 'LAN2' | 'Any' | $ipaddr, #Optional / Default is
'Any'
     override_notify_delay            => 'true' | 'false',                          #Optional / Default is
'false'
     override_source_port             => 'true' | 'false',                          #Optional / Default is
'false'
     query_source_interface           => 'VIP' | 'MGMT' | 'LAN2' | 'Any' | $ipaddr, #Optional / Default is
'Any'
     query_source_port                => $uint,                                     #Optional / Default is
undefined
     view_name                        => $string,                                   #Required
 );
```

MODULE METHODS

You can apply the following functions to a DNS view address settings object.

Infoblox::Grid::Member::DNS->dns_view_address_settings( )

Use this method to set or retrieve the list of address settings for Grid member views. See Infoblox::Grid::Member::DNS/dns_view_address_settings() for parameters and return values.

Example

```
#Get dns_view_address_settings value
my $value = $object->dns_view_address_settings();

#Construct view address setting
```

```
my $view_address_setting = Infoblox::Grid::Member::DNS::ViewAddressSetting->new('view_name' =>
'view1');

#Modify dns_view_address_settings value
$object->dns_view_address_settings([$view_address_setting]);
```

METHODS

This section describes all the methods that you can use to set and retrieve the attribute values for a DNS view address settings object.

enable_notify_source_port( )

Use this method to set or retrieve the flag that indicates whether the notify source port setting is enabled.

Setting this method to a defined value implicitly sets the override_source_ports method to 'true'. Setting the parameter to undefined (if enable_query_source_port, query_source_port, notify_source_port are undefined) causes the appliance to use the Grid member default and automatically resets the override_source_ports method to 'false'.

Note that when enable_notify_source_port is set to a defined value and override_source_ports is set to 'false', the last operation takes precedence. Thus the sequence $object->enable_notify_source_port('true'); $object->override_source_ports('false'); will set override_source_ports to 'false', and the sequence $object->override_source_ports('false'); $object->enable_notify_source_port('true'); will result in override_source_ports='true'.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

Specify 'true' to enable notify source port setting, and 'false' to disable it.

Returns

When you specify a parameter and the modification succeeds, the method returns 'true'. When the operation fails, the method returns 'false'.

When you do not specify a parameter, the method returns the attribute value.

Example

```
#Get enable_notify_source_port value
my $value = $object->enable_notify_source_port();

#Modify enable_notify_source_port value
$object->enable_notify_source_port('true');
```

enable_query_source_port( )

Use this method to set or retrieve the flag that indicates whether the query source port setting is enabled.

Setting this method to a defined value implicitly sets the override_source_ports method to 'true'. Setting the parameter to undefined (if enable_notify_source_port, query_source_port, notify_source_port are undefined) causes the appliance to use the Grid member default and automatically resets the override_source_ports method to 'false'.

Note that when enable_query_source_port is set to a defined value and override_source_ports is set to 'false', the last operation takes precedence. Thus the sequence $object->enable_query_source_port('true'); $object->override_source_ports('false'); will set override_source_ports to 'false', and the sequence $object->override_source_ports('false'); $object->enable_query_source_port('true'); will result in override_source_ports='true'.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

Specify 'true' to enable query source port setting, and 'false' to disable it.

Returns

When you specify a parameter and the modification succeeds, the method returns 'true'. When the operation fails, the method returns 'false'.

When you do not specify a parameter, the method returns the attribute value.

Example

```
#Get enable_query_source_port value
my $value = $object->enable_query_source_port();

#Modify enable_query_source_port value
$object->enable_query_source_port('true');
```

notify_delay( )

Use this method to set or retrieve the minimum delay (in seconds) between a set of NOTIFY messages.

Setting this method to a defined value implicitly sets the override_notify_delay method to 'true'. Setting the parameter to undefined causes the appliance to use the Grid DNS default and automatically resets the override_notify_delay method to 'false'.

Note that when notify_delay is set to a defined value and override_notify_delay is set to 'false', the last operation takes precedence. Thus the sequence $object->notify_delay('true'); $object->override_notify_delay('false'); will set override_notify_delay to 'false', and the sequence $object->override_notify_delay('false'); $object->notify_delay('true'); will result in override_notify_delay='true'.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

The valid value is an unsigned integer between 5 and 86400. The default value is 5.

Returns

When you specify a parameter and the modification succeeds, the method returns 'true'. When the operation fails, the method returns 'false'.

When you do not specify a parameter, the method returns the attribute value.

Example

```
#Get notify_delay value
my $value = $object->notify_delay();

#Modify notify_delay value
$object->notify_delay(10000);
```

notify_source_port( )

Use this method to set or retrieve the notify source port number.

Setting this method to a defined value implicitly sets the override_source_ports method to 'true'. Setting the parameter to undefined (if enable_query_source_port, enable_notify_source_port, query_source_port are undefined) causes the appliance to use the Grid member default and automatically resets the override_source_ports method to 'false'.

Note that when notify_source_port is set to a defined value and override_source_ports is set to 'false', the last operation takes precedence. Thus the sequence $object->notify_source_port('true'); $object->override_source_ports('false'); will set override_source_ports to 'false', and the sequence $object->override_source_ports('false'); $object->notify_source_port('true'); will result in override_source_ports='true'.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

The valid value is an unsigned integer between 1 and 63999.

Returns

When you specify a parameter and the modification succeeds, the method returns 'true'. When the operation fails, the method returns 'false'.

When you do not specify a parameter, the method returns the attribute value.

Example

```
#Get notify_source_port value
my $value = $object->notify_source_port();

#Modify notify_source_port value
$object->notify_source_port(10000);
```

notify_transfer_source_interface( )

Use this method to specify which IP address is used as the source for notify and zone transfer operations.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

Valid values are 'VIP', 'MGMT', 'LAN2', 'Any' or an IP address. The default is 'VIP'.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get notify_transfer_source_interface value
my $value = $object->notify_transfer_source_interface();

#Modify notify_transfer_source_interface value
$object->notify_transfer_source_interface('Any');
```

query_source_interface( )

Use this method to specify which IP address is used as the source for query operations.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

Valid values are 'VIP', 'MGMT', 'LAN2', 'Any' or an IP address. Default is 'VIP'.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get query_source_interface value
my $value = $object->query_source_interface();

#Modify query_source_interface value
$object->query_source_interface('Any');
```

override_notify_delay( )

The override_notify_delay attribute controls whether the notify_delay method values of the member are used, instead of the Grid member default.

The override_notify_delay attribute can be specified explicitly. It is also set implicitly when notify_delay is set to a defined value.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

Set the parameter to 'true' to override the member-level setting for notify_delay. Set the parameter to 'false' to inherit the member-level setting for notify_delay.

The default value of this parameter is 'false'.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get override_notify_delay value
my $value = $object->override_notify_delay();

#Modify override_notify_delay value
$object->override_notify_delay('true');
```

override_source_ports( )

The override_source_ports attribute controls whether the values of enable_notify_source_port, enable_query_source_port, query_source_port and notify_source_port for the member are used, instead of the Grid member default.

The override_source_ports attribute can be specified explicitly. It is also set implicitly when any of enable_notify_source_port, enable_query_source_port, query_source_port, notify_source_port is set to a defined value.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

Set the parameter to true to override the member-level setting for enable_notify_source_port, enable_query_source_port,

query_source_port, notify_source_port. Set the parameter to 'false' to inherit the member-level setting for source_ports.

The default value of this parameter is 'false'.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get override_source_ports value
my $value = $object->override_source_ports();

#Modify override_source_ports value
$object->override_source_ports('true');
```

query_source_port( )

Use this method to set or retrieve the query source port number.

Setting this method to a defined value implicitly sets the override_source_ports method to 'true'. Setting the parameter to undefined (if enable_query_source_port, enable_notify_source_port, notify_source_port are undefined) causes the appliance to use the Grid member default and automatically resets the override_source_ports method to 'false'.

Note that when query_source_port is set to a defined value and override_source_ports is set to 'false', the last operation takes precedence. Thus the sequence $object->query_source_port('true'); $object->override_source_ports('false'); will set override_source_ports to 'false', and the sequence $object->override_source_ports('false'); $object->query_source_port('true'); will result in override_source_ports='true'.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

The valid value is an unsigned integer between 1 and 63999.

Returns

When you specify a parameter and the modification succeeds, the method returns 'true'. When the operation fails, the method returns 'false'.

When you do not specify a parameter, the method returns the attribute value.

Example

```
#Get query_source_port value
my $value = $object->query_source_port();

#Modify query_source_port value
$object->query_source_port(10000);
```

view_name( )

Use this method to set or retrieve the DNS view name.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

The valid value is a desired DNS view name in a string format.

Returns

When you specify a parameter and the modification succeeds, the method returns 'true'. When the operation fails, the method returns 'false'.

When you do not specify a parameter, the method returns the attribute value.

Example

```
#Get view_name value
my $value = $object->view_name();

#Modify view_name value
$object->view_name('view1');
```

AUTHOR

Infoblox Inc. http://www.infoblox.com/

SEE ALSO

Infoblox::Grid::Member::DNS, Infoblox::Grid::Member::DNS/dns_view_address_settings(),

---

COPYRIGHT

Copyright (c) 2017 Infoblox Inc.

Infoblox::Grid::Member::FileDistribution - Grid Member FileDistribution object.

- NAME
- DESCRIPTION
- CONSTRUCTOR
- SESSION METHODS
  - Infoblox::Session->get( )
  - Infoblox::Session->modify( )
- METHODS
  - allow_uploads( )
  - name( )
  - enable_tftp( )
  - tftp_port( )
  - tftp_allow_access( )
  - enable_http( )
  - enable_http_acl( )
  - http_allow_access( )
  - enable_ftp( )
  - ftp_port( )
  - ftp_allow_access( )
  - ftp_login_banner( )
  - ftp_enable_passive_mode( )
  - ftp_enable_file_listing( )
  - override_allow_uploads( )
- SAMPLE CODE
- AUTHOR
- SEE ALSO
- COPYRIGHT

NAME

Infoblox::Grid::Member::FileDistribution - Grid Member FileDistribution object.

DESCRIPTION

You can configure the file distribution properties of a grid member. You can enable and disable TFTP, HTTP, and FTP services, and set their operational parameters.

CONSTRUCTOR

The Infoblox::Grid::Member::FileDistribution object is automatically generated upon the successful installation of the Infoblox appliance. It does not require manual construction.

SESSION METHODS

This section describes all the methods in the Infoblox::Session module that you can apply to a grid member FileDistribution object.

Infoblox::Session->get( )

Use this method to retrieve all the matching objects from the Infoblox appliance. See Infoblox::Session->get() for parameters and return values.

Key Reference

Apply the following attributes to get a specific grid member FileDistribution object:

name - Required. Host name in string format

Example

```
my $object   = $session->get(
   object    => "Infoblox::Grid::Member::FileDistribution",
   name      => "infoblox.localdomain",
    );
```

Infoblox::Session->modify( )

Use this method to modify an object in the Infoblox appliance. See Infoblox::Session->modify() for parameters and return values.

If you have applied an access control list to an object, you must restart services after you update the access control list for the changes to take effect. See Infoblox::Session->restart() method.

Example

```
#Use method to modify the enable_tftp Property.
$object->enable_tftp("true");
#Submit modification
my $response = $session->modify( $object );
```

METHODS

This section describes all the methods that you can use to configure and retrieve the attribute values of the file distribution properties of a grid member.

allow_uploads( )

Use this method to enable or disable file uploads for a member.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Setting this method to a defined value implicitly sets the override_allow_uploads method to "true". Setting the parameter to undefined causes the appliance to use the grid default and automatically resets the override_allow_uploads attribute to "false".

Note that when allow_uploads is set to a defined value and override_dallow_uploads is set to "false", the last operation takes precedence. Thus the sequence $object->allow_uploads("true"); $object->override_allow_uploads("false"); will set override_allow_uploads to "false", and the sequence $object->override_allow_uploads("false"); $object->allow_uploads("true"); will result in override_allow_uploads="true".

Parameter

Specify "true" to enable file uploads or "false" to disable it. The default value is "false".

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Getting allow_uploads
$file_distribution->allow_uploads( );
#Modifying allow_uploads
$file_distribution->allow_uploads("true");
```

name( )

Use this method to retrieve the grid name. This is a required argument.

Parameter

None.

Returns

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get name
my $name = $file_distribution->name();
```

enable_tftp( )

Use this method to enable or disable the TFTP service of a member.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

Specify "true" to enable the TFTP service or "false" to disable it. The default value is "false".

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Getting enable_tftp
$file_distribution->enable_tftp( );
#Modifying enable_tftp
$file_distribution->enable_tftp("true");
```

tftp_port( )

Use this method to specify the port to listen on for the TFTP file distribution service.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

Port number for TFTP file distribution service. Valid values are between 1 and 63999. The default is 69.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Getting tftp_port
$file_distribution->tftp_port( );
#Modifying tftp_port
$file_distribution->tftp_port("1069");
```

tftp_allow_access( )

Use this method to allow TFTP file distribution access from specified IPv4 addresses and networks.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

Valid value is an Infoblox::Grid::NamedACL object or an array reference that contains IPv4 addresses and networks. To reject TFTP file distribution access, add '!' as a prefix to IP addresses and networks. Specify "any" to allow any address or set to an empty array to clear the list.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails. If you did not specify a parameter, the method returns the attribute value.

Example

```
#Getting tftp_allow_access
my $file_distribution = $file_distribution->tftp_allow_access( );
#Modifying tftp_allow_access
$file_distribution->tftp_allow_access(["any"]);
#Modifying tftp_allow_access
$file_distribution->tftp_allow_access(["10.0.0.10", "20.0.0.0/24", "!"."30.0.0.30",
"!"."40.10.0.0/24"]);
#Modifying tftp_allow_access
$file_distribution->tftp_allow_access([]);
#Use Infoblox::Grid::NamedACL
$file_distribution->tftp_allow_access($nacl);
```

enable_http( )

Use this method to enable or disable the HTTP file distribution service of a member.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

Specify "true" to enable HTTP file distribution service or "false" to disable it. The default value is "false".

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Getting enable_http
$file_distribution->enable_http( );
#Modifying enable_http
$file_distribution->enable_http("true");
```

enable_http_acl( )

Use this method to enable or disable the ACLs for the HTTP file distribution service of a member.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

Specify "true" to enable ACLs for the HTTP file distribution service or "false" to disable it. The default value is "false".

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Getting enable_http_acl
$file_distribution->enable_http_acl( );
#Modifying enable_http_acl
$file_distribution->enable_http_acl("true");
```

http_allow_access( )

Use this method to allow HTTP file distribution access from specified IPv4 addresses and networks. enable_http and enable_http_acl must be set to 'true' in order for this http_allow_access to be usable. Set to an empty array to clear the list.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

Valid value is an Infoblox::Grid::NamedACL object or an array reference that contains IPv4 addresses and networks.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails. If you did not specify a parameter, the method returns the attribute value.

Example

```
#Getting http_allow_access
my $file_distribution = $file_distribution->http_allow_access( );
#Modifying http_allow_access
$file_distribution->http_allow_access(["10.0.0.10", "20.0.0.0/24", "30.0.0.30", "40.10.0.0/24"]);
#Modifying http_allow_access
$file_distribution->http_allow_access([]);
#Use Infoblox::Grid::NamedACL
$file_distribution->http_allow_access($nacl);
```

enable_ftp( )

Use this method to enable or disable the FTP service of a member.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

Specify "true" to enable the FTP service or "false" to disable it. The default value is "false".

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Getting enable_ftp
$file_distribution->enable_ftp( );
#Modifying enable_ftp
$file_distribution->enable_ftp("true");
```

ftp_port( )

Use this method to specify the port to listen on for the FTP file distribution service.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

Port number for the FTP file distribution service. Valid values are between 1 and 63999. The default is 21.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Getting ftp_port
$file_distribution->ftp_port( );
#Modifying ftp_port
$file_distribution->ftp_port("1022");
```

ftp_allow_access( )

Use this method to allow FTP file distribution access from specified IPv4 addresses and networks.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

Valid value is an Infoblox::Grid::NamedACL object or an array reference that contains IPv4 addresses and networks. To reject FTP file distribution access, add '!' as a prefix to IP addresses and networks. Specify "any" to allow any address or set to an empty array to clear the list.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails. If you did not specify a parameter, the method returns the attribute value.

Example

```
#Getting ftp_allow_access
my $file_distribution = $file_distribution->ftp_allow_access( );
#Modifying ftp_allow_access
$file_distribution->ftp_allow_access(["any"]);
#Modifying ftp_allow_access
$file_distribution->ftp_allow_access(["10.0.0.10", "20.0.0.0/24", "!"."30.0.0.30",
"!"."40.10.0.0/24"]);
#Modifying ftp_allow_access
$file_distribution->ftp_allow_access([]);
#Use Infoblox::Grid::NamedACL
$file_distribution->ftp_allow_access($nacl);
```

ftp_login_banner( )

Use this method to specify the login banner for the FTP file distribution service.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

The login banner for the FTP file distribution service. Valid values are strings of up to 256 characters.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Getting ftp_login_banner
$file_distribution->ftp_login_banner( );
#Modifying ftp_login_banner
$file_distribution->ftp_login_banner("This is my new FTP login banner");
```

ftp_enable_passive_mode( )

Use this method to enable or disable the FTP service in passive mode.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

Specify "true" to enable the FTP service in passive mode or "false" to disable it. The default value is "false".

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Getting ftp_enable_passive_mode
$file_distribution->ftp_enable_passive_mode( );
#Modifying ftp_enable_passive_mode
$file_distribution->ftp_enable_passive_mode("true");
```

ftp_enable_file_listing( )

Use this method to enable or disable file listing for the FTP service.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

Specify "true" to enable file listing or "false" to disable it. The default value is "false".

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Getting ftp_enable_file_listing
$file_distribution->ftp_enable_file_listing( );
#Modifying ftp_enable_file_listing
$file_distribution->ftp_enable_file_listing("true");
```

override_allow_uploads( )

The override_allow_uploads attribute controls whether the allow_uploads value in the member is used, instead of the grid default.

The override_allow_uploads attribute can be specified explicitly. It is also set implicitly when allow_uploads is set to a defined value.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

Specify "true" to set the override_allow_uploads flag or "false" to deactivate/unset it. The default value is "false".

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get override_allow_uploads for the member
my $type = $file_distribution->override_allow_uploads();
#Modify override_allow_uploads for the member
$file_distribution->override_allow_uploads("true");
```

SAMPLE CODE

The following sample code demonstrates the different functions that can be applied to an object, such as modify and get. This sample also includes error handling for the operations.

#Preparation prior to member FileDistribution object operations

```
#PROGRAM STARTS: Include all the modules that will be used
```

```
use strict;
use Infoblox;

#Create a session to the Infoblox appliance
my $session = Infoblox::Session->new(
    master   => "192.168.1.2",
    username => "admin",
    password => "infoblox"
);
unless ($session) {
   die("Construct session failed: ",
       Infoblox::status_code() . ":" . Infoblox::status_detail());
}
print "Session created successfully\n";
```

#Get and modify a Member FileDistribution object

```
# Get and modify the member FileDistribution property object. Note that the grid member named
infoblox.localdomain was previously created.
my $object = $session->get(
    object => "Infoblox::Grid::Member::FileDistribution",
    name   => "infoblox.localdomain"
);

 unless ($object) {
      die("get member FileDistribution failed: ",
      $session->status_code() . ":" . $session->status_detail());
  }
 print "member FileDistribution get successful\n";

#Modifying the value of the specified object
$object->enable_tftp("true");
$object->tftp_allow_access(["any"]);
$object->enable_http("true");
$object->enable_http_acl("true");
$object->http_allow_access(["10.0.0.10", "20.0.0.0/24"]);
$object->enable_ftp("true");
$object->ftp_allow_access(["10.0.0.10", "20.0.0.0/24", "!"."30.0.0.30", "!"."40.10.0.0/24"]);
$object->tftp_port(1069);
$object->ftp_port(1022);
$object->ftp_login_banner("My new banner");
$object->ftp_enable_passive_mode("true");

 #Apply the changes
 $session->modify($object)
  or die("modify member FileDistribution failed: ",
      $session->status_code() . ":" . $session->status_detail());

print "FileDistribution member object modified successfully \n";

####PROGRAM ENDS####
```

AUTHOR

Infoblox Inc. http://www.infoblox.com/

SEE ALSO

Infoblox::Grid::FileDistributionDir, Infoblox::Session, Infoblox::Session->get(), Infoblox::Session->modify(), Infoblox::Grid::NamedACL

COPYRIGHT

Copyright (c) 2017 Infoblox Inc.

Infoblox::Grid::Member::Interface - Interface member object.

- NAME
- DESCRIPTION
- CONSTRUCTOR
- MODULE METHODS
  - Infoblox::Grid::Member->additional_ip_list( )
  - Infoblox::Grid::Member::DNS->additional_ip_list( )
- METHODS
  - anycast( )
  - cidr( )
  - comment( )
  - dscp( )
  - enable_bgp( )
  - enable_ospf( )
  - enable_ipv6_auto_config( )
  - gateway( )
  - interface( )
  - ipv4addr( )
  - ipv6addr( )
  - subnet_mask( )
  - override_dscp( )
  - vlan_id( )
- SAMPLE CODE
- AUTHOR
- SEE ALSO
- COPYRIGHT

NAME

Infoblox::Grid::Member::Interface - Interface member object.

DESCRIPTION

An Interface member object represents additional interface information that can be used at the member level. A single IPv6 address may be added to each member by using an Interface object.

CONSTRUCTOR

```
my  $interface = Infoblox::Grid::Member::Interface->new(
    anycast     => "true" | "false",                          #Optional / Default is "false"
    cidr        => $prefix,                                   #Optional / Default is undefined
    comment     => $string,                                   #Optional / Default is undefined
    enable_bgp  => "true" | "false",                          #Optional / Default is "false"
    enable_ospf => "true" | "false",                          #Optional / Default is "false"
    enable_ipv6_auto_config => "true" | "false",              #Optional / Default is "false"
    gateway     => $ipv4addr | $ipv6addr,                     #Optional / Default is undefined
    interface   => "LAN_HA" | "LOOPBACK" | "MGMT" | "LAN2",   #Optional / Default is "LOOPBACK"
    ipv4addr    => $ipv4addr,                                 #Optional / Default is undefined
    ipv6addr    => $ipv6addr,                                 #Optional / Default is undefined
    subnet_mask => $netmask                                   #Optional / Default is undefined
    dscp          => $uint,                                   #Optional / Default is undefined
    override_dscp => "true" | "false",                        #Optional / Default is "false"
    vlan_id       => $uint,                                   #Optional / Default is undefined
);
```

MODULE METHODS

The following functions are available to be applied to an Interface object.

Infoblox::Grid::Member->additional_ip_list( )

Use this function to specify Interface for the Grid Member. See Infoblox::Grid::Member->additional_ip_list() for parameters and return values.

Example 1

```
#Construct an Interface object
my $interface1 = Infoblox::Grid::Member::Interface->new(
```

```
    ipv4addr    => "10.2.3.4",
    anycast     => "true",
    interface   => "LOOPBACK",
    subnet_mask => "255.255.255.255",
    comment     => "this is a loopback interface"
);

# Configure Interface on the Grid Member object
my $response = $Grid_Member->additional_ip_list([$interface1]);
```

Example 2

```
#Construct an IPv6 Interface object for member addition
my $interface6 = Infoblox::Grid::Member::Interface->new(
    ipv6addr    => "2001::1234",
    gateway     => "2001::1111",
    anycast     => "false",
    interface   => "LAN_HA",
    cidr        => "64",
    comment     => "this is an IPv6 address"
);

# Configure Interface on the Grid Member object
my $response = $Grid_Member->additional_ip_list([$interface6]);
```

Example 3

```
#Construct an IPv6 Interface object for member addition with IPv6 auto-configuration turned on
my $interface6 = Infoblox::Grid::Member::Interface->new(
    ipv6addr    => "2001::1234",
    enable_ipv6_auto_config  => "true",
    anycast     => "false",
    interface   => "LAN_HA",
    cidr        => "64",
    comment     => "this is an IPv6 address with auto-configuration turned on"
);

# Configure Interface on the Grid Member object
my $response = $Grid_Member->additional_ip_list([$interface6]);
```

Infoblox::Grid::Member::DNS->additional_ip_list( )

Use this function to specify Interface for the Grid Member DNS. See Infoblox::Grid::Member::DNS->additional_ip_list() for parameters and return values.

Example

```
#Construct an Interface object
my $interface2 = Infoblox::Grid::Member::Interface->new(
    ipv4addr    => "11.22.33.44",
    anycast     => "true",
    interface   => "LOOPBACK",
    subnet_mask => "255.0.0.0",
    comment     => "this is a loopback interface"
);

# Configure Interface on the Grid Member DNS object
my $response = $Grid_Member_DNS->additional_ip_list([$interface2]);
```

METHODS

This section describes all the methods that can be used to configure and retrieve the attribute values of an Interface object.

anycast( )

Use this method to enable/disable anycast of the Interface object.

When anycast is enabled the Interface is served for anycast usage.

Parameter

Specify "true" to enable anycast or "false" to disable it. Default value is "false".

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

Example

```
#Get anycast
my $anycast = $interface->anycast();
#Modify anycast
$interface->anycast("false");
```

cidr( )

Use this method to set or retrieve the CIDR netmask format of the Inteface object. Required only when ipv6addr is specified.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

Specify the netmask in CIDR format (example: "8" should be specified for netmask 255.0.0.0).

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get cidr
my $cidr = $interface->cidr();
#Modify cidr
$interface->cidr("16");
```

comment( )

Use this method to set or retrieve a descriptive comment of Interface object.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

Desired comment in string format with a maximum of 512 bytes. Default value is undefined.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get comment
my $comment = $interface->comment();
#Modify comment
$interface->comment("Modifying the Interface comment");
```

dscp( )

Use this method to set or retrieve the DSCP value of the Interface object.

Setting this method to a defined value implicitly sets the override_dscp method to "true". Setting the parameter to undefined causes the appliance to use the member default and automatically resets the override_dscp attribute to "false".

Note that when dscp is set to a defined value and override_dscp is set to "false", the last operation takes precedence. Thus the sequence $object->dscp(11); $object->override_dscp("false"); will set override_dscp to "false", and the sequence $object->override_dscp("false"); $object->dscp(11); will result in override_dscp="true".

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

Valid values are integers between 0 and 63, inclusive. The default value is undefined.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get attribute value
```

```
my $value = $interface->dscp();
#Modify attribute value
$interface->dscp(11);
```

enable_bgp( )

Use this method to set or retrieve the BGP advertisement setting for this interface.

Parameter

Specify "true" to enable BGP advertisements on this interface or "false" to disable them. Enabling advertisements requires the anycast attribute to be set to "true".

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

Example

```
#Get enable_bgp
my $enable_bgp = $interface->enable_bgp();
#Modify enable_bgp
$interface->enable_bgp("true");
```

enable_ospf( )

Use this method to set or retrieve the OSPF advertisement setting for this interface.

Parameter

Specify "true" to enable OSPF advertisement on this interface or "false" to disable it. Enabling advertisement requires the anycast attribute to be set to "true".

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

Example

```
#Get enable_ospf
my $enable_ospf = $interface->enable_ospf();
#Modify enable_ospf
$interface->enable_ospf("true");
```

enable_ipv6_auto_config( )

Use this method to enable/disable IPv6 auto-configuration of the Interface object.

When IPv6 auto-configuration is enabled, a gateway is not needed since it will be automatically discovered.

Parameter

Specify "true" to enable IPv6 auto-configuration or "false" to disable it. Default value is "false".

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

Example

```
#Get IPv6 auto-configuration
my $ipv6_autoconfig_enabled = $interface->enable_ipv6_auto_config();
#Modify IPv6 auto-configuration
$interface->enable_ipv6_auto_config("true");
```

gateway( )

Use this method to set or retrieve the gateway of the Interface object.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

Specify the gateway of the Interface in IPv4 address format or IPV6 address format. An IPv4 address is a 32-bit number in dotted decimal notation. It consists of four 8-bit groups of decimal digits separated by decimal points (example: 192.168.1.2). An IPv6 address is a 128-bit number in colon hexadecimal notation. It consists of eight 16-bit groups of hexadecimal digits separated by colons (example: 12ab:0000:0000:0123:4567:89ab:0000:cdef).

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get gateway
my $gateway = $interface->gateway();
#Modify gateway
$interface->gateway("10.1.1.1");
```

interface( )

Use this method to set or retrieve the interface type of this Interface object.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

Specify the interface type in string format. Valid values are "LOOPBACK", "LAN_HA", "MGMT" or "LAN2". Default value is "LOOPBACK".

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get interface
my $interface = $interface->interface();
#Modify interface
$interface->interface("LAN2");
```

ipv4addr( )

Use this method to set or retrieve the IPV4 address used for the Interface object.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

An IPv4 address is a 32-bit number in dotted decimal notation. It consists of four 8-bit groups of decimal digits separated by decimal points (example: 192.168.1.2).

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get ipv4addr
my $ipv4addr = $interface->ipv4addr();
#Modify ipv4addr
$interface->ipv4addr("2.2.2.2");
```

ipv6addr( )

Use this method to set or retrieve the IPV6 address used for the Interface object.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

An IPv6 address is a 128-bit number in colon hexadecimal notation. It consists of eight 16-bit groups of hexadecimal digits separated by colons (example: 12ab:0000:0000:0123:4567:89ab:0000:cdef). IPv6 gateway, and an CIDR are used in conjunctions with ipv6addr ( ).

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get ipv6addr
my $ipv6addr = $interface->ipv6addr();
#Modify ipv6addr
$interface->ipv6addr("aaaa:1111:bbbb:2222:cccc:3333:dddd:5555");
```

subnet_mask( )

Use this method to set or retrieve the subnet mask of the Interface object.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

Specify the subnet mask of the Interface in IPv4 address format. An IPv4 address is a 32-bit number in dotted decimal notation. It consists of four 8-bit groups of decimal digits separated by decimal points (example: 255.0.0.0).

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get subnet_mask
my $ipv4_subnet_mask = $interface->subnet_mask();
#Modify subnet_mask
$interface->subnet_mask("255.255.255.255");
```

override_dscp( )

The override_dscp attribute contols whether the DSCP value is used, instead of the Member default.

The override_dscp attribute can be specified explicitly. It is also set implicitly when DSCP is set to a defined value.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

Specify "true" to set the override_dscp flag or "false" to deactivate/unset it. The default value is "false".

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get override_dscp from the member
my $override = $interface->override_dscp();
#Modify override_dscp
$interface->override_dscp("true");
```

vlan_id( )

Use this method to set or retrieve the VLAN ID of the Interface object.

Parameter

Valid values are unsigned integers between 1 and 4095, inclusive. The default value is undefined.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get attribute value
my $attr = $interface->vlan_id();
#Modify attribute value
$interface->vlan_id(10);
```

SAMPLE CODE

The following sample code demonstrates the different functions that can be applied to an Interface object such as add, modify, and remove. Also, this sample includes error handling for the operations.

#Preparation prior to Interface object insertion

```
use strict;
use warnings;
use Infoblox;

my $host_name = "infoblox.localdomain";
my $host_ip   = "192.168.1.2";
#Creating a session to Infoblox appliance.
my $session = Infoblox::Session->new(
    master   => $host_ip,
    username => "admin",
    password => "infoblox"
);

unless ($session) {
    die(qq(constructor for session failed: ),
        join(":", Infoblox::status_code(), Infoblox::status_detail()));
}
print "Session created successfully \n";
```

#Create an Interface object

```
#Creating Interface object.
my $interface1 = Infoblox::Grid::Member::Interface->new(
    ipv4addr     => "10.2.3.4",
    anycast      => "false",
    interface    => "LOOPBACK",
    subnet_mask  => "255.255.255.255",
   ) or die(qq(Constructor for interface1 failed: ),
   join(":", Infoblox::status_code(), Infoblox::status_detail()));
print "Interface object constructed successfully \n";
```

#Add Interface object to Grid Member

```
#Getting the member object from appliance through session.
my @result = $session->get(
    object => "Infoblox::Grid::Member",
    name   => $host_name,
    ) or die(qq(Get Grid Member failed: ),
    join(":", $session->status_code(), $session->status_detail()));
print "Get Grid Member successful\n";

my $result = $result[0];
if ($result) {
    #Modifying the value of the Interface method from the member object.
    $result->additional_ip_list([$interface1]);
    #Applying the changes to appliance through session.
    $session->modify($result)
        or die(qq(Modify Grid Member failed: ),
        join(":", $session->status_code(), $session->status_detail()));
}
print "Interface object added to Grid Member successfully\n";
```

#Modify Interface object

```
#Modifying the comment of the Interface object.
$interface1->comment("Modified Comment");

#Apply changes to the Grid Member object.
$result->additional_ip_list([$interface1]);

#Update Grid Member object through the Infoblox session.
$session->modify($result)
  or die("Modify Grid Member object failed: ",
  $session->status_code() . ":" . $session->status_detail());
print "Grid Member object with modified Interface updated to Infoblox appliance successfully\n";
```

#Remove Interface object

```
#Remove the Interface from the Grid Member object.
$result->additional_ip_list([]);

#Update Grid Member object through the Infoblox session.
```

```
$session->modify($result)
  or die("Modify Grid Member object failed: ",
  $session->status_code() . ":" . $session->status_detail());
print "Removed Interface object from Grid Member successfully\n";

####PROGRAM ENDS####
```

AUTHOR

Infoblox Inc. http://www.infoblox.com/

SEE ALSO

Infoblox::Grid::Member, Infoblox::Session , Infoblox::Grid::Member::DNS

COPYRIGHT

Copyright (c) 2017 Infoblox Inc.

Infoblox::Grid::Member::IPv6StaticRoute - Static route member object

- NAME
- DESCRIPTION
- CONSTRUCTOR
- MODULE METHODS
  - Infoblox::Grid::Member->ipv6_static_routes( )
- METHODS
  - address( )
  - cidr( )
  - gateway( )
- AUTHOR
- SEE ALSO
- COPYRIGHT

NAME

Infoblox::Grid::Member::IPv6StaticRoute - Static route member object

DESCRIPTION

An IPv6StaticRoute object provides IPv6 static route information for a grid member.

CONSTRUCTOR

```
my $route = Infoblox::Grid::Member::IPv6StaticRoute->new(
        "address" => $ip_address,        #Required
        "gateway" => $ip_address,        #Required
        "cidr"    => $cidr,                      #Required
);
```

MODULE METHODS

The following functions can be applied to an IPv6staticroute object.

Infoblox::Grid::Member->ipv6_static_routes( )

Use this function to specify an IPv6staticroute object for the grid member. See Infoblox::Grid::Member->ipv6_static_routes() for parameters and return values.

Example

```
#Get static routes list
my $ipv6_static_routes = $grid_member->ipv6_static_routes();
#Modify static routes list
my $route = Infoblox::Grid::Member::IPv6StaticRoute->new(
        "address" => "2002::5",
        "cidr"    => 64
        "gateway" => "2002::1",
);
$grid_member->ipv6_static_routes([$route]);
```

METHODS

This section describes all the methods that can be used to configure and retrieve the attribute values of an IPv6 static route object.

address( )

Use this method to set or retrieve the IPv6 address of an IPv6 static route object.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

An IPv6 address is a 128-bit number represented as eight groups of four hexadecimal digits (i.e. two octets), each group separated by a `colon(:)` (example: 2001:0db8:85a3:0000:0000:8a2e:0370:7334). Leading zeroes and groups of consecutive zeros within the value can be omitted to produce a more compact representation (example: 2001::1, ::1).

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get the address
my $address= $route->address();
#Modify the address
$route->address("2002::5");
```

cidr( )

Use this method to set or retrieve the cidr of the IPv6 static route object.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

Specify the netmask in CIDR format.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get the cidr
my $cidr= $route->cidr();
#Modify the cidr
$route->cidr(64);
```

gateway( )

Use this method to set or retrieve the gateway of the IPv6 static route object.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

An IPv6 address is a 128-bit number represented as eight groups of four hexadecimal digits (i.e. two octets), each group separated by a `colon(:)` (example: 2001:0db8:85a3:0000:0000:8a2e:0370:7334). Leading zeroes and groups of consecutive zeros within the value can be omitted to produce a more compact representation (example: 2001::1, ::1).

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get the gateway address
my $gateway= $route->gateway();
#Modify the gateway address
$route->gateway("2002::1");
```

AUTHOR

Infoblox Inc. http://www.infoblox.com/

SEE ALSO

Infoblox::Grid::Member

COPYRIGHT

Copyright (c) 2017 Infoblox Inc.

Infoblox::Grid::Member::License - License object.

- NAME
- DESCRIPTION
- CONSTRUCTOR
- SESSION METHODS
  - Infoblox::Session->add( )
  - Infoblox::Session->get( )
  - Infoblox::Session->search( )
  - Infoblox::Session->remove( )
  - Infoblox::Grid::Member->node1_licenses( ) / Infoblox::Grid::Member->node2_licenses( )
- METHODS
  - expiration( )
  - expiration_status( )
  - key( )
  - kind( )
  - hwid( )
  - limit( )
  - limit_context( )
  - type( )
- SAMPLE CODE
- AUTHOR
- SEE ALSO
- COPYRIGHT

NAME

Infoblox::Grid::Member::License - License object.

DESCRIPTION

Licenses are pre-installed on an Infoblox device according to the software packages ordered at the time of purchase. You can upgrade an Infoblox device with the Keystone license. Contact Infoblox Technical Support to get the Keystone license and install it on the device.

CONSTRUCTOR

```
my $license = Infoblox::Grid::Member::License->new(
    key  => $string,    #Required
);
```

SESSION METHODS

This section describes all the methods in the Infoblox::Session module that you can apply to a License object.

Infoblox::Session->add( )

Use this method to add a License object to the Infoblox device. See Infoblox::Session->add() for parameters and return values.

Example

```
#Construct an object
my $license = Infoblox::Grid::Member::License->new(
    key  => "EQAAAIMhrdkGjSnnmBlgQSA14b16"
);

# Submit for addition
my $response = $session->add( $license );
```

Infoblox::Session->get( )

Use this method to retrieve the existing objects from an Infoblox appliance. See Infoblox::Session->get() for parameters and return values.

Key Reference

```
Apply the following attributes to search the License object(s):

    "type"  - Optional. The license type.
    "limit" - Optional. The limitation of license.
    "key"   - Optional. The license string.
```

Example

```
my $retrieved_objs = $session->get(object => "Infoblox::Grid::Member::License");
```

Infoblox::Session->search( )

Use this method to retrieve the existing objects from an Infoblox appliance. See Infoblox::Session->search() for parameters and return values.

Key Reference

```
Apply the following attributes to search the License object(s):

    "type"  - Optional. The license type.
    "limit" - Optional. The limitation of license.
    "key"   - Optional. The license string.
```

Example

```
my @retrieved_objs = $session->search(
    object => "Infoblox::Grid::Member::License",
    type   => "DNS"
);
```

Infoblox::Session->remove( )

Use this method to remove a License object from the Infoblox device. See Infoblox::Session->remove() for parameters and return values.

Example

```
# Remove the object
my $response = $session->remove( $license );
```

Infoblox::Grid::Member->node1_licenses( ) / Infoblox::Grid::Member->node2_licenses( )

Use this function to retrieve the licenses of a specific Infoblox grid member.

Example

```
#Retrieve the licenses from a grid member
my $licenses = $grid_member->node1_licenses();
```

METHODS

This section describes all the methods that you can use to set or retrieve the attribute values of a License object.

expiration( )

Use this method to retrieve the license expiration timestamp. This is a read-only attribute.

Parameter

None

Returns

The method returns the number of seconds elapsed since January 1, 1970, in UTC format.

Example

```
#Get expiration value
my $expiration = $object->expiration();
```

expiration_status( )

Use this method to retrieve the license expiration status. This is a read-only attribute.

Parameter

None

Returns

The method returns the attribute value.

Example

```
#Get expiration_status value
my $expiration_status = $object->expiration_status();
```

key( )

Use this method to set or retrieve the license key.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

Specify the string with the license key of the device.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get key
my $key = $license->key();

#Modify key
$license->key("EQAAAKsmItQcEWiFb6csgab2VeQ7");
```

kind( )

Use this method to retrieve the overall type of license. This is a read-only attribute.

Parameter

None.

Returns

The method returns the attribute value. Valid values are:

```
"Static"  - The overall license type is static license.

"Dynamic" - The overall license type is dynamic license.
```

Example

```
#Get the overall type of license
my $kind = $license->kind();
```

hwid( )

Use this method to retrieve the hardware ID of the physical node on which the license is installed. This is a read-only attribute.

Parameter

None

Returns

The method returns the attribute value.

Example

```
#Get the hardware ID
my $hwid = $license->hwid();
```

limit( )

Use this method to retrieve the license limit value. This is a read-only attribute.

Parameter

None

Returns

The method returns the attribute value.

Example

```
#Get limit value
my $limit = $object->limit();
```

limit_context( )

Use this method to retrieve the context of the license limit. This is a read-only attribute.

Parameter

None

Returns

The method returns the attribute value.

Example

```
#Get limit_context value
my $limit_context = $object->limit_context();
```

type( )

Use this method to retrieve the type of license.

Returns

The method returns the attribute value. Valid values are:

```
"AnyCast"                   - Enables the AnyCast services.

"Cloud Network Automation"  - Enable cloud functionality for the GUI.

"Cloud Platform"            - Enable cloud API service on the Cloud Platform Appliance.

"DNSCacheAcceleration"      - Enables the DNS Cache Acceleration service.

"DDI Trial"                 - Enables the DDI Trial mode.

"DHCP"                      - Enables the Dynamic Host Configuration Protocol (DHCP) service.

"Discovery"                 - Enables the Discovery service.

"DNS"                       - Enables the Domain Name System (DNS) service.

"DNS Query Rewrite"         - Enables support for DNS Query Rewrite.

"FireEye"                   - Enables support for FireEye zones.

"KeystoneDVS"               - Enables Grid-based global management and reporting for Domain Name
System (DNS), Dynamic Host Configuration Protocol (DHCP), and IP address management (IPAM) appliance
solutions.

"KeystoneSup"               - Enables the Grid maintenance service.

"IF-MAP Service"            - Enables the IF-MAP service.

"IF-MAP Federation"         - Enables support for IF-MAP Federation connectors.

"IF-MAP Evaluation"         - Enables support for IF-MAP Evaluation.

"IPAM Freeware"             - Enables IPAM Express.

"LDAP"                      - Enables the Lightweight Directory Access Protocol (LDAP) service.

"Load Balancer"             - Enables support for Load Balancer.

"Multi-Grid Management"     - Enables Multi-Grid Management.

"MS Management"             - Enables Microsoft Management.

"NIOSMaintenance"           - Enables nonstop core network services such as Domain Name System
(DNS), Dynamic Host Configuration Protocol (DHCP), and IP address management (IPAM).

"OEM"                       - Enables OEM mode.

"Query Redirection"         - Enables the Query Redirection service.

"RADIUS"                    - Enables the RADIUS service.

"Reporting"                 - Enables the Reporting service.

"Response Policy Zones"     - Enables support for Response Policy zones.

"Security Ecosystem"        - Enables support for Security Ecosystem services, such as the TAXII
```

```
server.

 "Trinzic Automation Engine" - Enables the Network Automation Engine.

 "Threat Protection"         - Enables the Threat Protection service.

 "Threat Protection Update"  - Enables the Threat Protection Update service.

 "Threat Protection Software Add-On" - Enables the Threat Protection Software Add-On.

 "vNIOS"                     - Enables support for a specific vNIOS virtual appliance model.

 "Threat Analytics"          - Enables the Threat Analytics service.

 "Security Ecosystem"        - Enables the Security Ecosystem feature.

 "Flex Grid Activation"      - Allows all features to be enabled unless there is a conflict with
hardware or existing enabled services.
```

Example

```
 #Get type
 my $type = $license->type();
```

SAMPLE CODE

The following sample code demonstrates different operations that can be applied to a License object, such as create and remove an object. This sample code also includes error handling for the operations.

#Preparation prior to a license insertion

```
#PROGRAM STARTS: Include all the modules that will be used
use strict;
use Infoblox;

#Create a session to the Infoblox device
my $session = Infoblox::Session->new(
    master   => "192.168.1.2",
    username => "admin",
    password => "infoblox"
);

unless ($session) {
   die("Construct session failed: ",
       Infoblox::status_code() . ":" . Infoblox::status_detail());
}
print "Session created successfully\n";
```

#Add License object to Infoblox device

```
#Construct a License object

my $license = Infoblox::Grid::Member::License->new(
    key  => "EQAAAIMhrdkGjSnnmBlgQSA14b16"
);

unless ($license) {
   die("Construct License object failed: ",
       Infoblox::status_code() . ":" . Infoblox::status_detail());
}
print "License object created successfully\n";

#Adding the license object to Infoblox Appliance through session.
$session->add($license)
    or die("Add License failed: ",
           $session->status_code() . ":" . $session->status_detail() . "\nPlease contact Infoblox
Technical Support to get the license key and add them to the device\n");

print "Added License to Infoblox Appliance successfully.\n";

####PROGRAM ENDS####
```

AUTHOR

Infoblox Inc. http://www.infoblox.com/

SEE ALSO

Infoblox::Session

COPYRIGHT

Copyright (c) 2017 Infoblox Inc.

Infoblox::Grid::Member::OSPF - OSPF member object.

- NAME
- DESCRIPTION
- CONSTRUCTOR
- MODULE METHODS
  - Infoblox::Grid::Member->ospf_list( )
- METHODS
  - area_id( )
  - area_type( )
  - authentication_key( )
  - authentication_type( )
  - bfd_template( )
  - comment( )
  - cost( )
  - dead_interval( )
  - enable_bfd( )
  - hello_interval( )
  - interface( )
  - is_ipv4( )
  - key_id( )
  - retransmit_interval( )
  - transmit_delay( )
- SAMPLE CODE
- AUTHOR
- SEE ALSO
- COPYRIGHT

NAME

Infoblox::Grid::Member::OSPF - OSPF (Open Shortest Path First) member object.

DESCRIPTION

A OSPF member represents the configuration for a OSPF area used at the grid member level.

CONSTRUCTOR

```
my  $ospf = Infoblox::Grid::Member::OSPF->new(
    area_id             => $num | $ipv4addr,                        #Required
    interface           => "LAN_HA" | $ipaddr                       #Required
    area_type           => "standard" | "stub" | "nssa",            #Optional / Default is "standard"
    authentication_key  => $string | undef,                         #Optional / Default is undefined
    authentication_type => "none" | "simple" | "message-digest",    #Optional / Default is "none"
    bfd_template        => $string | undef,                         #Optional / Default is undefined
    enable_bfd          => "true" | "false"                         #Optional / Default is "false"
    comment             => $string | undef,                         #Optional / Default is undefined
    cost                => $num | "AUTO",                           #Optional / Default is "AUTO"
    dead_interval       => $num,                                    #Optional / Default is 40
    hello_interval      => $num,                                    #Optional / Default is 10
    key_id              => $num | undef,                            #Optional / Default is 1
    retransmit_interval => $num,                                    #Optional / Default is 5
    transmit_delay      => $num                                     #Optional / Default is 1
    is_ipv4             => "true" | "false"                         #Optional / Default is "true"
);
```

MODULE METHODS

The following functions are available to be applied to a OSPF object.

Infoblox::Grid::Member->ospf_list( )

Use this function to specify OSPF for the Grid Member. See Infoblox::Grid::Member->ospf_list() for parameters and return values.

Example

```
#Construct a OSPF object
my $ospf = Infoblox::Grid::Member::OSPF->new(
    area_id             => "20",
```

```
    area_type           => "standard",
    interface           => "LAN_HA",
    authentication_type => "simple",
    authentication_key  => "infoblox"
);

# Configure OSPF on the Grid Member object
my $response = $Grid_Member->ospf_list([$ospf]);
```

METHODS

This section describes all the methods that can be used to configure and retrieve the attribute values of a OSPF object.

area_id( )

Use this method to set or retrieve the area id value of the OSPF object.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

Specify the area id in a 32-bit integer (range from 0 to 4294967295) format or in IPv4 address format. An IPv4 address is a 32-bit number in dotted decimal notation. It consists of four 8-bit groups of decimal digits separated by decimal points (example: 192.168.1.2).

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get area_id
my $area_id = $ospf->area_id();
#Modify area_id
$ospf->area_id(1);
```

area_type( )

Use this method to set or retrieve the OSPF area type.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

Specify the area type in string format. Valid values are "standard", "stub" OR "nssa"(Not-so-stubby). Default value is "standard".

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get area_type
my $area_type = $ospf->area_type();
#Modify area_type
$ospf->area_type("stub");
```

authentication_key( )

Use this method to set the authentication password to use for OSPF. This is a write-only attribute. The authentication key is valid only when authentication type is "simple" or "message-digest".

Include the specified parameter to set the attribute value.

Parameter

Specify the authentication key in string format. The authentication key can have a maximum of 8 bytes if authentication type is "simple" or it can have a maximum of 16 bytes if the authentication type is "message-digest". Default value is undefined.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

Example

```
#Modify authentication_key
$ospf->authentication_key("infoblox");
```

authentication_type( )

Use this method to set or retrieve the authentication type used for the OSPF advertisement.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

Specify the authentication type in string format. Valid values are "none", "simple" or "message-digest". Default value is "none".

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get authentication_type
my $authentication_type = $ospf->authentication_type();
#Modify authentication_type
$ospf->authentication_type("message-digest");
```

bfd_template( )

Use this method to set or retrieve the Bidirectional Forwarding Detection (BFD) template name.

BFD template is used to configure advanced BFD settings such as timer intervals, authentication.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

Desired BFD template in string format. Default value is undefined.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get bfd_template value
my $bfd_template = $object->bfd_template();
#Modify bfd_template value
$object->bfd_template("template1");
```

comment( )

Use this method to set or retrieve a descriptive comment of the OSPF configuration.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

Desired comment in string format with a maximum of 256 bytes. Default value is undefined.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get comment
my $comment = $ospf->comment();
#Modify comment
$ospf->comment("Modifying the OSPF comment");
```

cost( )

Use this method to set or retrieve the cost metric associated with the OSPF advertisement. If "AUTO" is provided, cost metric will be generated automatically.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

Specify the cost metric in a 16-bit integer (range from 0 to 65535) format or in string format as "AUTO". Default value is "AUTO".

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get cost
my $cost = $ospf->cost();
#Modify cost
$ospf->cost(1);
```

dead_interval( )

Use this method to set or retrieve the dead interval value of OSPF. The dead interval describes the time taken in seconds to wait before declaring that the device is unavailable and down.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

Specify the dead interval in a 16-bit integer (range from 0 to 65535) format that represents the duration in seconds. Default value is 40.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get dead_interval
my $dead_interval = $ospf->dead_interval();
#Modify dead_interval
$ospf->dead_interval(10);
```

enable_bfd( )

Use this method to set or retrieve the flag that indicates whether the Bidirectional Forwarding Detection (BFD) fallover is enabled or not.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

Specify 'true' to enable BFD fallover and false to disable it. The default value is 'false'.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get enable_bfd value
my $enable_bfd = $object->enable_bfd();
#Modify enable_bfd value
$object->enable_bfd('true');
```

hello_interval( )

Use this method to set or retrieve the hello interval value of OSPF. The hello interval specifies how often to send OSPF hello advertisement, in seconds.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

Specify the hello interval in a 16-bit integer (range from 0 to 65535) format that represents the duration in seconds. Default value is 10.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get hello_interval
my $hello_interval = $ospf->hello_interval();
#Modify hello_interval
$ospf->hello_interval(20);
```

interface( )

Use this method to set or retrieve the interface that send out OSPF advertisement information.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

Valid value is "LAN_HA" or an IPv4 or IPv6 address of the VLAN interface.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get interface
my $interface = $ospf->interface();
#Modify interface
$ospf->interface("LAN_HA");
```

is_ipv4( )

Use this method to set or retrieve the OSPF protocol version.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

Specify "true" if the IPv4 version of OSPF is used, or "false" if the IPv6 version of OSPF is used. Default value is "true".

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get protocol version
my $ospf_is_ipv4 = $ospf->is_ipv4();
#Modify protocol version
$ospf->is_ipv4("true");
$ospf->is_ipv4("false");
```

key_id( )

Use this method to set or retrieve the hash key identifier to use for "message-digest" authentication. The hash key identifier is valid only when authentication type is "message-digest".

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

Specify the hash key identifier in a 8-bit integer (range from 0 to 255) format. Default value is 1.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get key_id
my $key_id = $ospf->key_id();
#Modify key_id
$ospf->key_id(100);
```

retransmit_interval( )

Use this method to set or retrieve the retransmit interval time of OSPF. The retransmit interval describes the time taken in seconds to wait before retransmitting OSPF advertisement.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

Specify the retransmit interval in a 16-bit integer (range from 0 to 65535) format that represents the duration in seconds. Default value is 5.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get retransmit_interval
my $retransmit_interval = $ospf->retransmit_interval();
#Modify retransmit_interval
$ospf->retransmit_interval(50);
```

transmit_delay( )

Use this method to set or retrieve the transmit delay value of OSPF. The transmit delay describes the time taken in seconds to wait before sending an advertisement.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

Specify the transmit delay in a 16-bit integer (range from 0 to 65535) format that represents the duration in seconds. Default value is 1.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get transmit_delay
my $transmit_delay = $ospf->transmit_delay();
#Modify transmit_delay
$ospf->transmit_delay(60);
```

SAMPLE CODE

The following sample code demonstrates the different functions that can be applied to an OSPF object such as add, modify, and remove. Also, this sample includes error handling for the operations.

#Preparation prior to OSPF object insertion

```
use strict;
use warnings;
use Infoblox;

my $host_name = "infoblox.localdomain";
my $host_ip = "192.168.1.2";
#Creating a session to Infoblox appliance.
my $session = Infoblox::Session->new(
    master   => $host_ip,
    username => "admin",
    password => "infoblox"
);
```

```
unless ($session) {
    die(qq(constructor for session failed: ),
        join(":", Infoblox::status_code(), Infoblox::status_detail()));
}
print "Session created successfully \n";
```

#Create a grid member OSPF object

```
#Creating OSPF object.
my $ospf1 = Infoblox::Grid::Member::OSPF->new(
    area_id             => "20",
    area_type           => "standard",
    interface           => "LAN_HA",
    authentication_type => "simple",
    authentication_key  => "infoblox"
   ) or die(qq(Constructor for ospf1 failed: ),
   join(":", Infoblox::status_code(), Infoblox::status_detail()));
print "OSPF object constructed successfully \n";
```

#Add OSPF object to Grid Member

```
#Getting the member object from appliance through session.
my @result = $session->get(
    object => "Infoblox::Grid::Member",
    name   => $host_name,
    ) or die(qq(Get Grid Member failed: ),
    join(":", $session->status_code(), $session->status_detail()));
print "Get Grid Member successful\n";

my $result = $result[0];
if ($result) {
    #Modifying the value of the ospf method from the member object.
    $result->ospf_list([$ospf1]);
    #Applying the changes to appliance through session.
    $session->modify($result)
        or die(qq(Modify Grid Member failed: ),
        join(":", $session->status_code(), $session->status_detail()));
}
print "OSPF object added to Grid Member successfully\n";
```

#Modify OSPF object

```
#Modifying the area id of the OSPF object.
$ospf1->area_id("50");

#Apply changes to the Grid Member object.
$result->ospf_list([$ospf1]);

#Update Grid Member object through the Infoblox session.
$session->modify($result)
  or die("Modify Grid Member object failed: ",
  $session->status_code() . ":" . $session->status_detail());
print "Grid Member object with modified OSPF updated to Infoblox appliance successfully\n";
```

#Remove OSPF object

```
#Remove the OSPF from the Grid Member object.
$result->ospf_list([]);

#Update Grid Member object through the Infoblox session.
$session->modify($result)
  or die("Modify Grid Member object failed: ",
  $session->status_code() . ":" . $session->status_detail());
print "Removed OSPF object from Grid Member successfully\n";

####PROGRAM ENDS####
```

AUTHOR

Infoblox Inc. http://www.infoblox.com/

SEE ALSO

Infoblox::Grid::Member, Infoblox::Session, Infoblox::Session->get(), Infoblox::Session->modify()

COPYRIGHT

Copyright (c) 2017 Infoblox Inc.

Infoblox::Grid::Member::ParentalControl - A Parental Control member object.

- NAME
- DESCRIPTION
- SESSION METHODS
  - Infoblox::Session->get( )
  - Infoblox::Session->modify( )
  - Infoblox::Session->search( )
- METHODS
  - enable_service( )
  - name( )
- AUTHOR
- SEE ALSO
- COPYRIGHT

NAME

Infoblox::Grid::Member::ParentalControl - A Parental Control member object.

DESCRIPTION

The Parental Control member object - provides information and configuration for member Parental Control feature.

SESSION METHODS

This section describes all the methods in the Infoblox::Session module that you can apply to a Parental Control member object.

Note that Parental Control member is automatically created when Infoblox::Grid::Member object is created and cannot be created and/or removed.

Infoblox::Session->get( )

Use this method to retrieve all the matching objects from the Infoblox appliance. See Infoblox::Session->get() for parameters and return values.

Key Reference

Apply the following attributes to get a specific Parental Control member object:

```
name - Optional. The Parental Control member host name.
```

Example

```
#Get member with name 'collector1.com'
my $object = $session->get(
    object => 'Infoblox::Grid::Member::ParentalControl',
    name   => 'collector1.com',
);
```

Infoblox::Session->modify( )

Use this method to modify an object in the Infoblox appliance. See Infoblox::Session->modify() for parameters and return values.

Example

```
#Modify enable_service value
$object->enable_service('true');

#Submit modification
my $response = $session->modify($object);
```

Infoblox::Session->search( )

Use this method to retrieve all the matching objects from the Infoblox appliance. See Infoblox::Session->search() for parameters and return values.

Key Reference

Apply the following attributes to get a specific Parental Control member object:

```
name - Optional. The Parental Control member host name.
```

Example

```
#Get member with name 'collector1.com'
my $object = $session->get(
    object => 'Infoblox::Grid::Member::ParentalControl',
    name   => 'collector1.com',
);
```

METHODS

This section describes all the methods that can be used to configure and retrieve the attribute values of a Parental Control member object.

enable_service( )

Use this method to set or retrieve the flag that indicates whether the Parental Control service is enabled or disabled.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

Specify 'true' to enable the Parental Control service or 'false' to disable/deactivate it. The default value is 'false'.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get enable_service value
my $enable_service = $object->enable_service();

#Modify enable_service value
$object->enable_service('true');
```

name( )

Use this method to retrieve the Parental Control member host name. This is a read-only attribute.

Omit the parameter to retrieve the attribute value.

Parameter

None

Returns

The method returns the attribute value.

Example

```
#Get name value
my $name = $object->name();
```

AUTHOR

Infoblox Inc. http://www.infoblox.com/

SEE ALSO

Infoblox::Session, Infoblox::Session->get(), Infoblox::Session->modify(), Infoblox::Session->search(),

---

COPYRIGHT

Copyright (c) 2017 Infoblox Inc.

Infoblox::Grid::Member::PNodeToken - Physical Node Token object.

- NAME
- DESCRIPTION
- CONSTRUCTOR
- METHODS
  - Infoblox::Grid::Member->physical_oid( )
  - Infoblox::Grid::Member->token_exp_date( )
  - Infoblox::Grid::Member->token( )
- SAMPLE CODE
- AUTHOR
- SEE ALSO
- COPYRIGHT

NAME

Infoblox::Grid::Member::PNodeToken - Physical Node Token object.

DESCRIPTION

This object is used as a permission token for a pre-provisioned member to automatically join the Grid.

CONSTRUCTOR

This object actually have no constructor, it is only returned by create_token and read_token function calls of Infoblox::Grid::Member class.

METHODS

This section describes all the methods in the Infoblox::Session module that you can apply to a License object.

Infoblox::Grid::Member->physical_oid( )

Use this method to retrieve the OID of the physical node for which the token is created.

Parameter

None

Returns

The method returns the attribute value, which is an integer number.

Example

```
# get the physical_oid value for the token
$pnode_token_obj->physical_oid()
```

Infoblox::Grid::Member->token_exp_date( )

Use this method to retrieve the date and time when the token will expire. The value is presented as a Unix timestamp.

Parameter

None

Returns

The method returns the attribute value, which is an integer number.

Example

```
# get the token_exp_date value for the token
$pnode_token_obj->token_exp_date()
```

Infoblox::Grid::Member->token( )

Use this method to retrieve the value (in text string) of the physical node token.

Parameter

None

Returns

The method returns the attribute value, which is a string.

Example

```
# get the token value itself
$pnode_token_obj->token()
```

SAMPLE CODE

The following sample code demonstrates how to create a token for the newly created pre-provisioned member.

```
#!/usr/bin/perl

use strict;
use Infoblox;

# get session for all subsequent operations
my $session = Infoblox::Session->new(master => 'localhost', username => 'admin', password => 'infoblox');

# create an object for the Grid member
my $grid_member = Infoblox::Grid::Member->new(
     name                    => "hostname.com",
     ipv4addr                => "3.0.0.2",
     mask                    => "255.0.0.0",
     gateway                 => "3.0.0.1",
     platform                => "VNIOS",
);
# add member
$session->add($grid_member);

# create pre-provisioning info objects
my $hw = Infoblox::Grid::Member::PreProvisioning::Hardware->new(
    hwtype          => 'IB-VNIOS',
    hwmodel         => 'CP-V1400',
);
my $pp = Infoblox::Grid::Member::PreProvisioning->new(
    hardware_info   => [$hw],
    licenses        => ['dns', 'dhcp'],
);
$grid_member->pre_provisioning($pp);
# ...and add pre-provisioning info to the Grid member
$session->modify($grid_member);

# Create tokens for the Grid member
# (actually the only token, as the member is not a HA-pair)
my $tokens_ref = $grid_member->create_token();

# Read all tokens for the grid member.
# Actually, we can skip the read_token call,
# as we already have tokens from previous
# create_token call in this example
# $tokens_ref = $grid_member->read_token();

if (defined $tokens_ref) {
   my @tokens = @$tokens_ref;
   # Iterate through all tokens in the array and
   # print their values.
   for (my $i = 0; $i < scalar @tokens; $i++) {
      print "Token $i:\n";
      print "physical_oid: " . $tokens[$i]->physical_oid(). "\n";
      print "token_exp_date: " . $tokens[$i]->token_exp_date() . "\n";
      print "token: " . $tokens[$i]->token() . "\n";
   }
}
```

AUTHOR

Infoblox Inc. http://www.infoblox.com/

SEE ALSO

Infoblox::Grid::Member

COPYRIGHT

Copyright (c) 2017 Infoblox Inc.

Infoblox::Grid::Member::PreProvisioning - Pre-Provisioned member object.

- NAME
- DESCRIPTION
- CONSTRUCTOR
- METHODS
  - hardware_info( )
  - licenses( )
- AUTHOR
- SEE ALSO
- COPYRIGHT

NAME

Infoblox::Grid::Member::PreProvisioning - Pre-Provisioned member object.

DESCRIPTION

An Infoblox::Grid::Member::PreProvisioning object represents pre-provisioned data for a Grid member.

CONSTRUCTOR

```
my $pp = Infoblox::Grid::Member::PreProvisioning->new(
    hardware_info   => [$hw1, $hw2, ...],                  # Optional / an array of
Infoblox::Grid::Member::PreProvisioning::Hardware objects
    licenses        => [$lic1, $lic2, ...],                # Optional / Default value is undefined
);
```

METHODS

This section describes all the methods that can be used to configure and retrieve the attribute values of an Infoblox::Grid::Member::PreProvisioning object.

hardware_info( )

Use this method to set or retrieve the hardware information.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

Valid value is an array of Infoblox::Grid::Member::PreProvisioning::Hardware objects.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get get hardware_info()
my $hw = $pp->hardware_info();
#Modify hardware_info
$pp->hardware_info([$hw1, $hw2]);
```

licenses( )

Use this method to set or retrieve pre-provisioning licenses.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

Valid value is an array of valid licenses for each IB model type for the pre-provisioinig feature.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get get license()
my $lics = $pp->license();
#Modify hardwary_info
$pp->license(['dns', 'dhcp']);
```

AUTHOR

Infoblox Inc. http://www.infoblox.com/

SEE ALSO

Infoblox::Grid::Member, Infoblox::Grid::Member::PreProvisioning::Hardware

---

COPYRIGHT

Copyright (c) 2017 Infoblox Inc.

Infoblox::Grid::Member::PreProvisioning::Hardware - PreProvisioning hardware info object.

- NAME
- DESCRIPTION
- CONSTRUCTOR
- METHODS
  - hwtype( )
  - hwmodel( )
- AUTHOR
- SEE ALSO
- COPYRIGHT

NAME

Infoblox::Grid::Member::PreProvisioning::Hardware - PreProvisioning hardware info object.

DESCRIPTION

An Infoblox::Grid::Member::PreProvisioning::Hardware object represents pre-provisioned data for the hardware model.

CONSTRUCTOR

```
 my $hw = Infoblox::Grid::Member::PreProvisioning->new(
     hwtype          => $string,                  # Optional / one of IB-100,IB-810, IB-820, IB-1410, IB-1420, IB-2210, IB-FLEX,
                                                  # IB-2220, IB-4010, IB-4030, PT-1400, PT-2200, PT-4000, IB-RSP2, IB-VM, IB-VNIOS
     hwmodel         => $string,                  # Optional / one of:
                                                  # - for IB-4010 are Rev1, Rev2;
                                                  # - for IB-4030 are Rev1, Rev2;
                                                  # - for PT-4000 is Rev2;
                                                  # - for IB-RSP2 is [TBD];
                                                  # - for IB-VNIOS are IB-VM-100, IB-VM-810, IB-VM-820, IB-VM-RSP, IB-VM-1410, IB-VM-1420, IB-VM-2210, IB-VM-2220, IB-VM-4010;
 );
```

METHODS

This section describes all the methods that can be used to configure and retrieve the attribute values of an Infoblox::Grid::Member::PreProvisioning::Hardware object.

hwtype( )

Use this method to set or retrieve the hardware type.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

Valid value is a valid hardware type string.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get get hwtype()
my $hwt = $hw->hwtype();
#Modify hwtype
$hw->hwtype('PT-4000');
```

hwmodel( )

Use this method to set or retrieve the hardware model.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

Valid value is a valid hardware model string.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get get hwmodel()
my $hwm = $hw->hwmodel();
#Modify hwmodel
$hw->hwmodel('Rev2');
```

AUTHOR

Infoblox Inc. http://www.infoblox.com/

SEE ALSO

Infoblox::Grid::Member, Infoblox::Grid::Member::PreProvisioning

---

COPYRIGHT

Copyright (c) 2017 Infoblox Inc.

Infoblox::Grid::Member::QueryFQDNParameter - A FQDN query parameters.

- NAME
- DESCRIPTION
- CONSTRUCTOR
- MODULE METHODS
  - Infoblox::Session->query_fqdn_on_member( )
- METHODS
  - fqdn( )
  - member( )
  - name_server( )
  - record_type( )
  - recursive_query( )
- AUTHOR
- SEE ALSO
- COPYRIGHT

NAME

Infoblox::Grid::Member::QueryFQDNParameter - A FQDN query parameters.

DESCRIPTION

A FQDN query call parameter objects. This object is intended to use only as Infoblox::Session->query_fqdn_on_member() parameter.

CONSTRUCTOR

```
 my $params = Infoblox::Grid::Member::QueryFQDNParameter->new(
     fqdn            => $string,                                          # Required
     member          => $string,                                          # Optional / Default is GM
     name_server     => $ipv4addr,                                        # Optional / Default is local
nameserver
     record_type     => 'ANY' | 'A' | 'AAAA' | 'CNAME' | 'DNAME' | 'MX' |
                        'NAPTR' | 'NS' | 'PTR' | 'SRV' | 'TXT' | 'AXFR'   # Optional / Default is 'ANY'
     recursive_query => 'true' | 'false',                                 # Optional / Default is 'true'
 );
```

MODULE METHODS

Infoblox::Session->query_fqdn_on_member( )

Use this method to query FQDN on the specified member.

This method forces Grid Master to execute RPC call to invoke dog on the member for a specific FQDN.

See Infoblox::Session->query_fqdn_on_member() for parameters and return values.

Example

```
#Construct parameters object
my $query_fqdn = Infoblox::Grid::Member::QueryFQDNParameter->new(
    fqdn        => 'bar.com',
    member      => 'member1',
    name_server => '5.5.5.5',
    record_type => 'A',
);

#Query
my $response = $session->query_fqdn_on_member($query_fqdn);
```

METHODS

This section describes all the methods that you can use to set or retrieve the attribute values of the object.

fqdn( )

Use this method to set or retrieve a fully-qualified DNS query name.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

The valid value is a FQDN to query in string format.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get fqdn value
my $fqdn = $object->fqdn();

#Modify fqdn value
$object->fqdn($fqdn);
```

member( )

Use this method to set or retrieve a member name to invoke dig on.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

The valid value is a string that contains the Grid Member host name. Default is undefined, which causes dig invokation on Grid Master.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get member value
my $member = $object->member();

#Modify member value
$object->member('localhost.localdomain');
```

name_server( )

Use this method to set or retrieve a nameserver to query.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

The valid value is a string that contains an IPv4 address.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get name_server value
my $value = $object->name_server();

#Modify name_server value
$object->name_server('5.5.5.5');
```

record_type( )

Use this method to set or retrieve the resource record type to query.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

The valid values are 'ANY', 'A', 'AAAA', 'CNAME', 'DNAME', 'MX', 'NAPTR', 'NS', 'PTR', 'SRV', 'TXT' and 'AXFR'. The default value is 'ANY'.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get record_type value
my $value = $object->record_type();

#Modify record_type value
$object->record_type('AAAA');
```

recursive_query( )

Use this method to set or retrieve the recursive query flag.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

Specify 'true' to set recursive query flag and 'false' to unset/deactivate it. The default value is 'true'.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get recursive_query value
my $value = $object->recursive_query();

#Modify recursive_query value
$object->recursive_query('false');
```

AUTHOR

Infoblox Inc. http://www.infoblox.com/

SEE ALSO

Infoblox::Session->query_fqdn_on_member()

COPYRIGHT

Copyright (c) 2017 Infoblox Inc.

Infoblox::Grid::Member::QueryFQDNResponse - A FQDN query response.

- NAME
- DESCRIPTION
- METHODS
  - result( )
  - result_text( )
  - dig_started( )
  - source_address( )
- AUTHOR
- SEE ALSO
- COPYRIGHT

NAME

Infoblox::Grid::Member::QueryFQDNResponse - A FQDN query response.

DESCRIPTION

A FQDN query call response objects. This object is returned from Infoblox::Session->query_fqdn_on_member() function call.

METHODS

This section describes all the methods that you can use to set or retrieve the attribute values of the object.

Note that this is a read-only object.

result( )

Use this method to set or retrieve a dig request return code.

Returns

The method returns the attribute value, which can be 'NOERROR', 'FORMERR', 'SERVFAIL', 'NXDOMAIN', 'NOTIMP', 'REFUSED' or 'INTERNAL_ERROR'.

Example

```
#Get result value
my $result = $object->result();
```

result_text( )

Use this method to set or retrieve a dig result multilined text.

Returns

The method returns the attribute value, a dig result text in string format.

Example

```
#Get result_text value
my $result_text = $object->result_text();
```

dig_started( )

Use this method to set or retrieve a timestamp since dig started.

Returns

The method returns the attribute value, the number of seconds that have elapsed since January 1st, 1970.

Example

```
#Get dig_started value
my $dig_started = $object->dig_started();
```

source_address( )

Use this method to set or retrieve a source IP address of the query.

Returns

The method returns the attribute value, the IP address in string format.

Example

```
#Get source_address value
my $source_address = $object->source_address();
```

AUTHOR

Infoblox Inc. http://www.infoblox.com/

SEE ALSO

Infoblox::Session->query_fqdn_on_member()

COPYRIGHT

Copyright (c) 2017 Infoblox Inc.

Infoblox::Grid::Member::RestartServiceStatus - The restart status for member services.

- NAME
- DESCRIPTION
- SESSION METHODS
  - Infoblox::Session->get( )
  - Infoblox::Session->search( )
- METHODS
  - dhcp_status( )
  - dns_status( )
  - member( )
  - radius_status( )
  - reporting_status( )
- AUTHOR
- SEE ALSO
- COPYRIGHT

NAME

Infoblox::Grid::Member::RestartServiceStatus - The restart status for member services.

DESCRIPTION

The object provides information about the restart status for member services.

SESSION METHODS

This section describes all the methods in an Infoblox::Session module that you can apply to an object.

Infoblox::Session->get( )

Use this method to retrieve all the matching objects from the Infoblox appliance. See Infoblox::Session->get() for parameters and return values.

Key References

```
Apply the following attributes to get a specific object:

 member - Optional. The Grid member name.
```

Example

```
my @retrieved_objs = $session->get(
    'object' => 'Infoblox::Grid::Member::RestartServiceStatus',
    'member' => 'hostname.com',
);
```

Infoblox::Session->search( )

Use this method to search for objects in the Infoblox appliance. See Infoblox::Session->search() for parameters and return values.

Key References

```
Apply the following attributes to search for an object:

 member - Optional. The Grid member name.
```

Example

```
my @retrieved_objs = $session->search(
    'object' => 'Infoblox::Grid::Member::RestartServiceStatus',
    'member' => 'hostname.com',
);
```

METHODS

This section describes all the methods that you can use to configure and retrieve the attribute values of an object.

dhcp_status( )

Use this method to retrieve the restart status for the member DHCP service. This is a read-only attribute.

Parameter

None

Returns

The method returns the attribute value that can be 'CONFIG_ERROR', 'DISABLED', 'NO', 'NO_PERMISSION', 'NO_REQUEST', 'OFFLINE', 'REQUESTING', 'RESTART_PENDING' or 'YES'.

Example

```
# Get attribute value
my $value = $object->dhcp_status();
```

dns_status( )

Use this method to retrieve the restart status for the member DNS service. This is a read-only attribute.

Parameter

None

Returns

The method returns the attribute value that can be 'CONFIG_ERROR', 'DISABLED', 'NO', 'NO_PERMISSION', 'NO_REQUEST', 'OFFLINE', 'REQUESTING', 'RESTART_PENDING' or 'YES'.

Example

```
# Get attribute value
my $value = $object->dns_status();
```

member( )

Use this method to retrieve the Grid member name. This is a read-only attribute.

Parameter

None

Returns

The method returns the attribute value.

Example

```
# Get attribute value
my $value = $object->member();
```

radius_status( )

Use this method to retrieve the restart status for the member RADIUS service. This is a read-only attribute.

Parameter

None

Returns

The method returns the attribute value that can be 'CONFIG_ERROR', 'DISABLED', 'NO', 'NO_PERMISSION', 'NO_REQUEST', 'OFFLINE', 'REQUESTING', 'RESTART_PENDING' or 'YES'.

Example

```
# Get attribute value
my $value = $object->radius_status();
```

reporting_status( )

Use this method to retrieve the restart status for the member reporting service. This is a read-only attribute.

Parameter

None

Returns

The method returns the attribute value that can be 'CONFIG_ERROR', 'DISABLED', 'NO', 'NO_PERMISSION', 'NO_REQUEST', 'OFFLINE', 'REQUESTING', 'RESTART_PENDING' or 'YES'.

Example

```
# Get attribute value
my $value = $object->reporting_status();
```

AUTHOR

Infoblox Inc. http://www.infoblox.com/

SEE ALSO

Infoblox::Session

COPYRIGHT

Copyright (c) 2017 Infoblox Inc.

Infoblox::Grid::Member::ServiceStatus - Service Status object.

- NAME
- DESCRIPTION
- MODULE METHODS
  - Infoblox::Grid::Member->service_status( ), Infoblox::Grid::Member->node1_service_status( ), Infoblox::Grid::Member->node2_service_status( )
- METHODS
  - description( )
  - status( )
  - service( )
- AUTHOR
- SEE ALSO
- COPYRIGHT

NAME

Infoblox::Grid::Member::ServiceStatus - Service Status object.

DESCRIPTION

This object is used to report the service status on an Infoblox appliance. All methods are to be considered read-only.

MODULE METHODS

The following functions can be applied to a ServiceStatus object.

Infoblox::Grid::Member->service_status( ), Infoblox::Grid::Member->node1_service_status( ), Infoblox::Grid::Member->node2_service_status( )

Use this function to retrieve a reference to an array of Infoblox::Grid::Member::ServiceStatus objects that represents the service status for an Infoblox appliance.

Example

```
#Retrieve the member object
my $member = $session->get(object => 'Infoblox::Grid::Member', name => 'hostname.com');

# Access its service status
my @services = @{$member->service_status()};
print $services[0]->status();
```

METHODS

This section describes all the methods that you can use to retrieve the attribute value of an appliance's service status.

description( )

Use this method to retrieve the description of the current ServiceStatus object.

Returns

The method returns the attribute value, which depends on the value of the 'service' method:

```
service value (for service_status) returned by this method
==========================================================
DHCP - DHCP service status
DNS - DNS service status
NTP - NTP service status
TFTP -  Percentage of the file distribution storage space that is used
HTTP_FILE_DIST - Percentage of the file distribution storage space that is used
FTP - Percentage of the file distribution storage space that is used
DNS_CACHE_ACCELERATION - DNS Cache Acceleration service status.

service value (for node1/2_service_status) returned by this method
===================================================================
NODE_STATUS - Node status
CPU1_TEMP - CPU1 temperature in Celsius
CPU2_TEMP - CPU2 temperature in Celsius
DISK_USAGE - Disk usage percentage
ENET_LAN - LAN1 interface IP
ENET_LAN2 - LAN2 interface IP
ENET_HA - HA interface IP
ENET_MGMT - MGMT interface IP
```

```
LCD - State of the LCD process (running/not running)
MEMORY - Memory usage percentage
REPLICATION - HA replication state
DB_OBJECT - Database capacity usage percentage
RAID_DISK1 - Disk status and type
RAID_DISK2 - Disk status and type
RAID_DISK3 - Disk status and type
RAID_DISK4 - Disk status and type
FAN1 - Fan 1 speed
FAN2 - Fan 2 speed
FAN3 - Fan 3 speed
POWER_SUPPLY - Power status
NTP_SYNC - Indicates if there are "reachable" NTP associations
TEMP_SYS - System temperature in Celsius
RAID_BATTERY - RAID battery status
CPU_USAGE - CPU Usage
BGP - BGP service status
OSPF - OSPF service status
OSPF6 - OSPFv6 service status
SWAP_USAGE - Swap Usage
```

Example

```
#Get description
my $description = $service_status->description();
```

status( )

Use this method to retrieve the service status.

Returns

The method returns the attribute value. Valid values are "Failed", "Warning", "Working", and "Inactive"

Example

```
#Get status
my $status = $service_status->status();
```

service( )

Use this method to retrieve the service identifier.

Returns

The method returns the attribute value. Possible values when the object is retrieved via service_status are:

ANALYTICS, ATP, CAPTIVE_PORTAL, CLOUD_API, DHCP, DISCOVERY, DNS, DNS_CACHE_ACCELERATION, FTP, HSM, HTTP_FILE_DIST, NTP, REPORTING, TAXII, TFTP

Possible values when the object is retrieved via node1/2_service_status are:

AUTH_NAMED, BGP, CORE_FILES, CPU1_TEMP, CPU2_TEMP, CPU_USAGE, CPU_USAGE, DB_OBJECT, DISCOVERY_CAPACITY, DISK_SIZE, DISK_USAGE, ENET_HA, ENET_LAN, ENET_LAN2, ENET_MGMT, EXTERNAL_STORAGE, FAN1, FAN2, FAN3, FAN4, FAN5, FAN6, FAN7, FAN8, JOIN_STATUS, LCD, MEMORY, MGM_SERVICE, NETWORK_CAPACITY, NODE_STATUS, NTP_SYNC, OSPF, OSPF6, PASSIVE_HA_CONNECTIVITY, POWER1, POWER2, POWER3, POWER4, POWER_SUPPLY, RAID_BATTERY, RAID_DISK1, RAID_DISK2, RAID_DISK3, RAID_DISK4, RAID_DISK5, RAID_DISK6, RAID_DISK7, RAID_DISK8, RAID_SUMMARY, REPLICATION, SFP_HA, SFP_LAN, SFP_LAN2, SFP_MGMT, SNIC_CHIP_TEMP, SNIC_CORE_UTIL, SNIC_PCB_TEMP, SNIC_UTIL, SUBGRID_CONN, SWAP_USAGE, SYS_TEMP, VPN_CERT

Example

```
#Get service
my $service = $service_status->service();
```

AUTHOR

Infoblox Inc. http://www.infoblox.com/

SEE ALSO

Infoblox::Grid::Member/>

COPYRIGHT

Copyright (c) 2017 Infoblox Inc.

Infoblox::Grid::Member::StaticRoute - Static route member object

- NAME
- DESCRIPTION
- CONSTRUCTOR
- MODULE METHODS
  - Infoblox::Grid::Member->static_routes( )
- METHODS
  - network( )
  - gateway( )
- SAMPLE CODE
- AUTHOR
- SEE ALSO
- COPYRIGHT

NAME

Infoblox::Grid::Member::StaticRoute - Static route member object

DESCRIPTION

A StaticRoute object provides static route information for a grid member.

CONSTRUCTOR

```
my $route = Infoblox::Grid::Member::StaticRoute->new(
        "network" => $network,          #Required
        "gateway" => $ip_address,       #Required
);
```

MODULE METHODS

The following functions can be applied to a StaticRoute object.

Infoblox::Grid::Member->static_routes( )

Use this function to specify a StaticRoute object for the grid member. See Infoblox::Grid::Member->static_routes() for parameters and return values.

Example

```
#Get static routes list
my $static_routes = $grid_member->static_routes();
#Modify static routes list
my $route = Infoblox::Grid::Member::StaticRoute->new(
        "network" => "192.168.2.0/24",
        "gateway" => "192.168.1.1",
);
$grid_member->static_routes([$route]);
```

METHODS

This section describes all the methods that can be used to configure and retrieve the attribute values of a StaticRoute object.

network( )

Use this method to set or retrieve the network address of a StaticRoute object.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

Specify the address and netmask of network in CIDR format (example: 192.168.2.0/24).

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get network address
my $network= $route->network();
#Modify network address
$route-->network("192.168.2.0/24");
```

gateway( )

Use this method to set or retrieve the gateway of the StaticRoute object.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

Specify the gateway address in IPv4 address format. An IPv4 address is a 32-bit number in dotted decimal notation. It consists of four 8-bit groups of decimal digits separated by decimal points (example: 192.168.1.2).

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get gateway address
my $gateway= $route->gateway();
#Modify gateway address
$route-->gateway("192.168.2.0");
```

SAMPLE CODE

The following sample code demonstrates the different functions that can be applied to a StaticRoute object such as add, modify, and remove. Also, this sample includes error handling for the operations.

#Preparation prior to a StaticRoute object insertion

```
use strict;
use warnings;
use Infoblox;

my $host_name = "infoblox.localdomain";
my $host_ip   = "192.168.1.2";
#Creating a session to the Infoblox appliance.
my $session = Infoblox::Session->new(
    master   => $host_ip,
    username => "admin",
    password => "infoblox"
);

unless ($session) {
    die(qq(constructor for session failed: ),
        join(":", Infoblox::status_code(), Infoblox::status_detail()));
}
print "Session created successfully \n";
```

#Create a StaticRoute object

```
#Creating a StaticRoute object.
my $route = Infoblox::Grid::Member::StaticRoute->new(
       "network" => "192.168.2.0/24",
       "gateway" => "192.168.1.1",
) or die(qq(Constructor for StaticRoute failed: ),
   join(":", Infoblox::status_code(), Infoblox::status_detail()));
print "StaticRoute object constructed successfully \n";
```

#Add a StaticRoute object to the static routes list of the grid member

```
#Getting the Grid Member object from the appliance through a session.
my @result = $session->get(
    object => "Infoblox::Grid::Member",
    name   => $host_name,
    ) or die(qq(Get Grid Member failed: ),
    join(":", $session->status_code(), $session->status_detail()));
print "Get Grid Member successful\n";
```

```
my $result = $result[0];
if ($result) {
    #Modifying the value of the StaticRoute method from the member object.
    $result->static_routes([$route]);
    #Applying the changes to appliance through session.
    $session->modify($result)
        or die(qq(Modify Grid Member failed: ),
        join(":", $session->status_code(), $session->status_detail()));
}
print "StaticRoute object added to Grid Member successfully\n";
```

#Remove the static routes list

```
#Remove the StaticRoute object from the Grid Member object.
$result->static_routes([]);

#Update the Grid Member object through the Infoblox session.
$session->modify($result)
  or die("Modify Grid Member object failed: ",
  $session->status_code() . ":" . $session->status_detail());
print "Removed StaticRoute object from Grid Member successfully\n";

####PROGRAM ENDS####
```

AUTHOR

Infoblox Inc. http://www.infoblox.com/

SEE ALSO

Infoblox::Grid::Member

COPYRIGHT

Copyright (c) 2017 Infoblox Inc.

Infoblox::Grid::Member::Taxii - A Taxii Member object.

- NAME
- DESCRIPTION
- SESSION METHODS
  - Infoblox::Session->get( )
  - Infoblox::Session->modify( )
  - Infoblox::Session->search( )
- METHODS
  - enable_service( )
  - ipv4addr( )
  - ipv6addr( )
  - name( )
  - taxii_rpz_config( )
- AUTHOR
- SEE ALSO
- COPYRIGHT

NAME

Infoblox::Grid::Member::Taxii - A Taxii Member object.

DESCRIPTION

The Taxii Member object provides information on Taxii service configuration such as the start/stop flag and RPZ (Response Policy Zone) configuration.

SESSION METHODS

This section describes all the methods in an Infoblox::Session module that you can apply to a Taxii Member object.

Infoblox::Session->get( )

Use this method to retrieve all the matching objects from the Infoblox appliance. See Infoblox::Session->get() for parameters and return values.

Key Reference

```
Apply the following attribute to get a specific object:

  ipv4addr - Optional. IPv4 Member address.
  ipv6addr - Optional. IPv6 Member address.
  name     - Optional. The host name of the parent member.
```

Example

```
#Get Taxii Member object with name 'taxii_member.com'
my @taxii_member = $session->get(
    object => 'Infoblox::Grid::Member::Taxii',
    name   => 'taxii_member.com',
);
```

Infoblox::Session->modify( )

Use this method to modify an object in the Infoblox appliance. See Infoblox::Session->modify() for parameters and return values.

Example

```
#Modify enable_service value
$taxii_member->enable_service('true');

#Submit modification
$session->modify($taxii_member);
```

Infoblox::Session->search( )

Use this method to search for objects in the Infoblox appliance. See Infoblox::Session->search() for parameters and return values.

Key Reference

```
Apply the following attribute to search a specific object:
```

```
  ipv4addr - Optional. IPv4 Member address (regular expression).
  ipv6addr - Optional. IPv6 Member address (regular expression).
  name     - Optional. The host name of the parent member.
```

Example

```
#Search for Taxii Member object with name starting with 'taxii'
my @taxii_member = $session->get(
    object => 'Infoblox::Grid::Member::Taxii',
    name   => 'taxii.*',
);
```

METHODS

This section describes all the methods that you can use to set or retrieve the attribute values of the object.

enable_service( )

Use this method to set or retrieve the flag that indicates whether the Taxii service is running on the given member or not.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

Specify 'true' to start/enable Taxii service or 'false' to stop/deactivate it.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get enable_service value
my $enable_service = $object->enable_service();

#Modify enable_service value
$object->enable_service('true');
```

ipv4addr( )

Use this method to retrieve the IPv4 address of the Taxii member. This is a read-only attribute.

Omit the parameter to retrieve the attribute value.

Parameter

None

Returns

The method returns the attribute value.

Example

```
#Get ipv4addr value
my $ipv4addr = $object->ipv4addr();
```

ipv6addr( )

Use this method to retrieve the IPv6 address of the Taxii member. This is a read-only attribute.

Omit the parameter to retrieve the attribute value.

Parameter

None

Returns

The method returns the attribute value.

Example

```
#Get ipv6addr value
my $ipv6addr = $object->ipv6addr();
```

name( )

Use this method to retrieve the name of the Taxii member. This is a read-only attribute.

Omit the parameter to retrieve the attribute value.

Parameter

None

Returns

The method returns the attribute value.

Example

```
#Get name value
my $name = $object->name();
```

taxii_rpz_config( )

Use this method to set or retrieve the Taxii service RPZ configuration list.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

The valid value is an array of Infoblox::Grid::Member::Taxii::RPZConfig.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get taxii_rpz_config value
my $taxii_rpz_config = $object->taxii_rpz_config();

#Modify taxii_rpz_config value
$object->taxii_rpz_config([$conf1, $conf2]);
```

AUTHOR

Infoblox Inc. http://www.infoblox.com/

SEE ALSO

Infoblox::Session, Infoblox::Session->get(), Infoblox::Session->modify(), Infoblox::Session->search(), Infoblox::Grid::Member::Taxii::RPZConfig

---

COPYRIGHT

Copyright (c) 2017 Infoblox Inc.

Infoblox::Grid::Member::Taxii::RPZConfig - A Taxii Member RPZ Configuration object.

- NAME
- DESCRIPTION
- CONSTRUCTOR
- MODULE METHODS
  - Infoblox::Grid::Member::Taxii->taxii_rpz_config( )
- METHODS
  - collection_name( )
  - zone( )
- AUTHOR
- SEE ALSO
- COPYRIGHT

NAME

Infoblox::Grid::Member::Taxii::RPZConfig - A Taxii Member RPZ Configuration object.

DESCRIPTION

The Taxii Member RPZ Configuration object provides configuration for RPZ rule creation through the Taxii protocol. Each Grid member running the Taxii server can support a separate set of response policy zones. These zones will be accessible via a user configurable name for the STIX collection.

CONSTRUCTOR

```
my $rpz_config = Infoblox::Grid::Member::Taxii::RPZConfig->new(
    collection_name => $string, #Required
    zone            => $rpzone, #Required
);
```

MODULE METHODS

Infoblox::Grid::Member::Taxii->taxii_rpz_config( )

Use this method to set or retrieve the Taxii service RPZ configuration list. See Infoblox::Grid::Member::Taxii->taxii_rpz_config() for parameters and return values.

Example

```
#Get taxii_rpz_config value
my $taxii_rpz_config = $object->taxii_rpz_config();

#Modify taxii_rpz_config value
$object->taxii_rpz_config([$conf1, $conf2]);
```

METHODS

collection_name( )

Use this method to set or retrieve the STIX collection name.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

The valid value is a desired collection name in sstring format.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get collection_name value
my $collection_name = $object->collection_name();

#Modify collection_name value
```

```
$object->collection_name('collection1');
```

zone( )

Use this method to set or retrieve the response policy zone that can be modified through the Taxii protocol.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

The valid value is an Infoblox::DNS::Zone object.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get zone value
my $zone = $object->zone();

#Modify zone value
$object->zone($rp_zone);
```

AUTHOR

Infoblox Inc. http://www.infoblox.com/

SEE ALSO

Infoblox::Grid::Member::Taxii, Infoblox::Grid::Member::Taxii->taxii_rpz_config(), Infoblox::DNS::Zone

---

COPYRIGHT

Copyright (c) 2017 Infoblox Inc.

Infoblox::Grid::Member::ThreatProtection - Member Threat Protection object.

- NAME
- DESCRIPTION
- SESSION METHODS
  - Infoblox::Session->get( )
  - Infoblox::Session->modify( )
  - Infoblox::Session->search( )
- METHODS
  - current_ruleset( )
  - enable_nat_rules( )
  - enable_service( )
  - events_per_second_per_rule( )
  - grid_member( )
  - hardware_model( )
  - hardware_type( )
  - override_events_per_second_per_rule( )
  - disable_multiple_dns_tcp_request( )
  - nat_rules( )
  - override_current_ruleset( )
  - override_disable_multiple_dns_tcp_request( )
  - override_enable_nat_rules( )
  - profile( )
- AUTHOR
- SEE ALSO
- COPYRIGHT

NAME

Infoblox::Grid::Member::ThreatProtection - Member Threat Protection object.

DESCRIPTION

This object provides information about the member Threat protection settings.

SESSION METHODS

The following functions can be applied to a Grid::Member::ThreatProtection object.

Infoblox::Session->get( )

Use this method to retrieve all the matching objects from the Infoblox appliance. See Infoblox::Session->get() for parameters and return values.

Key References

```
Apply the following attributes to get a specific Member Threat Protection object.

 grid_member    - Optional. The Grid member name.
 hardware_model - Optional. The Grid member hardware model name.
 hardware_type  - Optional. The Grid member hardware type.
 profile        - Optional. The Threat Protection profile object.
```

Example

```
my @retrieved_objs = $session->get(
    'object'      => 'Infoblox::Grid::Member::ThreatProtection',
    'grid_member' => 'host.com',
);
```

Infoblox::Session->modify( )

Use this method to modify an object in the Infoblox appliance. See Infoblox::Session->modify() for parameters and return values.

Example

```
# Use this method to modify attributes of an member object.
$member->enable_service('true');
$member->override_events_per_second_per_rule('true');
$member->events_per_second_per_rule('2');
$member->override_disable_multiple_dns_tcp_request('true');
```

```
$member->disable_multiple_dns_tcp_request('true');

# Submit modification
my $response = $session->modify( $member );
```

Infoblox::Session->search( )

Use this method to retrieve all the matching objects from the Infoblox appliance. See Infoblox::Session->search() for parameters and return values.

Key References

```
Apply the following attributes to search a specific Member Threat Protection object.

 grid_member    - Optional. The Grid member name.
 hardware_model - Optional. The Grid member hardware model name (regular expression).
 hardware_type  - Optional. The Grid member hardware type (regular expression).
 profile        - Optional. The Threat Protection profile object.
```

Example

```
my @retrieved_objs = $session->search(
    'object'      => 'Infoblox::Grid::Member::ThreatProtection',
    'grid_member' => 'host.com',
);
```

METHODS

This object provides information about the member threat protection settings.

current_ruleset( )

Use this method to set or retrieve the current member ruleset.

Setting this method to a defined value implicitly sets the override_current_ruleset method to "true". Setting the parameter to undefined causes the appliance to use the grid default and automatically resets the override_current_ruleset attribute to "false".

Note that when current_ruleset is set to a defined value and override_dcurrent_ruleset is set to "false", the last operation takes precedence. Thus the sequence $object->current_ruleset($rs); $object->override_current_ruleset("false"); will set override_current_ruleset to "false", and the sequence $object->override_current_ruleset("false"); $object->current_ruleset($rs); will result in override_current_ruleset="true".

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

Valid value is an Infoblox::Grid::ThreatProtection::Ruleset object.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get attribute value
my $value = $obj->current_ruleset();
#Modify attribute value
$obj->current_ruleset($rs);
```

enable_nat_rules( )

Use this method to enable or disable NAT (Network Address Translation) mapping for threat protection at the member level.

Setting this method to a defined value implicitly sets the override_enable_nat_rules method to "true". Setting the parameter to undefined causes the appliance to use the Grid default and automatically resets the override_enable_nat_rules attribute to "false".

Note that when enable_nat_rules is set to a defined value and override_denable_nat_rules is set to "false", the last operation takes precedence. Thus the sequence $object->enable_nat_rules($nat_rules); $object->override_enable_nat_rules("false"); will set override_enable_nat_rules to "false", and the sequence $object->override_enable_nat_rules("false"); $object->enable_nat_rules($nat_rules); will result in override_enable_nat_rules="true".

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Paramter

Specify 'true' to enable NAT mapping for threat protection or 'false' to disable it. The default value is 'false.'

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get attribute value
my $value = $object->enable_nat_rules();

#Modify attribute value
$object->enable_nat_rules('true');
```

enable_service( )

Use this method to enable or disable the Threat protection service on the member.

Parameter

Set the parameter to "true" to enable Threat protection service on the member. Set the parameter to "false" to disable the service.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
# Get attribute value
my $value = $object->enable_service();
# Modify attribute value
$object->enable_service('true');
```

events_per_second_per_rule( )

Use this method to set or retrieve the number of events logged per second per rule.

Parameter

The number of events logged per second per rule in string format.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
# Get attribute value
my $value = $object->events_per_second_per_rule();
# Modify attribute value
$object->events_per_second_per_rule('2');
```

grid_member( )

Use this method to retrieve a Grid member name. This is a read-only attribute.

Parameter

None

Returns

The method returns the attribute value.

Example

```
# Get attribute value
my $value = $object->grid_member();
```

hardware_model( )

Use this method to retrieve the hardware model name of the Grid member. This is a read-only attribute.

Parameter

None

Returns

The method returns the attribute value.

Example

```
#Get hardware_model value
my $value = $object->hardware_model();
```

hardware_type( )

Use this method to retrieve the type of the Grid member hardware. This is a read-only attribute.

Parameter

None

Returns

The method returns the attribute value.

Example

```
#Get hardware_type value
my $value = $object->hardware_type();
```

override_events_per_second_per_rule( )

Use this method to set or retrieve the override flag for the number of events logged per second per rule.

Parameter

Specify "true" to set the override_events_per_second_per_rule flag or "false" to deactivate/unset it. The default value is "false".

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
# Get attribute value
my $value = $object->override_events_per_second_per_rule();
# Modify attribute value
$object->override_events_per_second_per_rule('true');
```

disable_multiple_dns_tcp_request( )

Use this method to disable multiple BIND responses via TCP connection.

Setting this method to a defined value implicitly sets the override_disable_multiple_dns_tcp_request method to "true". Setting the parameter to undefined causes the appliance to use the Grid default and automatically resets the override_disable_multiple_dns_tcp_request attribute to "false".

Note that when config is set to a defined value and override_disable_multiple_dns_tcp_request is set to "false", the last operation takes precedence. Thus the sequence $object->disable_multiple_dns_tcp_request($value); $object->override_disable_multiple_dns_tcp_request("false"); will set override_disable_multiple_dns_tcp_request to "false", and the sequence $object->override_disable_multiple_dns_tcp_request("false"); $object->disable_multiple_dns_tcp_request($value); will result in override_disable_multiple_dns_tcp_request="true".

Parameter

Set the parameter to "true" to disable multiple TCP requests. Set the parameter to "false" to enable multiple TCP requests.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
# Get attribute value
```

```
my $value = $object->disable_multiple_dns_tcp_request();
# Modify attribute value
$object->disable_multiple_dns_tcp_request('true');
```

nat_rules( )

Use this method to set or retrieve the list of NAT mapping rules cofigured for the member.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

The valid value is an array of Infoblox::Grid::ThreatProtection::NATRule objects.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get attribute value
my $value = $obj->nat_rules();
#Modify attribute value
$obj->nat_rules($nat_rules);
```

override_current_ruleset( )

The override_current_ruleset attribute controls whether the member's current ruleset value is used, instead of the Grid default.

The override_current_ruleset attribute can be specified explicitly. It is also set implicitly when current_ruleset is set to a defined value.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

Specify "true" to set the override_current_ruleset flag or "false" to deactivate/unset it. The default value is "false".

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get override_current_ruleset from the member
my $override = $object->override_current_ruleset();
#Modify override_current_ruleset
$object->override_current_ruleset("true");
```

override_disable_multiple_dns_tcp_request( )

Use this method to set or retrieve the override flag for disabling multiple BIND responses via TCP connection.

Parameter

Specify "true" to set the override_disable_multiple_dns_tcp_request flag or "false" to unset it. The default value is "false".

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
# Get attribute value
my $value = $object->override_disable_multiple_dns_tcp_request();
# Modify attribute value
$object->override_disable_multiple_dns_tcp_request('true');
```

override_enable_nat_rules( )

The override_enable_nat_rules attribute controls whether the member's current enable_nat_rules value is used, instead of the Grid default.

The override_enable_nat_rules attribute can be specified explicitly. It is also set implicitly when enable_nat_rules is set to a defined value.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

Specify "true" to set the override_enable_nat_rules flag or "false" to deactivate/unset it. The default value is "false".

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get override_enable_nat_rules from the member
my $override = $object->override_enable_nat_rules();
#Modify override_enable_nat_rules
$object->override_enable_nat_rules("true");
```

profile( )

Use this method to set or retrieve the Threat Protection profile that is used to group this member.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

The valid value is an Infoblox::Grid::ThreatProtection::Profile object.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get profile value
my $value = $object->profile();

#Modify profile value
$object->profile($profile);
```

AUTHOR

Infoblox Inc. http://www.infoblox.com/

SEE ALSO

Infoblox::Session, Infoblox::Session->get(), Infoblox::Session->modify(), Infoblox::Session->search(), Infoblox::Grid::ThreatProtection::NATRule, Infoblox::Grid::ThreatProtection::Profile, Infoblox::Grid::ThreatProtection,

---

COPYRIGHT

Copyright (c) 2017 Infoblox Inc.

Infoblox::Grid::Member::ThreatProtection::Rule - Member Threat Protection Rule object.

- NAME
- DESCRIPTION
- SESSION METHODS
  - Infoblox::Session->get( )
  - Infoblox::Session->modify( )
  - Infoblox::Session->search( )
- METHODS
  - config( )
  - disable( )
  - member( )
  - override_config( )
  - override_disable( )
  - rule( )
  - sid( )
- AUTHOR
- SEE ALSO
- COPYRIGHT

NAME

Infoblox::Grid::Member::ThreatProtection::Rule - Member Threat Protection Rule object.

DESCRIPTION

This object provides information about the member Threat protection rule settings.

SESSION METHODS

The following functions can be applied to a Grid::Member::ThreatProtection::Rule object.

Infoblox::Session->get( )

Use this method to retrieve all the matching objects from the Infoblox appliance. See Infoblox::Session->get() for parameters and return values.

Key References

```
Apply the following attributes to get a specific Member Threat Protection::Rule object.
 sid    - Optional. The Rule ID.
 rule   - Optional. The Threat Protection Rule object.
 member - Optional. The Member Threat Protection object.
```

Example

```
my @retrieved_objs = $session->get(
    'object' => 'Infoblox::Grid::Member::ThreatProtection::Rule',
    'sid' => '211111'
);
```

Infoblox::Session->modify( )

Use this method to modify an object in the Infoblox appliance. See Infoblox::Session->modify() for parameters and return values.

Example

```
 my $rule = $session->search('object' => 'Infoblox::Grid::ThreatProtection::Rule', 'sid' =>
'211111');
 my $member = $session->get('object' => 'Infoblox::Grid::Member::ThreatProtection');
 my $member_rule = $session->search('object' => 'Infoblox::Grid::Member::ThreatProtection::Rule',
'member' => $member, 'rule' => $rule);

 my $conf = $member_rule->config();
 $conf->action('PASS');

 # Use this method to modify attributes of object.
 $member_rule->disable('true');
 $member_rule->override_disable('true');
 $member_rule->config($conf);
 $member_rule->override_config('true');
```

```
# Submit modification
my $response = $session->modify( $member_rule );
```

Infoblox::Session->search( )

Use this method to search for objects in the Infoblox appliance. See Infoblox::Session->search() for parameters and return values.

Key References

```
Apply the following attributes to get a specific Grid Threat Protection::Rule object.
 sid    - Optional. The Rule ID.
 rule   - Optional. The Threat Protection Rule object.
 member - Optional. The Member Threat Protection object.
```

Example

```
my $rule = $session->search('object' => 'Infoblox::Grid::ThreatProtection::Rule', 'sid' =>
'211111');
my $member = $session->get('object' => 'Infoblox::Grid::Member::ThreatProtection');
my $member_rule = $session->search('object' => 'Infoblox::Grid::Member::ThreatProtection::Rule',
    'member' => $member,
    'rule' => $rule,
);
```

METHODS

config( )

Use this method to set or retrieve rule configuration information.

Setting this method to a defined value implicitly sets the override_config method to "true". Setting the parameter to undefined causes the appliance to use the Grid default and automatically resets the override_config attribute to "false".

Note that when config is set to a defined value and override_config is set to "false", the last operation takes precedence. Thus the sequence $object->config($value); $object->override_config("false"); will set override_config to "false", and the sequence $object->override_config("false"); $object->config($value); will result in override_config="true".

Parameter

Valid value is reference on Infoblox::Grid::ThreatProtection::RuleConfig object.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
# Get attribute value
my $value = $object->config();
$value->log_severity('INFORMATIONAL');
# Modify attribute value
$object->config($value);
```

disable( )

Use this method to set or indicate whether the rule is enabled or disabled for the member.

Setting this method to a defined value implicitly sets the override_disable method to "true". Setting the parameter to undefined causes the appliance to use the Grid default and automatically resets the override_disable attribute to "false".

Note that when disable is set to a defined value and override_disable is set to "false", the last operation takes precedence. Thus the sequence $object->disable('true'); $object->override_disable("false"); will set override_disable to "false", and the sequence $object->override_disable("false"); $object->disable('true'); will result in override_disable="true".

Parameter

Valid value is "true" or "false".

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
# Get attribute value
my $value = $object->disable();
# Modify attribute value
$object->disable(true);
```

member( )

Use this method to retrieve reference to the member object. This is a read-only attribute.

Parameter

None

Returns

The method returns the attribute value.

Example

```
# Get attribute value
my $value = $object->member();
```

override_config( )

Use this method to set or retrieve the override flag for the rule configuration.

Parameter

Valid value is "true" or "false".

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
# Get attribute value
my $value = $object->override_config();
# Modify attribute value
$object->override_config(true);
```

override_disable( )

Use this method to set or retrieve the override flag for the disabled field.

Parameter

Valid value is "true" or "false".

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
# Get attribute value
my $value = $object->override_disable();
# Modify attribute value
$object->override_disable(true);
```

rule( )

Use this method to retrieve reference to the rule object. This is a read-only attribute.

Parameter

None

Returns

The method returns the attribute value.

Example

```
# Get attribute value
my $value = $object->rule();
```

sid( )

Use this method to retrieve the Rule ID. This is a read-only attribute.

Parameter

None

Returns

The method returns the attribute value.

Example

```
# Get attribute value
my $value = $object->sid();
```

AUTHOR

Infoblox Inc. http://www.infoblox.com/

SEE ALSO

Infoblox::Session, Infoblox::Grid::Member::ThreatProtection, Infoblox::Grid::ThreatProtection::Rule, Infoblox::Grid::ThreatProtection::RuleConfig, Infoblox::Grid::ThreatProtection::RuleParam

---

COPYRIGHT

Copyright (c) 2017 Infoblox Inc.

Infoblox::Grid::MSServer - Microsoft Grid Server object.

- NAME
- DESCRIPTION
- CONSTRUCTOR
- Infoblox::DNS::Zone Considerations
  - Methods with no effect
  - Applicable methods
- Infoblox::DNS considerations
- Infoblox::Grid::DNS considerations
- SESSION METHODS
  - Infoblox::Session->add( )
  - Infoblox::Session->get( )
  - Infoblox::Session->modify( )
  - Infoblox::Session->remove( )
  - Infoblox::Session->search( )
- METHODS
  - address()
  - ad_domain( )
  - ad_sites()
  - ad_user()
  - comment()
  - disable()
  - login()
  - password()
  - network_view()
  - dns_view()
  - log_destination( )
  - max_connection( )
  - managing_member()
  - override_log_destination( )
  - override_max_connection( )
  - override_rpc_timeout( )
  - read_only()
  - root_ad_domain( )
  - rpc_timeout( )
  - synchronization_interval()
  - logging_mode()
  - version()
  - is_AD()
  - extattrs( )
  - extensible_attributes( )
- SAMPLE CODE
- AUTHOR
- SEE ALSO
- COPYRIGHT

NAME

Infoblox::Grid::MSServer - Microsoft (r) Grid Server object.

DESCRIPTION

A grid Microsoft DNS Server object is a single Microsoft DNS server that is managed by a grid member.

CONSTRUCTOR

```
 my $ms_member = Infoblox::Grid::MSServer->new(
     address                 => $ip_or_fqdn,                                 #Required
     ad_sites                => $ad_sites,                                   #Optional
     ad_user                 => $ad_user,                                    #Optional
     comment                 => $string,                                     #Optional / Default value
is empty
     disable                 => "true" | "false"                             #Optional / Default value
is "false"
     extattrs                => { $string => $extattr, ... },                #Optional / Default is
undefined
     extensible_attributes   => { $string => $string | $num,
                                  $string => [ $string | $num, ... ], ... }, #Optional / Default is
undefined
```

```
     login                    => $string,                                    #Required
     password                 => $string,                                    #Optional / Default value
is empty
     managing_member          => $Member,                                    #Optional / Default value
is empty
     read_only                => "true" | "false",                           #Optional / Default value
is "true"
     synchronization_interval => $num,                                       #Optional / Default value
is 2
     logging_mode             => "minimum" | "normal" | "advanced" | "full", #Optional / Default value
is "normal"
     network_view             => $NetworkView,                               #Optional / Default is
"default" network view
     dns_view                 => $DnsView,                                   #Optional / Default is
"default" view
     log_destination          =>  "SYSLOG" | "MSLOG",                        #Optional / Default is
undefined
     override_log_destination => "true" | "false",                           #Optional / Default value
is "false"
     max_connection           => $num,                                       #Optional / Default is
undefined
     override_max_connection  => "true" | "false",                           #Optional / Default value
is "false"
     rpc_timeout              => $num,                                       #Optional / Default is
undefined
     override_rpc_timeout     => "true" | "false",                           #Optional / Default value
is "false"
 );
```

You cannot set both extattrs and extensible_attributes attributes at the same time.

Infoblox::DNS::Zone Considerations

The following apply to zones assigned to Microsoft DNS servers:

Methods with no effect

The following methods have no effect on a Microsoft DNS server that is assigned to a zone:

allow_transfer()
allow_query( )
allow_update( )
allow_update_forwarding( )
bind_check_names_policy( )
create_underscore_zones( )
disable_forwarding( )
forward_only( )
forward_to( )
import_from( )
override_delegated_ttl( )
override_grid_zone_timer( )
override_grid_email( )
override_serial_number( )
prefix( )
remove_sub_zones( )
soa_negative_ttl( )
shared_record_groups( )

Values set or retrieved using the member that manages the Microsoft DNS server apply only to that grid member if it is assigned to a zone.

Applicable methods

The following methods apply to any grid member or Microsoft DNS server assigned to the zone:

comment( )
delegated_ttl( )
disable( )
email( )
extensible_attributes( )
host_name_restriction_policy( )
locked( )
name( )
ns_group( )
multiple_primaries( )
secondaries( )
soa_default_ttl( )

soa_expire( )
member_soa_mnames( )
soa_refresh( )
soa_retry( )
member_soa_serials( )
stub_from( )
stub_members( )
zone( )

Infoblox::DNS considerations

The following objects cannot be used with a Microsoft DNS Server object or with a Zone object whose primary server is a Microsoft DNS Server object:

Infoblox::DNS::BulkHost
Infoblox::DNS::DnssecTrustedKey
Infoblox::DNS::Host
Infoblox::DNS::SRG
Infoblox::DNS::RootNameServer
Infoblox::DNS::Sortlist
Infoblox::DNS::TSIGKey
Infoblox::DNS::Record

DNAME (only with Windows Server 2008 and later)
DS, RRSIG, DNSKEY, NSEC, NSEC3, NSEC3PARAM (read-only on Windows Server 2008 R2 and later)

Infoblox::Grid::DNS considerations

The following methods can be applied to DNS zones managed by a Microsoft DNS server:

default_ttl( )
email( )
host_name_restriction_policy( )
zone_deletion_double_confirm( )

The following methods have no effect on any zone managed by a Microsoft DNS server:

allow_bulkhost_ddns( )
allow_gss_tsig_zone_updates( )
allow_transfer( )
allow_query( )
allow_recursive_query( )
allow_update( )
bind_check_names_policy( )
blackhole_list( )
bulk_host_name_template( )
bulk_host_name_templates( )
dnssec_enabled( )
dnssec_expired_signatures_enabled( )
`dnssec_ksk_algorithm()`
`dnssec_ksk_rollover_interval()`
`dnssec_ksk_size()`
dnssec_signature_expiration( )
dnssec_validation_enabled( )
dnssec_trusted_keys( )
`dnssec_zsk_algorithm()`
`dnssec_zsk_rollover_interval()`
`dnssec_zsk_size()`
enable_blackhole( )
expire_after( )
forward_only( )
forward_updates( )
forwarders( )
logging_categories( )
logging_facility( )
member_secondary_notify( )
negative_ttl( )
notify_source_port( )
nsgroup_default( )
nsgroups( )
query_source_port( )
record_name_policies( )
recursive_query_list( )
refresh_timer( )

retry_timer( )
rootNS( )
sortlist( )
transfer_excluded_servers( )
transfer_format( )

SESSION METHODS

This section describes all the methods in an Infoblox::Session module that you can apply to a grid member object.

Infoblox::Session->add( )

Use this method to add an object to the Infoblox appliance. See Infoblox::Session->add() for parameters and return values.

Example

```
#Construct an object
my $grid_msserver = Infoblox::Grid::MSServer->new(
                                                  address                  => '10.0.0.1',
                                                  comment                  => 'This is an MS DNS Server',
                                                  disable                  => 'true',
                                                  extensible_attributes    => { Site => 'Main Office'},
                                                  login                    => 'user',
                                                  password                 => 'password',
                                                  managing_member          => 'managing.member.com',
                                                  read_only                => 'false',
                                                  synchronization_interval => 5,
                                                  logging_mode             => 'minimum',
    );
#Submit for addition
my $response = $session->add( $grid_msserver );
```

Infoblox::Session->get( )

Use this method to retrieve all the matching objects from the Infoblox appliance. See Infoblox::Session->get() for parameters and return values.

Key References

```
Apply the following attributes to get a specific grid member object:

 address - Optional. A member address or name in FQDN format.
 comment - Optional. A comment in string format
 read_only - Optional. A boolean value
 managing_member - Optional. A name in FQDN format.
 network_view  -  Optional. The network view served by the member.
 dns_view -  Optional. The DNS view served by the member.
 extattrs - Optional. A hash reference containing extensible attributes.
 extensible_attributes - Optional. A hash reference containing extensible attributes.
```

Example

```
my @retrieved_objs = $session->get(
    object  => "Infoblox::Grid::MSServer",
    address => "10.0.0.1");
```

Infoblox::Session->modify( )

Use this method to modify an object in the Infoblox appliance. See Infoblox::Session->modify() for parameters and return values.

Example

```
# Use this method to modify the comment of a Microsoft DNS server managed by the grid.
$grid_msserver->comment("This is modified object");
# Submit modification
my $response = $session->modify( $grid_msserver );
```

Infoblox::Session->remove( )

Use this method to remove an object from the Infoblox appliance. See Infoblox::Session->remove() for parameters and return values.

To remove a specific object, first use `get()` or `search()` to retrieve the specific object, and then submit this object for removal.

Example

```
#Get the objects with the same name
my @retrieved_objs = $session->get(
```

```
    object => "Infoblox::Grid::MSServer",
    name   => "10.0.0.1");
#Find the desired object from the retrieved list.
my $desired_gridmsserver = $retrieved_objs[0];
# Submit for removal
my $response = $session->remove( $desired_gridmsserver );
```

Infoblox::Session->search( )

Use this method to search for Microsoft DNS Server objects in the Infoblox appliance. See Infoblox::Session->search() for parameters and return values.

Key References

Apply the following attributes to search for a Microsoft DNS Server object

```
address - Optional. A member address or name in FQDN format (regular expression).
comment - Optional. A comment in string format (regular expression).
read_only - Optional. A boolean value
managing_member - Optional. The FQDN of the grid member that manages the Microsoft DNS server?
network_view  -  Optional. The network view served by the member.
dns_view -  Optional. The DNS view served by the member.
extattrs - Optional. A hash reference containing extensible attributes.
extensible_attributes   - Optional. A hash reference that contains extensible attributes.
```

For more information about searching extensible attributes, see Infoblox::Grid::ExtensibleAttributeDef/Searching Extensible Attributes.

Example

```
# search for all matching grid msserver objects
my @retrieved_objs = $session->search(
    object  => "Infoblox::Grid::MSServer",
    extensible_attributes => { Site => 'Local'},
    );
```

METHODS

This section describes all the methods that you can use to configure and retrieve the attribute values of a Microsoft DNS server.

address()

Retrieve or set the IP address or FQDN of the Microsoft DNS server.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

The Microsoft DNS server name in FQDN format or IPV4 format

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get address
my $address = $msserver->address();
```

ad_domain( )

Use this method to retrieve the Active Directory domain name of the Microsoft Server. This is a read-only attribute.

Omit the parameter to retrieve the attribute value.

Parameter

None

Returns

The method returns the attribute value.

Example

```
#Get ad_domin value
my $ad_domain = $object->ad_domain();
```

ad_sites()

Use this method to set or retrieve structure containing Active Directory Sites information.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

The valid value is an Infoblox::Grid::MSServer::AdSites object.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get ad_sites value
my $ad_sites = $msserver->ad_sites();

#Modify ad_sites value
my $res = $msserver->ad_sites($ad_sites);
```

ad_user()

Use this method to set or retrieve the object that contains Active Directory User synchronization information.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

The valid value is an Infoblox::Grid::MSServer::AdUser object.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get ad_user value
my $ad_user = $msserver->ad_user();

#Modify ad_user value
my $res = $msserver->ad_user($ad_user);
```

comment()

Retrieve or set a user-defined string related to this Microsoft DNS server.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

Desired comment in string format with a maximum of 256 bytes.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get comment
my $comment = $msserver->comment();
#Modify comment
$msserver->comment("Modifying the MS DNS server comment");
```

disable()

Retrieve or set the `disable()` flag of this Microsoft DNS server.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

Specify "true" to set the disable flag or "false" to deactivate/unset it. The default value is "false".

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get disable
my $disable = $msserver->disable();
#Modify disable
$msserver->disable("true");
```

login()

Retrieve or set the login name that the grid member uses to connect to the Microsoft DNS server.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

When setting a value that requires a domain name, prepend the domain information followed by a backslash to the username: "domain\username".

Parameter

The valid value is a string that contains the login name.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get login
my $login = $msserver->login();
#Modify login
$msserver->login("newuser");
```

password()

Set the password that the grid member uses to connect to the Microsoft DNS server. This field is used only if the login field for this member is defined. This is a write-only attribute.

This field can not be retrieved.

Parameter

The valid value is a string that contains the password.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

Example

```
#Set the password
$msserver->password("iG&ojG97Y");
```

network_view()

Retrieve or set the name of the network view associated with this member.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

The valid value is a string that contains the name of the network view associated with this member.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get network_view
```

```
my $network_view = $msserver->network_view();
#Modify network_view
$msserver->network_view("external");
```

dns_view()

Retrieve or set the name of the DNS view that applies to this Microsoft DNS server. If this field is set to undefined, the default view is used. The DNS view must be set to a DNS view linked to the Microsoft DNS server network view.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

The valid value is a string that contains the name of the DNS view associated with the Microsoft DNS server

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get dns_view
my $dns_view = $msserver->dns_view();
#Modify dns_view
$msserver->dns_view("external");
```

log_destination( )

Use this method to direct the logging of synchronization messages to the syslog or mslog.

Setting this method to a defined value implicitly sets the override_log_destination method to "true". Setting the parameter to undefined causes the appliance to use the grid default and automatically resets the override_log_destination method to "false".

Note that when log_destination is set to a defined value and override_log_destination is set to "false", the last operation takes precedence. Thus the sequence $object->log_destination("MSLOG"); $object->override_log_destination("false"); will set override_log_destination to "false", and the sequence $object->override_log_destination("false"); $object->log_destination("MSLOG"); will result in override_log_destination="true".

Parameter

Valid values are "SYSLOG" and "MSLOG".

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get log_destination value
my $log_destination = $msserver->log_destination();
#Modify log_destination value
$msserver->log_destination("SYSLOG");
```

max_connection( )

Use this method to set or retrieve the maximum number of connections to a Microsoft server.

Setting this method to a defined value implicitly sets the override_max_connection method to "true". Setting the parameter to undefined causes the appliance to use the grid default and automatically resets the override_max_connection method to "false".

Note that when max_connection is set to a defined value and override_max_connection is set to "false", the last operation takes precedence. Thus the sequence $object->max_connection(20); $object->override_max_connection("false"); will set override_max_connection to "false", and the sequence $object->override_max_connection("false"); $object->max_connection(20); will result in override_max_connection="true".

Parameter

Valid values are integers between 2 and 40 inclusive.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get max_connection value
my $max_connection = $msserver->max_connection();
#Modify max_connection value
$msserver->max_connection(20);
```

managing_member()

Retrieve or set the grid member responsible for the synchronization of this Microsoft DNS server.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

The valid value is a string that contains the name of the grid member.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get managing_member
my $managing_member = $msserver->managing_member();
#Modify managing_member
$msserver->managing_member("example.member.com");
```

override_log_destination( )

The override_log_destination attribute controls whether the log_destination method values of the Microsoft Server are used, instead of the Grid default.

The override_log_destination attribute can be specified explicitly. It is also set implicitly when log_destination is set to a defined value.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

Set the parameter to "true" to override the Grid-level setting for log_destination. Set the parameter to "false" to inherit the Grid-level setting for log_destination. The default value is "false".

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get override_log_destination
my $override_log_destination = $msserver->override_log_destination();
#Modify override_log_destination
$msserver->override_log_destination("true");
```

override_max_connection( )

The override_max_connection attribute controls whether the max_connection method values of the Microsoft Server are used, instead of the Grid default.

The override_max_connection attribute can be specified explicitly. It is also set implicitly when max_connection is set to a defined value.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

Set the parameter to "true" to override the grid-level setting for max_connection. Set the parameter to "false" to inherit the grid-level setting for max_connection. The default value is "false".

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get override_max_connection
my $override_max_connection = $msserver->override_max_connection();
#Modify override_max_connection
$msserver->override_max_connection("true");
```

override_rpc_timeout( )

The override_rpc_timeout attribute controls whether the rpc_timeout method values of the Microsoft Server are used, instead of the Grid default.

The override_rpc_timeout attribute can be specified explicitly. It is also set implicitly when rpc_timeout is set to a defined value.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

Set the parameter to "true" to override the grid-level setting for rpc_timeout. Set the parameter to "false" to inherit the grid-level setting for rpc_timeout. The default value is "false".

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get override_rpc_timeout
my $override_rpc_timeout = $msserver->override_rpc_timeout();
#Modify override_rpc_timeout
$msserver->override_rpc_timeout("true");
```

read_only()

Retrieve or set the synchronization mode of this Microsoft DNS server.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Setting the read_only mode to true synchronizes all data from the Microsoft DNS server to the grid and forbids any change on the grid that could be synchronized to the Microsoft DNS server.

Parameter

Specify "true" to set the disable flag or "false" to deactivate/unset it. The default value is "true".

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get read_only
my $read_only = $msserver->read_only();
#Modify read_only
$msserver->read_only("false");
```

root_ad_domain( )

Use this method to retrieve the Active Directory root domain name of the Microsoft Server. This is a read-only attribute.

Omit the parameter to retrieve the attribute value.

Parameter

None

Returns

The method returns the attribute value.

Example

```
#Get ad_domin value
my $root_ad_domain = $object->root_ad_domain();
```

rpc_timeout( )

Use this method to set or retrieve the timeout value in seconds for RPC connections to Microsoft Server.

Setting this method to a defined value implicitly sets the override_rpc_timeout method to "true". Setting the parameter to undefined causes the appliance to use the grid default and automatically resets the override_rpc_timeout method to "false".

Note that when rpc_timeout is set to a defined value and override_rpc_timeout is set to "false", the last operation takes precedence. Thus the sequence $object->rpc_timeout(10); $object->override_rpc_timeout("false"); will set override_rpc_timeout to "false", and the sequence $object->override_rpc_timeout("false"); $object->rpc_timeout(10); will result in override_rpc_timeout="true".

Parameter

Valid values are integers between 0 and 65535 inclusive.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get rpc_timeout value
my $rpc_timeout = $msserver->rpc_timeout();
#Modify rpc_timeout value
$msserver->rpc_timeout(10);
```

synchronization_interval()

Retrieve or set the synchronization interval of this Microsoft DNS server.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

The synchronization interval controls how often data on the Microsoft DNS server is synchronized with the grid. The value is specified in minutes.

Parameter

The valid value is a number that specifies the synchronization interval in minutes. The default value is 2 minutes.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get synchronization_interval
my $synchronization_interval = $msserver->synchronization_interval();
#Modify synchronization_interval
$msserver->synchronization_interval(5);
```

logging_mode()

Retrieve or set the synchronization logging verbosity of this Microsoft DNS server.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

The valid value is "minimum", "normal", "advanced", or "full". The default value is "normal".

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get logging_mode
my $logging_mode = $msserver->logging_mode();
#Modify logging_mode
$msserver->logging_mode("advanced");
```

version()

Retrieve the version of this Microsoft DNS server. The returned version is a string, such as "Microsoft Windows Server 2003".

Returns

The method returns the attribute value.

Example

```
#Get version
my $version = $msserver->version();
```

is_AD()

Retrieve the Active Directory flag of the Microsoft DNS server. The method returns "true" if the Microsoft DNS server has Active Directory capabilities or "false" if it does not.

Returns

The method returns the attribute value.

Example

```
#Get is_AD
my $is_AD = $msserver->is_AD();
```

extattrs( )

Use this method to set or retrieve the extensible attributes associated with a Microsoft DNS server object.

Parameter

Valid value is a hash reference containing the names of extensible attributes and their associated values (Infoblox::Grid::Extattr objects).

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get extattrs
my $ref_extattrs = $msserver->extattrs();
#Modify extattrs
$msserver->extattrs({ 'Site' => $extattr1, 'Administrator' => $extattr2 });
```

extensible_attributes( )

Use this method to set or retrieve the extensible attributes associated with a Microsoft DNS server.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

For valid values for extensible attributes, see Infoblox::Grid::ExtensibleAttributeDef/Extensible Attribute Values.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get extensible attributes
my $ref_extensible_attributes = $msserver->extensible_attributes();
#Modify extensible attributes
$msserver->extensible_attributes({ 'Site' => 'Santa Clara', 'Administrator' => [ 'Peter', 'Tom' ]
});
```

SAMPLE CODE

The following sample code demonstrates the different functions that can be applied to an object such as add, get, modify, search and remove. This sample also includes error handling for the operations.

#Preparation prior to getting and modifying a grid msserver object

```
use strict;
use Infoblox;
```

```
#refers to Infoblox Appliance IP address
my $host_ip = "192.168.1.2";

#Create a session to the Infoblox appliance

my $session = Infoblox::Session->new(
    master   => $host_ip,
    username => "admin",
    password => "infoblox"
);
unless ($session) {
       die("Construct session failed: ",
               Infoblox::status_code() . ":" . Infoblox::status_detail());
}
print "Session created successfully\n";

#Create a grid member object

my $grid_member = Infoblox::Grid::Member->new(
    name          => "hostname.com",
    ipv4addr      => "3.0.0.2",
    mask          => "255.0.0.0",
    gateway       => "3.0.0.1",
    comment       => "Script Generated"
    );
unless ($grid_member) {
       die("Construct grid member object failed: ",
               Infoblox::status_code() . ":" . Infoblox::status_detail());
}
print "Grid member object created successfully\n";

#Adding the member object to appliance through session
my $response = $session->add( $grid_member );

unless ($response) {
    die("Add grid member failed: ",
           $session->status_code() . ":" . $session->status_detail());
}
print"Grid member added successfully \n";

my $msserver = Infoblox::Grid::MSServer->new(
                                          address                  => '10.0.0.1',
                                          comment                  => 'basic member',
                                          disable                  => 'true',
                                          extensible_attributes    => { Site => 'Local'},
                                          login                    => 'user',
                                          password                 => 'password',
                                          managing_member          => 'hostname.com',
                                          read_only                => 'false',
                                          synchronization_interval => 5,
                                          logging_mode             => 'minimum',
                                         );

unless ($msserver) {
       die("Construct grid MS DNS server object failed: ",
               Infoblox::status_code() . ":" . Infoblox::status_detail());
}
print "Grid MS DNS server object created successfully\n";

#Adding the member object to appliance through session
$response = $session->add( $msserver );

unless ($response) {
    die("Add grid MS DNS server failed: ",
           $session->status_code() . ":" . $session->status_detail());
}
print"Grid MS DNS server added successfully \n";
```

#Search for a grid Microsoft DNS server object

```
 my @retrieved_objs = $session->search(
     object => "Infoblox::Grid::MSServer",
     comment => "basic.*");
 my $object = $retrieved_objs[0];

 unless ($object) {
        die
```

```
            ("Search grid MS DNS server object failed: ",
                $session->status_code() . ":" . $session->status_detail());
 }
 print "Search grid MS DNS server object found at least 1 matching entry\n";
```

#Get and modify the msserver object

```
#Get grid member through session
my @retrieved_objs = $session->get(
    object => "Infoblox::Grid::MSServer",
    address => "10.0.0.1"
    );
my $msserver = $retrieved_objs[0];

unless ($msserver) {
    die("Get grid MS DNS server object failed: ",
           $session->status_code() . ":" . $session->status_detail());
}
print"Get grid MS DNS server successful \n";

# Modify one of the attributes of the selected Microsoft DNS server.

$msserver->comment("Modified MS DNS server object");

#Applying the changes
$session->modify($msserver)
   or die("Modify grid MS DNS server failed",
            $session->status_code() . ":" . $session->status_detail());
print"Grid MS DNS server modified successfully \n";
```

#Remove the created objects object

```
#Get grid member through session
my @retrieved_objs = $session->get(
    object => "Infoblox::Grid::MSServer",
    address => "10.0.0.1"
    );
my $desired_msserver = $retrieved_objs[0];

unless ($desired_msserver) {
    die("Get grid MS DNS server object failed: ",
           $session->status_code() . ":" . $session->status_detail());
}
print"Get grid member successful \n";

#Submit the object for removal
$session->remove( $desired_msserver )
    or die("Remove grid MS DNS server failed",
           $session->status_code() . ":" . $session->status_detail());
print"Grid MS DNS server removed successfully \n";

#Get grid member through session
my @retrieved_objs = $session->get(
    object => "Infoblox::Grid::Member",
    name   => "hostname.com"
    );
my $desired_grid_member = $retrieved_objs[0];

unless ($desired_grid_member) {
    die("Get grid member object failed: ",
           $session->status_code() . ":" . $session->status_detail());
}
print"Get grid member successful \n";

#Submit the object for removal
$session->remove( $desired_grid_member )
    or die("Remove grid member failed",
           $session->status_code() . ":" . $session->status_detail());
print"Grid member removed successfully \n";

####PROGRAM ENDS####
```

AUTHOR

Infoblox Inc. http://www.infoblox.com/

SEE ALSO

Infoblox::Session, Infoblox::Grid::MSServer::AdSites, Infoblox::Grid::MSServer::AdUser.

COPYRIGHT

Copyright (c) 2017 Infoblox Inc.

Infoblox::Grid::MSServer::AdSites - the Active Directory Site object.

- NAME
- DESCRIPTION
- SESSION METHODS
- METHODS
  - default_ip_site_link( )
  - last_sync_ts( )
  - last_sync_detail( )
  - last_sync_status( )
  - ldap_timeout( )
  - ldap_auth_port( )
  - ldap_encryption( )
  - login_name( )
  - login_password( )
  - managed( )
  - override_default_ip_site_link( )
  - override_login( )
  - override_ldap_timeout( )
  - override_synchronization_interval( )
  - read_only( )
  - synchronization_interval( )
- AUTHOR
- SEE ALSO
- COPYRIGHT

NAME

Infoblox::Grid::MSServer::AdSites - the Active Directory Site object.

DESCRIPTION

The object provides information about the Active Directory Site.

SESSION METHODS

The object does not support any session methods. It is retrieved as a part of the Microsoft Server object.

METHODS

this section describes all the methods that you can use to set or retrieve the attribute values of the object.

default_ip_site_link( )

Use this method to set or retrieve the default IP site link name.

Setting this method to a defined value implicitly sets the override_default_ip_site_link method to "true". Setting the parameter to undefined causes the appliance to use the Grid default and automatically resets the override_default_ip_site_link attribute to "false".

Note that when default_ip_site_link is set to a defined value and override_ddefault_ip_site_link is set to "false", the last operation takes precedence. Thus the sequence $object->default_ip_site_link('LAX'); $object->override_default_ip_site_link("false"); will set override_default_ip_site_link to "false", and the sequence $object->override_default_ip_site_link("false"); $object->default_ip_site_link('LAX'); will result in override_default_ip_site_link="true".

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

Valid value is a string with the default IP site link name.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get default_ip_site_link value
my $default_ip_site_link = $ad_sites->default_ip_site_link();
#Modify default_ip_site_link value
$ad sites  default ip site link
```

```
                ->                      ('Site1');
```

last_sync_ts( )

Use this method to retrieve a timestamp of the last LDAP synchronization attempt. This is a read-only attribute.

Parameter

None

Returns

The method returns the attribute value.

Example

```
#Get last_sync_ts value
my $last_sync_ts = $ad_sites->last_sync_ts();
```

last_sync_detail( )

Use this method to retrieve a string containing details of the last LDAP synchronization attempt. This is a read-only attribute.

Parameter

None

Returns

The method returns the attribute value.

Example

```
#Get last_sync_detail value
my $last_sync_detail = $ad_sites->last_sync_detail();
```

last_sync_status( )

Use this method to retrieve a string containing status of the last LDAP synchronization attempt. This is a read-only attribute.

Parameter

None

Returns

The method returns the attribute value that can be 'OK', 'WARNING' or 'ERROR'.

Example

```
#Get last_sync_status value
my $last_sync_status = $ad_sites->last_sync_status();
```

ldap_timeout( )

Use this method to set or retrieve an LDAP connection timeout interval in seconds for the Microsoft Server.

Setting this method to a defined value implicitly sets the override_ldap_timeout method to "true". Setting the parameter to undefined causes the appliance to use the Grid default and automatically resets the override_ldap_timeout attribute to "false".

Note that when ldap_timeout is set to a defined value and override_dldap_timeout is set to "false", the last operation takes precedence. Thus the sequence $object->ldap_timeout(10); $object->override_ldap_timeout("false"); will set override_ldap_timeout to "false", and the sequence $object->override_ldap_timeout("false"); $object->ldap_timeout(10); will result in override_ldap_timeout="true".

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

Valid value is an unsigned integer.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get ldap_timeout value
my $ldap_timeout = $ad_sites->ldap_timeout();
#Modify ldap_timeout value
$ad_sites->ldap_timeout(10);
```

ldap_auth_port( )

Use this method to set or retrieve an LDAP TCP connection port for the Microsoft Server.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

Valid values are unsinged integers between 0 and 65535.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get ldap_auth_port value
my $ldap_auth_port = $ad_sites->ldap_auth_port();
#Modify ldap_auth_port value
$ad_sites->ldap_auth_port(636);
```

ldap_encryption( )

Use this method to set or retrieve an encryption for the LDAP connection for the Microsoft Server.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

Valid value is 'SSL' or 'NONE'.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get ldap_encryption value
my $ldap_encryption = $ad_sites->ldap_encryption();
#Modify ldap_encryption value
$ad_sites->ldap_encryption('SSL');
```

login_name( )

Use this method to set or retrieve the Microsoft Server login name.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

When setting a value that requires a domain name, prepend the domain information followed by a backslash to the username: "domain\username".

Parameter

The valid value is a string that contains the login name.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get login value
my $login = $ad_sites->login();
#Modify login value
$ad_sites->login("newuser");
```

login_password( )

Use this method to set a password for the Microsoft Server. This field is used only if the login field for this member is defined. This is a write-only attribute.

This field cannot be retrieved.

Parameter

The valid value is a string that contains the password.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

Example

```
#Set the password
$ad_sites->password("iG&ojG97Y");
```

managed( )

Use this method to set or retrieve the flag that indicates whether the sites of the Microsoft Server are to be synchronized with the assigned managing member or not.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

Specify "true" to set the disable flag or "false" to deactivate/unset it. The default value is "false".

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get managed value
$managed = $ad_sites->managed();
#Modify managed value
$ad_sites->managed("true");
```

override_default_ip_site_link( )

The override_default_ip_site_link attribute controls whether the default_ip_site_link method values of the Microsoft Server are used, instead of the Grid default.

The override_default_ip_site_link attribute can be specified explicitly. It is also set implicitly when default_ip_site_link is set to a defined value.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

Set the parameter to "true" to override the Grid-level setting for default_ip_site_link. Set the parameter to "false" to inherit the Grid-level setting for default_ip_site_link. The default value is "false".

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get override_default_ip_site_link
my $override_default_ip_site_link = $ad_sites->override_default_ip_site_link();
#Modify override_default_ip_site_link
$ad_sites->override_default_ip_site_link("true");
```

override_login( )

The override_login attribute controls whether the login_name method values of the Microsoft Server are used, instead of the Grid default.

The override_login attribute can be specified explicitly. It is also set implicitly when login_name is set to a defined value.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

Set the parameter to "true" to override the Grid-level setting for login_name. Set the parameter to "false" to inherit the Grid-level setting for login_name. The default value is "false".

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get override_login
my $override_login = $ad_sites->override_login();
#Modify override_login
$ad_sites->override_login("true");
```

override_ldap_timeout( )

The override_ldap_timeout attribute controls whether the ldap_timeout method values of the Microsoft Server are used, instead of the Grid default.

The override_ldap_timeout attribute can be specified explicitly. It is also set implicitly when ldap_timeout is set to a defined value.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

Set the parameter to "true" to override the Grid-level setting for ldap_timeout. Set the parameter to "false" to inherit the Grid-level setting for ldap_timeout. The default value is "false".

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get override_ldap_timeout
my $override_ldap_timeout = $ad_sites->override_ldap_timeout();
#Modify override_ldap_timeout
$ad_sites->override_ldap_timeout("true");
```

override_synchronization_interval( )

The override_synchronization_interval attribute controls whether the synchronization_interval method values of the Microsoft Server are used, instead of the Grid default.

The override_synchronization_interval attribute can be specified explicitly. It is also set implicitly when synchronization_interval is set to a defined value.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

Set the parameter to "true" to override the Grid-level setting for synchronization_interval. Set the parameter to "false" to inherit the Grid-level setting for synchronization_interval. The default value is "false".

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get override_synchronization_interval
my $override_synchronization_interval = $ad_sites->override_synchronization_interval();
#Modify override_synchronization_interval
$ad_sites->override_synchronization_interval("true");
```

read_only( )

Use this method to enable or disable read-only synchronization of the sites for the Active Directory Domain.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

Specify "true" to set the disable flag or "false" to deactivate/unset it.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get read_only value
$read_only = $ad_sites->read_only();
#Modify read_only value
$ad_sites->read_only("true");
```

synchronization_interval( )

Use this method to set or retrieve the synchronization interval of the sites for the Active Directory Domain.

Setting this method to a defined value implicitly sets the override_synchronization_interval method to "true". Setting the parameter to undefined causes the appliance to use the Grid default and automatically resets the override_synchronization_interval attribute to "false".

Note that when synchronization_interval is set to a defined value and override_dsynchronization_interval is set to "false", the last operation takes precedence. Thus the sequence $object->synchronization_interval(10); $object->override_synchronization_interval("false"); will set override_synchronization_interval to "false", and the sequence $object->override_synchronization_interval("false"); $object->synchronization_interval(10); will result in override_synchronization_interval="true".

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

Valid value is an unsigned integer.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get synchronization_interval value
my $synchronization_interval = $ad_sites->synchronization_interval();
#Modify synchronization_interval value
$ad_sites->synchronization_interval(10);
```

AUTHOR

Infoblox Inc. http://www.infoblox.com/

SEE ALSO

Infoblox::Grid::MSServer

COPYRIGHT

Copyright (c) 2017 Infoblox Inc.

Infoblox::Grid::MSServer::AdSites::Domain - the Active Directory Domain object.

- NAME
- DESCRIPTION
- SESSION METHODS
  - Infoblox::Session->get( )
  - Infoblox::Session->search( )
- METHODS
  - ms_sync_master_name( )
  - ea_definition( )
  - network_view( )
  - netbios( )
  - name( )
  - read_only( )
- AUTHOR
- SEE ALSO
- COPYRIGHT

NAME

Infoblox::Grid::MSServer::AdSites::Domain - the Active Directory Domain object.

DESCRIPTION

The object provides information about the Active Directory Domain.

SESSION METHODS

Infoblox::Session->get( )

Use this method to retrieve all the matching objects from the Infoblox appliance. See Infoblox::Session->get() for parameters and return values.

Key Reference

```
Apply the following attribute to get a specific object:

  name          - Optional. The Active Directory Domain name in DNS form.
  netbios       - Optional. The Active Directory Domain name in NetBIOS form.
  network_view   - Optional. The network view which the Active Directory Domain is associated with.
  ea_definition - Optional. Extensible attribute definition that is linked with this Active
Directory Domain.
```

Example

```
my @retrieved_objs = $session->get(
   name          => 'domain.com',
   netbios       => 'DOMAIN',
   network_view  => 'external',
   ea_definition => 'DOMAIN.external Site',
);
```

Infoblox::Session->search( )

Use this method to search for objects in the Infoblox appliance. See Infoblox::Session->search() for parameters and return values.

Key Reference

```
Apply the following attribute to get a specific object:

  ea_definition - Optional. Extensible attribute definition that is linked with this Active
Directory Domain.
  name          - Optional. The Active Directory Domain name in DNS form.
  netbios       - Optional. The Active Directory Domain name in NetBIOS form.
  network_view   - Optional. The network view which the Active Directory Domain is associated with.
```

Example

```
my @retrieved_objs = $session->search(
   netbios => 'DOMAIN*',
);
```

METHODS

ms_sync_master_name( )

Use this method to retrieve IP address or FQDN of the managing master Microsoft Server. This is a read-only attribute.

Parameter

None

Returns

The method returns the attribute value.

Example

```
#Get ms_sync_master_name value
my $sync_master_name = $domain->ms_sync_master_name();
```

ea_definition( )

Use this method to retrieve extensible attribute definition that is linked with the Active Directory Domain. This is a read-only attribute.

Parameter

None

Returns

The method returns the attribute value.

Example

```
#Get ea_definition value
my $ea_definition = $domain->ea_definition();
```

network_view( )

Use this method to retrieve the network view which the Active Directory Domain is associated with. This is a read-only attribute.

Parameter

None

Returns

The method returns the attribute value.

Example

```
#Get network_view value
my $network_view = $domain->network_view();
```

netbios( )

Use this method to retrieve the Active Directory Domain name in NetBIOS form. This is a read-only attribute.

Parameter

None

Returns

The method returns the attribute value.

Example

```
#Get netbios value
my $netbios = $domain->netbios();
```

name( )

Use this method to retrieve the Active Directory Domain name in DNS form. This is a read-only attribute.

Parameter

None

Returns

The method returns the attribute value.

Example

```
#Get name value
my $name = $domain->name();
```

read_only( )

Use this method to retrieve the flag that indicates if this Active Directory Domain is synchronized in Read-Only or Read/Write mode. This is a read-only attribute.

Parameter

None

Returns

The valid return value is "true" or "false".

Example

```
#Get read_only value
my $read_only = $domain->read_only();
```

AUTHOR

Infoblox Inc. http://www.infoblox.com/

SEE ALSO

Infoblox::Session, Infoblox::Session->get(), Infoblox::Session->search()

COPYRIGHT

Copyright (c) 2017 Infoblox Inc.

Infoblox::Grid::MSServer::AdSites::Site - object that represents Active Directory Site.

- NAME
- DESCRIPTION
- CONSTRUCTOR
- SESSION METHODS
  - Infoblox::Session->add( )
  - Infoblox::Session->remove( )
  - Infoblox::Session->get( )
  - Infoblox::Session->modify( )
  - Infoblox::Session->search( )
- METHODS
  - domain( )
  - move_subnets( )
  - name( )
  - networks( )
- AUTHOR
- SEE ALSO
- COPYRIGHT

NAME

Infoblox::Grid::MSServer::AdSites::Site - object that represents Active Directory Site.

DESCRIPTION

The object provides information about Active Directory Site.

CONSTRUCTOR

```
my $ms_adsite = Infoblox::Grid::MSServer::AdSites::Site->new(
   domain   => $adsites_domain,                #Required
   name     => $string,                        #Required
   networks => [$network1, $network2, ...],    #Optional / Default is undefined
);
```

SESSION METHODS

This section describes all the methods in an Infoblox::Session module that you can apply to the Active Directory Site.

Infoblox::Session->add( )

Use this method to add an object to the Infoblox appliance. See Infoblox::Session->add() for parameters and return values.

Example

```
my $site = Infoblox::Grid::MSServer::AdSites::Site->new(
    domain   => $domain,
    name     => 'Site0',
    networks => [$network1, $network2],
);
```

Infoblox::Session->remove( )

Use this method to remove an object from the Infoblox appliance. See Infoblox::Session->remove() for parameters and return values.

To remove a specific object, first use `get()` or `search()` to retrieve the specific object, and then submit this object for removal.

Example

```
my $site = $session->get(
    domain => $domain,
    name   => 'Site0',
);

$session->remove($site);
```

Infoblox::Session->get( )

Use this method to retrieve all the matching objects from the Infoblox appliance. See Infoblox::Session->get() for parameters and return values.

Key Reference

Apply the following attribute to get a specific object:

```
   name   - Optional. The Active Directory Site name.
   domain - Optional. An Infoblox::Grid::MSServer::AdSites::Domain object.
```

Example

```
my @retrieved_objs = $session->get(
   name   => 'Site0',
   domain => $domain,
);
```

Infoblox::Session->modify( )

Use this method to modify an object in the Infoblox appliance. See Infoblox::Session->modify() for parameters and return values.

Example

```
$site->name('Site2');
$session->modify($site);
```

Infoblox::Session->search( )

Use this method to search for objects in the Infoblox appliance. See Infoblox::Session->search() for parameters and return values.

Key Reference

```
Apply the following attribute to get a specific object:

  name   - Optional. The Active Directory Site name.
  domain - Optional. An Infoblox::Grid::MSServer::AdSites::Domain object.
```

Example

```
my @retrieved_objs = $session->search(
   name   => 'Site*',
);
```

METHODS

domain( )

Use this method to retrieve Active Directory Domain to which this Active Directory belongs.

Note that you can create a Site object with a given Active Directory Domain but once you've created, you will not be able modify a Domain of existing objects.

Parameter

Valid value is an Infoblox::Grid::MSServer::AdSites::Domain object.

Returns

The method returns the attribute value.

Example

```
#Get domain value
my $domain = $site->domain();
```

move_subnets( )

Use this method to move subnets beetween Active Directory Domain Sites.

Parameters

```
 networks - a list of Infoblox::DHCP::Network and Infoblox::DHCP::IPv6Network objects to perform the
move operation on.
```

Returns

The method returns true when the modification succeeds, and returns false when the operation fails.

Example

```
#move subnets to the site
$site->move_subnets(
   networks => [$network1, $network2],
);
```

name( )

Use this method to set or retrieve the Active Directory Site name.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

The valid value is a string that contains the Active Directory Site name.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get name value
my $name = $site->name();
#Modify name value
$site->name('Site1');
```

networks( )

Use this method to set or retrieve the list of networks associated with this Active Directory Site.

Parameter

The valid value is a list of Infoblox::DHCP::Network, Infoblox::DHCP::NetworkContainer, Infoblox::DHCP::IPv6Network and Infoblox::DHCP::IPv6NetworkContainer objects.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get networks value
my @networks = $site->networks();
#Modify networks value
$site->networks([$network1, $network2, $network3]);
```

AUTHOR

Infoblox Inc. http://www.infoblox.com/

SEE ALSO

Infoblox::Session, Infoblox::Session->get(), Infoblox::Session->search(), Infoblox::Session->add(), Infoblox::Session->remove(), Infoblox::Session->modify(), Infoblox::Grid::MSServer::AdSites::Domain, Infoblox::DHCP::Network, Infoblox::DHCP::IPv6Network

---

COPYRIGHT

Copyright (c) 2017 Infoblox Inc.

Infoblox::Grid::MSServer::AdUser - the Active Directory User object.

- NAME
- DESCRIPTION
- CONSTRUCTOR
- SESSION METHODS
- MODULE METHODS
  - Infoblox::Grid::MSServer->ad_user( )
- METHODS
  - enable_user_sync( )
  - last_sync_time( )
  - last_sync_detail( )
  - last_sync_status( )
  - login_name( )
  - login_password( )
  - override_enable_user_sync( )
  - override_login( )
  - override_synchronization_interval( )
  - synchronization_interval( )
- AUTHOR
- SEE ALSO
- COPYRIGHT

NAME

Infoblox::Grid::MSServer::AdUser - the Active Directory User object.

DESCRIPTION

The object provides information about synchronization of the Active Directory users.

CONSTRUCTOR

```
my $ad_user = Infoblox::Grid::MSServer::AdUser->new(
   enable_user_sync                => "true" | "false", # Optional / Default is 'true'
   login_name                      => $string,          # Optional / Default is undefined
   login_password                  => $string,          # Optional / Default is undefined
   synchronization_interval        => $uint,            # Optional / Default is undefined
   override_login                  => "true" | "false", # Optional / Default is 'false'
   override_enable_user_sync       => "true" | "false", # Optional / Default is 'false'
   override_synchronization_interval => "true" | "false", # Optional / Default is 'false'
```

SESSION METHODS

The object does not support any session methods. It is retrieved as a part of the Microsoft Server object.

MODULE METHODS

The following functions can be applied to a Active Directory User object.

Infoblox::Grid::MSServer->ad_user( )

Use this method to set or retrieve structure containing Active Directory User synchronization information. See Infoblox::Grid::MSServer->ad_user() for parameters and return values.

Example

```
#Get ad_user value
my $ad_user = $msserver->ad_user();
#Modify ad_user value
my $res = $msserver->ad_user($ad_user);
```

METHODS

this section describes all the methods that you can use to set or retrieve the attribute values of the object.

enable_user_sync( )

Use this method to enable or disable the Active Directory user information synchronization.

Setting this method to a defined value implicitly sets the override_enable_user_sync method to "true". Setting the parameter to undefined causes the appliance to use the grid default and automatically resets the override_enable_user_sync method to "false".

Note that when enable_user_sync is set to a defined value and override_enable_user_sync is set to "false", the last operation takes precedence. Thus the sequence $object->enable_user_sync("true"); $object->override_enable_user_sync("false"); will set override_enable_user_sync to "false", and the sequence $object->override_enable_user_sync("false"); $object->enable_user_sync("true"); will result in override_enable_user_sync="true".

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

Specify "true" to enable Active Directory user information synchronization or "false" to forbid it. Default value is "false".

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get enable_user_sync value
my $enable_user_sync = $ad_user->enable_user_sync();
#Modify enable_user_sync value
$ad_user->enable_user_sync("true");
```

last_sync_time( )

Use this method to retrieve the timestamp of the last Active Directory user information synchronization attempt. This is a read-only attribute.

Parameter

None

Returns

The method returns the attribute value.

Example

```
#Get last_sync_time value
my $last_sync_time = $ad_user->last_sync_time();
```

last_sync_detail( )

Use this method to retrieve a string containing details of the last Active Directory user information synchronization attempt. This is a read-only attribute.

Parameter

None

Returns

The method returns the attribute value.

Example

```
#Get last_sync_detail value
my $last_sync_detail = $ad_user->last_sync_detail();
```

last_sync_status( )

Use this method to retrieve a string containing status of the last Active Directory user information synchronization attempt. This is a read-only attribute.

Parameter

None

Returns

The method returns the attribute value that can be 'OK', 'WARNING' or 'ERROR'.

Example

```
#Get last_sync_status value
my $last_sync_status = $ad_user->last_sync_status();
```

login_name( )

Use this method to set or retrieve the Active Directory user login name.

Setting this method to a defined value implicitly sets the override_login method to "true". Setting the parameter to undefined causes the appliance to use the Grid default and automatically resets the override_login attribute to "false".

Note that when login_name is set to a defined value and override_dlogin_name is set to "false", the last operation takes precedence. Thus the sequence $object->login_name('newuser'); $object->override_login("false"); will set override_login to "false", and the sequence $object->override_login("false"); $object->login_name('newuser'); will result in override_login="true".

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

When setting a value that requires a domain name, prepend the domain information followed by a backslash to the username: "domain\username".

Parameter

The valid value is a string that contains the login name.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get login value
my $login = $ad_user->login_name();
#Modify login value
$ad_user->login_name("newuser");
```

login_password( )

Use this method to set a password for the Active Directory user. This field is used only if the login field for this member is defined. This is a write-only attribute.

This field cannot be retrieved.

Parameter

The valid value is a string that contains the password.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

Example

```
#Set the password
$ad_user->login_password("iG&ojG97Y");
```

override_enable_user_sync( )

The override_enable_user_sync attribute controls whether the enable_user_sync method values of the Active Directory user are used, instead of the Grid default.

The override_enable_user_sync attribute can be specified explicitly. It is also set implicitly when enable_user_sync is set to a defined value.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

Set the parameter to "true" to override the Grid-level setting for enable_user_sync. Set the parameter to "false" to inherit the Grid-level setting for enable_user_sync. The default value is "false".

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get override_enable_user_sync
my $override_enable_user_sync = $ad_user->override_enable_user_sync();
#Modify override_enable_user_sync
$ad_user->override_enable_user_sync("true");
```

override_login( )

The override_login attribute controls whether the login_name method values of the Active Directory user are used, instead of the Grid default.

The override_login attribute can be specified explicitly. It is also set implicitly when login_name is set to a defined value.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

Set the parameter to "true" to override the Grid-level setting for login_name. Set the parameter to "false" to inherit the Grid-level setting for login_name. The default value is "false".

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get override_login
my $override_login = $ad_user->override_login();
#Modify override_login
$ad_user->override_login("true");
```

override_synchronization_interval( )

The override_synchronization_interval attribute controls whether the synchronization_interval method values of the Active Directory user are used, instead of the Grid default.

The override_synchronization_interval attribute can be specified explicitly. It is also set implicitly when synchronization_interval is set to a defined value.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

Set the parameter to "true" to override the Grid-level setting for synchronization_interval. Set the parameter to "false" to inherit the Grid-level setting for synchronization_interval. The default value is "false".

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get override_synchronization_interval
my $override_synchronization_interval = $ad_user->override_synchronization_interval();
#Modify override_synchronization_interval
$ad_user->override_synchronization_interval("true");
```

synchronization_interval( )

Use this method to set or retrieve the synchronization interval of the Active Directory user.

Setting this method to a defined value implicitly sets the override_synchronization_interval method to "true". Setting the parameter to undefined causes the appliance to use the Grid default and automatically resets the override_synchronization_interval attribute to "false".

Note that when synchronization_interval is set to a defined value and override_dsynchronization_interval is set to "false", the last operation takes precedence. Thus the sequence $object->synchronization_interval(10); $object->override_synchronization_interval("false"); will set override_synchronization_interval to "false", and the sequence $object->override_synchronization_interval("false"); $object->synchronization_interval(10); will result in override_synchronization_interval="true".

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

The valid value is an unsigned integer.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get synchronization_interval value
my $synchronization_interval = $ad_user->synchronization_interval();
#Modify synchronization_interval value
$ad_user->synchronization_interval(10);
```

AUTHOR

Infoblox Inc. http://www.infoblox.com/

SEE ALSO

Infoblox::Grid::MSServer

COPYRIGHT

Copyright (c) 2017 Infoblox Inc.

Infoblox::Grid::MSServer::AdUser::Data - the Active Directory User Data object.

- NAME
- DESCRIPTION
- SESSION METHODS
- METHODS
  - active_users_count( )
- AUTHOR
- SEE ALSO
- COPYRIGHT

NAME

Infoblox::Grid::MSServer::AdUser::Data - the Active Directory User Data object.

DESCRIPTION

The object provides information about the Active Directory users.

SESSION METHODS

The object does not support any session methods. It is retrieved as a part of the following objects: Infoblox::IPAM::Address, Infoblox::IPAM::Statistics, Infoblox::DHCP::Network, Infoblox::DHCP::SharedNetwork, Infoblox::DHCP::Range, Infoblox::DHCP::FixedAddr, Infoblox::DHCP::IPv6FixedAddr, Infoblox::DHCP::HostAddr, Infoblox::DNS::Host, Infoblox::DNS::Record::A, Infoblox::DNS::Record::AAAA, Infoblox::DNS::Record::PTR, Infoblox::Grid::Discovery::Device, Infoblox::Grid::CloudAPI::VMAddress.

METHODS

this section describes all the methods that you can use to set or retrieve the attribute values of the object.

active_users_count( )

Use this method to retrieve the number of active users. This is a read-only attribute.

Parameter

None

Returns

The valid return value is an unsigned integer.

Example

```
#get active_users_count
my $active_users_count = $data->active_users_count();
```

AUTHOR

Infoblox Inc. http://www.infoblox.com/

SEE ALSO

Infoblox::IPAM::Address, Infoblox::IPAM::Statistics, Infoblox::DHCP::Network, Infoblox::DHCP::SharedNetwork, Infoblox::DHCP::Range, Infoblox::DHCP::FixedAddr, Infoblox::DHCP::IPv6FixedAddr, Infoblox::DHCP::HostAddr, Infoblox::DNS::Host, Infoblox::DNS::Record::A, Infoblox::DNS::Record::AAAA, Infoblox::DNS::Record::PTR, Infoblox::Grid::Discovery::Device, Infoblox::Grid::CloudAPI::VMAddress.

COPYRIGHT

Copyright (c) 2017 Infoblox Inc.

Infoblox::Grid::MSServer::DCNSRecordCreation - An Infoblox Active Directory Domain Controller object

- NAME
- DESCRIPTION
- CONSTRUCTOR
- SESSION METHODS
- MODULE METHODS
  - Infoblox::DNS::Zone->ms_dc_ns_record_creation( )
- METHODS
  - address( )
  - comment( )
- AUTHOR
- SEE ALSO
- COPYRIGHT

NAME

Infoblox::Grid::MSServer::DCNSRecordCreation - An Infoblox Active Directory Domain Controller object

---

DESCRIPTION

The Active Directory Domain Controller object represents the Active Directory Domain Controller which is allowed for NS Record creation in a DNS Zone.

CONSTRUCTOR

```
my $dcncrecordcreation = Infoblox::Grid::MSServer::DCNSRecordCreation->new(
    address => $ipv4addr | $ipv6addr, #Required
    comment => $string,               #Optional / Default is undefined
);
```

SESSION METHODS

The object does not support any session methods. It is retrieved as a part of the DNS Zone object. It also can be retrieved as a result of Infoblox::Session->get_ms_dc_ns_record_creation_list() function.

MODULE METHODS

The following functions can be applied to a AD Domain Controller object.

Infoblox::DNS::Zone->ms_dc_ns_record_creation( )

Use this method to set or retrieve the list of Active Directory Domain Controllers which are allowed to create NS Record for the DNS Zone. See Infoblox::DNS::Zone->ms_dc_ns_record_creation() for parameters and return values.

Example

```
#Get ms_dc_ns_record_creation
my $ms_dc_ns_record_creation = $zone->ms_dc_ns_record_creation();

#Modify ms_dc_ns_record_creation
$zone->ms_dc_ns_record_creation([$msdcns1, $msdcns1]);
```

METHODS

this section describes all the methods that you can use to set or retrieve the attribute values of the object.

address( )

Use this method to set or retrieve the address of a Domain Controller.

Parameter

The valid value is a desired IPv4 or IPv6 address in a string format.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get address value
my $address = $object->address();

#Set address value
$object->address('10.0.0.1');
```

comment( )

Use this method to set or retrieve the descriptive comment.

Parameter

The valid value is a desired comment in a string format.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get comment value
my $address = $object->comment();

#Set comment value
$object->comment('desired-comment');
```

AUTHOR

Infoblox Inc. http://www.infoblox.com/

SEE ALSO

Infoblox::DNS::Zone->ms_dc_ns_record_creation(), Infoblox::Session->get_ms_dc_ns_record_creation_list()

COPYRIGHT

Copyright (c) 2017 Infoblox Inc.

Infoblox::Grid::MSServer::DHCP - Grid Microsoft Server DHCP object.

- NAME
- DESCRIPTION
- SESSION METHODS
  - Infoblox::Session->get( )
  - Infoblox::Session->search( )
  - Infoblox::Session->modify( )
- METHODS
  - manage_dhcp()
  - enable_dhcp()
  - status()
  - status_detail()
  - status_last_updated()
  - last_sync_ts()
  - last_sync_status()
  - last_sync_detail()
  - login_name( )
  - login_password( )
  - extattrs( )
  - extensible_attributes()
  - address()
  - broadcast_address()
  - enable_ddns()
  - enable_invalid_mac( )
  - enable_monitoring( )
  - ddns_server_always_updates()
  - ms_options( )
  - comment()
  - read_only()
  - name()
  - static_hosts()
  - supports_failover( )
  - dynamic_hosts()
  - dhcp_utilization()
  - dhcp_utilization_status()
  - override_enable_invalid_mac( )
  - override_enable_monitoring( )
  - override_login( )
  - override_synchronization_interval( )
  - synchronization_interval( )
- AUTHOR
- SEE ALSO
- COPYRIGHT

NAME

Infoblox::Grid::MSServer::DHCP - Grid Microsoft (r) Server DHCP object.

DESCRIPTION

You can use Infoblox::Grid::MSServer::DHCP to configure some of the DHCP properties of a Microsoft DHCP server, including enabling/disabling the DHCP service. The service configurations of a grid are not inherited by this object.

SESSION METHODS

This section describes all the methods in an Infoblox::Session module that you can apply to a Microsoft Server DHCP object.

Infoblox::Session->get( )

Use this method to retrieve all the matching objects from the Infoblox appliance. See Infoblox::Session->get() for parameters and return values.

Key References

```
 Apply the following attributes to get a specific grid member object:

  address - Required. A Microsoft server address or name in FQDN (Fully Qualified Domain Name)
format.
```

Example

```
my @retrieved_objs = $session->get(
    object  => "Infoblox::Grid::MSServer::DHCP",
    address => "10.0.0.1");
```

Infoblox::Session->search( )

Use this method to retrieve all the matching objects from the Infoblox appliance. See Infoblox::Session->search() for parameters and return values.

Key References

```
Apply the following attributes to search for a specific grid member object:

  address - Required. A Microsoft server address or name in FQDN (Fully Qualified Domain Name)
format. (Regular expressions are supported)
```

Example

```
my @retrieved_objs = $session->search(
    object  => "Infoblox::Grid::MSServer::DHCP",
    address => "domain.*");
```

Infoblox::Session->modify( )

Use this method to modify an object in the Infoblox appliance. See Infoblox::Session->modify() for parameters and return values.

Example

```
# Use this method to modify a field of a grid Microsoft DHCP server.
$grid_msserver_dhcp->enable_dhcp('true');
# Submit modification
my $response = $session->modify( $grid_msserver_dhcp );
```

METHODS

This section describes all the methods that you can use to configure and retrieve the attribute values of a grid Microsoft DHCP server.

manage_dhcp()

Retrieve or set the managed state of the Microsoft DHCP server. When setting the managed state, use true to enable or false to disable synchronization of the DHCP server. Setting the status does not stop or start the actual DHCP server, but controls whether it is synchronized.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

Specify "true" to set the disable flag or "false" to deactivate/unset it.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get manage_dhcp
my $manage_dhcp = $msserver_dhcp->manage_dhcp();
#Modify manage_dhcp
$msserver_dhcp->manage_dhcp("true");
```

enable_dhcp()

Set the DHCP service status of the Microsoft DHCP server. When setting the service status, use true to enable or false to disable the DHCP service on the Microsoft DHCP server. Setting the service status stops or starts the actual DHCP service. This only applies if `manage_dhcp()` is true.

When setting the service status, the Microsoft DHCP server will be started or stopped at the next synchronization.

Include the specified parameter to set the attribute value. This setting is write-only. To get the actual status of the service, use the `status()` method.

Parameter

Specify "true" to set the disable flag or "false" to deactivate/unset it.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get enable_dhcp
my $enable_dhcp = $msserver_dhcp->enable_dhcp();
#Modify enable_dhcp
$msserver_dhcp->enable_dhcp("true");
```

status()

Retrieve the running status of the DHCP service of the Microsoft DHCP server. The running status is "running", "stopped", or "error". The returned status is the latest known status as of the date returned by the `status_last_updated()` attribute.

Returns

The method returns the attribute value.

Example

```
#Get status
my $status = $msserver_dhcp->status();
```

status_detail()

Retrieve the detailed running status of the DHCP service of the Microsoft DHCP server. The returned status is the latest known status.

Returns

The method returns the attribute value.

Example

```
#Get status
my $status = $msserver_dhcp->status();
```

status_last_updated()

Retrieve the time when the DHCP service status of the Microsoft DHCP server was last updated.

Returns

The method returns the attribute value.

Example

```
#Get status_last_updated
my $status_last_updated = $msserver_dhcp->status_last_updated();
```

last_sync_ts()

Retrieve the timestamp of the end of the last synchronization attempt related to all DHCP data, except leases.

Returns

The method returns the attribute value.

Example

```
#Get last_sync_ts
my $last_sync_ts = $msserver_dhcp->last_sync_ts();
```

last_sync_status()

Retrieve the result of the last synchronization attempt, if last_sync_ts is defined. Valid values are "Ok", "Warning", or "Error".

Returns

The method returns the attribute value.

Example

```
#Get last_sync_status
my $last_sync_status = $msserver_dhcp->last_sync_status();
```

last_sync_detail()

Retrieve the detailed result of the last synchronization attempt, if last_sync_ts is defined.

Returns

The method returns the attribute value.

Example

```
#Get last_sync_detail
my $last_sync_detail = $msserver_dhcp->last_sync_detail();
```

login_name( )

Use this method to set or retrieve the Microsoft Server DHCP login name.

Setting this method to a defined value implicitly sets the override_login method to "true". Setting the parameter to undefined causes the appliance to use the Grid default and automatically resets the override_login attribute to "false".

Note that when login_name is set to a defined value and override_dlogin_name is set to "false", the last operation takes precedence. Thus the sequence $object->login_name('newuser'); $object->override_login("false"); will set override_login to "false", and the sequence $object->override_login("false"); $object->login_name('newuser'); will result in override_login="true".

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

When setting a value that requires a domain name, prepend the domain information followed by a backslash to the username: "domain\username".

Parameter

The valid value is a string that contains the login name.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get login value
my $login = $msserver_dhcp->login_name();
#Modify login value
$msserver_dhcp->login_name("newuser");
```

login_password( )

Use this method to set a password for the Microsoft Server DHCP. This field is used only if the login field for this member is defined. This is a write-only attribute.

This field cannot be retrieved.

Parameter

The valid value is a string that contains the password.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

Example

```
#Set the password
$msserver_dhcp->login_password("iG&ojG97Y");
```

extattrs( )

Use this method to retrieve the extensible attributes associated with a Grid Microsoft DHCP Server object. This attribute is read-only and cannot be set.

Parameter

None.

Returns

The method returns the attribute value.

Example

```
#Get extattrs
my $ref_extattrs = $msserver_dhcp->extattrs();
```

extensible_attributes()

Use this method to retrieve the extensible attributes of the associated Grid Microsoft DHCP Server object.

This method is read-only. Use the Infoblox::Grid::MSServer object to set or modify the extensible attributes of the Grid Microsoft DHCP Server object.

Returns

The method returns the attribute value.

Example

```
#Get extensible_attributes
my $extensible_attributes = $msserver_dhcp->extensible_attributes();
```

address()

Use this method to retrieve the IP address or FQDN of the associated Grid Microsoft DHCP Server object.

This method is read-only. Use the Infoblox::Grid::MSServer object to set or modify the Grid Microsoft DHCP Server object address.

Returns

The method returns the attribute value.

Example

```
#Get address
my $address = $msserver_dhcp->address();
```

broadcast_address()

Use this method to set or retrieve the broadcast address of the associated Grid Microsoft DHCP Server object.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Returns

The Microsoft DHCP server broadcast address in IPV4 format.

Example

```
#Get address
my $address = $msserver_dhcp->broadcast_address();
#Modify broadcast_address
$msserver_dhcp->broadcast_address("10.0.0.1");
```

enable_ddns()

Retrieve or set the dynamic DNS flag of the Microsoft DHCP server.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

Specify "true" to set the disable flag or "false" to deactivate/unset it.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get enable_ddns
my $enable_ddns = $msserver_dhcp->enable_ddns();
#Modify enable_ddns
$msserver_dhcp->enable_ddns("true");
```

enable_invalid_mac( )

Use this method to allow invalid MAC address synchronization in DHCP leases and fixed addresses.

Setting this method to a defined value implicitly sets the override_enable_invalid_mac method to "true". Setting the parameter to undefined causes the appliance to use the grid default and automatically resets the override_enable_invalid_mac method to "false".

Note that when enable_invalid_mac is set to a defined value and override_enable_invalid_mac is set to "false", the last operation takes precedence. Thus the sequence $object->enable_invalid_mac("true"); $object->override_enable_invalid_mac("false"); will set override_enable_invalid_mac to "false", and the sequence $object->override_enable_invalid_mac("false"); $object->enable_invalid_mac("true"); will result in override_enable_invalid_mac="true".

Parameter

Specify "true" to allow invalid MAC address synchronization or "false" to forbid it. Default value is "false".

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get enable_invalid_mac value
my $enable_invalid_mac = $msserver_dhcp->enable_invalid_mac();
#Modify enable_invalid_mac value
$msserver_dhcp->enable_invalid_mac("true");
```

enable_monitoring( )

Use this method to enable or disable the monitoring and control of DHCP service.

Setting this method to a defined value implicitly sets the override_enable_monitoring method to "true". Setting the parameter to undefined causes the appliance to use the grid default and automatically resets the override_enable_monitoring method to "false".

Note that when enable_monitoring is set to a defined value and override_enable_monitoring is set to "false", the last operation takes precedence. Thus the sequence $object->enable_monitoring("true"); $object->override_enable_monitoring("false"); will set override_enable_monitoring to "false", and the sequence $object->override_enable_monitoring("false"); $object->enable_monitoring("true"); will result in override_enable_monitoring="true".

Parameter

Specify "true" to allow invalid mac address synchronization or "false" to forbid it. Default value is "false".

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get enable_monitoring value
my $enable_monitoring = $msserver_dhcp->enable_dhcp_monitoring();
#Modify enable_monitoring value
$msserver_dhcp->enable_monitoring("true");
```

ddns_server_always_updates()

Retrieve or set the "Always update DDNS" flag of the Microsoft DHCP server. Use "true" to always allow dynamic DNS updates, or use "false" to only update when requested by the client. The default value is "false".

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

Specify "true" to set the disable flag or "false" to deactivate/unset it.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get ddns_server_always_updates
my $ddns_server_always_updates = $msserver_dhcp->ddns_server_always_updates();
#Modify ddns_server_always_updates
$msserver_dhcp->ddns_server_always_updates("true");
```

ms_options( )

Use this method to set or retrieve the custom DHCP options that are associated with the specified Microsoft DHCP server.

Note: You can set only options that are included in both the Microsoft server and NIOS predefined option definitions. Options that are not common to both are read-only and cannot be modified.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

The valid value is an array reference that contains defined Infoblox::DHCP::MSOption objects. The default value is empty.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get options
my $ref_options = $msserver_dhcp->ms_options();
#Modify options
$msserver_dhcp->ms_options([$option12]); #$option12 is an Infoblox::DHCP::MSOption object
```

comment()

Use this method to retrieve the comment associated with the Grid Microsoft DHCP Server object.

This method is read-only. Use the Infoblox::Grid::MSServer object to set or modify the Grid Microsoft DHCP Server object comment.

Returns

The method returns the attribute value.

Example

```
#Get comment
my $comment = $msserver_dhcp->comment();
```

read_only()

Use this method to retrieve the read_only flag of the associated Grid Microsoft DHCP Server object.

This method is read-only. Use the Infoblox::Grid::MSServer object to set or modify the Grid Microsoft DHCP Server object read_only status.

Returns

The method returns the attribute value.

Example

```
#Get read_only
my $read_only = $msserver_dhcp->read_only();
```

name()

Use this method to retrieve the name of the associated Grid Microsoft DHCP Server object.

This method is read-only. Use the Infoblox::Grid::MSServer object to set or modify the Grid Microsoft DHCP Server object name.

Returns

The method returns the attribute value.

Example

```
#Get name
my $name = $msserver_dhcp->name();
```

static_hosts()

Use this method to retrieve the number of static addresses (fixed addresses and reservations) associated with the Microsoft server.

Returns

The method returns the attribute value.

Example

```
#Get static_hosts
my $static_hosts = $msserver_dhcp->static_hosts();
```

supports_failover( )

Use this method to retrieve the flag used to indicate whether DHCP supports failover. This is a read-only attribute.

Parameter

None

Returns

The method returns the attribute value.

Example

```
#Get supports_failover value
my $supports_failover = msserver_dhcp->supports_failover();
```

dynamic_hosts()

Use this method to retrieve the number of active leases associated with the Microsoft server.

Returns

The method returns the attribute value.

Example

```
#Get dynamic_hosts
my $dynamic_hosts = $msserver_dhcp->dynamic_hosts();
```

dhcp_utilization()

Use this method to retrieve the DHCP utilization status of the Microsoft server. The returned value is between 0 and 1000. 1000 indicates 100% utilization.

Returns

The method returns the attribute value.

Example

```
#Get dhcp_utilization
my $dhcp_utilization = $msserver_dhcp->dhcp_utilization();
```

dhcp_utilization_status()

Use this method to retrieve the DHCP utilization status of the Microsoft server. The returned value is 'Low', 'Normal', 'High', or 'Full', and depend on the value of the low- and high-water mark parameters.

Returns

The method returns the attribute value.

Example

```
#Get dhcp_utilization_status
my $dhcp_utilization_status = $msserver_dhcp->dhcp_utilization_status();
```

override_enable_invalid_mac( )

The override_enable_invalid_mac attribute controls whether the enable_invalid_mac method values of the Microsoft Server DHCP object are used, instead of the Grid default.

The override_enable_invalid_mac attribute can be specified explicitly. It is also set implicitly when enable_invalid_mac is set to a defined value.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

Set the parameter to "true" to override the grid-level setting for enable_invalid_mac. Set the parameter to "false" to inherit the

grid-level setting for enable_invalid_mac. The default value is "false".

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get override_enable_invalid_mac
my $override_enable_invalid_mac = $msserver_dhcp->override_enable_invalid_mac();
#Modify override_enable_invalid_mac
$msserver_dhcp->override_enable_invalid_mac("true");
```

override_enable_monitoring( )

The override_enable_monitoring attribute controls whether the enable_monitoring method values of the Microsoft Server DHCP object are used, instead of the Grid default.

The override_enable_monitoring attribute can be specified explicitly. It is also set implicitly when enable_monitoring is set to a defined value.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

Set the parameter to "true" to override the grid-level setting for enable_monitoring. Set the parameter to "false" to inherit the grid-level setting for enable_monitoring. The default value is "false".

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get override_enable_monitoring
my $override_enable_monitoring = $msserver_dhcp->override_enable_monitoring();
#Modify override_enable_monitoring
$msserver_dhcp->override_enable_monitoring("true");
```

override_login( )

The override_login attribute controls whether the login_name method values of the Microsoft Server DHCP are used, instead of the Grid default.

The override_login attribute can be specified explicitly. It is also set implicitly when login_name is set to a defined value.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

Set the parameter to "true" to override the Grid-level setting for login_name. Set the parameter to "false" to inherit the Grid-level setting for login_name. The default value is "false".

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get override_login
my $override_login = $msserver_dhcp->override_login();
#Modify override_login
$msserver_dhcp->override_login("true");
```

override_synchronization_interval( )

The override_synchronization_interval attribute controls whether the synchronization_interval method values of the Microsoft Server DHCP are used, instead of the Grid default.

The override_synchronization_interval attribute can be specified explicitly. It is also set implicitly when synchronization_interval is set to a defined value.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

Set the parameter to "true" to override the Grid-level setting for synchronization_interval. Set the parameter to "false" to inherit the Grid-level setting for synchronization_interval. The default value is "false".

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get override_synchronization_interval
my $override_synchronization_interval = $msserver_dhcp->override_synchronization_interval();
#Modify override_synchronization_interval
$msserver_dhcp->override_synchronization_interval("true");
```

synchronization_interval( )

Use this method to set or retrieve the synchronization interval of the Microsoft Server DHCP.

Setting this method to a defined value implicitly sets the override_synchronization_interval method to "true". Setting the parameter to undefined causes the appliance to use the Grid default and automatically resets the override_synchronization_interval attribute to "false".

Note that when synchronization_interval is set to a defined value and override_dsynchronization_interval is set to "false", the last operation takes precedence. Thus the sequence $object->synchronization_interval(10); $object->override_synchronization_interval("false"); will set override_synchronization_interval to "false", and the sequence $object->override_synchronization_interval("false"); $object->synchronization_interval(10); will result in override_synchronization_interval="true".

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

The valid value is an unsigned integer.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get synchronization_interval value
my $synchronization_interval = $msserver_dhcp->synchronization_interval();
#Modify synchronization_interval value
$msserver_dhcp->synchronization_interval(10);
```

AUTHOR

Infoblox Inc. http://www.infoblox.com/

SEE ALSO

Infoblox::Grid::MSServer, Infoblox::DHCP::MSOption

COPYRIGHT

Copyright (c) 2017 Infoblox Inc.

Infoblox::Grid::MSServer::DNS - Grid Microsoft Server DNS object.

- NAME
- DESCRIPTION
- SESSION METHODS
  - Infoblox::Session->get( )
  - Infoblox::Session->modify( )
  - Infoblox::Session->search( )
- METHODS
  - manage_dns()
  - enable_dns()
  - enable_dns_reports_sync( )
  - enable_monitoring( )
  - login_name( )
  - login_password( )
  - override_enable_dns_reports_sync( )
  - override_enable_monitoring( )
  - override_login( )
  - override_synchronization_interval( )
  - status()
  - status_detail()
  - status_last_updated()
  - synchronization_interval( )
  - last_sync_ts()
  - last_sync_status()
  - last_sync_detail()
  - extattrs( )
  - extensible_attributes()
  - address()
  - name()
  - forwarders()
- AUTHOR
- SEE ALSO
- COPYRIGHT

NAME

Infoblox::Grid::MSServer::DNS - Grid Microsoft (r) Server DNS object.

DESCRIPTION

A Grid Microsoft Server DNS object is a single Microsoft DNS server managed by a Grid member.

You can use Infoblox::Grid::MSServer::DNS to configure some of the DNS properties of a Microsoft DNS server, including enabling/disabling the DNS service. The service configurations of a Grid are not inherited by this object.

SESSION METHODS

This section describes all the methods in an Infoblox::Session module that you can apply to a Microsoft Server DNS object.

Infoblox::Session->get( )

Use this method to retrieve all the matching objects from the Infoblox appliance. See Infoblox::Session->get() for parameters and return values.

Key References

```
Apply the following attributes to get a specific Grid member object:

 address     - Optional. A member address or name in FQDN (Fully Qualified Domain Name) format.
 server_ip   - Optional. A member address.
 server_name - Optional. A member name in FQDN (Fully Qualified Domain Name) format.
```

Example

```
my @retrieved_objs = $session->get(
    object    => "Infoblox::Grid::MSServer::DNS",
    server_ip => "10.0.0.1");
```

Infoblox::Session->modify( )

Use this method to modify an object in the Infoblox appliance. See Infoblox::Session->modify() for parameters and return values.

Example

```
# Use this method to modify the comment of a Grid Microsoft DNS server.
$grid_msserver_dns->enable_dns('true');
# Submit modification
my $response = $session->modify( $grid_msserver_dns );
```

Infoblox::Session->search( )

Use this method to retrieve all the matching objects from the Infoblox appliance. See Infoblox::Session->search() for parameters and return values.

Key References

```
Apply the following attributes to search for a specific Grid member object:

 server_ip   - Required. A member address.
 server_name - Optional. A member name in FQDN (Fully Qualified Domain Name) format.
```

Example

```
my @retrieved_objs = $session->search(
    object      => "Infoblox::Grid::MSServer::DNS",
    server_name => "domain.*");
```

METHODS

This section describes all the methods that you can use to configure and retrieve the attribute values of a Grid Microsoft DNS server.

manage_dns()

Retrieve or set the managed state of the Microsoft DNS server. When setting the managed state, use true to enable or false to disable synchronization of the DNS server. Setting the status does not stop or start the actual DNS server but controls whether it is synchronized.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

Specify "true" to set the disable flag or "false" to deactivate/unset it.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get manage_dns
my $manage_dns = $msserver_dns->manage_dns();
#Modify manage_dns
$msserver_dns->manage_dns("true");
```

enable_dns()

Set the DNS service status of the Microsoft DNS server. When setting the service status, use true to enable or false to disable the DNS service on the Microsoft DNS server. Setting the service status stops or starts the actual DNS service. This only applies if `manage_dns()` is true.

When setting the service status, the Microsoft DNS server will be started or stopped at the next synchronization.

Include the specified parameter to set the attribute value. This setting is read-only. To get the actual status of the service, use the `status()` method.

Parameter

Specify "true" to set the disable flag or "false" to deactivate/unset it.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get enable_dns
my $enable_dns = $msserver_dns->enable_dns();
#Modify enable_dns
$msserver_dns->enable_dns("true");
```

enable_dns_reports_sync( )

Use this method to enable or disable the synchronization of DNS reporting data for the Microsoft server.

Setting this method to a defined value implicitly sets the override_enable_dns_reports_data_sync method to "true". Setting the parameter to undefined causes the appliance to use the Grid default and automatically resets the override_enable_dns_reports_sync method to "false".

Note that when enable_dns_reports_sync is set to a defined value and override_enable_dns_reports_sync is set to "false", the last operation takes precedence. Thus the sequence $object->enable_dns_reports_sync("false"); $object->override_enable_dns_reports_sync("false"); will set override_enable_dns_reports_sync to "false", and the sequence $object->override_enable_dns_reports_sync("false"); $object->enable_dns_reports_sync("false"); will result in override_enable_dns_reports_sync="true".

Parameter

Specify "true" to enable the synchronization of DNS reporting data from the Microsoft server. Specify "false" to disable the synchronization of DNS reporting data from the Microsoft server. Default value is "false".

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get enable_dns_reports_sync
my $enable_dns_reports_sync = $msserver_dns->enable_dns_reports_sync();
#Modify enable_dns_reports_sync
$msserver_dns->enable_dns_reports_sync("true");
```

enable_monitoring( )

Use this method to enable or disable the monitoring and control of DNS service on Microsoft Server.

Setting this method to a defined value implicitly sets the override_enable_monitoring method to "true". Setting the parameter to undefined causes the appliance to use the grid default and automatically resets the override_enable_monitoring method to "false".

Note that when enable_monitoring is set to a defined value and override_enable_monitoring is set to "false", the last operation takes precedence. Thus the sequence $object->enable_monitoring("true"); $object->override_enable_monitoring("false"); will set override_enable_monitoring to "false", and the sequence $object->override_enable_monitoring("false"); $object->enable_monitoring("true"); will result in override_enable_monitoring="true".

Parameter

Specify "true" to enable monitoring and control of DNS service on all Microsoft servers in the Grid and "false" to forbid it. Default value is "false".

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get enable_monitoring value
my $enable_monitoring = $msserver_dns->enable_monitoring();
#Modify enable_monitoring value
$msserver_dns->enable_monitoring("true");
```

login_name( )

Use this method to set or retrieve the Microsoft Server DNS login name.

Setting this method to a defined value implicitly sets the override_login method to "true". Setting the parameter to undefined causes the appliance to use the Grid default and automatically resets the override_login attribute to "false".

Note that when login_name is set to a defined value and override_dlogin_name is set to "false", the last operation takes precedence. Thus the sequence $object->login_name('newuser'); $object->override_login("false"); will set override_login to "false", and the sequence $object->override_login("false"); $object->login_name('newuser'); will result in override_login="true".

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

When setting a value that requires a domain name, prepend the domain information followed by a backslash to the username: "domain\username".

Parameter

The valid value is a string that contains the login name.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get login value
my $login = $msserver_dns->login_name();
#Modify login value
$msserver_dns->login_name("newuser");
```

login_password( )

Use this method to set a password for the Microsoft Server DNS. This field is used only if the login field for this member is defined. This is a write-only attribute.

This field cannot be retrieved.

Parameter

The valid value is a string that contains the password.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

Example

```
#Set the password
$msserver_dns->login_password("iG&ojG97Y");
```

override_enable_dns_reports_sync( )

The override_enable_dns_reports_sync attribute controls whether the enable_dns_reports_sync method values of the Microsoft Server DNS object are used, instead of the Grid default.

The override_enable_dns_reports_sync attribute can be specified explicitly. It is also set implicitly when enable_dns_reports_sync is set to a defined value.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

Set the parameter to "true" to override the Grid-level setting for enable_dns_reports_sync. Set the parameter to "false" to inherit the Grid-level setting for enable_dns_reports_sync. The default value is "false".

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get override_enable_dns_reports_sync
my $override_enable_dns_reports_sync = $msserver_dns->override_enable_dns_reports_sync();
#Modify override_enable_dns_reports_sync
$msserver_dns->override_enable_dns_reports_sync("true");
```

override_enable_monitoring( )

The override_enable_monitoring attribute controls whether the enable_monitoring method values of the Microsoft Server DNS object are used, instead of the Grid default.

The override_enable_monitoring attribute can be specified explicitly. It is also set implicitly when enable_monitoring is set to a defined value.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

Set the parameter to "true" to override the Grid-level setting for enable_monitoring. Set the parameter to "false" to inherit the Grid-level setting for enable_monitoring. The default value is "false".

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get override_enable_monitoring
my $override_enable_monitoring = $msserver_dns->override_enable_monitoring();
#Modify override_enable_monitoring
$msserver_dns->override_enable_monitoring("true");
```

override_login( )

The override_login attribute controls whether the login_name method values of the Microsoft Server DNS are used, instead of the Grid default.

The override_login attribute can be specified explicitly. It is also set implicitly when login_name is set to a defined value.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

Set the parameter to "true" to override the Grid-level setting for login_name. Set the parameter to "false" to inherit the Grid-level setting for login_name. The default value is "false".

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get override_login
my $override_login = $msserver_dns->override_login();
#Modify override_login
$msserver_dns->override_login("true");
```

override_synchronization_interval( )

The override_synchronization_interval attribute controls whether the synchronization_interval method values of the Microsoft Server DNS are used, instead of the Grid default.

The override_synchronization_interval attribute can be specified explicitly. It is also set implicitly when synchronization_interval is set to a defined value.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

Set the parameter to "true" to override the Grid-level setting for synchronization_interval. Set the parameter to "false" to inherit the Grid-level setting for synchronization_interval. The default value is "false".

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get override_synchronization_interval
my $override_synchronization_interval = $msserver_dns->override_synchronization_interval();
#Modify override_synchronization_interval
$msserver_dns->override_synchronization_interval("true");
```

status()

Retrieve the running status of the DNS service of the Microsoft DNS server. Returns a status that is "running", "stopped", or "error". The returned status is the latest known status as of the date returned by the `status_last_updated()` attribute.

Returns

The method returns the attribute value.

Example

```
#Get status
my $status = $msserver_dns->status();
```

status_detail()

Retrieve the detailed running status of the DNS service of the Microsoft DNS server. The returned status is the latest known status.

Returns

The method returns the attribute value.

Example

```
#Get status
my $status = $msserver_dns->status();
```

status_last_updated()

Retrieve the time when the DNS service status of the Microsoft DNS server was last updated.

Returns

The method returns the attribute value.

Example

```
#Get status_last_updated
my $status_last_updated = $msserver_dns->status_last_updated();
```

synchronization_interval( )

Use this method to set or retrieve the synchronization interval of the Microsoft Server DNS.

Setting this method to a defined value implicitly sets the override_synchronization_interval method to "true". Setting the parameter to undefined causes the appliance to use the Grid default and automatically resets the override_synchronization_interval attribute to "false".

Note that when synchronization_interval is set to a defined value and override_dsynchronization_interval is set to "false", the last operation takes precedence. Thus the sequence $object->synchronization_interval(10); $object->override_synchronization_interval("false"); will set override_synchronization_interval to "false", and the sequence $object->override_synchronization_interval("false"); $object->synchronization_interval(10); will result in override_synchronization_interval="true".

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

The valid value is an unsigned integer.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get synchronization_interval value
my $synchronization_interval = $msserver_dns->synchronization_interval();
#Modify synchronization_interval value
$msserver_dns->synchronization_interval(10);
```

last_sync_ts()

Retrieve the timestamp of the end of the last synchronization attempt related to zone list and DNS properties.

Returns

The method returns the attribute value.

Example

```
#Get last_sync_ts
```

```
my $last_sync_ts = $msserver_dns->last_sync_ts();
```

last_sync_status()

Retrieve the result of the last synchronization attempt, if last_sync_ts is defined. Valid values are "Ok", "Warning", or "Error".

Returns

The method returns the attribute value.

Example

```
#Get last_sync_status
my $last_sync_status = $msserver_dns->last_sync_status();
```

last_sync_detail()

Retrieve the detailed result of the last synchronization attempt, if last_sync_ts is defined.

Returns

The method returns the attribute value.

Example

```
#Get last_sync_detail
my $last_sync_detail = $msserver_dns->last_sync_detail();
```

extattrs( )

Use this method to retrieve the extensible attributes associated with a Grid Microsoft DNS Server object. This attribute is read-only and cannot be set.

Parameter

None.

Returns

The method returns the attribute value.

Example

```
#Get extattrs
my $ref_extattrs = $msserver_dns->extattrs();
```

extensible_attributes()

Use this method to retrieve the extensible attributes of the associated Grid Microsoft DNS Server object.

This method is read-only. Use the Infoblox::Grid::MSServer object to set or modify the extensible attributes of the Grid Microsoft DNS Server object.

Returns

The method returns the attribute value.

Example

```
#Get extensible_attributes
my $extensible_attributes = $msserver_dns->extensible_attributes();
```

address()

Use this method to retrieve the IP address or FQDN of the associated Grid Microsoft DNS Server object.

This method is read-only. Use the Infoblox::Grid::MSServer object to set or modify the IP address of the Grid Microsoft DNS Server object.

Returns

The method returns the attribute value.

Example

```
#Get address
my $address = $msserver_dns->address();
```

name()

Use this method to retrieve the FQDN of the associated Grid Microsoft DNS Server object.

This method is read-only. Use the Infoblox::Grid::MSServer object to set or modify the IP address of the Grid Microsoft DNS Server object.

Returns

The method returns the attribute value.

Example

```
#Get address
my $name = $msserver_dns->name();
```

forwarders()

Use this method to specify forwarders for the Microsoft DNS server. A forwarder is essentially a name server to which other name servers first send all of their off-site queries. The forwarder builds up a cache of information, avoiding the need for the other name servers to send queries off-site.

This setting is a per Microsoft DNS server setting. It doesn't override the Grid-level setting and the Grid-level setting does not apply if no value is defined for this Microsoft DNS server.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

Specify an array of IPV4/IPV6 addresses to set the list of forwarders.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get forwarders
my $forwarders = $msserver_dns->forwarders();
#Modify forwarders
$msserver_dns->forwarders(['10.0.0.1','10.0.0.2']);
```

AUTHOR

Infoblox Inc. http://www.infoblox.com/

SEE ALSO

Infoblox::Grid::MSServer

COPYRIGHT

Copyright (c) 2017 Infoblox Inc.

Infoblox::Grid::NamedACL - Named ACL object.

- NAME
- DESCRIPTION
- CONSTRUCTOR
- SESSION METHODS
  - Infoblox::Session->get( )
  - Infoblox::Session->modify( )
  - Infoblox::Session->remove( )
  - Infoblox::Session->search( )
- MODULE METHODS
- METHODS
  - access_list( )
  - comment( )
  - exploded_access_list( )
  - extattrs( )
  - extensible_attributes( )
  - name( )
- SAMPLE CODE
- AUTHOR
- SEE ALSO
- COPYRIGHT

NAME

Infoblox::Grid::NamedACL - Named ACL object.

DESCRIPTION

Named ACL object represents a named list of access control entries.

CONSTRUCTOR

```
 my $nacl = Infoblox::Grid::NamedACL->new(
    'name'                  => $string,
# Required
    'comment'               => $string,
# Optional
    'access_list'           => [ $ipv4addr | $ipv4_subnet/prefix | $ipv6addr |
# Optional
                                 $ipv6_subnet/prefix | $TSIGKey | $NamedACL, ... ],
    'extattrs'              => { $string => $extattr, ... },
# Optional / Default is undefined
    'extensible_attributes' => { $string => $string | $num, $string => [ $string | $num, ... ], ... },
# Optional
 );
```

You cannot set both extattrs and extensible_attributes attributes at the same time.

SESSION METHODS

This section describes all the methods in the Infoblox::Session module that you can apply to a Named ACL object.

Infoblox::Session->get( )

Use this method to retrieve all the matching objects from the Infoblox appliance. See Infoblox::Session->get() for parameters and return values.

Key References

```
Apply the following attributes to get a specific Named ACL object:

 name    - Optional. The name of Named ACL object.
 comment - Optional. The comment in string format.
 extattrs- Optional. A hash reference containing extensible attributes.
```

Example

```
my @retrieved_objs = $session->get(
   object => 'Infoblox::Grid::NamedACL',
   name   => 'nacl',
```

```
);
```

Infoblox::Session->modify( )

Use this method to modify an object in the Infoblox appliance. See Infoblox::Session->modify() for parameters and return values.

If you have applied an access control list to an object, you must restart services after you update the access control list for the changes to take effect. See Infoblox::Session->restart() method.

Example

```
# Use this method to modify comment of Named ACL object.
$nacl->comment('new comment');
# Submit modification
my $response = $session->modify( $nacl );
```

Infoblox::Session->remove( )

Use this method to remove an object from the Infoblox appliance. See Infoblox::Session->remove() for parameters and return values.

To remove a specific object, first use `get()` or `search()` to retrieve the specific object, and then submit this object for removal.

Example

```
# Get the objects with the same name
my @retrieved_objs = $session->get(
    object => "Infoblox::Grid::NamedACL",
    name   => "nacl");
# Find the desired object from the retrieved list.
my $desired_nacl = $retrieved_objs[0];
# Submit for removal
my $response = $session->remove( $desired_nacl );
```

Infoblox::Session->search( )

Use this method to search for Named ACL objects in the Infoblox appliance. See Infoblox::Session->search() for parameters and return values.

Key References

```
Apply the following attributes to search for Named ACL objects:

 name    - Optional. The name of Named ACL object (regular expression).
 comment - Optional. The comment in string format (regular expression).
 extattrs- Optional. A hash reference containing extensible attributes.
```

Example

```
my @retrieved_objs = $session->search(
   object => 'Infoblox::Grid::NamedACL',
   name   => 'nacl',
);
```

MODULE METHODS

Retrieved Named ACL objects in an access list are considered 'partial' objects. See Infoblox::Session->fill_partial_object() for more information on how it can be converted to a 'full' API object.

METHODS

This section describes all the methods that you can use to configure and retrieve the attribute values of Named ACL object.

access_list( )

Use this method to set or retrieve an access list of Named ACL object.

Parameter

The valid value is an array reference that contains IPv4/IPv6 addresses, networks, Infoblox::DNS::TSIGKey objects and/or Infoblox::Grid::NamedACL objects.

To deny queries from certain IP addresses and networks, add "!" as a prefix to the IP addresses and networks.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
# Get attribute value
my $value = $nacl->access_list();
# Modify attribute value
$nacl->access_list(['10.0.0.1', $tsigkey, $nacl]);
```

comment( )

Use this method to set or retrieve a descriptive comment about Named ACL object.

Parameter

Desired comment in string format with a maximum of 256 bytes.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
# Get attribute value
my $value = $nacl->comment();
# Modify attribute value
$nacl->comment('modified comment');
```

exploded_access_list( )

Use this method to retrieve the exploded access list of the Named ACL object. This is a read-only attribute.

Parameter

None.

Returns

The method returns the attribute value.

The valid value is an array reference that contains IPv4/IPv6 addresses, networks and/or Infoblox::DNS::TSIGKey objects.

Example

```
# Get attribute value
my $value = $nacl->exploded_access_list();
```

extattrs( )

Use this method to set or retrieve the extensible attributes associated with a Named ACL object.

Parameter

Valid value is a hash reference containing the names of extensible attributes and their associated values (Infoblox::Grid::Extattr objects).

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get extattrs
my $ref_extattrs = $nacl->extattrs();
#Modify extattrs
$nacl->extattrs({ 'Site' => $extattr1, 'Administrator' => $extattr2 });
```

extensible_attributes( )

Use this method to set or retrieve the extensible attributes associated with Named ACL object.

Parameter

For valid values for extensible attributes, see Infoblox::Grid::ExtensibleAttributeDef/Extensible Attribute Values.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
# Get attribute value
my $value = $nacl->extensible_attributes();
# Modify attribute value
$nacl->extensible_attributes({ 'Site' => 'Santa Clara', 'Administrator' => [ 'Peter', 'Tom' ] });
```

name( )

Use this method to set or retrieve the name of the Named ACL object.

Parameter

The name of the Named ACL object in string format.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
# Get attribute value
my $value = $nacl->name();
# Modify attribute value
$nacl->name('new_nacl');
```

SAMPLE CODE

The following sample code demonstrates the different functions that can be applied to an object such as add, get, modify, search and remove. This sample also includes error handling for the operations.

#Preparation prior to getting and modifying an object

```
use strict;
use Infoblox;

#refers to Infoblox Appliance IP address
my $host_ip = "192.168.1.2";

#Create a session to the Infoblox appliance

my $session = Infoblox::Session->new(
    master   => $host_ip,
    username => "admin",
    password => "infoblox"
);
unless ($session) {
       die("Construct session failed: ",
               Infoblox::status_code() . ":" . Infoblox::status_detail());
}
print "Session created successfully\n";
```

#Create an object

```
my $nacl = Infoblox::Grid::NamedACL->new(
    name          => "NACL",
    access_list   => ['10.0.0.1', '20.0.0.0/24'],
    comment       => "Script Generated",
    );
unless ($nacl) {
       die("Construct object failed: ",
               Infoblox::status_code() . ":" . Infoblox::status_detail());
}
print "The object created successfully\n";

#Adding the object to appliance through session
my $response = $session->add( $nacl );
```

```
unless ($response) {
    die("Add object failed: ",
           $session->status_code() . ":" . $session->status_detail());
}
print"The object was added successfully \n";
```

#Search for the object

```
 my @retrieved_objs = $session->search(
     object => "Infoblox::Grid::NamedACL",
     name   => ".*");
 my $object = $retrieved_objs[0];

 unless ($object) {
        die("Search the object failed: ",
                $session->status_code() . ":" . $session->status_detail());
 }
 print "Search the object found at least 1 matching entry\n";
```

#Get and modify the object

```
#Get the object through session
my @retrieved_objs = $session->get(
    object => "Infoblox::Grid::NamedACL",
    name   => "NACL"
    );
my $nacl = $retrieved_objs[0];

unless ($nacl) {
    die("Get the object failed: ",
           $session->status_code() . ":" . $session->status_detail());
}
print"Retrieved the object successfully \n";

# Modify one of the attributes of selected object.

$nacl->comment("Modified object");

#Applying the changes
$session->modify($nacl)
   or die("Modify the object failed",
            $session->status_code() . ":" . $session->status_detail());
print"The object was modified successfully \n";
```

#Remove the object

```
#Get the object through session
my @retrieved_objs = $session->get(
    object => "Infoblox::Grid::NamedACL",
    name   => "NACL"
    );
my $desired_nacl = $retrieved_objs[0];

unless ($desired_nacl) {
    die("Get the object failed: ",
           $session->status_code() . ":" . $session->status_detail());
}
print"Retrieved the object successfully \n";

#Submit the object for removal
$session->remove( $desired_nacl )
    or die("Remove the object failed",
           $session->status_code() . ":" . $session->status_detail());
print"The object was removed successfully \n";

####PROGRAM ENDS####
```

AUTHOR

Infoblox Inc. http://www.infoblox.com/

SEE ALSO

Infoblox::Session, Infoblox::DNS::TSIGKey, Infoblox::Grid::ExtensibleAttributeDef

---

COPYRIGHT

Copyright (c) 2017 Infoblox Inc.

Infoblox::Grid::NatGroup - NAT Group object.

- NAME
- DESCRIPTION
- CONSTRUCTOR
- MODULE METHODS
  - Infoblox::Grid->nat_group_list( )
- METHODS
  - comment( )
  - name( )
- SAMPLE CODE
- AUTHOR
- SEE ALSO
- COPYRIGHT

NAME

Infoblox::Grid::NatGroup - NAT Group object.

DESCRIPTION

NAT Groups are necessary if the Grid master is behind a NAT device and there memberies behind both side of the NAT device. Any member on the same side as the master go into the same NAT group as the master and use their interface address for Grid communication with each other. Grid members on the other side of that NAT device do not go into the same NAT group as the master and use master's NAT address for Grid communication.

CONSTRUCTOR

```
#Constructor for an NAT Group object
my $nat_group = Infoblox::Grid::NatGroup->new (
        name    => $string      #Required
        comment => $string      #Optional / Default is empty
    );
```

MODULE METHODS

The following functions are available to be applied to an NatGroup object

Infoblox::Grid->nat_group_list( )

Use this function to specify a NatGroup. See Infoblox::Grid->nat_group_list() for parameters and return values.

Example

```
# Retrieve the list of NatGroup objects from Grid object.
my @retrieved_objs = $session->get(
    object => "Infoblox::Grid",
    name   => "Infoblox" );

my $Grid_obj = $retrieved_objs[0];

# construct a NatGroup object
my $nat_group = Infoblox::Grid::NatGroup->new (
        name => "my_nat_group",
        comment => "inside NAT device");

# Modify Nat Group list.
$Grid_obj->nat_group_list([$nat_group]);
```

METHODS

This section describes all the methods that you can use to configure and retrieve the attribute value of a NatGroup.

comment( )

Use this method to set or retrieve the descriptive comment.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

Desired comment in string format with a maximum of 256 bytes.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get comment
my $comment = $nat_group->comment();
#Modify comment
$nat_group->comment("new comment");
```

name( )

Use this method to set or retrieve a NAT group name

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

Desired name in string format with a maximum of 256 bytes.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

Example

```
#Get NAT group name
my $nat_group_name = $nat_group->name();
#Modify NAT group name
$nat_group->name("new_group_name");
```

SAMPLE CODE

The following sample code demonstrates different operations that can be applied to an object such as create, modify, and remove an object. This sample code also includes error handling for the operations.

```
#PROGRAM STARTS: Include all the modules that will be used
use strict;
use Infoblox;

#Create a session to the Infoblox appliance
my $session = Infoblox::Session->new(
    master   => "192.168.1.2",
    username => "admin",
    password => "infoblox"
);
unless ($session) {
   die("Construct session failed: ",
       Infoblox::status_code() . ":" . Infoblox::status_detail());
}
print "Session created successfully\n";
```

#Add a NAT Group object

```
my @result_array = $session->get( "object" => "Infoblox::Grid", "name" => "Infoblox");
if( scalar( @result_array ) > 0 )
 {
        my $Grid_obj = $result_array[0];
        unless ($Grid_obj) {
                die("Get Grid object failed: ",
        $session->status_code() . ":" . $session->status_detail());
        }
         # construct a NatGroup object
         my $nat_group = Infoblox::Grid::NatGroup->new (
              name => "my_nat_group",
              comment => "inside NAT device");

         # Add Nat Group to Grid object.
         $Grid_obj->nat_group_list([$nat_group]);
```

```
          $session->modify($Grid_obj);
          print "NAT Group added to Grid object sucessfully\n";
           }
```

#Modify existing NAT Group object

```
 @result_array = $session->get( "object" => "Infoblox::Grid", "name" => "Infoblox");
 if( scalar( @result_array ) > 0 )
  {
         my $Grid_obj = $result_array[0];
         unless ($Grid_obj) {
                 die("Get Grid object failed: ",
         $session->status_code() . ":" . $session->status_detail());
         }

          # get Nat Group from Grid object.
          my $nat_groups_list = $Grid_obj->nat_group_list();
          my $nat_group = @{$nat_groups_list}[0];

          #modify the NAT group name
          $nat_group->name("new_nat_group_name");

          # Add Nat Group to Grid object.
          $Grid_obj->nat_group_list([$nat_group]);

          $session->modify($Grid_obj);
          print "NAT Group modified on Grid object sucessfully\n";
           }
```

#Remove existing NAT Group object

```
 @result_array = $session->get( "object" => "Infoblox::Grid", "name" => "Infoblox");
 if( scalar( @result_array ) > 0 )
  {
         my $Grid_obj = $result_array[0];
         unless ($Grid_obj) {
                 die("Get Grid object failed: ",
         $session->status_code() . ":" . $session->status_detail());
         }

          # empty Nat Group list on Grid object.
          $Grid_obj->nat_group_list([]);

          $session->modify($Grid_obj);
          print "NAT Group removed on Grid object sucessfully\n";
           }

 ####PROGRAM ENDS####
```

AUTHOR

Infoblox Inc. http://www.infoblox.com/

SEE ALSO

Infoblox::Session, Infoblox::Session->get(), Infoblox::Session->modify()

COPYRIGHT

Copyright (c) 2017 Infoblox Inc.

Infoblox::Grid::NTPAccess - Grid Network Time Protocol access object.

- NAME
- DESCRIPTION
- CONSTRUCTOR
- MODULE METHODS
  - Infoblox::Grid->ntp_access_list( )
  - Infoblox::Grid::Member->ntp_access_list( )
- METHODS
  - address( )
  - service( )
- AUTHOR
- SEE ALSO
- COPYRIGHT

NAME

Infoblox::Grid::NTPAccess - Grid Network Time Protocol (NTP) access object.

DESCRIPTION

The Grid NTP access object is used to specify the access list for the NTP service.

CONSTRUCTOR

```
#Constructor for an Grid NTP access object
 my $ntp_access = Infoblox::Grid::NTPAccess->new(
    address         =>  $ipv4addr | "Any"   #Required
    service         => "Time" | "Time and NTP control"          #Optional / Default "Time"
   );
```

This constructed object can be used only when NTP is set to "true" at the grid level.

MODULE METHODS

This section describes all the functions that can be applied to an NTP access object.

Infoblox::Grid->ntp_access_list( )

Infoblox::Grid::Member->ntp_access_list( )

Use this function to specify the NTP access for the grid/member.

Example

```
# Create NTP access objects
my $ntpac1 = Infoblox::Grid::NTPAccess->new(
                                            address => "2.2.2.2",
                                            service => "Time",
                                           );

my $ntpac2 = Infoblox::Grid::NTPAccess->new(
                                            address => "4.4.4.4",
                                            service => "Time and NTP control",
                                           );


$grid->ntp_access_list([$ntpac1,$ntpac2]);

my $grid = $session->get(
   object  => "Infoblox::Grid",
   name    => "Infoblox"
 );

 unless ($grid) {
     die("get Grid failed:" ,
          $session->status_code(), $session->status_detail());
 }

$grid->ntp_access_list([$ntpac1,$ntpac2]);
```

```
#Applying the changes to appliance through session.
$session->modify($grid)
     or die("modify Grid failed:" ,
      $session->status_code(), $session->status_detail());
```

METHODS

This section describes all the methods that can be used to configure and retrieve the attribute value of an NTP access object.

address( )

Use this method to set or retrieve the IPv4 address of the external NTP client. Use "Any" to apply this permission to all clients.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

Valid values are an IPv4 address of the external NTP client, or "Any".

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get address
my $address = $ntp_access->address();
#Modify address
$ntp_access->address("4.4.4.4");
```

service( )

Use this method to specify to which service this NTP Access object applies.

Parameter

Valid values are "Time" or "Time and NTP control".

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get service
my $service = $ntp_access->service();
#Modify service
$ntp_access->service("Time and NTP control");
```

AUTHOR

Infoblox Inc. http://www.infoblox.com/

SEE ALSO

Infoblox::Session, Infoblox::Grid::Member, Infoblox::Grid,Infoblox::Session->get(), Infoblox::Session->modify()

COPYRIGHT

Copyright (c) 2017 Infoblox Inc.

Infoblox::Grid::NTPKey - Grid Network Time Protocol key object.

- NAME
- DESCRIPTION
- CONSTRUCTOR
- MODULE METHODS
  - Infoblox::Grid->ntp_authentication_key( )
  - Infoblox::Grid::Member->ntp_authentication_key( )
- METHODS
  - key_number( )
  - key_string( )
  - key_type( )
- SAMPLE CODE
- AUTHOR
- SEE ALSO
- COPYRIGHT

NAME

Infoblox::Grid::NTPKey - Grid Network Time Protocol (NTP) key object.

DESCRIPTION

Grid NTPKey object is used by NTP server to authenticate the clients.

CONSTRUCTOR

```
#Constructor for an Grid NTPKey object
 my $ntp_key = Infoblox::Grid::NTPKey->new(
    key_number    =>  $num,                  #Required
    key_string    =>  $string,               #Required
    key_type      =>  "M" | "S" | "A" | "N"  #Optional / Default is "M"
  );
```

MODULE METHODS

The following functions are available to be applied to an NTPKey object.

Infoblox::Grid->ntp_authentication_key( )

Use this function to specify a NTP key at the grid level on the Infoblox appliance. See Infoblox::Grid->ntp_authentication_key() for parameters and return values.

Example

```
#Create a NTPKey object.
my $ntp_key = Infoblox::Grid::NTPKey->new(
                  key_number => 1111,
                  key_type   => "M",
                  key_string => "aaaa"
                );

#Configure NTPKey on the Infoblox grid object
my $responce = $grid->ntp_authentication_key()([$ntp_key]);
```

Infoblox::Grid::Member->ntp_authentication_key( )

Use this function to specify a NTP key at the member level on the Infoblox appliance. See Infoblox::Grid::Member->ntp_authentication_key() for parameters and return values.

Example

```
 #Create a NTPKey object.
  my $ntp_key = Infoblox::Grid::NTPKey->new(
                  key_number => 1111,
                  key_type   => "M",
                  key_string => "aaaa"
                );

#Configure NTPKey on the Infoblox member object
```

```
my $responce = $member->ntp_authentication_key()([$ntp_key]);
```

METHODS

This section describes all the methods that you can used to configure and retrieve the attribute value of a NTPKey object.

key_number( )

Use this method to set or retrieve the key number of the NTPKey object.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

A positive integer number. Value must be between 1 and 65534.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get key_number
my $number = $ntp_key->key_number();
#Modify key_number
$ntp_key->key_number("3333");
```

key_string( )

Use this method to set or retrieve the key string of the NTPKey object.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

Desired key_string in string format defined in the key_type.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get key_string
my $string = $ntp_key->key_string();
#Modify key_string
$ntp_key->key_string("d3e54352e5548080");
```

key_type( )

Use this method to set or retrieve the key type of the NTPKey object.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

Valid values are :"M","S","N","A".The default value is "M".

```
 "M" - For  Message Digest (MD5) in ASCII format and the key_string must be a 1-31 character ASCII
using
          MD5;
 "S" - For Data Encryption Standard (DES) in hex format and key_string must be a 64-bit hexadecimal
         number in DES. The high order 7 bits of each octet form the 56-bit key, and the low order
bit of each
        octet is given a value,  so that the octet maintains odd parity. You must specify leading
zeros so the key
        is exactly 16 hexadecimal digits long and maintains odd parity;

 "A" - For Data Encryption Standard (DES) in ASCII format and the key_string must be a 1-8 character
          ASCII using DES;
 "N" - For Data Encryption Standard (DES) in NTP format and the key_string must be a 64-bit
hexadecimal in
          NTP format. It is the same as the "S" format, but the bits in each octet have been rotated
```

```
one bit right,
          so the parity bit is in the high order bit of the octet. You must specify leading zeros and
odd parity must
          be maintained.
```

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get key_type
my $type = $ntp_key->key_type();
#Modify key_type
$ntp_key->ntp_key("N");
```

SAMPLE CODE

The following sample code demonstrates different operations that can be applied to an object such as create, modify, and remove an object. Also this sample code also includes error handling for the operations.

```
#PROGRAM STARTS: Include all the modules that will be used
 use strict;
 use Infoblox;

 #Create a session to the Infoblox appliance
 my $session = Infoblox::Session->new(
    master   => "192.168.1.2",
    username => "admin",
    password => "infoblox"
 );
 unless ($session) {
   die("Construct session failed: ",
       Infoblox::status_code() . ":" . Infoblox::status_detail());
 }
 print "Session created successfully\n";
```

#Adding an NTPKey object

```
#Create a NTPKey object.
my $ntp_key = Infoblox::Grid::NTPKey->new(
   key_number => 65533,
   key_type   => "M",
   key_string => "aaaa"
);

unless($ntp_key) {
       die("Construct NTP Key object failed: ",
            Infoblox::status_code() . ":" . Infoblox::status_detail());
}
print "NTP Key object created successfully\n";

#Create a NTPServer object.
 my $ntp_server = Infoblox::Grid::NTPServer->new(
    address        => "3.3.3.3",
);

unless($ntp_server) {
       die("Construct NTP server object failed: ",
            Infoblox::status_code() . ":" . Infoblox::status_detail());
}
print "NTP Server object created successfully\n";

my @grid_objs = $session->get(
   object  => "Infoblox::Grid",
   name    => "Infoblox"
 );

 my $grid = $grid_objs[0];
 unless ($grid) {
       die("Get Grid object failed: ",
            $session->status_code() . ":" . $session->status_detail());
  }
  print "Get Grid object found at least 1 matching entry\n";
```

```
$grid->enable_ntp("true");
$grid->ntp_authentication_key([$ntp_key]);
$grid->ntp_server([$ntp_server]);

#Applying the changes to appliance through session.
$session->modify($grid)
     or die("modify Grid failed:" ,
      $session->status_code(), $session->status_detail());
print "Grid modified successfully for adding NTPKey object\n";
```

#Modifying an NTPKey object

```
my $ntp_key = Infoblox::Grid::NTPKey->new(
   key_number => 65532,
   key_type   => "M",
   key_string => "aaaa"
);

my @grid_objs = $session->get(
           object => "Infoblox::Grid",
           name   => "Infoblox"
       );

my $grid = $grid_objs[0];
unless ($grid) {
       die("Get Grid object failed: ",
            $session->status_code() . ":" . $session->status_detail());
}
print "Get Grid object found at least 1 matching entry\n";

$grid->ntp_authentication_key([$ntp_key]);

#Applying the changes to appliance through session.
$session->modify($grid)
     or die("modify Grid failed:" ,
      $session->status_code(), $session->status_detail());
print "Grid modified successfully for modifying NTPKey object\n";
```

#Removing an NTPKey object

```
#Get Infoblox grid object
my @grid_objs = $session->get(
           object => "Infoblox::Grid",
           name   => "Infoblox"
       );

my $grid = $grid_objs[0];
unless ($grid) {
       die("Get Grid object failed: ",
            $session->status_code() . ":" . $session->status_detail());
}
print "Get Grid object found at least 1 matching entry\n";

$grid->ntp_authentication_key([]);
#Applying the changes to appliance through session.
$session->modify($grid)
    or die("modify Grid failed for removing NTPKey object:" ,
     $session->status_code(), $session->status_detail());
print "Grid modified successfully for removing NTPKey object\n";
####PROGRAM ENDS####
```

AUTHOR

Infoblox Inc. http://www.infoblox.com/

SEE ALSO

Infoblox::Session, Infoblox::Grid::NTPServer, Infoblox::Grid,Infoblox::Session->get(), Infoblox::Session->modify()

COPYRIGHT

Copyright (c) 2017 Infoblox Inc.

Infoblox::Grid::NTPServer - Grid Network Time Protocol server object.

- NAME
- DESCRIPTION
- CONSTRUCTOR
- MODULE METHODS
  - Infoblox::Grid->ntp_server( )
- METHODS
  - address( )
  - authentication( )
  - burst( )
  - iburst( )
  - ntp_key( )
- SAMPLE CODE
- AUTHOR
- SEE ALSO
- COPYRIGHT

NAME

Infoblox::Grid::NTPServer - Grid Network Time Protocol (NTP) server object.

DESCRIPTION

The Grid NTP server object is used to synchronize the date and time for an Infoblox appliance.

CONSTRUCTOR

```
#Constructor for an Grid NTP server object
 my $ntp_server = Infoblox::Grid::NTPServer->new(
    address        =>  $ipv4addr | $ipv6addr | $fqdn    #Required
    authentication => "true" | "false",                 #Optional / Default "false"
    burst          => "true" | "false",                 #Optional / Default "true"
    iburst         => "true" | "false",                 #Optional / Default "true"
    ntp_key        => [$NTPKey1, $NTPKey2,...]          #Optional / Default is undefined
   );
```

This constructed object can be used only when NTP is set to "true" at the grid level.

MODULE METHODS

This section describes all the functions that can be applied to a NTP server object.

Infoblox::Grid->ntp_server( )

Use this function to specify the NTP server for the grid.

Example

```
#Create a NTPKey object.
my $ntp_key1 = Infoblox::Grid::NTPKey->new(
   key_number => 1111,
   key_type   => "M",
   key_string => "aaaa"
);

#Create a NTPServer object.
 my $ntp_server = Infoblox::Grid::NTPServer->new(
    address        => "3.3.3.3",
    authentication => "true",
    ntp_key        => $ntp_key1
);

my $grid = $session->get(
   object  => "Infoblox::Grid",
   name    => "Infoblox"
 );

 unless ($grid) {
     die("get Grid failed:" ,
```

```
            $session->status_code(), $session->status_detail());
 }

$grid->enable_ntp("true");
$grid->ntp_authentication_key([$ntp_key1]);
$grid->ntp_server([$ntp_server]);

#Applying the changes to appliance through session.
$session->modify($grid)
      or die("modify Grid failed:" ,
       $session->status_code(), $session->status_detail());
```

METHODS

This section describes all the methods that can be used to configure and retrieve the attribute value of a NTP server object.

address( )

Use this method to set or retrieve the IPv4 or IPv6 address or FQDN of the external NTP server.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

Valid values are IPv4 or IPv6 address or FQDN of the external NTP server.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get address
my $address = $ntp_server->address();
#Modify address
$ntp_server->address("4.4.4.4");
```

authentication( )

Use this method to enable or disable the authentication for an NTP server.

Parameter

Specify "true" to enable authentication or "false" to disable it. Default value is "false".

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get authentication
my $authentication = $ntp_server->authentication();
#Modify authentication
$ntp_server->authentication("false");
```

burst( )

Use this method to enable or disable BURST operating mode. In BURST operating mode, when the external server is reachable and a valid source of synchronization is available, NTP sends a burst of 8 packets with a 2 second interval between packets.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

Specify "true" to eanble the BURST operating mode or "false" to disable it. The default value is "true".

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get burst
my $burst = $ntp_server->burst();
#Modify burst
$ntp_server->burst("false");
```

iburst( )

Use this method to enable or disable IBURST operating mode. In IBURST operating mode, when the external server is unreachable, NTP server sends a burst of 8 packets with a 2 second interval between packets.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

Specify "true" to eanble the IBURST operating mode or "false" to disable it. The default value is "true".

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get iburst
my $iburst = $ntp_server->iburst();
#Modify iburst
$ntp_server->iburst("false");
```

ntp_key( )

Use this method to set or retrieve the ntp_key used for authentication.

Parameter

Valid value is an array reference that contains Infoblox::Grid::NTPKey objects.

See Infoblox::Grid::NTPKey for parameters and return values.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Create a NTPKey object.
my $ntp_key2 = Infoblox::Grid::NTPKey->new(
  key_number => 2222,
  key_type   => "n",
  key_string =>"d3e54352e5548080" );

#Modify NTPServer object
my $ntp_server = Infoblox::Grid::NTPServer->new(
   address        => "3.3.3.3",
   authentication => "true",
   ntp_key        => $ntp_key2
);
```

SAMPLE CODE

The following sample code demonstrates different operations that can be applied to an object such as create, modify, and remove an object. Also this sample code also includes error handling for the operations.

```
#PROGRAM STARTS: Include all the modules that will be used
 use strict;
 use Infoblox;

 #Create a session to the Infoblox appliance
 my $session = Infoblox::Session->new(
    master   => "192.168.1.2",
    username => "admin",
    password => "infoblox"
 );
```

```
  unless ($session) {
    die("Construct session failed: ",
        Infoblox::status_code() . ":" . Infoblox::status_detail());
  }
  print "Session created successfully\n";
```

#Adding an NTP server object #Get the grid object. my $result = $session->get(object => "Infoblox::Grid", name => "Infoblox"); unless ($result) { die("get Grid failed:" , $session->status_code(), $session->status_detail()); }

```
  #Create a NTPKey object.
  my $ntp_key1 = Infoblox::Grid::NTPKey->new(
      key_number => 3333,
      key_type   => "S",
      key_string => "a7cb86a4cba80101"
  );

  #Create a NTPServer object.
  my $ntp_server = Infoblox::Grid::NTPServer->new(
      address        => "3.3.3.3",
      authentication => "true",
      ntp_key        => $ntp_key1
  );
   #Modifying the value of the ntp_authentication_key and ntp_server methods, from the grid object.
  $result->enable_ntp("true");
  $result->ntp_authentication_key([$ntp_key1]);
  $result->ntp_server([$ntp_server]);

  #Applying the changes to appliance through session.
  $session->modify($result)
       or die("modify Grid failed:" ,
        $session->status_code(), $session->status_detail());
   print "\n  Modify Grid successful \n";
```

#Modifying an NTP server object #Create a NTPServer object. my $ntp_server = Infoblox::Grid::NTPServer->new(address => "3.3.3.3", authentication => "false",);

```
 $result->ntp_server([$ntp_server]);

 #Applying the changes to appliance through session.
 $session->modify($result)
      or die("modify Grid failed:" ,
       $session->status_code(), $session->status_detail());
 print "\n  Modify grid successful for NTP server object\n";
```

#Removing an NTP server object #Get the grid object. my $result = $session->get(object => "Infoblox::Grid", name => "Infoblox"); unless ($result) { die("get Grid failed:" , $session->status_code(), $session->status_detail()); }

```
 $result->enable_ntp("false");
 $result->ntp_server([]);
 #Applying the changes to appliance through session.
 $session->modify($result)
      or die("modify Grid failed:" ,
       $session->status_code(), $session->status_detail());
 print "\n  Modify grid successful for NTP server object\n";

 ####PROGRAM ENDS####
```

AUTHOR

Infoblox Inc. http://www.infoblox.com/

SEE ALSO

Infoblox::Session, Infoblox::Grid::NTPKey, Infoblox::Grid,Infoblox::Session->get(), Infoblox::Session->modify()

COPYRIGHT

Copyright (c) 2017 Infoblox Inc.

Infoblox::Grid::ObjectsChangesTrackingSetting - the object changes tracking setting.

- NAME
- DESCRIPTION
- CONSTRUCTOR
- MODULE METHODS
  - Infoblox::Grid->objects_changes_tracking_setting( )
- METHODS
  - enable( )
  - enable_completion( )
  - max_objs_to_track( )
  - max_time_to_track( )
  - state( )
- AUTHOR
- SEE ALSO
- COPYRIGHT

NAME

Infoblox::Grid::ObjectsChangesTrackingSetting - the object changes tracking setting.

DESCRIPTION

The object changes tracking setting object is used to display the object changes state and to enable or disable the object changes tracking feature.

CONSTRUCTOR

```
my $object_changes = Infoblox::Grid::ObjectsChangesTrackingSetting->new(
    enable => 'true' | 'false', #Optional / Default is 'false'
);
```

MODULE METHODS

+The following functions can be applied to an object changes tracking setting object.

Infoblox::Grid->objects_changes_tracking_setting( )

Use this function to specify the object changes tracking settings for the Grid. See Infoblox::Grid/objects_changes_tracking_setting() for parameters and return values.

Example

```
#Get objects_changes_tracking_setting value
my $value = $object->objects_changes_tracking_setting();

#Construct objects_changes_tracking_setting value
my $objects_changes_tracking_setting = Infoblox::Grid::ObjectsChangesTrackingSetting->new(
    enable => 'true',
);

#Modify objects_changes_tracking_setting value
$object->objects_changes_tracking_setting($objects_changes_tracking_setting);
```

METHODS

This section describes all the methods that you can use to set and retrieve the attribute values of an object changes tracking setting object.

enable( )

Use this method to set or retrieve the flag that indicates whether the object changes tracking feature is enabled or disabled.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

Specify 'true' to enable the object changes tracking feature or 'false' to disable it.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get enable value
my $value = $object->enable();

#Modify enable value
$object->enable('true');
```

enable_completion( )

Use this method to retrieve the object changes tracking completion percentage. This is a read-only attribute.

Omit the parameter to retrieve the attribute value.

Parameter

None

Returns

The valid return value is an unsigned integer between 0 and 100.

Example

```
#Get enable_completion value
my $value = $object->enable_completion();
```

max_objs_to_track( )

Use this method to set or retrieve the maximum number of objects retained for change tracking.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

The valid value is an unsigned integer.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get max_objs_to_track value
my $value = $object->max_objs_to_track();

#Modify max_objs_to_track value
$object->max_objs_to_track(10);
```

max_time_to_track( )

Use this method to set or retrieve the maximum time period (in seconds) to track the deleted object changes.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

The valid value is an unsigned integer that represents time period (in seconds).

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get max_time_to_track value
my $value = $object->max_time_to_track();

#Modify max_time_to_track value
```

```
$object->max_time_to_track(10);
```

state( )

Use this method to retrieve the object changes tracking state. This is a read-only attribute.

Omit the parameter to retrieve the attribute value.

Parameter

None

Returns

The valid values are 'ENABLING', 'ENABLED', 'DISABLED' and 'ENABLING_ERROR'.

Example

```
#Get state value
my $value = $object->state();
```

AUTHOR

Infoblox Inc. http://www.infoblox.com/

SEE ALSO

Infoblox::Grid, Infoblox::Grid/objects_changes_tracking_setting()

COPYRIGHT

Copyright (c) 2017 Infoblox Inc.

Infoblox::Grid::ParentalControl::MSP - A Parental Control managed service provider object.

- NAME
- DESCRIPTION
- CONSTRUCTOR
- MODULE METHODS
  - Infoblox::Grid::ParentalControl::SubscriberSite->msps( )
- METHODS
  - address( )
- AUTHOR
- SEE ALSO
- COPYRIGHT

NAME

Infoblox::Grid::ParentalControl::MSP - A Parental Control managed service provider object.

DESCRIPTION

The Parental Control managed service provider object is used to configure list of service providers that are managed by the Parental Control subscriber site.

CONSTRUCTOR

```
my $nas_gateway = Infoblox::Grid::ParentalControl::MSP->new(
    address => $ipv4addr, # Required
);
```

MODULE METHODS

The following functions are available to be applied to a Parental Control managed service provider object.

Infoblox::Grid::ParentalControl::SubscriberSite->msps( )

Use this method to set or retrieve the list of Parental Control managed service providers.

See Infoblox::Grid::ParentalControl::SubscriberSite->msps() for parameters and return values.

Example

```
#Get msps value
my $value = $object->msps();

#Construct Parental Control MSP
my $msp = Infoblox::Grid::ParentalControl::MSP->new(
    address => '10.0.10.10',
);

#Modify msps value
$object->msps([$msp]);
```

METHODS

This section describes all the methods that can be used to configure and retrieve the attribute values of a Parental Control managed service provider object.

address( )

Use this method to set or retrieve the managed service provider IP address.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

The valid value is desired IPv4 address in a string format.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get address value
my $address = $object->address();

#Modify address value
$object->address('10.0.0.10');
```

AUTHOR

Infoblox Inc. http://www.infoblox.com/

SEE ALSO

Infoblox::Grid::ParentalControl::SubscriberSite, Infoblox::Grid::ParentalControl::SubscriberSite->msps(),

---

COPYRIGHT

Copyright (c) 2017 Infoblox Inc.

Infoblox::Grid::ParentalControl::NAS::AVP - A NAS accounting value pair object.

- NAME
- DESCRIPTION
- CONSTRUCTOR
- SESSION METHODS
  - Infoblox::Session->add( )
  - Infoblox::Session->get( )
  - Infoblox::Session->modify( )
  - Infoblox::Session->remove( )
  - Infoblox::Session->search( )
- METHODS
  - comment( )
  - domain_types( )
  - is_restricted( )
  - name( )
  - type( )
  - user_defined( )
  - value_type( )
  - vendor_id( )
  - vendor_type( )
- AUTHOR
- SEE ALSO
- COPYRIGHT

NAME

Infoblox::Grid::ParentalControl::NAS::AVP - A NAS accounting value pair object.

DESCRIPTION

The NAS accounting value pair object is used in Parental Control Subscriber configuration for extracting accounting information from accounting protocols logs.

CONSTRUCTOR

```
my $avp = Infoblox::Grid::ParentalControl::NAS::AVP->new(
    name          => $string,                                          #Required
    value_type    => 'BYTE' | 'DATE' | 'INTEGER' | 'INTEGER64',
                     | 'IPADDR' | 'IPV6ADDR' | 'IPV6IFID' | 'IPV6PREFIX',
                     | 'OCTETS' | 'SHORT' | 'STRING'                   #Required
    type          => $uint,                                            #Required
    vendor_id     => $uint,                                            #Optional / Default is undefined
    vendor_type   => $uint,                                            #Optional / Default is undefined
    comment       => $string,                                          #Optional / Default is undefined
    is_restricted => 'true' | 'false',                                 #Optional / Default is 'false'
    user_defined  => 'true' | 'false',                                 #Optional / Default is 'true'

);
```

SESSION METHODS

This section describes all the methods in the Infoblox::Session module that you can apply to a accounting value pair object.

Infoblox::Session->add( )

Use this method to add an object to the Infoblox appliance. See Infoblox::Session->add() for parameters and return values.

Example

```
#Construct NAS AVP object
my $nas_avp = Infoblox::Grid::ParentalControl::NAS::AVP->new(
    name        => 'address_avp1',
    value_type  => 'ADDRESS',
    type        => 10,
    vendor_id   => 1,
    vendor_type => 20,
);

#Submit for addition
```

```
my $response = $session->add($nas_avp);
```

Infoblox::Session->get( )

Use this method to retrieve all the matching objects from the Infoblox appliance. See Infoblox::Session->get() for parameters and return values.

Key Reference

Apply the following attributes to get a specific NAS AVP object:

```
name        - Optional. The string that contains NAS AVP object name.
value_type  - Optional. The string that represents the NAS AVP value type.
type        - Optional. The unsinged integer that represents the NAS AVP type.
vendor_id   - Optional. The unsigned integer that represents the NAS AVP vendor identifier.
vendor_type - Optional. The unsigned integer that represents the NAS AVP vendor type.
comment     - Optional. The descriptive comment in string format.
```

Example

```
#Get object matching the given name
my $object = $session->get(
    object => 'Infoblox::Grid::ParentalControl::NAS::AVP',
    name   => 'address_avp1',
);
```

Infoblox::Session->modify( )

Use this method to modify an object in the Infoblox appliance. See Infoblox::Session->modify() for parameters and return values.

Example

```
#Modify vendor_id value
$object->vendor_id(10);

#Submit modification
my $response = $session->modify($object);
```

Infoblox::Session->remove( )

Use this method to remove an object from the Infoblox appliance. See Infoblox::Session->remove() for parameters and return values.

To remove a specific object, use `get()` or `search()` to retrieve the specific object first, and then submit this object for removal.

Example

```
#Get the NAS AVP object matching the given name
my @retrieved_objs = $session->get(
    object => 'Infoblox::Grid::ParentalControl::NAS::AVP',
    name   => 'address_avp1',
);

#Find the desired object from the retrieved list
my $object = @retrieved_objs[0];

#Submit for removal
my $response = $session->remove($object);
```

Infoblox::Session->search( )

Use this method to retrieve all the matching objects from the Infoblox appliance. See Infoblox::Session->search() for parameters and return values.

Key Reference

Apply the following attributes to get a specific NAS AVP object:

```
name        - Optional. The string that contains NAS AVP object name (regexp).
value_type  - Optional. The string that represents the NAS AVP value type (exact value).
type        - Optional. The unsinged integer that represents the NAS AVP type (exact value).
vendor_id   - Optional. The unsigned integer that represents the NAS AVP vendor identifier (exact
value).
vendor_type - Optional. The unsigned integer that represents the NAS AVP vendor type (exact value).
comment     - Optional. The descriptive comment in string format (regexp).
```

Example

```
#Search for object matching the given name pattern
my @objects = $session->search(
    object => 'Infoblox::Grid::ParentalControl::NAS::AVP',
```

```
        name    => 'address_avp.*',
);
```

METHODS

This section describes all the methods that can be used to configure and retrieve the attribute values of a accounting value pair object.

comment( )

Use this method to set or retrieve the descriptive comment for the AVP.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

The valid value is a desired comment in a string format.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get comment value
my $comment = $object->comment();

#Modify comment value
$object->comment('comment changed');
```

domain_types( )

Use this method to set or retrieve the domain types that are applicable to AVP.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

The valid value is array that contains following values: 'ANCILLARY', 'IP_SPACE_DIS', 'NAS_CONTEXT', 'SUBS_ID'.

By default all domain types are applicable for AVP.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get domain_types value
my $domain_types = $object->domain_types();

#Modify domain_types value
$object->domain_types(['ANCILLARY', 'IP_SPACE_DIS']);
```

is_restricted( )

Use this method to set or retrieve the flag that indicates whether the AVP is restricted to particular AVP domains or not.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

Specify 'true' to enable AVP domain restriction AVP and 'false' to disable it. The default value is 'false'.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get is_restricted value
my $is_restricted = $object->is_restricted();
```

```
#Modify is_restricted value
$object->is_restricted('true');
```

name( )

Use this method to set or retrieve the AVP name.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

The valid value is a desired AVP name in a string format.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get name value
my $name = $object->name();

#Modify name value
$object->name('address_avp1');
```

type( )

Use this method to set or retrieve the AVP vendor type.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

The valid value is an unsigned integer between 1 and 255.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get type value
my $type = $object->type();

#Modify type value
$object->type(11);
```

user_defined( )

Use this method to retrieve the flag that indicates whether this AVP is user-defined or a system default. This is a read-only attribute.

Parameter

None

Returns

The method returns the attribute value.

Example

```
#Get user_defined value
my $user_defined = $object->user_defined();
```

value_type( )

Use this method to set or retrieve the AVP value type.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

The valid values are 'BYTE', 'DATE', 'INTEGER', 'INTEGER64', 'IPADDR', 'IPV6ADDR', 'IPV6IFID', 'IPV6PREFIX', 'OCTETS', 'SHORT' and 'STRING'.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get value_type value
my $value_type = $object->value_type();

#Modify value_type value
$object->value_type('DATE');
```

vendor_id( )

Use this method to set or retrieve the AVP vendor identifier.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

The valid value is an unsigned integer between 1 and 65535.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get vendor_id value
my $vendor_id = $object->vendor_id();

#Modify vendor_id value
$object->vendor_id(11);
```

vendor_type( )

Use this method to set or retrieve the AVP vendor specific type.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

The valid value is an unsigned integer between 1 and 255.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get vendor_type value
my $vendor_type = $object->vendor_type();

#Modify vendor_type value
$object->vendor_type(11);
```

AUTHOR

Infoblox Inc. http://www.infoblox.com/

SEE ALSO

Infoblox::Session, Infoblox::Session->add(), Infoblox::Session->get(), Infoblox::Session->modify(), Infoblox::Session->remove(), Infoblox::Session->search(),

---

COPYRIGHT

Copyright (c) 2017 Infoblox Inc.

Infoblox::Grid::ParentalControl::NAS::Gateway - A Parental Control network accounting server gateway object.

- NAME
- DESCRIPTION
- CONSTRUCTOR
- MODULE METHODS
  - Infoblox::Grid::ParentalControl::SubscriberSite->nas_gateways( )
- METHODS
  - address( )
  - comment( )
  - message_rate( )
  - name( )
  - send_ack( )
  - shared_secret( )
- AUTHOR
- SEE ALSO
- COPYRIGHT

NAME

Infoblox::Grid::ParentalControl::NAS::Gateway - A Parental Control network accounting server gateway object.

DESCRIPTION

The Parental Control network accounting server (NAS) gateway object provides configuration for transmission of authentication protocol accounting infromation to the NIOS accounting collector.

CONSTRUCTOR

```
my $nas_gateway = Infoblox::Grid::ParentalControl::NAS::Gateway->new(
    name          => $string,                    # Required
    address       => $ipv4addr,                  # Required
    shared_secret => $string,                    # Optional
    send_ack      => 'true' | 'false',           # Optional / Default is 'false'
    comment       => $string,                    # Optional / Default is undefined
);
```

MODULE METHODS

The following functions are available to be applied to a Parental Control network accounting server gateway object.

Infoblox::Grid::ParentalControl::SubscriberSite->nas_gateways( )

Use this method to set or retrieve the NAS gateways associated with subscriber site.

See Infoblox::Grid::ParentalControl::SubscriberSite->nas_gateways() for parameters and return values.

Example

```
#Get nas_gateways value
my $nas_gateways = $object->nas_gateways();

#Construct first gateway
my $gw1 = Infoblox::Grid::ParentalControl::NAS::Gateway->new(
    name          => 'gateway-1',
    address       => '10.1.1.1',
    shared_secret => 'secret',
);

#Construct first gateway
my $gw2 = Infoblox::Grid::ParentalControl::NAS::Gateway->new(
    name          => 'gateway-2',
    address       => '2001:db8::1',
    shared_secret => 'secret',
    send_ack      => 'true',
);

#Modify nas_gateways value
$object->nas_gateways([$gateway1, $gateway2]);
```

METHODS

This section describes all the methods that can be used to configure and retrieve the attribute values of a Parental Control network accounting server gateway object.

address( )

Use this method to set or retrieve the NAS gateway IP address.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

The valid value is desired IPv4 address in a string format.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get address value
my $address = $object->address();

#Modify address value
$object->address('10.0.0.10');
```

comment( )

Use this method to set or retrieve the NAS gateway comment.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

The valid value is a desired comment in a string format.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get comment value
my $comment = $object->comment();

#Modify comment value
$object->comment('nas gateway comment');
```

message_rate( )

Use this method to retrieve the vendor specific accounting messages rate. This is a read-only attribute.

Omit the parameter to retrieve the attribute value.

Parameter

None

Returns

The method returns the attribute value.

Example

```
#Get message_rate value
my $message_rate = $object->message_rate();
```

name( )

Use this method to set or retrieve the NAS gateway object name.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

The valid value is a desired name in a string format.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get name value
my $name = $object->name();

#Modify name value
$object->name('accounting_log_server');
```

send_ack( )

Use this method to set or retrieve the flag that indicates whether the acknowledgement of the accounting packets is enabled or disabled.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

Specify 'true' to enable acknowledgement of the accounting packets or 'false' to disable it. The default value is 'false'.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get send_ack value
my $send_ack = $object->send_ack();

#Modify send_ack value
$object->send_ack('true');
```

shared_secret( )

Use this method to set or retrieve the MD5 phrase for the accounting data. This is a write-only attribute.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

The valid value is a desired MD5 phrase in a string format.

Returns

The method returns true when the modification succeeds, and returns false when the operation fails.

Example

```
#Modify shared_secret value
$object->shared_secret('secret');
```

AUTHOR

Infoblox Inc. http://www.infoblox.com/

SEE ALSO

Infoblox::Grid::ParentalControl::SubscriberSite, Infoblox::Grid::ParentalControl::SubscriberSite->nas_gateways(),

---

COPYRIGHT

Copyright (c) 2017 Infoblox Inc.

Infoblox::Grid::ParentalControl::SiteMember - A Parental Control site member object.

- NAME
- DESCRIPTION
- CONSTRUCTOR
- MODULE METHODS
  - Infoblox::Grid::ParentalControl::SubscriberSite->members( )
- METHODS
  - name( )
  - type( )
- AUTHOR
- SEE ALSO
- COPYRIGHT

NAME

Infoblox::Grid::ParentalControl::SiteMember - A Parental Control site member object.

DESCRIPTION

The Parental Control site member object associate particular Grid member with it's role in process of collecting and consuming the authentication information and the Parental Control site the feature is deployed on.

CONSTRUCTOR

```
my $site_member = Infoblox::Grid::ParentalControl::SiteMember->new(
    name => $string,                                  #Required
    type => 'COLLECTOR' | 'NONE' | 'BOTH' | 'CLIENT', #Optional / Default is 'NONE'
);
```

MODULE METHODS

The following functions are available to be applied to a Parental Control site member object.

Infoblox::Grid::ParentalControl::SubscriberSite->members( )

Use this method to set or retrieve the Grid members associatied with subscriber site.

See Infoblox::Grid::ParentalControl::SubscriberSite->members() for parameters and return values.

Example

```
#Get members value
my $members = $object->members();

#Construct first member
my $m1 = Infoblox::Grid::ParentalControl::SiteMember->new(
    name => 'infoblox.localdomain',
    type => 'NONE',
);

#Construct second member
my $m2 = Infoblox::Grid::ParentalControl::SiteMember->new(
    name => 'imc-collector',
    type => 'COLLECTOR',
);

#Modify members value
$object->members([$m1, $m2]);
```

METHODS

This section describes all the methods that can be used to configure and retrieve the attribute values of a Parental Control site member object.

name( )

Use this method to set or retrieve the name of the member.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

The valid value is a desired name in a string format.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get name value
my $name = $object->name();

#Modify name value
$object->name('my_collector');
```

type( )

Use this method to set or retrieve the type of the member.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

The valid values are 'BOTH', 'COLLECTOR', 'CLIENT' and 'NONE'. The default value is 'NONE'.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get type value
my $type = $object->type();

#Modify type value
$object->type('CLIENT');
```

AUTHOR

Infoblox Inc. http://www.infoblox.com/

SEE ALSO

Infoblox::Grid::ParentalControl::SubscriberSite, Infoblox::Grid::ParentalControl::SubscriberSite->members(),

---

COPYRIGHT

Copyright (c) 2017 Infoblox Inc.

Infoblox::Grid::ParentalControl::Subscriber - Parental Control subscriber properties.

- NAME
- DESCRIPTION
- SESSION METHODS
  - Infoblox::Session->get( )
  - Infoblox::Session->modify( )
- METHODS
  - ancillaries( )
  - enable_parental_control( )
  - enable_mgmt_only_nas( )
  - interim_accounting_interval( )
  - ip_anchors( )
  - ip_space_discriminator( )
  - ip_space_discriminator_regexp( )
  - ip_space_discriminator_subexpression( )
  - nas_context_info( )
  - subscriber_id( )
  - subscriber_id_regexp( )
  - subscriber_id_subexpression( )
- SEE ALSO
- COPYRIGHT

NAME

Infoblox::Grid::ParentalControl::Subscriber - Parental Control subscriber properties.

DESCRIPTION

The Parental Control subscriber properties contains user defined Radius subscriber information of which will be used by DNS/RPZ for reporting and logging violations.

Note that this object cannot be added or deleted. Only get and modify operations are supported.

SESSION METHODS

This section describes all the methods in the Infoblox::Session module that you can apply to the object.

Infoblox::Session->get( )

Use this method to retrieve all the matching objects from the Infoblox appliance. See Infoblox::Session->get() for parameters and return values.

Example

```
#Get object
my $object = $session->get(
    object => 'Infoblox::Grid::ParentalControl::Subscriber',
);
```

Infoblox::Session->modify( )

Use this method to modify an object in the Infoblox appliance. See Infoblox::Session->modify() for parameters and return values.

Example

```
#Modify interim_accounting_interval value
$object->interim_accounting_interval(20);

#Submit modification
my $response = $session->modify($object);
```

METHODS

This section describes all the methods that can be used to configure and retrieve the attribute values of the object.

ancillaries( )

Use this method to set or retrieve the ordered list of ancillary AVP fields.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

The valid value is an array reference that contains existing AVP names in string format.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get ancillaries value
my $ancillaries = $object->ancillaries();

#Modify ancillaries value
$object->ancillaries(['avp1', 'avp2']);
```

enable_parental_control( )

Use this method to set or retrieve the flag that indicates whether the Parental Control feature is enabled.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

Specify 'true' to enable Parental Control and 'false to disable it. The default value is 'false'.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get enable_parental_control value
my $enable_parental_control = $object->enable_parental_control();

#Modify enable_parental_control value
$object->enable_parental_control('true');
```

enable_mgmt_only_nas( )

Use this method to set or retrieve the flag that indicates whether the NAS RADIUS traffic is accepted over MGMT port only.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

Specify 'true' to restrict NAS RADIUS traffic to MGMT port only, and 'false' to disable this restriction. The default value is 'false'.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get enable_mgmt_only_nas value
my $enable_mgmt_only_nas = $object->enable_mgmt_only_nas();

#Modify enable_mgmt_only_nas value
$object->enable_mgmt_only_nas('true');
```

interim_accounting_interval( )

Use this method to set or retrieve the time that is required for collector to be fully populated.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

The valid value is unsigned integer between 1 and 200.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get interim_accounting_interval value
my $interim_accounting_interval = $object->interim_accounting_interval();

#Modify interim_accounting_interval value
$object->interim_accounting_interval(20);
```

ip_anchors( )

Use this method to set or retrieve the ordered list of IP anchors AVPs.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

The valid value is an array reference that contains existing AVP names in string format.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get ip_anchors value
my $ip_anchors = $object->ip_anchors();

#Modify ip_anchors value
$object->ip_anchors(['avp1', 'avp2']);
```

ip_space_discriminator( )

Use this method to set or retrieve the IP space discriminator AVP.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

The valid value is existing AVP name in string format.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get ip_space_discriminator value
my $ip_space_discriminator = $object->ip_space_discriminator();

#Modify ip_space_discriminator value
$object->ip_space_discriminator('avp1');
```

ip_space_discriminator_regexp( )

Use this method to set or retrieve the regular expression for IP space discriminator information extraction.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

The valid value is an unsigned integer in string format.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get ip_space_discriminator_regexp value
```

```
my $ip_space_discriminator_regexp = $object->ip_space_discriminator_regexp();

#Modify ip_space_discriminator_regexp value
$object->ip_space_discriminator_regexp('regexp');
```

ip_space_discriminator_subexpression( )

Use this method to set or retrieve the regular expression capturing group number to extract the IP space discriminator information.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

The valid value is unsigned integer between 0 and 8. 0 group returns full matched expression.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get ip_space_discriminator_subexpression value
my $ip_space_discriminator_subexpression = $object->ip_space_discriminator_subexpression();

#Modify ip_space_discriminator_subexpression value
$object->ip_space_discriminator_subexpression(2);
```

nas_context_info( )

Use this method to set or retrieve the subscriber NAS context information AVP.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

The valid value is existing AVP name in string format.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get nas_context_info value
my $nas_context_info = $object->nas_context_info();

#Modify nas_context_info value
$object->nas_context_info('avp1');
```

subscriber_id( )

Use this method to set or retrieve the subscriber ID AVP.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

The valid value is existing AVP name in string format.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get subscriber_id value
my $subscriber_id = $object->subscriber_id();

#Modify subscriber_id value
$object->subscriber_id('avp1');
```

subscriber_id_regexp( )

Use this method to set or retrieve the regular expression for extracting subscriber ID information.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

The valid value is desired regular expression in string format.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get subscriber_id_regexp value
my $subscriber_id_regexp = $object->subscriber_id_regexp();

#Modify subscriber_id_regexp value
$object->subscriber_id_regexp('regexp');
```

subscriber_id_subexpression( )

Use this method to set or retrieve the regular expression capturing group number to extract the subscriber ID information.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

The valid value is unsigned integer between 0 and 8. 0 group returns full matched expression.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get subscriber_id_subexpression value
my $subscriber_id_subexpression = $object->subscriber_id_subexpression();

#Modify subscriber_id_subexpression value
$object->subscriber_id_subexpression(2);
```

SEE ALSO

Infoblox::Session, Infoblox::Session->get(), Infoblox::Session->modify(),

---

COPYRIGHT

Copyright (c) 2017 Infoblox Inc.

Infoblox::Grid::ParentalControl::SubscriberSite - a Parental Control subscriber site object

- NAME
- DESCRIPTION
- CONSTRUCTOR
- SESSION METHODS
  - Infoblox::Session->add( )
  - Infoblox::Session->get( )
  - Infoblox::Session->modify( )
  - Infoblox::Session->remove( )
  - Infoblox::Session->search( )
- METHODS
  - comment( )
  - blocking_ipv4_vip1( )
  - blocking_ipv4_vip2( )
  - blocking_ipv6_vip1( )
  - blocking_ipv6_vip2( )
  - extattrs( )
  - extensible_attributes( )
  - maximum_subscribers( )
  - members( )
  - msps( )
  - name( )
  - nas_gateways( )
  - nas_port( )
- AUTHOR
- SEE ALSO
- COPYRIGHT

NAME

Infoblox::Grid::ParentalControl::SubscriberSite - a Parental Control subscriber site object

---

DESCRIPTION

CONSTRUCTOR

```
 my $subscriber_site = Infoblox::Grid::ParentalControl::SubscriberSite->new(
    name                  => $string,                                                  #
Required
    maximum_subscribers   => $uint,                                                    #
Required
    nas_port              => $uint,                                                    #
Required
    comment               => $string,                                                  #
Optional / Default is undefined
    members               => [$site_member1, $site_member2, ...],                      #
Optional / Default is empty list
    msps                  => [$msp1, $msp2, ...],                                      #
Optional / Default is empty list
    nas_gateways          => [$gateway1, $gateway2, ...],                              #
Optional / Default is empty list
    blocking_ipv4_vip1    => $ipv4addr,                                                #
Optional / Default is undefined
    blocking_ipv6_vip1    => $ipv6addr,                                                #
Optional / Default is undefined
    blocking_ipv4_vip2    => $ipv4addr,                                                #
Optional / Default is undefined
    blocking_ipv6_vip2    => $ipv6addr,                                                #
Optional / Default is undefined
    extattrs              => { $string => $extattr, ... },                             #
Optional / Default is undefined
    extensible_attributes => {$string => $string | $num, $string => [ $string | $num, ... ], ... } #
Optional / Default is undefined
 );
```

You cannot set both extattrs and extensible_attributes attributes at the same time.

SESSION METHODS

The following functions can be applied to a Infoblox::Grid::ParentalControl::SubscriberSite object.

Infoblox::Session->add( )

Use this method to add an object to the Infoblox appliance. See Infoblox::Session->add() for parameters and return values.

Example

```
#Construct an object
my $object = Infoblox::Grid::ParentalControl::SubscriberSite->new(
   name                => 'site1',
   maximum_subscribers => '20000',
   nas_port            => '1919',
   comment             => 'site1 comment',
);

#Submit for addition
my $response = $session->add($object);
```

Infoblox::Session->get( )

Use this method to retrieve all the matching objects from the Infoblox appliance. See Infoblox::Session->get() for parameters and return values.

Key References

```
 Apply the following attributes to get a specific Infoblox::Grid::ParentalControl::SubscriberSite
object.
  name                - Optional. The Parental Control subscriber site name.
  comment             - Optional. The Parental Control subscriber site comment.
  extattrs            - Optional. A hash reference containing Infoblox::Grid::Extattr object.
  extensible_attributes - Optional. A hash reference containing extensible attributes.
  blocking_ipv4_vip1    - Optional. The Parental Control primary VIP IPv4 address of the blocking
server.
  blocking_ipv6_vip1    - Optional. The Parental Control primary VIP IPv6 address of the blocking
server.
  blocking_ipv4_vip2    - Optional. The Parental Control secondary VIP IPv4 address of the blocking
server.
  blocking_ipv6_vip2    - Optional. The Parental Control secondary VIP IPv6 address of the blocking
server.
```

Example

```
# search for the object which name is 'test.com'
my @retrieved_objs = $session->get(
    'object' => 'Infoblox::Grid::ParentalControl::SubscriberSite',
    'name'   => 'test.com',
);
```

Infoblox::Session->modify( )

Use this method to modify an object in the Infoblox appliance. See Infoblox::Session->modify() for parameters and return values.

Example

```
# modify object object
$object->comment('comment change');

# Submit modification
my $response = $session->modify($object);
```

Infoblox::Session->remove( )

Use this method to remove an object from the Infoblox appliance. See Infoblox::Session->remove() for parameters and return values.

To remove a specific object, first use `get()` or `search()` to retrieve the specific object, and then submit this object for removal.

Example

```
#Get the objects with the same name
my @retrieved_objs = $session->get(
    object => 'Infoblox::Grid::ParentalControl::SubscriberSite',
    name   => 'site1');

#Find the desired object from the retrieved list.
my $desired_object = $retrieved_objs[0];

# Submit for removal
my $response = $session->remove($desired_object);
```

Infoblox::Session->search( )

Use this method to search for objects in the Infoblox appliance. See Infoblox::Session->search() for parameters and return values.

Key References

```
 Apply the following attributes to search for a specific
Infoblox::Grid::ParentalControl::SubscriberSite object.
  name                  - Optional. The Parental Control subscriber site name (regexp).
  comment               - Optional. The Parental Control subscriber site comment (regexp).
  extattrs              - Optional. A hash reference containing Infoblox::Grid::Extattr object.
  extensible_attributes - Optional. A hash reference containing extensible attributes.
  blocking_ipv4_vip1    - Optional. The Parental Control primary VIP IPv4 address of the blocking
server (regexp).
  blocking_ipv6_vip1    - Optional. The Parental Control primary VIP IPv6 address of the blocking
server (regexp).
  blocking_ipv4_vip2    - Optional. The Parental Control secondary VIP IPv4 address of the blocking
server (regexp).
  blocking_ipv6_vip2    - Optional. The Parental Control secondary VIP IPv6 address of the blocking
server (regexp).
```

Example

```
 # search for objects which name contains test and primary IPv6 address of the blocking server
contains '2:'
 my @retrieved_objs = $session->search(
     'object'             => 'Infoblox::Grid::ParentalControl::SubscriberSite',
     'name'               => 'test.*',
     'blocking_ipv6_vip1' => '2:.*',
 );

 # search for objects defining 'Santa Clara' for 'Site' extensible attribute
 my @retrieved_objs = $session->search(
     object               => 'Infoblox::Grid::ParentalControl::SubscriberSite',
     extensible_attributes => {'Site' => 'Santa Clara'},
 );
```

METHODS

This section describes all the methods that can be used to configure and retrieve the attribute values of a Parental Control subscriber site object.

comment( )

Use this method to set or retrieve Parental Control subscriber site comment.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

The valid value is desired comment in string format.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
 #Get comment
 my $value = $object->comment();

 #Modify comment
 $object->comment('new comment');
```

blocking_ipv4_vip1( )

Use this method to set or retrieve the primary IPv4 address of the blocking server.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

The valid value is a desired IPv4 address in a string format.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get blocking_ipv4_vip1 value
my $blocking_ipv4_vip1 = $object->blocking_ipv4_vip1();

#Modify blocking_ipv4_vip1 value
$object->blocking_ipv4_vip1('10.0.10.1');
```

blocking_ipv4_vip2( )

Use this method to set or retrieve the secondary IPv4 address of the blocking server.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

The valid value is a desired IPv4 address in a string format.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get blocking_ipv4_vip2 value
my $blocking_ipv4_vip2 = $object->blocking_ipv4_vip2();

#Modify blocking_ipv4_vip2 value
$object->blocking_ipv4_vip2('10.0.10.1');
```

blocking_ipv6_vip1( )

Use this method to set or retrieve the primary IPv6 address of the blocking server.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

The valid value is a desired IPv6 address in a string format.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get blocking_ipv6_vip1 value
my $blocking_ipv6_vip1 = $object->blocking_ipv6_vip1();

#Modify blocking_ipv6_vip1 value
$object->blocking_ipv6_vip1('2001:db8::1');
```

blocking_ipv6_vip2( )

Use this method to set or retrieve the secondary IPv6 address of the blocking server.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

The valid value is a desired IPv6 address in a string format.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get blocking_ipv6_vip2 value
my $blocking_ipv6_vip2 = $object->blocking_ipv6_vip2();

#Modify blocking_ipv6_vip2 value
$object->blocking_ipv6_vip2('2002:db8::1');
```

extattrs( )

Use this method to set or retrieve the extensible attributes associated with a Parental Control subscriber site object.

Parameter

Valid value is a hash reference that contains the names of extensible attributes and their associated values (Infoblox::Grid::Extattr objects).

Returns

If you have specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get extattrs value
my $extattrs = $object->extattrs();

#Modify extattrs value
$object->extattrs({'Site' => $extattr1, 'Administrator' => $extattr2});
```

extensible_attributes( )

Use this method to set or retrieve the extensible attributes associated with a Parental Control subscriber site object.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

For valid values for extensible attributes, see Infoblox::Grid::ExtensibleAttributeDef/Extensible Attribute Values.

Returns

If you have specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get extensible attributes value
my $extensible_attributes = $object->extensible_attributes();
#Modify extensible attributes
$object->extensible_attributes({'Site' => 'Santa Clara', 'Administrator' => ['Peter', 'Tom']});
```

maximum_subscribers( )

Use this method to set or retrieve the port number to reach the collector.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

The valid value is an unsigned integer between 10 000 and 10 000 000.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get maximum_subscribers value
my $maximum_subscribers = $object->maximum_subscribers();

#Modify maximum_subscribers value
$object->maximum_subscribers('20000');
```

members( )

Use this method to set or retrieve the Grid members associatied with subscriber site.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

The valid value is an array of Infoblox::Grid::ParentalControl::SiteMember.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Construct first member
my $m1 = Infoblox::Grid::ParentalControl::SiteMember->new(
    name => 'infoblox.localdomain',
    type => 'NONE',
);

#Construct second member
my $m2 = Infoblox::Grid::ParentalControl::SiteMember->new(
    name => 'imc-collector',
    type => 'COLLECTOR',
);

#Get members value
my $members = $object->members();

#Modify members value
$object->members([$m1, $m2]);
```

msps( )

Use this method to set or retrieve the list of Parental Control managed service providers.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

The valid value is an array of Infoblox::Grid::ParentalControl::MSP objects.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Exmaple

```
#Get msps value
my $value = $object->msps();

#Construct Parental Control MSP
my $msp = Infoblox::Grid::ParentalControl::MSP->new(address => '10.0.10.10');

#Modify msps value
$object->msps([$msp]);
```

name( )

Use this method to set or retrieve a name of the Parental Control subscriber site.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

The valid value is a desired name in string format.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Exmaple

```
#Get name value
my $value = $object->name();
```

```
#Modify name value
$object->name('test2');
```

nas_gateways( )

Use this method to set or retrieve the NAS gateways associated with subscriber site.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

The valid value is an array of Infoblox::Grid::ParentalControl::NAS::Gateway.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get nas_gateways value
my $nas_gateways = $object->nas_gateways();

#Construct first gateway
my $gw1 = Infoblox::Grid::ParentalControl::NAS::Gateway->new(
    name          => 'gateway-1',
    address       => '10.1.1.1',
    shared_secret => 'secret',
);

#Construct first gateway
my $gw2 = Infoblox::Grid::ParentalControl::NAS::Gateway->new(
    name          => 'gateway-2',
    address       => '2001:db8::1',
    shared_secret => 'secret',
    send_ack      => 'true',
);

#Modify nas_gateways value
$object->nas_gateways([$gw1, $gw2]);
```

nas_port( )

Use this method to set or retrieve the port number to reach the collector.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

The valid value is an unsigned integer between 1 and 65535.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get nas_port value
my $nas_port = $object->nas_port();

#Modify nas_port value
$object->nas_port('1919');
```

AUTHOR

Infoblox Inc. http://www.infoblox.com/

SEE ALSO

Infoblox::Session, Infoblox::Session->add(), Infoblox::Session->get(), Infoblox::Session->remove(), Infoblox::Session->search(), Infoblox::Session->modify(), Infoblox::Grid::ParentalControl::SiteMember, Infoblox::Grid::ParentalControl::MSP, Infoblox::Grid::ParentalControl::NAS::Gateway

---

COPYRIGHT

Copyright (c) 2017 Infoblox Inc.

Infoblox::Grid::PersonalSmartFolder - Personal Smart Folder object.

- NAME
- DESCRIPTION
- CONSTRUCTOR
- SESSION METHODS
  - Infoblox::Session->add( )
  - Infoblox::Session->get( )
  - Infoblox::Session->modify( )
  - Infoblox::Session->remove( )
  - Infoblox::Session->search( )
- METHODS
  - comment( )
  - group_bys( )
  - name( )
  - query_items( )
- SAMPLE CODE
- AUTHOR
- SEE ALSO
- COPYRIGHT

NAME

Infoblox::Grid::PersonalSmartFolder - Personal Smart Folder object.

DESCRIPTION

A Personal Smart Folder object is used for data organization.

CONSTRUCTOR

```
 my $group = Infoblox::Grid::PersonalSmartFolder->new(
     name                => $string,                                          #Required
     comment             => $string,                                          #Optional
/ Default is undefined
     group_bys           => [$GroupbyItem1, $GroupbyItem2],                   #Optional
/ Default is undefined
     query_items         => [$QueryItem1, $QueryItem2],                       #Optional
/ Default is undefined
 );
```

SESSION METHODS

This section describes all the methods in the Infoblox::Session module that you can apply to a Personal Smart Folder object.

Infoblox::Session->add( )

Use this method to add an object to the Infoblox appliance. See Infoblox::Session->add() for parameters and return values.

Example

```
#Construct the QueryItem in-memory object
my $query_item1 = Infoblox::Grid::SmartFolder::QueryItem->new(
    name                => "location",
    operator            => "eq",
    value               => "Santa Clara",
);

#Construct the PersonalSmartFolder object
my $group_by_state = Infoblox::Grid::SmartFolder::GroupBy->new(
                                                              enable_grouping => 'true',
                                                              value => "State",
                                                              value_type => "EXTATTR",
                                                             );

my $smart_folder = Infoblox::Grid::PersonalSmartFolder->new(
    name                => "smartfolder1",
    comment             => "personal smart folder",
    group_bys           => [$group_by_state],
    query_items         => [$query_item1],
```

```
);
# Submit for addition
my $response = $session->add( $smart_folder );
```

Infoblox::Session->get( )

Use this method to retrieve all the matching objects from the Infoblox appliance. See Infoblox::Session->get() for parameters and return values.

Key References

```
Apply the following attributes to get a specific PersonalSmartFolder object:

 name      - Required. PersonalSmartFolder name in string format.
 comment   - Optional. Comment in string format.
```

Example

```
my @retrieved_objs = $session->get(
    object     => "Infoblox::Grid::PersonalSmartFolder",
    name       => "smartfolder1" );
```

Infoblox::Session->modify( )

Use this method to modify an object in the Infoblox appliance. See Infoblox::Session->modify() for parameters and return values.

Example

```
# Use this method to modify the comment.
$smart_folder->comment("This is a modified comment");
# Submit modification
my $response = $session->modify( $smart_folder );
```

Infoblox::Session->remove( )

Use this method to remove an object from the Infoblox appliance. See Infoblox::Session->remove() for parameters and return values.

To remove a specific object, use `get()` or `search()` to retrieve the specific object first, and then submit this object for removal.

Example

```
# Get the PersonalSmartFolder objects using a name
my @retrieved_objs = $session->get(
    object     => "Infoblox::Grid::PersonalSmartFolder",
    name       => "smartfolder1" );
# find the desired object from the retrieved list.
my $desired_smart_folder = $retrieved_objs[0];
# Submit for removal
my $response = $session->remove( $desired_smart_folder );
```

Infoblox::Session->search( )

Use this method to search for Personal Smart Folder objects in the Infoblox appliance. See Infoblox::Session->search() for parameters and return values.

Key References

```
Apply the following attributes to search for a specific PersonalSmartFolder object:

 name      - Optional. PersonalSmartFolder name in string format (regular expression).
 comment   - Optional. Comment in string format (regular expression).
```

Example

```
# search for all PersonalSmartFolder objects that start with "smartfolder"
my @retrieved_objs = $session->search(
    object => "Infoblox::Grid::PersonalSmartFolder",
    name   => "smartfolder.*" );
```

METHODS

This section describes all the methods that you can use to set and retrieve the attribute values of a PersonalSmartFolder object.

comment( )

Use this method to set or retrieve the comment.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

Pertinent information about the personal smart folder. Comment in string format with a maximum of 256 bytes. The default value is undefined.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get comment
my $comment = $smart_folder->comment();
#Modify comment
$smart_folder->comment("This is the modified comment for smartfolder1");
```

group_bys( )

Use this method to set or retrieve the group_bys value.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

The valid value is an array reference that contains Infoblox::Grid::SmartFolder::GroupBy object(s). The default value is undefined.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
my $group_by_state = Infoblox::Grid::SmartFolder::GroupBy->new(
                                                              enable_grouping => 'true',
                                                              value => "State",
                                                              value_type => "EXTATTR",
                                                             );

#Get group_by value
my $group_by = $smart_folder->group_by();
#Modify group_by value
$smart_folder->group_bys([$group_by_state]);
```

name( )

Use this method to set or retrieve the name of a personal smart folder.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

Text with the name of the personal smart folder.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get name
my $name = $smart_folder->name();
```

query_items( )

Use this method to set or retrieve the query items. If query items are not specified when you `add()` a smart folder, the server automatically assigns a default query item for the smart folder. This default query item is "type=Network/Zone/Range/Member". This ensures that objects are included in searches for these 4 types, by default. When you `modify()` a smart folder, the server keeps the query items provided by the client and does not add the default items.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

The valid value is an array reference that contains Infoblox::Grid::SmartFolder::QueryItem object(s). The default value is undefined.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get query items
my $query_items = $smart_folder->query_items();

#Modify query items
my $query_item1 = Infoblox::Grid::SmartFolder::QueryItem->new(
    name                => "location",
    operator            => "eq",
    value               => "Santa Clara",
);
$smart_folder->query_items([$query_items1]);
```

SAMPLE CODE

The following sample code demonstrates the different functions that can be applied to an object, such as add, search, modify, and remove. This sample also includes error handling for the operations.

#Preparation prior to an PersonalSmartFolder object insertion

```
#PROGRAM STARTS: Include all the modules that will be used
use strict;
use Infoblox;

#Create a session to the Infoblox appliance

my $session = Infoblox::Session->new(
              master   => "192.168.1.2", #appliance host ip
              username => "admin",       #appliance user login
              password => "infoblox"     #appliance password
              );

unless ($session) {
       die("Construct session failed: ",
              Infoblox::status_code() . ":" . Infoblox::status_detail());
}
print "Session created successfully\n";

# Add a network with extensible attributes

my %extensible_attributes=(
    'Region' => 'Santa Clara County',
    'Country' => 'USA',
    'State' => 'California',
    'Site' => 'Santa Clara',
    'Building' => 'HQ',
    'VLAN' => '1',
);

my $network = Infoblox::DHCP::Network->new(
    network => "10.0.0.0/24",
    extensible_attributes => \%extensible_attributes,
);
unless ($network) {
    die("Construct Network failed: ",
        Infoblox::status_code() . ":" . Infoblox::status_detail());
}
print "Network created successfully\n";

$session->add($network)
    or die("Add Network object failed: ",
    $session->status_code() . ":" . $session->status_detail());
print "Network object added to server successfully\n";
```

#Create a PersonalSmartFolder object

```
my $query_item = Infoblox::Grid::SmartFolder::QueryItem->new(
    name                    => "Country",
    is_extensible_attribute => "true",
    operator                => "eq",
    value                   => "USA",
);

unless ($query_item) {
    die("Construct query item object failed: ",
        Infoblox::status_code() . ":" . Infoblox::status_detail());
}
print "Smart folder query item object created successfully\n";

my $smart_folder = Infoblox::Grid::PersonalSmartFolder->new(
    name                    => "my_folder_personal",
    comment                 => "my personal folder",
    query_items             => [$query_item],
);

unless ($smart_folder) {
    die("Construct personal smart folder object failed: ",
        Infoblox::status_code() . ":" . Infoblox::status_detail());
}
print "Smart folder object created successfully\n";

#Add the PersonalSmartFolder object to the Infoblox appliance through a session
$session->add($smart_folder)
    or die("Add PersonalSmartFolder object failed: ",
    $session->status_code() . ":" . $session->status_detail());
print "PersonalSmartFolder object added to server successfully\n";
```

#Search for a PersonalSmartFolder

```
my @retrieved_objs = $session->search(
    object => "Infoblox::Grid::PersonalSmartFolder",
    name   => "my_folder.*"
);
my $object = $retrieved_objs[0];

unless ($object) {
    die("Search for a PersonalSmartFolder object failed: ",
        $session->status_code() . ":" . $session->status_detail());
}
print "Search for a PersonalSmartFolder object found at least 1 matching entry\n";
```

#Get and modify a PersonalSmartFolder object

```
#Get the PersonalSmartFolder object from Infoblox appliance through a session
my @retrieved_objs = $session->get(
    object => "Infoblox::Grid::PersonalSmartFolder",
    name   => "my_folder_personal"
);
my $object = $retrieved_objs[0];

unless ($object) {
    die("Get PersonalSmartFolder object failed: ",
        $session->status_code() . ":" . $session->status_detail());
}
print "Get PersonalSmartFolder object found at least 1 matching entry\n";

#Modify the PersonalSmartFolder object
$object->name("my_folder_personal_renamed");
$object->comment("modified personal smart folder");

#Apply the changes.
$session->modify($object)
    or die("Modify PersonalSmartFolder object failed: ",
    $session->status_code() . ":" . $session->status_detail());
print "PersonalSmartFolder object modified successfully \n";
```

#Remove a PersonalSmartFolder object

```
#Get the PersonalSmartFolder object through the session
my @retrieved_objs = $session->get(
    object => "Infoblox::Grid::PersonalSmartFolder",
    name   => "my_folder_personal_renamed"
```

```
);
my $object = $retrieved_objs[0];
unless ($object) {
    die("Get PersonalSmartFolder object failed: ",
        $session->status_code() . ":" . $session->status_detail());
}
print "Get PersonalSmartFolder object found at least 1 matching entry\n";

#Submit the object for removal
$session->remove($object)
    or die("Remove PersonalSmartFolder object failed: ",
    $session->status_code() . ":" . $session->status_detail());
print "PersonalSmartFolder object removed successfully \n";
```

#Cleanup

```
#Get the Network object through the session
my @retrieved_objs = $session->get(
    object  => "Infoblox::DHCP::Network",
    network => "10.0.0.0/24"
);
my $object = $retrieved_objs[0];
unless ($object) {
    die("Get Network object failed: ",
        $session->status_code() . ":" . $session->status_detail());
}
print "Get Network object found at least 1 matching entry\n";

#Submit the object for removal
$session->remove($object)
    or die("Remove Network object failed: ",
    $session->status_code() . ":" . $session->status_detail());
print "Network object removed successfully \n";

####PROGRAM ENDS####
```

AUTHOR

Infoblox Inc. http://www.infoblox.com/

SEE ALSO

Infoblox::Grid::SmartFolderChildren, Infoblox::Grid::PersonalSmartFolder, Infoblox::Session->add(), Infoblox::Session->get(), Infoblox::Session->modify(), Infoblox::Session->remove(), Infoblox::Session->search(),Infoblox::Session

---

COPYRIGHT

Copyright (c) 2017 Infoblox Inc.

Infoblox::Grid::RecordNamePolicy - Manages the DNS record name policies defined at the grid level.

- NAME
- DESCRIPTION
- CONSTRUCTOR
- MODULE METHODS
  - Infoblox::Grid::DNS->record_name_policies( )
- METHODS
  - name( )
  - regex( )
- SAMPLE CODE
- AUTHOR
- SEE ALSO
- COPYRIGHT

NAME

Infoblox::Grid::RecordNamePolicy - Manages the DNS record name policies defined at the grid level.

DESCRIPTION

Record name policy defines the DNS record name format restrictions at the grid level.

CONSTRUCTOR

```
my $policy = Infoblox::Grid::RecordNamePolicy->new(
             name  => $string, #Required
             regex => $regex   #Required
);
```

MODULE METHODS

The following functions are available to apply to the record name policy object.

Infoblox::Grid::DNS->record_name_policies( )

Use this function to specify a record name policy at the grid level in the Infoblox appliance.

Example

```
#Create a RecordNamePolicy object.
my $policy = Infoblox::Grid::RecordNamePolicy->new(
                 name  => "test",
                 regex => "^[a-zA-Z0-9]+\$",
               );

#Configure record name policy on the Infoblox grid object
my $response = $grid_dns->record_name_policies([$policy]);
```

METHODS

This section describes all the methods that can be used to configure and retrieve the attributes values of a record name policy object.

name( )

Use this method to set or retrieve the record name policy name.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

String with the name of the record name policy.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get name
my $name = $policy->name();
#Modify name
$policy->name("new_name");
```

regex( )

Use this method to set or retrieve the record name policy in regular expression format.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

The record name policy in regular expression format. It should follow POSIX regular expression rules.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get regex
my $regex = $policy->regex();
#Modify regex
$policy->regex("^[a-zA-Z0-9]*\$");
```

SAMPLE CODE

The following sample code demonstrates different operations that can be applied to a record name policy object, such as create, modify, and remove. This sample code also includes error handling for the operations.

```
#PROGRAM STARTS: Include all the modules that will be used
use strict;
use Infoblox;

#Create a session to the Infoblox appliance
my $session = Infoblox::Session->new(
    master   => "192.168.1.2",
    username => "admin",
    password => "infoblox"
);
unless ($session) {
   die("Construct session failed: ",
       Infoblox::status_code() . ":" . Infoblox::status_detail());
}
print "Session created successfully\n";
```

#Create a record name policy object

```
my $policy1 = Infoblox::Grid::RecordNamePolicy->new(
    name  => "Allow Any",
    regex => ".+",
    );
unless ($policy1) {
     die("Construct record name policy1 object failed: ",
           Infoblox::status_code() . ":" . Infoblox::status_detail());
}
print "Record name policy1 object created successfully\n";

my $policy2 = Infoblox::Grid::RecordNamePolicy->new(
      name  => "Allow Underscore",
      regex => "^[-a-zA-Z0-9_.]+\$",
    );

unless ($policy2) {
     die("Construct record name policy1 object failed: ",
           Infoblox::status_code() . ":" . Infoblox::status_detail());
}
print "Record name policy2 object created successfully\n";

my $policy3 = Infoblox::Grid::RecordNamePolicy->new(
      name  => "Strict Hostname Checking",
```

```
      regex => "^[a-zA-Z0-9]\$|^[a-zA-Z0-9][-a-zA-Z0-9.]*[a-zA-Z0-9]\$",
    );

unless ($policy3) {
     die("Construct record name policy3 object failed: ",
           Infoblox::status_code() . ":" . Infoblox::status_detail());
}
print "Record name policy3 object created successfully\n";

my $policy4 = Infoblox::Grid::RecordNamePolicy->new(
    name  => "test",
    regex => "^[a-zA-Z0-9]+\$",
    );
unless ($policy4) {
     die("Construct record name policy3 object failed: ",
           Infoblox::status_code() . ":" . Infoblox::status_detail());
}
print "Record name policy4 object created successfully\n";
```

#Get the Member DNS object to add a record name policy object to it

```
my $grid_dns = $session->get( "object" => "Infoblox::Grid::DNS", "grid" => "Infoblox");

unless ($grid_dns) {
         die("Get Grid DNS failed: ",
             $session->status_code() . ":" . $session->status_detail());
 }
$grid_dns->record_name_policies([$policy1,$policy2,$policy3,$policy4]);
$session->modify($grid_dns)
      or die("modify Grid DNS failed:" ,
      $session->status_code(), $session->status_detail());
print "Modify grid DNS successful for record name policy object\n";

####PROGRAM ENDS####
```

AUTHOR

Infoblox Inc. http://www.infoblox.com/

SEE ALSO

Infoblox::Grid::DNS, Infoblox::Session, Infoblox::Session->get(), Infoblox::Session->modify()

COPYRIGHT

Copyright (c) 2017 Infoblox Inc.

Infoblox::Grid::RestartBannerSetting - The restart banner setting.

- NAME
- DESCRIPTION
- CONSTRUCTOR
- MODULE METHODS
  - Infoblox::Grid->restart_banner_setting( )
- METHODS
  - enable( )
  - enable_double_confirmation( )
- AUTHOR
- SEE ALSO
- COPYRIGHT

NAME

Infoblox::Grid::RestartBannerSetting - The restart banner setting.

DESCRIPTION

The Restart Banner Setting object controls the banner displayed for Grid restart.

CONSTRUCTOR

```
my $restart = Infoblox::Grid::RestartBannerSetting->new(
    enable                    => "true" | "false", #Mandatory
    enable_double_confirmation => "true" | "false", #Optional / Default is "false"
);
```

MODULE METHODS

The following functions are available to apply to a restart banner setting object.

Infoblox::Grid->restart_banner_setting( )

Use this function to specify the restart banner settings for the Grid. See Infoblox::Grid->restart_banner_setting() for parameters and return values.

Example

```
my $grid = $session->get(object => 'Infoblox::Grid');

my $banner = Infoblox::Grid::RestartBannerSetting->new(enable => 'true');
$grid->restart_banner_setting($banner);
my $response = $session->modify($grid);
unless($response) {
     die("Modify grid failed: ",
           session->status_code() . ":" . session->status_detail());
}
print "Grid banner setting modified successfully\n";
```

METHODS

This section describes all the methods that you can use to set and retrieve the attribute values of a Restart Banner Setting object.

enable( )

Use this method to set the enable flag.

Parameter

Specify "true" to enable the banner or "false" to disable it.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get enable
my $enable = $banner->enable();
#Modify enable
$banner->enable("true");
```

enable_double_confirmation( )

Use this method to set the enable_double_confirmation flag. If this flag is set to "true" the user is required to input name before restarting services.

Parameter

Specify "true" to enable double confirmation or "false" to disable it.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get enable_double_confirmation
my $enable_double_confirmation = $banner->enable_double_confirmation();
#Modify enable_double_confirmation
$banner->enable_double_confirmation("true");
```

AUTHOR

Infoblox Inc. http://www.infoblox.com/

SEE ALSO

Infoblox::Grid

COPYRIGHT

Copyright (c) 2017 Infoblox Inc.

Infoblox::Grid::RIR - Regional Internet Registry object.

- NAME
- DESCRIPTION
- CONSTRUCTOR
- SESSION METHODS
  - Infoblox::Session->get( )
  - Infoblox::Session->modify( )
  - Infoblox::Session->search( )
- METHODS
  - communication_mode( )
  - name( )
  - email( )
  - url( )
  - override_email( )
  - override_url( )
- AUTHOR
- SEE ALSO
- COPYRIGHT

NAME

Infoblox::Grid::RIR - Regional Internet Registry (RIR) object.

DESCRIPTION

Regional Internet Registry (RIR) object.

CONSTRUCTOR

```
my $rir = Infoblox::Grid::RIR->new(
   communication_mode    => 'API' | 'EMAIL' | 'NONE', # Required
   name                  => 'RIPE',                   # Optional, Default is 'RIPE'
   email                 => $string,                  # Optional
   override_email        => 'true' | 'false',         # Optional, Default is 'false'
   url                   => $string,                  # Optional
   override_url          => 'true' | 'false',         # Optional, Default is 'false'
);
```

SESSION METHODS

This section describes all the methods in an Infoblox::Session module that you can apply to a RIR object.

Support for RIR/SWIP must be enabled at the Grid level. See Infoblox::Grid->enable_rir_swip() method.

Infoblox::Session->get( )

Use this method to retrieve all the matching objects from the Infoblox appliance. See Infoblox::Session->get() for parameters and return values.

Key References

```
Apply the following attributes to get a specific RIR object:

 name    - Optional. The RIR name.
```

Example

```
my @retrieved_objs = $session->get(
   object => 'Infoblox::Grid::RIR',
   name   => 'RIPE',
);
```

Infoblox::Session->modify( )

Use this method to modify an object in the Infoblox appliance. See Infoblox::Session->modify() for parameters and return values.

Example

```
# Use this method to modify email of a RIR object.
```

```
$rir->email('support@infoblox.com');
# Submit modification
my $response = $session->modify( $rir );
```

Infoblox::Session->search( )

Use this method to search for RIR objects in the Infoblox appliance. See Infoblox::Session->search() for parameters and return values.

Key References

```
Apply the following attributes to search a specific RIR object:

 name         - Optional. The RIR name.
```

Example

```
my @retrieved_objs = $session->search(
   object => 'Infoblox::Grid::RIR',
   name   => 'RIPE',
);
```

METHODS

This section describes all the methods that you can use to configure and retrieve the attribute values of a RIR object.

communication_mode( )

Use this method to set or retrieve a communication mode for RIR.

Parameter

Valid values are 'API', 'EMAIL' and 'NONE'. The default value is 'API'.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get attribute value
my $value = $rir->communication_mode();
#Modify attribute value
$rir->communication_mode('EMAIL');
```

name( )

Use this method to set or retrieve a name of RIR.

Parameter

Valid value is 'RIPE'. The default value is 'RIPE'.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get attribute value
my $value = $rir->name();
#Modify attribute value
$rir->name('RIPE');
```

email( )

Use this method to set or retrieve an email address for RIR.

Parameter

Email address in string format.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get attribute value
my $value = $rir->email();
#Modify attribute value
$rir->email('support@infoblox.com');
```

url( )

Use this method to set or retrieve the Web API URL for RIR.

Parameter

Web API URL in string format.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get attribute value
my $value = $rir->url();
#Modify attribute value
$rir->url('http://www.infoblox.com');
```

override_email( )

Use this method to set or retrieve the override_email flag of RIR.

Parameter

Specify "true" to override the predefined email address or "false" to use the predefined email address. The default value is "false".

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get attribute value
my $value = $rir->override_email();
#Modify attribute value
$rir->override_email('true');
```

override_url( )

Use this method to set or retrieve the override_url flag of RIR.

Parameter

Specify "true" to override the predefined Web API URL or "false" to use the predefined Web API URL. The default value is "false".

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get attribute value
my $value = $rir->override_url();
#Modify attribute value
$rir->override_url('true');
```

AUTHOR

Infoblox Inc. http://www.infoblox.com/

SEE ALSO

Infoblox::Session, Infoblox::Grid->enable_rir_swip()

COPYRIGHT

Copyright (c) 2017 Infoblox Inc.

Infoblox::Grid::RIR::Organization - Regional Internet Registry organization object.

- NAME
- DESCRIPTION
- CONSTRUCTOR
- SESSION METHODS
  - Infoblox::Session->add( )
  - Infoblox::Session->get( )
  - Infoblox::Session->modify( )
  - Infoblox::Session->remove( )
  - Infoblox::Session->search( )
- METHODS
  - id( )
  - rir( )
  - extattrs( )
  - extensible_attributes( )
  - name( )
  - maintainer( )
  - password( )
  - sender_email( )
- SAMPLE CODE
- AUTHOR
- SEE ALSO
- COPYRIGHT

NAME

Infoblox::Grid::RIR::Organization - Regional Internet Registry (RIR) organization object.

DESCRIPTION

Regional Internet Registry (RIR) organization object.

CONSTRUCTOR

```
my $rir_org = Infoblox::Grid::RIR::Organization->new(
   id                    => $string,                       # Required
   rir                   => $rir,                          # Required, an Infoblox::Grid::RIR object
   extattrs              => { $string => $extattr, ... },  # Required
   extensible_attributes => $extattrs,                     # Required
   name                  => $string,                       # Required
   maintainer            => $string,                       # Required
   password              => $string,                       # Required, write-only
   sender_email          => $string,                       # Required
);
```

You cannot set both extattrs and extensible_attributes attributes at the same time.

SESSION METHODS

This section describes all the methods in an Infoblox::Session module that you can apply to RIR organization object.

Support for RIR/SWIP must be enabled at the Grid level. See Infoblox::Grid->enable_rir_swip() method.

Infoblox::Session->add( )

Use this method to add an object to the Infoblox appliance. See Infoblox::Session->add() for parameters and return values.

Example

```
#Construct an object
my $rir_org = Infoblox::Grid::RIR::Organization->new(
   id                    => 'ORG_ID',
   name                  => 'RIR organization',
   maintainer            => 'support',
   password              => 'infoblox',
   sender_email          => 'support@infoblox.com',
   rir                   => $rir,
   extensible_attributes' => {'RIPE Registry Source' => 'RIPE',
```

```
                                    'RIPE Country' => 'United States (US)',
                                    'RIPE Admin Contact' => 'CA1-TEST',
                                    'RIPE Technical Contact' => 'CT1-TEST',
                                    'RIPE Email' => 'admin@infoblox.com'},
);


#Submit for addition
my $response = $session->add( $rir_org );
```

Infoblox::Session->get( )

Use this method to retrieve all the matching objects from the Infoblox appliance. See Infoblox::Session->get() for parameters and return values.

Key References

```
Apply the following attributes to get a specific RIR organization object:

 id           - Optional. The RIR Organization ID.
 name         - Optional. The RIR organization name.
 extattrs     - Optional. A hash reference containing extensible attributes.
```

Example

```
my @retrieved_objs = $session->get(
   object => 'Infoblox::Grid::RIR::Organization',
   name   => 'RIR organization',
);
```

Infoblox::Session->modify( )

Use this method to modify an object in the Infoblox appliance. See Infoblox::Session->modify() for parameters and return values.

Example

```
# Use this method to set password of a RIR organization object.
$rir_org->password('infoblox');
# Submit modification
my $response = $session->modify( $rir_org );
```

Infoblox::Session->remove( )

Use this method to remove an object from the Infoblox appliance. See Infoblox::Session->remove() for parameters and return values.

To remove a specific object, first use `get()` or `search()` to retrieve the specific object, and then submit this object for removal.

Example

```
#Get the objects with the same name
my @retrieved_objs = $session->get(
   object => 'Infoblox::Grid::RIR::Organization',
   name   => 'RIR organization',
);
#Find the desired object from the retrieved list.
my $desired_rir_org = $retrieved_objs[0];
# Submit for removal
my $response = $session->remove( $desired_rir_org );
```

Infoblox::Session->search( )

Use this method to search for RIR organization objects in the Infoblox appliance. See Infoblox::Session->search() for parameters and return values.

Key References

```
Apply the following attributes to search a specific RIR organization object:

 id           - Optional. The RIR Organization ID.
 name         - Optional. The RIR organization name.
 extattrs     - Optional. A hash reference containing extensible attributes.
```

Example

```
my @retrieved_objs = $session->search(
   object => 'Infoblox::Grid::RIR::Organization',
   name   => 'RIR organization',
);
```

METHODS

This section describes all the methods that you can use to configure and retrieve attribute values of the RIR organization object.

id( )

Use this method to set or retrieve RIR Organization ID.

Parameter

RIR Organization ID in string format.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get attribute value
my $value = $rir_org->id();
#Modify attribute value
$rir_org->id('ORG-MC1-RIPE');
```

rir( )

Use this method to set or retrieve RIR object which is associated with the organization.

Parameter

An Infoblox::Grid::RIR object.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get attribute value
my $value = $rir_org->rir();
#Modify attribute value
$rir_org->rir($rir);
```

extattrs( )

Use this method to set or retrieve the extensible attributes associated with a RIR organization object.

Parameter

Valid value is a hash reference containing the names of extensible attributes and their associated values (Infoblox::Grid::Extattr objects).

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get extattrs
my $ref_extattrs = $rir_org->extattrs();
#Modify extattrs
$rir_org->extattrs({ 'Site' => $extattr1, 'Administrator' => $extattr2 });
```

extensible_attributes( )

Use this method to set or retrieve the extensible attributes associated with a RIR organization object.

Parameter

For valid values for extensible attributes, see Infoblox::Grid::ExtensibleAttributeDef/Extensible Attribute Values and Infoblox::Grid->enable_rir_swip().

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get attribute value
my $value = $rir_org->extensible_attributes();
#Modify attribute value
$rir_org->extensible_attributes({'RIPE Registry Source' => 'TEST',
                                 'RIPE Country' => 'Canada (CA)',
                                 'RIPE Admin Contact' => 'CA2-TEST',
                                 'RIPE Technical Contact' => 'CT2-TEST',
                                 'RIPE Email' => 'support@infoblox.com'});
```

name( )

Use this method to set or retrieve RIR organization name.

Parameter

RIR organization name in string format.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get attribute value
my $value = $rir_org->name();
#Modify attribute value
$rir_org->name('New RIR organization');
```

maintainer( )

Use this method to set or retrieve the RIR organization maintainer.

Parameter

RIR organization maintainer in string format.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get attribute value
my $value = $rir_org->maintainer();
#Modify attribute value
$rir_org->maintainer('support');
```

password( )

Use this method to set the password for RIR organization's maintainer. This is a write-only attribute.

Parameter

The password for RIR organization's maintainer in string format.

Returns

The method returns true when the modification succeeds, and returns false when the operation fails.

Example

```
#Set attribute value
$rir_org->password('infoblox');
```

sender_email( )

Use this method to set or retrieve the sender email address for RIR organization.

Parameter

The sender email address for RIR organization in string format.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get attribute value
my $value = $rir_org->sender_email();
#Modify attribute value
$rir_org->sender_email('support@infoblox.com');
```

SAMPLE CODE

The following sample code demonstrates the different functions that can be applied to an object such as add, get, modify, search and remove. This sample also includes error handling for the operations.

#Preparation prior to getting and modifying RIR organization object

```
use strict;
use Infoblox;

#refers to Infoblox Appliance IP address
my $host_ip = "192.168.1.2";

#Create a session to the Infoblox appliance

my $session = Infoblox::Session->new(
    master   => $host_ip,
    username => "admin",
    password => "infoblox"
);
unless ($session) {
       die("Construct session failed: ",
               Infoblox::status_code() . ":" . Infoblox::status_detail());
}
print "Session created successfully\n";

my $grid = $self->{'session'}->get('object' => 'Infoblox::Grid');

unless ($grid) {
    die("Get Grid object failed: ",
           $session->status_code() . ":" . $session->status_detail());
}
print"Get Grid successful \n";

$grid->enable_rir_swip('true');

$session->modify($grid)
   or die("Modify Grid failed",
            $session->status_code() . ":" . $session->status_detail());
print "Grid modified successfully \n";
```

#Create RIR organization object

```
my $rir = $self->{'session'}->get('object' => 'Infoblox::Grid::RIR', 'name' => 'RIPE');

unless ($rir) {
    die("Get RIR object failed: ",
           $session->status_code() . ":" . $session->status_detail());
}
print"Get RIR successful \n";

my $rir_org = Infoblox::Grid::RIR::Organization->new(
       'id'            => 'RIR_ORG_ID',
       'name'          => 'RIR organization',
       'maintainer'    => 'support',
       'password'      => 'infoblox',
       'sender_email'  => 'support@infoblox.com',
```

```
        'rir'           => $rir,
        'extensible_attributes' => {'RIPE RIPE Registry Source' => 'RIPE',
                                    'RIPE Country' => 'United States (US)',
                                    'RIPE Admin Contact' => 'CA1-TEST',
                                    'RIPE Technical Contact' => 'CT1-TEST',
                                    'RIPE Email' => 'admin@infoblox.com'},
);

unless ($rir_org) {
    die("Construct RIR organization object failed: ",
            $session->status_code() . ":" . $session->status_detail());
}
print"RIR organization object created successfully\n";

my $response = $session->add($rir_org);

unless ($response) {
    die("Add RIR organization failed: ",
            $session->status_code() . ":" . $session->status_detail());
}
print"RIR organization added successfully \n";
```

#Search for RIR organization object

```
my @retrieved_objs = $session->search(
   object => 'Infoblox::Grid::RIR::Organization',
   id     => 'RIR_ORG_ID',
);

$rir_org = $retrieved_objs[0];

unless ($rir_org) {
        die("Search RIR organization object failed: ",
                $session->status_code() . ":" . $session->status_detail());
}
print "Search RIR organization object found at least 1 matching entry\n";
```

#Get and modify RIR organization object

```
@retrieved_objs = $session->get(
   object => 'Infoblox::Grid::RIR::Organization',
   id     => 'RIR_ORG_ID',
);

$rir_org = $retrieved_objs[0];

unless ($rir_org) {
    die("Get RIR organization object failed: ",
            $session->status_code() . ":" . $session->status_detail());
}
print"Get RIR organization successful \n";

$rir_org->maintainer("admin");

$session->modify($rir_org)
   or die("Modify RIR organization failed",
           $session->status_code() . ":" . $session->status_detail());
print "RIR organization modified successfully \n";
```

#Remove RIR organization object

```
@retrieved_objs = $session->get(
   object => 'Infoblox::Grid::RIR::Organization',
   id     => 'RIR_ORG_ID',
);

$rir_org = $retrieved_objs[0];

unless ($rir_org) {
    die("Get RIR organization object failed: ",
            $session->status_code() . ":" . $session->status_detail());
}
print"Get RIR organization successful \n";

$session->remove( $rir_org )
    or die("Remove RIR organization failed",
```

```
            $session->status_code() . ":" . $session->status_detail());
print"RIR organization removed successfully \n";

####PROGRAM ENDS####
```

AUTHOR

Infoblox Inc. http://www.infoblox.com/

SEE ALSO

Infoblox::Session, Infoblox::Grid::RIR, Infoblox::Grid::ExtensibleAttributeDef/Extensible Attribute Values, Infoblox::Grid->enable_rir_swip()

COPYRIGHT

Copyright (c) 2017 Infoblox Inc.

Infoblox::Grid::RootNameServer - Custom root name server object.

- NAME
- DESCRIPTION
- CONSTRUCTOR
- MODULE METHODS
  - Infoblox::Grid::DNS->rootNS( )
- METHODS
  - host_name( )
  - ipv4addr( )
  - ipv6addr( )
- SAMPLE CODE
- AUTHOR
- SEE ALSO
- COPYRIGHT

NAME

Infoblox::Grid::RootNameServer - Custom root name server object.

DESCRIPTION

The custom root name server provides host name and IP address information of internal root name server. When the Infoblox appliance receives a recursive query for DNS data that it does not have, it will use the internal root name server specified, instead of using the internet root name server.

CONSTRUCTOR

```
my $rootns = Infoblox::Grid::RootNameServer->new(
             host_name => $fqdn,      #Required
             ipv4addr  => $ipv4addr   #Required if $ipv6addr is not present
             ipv6addr  => $ipv6addr   #Required if $ipv4addr is not present
);
```

MODULE METHODS

The following functions are available to be applied to a custom root name server object.

Infoblox::Grid::DNS->rootNS( )

Use this function to specify custom root name servers at the grid level on Infoblox appliance. See Infoblox::Grid::DNS->rootNS() for parameters and return values.

Example

```
#Construct a custom root name server object
my $rootns1 = Infoblox::Grid::RootNameServer->new(
             host_name => "qe.infoblox.com",
             ipv4addr  => "1.1.1.1"
);

#Construct a custom root name server object
my $rootns2 = Infoblox::Grid::RootNameServer->new(
             host_name => "doc.infoblox.com",
             ipv6addr  => "2006::132"
);

# Configure custom root name server on the grid DNS object
my $response = $Grid_DNS->rootNS([$rootns1, $rootns2]);
```

METHODS

This section describes all the methods that can be used to configure and retrieve the attribute values of a custom root name server.

host_name( )

Use this method to set or retrieve the host name of the custom root name server.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

Specify the host name in FQDN (Fully Qualified Domain Name) format. The FQDN consists of the host name followed by the domain name (example: abc.com). A host name can have a maximum of 256 characters.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get host name
my $host_name = $rootns->host_name();
#Modify host name
$rootns->host_name("qa.infoblox.com");
```

ipv4addr( )

Use this method to set or retrieve the IPv4 address of the custom root name server. Required only when ipv6 is not specified.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

An IPv4 address is a 32-bit number in dotted decimal notation. It consists of four 8-bit groups of decimal digits separated by decimal points (example: 192. 168.1.2).

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get ipv4addr
my $ipv4addr = $rootns->ipv4addr();
#Modify ipv4addr
$rootns->ipv4addr("2.2.2.2");
```

ipv6addr( )

Use this method to set or retrieve the IPv6 address of the custom root name server. Required only when ipv4 is not specified.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

An IPv6 address is a 128-bit number in colon hexadecimal notation. It consists of eight groups of four hexadecimal digits separated by colons (example: 12ab:0000:0000:0123:4567:89ab:0000:cdef).

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
# Getting ipv6addr
my $ipv6addr = $rootns->ipv6addr();
# Modifying ipv6addr
$rootns->ipv6addr("12ab::345");
```

SAMPLE CODE

The following sample code demonstrates the different functions that can be applied to a custom root name server object such as add, modify, and remove. Also, this sample includes error handling for the operations.

#Preparation prior to a custom root name server object insertion

```
#PROGRAM STARTS: Include all the modules that will be used
use strict;
use Infoblox;
```

```
#Create a session to the Infoblox appliance
my $session = Infoblox::Session->new(
              master   => "192.168.1.2", #appliance host ip
              username => "admin",       #appliance user login
              password => "infoblox"     #appliance password
);
unless ($session) {
       die("Construct session failed: ",
            Infoblox::status_code() . ":" . Infoblox::status_detail());
}
print "Session created successfully\n";
```

#Create a custom root name server object

```
my $rootns = Infoblox::Grid::RootNameServer->new(
             host_name => "qe.infoblox.com",
             ipv4addr  => "1.1.1.1"
);
unless($rootns) {
       die("Construct custom root name server object failed: ",
            Infoblox::status_code() . ":" . Infoblox::status_detail());
}
print "Root name server object created successfully\n";
```

#Get the Grid DNS object and add custom root name server to it

```
#Get the Grid DNS object
my @retrieved_objs = $session->get(
    object => "Infoblox::Grid::DNS",
    grid   => "Infoblox"
);
my $object = $retrieved_objs[0];
unless ($object) {
       die("Get Grid DNS object failed: ",
            $session->status_code() . ":" . $session->status_detail());
}
print "Get Grid DNS object found at least 1 matching entry\n";

#Apply the changes to the Grid DNS object
$object->rootNS([$rootns]);

#Apply the changes
$session->modify($object)
    or die("Modify Grid DNS object failed: ",
            $session->status_code() . ":" . $session->status_detail());
print "Grid DNS object with custom root name server updated to Infoblox appliance successfully\n";
```

#Modify custom root name server

```
#Modify existing custom root name server object

#Modifying the host name of the custom root name server object.
$rootns->host_name("eng.infoblox.com");

#Modifying the IPV4 address of the custom root name server object.
$rootns->ipv4addr("3.3.3.3");

#Apply changes to the Grid DNS object.
$object->rootNS([$rootns]);

#Update Grid DNS object through the Infoblox session.
$session->modify($object)
            or die("modify Grid DNS object failed: ",
                $session->status_code() . ":" . $session->status_detail());
       print "Grid DNS object with modified custom root name server updated to Infoblox appliance
successfully\n";
```

#Remove custom root name server

```
#Apply changes to the Grid DNS object to use default internet root name server.
$object->rootNS(undef);

#Update Grid DNS object through the Infoblox session.
$session->modify($object)
            or die("Remove custom root name server from Grid DNS object failed: ",
                $session->status_code() . ":" . $session->status_detail());
       print "Removed custom root name server from Grid DNS object successfully\n";
```

```
####PROGRAM ENDS####
```

AUTHOR

Infoblox Inc. http://www.infoblox.com/

SEE ALSO

Infoblox::Session, Infoblox::Grid::DNS

COPYRIGHT

Copyright (c) 2017 Infoblox Inc.

Infoblox::Grid::RPZ::ThreatDetails - RPZ ThreatDetails object.

- NAME
- DESCRIPTION
- SESSION METHODS
- METHODS
  - active( )
  - danger_level( )
  - description( )
  - first_identified( )
  - known( )
  - last_seen( )
  - name( )
  - public_description( )
  - short_description( )
- AUTHOR
- SEE ALSO
- COPYRIGHT

NAME

Infoblox::Grid::RPZ::ThreatDetails - RPZ ThreatDetails object.

DESCRIPTION

Use this object to fetch ThreatStop details via ThreatStop REST API.

SESSION METHODS

The object does not support any session methods. It is retrieved as a response for Infoblox::Session->get_rpz_threat_details().

METHODS

This section describes all the methods that you can use to set or retrieve the attribute values of the object.

active( )

Use this method to retrieve the rule status whether it is active or not. This is a read-only attribute.

Parameter

None

Returns

The method returns attribute value.

Example

```
#Get active
my $active = $rpz_threat_details->active();
```

danger_level( )

Use this method to retrieve the value of a danger level. This is a read-only attribute.

Parameter

None

Returns

The method returns attribute value.

Example

```
#Get danger_level
my $danger_level = $rpz_threat_details->danger_level();
```

description( )

Use this method to retrieve the description. This is a read-only attribute.

Parameter

None

Returns

The method returns attribute value.

Example

```
#Get description
my $description = $rpz_threat_details->description();
```

first_identified( )

Use this method to retrieve the timestamp when the threat was first identified. This is a read-only attribute.

Parameter

None

Returns

The method returns attribute value.

Example

```
#Get first_identified
my $first_identified = $rpz_threat_details->first_identified();
```

known( )

Use this method to retrieve the rule status whether it is known or not. This is a read-only attribute.

Parameter

None

Returns

The method returns attribute value.

Example

```
#Get known
my $known = $rpz_threat_details->known();
```

last_seen( )

Use this method to retrieve the timestamp when the threat was last seen. This is a read-only attribute.

Parameter

None

Returns

The method returns attribute value.

Example

```
#Get last_seen
my $last_seen = $rpz_threat_details->last_seen();
```

name( )

Use this method to retrieve the threat name. This is a read-only attribute.

Parameter

None

Returns

The method returns attribute value.

Example

```
#Get name
my $name = $rpz_threat_details->name();
```

public_description( )

Use this method to retrieve the threat public description. This is a read-only attribute.

Parameter

None

Returns

The method returns attribute value.

Example

```
#Get public_description
my $public_description = $rpz_threat_details->public_description();
```

short_description( )

Use this method to retrieve the threat short description. This is a read-only attribute.

Parameter

None

Returns

The method returns attribute value.

Example

```
#Get short_description
my $short_description = $rpz_threat_details->short_description();
```

AUTHOR

Infoblox Inc. http://www.infoblox.com/

SEE ALSO

Infoblox::Session->get_rpz_threat_details()

COPYRIGHT

Copyright (c) 2017 Infoblox Inc.

Infoblox::Grid::ScheduledBackup - Scheduled Backup object.

- NAME
- DESCRIPTION
- CONSTRUCTOR
- MODULE METHODS
  - Infoblox::Grid->scheduled_backup( )
- METHODS
  - backup_type( )
  - keep_local_copy( )
  - backup_frequency( )
  - weekday( )
  - execute( )
  - hour_of_day( )
  - minute_of_hour( )
  - nios_data( )
  - discovery_data( )
  - splunk_app_data( )
  - user( )
  - password( )
  - backup_server( )
  - path( )
  - disabled( )
  - source( )
  - status( )
  - operation( )
  - restore_type( )
  - restore_host( )
  - restore_username( )
  - restore_password( )
  - restore_directory( )
- SAMPLE CODE
- AUTHOR
- SEE ALSO
- COPYRIGHT

NAME

Infoblox::Grid::ScheduledBackup - Scheduled Backup object.

DESCRIPTION

The Scheduled Backup object is used to schedule system backups.

CONSTRUCTOR

```
 #Constructor for a ScheduledBackup object
 my $backup = Infoblox::Grid::ScheduledBackup->new (
         backup_type => "LOCAL" | "TFTP" | "FTP" | "SCP",    #Optional / Default is "LOCAL"
         keep_local_copy  => "true" | "false",               #Optional / Default is "false"
         backup_frequency => "Weekly" | "Daily" | "Hourly", #Optional / Default is "Daily"
         weekday => "0" | "1" | "2" | "3" | "4" | "5" | "6" | "7" | "Sunday" | "Monday" | "Tuesday" |
"Wednesday" | "Thursday" | "Friday" | "Saturday"  #Optional / Default is "0"
         hour_of_day      => $num,                           #Optional / Default is 3
         minute_of_hour   => $num,                           #Optional / Default is 0
         user             => $string,                        #Optional / Default is undefined
         password         => $string,                        #Optional / Default is undefined
         backup_server    => $string,                        #Optional / Default is undefined
         path             => $string,                        #Optional / Default is undefined
         disabled         => "true" | "false",               #Optional / Default is "false"

         source             => "DB" | "REPORTING"            #Optional / Default is "DB"
         status             => "IDLE" | "TRIGGERED" | "IN_PROGRESS" | "FINISHED" | "FAILED", #Optional /
Default is "IDLE"
         operation_state    => 'TRIGGERED',                  #Optional / Default is undefined
         operation          => "NONE" | "BACKUP" | "RESTORE", #Optional / Default is "NONE"
         restore_type       => "FTP" | "SCP",                #Optional / Default is "FTP"
         restore_host       => $string,                      #Optional / Default is undefined
         restore_username   => $string,                      #Optional / Default is undefined
         restore_password   => $string,                      #Optional / Default is undefined
         restore_directory  => $string,                      #Optional / Default is undefined
         nios_data          => 'true' | 'false',             #Optional / If any of nios_data and
discovery_data is not
                                                             #defined, nios_data is 'true'.
```

```
        discovery_data    => 'true' | 'false',              #Optional / If any of nios_data and
discovery_data is not
                                                            #defined, discovery_data is 'false'.
        splunk_app_data   => 'true' | 'false',              #Optional / Default is 'false'
         );
```

MODULE METHODS

The following functions can be applied to a ScheduledBackup object.

Infoblox::Grid->scheduled_backup( )

Use this function to specify a ScheduledBackup object. See Infoblox::Grid->scheduled_backup() for parameters and return values.

Example

```
# Retrieve the list of ScheduledBackup objects from the Grid object.
my @retrieved_objs = $session->get(
    object => "Infoblox::Grid",
    name   => "Infoblox"
);

my $Grid_obj = $retrieved_objs[0];

# construct a ScheduledBackup object
my $backup = Infoblox::Grid::ScheduledBackup->new (
    backup_type => "LOCAL",
    backup_frequency => "Weekly",
    weekday => "0", # "0" is the same as "Sunday"
    hour_of_day => "4",
    minute_of_hour => "0",
    user  => "username",
    password => "user-password",
    backup_server  => "my-backup-server",
    path => "/tmp/infoblox/database.tar.gz",
    disabled => "false",
);

# Modify the scheduled backup list
$Grid_obj->scheduled_backup($backup);
```

METHODS

This section describes all the methods that you can use to configure and retrieve the attribute values of a ScheduledBackup object.

backup_type( )

Use this method to set or retrieve the destination of the backup files. You can back up system files to a local directory, a TFTP server, an FTP server, or an SSH server that supports SCP.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

Valid values are "LOCAL", "TFTP", "FTP" or "SCP". The default value is "LOCAL".

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get backup_type
my $backup_type = $backup->backup_type();
#Modify backup_type
$backup->backup_type("FTP");
```

keep_local_copy( )

Use this method to control whether a local backup is created in addition to a remote one. This option works only if you also specify a remote backup type (TFTP, FTP, or SCP). You cannot enable this option using the "LOCAL" backup type.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

Valid values are "true" and "false". The default value is "false".

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get keep_local_copy
my $keep_local_copy = $backup->keep_local_copy();
#Modify keep_local_copy
$backup->backup_type("true");
```

backup_frequency( )

Use this method to set or retrieve the frequency of backups.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

Valid values are "Weekly", "Daily" or "Hourly". The default value is "Daily".

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

Example

```
#Get backup frequency
my $backup_backup_frequency = $backup->backup_frequency();
#Modify backup frequency
$backup->backup_frequency("Weekly");
```

weekday( )

Use this method to set or retrieve the day of the week when the backup is performed. This is required only when backup_frequency is set to "Weekly".

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

Valid values are "0", "1", "2", "3", "4", "5", "6", "7", "Sunday", "Monday", "Tuesday", "Wednesday", "Thursday", "Friday" or "Saturday". "0" means "Sunday", "1" - "Monday". etc. The default value is "6" (Saturday).

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get weekday
my $weekday = $backup->weekday();
#Modify weekday
$backup->weekday("0");
```

execute( )

Use this method to set the state for Reporting scheduled backup / restore operation. This is a write-only attribute.

Parameter

The valid value is 'TRIGGER'. The default is undefined.

Returns

None

Example

```
#Modify operation_state value
$object->execute('TRIGGER');
```

hour_of_day( )

Use this method to set or retrieve the hour when the backup is performed.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

A number from 0 through 23. The default value is 3 (3:00 A.M.).

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get hour_of_day
my $hour_of_day = $backup->hour_of_day();
#Modify hour_of_day
$backup->hour_of_day("4");
```

minute_of_hour( )

Use this method to set or retrieve the minute of the hour when the backup is performed.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

A number from 0 through 59. The default value is 0.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get minute_of_hour
my $minute_of_hour = $backup->minute_of_hour();
#Modify minute_of_hour
$backup->minute_of_hour("5");
```

nios_data( )

Use this method to enable or disable the backup and restore of NIOS data.

If any of nios_data and discovery_data is not defined, nios_data is 'true'.

Parameter

Set the parameter to "true" to enable backup/restore of NIOS data. Set the parameter to "false" to disable the object. The default value of this parameter is "false".

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get nios_data
my $nios_data = $backup->nios_data();
#Modify nios_data
$backup->nios_data("true");
```

discovery_data( )

Use this method to enable or disable the backup and restore of Network Automation data.

If any of nios_data and discovery_data is not defined, discovery_data is 'false'.

Parameter

Set the parameter to "true" to enable backup/restore of Network Automation data. Set the parameter to "false" to disable the object. The default value of this parameter is "false".

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get discovery_data
my $nios_data = $backup->discovery_data();
#Modify discovery_data
$backup->discovery_data("true");
```

splunk_app_data( )

Use this method to enable or disable the backup and restore of Splunk application data.

Parameter

Set the parameter to "true" to enable backup/restore of Splunk application data. Set the parameter to "false" to disable the object. The default value of this parameter is "false".

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get splunk_app_data
my $nios_data = $backup->splunk_app_data();
#Modify splunk_app_data
$backup->splunk_app_data("true");
```

user( )

Use this method to set or retrieve the user name on the backup server. This is required only when backup_type is set to "FTP" or "SCP".

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

The user name in string format with a maximum of 1024 bytes.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get user
my $user = $backup->user();
#Modify user
$backup->user("admin1");
```

password( )

Use this method to set the user password on the backup server. This is required only when backup_type is set to "FTP" or "SCP". This is a write-only attribute.

Parameter

The password in string format with a maximum of 64 bytes.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

Example

```
#Modify password
$backup->password("userpassword");
```

backup_server( )

Use this method to set or retrieve the IP address of the backup server. This is required only when backup_type is set to "FTP", "TFTP" or "SCP".

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

The IP address of the backup server.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get backup_server
my $backup_server = $backup->backup_server();
#Modify backup_server
$backup->backup_server("my-backup-server");
```

path( )

Use this method to set or retrieve the directory path to the backup file stored on the server. This is required only when backup_type is set to "FTP" or "SCP".

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

Desired path in string format with a maximum of 1024 bytes.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get path
my $path = $backup->path();
#Modify path
$backup->path("/tmp/infoblox/database.tar.gz");
```

disabled( )

Use this method to disable the ScheduledBackup object.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

Set the parameter to "true" to disable the ScheduledBackup object. Set the parameter to "false" to enable the object. The default value of this parameter is "false".

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get disabled
my $disabled = $backup->disabled();
#Modify disabled
$backup->disabled("true");
```

source( )

Use thi method to set or retrieve the source of ScheduledBackup object.

Parameter

Can be set to "DB" to operate with Grid ScheduledBackup object or to "REPORTING" to operate with Reporting ScheduledBackup object.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get source
my $source = $backup->source();
#Modify source
$backup->source("REPORTING");
```

status( )

Use this method to set or retrieve the status of the Reporting ScheduledBackup object.

Parameter

Either "TRIGGERED" or "IN_PROGRESS" to identify that Reporting backup/restore is running. Status "FINISHED" means Reporting backup/restore succeeded. Status "FAILED" means Reporting backup/restore failed. The default value of this parameter is "IDLE".

Set status to "TRIGGERED" to run a Reporting manual backup. Use this for a Reporting manual backup only.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get status
my $status = $backup->status();
#Modify status
$backup->status("TRIGGERED");
```

operation( )

Use this method to set or retrieve the operation of the Reporting ScheduledBackup object.

Parameter

Specify "BACKUP" to indicate that a Reporting backup must be performed, or specify "RESTORE" to indicate that a Reporting restore must be performed. The default value is "NONE".

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get operation
my $operation = $backup->operation();
#Modify operation
$backup->operation("RESTORE");
```

restore_type( )

Use this method to set or retrieve the destination of the restore files when the operation is set to "RESTORE". You can back up system files to an FTP server, or an SSH server that supports SCP.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

Valid values are "FTP" or "SCP". The default value is "FTP".

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get restore_type
my $restore_type = $backup->restore_type();
#Modify restore_type
$backup->restore_type("FTP");
```

restore_host( )

Use this method to set or retrieve the IP address of the restore server when the operation is set to "RESTORE".

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

The IP address of the restore server.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get backup_server
my $restore_host = $backup->restore_host();
#Modify restore_host
$backup->restore_host("my-restore-server");
```

restore_username( )

Use this method to set or retrieve the user name that is used to log in to the restore server. This is required only when "RESTORE" operation is performed.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

The user name in string format with a maximum of 1024 bytes.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get user
my $restore_username = $backup->restore_username();
#Modify user
$backup->restore_username("admin1");
```

restore_password( )

Use this method to set the user password that is used to log in to the restore server. This is required only when "RESTORE" operation is performed. This is a write-only attribute.

Parameter

The restore_password in string format with a maximum of 64 bytes.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

Example

```
#Modify restore_password
$backup->restore_password("userpassword");
```

restore_directory( )

Use this method to set or retrieve the directory path to the restored file on the server. This is required only when "RESTORE" operation is performed.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

Desired path in string format with a maximum of 1024 bytes.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get restore_directory
my $restore_directory = $backup->restore_directory();
#Modify restore_directory
$backup->restore_directory("/tmp");
```

SAMPLE CODE

The following sample code demonstrates different operations that can be applied to an object such as create, modify, and remove an object. This sample code also includes error handling for the operations.

```
#PROGRAM STARTS: Include all the modules that will be used
use strict;
use Infoblox;

#Create a session to the Infoblox appliance
my $session = Infoblox::Session->new(
    master   => "192.168.1.2",
    username => "admin",
    password => "infoblox"
);
unless ($session) {
   die("Construct session failed: ",
       Infoblox::status_code() . ":" . Infoblox::status_detail());
}
print "Session created successfully\n";
```

#Add a ScheduledBackup object

```
my @result_array = $session->get( "object" => "Infoblox::Grid", "name" => "Infoblox");
if( scalar( @result_array ) > 0 )
 {
        my $Grid_obj = $result_array[0];
        unless ($Grid_obj) {
                die("Get Grid object failed: ",
        $session->status_code() . ":" . $session->status_detail());
        }
        # construct a ScheduledBackup object
        my $backup = Infoblox::Grid::ScheduledBackup->new (
            backup_type => "LOCAL",
            backup_frequency => "Weekly",
            weekday => "0", # "0" is the same as "Sunday"
            hour_of_day => "4",
            minute_of_hour => "0",
            user  => "username",
            password => "user-password",
            backup_server  => "10.0.3.4",
            path => "/tmp/infoblox/database.tar.gz",
            disabled => "false",
        );

        # Add ScheduledBackup to Grid object.
        $Grid_obj->scheduled_backup($backup);

        $session->modify($Grid_obj)
            or die("Modify Grid failed: ",$session->status_code(). ":" .$session->status_detail());
        print "ScheduledBackup added to Grid object sucessfully\n";
         } else {
             print "No Grid object found.";
         }
```

#Modify an existing ScheduledBackup object

```
@result_array = $session->get( "object" => "Infoblox::Grid", "name" => "Infoblox");
if( scalar( @result_array ) > 0 )
```

```
{
        my $Grid_obj = $result_array[0];
        unless ($Grid_obj) {
                die("Get Grid object failed: ",
        $session->status_code() . ":" . $session->status_detail());
        }

         # get ScheduledBackup from Grid object.
         my $backup= $Grid_obj->scheduled_backup();

         #modify the backup frequency
         $backup->backup_frequency("Daily");

         # Add ScheduledBackup to Grid object.
         $Grid_obj->scheduled_backup($backup);

         $session->modify($Grid_obj)
            or die("Modify Grid failed: ",$session->status_code(). ":" .$session->status_detail());
         print "ScheduledBackup modified on Grid object sucessfully\n";
          } else {
              print "No Grid object found.";
          }
```

#Remove an existing ScheduledBackup object

```
@result_array = $session->get( "object" => "Infoblox::Grid", "name" => "Infoblox");
if( scalar( @result_array ) > 0 )
 {
        my $Grid_obj = $result_array[0];
        unless ($Grid_obj) {
                die("Get Grid object failed: ",
        $session->status_code() . ":" . $session->status_detail());
        }

         # empty ScheduledBackup list on Grid object.
         $Grid_obj->scheduled_backup([]);

         $session->modify($Grid_obj)
            or die("Modify Grid failed: ",$session->status_code(). ":" .$session->status_detail());
         print "ScheduledBackup removed on Grid object sucessfully\n";
          } else {
              print "No Grid object found.";
          }

####PROGRAM ENDS####
```

AUTHOR

Infoblox Inc. http://www.infoblox.com/

SEE ALSO

Infoblox::Grid, Infoblox::Grid->scheduled_backup()

COPYRIGHT

Copyright (c) 2017 Infoblox Inc.

Infoblox::Grid::ScheduledTask - scheduled task object.

- NAME
- DESCRIPTION
- CONSTRUCTOR
- SESSION METHODS
  - Infoblox::Session->get( )
  - Infoblox::Session->modify( )
  - Infoblox::Session->remove( )
  - Infoblox::Session->search( )
- METHODS
  - approval_status( )
  - approver( )
  - approver_comment( )
  - changed_objects( )
  - dependent_tasks( )
  - execution_details( )
  - execution_details_type( )
  - execution_status( )
  - execution_time( )
  - member( )
  - is_network_insight_task( )
  - re_execute_task( )
  - scheduled_time( )
  - submit_time( )
  - submitter_comment( )
  - submitter( )
  - task_id( )
  - ticket_number( )
  - task_type( )
  - time_zone( )
  - predecessor_task( )
- SAMPLE CODE
- SCHEDULED DATE AND TIME FORMAT
- AUTHOR
- SEE ALSO
- COPYRIGHT

NAME

Infoblox::Grid::ScheduledTask - scheduled task object.

DESCRIPTION

A scheduled task object is used to retrieve information about scheduled tasks. Scheduled tasks are commands that are scheduled by the Infoblox::Session methods, Infoblox::Session->add(), Infoblox::Session->modify() and Infoblox::Session->remove(), and are called with the scheduled_at parameter.

CONSTRUCTOR

The Infoblox::Grid::ScheduledTask object is automatically generated after the Infoblox::Session methods, Infoblox::Session->add(), Infoblox::Session->modify() and Infoblox::Session->remove(), are called with the scheduled_at parameter. It does not require manual construction.

SESSION METHODS

This section describes all the methods in the Infoblox::Session module that you can apply to a scheduled task object.

Infoblox::Session->get( )

Use this method to retrieve all the matching objects from the Infoblox appliance. See Infoblox::Session->get() for parameters and return values.

Key References

Apply the following attributes to get a specific scheduled task object:

member - The Grid member name.

scheduled_time - The time when the task is scheduled to occur.

submit_time - The time when the scheduled task was submitted.

submitter - The name of the user who submitted the task (regular expression).

task_id - The identifier of a scheduled task.

action - The type of action performed. When used, the search returns ScheduledTask objects that have ScheduledTask::ChangedObject objects in the changed_objects list with matching 'action' attributes. Possible values are:

```
Add
Modify
Delete
Network Discovery
Restart Services
```

object_name - The name of the changed object (regular expression). When used, the search returns ScheduledTask objects that have ScheduledTask::ChangedObject objects in the changed_objects list with matching 'object_name' attirbutes.

object_type - The type of the changed object (regular expression). When used, the search returns ScheduledTask objects that have ScheduledTask::ChangedObject objects in the changed_objects list with matching 'object_type' attributes. Possible values are:

```
A Record
AAAA Record
Authoritative Zone
Bulk Host
CNAME Record
Delegated Zone
DHCP Range
DNAME Record
DNS View
Fixed Address
Forward Zone
Host Record
IPv4 Network
IPv4 Network Container
IPv6 Network
IPv6 Network Container
MX Record
NS Record
PTR Record
Reservation
Roaming Host
Shared A Record
Shared AAAA Record
Shared MX Record
Shared Network
Shared Record Group
Shared SRV Record
Shared TXT Record
SRV Record
Stub Zone
TXT Record
```

Example

```
my @retrieved_objs = $session->get(
    object     => "Infoblox::Grid::ScheduledTask",
    task_id    => "34" );
```

Infoblox::Session->modify( )

Use this method to modify an object in the Infoblox appliance. See Infoblox::Session->modify() for parameters and return values.

Example

```
# Use this method to modify the object.
$task->scheduled_time( "2008-10-21T10:56:33Z" );
# Submit modification
my $response = $session->modify( $task );
```

Infoblox::Session->remove( )

Use this method to remove an object from the Infoblox appliance. See Infoblox::Session->remove() for parameters and return values.

To remove a specific object, use `get()` or `search()` to retrieve the specific object first, and then submit this object for removal.

Example

```
# Get the ScheduledTask objects
my @retrieved_objs = $session->get(
    object     => "Infoblox::Grid::ScheduledTask",
    submitter  => "admin" );
# find the desired object from the retrieved list.
```

```
my $task = $retrieved_objs[0];
# Submit for removal
my $response = $session->remove( $task );
```

Infoblox::Session->search( )

Use this method to search for scheduled task objects in the Infoblox appliance. See Infoblox::Session->search() for parameters and return values.

Key References

The search can be performed on any combination of the following attributes:

member - The Grid member name.

scheduled_time - The time when the task is scheduled to occur. This method supports searching with limit parameters (see below).

submit_time - The time when the scheduled task was submitted. This method supports searching with limit parameters (see below).

submitter - The name of the user who submitted the task (regular expression).

task_id - The identifier of a scheduled task.

action - The type of action performed. When used, the search returns ScheduledTask objects that have ScheduledTask::ChangedObject objects in the changed_objects list with matching 'action' attributes. Possible values are:

```
Add
Modify
Delete
Network Discovery
Restart Services
```

object_name - The name of the changed object (regular expression). When used, the search returns ScheduledTask objects that have ScheduledTask::ChangedObject objects in the changed_objects list with matching 'object_name' attirbutes.

object_type - The type of the changed object (regular expression). When used, the search returns ScheduledTask objects that have ScheduledTask::ChangedObject objects in the changed_objects list with matching 'object_type' attributes. Possible values are:

```
A Record
AAAA Record
Authoritative Zone
Bulk Host
CNAME Record
Delegated Zone
DHCP Range
DNAME Record
DNS View
Fixed Address
Forward Zone
Host Record
IPv4 Network
IPv4 Network Container
IPv6 Network
IPv6 Network Container
MX Record
NS Record
PTR Record
Reservation
Roaming Host
Shared A Record
Shared AAAA Record
Shared MX Record
Shared Network
Shared Record Group
Shared SRV Record
Shared TXT Record
SRV Record
Stub Zone
TXT Record
```

See Infoblox::Session/search with limit parameters for more information on this functionality.

Example

```
# search for all ScheduledTask objects
my @retrieved_objs = $session->search(
    object  => "Infoblox::Grid::ScheduledTask",
    scheduled_time => ">=< 2008-10-21T10:56:33Z,2008-11-21T10:56:33Z");
```

METHODS

This section describes all the methods that you can use to set and retrieve the attribute values of a ScheduledTask object.

approval_status( )

Use this method to set or retrieve the approval status if approvals are required for this task.

Parameter

Valid values are 'NONE', 'APPROVED', 'PENDING' and 'REJECTED'.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get approval_status value
my $approval_status = $task->approval_status();

#Modify approval_status value
$task->approval_status('APPROVED');
```

approver( )

Use this method to retrieve the username of the approver. This is a read-only attribute.

Parameter

none

Returns

The method returns the attribute value.

Example

```
#Get approver value
my $approver = $task->approver();
```

approver_comment( )

Use this method to set or retrieve the approver comment.

Parameter

The approver comment in string format.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get approver_comment value
my $approver_comment = $task->approver_comment();

#Modify approver_comment value
$task->approver_comment('ok');
```

changed_objects( )

Use this method to retrieve information about the objects that are scheduled to be changed.

Parameter

none

Returns

The method returns a reference to an array of Infoblox::Grid::ScheduledTask::ChangedObject objects.

Example

```
#Get changed_objects
my $changed_objects = $task->changed_objects();
my $obj = $changed_objects->[0];
```

dependent_tasks( )

Use this method to retrieve an array of of tasks dependent on this one. This is a read-only attribute.

Parameter

None

Returns

The method returns a reference to an array of Infoblox::Grid::ScheduledTask objects.

Example

```
#Get array of ScheduledTask objects
my @dependent_tasks = $task->dependent_tasks();
```

execution_details( )

Use this method to retrieve the list of details generated by the execution. This is a read-only field.

Parameters

None

Returns

The method returns the array of strings that contains details generated by the execution.

Example

```
#Get execution_details value
my $details = $task->execution_details();
```

execution_details_type( )

Use this method to retrieve the type of details generated by the execution after the task has been completed. This is a read-only field.

Parameters

None

Returns

The method returns the attribute value that can be 'NONE' or 'WARNING'.

Example

```
#Get execution_details_type value
my $details = $task->execution_details_type();
```

execution_status( )

Use this method to retrieve the status of the task. This is a read-only attribute.

Parameter

none

Returns

The method returns the attribute value that can be 'COMPLETED', 'FAILED' or 'PENDING'.

Example

```
#Get execution_status
my $execution_status = $task->execution_status();
```

execution_time( )

Use this method to retrieve the execution time. This is a read-only attribute.

Parameter

none

Returns

The method returns the attribute value.

Example

```
#Get execution_time
my $execution_time = $task->execution_time();
```

member( )

Use this method to retrieve the Grid member that has created this task. This is a read-only attribute.

Parameter

None

Returns

The method returns the attribute value.

Example

```
#Get member
my $member = $task->member();
```

is_network_insight_task( )

Use this method to retrieve the flag that indicates if this task is related to network insight. This is a read-only attribute.

Parameter

none

Returns

This method returns the attribute value.

Example

```
#Get value
$value = $task->is_network_insight_task();
```

re_execute_task( )

Use this method to set failed discovery task to be re-executed.

If this flag is set to 'true' a failed Network Insight discovery task will be re-executed, a new task will be created cloned from this one and restarted.

Parameter

Boolean to set failed netmri task to be re-executed. Only "true" value makes sense.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
# Get value
$value = $task->re_execute_task();

# Set value
$task->re_execute_task('true');
```

scheduled_time( )

Use this method to set or retrieve the date and time the task is scheduled to occur.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

Scheduled date and time in ISO 8601 extended format (e.g., 2008-10-21T10:56:33Z or 2008-10-21T10:56:33.001Z).

Use case insensitive value "now" to execute a scheduled task immediately.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get scheduled_time value
my $scheduled_time = $task->scheduled_time();

#Modify scheduled_time value
$task->scheduled_time("2008-10-21T10:56:33Z");
# Submit modification
$session->modify($task);

# Do it now
$task->scheduled_time("Now");
# Submit modification
$session->modify($task);
```

submit_time( )

Use this method to retrieve the time the scheduled task was submitted (when appropriate Session method was called with parameter scheduled_time).

Parameter

none

Returns

The method returns the submitted date and time in ISO 8601 extended format (e.g., 2008-10-21T10:56:00Z or 2008-10-21T10:56:00.001Z).

Example

```
#Get submit_time
my $submit_time = $task->submit_time();
```

submitter_comment( )

Use this method to set or retrieve the submitter comment.

Parameter

The submitter comment in string format.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get submitter_comment value
my $submitter_comment = $task->submitter_comment();

#Modify submitter_comment value
$task->submitter_comment('ok');
```

submitter( )

Use this method to retrieve the name of the user who submitted the task.

Parameter

none

Returns

The method returns the user name in string format.

Example

```
#Get submitter
my $submitter = $task->submitter();
```

task_id( )

Use this method to retrieve the identifier of a scheduled task.

Parameter

none

Returns

The method returns the identifier in numeric format.

Example

```
#Get task_id
my $task_id = $task->task_id();
```

ticket_number( )

Use this method to set or retrieve the ticket number.

Parameter

The ticket number in string format.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get ticket_number value
my $ticket_number = $task->ticket_number();

#Modify ticket_number value
$task->ticket_number('ok');
```

task_type( )

Use this method to retrieve the scheduled task type. This is a read-only attribute.

Parameter

none

Returns

The method returns the attribute value. Valid return value is "OBJECT_CHANGE" and "PORT_CONTROL".

Example

```
#Get task_type value
my $task_type = $task->task_type();
```

time_zone( )

Use this method to set or retrieve the time zone.

Parameter

The UTC string that represents the time zone. For example "(UTC - 6:00) Central Time (US and Canada)". The default value is "(UTC) Coordinated Universal Time".

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get attribute value
my $value = $schedule_setting->time_zone();

#Modify attribute value
$schedule_setting->time_zone("(UTC - 6:00) Central Time (US and Canada)");
```

predecessor_task( )

Use this method to retrieve the predecessor task. This is a read-only attribute.

Parameter

none

Returns

The method returns an Infoblox::Grid::ScheduledTask object.

Example

```
#Get predecessor_task value
my $predecessor_task = $task->predecessor_task();
```

SAMPLE CODE

The following sample code demonstrates the different functions that can be applied to an object, such as add, search, modify, and remove. This sample also includes error handling for the operations.

#Preparation prior to a ScheduledTask object insertion

```
#PROGRAM STARTS: Include all the modules that will be used
use strict;
use Infoblox;

#Create a session to the Infoblox appliance

my $session = Infoblox::Session->new(
               master   => "192.168.1.2", #appliance host ip
               username => "admin",       #appliance user login
               password => "infoblox"     #appliance password
               );

unless ($session) {
       die("Construct session failed: ",
               Infoblox::status_code() . ":" . Infoblox::status_detail());
}
print "Session created successfully\n";

# Add a network
my $network = Infoblox::DHCP::Network->new(
    network => "10.0.0.0/24",
);
unless ($network) {
    die("Construct Network failed: ",
        Infoblox::status_code() . ":" . Infoblox::status_detail());
}
print "Network created successfully\n";

$session->add($network)
    or die("Add Network object failed: ",
    $session->status_code() . ":" . $session->status_detail());
print "Network object added to server successfully\n";
```

#Create a ScheduledTask object

```
# Create a fixed address
my $fixed_addr = Infoblox::DHCP::FixedAddr->new(
    ipv4addr    => '10.0.0.3',
    mac         => '11:22:33:44:55:66'
);
unless ($fixed_addr) {
    die("Construct FixedAddr failed: ",
        Infoblox::status_code() . ":" . Infoblox::status_detail());
}
print "FixedAddr created successfully\n";

# Schedule the addition of a fixed address
$session->add($fixed_addr, scheduled_at => "2020-01-01T14:52:00Z")
    or die("Schedule fixed address add failed: ",
    $session->status_code() . ":" . $session->status_detail());
print "Fixed address add scheduled successfully\n";
```

#Search for a ScheduledTask object

```
my @retrieved_objs = $session->search(
    object    => "Infoblox::Grid::ScheduledTask",
    submitter => "admin"
);
my $object = $retrieved_objs[0];

unless ($object) {
    die("Search for a ScheduledTask object failed: ",
        $session->status_code() . ":" . $session->status_detail());
}
print "Search for a ScheduledTask object found at least 1 matching entry\n";
```

#Get and modify a ScheduledTask object

```
#Get the ScheduledTask object from Infoblox appliance through a session
my @retrieved_objs = $session->get(
    object         => "Infoblox::Grid::ScheduledTask",
    scheduled_time => "2020-01-01T14:52:33Z"
);
my $object = $retrieved_objs[0];

unless ($object) {
    die("Get ScheduledTask object failed: ",
        $session->status_code() . ":" . $session->status_detail());
}
print "Get ScheduledTask object found at least 1 matching entry\n";

#Modify the ScheduledTask object
$object->scheduled_time("2020-02-01T14:52:33Z");

#Apply the changes.
$session->modify($object)
    or die("Modify ScheduledTask object failed: ",
    $session->status_code() . ":" . $session->status_detail());
print "ScheduledTask object modified successfully \n";
```

#Remove a ScheduledTask object

```
#Get the ScheduledTask object through the session
my @retrieved_objs = $session->get(
    object => "Infoblox::Grid::ScheduledTask",
    action => "Add"
);
my $object = $retrieved_objs[0];
unless ($object) {
    die("Get ScheduledTask object failed: ",
        $session->status_code() . ":" . $session->status_detail());
}
print "Get ScheduledTask object found at least 1 matching entry\n";

#Submit the object for removal
$session->remove($object)
    or die("Remove ScheduledTask object failed: ",
    $session->status_code() . ":" . $session->status_detail());
print "ScheduledTask object removed successfully \n";
```

#Cleanup

```
#Get the Network object through the session
my @retrieved_objs = $session->get(
    object  => "Infoblox::DHCP::Network",
    network => "10.0.0.0/24"
);
my $object = $retrieved_objs[0];
unless ($object) {
    die("Get Network object failed: ",
        $session->status_code() . ":" . $session->status_detail());
}
print "Get Network object found at least 1 matching entry\n";

#Submit the object for removal
$session->remove($object)
    or die("Remove Network object failed: ",
    $session->status_code() . ":" . $session->status_detail());
print "Network object removed successfully \n";
```

```
####PROGRAM ENDS####
```

SCHEDULED DATE AND TIME FORMAT

Scheduled date and time are represented in ISO 8601 extended format (e.g., 2008-10-21T10:56:00Z or 2008-10-21T10:56:00.001Z) with some restrictions:

- Maximal value for year field is 2037.

When the scheduling parameter is used, changes to objects do not occur immediately. Instead, the Infoblox::Grid::ScheduledTask object is created, and then executed at the time specified in this parameter.

AUTHOR

Infoblox Inc. http://www.infoblox.com/

SEE ALSO

Infoblox::Grid::ScheduledTask, Infoblox::Grid::ScheduledTask::ChangedObject, Infoblox::Session->add(), Infoblox::Session->get(), Infoblox::Session->modify(), Infoblox::Session->remove(), Infoblox::Session->search(),Infoblox::Session

COPYRIGHT

Copyright (c) 2017 Infoblox Inc.

Infoblox::Grid::ScheduledTask::ChangedObject - Scheduled Task Changed Object.

- NAME
- DESCRIPTION
- CONSTRUCTOR
- MODULE METHODS
  - Infoblox::Grid::ScheduledTask->changed_objects( )
- METHODS
  - action( )
  - changed_properties( )
  - object_name( )
  - object_type( )
- SAMPLE CODE
- AUTHOR
- SEE ALSO
- COPYRIGHT

NAME

Infoblox::Grid::ScheduledTask::ChangedObject - Scheduled Task Changed Object.

DESCRIPTION

A Scheduled Task Changed Object is used to retrieve information about objects with changes that were scheduled in Infoblox::Grid::ScheduledTask.

CONSTRUCTOR

The Infoblox::Grid::ScheduledTask::ChangedObject is a read-only object and does not require manual construction.

MODULE METHODS

The following functions can be applied to a ChangedObject object.

Infoblox::Grid::ScheduledTask->changed_objects( )

Use this function to retrieve the changed objects list. See Infoblox::Grid::ScheduledTask->changed_objects() for parameters and return values.

Example

```
# Retrieve the list of ScheduledTask objects
my @retrieved_objs = $session->get(
    object => "Infoblox::Grid::ScheduledTask",
    submitter => "admin" );

my $task = $retrieved_objs[0];

# get ChangedObject
my $changed_objects = $task->changed_objects();
my $obj = $changed_objects->[0];
```

METHODS

This section describes all the methods that you can use to retrieve the attribute values of a ChangedObject object.

action( )

Use this method to retrieve the operation performed on the objects.

Parameter

none

Returns

The method returns the operation in string format. Possible values are:

"Network Discovery" for network_discovery_control( ) service,

"Restart Services" for restart( ) services,

"Add", "Modify", "Delete" for all other objects.

Example

```
#Get action
my $action = $changed_object->action();
```

changed_properties( )

Use this method to retrieve the list of changed properties in the object.

Parameter

none

Returns

The method returns a reference to an array of strings. Each string in the array describes the changes. The descriptions are similiar to the entries in the Change Audit Log column in the GUI.

Example

```
#Get changed_properties value
my $changed_properties = $changed_object->changed_properties();
for (@$changed_properties) {
    print $_,"\n";
}
```

object_name( )

Use this method to retrieve the name of the changed object.

Parameter

none

Returns

The method returns the object name in string format.

Example

```
#Get object_name
my $object_name = $changed_object->object_name();
```

object_type( )

Use this method to retrieve the type of the changed object.

Parameter

none

Returns

The method returns an object type in string format, such as "A Record" or "Shared TXT Record".

Example

```
#Get object_type
my $object_type = $changed_object->object_type();
```

SAMPLE CODE

The following sample code demonstrates the different functions that can be applied to an object, such as add, search, modify, and remove. This sample also includes error handling for the operations.

#Preparation prior to a ScheduledTask object insertion

```
#PROGRAM STARTS: Include all the modules that will be used
use strict;
use Infoblox;

#Create a session to the Infoblox appliance
```

```
my $session = Infoblox::Session->new(
               master   => "192.168.1.2", #appliance host ip
               username => "admin",       #appliance user login
               password => "infoblox"     #appliance password
               );

unless ($session) {
       die("Construct session failed: ",
               Infoblox::status_code() . ":" . Infoblox::status_detail());
}
print "Session created successfully\n";

my $network = Infoblox::DHCP::Network->new(
    network => "10.0.0.0/24",
);
unless ($network) {
    die("Construct Network failed: ",
        Infoblox::status_code() . ":" . Infoblox::status_detail());
}
print "Network created successfully\n";

$session->add($network)
    or die("Add Network object failed: ",
    $session->status_code() . ":" . $session->status_detail());
print "Network object added to server successfully\n";
```

#Create a ScheduledTask object

```
# Create a fixed address
my $fixed_addr = Infoblox::DHCP::FixedAddr->new(
    ipv4addr    => '10.0.0.3',
    mac         => '11:22:33:44:55:66'
);
unless ($fixed_addr) {
    die("Construct FixedAddr failed: ",
        Infoblox::status_code() . ":" . Infoblox::status_detail());
}
print "FixedAddr created successfully\n";

# Schedule the addition of a fixed address
$session->add($fixed_addr, scheduled_at => "2020-01-01T14:52:00Z")
    or die("Schedule fixed address add failed: ",
    $session->status_code() . ":" . $session->status_detail());
print "Fixed address add scheduled successfully\n";
```

#Search for a ScheduledTask

```
my @retrieved_objs = $session->search(
    object    => "Infoblox::Grid::ScheduledTask",
    submitter => "admin"
);
my $object = $retrieved_objs[0];

unless ($object) {
    die("Search for a ScheduledTask object failed: ",
        $session->status_code() . ":" . $session->status_detail());
}
print "Search for a ScheduledTask object found at least 1 matching entry\n";
my $objs = $object->changed_objects;
my $changed_obj = $objs->[0];
my $action = $changed_obj->action();
my $object_name= $changed_obj->object_name();
my $object_type= $changed_obj->object_type();
my $changed_properties= $changed_obj->changed_properties();
my $property1 = $changed_properties->[0];
```

#Get and modify a ScheduledTask object

```
#Get the ScheduledTask object from Infoblox appliance through a session
my @retrieved_objs = $session->get(
    object         => "Infoblox::Grid::ScheduledTask",
    scheduled_time => "2020-01-01T14:52:00Z"
);
my $object = $retrieved_objs[0];

unless ($object) {
```

```
    die("Get ScheduledTask object failed: ",
        $session->status_code() . ":" . $session->status_detail());
}
print "Get ScheduledTask object found at least 1 matching entry\n";

#Modify the ScheduledTask object
$object->scheduled_time("2020-02-01T14:52:00Z");

#Apply the changes.
$session->modify($object)
    or die("Modify ScheduledTask object failed: ",
    $session->status_code() . ":" . $session->status_detail());
print "ScheduledTask object modified successfully \n";
```

#Remove a ScheduledTask object

```
#Get the ScheduledTask object through the session
my @retrieved_objs = $session->get(
    object => "Infoblox::Grid::ScheduledTask",
    action => "Add"
);
my $object = $retrieved_objs[0];
unless ($object) {
    die("Get ScheduledTask object failed: ",
        $session->status_code() . ":" . $session->status_detail());
}
print "Get ScheduledTask object found at least 1 matching entry\n";

#Submit the object for removal
$session->remove($object)
    or die("Remove ScheduledTask object failed: ",
    $session->status_code() . ":" . $session->status_detail());
print "ScheduledTask object removed successfully \n";
```

#Cleanup

```
#Get the Network object through the session
my @retrieved_objs = $session->get(
    object  => "Infoblox::DHCP::Network",
    network => "10.0.0.0/24"
);
my $object = $retrieved_objs[0];
unless ($object) {
    die("Get Network object failed: ",
        $session->status_code() . ":" . $session->status_detail());
}
print "Get Network object found at least 1 matching entry\n";

#Submit the object for removal
$session->remove($object)
    or die("Remove Network object failed: ",
    $session->status_code() . ":" . $session->status_detail());
print "Network object removed successfully \n";

####PROGRAM ENDS####
```

AUTHOR

Infoblox Inc. http://www.infoblox.com/

SEE ALSO

Infoblox::Grid::ScheduledTask, Infoblox::Grid::ScheduledTask->changed_objects()

COPYRIGHT

Copyright (c) 2017 Infoblox Inc.

Infoblox::Grid::ScheduleSetting - a schedule setting object.

- NAME
- DESCRIPTION
- CONSTRUCTOR
- SESSION METHODS
- METHODS
  - day_of_month( )
  - disable( )
  - every( )
  - frequency( )
  - hour_of_day( )
  - month( )
  - year( )
  - minutes_past_hour( )
  - repeat( )
  - time_zone( )
  - weekdays( )
- AUTHOR
- SEE ALSO
- COPYRIGHT

NAME

Infoblox::Grid::ScheduleSetting - a schedule setting object.

DESCRIPTION

An object that provides the ability to set the schedule for a discovery task.

CONSTRUCTOR

```
my $schedule_setting = Infoblox::Grid::ScheduleSetting->new(
   frequency           => "DAILY" | "HOURLY" | "MONTHLY" | "WEEKLY",      # optional
   day_of_month        => $uint,                                          # optional
   every               => $uint,                                          # optional
   weekdays            => ["MONDAY", "THURSDAY", .... ],                  # optional
   hour_of_day         => $uint,                                          # optional
   minutes_past_hour   => $uint,                                          # optional
   month               => $uint,                                          # optional
   year                => $uint,                                          # optional
   repeat              => "ONCE" | "RECUR",                               # optional
   disable             => "true" | "false",                               # optional
   time_zone           => $time_zone,                                     # optional
);
```

SESSION METHODS

The object does not support any session methods.

METHODS

This section describes all the methods that you can use to configure and retrieve attribute values of the scheduling object.

day_of_month( )

Use this method to set or retrieve the scheduled day of month when the discovery is triggered.

Parameter

Valid value is an unsigned integer between 1-31.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get attribute value
my $value = $schedule_setting->day_of_month();
#Modify attribute value
$schedule_setting->day_of_month(10);
```

disable( )

Use this method to enable or disable scheduling of a discovery task.

Parameter

Specify "true" to disable scheduling or "false" to enable it. The default value is "false".

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get attribute value
my $value = $schedule_setting->disable();
#Modify attribute value
$schedule_setting->disable("true");
```

every( )

Use this method to set or retrieve the recurrence pattern of a recurring discovery.

Parameter

Valid value is an unsigned integer.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get attribute value
my $value = $schedule_setting->every();
#Modify attribute value
$schedule_setting->every(2);
```

frequency( )

Use this method ot set or retrieve the recurrence pattern of a recurring discovery.

Parameter

Valid value is "DAILY", "HOURLY", "MONTHLY" or "WEEKLY".

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get attribute value
my $value = $schedule_setting->frequency();
#Modify attribute value
$schedule_setting->frequency("MONTHLY");
```

hour_of_day( )

Use this method to set or retrieve the hour, based on the 24-hour clock notation, when the discovery should start. For example, use 18 for 18:00.

Parameter

Valid value is an unsigned integer between 0-23.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get attribute value
my $value = $schedule_setting->hour_of_day();
#Modify attribute value
$schedule_setting->hour_of_day(5);
```

month( )

Use this method to set or retrieve the month of the year when the discovery should start. For example, use 1 for January, 2 for February, and so on.

Parameter

Valid value is an unsigned integer between 1 and 12.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get attribute value
my $value = $schedule_setting->month();
#Modify attribute value
$schedule_setting->month(10);
```

year( )

Use this method to set or retrieve the year when the discovery should start.

Parameter

Valid value is an unsigned integer.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get attribute value
my $value = $schedule_setting->year();
#Modify attribute value
$schedule_setting->year(2015);
```

minutes_past_hour( )

Use this method to set or retrieve the minute, based on the 24-hour clock notation, when the discovery should start. For example, use 59 for 00:59.

Parameter

Valid value is an unsigned integer between 0-59.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get attribute value
my $value = $schedule_setting->minutes_past_hour();
#Modify attribute value
$schedule_setting->minutes_past_hour(10);
```

repeat( )

Use this method to set or retrieve the flag that indicates if the discovery is scheduled for once or recurring.

Parameter

Valid value is "ONCE" or "RECUR".

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get attribute value
my $value = $schedule_setting->repeat();
#Modify attribute value
$schedule_setting->repeat("RECUR");
```

time_zone( )

Use this method to set or retrieve the time zone.

Parameter

The UTC string that represents the time zone. For example "(UTC - 6:00) Central Time (US and Canada)". The default value is "(UTC) Coordinated Universal Time".

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get attribute value
my $value = $schedule_setting->time_zone();
#Modify attribute value
$schedule_setting->time_zone("(UTC - 6:00) Central Time (US and Canada)");
```

weekdays( )

Use this method to set or retrieve the days of the week when the discovery is triggered.

Parameter

Valid value is a reference to an array that can contain the following values: 'SUNDAY', 'MONDAY', 'TUESDAY', 'WEDNESDAY', 'THURSDAY', 'FRIDAY' and 'SATURDAY'.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get attribute value
my $value = $schedule_setting->weekdays();
#Modify attribute value
$schedule_setting->weekdays(["MONDAY", "FRIDAY"]);
```

AUTHOR

Infoblox Inc. http://www.infoblox.com/

SEE ALSO

Infoblox::IPAM::DiscoveryTask

COPYRIGHT

Copyright (c) 2017 Infoblox Inc.

Infoblox::Grid::ServiceRestart - a Grid Service Restart setting object.

- NAME
- DESCRIPTION
- CONSTRUCTOR
- SESSION METHODS
- MODULE METHODS
  - Infoblox::Grid::DNS->restart_setting( )
  - Infoblox::Grid::DHCP->restart_setting( )
- METHODS
  - delay( )
  - restart_offline( )
  - timeout( )
- AUTHOR
- SEE ALSO
- COPYRIGHT

NAME

Infoblox::Grid::ServiceRestart - a Grid Service Restart setting object.

DESCRIPTION

The Grid Service Restart setting object provides information about service restart configurations.

CONSTRUCTOR

```
my $service_restart = Infoblox::Grid::ServiceRestart->new(
   delay           => $uint,              # Optional / Default is 10
   restart_offline => 'true' | 'false', # Optional / Default is 'true'
   timeout         => $int,               # Optional / Default is 60
```

SESSION METHODS

The object does not support any session methods.

MODULE METHODS

The following functions are available to apply to a Grid Service Restart object.

Infoblox::Grid::DNS->restart_setting( )

Use this method to set or retrive Grid DNS service restart settings. See Infoblox::Grid::DNS->restart_setting() for parameters and return values.

Example

```
#Get restart_setting
my $restart_setting = $grid_dns->restart_setting();
#Modify the retry time
$grid_dns->restart_setting($restart_setting);
```

Infoblox::Grid::DHCP->restart_setting( )

Use this method to set or retrive Grid DHCP service restart settings. See Infoblox::Grid::DHCP->restart_setting() for parameters and return values.

Example

```
#Get restart_setting
my $restart_setting = $grid_dns->restart_setting();
#Modify the retry time
$grid_dns->restart_setting($restart_setting);
```

METHODS

delay( )

Use this method to set or retrieve restart delay in seconds between Grid Service Groups.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

The valid value is an unsigned integer. The default value is 10.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get delay
my $delay = $svcrst->delay();
#Modify delay
$svcrsts->delay(50);
```

restart_offline( )

Use this method to enable or disable the restart of offline Grid Members.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

Specify 'true' to enable the restart of offline Grid Services or 'false' to disable it. The default value is 'true'.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get restart_offline
my $restart_offline = $svcrst->restart_offline();
#Modify restart_offline
$svcrsts->restart_offline('false');
```

timeout( )

Use this method to set or retrieve timeout interval in seconds for Grid Service Groups restart.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

The valid value is an integer greater or equal to -1 (where -1 means infinite). The default value is 10.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get timeout
my $timeout = $svcrst->timeout();
#Modify timeout
$svcrst->timeout(50);
```

AUTHOR

Infoblox Inc. http://www.infoblox.com/

SEE ALSO

Infoblox::Grid::DNS, Infoblox::Grid::DNS->restart_setting(), Infoblox::Grid::DHCP, Infoblox::Grid::DHCP->restart_setting()

COPYRIGHT

Infoblox::Grid::ServiceRestart - a Grid Service Restart setting object.

Copyright (c) 2017 Infoblox Inc.

Infoblox::Grid::ServiceRestart::Group - a Grid Service Restart Group object.

- NAME
- DESCRIPTION
- CONSTRUCTOR
- SESSION METHODS
  - Infoblox::Session->add( )
  - Infoblox::Session->get( )
  - Infoblox::Session->modify( )
  - Infoblox::Session->remove( )
  - Infoblox::Session->search( )
- METHODS
  - comment( )
  - extattrs( )
  - extensible_attributes( )
  - is_default( )
  - last_updated_time( )
  - members( )
  - mode( )
  - name( )
  - position( )
  - recurring_schedule( )
  - requests( )
  - service( )
  - status( )
- AUTHOR
- SEE ALSO
- COPYRIGHT

NAME

Infoblox::Grid::ServiceRestart::Group - a Grid Service Restart Group object.

DESCRIPTION

The Grid Service Restart Group object provides information about restart of particular services and members, their order and periodicity.

CONSTRUCTOR

```
 my $svcrst_group = Infoblox::Grid::ServiceRestart::Group->new(
    name                  => $string,                                                  #
Required
    service               => 'DHCP' | 'DNS',                                           #
Required
    comment               => $string,                                                  #
Optional / Default is undefined
    mode                  => 'SIMULTANEOUS' | 'SEQUENTIAL',                            #
Optional / Default is
                                                                                       #
'SEQUENTIAL' for default restart groups /                                              #
'SIMULTANEOUS' for non-default restart groups
    members               => $string_array,                                            #
Optional / Default is undefined
    recurring_schedule    => $svcrst_schedule,                                         #
Optional / Default is undefined
    extattrs              => { $string => $extattr, ... },                             #
Optional / Default is undefined
    extensible_attributes => { $string => $string | $num, $string => [ $string | $num, ... ], ... } #
Optional / Default is undefined
 );
```

SESSION METHODS

This section describes all the methods in a Session module that you can apply to a Grid Service Restart Group object.

Infoblox::Session->add( )

Use this method to add an object to the Infoblox appliance. See Infoblox::Session->add() for parameters and return values.

Example

```
#Construct an object
my $svcrst_group = Infoblox::Grid::ServiceRestart::Group->new(
   name    => 'test_group',
   service => 'DHCP',
);

#Submit for addition
my $response = $session->add($svcrst_group);
```

Infoblox::Session->get( )

Use this method to retrieve all the matching objects from the Infoblox appliance. See Infoblox::Session->get() for parameters and return values.

Key Reference

```
Apply the following attribute to get a specific object:

  comment               - Optional. A Grid Service Restart Group comment.
  extattrs              - Optional. A hash reference containing extensible attributes.
  extensible_attributes - Optional. A hash reference that contains extensible attributes.
  is_default            - Optional. Flag which indicates whether the Grid Service Restart Group is a
default restart group or not.
  name                  - Optional. A Grid Service Restart Group name.
  position              - Optional. A Grid Service Restart Group position in Restart Service Order
object.
  service               - Optional. A Grid Service Restart Group service name.
```

Example

```
my @retrieved_objs = $session->get(
    object => 'Infoblox::Grid::ServiceRestart::Group',
    name   => 'test_group',
);

my @retrieved_objs = $session->get(
    object => 'Infoblox::Grid::ServiceRestart::Group',
    extensible_attributes => { 'Site' => 'Santa Clara' }
);
```

Infoblox::Session->modify( )

Use this method to modify an object in the Infoblox appliance. See Infoblox::Session->modify() for parameters and return values.

Example

```
#Use method to modify the comment.
$svcrst_group->comment('this is a modified comment');
#Submit modification
my $response = $session->modify($svcrst_group);
```

Infoblox::Session->remove( )

Use this method to remove an object from the Infoblox appliance. See Infoblox::Session->remove() for parameters and return values.

To remove a specific object, first use `get()` or `search()` to retrieve the specific object, and then submit this object for removal.

Example

```
#Get the objects with the same name
my @retrieved_objs = $session->get(
    object => 'Infoblox::Grid::ServiceRestart::Group',
    name   => 'svcrst_group1',
);
#Find the desired object from the retrieved list.
my $desired_svcrst_group = $retrieved_objs[0];
#Submit for removal
my $response = $session->remove($desired_svcrst_group);
```

Infoblox::Session->search( )

Use this method to search for objects in the Infoblox appliance. See Infoblox::Session->search() for parameters and return values.

Key Reference

```
Apply the following attribute to get a specific object:

  comment               - Optional. A Grid Service Restart Group comment.
  extattrs              - Optional. A hash reference containing extensible attributes.
```

```
  extensible_attributes - Optional. A hash reference that contains extensible attributes.
  is_default            - Optional. Flag which indicates whether the Grid Service Restart Group is a
default restart group or not.
  name                  - Optional. A Grid Service Restart Group name.
  position              - Optional. A Grid Service Restart Group position in Restart Service Order
object.
  service               - Optional. A Grid Service Restart Group service name.
```

Example

```
my @retrieved_objs = $session->search(
    object => 'Infoblox::Grid::ServiceRestart::Group',
    name   => 'svcrst.*',
);

my @retrieved_objs = $session->search(
    object => 'Infoblox::Grid::ServiceRestart::Group',
    extensible_attributes => { 'Site' => 'Santa Clara' }
);
```

METHODS

This section describes all the methods that you can use to set or retrieve the attribute values of the object.

comment( )

Use this method to set or retrieve Grid Service Restart Group object comment.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

The valid value is a desired comment in string format.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get comment
my $comment = $svcrst_group->comment();
#Modify comment
$svcrst_group->comment(50);
```

extattrs( )

Use this method to set or retrieve the extensible attributes associated with a Grid Service Restart Group object.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

Valid value is a hash reference containing the names of extensible attributes and their associated values (Infoblox::Grid::Extattr objects).

Returns

If you specified a parameter, the method returns 'true' when the modification succeeds, and returns 'false' when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#get extattrs value
my $ref_extattrs = $svcrst_group->extattrs();

#Modify extattrs
$svcrst_group->extattrs({ 'Site' => $extattr1, 'Administrator' => $extattr2 });
```

extensible_attributes( )

Use this method to set or retrieve the extensible attributes associated with a Grid Service Restart Group object.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

For valid values for extensible attributes, see Infoblox::Grid::ExtensibleAttributeDef/Extensible Attribute Values.

Returns

If you specified a parameter, the method returns 'true' when the modification succeeds, and returns 'false' when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get extensible attributes
my $ref_extensible_attributes = $svcrst_group->extensible_attributes();

#Modify extensible attributes
$svcrst_group->extensible_attributes({'Site' => 'Santa Clara', 'Administrator' => ['Peter',
'Tom']});
```

is_default( )

Use this method to retrieve the flag which indicates whether the Grid Service Restart Group is a default group or not. This is a read-only attribute.

Omit the parameter to retrieve the attribute value.

Parameter

None

Returns

The method returns attribute value.

Example

```
#Get is_default
my $is_default = $svcrst_group->is_default();
```

last_updated_time( )

Use this method to retrieve time of the last restart of the Grid Service Restart Group. This is the read-only attribute.

Omit the parameter tor retrieve the attribute value.

Parameter

None

Returns

The method returns the number of seconds that have elapsed since January 1st, 1970 UTC.

Example

```
#Get last_updated_time
my $last_updated_time = $svcrst_group->last_updated_time()
```

members( )

Use this method to set or retrieve Grid Members associated with the Grid Service Restart Group.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

The valid value is an array reference of desired Grid Members names.

Returns

If you specified a parameter, the method returns 'true' when the modification succeeds, and returns 'false' when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get members
my $members = $svcrst_group->members();
#Modify members
$svcrst_group->members(['foo.com', 'bar.com']);
```

mode( )

Use this method to set or retrieve the restart mode of Grid Members in the Grid Service Restart Group.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Note that if the Service Restart Group is a default one, the default value is 'SEQUENTIAL', 'SIMULTANEOUS' otherwise.

Parameter

The valid values are 'SEQUENTIAL' and 'SIMULTANEOUS'.

Returns

If you specified a parameter, the method returns 'true' when the modification succeeds, and returns 'false' when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get mode
my $mode = $svcrst_group->mode();
#Modify mode
$svcrst_group->mode('SEQUENTIAL');
```

name( )

Use this method to set or retrieve the Grid Service Restart Group name.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

The valid value is a desired name in a string format.

Returns

If you specified a parameter, the method returns 'true' when the modification succeeds, and returns 'false' when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get name
my $name = $svcrst_group->name();
#Modify name
$svcrst_group->name('new_name');
```

position( )

Use this method to retrieve the restart position of the Grid Service Restart Group. This is a read-only attribute.

Omit the parameter to retrieve the attribute value.

Parameter

None

Returns

The method returns the attribute value.

Example

```
#Get position
my $position = $svcrst_group->position();
```

recurring_schedule( )

Use this method to set or retrieve the recurring schedule of the Grid Service Restart Group.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

The valid value is an Infoblox::Grid::ServiceRestart::Group::Schedule object.

Returns

If you specified a parameter, the method returns 'true' when the modification succeeds, and returns 'false' when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get recurring_schedule
my $recurring_schedule = $svcrst_group->recurring_schedule();
#Modify recurring_schedule
$svcrst_group->recurring_schedule($svcrst_sched);
```

requests( )

Use this method to retrieve the array of Grid Service Restart Requests. This is a read-only attribute.

Omit the parameter to retrieve the attribute value.

Parameter

None

Returns

The valid return value is an array of Infoblox::Grid::ServiceRestart::Request objects.

Example

```
#Get requests
my $requests = $svcrst_group->requests();
```

service( )

Use this method to set or retrieve the service name for the Grid Service Restart Group.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

The valid value is 'DHCP' or 'DNS'.

Returns

If you specified a parameter, the method returns 'true' when the modification succeeds, and returns 'false' when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get service
my $service = $svcrst_group->service();
#Modify service
$svcrst_group->service('DHCP');
```

status( )

Use this method to retrieve the Grid Service Restart Group restart status. This is a read-only attribute.

Omit the parameter to retrieve the attribute value.

Parameter

None

Returns

The valid return value is an Infoblox::Grid::ServiceRestart::Status object.

Example

```
#Get status
my $status = $svcrst_group->status();
```

AUTHOR

Infoblox Inc. http://www.infoblox.com/

SEE ALSO

Infoblox::Session, Infoblox::Session->add(), Infoblox::Session->get(), Infoblox::Session->modify(), Infoblox::Session->remove(), Infoblox::Session->search(), Infoblox::Grid::Extattr, Infoblox::Grid::ExtensibleAttributeDef/Extensible Attribute Values, Infoblox::Grid::ServiceRestart::Group::Schedule, Infoblox::Grid::ServiceRestart::Request, Infoblox::Grid::ServiceRestart::Status.

COPYRIGHT

Copyright (c) 2017 Infoblox Inc.

Infoblox::Grid::ServiceRestart::Group::Order - A Grid Service Restart Group Order object.

- NAME
- DESCRIPTION
- CONSTRUCTOR
- SESSION METHODS
  - Infoblox::Session->add( )
- METHODS
  - groups( )
- AUTHOR
- SEE ALSO
- COPYRIGHT

NAME

Infoblox::Grid::ServiceRestart::Group::Order - A Grid Service Restart Group Order object.

DESCRIPTION

The Grid Service Restart Group Order object controls the restart order of the Service Restart Group for a particular service ('DHCP' or 'DNS'). To specify a restart order, gather all the groups you want to restart for a particular service and then place the Restart Service Groups in the desired order.

Note that the default groups should be placed at the end of the list.

CONSTRUCTOR

```
my $group_order = Infoblox::Grid::ServiceRestart::Group::Order->new(
    groups => [$group1, $group2, ..., $default_group], #Required
);
```

SESSION METHODS

This section describes all the methods in a Session module that you can apply to a Grid Service Restart Group object.

Infoblox::Session->add( )

Use this method to add an object to the Infoblox appliance. See Infoblox::Session->add() for parameters and return values.

Example

```
#Construct an object
my $svcrst_group_order = Infoblox::Grid::ServiceRestart::Group::Order->new(
   groups => [$group1, $group2, $default_group],
);

#Submit for addition
my $response = $session->add($svcrst_group_order);
```

METHODS

This section describes all the methods that you can use to set or retrieve the attribute values of the object.

groups( )

Use this method to set or retrieve the ordered list for Grid Service Restart Groups.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

The valid value is an array of Infoblox::Grid::ServiceRestart::Group objects.

Note that all groups for the specified service ('DHCP' or 'DNS') are required. Default groups should be placed at the end of the list.

Returns

If you specified a parameter, the method returns 'true' when the modification succeeds, and returns 'false' when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get groups
my $groups = $svcrst_gr_order->groups();
#Modify groups
$svcrst_gr_order->groups([$group1, $group2, $group3]);
```

AUTHOR

Infoblox Inc. http://www.infoblox.com/

SEE ALSO

Infoblox::Session, Infoblox::Session->add(), Infoblox::Grid::ServiceRestart::Group.

COPYRIGHT

Copyright (c) 2017 Infoblox Inc.

Infoblox::Grid::ServiceRestart::Group::Schedule - A Grid Service Restart Group Schedule object.

- NAME
- DESCRIPTION
- CONSTRUCTOR
- MODULE METHODS
  - Infoblox::Grid::ServiceRestart::Group->recurring_schedule( )
- METHODS
  - services( )
  - mode( )
  - force( )
  - schedule( )
- AUTHOR
- SEE ALSO
- COPYRIGHT

NAME

Infoblox::Grid::ServiceRestart::Group::Schedule - A Grid Service Restart Group Schedule object.

DESCRIPTION

The Grid Service Restart Group Schedule object provides information about restart service schedule settings for Service Restart Group.

CONSTRUCTOR

```
my $svcrst_schedule = Infoblox::Grid::ServiceRestart::Group::Schedule->new(
   schedule => $schedule,                                  # Required
   services => $array_of_services,                         # Optional / Default is ['ALL']
   mode     => 'GROUPED' | 'SEQUENTIAL' | 'SIMULTANEOUS', # Optional / Default is 'GROUPED'
   force    => 'true' | 'false',                           # Optional / Default is 'false'
);
```

MODULE METHODS

Infoblox::Grid::ServiceRestart::Group->recurring_schedule( )

Use this method to set or retrieve the Grid Service Restart Group Schedule. See Infoblox::Grid::ServiceRestart::Group->recurring_schedule() for parameters and return values.

Example

```
#Get recurring_schedule
my $recurring_schedule = $svcrst_group->recurring_schedule();
#Modify recurring_schedule
$svcrst_group->recurring_schedule($sched);
```

METHODS

This section describes all the methods that you can use to set or retrieve the attribute values of the object.

services( )

Use this method to set or retrieve the list of services for a scheduled restart for a particular Grid Service Restart Group.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

The valid value is an array which contains following values: 'ALL', 'DNS', 'DHCP', 'DHCPV4', 'DHCPV6'.

Returns

If you specified a parameter, the method returns 'true' when the modification succeeds, and returns 'false' when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get services
my $services = $svcrst_schedule->services();
#Modify services
$svcrst_schedule->services('DHCP');
```

mode( )

Use this method to set or retrieve the service restart mode for a scheduled restart for a particular Grid Service Restart Group.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

The valid values are 'GROUPED', 'SEQUENTIAL' and 'SIMULTANEOUS'.

Returns

If you specified a parameter, the method returns 'true' when the modification succeeds, and returns 'false' when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get mode
my $mode = $svcrst_schedule->mode();
#Modify mode
$svcrst_schedule->mode('SEQUENTIAL');
```

force( )

Use this method to set or retrieve the flag that indicates whether a forced scheduled restart will be performed for a particular Grid Service Restart Group.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

Specify 'true' to set the flag, and 'false' to unset/deactivate it. The default value is 'false'.

Returns

If you specified a parameter, the method returns 'true' when the modification succeeds, and returns 'false' when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get force
my $force = $svcrst_schedule->force();
#Modify force
$svcrst_schedule->force('true');
```

schedule( )

Use this method to set or retrieve the schedule settings for a particular Grid Service Restart Group.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

The valid value is an Infoblox::Grid::ScheduleSetting object.

Returns

If you specified a parameter, the method returns 'true' when the modification succeeds, and returns 'false' when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get schedule
my $schedule = $svcrst_schedule->schedule();
#Modify schedule
$svcrst_schedule->schedule($sched);
```

Infoblox::Grid::ServiceRestart::Group::Schedule - A Grid Service Restart Group Schedule object.

AUTHOR

Infoblox Inc. http://www.infoblox.com/

SEE ALSO

Infoblox::Grid::ServiceRestart::Group, Infoblox::Grid::ServiceRestart::Group->recurring_schedule(), Infoblox::Grid::ScheduleSetting.

COPYRIGHT

Copyright (c) 2017 Infoblox Inc.

Infoblox::Grid::ServiceRestart::Request - A Grid Service Restart Request object.

- NAME
- DESCRIPTION
- SESSION METHODS
  - Infoblox::Session->get( )
  - Infoblox::Session->search( )
- MODULE METHODS
  - Infoblox::Grid::ServiceRestart::Group->requests( )
- METHODS
  - error( )
  - forced( )
  - group( )
  - last_updated_time( )
  - member( )
  - needed( )
  - order( )
  - result( )
  - service( )
  - state( )
- AUTHOR
- SEE ALSO
- COPYRIGHT

NAME

Infoblox::Grid::ServiceRestart::Request - A Grid Service Restart Request object.

DESCRIPTION

The Grid Service Restart Request object provides information and statistics about restart service routine of the Restart Service Group.

SESSION METHODS

This section describes all the methods in a Session module that you can apply to a Grid Service Restart Request object.

Infoblox::Session->get( )

Use this method to retrieve all the matching objects from the Infoblox appliance. See Infoblox::Session->get() for parameters and return values.

Key Reference

```
Apply the following attribute to get a specific object:

group  - Optional. A Grid Service Restart Group object.
member - Optional. A Grid Member name or object.
```

Example

```
my @retrieved_objs = $session->get(
    object => 'Infoblox::Grid::ServiceRestart::Request',
    group  => $group,
);
```

Infoblox::Session->search( )

Use this method to search for objects in the Infoblox appliance. See Infoblox::Session->search() for parameters and return values.

Key Reference

```
Apply the following attribute to search for a specific object:

group  - A Grid Service Restart Group object.
member - A Grid Member name or object.
```

Example

```
my @retrieved_objs = $session->search(
    object => 'Infoblox::Grid::ServiceRestart::Request',
    group  => $group,
```

```
);
```

MODULE METHODS

The following functions are available to apply to a Grid Service Restart Request object.

Infoblox::Grid::ServiceRestart::Group->requests( )

Use this method to retrieve restart requests for the Grid Service Restart Group. This is a read-only attribute. See Infoblox::Grid::ServiceRestart::Group->requests() for parameters and return values.

Example

```
#Get requests
my $requests = $svcrst_group->requests();
```

METHODS

This section describes all the methods that you can use to set or retrieve the attribute values of the object.

error( )

Use this method to retrieve error messages for failed restart requests. This is a read-only attribute.

Omit the parameter to retrieve the attribute value.

Parameter

None

Returns

The method returns the attribute value.

Example

```
#Get error
my $error = $svcrst_request->error();
```

forced( )

Use this method to retrieve the flag that indicates whether this is force restart request or not. This is a read-only attribute.

Omit the parameter to retrieve the attribute value.

Parameter

None

Returns

The method returns the attribute value

Example

```
#Get forced
my $forced = $svcrst_request->forced();
```

group( )

Use this method to retrieve the Grid Service Restart Group object that is associated with the request. This is a read-only attribute.

Omit the parameter to retrieve the attribute value.

Parameter

None

Returns

The valid return value is an Infoblox::Grid::ServiceRestart::Group object.

Example

```
#Get group
my $group = $svcrst_request->group();
```

last_updated_time( )

Use this method to retrieve the time when the request status has changed. This is a read-only attribute.

Omit the parameter to retrieve the attribute value.

Parameter

None

Returns

The method returns the number of seconds that have elapsed since January 1st, 1970 UTC.

Example

```
#Get last_updated_time
my $last_updated_time = $svcrst_request->last_updated_time();
```

member( )

Use this method to retrieve the Grid Member name for the restart. This is a read-only attribute.

Omit the parameter to retrieve the attribute value.

Parameter

None

Returns

The method returns the attribute value.

Example

```
#Get member
my $member = $svcrst_request->member();
```

needed( )

Use this method to retrieve the value that indicates whether a restart is required or not. This is a read-only attribute.

Omit the parameter to retrieve the attribute value.

Parameter

None

Returns

The valid return values are 'UNKNOWN', 'CHECKING', 'YES', 'NO' and 'FAILURE'.

Example

```
#Get needed
my $needed = $svcrst_request->needed();
```

order( )

Use this method to retrieve the restart order for the Grid Service Restart Request. This is a read-only attribute.

Omit the parameter to retrieve the attribute value.

Parameter

None

Returns

The method returns attribute value.

Example

```
#Get order
my $order = $svcrst_request->order();
```

result( )

Use this method to retrieve the result of a service restart for a particular Grid Service Restart Request. This is a read-only attribute.

Omit the parameter to retrieve the attribute value.

Parameter

None

Returns

The valid return values are 'SUCCESS', 'TIMEOUT', 'FAILURE' and 'NORESTART'.

Example

```
#Get result
my $result = $svcrst_request->result();
```

service( )

Use this method to retrieve the service to restart for a particular Grid Service Restart Request. This is a read-only attribute.

Omit the parameter to retrieve the attribute value.

Parameter

None

Returns

The valid return values are 'DNS', 'DHCP', 'DHCPV4' and 'DHCPV6'.

Example

```
#Get service
my $service = $svcrst_request->service();
```

state( )

Use this method to retrieve the state of a particular Grid Service Restart Request. This is a read-only attribute.

Omit the parameter to retrieve the attribute value.

Parameter

None

Returns

The valid return values are 'NEW', 'QUEUED', 'PROCESSING' and 'FINISHED'.

Example

```
#Get state
my $state = $svcrst_request->state();
```

AUTHOR

Infoblox Inc. http://www.infoblox.com/

SEE ALSO

Infoblox::Session, Infoblox::Session->get(), Infoblox::Session->search(), Infoblox::Grid::ServiceRestart::Group, Infoblox::Grid::ServiceRestart::Group->requests().

COPYRIGHT

Copyright (c) 2017 Infoblox Inc.

Infoblox::Grid::ServiceRestart::Request::ChangedObject - A Grid service restart request changed object.

- NAME
- DESCRIPTION
- SESSION METHODS
  - Infoblox::Session->get( )
  - Infoblox::Session->search( )
- METHODS
  - action( )
  - changed_time( )
  - changed_properties( )
  - object_name( )
  - object_type( )
  - user_name( )
- AUTHOR
- SEE ALSO
- COPYRIGHT

NAME

Infoblox::Grid::ServiceRestart::Request::ChangedObject - A Grid service restart request changed object.

DESCRIPTION

The Grid service restart request changed object provides information about changes that are waiting for the restart.

SESSION METHODS

This section describes all the methods in a Session module that you can apply to a Grid service restart request changed object.

Infoblox::Session->get( )

Use this method to retrieve all the matching objects from the Infoblox appliance. See Infoblox::Session->get() for parameters and return values.

Key Reference

```
Apply the following attribute to get a specific object:

 source  - Infoblox::Grid::ServiceRestartGroup or Infoblox::Grid::Member object.
 service - string that contains service that is waiting for restart. The default value is 'ALL'.
```

Example

```
my @retrieved_objs = $session->get(
    object  => 'Infoblox::Grid::ServiceRestart::Request::ChangedObject',
    service => 'DHCPV6',
);
```

Infoblox::Session->search( )

Use this method to search for objects in the Infoblox appliance. See Infoblox::Session->search() for parameters and return values.

Key Reference

```
Apply the following attribute to search for a specific object:

 source  - Infoblox::Grid::ServiceRestartGroup or Infoblox::Grid::Member object.
 service - string that contains service that is waiting for changes. The default value is 'ALL'.
```

Example

```
my @retrieved_objs = $session->search(
    object  => 'Infoblox::Grid::ServiceRestart::Request::ChangedObject',
    service => 'DHCPV6',
);
```

METHODS

This section describes all the methods that you can use to set or retrieve the attribute values of the object.

action( )

Use this method to retrieve the operation on the changed object. This is a read-only attribute.

Parameter

None

Returns

The method returns the attribute value.

Example

```
#Get action value
my $value = $object->action();
```

changed_time( )

Use this method to retrieve the timestamp of the change. This is a read-only attribute.

Parameter

None

Returns

The method returns the number of seconds elapsed since January 1, 1970, in UTC format.

Example

```
#Get changed_time value
my $changed_time = $object->changed_time();
```

changed_properties( )

Use this method to retrieve the list of properties changed in the object. This is a read-only attribute.

Parameter

None

Returns

The method returns a reference to an array of strings. Each string in the array describes the changes. The descriptions are similiar to the entries in the Change Audit Log column in the GUI.

Example

```
#Get changed_properties value
my $changed_properties = $object->changed_properties();
```

object_name( )

Use this method to retrieve the name of the changed object. This is a read-only attribute.

Parameter

none

Returns

The method returns the object name in string format.

Example

```
#Get object_name value
my $value = $object->object_name();
```

object_type( )

Use this method to retrieve the type of the changed object. This is a read-only attribute.

Parameter

None

Returns

The method returns the attribute value.

Example

```
#Get object_type value
my $value = $object->object_type();
```

user_name( )

Use this method to retrieve the user name who have changed the object properties. This is a read-only attribute.

Parameter

None

Returns

The method returns the attribute value.

Example

```
#Get user_name value
my $value = $object->user_name();
```

AUTHOR

Infoblox Inc. http://www.infoblox.com/

SEE ALSO

Infoblox::Session, Infoblox::Session->get(), Infoblox::Session->search(),

---

COPYRIGHT

Copyright (c) 2017 Infoblox Inc.

Infoblox::Grid::ServiceRestart::Status - A Grid Service Restart Status object.

- NAME
- DESCRIPTION
- SESSION METHODS
  - Infoblox::Session->get( )
  - Infoblox::Session->search( )
- MODULE METHODS
  - Infoblox::Grid::ServiceRestart::Group->status( )
  - Infoblox::Grid->restart_status( )
- METHODS
  - failures( )
  - finished( )
  - parent( )
  - grouped( )
  - needed_restart( )
  - no_restart( )
  - pending( )
  - pending_restart( )
  - processing( )
  - restarting( )
  - success( )
  - timeouts( )
- AUTHOR
- SEE ALSO
- COPYRIGHT

NAME

Infoblox::Grid::ServiceRestart::Status - A Grid Service Restart Status object.

DESCRIPTION

The Grid Service Restart Status object provides information and statistics about restart service routine of the Grid or Restart Service Group.

SESSION METHODS

This section describes all the methods in a Session module that you can apply to a Grid Service Restart Status object.

Infoblox::Session->get( )

Use this method to retrieve all the matching objects from the Infoblox appliance. See Infoblox::Session->get() for parameters and return values.

Key Reference

```
Apply the following attribute to get a specific object:

parent  - Optional. A Grid Service Restart Group or Grid object.
grouped - Optional. A grouping type of the status which can be either 'GRID' or 'GROUP'.
```

Example

```
my @retrieved_objs = $session->get(
    object  => 'Infoblox::Grid::ServiceRestart::Status',
    parent  => $test_group,
    grouped => 'GROUP',

);
```

Infoblox::Session->search( )

Use this method to search for objects in the Infoblox appliance. See Infoblox::Session->search() for parameters and return values.

Key Reference

```
Apply the following attribute to search for a specific object:

parent  - Optional. A Grid Service Restart Group or Grid object.
grouped - Optional. A grouping type of the status which can be either 'GRID' or 'GROUP'.
```

Example

```
my @retrieved_objs = $session->get(
    object  => 'Infoblox::Grid::ServiceRestart::Status',
    parent  => $test_group,
    grouped => 'GROUP',

);
```

MODULE METHODS

The following functions are available to apply to a Grid Service Restart Status object.

Infoblox::Grid::ServiceRestart::Group->status( )

Use this method to retrieve the Grid Service Restart restart status. This is a read-only attribute. See Infoblox::Grid::ServiceRestart::Group->status() for parameters and return values.

Example

```
#Get status
my $status = $svcrst_group->status();
```

Infoblox::Grid->restart_status( )

Use this method to retrieve the Grid restart status. This is a read-only attribute. See Infoblox::Grid->restart_status() for parameters and return values.

Example

```
#Get status
my $status = $grid->restart_status();
```

METHODS

This section describes all the methods that you can use to set or retrieve the attribute values of the object.

failures( )

Use this method to retrieve the number of failed service restart requests. This is a read-only attribute.

Omit the parameter to retrieve the attribute value.

Parameter

None

Returns

The method returns attribute value.

Example

```
#Get failures
my $failures = $svcrst_status->failures();
```

finished( )

Use this method to retrieve the number of completed service restart requests. This is a read-only attribute.

Omit the parameter to retrieve the attribute value.

Parameter

None

Returns

The method returns attribute value.

Example

```
#Get finished
my $finished = $svcrst_status->finished();
```

parent( )

Use this method to retrieve the Grid or Grid Service Restart Group objects. This is a read-only attribute.

Omit the parameter to retrieve the attribute value.

Parameter

None

Returns

The valid return value can be either Infoblox::Grid or Infoblox::Grid::ServiceRestart::Group object.

Example

```
#Get group
my $group = $svcrst_status->group();
```

grouped( )

Use this method to retrieve the grouping type of the status which can be either 'GRID' or 'GROUP'. This is a read-only attribute.

Omit the parameter to retrieve the attribute value.

Parameter

None

Returns

The method returns attribute value.

Example

```
#Get failures
my $failures = $svcrst_status->failures();
```

needed_restart( )

Use this method to retrieve the number of created but unprocessed restart requests. This is a read-only attribute.

Omit the parameter to retrieve the attribute value.

Parameter

None

Returns

The method returns attribute value.

Example

```
#Get no_restart
my $needed_restart = $svcrst_status->needed_restart();
```

no_restart( )

Use this method to retrieve the number of service restarts that are not required. This is a read-only attribute.

Omit the parameter to retrieve the attribute value.

Parameter

None

Returns

The method returns attribute value.

Example

```
#Get no_restart
my $no_restart = $svcrst_status->no_restart();
```

pending( )

Use this method to retrieve the number of pending service restart requests. This is a read-only attribute.

Omit the parameter to retrieve the attribute value.

Parameter

None

Returns

The method returns attribute value.

Example

```
#Get pending
my $pending = $svcrst_status->pending();
```

pending_restart( )

Use this method to retrieve the number of required force restart requests that are still pending. This is a read-only attribute.

Omit the parameter to retrieve the attribute value.

Parameter

None

Returns

The method returns attribute value.

Example

```
#Get pending_restart
my $pending_restart = $svcrst_status->pending_restart();
```

processing( )

Use this method to retrieve the number of pending restart requests that are not required and not forced. This is a read-only attribute.

Omit the parameter to retrieve the attribute value.

Parameter

None

Returns

The method returns attribute value.

Example

```
#Get processing
my $processing = $svcrst_status->processing();
```

restarting( )

Use this method to retrieve the number of restarting services. This is a read-only attribute.

Omit the parameter to retrieve the attribute value.

Parameter

None

Returns

The method returns attribute value.

Example

```
#Get restarting
my $restarting = $svcrst_status->restarting();
```

success( )

Use this method to retrieve the number of successful service restart requests. This is a read-only attribute.

Omit the parameter to retrieve the attribute value.

Parameter

None

Returns

The method returns attribute value.

Example

```
#Get success
my $success = $svcrst_status->success();
```

timeouts( )

Use this method to retrieve the number of timeouts for service restart requests. This is a read-only attribute.

Omit the parameter to retrieve the attribute value.

Parameter

None

Returns

The method returns attribute value.

Example

```
#Get timeouts
my $timeouts = $svcrst_status->timeouts();
```

AUTHOR

Infoblox Inc. http://www.infoblox.com/

SEE ALSO

Infoblox::Session, Infoblox::Session->get(), Infoblox::Session->search(), Infoblox::Grid::ServiceRestart::Group, Infoblox::Grid::ServiceRestart::Group->status(), Infoblox::Grid, Infoblox::Grid->restart_status().

COPYRIGHT

Copyright (c) 2017 Infoblox Inc.

Infoblox::Grid::SmartFolder::GroupBy - Smart Folder Group By object

- NAME
- DESCRIPTION
- CONSTRUCTOR
- MODULE METHODS
  - Infoblox::Grid::GlobalSmartFolder->group_bys( )
  - Infoblox::Grid::PersonalSmartFolder->group_bys( )
- METHODS
  - value( )
  - value_type( )
  - enable_grouping( )
- SAMPLE CODE
- AUTHOR
- SEE ALSO
- COPYRIGHT

NAME

Infoblox::Grid::SmartFolder::GroupBy - Smart Folder Group By object

---

DESCRIPTION

Defines grouping information in a global or personal smart folder.

CONSTRUCTOR

```
my $group_by = Infoblox::Grid::SmartFolder::GroupBy->new(
    value                => $string,            #Required
    enable_grouping      => "true" | "false",   #Required
    value_type           => "EXTATTR" | "NORMAL", #Optional / Default is "EXTATTR"
);
```

MODULE METHODS

The following functions can be applied to a GroupBy object.

Infoblox::Grid::GlobalSmartFolder->group_bys( )

Use this function to specify a list of "group by" items to store in a global smart folder.

Example

```
#Construct an object
my $group_by_state = Infoblox::Grid::SmartFolder::GroupBy->new(
                                                    enable_grouping => 'true',
                                                    value => "State",
                                                    value_type => "EXTATTR",
                                                   );

my $group_by_vcluster = Infoblox::Grid::SmartFolder::GroupBy->new(
                                                  enable_grouping => 'true',
                                                  value => "v_cluster",
                                                  value_type => "NORMAL",
                                                 );
# Configure group_by items
my $response = $GlobalSmartFolder->group_bys([$group_by_state,$group_by_vcluster]);
```

Infoblox::Grid::PersonalSmartFolder->group_bys( )

Use this function to specify a list of "group by" items to store in a personal smart folder.

Example

```
#Construct an object
my $group_by_state = Infoblox::Grid::SmartFolder::GroupBy->new(
                                                    enable_grouping => 'true',
                                                    value => "State",
                                                    value_type => "EXTATTR",
                                                   );
```

```
my $group_by_vcluster = Infoblox::Grid::SmartFolder::GroupBy->new(
                                                        enable_grouping => 'true',
                                                        value => "v_cluster",
                                                        value_type => "NORMAL",
                                                       );
# Configure group_by items
my $response = $PersonalSmartFolder->group_bys([$group_by_state,$group_by_vcluster]);
```

METHODS

This section describes all the methods that you can use to configure and retrieve the attribute values of a SmartFolder::GroupBy object.

value( )

Use this method to specify the name of the attribute (when value_type is "EXTATTR" or "NORMAL"). The possible values for the value attribute are v_datacenter, v_cluster, v_host, v_entity_type, port_vlan_name, network_component_name, network_component_port_name, and any valid extensible attribute in the appliance.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

Valid values are strings corresponding to extensible attributes, object types, or object names.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
# Getting value
my $value = $group_by->value();
# Modifying value
$group_by->value("Country");
```

value_type( )

Use this method to set or retrieve the value type.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

"EXTATTR" and "NORMAL" are supported.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
# Getting value_type
my $value_type = $group_by->value_type();
# Modifying value_type
$group_by->value_type("EXTATTR");
```

enable_grouping( )

Use this method to set or retrieve the enable grouping attribute.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

Specify "true" to set the enable_grouping flag or "false" to deactivate/unset it. "group by" rules where enable_grouping is set to "false" will be ignored when retrieving the corresponding smart folder.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
# Getting enable_grouping
my $enable_grouping = $group_by->enable_grouping();
# Modifying enable_grouping
$group_by->enable_grouping("true");
```

SAMPLE CODE

##Preparation prior to a smart folder group by object insertion

```
#PROGRAM STARTS: Include all the modules that will be used
use strict;
use Infoblox;

#Create a session to the Infoblox appliance
my $session = Infoblox::Session->new(
    master   => "192.168.1.2",
    username => "admin",
    password => "infoblox"
);
unless ($session) {
   die("Construct session failed: ",
       Infoblox::status_code() . ":" . Infoblox::status_detail());
}
print "Session created successfully\n";
```

#Create a smart folder query item object

```
my $group_by_state = Infoblox::Grid::SmartFolder::GroupBy->new(
                                                        enable_grouping => 'true',
                                                        value => "State",
                                                        value_type => "EXTATTR",
                                                       );
unless ($group_by_state){
     die("Construct smart folder group_by item failed: ",
           Infoblox::status_code() . ":" . Infoblox::status_detail());
}

print "SmartFolder::GroupBy object created successfully\n";
```

#Create a personal smart folder and use $group_by

```
 #create a personal smart folder and use $query_item
 my $smart_folder = Infoblox::Grid::PersonalSmartFolder->new(
    name         => "my_personal_folder",
    group_bys    => [$group_by_state],
  );

#put smart folder into session
$session->add($smart_folder)
  or die("Smart folder creation failed: ",
           Infoblox::status_code() . ":" . Infoblox::status_detail());

print "Smart folder creation successful\n";
```

#Clean up

```
my @result = $session->get(
    object => "Infoblox::Grid::PersonalSmartFolder",
    name   => "my_personal_folder"
);

unless (scalar(@result) == 0) {
    $session->remove($result[0]);
}

####PROGRAM ENDS####
```

AUTHOR

Infoblox Inc. http://www.infoblox.com/

SEE ALSO

Infoblox::Session, Infoblox::Session->add(), Infoblox::Session->get(), Infoblox::Session->modify(),Infoblox::Grid::GlobalSmartFolder,Infoblox::Grid::PersonalSmartFolder

---

COPYRIGHT

Copyright (c) 2017 Infoblox Inc.

Infoblox::Grid::SmartFolder::GroupByValue - Smart Folder Group By Value object

- NAME
- DESCRIPTION
- CONSTRUCTOR
- MODULE METHODS
- METHODS
  - name( )
  - value( )
- AUTHOR
- SEE ALSO
- COPYRIGHT

NAME

Infoblox::Grid::SmartFolder::GroupByValue - Smart Folder Group By Value object

DESCRIPTION

Defines a group value as part of an Infoblox::Grid::SmartFolderChildren query.

CONSTRUCTOR

```
my $group_by_value = Infoblox::Grid::SmartFolder::GroupByValue->new(
    name                    => $string,             #Required
    value                   => $string,             #Required
);
```

MODULE METHODS

METHODS

This section describes all the methods that you can use to configure a SmartFolder::GroupByValue object.

name( )

Use this method to specify the extensible attribute, object type, or object name that is used to group objects.

Parameter

Valid values are strings corresponding to the names.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

value( )

Use this method to specify the value of the extensible attribute, object type, or object name.

Parameter

Valid values are strings corresponding to the required extensible or normal attribute values.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

AUTHOR

Infoblox Inc. http://www.infoblox.com/

SEE ALSO

Infoblox::Grid::SmartFolderChildren

COPYRIGHT

Copyright (c) 2017 Infoblox Inc.

Infoblox::Grid::SmartFolder::QueryItem - Smart Folder Query Item object

- NAME
- DESCRIPTION
- CONSTRUCTOR
- MODULE METHODS
  - Infoblox::Grid::GlobalSmartFolder->query_items( )
- METHODS
  - is_extensible_attribute( )
  - name( )
  - operator( )
  - value( )
- SAMPLE CODE
- AUTHOR
- SEE ALSO
- COPYRIGHT

NAME

Infoblox::Grid::SmartFolder::QueryItem - Smart Folder Query Item object

---

DESCRIPTION

Manipulates a Smart Folder Query Item object that contains information about a query to be stored in a global or personal smart folder.

CONSTRUCTOR

```
 my $query_item = Infoblox::Grid::SmartFolder::QueryItem->new(
     name                   => $string,    #Required
     is_extensible_attribute => $string,    #Optional / Default is "true"
     operator               => "eq" | "ne" | "lt" | "gt" | "leq" | "geq" | "begins with" | "does not begin
with" | "ends with" | "does not end with" | "has value" | "does not have value" | "within" | "on" | "today"
| "yesterday" | "this week" | "this month",    #Required
     value                  => $string | $num | $boolean | type of extensible attribute definition,
#Optional / Default is undefined
 );
```

MODULE METHODS

The following functions can be applied to a query item object.

Infoblox::Grid::GlobalSmartFolder->query_items( )

Use this function to specify a list of query items to store in a global smart folder.

Example

```
#Construct an object
  my $query_item1 = Infoblox::Grid::SmartFolder::QueryItem->new(
    name                   => "Building",
    is_extensible_attibute => "true",
    operator               => "eq",
    value                  => "office1"
);

#Construct an object
  my $query_item2 = Infoblox::Grid::SmartFolder::QueryItem->new(
    name                   => "address",
    is_extensible_attibute => "false",
    operator               => "ne",
    value                  => "10.0.0.1"
);

# Configure query items
my $response = $GlobalSmartFolder->query_items([$query_item1,$query_item2]);
```

METHODS

This section describes all the methods that you can use to configure and retrieve the attribute values of a SmartFolder::QueryItem object.

is_extensible_attribute( )

Use this method to indicate whether this smart folder query item is an extensible or not. When the attribute is_extensible_attribute is "true", the query item is an extensible attribute. When it is "false", it is a normal attribute.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

Valid values are "true" or "false".

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
# Getting is_extensible_attribute
my $is_extensible_attribute = $query_item->is_extensible_attribute();
# Modifying is_extensible_attribute
$query_item->is_extensible_attribute("true");
```

name( )

Use this method to set or retrieve the name of the smart folder query item. The possible values for the name attribute are

```
address
circuit_id
comment
disabled
duid
is_leaf_network
last_discovered_timestamp
locked
mac_address
name
network_component_name
network_component_port_name
network_view
port_vlan_name
record_type
remote_id
server_association_type
service
service_status
type
unmanaged
v_cluster
v_datacenter
v_entity_type
v_host
was_conflict
zone_func
zone_type
```

or any valid extensible attribute definition name present in the system.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

Desired name for the smart folder query item.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails. If you did not specify a parameter, the method returns the attribute value.

Example

```
# Getting name
my $name = $query_item->name();
# Modifying name
$query_item->name("Region");
```

operator( )

Use this method to set or retrieve the operator used in the query.

Parameter

Valid values are any of the following strings: "eq", "ne", "lt", "gt", "geq", "leq", "begins with", "does not begin with", "ends with", "does not end with", "has value", "does not have value", "within", "on", "today", "yesterday", "this week", "this month" and "inheritance state equals".

"has_value" and "does not have value" apply to extensible attribute values only.

For valid values for 'value' parameter for the "within" operator, please see the description of the 'value' method.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
# Getting operator
my $operator = $query_item->operator();
# Modifying operator
$query_item->operator("today");
```

value( )

Use this method to set or retrieve the value used in the query.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

Desired value in string format.

For the "within" operator, the following format is supported: n [d/w/m], where n is the number of days/weeks/months depending on the unit following it. The unit is required. For example, 4d (4 days), 1w (1 week), or 7m (7 months)

For the following query items, a boolean value 'true' or 'false' will be expected: "disabled", "is_leaf_network", "locked".

For a "type" query item, a valid value is one of:

```
'All'
'AllNetwork'
'AllTemplate'
'DhcpFailoverAssoc'
'DhcpFilter'
'DHCPObjects'
'DhcpRange'
'DhcpRangeTemplate'
'DNSObjects'
'FixedAddress'
'FixedAddressTemplate'
'Host'
'IPAMObjects'
'IPv4DHCPObjects'
'IPv6DhcpFilter'
'IPv6DHCPObjects'
'IPv6DhcpRange'
'IPv6DhcpRangeTemplate'
'IPv6FixedAddress'
'IPv6FixedAddressTemplate'
'IPv6Lease'
'IPv6Network'
'IPv6NetworkTemplate'
'IPv6OptionSpace'
'IPv6SharedNetwork'
'MacFilterAddress'
'Member'
'Network'
'NetworkTemplate'
'Network/Zone/Range/Member'
'NsGroup'
'OptionFilterMatchRule'
'OptionSpace'
'Reservation'
'ResourceRecord'
'RoamingHost'
'SharedNetwork'
'SharedRecordGroup'
'View'
'Zone'
```

For a "record_type" query item a valid value is one of:

```
'A'
'AAAA'
'All types'
'CNAME'
'DNAME'
'MX'
```

```
'PTR'
'SRV'
'TXT'
```

For a "service_status" query item a valid value is one of:

```
'FAILED'
'INACTIVE'
'OFFLINE'
'WARNING'
'WORKING'
```

For a "zone_func" query item a valid value is one of:

```
'Forward-mapping Zone'
'IPv4 Reverse-mapping Zone'
'IPv6 Reverse-mapping Zone'
```

For a "zone_type" query item a valid value is one of:

```
'Authoritative'
'Delegation'.
'Forward'
'Stub'
```

For a "v_type" query item a valid value is one of:

```
'HostSystem'
'VirtualCenter'
'VirtualMachine'
```

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
# Getting value
my $value = $query_item->value();
# Modifying value
$query_item->value("Santa Clara");
```

SAMPLE CODE

The following sample code demonstrates the different functions that can be applied to an object such as add, modify. This sample also includes error handling for the operations.

##Preparation prior to a smart folder query item object insertion

```
#PROGRAM STARTS: Include all the modules that will be used
use strict;
use Infoblox;

#Create a session to the Infoblox appliance
my $session = Infoblox::Session->new(
    master   => "192.168.1.2",
    username => "admin",
    password => "infoblox"
);
unless ($session) {
   die("Construct session failed: ",
       Infoblox::status_code() . ":" . Infoblox::status_detail());
}
print "Session created successfully\n";
```

#Create a smart folder query item object

```
my $query_item = Infoblox::Grid::SmartFolder::QueryItem->new(
    name                      => "Country",
    is_extensible_attribute   => "true",
    operator                  => "eq",
    value                     => "Canada",
);
unless ($query_item){
     die("Construct smart folder query item failed: ",
           Infoblox::status_code() . ":" . Infoblox::status_detail());
}
```

```
print "SmartFolder::QueryItem object created successfully\n";
```

#Create a personal smart folder and use $query_item

```
#create a personal smart folder and use $query_item
my $smart_folder = Infoblox::Grid::PersonalSmartFolder->new(
   name         => "my_personal_folder",
   query_items  => [$query_item],
 );

#put smart folder into session
$session->add($smart_folder)
  or die("Smart folder creation failed: ",
           Infoblox::status_code() . ":" . Infoblox::status_detail());

print "Smart folder creation successful\n";
```

#Get and modify the query items

```
#get the personal smart folder object from session
my @result = $session->get(
    object => "Infoblox::Grid::PersonalSmartFolder",
    name   => "my_personal_folder"
);

unless (scalar(@result) == 0) {
    my $smart_folder = $result[0];

    if ($smart_folder) {

      #modify query_items
      $query_item->value("France");
      $smart_folder->query_items([$query_item]);

      #update session
      $session->modify($smart_folder)
        or die("modify smart_folder my_personal_folder failed: ",
            $session->status_code(), $session->status_detail());
          }
       } else {
         print "No smart folder found.";
       }
```

#Clean up

```
@result = $session->get(
    object => "Infoblox::Grid::PersonalSmartFolder",
    name   => "my_personal_folder"
);

unless (scalar(@result) == 0) {
    $session->remove($result[0]);
}

####PROGRAM ENDS####
```

AUTHOR

Infoblox Inc. http://www.infoblox.com/

SEE ALSO

Infoblox::Session, Infoblox::Session->add(), Infoblox::Session->get(), Infoblox::Session->modify(),Infoblox::Grid::GlobalSmartFolder,Infoblox::Grid::PersonalSmartFolder

---

COPYRIGHT

Copyright (c) 2017 Infoblox Inc.

Infoblox::Grid::SmartFolderChildren - Smart Folder retrieval item.

- NAME
- DESCRIPTION
- CONSTRUCTOR
- SESSION METHODS
  - Infoblox::Session->search( )
- METHODS
  - is_matching_object( )
  - value( )
  - value_type( )
  - comment( )
  - extattrs( )
  - extensible_attributes( )
- SAMPLE CODE
- AUTHOR
- SEE ALSO
- COPYRIGHT

NAME

Infoblox::Grid::SmartFolderChildren - Smart Folder retrieval item.

DESCRIPTION

This module helps read the objects that are associated with either a smart folder (global or personal) or a set of queries that users can make without saving a smart folder object on the appliance.

This interface can be used for both "filtering" and "grouping" the results of smart folder associated objects.

CONSTRUCTOR

The object cannot be constructed. It can only be returned as part of a search.

SESSION METHODS

This section describes all the methods in the Infoblox::Session module that you can apply to a personal smart folder object.

Infoblox::Session->search( )

Use this method to search for SmartFolderChildren objects in the Infoblox appliance. See Infoblox::Session->search() for parameters and return values.

Key References

```
 Apply the following attributes to search for a specific SmartFolderChildren object:

  query_items                  - Required. A list of Infoblox::Grid::SmartFolder::QueryItem objects.
  smart_folder_name            - Required. PersonalSmartFolder/GlobalSmartFolder name in string
format.
  smart_folder_type            - Optional. "Global" / "Personal", "Global" if not specified.
  smart_folder_group_bys       - Optional. A list of Infoblox::Grid::SmartFolder::GroupBy objects.
  smart_folder_group_by_values - Optional. A list of Infoblox::Grid::SmartFolder::GroupByValue
objects.

 Both "smart_folder_name" and "query_items" are marked "Required" but they cannot be specified
together. Note the following guidelines:

 - "query_items" cannot be specified when "smart_folder_name" is specified because if
"smart_folder_name" is specified, all the properties of a query can be derived from that smart
folder.

 - "query_items" can be specified with "smart_folder_group_bys". In this case, the results are
grouped by specified group_bys attribute and a list of intermediate objects are returned.

 - When only "smart_folder_name" is specified, it runs like a query using all properties from this
folder and returning the results accordingly.

 - When "smart_folder_group_bys" is specified with "smart_folder_name", the query is used from the
specified smart folder; however, the results are shown from the level listed in
"smart_folder_group_bys".

 - "smart_folder_group_bys" by itself cannot be an argument since it does not provide sufficient
input.
```

```
 - "smart_folder_group_by_values" by itself cannot be an argument since it does not provide
sufficient input.

 - For the non-saved smart folder query, the "smart_folder_group_by_values" can be used to specify
the level of results to return. "smart_folder_group_bys" must also be specified in this case.
```

Example

```
  Please see the 'SAMPLE CODE' section below.
```

METHODS

This section describes all the methods that you can use to retrieve the attribute values of a SmartFolderChildren object.

is_matching_object( )

Use this method to retrieve the is_matching_object flag.

Parameter

The flag will be "true" if a matching object is returned inside the value(); if not, the attribute `value()` will contain the name of the intermediate object.

value( )

Use this method to retrieve the value of the object.

Parameter

This attribute will contain either an actual object matching the query (if is_matching_object is "true") or a string describing the name of the intermediate object.

Intermediate objects are the extensible attribute value objects for which the grouping was requested. Matching objects are the DB objects that matched the query specified.

value_type( )

Use this method to retrieve the type of the returned value.

Parameter

If is_matching_object is "false", this will contain one of: "STRING" / "INTEGER" / "DATE" / "BOOLEAN".

comment( )

Use this method to retrieve the comment.

Parameter

This attribute will return the value of the `comment()` attribute of the associated object, if `is_matching_object()` is "true". It will return undef if is_matching_object is "false".

extattrs( )

Use this method to retrieve the extensible attributes. This attribute is read-only and cannot be set.

Parameter

This attribute will return the value of the `extattrs()` attribute of the associated object (if `is_matching_object()` is "true"). It will return undef if is_matching_object is "true"). It will return undef if is_matching_object is "false".

extensible_attributes( )

Use this method to retrieve the extensible attributes.

Parameter

This attribute will return the value of the `extensible_attributes()` attribute of the associated object (if `is_matching_object()` is "true"). It will return undef if is_matching_object is "false".

SAMPLE CODE

```
USE-CASES & EXAMPLES:

#########################################################################
```

```
# Case 1 : Fetching objects based on the query defined for an existing folder
##############################################################################

my @retrieved_objs = $session->search(
            object =>"Infoblox::Grid::SmartFolderChildren" ,
            smart_folder_name => "global_smart_folder_1"
          );

my $object = $retrieved_objs[0];

if ($object->is_matching_object())
{
   print $object->comment();
   # $object->extensible_attributes() will contain the object's
   # extensible attributes
   #
   # This will be the actual matching object
   # $object->value();
}
else
{
   # Name of the intermediate group, use this to fetch another level if needed
   # This will be "USA", "Canada" etc
   # if the grouping was on "Country" for example
   print $object->value();
   print $object->value_type();
}

##############################################################################
# Case 2 : Fetching objects based on the smart folder and a given group_by values
##############################################################################

my $groupbyvalue1 = Infoblox::Grid::SmartFolder::GroupByValue->new(
          name => "Country",
          value => "USA",
);
my $groupbyvalue2 = Infoblox::Grid::SmartFolder::GroupByValue->new(
          name => "State",
          value => "CA",
);

my  @retrieved_objs = $session->search(
      object =>"Infoblox::Grid::SmartFolderChildren" ,
      smart_folder_name => "global_smart_folder_1",
      smart_folder_group_by_values => [$groupbyvalue1, $groupbyvalue2],
         );

my $object = $retrieved_objs[0];
if ($object->is_matching_object())
{
   print $object->comment();
   # $object->extensible_attributes() will contain the object's
   # extensible attributes
   #
   # This will be the actual matching object
   # $object->value();
}
else
{
   # Name of the intermediate group, use this to fetch another level if needed
   print $object->value();
   print $object->value_type();
}

##############################################################################
# Case 3 : Fetching objects based on the not-saved smart folder query, no grouping
##############################################################################

my  @retrieved_objs = $session->search(
      object =>"Infoblox::Grid::SmartFolderChildren" ,
      #This query will only return intermediate objects
      query_items => [queryitem1, queryitem2],
         );

my $object = $retrieved_objs[0];
if ($object->is_matching_object())
{
   print $object->comment();
   # $object->extensible_attributes() will contain the object's
   # extensible attributes
   #
   # This will be the actual matching object
```

```
    # $object->value();

}
else
{
    # This shall never happen
}

#########################################################################
# Case 4 : Fetching objects based on the not-saved smart folder query & with top level grouping
#########################################################################

my  @retrieved_objs = $session->search(
      object =>"Infoblox::Grid::SmartFolderChildren" ,
      query_items => [queryitem1, queryitem2],
      smart_folder_group_bys => [$groupby1, $groupby2],
         );

my $object = $retrieved_objs[0];
if ($object->is_matching_object())
{
    # This shall never happen
}
else
{
    # Name of the intermediate group, use this to fetch another level if needed
    print $object->value();
    print $object->value_type();
}

#########################################################################
# Case 5 : Fetching objects based on the not-saved smart folder query & with second level grouping
#########################################################################

my  @retrieved_objs = $session->search(
      object =>"Infoblox::Grid::SmartFolderChildren" ,
      query_items => [$queryitem1, $queryitem2],
      smart_folder_group_bys => [$groupby1, $groupby2],
      smart_folder_group_by_values => [$groupbyvalue1],
         );

my $object = $retrieved_objs[0];
if ($object->is_matching_object())
{
    # This shall never happen
}
else
{
    # Name of the intermediate group, use this to fetch another level if needed
    # The intermediate objects grouped by $groupby2 are returned in this case
    print $object->value();
    print $object->value_type();
}

#########################################################################
# Case 6 : Fetching objects based on the defined SF query & grouping by a given rule "value"
#########################################################################

my $groupbyvalue1 = Infoblox::Grid::SmartFolder::GroupByValue->new(
          name => "Country",
          value => "USA",
);

my  @retrieved_objs = $session->search(
      object =>"Infoblox::Grid::SmartFolderChildren" ,
      smart_folder_name => "global_smart_folder_1"
      smart_folder_group_by_values => [$groupbyvalue1],
   );

my $object = $retrieved_objs[0];

if ($object->is_matching_object())
{
    print $object->comment();
    # $object->extensible_attributes() will contain the object's
    # extensible attributes
    #
    # This will be the actual matching object
    # $object->value();
}
else
```

```
{
   # Name of the intermediate group, use this to fetch another level if needed
   print $object->value();
   print $object->value_type();
}

##########################################################################
# Case 7 : Fetching next levels of data from result of previous one (NESTED example)
##########################################################################

# EXAMPLE: Get objects for Country USA-> State CA-> DB Objects.

#Add Personal Smart Folder with 2 levels of grouping
#Use this smart folder to fetch intermediate objects from first level (groups)
#Use the result of this first level and fetch second level of intermediate objects (groups)
#Use the result of this second level and fetch the final / third level of DB protocol objects

my $groupby1 = Infoblox::Grid::SmartFolder::GroupBy->new(
            enable_grouping => True,
            value => "Country",
            value_type => "EXTATTR",
);
my $groupby2 = Infoblox::Grid::SmartFolder::GroupBy->new(
            enable_grouping => True,
            value => "State",
            value_type => "EXTATTR",
);

my  $smartfolder = Infoblox::Grid::PersonalSmartFolder->new(
             name => "personal_smart_folder_1",
             comment => "personal smart folder 1 comment",
             enable_grouping => True,
             group_bys => [$groupby1, $groupby2],
             query_items => [],
              );
my $response = $session->add($smartfolder)

my  @retrieved_objs = $session->search(
      object =>"Infoblox::Grid::SmartFolderChildren" ,
      smart_folder_name => "personal_smart_folder_1",
      smart_folder_type => "Personal",
   );

my $object = $retrieved_objs[0];

if ($object->is_matching_object())
{
        # This will not happen since intermediate objects are returned in this case
}
else
{
   # Name of the intermediate group, use this to fetch another level if needed
   my $intermediate_group_value = $object->value();
        if ($intermediate_group_value eq "USA")
        {
            my $groupbyvalue_usa = Infoblox::Grid::SmartFolder::GroupByValue->new(
                name => "Country",
                value => "USA",
            );
            my  @retrieved_objs = $session->search(
                object =>"Infoblox::Grid::SmartFolderChildren" ,
                smart_folder_name => "personal_smart_folder_1"
                smart_folder_group_by_values => [$groupbyvalue_usa],
            );

            my $object = $retrieved_objs[0];

            my $intermediate2_group_value = $object->value();
            if ($intermediate2_group_value eq "CA")
            {
                  my $groupbyvalue_ca = Infoblox::Grid::SmartFolder::GroupByValue->new(
                      name => "State",
                      value => "CA",
                  );
                  my  @retrieved_objs = $session->search(
                      object =>"Infoblox::Grid::SmartFolderChildren" ,
                      smart_folder_name => "personal_smart_folder_1"
                      smart_folder_type => "Personal",
```

```
        smart_folder_group_by_values => [$groupbyvalue_ca],
);

my $object = $retrieved_objs[0];

 if ($object->comment() eq "I am a network object")
 {
     # $object here represents the Database object
 }
            }
  }

####PROGRAM ENDS####
```

AUTHOR

Infoblox Inc. http://www.infoblox.com/

SEE ALSO

Infoblox::Grid::PersonalSmartFolder, Infoblox::Session->add(), Infoblox::Session->fetch_smart_folder_objects(), Infoblox::Session->get(), Infoblox::Session->modify(), Infoblox::Session->remove(), Infoblox::Session->search(),Infoblox::Session

---

COPYRIGHT

Copyright (c) 2017 Infoblox Inc.

Infoblox::Grid::SNMP::Admin - Manages a SNMP Admin object.

- NAME
- DESCRIPTION
- CONSTRUCTOR
- MODULE METHODS
  - Infoblox::Grid->snmp_admin( )
  - Infoblox::Grid::Member->snmp_admin( )
- METHODS
  - sysContact( )
  - sysContact_node1( )
  - sysContact_node2( )
  - sysDescr( )
  - sysDescr_node1( )
  - sysDescr_node2( )
  - sysLocation( )
  - sysLocation_node1( )
  - sysLocation_node2( )
  - sysName( )
  - sysName_node1( )
  - sysName_node2( )
- SAMPLE CODE
- AUTHOR
- SEE ALSO
- COPYRIGHT

NAME

Infoblox::Grid::SNMP::Admin - Manages a SNMP Admin object.

DESCRIPTION

SNMP (Simple Network Management Protocol) is a protocol that administrators use to manage network devices and monitor their processes. An SNMP-managed appliance, such as an Infoblox appliance, has an SNMP agent that collects data and stores them as objects in MIBs (Management Information Bases). You can configure an Infoblox appliance as an SNMP-managed appliance. Infoblox appliances support SNMP versions 1, 2, and 3. You can enter values for sysContact, sysLocation, sysName, sysDescr managed objects in MIB-II, the standard MIB defined in RFC 1213. Once these values are set on the appliance, the administrators can send queries for these values from management systems that are allowed to send queries to the appliance.

CONSTRUCTOR

For SNMP Admin objects used as part of an HA pair

```
my $snmp = Infoblox::Grid::SNMP::Admin->new(
    sysContact_node1  => $string,      #Optional / Default is empty
    sysDescr_node1    => $string,      #Optional / Default is empty
    sysLocation_node1 => $string,  #Optional / Default is empty
    sysName_node1     => $string,  #Optional / Default is hostname of appliance
    sysContact_node2  => $string,      #Optional / Default is empty
    sysDescr_node2    => $string,      #Optional / Default is empty
    sysLocation_node2 => $string,  #Optional / Default is empty
    sysName_node2     => $string,  #Optional / Default is hostname of appliance
 );
```

otherwise

```
my $snmp = Infoblox::Grid::SNMP::Admin->new(
    sysContact  => $string,  #Optional / Default is empty
    sysDescr    => $string,  #Optional / Default is empty
    sysLocation => $string,  #Optional / Default is empty
    sysName     => $string,  #Optional / Default is hostname of appliance
 );
```

When the SNMP Admin object is not used as part of an HA pair, sysContact/sysContact_node1, sysDescr/sysDescr_node1, ... are equivalent. The _node2 fields will be empty.

MODULE METHODS

The following functions are available to apply to an SNMP admin object.

Infoblox::Grid->snmp_admin( )

Use this function to specify an SNMP admin at the grid level. See Infoblox::Grid->snmp_admin() for parameters and return values.

Example

```
#Construct an object
 my $snmp = Infoblox::Grid::SNMP::Admin->new(
   sysContact  => "Somebody",
   sysDescr    => "SunOS noc1 5.6 Generic_105181-03 sun4u",
   sysLocation => "Infoblox Headquarter",
   sysName     => "noc1"
);

#Configure snmp at grid level
my $response = $grid->snmp_admin($snmp);
```

Infoblox::Grid::Member->snmp_admin( )

Use this function to specify an SNMP admin at the member level. See Infoblox::Grid::Member->snmp_admin() for parameters and return values.

Example

```
#Construct an object
  my $snmp = Infoblox::Grid::SNMP::Admin->new(
    sysContact_node1  => "Somebody",
    sysDescr_node1    => "SunOS noc1 5.6 Generic_105181-03 sun4u",
    sysLocation_node1 => "Infoblox Headquarter",
    sysName_node1     => "noc1"
);

# Configure snmp at member level
$member->snmp_admin($snmp);
```

METHODS

This section describes all the methods that you can use to configure and retrieve the attribute values of an SNMP admin object.

sysContact( )

Use this method to set or retrieve the name of the contact person for the Simple Network Management Protocol (SNMP) agent host.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

The name of the contact person in string format. The maximum length of the name is 50 characters.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get sysContact
my $sysContact = $snmp->sysContact();
#Modify sysContact
$snmp->sysContact("Infoblox");
```

sysContact_node1( )

Use this method to set or retrieve the name of the contact person for the Simple Network Management Protocol (SNMP) agent host on node 1 of an HA pair.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

The name of the contact person in string format. The maximum length of the name is 50 characters.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get sysContact_node1
my $sysContact_node1 = $snmp->sysContact_node1();
#Modify sysContact_node1
$snmp->sysContact_node1("Infoblox");
```

sysContact_node2( )

Use this method to set or retrieve the name of the contact person for the Simple Network Management Protocol (SNMP) agent host on node 2 of an HA pair.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

The name of the contact person in string format. The maximum length of the name is 50 characters.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get sysContact_node2
my $sysContact_node2 = $snmp->sysContact_node2();
#Modify sysContact_node2
$snmp->sysContact_node2("Infoblox");
```

sysDescr( )

Use this method to set or retrieve the name and version ID of the system's hardware type, the software Operational System (OS), and networking software that the appliance is running.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

Useful information about the hardware and software described in string format. The maximum length of the sysDescr is 50 characters.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get sysDescr
my $sysDescr = $snmp->sysDescr();
#Modify sysDescr
$snmp->sysDescr("SunOS noc2 5.6 Generic_105181-03 sun5u");
```

sysDescr_node1( )

Use this method to set or retrieve the name and version ID of the system's hardware type, the software Operational System (OS), and networking software that the appliance is running on node 1 of an HA pair.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

Useful information about the hardware and software described in string format. The maximum length of the sysDescr_node1 is 50 characters.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get sysDescr_node1
my $sysDescr_node1 = $snmp->sysDescr_node1();
#Modify sysDescr_node1
$snmp->sysDescr_node1("SunOS noc2 5.6 Generic_105181-03 sun5u");
```

sysDescr_node2( )

Use this method to set or retrieve the name and version ID of the system's hardware type, the software Operational System (OS), and networking software that the appliance is running on node 2 of an HA pair.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

Useful information about the hardware and software described in string format. The maximum length of the sysDescr_node2 is 50 characters.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get sysDescr_node2
my $sysDescr_node2 = $snmp->sysDescr_node2();
#Modify sysDescr_node2
$snmp->sysDescr_node2("SunOS noc2 5.6 Generic_105181-03 sun5u");
```

sysLocation( )

Use this method to set or retrieve the physical location of the appliance.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

The physical location of the appliance in string format. The maximum length of the sysLocation is 50 characters.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails. If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get sysLocation
my $sysLocation = $snmp->sysLocation();
#Modify sysLocation
$snmp->sysLocation("Infoblox Headquarter5");
```

sysLocation_node1( )

Use this method to set or retrieve the physical location of the appliance on node 1 of an HA pair.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

The physical location of the appliance in string format. The maximum length of the sysLocation_node1 is 50 characters.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails. If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get sysLocation_node1
my $sysLocation_node1 = $snmp->sysLocation_node1();
#Modify sysLocation_node1
$snmp->sysLocation_node1("Infoblox Headquarter5");
```

sysLocation_node2( )

Use this method to set or retrieve the physical location of the appliance on node 2 of an HA pair.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

The physical location of the appliance in string format. The maximum length of the sysLocation_node2 is 50 characters.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails. If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get sysLocation_node2
my $sysLocation_node2 = $snmp->sysLocation_node2();
#Modify sysLocation_node2
$snmp->sysLocation_node2("Infoblox Headquarter5");
```

sysName( )

Use this method to set or retrieve the fully qualified domain name (FQDN) of the appliance.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

The fully qualified domain name (FQDN) of the appliance in string format. The maximum length of the sysName is 50 characters. The default value of sysName is the hostname of the Infoblox appliance.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get sysName
my $sysName = $snmp->sysName();
#Modify sysName
$snmp->sysName("Infobloxnoc2");
```

sysName_node1( )

Use this method to set or retrieve the fully qualified domain name (FQDN) of the appliance on node 1 of an HA pair.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

The fully qualified domain name (FQDN) of the appliance in string format. The maximum length of the sysName_node1 is 50 characters. The default value of sysName_node1 is the hostname of the Infoblox appliance.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get sysName_node1
my $sysName_node1 = $snmp->sysName_node1();
#Modify sysName_node1
$snmp->sysName_node1("Infobloxnoc2");
```

sysName_node2( )

Use this method to set or retrieve the fully qualified domain name (FQDN) of the appliance on node 2 of an HA pair.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

The fully qualified domain name (FQDN) of the appliance in string format. The maximum length of the sysName_node2 is 50 characters. The default value of sysName_node2 is the hostname of the Infoblox appliance.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get sysName_node2
my $sysName_node2 = $snmp->sysName_node2();
#Modify sysName_node2
$snmp->sysName_node2("Infobloxnoc2");
```

SAMPLE CODE

The following sample code demonstrates the different functions that can be applied to an object such as add, modify and remove. This sample also includes error handling for the operations.

#Preparation prior to an SNMP admin object insertion

```
#PROGRAM STARTS: Include all the modules that will be used
use strict;
use Infoblox;

#Create a session to the Infoblox appliance
my $session = Infoblox::Session->new(
    master   => "192.168.1.2",
    username => "admin",
    password => "infoblox"
);
unless ($session) {
   die("Construct session failed: ",
       Infoblox::status_code() . ":" . Infoblox::status_detail());
}
print "Session created successfully\n";
```

#Creating an SNMP admin object

```
my $snmp = Infoblox::Grid::SNMP::Admin->new(
    sysContact  => "Somebody",
    sysDescr    => "SunOS noc1 5.6 Generic_105181-03 sun4u",
    sysLocation => "Infoblox Headquarter",
    sysName     => "noc1"
    );
unless($snmp) {
     die("Construct a SNMP Admin failed: ",
           Infoblox::status_code() . ":" . Infoblox::status_detail());
}
print "SNMP Admin object created successfully\n";
```

#Getting the grid object from the appliance through session

```
 my @result = $session->get(
    object => "Infoblox::Grid",
    name   => "Infoblox"
    );

my $grid = $result[0];
unless ($grid) {
       die("Get Grid object failed: ",
            $session->status_code() . ":" . $session->status_detail());
}
print "Get Grid object found at least 1 matching entry\n";

      #Modifying the value of the specified object.
        $grid->snmp_admin($snmp);  #Applying the changes.
        $session->modify($grid);
        print "SNMP Admin modified successfully\n";
```

#Modify SNMP admin attribute sysName

```
 #Modify existing attribute sysName
$snmp->sysName("noc2.infoblox.com");

 #Apply changes to the Grid object.
$grid->snmp_admin($snmp);

 #Update Grid object through the Infoblox session.
$session->modify($grid)
```

```
            or die("modify Grid object failed: ",
                $session->status_code() . ":" . $session->status_detail());
        print "Modified SNMP Admin sysName of Grid object successful\n";
```

#Remove SNMP admin attribute from Grid object

```
  #Apply changes to the Grid object to Un-def SNMP Admin object.
  $grid->snmp_admin(undef);

  #Update Grid object through the Infoblox session.
  $session->modify($grid)
            or die("Remove SNMP Admin from Grid object failed: ",
                $session->status_code() . ":" . $session->status_detail());
        print "Removed SNMP Admin from Grid object successful\n";

 ####PROGRAM ENDS####
```

AUTHOR

Infoblox Inc. http://www.infoblox.com/

SEE ALSO

Infoblox::Session,Infoblox::Session->get(), Infoblox::Session->modify(), Infoblox::Grid

---

COPYRIGHT

Copyright (c) 2017 Infoblox Inc.

Infoblox::Grid::SNMP::IPAMThreshold - IPAM Utilization SNMP Threshold object.

- NAME
- DESCRIPTION
- CONSTRUCTOR
- METHODS
  - reset_value( )
  - trigger_value( )
- AUTHOR
- COPYRIGHT

NAME

Infoblox::Grid::SNMP::IPAMThreshold - IPAM Utilization SNMP Threshold object.

DESCRIPTION

The object provides information about the IPAM utilization SNMP threshold.

CONSTRUCTOR

```
my $obj = Infoblox::Grid::SNMP::IPAMThreshold->new(
    'reset_value'   => $uint,     # optional
    'trigger_value' => $uint,     # optional
);
```

METHODS

This section describes all the methods that can be used to retrieve the attribute values of an object.

reset_value( )

Use this method to set or retrieve a reset threshold value for the trap.

Returns

Valid value is unsigned integer between 0 and 100 inclusive.

Parameter

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get attrubute value
my $value = $obj->reset_value();
#Set attribute value
$obj->reset_value(55);
```

trigger_value( )

Use this method to set or retrieve a trigger threshold value for the trap.

Returns

Valid value is unsigned integer between 0 and 100 inclusive.

Parameter

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get attrubute value
my $value = $obj->trigger_value();
#Set attribute value
$obj->trigger_value(55);
```

AUTHOR

Infoblox Inc. http://www.infoblox.com/

COPYRIGHT

Copyright (c) 2017 Infoblox Inc.

Infoblox::Grid::SNMP::IPAMTrap - IPAM Utilization SNMP Trap object.

- NAME
- DESCRIPTION
- CONSTRUCTOR
- METHODS
  - enable_email_warnings( )
  - enable_snmp_warnings( )
- AUTHOR
- COPYRIGHT

NAME

Infoblox::Grid::SNMP::IPAMTrap - IPAM Utilization SNMP Trap object.

DESCRIPTION

The object provides information about the IPAM utilization SNMP trap.

CONSTRUCTOR

```
my $obj = Infoblox::Grid::SNMP::IPAMTrap->new(
    'enable_email_warnings' => 'true' | 'false',      # optional
    'enable_snmp_warnings'  => 'true' | 'false',      # optional
);
```

METHODS

This section describes all the methods that can be used to retrieve the attribute values of an object.

enable_email_warnings( )

Use this method to enable or disable email notifications.

Returns

Specify 'true' to enable email notifications or 'false' to disable it. The default value is 'true'.

Parameter

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get attrubute value
my $value = $obj->enable_email_warnings();
#Set attribute value
$obj->enable_email_warnings('true');
```

enable_snmp_warnings( )

Use this method to enable or disable SNMP notifications.

Returns

Specify 'true' to enable SNMP notifications or 'false' to disable it. The default value is 'true'.

Parameter

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get attrubute value
my $value = $obj->enable_snmp_warnings();
#Set attribute value
$obj->enable_snmp_warnings('true');
```

AUTHOR

Infoblox Inc. http://www.infoblox.com/

COPYRIGHT

Copyright (c) 2017 Infoblox Inc.

Infoblox::Grid::SNMP::QueriesUser - Grid SNMP QueriesUser object.

- NAME
- DESCRIPTION
- CONSTRUCTOR
- METHODS
    - user()
- AUTHOR
- SEE ALSO
- COPYRIGHT

NAME

Infoblox::Grid::SNMP::QueriesUser - Grid SNMP QueriesUser object.

DESCRIPTION

The Grid SNMP Queriesuser object is used to set or retrieve the user name of an SNMPv3 user.

CONSTRUCTOR

```
my $query_user1 = Infoblox::Grid::SNMP::QueriesUser->new(
          user => $snmp_user1,
          comment => "query comment 1"
);
```

METHODS

This section describes all the methods that you can use to configure and retrieve the attribute values of an SNMPv3 QueriesUser object.

user()

Use this method to set or retrieve the user name of an SNMPv3 user.

Parameter

A reference to the Infoblox::Grid::SNMP::User object.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get query user
my $query_user=$grid->snmpv3_query_users();
#Set query user
$grid->snmpv3_query_users([$query_user1,$query_user2];
```

AUTHOR

Infoblox Inc. http://www.infoblox.com/

SEE ALSO

Infoblox::Grid::SNMP::User

COPYRIGHT

Copyright (c) 2017 Infoblox Inc.

Infoblox::Grid::SNMP::ThresholdTrap - Threshold Trap object.

- NAME
- DESCRIPTION
- CONSTRUCTOR
- MODULE METHODS
  - Infoblox::Grid->threshold_traps( )
  - Infoblox::Grid::Member->threshold_traps( )
- METHODS
  - trap_reset( )
  - trap_trigger( )
  - trap_type( )
- SAMPLE CODE
- AUTHOR
- SEE ALSO
- COPYRIGHT

NAME

Infoblox::Grid::SNMP::ThresholdTrap - Threshold Trap object.

DESCRIPTION

The Grid SNMP ThresholdTrap object is used to set the values of the thresholds of the SNMP traps.

CONSTRUCTOR

Use the following attributes to construct an Infoblox::Grid::SNMP::ThresholdTrap object.

```
my $threshold_trap = Infoblox::Grid::SNMP::ThresholdTrap->new(
    trap_type       =>   $string,      #Required
    trap_reset      =>   $uint,        #Optional
    trap_trigger    =>   $uint,        #Optional
);
```

MODULE METHODS

This section describes all the methods in the other modules that can be applied to a Threshold Trap object.

Infoblox::Grid->threshold_traps( )

Use this function to specify threshold traps at the grid level. See Infoblox::Grid->threshold_traps() for parameters and return values.

Example

```
#Get threshold traps
my $threshold_traps = $grid->threshold_traps();
#Modify threshold traps
my $threshold_traps = $grid->threshold_traps();
foreach(@$threshold_traps){if $_->trap_name() eq 'Rootfs'; $_->trap_reset=60;} #Change value for
'Rootfs' trap
$grid->threshold_traps($threshold_traps);
$session->modify($grid);
```

Infoblox::Grid::Member->threshold_traps( )

Use this function to specify threshold traps at the member level. See Infoblox::Grid::Member->threshold_traps() for parameters and return values.

Example

```
#Get threshold traps
my $threshold_traps = $member->threshold_traps();
#Modify threshold traps
my $threshold_traps = $member->threshold_traps();
foreach(@$threshold_traps){if $_->trap_name() eq 'Rootfs'; $_->trap_reset=60;} #Change value for
'Rootfs' trap
$grid->threshold_traps($threshold_traps);
$session->modify($member);
```

METHODS

This section describes all the methods that can be used to retrieve the attribute values of a Threshold Trap object.

trap_reset( )

Use this method to specify the threshold value to reset the trap.

Parameter

The trap_reset parameter is a positive integer. The default value depends on the trap_type.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get the trap reset value
my $trap_reset = $threshold_trap->trap_reset();
#Set the trap reset value
$threshold_trap->trap_reset();
```

trap_trigger( )

Use this method to specify the threshold value to trigger the trap. The default value depends on the trap_type.

Parameter

The trap_trigger parameter is a positive integer. The default value depends on the trap_type.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get the trap trigger
my $trap_trigger = $threshold_trap->trap_trigger();
#Set the trap trigger
$threshold_trap->trap_trigger();
```

trap_type( )

Use this method to get the type of a given trap.

Parameter

String that contains the trap type.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get the trap type
my $trap_type = $threshold_trap->trap_type();
#Set the trap type
$threshold_trap->trap_type();
```

SAMPLE CODE

The following sample code demonstrates the session methods of a Threshold Trap object.

#Preparation prior to getting and modifying a grid member object

```
use strict;
use Infoblox;

my $session = Infoblox::Session->new(
```

```
            master   => "192.168.2.1",
            username => "admin",
            password => "infoblox"
  );

  unless ($session) {
     die("Session construction failed: ",
         Infoblox::status_code() . ":" . Infoblox::status_detail());
  }
  print "Session created successfully\n";
```

#Getting the grid level trap list from appliance through session

```
  #Getting the Grid object
  my @result = $session->get(
     object => "Infoblox::Grid",
     name   => "Infoblox"
     ); my $grid = $result[0];
  unless ($grid) {
        die("Get Grid object failed: ",
            $session->status_code() . ":" . $session->status_detail());
  }
  print "Get Grid object found at least 1 matching entry\n";

  #Extracting threshold traps
  my $ref_threshold_traps = $grid->threshold_traps();
  unless($ref_threshold_traps && ref($ref_threshold_traps) eq 'ARRAY' ){
      die("Get threshold traps list failed: ",
          $session->status_code() . ":" . $session->status_detail());
  }
  my @current_threshold_traps = @$ref_threshold_traps;
```

#Modify threshold traps list

```
  #Delete some traps from the list
  my @new_list = grep {$_->trap_type() ne "NetworkCapacity" && $_->trap_type() ne "Rootfs" }
@current_threshold_traps;
  #Create new "Rootfs" threshold trap
  my $new_rootfs_trap = Infoblox::Grid::SNMP::ThresholdTrap->new(
       trap_type => "Rootfs",
       trap_trigger => 95,
       trap_reset => 80,
   );
  unless($new_rootfs_trap){
      die("Creating new threshold trap failed: ",
          Infoblox::status_code() . ":" . Infoblox::status_detail());
  }
  push @new_list, $new_rootfs_trap;
  #Modify grid object
  unless($grid->threshold_traps(\@new_list)){
      die("Modifying threshold traps failed: ",
          Infoblox::status_code() . ":" . Infoblox::status_detail());
  }
  B<#Update Grid object through the Infoblox session.>
  $session->modify($grid)
             or die("modify Grid object failed: ",
                 $session->status_code() . ":" . $session->status_detail());
        print "Modified threshold traps of Grid object successful\n";
```

#Restore the old threshold traps

```
  #Apply changes to the Grid object to Un-def SNMP Admin object.
  $grid->threshold_traps(\@current_threshold_traps);  #Update Grid object through the Infoblox session.
  $session->modify($grid)
             or die("Restore threshold traps for Grid object failed: ",
                 $session->status_code() . ":" . $session->status_detail());

        print "Restored Grid object successful\n";

        ####PROGRAM ENDS####
```

AUTHOR

Infoblox Inc. http://www.infoblox.com/

SEE ALSO

Infoblox::Session, Infoblox::Session->get()

COPYRIGHT

Copyright (c) 2017 Infoblox Inc.

Infoblox::Grid::SNMP::TrapNotification - Trap Notification object.

- NAME
- DESCRIPTION
- CONSTRUCTOR
- MODULE METHODS
  - Infoblox::Grid->trap_notifications( )
  - Infoblox::Grid::Member->trap_notifications( )
- METHODS
  - enable_email( )
  - enable_trap( )
  - trap_type( )
- AUTHOR
- SEE ALSO
- COPYRIGHT

NAME

Infoblox::Grid::SNMP::TrapNotification - Trap Notification object.

DESCRIPTION

This object represents the trap notification settings of the Infoblox::Grid and Infoblox::Grid::Member object. The object allows you to control which trap categories are enabled, and for which trap categories mail notification will be sent.

CONSTRUCTOR

Use the following parameters to construct the Infoblox::Grid::SNMP::TrapNotification object

```
my $trap_notification = Infoblox::Grid::SNMP::TrapNotification->new(
   trap_type       =>  $string,              #Required
   enable_email    =>  "true" | "false",     #Optional / Default is  "false"
   enable_trap     =>  "true" | "false",     #Optional / Default is  "true"
);
```

MODULE METHODS

The following functions are available to apply to an SNMP Trap Notification object.

Infoblox::Grid->trap_notifications( )

Use this function to specify trap notification settings at the grid level. See Infoblox::Grid->trap_notifications() for parameters and return values.

Example

```
#Get a reference to a trap notifications array
my $trap_notifications = $grid->trap_notifications();
#Modify trap notifications
my $notifications = $grid->trap_notifications();
foreach(@$notifications){if $_->trap_type() eq 'CPU'; $_->enable_email="true";} #Change value for
'CPU' trap
$grid->trap_notifications($notifications);
$session->modify($grid);
```

Infoblox::Grid::Member->trap_notifications( )

Use this function to specify a trap notifications at the member level. See Infoblox::Grid::Member->trap_notifications() for parameters and return values.

Example

```
#Get a reference to a trap notifications array
my $trap_notifications = $member->trap_notifications();
#Modify trap notifications
my $notifications = $member->trap_notifications();
foreach(@$notifications){if $_->trap_type() eq 'CPU'; $_->enable_email="true";} #Change value for
'CPU' trap
$member->trap_notifications($notifications);
$session->modify($member);
```

METHODS

This section describes all the methods that can be used to retrieve the attribute values of a Trap Notification object.

enable_email( )

Use this method to retrieve the enable_email flag of a Trap Notification object.

Parameter

none

Returns

The method returns the enable_email flag value.

Example

```
#Get  enable_email flag
my $enable_email = $trap_notification->enable_email();
#Set  enable_email flag
$trap_notification->enable_email();
```

enable_trap( )

Use this method to retrieve the enable_trap flag of a Trap Notification object.

Parameter

none

Returns

The method returns the enable_trap flag attribute value.

Example

```
#Get  enable_trap flag
my $enable_trap = $trap_notification->enable_trap();
#Set  enable_trap flag
$trap_notification->enable_trap();
```

trap_type( )

Use this method to retrieve the category of a trap in the Trap Notification object.

Parameter

none

Returns

The method returns the category of the trap attribute value.

Example

```
#Get category of trap
my $trap_type = $trap_notification->trap_type();
#Set category of trap
$trap_notification->trap_type();
```

AUTHOR

Infoblox Inc. http://www.infoblox.com/

SEE ALSO

Infoblox::Grid, Infoblox::Grid::Member

COPYRIGHT

Copyright (c) 2017 Infoblox Inc.

Infoblox::Grid::SNMP::TrapReceiver - Grid SNMP TrapReceiver object.

- NAME
- DESCRIPTION
- CONSTRUCTOR
- METHODS
  - address()
  - user()
- AUTHOR
- SEE ALSO
- COPYRIGHT

NAME

Infoblox::Grid::SNMP::TrapReceiver - Grid SNMP TrapReceiver object.

DESCRIPTION

The Grid SNMP TrapReceiver object is used to set or retrieve the IP address and user name of a trap receiver.

CONSTRUCTOR

```
my $Infoblox::Grid::SNMP::TrapReceiver->new(
    address => $ip_address,   #Required
    user => $snmpv3user       #Required
);
```

METHODS

This section describes all the methods that you can use to configure and retrieve the attribute values of an SNMPv3 TrapReceiver object.

address()

Use this method to set or retrieve the IP address of the trap receiver.

Parameter

IPv4 address of the trap receiver

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get trap receiver address
my $ip=$trap_receiver->address();
#Set trap receiver address
$trap_receiver->address("192.168.1.2");
```

user()

Use this method to set or retrieve the user name for the trap receiver.

Parameter

A reference to the Infoblox::Grid::SNMP::User object or the user name in string format.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get trap receiver user
my $user=$trap_receiver->user();
#Set trap receiver user
$trap_receiver->user($user);
```

Infoblox::Grid::SNMP::TrapReceiver - Grid SNMP TrapReceiver object.

AUTHOR

Infoblox Inc. http://www.infoblox.com/

SEE ALSO

Infoblox::Grid::SNMP::User

COPYRIGHT

Copyright (c) 2017 Infoblox Inc.

Infoblox::Grid::SNMP::User - Grid SNMP User object

- NAME
- DESCRIPTION
- CONSTRUCTOR
- SESSION METHODS
  - Infoblox::Session->get( )
  - Infoblox::Session->modify( )
  - Infoblox::Session->search( )
  - Infoblox::Session->remove( )
- METHODS
  - auth_password( )
  - auth_protocol( )
  - comment( )
  - disable( )
  - engine_id( )
  - extattrs( )
  - name( )
  - privacy_password( )
  - privacy_protocol( )
- AUTHOR
- SEE ALSO
- COPYRIGHT

NAME

Infoblox::Grid::SNMP::User - Grid SNMP User object

DESCRIPTION

The Grid SNMP User object is used to set, get, search, modify, or remove an SNMPv3 user. An SNMPv3 user is a managment system that sends queries and receives traps from the appliance.

CONSTRUCTOR

```
#Constructor for a Grid SNMP User object
my $snmp_user = Infoblox::Grid::SNMP::User->new(
        name                => $string,                 #Required
        auth_protocol       => "none"| "md5" | "sha",   #Required
        privacy_protocol    => "none"| "des" | "aes",   #Required
        auth_password       => $string,                 #Optional. Required for MD5/SHA authentication
        comment             => $string,                 #Optional / Default is undefined
        disable             => "true" | "false",        #Optional / Default is "false"
        privacy_password    => $string,                 #Optional. Required for MD5/SHA authentication
        extattrs            => { $string => $extattr, ... },      #Optional / Default is undefined
 );
```

SESSION METHODS

This section describes all the methods in Infoblox::Session that you can apply to a Grid SNMP User object.

Infoblox::Session->get( )

Use this method to retrieve existing objects from the Infoblox appliance. See Infoblox::Session->get() for parameters and return values.

Key Reference

```
Apply the following attributes to get a Grid SNMP User object:

name     -  user name in string format
comment  -  comment string
extattrs - Optional. A hash reference containing extensible attributes.
```

Example

```
my @retrieved_objs = $session->get(
        object      => "Infoblox::Grid::SNMP::User",
        name        => "testuser" );
```

Infoblox::Session->modify( )

Use this method to modify objects in the Infoblox appliance. See Infoblox::Session->modify() for parameters and return values.

Example

```
# Use this method to modify the comment
$snmp_user->comment("This is a modified comment");
# Submit modification
my $response = $session->modify( $snmp_user );
```

Infoblox::Session->search( )

Use this method to search for Grid SNMP User objects in the Infoblox appliance. See Infoblox::Session->search() for parameters and return values.

Key References

```
Apply the following attributes to search for a Grid SNMP User object:

 name -  user name (regular expression)
 comment - comment string (regular expression)
 extattrs - Optional. A hash reference containing extensible attributes.
```

Example

```
# Search for all Grid SNMP User objects that start with "test"
my @retrieved_objs = $session->search(
    object      => "Infoblox::Grid::SNMP::User",
    name        => "test.*",
);
```

Infoblox::Session->remove( )

Use this method to remove an object from the Infoblox appliance. See Infoblox::Session->remove() for parameters and return values.

To remove a specific object, first use `get()` or `search()` to retrieve the object, and then submit it for removal.

Example

```
# Get Grid SNMP User objects with the same starting name
my @retrieved_objs = $session->get(
    object      => "Infoblox::Grid::SNMP::User",
    name        => "testuser" );
# find the desired object from the retrieved list.
my $desired_user = $retrieved_objs[0];
# Submit for removal
my $response = $session->remove( $desired_user );
```

METHODS

This section describes all the methods that you can use to configure and retrieve the attribute values of an SNMPv3 user.

auth_password( )

Use this method to provide an authentication password for the user. This is a write-only attribute.

Parameter

Authentication password of the user in string format.

Returns

If a parameter is specified, the method returns true when the modification succeeds, and returns false when the operation fails.

Example

```
#Set the password
$snmp_user->auth_password("mysecret");
```

auth_protocol( )

Use this method to specify the authentication protocol of the user.

Parameter

Possible values for the authentication protocol are:

```
none
```

```
md5
sha
```

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get auth_protocol
my $proto = $snmp_user->auth_protocol();
#Set the protocol
$snmp_user->auth_protocol("md5");
```

comment( )

Use this method to provide a comment for the SNMPv3 User object.

Parameter

Comment in string format with a maximum of 256 bytes.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get comment
my $comment = $snmp_user->comment();
#Set the comment
$snmp_user->comment("This is a sample comment");
```

disable( )

Use this method to configure or retrieve the "disable" flag.

The default value is false; that is, the user is enabled.

Parameter

Specify "true" to set the disable flag or "false" to deactivate/unset it. The default value is "false".

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get disable flag
my $disabled = $snmp_user->disable();
#Set the disable flag
$snmp_user->disable("true");
```

engine_id( )

Use this method to retrieve the SNMP engine ID.

This field is read-only.

Parameter

None

Returns

When called without parameters, the method returns the attribute value. Calling this method with a parameter results in an error.

Example

```
#Get engine_id
my $engine_id = $snmp_user->engine_id();
```

extattrs( )

Use this method to set or retrieve the extensible attributes associated with a SNMP user object.

Parameter

Valid value is a hash reference containing the names of extensible attributes and their associated values (Infoblox::Grid::Extattr objects).

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get extattrs
my $ref_extattrs = $snmp_user->extattrs();
#Modify extattrs
$snmp_user->extattrs({ 'Site' => $extattr1, 'Administrator' => $extattr2 });
```

name( )

Use this method to set or retrieve the name of an SNMPv3 user.

Parameter

Text with the name of the user.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get name
my $name = $snmp_user->name();
#Modify name
$snmp_user->name("test_user");
```

privacy_password( )

Use this method to provide a password for the privacy protocol. This is a write-only attribute.

Parameter

Password in string format.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

Example

```
#Set the password
$snmp_user->privacy_password("mysecret");
```

privacy_protocol( )

Use this method to specify encryption protocol for the SNMP packets.

Parameter

Possible values for the encryption protocol are:

```
none
des
aes
```

The values are case insensitive.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get privacy_protocol
my $proto = $snmp_user->privacy_protocol();
#Set the protocol
$snmp_user->privacy_protocol("aes");
```

AUTHOR

Infoblox Inc. http://www.infoblox.com/

SEE ALSO

Infoblox::Grid::SNMP::TrapReceiver,Infoblox::Session, Infoblox::Session->get(), Infoblox::Session->modify()

COPYRIGHT

Copyright (c) 2017 Infoblox Inc.

Infoblox::Grid::SyslogBackupServer - Syslog Backup Server object.

- NAME
- DESCRIPTION
- CONSTRUCTOR
- MODULE METHODS
  - Infoblox::Grid->syslog_backup_servers( )
  - Infoblox::Grid::Member->syslog_backup_servers( )
- METHODS
  - address( )
  - directory_path( )
  - enable( )
  - password( )
  - port( )
  - protocol( )
  - username( )
- AUTHOR
- SEE ALSO
- COPYRIGHT

NAME

Infoblox::Grid::SyslogBackupServer - Syslog Backup Server object.

DESCRIPTION

The Syslog Backup Server object contains configuration for external servers that are used for replication of rotated log files through SCP or FTP.

CONSTRUCTOR

```
my $syslog_backup_server = Infoblox::Grid::SyslogBackupServer->new(
    address        => $ipv4addr | $ipv6addr, # Optional
    directory_path => $string,               # Optional
    enable         => 'true' | 'false',      # Optional
    password       => $string,               # Optional
    port           => $uint,                 # Optional
    protocol       => 'SCP' | 'FTP',         # Optional
    username       => $string,               # Optional
);
```

MODULE METHODS

The following functions are available to be applied to a syslog backup server object.

Infoblox::Grid->syslog_backup_servers( )

Use this method to set or retrieve an array that contains Grid-level syslog backup server settings. See Infoblox::Grid->syslog_backup_servers() for parameters and return values.

Example

```
#get syslog_backup_servers value
my $servers = $grid->syslog_backup_servers();

#modify syslog_backup_servers value
$grid->syslog_backup_servers([$server1, $server2]);
```

Infoblox::Grid::Member->syslog_backup_servers( )

Use this method to set or retrieve an array that contains syslog backup server settings for a Grid member. See Infoblox::Grid::Member->syslog_backup_servers() for parameters and return values.

Example

```
#get syslog_backup_servers value
my $servers = $grid_member->syslog_backup_servers();

#modify syslog_backup_servers value
$grid_member->syslog_backup_servers([$server1, $server2]);
```

METHODS

address( )

Use this method to set or retrieve the IPv4 or IPv6 address of a syslog backup server.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

An IPv4 address is a 32-bit number in dotted decimal notation. It consists of four 8-bit groups of decimal digits separated by decimal points (example: 192.168.1.2).

An IPv6 address is a 128-bit number represented as eight groups of four hexadecimal digits (i.e. two octets), with each group separated by a `colon(:)` (example: 2001:0db8:85a3:0000:0000:8a2e:0370:7334). Leading zeroes and groups of consecutive zeros within the value can be omitted to produce a more compact representation (example: 2001::1, ::1).

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#get address value
my $address = $server->address();

#modify address value
$server->address('192.168.1.5');
```

directory_path( )

Use this method to set or retrieve the path for rotated syslog file replication.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

The valid value is a desired path in a string format.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#get directory_path value
my $directory_path = $server->directory_path();

#modify directory_path value
$server->directory_path('/path/to/directory');
```

enable( )

Use this method to enable or disable the syslog backup server.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

Specify 'true' to enable syslog backup server or 'false' to disable it. The default value is 'true'.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#get enable value
my $enable = $server->enable();

#modify enable value
```

```
$server->enable('true');
```

password( )

Use this method to set the syslog backup server password. This is a write-only attribute.

Include the specified parameter to set the attribute value.

Parameter

The valid value is a desired password in a string format.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

Example

```
#modify password value
$server->password('infoblox');
```

port( )

Use this method to set or retrieve the transport layer port for connection to the syslog backup server.

Include the specified parameter to set the attribute value.

Parameter

The valid value is an unsigned integer between 0 and 65535.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#get port value
my $port = $server->port();

#modify port value
$server->port(22);
```

protocol( )

Use this method to set or retireve the transport layer protocol for the syslog backup server connection.

Include the specified parameter to set the attribute value.

Parameter

The valid values are 'FTP' and 'SCP'.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#get protocol value
my $protocol = $server->protocol();

#modify protocol value
$server->protocol('SCP');
```

username( )

Use this method to set or retrieve the username for the syslog backup server.

Include the specified parameter to set the attribute value.

Parameter

Valid value is a desired username in string format.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#get username value
my $username = $server->username();

#modify username value
$server->username('admin');
```

AUTHOR

Infoblox Inc. http://www.infoblox.com/

SEE ALSO

Infoblox::Grid, Infoblox::Grid::Member

COPYRIGHT

Copyright (c) 2017 Infoblox Inc.

Infoblox::Grid::SyslogServer - Syslog server object.

- NAME
- DESCRIPTION
- CONSTRUCTOR
- MODULE METHODS
  - Infoblox::Grid->syslog_server( )
  - Infoblox::Grid::Member->syslog_server( )
- METHODS
  - address( )
  - category_list( )
  - certificate( )
  - connection_type( )
  - local_interface( )
  - message_source( )
  - only_category_list( )
  - port( )
  - severity( )
- SAMPLE CODE
- AUTHOR
- SEE ALSO
- COPYRIGHT

NAME

Infoblox::Grid::SyslogServer - Syslog server object.

DESCRIPTION

Syslog is a widely used mechanism for logging system events. Infoblox appliances generate syslog messages which can be viewed through the system log viewer and downloaded to a directory on your management station. In addition, the Infoblox appliance can be configured to send the messages to one or more external syslog servers for later analysis. Syslog messages provide information about appliance operations and processes. Also, the appliance can include audit log messages and specific BIND messages among the messages sent to the syslog server.

CONSTRUCTOR

```
 my $syslog_server = Infoblox::Grid::SyslogServer->new(
        address             => $address,                          #Required
        connection_type     => "tcp" | "udp" | "stcp",            #Optional / Default is "udp"
        local_interface     => "any" | "mgmt" | "lan",            #Optional / Default is "any"
        message_source      => "any" | "internal" | "external",   #Optional / Default is "any"
        port                => $num,                              #Optional / Default is 514 for TCP/UDP,
6514 for STCP
        severity            => "debug" | "info" | "notice" | "warning" | "err" | "crit" | "alert" | "emerg",
#Optional / Default is "debug"
        certificate         => $fname,  #Optional / Default is undefined

 );
```

MODULE METHODS

The following functions are available to be applied to a syslog server object.

Infoblox::Grid->syslog_server( )

Use this function to specify a syslog server at the grid level on the Infoblox appliance. See Infoblox::Grid->syslog_server() for parameters and return values.

Example

```
       #Construct a syslog server object
       my $syslog_server1 = Infoblox::Grid::SyslogServer->new(
                               address => "1.1.1.1",
                             );

       #Configure syslog server on the Infoblox grid object
       my $responce = $grid->syslog_server([$syslog_server1]);
```

Infoblox::Grid::Member->syslog_server( )

Use this function to specify a syslog server at the member level on the Infoblox appliance. See Infoblox::Grid::Member->syslog_server() for parameters and return values.

Example

```
#Construct a syslog server object
my $syslog_server2 = Infoblox::Grid::SyslogServer->new(
                       address => "2.2.2.2",
                       severity => "debug",
                     );

#Configure syslog server on the grid member object
my $responce = $grid_member->syslog_server([$syslog_server2]);
```

METHODS

This section describes all the methods that can be used to configure and retrieve the attribute value of a syslog server object.

address( )

Use this method to set or retrieve the IPv4 or IPv6 address of a syslog server.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

An IPv4 address is a 32-bit number in dotted decimal notation. It consists of four 8-bit groups of decimal digits separated by decimal points (example: 192.168.1.2).

An IPv6 address is a 128-bit number represented as eight groups of four hexadecimal digits (i.e. two octets), with each group separated by a `colon(:)` (example: 2001:0db8:85a3:0000:0000:8a2e:0370:7334). Leading zeroes and groups of consecutive zeros within the value can be omitted to produce a more compact representation (example: 2001::1, ::1).

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#get address
my $address = $syslog_server->address();
#Modify address
$syslog_server->address("3.3.3.3");
```

category_list( )

Use this method to set or retrieve the list of syslog categories for forwarding.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

The valid value is a list containing these values: 'NON_CATEGORIZED', 'ATP', 'DNS_CLIENT', 'DNS_CONFIG', 'DNS_DATABASE', 'DNS_DNSSEC', 'DNS_GENERAL', 'DNS_LAME_SERVERS', 'DNS_NETWORK', 'DNS_NOTIFY', 'DNS_QUERIES', 'DNS_QUERY_REWRITE', 'DNS_RESOLVER', 'DNS_RESPONSES', 'DNS_RPZ', 'DNS_SECURITY', 'DNS_UPDATE', 'DNS_UPDATE_SECURITY', 'DNS_XFER_IN', 'DNS_XFER_OUT', 'DNS_UNBOUND', 'DTC_HEALTHD', 'DTC_IDNSD', 'DHCPD', 'NTP', 'FTPD', 'TFTPD', 'CLOUD_API', 'MS_DNS_SERVER', 'MS_CONNECT_STATUS', 'MS_DNS_ZONE', 'MS_DHCP_SERVER', 'MS_DHCP_LEASE', 'MS_DHCP_CLEAR_LEASE', 'MS_SITES', 'MS_AD_USERS', 'AUTH_COMMON', 'AUTH_NON_SYSTEM', 'AUTH_UI_API', 'AUTH_ACTIVE_DIRECTORY', 'AUTH_RADIUS', 'AUTH_TACACS', 'AUTH_LDAP', 'OUTBOUND_API'.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get category_list
my $category_list = $syslog_server->category_list();

#Modify category_list
$syslog_server->category_list(['DNS_CLIENT']);
```

certificate( )

Use this method to set a path to the file that contains a client certificate. This is a write-only attribute.

Note that the certificate must be set in order to establish an STCP connection.

Parameter

The valid value is a string that contains the path to the client certificate file.

Returns

The method returns true when the modification succeeds, and returns false when the operation fails.

Example

```
#Modify certificate
$syslog_server->certificate('/path/to/certificate.pem');
```

connection_type( )

Use this method to set or retrieve the connection type to the external syslog server. This method specifies whether the appliance uses TCP or UDP to connect to the external syslog server.

Note that the certificate must be set in order to establish an STCP connection.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

Valid values are: `"tcp", "udp", "stcp"`. The default value is "udp".

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#get connection_type
my $connection_type = $syslog_server->connection_type();
#Modify connection_type
$syslog_server->connection_type("tcp");
```

local_interface( )

Use this method to set or retrieve the interface through which the appliance sends syslog messages to the syslog server.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

Valid values are: `"any", "mgmt", "lan"`. The default value is "any".

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#get local_interface
my $local_interface = $syslog_server->local_interface();
#Modify local_interface
$syslog_server->local_interface("lan");
```

message_source( )

Use this method to set or retrieve the message source value of the syslog server. This method specifies which syslog messages the appliance sends to the external syslog server:

```
 Any     : The appliance sends both internal and external syslog messages.
 Internal: The appliance sends syslog messages that it generates.
 External: The appliance sends syslog messages that it receives from other appliances, such as syslog
servers and routers.
```

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

Valid values are: "any", "internal", "external". The default value is "any".

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#get message_source
my $message_source = $syslog_server->message_source();
#Modify message_source
$syslog_server->message_source("internal");
```

only_category_list( )

Use this method to enable or disable forwarding of syslog messages that belong to the list of categories.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

Specify "true" to enable forwarding or "false" to disable it. The default value is "false".

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get only_category_list
my $only_category_list = $syslog_server->only_category_list();

#Modify only_category_list
$syslog_server->only_category_list("true");
```

port( )

Use this method to set or retrieve the destination port number.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

Valid values are between 1 and 65535. The default value is 514.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#get port
my $port = $syslog_server->port();
#Modify port
$syslog_server->port("515");
```

severity( )

Use this method to set or retrieve the severity level for the syslog server. When you set a severity level, grid members send messages for that severity level plus all messages for all severity levels above it. The lowest severity level is "debug", and the highest severity level is "emerg".

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

Valid values are: "debug", "info", "notice", "warning", "err", "crit", "alert", "emerg". The default value is "debug".

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#get severity
my $severity = $syslog_server->severity();
#Modify severity
$syslog_server->severity("info");
```

SAMPLE CODE

The following sample code demonstrates the different functions that can be applied to a syslog server object such as add, modify, and remove. Also, this sample code includes error handling for the operations.

#Preparation prior to a syslog server object insertion

```
use strict;
use Infoblox;

#Create a session to the Infoblox appliance
my $session = Infoblox::Session->new(
    master   => "192.168.1.2",   #appliance host ip
    username => "admin",         #appliance user login
    password => "infoblox"       #appliance password
);
unless ($session) {
   die("Construct session failed: ",
       Infoblox::status_code() . ":" . Infoblox::status_detail());
}
print "Session created successfully.\n";
```

#Create a syslog server object

```
my $syslog_server = Infoblox::Grid::SyslogServer->new(
                    address => "1.1.1.1",
                    severity => "debug"
                   );

unless($syslog_server) {
       die("Construct syslog server object failed: ",
            Infoblox::status_code() . ":" . Infoblox::status_detail());
}
print "syslog server object created successfully\n";
```

#Get Infoblox grid object and add syslog server to it

```
#Get Infoblox grid object
my @grid_objs = $session->get(
           object => "Infoblox::Grid",
           name   => "Infoblox"
       );

my $grid = $grid_objs[0];
unless ($grid) {
       die("Get Grid object failed: ",
            $session->status_code() . ":" . $session->status_detail());
}
print "Get Grid object found at least 1 matching entry\n";

#Apply the changes to the Grid object
$grid->syslog_server([$syslog_server]);

#Apply the changes
$session->modify($grid)
    or die("Modify Grid object failed: ",
            $session->status_code() . ":" . $session->status_detail());

print "Grid object with syslog server updated to Infoblox appliance successfully\n";
```

#Modify syslog server object

```
#Get Infoblox grid object
my @grid_objs = $session->get(
           object => "Infoblox::Grid",
```

```
            name   => "Infoblox"
        );

 unless(scalar(@grid_objs) == 0){
        my $grid = $grid_objs[0];
        if($grid){
                my $syslog_obj = $grid->syslog_server();
                my @sys_servers = @{$syslog_obj};
                my $sys_server = $sys_servers[0];

                #Modifying the address of the syslog server object.
                $sys_server->address("2.2.2.2");

                #Modifying the connection_type of the syslog server object.
                $sys_server->connection_type("tcp");

                #Modifying the local_interface of the syslog server object.
                $sys_server->local_interface("lan");

                #Modifying the message_source of the syslog server object.
                $sys_server->message_source("external");

                #Modifying the port of the syslog server object.
                $sys_server->port("516");

                #Modifying the severity of the syslog server object.
                $sys_server->severity("warning");

                #Apply changes to the Grid object.
                $grid->syslog_server([$sys_server]);

                #Update Grid object through the Infoblox session.
                $session->modify($grid)
                             or die("modify Grid object failed: ",
                                 $session->status_code() . ":" . $session->status_detail());

                print "Grid object with modified syslog server updated to Infoblox appliance
successfully\n";
                        }
                 }
```

#Remove syslog server

```
 #Get Infoblox grid object
 my @grid_objs = $session->get(
            object => "Infoblox::Grid",
            name   => "Infoblox"
        );

 my $grid = $grid_objs[0];
 unless ($grid) {
        die("Get Grid object failed: ",
             $session->status_code() . ":" . $session->status_detail());
 }
 print "Get Grid object found at least 1 matching entry\n";

 #Remove syslog servers from the Grid object.
 $grid->syslog_server(undef);

 #Update Grid object through the Infoblox session.
 $session->modify($grid)
             or die("Remove syslog server from Grid object failed: ",
                 $session->status_code() . ":" . $session->status_detail());

 print "Removed syslog server from Grid object successfully\n";

 ####PROGRAM ENDS####
```

AUTHOR

Infoblox Inc. http://www.infoblox.com/

SEE ALSO

Infoblox::Session,Infoblox::Grid,Infoblox::Grid::Member

COPYRIGHT

Copyright (c) 2017 Infoblox Inc.

Infoblox::Grid::TestResult - A result object returned by various test functions.

- NAME
- DESCRIPTION
- SESSION METHODS
- METHODS
  - error_message( )
  - error_messages( )
  - overall_status( )
- AUTHOR
- SEE ALSO
- COPYRIGHT

NAME

Infoblox::Grid::TestResult - A result object returned by various test functions.

DESCRIPTION

The result object represents the evaluation status for various test operations (eg Threat Analytics server connectivity, Threat Protection server connectivity).

SESSION METHODS

The object does not support any session methods.

METHODS

This section describes all the methods that you can use to retrieve the attribute values of a test result object.

error_message( )

Use this method to retrieve the single error messages returned by the test operation. This is a read-only attribute.

Parameter

None

Returns

The method returns the attribute value, that is an error message in string format.

Example

```
#Get error_message value
my $error_message = $object->error_message();
```

error_messages( )

Use this method to retrieve the list of error messages returned by the test operation. This is a read-only attribute.

Parameter

None

Returns

The method returns an array reference that contains error messages in a string format.

Example

```
#Get error_messages value
my $error_messages = $object->error_messages();
```

overall_status( )

Use this method to retrieve the overall status returned by the test operation. This is a read-only attribute.

Parameter

None

Returns

The method returns the attribute value.

Example

```
#Get overall_status value
my $overall_status = $object->overall_status();
```

AUTHOR

Infoblox Inc. http://www.infoblox.com/

SEE ALSO

Infoblox::Grid::ThreatProtection->test_threat_protection_server_connectivity(), Infoblox::Grid::ThreatAnalytics->test_threat_analytics_server_connectivity(),

---

COPYRIGHT

Copyright (c) 2017 Infoblox Inc.

Infoblox::Grid::ThreatAnalytics - Grid Threat Analytics object.

- NAME
- DESCRIPTION
- SESSION METHODS
  - Infoblox::Session->get( )
  - Infoblox::Session->modify( )
- METHODS
  - current_moduleset( )
  - dns_tunnel_black_list_rpz_zones( )
  - download_threat_analytics_moduleset_update( )
  - enable_auto_download( )
  - enable_scheduled_download( )
  - last_checked_for_update( )
  - last_module_update_time( )
  - last_module_update_version( )
  - last_whitelist_update_time( )
  - last_whitelist_update_version( )
  - module_update_policy( )
  - scheduled_download( )
  - test_threat_analytics_server_connectivity( )
  - update_threat_analytics_moduleset( )
- AUTHOR
- SEE ALSO
- COPYRIGHT

NAME

Infoblox::Grid::ThreatAnalytics - Grid Threat Analytics object.

DESCRIPTION

The object provides information about Grid Threat Analytics settings.

SESSION METHODS

The following functions can be applied to a Infoblox::Grid::ThreatAnalytics object.

Infoblox::Session->get( )

Use this method to retrieve all the matching objects from the Infoblox appliance. See Infoblox::Session->get() for parameters and return values.

Key References

```
Apply the following attributes to get a specific Grid Threat Analytics object
 object - Required. It must be set to "Infoblox::Grid::ThreatAnalytics".
```

Example

```
my @retrieved_objs = $session->get(
    'object' => 'Infoblox::Grid::ThreatAnalytics',
);
```

Infoblox::Session->modify( )

Use this method to modify an object in the Infoblox appliance. See Infoblox::Session->modify() for parameters and return values.

Example

```
# Use this method to modify some of the attributes of the retrieved_objs object.
$retrieved_objs->dns_tunnel_black_list_rpz_zones([$rpz]);
# Submit modification
my $response = $session->modify($retrieved_objs);
```

METHODS

This section describes all the methods that you can use to configure and retrieve values of the Grid Threat Analytics object.

current_moduleset( )

Use this method to set or retrieve the current threat analytics model set

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

The valid value is an Infoblox::Grid::ThreatAnalytics::ModuleSet object.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get attribute value
my $current_moduleset = $obj->current_moduleset();
#Modify attribute value
$obj->current_moduleset($current_moduleset);
```

dns_tunnel_black_list_rpz_zones( )

Use this method to set or retrieve the list of Response Policy Zones for DNS tunnelling requests.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

The valid value is an array reference that contains Infoblox::DNS::Zone objects.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get attribute value
my $rpz_zones = $obj->dns_tunnel_black_list_rpz_zones();
#Modify attribute value
$obj->dns_tunnel_black_list_rpz_zones([$rpz_zone]);
```

download_threat_analytics_moduleset_update( )

Use this method to download and install the latest threar analytics module set update, available from the Infoblox threat analytics download server.

Key Reference

None

Returns

The method returns 1 when the operation succeeds, and 0 when the operation fails.

Example

```
my $res = $obj->download_threat_analytics_moduleset_update();
```

enable_auto_download( )

Use this method to set or retrieve the flag that indicates whether the automatic threat analytics module set download is enabled or disabled.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

Specify 'true' to enable the automatic download of the threat analytics module set or 'false' to disable it. The default value is 'false'.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get enable_auto_download value
my $enable_auto_download = $obj->enable_auto_download();
#Modify enable_auto_download value
$obj->enable_auto_download('false');
```

enable_scheduled_download( )

Use this method to set or retrieve the flag that indicates whether the scheduled download of the threat analytics module set is enabled or disabled.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

Specify 'true' to enable the scheduled download of the threat analytics module set or 'false' to disable it. The default value is 'false'.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get enable_scheduled_download value
my $enable_scheduled_download = $obj->enable_scheduled_download();
#Modify enable_scheduled_download value
$obj->enable_scheduled_download('false');
```

last_checked_for_update( )

Use this method to retrieve the last checked for update time for the analytics module set.

Omit the parameter to retrieve the attribute value.

Parameter

None

Returns

The method returns the number of seconds that have elapsed since January 1st, 1970 UTC.

Example

```
#Get last_checked_for_update value
my $last_checked_for_update = $obj->last_checked_for_update();
```

last_module_update_time( )

Use this method to retrieve the last update time for the threat analytics module set.

Omit the parameter to retrieve the attribute value.

Parameter

None

Returns

The method returns the number of seconds that have elapsed since January 1st, 1970 UTC.

Example

```
#Get last_module_update_time value
my $last_module_update_time = $obj->last_module_update_time();
```

last_module_update_version( )

Use this method to retrieve the version number of the last updated threat analytics module set.

Omit the parameter to retrieve the attribute value.

Parameter

None

Returns

The method returns the attribute value.

Example

```
#Get last_module_update_version value
my $last_module_update_version = $obj->last_module_update_version();
```

last_whitelist_update_time( )

Use this method to retrieve the last update time for the threat analytics whitelist.

Omit the parameter to retrieve the attribute value.

Parameter

None

Returns

The method returns the number of seconds that have elapsed since January 1st, 1970 UTC.

Example

```
#Get last_whitelist_update_time value
my $last_whitelist_update_time = $obj->last_whitelist_update_time();
```

last_whitelist_update_version( )

Use this method to retrieve the version number of the last updated threat analytics whitelist.

Omit the parameter to retrieve the attribute value.

Parameter

None

Returns

The method returns the attribute value.

Example

```
#Get last_whitelist_update_version value
my $last_whitelist_update_version = $obj->last_whitelist_update_version();
```

module_update_policy( )

Use this method to set or retrieve the updated policy for the threat analytics module set.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

The valid values are 'AUTOMATIC' and 'MANUAL'.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get module_update_policy value
my $module_update_policy = $obj->module_update_policy();
#Modify module_update_policy value
$obj->module_update_policy('MANUAL');
```

scheduled_download( )

Use this method to set or retrieve the schedule settings for the threat analytics module set download.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

The valid value is an Infoblox::Grid::ScheduleSetting object.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get scheduled_download value
my $scheduled_download = $obj->scheduled_download();
#Modify scheduled_download value
$obj->scheduled_download($sched_download);
```

test_threat_analytics_server_connectivity( )

Use this method ot test the connectivity with the Infoblox threat analytics download server.

Key Reference

None

Returns

The method returns an Infoblox::Grid::TestResult object if connectivity test occurs and 0 otherwise.

Example

```
my $res = $obj->test_threat_analytics_server_connectivity();
```

update_threat_analytics_moduleset( )

Use this method to update the threat analytics module set

Note that to update the threat analytics module set, the threat analytics update file should be uploaded via Infoblox::Session->import_data() method

Key Reference

```
mode - Optional. Specify 'TEST' to check uploaded module set or 'EXECUTE' to apply updates to the
current module set. The default value is 'EXECUTE'.
```

Returns

The method returns 1 when the operation succeeds, and 0 when the operation fails.

Example

```
my $res = $obj->update_threat_analytics_moduleset(mode => 'TEST');
```

AUTHOR

Infoblox Inc. http://www.infoblox.com/

SEE ALSO

Infoblox::Session, Infoblox::Session->get(), Infoblox::Session->modify(), Infoblox::DNS::Zone, Infoblox::Grid::ScheduleSetting, Infoblox::Grid::ThreatAnalytics::ModuleSet

---

COPYRIGHT

Copyright (c) 2017 Infoblox Inc.

Infoblox::Grid::ThreatAnalytics::Member - Member Analytics object.

- NAME
- DESCRIPTION
- SESSION METHODS
  - Infoblox::Session->get( )
  - Infoblox::Session->modify( )
  - Infoblox::Session->search( )
- METHODS
  - comment( )
  - enable_service( )
  - ipv4addr( )
  - ipv6addr( )
  - move_black_list_rpz_cnames_to_threat_analytics_whitelist( )
  - name( )
  - status( )
- AUTHOR
- SEE ALSO
- COPYRIGHT

NAME

Infoblox::Grid::ThreatAnalytics::Member - Member Analytics object.

DESCRIPTION

This object provides information about the member Threat Analytics settings.

SESSION METHODS

The following functions can be applied to a Infoblox::Grid::ThreatAnalytics::Member object.

Infoblox::Session->get( )

Use this method to retrieve all the matching objects from the Infoblox appliance. See Infoblox::Session->get() for parameters and return values.

Key References

```
Apply the following attributes to get a specific Member Threat Analytics object.

   comment  - The Threat Analytics member comment.
   ipv4addr - The Threat Analytics member IPv4 address.
   ipv6addr - The Threat Analytics member IPv6 address.
   name     - The Threat Analytics member name.
   status   - The Threat Analytics member status.
```

Example

```
my @retrieved_objs = $session->get(
    'object'      => 'Infoblox::Grid::ThreatAnalytics::Member',
    'name'        => 'host.com',
);
```

Infoblox::Session->modify( )

Use this method to modify an object in the Infoblox appliance. See Infoblox::Session->modify() for parameters and return values.

Example

```
# Use this method to modify attributes of an member object.
$member->enable_service('true');

# Submit modification
my $response = $session->modify($member);
```

Infoblox::Session->search( )

Use this method to retrieve all the matching objects from the Infoblox appliance. See Infoblox::Session->search() for parameters and return values.

Key References

```
Apply the following attributes to search for a Member Threat Analytics objects.

   comment  - The Threat Analytics member comment.
   ipv4addr - The Threat Analytics member IPv4 address.
   ipv6addr - The Threat Analytics member IPv6 address.
   name     - The Threat Analytics member name.
   status   - The Threat Analytics member status.
```

Example

```
my @retrieved_objs = $session->search(
    'object'      => 'Infoblox::Grid::ThreatAnalytics::Member',
    'name'        => 'host.*',
);
```

METHODS

This object provides information about the member Threat Analytics settings.

comment( )

Use this method to retrieve the descriptive comment for the Threat Analytics member object. This is a read-only attribute.

Omit the parameter to retrieve the attribute value.

Parameter

None

Returns

The method returns the attribute value.

Example

```
#Get comment value
my $comment = $analytics_member->comment();
```

enable_service( )

Use this method to set or retrieve the flag which indicates whether the Threat Analytics engine is enabled or disabled.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

Specify 'true' to enable Threat Analytics engine and 'false' to disable it. The default value is 'false'.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get enable_service value
my $enable_service = $analytics_member->enable_service();
#Modify enable_service value
$analytics_member->enable_service('true');
```

ipv4addr( )

Use this method to retrieve the IPv4 address of the Threat Analytics member object. This is a read-only attribute.

Omit the parameter to retrieve the attribute value.

Parameter

None

Returns

The method returns the attribute value.

Example

```
#Get ipv4addr value
```

```
my $ipv4addr = $analytics_member->ipv4addr();
```

ipv6addr( )

Use this method to retrieve the IPv6 address of the Threat Analytics member object. This is a read-only attribute.

Omit the parameter to retrieve the attribute value.

Parameter

None

Returns

The method returns the attribute value.

Example

```
#Get ipv6addr value
my $ipv6addr = $analytics_member->ipv6addr();
```

move_black_list_rpz_cnames_to_threat_analytics_whitelist( )

Use this method to replace DNS RPZ CNAME records with appropriate Threat Analytics Whitelist objects.

Parameter

The valid value is an array reference that contains Infoblox::DNS::RPZRecord::CNAME objects.

Returns

The method returns true when the operation succeeds, and returns false when the operation fails.

Example

```
#move cnames to analytics whitelist
$analytics_member->move_black_list_rpz_cnames_to_threat_analytics_whitelist(@cnames);
```

name( )

Use this method to retrieve the name of the Threat Analytics member object. This is a read-only attribute.

Omit the parameter to retrieve the attribute value.

Parameter

None

Returns

The method returns the attribute value.

Example

```
#Get name value
my $name = $analytics_member->name();
```

status( )

Use this method to retrieve the status of the Threat Analytics member object. This is a read-only attribute.

Omit the parameter to retrieve the attribute value.

Parameter

None

Returns

The valid return values are 'FAILED', 'INACTIVE', 'UNKNOWN', 'WARNING' and 'WORKING'.

Example

```
#Get status value
my $status = $analytics_member->status();
```

AUTHOR

Infoblox Inc. http://www.infoblox.com/

SEE ALSO

Infoblox::Session, Infoblox::Session->get(), Infoblox::Session->modify(), Infoblox::Session->search(), Infoblox::DNS::RPZRecord::CNAME

COPYRIGHT

Copyright (c) 2017 Infoblox Inc.

Infoblox::Grid::ThreatAnalytics::ModuleSet - A Grid Threat Analytics moduleset object.

- NAME
- DESCRIPTION
- SESSION METHODS
  - Infoblox::Session->get( )
  - Infoblox::Session->search( )
- METHODS
  - version( )
- AUTHOR
- SEE ALSO
- COPYRIGHT

NAME

Infoblox::Grid::ThreatAnalytics::ModuleSet - A Grid Threat Analytics moduleset object.

DESCRIPTION

The Grid Threat Analytics moduleset represents an uploaded Apache Spark Module.

SESSION METHODS

The following functions can be applied to a Threat Analytics moduleset object.

Infoblox::Session->get( )

Use this method to retrieve all the matching objects from the Infoblox appliance. See Infoblox::Session->get() for parameters and return values.

Key References

```
Apply the following attributes to get a specific Threat Analytics moduleset object.

 version - Optional. The Threat Analytics moduleset version.
```

Example

```
my @retrieved_objs = $session->get(
    'object' => 'Infoblox::Grid::ThreatAnalytics::ModuleSet',
    'version' => '1.0',
);
```

Infoblox::Session->search( )

Use this method to search for objects in the Infoblox appliance. See Infoblox::Session->search() for parameters and return values.

Key References

```
Apply the following attributes to get a specific Threat Analytics moduleset object.

 version - Optional. The Threat Analytics moduleset version.
```

Example

```
my @retrieved_objs = $session->get(
    'object' => 'Infoblox::Grid::ThreatAnalytics::ModuleSet',
    'version' => '1.*',
);
```

METHODS

version( )

Use this method to retrieve the Threat Analytics moduleset version.

Omit the parameter to retrieve the attribute value.

Parameter

None

Returns

The method returns the attribute value.

Example

```
#Get version
my $version = $object->version();
```

AUTHOR

Infoblox Inc. http://www.infoblox.com/

SEE ALSO

Infoblox::Session, Infoblox::Session->get(), Infoblox::Session->search()

COPYRIGHT

Copyright (c) 2017 Infoblox Inc.

Infoblox::Grid::ThreatAnalytics::WhiteList - a Grid Threat Analytics White List object

- NAME
- DESCRIPTION
- CONSTRUCTOR
- SESSION METHODS
  - Infoblox::Session->add( )
  - Infoblox::Session->get( )
  - Infoblox::Session->modify( )
  - Infoblox::Session->remove( )
  - Infoblox::Session->search( )
- METHODS
  - comment( )
  - disable( )
  - fqdn( )
  - status( )
- AUTHOR
- SEE ALSO
- COPYRIGHT

NAME

Infoblox::Grid::ThreatAnalytics::WhiteList - a Grid Threat Analytics White List object

---

DESCRIPTION

The Grid Threat Analytics White List provides information about allowed FQDNs in the DNS tunneling detection feature.

CONSTRUCTOR

```
my $whitelist = Infoblox::Grid::ThreatAnalytics::WhiteList->new(
   fqdn    => $fqdn,            # Required
   comment => $string,          # Optional / Default is undefined
   disable => 'true' | 'false', # Optional / Default is 'false'
);
```

SESSION METHODS

The following functions can be applied to a Infoblox::Grid::ThreatAnalytics::WhiteList object.

Infoblox::Session->add( )

Use this method to add an object to the Infoblox appliance. See Infoblox::Session->add() for parameters and return values.

Example

```
#Construct an object
my $whitelist = Infoblox::Grid::ThreatAnalytics::WhiteList->new(
   fqdn => 'test.com',
   comment => 'test comment',
   disable => 'true',
);

#Submit for addition
my $response = $session->add( $whitelist );
```

Infoblox::Session->get( )

Use this method to retrieve all the matching objects from the Infoblox appliance. See Infoblox::Session->get() for parameters and return values.

Key References

```
Apply the following attributes to get a specific Infoblox::Grid::ThreatAnalytics::WhiteList object.
 fqdn    - Optional. The Grid Threat Analytics White List FQDN.
 status  - Optional. The Grid Threat Analytics White List status.
 comment - Optional. The Grid Threat Analytics White List comment.
```

Example

```
my @retrieved_objs = $session->get(
    'object' => 'Infoblox::Grid::ThreatAnalytics::WhiteList',
    'fqdn'   => 'test.com',
);
```

Infoblox::Session->modify( )

Use this method to modify an object in the Infoblox appliance. See Infoblox::Session->modify() for parameters and return values.

Example

```
# modify whitelist object
$whitelist->comment('comment change');

# Submit modification
my $response = $session->modify( $whitelist );
```

Infoblox::Session->remove( )

Use this method to remove an object from the Infoblox appliance. See Infoblox::Session->remove() for parameters and return values.

To remove a specific object, first use `get()` or `search()` to retrieve the specific object, and then submit this object for removal.

Note that 'SYSTEM' Grid Threat Analytics White Lists cannot be removed.

Example

```
#Get the objects with the same name
my @retrieved_objs = $session->get(
    object => 'Infoblox::Grid::ThreatAnalytics::WhiteList',
    fqdn   => 'test.com');

#Find the desired object from the retrieved list.
my $desired_whitelist = $retrieved_objs[0];

# Submit for removal
my $response = $session->remove( $desired_whitelist );
```

Infoblox::Session->search( )

Use this method to search for objects in the Infoblox appliance. See Infoblox::Session->search() for parameters and return values.

Key References

```
 Apply the following attributes to search for a specific Infoblox::Grid::ThreatAnalytics::WhiteList
object.
  fqdn    - Optional. The Grid Threat Analytics White List FQDN.
  status  - Optional. The Grid Threat Analytics White List status.
  comment - Optional. The Grid Threat Analytics White List comment.
```

Example

```
my @retrieved_objs = $session->search(
    'object' => 'Infoblox::Grid::ThreatAnalytics::WhiteList',
    'fqdn'   => 'test.*',
);
```

METHODS

comment( )

Use this method to set or retrieve Grid Threat Analytics White List comment.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

The valid value is desired comment in string format.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get comment
my $comment = $whitelist->comment();

#Modify comment
$whitelist->comment('new comment');
```

disable( )

Use this method to disable or enable Grid Threat Analytics White List.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

Specify 'true' to disable Grid Threat Analytics White List or 'false' to enable it. The default value is 'false'.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Exmaple

```
#Get disable
my $disable = $whitelist->disable();

#Modify disable
$whitelist->disable('true');
```

fqdn( )

Use this method to set or retrieve a fully-qualified domain name (FQDN) of the Grid Threat Analytics White List.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

The valid value is a desired FQDN in string format.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Exmaple

```
#Get fqdn
my $fqdn = $whitelist->fqdn();

#Modify fqdn
$whitelist->fqdn('test2.com');
```

status( )

Use this method to retrieve the status of Grid Threat Analytics White List. This is a read-only attribute.

Omit the parameter to retrieve the attribute value.

Parameter

None

Returns

The valid return value is 'CUSTOM', 'BOTH' or 'SYSTEM'.

Example

```
#Get status
my $status = $whitelist->status();
```

AUTHOR

Infoblox Inc. http://www.infoblox.com/

SEE ALSO

Infoblox::Session, Infoblox::Session->add(), Infoblox::Session->get(), Infoblox::Session->remove(), Infoblox::Session->search(), Infoblox::Session->modify()

COPYRIGHT

Copyright (c) 2017 Infoblox Inc.

Infoblox::Grid::ThreatInsight::CloudClient - Grid Threat Insight Cloud Client object.

- NAME
- DESCRIPTION
- SESSION METHODS
  - Infoblox::Session->get( )
  - Infoblox::Session->modify( )
- METHODS
  - blacklist_rpz_list( )
  - enable( )
  - force_refresh( )
  - interval( )
- AUTHOR
- SEE ALSO
- COPYRIGHT

NAME

Infoblox::Grid::ThreatInsight::CloudClient - Grid Threat Insight Cloud Client object.

DESCRIPTION

Use this object to configure the cloud client that is used for integrating with Infoblox Threat Insight in the cloud.

SESSION METHODS

The following functions can be applied to a Infoblox::Grid::ThreatInsight::CloudClient object.

Infoblox::Session->get( )

Use this method to retrieve all the matching objects from the Infoblox appliance. See Infoblox::Session->get() for parameters and return values.

Example

```
my @retrieved_objs = $session->get(
    'object' => 'Infoblox::Grid::ThreatInsight::CloudClient',
);
```

Infoblox::Session->modify( )

Use this method to modify an object in the Infoblox appliance. See Infoblox::Session->modify() for parameters and return values.

Example

```
# Use this method to modify some of the attributes of the object.
$object->enable('true');

# Submit modification
my $response = $session->modify($object);
```

METHODS

This section describes all the methods that you can use to configure and retrieve values of the Grid Threat Insight Cloud Client object.

blacklist_rpz_list( )

Use this method to set or retrieve the list of RPZs to which domains detected by the Threat Insight Cloud Client are targeted.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

The valid value is an array of Infoblox::DNS::Zone objects.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get blacklist_rpz_list value
my $blacklist_rpz_list = $object->blacklist_rpz_list();

#Modify blacklist_rpz_list value
$object->blacklist_rpz_list([$rpz_zone]);
```

enable( )

Use this method to set or retrieve the flag that indicates whether Threat Insight on the Cloud Client is enabled.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

Specify 'true' to enable the Threat Insight in the Cloud Client and 'false' to disable it. The default is 'false'.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get enable value
my $enable = $object->enable();

#Modify enable value
$object->enable('true');
```

force_refresh( )

Use this method to set the flag that indicates whether force refresh for Response Policy Zones is enabled. This is a write-only attribute.

Note that force refresh takes effect only if at least one new Response Policy Zone is configured. Otherwise, the corresponding CLI command is used.

Parameter

Specify 'true' to enable force refresh for the Response Policy Zone and 'false' to disable it. The default is 'true'.

Parameter

The method returns true when the modification succeeds, and returns false when the operation fails.

Example

```
#Modify force_refresh value
$object->force_refresh('false');
```

interval( )

Use this method to set or retrieve the time interval (in seconds) for the Threat Insight Cloud Client integration.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

The valid value is an unsigned integer between 60 and 3600. The default value is 600.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get interval value
my $interval = $object->interval();

#Modify interval value
$object->interval(3000);
```

AUTHOR

Infoblox::Grid::ThreatInsight::CloudClient - Grid Threat Insight Cloud Client object.

Infoblox Inc. http://www.infoblox.com/

SEE ALSO

Infoblox::Session, Infoblox::Session->get(), Infoblox::Session->modify(), Infoblox::DNS::Zone.

COPYRIGHT

Copyright (c) 2017 Infoblox Inc.

Infoblox::Grid::ThreatProtection - Grid Threat Protection object.

- NAME
- DESCRIPTION
- SESSION METHODS
  - Infoblox::Session->get( )
  - Infoblox::Session->modify( )
- METHODS
  - current_ruleset( )
  - enable_nat_rules( )
  - events_per_second_per_rule( )
  - enable_auto_download( )
  - enable_scheduled_download( )
  - rule_update_policy( )
  - last_checked_for_update( )
  - last_rule_update_version( )
  - nat_rules( )
  - scheduled_download( )
  - test_threat_protection_server_connectivity( )
  - disable_multiple_dns_tcp_request( )
- AUTHOR
- SEE ALSO
- COPYRIGHT

NAME

Infoblox::Grid::ThreatProtection - Grid Threat Protection object.

DESCRIPTION

This object provides information about the Grid Threat protection settings.

SESSION METHODS

The following functions can be applied to a Grid::ThreatProtection object.

Infoblox::Session->get( )

Use this method to retrieve all the matching objects from the Infoblox appliance. See Infoblox::Session->get() for parameters and return values.

Key References

```
Apply the following attributes to get a specific Grid Threat Protection object
 object - Required. It must be set to "Infoblox::Grid::ThreatProtection".
```

Example

```
my @retrieved_objs = $session->get(
    'object' => 'Infoblox::Grid::ThreatProtection',
);
```

Infoblox::Session->modify( )

Use this method to modify an object in the Infoblox appliance. See Infoblox::Session->modify() for parameters and return values.

Example

```
# Use this method to modify some of the attributes of the retrieved_objs object.
$retrieved_objs->events_per_second_per_rule('2');
$retrieved_objs->rule_update_policy('MANUAL');
# Submit modification
my $response = $session->modify( $retrieved_objs );
```

METHODS

This section describes all the methods that you can use to configure and retrieve values of the Threat Protection object.

current_ruleset( )

Use this method to set or retrieve the current Grid ruleset.

Parameter

Valid value is an Infoblox::Grid::ThreatProtection::Ruleset object.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get attribute value
my $value = $obj->current_ruleset();
#Modify attribute value
$obj->current_ruleset($rs);
```

enable_nat_rules( )

Use this method to enable or disable the NAT Threat Protection feature.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Paramter

Specify 'true' to enable NAT mapping support for threat protection or 'false' to disable it. The default value is 'false'.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get attribute value
my $value = $object->enable_nat_rules();

#Modify attribute value
$object->enable_nat_rules('true');
```

events_per_second_per_rule( )

Use this method to set or retrieve the number of events logged per second per rule.

Parameter

The number of events logged per second per rule in string format.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
# Get attribute value
my $value = $object->events_per_second_per_rule();
# Modify attribute value
$object->events_per_second_per_rule('2');
```

enable_auto_download( )

Use this method to enable or disable the auto download service.

Parameter

Specify 'true' to enable the auto download service or 'false' to disable it. The default value is 'false'.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get attribute value
my $value = $obj->enable_auto_download();
#Modify attribute value
$obj->enable_auto_download('true');
```

enable_scheduled_download( )

Use this method to enable or disable the scheduled download. The default frequency is once every 24 hours if it is disabled.

Parameter

Specify 'true' to enable the scheduled download or 'false' to disable it. The default value is 'false'.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get attribute value
my $value = $obj->enable_scheduled_download();
#Modify attribute value
$obj->enable_scheduled_download('true');
```

rule_update_policy( )

Use this method to set or retrieve the updated rule policy.

Parameter

Valid value is 'AUTOMATIC' or 'MANUAL'.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
# Get attribute value
my $value = $object->rule_update_policy();
# Modify attribute value
$object->rule_update_policy('MANUAL');
```

last_checked_for_update( )

Use this method to retrieve the time when the Grid last checked for updates.

Parameter

None

Returns

The method returns the attribute value, the number of seconds that have elapsed since January 1st, 1970 UTC.

Example

```
# Get attribute value
my $value = $object->last_checked_for_update();
```

last_rule_update_version( )

Use this method to retrieve the last rule update version. This is a read-only attribute.

Parameter

None

Returns

The method returns the attribute value.

Example

```
# Get attribute value
my $value = $object->last_rule_update_version();
```

nat_rules( )

Use this method to set or retrieve the list of NAT mapping rules for threat protection.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

The valid value is an array of Infoblox::Grid::ThreatProtection::NATRule objects.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get attribute value
my $value = $obj->nat_rules();
#Modify attribute value
$obj->nat_rules($nat_rules);
```

scheduled_download( )

Use this method to set or retrieve the schedule setting for automatic rule update.

Parameter

Valid value is an Infoblox::Grid::ScheduleSetting object.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get attribute value
my $value = $obj->scheduled_download();
#Modify attribute value
$obj->scheduled_download($schedule);
```

test_threat_protection_server_connectivity( )

Use this method ot test the connectivity with the Infoblox threat protection server.

Key Reference

None

Returns

The method returns an Infoblox::Grid::TestResult object if connectivity test occurs and 0 otherwise.

Example

```
my $res = $obj->test_threat_protection_server_connectivity();
```

disable_multiple_dns_tcp_request( )

Use this method to disable multiple BIND responses via TCP connection.

Parameter

Set the parameter to "true" to disable multiple TCP requests. Set the parameter to "false" to enable multiple TCP requests.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
# Get attribute value
my $value = $object->disable_multiple_dns_tcp_request();
# Modify attribute value
$object->disable_multiple_dns_tcp_request('true');
```

AUTHOR

Infoblox Inc. http://www.infoblox.com/

SEE ALSO

Infoblox::Session, Infoblox::Grid::Member::ThreatProtection, Infoblox::Grid::ScheduleSetting, Infoblox::Grid::ThreatProtection::NATRule

COPYRIGHT

Copyright (c) 2017 Infoblox Inc.

Infoblox::Grid::ThreatProtection::NATPort - a NAT Threat Protection Port object.

- NAME
- DESCRIPTION
- CONSTRUCTION
- SESSION METHODS
- MODULE METHODS
  - Infoblox::Grid::ThreatProtection::NATRule->nat_ports( )
- METHODS
  - block_size( )
  - end_port( )
  - start_port( )
- AUTHOR
- SEE ALSO
- COPYRIGHT

NAME

Infoblox::Grid::ThreatProtection::NATPort - a NAT Threat Protection Port object.

DESCRIPTION

The Threat Protection NAT Port object provides information about the port blocks configured for NAT mapping.

CONSTRUCTION

```
my $nat_port = Infoblox::Grid::ThreatProtection::NATPort->new(
   start_port => $uint, #Required
   end_port   => $uint, #Required
   block_size => $uint, #Required
);
```

SESSION METHODS

The object does not support any session methods.

MODULE METHODS

The following functions are available to apply to a NAT Port object.

Infoblox::Grid::ThreatProtection::NATRule->nat_ports( )

Use this method to set or retrieve NAT Port configuration for the NAT Threat Protection Rule. See Infoblox::Grid::ThreatProtection::NATRule->nat_ports() for parameters and return values.

Example

```
#Get nat_ports
my $nat_ports = $nat_rule->nat_ports();

#Modify nat_ports
$nat_rule->nat_ports($nat_ports);
```

METHODS

This section describes all the methods that you can use to configure and retrieve the attribute values of a Threat Protection NAT Port.

block_size( )

Use this method to set or retrieve the block size for the NAT Port configuration object.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

The valid value is an unsigned integer.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get attribute value
my $value = $object->block_size();

#Modify attribute value
$object->block_size(20);
```

end_port( )

Use this method to set or retrieve the end port value for the NAT port configuration object.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

The valid value is an unsigned integer.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get attribute value
my $value = $object->end_port();

#Modify attribute value
$object->end_port(10);
```

start_port( )

Use this method to set or retrieve the start port value for the NAT port configuration object.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

The valid value is an unsigned integer.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get attribute value
my $value = $object->start_port();

#Modify attribute value
$object->start_port(5);
```

AUTHOR

Infoblox Inc. http://www.infoblox.com/

SEE ALSO

Infoblox::Grid::ThreatProtection::NATRule, Infoblox::Grid::ThreatProtection::NATRule->nat_ports()

COPYRIGHT

Copyright (c) 2017 Infoblox Inc.

Infoblox::Grid::ThreatProtection::NATRule - a NAT Threat Protection Rule object.

- NAME
- DESCRIPTION
- CONSTRUCTOR
- SESSION METHODS
- MODULE METHODS
  - Infoblox::Grid::ThreatProtection->nat_rules( )
  - Infoblox::Grid::Member::ThreatProtection->nat_rules( )
- METHODS
  - address( )
  - cidr( )
  - end_address( )
  - nat_ports( )
  - network( )
  - rule_type( )
  - start_address( )
- AUTHOR
- SEE ALSO
- COPYRIGHT

NAME

Infoblox::Grid::ThreatProtection::NATRule - a NAT Threat Protection Rule object.

DESCRIPTION

The NAT Threat Protection Rule object provides information about NAT rules configured for the threat protection NAT mapping feature.

CONSTRUCTOR

```
my $nat_rule = Infoblox::Grid::ThreatProtection::NATRule->new(
    rule_type     => 'ADDRESS' | 'NETWORK' | 'RANGE', #Required
    address       => $ipaddress,                      #Required if rule_type is 'ADDRESS'
    network       => $ipaddress,                      #Required if rule_type is 'NETWORK'
    cidr          => $uint,                           #Required if rule_type is 'NETWORK'
    start_address => $ipaddress,                      #Required if rule_type is 'RANGE'
    end_address   => $ipaddress,                      #Required if rule_type is 'RANGE'
    nat_ports     => $nat_ports,                      #Optional
);
```

SESSION METHODS

The object does not support any session methods.

MODULE METHODS

The following functions are available to apply to a NAT Port object.

Infoblox::Grid::ThreatProtection->nat_rules( )

Use this method to set or retrieve the list of rules configured for the threat protection NAT mapping feature. See Infoblox::Grid::ThreatProtection->nat_rules() for parameters and return values.

Example

```
#Get nat_rules
my $nat_rules = $nat_rule->nat_rules();

#Modify nat_rules
$nat_rule->nat_rules($nat_rules);
```

Infoblox::Grid::Member::ThreatProtection->nat_rules( )

Use this method to set or retrieve the list of rules configured for the threat protection NAT mapping feature. See Infoblox::Grid::Member::ThreatProtection->nat_rules() for parameters and return values.

Example

```
#Get nat_rules
my $nat_rules = $nat_rule->nat_rules();

#Modify nat_rules
$nat_rule->nat_rules($nat_rules);
```

METHODS

This section describes all the methods that you can use to configure and retrieve the attribute values of a Threat Protection NAT Port.

address( )

Use this method to set or retrieve the IP address for the threat protection NAT mapping rule.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

The valid value is a desired IP address in string format.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get attribute value
my $value = $object->address();

#Modify attribute value
$object->address('1.1.1.1');
```

cidr( )

Use this method to set or retrieve the network CIDR for the threat protection NAT mapping rule.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

The valid value is an unsigned integer.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get attribute value
my $value = $object->cidr();

#Modify attribute value
$object->cidr(8);
```

end_address( )

Use this method to set or retrieve the end address for the range of the threat protection NAT mapping rule.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

The valid value is a desired IP address in string format.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get attribute value
my $value = $object->end_address();
```

```
#Modify attribute value
$object->end_address('1.1.0.255');
```

nat_ports( )

Use this method to set or retrieve the NAT port configuration for the threat protection NAT mapping rule.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

The valid value is an array of Infoblox::Grid::ThreatProtection::NATPort objects.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get attribute value
my $value = $object->nat_ports();

#Modify attribute value
$object->nat_ports($nat_ports);
```

network( )

Use this method to set or retrieve the network address for the threat protection NAT mapping rule.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

The valid value is an unsigned integer.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get attribute value
my $value = $object->network();

#Modify attribute value
$object->network('10.0.0.0');
```

rule_type( )

Use this method to set or retrieve the rule type for the threat protection NAT mapping rule.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

The valid values are 'ADDRESS', 'RANGE', 'NETWORK'.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get attribute value
my $value = $object->rule_type();

#Modify attribute value
$object->rule_type('RANGE');
```

start_address( )

Use this method to set or retrieve the start address of the range for the threat protection NAT mapping rule.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

The valid value is a desired IP address in string format.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get attribute value
my $value = $object->start_address();

#Modify attribute value
$object->start_address('1.1.0.255');
```

AUTHOR

Infoblox Inc. http://www.infoblox.com/

SEE ALSO

Infoblox::Grid::ThreatProtection::NATPort, Infoblox::Grid::ThreatProtection->nat_rules(), Infoblox::Grid::ThreatProtection, Infoblox::Grid::Member::ThreatProtection->nat_rules(), Infoblox::Grid::Member::ThreatProtection.

COPYRIGHT

Copyright (c) 2017 Infoblox Inc.

Infoblox::Grid::ThreatProtection::Profile - Threat Protection profile object.

- NAME
- DESCRIPTION
- CONSTRUCTOR
- SESSION METHODS
  - Infoblox::Session->add( )
  - Infoblox::Session->get( )
  - Infoblox::Session->modify( )
  - Infoblox::Session->remove( )
  - Infoblox::Session->search( )
- METHODS
  - comment( )
  - current_ruleset( )
  - disable_multiple_dns_tcp_request( )
  - events_per_second_per_rule( )
  - extattrs( )
  - extensible_attributes( )
  - members( )
  - name( )
  - override_events_per_second_per_rule( )
  - override_current_ruleset( )
  - override_disable_multiple_dns_tcp_request( )
  - source( )
- SAMPLE CODE
- AUTHOR
- SEE ALSO
- COPYRIGHT

NAME

Infoblox::Grid::ThreatProtection::Profile - Threat Protection profile object.

DESCRIPTION

The Threat Protection profile object facilitates configuring groups of Threat Protection members that have similar traffic properties. A member can be either associated with a Threat Protection profile or inherit the ruleset from the Grid or override the ruleset individually at the member level.

CONSTRUCTOR

```
 my $object = Infoblox::Grid::ThreatProtection::Profile->new(
     name                                    => $string,
#Required
     comment                                 => $string,
#Optional / Default is undefined
     current_ruleset                         => $ruleset,
#Optional / Default is undefined
     disable_multiple_dns_tcp_request        => 'true' | 'false',
#Optional / Default is 'true'
     events_per_second_per_rule              => $uint,
#Optional / Default is undefined
     extattrs                                => { $string => $extattr, ... },
#Optional / Default is undefined
     extensible_attributes                   => {$string => $string | $num, $string => [ $string | $num,
... ], ... }, #Optional / Default is undefined
     members                                 => [$host_name | ... ],
#Optional / Default is empty list
     override_current_ruleset                => 'true' | 'false',
#Optional / Default is 'false'
     override_disable_multiple_dns_tcp_request => 'true' | 'false',
#Optional / Default is 'false'
     override_events_per_second_per_rule     => 'true' | 'false',
#Optional / Default is 'false'
     source                                  => $tp_member | $tp_profile,
#Optional / Default is undefined
 );
```

You cannot set both extattrs and extensible_attributes attributes at the same time.

SESSION METHODS

The following functions can be applied to a Infoblox::Grid::ThreatProtection::Profile object.

Infoblox::Session->add( )

Use this method to add the object to the Infoblox appliance. See Infoblox::Session->add() for parameters and return values.

Example

```
#Construct object
my $object = Infoblox::Grid::ThreatProtection::Profile->new(
    name => 'profile1',
);

#Submit object for addition
$session->add($object);
```

Infoblox::Session->get( )

Use this method to retrieve all the matching objects from the Infoblox appliance. See Infoblox::Session->get() for parameters and return values.

Key References

```
Apply the following attributes to get a specific Threat Protection profile object:

    name                  - Optional. The Threat Protection profile name.
    comment               - Optional. The Threat Protection profile comment.
    extattrs              - Optional. A hash reference containing Infoblox::Grid::Extattr object.
    extensible_attributes - Optional. A hash reference containing extensible attributes.
```

Example

```
my @retrieved_objs = $session->get(
    'object' => 'Infoblox::Grid::ThreatProtection::Profile',
    'name'   => 'profile1',
);
```

Infoblox::Session->modify( )

Use this method to modify an object in the Infoblox appliance. See Infoblox::Session->modify() for parameters and return values.

Example

```
#Modify events_per_second_per_rule value
$object->events_per_second_per_rule('2');

#Submit modification
my $response = $session->modify($object);
```

Infoblox::Session->remove( )

Use this method to remove the object from the Infoblox appliance. See Infoblox::Session->remove() for parameters and return values.

To remove a specific object, first use `get()` or `search()` to retrieve the specific DHCP range object, and then submit this object for removal.

Example

```
#Get the objects
my @retrieved_objs = $session->get(
    object => 'Infoblox::Grid::ThreatProtection::Profile',
    name   => 'profile1',
);

#Find the desired object from the retrieved list
my $desired_object = $retrieved_objs[0];

#Submit for removal
my $response = $session->remove($desired_object);
```

Infoblox::Session->search( )

Use this method to search for objects in the Infoblox appliance. See Infoblox::Session->search() for parameters and return values.

Key References

```
Apply the following attributes to search for the Threat Protection profile object:
```

```
name                  - Optional. The Threat Protection profile name (regular expression).
comment               - Optional. The Threat Protection profile comment (regular expression).
extattrs              - Optional. A hash reference containing Infoblox::Grid::Extattr object.
extensible_attributes - Optional. A hash reference containing extensible attributes.
```

For more information about searching extensible attributes, see Infoblox::Grid::ExtensibleAttributeDef/Searching Extensible Attributes.

Examples

```
# search for objects
my @retrieved_objs = $session->search(
    object      => 'Infoblox::Grid::ThreatProtection::Profile',
    comment     => '.*comment',
);

# search for objects defining 'Santa Clara' for 'Site' extensible attribute
my @retrieved_objs = $session->search(
    object               => 'Infoblox::Grid::ThreatProtection::Profile',
    extensible_attributes => {'Site' => 'Santa Clara'},
);
```

METHODS

This object provides information about the Threat Protection profile settings.

comment( )

Use this method to set or retrieve the Threat Protection profile descriptive comment.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

Valid value is a desired comment in a string format.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get comment value
my $value = $object->comment();

#Modify comment value
$object->comment('new comment');
```

current_ruleset( )

Use this method to set or retrieve the current Threat Protection profile ruleset.

Setting this method to a defined value implicitly sets the override_current_ruleset method to 'true'. Setting the parameter to undefined causes the appliance to use the Grid default and automatically resets the override_current_ruleset attribute to 'false'.

Note that when current_ruleset is set to a defined value and override_dcurrent_ruleset is set to 'false', the last operation takes precedence. Thus the sequence $object->current_ruleset($rs); $object->override_current_ruleset('false'); will set override_current_ruleset to 'false', and the sequence $object->override_current_ruleset('false'); $object->current_ruleset($rs); will result in override_current_ruleset='true'.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

Valid value is an Infoblox::Grid::ThreatProtection::Ruleset object.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get current_ruleset value
my $value = $object->current_ruleset();
```

```
#Modify current_ruleset value
$object->current_ruleset($rs);
```

disable_multiple_dns_tcp_request( )

Use this method to disable multiple BIND responses via TCP connection.

Setting this method to a defined value implicitly sets the override_disable_multiple_dns_tcp_request method to 'true'. Setting the parameter to undefined causes the appliance to use the Grid default and automatically resets the override_disable_multiple_dns_tcp_request attribute to 'false'.

Note that when config is set to a defined value and override_disable_multiple_dns_tcp_request is set to 'false', the last operation takes precedence. Thus the sequence $object->disable_multiple_dns_tcp_request($value); $object->override_disable_multiple_dns_tcp_request('false'); will set override_disable_multiple_dns_tcp_request to 'false', and the sequence $object->override_disable_multiple_dns_tcp_request('false'); $object->disable_multiple_dns_tcp_request($value); will result in override_disable_multiple_dns_tcp_request='true'.

Parameter

Set the parameter to 'true' to disable multiple TCP requests. Set the parameter to 'false' to enable multiple TCP requests.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
# Get attribute value
my $value = $object->disable_multiple_dns_tcp_request();
# Modify attribute value
$object->disable_multiple_dns_tcp_request('true');
```

events_per_second_per_rule( )

Use this method to set or retrieve the number of events logged per second per rule.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

The number of events logged per second per rule in string format.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get events_per_second_per_rule value
my $value = $object->events_per_second_per_rule();

#Modify events_per_second_per_rule value
$object->events_per_second_per_rule('2');
```

extattrs( )

Use this method to set or retrieve the extensible attributes that are associated with a Threat Protection profile object.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

Valid value is a hash reference that contains the names of extensible attributes and their associated values (Infoblox::Grid::Extattr objects).

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get extattrs value
```

```
my $value = $object->extattrs();

#Modify extattrs value
$object->extattrs({'Site' => $extattr1, 'Administrator' => $extattr2});
```

extensible_attributes( )

Use this method to set or retrieve the extensible attributes that are associated with a Threat Protection profile object.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

For valid values for extensible attributes, see Infoblox::Grid::ExtensibleAttributeDef/Extensible Attribute Values.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get extensible attributes value
my $value = $object->extensible_attributes();

#Modify extensible attributes value
$object->extensible_attributes({'Site' => 'Santa Clara', 'Administrator' => ['Peter', 'Tom']});
```

members( )

Use this method to set or retrieve the list of members that are using the Threat Protection profile.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

The valid value is an array reference that contains desired Threat Protection member names in a string format.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get members value
my $value = $object->members();

#Modify members value
$object->members(['infoblox.localdomain']);
```

name( )

Use this method to retrieve a Threat Protection profile name.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

The valid value is a desired Threat Protection profile name in a string format.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get name value
my $value = $object->name();

#Modify name value
$object->name('new_name');
```

override_events_per_second_per_rule( )

Use this method to set or retrieve the override flag for the number of events logged per second per rule.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

Specify 'true' to set the override_events_per_second_per_rule flag or 'false' to deactivate/unset it. The default value is 'false'.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get override_events_per_second_per_rule value
my $value = $object->override_events_per_second_per_rule();

#Modify override_events_per_second_per_rule value
$object->override_events_per_second_per_rule('true');
```

override_current_ruleset( )

The override_current_ruleset attribute controls whether the current ruleset value is used for Threat Protection profile instead of the Grid default.

The override_current_ruleset attribute can be specified explicitly. It is also set implicitly when current_ruleset is set to a defined value.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

Specify 'true' to set the override_current_ruleset flag or 'false' to deactivate/unset it. The default value is 'false'.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get override_current_ruleset value
my $override = $object->override_current_ruleset();

#Modify override_current_ruleset
$object->override_current_ruleset('true');
```

override_disable_multiple_dns_tcp_request( )

The override_disable_multiple_dns_tcp_request attribute controls whether the Threat Protection profile disable_multiple_dns_tcp_request value is used instead of the Grid default.

The override_disable_multiple_dns_tcp_request attribute can be specified explicitly. It is also set implicitly when disable_multiple_dns_tcp_request is set to a defined value.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

Specify 'true' to set the override_disable_multiple_dns_tcp_request flag or 'false' to unset it. The default value is 'false'.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get override_disable_multiple_dns_tcp_request value
my $value = $object->override_disable_multiple_dns_tcp_request();

#Modify override_disable_multiple_dns_tcp_request value
$object->override_disable_multiple_dns_tcp_request('true');
```

source( )

Use this method to set the source for cloning the profile settings. This is a write-only attribute and can be used only on create operation.

Include the specified parameter to set the attribute value.

Parameter

The valid value is an Infoblox::Grid::Member::ThreatProtection or Infoblox::Grid::ThreatProtection::Profile object reference.

Returns

The method returns true when the modification succeeds, and returns false when the operation fails.

Example

```
#Modify source value
$object->source($profile1);
```

SAMPLE CODE

The following sample code demonstrates the different functions that can be applied to an object, such as add, search, modify, and remove. This sample also includes error handling for the operations.

#Preparation prior to a Threat Protection profile insertion

```
#PROGRAM STARTS: Include all the modules that will be used
use strict;
use Infoblox;

#Create a session to the Infoblox appliance

my $session = Infoblox::Session->new(
               master   => "192.168.1.2", #appliance host ip
               username => "admin",       #appliance user login
               password => "infoblox"     #appliance password
               );

unless ($session) {
       die("Construct session failed: ",
               Infoblox::status_code() . ":" . Infoblox::status_detail());
}
print "Session created successfully\n";
```

#Create a Threat Protection profile

```
my $atp_profile = Infoblox::Grid::ThreatProtection::Profile->new(
               name    => "atp_profile1",
               comment => "add TP profile",
               );

unless ($atp_profile) {
       die("Construct Threat Protection profile failed: ",
               Infoblox::status_code() . ":" . Infoblox::status_detail());
}

#Add the Threat Protection profile object to the Infoblox appliance through a session
$session->add($atp_profile)
       or die("Add Threat Protection profile failed: ",
                       $session->status_code() . ":" . $session->status_detail());
print "Threat Protection profile object added to server successfully\n";
```

#Search for a specific Threat Protection profile

```
#Search all Threat Protection profiles that matches "atp"
my @retrieved_objs = $session->search(
               object => "Infoblox::Grid::ThreatProtection::Profile",
               name   => 'atp'
               );
my $object = $retrieved_objs[0];

unless ($object) {
       die("Search Threat Protection profile failed: ",
               $session->status_code() . ":" . $session->status_detail());
}
print "Search Threat Protection profile object found at least 1 matching entry\n";
```

#Get and modify a Threat Protection profile

```
#Get a Threat Protection profile through the session
my @retrieved_objs = $session->get(
               object => "Infoblox::Grid::ThreatProtection::Profile",
               name   => "atp_profile1"
               );
my $object = $retrieved_objs[0];

unless ($object) {
       die("Get Threat Protection profile failed: ",
               $session->status_code() . ":" . $session->status_detail());
}
print "Get Threat Protection profile object found at least 1 matching entry\n";

#Modify one of the attributes of the specified Threat Protection profile
$object->comment("this is new comment");

#Apply the changes
$session->modify($object)
       or die("Modify Threat Protection profile failed: ",
               $session->status_code() . ":" . $session->status_detail());
print "Threat Protection profile object modified successfully \n";
```

#Remove a Threat Protection profile

```
#Get Threat Protection profile through the session
my @retrieved_objs = $session->get(
    object => "Infoblox::Grid::ThreatProtection::Profile",
    name   => "atp_profile1"
);
my $object = $retrieved_objs[0];
unless ($object) {
    die("Get Threat Protection profile failed: ",
        $session->status_code() . ":" . $session->status_detail());
}
print "Get Threat Protection profile object found at least 1 matching entry\n";

#Submit the object for removal
$session->remove($object)
       or die("Remove Threat Protection profile failed: ",
               $session->status_code() . ":" . $session->status_detail());
print "Threat Protection profile object removed successfully \n";

####PROGRAM ENDS####
```

AUTHOR

Infoblox Inc. http://www.infoblox.com/

SEE ALSO

Infoblox::Session, Infoblox::Session->add(), Infoblox::Session->get(), Infoblox::Session->modify(), Infoblox::Session->remove(), Infoblox::Session->search(), Infoblox::Grid::ThreatProtection::Ruleset, Infoblox::Grid::Member::ThreatProtection, Infoblox::Grid::ThreatProtection::Profile,

---

COPYRIGHT

Copyright (c) 2017 Infoblox Inc.

Infoblox::Grid::ThreatProtection::Profile::Rule - Threat Protection profile rule object.

- NAME
- DESCRIPTION
- SESSION METHODS
  - Infoblox::Session->get( )
  - Infoblox::Session->modify( )
  - Infoblox::Session->search( )
- METHODS
  - config( )
  - disable( )
  - override_config( )
  - override_disable( )
  - profile( )
  - rule( )
  - sid( )
- SAMPLE CODE
- AUTHOR
- SEE ALSO
- COPYRIGHT

NAME

Infoblox::Grid::ThreatProtection::Profile::Rule - Threat Protection profile rule object.

DESCRIPTION

This object provides information about the Threat Protection profile rule settings.

SESSION METHODS

The following functions can be applied to a Infoblox::Grid::ThreatProtection::Profile::Rule object.

Infoblox::Session->get( )

Use this method to retrieve all the matching objects from the Infoblox appliance. See Infoblox::Session->get() for parameters and return values.

Key References

```
Apply the following attributes to get a specific Threat Protection profile rule object.

 sid     - Optional. The Rule ID.
 rule    - Optional. The Threat Protection rule object.
 profile - Optional. The Threat Protection profile object.
```

Example

```
my @retrieved_objs = $session->get(
    'object' => 'Infoblox::Grid::ThreatProtection::Profile::Rule',
    'sid'    => '211111'
);
```

Infoblox::Session->modify( )

Use this method to modify an object in the Infoblox appliance. See Infoblox::Session->modify() for parameters and return values.

Example

```
 my $rule = $session->search('object' => 'Infoblox::Grid::ThreatProtection::Rule', 'sid' =>
'211111');
 my $object = $session->search('object' => 'Infoblox::Grid::ThreatProtection::Profile::Rule', 'rule'
=> $rule);

 my $conf = $object->config();
 $conf->action('PASS');

 # Use this method to modify attributes of object.
 $object->disable('true');
 $object->override_disable('true');
 $object->config($conf);
```

```
$object->override_config('true');
# Submit modification
my $response = $session->modify( $object );
```

Infoblox::Session->search( )

Use this method to search for objects in the Infoblox appliance. See Infoblox::Session->search() for parameters and return values.

Key References

```
Apply the following attributes to get a specific Grid Threat Protection::Rule object.

 sid     - Optional. The Rule ID.
 rule    - Optional. The Threat Protection rule object.
 profile - Optional. The Threat Protection profile object.
```

Example

```
#Retrieve the Threat Protection rule from appliance
my $rule = $session->search(
    'object' => 'Infoblox::Grid::ThreatProtection::Rule',
    'sid'    => '211111',
);

my $object = $session->search(
    'object' => 'Infoblox::Grid::ThreatProtection::Profile::Rule',
    'rule'   => $rule,
);
```

METHODS

This object provides information about the Threat Protection profile rule settings.

config( )

Use this method to set or retrieve the rule configuration information.

Setting this method to a defined value implicitly sets the override_config method to 'true'. Setting the parameter to undefined causes the appliance to use the Grid default and automatically resets the override_config attribute to 'false'.

Note that when config is set to a defined value and override_config is set to 'false', the last operation takes precedence. Thus the sequence $object->config($value); $object->override_config('false'); will set override_config to 'false', and the sequence $object->override_config('false'); $object->config($value); will result in override_config='true'.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

Valid value is reference on Infoblox::Grid::ThreatProtection::RuleConfig object.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get config value
my $value = $object->config();

#Modify config value
$value->log_severity('INFORMATIONAL');
$object->config($value);
```

disable( )

Use this method to set or indicate whether the Threat Protection profile rule is disabled.

Setting this method to a defined value implicitly sets the override_disable method to 'true'. Setting the parameter to undefined causes the appliance to use the Grid default and automatically resets the override_disable attribute to 'false'.

Note that when disable is set to a defined value and override_disable is set to 'false', the last operation takes precedence. Thus the sequence $object->disable('true'); $object->override_disable('false'); will set override_disable to 'false', and the sequence $object->override_disable('false'); $object->disable('true'); will result in override_disable='true'.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

Specify 'true' to disable Threat Protection profile rule and 'false' to enable it.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get disable value
my $value = $object->disable();

#Modify disable value
$object->disable('true');
```

override_config( )

The override_config attribute controls whether the current config value is used for the Threat Protection profile rule instead of the Grid default.

The override_config attribute can be specified explicitly. It is also set implicitly when config is set to a defined value.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

Specify 'true' to set the override_config flag or 'false' to deactivate/unset it. The default value is 'false'.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get override_config value
my $value = $object->override_config();

#Modify override_config value
$object->override_config('true');
```

override_disable( )

The override_disable attribute controls whether the current disable value is used for the Threat Protection profile rule instead of the Grid default.

The override_disable attribute can be specified explicitly. It is also set implicitly when disable is set to a defined value.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

Specify 'true' to set the override_disable flag or 'false' to deactivate/unset it. The default value is 'false'.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get override_disable value
my $value = $object->override_disable();

#Modify override_disable value
$object->override_disable('true');
```

profile( )

Use this method to retrieve the Threat Protection profile object. This is a read-only attribute.

Omit the parameter to retrieve the attribute value.

Parameter

None

Returns

The method returns the attribute value, an Infoblox::Grid::ThreatProtection::Profile object.

Example

```
#Get profile value
my $value = $object->profile();
```

rule( )

Use this method to retrieve the Threat Protection rule object. This is a read-only attribute.

Omit the parameter to retrieve the attribute value.

Parameter

None

Returns

The method returns the attribute value, an Infoblox::Grid::ThreatProtection::Rule object.

Example

```
#Get rule value
my $value = $object->rule();
```

sid( )

Use this method to retrieve the Rule ID. This is a read-only attribute.

Omit the parameter to retrieve the attribute value.

Parameter

None

Returns

The method returns the attribute value.

Example

```
#Get sid value
my $value = $object->sid();
```

SAMPLE CODE

The following sample code demonstrates the different functions that can be applied to an object, such as add, search, modify, and remove. This sample also includes error handling for the operations.

#Preparation prior to a Threat Protection profile rule insertion

```
#PROGRAM STARTS: Include all the modules that will be used
use strict;
use Infoblox;

#Create a session to the Infoblox appliance

my $session = Infoblox::Session->new(
              master   => "192.168.1.2", #appliance host ip
              username => "admin",       #appliance user login
              password => "infoblox"     #appliance password
              );

unless ($session) {
       die("Construct session failed: ",
              Infoblox::status_code() . ":" . Infoblox::status_detail());
}
print "Session created successfully\n";

#Create a Threat Protection profile
```

```
my $atp_profile = Infoblox::Grid::ThreatProtection::Profile->new(
                name    => "atp_profile1",
                comment => "add TP profile",
                );

unless ($atp_profile) {
       die("Construct Threat Protection profile failed: ",
                Infoblox::status_code() . ":" . Infoblox::status_detail());
}

#Add the Threat Protection profile object to the Infoblox appliance through a session
$session->add($atp_profile)
       or die("Add Threat Protection profile failed: ",
                        $session->status_code() . ":" . $session->status_detail());
print "Threat Protection profile object added to server successfully\n";
```

#Search for a specific Threat Protection profile rule

```
#Search all Threat Protection profile rules that matches "atp_profile1" profile
my @retrieved_objs = $session->search(
                object  => "Infoblox::Grid::ThreatProtection::Profile::Rule",
                profile => $atp_profile,
                );
my $object = $retrieved_objs[0];

unless ($object) {
       die("Search Threat Protection profile rule failed: ",
                $session->status_code() . ":" . $session->status_detail());
}
print "Search Threat Protection profile rule object found at least 1 matching entry\n";
```

#Get and modify a Threat Protection profile rule

```
#Get a Threat Protection profile rule through the session
my @retrieved_objs = $session->get(
                object  => "Infoblox::Grid::ThreatProtection::Profile::Rule",
                profile => $atp_profile,
                );
my $object = $retrieved_objs[0];

unless ($object) {
       die("Get Threat Protection profile rule failed: ",
                $session->status_code() . ":" . $session->status_detail());
}
print "Get Threat Protection profile rule object found at least 1 matching entry\n";

#Modify one of the attributes of the specified Threat Protection profile rule
$object->disable("true");

#Apply the changes
$session->modify($object)
       or die("Modify Threat Protection profile rule failed: ",
                $session->status_code() . ":" . $session->status_detail());
print "Threat Protection profile rule object modified successfully \n";

####PROGRAM ENDS####
```

AUTHOR

Infoblox Inc. http://www.infoblox.com/

SEE ALSO

Infoblox::Session, Infoblox::Session->get(), Infoblox::Session->modify(), Infoblox::Session->search(), Infoblox::Grid::ThreatProtection::Ruleset, Infoblox::Grid::ThreatProtection::RuleConfig, Infoblox::Grid::ThreatProtection::Rule, Infoblox::Grid::ThreatProtection::Profile,

---

COPYRIGHT

Copyright (c) 2017 Infoblox Inc.

Infoblox::Grid::ThreatProtection::Rule - Grid Threat Protection Rule object.

- NAME
- DESCRIPTION
- CONSTRUCTOR
- SESSION METHODS
  - Infoblox::Session->add( )
  - Infoblox::Session->get( )
  - Infoblox::Session->modify( )
  - Infoblox::Session->remove( )
  - Infoblox::Session->ruleset( )
  - Infoblox::Session->search( )
- METHODS
  - allowed_actions( )
  - category( )
  - comment( )
  - config( )
  - description( )
  - disable( )
  - is_factory_reset_enabled( )
  - name( )
  - sid( )
  - template( )
  - type( )
- AUTHOR
- SEE ALSO
- COPYRIGHT

NAME

Infoblox::Grid::ThreatProtection::Rule - Grid Threat Protection Rule object.

DESCRIPTION

This object provides information about the Grid Threat protection rule settings.

CONSTRUCTOR

```
my $object = Infoblox::Grid::ThreatProtection::Rule->new(
    'comment'     => $string                                    # Optional
    'disable'     => 'true' | 'false',                          # Optional / Default is 'false'
    'config'      => $config,                                   # Required
    'template'    => $template,                                 # Required
);
```

SESSION METHODS

The following functions can be applied to a Grid::ThreatProtection::Rule object.

Infoblox::Session->add( )

Use this method to add an object to the Infoblox appliance. See Infoblox::Session->add() for parameters and return values.

Example

```
    #Get Rule template Object
    my $template = $session->search('object' => 'Infoblox::Grid::ThreatProtection::RuleTemplate', 'name'
  => 'Whitelist source IPv6 address');
    my $config = $template->default_config();
    my $params = $config->params();
    $params->[0]->value('2001::1');
    #Construct an object
    my $custom_rule = Infoblox::Grid::ThreatProtection::Rule->new(
                      'template'    => $template,
                      'comment'     => 'Test Custom Rule',
                      'config'      => $config,
                      'disable'     => 'false');

    #Submit for addition
    my $response = $session->add( $custom_rule );
```

Infoblox::Session->get( )

Use this method to retrieve all the matching objects from the Infoblox appliance. See Infoblox::Session->get() for parameters and return values.

Key References

```
Apply the following attributes to get a specific Grid Threat Protection::Rule object.
 sid    - Optional. The Rule ID.
 name   - Optional. The name of Threat Protection Rule.
```

Example

```
my @retrieved_objs = $session->get(
    'object' => 'Infoblox::Grid::ThreatProtection::Rule',
    'name' => 'Whitelist source IPv6 address',
);
```

Infoblox::Session->modify( )

Use this method to modify an object in the Infoblox appliance. See Infoblox::Session->modify() for parameters and return values.

Example

```
my $modify_rule_template = $session->search('object' =>
'Infoblox::Grid::ThreatProtection::RuleTemplate', 'name' => 'Whitelist source IP address');
my $config = $modify_rule_template->default_config();
$conf->action('PASS');
# Use this method to modify attributes of object.
$retrieved_objs->comment('Comment of Custom Rule');
$retrieved_objs->disable('true');
$retrieved_objs->config($config);
$retrieved_objs->template($modify_rule_template);
# Submit modification
my $response = $session->modify( $retrieved_objs );
```

Infoblox::Session->remove( )

Use this method to remove an object from the Infoblox appliance. See Infoblox::Session->remove() for parameters and return values.

To remove a specific object, first use `get()` or `search()` to retrieve the specific object, and then submit this object for removal.

Example

```
#Get the objects
my @retrieved_objs = $session->get(
    'object' => 'Infoblox::Grid::ThreatProtection::Rule',
    'name' => 'Whitelist source IPv6 address',
);
#Find the desired object from the retrieved list.
my $desired_object = $retrieved_objs[0];
# Submit for removal
my $response = $session->remove( $desired_object );
```

Infoblox::Session->ruleset( )

Use this method to retrieve the ruleset. See Infoblox::Session->add() for parameters and return values.

Example

```
#Get the object
my $rule = $self->{'session'}->search(
    'object' => 'Infoblox::Grid::ThreatProtection::Rule',
    'sid' => '111111',
);

# Retrieve ruleset
my $ruleset = $rule->ruleset();
```

Infoblox::Session->search( )

Use this method to search for objects in the Infoblox appliance. See Infoblox::Session->search() for parameters and return values.

Key References

```
Apply the following attributes to get a specific Grid Threat Protection::Rule object.
```

```
sid    - Optional. The Rule ID.
name   - Optional. The name of Threat Protection Rule.
```

Example

```
my @retrieved_objs = $session->search(
    'object' => 'Infoblox::Grid::ThreatProtection::Rule',
    'name' = 'Whitelist source IP network';
);
```

METHODS

This section describes all the methods that you can use to configure and retrieve values of the Threat Protection Rule.

allowed_actions( )

Use this method to retrieve the allowed actions for the rule. This is a read-only attribute.

Parameter

None

Returns

The method returns the attribute value.

Example

```
# Get attribute value
my $value = $object->allowed_actions();
```

category( )

Use this method to retrieve reference to the rule category. This is a read-only attribute.

Parameter

None

Returns

The method returns the attribute value.

Example

```
# Get attribute value
my $value = $object->category();
```

comment( )

Use this method to set or retrieve user editable comments.

Parameter

Comments in string format.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
# Get attribute value
my $value = $object->comment();
# Modify attribute value
$object->comment('New comment');
```

config( )

Use this method to set or retrieve information about the rule configuration.

This method is required for creating custom rules. The server will set all undefined members of the object to include the 'params' array from the template.

Parameter

Valid value is reference on Infoblox::Grid::ThreatProtection::RuleConfig object.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
# Get attribute value
my $value = $object->config();
$value->log_severity('INFORMATIONAL');
# Modify attribute value
$object->config($value);
```

description( )

Use this method to retrieve the description for the rule. This is a read-only attribute.

Parameter

None

Returns

The method returns the attribute value.

Example

```
# Get attribute value
my $value = $object->description();
```

disable( )

Use this method to set or indicate whether the rule is enabled or disabled for the Grid.

Parameter

Valid value is "true" or "false". The default value is "false".

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
# Get attribute value
my $value = $object->disable();
# Modify attribute value
$object->disable('true');
```

is_factory_reset_enabled( )

Use this method to retrieve the flag that indicates whether the factory reset of this object is enabled or disabled. This is a read-only attribute.

Parameter

None

Returns

The method returns the attribute value.

Example

```
# Get attribute value
my $value = $object->is_factory_reset_enabled();
```

name( )

Use this method to retrieve the rule name. This is a read-only attribute.

Parameter

None

Returns

The method returns the attribute value.

Example

```
# Get attribute value
my $value = $object->name();
```

sid( )

Use this method to retrieve the Rule ID. This is a read-only attribute.

Parameter

None

Returns

The method returns the attribute value.

Example

```
# Get attribute value
my $value = $object->sid();
```

template( )

Use this method to set or retrieve reference to the template object.

Parameter

Valid value is reference on Infoblox::Grid::ThreatProtection::RuleTemplate object.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
# Get attribute value
my $value = $object->template();
# Get Rule Template Object.
my $modify_rule_template = $session->search('object' =>
'Infoblox::Grid::ThreatProtection::RuleTemplate', 'name' => 'Whitelist source IP network');
$object->template($modify_rule_template);
# Submit modification
my $response = $session->modify( $object );
```

type( )

Use this method to retrieve the rule type. This is a read-only attribute.

Parameter

None

Returns

The method returns the attribute value.

Example

```
# Get attribute value
my $value = $object->type();
```

AUTHOR

Infoblox Inc. http://www.infoblox.com/

SEE ALSO

Infoblox::Session, Infoblox::Grid::ThreatProtection, Infoblox::Grid::Member::ThreatProtection::Rule, Infoblox::Grid::ThreatProtection::RuleConfig, Infoblox::Grid::ThreatProtection::RuleParam

---

COPYRIGHT

Copyright (c) 2017 Infoblox Inc.

Infoblox::Grid::ThreatProtection::RuleCategory - The rule category settings.

- NAME
- DESCRIPTION
- SESSION METHODS
  - Infoblox::Session->get( )
  - Infoblox::Session->search( )
- METHODS
  - is_factory_reset_enabled( )
  - name( )
- AUTHOR
- SEE ALSO
- COPYRIGHT

NAME

Infoblox::Grid::ThreatProtection::RuleCategory - The rule category settings.

DESCRIPTION

This object provides information about the rule category settings.

SESSION METHODS

The following functions can be applied to a Grid::ThreatProtection::RuleCategory object.

Infoblox::Session->get( )

Use this method to retrieve all the matching objects from the Infoblox appliance. See Infoblox::Session->get() for parameters and return values.

Key References

```
Apply the following attributes to get a specific Rule Category object.
 name   - Optional. The name of the Rule Category object.
```

Example

```
my @retrieved_objs = $session->get(
    'object' => 'Infoblox::Grid::ThreatProtection::RuleCategory',
    'name' => 'Whitelist source IP address',
);
```

Infoblox::Session->search( )

Use this method to search for objects in the Infoblox appliance. See Infoblox::Session->search() for parameters and return values.

Key References

```
Apply the following attributes to get a specific Rule Category object.
 name   - Optional. The name of the Rule Category object.
```

Example

```
my @retrieved_objs = $session->search(
    'object' => 'Infoblox::Grid::ThreatProtection::RuleCategory',
    'name' => 'Whitelist source IP address',
);
```

METHODS

This section describes all the methods that you can use to set or retrieve the attribute values of the struct.

is_factory_reset_enabled( )

Use this method to retrieve the flag that indicates whether the factory reset of this object is enabled or disabled. This is a read-only attribute.

Parameter

None

Returns

The method returns the attribute value.

Example

```
# Get attribute value
my $value = $object->is_factory_reset_enabled();
```

name( )

Use this method to retrieve the category name. This is a read-only attribute.

Parameter

None

Returns

The method returns the attribute value.

Example

```
# Get attribute value
my $value = $object->name();
```

AUTHOR

Infoblox Inc. http://www.infoblox.com/

SEE ALSO

Infoblox::Session, Infoblox::Grid::ThreatProtection, Infoblox::Grid::ThreatProtection::Rule

---

COPYRIGHT

Copyright (c) 2017 Infoblox Inc.

Infoblox::Grid::ThreatProtection::RuleConfig - The rule configuration.

- NAME
- DESCRIPTION
- CONSTRUCTOR
- SESSION METHODS
- METHODS
  - action( )
  - log_severity( )
  - params( )
- AUTHOR
- SEE ALSO
- COPYRIGHT

NAME

Infoblox::Grid::ThreatProtection::RuleConfig - The rule configuration.

DESCRIPTION

This is an in-memory struct.

CONSTRUCTOR

```
my $object = Infoblox::Grid::ThreatProtection::RuleConfig->new(
    'action'          => 'ALERT' | 'PASS' | 'DROP',                              # optional
    'log_severity'    => 'CRITICAL' | 'MAJOR' | 'WARNING' | 'INFORMATIONAL',     # optional
    'params'          => [$param1, $param2, ...],                                # required
);
```

SESSION METHODS

The object does not support any session methods.

METHODS

This section describes all the methods that you can use to set or retrieve the attribute values of the struct.

action( )

Use this method to set or retrieve the action of the rule.

Parameter

Valid value is 'ALERT', 'PASS' or 'DROP'.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
# Get attribute value
my $value = $object->action();
# Modify attribute value
$object->action('DROP');
```

log_severity( )

Use this method to set or retrieve the severity for the rule log.

Parameter

Valid value is 'CRITICAL', 'MAJOR', 'WARNING' or 'INFORMATIONAL'.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
# Get attribute value
my $value = $object->log_severity();
# Modify attribute value
$object->log_severity('WARNING');
```

params( )

Use this method to set or retrieve rule parameters.

Parameter

Valid value is an array that contains Infoblox::Grid::ThreatProtection::RuleParam structs.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
# Get attribute value
my $value = $object->params();

# create expected RuleParam struct
my $param = Infoblox::Grid::ThreatProtection::RuleParam->new(
    'name' => 'SRC_IP',
    'value' => 'abcd::/64',
);

# Modify attribute value
$object->params([$param]);
```

AUTHOR

Infoblox Inc. http://www.infoblox.com/

SEE ALSO

Infoblox::Grid::ThreatProtection::Rule, Infoblox::Grid::ThreatProtection::RuleParam

---

COPYRIGHT

Copyright (c) 2017 Infoblox Inc.

Infoblox::Grid::ThreatProtection::RuleParam - A Grid Threat Protection Rule parameter object.

- NAME
- DESCRIPTION
- CONSTRUCTOR
- SESSION METHODS
- MODULE METHODS
  - Infoblox::Grid::ThreatProtection::RuleConfig->params( )
- METHODS
  - description( )
  - min( )
  - max( )
  - name( )
  - read_only( )
  - syntax( )
  - value( )
- AUTHOR
- SEE ALSO
- COPYRIGHT

NAME

Infoblox::Grid::ThreatProtection::RuleParam - A Grid Threat Protection Rule parameter object.

DESCRIPTION

The Grid Threat Protection Rule parameter object represents information about Threat Protection Rule Config parameters.

CONSTRUCTOR

```
my $object = Infoblox::Grid::ThreatProtection::RuleParam->new(
    'name'  => $string,     # Required
    'value' => $string,     # Required
);
```

SESSION METHODS

The object does not support any session methods.

MODULE METHODS

This section describes all the functions that you can apply to an ThreatProtection RuleParam object.

Infoblox::Grid::ThreatProtection::RuleConfig->params( )

Use this method to set or retrieve rule parameters. See Infoblox::Grid::ThreatProtection::RuleConfig->params()

Example

```
my $rule_param = Infoblox::Grid::ThreatProtection::RuleParam->new(
  'name'  => 'SRC_IP',
  'value' => 'abcd::',
);

my $rule_config = Infoblox::Grid::ThreatProtection::RuleConfig->new(
  'action'              => 'PASS',
  'log_severity'        => 'MAJOR',
  'params'              => [$rule_param]);
```

METHODS

This section describes all the methods that you can use to set or retrieve the attribute values of the struct.

description( )

Use this method to retrieve the parameter description. This is a read-only attribute.

Parameter

None

Returns

The method returns the attribute value.

Example

```
# Get attribute value
my $value = $object->description();
```

min( )

Use this method to retrieve the minimum value of the parameter if the parameter syntax is UNSIGNED_INT. This is a read-only attribute.

Parameter

None

Returns

The method returns the attribute value.

Example

```
# Get attribute value
my $value = $object->min();
```

max( )

Use this method to retrieve the maximum value of the parameter if the parameter syntax is UNSIGNED_INT. This is a read-only attribute.

Parameter

None

Returns

The method returns the attribute value.

Example

```
# Get attribute value
my $value = $object->max();
```

name( )

Use this method to set or retrieve the parameter name.

Parameter

The parameter name in string format.

Returns

The method returns the attribute value.

Example

```
# Get attribute value
my $value = $object->name();
# Modify attribute value
$object->name('SRC_IP');
```

read_only( )

Use this method to indicate if the parameter is read-only. This is a read-only attribute.

Parameter

None

Returns

The method returns the attribute value.

Example

```
# Get attribute value
my $value = $object->read_only();
```

syntax( )

Use this method to retrieve the parameter syntax. This is a read-only attribute.

Parameter

None

Returns

The method returns the attribute value.

Example

```
# Get attribute value
my $value = $object->syntax();
```

value( )

Use this method to set or retrieve the parameter value.

Parameter

The parameter value in string format.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
# Get attribute value
my $value = $object->value();
# Modify attribute value
$object->value('15');
```

AUTHOR

Infoblox Inc. http://www.infoblox.com/

SEE ALSO

Infoblox::Grid::ThreatProtection::Rule, Infoblox::Grid::ThreatProtection::RuleConfig

---

COPYRIGHT

Copyright (c) 2017 Infoblox Inc.

Infoblox::Grid::ThreatProtection::Ruleset - A Grid Threat Protection Ruleset object.

- NAME
- DESCRIPTION
- SESSION METHODS
  - Infoblox::Session->get( )
  - Infoblox::Session->search( )
  - Infoblox::Session->modify( )
  - Infoblox::Session->remove( )
- METHODS
  - add_type( )
  - added_time( )
  - comment( )
  - do_not_delete( )
  - is_factory_reset_enabled( )
  - used_by( )
  - version( )
- AUTHOR
- SEE ALSO
- COPYRIGHT

NAME

Infoblox::Grid::ThreatProtection::Ruleset - A Grid Threat Protection Ruleset object.

DESCRIPTION

This object represents the Grid Threat Protection ruleset.

SESSION METHODS

This section describes all the methods in an Infoblox::Session module that you can apply to a Grid ruleset object.

Infoblox::Session->get( )

Use this method to retrieve all the ruleset objects from the Infoblox appliance. See Infoblox::Session->get() for parameters and return values.

Example

```
my @retrieved_objs = $session->get(
    'object' => 'Infoblox::Grid::ThreatProtection::Ruleset',
);
```

Infoblox::Session->search( )

Use this method to retrieve all the ruleset objects from the Infoblox appliance. See Infoblox::Session->search() for parameters and return values.

Example

```
my @retrieved_objs = $session->search(
    'object' => 'Infoblox::Grid::ThreatProtection::Ruleset',
);
```

Infoblox::Session->modify( )

Use this method to modify an object in the Infoblox appliance. See Infoblox::Session->modify() for parameters and return values.

If you have applied an access control list to an object, you must restart services after you update the access control list for the changes to take effect. See Infoblox::Session->restart() method.

Example

```
# Use this method to modify comments for the Grid member.
$ruleset->comment("This is modified object");
# Submit modification
my $response = $session->modify( $ruleset );
```

Infoblox::Session->remove( )

Use this method to remove an object from the Infoblox appliance. See Infoblox::Session->remove() for parameters and return values.

To remove a specific object, first use `get()` or `search()` to retrieve the specific object, and then submit this object for removal.

Example

```
# Submit for removal
my $response = $session->remove( $desired_ruleset );
```

METHODS

This section describes all the methods that you can use to set or retrieve the attribute values of the struct.

add_type( )

Use this method to retrieve how the ruleset was added. This is a read-only attribute.

Parameter

None

Returns

The method returns the attribute value, which is either "AUTOMATIC" or "MANUAL".

Example

```
# Get attribute value
my $value = $object->add_type();
```

added_time( )

Use this method to retrieve the time when the ruleset was added. This is a read-only attribute.

Parameter

None

Returns

The method returns the attribute value, the number of seconds that have elapsed since January 1st, 1970 UTC.

Example

```
# Get attribute value
my $value = $object->added_time();
```

comment( )

Use this method to set or retrieve a descriptive comment.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

Desired comment in string format with a maximum of 256 bytes.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get comment
my $comment = $object->comment();
#Modify comment
$object->comment("Modified comment");
```

do_not_delete( )

Use this method to set or retrieve the do_not_delete attribute of the ruleset. If set to "true", the ruleset will not be deleted during a ruleset upgrade.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

None

Returns

The method returns the attribute value, which is "true" or "false".

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get do_not_delete from the ruleset
my $do_not_delete = $object->do_not_delete();
#Modify do_not_delete
$object->do_not_delete("true");
```

is_factory_reset_enabled( )

Use this method to retrieve the flag that indicates whether the factory reset of this object is enabled or disabled. This is a read-only attribute.

Parameter

None

Returns

The method returns the attribute value.

Example

```
# Get attribute value
my $value = $object->is_factory_reset_enabled();
```

used_by( )

Use this method to retrieve the users of the ruleset. This is a read-only attribute.

Parameter

None

Returns

The method returns the attribute value, which is a list of strings.

Example

```
# Get attribute value
my $value = $object->used_by();
```

version( )

Use this method to retrieve the ruleset version. This is a read-only attribute.

Parameter

None

Returns

The method returns the attribute value.

Example

```
# Get attribute value
my $value = $object->version();
```

AUTHOR

Infoblox Inc. http://www.infoblox.com/

SEE ALSO

Infoblox::Grid::ThreatProtection, Infoblox::Grid::Member::ThreatProtection

---

COPYRIGHT

Copyright (c) 2017 Infoblox Inc.

Infoblox::Grid::ThreatProtection::RuleTemplate - Grid Threat Protection Rule Template object.

- NAME
- DESCRIPTION
- CONSTRUCTOR
- SESSION METHODS
  - Infoblox::Session->get( )
  - Infoblox::Session->search( )
- METHODS
  - allowed_actions( )
  - category( )
  - default_config( )
  - description( )
  - name( )
  - ruleset( )
  - sid( )
  - versions( )
- AUTHOR
- SEE ALSO
- COPYRIGHT

NAME

Infoblox::Grid::ThreatProtection::RuleTemplate - Grid Threat Protection Rule Template object.

DESCRIPTION

This object provides information about the Grid Threat protection rule template settings.

CONSTRUCTOR

```
my $object = Infoblox::Grid::ThreatProtection::RuleTemplate->new(
   'name' => $string,        # Required
);
```

SESSION METHODS

The following functions can be applied to a Grid::ThreatProtection::RuleTemplate object.

Infoblox::Session->get( )

Use this method to retrieve all the matching objects from the Infoblox appliance. See Infoblox::Session->get() for parameters and return values.

Key References

```
Apply the following attributes to get a specific Grid Threat Protection RuleTemplate object.
 sid    - Optional. The Rule ID.
 name   - Optional. The name of the Threat protection rule template.
```

Example

```
my @retrieved_objs = $session->get(
    'object' => 'Infoblox::Grid::ThreatProtection::RuleTemplate',
    'name' => 'Whitelist source IP network',
);
```

Infoblox::Session->search( )

Use this method to search for objects in the Infoblox appliance. See Infoblox::Session->search() for parameters and return values.

Key References

```
Apply the following attributes to get a specific Grid Threat Protection::RuleTemplate object.
 sid     - Optional. The Rule ID.
 name    - Optional. The name of the Threat protection rule template.
 ruleset - Optional. An
L<Infoblox::Grid::ThreatProtection::Ruleset|Infoblox::Grid::ThreatProtection::Ruleset> object
representing a reference to the ruleset object.
```

Example

```
my @retrieved_objs = $session->search(
    'object' => 'Infoblox::Grid::ThreatProtection::RuleTemplate',
    'name' => 'Whitelist source IP network',
);
```

METHODS

allowed_actions( )

Use this method to retrieve allowed actions for the rule template. This is a read-only attribute.

Parameter

None

Returns

The method returns the attribute value.

Example

```
# Get attribute value
my $value = $object->allowed_actions();
```

category( )

Use this method to retrieve reference to the rule category. This is a read-only attribute.

Parameter

None

Returns

The method returns the attribute value.

Example

```
# Get attribute value
my $value = $object->category();
```

default_config( )

Use this method to retrieve default rule configuration information. This is a read-only attribute.

Parameter

None

Returns

The method returns the attribute value.

Example

```
# Get attribute value
my $value = $object->default_config();
```

description( )

Use this method to retrieve the description for the rule template. This is a read-only attribute.

Parameter

None

Returns

The method returns the attribute value.

Example

```
# Get attribute value
my $value = $object->description();
```

name( )

Use this method to set or retrieve the name for the rule template. This is a read-only attribute if the object was retrieved from the server.

Parameter

The name in string format.

Returns

The method returns the attribute value.

Example

```
# Get attribute value
my $value = $object->name();
# Modify attribute value
$object->name('Whitelist source IP network');
```

ruleset( )

Use this method to retrieve a reference to the ruleset object. This is a read-only attribute.

Parameter

None

Returns

An Infoblox::Grid::ThreatProtection::Ruleset object.

Example

```
#Get ruleset value
my $value = $object->ruleset();
```

sid( )

Use this method to retrieve the Rule ID. This is a read-only attribute.

Parameter

None

Returns

The method returns the attribute value.

Example

```
# Get attribute value
my $value = $object->sid();
```

versions( )

Use this method to retrieve information about the rule version. This is a read-only attribute.

Parameter

None

Returns

The method returns the attribute value.

Example

```
# Get attribute value
my $value = $object->versions();
```

AUTHOR

Infoblox Inc. http://www.infoblox.com/

SEE ALSO

Infoblox::Session, Infoblox::Grid::ThreatProtection, Infoblox::Grid::ThreatProtection::Rule, Infoblox::Grid::ThreatProtection::RuleConfig, Infoblox::Grid::ThreatProtection::RuleParam, Infoblox::Grid::ThreatProtection::Ruleset

---

COPYRIGHT

Copyright (c) 2017 Infoblox Inc.

Infoblox::Grid::ThreatProtection::StatInfo - Statistics information.

- NAME
- DESCRIPTION
- SESSION METHODS
- METHODS
  - critical( )
  - informational( )
  - major( )
  - timestamp( )
  - warning( )
- AUTHOR
- SEE ALSO
- COPYRIGHT

NAME

Infoblox::Grid::ThreatProtection::StatInfo - Statistics information.

DESCRIPTION

This is an in-memory struct.

SESSION METHODS

The object does not support any session methods.

METHODS

This section describes all the methods that you can use to set or retrieve the attribute values of the struct.

critical( )

Use this method to retrieve the number of critical events. This is a read-only attribute.

Parameter

None

Returns

The method returns the attribute value.

Example

```
# Get attribute value
my $value = $object->critical();
```

informational( )

Use this method to retrieve the number of informational events. This is a read-only attribute.

Parameter

None

Returns

The method returns the attribute value.

Example

```
# Get attribute value
my $value = $object->informational();
```

major( )

Use this method to retrieve the number of major events. This is a read-only attribute.

Parameter

None

Returns

The method returns the attribute value.

Example

```
# Get attribute value
my $value = $object->major();
```

timestamp( )

Use this method to retrieve the timestamp when the following data is being collected. This is a read-only attribute.

Parameter

None

Returns

The method returns the attribute value.

Example

```
# Get attribute value
my $value = $object->timestamp();
```

warning( )

Use this method to retrieve the number of warning events. This is a read-only attribute.

Parameter

None

Returns

The method returns the attribute value.

Example

```
# Get attribute value
my $value = $object->warning();
```

AUTHOR

Infoblox Inc. http://www.infoblox.com/

SEE ALSO

Infoblox::Grid::ThreatProtection::Rule, Infoblox::Grid::ThreatProtection::Statistics

---

COPYRIGHT

Copyright (c) 2017 Infoblox Inc.

Infoblox::Grid::ThreatProtection::Statistics - Threat protection statistics.

- NAME
- DESCRIPTION
- SESSION METHODS
  - Infoblox::Session->get( )
  - Infoblox::Session->search( )
- METHODS
  - stat_infos( )
  - grid_member( )
- AUTHOR
- SEE ALSO
- COPYRIGHT

NAME

Infoblox::Grid::ThreatProtection::Statistics - Threat protection statistics.

DESCRIPTION

This object provides information about Threat protection statistics.

SESSION METHODS

The following functions can be applied to a Grid::ThreatProtection::Statistics object.

Infoblox::Session->get( )

Use this method to retrieve all the matching objects from the Infoblox appliance. See Infoblox::Session->get() for parameters and return values.

Key References

```
Apply the following attributes to get a specific Threat Protection Statistics object.
 grid_member   - Optional. The Grid member name.
```

Example

```
my @retrieved_objs = $session->get(
    'object'      => 'Infoblox::Grid::ThreatProtection::Statistics',
    'grid_member' => 'member.com',
);
```

Infoblox::Session->search( )

Use this method to search for objects in the Infoblox appliance. See Infoblox::Session->search() for parameters and return values.

Key References

```
Apply the following attributes to get a specific Threat Protection Statistics object.
 grid_member   - Optional. The Grid member name.
```

Example

```
my @retrieved_objs = $session->search(
    'object'      => 'Infoblox::Grid::ThreatProtection::Statistics',
    'grid_member' => 'member.com',
);
```

METHODS

stat_infos( )

Use this method to retrieve event statistics. This is a read-only attribute.

Parameter

None

Returns

The method returns the attribute value that can be an array reference of Infoblox::Grid::ThreatProtection::StatInfo objects.

Example

```
# Get attribute value
my $value = $object->stat_infos();
```

grid_member( )

Use this method to retrieve a Grid member name. This is a read-only attribute.

Parameter

None

Returns

The method returns the attribute value.

Example

```
# Get attribute value
my $value = $object->grid_member();
```

AUTHOR

Infoblox Inc. http://www.infoblox.com/

SEE ALSO

Infoblox::Session, Infoblox::Grid::ThreatProtection, Infoblox::Grid::ThreatProtection::Rule, Infoblox::Grid::ThreatProtection::StatInfo

---

COPYRIGHT

Copyright (c) 2017 Infoblox Inc.

Infoblox::Grid::TimeZone - Grid time zone object.

- NAME
- DESCRIPTION
- CONSTRUCTOR
- SESSION METHODS
  - Infoblox::Session->get( )
  - Infoblox::Session->modify( )
- METHODS
  - ntp_enabled( )
  - time_zone( )
- SAMPLE CODE
- AUTHOR
- SEE ALSO
- COPYRIGHT

NAME

Infoblox::Grid::TimeZone - Grid time zone object.

DESCRIPTION

Grid TimeZone object is used to set the time zone for an Infoblox appliance.

CONSTRUCTOR

```
#Constructor for an Grid Time zone object
my $time_zone = Infoblox::Grid::TimeZone->new (
        time_zone     => $time_zone             #Required
        ntp_enabled   => "true" | "false"       #Optional / Default "false"
    );
```

SESSION METHODS

This section describes all the methods in Infoblox::Session that you can apply to a grid TimeZone object.

Infoblox::Session->get( )

Use this method to retrieve existing objects from the Infoblox appliance. See Infoblox::Session->get() for parameters and return values.

Key Reference

Apply the following attributes to get a grid TimeZone object:

none.

Example

```
my @result_array = $session->get( "object" => "Infoblox::Grid::TimeZone");
```

Infoblox::Session->modify( )

Use this method to modify objects in the Infoblox appliance. See Infoblox::Session->modify() for parameters and return values.

Example

```
# Use method to modify the time zone.
$object->time_zone( "(UTC - 8:00) Pacific Time (US and Canada), Tijuana" );
$object->ntp_enabled( "false" );
# Submit modification
my $response = $session->modify( $object );
```

METHODS

This section describes all the methods that you can use to configure and retrieve the attribute value of a time zone.

ntp_enabled( )

Use this method to enable/disable NTP (Network Time Protocol).

Parameter

Specify "true" to enable NTP or "false" to disable it. Default value is "false".

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

Example

```
#Get ntp_enabled
my $enable = $object->ntp_enabled();
#Modify ntp_enabled
$object->ntp_enabled("true");
```

time_zone( )

Use this method to set or retrieve a time_zone.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

Desired time in string format. Following are the available Time Zones:

```
'(UTC - 12:00)'
'(UTC - 11:00) Midway Island, Samoa'
'(UTC - 10:00) Hawaii'
'(UTC - 9:00) Alaska'
'(UTC - 8:00) Pacific Time (US and Canada), Tijuana'
'(UTC - 7:00) Arizona'
'(UTC - 7:00) Mountain Time (US and Canada)'
'(UTC - 6:00) Central Time (US and Canada)'
'(UTC - 6:00) Mexico City, Tegucigalpa'
'(UTC - 6:00) Saskatchewan'
'(UTC - 5:00) Bogota, Lima, Quito'
'(UTC - 5:00) Eastern Time (US and Canada)'
'(UTC - 5:00) Indiana (East)'
'(UTC - 4:00) Caracas'
'(UTC - 4:00) Atlantic Time (Canada)'
'(UTC - 4:00) Santiago'
'(UTC - 3:30) Newfoundland'
'(UTC - 3:00) Brasilia'
'(UTC - 3:00) Buenos Aires'
'(UTC - 2:00) Mid-Atlantic'
'(UTC - 1:00) Azores'
'(UTC) Coordinated Universal Time'
'(UTC) Dublin'
'(UTC) Lisbon'
'(UTC) London'
'(UTC + 1:00) Casablanca'
'(UTC + 1:00) Amsterdam, Berlin, Bern, Rome, Stockholm, Vienna'
'(UTC + 1:00) Belgrade, Bratislava, Budapest, Ljubljana, Prague'
'(UTC + 1:00) Brussels, Copenhagen, Madrid, Paris'
'(UTC + 1:00) Sarajevo, Skopje, Sofija, Warsaw, Zagreb'
'(UTC + 2:00) Athens, Vilnius'
'(UTC + 2:00) Bucharest'
'(UTC + 2:00) Cairo'
'(UTC + 2:00) Harare'
'(UTC + 2:00) Helsinki'
'(UTC + 2:00) Jerusalem'
'(UTC + 2:00) Kaliningrad'
'(UTC + 3:00) Baghdad, Istanbul, Kuwait, Minsk, Riyadh'
'(UTC + 3:00) Nairobi'
'(UTC + 3:00) Moscow, St    .   Petersburg, Volgograd'
'(UTC + 3:30) Tehran'
'(UTC + 4:00) Dubai'
'(UTC + 4:00) Baku'
'(UTC + 4:00) Samara'
'(UTC + 4:30) Kabul'
'(UTC + 5:00) Ekaterinburg'
'(UTC + 5:00) Islamabad, Karachi'
'(UTC + 5:30) Bombay, Calcutta, Madras, New Delhi'
'(UTC + 5:30) Colombo'
'(UTC + 6:00) Dhaka'
'(UTC + 6:00) Omsk'
'(UTC + 6:30) Rangoon'
'(UTC + 7:00) Bangkok, Hanoi'
'(UTC + 7:00) Krasnoyarsk'
'(UTC + 8:00) Beijing, Chongqing, Shanghai'
'(UTC + 8:00) Hong Kong'
'(UTC + 8:00) Irkutsk'
'(UTC + 8:00) Perth'
'(UTC + 8:00) Singapore'
'(UTC + 8:00) Taipei'
'(UTC + 9:00) Osaka, Sapporo, Tokyo'
'(UTC + 9:00) Seoul'
'(UTC + 9:00) Yakutsk'
'(UTC + 9:30) Adelaide'
'(UTC + 9:30) Darwin'
'(UTC + 10:00) Brisbane'
```

```
'(UTC + 10:00) Canberra, Sydney'
'(UTC + 10:00) Melbourne, Victoria'
'(UTC + 10:00) Guam'
'(UTC + 10:00) Hobart'
'(UTC + 10:00) Vladivostok'
'(UTC + 11:00) Solomon Islands'
'(UTC + 11:00) Magadan'
'(UTC + 12:00) Anadyr'
'(UTC + 12:00) Auckland'
'(UTC + 12:00) Fiji'
'(UTC + 12:00) Marshall Islands'
```

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

Example

```
#Get time_zone
my $time_zone = $object->time_zone();
#Modify time_zone
$object->time_zone("(UTC - 8:00) Pacific Time (US and Canada), Tijuana");
```

SAMPLE CODE

The following sample code demonstrates different operations that can be applied to an object such as create, modify, and remove an object. This sample code also includes error handling for the operations.

```
#PROGRAM STARTS: Include all the modules that will be used
use strict;
use Infoblox;

#Create a session to the Infoblox appliance
my $session = Infoblox::Session->new(
    master   => "192.168.1.2",
    username => "admin",
    password => "infoblox"
);
unless ($session) {
   die("Construct session failed: ",
       Infoblox::status_code() . ":" . Infoblox::status_detail());
}
print "Session created successfully\n";

my @result_array = $session->get( "object" => "Infoblox::Grid::TimeZone");
if( defined @result_array  and scalar( @result_array ) > 0 )
 {
        my $object = $result_array[0];
        unless ($object) {
                die("Get on Time zone failed: ",
        $session->status_code() . ":" . $session->status_detail());
        }
#Modify time_zone
$object->time_zone("(UTC - 8:00) Pacific Time (US and Canada), Tijuana");
my $enable = $object->ntp_enabled();
$object->ntp_enabled( "true" );
$session->modify($object)
   or die("Modify on Time zone failed: ",
           $session->status_code() . ":" . $session->status_detail());
print "Grid Time zone modified successfully \n";
}
else
{
        print "No  TimeZone Structure found !!!\n";
}

####PROGRAM ENDS####
```

AUTHOR

Infoblox Inc. http://www.infoblox.com/

SEE ALSO

Infoblox::Session, Infoblox::Session->get(), Infoblox::Session->modify()

COPYRIGHT

Copyright (c) 2017 Infoblox Inc.

Infoblox::Grid::UpdatesDownloadMemberConfig - An Updates Download Member Configuration object.

- NAME
- DESCRIPTION
- CONSTRUCTOR
- SESSION METHODS
- MODULE METHODS
  - Infoblox::Grid->updates_download_member_config( )
- METHODS
  - interface( )
  - is_online( )
  - member( )
- AUTHOR
- SEE ALSO
- COPYRIGHT

NAME

Infoblox::Grid::UpdatesDownloadMemberConfig - An Updates Download Member Configuration object.

DESCRIPTION

The Updates Download Member Configuration object provides information and settings for configuring Member that is responsible for updates download.

CONSTRUCTOR

```
my $http_proxy_setting = Infoblox::Grid::UpdatesDownloadMemberConfig->new(
    member    => $string | undef,                  #Required
    interface => 'ANY' | 'VIP' | 'LAN2' | 'MGMT', #Required
);
```

SESSION METHODS

The object does not support any session methods.

MODULE METHODS

The following functions are available to be applied to a Updates Download Member Configuration object.

Infoblox::Grid->updates_download_member_config( )

Use this method to set or retrieve the list of updates download member configurations. See Infoblox::Grid->updates_download_member_config() for parameters andreturn values.

Example

```
#Get updates_download_member_config value
my $updates_download_member_config = $grid->updates_download_member_config();

#Modify updates_download_member_config value
$grid->updates_download_member_config([$conf1, $conf2]);
```

METHODS

This section describes all the methods that you can use to configure and retrieve the attribute values of a Updates Download Member Configuration object.

interface( )

Use this method to set or retrieve the source interface for updates download requests.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

The valid values are 'ANY', 'VIP', 'LAN2' and 'MGMT'.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get interface value
my $interface = $obj->interface();

#Modify interface value
$obj->interface('ANY');
```

is_online( )

Use this method to retrieve the flag that indicates whether the updates download member is on-line or not.

Omit the parameter to retrieve the attribute value.

Parameter

None

Returns

The method returns the attribute value.

Example

```
#Get is_online value
my $is_online = $obj->is_online();
```

member( )

Use this method to set or retrieve the updates download member name.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

The valid value is a desired member's name in a string format or undef for Grid Master.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get member value
my $member = $obj->member();

#Modify member value
$obj->member('desired-member.com');

#Modify member value to Grid Master
$obj->member(undef);
```

AUTHOR

Infoblox Inc. http://www.infoblox.com/

SEE ALSO

Infoblox::Grid, Infoblox::Grid->updates_download_member_config()

COPYRIGHT

Copyright (c) 2017 Infoblox Inc.

Infoblox::Grid::UpgradeGroup - Upgrade Group object.

- NAME
- DESCRIPTION
- CONSTRUCTOR
- SESSION METHODS
  - Infoblox::Session->add( )
  - Infoblox::Session->get( )
  - Infoblox::Session->modify( )
  - Infoblox::Session->remove( )
  - Infoblox::Session->search( )
  - Infoblox::Grid::UpgradeSchedule->groups( )
  - Infoblox::Grid::DistributionSchedule->groups( )
- METHODS
  - comment( )
  - distribution_policy( )
  - distribution_dependent_group( )
  - distribution_time( )
  - members( )
  - name( )
  - time_zone( )
  - upgrade_dependent_group( )
  - upgrade_policy( )
  - upgrade_time( )
- SAMPLE CODE
- AUTHOR
- SEE ALSO
- COPYRIGHT

NAME

Infoblox::Grid::UpgradeGroup - Upgrade Group object.

DESCRIPTION

This object enables the configuration of upgrade groups for the Infoblox Grid.

CONSTRUCTOR

```
 my $group = Infoblox::Grid::UpgradeGroup->new(
         name                          => $string, #Required
         comment                       => $string, #Optional / Default is undefined
         distribution_policy           => "simultaneously" | "sequentially", #Optional / Default is
simultaneously
         upgrade_policy                => "simultaneously" | "sequentially", #Optional / Default is
simultaneously
         distribution_dependent_group  => $string, #Optional / Default is undefined
         upgrade_dependent_group       => $string, #Optional / Default is undefined
         distribution_time             => $timestamp, #See below
         upgrade_time                  => $timestamp, #See below
         members                       => [$m1, $m2, ...], #Optional / Default is undefined
 );
```

SESSION METHODS

This section describes all the methods in the Infoblox::Session module that you can apply to an Upgrade Group object.

Infoblox::Session->add( )

Use this method to add an object to the Infoblox appliance. See Infoblox::Session->add() for parameters and return values.

Example

```
       my $member1 = Infoblox::Grid::UpgradeGroup::Member->new(
                                                              member => '192.168.1.2',
                                                             );
       unless($member1) {
            die("Construct member failed: ",
                  Infoblox::status_code() . ":" . Infoblox::status_detail());
```

```
}
print "Member object created successfully\n";

#Create the upgrade group object with this member
my $sgu = Infoblox::Grid::UpgradeGroup->new(
                                            name                          => "groupname1",
                                            comment                       => "group name 1 comment",
                                            distribution_policy           => "sequentially",
                                            upgrade_policy                => "simultaneously",
                                            distribution_dependent_group  => 'distgroup1',
                                            upgrade_dependent_group       => 'distgroup2',
                                            distribution_time             => '2020-12-22T11:22:33Z',
                                            upgrade_time                  => '2021-12-22T11:22:33Z',
                                            members                       => [$member1],
                                            );

my $response = $session->add($sgu)
unless($response) {
     die("Add upgrade group failed: ",
           session->status_code() . ":" . session->status_detail());
}
print "Upgrade group created successfully\n";
```

Infoblox::Session->get( )

Use this method to retrieve all the matching objects from the Infoblox appliance. See Infoblox::Session->get() for parameters and return values.

Key References

```
Apply the following attributes to get a specific Upgrade Group object:

 name          - Optional. Group name in string format.
 comment       - Optional. Comment in string format.
```

Example

```
my @retrieved_objs = $session->get(
    object      => "Infoblox::Grid::UpgradeGroup",
    name        => "testgroup" );
```

Infoblox::Session->modify( )

Use this method to modify an object in the Infoblox appliance. See Infoblox::Session->modify() for parameters and return values.

Example

```
# Use this method to modify the comment.
$upgradegroup->comment("This is a modified comment");
# Submit modification
my $response = $session->modify( $upgradegroup );
```

Infoblox::Session->remove( )

Use this method to remove an object from the Infoblox appliance. See Infoblox::Session->remove() for parameters and return values.

To remove a specific object, use `get()` or `search()` to retrieve the specific object first, and then submit this object for removal.

Example

```
# Get the group object
my @retrieved_objs = $session->get(
    object      => "Infoblox::Grid::UpgradeGroup",
    name        => "testgroup" );
# Find the desired object from the retrieved list.
my $desired_group = $retrieved_objs[0];
# Submit for removal
my $response = $session->remove( $desired_group );
```

Infoblox::Session->search( )

Use this method to search for Upgrade Group objects in the Infoblox appliance. See Infoblox::Session->search() for parameters and return values.

Key References

```
Apply the following attributes to search for a specific Upgrade Group object:
```

```
 name          - Optional. Group name in string format (regular expression).
 comment       - Optional. Comment in string format (regular expression).
```

Example

```
# Search for all Upgrade Group objects that start with "test"
my @retrieved_objs = $session->search(
    object      => "Infoblox::Grid::UpgradeGroup",
    name        => "test.*",
);
```

Infoblox::Grid::UpgradeSchedule->groups( )

Use this function to specify the groups that belong to this upgrade schedule. See Infoblox::Grid::UpgradeSchedule->groups() for parameters and return values.

Infoblox::Grid::DistributionSchedule->groups( )

Use this function to specify the groups that belong to this distribution schedule. See Infoblox::Grid::DistributionSchedule->groups() for parameters and return values.

METHODS

This section describes all the methods that you can use to set and retrieve the attribute values of an upgrade group object.

comment( )

Use this method to set or retrieve the comment.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

Comment in string format with a maximum of 256 bytes. The default value is undefined.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get comment
my $comment = $upgradegroup->comment();
#Modify comment
$upgradegroup->comment("descriptive comment");
```

distribution_policy( )

Use this method to set or retrieve the distribution policy of this Upgrade Group.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

The distribution policy. Valid values are "sequentially" and "simultaneously". The default is "simultaneously".

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get distribution_policy
my $distribution_policy = $upgradegroup->distribution_policy();
#Modify distribution_policy
$upgradegroup->distribution_policy("sequentially");
```

distribution_dependent_group( )

Use this method to set or retrieve the distribution dependent group.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

The name of the distribution dependent group.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get distribution_dependent_group
my $distribution_dependent_group = $upgradegroup->distribution_dependent_group();
#Modify distribution_dependent_group
$upgradegroup->distribution_dependent_group("group2");
```

distribution_time( )

Use this method to set or retrieve the distribution time.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Note this member is inserted only if the Upgrade Group is submitted for modification as part of an Infoblox::Grid::DistributionSchedule update, this member will not be changed if the Upgrade Group is submitted for modification as a stand-alone object.

Note that if distribution_dependent_group is set, distribution_time will be ignored.

Parameter

The start time of the distribution in ISO 8601 extended format (e.g. 2008-10-21T10:56:00Z or 2008-10-21T10:56:00.001Z).

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get distribution_time
my $distribution_time = $upgradegroup->distribution_time();
#Modify distribution_time
$upgradegroup->distribution_time("2008-10-21T10:56:00.001Z");
```

members( )

Use this method to set or retrieve the members of this upgrade group.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

Array reference that contains Infoblox::Grid::UpgradeGroup::Member object. The default value is empty.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get members
my $members = $upgradegroup->members();
#Modify members
$upgradegroup->members([$ug1, $ug2, $ug3]);
```

name( )

Use this method to set or retrieve the name of this upgrade group.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

The name in string format.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get name
my $name = $upgradegroup->name();
#Modify name
$upgradegroup->name("group2");
```

time_zone( )

Use this method to retrieve the time_zone for this upgrade group.

Parameter

None.

Returns

The method returns the attribute value.

Example

```
#Get time_zone
my $time_zone = $upgradegroup->time_zone();
```

upgrade_dependent_group( )

Use this method to set or retrieve the upgrade dependent group.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

The name of the upgrade dependent group.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get upgrade_dependent_group
my $upgrade_dependent_group = $upgradegroup->upgrade_dependent_group();
#Modify upgrade_dependent_group
$upgradegroup->upgrade_dependent_group("group2");
```

upgrade_policy( )

Use this method to set or retrieve the upgrade_policy.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

The upgrade policy. Valid values are "sequentially" and "simultaneously". The default is "simultaneously".

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get upgrade_policy
my $upgrade_policy = $upgradegroup->upgrade_policy();
#Modify upgrade_policy
$upgradegroup->upgrade_policy("sequentially");
```

upgrade_time( )

Use this method to set or retrieve the upgrade time.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Note this member is inserted only if the Upgrade Group is submitted for modification as part of an Infoblox::Grid::UpgradeSchedule update, this member will not be changed if the Upgrade Group is submitted for modification as a stand-alone object.

Parameter

The start time of the upgrade in ISO 8601 extended format (e.g. 2008-10-21T10:56:00Z or 2008-10-21T10:56:00.001Z).

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get upgrade_time
my $upgrade_time = $upgradegroup->upgrade_time();
#Modify upgrade_time
$upgradegroup->upgrade_time("2008-10-21T10:56:00.001Z");
```

SAMPLE CODE

The following sample code demonstrates the different functions that can be applied to an object, such as add, search, modify, and remove. This sample also includes error handling for the operations.

#Preparation prior to an Upgrade Group insertion

```
#PROGRAM STARTS: Include all the modules that will be used
use strict;
use Infoblox;

#Create a session to the Infoblox appliance

my $session = Infoblox::Session->new(
               master   => "127.0.0.1", #appliance host ip
               username => "admin",       #appliance user login
               password => "infoblox"     #appliance password
               );

unless ($session) {
       die("Construct session failed: ",
               Infoblox::status_code() . ":" . Infoblox::status_detail());
}
print "Session created successfully\n";
```

#Create a few Upgrade Group objects

```
my $group1 = Infoblox::Grid::UpgradeGroup->new(
                                          name => "group1",
                                         );

unless ($group1) {
       die("Construct group object failed: ",
               Infoblox::status_code() . ":" . Infoblox::status_detail());
}
print "Group object created successfully\n";

my $group2 = Infoblox::Grid::UpgradeGroup->new(
                                          name => "group2",
                                         );

$session->add($group1)
       or die("Add Upgrade Group object failed: ",
                       $session->status_code() . ":" . $session->status_detail());
print "Upgrade Group object added to server successfully\n";

unless ($group2) {
       die("Construct group object failed: ",
               Infoblox::status_code() . ":" . Infoblox::status_detail());
}
print "Group object created successfully\n";

$session->add($group2)
```

```
        or die("Add Upgrade Group object failed: ",
                        $session->status_code() . ":" . $session->status_detail());
print "Upgrade Group object added to server successfully\n";

# Note that this has to be a valid Grid member name
my $member = Infoblox::Grid::UpgradeGroup::Member->new(
                                                        member      => 'grid_member.com',
                                                       );

my $group3 = Infoblox::Grid::UpgradeGroup->new(
                                               name                          => "group3",
                                               comment                       => "descriptive comment",
                                               distribution_policy           => "sequentially",
                                               upgrade_policy                => "simultaneously",
                                               distribution_dependent_group  => 'group1',
                                               upgrade_dependent_group       => 'group2',
                                               distribution_time             => '2020-12-22T11:22:33Z',
                                               upgrade_time                  => '2021-12-22T11:22:33Z',
                                               members                       => [$member],
                                              );

unless ($group3) {
        die("Construct group object failed: ",
                Infoblox::status_code() . ":" . Infoblox::status_detail());
}
print "Group object created successfully\n";

#Add the upgrade group object to the Infoblox appliance through a session
$session->add($group3)
        or die("Add Upgrade Group object failed: ",
                        $session->status_code() . ":" . $session->status_detail());
print "Upgrade Group object added to server successfully\n";
```

#Search for all groups

```
my @retrieved_objs = $session->search(
                object      => "Infoblox::Grid::UpgradeGroup",
                name        => "group.*",
               );
my $object = $retrieved_objs[0];

unless ($object) {
        die("Search Group object failed: ",
                $session->status_code() . ":" . $session->status_detail());
}
print "Search Upgrade Group objects found at least 1 matching entry\n";
```

#Get and modify a Group object

```
#Get the object from an Infoblox appliance through a session
my @retrieved_objs = $session->get(
            object      => "Infoblox::Grid::UpgradeGroup",
            name        => "group3",
);
my $object = $retrieved_objs[0];

unless ($object) {
        die("Get Group object failed: ",
                $session->status_code() . ":" . $session->status_detail());
}
print "Get Upgrade Group object found at least 1 matching entry\n";

#Modify the upgrade policy of the Group object
$object->upgrade_policy("sequentially");

#Apply the changes.
$session->modify($object)
        or die("Modify Group object failed: ",
                $session->status_code() . ":" . $session->status_detail());
print "Group object modified successfully \n";
```

#Remove the Group object

```
#Get the Group object through the session
my @retrieved_objs = $session->get(
            object      => "Infoblox::Grid::UpgradeGroup",
            name        => "group3",
```

```
);
my $object = $retrieved_objs[0];
unless ($object) {
    die("Get Group object failed: ",
        $session->status_code() . ":" . $session->status_detail());
}
print "Get Group object found at least 1 matching entry\n";

#Submit the object for removal
$session->remove($object)
       or die("Remove Group object failed: ",
                $session->status_code() . ":" . $session->status_detail());
print "Upgrade Group object removed successfully \n";
```

#Remove the other created objects

```
$session->remove($group2)
       or die("Remove Group object failed: ",
                $session->status_code() . ":" . $session->status_detail());

$session->remove($group1)
       or die("Remove Group object failed: ",
                $session->status_code() . ":" . $session->status_detail());

####PROGRAM ENDS####
```

AUTHOR

Infoblox Inc. http://www.infoblox.com/

SEE ALSO

Infoblox::Grid::UpgradeGroup::Member,Infoblox::Session->add(), Infoblox::Session->get(), Infoblox::Session->modify(), Infoblox::Session->remove(), Infoblox::Session->search(), Infoblox::Session

---

COPYRIGHT

Copyright (c) 2017 Infoblox Inc.

Infoblox::Grid::UpgradeGroup::Member - Upgrade Group Member object

- NAME
- DESCRIPTION
- CONSTRUCTOR
- MODULE METHODS
  - Infoblox::Grid::UpgradeGroup->members( )
- METHODS
  - member( )
  - time_zone( )
- AUTHOR
- SEE ALSO
- COPYRIGHT

NAME

Infoblox::Grid::UpgradeGroup::Member - Upgrade Group Member object

DESCRIPTION

This object is used to represent Upgrade Group Grid members.

CONSTRUCTOR

```
my $member = Infoblox::Grid::UpgradeGroup::Member->new(
    member          => $string, # Required, name of the member
);
```

MODULE METHODS

The following functions are available to apply to an Upgrade Group Member object.

Infoblox::Grid::UpgradeGroup->members( )

Use this function to specify the members that belong to this upgrade group. See Infoblox::Grid::UpgradeGroup->members() for parameters and return values.

Example

```
my $member1 = Infoblox::Grid::UpgradeGroup::Member->new(
                                                        member => 'hostname.com',
                                                       );
unless($member1) {
     die("Construct member failed: ",
           Infoblox::status_code() . ":" . Infoblox::status_detail());
}
print "Member object created successfully\n";

#Create the upgrade group object with this member
my $sgu = Infoblox::Grid::UpgradeGroup->new(
                                            name                          => "groupname1",
                                            comment                       => "group name 1 comment",
                                            distribution_policy           => "sequentially",
                                            upgrade_policy                => "simultaneously",
                                            distribution_dependent_group  => 'distgroup1',
                                            upgrade_dependent_group       => 'distgroup2',
                                            distribution_time             => '2020-12-22T11:22:33Z',
                                            upgrade_time                  => '2021-12-22T11:22:33Z',
                                            members                       => [$member1],
                                            );

my $response = $session->add($sgu)
unless($response) {
     die("Add upgrade group failed: ",
           session->status_code() . ":" . session->status_detail());
}
print "Upgrade group created successfully\n";
```

METHODS

This section describes all the methods that you can use to set and retrieve the attribute values of an Upgrade Group Member object.

member( )

Use this method to set or retrieve the name of the member.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

The name of the grid member.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get member
my $member = $member->member();
#Modify member
$member->member("192.168.1.3");
```

time_zone( )

Use this method to retrieve the time zone of the member.

Parameter

none

Returns

The method returns the attribute value.

Example

```
#Get time_zone
my $time_zone = $member->time_zone();
```

AUTHOR

Infoblox Inc. http://www.infoblox.com/

SEE ALSO

Infoblox::Grid::UpgradeGroup

COPYRIGHT

Copyright (c) 2017 Infoblox Inc.

Infoblox::Grid::UpgradeSchedule - Information about the Grid's upgrade schedule.

- NAME
- DESCRIPTION
- CONSTRUCTOR
- SESSION METHODS
  - Infoblox::Session->get( )
  - Infoblox::Session->modify( )
- METHODS
  - active( )
  - groups( )
  - start_time( )
  - time_zone( )
- AUTHOR
- SEE ALSO
- COPYRIGHT

NAME

Infoblox::Grid::UpgradeSchedule - Information about the Grid's upgrade schedule.

DESCRIPTION

The Upgrade Schedule object is used to set and view the upgrade schedule of an Infoblox Grid.

CONSTRUCTOR

The Infoblox::Grid::UpgradeSchedule object is automatically generated. It does not require manual construction.

SESSION METHODS

This section describes all the methods in an Infoblox::Session that can be applied to an Upgrade Schedule object.

Infoblox::Session->get( )

Use this method to retrieve the existing objects from an Infoblox appliance. See Infoblox::Session->get() for parameters and return values.

Example

```
my $schedule = $session->get(object => "Infoblox::Grid::UpgradeSchedule");
```

Infoblox::Session->modify( )

Use this method to modify objects in the Infoblox appliance. See Infoblox::Session->modify() for parameters and return values.

Example

```
# Submit modification
my $response = $session->modify( $schedule );
```

METHODS

This section describes all the methods that can be used to set or retrieve the attribute values of an Upgrade Schedule object.

active( )

Use this method to set or retrieve the "active" flag.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

Specify "true" to set the active flag or "false" to deactivate/unset it. The default value is "false".

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get active
my $active = $schedule->active();
#Modify active
$schedule->active("true");
```

groups( )

Use this method to set or retrieve the list of groups affected by this upgrade schedule.

Parameter

Array reference that contains each Infoblox::Grid::UpgradeGroup object.

Note that the members of this list must have their upgrade_time member set in order for the modification to succeed. This means that it is possible to retrieve an upgrade schedule but not be allowed to modify it without setting the upgrade_time values for the groups.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

Example

```
#Get the group list
my $grouplist = $schedule->groups();
#Modify it
$grid->groups([$ug1, $ug2, $ug3 ]);
```

start_time( )

Use this method to set or retrieve the start time of the upgrade.

Parameter

The start time of the upgrade in ISO 8601 extended format (e.g. 2008-10-21T10:56:00Z or 2008-10-21T10:56:00.001Z).

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get start_time
my $start_time = $schedule->start_time();
#Modify start_time
$schedule->start_time("2020-12-21T11:22:33Z");
```

time_zone( )

Use this method to set or retrieve the time zone of the upgrade schedule.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

The UTC string that represents the time zone. For example "(UTC - 6:00) Central Time (US and Canada)". Default value is undefined.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get time_zone
my $time_zone = $schedule->time_zone();
#Modify time_zone
$schedule->time_zone("(UTC - 6:00) Central Time (US and Canada)");
```

AUTHOR

Infoblox Inc. http://www.infoblox.com/

SEE ALSO

Infoblox::Session, Infoblox::Session->get()

COPYRIGHT

Copyright (c) 2017 Infoblox Inc.

Infoblox::Grid::UpgradeStatus - Information about the Grid's upgrade status.

- NAME
- DESCRIPTION
- CONSTRUCTOR
- SESSION METHODS
  - Infoblox::Session->get( )
- METHODS
  - allow_distribution( )
  - allow_upgrade( )
  - allow_upgrade_cancel( )
  - allow_upgrade_pause( )
  - allow_upgrade_resume( )
  - allow_upgrade_scheduling( )
  - allow_upgrade_test( )
  - allow_upload( )
  - distribution_schedule_active( )
  - grid_state( )
  - group_state( )
  - message( )
  - status_time( )
  - upgrade_schedule_active( )
  - upgrade_state( )
  - upgrade_test_status( )
- AUTHOR
- SEE ALSO
- COPYRIGHT

NAME

Infoblox::Grid::UpgradeStatus - Information about the Grid's upgrade status.

DESCRIPTION

The Upgrade Status object is used to view the upgrade status of an Infoblox Grid.

CONSTRUCTOR

The Infoblox::Grid::UpgradeStatus object is automatically generated. It does not require manual construction.

SESSION METHODS

This section describes all the methods in an Infoblox::Session that can be applied to an Upgrade Status object.

Infoblox::Session->get( )

Use this method to retrieve the existing objects from an Infoblox appliance. See Infoblox::Session->get() for parameters and return values.

Example

```
my @statistics = $session->get(object => "Infoblox::Grid::UpgradeStatus");
```

METHODS

This section describes all the methods that can be used to retrieve the attribute values of an Upgrade Status object.

allow_distribution( )

Use this method to determine if distribution is allowed for the Grid.

Parameter

none

Returns

The method returns the attribute value. Valid values are "true" or "false".

Example

```
#Get allow_distribution
my $allow_distribution = $upgstatus->allow_distribution();
```

allow_upgrade( )

Use this method to determine if upgrade is allowed for the Grid.

Parameter

none

Returns

The method returns the attribute value. Valid values are "true" or "false".

Example

```
#Get allow_upgrade
my $allow_upgrade = $upgstatus->allow_upgrade();
```

allow_upgrade_cancel( )

Use this method to determine if the Grid is allowed to cancel an upgrade.

Parameter

none

Returns

The method returns the attribute value. Valid values are "true" or "false".

Example

```
#Get allow_upgrade_cancel
my $allow_upgrade_cancel = $upgstatus->allow_upgrade_cancel();
```

allow_upgrade_pause( )

Use this method to determine if the Grid is allowed to pause an upgrade.

Parameter

none

Returns

The method returns the attribute value. Valid values are "true" or "false".

Example

```
#Get allow_upgrade_pause
my $allow_upgrade_pause = $upgstatus->allow_upgrade_pause();
```

allow_upgrade_resume( )

Use this method to determine if the Grid is allowed to resume an upgrade.

Parameter

none

Returns

The method returns the attribute value. Valid values are "true" or "false".

Example

```
#Get allow_upgrade_resume
my $allow_upgrade_resume = $upgstatus->allow_upgrade_resume();
```

allow_upgrade_scheduling( )

Use this method to determine if the Grid is allowed to schedule an upgrade.

Parameter

none

Returns

The method returns the attribute value. Valid values are "true" or "false".

Example

```
#Get allow_upgrade_scheduling
my $allow_upgrade_scheduling = $upgstatus->allow_upgrade_scheduling();
```

allow_upgrade_test( )

Use this method to determine if the Grid is allowed to test an upgrade.

Parameter

none

Returns

The method returns the attribute value. Valid values are "true" or "false".

Example

```
#Get allow_upgrade_test
my $allow_upgrade_test = $upgstatus->allow_upgrade_test();
```

allow_upload( )

Use this method to determine if the Grid is allowed to upload a build.

Parameter

none

Returns

The method returns the attribute value. Valid values are "true" or "false".

Example

```
#Get allow_upload
my $allow_upload = $upgstatus->allow_upload();
```

distribution_schedule_active( )

Use this method to determine if the distribution schedule is active for the Grid.

Parameter

none

Returns

The method returns the attribute value. Valid values are "true" or "false".

Example

```
#Get distribution_schedule_active
my $distribution_schedule_active = $upgstatus->distribution_schedule_active();
```

grid_state( )

Use this method to retrieve the Grid state.

Parameter

none

Returns

The method returns the attribute value. Valid values are

```
"NONE", "DISTRIBUTING_COMPLETE", "DISTRIBUTING_ENDED", "DISTRIBUTING_PAUSED",
"DISTRIBUTING", "DISTRIBUTING_FAILED", "DEFAULT", "UPGRADING", "REVERT",
"TEST_UPGRADING", "UPGRADING_COMPLETE", "UPGRADING_FAILED",
"REVERTING_COMPLETE", "DOWNGRADING_COMPLETE", "UPLOADED",
```

```
"DOWNGRADING_FAILED", "UPGRADING_PAUSED"
```

Example

```
#Get grid_state
my $grid_state = $upgstatus->grid_state();
```

group_state( )

Use this method to retrieve the group state.

Parameter

none

Returns

The method returns the attribute value. Valid values are

```
"GROUP_NONE", "GROUP_DISTRIBUTING_WAITING", "GROUP_DISTRIBUTING",
"GROUP_DISTRIBUTING_COMPLETE", "GROUP_UPGRADING_WAITING", "GROUP_UPGRADING",
"GROUP_UPGRADING_COMPLETE", "GROUP_DISTRIBUTING_FAILED"
```

Example

```
#Get group_state
my $group_state = $upgstatus->group_state();
```

message( )

Use this method to retrieve the grid message.

Parameter

none

Returns

The method returns the attribute value.

Example

```
#Get message
my $message = $upgstatus->message();
```

status_time( )

Use this method to retrieve the status time.

Parameter

none

Returns

The method returns the attribute value.

Example

```
#Get status_time
my $status_time = $upgstatus->status_time();
```

upgrade_schedule_active( )

Use this method to determine if the upgrade schedule is active.

Parameter

none

Returns

The method returns the attribute value. Valid values are "true" or "false".

Example

```
#Get upgrade_schedule_active
my $upgrade_schedule_active = $upgstatus->upgrade_schedule_active();
```

upgrade_state( )

Use this method to retrieve the upgrade state of the Grid.

Parameter

none

Returns

The method returns the attribute value. Valid values are "NONE" or "PROGRESSING".

Example

```
#Get upgrade_state
my $upgrade_state = $upgstatus->upgrade_state();
```

upgrade_test_status( )

Use this method to retrieve the upgrade test status of the Grid.

Parameter

none

Returns

The method returns the attribute value. Valid values are "NONE", "PROGRESSING", "COMPLETED" or "FAILED".

Example

```
#Get upgrade_test_status
my $upgrade_test_status = $upgstatus->upgrade_test_status();
```

AUTHOR

Infoblox Inc. http://www.infoblox.com/

SEE ALSO

Infoblox::Session, Infoblox::Session->get()

COPYRIGHT

Copyright (c) 2017 Infoblox Inc.

Infoblox::Grid::VDiscoveryTask - the vDiscovery task.

- NAME
- DESCRIPTION
- CONSTRUCTOR
- SESSION METHODS
  - Infoblox::Session->add( )
  - Infoblox::Session->get( )
  - Infoblox::Session->remove( )
  - Infoblox::Session->search( )
  - Infoblox::Session->modify( )
- METHODS
  - name( )
  - member( )
  - scheduled_run( )
  - state( )
  - enabled( )
  - comment( )
  - public_network_view_mapping_policy( )
  - public_network_view( )
  - private_network_view_mapping_policy( )
  - private_network_view( )
  - driver_type( )
  - fqdn_or_ip( )
  - protocol( )
  - port( )
  - username( )
  - password( )
  - state_msg( )
  - last_run_time( )
  - update_metadata( )
  - merge_data( )
  - auto_consolidate_managed_tenant( )
  - auto_consolidate_managed_vm( )
  - auto_consolidate_cloud_ea( )
  - auto_create_dns_record( )
  - auto_create_dns_record_type( )
  - auto_create_dns_hostname_template( )
  - vdiscovery_control( )
  - credentials_type( )
  - allow_unsecured_connection( )
  - update_dns_view_public_ip( )
  - dns_view_public_ip( )
  - update_dns_view_private_ip( )
  - dns_view_private_ip( )
  - identity_version( )
  - domain_name( )
- AUTHOR
- SEE ALSO
- COPYRIGHT

NAME

Infoblox::Grid::VDiscoveryTask - the vDiscovery task.

DESCRIPTION

The object provides information about the vDiscovery task.

CONSTRUCTOR

```
my $object = Infoblox::Grid::VDiscoveryTask->new(
    'name'                                => $string,
    'member'                              => $string,
    'public_network_view_mapping_policy' => 'DIRECT' | 'AUTO_CREATE',
    'private_network_view_mapping_policy'=> 'DIRECT' | 'AUTO_CREATE',
    'driver_type'                         => 'AWS' | 'AZURE' | 'OPENSTACK' | 'VMWARE',
    'fqdn_or_ip'                          => $string,
    'protocol'                            => 'HTTP' | 'HTTPS',
    'port'                                => $uint,
    'username'                            => $string,
    'password'                            => $string,
    'update_metadata'                     => 'true' | 'false',
    'merge_data'                          => 'true' | 'false',
    'auto_consolidate_managed_tenant'     => 'true' | 'false',
```

```
    'auto_consolidate_managed_vm'        => 'true' | 'false',
    'auto_consolidate_cloud_ea '         => 'true' | 'false',
    'auto_create_dns_record'             => 'true' | 'false',
    'auto_create_dns_record_type'        => 'HOST_RECORD' | 'A_PTR_RECORD',
    'auto_create_dns_hostname_template'  => $string,
    'credentials_type'                   => 'DIRECT' | 'INDIRECT',
    'allow_unsecured_connection'         => 'true' | 'false',
    'update_dns_view_public_ip'          => 'true' | 'false',
    'dns_view_public_ip'                 => $string,
    'update_dns_view_private_ip'         => 'true' | 'false',
    'dns_view_private_ip'                => $string,
    'identity_version'                   => 'KEYSTONE_V2' | 'KEYSTONE_V3',
    'domain_name'                        => $string,
);
```

SESSION METHODS

This section describes all the methods in Infoblox::Session that you can apply to a vDiscovery task object.

Infoblox::Session->add( )

Use this method to add an object to the Infoblox appliance. See Infoblox::Session->add() for parameters and return values.

Example

```
#Construct an object
my $task = Infoblox::Grid::VDiscoveryTask->new(
   name             => "taskname",
   comment          => "descriptive comment",
);
# Submit for addition
my $response = $session->add( $task );
```

Infoblox::Session->get( )

Use this method to retrieve all the matching objects from the Infoblox appliance. See Infoblox::Session->get() for parameters and return values.

Key References

```
Apply the following attributes to get a specific grid object:

  name                  - Optional. A name of the cloud discovery task in string format.
  member                - Optional. A host name of the member the cloud discovery task is assigned toin string
format.
  fqdn_or_ip            - Optional. A FQDN or IP of the cloud management platform in string format.
  protocol              - Optional. A connection protocol used for connecting to the cloud management platform
in string format.
  port                  - Optional. A connection port used for connecting to the cloud management platform in
string format.
  public_network_view   - Optional. A network view name for public IPs in string format.
  private_network_view  - Optional. A network view name for private IPs in string format.
  extattrs              - Optional. A hash reference containing extensible attributes.
  extensible_attributes - Optional. A hash reference containing extensible attributes.
  comment               - Optional. A comment in string format.
```

Example

```
my @retrieved_objs = $session->get(
    object => "Infoblox::Grid::VDiscoveryTask",
    name   => "taskname",
);
```

Infoblox::Session->remove( )

Use this method to submit an object for removal from the Infoblox appliance. See Infoblox::Session->remove() for parameters and return values.

To remove a specific object, first use `get()` or `search()` to retrieve the object, and then submit this object for removal.

Example

```
# Get the objects with the same name
my @retrieved_objs = $session->get(
    object => "Infoblox::Grid::VDiscoveryTask",
    name   => "taskname",
);

# Find the desired object on the retrieved list.
my $desired_task = $retrieved_objs[0];
# Submit for removal
my $response = $session->remove( $desired_task );
```

Infoblox::Session->search( )

Use this method to search for objects in the Infoblox appliance. See Infoblox::Session->search() for parameters and return values.

Key References

```
 Apply the following attributes to search for a task object:

  name                  - Optional. A name of the cloud discovery task in string format.
  member                - Optional. A host name of the member the cloud discovery task is assigned toin string
format.
  fqdn_or_ip            - Optional. A FQDN or IP of the cloud management platform in string format.
  protocol              - Optional. A connection protocol used for connecting to the cloud management platform
in string format.
  port                  - Optional. A connection port used for connecting to the cloud management platform in
string format.
  public_network_view   - Optional. A network view name for public IPs in string format.
  private_network_view  - Optional. A network view name for private IPs in string format.
  extattrs              - Optional. A hash reference containing extensible attributes.
  extensible_attributes - Optional. A hash reference containing extensible attributes.
  comment               - Optional. A comment in string format.
```

For more information about searching extensible attributes, see Infoblox::MasterGrid::ExtensibleAttributeDef/Searching Extensible Attributes.

Example

```
# search for all map objects objects that have "sub" in their name
my @retrieved_objs = $session->search(
    object => "Infoblox::Grid::VDiscoveryTask",
    name   => "sub.*",
);
```

Infoblox::Session->modify( )

Use this method to modify objects in the Infoblox appliance. See Infoblox::Session->modify() for parameters and return values.

Example

```
# Use this method to modify the name
$task->comment("some other comment");
# Submit modification
my $response = $session->modify( $task );
```

METHODS

This section describes all the methods that you can use to set or retrieve the attribute values of the discovered data object.

name( )

Use this method to set or retrieve the name of the vDiscovery task.

Parameter

The the name of the vDiscovery task in string format. The name uniquely identify a vDiscovery task.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
# Get attribute value
my $value = $object->name();
# Modify attribute value
$object->name('name');
```

member( )

Use this method to set or retrieve the host name of the member to which the vDiscovery task is assigned.

Parameter

The host name of the member (in string format) to which the vDiscovery task is assigned.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
# Get attribute value
my $value = $object->member();
# Modify attribute value
$object->member('member');
```

scheduled_run( )

Use this method to set or retrieve the schedule setting for the vDiscovery task.

Parameter

Valid value is an Infoblox::Grid::ScheduleSetting object.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
# Get attribute value
my $value = $object->scheduled_run();
# Modify attribute value
$object->scheduled_run($schedule);
```

state( )

Use this method to retrieve the current state of the vDiscovery task. This is a read-only attribute.

Parameter

None

Returns

Valid values are 'IDLE' (waiting), 'READY' (ready run), 'RUNNING' (the task is running), 'COMPLETE' (successfully completes discovery), 'COLLECTION_COMPLETE' (successfully completed collection), 'WARNING' (discovery completed with errors), 'ERROR' (fails to discover), 'CANCEL_PENDING' (cancelling equals pending). The default value is 'IDLE'.

Example

```
# Get attribute value
my $value = $object->state();
```

enabled( )

Use this method to set or retrieve the flag used to control whether the vDiscovery task is running or not.

Parameter

Specify "true" to enable port scanning or "false" to disable it.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
# Get attribute value
my $value = $object->enabled();
# Modify attribute value
$object->enabled('true');
```

comment( )

Use this method to set or retrieve the comment of the task.

Parameter

The comment of the task in string format.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
# Get attribute value
my $value = $object->comment();
# Modify attribute value
$object->comment('comment');
```

public_network_view_mapping_policy( )

Use this method to set or retrieve the mapping policy for the network view for public IPs in discovered data.

Parameter

Map the policy using a specified network view ('DIRECT') or map the policy using the network view resolved from the tenant ID ('AUTO_CREATE').

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
# Get attribute value
my $value = $object->public_network_view_mapping_policy();
# Modify attribute value
$object->public_network_view_mapping_policy('DIRECT');
```

public_network_view( )

Use this method to set or retrieve the name of the network view for public IPs.

Parameter

The name of the network view for public IPs in string format.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
# Get attribute value
my $value = $object->public_network_view();
# Modify attribute value
$object->public_network_view('public_network_view');
```

private_network_view_mapping_policy( )

Use this method to set or retrieve the mapping policy for the network view for private IPs in discovered data.

Parameter

Map the policy using a specified network view ('DIRECT') or map the policy using the network view resolved from the tenant ID ('AUTO_CREATE').

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
# Get attribute value
my $value = $object->private_network_view_mapping_policy();
# Modify attribute value
$object->private_network_view_mapping_policy('DIRECT');
```

private_network_view( )

Use this method to set or retrieve the name of the network view for private IPs.

Parameter

The name of the network view for private IPs in string format.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
# Get attribute value
my $value = $object->private_network_view();
# Modify attribute value
$object->private_network_view('private_network_view');
```

driver_type( )

Use this method to set or retrieve the type of discovery driver that applies to the vDiscovery task.

Parameter

The type of discovery driver that applies to the vDiscovery task in string format. Valid value is 'AWS' (Amazon Web Service), 'AZURE'(Microsoft Azure), OPENSTACK' (OpenStack) or 'VMWARE' (VMWare).

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
# Get attribute value
my $value = $object->driver_type();
# Modify attribute value
$object->driver_type('VMWARE');
```

fqdn_or_ip( )

Use this method to set or retrieve the FQDN or IP of the Cloud Management Platform.

Parameter

The FQDN or IP of the Cloud Management Platform in string format.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
# Get attribute value
my $value = $object->fqdn_or_ip();
# Modify attribute value
$object->fqdn_or_ip('fqdn_or_ip');
```

protocol( )

Use this method to set or retrieve the connection protocol used for connecting to the Cloud Management Platform.

Parameter

The connection protocol used for connecting to the Cloud Management Platform in string format. Valid value is 'HTTP' or 'HTTPS'.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
# Get attribute value
my $value = $object->protocol();
# Modify attribute value
$object->protocol('HTTP');
```

port( )

Use this method to set or retrieve the connection port used for connecting to the Cloud Management Platform.

Parameter

The connection port used for connecting to the Cloud Management Platform in string format.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
# Get attribute value
my $value = $object->port();
# Modify attribute value
$object->port('port');
```

username( )

Use this method to set or retrieve the username used for connecting to the Cloud Management Platform. This attribute is mandatory if credentials_type is "DIRECT".

Parameter

The username used for connecting to the Cloud Management Platform in string format.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
# Get attribute value
my $value = $object->username();
# Modify attribute value
$object->username('username');
```

password( )

Use this method to set the password used for connecting to the Cloud Management Platform. This is a write-only attribute. This attribute is mandatory if credentials_type is "DIRECT".

Parameter

The password used for connecting to the Cloud Management Platform in string format.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

Example

```
# Modify attribute value
$object->password('password');
```

state_msg( )

Use this method to retrieve the state message of the complete discovery process. This is a read-only attribute.

Parameter

None

Returns

The method returns the attribute value.

Example

```
# Get attribute value
my $value = $object->state_msg();
```

last_run_time( )

Use this method to set or retrieve the timestamp of the last vDiscovery task.

Parameter

None

Returns

The method returns the attribute value.

Example

```
# Get attribute value
my $value = $object->last_run_time();
```

update_metadata( )

Use this method to control whether to update the metadata as a result of the vDiscovery or not.

Parameter

Specify "true" to update the metadata as a result of the vDiscovery or "false" to not update it.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
# Get attribute value
my $value = $object->update_metadata();
# Modify attribute value
$object->update_metadata('true');
```

merge_data( )

Use this method to control whether to replace the old data with the new one or not.

Parameter

Specify "true" to replace the old data with the new one or "false" to not replace it.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
# Get attribute value
my $value = $object->merge_data();
# Modify attribute value
$object->merge_data('true');
```

auto_consolidate_managed_tenant( )

Use this method to control whether to replace managed tenant with discovered tenant data.

Parameter

Specify "true" to replace managed tenant with discovered tenant data or "false" to not replace it.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
# Get attribute value
my $value = $object->auto_consolidate_managed_tenant();
# Modify attribute value
$object->auto_consolidate_managed_tenant('true');
```

auto_consolidate_managed_vm( )

Use this method to control whether to replace managed virtual machine with discovered virtual machine data.

Parameter

Specify "true" to replace managed virtual machine with discovered virtual machine data or "false" to not replace it.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
# Get attribute value
my $value = $object->auto_consolidate_managed_vm();
# Modify attribute value
$object->auto_consolidate_managed_vm('true');
```

auto_consolidate_cloud_ea( )

Use this method to control whether to insert or update cloud EAs with discovered data.

Parameter

Specify "true" to insert or update cloud EAs with discovered data or "false" to not do it.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
# Get attribute value
my $value = $object->auto_consolidate_cloud_ea();
# Modify attribute value
$object->auto_consolidate_cloud_ea('true');
```

auto_create_dns_record( )

Use this method to set or retrieve the flag that indicates whether the automatic creation and update of DNS records using discovered data is enabled or disabled.

Parameter

Specify 'true' to enable automatic creation and update of DNS records with discovered data or 'false' to disable it. The default value is 'false'.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
# Get attribute value
my $value = $object->auto_create_dns_record();
# Modify attribute value
$object->auto_create_dns_record('false');
```

auto_create_dns_record_type( )

Use this method to set or retrieve the type of record to create if the auto-creation of DNS records is enabled.

Parameter

The type of record to create if the auto-creation of DNS records is enabled. Valid values are 'HOST_RECORD' (host record) or 'A_PTR_RECORD' (A (AAAA) record and ptr record combination). Default is 'HOST_RECORD'.

Parameter

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
# Get attribute value
my $value = $object->auto_create_dns_record_type();
# Modify attribute value
$object->auto_create_dns_record_type('HOST_RECORD');
```

auto_create_dns_hostname_template( )

Use this method to set or retrieve the template string used to generate host names.

Parameter

Template string used to generate host names.

Note that using double quotes may lead to interpolation of variables. Instead, use single quotes or encapsulated '$' characters.

Parameter

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
# Get attribute value
my $value = $object->auto_create_dns_hostname_template();
# Modify attribute value
$object->auto_create_dns_hostname_template('${vm_name}.test.com');
```

vdiscovery_control( )

Use this method to start a task or to cancel a task.

Parameter

action - Action being requested in string format. Support starting ('START') an IDLE vDiscovery task or cancelling ('CANCEL') a RUNNING vDiscovery task. Valid value is 'START' or 'CANCEL'.

Returns

The method returns "true" if the action has been performed successfully , and returns "false" when the actions fails.

Example

```
my $res = $cdiscovery->vdiscovery_control('action' => 'START');
```

credentials_type( )

Use this method to set or retrieve the type of credentials used for connecting to the Cloud Management Platform.

Parameter

The credentials type used for connecting to the Cloud Management Platform in string format. Valid values are 'DIRECT' or 'INDIRECT'. Default is 'DIRECT'.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
# Get attribute value
my $value = $object->credentials_type();
# Modify attribute value
$object->credentials_type('INDIRECT');
```

allow_unsecured_connection( )

Use this method to set or retrieve the flag that indicates whether the remote SSL certificate is validated or not. If set to 'true', the certificate will not be validated.

Parameter

Specify 'true' to disable the validation of the remote SSL certificate or 'false' to enable it. The default value is 'false'.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
# Get attribute value
my $value = $object->allow_unsecured_connection();
# Modify attribute value
$object->allow_unsecured_connection('false');
```

update_dns_view_public_ip( )

Use this method to set or retrieve the flag that indicates whether the appliance should use a specific DNS view for public IPs.

Parameter

Specify 'true' to use a specific DNS view for public IPs or 'false' to not use.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
# Get attribute value
my $value = $object->update_dns_view_public_ip();
# Modify attribute value
$object->update_dns_view_public_ip('false');
```

dns_view_public_ip( )

The DNS view name for public IPs. If you configure this option, you must also set update_dns_view_public_ip to true. Otherwise, set update_dns_view_public_ip to false.

Parameter

The valid value is DNS view name in string format.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
# Get attribute value
my $value = $object->dns_view_public_ip();
# Modify attribute value
$object->dns_view_public_ip('public_dns_view');
```

update_dns_view_private_ip( )

Use this method to set or retrieve the flag that indicates whether the appliance should use a specific DNS view for private IPs.

Parameter

Specify 'true' to use a specific DNS view for private IPs or 'false' to not use.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
# Get attribute value
my $value = $object->update_dns_view_private_ip();
# Modify attribute value
$object->update_dns_view_private_ip('false');
```

dns_view_private_ip( )

The DNS view name for private IPs. If you configure this option, you must also set update_dns_view_private_ip to true. Otherwise, set update_dns_view_private_ip to false.

Parameter

The valid value is DNS view name in string format.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
# Get attribute value
my $value = $object->dns_view_private_ip();
# Modify attribute value
$object->dns_view_private_ip('private_dns_view');
```

identity_version( )

Use this method to set or retrieve the OpenStack identity service version.

Parameter

The OpenStack identity service version in string format. Valid value is 'KEYSTONE_V2', 'KEYSTONE_V3'. Default value is 'KEYSTONE_V2'.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
# Get attribute value
my $value = $object->identity_version();
# Modify attribute value
$object->identity_version('KEYSTONE_V3');
```

domain_name( )

Use this method to set or retrieve the name of the domain to use with Keystone v3.

Parameter

The name of the domain to use with Keystone v3. Must be set if 'identity_version' set to 'KEYSTONE_V3'.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
# Get attribute value
my $value = $object->domain_name();
# Modify attribute value
$object->domain_name('Default');
```

AUTHOR

Infoblox Inc. http://www.infoblox.com/

SEE ALSO

Infoblox::IPAM::DiscoveryTask::VServer,Infoblox::DHCP::Member,Infoblox::DHCP::Network,Infoblox::IPAM::TCPPort,Infoblox::Grid::ScheduleSetting

---

COPYRIGHT

Copyright (c) 2017 Infoblox Inc.

Infoblox::Grid::X509Certificate - An X509Certificate object.

- NAME
- DESCRIPTION
- CONSTRUCTOR
- SESSION METHODS
  - Infoblox::Session->get( )
  - Infoblox::Session->remove( )
  - Infoblox::Session->search( )
- METHODS
  - issuer( )
  - serial( )
  - distinguished_name( )
  - valid_not_before( )
  - valid_not_after( )
- AUTHOR
- SEE ALSO
- COPYRIGHT

NAME

Infoblox::Grid::X509Certificate - An X509Certificate object.

DESCRIPTION

An X509Certificate object.

CONSTRUCTOR

SESSION METHODS

This section describes all the methods in an Infoblox::Session module that you can apply to an X509Certificate object.

Infoblox::Session->get( )

Use this method to retrieve all the matching objects from the Infoblox appliance. See Infoblox::Session->get() for parameters and return values.

Key References

```
 Apply the following attributes to get for an X509Certificate object:

  serial              - Optional. The certificate serial number in hex format.
  issuer              - Optional. The name of the issuer.
  distinguished_name  - Optional. The certificate subject name.
  valid_not_after     - Optional. String that contains a desired value of the certificate expiration
date in ISO 8601 format.
  valid_not_before    - Optional. String that contains a desired value of the certificate start date
in ISO 8601 format.
```

Example

```
 # Retrieve X509Certificate objects with specify certificate subject name
 my @retrieved_objs = $session->get(
        object            => 'Infoblox::Grid::X509Certificate',
        distinguished_name => 'CN="10.34.1.106",OU="Engineering",O="Infoblox Inc",L="Santa
Clara",ST="California",C="US"',
 );
```

Infoblox::Session->remove( )

Use this method to remove an object from the Infoblox appliance. See Infoblox::Session->remove() for parameters and return values.

To remove a specific object, first use `get()` or `search()` to retrieve the specific object, and then submit this object for removal.

Example

```
 # Get X509Certificate object
 my @retrieved_objs = $session->get(
        object            => 'Infoblox::Grid::X509Certificate',
        distinguished_name => 'CN="10.34.1.106",OU="Engineering",O="Infoblox Inc",L="Santa
```

```
Clara",ST="California",C="US"',
 );

 # Find the desired object from the retrieved list
 my $x509cert = $retrieved_objs[0];

 # Submit for removal
 my $response = $session->remove( $x509cert );
```

Infoblox::Session->search( )

Use this method to search for objects in the Infoblox appliance. See Infoblox::Session->search() for parameters and return values.

Key References

```
 Apply the following attributes to search for an X509Certificate object:

  serial              - Optional. The certificate serial number in hex format.
  issuer              - Optional. The name of the issuer.
  distinguished_name  - Optional. The certificate subject name.
  valid_not_after     - Optional. String that contains a desired value of the certificate expiration
date in ISO 8601 format.
  valid_not_before    - Optional. String that contains a desired value of the certificate start date
in ISO 8601 format.
```

Example

```
 # Retrieve X509Certificate objects with specify certificate subject name
 my @retrieved_objs = $session->search(
        object             => 'Infoblox::Grid::X509Certificate',
        distinguished_name => 'CN="10.34.1.106",OU="Engineering",O="Infoblox Inc",L="Santa
Clara",ST="California",C="US"',
 );
```

METHODS

This section describes all the methods that you can use to configure and retrieve the attribute values of a X509Certificate object.

issuer( )

Use this method to retrieve the issuer subject name. This method does not support any argument because the attribute is read-only and is set by the server.

Parameter

None

Returns

The method returns the attribute value.

Example

```
 # Get attribute value
 my $issuer = $x509cert->issuer();
```

serial( )

Use this method to retrieve the certificate serial number in hex format. This method does not support any argument because the attribute is read-only and is set by the server.

Parameter

None

Returns

The method returns the attribute value.

Example

```
 # Get attribute value
 my $serial = $x509cert->serial();
```

distinguished_name( )

Use this method to retrieve the certificate subject name. This method does not support any argument because the attribute is read-only and is set by the server.

Parameter

None

Returns

The method returns the attribute value.

Example

```
# Get attribute value
my $distinguished_name = $x509cert->distinguished_name();
```

valid_not_before( )

Use this method to retrieve the date before the certificate becomes invalid in ISO 8601 format. This method does not support any argument because the attribute is read-only and is set by the server.

Parameter

None

Returns

The method returns the attribute value.

Example

```
# Get attribute value
my $valid_not_before = $x509cert->valid_not_before();
```

valid_not_after( )

Use this method to retrieve the date after the certificate becomes invalid in ISO 8601 format. This method does not support any argument because the attribute is read-only and is set by the server.

Parameter

None

Returns

The method returns the attribute value.

Example

```
# Get attribute value
my $valid_not_after = $x509cert->valid_not_after();
```

AUTHOR

Infoblox Inc. http://www.infoblox.com/

SEE ALSO

Infoblox::Session

COPYRIGHT

Copyright (c) 2017 Infoblox Inc.

Infoblox::IFMap::CACertificate - IF-MAP CA Certificate object.

- NAME
- DESCRIPTION
- CONSTRUCTOR
- SESSION METHODS
  - Infoblox::Session->add( )
  - Infoblox::Session->get( )
  - Infoblox::Session->remove( )
  - Infoblox::Session->search( )
- METHODS
  - certificate( )
  - subject( )
  - issuer( )
  - valid_not_before( )
  - valid_not_after( )
- SAMPLE CODE
- AUTHOR
- SEE ALSO
- COPYRIGHT

NAME

Infoblox::IFMap::CACertificate - IF-MAP CA Certificate object.

DESCRIPTION

You can configure the Infoblox appliance as a metadata access point (MAP) and implement the IF-MAP protocol defined by the Trusted Computing Group (TCG, www.trustedcomputinggroup.com). This module provides an interface for configuring CA certificates that the IF-MAP server uses to authenticate IF-MAP clients.

CONSTRUCTOR

```
my $ca_cert=Infoblox::IFMap::CACertificate->new(
    certificate      => $string, #Required
);
```

SESSION METHODS

This section describes all the methods in an Infoblox::Session module that you can apply to an IF-MAP CA Certificate object.

Infoblox::Session->add( )

Use this method to add a new CA certificate for the IF-MAP server. See Infoblox::Session->add( ) for parameters and return values.

Example

```
# Read the certificate contents from the file
open CERTFILE "<cert.pem" or die "Unable to open certificate file";
my $pem_cert = join('', <CERTFILE>);

# Create a new CA certificate object
my $cacert = Infoblox::IFMap::CACertificate->new(
certificate => $pem_cert
);

# Add the CA certificate
my $response = $session->add( $cacert );
```

Infoblox::Session->get( )

Use this method to retrieve the CA certificates configured for the IF-MAP server. See Infoblox::Session->get( ) for parameters and return values.

Key References

Apply the following attributes to get a specific IF-MAP CA certificate object:

```
subject                - Optional. String that contains a certificate subject.
issuer                 - Optional. String that contains a certificate issuer.
```

```
 valid_not_after        - Optional. String that contains a desired value of the certificate
expiration date in ISO 8601 format.
 valid_not_before       - Optional. String that contains a desired value of the certificate start
date in ISO 8601 format.
```

Example

```
# Retrieve CA certificate with the subject = "/CN=verisign.com/OU=Verisign"
my @retrieved_objs = $session->get(
object => "Infoblox::IFMap::CACertificate",
subject => "/CN=verisign.com/OU=Verisign"
);
```

Infoblox::Session->remove( )

Use this method to remove a CA certificate.

Examples

```
# Get the CA certificate
my @retrieved_objs = $session->get(
object => "Infoblox::IFMap::CACertificate",
subject   => "/CN=verisign.com/OU=Verisign"
);

# Find the desired object from the retrieved list.
my $desired_cert = $retrieved_objs[0];

# Submit for removal
my $response = $session->remove( $desired_cert );
```

Infoblox::Session->search( )

Use this method to search for the CA certificates configured for the IF-MAP server. You can specify the following attributes in a search: subject, issuer, valid_not_before, and valid_not_after.

Key References

Apply the following attributes to search for a specific IF-MAP CA certificate object:

```
 subject                - Optional. Regex for the certificate subject.
 issuer                 - Optional. Regex for the certificate issuer.
 valid_not_after        - Optional. String that contains the desired certificate expiration date in
ISO 8601 format. This method supports searching with limit parameters  (see below).
 valid_not_before       - Optional. String that contains the desired certificate start date in ISO
8601 format. This method supports searching with limit parameters  (see below).
```

See Infoblox::Session/search with limit parameters for more information on this functionality.

Examples

```
# Search for IF-MAP CA certificates issued by Verisign
# and with valid not before time in 2010
my @retrieved_objs = $session->search(
object => "Infoblox::IFMap::CACertificate",
issuer => ".*verisign.*",
valid_not_before => ">=< 2010-01-01T00:00:00Z,2011-01-01T00:00:00Z",
);
```

METHODS

certificate( )

Use this method to retrieve or set the value of the certificate attribute. If no argument is passed, the method returns the current attribute value. The value must be a single certificate in PEM format.

Parameter

The valid value is a string that contains the certificate in PEM format. This is a required attribute.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Examples

```
# Get certificate
```

```
my $pem_cert = $cert_object->certificate( );

# Set certificate
open CERTFILE "<cert.pem" or die "Unable to open certificate file";
my $pem_cert = join('', <CERTFILE>);
$cert_object->certificate( $pem_cert );
```

subject( )

Use this method to retrieve the value of the subject attribute. This method does not support any argument because the attribute is read-only and is set by the server. It has a value only when it reads an existing certificate from the Infoblox::IFMap::CACertificate object retrieved from the appliance.

Parameter

N/A.

Returns

The method returns the attribute value.

This is a read-only attribute. If you specify a parameter, the method returns an error.

Examples

```
# Get subject
my $subject = $certificate->subject( );
```

issuer( )

Use this method to retrieve the value of the =issuer= attribute. This method does not support any argument because the attribute is read-only and is set by the server. It has a value only when it reads an existing certificate from the =Infoblox::IFMap::CACertificate= object.

Parameter

N/A.

Returns

The method returns the attribute value.

This is a read-only attribute. If you specify a parameter, the method returns an error.

Examples

```
# Get subject
my $issuer = $certificate->issuer( );
```

valid_not_before( )

Use this method to retrieve the value of the valid_not_before attribute. This method does not support any argument because the attribute is read-only and is set by the server. It has a value only when it reads an existing certificate from the Infoblox::IFMap::CACertificate object. The returned value is the date before the certificate becomes invalid in ISO 8601 format.

Parameter

N/A.

Returns

The method returns the attribute value.

This is a read-only attribute. If you specify a parameter, the method returns an error.

Examples

```
# Get valid_not_before attribute
my $valid_not_before = $certificate->valid_not_before( );
```

valid_not_after( )

Use this method to retrieve the value of the valid_not_after attribute. This method does not support any argument because it is read-only and is set by the server. It has a value only when it reads an existing certificate from the Infoblox::IFMap::CACertificate object. The returned value is the date after the certificate becomes invalid in the ISO 8601 format.

Parameter

N/A.

Returns

The method returns the attribute value.

This is a read-only attribute. If you specify a parameter, the method returns an error.

Examples

```
# Get operation
my $valid_not_after = $certificate->valid_not_after( );
```

SAMPLE CODE

The following sample code demonstrates the different functions that can be applied to an object, such as add, search and remove. This sample also includes error handling for the operations.

#Preparations

```
#PROGRAM STARTS: Include all the modules that will be used
use strict;
use Infoblox;
#Create a session to the Infoblox appliance
my $session = Infoblox::Session->new(
               master   => "192.168.1.2", #appliance host ip
               username => "admin",       #appliance user login
               password => "infoblox"     #appliance password
);
unless ($session) {
       die("Construct session failed: ",
            Infoblox::status_code( ) . ":" . Infoblox::status_detail( ));
}
print "Session created successfully\n";

#Get a certificate from the external file
local *CERTFILE;
open CERTFILE, "<API_unit_test_sample_code/ca_cert.pem" or die "Unable to open certificate file";
my $pem_cert = join('', <CERTFILE>);
close(CERTFILE);
```

#Create CA certificate object

```
my $ca_certificate= Infoblox::IFMap::CACertificate->new(
                                                          certificate => $pem_cert
                                                        );
unless($ca_certificate) {
       die("Construct CA certificate object failed: ",
               Infoblox::status_code( ) . ":" . Infoblox::status_detail( ));
}
print "IFMap CA Certificate object created successfully\n";
```

#Add the CA Certificate object to the Infoblox appliance through a session

```
$session->add($ca_certificate)
    or die("Add IFMap CA certificate object failed: ",
                        $session->status_code( ) . ":" . $session->status_detail( ));
print "IFMap CA Certificate object added to server successfully\n";

#Get the IFMap CA Certificate object using subject attribute
my @retrieved_objs=$session->get(
                                  object => "Infoblox::IFMap::CACertificate",
                                  subject => 'CN="Test CA 2",OU="Engineering",O="Infoblox",L="Santa
Clara",ST="CA",C="US"'
                                );
my $object = $retrieved_objs[0];
unless($object){
    die("Get IFMap CA Certificate object failed: ",
                $session->status_code( ) . ":" . $session->status_detail( ));
}
print "Get IFMap CA Certificate object found at least 1 matching entry\n";
```

#Search for IFMap CA certificate objects

```
my @retrieved_objs=$session->search(
                                  object => "Infoblox::IFMap::CACertificate",
                                  subject => '.*Engineering.*',
```

```
                                    valid_not_before => '>=< 2010-01-01T08:00:00Z,2010-10-01T00:00:00Z'
                                );
 my $object = $retrieved_objs[0];
 unless($object){
     die("Search for IFMap CA Certificate object failed: ",
                 $session->status_code( ) . ":" . $session->status_detail( ));
 }
 print "Searching IFMap CA Certificate object found at least 1 matching entry\n";
```

#Remove IFMap CA Certificate object

```
 #Get IFMap CA Certificate object
 my @retrieved_objs=$session->get(
                                    object => "Infoblox::IFMap::CACertificate",
                                    subject => 'CN="Test CA 2",OU="Engineering",O="Infoblox",L="Santa
Clara",ST="CA",C="US"'
                                );
 my $object = $retrieved_objs[0];
 unless($object){
     die("Get IFMap CA certificate object failed: ",
                 $session->status_code( ) . ":" . $session->status_detail( ));
 }
 print "Get IFMap CA certificate object found at least 1 matching entry\n";

 #submit the object for removal
 $session->remove($object)
     or die("Remove IFMap CA certificate failed: ",
             $session->status_code( ) . ":" . $session->status_detail( ));
 print "IFMap CA certificate object removed successfully \n";
```

####PROGRAM ENDS####

AUTHOR

Infoblox Inc. http://www.infoblox.com/

SEE ALSO

Infoblox::Session,Infoblox::Session->add( ),Infoblox::Session->get( ),Infoblox::Session->remove( ),Infoblox::Session->search( )

COPYRIGHT

Copyright (c) 2017 Infoblox Inc.

Infoblox::IFMap::Client::Credentials - IF-MAP Credentials object

- NAME
- DESCRIPTION
- CONSTRUCTOR
- METHODS
  - authentication( )
  - password( )
  - server_ca_cert( )
  - server_url( )
  - username( )
- AUTHOR
- SEE ALSO
- COPYRIGHT

NAME

Infoblox::IFMap::Client::Credentials - IF-MAP Credentials object

DESCRIPTION

This object represents the credentials used by an IF-MAP Federation Connector object.

CONSTRUCTOR

```
my $credentials = Infoblox::IFMap::Federation::Connector::Credentials->new(
    authentication => <"basic" | "certificate">,              #Required
    password => <STRING>,                                     #Required if authentication = "basic"
    server_ca_cert => $cacert,                                #Optional
    server_url => <STRING>,                                   #Optional
    username => <STRING>,                                     #Required if authentication = "basic"
);
```

METHODS

This section describes all the methods that you can use to set and retrieve the attribute values of an IF-MAP Credentials object.

authentication( )

Use this method to set or retrieve the value of the authentication attribute.

Parameter

A string that represents the attribute. Valid strings are "basic" or "certificate".

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the authentication attribute.

Example

```
# Get authentication
my $authentication = $credentials->authentication( );

# Change it
$credentials->authentication("basic");
```

password( )

Use this method to set the value of the password attribute. This value is not returned by the server when you use the `get()` and `search()` session methods. This password is used for Basic Authentication and is required if authentication is set to "basic". This is a write-only attribute.

Parameter

A string that specifies the password.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

Example

```
# Set it
$credentials->password("secret");
```

server_ca_cert( )

Use this method to set or retrieve the value of the server_ca_cert attribute. This configures the CA certificate to use for server certificate validation.

Parameter

An Infoblox::IFMap::CACertificate object.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute server_ca_cert.

Example

```
# Get server_ca_cert
my $server_ca_cert = $credentials->server_ca_cert( );

# Change it
$credentials->server_ca_cert($ca_cert);
```

server_url( )

Use this method to set or retrieve the value of the server_url attribute. It specifies the URL of the federated IF-MAP service.

Parameter

The URL of the IF-MAP service. For example "https://ifmap.infoblox.com:443/ifmap".

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute server_url.

Example

```
# Get server_url
my $server_url = $credentials->server_url( );

# Change it
$credentials->server_url("https://ifmap.infoblox.com:443/ifmap");
```

username( )

Use this method to set or retrieve the value of the username attribute. This username is used for Basic Authentication and is required if authentication is set to "basic". It has to be unique per server, specified by the server_url attribute.

Parameter

A string that represents the attribute.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute username.

Example

```
# Get username
my $username = $credentials->username( );

# Change it
$credentials->username("user1");
```

AUTHOR

Infoblox Inc. http://www.infoblox.com/

SEE ALSO

Infoblox::IFMap::CACertificate

COPYRIGHT

Copyright (c) 2017 Infoblox Inc.

Infoblox::IPAM::Address - IPAM address object

- NAME
- DESCRIPTION
- CONSTRUCTOR
- SESSION METHODS
  - Infoblox::Session->get( )
  - Infoblox::Session->remove( )
- METHODS
  - conflict_types( )
  - dhcp_client_identifier( )
  - discovered_data( )
  - discovered_name( )
  - discover_now_status( )
  - duid( )
  - discoverer( )
  - extattrs( )
  - extensible_attributes( )
  - fingerprint( )
  - first_discovered( )
  - ip_address( )
  - is_conflict( )
  - is_invalid_mac( )
  - last_discovered( )
  - lease_state( )
  - mac_address( )
  - ms_ad_user_data( )
  - name( )
  - netbios_name( )
  - network_component_description( )
  - network_component_ip( )
  - network_component_name( )
  - network_component_port_description( )
  - network_component_port_name( )
  - network_component_port_number( )
  - network_component_type( )
  - network_view( )
  - objects( )
  - os( )
  - port_duplex( )
  - port_link_status( )
  - port_speed( )
  - port_status( )
  - port_vlan_description( )
  - port_vlan_name( )
  - port_vlan_number( )
  - reserved_port( )
  - status( )
  - types( )
  - usage( )
  - username( )
  - v_cluster( )
  - v_datacenter( )
  - v_host( )
  - v_name( )
  - v_netadapter( )
  - v_switch( )
  - v_type( )
- SAMPLE CODE
- AUTHOR
- SEE ALSO
- COPYRIGHT

NAME

Infoblox::IPAM::Address - IPAM address object

DESCRIPTION

The IPAM address object is used to view IPv4 and IPv6 address management information in an Infoblox appliance.

CONSTRUCTOR

The Infoblox::IPAM::Address object is a read-only object and does not require manual construction.

SESSION METHODS

This section describes all the methods in an Infoblox::Session that can be applied to an IPAM address object.

Infoblox::Session->get( )

Use this method to retrieve the existing objects from an Infoblox appliance. See Infoblox::Session->get() for parameters and return values.

Note that for IPv4 addresses, both used and unused addresses are returned unless 'status' is specified; but IPv6 addresses will only return used addresses.

'discover_now_status' will not be returned by default, use 'return_methods' to request it if needed.

Key Reference

```
 Apply the following attributes to get the IPAM address object(s). Note that at least one of the
following is required: address, start_addr, end_addr, or network.
```

address - Optional. An IPv4 or IPv6 address. Get IPAM information for this address. All other optional arguments are ignored, except for network_view, if 'address' is present.

network - Optional. The IPv4 or IPv6 network, in CIDR notation (e.g 1.0.0.0/8 or 2000::/64), where the IPAM address is located.

network_view - Optional. The network view in which the network is located. The default is the system-defined default network view.

start_addr - Optional. An IPv4 or IPv6 address. Start address of an IPAM address scope. If the start address is not in the specified network, an error is returned.

end_addr - Optional. An IPv4 or IPv6 address. End address of an IPAM address scope. If the end address is not in the specified network, an error is returned.

status - Optional. The filter type of the IPAM address. Valid values are "used", "unused" and "all". Default value is "all". This argument applies to IPv4 addresses only. When the value is returned ", Unmanaged" or ", Conflict" might be appended to the status member when applicable.

type - Optional. This is exactly the same as the 'status' argument, and is being deprecated in favor of the 'status' argument. This argument applies to IPv4 addresses only.

extattrs - Optional. A hash reference containing extensible attributes.

extensible_attributes - Optional. A hash reference containing extensible attributes.

fingerprint - Optional. DHCP fingerprint description.

discovered_duid - Optional. Discovered DHCPv6 UID.

discovered_name - Optional. Discovered name.

discoverer - Optional. Discoverer name.

first_discovered - Optional. Time of the first discovery.

last_discovered - Optional. Time of the last discovery.

netbios - Optional. Discovered NetBIOS name.

network_component_description - Optional. Discovered network component description.

network_component_ip - Optional. Discovered network component IP address.

network_component_name - Optional. Discovered network component name.

network_component_port_description - Optional. Discovered network component port description.

network_component_port_name - Optional. Discovered network component port name.

network_component_port_number - Optional. Discovered network component port number.

network_component_type - Optional. Discovered network component type.

os - Optional. Discovered OS.

port_duplex - Optional. Discovered port duplex settings.

port_link_status - Optional. Discovered port link status.

port_speed - Optional. Discovered port speed settings.

port_status - Optional. Discovered port status settings.

port_vlan_description - Optional. Discovered port VLAN description.

port_vlan_name - Optional. Discovered port VLAN name.

port_vlan_number - Optional. Discovered port VLAN number.

v_cluster - Optional. Discovered VMware cluster name.

v_datacenter - Optional. Discovered VMware datacenter name.

v_host - Optional. Discovered VMware server name.

v_name - Optional. Discovered VMware host or virtual machine name.

v_netadapter - Optional. Discovered VMware physical network adapter name.

v_switch - Optional. Discovered VMware virtual switch name.

v_type - Optional. Discovered VMware entity type (possible values: "VirtualMachine", "HostSystem", "VirtualCenter").

Example

```
# case 1
my @ipam_objects = $session->get( 'object'     => 'Infoblox::IPAM::Address',
                                  'network'  => '10.0.0.0/8');
```

Since there are no start and end addresses specified, the appliance returns a maximum of 2000 IP addresses, which is the maximum page size of an administrator. If the total number of IP addresses in the network is less than 2000, then it returns all IP addresses.

```
# case 2
my @ipam_objects = $session->get( 'object'     => 'Infoblox::IPAM::Address',
                                  'network'    => '10.0.0.0/8',
                                  'start_addr' => '10.0.0.100');
```

This call returns a maximum of 2000 IPAM addresses starting from 10.0.0.100.

```
# case 3
my @ipam_objects = $session->get( 'object'     => 'Infoblox::IPAM::Address',
                                  'network'    => '10.0.0.0/8',
                                  'start_addr' => '10.0.0.100'
                                  'end_addr'   => '10.0.0.200');
```

This call returns 101 IPAM addresses from 10.0.0.100 to 10.0.0.200.

```
# case 4
my @ipam_objects = $session->get( 'object'     => 'Infoblox::IPAM::Address',
                                  'address'    => '10.0.0.10');
```

This call returns the IPAM address object for 10.0.0.10.

```
# case 5
my @ipam_objects = $session->get( 'object'     => 'Infoblox::IPAM::Address',
                                  'network'  => '10.0.0.0/8',
                                  'start_addr' => '10.0.0.100',
                                  'end_addr'  => '10.0.0.200',
                                  'status'          => 'used' );
```

This call returns used IPAM addresses from 10.0.0.100 to 10.0.0.200. Note that the filter is applied to the specified scope. Therefore, the number of returned objects could be less than 101, if only a few IP addresses are used in this scope.

```
# case 6
my @ipam_objects = $session->get( 'object        => 'Infoblox::IPAM::Address',
                                  'network'      => '10.0.0.0/8',
                                  'network_view' => 'my_network_view');
```

Same as case 1, but this uses the 'my_network_view' network view rather than the system-defined default view.

```
# case 7
my @ipam_objects = $session->get( 'object'     => 'Infoblox::IPAM::Address',
                                  'address'    => '10.0.0.5');
```

```
# case 8
my @ipam_objects = $session->get( 'object'     => 'Infoblox::IPAM::Address',
                                  'network'  => '2000::/64');
```

Since there are no start and end addresses specified, the appliance returns a maximum of 2000 used IP addresses, which is the maximum page size of an administrator. If the total number of IP addresses in the network is less than 2000, then it returns all IP addresses.

```
# case 9
my @ipam_objects = $session->get( 'object'     => 'Infoblox::IPAM::Address',
                                  'network'    => '2000::/64',
                                  'start_addr' => '2000::11');
```

This call returns a maximum of 2000 used IPv6 addresses starting from 2000::11.

```
# case 10
my @ipam_objects = $session->get( 'object'     => 'Infoblox::IPAM::Address',
                                  'network'    => '2000::/64',
                                  'start_addr' => '2000::11',
                                  'end_addr'   => '2000::31');
```

This call returns used IPv6addresses from '2000::11' to '2000::31'.

Infoblox::Session->remove( )

Use this method to remove an object from the Infoblox appliance. See Infoblox::Session->remove() for parameters and return values.

To remove a specific object, first use `get()` to retrieve the specific object, and then submit this object for removal.

Example

```
# Get the object under the same name
my @result_array = $session->get(
      object        => "Infoblox::IPAM::Address" ,
      network       => "2001::/64",
);

# find the desired object from the retrieved list.
my $desired_ipam_address = $result_array[0];

# Submit for removal
my $response = $session->remove( $desired_ipam_address );
```

METHODS

This section describes all the methods that can be used to retrieve the attribute values of an IPAM address object.

conflict_types( )

Use this method to retrieve a list of Discovery conflict types. This attribute is read-only.

Parameter

none

Returns

The method returns the attribute value.

Example

```
#Get conflict_types
my $value = $ipam_address->conflict_types();
```

dhcp_client_identifier( )

Use this method to retrieve the DHCP client identifier of an IPAM address object. This method applies to IPv4 addresses only.

Parameter

none

Returns

The method returns the DHCP client identifier attribute value.

Example

```
#Get DHCP client identifier
my $identifier = $ipam_address->dhcp_client_identifier();
```

discovered_data( )

Use this method to retrieve the discovered data of an IPAM address object. This is a read-only attribute.

Omit the parameter to retrieve the attribute value.

Parameter

None

Returns

The valid return value is an Infoblox::Grid::Discovery::Data object.

Example

```
#Get discovered_data value
my $discovered_data = $object->discovered_data();
```

discovered_name( )

Use this method to retrieve the discovered name of an IPAM address object.

Parameter

none

Returns

The method returns the discovered name.

Example

```
#Get discovered_name
my $discovered_name = $ipam_address->discovered_name();
```

discover_now_status( )

Use this method to retrieve the current discovery status. This attribute is read-only.

'discover_now_status' will not be returned by default, use 'return_methods' to request it if needed.

Parameter

None.

Returns

The method returns the attribute value that can be 'COMPLETE', 'FAILED', 'NONE', 'PENDING' or 'RUNNING'.

Example

```
#Get discover_now_status
my $discover_now_status = $ipam_address->discover_now_status();
```

duid( )

Use this method to retrieve the DHCPv6 Unique Identifier (DUID) of an IPAM address object.

Parameter

none

Returns

The method returns the DUID value.

Example

```
#Get mac address
my $duid = $ipam_address->duid();
```

discoverer( )

Use this method to retrieve the discoverer of an IPAM address object.

Parameter

none

Returns

The method returns the discoverer value.

Example

```
#Get discoverer
my $discoverer = $ipam_address->discoverer();
```

extattrs( )

Use this method to retrieve the extensible attributes associated with an IPAM address object. This attribute is read-only and cannot be set.

Parameter

None.

Returns

The method returns the attribute value.

Example

```
#Get extattrs
my $ref_extattrs = $ipam_address->extattrs();
```

extensible_attributes( )

Use this method to retrieve the extensible attributes associated with an IPAM address object.

Parameter

none

Returns

The method returns a reference to a hash containing the extensible attributes.

Example

```
#Get extensible attributes
my $ref_extensible_attributes = $ipam_address->extensible_attributes();
```

fingerprint( )

Use this method to retrieve DHCP fingerprint description of an IPAM address object. This is a read-only attribute.

Parameter

none

Returns

The method returns the attribute value.

Example

```
#Get fingerprint
my $fingerprint = $ipam_address->fingerprint();
```

first_discovered( )

Use this method to retrieve the date and time that the IPAM address object was first discovered.

Parameter

none

Returns

The method returns the date and time that the IPAM address object was fisrt discovered.

Example

```
#Get first_discovered
my $first_discovered = $ipam_address->first_discovered();
```

ip_address( )

Use this method to retrieve the IP addresses of an IPAM address object.

Parameter

none

Returns

The method returns the IP address value. The value can be an IPv4 or IPv6 address.

Example

```
#Get IP address
my $ip_address = $ipam_address->ip_address();
```

is_conflict( )

Use this method to find out whether this IPAM address is in conflict with another IPAM address in the system. This method applies to IPv4 addresses only.

Parameter

none

Returns

The method returns true if the IPAM address is in conflict with another in the system, false otherwise.

Example

```
#Get is_conflict
my $conflict = $ipam_address->is_conflict();
```

is_invalid_mac( )

Use this method to retrieve the flag used to indicate whether an invalid MAC address is used. This is a read-only attribute.

Parameter

None

Returns

The method returns the attribute value.

Example

```
#Get is_invalid_mac value
my $is_invalid_mac = $ipam_address->is_invalid_mac();
```

last_discovered( )

Use this method to retrieve the last discovered time of an IPAM address object.

Parameter

none

Returns

The method returns the last discovered value.

Example

```
#Get last discovered
my $last_discovered = $ipam_address->last_discovered();
```

lease_state( )

Use this method to retrieve the lease state of an IPAM address object. This method applies to IPv4 addresses only.

Parameter

none

Returns

The method returns the lease state value. A valid state is any of the following string values: free, backup, active, expired, released, abandoned, and reset.

Example

```
#Get lease state
my $lease_state = $ipam_address->lease_state();
```

mac_address( )

Use this method to retrieve the MAC address of an IPAM address object.

Parameter

none

Returns

The method returns the MAC address value.

Example

```
#Get mac address
my $mac_address = $ipam_address->mac_address();
```

ms_ad_user_data( )

Use this method to retrieve Microsoft Active Directory users related information. This is a read-only attribute.

Parameter

None

Returns

The valid return value is an Infoblox::Grid::MSServer::AdUser::Data object.

Example

```
#Get ms_ad_user_data
my $ms_ad_user_data = $ipam_address->ms_ad_user_data();
```

name( )

Use this method to retrieve the name of an IPAM address object.

Parameter

none

Returns

The method returns the name value.

Example

```
#Get name
my $name = $ipam_address->name();
```

netbios_name( )

Use this method to retrieve the NETBIOS name of an IPAM address object.

Parameter

none

Returns

The method returns the NETBIOS name value.

Example

```
#Get netbios name
```

```
my $netbios_name = $ipam_address->netbios_name();
```

network_component_description( )

Use this method to retrieve the description of the network device that is connected to the IPAM address object.

Parameter

none

Returns

The method returns the network device description.

Example

```
#Get network_component_description
my $network_component_description = $ipam_address->network_component_description();
```

network_component_ip( )

Use this method to retrieve the IP address of the network device that is connected to the IPAM address object.

Parameter

none

Returns

The method returns the network device IP address.

Example

```
#Get network_component_ip
my $network_component_ip = $ipam_address->network_component_ip();
```

network_component_name( )

Use this method to retrieve the name of the network device that is connected to the IPAM address object.

Parameter

none

Returns

The method returns the network device name.

Example

```
#Get network_component_name
my $network_component_name = $ipam_address->network_component_name();
```

network_component_port_description( )

Use this method to retrieve the description of the network device port that is connected to the IPAM address object.

Parameter

none

Returns

The method returns the network device port description.

Example

```
#Get network_component_port_description
my $network_component_port_description = $ipam_address->network_component_port_description();
```

network_component_port_name( )

Use this method to retrieve the name of the network device port that is connected to the IPAM address object.

Parameter

none

Returns

The method returns the network device port name.

Example

```
#Get network_component_port_name
my $network_component_port_name = $ipam_address->network_component_port_name();
```

network_component_port_number( )

Use this method to retrieve the number of the network device port that is connected to the IPAM address object.

Parameter

none

Returns

The method returns the network device port number.

Example

```
#Get network_component_port_number
my $network_component_port_number = $ipam_address->network_component_port_number();
```

network_component_type( )

Use this method to retrieve the type of the network device that is connected to the IPAM address object.

Parameter

none

Returns

The method returns the network device type.

Example

```
#Get network_component_type
my $network_component_type = $ipam_address->network_component_type();
```

network_view( )

Use this method to retrieve the network view name of an IPAM address object.

Parameter

none

Returns

The method returns the network view name.

Example

```
#Get network view
my $nview_name = $ipam_address->network_view();
```

objects( )

Use this method to retrieve the IPAM address object's associated objects.

Parameter

none

Returns

The method returns a reference to an array of associated objects. Note that this method has to be explicitly requested via Infoblox::Session's return_methods member.

Associated objects can be one or more of the following PAPI objects:

```
Infoblox::DHCP::FixedAddr
Infoblox::DHCP::IPv6FixedAddr
Infoblox::DHCP::Lease
Infoblox::DNS::BulkHost
```

```
Infoblox::DNS::Host
Infoblox::DNS::Record::A
Infoblox::DNS::Record::AAAA
Infoblox::DNS::Record::PTR
```

Example

```
my @retrieved_obj = $session->get(
                                    object     => 'Infoblox::IPAM::Address',
                                    start_addr => '10.0.0.0',
                                    end_addr   => '10.0.0.9',
                                    return_methods => ['+objects'],
                                   );

# Get the associated objects (if any)
my $associated_ref = $retrieved_obj[0]->objects();
```

os( )

Use this method to retrieve the OS of an IPAM address object.

Parameter

none

Returns

The method returns the OS value.

Example

```
#Get os
my $os = $ipam_address->os();
```

port_duplex( )

Use this method to retrieve the duplex setting of the network device port that is connected to the IPAM address object.

Parameter

none

Returns

The method returns the port duplex setting. Returned values, if any, are one of the following:

```
Full
Half
```

Example

```
#Get port_duplex
my $port_duplex = $ipam_address->port_duplex();
```

port_link_status( )

Use this method to retrieve the link status of the network device port that is connected to the IPAM address object.

Parameter

none

Returns

The method returns the port link status of the network device. Returned values, if any, are one of the following:

```
Connected
Not Connected
Unknown
```

Example

```
#Get port_link_status
my $port_link_status = $ipam_address->port_link_status();
```

port_speed( )

Use this method to retrieve the speed of the network device port that is connected to the IPAM address object.

Parameter

none

Returns

The method returns the port speed. Returned values, if any, are one of the following:

```
10M
100M
1G
10G
100G
Unknown
```

Example

```
#Get port_speed
my $port_speed = $ipam_address->port_speed();
```

port_status( )

Use this method to retrieve the status of the network device port that is connected to the IPAM address object.

Parameter

none

Returns

The method returns the network device port status. Returned values, if any, are one of the following:

```
Up
Down
Unknown
```

Example

```
#Get port_status
my $port_status = $ipam_address->port_status();
```

port_vlan_description( )

Use this method to retrieve the VLAN description of the network device port that is connected to the IPAM address object.

Parameter

none

Returns

The method returns the network device port VLAN description.

Example

```
#Get port_vlan_description
my $port_vlan_description = $ipam_address->port_vlan_description();
```

port_vlan_name( )

Use this method to retrieve the VLAN name of the network device port that is connected to the IPAM address object.

Parameter

none

Returns

The method returns the network device port VLAN name.

Example

```
#Get port_vlan_name
my $port_vlan_name = $ipam_address->port_vlan_name();
```

port_vlan_number( )

Use this method to retrieve the VLAN number of the network device port that is connected to the IPAM address object.

Parameter

none

Returns

The method returns the network device port VLAN number.

Example

```
#Get port_vlan_number
my $port_vlan_number = $ipam_address->port_vlan_number();
```

reserved_port( )

Use this method to retrieve the reserved port. This attribute is read-only.

Parameter

none

Returns

The method returns the attribute value.

Example

```
#Get reserved_port
my $value = $ipam_address->reserved_port();
```

status( )

Use this method to retrieve the status of an IPAM address object.

Parameter

none

Returns

The method returns the status value.

Example

```
#Get status
my $status = $ipam_address->status();
```

types( )

Use this method to retrieve the type of an IPAM address object.

Parameter

none

Returns

The method returns the type value. A valid value is a comma-separated list that contains a subset of the following values: A, FA, RESERVATION, RANGE, LEASE, UNMANAGED, HOST, BULKHOST, PTR, NETWORK, and BROADCAST. Note that in the subset of values, "FA" refers to "fixed address".

Example

```
#Get types
my $types= $ipam_address->types();
```

usage( )

Use this method to retrieve the usage of an IPAM address object.

Parameter

none

Returns

The method returns the usage value. A valid value is a comma-separated list that contains a subset of the following values: DNS, DHCP.

Example

```
#Get usage
my $usage= $ipam_address->usage();
```

username( )

Use this method to retrieve the user name of an IPAM address object. This method applies to IPv4 addresses only.

Parameter

none

Returns

The method returns the user name.

Example

```
#Get username
my $username= $ipam_address->username();
```

v_cluster( )

Use this method to retrieve the name of the VMware cluster associated with the IPAM Address object. This method is read-only.

Parameter

none

Returns

This method returns a string that contains the VMware cluster name. The default value is an empty string.

Example

```
#Get v_cluster
my $v_cluster = $ipam_address->v_cluster();
```

v_datacenter( )

Use this method to retrieve the name of the VMware datacenter associated with the IPAM Address object. This method is read-only.

Parameter

none

Returns

This method returns a string that contains the VMware datacenter name. The default value is an empty string.

Example

```
#Get v_datacenter
my $v_datacenter = $ipam_address->v_datacenter();
```

v_host( )

Use this method to retrieve the name of the VMware host associated with the IPAM Address object. This method is read-only.

Parameter

none

Returns

This method returns a string that contains the VMware host name. The default value is an empty string.

Example

```
#Get v_host
my $v_host = $ipam_address->v_host();
```

v_name( )

Use this method to retrieve the name of the VMware entity associated with the IPAM Address object. This method is read-only.

Parameter

none

Returns

This method returns a string that contains the VMware entity name. The default value is an empty string.

Example

```
#Get v_name
my $v_name = $ipam_address->v_name();
```

v_netadapter( )

Use this method to retrieve the name of the physical network adapter through which the VMware entity is connected to the appliance. This method is read-only.

Parameter

none

Returns

This method returns a string that contains the network adapter name. The default value is an empty string.

Example

```
#Get v_netadapter
my $v_netadapter = $ipam_address->v_netadapter();
```

v_switch( )

Use this method to retrieve the name of the virtual switch through which the VMware entity is connected to the appliance. This method is read-only.

Parameter

none

Returns

This method returns a string that contains the virtual switch name. The default value is an empty string.

Example

```
#Get v_switch
my $v_switch = $ipam_address->v_switch();
```

v_type( )

Use this method to retrieve the type of VMware entity associated with the IPAM Address object. This method is read-only.

Parameter

none

Returns

This method returns a string that contains the VMware entity type. The default value is an empty string. The returned value, if any, can be one of the following:

```
VirtualMachine
HostSystem
VirtualCenter
```

Example

```
#Get v_type
my $v_type = $ipam_address->v_type();
```

SAMPLE CODE

The following sample code demonstrates the get session method on an IPAM address object.

```
#PROGRAM STARTS: Include all the modules that will be used
use strict;
use Infoblox;
```

```
my $host_ip = "192.168.1.2";
my $host_name = "localhost";
my $node_ip = "192.168.1.2";

#Create a session to the Infoblox appliance
my $session = Infoblox::Session->new(
               master   => $host_ip,
               username => "admin",
               password => "infoblox"
               );

unless ($session) {
       die("Construct session failed: ",
               Infoblox::status_code() . ":" . Infoblox::status_detail());
}
print "Session created successfully\n";

my $memberdhcp = Infoblox::DHCP::Member->new(
          name     => $host_name,
          ipv4addr => $node_ip,
     );

#Create a large network object
my $network_large = Infoblox::DHCP::Network->new(
                                          "network" => "10.0.0.0/8",
                                          "members" => [$memberdhcp],
                                         );

$session->add($network_large)
   or die("Add network failed: " . $session->status_code() . ": " . $session->status_detail());

#Create another member
my $new_member = Infoblox::Grid::Member->new(
       grid      => "0",
       name      => "foo.com",
       ipv4addr  => "3.0.0.2",
       mask      => "255.255.255.0",
       gateway   => "3.0.0.1",
     );
$session->add($new_member)
   or die("Add grid member failed: " . $session->status_code() . ": " . $session->status_detail());

my $memberdhcp2 = Infoblox::DHCP::Member->new(
          name     => "foo.com",
          ipv4addr => "3.0.0.2",
     );

#Create a network view
my $nview = Infoblox::DHCP::View->new(
                                   "name" => "my_network_view",
                                  );
$session->add($nview)
   or die("Add network view failed: " . $session->status_code() . ": " . $session->status_detail());

#Create a DNS view associated with the network view
my $view = Infoblox::DNS::View->new(
  "name" => "my_dns_view",
  "network_view" => $nview,
);
$session->add($view)
   or die("Add DNS view failed: " . $session->status_code() . ": " . $session->status_detail());

#Create a DNS Zone in a non-default DNS view
my $zone = Infoblox::DNS::Zone->new(
  "name" => "test.com",
  "views" => [$view],
);
$session->add($zone)
   or die("Add DNS Zone failed: " . $session->status_code() . ": " . $session->status_detail());

#Create a small IPv4 network object with a non-default network view
my $network_small = Infoblox::DHCP::Network->new(
                                          "network" => "20.0.0.0/24",
                                          "members" => [$memberdhcp2],
                                          "network_view" => $nview,
                                         );
```

```
$session->add($network_small)
   or die("Add network failed: " . $session->status_code() . ": " . $session->status_detail());

#Create an IPv6 network object with a non-default network view
my $ipv6_network = Infoblox::DHCP::IPv6Network->new(
    "network" => "2001::/64",
    "network_view" => $nview,
);
$session->add($ipv6_network)
   or die("Add IPv6 network failed: " . $session->status_code() . ": " . $session->status_detail());

#Construct an IPv4 DHCP range object
my $dhcp_range = Infoblox::DHCP::Range->new(
        start_addr              => "10.0.0.1",
        end_addr                => "10.0.0.10",
        authority               => "true",
        bootfile                => "bootfile1.com",
        bootserver              => "someserver.com",
        comment                 => "add range",
        ddns_generate_hostname  => "true",
        deny_all_clients        => "true",
        deny_bootp              => "true",
        disable                 => "true",
        enable_ddns             => "true",
        member                  => $memberdhcp,
        nextserver              => "3.3.3.3",
        pxe_lease_time          => "3600",
        recycle_leases          => "true",
);

$session->add($dhcp_range)
   or die("Add DHCP range failed: " . $session->status_code() . ": " . $session->status_detail());

#Construct an IPv4 DHCP fixed address object
my $fixed_addr = Infoblox::DHCP::FixedAddr->new(
      "ipv4addr"       => "10.0.0.15",
      "mac"            => "11:11:11:11:11:11",
      "dhcp_client_identifier" => "",
      "client_identifier_prepend_zero"   => "false",
      "match_client"   => "MAC",
      "network"        => "10.0.0.0/8",
      "bootfile"       => "bootfile",
      "bootserver"     => "bootserver.com",
      "comment"        => "add fixed address",
      "deny_bootp"     => "false",
      "disable"        => "false",
      "dynamic"        => "false",
      "enable_ddns"    => "false",
      "nextserver"     => "3.3.3.3",
      "pxe_lease_time" => "3600",
);

$session->add($fixed_addr)
   or die("Add DHCP range failed: " . $session->status_code() . ": " . $session->status_detail());

#Construct IPv6 Host and AAAA record objects in a non-default DNS view
my $host = Infoblox::DNS::Host->new(
      "name"           => "host1.test.com",
      "ipv6addrs"      => ["2001::1", "2001::2", "2001::5"],
      "views"          => [$view],
);

$session->add($host)
   or die("Add host failed: " . $session->status_code() . ": " . $session->status_detail());

my $aaaa = Infoblox::DNS::Record::AAAA->new(
      "name"           => "myhost1.test.com",
      "ipv6addr"       => "2001::1",
      "views"          => [$view],
);

$session->add($aaaa)
   or die("Add AAAA record failed: " . $session->status_code() . ": " . $session->status_detail());

   # Case 1: Get IPAM address objects using 'network'
   # This should return 2000 IPAM addresses
   my @ipam objects   $session  get
```

```
                    =          ->    ('object'     => 'Infoblox::IPAM::Address',
                                     'network'    => '10.0.0.0/8');
    unless( scalar(@ipam_objects) == 2000) {
        die("Returned IPAM addresses number is not correct");
    }
    print("Returned IPAM addresses number correctly\n");

    # Case 2: Get IPAM address objects using 'start_addr'
    # This should return 2000 IPAM addresses
    my @ipam_objects = $session->get('object'     => 'Infoblox::IPAM::Address',
                                     'network'    => '10.0.0.0/8',
                                     'start_addr' => '10.0.0.10');
    unless( scalar(@ipam_objects) == 2000) {
        die("Returned IPAM addresses number is not correct");
    }
    print("Returned IPAM addresses number correctly\n");

    # Case 3: Get IPAM address objects using 'start_addr' and 'end_addr'
       # This should return the specified number of IPAM addresses
    my @ipam_objects = $session->get('object'     => 'Infoblox::IPAM::Address',
                                     'network'    => '10.0.0.0/8',
                                     'start_addr' => '10.0.0.10',
                                     'end_addr'   => '10.0.0.100');

    unless( scalar(@ipam_objects) == 91 ) {
        die("Returned IPAM addresses number is not correct");
    }
    print("Returned IPAM addresses number correctly\n");

    # Case 4: Get IPAM address objects using 'address'
    # This should return one IPAM address
    my @ipam_objects = $session->get('object'     => 'Infoblox::IPAM::Address',
                                     'address'    => '10.0.0.10');
    unless( scalar(@ipam_objects) == 1 ) {
        die("Returned IPAM addresses number is not correct");
    }
    print("Returned IPAM addresses number correctly\n");

   # Case 5: Get IPAM address objects using a 'start_addr' that is close to the end of the network
   # This should return IPAM addresses from the start_addr to the end of the  network
    my @ipam_objects = $session->get('object'     => 'Infoblox::IPAM::Address',
                                     'network'    => '10.0.0.0/8',
                                     'start_addr' => '10.255.255.100');

    unless( scalar(@ipam_objects) == 156 ) {
        die("Returned IPAM addresses number is not correct");
    }
    print("Returned IPAM addresses number correctly\n");

    # Case 6: Get IPAM address objects using 'start_addr' and 'end_addr'
       # but the total number of IP addresses in this scope is more than 2000.
       # This should return 2000 IPAM addresses starting with the start_addr.
    my @ipam_objects = $session->get('object'     => 'Infoblox::IPAM::Address',
                                     'network'    => '10.0.0.0/8',
                                     'start_addr' => '10.0.0.100',
                                     'end_addr'   => '10.255.255.100');

    unless( scalar(@ipam_objects) == 2000 ) {
        die("Returned IPAM addresses number is not correct");
    }
    print("Returned IPAM addresses number correctly\n");

   # Case 7: Get IPAM address objects from a small network.
   # This should return all IP addresses in this network.
    my @ipam_objects = $session->get('object'       => 'Infoblox::IPAM::Address',
                                     'network'      => '20.0.0.0/24',
                                     'network_view' => 'my_network_view');

    unless( scalar(@ipam_objects) == 256 ) {
        die("Returned IPAM addresses number is not correct");
    }
    print("Returned IPAM addresses number correctly\n");

    # Case 8: Get IPAM address objects using 'type = used'
       # This should return a total of 11 IP addresses in this network
       # which includes 10 in the range and 1 fixed address
    my @ipam_objects = $session->get('object'     => 'Infoblox::IPAM::Address',
                                     'network'    => '10.0.0.0/8',
                                     'start_addr' => '10.0.0.0',
```

```
                                    'end_addr'   => '10.0.0.100',
                                    'type'       => 'used');

unless( scalar(@ipam_objects) == 11 ) {
    die("Returned IPAM addresses number is not correct");
}
print("Returned IPAM addresses number correctly\n");

# Case 9: Get IPAM address objects using 'type = unused'
   # This should return a total of 90 (101 - 11 = 90) IP addresses in this network
   # which excludes 10 in the range and 1 fixed address
my @ipam_objects = $session->get('object'     => 'Infoblox::IPAM::Address',
                                    'network'    => '10.0.0.0/8',
                                    'start_addr' => '10.0.0.0',
                                    'end_addr'   => '10.0.0.100',
                                    'type'       => 'unused');

unless( scalar(@ipam_objects) == 90 ) {
    die("Returned IPAM addresses number is not correct");
}
print("Returned IPAM addresses number correctly\n");

# Case 10: Get IPAM address objects using 'address'
   # This should return one IPAM address object for later tests
my @ipam_objects = $session->get('object'     => 'Infoblox::IPAM::Address',
                                    'address'    => '10.0.0.15');
unless( scalar(@ipam_objects) == 1 ) {
    die("Returned IPAM addresses number is not correct");
}
print("Returned IPAM addresses number correctly\n");

my $ipam_object = $ipam_objects[0];

unless($ipam_object->status() eq "Used") {
    die("Returned IPAM address object's status is wrong");
}
print("Returned IPAM address object's status correctly\n");

unless($ipam_object->usage() eq "DHCP") {
    die("Returned IPAM address object's usage is wrong");
}
print("Returned IPAM address object's usage correctly\n");

unless($ipam_object->mac_address() eq $fixed_addr->mac()) {
    die("Returned IPAM address object's mac address is wrong");
}
print("Returned IPAM address object's mac address correctly\n");

unless($ipam_object->dhcp_client_identifier() eq $fixed_addr->dhcp_client_identifier()) {
    die("Returned IPAM address object's dhcp client identifier is wrong");
}
print("Returned IPAM address object's dhcp client identifier correctly\n");

# Case 11: Get IPv6 IPAM address objects using 'network'
# This should return 3 used IPAM addresses
my @ipam_objects = $session->get('object'     => 'Infoblox::IPAM::Address',
                                    'network'    => '2001::/64',
                                    'network_view' => 'my_network_view');
unless( scalar(@ipam_objects) == 3) {
    die("Returned IPAM addresses number is not correct");
}
print("Returned IPAM addresses number correctly\n");

# Case 12: Get IPv6 IPAM address objects using 'start_addr'
# This should return 2 used IPAM addresses (2001::2, 2001::5)
my @ipam_objects = $session->get('object'     => 'Infoblox::IPAM::Address',
                                    'network'    => '2001::/64',
                                    'start_addr' => '2001::2',
                                    'network_view' => 'my_network_view');
unless( scalar(@ipam_objects) == 2) {
    die("Returned IPAM addresses number is not correct");
}
print("Returned IPAM addresses number correctly\n");

# Case 13: Get IPv6 IPAM address objects using 'start_addr' and 'end_addr'
   # This should return 2 used IPAM addresses
my @ipam_objects = $session->get('object'     => 'Infoblox::IPAM::Address',
                                    'network'    => '2001::/64',
```

```
                                    'start_addr' => '2001::1',
                                    'end_addr'   => '2001::3',
                                    'network_view' => 'my_network_view');
    unless( scalar(@ipam_objects) == 2 ) {
        die("Returned IPAM addresses number is not correct");
    }
    print("Returned IPAM addresses number correctly\n");

    # Case 14: Get IPv6 IPAM address objects using 'address'
    # This should return one IPAM address
    my @ipam_objects = $session->get('object'     => 'Infoblox::IPAM::Address',
                                    'address'    => '2001::5',
                                    'network_view' => 'my_network_view');
    unless( scalar(@ipam_objects) == 1 ) {
        die("Returned IPAM addresses number is not correct");
    }
    print("Returned IPAM addresses number correctly\n");

    my $ipam_object = $ipam_objects[0];

    unless($ipam_object->ip_address() eq "2001::5") {
        die("Returned IPAM address object's address is wrong");
    }
    print("Returned IPAM address object's address correctly\n");

    unless($ipam_object->status() eq "Used") {
        die("Returned IPAM address object's status is wrong");
    }
    print("Returned IPAM address object's status correctly\n");

    unless($ipam_object->usage() eq "DNS") {
        die("Returned IPAM address object's usage is wrong");
    }
    print("Returned IPAM address object's usage correctly\n");

# Remove IPAM Address object
unless($session->remove($ipam_object)) {
        die("Remove IPAM Address object failed");
}
print "Removed IPAM Address correctly\n";

my @retrieved_objs = $session->get(
     object       => "Infoblox::DHCP::Network" ,
     network      => "10.0.0.0/8",
);
my $network = $retrieved_objs[0];

unless($session->remove($network)) {
        die("Remove Network object failed");
}
print "Removed network correctly\n";

@retrieved_objs = $session->get(
     object       => "Infoblox::DHCP::Network" ,
     network      => "20.0.0.0/24",
     network_view => "my_network_view",
);

$network = $retrieved_objs[0];

unless($session->remove($network)) {
        die("Remove Network object failed");
}
print "Removed network correctly\n";

$view = $session->get(
    object  => "Infoblox::DNS::View",
    name    => "my_dns_view",
);
unless($session->remove($view)) {
        die("Remove DNS View object failed");
}
print "Removed DNS view correctly\n";

$nview = $session->get(
    object  => "Infoblox::DHCP::View",
```

```
    name    => "my_network_view",
);
unless($session->remove($nview)) {
        die("Remove Network View object failed");
}
print "Removed network view correctly\n";

$new_member = $session->get(
    object  => "Infoblox::Grid::Member",
    name    => "foo.com",
);
unless($session->remove($new_member)) {
        die("Remove Grid Member object failed");
}
print "Removed grid member correctly\n";

####PROGRAM ENDS####
```

AUTHOR

Infoblox Inc. http://www.infoblox.com/

SEE ALSO

Infoblox::Session, Infoblox::DHCP::Network, Infoblox::DHCP::View, Infoblox::DHCP::FixedAddr, Infoblox::DHCP::Range, Infoblox::Session->get(), Infoblox::Grid::MSServer::AdUser::Data, Infoblox::Grid::Discovery::Data

---

COPYRIGHT

Copyright (c) 2017 Infoblox Inc.

Infoblox::IPAM::DiscoveryTask - Network discovery configuration

- NAME
- DESCRIPTION
- CONSTRUCTOR
- SESSION METHODS
  - Infoblox::Session->get( )
  - Infoblox::Session->modify( )
- METHODS
  - disable_ip_scanning( )
  - disable_vmware_scanning( )
  - discovery_task_oid( )
  - member( )
  - mode( )
  - merge_data( )
  - networks( )
  - network_view( )
  - ping_retries( )
  - ping_timeout( )
  - scheduled_run( )
  - state( )
  - status( )
  - tcp_ports( )
  - tcp_scan_technique( )
  - update_metadata( )
  - vservers( )
  - v_network_view( )
- SAMPLE CODE
- AUTHOR
- SEE ALSO
- COPYRIGHT

NAME

Infoblox::IPAM::DiscoveryTask - Network discovery configuration

DESCRIPTION

This class represents the condiguration of network discovery jobs. Configuration parameters in this class have control over the behavior of network discovery jobs.

CONSTRUCTOR

Both current and scheduled Infoblox::IPAM::DiscoveryTask objects are automatically generated upon the successful installation of the Infoblox appliance. You do not need to create it.

SESSION METHODS

This section describes all the methods in the Infoblox::Session module that you can apply to a discovery task object.

Infoblox::Session->get( )

Use this method to retrieve matching objects from the Infoblox appliance. See Infoblox::Session->get() for parameters and return values.

Key Reference

```
Apply the following attributes to get a specific IPAM network discovery task object:

 object - Required. It should be set to "Infoblox::IPAM::DiscoveryTask".
 discovery_task_oid - Optional. The discovery task identifier that can be "current" or "scheduled".
```

Example

```
my @retrieved_objs = $session->get(
    object => "Infoblox::IPAM::DiscoveryTask",
    discovery_task_oid => "current",
    );
```

Infoblox::Session->modify( )

Use this method to modify objects in the Infoblox appliance. See Infoblox::Session->modify() for parameters and return values.

Example

```
#Use this method to modify the ping_timeout.
$discovery_task->ping_timeout("500");
#Submit modification
my $response = $session->modify( $discovery_task);
```

METHODS

This section describes all the methods that you can use to configure and retrieve the attribute values of the network discovery task.

disable_ip_scanning( )

Use this method to enable or disable IP discovery. This is a boolean value.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

Set the disable_ip_scanning attribute to "false" to enable IP discovery. Set the attribute to "true" to disable IP discovery. The default value is "false".

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get disable_ip_scanning attribute
my $disable_ip_scanning = $discovery_task->disable_ip_scanning();
#Modify disable_ip_scanning attribute
$discovery_task->disable_ip_scanning("true");
```

disable_vmware_scanning( )

Use this method to enable or disable VM discovery. This is a boolean value.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

Set the disable_vmware_scanning attribute to "false" to enable VM discovery. Set the attribute to "true" to disable VM discovery. The default value is "true".

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get disable_vmware_scanning attribute
my $disable_vmware_scanning = $discovery_task->disable_vmware_scanning();
#Modify disable_vmware_scanning attribute
$discovery_task->disable_vmware_scanning("true");
```

discovery_task_oid( )

Use this method to retrieve the discovery task identifier. This attribute is read-only and cannot be set.

Parameter

None.

Returns

The method returns the attribute value that can be "current" or "scheduled".

Example

```
#Get discovery_task_oid attribute
my $discovery_task_oid = $discovery_task->discovery_task_oid();
```

member( )

Use this method to set or retrieve the grid member that will run the discovery. Member is required for network discovery task object.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

Valid value is a defined Infoblox::DHCP::Member object. The default value is empty.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get member
my $member = $discovery_task->member();
#Modify member
$discovery_task->member($member1); #$member1 is an Infoblox::DHCP::Member object
```

mode( )

Use this method to set or retrieve the network discovery mode.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

Network discovery mode. Valid values when setting the mode are "FULL", "ICMP", "NETBIOS" and "TCP". Default value is "FULL".

When retrieving the network discovery mode, you can get any value listed above plus the value "CSV" if a CSV discovery job is running.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get mode
my $mode = $discovery_task->mode();
#Modify mode
$discovery_task->mode("ICMP");
```

merge_data( )

Use this method to set or retrieve the merge data flag that a network discovery uses.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

The flag indicates whether the GM sorter replaces or merges the discovered data with existing data. The valid values are "true" and "false". The default value is "true".

Returns

If you specified a parameter, the method returns "true" when the modification succeeds, and it returns "false" when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get merge_data
my $merge_data = $discovery_task->merge_data();
#Modify merge_data
$discovery_task->merge_data("false");
```

networks( )

Use this method to retrieve the DHCP networks on which network discovery jobs search for IP addresses.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

The valid value is an array reference that contains Infoblox::DHCP::Network objects. Networks must contain at least one Infoblox::DHCP::Network object.

Note: The discovery task object returned from the appliance may or may not contain Infoblox::DHCP::Network objects. Networks must contain at least one Infoblox::DHCP::Network object when calling modify

function.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
my $network1 = Infoblox::DHCP::Network->new(
                                        "network" => "10.0.0.0/24",
                                );
#Get networks
my $networks = $discovery_task->networks();
#Modify networks
$discovery_task->networks([$network1]);
```

network_view( )

Use this method to set or retrieve the network view of a discovery task.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

The default value is the default network view.

Parameter

The valid value is a defined Infoblox::DHCP::View object. The default value is the system-defined network view.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get network view
my $nview = $discovery_task->network_view();
#Modify network view, with an Infoblox::DHCP::View object
$discovery_task->network_view($nview);
```

ping_retries( )

Use this method to set or retrieve the network discovery ping retry times.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

Network discovery ping retry times. Minimium value is 1 and maximum value is 5. Default value is 2.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get ping_retries
my $ping_retries = $discovery_task->ping_retries();
#Modify ping_retries
$discovery_task->ping_retries("4");
```

ping_timeout( )

Use this method to set or retrieve the network discovery ping timeout.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

Network discovery ping timeout. Minimium value is 1 ms and maximum value is 4000 ms. Default value is 1000 ms.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get ping_timeout
my $ping_timeout = $discovery_task->ping_timeout();
#Modify ping_timeout
$discovery_task->ping_timeout("500");
```

scheduled_run( )

Use this method to set or retrieve the schedule of a discovery task.

Parameter

Valid value is a Infoblox::Grid::ScheduleSetting object. The default value is undef.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get scheduled_run
my $scheduled_run = $discovery_task->scheduled_run();
#Modify scheduled_run
$discovery_task->scheduled_run($schedule_setting);
```

state( )

Use this method to retrieve the network discovery state. The discovery state is read-only and cannot be set.

Parameter

None

Returns

Return the state of network discovery.

Example

```
#Get state
my $state = $discovery_task->state();
```

status( )

Use this method to retrieve the network discovery status. The discovery status is read-only and cannot be set.

Parameter

None

Returns

Return the status of network discovery.

Example

```
#Get status
my $status = $discovery_task->status();
```

tcp_ports( )

Use this method to set or retrieve the TCP ports that the network discovery jobs use.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

The valid value is an array reference that contains Infoblox::IPAM::TCPPort objects. Default TCP ports are:

```
ftp               21/tcp      # File Transfer [Control]
ssh               22/tcp      # Secure Shell Login
```

```
telnet            23/tcp     #
smtp              25/tcp     # Simple Mail Transfer
finger            79/tcp     #
http              80/tcp     # World Wide Web HTTP
kerberos-sec      88/tcp     # Kerberos (v5)
pop3              110/tcp    # PostOffice V.3
rpcbind           111/tcp    # portmapper, rpcbind
auth              113/tcp    # ident, tap, Authentication Service
msrpc             135/tcp    # Microsoft RPC services
netbios-ns        137/tcp    # NETBIOS Name Service
netbios-dgm       138/tcp    # NETBIOS Datagram Service
netbios-ssn       139/tcp    # NETBIOS Session Service
imap              143/tcp    # Interim Mail Access Protocol v2
https             443/tcp    # secure http (SSL)
microsoft-ds      445/tcp    # SMB directly over IP
login             513/tcp    # BSD rlogind(8)
shell             514/tcp    # BSD rshd(8)
printer           515/tcp    # spooler (lpd)
klogin            543/tcp    # Kerberos (v4/v5)
kerberos          750/tcp    # kdc Kerberos (v4)
kdm               1024/tcp   # K Display Manager (KDE version of xdm)
NFS-or-IIS        1025/tcp   # IIS, NFS, or listener RFS remote_file_sharing
```

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get tcp_ports
my $ports = $discovery_task->tcp_ports();

#Modify tcp_ports
my $port = Infoblox::IPAM::TCPPort->new(
       number => '333',
       comment => 'test_port');
push @$ports , $port;

$discovery_task->tcp_ports($ports);
```

tcp_scan_technique( )

Use this method to set or retrieve the TCP scan technique that the network discovery jobs use.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

Network discovery TCP scan technique. The scan techniques are identical to Nmap techniques. Valid values are "SYN" and "CONNECT". The default value is "SYN".

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get tcp_scan_technique
my $tcp_scan_technique= $discovery_task->tcp_scan_technique();
#Modify tcp_scan_technique
$discovery_task->tcp_scan_technique("CONNECT");
```

update_metadata( )

Use this method to set or retrieve the updated metadata flag that the network discovery jobs use.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

The flag indicates whether the appliance updates the metadata of the discovered objects. The valid values are "true" and "false". The default value is "true". If the value is "false", only the unmanaged objects are updated.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get update_metadata
my $update_metadata = $discovery_task->update_metadata();
#Modify update_metadata
$discovery_task->update_metadata("false");
```

vservers( )

Use this method to set or retrieve the list of VMware vSphere servers for VM discovery.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

The valid value is an array reference that contains the Infoblox::IPAM::DiscoveryTask::VServer objects. The default value is a reference to an empty list.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get vservers
my $vservers = $discovery_task->vservers();
#Modify vservers
my $vserver1 = Infoblox::IPAM::DiscoveryTask::VServer->new(
       fqdn_or_ip => "vcenter.com",
       username => "test1",
       password => "test1",
   );
my $vserver2 = Infoblox::IPAM::DiscoveryTask::VServer->new(
       fqdn_or_ip => "10.2.3.4",
       username => "test2",
       password => "test2",
   );
$discovery_task->vservers([$vserver1,$vserver2]);
```

v_network_view( )

Use this method to set or retrieve the network view of a VM discovery.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

The default value is the default network view.

Parameter

The valid value is a defined Infoblox::DHCP::View object. The default value is the system-defined network view.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get network view
my $nview = $discovery_task->v_network_view();
#Modify network view, with an Infoblox::DHCP::View object
$discovery_task->v_network_view($nview);
```

SAMPLE CODE

The following sample code demonstrates different operations that can be applied to a network discovery task object, such as getting and modifying an object. This sample code also includes error handling for the operations.

#Preparation prior to the get and modify operations on a network discovery task object

```
#PROGRAM STARTS: Include all the modules that will be used
use strict;
use Infoblox;

#refers to Infoblox Appliance IP address
```

```
my $host_ip   = "192.168.1.2";
my $host_name = "infoblox.localdomain";

#Create a session to the Infoblox appliance
my $session = Infoblox::Session->new(
    master   => $host_ip,
    username => "admin",
    password => "infoblox"
    )
    or die("Constructor for session failed: ",
               Infoblox::status_code(). ":" .Infoblox::status_detail());

print"Session created successfully.\n";

my  @result_array = $session->get(
      object  => "Infoblox::IPAM::DiscoveryTask" ,
);

my $discovery_task = $result_array[0];
unless (ref($discovery_task) eq 'Infoblox::IPAM::DiscoveryTask') {
       die("Get discovery task failed");
}
print "Get discovery task successfully\n";

#Create the Network object
my $network = Infoblox::DHCP::Network->new(
       network => "10.0.0.0/24",
);

$session->add($network)
   or die("Add network failed: " .
              $session->status_code() . ":" . $session->status_detail());
#modify networks
$discovery_task->networks([$network]);

# modify ping_timeout
$discovery_task->ping_timeout("500");

# modify tcp_ports
my $ports = $discovery_task->tcp_ports();

my $port = Infoblox::IPAM::TCPPort->new(
       number => '333',
       comment => 'test_port');
push @$ports , $port;

$discovery_task->tcp_ports($ports);

# submit changes
$session->modify( $discovery_task )
       or die("Modify discovery task failed: " .
              $session->status_code() . ":" . $session->status_detail());
print "Modify discovery task successfully\n";

# clean up
# remove network
$session->remove( $network)
   or die("Remove network failed: " .
         $session->status_code() . ":" . $session->status_detail());

####### Program Ends ############################
```

AUTHOR

Infoblox Inc. http://www.infoblox.com/

SEE ALSO

Infoblox::IPAM::DiscoveryTask::VServer,Infoblox::DHCP::Member,Infoblox::DHCP::Network,Infoblox::IPAM::TCPPort,Infoblox::Session,Infoblox::Session->get(),Infoblox::Session->modify(),Infoblox::Grid::ScheduleSetting

COPYRIGHT

Copyright (c) 2017 Infoblox Inc.

Infoblox::IPAM::DiscoveryTask::VServer - VMWare discovery server object.

- NAME
- DESCRIPTION
- CONSTRUCTOR
- MODULE METHODS
  - Infoblox::IPAM::DiscoveryTask->vservers( )
- METHODS
  - disable( )
  - encryption( )
  - fqdn_or_ip( )
  - password( )
  - port( )
  - username( )
- AUTHOR
- SEE ALSO
- COPYRIGHT

NAME

Infoblox::IPAM::DiscoveryTask::VServer - VMWare discovery server object.

DESCRIPTION

This object contains information about the vSphere servers on which the VM discovery is performed.

CONSTRUCTOR

```
my $vserver = Infoblox::IPAM::DiscoveryTask::VServer->new(
        fqdn_or_ip    => $string,           #Required.
        password      => $string,           #Required.
        username      => $string,           #Required.
        disable       => "true" | "false",  #Optional. Default is "false".
        encryption    => "true" | "false",  #Optional. Default is "true".
        port          => $num,              #Optional. Default is 443.
);
```

MODULE METHODS

You can apply the following functions to a VMware vSphere server.

Infoblox::IPAM::DiscoveryTask->vservers( )

Use this function to specify the vSphere server on which the VM discovery is performed. See Infoblox::IPAM::DiscoveryTask->vservers( ) for parameters and returned values.

Example

```
    my $vserver1 = Infoblox::IPAM::DiscoveryTask::VServer->new(
          fqdn_or_ip => "vcenter.com",
          username => "test1",
          password => "test1",
       );
    my $vserver2 = Infoblox::IPAM::DiscoveryTask::VServer->new(
          fqdn_or_ip => "10.2.3.4",
          username => "test2",
          password => "test2",
       );
     #Modifying vserver
     $discovery_task->vservers([$vserver1, $vserver2]);
```

METHODS

This section describes all the methods that can be used to retrieve the attribute values of a vSphere server.

disable( )

Use this method to set or retrieve the disable flag of a vSphere server.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

Specify "true" to disable the vSphere server or "false" to enable the server. The default value is "false".

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get disable flag
my $disable = $vserver->disable();
#Modify disable
$vserver->disable("true");
```

encryption( )

Use this method to set or retrieve the encryption flag of a vSphere server.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

Specify "true" to enable encryption or specify "false" to disable encryption on the vSphere server. The default value is "true".

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get encryption
my $encryption = $vserver->encryption();
#Modify encryption
$vserver->encryption("false");
```

fqdn_or_ip( )

Use this method to retrieve the FQDN (Fully Qualified Domain Name) or IP address of the vShpere server.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

The string that contains the FQDN or the IP address of the vSphere server.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get fqdn_or_ip
my $fqdn_or_ip = $vserver->fqdn_or_ip();
#Modify fqdn_or_ip
$vserver->fqdn_or_ip("vdiscovery.example.com");
```

password( )

Use this method to set the user password on the vSphere server. This is a write-only attribute.

Parameter

String containing password.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

Example

```
#Modify password
$vserver->password("infobloxx");
```

port( )

Use this method to retrieve the connection port that the vSphere server uses.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

Valid values are between 1 and 65535. The default value is 443.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get port
my $port = $vserver->port();
#Modify port
$vserver->port(444);
```

username( )

Use this method to set or retrieve the user name on the VSphere server.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

The username in the string format.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get username
my $username = $vserver->username();
#Modify username
$vserver->username("someuser");
```

AUTHOR

Infoblox Inc. http://www.infoblox.com/

SEE ALSO

Infoblox::IPAM::DiscoveryTask, Infoblox::IPAM::DiscoveryTask->vservers()

COPYRIGHT

Copyright (c) 2017 Infoblox Inc.

Infoblox::IPAM::Statistics - Manages IPAM statistics.

- NAME
- DESCRIPTION
- CONSTRUCTOR
- SESSION METHODS
  - Infoblox::Session->get( )
  - Infoblox::Session->search( )
- METHODS
  - utilization( )
  - conflict_count( )
  - ms_ad_user_data( )
  - unmanaged_count( )
  - utilization_update( )
- SAMPLE CODE
- AUTHOR
- SEE ALSO
- COPYRIGHT

NAME

Infoblox::IPAM::Statistics - Manages IPAM statistics.

DESCRIPTION

The IPAM statistics object is used to view the IPAM statistics of the network or network container in an Infoblox appliance.

CONSTRUCTOR

The Infoblox::IPAM::Statistics object does not require manual construction.

SESSION METHODS

This section describes all the methods in an Infoblox::Session that can be applied to a IPAM statistics object.

Infoblox::Session->get( )

Use this method to retrieve the existing objects from an Infoblox appliance. See Infoblox::Session->get() for parameters and return values.

Key Reference

```
 Apply the following attributes to get the IPAM statistics:

 network      - Required. An IPv4 address in dotted decimal notation followed by the network prefix
(e.g 1.0.0.0/8).
 network_view - Optional. The name of the network view in which the network object is located. By
default, all network views are searched.
```

Example

```
my $ipam_stats = $session->get(
         object  => "Infoblox::IPAM::Statistics",
         network => "1.0.0.0/8",
         network_view => "my_network_view");
```

Infoblox::Session->search( )

Use this method to search for an IPAM statistics object from an Infoblox appliance. See Infoblox::Session->search() for parameters and return values.

Key Reference

```
 Apply the following attributes to get the IPAM statistics:

 network      - Required. An IPv4 address in dotted decimal notation followed by the network prefix
(e.g 1.0.0.0/8) (regular expression).
 network_view - Optional. The name of the network view in which the network object is located. By
default, all network views are searched.
```

Example

```
my $ipam_stats = $session->search(
         object  => "Infoblox::IPAM::Statistics",
         network => '1\.0\.0\..*',
         network_view => "my_network_view");
```

METHODS

This section describes all the methods that can be used to retrieve the attribute values of a IPAM statistics object.

utilization( )

Use this method to retrieve the network utilization.

Parameter

none

Returns

The method returns the network utilization in percentage.

Example

```
#Get utilization
my $utilization = $ipam_stats->utilization();
```

conflict_count( )

Use this method to retrieve the number of conflicts discovered via network discovery. This method is only valid for a Network object.

Parameter

none

Returns

The method returns the number of discovered conflicts. For a NetworkContainer object, the return value is undefined.

Example

```
#Get conflict_count
my $conflict_count = $ipam_stats->conflict_count();
```

ms_ad_user_data( )

Use this method to retrieve Microsoft Active Directory users related information. This is a read-only attribute.

Parameter

None

Returns

The valid return value is an Infoblox::Grid::MSServer::AdUser::Data object.

Example

```
#Get ms_ad_user_data
my $ms_ad_user_data = $ipam_stats->ms_ad_user_data();
```

unmanaged_count( )

Use this method to retrieve the number of unmanaged IP addresses as discovered by network discovery. This method is only valid for a Network object.

Parameter

none

Returns

The method returns the number of unmanaged IP addresses. For a NetworkContainer object, the return value is undefined.

Example

```
#Get unmanaged_count
my $unmanaged_count = $ipam_stats->unmanaged_count();
```

utilization_update( )

Use this method to retrieve the time that the utilization statistics were updated last. This method is only valid for a Network object.

Parameter

none

Returns

The method returns a timestamp value in the ISO 8601 extended format for date and time (e.g. 2008-10-21T10:56:33Z or 2008-10-21T10:56:33.001Z). For a NetworkContainer object, the return value is undefined.

Example

```
#Get utilization_update
my $utilization_update = $ipam_stats->utilization_update();
```

SAMPLE CODE

The following sample code demonstrates the get and search session methods on an IPAM statistics object. Also, this sample code also includes error handling for the operations.

```
#PROGRAM STARTS: Include all the modules that will be used
use strict;
use Infoblox;
```

#Preparation prior to getting IPAM statistics

```
#Create a session to the Infoblox appliance
my $session = Infoblox::Session->new(
    master   => "192.168.1.2",
    username => "admin",
    password => "infoblox"
);
unless ($session) {
   die("Construct session failed: ",
       Infoblox::status_code() . ":" . Infoblox::status_detail());
}
print "Session created successfully\n";

my $member = Infoblox::DHCP::Member->new(
    name     => "infoblox.localdomain",
    ipv4addr => "192.168.1.2",
);

unless($member) {
     die("Construct member failed: ",
           Infoblox::status_code() . ":" . Infoblox::status_detail());
}
print "Member object created successfully\n";

#Create a Network View object
my $network_view = Infoblox::DHCP::View->new(
                   name => "my_network_view",
);
unless($network_view) {
            die("Construct Network View object failed: ",
                  Infoblox::status_code() . ":" . Infoblox::status_detail());
}
print "Network View object created successfully\n";

#Add the Network View object into the Infoblox appliance through a session
$session->add($network_view)
     or die("Add Network View object failed: ",
           $session->status_code() . ":" . $session->status_detail());
print "Network View added successfully\n";

#Create the Network object with this member
my $network = Infoblox::DHCP::Network->new(
                   network => "1.0.0.0/255.0.0.0",
                   members => [ $member ],
                   network_view => $network_view,
);
unless($network) {
```

```
            die("Construct Network object failed: ",
                  Infoblox::status_code() . ":" . Infoblox::status_detail());
}
print "Network object created successfully\n";

#Add the Network object into the Infoblox appliance through a session
$session->add($network)
     or die("Add Network object failed: ",
           $session->status_code() . ":" . $session->status_detail());
print "Network added successfully\n";

#Create the DHCP Range object with this member
my $dhcp_range = Infoblox::DHCP::Range->new(
    network    => "1.0.0.0/8",
    start_addr => "1.0.0.1",
    end_addr   => "1.0.0.10",
    member     => $member,
    network_view => $network_view,
);
unless($dhcp_range) {
     die("Construct DHCP Range failed: ",
           Infoblox::status_code() . ":" . Infoblox::status_detail());
}
print "DHCP range object created successfully\n";

$session->add($dhcp_range)
     or die("Add DHCP range object failed: ",
           $session->status_code() . ":" . $session->status_detail());
print "DHCP range added successfully\n";

#Create the Fixed Address object
my $fixed_address = Infoblox::DHCP::FixedAddr->new(
                                              "network" => "1.0.0.0/8",
                                              "ipv4addr" => "1.0.0.2",
                                              "mac" => "22:22:22:22:22:22",
                                              "network_view" => $network_view,
                                              );
unless($fixed_address) {
     die("Construct DHCP Fixed address failed: ",
           Infoblox::status_code() . ":" . Infoblox::status_detail());
}
print "DHCP Fixed address object created successfully\n";

$session->add($fixed_address)
     or die("Add DHCP Fixed address object failed: ",
           $session->status_code() . ":" . $session->status_detail());
print "DHCP Fixed address added successfully\n";
```

#Get the IPAM statistics

```
 my @result_array = $session->get(
          object  => "Infoblox::IPAM::Statistics",
          network => "1.0.0.0/8",
          network_view => "my_network_view");

 my $ipam_stats = $result_array[0];
 unless ($ipam_stats) {
      die("Get IPAM statistics failed: ",
            $session->status_code() . ":" . $session->status_detail());
 }
 print "Get IPAM Statistics successful\n";

 print "Utilization for '1.0.0.0/8': ", $ipam_stats->utilization(), "\n";
```

#Search for the IPAM statistics

```
 my @result_array = $session->search(
          object  => "Infoblox::IPAM::Statistics",
          network => '1\.0\.0\..*',
          network_view => "my_network_view");

 my $ipam_stats = $result_array[0];
 unless ($ipam_stats) {
      die("Get IPAM statistics failed: ",
            $session->status_code() . ":" . $session->status_detail());
 }
```

```
print "Get IPAM Statistics successful\n";

print "Utilization for '1.0.0.0/8': ", $ipam_stats->utilization(), "\n";
```

#Clean up

```
# Remove the IPAM Statistics object that was just retrieved
$session->remove($ipam_stats);

# Retrieve and remove the Network View object
my $network_view = $session->get(
        object => "Infoblox::DHCP::View",
        name => "my_network_view",
);
unless ($network_view) {
     die("Get Network View failed: ",
           $session->status_code() . ":" . $session->status_detail());
}
$session->remove($network_view);

####PROGRAM ENDS####
```

AUTHOR

Infoblox Inc. http://www.infoblox.com/

SEE ALSO

Infoblox::Session, Infoblox::DHCP::Network,Infoblox::DHCP::View,Infoblox::DHCP::FixedAddr,Infoblox::DHCP::Range,Infoblox::Session->add(),Infoblox::Session->get(), Infoblox::Grid::MSServer::AdUser::Data

---

COPYRIGHT

Copyright (c) 2017 Infoblox Inc.

Infoblox::IPAM::TCPPort - Network discovery TCP port

- NAME
- DESCRIPTION
- CONSTRUCTOR
- SESSION METHODS
- METHODS
  - comment( )
  - number( )
- SAMPLE CODE
- AUTHOR
- SEE ALSO
- COPYRIGHT

NAME

Infoblox::IPAM::TCPPort - Network discovery TCP port

DESCRIPTION

This class represents the TCP port that network discovery jobs use to find IP addresses.

CONSTRUCTOR

```
my $tcp_port = Infoblox::IPAM::TCPPort->new(
     number             => $string,             #Required.
     comment            => $string,             #Optional.
);
```

SESSION METHODS

This object does not have a session method. You must use it with Infoblox::IPAM::DiscoveryTask class.

METHODS

This section describes all the methods that you can use to configure and retrieve the attribute values of a TCP port.

comment( )

Use this method to set or retrieve a descriptive comment for a TCP port.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

Enter a descriptive comment for the TCP port in string format.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
# Get comment
my $comment = $tcp_port->comment();
# Modify comment
$tcp_port->comment("new port");
```

number( )

Use this method to set or retrieve the TCP port number.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

Enter a number the TCP port.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
# Get port number
my $port_number = $tcp_port->number();
# Modify port number
$tcp_port->number("222");
```

SAMPLE CODE

Please see sample code in Infoblox::IPAM::DiscoveryTask.

AUTHOR

Infoblox Inc. http://www.infoblox.com/

SEE ALSO

Infoblox::IPAM::DiscoveryTask

COPYRIGHT

Copyright (c) 2017 Infoblox Inc.

Infoblox::LDAP::AuthService - An LDAP Authentication Service object.

- NAME
- DESCRIPTION
- CONSTRUCTOR
- SESSION METHODS
  - Infoblox::Session->add( )
  - Infoblox::Session->get( )
  - Infoblox::Session->modify( )
  - Infoblox::Session->remove( )
  - Infoblox::Session->search( )
- METHODS
  - comment( )
  - disable( )
  - ea_mapping( )
  - mode( )
  - name( )
  - ldap_group_attribute( )
  - ldap_group_authentication_type( )
  - ldap_user_attribute( )
  - recovery_interval( )
  - retries( )
  - search_scope( )
  - servers( )
  - timeout( )
- SAMPLE CODE
- AUTHOR
- SEE ALSO
- COPYRIGHT

NAME

Infoblox::LDAP::AuthService - An LDAP Authentication Service object.

DESCRIPTION

The LDAP Authentication Service.

CONSTRUCTOR

```
 my $service = Infoblox::LDAP::AuthService->new(
     name                          => $string,                                  # Required
     servers                       => [$server1, $server2, ...],                # Required, These are
Infoblox::LDAP::Server objects
     timeout                       => $uint,                                    # Optional, Default is 5
     retries                       => $uint,                                    # Optional, Default is 5
     recovery_interval             => $uint,                                    # Optional, Default is 30
     mode                          => 'ORDERED_LIST'| 'ROUND_ROBIN',            # Optional, Default is
'ORDERED_LIST'
     ldap_user_attribute           => $string,                                  # Required
     ldap_group_attribute          => $string,                                  # Optional, Default is 'memberOf'
     search_scope                  => BASE' | 'ONELEVEL' | 'SUBTREE',          # Optional, Default is 'ONELEVEL'
     ea_mapping                    => [$ea_mapping1, $ea_mapping, ...],         # Optional, These are
Infoblox::LDAP::EA_Mapping objects
     comment                       => $string,                                  # Optional
     disable                       => 'true' | 'false',                         # Optional, Default is 'false'
     ldap_group_authentication_type => 'GROUP_ATTRIBUTE' | 'POSIX_GROUP',       # Optional, Default is
'GROUP_ATTRIBUTE'
 );
```

SESSION METHODS

This section describes all the methods in an Infoblox::Session module that you can apply to an LDAP Authentication Service object.

Infoblox::Session->add( )

Use this method to add an object to the Infoblox appliance. See Infoblox::Session->add() for parameters and return values.

Example

```
#Construct an object
my $service = Infoblox::LDAP::AuthService->new(
   name                => 'LDAPAuthService',
   servers             => [$server],
   ldap_user_attribute => 'cn',
);

#Submit for addition
my $response = $session->add( $service );
```

Infoblox::Session->get( )

Use this method to retrieve all the matching objects from the Infoblox appliance. See Infoblox::Session->get() for parameters and return values.

Key References

```
Apply the following attributes to get a specific LDAP Authentication Service object:

 name         - Optional. The LDAP Authentication Service name.
 comment      - Optional. The comment.
 search_scope - Optional. The starting point of the LDAP search.
 mode         - Optional. The LDAP authentication mode.
```

Example

```
my @retrieved_objs = $session->get(
   object => 'Infoblox::LDAP::AuthService',
   name   => 'LDAPAuthService',
);
```

Infoblox::Session->modify( )

Use this method to modify an object in the Infoblox appliance. See Infoblox::Session->modify() for parameters and return values.

Example

```
# Use this method to modify comment of a LDAP Authentication Service.
$service->comment("This is modified object");
# Submit modification
my $response = $session->modify( $service );
```

Infoblox::Session->remove( )

Use this method to remove an object from the Infoblox appliance. See Infoblox::Session->remove() for parameters and return values.

To remove a specific object, first use `get()` or `search()` to retrieve the specific object, and then submit this object for removal.

Example

```
#Get the objects with the same name
my @retrieved_objs = $session->get(
   object => 'Infoblox::LDAP::AuthService',
   name   => 'LDAPAuthService',
)
#Find the desired object from the retrieved list.
my $desired_device = $retrieved_objs[0];
# Submit for removal
my $response = $session->remove( $desired_device );
```

Infoblox::Session->search( )

Use this method to search for device objects in the Infoblox appliance. See Infoblox::Session->search() for parameters and return values.

Key References

```
Apply the following attributes to search a specific LDAP Authentication Service object:

 name         - Optional. The LDAP Authentication Service name.
 comment      - Optional. The comment.
 search_scope - Optional. The starting point of the LDAP search.
 mode         - Optional. The LDAP authentication mode.
```

Example

```
my @retrieved_objs = $session->search(
   object => 'Infoblox::LDAP::AuthService',
```

```
    name    => 'LDAPAuthService',
);
```

METHODS

This section describes all the methods that you can use to configure and retrieve the attribute values of an LDAP Authentication Service object.

comment( )

Use this method to set or retrieve a descriptive comment.

Parameter

A desired comment in string format with a maximum of 256 bytes.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get attribute value
my $value = $service->comment();
#Modify attribute value
$service->comment('LDAP Authentication Service');
```

disable( )

Use this method to enable or disable an LDAP Authentication Service.

Parameter

Specify "true" to disable an LDAP Authentication Service or "false" to enable it. The default value is 'false'.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get attribute value
my $value = $service->disable();
#Modify attribute value
$service->disable('true');
```

ea_mapping( )

Use this method to set or retrieve mapping of LDAP fields to extensible attributes.

Parameter

An array of an Infoblox::LDAP::EA_Mapping objects.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get attribute value
my $value = $service->ea_mapping();
#Modify attribute value
$service->ea_mapping([$ea_mapping1, $ea_mapping2]);
```

mode( )

Use this method to set or retrieve an LDAP authentication mode.

Parameter

Valid values are 'ORDERED_LIST' and 'ROUND_ROBIN'. The default value is 'ORDERED_LIST'.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get attribute value
my $value = $service->mode();
#Modify attribute value
$service->mode('ROUND_ROBIN');
```

name( )

Use this method to set or retrieve an LDAP Authentication Service name.

Parameter

An LDAP Authentication Service name in string format.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get attribute value
my $value = $service->name();
#Modify attribute value
$service->name('NewLDAPAuthService');
```

ldap_group_attribute( )

Use this method to set or retrieve the name of the LDAP attribute that defines group membership.

Parameter

A name of the LDAP attribute in string format. The default value is 'memberOf'.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get attribute value
my $value = $service->ldap_group_attribute();
#Modify attribute value
$service->ldap_group_attribute('memberOf');
```

ldap_group_authentication_type( )

Use this method to set or retrieve the LDAP group authentication type.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

The valid values are 'GROUP_ATTRIBUTE' and 'POSIX_GROUP'.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get ldap_group_authentication_type value
my $value = $service->ldap_group_authentication_type();
#Modify ldap_group_authentication_type value
$service->ldap_group_authentication_type('POSIX_GROUP');
```

ldap_user_attribute( )

Use this method to set or retrieve an LDAP userid attribute that is used for search.

Parameter

An LDAP userid attribute in string format.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get attribute value
my $value = $service->ldap_user_attribute();
#Modify attribute value
$service->ldap_user_attribute('cn');
```

recovery_interval( )

Use this method to set or retrieve the period of time to wait before trying to contact an LDAP server that has been marked as 'down'.

Parameter

The period of time in seconds. The default value is 30.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get attribute value
my $value = $service->recovery_interval();
#Modify attribute value
$service->recovery_interval(40);
```

retries( )

Use this method to set or retrieve maximum number of authentication attempts.

Parameter

Maximum number of authentication attempts. The default value is 5.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get attribute value
my $value = $service->retries();
#Modify attribute value
$service->retries(4);
```

search_scope( )

Use this method to set or retrieve the starting point of an LDAP search.

Parameter

Valid values are 'BASE', 'ONELEVEL' and 'SUBTREE'. The default value is 'ONELEVEL'.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get attribute value
my $value = $service->search_scope();
#Modify attribute value
$service->search_scope('ONELEVEL');
```

servers( )

Use this method to set or retrieve the list of LDAP servers used for authentication.

Parameter

An array of an Infoblox::LDAP::Server objects.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get attribute value
my $value = $service->servers();
#Modify attribute value
$service->servers([$server1, $server2]);
```

timeout( )

Use this method to set or retrieve timeout for authentication.

Parameter

Timout in seconds. The default value is 5.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get attribute value
my $value = $service->timeout();
#Modify attribute value
$service->timeout(4);
```

SAMPLE CODE

The following sample code demonstrates the different functions that can be applied to an object such as add, get, modify, search and remove. This sample also includes error handling for the operations.

#Preparation prior to getting and modifying a LDAP Authentication Service object

```
use strict;
use Infoblox;

#refers to Infoblox Appliance IP address
my $host_ip = "192.168.1.2";

#Create a session to the Infoblox appliance

my $session = Infoblox::Session->new(
    master   => $host_ip,
    username => "admin",
    password => "infoblox"
);
unless ($session) {
       die("Construct session failed: ",
               Infoblox::status_code() . ":" . Infoblox::status_detail());
}
print "Session created successfully\n";
```

#Create a LDAP Authentication Service object

```
my $server = Infoblox::LDAP::Server->new(
    'address' => '10.0.0.1',
```

```
    'base_dn' => 'dc=infoblox,dc=com',
);

unless ($server) {
       die("Construct LDAP server object failed: ",
               Infoblox::status_code() . ":" . Infoblox::status_detail());
}
print "LDAP server object created successfully\n";

my $service = Infoblox::LDAP::AuthService->new(
    'name'                => 'papi-ldap-service',
    'servers'             => [$server],
    'ldap_user_attribute' => 'cn',
);

unless ($service) {
       die("Construct LDAP Authentication Service object failed: ",
               Infoblox::status_code() . ":" . Infoblox::status_detail());
}
print "LDAP Authentication Service object created successfully\n";

my $response = $session->add($service);

unless ($response) {
    die("Add LDAP Authentication Service failed: ",
          $session->status_code() . ":" . $session->status_detail());
}
print "LDAP Authentication Service added successfully \n";
```

#Search for a LDAP Authentication Service object

```
my @retrieved_objs = $session->search(
   object => 'Infoblox::LDAP::AuthService',
   name   => 'papi-ldap-service',
);

$service = $retrieved_objs[0];

unless ($service) {
        die("Search LDAP Authentication Service object failed: ",
                $session->status_code() . ":" . $session->status_detail());
}
print "Search LDAP Authentication Service object found at least 1 matching entry\n";
```

#Get and modify the LDAP Authentication Service object

```
@retrieved_objs = $session->get(
   object => 'Infoblox::LDAP::AuthService',
   name   => 'papi-ldap-service',
);

$service = $retrieved_objs[0];

unless ($service) {
    die("Get LDAP Authentication Service object failed: ",
          $session->status_code() . ":" . $session->status_detail());
}
print"Get LDAP Authentication Service successful \n";

$service->comment("This is modified object");

$session->modify($service)
   or die("Modify LDAP Authentication Service failed",
           $session->status_code() . ":" . $session->status_detail());
print "LDAP Authentication Service modified successfully \n";
```

#Remove the LDAP Authentication Service object

```
@retrieved_objs = $session->get(
   object => 'Infoblox::LDAP::AuthService',
   name   => 'papi-ldap-service',
);

$service = $retrieved_objs[0];

unless ($service) {
```

```
    die("Get LDAP Authentication Service object failed: ",
            $session->status_code() . ":" . $session->status_detail());
}
print"Get LDAP Authentication Service successful \n";

$session->remove( $service )
    or die("Remove LDAP Authentication Service failed",
            $session->status_code() . ":" . $session->status_detail());
print "LDAP Authentication Service removed successfully \n";

####PROGRAM ENDS####
```

AUTHOR

Infoblox Inc. http://www.infoblox.com/

SEE ALSO

Infoblox::LDAP::Server, Infoblox::LDAP::EA_Mapping

COPYRIGHT

Copyright (c) 2017 Infoblox Inc.

Infoblox::LDAP::EA_Mapping - An LDAP Extensible Attribute Mapping object.

- NAME
- DESCRIPTION
- CONSTRUCTOR
- SESSION METHODS
- METHODS
  - attr_name( )
  - ext_attr_def( )
- AUTHOR
- SEE ALSO
- COPYRIGHT

NAME

Infoblox::LDAP::EA_Mapping - An LDAP Extensible Attribute Mapping object.

DESCRIPTION

Mapping of LDAP field to Extensible Attribute.

CONSTRUCTOR

```
my $ea_mapping = Infoblox::LDAP::EA_Mapping->new(
   attr_name    => $string,
   ext_attr_def => $attr_def,                    # An Infoblox::Grid::ExtensibleAttributeDef object
);
```

SESSION METHODS

The LDAP Extensible Attribute Mapping object does not support any session methods.

METHODS

This section describes all the methods that you can use to configure and retrieve the attribute values of an LDAP Extensible Attribute Mapping object.

attr_name( )

Use this method to set or retrieve the name of an LDAP attribute.

Parameter

The name of an LDAP attribute in string format.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get attribute value
my $value = $ea_mapping->attr_name();
#Modify attribute value
$ea_mapping->attr_name('homephone');
```

ext_attr_def( )

Use this method to set or retrieve the definition of an extensible attribute.

Parameter

An Infoblox::Grid::ExtensibleAttributeDef object.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get attribute value
my $value = $ea_mapping->ext_attr_def();
#Modify attribute value
$ea_mapping->ext_attr_def($attr_def);
```

AUTHOR

Infoblox Inc. http://www.infoblox.com/

SEE ALSO

Infoblox::LDAP::AuthService, Infoblox::Grid::ExtensibleAttributeDef

---

COPYRIGHT

Copyright (c) 2017 Infoblox Inc.

Infoblox::LDAP::Server - An LDAP Server object.

- NAME
- DESCRIPTION
- CONSTRUCTOR
- SESSION METHODS
- METHODS
  - address( )
  - authentication_type( )
  - base_dn( )
  - bind_password( )
  - bind_user_dn( )
  - comment( )
  - encryption( )
  - disable( )
  - port( )
  - use_mgmt_port( )
  - version( )
- AUTHOR
- SEE ALSO
- COPYRIGHT

NAME

Infoblox::LDAP::Server - An LDAP Server object.

DESCRIPTION

The LDAP Server is used for authentication.

CONSTRUCTOR

```
my $server = Infoblox::LDAP::Server->new(
    address             => $address_or_fqdn,              # Required
    base_dn             => $string,                       # Required
    bind_user_dn        => $string,                       # Required if authentication_type is
'AUTHENTICATED'
    bind_password       => $string,                       # Required if authentication_type is
'AUTHENTICATED', write-only
    port                => $uint,                         # Optional, Default is 636
    version             => 'V2' | 'V3',                   # Optional, Default is 'V3'
    comment             => $string,                       # Optional
    authentication_type => 'ANONYMOUS' | 'AUTHENTICATED', # Optional, Default is 'ANONYMOUS'
    encryption          => 'NONE' | 'SSL',                # Optional, Default is 'SSL'
    use_mgmt_port       => 'true' | 'false',              # Optional, Default is 'false'
    disable             => 'true' | 'false',              # Optional, Default is 'false'
);
```

SESSION METHODS

The LDAP Server object does not support any session methods.

METHODS

This section describes all the methods that you can use to configure and retrieve the attribute values of an LDAP server object.

address( )

Use this method to set or retrieve an IP address or FQDN of an LDAP server.

Parameter

An IP address or FQDN of an LDAP server in string format.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get attribute value
my $value = $server->address();
#Modify attribute value
$server->address('10.0.0.1');
```

authentication_type( )

Use this method to set or retrieve an authentication type.

Parameter

Valid values are 'ANONYMOUS' and 'AUTHENTICATED'. The default value is 'ANONYMOUS'.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get attribute value
my $value = $server->authentication_type();
#Modify attribute value
$server->authentication_type('AUTHENTICATED');
```

base_dn( )

Use this method to set or retrieve a base DN for the LDAP server.

Parameter

A base DN in string format.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get attribute value
my $value = $server->base_dn();
#Modify attribute value
$server->base_dn('dc=infoblox,dc=com');
```

bind_password( )

Use this method to set a user password for authentication. This attribute is required if authentication_type attribute value is 'AUTHENTICATED'. This is a write-only attribute.

Parameter

An user password in string format.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

Example

```
#Modify attribute value
$server->bind_password('password');
```

bind_user_dn( )

Use this method to set or retrieve an user DN for authentication. This attribute is required if authentication_type attribute value is 'AUTHENTICATED'.

Parameter

An user DN in string format.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get attribute value
my $value = $server->bind_user_dn();
#Modify attribute value
$server->bind_user_dn('cn=user,dc=infoblox,dc=com');
```

comment( )

Use this method to set or retrieve a descriptive comment.

Parameter

A desired comment in string format with a maximum of 256 bytes.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get attribute value
my $value = $server->comment();
#Modify attribute value
$server->comment('LDAP object');
```

encryption( )

Use this method to set or retrieve an encryption type.

Parameter

Valid values are 'NONE' and 'SSL'. The default value is 'SSL'.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get attribute value
my $value = $server->encryption();
#Modify attribute value
$server->encryption('SSL');
```

disable( )

Use this method to enable or disable an LDAP server.

Parameter

Specify "true" to disable an LDAP server or "false" to enable it. The default value is 'false'.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get attribute value
my $value = $server->disable();
#Modify attribute value
$server->disable('true');
```

port( )

Use this method to set or retrieve a port number.

Parameter

A port number. The default value is 636.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get attribute value
my $value = $server->port();
#Modify attribute value
$server->port(389);
```

use_mgmt_port( )

Use this method to set or retrieve the flag that allows connections through the management interface.

Parameter

Specify "true" to allow or "false" to deny it. The default value is 'false'.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get attribute value
my $value = $server->use_mgmt_port();
#Modify attribute value
$server->use_mgmt_port('true');
```

version( )

Use this method to set or retrieve an LDAP server version.

Parameter

Valid values are 'V2' and 'V3'. The default value is 'V3'.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get attribute value
my $value = $server->version();
#Modify attribute value
$server->version('V2');
```

AUTHOR

Infoblox Inc. http://www.infoblox.com/

SEE ALSO

Infoblox::LDAP::AuthService

COPYRIGHT

Copyright (c) 2017 Infoblox Inc.

Infoblox::MasterGrid::Grid - Manages the Master Grid settings.

- NAME
- DESCRIPTION
- CONSTRUCTOR
- SESSION METHODS
  - Infoblox::Session->get( )
  - Infoblox::Session->search( )
  - Infoblox::Session->modify( )
- METHODS
  - address( )
  - connection_timestamp( )
  - detached( )
  - enabled( )
  - joined( )
  - last_event( )
  - last_event_details( )
  - last_sync_timestamp( )
  - port( )
  - status( )
- AUTHOR
- SEE ALSO
- COPYRIGHT

NAME

Infoblox::MasterGrid::Grid - Manages the Master Grid settings.

DESCRIPTION

This class manages the Master Grid.

CONSTRUCTOR

The Infoblox::MasterGrid::Grid object is automatically generated when a Grid successfully joins the Master Grid. It does not require manual construction.

SESSION METHODS

This section describes all the methods in Infoblox::Session that you can apply to a Master Grid object.

Infoblox::Session->get( )

Use this method to retrieve the existing objects from the Infoblox appliance. See Infoblox::Session->get() for parameters and return values.

Key References

```
Apply the following attributes to get a specific Master Grid object:

name    - Optional. The Master Grid name in string format.
port    - Optional. The Master Grid port number.
```

Example

```
my $grid = $session->get(
    object => "Infoblox::MasterGrid::Grid");
```

Infoblox::Session->search( )

Use this method to search for the existing objects from the Infoblox appliance. See Infoblox::Session->search() for parameters and return values.

Key References

```
Apply the following attributes to get a specificMaster Grid object:

name    - Optional. The Master Grid name in string format. Regular expression is supported.
port    - Optional. The Master Grid port number.
```

Example

```
my $grid = $session->search(
    object => "Infoblox::MasterGrid::Grid");
```

Infoblox::Session->modify( )

Use this method to modify objects in the Infoblox appliance. See Infoblox::Session->modify() for parameters and return values.

Example

```
# Use this method to modify the enabled status
$grid->enabled("false");
# Submit modification
my $response = $session->modify( $grid );
```

METHODS

This section describes all the methods that you can use to configure and retrieve the attribute values of a Master Grid.

address( )

Use this method to set or retrieve the domain name or IP address for the Master Grid.

Parameter

Valid values are a domain name in FQDN format or an IP address.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get address
my $address = $grid->address();
#Modify address
$grid->address("10.0.0.1");
```

connection_timestamp( )

Use this method to retrieve the timestamp that indicates when the connection to the Master Grid was established.

Parameter

none

Returns

The method returns the timestamp.

Example

```
#Get the Master Grid connection_timestamp
my $connection_timestamp = $mg->connection_timestamp();
```

detached( )

Use this method to retrieve the detached flag for the Master Grid.

Parameter

none

Returns

The method returns "true" or "false".

Example

```
#Get detached flag
my $detached = $grid->detached();
```

enabled( )

Use this method to set or retrieve the enabled flag for the Master Grid.

Parameter

Valid values are "true" or "false".

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get enabled flag
my $enabled = $grid->enabled();
#Modify enabled flag
$grid->enabled("false");
```

joined( )

Use this method to retrieve the status of the Grid's join operation to the Master Grid.

Parameter

none

Returns

The method returns "true" or "false".

Example

```
#Get the join status of the Grid to the Master Grid
my $joined = $mg->joined();
```

last_event( )

Use this method to retrieve the Master Grid's last event.

Parameter

none

Returns

The method returns the Master Grid's last event.

Example

```
#Get the Master Grid last_event
my $last_event = $mg->last_event();
```

last_event_details( )

Use this method to retrieve the details of the Master Grid's last event.

Parameter

none

Returns

The method returns details on the Master Grid's last event.

Example

```
#Get the Master Grid last_event_details
my $last_event_details = $mg->last_event_details();
```

last_sync_timestamp( )

Use this method to retrieve the timestamp for the last synchronization operation with the Master Grid.

Parameter

none

Returns

The method returns the timestamp.

Example

```
#Get the Master Grid last_sync_timestamp
my $last_sync_timestamp = $mg->last_sync_timestamp();
```

port( )

Use this method to set or retrieve the Master Grid port to which the Grid connects.

Parameter

Valid value is a port number from 0 to 65535.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get port
my $port = $grid->port();
#Modify port
$grid->port(1234);
```

status( )

Use this method to retrieve the Master Grid's status.

Parameter

none

Returns

The method returns the Master Grid's status in string format.

Example

```
#Get the Master Grid status
my $status = $mg->status();
```

AUTHOR

Infoblox Inc. http://www.infoblox.com/

SEE ALSO

Infoblox::Session, Infoblox::Session->get(), Infoblox::Session->search(), Infoblox::Session->modify()

COPYRIGHT

Copyright (c) 2017 Infoblox Inc.

Infoblox::Notification::REST::Endpoint - A REST API endpoint object.

- NAME
- DESCRIPTION
- CONSTRUCTOR
- SESSION METHODS
  - Infoblox::Session->add( )
  - Infoblox::Session->get( )
  - Infoblox::Session->modify( )
  - Infoblox::Session->remove( )
  - Infoblox::Session->search( )
- METHODS
  - client_certificate( )
  - client_certificate_subject( )
  - client_certificate_valid_from( )
  - client_certificate_valid_to( )
  - comment( )
  - disable( )
  - extattrs( )
  - extensible_attributes( )
  - name( )
  - outbound_members( )
  - outbound_member_type( )
  - log_level( )
  - password( )
  - server_cert_validation( )
  - template_instance( )
  - timeout( )
  - uri( )
  - username( )
  - vendor_identifier( )
  - wapi_user_name( )
  - wapi_user_password( )
- AUTHOR
- SEE ALSO
- COPYRIGHT

NAME

Infoblox::Notification::REST::Endpoint - A REST API endpoint object.

DESCRIPTION

The REST API endpoint object provides configuration for third party REST API servers integration.

CONSTRUCTOR

```
 my $endpoint = Infoblox::Notification::REST::Endpoint->new(
     name                  => $string,                                                  #
Required
     uri                   => $string,                                                  #
Required
     outbound_member_type  => 'GM' | 'MEMBER',                                          #
Required
     client_certificate    => $path,                                                    #
Optional / Default is undefined
     comment               => $string,                                                  #
Optional / Default is undefined
     disable               => 'true' | 'false',                                         #
Optional / Default is 'true'
     extattrs              => { $string => $extattr, ... },                             #
Optional / Default is undefined
     extensible_attributes => {$string => $string | $num, $string => [ $string | $num, ... ], ... } #
Optional / Default is undefined
     log_level             => 'WARNING' | 'DEBUG' | 'INFO' | 'ERROR',                   #
Optional / Default is 'WARNING'
     outbound_members      => [$string, $string, ...],                                  #
Optional / Default is empty list
     password              => $string,                                                  #
Optional / Default is undefined
     template_instance     => $template_instance                                        #
Optional / Default is undefined
     timeout               => $uint,                                                    #
```

```
Optional / Default is 0
     username               => $string,                                                  #
Optional / Default is undefined
     vendor_identifier      => $string,                                                  #
Optional / Default is undefined
     server_cert_validation => 'CA_CERT' | 'CA_CERT_NO_HOSTNAME' | 'NO_VALIDATION',      #
Optional / Default is 'CA_CERT'
     wapi_user_name         => $string,                                                  #
Optional / Default is undefined
     wapi_user_password     => $string,                                                  #
Optional / Default is undefined

 );
```

You cannot set both extattrs and extensible_attributes attributes at the same time.

SESSION METHODS

This section describes all the methods in an Infoblox::Session module that you can apply to a REST API endpoint object.

Infoblox::Session->add( )

Use this method to add the object to the Infoblox appliance. See Infoblox::Session->add() for parameters and return values.

Example

```
#Construct an object
my $object = Infoblox::Notification::REST::Endpoint->new(
    uri                  => 'http://10.0.0.1/office',
    name                 => 'restapi_endpoint',
    outbound_member_type => 'GM',
);

#Submit for addition
my $response = $session->add($object);
```

Infoblox::Session->get( )

Use this method to retrieve all the matching objects from the Infoblox appliance. See Infoblox::Session->get() for parameters and return values.

Key References

```
Apply the following attributes to get a specific REST API endpoint object:

 comment                    - Optional. The REST API endpoint comment in string format.
 name                       - Optional. The REST API endpoint name in string format.
 uri                        - Optional. The REST API endpoint URI in string format.
 vendor_identifier          - Optional. The REST API endpoint vendor identifier in string format.
 extattrs                   - Optional. A hash reference containing Infoblox::Grid::Extattr object.
 extensible_attributes      - Optional. A hash reference containing extensible attributes.
```

Examples

```
my @retrieved_objs = $session->get(
    object => 'Infoblox::Notification::REST::Endpoint',
    uri    => 'http://10.0.0.1/office',
);

my @retrieved_objs = $session->get(
    object               => 'Infoblox::Notification::REST::Endpoint',
    extensible_attributes => {'Site' => 'Santa Clara'},
);
```

Infoblox::Session->modify( )

Use this method to modify the object in the Infoblox appliance. See Infoblox::Session->modify() for parameters and return values.

Example

```
#Modify comment value
$object->comment('this is a modified comment');

#Submit modification
my $response = $session->modify($object);
```

Infoblox::Session->remove( )

Use this method to remove the object from the Infoblox appliance. See Infoblox::Session->remove() for parameters and return values.

To remove a specific object, first use `get()` or `search()` to retrieve the specific REST API endpoint object, and then submit this object for removal.

Example

```
#Get the objects
my @retrieved_objs = $session->get(
    object       => 'Infoblox::Notification::REST::Endpoint',
    uri          => 'http://10.0.0.1/office',
);

#Find the desired object from the retrieved list
my $desired_object = $retrieved_objs[0];

#Submit for removal
my $response = $session->remove($desired_object);
```

Infoblox::Session->search( )

Use this method to search for objects in the Infoblox appliance. See Infoblox::Session->search() for parameters and return values.

Key References

```
Apply the following attributes to search for the REST API endpoint object:

 comment                   - Optional. The REST API endpoint comment in string format (regexp).
 name                      - Optional. The REST API endpoint name in string format (regexp).
 uri                       - Optional. The REST API endpoint URI in string format (regexp).
 vendor_identifier         - Optional. The REST API endpoint vendor identifier in string format
(regexp).
 extattrs                  - Optional. A hash reference containing Infoblox::Grid::Extattr object.
 extensible_attributes     - Optional. A hash reference containing extensible attributes.
```

For more information about searching extensible attributes, see Infoblox::Grid::ExtensibleAttributeDef/Searching Extensible Attributes.

Examples

```
# search for objects
my @retrieved_objs = $session->search(
    object           => 'Infoblox::Notification::REST::Endpoint',
    uri              => 'http.*',
    vendor_identifier => 'Carbon',
);

# search for objects defining 'Santa Clara' for 'Site' extensible attribute
my @retrieved_objs = $session->search(
    object                => 'Infoblox::DHCP::Range',
    extensible_attributes => {'Site' => 'Santa Clara'},
);
```

METHODS

This section describes all the methods that you can use to configure and retrieve the attribute values of a REST API endpoint.

client_certificate( )

Use this method to set the path to a client certificate file. This is a write-only attribute.

Include the specified parameter to set the attribute value.

Parameter

The valid value is a path to a certificate in string format.

Returns

If you have specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

Example

```
#Modify client_certificate value
$object->client_certificate('/path/to/cert.pem');
```

client_certificate_subject( )

Use this method to retrieve the client certificate subject. This is a read-only attrbiute.

Omit the parameter to retrieve the attribute value.

Parameter

None

Returns

The method returns the attribute value.

Example

```
#Get client_certificate_subject value
my $client_certificate_subject = $object->client_certificate_subject();
```

client_certificate_valid_from( )

Use this method to retrieve the time from which the client certificate is valid. This is a read-only attrbiute.

Omit the parameter to retrieve the attribute value.

Parameter

None

Returns

The method returns the attribute value. The number of seconds that have elapsed since January 1st, 1970 UTC.

Example

```
#Get client_certificate_valid_from value
my $client_certificate_valid_from = $object->client_certificate_valid_from();
```

client_certificate_valid_to( )

Use this method to retrieve the bulk client certificate expiration time. This is a read-only attrbiute.

Omit the parameter to retrieve the attribute value.

Parameter

None

Returns

The method returns the attribute value. The number of seconds that have elapsed since January 1st, 1970 UTC.

Example

```
#Get client_certificate_valid_to value
my $client_certificate_valid_to = $object->client_certificate_valid_to();
```

comment( )

Use this method to set or retrieve the REST API endpoint comment.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

The valid value is a desired comment in string format.

Returns

If you have specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get comment value
my $comment = $object->comment();

#Modify comment value
```

```
$object->comment('modified comment');
```

disable( )

Use this method to set or retrieve the flag that indicates whether the REST API endpoint is enabled or disabled.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

Specify 'true' to disable the REST API endpoint or 'false' to enable it. The default value is 'true'.

Returns

If you have specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get disable value
my $disable = $object->disable();

#Modify disable value
$object->disable('false');
```

extattrs( )

Use this method to set or retrieve the extensible attributes associated with a REST API endpoint object.

Parameter

Valid value is a hash reference that contains the names of extensible attributes and their associated values (Infoblox::Grid::Extattr objects).

Returns

If you have specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get extattrs value
my $extattrs = $object->extattrs();

#Modify extattrs value
$object->extattrs({'Site' => $extattr1, 'Administrator' => $extattr2});
```

extensible_attributes( )

Use this method to set or retrieve the extensible attributes associated with a REST API endpoint object.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

For valid values for extensible attributes, see Infoblox::Grid::ExtensibleAttributeDef/Extensible Attribute Values.

Returns

If you have specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get extensible attributes value
my $extensible_attributes = $object->extensible_attributes();
#Modify extensible attributes
$object->extensible_attributes({'Site' => 'Santa Clara', 'Administrator' => ['Peter', 'Tom']});
```

name( )

Use this method to set or retrieve the REST API endpoint name.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

The valid value is a desired name in string format.

Note that this field will default to URI value if no value is specified.

Returns

If you have specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get name value
my $name = $object->name();

#Modify name value
$object->name('modified_name');
```

outbound_members( )

Use this method to set or retrieve the list of REST API endpoint members which are responsible for event generation.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

The valid value is an array that contains Grid member names in a string format.

Returns

If you have specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get outbound_members value
my $outbound_members = $object->outbound_members();

#Modify outbound_members value
$object->outbound_members(['member1', 'member2']);
```

outbound_member_type( )

Use this method to set or retrieve the REST API endpoint member which is responsible for event generation.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

The valid values are 'GM' and 'MEMBER'.

Returns

If you have specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get outbound_member_type value
my $outbound_member_type = $object->outbound_member_type();

#Modify outbound_member_type value
$object->outbound_member_type('MEMBER');
```

log_level( )

Use this method to set or retrieve the REST API endpoint logging level.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

The valid values are 'DEBUG', 'ERROR', 'INFO', and 'WARNING'.

Returns

If you have specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get log_level value
my $value = $object->log_level();

#Modify log_level value
$object->log_level('INFO');
```

password( )

Use this method to set or retrieve the REST API endpoint auth password. This is a write-only attribute.

Include the specified parameter to set the attribute value.

Parameter

The valid value is a desired password in string format.

Returns

The method returns true when the modification succeeds, and returns false when the operation fails.

Example

```
#Modify password value
$object->password('usr1');
```

server_cert_validation( )

Use this method to set or retrieve the server certificate validation type.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

The valid values are 'CA_CERT', 'CA_CERT_NO_HOSTNAME' and 'NO_VALIDATION'. The default value is 'CA_CERT'.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get server_cert_validation value
my $value = $object->server_cert_validation();

#Modify server_cert_validation value
$object->server_cert_validation('NO_VALIDATION');
```

template_instance( )

Use this method to set or retrieve the REST API endpoint REST API template instance.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Note that 'parameters' value of a template instance object cannot be changed for REST API endpoint.

Parameter

The valid value is an Infoblox::Notification::REST::TemplateInstance object.

Returns

If you have specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get template_instance value
my $value = $object->template_instance();

#Modify template_instance value
$object->template_instance($instance);
```

timeout( )

Use this method to set or retrieve the REST API endpoint management session timeout.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

The valid value is an unsigned integer that represents the number of seconds for the management session timeout.

Returns

If you have specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get timeout value
my $value = $object->timeout();

#Modify timeout value
$object->timeout(100);
```

uri( )

Use this method to set or retrieve the REST API endpoint URI.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

The valid value is a desired URI in string format which should be valid URL.

Returns

If you have specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get uri value
my $uri = $object->uri();

#Modify uri value
$object->uri('http://10.0.0.1/new_uri');
```

username( )

Use this method to set or retrieve the REST API endpoint auth username.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

The valid value is a desired username in string format.

Returns

If you have specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get username value
my $username = $object->username();

#Modify username value
$object->username('usr1');
```

vendor_identifier( )

Use this method to set or retrieve the REST API endpoint vendor identifier.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

The valid value is a desired vendor identifier in string format.

Returns

If you have specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get vendor_identifier value
my $vendor_identifier = $object->vendor_identifier();

#Modify vendor_identifier value
$object->vendor_identifier('CarbonBlack');
```

wapi_user_name( )

Use this method to set or retrieve the user name for WAPI integration.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

The valid value is a desired user name in string format.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get wapi_user_name value
my $value = $object->wapi_user_name();

#Modify wapi_user_name value
$object->wapi_user_name('admin');
```

wapi_user_password( )

Use this method to set the user password for WAPI integration. This is a write-only attribute.

Include the specified parameter to set the attribute value.

Parameter

The valid value is a desired user password in string format.

Returns

The method returns true when the modification succeeds, and returns false when the operation fails.

Example

```
#Modify wapi_user_password value
$object->wapi_user_password('infoblox');
```

AUTHOR

Infoblox Inc. http://www.infoblox.com/

SEE ALSO

Infoblox::Session, Infoblox::Session->add(), Infoblox::Session->get(), Infoblox::Session->modify(), Infoblox::Session->remove(), Infoblox::Session->search(), Infoblox::Grid::Extattr, Infoblox::Grid::ExtensibleAttributeDef/Searching Extensible Attributes,

---

COPYRIGHT

Copyright (c) 2017 Infoblox Inc.

Infoblox::Notification::REST::Template - REST API template object.

- NAME
- DESCRIPTION
- CONSTRUCTOR
- SESSION METHODS
  - Infoblox::Session->get( )
  - Infoblox::Session->modify( )
  - Infoblox::Session->remove( )
  - Infoblox::Session->search( )
- METHODS
  - action_name( )
  - added_on( )
  - comment( )
  - content( )
  - event_type( )
  - name( )
  - outbound_type( )
  - template_type( )
  - vendor_identifier( )
- AUTHOR
- SEE ALSO
- COPYRIGHT

NAME

Infoblox::Notification::REST::Template - REST API template object.

DESCRIPTION

The REST API template object used to define particular template configuration.

CONSTRUCTOR

```
my $templ = Infoblox::Notification::REST::Template->new(
    name          => $string,        #Required
    content       => $string,        #Required
    comment       => $string,        #Optional / Default is undefined
    outbound_type => 'REST' | 'DXL', #Optional / Default is 'REST'
);
```

SESSION METHODS

This section describes all the methods in an Infoblox::Session module that you can apply to a REST API template object.

Note that template object does not support 'add' method. Use Infoblox::Session->import_data() of a 'restapi_template' type instead.

Infoblox::Session->get( )

Use this method to retrieve all the matching objects from the Infoblox appliance. See Infoblox::Session->get() for parameters and return values.

Key References

```
Apply the following attributes to get a specific REST API template object:

 comment       - Optional. The REST API template comment in string format.
 name          - Optional. The REST API template name in string format.
 action_name   - Optional. The REST API template action name in string format.
 outbound_type - Optional. The template type.
```

Examples

```
my @retrieved_objs = $session->get(
    object => 'Infoblox::Notification::REST::Template',
    name   => 'template1',
);
```

Infoblox::Session->modify( )

Use this method to modify the object in the Infoblox appliance. See Infoblox::Session->modify() for parameters and return values.

Example

```
#Modify comment value
$object->comment('this is a modified comment');

#Submit modification
my $response = $session->modify($object);
```

Infoblox::Session->remove( )

Use this method to remove the object from the Infoblox appliance. See Infoblox::Session->remove() for parameters and return values.

To remove a specific object, first use `get()` or `search()` to retrieve the specific REST API template object, and then submit this object for removal.

Example

```
#Get the objects
my @retrieved_objs = $session->get(
    object => 'Infoblox::Notification::REST::Template',
    name   => 'template1',
);

#Find the desired object from the retrieved list
my $desired_object = $retrieved_objs[0];

#Submit for removal
my $response = $session->remove($desired_object);
```

Infoblox::Session->search( )

Use this method to search for objects in the Infoblox appliance. See Infoblox::Session->search() for parameters and return values.

Key References

```
Apply the following attributes to search for the REST API template object:

 comment       - Optional. The REST API template comment in string format (regexp).
 name          - Optional. The REST API template name in string format (regexp).
 action_name   - Optional. The REST API template action name in string format (regexp).
 outbound_type - Optional. The template type.
```

Examples

```
# search for objects
my @retrieved_objs = $session->search(
    object => 'Infoblox::Notification::REST::Template',
    name   => 'templ.*',
);
```

METHODS

This section describes all the methods that you can use to configure and retrieve the attribute values of a REST API template instance object.

action_name( )

Use this method to retrieve the template action name. This is a read-only attribute.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

None

Returns

The method returns the attribute value.

Example

```
#Get action_name value
my $action_name = $object->action_name();
```

added_on( )

Use this method to retrieve the timestamp the template was added. This is a read-only attribute.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

None

Returns

The method returns the attribute value, the number of seconds that have elapsed since January 1st, 1970.

Example

```
#Get added_on value
my $added_on = $object->added_on();
```

comment( )

Use this method to set or retrieve the descriptive comment.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

The valid value is a desired comment in string format.

Returns

If you have specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get comment value
my $comment = $object->comment();

#Modify comment value
$object->comment('new comment');
```

content( )

Use this method to set or retrieve the template content.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

The valid value is an well formed JSON string.

Returns

If you have specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get content value
my $content = $object->content();

#Modify content value
$object->content($content);
```

event_type( )

Use this method to retrieve the template event type. This is a read-only attribute.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

None

Returns

The method returns the attribute value.

Example

```
#Get event_type value
my $event_type = $object->event_type();
```

name( )

Use this method to set or retrieve the template name.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

The valid value is a desired name in string format.

Returns

If you have specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get name value
my $name = $object->name();

#Modify name value
$object->name('new_name');
```

outbound_type( )

Use this method to set or retrieve the outbound type for the template.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

The valid value is 'REST' or 'DXL'. The default value is 'REST'.

Returns

If you have specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get outbound_type value
my $outbound_type = $object->outbound_type();

#Modify outbound_type value
$object->outbound_type('DXL');
```

template_type( )

Use this method to retrieve the template type. This is a read-only attribute.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

None

Returns

The method returns the attribute value.

Example

```
#Get template_type value
my $template_type = $object->template_type();
```

vendor_identifier( )

Use this method to retrieve the template vendor identifier. This is a read-only attribute.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

None

Returns

The method returns the attribute value.

Example

```
#Get vendor_identifier value
my $vendor_identifier = $object->vendor_identifier();
```

AUTHOR

Infoblox Inc. http://www.infoblox.com/

SEE ALSO

Infoblox::Session, Infoblox::Session->get(), Infoblox::Session->modify(), Infoblox::Session->remove(), Infoblox::Session->search(), Infoblox::Session->import_data()

COPYRIGHT

Copyright (c) 2017 Infoblox Inc.

Infoblox::Notification::REST::TemplateInstance - A REST API template instance.

- NAME
- DESCRIPTION
- CONSTRUCTOR
- SESSION METHODS
- MODULE METHODS
  - Infoblox::Notification::Rule->template_instance( )
  - Infoblox::Notification::REST::Endpoint->template_instance( )
- METHODS
  - template( )
  - parameters( )
- AUTHOR
- SEE ALSO
- COPYRIGHT

NAME

Infoblox::Notification::REST::TemplateInstance - A REST API template instance.

DESCRIPTION

The REST API template instance is used to configure REST API template setting for notification rule and REST API endpoint.

CONSTRUCTOR

```
my $templ_instance = Infoblox::Notification::REST::TemplateInstance->new(
    template   => $string,                  #Required
    parameters => [$param1, $param2, ...], #Optional / Default is empty list
);
```

SESSION METHODS

The object does not have any session methods.

MODULE METHODS

This section describes all the functions that you can apply to REST API template instance object.

Infoblox::Notification::Rule->template_instance( )

Use this method to set or retrieve the REST API template instance. See Infoblox::Notification::Rule->template_instance() for parameters and return values.

Example

```
#Get template_instance value
my $template_instance = $object->template_instance();

#Modify template_instance value
$object->template_instance($instance);
```

Infoblox::Notification::REST::Endpoint->template_instance( )

Use this method to set or retrieve the REST API template instance. See Infoblox::Notification::REST::Endpoint->template_instance() for parameters and return values.

Example

```
#Get template_instance value
my $template_instance = $object->template_instance();

#Modify template_instance value
$object->template_instance($instance);
```

METHODS

This section describes all the methods that you can use to configure and retrieve the attribute values of a REST API template instance

object.

template( )

Use this method to set or retrieve the REST API template name.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

The valid value is an Infoblox::Notification::REST::Template object name.

Returns

If you have specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get template value
my $template = $object->template();

#Modify template value
$object->template('template1');
```

parameters( )

Use this method to set or retrieve the list of template parameters.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

The valid value is array of Infoblox::Notification::REST::TemplateParameter objects.

Returns

If you have specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get parameters value
my $parameters = $object->parameters();

#Modify parameters value
$object->parameters([$param1, $param2]);
```

AUTHOR

Infoblox Inc. http://www.infoblox.com/

SEE ALSO

Infoblox::Notification::REST::Template, Infoblox::Notification::REST::TemplateParameter

---

COPYRIGHT

Copyright (c) 2017 Infoblox Inc.

Infoblox::Notification::REST::TemplateParameter - A REST API template parameter.

- NAME
- DESCRIPTION
- CONSTRUCTOR
- SESSION METHODS
- MODULE METHODS
  - Infoblox::Notification::REST::TemplateInstance->parameters( )
- METHODS
  - default_value( )
  - name( )
  - syntax( )
  - value( )
- AUTHOR
- SEE ALSO
- COPYRIGHT

NAME

Infoblox::Notification::REST::TemplateParameter - A REST API template parameter.

DESCRIPTION

The REST API template parameter is used to configure REST API template instance variables.

CONSTRUCTOR

```
my $templ_parameter = Infoblox::Notification::REST::TemplateParameter->new(
    name          => $string,                 # Required
    syntax        => 'STR' | 'INT' | 'BOOL', # Required
    value         => $string,                 # Optional / Default is undefined
);
```

SESSION METHODS

The object does not have any session methods.

MODULE METHODS

This section describes all the functions that you can apply to REST API template parameter object.

Infoblox::Notification::REST::TemplateInstance->parameters( )

Use this method to set or retrieve the list of REST API template parameters. See Infoblox::Notification::REST::TemplateInstance->parameters() for parameters and return values.

Example

```
#Get parameters value
my $parameters = $object->parameters();

#Modify parameters value
$object->parameters([[$param1, $param2]);
```

METHODS

This section describes all the methods that you can use to configure and retrieve the attribute values of a REST API template parameter object.

default_value( )

Use this method to retrieve the template parameter default value. This is a read-only attribute.

Omit the parameter to retrieve the attribute value.

Parameter

None

Returns

The method returns the attribute value.

Example

```
#Get default_value value
my $default_value = $object->default_value();
```

name( )

Use this method to set or retrieve the template parameter name.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

The valid value is desired name in string format.

Returns

If you have specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get name value
my $name = $object->name();

#Modify name value
$object->name('param1');
```

syntax( )

Use this method to set or retrieve the template parameter syntax.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

The valid values are 'BOOL', 'STR' and 'INT'.

Returns

If you have specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get syntax value
my $syntax = $object->syntax();

#Modify syntax value
$object->syntax('INT');
```

value( )

Use this method to set or retrieve the template parameter value.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

The desired value in string format.

Returns

If you have specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get value value
my $value = $object->value();

#Modify value value
$object->value('val');
```

AUTHOR

Infoblox Inc. http://www.infoblox.com/

SEE ALSO

Infoblox::Notification::REST::TemplateInstance, Infoblox::Notification::REST::TemplateInstance->parameters()

COPYRIGHT

Copyright (c) 2017 Infoblox Inc.

Infoblox::Notification::Rule - A notification rule object.

- NAME
- DESCRIPTION
- CONSTRUCTOR
- SESSION METHODS
  - Infoblox::Session->add( )
  - Infoblox::Session->get( )
  - Infoblox::Session->modify( )
  - Infoblox::Session->remove( )
  - Infoblox::Session->search( )
- METHODS
  - all_members( )
  - comment( )
  - disable( )
  - enable_event_deduplication( )
  - enable_event_deduplication_log( )
  - event_deduplication_fields( )
  - event_deduplication_lookback_period( )
  - event_type( )
  - expression_list( )
  - name( )
  - notification_action( )
  - notification_target( )
  - override_publish_settings( )
  - publish_settings( )
  - selected_members( )
  - template_instance( )
- AUTHOR
- SEE ALSO
- COPYRIGHT

NAME

Infoblox::Notification::Rule - A notification rule object.

DESCRIPTION

The notification rule object provides configuration for notification of endpoints when the appropriate event occurs.

CONSTRUCTOR

```
 my $notification_rule = Infoblox::Notification::Rule->new(
     event_type                          => 'DNS_RPZ' | 'DHCP_LEASES' | 'SECURITY_ADP' |
                                            'IPAM',                                          # Required
     expression_list                     => [$rule_op1, $rule_op2, ...],                      # Required
     name                                => $string,                                          # Required
     notification_action                 => 'CISCOISE_QUARANTINE' | 'CISCOISE_PUBLISH' |
                                            'RESTAPI_TEMPLATE_INSTANCE',                     # Required
     notification_target                 => $ise_endpoint | $rest_endpoint | $dxl_endpoint    # Required
     all_members                         => 'true' | 'false',                                 # Optional /
if no selected_members given default is 'true'
     comment                             => $string,                                          # Optional /
Default is undefined
     disable                             => 'true' | 'false',                                 # Optional /
Default is 'false'
     override_publish_settings           => 'true' | 'false',                                 # Optional
default is 'false'
     publish_settings                    => $publish_settings,                                # Optional /
Default is undefined
     selected_members                    => [$member1, $member2, ...],                        # Optional /
Default is empty list
     template_instance                   => $instance,                                        # Optional /
Default is undefined
     enable_event_deduplication          => 'true' | 'false',                                 # Optional /
Default is 'false'
     event_deduplication_lookback_period => $uint,                                            # Optional /
Default is 600
     enable_event_deduplication_log      => 'true' | 'false',                                 # Optional /
Default is 'false'
     event_deduplication_fields          => [ 'SOURCE_IP' | 'QUERY_NAME' | 'RPZ_POLICY' |
                                              'RPZ_TYPE' | 'QUERY_TYPE' | 'NETWORK' |
```

```
                                                'NETWORK_VIEW', ... ],                  # Optional /
Default is empty list
                                                                                        # Required if
enable_event_deduplication is set to 'true'
 );
```

SESSION METHODS

This section describes all the methods in an Infoblox::Session module that you can apply to a notification rule object.

Infoblox::Session->add( )

Use this method to add the object to the Infoblox appliance. See Infoblox::Session->add() for parameters and return values.

Example

```
#Construct an object
my $object = Infoblox::Notification::Rule->new(
    event_type          => 'DNS_RPZ',
    expression_list     => [$rule_start_list, $rule_op1, $rule_op2, $rule_end_list],
    name                => 'ise_rule1',
    notification_action => 'CISCOISE_QUARANTINE',
    notification_target => $endpoint,
);

#Submit for addition
my $response = $session->add($object);
```

Infoblox::Session->get( )

Use this method to retrieve all the matching objects from the Infoblox appliance. See Infoblox::Session->get() for parameters and return values.

Key References

```
Apply the following attributes to get a specific notification rule object:

 comment             - Optional. The notification rule comment in string format.
 name                - Optional. The notification rule name in string format.
 event_type          - Optional. The notification rule event type in string format.
 notification_action - Optional. The notification rule notification action in string format.
 notification_target - Optional. The Infoblox::CiscoISE::Endpoint,
Infoblox::Notification::REST::Endpoint or Infoblox::DXL::Endpoint object.
```

Examples

```
my @retrieved_objs = $session->get(
    object => 'Infoblox::Notification::Rule',
    name   => 'ise_rule1',
);
```

Infoblox::Session->modify( )

Use this method to modify the object in the Infoblox appliance. See Infoblox::Session->modify() for parameters and return values.

Example

```
#Modify comment value
$object->comment('this is a modified comment');

#Submit modification
my $response = $session->modify($object);
```

Infoblox::Session->remove( )

Use this method to remove the object from the Infoblox appliance. See Infoblox::Session->remove() for parameters and return values.

To remove a specific object, first use `get()` or `search()` to retrieve the specific DHCP range object, and then submit this object for removal.

Example

```
#Get the objects
my @retrieved_objs = $session->get(
    object => 'Infoblox::Notification::Rule',
    name   => 'ise_rule1',
```

```
);

#Find the desired object from the retrieved list
my $desired_object = $retrieved_objs[0];

#Submit for removal
my $response = $session->remove($desired_object);
```

Infoblox::Session->search( )

Use this method to search for objects in the Infoblox appliance. See Infoblox::Session->search() for parameters and return values.

Key References

```
Apply the following attributes to search for the notification rule object:

 comment             - Optional. The notification rule comment in string format (regexp).
 name                - Optional. The notification rule name in string format (regexp).
 event_type          - Optional. The notification rule event type in string format (exact)
 notification_action - Optional. The notification rule notification action in string format (exact).
 notification_target - Optional. The Infoblox::CiscoISE::Endpoint,
Infoblox::Notification::REST::Endpoint or Infoblox::DXL::Endpoint object.
```

Examples

```
# search for objects
my @retrieved_objs = $session->search(
    object  => 'Infoblox::Notification::Rule',
    name    => '^ise.*',
    comment => '.*comment',
);
```

METHODS

This section describes all the methods that you can use to configure and retrieve the attribute values of a notification rule.

all_members( )

Use this method to set or retrieve the flag that indicates whether the notification rule engine is run on all member or on selected members.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

Specify 'true' to run notification rule engine on all members, or 'false' to run it on selected members. The default value unless selected_members are given is 'true'.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get all_members value
my $all_members = $object->all_members();

#Modify all_members value
$object->all_members('false');
```

comment( )

Use this method to set or retrieve the notification rule comment.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

The valid value is a desired comment in string format.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get comment value
my $comment = $object->comment();

#Modify comment value
$object->comment('ise comment');
```

disable( )

Use this method to set or retrieve the flag that indicates whether the notification rule is enabled or disabled.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

Specify 'true' to disable the notification rule or 'false' to enable it. The default value is 'false'.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get disable value
my $disable = $object->disable();

#Modify disable value
$object->disable('false');
```

enable_event_deduplication( )

Use this method to set or retrieve the flag that indicates whether the notification rule for event deduplication is enabled or disabled.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameters

Specify 'true' to enable event deduplication or 'false' to disable it. The default value is 'false'.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get enable_event_deduplication value
my $enable_event_deduplication = $object->enable_event_deduplication();

#Modify enable_event_deduplication value
$object->enable_event_deduplication('true');
```

enable_event_deduplication_log( )

Use this method to set or retrieve the flag that indicates whether the notification rule event deduplication syslog is enabled or disabled.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameters

Specify 'true' to enable event deduplication syslog or 'false' to disable it. The default value is 'false'.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get enable_event_deduplication_log value
my $enable_event_deduplication_log = $object->enable_event_deduplication_log();

#Modify enable_event_deduplication_log value
$object->enable_event_deduplication_log('true');
```

event_deduplication_fields( )

Use this method to set or retrieve the fields in the notification rule for event deduplication.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameters

The valid value is an array reference that contains following values: 'SOURCE_IP', 'QUERY_NAME', 'RPZ_POLICY', 'RPZ_TYPE', 'QUERY_TYPE', 'NETWORK', 'NETWORK_VIEW'.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get event_deduplication_fields value
my $event_deduplication_fields = $object->event_deduplication_fields();

#Modify event_deduplication_fields value
$object->event_deduplication_fields(['SOURCE_IP', 'QUERY_TYPE']);
```

event_deduplication_lookback_period( )

Use this method to set or retrieve the lookback period for the notification rule for event deduplication.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameters

The valid value is an unsigned integer between 5 and 900 that represents the lookback period (in seconds).

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get event_deduplication_lookback_period value
my $event_deduplication_lookback_period = $object->event_deduplication_lookback_period();

#Modify event_deduplication_lookback_period value
$object->event_deduplication_lookback_period(300);
```

event_type( )

Use this method to set or retrieve the notification rule event type.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

The valid values are 'DNS_RPZ', 'IPAM', 'DHCP_LEASES', 'ANALYTICS_DNS_TUNNEL', 'SECURITY_ADP', 'DB_CHANGE_DHCP_FIXED_ADDRESS_IPV4', 'DB_CHANGE_DHCP_FIXED_ADDRESS_IPV6', 'DB_CHANGE_DHCP_NETWORK_IPV4', 'DB_CHANGE_DHCP_NETWORK_IPV6', 'DB_CHANGE_DHCP_RANGE_IPV4', 'DB_CHANGE_DHCP_RANGE_IPV6', 'DB_CHANGE_DNS_HOST_ADDRESS_IPV4' and 'DB_CHANGE_DNS_HOST_ADDRESS_IPV6'.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get event_type value
my $event_type = $object->event_type();

#Modify event_type value
$object->event_type('SECURITY_ADP');
```

expression_list( )

Use this method to set or retrieve the notification rule condition expression.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

The valid value is an array of Infoblox::Notification::RuleExpressionOp objects.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get expression_list value
my $expression_list = $object->expression_list();

#Set event_type value
$object->event_type('SECURITY_ADP');

#Construct ('SECURITY_AD_RULE_SEVERY' LESS_SEVERE 'MAJOR'
# AND SOURCE_IP EQUALS '10.0.0.10') expression

my $start_list = Infoblox::Grid::ExpressionOp->new(
    op1_type => 'LIST',
    op       => 'AND',
);

my $op1 = Infoblox::Grid::ExpressionOp->new(
    op1_type => 'FIELD',
    op1      => 'SECURITY_AD_RULE_SEVERY',
    op       => 'LE',
    op2      => 'MAJOR',
    op2_type => 'STRING',
);

my $op2 = Infoblox::Grid::ExpressionOp->new(
    op1_type => 'FIELD',
    op1      => 'SOURCE_IP',
    op       => 'EQ',
    op2      => '10.0.0.10',
    op2_type => 'STRING',
);

my $end_list = Infoblox::Grid::ExpressionOp->new(
    op1_type => 'LIST',
    op       => 'ENDLIST',
);

#Modify expression_list value
$object->expression_list([$start_list, $op1, $op2, $end_list]);
```

name( )

Use this method to set or retrieve the notification rule name.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

The valid value is a desired name in string format.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get name value
my $name = $object->name();

#Modify name value
$object->name('rule1');
```

notification_action( )

Use this method to set or retrieve the notification rule action.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

The valid values are 'CISCOISE_QUARANTINE', 'CISCOISE_PUBLISH' and 'RESTAPI_TEMPLATE_INSTANCE'.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get notification_action value
my $notification_action = $object->notification_action();

#Modify notification_action value
$object->notification_action('CISCOISE_QUARANTINE');
```

notification_target( )

Use this method to set or retrieve the notification rule target.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

The valid value is an Infoblox::CiscoISE::Endpoint, Infoblox::Notification::REST::Endpoint or Infoblox::DXL::Endpoint object.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get notification_target value
my $notification_target = $object->notification_target();

#Modify notification_target value
$object->notification_target($endpoint);
```

override_publish_settings( )

The override_publish_settings attribute controls whether the publish_settings method values of the member are used, instead of the target endpoint default.

The override_publish_settings attribute can be specified explicitly. It is also set implicitly when publish_settings is set to a defined value.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

Set the parameter to "true" to override the endpoint level setting for publish_settings. Set the parameter to "false" to inherit the endpoint level setting for publish_settings.

The default value of this parameter is "false".

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get override_publish_settings value
my $override_publish_settings = $object->override_publish_settings( );
#Modify override_publish_settings value
$object->override_publish_settings("true");
```

publish_settings( )

Use this method to set or retrieve the notification rule publish settings.

Setting this method to a defined value implicitly sets the override_publish_settings method to "true". Setting the parameter to undefined causes the appliance to use the endpoint default and automatically resets the override_publish_settings method to "false".

Note that when publish_settings is set to a defined value and override_publish_settings is set to "false", the last operation takes precedence. Thus the sequence $object->publish_settings($value); $object->override_publish_settings("false"); will set override_publish_settings to "false", and the sequence $object->override_publish_settings("false"); $object->publish_settings($value); will result in override_publish_settings="true".

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

The valid value is an Infoblox::CiscoISE::PublishSetting object.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get publish_settings value
my $publish_settings = $object->publish_settings();

#Modify publish_settings value
$object->publish_settings($publish_settings);
```

selected_members( )

Use this method to set or retrieve the member host names on which you run the publish engine.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

The valid value is an array of desired selected member host names in string format.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get selected_members value
my $selected_members = $object->selected_members();

#Modify selected_members value
$object->selected_members(['member1.com', 'member2.com']);
```

template_instance( )

Use this method to set or retrieve the REST API template instance.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

The valid value is an Infoblox::Notification::REST::TemplateInstance object.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get template_instance value
my $template_instance = $object->template_instance();

#Modify template_instance value
$object->template_instance($instance);
```

AUTHOR

Infoblox Inc. http://www.infoblox.com/

SEE ALSO

Infoblox::Session, Infoblox::Session->add(), Infoblox::Session->get(), Infoblox::Session->modify(), Infoblox::Session->remove(), Infoblox::Session->search(), Infoblox::CiscoISE::PublishSetting, Infoblox::CiscoISE::Endpoint, Infoblox::DXL::Endpoint, Infoblox::Notification::RuleExpressionOp, Infoblox::Notification::REST::Endpoint, Infoblox::Notification::REST::TemplateInstance,

---

COPYRIGHT

Copyright (c) 2017 Infoblox Inc.

Infoblox::Notification::RuleExpressionOp - A notification rule expression operand object.

- NAME
- DESCRIPTION
- CONSTRUCTOR
- SESSION METHODS
- MODULE METHODS
  - Infoblox::Notification::Rule->expression_list( )
- METHODS
  - op( )
  - op1( )
  - op1_type( )
  - op2( )
  - op2_type( )
- AUTHOR
- SEE ALSO
- COPYRIGHT

NAME

Infoblox::Notification::RuleExpressionOp - A notification rule expression operand object.

DESCRIPTION

The expression operand object is used to build expression lists. The allowed values for the expression operand object depend on the object they appear to be a part of.

CONSTRUCTOR

```
 my $expression = Infoblox::Notification::RuleExpressionOp->new(
     op       => 'AND' | 'OR' | 'ENDLIST' | 'EQ' | 'REGEX' |
                 | 'MATCH_CIDR' | 'MATCH_RANGE' | 'LE' | 'GE' | 'CONTAINED_IN'            # Required
     op1      => 'ADDRESS_TYPE' | 'DHCP_FINGERPRINT' |
                 'DHCP_LEASE_STATE' | 'DHCP_IP_ADDRESS' | 'DISABLE' |
                 'DNS_RPZ_ACTION_POLICY' | 'DNS_RPZ_NAME' | 'DNS_RPZ_RULE_NAME' |
                 'DNS_RPZ_TYPE' | 'DUID' | 'HOST' | 'IPV4_ADDRESS' |
                 'IPV6_ADDRESS' | 'IPV6_PREFIX' | 'IPV6_PREFIX_BITS' | 'MAC' |
                 'NAME' | 'NETWORK' | 'NETWORK_VIEW' | 'SECURITY_ADP_RULE_MESSAGE' |
                 'SECURITY_ADP_RULE_SEVERITY' | 'SECURITY_ADP_SID' |
                 'SERVER_ASSOC_TYPE_IPV4', 'SERVER_ASSOC_TYPE_IPV6' | 'SOURCE_IP',        # Optional /
Default is undefined
     op2      => $string,                                                                 # Optional /
Default is undefined
     op1_type => 'FIELD' | 'LIST',                                                        # Optional /
Default is undefined
     op2_type => 'STRING',                                                                # Optional /
Default is undefined
 );
```

SESSION METHODS

The object does not support any session methods.

MODULE METHODS

Infoblox::Notification::Rule->expression_list( )

Use this method to set or retrieve the notification rule condition expression. See Infoblox::Notification::Rule->expression_list() for parameters and return values.

Example

```
#Get expression_list value
my $expression_list = $object->expression_list();

#Modify expression_list value
$object->expression_list([$start_list, $op1, $op2, $end_list]);
```

METHODS

op( )

Use this method to set or retrieve the operation name.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

The valid values are 'AND', 'OR', 'ENDLIST', 'EQ', 'NOT_EQ', 'LE', 'REGEX', 'GE', 'CONTAINED_IN', 'MATCH_CIDR' and 'MATCH_RANGE'.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get op value
my $op = $object->op();

#Modify op value
$object->op('OR');
```

op1( )

Use this method to set or retrieve the first operand value.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

The valid values are 'ADDRESS_TYPE', 'DHCP_FINGERPRINT' , 'DHCP_LEASE_STATE', 'DHCP_IP_ADDRESS', 'DISABLE' , 'DNS_RPZ_ACTION_POLICY', 'DNS_RPZ_NAME', 'DNS_RPZ_RULE_NAME' , 'DNS_RPZ_TYPE', 'DUID', 'HOST', 'IPV4_ADDRESS' , 'IPV6_ADDRESS', 'IPV6_PREFIX', 'IPV6_PREFIX_BITS', 'MAC' , 'NAME', 'NETWORK', 'NETWORK_VIEW', 'SECURITY_ADP_RULE_MESSAGE' , 'SECURITY_ADP_RULE_SEVERITY', 'SECURITY_ADP_SID' , 'SERVER_ASSOC_TYPE_IPV4', 'SERVER_ASSOC_TYPE_IPV6', 'SOURCE_IP'.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get op1 value
my $op1 = $object->op1();

#Modify op1 value
$object->op1('SOURCE_IP');
```

op1_type( )

Use this method to set or retrieve the first operand type.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

The valid values are 'FIELD', 'LIST' and 'STRING'.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get op1_type value
my $op1_type = $object->op1_type();

#Modify op1_type value
$object->op1_type('LIST');
```

op2( )

Use this method to set or retrieve the second operand value.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

The valid value is a string that contains the second operand value.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get op2 value
my $op2 = $object->op2();

#Modify op2 value
$object->op2('OP2_VAL');
```

op2_type( )

Use this method to set or retrieve the second operand type.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

The valid value is 'STRING'.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get op2_type value
my $op2_type = $object->op2_type();

#Modify op2_type value
$object->op2_type('STRING');
```

AUTHOR

Infoblox Inc. http://www.infoblox.com/

SEE ALSO

Infoblox::Notification::Rule, Infoblox::Notification::Rule->expression_list(),

---

COPYRIGHT

Copyright (c) 2017 Infoblox Inc.

Infoblox::OCSP::Responder - an OCSP authentication responder.

- NAME
- DESCRIPTION
- CONSTRUCTOR
- MODULE METHODS
  - Infoblox::Grid::Admin::CertificateAuthService->ocsp_responders( )
- METHODS
  - certificate( )
  - comment( )
  - disabled( )
  - address( )
  - port( )
- AUTHOR
- SEE ALSO
- COPYRIGHT

NAME

Infoblox::OCSP::Responder - an OCSP authentication responder.

DESCRIPTION

An Infoblox::OCSP::Responder object represents an OCSP authentication responder.

CONSTRUCTOR

```
my $responder = Infoblox::OCSP::Responder->new(
     address      => $string,              # Required
     certificate  => $string,              # Required
     port         => $uint,                # Optional / Default value is 80
     comment      => $string,              # Optional
     disabled     => "true" | "false",     # Optional / Default value is "false"
);
```

MODULE METHODS

The following functions can be applied to an OCSP authentication responder object.

Infoblox::Grid::Admin::CertificateAuthService->ocsp_responders( )

Use this function to add or retrieve the OCSP authentication responder object in the Certificate Authentication Service object. See Infoblox::Grid::Admin::CertificateAuthService->ocsp_responders() for parameters and return values.

Example

```
#Get ocsp_responders
my $responder_list = $auth_service->ocsp_responders();

#Modify ocsp_responders
my $responder1 = Infoblox::OCSP::Responder->new(
     fqdn_or_ip  => 'domain.com',
     certificate => '/tmp/cert1.pem',
);

my $responder2 = Infoblox::OCSP::Responder->new(
     fqdn_or_ip  => '192.168.1.10',
     certificate => '/tmp/cert2.pem',
);

$auth_service->ocsp_responders([$responder1, $responder2]);
```

METHODS

This section describes all the methods that can be used to configure and retrieve the attribute values of an Infoblox::OCSP::Responder object.

certificate( )

Use this method to set the OCSP responder certificate. This attribute is write-only and cannot be retrieved.

Parameter

String with a reference to a file.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

Example

```
#Set certificate
$responder->certificate('/tmp/cer.pem');
```

comment( )

Use this method to set or retrieve a descriptive comment.

Parameter

Desired comment in string format with a maximum of 256 bytes.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get comment
my $comment = $responder->comment();

#Modify comment
$responder->comment("Modified comment");
```

disabled( )

Use this method to set or retrieve the disabled flag for the OCSP authentication responder.

Parameter

Specify "true" to disable the OCSP authentication respondert or "false" to enable it. Default value is "false".

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value as "true" or "false".

Example

```
#Get disabled flag
my $disabled = $responder->disabled();

#Modify disabled flag
$responder->disabled("true");
```

address( )

Use this method to set or retrieve the IP address or the domain name for the OCSP authentication responder.

Parameter

String with the IP address or the domain name for the OCSP authentication responder.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get address
my $address = $responder->address();

#Modify address
```

```
$responder->address("192.168.1.2");
```

port( )

Use this method to set or retrieve the TCP Port on which the OCSP client communicates with the OCSP responder.

Parameter

An unsigned integer. Default value is 80.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get port
my $port = $responder->port();

#Modify port
$responder->port(90);
```

AUTHOR

Infoblox Inc. http://www.infoblox.com/

SEE ALSO

Infoblox::Grid::Admin::CertificateAuthService

COPYRIGHT

Copyright (c) 2017 Infoblox Inc.

Infoblox::Session - Session object to manipulate data within a grid.

- NAME
- DESCRIPTION
- CONSTRUCTOR
- METHODS
  - add( )
  - cc_mode( )
  - clear_dns_cache( )
  - clear_outbound_worker_log( )
  - clear_unmanaged( )
  - copy_records( )
  - copy_rpz_records( )
  - csv_import_control( )
  - get_all_template_vendor_id( )
  - get_rpz_members( )
  - get_template_schema_versions( )
  - default_maximum_objects_returned( )
  - default_partial_subobjects( )
  - empty_recycle_bin( )
  - export_data( )
  - fill_partial_object( )
  - gen_tsig_key( )
  - gen_hsm_client_cert( )
  - get( )
  - get_ksk_rollover( )
  - get_rpz_threat_details( )
  - get_ms_ad_dc_ns_record_creation_list( )
  - get_grid_revert_status( )
  - grid_upgrade( )
  - import_data( )
  - join_grid( )
  - leave_multi_grid( )
  - logout( )
  - member_upgrade( )
  - modify( )
  - new_cursor( )
  - network_discovery_control( )
  - publish_changes( )
  - query_fqdn_on_member( )
  - refresh_hsm( )
  - remove( )
  - remove_data( )
  - remove_eap_ca_cert( )
  - remove_tftp_file( )
  - reset_discovery_timestamp( )
  - reset_discovery_data( )
  - reset_threat_protection_object( )
  - restart( )
  - restart_status( )
  - rollback_db_snapshot( )
  - rotate_log_files( )
  - run_scavenging( )
  - save_db_snapshot( )
  - search( )
  - search with limit parameters
  - server_version( )
  - set_dhcp_failover_partner_down( )
  - set_dhcp_failover_secondary_recover( )
  - skip_member_upgrade( )
  - status_code( )
  - status_detail( )
  - test( )
  - test_hsm_status( )
  - upgrade_group_now( )
- SAMPLE CODE
- SCHEDULED OBJECT USAGE DETAILS
- AUTHOR
- SEE ALSO
- COPYRIGHT

NAME

Infoblox::Session - Session object to manipulate data within a grid.

DESCRIPTION

The Infoblox::Session object is the key object that is used to manipulate data within a grid. Creating a Session object establishes a communications channel with the grid master.

CONSTRUCTOR

```
 my $session = Infoblox::Session->new(
     master   => $address,  #Required
     password => $string,   #Required unless SSL_cert_file and SSL_key_file provided
     username => $string,   #Required unless SSL_cert_file and SSL_key_file provided
     new_password => $string, #Optional
     timeout  => $num,      #Optional, default is to use the LWP::UserAgent default timeout (180 seconds)
     connection_timeout => $num  #Optional, see below for more details
     default_maximum_objects_returned  => $num, #Optional, default is to return all matching objects for
search operations
     default_partial_subobjects  => 0 / 1, #Optional, default is 0 (no partial subobjects for search
operations)

     # The following options will be passed to LWP::UserAgent if specified. Please see the module
documentation for more details.
     verify_hostname => 0 / 1, #Optional. If set to 1, ensures that the hostname of the server will match
the hostname in its certificate.
     SSL_ca_path => $string, #Optional. The path to a directory that contains the Certificate Authority
certificates. Ignored unless verify_hostname is 1.
     SSL_ca_file => $string, #Optional. The path to a file that contains the Certificate Authority
certificates. Ignored unless verify_hostname is 1.
     SSL_cert_file => $string, #Optional, The path to a file that contains the OCSP Client Certificate.
SSL_key_file and SSL_cert_file should be defined together.
     SSL_key_file  => $string, #Optional, The path to a file that contains the OCSP Client Private Key.
SSL_key_file and SSL_cert_file should be defined together.
      );
```

If connection_timeout is provided and is greater than zero, the API will try to connect to the master's IP address with the specified connection_timeout first; and if it's successful, it will proceed to the log-in process.

The new_password option is required when the force password reset policy is enabled or when the user password is expired.

When the product is in Common Criteria mode, verify_hostname is automatically forced to 1. If verify_hostname is set to 1, SSL_ca_path and SSL_ca_file must be set unless the server certificate is signed by a Certificate Authority recognized by libwww-perl.

Note that SSL errors are not reported by the Perl API when debugging SSL connection issues (1006 'can't connect' errors). Infoblox recommends that you add either

```
 use IO::Socket::SSL qw(debug3);
```

or

```
 use Net::SSLeay;
 $Net::SSLeay::trace = 3;
```

to your Perl program before the "use Infoblox; line" to receive more complete debugging information.

For Common Criteria and SSL operation with certificate options in general, older versions of libwww-perl and supporting libraries might not work correctly. The following have been tested: libwww-perl 6.03, LWP::Protocol::https 6.02, IO::Socket::SSL 1.52, Net::SSLeay 1.42. Infoblox recommends performing a negative test in your environment to make sure connections with incorrect certificates will fail as expected.

For IPv6 operations, the PAPI library should also be imported with the :ipv6connection option. Note that older versions of libwww-perl might not allow connection to explicit IPv6 addresses, but only to Grid members via their FQDNs (Fully Qualified Domain Names).

METHODS

This section describes all the methods that you can use to configure and retrieve the attribute values of a Session object.

add( )

Use this method to add an object to the Infoblox appliance.

Parameter

The Infoblox object to be created in the Infoblox appliance. See individual objects for the necessary parameters.

scheduled_at - Optional. Scheduled date and time in ISO 8601 extended format (e.g., 2008-10-21T10:56:00Z or 2008-10-21T10:56:00.001Z) or 'now' which will set scheduled date and time to the current date and time. See details.

predecessor_task - Optional. An Infoblox::Grid::ScheduledTask object this task depends on. It is mandatory to set scheduled_at parameter if predecessor_task parameter is set.

approval_comment - Optional. Comment provided by the submitter on approval.

approval_ticket_number - Optional. Ticket number provided by the submitter on approval.

ip_validation - Optional. A list of Infoblox::Grid::IPValidationInfo objects.

Returns

The method returns true when the operation succeeds, and returns false when the operation fails. When an operation fails, the error code and error message are reported in the `status_code()` and `status_detail()` attributes.

Example

```
#Construct an object
my $bind_a = Infoblox::DNS::Record::A->new (
    ipv4addr => "1.1.1.1",
    name     => "bind_a.domain.com" );
# Submit for add
my $response = $session->add( $bind_a );
my $ipam_network = Infoblox::Grid::IPAM::Network->new(
   address => '10.20.0.0',
);

my $response = $session->add($ipam_network);
```

cc_mode( )

Use this method to determine if appliance is in Common Criteria mode.

Parameter

None.

Returns

The method returns "true" when the appliance is in Common Criteria mode.

Example

```
#Get cc_mode
my $cc_mode = $session->cc_mode();
```

clear_dns_cache( )

Use this method to clear the DNS cache.

Parameter

member - Required. The IPv4 address of the member on which you want to clear the DNS cache.

domain - Optional. The name of the domain to which the clear operation will be limited. The default is all domains.

view - Optional. The name of the DNS view to which the clear operation will be limited. The default is all views.

clear_full_tree - Optional. The flag that indicates whether the clear operation should be performed on all subdomains of the domain or not. The default is 'false'.

Returns

The method returns true when the operation succeeds, and returns false when the operation fails. When an operation fails, the error code and error message are reported in the `status_code()` and `status_detail()` attributes.

Example

```
# clear the entire DNS cache
my $response = $session->clear_dns_cache(
       member => "192.168.1.2");

unless($response) {
   die("Clear DNS cache failed");
}

# clear a DNS view from the DNS cache
$response = $session->clear_dns_cache(
       member => "192.168.1.2",
```

```
        view => "new_view",
);

unless($response) {
   die("Clear DNS view from DNS cache failed");
}

# clear a domain (including it's all subdomains) from the DNS cache
$response = $session->clear_dns_cache(
        member          => "192.168.1.2",
        domain          => "foo.bar.com",
        clear_full_tree => "true",
);

unless($response) {
   die("Clear the domain \"foo.bar.com\" from DNS cache failed");
}
```

clear_outbound_worker_log( )

Use this method to clear the outbound worker log.

Parameters

`endpoint - Required. The endpoint name.`

Returns

The valid return value is 0 if operation fails to occur or Infoblox::Grid::TestResult otherwise.

Example

```
#Clear outbound worker log
my $result = $session->clear_outbound_worker_log(
    endpoint => 'dxl_endpoint1');
```

clear_unmanaged( )

Use this method to clear unmanaged addresses.

Parameter

type - Required. Valid values are "GLOBAL", "NETWORK", and "IP_ADDRESS".

network - Optional. Required when type is "NETWORK" or "IP_ADDRESS", and the ip_address value is an IPv4 address.

network_view - Optional. The network view to which the clear operation will be limited. The default value is the system-defined default network view.

ip_address - Optional. Required when type is "IP_ADDRESS". The value can be an IPv4 or IPv6 address.

Returns

The method returns true when the operation succeeds, and returns false when the operation fails. When an operation fails, the error code and error message are reported in the `status_code()` and `status_detail()` attributes.

Example

```
# type is GLOBAL
my $response = $session->clear_unmanaged(
        type => 'GLOBAL');

unless($response) {
   die("Clear unmanaged address failed");
}

# with a non-default network view
my $nview = Infoblox::DHCP::View->new(
        name => 'netview',
);
my $response = $session->clear_unmanaged(
        type => 'GLOBAL',
        network_view => $nview);

unless($response) {
   die("Clear unmanaged address failed");
}
```

```
# type is NETWORK
$response = $session->clear_unmanaged(
        type => 'NETWORK',
        network => '10.0.0.0/24');

unless($response) {
   die("Clear unmanaged address failed");
}

# type is IP_ADDRESS
$response = $session->clear_unmanaged(
        type => 'IP_ADDRESS',
        network => '10.0.0.0/24',
        ip_address => '10.0.0.10');

unless($response) {
   die("Clear unmanaged address failed");
}

####PROGRAM ENDS####
```

copy_records( )

Copy records between DNS views.

Parameter

dest_view - Required. The destination DNS view.

dest_zone - Required. The destination DNS zone.

view - Required. The source DNS view.

zone - Required. The source DNS zone.

clear_dest_zone - Optional. Remove all records in the destination zone before copying the records. Default is "false".

copy_all_rr - Optional. Copy all records. Default is "false".

copy_bind_srv - Optional. Copy SRV records. Default is "false".

copy_bind_txt - Optional. Copy TXT records. Default is "false".

copy_bind_ptr - Optional. Copy PTR records. Default is "false".

copy_bulk_host - Optional. Copy Bulk Host records. Default is "false".

copy_bind_a - Optional. Copy A records. Default is "false".

copy_bind_aaaa - Optional. Copy AAAA records. Default is "false".

copy_bind_cname - Optional. Copy CNAME records. Default is "false".

copy_bind_dname - Optional. Copy DNAME records. Default is "false".

copy_bind_mx - Optional. Copy MX records. Default is "false".

copy_bind_naptr - Optional. Copy NAPTR records. Default is "false".

copy_host - Optional. Copy Host records. Default is "false".

replace_existing_records - Optional. Replace any existing records with the copied records if duplicate records are not allowed. Default is "false".

Returns

The method returns true when the modification succeeds, and returns false when the operation fails.

Example

```
#copy records from default view to new_view
$session->copy_records(
    view                    => "default",
    zone                    => "domain.com",
    dest_view               => "new_view",
    dest_zone               => "domain.com",
    copy_bind_srv           => "true",
    copy_bind_txt           => "true",
    copy_bind_ptr           => "false",
```

```
        copy_bulk_host           => "false",
        copy_bind_a              => "true",
        copy_bind_aaaa           => "true",
        copy_bind_cname          => "true",
        copy_bind_dname          => "true",
        copy_bind_mx             => "true",
        copy_host                => "true",
        replace_existing_records => "false",
        clear_dest_zone          => "false",
        copy_all_rr              => "false" );
```

copy_rpz_records( )

Copy RPZ records (rules) between DNS views.

Parameter

dest_view - Required. The destination DNS view.

dest_zone - Required. The destination DNS zone.

view - Required. The source DNS view.

zone - Required. The source DNS zone.

clear_dest_zone - Optional. Remove all records in the destination zone before copying the records. Default is "false".

copy_all_rr - Optional. Copy all records. Default is "false".

copy_passthru_ip_addr - Optional. Copy PASSTHRUIPADDR rules. Default is "false".

copy_passthru_domain - Optional. Copy PASSTHRUDOMAIN rules. Default is "false".

copy_block_nxd_ip_addr - Optional. Copy BLOCKNXDOMAINIPADDR rules. Default is "false".

copy_block_nxd_domain - Optional. Copy BLOCKNXDOMAINDOMAIN rules. Default is "false".

copy_block_nodata_ip_addr - Optional. Copy BLOCKNODATAIPADDR rules. Default is "false".

copy_block_nodata_domain - Optional. Copy BLOCKNODATADOMAIN rules. Default is "false".

copy_subst_cname - Optional. Copy SUBSTITUTE_CNAME rules. Default is "false".

copy_subst_a - Optional. Copy SUBSTITUTE_A_RECORD rules. Default is "false".

copy_subst_aaaa - Optional. Copy SUBSTITUTE_AAAA_RECORD rules. Default is "false".

copy_subst_mx - Optional. Copy SUBSTITUTE_MX_RECORD rules. Default is "false".

copy_subst_ptr - Optional. Copy SUBSTITUTE_PTR_RECORD rules. Default is "false".

copy_subst_naptr - Optional. Copy SUBSTITUTE_NAPTR_RECORD rules. Default is "false".

copy_subst_srv - Optional. Copy SUBSTITUTE_SRV_RECORD rules. Default is "false".

copy_subst_txt - Optional. Copy SUBSTITUTE_TXT_RECORD rules. Default is "false".

copy_subst_ipv4_addr - Optional. Copy SUBSTITUTE_IPV4ADDRESS_RECORD rules. Default is "false".

copy_subst_ipv6_addr - Optional. Copy SUBSTITUTE_IPV6ADDRESS_RECORD rules. Default is "false".

copy_subst_ipaddr_cname - Optional. Copy SUBSTITUTE_IPADDR_CNAME rules. Default is "false".

copy_subst_client_ipaddr_cname - Optional. Copy SUBSTITUTE_CLIENTIPADDR_CNAME rules. Default is "false".

copy_passthru_client_ipaddr - Optional. Copy PASSTHRUCLIENTIPADDR rules. Default is "false".

copy_block_nxdomain_client_ipaddr - Optional. Copy BLOCKNXDOMAINCLIENTIPADDR rules. Default is "false".

copy_block_no_data_client_ipaddr - Optional. Copy BLOCKNODATACLIENTIPADDR rules. Default is "false".

replace_existing_records - Optional. Replace any existing records with the copied records if duplicate records are not allowed. Default is "false".

Returns

The method returns true when the modification succeeds, and returns false when the operation fails.

Example

```
#copy rules (records) from default view to new_view
$session->copy_rpz_records(
    view                      => "default",
    zone                      => "rpz.com",
    dest_view                 => "new_view",
    dest_zone                 => "rpz.com",
    copy_subst_ipv4_addr      => "true",
    copy_subst_cname          => "true",
    copy_passthru_ip_addr     => "false",
    replace_existing_records => "false",
    clear_dest_zone           => "false",
    copy_all_rr               => "false" );
```

csv_import_control( )

Use this method to control the execution of the CSV import task.

Key Reference

```
action    - Required. Valid value is 'STOP' which stops the CSV import task.
import_id - Required. The id of the import task to control.
```

Returns

The method returns 1 if operation succeeds.

Example

```
#Stop CSV Import Task
my $result = $session->csv_import_control(action => 'STOP', import_id => '123456');
```

get_all_template_vendor_id( )

Use this method to retrieve the list of vendor identifiers associated with REST API templates.

Parameter

outbound_type - Optional. The outbound type for the vendor identifier. The valid values are 'REST' and 'DXL'. Default is 'REST'.

Returns

The method returns an array of vendor identifiers in a string format.

Example

```
#get vendor identifiers
my $result = $session->get_all_template_vendor_id(outbound_type => 'DXL');
```

get_rpz_members( )

Use this method to get a list of Infoblox::DNS::Member objects that are conformed to passed arguments.

Parameter

rpz_licensed - Optional. Requests members that have RPZ license properly installed. Default is "false".

rpz_enabled - Optionl. Requests members that have RPZ license properly installed and recursion enabled. Default is "false".

view - Optional. The DNS view to which the zone being added or updated belongs. If omitted, the default DNS view is used.

Returns

The method returns a list of Infoblox::DNS::Member objects when the requested members are found. It returns an empty list if no member conforms to the request.

Example

```
#get RPZ enabled members
$session->get_rpz_members(
       rpz_enabled = > "true");
```

get_template_schema_versions( )

Use this method to retrieve the list of schema versions for the RESTful API templates.

Parameter

```
 schema_type - Required. The REST API template type. The valid values are 'REST_EVENT' and
'REST_ENDPOINT'.
```

Returns

The method returns an array of template schema versions in a string format.

Example

```
#get template schema versions
my $result = $session->get_template_schema_versions();
```

default_maximum_objects_returned( )

Use this method to set or retrieve the default for the maximum objects returned in `search()` and `get()` operations.

Parameter

An unsigned integer.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

Example

```
#Set a new default
$session->default_maximum_objects_returned(100);
```

default_partial_subobjects( )

Use this method to set or retrieve the default for the partial subobjects feature in `search()` operations.

Parameter

Valid parameters are 0 or 1, where 1 means that the partial subobjects functionality is enabled.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

Example

```
#Enable partial_subobjects for searches
$session->default_partial_subobjects(1);
```

empty_recycle_bin( )

Remove the DNS/DHCP data from the recycle bin.

Parameter

none

Returns

The method returns true when the modification succeeds, and returns false when the operation fails.

Example

```
#empty recycle bin
my $response = $session->empty_recycle_bin ();
```

export_data( )

Use this method to download data from the Infoblox appliance to the local system.

Parameter

path - Required. The absolute path of the destination file for the data you want to export.

type - Required. The data type to export. Following is a list of available data types for export:

```
 +-------------------------------------------------------------------------------------------------
----------------+
 | admin_cert       - Export a member certificate. Exported file is in "pem" format.
|
 | member           - Required. The member from which you want to export the certificate.
|
 +-------------------------------------------------------------------------------------------------
```

```
----------------+
 | backup          - Export Infoblox appliance database. Exported file is in "tar.gz" format.
|
 | backup_type     - Optional. Valid values are LOCAL and SCP. Default is LOCAL.
|
 | backup_server   - Optional. The IPv4 address of the backup server.
|
 | user            - Optional. The user name of the backup server.
|
 | password        - Optional. The user password of the backup server.
|
 | nios_data       - Optional. Export NIOS data. Valid value is "true" or "false".
|
 |                   If nios_data, splunk_app_data and discovery_data are not defined, nios_data is
'true'.          |
 | discovery_data  - Optional. Export Network Automation data. Valid value is "true" or "false".
|
 |                   If nios_data, splunk_app_data and discovery_data are not defined, discovery_data
is 'false'.    |
 | splunk_app_data - Optional. Export Splunk application data. Valid value is "true" or "false".
|
 |                   If nios_data, splunk_app_data and discovery_data are not defined,
splunk_app_data is 'false'.   |
 +-------------------------------------------------------------------------------------------------
----------------+
 | captive_portal_cert   - Exports the server certificate of the captive portal.
|
 | dn                    - Required. The distinguished name (dn) of the certificate subject.
|
 +-------------------------------------------------------------------------------------------------
----------------+
 | csv               - Export data in a CSV file
|
|
|
 | object_type       - Required. The object type of the object to be exported
|
 | separator         - Optional. CSV separator character. Valid values are 'comma', 'semicolon',
'space' and 'tab'.  |
 |                     The default is 'comma'.
|
|
|
 | <various>         - Optional. A list of search parameters that identify the object(s) you want to
|
 |                     export. See the individual API object pages for the parameters,
which are           |
 |                     the same as the parameters that are passed to $session->search().
This document     |
 |                     provides information about the supported objects.
|
|
|
 |                     If the objects you want to export have a search parameter named
type, path,         |
 |                     object_type or separator, pass the search parameter with a prepended
|
 |                     underscore, such as '_type', '_separator', etc.
|
 +-------------------------------------------------------------------------------------------------
----------------+
 | csv_snapshot_file - Export the CSV import snapshot file.
|
|
|
 | import_id         - Optional. The ID of the import task for which the snapshot file was exported.
|
 |                     If not specified, the current export task will be used.
|
 +-------------------------------------------------------------------------------------------------
----------------+
 | csv_uploaded_file - Export the original CSV import file.
|
|
|
 | import_id         - Optional. The ID of the import task for which the original CSV file was
exported.            |
 |                     If not specified, the current export task will be used.
|
 +-------------------------------------------------------------------------------------------------
----------------+
 | csv_error_log  - Export the CSV import error log file.
|
 | import_id      - Optional. The ID of the import task for the log file you want to export. If not
specified,       |
 |                  the current export task ID is used.
|
 +-------------------------------------------------------------------------------------------------
----------------+
 | dnsCache       - Export the DNS dnsCache.db file of a member. Exported file is in "tar.gz" format.
The            |
 |                  following parameter is also available:
|
 | member         - Required. The member from which you want to download the dns cache file.
|
 +-------------------------------------------------------------------------------------------------
----------------+
```

```
 | dnsAccelerationCache - Export the DNS acceleration cache of a member. Exported file is in "tar.gz"
format. The    |
 |                        following parameter is also available:
|
 | member               - Required. The member from which you want to download the DNS acceleration
cache file.      |
 +-------------------------------------------------------------------------------------------------
---------------+
 | dnsStats       - Export the DNS statistics file of a member. Exported file is in "tar.gz" format.
The            |
 |                  following parameter is also available:
|
 | member         - Required. The member from which you want to download the DNS statistics file.
|
 +-------------------------------------------------------------------------------------------------
---------------+
 | dhcpd_conf     - Export the DHCP configuration file of a member. Exported file is in "tar.gz"
format. The         |
 |                  following parameter is also available:
|
 | member         - Required. The member from which you want to download the DHCP configuration file.
|
 +-------------------------------------------------------------------------------------------------
---------------+
 | dhcpdv6_conf   - Export the DHCPv6 configuration file of a member. Exported file is in "tar.gz"
format. The       |
 |                  following parameter is also available:
|
 | member         - Required. The member from which you want to download the DHCPv6 configuration
file.              |
 +-------------------------------------------------------------------------------------------------
---------------+
 | dhcp_expert_mode_config - Export the DHCP configuration file of a member in expert mode. Exported
file is in      |
 |                           "dhcpd.conf" format. The following parameter is also available:
|
 | member                  - Required. The member from which you want to download the DHCP
configuration export mode |
 |                           file.
|
 +-------------------------------------------------------------------------------------------------
---------------+
 | dns_conf     - Export the DNS configuration file of a member. Exported file is in "tar.gz" format.
The            |
 |                  following parameter is also available:
|
 | member       - Required. The member from which you want to download the DNS configuration file.
|
 +-------------------------------------------------------------------------------------------------
---------------+
 | ds_records     - Export the keyset associated with a given zone. This is useful for sending to the
parent zone    |
 |                  for signing.
|
 | zone           - Required. The zone that contain the keys to be exported.
|
 |                  Specify either the FQDN of the zone or the Infoblox::DNS::Zone object.
|
 | view           - Optional. The DNS view that contains the zone. Specify either the DNS view name
|
 |                  or Infoblox::DNS::View object. The default is the "default" view.
|
 +-------------------------------------------------------------------------------------------------
---------------+
 | eap_ca_cert    - Export an Extensible Authentication Protocol Certificate Authority (EAP CA)
certificate from the |
 |                  Infoblox appliance. Exported file is in "pem" format. The following parameter is
also            |
 |                  available:
|
 | dn             - Required. The distinguished name (dn) of your certificate's subject.
|
 |                           Examples of dn: CN="Testing",OU="Sales",O="Infoblox",L="Santa
Clara",ST="CA",C="US"    |
 |                                           O="My Company",ST="CA",C="US"
|
 |                                           CN="Test Common Name"
|
 +-------------------------------------------------------------------------------------------------
---------------+
 | generate_captive_portal_csr    - Generate and export a certificate signing request (CSR) for a
captive portal     |
 |                    Exported file is in "pem" format.
|
 | cn               - Required. Common name for the certificate.
|
 | member           - Required. The member for which you want to generate the CSR.
|
 | key_size         - Optional.  Number of bits in the key. It must be 1024, 2048 or 4096. Default is
2048.          |
 | algorithm        - Optional.  Digest algorithm. It must be either SHA-1 or SHA-256. Default is
SHA-256.         |
 | org_unit         - Optional.  Organizational unit used to construct the distinguished name of the
subject.         |
 | org              - Optional.  Organization used to construct the distinguished name of the
subject.           |
 | locality         - Optional.  Location (e.g., city, town) used to construct the distinguished name
```

```
of the subject.|
 | state            - Optional.  State or province used to construct the distinguished name of the
subject.          |
 | country          - Optional.  2-letter country code used to construct the distinguished name of
the subject.      |
 | email            - Optional.  Contact email address used to construct the distinguished name of
the subject.      |
 | comment          - Optional.  A descriptive comment about the certificate to be signed.
|
 +-------------------------------------------------------------------------------------------------
----------------+
 | generate_csr     - Generate and export a certificate signing request (CSR) for a grid member
|
 |                    Exported file is in "pem" format.
|
 | cn               - Required. Common name for the certificate.
|
 | member           - Required. The member for which you want to generate the CSR.
|
 | key_size         - Optional.  Number of bits in the key. It must be 1024, 2048 or 4096. Default is
2048.         |
 | algorithm        - Optional.  Digest algorithm. It must be either SHA-1 or SHA-256. Default is
SHA-256.          |
 | org_unit         - Optional.  Organizational unit used to construct the distinguished name of the
subject.        |
 | org              - Optional.  Organization used to construct the distinguished name of the
subject.          |
 | locality         - Optional.  Location (e.g., city, town) used to construct the distinguished name
of the subject.|
 | state            - Optional.  State or province used to construct the distinguished name of the
subject.          |
 | country          - Optional.  2-letter country code used to construct the distinguished name of
the subject.      |
 | email            - Optional.  Contact e-mail address used to construct the distinguished name of
the subject.     |
 | comment          - Optional.  A descriptive comment about the certificate to be signed.
|
 +-------------------------------------------------------------------------------------------------
----------------+
 | generate_ifmap_dhcp_csr - Generate and export a certificate signing request (CSR) for the IF-MAP
DHCP              |
 |                    Exported file is in "pem" format.
|
 | cn               - Required. Common name for the certificate.
|
 | member           - Required. The member for which you want to generate the CSR.
|
 | key_size         - Optional.  Number of bits in the key. It must be 1024, 2048 or 4096. Default is
2048.         |
 | algorithm        - Optional.  Digest algorithm. It must be either SHA-1 or SHA-256. Default is
SHA-256.          |
 | org_unit         - Optional.  Organizational unit used to construct the distinguished name of the
subject.        |
 | org              - Optional.  Organization used to construct the distinguished name of the
subject.          |
 | locality         - Optional.  Location (e.g., city, town) used to construct the distinguished name
of the subject.|
 | state            - Optional.  State or province used to construct the distinguished name of the
subject.          |
 | country          - Optional.  2-letter country code used to construct the distinguished name of
the subject.      |
 | email            - Optional.  Contact email address used to construct the distinguished name of
the subject.      |
 | comment          - Optional.  A descriptive comment about the certificate to be signed.
|
 +-------------------------------------------------------------------------------------------------
----------------+
 | generate_self_signed_admin_cert - Generate and export a self-signed certificate for a grid member
|
 |                    Exported file is in "pem" format.
|
 | cn               - Required. Common name for the certificate.
|
 | days_valid       - Required. Validity period for the certificate.
|
 | member           - Required. The member for which you want to generate the certificate.
|
 | key_size         - Optional.  Number of bits in the key. It must be 1024, 2048 or 4096. Default is
2048.         |
 | algorithm        - Optional.  Digest algorithm. It must be either SHA-1 or SHA-256. Default is
SHA-256.          |
 | org_unit         - Optional.  Organizational unit used to construct the distinguished name of the
subject.        |
 | org              - Optional.  Organization used to construct the distinguished name of the
subject.          |
 | locality         - Optional.  Location (e.g., city, town) used to construct the distinguished name
of the subject.|
 | state            - Optional.  State or province used to construct the distinguished name of the
subject.          |
 | country          - Optional.  2-letter country code used to construct the distinguished name of
the subject.      |
 | email            - Optional.  Contact email address used to construct the distinguished name of
the subject.      |
 | comment          - Optional.  A descriptive comment about the certificate to be signed.
|
 +-------------------------------------------------------------------------------------------------
----------------+
```

```
 | generate_self_signed_captive_portal_cert - Generate and export a self-signed certificate for a
captive portal     |
 |                    Exported file is in "pem" format.
|
 | cn                - Required. Common name for the certificate.
|
 | days_valid        - Required. Validity period for the certificate.
|
 | member            - Required. The member for which you want to generate the certificate.
|
 | key_size          - Optional.  Number of bits in the key. It must be 1024, 2048 or 4096. Default is
2048.          |
 | algorithm         - Optional.  Digest algorithm. It must be either SHA-1 or SHA-256. Default is
SHA-256.          |
 | org_unit          - Optional.  Organizational unit used to construct the distinguished name of the
subject.         |
 | org               - Optional.  Organization used to construct the distinguished name of the
subject.           |
 | locality          - Optional.  Location (e.g., city, town) used to construct the distinguished name
of the subject.|
 | state             - Optional.  State or province used to construct the distinguished name of the
subject.          |
 | country           - Optional.  2-letter country code used to construct the distinguished name of
the subject.       |
 | email             - Optional.  Contact email address used to construct the distinguished name of
the subject.       |
 | comment           - Optional.  A descriptive comment about the certificate to be signed.
|
 +-------------------------------------------------------------------------------------------------
----------------+
 | generate_self_signed_ifmap_dhcp_cert - Generate and export a self-signed certificate for the IF-
MAP DHCP           |
 |                    Exported file is in "pem" format.
|
 | cn                - Required. Common name for the certificate.
|
 | days_valid        - Required. Validity period for the certificate.
|
 | member            - Required. The member for which you want to generate the certificate.
|
 | key_size          - Optional.  Number of bits in the key. It must be 1024, 2048 or 4096. Default is
2048.          |
 | algorithm         - Optional.  Digest algorithm. It must be either SHA-1 or SHA-256. Default is
SHA-256.          |
 | org_unit          - Optional.  Organizational unit used to construct the distinguished name of the
subject.         |
 | org               - Optional.  Organization used to construct the distinguished name of the
subject.           |
 | locality          - Optional.  Location (e.g., city, town) used to construct the distinguished name
of the subject.|
 | state             - Optional.  State or province used to construct the distinguished name of the
subject.          |
 | country           - Optional.  2-letter country code used to construct the distinguished name of
the subject.       |
 | email             - Optional.  Contact email address used to construct the distinguished name of
the subject.       |
 | comment           - Optional.  A descriptive comment about the certificate to be signed.
|
 +-------------------------------------------------------------------------------------------------
----------------+
 | ifmap_dhcp_cert   - Export an IF-MAP DHCP certificate. Exported file is in "pem" format.
|
 | member            - Required. The member from which you want to export the certificate.
|
 +-------------------------------------------------------------------------------------------------
----------------+
 | log_files         - Export the specified log files from a member. Exported file is in "tar.gz"
format (except
                     'OUTBOUND' in case of which the exported file format is plane text. The
following parameters   |
                     are also available:
|
 | member            - The member from which you are downloading the logs.
|
 |                     You must specify a member, endpoint or a Microsoft(r) server.
|
 | msserver          - The Microsoft server from which you are downloading the logs. You must specify
a member or     |
 |                     a Microsoft server.
|
 | endpoint          - The REST API endpoint name.
|
 | node_type         - Optional. Node type. Valid values are "active" and "passive". The default is
"active".          |
 | log_type          - Optional. Log type. Valid values are "audit", "msmgmt", "outbound" and
"syslog". The default   |
 |                     is "syslog".
|
 | include_rotated   - Optional. Include rotated out log files. The default is false
|
 +-------------------------------------------------------------------------------------------------
----------------+
 | lease_history     - Export the DHCP lease history. Exported file is in "gz" format.
|
 | start_time      - Optional. The start time of the exported lease history in the format yyyy-mm-
ddThh:mm:ss,         |
 |                   yyyy/mm/dd hh:mm:ss, or mm/dd/yyyy hh:mm:ss.
```

```
|
 | end_time       - Optional. The end time of the exported lease history in the format yyyy-mm-
ddThh:mm:ss,          |
 |                  yyyy/mm/dd hh:mm:ss, or mm/dd/yyyy hh:mm:ss.
|
 +-------------------------------------------------------------------------------------------------
----------------+
 | snmp_mibs      - Export the Infoblox SNMP MIB files. Exported file is in "tar.gz" format.
|
 +-------------------------------------------------------------------------------------------------
----------------+
 | support_bundle        - Export the support bundle from a member. Exported file is in "tar.gz"
format. The         |
 |                         following parameters are also available:
|
 | member                - Required. The member from which you are downloading the support bundle.
|
 | filter_optional_files - Optional. Filter optional files. Default is "true".
|
 | core_files            - Optional. Include core files in support bundle. Default is "true".
|
 | log_files             - Optional. Include log files in support bundle. Default is "true".
|
 | rotate_log_files      - Optional. Include rotate_log files in support bundle. Default is "true".
|
 | cached_zone_data      - Optional. Include cached zone data files in support bundle. Default is
"false".            |
 +-------------------------------------------------------------------------------------------------
----------------+
 | serial_numbers - Export the serial numbers of all nodes in the grid. Exported file is in plain
text format.        |
 +-------------------------------------------------------------------------------------------------
----------------+
 | traffic_capture_file  - Export the member's traffic capture file. The exported file is in "tar.gz"
format          |
 |
|
 | member           - Required: The member from which you are downloading the traffic capture file.
|
 +-------------------------------------------------------------------------------------------------
----------------+
 | trust_anchors  - Export trust anchors, i.e. our public keys to allow creation of the island of
security.           |
 |
|
 | zone           - Required. The zone that contains the keys to be exported.
|
 |                  Specify either the FQDN of the zone or the Infoblox::DNS::Zone object.
|
 | view           - Optional. The DNS view that contains the zone. Specify either the DNS view name
or the           |
 |                  Infoblox::DNS::View object. The default is the "default" view.
|
 +-------------------------------------------------------------------------------------------------
----------------+
 | restapi_template - Export the REST API template in JSON format.
|
 |
|
 | template         - Required. An Infoblox::Notification::REST::Template object.
|
 +-------------------------------------------------------------------------------------------------
----------------+
 | restapi_template_schema - Export the REST API template schema in JSON format.
|
 |
|
 | schema_type             - Requred. Schema type to export: 'REST_ENDPOINT' or 'REST_EVENT'.
|
 | version                 - Optional. Schema version to export. The default is the latest version.
|
 +-------------------------------------------------------------------------------------------------
----------------+
 | dxl_endpoint_certs - Export DXL endpoint client and CA certificates. Path argument is the
destination for the     |
 |                     DXL endpoint client certificate.
|
 |
|
 | ca_cert_path - Required. The absolute path of the destination file for the CA certificate you want
to export.      |
 +-------------------------------------------------------------------------------------------------
----------------+
 | safenet_client_cert   - Export a SafeNet HSM client certificate. Exported file is in "pem" format.
|
 | member                - Required. The member from which you want to export the certificate.
|
 | algorithm             - Required. Signature algorithm of a new SafeNet HSM client certificate.
|
 |                         The value can be 'RSASHA1' or 'RSASHA256'.
|
 +-------------------------------------------------------------------------------------------------
----------------+
 | namedacl(type)        - Validate access control items of the Named ACL object. If access control
items of          |
 |                         Named ACL object are ok, the server doesn't return a result file,
otherwise exported file |
```

```
 |                          is in CSV format.
|
 | namedacl              - Required. The Named ACL object.
|
 +-------------------------------------------------------------------------------------------------
----------------+
```

For CSV export, the following objects are supported:

```
 Infoblox::DHCP::FailOver
 Infoblox::DHCP::Filter::Fingerprint
 Infoblox::DHCP::Filter::MAC
 Infoblox::DHCP::Filter::NAC
 Infoblox::DHCP::Filter::Option
 Infoblox::DHCP::Filter::RelayAgent
 Infoblox::DHCP::Fingerprint
 Infoblox::DHCP::FixedAddr
 Infoblox::DHCP::HostAddr
 Infoblox::DHCP::IPv6DhcpRange
 Infoblox::DHCP::IPv6FixedAddress
 Infoblox::DHCP::IPv6HostAddr
 Infoblox::DHCP::IPv6Network
 Infoblox::DHCP::IPv6NetworkContainer
 Infoblox::DHCP::IPv6NetworkContainer
 Infoblox::DHCP::IPv6OptionDefinition
 Infoblox::DHCP::IPv6OptionSpace
 Infoblox::DHCP::IPv6SharedNetwork
 Infoblox::DHCP::Lease
 Infoblox::DHCP::MAC
 Infoblox::DHCP::Network
 Infoblox::DHCP::NetworkContainer
 Infoblox::DHCP::NetworkContainer
 Infoblox::DHCP::OptionDefinition
 Infoblox::DHCP::OptionSpace
 Infoblox::DHCP::Range
 Infoblox::DHCP::SharedNetwork
 Infoblox::DHCP::View
 Infoblox::DNS::AllRPZRecords
 Infoblox::DNS::AllRecords
 Infoblox::DNS::BulkHost
 Infoblox::DNS::Host
 Infoblox::DNS::RPZRecord::A
 Infoblox::DNS::RPZRecord::AAAA
 Infoblox::DNS::RPZRecord::AAAAIpAddress
 Infoblox::DNS::RPZRecord::AIpAddress
 Infoblox::DNS::RPZRecord::CNAME
 Infoblox::DNS::RPZRecord::CNAME::ClientIpAddress
 Infoblox::DNS::RPZRecord::CNAME::ClientIpAddressDN
 Infoblox::DNS::RPZRecord::CNAME::IpAddress
 Infoblox::DNS::RPZRecord::CNAME::IpAddressDN
 Infoblox::DNS::RPZRecord::MX
 Infoblox::DNS::RPZRecord::NAPTR
 Infoblox::DNS::RPZRecord::PTR
 Infoblox::DNS::RPZRecord::SRV
 Infoblox::DNS::RPZRecord::TXT
 Infoblox::DNS::Record::A
 Infoblox::DNS::Record::AAAA
 Infoblox::DNS::Record::Alias
 Infoblox::DNS::Record::CNAME
 Infoblox::DNS::Record::DNAME
 Infoblox::DNS::Record::MX
 Infoblox::DNS::Record::NAPTR
 Infoblox::DNS::Record::NS
 Infoblox::DNS::Record::PTR
 Infoblox::DNS::Record::SRV
 Infoblox::DNS::Record::TLSA
 Infoblox::DNS::Record::TXT
 Infoblox::DNS::Ruleset
 Infoblox::DNS::View
 Infoblox::DNS::Zone
 Infoblox::DTC::Topology
 Infoblox::DTC::Topology::Rule
 Infoblox::Grid::Analytics::WhiteList
 Infoblox::Grid::DHCP
 Infoblox::Grid::DNS
 Infoblox::Grid::DNS::DNS64Group
 Infoblox::Grid::DNS::Nsgroup
 Infoblox::Grid::Member
 Infoblox::Grid::Member::DHCP
 Infoblox::Grid::Member::DNS
 Infoblox::Grid::NamedACL
 Infoblox::Grid::ThreatProtection::Profile
 Infoblox::Grid::ThreatProtection::Profile::Rule
 Infoblox::Grid::UpgradeGroup
```

For more information about the CSV file format please refer to the Infoblox CSV Import Reference.

Returns

The method returns true when the modification succeeds, and returns false when the operation fails. The exported data is stored in the path specified.

Example

```
#Export backup
$session->export_data(
    type   => "backup",
    path   => "/tmp/infoblox/database.tar.gz" );

#Secured backup
my $result = $session->export_data(
    type   => "backup",
    backup_type => "SCP",
    path   => "/tmp/infoblox/database.tar.gz",
    backup_server => "192.168.0.5",
    user => "testuser",
    password => "test-password",
    );

#CSV export
$session->export_data(
                      type        => "csv",
                      path        => "/tmp/infoblox/exported_data.csv",
                      object_type => "Infoblox::DNS::Record::A",
                      separator   => "comma",

                      # search parameters
                      name        => ".*test_exportimport.com");

#CSV error log export
$session->export_data(
                      type      => "csv_error_log",
                      path      => "/tmp/infoblox/error_log.txt",
                      import_id => 12345);

#Export dnsCache.db
my $result = $session->export_data(
    type   => "dnsCache",
    path   => "/tmp/infoblox/dnsCache.tar.gz",
    member => "192.168.1.2" );

#Export dnsStats
my $result = $session->export_data(
    type   => "dnsStats",
    path   => "/tmp/infoblox/dnsStats.tar.gz",
    member => "192.168.1.2" );

#Export dhcpd_conf
my $result = $session->export_data(
    type   => "dhcpd_conf",
    path   => "/tmp/infoblox/dhcpd_conf.tar.gz",
    member => "192.168.1.2" );

#Export dhcpdv6_conf
my $result = $session->export_data(
    type   => "dhcpdv6_conf",
    path   => "/tmp/infoblox/dhcpdv6_conf.tar.gz",
    member => "192.168.1.2" );

#Export dhcp_expert_mode_config
my $result = $session->export_data(
    type   => "dhcp_expert_mode_config",
    path   => "/tmp/infoblox/dhcpd_expert.conf",
    member => "192.168.1.2" );

#Export dns_conf
my $result = $session->export_data(
    type   => "dns_conf",
    path   => "/tmp/infoblox/dns_conf.tar.gz",
    member => "192.168.1.2" );

#Export DS records
my $result = $session->export_data(
    type   => "ds_records",
    zone   => "mydnsseczone.com",
    view   => "dnssec_view",
    path   => "/tmp/infoblox/ds_records.txt" );

#Export captive_portal_cert
my $result = $session->export_data(
    type => "captive_portal_cert",
    path => "/tmp/infoblox/captive_portal_cert.pem",
    dn   => 'CN="My CA Certificate",C="US"' );
```

```
#Export eap_ca_cert
my $result = $session->export_data(
    type => "eap_ca_cert",
    path => "/tmp/infoblox/eap_ca_cert.pem",
    dn   => 'CN="My CA Certificate",C="US"' );

#Generate and export a CSR For captive portal
my $result = $session->export_data(
    type     => "generate_captive_portal_csr",
    path     => "/tmp/infoblox/csr.pem",
    cn       => "My Signing Request",
    key_size =>  2048,
    org_unit => "My Dept.",
    org      => "My Company, Inc.",
    locality => "Santa Clara",
    state    => "California",
    member   => "192.168.1.2",
    country  => "US",
    email    => "me\@mycompany.com",
    comment  => "This is my certificate." );

#Generate and export a CSR For IF-MAP DHCP
my $result = $session->export_data(
    type     => "generate_ifmap_dhcp_csr",
    path     => "/tmp/infoblox/dhcp_csr.pem",
    cn       => "My Signing Request",
    key_size =>  2048,
    org_unit => "My Dept.",
    org      => "My Company, Inc.",
    locality => "Santa Clara",
    state    => "California",
    member   => "192.168.1.2",
    country  => "US",
    email    => "me\@mycompany.com",
    comment  => "This is my certificate." );

#Generate and export a self-signed admin certificate
my $result = $session->export_data(
    type       => "generate_self_signed_admin_cert",
    path       => "/tmp/infoblox/selfsigned.pem",
    cn         => "Self-Signed Admin Cert",
    member     => "192.168.1.2",
    days_valid => 365,
    key_size   => 2048,
    org_unit   => "My Dept.",
    org        => "My Company, Inc.",
    locality   => "Santa Clara",
    state      => "California",
    country    => "US",
    email      => "me\@mycompany.com",
    comment    => "This certificate is self-signed." );

#Generate and export a self-signed captive portal certificate
my $result = $session->export_data(
    type       => "generate_self_signed_captive_portal_cert",
    path       => "/tmp/infoblox/selfsigned.pem",
    cn         => "Self-Signed Captive Portal Cert",
    member     => "192.168.1.2",
    days_valid => 365,
    key_size   => 2048,
    org_unit   => "My Dept.",
    org        => "My Company, Inc.",
    locality   => "Santa Clara",
    state      => "California",
    country    => "US",
    email      => "me\@mycompany.com",
    comment    => "This certificate is self-signed." );

#Generate and export a self-signed IF-MAP DHCP certificate
my $result = $session->export_data(
    type       => "generate_self_signed_ifmap_dhcp_cert",
    path       => "/tmp/infoblox/selfsigned.pem",
    cn         => "Self-Signed IF-MAP DHCP Cert",
    member     => "192.168.1.2",
    days_valid => 365,
    key_size   => 2048,
```

```
        org_unit   => "My Dept.",
        org        => "My Company, Inc.",
        locality   => "Santa Clara",
        state      => "California",
        country    => "US",
        email      => "me\@mycompany.com",
        comment    => "This certificate is self-signed." );

    #Generate and export a CSR
    my $result = $session->export_data(
        type     => "generate_csr",
        path     => "/tmp/infoblox/csr.pem",
        cn       => "My Signing Request",
        key_size =>  2048,
        org_unit => "My Dept.",
        org      => "My Company, Inc.",
        locality => "Santa Clara",
        state    => "California",
        member   => "192.168.1.2",
        country  => "US",
        email    => "me\@mycompany.com",
        comment  => "This is my certificate." );

    #Export an IF-MAP DHCP certificate
    my $result = $session->export_data(
        type   => "ifmap_dhcp_cert",
        member => "192.168.1.2",
        path   => "/tmp/infoblox/ifmap_cert.pem");

    #Export lease_history
    $session->export_data(
        type   => "lease_history",
        path   => "/tmp/infoblox/lease_history.gz",
        start_time  => "2005/09/01 00:00:00",
        end_time    => "2005/09/01 11:59:59" );

    #Export the audit log
    $session->export_data(
        type   => "log_files",
        path   => "/tmp/infoblox/auditlog.tar.gz",
        member => "192.168.1.2",
        node_type => "active",
        log_type => "audit");

    #Export snmp_mibs
    $session->export_data(
        type   => "snmp_mibs",
        path   => "/tmp/infoblox/snmp_mibs.tar.gz" );

    #Export support_bundle
    my $result = $session->export_data(
        type   => "support_bundle",
        path   => "/tmp/infoblox/supportBundle.tar.gz",
        member => "192.168.1.2" );

    #Export support_bundle but exclude core files
    my $result = $session->export_data(
        type   => "support_bundle",
        path   => "/tmp/infoblox/supportBundle.tar.gz",
        core_files => 'false',
        member => "192.168.1.2" );

    #Export support_bundle but exclude log files
    my $result = $session->export_data(
        type   => "support_bundle",
        path   => "/tmp/infoblox/supportBundle.tar.gz",
        log_files => 'false',
        member => "192.168.1.2" );

    #Export serial number of all nodes
    my $result = $session->export_data(
        type   => "serial_numbers",
        path   => "/tmp/infoblox/serial_nos.txt" );

    #Export trust anchors
    my $result= $session->export_data(
        type   => "trust_anchors",
```

```
        zone    => "mysignedzone.com",
        view    => "my_dnssec_view",
        path    => "/tmp/infoblox/trusted_keys.txt" );
```

fill_partial_object( )

Takes an object returned by a search method call with return_methods and fills it.

Parameter

object - Required. An object or a reference to an array of objects returned by a search method call using return_methods call, or from an AllRecords 'record' member. It is legal, but meaningless, to call this method for complete objects.

Returns

The method will return a complete object corresponding to the 'partial' object passed as a parameter. The object returned may be the same object (modified to have all methods initialized) or a new object. In the case of an error, undef will be returned and the appropriate status code will be set.

Example

```
# $partial_bind_object is an object returned by a search using return_methods
my $full_bind_object = $session->fill_partial_object($partial_bind_object);

# @partial_objects is an array of objects returned by a search using return_methods
my @full_objects = $session->fill_partial_object(\@partial_objects);
```

gen_tsig_key( )

Generates a TSIG key.

Parameter

The following parameters are supported:

```
algorithm => "HMAC-MD5" | "HMAC-SHA256" # Optional. If not specified, the algorithm is "HMAC-MD5".
keylen => 128 | 256 | 512 # Optional. If not specified, the key length is 128.
```

Note that the algorithm "HMAC-SHA256" does not support a key length of 512.

Returns

The method returns a randomly generated TSIG key.

Example

```
my $tsig = $session->gen_tsig_key(keylen => 256, algorithm => 'HMAC-SHA256');
```

gen_hsm_client_cert( )

Generate a new SafeNet HSM client certificate.

Parameter

member - Required. The member for which you want to generate a new SafeNet HSM client certificate.

algorithm - Required. Signature algorithm of a new SafeNet HSM client certificate. The value can be 'RSASHA1' or 'RSASHA256'.

Returns

The method returns true when a new SafeNet HSM client certificate is generated successfully, and returns false when the operation fails.

Example

```
my $result = $session->gen_hsm_client_cert('member' => '192.168.1.2', 'algorithm' => 'RSASHA1');
```

get( )

Use this method to retrieve all the matching objects from the Infoblox appliance.

Parameter

object - Required. The Infoblox object to retrieve.

key reference - Required. The attribute and value that refer to the desired `object(s)` to retrieve. See individual objects for available key references.

Returns

Return an array in list context containing all matching objects. For failures and objects not found, it returns an empty array in list context and undefined in scalar context. When an operation fails, the error code and error message are reported in the `status_code()` and `status_detail()` attributes.

Example

```
#Getting an A record from the Infoblox appliance.
my @retrieved_objs = $session->get(
    object => "Infoblox::DNS::Record::A",
    name   => "bind_a.domain.com"  );

unless (@retrieved_objs) {
    die("Get DNS record A failed: ",
       $session->status_code() . ":" . $session->status_detail());
}
```

get_ksk_rollover( )

Use this method to retrieve the KSK rollover information.

Parameter

days - Optional. The number of days since the warning was sent and before the rollover period expires. The default is 7 days.

Returns

The method returns a reference to an array that contains references to hashes representing the zones' KSK information. These hashes contain the following key/value pairs:

```
 name => The zone name
 view => The name of the view to which the zone belongs.
 days => The number of days that are left before the rollover period expires. If the rollover period
has expired, this will be a negative number.
```

Example

```
#Retrieve the KSK information:
my $resultref = $session->get_ksk_rollover();

#Assuming that there is at least one zone requiring rollover:
$zone = @$resultref[0]

#Now $zone contains the above parameters. To print the zone name:
print $zone->{'name'};
```

get_rpz_threat_details( )

Use this method to make a RESTful API call to the ThreatStop RESTful API to receive threat details.

Include the specified parameter to retrieve the attribute value.

Parameter

The valid value is a string containing rpz_rule_name. This is a required parameter.

Returns

If you specified a parameter, the method returns an Infoblox::Grid::RPZ::ThreatDetails structure when the fetch succeeds.

Example

```
#Retrieve rpz_threat_details:
my $rpz_threat_details = $session->get_rpz_threat_details(rpz_rule_name => "b.com.a.com");
```

get_ms_ad_dc_ns_record_creation_list( )

Use this method to get the list of Domain Controllers allowed for creation of NS Record.

Key Reference

```
 zone       - Required if query_type is 'ZONE' or no 'ms_server' is given. The Infoblox::DNS::Zone
object.
 ms_server  - Required if query_type is 'SERVERS_IN_DOMAIN' or no 'zone' is given. The
Infoblox::Grid::MSServer object.
 query_type - Optional. Default value is 'ZONE' if zone argument is given, 'SERVERS_IN_DOMAIN' if
ms_server argument is given.
```

Returns

The valid return value is an array of Infoblox::Grid::MSServer::DCNSRecordCreation objects or empty list.

Example

```
my $dcnsrecordcr = $session->get_ms_ad_dc_ns_record_creation_list(zone => $dns_zone);
```

get_grid_revert_status( )

Retrieves the revert status of an Infoblox Grid.

Parameter

none

Returns

The method returns a reference to a hash containing the following members.

```
revert_allowed => "true" | "false". Indicates whether revert is allowed for the Grid.
revert_version => $string. The version to which the Grid reverts.
```

Example

```
my $revertref = $session->get_grid_revert_status();
```

grid_upgrade( )

This method will perform the specified function on the Grid.

Parameter

The following required parameter is supported:

```
action => one of the following: "UPGRADE_PAUSE", "UPGRADE_RESUME", "DISTRIBUTION_PAUSE",
"DISTRIBUTION_RESUME", "DISTRIBUTION_START", "DISTRIBUTION_STOP", "DOWNGRADE", "REVERT", "UPGRADE",
"UPGRADE_TEST_START", "UPGRADE_TEST_STOP", or "UPLOAD".
```

Returns

The method returns true when the operation succeeds, and returns false when the operation fails.

Example

```
#grid upgrade
my $response = $session->grid_upgrade(action => "UPGRADE_PAUSE");
```

import_data( )

Use this method to upload data to the Infoblox appliance from the local system.

Parameter

path - Required. The complete path of the file to import.

type - Required. The data type to import. Following is a list of available data types for import:

```
 +-------------------------------------------------------------------------------------------------
----------------+
 | threat_analytics_file_upload - Import Apache Spark or Threat Analytics Whitelist update.
|
 +-------------------------------------------------------------------------------------------------
----------------+
 | restapi_template - Import the REST API template from file in JSON format.
|
 | overwrite        - Optional. If 'true' overwrites the existing template based on the template
name, vendor        |
 |                    identifier, event type, and action type. The default is 'false'.
|
 +-------------------------------------------------------------------------------------------------
----------------+
 | admin_cert       - Import a certificate signed by a CA for a certificate signing request
previously given by a    |
 |                    member. The input file should be in "pem" format.
|
 | member           - Required. The member to which you want to upload the certificate.
|
 +-------------------------------------------------------------------------------------------------
----------------+
 | backup          - Restore an existing database backup. Supported file type is "tar.gz". The
following parameters  |
 |                   are also available:
|
 | force           - Optional. Force restore a database that was backed up from
```

```
|
 |                   a different grid. Default is "false".
|
 | keep_grid_ip    - Optional. Used with the force restore option to retain the current grid IP
address and ignore   |
 |                   the IP address from the backup. Default is "true".
|
 | nios_data       - Optional. Restore NIOS data. Valid value is "true" or "false".
|
 |                   If nios_data, discovery_data and splunk_app_data are not defined, nios_data is
'true'.          |
 | discovery_data  - Optional. Restore Network Automation data. Valid value is "true" or "false".
|
 |                   If nios_data, splunk_app_data and discovery_data are not defined, discovery_data
is 'false'.    |
 | splunk_app_data - Optional. Restore Splunk application data. Vali value is "true" or "false".
|
 |                   If nios_data, splunk_app_data and discovery_data are not defined,
splunk_app_data is 'false'.  |
 +-------------------------------------------------------------------------------------------------
----------------+
 | dsb             - Import device support bundle. Supported file type is "tar.gz".
|
 +-------------------------------------------------------------------------------------------------
----------------+
 | captive_portal_cert   - Import a server certificate for the captive portal.
|
 |                         Supported file type is "pem".
|
 | member                - Required. The captive portal to which you want to upload the certificate.
|
 +-------------------------------------------------------------------------------------------------
----------------+
 | captive_portal_file   - Import a file to be used by the captive portal.
|
 |                         Supported file types are .jpg, .gif and .png (when cp_type is one of
'logo', 'footer' or  |
 |                         'header') or .txt (when cp_type is set to 'aup')
|
 | member                - Required. The captive portal to which you want to upload the file.
|
 | cp_type               - Required. Indicates how the file is used in the captive portal. Valid
values are:          |
 |                                   'logo', 'footer', 'header', 'aup'.
|
 +-------------------------------------------------------------------------------------------------
----------------+
 | clone           - For cloned data from the backup of another system. Supported file type is
"tar.gz".              |
 +-------------------------------------------------------------------------------------------------
----------------+
 | csv               - Import DNS and DHCP data via CSV files. See export_data/csv for the supported
object types.   |
 |                     For more information about the CSV file format, refer to the Infoblox CSV
Import Reference.   |
 |                     Use Infoblox::Grid::CSVImportStatus to check import task status.
|
 |
|
 | csv_action        - Optional. Valid values are "start" and "test". The default value is "start".
|
 |                               You cannot set csv_action to "test" if operation is not "replace".
|
 | operation         - Optional. Valid values are "insert", "update", "replace", "custom" and
"delete".              |
 |                               The default value is "insert".
|
 | override          - Optional. Valid values are "override" and "merge". The default value is
"override".           |
 | separator         - Optional. CSV separator character. Valid values are 'comma', 'semicolon',
'space' or 'tab'.   |
 |                               The default value is 'comma'.
|
 | continue_on_error - Optional. Continue import in case of errors. Valid values are "true" or
"false".              |
 |                               The default is "false". If the value is "true", the import continues
even if there  |
 |                               are errors. Note that file format errors are reported immediately.
|
 +-------------------------------------------------------------------------------------------------
----------------+
 | dhcp_expert_mode_config - Import a custom DHCP configuration file from the local system to the
Infoblox           |
 |                           appliance. Supported file type is "dhcpd.conf". The following parameter
is also         |
 |                           available:
|
 | member                  - Required. The member to which you want to upload the DHCP configuration
expert mode     |
 |                           file.
|
 +-------------------------------------------------------------------------------------------------
----------------+
 | downgrade       - NIOS image file that will be used to downgrade the Grid.
|
 +-------------------------------------------------------------------------------------------------
----------------+
```

```
 | ds_records     - Import DS records from a file. The file format is either a DSset (DS records) or
a keyset        |
 |                  (DNSKEY records).
|
 | zone           - Required.  The zone that will contain the records.
|
 |                  Specify either the FQDN of the zone ore the Infoblox::DNS::Zone object.
|
 | view           - Optional. The DNS view that contains the zone. Specify either the DNS view name
or the          |
 |                  Infoblox::DNS::View object. The default is the "default" view.
|
 +-------------------------------------------------------------------------------------------------
----------------+
 | eap_ca_cert    - Import an Extensible Authentication Protocol Certificate Authority (EAP CA)
certificate.       |
 |                  Supported file type is "pem".
|
 +-------------------------------------------------------------------------------------------------
---------------+
 | ifmap_dhcp_cert  - Import a certificate signed by a CA for a certificate signing request
previously given by a    |
 |                    member for the IF-MAP DHCP service. The input file must be in "pem" format.
|
 | member           - Required. The member to which you want to upload the certificate.
|
 +-------------------------------------------------------------------------------------------------
----------------+
 | join_multi_grid    - Joins the Master Grid.
|
 | token              - Required. The token string corresponding to the file to be imported.
|
 | address            - Required. The Multi-Grid Master's address or FQDN.
|
 | port               - Optional. The Multi-Grid Master's port. If it is not specified, 1194 is used.
|
 | grid_name          - Optional. The SubGrid name. The default value is 'Infoblox'.
|
 | use_mgmt_port      - Optional. Use the MGMT port to connect to the grid.
|
 |                      The valid values are 'true' and 'false'. The default value is 'false'.
|
 +-------------------------------------------------------------------------------------------------
----------------+
 | leases           - Import DHCP leases from the local system to the Infoblox appliance. Supported
file type is     |
 |                    "dhcpd.leases". The following parameters are also available:
|
 | format           - Optional. The format of the data file. Only "ISC" is supported. Default is
"ISC".           |
 | lease_precedence - Indicates whether existing leases are overwritten. Valid values are "keep-
previous",        |
 |                    "replace-previous", and "keep-newest". Default is "keep-newest".
|
 | network_view     - Optional. The name of the network view from which the lease data is imported.
|
 |                    The default value is the system-defined default network view.
|
 +-------------------------------------------------------------------------------------------------
----------------+
 | leases6          - Import IPv6 DHCP leases from the local system to the Infoblox appliance.
Supported file type is|
 |                    "dhcpd.leases". The following parameters are also available:
|
 | format           - Optional. The format of the data file. Only "ISC" is supported. Default is
"ISC".           |
 | lease_precedence - Indicates whether existing leases are overwritten. Valid values are "keep-
previous",        |
 |                    "replace-previous", and "keep-newest". Default is "keep-newest".
|
 | network_view     - Optional. The name of the network view from which the lease data is imported.
|
 |                    The default value is the system-defined default network view.
|
 +-------------------------------------------------------------------------------------------------
----------------+
 | lease_history  - Import the DHCP lease history. Supported file type is "gz".
|
 +-------------------------------------------------------------------------------------------------
----------------+
 | tftp_file      - Import file from the local system to a TFTP directory on the Infoblox appliance.
All file types  |
 |                  are supported. The following parameter is also available:
|
 | directory      - Required. The TFTP directory in which the file is stored.
|
 +-------------------------------------------------------------------------------------------------
----------------+
 | discovery_csv  - Import the file from the local system to the Infoblox appliance. Supported file
type is a CSV    |
 |                  file using comma as field sepearator. The following parameter is also available:
|
 | network_view   - Optional. The name of the network view from which the CSV data is imported.
|
 |                  The default value is the system-defined default network view.
|
 | merge_data     - The flag indicates whether the GM sorter replaces or merges the discovered data
```

```
with existing    |
 |                  data. The valid values are "true" and "false". The default value is "true".
|
 +-------------------------------------------------------------------------------------------------------
---------------+
 | upgrade        - NIOS image file that will be used to upgrade the Grid.
|
 +-------------------------------------------------------------------------------------------------------
---------------+
 | uploadkeytab   - Upload keytab file to the server not assigning the keys. The function has no
parameters. Note    |
 |                  that method with such type returns
L<Infoblox::Grid::KerberosKey|Infoblox::Grid::KerberosKey>    |
 |                  objects list if the upload succeeds, and returns false when the operation fails.
|
 +-------------------------------------------------------------------------------------------------------
---------------+
 | trusted_ca_cert  - Import a generic Certificate Authority (CA) certificate.  Supported file type
is "pem".        |
 |                    Allows multiple certificates per single file.
|
 +-------------------------------------------------------------------------------------------------------
---------------+
 | threat_protection_rule_update - Import Threat Protection rules to the Infoblox appliance.
Supported file type is  |
 |                                 "bin2" provided by Infoblox Inc.
|
 +-------------------------------------------------------------------------------------------------------
---------------+
 | dxl_endpoint_brokers - Import DXL endpoint brokers as configuration file and retrieve the list of
|
 |                        Infoblox::DXL::Endpoint::Broker objects.
|
 +-------------------------------------------------------------------------------------------------------
---------------+
```

Returns

The method returns true when the modification succeeds, and returns false when the operation fails.

Example

```
#import backup
$session->import_data(
    type  => "backup",
    force => "false",
    path  => "/tmp/infoblox/database.tar.gz" );

#csv import
$session->import_data(
    type  => "csv",
    continue_on_error => 'true',
    operation => 'update',
    override => 'override',
    path  => "/tmp/infoblox/import_data.csv" );

#import clone
$session->import_data(
    type => "clone",
    path => "/tmp/infoblox/clone_database.tar.gz" );

#import dhcp_expert_mode_config
$session->import_data(
    type   => "dhcp_expert_mode_config",
    member => "192.168.1.2",
    path   => "/tmp/infoblox/dhcpd.conf" );

#import captive_portal_cert
$session->import_data(
    type => "captive_portal_cert",
    path => "/tmp/infoblox/eap_ca_cert.pem",
    member => "192.168.1.2" );

#import captive_portal_file
$session->import_data(
    type => "captive_portal_file",
    path => "/tmp/infoblox/logo_file.jpg",
    member => "192.168.1.2",
    cp_type => "logo" );

#import eap_ca_cert
$session->import_data(
    type => "eap_ca_cert",
    path => "/tmp/infoblox/eap_ca_cert.pem" );
```

```
#import the  IF-MAP DHCP certificate
$session->import_data(
    type => "ifmap_dhcp_cert",
    path => "/tmp/infoblox/ifmap_dhcp_cert.pem" );

#upload keytab
my $kerberos_keys = $session->import_data(
    type => "uploadkeytab",
    path => "/tmp/infoblox/keytab.key");

#import leases
$session->import_data(
    type             => "leases",
    format           => "ISC",
    path             => "/tmp/infoblox/dhcpd.leases",
    lease_precedence => "keep-newest" );

#import leases from a non-default network view
$session->import_data(
    type             => "leases",
    format           => "ISC",
    path             => "/tmp/infoblox/dhcpd.leases",
    lease_precedence => "keep-newest",
    network_view     => "netview" );

#import lease_history
 $session->import_data(
    type => "lease_history",
    path => "/tmp/infoblox/lease_history.gz" );

#import tftp_file
$session->import_data(
    type      => "tftp_file",
    directory => "/",
    path      => "/tmp/infoblox/voip_settings.txt" );

#import discovery CSV file in a given network view
$session->import_data(
    type             => "discovery_csv",
    network_view     => "netview" );

#import DS records
$session->import_data(
    type             => "ds_records",
    path             => "/tmp/infoblox/ds_records.txt",
    zone             => "myzone.test.com",
    view             => "dns_view", );
```

join_grid( )

Join an Infoblox appliance to an existing grid.

Parameter

grid_name - Required. The name of the grid.

master - Required. The virtual IP address of the grid master.

shared_secret - Required. The shared secret string of the grid.

Returns

The method returns true when the modification succeeds, and returns false when the operation fails.

Example

```
my $result = $session->join_grid(
    grid_name     => "infoblox",
    master        => "192.168.1.2",
    shared_secret => "test" );
```

leave_multi_grid( )

Disconnect an Infoblox appliance from the Master Grid.

Returns

The method returns true when the modification succeeds, and returns false when the operation fails.

Example

```
my $result = $session->leave_multi_grid();
```

logout( )

Use this function to log out of the current session.

If you want to log in to the server again, create a new Infoblox::Session object.

Parameter

None

Returns

None

Example

```
$session->logout();
```

member_upgrade( )

Use this function to upgrade a single member that was reverted during the staged upgrade process or to revert a single member if it does not behave properly after an upgrade.

Parameter

The following parameters are supported:

```
member - The FQDN of the member to be upgraded. # Required
action - The action to execute. # Optional
```

The action parameter can be "UPGRADE" or "REVERT". If not specified, the action parameter defaults to "UPGRADE".

Returns

The method returns true when the operation succeeds, and returns false when the operation fails.

Example

```
#upgrade the member
my $response = $session->member_upgrade(member => "hostname.com");
#revert the member
my $response = $session->member_upgrade(member => "hostname.com", action => "REVERT");
```

modify( )

Use this method to modify an object in the Infoblox appliance.

To modify a specific object, use `get()` or `search()` to retrieve the specific object, make the desired modifications to this object, and then submit it for modification.

Parameter

The object to be modified in the Infoblox appliance. See individual objects for the necessary parameters.

Parameter

scheduled_at - Optional. The scheduled date and time in ISO 8601 extended format (e.g. 2008-10-21T10:56:00Z or 2008-10-21T10:56:00.001Z) or 'now' which will set scheduled date and time to the current date and time. See details.

predecessor_task - Optional. An Infoblox::Grid::ScheduledTask object this task depends on. It is mandatory to set scheduled_at parameter if predecessor_task parameter is set.

approval_comment - Optional. Comment provided by the submitter on approval.

approval_ticket_number - Optional. Ticket number provided by the submitter on approval.

ip_validation - Optional. A list of Infoblox::Grid::IPValidationInfo objects.

Returns

The method returns true when the modification succeeds, and returns false when the operation fails. When an operation fails, the error code and error message are reported in the `status_code()` and `status_detail()` attributes.

Example

```
# Use get() to retrieve the object.
my @retrieved_obj = $session->get(
    object => "Infoblox::DNS::Record::A",
    name   => "bind_a.domain.com" );
# select the specific object.
my $bind_a = $retrieved_obj[0];
# Use object method to modify the comment.
$bind_a->comment("this is a modified comment");
# Submit modification
my $response = $session->modify( $bind_a );
```

new_cursor( )

Use this method to return a cursor object. See Infoblox::Cursor for more information.

network_discovery_control( )

Use this method to control the network discovery task

Parameter

action - Required. Valid values are "START", "PAUSE", "RESUME", and "END".

scheduled_at - Optional. Scheduled time in ISO 8601 extended format for date and time (e.g. 2008-10-21T10:56:00Z or 2008-10-21T10:56:00.001Z). Note that this can only be used with the "START" action.

Returns

The method returns true when the operation succeeds, and returns false when the operation fails. When an operation fails, the error code and error message are reported in the `status_code()` and `status_detail()` attributes.

Example

```
# start network discovery
my $response = $session->network_discovery_control(
       action => 'START');

unless($response) {
   die("Start network discovery failed");
}
```

publish_changes( )

Use this method to publish configuration changes.

Parameter

grid_member - Optional. The Grid member name.

services - Required. The service name. The valid value is 'ALL' or 'ATP'.

member_order - Required if Grid_member is not defined. Otherwise this parameter is not used. Option to publish changes on all members with or without delay. The valid value is 'SEQUENTIALLY' or 'SIMULTANEOUSLY'.

sequential_delay - Optional if Grid_member is not defined. Otherwise this parameter is not used. The delay between publishing configuration changes on members in seconds.

Returns

The method returns true when the operation succeeds, and returns false when the operation fails. When an operation fails, the error code and error message are reported in the `status_code()` and `status_detail()` attributes.

Example

```
# publish changes on the Grid
my $res = $session->publish_changes(
   'sequential_delay' => 1,
   'member_order'     => 'SIMULTANEOUSLY',
   'services'         => 'ALL',
);

# publish changes on the Grid member
$res = $session->publish_changes(
   'grid_member' => 'infoblox.localdomain',
   'services'    => 'ALL',
);
```

query_fqdn_on_member( )

Use this method to query FQDN on the specified member.

This methods forces Grid Master to execute RPC call to invoke dig on the member for a specific FQDN.

Parameter

The valid value is an Infoblox::Grid::Member::QueryFQDNParameter object or HASH that contains following values:

```
fqdn            - Required. FQDN to query.
member          - Optional. Member to invoke dig on. The default is Grid Master.
name_server     - Optional. Nameserver to query.
record_type     - Optional. Query resource record type. The default is ANY.
recursive_query - Optional. 'true' to set recursive query flag. The default is 'true'.
```

Returns

The method returns an Infoblox::Grid::Member::QueryFQDNResponse object.

Example

```
#Recursively query any record for the domain, 'foo.com' on the Grid Master.
my $response = $session->query_fqdn_on_member(fqdn => 'foo.com');

#Recursively query nameserver 5.5.5.5 for A record for 'bar.com'.
#domain via 'member1' dig
my $response = $session->query_fqdn_on_member(
    fqdn        => 'bar.com',
    member      => 'member1',
    name_server => '5.5.5.5',
    record_type => 'A',
);

#Same, but using Infoblox::Grid::Member::QueryFQDNParameter

#Construct parameters object
my $query_fqdn = Infoblox::Grid::Member::QueryFQDNParameter->new(
    fqdn        => 'bar.com',
    member      => 'member1',
    name_server => '5.5.5.5',
    record_type => 'A',
);

#Query
my $response = $session->query_fqdn_on_member($query_fqdn);
```

refresh_hsm( )

Use this method to re-sync with the HSM data repository.

Parameter

hsm_group - Required. An HSM group object.

Returns

The method returns one of the following values : "PASSED", "INACTIVE" , "FAILED".

Example

```
# Refresh HSM
my $response = $session->refresh_hsm(
       hsm_group => $hsm_group_object);

unless($response) {
   die("Refresh HSM failed");
}
```

remove( )

Use this method to remove an object from the Infoblox appliance.

To remove a specific object, first use `get()` or `search()` to retrieve the specific object, and then submit this object for removal.

Parameter

The object to be removed from the Infoblox appliance. See individual objects for the necessary parameters.

Parameter

scheduled_at - Optional. Scheduled time in ISO 8601 extended format for date and time (e.g. 2008-10-21T10:56:00Z or 2008-10-21T10:56:00.001Z) or 'now' which will set scheduled date and time to the current date and time. See details.

predecessor_task - Optional. An Infoblox::Grid::ScheduledTask object this task depends on. It is mandatory to set scheduled_at parameter if predecessor_task parameter is set.

approval_comment - Optional. Comment provided by the submitter on approval.

approval_ticket_number - Optional. Ticket number provided by the submitter on approval.

Returns

The method returns true when the modification succeeds, and returns false when the operation fails.

Example

```
# Use get() to retrieve the object.
my @retrieved_obj = $session->get(
    object => "Infoblox::DNS::Record::A",
    name   => "bind_a.domain.com" );
# Select the specific object.
my $bind_a = $retrieved_obj[0];
# Submit for removal
my $response = $session->remove( $bind_a );
```

remove_data( )

Use this method to remove data from the Infoblox appliance or purge DHCP IF-MAP data.

Parameter

type - Required. The type of data to remove. Following is a list of data types that can be removed:

```
 ------------------------------------------------------------------------------------------------------------------------
 | nac_auth_cache           - Purge the NAC authentication cache.
|
 | member                   - Required. The IPv4 address of the member on which the NAC authentication cache is cleared. |
 | mac_addr                 - Optional. The MAC address for which the authentication result in the NAC authentication    |
 |                            cache is to be cleared. if this field is not present, the entire cache will be flushed.    |
 ------------------------------------------------------------------------------------------------------------------------
 | ifmap_dhcp_data          - Sends a purge request that removes any IF-MAP data published by the DHCP IF-MAP client     |
 |                            running on this member. The following parameters are also available:
|
 |
|
 | member                   - Required. The IP address of the grid member.
|
 ------------------------------------------------------------------------------------------------------------------------
```

Returns

The method returns true when the modification succeeds, and returns false when the operation fails.

Example

```
# purge DHCP IF-MAP data published by member 1.2.3.4
$session->remove_data(
    "type"      => "ifmap_dhcp_data",
    "member" => "1.2.3.4");

# purge the entire NAC authentication cache for member 1.2.3.4
$session->remove_data(
    "type"      => "nac_auth_cache",
    "member" => "1.2.3.4");

# Clear authentication data for MAC address a1:a1:a1:a1:a1:a1 from the NAC authentication cache for member 1.2.3.4
$session->remove_data(
    "type"          => "nac_auth_cache",
    "member"    => "1.2.3.4",
    "mac_addr" => "a1:a1:a1:a1:a1:a1");
```

remove_eap_ca_cert( )

Remove an Extensible Authentication Protocol Certificate Authority (EAP CA) certificate.

Parameter

dn - Required. Distinguished Name (DN) of the Certificate Authority (CA) certificate you want to remove.

Returns

The method returns true when the modification succeeds, and returns false when the operation fails.

Example

```
my $result = $session->remove_eap_ca_cert(
    dn => 'CN="My Server Certificate",C="US"' );
```

remove_tftp_file( )

Remove a file from a TFTP directory on the Infoblox appliance.

Parameter

directory - Required. The parent directory of the file in TFTP storage.

name - Required. The name of the file.

Returns

The method returns true when the modification succeeds, and returns false when the operation fails.

Example

```
my $result = $session->remove_tftp_file(
    directory => "/",
    name      => "voip_settings.txt" );
```

reset_discovery_timestamp( )

This method is deprecated and is provided only to maintain backward compatibility with previous releases.

Calling this method will perform the same action as calling the **reset_discovery_data()** method.

reset_discovery_data( )

Use this method to reset the discovery related attributes for an IP address.

The following attributes are reset:

```
os
netbios
first_discovered
last_discovered
network_component_type
network_component_name
network_component_description
network_component_ip
network_component_port_number
network_component_port_name
network_component_port_description
port_vlan_name
port_vlan_number
port_speed
port_duplex
port_status
port_link_status
discovered_name
discoverer
```

Parameter

type - Required. Valid values are "GLOBAL", "NETWORK", and "IP_ADDRESS".

network - Optional. Required when type is "NETWORK" or "IP_ADDRESS", and the ip_address value is an IPv4 address..

network_view - Optional. The network view to which the reset operation will be limited. The default value is the system-defined default network view.

ip_address - Optional. Required when type is "IP_ADDRESS". The value can be an IPv4 or IPv6 address.

Returns

The method returns true when the operation succeeds, and returns false when the operation fails. When an operation fails, the

error code and error message are reported in the `status_code()` and `status_detail()` attributes.

Example

```
# type is GLOBAL
my $response = $session->reset_discovery_data(
       type => 'GLOBAL');

unless($response) {
   die("reset discovery datafailed");
}

# with a non-default network view
my $nview = Infoblox::DHCP::View->new(
       name => 'netview',
);
my $response = $session->reset_discovery_data(
       type => 'GLOBAL',
       network_view => $nview);

unless($response) {
   die("reset discovery data failed");
}

# type is NETWORK
$response = $session->reset_discovery_data(
       type => 'NETWORK',
       network => '10.0.0.0/24');

unless($response) {
   die("reset discovery data failed");
}

# type is IP_ADDRESS
$response = $session->reset_discovery_data(
       type => 'IP_ADDRESS',
       network => '10.0.0.0/24',
       ip_address => '10.0.0.10');

unless($response) {
   die("reset discovery data failed");
}
```

reset_threat_protection_object( )

Use this method to reset the Threat Protection object to the factory defaults.

Key Reference

```
 object              - Required. An Infoblox::Grid::ThreatProtection::Rule,
Infoblox::Grid::ThreatProtection::Ruleset or Infoblox::Grid::ThreatProtection::RuleCategory.
 delete_custom_rules - Optional. Flag that indicates whether the custom rules are removed or not. The
default value is 'true'.
```

Returns

The method returns true when the operation succeeds, and returns false when the operation fails. When an operation fails, the error code and error message are reported in the `status_code()` and `status_detail()` attributes.

Example

```
my $res = $session->reset_threat_protection_object(
    object => $rule,
    delete_custom_rules => 'false'
);
```

restart( )

Restart a service on the Infoblox appliance.

Parameter

mode - Optional. The restart service mode. Valid values are 'GROUPED', 'SEQUENTIAL', 'SIMULTANEOUS'. Default is 'SIMULTANEOUS'.

force_restart - Optional. Valid values are "true" or "false".

members - Optional. The restart service members. Valid value is an array of Grid Member objects. Note that objects should be

retrieved from the server first. Only one parameter for members and groups can be used in the same call.

groups - Optional. The restart service groups. Valid value is an array of Grid Restart Service Group objects. Note that objects should be retrieved from the server first. Only one parameter for members and groups can be used in the same call.

services - Optional. Valid value is an array, which can contain the following values: 'ALL', 'DHCP', 'DHCPV4', 'DHCPV6' and 'DNS'. Default is ['ALL'].

scheduled_at - Optional. Scheduled time in ISO 8601 extended format for date and time (e.g. 2008-10-21T10:56:00Z or 2008-10-21T10:56:00.001Z).

user_name - Optional. Name of the user requesting the restart.

Returns

The method returns true when the modification succeeds, and returns false when the operation fails. If no attribute is specified, the method returns the pending schedule.

Example

```
my $result = $session->restart(
   groups        => [$group1, $group2],
   mode          => 'GROUPED',
   force_restart => 'true',
   services      => ['DNS', 'DHCPV6'],
   user_name     => 'ib-admin',
);
```

restart_status( )

Use this method to retrieve all the services that need to restart.

Parameter

None

Returns

The method returns an array of strings for all the services that need to restart ('dhcp', 'dns').

Example

```
my @restart_list = $session->restart_status();
```

rollback_db_snapshot( )

Use this method to rollback OneDB snapshot if one exists.

Key Reference

None

Returns

The method returns true when the operation succeeds, and returns false when the operation fails. When an operation fails, the error code and error message are reported in the `status_code()` and `status_detail()` attributes.

Example

```
my $res = $session->rollback_db_snapshot();
```

rotate_log_files( )

Use this method to rotate the specified log file for the Grid Master or specified member.

Parameter

log_type - Required. The type of log file. Valid values are 'audit', 'msmgmt', or 'sys'.

member - Optional. Hostname of the Grid member on which log rotation is performed. Only required if 'log_type' is 'sys' or 'msmgmt'.

node_type - Optional. HA node type ('ACTIVE' or 'PASSIVE'). Only required if 'log_type' is 'sys' or 'msmgmt'.

server - Optional. IP address or FQDN of the Microsoft server. Only required if 'log_type' is 'msmgmt'.

Returns

The method returns '1' when there are no errors and sets session error code and text when there is an error.

Example

```
# rotate the syslog
my %args;
$args{'log_type'} = "sys";
$args{'node_type'} = "ACTIVE";
$args{'member'} = "infoblox.localdomain";

$session->rotate_log_files(%args);
```

run_scavenging( )

Use this method to perform the scavenging of the DNS Records.

Key Reference

object - Required. The DNS object on which scavenging is performed. The valid values are Infoblox::Grid::DNS, Infoblox::DNS::Zone and Infoblox::DNS::View. action - Optional. The scavenging action to perform. The valid values are 'ANALYZE', 'RECLAIM', 'ANALYZE_RECLAIM' and 'RESET'. The default value is 'ANALYZE'. scheduled_at - Optional. Scheduled date and time in ISO 8601 extended format (e.g., 2008-10-21T10:56:00Z or 2008-10-21T10:56:00.001Z) or 'now' which will set scheduled date and time to the current date and time. See details.

Returns

The method returns 1 when the operation succeeds and sets session error code an text when the operation fails.

Example

```
my $res = $session->run_scavenging(
    object => $grid_dns,
    action => 'RESET',
);
```

save_db_snapshot( )

Use this method to take OneDB snapshot.

Key Reference

`comment - Required. A descriptive comment in string format.`

Returns

The method returns true when the operation succeeds, and returns false when the operation fails. When an operation fails, the error code and error message are reported in the `status_code()` and `status_detail()` attributes.

Example

```
my $res = $session->save_db_snapshot(comment => 'save snapshot demo');
```

search( )

Use this method to search and retrieve all the matching objects from the Infoblox appliance.

Parameter

object - Required. The Infoblox object to retrieve.

return_methods - Optional. The member fields to return.

maximum_objects_returned - Optional. The maximum number of objects to retrieve (this overrides default_maximum_objects_returned if specified, but will be overridden by an individual object's limit if it is lower).

partial_subobjects - Optional. Indicates whether the partial subobjects feature is enabled (this overrides default_partial_subobjects for this call).

key reference - Required. The attribute and value that refer to the desired `object(s)` to retrieve. Many key references are given as regular expressions. See individual objects for available key references.

Returns

Return an array in list context containing all matching objects. For failures or objects not found, it returns an empty array in list context and undefined in scalar context. When an operation fails, the error code and error message are reported in the `status_code()` and `status_detail()` attributes.

return_methods can be specified in the following way:

```
 +method_name - This adds the specified method to the returned methods (useful for "optional"
methods).
 -method_name - This returns all methods, except the specified method.
  method_name - This returns only the specified method.
```

The return_methods above can be specified together. Note that specifying -method_name in a list that contains method_name entries will be ignored (because only method_name entries will be returned). Specifying +method_name for members that are not optional will similarly be ignored. Following are usage examples:

```
 ['method1','method2']  = Returns only method1 and method2.
 ['-method1']           = Returns all object methods, except method1.
 ['+method1']           = Returns all object methods and method1, if method1 is not a method that is
returned by default.
```

Note that objects returned when method_name and/or -method_name are specified cannot be used directly for modify( ) calls; fill_partial_object must be called for them first.

Member methods that represent actions/operations rather than values (e.g., Infoblox::DHCP::NetworkContainer->resize()) cannot be specified as return_methods.

When the partial subobjects feature is enabled, objects returned from searches do not retrieve all of their subobjects (if any), but only a few of their fields. These subobjects are supported:

```
 Infoblox::DNS::View  - only 'name' and 'comment' will be returned
 Infoblox::DHCP::View - only 'name' and 'comment' will be returned
```

Therefore, if an object that is returned by 'search' has one of the supported subobjects as a method value, then the subobject is treated as if it was retrieved with 'return_methods' set and can be completed by calling fill_partial_object.

Example

```
 #Searching for all A records under "domain.com" from the Infoblox appliance.
 my @retrieved_objs = $session->search(
     object => "Infoblox::DNS::Record::A",
     name   => '.*\.domain\.com' );

 unless (@retrieved_objs) {
     die("Search DNS record A failed: ",
        $session->status_code() . ":" . $session->status_detail());
 }

 #Searching for all A records under "domain.com" from the Infoblox appliance and returning only their
addresses and TTL's.
 my @retrieved_objs = $session->search(
     object => "Infoblox::DNS::Record::A",
     name   => '.*\.domain\.com',
     return_methods => ['ttl','ipv4addr']
 );

 unless (@retrieved_objs) {
     die("Search DNS record A failed: ",
        $session->status_code() . ":" . $session->status_detail());
 }

 #Searching for all A records under "domain.com" from the Infoblox appliance and returning everything
but their TTL's.
 my @retrieved_objs = $session->search(
     object => "Infoblox::DNS::Record::A",
     name   => '.*\.domain\.com',
     return_methods => ['-ttl']
 );

 unless (@retrieved_objs) {
     die("Search DNS record A failed: ",
        $session->status_code() . ":" . $session->status_detail());
 }

 #Searching for all A records under "domain.com" from the Infoblox appliance and returning their
views member as a partial object.
 my @retrieved_objs = $session->search(
     object => "Infoblox::DNS::Record::A",
     name   => '.*\.domain\.com',
     partial_subobjects => 1
 );

 unless (@retrieved_objs) {
     die("Search DNS record A failed: ",
        $session->status_code() . ":" . $session->status_detail());
```

```
}
```

search with limit parameters

Some object methods support searches where limits are specified. Assuming that 'X' and 'Y' are valid searchable values for a particular method, the following are supported:

'X' - Matches, as usual, if the method value is EXACTLY X.
'> X' or '< X' - Matches if the method value is GREATER or LESS than X respectively.
'>= X' or '<= X' - Matches if the method value is GREATER THAN OR EQUALS or is LESS THAN OR EQUALS X respectively.
'!= X' or '/= X' - Matches if the method value DOES NOT EQUAL X.
'>=< X,Y' - Matches if the method value is GREATER THAN OR EQUALS X AND is LESS THAN OR EQUALS Y.

Following are examples of limit searches:

```
# This searches for tasks scheduled exactly on 2011-09-18T10:20:20Z
@result = $session->search(
                           object         => "Infoblox::Grid::ScheduledTask",
                           scheduled_time => '2011-09-18T10:20:20Z',
                          );

# This searches for tasks scheduled before 2011-09-18T10:20:20Z
@result = $session->search(
                           object         => "Infoblox::Grid::ScheduledTask",
                           scheduled_time => '< 2011-09-18T10:20:20Z',
                          );

# This searches for tasks scheduled before or on 2011-09-18T10:20:20Z
@result = $session->search(
                           object         => "Infoblox::Grid::ScheduledTask",
                           scheduled_time => '<= 2011-09-18T10:20:20Z',
                          );

# This searches for tasks scheduled not on 2011-09-18T11:20:20Z
@result = $session->search(
                           object         => "Infoblox::Grid::ScheduledTask",
                           scheduled_time => '!= 2011-09-18T11:20:20Z',
                          );

# This searches for tasks scheduled between 2011-09-18T10:20:20Z and 2011-10-18T10:20:20Z, inclusive
@result = $session->search(
                           object         => "Infoblox::Grid::ScheduledTask",
                           scheduled_time => '>=< 2011-09-18T10:20:20Z,2011-10-18T10:20:20Z',
                          );
```

server_version( )

Check the current server version.

Parameter

none

Returns

The method returns the current Infoblox appliance version.

Example

```
my $version = $session->server_version( );
```

set_dhcp_failover_partner_down( )

Set a DHCP failover association peer to the Partner Down state.

Parameter

dhcp_failover - Required. The name of the DHCP failover association with the peer that you are setting to the Partner Down state.

Parameter

peer_type - Required. The peer that you are setting to the Partner Down state. Valid values are 'primary' or 'secondary'.

Returns

The method returns true when the operation succeeds, and returns false when the operation fails.

Example

```
my $result = $session->set_dhcp_failover_partner_down(
   dhcp_failover => 'test_failover',
   peer_type     => 'primary'
);
```

set_dhcp_failover_secondary_recover( )

Set the secondary peer to the Recover state.

Parameter

dhcp_failover - Required. The name of the DHCP failover association with the secondary peer that you are setting to the Recover state.

Returns

The method returns true when the operation succeeds, and returns false when the operation fails.

Example

```
my $result = $session->set_dhcp_failover_secondary_recover(
   dhcp_failover => 'test_failover'
);
```

skip_member_upgrade( )

This function allows the specified member to skip the upgrade process.

Parameter

The following required parameter is supported:

```
member - The FQDN of the member that will skip the upgrade process.
```

Returns

The method returns true when the operation succeeds, and returns false when the operation fails.

Example

```
#skip member
my $response = $session->skip_member_upgrade(member => "hostname.com");
```

status_code( )

Get the status code of the last function called.

Parameter

none

Returns

The method returns the status code of the last function/operation called.

Example

```
my $status_code = $session->status_code( );
```

status_detail( )

Get the status detail of the last function called.

Parameter

none

Returns

The method returns the status detail of the last function/operation called.

Example

```
my $status_detail = $session->status_detail( );
```

test( )

A generic test function.

Parameter

object - Required. An Infoblox object. timeout - Optional, if object type is 'Infoblox::LDAP::Server'. The timeout in seconds. ldap_service - Optional, if object type is 'Infoblox::LDAP::Server'. An Infoblox::LDAP::AuthService object that has been retrieved from the appliance. certificate_auth_service - Optional, if object type is 'Infoblox::Grid::Admin::CertificateAuthService'. An Infoblox::Grid::Admin::CertificateAuthService or it's name in a string format. member - Optional, if object type is 'Infoblox::Grid::SyslogBackupServer'. The name of the Grid member which performs the connection test. By default, the connection test is performed from the Grid Master. event_text - Required, if object type is 'Infoblox::Notification::Rule'. An event to trigger notification rule in a JSON text string.

Object type	Description
Infoblox::OCSP::Responder	- Performs validation of OCSP responder object. The valid return value is one of the following: "CANNOT_RESOLVE_FQDN", "CANNOT_CONNECT", "FAILED_TEST", "TEST_OK", or "TEST_OK_RESPONSE_NOT_VERIFIED".
Infoblox::LDAP::Server	- Tests connectivity to the LDAP server.
Infoblox::CiscoISE::Endpoint	- Tests connectivity to the CiscoISE endpoint. The valid return values are 'CANNOT_RESOLVE_FQDN', 'FAIL', 'OK' and 'WARN'.
Infoblox::Notification::REST::Endpoint	- Tests connectivity to the REST API endpoint. The valid return value is 0 if connectivity tests fail to occur or Infoblox::Grid::TestResult otherwise.
Infoblox::Notification::Rule	- Tests applicability of event to be triggered.
Infoblox::DXL::Endpoint::Broker	- Tests connectivity to the DXL broker. Parameters: endpoint - Optional. The Infoblox::DXL::Endpoint object. client_certificate - Optional. The DXL broker client certificate. Note that at least one of the arguments ('endpoint' and 'client_certificate') should be set in order to perform the connectivity test. The valid return value is 0 if the connectivity tests fail to occur or Infoblox::Grid::TestResult otherwise.
Infoblox::Grid::SyslogServer	- Tests connectivity to the Syslog server.
Infoblox::Grid::SyslogBackupServer	- Tests connectivity to the Syslog backup server.

Returns

The method returns a test result.

Example

```
#Performs validation of OCSP responder object
my $ocsp_responder = Infoblox::OCSP::Responder->new(
                     'address'     => 'domain.com',
                     'certificate' => 'responder.pem',
);

my $result = $session->test('object' => $ocsp_responder);

my $syslog_server = Infoblox::Grid::SyslogServer->new(
   address         => '10.32.1.55',
   connection_type => 'tcp',
   port            => 514,
);

my $result = $session->test('object' => $syslog_server);

my $backup_syslog_server = Infoblox::Grid::BackupSyslogServer->new(
   address => '10.32.1.55',
);

my $result = $session->test('object' => $backup_syslog_server);
```

```
my $notif_rule = Infoblox::Notification::Rule->new(
                            event_type          => 'DNS_RPZ',
                            expression_list     => [@rule_expr1],
                            name                => 'notif_rule1',
                            notification_action => 'RESTAPI_TEMPLATE_INSTANCE',
                            notification_target => $endpoint,
                            template_instance   => $instances[0],
                            disable             => 'false',
   );
my $event_text = '{"destination_ip": "10.0.0.0","timestamp": "1973-03-03T09:46:41Z",'.
                 '"vnode_oid": 0, "thread_id": 0, "sequence_id": 0, "member_name":
"infoblox.localdomain",'.
                 '"rpz_policy": "PASSTHRU", "member_ip": "192.168.1.2", "source_ip": "10.0.0.55",'
.
                 '"network.network_view": "test_view", "rpz_severity": "WARNING"}';

my $result = $session->test('object' => $notif_rule, event_text => $event_text);
```

test_hsm_status( )

Performs a signing test using an active HSM group.

Parameter

hsm_group - Required. An HSM group object.

Returns

The method returns the status detail of signing test. The valid return value is one of the following : "PASSED", "INACTIVE","KEY_GEN", "SIGNING" .

Example

```
my $response = $session->test_hsm_status(
                          hsm_group => $hsm_group_object);

unless($response) {
   die("Test HSM status failed");
```

upgrade_group_now( )

This function allows the specified group to immediately upgrade.

Parameter

The following required parameter is supported:

group_name - The name of the group to upgrade.

Returns

The method returns true when the operation succeeds, and returns false when the operation fails.

Example

```
#Upgrade group now
my $response = $session->upgrade_group_now(group_name => "group1");
```

SAMPLE CODE

#Create an Infoblox::Session object

```
#PROGRAM STARTS: Include all the modules that will be used
use strict;
use Infoblox;

#Create a session to the Infoblox appliance
my $session = Infoblox::Session->new(
    master   => "192.168.1.2",
    username => "admin",
    password => "infoblox"
);
if ($session->status_code()) {
   die("Construct session failed: ",
       $session->status_code() . ":" . $session->status_detail());
```

```
}
print "Session created successfully\n";
```

#Add a DNS zone object

```
#Construct the zone
my $zone = Infoblox::DNS::Zone->new(name => "domain.com");
unless ($zone) {
   die("Construct zone failed: ",
       $session->status_code() . ":" . $session->status_detail());
}
print "Zone object created successfully\n";

#Add the zone to the Infoblox appliance
$session->add($zone)
    or die("Add zone failed: ",
        $session->status_code() . ":" . $session->status_detail());
print "Zone added successfully\n";
```

#Restart service

```
$session->restart(
     "member" => "192.168.1.2",
     "when"   => "03/20/2007 01:00:00",
     "time_zone" => "(UMT - 6:00) Central Time (US and Canada)"
);

#To cancel the previous restart schedule.

 $session->restart("cancel" => "true");
```

#Get restart services

```
my @services = $session->restart_status();
```

#Get zone object

```
my @retrieved_objs = $session->get(
    object    => "Infoblox::DNS::Zone",
    name      => "domain.com" );
my $object = $retrieved_objs[0];
unless ($object) {
    die("Get Zone object failed: ",
        $session->status_code() . ":" . $session->status_detail());
}
print "Get DNS zone object found at least 1 matching entry\n";
```

#Remove zone object

```
$session->remove( $object )
  or die("Remove DNS zone failed: ",
         $session->status_code() . ":" . $session->status_detail());
print "DNS Zone object removed successfully \n";
```

#Show the current version

```
my $version = $session->server_version();
print "Current server version is $version\n";
```

#Generate a TSIG key

```
my $tsig = $session->gen_tsig_key();
print "Generated a TSIG key: $tsig\n";
```

#Show status_code and status_detail

```
my $status_code = $session->status_code();
my $status_detail = $session->status_detail();
print "Previous operation status_code is: $status_code\n";
print "Previous operation status_detail is: $status_detail\n";
```

#clear unmanaged address

```
# Creating new network and checking
my $network = Infoblox::DHCP::Network->new(network => "10.0.0.0/24");
unless($network) {
   die("Create network object failed.");
```

```
}

# Adding into Session and testing
$session->add($network)
   or die("Add network failed.");

# Creating a new network view
my $network_view = Infoblox::DHCP::View->new(name => "netview");
unless($network_view) {
   die("Create network view object failed.");
}
$session->add($network_view)
   or die("Add network view failed.");

# Creating a second network in the network view
my $network2 = Infoblox::DHCP::Network->new(
                  network => "10.0.0.0/24",
                  network_view => $network_view);
unless($network2) {
   die("Create second network object failed.");
}
$session->add($network2)
   or die("Add second network object failed.");

# type is GLOBAL
my $response = $session->clear_unmanaged(
   type => 'GLOBAL');

unless($response) {
   die("Clear unmanaged failed");
}

my $response = $session->clear_unmanaged(
   type => 'GLOBAL',
   network_view => $network_view);

unless($response) {
   die("Clear unmanaged failed");
}
# type is NETWORK
$response = $session->clear_unmanaged(
   type => 'NETWORK',
   network => '10.0.0.0/24');

unless($response) {
   die("Clear unmanaged failed");
}

$response = $session->clear_unmanaged(
   type => 'NETWORK',
   network => '10.0.0.0/24',
   network_view => $network_view);

unless($response) {
   die("Clear unmanaged failed");
}

# type is IP_ADDRESS
$response = $session->clear_unmanaged(
   type => 'IP_ADDRESS',
   network => '10.0.0.0/24',
   ip_address => '10.0.0.10');

unless($response) {
   die("Clear unmanaged failed");
}

$response = $session->clear_unmanaged(
   type => 'IP_ADDRESS',
   network => '10.0.0.0/24',
   network_view => $network_view,
   ip_address => '10.0.0.10');

unless($response) {
   die("Clear unmanaged failed");
}
```

#reset discovery data

```
# type is GLOBAL
my $response = $session->reset_discovery_data(
   type => 'GLOBAL');

unless($response) {
   die("reset discovery data failed");
}

my $response = $session->reset_discovery_data(
   type => 'GLOBAL',
   network_view => $network_view);

unless($response) {
   die("reset discovery data failed");
}

# type is NETWORK
$response = $session->reset_discovery_data(
   type => 'NETWORK',
   network => '10.0.0.0/24');

unless($response) {
   die("reset discovery data failed");
}

$response = $session->reset_discovery_data(
   type => 'NETWORK',
   network => '10.0.0.0/24',
   network_view => $network_view);

unless($response) {
   die("reset discovery data failed");
}

# type is IP_ADDRESS
$response = $session->reset_discovery_data(
   type => 'IP_ADDRESS',
   network => '10.0.0.0/24',
   ip_address => '10.0.0.10');

unless($response) {
   die("reset discovery data failed");
}

$response = $session->reset_discovery_data(
   type => 'IP_ADDRESS',
   network => '10.0.0.0/24',
   network_view => $network_view,
   ip_address => '10.0.0.10');

unless($response) {
   die("reset discovery data failed");
}
```

#Control the network discovery task

```
# stop network discovery
my $response = $session->network_discovery_control(
       action => 'END',
       network => '10.0.0.0/24');

unless($response) {
   die("Stop network discovery failed");
}
```

#Clean up

```
#Remove the network
$network = $session->get(
             object => 'Infoblox::DHCP::Network',
             network => '10.0.0.0/24',
             network_view => 'default');
unless($network) {
   die("network '10.0.0.0/24' couldn't be retrieved");
}
```

```
$session->remove($network)
   or die("Remove DHCP network failed: ",
               $session->status_code() . ":" . $session->status_detail());
print "DHCP network object removed successfully \n";

#Remove the network view
# (network2 will be removed with the view)
$network_view = $session->get(
             object => 'Infoblox::DHCP::View',
             name => 'netview');
unless($network_view) {
   die("network view 'netview' couldn't be retrived");
}
$session->remove($network_view)
   or die("Remove DHCP network view failed: ",
               $session->status_code() . ":" . $session->status_detail());
print "DHCP network view object removed successfully \n";
```

SCHEDULED OBJECT USAGE DETAILS

The following are the object types for which you can schedule tasks:

```
Infoblox::DHCP::FixedAddr
Infoblox::DHCP::NetworkContainer
Infoblox::DHCP::IPv6NetworkContainer
Infoblox::DHCP::Network
Infoblox::DHCP::IPv6Network
Infoblox::DHCP::SharedNetwork
Infoblox::DHCP::Range
Infoblox::DHCP::RoamingHost
Infoblox::DNS::BulkHost
Infoblox::DNS::Host
Infoblox::DNS::Record::A
Infoblox::DNS::Record::AAAA
Infoblox::DNS::Record::Alias
Infoblox::DNS::Record::CNAME
Infoblox::DNS::Record::DNAME
Infoblox::DNS::Record::MX
Infoblox::DNS::Record::PTR
Infoblox::DNS::Record::SRV
Infoblox::DNS::Record::TLSA
Infoblox::DNS::Record::TXT
Infoblox::DNS::SharedRecord::A
Infoblox::DNS::SharedRecord::AAAA
Infoblox::DNS::SharedRecord::MX
Infoblox::DNS::SharedRecord::SRV
Infoblox::DNS::SharedRecord::TXT
Infoblox::DNS::Zone
Infoblox::DNS::SRG
Infoblox::DNS::View
Infoblox::DTC::LBDN
Infoblox::DTC::Pool
Infoblox::DTC::Server
Infoblox::DTC::Topology
Infoblox::DTC::Monitor::HTTP
Infoblox::DTC::Monitor::SIP
Infoblox::DTC::Monitor::ICMP
Infoblox::DTC::Monitor::PDP
Infoblox::DTC::Monitor::TCP
```

The following are the functions for which you can schedule tasks:

```
restart
network_discovery_control
```

To allow limited-access admins to use the scheudling feature, add the permission with resource_type = "Scheduling" to the appropriate admin group (see Infoblox::Grid::Admin::Permission).

AUTHOR

Infoblox Inc. http://www.infoblox.com/

SEE ALSO

Infoblox::DNS::Record::A, Infoblox::DNS::Zone, Infoblox::DNS::View, Infoblox::DHCP::FailOver, Infoblox::DHCP::Network, Infoblox::DHCP::NetworkContainer, Infoblox::DHCP::View, Infoblox::Grid::CSVImportStatus, Infoblox::Grid::Member::QueryFQDNParameter, Infoblox::Grid::Member::QueryFQDNResponse, Infoblox::Grid::GlobalSmartFolder, Infoblox::Grid::PersonalSmartFolder, Infoblox::Grid::ScheduledTask

---

COPYRIGHT

Infoblox::Session - Session object to manipulate data within a grid.

Copyright (c) 2017 Infoblox Inc.

/storage/data/builds/nios-repo/products/one/script/temp-distrib/Infoblox/Util.pm

- SUMMARY
- DNSSEC ALGORITHMS
- __dnssec_algorithm_convert_helper__( )

SUMMARY

```
@dnssec_allowed_algorithms

&ibap_i2o_dnssec_key_params
&ibap_o2i_dnssec_key_params
&dnssec_algorithm_size_accessor
```

DNSSEC ALGORITHMS

```
algorithm mappings for dnssec KSK and ZSK  algorithms.
nst - value of 'next_secure_type'. as per DNSSECENH FS
NSEC3RSASHA256  and NSEC3RSASHA512 should be  replaced
with NSEC analogs.

%_alg_hash, %_rev_alg_hash  and  such  methods  as
__algorithm_convert_helper__, ibap_*_key_algorithm
methods  are  left  as  well  as  they are used by
Infoblox::DNS::DNSSecKeyRecord.
```

table 1

algorithm	maps	nst
RSAMD5	RSAMD5	NSEC
DSA	DSA	NSEC
RSASHA1	RSASHA1	NSEC
NSEC3DSA	DSA	NSEC3
NSEC3RSASHA1	RSASHA1	NSEC3
RSASHA256	RSASHA256	NSEC
RSASHA512	RSASHA512	NSEC
NSEC3RSASHA256	RSASHA256	NSEC3
NSEC3RSASHA512	RSASHA512	NSEC3
1	RSAMD5	NSEC
3	DSA	NSEC
5	RSASHA1	NSEC
8	RSASHA256	NSEC
10	RSASHA512	NSEC
6	DSA	NSEC3
7	RSASHA1	NSEC3

__dnssec_algorithm_convert_helper__( )

description:

```
use this method to convert old algorithm names
to a new algorithm values as per table 1.
```

parameters:

```
value   - required. one of @dnssec_allowed_algorithm
field   - required. 'dnssec_(ksk|zsk)_algorithm'
```

returns:

```
in array context both 'algorithm' and 'next_secure_type'
returned. in scalar context only 'next_secure_type' is returned
```

example:

```
my ($algorithm, $next_secure_type) = __dnssec_algorithm_convert_helper__('NSEC3RSASHA256');
```

Infoblox::CiscoISE::PublishSetting - A Cisco ISE subscribe settings.

- NAME
- DESCRIPTION
- CONSTRUCTOR
- SESSION METHODS
- MODULE METHODS
  - Infoblox::CiscoISE::Endpoint->subscribe_settings( )
- METHODS
  - enabled_attributes( )
  - mapped_ea_attributes( )
- AUTHOR
- SEE ALSO
- COPYRIGHT

NAME

Infoblox::CiscoISE::PublishSetting - A Cisco ISE subscribe settings.

DESCRIPTION

The Cisco ISE subscribe setting contains list of extensible attributes that are enabled for subscription from Cisco ISE server.

CONSTRUCTOR

```
 my $subscribe_setting = Infoblox::CiscoISE::PublishSetting->new(
     enabled_attributes   => ['DOMAINNAME' | 'ENDPOINT_PROFILE' | 'SECURITY_GROUP' | 'SESSION_STATE' |
'SSID' | 'USERNAME' | 'VLAN', ...], #Required
     mapped_ea_attributes => [$ea_assoc1, $ea_assoc2, ...],
#Optional / Default is undefined
 );
```

SESSION METHODS

The object does not have any session methods.

MODULE METHODS

This section describes all the functions that you can apply to Cisco ISE subscribe settings object.

Infoblox::CiscoISE::Endpoint->subscribe_settings( )

Use this method to set or retrieve the Cisco ISE endpoint subscribe settings. See Infoblox::CiscoISE::Endpoint->subscribe_settings() for parameters and return values.

Example

```
#Get subscribe_settings value
my $subscribe_settings = $object->subscribe_settings();

#Modify subscribe_settings value
$object->subscribe_settings($subscribe_settings);
```

METHODS

This section describes all the methods that you can use to configure and retrieve the attribute values of a Cisco ISE subscribe settings.

enabled_attributes( )

Use this method to set or retrieve the list of attributes that are enabled for subscription from the Cisco ISE server.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

The valid value is an array that contains following values: 'DOMAINNAME', 'ENDPOINT_PROFILE', 'SECURITY_GROUP', 'SESSION_STATE', 'SSID', 'USERNAME', 'VLAN'.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get enabled_attributes value
my $enabled_attributes = $object->enabled_attributes();

#Modify enabled_attributes value
$object->enabled_attributes(['DOMAINNAME', 'SECURITY_GROUP']);
```

mapped_ea_attributes( )

Use this method to set or retrieve the list of extensible attribute associations that are mapped to Cisco ISE attributes.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

The valid value is an array of Infoblox::CiscoISE::EAAssociation objects.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get mapped_ea_attributes value
my $mapped_ea_attributes = $object->mapped_ea_attributes();

#Modify mapped_ea_attributes value
$object->mapped_ea_attributes([$ea_assoc1, $ea_assoc2]);
```

AUTHOR

Infoblox Inc. http://www.infoblox.com/

SEE ALSO

Infoblox::CiscoISE::Endpoint, Infoblox::CiscoISE::Endpoint->subscribe_settings(), Infoblox::CiscoISE::EAAssociation,

---

COPYRIGHT

Copyright (c) 2017 Infoblox Inc.

Infoblox::Session - Session object to manipulate data within a grid.

- NAME
- DESCRIPTION
- CONSTRUCTOR
- METHODS
  - add( )
  - cc_mode( )
  - clear_dns_cache( )
  - clear_outbound_worker_log( )
  - clear_unmanaged( )
  - copy_records( )
  - copy_rpz_records( )
  - csv_import_control( )
  - get_all_template_vendor_id( )
  - get_rpz_members( )
  - get_template_schema_versions( )
  - default_maximum_objects_returned( )
  - default_partial_subobjects( )
  - empty_recycle_bin( )
  - export_data( )
  - fill_partial_object( )
  - gen_tsig_key( )
  - gen_hsm_client_cert( )
  - get( )
  - get_ksk_rollover( )
  - get_rpz_threat_details( )
  - get_ms_ad_dc_ns_record_creation_list( )
  - get_grid_revert_status( )
  - grid_upgrade( )
  - import_data( )
  - join_grid( )
  - leave_multi_grid( )
  - logout( )
  - member_upgrade( )
  - modify( )
  - new_cursor( )
  - network_discovery_control( )
  - publish_changes( )
  - query_fqdn_on_member( )
  - refresh_hsm( )
  - remove( )
  - remove_data( )
  - remove_eap_ca_cert( )
  - remove_tftp_file( )
  - reset_discovery_timestamp( )
  - reset_discovery_data( )
  - reset_threat_protection_object( )
  - restart( )
  - restart_status( )
  - rollback_db_snapshot( )
  - rotate_log_files( )
  - run_scavenging( )
  - save_db_snapshot( )
  - search( )
  - search with limit parameters
  - server_version( )
  - set_dhcp_failover_partner_down( )
  - set_dhcp_failover_secondary_recover( )
  - skip_member_upgrade( )
  - status_code( )
  - status_detail( )
  - test( )
  - test_hsm_status( )
  - upgrade_group_now( )
- SAMPLE CODE
- SCHEDULED OBJECT USAGE DETAILS
- AUTHOR
- SEE ALSO
- COPYRIGHT

NAME

Infoblox::Session - Session object to manipulate data within a grid.

DESCRIPTION

The Infoblox::Session object is the key object that is used to manipulate data within a grid. Creating a Session object establishes a communications channel with the grid master.

CONSTRUCTOR

```
 my $session = Infoblox::Session->new(
     master   => $address,  #Required
     password => $string,   #Required unless SSL_cert_file and SSL_key_file provided
     username => $string,   #Required unless SSL_cert_file and SSL_key_file provided
     new_password => $string, #Optional
     timeout  => $num,      #Optional, default is to use the LWP::UserAgent default timeout (180 seconds)
     connection_timeout => $num  #Optional, see below for more details
     default_maximum_objects_returned  => $num, #Optional, default is to return all matching objects for
search operations
     default_partial_subobjects  => 0 / 1, #Optional, default is 0 (no partial subobjects for search
operations)

     # The following options will be passed to LWP::UserAgent if specified. Please see the module
documentation for more details.
     verify_hostname => 0 / 1, #Optional. If set to 1, ensures that the hostname of the server will match
the hostname in its certificate.
     SSL_ca_path => $string, #Optional. The path to a directory that contains the Certificate Authority
certificates. Ignored unless verify_hostname is 1.
     SSL_ca_file => $string, #Optional. The path to a file that contains the Certificate Authority
certificates. Ignored unless verify_hostname is 1.
     SSL_cert_file => $string, #Optional, The path to a file that contains the OCSP Client Certificate.
SSL_key_file and SSL_cert_file should be defined together.
     SSL_key_file  => $string, #Optional, The path to a file that contains the OCSP Client Private Key.
SSL_key_file and SSL_cert_file should be defined together.
      );
```

If connection_timeout is provided and is greater than zero, the API will try to connect to the master's IP address with the specified connection_timeout first; and if it's successful, it will proceed to the log-in process.

The new_password option is required when the force password reset policy is enabled or when the user password is expired.

When the product is in Common Criteria mode, verify_hostname is automatically forced to 1. If verify_hostname is set to 1, SSL_ca_path and SSL_ca_file must be set unless the server certificate is signed by a Certificate Authority recognized by libwww-perl.

Note that SSL errors are not reported by the Perl API when debugging SSL connection issues (1006 'can't connect' errors). Infoblox recommends that you add either

```
 use IO::Socket::SSL qw(debug3);
```

or

```
 use Net::SSLeay;
 $Net::SSLeay::trace = 3;
```

to your Perl program before the "use Infoblox; line" to receive more complete debugging information.

For Common Criteria and SSL operation with certificate options in general, older versions of libwww-perl and supporting libraries might not work correctly. The following have been tested: libwww-perl 6.03, LWP::Protocol::https 6.02, IO::Socket::SSL 1.52, Net::SSLeay 1.42. Infoblox recommends performing a negative test in your environment to make sure connections with incorrect certificates will fail as expected.

For IPv6 operations, the PAPI library should also be imported with the :ipv6connection option. Note that older versions of libwww-perl might not allow connection to explicit IPv6 addresses, but only to Grid members via their FQDNs (Fully Qualified Domain Names).

METHODS

This section describes all the methods that you can use to configure and retrieve the attribute values of a Session object.

add( )

Use this method to add an object to the Infoblox appliance.

Parameter

The Infoblox object to be created in the Infoblox appliance. See individual objects for the necessary parameters.

scheduled_at - Optional. Scheduled date and time in ISO 8601 extended format (e.g., 2008-10-21T10:56:00Z or 2008-10-21T10:56:00.001Z) or 'now' which will set scheduled date and time to the current date and time. See details.

predecessor_task - Optional. An Infoblox::Grid::ScheduledTask object this task depends on. It is mandatory to set scheduled_at parameter if predecessor_task parameter is set.

approval_comment - Optional. Comment provided by the submitter on approval.

approval_ticket_number - Optional. Ticket number provided by the submitter on approval.

ip_validation - Optional. A list of Infoblox::Grid::IPValidationInfo objects.

Returns

The method returns true when the operation succeeds, and returns false when the operation fails. When an operation fails, the error code and error message are reported in the `status_code()` and `status_detail()` attributes.

Example

```
#Construct an object
my $bind_a = Infoblox::DNS::Record::A->new (
    ipv4addr => "1.1.1.1",
    name     => "bind_a.domain.com" );
# Submit for add
my $response = $session->add( $bind_a );
my $ipam_network = Infoblox::Grid::IPAM::Network->new(
   address => '10.20.0.0',
);

my $response = $session->add($ipam_network);
```

cc_mode( )

Use this method to determine if appliance is in Common Criteria mode.

Parameter

None.

Returns

The method returns "true" when the appliance is in Common Criteria mode.

Example

```
#Get cc_mode
my $cc_mode = $session->cc_mode();
```

clear_dns_cache( )

Use this method to clear the DNS cache.

Parameter

member - Required. The IPv4 address of the member on which you want to clear the DNS cache.

domain - Optional. The name of the domain to which the clear operation will be limited. The default is all domains.

view - Optional. The name of the DNS view to which the clear operation will be limited. The default is all views.

clear_full_tree - Optional. The flag that indicates whether the clear operation should be performed on all subdomains of the domain or not. The default is 'false'.

Returns

The method returns true when the operation succeeds, and returns false when the operation fails. When an operation fails, the error code and error message are reported in the `status_code()` and `status_detail()` attributes.

Example

```
# clear the entire DNS cache
my $response = $session->clear_dns_cache(
       member => "192.168.1.2");

unless($response) {
   die("Clear DNS cache failed");
}

# clear a DNS view from the DNS cache
$response = $session->clear_dns_cache(
       member => "192.168.1.2",
```

```
        view => "new_view",
);

unless($response) {
   die("Clear DNS view from DNS cache failed");
}

# clear a domain (including it's all subdomains) from the DNS cache
$response = $session->clear_dns_cache(
        member          => "192.168.1.2",
        domain          => "foo.bar.com",
        clear_full_tree => "true",
);

unless($response) {
   die("Clear the domain \"foo.bar.com\" from DNS cache failed");
}
```

clear_outbound_worker_log( )

Use this method to clear the outbound worker log.

Parameters

`endpoint - Required. The endpoint name.`

Returns

The valid return value is 0 if operation fails to occur or Infoblox::Grid::TestResult otherwise.

Example

```
#Clear outbound worker log
my $result = $session->clear_outbound_worker_log(
    endpoint => 'dxl_endpoint1');
```

clear_unmanaged( )

Use this method to clear unmanaged addresses.

Parameter

type - Required. Valid values are "GLOBAL", "NETWORK", and "IP_ADDRESS".

network - Optional. Required when type is "NETWORK" or "IP_ADDRESS", and the ip_address value is an IPv4 address.

network_view - Optional. The network view to which the clear operation will be limited. The default value is the system-defined default network view.

ip_address - Optional. Required when type is "IP_ADDRESS". The value can be an IPv4 or IPv6 address.

Returns

The method returns true when the operation succeeds, and returns false when the operation fails. When an operation fails, the error code and error message are reported in the `status_code()` and `status_detail()` attributes.

Example

```
# type is GLOBAL
my $response = $session->clear_unmanaged(
        type => 'GLOBAL');

unless($response) {
   die("Clear unmanaged address failed");
}

# with a non-default network view
my $nview = Infoblox::DHCP::View->new(
        name => 'netview',
);
my $response = $session->clear_unmanaged(
        type => 'GLOBAL',
        network_view => $nview);

unless($response) {
   die("Clear unmanaged address failed");
}
```

```
# type is NETWORK
$response = $session->clear_unmanaged(
        type => 'NETWORK',
        network => '10.0.0.0/24');

unless($response) {
   die("Clear unmanaged address failed");
}

# type is IP_ADDRESS
$response = $session->clear_unmanaged(
        type => 'IP_ADDRESS',
        network => '10.0.0.0/24',
        ip_address => '10.0.0.10');

unless($response) {
   die("Clear unmanaged address failed");
}

####PROGRAM ENDS####
```

copy_records( )

Copy records between DNS views.

Parameter

dest_view - Required. The destination DNS view.

dest_zone - Required. The destination DNS zone.

view - Required. The source DNS view.

zone - Required. The source DNS zone.

clear_dest_zone - Optional. Remove all records in the destination zone before copying the records. Default is "false".

copy_all_rr - Optional. Copy all records. Default is "false".

copy_bind_srv - Optional. Copy SRV records. Default is "false".

copy_bind_txt - Optional. Copy TXT records. Default is "false".

copy_bind_ptr - Optional. Copy PTR records. Default is "false".

copy_bulk_host - Optional. Copy Bulk Host records. Default is "false".

copy_bind_a - Optional. Copy A records. Default is "false".

copy_bind_aaaa - Optional. Copy AAAA records. Default is "false".

copy_bind_cname - Optional. Copy CNAME records. Default is "false".

copy_bind_dname - Optional. Copy DNAME records. Default is "false".

copy_bind_mx - Optional. Copy MX records. Default is "false".

copy_bind_naptr - Optional. Copy NAPTR records. Default is "false".

copy_host - Optional. Copy Host records. Default is "false".

replace_existing_records - Optional. Replace any existing records with the copied records if duplicate records are not allowed. Default is "false".

Returns

The method returns true when the modification succeeds, and returns false when the operation fails.

Example

```
#copy records from default view to new_view
$session->copy_records(
    view                    => "default",
    zone                    => "domain.com",
    dest_view               => "new_view",
    dest_zone               => "domain.com",
    copy_bind_srv           => "true",
    copy_bind_txt           => "true",
    copy_bind_ptr           => "false",
```

```
        copy_bulk_host           => "false",
        copy_bind_a              => "true",
        copy_bind_aaaa           => "true",
        copy_bind_cname          => "true",
        copy_bind_dname          => "true",
        copy_bind_mx             => "true",
        copy_host                => "true",
        replace_existing_records => "false",
        clear_dest_zone          => "false",
        copy_all_rr              => "false" );
```

copy_rpz_records( )

Copy RPZ records (rules) between DNS views.

Parameter

dest_view - Required. The destination DNS view.

dest_zone - Required. The destination DNS zone.

view - Required. The source DNS view.

zone - Required. The source DNS zone.

clear_dest_zone - Optional. Remove all records in the destination zone before copying the records. Default is "false".

copy_all_rr - Optional. Copy all records. Default is "false".

copy_passthru_ip_addr - Optional. Copy PASSTHRUIPADDR rules. Default is "false".

copy_passthru_domain - Optional. Copy PASSTHRUDOMAIN rules. Default is "false".

copy_block_nxd_ip_addr - Optional. Copy BLOCKNXDOMAINIPADDR rules. Default is "false".

copy_block_nxd_domain - Optional. Copy BLOCKNXDOMAINDOMAIN rules. Default is "false".

copy_block_nodata_ip_addr - Optional. Copy BLOCKNODATAIPADDR rules. Default is "false".

copy_block_nodata_domain - Optional. Copy BLOCKNODATADOMAIN rules. Default is "false".

copy_subst_cname - Optional. Copy SUBSTITUTE_CNAME rules. Default is "false".

copy_subst_a - Optional. Copy SUBSTITUTE_A_RECORD rules. Default is "false".

copy_subst_aaaa - Optional. Copy SUBSTITUTE_AAAA_RECORD rules. Default is "false".

copy_subst_mx - Optional. Copy SUBSTITUTE_MX_RECORD rules. Default is "false".

copy_subst_ptr - Optional. Copy SUBSTITUTE_PTR_RECORD rules. Default is "false".

copy_subst_naptr - Optional. Copy SUBSTITUTE_NAPTR_RECORD rules. Default is "false".

copy_subst_srv - Optional. Copy SUBSTITUTE_SRV_RECORD rules. Default is "false".

copy_subst_txt - Optional. Copy SUBSTITUTE_TXT_RECORD rules. Default is "false".

copy_subst_ipv4_addr - Optional. Copy SUBSTITUTE_IPV4ADDRESS_RECORD rules. Default is "false".

copy_subst_ipv6_addr - Optional. Copy SUBSTITUTE_IPV6ADDRESS_RECORD rules. Default is "false".

copy_subst_ipaddr_cname - Optional. Copy SUBSTITUTE_IPADDR_CNAME rules. Default is "false".

copy_subst_client_ipaddr_cname - Optional. Copy SUBSTITUTE_CLIENTIPADDR_CNAME rules. Default is "false".

copy_passthru_client_ipaddr - Optional. Copy PASSTHRUCLIENTIPADDR rules. Default is "false".

copy_block_nxdomain_client_ipaddr - Optional. Copy BLOCKNXDOMAINCLIENTIPADDR rules. Default is "false".

copy_block_no_data_client_ipaddr - Optional. Copy BLOCKNODATACLIENTIPADDR rules. Default is "false".

replace_existing_records - Optional. Replace any existing records with the copied records if duplicate records are not allowed. Default is "false".

Returns

The method returns true when the modification succeeds, and returns false when the operation fails.

Example

```
#copy rules (records) from default view to new_view
$session->copy_rpz_records(
    view                      => "default",
    zone                      => "rpz.com",
    dest_view                 => "new_view",
    dest_zone                 => "rpz.com",
    copy_subst_ipv4_addr      => "true",
    copy_subst_cname          => "true",
    copy_passthru_ip_addr     => "false",
    replace_existing_records => "false",
    clear_dest_zone           => "false",
    copy_all_rr               => "false" );
```

csv_import_control( )

Use this method to control the execution of the CSV import task.

Key Reference

```
action    - Required. Valid value is 'STOP' which stops the CSV import task.
import_id - Required. The id of the import task to control.
```

Returns

The method returns 1 if operation succeeds.

Example

```
#Stop CSV Import Task
my $result = $session->csv_import_control(action => 'STOP', import_id => '123456');
```

get_all_template_vendor_id( )

Use this method to retrieve the list of vendor identifiers associated with REST API templates.

Parameter

outbound_type - Optional. The outbound type for the vendor identifier. The valid values are 'REST' and 'DXL'. Default is 'REST'.

Returns

The method returns an array of vendor identifiers in a string format.

Example

```
#get vendor identifiers
my $result = $session->get_all_template_vendor_id(outbound_type => 'DXL');
```

get_rpz_members( )

Use this method to get a list of Infoblox::DNS::Member objects that are conformed to passed arguments.

Parameter

rpz_licensed - Optional. Requests members that have RPZ license properly installed. Default is "false".

rpz_enabled - Optionl. Requests members that have RPZ license properly installed and recursion enabled. Default is "false".

view - Optional. The DNS view to which the zone being added or updated belongs. If omitted, the default DNS view is used.

Returns

The method returns a list of Infoblox::DNS::Member objects when the requested members are found. It returns an empty list if no member conforms to the request.

Example

```
#get RPZ enabled members
$session->get_rpz_members(
       rpz_enabled = > "true");
```

get_template_schema_versions( )

Use this method to retrieve the list of schema versions for the RESTful API templates.

Parameter

```
 schema_type - Required. The REST API template type. The valid values are 'REST_EVENT' and
'REST_ENDPOINT'.
```

Returns

The method returns an array of template schema versions in a string format.

Example

```
#get template schema versions
my $result = $session->get_template_schema_versions();
```

default_maximum_objects_returned( )

Use this method to set or retrieve the default for the maximum objects returned in `search()` and `get()` operations.

Parameter

An unsigned integer.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

Example

```
#Set a new default
$session->default_maximum_objects_returned(100);
```

default_partial_subobjects( )

Use this method to set or retrieve the default for the partial subobjects feature in `search()` operations.

Parameter

Valid parameters are 0 or 1, where 1 means that the partial subobjects functionality is enabled.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

Example

```
#Enable partial_subobjects for searches
$session->default_partial_subobjects(1);
```

empty_recycle_bin( )

Remove the DNS/DHCP data from the recycle bin.

Parameter

none

Returns

The method returns true when the modification succeeds, and returns false when the operation fails.

Example

```
#empty recycle bin
my $response = $session->empty_recycle_bin ();
```

export_data( )

Use this method to download data from the Infoblox appliance to the local system.

Parameter

path - Required. The absolute path of the destination file for the data you want to export.

type - Required. The data type to export. Following is a list of available data types for export:

```
 +-------------------------------------------------------------------------------------------------
----------------+
 | admin_cert       - Export a member certificate. Exported file is in "pem" format.
|
 | member           - Required. The member from which you want to export the certificate.
|
 +-------------------------------------------------------------------------------------------------
```

```
----------------+
 | backup          - Export Infoblox appliance database. Exported file is in "tar.gz" format.
|
 | backup_type     - Optional. Valid values are LOCAL and SCP. Default is LOCAL.
|
 | backup_server   - Optional. The IPv4 address of the backup server.
|
 | user            - Optional. The user name of the backup server.
|
 | password        - Optional. The user password of the backup server.
|
 | nios_data       - Optional. Export NIOS data. Valid value is "true" or "false".
|
 |                   If nios_data, splunk_app_data and discovery_data are not defined, nios_data is
'true'.          |
 | discovery_data  - Optional. Export Network Automation data. Valid value is "true" or "false".
|
 |                   If nios_data, splunk_app_data and discovery_data are not defined, discovery_data
is 'false'.    |
 | splunk_app_data - Optional. Export Splunk application data. Valid value is "true" or "false".
|
 |                   If nios_data, splunk_app_data and discovery_data are not defined,
splunk_app_data is 'false'.   |
 +-------------------------------------------------------------------------------------------------
----------------+
 | captive_portal_cert   - Exports the server certificate of the captive portal.
|
 | dn                    - Required. The distinguished name (dn) of the certificate subject.
|
 +-------------------------------------------------------------------------------------------------
----------------+
 | csv               - Export data in a CSV file
|
|
|
 | object_type       - Required. The object type of the object to be exported
|
 | separator         - Optional. CSV separator character. Valid values are 'comma', 'semicolon',
'space' and 'tab'.  |
 |                     The default is 'comma'.
|
|
|
 | <various>         - Optional. A list of search parameters that identify the object(s) you want to
|
 |                     export. See the individual API object pages for the parameters,
which are           |
 |                     the same as the parameters that are passed to $session->search().
This document     |
 |                     provides information about the supported objects.
|
|
|
 |                     If the objects you want to export have a search parameter named
type, path,         |
 |                     object_type or separator, pass the search parameter with a prepended
|
 |                     underscore, such as '_type', '_separator', etc.
|
 +-------------------------------------------------------------------------------------------------
----------------+
 | csv_snapshot_file - Export the CSV import snapshot file.
|
|
|
 | import_id         - Optional. The ID of the import task for which the snapshot file was exported.
|
 |                     If not specified, the current export task will be used.
|
 +-------------------------------------------------------------------------------------------------
----------------+
 | csv_uploaded_file - Export the original CSV import file.
|
|
|
 | import_id         - Optional. The ID of the import task for which the original CSV file was
exported.            |
 |                     If not specified, the current export task will be used.
|
 +-------------------------------------------------------------------------------------------------
----------------+
 | csv_error_log  - Export the CSV import error log file.
|
 | import_id      - Optional. The ID of the import task for the log file you want to export. If not
specified,       |
 |                  the current export task ID is used.
|
 +-------------------------------------------------------------------------------------------------
----------------+
 | dnsCache       - Export the DNS dnsCache.db file of a member. Exported file is in "tar.gz" format.
The            |
 |                  following parameter is also available:
|
 | member         - Required. The member from which you want to download the dns cache file.
|
 +-------------------------------------------------------------------------------------------------
----------------+
```

```
 | dnsAccelerationCache - Export the DNS acceleration cache of a member. Exported file is in "tar.gz"
format. The    |
 |                        following parameter is also available:
|
 | member               - Required. The member from which you want to download the DNS acceleration
cache file.      |
 +-------------------------------------------------------------------------------------------------
---------------+
 | dnsStats       - Export the DNS statistics file of a member. Exported file is in "tar.gz" format.
The            |
 |                  following parameter is also available:
|
 | member         - Required. The member from which you want to download the DNS statistics file.
|
 +-------------------------------------------------------------------------------------------------
---------------+
 | dhcpd_conf     - Export the DHCP configuration file of a member. Exported file is in "tar.gz"
format. The         |
 |                  following parameter is also available:
|
 | member         - Required. The member from which you want to download the DHCP configuration file.
|
 +-------------------------------------------------------------------------------------------------
---------------+
 | dhcpdv6_conf   - Export the DHCPv6 configuration file of a member. Exported file is in "tar.gz"
format. The       |
 |                  following parameter is also available:
|
 | member         - Required. The member from which you want to download the DHCPv6 configuration
file.              |
 +-------------------------------------------------------------------------------------------------
---------------+
 | dhcp_expert_mode_config - Export the DHCP configuration file of a member in expert mode. Exported
file is in      |
 |                           "dhcpd.conf" format. The following parameter is also available:
|
 | member                  - Required. The member from which you want to download the DHCP
configuration export mode |
 |                           file.
|
 +-------------------------------------------------------------------------------------------------
---------------+
 | dns_conf     - Export the DNS configuration file of a member. Exported file is in "tar.gz" format.
The            |
 |                  following parameter is also available:
|
 | member       - Required. The member from which you want to download the DNS configuration file.
|
 +-------------------------------------------------------------------------------------------------
---------------+
 | ds_records     - Export the keyset associated with a given zone. This is useful for sending to the
parent zone    |
 |                  for signing.
|
 | zone           - Required. The zone that contain the keys to be exported.
|
 |                  Specify either the FQDN of the zone or the Infoblox::DNS::Zone object.
|
 | view           - Optional. The DNS view that contains the zone. Specify either the DNS view name
|
 |                  or Infoblox::DNS::View object. The default is the "default" view.
|
 +-------------------------------------------------------------------------------------------------
---------------+
 | eap_ca_cert    - Export an Extensible Authentication Protocol Certificate Authority (EAP CA)
certificate from the |
 |                  Infoblox appliance. Exported file is in "pem" format. The following parameter is
also            |
 |                  available:
|
 | dn             - Required. The distinguished name (dn) of your certificate's subject.
|
 |                           Examples of dn: CN="Testing",OU="Sales",O="Infoblox",L="Santa
Clara",ST="CA",C="US"    |
 |                                           O="My Company",ST="CA",C="US"
|
 |                                           CN="Test Common Name"
|
 +-------------------------------------------------------------------------------------------------
---------------+
 | generate_captive_portal_csr    - Generate and export a certificate signing request (CSR) for a
captive portal     |
 |                    Exported file is in "pem" format.
|
 | cn               - Required. Common name for the certificate.
|
 | member           - Required. The member for which you want to generate the CSR.
|
 | key_size         - Optional.  Number of bits in the key. It must be 1024, 2048 or 4096. Default is
2048.          |
 | algorithm        - Optional.  Digest algorithm. It must be either SHA-1 or SHA-256. Default is
SHA-256.         |
 | org_unit         - Optional.  Organizational unit used to construct the distinguished name of the
subject.         |
 | org              - Optional.  Organization used to construct the distinguished name of the
subject.           |
 | locality         - Optional.  Location (e.g., city, town) used to construct the distinguished name
```

```
of the subject.|
 | state            - Optional.  State or province used to construct the distinguished name of the
subject.          |
 | country          - Optional.  2-letter country code used to construct the distinguished name of
the subject.      |
 | email            - Optional.  Contact email address used to construct the distinguished name of
the subject.      |
 | comment          - Optional.  A descriptive comment about the certificate to be signed.
|
 +-------------------------------------------------------------------------------------------------
----------------+
 | generate_csr     - Generate and export a certificate signing request (CSR) for a grid member
|
 |                    Exported file is in "pem" format.
|
 | cn               - Required. Common name for the certificate.
|
 | member           - Required. The member for which you want to generate the CSR.
|
 | key_size         - Optional.  Number of bits in the key. It must be 1024, 2048 or 4096. Default is
2048.         |
 | algorithm        - Optional.  Digest algorithm. It must be either SHA-1 or SHA-256. Default is
SHA-256.          |
 | org_unit         - Optional.  Organizational unit used to construct the distinguished name of the
subject.        |
 | org              - Optional.  Organization used to construct the distinguished name of the
subject.          |
 | locality         - Optional.  Location (e.g., city, town) used to construct the distinguished name
of the subject.|
 | state            - Optional.  State or province used to construct the distinguished name of the
subject.          |
 | country          - Optional.  2-letter country code used to construct the distinguished name of
the subject.      |
 | email            - Optional.  Contact e-mail address used to construct the distinguished name of
the subject.     |
 | comment          - Optional.  A descriptive comment about the certificate to be signed.
|
 +-------------------------------------------------------------------------------------------------
----------------+
 | generate_ifmap_dhcp_csr - Generate and export a certificate signing request (CSR) for the IF-MAP
DHCP              |
 |                    Exported file is in "pem" format.
|
 | cn               - Required. Common name for the certificate.
|
 | member           - Required. The member for which you want to generate the CSR.
|
 | key_size         - Optional.  Number of bits in the key. It must be 1024, 2048 or 4096. Default is
2048.         |
 | algorithm        - Optional.  Digest algorithm. It must be either SHA-1 or SHA-256. Default is
SHA-256.          |
 | org_unit         - Optional.  Organizational unit used to construct the distinguished name of the
subject.        |
 | org              - Optional.  Organization used to construct the distinguished name of the
subject.          |
 | locality         - Optional.  Location (e.g., city, town) used to construct the distinguished name
of the subject.|
 | state            - Optional.  State or province used to construct the distinguished name of the
subject.          |
 | country          - Optional.  2-letter country code used to construct the distinguished name of
the subject.      |
 | email            - Optional.  Contact email address used to construct the distinguished name of
the subject.      |
 | comment          - Optional.  A descriptive comment about the certificate to be signed.
|
 +-------------------------------------------------------------------------------------------------
----------------+
 | generate_self_signed_admin_cert - Generate and export a self-signed certificate for a grid member
|
 |                    Exported file is in "pem" format.
|
 | cn               - Required. Common name for the certificate.
|
 | days_valid       - Required. Validity period for the certificate.
|
 | member           - Required. The member for which you want to generate the certificate.
|
 | key_size         - Optional.  Number of bits in the key. It must be 1024, 2048 or 4096. Default is
2048.         |
 | algorithm        - Optional.  Digest algorithm. It must be either SHA-1 or SHA-256. Default is
SHA-256.          |
 | org_unit         - Optional.  Organizational unit used to construct the distinguished name of the
subject.        |
 | org              - Optional.  Organization used to construct the distinguished name of the
subject.          |
 | locality         - Optional.  Location (e.g., city, town) used to construct the distinguished name
of the subject.|
 | state            - Optional.  State or province used to construct the distinguished name of the
subject.          |
 | country          - Optional.  2-letter country code used to construct the distinguished name of
the subject.      |
 | email            - Optional.  Contact email address used to construct the distinguished name of
the subject.      |
 | comment          - Optional.  A descriptive comment about the certificate to be signed.
|
 +-------------------------------------------------------------------------------------------------
----------------+
```

```
 | generate_self_signed_captive_portal_cert - Generate and export a self-signed certificate for a
captive portal     |
 |                    Exported file is in "pem" format.
|
 | cn                - Required. Common name for the certificate.
|
 | days_valid        - Required. Validity period for the certificate.
|
 | member            - Required. The member for which you want to generate the certificate.
|
 | key_size          - Optional.  Number of bits in the key. It must be 1024, 2048 or 4096. Default is
2048.          |
 | algorithm         - Optional.  Digest algorithm. It must be either SHA-1 or SHA-256. Default is
SHA-256.          |
 | org_unit          - Optional.  Organizational unit used to construct the distinguished name of the
subject.         |
 | org               - Optional.  Organization used to construct the distinguished name of the
subject.           |
 | locality          - Optional.  Location (e.g., city, town) used to construct the distinguished name
of the subject.|
 | state             - Optional.  State or province used to construct the distinguished name of the
subject.          |
 | country           - Optional.  2-letter country code used to construct the distinguished name of
the subject.       |
 | email             - Optional.  Contact email address used to construct the distinguished name of
the subject.       |
 | comment           - Optional.  A descriptive comment about the certificate to be signed.
|
 +-------------------------------------------------------------------------------------------------
----------------+
 | generate_self_signed_ifmap_dhcp_cert - Generate and export a self-signed certificate for the IF-
MAP DHCP           |
 |                    Exported file is in "pem" format.
|
 | cn                - Required. Common name for the certificate.
|
 | days_valid        - Required. Validity period for the certificate.
|
 | member            - Required. The member for which you want to generate the certificate.
|
 | key_size          - Optional.  Number of bits in the key. It must be 1024, 2048 or 4096. Default is
2048.          |
 | algorithm         - Optional.  Digest algorithm. It must be either SHA-1 or SHA-256. Default is
SHA-256.          |
 | org_unit          - Optional.  Organizational unit used to construct the distinguished name of the
subject.         |
 | org               - Optional.  Organization used to construct the distinguished name of the
subject.           |
 | locality          - Optional.  Location (e.g., city, town) used to construct the distinguished name
of the subject.|
 | state             - Optional.  State or province used to construct the distinguished name of the
subject.          |
 | country           - Optional.  2-letter country code used to construct the distinguished name of
the subject.       |
 | email             - Optional.  Contact email address used to construct the distinguished name of
the subject.       |
 | comment           - Optional.  A descriptive comment about the certificate to be signed.
|
 +-------------------------------------------------------------------------------------------------
----------------+
 | ifmap_dhcp_cert   - Export an IF-MAP DHCP certificate. Exported file is in "pem" format.
|
 | member            - Required. The member from which you want to export the certificate.
|
 +-------------------------------------------------------------------------------------------------
----------------+
 | log_files         - Export the specified log files from a member. Exported file is in "tar.gz"
format (except
                     'OUTBOUND' in case of which the exported file format is plane text. The
following parameters   |
                     are also available:
|
 | member            - The member from which you are downloading the logs.
|
 |                     You must specify a member, endpoint or a Microsoft(r) server.
|
 | msserver          - The Microsoft server from which you are downloading the logs. You must specify
a member or     |
 |                     a Microsoft server.
|
 | endpoint          - The REST API endpoint name.
|
 | node_type         - Optional. Node type. Valid values are "active" and "passive". The default is
"active".          |
 | log_type          - Optional. Log type. Valid values are "audit", "msmgmt", "outbound" and
"syslog". The default   |
 |                     is "syslog".
|
 | include_rotated   - Optional. Include rotated out log files. The default is false
|
 +-------------------------------------------------------------------------------------------------
----------------+
 | lease_history     - Export the DHCP lease history. Exported file is in "gz" format.
|
 | start_time      - Optional. The start time of the exported lease history in the format yyyy-mm-
ddThh:mm:ss,         |
 |                   yyyy/mm/dd hh:mm:ss, or mm/dd/yyyy hh:mm:ss.
```

```
|
 | end_time       - Optional. The end time of the exported lease history in the format yyyy-mm-
ddThh:mm:ss,          |
 |                  yyyy/mm/dd hh:mm:ss, or mm/dd/yyyy hh:mm:ss.
|
 +-------------------------------------------------------------------------------------------------
----------------+
 | snmp_mibs      - Export the Infoblox SNMP MIB files. Exported file is in "tar.gz" format.
|
 +-------------------------------------------------------------------------------------------------
----------------+
 | support_bundle        - Export the support bundle from a member. Exported file is in "tar.gz"
format. The         |
 |                         following parameters are also available:
|
 | member                - Required. The member from which you are downloading the support bundle.
|
 | filter_optional_files - Optional. Filter optional files. Default is "true".
|
 | core_files            - Optional. Include core files in support bundle. Default is "true".
|
 | log_files             - Optional. Include log files in support bundle. Default is "true".
|
 | rotate_log_files      - Optional. Include rotate_log files in support bundle. Default is "true".
|
 | cached_zone_data      - Optional. Include cached zone data files in support bundle. Default is
"false".            |
 +-------------------------------------------------------------------------------------------------
----------------+
 | serial_numbers - Export the serial numbers of all nodes in the grid. Exported file is in plain
text format.        |
 +-------------------------------------------------------------------------------------------------
----------------+
 | traffic_capture_file  - Export the member's traffic capture file. The exported file is in "tar.gz"
format          |
 |
|
 | member           - Required: The member from which you are downloading the traffic capture file.
|
 +-------------------------------------------------------------------------------------------------
----------------+
 | trust_anchors  - Export trust anchors, i.e. our public keys to allow creation of the island of
security.           |
 |
|
 | zone           - Required. The zone that contains the keys to be exported.
|
 |                  Specify either the FQDN of the zone or the Infoblox::DNS::Zone object.
|
 | view           - Optional. The DNS view that contains the zone. Specify either the DNS view name
or the           |
 |                  Infoblox::DNS::View object. The default is the "default" view.
|
 +-------------------------------------------------------------------------------------------------
----------------+
 | restapi_template - Export the REST API template in JSON format.
|
 |
|
 | template         - Required. An Infoblox::Notification::REST::Template object.
|
 +-------------------------------------------------------------------------------------------------
----------------+
 | restapi_template_schema - Export the REST API template schema in JSON format.
|
 |
|
 | schema_type             - Requred. Schema type to export: 'REST_ENDPOINT' or 'REST_EVENT'.
|
 | version                 - Optional. Schema version to export. The default is the latest version.
|
 +-------------------------------------------------------------------------------------------------
----------------+
 | dxl_endpoint_certs - Export DXL endpoint client and CA certificates. Path argument is the
destination for the     |
 |                     DXL endpoint client certificate.
|
 |
|
 | ca_cert_path - Required. The absolute path of the destination file for the CA certificate you want
to export.      |
 +-------------------------------------------------------------------------------------------------
----------------+
 | safenet_client_cert   - Export a SafeNet HSM client certificate. Exported file is in "pem" format.
|
 | member                - Required. The member from which you want to export the certificate.
|
 | algorithm             - Required. Signature algorithm of a new SafeNet HSM client certificate.
|
 |                         The value can be 'RSASHA1' or 'RSASHA256'.
|
 +-------------------------------------------------------------------------------------------------
----------------+
 | namedacl(type)        - Validate access control items of the Named ACL object. If access control
items of          |
 |                         Named ACL object are ok, the server doesn't return a result file,
otherwise exported file |
```

```
 |                          is in CSV format.
|
 | namedacl              - Required. The Named ACL object.
|
 +-------------------------------------------------------------------------------------------------
----------------+
```

For CSV export, the following objects are supported:

```
 Infoblox::DHCP::FailOver
 Infoblox::DHCP::Filter::Fingerprint
 Infoblox::DHCP::Filter::MAC
 Infoblox::DHCP::Filter::NAC
 Infoblox::DHCP::Filter::Option
 Infoblox::DHCP::Filter::RelayAgent
 Infoblox::DHCP::Fingerprint
 Infoblox::DHCP::FixedAddr
 Infoblox::DHCP::HostAddr
 Infoblox::DHCP::IPv6DhcpRange
 Infoblox::DHCP::IPv6FixedAddress
 Infoblox::DHCP::IPv6HostAddr
 Infoblox::DHCP::IPv6Network
 Infoblox::DHCP::IPv6NetworkContainer
 Infoblox::DHCP::IPv6NetworkContainer
 Infoblox::DHCP::IPv6OptionDefinition
 Infoblox::DHCP::IPv6OptionSpace
 Infoblox::DHCP::IPv6SharedNetwork
 Infoblox::DHCP::Lease
 Infoblox::DHCP::MAC
 Infoblox::DHCP::Network
 Infoblox::DHCP::NetworkContainer
 Infoblox::DHCP::NetworkContainer
 Infoblox::DHCP::OptionDefinition
 Infoblox::DHCP::OptionSpace
 Infoblox::DHCP::Range
 Infoblox::DHCP::SharedNetwork
 Infoblox::DHCP::View
 Infoblox::DNS::AllRPZRecords
 Infoblox::DNS::AllRecords
 Infoblox::DNS::BulkHost
 Infoblox::DNS::Host
 Infoblox::DNS::RPZRecord::A
 Infoblox::DNS::RPZRecord::AAAA
 Infoblox::DNS::RPZRecord::AAAAIpAddress
 Infoblox::DNS::RPZRecord::AIpAddress
 Infoblox::DNS::RPZRecord::CNAME
 Infoblox::DNS::RPZRecord::CNAME::ClientIpAddress
 Infoblox::DNS::RPZRecord::CNAME::ClientIpAddressDN
 Infoblox::DNS::RPZRecord::CNAME::IpAddress
 Infoblox::DNS::RPZRecord::CNAME::IpAddressDN
 Infoblox::DNS::RPZRecord::MX
 Infoblox::DNS::RPZRecord::NAPTR
 Infoblox::DNS::RPZRecord::PTR
 Infoblox::DNS::RPZRecord::SRV
 Infoblox::DNS::RPZRecord::TXT
 Infoblox::DNS::Record::A
 Infoblox::DNS::Record::AAAA
 Infoblox::DNS::Record::Alias
 Infoblox::DNS::Record::CNAME
 Infoblox::DNS::Record::DNAME
 Infoblox::DNS::Record::MX
 Infoblox::DNS::Record::NAPTR
 Infoblox::DNS::Record::NS
 Infoblox::DNS::Record::PTR
 Infoblox::DNS::Record::SRV
 Infoblox::DNS::Record::TLSA
 Infoblox::DNS::Record::TXT
 Infoblox::DNS::Ruleset
 Infoblox::DNS::View
 Infoblox::DNS::Zone
 Infoblox::DTC::Topology
 Infoblox::DTC::Topology::Rule
 Infoblox::Grid::Analytics::WhiteList
 Infoblox::Grid::DHCP
 Infoblox::Grid::DNS
 Infoblox::Grid::DNS::DNS64Group
 Infoblox::Grid::DNS::Nsgroup
 Infoblox::Grid::Member
 Infoblox::Grid::Member::DHCP
 Infoblox::Grid::Member::DNS
 Infoblox::Grid::NamedACL
 Infoblox::Grid::ThreatProtection::Profile
 Infoblox::Grid::ThreatProtection::Profile::Rule
 Infoblox::Grid::UpgradeGroup
```

For more information about the CSV file format please refer to the Infoblox CSV Import Reference.

Returns

The method returns true when the modification succeeds, and returns false when the operation fails. The exported data is stored in the path specified.

Example

```
#Export backup
$session->export_data(
    type   => "backup",
    path   => "/tmp/infoblox/database.tar.gz" );

#Secured backup
my $result = $session->export_data(
    type   => "backup",
    backup_type => "SCP",
    path   => "/tmp/infoblox/database.tar.gz",
    backup_server => "192.168.0.5",
    user => "testuser",
    password => "test-password",
    );

#CSV export
$session->export_data(
                      type        => "csv",
                      path        => "/tmp/infoblox/exported_data.csv",
                      object_type => "Infoblox::DNS::Record::A",
                      separator   => "comma",

                      # search parameters
                      name        => ".*test_exportimport.com");

#CSV error log export
$session->export_data(
                      type      => "csv_error_log",
                      path      => "/tmp/infoblox/error_log.txt",
                      import_id => 12345);

#Export dnsCache.db
my $result = $session->export_data(
    type   => "dnsCache",
    path   => "/tmp/infoblox/dnsCache.tar.gz",
    member => "192.168.1.2" );

#Export dnsStats
my $result = $session->export_data(
    type   => "dnsStats",
    path   => "/tmp/infoblox/dnsStats.tar.gz",
    member => "192.168.1.2" );

#Export dhcpd_conf
my $result = $session->export_data(
    type   => "dhcpd_conf",
    path   => "/tmp/infoblox/dhcpd_conf.tar.gz",
    member => "192.168.1.2" );

#Export dhcpdv6_conf
my $result = $session->export_data(
    type   => "dhcpdv6_conf",
    path   => "/tmp/infoblox/dhcpdv6_conf.tar.gz",
    member => "192.168.1.2" );

#Export dhcp_expert_mode_config
my $result = $session->export_data(
    type   => "dhcp_expert_mode_config",
    path   => "/tmp/infoblox/dhcpd_expert.conf",
    member => "192.168.1.2" );

#Export dns_conf
my $result = $session->export_data(
    type   => "dns_conf",
    path   => "/tmp/infoblox/dns_conf.tar.gz",
    member => "192.168.1.2" );

#Export DS records
my $result = $session->export_data(
    type   => "ds_records",
    zone   => "mydnsseczone.com",
    view   => "dnssec_view",
    path   => "/tmp/infoblox/ds_records.txt" );

#Export captive_portal_cert
my $result = $session->export_data(
    type => "captive_portal_cert",
    path => "/tmp/infoblox/captive_portal_cert.pem",
    dn   => 'CN="My CA Certificate",C="US"' );
```

```
#Export eap_ca_cert
my $result = $session->export_data(
    type => "eap_ca_cert",
    path => "/tmp/infoblox/eap_ca_cert.pem",
    dn   => 'CN="My CA Certificate",C="US"' );

#Generate and export a CSR For captive portal
my $result = $session->export_data(
    type     => "generate_captive_portal_csr",
    path     => "/tmp/infoblox/csr.pem",
    cn       => "My Signing Request",
    key_size =>  2048,
    org_unit => "My Dept.",
    org      => "My Company, Inc.",
    locality => "Santa Clara",
    state    => "California",
    member   => "192.168.1.2",
    country  => "US",
    email    => "me\@mycompany.com",
    comment  => "This is my certificate." );

#Generate and export a CSR For IF-MAP DHCP
my $result = $session->export_data(
    type     => "generate_ifmap_dhcp_csr",
    path     => "/tmp/infoblox/dhcp_csr.pem",
    cn       => "My Signing Request",
    key_size =>  2048,
    org_unit => "My Dept.",
    org      => "My Company, Inc.",
    locality => "Santa Clara",
    state    => "California",
    member   => "192.168.1.2",
    country  => "US",
    email    => "me\@mycompany.com",
    comment  => "This is my certificate." );

#Generate and export a self-signed admin certificate
my $result = $session->export_data(
    type       => "generate_self_signed_admin_cert",
    path       => "/tmp/infoblox/selfsigned.pem",
    cn         => "Self-Signed Admin Cert",
    member     => "192.168.1.2",
    days_valid => 365,
    key_size   => 2048,
    org_unit   => "My Dept.",
    org        => "My Company, Inc.",
    locality   => "Santa Clara",
    state      => "California",
    country    => "US",
    email      => "me\@mycompany.com",
    comment    => "This certificate is self-signed." );

#Generate and export a self-signed captive portal certificate
my $result = $session->export_data(
    type       => "generate_self_signed_captive_portal_cert",
    path       => "/tmp/infoblox/selfsigned.pem",
    cn         => "Self-Signed Captive Portal Cert",
    member     => "192.168.1.2",
    days_valid => 365,
    key_size   => 2048,
    org_unit   => "My Dept.",
    org        => "My Company, Inc.",
    locality   => "Santa Clara",
    state      => "California",
    country    => "US",
    email      => "me\@mycompany.com",
    comment    => "This certificate is self-signed." );

#Generate and export a self-signed IF-MAP DHCP certificate
my $result = $session->export_data(
    type       => "generate_self_signed_ifmap_dhcp_cert",
    path       => "/tmp/infoblox/selfsigned.pem",
    cn         => "Self-Signed IF-MAP DHCP Cert",
    member     => "192.168.1.2",
    days_valid => 365,
    key_size   => 2048,
```

```
        org_unit   => "My Dept.",
        org        => "My Company, Inc.",
        locality   => "Santa Clara",
        state      => "California",
        country    => "US",
        email      => "me\@mycompany.com",
        comment    => "This certificate is self-signed." );

    #Generate and export a CSR
    my $result = $session->export_data(
        type     => "generate_csr",
        path     => "/tmp/infoblox/csr.pem",
        cn       => "My Signing Request",
        key_size =>  2048,
        org_unit => "My Dept.",
        org      => "My Company, Inc.",
        locality => "Santa Clara",
        state    => "California",
        member   => "192.168.1.2",
        country  => "US",
        email    => "me\@mycompany.com",
        comment  => "This is my certificate." );

    #Export an IF-MAP DHCP certificate
    my $result = $session->export_data(
        type   => "ifmap_dhcp_cert",
        member => "192.168.1.2",
        path   => "/tmp/infoblox/ifmap_cert.pem");

    #Export lease_history
    $session->export_data(
        type   => "lease_history",
        path   => "/tmp/infoblox/lease_history.gz",
        start_time  => "2005/09/01 00:00:00",
        end_time    => "2005/09/01 11:59:59" );

    #Export the audit log
    $session->export_data(
        type   => "log_files",
        path   => "/tmp/infoblox/auditlog.tar.gz",
        member => "192.168.1.2",
        node_type => "active",
        log_type => "audit");

    #Export snmp_mibs
    $session->export_data(
        type   => "snmp_mibs",
        path   => "/tmp/infoblox/snmp_mibs.tar.gz" );

    #Export support_bundle
    my $result = $session->export_data(
        type   => "support_bundle",
        path   => "/tmp/infoblox/supportBundle.tar.gz",
        member => "192.168.1.2" );

    #Export support_bundle but exclude core files
    my $result = $session->export_data(
        type   => "support_bundle",
        path   => "/tmp/infoblox/supportBundle.tar.gz",
        core_files => 'false',
        member => "192.168.1.2" );

    #Export support_bundle but exclude log files
    my $result = $session->export_data(
        type   => "support_bundle",
        path   => "/tmp/infoblox/supportBundle.tar.gz",
        log_files => 'false',
        member => "192.168.1.2" );

    #Export serial number of all nodes
    my $result = $session->export_data(
        type   => "serial_numbers",
        path   => "/tmp/infoblox/serial_nos.txt" );

    #Export trust anchors
    my $result= $session->export_data(
        type   => "trust_anchors",
```

```
        zone    => "mysignedzone.com",
        view    => "my_dnssec_view",
        path    => "/tmp/infoblox/trusted_keys.txt" );
```

fill_partial_object( )

Takes an object returned by a search method call with return_methods and fills it.

Parameter

object - Required. An object or a reference to an array of objects returned by a search method call using return_methods call, or from an AllRecords 'record' member. It is legal, but meaningless, to call this method for complete objects.

Returns

The method will return a complete object corresponding to the 'partial' object passed as a parameter. The object returned may be the same object (modified to have all methods initialized) or a new object. In the case of an error, undef will be returned and the appropriate status code will be set.

Example

```
# $partial_bind_object is an object returned by a search using return_methods
my $full_bind_object = $session->fill_partial_object($partial_bind_object);

# @partial_objects is an array of objects returned by a search using return_methods
my @full_objects = $session->fill_partial_object(\@partial_objects);
```

gen_tsig_key( )

Generates a TSIG key.

Parameter

The following parameters are supported:

```
algorithm => "HMAC-MD5" | "HMAC-SHA256" # Optional. If not specified, the algorithm is "HMAC-MD5".
keylen => 128 | 256 | 512 # Optional. If not specified, the key length is 128.
```

Note that the algorithm "HMAC-SHA256" does not support a key length of 512.

Returns

The method returns a randomly generated TSIG key.

Example

```
my $tsig = $session->gen_tsig_key(keylen => 256, algorithm => 'HMAC-SHA256');
```

gen_hsm_client_cert( )

Generate a new SafeNet HSM client certificate.

Parameter

member - Required. The member for which you want to generate a new SafeNet HSM client certificate.

algorithm - Required. Signature algorithm of a new SafeNet HSM client certificate. The value can be 'RSASHA1' or 'RSASHA256'.

Returns

The method returns true when a new SafeNet HSM client certificate is generated successfully, and returns false when the operation fails.

Example

```
my $result = $session->gen_hsm_client_cert('member' => '192.168.1.2', 'algorithm' => 'RSASHA1');
```

get( )

Use this method to retrieve all the matching objects from the Infoblox appliance.

Parameter

object - Required. The Infoblox object to retrieve.

key reference - Required. The attribute and value that refer to the desired `object(s)` to retrieve. See individual objects for available key references.

Returns

Return an array in list context containing all matching objects. For failures and objects not found, it returns an empty array in list context and undefined in scalar context. When an operation fails, the error code and error message are reported in the `status_code()` and `status_detail()` attributes.

Example

```
#Getting an A record from the Infoblox appliance.
my @retrieved_objs = $session->get(
    object => "Infoblox::DNS::Record::A",
    name   => "bind_a.domain.com"  );

unless (@retrieved_objs) {
    die("Get DNS record A failed: ",
       $session->status_code() . ":" . $session->status_detail());
}
```

get_ksk_rollover( )

Use this method to retrieve the KSK rollover information.

Parameter

days - Optional. The number of days since the warning was sent and before the rollover period expires. The default is 7 days.

Returns

The method returns a reference to an array that contains references to hashes representing the zones' KSK information. These hashes contain the following key/value pairs:

```
 name => The zone name
 view => The name of the view to which the zone belongs.
 days => The number of days that are left before the rollover period expires. If the rollover period
has expired, this will be a negative number.
```

Example

```
#Retrieve the KSK information:
my $resultref = $session->get_ksk_rollover();

#Assuming that there is at least one zone requiring rollover:
$zone = @$resultref[0]

#Now $zone contains the above parameters. To print the zone name:
print $zone->{'name'};
```

get_rpz_threat_details( )

Use this method to make a RESTful API call to the ThreatStop RESTful API to receive threat details.

Include the specified parameter to retrieve the attribute value.

Parameter

The valid value is a string containing rpz_rule_name. This is a required parameter.

Returns

If you specified a parameter, the method returns an Infoblox::Grid::RPZ::ThreatDetails structure when the fetch succeeds.

Example

```
#Retrieve rpz_threat_details:
my $rpz_threat_details = $session->get_rpz_threat_details(rpz_rule_name => "b.com.a.com");
```

get_ms_ad_dc_ns_record_creation_list( )

Use this method to get the list of Domain Controllers allowed for creation of NS Record.

Key Reference

```
 zone       - Required if query_type is 'ZONE' or no 'ms_server' is given. The Infoblox::DNS::Zone
object.
 ms_server  - Required if query_type is 'SERVERS_IN_DOMAIN' or no 'zone' is given. The
Infoblox::Grid::MSServer object.
 query_type - Optional. Default value is 'ZONE' if zone argument is given, 'SERVERS_IN_DOMAIN' if
ms_server argument is given.
```

Returns

The valid return value is an array of Infoblox::Grid::MSServer::DCNSRecordCreation objects or empty list.

Example

```
my $dcnsrecordcr = $session->get_ms_ad_dc_ns_record_creation_list(zone => $dns_zone);
```

get_grid_revert_status( )

Retrieves the revert status of an Infoblox Grid.

Parameter

none

Returns

The method returns a reference to a hash containing the following members.

```
revert_allowed => "true" | "false". Indicates whether revert is allowed for the Grid.
revert_version => $string. The version to which the Grid reverts.
```

Example

```
my $revertref = $session->get_grid_revert_status();
```

grid_upgrade( )

This method will perform the specified function on the Grid.

Parameter

The following required parameter is supported:

```
action => one of the following: "UPGRADE_PAUSE", "UPGRADE_RESUME", "DISTRIBUTION_PAUSE",
"DISTRIBUTION_RESUME", "DISTRIBUTION_START", "DISTRIBUTION_STOP", "DOWNGRADE", "REVERT", "UPGRADE",
"UPGRADE_TEST_START", "UPGRADE_TEST_STOP", or "UPLOAD".
```

Returns

The method returns true when the operation succeeds, and returns false when the operation fails.

Example

```
#grid upgrade
my $response = $session->grid_upgrade(action => "UPGRADE_PAUSE");
```

import_data( )

Use this method to upload data to the Infoblox appliance from the local system.

Parameter

path - Required. The complete path of the file to import.

type - Required. The data type to import. Following is a list of available data types for import:

```
 +-------------------------------------------------------------------------------------------------
----------------+
 | threat_analytics_file_upload - Import Apache Spark or Threat Analytics Whitelist update.
|
 +-------------------------------------------------------------------------------------------------
----------------+
 | restapi_template - Import the REST API template from file in JSON format.
|
 | overwrite        - Optional. If 'true' overwrites the existing template based on the template
name, vendor        |
 |                    identifier, event type, and action type. The default is 'false'.
|
 +-------------------------------------------------------------------------------------------------
----------------+
 | admin_cert       - Import a certificate signed by a CA for a certificate signing request
previously given by a    |
 |                    member. The input file should be in "pem" format.
|
 | member           - Required. The member to which you want to upload the certificate.
|
 +-------------------------------------------------------------------------------------------------
----------------+
 | backup          - Restore an existing database backup. Supported file type is "tar.gz". The
following parameters  |
 |                   are also available:
|
 | force           - Optional. Force restore a database that was backed up from
```

```
|
 |                   a different grid. Default is "false".
|
 | keep_grid_ip    - Optional. Used with the force restore option to retain the current grid IP
address and ignore   |
 |                   the IP address from the backup. Default is "true".
|
 | nios_data       - Optional. Restore NIOS data. Valid value is "true" or "false".
|
 |                   If nios_data, discovery_data and splunk_app_data are not defined, nios_data is
'true'.          |
 | discovery_data  - Optional. Restore Network Automation data. Valid value is "true" or "false".
|
 |                   If nios_data, splunk_app_data and discovery_data are not defined, discovery_data
is 'false'.    |
 | splunk_app_data - Optional. Restore Splunk application data. Vali value is "true" or "false".
|
 |                   If nios_data, splunk_app_data and discovery_data are not defined,
splunk_app_data is 'false'.  |
 +-------------------------------------------------------------------------------------------------
----------------+
 | dsb             - Import device support bundle. Supported file type is "tar.gz".
|
 +-------------------------------------------------------------------------------------------------
----------------+
 | captive_portal_cert   - Import a server certificate for the captive portal.
|
 |                         Supported file type is "pem".
|
 | member                - Required. The captive portal to which you want to upload the certificate.
|
 +-------------------------------------------------------------------------------------------------
----------------+
 | captive_portal_file   - Import a file to be used by the captive portal.
|
 |                         Supported file types are .jpg, .gif and .png (when cp_type is one of
'logo', 'footer' or  |
 |                         'header') or .txt (when cp_type is set to 'aup')
|
 | member                - Required. The captive portal to which you want to upload the file.
|
 | cp_type               - Required. Indicates how the file is used in the captive portal. Valid
values are:          |
 |                                   'logo', 'footer', 'header', 'aup'.
|
 +-------------------------------------------------------------------------------------------------
----------------+
 | clone           - For cloned data from the backup of another system. Supported file type is
"tar.gz".              |
 +-------------------------------------------------------------------------------------------------
----------------+
 | csv               - Import DNS and DHCP data via CSV files. See export_data/csv for the supported
object types.   |
 |                     For more information about the CSV file format, refer to the Infoblox CSV
Import Reference.   |
 |                     Use Infoblox::Grid::CSVImportStatus to check import task status.
|
 |
|
 | csv_action        - Optional. Valid values are "start" and "test". The default value is "start".
|
 |                               You cannot set csv_action to "test" if operation is not "replace".
|
 | operation         - Optional. Valid values are "insert", "update", "replace", "custom" and
"delete".              |
 |                               The default value is "insert".
|
 | override          - Optional. Valid values are "override" and "merge". The default value is
"override".           |
 | separator         - Optional. CSV separator character. Valid values are 'comma', 'semicolon',
'space' or 'tab'.   |
 |                               The default value is 'comma'.
|
 | continue_on_error - Optional. Continue import in case of errors. Valid values are "true" or
"false".              |
 |                               The default is "false". If the value is "true", the import continues
even if there  |
 |                               are errors. Note that file format errors are reported immediately.
|
 +-------------------------------------------------------------------------------------------------
----------------+
 | dhcp_expert_mode_config - Import a custom DHCP configuration file from the local system to the
Infoblox           |
 |                           appliance. Supported file type is "dhcpd.conf". The following parameter
is also         |
 |                           available:
|
 | member                  - Required. The member to which you want to upload the DHCP configuration
expert mode     |
 |                           file.
|
 +-------------------------------------------------------------------------------------------------
----------------+
 | downgrade       - NIOS image file that will be used to downgrade the Grid.
|
 +-------------------------------------------------------------------------------------------------
----------------+
```

```
 | ds_records     - Import DS records from a file. The file format is either a DSset (DS records) or
a keyset        |
 |                  (DNSKEY records).
|
 | zone           - Required.  The zone that will contain the records.
|
 |                  Specify either the FQDN of the zone ore the Infoblox::DNS::Zone object.
|
 | view           - Optional. The DNS view that contains the zone. Specify either the DNS view name
or the          |
 |                  Infoblox::DNS::View object. The default is the "default" view.
|
 +-------------------------------------------------------------------------------------------------
----------------+
 | eap_ca_cert    - Import an Extensible Authentication Protocol Certificate Authority (EAP CA)
certificate.       |
 |                  Supported file type is "pem".
|
 +-------------------------------------------------------------------------------------------------
---------------+
 | ifmap_dhcp_cert  - Import a certificate signed by a CA for a certificate signing request
previously given by a    |
 |                    member for the IF-MAP DHCP service. The input file must be in "pem" format.
|
 | member           - Required. The member to which you want to upload the certificate.
|
 +-------------------------------------------------------------------------------------------------
----------------+
 | join_multi_grid    - Joins the Master Grid.
|
 | token              - Required. The token string corresponding to the file to be imported.
|
 | address            - Required. The Multi-Grid Master's address or FQDN.
|
 | port               - Optional. The Multi-Grid Master's port. If it is not specified, 1194 is used.
|
 | grid_name          - Optional. The SubGrid name. The default value is 'Infoblox'.
|
 | use_mgmt_port      - Optional. Use the MGMT port to connect to the grid.
|
 |                      The valid values are 'true' and 'false'. The default value is 'false'.
|
 +-------------------------------------------------------------------------------------------------
----------------+
 | leases           - Import DHCP leases from the local system to the Infoblox appliance. Supported
file type is     |
 |                    "dhcpd.leases". The following parameters are also available:
|
 | format           - Optional. The format of the data file. Only "ISC" is supported. Default is
"ISC".           |
 | lease_precedence - Indicates whether existing leases are overwritten. Valid values are "keep-
previous",        |
 |                    "replace-previous", and "keep-newest". Default is "keep-newest".
|
 | network_view     - Optional. The name of the network view from which the lease data is imported.
|
 |                    The default value is the system-defined default network view.
|
 +-------------------------------------------------------------------------------------------------
----------------+
 | leases6          - Import IPv6 DHCP leases from the local system to the Infoblox appliance.
Supported file type is|
 |                    "dhcpd.leases". The following parameters are also available:
|
 | format           - Optional. The format of the data file. Only "ISC" is supported. Default is
"ISC".           |
 | lease_precedence - Indicates whether existing leases are overwritten. Valid values are "keep-
previous",        |
 |                    "replace-previous", and "keep-newest". Default is "keep-newest".
|
 | network_view     - Optional. The name of the network view from which the lease data is imported.
|
 |                    The default value is the system-defined default network view.
|
 +-------------------------------------------------------------------------------------------------
----------------+
 | lease_history  - Import the DHCP lease history. Supported file type is "gz".
|
 +-------------------------------------------------------------------------------------------------
----------------+
 | tftp_file      - Import file from the local system to a TFTP directory on the Infoblox appliance.
All file types  |
 |                  are supported. The following parameter is also available:
|
 | directory      - Required. The TFTP directory in which the file is stored.
|
 +-------------------------------------------------------------------------------------------------
----------------+
 | discovery_csv  - Import the file from the local system to the Infoblox appliance. Supported file
type is a CSV    |
 |                  file using comma as field sepearator. The following parameter is also available:
|
 | network_view   - Optional. The name of the network view from which the CSV data is imported.
|
 |                  The default value is the system-defined default network view.
|
 | merge_data     - The flag indicates whether the GM sorter replaces or merges the discovered data
```

```
with existing    |
 |                  data. The valid values are "true" and "false". The default value is "true".
|
 +-------------------------------------------------------------------------------------------------------
---------------+
 | upgrade        - NIOS image file that will be used to upgrade the Grid.
|
 +-------------------------------------------------------------------------------------------------------
---------------+
 | uploadkeytab   - Upload keytab file to the server not assigning the keys. The function has no
parameters. Note    |
 |                  that method with such type returns
L<Infoblox::Grid::KerberosKey|Infoblox::Grid::KerberosKey>    |
 |                  objects list if the upload succeeds, and returns false when the operation fails.
|
 +-------------------------------------------------------------------------------------------------------
---------------+
 | trusted_ca_cert  - Import a generic Certificate Authority (CA) certificate.  Supported file type
is "pem".        |
 |                    Allows multiple certificates per single file.
|
 +-------------------------------------------------------------------------------------------------------
---------------+
 | threat_protection_rule_update - Import Threat Protection rules to the Infoblox appliance.
Supported file type is  |
 |                                 "bin2" provided by Infoblox Inc.
|
 +-------------------------------------------------------------------------------------------------------
---------------+
 | dxl_endpoint_brokers - Import DXL endpoint brokers as configuration file and retrieve the list of
|
 |                        Infoblox::DXL::Endpoint::Broker objects.
|
 +-------------------------------------------------------------------------------------------------------
---------------+
```

Returns

The method returns true when the modification succeeds, and returns false when the operation fails.

Example

```
#import backup
$session->import_data(
    type  => "backup",
    force => "false",
    path  => "/tmp/infoblox/database.tar.gz" );

#csv import
$session->import_data(
    type  => "csv",
    continue_on_error => 'true',
    operation => 'update',
    override => 'override',
    path  => "/tmp/infoblox/import_data.csv" );

#import clone
$session->import_data(
    type => "clone",
    path => "/tmp/infoblox/clone_database.tar.gz" );

#import dhcp_expert_mode_config
$session->import_data(
    type   => "dhcp_expert_mode_config",
    member => "192.168.1.2",
    path   => "/tmp/infoblox/dhcpd.conf" );

#import captive_portal_cert
$session->import_data(
    type => "captive_portal_cert",
    path => "/tmp/infoblox/eap_ca_cert.pem",
    member => "192.168.1.2" );

#import captive_portal_file
$session->import_data(
    type => "captive_portal_file",
    path => "/tmp/infoblox/logo_file.jpg",
    member => "192.168.1.2",
    cp_type => "logo" );

#import eap_ca_cert
$session->import_data(
    type => "eap_ca_cert",
    path => "/tmp/infoblox/eap_ca_cert.pem" );
```

```
#import the  IF-MAP DHCP certificate
$session->import_data(
    type => "ifmap_dhcp_cert",
    path => "/tmp/infoblox/ifmap_dhcp_cert.pem" );

#upload keytab
my $kerberos_keys = $session->import_data(
    type => "uploadkeytab",
    path => "/tmp/infoblox/keytab.key");

#import leases
$session->import_data(
    type             => "leases",
    format           => "ISC",
    path             => "/tmp/infoblox/dhcpd.leases",
    lease_precedence => "keep-newest" );

#import leases from a non-default network view
$session->import_data(
    type             => "leases",
    format           => "ISC",
    path             => "/tmp/infoblox/dhcpd.leases",
    lease_precedence => "keep-newest",
    network_view     => "netview" );

#import lease_history
 $session->import_data(
    type => "lease_history",
    path => "/tmp/infoblox/lease_history.gz" );

#import tftp_file
$session->import_data(
    type      => "tftp_file",
    directory => "/",
    path      => "/tmp/infoblox/voip_settings.txt" );

#import discovery CSV file in a given network view
$session->import_data(
    type             => "discovery_csv",
    network_view     => "netview" );

#import DS records
$session->import_data(
    type             => "ds_records",
    path             => "/tmp/infoblox/ds_records.txt",
    zone             => "myzone.test.com",
    view             => "dns_view", );
```

join_grid( )

Join an Infoblox appliance to an existing grid.

Parameter

grid_name - Required. The name of the grid.

master - Required. The virtual IP address of the grid master.

shared_secret - Required. The shared secret string of the grid.

Returns

The method returns true when the modification succeeds, and returns false when the operation fails.

Example

```
my $result = $session->join_grid(
    grid_name     => "infoblox",
    master        => "192.168.1.2",
    shared_secret => "test" );
```

leave_multi_grid( )

Disconnect an Infoblox appliance from the Master Grid.

Returns

The method returns true when the modification succeeds, and returns false when the operation fails.

Example

```
my $result = $session->leave_multi_grid();
```

logout( )

Use this function to log out of the current session.

If you want to log in to the server again, create a new Infoblox::Session object.

Parameter

None

Returns

None

Example

```
$session->logout();
```

member_upgrade( )

Use this function to upgrade a single member that was reverted during the staged upgrade process or to revert a single member if it does not behave properly after an upgrade.

Parameter

The following parameters are supported:

```
member - The FQDN of the member to be upgraded. # Required
action - The action to execute. # Optional
```

The action parameter can be "UPGRADE" or "REVERT". If not specified, the action parameter defaults to "UPGRADE".

Returns

The method returns true when the operation succeeds, and returns false when the operation fails.

Example

```
#upgrade the member
my $response = $session->member_upgrade(member => "hostname.com");
#revert the member
my $response = $session->member_upgrade(member => "hostname.com", action => "REVERT");
```

modify( )

Use this method to modify an object in the Infoblox appliance.

To modify a specific object, use `get()` or `search()` to retrieve the specific object, make the desired modifications to this object, and then submit it for modification.

Parameter

The object to be modified in the Infoblox appliance. See individual objects for the necessary parameters.

Parameter

scheduled_at - Optional. The scheduled date and time in ISO 8601 extended format (e.g. 2008-10-21T10:56:00Z or 2008-10-21T10:56:00.001Z) or 'now' which will set scheduled date and time to the current date and time. See details.

predecessor_task - Optional. An Infoblox::Grid::ScheduledTask object this task depends on. It is mandatory to set scheduled_at parameter if predecessor_task parameter is set.

approval_comment - Optional. Comment provided by the submitter on approval.

approval_ticket_number - Optional. Ticket number provided by the submitter on approval.

ip_validation - Optional. A list of Infoblox::Grid::IPValidationInfo objects.

Returns

The method returns true when the modification succeeds, and returns false when the operation fails. When an operation fails, the error code and error message are reported in the `status_code()` and `status_detail()` attributes.

Example

```
# Use get() to retrieve the object.
my @retrieved_obj = $session->get(
    object => "Infoblox::DNS::Record::A",
    name   => "bind_a.domain.com" );
# select the specific object.
my $bind_a = $retrieved_obj[0];
# Use object method to modify the comment.
$bind_a->comment("this is a modified comment");
# Submit modification
my $response = $session->modify( $bind_a );
```

new_cursor( )

Use this method to return a cursor object. See Infoblox::Cursor for more information.

network_discovery_control( )

Use this method to control the network discovery task

Parameter

action - Required. Valid values are "START", "PAUSE", "RESUME", and "END".

scheduled_at - Optional. Scheduled time in ISO 8601 extended format for date and time (e.g. 2008-10-21T10:56:00Z or 2008-10-21T10:56:00.001Z). Note that this can only be used with the "START" action.

Returns

The method returns true when the operation succeeds, and returns false when the operation fails. When an operation fails, the error code and error message are reported in the `status_code()` and `status_detail()` attributes.

Example

```
# start network discovery
my $response = $session->network_discovery_control(
       action => 'START');

unless($response) {
   die("Start network discovery failed");
}
```

publish_changes( )

Use this method to publish configuration changes.

Parameter

grid_member - Optional. The Grid member name.

services - Required. The service name. The valid value is 'ALL' or 'ATP'.

member_order - Required if Grid_member is not defined. Otherwise this parameter is not used. Option to publish changes on all members with or without delay. The valid value is 'SEQUENTIALLY' or 'SIMULTANEOUSLY'.

sequential_delay - Optional if Grid_member is not defined. Otherwise this parameter is not used. The delay between publishing configuration changes on members in seconds.

Returns

The method returns true when the operation succeeds, and returns false when the operation fails. When an operation fails, the error code and error message are reported in the `status_code()` and `status_detail()` attributes.

Example

```
# publish changes on the Grid
my $res = $session->publish_changes(
   'sequential_delay' => 1,
   'member_order'     => 'SIMULTANEOUSLY',
   'services'         => 'ALL',
);

# publish changes on the Grid member
$res = $session->publish_changes(
   'grid_member' => 'infoblox.localdomain',
   'services'    => 'ALL',
);
```

query_fqdn_on_member( )

Use this method to query FQDN on the specified member.

This methods forces Grid Master to execute RPC call to invoke dig on the member for a specific FQDN.

Parameter

The valid value is an Infoblox::Grid::Member::QueryFQDNParameter object or HASH that contains following values:

```
fqdn            - Required. FQDN to query.
member          - Optional. Member to invoke dig on. The default is Grid Master.
name_server     - Optional. Nameserver to query.
record_type     - Optional. Query resource record type. The default is ANY.
recursive_query - Optional. 'true' to set recursive query flag. The default is 'true'.
```

Returns

The method returns an Infoblox::Grid::Member::QueryFQDNResponse object.

Example

```
#Recursively query any record for the domain, 'foo.com' on the Grid Master.
my $response = $session->query_fqdn_on_member(fqdn => 'foo.com');

#Recursively query nameserver 5.5.5.5 for A record for 'bar.com'.
#domain via 'member1' dig
my $response = $session->query_fqdn_on_member(
    fqdn        => 'bar.com',
    member      => 'member1',
    name_server => '5.5.5.5',
    record_type => 'A',
);

#Same, but using Infoblox::Grid::Member::QueryFQDNParameter

#Construct parameters object
my $query_fqdn = Infoblox::Grid::Member::QueryFQDNParameter->new(
    fqdn        => 'bar.com',
    member      => 'member1',
    name_server => '5.5.5.5',
    record_type => 'A',
);

#Query
my $response = $session->query_fqdn_on_member($query_fqdn);
```

refresh_hsm( )

Use this method to re-sync with the HSM data repository.

Parameter

hsm_group - Required. An HSM group object.

Returns

The method returns one of the following values : "PASSED", "INACTIVE" , "FAILED".

Example

```
# Refresh HSM
my $response = $session->refresh_hsm(
       hsm_group => $hsm_group_object);

unless($response) {
   die("Refresh HSM failed");
}
```

remove( )

Use this method to remove an object from the Infoblox appliance.

To remove a specific object, first use `get()` or `search()` to retrieve the specific object, and then submit this object for removal.

Parameter

The object to be removed from the Infoblox appliance. See individual objects for the necessary parameters.

Parameter

scheduled_at - Optional. Scheduled time in ISO 8601 extended format for date and time (e.g. 2008-10-21T10:56:00Z or 2008-10-21T10:56:00.001Z) or 'now' which will set scheduled date and time to the current date and time. See details.

predecessor_task - Optional. An Infoblox::Grid::ScheduledTask object this task depends on. It is mandatory to set scheduled_at parameter if predecessor_task parameter is set.

approval_comment - Optional. Comment provided by the submitter on approval.

approval_ticket_number - Optional. Ticket number provided by the submitter on approval.

Returns

The method returns true when the modification succeeds, and returns false when the operation fails.

Example

```
# Use get() to retrieve the object.
my @retrieved_obj = $session->get(
    object => "Infoblox::DNS::Record::A",
    name   => "bind_a.domain.com" );
# Select the specific object.
my $bind_a = $retrieved_obj[0];
# Submit for removal
my $response = $session->remove( $bind_a );
```

remove_data( )

Use this method to remove data from the Infoblox appliance or purge DHCP IF-MAP data.

Parameter

type - Required. The type of data to remove. Following is a list of data types that can be removed:

```
 ------------------------------------------------------------------------------------------------------------------------
 | nac_auth_cache           - Purge the NAC authentication cache.
|
 | member                   - Required. The IPv4 address of the member on which the NAC authentication cache is cleared. |
 | mac_addr                 - Optional. The MAC address for which the authentication result in the NAC authentication    |
 |                            cache is to be cleared. if this field is not present, the entire cache will be flushed.    |
 ------------------------------------------------------------------------------------------------------------------------
 | ifmap_dhcp_data          - Sends a purge request that removes any IF-MAP data published by the DHCP IF-MAP client     |
 |                            running on this member. The following parameters are also available:
|
 |
|
 | member                   - Required. The IP address of the grid member.
|
 ------------------------------------------------------------------------------------------------------------------------
```

Returns

The method returns true when the modification succeeds, and returns false when the operation fails.

Example

```
# purge DHCP IF-MAP data published by member 1.2.3.4
$session->remove_data(
    "type"      => "ifmap_dhcp_data",
    "member" => "1.2.3.4");

# purge the entire NAC authentication cache for member 1.2.3.4
$session->remove_data(
    "type"      => "nac_auth_cache",
    "member" => "1.2.3.4");

# Clear authentication data for MAC address a1:a1:a1:a1:a1:a1 from the NAC authentication cache for member 1.2.3.4
$session->remove_data(
    "type"          => "nac_auth_cache",
    "member"    => "1.2.3.4",
    "mac_addr" => "a1:a1:a1:a1:a1:a1");
```

remove_eap_ca_cert( )

Remove an Extensible Authentication Protocol Certificate Authority (EAP CA) certificate.

Parameter

dn - Required. Distinguished Name (DN) of the Certificate Authority (CA) certificate you want to remove.

Returns

The method returns true when the modification succeeds, and returns false when the operation fails.

Example

```
my $result = $session->remove_eap_ca_cert(
    dn => 'CN="My Server Certificate",C="US"' );
```

remove_tftp_file( )

Remove a file from a TFTP directory on the Infoblox appliance.

Parameter

directory - Required. The parent directory of the file in TFTP storage.

name - Required. The name of the file.

Returns

The method returns true when the modification succeeds, and returns false when the operation fails.

Example

```
my $result = $session->remove_tftp_file(
    directory => "/",
    name      => "voip_settings.txt" );
```

reset_discovery_timestamp( )

This method is deprecated and is provided only to maintain backward compatibility with previous releases.

Calling this method will perform the same action as calling the **reset_discovery_data()** method.

reset_discovery_data( )

Use this method to reset the discovery related attributes for an IP address.

The following attributes are reset:

```
os
netbios
first_discovered
last_discovered
network_component_type
network_component_name
network_component_description
network_component_ip
network_component_port_number
network_component_port_name
network_component_port_description
port_vlan_name
port_vlan_number
port_speed
port_duplex
port_status
port_link_status
discovered_name
discoverer
```

Parameter

type - Required. Valid values are "GLOBAL", "NETWORK", and "IP_ADDRESS".

network - Optional. Required when type is "NETWORK" or "IP_ADDRESS", and the ip_address value is an IPv4 address..

network_view - Optional. The network view to which the reset operation will be limited. The default value is the system-defined default network view.

ip_address - Optional. Required when type is "IP_ADDRESS". The value can be an IPv4 or IPv6 address.

Returns

The method returns true when the operation succeeds, and returns false when the operation fails. When an operation fails, the

error code and error message are reported in the `status_code()` and `status_detail()` attributes.

Example

```
# type is GLOBAL
my $response = $session->reset_discovery_data(
       type => 'GLOBAL');

unless($response) {
   die("reset discovery datafailed");
}

# with a non-default network view
my $nview = Infoblox::DHCP::View->new(
       name => 'netview',
);
my $response = $session->reset_discovery_data(
       type => 'GLOBAL',
       network_view => $nview);

unless($response) {
   die("reset discovery data failed");
}

# type is NETWORK
$response = $session->reset_discovery_data(
       type => 'NETWORK',
       network => '10.0.0.0/24');

unless($response) {
   die("reset discovery data failed");
}

# type is IP_ADDRESS
$response = $session->reset_discovery_data(
       type => 'IP_ADDRESS',
       network => '10.0.0.0/24',
       ip_address => '10.0.0.10');

unless($response) {
   die("reset discovery data failed");
}
```

reset_threat_protection_object( )

Use this method to reset the Threat Protection object to the factory defaults.

Key Reference

```
 object              - Required. An Infoblox::Grid::ThreatProtection::Rule,
Infoblox::Grid::ThreatProtection::Ruleset or Infoblox::Grid::ThreatProtection::RuleCategory.
 delete_custom_rules - Optional. Flag that indicates whether the custom rules are removed or not. The
default value is 'true'.
```

Returns

The method returns true when the operation succeeds, and returns false when the operation fails. When an operation fails, the error code and error message are reported in the `status_code()` and `status_detail()` attributes.

Example

```
my $res = $session->reset_threat_protection_object(
    object => $rule,
    delete_custom_rules => 'false'
);
```

restart( )

Restart a service on the Infoblox appliance.

Parameter

mode - Optional. The restart service mode. Valid values are 'GROUPED', 'SEQUENTIAL', 'SIMULTANEOUS'. Default is 'SIMULTANEOUS'.

force_restart - Optional. Valid values are "true" or "false".

members - Optional. The restart service members. Valid value is an array of Grid Member objects. Note that objects should be

retrieved from the server first. Only one parameter for members and groups can be used in the same call.

groups - Optional. The restart service groups. Valid value is an array of Grid Restart Service Group objects. Note that objects should be retrieved from the server first. Only one parameter for members and groups can be used in the same call.

services - Optional. Valid value is an array, which can contain the following values: 'ALL', 'DHCP', 'DHCPV4', 'DHCPV6' and 'DNS'. Default is ['ALL'].

scheduled_at - Optional. Scheduled time in ISO 8601 extended format for date and time (e.g. 2008-10-21T10:56:00Z or 2008-10-21T10:56:00.001Z).

user_name - Optional. Name of the user requesting the restart.

Returns

The method returns true when the modification succeeds, and returns false when the operation fails. If no attribute is specified, the method returns the pending schedule.

Example

```
my $result = $session->restart(
   groups        => [$group1, $group2],
   mode          => 'GROUPED',
   force_restart => 'true',
   services      => ['DNS', 'DHCPV6'],
   user_name     => 'ib-admin',
);
```

restart_status( )

Use this method to retrieve all the services that need to restart.

Parameter

None

Returns

The method returns an array of strings for all the services that need to restart ('dhcp', 'dns').

Example

```
my @restart_list = $session->restart_status();
```

rollback_db_snapshot( )

Use this method to rollback OneDB snapshot if one exists.

Key Reference

None

Returns

The method returns true when the operation succeeds, and returns false when the operation fails. When an operation fails, the error code and error message are reported in the `status_code()` and `status_detail()` attributes.

Example

```
my $res = $session->rollback_db_snapshot();
```

rotate_log_files( )

Use this method to rotate the specified log file for the Grid Master or specified member.

Parameter

log_type - Required. The type of log file. Valid values are 'audit', 'msmgmt', or 'sys'.

member - Optional. Hostname of the Grid member on which log rotation is performed. Only required if 'log_type' is 'sys' or 'msmgmt'.

node_type - Optional. HA node type ('ACTIVE' or 'PASSIVE'). Only required if 'log_type' is 'sys' or 'msmgmt'.

server - Optional. IP address or FQDN of the Microsoft server. Only required if 'log_type' is 'msmgmt'.

Returns

The method returns '1' when there are no errors and sets session error code and text when there is an error.

Example

```
# rotate the syslog
my %args;
$args{'log_type'} = "sys";
$args{'node_type'} = "ACTIVE";
$args{'member'} = "infoblox.localdomain";

$session->rotate_log_files(%args);
```

run_scavenging( )

Use this method to perform the scavenging of the DNS Records.

Key Reference

object - Required. The DNS object on which scavenging is performed. The valid values are Infoblox::Grid::DNS, Infoblox::DNS::Zone and Infoblox::DNS::View. action - Optional. The scavenging action to perform. The valid values are 'ANALYZE', 'RECLAIM', 'ANALYZE_RECLAIM' and 'RESET'. The default value is 'ANALYZE'. scheduled_at - Optional. Scheduled date and time in ISO 8601 extended format (e.g., 2008-10-21T10:56:00Z or 2008-10-21T10:56:00.001Z) or 'now' which will set scheduled date and time to the current date and time. See details.

Returns

The method returns 1 when the operation succeeds and sets session error code an text when the operation fails.

Example

```
my $res = $session->run_scavenging(
    object => $grid_dns,
    action => 'RESET',
);
```

save_db_snapshot( )

Use this method to take OneDB snapshot.

Key Reference

`comment - Required. A descriptive comment in string format.`

Returns

The method returns true when the operation succeeds, and returns false when the operation fails. When an operation fails, the error code and error message are reported in the `status_code()` and `status_detail()` attributes.

Example

```
my $res = $session->save_db_snapshot(comment => 'save snapshot demo');
```

search( )

Use this method to search and retrieve all the matching objects from the Infoblox appliance.

Parameter

object - Required. The Infoblox object to retrieve.

return_methods - Optional. The member fields to return.

maximum_objects_returned - Optional. The maximum number of objects to retrieve (this overrides default_maximum_objects_returned if specified, but will be overridden by an individual object's limit if it is lower).

partial_subobjects - Optional. Indicates whether the partial subobjects feature is enabled (this overrides default_partial_subobjects for this call).

key reference - Required. The attribute and value that refer to the desired `object(s)` to retrieve. Many key references are given as regular expressions. See individual objects for available key references.

Returns

Return an array in list context containing all matching objects. For failures or objects not found, it returns an empty array in list context and undefined in scalar context. When an operation fails, the error code and error message are reported in the `status_code()` and `status_detail()` attributes.

return_methods can be specified in the following way:

```
 +method_name - This adds the specified method to the returned methods (useful for "optional"
methods).
 -method_name - This returns all methods, except the specified method.
  method_name - This returns only the specified method.
```

The return_methods above can be specified together. Note that specifying -method_name in a list that contains method_name entries will be ignored (because only method_name entries will be returned). Specifying +method_name for members that are not optional will similarly be ignored. Following are usage examples:

```
 ['method1','method2']  = Returns only method1 and method2.
 ['-method1']           = Returns all object methods, except method1.
 ['+method1']           = Returns all object methods and method1, if method1 is not a method that is
returned by default.
```

Note that objects returned when method_name and/or -method_name are specified cannot be used directly for modify( ) calls; fill_partial_object must be called for them first.

Member methods that represent actions/operations rather than values (e.g., Infoblox::DHCP::NetworkContainer->resize()) cannot be specified as return_methods.

When the partial subobjects feature is enabled, objects returned from searches do not retrieve all of their subobjects (if any), but only a few of their fields. These subobjects are supported:

```
 Infoblox::DNS::View  - only 'name' and 'comment' will be returned
 Infoblox::DHCP::View - only 'name' and 'comment' will be returned
```

Therefore, if an object that is returned by 'search' has one of the supported subobjects as a method value, then the subobject is treated as if it was retrieved with 'return_methods' set and can be completed by calling fill_partial_object.

Example

```
 #Searching for all A records under "domain.com" from the Infoblox appliance.
 my @retrieved_objs = $session->search(
     object => "Infoblox::DNS::Record::A",
     name   => '.*\.domain\.com' );

 unless (@retrieved_objs) {
     die("Search DNS record A failed: ",
        $session->status_code() . ":" . $session->status_detail());
 }

 #Searching for all A records under "domain.com" from the Infoblox appliance and returning only their
addresses and TTL's.
 my @retrieved_objs = $session->search(
     object => "Infoblox::DNS::Record::A",
     name   => '.*\.domain\.com',
     return_methods => ['ttl','ipv4addr']
 );

 unless (@retrieved_objs) {
     die("Search DNS record A failed: ",
        $session->status_code() . ":" . $session->status_detail());
 }

 #Searching for all A records under "domain.com" from the Infoblox appliance and returning everything
but their TTL's.
 my @retrieved_objs = $session->search(
     object => "Infoblox::DNS::Record::A",
     name   => '.*\.domain\.com',
     return_methods => ['-ttl']
 );

 unless (@retrieved_objs) {
     die("Search DNS record A failed: ",
        $session->status_code() . ":" . $session->status_detail());
 }

 #Searching for all A records under "domain.com" from the Infoblox appliance and returning their
views member as a partial object.
 my @retrieved_objs = $session->search(
     object => "Infoblox::DNS::Record::A",
     name   => '.*\.domain\.com',
     partial_subobjects => 1
 );

 unless (@retrieved_objs) {
     die("Search DNS record A failed: ",
        $session->status_code() . ":" . $session->status_detail());
```

```
}
```

search with limit parameters

Some object methods support searches where limits are specified. Assuming that 'X' and 'Y' are valid searchable values for a particular method, the following are supported:

'X' - Matches, as usual, if the method value is EXACTLY X.
'> X' or '< X' - Matches if the method value is GREATER or LESS than X respectively.
'>= X' or '<= X' - Matches if the method value is GREATER THAN OR EQUALS or is LESS THAN OR EQUALS X respectively.
'!= X' or '/= X' - Matches if the method value DOES NOT EQUAL X.
'>=< X,Y' - Matches if the method value is GREATER THAN OR EQUALS X AND is LESS THAN OR EQUALS Y.

Following are examples of limit searches:

```
# This searches for tasks scheduled exactly on 2011-09-18T10:20:20Z
@result = $session->search(
                           object         => "Infoblox::Grid::ScheduledTask",
                           scheduled_time => '2011-09-18T10:20:20Z',
                          );

# This searches for tasks scheduled before 2011-09-18T10:20:20Z
@result = $session->search(
                           object         => "Infoblox::Grid::ScheduledTask",
                           scheduled_time => '< 2011-09-18T10:20:20Z',
                          );

# This searches for tasks scheduled before or on 2011-09-18T10:20:20Z
@result = $session->search(
                           object         => "Infoblox::Grid::ScheduledTask",
                           scheduled_time => '<= 2011-09-18T10:20:20Z',
                          );

# This searches for tasks scheduled not on 2011-09-18T11:20:20Z
@result = $session->search(
                           object         => "Infoblox::Grid::ScheduledTask",
                           scheduled_time => '!= 2011-09-18T11:20:20Z',
                          );

# This searches for tasks scheduled between 2011-09-18T10:20:20Z and 2011-10-18T10:20:20Z, inclusive
@result = $session->search(
                           object         => "Infoblox::Grid::ScheduledTask",
                           scheduled_time => '>=< 2011-09-18T10:20:20Z,2011-10-18T10:20:20Z',
                          );
```

server_version( )

Check the current server version.

Parameter

none

Returns

The method returns the current Infoblox appliance version.

Example

```
my $version = $session->server_version( );
```

set_dhcp_failover_partner_down( )

Set a DHCP failover association peer to the Partner Down state.

Parameter

dhcp_failover - Required. The name of the DHCP failover association with the peer that you are setting to the Partner Down state.

Parameter

peer_type - Required. The peer that you are setting to the Partner Down state. Valid values are 'primary' or 'secondary'.

Returns

The method returns true when the operation succeeds, and returns false when the operation fails.

Example

```
my $result = $session->set_dhcp_failover_partner_down(
   dhcp_failover => 'test_failover',
   peer_type     => 'primary'
);
```

set_dhcp_failover_secondary_recover( )

Set the secondary peer to the Recover state.

Parameter

dhcp_failover - Required. The name of the DHCP failover association with the secondary peer that you are setting to the Recover state.

Returns

The method returns true when the operation succeeds, and returns false when the operation fails.

Example

```
my $result = $session->set_dhcp_failover_secondary_recover(
   dhcp_failover => 'test_failover'
);
```

skip_member_upgrade( )

This function allows the specified member to skip the upgrade process.

Parameter

The following required parameter is supported:

```
member - The FQDN of the member that will skip the upgrade process.
```

Returns

The method returns true when the operation succeeds, and returns false when the operation fails.

Example

```
#skip member
my $response = $session->skip_member_upgrade(member => "hostname.com");
```

status_code( )

Get the status code of the last function called.

Parameter

none

Returns

The method returns the status code of the last function/operation called.

Example

```
my $status_code = $session->status_code( );
```

status_detail( )

Get the status detail of the last function called.

Parameter

none

Returns

The method returns the status detail of the last function/operation called.

Example

```
my $status_detail = $session->status_detail( );
```

test( )

A generic test function.

Parameter

object - Required. An Infoblox object. timeout - Optional, if object type is 'Infoblox::LDAP::Server'. The timeout in seconds. ldap_service - Optional, if object type is 'Infoblox::LDAP::Server'. An Infoblox::LDAP::AuthService object that has been retrieved from the appliance. certificate_auth_service - Optional, if object type is 'Infoblox::Grid::Admin::CertificateAuthService'. An Infoblox::Grid::Admin::CertificateAuthService or it's name in a string format. member - Optional, if object type is 'Infoblox::Grid::SyslogBackupServer'. The name of the Grid member which performs the connection test. By default, the connection test is performed from the Grid Master. event_text - Required, if object type is 'Infoblox::Notification::Rule'. An event to trigger notification rule in a JSON text string.

Object type	Description
Infoblox::OCSP::Responder	- Performs validation of OCSP responder object. The valid return value is one of the following: "CANNOT_RESOLVE_FQDN", "CANNOT_CONNECT", "FAILED_TEST", "TEST_OK", or "TEST_OK_RESPONSE_NOT_VERIFIED".
Infoblox::LDAP::Server	- Tests connectivity to the LDAP server.
Infoblox::CiscoISE::Endpoint	- Tests connectivity to the CiscoISE endpoint. The valid return values are 'CANNOT_RESOLVE_FQDN', 'FAIL', 'OK' and 'WARN'.
Infoblox::Notification::REST::Endpoint	- Tests connectivity to the REST API endpoint. The valid return value is 0 if connectivity tests fail to occur or Infoblox::Grid::TestResult otherwise.
Infoblox::Notification::Rule	- Tests applicability of event to be triggered.
Infoblox::DXL::Endpoint::Broker	- Tests connectivity to the DXL broker. Parameters: endpoint - Optional. The Infoblox::DXL::Endpoint object. client_certificate - Optional. The DXL broker client certificate. Note that at least one of the arguments ('endpoint' and 'client_certificate') should be set in order to perform the connectivity test. The valid return value is 0 if the connectivity tests fail to occur or Infoblox::Grid::TestResult otherwise.
Infoblox::Grid::SyslogServer	- Tests connectivity to the Syslog server.
Infoblox::Grid::SyslogBackupServer	- Tests connectivity to the Syslog backup server.

Returns

The method returns a test result.

Example

```
#Performs validation of OCSP responder object
my $ocsp_responder = Infoblox::OCSP::Responder->new(
                     'address'     => 'domain.com',
                     'certificate' => 'responder.pem',
);

my $result = $session->test('object' => $ocsp_responder);

my $syslog_server = Infoblox::Grid::SyslogServer->new(
   address         => '10.32.1.55',
   connection_type => 'tcp',
   port            => 514,
);

my $result = $session->test('object' => $syslog_server);

my $backup_syslog_server = Infoblox::Grid::BackupSyslogServer->new(
   address => '10.32.1.55',
);

my $result = $session->test('object' => $backup_syslog_server);
```

```
my $notif_rule = Infoblox::Notification::Rule->new(
                            event_type          => 'DNS_RPZ',
                            expression_list     => [@rule_expr1],
                            name                => 'notif_rule1',
                            notification_action => 'RESTAPI_TEMPLATE_INSTANCE',
                            notification_target => $endpoint,
                            template_instance   => $instances[0],
                            disable             => 'false',
   );
my $event_text = '{"destination_ip": "10.0.0.0","timestamp": "1973-03-03T09:46:41Z",'.
                 '"vnode_oid": 0, "thread_id": 0, "sequence_id": 0, "member_name":
"infoblox.localdomain",'.
                 '"rpz_policy": "PASSTHRU", "member_ip": "192.168.1.2", "source_ip": "10.0.0.55",'
.
                 '"network.network_view": "test_view", "rpz_severity": "WARNING"}';

my $result = $session->test('object' => $notif_rule, event_text => $event_text);
```

test_hsm_status( )

Performs a signing test using an active HSM group.

Parameter

hsm_group - Required. An HSM group object.

Returns

The method returns the status detail of signing test. The valid return value is one of the following : "PASSED", "INACTIVE","KEY_GEN", "SIGNING" .

Example

```
my $response = $session->test_hsm_status(
                          hsm_group => $hsm_group_object);

unless($response) {
   die("Test HSM status failed");
```

upgrade_group_now( )

This function allows the specified group to immediately upgrade.

Parameter

The following required parameter is supported:

group_name - The name of the group to upgrade.

Returns

The method returns true when the operation succeeds, and returns false when the operation fails.

Example

```
#Upgrade group now
my $response = $session->upgrade_group_now(group_name => "group1");
```

SAMPLE CODE

#Create an Infoblox::Session object

```
#PROGRAM STARTS: Include all the modules that will be used
use strict;
use Infoblox;

#Create a session to the Infoblox appliance
my $session = Infoblox::Session->new(
    master   => "192.168.1.2",
    username => "admin",
    password => "infoblox"
);
if ($session->status_code()) {
   die("Construct session failed: ",
       $session->status_code() . ":" . $session->status_detail());
```

```
}
print "Session created successfully\n";
```

#Add a DNS zone object

```
#Construct the zone
my $zone = Infoblox::DNS::Zone->new(name => "domain.com");
unless ($zone) {
   die("Construct zone failed: ",
       $session->status_code() . ":" . $session->status_detail());
}
print "Zone object created successfully\n";

#Add the zone to the Infoblox appliance
$session->add($zone)
    or die("Add zone failed: ",
        $session->status_code() . ":" . $session->status_detail());
print "Zone added successfully\n";
```

#Restart service

```
$session->restart(
     "member" => "192.168.1.2",
     "when"   => "03/20/2007 01:00:00",
     "time_zone" => "(UMT - 6:00) Central Time (US and Canada)"
);

#To cancel the previous restart schedule.

 $session->restart("cancel" => "true");
```

#Get restart services

```
my @services = $session->restart_status();
```

#Get zone object

```
my @retrieved_objs = $session->get(
    object    => "Infoblox::DNS::Zone",
    name      => "domain.com" );
my $object = $retrieved_objs[0];
unless ($object) {
    die("Get Zone object failed: ",
        $session->status_code() . ":" . $session->status_detail());
}
print "Get DNS zone object found at least 1 matching entry\n";
```

#Remove zone object

```
$session->remove( $object )
  or die("Remove DNS zone failed: ",
         $session->status_code() . ":" . $session->status_detail());
print "DNS Zone object removed successfully \n";
```

#Show the current version

```
my $version = $session->server_version();
print "Current server version is $version\n";
```

#Generate a TSIG key

```
my $tsig = $session->gen_tsig_key();
print "Generated a TSIG key: $tsig\n";
```

#Show status_code and status_detail

```
my $status_code = $session->status_code();
my $status_detail = $session->status_detail();
print "Previous operation status_code is: $status_code\n";
print "Previous operation status_detail is: $status_detail\n";
```

#clear unmanaged address

```
# Creating new network and checking
my $network = Infoblox::DHCP::Network->new(network => "10.0.0.0/24");
unless($network) {
   die("Create network object failed.");
```

```
}

# Adding into Session and testing
$session->add($network)
   or die("Add network failed.");

# Creating a new network view
my $network_view = Infoblox::DHCP::View->new(name => "netview");
unless($network_view) {
   die("Create network view object failed.");
}
$session->add($network_view)
   or die("Add network view failed.");

# Creating a second network in the network view
my $network2 = Infoblox::DHCP::Network->new(
                  network => "10.0.0.0/24",
                  network_view => $network_view);
unless($network2) {
   die("Create second network object failed.");
}
$session->add($network2)
   or die("Add second network object failed.");

# type is GLOBAL
my $response = $session->clear_unmanaged(
   type => 'GLOBAL');

unless($response) {
   die("Clear unmanaged failed");
}

my $response = $session->clear_unmanaged(
   type => 'GLOBAL',
   network_view => $network_view);

unless($response) {
   die("Clear unmanaged failed");
}
# type is NETWORK
$response = $session->clear_unmanaged(
   type => 'NETWORK',
   network => '10.0.0.0/24');

unless($response) {
   die("Clear unmanaged failed");
}

$response = $session->clear_unmanaged(
   type => 'NETWORK',
   network => '10.0.0.0/24',
   network_view => $network_view);

unless($response) {
   die("Clear unmanaged failed");
}

# type is IP_ADDRESS
$response = $session->clear_unmanaged(
   type => 'IP_ADDRESS',
   network => '10.0.0.0/24',
   ip_address => '10.0.0.10');

unless($response) {
   die("Clear unmanaged failed");
}

$response = $session->clear_unmanaged(
   type => 'IP_ADDRESS',
   network => '10.0.0.0/24',
   network_view => $network_view,
   ip_address => '10.0.0.10');

unless($response) {
   die("Clear unmanaged failed");
}
```

#reset discovery data

```
# type is GLOBAL
my $response = $session->reset_discovery_data(
   type => 'GLOBAL');

unless($response) {
   die("reset discovery data failed");
}

my $response = $session->reset_discovery_data(
   type => 'GLOBAL',
   network_view => $network_view);

unless($response) {
   die("reset discovery data failed");
}

# type is NETWORK
$response = $session->reset_discovery_data(
   type => 'NETWORK',
   network => '10.0.0.0/24');

unless($response) {
   die("reset discovery data failed");
}

$response = $session->reset_discovery_data(
   type => 'NETWORK',
   network => '10.0.0.0/24',
   network_view => $network_view);

unless($response) {
   die("reset discovery data failed");
}

# type is IP_ADDRESS
$response = $session->reset_discovery_data(
   type => 'IP_ADDRESS',
   network => '10.0.0.0/24',
   ip_address => '10.0.0.10');

unless($response) {
   die("reset discovery data failed");
}

$response = $session->reset_discovery_data(
   type => 'IP_ADDRESS',
   network => '10.0.0.0/24',
   network_view => $network_view,
   ip_address => '10.0.0.10');

unless($response) {
   die("reset discovery data failed");
}
```

#Control the network discovery task

```
# stop network discovery
my $response = $session->network_discovery_control(
       action => 'END',
       network => '10.0.0.0/24');

unless($response) {
   die("Stop network discovery failed");
}
```

#Clean up

```
#Remove the network
$network = $session->get(
             object => 'Infoblox::DHCP::Network',
             network => '10.0.0.0/24',
             network_view => 'default');
unless($network) {
   die("network '10.0.0.0/24' couldn't be retrieved");
}
```

```
$session->remove($network)
   or die("Remove DHCP network failed: ",
               $session->status_code() . ":" . $session->status_detail());
print "DHCP network object removed successfully \n";

#Remove the network view
# (network2 will be removed with the view)
$network_view = $session->get(
             object => 'Infoblox::DHCP::View',
             name => 'netview');
unless($network_view) {
   die("network view 'netview' couldn't be retrived");
}
$session->remove($network_view)
   or die("Remove DHCP network view failed: ",
               $session->status_code() . ":" . $session->status_detail());
print "DHCP network view object removed successfully \n";
```

SCHEDULED OBJECT USAGE DETAILS

The following are the object types for which you can schedule tasks:

```
Infoblox::DHCP::FixedAddr
Infoblox::DHCP::NetworkContainer
Infoblox::DHCP::IPv6NetworkContainer
Infoblox::DHCP::Network
Infoblox::DHCP::IPv6Network
Infoblox::DHCP::SharedNetwork
Infoblox::DHCP::Range
Infoblox::DHCP::RoamingHost
Infoblox::DNS::BulkHost
Infoblox::DNS::Host
Infoblox::DNS::Record::A
Infoblox::DNS::Record::AAAA
Infoblox::DNS::Record::Alias
Infoblox::DNS::Record::CNAME
Infoblox::DNS::Record::DNAME
Infoblox::DNS::Record::MX
Infoblox::DNS::Record::PTR
Infoblox::DNS::Record::SRV
Infoblox::DNS::Record::TLSA
Infoblox::DNS::Record::TXT
Infoblox::DNS::SharedRecord::A
Infoblox::DNS::SharedRecord::AAAA
Infoblox::DNS::SharedRecord::MX
Infoblox::DNS::SharedRecord::SRV
Infoblox::DNS::SharedRecord::TXT
Infoblox::DNS::Zone
Infoblox::DNS::SRG
Infoblox::DNS::View
Infoblox::DTC::LBDN
Infoblox::DTC::Pool
Infoblox::DTC::Server
Infoblox::DTC::Topology
Infoblox::DTC::Monitor::HTTP
Infoblox::DTC::Monitor::SIP
Infoblox::DTC::Monitor::ICMP
Infoblox::DTC::Monitor::PDP
Infoblox::DTC::Monitor::TCP
```

The following are the functions for which you can schedule tasks:

```
restart
network_discovery_control
```

To allow limited-access admins to use the scheudling feature, add the permission with resource_type = "Scheduling" to the appropriate admin group (see Infoblox::Grid::Admin::Permission).

AUTHOR

Infoblox Inc. http://www.infoblox.com/

SEE ALSO

Infoblox::DNS::Record::A, Infoblox::DNS::Zone, Infoblox::DNS::View, Infoblox::DHCP::FailOver, Infoblox::DHCP::Network, Infoblox::DHCP::NetworkContainer, Infoblox::DHCP::View, Infoblox::Grid::CSVImportStatus, Infoblox::Grid::Member::QueryFQDNParameter, Infoblox::Grid::Member::QueryFQDNResponse, Infoblox::Grid::GlobalSmartFolder, Infoblox::Grid::PersonalSmartFolder, Infoblox::Grid::ScheduledTask

---

COPYRIGHT

Infoblox::Session - Session object to manipulate data within a grid.

Copyright (c) 2017 Infoblox Inc.

Infoblox::Grid::ExtensibleAttributeDef - Extensible Attribute Definition object.

- NAME
- DESCRIPTION
- Searching Extensible Attributes
  - String Searching
  - Integer Searching
  - Date Searching
- Extensible Attribute Values
- CONSTRUCTOR
- SESSION METHODS
  - Infoblox::Session->add( )
  - Infoblox::Session->get( )
  - Infoblox::Session->modify( )
  - Infoblox::Session->remove( )
  - Infoblox::Session->search( )
- METHODS
  - allowed_object_types( )
  - audit_log( )
  - comment( )
  - default_value( )
  - descendants_action( )
  - inheritable( )
  - list_values( )
  - max( )
  - min( )
  - multiple( )
  - cloud_api( )
  - grid_master( )
  - mgm_private( )
  - name( )
  - namespace( )
  - type( )
  - user_input( )
- SAMPLE CODE
- AUTHOR
- SEE ALSO
- COPYRIGHT

NAME

Infoblox::Grid::ExtensibleAttributeDef - Extensible Attribute Definition object.

DESCRIPTION

The Extensible Attribute Definition object is used to create or modify the definition of an extensible attribute. After the definition is created, the attribute can be defined on other Infoblox objects.

Searching Extensible Attributes

When searching for integer, string and date extensible attributes, the following constructs are supported, in addition to specifying the actual desired value.

String Searching

The string search can be modified by specifying the string in the following format:

```
 stringattr => { value => $wantedstring, ... options ... }
```

the following options are supported:

```
 case_insensitive => "true" | "false" # Optional. If "true", the string search will be case insensitive.
The default is "false".
```

Following is an example of a string search:

```
 @result = $session->search(object => 'Infoblox::DNS::Record::A',
                            extensible_attributes => { stringattr => { value => "some string",
case_insensitive => "true"});
```

Integer Searching

Integer searches support the following where X and Y are integers:

'> X' or '< X' - Matches if the extensible attribute value is GREATER or LESS than X respectively.
'>= X' or '<= X' - Matches if the extensible attribute value is GREATER THAN OR EQUALS or is LESS THAN OR EQUALS X respectively.
'!= X' or '/= X' - Matches if the extensible attribute value DOES NOT EQUAL X.
'>=< X,Y' - Matches if the extensible attribute value is GREATER THAN OR EQUALS X AND is LESS THAN OR EQUALS Y.

Following are examples of integer searches:

```
@result = $session->search(object => 'Infoblox::DNS::Record::A',
                           extensible_attributes => { integerattr => '<= 5'});

@result = $session->search(object => 'Infoblox::DNS::Record::A',
                           extensible_attributes => { integerattr => '!= 5'});

@result = $session->search(object => 'Infoblox::DNS::Record::A',
                           extensible_attributes => { integerattr => '>=< 5,10'});
```

Date Searching

Valid values for dates when searching are:

The number of seconds that have elapsed since January 1st, 1970 UTC.
A string matching 'YYYY-MM-DD'.
'today', 'yesterday', 'tomorrow'.
'this week', 'this month'.

Date searches support the following where X and Y are dates:

'> X' or '< X' - Matches if the extensible attribute value is GREATER or LESS than X respectively.
'>= X' or '<= X' - Matches if the extensible attribute value is GREATER THAN OR EQUALS or is LESS THAN OR EQUALS X respectively.
'>=< X,Y' - Matches if the extensible attribute value is GREATER THAN OR EQUALS X AND is LESS THAN OR EQUALS Y. This is supported only for epoch or YYYY-MM-DD strings.
'- n' or '+ n' where n is an integer - Matches if the extensible attribute value is within the past or next n days respectively, excluding today.

Following are examples of date searches:

```
@result = $session->search(object => 'Infoblox::DNS::Record::A',
                           extensible_attributes => { dateattr => '<= yesterday'});

@result = $session->search(object => 'Infoblox::DNS::Record::A',
                           extensible_attributes => { dateattr => 'this week'});

@result = $session->search(object => 'Infoblox::DNS::Record::A',
                           extensible_attributes => { dateattr => '>=< 2010-10-20,2010-11-01'});

@result = $session->search(object => 'Infoblox::DNS::Record::A',
                           extensible_attributes => { dateattr => '- 3'});
```

Extensible Attribute Values

The valid value for an extensible attribute is a hash reference containing the names of extensible attributes and their associated values.

The names must be those of existing Infoblox::Grid::ExtensibleAttributeDef objects in the database.

For email, URL, and string types, the value is a string with a maximum of 256 characters.

For a list, the value is a string with a maximum of 64 characters.

For an integer, the value is an integer from -2147483648 through 2147483647.

For a date, the value is the number of seconds that have elapsed since January 1st, 1970 UTC or a string of the format YYYY-MM-DD. Any date entered will be converted to an epoch time corresponding to that day at 11:01am UTC.

The default value for the member is undefined.

CONSTRUCTOR

```
my $ext_attr_def = Infoblox::Grid::ExtensibleAttributeDef->new(
```

```
    name                => $string,                                                    #Required
    type                => "string" | "integer" | "email" | "date" | "list" | "url",   #Optional
/ Default is "string"
    comment             => $string,                                                    #Optional
/ Default is undefined
    audit_log           => "true" | "false",                                           #Optional
/ Default is "false"
    inheritable         => "true" | "false",                                           #Optional
/ Default is "false"
    multiple            => "true" | "false",                                           #Optional
/ Default is "false"
    cloud_api           => "true" | "false",                                           #Optional
/ Default is "false"
    mgm_private         => "true" | "false",                                           #Optional
/ Default is "false"
    grid_master         => "true" | "false",                                           #Optional
/ Default is "false"
    default_value       => $string | $num,                                             #Optional
/ Default is undefined
    list_values         => [$Value1, $Value2, ...],                                    #Optional
/ Default is undefined (Mandatory if Extensible Attributes of type='list' is defined; optional otherwise)
    min                 => $num,                                                       #Optional
/ Default is undefined
    max                 => $num,                                                       #Optional
/ Default is undefined
    allowed_object_types=> [$string, $string, ...]                                     #Optional
/ Default is undefined
    user_input          => 'recommended' | 'required' | 'optional',                    #Optional
/ Default is 'optional'
    descendants_action  => $descendants,                                               #Optional
/ Default is undefined
 );
```

SESSION METHODS

This section describes all the methods in the Infoblox::Session module that you can apply to an Extensible Attribute Definition object.

Infoblox::Session->add( )

Use this method to add an object to the Infoblox appliance. See Infoblox::Session->add() for parameters and return values.

Example

```
# Construct an Extensible Attribute Definition object
my $ext_attr_def = Infoblox::Grid::ExtensibleAttributeDef->new(
    name                => "test_ext_attr",
    comment             => "test extensible attribute",
    type                => "integer"
);
# Submit for addition
my $response = $session->add( $ext_attr_def );
```

Infoblox::Session->get( )

Use this method to retrieve all the matching objects from the Infoblox appliance. See Infoblox::Session->get() for parameters and return values.

Key References

```
Apply the following attributes to get a specific Extensible Attribute Definition object:

 name      - Required. Extensible Attribute Definition name in string format.
 namespace - Optional. Extensible Attribute Definition namespace.
```

Example

```
my @retrieved_objs = $session->get(
    object     => "Infoblox::Grid::ExtensibleAttributeDef",
    name       => "test_ext_attr" );
```

Infoblox::Session->modify( )

Use this method to modify an object in the Infoblox appliance. See Infoblox::Session->modify() for parameters and return values.

Example

```
# Use this method to modify the comment.
$ext_attr_def->comment("This is a modified comment");
# Submit modification
```

```
my $response = $session->modify( $ext_attr_def );
```

Infoblox::Session->remove( )

Use this method to remove an object from the Infoblox appliance. See Infoblox::Session->remove() for parameters and return values.

To remove a specific object, use `get()` or `search()` to retrieve the specific object first, and then submit this object for removal.

Example

```
# Get the Extensible Attribute Definition object matching the given name
my @retrieved_objs = $session->get(
    object     => "Infoblox::Grid::ExtensibleAttributeDef",
    name       => "test_ext_attr" );
# Find the desired object from the retrieved list.
my $ext_attr_def = $retrieved_objs[0];
# Submit for removal
my $response = $session->remove( $ext_attr_def );
```

Infoblox::Session->search( )

Use this method to search for Extensible Attribute Definition objects in the Infoblox appliance. See Infoblox::Session->search() for parameters and return values.

Key References

```
Apply the following attributes to search for a specific Extensible Attribute Definition object:

 name      - Optional. Extensible Attribute Definition name (regular expression).
 comment   - Optional. Extensible Attribute Definition comment (regular expression).
 namespace - Optional. Extensible Attribute Definition namespace.
```

For more information about searching extensible attributes, see Infoblox::Grid::ExtensibleAttributeDef/Searching Extensible Attributes.

Example

```
# search for all Extensible Attribute Definition objects starting with "test"
my @retrieved_objs = $session->search(
    object => "Infoblox::Grid::ExtensibleAttributeDef",
    name   => "test.*" );
```

METHODS

This section describes all the methods that you can use to set and retrieve the attribute values of an Extensible Attribute Definition object.

allowed_object_types( )

Use this method to set or retrieve the object types on which the extensible attribute can be defined.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

Parameter is an array reference that contains the list of object types on which the extensible attribute can be defined. The following object types are supported here:

```
Infoblox::Grid::Member
Infoblox::Grid::Admin::User
Infoblox::Grid::Admin::Role
Infoblox::DHCP::FailOver
Infoblox::DHCP::Filter::Fingerprint
Infoblox::DHCP::Filter::MAC
Infoblox::DHCP::Filter::NAC
Infoblox::DHCP::Filter::Option
Infoblox::DHCP::Filter::RelayAgent
Infoblox::DHCP::Fingerprint
Infoblox::DHCP::FixedAddr
Infoblox::DHCP::FixedAddrTemplate
Infoblox::DHCP::IPv6FixedAddr
Infoblox::DHCP::IPv6FixedAddrTemplate
Infoblox::DHCP::IPv6Network
Infoblox::DHCP::IPv6Network
Infoblox::DHCP::IPv6NetworkContainer
Infoblox::DHCP::IPv6NetworkContainer
Infoblox::DHCP::IPv6NetworkTemplate
Infoblox::DHCP::IPv6Range
Infoblox::DHCP::IPv6RangeTemplate
Infoblox::DHCP::IPv6SharedNetwork
Infoblox::DHCP::MAC
Infoblox::DHCP::MSSuperscope
Infoblox::DHCP::Network
```

```
Infoblox::DHCP::NetworkContainer
Infoblox::DHCP::NetworkTemplate
Infoblox::DHCP::Range
Infoblox::DHCP::RangeTemplate
Infoblox::DHCP::RoamingHost
Infoblox::DHCP::SharedNetwork
Infoblox::DHCP::View
Infoblox::DNS::BulkHost
Infoblox::DNS::Host
Infoblox::DNS::RPZRecord::A
Infoblox::DNS::RPZRecord::AAAA
Infoblox::DNS::RPZRecord::AAAAIpAddress
Infoblox::DNS::RPZRecord::AIpAddress
Infoblox::DNS::RPZRecord::CNAME
Infoblox::DNS::RPZRecord::CNAME::ClientIpAddress
Infoblox::DNS::RPZRecord::CNAME::ClientIpAddressDN
Infoblox::DNS::RPZRecord::CNAME::IpAddress
Infoblox::DNS::RPZRecord::CNAME::IpAddressDN
Infoblox::DNS::RPZRecord::MX
Infoblox::DNS::RPZRecord::NAPTR
Infoblox::DNS::RPZRecord::PTR
Infoblox::DNS::RPZRecord::SRV
Infoblox::DNS::RPZRecord::TXT
Infoblox::DNS::Record::A
Infoblox::DNS::Record::AAAA
Infoblox::DNS::Record::Alias
Infoblox::DNS::Record::CNAME
Infoblox::DNS::Record::DNAME
Infoblox::DNS::Record::MX
Infoblox::DNS::Record::NS
Infoblox::DNS::Record::PTR
Infoblox::DNS::Record::SRV
Infoblox::DNS::Record::TLSA
Infoblox::DNS::Record::TXT
Infoblox::DNS::SRG
Infoblox::DNS::SharedRecord::A
Infoblox::DNS::SharedRecord::AAAA
Infoblox::DNS::SharedRecord::CNAME
Infoblox::DNS::SharedRecord::MX
Infoblox::DNS::SharedRecord::SRV
Infoblox::DNS::SharedRecord::TXT
Infoblox::DNS::View
Infoblox::DNS::Zone
Infoblox::DTC::LBDN
Infoblox::DTC::Monitor::HTTP
Infoblox::DTC::Monitor::ICMP
Infoblox::DTC::Monitor::PDP
Infoblox::DTC::Monitor::SIP
Infoblox::DTC::Monitor::SNMP
Infoblox::DTC::Monitor::TCP
Infoblox::DTC::Pool
Infoblox::DTC::Server
Infoblox::DTC::Topology
Infoblox::Grid::AllEndpoints
Infoblox::Grid::DNS
Infoblox::Grid::DNS::DNS64Group
Infoblox::Grid::DNS::Nsgroup
Infoblox::Grid::DNS::Nsgroup::DelegationMember
Infoblox::Grid::DNS::Nsgroup::ForwardStubServer
Infoblox::Grid::DNS::Nsgroup::ForwardingMember
Infoblox::Grid::Discovery::Device
Infoblox::Grid::Discovery::DeviceInterface
Infoblox::Grid::MSServer
Infoblox::Grid::Member::DNS
Infoblox::Grid::NamedACL
Infoblox::Grid::ParentalControl::SubscriberSite
Infoblox::Grid::RIR::Organization
Infoblox::Grid::SNMP::User
Infoblox::Grid::ServiceRestart::Group
```

If the parameter is undefined, the extensible attribute can be defined on all object types. The default value is undefined.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
# Get allowed object types
my $ref_allowed_object_types = $ext_attr_def->allowed_object_types();
# Modify allowed object types
$ext_attr_def->allowed_object_types(['Infoblox::DHCP::Range', 'Infoblox::DHCP::RangeTemplate']);
```

audit_log( )

Use this method to set or retrieve the "audit_log" flag.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

Specify "true" to set the audit_log flag or "false" to deactivate/unset it. When the flag is set the extensible attribute name and value assigned to an object are logged in the audit log when a change occurs on the object. The default value for this field is "false".

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
# Get audit_log flag
my $audit_log = $ext_attr_def->audit_log();
# Modify audit_log flag
$ext_attr_def->audit_log("true");
```

comment( )

Use this method to set or retrieve the comment.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

Comment in string format with a maximum of 256 characters. The default value is undefined.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get comment
my $comment = $ext_attr_def->comment();
#Modify comment
$ext_attr_def->comment("This is the modified comment for extensible attribute definition");
```

default_value( )

Use this method to set or retrieve the value used to pre-populate the attribute value in the GUI.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

Default value used to pre-populate the attribute value in the GUI. For email, URL, and string types, the value is a string with a maximum of 256 characters. For a list, the value is a string with a maximum of 64 characters. For an integer, the value is an integer from -2147483648 through 2147483647. For a date, the value is the number of seconds that have elapsed since January 1st, 1970 UTC. The default value is undefined.

Note that, for a date, the time value is automatically modified to 11:01:00 before insertion, so you may not get the same value that you've inserted upon retrieval. e.g. 1000000 (1970-01-12T13:46:40Z) will be returned as 990060 (1970-01-12T11:01:00Z).

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
# Get default value of an extensible attribute of type string
my $ext_attr_def = $ext_attr_def->default_value();
# Modify default value
$ext_attr_def->default_value("Router");
```

descendants_action( )

Use this method to set the actions for extensible attributes that exist on descendants. This attribute is write-only and cannot be retrieved.

Parameter

Valid value is an Infoblox::Grid::ExtensibleAttributeDef::Descendants object.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

Example

```
#Modify attribute value
$ext_attr_def->descendants_action($descendants);
```

inheritable( )

Use this method to set or retrieve the "inheritable" flag.

Parameter

Specify "true" to set the inheritable flag or "false" to deactivate/unset it. The default value is "false".

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
# Get inheritable flag
my $inheritable = $ext_attr_def->inheritable();
# Modify inheritable flag
$ext_attr_def->inheritable("true");
```

list_values( )

Use this method to set or retrieve the list of values that can be defined for the list type.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

Parameter is an array reference that contains Infoblox::Grid::ExtensibleAttributeDef::ListValue values associated with an extensible attribute object defined as the "list" type. This parameter is not used for other types. The default value for this field is undefined.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
# Get list of values
my $ref_list_values = $ext_attr_def->list_values();

# Add a new value to the list
my $ref_list_values = $ext_attr_def->list_values();
push @$ref_list_values, Infoblox::Grid::ExtensibleAttributeDef::ListValue->new(value => 'Santa
Clara');
$ext_attr_def->list_values($ref_list_values);

# Rename a value in the list
my $ref_list_values = $ext_attr_def->list_values();
foreach my $value (@$ref_list_values)
{
   if ($value->value() eq 'San Francisco')
   {
       $value->value('Los Angeles');
   }
}
$ext_attr_def->list_values($ref_list_values);
```

max( )

Use this method to set or retrieve the maximal length of string for string type or maximal value for integer type.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

For a string, the value is an integer from 1 through 256. For an integer, the value is an integer from -2147483648 through

2147483647. This parameter is not used for other types. Using the Infoblox::Session->modify() function you can only define a value greater than the current one. The default value is undefined.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
# Get max
my $ext_attr_def = $ext_attr_def->max();
# Modify max
$ext_attr_def->max(23);
```

min( )

Use this method to set or retrieve the minimal length of string for string type or minimal value for integer type.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

For a string, the value is an integer from 1 through 256. For an integer, the value is an integer from -2147483648 through 2147483647. This parameter is not used for other types. Using the Infoblox::Session->modify() function you can only define a value lesser than the current one. The default value is undefined.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
# Get min
my $ext_attr_def = $ext_attr_def->min();
# Modify min
$ext_attr_def->min(3);
```

multiple( )

Use this method to set or retrieve the "multiple" flag.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

Specify "true" to set the multiple flag or "false" to deactivate/unset it. When the flag is set multiple values can be assigned to an extensible attribute. Using the Infoblox::Session->modify() function you can change this flag from "false" to "true". The reverse operation is not allowed. The default value for this field is "false".

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
# Get multiple flag
my $audit_log = $ext_attr_def->multiple();
# Modify multiple flag
$ext_attr_def->multiple("true");
```

cloud_api( )

Use this method to set or retrieve the "cloud_api" flag.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

Specify "true" to set the cloud_api flag or "false" to deactivate or unset it. The default value is "false".

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
# Get cloud_api flag
my $cloud_api = $ext_attr_def->cloud_api();
# Modify cloud_api flag
$ext_attr_def->cloud_api("true");
```

grid_master( )

Use this method to set or retrieve the "grid_master" flag.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

Specify "true" to set the grid_master flag or "false" to deactivate or unset it. The default value is "false".

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
# Get grid_master flag
my $grid_master = $ext_attr_def->grid_master();
# Modify grid_master flag
$ext_attr_def->grid_master("true");
```

mgm_private( )

Use this method to set or retrieve the "mgm_private" flag.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

Specify "true" to set the mgm_private flag or "false" to deactivate or unset it. The default value is "false".

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
# Get mgm_private flag
my $mgm_private = $ext_attr_def->mgm_private();
# Modify mgm_private flag
$ext_attr_def->mgm_private("true");
```

name( )

Use this method to set or retrieve the name of an extensible attribute definition.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

Name of the extensible attribute with a maximum of 128 characters.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
# Get name
my $name = $ext_attr_def->name();
# Modify name
$ext_attr_def->name("Class");
```

namespace( )

Use this method to retrieve the extensible attribute definition namespace. This is a read-only attribute.

Specific extensible attribute definitions with 'RIPE' namespace will be added only when support for RIR/SWIP is enabled at the Grid level for the first time. See Infoblox::Grid->enable_rir_swip() method.

Parameter

None

Returns

The method returns the attribute value that can be 'default', 'MSADSITES', 'RIPE' or 'CLOUD'.

Example

```
# Get namespace
$namespace = $ext_attr_def->namespace();
```

type( )

Use this method to set or retrieve the type of an extensible attribute.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

Parameter is a string. The default value for this field is "string". Currenty the supported object types are 'string', 'integer', 'email', 'date', 'list' and 'url'. Modification of the type using the Infoblox::Session->modify() function is not allowed.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get type
my $type = $ext_attr_def->type();
```

user_input( )

Use this method to set or retrieve the flag that indicates whether the extensible attribute value is required, recommended or optional.

Parameter

Valid values are 'recommended', 'required' or 'optional'. The default value is 'optional'.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get user_input
my $user_input = $ext_attr_def->user_input();
#Modify user_input
$ext_attr_def->user_input('required');
```

SAMPLE CODE

The following sample code demonstrates the different functions that can be applied to an object, such as add, get, modify, and remove. This sample also includes error handling for the operations.

#Preparation prior to an Extensible Attribute Definition object insertion

```
#PROGRAM STARTS: Include all the modules that will be used
use strict;
use Infoblox;

#Create a session to the Infoblox appliance

my $session = Infoblox::Session->new(
                                        #appliance host ip
```

```
                master   => "192.168.1.2",
                username => "admin",        #appliance user login
                password => "infoblox"      #appliance password
                );

unless (($session) {
        die("Construct session failed: ",
                Infoblox::status_code() . ":" . Infoblox::status_detail());
}
print "Session created successfully\n";
```

#Create an Extensible Attribute Definition object

```
my $ext_attr_def = Infoblox::Grid::ExtensibleAttributeDef->new(
    name                => "test_extensible_atttribute",
    comment             => "test extensibleatttribute",
    type                => "integer",
    min                 => -50,
    max                 => 50,
);

unless (($ext_attr_def) {
        die("Construct extensible attribute definition object failed: ",
                Infoblox::status_code() . ":" . Infoblox::status_detail());
}
print "Extensible Attribute Definition object created successfully\n";

#Add this object to the Infoblox appliance through a session
$session->add($ext_attr_def)
        or die("Add Extensible Attribute Definition object failed: ",
                        $session->status_code() . ":" . $session->status_detail());
print "Extensible Attribute Definition object added to server successfully\n";
```

#Get and modify an Extensible Attribute Definition object

```
#Get the object from Infoblox appliance through a session
my @retrieved_objs = $session->get(
    object => "Infoblox::Grid::ExtensibleAttributeDef",
    name   => "test_extensible_atttribute"
);
my $object = $retrieved_objs[0];

unless (($object) {
        die("Get Extensible Attribute Definition object failed: ",
                $session->status_code() . ":" . $session->status_detail());
}
print "Get Extensible Attribute Definition object found at least 1 matching entry\n";

#Modify the object
$object->comment("this is a new comment");
$object->min(-100);
$object->max(100);

#Apply the changes.
$session->modify($object)
        or die("Modify Extensible Attribute definition object failed: ",
                $session->status_code() . ":" . $session->status_detail());
print "Extensible Attribute definition object modified successfully \n";

#Accessor methods
print "name: " . $object->name() . "\n";
print "comment: " . $object->comment() . "\n";
print "min: " . $object->min() . "\n";
print "max: " . $object->max() . "\n";
```

#Remove an Extensible Attribute Definition object

```
#Get the object through the session
my @retrieved_objs = $session->get(
    object => "Infoblox::Grid::ExtensibleAttributeDef",
    name   => "test_extensible_atttribute"
);
my $object = $retrieved_objs[0];
unless (($object) {
    die("Get Extensible Attribute Definition object failed: ",
        $session->status_code() . ":" . $session->status_detail());
}
```

```
print "Get Extensible Attribute Definition object found at least 1 matching entry\n";

#Submit the object for removal
$session->remove($object)
       or die("Remove Extensible Attribute Definition object failed: ",
                $session->status_code() . ":" . $session->status_detail());
print "Extensible Attribute Definition object removed successfully \n";

####PROGRAM ENDS####
```

AUTHOR

Infoblox Inc. http://www.infoblox.com/

SEE ALSO

Infoblox::Session->add(), Infoblox::Session->get(), Infoblox::Session->modify(), Infoblox::Session->remove(), Infoblox::Session->search(),Infoblox::Session,Infoblox::Grid::ExtensibleAttributeDef::Descendants

---

COPYRIGHT

Copyright (c) 2019 Infoblox Inc.

Infoblox::Grid::Extattr - an extensible attribute representation for an attribute.

- NAME
- DESCRIPTION
- CONSTRUCTOR
- SESSION METHODS
- METHODS
  - descendants_action( )
  - inheritance_operation( )
  - inheritance_source( )
  - value( )
- AUTHOR
- SEE ALSO
- COPYRIGHT

NAME

Infoblox::Grid::Extattr - an extensible attribute representation for an attribute.

DESCRIPTION

An object that provides an extensible attribute representation for an attribute.

CONSTRUCTOR

```
my $extattr = Infoblox::Grid::Extattr->new(
   value                => $scalar | [$scalar1, $scalar2, ...],      # required
   inheritance_operation => 'DELETE' | 'INHERIT' | 'OVERRIDE',       # optional
   descendants_action    => $descendants,                            # optional
);
```

SESSION METHODS

The object does not support any session methods.

METHODS

This section describes all the methods that you can use to configure and retrieve the attribute values of the extensible attribute representation for an attribute.

descendants_action( )

Use this method to set the actions for extensible attributes that exist on descendants. This attribute is write-only and cannot be retrieved.

Parameter

Valid value is an Infoblox::Grid::ExtensibleAttributeDef::Descendants object.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

Example

```
#Modify attribute value
$extattr->descendants_action($descendants);
```

inheritance_operation( )

Use this method to set the operation that will be performed on an extensible attribute. This attribute is write-only and cannot be retrieved.

Parameter

Valid value is 'DELETE', 'INHERIT' or 'OVERRIDE'.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

Example

```
#Modify attribute value
$extattr->inheritance_operation('INHERIT');
```

inheritance_source( )

Use this method to retrieve the object from which the extensible attribute was inherited. This attribute is read-only and cannot be set

This object is considered a 'partial' object. See Infoblox::Session->fill_partial_object() for more information on how it can be converted to a 'full' API object.

Parameter

None.

Returns

The method returns the attribute value.

Example

```
#Get attribute value
my $value = $extattr->inheritance_source();
```

value( )

Use this method to set or retrieve the value of the extensible attribute representation for an attribute.

Parameter

For email, URL, and string types, the value is a string with a maximum of 256 characters.

For a list, the value is a reference to an array of strings with a maximum of 64 characters.

For an integer, the value is an integer from -2147483648 through 2147483647.

For a date, the value is the number of seconds that have elapsed since January 1st, 1970 UTC or a string in YYYY-MM-DD format. Any date entered will be converted to an epoch time corresponding to that day at 11:01am UTC.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get attribute value
my $value = $extattr->value();
#Modify attribute value
$extattr->value('www.infoblox.com');
```

AUTHOR

Infoblox Inc. http://www.infoblox.com/

SEE ALSO

Infoblox::Session, Infoblox::Grid::ExtensibleAttributeDef::Descendants

---

COPYRIGHT

Copyright (c) 2017 Infoblox Inc.

Infoblox::CiscoISE::PublishSetting - A Cisco ISE publish settings.

- NAME
- DESCRIPTION
- CONSTRUCTOR
- SESSION METHODS
- MODULE METHODS
  - Infoblox::CiscoISE::Endpoint->publish_settings( )
  - Infoblox::Notification::Rule->publish_settings( )
- METHODS
  - enabled_attributes( )
- AUTHOR
- SEE ALSO
- COPYRIGHT

NAME

Infoblox::CiscoISE::PublishSetting - A Cisco ISE publish settings.

DESCRIPTION

The Cisco ISE publish setting contains a list of extensible attributes that are enabled for publishing to a Cisco ISE server.

CONSTRUCTOR

```
my $publish_setting = Infoblox::CiscoISE::PublishSetting->new(
    enabled_attributes => ['CLIENT_ID' | 'FINGERPRINT' | 'HOSTNAME' | 'IPADDRESS' |
                           'INFOBLOX_MEMBER' | 'LEASE_STATE' | 'LEASE_START_TIME' |
                           'LEASE_END_TIME' | 'MAC_OR_DUID' | 'NETBIOS_NAME', ...],
);
```

SESSION METHODS

The object does not have any session methods.

MODULE METHODS

This section describes all the functions that you can apply to Cisco ISE publish settings object.

Infoblox::CiscoISE::Endpoint->publish_settings( )

Use this method to set or retrieve the Cisco ISE endpoint publish settings. See Infoblox::CiscoISE::Endpoint->publish_settings() for parameters and return values.

Example

```
#Get publish_settings value
my $publish_settings = $object->publish_settings();

#Modify publish_settings value
$object->publish_settings($publish_settings);
```

Infoblox::Notification::Rule->publish_settings( )

Use this method to set or retrieve the notification rule publish settings. See Infoblox::Notification::Rule->publish_settings() for parameters and return values.

Example

```
#Get publish_settings value
my $publish_settings = $object->publish_settings();

#Modify publish_settings value
$object->publish_settings($publish_settings);
```

METHODS

This section describes all the methods that you can use to configure and retrieve the attribute values of a Cisco ISE publish settings.

enabled_attributes( )

Use this method to set or retrieve the list of extensible attributes that are enabled for publishing to a Cisco ISE server.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

The valid value is an array that contains following values: 'CLIENT_ID', 'FINGERPRINT', 'HOSTNAME', 'IPADDRESS', 'INFOBLOX_MEMBER', 'LEASE_STATE', 'LEASE_START_TIME', 'LEASE_END_TIME', 'MAC_OR_DUID', 'NETBIOS_NAME'.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get enabled_attributes value
my $enabled_attributes = $object->enabled_attributes();

#Modify enabled_attributes value
$object->enabled_attributes(['CLIENT_ID', 'IPADDRESS']);
```

AUTHOR

Infoblox Inc. http://www.infoblox.com/

SEE ALSO

Infoblox::CiscoISE::Endpoint, Infoblox::CiscoISE::Endpoint->publish_settings(), Infoblox::Notification::Rule, Infoblox::Notification::Rule->publish_settings(),

---

COPYRIGHT

Copyright (c) 2017 Infoblox Inc.

Infoblox::CiscoISE::Endpoint - A Cisco ISE endpoint object.

- NAME
- DESCRIPTION
- CONSTRUCTOR
- SESSION METHODS
  - Infoblox::Session->add( )
  - Infoblox::Session->get( )
  - Infoblox::Session->modify( )
  - Infoblox::Session->remove( )
  - Infoblox::Session->search( )
- METHODS
  - address( )
  - bulk_download_certificate( )
  - bulk_download_certificate_subject( )
  - bulk_download_certificate_valid_from( )
  - bulk_download_certificate_valid_to( )
  - client_certificate( )
  - client_certificate_subject( )
  - client_certificate_valid_from( )
  - client_certificate_valid_to( )
  - comment( )
  - connection_status( )
  - connection_timeout( )
  - disable( )
  - extattrs( )
  - extensible_attributes( )
  - network_view( )
  - publish_settings( )
  - resolved_address( )
  - resolved_secondary_address( )
  - secondary_address( )
  - subscribe_settings( )
  - subscribing_member( )
  - type( )
  - version( )
- AUTHOR
- SEE ALSO
- COPYRIGHT

NAME

Infoblox::CiscoISE::Endpoint - A Cisco ISE endpoint object.

DESCRIPTION

The CiscoISE endpoint object provides configuration for third party Cisco ISE servers integration.

CONSTRUCTOR

```
 my $ise_endpoint = Infoblox::CiscoISE::Endpoint->new(
     address                  => $fqdn | $ipaddr,                                                  #
Required
     bulk_download_certificate => $path,                                                           #
Required
     client_certificate       => $path,                                                            #
Required
     subscribe_settings       => $subscribe_settings,                                              #
Required
     subscribing_member       => $string,                                                          #
Required
     version                  => 'VERSION_1_3' | 'VERSION_1_4' | 'VERSION_2_0',                    #
Required
     comment                  => $string,                                                          #
Optional / Default is undefined
     connection_timeout       => $uint,                                                            #
Optional / Default is 10
     disable                  => 'true' | 'false',                                                 #
Optional / Default is 'true'
     extattrs                 => { $string => $extattr, ... },                                     #
Optional / Default is undefined
     extensible_attributes    => {$string => $string | $num, $string => [ $string | $num, ... ], ... } #
```

```
Optional / Default is undefined
     network_view              => $string,                                    #
Optional / Default is 'default'
     publish_settings          => $publish_settings,                          #
Optional / Default is undefined
     secondary_address         => $fqdn | $ipaddr,                            #
Optional / Default is undefined
     type                      => 'TYPE_CISCO',                               #
Optional / Default is 'TYPE_CISCO',
 );
```

You cannot set both extattrs and extensible_attributes attributes at the same time.

SESSION METHODS

This section describes all the methods in an Infoblox::Session module that you can apply to a Cisco ISE endpoint object.

Infoblox::Session->add( )

Use this method to add the object to the Infoblox appliance. See Infoblox::Session->add() for parameters and return values.

Example

```
#Construct an object
my $object = Infoblox::CiscoISE::Endpoint->new(
    address                   => '10.0.0.1',
    bulk_download_certificate => '/path/to/bulk_download_cert.pem',
    client_certificate        => '/path/to/client_cert.pem',
    subscribe_settings        => $subscribe_settings,
    subscribing_member        => 'infoblox.com',
    version                   => 'VERSION_1_3',
);

#Submit for addition
my $response = $session->add($object);
```

Infoblox::Session->get( )

Use this method to retrieve all the matching objects from the Infoblox appliance. See Infoblox::Session->get() for parameters and return values.

Key References

```
Apply the following attributes to get a specific Cisco ISE endpoint object:

 address                   - Optional. The FQDN or IP address of the Cisco ISE endpoint.
 comment                   - Optional. The Cisco ISE endpoint comment in string format
 extattrs                  - Optional. A hash reference containing Infoblox::Grid::Extattr object.
 extensible_attributes     - Optional. A hash reference containing extensible attributes.
 network_view              - Optional. The Cisco ISE endpoint network view name.
 resolved_address          - Optional. The resolved IP address of the Cisco ISE endpoint.
 resolved_secondary_address - Optional. The resolved secondary IP address of the Cisco ISE endpoint.
 secondary_address         - Optional. The secondary FQDN or IP address of the Cisco ISE endpoint.
 subscribing_member        - Optional. The Cisco ISE endpoint subscribing member name.
 type                      - Optional. The Cisco ISE endpoint type.
 version                   - Optional. The Cisco ISE endpoint version.
```

Examples

```
my @retrieved_objs = $session->get(
    object  => 'Infoblox::CiscoISE::Endpoint',
    address => '10.0.0.1',
);

my @retrieved_objs = $session->get(
    object               => 'Infoblox::CiscoISE::Endpoint',
    extensible_attributes => {'Site' => 'Santa Clara'},
);
```

Infoblox::Session->modify( )

Use this method to modify the object in the Infoblox appliance. See Infoblox::Session->modify() for parameters and return values.

Example

```
#Modify comment value
$object->comment('this is a modified comment');

#Submit modification
```

```
my $response = $session->modify($object);
```

Infoblox::Session->remove( )

Use this method to remove the object from the Infoblox appliance. See Infoblox::Session->remove() for parameters and return values.

To remove a specific object, first use `get()` or `search()` to retrieve the specific DHCP range object, and then submit this object for removal.

Example

```
#Get the objects
my @retrieved_objs = $session->get(
    object       => 'Infoblox::CiscoISE::Endpoint',
    address      => '10.0.0.1',
    newtork_view => 'default',
);

#Find the desired object from the retrieved list
my $desired_object = $retrieved_objs[0];

#Submit for removal
my $response = $session->remove($desired_object);
```

Infoblox::Session->search( )

Use this method to search for objects in the Infoblox appliance. See Infoblox::Session->search() for parameters and return values.

Key References

```
Apply the following attributes to search for the Cisco ISE endpoint object:

 address                    - Optional. The FQDN or IP address of the Cisco ISE endpoint (regexp).
 comment                    - Optional. The Cisco ISE endpoint comment in string format (regexp).
 extattrs                   - Optional. A hash reference containing Infoblox::Grid::Extattr object.
 extensible_attributes      - Optional. A hash reference containing extensible attributes.
 network_view               - Optional. The Cisco ISE endpoint network view name (exact).
 resolved_address           - Optional. The resolved IP address of the Cisco ISE endpoint (regexp).
 resolved_secondary_address - Optional. The resolved secondary IP address of the Cisco ISE endpoint
(regexp).
 secondary_address          - Optional. The secondary FQDN or IP address of the Cisco ISE endpoint
(regexp).
 subscribing_member         - Optional. The Cisco ISE endpoint subscribing member name (exact).
 type                       - Optional. The Cisco ISE endpoint type (exact).
 version                    - Optional. The Cisco ISE endpoint version (exact).
```

For more information about searching extensible attributes, see Infoblox::Grid::ExtensibleAttributeDef/Searching Extensible Attributes.

Examples

```
# search for objects
my @retrieved_objs = $session->search(
    object       => 'Infoblox::CiscoISE::Endpoint',
    network_view => 'default',
    address      => '10.*',
    comment      => '.*comment',
);

# search for objects defining 'Santa Clara' for 'Site' extensible attribute
my @retrieved_objs = $session->search(
    object               => 'Infoblox::DHCP::Range',
    extensible_attributes => {'Site' => 'Santa Clara'},
);
```

METHODS

This section describes all the methods that you can use to configure and retrieve the attribute values of a Cisco ISE endpoint.

address( )

Use this method to set or retrieve the fully-qualified domain name (FQDN) or IP address of the Cisco ISE endpoint.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

The valid value is a desired IP address or FQDN in string format.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get address value
my $address = $object->address();

#Modify address value
$object->address('foo.com');
```

bulk_download_certificate( )

Use this method to set the path to a bulk download certificate file. This is a write-only attribute.

Include the specified parameter to set the attribute value.

Parameter

The valid value is a path to a certificate in string format.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

Example

```
#Modify bulk_download_certificate value
$object->bulk_download_certificate('/path/to/cert.pem');
```

bulk_download_certificate_subject( )

Use this method to retrieve the bulk download certificate subject. This is a read-only attrbiute.

Omit the parameter to retrieve the attribute value.

Parameter

None

Returns

The method returns the attribute value.

Example

```
#Get bulk_download_certificate_subject value
my $bulk_download_certificate_subject = $object->bulk_download_certificate_subject();
```

bulk_download_certificate_valid_from( )

Use this method to retrieve the time from which the bulk download certificate is valid. This is a read-only attrbiute.

Omit the parameter to retrieve the attribute value.

Parameter

None

Returns

The method returns the attribute value. The number of seconds that have elapsed since January 1st, 1970 UTC.

Example

```
#Get bulk_download_certificate_valid_from value
my $bulk_download_certificate_valid_from = $object->bulk_download_certificate_valid_from();
```

bulk_download_certificate_valid_to( )

Use this method to retrieve the bulk download certificate expiration time. This is a read-only attrbiute.

Omit the parameter to retrieve the attribute value.

Parameter

None

Returns

The method returns the attribute value. The number of seconds that have elapsed since January 1st, 1970 UTC.

Example

```
#Get bulk_download_certificate_valid_to value
my $bulk_download_certificate_valid_to = $object->bulk_download_certificate_valid_to();
```

client_certificate( )

Use this method to set the path to a client certificate file. This is a write-only attribute.

Include the specified parameter to set the attribute value.

Parameter

The valid value is a path to a certificate in string format.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

Example

```
#Modify client_certificate value
$object->client_certificate('/path/to/cert.pem');
```

client_certificate_subject( )

Use this method to retrieve the client certificate subject. This is a read-only attrbiute.

Omit the parameter to retrieve the attribute value.

Parameter

None

Returns

The method returns the attribute value.

Example

```
#Get client_certificate_subject value
my $client_certificate_subject = $object->client_certificate_subject();
```

client_certificate_valid_from( )

Use this method to retrieve the time from which the client certificate is valid. This is a read-only attrbiute.

Omit the parameter to retrieve the attribute value.

Parameter

None

Returns

The method returns the attribute value. The number of seconds that have elapsed since January 1st, 1970 UTC.

Example

```
#Get client_certificate_valid_from value
my $client_certificate_valid_from = $object->client_certificate_valid_from();
```

client_certificate_valid_to( )

Use this method to retrieve the bulk client certificate expiration time. This is a read-only attrbiute.

Omit the parameter to retrieve the attribute value.

Parameter

None

Returns

The method returns the attribute value. The number of seconds that have elapsed since January 1st, 1970 UTC.

Example

```
#Get client_certificate_valid_to value
my $client_certificate_valid_to = $object->client_certificate_valid_to();
```

comment( )

Use this method to set or retrieve the Cisco ISE endpoint comment.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

The valid value is a desired comment in string format.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get comment value
my $comment = $object->comment();

#Modify comment value
$object->comment('ise comment');
```

connection_status( )

Use this method to retrieve the Cisco ISE endpoint connection status. This is a read-only attrbiute.

Omit the parameter to retrieve the attribute value.

Parameter

None

Returns

The method returns the attribute value.

Example

```
#Get connection_status value
my $connection_status = $object->connection_status();
```

connection_timeout( )

Use this method to set or retrieve the timeout value (in seconds) for the Cisco ISE endpoint connections.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

The valid value is an unsigned integer between 1 and 180. The default value is 10.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get connection_timeout value
my $connection_timeout = $object->connection_timeout();

#Modify connection_timeout value
$object->connection_timeout('20');
```

disable( )

Use this method to set or retrieve the flag that indicates whether the Cisco ISE endpoint is enabled or disabled.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

Specify 'true' to disable the Cisco ISE endpoint or 'false' to enable it. The default value is 'true'.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get disable value
my $disable = $object->disable();

#Modify disable value
$object->disable('false');
```

extattrs( )

Use this method to set or retrieve the extensible attributes associated with a Cisco ISE endpoint object.

Parameter

Valid value is a hash reference that contains the names of extensible attributes and their associated values (Infoblox::Grid::Extattr objects).

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get extattrs value
my $extattrs = $object->extattrs();

#Modify extattrs value
$dhcp_range->extattrs({'Site' => $extattr1, 'Administrator' => $extattr2});
```

extensible_attributes( )

Use this method to set or retrieve the extensible attributes associated with a Cisco ISE endpoint object.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

For valid values for extensible attributes, see Infoblox::Grid::ExtensibleAttributeDef/Extensible Attribute Values.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get extensible attributes value
my $extensible_attributes = $object->extensible_attributes();
#Modify extensible attributes
$object->extensible_attributes({'Site' => 'Santa Clara', 'Administrator' => ['Peter', 'Tom']});
```

network_view( )

Use this method to set or retrieve the Cisco ISE endpoint network view name.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

The valid value is a desired network view name in string format. The default value is 'default'.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get network_view value
my $network_view = $object->network_view();

#Modify network_view value
$object->network_view('internal');
```

publish_settings( )

Use this method to set or retrieve the Cisco ISE endpoint publish settings.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

The valid value is an Infoblox::CiscoISE::PublishSetting object.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get publish_settings value
my $publish_settings = $object->publish_settings();

#Modify publish_settings value
$object->publish_settings($publish_settings);
```

resolved_address( )

Use this method to retrieve the resolved IP address of the Cisco ISE endpoint. This is a read-only attribute.

Omit the parameter to retrieve the attribute value.

Parameter

None

Returns

The method returns the attribute value.

Example

```
#Get resolved_address value
my $resolved_address = $object->resolved_address();
```

resolved_secondary_address( )

Use this method to retrieve the resolved secondary IP address of the Cisco ISE endpoint. This is a read-only attribute.

Omit the parameter to retrieve the attribute value.

Parameter

None

Returns

The method returns the attribute value.

Example

```
#Get resolved_secondary_address value
my $resolved_secondary_address = $object->resolved_secondary_address();
```

secondary_address( )

Use this method to set or retrieve the secondary fully-qualified domain name (FQDN) or IP address of the Cisco ISE endpoint.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

The valid value is a desired IP address or FQDN in string format.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get secondary_address value
my $secondary_address = $object->secondary_address();

#Modify secondary_address value
$object->secondary_address('foo.com');
```

subscribe_settings( )

Use this method to set or retrieve the Cisco ISE endpoint subscribe settings.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

The valid value is an Infoblox::CiscoISE::SubscribeSetting object.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get subscribe_settings value
my $subscribe_settings = $object->subscribe_settings();

#Modify subscribe_settings value
$object->subscribe_settings($subscribe_settings);
```

subscribing_member( )

Use this method to set or retrieve the Cisco ISE endpoint subscribing member host name.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

The valid value is a desired subscribing member host name in string format. The default value is 'default'.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get subscribing_member value
my $subscribing_member = $object->subscribing_member();

#Modify subscribing_member value
$object->subscribing_member('infoblox.com');
```

type( )

Use this method to set or retrieve the Cisco ISE endpoint type.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

The valid value is 'TYPE_CISCO'.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get type value
my $type = $object->type();

#Modify type value
$object->type('TYPE_CISCO');
```

version( )

Use this method to set or retrieve the Cisco ISE endpoint version.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

The valid values are 'VERSION_1_3', 'VERSION_1_4' and 'VERSION_2_0'.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get version value
my $version = $object->version();

#Modify version value
$object->version('VERSION_2_0');
```

AUTHOR

Infoblox Inc. http://www.infoblox.com/

SEE ALSO

Infoblox::Session, Infoblox::Session->add(), Infoblox::Session->get(), Infoblox::Session->modify(), Infoblox::Session->remove(), Infoblox::Session->search(), Infoblox::Grid::Extattr, Infoblox::Grid::ExtensibleAttributeDef/Searching Extensible Attributes, Infoblox::CiscoISE::PublishSetting, Infoblox::CiscoISE::SubscribeSetting,

---

COPYRIGHT

Copyright (c) 2017 Infoblox Inc.

Infoblox::Notification::Rule - A notification rule object.

- NAME
- DESCRIPTION
- CONSTRUCTOR
- SESSION METHODS
  - Infoblox::Session->add( )
  - Infoblox::Session->get( )
  - Infoblox::Session->modify( )
  - Infoblox::Session->remove( )
  - Infoblox::Session->search( )
- METHODS
  - all_members( )
  - comment( )
  - disable( )
  - enable_event_deduplication( )
  - enable_event_deduplication_log( )
  - event_deduplication_fields( )
  - event_deduplication_lookback_period( )
  - event_type( )
  - expression_list( )
  - name( )
  - notification_action( )
  - notification_target( )
  - override_publish_settings( )
  - publish_settings( )
  - selected_members( )
  - template_instance( )
- AUTHOR
- SEE ALSO
- COPYRIGHT

NAME

Infoblox::Notification::Rule - A notification rule object.

DESCRIPTION

The notification rule object provides configuration for notification of endpoints when the appropriate event occurs.

CONSTRUCTOR

```
 my $notification_rule = Infoblox::Notification::Rule->new(
     event_type                          => 'DNS_RPZ' | 'DHCP_LEASES' | 'SECURITY_ADP' |
                                            'IPAM',                                          # Required
     expression_list                     => [$rule_op1, $rule_op2, ...],                      # Required
     name                                => $string,                                          # Required
     notification_action                 => 'CISCOISE_QUARANTINE' | 'CISCOISE_PUBLISH' |
                                            'RESTAPI_TEMPLATE_INSTANCE',                     # Required
     notification_target                 => $ise_endpoint | $rest_endpoint | $dxl_endpoint    # Required
     all_members                         => 'true' | 'false',                                 # Optional /
if no selected_members given default is 'true'
     comment                             => $string,                                          # Optional /
Default is undefined
     disable                             => 'true' | 'false',                                 # Optional /
Default is 'false'
     override_publish_settings           => 'true' | 'false',                                 # Optional
default is 'false'
     publish_settings                    => $publish_settings,                                # Optional /
Default is undefined
     selected_members                    => [$member1, $member2, ...],                        # Optional /
Default is empty list
     template_instance                   => $instance,                                        # Optional /
Default is undefined
     enable_event_deduplication          => 'true' | 'false',                                 # Optional /
Default is 'false'
     event_deduplication_lookback_period => $uint,                                            # Optional /
Default is 600
     enable_event_deduplication_log      => 'true' | 'false',                                 # Optional /
Default is 'false'
     event_deduplication_fields          => [ 'SOURCE_IP' | 'QUERY_NAME' | 'RPZ_POLICY' |
                                              'RPZ_TYPE' | 'QUERY_TYPE' | 'NETWORK' |
```

```
                                                'NETWORK_VIEW', ... ],                  # Optional /
Default is empty list
                                                                                        # Required if
enable_event_deduplication is set to 'true'
 );
```

SESSION METHODS

This section describes all the methods in an Infoblox::Session module that you can apply to a notification rule object.

Infoblox::Session->add( )

Use this method to add the object to the Infoblox appliance. See Infoblox::Session->add() for parameters and return values.

Example

```
#Construct an object
my $object = Infoblox::Notification::Rule->new(
    event_type          => 'DNS_RPZ',
    expression_list     => [$rule_start_list, $rule_op1, $rule_op2, $rule_end_list],
    name                => 'ise_rule1',
    notification_action => 'CISCOISE_QUARANTINE',
    notification_target => $endpoint,
);

#Submit for addition
my $response = $session->add($object);
```

Infoblox::Session->get( )

Use this method to retrieve all the matching objects from the Infoblox appliance. See Infoblox::Session->get() for parameters and return values.

Key References

```
Apply the following attributes to get a specific notification rule object:

 comment             - Optional. The notification rule comment in string format.
 name                - Optional. The notification rule name in string format.
 event_type          - Optional. The notification rule event type in string format.
 notification_action - Optional. The notification rule notification action in string format.
 notification_target - Optional. The Infoblox::CiscoISE::Endpoint,
Infoblox::Notification::REST::Endpoint or Infoblox::DXL::Endpoint object.
```

Examples

```
my @retrieved_objs = $session->get(
    object => 'Infoblox::Notification::Rule',
    name   => 'ise_rule1',
);
```

Infoblox::Session->modify( )

Use this method to modify the object in the Infoblox appliance. See Infoblox::Session->modify() for parameters and return values.

Example

```
#Modify comment value
$object->comment('this is a modified comment');

#Submit modification
my $response = $session->modify($object);
```

Infoblox::Session->remove( )

Use this method to remove the object from the Infoblox appliance. See Infoblox::Session->remove() for parameters and return values.

To remove a specific object, first use `get()` or `search()` to retrieve the specific DHCP range object, and then submit this object for removal.

Example

```
#Get the objects
my @retrieved_objs = $session->get(
    object => 'Infoblox::Notification::Rule',
    name   => 'ise_rule1',
```

```
);

#Find the desired object from the retrieved list
my $desired_object = $retrieved_objs[0];

#Submit for removal
my $response = $session->remove($desired_object);
```

Infoblox::Session->search( )

Use this method to search for objects in the Infoblox appliance. See Infoblox::Session->search() for parameters and return values.

Key References

```
Apply the following attributes to search for the notification rule object:

 comment             - Optional. The notification rule comment in string format (regexp).
 name                - Optional. The notification rule name in string format (regexp).
 event_type          - Optional. The notification rule event type in string format (exact)
 notification_action - Optional. The notification rule notification action in string format (exact).
 notification_target - Optional. The Infoblox::CiscoISE::Endpoint,
Infoblox::Notification::REST::Endpoint or Infoblox::DXL::Endpoint object.
```

Examples

```
# search for objects
my @retrieved_objs = $session->search(
    object  => 'Infoblox::Notification::Rule',
    name    => '^ise.*',
    comment => '.*comment',
);
```

METHODS

This section describes all the methods that you can use to configure and retrieve the attribute values of a notification rule.

all_members( )

Use this method to set or retrieve the flag that indicates whether the notification rule engine is run on all member or on selected members.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

Specify 'true' to run notification rule engine on all members, or 'false' to run it on selected members. The default value unless selected_members are given is 'true'.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get all_members value
my $all_members = $object->all_members();

#Modify all_members value
$object->all_members('false');
```

comment( )

Use this method to set or retrieve the notification rule comment.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

The valid value is a desired comment in string format.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get comment value
my $comment = $object->comment();

#Modify comment value
$object->comment('ise comment');
```

disable( )

Use this method to set or retrieve the flag that indicates whether the notification rule is enabled or disabled.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

Specify 'true' to disable the notification rule or 'false' to enable it. The default value is 'false'.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get disable value
my $disable = $object->disable();

#Modify disable value
$object->disable('false');
```

enable_event_deduplication( )

Use this method to set or retrieve the flag that indicates whether the notification rule for event deduplication is enabled or disabled.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameters

Specify 'true' to enable event deduplication or 'false' to disable it. The default value is 'false'.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get enable_event_deduplication value
my $enable_event_deduplication = $object->enable_event_deduplication();

#Modify enable_event_deduplication value
$object->enable_event_deduplication('true');
```

enable_event_deduplication_log( )

Use this method to set or retrieve the flag that indicates whether the notification rule event deduplication syslog is enabled or disabled.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameters

Specify 'true' to enable event deduplication syslog or 'false' to disable it. The default value is 'false'.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get enable_event_deduplication_log value
my $enable_event_deduplication_log = $object->enable_event_deduplication_log();

#Modify enable_event_deduplication_log value
$object->enable_event_deduplication_log('true');
```

event_deduplication_fields( )

Use this method to set or retrieve the fields in the notification rule for event deduplication.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameters

The valid value is an array reference that contains following values: 'SOURCE_IP', 'QUERY_NAME', 'RPZ_POLICY', 'RPZ_TYPE', 'QUERY_TYPE', 'NETWORK', 'NETWORK_VIEW'.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get event_deduplication_fields value
my $event_deduplication_fields = $object->event_deduplication_fields();

#Modify event_deduplication_fields value
$object->event_deduplication_fields(['SOURCE_IP', 'QUERY_TYPE']);
```

event_deduplication_lookback_period( )

Use this method to set or retrieve the lookback period for the notification rule for event deduplication.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameters

The valid value is an unsigned integer between 5 and 900 that represents the lookback period (in seconds).

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get event_deduplication_lookback_period value
my $event_deduplication_lookback_period = $object->event_deduplication_lookback_period();

#Modify event_deduplication_lookback_period value
$object->event_deduplication_lookback_period(300);
```

event_type( )

Use this method to set or retrieve the notification rule event type.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

The valid values are 'DNS_RPZ', 'IPAM', 'DHCP_LEASES', 'ANALYTICS_DNS_TUNNEL', 'SECURITY_ADP', 'DB_CHANGE_DHCP_FIXED_ADDRESS_IPV4', 'DB_CHANGE_DHCP_FIXED_ADDRESS_IPV6', 'DB_CHANGE_DHCP_NETWORK_IPV4', 'DB_CHANGE_DHCP_NETWORK_IPV6', 'DB_CHANGE_DHCP_RANGE_IPV4', 'DB_CHANGE_DHCP_RANGE_IPV6', 'DB_CHANGE_DNS_HOST_ADDRESS_IPV4' and 'DB_CHANGE_DNS_HOST_ADDRESS_IPV6'.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get event_type value
my $event_type = $object->event_type();

#Modify event_type value
$object->event_type('SECURITY_ADP');
```

expression_list( )

Use this method to set or retrieve the notification rule condition expression.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

The valid value is an array of Infoblox::Notification::RuleExpressionOp objects.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get expression_list value
my $expression_list = $object->expression_list();

#Set event_type value
$object->event_type('SECURITY_ADP');

#Construct ('SECURITY_AD_RULE_SEVERY' LESS_SEVERE 'MAJOR'
# AND SOURCE_IP EQUALS '10.0.0.10') expression

my $start_list = Infoblox::Grid::ExpressionOp->new(
    op1_type => 'LIST',
    op       => 'AND',
);

my $op1 = Infoblox::Grid::ExpressionOp->new(
    op1_type => 'FIELD',
    op1      => 'SECURITY_AD_RULE_SEVERY',
    op       => 'LE',
    op2      => 'MAJOR',
    op2_type => 'STRING',
);

my $op2 = Infoblox::Grid::ExpressionOp->new(
    op1_type => 'FIELD',
    op1      => 'SOURCE_IP',
    op       => 'EQ',
    op2      => '10.0.0.10',
    op2_type => 'STRING',
);

my $end_list = Infoblox::Grid::ExpressionOp->new(
    op1_type => 'LIST',
    op       => 'ENDLIST',
);

#Modify expression_list value
$object->expression_list([$start_list, $op1, $op2, $end_list]);
```

name( )

Use this method to set or retrieve the notification rule name.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

The valid value is a desired name in string format.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get name value
my $name = $object->name();

#Modify name value
$object->name('rule1');
```

notification_action( )

Use this method to set or retrieve the notification rule action.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

The valid values are 'CISCOISE_QUARANTINE', 'CISCOISE_PUBLISH' and 'RESTAPI_TEMPLATE_INSTANCE'.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get notification_action value
my $notification_action = $object->notification_action();

#Modify notification_action value
$object->notification_action('CISCOISE_QUARANTINE');
```

notification_target( )

Use this method to set or retrieve the notification rule target.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

The valid value is an Infoblox::CiscoISE::Endpoint, Infoblox::Notification::REST::Endpoint or Infoblox::DXL::Endpoint object.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get notification_target value
my $notification_target = $object->notification_target();

#Modify notification_target value
$object->notification_target($endpoint);
```

override_publish_settings( )

The override_publish_settings attribute controls whether the publish_settings method values of the member are used, instead of the target endpoint default.

The override_publish_settings attribute can be specified explicitly. It is also set implicitly when publish_settings is set to a defined value.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

Set the parameter to "true" to override the endpoint level setting for publish_settings. Set the parameter to "false" to inherit the endpoint level setting for publish_settings.

The default value of this parameter is "false".

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get override_publish_settings value
my $override_publish_settings = $object->override_publish_settings( );
#Modify override_publish_settings value
$object->override_publish_settings("true");
```

publish_settings( )

Use this method to set or retrieve the notification rule publish settings.

Setting this method to a defined value implicitly sets the override_publish_settings method to "true". Setting the parameter to undefined causes the appliance to use the endpoint default and automatically resets the override_publish_settings method to "false".

Note that when publish_settings is set to a defined value and override_publish_settings is set to "false", the last operation takes precedence. Thus the sequence $object->publish_settings($value); $object->override_publish_settings("false"); will set override_publish_settings to "false", and the sequence $object->override_publish_settings("false"); $object->publish_settings($value); will result in override_publish_settings="true".

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

The valid value is an Infoblox::CiscoISE::PublishSetting object.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get publish_settings value
my $publish_settings = $object->publish_settings();

#Modify publish_settings value
$object->publish_settings($publish_settings);
```

selected_members( )

Use this method to set or retrieve the member host names on which you run the publish engine.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

The valid value is an array of desired selected member host names in string format.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get selected_members value
my $selected_members = $object->selected_members();

#Modify selected_members value
$object->selected_members(['member1.com', 'member2.com']);
```

template_instance( )

Use this method to set or retrieve the REST API template instance.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

The valid value is an Infoblox::Notification::REST::TemplateInstance object.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get template_instance value
my $template_instance = $object->template_instance();

#Modify template_instance value
$object->template_instance($instance);
```

AUTHOR

Infoblox Inc. http://www.infoblox.com/

SEE ALSO

Infoblox::Session, Infoblox::Session->add(), Infoblox::Session->get(), Infoblox::Session->modify(), Infoblox::Session->remove(), Infoblox::Session->search(), Infoblox::CiscoISE::PublishSetting, Infoblox::CiscoISE::Endpoint, Infoblox::DXL::Endpoint, Infoblox::Notification::RuleExpressionOp, Infoblox::Notification::REST::Endpoint, Infoblox::Notification::REST::TemplateInstance,

---

COPYRIGHT

Copyright (c) 2017 Infoblox Inc.

Infoblox::CiscoISE::EAAssociation - A Cisco ISE extensible attributes association.

- NAME
- DESCRIPTION
- CONSTRUCTOR
- SESSION METHODS
- MODULE METHODS
  - Infoblox::CiscoISE::SubscribeSetting->mapped_ea_attributes( )
- METHODS
  - name( )
  - mapped_ea( )
- AUTHOR
- SEE ALSO
- COPYRIGHT

NAME

Infoblox::CiscoISE::EAAssociation - A Cisco ISE extensible attributes association.

DESCRIPTION

The Cisco ISE extensible attribute association provides information on NIOS extensible attributes mapping on Cisco ISE attributes.

CONSTRUCTOR

```
my $ea_assoc = Infoblox::CiscoISE::EAAssociation->new(
    name      => 'ACCOUNT_SESSION_ID' | 'AUDIT_SESSION_ID' |
                 'EPS_STATUS' | 'IP_ADDRESS' |'MAC' |'NAS_IP_ADDRESS' | 'NAS_PORT_ID' |
                 'POSTURE_STATUS' |'POSTURE_TIMESTAMP',                                  #Required
    mapped_ea => $string,                                                                #Required
```

SESSION METHODS

The object does not have any session methods.

MODULE METHODS

This section describes all the functions that you can apply to Cisco ISE extensible attributes association object.

Infoblox::CiscoISE::SubscribeSetting->mapped_ea_attributes( )

Use this method to set or retrieve the list of extensible attribute associations that are mapped to Cisco ISE attributes. See Infoblox::CiscoISE::SubscribeSetting->mapped_ea_attributes() for parameters and return values.

Example

```
#Get mapped_ea_attributes value
my $mapped_ea_attributes = $object->mapped_ea_attributes();

#Modify mapped_ea_attributes value
$object->mapped_ea_attributes([$ea_assoc1, $ea_assoc2]);
```

METHODS

This section describes all the methods that you can use to configure and retrieve the attribute values of a Cisco ISE extensible attribute association object.

name( )

Use this method to set or retrieve the Cisco ISE attribute name.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

The valid values are 'ACCOUNT_SESSION_ID', 'AUDIT_SESSION_ID', 'EPS_STATUS', 'IP_ADDRESS', 'MAC', 'NAS_IP_ADDRESS', 'NAS_PORT_ID', 'POSTURE_STATUS' and 'POSTURE_TIMESTAMP',

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get name value
my $name = $object->name();

#Modify name value
$object->name('EPS_STATUS');
```

mapped_ea( )

Use this method to set or retrieve the NIOS extensible attribute definition name.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

The valid value is a desired extensible attribute definition name in string format.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get mapped_ea value
my $mapped_ea = $object->mapped_ea();

#Modify mapped_ea value
$object->mapped_ea('nios_ead');
```

AUTHOR

Infoblox Inc. http://www.infoblox.com/

SEE ALSO

Infoblox::CiscoISE::SubscribeSetting, Infoblox::CiscoISE::SubscribeSetting->mapped_ea_attributes(),

---

COPYRIGHT

Copyright (c) 2017 Infoblox Inc.

Infoblox::DNS::AllRecords - A synthetic object used to return all record object types belonging to a zone.

- NAME
- DESCRIPTION
- SESSION METHODS
  - Infoblox::Session->search( )
- METHODS
  - comment( )
  - creator( )
  - ddns_principal( )
  - ddns_protected( )
  - disable( )
  - dtc_obscured( )
  - last_queried( )
  - name( )
  - reclaimable( )
  - record( )
  - ttl( )
  - type( )
  - view( )
  - zone( )
- SAMPLE CODE
- AUTHOR
- SEE ALSO
- COPYRIGHT

NAME

Infoblox::DNS::AllRecords - A synthetic object used to return all record object types belonging to a zone.

DESCRIPTION

By searching on this object, all the record objects associated with a specific zone can be retrieved

---

SESSION METHODS

This section describes all the methods in Infoblox::Session module that you can apply to a DNS AllRecords object.

Infoblox::Session->search( )

Use this method to search for record objects in the Infoblox appliance. See Infoblox::Session->search() for parameters and return values.

Key References

```
 Apply the following attributes to search for a DNS AllRecords object:

  zone           - Required. A zone name in FQDN format.
  creator        - Optional. The creator of the record. The valid values are 'SYSTEM', 'DYNAMIC' and
'STATIC'.
  ddns_principal - Optional. The GSS-TSIG principal that owns the record.
  view           - Optional. The DNS view in which the records are located. By default, the default
DNS views is searched.
  name           - Optional. The name of the matched record. (regular expression).
  reclaimable    - Optional. The flag that indicates whether the record is reclaimable or not.
  type           - Optional. The type of record to search for. By default, all record types are
searched. Following is the list of valid values for this search field:

                 'All A Records',
                 'All AAAA Records',
                 'All Alias Records',
                 'All Bulk Host',
                 'All CNAME Records',
                 'All DNAME Records',
                 'All DNSKEY Records',
                 'All DS Records',
                 'All DTCLBDN Records',
                 'All Host Address',
                 'All Host Alias',
                 'All Host Records',
                 'All IPV6 Host Address',
                 'All LBDN Records',
                 'All MX Records',
                 'All NAPTR Records',
                 'All NS Records',
                 'All NSEC Records',
                 'All NSEC3 Parameter Records',
```

```
                    'All NSEC3 Records',
                    'All PTR Records',
                    'All RSIG Records',
                    'All Records',
                    'All SEC Records',
                    'All SOA Records',
                    'All SRV Records',
                    'All TLSA Records',
                    'All Shared A Records',
                    'All Shared AAAA Records',
                    'All Shared MX Records',
                    'All Shared SRV Records',
                    'All Shared TXT Records',
                    'All TXT Records',
```

Example

```
# search for all records in the "domain.com" zone in the default DNS view
my @retrieved_objs = $session->search(
    object => "Infoblox::DNS::AllRecords",
    zone   => 'domain.com',
);

# search for all TXT records that match the 'rec' name in the "domain.com" zone in the default DNS
view
my @retrieved_objs = $session->search(
    object => "Infoblox::DNS::AllRecords",
    zone   => 'domain.com',
    type   => 'All TXT Records',
    name   => 'rec',
);
```

METHODS

This section describes all the methods that you can use to retrieve the attribute values of an AllRecords object. All methods are read-only and cannot be set.

comment( )

Use this method to retrieve the descriptive comment of the record associated with the AllRecords object.

Returns

The method returns the attribute value.

Example

```
#Get comment
my $comment = $allrecord->comment();
```

creator( )

Use this method to retrieve the resource record creator.

Returns

The method returns the attribute value.

Example

```
#Get creator
my $creator = $allrecord->creator();
```

ddns_principal( )

Use this method to retrieve the GSS-TSIG principal that owns the record.

Returns

The method returns the attribute value.

Example

```
#Get ddns_principal
my $ddns_principal = $allrecord->ddns_principal();
```

ddns_protected( )

Use this method to retrieve the flag that indicates whether this record is protected from DDNS updates or not.

Returns

The method returns the attribute value.

Example

```
#Get ddns_protected
my $ddns_protected = $allrecord->ddns_protected();
```

disable( )

Use this method to retrieve the disable flag of of the record associated with the AllRecords object (if present).

Returns

The method returns the attribute value.

Example

```
#Get disable
my $disable = $allrecord->disable();
```

dtc_obscured( )

Use this method to retrieve the LBDN object that obscures this record. This is a read-only attribute.

Parameter

None

Returns

The method returns attribute value.

Example

```
#Get dtc_obscured
my $dtc_obscured = $allrecord->dtc_obscured();
```

last_queried( )

Use this method to retrieve the time the associated record was last queried on.

Returns

The method returns the attribute value, the number of seconds that have elapsed since January 1st, 1970 UTC.

Example

```
#Get last_queried
my $last_queried = $allrecord->last_queried();
```

name( )

Use this method to retrieve the name of of the record associated with the AllRecords object. Note that this value could differ from the value of the associated record's name field.

Returns

The method returns the attribute value.

Example

```
#Get name
my $name = $allrecord->name();
```

reclaimable( )

Use this method to retrieve the flag that indicates whether the record is reclaimable or not.

Parameter

None

Returns

The method returns the attribute value.

Example

```
#Get reclaimable
my $reclaimable = $object->reclaimable();
```

record( )

Use this method to retrieve the record object associated with the AllRecords object.

This object is considered a 'partial' object. See Infoblox::Session->fill_partial_object() for more information on how it can be converted to a 'full' API object.

If the AllRecords object refers to a record not currently supported in the Perl API, the member value will be undefined.

Returns

The method returns the attribute value.

Example

```
#Get record
my $record = $allrecord->record();
```

ttl( )

Use this method to retrieve the TTL value of of the record associated with the AllRecords object (if present).

Returns

The method returns the attribute value.

Example

```
#Get ttl
my $ttl = $allrecord->ttl();
```

type( )

Use this method to retrieve the type of the record associated with the AllRecords object. This is a descriptive string that identifies the record to which this AllRecords object refers. (For example, 'A record', 'MX record', ...)

Returns

The method returns the attribute value.

Example

```
#Get type
my $type = $allrecord->type();
```

view( )

Use this method to retrieve the DNS view name of the record associated with the AllRecords object.

Returns

The method returns the attribute value.

Example

```
#Get view
my $view = $allrecord->view();
```

zone( )

Use this method to retrieve the zone name of the record associated with the AllRecords object.

Returns

The method returns the attribute value.

Example

```
#Get zone
my $zone = $allrecord->zone();
```

SAMPLE CODE

```
 In cases where a large number of matching records exist on the appliance, only a limited number will be
returned
 at a time. If all of the matching records are required, please consider using the paging support via
Infoblox::Cursor.

 #Code sample fragment Starts:

    $cursor=$session->new_cursor(
                                 fetch_size => 100,
                                 object => "Infoblox::DNS::AllRecords",
                                 zone   => 'domain.com',
                                 view   => 'default',
                                );

    while ($current_record = $cursor->fetch()) {
        # $current_record will contain a matching AllRecords object
    }

    # All matching AllRecords will be returned
    # when the cycle ends.

 If you want full record objects for some of the above, you can add the following code

 #Code sample fragment Starts:

     if( ... some criteria ...) {
         push @temp, $current_record->record();
     }

 ####Code Fragment Ends####

 inside the while() cycle and afterwards call

 #Code sample fragment Starts:

 @full_record_objects=$session->fill_partial_object(\@temp);

 ####Code Fragment Ends####

 to fill the wanted objects.
```

AUTHOR

Infoblox Inc. http://www.infoblox.com/

SEE ALSO

Infoblox::Session, Infoblox::Session->search()/>

COPYRIGHT

Copyright (c) 2017 Infoblox Inc.

Infoblox::DNS::Zone - DNS Zone object

- NAME
- DESCRIPTION
- CONSTRUCTOR
- SESSION METHODS
  - Infoblox::Session->add( )
  - Infoblox::Session->get( )
  - Infoblox::Session->modify( )
  - Infoblox::Session->remove( )
  - Infoblox::Session->search( )
- METHODS
  - ad_servers( )
  - allow_gss_tsig_for_underscore_zone( )
  - allow_gss_tsig_zone_updates( )
  - allow_ptr_creation_in_parent( )
  - allow_query( )
  - allow_transfer( )
  - allow_update( )
  - allow_update_forwarding( )
  - bind_check_names_policy( )
  - comment( )
  - cloud_info( )
  - convert_a_host_ptr_to_bulk_host( )
  - copy_xfer_to_notify( )
  - create_ptr_for_bulk_hosts( )
  - create_ptr_for_hosts( )
  - create_underscore_zones( )
  - ddns_force_creation_timestamp_update( )
  - ddns_restrict_patterns_list( )
  - ddns_restrict_patterns( )
  - ddns_restrict_static( )
  - ddns_restrict_protected( )
  - ddns_restrict_secure( )
  - ddns_principal_tracking( )
  - ddns_principal_group( )
  - delegate_to( )
  - delegated_ttl( )
  - delegation_ns_group( )
  - disable( )
  - disable_forwarding( )
  - dnssec_enable_ksk_auto_rollover( )
  - dnssec_keys( )
  - dnssec_ksk_email_notification_enabled( )
  - dnssec_ksk_rollover_date( )
  - dnssec_ksk_rollover_notification_config( )
  - dnssec_ksk_snmp_notification_enabled( )
  - dns_email( )
  - dns_integrity_check_enable( )
  - dns_integrity_frequency( )
  - dns_integrity_member( )
  - dns_integrity_verbose_logging( )
  - dns_name( )
  - dns_soa_mname( )
  - dnssec_ksk_algorithms( )
  - dnssec_ksk_rollover_interval()
  - dnssec_nsec3_iterations( )
  - dnssec_nsec3_salt_min_length( )
  - dnssec_nsec3_salt_max_length( )
  - dnssec_resign( )
  - dnssec_signature_expiration( )
  - dnssec_signed( )
  - dnssec_zsk_algorithms( )
  - dnssec_zsk_rollover_date( )
  - dnssec_zsk_rollover_interval()
  - dnssec_zsk_rollover_mechanism( )
  - do_host_abstraction( )
  - enable_ad_servers( )
  - email( )
  - execute_dns_parent_check( )
  - extattrs( )
  - extensible_attributes( )
  - fireeye_rule_mapping( )

- forward_external_ns_group( )
- forward_ns_group( )
- forward_only( )
- forward_to( )
- host_name_restriction_policy( )
- import_from( )
- initiate_ksk_rollover( )
- initiate_zsk_rollover( )
- is_default( )
- is_multimaster( )
- last_queried( )
- rpz_drop_ip_rule_enabled( )
- rpz_drop_ip_rule_min_prefix_length_ipv4( )
- rpz_drop_ip_rule_min_prefix_length_ipv6( )
- rpz_policy( )
- restart_if_needed( )
- rpz_type( )
- rpz_severity( )
- substitute_name( )
- rpz_priority( )
- rpz_last_updated_time( )
- rpz_severity( )
- is_feed_zone( )
- locked( )
- members( )
- member_soa_serials( )
- member_soa_mnames( )
- ms_dc_ns_record_creation( )
- name( )
- network_associations( )
- next_secure_type( )
- notify_delay( )
- `ms_ad_integrated()`
- `ms_managed()`
- `ms_read_only()`
- `ms_ddns_mode()`
- `ms_allow_transfer()`
- ms_sync_disable( )
- ns_group( )
- override_copy_xfer_to_notify( )
- override_ddns_force_creation_timestamp_update( )
- override_ddns_patterns_restriction( )
- override_ddns_restrict_static( )
- override_ddns_restrict_protected( )
- override_ddns_principal_security( )
- override_delegated_ttl( )
- override_grid_zone_timer( )
- override_grid_email( )
- override_scavenging_settings( )
- override_rpz_drop_ip_rule( )
- override_serial_number( )
- prefix( )
- multiple_primaries( )
- primary_shared_with_ms_parent_delegation( )
- scavenging_settings( )
- remove_sub_zones( )
- secondaries( )
- resource_record_monitored_since( )
- soa_default_ttl( )
- soa_expire( )
- soa_mname( )
- soa_negative_ttl( )
- soa_refresh( )
- soa_retry( )
- soa_serial_number( )
- shared_record_groups( )
- stub_external_ns_group( )
- stub_from( )
- stub_members( )
- stub_ns_group( )
- views( )
- zone( )
- zone_monitored_since( )

- SAMPLE CODE

- AUTHOR
- SEE ALSO
- COPYRIGHT

NAME

Infoblox::DNS::Zone - DNS Zone object

DESCRIPTION

A zone is a portion of the domain name space for which an Infoblox appliance or another name server is authoritative (for example, has the start of authority [SOA] record). A zone can also be delegated or forwarded. Zones are the primary objects used to manage DNS data and DNS services.

Note that you must specify only one view for the attribute "views".

CONSTRUCTOR

```
 my $zone = Infoblox::DNS::Zone->new(
     name                                    => $fqdn | $ipv4_subnet/prefix | $ipv6_subnet/prefix,
# Required
     ad_servers                              => [ $ipv4addr | $ipv6addr ],
# Optional / Default is empty list
     allow_gss_tsig_for_underscore_zone      => "true" | "false",
# Optional / Default is "false"
     allow_gss_tsig_zone_updates             => "true" | "false",
# Optional / Default is "false"
     allow_query                             => [ $ipv4addr | $ipv4_subnet/prefix | $ipv6addr |
                                                  $ipv6_subnet/prefix | $TSIGKey ,... ] | $nacl |
undef,                  # Optional / Default is undefined
     allow_transfer                          => [ $ipv4addr | $ipv4_subnet/prefix | $ipv6addr |
                                                  $ipv6_subnet/prefix | $TSIGKey ,... ] | $nacl |
undef,                  # Optional / Default is undefined
     allow_update                            => [ $ipv4addr | $ipv4_subnet/prefix | $ipv6addr |
                                                  $ipv6_subnet/prefix | $TSIGKey ,... ] | $nacl |
undef,                  # Optional / Default is undefined
     allow_update_forwarding                 => [ $ipv4addr | $ipv4_subnet/prefix | $ipv6addr |
                                                  $ipv6_subnet/prefix | $TSIGKey ,... ] | $nacl |
undef,                  # Optional / Default is undefined
     bind_check_names_policy                 => [ "fail" | "warn" | undef],
# Optional / Default is undefined
     comment                                 => $string,
# Optional / Default is undefined
     create_underscore_zones                 => "true" | "false",
# Optional / Default is "false"
     delegate_to                             => [ $Nameserver1, $Nameserver2 ... ],
# Optional / Default is undefined
     delegated_ttl                           => $num,
# Optional / Default is undefined
     delegation_ns_group                     => $string | $delegation_ns_group
# Optional / Default is undefined
     disable                                 => "true" | "false",
# Optional / Default is "false"
     disable_forwarding                      => "true" | "false",
# Optional / Default is "false"
     dnssec_enable_ksk_auto_rollover         => "true" | "false",
# Optional / Default is "false"
     dnssec_keys                             => $keys,
# Optional / Default is empty list
     dnssec_zsk_algorithms                   => [$algorithm1, $algorithm2, ...],
# Optional / Default is undefined
     dnssec_ksk_email_notifiaction_enabled   => "true" | "false",
# Optional / Default is false
     dnssec_ksk_rollover_date                => $date,
# Optional / Default is undefined
     dnssec_ksk_rollover_interval            => $num,
# Optional / Default is undefined
     dnssec_ksk_rollover_notification_config => "NONE" | "ALL" | "REQUIRE_MANUAL_INTERVENTION",
# Optional / Default is undefined
     dnssec_ksk_email_notification_enabled   => "true" | "false",
# Optional / Default is "false"
     dnssec_ksk_snmp_notification_enabled    => "true" | "false",
# Optional / Default is "false"
     dnssec_nsec3_iterations                 => $num,
# Optional / Default is undefined
     dnssec_nsec3_salt_min_length            => $num,
# Optional / Default is undefined
                                                $num
```

```
      dnssec_nsec3_salt_max_length                => ,
# Optional / Default is undefined
      dnssec_signature_expiration                 => $num,
# Optional / Default is undefined
      dnssec_signed                               => "true" | "false",
# Optional / Default is undefined
      dnssec_zsk_algorithms                       => [$algorithm1, $algorithm2, ...],
# Optional / Default is undefined
      dnssec_zsk_rollover_date                    => $date,
# Optional / Default is undefined
      dnssec_zsk_rollover_mechanism               => "DOUBLE_SIGN" | "PRE_PUBLISH",
# Optional / Default is undefined
      dnssec_zsk_rollover_interval                => $num,
# Optional / Default is undefined
      next_secure_type                            => "NSEC" | "NSEC3",
# Optional / Default is undefined
      enable_ad_servers                           => "true" | "false",
# Optional / Default is "false"
      allow_ptr_creation_in_parent                => "true" | "false",
# Optional / Default is "false"
      email                                       => $email,
# Optional / Default is undefined
      extattrs                                    => { $string => $extattr, ... },
# Optional / Default is undefined
      extensible_attributes                       => { $string => $string | $num, $string => [ $string |
$num, ... ], ... }    # Optional / Default is undefined
      fireeye_rule_mapping                        => $rulemap,
# Optional / Default is undefined
      forward_only                                => "true" | "false",
# Optional / Default is "false"
      forward_to                                  => [ $Member1, $Member2, $Nameserver1,
$Nameserver2,...],                    # Optional / Default is undefined
      forward_ns_group                            => $string | $forward_ns_group
# Optional / Default is undefined
      forward_external_ns_group                   => $string
# Optional / Default is undefined
      host_name_restriction_policy                => $string,
# Optional / Default is undefined
      import_from                                 => $ipv4addr,
# Optional / Default is undefined
      is_default                                  => "true" | "false",
# Optional / Default is "false"
      is_multimaster                              => "true" | "false",
# Optional / Default is "false"
      locked                                      => "true" | "false",
# Optional / Default is "false"
      members                                     => [ $Member1, $Member2,...],
# Optional / Default is undefined
      member_soa_mnames                           => [$soa_mname1, $soa_mname2, ...],
# Optional / Default is undefined
      member_soa_serials                          => [$soa_serial1, $soa_serial2, ...],
# Optional / Default is undefined
      ms_ad_integrated                            => "true" | "false"
# Optional / Default is "false"
      ms_allow_transfer                           => "Any" | "NSOnly" | "None" | [ $ipv4addr1, ... ]
# Optional / Default is "NSOnly"
      ms_ddns_mode                                => "Secure" | "None" | "Any"
# Optional / Default is undefined
      ms_sync_disable                             => "true" | "false"
# Optional / Default is "false"
      notify_delay                                => $num,
# Optional / Default is undefined
      multiple_primaries                          => [$Member1, ...] | [$Nameserver1, ...] | [$MSServer1,
...],                 # Optional / Default is undefined
      ns_group                                    => $string | $ns_group
# Optional / Default is undefined
      override_delegated_ttl                      => "true" | "false",
# Optional / Default is "false"
      override_grid_zone_timer                    => "true" | "false",
# Optional / Default is "false"
      override_grid_email                         => "true" | "false",
# Optional / Default is "false"
      override_serial_number                      => "true" | undef,
# Optional / Default is undef
      prefix                                      => $string,
# Optional / Default is undefined
      scavenging_settings                         => $scavenging_settings,
# Optional / Default is undefined
      remove_sub_zones                            => "true" | "false",
# Optional / Default is undefined
      rpz_type                                    => "LOCAL" | "FIREEYE" | "FEED",
# Optional / Default is undefined
      rpz_severity                                => "CRITICAL" | "MAJOR" | "WARNING" | "INFORMATIONAL",
```

```
  # Optional / Default is undefined
      secondaries                                => [ $Member1, $Member2, $Nameserver1, $MSServer2, ...],
  # Optional / Default is empty
      soa_default_ttl                            => $num,
  # Optional / Default is undefined
      soa_expire                                 => $num,
  # Optional / Default is undefined
      soa_mname                                  => $fqdn,
  # Optional / Default is undefined
      soa_negative_ttl                           => $num,
  # Optional / Default is undefined
      soa_retry                                  => $num,
  # Optional / Default is undefined
      soa_refresh                                => $num,
  # Optional / Default is undefined
      soa_serial_number                          => $num,
  # Optional / Default is 1
      shared_record_groups                       => [ $shared_record_group1_name, ... ],
  # Optional / Default is undefined
      stub_from                                  => [ $Nameserver1, $Nameserver2, ... ],
  # Optional / Default is undefined
      stub_members                               => [ $Member1, $Member2, $MSServer1, $MSServer2, ...],
  # Optional / Default is undefined
      stub_ns_group                              => $string | $stub_ns_group
  # Optional / Default is undefined
      stub_external_ns_group                     => $string
  # Optional / Default is undefined
      views                                      => [$View1],
  # Optional / Default is "default" view
      rpz_policy                                 => "GIVEN" | "PASSTHRU" | "NXDOMIAN" | "NODATA" |
  "SUBSTITUTE" | "DISABLED", # Optional / Default is undefined
      substitute_name                            => $string,
  # Optional / Default is empty
      copy_xfer_to_notify                        => "true" | "false",
  # Optional / Default is "false"
      override_copy_xfer_to_notify               => "true" | "false",
  # Optional / Default is "false"
      dns_integrity_check_enable                 => "true" | "false",
  # Optional / Default is "false"
      dns_integrity_frequency                    => $uint,
  # Optional / Default is undefined
      dns_integrity_member                       => $hostname,
  # Optional / Default is undefined
      dns_integrity_verbose_logging              => "true" | "false",
  # Optional / Default is undefined
      ddns_restrict_patterns_list                => [$string, $string, ...],
  # Optional / Default is undefined
      ddns_restrict_patterns                     => "true" | "false",
  # Optional / Default is 'false'
      ddns_restrict_static                       => "true" | "false",
  # Optional / Default is 'false'
      ddns_restrict_protected                    => "true" | "false",
  # Optional / Default is 'false'
      ddns_restrict_secure                       => "true" | "false",
  # Optional / Default is 'false'
      ddns_principal_tracking                    => "true" | "false",
  # Optional / Default is 'false'
      ddns_principal_group                       => $principal_group,
  # Optional / Default is undefined,
      ddns_force_creation_timestamp_update       => "true" | "false",
  # Optional / Default is 'false'
      override_ddns_patterns_restriction         => "true" | "false",
  # Optional / Default is 'false'
      override_ddns_restrict_static              => "true" | "false",
  # Optional / Default is 'false'
      override_ddns_restrict_protected           => "true" | "false",
  # Optional / Default is 'false'
      override_ddns_principal_security           => "true" | "false",
  # Optional / Default is 'false'
      override_ddns_force_creation_timestamp_update => "true" | "false",
  # Optional / Default is 'false'
      override_scavenging_settings               => "true" | "false",
  # Optional / Default is 'false'
      override_rpz_drop_ip_rule                  => "true" | "false" | undef,
  # Optional / Default is "false"
      rpz_drop_ip_rule_enabled                   => "true" | "false" | undef,
  # Optional / Default is "false"
      rpz_drop_ip_rule_min_prefix_length_ipv4    => $uint | undef,
  # Optional / Default is 29
      rpz_drop_ip_rule_min_prefix_length_ipv6    => $uint | undef,
  # Optional / Default is 112
      ms_dc_ns_record_creation                   => $msdcnsrecordcreation,
  # Optional / Default is empty list
   );
```

You cannot set both extattrs and extensible_attributes attributes at the same time.

SESSION METHODS

This section describes all the methods in the Infoblox::Session module that you can apply to a DNS Zone object.

Infoblox::Session->add( )

Use this method to add an object to the Infoblox appliance. See Infoblox::Session->add() for parameters and return values.

If you have applied an access control list to an object, you must restart services after you update the access control list for the changes to take effect. See Infoblox::Session->restart() method.

Example

```
#Construct an object
my $zone = Infoblox::DNS::Zone->new (
    name     => "domain.com",
    comment  => "this is a demo zone",
    disable  => "false",
    views    => [$view1]
    );
#Submit for addition
my $response = $session->add( $zone );
```

Infoblox::Session->get( )

Use this method to retrieve all the matching objects from the Infoblox appliance. See Infoblox::Session->get() for parameters and return values.

Key References

```
Apply the following attributes to get a specific DNS Zone object:

 name - Required. A zone name in string format.
 view - Optional. The DNS view in which the zone is located. The default is all DNS views.
 extattrs      - Optional. A hash reference containing extensible attributes.
 extensible_attributes - Optional. A hash reference that contains extensible attributes.
 zone - Optional. The parent zone name in string format.
```

Examples

```
my @retrieved_objs = $session->get(
    object => "Infoblox::DNS::Zone",
    name   => "domain.com",
    view   => "default" );

my @retrieved_objs = $session->get(
    object => "Infoblox::DNS::Zone",
    extensible_attributes => { 'Site' => 'Santa Clara' });
```

Infoblox::Session->modify( )

Use this method to modify an object in the Infoblox appliance. See Infoblox::Session->modify() for parameters and return values.

If you have applied an access control list to an object, you must restart services after you update the access control list for the changes to take effect. See Infoblox::Session->restart() method.

Example

```
#Use method to modify the comment.
$zone->comment("this is a modified comment");
#Submit modification
my $response = $session->modify( $zone );
```

Infoblox::Session->remove( )

Use this method to remove an object from the Infoblox appliance. See Infoblox::Session->remove() for parameters and return values.

To remove a specific object, first use `get()` or `search()` to retrieve the specific object, and then submit this object for removal.

Example

```
#Get the objects with the same name
my @retrieved_objs = $session->get(
    object => "Infoblox::DNS::Zone",
```

```
    name   => "domain.com",
    view   => "default" );
#Find the desired object from the retrieved list.
my $desired_zone = $retrieved_objs[0];
#Submit for removal
my $response = $session->remove( $desired_zone );
```

Infoblox::Session->search( )

Use this method to search for DNS Zone objects in the Infoblox appliance. See Infoblox::Session->search() for parameters and return values.

Key References

```
Apply the following attributes to search for a DNS Zone object:

 name - Required. A host name in string format (regular expression).
 view - Optional. The DNS view in which the zone is located. The default is all DNS views.
 extattrs     - Optional. A hash reference containing extensible attributes.
 extensible_attributes - Optional. A hash reference containing extensible attributes.
 comment - Optional. A comment in string format (regular expression).
 primary_association_type - Optional. "Microsoft" is the only allowed value. If it is specified, the
search returns only zones with primary servers that are Microsoft (r) DNS servers.
 dnssec_ksk_rollover_date - Optional. Value is the number of seconds that have elapsed since January
1st, 1970 UTC for the next KSK rollover.
 dnssec_zsk_rollover_date - Optional. Value is the number of seconds that have elapsed since January
1st, 1970 UTC for the next ZSK rollover.
 zone - Optional. The parent zone name in string format (regular expression).
```

For more information about searching extensible attributes, see Infoblox::Grid::ExtensibleAttributeDef/Searching Extensible Attributes.

Examples

```
# search for all DNS zone objects under com in default view
my @retrieved_objs = $session->search(
    object => "Infoblox::DNS::Zone",
    name   => ".*\\.com",
    view   => "default" );

# search for all DNS zone objects defining "Santa Clara" for "Site" extensible attribute
my @retrieved_objs = $session->get(
    object => "Infoblox::DNS::Zone",
    extensible_attributes => { 'Site' => 'Santa Clara' });
```

METHODS

This section describes all the methods that you can use to configure and retrieve the attribute values of a zone.

ad_servers( )

Use this method to set or retrieve the list of AD domain controllers that are allowed to send GSS-TSIG unsigned updates.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

The valid value is an array reference that contains IPv4 or IPv6 addresses.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
# Get ad_servers
my $ad_servers = $zone->ad_servers();
# Modify ad_servers
$zone->ad_servers(["1.2.3.4", "dead::beef"]);
```

allow_gss_tsig_for_underscore_zone( )

Use this method to set or retrieve the flag that allows DHCP clients to perform GSS-TSIG signed updates for underscore zones.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

Specify "true" to set the disable flag or "false" to deactivate/unset it. The default value is "false".

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
# Get allow_gss_tsig_for_underscore_zone.
my $allow_gss_tsig_for_underscore_zone = $zone->allow_gss_tsig_for_underscore_zone();
# Modify allow_gss_tsig_for_underscore_zone.
$zone->allow_gss_tsig_for_underscore_zone("true");
```

allow_gss_tsig_zone_updates( )

Use this method to enable/disable the zone to accept GSS-TSIG updates. This setting overrides the grid and member GSS-TSIG zone update settings.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

Specify "true" to set the disable flag or "false" to deactivate/unset it. The default value is "false".

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
# Get allow_gss_tsig_zone_updates.
my $allow_gss_tsig_zone_updates = $zone->allow_gss_tsig_zone_updates();
# Modify allow_gss_tsig_zone_updates.
$zone->allow_gss_tsig_zone_updates("true");
```

allow_ptr_creation_in_parent( )

This flag controls whether automatic generation of RFC 2317 CNAMEs for delegated reverse zones overwrite existing PTR records. The default behavior is to overwrite all the existing records in the range, this corresponds to "allow_ptr_creation_in_parent" set to "false". However, when this flag is set to "true" the existing PTR records are not overwritten.

Parameter

Specify "true" to set the flag or "false" to deactivate/unset it. The default value is "false".

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
# Get allow_ptr_creation_in_parent.
my $allow_ptr_creation_in_parent = $zone->allow_ptr_creation_in_parent();
# Modify allow_ptr_creation_in_parent.
$zone->allow_ptr_creation_in_parent("true");
```

allow_query( )

Use this method to set or retrieve the allow_query flag of a DNS zone.

You can allow or reject queries from specified IP addresses and networks. You can also specify TSIG keys for the allowed hosts. This setting overrides the member-level query settings.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

The valid value is an Infoblox::Grid::NamedACL object or an array reference that contains IPv4/IPv6 addresses, networks, and/or Infoblox::DNS::TSIGKey objects. To deny queries from certain IP addresses and networks, add "!" as a prefix to the IP addresses and networks. Specify "any" to allow any query, or undefine to use the Grid::Member::DNS level allow_query list. The default value is undefined.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Getting allow_query
my $allow_query = $zone->allow_query();
#Modifying allow_query
$zone->allow_query(["any"]);

#Specifying TSIG keys
my $tsig_key1 = $session->gen_tsig_key();
my $query_key = Infoblox::DNS::TSIGKey->new(
   name => "query_key1",
   key  => $tsig_key1
);
#Modifying allow_query
$zone->allow_query( [ "10.0.0.10" , "20.0.0.0/24","2001::123", $key1,
"2004::444/64","!"."30.0.0.30" , "!"."40.10.0.0/24","!"."2005::222/64"] );
##Un-override allow_query
$zone->allow_query(undef)
#Use Infoblox::Grid::NamedACL
$zone->allow_query($nacl);
```

allow_transfer( )

Use this method to set or retrieve the allow_transfer list of a DNS zone.

You can allow or reject zone transfers from specified IP addresses and networks. This setting overrides the member-level zone transfer settings. You can also provide Transaction Signature (TSIG) keys for clients that are allowed to do zone transfers.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

The valid value is an Infoblox::Grid::NamedACL object or an array reference that contains IPv4/IPv6 addresses and networks. It is also possible to specify the TSIG keys of clients that are allowed to perform zone transfers. TSIG keys are used to authenticate zone transfer requests and replies. The same key name and key value must be on the primary and secondary name servers for TSIG-authenticated zone transfers to occur. To deny zone transfers from certain IP addresses and networks, add "!" as a prefix to the IP addresses and networks. Specify "any" to allow any transfer, or undefine to use the Grid::Member::DNS level allow_transfer list. The default value is undefined.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Getting allow_transfer
my $allow_transfer = $zone->allow_transfer( );
#Modifying allow_transfer
$zone->allow_transfer(["any"]);

#Specifying TSIG keys
my $tsig_key1 = $session->gen_tsig_key();
my $key1 = Infoblox::DNS::TSIGKey->new(
   name => "key1",
   key  => $tsig_key1
);
my $key2 = Infoblox::DNS::TSIGKey->new(
   name => "tsig_xfer",
   key  => ":2xCOMPAT"
   );
#Modifying allow_transfer
$zone->allow_transfer(["10.0.0.10", "20.0.0.0/24","2001::1234", $key1, "2001::333/64", $key2,
"!"."30.0.0.30", "!"."40.10.0.0/24","!"."2008::543","!"."2007::456/64"]);
#Use Grid::Member::DNS level allow_transfer list
$zone->allow_transfer(undef)
#Use Infoblox::Grid::NamedACL
$zone->allow_transfer($nacl);
```

allow_update( )

Use this method to set or retrieve the allow_update flag of a DNS zone.

You can allow or deny dynamic updates from specified IP addresses and networks. This setting overrides member-level update settings. You can also provide TSIG keys for clients that are allowed to do zone updates.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

The valid value is an Infoblox::Grid::NamedACL object or an array reference that contains IPv4/IPv6 addresses, networks and/or Infoblox::DNS::TSIGKey objects. To deny updates from certain IP addresses and networks, add "!" as a prefix to the IP addresses and networks. Specify "any" to allow any update, or undefine to use the Grid::Member::DNS allow_update list. Note: You should also undefine the update_keys list to use Grid::Member::DNS settings. The default value is undefined.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Getting allow_update
my $allow_update = $zone->allow_update();
#Modifying allow_update
$zone->allow_update(["any"]);

#Creating a TSIGKey object.
my $update_key1 = $session->gen_tsig_key();
my $up_key1 = Infoblox::DNS::TSIGKey->new(
   name => "update_key",
   key  => $update_key1
   );

#Modifying allow_update
$zone->allow_update(["10.0.0.10", $up_key1, "20.0.0.0/24","2006::987","2007::777/64",
"!"."30.0.0.30", "!"."40.10.0.0/24"]);

#Use Grid::Member::DNS level allow_update list
$zone->allow_update(undef)
#Use Infoblox:Grid::NamedACL
$zone->allow_update($nacl);
```

allow_update_forwarding( )

Use this method to set or retrieve the allow_update_forwarding flag of a DNS zone.

You can allow or deny dynamic updates forwarded from specified IP addresses and networks. This setting overrides member-level update settings. You can also provide TSIG keys for clients that are allowed to do zone updates.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

The valid value is an Infoblox::Grid::NamedACL object or an array reference that contains IPv4/IPv6 addresses, networks and/or Infoblox::DNS::TSIGKey objects. To deny updates from certain IP addresses and networks, add "!" as a prefix to the IP addresses and networks. Specify "any" to allow any update, or undefine to use the Grid::Member::DNS allow_update list. Note: You should also undefine the update_keys list to use Grid::Member::DNS settings. The default value is undefined.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Getting allow_update_forwarding
my $allow_update_forwarding = $zone->allow_update_forwarding();
#Modifying allow_update_forwarding
$zone->allow_update_forwarding(["any"]);

#Creating a TSIGKey object.
my $update_key1 = $session->gen_tsig_key();
my $up_key1 = Infoblox::DNS::TSIGKey->new(
   name => "update_key",
   key  => $update_key1
   );

#Modifying allow_update_forwarding
```

```
  $zone->allow_update_forwarding(["10.0.0.10", $up_key1,
"20.0.0.0/24","2009::654/64","!"."30.0.0.30", "!"."40.10.0.0/24","!"."2007::432/64"]);

  ##Un-override allow_update_forwarding
  $zone->allow_update_forwarding(undef);
  #Use Infoblox::Grid::NamedACL
  $zone->allow_update_forwarding($nacl);
```

bind_check_names_policy( )

Use this method to set or retrieve the BIND check names policy, which indicates the action the appliance takes when it encounters host names that do not comply with the Strict Hostname Checking policy. This method applies only if the host name restriction policy is set to "Strict Hostname Checking".

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

The allowed values of the parameter are "fail", "warn" or undefined. This indicates the desired BIND behavior in the case of illegal hostnames. The default value is undefined, which means the appliance ignores illegal hostnames.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get bind_check_names_policy value
my $names_policy = $zone->bind_check_names_policy();
#Modify bind_check_names_policy
$zone->bind_check_names_policy("warn");
```

comment( )

Use this method to set or retrieve a descriptive comment about the zone.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

Desired comment in string format with a maximum of 256 bytes.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get comment
my $comment = $zone->comment();
#Modify comment
$zone->comment("Modifying the DNS Zone comment");
```

cloud_info( )

Use this method to retrieve cloud API related information for the Infoblox::DNS::Zone object.

Parameter

None

Returns

The method returns the attribute value.

Example

```
# Get cloud_info
my $cloud_info = $object->cloud_info();
```

convert_a_host_ptr_to_bulk_host( )

Use this method to convert all A, HOST and PTR records that have sequential IP addresses to bulk host records.

The zone object must be retrieved from the server before calling this method.

The method returns true when the modification succeeds, and returns false when the operation fails.

Parameter

The parameter 'create_reverse_mapping' toggles if the method creates records in the matching reverse-mapping zone.

The parameter is set to 'True' if records are created in the reverse-mapping zone. Otherwise, it is set to 'False'.

Returns

Example

```
my %args;
$args{'create_reverse_mapping'} = 'true';
$zone->convert_a_host_ptr_to_bulk_host(\%args);
```

copy_xfer_to_notify( )

Use this method to enable/disable copying of the allowed IPs from zone transfer list into also-notify statement in named.conf.

Parameter

Set the parameter to "true" to enable copying of the allowed IPs from zone transfer list into also-notify statement in named.conf. Set the parameter to "false" to disable it.

The default value of the parameter is "false".

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
# Get attribute value
my $value = $zone->copy_xfer_to_notify();
# Modify attribute value
$zone->copy_xfer_to_notify('true');
```

create_ptr_for_bulk_hosts( )

Use this method to set the create_ptr_for_bulk_hosts property used in zone import operations. This is a write-only attribute.

Set this flag to indicate that Infoblox PTR records will be automatically created for Bulk Host records if necessary when the zone data is imported.

Parameter

Specify "true" to trigger host creation or "false" to disable it. The default value is "false".

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

Example

```
#Modify create_ptr_for_bulk_hosts
$zone->create_ptr_for_bulk_hosts("true");
```

create_ptr_for_hosts( )

Use this method to set the create_ptr_for_hosts property used in zone import operations. This is a write-only attribute.

Set this flag to indicate that Infoblox PTR records will be automatically created for Host records if necessary when the zone data is imported.

Parameter

Specify "true" to trigger host creation or "false" to disable it. The default value is "false".

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

Example

```
#Modify create_ptr_for_bulk_hosts
$zone->create_ptr_for_bulk_hosts("true");
```

create_underscore_zones( )

Use this method to set or retrieve the create_underscore_zones flag of a DNS zone.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

Specify "true" to set the create_underscore_zones flag or "false" to deactivate/unset it. The default value is "false".

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get create_underscore_zones
my $create_underscore_zones = $zone->create_underscore_zones();
#Modify create_underscore_zones
$zone->create_underscore_zones("true");
```

ddns_force_creation_timestamp_update( )

Use this method to set or retrieve the flag that indicates whether the creation timestamp of resource records (RR) should be updated when DDNS update happens even if there is no change to the RR.

Setting this parameter to defined value, implicitly sets the override_ddns_force_creation_timestamp_update attribute to "true". Setting this parameter to undefined causes the appliance to use the Grid-level ddns_force_creation_timestamp_update flag and automatically sets the override_ddns_force_creation_timestamp_update attribute to "false".

Note that when ddns_force_creation_timestamp_update contains a value and override_ddns_force_creation_timestamp_update is set to "false", the last operation takes precedence. Thus the sequence $object->ddns_force_creation_timestamp_update("true"); $object->override_ddns_force_creation_timestamp_update("false"); sets override_ddns_force_creation_timestamp_update to "false", and the sequence $object->override_ddns_force_creation_timestamp_update("false"); $object->ddns_force_creation_timestamp_update("true"); sets override_ddns_force_creation_timestamp_update to "true".

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

Specify 'true' to update the creation timestamp of RR on every DDNS update, and 'false' to update the creation timestamp only when RR changes during a DDNS update. The default is 'false'.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get ddns_force_creation_timestamp_update value
$value = $object->ddns_force_creation_timestamp_update();

#modify ddns_force_creation_timestamp_update value
$object->ddns_force_creation_timestamp_update('true');
```

ddns_restrict_patterns_list( )

Use this method to set or retrieve an unordered list of restriction patterns for an option of to restrict DDNS updates based on FQDN patterns.

Setting this parameter to any list, implicitly sets the override_ddns_patterns_restriction attribute to "true". Setting this parameter to undefined causes the appliance to use the Grid-level DDNS update restrictions and automatically sets the override_ddns_patterns_restriction attribute to "false" if "ddns_restrict_patterns" is also undefined. But if "ddns_restrict_patterns" contains a value, then the override_ddns_patterns_restriction attribute is set to "true".

Note that when ddns_restrict_patterns_list contains a valid array reference and override_ddns_patterns_restriction is set to "false", the last operation takes precedence. Thus the sequence $zone->ddns_restrict_patterns_list([$pattern1, $pattern2]); $zone->override_ddns_patterns_restriction("false"); sets override_ddns_patterns_restriction to "false", and the sequence $zone->override_ddns_patterns_restriction("false"); $zone->ddns_restrict_patterns_list([$pattern1, $pattern2]); sets override_ddns_patterns_restriction to "true".

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

The valid value is an array of FQDNs in a string format.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#get ddns_restrict_patterns_list value
$ddns_restrict_patterns_list = $zone->ddns_restrict_patterns_list();

#modify ddns_restrict_patterns_list value
$zone->ddns_restrict_patterns_list(['foo.com', 'bar.com']);
```

ddns_restrict_patterns( )

Use this method to set or retrieve the flag which indicates whether an option to restrict DDNS update request based on FQDN patterns is enabled or disabled.

Setting this parameter to defined value, implicitly sets the override_ddns_patterns_restriction attribute to "true". Setting this parameter to undefined causes the appliance to use the Grid-level DDNS update restrictions and automatically sets the override_ddns_patterns_restriction attribute to "false" if "ddns_restrict_patterns_list" is also undefined. But if "ddns_restrict_patterns_list" contains a value, then the override_ddns_patterns_restriction attribute is set to "true".

Note that when ddns_restrict_patterns contains a value and override_ddns_patterns_restriction is set to "false", the last operation takes precedence. Thus the sequence $zone->ddns_restrict_patterns("true"); $zone->override_ddns_patterns_restriction("false"); sets override_ddns_patterns_restriction to "false", and the sequence $zone->override_ddns_patterns_restriction("false"); $zone->ddns_restrict_patterns("true"); sets override_ddns_patterns_restriction to "true".

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

Specify 'true' to enable DDNS update request restriction base on FQDN patterns and 'false to disable it. The default value is 'false'.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#get ddns_restrict_patterns value
$ddns_restrict_patterns = $zone->ddns_restrict_patterns();

#modify ddns_restrict_patterns value
$zone->ddns_restrict_patterns('true');
```

ddns_restrict_static( )

Use this method to set or retrieve the flag which indicates whether an option to restrict DDNS update request to resource records which are marked as 'STATIC' is enabled or disabled.

Setting this parameter to defined value, implicitly sets the override_ddns_restrict_static attribute to "true". Setting this parameter to undefined causes the appliance to use the Grid-level ddns_restrict_static flag and automatically sets the override_ddns_restrict_static attribute to "false".

Note that when ddns_restrict_static contains a value and override_ddns_restrict_static is set to "false", the last operation takes precedence. Thus the sequence $zone->ddns_restrict_static("true"); $zone->override_ddns_restrict_static("false"); sets override_ddns_restrict_static to "false", and the sequence $zone->override_ddns_restrict_static("false"); $zone->ddns_restrict_static("true"); sets override_ddns_restrict_static to "true".

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

Specify 'true' to enable DDNS update requests to resource records which are marked as 'STATIC' and 'false' to disable it. The default value is 'false'.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#get ddns_restrict_static value
$ddns_restrict_static = $zone->ddns_restrict_static();

#modify ddns_restrict_static value
$zone->ddns_restrict_static('true');
```

ddns_restrict_protected( )

Use this method to set or retrieve the flag which indicates whether an option to restrict DDNS update request to protected resource records is enabled or disabled.

Setting this parameter to defined value, implicitly sets the override_ddns_restrict_protected attribute to "true". Setting this parameter to undefined causes the appliance to use the Grid-level ddns_restrict_protected flag and automatically sets the override_ddns_restrict_protected attribute to "false".

Note that when ddns_restrict_protected contains a value and override_ddns_restrict_protected is set to "false", the last operation takes precedence. Thus the sequence $zone->ddns_restrict_protected("true"); $zone->override_ddns_restrict_protected("false"); sets override_ddns_restrict_protected to "false", and the sequence $zone->override_ddns_restrict_protected("false"); $zone->ddns_restrict_protected("true"); sets override_ddns_restrict_protected to "true".

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

Specify 'true' to enable DDNS update requests to protected resource records and 'false' to disable it. The default value is 'false'.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#get ddns_restrict_protected value
$ddns_restrict_protected = $zone->ddns_restrict_protected();

#modify ddns_restrict_protected value
$zone->ddns_restrict_protected('true');
```

ddns_restrict_secure( )

Use this method to set or retrieve the flag which indicates whether DDNS update request for principal other than target resource record's principal is restricted.

Setting this parameter to a defined value, implicitly sets the override_ddns_principal_security attribute to "true". Setting this parameter to undefined causes the appliance to use the Grid-level DDNS principal security setting and automatically sets the override_ddns_principal_security attribute to "false" if "ddns_principal_tracking" and "ddns_principal_group" are also undefined. But if "ddns_principal_tracking" or "ddns_principal_group" contains a value, then the override_ddns_principal_security attribute is set to "true".

Note that when ddns_restrict_secure contains a value and override_ddns_principal_security is set to "false", the last operation takes precedence. Thus the sequence $zone->ddns_restrict_secure("true"); $zone->override_ddns_principal_security("false"); sets override_ddns_principal_security to "false", and the sequence $zone->override_ddns_principal_security("false"); $zone->ddns_restrict_secure("true"); sets override_ddns_principal_security to "true".

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

Specify 'true' to enable DDNS update requests restriction for principal other than target resource record's principal and 'false' to disable it. The default value is 'false'.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#get ddns_restrict_secure value
$ddns_restrict_secure = $zone->ddns_restrict_secure();
```

```
#modify ddns_restrict_secure value
$zone->ddns_restrict_secure('true');
```

ddns_principal_tracking( )

Use this method to set or retrieve the flag which indicates whether the DDNS principal track is enabled or disabled.

DDNS principal tracking includes putting principal name into resource record affected by update.

Setting this parameter to a defined value, implicitly sets the override_ddns_principal_security attribute to "true". Setting this parameter to undefined causes the appliance to use the Grid-level DDNS principal security setting and automatically sets the override_ddns_principal_security attribute to "false" if "ddns_restrict_secure" and "ddns_principal_group" are also undefined. But if "ddns_restrict_secure" or "ddns_principal_group" contains a value, then the override_ddns_principal_security attribute is set to "true".

Note that when ddns_principal_tracking contains a value and override_ddns_principal_security is set to "false", the last operation takes precedence. Thus the sequence $zone->ddns_principal_tracking("true"); $zone->override_ddns_principal_security("false"); sets override_ddns_principal_security to "false", and the sequence $zone->override_ddns_principal_security("false"); $zone->ddns_principal_tracking("true"); sets override_ddns_principal_security to "true".

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

Specify 'true' to enable DDNS principal tracking and 'false' to disable it. The default value is 'false'.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#get ddns_principal_tracking value
$ddns_principal_tracking = $zone->ddns_principal_tracking();

#modify ddns_principal_tracking value
$zone->ddns_principal_tracking('true');
```

ddns_principal_group( )

Use this method to set or retrieve the DDNS principal group.

Setting this parameter to a defined value, implicitly sets the override_ddns_principal_security attribute to "true". Setting this parameter to undefined causes the appliance to use the Grid-level DDNS principal security setting and automatically sets the override_ddns_principal_security attribute to "false" if "ddns_restrict_secure" and "ddns_principal_tracking" are also undefined. But if "ddns_restrict_secure" or "ddns_principal_tracking" contains a value, then the override_ddns_principal_security attribute is set to "true".

Note that when ddns_principal_group contains a value and override_ddns_principal_security is set to "false", the last operation takes precedence. Thus the sequence $zone->ddns_principal_group($principal_group); $zone->override_ddns_principal_security("false"); sets override_ddns_principal_security to "false", and the sequence $zone->override_ddns_principal_security("false"); $zone->ddns_principal_group($principal_group); sets override_ddns_principal_security to "true".

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

The valid value is an Infoblox::DNS::DDNS::PrincipalCluster::Group object.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#get ddns_principal_group value
$ddns_principal_group = $zone->ddns_principal_group();

#modify ddns_principal_group value
$zone->ddns_principal_group($principal_group);
```

delegate_to( )

Use this method to set or retrieve the delegate_to flag of a DNS zone.

This provides information for the remote name server that maintains data for the delegated zone. The Infoblox appliance redirects queries

for data for the delegated zone to this remote name server.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

The valid value is an array reference that contains Infoblox::DNS::Nameserver objects.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get delegate_to
my $delegate_to = $zone->delegate_to();
#Modify delegate_to
my $nameserver1 = Infoblox::DNS::Nameserver->new(
       name     => "ns1.del.com",
       ipv4addr => "11.11.11.11",
   );
my $nameserver2 = Infoblox::DNS::Nameserver->new(
       name     => "test",
       ipv6addr => "2006::b671",
   );
$zone->delegate_to([$nameserver1, $nameserver2]);
```

delegated_ttl( )

Use this method to set or retrieve the delegated_ttl value of a delegated DNS zone.

You can specify the Time to Live (TTL) values of auto-generated NS and glue records for a delegated zone. This value is the number of seconds that data is cached.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

A 32-bit integer (range from 0 to 4294967295) that represents the delegated_ttl in seconds. The default value is undefined.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails. When delegated_ttl is modified, override_delegated_ttl will be set to 'true' implicitly.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get delegated_ttl
my $delegated_ttl = $zone->delegated_ttl();
#Modify delegated_ttl
$zone->delegated_ttl("3600");
```

delegation_ns_group( )

Use this method to set or retrieve the delegation name servers group for the Delegation DNS Zone object.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

The valid values are string that contains name of the group or Infoblox::Grid::DNS::Nsgroup::DelegationMember.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Construct external nameserver
my $ns1 = Infoblox::DNS::Nameserver->new(
    name     => 'ns1.domain0.extra',
    ipv4addr => '5.5.5.5',
```

```
);

#Create delegation nameservers group
my $delegation_group = Infoblox::Grid::DNS::DelegationMember->new(
    name               => 'delegation_ns_group_A',
    delegation_servers => [$member],
);

#Submit created delegation nameserver group to session
$session->add($delegation_group);

#Get delegation_ns_group
my $delegation_ns_group = $zone->delegation_ns_group();

#Modify delegation_ns_group
$zone->delegation_ns_group("delegation_ns_group_A");
```

disable( )

Use this method to set or retrieve the disable flag of a DNS zone.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

Specify "true" to set the disable flag or "false" to deactivate/unset it. The default value is "false".

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get disable
my $disable = $zone->disable();
#Modify disable
$zone->disable("true");
```

disable_forwarding( )

Use this method to set or retrieve the disable_forwarding flag of a DNS zone.

Set this flag to indicate that the name servers hosting the zone should not forward queries that end with the domain name of the zone to any configured forwarders.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

Specify "true" to disable forwarding or "false" to enable it. The default value is "false".

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get disable_forwarding
my $disable_forwarding = $zone->disable_forwarding();
#Modify disable_forwarding
$zone->disable_forwarding("true");
```

dnssec_enable_ksk_auto_rollover( )

Use this method to enable automatic rollover for Key-Signing key.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

Specify "true" to enable automatic rollover for Key-signing key or "false" to disable it. The default value is "false".

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get dnssec_enable_ksk_auto_rollover value
my $enable_ksk_auto_rollover = $zone->dnssec_enable_ksk_auto_rollover();
#Modify dnssec_enable_ksk_auto_rollover value
$zone->dnssec_enable_ksk_auto_rollover("true");
```

dnssec_keys( )

Use this method to set or retrieve the list of public keys.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

The valid value is a list of Infoblox::DNS::DNSSecKey objects.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get dnssec_keys value
my $keys = $zone->dnssec_keys();
#Modify dnssec_keys value
$zone->dnssec_keys($keys);
```

dnssec_ksk_email_notification_enabled( )

Use this method to enable or disable email notifications for KSK related events.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

Specify "true" to enable email notification or "false" to disable it. The default value is "false".

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get dnssec_ksk_email_notification_enabled value
my $value = $zone->dnssec_ksk_email_notification_enabled();
#Modify dnssec_ksk_email_notification_enabled value
$zone->dnssec_ksk_email_notification_enabled("true");
```

dnssec_ksk_rollover_date( )

Use this method to set or retrieve the time for the next KSK rollover.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

The valid value is the number of seconds that have elapsed since January 1st, 1970 UTC.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get dnssec_ksk_rollover_date value
my $value = $zone->dnssec_ksk_rollover_date_value();
#Modify dnssec_ksk_rollover_date value
$zone->dnssec_ksk_rollover_date_value($date);
```

dnssec_ksk_rollover_notification_config( )

Use this method to specify the type of events users will be notified.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

The valid values are "NONE", "ALL" and "REQUIRE_MANUAL_INTERVENTION". Default value is undefined.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get dnssec_ksk_rollover_notification_config value
my $value = $zone->dnssec_ksk_rollover_notification_config();
#Modify dnssec_ksk_rollover_notification_config value
$zone->dnssec_ksk_rollover_notification_config("ALL");
```

dnssec_ksk_snmp_notification_enabled( )

Use this method to enable or disable SNMP notifications for KSK related events.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

Specify "true" to enable SNMP notification or "false" to disable it. The default value is "false".

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get dnssec_ksk_snmp_notification_enabled value
my $value = $zone->dnssec_ksk_snmp_notification_enabled();
#Modify dnssec_ksk_snmp_notification_enabled value
$zone->dnssec_ksk_snmp_notification_enabled("true");
```

dns_email( )

Use this method to retrieve the email address (in punycode format) for the administrator who manages DNS zones. This is a read-only attribute.

Parameter

None

Returns

The method returns the attribute value.

Example

```
# Get attribute value
my $value = $zone->dns_email();
```

dns_integrity_check_enable( )

Use this method to set or retrieve the flag used to indicate whether this authoritative zone is monitored for discrepancies or not.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

Specify "true" to enable authoritative zone monitoring for discrepancies, or "false" do disable it. The default value is "false".

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get dns_integrity_check_enable value
my $value = $zone->dns_integrity_check_enable();
#Modify dns_integrity_check_enable value
$zone->dns_integrity_check_enable("true");
```

dns_integrity_frequency( )

Use this method to set or retrieve the frequency (in seconds) for monitoring authoritative zones.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

The valid values are unsigned integers between 900 and 65535. The default value is undefined.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get dns_integrity_frequency value
my $value = $zone->dns_integrity_frequency();
#Modify dns_integrity_frequency value
$zone->dns_integrity_frequency(1000);
```

dns_integrity_member( )

Use this method to set or retrieve the authoritative zone member on which DNS integrity check is performed.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

The valid value is authoritative zone member name in string format. The default value is undefined.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get dns_integrity_member value
my $value = $zone->dns_integrity_member();
#Modify dns_integrity_member value
$zone->dns_integrity_member("test.localdomain");
```

dns_integrity_verbose_logging( )

Use this method to specify whether we should put more details in the authoritative zone syslog for troubleshooting or not.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

Specify "true" to put more details in syslog, or "false" to put less details in syslog. Default value is "false".

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get dns_integrity_verbose_logging value
my $value = $zone->dns_integrity_verbose_logging();
#Modify dns_integrity_verbose_logging value
$zone->dns_integrity_verbose_logging("true");
```

dns_name( )

Use this method to retrieve the name, in punycode format, of a DNS zone. This is a read-only attribute.

Parameter

None

Returns

The method returns the attribute value.

Example

```
# Get attribute value
my $value = $zone->dns_name();
```

dns_soa_mname( )

Use this method to retrieve the primary server name, in punycode format, of a DNS zone. This is a read-only attribute.

Parameter

None

Returns

The method returns the attribute value.

Example

```
# Get attribute value
my $value = $zone->dns_soa_mname();
```

dnssec_ksk_algorithms( )

Use this method to set or retrieve the DNSSEC key-signing key algorithms of the zone.

This property determines which algorithms to use when generating the keys for signing the DNSKEY records.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

Currently there are two major versions of the DNSSEC zone encryption mechanism. The difference between the two is in their implementation of the Next Secure (NSEC) records. The older version, which is referred to as NSEC, is based on RFCs 4033-4035. The newer version, which is referred to as NSEC3, is described in RFC 5155.

The valid value is a list containing Infoblox::DNS::DNSSecKeyAlgorithm objects.

The dnssec_ksk_algorithms parameter inherits the grid setting only when it and all the following DNSSEC parameters are undefined: dnssec_ksk_rollover_interval, dnssec_signature_expiration, dnssec_zsk_algorithms, dnssec_zsk_rollover_interval.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
# Getting algorithm
my $algorithms = $zone->dnssec_ksk_algorithms();

# Modifying algorithm
$gzone->dnssec_ksk_algorithms([$alg1, $alg2]);

#Inheriting the grid DNSSEC settings
$zone->dnssec_ksk_algorithms(undef);
$zone->dnssec_ksk_rollover_interval(undef);
$zone->dnssec_signature_expiration(undef
$zone->dnssec_zsk_algorithms(undef);
$zone->dnssec_zsk_rollover_interval(undef);
```

dnssec_ksk_rollover_interval()

Use this method to set or retrieve the rollover interval of the KSK keys.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

A positive integer that specifies the number of seconds between KSK rollovers. Normally, this parameter should be larger than the ZSK rollover interval. The DNSSEC key-signing key rollover interval (in seconds) must be between 1 day and seconds to Jan 2, 2038.

The dnssec_ksk_rollover_interval parameter inherits the grid setting only when it and all the following DNSSEC parameters are undefined: dnssec_ksk_algorithms, dnssec_signature_expiration, dnssec_zsk_algorithms, dnssec_zsk_rollover_interval. When at least one of the parameters is specified, then the dnssec_ksk_rollover_interval parameter uses the value 31536000 seconds (1 year), unless a different value is explicitly specified.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
# Getting dnssec_ksk_rollover_interval
my $ksk_rollover = $zone->dnssec_ksk_rollover_interval();

# Modifying dnssec_ksk_rollover_interval
$zone->dnssec_ksk_rollover_interval(86400);

#Use the grid setting
$zone->dnssec_ksk_algorithms(undef);
$zone->dnssec_ksk_rollover_interval(undef);
$zone->dnssec_signature_expiration(undef);
$zone->dnssec_zsk_algorithms(undef);
$zone->dnssec_zsk_rollover_interval(undef);
```

dnssec_nsec3_iterations( )

Use this method to define the number of iterations for NSEC3 hashing.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

The valid value are integers between 0 and 65535. Default value is undefined.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get dnssec_nsec3_iterations value
my $iteration = $zone->dnssec_nsec3_iterations();
#Modify dnssec_nsec3_iterations value
$zone->dnssec_nsec3_iterations();
```

dnssec_nsec3_salt_min_length( )

Use this method to define the minimum length for the NSEC3 salt.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

The valid value are integers between 0 and 65535. Default value is undefined.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get dnssec_nsec3_salt_min_length value
my $salt_min_length = $zone->dnssec_nsec3_salt_min_length();
#Modify dnssec_nsec3_salt_min_length value
$zone->dnssec_nsec3_salt_min_length();
```

dnssec_nsec3_salt_max_length( )

Use this method to define the maximum legnth for the NSEC3 salt.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

The valid value are integers between 0 and 65535. Default value is undefined.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get dnssec_nsec3_salt_max_length value
my $salt_max_length = $zone->dnssec_nsec3_salt_max_length();
#Modify dnssec_nsec3_salt_max_length value
$zone->dnssec_nsec3_salt_max_length();
```

dnssec_resign( )

Use this method to initiate the rollover of the key-signing key and the rollover of the zone-signing key for the DNSSEC signed zone.

Parameter

None

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

Example

```
#Do KSK/ZSK rollover
unless($zone->dnssec_resign()){ warn "KSK/ZSK rollover failed!";}
```

dnssec_signature_expiration( )

Use this method to set or retrieve the expiration time for DNSSEC signatures.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

The signature expiration time in seconds. The minimum value is 86400 (1 day), the maximum value is the number of seconds from the time you set it to January 2038.

The dnssec_signature_expiration parameter inherits the grid setting only when it and all the following DNSSEC parameters are undefined: dnssec_ksk_algorithms, dnssec_ksk_rollover_interval, dnssec_zsk_algorithms, dnssec_zsk_rollover_interval. When at least one of the parameters is specified, then the dnssec_signature_expiration parameter uses the value 2592000 seconds (30 days), unless a different value is explicitly specified.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
# Getting dnssec_signature_expiration
my $signature_expiration = $zone->dnssec_signature_expiration();

# Modifying dnssec_signature_expiration
$zone->dnssec_signature_expiration(31536000);

#Use the grid setting
$zone->dnssec_ksk_algorithms(undef);
$zone->dnssec_ksk_rollover_interval(undef);
$zone->dnssec_signature_expiration(undef);
$zone->dnssec_zsk_algorithms(undef);
$zone->dnssec_zsk_rollover_interval(undef);
```

dnssec_signed( )

Use this method to check if the zone is DNSSEC signed and to sign/unsign the zone.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value. Note that for this method it is not necessary to use the call $session->`modify()` on the zone object for the setting to take effect.

Parameter

Set the parameter to "true" to sign the zone, set the parameter to "false" to unsign the zone. The default value of this parameter is "false".

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns "true" if the zone is signed and "false" if it is unsigned.

Example

```
# Checking whether the zone is signed
my $is_signed = $zone->dnssec_signed();

# Signing the zone
$zone->dnssec_signed("true");

# Unsigning the zone
$zone->dnssec_signed("false");
```

dnssec_zsk_algorithms( )

Use this method to set or retrieve the DNSSEC zone-signing key algorithms of the zone.

This property determines which algorithms to use when generating the keys for signing the DNSKEY records.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

Currently there are two major versions of the DNSSEC zone encryption mechanism. The difference between the two is in their implementation of the Next Secure (NSEC) records. The older version, which is referred to as NSEC, is based on RFCs 4033-4035. The newer version, which is referred to as NSEC3, is described in RFC 5155.

The valid value is a list containing Infoblox::DNS::DNSSecKeyAlgorithm objects.

The dnssec_zsk_algorithms parameter inherits the grid setting only when it and all the following DNSSEC parameters are undefined: dnssec_ksk_rollover_interval, dnssec_signature_expiration, dnssec_ksk_algorithms, dnssec_zsk_rollover_interval.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
# Getting algorithm
my $algorithms = $zone->dnssec_zsk_algorithms();

# Modifying algorithm
$gzone->dnssec_zsk_algorithms([$alg1, $alg2]);

#Inheriting the grid DNSSEC settings
$zone->dnssec_ksk_algorithms(undef);
$zone->dnssec_ksk_rollover_interval(undef);
$zone->dnssec_signature_expiration(undef
$zone->dnssec_zsk_algorithms(undef);
$zone->dnssec_zsk_rollover_interval(undef);
```

dnssec_zsk_rollover_date( )

Use this method to set or retrieve the time for the next ZSK rollover.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

The valid value is the number of seconds that have elapsed since January 1st, 1970 UTC.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get dnssec_zsk_rollover_date value
my $value = $zone->dnssec_zsk_rollover_date_value();
#Modify dnssec_zsk_rollover_date value
$zone->dnssec_zsk_rollover_date_value($date);
```

dnssec_zsk_rollover_interval()

Use this method to set or retrieve the zone-signing key rollover interval.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

A positive integer that specifies the number of seconds between ZSK rollovers. Normally, this parameter should be smaller than the KSK rollover interval. The DNSSEC zone-signing key over interval (in seconds) must be between 1 day and the number of seconds from present to Jan 2038.

The dnssec_zsk_rollover_interval parameter inherits the grid setting only when it and all the following DNSSEC parameters are undefined: dnssec_ksk_algorithms, dnssec_ksk_rollover_interval, dnssec_signature_expiration, dnssec_zsk_algorithms. When at least one of the parameters is specified, then the dnssec_zsk_rollover_interval parameter uses the value 2592000 seconds (30 days), unless a different value is explicitly specified.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
# Getting dnssec_zsk_rollover_interval
my $zsk_rollover = $zone->dnssec_zsk_rollover_interval();

# Modifying dnssec_ksk_rollover_interval
$zone->dnssec_zsk_rollover_interval(86400);

#Use the grid setting
$zone->dnssec_ksk_algorithms(undef);
$zone->dnssec_ksk_rollover_interval(undef);
$zone->dnssec_signature_expiration(undef);
$zone->dnssec_zsk_algorithms(undef);
$zone->dnssec_zsk_rollover_interval(undef);
```

dnssec_zsk_rollover_mechanism( )

Use this method to set or retrive the ZSK rollover mechanism type.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

The valid values are "DOUBLE_SING" and "PRE_PUBLISH". Default value is undefined.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get dnssec_zsk_rollover_mechanism value
my $value = $zone->dnssec_zsk_rollover_mechanism();
#Modify dnssec_zsk_rollover_mechanism value
$zone->dnssec_zsk_rollover_mechanism("DOUBLE_SIGN");
```

do_host_abstraction( )

Use this method to set the do_host_abstraction property used in zone import operations. This is a write-only attribute.

Set this flag to indicate that Infoblox host records will be automatically created from A records when the zone data is imported.

Parameter

Specify "true" to trigger host creation or "false" to disable it. The default value is "false".

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

Example

```
#Modify do_host_abstraction
$zone->do_host_abstraction("true");
```

enable_ad_servers( )

Use this method to set or retrieve the flag that allows GSS-TSIG unsigned updates from AD domain controllers.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

Specify "true" to set the flag or "false" to deactivate/unset it. The default value is "false".

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
# Get enable_ad_servers.
my $enable_ad_servers = $zone->enable_ad_servers();
# Modify enable_ad_servers.
$zone->enable_ad_servers("true");
```

email( )

Use this method to set or retrieve the e-mail address of an administrator of a DNS zone.

This setting overrides grid-level e-mail settings.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

The attribute value can be in unicode format.

Parameter

The e-mail address of an administrator.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get email
my $email = $zone->email();
#Modify email
$zone->email("admin\@infoblox.com");
```

execute_dns_parent_check( )

Use this method to perform an one-time DNS integrity check for the selected zone.

Parameter

None

Returns

Returns "true" value upon the successful completion, and returns fail string otherwise.

Example

```
#perform one-time check for zone
$zone->execute_dns_parent_check();
```

extattrs( )

Use this method to set or retrieve the extensible attributes associated with a DNS zone object.

Parameter

Valid value is a hash reference containing the names of extensible attributes and their associated values (Infoblox::Grid::Extattr objects).

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get extattrs
my $ref_extattrs = $zone->extattrs();
#Modify extattrs
$zone->extattrs({ 'Site' => $extattr1, 'Administrator' => $extattr2 });
```

extensible_attributes( )

Use this method to set or retrieve the extensible attributes associated with a DNS zone.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

For valid values for extensible attributes, see Infoblox::Grid::ExtensibleAttributeDef/Extensible Attribute Values.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get extensible attributes
my $ref_extensible_attributes = $zone->extensible_attributes();
#Modify extensible attributes
$zone->extensible_attributes({ 'Site' => 'Santa Clara', 'Administrator' => [ 'Peter', 'Tom' ] });
```

fireeye_rule_mapping( )

Use this method to set or retrieve the FireEye rule mapping.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Returns

An the Infoblox::DNS::FireEye::RuleMapping manpage object.

Example

```
#Get fireeye_rule_mapping
my $fireeye_rule_mapping = $zone->fireeye_rule_mapping();
#Modify fireeye_rule_mapping
$zone->fireeye_rule_mapping($rm);
```

forward_external_ns_group( )

Use this method to set or retrieve the external nameservers group name for forward DNS zone.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

The valid value is a desired external name servers group name in a string format.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Construct external nameserver
my $ns1 = Infoblox::DNS::Nameserver->new(
    name     => 'ns1.domain0.extra',
    ipv4addr => '5.5.5.5',
);

#Construct external nameservers group
my $ext = Infoblox::DNS::Nsgroup::ForwardStubServer->new(
    name             => 'extsrvg1',
    external_servers => [$ns1],
);

#Submit created forward_external nameservers group to session
$session->add($ext);

#Get forward_external_ns_group value
my $forward_external_ns_group = $zone->forward_external_ns_group();

#Modify forward_external_ns_group value
$zone->forward_external_ns_group('extsrvg1');
```

forward_ns_group( )

Use this method to set or retrieve the name servers group for the Forward DNS Zone object.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

The valid values are string that contains name of the group or Infoblox::Grid::DNS::Nsgroup::ForwardingMember.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Construct external nameserver
my $ns1 = Infoblox::DNS::Nameserver->new(
    name     => 'ns1.domain0.extra',
    ipv4addr => '5.5.5.5',
);

#Construct dns member
my $member = Infoblox::DNS::Member->new(
    ipv4addr   => '192.168.1.2',
    forward_to => [$ns1],
);

#Create forward nameservers group
my $forward_group = Infoblox::Grid::DNS::ForwardingMember->new(
    name               => 'forward_ns_group_A',
    forwarding_servers => [$member],
);

#Submit created forward nameserver group to session
$session->add($forward_group);

#Get forward_ns_group
my $forward_ns_group = $zone->forward_ns_group();

#Modify forward_ns_group
$zone->forward_ns_group("forward_ns_group_A");
```

forward_only( )

Use this method to set or retrieve the forward_only flag of a DNS forward zone. When you set this flag, the appliance sends queries to forwarders only, and not to other internal or Internet root servers.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

Specify "true" to send queries to forwarders only. Specify "false" to disable it. Default is "false."

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Getting forward_only
my $forward_only = $fwd_zone->forward_only( );
#Modifying forward_only
$fwd_zone->forward_only("true");
```

forward_to( )

Use this method to set or retrieve the forward_to flag of a DNS zone.

This method provides information for the remote name server to which you want the Infoblox appliance to forward queries for a specified domain name.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

The valid value is an array reference that contains Infoblox::DNS::Member objects for grid secondary servers and Infoblox::DNS::Nameserver objects for external secondary servers. The default value is undefined.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get forward_to
my $forward_to = $zone->forward_to();
#Modify forward_to
my $member = Infoblox::DNS::Member->new(
        name     => "hostname.com",
        ipv4addr => "3.0.0.2",
   );

my $nameserver = Infoblox::DNS::Nameserver->new(
        name     => "ns1.forwd.com",
        ipv4addr => "11.11.11.11",
   );

$zone->forward_to([$member, $nameserver]);
```

host_name_restriction_policy( )

Use this method to set or retrieve the host_name_restriction_policy flag of a DNS zone.

This setting overrides the member-level host name restriction policy settings.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

String with the name of predefined record policies or user-defined record policies. The default value is undefined, which is to follow the Record Name policy specified in Infoblox::Grid::Member::DNS level.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get host name restriction policy
my $host_name_restriction_policy = $zone->host_name_restriction_policy();
#Modify host name restriction policy
$zone->host_name_restriction_policy("Allow Any");
```

import_from( )

Use this method to set or retrieve the import_from flag of a DNS zone.

This method specifies the IP address of the Infoblox appliance from which zone data is imported. This is only valid for authoritative zones. Setting this address to '255.255.255.255' and do_host_abstraction to 'true' will create Host records from A records in this zone without importing zone data.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

An IPv4 address is a 32-bit number with the basic format representation of `X.X.X.X` where x refers to decimal digits (8 bits), for example: `128.6.4.7`. The default value is undefined.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Modify import_from
$zone->import_from("192.168.1.2");
```

initiate_ksk_rollover( )

Use this method to initiate the rollover of the key-signing key for the DNSSEC signed zone.

Parameter

None

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

Example

```
#Do KSK rollover
unless($zone->initiate_ksk_rollover()){ warn "KSK rollover failed!";}
```

initiate_zsk_rollover( )

Use this method to initiate the rollover of the Zone-signing key for the DNSSEC signed zone.

Parameter

None

Returns

Returns true value upon the successful completion, and returns false value otherwise.

Example

```
#Do ZSK rollover
unless($zone->initiate_zsk_rollover()){ warn "ZSK rollover failed!";}
```

is_default( )

Use this method to set or retrieve the is_default flag of a DNS zone.

This flag is used to determine the default zone for the zone associations of a DHCP network. This is set or returned only for zone objects set in the zone-associations method of the DHCP network.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

Specify "true" to set the is_default flag or "false" to deactivate/unset it. The default value is "false".

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get is_default
my $is_default = $zone->is_default();
```

```
#Modify is_default
$zone->is_default("true");
```

is_multimaster( )

Use this method to set or retrieve the is_multimaster flag.

Parameter

Specify "true" to set the is_multimaster flag or "false" to deactivate/unset it. The default value is "false".

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
# Get the attribute value
my $attr = $zone->is_multimaster();
# Modify the attribute value
$zone->is_multimaster('true');
```

last_queried( )

Use this method to retrieve the time the zone was last queried on.

Returns

The method returns the attribute value, the number of seconds that have elapsed since January 1st, 1970 UTC.

Example

```
#Get last_queried
my $last_queried = $zone->last_queried();
```

rpz_drop_ip_rule_enabled( )

Use this method to set or retreive the flag that indicates whether the DNS server ignores RPZ-IP triggers with large prefix lengths or not.

Setting this method to a defined value implicitly sets the override_rpz_drop_ip_rule method to "true". Setting the parameter to undefined if rpz_drop_ip_rule_min_prefix_length_ipv4 and rpz_drop_ip_rule_min_prefix_length_ipv6 are undefined causes the appliance to use the Grid default and automatically resets the override_rpz_drop_ip_rule attribute to "false".

Note that when rpz_drop_ip_rule_enabled is set to a defined value and override_rpz_drop_ip_rule is set to "false", the last operation takes precedence. Thus the sequence $object->rpz_drop_ip_rule_enabled("true"); $object->override_rpz_drop_ip_rule("false"); will set override_rpz_drop_ip_rule to "false", and the sequence $object->override_rpz_drop_ip_rule("false"); $object->rpz_drop_ip_rule_enabled("true"); will result in override_rpz_drop_ip_rule="true".

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

Specify 'true' to enable ignoring of RPZ-IP triggers with large prefix lengths or 'false' to disable it. The default value is 'false'.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
# Get attribute value
my $value = $zone->rpz_drop_ip_rule_enabled();
# Modify attribute value
$zone->rpz_drop_ip_rule_enabled("true");
```

rpz_drop_ip_rule_min_prefix_length_ipv4( )

Use this method to set or retreive minimum prefix length for IPv4 RPZ-IP triggers.

Setting this method to a defined value implicitly sets the override_rpz_drop_ip_rule method to "true". Setting the parameter to undefined if rpz_drop_ip_rule_enabled and rpz_drop_ip_rule_min_prefix_length_ipv6 are undefined causes the appliance to use the Grid default and automatically resets the override_rpz_drop_ip_rule attribute to "false".

Note that when rpz_drop_ip_rule_min_prefix_length_ipv4 is set to a defined value and override_rpz_drop_ip_rule is set to "false", the last

operation takes precedence. Thus the sequence $object->rpz_drop_ip_rule_min_prefix_length_ipv4("20"); $object->override_rpz_drop_ip_rule("false"); will set override_rpz_drop_ip_rule to "false", and the sequence $object->override_rpz_drop_ip_rule("false"); $object->rpz_drop_ip_rule_min_prefix_length_ipv4("20"); will result in override_rpz_drop_ip_rule="true".

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

The valid value is an unsigned integer between 1 and 31 that represents prefix length (in bits) for IPv4 RPZ-IP triggers.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
# Get attribute value
my $value = $zone->rpz_drop_ip_rule_min_prefix_length_ipv4();
# Modify attribute value
$zone->rpz_drop_ip_rule_min_prefix_length_ipv4("16");
```

rpz_drop_ip_rule_min_prefix_length_ipv6( )

Use this method to set or retreive minimum prefix length for IPv6 RPZ-IP triggers.

Setting this method to a defined value implicitly sets the override_rpz_drop_ip_rule method to "true". Setting the parameter to undefined if rpz_drop_ip_rule_enabled and rpz_drop_ip_rule_min_prefix_length_ipv4 are undefined causes the appliance to use the Grid default and automatically resets the override_rpz_drop_ip_rule attribute to "false".

Note that when rpz_drop_ip_rule_min_prefix_length_ipv6 is set to a defined value and override_rpz_drop_ip_rule is set to "false", the last operation takes precedence. Thus the sequence $object->rpz_drop_ip_rule_min_prefix_length_ipv6("20"); $object->override_rpz_drop_ip_rule("false"); will set override_rpz_drop_ip_rule to "false", and the sequence $object->override_rpz_drop_ip_rule("false"); $object->rpz_drop_ip_rule_min_prefix_length_ipv6("20"); will result in override_rpz_drop_ip_rule="true".

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

The valid value is an unsigned integer between 1 and 127 that represents prefix length (in bits) for IPv6 RPZ-IP triggers.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
# Get attribute value
my $value = $zone->rpz_drop_ip_rule_min_prefix_length_ipv6();
# Modify attribute value
$zone->rpz_drop_ip_rule_min_prefix_length_ipv6("120");
```

rpz_policy( )

Use this method to set or retrieve the override policy for the Response Policy Zone.

This property determines the generic override rule for all records in this Response Policy Zone.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

The valid value is one of "GIVEN", "PASSTHRU", "NXDOMAIN", "NODATA", "SUBSTITUTE", "DISABLED". The default value is undefined.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
# Getting rpz_policy
```

```
my $rpz_policy = $zone->rpz_policy();

# Modifying rpz_policy
$zone->rpz_policy("PASSTHRU");
```

restart_if_needed( )

Use this method to set the flag that indicates whether DHCP service should be restarted on a member that is serving DHCP for the changed object referenced in a cloud API request.

Parameter

Specify true to restart the DHCP service.

Returns

None

Example

```
# Modify
$zone->restart_if_needed('true');
```

rpz_type( )

Use this method to set or retrieve the type of the Response Policy Zone.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

The valid value is one of "LOCAL", "FIREEYE", "FEED". The default value is undefined.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
# Getting rpz_type
my $rpz_type = $zone->rpz_type();

# Modifying rpz_type
$zone->rpz_type("LOCAL");
```

rpz_severity( )

Use this method to set or retrieve the type of severity for Response Policy Zone.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

The valid value is CRITICAL, MAJOR, WARNING, or INFORMATIONAL. The default value is undefined.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
# Getting rpz_severity
my $rpz_severity = $zone->rpz_severity();

# Modifying rpz_type
$zone->rpz_severity("MAJOR");
```

substitute_name( )

Use this method to retrieve the substitution name when the RPZ policy is set to "SUBSTITUTE".

You must specify a parameter if rpz_policy is set to "SUBSTITUTE".

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
# Getting algorithm
my $substitution_name = $zone->substitute_name();

# Modifying algorithm
$zone->substitute_name("something");
```

rpz_priority( )

Use this method to retrieve the priority of zones among all Response Policy Zones.

This attribute is readonly and you can use an Infoblox::DNS::OrderedResponsePolicyZones object to reorder Response Policy Zones in a DNS view.

Returns

This method returns the attribute value in integer format. "1" indicates the highest priority.

Example

```
#Get rpz_priority
my $rpz_priority = $zone->rpz_priority();
```

rpz_last_updated_time( )

Use this method to retrieve the date and time the Response Policy Zone was last updated.

Returns

This method returns the attribute value in number of seconds that have elapsed since January 1st, 1970 UTC.

Example

```
#Get rpz_last_updated_time
my $last_updated_time = $zone->rpz_last_updated_time();
```

rpz_severity( )

Use this method to set or retrieve the severity of this response policy zone.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

The valid value is one of the following: CRITICAL, MAJOR, WARNING or INFORMATIONAL.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get rpz_severity
my $severity = $zone->rpz_severity();

# Modify rpz_severity
$zone->rpz_severity("MAJOR");
```

is_feed_zone( )

Use this method to retrieve the boolena value that indicates whether the Response Policy Zone is a "feed" zone or not.

Returns

This method returns the attribute value. "True" means the response policy zone is a feed zone that has an assigned external primary.

Example

```
#Get is_feed_zone
my $is_feed_zone = $zone->is_feed_zone();
```

locked( )

Use this method to set or retrieve the locked flag of a DNS zone.

If you enable this flag other administrators cannot make changes that would "collide". This is for administration purposes only. The zone will continue to serve DNS data even when it is locked.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

Specify "true" to set the locked flag or "false" to deactivate/unset it. The default value is "false".

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get locked
my $locked = $zone->locked();
#Modify locked
$zone->locked("true");
```

members( )

Use this method to set or retrieve the members flag of a DNS forward zone.

This method specifies the grid member that serves the forward zone.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

The valid value is an array reference that contains Infoblox::DNS::Member object(s). The default value is undefined.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get members
my $members = $zone->members();
#Modify members
my $memberns1 = Infoblox::DNS::Member->new(
      name => "member1.forwd.com",
  );

my $memberns2 = Infoblox::DNS::Member->new(
      name => "member2.forwd.com",
  );

$zone->members([$memberns1,$memberns2]);
```

member_soa_serials( )

Use this method to retrieve per-member SOA serial information. This is a read-only attribute.

Parameter

None.

Returns

The method returns the attribute value that contains Infoblox::DNS::Member::SoaSerial object(s).

Example

```
# Get the attribute value
my $member_soa_serials = $zone->member_soa_serials();
```

member_soa_mnames( )

Use this method to set or retrieve per-member SOA MNAME information.

Parameter

The valid value is an array reference that contains Infoblox::DNS::Member::SoaMname objects.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
# Get the attribute value
my $member_soa_mnames = $zone->member_soa_mnames();
# Modify the attribute value
$zone->member_soa_mnames([$soa_mname]);
```

ms_dc_ns_record_creation( )

Use this method to set or retrieve the list of Active Directory Domain Controllers which are allowed to create NS Record for the DNS Zone.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameters

The valid value is an array of Infoblox::Grid::MSServer::DCNSRecordCreation objects.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get ms_dc_ns_record_creation
my $ms_dc_ns_record_creation = $zone->ms_dc_ns_record_creation();

#Modify ms_dc_ns_record_creation
$zone->ms_dc_ns_record_creation([$msdcns1, $msdcns1]);
```

name( )

Use this method to set or retrieve the name of a DNS zone.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

The attribute value can be in unicode format.

Parameter

Zone name. For a reverse zone, this is in "address/cidr" format. For other zones, this is in FQDN (Fully Qualified Domain Name) format. A zone name can have a maximum of 256 characters.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get name
my $name = $zone->name();
```

network_associations( )

Use this method to retrieve the associated network/network container information. This is a read-only method.

Parameter

No parameter need to be specified

Returns

This method returns an array reference that contains Infoblox::DHCP::Network, Infoblox::DHCP::NetworkContainer, Infoblox::DHCP::IPv6Network, and Infoblox::DHCP::IPv6NetworkContainer objects.

Examples

```
#Getting network associations
my $network_associations = $zone->network_associations();
```

next_secure_type( )

Use this method to set or retrieve resource record type.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

Valid value is 'NSEC' or 'NSEC3'.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get the next_secure_type value
my $next_secure_type = $zone->next_secure_type();
#Modify the next_secure_type value
$zone->next_secure_type('NSEC3');
```

notify_delay( )

Use this method to specify with how many seconds of delay the notify messages are sent to secondaries.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

Valid values are unsigned integer between 5 and 86400, inclusive. Default value is undefined.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Getting notify_delay
my $notify_delay = $zone->notify_delay( );
#Modifying notify_delay
$zone->notify_delay(5);
```

ms_ad_integrated()

Use this method to set or retrieve the Active Directory-integrated flag of a zone. This attribute is valid only when `ms_managed()` is "Stub", "Auth_primary", or "Auth_both".

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

Specify "true" to set the Active Directory-integrated flag or "false" to unset it. The default value is "false".

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get ms_ad_integrated
my $ms_ad_integrated = $zone->ms_ad_integrated();
#Modify ms_ad_integrated
$zone->ms_ad_integrated("true");
```

ms_managed()

Use this method to retrieve the flag that indicates whether the zone is assigned to a Microsoft DNS server. This flag returns the

authoritative name server type of the Microsoft DNS server. The value can be:

"None" if the zone is not assigned to any Microsoft DNS server
"Stub" if the zone is assigned to a Microsoft DNS server as a stub zone
"Auth_primary" if only the primary server of the zone is a Microsoft DNS server
"Auth_secondary" if only the secondary server of the zone is a Microsoft DNS server
"Auth_both" if both the primary and secondary servers of the zone are Microsoft DNS servers

Returns

The method returns the attribute value.

Example

```
#Get ms_managed
my $ms_managed = $zone->ms_managed();
```

ms_read_only()

Use this method to retrieve the flag that indicates if a grid member manages the zone served by a Microsoft DNS server in read-only mode. This flag returns "true" when a grid member manages the zone in read-only mode. Otherwise, it returns "false".

When the zone has the `ms_read_only()` flag set to "true", no change can be made to this zone.

Returns

The method returns the attribute value.

Example

```
#Get ms_read_only
my $ms_read_only = $zone->ms_read_only();
```

ms_ddns_mode()

Use this method to set or retrieve the flag that indicates whether an Active Directory-integrated zone with a Microsoft DNS server as primary allows dynamic updates. This flag returns "Secure" if the zone allows secure updates only, "None" if the zone forbids dynamic updates, or "Any" if the zone accepts both secure and nonsecure updates. This flag is valid only if `ms_managed()` is either "Auth_primary" or "Auth_both". If the flag `ms_ad_integrated()` is false, the value "Secure" is not allowed.

Parameter

Specify "Secure", "None", or "Any" to control the dynamic updates that an Active Directory-integrated zone is allowed to accept. The default value is undefined.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get ms_ddns_mode
my $ms_ddns_mode = $zone->ms_ddns_mode();
#Modify ms_ddns_mode
$zone->ms_ddns_mode("None");
```

ms_allow_transfer()

Use this method to set or retrieve the list of DNS clients that are allowed to perform zone transfers from a Microsoft DNS server.

This setting applies only to zones with Microsoft DNS servers that are either primary or secondary servers. This setting does not inherit any value from the grid or from any member that defines an `allow_transfer()` value. This setting does not apply to any grid member. Use the `allow_transfer()` attribute to control allowed clients on grid members.

Valid values are:

An array that contains a list of IPv4 addresses
The keyword "Any" to allow any client
The keyword "NSOnly" to allow only the nameservers listed in the zone
The keyword "None" to deny all zone transfer requests

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

Specify one of the allowed values to identify the DNS clients that are allowed to perform zone transfers from a Microsoft DNS

server. The default is "NSOnly".

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get ms_allow_transfer
my $ms_allow_transfer = $zone->ms_allow_transfer();
#Modify ms_allow_transfer
$zone->ms_allow_transfer("Any");
```

ms_sync_disable( )

Use this method to disable Microsoft Server synchronization for this zone.

Parameter

Set the parameter to "true" to disable Microsoft Server synchronization for this zone. Set the parameter to "false" to enable Microsoft Server synchronization for this zone.

If you did not specify a parameter, the method returns the attribute value.

Return

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get ms_sync_disable value
my $ms_sync_disable = $zone->ms_sync_disable();

#Set ms_sync_disable value
$zone->ms_sync_disable("true");
```

ns_group( )

Use this method to set or retrieve the name servers group for the DNS Authoritative and Response Policy Zone objects.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

The valid values are string that contains name of the group or Infoblox::Grid::DNS::Nsgroup.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get ns_group
my $ns_group = $zone->ns_group();

#Modify ns_group
$zone->ns_group("ns_group_A");
```

override_copy_xfer_to_notify( )

The override_copy_xfer_to_notify attribute controls whether the copy_xfer_to_notify method values of the DNS zone are used, instead of the member default.

The override_copy_xfer_to_notify attribute can be specified explicitly. It is also set implicitly when copy_xfer_to_notify is set to a defined value.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

Set the parameter to "true" to override the member-level setting for copy_xfer_to_notify. Set the parameter to "false" to inherit the member-level setting for copy_xfer_to_notify.

The default value of this parameter is "false".

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Getting override_copy_xfer_to_notify
my $override_copy_xfer_to_notify = $zone->override_copy_xfer_to_notify( );
#Modifying override_copy_xfer_to_notify
$zone->override_copy_xfer_to_notify("true");
```

override_ddns_force_creation_timestamp_update( )

The override_ddns_force_creation_timestamp_update attribute controls whether the ddns_force_creation_timestamp_update method value of the DNS View is used, instead of the Grid default.

The override_ddns_force_creation_timestamp_update attribute can be specified explicitly. It is also set implicitly when ddns_force_creation_timestamp_update is set to a defined value.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

Set the parameter to "true" to override the Grid-level setting for ddns_force_creation_timestamp_update. Set the parameter to "false" to inherit the Grid-level setting for ddns_force_creation_timestamp_update.

The default value of this parameter is "false".

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get override_ddns_force_creation_timestamp_update value
my $value = $object->override_ddns_force_creation_timestamp_update();

#Modify override_ddns_force_creation_timestamp_update value
$object->override_ddns_force_creation_timestamp_update("true");
```

override_ddns_patterns_restriction( )

The override_ddns_patterns_restriction attribute controls whether the ddns_restrict_patterns_list and ddns_restrict_patterns methods values of the DNS Zone are used, instead of the Grid default.

The override_ddns_patterns_restriction attribute can be specified explicitly. It is also set implicitly when ddns_restrict_patterns_list or ddns_restrict_patterns is set to a defined value.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

Set the parameter to "true" to override the Grid-level setting for ddns_restrict_patterns_list and ddns_restrict_patterns. Set the parameter to "false" to inherit the Grid-level setting for ddns_restrict_patterns_list and ddns_restrict_patterns.

The default value of this parameter is "false".

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get override_ddns_patterns_restriction
my $override_ddns_patterns_restriction = $zone->override_ddns_patterns_restriction();
#Modify override_ddns_patterns_restriction
$zone->override_ddns_patterns_restriction("true");
```

override_ddns_restrict_static( )

The override_ddns_restrict_static attribute controls whether the ddns_restrict_static method value of the DNS Zone is used, instead of the

Grid default.

The override_ddns_restrict_static attribute can be specified explicitly. It is also set implicitly when ddns_restrict_static is set to a defined value.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

Set the parameter to "true" to override the Grid-level setting for ddns_restrict_static. Set the parameter to "false" to inherit the Grid-level setting for ddns_restrict_static.

The default value of this parameter is "false".

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get override_ddns_restrict_static
my $override_ddns_restrict_static = $zone->override_ddns_restrict_static();
#Modify override_ddns_restrict_static
$zone->override_ddns_restrict_static("true");
```

override_ddns_restrict_protected( )

The override_ddns_restrict_protected attribute controls whether the ddns_restrict_protected method value of the DNS Zone is used, instead of the Grid default.

The override_ddns_restrict_protected attribute can be specified explicitly. It is also set implicitly when ddns_restrict_protected is set to a defined value.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

Set the parameter to "true" to override the Grid-level setting for ddns_restrict_protected. Set the parameter to "false" to inherit the Grid-level setting for ddns_restrict_protected.

The default value of this parameter is "false".

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get override_ddns_restrict_protected
my $override_ddns_restrict_protected = $zone->override_ddns_restrict_protected();
#Modify override_ddns_restrict_protected
$zone->override_ddns_restrict_protected("true");
```

override_ddns_principal_security( )

The override_ddns_principal_security attribute controls whether the ddns_restrict_secure, ddns_principal_tracking and ddns_principal_group methods values of the DNS Zone are used, instead of the Grid default.

The override_ddns_principal_security attribute can be specified explicitly. It is also set implicitly when ddns_restrict_secure, ddns_principal_tracking or ddns_principal_group is set to a defined value.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

Set the parameter to "true" to override the Grid-level setting for ddns_restrict_secure, ddns_principal_tracking and ddns_principal_group. Set the parameter to "false" to inherit the Grid-level setting for ddns_restrict_secure, ddns_principal_tracking and ddns_principal_group.

The default value of this parameter is "false".

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get override_ddns_principal_security
my $override_ddns_principal_security = $zone->override_ddns_principal_security();
#Modify override_ddns_principal_security
$zone->override_ddns_principal_security("true");
```

override_delegated_ttl( )

Use this method to set or retrieve the override_delegated_ttl flag of a DNS zone.

This flag indicates whether to override the TTL values of auto-generated NS and GLUE records for a delegated zone. If you set it to "true", then you must specify the 'delegated_ttl' value.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

Specify "true" to override the TTL values or "false" to have these records inherit the TTL values from their parent zone. The default value is "false".

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get override_delegated_ttl
my $override_delegated_ttl = $zone->override_delegated_ttl();
#Modify override_delegated_ttl
$zone->override_delegated_ttl("true");
```

override_grid_zone_timer( )

Use this method to set or retrieve the override_grid_zone_timer flag of a DNS zone.

This flag indicates whether to override the grid's zone timer settings. If you set it to "true", then you must specify the following properties: soa_refresh, soa_retry, soa_expire, soa_default_ttl, soa_negative_ttl.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

Specify "true" to override grid zone timer settings or "false" to use the grid zone timer settings. The default value is "false".

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get override_grid_zone_timer
my $override_grid_zone_timer = $zone->override_grid_zone_timer();
#Modify override_grid_zone_timer
$zone->override_grid_zone_timer("true");
```

override_grid_email( )

Use this method to set or retrieve the override_grid_email flag of a DNS zone.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

Specify "true" to override grid e-mail settings or "false" to use the grid e-mail settings. The default value is "false".

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get override_grid_email
my $override_grid_email = $zone->override_grid_email();
#Modify override_grid_email
$zone->override_grid_email("false");
```

override_scavenging_settings( )

The override_scavenging_settings attribute controls whether the scavenging_settings method values of the DNS Zone are used, instead of the Grid default.

The override_scavenging_settings attribute can be specified explicitly. It is also set implicitly when scavenging_settings is set to a defined value.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

Set the parameter to "true" to override the Grid-level setting for scavenging_settings. Set the parameter to "false" to inherit the Grid-level setting for scavenging_settings.

The default value of this parameter is "false".

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get override_scavenging_settings value
my $override_scavenging_settings = $object->override_scavenging_settings( );

#Modify override_scavenging_settings value
$object->override_scavenging_settings("true");
```

override_rpz_drop_ip_rule( )

The override_rpz_drop_ip_rule attribute controls whether the rpz_drop_ip_rule_enabled, rpz_drop_ip_rule_min_prefix_length_ipv4 and rpz_drop_ip_rule_min_prefix_length_ipv6 method values of the member are used, instead of the Grid default.

The override_rpz_drop_ip_rule attribute can be specified explicitly. It is also set implicitly when one of rpz_drop_ip_rule_enabled, rpz_drop_ip_rule_min_prefix_length_ipv4 or rpz_drop_ip_rule_min_prefix_length_ipv6 methods is set to a defined value.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

Set the parameter to "true" to override the Grid-level setting for rpz_drop_ip_rule_enabled, rpz_drop_ip_rule_min_prefix_length_ipv4 and rpz_drop_ip_rule_min_prefix_length_ipv6 methods. Set the parameter to "false" to inherit the Grid-level setting for rpz_drop_ip_rule_enabled, rpz_drop_ip_rule_min_prefix_length_ipv4 and rpz_drop_ip_rule_min_prefix_length_ipv6 methods.

The default value of this parameter is "false".

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Getting override_rpz_drop_ip_rule
my $override_rpz_drop_ip_rule = $zone->override_rpz_drop_ip_rule( );
#Modifying override_rpz_drop_ip_rule
$zone->override_rpz_drop_ip_rule("true");
```

override_serial_number( )

Use this method to set the override_serial_number flag of a DNS zone.

If set this flag to 'true', soa_serial_number can be set. This value behaves somewhat unusually in that the server always resets it to undef in the response, even if it is true in the request.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

Specify "true" to override serial number. The default value is undef

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get override_serial_number
my $override_serial_number = $zone->override_serial_number();
#Modify override_serial_number
$zone->override_serial_number("true");
```

prefix( )

Use this method to set or retrieve the prefix flag of a DNS zone.

Use this method only when the netmask is greater than 24 bits; that is, for a mask between 25 and 31 bits. Enter a prefix, such as the name of the allocated address block. The prefix can be alphanumeric characters; for example: 128/26 , 128-189 , or sub-B . This is only valid for authoritative reverse zones.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

This prefix can be anything such as the address range (examples: 0-127, 0/127) or descriptions (examples: first-network, customer1).A prefix can have a maximum of 63 bytes. The default value is undefined.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get prefix
my $prefix = $zone->prefix();
#Modify prefix
$zone->prefix("first-network");
```

multiple_primaries( )

Use this method to set or retrieve the primary flag of a DNS zone.

This method specifies Grid primary servers, external primary servers, or Microsoft DNS servers that are primary servers for the zone. Although a zone typically has just one primary name server, you can specify up to ten independent servers for a single zone.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

The valid value is an array reference that contains Infoblox::DNS::Member objects for Grid primary servers, or Infoblox::DNS::Nameserver objects for external primary servers, or Infoblox::DNS::MSServer objects for Microsoft DNS servers. The default value is undefined.

To set two and more Infoblox::DNS::Member objects you should set the is_multimaster flag to "true".

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get multiple_primaries
my $multiple_primaries = $zone->multiple_primaries();
#Modify multiple_primaries
my $memberns1 = Infoblox::DNS::Member->new(
       name     => "member1.server.com",
       ipv4addr => "1.1.1.1",
   );

$zone->multiple_primaries([$memberns1]);
```

primary_shared_with_ms_parent_delegation( )

Use this method to retrieve the primary_shared_with_ms_parent_delegation flag. If the zone is a subzone and the primary server of the parent zone is a Microsoft server, it will be set to "true" if the FQDN and IP address in the delegation zone's NS record matches the FQDN and IP address of the authoritative name server of the zone.

Parameter

None.

Returns

The method returns the attribute value.

Example

```
#Get primary_shared_with_ms_parent_delegation
my $primary_shared_with_ms_parent_delegation = $member->primary_shared_with_ms_parent_delegation();
```

scavenging_settings( )

Use this method to set or retrieve the zone-level scavenging settings.

Setting this parameter implicitle sets the override_scavenging_settings attribute to "true". Setting the parameter to undefined causes the appliance to use the Grid-level settings and automatically sets the override_scavenging_settings attribute to "false".

Note that when scavenging_settings is set and override_scavenging_settings is set to "false", the last operation takes precedence. Thus the sequence $object->scavenging_settings($recl_settings); $object->override_scavenging_settings("false"); sets override_scavenging_settings to "false", and the sequence $object->override_scavenging_settings("false"); $object->scavenging_settings($recl_settings); sets override_scavenging_settings to "true".

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

The valid value is an Infoblox::Grid::DNS::ScavengingSetting object.

Parameter

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get scavenging_settings value
my $scavenging_settings = $object->scavenging_settings();

#Modify scavenging_settings value
$object->scavenging_settings($scavenging_settings);
```

remove_sub_zones( )

Use this method to set or retrieve the remove_sub_zones flag of a DNS zone.

You can remove the sub-zones and resource records of the selected zone.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

Specify "true" to remove sub zones or "false" to not to remove sub zones. The default value is undefined, which is to remove all subzones.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get remove_sub_zones
my $remove_sub_zones = $zone->remove_sub_zones();
#Modify remove_sub_zones
$zone->remove_sub_zones("true");
```

secondaries( )

Use this method to set or retrieve the secondaries flag of a DNS zone.

You can specify grid secondary servers, external secondary servers, or Microsoft DNS servers that are secondary servers for the zone.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

The valid value is an array reference that contains Infoblox::DNS::Member objects for grid secondary servers, Infoblox::DNS::Nameserver objects for external secondary servers, and Infoblox::DNS::MSServer objects for Microsoft DNS servers. The default value is empty.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get secondaries
my $secondaries = $zone->secondaries();
#Modify secondaries
my $memberns1 = Infoblox::DNS::Member->new(
        name     => "member1.server.com",
        ipv4addr => "1.1.1.1",
    );

my $nameserver1 = Infoblox::DNS::Nameserver->new(
        name     => "ns1.exter.com",
        ipv4addr => "11.11.11.11",
    );

$zone->secondaries([$memberns1, $nameserver1]);
```

resource_record_monitored_since( )

Use this method to retrieve the time data collection for "DNS Resource Records Last Queried" was turned on for the zone.

Returns

The method returns the attribute value, the number of seconds that have elapsed since January 1st, 1970 UTC.

Example

```
#Get resource_record_monitored_since
my $resource_record_monitored_since = $zone->resource_record_monitored_since();
```

soa_default_ttl( )

Use this method to set or retrieve the soa_default_ttl flag of a DNS zone.

You can set the Time to Live (TTL) value of the SOA record of a zone. This value is the number of seconds that data is cached.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

A 32-bit integer (range from 0 to 4294967295) that represents the soa_default_ttl in seconds. The default value is undefined.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get soa_default_ttl
my $soa_default_ttl = $zone->soa_default_ttl();
#Modify soa_default_ttl
$zone->soa_default_ttl("3600");
```

soa_expire( )

Use this method to set or retrieve the soa_expire flag of a DNS zone.

This setting defines the amount of time, in seconds, after which the secondary server stops giving out answers about the zone because the

zone data is too old to be useful. The default is one week.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

A 32-bit integer (range from 0 to 4294967295) that represents the soa_expire in seconds. The default value is undefined.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get soa_expire
my $soa_expire = $zone->soa_expire();
#Modify soa_expire
$zone->soa_expire("3600");
```

soa_mname( )

Use this method to set or retrieve the soa_mname flag of a DNS zone.

The Infoblox appliance allows you to change the name of the primary server on the SOA record that is automatically created when you initially configure a zone. Use this method to change the name of the primary server on the SOA record. For example, you may want to hide the primary server for a zone. If your device is named dns1.zone.tld, and for security reasons, you want to show a secondary server called dns2.zone.tld as the primary server. To do so, you would go to dns1.zone.tld zone (being the true primary) and change the primary server on the SOA to dns2.zone.tld to hide the true identity of the real primary server.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

The attribute value can be in unicode format.

Parameter

Primary server name in FQDN (Fully Qualified Domain Name) format. The default value is undefined.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get soa_mname
my $soa_mname = $zone->soa_mname();
#Modify soa_mname
$zone->soa_mname("soamname.com");
```

soa_negative_ttl( )

Use this method to set or retrieve the soa_negative_ttl flag of a DNS zone.

The negative Time to Live (TTL) value of the SOA of the zone indicates how long a secondary server can cache data for "Does Not Respond" responses.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

A 32-bit integer (range from 0 to 4294967295) that represents the soa_negative_ttl in seconds. The default value is undefined.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get soa_negative_ttl
my $soa_negative_ttl = $zone->soa_negative_ttl();
#Modify soa_negative_ttl
$zone->soa_negative_ttl("3600");
```

soa_refresh( )

Use this method to set or retrieve the soa_refresh flag of a DNS zone.

This indicates the interval at which a secondary server sends a message to the primary server for a zone to check that its data is current, and retrieve fresh data if it is not.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

A 32-bit integer (range from 0 to 4294967295) that represents the soa_refresh in seconds. The default value is undefined.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get soa_refresh
my $soa_refresh = $zone->soa_refresh();
#Modify soa_refresh
$zone->soa_refresh("1000");
```

soa_retry( )

Use this method to set or retrieve the soa_retry flag of a DNS zone.

This indicates how long a secondary server must wait before attempting to recontact the primary server after a connection failure between the two servers occurs.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

A 32-bit integer (range from 0 to 4294967295) that represents the soa_retry in seconds. The default value is undefined.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get soa_retry
my $soa_retry = $zone->soa_retry();
#Modify soa_retry
$zone->soa_retry("1000");
```

soa_serial_number( )

Use this method to set or retrieve the soa_serial_number flag of a DNS zone.

The serial number in the SOA record incrementally changes every time the record is modified. The Infoblox appliance allows you to change the serial number (in the SOA record) for the primary server so it is higher than the secondary server, thereby ensuring zone transfers come from the primary server (as they should).

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

A 32-bit integer (range from 0 to 4294967295) that represents the soa serial number. The default value is 1.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get soa_serial_number
my $soa_serial_number = $zone->soa_serial_number();
#Modify soa_serial_number
$zone->soa_serial_number("222222");
```

shared_record_groups( )

Use this method to set or retrieve the associated shared record groups of a DNS zone.

If a shared record group is associated with a zone, then all shared records in a shared record group will be shared in the zone. For more information about shared record groups and shared records, please refer to Infoblox Administrator Guide.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

Array reference containing the name of shared record groups associated with this zone. The name of shared record group is in string format. The default value is undefined.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get shared record groups
my $srgs = $zone->shared_record_groups();
#Modify shared record groups. Assume "test_srg" and "another_srg" are the name
#of already defined shared record groups.
$zone->shared_record_groups(["test_srg", "another_srg"]);
```

stub_external_ns_group( )

Use this method to set or retrieve the external nameservers group name for stub DNS zone.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

The valid value is a desired external name servers group name in a string format.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Construct external nameserver
my $ns1 = Infoblox::DNS::Nameserver->new(
    name     => 'ns1.domain0.extra',
    ipv4addr => '5.5.5.5',
);

#Construct external nameservers group
my $ext = Infoblox::DNS::Nsgroup::ForwardStubServer->new(
    name             => 'extsrvg1',
    external_servers => [$ns1],
);

#Submit created stub_external nameservers group to session
$session->add($ext);

#Get stub_external_ns_group value
my $stub_external_ns_group = $zone->stub_external_ns_group();

#Modify stub_external_ns_group value
$zone->stub_external_ns_group('extsrvg1');
```

stub_from( )

Use this method to set or retrieve the stub_from servers of a DNS zone.

This method specifies the primary servers (masters) of the stub zone.

User can also specify grid member server IP/name pair in this list of primary servers.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

The valid value is an array reference that contains Infoblox::DNS::Nameserver object(s). The default value is undefined.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get stub_from
my $stub_from = $zone->stub_from();
#Modify stub_from
my $nameserver1 = Infoblox::DNS::Nameserver->new(
        name     => "ns1.stub.com",
        ipv4addr => "11.11.11.11",
    );
my $nameserver2 = Infoblox::DNS::Nameserver->new(
        name     => "test",
        ipv4addr => "2006::b671",
    );
#Specify a grid member server IP and name
my $nameserver3 = Infoblox::DNS::Nameserver->new(
        name     => "ns.member.com",
        ipv4addr => "192.168.1.2",
    );
$zone->stub_from([$nameserver1, $nameserver2, $nameserver3]);
```

stub_members( )

Use this method to set or retrieve the stub_members flag of a DNS zone.

This method specifies the grid member or Microsoft DNS servers of the stub zone.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

The valid value is an array reference that contains Infoblox::DNS::Member `object(s)` for grid DNS servers and Infoblox::DNS::MSServer `object(s)` for Microsoft DNS Servers. The default value is undefined.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get stub_members
my $stub_members = $zone->stub_members();
#Modify stub_members
my $memberns1 = Infoblox::DNS::Member->new(
        name     => "member1.stub.com",
        ipv4addr => "3.3.3.3",
    );

my $memberns2 = Infoblox::DNS::Member->new(
        name     => "member2.stub.com",
        ipv4addr => "4.4.4.4",
    );

$zone->stub_members([$memberns1,$memberns2 ]);
```

stub_ns_group( )

Use this method to set or retrieve the name servers group for the Stub DNS Zone object.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

The valid values are string that contains name of the group or Infoblox::Grid::DNS::Nsgroup::StubMember.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Create stub nameservers group
my $stub_group = Infoblox::Grid::DNS::StubMember->new(
    name         => 'stub_ns_group_A',
    stub_members => ['infoblox.localdomain'],
);

#Submit created stub nameserver group to session
$session->add($stub_group);

#Get stub_ns_group
my $stub_ns_group = $zone->stub_ns_group();

#Modify stub_ns_group
$zone->stub_ns_group('stub_ns_group_A');
```

views( )

Use this method to set or retrieve the views flag of a DNS zone.

You can list the views in which the zone is located.

When the `ms_managed()` flag is not None, the method is limited to retrieving the list of DNS views only. Setting the DNS view of a zone served by a Microsoft DNS server is done from the Microsoft server.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

The valid value is an array reference that contains the Infoblox::DNS::View object. The default value is the "default" view.

Note that the array size must be 1.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get views
my $views = $zone->views();
#Modify views
my $internal_view = Infoblox::DNS::View->new(
                "name"    => "internal",
                "comment" => "insert internal",
                "disable" => "false",
                );

$zone->views([$internal_view]);
```

zone( )

Use this method to retrieve the parent zone value of a DNS zone. A parent zone can be of any type, except that a delegated child zone can not have a parent zone of delegated type. The zone name can be empty.

Parameter

None

Returns

The method returns the attribute value.

Example

```
#Get zone
my $zone1 = $zone->zone();
```

zone_monitored_since( )

Use this method to retrieve the time data collection for "DNS Zones Last Queried" was turned on for the zone.

Returns

The method returns the attribute value, the number of seconds that have elapsed since January 1st, 1970 UTC.

Example

```
#Get zone_monitored_since
my $zone_monitored_since = $zone->zone_monitored_since();
```

SAMPLE CODE

The following sample code demonstrates the different functions that can be applied to an object, such as add, search, modify, and remove. This sample also includes error handling for the operations.

#Preparation prior to a zone object insertion

```
#PROGRAM STARTS: Include all the modules that will be used
use strict;
use Infoblox;

#Create a session to the Infoblox appliance
my $host_ip =  "192.168.1.2";
my $session = Infoblox::Session->new(
    master   => $host_ip,
    username => "admin",
    password => "infoblox"
    );
unless($session){
        die("Constructor for session failed: ",
               Infoblox::status_code(). ":" . Infoblox::status_detail());
}
print "Session created successfully.\n";
```

#Create a DNS zone.

```
my $memberns1 = Infoblox::DNS::Member->new(
    name     => "infoblox.localdomain",
    ipv4addr => $host_ip,
    lead     => "false",
    stealth  => "false"
);

my $nameserver1 = Infoblox::DNS::Nameserver->new(
    name     => "ns1.test.com",
    ipv4addr => "1.1.1.1",
    stealth  => "false",
    TSIGname => "tsig_name",
    TSIGkey  => "ZX1jVJl7C58GT/sc7Q3ucA==",
);

my $nameserver2 = Infoblox::DNS::Nameserver->new(
    name     => "ns2.test.com",
    ipv4addr => "2.2.2.2",
    stealth  => "false",
    TSIGname => "tsig_name2",
    TSIGkey  => "qwqwqwqwqwqwqwqwqwqA==",
);

my $default_view = Infoblox::DNS::View->new(name => "default");

#To add zone test.com in the default view
my $firstzone = Infoblox::DNS::Zone->new(
    name        => "test.com",
    views       => [$default_view],
    email       => "admin\@infoblox.com",
    comment     => "add a zone test.com",
    multiple_primaries => [$memberns1],
    secondaries => [$nameserver1, $nameserver2],
    );

unless($firstzone){
       die("Construct test.com zone object failed: ",
               Infoblox::status_code(). ":" .Infoblox::status_detail());
       }
print "test.com zone object created successfully.\n";

$session->add($firstzone)
    or die("Add zone for test.com failed: ",
               $session->status_code(). ":" .$session->status_detail());
```

```
print"Zone test.com added successfully.\n";

#Create a reverse zone.
my $revzone1 = Infoblox::DNS::Zone->new(
    name    => "10.1.4.0/24",
    email   => "admin\@infoblox.com",
    comment => "add a zone 4.168.192.in-addr.arpa",
    multiple_primaries => [$memberns1],
    );

unless($revzone1){
     die("Constructor for zone 4.1.10.in-addr.arpa failed: ",
                Infoblox::status_code(). ":" .Infoblox::status_detail());
       }
print "4.1.10.in-addr.arpa zone object created successfully.\n";

$session->add($revzone1)
    or die("Add zone for 4.1.10.in-addr.arpa failed: ",
                $session->status_code(). ":" .$session->status_detail());

print"Zone 4.1.10.in-addr.arpa added successfully.\n";

#Create a reverse zone with a prefix.
my $revzone2 = Infoblox::DNS::Zone->new(
    name    => "10.168.192.in-addr.arpa",
    email   => "admin\@infoblox.com",
    comment => "add a zone 0-63.10.168.192.in-addr.arpa",
    prefix  => "0-63",
    );

unless($revzone2){
        die("Constructor for zone 0-63.10.168.192.in-addr.arpa failed: ",
               Infoblox::status_code(). ":" .Infoblox::status_detail());
       }
print "0-63.10.168.192.in-addr.arpa zone object created successfully.\n";

$session->add($revzone2)
    or die("Add zone for 0-63.10.168.192.in-addr.arpa failed: ",
              $session->status_code(). ":" .$session->status_detail());

print"Zone 0-63.10.168.192.in-addr.arpa added successfully.\n";
```

#Search for a specific DNS zone

```
#Searches zones that ends with com
my @search_result = $session->search(
    object => "Infoblox::DNS::Zone",
    name   => ".*\\.com"
);

my $search_obj = $search_result[0];
unless($search_obj){
       die("Search zone failed: ",
                $session->status_code() . ":" . $session->status_detail());
       }

print "Search zone object found at least 1 matching entry\n";
```

#Get and modify a DNS zone

```
#Get zone test.com through session
my @result_array = $session->get(
    object => "Infoblox::DNS::Zone",
    name   => "test.com",
    view   => "default"
);
my $for_obj = $result_array[0];
unless($for_obj){
       die("Get zone test.com failed: ",
       $session->status_code() . ":" . $session->status_detail());
       }
print "Get test.com zone object found at least 1 matching entry\n";

#Modify one of the attributes of the specified zone
$for_obj->comment("this is a modified comment.");

#Apply the change
```

```
    $session->modify($result_array[0])
        or die("Modify zone for test.com failed: ",
               $session->status_code(). ":" .$session->status_detail());

 print "test.com zone object modified successfully.\n";

#Get and modify zone 4.1.10.in-addr.arpa.
my @result = $session->get(
    object => "Infoblox::DNS::Zone",
    name   => "4.1.10.in-addr.arpa"
);

my $rev_object = $result[0];
unless ($rev_object) {
       die("Get zone 4.1.10.in-addr.arpa failed: ",
                $session->status_code() . ":" . $session->status_detail());
       }
print "Get 4.1.10.in-addr.arpa zone object found at least 1 matching entry\n";

#Modify one of the attributes of the specified zoe
$rev_object->secondaries([$nameserver2]);

#Apply the changes
$session->modify($rev_object)
            or die("Modify reverse zone 4.1.10.in-addr.arpa failed: ",
                        $session->status_code(). ":" .$session->status_detail());

print "4.1.10.in-addr.arpa zone object modified successfully.\n";
```

#Remove a DNS zone

```
#Remove zone test.com from a view.
$session->remove(
    object => "Infoblox::DNS::Zone",
    name   => "test.com",
    views  => [$default_view]
    )
    or die("Remove zone test.com failed: ",
             $session->status_code(). ":" .$session->status_detail());

print "Remove zone test.com successfull\n";

#Remove zone 4.1.10.in-addr.arpa from a view.
$session->remove(
    object => "Infoblox::DNS::Zone",
    name   => "4.1.10.in-addr.arpa",
    views  => [$default_view]
    )
    or die("Remove zone 4.1.10.in-addr.arpa failed: ",
             $session->status_code(). ":" .$session->status_detail());

print "Remove zone 4.1.10.in-addr.arpa successfull\n";

#Remove zone with prefix from a view.
$session->remove(
    object => "Infoblox::DNS::Zone",
    name    => "10.168.192.in-addr.arpa",
    prefix  => "0-63",
   )
   or die("Remove zone 0-63.10.168.192.in-addr.arpa failed: ",
             $session->status_code(). ":" .$session->status_detail());

print "Remove zone 0-63.10.168.192.in-addr.arpa successfull\n";

####PROGRAM ENDS####
```

AUTHOR

Infoblox Inc. http://www.infoblox.com/

SEE ALSO

Infoblox::DNS::Member, Infoblox::DNS::Nameserver, Infoblox::DNS::SRG, Infoblox::DNS::TSIGKey, Infoblox::DNS::View, Infoblox::Session, Infoblox::Session->add(), Infoblox::Session->get(), Infoblox::Session->modify(), Infoblox::Session->remove(), Infoblox::Session->search(), Infoblox::DNS::Member::SoaSerial, Infoblox::DNS::Member::Mname,

Infoblox::DNS::DNSSecKeyAlgorithm, Infoblox::Grid::MSServer::DCNSRecordCreation, Infoblox::Grid::DNS::Nsgroup, Infoblox::Grid::DNS::Nsgroup, Infoblox::Grid::DNS::Nsgroup,

---

COPYRIGHT

Copyright (c) 2017 Infoblox Inc.

Infoblox::DHCP::FixedAddr - DHCP Fixed Address object.

- NAME
- DESCRIPTION
- CONSTRUCTOR
- SESSION METHODS
  - Infoblox::Session->add( )
  - Infoblox::Session->get( )
  - Infoblox::Session->modify( )
  - Infoblox::Session->remove( )
  - Infoblox::Session->search( )
- METHODS
  - allow_telnet( )
  - bootfile( )
  - agent_circuit_id( )
  - agent_remote_id( )
  - always_update_dns( )
  - bootserver( )
  - client_identifier_prepend_zero( )
  - cli_credentials( )
  - comment( )
  - configure_for_dhcp( )
  - cloud_info( )
  - ddns_domainname( )
  - ddns_hostname( )
  - deny_bootp( )
  - device_description( )
  - device_location( )
  - device_type( )
  - device_vendor( )
  - dhcp_client_identifier( )
  - disable( )
  - discover_now_status( )
  - discovered_data( )
  - discovered_mac( )
  - discovered_name( )
  - discoverer( )
  - enable_ddns( )
  - extattrs( )
  - extensible_attributes( )
  - first_discovered( )
  - ignore_dhcp_option_list_request( )
  - ipv4addr( )
  - is_invalid_mac( )
  - last_discovered( )
  - logic_filters( )
  - mac( )
  - match_client( )
  - member( )
  - ms_ad_user_data( )
  - name( )
  - netbios( )
  - ms_options( )
  - network( )
  - network_component_description( )
  - network_component_ip( )
  - network_component_name( )
  - network_component_port_description( )
  - network_component_port_name( )
  - network_component_port_number( )
  - network_component_type( )
  - network_view( )
  - nextserver( )
  - options( )
  - os( )
  - override_ddns_domainname( )
  - override_cli_credentials( )
  - override_logic_filters( )
  - port_duplex( )
  - port_link_status( )
  - port_speed( )
  - port_status( )
  - port_vlan_description( )

- port_vlan_name( )
- port_vlan_number( )
- pxe_lease_time( )
- reserved_interface( )
- template( )
- v_cluster( )
- v_datacenter( )
- v_host( )
- v_name( )
- v_netadapter( )
- v_switch( )
- v_type( )

- SAMPLE CODE
- AUTHOR
- SEE ALSO
- COPYRIGHT

NAME

Infoblox::DHCP::FixedAddr - DHCP Fixed Address object.

DESCRIPTION

A Fixed Address is a specific IP address that a DHCP server always assigns when a lease request comes from a particular MAC address of the client.

CONSTRUCTOR

```
 my $fixed_addr = Infoblox::DHCP::FixedAddr -> new(
       ipv4addr                          => $ipv4addr,                  #Required
       agent_circuit_id                  => $circuit_id,                #Optional / Default is undefined
       agent_remote_id                   => $remote_id,                 #Optional / Default is undefined
       always_update_dns                 => "true" | "false",           #Optional / Default is "false"
       bootfile                          => $filename | undef,          #Optional / Default is undefined
       bootserver                        => $ipv4addr | $fqdn | undef,  #Optional / Default is undefined
       client_identifier_prepend_zero    => "true" | "false",           #Optional / Default is "false"
       comment                           => $string,                    #Optional / Default is empty
       configure_for_dhcp                => "true" | "false",           #Optional / Default is "false"
                                                                        # When both the IP address and MAC
address are present, the default value is "true"
                                                                        # Applicable only when adding this
fixed address to a host
       ddns_domainname                   => $string,                    #Optional / Default is empty
       ddns_hostname                     => $string,                    #Optional / Default is empty
       deny_bootp                        => "true" | "false" | undef,   #Optional / Default is undefined
       dhcp_client_identifier            => $string,                    #Optional / Default is empty
       disable                           => "true" | "false",           #Optional / Default is "false"
       enable_ddns                       => "true" | "false",           #Optional / Default is "false"
       extattrs                          => { $string => $extattr, ... },  #Optional / Default is undefined
       extensible_attributes             => { $string => $string | $num, $string => [ $string | $num, ... ],
... } #Optional / Default is undefined
       ignore_dhcp_option_list_request   => "true" | "false" | undef,   #Optional / Default is "false"
       last_discovered                   => $string,                    #Optional / Default is empty
       mac                               => $mac_addr,                  #Optional / Default is empty
       match_client                      => $string,                    #Optional / Default is "MAC"
                                                                        #Possible values are:
                                                                        #
"MAC","CLIENT_IDENTIFIER","RESERVED",
                                                                        # "CIRCUIT_ID", "REMOTE_ID"
       member                            => $msmember,                  #Optional
       ms_options                        => [$msopt1, $msopt2, ...]     #Optional / Default is empty
       name                              => $name,                      #Optional
       network                           => $network,                   #Optional
       network_view                      => $NetworkView,               #Optional / Default is "default"
network view
       nextserver                        => $ipv4addr | $fqdn | undef,  #Optional / Default is undefined
       options                           => [$option1, $option2, ...]   #Optional / Default is empty
       override_ddns_domainname          => "true" | "false",           #Optional / Default value is "false"
       pxe_lease_time                    => $time,                      #Optional / Default is empty
       template                          => $string,                    #Optional / Default is undefined
       cli_credentials                   => [$cli1, ...],               #Optional
       override_cli_credentials          => "true" | "false",           #Optional / Default is "false"
       reserved_interface                => $interface,                 #Optional
       device_type                       => $string,                    #Optional
```

```
        device_vendor                   => $string,                     #Optional
        device_location                 => $string,                     #Optional
        device_description              => $string,                     #Optional
        allow_telnet                    => "true" | "false",            #Optional / Default is "false"
        logic_filters                   => [ $filter_name | $filter_obj, ...], # Optional / Default is
undefined
        override_logic_filters          => "true' | 'false",                    # Optional / Default value is
"false"
 );
```

You cannot set both extattrs and extensible_attributes attributes at the same time.

SESSION METHODS

This section describes all the methods in Infoblox::Session module that you can apply to a DHCP Fixed Address object.

Infoblox::Session->add( )

Use this method to add a DHCP Fixed Address object to the Infoblox appliance. See Infoblox::Session->add() for parameters and return values.

Example

```
        #Construct a DHCP Fixed Address object
        my $fixed_addr = Infoblox::DHCP::FixedAddr->new(
                "ipv4addr"                          => "10.0.0.5",
                "bootfile"                          => "bootfile",
                "bootserver"                        => "bootserver.com",
                "client_identifier_prepend_zero"    => "true",
                "comment"                           => "add fixed address using uid",
                "ddns_domainname"                   => "ddns.domain.com",
                "ddns_hostname"                     => "ddns.host.domain.com",
                "deny_bootp"                        => "false",
                "dhcp_client_identifier"            => "foo",
                "disable"                           => "false",
                "enable_ddns"                       => "false",
                "match_client"                      => "MAC",
                "network"                           => "10.0.0.0/8",
                "network_view"                      => $nview,
                "nextserver"                        => "3.3.3.3",
                "options"                           => [ $option1,  $option2, $option3,  $option4, $option5,
      $option6 ],
                "pxe_lease_time"                    => "3600",
         );

         #Submit for addition
         my $response = $session->add($fixed_addr)
```

Infoblox::Session->get( )

Use this method to retrieve all the matching objects from the Infoblox appliance. See Infoblox::Session->get() for parameters and return values.

'discover_now_status' will not be returned by default, use 'return_methods' to request it if needed.

Key References

```
        Apply the following attributes to get a specific DHCP Fixed Address object:

          ipv4addr                - Optional. IPV4 address of the fixed address.
          mac                     - Optional. MAC address of the fixed address.
          dhcp_client_identifier  - Optional, DHCP option 61 client identifier.
          network                 - Optional. Network address of the fixed address.
          network_view            - Optional. Network view in which the fixed address is located. By default,
      all network views are searched.
          extattrs                - Optional. A hash reference containing extensible attributes.
          extensible_attributes   - Optional. A hash reference containing extensible attributes.
          v_cluster               - Optional. Discovered VMware cluster name.
          v_datacenter            - Optional. Discovered VMware datacenter name.
          v_host                  - Optional. Discovered VMware server name.
          v_name                  - Optional. Discovered VMware host or virtual machine name.
          v_netadapter            - Optional. Discovered VMware physical network adapter name.
          v_switch                - Optional. Discovered VMware virtual switch name.
          v_type                  - Optional. Discovered VMware entity type (possible values:
      "VirtualMachine", "HostSystem", "VirtualCenter").
```

Examples

```
        #Get DHCP Fixed Address object through the session
```

```
my @retrieved_objs = $session->get(
    object => "Infoblox::DHCP::FixedAddr",
    mac    => "11:11:11:11:11.11",
);
my @retrieved_objs = $session->get(
    object => "Infoblox::DHCP::FixedAddr",
    dhcp_client_identifier => "foo",
);
my @retrieved_objs = $session->get(
    object       => "Infoblox::DHCP::FixedAddr",
    ipv4addr     => "10.0.0.4",
    network_view => "default",
);

my @retrieved_objs = $session->get(
   object                => "Infoblox::DHCP::FixedAddr",
   extensible_attributes => { 'Site' => 'Santa Clara' });

 # get FixedAddr by v_type:
 my @retrieved_objs = $session->get(
   object    => 'Infoblox::DHCP::FixedAddr',
   v_type    => 'VirtualMachine');
```

Infoblox::Session->modify( )

Use this method to modify a DHCP Fixed Address object in the Infoblox appliance. See Infoblox::Session->modify() for parameters and return values.

Example

```
#Use method to modify the pxe_lease_time
$fixed_address->pxe_lease_time("1234");
#Submit modification
my $response = $session->modify( $fixed_address );
```

Infoblox::Session->remove( )

Use this method to remove a DHCP Fixed Address object from the Infoblox appliance. See Infoblox::Session->remove() for parameters and return values.

To remove a specific object, first use `get()` or `search()` to retrieve the specific DHCP Fixed Address object, and then submit this object for removal.

Example

```
#Get DHCP Fixed Address object through the session
my @retrieved_objs = $session->get(
    object       => "Infoblox::DHCP::FixedAddr",
    mac          => "11:11:11:11:11:11",
    network_view => "default"
);
#Find the desired object from the retrieved list.
my $desired_fixed_addr = $retrieved_objs[0];
#Submit for removal
my $response = $session->remove( $desired_fixed_addr );
```

Infoblox::Session->search( )

Use this method to search for DHCP Fixed Address object and matching Host object with MAC address in the Infoblox appliance (see below). See Infoblox::Session->search() for parameters and return values.

Note that if the Infoblox library is invoked with the :hostaddress option, only DHCP Fixed Address objects are searched.

'discover_now_status' will not be returned by default, use 'return_methods' to request it if needed.

Key References

```
Apply the following attributes to search for a specific DHCP Fixed Address object:

  name          - Optional. Name of the fixed address. If name is passed it implies no_hostaddresses
  ipv4addr      - Optional. IPV4 address of the fixed address (regular expression).
  mac           - Optional. MAC address of the fixed address (regular expression).
  network       - Optional. Network address of the fixed address.
  network_view  - Optional. Network view in which the fixed address is located. By default, all
network views are searched.
  extattrs      - Optional. A hash reference containing extensible attributes.
  extensible_attributes - Optional. A hash reference containing extensible attributes.
  v_cluster     - Optional. Discovered VMware cluster name (regular expression).
  v_datacenter  - Optional. Discovered VMware datacenter name (regular expression).
```

```
  v_host        - Optional. Discovered VMware server name (regular expression).
  v_name        - Optional. Discovered VMware host or virtual machine name (regular expression).
  v_netadapter  - Optional. Discovered VMware physical network adapter name (regular expression).
  v_switch      - Optional. Discovered VMware virtual switch name (regular expression).
  v_type        - Optional. Discovered VMware entity type (possible values: "VirtualMachine",
"HostSystem", "VirtualCenter").
  no_hostaddresses - Optional. If specified and set to "true", only Fixed Address objects will be
returned by the search.
```

For more information about searching extensible attributes, see Infoblox::Grid::ExtensibleAttributeDef/Searching Extensible Attributes.

Examples

```
# Search for all DHCP Fixed Address objects that are specified in the network "10.0.0.0/8" in the
"default" network view.
my @retrieved_objs = $session->search(
    object       => "Infoblox::DHCP::FixedAddr",
    mac          => "11:11:11:11:11.*",
    network      => "10.0.0.0/8",
    network_view => "default"
);

# Search for all DHCP Fixed Address objects with "Santa Clara" for the "Site" extensible attribute
my @retrieved_objs = $session->search(
    object               => "Infoblox::DHCP::FixedAddr",
    extensible_attributes => { 'Site' => 'Santa Clara' }
);

# search FixedAddr by v_datacenter:
my @retrieved_objs = $session->search(
   object               => 'Infoblox::DHCP::FixedAddr',
   v_datacenter => 'test.*');
```

METHODS

This section describes all the methods that you can use to set and retrieve the attribute values of a DHCP Fixed Address object.

allow_telnet( )

Use this method to set or retrieve the flag that enables copying SSH credential to TELNET.

Parameter

Specify 'true' to copy SSH credential to TELNET or 'false' to disable it. The default value is 'false'.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
# Get attribute value
my $value = $fixed_addr->allow_telnet();
# Modify attribute value
$fixed_addr->allow_telnet('true');
```

bootfile( )

Use this method to set or retrieve the bootfile value of a DHCP Fixed Address object.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

This option is overridden independently from Infoblox::DHCP::FixedAddr->bootserver() and Infoblox::DHCP::FixedAddr->nextserver().

Parameter

The name of the file that the client must download. The default value is undefined.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get bootfile
my $bootfile = $fixed_addr->bootfile();
#Modify bootfile
$fixed_addr->bootfile("boot_file");
#Un-override bootfile
$fixed_addr->bootfile(undef);
```

agent_circuit_id( )

Use this method to set or retrieve the agent circuit ID of a DHCP Fixed Address object.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

Use the agent_circuit_id to assign fixed addresses to DHCP clients from whom the member DHCP server receives DHCPDISCOVER and DHCPREQUEST messages that include option 82. To enable the member DHCP server to match the agent circuit ID, set the "match_client" field to "CIRCUIT_ID".

Returns

If you specified a parameter, the method returns true when the modification succeeds and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get agent_circuit_id
my $uid = $fixed_addr->agent_circuit_id();
#Modify agent_circuit_id
$fixed_addr->agent_circuit_id("foo");
```

agent_remote_id( )

Use this method to set or retrieve the agent remote ID of a DHCP Fixed Address object.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

Use the agent_remote_id to assign fixed addresses to DHCP clients from whom the member DHCP server receives DHCPDISCOVER and DHCPREQUEST messages that include option 82. To enable the member DHCP server to match the agent remote ID, set the "match_client" field to "REMOTE_ID".

Returns

If you specified a parameter, the method returns true when the modification succeeds and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get agent_remote_id
my $uid = $fixed_addr->agent_remote_id();
#Modify agent_remote_id
$fixed_addr->agent_remote_id("foo");
```

always_update_dns( )

Use this method to set or retrieve the always_update_dns flag of a DHCP Fixed Address object.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

This flag only applies if the fixed address is associated with a Microsoft (r) server. If it is set to "true", the DHCP server will always update DNS, regardless of the requests from the DHCP clients. If this flag is set to false, clients will be allowed to update DNS when they request it.

Parameter

Specify "true" to set the always_update_dns flag or "false" to deactivate/unset it. The default value is "false".

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get always_update_dns
my $always_update_dns = $fixed_addr->always_update_dns();
#Modify always_update_dns
$fixed_addr->always_update_dns("false");
```

bootserver( )

Use this method to set or retrieve the bootserver of a DHCP Fixed Address object.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

This option is overridden independently from Infoblox::DHCP::FixedAddr->bootfile() and Infoblox::DHCP::FixedAddr->nextserver().

Parameter

The boot server IP address or name in FQDN (Fully Qualified Domain Name) format. The FQDN consists of the host name followed by the domain name (example: abc.com). A boot server name can have a maximum of 256 bytes. The default value is undefined.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get bootserver
my $bootserver = $fixed_addr->bootserver();
#Modify bootserver
$fixed_addr->bootserver("bootserver2.com");
#Un-override bootserver
$fixed_addr->bootserver(undef);
```

client_identifier_prepend_zero( )

Use this method prepend "\000" to the dhcp-client-identifier of a fixed address.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

When you set the client_identifier_prepend_zero parameter to "true", it prefixes a 0 (zero) to the DHCP client identifier whenever option 61 is used to assign fixed addresses. The following is an excerpt from "man dhcp-options":

Please be aware that some DHCP clients, when configured with client identifiers that are ASCII text, will prepend a zero to the ASCII text. So you may need to write:

```
option dhcp-client-identifier "\000foo"
```

rather than:

```
option dhcp-client-identifier "foo".
```

Returns

If you specified a parameter, the method returns true when the modification succeeds and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get client_identifier_prepend_zero
my $client_identifier_prepend_zero = $fixed_addr->client_identifier_prepend_zero();
#Modify dhcp_client_identifier
$fixed_addr->client_identifier_prepend_zero("true");
```

cli_credentials( )

Use this method to set or retrieve the discovery CLI credentials.

Setting this method to a defined value implicitly sets the override_cli_credentials method to "true". Setting the parameter to undefined causes the appliance to use the grid default and automatically resets the override_cli_credentials attribute to "false".

Note that when cli_credentials is set to a defined value and override_cli_credentials is set to "false", the last operation takes precedence. Thus the sequence $fixed_addr->cli_credentials([$cli]); $fixed_addr->override_cli_credentials("false"); will set override_cli_credentials to "false", and the sequence $fixed_addr->override_cli_credentials("false"); $fixed_addr->cli_credentials([$cli]); will result in override_cli_credentials="true".

Parameter

Valid value is an array reference that contains Infoblox::Grid::Discovery::CLICredential objects.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get cli_credentials
my $cli_credentials = $fixed_addr->cli_credentials();
#Modify cli_credentials
$fixed_addr->cli_credentials([$cli1, $cli2]);
```

comment( )

Use this method to set or retrieve the descriptive comment of a DHCP Fixed Address object.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

Desired comment in string format with a maximum of 256 bytes.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get comment
my $comment = $fixed_addr->comment();
#Modify comment
$fixed_addr->comment("Modified DHCP Fixed Address object comment");
```

configure_for_dhcp( )

Use this method to set or retrieve the configure_for_dhcp flag of a DHCP Fixed Address object.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

Specify "true" to set the configure_for_dhcp flag or "false" to deactivate/unset it. The default value is "false".

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get configure_for_dhcp
my $configure_for_dhcp = $fixed_addr->configure_for_dhcp();
#Modify configure_for_dhcp
$fixed_addr->configure_for_dhcp("true");
```

cloud_info( )

Use this method to retrieve cloud API related information for the Infoblox::DHCP::FixedAddr object.

Parameter

None

Returns

The method returns the attribute value.

Example

```
# Get cloud_info
my $cloud_info = $object->cloud_info();
```

ddns_domainname( )

Use this method to set or retrieve the DDNS domain name value of a DHCP Fixed Address object.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Setting this method to a defined value implicitly sets the override_ddns_domainname method to "true". Setting the parameter to undefined causes the appliance to use the grid default and automatically resets the override_ddns_domainname attribute to "false".

Note that when ddns_domainname is set to a defined value and override_dddns_domainname is set to "false", the last operation takes precedence. Thus the sequence $object->ddns_domainname("testdomain"); $object->override_ddns_domainname("false"); will set override_ddns_domainname to "false", and the sequence $object->override_ddns_domainname("false"); $object->ddns_domainname("testdomain"); will result in override_ddns_domainname="true".

Parameter

Specify the DDNS domain name.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get the DDNS domain name
my $ddns_domainname = $fixed_addr->ddns_domainname();
#Modify the DDNS domain name
$fixed_addr->ddns_domainname("ddns.domain.com");
```

ddns_hostname( )

Use this method to set or retrieve the DDNS host name value of a DHCP Fixed Address object.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

Specify the DDNS host name.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get the DDNS host name
my $ddns_hostname = $fixed_addr->ddns_hostname();
#Modify the DDNS host name
$fixed_addr->ddns_hostname("host.ddns.domain.com");
```

deny_bootp( )

Use this method to set or retrieve the deny_bootp flag of a DHCP Fixed Address object.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

Specify "true" to set the deny_bootp flag or "false" to deactivate/unset it. The default value is undefined which indicates that this attribute inherits the upper-level setting

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get deny_bootp
my $deny_bootp = $fixed_addr->deny_bootp();
#Modify deny_bootp
$fixed_addr->deny_bootp("true");
```

device_description( )

Use this method to set or retrieve the description of the discovery device.

Parameter

The device description in string format.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get device_description
my $device_description = $fixed_addr->device_description();
#Modify device_description
$fixed_addr->device_description('example');
```

device_location( )

Use this method to set or retrieve the location of the discovery device.

Parameter

The device location in string format.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get device_location
my $device_location = $fixed_addr->device_location();
#Modify device_location
$fixed_addr->device_location('example');
```

device_type( )

Use this method to set or retrieve the type of the discovery device.

Parameter

The device type in string format.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get device_type
my $device_type = $fixed_addr->device_type();
#Modify device_type
$fixed_addr->device_type('example');
```

device_vendor( )

Use this method to set or retrieve the vendor name of the discovery device.

Parameter

The device vendor name in string format.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get device_vendor
my $device_vendor = $fixed_addr->device_vendor();
#Modify device_vendor
$fixed_addr->device_vendor('example');
```

dhcp_client_identifier( )

Use this method to set or retrieve the dhcp-client-identifier value of a DHCP Fixed Address object.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

Use the dhcp_client_identifier or uid to assign fixed addresses if the DHCP clients send option 61. A DHCP client identifier can be a string or a MAC address. Set the "match_client" field to "CLIENT_IDENTIFIER" for fixed addresses to use the dhcp_client_identifier instead of the mac_address.

Returns

If you specified a parameter, the method returns true when the modification succeeds and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get dhcp_client_identifier
my $uid = $fixed_addr->dhcp_client_identifier();
#Modify dhcp_client_identifier
$fixed_addr->dhcp_client_identifier("foo");
or
$fixed_addr->dhcp_client_identifier("1:11::11::11::11::11::12");
```

disable( )

Use this method to set or retrieve the disable flag of a DHCP Fixed Address object.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

Specify "true" to set the disable flag or "false" to deactivate/unset it. The default value is "false".

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get disable
my $disable = $fixed_addr->disable();
#Modify disable
$fixed_addr->disable("true");
```

discover_now_status( )

Use this method to retrieve the current discovery status. This attribute is read-only.

'discover_now_status' will not be returned by default, use 'return_methods' to request it if needed.

Parameter

None.

Returns

The method returns the attribute value that can be 'COMPLETE', 'FAILED', 'NONE', 'PENDING' or 'RUNNING'.

Example

```
#Get discover_now_status
my $discover_now_status = $fixed_addr->discover_now_status();
```

discovered_data( )

Use this method to retrieve the discovered data of a DHCP Fixed Address object. This is a read-only attribute.

Omit the parameter to retrieve the attribute value.

Parameter

None

Returns

The valid return value is an Infoblox::Grid::Discovery::Data object.

Example

```
#Get discovered_data value
my $discovered_data = $object->discovered_data();
```

discovered_mac( )

Use this method to retrieve the discovered MAC address of a DHCP Fixed Address object.

Parameter

none

Returns

The method returns the discovered MAC address.

Example

```
#Get discovered_mac
my $discovered_mac = $fixed_addr->discovered_mac();
```

discovered_name( )

Use this method to retrieve the discovered name of a DHCP Fixed Address object.

Parameter

none

Returns

The method returns the discovered name.

Example

```
#Get discovered_name
my $discovered_name = $fixed_addr->discovered_name();
```

discoverer( )

Use this method to retrieve the discoverer of a DHCP Fixed Address object.

Parameter

none

Returns

The method returns the discoverer value.

Example

```
#Get discoverer
my $discoverer = $fixed_addr->discoverer();
```

enable_ddns( )

Use this method to set or retrieve the enable_ddns flag of a DHCP Fixed Address object.

If enable_ddns value is specified as "true", then the A and PTR records corresponding to the Fixed Address object will be placed on the DNS server.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

Specify "true" to set the enable_ddns flag or "false" to deactivate/unset it. The default value is "false".

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get enable_ddns
my $enable_ddns = $fixed_addr->enable_ddns();
#Modify enable_ddns
$fixed_addr->enable_ddns("true");
```

extattrs( )

Use this method to set or retrieve the extensible attributes associated with a DHCP Fixed Address object.

Parameter

Valid value is a hash reference containing the names of extensible attributes and their associated values (Infoblox::Grid::Extattr objects).

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get extattrs
my $ref_extattrs = $fixed_addr->extattrs();
#Modify extattrs
$fixed_addr->extattrs({ 'Site' => $extattr1, 'Administrator' => $extattr2 });
```

extensible_attributes( )

Use this method to set or retrieve the extensible attributes associated with a DHCP Fixed Address object

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

For valid values for extensible attributes, see Infoblox::Grid::ExtensibleAttributeDef/Extensible Attribute Values.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get extensible attributes
my $ref_extensible_attributes = $fixed_addr->extensible_attributes();
#Modify extensible attributes
$fixed_addr->extensible_attributes({ 'Site' => 'Santa Clara', 'Administrator' => [ 'Peter', 'Tom' ]
});
```

first_discovered( )

Use this method to retrieve the date and time that the DHCP Fixed Address object was first discovered.

Parameter

none

Returns

The method returns the date and time that the DHCP Fixed Address object was first discovered.

Example

```
#Get first_discovered
my $first_discovered = $fixed_addr->first_discovered();
```

ignore_dhcp_option_list_request( )

Use this method to set or retrieve the ignore_dhcp_option_list_request flag of a DHCP Fixed address object. If this flag is set to false all the defined DHCP options will be returned to the client.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

Specify "true" to set the ignore_dhcp_option_list_request flag or "false" to deactivate/unset it. The default value is false which indicates that this attribute inherit the member level setting..

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
# Get ignore_dhcp_option_list_request
my $ignore_dhcp_option_list_request = $fixed_addr->ignore_dhcp_option_list_request();
# Un-override ignore_dhcp_option_list_request
$fixed_addr->ignore_dhcp_option_list_request(undef);
# Modify ignore_dhcp_option_list_request
$fixed_addr->ignore_dhcp_option_list_request("true");
```

ipv4addr( )

Use this method to set or retrieve the ipv4addr value of a DHCP Fixed Address object.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

An IPv4 address is a 32-bit number in dotted decimal notation. It consists of four 8-bit groups of decimal digits separated by decimal points (example: 192.168.1.2).

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get ipv4addr
my $ipv4addr = $fixed_addr->ipv4addr();
#Modify ipv4addr
$fixed_addr->ipv4addr("10.0.0.5");
```

is_invalid_mac( )

Use this method to retrieve the flag used to indicate whether an invalid MAC address is used. This is a read-only attribute.

Parameter

None

Returns

The method returns the attribute value.

Example

```
#Get is_invalid_mac value
my $is_invalid_mac = $fixed_addr->is_invalid_mac();
```

last_discovered( )

Use this method to retrieve the time this object was last seen by a discovery job.

The default value for this field is empty.

Parameter

N/A

Returns

The method returns the attribute value.

Example

```
#Get last_discovered
my $last_discovered = $fixed_addr->last_discovered();
```

logic_filters( )

Use this method to set or retrieve all the logic (ordered) filters associated with the DHCP Fixed Address.

Filters are used to screen address assignments for the DHCP Fixed Address if the addresses are to be assigned to specific hosts.

Setting this method to a defined value implicitly sets the override_logic_filters method to "true". Setting the parameter to undefined causes the appliance to use the grid default and automatically resets the override_logic_filters attribute to "false".

Note that when logic_filters is set to a defined value and override_dlogic_filters is set to "false", the last operation takes precedence. Thus the sequence $object->logic_filters($filters); $object->override_logic_filters("false"); will set override_logic_filters to "false", and the sequence $object->override_logic_filters("false"); $object->logic_filters($filters); will result in override_logic_filters="true".

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

The valid value is an array reference that contains either the Infoblox::DHCP::Filter::MAC, Infoblox::DHCP::Filter::NAC, Infoblox::DHCP::Filter::Option objects or the names of the corresponding filters. The filters in the list must satisfy the following conditions:

Filter name must be globally unique.
Filters without expressions must be at the end of the list.
The filters should not be disabled.

The default value of the parameter is undefined.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get logic_filters
my $ref_logic_filters = $fixed_addr->logic_filters();
#Modify logic_filters
$fixed_addr->logic_filters(["filter_1","filter_2",$exl1]); #$exl1 is an Infoblox::DHCP::Filter::MAC,
Infoblox::DHCP::Filter::NAC or Infoblox::DHCP::Filter::Option object.
```

mac( )

```
Use this method to set or retrieve the MAC address value of a DHCP Fixed Address object.
```

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

MAC Address of the DHCP Fixed address object. Format for the hexadecimal strings for MAC addresses and vendor prefixes with colons or dashes. Both of the following formats are acceptable: 11:11:11:11:11:11 and 11-11-11-11-11-11.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get mac
my $mac = $fixed_addr->mac();
#Modify mac
$fixed_addr->mac("11::11::11::11::11::11::12");
```

match_client( )

Use this method to set or retrieve the match_client value of a DHCP Fixed Address object.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

```
    match_client "MAC": The fixed IP address is leased to the matching MAC address.

    match_client "CLIENT_IDENTIFIER": The fixed IP address is leased to the matching DHCP client
identifier

    match_client "RESERVED": The fixed IP address is reserved for later use with a MAC address that
only has zeros.

    match_client "CIRCUIT_ID": The fixed IP address is leased to the DHCP client with a matching
circuit ID. Note that
                  the "agent_circuit_id" field must be set in this case.

    match_client "REMOTE_ID": The fixed IP address is leased to the DHCP client with a matching
remote ID. Note that
                  the "agent_remote_id" field must be set in this case.
```

Returns

```
    If you specified a parameter, the method returns true when the modification succeeds, and returns
false when the operation fails.

    If you did not specify a parameter, the method returns the attribute value.
```

Example

```
#Get match_client
my $match_client = $fixed_addr->match_client();
#Modify match_client
$fixed_addr->match_client("CLIENT_IDENTIFIER");
```

member( )

Use this method to set or retrieve the Microsoft server associated with this fixed address.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

The valid value is a defined Infoblox::DHCP::MSServer object. The default value is empty.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get member
my $ref_member = $dhcp_fixedaddr->member();
#Modify member
$dhcp_fixedaddr->member($member1); #$member1 is an Infoblox::DHCP::MSServer object
```

ms_ad_user_data( )

Use this method to retrieve Microsoft Active Directory users related information. This is a read-only attribute.

Parameter

None

Returns

The valid return value is an Infoblox::Grid::MSServer::AdUser::Data object.

Example

```
#Get ms_ad_user_data
my $ms_ad_user_data = $dhcp_fixedaddr->ms_ad_user_data();
```

name( )

```
Use this method to set or retrieve the name of a DHCP Fixed Address object.
```

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

A string specifying the name of the fixed address.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get name
my $name = $fixed_addr->name();
#Modify name
$fixed_addr->name("fxname");
```

netbios( )

Use this method to retrieve the name in the NetBIOS reply that responded to a NetBIOS query.

Returns

The method returns the attribute value.

Example

```
#Get netbios
my $netbios = $fixed_addr->netbios();
```

ms_options( )

Use this method to set or retrieve the Microsoft server custom DHCP options associated with this DHCP fixed address.

Note: You can set only options that are specified in both the Microsoft server and NIOS option definitions. Options that are not common to both the Microsoft server and NIOS cannot be modified.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

Valid value is an array reference that contains defined Infoblox::DHCP::MSOption objects. The default value is empty.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get options
my $ref_options = $dhcp_fixedaddr->ms_options();
#Modify options
$dhcp_fixedaddr->ms_options([$option12]); #$option12 is an Infoblox::DHCP::MSOption object
```

network( )

Use this method to retrieve the network value of a DHCP Fixed Address object.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

An network is a 32-bit number in dotted decimal notation. It consists of four 8-bit groups of decimal digits separated by decimal points followed by prefix (example: 1.0.0.0/8).

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get network
my $network = $fixed_addr->network();
```

network_component_description( )

Use this method to retrieve the description of the network device that is connected to the DHCP Fixed Address object.

Parameter

none

Returns

The method returns the network device description.

Example

```
#Get network_component_description
my $network_component_description = $fixed_addr->network_component_description();
```

network_component_ip( )

Use this method to retrieve the IP address of the network device that is connected to the DHCP Fixed Address object.

Parameter

none

Returns

The method returns the IP address of the network device.

Example

```
#Get network_component_ip
my $network_component_ip = $fixed_addr->network_component_ip();
```

network_component_name( )

Use this method to retrieve the name of the network device that is connected to the DHCP Fixed Address object.

Parameter

none

Returns

The method returns the network device name.

Example

```
#Get network_component_name
my $network_component_name = $fixed_addr->network_component_name();
```

network_component_port_description( )

Use this method to retrieve the description of the network device port that is connected to the DHCP Fixed Address object.

Parameter

none

Returns

The method returns the description of the port on the network device.

Example

```
#Get network_component_port_description
my $network_component_port_description = $fixed_addr->network_component_port_description();
```

network_component_port_name( )

Use this method to retrieve the name of the network device port that is connected to the DHCP Fixed Address object.

Parameter

none

Returns

The method returns the name of the port on the network device.

Example

```
#Get network_component_port_name
my $network_component_port_name = $fixed_addr->network_component_port_name();
```

network_component_port_number( )

Use this method to retrieve the number of the network device port that is connected to the DHCP Fixed Address object.

Parameter

none

Returns

The method returns the network device port number.

Example

```
#Get network_component_port_number
my $network_component_port_number = $fixed_addr->network_component_port_number();
```

network_component_type( )

Use this method to retrieve the type of the network device that is connected to DHCP Fixed Address object.

Parameter

none

Returns

The method returns the network device type.

Example

```
#Get network_component_type
my $network_component_type = $fixed_addr->network_component_type();
```

network_view( )

Use this method to set or retrieve the network view of the DHCP fixed address.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

The default value is the "default" network view, which means the fixed address is in the default network view.

Parameter

Valid value is a defined Infoblox::DHCP::View object. The default network view value is the system-defined default network view object.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get network view
my $nview = $fixed_addr->network_view();
#Modify network view, with an Infoblox::DHCP::View object
$fixed_addr->network_view($nview);
```

nextserver( )

Use this method to set or retrieve the nextserver value of a DHCP Fixed Address object.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

This option is overridden independently from Infoblox::DHCP::FixedAddr->bootfile() and Infoblox::DHCP::FixedAddr->bootserver().

Parameter

The next server IP address or name in FQDN (Fully Qualified Domain Name) format. The FQDN consists of the host name followed by the domain name (example: abc.com). A next server name can have a maximum of 256 bytes. The default value is undefined.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get nextserver
my $nextserver = $fixed_addr->nextserver();
#Modify nextserver
$fixed_addr->nextserver("3.3.3.4");
#Un-override nextserver
$fixed_addr->nextserver(undef);
```

options( )

Use this method to set or retrieve the options of a DHCP Fixed Address object.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

See Infoblox::DHCP::Option for parameters and methods available for this object.

Parameter

Options describe network configuration settings and various services available on the network. These options occur as variable-length fields at the end of DHCP messages.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get options
my $options = $fixed_addr->options();
#Modify options
$fixed_addr->options([$option1, $option2]);
```

os( )

Use this method to retrieve the operating system associated with this fixed address.

Returns

The method returns the attribute value.

Example

```
#Get os
my $os = $fixed_addr->os();
```

override_ddns_domainname( )

The override_ddns_domainname attribute controls whether the ddns_domainname value in the DHCP Fixed Address is used, instead of the grid default.

The override_ddns_domainname attribute can be specified explicitly. It is also set implicitly when ddns_domainname is set to a defined value.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

Set the parameter to "true" to override the grid-level setting for ddns_domainname. Set the parameter to "false" to inherit the grid-level setting for ddns_domainname.

The default value of this parameter is "false".

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Getting override_ddns_domainname
my $override_ddns_domainname=$fixed_addr->override_ddns_domainname( );
#Modifying override_ddns_domainname
$fixed_addr->override_ddns_domainname("true");
```

override_cli_credentials( )

The override_cli_credentials attribute controls whether the cli_credentials value in the object is used, instead of the grid default.

The override_cli_credentials attribute can be specified explicitly. It is also set implicitly when cli_credentials is set to a defined value.

Parameter

Specify "true" to set the override_cli_credentials flag or "false" to deactivate/unset it. The default value is "false".

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
# Get attribute value
my $value = $fixed_addr->override_cli_credentials();
# Modify attribute value
$fixed_addr->override_cli_credentials('true');
```

override_logic_filters( )

The override_logic_filters attribute controls whether the logic_filters value of the DHCP Fixed Address is used, instead of the Grid default.

The override_logic_filters attribute can be specified explicitly. It is also set implicitly when logic_filters is set to a defined value.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

Set the parameter to "true" to override the Grid-level setting for logic_filters. Set the parameter to "false" to inherit the Grid-level setting for logic_filters.

The default value of this parameter is "false".

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Getting override_logic_filters
my $override_logic_filters=$fixed_addr->override_logic_filters( );
#Modifying override_logic_filters
$fixed_addr->override_logic_filters("true");
```

port_duplex( )

Use this method to retrieve the duplex setting of the network device port that is connected to the DHCP Fixed Address object.

Parameter

none

Returns

The method returns the port duplex setting. Returned values, if any, are one of the following:

```
Full
Half
```

Example

```
#Get port_duplex
my $port_duplex = $fixed_addr->port_duplex();
```

port_link_status( )

Use this method to retrieve the link status of the network device port that is connected to the DHCP Fixed Address object.

Parameter

none

Returns

The method returns the network device port link status. Returned values, if any, are one of the following:

```
Connected
Not Connected
Unknown
```

Example

```
#Get port_link_status
my $port_link_status = $fixed_addr->port_link_status();
```

port_speed( )

Use this method to retrieve the speed of the network device port that is connected to the DHCP Fixed Address object.

Parameter

none

Returns

The method returns the network device port speed. Returned values, if any, are one of the following:

```
10M
100M
1G
10G
100G
Unknown
```

Example

```
#Get port_speed
my $port_speed = $fixed_addr->port_speed();
```

port_status( )

Use this method to retrieve the status of the network device port that is connected to the DHCP Fixed Address object.

Parameter

none

Returns

The method returns the network device port status. Returned values, if any, are one of the following:

```
Up
Down
Unknown
```

Example

```
#Get port_status
my $port_status = $fixed_addr->port_status();
```

port_vlan_description( )

Use this method to retrieve the VLAN description of the network device port that is connected to the DHCP Fixed Address object.

Parameter

none

Returns

The method returns the VLAN description of the network device port.

Example

```
#Get port_vlan_description
my $port_vlan_description = $fixed_addr->port_vlan_description();
```

port_vlan_name( )

Use this method to retrieve the VLAN name of the network device port that is connected to the DHCP Fixed Address object.

Parameter

none

Returns

The method returns the network device port VLAN name.

Example

```
#Get port_vlan_name
my $port_vlan_name = $fixed_addr->port_vlan_name();
```

port_vlan_number( )

Use this method to retrieve the VLAN number of the network device port that is connected to the DHCP Fixed Address object.

Parameter

none

Returns

The method returns the port VLAN number.

Example

```
#Get port_vlan_number
my $port_vlan_number = $fixed_addr->port_vlan_number();
```

pxe_lease_time( )

Use this method to set or retrieve the pxe_lease_time value of a DHCP Fixed Address object.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

Enter appropriate values in the Days, Hours, Mins and Secs fields. User can specify the duration of time it takes a host to connect to a boot server, such as a TFTP server, and download the file it needs to boot.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get pxe_lease_time
my $pxe_lease_time = $fixed_addr->pxe_lease_time();
#Modify pxe_lease_time
$fixed_addr->pxe_lease_time("369");
```

reserved_interface( )

Use this method to set or retrieve the reserved discovery interface.

Parameter

Valid value is an Infoblox::Grid::Discovery::DeviceInterface object.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get reserved_interface
my $reserved_interface = $fixed_addr->reserved_interface();
#Modify reserved_interface
$fixed_addr->reserved_interface($interface);
```

template( )

Use this method to set or retrieve the fixed address template name for a DHCP Fixed Address object.

Template is the name of a DHCP Fixed Address template which will be used to create the DHCP Fixed Address.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

The valid name in sting format.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

The template parameter can be used to only add a new DHCP Fixed Address and cannot be used during object modify or returned during object get.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get template
my $template = $fixed_addr->template();
#Modify template
$fixed_addr->template("Template_Name");
```

v_cluster( )

Use this method to retrieve the name of the VMware cluster associated with the DHCP Fixed Address object. This method is read-only.

Parameter

none

Returns

This method returns a string that contains the VMware cluster name. The default value is an empty string.

Example

```
#Get v_cluster
my $v_cluster = $fixed_addr->v_cluster();
```

v_datacenter( )

Use this method to retrieve the name of the VMware datacenter associated with the DHCP Fixed Address object. This method is read-only.

Parameter

none

Returns

This method returns a string that contains the VMware datacenter name. The default value is an empty string.

Example

```
#Get v_datacenter
my $v_datacenter = $fixed_addr->v_datacenter();
```

v_host( )

Use this method to retrieve the name of the VMware host associated with the DHCP Fixed Address object. This method is read-only.

Parameter

none

Returns

This method returns a string that contains the VMware host name. The default value is an empty string.

Example

```
#Get v_host
my $v_host = $fixed_addr->v_host();
```

v_name( )

Use this method to retrieve the name of the VMware entity associated with the DHCP Fixed Address object. This method is read-only.

Parameter

none

Returns

This method returns a string that contains the VMware entity name. The default value is an empty string.

Example

```
#Get v_name
my $v_name = $fixed_addr->v_name();
```

v_netadapter( )

Use this method to retrieve the name of the physical network adapter through which the VMware entity is connected to the appliance. This method is read-only.

Parameter

none

Returns

This method returns a string that contains the network adapter name. The default value is an empty string.

Example

```
#Get v_netadapter
my $v_netadapter = $fixed_addr->v_netadapter();
```

v_switch( )

Use this method to retrieve the name of the virtual switch through which the VMware entity is connected to the appliance. This method is read-only.

Parameter

none

Returns

This method returns a string that contains the virtual switch name. The default value is an empty string.

Example

```
#Get v_switch
my $v_switch = $fixed_addr->v_switch();
```

v_type( )

Use this method to retrieve the type of VMware entity associated with the DHCP Fixed Address object. This method is read-only.

Parameter

none

Returns

This method returns a string that contains the VMware entity type. The default value is an empty string. The returned value, if any, can be one of the following:

```
VirtualMachine
HostSystem
VirtualCenter
```

Example

```
#Get v_type
my $v_type = $fixed_addr->v_type();
```

SAMPLE CODE

The following sample code demonstrates the different functions that can be applied to an object such as add, search, modify, and remove. This sample also includes error handling for the operations.

#Preparation prior to a DHCP Fixed Address object insertion

```
#PROGRAM STARTS: Include all the modules that will be used
```

```
use strict;
use Infoblox;

#Create a session to the Infoblox appliance

my $session = Infoblox::Session->new(
               master   => "192.168.1.2", #appliance host ip
               username => "admin",       #appliance user login
               password => "infoblox"     #appliance password
);

unless ($session) {
       die("Construct session failed: ",
               Infoblox::status_code() . ":" . Infoblox::status_detail());
}
print "Session created successfully\n";

#Create a network object
my $network = Infoblox::DHCP::Network->new(
                                          "network" => "10.0.0.0/8",
                                         );
unless($network) {
     die("Construct Network object failed: ",
           Infoblox::status_code() . ":" . Infoblox::status_detail());
}
print "Network object created successfully\n";

$session->add($network)
    or die("Add network failed: ",
           $session->status_code() . ":" . $session->status_detail());
print "Network added successfully\n";

my $option1 = Infoblox::DHCP::Option->new(
                                          "name"  => "lease-time",
                                          "value" => "50"
                                         );
unless($option1) {
     die("Construct option1 object failed: ",
           Infoblox::status_code() . ":" . Infoblox::status_detail());
}
print "option1 object created successfully\n";

my $option2 = Infoblox::DHCP::Option->new(
                                          "type"      => "routers",
                                          "ipv4addrs" => [ "10.0.0.1", "10.0.0.255" ],
                                         );
unless($option2) {
     die("Construct option2 object failed: ",
           Infoblox::status_code() . ":" . Infoblox::status_detail());
}
print "option2 object created successfully\n";
```

#Create a DHCP Fixed address object

```
#Create a DHCP Fixed Address object
my $fixed_addr = Infoblox::DHCP::FixedAddr->new(
    "ipv4addr"       => "10.0.0.6",
    "mac"            => "00:00:00:11:11:11",
    "network"        => "10.0.0.0/8",
    "comment"        => "add fixed address",
    "enable_ddns"    => "true",
    "pxe_lease_time" => "3600",
    "bootserver"     => "someserver.com",
    "nextserver"     => "3.3.3.3",
    "bootfile"       => "bootfile.com",
    "options"        => [ $option1 ,  $option2 ],
);
unless($fixed_addr) {
     die("Construct fixed address failed: ",
           Infoblox::status_code() . ":" . Infoblox::status_detail());
}
print "DHCP Fixed address object created successfully\n";

#Verify if the DHCP Fixed Address exists in the Infoblox appliance
my $object = $session->get(
       object => "Infoblox::DHCP::FixedAddr",
```

```
        mac     => "00:00:00:11:11:11"
);
unless ($object) {
   print "fixed address does not exist on server, safe to add the fixed address\n";
   $session->add($fixed_addr)
      or die("Add fixed address failed: ",
             $session->status_code() . ":" . $session->status_detail());
}
print "DHCP Fixed address added successfully\n";
```

#Create a DHCP Fixed Address object using a template

```
#Create a DHCP Fixed Address template object
my $fixed_addr_template = Infoblox::DHCP::FixedAddrTemplate->new(
    "name"                  => "custom_fa_template",
    "offset"                => "10",
    "number_of_addresses"   => "1",
    "comment"               => "add fixed address template",
    "enable_ddns"           => "TRUE",
);
unless($fixed_addr_template) {
     die("Construct fixed address failed: ",
           Infoblox::status_code() . ":" . Infoblox::status_detail());
}
print "DHCP Fixed address template object created successfully\n";

$session->add($fixed_addr_template)
      or die("Add fixed address template failed: ",
             $session->status_code() . ":" . $session->status_detail());
}
print "DHCP Fixed address template added successfully\n";

#Create a DHCP Fixed Address object
my $fixed_addr = Infoblox::DHCP::FixedAddr->new(
    "network"        => "10.0.0.0/8",
    "ipv4addr"       => "10.0.0.7",
    "match_client"   => "RESERVED",
    "template"       => "custom_fa_template"
);
unless($fixed_addr) {
     die("Construct fixed address failed: ",
           Infoblox::status_code() . ":" . Infoblox::status_detail());
}
print "DHCP Fixed address object created successfully\n";

$session->add($fixed_addr)
      or die("Add fixed address failed: ",
             $session->status_code() . ":" . $session->status_detail());
}
print "DHCP Fixed address added successfully\n";
```

#Search for a specific DHCP Fixed Address object

```
#Search for all DHCP Fixed address objects under the network 10.0.0.0/8
my @retrieved_objs = $session->search(
    object  => "Infoblox::DHCP::FixedAddr",
    mac     => "00:00:00:11:11.*",
    network => "10.0.0.0/8" );
my $object = $retrieved_objs[0];

unless ($object) {
       die("Search for Fixed Address object failed: ",
              $session->status_code() . ":" . $session->status_detail());
}
print "Search Fixed Address object found at least 1 matching entry\n";
```

#Get and modify a DHCP Fixed Address object

```
#Get DHCP Fixed Address object through the session
my @retrieved_objs = $session->get(
    object => "Infoblox::DHCP::FixedAddr",
    mac    => "00:00:00:11:11:11",
);
my $object = $retrieved_objs[0];
unless ($object) {
    die("Get Fixed Address object failed: ",
```

```
        $session->status_code() . ":" . $session->status_detail());
}
print "Get Fixed Address object found at least 1 matching entry\n";

#Modify one of the attributes of the obtained Fixed Address object
$object->pxe_lease_time("1234");

#Un-override bootfile
$object->bootfile(undef);

#Un-override bootserver
$object->bootserver(undef);

#Un-override nextserver
$object->nextserver(undef);

#Apply the changes
$session->modify($object)
    or die("Modify Fixed Address object failed: ",
           $session->status_code() . ":" . $session->status_detail());
print "Fixed Address object modified successfully \n";
```

#Remove a Fixed Address object

```
#Get Fixed Address object through the session
my @retrieved_objs = $session->get(
    object => "Infoblox::DHCP::FixedAddr",
    mac    => "00:00:00:11:11:11",
);
my $object = $retrieved_objs[0];
unless ($object) {
    die("Get Fixed Address object failed: ",
        $session->status_code() . ":" . $session->status_detail());
}
print "Get Fixed Address object found at least 1 matching entry\n";

#Submit the object for removal
$session->remove($object)
    or die("Remove Fixed Address object failed: ",
        $session->status_code() . ":" . $session->status_detail());
print "Fixed Address object removed successfully \n";

#Submit the Network object for removal
$session->remove($network)
    or die("Remove Network object failed: ",
        $session->status_code() . ":" . $session->status_detail());
print "Network object removed successfully \n";

#Submit the Fixed Address Template object for removal
$session->remove($fixed_addr_template)
    or die("Remove Fixed Address Template object failed: ",
        $session->status_code() . ":" . $session->status_detail());
print "Fixed Address Template object removed successfully \n";

####PROGRAM ENDS####
```

AUTHOR

Infoblox Inc. http://www.infoblox.com/

SEE ALSO

Infoblox::DHCP::Network, Infoblox::DHCP::View, Infoblox::DHCP::Range, Infoblox::DHCP::Lease, Infoblox::Session->add(), Infoblox::Session->get(), Infoblox::Session->modify(), Infoblox::Session->remove(), Infoblox::Session->search(), Infoblox::Session, Infoblox::DHCP::FixedAddrTemplate, Infoblox::Grid::Discovery::Data

---

COPYRIGHT

Copyright (c) 2017 Infoblox Inc.

Infoblox::DHCP::Network - DHCP Network object

- NAME
- DESCRIPTION
- CONSTRUCTOR
- SESSION METHODS
  - Infoblox::Session->add( )
  - Infoblox::Session->get( )
  - Infoblox::Session->modify( )
  - Infoblox::Session->remove( )
  - Infoblox::Session->search( )
- METHODS
  - authority( )
  - auto_create_reversezone( )
  - bootfile( )
  - bootserver( )
  - comment( )
  - cloud_info( )
  - ddns_domainname( )
  - ddns_generate_hostname( )
  - ddns_server_always_updates( )
  - ddns_ttl( )
  - ddns_update_fixed_addresses( )
  - ddns_use_option81( )
  - deny_bootp( )
  - disable( )
  - discovered_bridge_domain( )
  - discovered_tenant( )
  - discovery_basic_poll_setting( )
  - discovery_exclusion_range( )
  - discovery_member( )
  - discover_now_status( )
  - discovery_blackout_setting( )
  - discovery_engine_type( )
  - email_list( )
  - enable_ddns( )
  - enable_dhcp_thresholds( )
  - enable_email_warnings( )
  - enable_discovery( )
  - enable_ifmap_publishing( )
  - enable_immediate_discovery( )
  - enable_snmp_warnings( )
  - endpoint_sources( )
  - extattrs( )
  - extensible_attributes( )
  - high_water_mark( )
  - high_water_mark_reset( )
  - ignore_id( )
  - ignore_mac_addresses( )
  - ignore_dhcp_option_list_request( )
  - ipam_email_addresses( )
  - ipam_trap_settings( )
  - ipam_threshold_settings( )
  - lease_scavenge_time( )
  - logic_filters( )
  - low_water_mark( )
  - low_water_mark_reset( )
  - members( )
  - mgm_private( )
  - mgm_private_overridable( )
  - ms_ad_user_data( )
  - network( )
  - network_container( )
  - network_view( )
  - nextserver( )
  - next_available_ip( )
  - next_available_network( )
  - options( )
  - override_ddns_domainname( )
  - override_ddns_ttl( )
  - override_enable_ifmap_publishing( )
  - override_blackout_setting( )
  - override_ignore_id( )

- override_ipam_email_addresses( )
- override_ipam_threshold_settings( )
- override_ipam_trap_settings( )
- override_lease_scavenge_time( )
- override_logic_filters( )
- override_mgm_private( )
- override_subscribe_settings( )
- override_update_dns_on_lease_renewal( )
- port_control_blackout_setting
- pxe_lease_time( )
- recycle_leases( )
- resize( )
- restart_if_needed( )
- rir( )
- rir_organization( )
- rir_registration_action( )
- rir_registration_status( )
- last_rir_registration_update_sent( )
- last_rir_registration_update_status( )
- same_port_control_discovery_blackout( )
- send_rir_request( )
- subscribe_settings( )
- delete_reason( )
- template( )
- zone_associations( )
- unmanaged( )
- update_dns_on_lease_renewal( )

- SAMPLE CODE
- AUTHOR
- SEE ALSO
- COPYRIGHT

NAME

Infoblox::DHCP::Network - DHCP Network object

DESCRIPTION

When DHCP services are configured on an appliance, a network must be defined that it serves. After a network is created, you can then create all the subnetworks individually or create a parent network that encompasses all the subnetworks.

CONSTRUCTOR

```
  my $network = Infoblox::DHCP::Network->new(
                network                         => $ipv4_subnet/prefix,             #Required, when
creating a network
                ipv4addr                        => $ipv4addr,                       #Use this parameter
instead of "network", when using a template with 'allow_any_netmask' set to false to create a network.
                authority                       => "true" | "false"| undef,         #Optional / Default
is undefined
                auto_create_reversezone         => "true" | "false"| undef,         #Optional / Default
is undefined
                bootfile                        => $filename | undef,               #Optional / Default
is undefined
                bootserver                      => $ipv4addr | $fqdn | undef,       #Optional / Default
is undefined
                comment                         => $string| undef,                  #Optional / Default
is undefined
                ddns_domainname                 => $string| undef,                  #Optional / Default
is undefined
                ddns_generate_hostname          => "true" | "false" | undef,        #Optional / Default
is undefined
                ddns_server_always_updates      => "true" | "false",                #Optional / Default
is "true"
                ddns_ttl                        => $num,                            #Optional / Default
is "0"
                ddns_use_option81               => "true" | "false" | undef,        #Optional / Default
is undefined
                ddns_update_fixed_addresses     => "true" | "false" | undef,        #Optional / Default
is undefined
                deny_bootp                      => "true" | "false" | undef,        #Optional / Default
is undefined
                disable                         => "true" | "false",                #Optional / Default
is "false"
                email_list                      => [ "admin\@test.com",...] | undef,  #Optional / Default
```

```
  is undefined
                  enable_ddns                     => "true" | "false" | undef,          #Optional / Default
  is undefined
                  enable_dhcp_thresholds          => "true" | "false",                  #Optional / Default
  is "false"
                  enable_email_warnings           => "true" | "false",                  #Optional / Default
  is "false"
                  enable_snmp_warnings            => "true" | "false",                  #Optional / Default
  is "false"
                  extattrs                        => { $string => $extattr, ... },      #Optional / Default
  is undefined
                  extensible_attributes           => { $string => $string | $num, $string => [ $string |
  $num, ... ], ... } #Required if 'rir_organization' attribute is defined
  #Otherwise is optional
                  high_water_mark                 => $number,                           #Optional / Default
  is "95"
                  high_water_mark_reset           => $number,                           #Optional / Default
  is "95"
                  ignore_dhcp_option_list_request => "true" | "false" | undef,          #Optional / Default
  is undefined
                  ignore_id                       => 'NONE' | 'CLIENT' | 'MACADDR',     #Optional / Default
  is 'NONE'
                  ignore_mac_addresses            => [ "mac_addr", ... ] | undef        #Optional / Default
  is undefined
                  lease_scavenge_time             => $number,                           #Optional / Default
  is "-1"
                  low_water_mark                  => $number,                           #Optional / Default
  is "0"
                  low_water_mark_reset            => $number,                           #Optional / Default
  is "0"
                  members                         => [ $Member1,$Member2,...],          #Optional / Default
  is empty
                  mgm_private                     => "true" | "false",                  #Optional / Default
  is "false"
                  network_container               => $ipv4_subnet/prefix,               #Optional / Default
  is "/"
                  network_view                    => $NetworkView,                      #Optional / Default
  is "default" network view
                  nextserver                      => $ipv4addr | $fqdn | undef,         #Optional / Default
  is undefined
                  options                         => [$Option1,$Option2,...],           #Optional / Default
  is empty list
                  override_ddns_domainname        => "true" | "false",                  #Optional / Default
  value is "false"
                  override_ddns_ttl               => "true" | "false",                  #Optional / Default
  value is "false"
                  override_enable_ifmap_publishing     => "true" | "false",             #Optional / Default
  value is "false"
                  override_lease_scavenge_time    => "true" | "false",                  #Optional / Default
  value is "false"
                  override_mgm_private            => "true" | "false",                  #Optional / Default
  value is "false"
                  override_update_dns_on_lease_renewal => "true" | "false",             #Optional / Default
  value is "false"
                  override_ignore_id              => "true" | "false",                  #Optional / Default
  value is "false"
                  pxe_lease_time                  => $num | undef,                      #Optional / Default
  is undefined
                  recycle_leases                  => "true" | "false" | undef,          #Optional / Default
  is undefined
                  rir_organization                => $rir_org,                          #Optional / An
  Infoblox::Grid::RIR::Organization object
                  rir_registration_action         => 'NONE' | 'CREATE' | 'MODIFY' | 'DELETE',  # Optional /
  write-only
                  rir_registration_status         => 'NOT_REGISTERED' | 'REGISTERED',   #Optional / Default
  is 'NOT_REGISTERED'
                  send_rir_request                => 'true' | 'false',                  #Optional / write-
  only
                  delete_reason                   => $string,                           #Optional / write-
  only
                                                                                        #Required only if
  'rir_registration_action' is specified as 'DELETE'
                  template                        => $string | undef,                   #Optional / Default
  is undefined
                  update_dns_on_lease_renewal     => "true" | "false",                  #Optional / Default
  value is "false"
                  zone_associations               => [$zone1, $zone2, ... ] | undef,    #Optional / Default
  is an empty list
                  discovery_blackout_setting      => $blackout,                         #Optional
                  port_control_blackout_setting   => $blackout,                         #Optional
                  override_blackout_setting       => "true" | "false",                  #Optional / Default
  value is "false"
                  same_port_control_discovery_blackout => "true" | "false",             #Optional / Default
```

```
value is "false"
                ipam_trap_settings              => $ipam_trap,                          #Optional / An
Infoblox::Grid::SNMP::IPAMTrap object
                override_ipam_trap_settings     => "true" | "false",                    #Optional / Default
value is "false"
                ipam_threshold_settings         => $ipam_threshold,                     #Optional / An
Infoblox::Grid::SNMP::IPAMThreshold object
                override_ipam_threshold_settings => "true" | "false",                   #Optional / Default
value is "false"
                ipam_email_addresses            => [$email1, $email2, ...],             #Optional / Default
is an empty list
                override_ipam_email_addresses   => "true" | "false",                    #Optional / Default
value is "false"
                logic_filters                => [ $filter_name | $filter_obj, ...],     #Optional / Default
is undefined
                override_logic_filters      => "true' | 'false",                        #Optional / Default
value is "false"
                subscribe_settings          => $subscribe_settings,                     #Optional / Default
is undefined
                override_subscribe_settings => 'true' | 'false',                        #Optional / Default
is 'false'
                discovery_engine_type       => 'NETMRI' | 'NETWORK_INSIGHT' | 'VDISCOVERY' | 'UNKNOWN' |
'NONE', #Optional / Default is 'NONE'
                discovered_bridge_domain    => $string,                                 #Optional / Default
is undefined
                discovered_tenant           => $string,                                 #Optional / Default
is undefined
              );
```

You cannot set both extattrs and extensible_attributes attributes at the same time.

SESSION METHODS

This section describes all the methods in an Infoblox::Session module that can apply to a DHCP Network object.

Infoblox::Session->add( )

Use this method to add a network object to the Infoblox appliance. See Infoblox::Session->add() for parameters and return values. Notice the syntax difference between simple adding a network and adding a network using a template.

Example

```
#Adding a network.
#Construct an object
my $network = Infoblox::DHCP::Network->new(
        network => "20.0.0.0/8",
);

# Submit for adding a network
my $response = $session->add( $network );

#Adding a network using template
#Construct an object
my $network = Infoblox::DHCP::Network->new(
        ipv4addr => '20.0.0.0',
        template => "mytemplate",
);

# Submit for adding a network
my $response = $session->add( $network );
```

Infoblox::Session->get( )

Use this method to retrieve all the matching network objects from the Infoblox appliance. See Infoblox::Session->get() for parameters and return values.

'discover_now_status' will not be returned by default, use 'return_methods' to request it if needed.

Key References

```
  Apply the following attributes to get a specific DHCP Network object:

    network                 - Optional. The IP address and netmask of a network in CIDR format.
    network_container       - Optional. The network container, in CIDR format, to which this network
belongs.
    network_view            - Optional. The network view in which the network is located. By default,
all network views are searched (except when contain_address is specified -- see below).
    extattrs                - Optional. A hash reference containing extensible attributes.
    extensible_attributes   - Optional. A hash reference containing extensible attributes.
    comment                 - Optional. A comment in string format.
    contains_address        - Optional. An IPv4 address in string format. If specified, all other
```

```
search attributes are ignored, except for network_view. When a value for this attribute is provided,
network_view defaults to the system-defined default network view, unless specified. The returned
network is the smallest network enclosing this IPv4 address.
  member                  - Optional. The member or Microsoft server serving DHCP for the network.
The valid value is an Infoblox::DHCP::Member or Infoblox::DHCP::MSServer object.
  discovered_bridge_domain - Optional. A bridge domain name in string format.
  discovered_tenant        - Optional. A tenant name in string format.
```

Examples

```
my  @result_array = $session->get(
     object         => "Infoblox::DHCP::Network" ,
     network        => "20.0.0.0/8",
     network_view => "default"
);

my  @result_array = $session->get(
     object                => "Infoblox::DHCP::Network" ,
     extensible_attributes => { 'Site' => 'Santa Clara' }
);

my  @result_array = $session->get(
     object  => "Infoblox::DHCP::Network" ,
     comment => "my-comment",
);

my  @result_array = $session->get(
     object  => "Infoblox::DHCP::Network" ,
     network_container => "10.0.0.0/16",
);

# Returns the smallest network enclosing 10.0.0.1 in the
# system-defined default network view.
my  @result_array = $session->get(
     object  => "Infoblox::DHCP::Network" ,
     contains_address => "10.0.0.1",
);

# Returns the smallest network enclosing 10.0.0.1 in the
# specified network view ($network_view_object).
my  @result_array = $session->get(
     object  => "Infoblox::DHCP::Network" ,
     contains_address => "10.0.0.1",
     network_view => $network_view_object,
);

# get a DHCP Network by member
my $member = Infoblox::DHCP::Member->new(
     name => "ib-10-0-0-1.infoblox.com",
     ipv4addr => "10.0.0.1",
);

my  @result_array = $session->get(
     object => "Infoblox::DHCP::Network",
     member => $member,
);

my @result_array = $session->get(
     object  => "Infoblox::DHCP::Network" ,
     discovered_bridge_domain => "domain1",
);

my @result_array = $session->get(
     object  => "Infoblox::DHCP::Network" ,
     discovered_tenant => "tenant1",
);
```

Infoblox::Session->modify( )

Use this method to modify a network object in the Infoblox appliance. See Infoblox::Session->modify() for parameters and return values.

Example

```
#Use method to modify enable_ddns method.
$object->enable_ddns("true");
# Submit modification
my $response = $session->modify( $object );
```

Infoblox::Session->remove( )

Use this method to remove an object from the Infoblox appliance. See Infoblox::Session->remove() for parameters and return values.

To remove a specific object, first use `get()` or `search()` to retrieve the specific object, and then submit this object for removal.

Key References

```
remove_options - Optional. A hash reference containing the following attributes:

+-------------------------------------------------------------------------------------------+
| reason             - Optional. The reason for removing the network in string format with  |
|                      a maximum of 256 bytes.                                              |
| send_rir_request   - Optional. Boolean flag that indicates whether the appliance should  |
|                      send an API or email request for Regional Internet Registry (RIR).  |
|                      Used when 'rir_organization' attribute has been defined.             |
+-------------------------------------------------------------------------------------------+
```

Example

```
 # Get the object under the same name
my  @result_array = $session->get(

       object        => "Infoblox::DHCP::Network" ,
       network       => "20.0.0.0/8",
       network_view  => "default"
         );

# find the desired object from the retrieved list.
my $desired_network = $result_array[0];

# Submit for removal
my $response = $session->remove(
    $desired_network,
    remove_options => {'reason' => 'Moved to B3', send_rir_request => 'true'} );
```

Infoblox::Session->search( )

Use this method to search for a DHCP Network object in the Infoblox appliance. See Infoblox::Session->search() for parameters and return values.

'discover_now_status' will not be returned by default, use 'return_methods' to request it if needed.

Key References

```
 Apply the following attributes to search for a DHCP Network object:

  network                  - Optional. The IP address and netmask of a network in CIDR format.
  network_container        - Optional. The network container, in CIDR format, to which this network
belongs.
  network_view             - Optional. The network view in which the network is located. By default,
all network views are searched.
  extattrs                 - Optional. A hash reference containing extensible attributes.
  extensible_attributes    - Optional. A hash reference containing extensible attributes.
  comment                  - Optional. A comment in string format (regular expression).
  contains_address         - Optional. An IPv4 address in string format. If specified, all other
search attributes are ignored, except for network view. When a value for this attribute is provided,
network view defaults to the system-defined default network view, unless specified. The returned
network is the smallest network enclosing this IPv4 address.
  member                   - Optional. The member or Microsoft server serving DHCP for the network.
The valid value is an Infoblox::DHCP::Member or Infoblox::DHCP::MSServer object.
  discovered_bridge_domain - Optional. A bridge domain name in string format (regular expression).
  discovered_tenant        - Optional. A tenant name in string format (regular expression).
```

For more information about searching extensible attributes, see Infoblox::Grid::ExtensibleAttributeDef/Searching Extensible Attributes.

Examples

```
# search for all DHCP Network objects
    my @retrieved_objs = $session->search(
      object       => "Infoblox::DHCP::Network",
      network      => '20\.0\.0\..*',
      network_view => "default"
    );

# search network by comment
    my @retrieved_objs = $session->search(
    object    => "Infoblox::DHCP::Network",
    comment   => "my-comment",
     );

# search network by network container
```

```
 my @result_array = $session->search(
      object  => "Infoblox::DHCP::Network" ,
      network_container => "10.0.0.0/16",
 );

# search for all DHCP Network objects defining "Santa Clara" for "Site" extensible attribute
    my @retrieved_objs = $session->search(
      object               => "Infoblox::DHCP::Network",
      extensible_attributes => { 'Site' => 'Santa Clara' }
    );

# Returns the smallest network enclosing 10.0.0.1 in the
# system-defined default network view.
    my @result_array = $session->search(
       object  => "Infoblox::DHCP::Network" ,
       contains_address => "10.0.0.1",
    );

# Returns the smallest network enclosing 10.0.0.1 in the
# specified network view ($network_view_object).
    my @result_array = $session->search(
       object  => "Infoblox::DHCP::Network" ,
       contains_address => "10.0.0.1",
       network_view => $network_view_object,
    );

 # search for all DHCP Network objects by member
 my $member = Infoblox::DHCP::Member->new(
      name => "ib-10-0-0-1.infoblox.com",
      ipv4addr => "10.0.0.1",
 );

 my  @result_array = $session->search(
      object => "Infoblox::DHCP::Network",
      member => $member,
 );

 # search network by discovered bridge domain
 my @result_array = $session->search(
      object  => "Infoblox::DHCP::Network" ,
      discovered_bridge_domain => "domain.*",
 );

 # search network by discovered tenant
 my @result_array = $session->search(
      object  => "Infoblox::DHCP::Network" ,
      discovered_tenant => "tenant.*",
 );
```

METHODS

This section describes all the methods that can be used to configure and retrieve the attribute values of a DHCP Network object

authority( )

Use this method to set or retrieve the authority of a DHCP network object. This setting overrides the upper-level settings.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

Specify "true" to set the authority or "false" to deactivate/unset it. The default value for this field is undefined, which indicates that this attribute inherit the upper-level setting.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
 # Get authority
 my $authority = $network->authority()
 # Modify authority
 $network->authority("true");
```

auto_create_reversezone( )

Use this method to set the auto_create_reversezone flag of a new DHCP network object. You can set this flag only when creating a new network object.

Include the specified parameter to set the attribute value.

Parameter

Specify "true" to set the auto_create_reversezone flag or "false" to deactivate/unset it. The default value for this field is "false".

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

Example

```
# Modify auto_create_reversezone
$network_container->auto_create_reversezone("true");
```

bootfile( )

Use this method to set or retrieve a bootfile attribute of a DHCP network object. You can configure the DHCP server to support clients that use boot file name option in their DHCPREQUEST messages. This setting overrides the upper-level settings. The default value is undefined, which indicates that this attribute inherits the upper-level setting.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

This option is overridden independently from Infoblox::DHCP::Network->bootserver() and Infoblox::DHCP::Network->nextserver().

Parameter

The name, in string format, of the file that the client must download. The default value is undefined.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
# Get bootfile
my $bootfile = $network->bootfile();
# Modify bootfile
$network->bootfile("bootfile1");
#Un-override bootfile
$network->bootfile(undef);
```

bootserver( )

Use this method to set or retrieve a bootserver attribute of a DHCP network object. You can specify the name and/or IP address of the boot server that host needs to boot. This setting overrides the upper-level settings. The default value is undefined, which indicates that this attribute inherits the upper-level setting.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

This option is overridden independently from Infoblox::DHCP::Network->bootfile() and Infoblox::DHCP::Network->nextserver().

Parameter

The boot server IP address or name in FQDN (Fully Qualified Domain Name) format. The FQDN consists of the host name followed by the domain name (example: abc.com). A boot server name can have a maximum of 256 bytes. The default value is undefined.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
# Get bootserver
my $bootserver = $network->bootserver();
# Modify bootserver
$network->bootserver("abc.domain.com");
#Un-override bootserver
$network->bootserver(undef);
```

comment( )

Use this method to set or retrieve a descriptive comment of a DHCP network object.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

Enter a descriptive comment for the network in string format with a maximum of 256 bytes.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
# Get comment
my $comment = $network->comment();
# Modify comment
$network->comment("add a network");
```

cloud_info( )

Use this method to set or retrieve cloud API related information for the Infoblox::DHCP::Network object.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

The valid value is a Infoblox::Grid::CloudAPI::Info object.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
# Get cloud_info
my $cloud_info = $network_view->cloud_info();
# Modify cloud_info
$network_view->cloud_info($cloud_info_object);
```

ddns_domainname( )

Use this method to set or retrieve the ddns_domainname value of a DHCP Network object.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Setting this method to a defined value implicitly sets the override_ddns_domainname method to "true". Setting the parameter to undefined causes the appliance to use the grid default and automatically resets the override_ddns_domainname attribute to "false".

Note that when ddns_domainname is set to a defined value and override_dddns_domainname is set to "false", the last operation takes precedence. Thus the sequence $object->ddns_domainname("testdomain"); $object->override_ddns_domainname("false"); will set override_ddns_domainname to "false", and the sequence $object->override_ddns_domainname("false"); $object->ddns_domainname("testdomain"); will result in override_ddns_domainname="true".

Parameter

The dynamic DNS domain name in string format.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get ddns_domainname
my $ddns_domainname = $network->ddns_domainname();
#Modify ddns_domainname
$network->ddns_domainname("test_domain.com");
```

ddns_generate_hostname( )

Use this method to set or retrieve the ddns_generate_hostname flag of a DHCP network object. Enable this method to allow only the DHCP server for generating hostname. This setting overrides the upper-level settings.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

Specify "true" to generate hostname or "false" to deactivate/unset it.The default value for this field is undefined, which indicates that this attribute inherit the member level setting.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
# Get ddns_generate_hostname
my $ddns_generate_hostname = $network->ddns_generate_hostname()
# Modify ddns_generate_hostname
$network->ddns_generate_hostname("true");
#Un-override
$network->ddns_generate_hostname(undef);
```

ddns_server_always_updates( )

Use this method to set or retrieve the ddns_server_always_updates flag of a DHCP network object. Enable this method to allow only the DHCP server to update DNS,regardless of the requests from the DHCP clients.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

Specify "true" to set the flag that enables the DHCP server to always updates DNS or "false" to deactivate/unset it.The default value for this field is "true".

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
# Get ddns_server_always_updates
my $ddns_server_always_updates = $network->ddns_server_always_updates()
# Modify ddns_server_always_updates
$network->ddns_server_always_updates("true");
```

ddns_ttl( )

Use this method to set or retrieve the DNS update Time to Live (TTL) value of a DHCP network object.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Setting this method to a defined value implicitly sets the override_ddns_ttl method to "true". Setting the parameter to undefined causes the appliance to use the grid default and automatically resets the override_ddns_ttl attribute to "false".

Note that when ddns_ttl is set to a defined value and override_dddns_ttl is set to "false", the last operation takes precedence. Thus the sequence $object->ddns_ttl(12); $object->override_ddns_ttl("false"); will set override_ddns_ttl to "false", and the sequence $object->override_ddns_ttl("false"); $object->ddns_ttl(12); will result in override_ddns_ttl="true".

Parameter

A 32-bit integer (range from 0 to 4294967295) that represents the duration, in seconds, that the update is cached. Zero indicates that the update is not cached. The default value is 0.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get DNS Update ttl
my $dns_ttl = $network->ddns_ttl();
#Modify DNS Update ttl
```

```
$network->ddns_ttl(1200);
```

ddns_update_fixed_addresses( )

Use this method to set or retrieve the ddns_update_fixed_addresses flag of a DHCP network object. By default, the DHCP server does not update DNS when it allocates a fixed address to a client. You can configure the DHCP server to update the A and PTR records of a client with a fixed address. When this feature is enabled and the DHCP server adds A and PTR records for a fixed address, the DHCP server never discards the records. This setting overrides the upper-level settings.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

Specify "true" to enable the appliance to update DNS when it assigns a fixed address to a client or "false" to disable it. The default value for this field is undefined, which indicates that this attribute inherit the member level setting.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
# Get ddns_update_fixed_addresses
my $ddns_update_fixed_addresses= $network->ddns_update_fixed_addresses()
# Modify ddns_update_fixed_addresses
$network->ddns_update_fixed_addresses("true");
#Un-override
$network->ddns_update_fixed_addresses(undef);
```

ddns_use_option81( )

Use this method to set or retrieve the ddns_use_option81 flag of a DHCP network object. Enable this method to define the settings for option 81 at the network level. This setting overrides the member level settings.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

Specify "true" to Configure Support for Option 81 or "false" to deactivate/unset it. The default value for this field is undefined, which indicates that this attribute inherit the member level setting.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
# Get ddns_use_option81
my $ddns_use_option81 = $network->ddns_use_option81()
# Modify ddns_use_option81
$network->ddns_use_option81("true");
# un-override ddns_use_option81
$network->ddns_use_option81(undef);
```

deny_bootp( )

Use this method to set or retrieve the deny_bootp flag of a DHCP Network object.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

Specify "true" to set the deny_bootp flag or "false" to deactivate/unset it. The default value is undefined, which indicates that this attribute inherit the upper-level setting.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get deny_bootp
my $deny_bootp   $network  deny bootp
```

```
                =          ->          ();
#Modify deny_bootp
$network->deny_bootp("true");
```

disable( )

Use this method to set or retrieve the disable flag of a DHCP network object.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

Specify "true" to set the disable flag or "false" to deactivate/unset it. The default value for this field is "false".

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
# Get disable
my $disable = $network->disable()
# Modify disable
$network->disable("true");
```

discovered_bridge_domain( )

Use this method to set or retrieve the discovered bridge domain name of a DHCP Network object.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

The desired bridge domain name in a string format.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get bridge domain
my $bridge_domain = $network->discovered_bridge_domain();
#Modify bridge domain
$network->discovered_bridge_domain("Domain");
```

discovered_tenant( )

Use this method to set or retrieve the discovered tenant name of a DHCP Network object.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

The desired tenant name in a string format.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get tenant
my $tenant = $network->discovered_tenant();
#Modify tenant
$network->discovered_tenant("Tenant");
```

discovery_basic_poll_setting( )

Use this method to set or retrieve the discovery_basic_poll_setting value of a DHCP Network object.

Include the specified parameter to set the attribute value.

Parameter

The valid value is object of the Infoblox::Grid::Discovery::BasicPollSettings manpage.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get discovery_basic_poll_setting
my $discovery_basic_poll_setting = $network->discovery_basic_poll_setting();
#Modify discovery_basic_poll_setting
$network->discovery_basic_poll_setting($poll_setting);
```

discovery_exclusion_range( )

Use this method to set or retrieve the discovery_exclusion_range value of a DHCP Network object.

Parameter

The valid value is an array of the Infoblox::DHCP::ExclusionRange manpage objects.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get discovery_exclusion_range
my $discovery_exclusion_range = $network->discovery_exclusion_range();
#Modify discovery_exclusion_range
$network->discovery_exclusion_range( [ $exclusion1, $exclusion2 ]);
```

discovery_member( )

Use this method to set or retrieve the discovery_member value of a DHCP Network object.

Parameter

The discovery member name in string format.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get discovery_member
my $discovery_member = $network->discovery_member();
#Modify discovery_member
$network->discovery_member("test.localdomain");
```

discover_now_status( )

Use this method to retrieve the current discovery status. This attribute is read-only.

'discover_now_status' will not be returned by default, use 'return_methods' to request it if needed.

Parameter

None.

Returns

The method returns the attribute value that can be 'COMPLETE', 'FAILED', 'NONE', 'PENDING' or 'RUNNING'.

Example

```
#Get discover_now_status
my $discover_now_status = $network->discover_now_status();
```

discovery_blackout_setting( )

Use this method to set or retrieve the discovery blackout setting.

Setting this method to a defined value implicitly sets the override_blackout_setting method to "true". If port_control_blackout_setting is undefined, setting the parameter to undefined causes the appliance to use the grid default and automatically resets the override_blackout_setting attribute to "false".

Note that when discovery_blackout_setting is set to a defined value and override_blackout_setting is set to "false", the last operation takes precedence. Thus the sequence $network->discovery_blackout_setting($blackout); $network->override_blackout_setting("false"); will set override_blackout_setting to "false", and the sequence $network->override_blackout_setting("false"); $network->discovery_blackout_setting($blackout); will result in override_blackout_setting="true".

Parameter

Valid value is an Infoblox::Grid::Discovery::Properties::BlackoutSetting object.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get discovery_blackout_setting
my $discovery_blackout_setting = $network->discovery_blackout_setting();
#Modify discovery_blackout_setting
$network->discovery_blackout_setting($blackout);
```

discovery_engine_type( )

Use this method to retrieve the DHCP Network discovery engine type. This attribute is read-only.

Parameter

None.

Returns

The method returns the attribute value that can be 'NETMRI', 'NETWORK_INSIGHT', 'VDISCOVERY', 'UNKNOWN' or 'NONE'.

Example

```
#Get discovery_engine_type
my $discovery_engine_type = $network->discovery_engine_type();
```

email_list( )

Use this method to set or retrieve the e-mail lists to which the appliance sends threshold e-mail messages.

If you enter e-mail addresses in the email_list, then the appliance sends DHCP threshold alarms to the e-mail addresses.

Parameter

The valid value is an array reference that contains string objects.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get email_list
my @email_list = $network->email_list();
#Modify email_list
$network->email_list( [ "admin\@test.com" ] );
```

enable_ddns( )

Use this method to set or retrieve the dynamic DNS updates flag of a DHCP network object. The DHCP server can send DDNS updates to DNS servers in the same grid and to external DNS servers. This setting overrides the member level settings.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

Specify "true" to set the dynamic DNS updates flag or "false" to deactivate/unset it. The default value for this field is undefined, which indicates that this attribute inherit the member level setting.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
# Get enable_ddns
my $enable_ddns = $network->enable_ddns()
# Modify enable_ddns
$network->enable_ddns("true");
# Un-override enable_ddns
$network->enable_ddns(undef);
```

enable_dhcp_thresholds( )

Use this method to set or retrieve DHCP thresholds at the network level. These thresholds are the high and low watermarks. If a DHCP range usage goes above or below these percentage values, the Infoblox appliance generates a syslog message and sends an SNMP notification.

Enable this flag to set DHCP thresholds values.

Parameter

Specify "true" to set the enable_dhcp_thresholds flag or "false" to deactivate/unset it. The default value is undefined.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get enable_dhcp_thresholds
my $enable_dhcp_thresholds = $network->enable_dhcp_thresholds();
#Modify enable_dhcp_thresholds
$network->enable_dhcp_thresholds("false");
```

enable_email_warnings( )

Use this method to send DHCP threshold warnings through e-mail.

Enable this flag if you want the appliance to send DHCP threshold warnings to the "email_list" provided when DHCP threshold is enabled.

Parameter

Specify "true" to set the enable_email_warnings flag or "false" to deactivate/unset it. The default value is undefined.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get enable_email_warnings
my $enable_email_warnings = $network->enable_email_warnings();
#Modify enable_email_warnings
$network->enable_email_warnings("false");
```

enable_discovery( )

Use this method to set or retrieve the enable_discovery value of a DHCP Network object. This setting overrides the member level settings.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Setting this method to a defined value implicitly sets the override_enable_discovery method to "true". Setting the parameter to undefined causes the appliance to use the default from the upper level and automatically resets the override_enable_discovery attribute to "false".

Enable this flag if you want to set discovery for this object.

Parameter

Specify "true" to set the enable_discovery flag or "false" to deactivate/unset it. The default value is undefined.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get enable_discovery
my $enable_discovery = $network->enable_discovery();
#Modify enable_discovery
$network->enable_discovery("false");
```

enable_ifmap_publishing( )

Use this method to set or retrieve ifmap publishing the option of a DHCP Network object. This setting overrides the member level settings.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

Specify "true" to set ifmap publishing option or "false" to deactivate/unset it. The default value 'false', which indicates that this attribute inherits the member level setting.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
# Get enable ifmap publishing
my $enable_ifmap_publishing = $network->enable_ifmap_publishing()
# Modify enable_ifmap_publishing
$network->enable_ifmap_publishing("true");
# Un-override enable_ifmap_publishing
$network->enable_ifmap_publishing(undef);
```

enable_immediate_discovery( )

Use this method to set or retrieve enable_immediate_discovery value of a DHCP Network object.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

Specify "true" to set the enable_immediate_discovery flag or "false" to deactivate/unset it. The default value is undefined.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get enable_immediate_discovery
my $enable_immediate_discovery = $network->enable_immediate_discovery();
#Modify enable_immediate_discovery
$network->enable_immediate_discovery("false");
```

enable_snmp_warnings( )

Use this method to send DHCP threshold warnings through SNMP.

If the flag is enabled, then the appliance sends DHCP threshold warnings through SNMP messages when DHCP threshold is enabled.

Parameter

Specify "true" to set the enable_snmp_warnings flag or "false" to deactivate/unset it. The default value is undefined.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get enable_snmp_warnings
my $enable_snmp_warnings = $network->enable_snmp_warnings();
#Modify enable_snmp_warnings
$network->enable_snmp_warnings("false");
```

endpoint_sources( )

Use this method to retrieve the list of endpoints that provide object data. This is a read-only attribute.

Omit the parameter to retrieve the attribute value.

Parameter

None

Returns

The valid return value is an array of Infoblox::CiscoISE::Endpoint objects.

Example

```
#Get endpoint_sources value
my $endpoint_sources = $object->endpoint_sources();
```

extattrs( )

Use this method to set or retrieve the extensible attributes associated with a DHCP network object.

Parameter

Valid value is a hash reference containing the names of extensible attributes and their associated values (Infoblox::Grid::Extattr objects).

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get extattrs
my $ref_extattrs = $network->extattrs();
#Modify extattrs
$network->extattrs({ 'Site' => $extattr1, 'Administrator' => $extattr2 });
```

extensible_attributes( )

Use this method to set or retrieve the extensible attributes associated with a DHCP network object

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

For valid values for extensible attributes, see Infoblox::Grid::ExtensibleAttributeDef/Extensible Attribute Values.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get extensible attributes
my $ref_extensible_attributes = $network->extensible_attributes();
#Modify extensible attributes
$network->extensible_attributes({ 'Site' => 'Santa Clara', 'Administrator' => [ 'Peter', 'Tom' ] });
```

high_water_mark( )

Use this method to set the percentage value for DHCP range usage above which the Infoblox appliance generates a syslog message and sends a warning.

When the percent of allocated addresses in a DHCP range exceeds the high watermark, the appliance generates a syslog message and sends a warning. Default value for this option is 95.

Parameter

The valid value is in string format.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get high_water_mark
my $high_water_mark = $network->high_water_mark();
#Modify high_water_mark
$network->high_water_mark("80");
```

high_water_mark_reset( )

Use this method to set or retrieve the high_water_mark_reset value. If the percentage of allocated addresses drops below this value, a corresponding SNMP trap is reset.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

A number that specifies the percentage of allocated addresses. The range is from 1 to 100. The high watermark reset value must be lower than the high watermark value. The default value is 95.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get high_water_mark_reset
my $high_water_mark_reset = $network->high_water_mark_reset();
#Modify high_water_mark_reset
$network->high_water_mark_reset(70);
```

ignore_id( )

Use this method to set or retrieve the ignore_id type of a DHCP network object. Indicates whether the appliance will ignore DHCP client IDs or MAC addresses. This setting overrides the upper-level settings.

Parameter

Valid values are 'NONE', 'CLIENT', or 'MACADDR'. The default is 'NONE'.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
# Get attribute value
$value = $network->ignore_id();
# Modify attribute value
$network->ignore_id('CLIENT');
#Un-override ignore_id
$network->ignore_id(undef);
```

ignore_mac_addresses( )

Use this method to set or retrieve the ignore_mac_addresses list of a DHCP network object. A list of MAC addresses the appliance will ignore. This setting overrides the upper-level settings.

If you enter MAC addresses in the ignore_mac_addresses, then the appliance will ignore them. If you enter empty array, then the appliance will ignore any MAC address.

Parameter

The valid value is an array reference that contains string objects.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
# Get attribute value
$value = $network->ignore_mac_addresses();
# Modify attribute value
$network->ignore_mac_addresses(["11::11::11::11::11::11"]);
```

ignore_dhcp_option_list_request( )

Use this method to set or retrieve the ignore_dhcp_option_list_request flag of a DHCP Network object. If this flag is set to false, the appliance returns all the defined DHCP options to the client. This setting overrides the upper-level settings.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

Specify "true" to set the ignore_dhcp_option_list_request flag or "false" to deactivate/unset it. The default value is undefined, which indicates that this attribute inherits the upper-level setting.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
# Get ignore_dhcp_option_list_request
my $ignore_dhcp_option_list_request = $network->ignore_dhcp_option_list_request();
# Modify ignore_dhcp_option_list_request
$network->ignore_dhcp_option_list_request("true");
#Do not override ignore_dhcp_option_list_request
$network->ignore_dhcp_option_list_request(undef);
```

ipam_email_addresses( )

Use this method to set or retrieve a list of IPAM Utilization email addresses.

Setting this method to a defined value implicitly sets the override_ipam_email_addresses method to "true". Setting the parameter to undefined causes the appliance to use the default from the upper level and automatically resets the override_ipam_email_addresses attribute to "false".

Note that when ipam_email_addresses is set to a defined value and override_ipam_email_addresses is set to "false", the last operation takes precedence. Thus the sequence $object->ipam_email_addresses(['admin@infoblox.com']); $object->override_ipam_email_addresses("false"); will set override_ipam_email_addresses to "false", and the sequence $object->override_ipam_email_addresses("false"); $object->ipam_email_addresses(['admin@infoblox.com']); will result in override_ipam_email_addresses="true".

Parameter

Valid value is an array reference that contains email addresses in string format.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get ipam_email_addresses
my $ipam_email_addresses = $network->ipam_email_addresses();
#Modify ipam_email_addresses
$network->ipam_email_addresses($ipam_email_addresses);
```

ipam_trap_settings( )

Use this method to set or retrieve IPAM Utilization SNMP Trap settings.

Setting this method to a defined value implicitly sets the override_ipam_trap_settings method to "true". Setting the parameter to undefined causes the appliance to use the default from the upper level and automatically resets the override_ipam_trap_settings attribute to "false".

Note that when ipam_trap_settings is set to a defined value and override_ipam_trap_settings is set to "false", the last operation takes precedence. Thus the sequence $object->ipam_trap_settings($ipam_trap); $object->override_ipam_trap_settings("false"); will set override_ipam_trap_settings to "false", and the sequence $object->override_ipam_trap_settings("false"); $object->ipam_trap_settings($ipam_trap); will result in override_ipam_trap_settings="true".

Parameter

Valid value is an Infoblox::Grid::SNMP::IPAMTrap object.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get ipam_trap_settings
my $ipam_trap_settings = $network->ipam_trap_settings();
#Modify ipam_trap_settings
$network->ipam_trap_settings($ipam_trap_settings);
```

ipam_threshold_settings( )

Use this method to set or retrieve IPAM Utilization SNMP Threshold settings.

Setting this method to a defined value implicitly sets the override_ipam_threshold_settings method to "true". Setting the parameter to undefined causes the appliance to use the default from the upper level and automatically resets the override_ipam_threshold_settings attribute to "false".

Note that when ipam_threshold_settings is set to a defined value and override_ipam_threshold_settings is set to "false", the last operation takes precedence. Thus the sequence $object->ipam_threshold_settings($ipam_threshold_settings); $object->override_ipam_trap_settings("false"); will set override_ipam_threshold_settings to "false", and the sequence $object->override_ipam_threshold_settings("false"); $object->ipam_threshold_settings($ipam_threshold_settings); will result in override_ipam_threshold_settings="true".

Parameter

Valid value is an Infoblox::Grid::SNMP::IPAMThreashold object.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get ipam_threshold_settings
my $ipam_threshold_settings = $network->ipam_threshold_settings();
#Modify ipam_threshold_settings
$network->ipam_threshold_settings($ipam_threshold_settings);
```

lease_scavenge_time( )

Use this method to set or retrieve the lease_scavenge_time value of a DHCP Network object.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Setting this method to a defined value implicitly sets the override_lease_scavenge_time method to "true". Setting the parameter to undefined causes the appliance to use the default from the upper level and automatically resets the override_lease_scavenge_time attribute to "false".

Note that when lease_scavenge_time is set to a defined value and override_lease_scavenge_time is set to "false", the last operation takes precedence. Thus the sequence $object->lease_scavenge_time(604800); $object->override_lease_scavenge_time("false"); will set override_lease_scavenge_time to "false", and the sequence $object->override_lease_scavenge_time("false"); $object->lease_scavenge_time(604800); will result in override_lease_scavenge_time="true".

Parameter

An integer that specifies the period of time, in seconds, that free and backup leases remain in the database before they are automatically deleted. To disable lease scavenging, set the parameter to -1. The minimum positive value must be greater than 86400 seconds (1 day). The default value of this parameter is -1, which means that lease scavenging is disabled.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get lease_scavenge_time
my $lease_scavenge = $network->lease_scavenge_time();
#Modify lease_scavenge_time
$network->lease_scavenge_time(30 * 24 * 60 * 60);
```

logic_filters( )

Use this method to set or retrieve all the logic (ordered) filters associated with the DHCP Network.

Filters are used to screen address assignments in a DHCP Network if the addresses are to be assigned to specific hosts.

Setting this method to a defined value implicitly sets the override_logic_filters method to "true". Setting the parameter to undefined causes the appliance to use the grid default and automatically resets the override_logic_filters attribute to "false".

Note that when logic_filters is set to a defined value and override_dlogic_filters is set to "false", the last operation takes precedence. Thus the sequence $object->logic_filters($filters); $object->override_logic_filters("false"); will set override_logic_filters to "false", and the sequence $object->override_logic_filters("false"); $object->logic_filters($filters); will result in override_logic_filters="true".

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

The valid value is an array reference that contains either the Infoblox::DHCP::Filter::MAC, Infoblox::DHCP::Filter::NAC, Infoblox::DHCP::Filter::Option objects or the names of the corresponding filters. The filters in the list must satisfy the following conditions:

Filter name must be globally unique.
Filters without expressions must be at the end of the list.
The filters should not be disabled.

The default value of the parameter is undefined.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get logic_filters
my $ref_logic_filters = $network->logic_filters();
#Modify logic_filters
$network->logic_filters(["filter_1","filter_2",$exl1]); #$exl1 is an Infoblox::DHCP::Filter::MAC,
Infoblox::DHCP::Filter::NAC or Infoblox::DHCP::Filter::Option object.
```

low_water_mark( )

Use this method to set the percentage value for DHCP range usage below which the Infoblox appliance generates a syslog message and sends a warning.

When the percent of allocated addresses in a DHCP range goes below the low watermark, the appliance generates a syslog message and sends a warning. Default value for this option is 0.

Parameter

The valid value is in string format.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get low_water_mark
my $low_water_mark = $network->low_water_mark();
#Modify low_water_mark
$network->low_water_mark("10");
```

low_water_mark_reset( )

Use this method to set or retrieve the low_water_mark_reset value. If the percentage of allocated addresses exceeds this value, a

corresponding SNMP trap is reset.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

A number that specifies the percentage of allocated addresses. The range is from 1 to 100. The low watermark reset value must be higher than the low watermark value. The default low watermark reset number is 0.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get low_water_mark_reset
my $low_water_mark_reset = $network->low_water_mark_reset();
#Modify low_water_mark_reset
$network->low_water_mark_reset(10);
```

members( )

Use this method to set or retrieve the member or Microsoft (r) server that serves DHCP for a network object.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

The valid value is an array reference that contains Infoblox::DHCP::Member or Infoblox::DHCP::MSServer objects.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
my $member1 = Infoblox::DHCP::Member->new(
    name     => "infoblox.localdomain",
    ipv4addr => "192.168.1.2",
);

#Create the Network object with this member
my $network = Infoblox::DHCP::Network->new(
    network => "10.0.0.0/8",
    members => [$member1],
);
```

mgm_private( )

Use this method to set or retrieve the flag that controls whether the object should be synced to the MGM or not.

Setting this method to a defined value implicitly sets the override_blackout_setting method to "true". If discovery_blackout_setting is undefined, setting the parameter to undefined causes the appliance to use the grid default and automatically resets the override_blackout_setting attribute to "false".

Note that when mgm_private is set to a defined value and override_mgm_private is set to "false", the last operation takes precedence. Thus the sequence $network->mgm_private("true"); $network->override_mgm_private("false"); will set override_mgm_private to "false", and the sequence $network->override_mgm_private("false"); $network->mgm_private("true"); will result in override_mgm_private="true".

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

Specify 'true' to enable synchronization of the managed Grid with the MGM or 'false' to forbid it. The default value is 'false'.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Note that this member is loaded on on-demand basis, so to retrieve the value the object must have been retrieved from the session first.

Example

```
#Get mgm_private value
my $value = $object->mgm_private();

#Modify mgm_private value
$object->mgm_private('true');
```

mgm_private_overridable( )

Use this method to retrieve the flag that indicates whether the overriding of 'mgm_private' flag is allowed or not. This is a read-only attribute.

Parameter

None

Returns

The method returns the attribute value.

Note that this member is loaded on on-demand basis, so to retrieve the value the object must have been retrieved from the session first.

Example

```
#Get mgm_private_overridable value
my $value = $object->mgm_private_overridable();
```

ms_ad_user_data( )

Use this method to retrieve Microsoft Active Directory users related information. This is a read-only attribute.

Parameter

None

Returns

The valid return value is an Infoblox::Grid::MSServer::AdUser::Data object.

Example

```
#Get ms_ad_user_data
my $ms_ad_user_data = $network->ms_ad_user_data();
```

network( )

Use this method to retrieve the network attribute of a DHCP network object.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

An IPv4 address is a 32-bit number in dotted decimal notation. It consists of four 8-bit groups of decimal digits separated by decimal points (example: 192.168.1.2).

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get network
my $network = $network->network();
```

network_container( )

Use this method to set or retrieve the network container of a DHCP network object.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

Specify the IP address and netmask of a network in CIDR format.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get network container
my $network_container = $network->network_container();
#Modify network container
$network->network_container("50.0.0.0/8");
```

network_view( )

Use this method to set or retrieve the network view of the DHCP network.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

The default value is the "default" network view, which means the network is in the default network view.

Parameter

Valid value is a defined Infoblox::DHCP::View object. The default network view value is the system-defined default network view object.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get network view
my $nview = $network->network_view();
#Modify network view, with an Infoblox::DHCP::View object
$network->network_view($nview);
```

nextserver( )

Use this method to set or retrieve the nextserver attribute of a DHCP network object. You can specify the name and/or IP address of the next server that the host needs to boot. This setting overrides the upper-level settings. The default value is undefined, which indicates that this attribute inherits the upper-level setting.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

This option is overridden independently from Infoblox::DHCP::Network->bootfile() and Infoblox::DHCP::Network->bootserver().

Parameter

The next server IP address or name in FQDN (Fully Qualified Domain Name) format. The FQDN consists of the host name followed by the domain name (example: abc.com). A next server name can have a maximum of 256 bytes. The default value is undefined.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
# Get nextserver
my $nextserver = $network->nextserver();
# Modify nextserver
$network->nextserver("blue.domain.com");
#Un-override nextserver
$network->nextserver(undef);
```

next_available_ip( )

Use this method to retrieve the next available IP `address(es)` in the network that was retrieved. Note that to use this method, the object MUST be retrieved through a get or search call, or created with an add call.

Parameter

The method accepts the following optional parameters:

`requested_num - The number of IP addresses requested. It must be a number between 1 and 20.`

```
  excluded - A reference to an array of addresses to be excluded from the results.
```

Returns

This method returns the next available IP address(es) in the network. If requested_num is not specified, a string with the next available address will be returned; otherwise, the method will return a reference to an array of addresses.

Within a DHCP Network object, the next available IP address(es) are the first addresses in this network that match all of the following rules:

```
   * Are not part of the passed 'excluded' parameter (if specified).
   * There is no A, PTR, fixed address or host address record matching this IP address
   * There is no bulk host record with this IP address
   * There is no unmanaged record matching this IP address
   * The IP address is not the network or broadcast address of this network
   * The IP address is not in any DHCP range within this network (including any exclusion range)
```

Example

```
# Retrieve a particular network
my @retrieved_objs = $session->get(
                          object   => "Infoblox::DHCP::Network",
                          network  => "10.0.0.0/8"
);

$dhcp_network = $retrieved_objs[0];

# Print the next available IP address in the network
print $dhcp_network->next_available_ip() . "\n";

# This will retrieve the next 10 available addresses and print the first one.
my $addressesref = $dhcp_network->next_available_ip(requested_num => 10);
print @$addressesref[0] . "\n";

# This will retrieve the next 10 available addresses and exclude two.
my $addressesref = $dhcp_network->next_available_ip(requested_num => 10, excluded => ["10.0.0.1",
"10.0.0.2"]);
```

next_available_network( )

Use this method to retrieve the next available network(s) in the network. Note that to use this method, the object MUST be retrieved through a get or search call, or created with an add call.

Parameter

The method accepts the following parameters:

```
  cidr - (required) The CIDR of the requested networks. It must be a value between 1 and 32,
inclusive.
  requested_num - (optional) The number of networks requested. It must be a value between 1 and 20,
inclusive.
  excluded - (optional) A reference to an array of networks to be excluded from the results.
```

Returns

This method returns the next available network(s) in the network. If requested_num is not specified, a string containing the next available network will be returned; otherwise, the method will return a reference to an array of networks.

A network cannot be returned as the next available network if it cannot be inserted; such as when any of the following conditions exist:

- It's parent network has a DHCP configuration assigned and therefore cannot be converted to a network container.

- It overlaps with an existing network or network container, regardless of whether the existing network has a greater or smaller CIDR than the requested CIDR.

- Its network address or broadcast address is used by a fixed address or a host configured for DHCP.

Example

```
# Retrieve a particular network
my @retrieved_objs = $session->get(
                          object   => "Infoblox::DHCP::Network",
                          network  => "10.0.0.0/8"
);

$network = $retrieved_objs[0];

# Print the next available IPv6 network in the network container
print $network->next_available_network(cidr => 12) . "\n";
```

```
# This will retrieve the next 10 available networks and print the first one
my $addressesref = $network->next_available_network(requested_num => 10, cidr => 12);
print @$addressesref[0] . "\n";

# This will retrieve the next 10 available networks and exclude two.
my $addressesref = $network->next_available_network(cidr => 12, requested_num => 10, excluded =>
['10.0.0.0/12']);
```

options( )

Use this method to set or retrieve the options value of a member DHCP server. DHCP options provide network configuration settings and various services available on the network for DHCP.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

The valid value is an array reference that contains Infoblox::DHCP::Option objects.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get options
my $options = $network->options();
#Modify options
my $option1 = Infoblox::DHCP::Option->new(
    name => "lease-time",
    value => 50
);

$network->options([$option1]);
```

override_ddns_domainname( )

The override_ddns_domainname attribute controls whether the ddns_domainname value in the DHCP Network is used, instead of the grid default.

The override_ddns_domainname attribute can be specified explicitly. It is also set implicitly when ddns_domainname is set to a defined value.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

Set the parameter to "true" to override the grid-level setting for ddns_domainname. Set the parameter to "false" to inherit the grid-level setting for ddns_domainname.

The default value of this parameter is "false".

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Getting override_ddns_domainname
my $override_ddns_domainname=$network->override_ddns_domainname( );
#Modifying override_ddns_domainname
$network->override_ddns_domainname("true");
```

override_ddns_ttl( )

The override_ddns_ttl attribute controls whether the ddns_ttl value in the DHCP Network is used, instead of the grid default.

The override_ddns_ttl attribute can be specified explicitly. It is also set implicitly when ddns_ttl is set to a defined value.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

Set the parameter to "true" to override the grid-level setting for ddns_ttl. Set the parameter to "false" to inherit the grid-level

setting for ddns_ttl.

The default value of this parameter is "false".

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Getting override_ddns_ttl
my $override_ddns_ttl=$network->override_ddns_ttl( );
#Modifying override_ddns_ttl
$network->override_ddns_ttl("true");
```

override_enable_ifmap_publishing( )

The override_enable_ifmap_publishing controls whether the enable_ifmap_publishing in the DHCP Network is used, instead of the grid default.

The override_enable_ifmap_publishing can be specified explicitly. It is also set implicitly when enable_ifmap_publishing is set to a defined value.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

Set the parameter to "true" to override the grid-level setting for enable_ifmap_publishing. Set the parameter to "false" to inherit the grid-level setting for enable_ifmap_publishing.

The default value of this parameter is "false".

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Getting override_enable_ifmap_publishing
my $override_enable_ifmap_publishing=$network->override_enable_ifmap_publishing( );
#Modifying override_enable_ifmap_publishing
$network->override_enable_ifmap_publishing("true");
```

override_blackout_setting( )

The override_blackout_setting attribute controls whether the discovery_blackout_setting and port_control_blackout_setting values in the object are used, instead of the grid default.

The override_blackout_setting attribute can be specified explicitly. It is also set implicitly when discovery_blackout_setting or port_control_blackout_setting is set to a defined value.

Parameter

Specify "true" to set the override_blackout_setting flag or "false" to deactivate/unset it. The default value is "false".

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
# Get attribute value
my $value = $network->override_blackout_setting();
# Modify attribute value
$network->override_blackout_setting('true');
```

override_ignore_id( )

The override_ignore_id controls whether the ignore_id in the DHCP network is used, instead of the upper-level default.

The override_ignore_id can be specified explicitly. It is also set implicitly when ignore_id is set to a defined value.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

Set the parameter to "true" to override the upper-level setting for ignore_id and ignore_mac_addresses. Set the parameter to "false" to inherit the upper-level setting for ignore_id and ignore_mac_addresses.

The default value of this parameter is "false".

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get attrubute value
my $value=$network->override_ignore_id();
#Modifying attribute value
$network->override_ignore_id('true');
```

override_ipam_email_addresses( )

The override_ipam_email_addresses controls whether the ipam_email_addresses in the DHCP network is used, instead of the upper-level default.

The override_ipam_email_addresses can be specified explicitly. It is also set implicitly when ipam_email_addresses is set to a defined value.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

Set the parameter to "true" to override the upper-level setting for ipam_email_addresses. Set the parameter to "false" to inherit the upper-level setting for ipam_email_addresses.

The default value of this parameter is "false".

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get attrubute value
my $value=$network->override_ipam_email_addresses();
#Modifying attribute value
$network->override_ipam_email_addresses('true');
```

override_ipam_threshold_settings( )

The override_ipam_threshold_settings controls whether the ipam_threshold_settings in the DHCP network is used, instead of the upper-level default.

The override_ipam_threshold_settings can be specified explicitly. It is also set implicitly when ipam_threshold_settings is set to a defined value.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

Set the parameter to "true" to override the upper-level setting for ipam_threshold_settings. Set the parameter to "false" to inherit the upper-level setting for ipam_threshold_settings.

The default value of this parameter is "false".

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get attrubute value
my $value=$network->override_ipam_threshold_settings();
#Modifying attribute value
```

```
$network->override_ipam_threshold_settings('true');
```

override_ipam_trap_settings( )

The override_ipam_trap_settings controls whether the ipam_trap_settings in the DHCP network is used, instead of the upper-level default.

The override_ipam_trap_settings can be specified explicitly. It is also set implicitly when ipam_trap_settings is set to a defined value.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

Set the parameter to "true" to override the upper-level setting for ipam_trap_settings. Set the parameter to "false" to inherit the upper-level setting for ipam_trap_settings.

The default value of this parameter is "false".

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get attrubute value
my $value=$network->override_ipam_trap_settings();
#Modifying attribute value
$network->override_ipam_trap_settings('true');
```

override_lease_scavenge_time( )

The override_lease_scavenge_time attribute controls whether the lease_scavenge_time value in the DHCP network is used, instead of the upper-level default.

The override_lease_scavenge_time attribute can be specified explicitly. It is also set implicitly when lease_scavenge_time is set to a defined value.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

Set the parameter to "true" to override the upper-level setting for lease_scavenge_time. Set the parameter to "false" to inherit the setting for lease_scavenge_time from the upper level.

The default value of this parameter is "false".

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Getting override_lease_scavenge_time
my $override_lease_scavenge_time=$network->override_lease_scavenge_time( );
#Modifying override_lease_scavenge_time
$network->override_lease_scavenge_time("true");
```

override_logic_filters( )

The override_logic_filters attribute controls whether the logic_filters value of the DHCP Network is used, instead of the Grid default.

The override_logic_filters attribute can be specified explicitly. It is also set implicitly when logic_filters is set to a defined value.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

Set the parameter to "true" to override the Grid-level setting for logic_filters. Set the parameter to "false" to inherit the Grid-level setting for logic_filters.

The default value of this parameter is "false".

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Getting override_logic_filters
my $override_logic_filters=$network->override_logic_filters( );
#Modifying override_logic_filters
$network->override_logic_filters("true");
```

override_mgm_private( )

The override_mgm_private attribute controls whether the mgm_private value in the object is used, instead of the parent-level default.

The override_mgm_private attribute can be specified explicitly. It is also set implicitly when mgm_private is set to a defined value.

Note that setting override_mgm_private to 'false' sets the original 'mgm_private' value to 'false'.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

Set the parameter to 'true' to override the parent-level setting for mgm_private. Set the parameter to 'false' to inherit the parent-level setting for mgm_private.

The default value of this parameter is 'false'.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Note that this member is loaded on on-demand basis, so to retrieve the value the object must have been retrieved from the session first.

Example

```
#Get override_mgm_private value
my $value = $object->override_mgm_private( );

#Modify override_mgm_private value
$object->override_mgm_private('true');
```

override_subscribe_settings( )

The override_subscribe_settings attribute controls whether the subscribe_settings method values of the member are used, instead of the Grid default.

The override_subscribe_settings attribute can be specified explicitly. It is also set implicitly when subscribe_settings is set to a defined value.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

Set the parameter to "true" to override the Grid-level setting for subscribe_settings. Set the parameter to "false" to inherit the Grid-level setting for subscribe_settings.

The default value of this parameter is "false".

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get override_subscribe_settings value
my $override_subscribe_settings = $object->override_subscribe_settings( );
#Modify override_subscribe_settings value
$object->override_subscribe_settings("true");
```

override_update_dns_on_lease_renewal( )

The override_update_dns_on_lease_renewal attribute controls whether the update_dns_on_lease_renewal value in the DHCP Network is used, instead of the grid default.

The override_update_dns_on_lease_renewal attribute can be specified explicitly. It is also set implicitly when

update_dns_on_lease_renewal is set to a defined value.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

Set the parameter to "true" to override the grid-level setting for update_dns_on_lease_renewal. Set the parameter to "false" to inherit the grid-level setting for update_dns_on_lease_renewal.

The default value of this parameter is "false".

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

#Getting override_update_dns_on_lease_renewal my $override_update_dns_on_lease_renewal=$network->override_update_dns_on_lease_renewal( ); #Modifying override_update_dns_on_lease_renewal $network->override_update_dns_on_lease_renewal("true");

port_control_blackout_setting

Use this method to set or retrieve the port control blackout setting.

Setting this method to a defined value implicitly sets the override_blackout_setting method to "true". If discovery_blackout_setting is undefined, setting the parameter to undefined causes the appliance to use the grid default and automatically resets the override_blackout_setting attribute to "false".

Note that when port_control_blackout_setting is set to a defined value and override_blackout_setting is set to "false", the last operation takes precedence. Thus the sequence $network->port_control_blackout_setting($blackout); $network->override_blackout_setting("false"); will set override_blackout_setting to "false", and the sequence $network->override_blackout_setting("false"); $network->port_control_blackout_setting($blackout); will result in override_blackout_setting="true".

Parameter

Valid value is an Infoblox::Grid::Discovery::Properties::BlackoutSetting object.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get port_control_blackout_setting
my $port_control_blackout_setting = $network->port_control_blackout_setting();
#Modify port_control_blackout_setting
$network->port_control_blackout_setting($blackout);
```

pxe_lease_time( )

Use this method to set or retrieve the PXE lease time value of a DHCP Network object. Some hosts use PXE (Preboot Execution Environment) to boot remotely from a server. To better manage your IP resources, set a different lease time for PXE boot requests. You can configure the DHCP server to allocate an IP address with a shorter lease time to hosts that send PXE boot requests, so IP addresses are not leased longer than necessary. This setting overrides the upper-level settings. The default value is undefined, which indicates that this attribute inherits the upper-level setting.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

A 32-bit integer (range from 0 to 4294967295) that represents the duration, in seconds, that the update is cached. Zero indicates that the update is not cached. The default value is undefined.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get PXE Lease Time
my $pxe_lease_time = $network->pxe_lease_time();
#Modify PXE Lease Time
```

```
$network->pxe_lease_time(1100);
#Un-override PXE Lease Time
$network->pxe_lease_time(undef);
```

recycle_leases( )

Use this method to set or retrieve the recycle_leases flag at the network level.

If the flag is enabled, the leases are kept in the recycle bin until one week after expiration. When the flag is disabled, the leases are irrecoverably deleted.

Parameter

Specify "true" to enable the recycle_leases flag or "false" to disable it. The default value is undefined.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get recycle_leases
my $recycle_leases = $network->recycle_leases();
#Modify recycle_leases
$network->recycle_leases("false");
```

resize( )

Use this method to resize a network. Note that to use this method, the object MUST be retrieved through a get or search call, or it must have been created with an add call.

Parameter

prefix - Required. The bitmask prefix length after resizing.

auto_create_reversezone - Optional. Boolean flag indicating whether or not to automatically create the reverse zone.

reason - Optional. The reason for resizing the network in string format with a maximum of 256 bytes. Used when 'rir_organization' attribute has been defined.

send_rir_request - Optional. Boolean flag that indicates whether the appliance should send an API or email request for Regional Internet Registry (RIR). Used when 'rir_organization' attribute has been defined.

Returns

The method returns true when the operation succeeds, and returns false when the operation fails. When an operation fails, the error code and error message are reported in the Session object's `status_code()` and `status_detail()` attributes.

Example

```
#Get Network through the session
my @retrieved_objs = $session->get(
        object =>"Infoblox::DHCP::Network" ,
        network => "20.0.0.0/8"
    );
my $network = $retrieved_objs[0];

# Expand a /24 network to a /16 network, inserting
# a corresponding reverse zone in the process.
my $response = $network->resize(
  prefix        => "16"
  auto_create_reversezone => "true"
) or die("Resize Network failed: ",
     $session->status_code() . ":" . $session->status_detail());
```

restart_if_needed( )

Use this method to set the flag that indicates whether DHCP service should be restarted on a member that is serving DHCP for the changed object referenced in a cloud API request.

Parameter

Specify true to restart the DHCP service.

Returns

None

Example

```
# Modify
$network->restart_if_needed('true');
```

rir( )

Use this method to retrieve the name of the Regional Internet Registry (RIR) that allocated this address space. This is a read-only attribute.

Parameter

None

Returns

The method returns the attribute value that can be 'RIPE' or 'NONE'.

Example

```
#Get rir
my $rir = $network->rir();
```

rir_organization( )

Use this method to set or retrieve the Regional Internet Registry (RIR) organization object.

Parameter

An Infoblox::Grid::RIR::Organization object.

You must enable support for RIR/SWIP at the Grid level before you can set attribute values. See Infoblox::Grid->enable_rir_swip() method.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get rir_organization
my $rir_organization = $network->rir_organization();
#Modify rir_organization
$network->rir_organization($rir_org);
```

rir_registration_action( )

Use this method to set the registration action of Regional Internet Registry (RIR). This is a write-only attribute.

Parameter

Valid values are 'NONE', 'CREATE', 'MODIFY' or 'DELETE'.

Returns

The method returns true when the modification succeeds, and returns false when the operation fails.

Example

```
#Set rir_registration_action
$network->rir_registration_action('CREATE');
```

rir_registration_status( )

Use this method to set or retrieve the registration status of Regional Internet Registry (RIR).

Parameter

The registration status of Regional Internet Registry (RIR), which can be 'REGISTERED' or 'NOT_REGISTERED'.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get rir_registration_status
my $rir_registration_status = $network->rir_registration_status();
#Modify rir_registration_status
$network->rir_registration_status('NOT_REGISTERED');
```

last_rir_registration_update_sent( )

Use this method to retrieve the last registration update timestamp of Regional Internet Registry (RIR). This is a read-only attribute.

Parameter

None

Returns

The method returns the attribute value.

Example

```
#Get last_rir_registration_update_sent
my $last_rir_registration_update_sent = $network->last_rir_registration_update_sent();
```

last_rir_registration_update_status( )

Use this method to retrieve the last registration update status of Regional Internet Registry (RIR). This is a read-only attribute.

Parameter

None

Returns

The method returns the attribute value.

Example

```
#Get last_rir_registration_update_status
my $last_rir_registration_update_status = $network->last_rir_registration_update_status();
```

same_port_control_discovery_blackout( )

Use this method to set or retrieve the flag that enables using the same discovery blackout for port control blackout.

Parameter

Specify 'true' to use same discovery blackout for port control blackout or 'false' to disable it. The default value is 'false'.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
# Get attribute value
my $value = $network->same_port_control_discovery_blackout();
# Modify attribute value
$network->same_port_control_discovery_blackout('true');
```

send_rir_request( )

Use this method to set the flag that indicates whether the appliance should send an API or email request for Regional Internet Registry (RIR). This is a write-only attribute.

Parameter

Specify "true" to set the send_rir_request flag or "false" to unset it.

Returns

The method returns true when the modification succeeds, and returns false when the operation fails.

Example

```
#Set send_rir_request
$network->send_rir_request('true');
```

subscribe_settings( )

Use this method to set or retrieve the Cisco ISE subscribe settings.

Setting this method to a defined value implicitly sets the override_subscribe_settings method to "true". Setting the parameter to undefined causes the appliance to use the Cisco ISE endpoint default and automatically resets the override_subscribe_settings method to "false".

Note that when subscribe_settings is set to a defined value and override_subscribe_settings is set to "false", the last operation takes precedence. Thus the sequence $object->subscribe_settings($value); $object->override_subscribe_settings("false"); will set override_subscribe_settings to "false", and the sequence $object->override_subscribe_settings("false"); $object->subscribe_settings($value); will result in override_subscribe_settings="true".

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

The valid value is an Infoblox::CiscoISE::SubscribeSetting object.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get subscribe_settings value
my $subscribe_settings = $object->subscribe_settings();
#Modify subscribe_settings value
$object->subscribe_settings($settings);
```

delete_reason( )

Use this method to specify the reason for deleting the Regional Internet Registry (RIR) registration request. This is a write-only attribute. This field is required only if 'rir_registration_action' is specified as 'DELETE'.

Parameter

The reason for deleting the RIR registration request in string format with a maximum of 256 bytes.

Returns

The method returns true when the modification succeeds, and returns false when the operation fails.

Example

```
#Set delete_reason
$network->delete_reason('invalid');
```

template( )

Use this method to set or retrieve the network template name of a DHCP Network object.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

The valid name in string format.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get template
my $template = $network->template();
#Modify template
$network->template("Template_Name");
```

zone_associations( )

Use this method to set or retrieve the associated zone information. Use the is_default boolean field of the Infoblox::DNS::Zone object to determine or specify the default zone in zone associations of a DHCP network. If the value is 'true', the associated zone is the default zone. Otherwise, it is not the default zone. The default value is 'false'.

Parameter

The valid value is an array reference that contains Infoblox::DNS::Zone object(s). The default value is an empty list. If the network has a parent, it is possible to use the zone associations of the parent by setting the parameter to undef.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails. If you did not specify a parameter, the method returns the attribute value.

Examples

```
#Getting zone associations
my $zone_associations = $network->zone_associations();
#Modifying zone_associations, setting 'test1.com' as the default zone
my $zone1 = Infoblox::DNS::Zone->new(
   name      => "test1.com",
   is_default => "true",
);
my $zone2 = Infoblox::DNS::Zone->new(
   name      => "test2.com",
   is_default => "false",
);
$network->zone_associations([$zone1, $zone2]);
#Clear zone_assocations
$network->zone_associations([]);
##Remove the override for zone_associations
$network->zone_associations(undef)
```

unmanaged( )

Use this method to set or retrieve the network status, which indicates whether the network is unmanaged or not.

Parameter

Specify true to mark the network as unmanaged or false as managed.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
# Get network status
$unmanaged = $network->unmanaged();
# Modify network status
$network->unmanaged('true');
```

update_dns_on_lease_renewal( )

The update_dns_on_lease_renewal attribute controls whether the DHCP server updates DNS when a DHCP lease is renewed.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Setting this method to a defined value implicitly sets the override_update_dns_on_lease_renewal method to "true". Setting the parameter to undefined causes the appliance to use the grid default and automatically resets the override_update_dns_on_lease_renewal attribute to "false".

Note that when update_dns_on_lease_renewal is set to a defined value and override_update_dns_on_lease_renewal is set to "false", the last operation takes precedence. Thus the sequence $object->update_dns_on_lease_renewal("true"); $object->override_update_dns_on_lease_renewal("false"); will set override_update_dns_on_lease_renewal to "false", and the sequence $object->override_update_dns_on_lease_renewal("false"); $object->update_dns_on_lease_renewal("true"); will result in override_update_dns_on_lease_renewal="true".

Parameter

Specify "true" to set the update_dns_on_lease_renewal flag or "false" to deactivate/unset it.

The default value of this parameter is "false".

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

#Get update_dns_on_lease_renewal my $update_dns_on_lease_renewal=$network->update_dns_on_lease_renewal( ); #Modify update_dns_on_lease_renewal $network->update_dns_on_lease_renewal("true");

SAMPLE CODE

The following sample code demonstrates the different functions that can be applied to an object such as add, search, modify, and remove. This sample also includes error handling for the operations.

#Preparation prior to a DHCP Network insertion

```
#PROGRAM STARTS: Include all the modules that will be used
use strict;
use Infoblox;

#Create a session to the Infoblox appliance

my $session = Infoblox::Session->new(
master   => "192.168.1.2",
username => "admin",
password => "infoblox"
);
unless ($session) {
die("Construct session failed: ",
   Infoblox::status_code() . ":" . Infoblox::status_detail());
}
print "Session created successfully\n";
```

#Create a DHCP Network object

```
my $memberdhcp = Infoblox::DHCP::Member->new(
        name     => "infoblox.localdomain",
        ipv4addr => "192.168.1.2"
       );

my  $network = Infoblox::DHCP::Network->new(
        network => "10.0.0.0/8",
        comment => "add network",
        members => [$memberdhcp],
         );

my $response = $session->add($network)
   or die("Add Network failed: ",
         $session->status_code() . ":" . $session->status_detail());

print "Network added successfully\n";

#Search for a specific Network

#Search the network
my @retrieved_objs = $session->search(
object  => "Infoblox::DHCP::Network",
network => '10\.0\.0\..*'
);
my $object = $retrieved_objs[0];
unless ($object) {
die("Search Network failed: ",
    $session->status_code() . ":" . $session->status_detail());
}
print "Search Network found at least 1 matching entry\n";
```

#Get and modify a Network attribute

```
#Get Network through the session
my @retrieved_objs = $session->get(
       object =>"Infoblox::DHCP::Network" ,
       network => "10.0.0.0"
    );

my $object = $retrieved_objs[0];
```

```
unless ($object) {
die("Get Network object failed: ",
    $session->status_code() . ":" . $session->status_detail());
}
print "Get Network found at least 1 matching entry\n";
```

#Modify attributes of the specified Network attribute

```
$object->enable_ddns("true"); #Apply the changes

#Un-override bootfile
$object->bootfile(undef);

#Un-override bootserver
$object->bootserver(undef);

#Un-override nextserver
$object->nextserver(undef);

$session->modify($object)
or die("Modify Network attribute failed: ",
       $session->status_code() . ":" . $session->status_detail());

print "Network object modified successfully \n";
```

#Resize a network

```
#Get Network through the session
my @retrieved_objs = $session->get(
       object =>"Infoblox::DHCP::Network" ,
       network => "10.0.0.0/8"
    );
my $object = $retrieved_objs[0];

#Resize
my $response = $object->resize(
  prefix        => "16"
) or die("Resize Network failed: ",
     $session->status_code() . ":" . $session->status_detail());
```

#Remove a Network Object

```
  #Get Network object through the session
  my @retrieved_objs = $session->get(
       object  => "Infoblox::DHCP::Network" ,
       network => "10.0.0.0"
    );
```

my $object = $retrieved_objs[0];

unless ($object) { die("Get Network object failed: ", $session->status_code() . ":" . $session->status_detail()); }

```
print "Get Network object found at least 1 matching entry\n";
```

#Submit the object for removal

```
$session->remove($object)
or die("Remove Network failed: ",
    $session->status_code() . ":" . $session->status_detail());

print "Network removed successfully \n";

####PROGRAM ENDS####
```

AUTHOR

Infoblox Inc. http://www.infoblox.com/

SEE ALSO

Infoblox::Session, Infoblox::DHCP::View, Infoblox::DHCP::Member, Infoblox::DHCP::Option, Infoblox::Session->get(), Infoblox::Session->search(), Infoblox::Session->add(), Infoblox::Session->remove(), Infoblox::Session->modify(), Infoblox::Grid::MSServer::AdUser::Data

---

COPYRIGHT

Copyright (c) 2017 Infoblox Inc.

Infoblox::DNS::Host - DNS Host record object.

- NAME
- DESCRIPTION
- CONSTRUCTOR
- SESSION METHODS
  - Infoblox::Session->add( )
  - Infoblox::Session->get( )
  - Infoblox::Session->modify( )
  - Infoblox::Session->remove( )
  - Infoblox::Session->search( )
- METHODS
  - aliases( )
  - allow_telnet( )
  - cloud_info( )
  - comment( )
  - configure_for_dns( )
  - cli_credentials( )
  - ddns_protected( )
  - device_description( )
  - device_location( )
  - device_type( )
  - device_vendor( )
  - disable( )
  - dns_aliases( )
  - dns_name( )
  - extattrs( )
  - extensible_attributes( )
  - ipv4addrs( )
  - ipv6addrs( )
  - last_queried( )
  - ms_ad_user_data( )
  - name( )
  - network_view( )
  - override_cli_credentials( )
  - rrset_order( )
  - ttl( )
  - views( )
  - zone( )
- SAMPLE CODE
- AUTHOR
- SEE ALSO
- COPYRIGHT

NAME

Infoblox::DNS::Host - DNS Host record object.

DESCRIPTION

A host record defines attributes for a node, such as the name-to-address and address-to-name mapping. This alleviates having to specify an A record and a PTR record separately for the same node. A host can also define aliases and DHCP fixed address nodes. The zone must be created first before adding a host record for the zone.

Note that you must specify only one view for the attribute "views".

CONSTRUCTOR

```
 my $host = Infoblox::DNS::Host->new(
     ipv4addrs               => [$ipv4addr, $ipv4addr, $FixedAddr, ...], #Required
     name                    => $fqdn,                                   #Required
     aliases                 => [$fqdn, $fqdn, ...],                     #Optional / Default is empty
     comment                 => $string,                                 #Optional / Default is undefined
     configure_for_dns       => "true" | "false",                        #Optional / Default "true"
     disable                 => "true" | "false",                                #Optional / Default
"false"
     extattrs                => { $string => $extattr, ... },            #Optional / Default is undefined
     extensible_attributes   => { $string => $string | $num, $string => [ $string | $num, ... ], ... }
#Optional / Default is undefined
     ipv6addrs               => [$ipv6addr, $ipv6addr, ...],             #Optional / Default is empty
```

```
     network_view             => $NetworkView,                          #Optional / Default is "default"
network view
     rrset_order              => "cyclic" | "random" | "fixed",         #Optional / Default "cyclic"
     ttl                      => $num | undef,                              #Optional / Default is
undefined
     views                    => [$View]                                #Optional / Default is "default"
view
     cli_credentials          => [$cli1, ...],                          #Optional
     override_cli_credentials => "true" | "false",                      #Optional / Default is "false"
     device_type              => $string,                               #Optional
     device_vendor            => $string,                               #Optional
     device_location          => $string,                               #Optional
     device_description       => $string,                               #Optional
     allow_telnet             => "true" | "false",                      #Optional / Default is "false"
     ddns_protected           => "true" | "false",                      #Optional / Default is "false"
 );
```

You cannot set both extattrs and extensible_attributes attributes at the same time.

SESSION METHODS

This section describes all the methods in the Infoblox::Session module that you can apply to a DNS host record object.

Infoblox::Session->add( )

Use this method to add a host record object to the Infoblox appliance. See Infoblox::Session->add() for parameters and return values.

Example

```
#Construct an object
my $host = Infoblox::DNS::Host->new(
    name         => "host1.test.com",
    aliases      => ["one.foo.com", "two.foo.com"],
    comment      => "add a host host1.test.com",
    disable      => "true",
    ipv4addrs    => ["4.4.4.4"],
    ipv6addrs    => ["2001::1:2:3:4"],
    newtork_view => $nview,
    ttl          => 3600,
    views        => [$view1],
);

# Submit for addition
my $response = $session->add( $host );
```

Infoblox::Session->get( )

Use this method to retrieve all the matching objects from the Infoblox appliance. See Infoblox::Session->get() for parameters and return values.

Key References

```
Apply the following attributes to get a specific DNS host object:

  name          - Optional. A host name in string format.
  network_view  - Optional. The network view in which the host record is located. By default, all
network views are searched.
  view          - Optional. The DNS view in which the host record is located. By default, all DNS
views are searched. But if you omit this attribute and specify a zone, the appliance searches the
'default' view only.
  zone          - Optional. A zone name in FQDN format.
  comment       - Optional. A comment in string format (regular expression)
  mac           - Optional. A MAC address present in any FixedAddr object in the ipv4addrs member.
  ipv4addr      - Optional. An IP address in string format (regular expression).
  ipv6addr      - Optional. An IPv6 address in string format (regular expression).
  alias         - Optional. An alias of the host in string format (regular expression).
  extattrs      - Optional. A hash reference containing extensible attributes.
  extensible_attributes - Optional. A hash reference containing extensible attributes.
```

Examples

```
my @retrieved_objs = $session->get(
                     object       => "Infoblox::DNS::Host",
                     name         => "host1.test.com",
                     network_view => "default",
                     view         => "default" );

my @retrieved_objs = $session->get(
```

```
                        object                => "Infoblox::DNS::Host",
                        extensible_attributes => { 'Site' => 'Santa Clara' });

    # get all DNS host objects in zone "test.com" of default view
    my @retrieved_objs = $session->get(
                        object       => "Infoblox::DNS::Host",
                        zone         => "test.com",
                        network_view => "default",
                        view         => "default" );
```

Infoblox::Session->modify( )

Use this method to modify a host record object in the Infoblox appliance. See Infoblox::Session->modify() for parameters and return values.

Example

```
    # Use method to modify the comment.
    $host->comment("this is a modified comment");
    # Submit modification
    my $response = $session->modify( $host );
```

Infoblox::Session->remove( )

Use this method to remove a host record object from the Infoblox appliance. See Infoblox::Session->remove() for parameters and return values.

To remove a specific object, first use `get()` or `search()` to retrieve the specific host record object, and then submit this object for removal.

Example

```
    # Get the objects with the same name
    my @retrieved_objs = $session->get(
        object => "Infoblox::DNS::Host",
        name   => "host1.test.com",
        view   => "default" );
    # Find the desired object from the retrieved list.
    my $desired_host = $retrieved_objs[0];
    # Submit for removal
    my $response = $session->remove( $desired_host );
```

Infoblox::Session->search( )

Use this method to search for DNS host record objects in the Infoblox appliance. See Infoblox::Session->search() for parameters and return values.

Key References

```
    Apply the following attributes to search for a DNS host record object:

      name          - Optional. A host name in string format (regular expression).
      network_view  - Optional. The network view in which the host record is located. By default, all
    network views are searched.
      view          - Optional. The DNS view in which the host record is located. By default, all DNS
    views are searched. But if you omit this attribute and specify a zone, the appliance searches the
    'default' view only.
      zone          - Optional. A zone name in FQDN format.
      comment       - Optional. A comment in string format (regular expression).
      mac           - Optional. A MAC address present in any FixedAddr object in the ipv4addrs member
    (regular expression).
      ipv4addr      - Optional. An IP address in string format (regular expression).
      ipv6addr      - Optional. An IPv6 address in string format (regular expression).
      alias         - Optional. An alias of the host in string format (regular expression).
      extattrs      - Optional. A hash reference containing extensible attributes.
      extensible_attributes - Optional. A hash reference containing extensible attributes.
```

For more information about searching extensible attributes, see Infoblox::Grid::ExtensibleAttributeDef/Searching Extensible Attributes.

Examples

```
    # search for all DNS host objects that match ".test.com" in the default DNS view
    my @retrieved_objs = $session->search(
        object  => "Infoblox::DNS::Host",
        name    => '.*\.test\.com',
        view    => "default",
        comment => ".*modified comment" );

    # search for DNS host objects defining "Santa Clara" for "Site" extensible attribute
```

```
my @retrieved_objs = $session->search(
    object                => "Infoblox::DNS::Host",
    extensible_attributes => { 'Site' => 'Santa Clara' });

# search for all DNS host objects in zone "test.com" of default view
my @retrieved_objs = $session->search(
    object       => "Infoblox::DNS::Host",
    zone         => "test.com",
    network_view => "default",
    view         => "default" );
```

METHODS

This section describes all the methods that you can use to configure and retrieve the attribute values of a host record.

aliases( )

Use this method to set or retrieve the aliases of the host.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

The attribute value can be in unicode format.

Parameter

Array reference with the aliases for the host. The aliases of the host should be in Fully Qualified Domain Name (FQDN) format. The default value is undefined.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Getting aliases
my $ref_aliases = $host->aliases();
#Modifying aliases
$host->aliases(["one1.foo.com", "two2.foo.com"]);
```

allow_telnet( )

Use this method to set or retrieve the flag that enables copying SSH credential to TELNET.

Parameter

Specify 'true' to copy SSH credential to TELNET or 'false' to disable it. The default value is 'false'.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
# Get attribute value
my $value = $host->allow_telnet();
# Modify attribute value
$host->allow_telnet('true');
```

cloud_info( )

Use this method to retrieve cloud API related information for the Infoblox::DNS::Host object.

Parameter

None

Returns

The method returns the attribute value.

Example

```
# Get cloud_info
```

```
my $cloud_info = $object->cloud_info();
```

comment( )

Use this method to set or retrieve a descriptive comment.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

Desired comment in string format with a maximum of 256 bytes. The default value is undefined.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get comment
my $comment = $host->comment();
#Modify comment
$host->comment("Modifying the DNS host comment");
```

configure_for_dns( )

Use this method to set or retrieve the configure_for_dns flag of a DNS host.

When configure_for_dns is false the host will not have parent zone information.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

The default value for this field is true.

Parameter

Specify "true" to set the configure_for_dns flag or "false" to deactivate/unset it. The default value is "true".

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get configure_for_dns
my $configure_for_dns = $host->configure_for_dns();
#Modify configure_for_dns
$host->configure_for_dns("false");
```

cli_credentials( )

Use this method to set or retrieve the discovery CLI credentials.

Setting this method to a defined value implicitly sets the override_cli_credentials method to "true". Setting the parameter to undefined causes the appliance to use the grid default and automatically resets the override_cli_credentials attribute to "false".

Note that when cli_credentials is set to a defined value and override_cli_credentials is set to "false", the last operation takes precedence. Thus the sequence $host->cli_credentials([$cli]); $host->override_cli_credentials("false"); will set override_cli_credentials to "false", and the sequence $host->override_cli_credentials("false"); $host->cli_credentials([$cli]); will result in override_cli_credentials="true".

Parameter

Valid value is an array reference that contains Infoblox::Grid::Discovery::CLICredential objects.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get cli_credentials
my $cli_credentials = $host->cli_credentials();
#Modify cli_credentials
$host->cli_credentials([$cli1, $cli2]);
```

ddns_protected( )

Use this method to set or retrieve the flag that indicates whether DDNS updates for this record are allowed or not.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

Specify 'true' to protect record from DDNS updates and 'false' to allow DDNS updates for the specified record.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get ddns_protected value
my $ddns_protected = $object->ddns_protected();

#Modify ddns_protected value
$object->ddns_protected('true');
```

device_description( )

Use this method to set or retrieve the description of the discovery device.

Parameter

The device description in string format.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get device_description
my $device_description = $host->device_description();
#Modify device_description
$host->device_description('example');
```

device_location( )

Use this method to set or retrieve the location of the discovery device.

Parameter

The device location in string format.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get device_location
my $device_location = $host->device_location();
#Modify device_location
$host->device_location('example');
```

device_type( )

Use this method to set or retrieve the type of the discovery device.

Parameter

The device type in string format.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get device_type
my $device_type = $host->device_type();
#Modify device_type
$host->device_type('example');
```

device_vendor( )

Use this method to set or retrieve the vendor name of the discovery device.

Parameter

The device vendor name in string format.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get device_vendor
my $device_vendor = $host->device_vendor();
#Modify device_vendor
$host->device_vendor('example');
```

disable( )

Use this method to set or retrieve the disable flag of a DNS host.

The host can be disabled instead of removed. You can avoid removing and re-adding a host when a network appliance is repaired or relocated. When the changes to the physical appliance are complete, the host can be simply re-enabled.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

The default value for this field is false.

Parameter

Specify "true" to set the disable flag or "false" to deactivate/unset it. The default value is "false".

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get disable
my $disable = $host->disable();
#Modify disable
$host->disable("false");
```

dns_aliases( )

Use this method to retrieve the aliases, in punycode format, of the host. This is a read-only attribute.

Parameter

None

Returns

The method returns the attribute value.

Example

```
# Get attribute value
my $value = $host->dns_aliases();
```

dns_name( )

Use this method to retrieve the host name in punycode format. This is a read-only attribute.

Parameter

None

Returns

The method returns the attribute value.

Example

```
# Get attribute value
my $value = $host->dns_name();
```

extattrs( )

Use this method to set or retrieve the extensible attributes associated with a DNS Host object.

Parameter

Valid value is a hash reference containing the names of extensible attributes and their associated values (Infoblox::Grid::Extattr objects).

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get extattrs
my $ref_extattrs = $host->extattrs();
#Modify extattrs
$host->extattrs({ 'Site' => $extattr1, 'Administrator' => $extattr2 });
```

extensible_attributes( )

Use this method to set or retrieve the extensible attributes associated with a DNS Host object.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

For valid values for extensible attributes, see Infoblox::Grid::ExtensibleAttributeDef/Extensible Attribute Values.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get extensible attributes
my $ref_extensible_attributes = $host->extensible_attributes();
#Modify extensible attributes
$host->extensible_attributes({ 'Site' => 'Santa Clara', 'Administrator' => [ 'Peter', 'Tom' ] });
```

ipv4addrs( )

Use this method to set or retrieve the IPv4 addresses of the host.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

Valid value is an array reference that contains defined Infoblox::DHCP::FixedAddr objects or IPv4 addresses.

An IPv4 address is a 32-bit number in dotted decimal notation. It consists of four 8-bit groups of decimal digits separated by decimal points (example: 192.168.1.2).

If the Infoblox library is loaded with the :hostaddress option, the valid value is an array reference that contains Infoblox::DHCP::HostAddr objects.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Gett ipv4addrs
my $ref_ipv4addr = $host->ipv4addrs();
#Modify ipv4addrs
$host->ipv4addrs([$fixedaddr, "2.2.2.2"]); #$fixedaddr is an Infoblox::DHCP::FixedAddr object
```

ipv6addrs( )

Use this method to set or retrieve the IPv6 addresses of the host.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

Valid value is an array reference that contains IPv6 addresses in string format.

An IPv6 address is a string consisting of eight groups of four hexadecimal digits, where each group is separated by a colon (:) (example: 2001:0db8:85a3:0000:0000:8a2e:0370:7334). Various shortcuts exist to shorten the string representation of an IPv6 address, such as omitting the leading zeros of each group and replacing one or any number of consecutive groups of 0 value with two colons(::). i.e. The previous example can be written as 2001:db8:85a3::8a2e:370:7334.

When creating an Infoblox::DNS::Host object, setting this method to [] is not supported. If the Infoblox::DNS::Host object does not have associated IPv6 addresses, simply do not pass the ipv6addrs attribute to the constructor. If [] is set and the Host object is submitted for addition, the following error will be reported: "A host record requires at least one IP address." Setting the method to [] is supported for modify requests.

If the Infoblox library is loaded with the :hostaddress option, the valid value is an array reference that contains Infoblox::DHCP::IPv6HostAddr objects.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Gett ipv6addrs
my $ref_ipv6addr = $host->ipv6addrs();
#Modify ipv6addrs
$host->ipv6addrs(["2001::1", "2002::1:2:3:4"]);
```

last_queried( )

Use this method to retrieve the time when the associated record was last queried. This is a read-only attribute.

Parameter

None

Returns

The method returns the attribute value. The number of seconds that have elapsed since January 1st, 1970 UTC.

Example

```
#Get last_queried
my $last_queried = $host->last_queried();
```

ms_ad_user_data( )

Use this method to retrieve Microsoft Active Directory users related information. This is a read-only attribute.

Parameter

None

Returns

The valid return value is an Infoblox::Grid::MSServer::AdUser::Data object.

Example

```
#Get ms_ad_user_data
my $ms_ad_user_data = $host->ms_ad_user_data();
```

name( )

Use this method to set or retrieve the host name.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

The attribute value can be in unicode format.

Parameter

Host name in FQDN (Fully Qualified Domain Name) format. The FQDN consists of the host name followed by the domain name (example: abc.com). A host name can have a maximum of 256 bytes.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get name
my $name = $host->name();
#Modify name
$host-> name("host2.test.com");
```

network_view( )

Use this method to set or retrieve the network view of the DNS host.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

The default value is the "default" network view, which means the DNS host is in the default network view.

Parameter

Valid value is a defined Infoblox::DHCP::View object. The default network view value is the system-defined default network view object.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get network view
my $nview = $host->network_view();
#Modify network view, with an Infoblox::DHCP::View object
$host->network_view($nview);
```

override_cli_credentials( )

The override_cli_credentials attribute controls whether the cli_credentials value in the object is used, instead of the grid default.

The override_cli_credentials attribute can be specified explicitly. It is also set implicitly when cli_credentials is set to a defined value.

Parameter

Specify "true" to set the override_cli_credentials flag or "false" to deactivate/unset it. The default value is "false".

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
# Get attribute value
my $value = $host->override_cli_credentials();
# Modify attribute value
$host->override_cli_credentials('true');
```

rrset_order( )

Use this method to set or retrieve the rrset_order value. The value of this parameters specifies the order in which resource record sets are

returned.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

The possible values of the parameter are "cyclic", "random" and "fixed". The default value of this parameter is "cyclic".

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
# Get rrset_order
my $order = $host->rrset_order()
# Set fixed RR set order
$host->rrset_order("fixed");
```

ttl( )

Use this method to set or retrieve the Time to Live (TTL) value.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

The default value is undefined which indicates that the record inherits the TTL value of the zone.

Specify a TTL value to override the TTL value at the zone level.

Parameter

A 32-bit integer (range from 0 to 4294967295) that represents the duration in seconds that the record is cached. Zero indicates that the record should not be cached. The default value is undefined.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get ttl
my $ttl = $host->ttl();
#Modify ttl
$host->ttl(1800);
#Un-override ttl
$host->ttl(undef);
```

views( )

Use this method to set or retrieve the view of the DNS host.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

The default value is the "default" view, which means the DNS host is located under the default view.

Parameter

Array reference of defined Infoblox::DNS::View objects. The default view value is "default".

Note that the array size must be 1.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get views
my $ref_views = $host->views();
#Modify views, list of Infoblox::DNS::View objects
$host->views([$view1]);
```

zone( )

Use this method to retrieve the zone name of a DNS host object. This method is read-only and cannot be set.

Parameter

None

Returns

Returns the attribute value.

Example

```
# Get zone
my $zone = $host->zone();
```

SAMPLE CODE

The following sample code demonstrates the different functions that can be applied to a Host record object, such as add, search, modify, and remove. This sample also includes error handling for the operations.

#Preparation prior to a DNS host record insertion

```
#PROGRAM STARTS: Include all the modules that will be used

use strict;
use Infoblox;

#Create a session to the Infoblox Appliance
my $session = Infoblox::Session->new(
    master   => "192.168.1.2", #appliance host ip
    username => "admin",     #appliance user login
    password => "infoblox"   #appliance password
);

unless ($session) {
   die("Construct session failed: ",
       Infoblox::status_code() . ":" . Infoblox::status_detail());
}

#Create the zone prior to an host insertion
my $zone = Infoblox::DNS::Zone->new(name => "test.com");

unless ($zone) {
   die("Construct zone failed: ",
       Infoblox::status_code() . ":" . Infoblox::status_detail());
}

print "Zone object created successfully\n";

#Verify if the zone exists
my $object = $session->get(object => "Infoblox::DNS::Zone",
                           name => "test.com");
unless ($object) {
   print "Zone does not exist on server, safe to add the zone\n";
   $session->add($zone)
      or die("Add zone failed: ",
             $session->status_code() . ":" . $session->status_detail());
}
print "Zone added successfully\n";
```

#Create a DNS host record

```
#Construct a DNS host object

my $host = Infoblox::DNS::Host->new(
    name         => "host1.test.com",
    aliases      => ["one.foo.com", "two.foo.com"],
    comment      => "add a host host1.test.com",
    disable      => "true",
    ipv4addrs    => ["4.4.4.4"],
    ipv6addrs    => ["2001::4:4:4:4"],
    ttl          => 3600,
);

unless ($host) {
   die("Construct DNS host record failed: ",
```

```
        Infoblox::status_code() . ":" . Infoblox::status_detail());
}
print "DNS host object created successfully\n";

#Add the DNS host record object to Infoblox appliance through a session
$session->add($host)

    or die("Add host record failed: ",
           $session->status_code() . ":" . $session->status_detail());
     print "DNS host object added to server successfully\n";
```

#Search for a specific DNS host record

```
#Search host record host1.test.com
my @retrieved_objs = $session->search(
    object => "Infoblox::DNS::Host",
    name   => 'host1\.test\.com'
);
my $object = $retrieved_objs[0];
unless ($object) {
    die("Search host record failed: ",
        $session->status_code() . ":" . $session->status_detail());
}
print "Search DNS host object found at least 1 matching entry\n";

#Search all host records that start with "add" in the comment
my @retrieved_objs = $session->search(
    object  => "Infoblox::DNS::Host",
    comment => "add.*",
);
my $object = $retrieved_objs[0];
unless ($object) {
    die("Search host record failed: ",
        $session->status_code() . ":" . $session->status_detail());
}
print "Search DNS host object using regexp in comment field found at least 1 matching entry\n";

#Search all host records that start with "host1" in the zone
my @retrieved_objs = $session->search(
    object => "Infoblox::DNS::Host",
    name   => 'host1.*\.com',
 );
my $object = $retrieved_objs[0];
unless ($object) {
    die("Search host record failed: ",
        $session->status_code() . ":" . $session->status_detail());
}
print "Search DNS host object using regexp in name field found at least 1 matching entry\n";
```

#Get and modify a DNS host record

```
#Get host record through the session
my @retrieved_objs = $session->get(
    object => "Infoblox::DNS::Host",
    name   => "host1.test.com"
);
my $object = $retrieved_objs[0];

unless ($object) {
    die("Get host record failed: ",
        $session->status_code() . ":" . $session->status_detail());
}

#Modify one of the attributes of the specified host record
$object->comment("This is the modified comment");

#Applying the changes
$session->modify($object)
    or die("Modify host record failed: ",
           $session->status_code() . ":" . $session->status_detail());
print "DNS host object modified successfully \n";
```

#Remove a DNS host record.

```
#Get host record through the session
my @retrieved_objs = $session->get(
    object => "Infoblox::DNS::Host",
```

```
    name   => "host1.test.com"
);
my $object = $retrieved_objs[0];
unless ($object) {
    die("Get host record failed: ",
        $session->status_code() . ":" . $session->status_detail());
}

print "Get DNS host object successfull\n";

#Submit the object for removal

$session->remove($object)
    or die("Remove host record failed: ",
           $session->status_code() . ":" . $session->status_detail());
print "DNS host object removed successfully \n";

####PROGRAM ENDS####
```

AUTHOR

Infoblox Inc. http://www.infoblox.com/

SEE ALSO

Infoblox::DNS::View, Infoblox::DHCP::View, Infoblox::DNS::Zone, Infoblox::DHCP::FixedAddr,Infoblox::Session->add(), Infoblox::Session->get(), Infoblox::Session->modify(), Infoblox::Session->remove(), Infoblox::Session->search(), Infoblox::Session, Infoblox::Grid::MSServer::AdUser::Data

COPYRIGHT

Copyright (c) 2017 Infoblox Inc.

Infoblox::DNS::Record::A - DNS A record object.

- NAME
- DESCRIPTION
- CONSTRUCTOR
- SESSION METHODS
  - Infoblox::Session->add( )
  - Infoblox::Session->get( )
  - Infoblox::Session->modify( )
  - Infoblox::Session->remove( )
  - Infoblox::Session->search( )
- METHODS
  - comment( )
  - cloud_info( )
  - creator( )
  - creation_time( )
  - ddns_principal( )
  - ddns_protected( )
  - disable( )
  - discovered_data( )
  - discovered_name( )
  - discoverer( )
  - dns_name( )
  - first_discovered( )
  - forbid_reclamation( )
  - extattrs( )
  - extensible_attributes( )
  - ipv4addr( )
  - last_discovered( )
  - last_queried( )
  - mac_address( )
  - ms_ad_user_data( )
  - name( )
  - netbios( )
  - network_component_description( )
  - network_component_ip( )
  - network_component_name( )
  - network_component_port_description( )
  - network_component_port_name( )
  - network_component_port_number( )
  - network_component_type( )
  - os( )
  - port_duplex( )
  - port_link_status( )
  - port_speed( )
  - port_status( )
  - port_vlan_description( )
  - port_vlan_name( )
  - port_vlan_number( )
  - reclaimable( )
  - ttl( )
  - views( )
  - v_cluster( )
  - v_datacenter( )
  - v_host( )
  - v_name( )
  - v_netadapter( )
  - v_switch( )
  - v_type( )
  - zone( )
- SAMPLE CODE
- AUTHOR
- SEE ALSO
- COPYRIGHT

NAME

Infoblox::DNS::Record::A - DNS A record object.

DESCRIPTION

An A (address) record maps a domain name to an IPv4 address. To define a specific name-to-address mapping, add an A record to a previously defined authoritative forward-mapping zone.

Note that you must specify only one view for the attribute "views".

CONSTRUCTOR

```
 my $bind_a = Infoblox::DNS::Record::A->new (
     ipv4addr              => $ipv4addr,                                                                  #
Required
     name                  => $fqdn,                                                                      #
Required
     comment               => $string,                                                                    #
Optional / Default is empty
     disable               => "true" | "false"                                                            #
Optional / Default "false"
     extattrs              => { $string => $extattr, ... },                                               #
Optional / Default is undefined
     extensible_attributes => { $string => $string | $num, $string => [ $string | $num, ... ], ... },   #
Optional / Default is undefined
     last_discovered       => $string,                                                                    #
Optional / Default is empty
     ttl                   => $num | undef,                                                               #
Optional / Default is undefined
     views                 => [$View1],                                                                   #
Optional / Default is "default" view
     creator               => "DYNAMIC" | "STATIC",                                                       #
Optional / Default is "STATIC"
     ddns_protected        => "true" | "false",                                                           #
Optional / Default is "false"
     ddns_principal        => $string,                                                                    #
Optional / Default is undefined
     forbid_reclamation    => "true" | "false",                                                           #
Optional / Default is "false"
 );
```

You cannot set both extattrs and extensible_attributes attributes at the same time.

SESSION METHODS

This section describes all the methods in Infoblox::Session module that you can apply to a DNS A object.

Infoblox::Session->add( )

Use this method to add an object to the Infoblox appliance. See Infoblox::Session->add() for parameters and return values.

Example

```
#Construct an object
my $bind_a = Infoblox::DNS::Record::A->new (
    ipv4addr => "1.1.1.1",
    name     => "bind_a.domain.com",
    comment  => "this is a demo bind_a record",
    disable  => "true",
    ttl      => "33",
    views    => [$view1]  );
# Submit for addition
my $response = $session->add( $bind_a );
```

Infoblox::Session->get( )

Use this method to retrieve all the matching objects from the Infoblox appliance. See Infoblox::Session->get() for parameters and return values.

Key References

```
Apply the following attributes to get a specific DNS A object:

  name                  - Optional. A host name in string format.
  view                  - Optional. The DNS view in which the A record is located. By default, all
DNS views are searched. But if you omit this attribute and specify a zone, the appliance searches the
'default' view only.
  zone                  - Optional. A zone name in FQDN format.
  ipv4addr              - Optional. The address of the host record.
  reclaimable           - Optional. The flag that indicates whether the record is reclaimable or not.
  extattrs              - Optional. A hash reference containing extensible attributes.
  extensible_attributes - Optional. A hash reference containing extensible attributes.
  creator               - Optional. Record creator.
  ddns_principal        - Optional. GSS-TSIG principal that owns the record.
```

```
  v_cluster             - Optional. Discovered VMware cluster name.
  v_datacenter          - Optional. Discovered VMware datacenter name.
  v_host                - Optional. Discovered VMware server name.
  v_name                - Optional. Discovered VMware host or virtual machine name.
  v_netadapter          - Optional. Discovered VMware physical network adapter name.
  v_switch              - Optional. Discovered VMware virtual switch name.
  v_type                - Optional. Discovered VMware entity type (possible values: "VirtualMachine",
"HostSystem", "VirtualCenter").
```

Example

```
my @retrieved_objs = $session->get(
    object => "Infoblox::DNS::Record::A",
    name   => "bind_a.domain.com",
    view   => "default" );

#get all DNS A recods with the extensible attribute 'Site'
my @retrieved_objs = $session->get(
   object => "Infoblox::DNS::Record::A",
   extensible_attributes => { 'Site' => 'Santa Clara' });

# get DNS A records by v_type:
my @retrieved_objs = $session->get(
   object    => 'Infoblox::DNS::Record::A',
   v_type    => 'VirtualMachine');
```

Infoblox::Session->modify( )

Use this method to modify an object in the Infoblox appliance. See Infoblox::Session->modify() for parameters and return values.

Example

```
# Use method to modify the comment.
$bind_a->comment("this is a modified comment");
# Submit modification
my $response = $session->modify( $bind_a );
```

Infoblox::Session->remove( )

Use this method to submit to remove an object from the Infoblox appliance. See Infoblox::Session->remove() for parameters and return values.

To remove a specifc object, first use `get()` or `search()` to retrieve the specific object, and then submit this object for removal.

Key References

```
remove_options - Optional. A hash reference containing the following attributes:

+-------------------------------------------------------------------------------------------+
| remove_associated_ptr - Optional. Boolean flag that indicates whether the associated PTR  |
|                         record should be removed while deleting the specified A record.   |
+-------------------------------------------------------------------------------------------+
```

Example

```
# Get the objects with the same name
my @retrieved_objs = $session->get(
    object => "Infoblox::DNS::Record::A",
    name   => "bind_a.domain.com",
    view   => "default" );
# Find the desired object from the retrieved list.
my $desired_a = $retrieved_objs[0];
# Submit for removal
my $response = $session->remove(
    $desired_a,
    remove_options => {'remove_associated_ptr' => 'true'} );
```

Infoblox::Session->search( )

Use this method to search for DNS A record objects in the Infoblox appliance. See Infoblox::Session->search() for para meters and return values.

Key References

```
Apply the following attributes to search for a DNS A record object:

  name                  - Optional. A host name in string format (regular expression).
  view                  - Optional. The DNS view in which the A record is located. By default, all
DNS views are searched. But if you omit this attribute and specify a zone, the appliance searches the
'default' view only.
  zone                  - Optional. A zone name in FQDN format.
```

```
  comment               - Optional. A comment in string format (regular expression).
  ipv4addr              - Optional. The address of the host record (regular expression).
  reclaimable           - Optional. The flag that indicates whether the record is reclaimable or not.
  extattrs              - Optional. A hash reference containing extensible attributes.
  extensible_attributes - Optional. A hash reference containing extensible attributes.
  creator               - Optional. Record creator.
  ddns_principal        - Optional. GSS-TSIG principal that owns the record.
  v_cluster             - Optional. Discovered VMware cluster name (regular expression).
  v_datacenter          - Optional. Discovered VMware datacenter name (regular expression).
  v_host                - Optional. Discovered VMware server name (regular expression).
  v_name                - Optional. Discovered VMware host or virtual machine name (regular
expression).
  v_netadapter          - Optional. Discovered VMware physical network adapter name (regular
expression).
  v_switch              - Optional. Discovered VMware virtual switch name (regular expression).
  v_type                - Optional. Discovered VMware entity type (possible values: "VirtualMachine",
"HostSystem", "VirtualCenter").
```

For more information about searching extensible attributes, see Infoblox::Grid::ExtensibleAttributeDef/Searching Extensible Attributes.

Example

```
# search for all DNS A objects that match "domain.com" in the default DNS view
my @retrieved_objs = $session->search(
    object => "Infoblox::DNS::Record::A",
    name   => 'domain\.com',
    view   => "default" );

# search for all DNS A records in the "domain.com" zone of the default view
my @retrieved_objs = $session->search(
    object => "Infoblox::DNS::Record::A",
    zone   => "domain.com",
    view   => "default" );

#get all DNS A recods with the extensible attribute 'Site'
my @retrieved_objs = $session->search(
  object => "Infoblox::DNS::Record::A",
  extensible_attributes => { 'Site' => 'Santa Clara' });

# search DNS A records by v_datacenter:
my @retrieved_objs = $session->search(
   object       => "Infoblox::DNS::Record::A",
   v_datacenter => "test.*");
```

METHODS

This section describes all the methods that you can use to configure and retrieve the attribute values of an A record

comment( )

Use this method to set or retrieve the descriptive comment.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

Desired comment in string format with a maximum of 256 bytes.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get comment
my $comment = $bind_a->comment();
#Modify comment
$bind_a->comment("Modifying the DNS A comment");
```

cloud_info( )

Use this method to retrieve cloud API related information for the Infoblox::DNS::Record::A object.

Parameter

None

Returns

The method returns the attribute value.

Example

```
# Get cloud_info
my $cloud_info = $object->cloud_info();
```

creator( )

Use this method to set or retrieve the record creator.

Note that changing creator from or to 'SYSTEM' value is not allowed.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Paramter

The valid values are 'STATIC' and 'DYNAMIC'. The default value is 'STATIC'.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get creator value
my $creator = $object->creator();

#Modify creator value
$object->creator("DYNAMIC");
```

creation_time( )

Use this method to retrieve the creation time for the record. This is a read-only attribute.

Parameter

None

Returns

The valid return value is a number of seconds that have elapsed since January 1st, 1970 UTC.

Example

```
#Get creation_time value
my $creation_time = $object->creation_time();
```

ddns_principal( )

Use this method to set or retrive the GSS-TSIG principal that owns this record.

Note that you cannot set ddns_principal for 'STATIC' and 'SYSTEM' records.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Paramter

The GSS-TSIG principal FQDN (Fully Qualified Domain Name) format. The FQDN consists of the hostname followed by the domain name (example: abc.com). A hostname can have a maximum of 256 characters.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get ddns_principal value
my $ddns_principal = $object->ddns_principal();

#Modify ddns_principal value
$object->ddns_principal('foo.com');
```

ddns_protected( )

Use this method to set or retrieve the flag that indicates whether DDNS updates for this record are allowed or not.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

Specify 'true' to protect record from DDNS updates and 'false' to allow DDNS updates for the specified record.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get ddns_protected value
my $ddns_protected = $object->ddns_protected();

#Modify ddns_protected value
$object->ddns_protected('true');
```

disable( )

Use this method to set or retrieve the disable flag of a DNS record.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

The default value for this field is false. The DNS record is enabled.

Parameter

Specify "true" to set the disable flag or "false" to deactivate/unset it.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get disable
my $disable = $bind_a->disable();
#Modify disable
$bind_a->disable("true");
```

discovered_data( )

Use this method to retrieve the discovered data of an A Record object. This is a read-only attribute.

Omit the parameter to retrieve the attribute value.

Parameter

None

Returns

The valid return value is an Infoblox::Grid::Discovery::Data object.

Example

```
#Get discovered_data value
my $discovered_data = $object->discovered_data();
```

discovered_name( )

Use this method to retrieve the discovered name of an A Record object.

Parameter

none

Returns

The method returns the discovered name.

Example

```
#Get discovered_name
my $discovered_name = $bind_a->discovered_name();
```

discoverer( )

Use this method to retrieve the discoverer of an A Record object.

Parameter

none

Returns

The method returns the discoverer value.

Example

```
#Get discoverer
my $discoverer = $bind_a->discoverer();
```

dns_name( )

Use this method to retrieve the host name in punycode format. This is a read-only attribute.

Parameter

None

Returns

The method returns the attribute value.

Example

```
# Get attribute value
my $value = $bind_a->dns_name();
```

first_discovered( )

Use this method to retrieve the date and time that the A Record object was first discovered.

Parameter

none

Returns

The method returns the date and time when the A Record object was first discovered.

Example

```
#Get first_discovered
my $first_discovered = $bind_a->first_discovered();
```

forbid_reclamation( )

Use this method to set or retrieve the flag that indicates whether the reclamation is allowed for the record or not.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

Specify 'true' to forbid reclamation for the record and 'false' to allow it. The default value is 'false'.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get forbid_reclamation
my $forbid_reclamation = $object->forbid_reclamation();
#Modify forbid_reclamation
```

```
$object->forbid_reclamation('true');
```

extattrs( )

Use this method to set or retrieve the extensible attributes associated with a DNS A record object.

Parameter

Valid value is a hash reference containing the names of extensible attributes and their associated values (Infoblox::Grid::Extattr objects).

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get extattrs
my $ref_extattrs = $bind_a->extattrs();
#Modify extattrs
$bind_a->extattrs({ 'Site' => $extattr1, 'Administrator' => $extattr2 });
```

extensible_attributes( )

Use this method to set or retrieve the extensible attributes associated with a DNS A record.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

For valid values for extensible attributes, see Infoblox::Grid::ExtensibleAttributeDef/Extensible Attribute Values.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get extensible attributes
my $ref_extensible_attributes = $bind_a->extensible_attributes();
#Modify extensible attributes
$bind_a->extensible_attributes({ 'Site' => 'Santa Clara', 'Administrator' => [ 'Peter', 'Tom' ] });
```

ipv4addr( )

Use this method to set or retrieve the the IPv4 address.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

An IPv4 address is a 32-bit number in dotted decimal notation. It consists of four 8-bit groups of decimal digits separated by decimal points (example: 192. 168.1.2).

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get ipv4addr
my $ipv4addr = $bind_a->ipv4addr();
#Modify ipv4addr
$bind_a->ipv4addr("2.2.2.2");
```

last_discovered( )

Use this method to retrieve the time this object was last seen by a discovery job.

The default value for this field is empty.

Parameter

N/A

Returns

The method returns the attribute value.

Example

```
#Get last_discovered
my $last_discovered = $bind_a->last_discovered();
```

last_queried( )

Use this method to retrieve the time when the associated record was last queried. This is a read-only attribute.

Parameter

None

Returns

The method returns the attribute value. The number of seconds that have elapsed since January 1st, 1970 UTC.

Example

```
#Get last_queried
my $last_queried = $bind_a->last_queried();
```

mac_address( )

Use this method to retrieve the discovered MAC address of this object.

The default value for this field is empty.

Parameter

N/A

Returns

The method returns the attribute value.

Example

```
#Get mac_address
my $mac_address = $bind_a->mac_address();
```

ms_ad_user_data( )

Use this method to retrieve Microsoft Active Directory users related information. This is a read-only attribute.

Parameter

None

Returns

The valid return value is an Infoblox::Grid::MSServer::AdUser::Data object.

Example

```
#Get ms_ad_user_data
my $ms_ad_user_data = $bind_a->ms_ad_user_data();
```

name( )

Use this method to set or retrieve the host name.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

The attribute value can be in unicode format.

Parameter

Hostname in FQDN (Fully Qualified Domain Name) format. The FQDN consists of the hostname followed by the domain name (example: abc.com). A hostname can have a maximum of 256 bytes.

Infoblox also supports wildcard A records. A wildcard A record maps all the hostnames in a domain to a single IP address.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get name
my $name = $bind_a->name();
#Modify name
$bind_a->name("bind_a.domain.com");
```

netbios( )

Use this method to retrieve the name in the NetBIOS reply that responded to a NetBIOS query.

Returns

The method returns the attribute value.

Example

```
#Get netbios
my $netbios = $bind_a->netbios();
```

network_component_description( )

Use this method to retrieve the description of the network device that is connected to the A Record object.

Parameter

none

Returns

The method returns the network device description.

Example

```
#Get network_component_description
my $network_component_description = $bind_a->network_component_description();
```

network_component_ip( )

Use this method to retrieve the IP address of the network device that is connected to the A Record object.

Parameter

none

Returns

The method returns the network device IP address.

Example

```
#Get network_component_ip
my $network_component_ip = $bind_a->network_component_ip();
```

network_component_name( )

Use this method to retrieve the name of the network device that is connected to the A Record object.

Parameter

none

Returns

The method returns the network device name.

Example

```
#Get network_component_name
my $network_component_name = $bind_a->network_component_name();
```

network_component_port_description( )

Use this method to retrieve the description of the network device port that is connected to the A Record object.

Parameter

none

Returns

The method returns the network device port description.

Example

```
#Get network_component_port_description
my $network_component_port_description = $bind_a->network_component_port_description();
```

network_component_port_name( )

Use this method to retrieve the name of the network device port that is connected to the A Record object.

Parameter

none

Returns

The method returns the name of the port on the network device.

Example

```
#Get network_component_port_name
my $network_component_port_name = $bind_a->network_component_port_name();
```

network_component_port_number( )

Use this method to retrieve the number of the network device port that is connected to the A Record object.

Parameter

none

Returns

The method returns the network device port number.

Example

```
#Get network_component_port_number
my $network_component_port_number = $bind_a->network_component_port_number();
```

network_component_type( )

Use this method to retrieve the type of the network device that is connected to the A Record object.

Parameter

none

Returns

The method returns the network device type.

Example

```
#Get network_component_type
my $network_component_type = $bind_a->network_component_type();
```

os( )

Use this method to retrieve the operating system associated with this A record.

Returns

The method returns the attribute value.

Example

```
#Get os
my $os = $bind_a->os();
```

port_duplex( )

Use this method to retrieve the duplex setting of the network device port that is connected to the A Record object.

Parameter

none

Returns

The method returns the network device port duplex setting. Returned values, if any, are one of the following:

```
Full
Half
```

Example

```
#Get port_duplex
my $port_duplex = $bind_a->port_duplex();
```

port_link_status( )

Use this method to retrieve the link status of the network device port that is connected to the A Record object.

Parameter

none

Returns

The method returns the port link status. Returned values, if any, are one of the following:

```
Connected
Not Connected
Unknown
```

Example

```
#Get port_link_status
my $port_link_status = $bind_a->port_link_status();
```

port_speed( )

Use this method to retrieve the speed of the network device port that is connected to the A Record object.

Parameter

none

Returns

The method returns the network device port speed value. Returned values, if any, are one of the following:

```
10M
100M
1G
10G
100G
Unknown
```

Example

```
#Get port_speed
my $port_speed = $bind_a->port_speed();
```

port_status( )

Use this method to retrieve the status of the network device port that is connected to the A Record object.

Parameter

none

Returns

The method returns the network device port status. Returned values, if any, are one of the following:

```
Up
Down
Unknown
```

Example

```
#Get port_status
my $port_status = $bind_a->port_status();
```

port_vlan_description( )

Use this method to retrieve the VLAN description of the network device port that is connected to the A Record object.

Parameter

none

Returns

The method returns the VLAN description of the network device port.

Example

```
#Get port_vlan_description
my $port_vlan_description = $bind_a->port_vlan_description();
```

port_vlan_name( )

Use this method to retrieve the VLAN name of the network device port that is connected to the A Record object.

Parameter

none

Returns

The method returns the VLAN name of the network device port.

Example

```
#Get port_vlan_name
my $port_vlan_name = $bind_a->port_vlan_name();
```

port_vlan_number( )

Use this method to retrieve the VLAN number of the network device port that is connected to the A Record object.

Parameter

none

Returns

The method returns the network device port VLAN number.

Example

```
#Get port_vlan_number
my $port_vlan_number = $bind_a->port_vlan_number();
```

reclaimable( )

Use this method to retrieve the flag that indicates whether the record is reclaimable or not.

Parameter

None

Returns

The method returns the attribute value.

Example

```
#Get reclaimable
my $reclaimable = $object->reclaimable();
```

ttl( )

Use this method to set or retrieve the Time to Live (TTL) value.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

The default value is undefined which indicates that the record inherits the TTL value of the zone.

Specify a TTL value to override the TTL value at the zone level.

Parameter

A 32-bit integer (range from 0 to 4294967295) that represents the duration in seconds that the record is cached. Zero indicates that the record should not be cached.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get ttl
my $ttl = $bind_a->ttl();
#Modify ttl
$bind_a->ttl(1800);
#Un-override ttl
$bind_a->ttl(undef);
```

views( )

Use this method to set or retrieve the view of the A record.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

The default value is the "default" view, which means the A record is located under the default view.

Parameter

Array reference of defined Infoblox::DNS::View objects.

Note that the array size must be 1.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get views
my $ref_views = $bind_a->views();
#Modify views, list of Infoblox::DNS::View objects
$bind_a->views([$view1]);
```

v_cluster( )

Use this method to retrieve the name of the VMware cluster associated with the A Record object. This method is read-only.

Parameter

none

Returns

This method returns a string that contains the VMware cluster name. The default value is an empty string.

Example

```
#Get v_cluster
my $v_cluster = $bind_a->v_cluster();
```

v_datacenter( )

Use this method to retrieve the name of the VMware datacenter associated with the A Record object. This method is read-only.

Parameter

none

Returns

This method returns a string that contains the VMware datacenter name. The default value is an empty string.

Example

```
#Get v_datacenter
my $v_datacenter = $bind_a->v_datacenter();
```

v_host( )

Use this method to retrieve the name of the VMware host associated with the A Record object. This method is read-only.

Parameter

none

Returns

This method returns a string that contains the VMware host name. The default value is an empty string.

Example

```
#Get v_host
my $v_host = $bind_a->v_host();
```

v_name( )

Use this method to retrieve the name of the VMware entity associated with the A Record object. This method is read-only.

Parameter

none

Returns

This method returns a string that contains the VMware entity name. The default value is an empty string.

Example

```
#Get v_name
my $v_name = $bind_a->v_name();
```

v_netadapter( )

Use this method to retrieve the name of the physical network adapter through which the VMware entity is connected to the appliance. This method is read-only.

Parameter

none

Returns

This method returns a string that contains the network adapter name. The default value is an empty string.

Example

```
#Get v_netadapter
my $v_netadapter = $bind_a->v_netadapter();
```

v_switch( )

Use this method to retrieve the name of the virtual switch through which the VMware entity is connected to the appliance. This method is read-only.

Parameter

none

Returns

This method returns a string that contains the virtual switch name. The default value is an empty string.

Example

```
#Get v_switch
```

```
my $v_switch = $bind_a->v_switch();
```

v_type( )

Use this method to retrieve the type of VMware entity associated with the A Record object. This method is read-only.

Parameter

none

Returns

This method returns a string that contains the VMware entity type. The default value is an empty string. The returned value, if any, can be one of the following:

```
VirtualMachine
HostSystem
VirtualCenter
```

Example

```
#Get v_type
my $v_type = $bind_a->v_type();
```

zone( )

Use this method to retrieve the zone name of a DNS A record. This method is read-only and cannot be set.

Parameter

None

Returns

Returns the attribute value.

Example

```
# Get zone
my $zone = $bind_a->zone();
```

SAMPLE CODE

The following sample code demonstrates the different functions that can be applied to an object, such as add, search, modify, and remove. This sample also includes error handling for the operations.

#Preparation prior to a DNS A record insertion

```
#PROGRAM STARTS: Include all the modules that will be used
use strict;
use Infoblox;

#Create a session to the Infoblox Appliance
my $session = Infoblox::Session->new(
    master   => "192.168.1.2",
    username => "admin",
    password => "infoblox"
);
unless ($session) {
   die("Construct session failed: ",
       $session->status_code() . ":" . $session->status_detail());
}
print "Session created successfully\n";

#Create the zone prior to an A record insertion
my $zone = Infoblox::DNS::Zone->new(name => "domain.com");
unless ($zone) {
   die("Construct zone failed: ",
       Infoblox::status_code() . ":" . Infoblox::status_detail());
}
print "Zone object created successfully\n";

#Verify if the zone exists
my $object = $session->get(object => "Infoblox::DNS::Zone", name => "domain.com");
unless ($object) {
   print "Zone does not exist on server, safe to add the zone\n";
   $session->add($zone)
```

```
        or die("Add zone failed: ",
               $session->status_code() . ":" . $session->status_detail());
}
print "Zone added successfully\n";
```

#Create a DNS A record

```
#Construct a DNS A object
my $bind_a = Infoblox::DNS::Record::A->new(
    name     => "bind_a.domain.com",
    comment  => " this is a demo bind_a record ",
    ipv4addr => "1.1.1.1"
);
unless ($bind_a) {
   die("Construct DNS record A failed: ",
       Infoblox::status_code() . ":" . Infoblox::status_detail());
}
print "DNS A object created successfully\n";

#Add the DNS A record object to Infoblox Appliance through a session
$session->add($bind_a)
    or die("Add record A failed: ",
           $session->status_code() . ":" . $session->status_detail());
print "DNS A object added to server successfully\n";
```

#Search for a specific DNS A record

```
#Search all A records that match "domain.com"
my @retrieved_objs = $session->search(
    object => "Infoblox::DNS::Record::A",
    name   => 'domain\.com'
);
my $object = $retrieved_objs[0];
unless ($object) {
    die("Search record A failed: ",
        $session->status_code() . ":" . $session->status_detail());
}
print "Search DNS A object found at least 1 matching entry\n";

#Search all A records that start with "bind" and end with ".domain.com"
my @retrieved_objs = $session->search(
    object => "Infoblox::DNS::Record::A",
    name   => '^bind.*\.domain\.com\$'
);
my $object = $retrieved_objs[0];
unless ($object) {
    die("Search record A failed: ",
        $session->status_code() . ":" . $session->status_detail());
}
print "Search DNS A object using regexp found at least 1 matching entry\n";
```

#Get and modify a DNS A record

```
#Get A record through the session
my @retrieved_objs = $session->get(
    object => "Infoblox::DNS::Record::A",
    name   => "bind_a.domain.com"
);
my $object = $retrieved_objs[0];
unless ($object) {
    die("Get record A failed: ",
        $session->status_code() . ":" . $session->status_detail());
}
print "Get DNS A object found at least 1 matching entry\n";

#Modify one of the attributes of the specified A record
$object->ipv4addr("3.3.3.3");

#Apply the changes
$session->modify($object)
    or die("Modify record A failed: ",
           $session->status_code() . ":" . $session->status_detail());
print "DNS A object modified successfully \n";
```

#Remove a DNS A record

```
#Get A record through the session
```

```
my @retrieved_objs = $session->get(
    object => "Infoblox::DNS::Record::A",
    name   => "bind_a.domain.com"
);
my $object = $retrieved_objs[0];
unless ($object) {
    die("Get record A failed: ",
        $session->status_code() . ":" . $session->status_detail());
}
print "Get DNS A object found at least 1 matching entry\n";

#Submit the object for removal
$session->remove($object)
    or die("Remove record A failed: ",
        $session->status_code() . ":" . $session->status_detail());
print "DNS A object removed successfully \n";

####PROGRAM ENDS####
```

AUTHOR

Infoblox Inc. http://www.infoblox.com/

SEE ALSO

Infoblox::DNS::View, Infoblox::DNS::Zone, Infoblox::Session, Infoblox::Session->get(), Infoblox::Session->search(), Infoblox::Session->add(), Infoblox::Session->remove(), Infoblox::Session->modify(), Infoblox::Grid::MSServer::AdUser::Data, Infoblox::Grid::Discovery::Data

---

COPYRIGHT

Copyright (c) 2017 Infoblox Inc.

Infoblox::DNS::Record::AAAA - DNS AAAA record object.

- NAME
- DESCRIPTION
- CONSTRUCTOR
- SESSION METHODS
  - Infoblox::Session->add( )
  - Infoblox::Session->get( )
  - Infoblox::Session->modify( )
  - Infoblox::Session->remove( )
  - Infoblox::Session->search( )
- METHODS
  - comment( )
  - cloud_info( )
  - creator( )
  - creation_time( )
  - ddns_principal( )
  - ddns_protected( )
  - disable( )
  - discovered_data( )
  - discovered_duid( )
  - discovered_name( )
  - discoverer( )
  - dns_name( )
  - extattrs( )
  - extensible_attributes( )
  - first_discovered( )
  - forbid_reclamation( )
  - ipv6addr( )
  - last_discovered( )
  - last_queried( )
  - ms_ad_user_data( )
  - name( )
  - netbios( )
  - network_component_description( )
  - network_component_ip( )
  - network_component_name( )
  - network_component_port_description( )
  - network_component_port_name( )
  - network_component_port_number( )
  - network_component_type( )
  - os( )
  - port_duplex( )
  - port_link_status( )
  - port_speed( )
  - port_status( )
  - port_vlan_description( )
  - port_vlan_name( )
  - port_vlan_number ( )
  - reclaimable( )
  - ttl( )
  - views( )
  - zone( )
- SAMPLE CODE
- AUTHOR
- SEE ALSO
- COPYRIGHT

NAME

Infoblox::DNS::Record::AAAA - DNS AAAA record object.

DESCRIPTION

An AAAA record, also known as a quad A address record, maps a domain name to an IPv6 address. To define a specific name-to-address mapping, add an AAAA record to a previously defined authoritative forward-mapping zone.

Note that you must specify only one view for the attribute "views".

CONSTRUCTOR

```
 my $bindaaaa = Infoblox::DNS::Record::AAAA->new (
     ipv6addr              => $ipv6addr,                                                  #
Required
     name                  => $fqdn,                                                      #
Required
     comment               => $string,                                                    #
Optional / Default is empty
     extattrs              => { $string => $extattr, ... },                               #
Optional / Default is undefined
     extensible_attributes => { $string => $string | $num, $string => [ $string | $num, ... ], ... }, #
Optional / Default is undefined
     disable               => "true" | "false"                                            #
Optional / Default "false"
     ttl                   => $num | undef,                                               #
Optional / Default is undefined
     views                 => [$View1],                                                   #
Optional / Default is "default" view
     creator               => "DYNAMIC" | "STATIC",                                       #
Optional / Default is "STATIC"
     ddns_protected        => "true" | "false",                                           #
Optional / Default is "false"
     ddns_principal        => $string,                                                    #
Optional / Default is undefined
     forbid_reclamation    => "true" | "false",                                           #
Optional / Default is "false"
 );
```

You cannot set both extattrs and extensible_attributes attributes at the same time.

SESSION METHODS

This section describes all the methods in Infoblox::Session module that you can apply to a DNS AAAA record object.

Infoblox::Session->add( )

Use this method to add a AAAA record object to the Infoblox appliance. See Infoblox::Session->add() for parameters and return values.

Example

```
#Construct an object
my $bindaaaa = Infoblox::DNS::Record::AAAA->new(
    name     => "aaaa1.domain.com",
    comment  => "add a DNS AAAA aaaa1.domain.com",
    ipv6addr => "aaaa:1111:bbbb:2222:cccc:3333:dddd:4444",
    disable  => "true",
    ttl      => "33",
    views    => [$view1]  );
# Submit for addition
my $response = $session->add( $bindaaaa );
```

Infoblox::Session->get( )

Use this method to retrieve all the matching objects from the Infoblox appliance. See Infoblox::Session->get() for parameters and return values.

Key References

```
 Apply the following attributes to get a specific DNS AAAA object:

 name                              - Optional. A host name in string format.
 view                              - Optional. The DNS view in which the host record is located. By
default, all DNS views are searched. But if you omit this attribute and specify a zone, the appliance
searches the 'default' view only.
 zone                              - Optional. A zone name in FQDN format.
 ipv6addr                          - Optional. The address of the AAAA record.
 creator                           - Optional. Record creator.
 ddns_principal                    - Optional. GSS-TSIG principal that owns the record.
 reclaimable                       - Optional. The flag that indicates whether the record is
reclaimable or not.
 extattrs                          - Optional. A hash reference containing extensible attributes.
 extensible_attributes             - Optional. A hash reference containing extensible attributes.

 discovered_duid                   - Optional. Discovered DHCPv6 UID.
 discovered_name                   - Optional. Discovered name.
 discoverer                        - Optional. Discoverer name.
 first_discovered                  - Optional. Time of the first discovery.
 last_discovered                   - Optional. Time of the last discovery.
 netbios                           - Optional. Discovered NetBIOS name.
 network_component_description     - Optional. Discovered network component description.
 network_component_ip              - Optional. Discovered network component IP address.
 network_component_name            - Optional. Discovered network component name.
 network_component_port_description - Optional. Discovered network component port description.
```

```
network_component_port_name        - Optional. Discovered network component port name.
network_component_port_number      - Optional. Discovered network component port number.
network_component_type             - Optional. Discovered network component type.
os                                 - Optional. Discovered OS.
port_duplex                        - Optional. Discovered port duplex settings.
port_link_status                   - Optional. Discovered port link status.
port_speed                         - Optional. Discovered port speed settings.
port_status                        - Optional. Discovered port status settings.
port_vlan_description              - Optional. Discovered port VLAN description.
port_vlan_name                     - Optional. Discovered port VLAN name.
port_vlan_number                   - Optional. Discovered port VLAN number.
```

Example

```
my @retrieved_objs = $session->get(
                      object => "Infoblox::DNS::Record::AAAA",
                      name   => "aaaa1.domain.com",
                      view   => "default" );

# get all the DNS AAAA records in the "domain.com" zone of the default view
my @retrieved_objs = $session->get(
                      object => "Infoblox::DNS::Record::AAAA",
                      zone   => "domain.com",
                      view   => "default" );

#get all DNS AAAA recods with the extensible attribute 'Site'
my @retrieved_objs = $session->get(
   object => "Infoblox::DNS::Record::AAAA",
   extensible_attributes => { 'Site' => 'Santa Clara' });
```

Infoblox::Session->modify( )

Use this method to modify a AAAA record object in the Infoblox appliance. See Infoblox::Session->modify() for parameters and return values.

Example

```
# Use method to modify the comment.
$bindaaaa->comment("this is a modified comment");
# Submit modification
my $response = $session->modify( $bindaaaa );
```

Infoblox::Session->remove( )

Use this method to remove a AAAA record object from the Infoblox appliance. See Infoblox::Session->remove() for parameters and return values.

To remove a specifc object, first use `get()` or `search()` to retrieve the specific AAAA record object, and then submit this object for removal.

Key References

```
remove_options - Optional. A hash reference containing the following attributes:

+-------------------------------------------------------------------------------------------+
| remove_associated_ptr - Optional. Boolean flag that indicates whether the associated PTR  |
|                         record should be removed while deleting the specified AAAA record.|
+-------------------------------------------------------------------------------------------+
```

Example

```
# Get the objects with the same name
my @retrieved_objs = $session->get(
    object => "Infoblox::DNS::Record::AAAA",
    name   => "aaaa1.domain.com",
    view   => "default" );
# Find the desired object from the retrieved list.
my $desired_aaaa = $retrieved_objs[0];
# Submit for removal
my $response = $session->remove(
    $desired_aaaa,
    remove_options => {'remove_associated_ptr' => 'true'} );
```

Infoblox::Session->search( )

Use this method to search for DNS AAAA record objects in the Infoblox appliance. See Infoblox::Session->search() for parameters and return values.

Key References

```
 Apply the following attributes, to search for a DNS AAAA record object:

  name                                  - Optional. A host name in string format (regular
expression).
  view                                  - Optional. The DNS view in which the AAAA record is located.
By default, all DNS views are searched. But if you omit this attribute and specify a zone, the
appliance searches the 'default' view only.
  zone                                  - Optional. A zone name in FQDN format.
  comment                               - Optional. Comment in string format (regular expression).
  creator                               - Optional. Record creator.
  ddns_principal                        - Optional. GSS-TSIG principal that owns the record.
  reclaimable                           - Optional. The flag that indicates whether the record is
reclaimable or not.
  ipv6addr                              - Optional. The address of the AAAA record (regular
expression).

  discovered_duid                       - Optional. Discovered DHCPv6 UID (exact search only).
  discovered_name                       - Optional. Discovered name(regex).
  discoverer                            - Optional. Discoverer name(regex).
  first_discovered                      - Optional. Time of the first discovery (in ISO format, exact
search only).
  last_discovered                       - Optional. Time of the last discovery (in ISO format, exact
search only).
  netbios                               - Optional. Discovered NetBIOS name(regex).
  network_component_description         - Optional. Discovered network component description(regex).
  network_component_ip                  - Optional. Discovered network component IP address(regex).
  network_component_name                - Optional. Discovered network component name(regex).
  network_component_port_description    - Optional. Discovered network component port
description(regex).
  network_component_port_name           - Optional. Discovered network component port name(regex).
  network_component_port_number         - Optional. Discovered network component port number (exact
search only).
  network_component_type                - Optional. Discovered network component type(regex).
  os                                    - Optional. Discovered OS (regex).
  port_duplex                           - Optional. Discovered port duplex settings (exact search
only).
  port_link_status                      - Optional. Discovered port link status (exact search only).
  port_speed                            - Optional. Discovered port speed settings (exact search
only).
  port_status                           - Optional. Discovered port status settings (exact search
only).
  port_vlan_description                 - Optional. Discovered port VLAN description(regex).
  port_vlan_name                        - Optional. Discovered port VLAN name(regex).
  port_vlan_number                      - Optional. Discovered port VLAN number(exact search only).

  extattrs                              - Optional. A hash reference containing extensible
attributes.
  extensible_attributes                 - Optional. A hash reference containing extensible
attributes.
```

For more information about searching extensible attributes, see Infoblox::Grid::ExtensibleAttributeDef/Searching Extensible Attributes.

Example

```
# search for all DNS AAAA objects that match "domain.com" in the default DNS view
my @retrieved_objs = $session->search(
    object  => "Infoblox::DNS::Record::AAAA",
    name    => 'domain\.com',
    view    => "default",
    comment => ".*modified comment" );

# search for all DNS AAAA records in zone "domain.com" of default view
my @retrieved_objs = $session->search(
                     object => "Infoblox::DNS::Record::AAAA",
                     zone   => "domain.com",
                     view   => "default" );

# search all DNS AAAA recods with the extensible attribute 'Site'
my @retrieved_objs = $session->search(
   object => "Infoblox::DNS::Record::AAAA",
   extensible_attributes => { 'Site' => 'Santa Clara' });
```

METHODS

This section describes all the methods that you can use to configure and retrieve the attribute values of an AAAA record

comment( )

Use this method to set or retrieve the descriptive comment.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

Desired comment in string format with a maximum of 256 bytes.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get comment
my $comment = $bindaaaa->comment();
#Modify comment
$bindaaaa->comment("Modifying the DNS AAAA comment");
```

cloud_info( )

Use this method to retrieve cloud API related information for the Infoblox::DNS::Record::AAAA object.

Parameter

None

Returns

The method returns the attribute value.

Example

```
# Get cloud_info
my $cloud_info = $object->cloud_info();
```

creator( )

Use this method to set or retrieve the record creator.

Note that changing creator from or to 'SYSTEM' value is not allowed.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Paramter

The valid values are 'STATIC' and 'DYNAMIC'. The default value is 'STATIC'.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get creator value
my $creator = $object->creator();

#Modify creator value
$object->creator("DYNAMIC");
```

creation_time( )

Use this method to retrieve the creation time for the record. This is a read-only attribute.

Parameter

None

Returns

The valid return value is a number of seconds that have elapsed since January 1st, 1970 UTC.

Example

```
#Get creation_time value
my $creation_time = $object->creation_time();
```

ddns_principal( )

Use this method to set or retrive the GSS-TSIG principal that owns this record.

Note that you cannot set ddns_principal for 'STATIC' and 'SYSTEM' records.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Paramter

The GSS-TSIG principal FQDN (Fully Qualified Domain Name) format. The FQDN consists of the hostname followed by the domain name (example: abc.com). A hostname can have a maximum of 256 characters.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get ddns_principal value
my $ddns_principal = $object->ddns_principal();

#Modify ddns_principal value
$object->ddns_principal('foo.com');
```

ddns_protected( )

Use this method to set or retrieve the flag that indicates whether DDNS updates for this record are allowed or not.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

Specify 'true' to protect record from DDNS updates and 'false' to allow DDNS updates for the specified record.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get ddns_protected value
my $ddns_protected = $object->ddns_protected();

#Modify ddns_protected value
$object->ddns_protected('true');
```

disable( )

Use this method to set or retrieve the disable flag of a DNS record.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

The default value for this field is false. The DNS record is enabled.

Parameter

Specify "true" to set the disable flag or "false" to deactivate/unset it.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get disable
my $disable = $bindaaaa->disable();
#Modify disable
$bindaaaa->disable("true");
```

discovered_data( )

Use this method to retrieve the discovered data of an AAAA Record object. This is a read-only attribute.

Omit the parameter to retrieve the attribute value.

Parameter

None

Returns

The valid return value is an Infoblox::Grid::Discovery::Data object.

Example

```
#Get discovered_data value
my $discovered_data = $object->discovered_data();
```

discovered_duid( )

Use this method to retrieve the discovered DHCPv6 UID (DUID) of an AAAA record object.

Parameter

none

Returns

The method returns the discovered name.

Example

```
#Get discovered_duid
my $discovered_duid = $bind_aaaa->discovered_duid();
```

discovered_name( )

Use this method to retrieve the discovered name of an AAAA Record object.

Parameter

none

Returns

The method returns the discovered name.

Example

```
#Get discovered_name
my $discovered_name = $bindaaa->discovered_name();
```

discoverer( )

Use this method to retrieve the discoverer of an AAAA Record object.

Parameter

none

Returns

The method returns the discoverer value.

Example

```
#Get discoverer
my $discoverer = $bindaaa->discoverer();
```

dns_name( )

Use this method to retrieve the host name in punycode format. This is a read-only attribute.

Parameter

None

Returns

The method returns the attribute value.

Example

```
# Get attribute value
my $value = $bindaaaa->dns_name();
```

extattrs( )

Use this method to set or retrieve the extensible attributes associated with a DNS AAAA record object.

Parameter

Valid value is a hash reference containing the names of extensible attributes and their associated values (Infoblox::Grid::Extattr objects).

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get extattrs
my $ref_extattrs = $bindaaaa->extattrs();
#Modify extattrs
$bindaaaa->extattrs({ 'Site' => $extattr1, 'Administrator' => $extattr2 });
```

extensible_attributes( )

Use this method to set or retrieve the extensible attributes associated with a DNS AAAA record.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

For valid values for extensible attributes, see Infoblox::Grid::ExtensibleAttributeDef/Extensible Attribute Values.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get extensible attributes
my $ref_extensible_attributes = $bindaaaa->extensible_attributes();
#Modify extensible attributes
$bindaaaa->extensible_attributes({ 'Site' => 'Santa Clara', 'Administrator' => [ 'Peter', 'Tom' ]
});
```

first_discovered( )

Use this method to retrieve the date and time that the AAAA Record object was first discovered.

Parameter

none

Returns

The method returns the date and time when the AAAA Record object was first discovered.

Example

```
#Get first_discovered
my $first_discovered = $bindaaa->first_discovered();
```

forbid_reclamation( )

Use this method to set or retrieve the flag that indicates whether the reclamation is allowed for the record or not.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

Specify 'true' to forbid reclamation for the record and 'false' to allow it. The default value is 'false'.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get forbid_reclamation
my $forbid_reclamation = $object->forbid_reclamation();
#Modify forbid_reclamation
$object->forbid_reclamation('true');
```

ipv6addr( )

Use this method to set or retrieve the the IPv6 address.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

An IPv6 address is a 128-bit number in colon hexadecimal notation. It consists of eight 16-bit groups of hexadecimal digits separated by colons (example: 12ab:0000:0000:0123:4567:89ab:0000:cdef).

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get ipv6addr
my ipv6addr = $bindaaaa->ipv6addr();
#Modify ipv6addr
$bindaaaa->ipv6addr("aaaa:1111:bbbb:2222:cccc:3333:dddd:5555");
```

last_discovered( )

Use this method to retrieve the time this object was last seen by a discovery job.

This field is blank by default.

Parameter

N/A

Returns

The method returns the attribute value.

Example

```
#Get last_discovered
my $last_discovered = $bindaaa->last_discovered();
```

last_queried( )

Use this method to retrieve the time when the associated record was last queried. This is a read-only attribute.

Parameter

None

Returns

The method returns the attribute value. The number of seconds that have elapsed since January 1st, 1970 UTC.

Example

```
#Get last_queried
my $last_queried = $bindaaaa->last_queried();
```

ms_ad_user_data( )

Use this method to retrieve Microsoft Active Directory users related information. This is a read-only attribute.

Parameter

None

Returns

The valid return value is an Infoblox::Grid::MSServer::AdUser::Data object.

Example

```
#Get ms_ad_user_data
my $ms_ad_user_data = $bindaaaa->ms_ad_user_data();
```

name( )

Use this method to set or retrieve the host name.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

The attribute value can be in unicode format.

Parameter

Hostname in FQDN (Fully Qualified Domain Name) format. The FQDN consists of the hostname followed by the domain name (example: abc.com). A hostname can have a maximum of 256 bytes.

Infoblox also supports wildcard AAAA records. A wildcard AAAA record maps all the hostnames in a domain to a single IP address.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get name
my $name = $bindaaaa->name();
#Modify name
$bindaaaa-> name("aaaa2.domain.com");
```

netbios( )

Use this method to retrieve the name in the NetBIOS reply that responded to a NetBIOS query.

Returns

The method returns the attribute value.

Example

```
#Get NetBIOS
my $netbios = $bindaaa->netbios();
```

network_component_description( )

Use this method to retrieve the description of the network device that is connected to the AAAA Record object.

Parameter

none

Returns

The method returns the network device description.

Example

```
#Get network_component_description
my $network_component_description = $bindaaa->network_component_description();
```

network_component_ip( )

Use this method to retrieve the IP address of the network device that is connected to the AAAA Record object.

Parameter

none

Returns

The method returns the network device IP address.

Example

```
#Get network_component_ip
```

```
my $network_component_ip = $bindaaa->network_component_ip();
```

network_component_name( )

Use this method to retrieve the name of the network device that is connected to the AAAA Record object.

Parameter

none

Returns

The method returns the network device name.

Example

```
#Get network_component_name
my $network_component_name = $bindaaa->network_component_name();
```

network_component_port_description( )

Use this method to retrieve the description of the network device port that is connected to the AAAA Record object.

Parameter

none

Returns

The method returns the network device port description.

Example

```
#Get network_component_port_description
my $network_component_port_description = $bindaaa->network_component_port_description();
```

network_component_port_name( )

Use this method to retrieve the name of the network device port that is connected to the AAAA Record object.

Parameter

none

Returns

The method returns the name of the port on the network device.

Example

```
#Get network_component_port_name
my $network_component_port_name = $bindaaa->network_component_port_name();
```

network_component_port_number( )

Use this method to retrieve the number of the network device port that is connected to the AAAA Record object.

Parameter

none

Returns

The method returns the network device port number.

Example

```
#Get network_component_port_number
my $network_component_port_number = $bindaaa->network_component_port_number();
```

network_component_type( )

Use this method to retrieve the type of the network device that is connected to the AAAA Record object.

Parameter

none

Returns

The method returns the network device type.

Example

```
#Get network_component_type
my $network_component_type = $bindaaa->network_component_type();
```

os( )

Use this method to retrieve the operating system associated with this A record.

Returns

The method returns the attribute value.

Example

```
#Get OS
my $os = $bindaaa->os();
```

port_duplex( )

Use this method to retrieve the duplex setting of the network device port that is connected to the AAAA Record object.

Parameter

none

Returns

The method returns the network device port duplex setting. The returned value, if any, is one of the following:

```
Full
Half
```

Example

```
#Get port_duplex
my $port_duplex = $bindaaa->port_duplex();
```

port_link_status( )

Use this method to retrieve the link status of the network device port that is connected to the AAAA Record object.

Parameter

none

Returns

The method returns the port link status. The returned value, if any, is one of the following:

```
Connected
Not Connected
Unknown
```

Example

```
#Get port_link_status
my $port_link_status = $bindaaa->port_link_status();
```

port_speed( )

Use this method to retrieve the speed of the network device port that is connected to the AAAA Record object.

Parameter

none

Returns

The method returns the network device port speed value. The returned value, if any, is one of the following:

```
10M
100M
1G
10G
100G
```

```
Unknown
```

Example

```
#Get port_speed
my $port_speed = $bindaaa->port_speed();
```

port_status( )

Use this method to retrieve the status of the network device port that is connected to the AAAA Record object.

Parameter

none

Returns

The method returns the network device port status. The returned value, if any, is one of the following:

```
Up
Down
Unknown
```

Example

```
#Get port_status
my $port_status = $bindaaa->port_status();
```

port_vlan_description( )

Use this method to retrieve the VLAN description of the network device port that is connected to the AAAA Record object.

Parameter

none

Returns

The method returns the VLAN description of the network device port.

Example

```
#Get port_vlan_description
my $port_vlan_description = $bindaaa->port_vlan_description();
```

port_vlan_name( )

Use this method to retrieve the VLAN name of the network device port that is connected to the AAAA Record object.

Parameter

none

Returns

The method returns the VLAN name of the network device port.

Example

```
#Get port_vlan_name
my $port_vlan_name = $bindaaa->port_vlan_name();
```

port_vlan_number ( )

Use this method to retrieve the VLAN number of the network device port that is connected to the AAAA Record object.

Parameter

none

Returns

The method returns the network device port VLAN number.

Example

```
#Get port_vlan_number
my $port_vlan_number  = $bindaaa->port_vlan_number ();
```

reclaimable( )

Use this method to retrieve the flag that indicates whether the record is reclaimable or not.

Parameter

None

Returns

The method returns the attribute value.

Example

```
#Get reclaimable
my $reclaimable = $object->reclaimable();
```

ttl( )

Use this method to set or retrieve the Time to Live (TTL) value.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

The default value is undefined which indicates that the record inherits the TTL value of the zone.

Specify a TTL value to override the TTL value at the zone level.

Parameter

A 32-bit integer (range from 0 to 4294967295) that represents the duration in seconds that the record is cached. Zero indicates that the record should not be cached.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get ttl
my $ttl = $bindaaaa->ttl();
#Modify ttl
$bindaaaa->ttl(1800);
#Un-override ttl
$bindaaaa->ttl(undef);
```

views( )

Use this method to set or retrieve the view of the AAAA record.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

The default value is the "default" view, which means the AAAA record is located under the default view.

Parameter

Array reference of defined Infoblox::DNS::View objects.

Note that the array size must be 1.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get views
my $ref_views = $bindaaaa->views();
#Modify views, list of Infoblox::DNS::View objects
$bindaaaa->views([$view1]);
```

zone( )

Use this method to retrieve the zone name of a DNS AAAA record. This method is read-only and cannot be set.

Parameter

None

Returns

Returns the attribute value.

Example

```
# Get zone
my $zone = $bindaaaa->zone();
```

SAMPLE CODE

The following sample code demonstrates the different functions that can be applied to a DNAME record object, such as add, search, modify, and remove. This sample also includes error handling for the operations.

#Preparation prior to a DNS AAAA record insertion

```
#PROGRAM STARTS: Include all the modules that will be used

use strict;
use Infoblox;

#Create a session to the Infoblox Appliance
my $session = Infoblox::Session->new(
    master   => "192.168.1.2", #appliance host ip
    username => "admin",     #appliance user login
    password => "infoblox"   #appliance password
);

unless ($session) {
   die("Construct session failed: ",
       Infoblox::status_code() . ":" . Infoblox::status_detail());
}

#Create the zone prior to an AAAA record insertion
my $zone = Infoblox::DNS::Zone->new(name => "domain.com");

unless ($zone) {
   die("Construct zone failed: ",
       Infoblox::status_code() . ":" . Infoblox::status_detail());
}

print "Zone object created successfully\n";

#Verify if the zone exists
my $object = $session->get(object => "Infoblox::DNS::Zone",
                           name => "domain.com");
unless ($object) {
   print "Zone does not exist on server, safe to add the zone\n";
   $session->add($zone)
      or die("Add zone failed: ",
             $session->status_code() . ":" . $session->status_detail());
}
print "Zone added successfully\n";
```

#Create a DNS AAAA record

```
#Construct a DNS AAAA object

my $bindaaaa = Infoblox::DNS::Record::AAAA->new(
    name     => "aaaa1.domain.com",
    comment  => "add a DNS AAAA aaaa1.domain.com",
    ipv6addr => "aaaa:1111:bbbb:2222:cccc:3333:dddd:4444"
);

unless ($bindaaaa) {
   die("Construct DNS record AAAA failed: ",
       Infoblox::status_code() . ":" . Infoblox::status_detail());
}
print "DNS AAAA object created successfully\n";

#Add the DNS AAAA record object to Infoblox Appliance through a session
$session->add($bindaaaa)

    or die("Add record AAAA failed: ",
```

```
            $session->status_code() . ":" . $session->status_detail());
     print "DNS AAAA object added to server successfully\n";
```

#Search for a specific DNS AAAA record

```
#Search all AAAA records that match "domain.com"
my @retrieved_objs = $session->search(
    object => "Infoblox::DNS::Record::AAAA",
    name   => 'domain\.com'
);
my $object = $retrieved_objs[0];
unless ($object) {
    die("Search record AAAA failed: ",
        $session->status_code() . ":" . $session->status_detail());
}
print "Search DNS AAAA object found at least 1 matching entry\n";

#Search all AAAA records that start with "add" in the comment
my @retrieved_objs = $session->search(
    object    => "Infoblox::DNS::Record::AAAA",
    comment   => "add.*",
);
my $object = $retrieved_objs[0];
unless ($object) {
    die("Search record AAAA failed: ",
        $session->status_code() . ":" . $session->status_detail());
}
print "Search DNS AAAA object found at least 1 matching entry\n";

#Search all AAAA records that start with "aaaa1" and end with ".com"
my @retrieved_objs = $session->search(
    object => "Infoblox::DNS::Record::AAAA",
    name   => 'aaaa1.*\.com',
 );
my $object = $retrieved_objs[0];
unless ($object) {
    die("Search record AAAA failed: ",
        $session->status_code() . ":" . $session->status_detail());
}
print "Search DNS AAAA object using regexp found at least 1 matching entry\n";
```

#Get and modify a DNS AAAA record

```
#Get AAAA record through the session
my @retrieved_objs = $session->get(
    object => "Infoblox::DNS::Record::AAAA",
    name   => "aaaa1.domain.com"
);
my $object = $retrieved_objs[0];

unless ($object) {
    die("Get record AAAA failed: ",
        $session->status_code() . ":" . $session->status_detail());
}

#Modify one of the attributes of the specified AAAA record
$object->ipv6addr("aaaa:1111:bbbb:2222:cccc:3333:dddd:eeee");

#Applying the changes
$session->modify($object)

    or die("Modify record AAAA failed: ",
           $session->status_code() . ":" . $session->status_detail());
     print "DNS AAAA object modified successfully \n";
```

#Remove a DNS AAAA record.

```
#Get AAAA record through the session
my @retrieved_objs = $session->get(
    object   => "Infoblox::DNS::Record::AAAA",
    name     => "aaaa1.domain.com"
);
my $object = $retrieved_objs[0];
unless ($object) {
    die("Get record AAAA failed: ",
        $session->status_code() . ":" . $session->status_detail());
}
```

```
print "Get DNS AAAA object found at least 1 matching entry\n";

#Submit the object for removal

$session->remove($object)
    or die("Remove record AAAA failed: ",
           $session->status_code() . ":" . $session->status_detail());
print "DNS AAAA object removed successfully \n";

####PROGRAM ENDS####
```

AUTHOR

Infoblox Inc. http://www.infoblox.com/

SEE ALSO

Infoblox::DNS::View, Infoblox::DNS::Zone, Infoblox::Session, Infoblox::Session->get(), Infoblox::Session->search(), Infoblox::Session->add(), Infoblox::Session->remove(), Infoblox::Session->modify(), Infoblox::Grid::MSServer::AdUser::Data

---

COPYRIGHT

Copyright (c) 2017 Infoblox Inc.

Infoblox::DNS::Record::PTR - DNS PTR record object.

- NAME
- DESCRIPTION
- CONSTRUCTOR
- SESSION METHODS
  - Infoblox::Session->add( )
  - Infoblox::Session->get( )
  - Infoblox::Session->modify( )
  - Infoblox::Session->remove( )
  - Infoblox::Session->search( )
- METHODS
  - comment( )
  - cloud_info( )
  - creator( )
  - creation_time( )
  - ddns_principal( )
  - ddns_protected( )
  - disable( )
  - discovered_data( )
  - discovered_duid( )
  - discovered_name( )
  - discoverer( )
  - dns_name( )
  - dns_ptrdname( )
  - first_discovered( )
  - extattrs( )
  - extensible_attributes( )
  - forbid_reclamation( )
  - ipv4addr( )
  - ipv6addr( )
  - last_discovered( )
  - last_queried( )
  - mac_address( )
  - ms_ad_user_data( )
  - name( )
  - netbios( )
  - network_component_description( )
  - network_component_ip( )
  - network_component_name( )
  - network_component_port_description( )
  - network_component_port_name( )
  - network_component_port_number( )
  - network_component_type( )
  - os( )
  - port_duplex( )
  - port_link_status( )
  - port_speed( )
  - port_status( )
  - port_vlan_description( )
  - port_vlan_name( )
  - port_vlan_number( )
  - ptrdname( )
  - reclaimable( )
  - ttl( )
  - views( )
  - v_cluster( )
  - v_datacenter( )
  - v_host( )
  - v_name( )
  - v_netadapter( )
  - v_switch( )
  - v_type( )
  - zone( )
- SAMPLE CODE
- AUTHOR
- SEE ALSO
- COPYRIGHT

NAME

Infoblox::DNS::Record::PTR - DNS PTR record object.

DESCRIPTION

In a forward-mapping zone, a PTR (pointer) record maps a domain name to another domain name. In a reverse-mapping zone, a PTR (pointer) record maps an address to a domain name. To define a specific address-to-name mapping, add a PTR record to a previously defined authoritative reverse-mapping zone.

To create a PTR record, you need to specify a domain name and host name.

Note that you must specify only one view for the attribute "views".

CONSTRUCTOR

```
 #Constructor for an IPv4 PTR object
 my $bind_ptr = Infoblox::DNS::Record::PTR->new (
        ipv4addr              => $ipv4addr,                                                  #
Required
        ptrdname              => $fqdn,                                                      #
Required
        comment               => $string,                                                    #
Optional / Default is empty
        disable               => "true" | "false"                                            #
Optional / Default "false"
        extattrs              => { $string => $extattr, ... },                               #
Optional / Default is undefined
        extensible_attributes => { $string => $string|$num, $string => [ $string|$num, ...], ... }, #
Optional / Default is undefined
        last_discovered       => $string,                                                    #
Optional / Default is empty
        name                  => $fqdn,                                                      #
Optional
        ttl                   => $num | undef,                                               #
Optional / Default is undefined
        views                 => [$View1],                                                   #
Optional / Default is "default" view
        creator               => "DYNAMIC" | "STATIC",                                       #
Optional / Default is "STATIC"
        ddns_protected        => "true" | "false",                                           #
Optional / Default is "false"
        ddns_principal        => $string,                                                    #
Optional / Default is undefined
        forbid_reclamation    => "true" | "false",                                           #
Optional / Default is "false"
     );

 #Constructor for an IPv6 PTR object
 my $bind_ptr = Infoblox::DNS::Record::PTR->new (
        ipv6addr              => $ipv6addr,                                                  #
Required
        ptrdname              => $fqdn,                                                      #
Required
        comment               => $string,                                                    #
Optional / Default is empty
        disable               => "true" | "false"                                            #
Optional / Default "false"
        extattrs              => { $string => $extattr, ... },                               #
Optional / Default is undefined
        extensible_attributes => { $string => $string|$num, $string => [ $string|$num, ...], ... }, #
Optional / Default is undefined
        name                  => $fqdn,                                                      #
Optional
        ttl                   => $nun | undef,                                               #
Optional / Default is undefined
        views                 => [$View1],                                                   #
Optional / Default is "default" view
        creator               => "DYNAMIC" | "STATIC",                                       #
Optional / Default is "STATIC"
        ddns_protected        => "true" | "false",                                           #
Optional / Default is "false"
        ddns_principal        => $string,                                                    #
Optional / Default is undefined
        forbid_reclamation    => "true" | "false",                                           #
Optional / Default is "false"
     );

 #Constructor for an PTR object in Forward Mapping Zone
 #Both ipv4addr and ipv6addr must be undefined
 my $bind_ptr = Infoblox::DNS::Record::PTR->new (
        name                  => $fqdn,                                                      #
Required
                                 $fqdn                                                       #
```

```
        ptrdname              =>      ,
Required
        comment               => $string,                                          #
Optional / Default is empty
        disable               => "true" | "false"                                  #
Optional / Default "false"
        extattrs              => { $string => $extattr, ... },                     #
Optional / Default is undefined
        extensible_attributes => { $string => $string|$num, $string => [ $string|$num, ...], ... }, #
Optional / Default is undefined
        ttl                   => $nun | undef,                                     #
Optional / Default is undefined
        views                 => [$View1],                                         #
Optional / Default is "default" view
        creator               => "DYNAMIC" | "STATIC",                             #
Optional / Default is "STATIC"
        ddns_protected        => "true" | "false",                                 #
Optional / Default is "false"
        ddns_principal        => $string,                                          #
Optional / Default is undefined
        forbid_reclamation    => "true" | "false",                                 #
Optional / Default is "false"
    );
```

You cannot set both extattrs and extensible_attributes attributes at the same time.

SESSION METHODS

This section describes all the methods in Infoblox::Session that you can apply to a DNS PTR record object.

Infoblox::Session->add( )

Use this method to add an object to the Infoblox appliance. See Infoblox::Session->add() for parameters and return values.

Example

```
#Constructor for an IPv4 PTR object
my $bind_ptr = Infoblox::DNS::Record::PTR->new (
    ipv4addr => "10.0.0.1",
    ptrdname => "bind_ptr.domain.com",
    comment => "this is a demo bind_ptr record",
    disable => "true",
    ttl => "33",
    views => [$view1]  );
 # Submit for addition
 my $response = $session->add( $bind_ptr );

#Constructor for an IPv6 PTR object
my $bind_ptr = Infoblox::DNS::Record::PTR->new(
    ptrdname     => "ip6host.this.com",
    comment  => " this is a demo ipv6 bind_ptr record ",
    disable => "true",
    ttl => "333",
    ipv6addr => "1234:5678:9abc:1000::1");
 # Submit for addition
 my $response = $session->add( $bind_ptr );
```

Infoblox::Session->get( )

Use this method to retrieve existing objects from the Infoblox appliance. See Infoblox::Session->get() for parameters and return values.

Key Reference

```
 Apply the following attributes to get a specific DNS PTR object:

 name | ipv4addr | ipv6addr | ptrdname - Optional. name/ptrdname is in FQDN format. ipv4addr is an
IPv4 address with a 32-bit number in dotted decimal notation. ipv6addr is the IPv6 (128 bits) address
of the DNS PTR record. ptrdname is the domain name of the DNS PTR record.
 view                                  - Optional. The DNS view in which the PTR record is located.
By default, all DNS views are searched. But if you omit this attribute and specify a zone, the
appliance searches the 'default' view only.
 zone                                  - Optional. A zone name in FQDN format.
 creator                               - Optional. Record creator.
 ddns_principal                        - Optional. GSS-TSIG principal that owns the record.
 reclaimable                           - Optional. The flag that indicates whether the record is
reclaimable or not.
 extattrs                              - Optional. A hash reference containing extensible attributes.
 extensible_attributes                 - Optional. A hash reference containing extensible attributes.
 discovered_duid                       - Optional. Discovered DHCPv6 UID.
 discovered_name                       - Optional. Discovered name.
 discoverer                            - Optional. Discoverer name.
```

```
        first_discovered                        - Optional. Time of the first discovery.
        last_discovered                         - Optional. Time of the last discovery.
        netbios                                 - Optional. Discovered NetBIOS name.
        network_component_description           - Optional. Discovered network component description.
        network_component_ip                    - Optional. Discovered network component IP address.
        network_component_name                  - Optional. Discovered network component name.
        network_component_port_description      - Optional. Discovered network component port description.
        network_component_port_name             - Optional. Discovered network component port name.
        network_component_port_number           - Optional. Discovered network component port number.
        network_component_type                  - Optional. Discovered network component type.
        os                                      - Optional. Discovered OS.
        port_duplex                             - Optional. Discovered port duplex settings.
        port_link_status                        - Optional. Discovered port link status.
        port_speed                              - Optional. Discovered port speed settings.
        port_status                             - Optional. Discovered port status settings.
        port_vlan_description                   - Optional. Discovered port VLAN description.
        port_vlan_name                          - Optional. Discovered port VLAN name.
        port_vlan_number                        - Optional. Discovered port VLAN number.
        v_cluster                               - Optional. Discovered VMware cluster name.
        v_datacenter                            - Optional. Discovered VMware datacenter name.
        v_host                                  - Optional. Discovered VMware server name.
        v_name                                  - Optional. Discovered VMware host or virtual machine name.
        v_netadapter                            - Optional. Discovered VMware physical network adapter name.
        v_switch                                - Optional. Discovered VMware virtual switch name.
        v_type                                  - Optional. Discovered VMware entity type (possible values:
      "VirtualMachine", "HostSystem", "VirtualCenter").
```

Example

```
#To get an IPv4 PTR object by name
my @retrieved_objs = $session->get(
    object => "Infoblox::DNS::Record::PTR",
    name   => "1.0.0.10.in-addr.arpa",
    view   => "default" );

#To get an IPv4 PTR object by ipv4addr
my @retrieved_objs = $session->get(
    object   => "Infoblox::DNS::Record::PTR",
    ipv4addr => "10.0.0.1",
    view     => "default" );

#To get an IPv4 PTR object by ptrdname
my @retrieved_objs = $session->get(
    object   => "Infoblox::DNS::Record::PTR",
    ptrdname => "bind_ptr.domain.com",
    view     => "default" );

#To get an IPv6 PTR object by name
my @retrieved_objs = $session->get(
    object => "Infoblox::DNS::Record::PTR",
    name   => "1.0.0.0.0.0.0.0.0.0.0.0.0.0.0.0.0.0.0.0.0.0.0.0.0.0.1.c.b.a.9.8.7.6.5.4.3.2.1.ip6.arpa",
    view   => "default" );

#To get an IPv6 PTR object by ipv6addr
my @retrieved_objs = $session->get(
    object   => "Infoblox::DNS::Record::PTR",
    ipv6addr => "1234:5678:9abc:1000::1",
    view     => "default" );

#To get an IPv6 PTR object by ptrdname
my @retrieved_objs = $session->get(
    object   => "Infoblox::DNS::Record::PTR",
    ptrdname => "ip6host.this.com",
    view     => "default" );

# get all PTR object in zone "10.10.10.in-addr.arpa" of default view
my @retrieved_objs = $session->get(
    object => "Infoblox::DNS::Record::PTR",
    zone   => "10.10.10.in-addr.arpa",
    view   => "default"
    );

# get all PTR objects with a given extensible attribute
my @retrieved_objs = $session->get(
    object => "Infoblox::DNS::Record::PTR",
    extensible_attributes => { 'Site' => 'Santa Clara' });

# get all PTR objects with a given v_type:
my @retrieved_objs = $session->get(
  object    => "Infoblox::DNS::Record::PTR",
  v_type    => "VirtualMachine");
```

Infoblox::Session->modify( )

Use this method to modify objects in the Infoblox appliance. See Infoblox::Session->modify() for parameters and return values.

Example

```
# Use method to modify the comment for IPv4 or IPv6 object.
$bind_ptr->comment("this is a modified comment for bind PTR");
# Submit modification
my $response = $session->modify( $bind_ptr );
```

Infoblox::Session->remove( )

Use this function to submit to remove an object from the Infoblox appliance. See Infoblox::Session->remove() for parameters and return values.

To remove a specific object, first use `get()` or `search()` to retrieve the specific object, and then submit this object for removal.

Example

#Remove an object with a unique name

```
# Get the object with the same name for IPv4 object
my @retrieved_objs = $session->get(
    object => "Infoblox::DNS::Record::PTR",
    name   => "1.0.0.10.in-addr.arpa",
    view   => "default" );
# find the desired object from retrieved list.
my $desired_ptr = $retrieved_objs[0];
# Submit for removal
my $response = $session->remove( $desired_ptr );

# Get the object with the same name for an IPv6 object
my @retrieved_objs = $session->get(
    object => "Infoblox::DNS::Record::PTR",
    name   => "1.0.0.0.0.0.0.0.0.0.0.0.0.0.0.0.0.0.0.0.0.0.0.0.0.0.1.c.b.a.9.8.7.6.5.4.3.2.1.ip6.arpa",
    view   => "default" );
# find the desired object from retrieved list.
my $desired_ptr = $retrieved_objs[0];
# Submit for removal
my $response = $session->remove( $desired_ptr );
```

Infoblox::Session->search( )

Use this method to search for existing DNS PTR record object in the Infoblox appliance. See Infoblox::Session->search() for parameters and return values.

Key Reference

```
 Apply the following attributes to search for a DNS PTR record object:

 name | ipv4addr | ipv6addr | ptrdname - Optional. The IPv4 or IPv6 address of the PTR record, or the
domain name in FQDN (Fully Qualified Domain Name) format (regular expression).
 view                                - Optional. The DNS view in which the PTR record is located.
By default, all DNS views are searched. But if you omit this attribute and specify a zone, the
appliance searches the 'default' view only.
 comment                             - Optional. A comment in string format (regular expression).
 creator                             - Optional. Record creator.
 ddns_principal                      - Optional. GSS-TSIG principal that owns the record.
 reclaimable                         - Optional. The flag that indicates whether the record is
reclaimable or not.
 extattrs                            - Optional. A hash reference containing extensible attributes.
 extensible_attributes               - Optional. A hash reference containing extensible attributes.
 discovered_duid                     - Optional. Discovered DHCPv6 UID (exact search only).
 discovered_name                     - Optional. Discovered name(regex).
 discoverer                          - Optional. Discoverer name(regex).
 first_discovered                    - Optional. Time of the first discovery (in the ISO format,
exact search only).
 last_discovered                     - Optional. Time of the last discovery (in the ISO format,
exact search only).
 netbios                             - Optional. Discovered NetBIOS name(regex).
 network_component_description       - Optional. Discovered network component description(regex).
 network_component_ip                - Optional. Discovered network component IP address(regex).
 network_component_name              - Optional. Discovered network component name(regex).
 network_component_port_description  - Optional. Discovered network component port
description(regex).
 network_component_port_name         - Optional. Discovered network component port name(regex).
 network_component_port_number       - Optional. Discovered network component port number (exact
search only).
 network_component_type              - Optional. Discovered network component type(regex).
 os                                  - Optional. Discovered OS (regex).
 port_duplex                         - Optional. Discovered port duplex settings (exact search
only).
 port_link_status                    - Optional. Discovered port link status (exact search only).
 port_speed                          - Optional. Discovered port speed settings (exact search
only).
 port_status                         - Optional. Discovered port status settings (exact search
```

```
only).
 port_vlan_description                  - Optional. Discovered port VLAN description(regex).
 port_vlan_name                         - Optional. Discovered port VLAN name(regex).
 port_vlan_number                       - Optional. Discovered port VLAN number(exact search only).
 v_cluster                              - Optional. Discovered VMware cluster name (regular
expression).
 v_datacenter                           - Optional. Discovered VMware datacenter name (regular
expression).
 v_host                                 - Optional. Discovered VMware server name (regular
expression).
 v_name                                 - Optional. Discovered VMware host or virtual machine name
(regular expression).
 v_netadapter                           - Optional. Discovered VMware physical network adapter name
(regular expression).
 v_switch                               - Optional. Discovered VMware virtual switch name (regular
expression).
 v_type                                 - Optional. Discovered VMware entity type (possible values:
"VirtualMachine", "HostSystem", "VirtualCenter").
```

For more information about searching extensible attributes, see Infoblox::Grid::ExtensibleAttributeDef/Searching Extensible Attributes.

Example

```
 # search for all DNS IPv4 PTR objects in the default DNS view that match 10.0.0.0/8 and that contain
"bind" in the comment field
 my @retrieved_objs = $session->search(
     object  => "Infoblox::DNS::Record::PTR",
     name    => "10.in-addr.arpa",
     comment => "bind*",
     view    => "default" );

 my @retrieved_objs = $session->search(
     object   => "Infoblox::DNS::Record::PTR",
     ipv4addr => '10\..*\..*\..*',
     comment  => "bind*",
     view     => "default" );

 my @retrieved_objs = $session->search(
     object   => "Infoblox::DNS::Record::PTR",
     ptrdname => "bind_ptr.domain.com",
     comment  => "bind*",
     view     => "default" );

 # search for all DNS IPv6 PTR objects under 1234:5678:9abc::/48 which have bind in comment field in
default view
 my @retrieved_objs = $session->search(
     object  => "Infoblox::DNS::Record::PTR",
     name    => "1.0.0.0.0.0.0.0.0.0.0.0.0.0.0.0.0.0.0.0.0.0.0.0.1.c.b.a.9.8.7.6.5.4.3.2.1.ip6.arpa",
     comment => "bind*",
     view    => "default" );

 my @retrieved_objs = $session->search(
     object   => "Infoblox::DNS::Record::PTR",
     ipv6addr => "1234:5678:9abc:1000::1",
     comment  => "bind*",
     view     => "default" );

 my @retrieved_objs = $session->search(
     object   => "Infoblox::DNS::Record::PTR",
     ptrdname => "ip6host.this.com",
     comment  => "bind*",
     view     => "default" );

 # search for all PTR objects in the "10.10.10.in-addr.arpa" zone of the default view
 my @retrieved_objs = $session->search(
     object => "Infoblox::DNS::Record::PTR",
     zone   => "10.10.10.in-addr.arpa",
     view   => "default"
     );

 # search all PTR records with the extensible attribute 'Site'
 my @retrieved_objs = $session->search(
    object => "Infoblox::DNS::Record::MX",
    extensible_attributes => { 'Site' => 'Santa Clara' });

 # search all PTR records by v_datacenter:
 my @retrieved_objs = $session->search(
    object       => "Infoblox::DNS::Record::PTR",
    v_datacenter => "test.*");
```

METHODS

This section describes all the methods that you can use to configure and retrieve the attribute value of a PTR record.

comment( )

Use this method to set or retrieve the comment.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

Desired comment in string format with a maximum of 256 bytes.

Returns

If you specified a parameter,the method returns true when the modification succeeds,and returns false when the operation fails.

Example

```
#Get comment
my $comment = $bindptr->comment();
#Modify comment
$bindptr->comment("Modifying the DNS PTR comment");
```

cloud_info( )

Use this method to retrieve cloud API related information for the Infoblox::DNS::Record::PTR object.

Parameter

None

Returns

The method returns the attribute value.

Example

```
# Get cloud_info
my $cloud_info = $object->cloud_info();
```

creator( )

Use this method to set or retrieve the record creator.

Note that changing creator from or to 'SYSTEM' value is not allowed.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Paramter

The valid values are 'STATIC' and 'DYNAMIC'. The default value is 'STATIC'.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get creator value
my $creator = $object->creator();

#Modify creator value
$object->creator("DYNAMIC");
```

creation_time( )

Use this method to retrieve the creation time for the record. This is a read-only attribute.

Parameter

None

Returns

The valid return value is a number of seconds that have elapsed since January 1st, 1970 UTC.

Example

```
#Get creation_time value
my $creation_time = $object->creation_time();
```

ddns_principal( )

Use this method to set or retrive the GSS-TSIG principal that owns this record.

Note that you cannot set ddns_principal for 'STATIC' and 'SYSTEM' records.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Paramter

The GSS-TSIG principal FQDN (Fully Qualified Domain Name) format. The FQDN consists of the hostname followed by the domain name (example: abc.com). A hostname can have a maximum of 256 characters.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get ddns_principal value
my $ddns_principal = $object->ddns_principal();

#Modify ddns_principal value
$object->ddns_principal('foo.com');
```

ddns_protected( )

Use this method to set or retrieve the flag that indicates whether DDNS updates for this record are allowed or not.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

Specify 'true' to protect record from DDNS updates and 'false' to allow DDNS updates for the specified record.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get ddns_protected value
my $ddns_protected = $object->ddns_protected();

#Modify ddns_protected value
$object->ddns_protected('true');
```

disable( )

Use this method to set or retrieve the disable flag of a DNS record.

Include the specific parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

The default value for this field is false, therefore the DNS record is enabled.

Parameter

Specify "true" to set the disable flag or "false" to deactivate/unset it.

Returns

If you specified a parameter,the method returns true when the modification succeeds,and returns false when the operation fails.

Example

```
#Get disable
my $disable = $bindptr->disable();
#Modify disable
```

```
$bindptr->disable("true");
```

discovered_data( )

Use this method to retrieve the discovered data of a PTR Record object. This is a read-only attribute.

Omit the parameter to retrieve the attribute value.

Parameter

None

Returns

The valid return value is an Infoblox::Grid::Discovery::Data object.

Example

```
#Get discovered_data value
my $discovered_data = $object->discovered_data();
```

discovered_duid( )

Use this method to retrieve the discovered DHCPv6 UID (DUID) of a PTR record object.

Parameter

none

Returns

The method returns the discovered name.

Example

```
#Get discovered_duid
my $discovered_duid = $bindptr->discovered_duid();
```

discovered_name( )

Use this method to retrieve the discovered name of a PTR record object.

Parameter

none

Returns

The method returns the discovered name.

Example

```
#Get discovered_name
my $discovered_name = $bindptr->discovered_name();
```

discoverer( )

Use this method to retrieve the discoverer of a PTR record object.

Parameter

none

Returns

The method returns the discoverer value.

Example

```
#Get discoverer
my $discoverer = $bindptr->discoverer();
```

dns_name( )

Use this method to retrieve the FQDN, in punycode format, of the PTR record. This is a read-only attribute.

Parameter

None

Returns

The method returns the attribute value.

Example

```
# Get attribute value
my $value = $bindptr->dns_name();
```

dns_ptrdname( )

Use this method to retrieve the PTR domain name in punycode format. This is a read-only attribute.

Parameter

None

Returns

The method returns the attribute value.

Example

```
# Get attribute value
my $value = $bindptr->ptrdname();
```

first_discovered( )

Use this method to retrieve the date and time that the PTR record object was first discovered.

Parameter

none

Returns

The method returns the date and time that the PTR record object was first discovered.

Example

```
#Get first_discovered
my $first_discovered = $bindptr->first_discovered();
```

extattrs( )

Use this method to set or retrieve the extensible attributes associated with a PTR record object.

Parameter

Valid value is a hash reference containing the names of extensible attributes and their associated values (Infoblox::Grid::Extattr objects).

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get extattrs
my $ref_extattrs = $bindptr->extattrs();
#Modify extattrs
$bindptr->extattrs({ 'Site' => $extattr1, 'Administrator' => $extattr2 });
```

extensible_attributes( )

Use this method to set or retrieve the extensible attributes associated with a PTR record.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

For valid values for extensible attributes, see Infoblox::Grid::ExtensibleAttributeDef/Extensible Attribute Values.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get extensible attributes
my $ref_extensible_attributes = $bindptr->extensible_attributes();
#Modify extensible attributes
$bindptr->extensible_attributes({ 'Site' => 'Santa Clara', 'Administrator' => [ 'Peter', 'Tom' ] });
```

forbid_reclamation( )

Use this method to set or retrieve the flag that indicates whether the reclamation is allowed for the record or not.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

Specify 'true' to forbid reclamation for the record and 'false' to allow it. The default value is 'false'.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get forbid_reclamation
my $forbid_reclamation = $object->forbid_reclamation();
#Modify forbid_reclamation
$object->forbid_reclamation('true');
```

ipv4addr( )

Use this method to set or retrieve the IPv4 address.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

An IPv4 address is a 32-bit number in dotted decimal notation. It consists of four 8-bit groups of decimal digits separated by decimal points (example: 192.168.1.2).

Returns

If you specified a parameter,the method returns true when the modification succeeds,and returns false when the operation fails.

Example

```
#Get ipv4addr
my $ipv4addr = $bindptr->ipv4addr();
#Modify ipv4addr
$bind_ptr->ipv4addr("10.0.0.2");
```

ipv6addr( )

Use this method to set or retrieve the IPv6 address.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

The IPv6 (128 bits) of the DNS PTR record (example: aaaa:bbbb:cccc:dddd:eeee:ffff:0000:1111).

Returns

If you specified a parameter,the method returns true when the modification succeeds,and returns false when the operation fails.

Example

```
#Get ipv6addr
my $ipv6addr = $bindptr->ipv6addr();
#Modify ipv6addr
$bind_ptr->ipv6addr("1234:5678:9abc:1000::2");
```

last_discovered( )

Use this method to retrieve the time this object was last seen by a discovery job.

The default value for this field is empty.

Parameter

N/A

Returns

The method returns the attribute value.

Example

```
#Get last_discovered
my $last_discovered = $bind_ptr->last_discovered();
```

last_queried( )

Use this method to retrieve the time when the associated record was last queried. This is a read-only attribute.

Parameter

None

Returns

The method returns the attribute value. The number of seconds that have elapsed since January 1st, 1970 UTC.

Example

```
#Get last_queried
my $last_queried = $bind_ptr->last_queried();
```

mac_address( )

Use this method to retrieve the discovered MAC address of this object.

The default value for this field is empty.

Parameter

N/A

Returns

The method returns the attribute value.

Example

```
#Get mac_address
my $mac_address = $bind_ptr->mac_address();
```

ms_ad_user_data( )

Use this method to retrieve Microsoft Active Directory users related information. This is a read-only attribute.

Parameter

None

Returns

The valid return value is an Infoblox::Grid::MSServer::AdUser::Data object.

Example

```
#Get ms_ad_user_data
my $ms_ad_user_data = $bind_ptr->ms_ad_user_data();
```

name( )

Use this method to set or retrieve the FQDN of the PTR record.

Include the specified parameter to set the attribute value if both ipv4addr and ipv6addr are undefined: PTR in Forward Mapping Zone. Otherwise use ipv4addr or ipv6addr methods.

Omit the parameter to retrieve the attribute value.

The attribute value can be in unicode format.

Parameter

FQDN of the PTR record

Returns

The method returns the attribute value.

Example

```
#Get name
my $name = $bindptr->name();
#Modify name
$bindptr->name('bindptr.domain.com');
```

netbios( )

Parameter

None

Use this method to retrieve the name in the NetBIOS reply that responded to a NetBIOS query.

Returns

The method returns the attribute value.

Example

```
#Get netbios
my $netbios = $bind_ptr->netbios();
```

network_component_description( )

Use this method to retrieve the description of the network device that is connected to the PTR record object.

Parameter

none

Returns

The method returns the description of the network device.

Example

```
#Get network_component_description
my $network_component_description = $bindptr->network_component_description();
```

network_component_ip( )

Use this method to retrieve the IP address of the network device that is connected to the PTR record object.

Parameter

none

Returns

The method returns the network device IP address.

Example

```
#Get network_component_ip
my $network_component_ip = $bindptr->network_component_ip();
```

network_component_name( )

Use this method to retrieve the name of the network device that is connected to the PTR record object.

Parameter

none

Returns

The method returns the network device name.

Example

```
#Get network_component_name
my $network_component_name = $bindptr->network_component_name();
```

network_component_port_description( )

Use this method to retrieve the description of the network device port that is connected to the PTR record object.

Parameter

none

Returns

The method returns the description of the network device port.

Example

```
#Get network_component_port_description
my $network_component_port_description = $bindptr->network_component_port_description();
```

network_component_port_name( )

Use this method to retrieve the name of the network device port that is connected to the PTR record object.

Parameter

none

Returns

The method returns the network device port name.

Example

```
#Get network_component_port_name
my $network_component_port_name = $bindptr->network_component_port_name();
```

network_component_port_number( )

Use this method to retrieve the number of the network device port that is connected to the PTR record object.

Parameter

none

Returns

The method returns the network device port number value.

Example

```
#Get network_component_port_number
my $network_component_port_number = $bindptr->network_component_port_number();
```

network_component_type( )

Use this method to retrieve the type of the network device that is connected to the PTR record object.

Parameter

none

Returns

The method returns the network device type.

Example

```
#Get network_component_type
my $network_component_type = $bindptr->network_component_type();
```

os( )

Use this method to retrieve the operating system associated with this PTR record.

Returns

The method returns the attribute value.

Example

```
#Get os
my $os = $bind_ptr->os();
#Modify os
$bind_ptr->os("os-name");
```

port_duplex( )

Use this method to retrieve the duplex setting of the network device port that is connected to the PTR record object.

Parameter

none

Returns

The method returns the network device port duplex setting. Returned values, if any, are one of the following:

```
Full
Half
```

Example

```
#Get port_duplex
my $port_duplex = $bindptr->port_duplex();
```

port_link_status( )

Use this method to retrieve the link status of the network device port that is connected to the PTR record object.

Parameter

none

Returns

The method returns the network device port link status. Returned values, if any, are one of the following:

```
Connected
Not Connected
Unknown
```

Example

```
#Get port_link_status
my $port_link_status = $bindptr->port_link_status();
```

port_speed( )

Use this method to retrieve the speed of the network device port that is connected to the PTR record object.

Parameter

none

Returns

The method returns the network device port speed value. Returned values, if any, are one of the following:

```
10M
100M
1G
10G
100G
Unknown
```

Example

```
#Get port_speed
my $port_speed = $bindptr->port_speed();
```

port_status( )

Use this method to retrieve the status of the network device port that is connected to the PTR record object.

Parameter

none

Returns

The method returns the network device port status. Returned values, if any, are one of the following:

```
Up
Down
Unknown
```

Example

```
#Get port_status
my $port_status = $bindptr->port_status();
```

port_vlan_description( )

Use this method to retrieve the VLAN description of the network device port that is connected to the PTR record object.

Parameter

none

Returns

The method returns the VLAN description of the network device port that is connected to the PTR record object.

Example

```
#Get port_vlan_description
my $port_vlan_description = $bindptr->port_vlan_description();
```

port_vlan_name( )

Use this method to retrieve the VLAN name of the network device port that is connected to the PTR record object.

Parameter

none

Returns

The method returns the VLAN name of the network device port.

Example

```
#Get port_vlan_name
my $port_vlan_name = $bindptr->port_vlan_name();
```

port_vlan_number( )

Use this method to retrieve the VLAN number of the network device port that is connected to the PTR record object.

Parameter

none

Returns

The method returns the port VLAN number.

Example

```
#Get port_vlan_number
my $port_vlan_number = $bindptr->port_vlan_number();
```

ptrdname( )

Use this method to set or retrieve the PTR domain name.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

The attribute value can be in unicode format.

Parameter

Text with the domain name for the DNS PTR record.

Returns

If you specified a parameter,the method returns true when the modification succeeds,and returns false when the operation fails.

Example

```
#Get ptrdname
my $ptrdname = $bindptr->ptrdname();
#Modify ptrdname
$bindptr->ptrdname("new domain name");
```

reclaimable( )

Use this method to retrieve the flag that indicates whether the record is reclaimable or not.

Parameter

None

Returns

The method returns the attribute value.

Example

```
#Get reclaimable
my $reclaimable = $object->reclaimable();
```

ttl( )

Use this method to configure or retrieve the Time to Live (TTL) value.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

The default value is undefined; therefore the record inherits the TTL value from the zone level.

With a specified TTL value, the method overrides the zone values with the specified TTL value.

Parameter

A 32-bit integer (range from 0 to 4294967295) that represents the duration ,in seconds, that the record is cached. Zero indicates that the record should not be cached.

Returns

If you specified a parameter,the method returns true when the modification succeeds,and returns false when the operation fails.

Example

```
#Get ttl
my $ttl = $bindptr->ttl();
#Modify ttl
$bindptr->ttl(1800);
#Un-override ttl
$bindptr->ttl(undef);
```

views( )

Use this method to specify or retrieve the view in which the PTR record is located.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

The default value is the default view therefore the PTR record is referenced under the default view.

Parameter

Array reference of defined Infoblox::DNS::View objects.

Note that the array size must be 1.

Returns

If you specified a parameter,the method returns true when the modification succeeds,and returns false when the operation fails.

Example

```
#Get views
```

```
my $ref_views = $bindptr->views();
#Modify views, list of Infoblox::DNS::View objects
$bindptr->views([$view1]);
```

v_cluster( )

Use this method to retrieve the name of the VMware cluster associated with the PTR Record object. This method is read-only.

Parameter

none

Returns

This method returns a string that contains the VMware cluster name. The default value is an empty string.

Example

```
#Get v_cluster
my $v_cluster = $bindptr->v_cluster();
```

v_datacenter( )

Use this method to retrieve the name of the VMware datacenter associated with the PTR Record object. This method is read-only.

Parameter

none

Returns

This method returns a string that contains the VMware datacenter name. The default value is an empty string.

Example

```
#Get v_datacenter
my $v_datacenter = $bindptr->v_datacenter();
```

v_host( )

Use this method to retrieve the name of the VMware host associated with the PTR Record object. This method is read-only.

Parameter

none

Returns

This method returns a string that contains the VMware host name. The default value is an empty string.

Example

```
#Get v_host
my $v_host = $bindptr->v_host();
```

v_name( )

Use this method to retrieve the name of the VMware entity associated with the PTR Record object. This method is read-only.

Parameter

none

Returns

This method returns a string that contains the VMware entity name. The default value is an empty string.

Example

```
#Get v_name
my $v_name = $bindptr->v_name();
```

v_netadapter( )

Use this method to retrieve the name of the physical network adapter through which the VMware entity is connected to the appliance. This method is read-only.

Parameter

none

Returns

This method returns a string that contains the network adapter name. The default value is an empty string.

Example

```
#Get v_netadapter
my $v_netadapter = $bindptr->v_netadapter();
```

v_switch( )

Use this method to retrieve the name of the virtual switch through which the VMware entity is connected to the appliance. This method is read-only.

Parameter

none

Returns

This method returns a string that contains the virtual switch name. The default value is an empty string.

Example

```
#Get v_switch
my $v_switch = $bindptr->v_switch();
```

v_type( )

Use this method to retrieve the type of VMware entity associated with the PTR Record object. This method is read-only.

Parameter

none

Returns

This method returns a string that contains the VMware entity name. The default value is an empty string. The returend value, if any, can be one of the following:

```
VirtualMachine
HostSystem
VirtualCenter
```

Example

```
#Get v_type
my $v_type = $bindptr->v_type();
```

zone( )

Use this method to retrieve the zone name of a DNS PTR record. This method is read-only and cannot be set.

Parameter

None

Returns

Returns the attribute value.

Example

```
# Get zone
my $zone = $bindptr->zone();
```

SAMPLE CODE

The following sample code demonstrates different operations that can be applied to an object, such as create, search, modify, and remove an object. This sample code also includes error handling for the operations.

#Preparation prior to a DNS IPv4 PTR record insertion

```
#PROGRAM STARTS: Include all the modules that will be used
use strict;
```

```
use Infoblox;

#Create a session to the Infoblox appliance
my $session = Infoblox::Session->new(
    master   => "192.168.1.2",
    username => "admin",
    password => "infoblox"
);
unless ($session) {
   die("Construct session failed: ",
       $session->status_code() . ":" . $session->status_detail());
}
print "Session created successfully\n";

#Create the zone prior to a PTR record insertion
my $zone = Infoblox::DNS::Zone->new(name => "10.0.0.0/8");
unless ($zone) {
   die("Construct zone failed: ",
       Infoblox::status_code() . ":" . Infoblox::status_detail());
}
print "Zone object created successfully\n";

#Verify if the zone exists
my $object = $session->get(object => "Infoblox::DNS::Zone", name => "10.in-addr.arpa");
unless ($object) {
   print "Zone does not exist on server, safe to add the zone\n";
   $session->add($zone)
      or die("Add zone failed: ",
             $session->status_code() . ":" . $session->status_detail());
}
print "Zone added successfully\n";
```

#Create a DNS IPv4 PTR record

```
my $bind_ptr = Infoblox::DNS::Record::PTR->new(
   ptrdname     => "bind_ptr.domain.com",
   comment  => " this is a demo bind_ptr record ",
   ipv4addr => "10.0.0.1"
 );
 unless ($bind_ptr) {
    die("Construct DNS record PTR failed: ",
        Infoblox::status_code() . ":" . Infoblox::status_detail());
 }
print "DNS PTR object created successfully\n";

#Add the DNS PTR record object to the Infoblox appliance through a session
$session->add($bind_ptr)
    or die("Add record PTR failed: ",
           $session->status_code() . ":" . $session->status_detail());
print "DNS PTR object added to server successfully\n";
```

#Search for a specific DNS IPv4 PTR record

```
#Search for all PTR records that match "10.in-addr.arpa"
my @retrieved_objs = $session->search(
    "object" => "Infoblox::DNS::Record::PTR",
    "name" => '10\.in-addr.arpa',
);
my $object = $retrieved_objs[0];
unless ($object) {
    die("Search record PTR failed: ",
        $session->status_code() . ":" . $session->status_detail());
}
print "Search DNS PTR object found at least 1 matching entry\n";

#Search all PTR records that have "bind" in the comment
my @retrieved_objs = $session->search(
    object => "Infoblox::DNS::Record::PTR",
    name   => '10\.*\.in-addr.arpa',
    comment => "bind*"
);
my $object = $retrieved_objs[0];
unless ($object) {
    die("Search record PTR failed: ",
        $session->status_code() . ":" . $session->status_detail());
}
```

```
print "Search DNS PTR object using regexp found at least 1 matching entry\n";
```

#Get and modify a DNS IPv4 PTR record

```
#Get a PTR record through the session
my @retrieved_objs = $session->get(
    object => "Infoblox::DNS::Record::PTR",
    name   => "1.0.0.10.in-addr.arpa",
);
my $object = $retrieved_objs[0];
unless ($object) {
    die("Get record PTR failed: ",
        $session->status_code() . ":" . $session->status_detail());
}
print "Get DNS PTR object found at least 1 matching entry\n";
#Modify one of the attributes of the specified PTR record
$object->comment ("modified comment");

#Apply the changes
$session->modify($object)
    or die("Modify record PTR failed: ",
           $session->status_code() . ":" . $session->status_detail());
print "DNS PTR object modified successfully \n";
```

#Remove a DNS IPv4 PTR record

```
#Get a PTR record through the session
my @retrieved_objs = $session->get(
    object => "Infoblox::DNS::Record::PTR",
    name   => "1.0.0.10.in-addr.arpa",
);

 my $object = $retrieved_objs[0];
  unless ($object) {
    die("Get record PTR failed: ",
         $session->status_code() . ":" . $session->status_detail());
  }
  print "Get DNS PTR object found at least 1 matching entry\n";

#submit the object for removal
$session->remove($object)
    or die("Remove record PTR failed: ",
           $session->status_code() . ":" . $session->status_detail());
print "DNS PTR object removed successfully \n";
```

#Preparation prior to a DNS IPv6 PTR record insertion

```
#PROGRAM STARTS: Include all the modules that will be used
use strict;
use Infoblox;

#Create a session to the Infoblox appliance
my $session = Infoblox::Session->new(
    master   => "192.168.1.2",
    username => "admin",
    password => "infoblox"
);
unless ($session) {
   die("Construct session failed: ",
       $session->status_code() . ":" . $session->status_detail());
}
print "Session created successfully\n";

#Create the zone prior to a PTR record insertion
my $zone = Infoblox::DNS::Zone->new(name => "1234:5678:9abc::/48");
unless ($zone) {
   die("Construct zone failed: ",
       Infoblox::status_code() . ":" . Infoblox::status_detail());
}
print "Zone object created successfully\n";

#Verify if the zone exists
my $object = $session->get(object => "Infoblox::DNS::Zone", name => "c.b.a.9.8.7.6.5.4.3.2.1.ip6.arpa");
unless ($object) {
   print "Zone does not exist on server, safe to add the zone\n";
   $session->add($zone)
      or die("Add zone failed: ",
```

```
            $session->status_code() . ":" . $session->status_detail());
}
print "Zone added successfully\n";
```

#Create a DNS IPv6 PTR record

```
my $bind_ptr = Infoblox::DNS::Record::PTR->new(
   ptrdname     => "ip6host.this.com",
   comment  => " this is a demo bind_ptr record ",
   ipv6addr => "1234:5678:9abc:1000::1"
 );
 unless ($bind_ptr) {
    die("Construct DNS record PTR failed: ",
        Infoblox::status_code() . ":" . Infoblox::status_detail());
 }
print "DNS PTR object created successfully\n";
#Add the DNS PTR record object to the Infoblox  appliance through a session
       $session->add($bind_ptr)
           or die("Add record PTR failed: ",
                  $session->status_code() . ":" . $session->status_detail());
       print "DNS PTR object added to server successfully\n";
```

#Search for a specific DNS IPv6 PTR record

```
#Search for a PTR record in the zone
my @retrieved_objs = $session->search(
    "object" => "Infoblox::DNS::Record::PTR",
    "name" => "1.0.0.0.0.0.0.0.0.0.0.0.0.0.0.0.0.0.0.0.0.0.0.0.0.1.c.b.a.9.8.7.6.5.4.3.2.1.ip6.arpa",
);
my $object = $retrieved_objs[0];
unless ($object) {
    die("Search record PTR failed: ",
        $session->status_code() . ":" . $session->status_detail());
}
print "Search DNS PTR object found at least 1 matching entry\n";
```

#Get and modify a DNS IPv6 PTR record

```
#Get PTR record through the session
my @retrieved_objs = $session->get(
    object => "Infoblox::DNS::Record::PTR",
    name   => "1.0.0.0.0.0.0.0.0.0.0.0.0.0.0.0.0.0.0.0.0.0.0.0.0.1.c.b.a.9.8.7.6.5.4.3.2.1.ip6.arpa",
);
my $object = $retrieved_objs[0];
unless ($object) {
    die("Get record PTR failed: ",
        $session->status_code() . ":" . $session->status_detail());
}
print "Get DNS PTR object found at least 1 matching entry\n";

#Modify one of the attributes of the specified PTR record
$object->comment ("modified comment");

#Apply the changes
$session->modify($object)
    or die("Modify record PTR failed: ",
            $session->status_code() . ":" . $session->status_detail());
print "DNS PTR object modified successfully \n";
```

#Remove a DNS IPv6 PTR record

```
#Get PTR record through the session
my @retrieved_objs = $session->get(
    object => "Infoblox::DNS::Record::PTR",
    name   => "1.0.0.0.0.0.0.0.0.0.0.0.0.0.0.0.0.0.0.0.0.0.0.0.0.1.c.b.a.9.8.7.6.5.4.3.2.1.ip6.arpa",
);
my $object = $retrieved_objs[0];
unless ($object) {
    die("Get record PTR failed: ",
         $session->status_code() . ":" . $session->status_detail());
}
print "Get DNS PTR object found at least 1 matching entry\n";

#Submit the object for removal
$session->remove($object)
    or die("Remove record PTR failed: ",
           $session->status_code() . ":" . $session->status_detail());
```

```
print "DNS PTR object removed successfully \n";

####PROGRAM ENDS####
```

AUTHOR

Infoblox Inc. http://www.infoblox.com/

SEE ALSO

Infoblox::DNS::View, Infoblox::DNS::Zone, Infoblox::Session, Infoblox::Session->get(), Infoblox::Session->search(), Infoblox::Session->add(), Infoblox::Session->remove(), Infoblox::Session->modify(), Infoblox::Grid::MSServer::AdUser::Data, Infoblox::Grid::Discovery::Data

COPYRIGHT

Copyright (c) 2017 Infoblox Inc.

Infoblox::DNS::Record::TXT - DNS TXT record object.

- NAME
- DESCRIPTION
- CONSTRUCTOR
- SESSION METHODS
  - Infoblox::Session->add( )
  - Infoblox::Session->get( )
  - Infoblox::Session->modify( )
  - Infoblox::Session->remove( )
  - Infoblox::Session->search( )
- METHODS
  - comment( )
  - cloud_info( )
  - creator( )
  - creation_time( )
  - ddns_principal( )
  - ddns_protected( )
  - disable( )
  - dns_name( )
  - extattrs( )
  - extensible_attributes( )
  - forbid_reclamation( )
  - last_queried( )
  - name( )
  - reclaimable( )
  - text( )
  - ttl( )
  - views( )
  - zone( )
- SAMPLE CODE
- AUTHOR
- SEE ALSO
- COPYRIGHT

NAME

Infoblox::DNS::Record::TXT - DNS TXT record object.

DESCRIPTION

A TXT (text) record contains supplemental information for a host. SPF (Sender Policy Framework) records are specialized TXT records that identify the servers that send mail from a domain.

Note that you must specify only one view for the attribute "views".

CONSTRUCTOR

```
 my $txt = Infoblox::DNS::Record::TXT->new(
     name                  => $fqdn,                                                    # Required
     text                  => $string                                                   # Required
     comment               => $string,                                                  # Optional
/ Default is empty
     disable               => "true" | "false"                                          # Optional
/ Default "false"
     extattrs              => { $string => $extattr, ... },                             # Optional
/ Default is undefined
     extensible_attributes => { $string => $string|$num, $string => [ $string|$num, ...], ... }, # Optional
/ Default is undefined
     ttl                   => $num | undef,                                             # Optional
/ Default is undefined
     views                 => [$View1],                                                 # Optional
/ Default is "default" view
     creator               => "DYNAMIC" | "STATIC",                                     # Optional
/ Default is "STATIC"
     ddns_protected        => "true" | "false",                                         # Optional
/ Default is "false"
     ddns_principal        => $string,                                                  # Optional
/ Default is undefined
     forbid_reclamation    => "true" | "false",                                         # Optional
/ Default is "false"
 );
```

You cannot set both extattrs and extensible_attributes attributes at the same time.

SESSION METHODS

This section describes all the methods in Infoblox::Session module that you can apply to a TXT record object.

Infoblox::Session->add( )

Use this method to add an object to the Infoblox appliance. See Infoblox::Session->add() for parameters and return values.

Example

```
#Construct the TXT object
my $bind_txt = Infoblox::DNS::Record::TXT->new(
    name    => "txt.domain.com",
    text    => "this is text string",
    comment => "add TXT txt.domain.com",
    disable => "false",
    ttl     => "360",
    views   => [$view1]  );
# Submit for addition
my $response = $session->add( $bind_txt );
```

Infoblox::Session->get( )

Use this method to retrieve all the matching objects from the Infoblox appliance. See Infoblox::Session->get() for parameters and return values.

Key References

```
Apply the following attributes to get a specific DNS TXT object:

  name                  - Optional. Host name in string format.
  view                  - Optional. The DNS view in which the TXT record is located. By default, all
DNS views are searched. But if you omit this attribute and specify a zone, the appliance searches the
'default' view only.
  text                  - Optional. The text of the TXT record.
  creator               - Optional. Record creator.
  ddns_principal        - Optional. GSS-TSIG principal that owns the record.
  reclaimable           - Optional. The flag that indicates whether the record is reclaimable or not.
  extattrs              - Optional. A hash reference containing extensible attributes.
  extensible_attributes - Optional. A hash reference containing extensible attributes.
```

Example

```
my @retrieved_objs = $session->get(
    object => "Infoblox::DNS::Record::TXT",
    name   => "txt.domain.com",
    view   => "default" );

# get all DNS TXT records in the "domain.com" zone of the default view
my @retrieved_objs = $session->get(
    object => "Infoblox::DNS::Record::TXT",
    zone   => "domain.com",
    view   => "default" );

# get all DNS TXT records with a given extensible attribute
my @retrieved_objs = $session->get(
    object => "Infoblox::DNS::Record::TXT",
    extensible_attributes => { 'Site' => 'Santa Clara' });
```

Infoblox::Session->modify( )

Use this method to modify an object in the Infoblox appliance. See Infoblox::Session->modify() for parameters and return values.

Example

```
# Use method to modify the comment.
$bind_txt->comment("This is a modified comment");
# Submit modification
my $response = $session->modify( $bind_txt );
```

Infoblox::Session->remove( )

Use this method to remove an object from the Infoblox appliance. See Infoblox::Session->remove() for parameters and return values.

To remove a specifc object, first use `get()` or `search()` to retrieve the specific object, and then submit this object for removal.

Example

```
# Get the objects with the same name
my @retrieved_objs = $session->get(
    object => "Infoblox::DNS::Record::TXT",
    name   => "txt.domain.com",
    view   => "default" );
# find the desired object from the retrieved list.
my $desired_txt = $retrieved_objs[0];
# Submit for removal
my $response = $session->remove( $ desired_txt );
```

Infoblox::Session->search( )

Use this method to search for DNS TXT record objects in the Infoblox appliance. See Infoblox::Session->search() for parameters and return values.

Key References

```
 Apply the following attributes to search for a DNS TXT record object:

  name                  - Optional. A name in FQDN (Fully Qualified Domain Name) format (regular
expression).
  view                  - Optional. The DNS view in which the TXT record is located. By default, all
DNS views are searched. But if you omit this attribute and specify a zone, the appliance searches the
'default' view only.
  zone                  - Optional. A zone name in FQDN format.
  comment               - Optional. A comment in string format (regular expression).
  text                  - Optional. The text of the TXT record (regular expression).
  creator               - Optional. Record creator.
  ddns_principal        - Optional. GSS-TSIG principal that owns the record.
  reclaimable           - Optional. The flag that indicates whether the record is reclaimable or not.
  extattrs              - Optional. A hash reference containing extensible attributes.
  extensible_attributes - Optional. A hash reference containing extensible attributes.
```

For more information about searching extensible attributes, see Infoblox::Grid::ExtensibleAttributeDef/Searching Extensible Attributes.

Example

```
# search for all DNS TXT objects that match "domain.com" in the default DNS view
my @retrieved_objs = $session->search(
    object => "Infoblox::DNS::Record::TXT",
    name   => 'domain\.com',
    view   => "default" );

# search for all DNS TXT records in the "domain.com" zone of the default view
my @retrieved_objs = $session->search(
    object => "Infoblox::DNS::Record::TXT",
    zone   => "domain.com",
    view   => "default" );

# search all DNS TXT records with a given extensible attribute 'Site'
my @retrieved_objs = $session->search(
   object => "Infoblox::DNS::Record::TXT",
   extensible_attributes => { 'Site' => 'Santa Clara' });
```

METHODS

This section describes all the methods that you can use to configure and retrieve the attribute values of a TXT record

comment( )

Use this method to set or retrieve the descriptive comment.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

Desired comment in string format with a maximum of 256 bytes.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get comment
```

```
my $comment = $bind_txt->comment();
#Modify comment
$bind_txt->comment("Modifying the DNS TXT comment");
```

cloud_info( )

Use this method to retrieve cloud API related information for the Infoblox::DNS::Record::TXT object.

Parameter

None

Returns

The method returns the attribute value.

Example

```
# Get cloud_info
my $cloud_info = $object->cloud_info();
```

creator( )

Use this method to set or retrieve the record creator.

Note that changing creator from or to 'SYSTEM' value is not allowed.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Paramter

The valid values are 'STATIC' and 'DYNAMIC'. The default value is 'STATIC'.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get creator value
my $creator = $object->creator();

#Modify creator value
$object->creator("DYNAMIC");
```

creation_time( )

Use this method to retrieve the creation time for the record. This is a read-only attribute.

Parameter

None

Returns

The valid return value is a number of seconds that have elapsed since January 1st, 1970 UTC.

Example

```
#Get creation_time value
my $creation_time = $object->creation_time();
```

ddns_principal( )

Use this method to set or retrive the GSS-TSIG principal that owns this record.

Note that you cannot set ddns_principal for 'STATIC' and 'SYSTEM' records.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Paramter

The GSS-TSIG principal FQDN (Fully Qualified Domain Name) format. The FQDN consists of the hostname followed by the domain name (example: abc.com). A hostname can have a maximum of 256 characters.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get ddns_principal value
my $ddns_principal = $object->ddns_principal();

#Modify ddns_principal value
$object->ddns_principal('foo.com');
```

ddns_protected( )

Use this method to set or retrieve the flag that indicates whether DDNS updates for this record are allowed or not.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

Specify 'true' to protect record from DDNS updates and 'false' to allow DDNS updates for the specified record.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get ddns_protected value
my $ddns_protected = $object->ddns_protected();

#Modify ddns_protected value
$object->ddns_protected('true');
```

disable( )

Use this method to set or retrieve the disable flag of a DNS record.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

The default value for this field is false. The DNS record is enabled.

Parameter

Specify "true" to set the disable flag or "false" to deactivate/unset it.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get disable
my $disable = $bind_txt->disable();
#Modify disable
$bind_txt->disable("true");
```

dns_name( )

Use this method to retrieve the name in punycode format. This is a read-only attribute.

Parameter

None

Returns

The method returns the attribute value.

Example

```
# Get attribute value
my $value = $bind_txt->dns_name();
```

extattrs( )

Use this method to set or retrieve the extensible attributes associated with a DNS TXT record object.

Parameter

Valid value is a hash reference containing the names of extensible attributes and their associated values (Infoblox::Grid::Extattr objects).

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get extattrs
my $ref_extattrs = $bind_txt->extattrs();
#Modify extattrs
$bind_txt->extattrs({ 'Site' => $extattr1, 'Administrator' => $extattr2 });
```

extensible_attributes( )

Use this method to set or retrieve the extensible attributes associated with a DNS TXT record.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

For valid values for extensible attributes, see Infoblox::Grid::ExtensibleAttributeDef/Extensible Attribute Values.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get extensible attributes
my $ref_extensible_attributes = $bindtxt->extensible_attributes();
#Modify extensible attributes
$bindtxt->extensible_attributes({ 'Site' => 'Santa Clara', 'Administrator' => [ 'Peter', 'Tom' ] });
```

forbid_reclamation( )

Use this method to set or retrieve the flag that indicates whether the reclamation is allowed for the record or not.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

Specify 'true' to forbid reclamation for the record and 'false' to allow it. The default value is 'false'.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get forbid_reclamation
my $forbid_reclamation = $object->forbid_reclamation();
#Modify forbid_reclamation
$object->forbid_reclamation('true');
```

last_queried( )

Use this method to retrieve the time when the associated record was last queried. This is a read-only attribute.

Parameter

None

Returns

The method returns the attribute value. The number of seconds that have elapsed since January 1st, 1970 UTC.

Example

```
#Get last_queried
my $last_queried = $bindtxt->last_queried();
```

name( )

Use this method to set or retrieve the name.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

The attribute value can be in unicode format.

Parameter

The name of the DNS TXT object in Fully Qualified Domain Name (FQDN) format. The name can have a maximum of 256 bytes.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get name
my $name = $bind_txt->name();
#Modify name
$bind_txt->name("new.domain.com");
```

reclaimable( )

Use this method to retrieve the flag that indicates whether the record is reclaimable or not.

Parameter

None

Returns

The method returns the attribute value.

Example

```
#Get reclaimable
my $reclaimable = $object->reclaimable();
```

text( )

Use this method to set or retrieve the text.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

Text that you want to associate with the record. It can contain up to 255 bytes.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get name
my $name = $bind_txt->text();
#Modify name
$bind_txt->text("this is new text string");
```

ttl( )

Use this method to set or retrieve the Time to Live (TTL) value.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

The default value is undefined which indicates that the record inherits the TTL value of the zone.

Specify a TTL value to override the TTL value at the zone level.

Parameter

A 32-bit integer (range from 0 to 4294967295) that represents the duration in seconds that the record is cached. Zero indicates that the record should not be cached.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get ttl
my $ttl = $bind_txt->ttl();
#Modify ttl
$bind_txt->ttl(1800);
#Un-override ttl
$bind_txt->ttl(undef);
```

views( )

Use this method to set or retrieve the view of a TXT record.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

The default value is the "default" view, which means the TXT record is located under the default view.

Parameter

Array reference of defined Infoblox::DNS::View objects.

Note that the array size must be 1.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get views
my $ref_views = $bind_txt->views();
#Modify views, list of Infoblox::DNS::View objects
$bind_txt->views([$view1]);
```

zone( )

Use this method to retrieve the zone name of a DNS TXT record. This method is read-only and cannot be set.

Parameter

None

Returns

Returns the attribute value.

Example

```
# Get zone
my $zone = $bind_txt->zone();
```

SAMPLE CODE

The following sample code demonstrates the different functions that can be applied to an object, such as add, search, modify, and remove. This sample also includes error handling for the operations.

#Preparation prior to a DNS TXT record insertion

```
#PROGRAM STARTS: Include all the modules that will be used
use strict;
use Infoblox;

#Create a session to the Infoblox appliance
```

```
my $session = Infoblox::Session->new(
               master   => "192.168.1.2", #appliance host ip
               username => "admin",       #appliance user login
               password => "infoblox"     #appliance password
               );

unless ($session) {
       die("Construct session failed: ",
               Infoblox::status_code() . ":" . Infoblox::status_detail());
}
print "Session created successfully\n";

#Create the zone prior to a TXT record insertion
my $zone = Infoblox::DNS::Zone->new(name => "domain.com");
unless ($zone) {
       die("Construct zone failed: ",
               Infoblox::status_code() . ":" . Infoblox::status_detail());
}
print "Zone object created successfully\n";

#Verify if the zone exists
my $object = $session->get(object => "Infoblox::DNS::Zone", name => "domain.com");
unless ($object) {
       print "Zone does not exist on server, safe to add the zone\n";
       $session->add($zone)
               or die("Add zone failed: ",
                               $session->status_code() . ":" . $session->status_detail());
}
print "Zone added successfully\n";
```

#Create a DNS TXT record

```
my $bind_txt = Infoblox::DNS::Record::TXT->new(
               name    => "txt.domain.com",
               comment => "add TXT txt.domain.com",
               text    => "this is text string",
               );

unless ($bind_txt) {
       die("Construct DNS record TXT failed: ",
               Infoblox::status_code() . ":" . Infoblox::status_detail());
}

#Add the DNS TXT record object to the Infoblox appliance through a session
$session->add($bind_txt)
       or die("Add record TXT failed: ",
                       $session->status_code() . ":" . $session->status_detail());
print "DNS TXT object added to server successfully\n";
```

#Search for a specific DNS TXT record

```
#Search all TXT records in the zone that matches "domain.com"
my @retrieved_objs = $session->search(
               object => "Infoblox::DNS::Record::TXT",
               name   => 'domain\.com'
               );
my $object = $retrieved_objs[0];

unless ($object) {
       die("Search record TXT failed: ",
               $session->status_code() . ":" . $session->status_detail());
}
print "Search DNS TXT object found at least 1 matching entry\n";

#Search all TXT records with names that start with "txt" and end with "domain.com"
my @retrieved_objs = $session->search(
               object => "Infoblox::DNS::Record::TXT",
               name   => '^txt.*\.domain\.com\$'
               );
my $object = $retrieved_objs[0];

unless ($object) {
       die("Search record TXT failed: ",
               $session->status_code() . ":" . $session->status_detail());
}
print "Search DNS TXT object using regexp found at least 1 matching entry\n";
```

#Get and modify a DNS TXT record

```
#Get a TXT record through the session
my @retrieved_objs = $session->get(
               object => "Infoblox::DNS::Record::TXT",
               name   => "txt.domain.com"
               );
my $object = $retrieved_objs[0];

unless ($object) {
       die("Get record TXT failed: ",
               $session->status_code() . ":" . $session->status_detail());
}
print "Get DNS TXT object found at least 1 matching entry\n";

#Modify one of the attributes of the specified TXT record
$object->text("this is new text string");

#Apply the changes
$session->modify($object)
       or die("Modify record TXT failed: ",
               $session->status_code() . ":" . $session->status_detail());
print "DNS TXT object modified successfully \n";
```

#Remove a DNS TXT record

```
#Get TXT record through the session
my @retrieved_objs = $session->get(
    object => "Infoblox::DNS::Record::TXT",
    name   => "txt.domain.com"
);
my $object = $retrieved_objs[0];
unless ($object) {
    die("Get record TXT failed: ",
        $session->status_code() . ":" . $session->status_detail());
}
print "Get DNS TXT object found at least 1 matching entry\n";

#Submit the object for removal
$session->remove($object)
       or die("Remove record TXT failed: ",
               $session->status_code() . ":" . $session->status_detail());
print "DNS TXT object removed successfully \n";

####PROGRAM ENDS####
```

AUTHOR

Infoblox Inc. http://www.infoblox.com/

SEE ALSO

Infoblox::Session->add(), Infoblox::Session->get(), Infoblox::Session->modify(), Infoblox::Session->remove(), Infoblox::Session->search(), Infoblox::Session, Infoblox::DNS::View, Infoblox::DNS::Zone

---

COPYRIGHT

Copyright (c) 2017 Infoblox Inc.

Infoblox::DNS::Record::MX - DNS MX record object

- NAME
- DESCRIPTION
- CONSTRUCTOR
- SESSION METHODS
  - Infoblox::Session->add( )
  - Infoblox::Session->get( )
  - Infoblox::Session->modify( )
  - Infoblox::Session->remove( )
  - Infoblox::Session->search( )
- METHODS
  - comment( )
  - cloud_info( )
  - creator( )
  - creation_time( )
  - ddns_principal( )
  - ddns_protected( )
  - disable( )
  - dns_exchanger( )
  - dns_name( )
  - exchanger( )
  - extattrs( )
  - extensible_attributes( )
  - forbid_reclamation( )
  - last_queried( )
  - name( )
  - pref( )
  - reclaimable( )
  - ttl( )
  - views( )
  - zone( )
- SAMPLE CODE
- AUTHOR
- SEE ALSO
- COPYRIGHT

NAME

Infoblox::DNS::Record::MX - DNS MX record object

DESCRIPTION

An MX (mail exchanger) record maps a domain name to a mail exchanger. A mail exchanger is a server that either delivers or forwards mail. You can specify one or more mail exchangers for a zone, as well as the priority for using each mail exchanger. A standard MX record applies to a particular domain or subdomain. A wildcard MX record applies to a domain and all its subdomains.

Note that you must specify only one view for the attribute "views".

CONSTRUCTOR

```
my $mx = Infoblox::DNS::Record::MX->new(
    exchanger             => $fqdn,                                                    # Required
    name                  => $fqdn,                                                    # Required
    pref                  => $num,                                                     # Required
    comment               => $string,                                                  # Optional
/ Default is empty
    disable               => "true" | "false",                                         # Optional
/ Default "false"
    extattrs              => { $string => $extattr, ... },                             # Optional
/ Default is undefined
    extensible_attributes => { $string => $string|$num, $string => [ $string|$num, ...], ... }, # Optional
/ Default is undefined
    ttl                   => $num | undef,                                             # Optional
/ Default is undefined
    views                 => [$View1]                                                  # Optional
/ Default is "default" view
    creator               => "DYNAMIC" | "STATIC",                                     # Optional
/ Default is "STATIC"
    ddns_protected        => "true" | "false",                                         # Optional
/ Default is "false"
    ddns_principal        => $string,                                                  # Optional
```

```
/ Default is undefined
     forbid_reclamation    => "true" | "false",                                    # Optional
/ Default is "false"
 );
```

You cannot set both extattrs and extensible_attributes attributes at the same time.

SESSION METHODS

This section describes all the methods in Infoblox::Session module that you can apply to a DNS MX object.

Infoblox::Session->add( )

Use this method to add an object to the Infoblox appliance. See Infoblox::Session->add() for parameters and return values.

Example

```
#Construct an object
my $bindmx = Infoblox::DNS::Record::MX->new(
    name      => "bind_mx.domain.com",
    comment   => "add mx bind_mx.domain.com",
    pref      => 1,
    exchanger => "exchanger.domain.com",
);
# Submit for addition
my $response = $session->add( $bindmx );
```

Infoblox::Session->get( )

Use this method to retrieve all the matching objects from the Infoblox appliance. See Infoblox::Session->get() for parameters and return values.

Key References

```
Apply the following attributes to get a specific DNS MX object:

  name                  - Optional.  A domain name in string format
  view                  - Optional.  The DNS view in which the MX record is located. By default, all
DNS views are searched. But if you omit this attribute and specify a zone, the appliance searches the
'default' view only.
  zone                  - Optional. A zone name in FQDN format.
  exchanger             - Optional. The exchanger of the MX record.
  pref                  - Optional. The Preference of the MX record.
  creator               - Optional. Record creator.
  ddns_principal        - Optional. GSS-TSIG principal that owns the record.
  reclaimable           - Optional. The flag that indicates whether the record is reclaimable or not.
  extattrs              - Optional. A hash reference containing extensible attributes.
  extensible_attributes - Optional. A hash reference containing extensible attributes.
```

Example

```
 my @retrieved_objs = $session->get(
    object    => "Infoblox::DNS::Record::MX",
    name      => "bind_mx.domain.com",
    view      => "default" );

 # get all DNS MX records in the "domain.com" zone of the default view
 my @retrieved_objs = $session->get(
    object    => "Infoblox::DNS::Record::MX",
    zone      => "domain.com",
    view      => "default" );

# get all DNS MX records with a given extensible attribute
my @retrieved_objs = $session->get(
    object => "Infoblox::DNS::Record::MX",
    extensible_attributes => { 'Site' => 'Santa Clara' });
```

Infoblox::Session->modify( )

Use this method to modify an object in the Infoblox appliance. See Infoblox::Session->modify() for parameters and return values.

Example

```
#Use method to modify the comment.
$bindmx->comment("this is a modified comment");
# Submit modification
my $response = $session->modify( $bindmx );
```

Infoblox::Session->remove( )

Use this method to remove an object from the Infoblox appliance. See Infoblox::Session->remove() for parameters and return values.

To remove a specifc object, first use `get()` or `search()` to retrieve the specific object, and then submit this object for removal.

Example

```
# Get the object under the same name
my @retrieved_objs = $session->get(
    name      => "bind_mx.domain.com",
    comment   => "add mx bind_mx.domain.com",
    pref      => 1,
    exchanger => "exchanger.domain.com",
    view      => "default" );
# find the desired object from retrieved list.
my $desired_mx = $retrieved_objs[0];
# Submit for removal
my $response = $session->remove( $desired _mx );
```

Infoblox::Session->search( )

Use this method to search for DNS MX record objects in the Infoblox appliance. See Infoblox::Session->search() for parameters and return values.

Key References

```
Apply the following attributes to search for a DNS MX record object:

  name                  - Optional. A domain name in string format (regular expression).
  view                  - Optional. The DNS view in which the MX record is located. By default, all
DNS views are searched. But if you omit this attribute and specify a zone, the appliance searches the
'default' view only.
  zone                  - Optional. A zone name in FQDN format.
  comment               - Optional. A comment in string format (regular expression).
  exchanger             - Optional. The exchanger of the MX record (regular expression).
  pref                  - Optional. The Preference of the MX record (regular expression).
  creator               - Optional. Record creator.
  ddns_principal        - Optional. GSS-TSIG principal that owns the record.
  reclaimable           - Optional. The flag that indicates whether the record is reclaimable or not.
  extattrs              - Optional. A hash reference containing extensible attributes.
  extensible_attributes - Optional. A hash reference containing extensible attributes.
```

For more information about searching extensible attributes, see Infoblox::Grid::ExtensibleAttributeDef/Searching Extensible Attributes.

Example

```
# search for all DNS MX objects that match "domain.com" in the default DNS view
    my @retrieved_objs = $session->search(
    object => "Infoblox::DNS::Record::MX",
    name   => 'domain\.com',
    view   => "default" );

# search for all DNS MX objects in the domain.com zone of the default DNS view
    my @retrieved_objs = $session->search(
    object => "Infoblox::DNS::Record::MX",
    zone   => "domain.com",
    view   => "default" );

# search all DNS MX records with a given extensible attribute 'Site'
my @retrieved_objs = $session->search(
   object => "Infoblox::DNS::Record::MX",
   extensible_attributes => { 'Site' => 'Santa Clara' });
```

METHODS

This section describes all the methods that you can use to configure and retrieve the attribute values of an MX record

comment( )

Use this method to add or retrieve a descriptive comment.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

Enter a descriptive comment for this record in string format with a maximum of 256 bytes.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
# Getting comment
my $comment = $bindmx->comment();
# Modifying comment
$bindmx->comment("add mx2.domain.com");
```

cloud_info( )

Use this method to retrieve cloud API related information for the Infoblox::DNS::Record::MX object.

Parameter

None

Returns

The method returns the attribute value.

Example

```
# Get cloud_info
my $cloud_info = $object->cloud_info();
```

creator( )

Use this method to set or retrieve the record creator.

Note that changing creator from or to 'SYSTEM' value is not allowed.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Paramter

The valid values are 'STATIC' and 'DYNAMIC'. The default value is 'STATIC'.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get creator value
my $creator = $object->creator();

#Modify creator value
$object->creator("DYNAMIC");
```

creation_time( )

Use this method to retrieve the creation time for the record. This is a read-only attribute.

Parameter

None

Returns

The valid return value is a number of seconds that have elapsed since January 1st, 1970 UTC.

Example

```
#Get creation_time value
my $creation_time = $object->creation_time();
```

ddns_principal( )

Use this method to set or retrive the GSS-TSIG principal that owns this record.

Note that you cannot set ddns_principal for 'STATIC' and 'SYSTEM' records.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Paramter

The GSS-TSIG principal FQDN (Fully Qualified Domain Name) format. The FQDN consists of the hostname followed by the domain name (example: abc.com). A hostname can have a maximum of 256 characters.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get ddns_principal value
my $ddns_principal = $object->ddns_principal();

#Modify ddns_principal value
$object->ddns_principal('foo.com');
```

ddns_protected( )

Use this method to set or retrieve the flag that indicates whether DDNS updates for this record are allowed or not.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

Specify 'true' to protect record from DDNS updates and 'false' to allow DDNS updates for the specified record.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get ddns_protected value
my $ddns_protected = $object->ddns_protected();

#Modify ddns_protected value
$object->ddns_protected('true');
```

disable( )

Use this method to set or retrieve the disable flag of a DNS record.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

The default value for this field is false. The DNS record is enabled.

Parameter

Specify "true" to set the disable flag or "false" to deactivate/unset it.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
# Getting disable
my $disable = $bindmx->disable()
# Modifying disable
$bindmx->disable("true");
```

dns_exchanger( )

Use this method to retrieve the exchanger in punycode format. This is a read-only attribute.

Parameter

None

Returns

The method returns the attribute value.

Example

```
# Get attribute value
my $value = $bindmx->dns_exchanger();
```

dns_name( )

Use this method to retrieve the domain name in punycode format. This is a read-only attribute.

Parameter

None

Returns

The method returns the attribute value.

Example

```
# Get attribute value
my $value = $bindmx->dns_name();
```

exchanger( )

Use this method to set or retrieve the exchanger.

The name of the mail exchanger in Fully Qualified Domain Name (FQDN) format.

The attribute value can be in unicode format.

Parameter

Text with the name of the mail exchanger for the DNS MX record.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails. If you did not specify a parameter, the method returns the attribute value.

Example

```
# Getting exchanger
my $exchanger = $bindmx->exchanger();
# Modifying exchanger
$bindmx->exchanger("exchanger2.domain.com");
```

extattrs( )

Use this method to set or retrieve the extensible attributes associated with a DNS MX record object.

Parameter

Valid value is a hash reference containing the names of extensible attributes and their associated values (Infoblox::Grid::Extattr objects).

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get extattrs
my $ref_extattrs = $bindmx->extattrs();
#Modify extattrs
$bindmx->extattrs({ 'Site' => $extattr1, 'Administrator' => $extattr2 });
```

extensible_attributes( )

Use this method to set or retrieve the extensible attributes associated with a DNS MX record.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

For valid values for extensible attributes, see Infoblox::Grid::ExtensibleAttributeDef/Extensible Attribute Values.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get extensible attributes
my $ref_extensible_attributes = $bindmx->extensible_attributes();
#Modify extensible attributes
$bindmx->extensible_attributes({ 'Site' => 'Santa Clara', 'Administrator' => [ 'Peter', 'Tom' ] });
```

forbid_reclamation( )

Use this method to set or retrieve the flag that indicates whether the reclamation is allowed for the record or not.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

Specify 'true' to forbid reclamation for the record and 'false' to allow it. The default value is 'false'.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get forbid_reclamation
my $forbid_reclamation = $object->forbid_reclamation();
#Modify forbid_reclamation
$object->forbid_reclamation('true');
```

last_queried( )

Use this method to retrieve the time when the associated record was last queried. This is a read-only attribute.

Parameter

None

Returns

The method returns the attribute value. The number of seconds that have elapsed since January 1st, 1970 UTC.

Example

```
#Get last_queried
my $last_queried = $bindmx->last_queried();
```

name( )

Use this method to set or retrieve the domain name.

The name of the DNS MX object in Fully Qualified Domain Name (FQDN) format. If you want to define an MX record for a domain whose name matches the zone in which you define the MX record, leave this field empty. If you want to define an MX record for a subdomain, enter that subdomain name here. If you want to define an MX record for a domain and all its subdomains, enter an asterisk (*) here to create a wildcard MX record.

The attribute value can be in unicode format.

Parameter

Text with the name for the DNS MX record.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Getting name
```

```
my $name = $bindmx->name();
# Modifying name
$bindmx->name("mx2.domain.com");
```

pref( )

Use this method to set or retrieve the Preference value.

The preference value of the DNS MX object. Preference value of MX record for the zone range from 0 to 65535. Low values are more preferred.

Parameter

Enter an unsigned integer between 0-65535. The priority determines the order in which a client attempts to contact the target mail exchanger. The highest priority is 0 and is queried first.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
# Getting pref
my $pref = $bindmx->pref();
# Modifying pref
$bindmx->pref(3);
```

reclaimable( )

Use this method to retrieve the flag that indicates whether the record is reclaimable or not.

Parameter

None

Returns

The method returns the attribute value.

Example

```
#Get reclaimable
my $reclaimable = $object->reclaimable();
```

ttl( )

Use this method to configure or retrieve the Time to Live (TTL) value.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

The default value is undefined which indicates that the record inherits the TTL value of the zone.

Specify a TTL value to override the TTL value at the zone level.

Parameter

A 32-bit integer (range from 0 to 4294967295) that represents the duration in seconds that the record is cached. Zero indicates that the record should not be cached.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get ttl
my $ttl = $bindmx->ttl();
#Modify ttl
$bindmx->ttl(1800);
#Un-override ttl
$bindmx->ttl(undef);
```

views( )

Use this method to specify or retrieve the view of the MX record.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

The default value is the 'default' view, which means the MX record is located under the default view.

Parameter

Array reference of defined Infoblox::DNS::View objects.

Note that the array size must be 1.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get views
my $ref_views = $bindmx->views();
#Modify views, list of Infoblox::DNS::View objects
$bindmx->views([$view1]);
```

zone( )

Use this method to retrieve the zone name of a DNS MX record. This method is read-only and cannot be set.

Parameter

None

Returns

Returns the attribute value.

Example

```
# Get zone
my $zone = $bindmx->zone();
```

SAMPLE CODE

The following sample code demonstrates the different functions that can be applied to an object, such as add, search, modify, and remove. This sample also includes error handling for the operations.

#Preparation prior to a DNS MX record insertion

```
#PROGRAM STARTS: Include all the modules that will be used
use strict;
use Infoblox;

#Create a session to the Infoblox appliance
my $session = Infoblox::Session->new(
    master   => "192.168.1.2",
    username => "admin",
    password => "infoblox"
);
unless ($session) {
   die("Construct session failed: ",
       Infoblox::status_code() . ":" . Infoblox::status_detail());
}
print "Session created successfully\n";

#Create the zone prior to MX  record insertion
my $zone = Infoblox::DNS::Zone->new(name => "domain.com");
unless ($zone) {
   die("Construct zone failed: ",
     Infoblox::status_code() . ":" . Infoblox::status_detail());
}
print "Zone object created successfully\n";

#Verify if the zone exists
my $object = $session->get(object => "Infoblox::DNS::Zone", name => "domain.com");
unless ($object) {
   print "Zone does not exist on server, safe to add the zone\n";
   $session  add $zone
```

```
            ->   (     )
      or die("Add zone failed: ",
             $session->status_code() . ":" . $session->status_detail());
}
print "Zone added successfully\n";
```

#Create a DNS MX record

```
 #Construct a DNS MX object
  my $bindmx = Infoblox::DNS::Record::MX->new(
    name      => "bind_mx.domain.com",
    comment   => "add mx bind_mx.domain.com",
    pref      => 1,
    exchanger => "exchanger.domain.com",
  );

unless ($bindmx) {
   die("Construct DNS record MX failed: ",
       Infoblox::status_code() . ":" . Infoblox::status_detail());
}
print "DNS MX object created successfully\n"; #Add the DNS MX record object to Infoblox appliance through
a session
$session->add($bindmx)
    or die("Add record MX failed: ",
           $session->status_code() . ":" . $session->status_detail());
print "DNS MX object added to server successfully\n";
```

#Search for a specific DNS MX record

```
#Search all MX records that match "domain.com"
my @retrieved_objs = $session->search(
    object => "Infoblox::DNS::Record::MX",
    name   => 'domain\.com'
);
my $object = $retrieved_objs[0];
unless ($object) {
    die("Search record MX failed: ",
        $session->status_code() . ":" . $session->status_detail());
}
print "Search DNS MX object found at least 1 matching entry\n";

#Search all MX records that start with "bind" and end with "domain.com"
my @retrieved_objs = $session->search(
    object => "Infoblox::DNS::Record::MX",
    name   => '^bind.*\.domain\.com\$'
);
my $object = $retrieved_objs[0];
unless ($object) {
    die("Search record MX failed: ",
        $session->status_code() . ":" . $session->status_detail());
}
print "Search DNS MX object using regexp found at least 1 matching entry\n";
```

#Get and modify a DNS MX record

```
#Get MX record through the session
my  @retrieved_objs = $session->get(
    object    => "Infoblox::DNS::Record::MX",
    name      => "bind_mx.domain.com",
    pref      => 1,
    exchanger => "exchanger.domain.com",
);

my $object = $retrieved_objs[0];
unless ($object) {
    die("Get record MX failed: ",
        $session->status_code() . ":" . $session->status_detail());
}
print "Get DNS MX object found at least 1 matching entry\n";

#Modify one of the attributes of the specified MX record
$object->exchanger("exc.domain.com");

#Apply the changes
$session->modify($object)
    or die("Modify record MX failed: ",
           $session->status_code() . ":" . $session->status_detail());
```

```
print "DNS MX object modified successfully \n";
```

#Remove a DNS MX record

```
 #Get MX record through the session
 my @retrieved_objs = $session->get(
     object => "Infoblox::DNS::Record::MX",
     name   => "bind_mx.domain.com"
 );
  my $object = $retrieved_objs[0];
  unless ($object) {
   die("Get record MX failed: ",
       $session->status_code() . ":" . $session->status_detail());
 }
  print "Get DNS MX object found at least 1 matching entry\n";

#Submit the object for removal
$session->remove($object)
    or die("Remove record MX failed: ",
        $session->status_code() . ":" . $session->status_detail());
print "DNS MX object removed successfully \n";

####PROGRAM ENDS####
```

AUTHOR

Infoblox Inc. http://www.infoblox.com/

SEE ALSO

Infoblox::DNS::View, Infoblox::DNS::Zone, Infoblox::Session, Infoblox::Session->get(), Infoblox::Session->search(), Infoblox::Session->add(), Infoblox::Session->remove(), Infoblox::Session->modify()

COPYRIGHT

Copyright (c) 2017 Infoblox Inc.

Infoblox::DNS::Record::SRV - DNS SRV record object

- NAME
- DESCRIPTION
- CONSTRUCTOR
- SESSION METHODS
  - Infoblox::Session->add( )
  - Infoblox::Session->get( )
  - Infoblox::Session->modify( )
  - Infoblox::Session->remove( )
  - Infoblox::Session->search( )
- METHODS
  - comment( )
  - cloud_info( )
  - creator( )
  - creation_time( )
  - ddns_principal( )
  - ddns_protected( )
  - disable( )
  - dns_name( )
  - dns_target( )
  - extattrs( )
  - extensible_attributes( )
  - forbid_reclamation( )
  - last_queried( )
  - name( )
  - port( )
  - priority( )
  - reclaimable( )
  - target( )
  - ttl( )
  - views( )
  - weight( )
  - zone( )
- SAMPLE CODE
- AUTHOR
- SEE ALSO
- COPYRIGHT

NAME

Infoblox::DNS::Record::SRV - DNS SRV record object

DESCRIPTION

A DNS SRV object represents an SRV resource record, which is also known as a service record. An SRV record provides information on available services.

Note that you must specify only one view for the attribute "views".

CONSTRUCTOR

```
 my $srv = Infoblox::DNS::Record::SRV->new(
     name                  => $fqdn,                                                  # Required
     port                  => $num,                                                   # Required
     priority              => $num,                                                   # Required
     target                => $fqdn,                                                  # Required
     weight                => $num,                                                   # Required
     comment               => $string,                                                # Optional
/ Default is empty
     disable               => "true" | "false"                                        # Optional
/ Default "false"
     extattrs              => { $string => $extattr, ... },                           # Optional
/ Default is undefined
     extensible_attributes => { $string => $string|$num, $string => [ $string|$num, ...], ... }, # Optional
/ Default is undefined
     ttl                   => $num | undef,                                           # Optional
/ Default is undefined
     views                 => [$View1],                                               # Optional
/ Default is "default" view
     creator               => "DYNAMIC" | "STATIC",                                   # Optional
/ Default is "STATIC"
```

```
    ddns_protected        => "true" | "false",                                  # Optional
/ Default is "false"
    ddns_principal        => $string,                                           # Optional
/ Default is undefined
    forbid_reclamation    => "true" | "false",                                  # Optional
/ Default is "false"
 );
```

You cannot set both extattrs and extensible_attributes attributes at the same time.

SESSION METHODS

This section describes all the methods in Infoblox::Session that you can apply to a DNS SRV record object.

Infoblox::Session->add( )

Use this method to add an object to the Infoblox appliance. See Infoblox::Session->add() for parameters and return values.

Example

```
#Construct object
my $bind_srv = Infoblox::DNS::Record::SRV->new(
    name     => "bind_srv.domain.com",
    comment  => "add bind srv bind_srv.domain.com",
    priority => 1,
    port     => 1,
    weight   => 1,
    target   => "old.target.test.org",
);
# Submit for addition
my $response = $session->add( $bind_srv );
```

Infoblox::Session->get( )

Use this method to retrieve the existing objects from the Infoblox appliance. See Infoblox::Session->get() for parameters and return values.

Key Reference

```
Apply the following attributes to get a specified DNS SRV object:

  name                  - Optional. A name in string format.
  view                  - Optional. The DNS view in which the SRV record is located. By default, all
DNS views are searched. But if you omit this attribute and specify a zone, the appliance searches the
'default' view only.
  zone                  - Optional. A zone name in FQDN format.
  weight                - Optional. The weight of the SRV record.
  port                  - Optional. The port of the SRV record.
  priority              - Optional. The priority of the SRV record.
  target                - Optional. The target of the SRV record.
  creator               - Optional. Record creator.
  ddns_principal        - Optional. GSS-TSIG principal that owns the record.
  reclaimable           - Optional. The flag that indicates whether the record is reclaimable or not.
  extattrs              - Optional. A hash reference containing extensible attributes.
  extensible_attributes - Optional. A hash reference containing extensible attributes.
```

Example

```
my @retrieved_objs = $session->get(
    object => "Infoblox::DNS::Record::SRV",
    name     => "bind_srv.domain.com",
    view   => "default" );

# get all DNS SRV record in zone "domain.com" of default view
my @retrieved_objs = $session->get(
    object => "Infoblox::DNS::Record::SRV",
    zone     => "domain.com",
    view   => "default" );

# get all DNS SRV records with a given extensible attribute
my @retrieved_objs = $session->get(
    object => "Infoblox::DNS::Record::SRV",
    extensible_attributes => { 'Site' => 'Santa Clara' });
```

Infoblox::Session->modify( )

Use this method to modify objects in the Infoblox appliance. See Infoblox::Session->modify() for parameters and return values.

Example

```
# Use method to modify the comment.
$bind_srv->comment("this is a modified comment");
# Submit modification
my $response = $session->modify( $bind_srv);
```

Infoblox::Session->remove( )

Use this method to remove an object from the Infoblox appliance. See Infoblox::Session->remove() for parameters and return values.

To remove a specific object, first use `get()` or `search()` to retrieve the specific object, and then submit this object for removal.

Example

```
# Get the object under the same name
my @retrieved_objs = $session->get(
    object => "Infoblox::DNS::Record::SRV",
    name     => "bind_srv.domain.com",
    view   => "default" );
my $desired_srv= $retrieved_objs[0];
# Submit for removal
my $response = $session->remove( $desired_srv );
```

Infoblox::Session->search( )

Use this method to search for existing DNS SRV record objects in the Infoblox appliance. See Infoblox::Session->search() for parameters and return values.

Key Reference

```
 Apply the following attributes to search for a DNS SRV record object:

   name                  - Required. A name in FQDN (Fully Qualified Domain Name) format (regular
expression).
   view                  - Optional. The DNS view in which the SRV record is located. By default, all
DNS views are searched. But if you omit this attribute and specify a zone, the appliance searches the
'default' view only.
   zone                  - Optional. A zone name in FQDN format.
   comment               - Optional . A comment in string format (regular expression).
   weight                - Optional. The weight of the SRV record (regular expression).
   port                  - Optional. The port of the SRV record (regular expression).
   priority              - Optional. The priority of the SRV record (regular expression).
   target                - Optional. The target of the SRV record (regular expression).
   creator               - Optional. Record creator.
   ddns_principal        - Optional. GSS-TSIG principal that owns the record.
   reclaimable           - Optional. The flag that indicates whether the record is reclaimable or
not.
   extattrs              - Optional. A hash reference containing extensible attributes.
   extensible_attributes - Optional. A hash reference containing extensible attributes.
```

For more information about searching extensible attributes, see Infoblox::Grid::ExtensibleAttributeDef/Searching Extensible Attributes.

Example

```
# search for all DNS SRV objects in the default DNS view that match "domain.com" and that contain
"bind" in the comment field
my @retrieved_objs = $session->search(
    object => "Infoblox::DNS::Record::SRV",
    name     => 'domain\.com',
    comment  => "bind",
    view     => "default" );

# search for all DNS SRV records in the "domain.com" zone of the default view
my @retrieved_objs = $session->search(
    object => "Infoblox::DNS::Record::SRV",
    zone     => "domain.com",
    view   => "default" );

# search all DNS SRV records with the extensible attribute 'Site'
my @retrieved_objs = $session->search(
   object => "Infoblox::DNS::Record::SRV",
   extensible_attributes => { 'Site' => 'Santa Clara' });
```

METHODS

This section describes all the methods that you can use to configure and retrieve the attribute values of an SRV record.

comment( )

Use this method to add or retrieve a descriptive comment.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

Desired comment in string format with a maximum of 256 bytes.

Returns

If you specified a parameter,the method returns true when the modification succeeds,and returns false when the operation fails.

Example

```
#Get comment
my $comment = $bindsrv->comment();
#Modify comment
$bindsrv->comment("Modifying the DNS SRV comment");
```

cloud_info( )

Use this method to retrieve cloud API related information for the Infoblox::DNS::Record::SRV object.

Parameter

None

Returns

The method returns the attribute value.

Example

```
# Get cloud_info
my $cloud_info = $object->cloud_info();
```

creator( )

Use this method to set or retrieve the record creator.

Note that changing creator from or to 'SYSTEM' value is not allowed.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Paramter

The valid values are 'STATIC' and 'DYNAMIC'. The default value is 'STATIC'.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get creator value
my $creator = $object->creator();

#Modify creator value
$object->creator("DYNAMIC");
```

creation_time( )

Use this method to retrieve the creation time for the record. This is a read-only attribute.

Parameter

None

Returns

The valid return value is a number of seconds that have elapsed since January 1st, 1970 UTC.

Example

```
#Get creation_time value
my $creation_time = $object->creation_time();
```

ddns_principal( )

Use this method to set or retrive the GSS-TSIG principal that owns this record.

Note that you cannot set ddns_principal for 'STATIC' and 'SYSTEM' records.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Paramter

The GSS-TSIG principal FQDN (Fully Qualified Domain Name) format. The FQDN consists of the hostname followed by the domain name (example: abc.com). A hostname can have a maximum of 256 characters.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get ddns_principal value
my $ddns_principal = $object->ddns_principal();

#Modify ddns_principal value
$object->ddns_principal('foo.com');
```

ddns_protected( )

Use this method to set or retrieve the flag that indicates whether DDNS updates for this record are allowed or not.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

Specify 'true' to protect record from DDNS updates and 'false' to allow DDNS updates for the specified record.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get ddns_protected value
my $ddns_protected = $object->ddns_protected();

#Modify ddns_protected value
$object->ddns_protected('true');
```

disable( )

Use this method to set or retrieve the disable flag.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

The default value for this field is false, therefore the DNS record is enabled.

Parameter

Specify "true" to set the disable flag or "false" to deactivate/unset it.

Returns

If you specified a parameter,the method returns true when the modification succeeds,and returns false when the operation fails.

Example

```
#Get disable
my $disable = $bindsrv->disable();
#Modify disable
$bindsrv->disable("true");
```

dns_name( )

Use this method to retrieve the name in punycode format. This is a read-only attribute.

Parameter

None

Returns

The method returns the attribute value.

Example

```
# Get attribute value
my $value = $bindsrv->dns_name();
```

dns_target( )

Use this method to retrieve the target in punycode format. This is a read-only attribute.

Parameter

None

Returns

The method returns the attribute value.

Example

```
# Get attribute value
my $value = $bindsrv->dns_target();
```

extattrs( )

Use this method to set or retrieve the extensible attributes associated with a DNS SRV record object.

Parameter

Valid value is a hash reference containing the names of extensible attributes and their associated values (Infoblox::Grid::Extattr objects).

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get extattrs
my $ref_extattrs = $bindsrv->extattrs();
#Modify extattrs
$bindsrv->extattrs({ 'Site' => $extattr1, 'Administrator' => $extattr2 });
```

extensible_attributes( )

Use this method to set or retrieve the extensible attributes associated with a DNS SRV record.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

For valid values for extensible attributes, see Infoblox::Grid::ExtensibleAttributeDef/Extensible Attribute Values.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get extensible attributes
my $ref_extensible_attributes = $bindsrv->extensible_attributes();
#Modify extensible attributes
$bindsrv->extensible_attributes({ 'Site' => 'Santa Clara', 'Administrator' => [ 'Peter', 'Tom' ] });
```

forbid_reclamation( )

Use this method to set or retrieve the flag that indicates whether the reclamation is allowed for the record or not.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

Specify 'true' to forbid reclamation for the record and 'false' to allow it. The default value is 'false'.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get forbid_reclamation
my $forbid_reclamation = $object->forbid_reclamation();
#Modify forbid_reclamation
$object->forbid_reclamation('true');
```

last_queried( )

Use this method to retrieve the time when the associated record was last queried. This is a read-only attribute.

Parameter

None

Returns

The method returns the attribute value. The number of seconds that have elapsed since January 1st, 1970 UTC.

Example

```
#Get last_queried
my $last_queried = $bindsrv->last_queried();
```

name( )

Use this method to set or retrieve the name.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

The attribute value can be in unicode format.

Parameter

The name of the DNS SRV record.

Returns

If you specified a parameter,the method returns true when the modification succeeds,and returns false when the operation fails.

Example

```
#Get name
my $name = $bindsrv->name();
#Modify name
$bindsrv->name("modified_bind_srv.domain.com");
```

port( )

Use this method to add or retrieve a port.

Parameter

The port number for the DNS SRV record (range from 0 to 65535).

Returns

If you specified a parameter,the method returns true when the modification succeeds,and returns false when the operation fails.

Example

```
#Get port
my $port = $bindsrv->port();
#Modify port
$bindport->port("2");
```

priority( )

Use this method to add or retrieve the priority.

Parameter

Number with the priority for the DNS SRV record (range from 0 to 65535).

Returns

If you specified a parameter,the method returns true when the modification succeeds,and returns false when the operation fails.

Example

```
#Get priority
my $priority = $bindsrv->priority();
#Modify priority
$bindsrv->priority("2");
```

reclaimable( )

Use this method to retrieve the flag that indicates whether the record is reclaimable or not.

Parameter

None

Returns

The method returns the attribute value.

Example

```
#Get reclaimable
my $reclaimable = $object->reclaimable();
```

target( )

Use this method to set or retrieve the target. The target must be in Fully Qualified Domain Name (FQDN) format.

The attribute value can be in unicode format.

Parameter

Text with the target for the DNS SRV record.

Returns

If you specified a parameter,the method returns true when the modification succeeds,and returns false when the operation fails.

Example

```
#Get target
my $target = $bindsrv->target();
#Modify target
$bindsrv->target ("new.target.test.org");
```

ttl( )

Use this method to configure or retrieve the Time to Live (TTL) value.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

The default value is undefined; therefore the record inherits the TTL value from the zone level.

With a specified TTL value, the method overrides the zone values with the specified TTL value.

Parameter

A 32-bit integer (range from 0 to 4294967295) that represents the duration in seconds that the record is cached. Zero indicates that the record should not be cached.

Returns

If you specified a parameter,the method returns true when the modification succeeds,and returns false when the operation fails.

Example

```
#Get ttl
my $ttl = $bindsrv->ttl();
```

```
#Modify ttl
$bindsrv->ttl(1800);
#Un-override ttl
$bindsrv->ttl(undef);
```

views( )

Use this method to specify or retrieve the views in which the SRV record is located.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

The default value is the default view, therefore the SRV record is located under the default view.

Parameter

Array reference of defined Infoblox::DNS::View objects.

Note that the array size must be 1.

Returns

If you specified a parameter,the method returns true when the modification succeeds,and returns false when the operation fails.

Example

```
#Get views
my $ref_views = $bindsrv->views();
#Modify views, list of Infoblox::DNS::View objects
$bindsrv->views([$view1]);
```

weight( )

Use this method to set or retrieve the weight.

Parameter

The weight of the DNS SRV record.

Returns

If you specified a parameter,the method returns true when the modification succeeds,and returns false when the operation fails.

Example

```
#Get weight
my $weight = $bindsrv->weight ();
#Modify weight
$bindsrv->weight ("3");
```

zone( )

Use this method to retrieve the zone name of the DNS SRV record. This method is read-only and cannot be set.

Parameter

None

Returns

Returns the attribute value.

Example

```
# Get zone
my $zone = $bindsrv->zone();
```

SAMPLE CODE

The following sample code demonstrates different operations that can be applied to an object, such as create, search, modify, and remove an object. This sample code also includes error handling for the operations.

#Preparation prior to a DNS SRV record insertion

```
#PROGRAM STARTS: Include all the modules that will be used
use strict;
use Infoblox;

#Create a session to the Infoblox appliance
```

```
my $session = Infoblox::Session->new(
    master   => "192.168.1.2",
    username => "admin",
    password => "infoblox"
);
unless ($session) {
   die("Construct session failed: ",
       $session->status_code() . ":" . $session->status_detail());
}
print "Session created successfully\n";

#Create the zone prior to a SRV record insertion
my $zone = Infoblox::DNS::Zone->new(name => "domain.com");
unless ($zone) {
   die("Construct zone failed: ",
       Infoblox::status_code() . ":" . Infoblox::status_detail());
}
print "Zone object created successfully\n";

#Verify if the zone exists
my $object = $session->get(object => "Infoblox::DNS::Zone", name => "domain.com");
unless ($object) {
   print "Zone does not exist on server, safe to add the zone\n";
   $session->add($zone)
      or die("Add zone failed: ",
             $session->status_code() . ":" . $session->status_detail());
}
print "Zone added successfully\n";
```

#Create a DNS SRV record

```
my $bind_srv = Infoblox::DNS::Record::SRV->new(
   name     => "bind_srv.domain.com",
   comment  => "add bind bind_srv.domain.com",
   priority => 1,
   port     => 1,
   weight   => 1,
   target   => "old.target.test.org",
 );
 unless ($bind_srv) {
    die("Construct DNS record SRV failed: ",
        Infoblox::status_code() . ":" . Infoblox::status_detail());
 }
print "DNS SRV object created successfully\n";

#Add the DNS SRV record object to Infoblox  appliance through a session
$session->add($bind_srv)
    or die("Add record SRV failed: ",
           $session->status_code() . ":" . $session->status_detail());
print "DNS SRV object added to server successfully\n";
```

#Search for a specific DNS SRV record

```
#Search for SRV records that match bind_srv.domain.com
my @retrieved_objs = $session->search(
    "object" => "Infoblox::DNS::Record::SRV",
    "name" => 'bind_srv\.domain\.com',
);
my $object = $retrieved_objs[0];
unless ($object) {
    die("Search record SRV failed: ",
        $session->status_code() . ":" . $session->status_detail());
}
print "Search DNS SRV object found at least 1 matching entry\n";

#Search all SRV records that match "domain.com" and that contain "bind" in the comment field
my @retrieved_objs = $session->search(
    object => "Infoblox::DNS::Record::SRV",
    name   => 'domain\.com',
    comment => "bind"
);
my $object = $retrieved_objs[0];
unless ($object) {
    die("Search record SRV failed: ",
       $session->status_code() . ":" . $session->status_detail());
}
```

```
print "Search DNS SRV object using regexp found at least 1 matching entry\n";
```

#Get and modify a DNS SRV record

```
#Get a SRV record through the session
my @retrieved_objs = $session->get(
    object => "Infoblox::DNS::Record::SRV",
    name   => "bind_srv.domain.com",
);
my $object = $retrieved_objs[0];
unless ($object) {
    die("Get record SRV failed: ",
        $session->status_code() . ":" . $session->status_detail());
}
print "Get DNS SRV object found at least 1 matching entry\n";

#Modify one of the attributes of the specified SRV record
$object->comment ("modified comment for SRV rec");

#Apply the changes
$session->modify($object)
    or die("Modify record SRV failed: ",
            $session->status_code() . ":" . $session->status_detail());
print "DNS SRV object modified successfully \n";
```

#Remove a DNS SRV record

```
#Get a SRV record through the session
my @retrieved_objs = $session->get(
    object => "Infoblox::DNS::Record::SRV",
    name   => "bind_srv.domain.com",
);
my $object = $retrieved_objs[0];
unless ($object) {
    die("Get record SRV failed: ",
        $session->status_code() . ":" . $session->status_detail());
}
print "Get DNS SRV object found at least 1 matching entry\n";

#submit the object for removal
$session->remove($object)
    or die("Remove record SRV failed: ",
            $session->status_code() . ":" . $session->status_detail());
print "DNS SRV object removed successfully \n";

####PROGRAM ENDS####
```

AUTHOR

Infoblox Inc. http://www.infoblox.com/

SEE ALSO

Infoblox::DNS::View, Infoblox::DNS::Zone, Infoblox::Session, Infoblox::Session->get(), Infoblox::Session->search(), Infoblox::Session->add(), Infoblox::Session->remove(), Infoblox::Session->modify()

COPYRIGHT

Copyright (c) 2017 Infoblox Inc.

Infoblox::DNS::Record::CNAME - DNS CNAME record object.

- NAME
- DESCRIPTION
- CONSTRUCTOR
- SESSION METHODS
  - Infoblox::Session->add( )
  - Infoblox::Session->get( )
  - Infoblox::Session->modify( )
  - Infoblox::Session->remove( )
  - Infoblox::Session->search( )
- METHODS
  - canonical( )
  - comment( )
  - cloud_info( )
  - creator( )
  - creation_time( )
  - ddns_principal( )
  - ddns_protected( )
  - disable( )
  - dns_name( )
  - dns_canonical( )
  - extattrs( )
  - extensible_attributes( )
  - forbid_reclamation( )
  - last_queried( )
  - name( )
  - reclaimable( )
  - ttl( )
  - views( )
  - zone( )
- SAMPLE CODE
- AUTHOR
- SEE ALSO
- COPYRIGHT

NAME

Infoblox::DNS::Record::CNAME - DNS CNAME record object.

DESCRIPTION

A CNAME record maps an alias to a canonical name. You can use CNAME records in both forward- and IPv4 reverse-mapping zones to serve two different purposes. (At this time, you cannot use CNAME records with IPv6 reverse-mapping zones.)

Note that you must specify only one view for the attribute "views".

CONSTRUCTOR

```
  my $bind_cname =  Infoblox::DNS::Record::CNAME->new (
      canonical             => $fqdn,                                                                  #
Required
      name                  => $fqdn,                                                                  #
Required
      comment               => $string,                                                                #
Optional / Default is empty
      disable               => "true"|"false"                                                          #
Optional / Default "false"
      extattrs              => { $string => $extattr, ... },                                           #
Optional / Default is undefined
      extensible_attributes => { $string => $string|$num, $string => [ $string|$num, ...], ... },     #
Optional / Default is undefined
      ttl                   => $num|undef,                                                             #
Optional / Default is undefined
      views                 => [$View1],                                                               #
Optional / Default is "default" view
      creator               => "DYNAMIC" | "STATIC",                                                   #
Optional / Default is "STATIC"
      ddns_protected        => "true" | "false",                                                       #
Optional / Default is "false"
      ddns_principal        => $string,                                                                #
Optional / Default is undefined
                                                                                                       #
```

```
    forbid_reclamation     => "true" | "false",
Optional / Default is "false"
  );
```

You cannot set both extattrs and extensible_attributes attributes at the same time.

SESSION METHODS

This section describes all the methods in Infoblox::Session module that you can apply to a CNAME record object.

Infoblox::Session->add( )

Use this method to add an object to the Infoblox appliance. See Infoblox::Session->add() for parameters and return values.

Example

```
#Construct an object
my $bind_cname = Infoblox::DNS::Record::CNAME->new (
    name      => "bind_cname.domain.com",
    canonical => "cname.domain.com",
    comment   => "this is a demo bind_cname record",
    disable   => "true",
    ttl       => "33",
    views     => [$view1]  );
# Submit for addition
my $response = $session->add( $bind_cname );
```

Infoblox::Session->get( )

Use this method to retrieve all the matching objects from the Infoblox appliance. See Infoblox::Session->get() for parameters and return values.

Key References

```
Apply the following attributes to get a specific DNS CNAME object:

  name                 - Optional. A host name in string format.
  view                 - Optional. The DNS view in which the CNAME record is located. By default,
all DNS views are searched. But if you omit this attribute and specify a zone, the appliance searches
the 'default' view only.
  zone                 - Optional. A zone name in FQDN format.
  canonical            - Optional. A canonical name.
  creator              - Optional. Record creator.
  ddns_principal       - Optional. GSS-TSIG principal that owns the record.
  reclaimable          - Optional. The flag that indicates whether the record is reclaimable or not.
  extattrs             - Optional. A hash reference containing extensible attributes.
  extensible_attributes - Optional. A hash reference containing extensible attributes.
```

Example

```
my @retrieved_objs = $session->get(
    object => "Infoblox::DNS::Record::CNAME",
    name   => "bind_cname.domain.com",
    view   => "default" );

# get all DNS CNAME record in zone "domain.com" of default view
my @retrieved_objs = $session->get(
    object => "Infoblox::DNS::Record::CNAME",
    zone   => "domain.com",
    view   => "default" );

# get DNS CNAME record with canonical name "cname.domain.com" of default view
my @retrieved_objs = $session->get(
    object => "Infoblox::DNS::Record::CNAME",
    canonical => "cname.domain.com",
    view   => "default" );

# get all DNS CNAME record with a given extensible attribute
my @retrieved_objs = $session->get(
    object => "Infoblox::DNS::Record::CNAME",
    extensible_attributes => { 'Site' => 'Santa Clara' });
```

Infoblox::Session->modify( )

Use this method to modify an object in the Infoblox appliance. See Infoblox::Session->modify() for parameters and return values.

Example

```
# Use method to modify the comment.
$bind_cname->comment("this is a modified comment");
# Submit modification
my $response = $session->modify( $bind_cname );
```

Infoblox::Session->remove( )

Use this method to remove an object from the Infoblox appliance. See Infoblox::Session->remove() for parameters and return values.

To remove a specific object, first use `get()` or `search()` to retrieve the specific object, and then submit this object for removal.

Example

```
#Get the objects with the same name
my @retrieved_objs = $session->get(
  object => "Infoblox::DNS::Record::CNAME",
  name   => "bind_cname.domain.com",
  view   => "default");
#Find the desired object from the retrieved list.
my $desired_cname = $retrieved_objs[0];
# Submit for removal
my $response = $session->remove( $desired_cname );
```

Infoblox::Session->search( )

Use this method to search for DNS CNAME record objects in the Infoblox appliance. See Infoblox::Session->search() for parameters and return values.

Key References

```
 Apply the following attributes to search for a specific DNS CNAME object.

  name                 - Optional. A host name in string format (regular expression).
  view                 - Optional. The DNS view in which the CNAME record is located. By default,
all DNS views are searched. But if you omit this attribute and specify a zone, the appliance searches
the 'default' view only.
  zone                 - Optional. A zone name in FQDN format.
  canonical            - Optional. A canonical name (regular expression).
  comment              - Optional. A comment in string format (regular expression).
  creator              - Optional. Record creator.
  ddns_principal       - Optional. GSS-TSIG principal that owns the record.
  reclaimable          - Optional. The flag that indicates whether the record is reclaimable or not.
  extattrs             - Optional. A hash reference containing extensible attributes.
  extensible_attributes - Optional. A hash reference containing extensible attributes.
```

For more information about searching extensible attributes, see Infoblox::Grid::ExtensibleAttributeDef/Searching Extensible Attributes.

Example

```
# search for all DNS CNAME objects that match "domain.com" in the default DNS view
my @retrieved_objs = $session->search(
    object  => "Infoblox::DNS::Record::CNAME",
    name    => 'domain\.com',
    view    => "default",
    comment => "this is a modified comment" );

# search for all DNS CNAME record in zone "domain.com" of default view
my @retrieved_objs = $session->search(
    object => "Infoblox::DNS::Record::CNAME",
    zone   => "domain.com",
    view   => "default" );

# search DNS CNAME record with canonical name "cname.domain.com" of default view
my @retrieved_objs = $session->search(
    object => "Infoblox::DNS::Record::CNAME",
    canonical => "cname.domain.com",
    view   => "default" );

# search all DNS CNAME recods with the extensible attribute 'Site'
my @retrieved_objs = $session->search(
   object => "Infoblox::DNS::Record::CNAME",
   extensible_attributes => { 'Site' => 'Santa Clara' });
```

METHODS

This section describes all the methods that you can use to configure and retrieve the attribute values of a CNAME record.

canonical( )

Use this method to set or retrieve the canonical (or official) name of the host.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

The attribute value can be in unicode format.

Parameter

Canonical name in FQDN (Fully Qualified Domain Name) format. The FQDN consists of the hostname followed by the domain name (example: abc.com). A hostname can have a maximum of 256 bytes.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get canonical
my $canonical = $bind_cname->canonical();
#Modify canonical name
$bind_cname->canonical("bind_cname.domain.com");
```

comment( )

Use this method to set or retrieve the descriptive comment.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

Desired comment in string format with a maximum of 256 bytes.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get comment
my $comment = $bind_cname->comment();
#Modify comment
$bind_cname->comment("Modifying the DNS CNAME comment");
```

cloud_info( )

Use this method to retrieve cloud API related information for the Infoblox::DNS::Record::CNAME object.

Parameter

None

Returns

The method returns the attribute value.

Example

```
# Get cloud_info
my $cloud_info = $object->cloud_info();
```

creator( )

Use this method to set or retrieve the record creator.

Note that changing creator from or to 'SYSTEM' value is not allowed.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Paramter

The valid values are 'STATIC' and 'DYNAMIC'. The default value is 'STATIC'.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get creator value
my $creator = $object->creator();

#Modify creator value
$object->creator("DYNAMIC");
```

creation_time( )

Use this method to retrieve the creation time for the record. This is a read-only attribute.

Parameter

None

Returns

The valid return value is a number of seconds that have elapsed since January 1st, 1970 UTC.

Example

```
#Get creation_time value
my $creation_time = $object->creation_time();
```

ddns_principal( )

Use this method to set or retrive the GSS-TSIG principal that owns this record.

Note that you cannot set ddns_principal for 'STATIC' and 'SYSTEM' records.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Paramter

The GSS-TSIG principal FQDN (Fully Qualified Domain Name) format. The FQDN consists of the hostname followed by the domain name (example: abc.com). A hostname can have a maximum of 256 characters.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get ddns_principal value
my $ddns_principal = $object->ddns_principal();

#Modify ddns_principal value
$object->ddns_principal('foo.com');
```

ddns_protected( )

Use this method to set or retrieve the flag that indicates whether DDNS updates for this record are allowed or not.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

Specify 'true' to protect record from DDNS updates and 'false' to allow DDNS updates for the specified record.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get ddns_protected value
my $ddns_protected = $object->ddns_protected();

#Modify ddns_protected value
$object->ddns_protected('true');
```

disable( )

Use this method to set or retrieve the disable flag of a DNS record.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

The default value for this field is false. The DNS record is enabled.

Parameter

Specify "true" to set the disable flag or "false" to deactivate/unset it.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get disable
my $disable = $bind_cname->disable();
#Modify disable
$bind_cname->disable("true");
```

dns_name( )

Use this method to retrieve the host name in punycode format. This is a read-only attribute.

Parameter

None

Returns

The method returns the attribute value.

Example

```
# Get attribute value
my $value = $bind_cname->dns_name();
```

dns_canonical( )

Use this method to retrieve the canonical (or official) name, in punycode format, of the host. This is a read-only attribute.

Parameter

None

Returns

The method returns the attribute value.

Example

```
# Get attribute value
my $value = $bind_cname->dns_canonical();
```

extattrs( )

Use this method to set or retrieve the extensible attributes associated with a DNS CNAME record object.

Parameter

Valid value is a hash reference containing the names of extensible attributes and their associated values (Infoblox::Grid::Extattr objects).

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get extattrs
my $ref_extattrs = $bind_cname->extattrs();
```

```
#Modify extattrs
$bind_cname->extattrs({ 'Site' => $extattr1, 'Administrator' => $extattr2 });
```

extensible_attributes( )

Use this method to set or retrieve the extensible attributes associated with a DNS CNAME record.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

For valid values for extensible attributes, see Infoblox::Grid::ExtensibleAttributeDef/Extensible Attribute Values.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get extensible attributes
my $ref_extensible_attributes = $bind_cname->extensible_attributes();
#Modify extensible attributes
$bind_cname->extensible_attributes({ 'Site' => 'Santa Clara', 'Administrator' => [ 'Peter', 'Tom' ]
});
```

forbid_reclamation( )

Use this method to set or retrieve the flag that indicates whether the reclamation is allowed for the record or not.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

Specify 'true' to forbid reclamation for the record and 'false' to allow it. The default value is 'false'.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get forbid_reclamation
my $forbid_reclamation = $object->forbid_reclamation();
#Modify forbid_reclamation
$object->forbid_reclamation('true');
```

last_queried( )

Use this method to retrieve the time when the associated record was last queried. This is a read-only attribute.

Parameter

None

Returns

The method returns the attribute value. The number of seconds that have elapsed since January 1st, 1970 UTC.

Example

```
#Get last_queried
my $last_queried = $bind_cname->last_queried();
```

name( )

Use this method to set or retrieve the host name.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

The attribute value can be in unicode format.

Parameter

A name in FQDN (Fully Qualified Domain Name) format. The FQDN consists of the host_name followed by the domain name

(example: abc.com). A host_name can have a maximum of 256 bytes.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get name
my $name = $bind_cname->name();
#Modify name
$bind_cname->name("bind_cname2.domain.com");
```

reclaimable( )

Use this method to retrieve the flag that indicates whether the record is reclaimable or not.

Parameter

None

Returns

The method returns the attribute value.

Example

```
#Get reclaimable
my $reclaimable = $object->reclaimable();
```

ttl( )

Use this method to set or retrieve the Time to Live (TTL) value.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

The default value is undefined which indicates that the record inherits the TTL value of the zone.

Specify a TTL value to override the TTL value at the zone level.

Parameter

A 32-bit integer (range from 0 to 4294967295) that represents the duration, in seconds, that the record is cached. Zero indicates that the record should not be cached.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get ttl
my $ttl = $bind_cname->ttl();
#Modify ttl
$bind_cname->ttl(1800);
#Un-override ttl
$bind_cname->ttl(undef);
```

views( )

Use this method to set or retrieve the view of the CNAME record.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

The default value is the "default" view, which means the CNAME record is located under the default view.

Parameter

Array reference of defined Infoblox::DNS::View objects.

Note that the array size must be 1.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get views
my $ref_views = $bind_cname->views();
#Modify views, list of Infoblox::DNS::View objects
$bind_cname->views([$view1]);
```

zone( )

Use this method to retrieve the zone name of a DNS CNAME record. This method is read-only and cannot be set.

Parameter

None

Returns

Returns the attribute value.

Example

```
# Get zone
my $zone = $bind_cname->zone();
```

SAMPLE CODE

The following sample code demonstrates the different functions that can be applied to an object, such as add, search, modify, and remove. This sample code also includes error handling for the operations.

#Preparation prior to a DNS CNAME record insertion

```
#PROGRAM STARTS: Include all the modules that will be used
use strict;
use Infoblox;

#Create a session to the Infoblox appliance
my $session = Infoblox::Session->new(
    master   => "192.168.1.2",
    username => "admin",
    password => "infoblox"
);
unless ($session) {
   die("Construct session failed: ",
       Infoblox::status_code() . ":" . Infoblox::status_detail());
}
print "Session created successfully\n";

#Create the zone prior to a CNAME record insertion
my $zone = Infoblox::DNS::Zone->new(name => "domain.com");
unless ($zone) {
   die("Construct zone failed: ",
       Infoblox::status_code() . ":" . Infoblox::status_detail());
}
print "Zone object created successfully\n";

#Verify if the zone exists
my $object = $session->get(object => "Infoblox::DNS::Zone", name => "domain.com");
unless ($object) {
   print "Zone does not exist on server, safe to add the zone\n";
   $session->add($zone)
      or die("Add zone failed: ",
             $session->status_code() . ":" . $session->status_detail());
}
print "Zone added successfully\n";
```

#Create a DNS CNAME record

```
my $view1 = Infoblox::DNS::View->new(
    name      => "default" );

#Construct a DNS CNAME object
my $bind_cname = Infoblox::DNS::Record::CNAME->new(
    name      => "bind_cname.domain.com",
    canonical => "cname.domain.com",
```

```
    comment   => "this is a demo bind_cname record",
    ttl       => "33",
    views     => [$view1]);
unless ($bind_cname) {
   die("Construct DNS record CNAME failed: ",
       Infoblox::status_code() . ":" . Infoblox::status_detail());
}
print "DNS CNAME object created successfully\n";

#Add the DNS CNAME record object to the Infoblox appliance through a session
$session->add($bind_cname)
    or die("Add record cname failed: ",
            $session->status_code() . ":" . $session->status_detail());
print "DNS CNAME object added to server successfully\n";
```

#Search for a specific DNS CNAME record

```
#Search all CNAME records that match "domain.com"
my @retrieved_objs = $session->search(
    object => "Infoblox::DNS::Record::CNAME",
    name   => 'domain\.com'
);
my $object = $retrieved_objs[0];
unless ($object) {
    die("Search record CNAME failed: ",
        $session->status_code() . ":" . $session->status_detail());
}
print "Search DNS CNAME object found at least 1 matching entry\n";

#Search all CNAME records that start with "bind" and end with ".domain.com"
my @retrieved_objs = $session->search(
    object => "Infoblox::DNS::Record::CNAME",
    name   => '^bind.*\.domain\.com\$'
);
my $object = $retrieved_objs[0];
unless ($object) {
    die("Search record CNAME failed: ",
        $session->status_code() . ":" . $session->status_detail());
}
print "Search DNS CNAME object using regexp found at least 1 matching entry\n";
```

#Get and modify a DNS CNAME record

```
#Get CNAME record through the session
my @retrieved_objs = $session->get(
    object => "Infoblox::DNS::Record::CNAME",
    name   => "bind_cname.domain.com",
    view   => "default"
);
my $object = $retrieved_objs[0];
unless ($object) {
    die("Get record CNAME failed: ",
        $session->status_code() . ":" . $session->status_detail());
}
print "Get DNS CNAME object found at least 1 matching entry\n";

#Modify one of the attributes of the specified CNAME record
$object->disable("true");

#Apply the changes
$session->modify($object)
    or die("Modify record CNAME failed: ",
            $session->status_code() . ":" . $session->status_detail());
print "DNS CNAME object modified successfully \n";
```

#Remove a DNS CNAME record

```
#Get CNAME record through the session
my @retrieved_objs = $session->get(
    object => "Infoblox::DNS::Record::CNAME",
    name   => "bind_cname.domain.com",
    view   => "default"
);
$object = $retrieved_objs[0];
unless ($object) {
    die("Get record CNAME failed: ",
        $session->status_code() . ":" . $session->status_detail());
```

```
}
print "Get DNS CNAME object found at least 1 matching entry\n";

#Submit the object for removal
$session->remove( $object )
  or die("Remove record CNAME failed: ",
         $session->status_code() . ":" . $session->status_detail());
print "DNS CNAME object removed successfully \n";

####PROGRAM ENDS####
```

AUTHOR

Infoblox Inc. http://www.infoblox.com/

SEE ALSO

Infoblox::DNS::View, Infoblox::DNS::Zone, Infoblox::Session, Infoblox::Session->get(), Infoblox::Session->search(), Infoblox::Session->add(), Infoblox::Session->remove(), Infoblox::Session->modify()

COPYRIGHT

Copyright (c) 2017 Infoblox Inc.

Infoblox::DNS::Record::DNAME - DNAME record object.

- NAME
- DESCRIPTION
- CONSTRUCTOR
- SESSION METHODS
  - Infoblox::Session->add( )
  - Infoblox::Session->get( )
  - Infoblox::Session->modify( )
  - Infoblox::Session->remove( )
  - Infoblox::Session->search( )
- METHODS
  - comment( )
  - cloud_info( )
  - creator( )
  - creation_time( )
  - ddns_principal( )
  - ddns_protected( )
  - disable( )
  - dns_name( )
  - dns_target( )
  - extattrs( )
  - extensible_attributes( )
  - forbid_reclamation( )
  - last_queried( )
  - name( )
  - reclaimable( )
  - target( )
  - ttl( )
  - views( )
  - zone( )
- SAMPLE CODE
- AUTHOR
- SEE ALSO
- COPYRIGHT

NAME

Infoblox::DNS::Record::DNAME - DNAME record object.

DESCRIPTION

A DNAME record maps all the names in one domain to those in another domain, essentially substituting one domain name suffix with the other.

Note that you must specify only one view for the attribute "views".

CONSTRUCTOR

```
 my $bind_dname = Infoblox::DNS::Record::DNAME->new (
     name                  => $fqdn,                                                    # Required
     target                => $fqdn,                                                    # Required
     comment               => $string,                                                  # Optional
/ Defauilt is empty
     disable               => "true" | "false",                                         # Optional
/ Default "false"
     extattrs              => { $string => $extattr, ... },                             # Optional
/ Default is undefined
     extensible_attributes => { $string => $string|$num, $string => [ $string|$num, ...], ... }, # Optional
/ Default is undefined
     ttl                   => $num | undef,                                             # Optional
/ Default is undefined
     views                 => [$View1],                                                 # Optional
/ Default is "default" view
     creator               => "DYNAMIC" | "STATIC",                                     # Optional
/ Default is "STATIC"
     ddns_protected        => "true" | "false",                                         # Optional
/ Default is "false"
     ddns_principal        => $string,                                                  # Optional
/ Default is undefined
     forbid_reclamation    => "true" | "false",                                         # Optional
/ Default is "false"
```

```
);
```

You cannot set both extattrs and extensible_attributes attributes at the same time.

SESSION METHODS

This section describes all the methods in Infoblox::Session module that you can apply to a DNS DNAME object.

Infoblox::Session->add( )

Use this method to add a DNAME record object to the Infoblox appliance. See Infoblox::Session->add() for parameters and return values.

Example

```
#Construct object
my $bind_dname = Infoblox::DNS::Record::DNAME->new (
    name    => "bind_dname.domain.com",
    comment => "add dname bind_dname.domain.com",
    target  => "target.org",
    disable => "true",
    ttl     => "33",
    views   => [$view1]
    );

# Submit for addition
my $response = $session->add( $bind_dname );
```

Infoblox::Session->get( )

Use this method to retrieve existing DNAME record objects from the Infoblox appliance. See Infoblox::Session->get() for parameters and return values.

Key Reference

```
Apply the following attributes to get a specific DNS DNAME object:

  name                - Optional. A domain name in string format (regular expression).
  view                - Optional. The DNS view in which the DNAME record is located. By default,
all DNS views are searched. But if you omit this attribute and specify a zone, the appliance searches
the 'default' view only.
  zone                - Optional. A zone name in FQDN format.
  creator             - Optional. Record creator.
  ddns_principal      - Optional. GSS-TSIG principal that owns the record.
  reclaimable         - Optional. The flag that indicates whether the record is reclaimable or not.
  extattrs            - Optional. A hash reference containing extensible attributes.
  extensible_attributes - Optional. A hash reference containing extensible attributes.
```

Example

```
my @retrieved_objs = $session->get(
    object => "Infoblox::DNS::Record::DNAME",
    name   => "bind_dname.domain.com",
    view   => "default"
    );

# get all DNS DNAME record in zone "domain.com" of default view
my @retrieved_objs = $session->get(
    object => "Infoblox::DNS::Record::DNAME",
    name   => "domain.com",
    view   => "default"
    );

# get all DNS DNAME record with a given extensible attribute
my @retrieved_objs = $session->get(
    object => "Infoblox::DNS::Record::DNAME",
    extensible_attributes => { 'Site' => 'Santa Clara' });
```

Infoblox::Session->modify( )

Use this method to modify DNAME record objects in the Infoblox appliance. See Infoblox::Session->modify() for parameters and return values.

Example

```
# Use method to modify the comment.
$bind_dname->comment("this is a modified comment.");
# Submit modification
my $response = $session->modify( $bind_dname );
```

Infoblox::Session->remove( )

Use this method to remove a DNAME record object from the Infoblox appliance. See Infoblox::Session->remove() for parameters and return values.

To remove a specific object, first use `get()` or `search()` to retrieve the specific object, and then submit this object for removal.

Example

```
# Get the objects with the same name
my @retrieved_objs = $session->get(
    object => "Infoblox::DNS::Record::DNAME",
    name   => "bind_dname.domain.com",
    view   => "default" );
# Find the desired object from the retrieved list.
my $desired_dname = $retrieved_objs[0];
# Submit for removal
my $response = $session->remove( $desired_dname );
```

Infoblox::Session->search( )

Use this method to search for existing DNAME record objects in the Infoblox appliance. See Infoblox::Session->search() for parameters and return values.

Key Reference

```
Apply the following attributes to search for a DNS DNAME record object:

  name                  - Optional. A domain name in string format (regular expression).
  view                  - Optional. The DNS view in which the DNAME record is located. By default,
all DNS views are searched. But if you omit this attribute and specify a zone, the appliance searches
the 'default' view only.
  comment               - Optional. A comment in string format (regular expression).
  creator               - Optional. Record creator.
  ddns_principal        - Optional. GSS-TSIG principal that owns the record.
  reclaimable           - Optional. The flag that indicates whether the record is reclaimable or not.
  extattrs              - Optional. A hash reference containing extensible attributes.
  extensible_attributes - Optional. A hash reference containing extensible attributes.
```

For more information about searching extensible attributes, see Infoblox::Grid::ExtensibleAttributeDef/Searching Extensible Attributes.

Example

```
# search for all DNS DNAME objects that match "domain.com" in the default DNS view
my @retrieved_objs = $session->search(
    object => "Infoblox::DNS::Record::DNAME",
    name   => '.*\.domain\.com',
    view   => "default"
    );

# search for all DNS DNAME records in the "domain.com" zone of the default view
my @retrieved_objs = $session->search(
    object => "Infoblox::DNS::Record::DNAME",
    zone   => "domain.com",
    view   => "default"
    );

# search all DNS DNAME recods with the extensible attribute 'Site'
my @retrieved_objs = $session->search(
   object => "Infoblox::DNS::Record::DNAME",
   extensible_attributes => { 'Site' => 'Santa Clara' });
```

METHODS

This section describes all the methods that you can use to configure and retrieve the attribute values of a DNAME record.

comment( )

Use this method to set or retrieve the comment value.

Enter identifying text for this record, such as a meaningful note or reminder.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

Desired comment in string format with a maximum of 256 bytes.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get comment
my $comment = $binddname->comment();
#Modify comment
$binddname->comment("Modifying the DNAME comment");
```

cloud_info( )

Use this method to retrieve cloud API related information for the Infoblox::DNS::Record::DNAME object.

Parameter

None

Returns

The method returns the attribute value.

Example

```
# Get cloud_info
my $cloud_info = $object->cloud_info();
```

creator( )

Use this method to set or retrieve the record creator.

Note that changing creator from or to 'SYSTEM' value is not allowed.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Paramter

The valid values are 'STATIC' and 'DYNAMIC'. The default value is 'STATIC'.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get creator value
my $creator = $object->creator();

#Modify creator value
$object->creator("DYNAMIC");
```

creation_time( )

Use this method to retrieve the creation time for the record. This is a read-only attribute.

Parameter

None

Returns

The valid return value is a number of seconds that have elapsed since January 1st, 1970 UTC.

Example

```
#Get creation_time value
my $creation_time = $object->creation_time();
```

ddns_principal( )

Use this method to set or retrive the GSS-TSIG principal that owns this record.

Note that you cannot set ddns_principal for 'STATIC' and 'SYSTEM' records.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Paramter

The GSS-TSIG principal FQDN (Fully Qualified Domain Name) format. The FQDN consists of the hostname followed by the domain name (example: abc.com). A hostname can have a maximum of 256 characters.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get ddns_principal value
my $ddns_principal = $object->ddns_principal();

#Modify ddns_principal value
$object->ddns_principal('foo.com');
```

ddns_protected( )

Use this method to set or retrieve the flag that indicates whether DDNS updates for this record are allowed or not.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

Specify 'true' to protect record from DDNS updates and 'false' to allow DDNS updates for the specified record.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get ddns_protected value
my $ddns_protected = $object->ddns_protected();

#Modify ddns_protected value
$object->ddns_protected('true');
```

disable( )

Use this method to set or retrieve the disable value.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

The default value for this field is "false", therefore the DNS record is enabled.

Parameter

Specify "true" to set the disable flag or "false" to deactivate/unset it.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get disable
my $disable = $binddname->disable();
#Modify disable
$binddname->disable("true");
```

dns_name( )

Use this method to retrieve the host name in punycode format. This is a read-only attribute.

Parameter

None

Returns

The method returns the attribute value.

Example

```
# Get attribute value
my $value = $binddname->dns_name();
```

dns_target( )

Use this method to retrieve the target in punycode format. This is a read-only attribute.

Parameter

None

Returns

The method returns the attribute value.

Example

```
# Get attribute value
my $value = $binddname->dns_target();
```

extattrs( )

Use this method to set or retrieve the extensible attributes associated with a DNS DNAME record object.

Parameter

Valid value is a hash reference containing the names of extensible attributes and their associated values (Infoblox::Grid::Extattr objects).

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get extattrs
my $ref_extattrs = $binddname->extattrs();
#Modify extattrs
$binddname->extattrs({ 'Site' => $extattr1, 'Administrator' => $extattr2 });
```

extensible_attributes( )

Use this method to set or retrieve the extensible attributes associated with a DNS DNAME record.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

For valid values for extensible attributes, see Infoblox::Grid::ExtensibleAttributeDef/Extensible Attribute Values.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get extensible attributes
my $ref_extensible_attributes = $binddname->extensible_attributes();
#Modify extensible attributes
$binddname->extensible_attributes({ 'Site' => 'Santa Clara', 'Administrator' => [ 'Peter', 'Tom' ]
});
```

forbid_reclamation( )

Use this method to set or retrieve the flag that indicates whether the reclamation is allowed for the record or not.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

Specify 'true' to forbid reclamation for the record and 'false' to allow it. The default value is 'false'.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get forbid_reclamation
my $forbid_reclamation = $object->forbid_reclamation();
#Modify forbid_reclamation
$object->forbid_reclamation('true');
```

last_queried( )

Use this method to retrieve the time when the associated record was last queried. This is a read-only attribute.

Parameter

None

Returns

The method returns the attribute value. The number of seconds that have elapsed since January 1st, 1970 UTC.

Example

```
#Get last_queried
my $last_queried = $binddname->last_queried();
```

name( )

Use this method to set or retrieve the name value.

Enter the name of a sub domain. If you are adding a DNAME record for the entire zone, leave this field empty. This field is for adding a DNAME record for a sub domain within the selected zone.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

The attribute value can be in unicode format.

Parameter

Hostname in FQDN (Fully Qualified Domain Name) format. The FQDN consists of the hostname followed by the domain name (example: abc.com). A hostname can have a maximum of 256 bytes.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get name
my $name = $binddname->name();
#Modify name
$binddname->name("bind_dname1.domain.com");
```

reclaimable( )

Use this method to retrieve the flag that indicates whether the record is reclaimable or not.

Parameter

None

Returns

The method returns the attribute value.

Example

```
#Get reclaimable
my $reclaimable = $object->reclaimable();
```

target( )

Use this method to set or retrieve the target value.

Enter the domain name to which you want to map all the domain names specified in the Domain Name field.

Include the specified parameter to set the attribute value. Must use some parameter to add a DNAME record.

The attribute value can be in unicode format.

Parameter

Text with the target (domain-name) for the DNS DNAME record.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get name
my $target = $binddname->target();
#Modify name
$binddname->target("modified_target.org");
```

ttl( )

Use this method to set or retrieve the Time to Live (TTL) value.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

The default value is undefined; therefore the record inherits the TTL value from the zone level.

With a specified TTL value, the method overrides the zone values with the specified TTL value.

Parameter

A 32-bit integer (range from 0 to 4294967295) that represents the duration in seconds that the record is cached. Zero indicates that the record should not be cached.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get ttl
my $ttl = $binddname->ttl();
#Modify ttl
$binddname->ttl(1800);
#Un-override ttl
$binddname->ttl(undef);
```

views( )

Use this method to set or retrieve the views value.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

The default value is the default view, therefore the DNAME record is referenced under the default view.

Parameter

Array reference of defined Infoblox::DNS::View objects.

Note that the array size must be 1.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get views
my $ref_views = $binddname->views();
#Modify views, list of Infoblox::DNS::View objects
$binddname->views([$view1]);
```

zone( )

Use this method to retrieve the zone name of a DNS DNAME record. This method is read-only and cannot be set.

Parameter

None

Returns

Returns the attribute value.

Example

```
# Get zone
my $zone = $binddname->zone();
```

SAMPLE CODE

The following sample code demonstrates different operations that can be applied to a DNAME record object, such as create, search, modify, and remove an object. This sample code also includes error handling for the operations.

#Preparation prior to a DNAME record insertion

```
#PROGRAM STARTS: Include all the modules that will be used
use strict;
use Infoblox;

#Create a session to the Infoblox appliance
my $session = Infoblox::Session->new(
    master   => "192.168.1.2",
    username => "admin",
    password => "infoblox"
);
unless ($session) {
   die("Construct session failed: ",
       Infoblox::status_code() . ":" . Infoblox::status_detail());
}
print "Session created successfully\n";

#Create the zone prior to a DNAME record insertion
my $zone = Infoblox::DNS::Zone->new(name => "domain.com");
unless ($zone) {
   die("Construct zone failed: ",
       Infoblox::status_code() . ":" . Infoblox::status_detail());
}
print "Zone object created successfully\n";

#Verify if the zone exists
my $object = $session->get(object => "Infoblox::DNS::Zone", name => "domain.com");
unless ($object) {
   print "Zone does not exist on server, safe to add the zone\n";
   $session->add($zone)
      or die("Add zone failed: ",
             $session->status_code() . ":" . $session->status_detail());
}
print "Zone added successfully\n";
```

#Create a DNAME record

```
#Construct a DNAME object
my $bind_dname = Infoblox::DNS::Record::DNAME->new(
    name     => "bind_dname.domain.com",
    target   => "target.org",
    comment  => "this is a demo bind_dname record",
);
unless ($bind_dname) {
   die("Construct DNS record DNAME failed: ",
       Infoblox::status_code() . ":" . Infoblox::status_detail());
}
print "DNS DNAME object created successfully\n";
```

```
#Add the DNS DNAME record object to Infoblox appliance through a session
$session->add($bind_dname)
    or die("Add record DNAME failed: ",
            $session->status_code() . ":" . $session->status_detail());
print "DNS DNAME object added to server successfully\n";
```

#Search for a specific DNS DNAME record

```
#Search all DNAME records in the zone
my @retrieved_objs = $session->search(
    object => "Infoblox::DNS::Record::DNAME",
    name   => 'domain\.com'
);
my $object = $retrieved_objs[0];
unless ($object) {
    die("Search record DNAME failed: ",
        $session->status_code() . ":" . $session->status_detail());
}
print "Search DNS DNAME object found at least 1 matching entry\n";

#Search all DNAME records that start with "bind" and end with "domain.com"
my @retrieved_objs = $session->search(
    object => "Infoblox::DNS::Record::DNAME",
    name   => '^bind.*\.domain.com\$'
);
my $object = $retrieved_objs[0];
unless ($object) {
    die("Search record DNAME failed: ",
        $session->status_code() . ":" . $session->status_detail());
}
print "Search DNS DNAME object using regexp found at least 1 matching entry\n";
```

#Get and modify a DNS DNAME record

```
#Get DNAME record through the session
my @retrieved_objs = $session->get(
    object => "Infoblox::DNS::Record::DNAME",
    name   => "bind_dname.domain.com",
    target => "target.org"
);
my $object = $retrieved_objs[0];
unless ($object) {
    die("Get record DNAME failed: ",
        $session->status_code() . ":" . $session->status_detail());
}
print "Get DNS DNAME object found at least 1 matching entry\n";

#Modify one of the attributes of the specified DNAME record
$object->comment("This is a modified comment.");

#Apply the changes
$session->modify($object)
    or die("Modify record DNAME failed: ",
            $session->status_code() . ":" . $session->status_detail());
print "DNS DNAME object modified successfully \n";
```

#Remove a DNS DNAME record

```
#Get DNAME record through the session
my @retrieved_objs = $session->get(
    object => "Infoblox::DNS::Record::DNAME",
    name   => "bind_dname.domain.com"
);
my $object = $retrieved_objs[0];
unless ($object) {
    die("Get record DNAME failed: ",
        $session->status_code() . ":" . $session->status_detail());
}
print "Get DNS DNAME object found at least 1 matching entry\n";

#Submit the object for removal
$session->remove($object)
    or die("Remove record DNAME failed: ",
        $session->status_code() . ":" . $session->status_detail());
print "DNS DNAME object removed successfully \n";

####PROGRAM ENDS####
```

AUTHOR

Infoblox Inc. http://www.infoblox.com/

SEE ALSO

Infoblox::DNS::View, Infoblox::DNS::Zone, Infoblox::Session, Infoblox::Session->get(), Infoblox::Session->search(), Infoblox::Session->add(), Infoblox::Session->remove(), Infoblox::Session->modify()

COPYRIGHT

Copyright (c) 2017 Infoblox Inc.

Infoblox::DHCP::View - DHCP Network View object

- NAME
- DESCRIPTION
- CONSTRUCTOR
- SESSION METHODS
  - Infoblox::Session->add( )
  - Infoblox::Session->get( )
  - Infoblox::Session->modify( )
  - Infoblox::Session->remove( )
  - Infoblox::Session->search( )
- METHODS
  - associated_dns_views( )
  - cloud_info( )
  - comment( )
  - ddns_zone_primaries( )
  - extensible_attributes( )
  - external_ddns_primaries( )
  - mgm_private( )
  - modify_vrf_assignment()
  - internal_ddns_zones( )
  - internal_updates_view( )
  - ms_ad_user_data( )
  - name( )
- SAMPLE CODE
- AUTHOR
- SEE ALSO
- COPYRIGHT

NAME

Infoblox::DHCP::View - DHCP Network View object

DESCRIPTION

Network view provides the ability to manage multiple views of IP address spaces from NIOS. These address spaces could be overlapping or non-overlapping. When the network view is not specified during a write operation, the objects are automatically associated with the system-defined default network view.

A network view is a single routing domain, with its own networks and shared networks. You can manage the networks in one network view independently of the other network views.

CONSTRUCTOR

```
  my $network = Infoblox::DHCP::View->new(
     name                        => $string,
#Required
     comment                     => $string,
#Optional / Default is undefined
     ddns_zone_primaries         => [$ddns1, $ddns2, ... ],
#Optional / Default is undefined
     mgm_private                 => 'true' | 'false',
#Optional / Default is 'false'
     extensible_attributes       => { $string => $string|$num, $string => [ $string|$num, ...], ... },
#Optional / Default is undefined
     external_ddns_primaries     => [ $Nameserver1,$Nameserver2,...],
#Optional / Default is undefined
     internal_ddns_primaries     => [ $ForwardZone1,$ForwardZone2,...],
#Optional / Default is undefined
   );
```

SESSION METHODS

This section describes all the methods in an Infoblox::Session module that can apply to a DHCP Network View object.

Infoblox::Session->add( )

Use this method to add a network view object to the Infoblox appliance. See Infoblox::Session->add() for parameters and return values.

Example

```
#Construct an object
my $network_view = Infoblox::DHCP::View->new(
       name => "netview",
);

# Submit for adding network view
my $response = $session->add( $network_view );
```

Infoblox::Session->get( )

Use this method to retrieve all the matching network view objects from the Infoblox appliance. See Infoblox::Session->get() for parameters and return values.

Key References

```
Apply the following attributes to get a specific DHCP Network View object:

 name                  - Required. A name in string format.
 extensible_attributes - Optional. A hash reference containing extensible attributes.
```

Example

```
 my  @result_array = $session->get(
      object        => "Infoblox::DHCP::View" ,
      name          => "netview"
 );

# get all DHCP view objects with a given extensible attribute
my @retrieved_objs = $session->get(
   object => "Infoblox::DHCP::View",
   extensible_attributes => { 'Site' => 'Santa Clara' });
```

Infoblox::Session->modify( )

Use this method to modify a network object in the Infoblox appliance. See Infoblox::Session->modify() for parameters and return values.

Example

```
#Use method to modify comment method.
$object->comment("My modified comment");
# Submit modification
my $response = $session->modify( $object );
```

Infoblox::Session->remove( )

Use this method to remove an object from the Infoblox appliance. See Infoblox::Session->remove() for parameters and return values.

To remove a specifc object, first use `get()` or `search()` to retrieve the specific object, and then submit this object for removal.

Example

```
# Get the object under the same name
my  @result_array = $session->get(
      object        => "Infoblox::DHCP::View" ,
      name          => "netview"
 );

# find the desired object from the retrieved list.
my $desired_network_view = $result_array[0];

# Submit for removal
my $response = $session->remove( $desired_network_view );
```

Infoblox::Session->search( )

Use this method to search for a DHCP Network View object in the Infoblox appliance. See Infoblox::Session->search() for parameters and return values.

Key References

```
Apply the following attributes to search for a DHCP Network View object:

 name                  - Required. A name in string format (regular expression).
 extensible_attributes - Optional. A hash reference containing extensible attributes.
```

For more information about searching extensible attributes, see Infoblox::Grid::ExtensibleAttributeDef/Searching Extensible Attributes.

Example

```
# search for DHCP Network View object with name 'netview'
    my @retrieved_objs = $session->search(
      object       => "Infoblox::DHCP::Network",
      name         => "^netview\$"
    );

# search for DHCP views with the extensible attribute 'Site'
my @retrieved_objs = $session->search(
   object => "Infoblox::DHCP::View",
   extensible_attributes => { 'Site' => 'Santa Clara' });
```

METHODS

This section describes all the methods that can be used to configure and retrieve the attribute values of a DHCP Network View object

associated_dns_views( )

Use this method to retrieve the list of DNS views associated with this network view. This list is read-only and cannot be set.

Parameter

None

Returns

Returns an array reference that contains Infoblox::DNS:View object(s).

Example

```
# Get associated_dns_views
my $associated_dns_views = $network_view->associated_dns_views()
```

cloud_info( )

Use this method to set or retrieve cloud API related information for the Infoblox::DHCP::View object.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

The valid value is a Infoblox::Grid::CloudAPI::Info object.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
# Get cloud_info
my $cloud_info = $network_view->cloud_info();
# Modify cloud_info
$network_view->cloud_info($cloud_info_object);
```

comment( )

Use this method to set or retrieve a descriptive comment of a DHCP Network View object.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

Enter a descriptive comment for the network view in string format with a maximum of 256 bytes.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
# Get comment
my $comment = $network_view->comment();
# Modify comment
```

```
$network_view->comment("add a network view");
```

ddns_zone_primaries( )

Use this method to set or retrieve an ordered list of zone primaries that will receive DDNS updates.

Parameter

A valid value is a reference to an array of Infoblox::DHCP::DDNS objects.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
# Get the attribute value
$value = $network_view->ddns_zone_primaries();
# Modify the attribute value
$network_view->ddns_zone_primaries([$ddns]);
```

extensible_attributes( )

Use this method to set or retrieve the extensible attributes associated with a DHCP view object.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

For valid values for extensible attributes, see Infoblox::Grid::ExtensibleAttributeDef/Extensible Attribute Values.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get extensible attributes
my $ref_extensible_attributes = $firstview->extensible_attributes();
#Modify extensible attributes
$firstview->extensible_attributes({ 'Site' => 'Santa Clara', 'Administrator' => [ 'Peter', 'Tom' ]
});
```

external_ddns_primaries( )

Use this method to set or retrieve the network view-level external_ddns_primaries value. This specifies the external zone that is updated dynamically. The DHCP server can send dynamic updates to a specified external name server.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

A valid value is an array reference that contains Infoblox::DNS::Nameserver object(s).

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
# Get external_ddns_primaries
my $external_ddns_primaries = $network_view->external_ddns_primaries();

# Modify external_ddns_primaries
my $nameserver1 = Infoblox::DNS::Nameserver->new(
        ddns_zone => "ns1.del.com",
        ipv4addr  => "11.11.11.11",
    );
my $nameserver2 = Infoblox::DNS::Nameserver->new(
         ddns_zone => "3.2.1.in-addr.arpa",
         ipv4addr  => "22.22.22.22",
    );
```

```
# Nameserver with TSIG security
my $nameserver3 = Infoblox::DNS::Nameserver->new(
        ddns_zone     => 'grid_level.local',
        ipv4addr      => '10.102.30.122',
        TSIGalgorithm => 'HMAC-SHA256',
        TSIGname      => 'kk',
        TSIGkey       => '+NPBBVYjoNC32hfj8vepUw==',
);

# Nameserver with GSS-TSIG security
my $nameserver4 = Infoblox::DNS::Nameserver->new(
        ipv4addr               => '10.102.30.122',
        gss_tsig_domain        => 'AD-31.LOCAL',
        ddns_zone              => 'grid1_level.local',
        gss_tsig_dns_principal => 'DNS/w8s232-29.ad-31.local@AD-31.LOCAL'
);

$network_view->external_ddns_primaries([$nameserver1, $nameserver2, $nameserver3, $nameserver4]);

# Clear external_ddns_primaries list
$network_view->external_ddns_primaries([]);
```

mgm_private( )

Use this method to set or retrieve the flag that controls whether the object should be synced to the MGM or not.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

Specify 'true' to enable synchronization of the managed Grid with the MGM or 'false' to forbid it. The default value is 'false'.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get mgm_private value
my $value = $object->mgm_private();

#Modify mgm_private value
$object->mgm_private('true');
```

modify_vrf_assignment()

This method is used to assign (unassign) a list of the virtual networks (VRFs) to the network view.

Parameters

VRF membership modification mode: ASSIGN or UNASSIGN.

A valid list (array reference) that contains Infoblox::Grid::Member::Discovery::VRF object(s).

Returns

Returns true on success operation, otherwise false.

Example

```
# Get Network View
my $default_view = $session->get(object => 'Infoblox::DHCP::View', name => 'default');

# Get Infoblox::Grid::Member::Discovery::VRF objects
my $vrf1 = $session->get(object => 'Infoblox::Grid::Member::Discovery::VRF', name =>
'VirtualNetworkMember1');
my $vrf2 = $session->get(object => 'Infoblox::Grid::Member::Discovery::VRF', name =>
'VirtualNetworkMember2');

# Assign a list of the VRFs to the Network View
my $assign_result = $default_view->modify_vrf_assignment(
       vrfs => [ $vrf1, $vrf2 ],
       mode => 'ASSIGN'
);

# Unassign VRF from Network View
```

```
my $unassign_result = $default_view->modify_vrf_assignment(
        vrfs => [ $vrf2 ],
        mode => 'UNASSIGN'
);

# Check of VRF assignment via Infoblox::Grid::Member::Discovery::VRF object
$vrf = $session->get(object => 'Infoblox::Grid::Member::Discovery::VRF', name =>
'VirtualNetworkMember1');

# Get current network view name associated with VRF
print $vrf->network_view;
```

internal_ddns_zones( )

Use this method to set or retrieve forward mapping zones which are being served in the grid. Only an authoritative zone can be assigned to this list. All other types of zones will generate an error.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

A valid value is an array reference that contains Infoblox::DNS::Zone object(s).

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
# Get internal_ddns_zones
my $internal_ddns_zones = $network_view->internal_ddns_zones();

# Modify internal_ddns_zones
my $nameserver1 = Infoblox::DNS::Nameserver->new(
        name     => "ns1.del.com",
        ipv4addr => "11.11.11.11",
    );
my $nameserver2 = Infoblox::DNS::Nameserver->new(
        name     => "test",
        ipv6addr => "2006::b671",
    );
my $zone1 = Infoblox::DNS::Zone->new(
        name     => "test1.com",
        primary  => [$nameserver1],
        secondaries => [$nameserver2],
    );
my $zone2 = Infoblox::DNS::Zone->new(
        name     => "test2.com",
    );
$network_view->internal_ddns_zones([$zone1, $zone2]);

# Clear internal_ddns_zones list
$network_view->internal_ddns_zones([]);
```

internal_updates_view( )

Use this method to specify or retrieve the internal updates DNS view value of a network view. When you enable the appliance to send updates to grid members, you must specify which DNS view receives the updates.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

A valid value is an Infoblox::DNS::View object for internal dynamic updates.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
# Get internal_updates_view
my $internal_updates_view = $network_view->internal_updates_view();
# Modify internal_updates_view
```

```
my $otherview = Infoblox::DNS::View->new(
        name => "other",
    );
$network_view->internal_updates_view($otherview);
```

ms_ad_user_data( )

Use this method to retrieve Microsoft Active Directory users related information. This is a read-only attribute.

Parameter

None

Returns

The valid return value is an Infoblox::Grid::MSServer::AdUser::Data object.

Example

```
#Get ms_ad_user_data
my $ms_ad_user_data = $view->ms_ad_user_data();
```

name( )

Use this method to set or retrieve the name of a DHCP Network View object.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

The name of the network view is in string format with a maximum of 64 bytes.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
# Get name
my $name = $network_view->name()
# Modify name
$network_view->name("netview_renamed");
```

SAMPLE CODE

The following sample code demonstrates the different functions that can be applied to an object such as add, search, modify, and remove. This sample also includes error handling for the operations.

#Preparation prior to a DHCP Network View insertion

```
#PROGRAM STARTS: Include all the modules that will be used
use strict;
use Infoblox;

#Create a session to the Infoblox appliance

my $session = Infoblox::Session->new(
    master   => "192.168.1.2",
    username => "admin",
    password => "infoblox"
);
unless ($session) {
   die("Construct session failed: ",
       Infoblox::status_code() . ":" . Infoblox::status_detail());
}
print "Session created successfully\n";
```

#Create a DHCP Network View object

```
my $network_view = Infoblox::DHCP::View->new(
           name     => "netview",
          );

my $response = $session->add($network_view)
       or die("Add Network View failed: ",
```

```
            $session->status_code() . ":" . $session->status_detail());

print "Network View added successfully\n";
```

#Search for a specific Network View

```
#Search the network view
my @retrieved_objs = $session->search(
    object  => "Infoblox::DHCP::View",
    name => '^netview$',
);
my $object = $retrieved_objs[0];
unless ($object) {
    die("Search Network View failed: ",
        $session->status_code() . ":" . $session->status_detail());
}
print "Search Network found at least 1 matching entry\n";
```

#Get and modify a Network View attribute

```
#Get Network View through the session
my  @retrieved_objs = $session->get(
           object => "Infoblox::DHCP::View" ,
           name   => "netview",
        );

my $object = $retrieved_objs[0];

unless ($object) {
    die("Get Network View object failed: ",
        $session->status_code() . ":" . $session->status_detail());
}
print "Get Network View found at least 1 matching entry\n";

#Modify one of the attributes of the specified Network View attribute
$object->comment("updated comment"); #Apply the changes

$session->modify($object)
    or die("Modify Network View attribute failed: ",
           $session->status_code() . ":" . $session->status_detail());

print "Network View object modified successfully \n";
```

#Remove a Network View Object

```
 #Get Network View object through the session
 my @retrieved_objs = $session->get(
           object  => "Infoblox::DHCP::View" ,
           name    => "netview",
       );

   my $object = $retrieved_objs[0];

   unless ($object) {
    die("Get Network View object failed: ",
        $session->status_code() . ":" . $session->status_detail());
    }

print "Get Network View object found at least 1 matching entry\n";

#Submit the object for removal
$session->remove($object)
    or die("Remove Network View failed: ",
        $session->status_code() . ":" . $session->status_detail());

print "Network View removed successfully \n";

####PROGRAM ENDS####
```

AUTHOR

Infoblox Inc. http://www.infoblox.com/

SEE ALSO

Infoblox::Session, Infoblox::DNS::View, Infoblox::DHCP::Network, Infoblox::DNS::Zone, Infoblox::DNS::Nameserver, Infoblox::Session->get(), Infoblox::Session->search(), Infoblox::Session->add(), Infoblox::Session->remove(), Infoblox::Session->modify(), Infoblox::DHCP::DDNS, Infoblox::Grid::MSServer::AdUser::Data

---

COPYRIGHT

Copyright (c) 2017 Infoblox Inc.

Infoblox::Grid::Member::DHCP - DHCP member object.

- NAME
- DESCRIPTION
- CONSTRUCTOR
- SESSION METHODS
  - Infoblox::Session->get( )
  - Infoblox::Session->modify( )
- METHODS
  - auth_server_group( )
  - authority( )
  - authn_captive_portal
  - authn_captive_portal_authenticated_filter
  - authn_captive_portal_enabled
  - authn_captive_portal_guest_filter
  - auth_server_group_enabled
  - bootfile( )
  - bootserver( )
  - ddns_generate_hostname( )
  - ddns_retry_interval( )
  - ddns_server_always_updates( )
  - ddns_ttl( )
  - ddns_domainname( )
  - ddns_update_fixed_addresses( )
  - ddns_use_option81( )
  - ddns_zone_primaries( )
  - deny_bootp( )
  - dns_update_style( )
  - email_list( )
  - enable_ddns( )
  - enable_dhcp( )
  - enable_dhcpv6_service( )
  - enable_dhcp_on_lan2( )
  - enable_dhcp_on_ipv6_lan2( )
  - enable_dhcp_thresholds( )
  - enable_email_warnings( )
  - enable_fingerprint( )
  - enable_gss_tsig( )
  - enable_hostname_rewrite( )
  - enable_ifmap( )
  - enable_ifmap_publishing( )
  - enable_snmp_warnings( )
  - extattrs( )
  - extensible_attributes( )
  - gss_tsig_keys( )
  - high_water_mark( )
  - high_water_mark_reset( )
  - hostname_rewrite_policy( )
  - ifmap_client_credentials( )
  - ifmap_last_status_update( )
  - ifmap_password( )
  - ifmap_status( )
  - ifmap_status_details( )
  - ifmap_username( )
  - ignore_id( )
  - ignore_mac_addresses( )
  - ignore_dhcp_option_list_request( )
  - ipv6_ddns_enable_option_fqdn( )
  - ipv6_ddns_server_always_updates( )
  - ipv6_ddns_domainname( )
  - ipv6_ddns_hostname( )
  - ipv6_ddns_ttl( )
  - ipv6_dns_update_style( )
  - ipv6_domain_name( )
  - ipv6_domain_name_servers( )
  - ipv6_enable_ddns( )
  - ipv6_enable_gss_tsig( )
  - ipv6_enable_lease_scavenge( )
  - ipv6_remember_expired_client_association( )
  - ipv6_enable_retry_updates( )
  - ipv6_generate_hostname( )
  - ipv6_gss_tsig_keys( )
  - ipv6_kdc_server( )

- ipv6_lease_scavenge_time( )
- ipv6_microsoft_code_page( )
- ipv6_options( )
- ipv6_recycle_leases( )
- ipv6_retry_updates_interval( )
- ipv6_server_duid( )
- ipv6_update_dns_on_lease_renewal( )
- kdc_server( )
- ipv6_lease_scavenge_time( )
- lease_per_client_settings( )
- lease_scavenge_time( )
- log_lease_events( )
- logic_filters( )
- low_water_mark( )
- low_water_mark_reset( )
- override_dns_update_style( )
- override_enable_ifmap_publishing( )
- override_gss_tsig_keys( )
- override_ipv6_dns_update_style( )
- override_ipv6_gss_tsig_keys( )
- override_ipv6_lease_scavenge( )
- override_logic_filters( )
- name( )
- nextserver( )
- microsoft_code_page( )
- options( )
- option60_match_rules( )
- override_ddns_ttl( )
- override_enable_dhcpv6_service( )
- override_enable_fingerprint( )
- override_hostname_rewrite( )
- override_ignore_id( )
- override_ipv6_ddns_domainname( )
- override_ipv6_ddns_hostname( )
- override_ipv6_ddns_enable_option_fqdn( )
- override_ipv6_ddns_ttl( )
- override_ipv6_domain_name( )
- override_ipv6_domain_name_servers( )
- override_ipv6_enable_ddns( )
- override_ipv6_enable_gss_tsig( )
- override_ipv6_enable_retry_updates( )
- override_ipv6_generate_hostname( )
- override_ipv6_microsoft_code_page( )
- override_ipv6_options( )
- override_ipv6_recycle_leases( )
- override_ipv6_update_dns_on_lease_renewal( )
- override_lease_per_client_settings( )
- override_lease_scavenge_time( )
- override_log_lease_events( )
- override_preferred_lifetime( )
- override_update_dns_on_lease_renewal( )
- override_valid_lifetime( )
- ping_count( )
- ping_timeout( )
- preferred_lifetime( )
- pxe_lease_time( )
- recycle_leases( )
- retry_ddns_updates( )
- syslog_facility( )
- update_dns_on_lease_renewal( )
- valid_lifetime( )

- SAMPLE CODE
- AUTHOR
- SEE ALSO
- COPYRIGHT

NAME

Infoblox::Grid::Member::DHCP - DHCP member object.

DESCRIPTION

When you configure an Infoblox appliance to function as a DHCP server, you can set certain properties that control how the server

operates. You can also specify the configuration information the server includes in its DHCP messages. When a DHCP server assigns an IP address to a client, it can include configuration information the client needs to connect to the network and communicate with other hosts and appliances in the network.

Though you can set general DHCP properties at the grid level, you enable DHCP services at the member level only. Infoblox recommends that you configure the DHCP parameters before you enable DHCP on the Infoblox appliance.

CONSTRUCTOR

The Infoblox::Grid::Member::DHCP object is automatically generated upon the successful installation of the Infoblox appliance. It does not require manual construction.

SESSION METHODS

This section describes all the methods in the Infoblox::Session module that you can apply to a member DHCP object.

Infoblox::Session->get( )

Use this method to retrieve matching objects from the Infoblox appliance. See Infoblox::Session->get() for parameters and return values.

Key Reference

Apply the following attribute to get a specific member DHCP object:

```
name - Optional. A domain name in string format.
ipv4addr - Optional.   IPv4 address of the member
ipv6addr - Optional.   IPv6 address of the member
extattrs     - Optional. A hash reference containing extensible attributes.
extensible_attributes - Optional. A hash reference containing extensible attributes.
```

Example

```
my @retrieved_objs = $session->get(
    object => "Infoblox::Grid::Member::DHCP",
    name   => "infoblox.localdomain"
    );

# get all Grid::Member::DHCP objects with a given extensible attribute
my @retrieved_objs = $session->get(
    object => "Infoblox::Grid::Member::DHCP",
    extensible_attributes => { 'Site' => 'Santa Clara' });
```

Infoblox::Session->modify( )

Use this method to modify objects in the Infoblox appliance. See Infoblox::Session->modify() for parameters and return values.

Example

```
#Use method to modify dhcp service.
$member_dhcp->enable_dhcp("true");
#Submit modification
my $response = $session->modify( $member_dhcp );
```

METHODS

This section describes all the methods that you can use to configure and retrieve the attribute values of the DHCP properties of a grid member.

auth_server_group( )

Use this method to set or retrieve the Authentication Server Group object associated with this member.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

An Infoblox::Grid::Admin::RadiusAuthService object.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Getting auth_server_group

my $auth_server_group = $memberdhcp->auth_server_group( );
#Modifying auth_server_group
$auth_server_group->$memberdhcp($radius_service);
```

authority( )

Use this method to set or retrieve the authority flag of a grid member. This flag specifies if a DHCP server is authoritative for a domain.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

Specify "true" to set the authority flag or "false" to deactivate/unset it.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get authority
my $authority = $memberdhcp->authority();
#Modify authority
$memberdhcp->authority("true");
```

authn_captive_portal

The captive portal responsible for authentications for this DHCP member.

Parameter

The valid value is a name in FQDN format.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get authn_captive_portal
my $authn_captive_portal = $memberdhcp->authn_captive_portal();
#Modify authn_captive_portal
$memberdhcp->authn_captive_portal("cpmember.com");
```

authn_captive_portal_authenticated_filter

The MAC filter representing the authenticated range.

Parameter

The valid value is an Infoblox::DHCP::Filter::MAC object.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get authn_captive_portal_authenticated_filter
my $authn_captive_portal_authenticated_filter = $memberdhcp-
>authn_captive_portal_authenticated_filter();
#Modify authn_captive_portal_authenticated_filter
$memberdhcp->authn_captive_portal_authenticated_filter($macfilter);
```

authn_captive_portal_enabled

Use this method to set or retrieve the flag that controls if this DHCP member is enabled for captive portal authentication.

Parameter

Specify "true" to enable this feature or "false" to disable it. The default value is "false".

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get authn_captive_portal_enabled
my $authn_captive_portal_enabled = $memberdhcp->authn_captive_portal_enabled();
#Modify authn_captive_portal_enabled
$memberdhcp->authn_captive_portal_enabled("true");
```

authn_captive_portal_guest_filter

The MAC filter representing the guest range.

Parameter

The valid value is an Infoblox::DHCP::Filter::MAC object.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get authn_captive_portal_guest_filter
my $authn_captive_portal_guest_filter = $memberdhcp->authn_captive_portal_guest_filter();
#Modify authn_captive_portal_guest_filter
$memberdhcp->authn_captive_portal_guest_filter($macfilter);
```

auth_server_group_enabled

Use this method to set or retrieve the flag that controls if this DHCP member can send authentication requests to an authentication server group.

Parameter

Specify "true" to enable this feature or "false" to disable it. The default value is "false".

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get auth_server_group_enabled
my $auth_server_group_enabled = $memberdhcp->auth_server_group_enabled();
#Modify auth_server_group_enabled
$memberdhcp->auth_server_group_enabled("true");
```

bootfile( )

Use this method to set or retrieve the name of a file that DHCP clients need to boot. This setting overrides the grid level setting.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

This option is overridden independently from Infoblox::Grid::Member::DHCP->bootserver() and Infoblox::Grid::Member::DHCP->nextserver().

Parameter

The name of the file that the client must download. The default value is undefined.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get bootfile
my $bootfile = $memberdhcp->bootfile();
#Modify bootfile
$memberdhcp->bootfile("bootfile.txt");
#Un-override bootfile
$memberdhcp->bootfile(undef);
```

bootserver( )

Use this method to set or retrieve the name of the server on which a boot file is stored. This setting overrides the grid level setting.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

This option is overridden independently from Infoblox::Grid::Member::DHCP->bootfile() and Infoblox::Grid::Member::DHCP->nextserver().

Parameter

The boot server IP address or name in FQDN (Fully Qualified Domain Name) format. The FQDN consists of the host name followed by the domain name (example: abc.com). A boot server name can have a maximum of 256 bytes. The default value is undefined.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get bootserver
my $bootserver = $memberdhcp->bootserver();
#Modify bootserver
$memberdhcp->bootserver("hostname.com");
#Un-override bootserver
$memberdhcp->bootserver(undef);
```

ddns_generate_hostname( )

Use this method to set or retrieve the ddns_generate_hostname value. Set this value to enable/disable the ability of a member DHCP server to generate a host name and update DNS with this host name when it receives a DHCP REQUEST message that does not include a host name.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

Specify "true" to enable this feature or "false" to disable it.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get ddns_generate_hostname
my $ddns_generate_hostname = $memberdhcp->ddns_generate_hostname();
#Modify disable
$memberdhcp->ddns_generate_hostname("true");
```

ddns_retry_interval( )

Use this method to set or retrieve the ddns_retry_interval value. Set the retry interval when the member DHCP server makes repeated attempts to send DDNS updates to a DNS server.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

A 32-bit integer (range from 0 to 4294967295) that represents the retry interval in minutes. The default retry interval is five minutes.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get ddns_retry_interval
my $ddns_retry_interval = $memberdhcp->ddns_retry_interval();
#Modify ddns_retry_interval
$memberdhcp->ddns_retry_interval(10);
```

ddns_server_always_updates( )

Use this method to set or retrieve the ddns_server_always_updates value of a member DHCP server. Select this check box to allow only the DHCP server to update DNS,regardless of the requests from the DHCP clients. This setting overrides the grid level setting.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

Specify "true" to set ddns_server_always_updates flag or "false" to deactivate/unset it.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get ddns_server_always_updates
my $ddns_server_always_updates = $memberdhcp->ddns_server_always_updates();
#Modify ddns_server_always_updates
$memberdhcp->ddns_server_always_updates("true");
```

ddns_ttl( )

Use this method to set or retrieve the ddns_ttl value of a member DHCP server. The DDNS TTL (Dynamic DNS Time To Live) value specifies the number of seconds an IP address for the name is cached.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Setting this method to a defined value implicitly sets the override_ddns_ttl method to "true". Setting the parameter to undefined causes the appliance to use the Grid default and automatically resets the override_ddns_ttl attribute to "false".

Note that when ddns_ttl is set to a defined value and override_dddns_ttl is set to "false", the last operation takes precedence. Thus the sequence $object->ddns_ttl(12); $object->override_ddns_ttl("false"); will set override_ddns_ttl to "false", and the sequence $object->override_ddns_ttl("false"); $object->ddns_ttl(12); will result in override_ddns_ttl="true".

Parameter

A 32-bit integer (range from 0 to 4294967295) that specifies the number of seconds the IP address for the name is cached. Default value is half of lease time.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get ddns_ttl
my $ddns_ttl = $memberdhcp->ddns_ttl();
#Modify ddns_ttl
$memberdhcp->ddns_ttl(1000);
```

ddns_domainname( )

Use this method to set or retrieve the member DDNS domain name value.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Setting this method to a defined value implicitly sets the override_ddns_domainname method to "true". Setting the parameter to undefined causes the appliance to use the Grid default and automatically resets the override_ddns_domainname attribute to "false".

Note that when ddns_domainname is set to a defined value and override_ddns_domainname is set to "false", the last operation takes precedence. Thus the sequence $object->ddns_domainname("domain.com"); $object->override_ddns_domainname("false"); will set override_ddns_domainname to "false", and the sequence $object->override_ddns_domainname("false"); $object->ddns_domainname("domain.com"); will result in override_ddns_domainname="true".

Parameter

The DDNS domain name in FQDN (Fully Qualified Domain Name) format.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get ddns_domainname
my $ddns_domainname = $memberdhcp->ddns_domainname();
#Modify ddns_domainname
$memberdhcp->ddns_domainname("domain.com");
```

ddns_update_fixed_addresses( )

Use this method to set or retrieve the ddns_update_fixed_addresses value. Set this value to enable/disable the member DHCP server to update the A and PTR record of clients with a fixed address.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

Specify "true" to enable the member DHCP server to update the A and PTR record of clients with a fixed address or "false" to disable it.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get ddns_update_fixed_addresses
my $ddns_update_fixed_addresses = $memberdhcp->ddns_update_fixed_addresses();
#Modify ddns_update_fixed_addresses
$memberdhcp->ddns_update_fixed_addresses("true");
```

ddns_use_option81( )

Use this method to set or retrieve the ddns_use_option81 value of a member DHCP server. Set this value to enable/disable support for option 81.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

Specify "true" to enable support for option 81 or "false" to disable it.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get ddns_use_option81
my $ddns_use_option81 = $memberdhcp->ddns_use_option81();
#Modify ddns_use_option81
$memberdhcp->ddns_use_option81("true");
```

ddns_zone_primaries( )

Use this method to set or retrieve an ordered list of zone primaries that will receive DDNS updates.

Parameter

A valid value is a reference to an array of Infoblox::DHCP::DDNS objects.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
# Get the attribute value
$value = $memberdhcp->ddns_zone_primaries();
# Modify the attribute value
$memberdhcp->ddns_zone_primaries([$ddns]);
```

deny_bootp( )

Use this method to set or retrieve the deny_bootp value of a member DHCP server. Set this value to enable/disable deny_bootp and Grid level.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

Specify "true" to enable deny_bootp or "false" to disable it.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get deny_bootp
my $deny_bootp = $memberdhcp->deny_bootp();
#Modify deny_bootp
$memberdhcp->deny_bootp("true");
```

dns_update_style( )

Use this method to set or retrieve the update style for dynamic DNS updates (DHCPv4).

Setting this method to a defined value implicitly sets the override_dns_update_style method to 'true'. Setting the parameter to undefined causes the appliance to use the Grid default and automatically resets the override_dns_update_style attribute to 'false'.

Note that when dns_update_style is set to a defined value and override_dns_update_style is set to 'false', the last operation takes precedence. Thus the sequence $object->dns_update_style('INTERIM'); $object->override_dns_update_style('false'); will set override_dns_update_style to 'false', and the sequence $object->override_dns_update_style('false'); $object->dns_update_style('INTERIM'); will result in override_dns_update_style='true'.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

Valid values are 'INTERIM' and 'STANDARD'.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get dns_update_style value
my $value = $object->dns_update_style();

#Modify dns_update_style value
$object->dns_update_style('STANDARD');
```

email_list( )

Use this method to set or retrieve the email_list value of a member DHCP server.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

Array reference with the e-mail values.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get email_list
my $email_list = $memberdhcp->email_list();
#Modify email_list
$memberdhcp->email_list(["admin\@infoblox.com"]);
```

enable_ddns( )

Use this method to set or retrieve the enable_ddns value of a member DHCP server. Set this value to enable/disable the member DHCP server to send DDNS updates.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

Specify "true" to enable support for DDNS updates or "false" to disable it.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get enable_ddns
my $enable_ddns= $memberdhcp->enable_ddns();
#Modify enable_ddns
$memberdhcp->enable_ddns("true");
```

enable_dhcp( )

Use this method to set or retrieve the enable_dhcp value of a member. Set this value to enable/disable DHCP service.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

Specify "true" to enable DHCP or "false" to disable it.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get enable_dhcp
my $enable_dhcp= $memberdhcp->enable_dhcp();
#Modify enable_dhcp
$memberdhcp->enable_dhcp("true");
```

enable_dhcpv6_service( )

Use this method to set or retrieve the enable_dhcpv6_service flag. Set this flag to enable DHCPv6 service for the member.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Setting this method to a defined value implicitly sets the override_enable_dhcpv6_service method to "true". Setting the parameter to undefined causes the appliance to use the Grid default and automatically resets the override_enable_dhcpv6_service attribute to "false".

Note that when enable_dhcpv6_service is set to a defined value and override_denable_dhcpv6_service is set to "false", the last operation takes precedence. Thus the sequence $object->enable_dhcpv6_service("true"); $object->override_enable_dhcpv6_service("false"); will set override_enable_dhcpv6_service to "false", and the sequence $object->override_enable_dhcpv6_service("false"); $object->enable_dhcpv6_service("true"); will result in override_enable_dhcpv6_service="true".

Parameter

Specify "true" to enable this feature or "false" to disable it. The default value is "false".

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get enable_dhcpv6_service
my $enable_dhcpv6_service = $memberdhcp->enable_dhcpv6_service();
#Modify enable_dhcpv6_service
$memberdhcp->enable_dhcpv6_service("true");
```

enable_dhcp_on_lan2( )

Use this method to set or retrieve the enable_dhcp_on_lan2 value of a member. Set this value to enable/disable DHCP service on the LAN2 interface.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

Specify "true" to enable DHCP on the LAN2 interface or "false" to disable it.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get enable_dhcp_on_lan2
my $enable_dhcp_on_lan2 = $memberdhcp->enable_dhcp_on_lan2();
#Modify enable_dhcp_on_lan2
$memberdhcp->enable_dhcp_on_lan2("true");
```

enable_dhcp_on_ipv6_lan2( )

Use this method to set or retrieve the enable_dhcp_on_ipv6_lan2 value of a member. Set this value to enable/disable DHCP service on the IPv6 LAN2 interface.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

Specify "true" to enable DHCP on the IPv6 LAN2 interface or "false" to disable it.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get enable_dhcp_on_ipv6_lan2
my $enable_dhcp_on_ipv6_lan2 = $memberdhcp->enable_dhcp_on_ipv6_lan2();
#Modify enable_dhcp_on_ipv6_lan2
$memberdhcp->enable_dhcp_on_ipv6_lan2("true");
```

enable_dhcp_thresholds( )

Use this method to set or retrieve the enable_dhcp_thresholds value of a member DHCP server. Thresholds represents the watermarks above or below which address usage in a network is unexpected and might warrant your attention. This setting overrides the grid level setting.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

Specify "true" to enable DHCP thresholds or "false" to disable them.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get enable_dhcp_thresholds
my $enable_dhcp_thresholds = $memberdhcp->enable_dhcp_thresholds();
#Modify enable_dhcp_thresholds
$memberdhcp->enable_dhcp_thresholds("true");
```

enable_email_warnings( )

Use this method to set or retrieve the enable_email_warnings value of a member DHCP server. If we enable it and DHCP threshold is enabled, the Infoblox appliance sends an e-mail notification to an administrator if DHCP address usage crosses a high or low watermark threshold.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

Specify "true" to set enable_email_warnings flag or "false" to deactivate/unset it.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get enable_email_warnings
my $enable_email_warnings = $memberdhcp->enable_email_warnings();
#Modify enable_email_warnings
$memberdhcp->enable_email_warnings("true");
```

enable_fingerprint( )

Use this method to set or retrieve the member-level enable_fingerprint flag. If you enable this feature, the server will match a fingerprint for incoming lease requests.

Setting this method to a defined value implicitly sets the override_enable_fingerprint method to "true". Setting the parameter to undefined causes the appliance to use the Grid default and automatically resets the override_enable_fingerprint attribute to "false".

Parameter

Specify "true" to set the enable_fingerprint flag or "false" to deactivate/unset it. The default value is "true".

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get enable_fingerprint
my $enable_fingerprint = $memberdhcp->enable_fingerprint();
#Modify enable_fingerprint
$memberdhcp->enable_fingerprint("false");
```

enable_gss_tsig( )

Use this method to set or retrieve the member-level enable_gss_tsig flag.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

This is for DHCPv4 GSS-TSIG authentication.

Parameter

Specify "true" to enable GSS-TSIG authentication for DHCP IPv4 or "false" to disable it. The default value is undefined. Setting this flag overrides the enable_gss_tsig attribute of the grid.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get enable_gss_tsig
my $enable_gss_tsig = $memberdhcp->enable_gss_tsig();
#Modify enable_gss_tsig
$memberdhcp->enable_gss_tsig("true");
```

enable_hostname_rewrite( )

Use this method to set or retrieve the grid member's enable_hostname_rewrite flag.

Parameter

Boolean to enable or disable hostname rewrite for the member. The default value is "false".

Cannot enable the hostname rewrite policy along with the Microsoft code page. See microsoft_code_page().

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get enable_hostname_rewrite
$status = $memberdhcp->enable_hostname_rewrite();
#Modify enable_hostname_rewrite
$memberdhcp->enable_hostname_rewrite("true");
```

enable_ifmap( )

Use this method to enable or disable the IF-MAP DHCP client feature on this member. Enabling this feature requires a restart of DHCP services.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

Boolean to enable or disable the IF-MAP client. The default value is "false".

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get IF-MAP enable value
my $ifmap_enabled = $memberdhcp->enable_ifmap();
#Enable IF-MAP Client feature
$memberdhcp->enable_ifmap("true");
```

enable_ifmap_publishing( )

Use this method to set or retrieve the ifmap publishing option of a DHCP Member object. This setting overrides the member level settings.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

Specify "true" to set ifmap publishing option or "false" to deactivate/unset it. The default value is 'false', which indicates that this attribute inherits the member level setting.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
# Get enable_ifmap_publishing
my $enable_ifmap_publishing = $memberdhcp->enable_ifmap_publishing()
# Modify enable_ifmap_publishing
$memberdhcp->enable_ifmap_publishing("true");
# Un-override enable_ifmap_publishing
$memberdhcp->enable_ifmap_publishing(undef);
```

enable_snmp_warnings( )

Use this method to set or retrieve the enable_snmp_warnings value of member DHCP server. When DHCP threshold is enabled and DHCP address usage crosses a watermark threshold, the appliance sends an SNMP trap to the trap receiver that you defined for the grid member level.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

Specify "true" to set enable_snmp_warnings flag or "false" to deactivate/unset it.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get enable_snmp_warnings
my $enable_snmp_warnings = $memberdhcp->enable_snmp_warnings();
#Modify enable_snmp_warnings
$memberdhcp->enable_snmp_warnings("true");
```

extattrs( )

Use this method to retrieve the extensible attributes associated with a Grid Member DHCP object. This attribute is read-only and cannot be set.

Parameter

None.

Returns

The method returns the attribute value.

Example

```
#Get extattrs
my $ref_extattrs = $memberdhcp->extattrs();
```

extensible_attributes( )

Use this method to retrieve the extensible attributes associated with a Grid Member DHCP object.

Parameter

none

Returns

The method returns the attribute value.

Example

```
#Get extensible attributes
my $ref_extensible_attributes = $memberdhcp->extensible_attributes();
```

gss_tsig_keys( )

Use this method to set or retrieve the list of GSS-TSIG keys for a member DHCPv4 object.

Setting this method to a defined value implicitly sets the override_gss_tsig_keys method to "true". Setting the parameter to undefined causes the appliance to use the Grid default and automatically resets the override_gss_tsig_keys method to "false".

Note that when gss_tsig_keys is set to a defined value and override_gss_tsig_keys is set to "false", the last operation takes precedence. Thus the sequence $object->gss_tsig_keys($keys); $object->override_gss_tsig_keys("false"); will set override_gss_tsig_keys to "false", and the sequence $object->override_gss_tsig_keys("false"); $object->gss_tsig_keys($keys); will result in override_gss_tsig_keys="true".

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

The valid value is an array reference that contains Infoblox::Grid::KerberosKey objects.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get the Kerberos keys list
my $kerberos_keys_list = $member_dhcp->gss_tsig_keys();
```

```
#Modify the Kerberos keys list
my $kerberos_key1 = Infoblox::Cluster::KerberosKey->new(
       principal => 'host/appliance.ivan.test1@LADYBUG.LOCAL',
       );
my $kerberos_key2 = Infoblox::Cluster::KerberosKey->new(
       principal => 'host/appliance.ivan.test2@LADYBUG.LOCAL',
       );
$member_dhcp->gss_tsig_keys([$kerberos_key1, $kerberos_key2]);
```

high_water_mark( )

Use this method to set or retrieve the high_water_mark value of a member DHCP server. If the percent of allocated addresses exceeds this watermark, the appliance makes a syslog entry and if enabled, sends an e-mail notification.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

Number with the percentage value. Range is from 1 to 100. The high watermark value must be greater than the low watermark value. Default value is 95.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get high_water_mark
my $high_water_mark = $memberdhcp->high_water_mark();
#Modify high_water_mark
$memberdhcp->high_water_mark(80);
```

high_water_mark_reset( )

Use this method to set or retrieve the high_water_mark_reset value of a member DHCP server. If the percentage of allocated addresses drops below this value, a corresponding SNMP trap is reset.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

Number with the percentage value. Range is from 1 to 100. The high watermark value must be greater than the low watermark value. Default value is 85.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get high_water_mark_reset
my $high_water_mark_reset = $memberdhcp->high_water_mark_reset();
#Modify high_water_mark_reset
$memberdhcp->high_water_mark_reset(80);
```

hostname_rewrite_policy( )

Use this method to set or retrieve the name of the hostname rewrite policy that is in the protocol_hostname_rewrite_policies array of the Grid DHCP object. This attribute is mandatory if enable_hostname_rewrite is "true".

Parameter

The name, in string format, of the hostname rewrite policy.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get hostname_rewrite_policy
```

```
my $name = $memberdhcp->hostname_rewrite_policy();
#Modify hostname_rewrite_policy
$memberdhcp->hostname_rewrite_policy('new_policy');
```

ifmap_client_credentials( )

Use this method to set the IF-MAP client credentials used by the DHCP IF-MAP client on this member.

Include the specified parameter to set the attribute value.

Parameter

IF-MAP client credentials. The default value is undefined.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Create new IF-MAP client credentials
my $credential = Infoblox::IFMap::Client::Credentials->new(
                                                            authentication => 'basic',
                                                            username       => 'something',
                                                            password       => 'else',
                                                           );

#Modify IF-MAP client credentials
$memberdhcp->ifmap_client_credentials($credentials);
```

ifmap_last_status_update( )

Use this method to get the last status update of the IF-MAP member.

Parameter

None

Returns

The method returns the attribute value.

Example

```
#Get IF-MAP member last status update
$ifmap_last_status_update = $memberdhcp->ifmap_last_status_update();
```

ifmap_password( )

Use this method to set the IF-MAP client password used by the DHCP IF-MAP client on this member. The value can only be set or modified, but not retrieved. Note that use of this member has now been deprecated.

Include the specified parameter to set the attribute value.

Parameter

String for the IF-MAP client password. The default value is undef.

Returns

The method returns true when the modification succeeds, and returns false when the operation fails.

Example

```
#Modify IF-MAP client password
$memberdhcp->ifmap_password("secret");
```

ifmap_status( )

Use this method to get the IF-MAP member status.

Parameter

None

Returns

The method returns the attribute value.

Example

```
#Get IF-MAP member status
$ifmap_status = $memberdhcp->ifmap_status();
```

ifmap_status_details( )

Use this method to get the status details of the IF-MAP member.

Parameter

None

Returns

The method returns the attribute value.

Example

```
#Get IF-MAP member status details
$ifmap_status_details = $memberdhcp->ifmap_status_details();
```

ifmap_username( )

Use this method to set or retrieve the IF-MAP client username used by the DHCP IF-MAP client on this member. Note that use of this member has now been deprecated.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

String for the IF-MAP client username. The default value is undef.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get IF-MAP client username
my $username = $memberdhcp->ifmap_username();
#Modify IF-MAP client username
$memberdhcp->ifmap_username("ifmapclient");
```

ignore_id( )

Use this method to set or retrieve the ignore_id type of a grid member DHCP object. Indicates whether the appliance will ignore DHCP client IDs or MAC addresses. This setting overrides the upper-level settings.

Parameter

Valid values are 'NONE', 'CLIENT', or 'MACADDR'. The default is 'NONE'.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
# Get attribute value
$value = $memberdhcp->ignore_id();
# Modify attribute value
$memberdhcp->ignore_id('CLIENT');
#Un-override ignore_id
$memberdhcp->ignore_id(undef);
```

ignore_mac_addresses( )

Use this method to set or retrieve the ignore_mac_addresses list of a grid member DHCP object. A list of MAC addresses the appliance will ignore. This setting overrides the upper-level settings.

If you enter MAC addresses in the ignore_mac_addresses, then the appliance will ignore them. If you enter empty array, then the

appliance will ignore any MAC address.

Parameter

The valid value is an array reference that contains string objects.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
# Get attribute value
$value = $memberdhcp->ignore_mac_addresses();
# Modify attribute value
$memberdhcp->ignore_mac_addresses(["11::11::11::11::11::11"]);
```

ignore_dhcp_option_list_request( )

Use this method to set or retrieve the ignore_dhcp_option_list_request flag of a grid member DHCP. If this flag is set to true all available DHCP options will be returned to the client.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

Specify "true" to set the ignore_dhcp_option_list_request flag or "false" to deactivate/unset it.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get ignore_dhcp_option_list_request
my $ignore_dhcp_option_list_request = $memberdhcp->ignore_dhcp_option_list_request();
#Modify ignore_dhcp_option_list_request
$memberdhcp->ignore_dhcp_option_list_request("false");
#Do not override ignore_dhcp_option_list_request
$memberdhcp->ignore_dhcp_option_list_request(undef);
```

ipv6_ddns_enable_option_fqdn( )

Use this method to set or retrieve the ipv6_ddns_enable_option_fqdn value of a member DHCP server. This method controls whether the FQDN option sent by the client is to be used, or if the server can automatically generate the FQDN.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Setting this method to a defined value implicitly sets the override_ipv6_ddns_enable_option_fqdn method to "true". Setting the parameter to undefined causes the appliance to use the upper-level default and automatically resets the override_ipv6_ddns_enable_option_fqdn attribute to "false".

Note that when ipv6_ddns_enable_option_fqdn is set to a defined value and override_ipv6_ddns_enable_option_fqdn is set to "false", the last operation takes precedence. Thus the sequence $object->ipv6_ddns_enable_option_fqdn("true"); $object->override_ipv6_ddns_enable_option_fqdn("false"); will set override_ipv6_ddns_enable_option_fqdn to "false", and the sequence $object->override_ipv6_ddns_enable_option_fqdn("false"); $object->ipv6_ddns_enable_option_fqdn("true"); will result in override_ipv6_ddns_enable_option_fqdn="true".

Parameter

A 32-bit integer (range from 0 to 4294967295) that specifies the number of seconds the IP address for the name is cached. Default value is half of lease time.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get ipv6_ddns_enable_option_fqdn
my $ipv6_ddns_enable_option_fqdn = $memberdhcp->ipv6_ddns_enable_option_fqdn();
#Modify ipv6_ddns_enable_option_fqdn
$memberdhcp->ipv6_ddns_enable_option_fqdn("true");
```

ipv6_ddns_server_always_updates( )

Use this method to set or retrieve the ipv6_ddns_server_always_updates flag. Set this flag to "false" to update DNS only if requested by the client.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

Specify "true" to enable this feature or "false" to disable it. The default value is "false".

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get ipv6_ddns_server_always_updates
my $ipv6_ddns_server_always_updates = $memberdhcp->ipv6_ddns_server_always_updates();
#Modify ipv6_ddns_server_always_updates
$memberdhcp->ipv6_ddns_server_always_updates("true");
```

ipv6_ddns_domainname( )

Use this method to set or retrieve the member IPv6 DDNS domain name value.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Setting this method to a defined value implicitly sets the override_ipv6_ddns_domainname method to "true". Setting the parameter to undefined causes the appliance to use the Grid default and automatically resets the override_ipv6_ddns_domainname attribute to "false".

Note that when ipv6_ddns_domainname is set to a defined value and override_dipv6_ddns_domainname is set to "false", the last operation takes precedence. Thus the sequence $object->ipv6_ddns_domainname("domain.com"); $object->override_ipv6_ddns_domainname("false"); will set override_ipv6_ddns_domainname to "false", and the sequence $object->override_ipv6_ddns_domainname("false"); $object->ipv6_ddns_domainname("domain.com"); will result in override_ipv6_ddns_domainname="true".

Parameter

The DDNS domain name in FQDN (Fully Qualified Domain Name) format.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get ipv6_ddns_domainname
my $ipv6_ddns_domainname = $memberdhcp->ipv6_ddns_domainname();
#Modify ipv6_ddns_domainname
$memberdhcp->ipv6_ddns_domainname("domain.com");
```

ipv6_ddns_hostname( )

Use this method to set or retrieve the member IPv6 DDNS hostname value.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Setting this method to a defined value implicitly sets the override_ipv6_ddns_hostname method to "true". Setting the parameter to undefined causes the appliance to use the Grid default and automatically resets the override_ipv6_ddns_hostname attribute to "false".

Note that when ipv6_ddns_hostname is set to a defined value and override_dipv6_ddns_hostname is set to "false", the last operation takes precedence. Thus the sequence $object->ipv6_ddns_hostname("host.com"); $object->override_ipv6_ddns_hostname("false"); will set override_ipv6_ddns_hostname to "false", and the sequence $object->override_ipv6_ddns_hostname("false"); $object->ipv6_ddns_hostname("host.com"); will result in override_ipv6_ddns_hostname="true".

Parameter

The DDNS hostname in FQDN (Fully Qualified Domain Name) format.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get ipv6_ddns_hostname
my $ipv6_ddns_hostname = $memberdhcp->ipv6_ddns_hostname();
#Modify ipv6_ddns_hostname
$memberdhcp->ipv6_ddns_hostname("host.com");
```

ipv6_ddns_ttl( )

Use this method to set or retrieve the member IPv6 DDNS TTL value.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Setting this method to a defined value implicitly sets the override_ipv6_ddns_ttl method to "true". Setting the parameter to undefined causes the appliance to use the Grid default and automatically resets the override_ipv6_ddns_ttl attribute to "false".

Note that when ipv6_ddns_ttl is set to a defined value and override_dipv6_ddns_ttl is set to "false", the last operation takes precedence. Thus the sequence $object->ipv6_ddns_ttl(12345); $object->override_ipv6_ddns_ttl("false"); will set override_ipv6_ddns_ttl to "false", and the sequence $object->override_ipv6_ddns_ttl("false"); $object->ipv6_ddns_ttl(12345); will result in override_ipv6_ddns_ttl="true".

Parameter

The preferred DDNS TTL value in seconds. The default is undefined.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get ipv6_ddns_ttl
my $ipv6_ddns_ttl = $memberdhcp->ipv6_ddns_ttl();
#Modify ipv6_ddns_ttl
$memberdhcp->ipv6_ddns_ttl(12345);
```

ipv6_dns_update_style( )

Use this method to set or retrieve the update style for dynamic DNS updates (DHCPv6).

Setting this method to a defined value implicitly sets the override_ipv6_dns_update_style method to 'true'. Setting the parameter to undefined causes the appliance to use the Grid default and automatically resets the override_ipv6_dns_update_style attribute to 'false'.

Note that when ipv6_dns_update_style is set to a defined value and override_ipv6_dns_update_style is set to 'false', the last operation takes precedence. Thus the sequence $object->ipv6_dns_update_style('INTERIM'); $object->override_ipv6_dns_update_style('false'); will set override_ipv6_dns_update_style to 'false', and the sequence $object->override_ipv6_dns_update_style('false'); $object->ipv6_dns_update_style('INTERIM'); will result in override_ipv6_dns_update_style='true'.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

Valid values are 'INTERIM' and 'STANDARD'.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get ipv6_dns_update_style value
my $value = $object->ipv6_dns_update_style();

#Modify ipv6_dns_update_style value
$object->ipv6_dns_update_style('STANDARD');
```

ipv6_domain_name( )

Use this method to set or retrieve the ipv6_domain_name value of a Grid Member DHCP object.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Setting this method to a defined value implicitly sets the override_ipv6_domain_name method to "true". Setting the parameter to

undefined causes the appliance to use the upper level default and automatically resets the override_ipv6_domain_name attribute to "false".

Note that when ipv6_domain_name is set to a defined value and override_ipv6_domain_name is set to "false", the last operation takes precedence. Thus the sequence $object->ipv6_domain_name("testdomain"); $object->override_ipv6_domain_name("false"); will set override_ipv6_domain_name to "false", and the sequence $object->override_ipv6_domain_name("false"); $object->ipv6_domain_name("testdomain"); will result in override_ipv6_domain_name="true".

Parameter

The name of the client's domain for resolving names in DNS in string format.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get ipv6_domain_name
my $ipv6_domain_name = $memberdhcp->ipv6_domain_name();
#Modify ipv6_domain_name
$memberdhcp->ipv6_domain_name("test_domain.com");
```

ipv6_domain_name_servers( )

Use this method to set or retrieve the ipv6_domain_name_servers value of a Grid Member DHCP object.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Setting this method to a defined value implicitly sets the override_ipv6_domain_name_servers method to "true". Setting the parameter to undefined causes the appliance to use the upper-level default and automatically resets the override_ipv6_domain_name_servers attribute to "false".

Note that when ipv6_domain_name_servers is set to a defined value and override_ipv6_domain_name_servers is set to "false", the last operation takes precedence. Thus the sequence $object->ipv6_domain_name_servers("2001::1,2001::2"); $object->override_ipv6_domain_name_servers("false"); will set override_ipv6_domain_name_servers to "false", and the sequence $object->override_ipv6_domain_name_servers("false"); $object->ipv6_domain_name_servers("2001::1,2001::2"); will result in override_ipv6_domain_name_servers="true".

Parameter

The domain name server addresses in IPv6 address format.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get ipv6_domain_name_servers
my $ipv6_domain_name_servers = $memberdhcp->ipv6_domain_name_servers();
#Modify ipv6_domain_name_servers
$memberdhcp->ipv6_domain_name_servers("2001::1,2001::2");
```

ipv6_enable_ddns( )

Use this method to set or retrieve the ipv6_enable_ddns flag. Set this flag to enable DHCPv6 dynamic updates.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Setting this method to a defined value implicitly sets the override_ipv6_enable_ddns method to "true". Setting the parameter to undefined causes the appliance to use the Grid default and automatically resets the override_ipv6_enable_ddns attribute to "false".

Note that when ipv6_enable_ddns is set to a defined value and override_dipv6_enable_ddns is set to "false", the last operation takes precedence. Thus the sequence $object->ipv6_enable_ddns("true"); $object->override_ipv6_enable_ddns("false"); will set override_ipv6_enable_ddns to "false", and the sequence $object->override_ipv6_enable_ddns("false"); $object->ipv6_enable_ddns("true"); will result in override_ipv6_enable_ddns="true".

Parameter

Specify "true" to enable this feature or "false" to disable it. The default value is "false".

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get ipv6_enable_ddns
my $ipv6_enable_ddns = $memberdhcp->ipv6_enable_ddns();
#Modify ipv6_enable_ddns
$memberdhcp->ipv6_enable_ddns("true");
```

ipv6_enable_gss_tsig( )

Use this method to set or retrieve the member-level ipv6_enable_gss_tsig flag.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

This is for DHCPv6 GSS-TSIG authentication.

Setting this method to a defined value implicitly sets the override_ipv6_enable_gss_tsig method to "true". Setting the parameter to undefined causes the appliance to use the upper-level default and automatically resets the override_ipv6_enable_gss_tsig attribute to "false".

Note that when ipv6_enable_gss_tsig is set to a defined value and override_ipv6_enable_gss_tsig is set to "false", the last operation takes precedence. Thus the sequence $object->ipv6_enable_gss_tsig("true"); $object->override_ipv6_enable_gss_tsig("false"); will set override_ipv6_enable_gss_tsig to "false", and the sequence $object->override_ipv6_enable_gss_tsig("false"); $object->ipv6_enable_gss_tsig("true"); will result in override_ipv6_enable_gss_tsig="true".

Parameter

Specify "true" to enable GSS-TSIG authentication for DHCP IPv6 or "false" to disable it. The default value is undefined. Setting this flag overrides the ipv6_enable_gss_tsig attribute of the grid.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get ipv6_enable_gss_tsig
my $ipv6_enable_gss_tsig = $memberdhcp->ipv6_enable_gss_tsig();
#Modify ipv6_enable_gss_tsig
$memberdhcp->ipv6_enable_gss_tsig("true");
```

ipv6_enable_lease_scavenge( )

Use this method to set or retrieve the flag that indicates whether the DHCPv6 lease scavenging is enabled or disabled.

Setting this method to a defined value implicitly sets the override_ipv6_lease_scavenge method to "true". When you set the parameter to undefined and if ipv6_lease_scavenge_time or ipv6_remember_expired_client_association are defined, the appliance uses the default Grid value and automatically resets override_ipv6_lease_scavenge attribute to "false".

Note that when ipv6_enable_lease_scavenge is set to a defined value and override_ipv6_enable_lease_scavenge is set to "false", the last operation takes precedence. Thus, the sequence $object->ipv6_enable_lease_scavenge("true"); $object->override_ipv6_lease_scavenge("false"); sets override_ipv6_lease_scavenge to "false", and the sequence $object->override_ipv6_lease_scavenge("false"); $object->ipv6_enable_lease_scavenge("true"); results in override_ipv6_lease_scavenge="true".

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

Specify 'true' to enable DHCPv6 lease scavenging and 'false' to disable it. The default value is false.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get ipv6_enable_lease_scavenge
my $ipv6_enable_lease_scavenge = $memberdhcp->ipv6_enable_lease_scavenge();
#Modify ipv6_enable_lease_scavenge
$memberdhcp->ipv6_enable_lease_scavenge("true");
```

ipv6_remember_expired_client_association( )

Use this method to set or retrieve the flag that indicates whether the DHCPv6 expired lease binding is enabled or disabled.

Setting this method to a defined value implicitly sets the override_ipv6_lease_scavenge method to "true". When you set the parameter to undefined and if ipv6_enable_lease_scavenge or ipv6_lease_scavenge_time are defined, the appliance uses the default Grid value and automatically resets override_ipv6_lease_scavenge to "false".

Note that when ipv6_remember_expired_client_association is set to a defined value and override_ipv6_lease_scavenge is set to "false", the last operation takes precedence. Thus, the sequence $object->ipv6_remember_expired_client_association("true"); $object->override_ipv6_lease_scavenge("false"); sets override_ipv6_lease_scavenge to "false", and the sequence $object->override_ipv6_lease_scavenge("false"); $object->ipv6_remember_expired_client_association("true"); results in override_ipv6_lease_scavenge="true".

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

Specify 'true' to enable the DHCPv6 server to 'remember' the client and 'false' to disable it. The default value is false.

Returns

If you specified a parameter, the method returns true when the modification succeds, and return false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get ipv6_remember_expired_client_association
my $ipv6_remember_expired_client_association = $memberdhcp-
>ipv6_remember_expired_client_association();
#Modify ipv6_remember_expired_client_association
$memberdhcp->ipv6_remember_expired_client_association("true");
```

ipv6_enable_retry_updates( )

Use this method to set or retrieve the ipv6_enable_retry_updates flag. If this flag is set to "true", the DHCPv6 server retries failed dynamic DNS updates.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Setting this method to a defined value implicitly sets the override_ipv6_enable_retry_updates method to "true". Setting the parameter to undefined causes the appliance to use the Grid default and automatically resets the override_ipv6_enable_retry_updates attribute to "false".

Note that when ipv6_enable_retry_updates is set to a defined value and override_dipv6_enable_retry_updates is set to "false", the last operation takes precedence. Thus the sequence $object->ipv6_enable_retry_updates("true"); $object->override_ipv6_enable_retry_updates("false"); will set override_ipv6_enable_retry_updates to "false", and the sequence $object->override_ipv6_enable_retry_updates("false"); $object->ipv6_enable_retry_updates("true"); will result in override_ipv6_enable_retry_updates="true".

Parameter

Specify "true" to enable this feature or "false" to disable it. The default value is "true".

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get ipv6_enable_retry_updates
my $ipv6_enable_retry_updates = $memberdhcp->ipv6_enable_retry_updates();
#Modify ipv6_enable_retry_updates
$memberdhcp->ipv6_enable_retry_updates("true");
```

ipv6_generate_hostname( )

Use this method to set or retrieve the ipv6_generate_hostname flag. Set this flag to generate the hostname if it is not sent by the client.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Setting this method to a defined value implicitly sets the override_ipv6_generate_hostname method to "true". Setting the parameter to undefined causes the appliance to use the Grid default and automatically resets the override_ipv6_generate_hostname attribute to "false".

Note that when ipv6_generate_hostname is set to a defined value and override_dipv6_generate_hostname is set to "false", the last

operation takes precedence. Thus the sequence $object->ipv6_generate_hostname("true"); $object->override_ipv6_generate_hostname("false"); will set override_ipv6_generate_hostname to "false", and the sequence $object->override_ipv6_generate_hostname("false"); $object->ipv6_generate_hostname("true"); will result in override_ipv6_generate_hostname="true".

Parameter

Specify "true" to enable this feature or "false" to disable it. The default value is "false".

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get ipv6_generate_hostname
my $ipv6_generate_hostname = $memberdhcp->ipv6_generate_hostname();
#Modify ipv6_generate_hostname
$memberdhcp->ipv6_generate_hostname("true");
```

ipv6_gss_tsig_keys( )

Use this method to set or retrieve the list of GSS-TSIG keys for a member DHCPv6 object.

Setting this method to a defined value implicitly sets the override_ipv6_gss_tsig_keys method to "true". Setting the parameter to undefined causes the appliance to use the Grid default and automatically resets the override_ipv6_gss_tsig_keys method to "false".

Note that when ipv6_gss_tsig_keys is set to a defined value and override_ipv6_gss_tsig_keys is set to "false", the last operation takes precedence. Thus the sequence $object->ipv6_gss_tsig_keys($keys); $object->override_ipv6_gss_tsig_keys("false"); will set override_ipv6_gss_tsig_keys to "false", and the sequence $object->override_ipv6_gss_tsig_keys("false"); $object->ipv6_gss_tsig_keys($keys); will result in override_ipv6_gss_tsig_keys="true".

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

The valid value is an array reference that contains Infoblox::Grid::KerberosKey objects.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get the Kerberos keys list
my $kerberos_keys_list = $member_dhcp->ipv6_gss_tsig_keys();

#Modify the Kerberos keys list
my $kerberos_key1 = Infoblox::Cluster::KerberosKey->new(
       principal => 'host/appliance.ivan.test1@LADYBUG.LOCAL',
       );
my $kerberos_key2 = Infoblox::Cluster::KerberosKey->new(
       principal => 'host/appliance.ivan.test2@LADYBUG.LOCAL',
       );
$member_dhcp->ipv6_gss_tsig_keys([$kerberos_key1, $kerberos_key2]);
```

ipv6_kdc_server( )

Use this method to specify or retrieve the IPv4 address or FQDN of the Kerberos server for DHCPv6 GSS-TSIG authentication. This setting overrides the grid-level setting.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

The Kerberos server IPv4 address or hostname, with a maximum of 1023 bytes.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get ipv6_kdc_server
my $ipv6_kdc_server = $memberdhcp->ipv6_kdc_server();
#Modify ipv6_kdc_server
$memberdhcp->ipv6_kdc_server("10.32.0.8");
```

ipv6_lease_scavenge_time( )

Use this method to set or retrieve the Grid-level grace period in seconds to keep an expired lease before it gets deleted by the scavenging process.

Setting this method to a defined value implicitly sets the override_ipv6_lease_scavenge method to "true". When you set the parameter to undefined and if ipv6_enable_lease_scavenge or ipv6_remember_expired_client_association are defined, the appliance uses the default Grid value and automatically resets override_ipv6_lease_scavenge attribute to "false".

Note that when ipv6_lease_scavenge_time is set to a defined value and override_ipv6_enable_lease_scavenge is set to "false", the last operation takes precedence. Thus, the sequence $object->ipv6_lease_scavenge_time(32000); $object->override_ipv6_lease_scavenge("false"); sets override_ipv6_lease_scavenge to "false", and the sequence $object->override_ipv6_lease_scavenge("false"); $object->ipv6_lease_scavenge_time(32000); results in override_ipv6_lease_scavenge="true".

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

The valid value is an unsigned integer greater than or equal to 21600.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get ipv6_lease_scavenge_time
my $ipv6_lease_scavenge_time = $griddhcp->ipv6_lease_scavenge_time();
#Modify ipv6_lease_scavenge_time
```

ipv6_microsoft_code_page( )

Use this method to set or retrieve the Microsoft client DHCP IPv6 code page value of a grid member. This value is the hostname translation code page for Microsoft DHCP IPv6 clients and overrides the grid-level Microsoft DHCP IPv6 client code page.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Setting this method to a defined value implicitly sets the override_ipv6_microsoft_code_page method to "true". Setting the parameter to undefined causes the appliance to use the Grid default and automatically resets the override_ipv6_microsoft_code_page attribute to "false".

Note that when ipv6_microsoft_code_page is set to a defined value and override_dipv6_microsoft_code_page is set to "false", the last operation takes precedence. Thus the sequence $object->ipv6_microsoft_code_page("Cyrillic (ISO-5589-5)"); $object->override_ipv6_microsoft_code_page("false"); will set override_ipv6_microsoft_code_page to "false", and the sequence $object->override_ipv6_microsoft_code_page("false"); $object->ipv6_microsoft_code_page("Cyrillic (ISO-5589-5)"); will result in override_ipv6_microsoft_code_page="true".

Parameter

Microsoft DHCP client code page in string format. Valid values are:

None

Arabic (1256)

Arabic (ISO-8859-6)

Baltic (1257)

Baltic (775)

Baltic (ISO-8859-4)

Central Europe (1250)

Cyrillic (1251)

Cyrillic (855)

Cyrillic (ISO-8859-5)

Greek (1253)

Greek (737)

Greek (ISO-8859-7)

Hebrew (1255)

Hebrew (862)

Hebrew (ISO-8859-8)

Japanese Shift-JIS (932)

Korean (949)

Latin 1 (ISO-8859-1)

Latin 2 (ISO-8859-2)

Latin 3 (ISO-8859-3)

Latin 9 (ISO-8859-15)

Latin I (1252)

Latin II (852)

Multilingual Latin I (850)

Russian (866)

Simplified Chinese GBK (936)

Thai (874)

Traditional Chinese Big5 (950)

Turkish (1254)

Turkish (857)

Turkish (ISO-8859-9)

US (437)

Vietnam (1258)

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get ipv6_microsoft_code_page
my $ipv6_microsoft_code_page = $memberdhcp->ipv6_microsoft_code_page();
#Modify ipv6_microsoft_code_page
$memberdhcp->ipv6_microsoft_code_page("Cyrillic (ISO-8859-5)");
#Do not override ipv6_microsoft_code_page
$memberdhcp->ipv6_microsoft_code_page("None");
```

ipv6_options( )

Use this method to set or retrieve the ipv6 options.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Setting this method to a defined value implicitly sets the override_ipv6_options method to "true". Setting the parameter to undefined causes the appliance to use the Grid default and automatically resets the override_ipv6_options attribute to "false".

Note that when ipv6_options is set to a defined value and override_dipv6_options is set to "false", the last operation takes precedence. Thus the sequence $object->ipv6_options([$opt]); $object->override_ipv6_options("false"); will set override_ipv6_options to "false", and the sequence $object->override_ipv6_options("false"); $object->ipv6_options([$opt]); will result in override_ipv6_options="true".

Parameter

The valid value is an array reference that contains Infoblox::DHCP::Option object(s).

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get ipv6_options
my $ipv6_options = $memberdhcp->ipv6_options();
#Modify ipv6_options
my $option1 = Infoblox::DHCP::Option->new(
    name   => "dhcp6.rapid-commit",
    value  => "true"
);
$memberdhcp->ipv6_options([$option1);
```

ipv6_recycle_leases( )

Use this method to set or retrieve the ipv6_recycle_leases flag of the DHCP member.

If the flag is enabled, the leases are kept in recycle bin until one week after expiration. When the flag is disabled, the leases are irrecoverably deleted.

Setting this method to a defined value implicitly sets the override_ipv6_recycle_leases method to "true". Setting the parameter to undefined causes the appliance to use the Grid default and automatically resets the override_ipv6_recycle_leases attribute to "false".

Note that when ipv6_recycle_leases is set to a defined value and override_ipv6_recycle_leases is set to "false", the last operation takes precedence. Thus the sequence $object->ipv6_recycle_leases("true"); $object->override_ipv6_recycle_leases("false"); will set override_ipv6_recycle_leases to "false", and the sequence $objct->override_ipv6_recycle_leases("false"); $object->ipv6_recycle_leases("true"); will result in override_ipv6_recycle_leases="true".

Parameter

Specify "true" to set the ipv6_recycle_leases flag or "false" to deactivate/unset it. The default value is undefined.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get ipv6_recycle_leases
my $ipv6_recycle_leases = $memberdhcp->ipv6_recycle_leases();
#Modify ipv6_recycle_leases
$memberdhcp->ipv6_recycle_leases("false");
```

ipv6_retry_updates_interval( )

Use this method to set or retrieve the ipv6_retry_updates_interval value. Set the retry interval when the member DHCPv6 server makes repeated attempts to send DDNS updates to a DNS server.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

A 32-bit integer (range from 0 to 4294967295) that represents the retry interval in minutes. The default retry interval is five minutes.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get ipv6_retry_updates_interval
my $ipv6_retry_updates_interval = $memberdhcp->ipv6_retry_updates_interval();
#Modify ipv6_retry_updates_interval
$memberdhcp->ipv6_retry_updates_interval(10);
```

ipv6_server_duid( )

Use this method to set or retrieve the server DHCPv6 unique identifier (DUID) for the grid member.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

DUID of the grid member in string format.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get ipv6_server_duid
my $ipv6_server_duid = $memberdhcp->ipv6_server_duid();
#Modify ipv6_server_duid
$memberdhcp->ipv6_server_duid("false");
```

ipv6_update_dns_on_lease_renewal( )

Use this method to set or retrieve the ipv6_update_dns_on_lease_renewal flag. This attribute controls whether the DHCP server updates DNS when an IPv6 DHCP lease is renewed.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Setting this method to a defined value implicitly sets the override_ipv6_update_dns_on_lease_renewal method to "true". Setting the parameter to undefined causes the appliance to use the Grid default and automatically resets the override_ipv6_update_dns_on_lease_renewal attribute to "false".

Note that when ipv6_update_dns_on_lease_renewal is set to a defined value and override_dipv6_update_dns_on_lease_renewal is set to "false", the last operation takes precedence. Thus the sequence $object->ipv6_update_dns_on_lease_renewal("true"); $object->override_ipv6_update_dns_on_lease_renewal("false"); will set override_ipv6_update_dns_on_lease_renewal to "false", and the sequence $object->override_ipv6_update_dns_on_lease_renewal("false"); $object->ipv6_update_dns_on_lease_renewal("true"); will result in override_ipv6_update_dns_on_lease_renewal="true".

Parameter

Specify "true" to enable this feature or "false" to disable it. The default value is "false".

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get ipv6_update_dns_on_lease_renewal
my $ipv6_update_dns_on_lease_renewal = $memberdhcp->ipv6_update_dns_on_lease_renewal();
#Modify ipv6_update_dns_on_lease_renewal
$memberdhcp->ipv6_update_dns_on_lease_renewal("true");
```

kdc_server( )

Use this method to specify or retrieve the IPv4 address or FQDN of the Kerberos server for DHCPv4 GSS-TSIG authentication. This setting overrides the grid-level setting.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

The Kerberos server IPv4 address or FQDN, with a maximum of 1023 bytes.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get kdc_server
my $kdc_server = $memberdhcp->kdc_server();
#Modify  kdc_server
$memberdhcp->kdc_server("10.32.0.7");
```

ipv6_lease_scavenge_time( )

Use this method to set or retrieve the member-level grace period in seconds to keep an expired lease before it gets deleted by the scavenging process.

Setting this method to a defined value implicitly sets the override_ipv6_lease_scavenge method to "true". Setting the parameter to undefined unless ipv6_enable_lease_scavenge is defined causes the appliance to use the Grid default and automatically resets the override_ipv6_lease_scavenge attribute to "false".

Note that when ipv6_lease_scavenge_time is set to a defined value and override_dipv6_lease_scavenge_time is set to "false", the last operation takes precedence. Thus the sequence $object->ipv6_lease_scavenge_time("true"); $object->override_ipv6_lease_scavenge("false"); will set override_ipv6_lease_scavenge to "false", and the sequence $object->override_ipv6_lease_scavenge("false"); $object->ipv6_lease_scavenge_time("true"); will result in override_ipv6_lease_scavenge="true".

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

The valid value is an unsigned integer greater than or equal to 21600.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get ipv6_lease_scavenge_time
my $ipv6_lease_scavenge_time = $memberdhcp->ipv6_lease_scavenge_time();
#Modify ipv6_lease_scavenge_time
$memberdhcp->ipv6_lease_scavenge_time("22000");
```

lease_per_client_settings( )

Use this method to set or retrieve the Grid member setting that defines how DHCP server will free or release client leases.

Setting this method to a defined value implicitly sets the override_lease_per_client_settings method to "true". Setting the parameter to undefined causes the appliance to use the Grid default and automatically resets the override_lease_per_client_settings attribute to "false".

Note that when lease_per_client_settings is set to a defined value and override_lease_per_client_settings is set to "false", the last operation takes precedence. Thus the sequence $object->lease_per_client_settings("NEVER_RELEASE"); $object->override_lease_per_client_settings("false"); will set override_lease_per_client_settings to "false", and the sequence $object->override_lease_per_client_settings("false"); $object->lease_per_client_settings("NEVER_RELEASE"); will set override_lease_per_client_settings to "true".

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

The valid values are 'ONE_LEASE_PER_CLIENT', 'RELEASE_MATCHING_ID' and 'NEVER_RELEASE'.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get lease_per_client_settings value
my $lease_per_client_settings = $object->lease_per_client_settings();

#Modify lease_per_client_settings value
$object->lease_per_client_settings("NEVER_RELEASE");
```

lease_scavenge_time( )

Use this method to set or retrieve the member-level lease_scavenge_time value.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Setting this method to a defined value implicitly sets the override_lease_scavenge_time method to "true". Setting the parameter to undefined causes the appliance to use the Grid default and automatically resets the override_lease_scavenge_time attribute to "false".

Note that when lease_scavenge_time is set to a defined value and override_lease_scavenge_time is set to "false", the last operation takes precedence. Thus the sequence $object->lease_scavenge_time(604800); $object->override_lease_scavenge_time("false"); will set override_lease_scavenge_time to "false", and the sequence $object->override_lease_scavenge_time("false"); $object->lease_scavenge_time(604800); will result in override_lease_scavenge_time="true".

Parameter

An integer that specifies the period of time, in seconds, that free and backup leases remain in the database before they are automatically deleted. To disable lease scavenging, set the parameter to -1. The minimum positive value must be greater than 86400 seconds (1 day). The default value of this parameter is -1, which means that lease scavenging is disabled.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get lease_scavenge_time
my $lease_scavenge = $memberdhcp->lease_scavenge_time();
#Modify lease_scavenge_time
$memberdhcp->lease_scavenge_time(30 * 24 * 60 * 60);
```

log_lease_events( )

Use this method to set or retrieve the log_lease_events value of a member DHCP server. This value specifies whether the grid member logs lease events.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Setting this method to a defined value implicitly sets the override_log_lease_events method to "true". Setting the parameter to undefined causes the appliance to use the Grid default and automatically resets the override_log_lease_events attribute to "false".

Note that when log_lease_events is set to a defined value and override_dlog_lease_events is set to "false", the last operation takes precedence. Thus the sequence $object->log_lease_events("true"); $object->override_log_lease_events("false"); will set override_log_lease_events to "false", and the sequence $object->override_log_lease_events("false"); $object->log_lease_events("true"); will result in override_log_lease_events="true".

Parameter

Specify "true" to set log_lease_events flag or "false" to deactivate/unset it.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get log_lease_events
my $log_lease_events = $memberdhcp->log_lease_events();
#Modify log_lease_events
$memberdhcp->log_lease_events("true");
```

logic_filters( )

Use this method to set or retrieve all the logic (ordered) filters associated with the Member DHCP.

Filters are used to screen address assignments on a Member DHCP if the addresses are to be assigned to specific hosts.

Setting this method to a defined value implicitly sets the override_logic_filters method to "true". Setting the parameter to undefined causes the appliance to use the Grid default and automatically resets the override_logic_filters attribute to "false".

Note that when logic_filters is set to a defined value and override_dlogic_filters is set to "false", the last operation takes precedence. Thus the sequence $object->logic_filters($filters); $object->override_logic_filters("false"); will set override_logic_filters to "false", and the sequence $object->override_logic_filters("false"); $object->logic_filters($filters); will result in override_logic_filters="true".

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

The valid value is an array reference that contains either the Infoblox::DHCP::Filter::MAC, Infoblox::DHCP::Filter::NAC, Infoblox::DHCP::Filter::Option objects or the names of the corresponding filters. The filters in the list must satisfy the following conditions:

Filter name must be globally unique.
Filters without expressions must be at the end of the list.
The filters should not be disabled.

The default value of the parameter is undefined.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get logic_filters
my $ref_logic_filters = $memberdhcp->logic_filters();
#Modify logic_filters
$memberdhcp->logic_filters(["filter_1","filter_2",$exl1]); #$exl1 is an Infoblox::DHCP::Filter::MAC,
Infoblox::DHCP::Filter::NAC or Infoblox::DHCP::Filter::Option object.
```

low_water_mark( )

Use this method to set or retrieve the low_water_mark value of a member DHCP server. If the percent of allocated addresses drops below this watermark, the appliance makes a syslog entry and if enabled, sends an e-mail notification.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

Number with the percentage value. Range is from 0 to 100. The low watermark value must be lower than the high watermark value. The default low watermark number is 0.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get low_water_mark
my $low_water_mark = $memberdhcp->low_water_mark();
#Modify low_water_mark
$memberdhcp->low_water_mark(10);
```

low_water_mark_reset( )

Use this method to set or retrieve the low_water_mark_reset value of a member DHCP server. If the percentage of allocated addresses exceeds this value, a corresponding SNMP trap is reset.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

Number with the percentage value. Range is from 0 to 100. The low watermark reset value must be higher than the low watermark value. The default low watermark reset value is 0.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get low_water_mark_reset
my $low_water_mark_reset = $memberdhcp->low_water_mark_reset();
#Modify low_water_mark_reset
$memberdhcp->low_water_mark_reset(10);
```

override_dns_update_style( )

The override_dns_update_style controls whether the dns_update_style in the DHCP Member is used, instead of the Grid default.

The override_dns_update_style can be specified explicitly. It is also set implicitly when dns_update_style is set to a defined value.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

Set the parameter to 'true' to override the Grid-level setting for dns_update_style. Set the parameter to 'false' to inherit the Grid-level setting for dns_update_style.

The default value of this parameter is 'false'.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get override_dns_update_style value
my $value = $object->override_dns_update_style();

#Modify override_dns_update_style value
$object->override_dns_update_style('true');
```

override_enable_ifmap_publishing( )

The override_enable_ifmap_publishing controls whether the enable_ifmap_publishing in the DHCP Member is used, instead of the Grid default.

The override_enable_ifmap_publishing can be specified explicitly. It is also set implicitly when enable_ifmap_publishing is set to a defined value.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

Set the parameter to "true" to override the grid-level setting for enable_ifmap_publishing. Set the parameter to "false" to inherit the grid-level setting for enable_ifmap_publishing.

The default value of this parameter is "false".

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Getting override_enable_ifmap_publishing
my $override_enable_ifmap_publishing=$memberdhcp->override_enable_ifmap_publishing( );
#Modifying override_enable_ifmap_publishing
$memberdhcp->override_enable_ifmap_publishing("true");
```

override_gss_tsig_keys( )

The override_gss_tsig_keys attribute controls whether the gss_tsig_keys method values of the member are used, instead of the Grid default.

The override_gss_tsig_keys attribute can be specified explicitly. It is also set implicitly when gss_tsig_keys is set to a defined value.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

Set the parameter to "true" to override GSS-TSIG keys. Set the parameter to "false" to inherit the grid-level GSS-TSIG keys.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Getting override_gss_tsig_keys
my $override_gss_tsig_keys = $member_dhcp->override_gss_tsig_keys();
#Modyfying override_gss_tsig_keys
$member_dhcp->override_gss_tsig_keys("true");
```

override_ipv6_dns_update_style( )

The override_ipv6_dns_update_style controls whether the ipv6_dns_update_style is used instead of the Grid default in the DHCP member.

The override_ipv6_dns_update_style can be specified explicitly. It is also set implicitly when ipv6_dns_update_style is set to a defined value.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

Set the parameter to 'true' to override the Grid-level setting for ipv6_dns_update_style. Set the parameter to 'false' to inherit the Grid-level setting for ipv6_dns_update_style.

The default value of this parameter is 'false'.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get override_ipv6_dns_update_style value
my $value = $object->override_ipv6_dns_update_style();

#Modify override_ipv6_dns_update_style value
$object->override_ipv6_dns_update_style('true');
```

override_ipv6_gss_tsig_keys( )

The override_ipv6_gss_tsig_keys attribute controls whether the ipv6_gss_tsig_keys method values of the member are used, instead of the Grid default.

The override_ipv6_gss_tsig_keys attribute can be specified explicitly. It is also set implicitly when ipv6_gss_tsig_keys is set to a defined value.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

Set the parameter to "true" to override GSS-TSIG keys. Set the parameter to "false" to inherit the grid-level GSS-TSIG keys.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Getting override_ipv6_gss_tsig_keys
my $override_ipv6_gss_tsig_keys = $member_dhcp->override_ipv6_gss_tsig_keys();
#Modyfying override_ipv6_gss_tsig_keys
$member_dhcp->override_ipv6_gss_tsig_keys("true");
```

override_ipv6_lease_scavenge( )

The override_ipv6_lease_scavenge attribute controls whether the ipv6_enable_lease_scavenge, ipv6_lease_scavenge_time and ipv6_remember_expired_client_association methods values of the member are used, instead of the Grid default.

The override_ipv6_lease_scavenge attribute can be specified explicitly. It is also set implicitly when one of ipv6_enable_lease_scavenge, ipv6_lease_scavenge_time and ipv6_remember_expired_client_association is set to a defined value.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

Set the parameter to "true" to override settings for scavenging of DHCPv6 leases. Set the parameter to "false" to inherit settings for scavenging of DHCPv6 leases.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Getting override_ipv6_lease_scavenge
my $override_ipv6_lease_scavenge = $memberdhcp->override_ipv6_lease_scavenge();
#Modyfying override_ipv6_lease_scavenge
$memberdhcp->override_ipv6_lease_scavenge("true");
```

override_logic_filters( )

The override_logic_filters attribute controls whether the logic_filters value of the Member DHCP is used, instead of the Grid default.

The override_logic_filters attribute can be specified explicitly. It is also set implicitly when logic_filters is set to a defined value.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

Set the parameter to "true" to override the Grid-level setting for logic_filters. Set the parameter to "false" to inherit the Grid-level setting for logic_filters.

The default value of this parameter is "false".

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Getting override_logic_filters
my $override_logic_filters=$memberdhcp->override_logic_filters( );
#Modifying override_logic_filters
$memberdhcp->override_logic_filters("true");
```

name( )

Use this method to retrieve the grid name.

Parameter

None.

Returns

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get name
my $name = $memberdhcp->name();
```

nextserver( )

Use this method to set or retrieve the next server value of a member DHCP server. This value is the IP address or name of the boot file server on which the boot file is stored.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

This option is overridden independently from Infoblox::Grid::Member::DHCP->bootfile() and Infoblox::Grid::Member::DHCP->bootserver().

Parameter

The boot file server IP address or name in FQDN format. The FQDN consists of the host name followed by the domain name (example: abc.com). A boot file server name can have a maximum of 256 bytes.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get nextserver
my $nextserver = $memberdhcp->nextserver();
#Modify nextserver
$memberdhcp->nextserver("1.1.1.1");
#Un-override nextserver
$memberdhcp->nextserver(undef);
```

microsoft_code_page( )

Use this method to set or retrieve the Microsoft client DHCP IPv4 code page value of a grid member. This value is the hostname translation code page for Microsoft DHCP IPv4 clients and overrides the grid-level Microsoft DHCP IPv4 client code page.

Setting microsoft_code_page() to undef removes the override, enabling the appliance to use the grid-level microsoft_code_page setting.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

Microsoft DHCP client code page in string format. Valid values are the same as those for ipv6_microsoft_code_page. See ipv6_microsoft_code_page().

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get microsoft_code_page
my $microsoft_code_page = $memberdhcp->microsoft_code_page();
#Modify microsoft_code_page
$memberdhcp->microsoft_code_page("Cyrillic (ISO-8859-5)");
#Do not override microsoft_code_page
$memberdhcp->microsoft_code_page("None");
```

options( )

Use this method to set or retrieve the options value of a member DHCP server. DHCP options provide network configuration settings and various services available on the network for DHCP member. This setting overrides the grid level setting.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

The valid value is an array reference that contains Infoblox::DHCP::Option object(s).

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get options
my $options = $memberdhcp->options();
#Modify options
my $option1 = Infoblox::DHCP::Option->new(
    name  => "lease-time",
    value => 50
);

my $option2 = Infoblox::DHCP::Option->new(
    name  => "routers",
    value => "1.4.1.1, 2.2.2.2"
);

my $option3 = Infoblox::DHCP::Option->new(
    name  => "nameservers",
    value => "1.1.1.1, 2.2.2.2"
);

my $option4 = Infoblox::DHCP::Option->new(
    name  => "domain-name",
    value => "domainname.com",
);

my $option5 = Infoblox::DHCP::Option->new(
    name  => "broadcast",
    value => "255.0.0.0",
);

$memberdhcp->options([$option1, $option2, $option3, $option4, $option5]);
```

option60_match_rules( )

Use this method to set or retrieve the option60_match_rules value of a member DHCP server. An Option60MatchRule object allows specifying match rules for Option 60 globally. This setting overrides the grid level setting.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

The valid value is an array reference that contains Infoblox::DHCP::Option60MatchRule object(s).

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get option60_match_rules
my $options = $memberdhcp->option60_match_rules();
#Modify option60_match_rules
my $option60_match_rule = Infoblox::DHCP::Option60MatchRule->new(
                                  "option_space" => "infospace",
                                  "match_value" => "Nortel-i2004-A",
                                  "is_substring" => "true",
                                  "substring_offset" => "8",
                                  "substring_length" => "14"
                                );

$memberdhcp->option60_match_rules([$option60_match_rule]);
#Do not override option60_match_rules
$memberdhcp->option60_match_rules(undef);
```

override_ddns_ttl( )

The override_ddns_ttl attribute controls whether the ddns_ttl value in the member DHCP server is used, instead of the Grid default.

The override_ddns_ttl attribute can be specified explicitly. It is also set implicitly when ddns_ttl is set to a defined value.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

Set the parameter to "true" to override the grid-level setting for ddns_ttl. Set the parameter to "false" to inherit the grid-level setting for ddns_ttl.

The default value of this parameter is "false".

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Getting override_ddns_ttl
my $override_ddns_ttl=$memberdhcp->override_ddns_ttl( );
#Modifying override_ddns_ttl
$memberdhcp->override_ddns_ttl("true");
```

override_enable_dhcpv6_service( )

The override_enable_dhcpv6_service attribute controls whether the enable_dhcpv6_service value in the DHCP member is used, instead of the Grid default.

The override_enable_dhcpv6_service attribute can be specified explicitly. It is also set implicitly when enable_dhcpv6_service is set to a defined value.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

Set the parameter to "true" to override the grid-level setting for enable_dhcpv6_service. Set the parameter to "false" to inherit the grid-level setting for enable_dhcpv6_service.

The default value of this parameter is "false".

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Getting override_enable_dhcpv6_service
my $override_enable_dhcpv6_service=$memberdhcp->override_enable_dhcpv6_service( );
#Modifying override_enable_dhcpv6_service
$memberdhcp->override_enable_dhcpv6_service("true");
```

override_enable_fingerprint( )

Use this method to set or retrieve the override_enable_fingerprint flag.

Parameter

Boolean to enable or disable the override fibgerprint flag for the member. The default value is "false".

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get override_enable_fingerprint
my $status = $memberdhcp->override_enable_fingerprint();
#Modify override_enable_fingerprint
$memberdhcp->override_enable_fingerprint("true");
```

override_hostname_rewrite( )

Use this method to set or retrieve the override_hostname_rewrite flag.

Parameter

Boolean to enable or disable the override hostname rewrite policy flag for the member. The default value is "false".

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get override_hostname_rewrite
my $status = $memberdhcp->override_hostname_rewrite();
#Modify override_hostname_rewrite
$memberdhcp->override_hostname_rewrite("true");
```

override_ignore_id( )

The override_ignore_id controls whether the ignore_id in the DHCP member is used, instead of the grid-level default.

The override_ignore_id can be specified explicitly. It is also set implicitly when ignore_id is set to a defined value.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

Set the parameter to "true" to override the grid-level setting for ignore_id and ignore_mac_addresses. Set the parameter to "false" to inherit the grid-level setting for ignore_id and ignore_mac_addresses.

The default value of this parameter is "false".

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get attrubute value
my $value=$memberdhcp->override_ignore_id();
#Modifying attribute value
$memberdhcp->override_ignore_id('true');
```

override_ipv6_ddns_domainname( )

The override_ipv6_ddns_domainname attribute controls whether the ipv6_ddns_domainname value in the DHCP member is used, instead

of the Grid default.

The override_ipv6_ddns_domainname attribute can be specified explicitly. It is also set implicitly when ipv6_ddns_domainname is set to a defined value.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

Set the parameter to "true" to override the grid-level setting for ipv6_ddns_domainname. Set the parameter to "false" to inherit the grid-level setting for ipv6_ddns_domainname.

The default value of this parameter is "false".

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Getting override_ipv6_ddns_domainname
my $override_ipv6_ddns_domainname=$memberdhcp->override_ipv6_ddns_domainname( );
#Modifying override_ipv6_ddns_domainname
$memberdhcp->override_ipv6_ddns_domainname("true");
```

override_ipv6_ddns_hostname( )

The override_ipv6_ddns_hostname attribute controls whether the ipv6_ddns_hostname value in the DHCP member is used, instead of the Grid default.

The override_ipv6_ddns_hostname attribute can be specified explicitly. It is also set implicitly when ipv6_ddns_hostname is set to a defined value.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

Set the parameter to "true" to override the grid-level setting for ipv6_ddns_hostname. Set the parameter to "false" to inherit the grid-level setting for ipv6_ddns_hostname.

The default value of this parameter is "false".

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Getting override_ipv6_ddns_hostname
my $override_ipv6_ddns_hostname=$memberdhcp->override_ipv6_ddns_hostname( );
#Modifying override_ipv6_ddns_hostname
$memberdhcp->override_ipv6_ddns_hostname("true");
```

override_ipv6_ddns_enable_option_fqdn( )

Use this method to set or retrieve the override_ipv6_ddns_enable_option_fqdn value of a Grid Member DHCP object.

The override_ipv6_ddns_enable_option_fqdn attribute can be specified explicitly. It is also set implicitly when ipv6_ddns_enable_option_fqdn is set to a defined value.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

Set the parameter to "true" to override the upper-level setting for ipv6_ddns_enable_option_fqdn. Set the parameter to "false" to inherit the upper-level setting for ipv6_ddns_enable_option_fqdn.

The default value of this parameter is "false".

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get override_ipv6_ddns_enable_option_fqdn
my $override_ipv6_ddns_enable_option_fqdn = $memberdhcp->override_ipv6_ddns_enable_option_fqdn();
#Modify override_ipv6_ddns_enable_option_fqdn
$memberdhcp->override_ipv6_ddns_enable_option_fqdn("true");
```

override_ipv6_ddns_ttl( )

The override_ipv6_ddns_ttl attribute controls whether the ipv6_ddns_ttl value in the DHCP member is used, instead of the Grid default.

The override_ipv6_ddns_ttl attribute can be specified explicitly. It is also set implicitly when ipv6_ddns_ttl is set to a defined value.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

Set the parameter to "true" to override the grid-level setting for ipv6_ddns_ttl. Set the parameter to "false" to inherit the grid-level setting for ipv6_ddns_ttl.

The default value of this parameter is "false".

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Getting override_ipv6_ddns_ttl
my $override_ipv6_ddns_ttl=$memberdhcp->override_ipv6_ddns_ttl( );
#Modifying override_ipv6_ddns_ttl
$memberdhcp->override_ipv6_ddns_ttl("true");
```

override_ipv6_domain_name( )

Use this method to set or retrieve the override_ipv6_domain_name value of a Grid Member DHCP object.

The override_ipv6_domain_name attribute can be specified explicitly. It is also set implicitly when ipv6_domain_name is set to a defined value.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

Set the parameter to "true" to override the upper-level setting for ipv6_domain_name. Set the parameter to "false" to inherit the upper-level setting for ipv6_domain_name.

The default value of this parameter is "false".

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get override_ipv6_domain_name
my $override_ipv6_domain_name = $memberdhcp->override_ipv6_domain_name();
#Modify override_ipv6_domain_name
$memberdhcp->override_ipv6_domain_name("true");
```

override_ipv6_domain_name_servers( )

Use this method to set or retrieve the override_ipv6_domain_name_servers value of a Grid Member DHCP object.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

Set the parameter to "true" to override the upper-level setting for ipv6_domain_name_servers. Set the parameter to "false" to inherit the upper-level setting for ipv6_domain_name_servers.

The default value of this parameter is "false".

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get override_ipv6_domain_name_servers
my $override_ipv6_domain_name_servers = $memberdhcp->override_ipv6_domain_name_servers();
#Modify override_ipv6_domain_name_servers
$memberdhcp->override_ipv6_domain_name_servers("true");
```

override_ipv6_enable_ddns( )

The override_ipv6_enable_ddns attribute controls whether the ipv6_enable_ddns value in the DHCP member is used, instead of the Grid default.

The override_ipv6_enable_ddns attribute can be specified explicitly. It is also set implicitly when ipv6_enable_ddns is set to a defined value.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

Set the parameter to "true" to override the grid-level setting for ipv6_enable_ddns. Set the parameter to "false" to inherit the grid-level setting for ipv6_enable_ddns.

The default value of this parameter is "false".

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Getting override_ipv6_enable_ddns
my $override_ipv6_enable_ddns=$memberdhcp->override_ipv6_enable_ddns( );
#Modifying override_ipv6_enable_ddns
$memberdhcp->override_ipv6_enable_ddns("true");
```

override_ipv6_enable_gss_tsig( )

The override_ipv6_enable_gss_tsig attribute controls whether the ipv6_enable_gss_tsig value in the DHCP member is used, instead of the Grid default.

The override_ipv6_enable_gss_tsig attribute can be specified explicitly. It is also set implicitly when ipv6_enable_gss_tsig is set to a defined value.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

Set the parameter to "true" to override the grid-level setting for ipv6_enable_gss_tsig. Set the parameter to "false" to inherit the grid-level setting for ipv6_enable_gss_tsig.

The default value of this parameter is "false".

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Getting override_ipv6_enable_gss_tsig
my $override_ipv6_enable_gss_tsig=$memberdhcp->override_ipv6_enable_gss_tsig( );
#Modifying override_ipv6_enable_gss_tsig
$memberdhcp->override_ipv6_enable_gss_tsig("true");
```

override_ipv6_enable_retry_updates( )

The override_ipv6_enable_retry_updates attribute controls whether the ipv6_enable_retry_updates value of the DHCP member is used, instead of the Grid default.

The override_ipv6_enable_retry_updates attribute can be specified explicitly. It is also set implicitly when ipv6_enable_retry_updates is set to a defined value.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

Set the parameter to "true" to override the grid-level setting for ipv6_enable_retry_updates. Set the parameter to "false" to inherit the grid-level setting for ipv6_enable_retry_updates.

The default value of this parameter is "false".

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Getting override_ipv6_enable_retry_updates
my $override_ipv6_enable_retry_updates=$memberdhcp->override_ipv6_enable_retry_updates( );
#Modifying override_ipv6_enable_retry_updates
$memberdhcp->override_ipv6_enable_retry_updates("true");
```

override_ipv6_generate_hostname( )

The override_ipv6_generate_hostname attribute controls whether the ipv6_generate_hostname value in the DHCP member is used, instead of the Grid default.

The override_ipv6_generate_hostname attribute can be specified explicitly. It is also set implicitly when ipv6_generate_hostname is set to a defined value.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

Set the parameter to "true" to override the grid-level setting for ipv6_generate_hostname. Set the parameter to "false" to inherit the grid-level setting for ipv6_generate_hostname.

The default value of this parameter is "false".

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Getting override_ipv6_generate_hostname
my $override_ipv6_generate_hostname=$memberdhcp->override_ipv6_generate_hostname( );
#Modifying override_ipv6_generate_hostname
$memberdhcp->override_ipv6_generate_hostname("true");
```

override_ipv6_microsoft_code_page( )

The override_ipv6_microsoft_code_page attribute controls whether the ipv6_microsoft_code_page value in the DHCP member is used, instead of the Grid default.

The override_ipv6_microsoft_code_page attribute can be specified explicitly. It is also set implicitly when ipv6_microsoft_code_page is set to a defined value.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

Set the parameter to "true" to override the grid-level setting for ipv6_microsoft_code_page. Set the parameter to "false" to inherit the grid-level setting for ipv6_microsoft_code_page.

The default value of this parameter is "false".

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Getting override_ipv6_microsoft_code_page
my $override_ipv6_microsoft_code_page=$memberdhcp->override_ipv6_microsoft_code_page( );
```

```
#Modifying override_ipv6_microsoft_code_page
$memberdhcp->override_ipv6_microsoft_code_page("true");
```

override_ipv6_options( )

The override_ipv6_options attribute controls whether the ipv6_options value in the DHCP member is used, instead of the Grid default.

The override_ipv6_options attribute can be specified explicitly. It is also set implicitly when ipv6_options is set to a defined value.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

Set the parameter to "true" to override the grid-level setting for ipv6_options. Set the parameter to "false" to inherit the grid-level setting for ipv6_options.

The default value of this parameter is "false".

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Getting override_ipv6_options
my $override_ipv6_options=$memberdhcp->override_ipv6_options( );
#Modifying override_ipv6_options
$memberdhcp->override_ipv6_options("true");
```

override_ipv6_recycle_leases( )

The override_ipv6_recycle_leases attribute controls whether the ipv6_recycle_leases value in the DHCP member is used, instead of the Grid default.

The override_ipv6_recycle_leases attribute can be specified explicitly. It is also set implicitly when ipv6_recycle_leases is set to a defined value.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

Set the parameter to "true" to override the grid-level setting for ipv6_recycle_leases. Set the parameter to "false" to inherit the grid-level setting for ipv6_recycle_leases.

The default value of this parameter is "false".

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Getting override_ipv6_recycle_leases
my $override_ipv6_recycle_leases=$memberdhcp->override_ipv6_recycle_leases( );
#Modifying override_ipv6_recycle_leases
$memberdhcp->override_ipv6_recycle_leases("true");
```

override_ipv6_update_dns_on_lease_renewal( )

The override_ipv6_update_dns_on_lease_renewal attribute controls whether the ipv6_update_dns_on_lease_renewal value in the DHCP member is used, instead of the Grid default.

The override_ipv6_update_dns_on_lease_renewal attribute can be specified explicitly. It is also set implicitly when ipv6_update_dns_on_lease_renewal is set to a defined value.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

Set the parameter to "true" to override the grid-level setting for ipv6_update_dns_on_lease_renewal. Set the parameter to "false" to inherit the grid-level setting for ipv6_update_dns_on_lease_renewal.

The default value of this parameter is "false".

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
    #Getting override_ipv6_update_dns_on_lease_renewal
    my $override_ipv6_update_dns_on_lease_renewal=$memberdhcp-
>override_ipv6_update_dns_on_lease_renewal( );
    #Modifying override_ipv6_update_dns_on_lease_renewal
    $memberdhcp->override_ipv6_update_dns_on_lease_renewal("true");
```

override_lease_per_client_settings( )

Use this method to set or retrieve the override_lease_per_client_settings flag.

The override_lease_per_client_settings attribute can be specified explicitly. It is also set implicitly when lease_per_client_settings is set to a defined value.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

Set the parameter to "true" to override the grid-level setting for lease_per_client_settings. Set the parameter to "false" to inherit the grid-level setting for lease_per_client_settings.

The default value of this parameter is "false".

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get override_lease_per_client_settings value
my $value = $object->override_lease_per_client_settings();

#Modify override_lease_per_client_settings value
$object->override_lease_per_client_settings("true");
```

override_lease_scavenge_time( )

The override_lease_scavenge_time attribute controls whether the lease_scavenge_time value in the member DHCP server is used, instead of the Grid default.

The override_lease_scavenge_time attribute can be specified explicitly. It is also set implicitly when lease_scavenge_time is set to a defined value.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

Set the parameter to "true" to override the grid-level setting for lease_scavenge_time. Set the parameter to "false" to inherit the grid-level setting for lease_scavenge_time.

The default value of this parameter is "false".

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
    #Getting override_lease_scavenge_time
    my $override_lease_scavenge_time=$memberdhcp->override_lease_scavenge_time( );
    #Modifying override_lease_scavenge_time
    $memberdhcp->override_lease_scavenge_time("true");
```

override_log_lease_events( )

The override_log_lease_events attribute controls whether the log_lease_events value in the member DHCP server is used, instead of the Grid default.

The override_log_lease_events attribute can be specified explicitly. It is also set implicitly when log_lease_events is set to a defined value.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

Set the parameter to "true" to override the grid-level setting for log_lease_events. Set the parameter to "false" to inherit the grid-level setting for log_lease_events.

The default value of this parameter is "false".

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Getting override_log_lease_events
my $override_log_lease_events=$memberdhcp->override_log_lease_events( );
#Modifying override_log_lease_events
$memberdhcp->override_log_lease_events("true");
```

override_preferred_lifetime( )

The override_preferred_lifetime attribute controls whether the preferred_lifetime value in the DHCP member is used, instead of the Grid default.

The override_preferred_lifetime attribute can be specified explicitly. It is also set implicitly when preferred_lifetime is set to a defined value.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

Set the parameter to "true" to override the grid-level setting for preferred_lifetime. Set the parameter to "false" to inherit the grid-level setting for preferred_lifetime.

The default value of this parameter is "false".

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Getting override_preferred_lifetime
my $override_preferred_lifetime=$memberdhcp->override_preferred_lifetime( );
#Modifying override_preferred_lifetime
$memberdhcp->override_preferred_lifetime("true");
```

override_update_dns_on_lease_renewal( )

The override_update_dns_on_lease_renewal attribute controls whether the update_dns_on_lease_renewal value in the member DHCP server is used, instead of the Grid default.

The override_update_dns_on_lease_renewal attribute can be specified explicitly. It is also set implicitly when update_dns_on_lease_renewal is set to a defined value.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

Set the parameter to "true" to override the grid-level setting for update_dns_on_lease_renewal. Set the parameter to "false" to inherit the grid-level setting for update_dns_on_lease_renewal.

The default value of this parameter is "false".

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Getting override_update_dns_on_lease_renewal
my $override_update_dns_on_lease_renewal=$memberdhcp->override_update_dns_on_lease_renewal( );
#Modifying override_update_dns_on_lease_renewal
$memberdhcp->override_update_dns_on_lease_renewal("true");
```

override_valid_lifetime( )

The override_valid_lifetime attribute controls whether the valid_lifetime value in the Grid Member DHCP is used, instead of the upper-level default.

The override_valid_lifetime attribute can be specified explicitly. It is also set implicitly when valid_lifetime is set to a defined value.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

Set the parameter to "true" to override the grid-level setting for valid_lifetime. Set the parameter to "false" to inherit the upper-level setting for valid_lifetime.

The default value of this parameter is "false".

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Getting override_valid_lifetime
my $override_valid_lifetime=$memberdhcp->override_valid_lifetime( );
#Modifying override_valid_lifetime
$memberdhcp->override_valid_lifetime("true");
```

ping_count( )

Use this method to set or retrieve the ping_count value of a member DHCP server. This value specifies the number of pings that the Infoblox appliance sends to an IP address to verify that it is not in use.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

Number of pings. Range is from 0 to 10, where 0 disables pings. The default value is 1.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get ping_count
my $ping_count = $memberdhcp->ping_count();
#Modify ping_count
$memberdhcp->ping_count(4);
```

ping_timeout( )

Use this method to set or retrieve the ping_timeout value of a member DHCP server. This value indicates the number of seconds the appliance waits for a response to its ping.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

Ping timeout value in seconds. Valid values are 0.1, 0.5, 1, 2, 3, 4 and 5 seconds. The default value is 1.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get ping_timeout
my $ping_timeout = $memberdhcp->ping_timeout();
```

```
#Modify ping_timeout
$memberdhcp->ping_timeout(3);
```

preferred_lifetime( )

Use this method to set or retrieve preferred lifetime value.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Setting this method to a defined value implicitly sets the override_preferred_lifetime method to "true". Setting the parameter to undefined causes the appliance to use the Grid default and automatically resets the override_preferred_lifetime attribute to "false".

Note that when preferred_lifetime is set to a defined value and override_dpreferred_lifetime is set to "false", the last operation takes precedence. Thus the sequence $object->preferred_lifetime(12345); $object->override_preferred_lifetime("false"); will set override_preferred_lifetime to "false", and the sequence $object->override_preferred_lifetime("false"); $object->preferred_lifetime(12345); will result in override_preferred_lifetime="true".

Parameter

Preferred lifetime value in seconds. The default value is 27000.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get preferred_lifetime
my $preferred_lifetime = $memberdhcp->preferred_lifetime();
#Modify preferred_lifetime
$memberdhcp->preferred_lifetime(12345);
```

pxe_lease_time( )

Use this method to set or retrieve the pxe_lease_time value of a member DHCP server. You can specify the duration of time it takes a host to connect to a boot server, such as a TFTP server, and download the file it needs to boot.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

A 32-bit integer (range from 0 to 4294967295) that specifies the lease time in seconds. Default value is 43200 seconds.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get pxe_lease_time
my $pxe_lease_time = $memberdhcp->pxe_lease_time();
#Modify pxe_lease_time
$memberdhcp->pxe_lease_time(200);
```

recycle_leases( )

Use this method to set or retrieve the recycle_leases value of a member DHCP server. If you enable this check box, and then delete a DHCP range, the appliance stores active leases from this range up to one week after the leases expires.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

Specify "true" to set recycle_leases flag or "false" to deactivate/unset it.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get recycle_leases
my $recycle_leases = $memberdhcp->recycle_leases();
```

```
#Modify recycle_leases
$memberdhcp->recycle_leases("true");
```

retry_ddns_updates( )

Use this method to set or retrieve the retry_ddns_updates value of a member DHCP server. This value indicates whether the DHCP server makes repeated attempts to send DDNS updates to a DNS server.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

Specify "true" to enable the DHCP server to retry DDNS updates or "false" to disable this feature.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get retry_ddns_updates
my $retry_ddns_updates = $memberdhcp->retry_ddns_updates();
#Modify retry_ddns_updates
$memberdhcp->retry_ddns_updates("true");
```

syslog_facility( )

Use this method to set or retrieve the syslog_facility value of a member DHCP server. The syslog facility is the location on the syslog server to which you want to sort the syslog messages.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

Valid values are: `"daemon", "local0", "local1", "local2", "local3", "local4", "local5", "local6", "local7"`.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get syslog_facility
my $syslog_facility = $memberdhcp->syslog_facility();
#Modify syslog_facility
$memberdhcp->syslog_facility("daemon");
```

update_dns_on_lease_renewal( )

The update_dns_on_lease_renewal attribute controls whether the DHCP server updates DNS when a DHCP lease is renewed.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Setting this method to a defined value implicitly sets the override_update_dns_on_lease_renewal method to "true". Setting the parameter to undefined causes the appliance to use the Grid default and automatically resets the override_update_dns_on_lease_renewal attribute to "false".

Note that when update_dns_on_lease_renewal is set to a defined value and override_update_dns_on_lease_renewal is set to "false", the last operation takes precedence. Thus the sequence $object->update_dns_on_lease_renewal("true"); $object->override_update_dns_on_lease_renewal("false"); will set override_update_dns_on_lease_renewal to "false", and the sequence $object->override_update_dns_on_lease_renewal("false"); $object->update_dns_on_lease_renewal("true"); will result in override_update_dns_on_lease_renewal="true".

Parameter

Specify "true" to set the update_dns_on_lease_renewal flag or "false" to deactivate/unset it.

The default value of this parameter is "false".

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get update_dns_on_lease_renewal
my $update_dns_on_lease_renewal=$memberdhcp->update_dns_on_lease_renewal( );
#Modify update_dns_on_lease_renewal
$memberdhcp->update_dns_on_lease_renewal("true");
```

valid_lifetime( )

Use this method to set or retrieve the value of valid_lifetime for Grid Member DHCP.

Setting this method to a defined value implicitly sets the override_valid_lifetime method to "true". Setting the parameter to undefined causes the appliance to use the upper-level default and automatically resets the override_valid_lifetime attribute to "false".

Note that when valid_lifetime is set to a defined value and override_valid_lifetime is set to "false", the last operation takes precedence. Thus the sequence $object->valid_lifetime(20000); $object->override_valid_lifetime("false"); will set override_valid_lifetime to "false", and the sequence $object->override_valid_lifetime("false"); $object->valid_lifetime(20000); will result in override_valid_lifetime="true".

Parameter

Valid lifetime value in seconds. The default value is 43200.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get valid_lifetime
my $valid_lifetime = $memberdhcp->valid_lifetime();
#Modify valid_lifetime
$memberdhcp->valid_lifetime(12345);
```

SAMPLE CODE

The following sample code demonstrates different operations that can be applied to a member DHCP object, such as get and modify an object. This sample code also includes error handling for the operations.

#Preparation prior to operations on a member DHCP object

```
#PROGRAM STARTS: Include all the modules that will be used
use strict;
use Infoblox;

#Create a session to the Infoblox appliance
my $session = Infoblox::Session->new(
    master   => "192.168.1.2",
    username => "admin",
    password => "infoblox"
);
unless ($session) {
   die("Construct session failed: ",
       Infoblox::status_code() . ":" . Infoblox::status_detail());
}
print "Session created successfully\n";
```

#Get and modify the member DHCP property object.

```
#Get the member DHCP object. Considering the id grid member named infoblox.localdomain was previously
created.
my @object = $session->get(
    object => "Infoblox::Grid::Member::DHCP",
    name   => "infoblox.localdomain"
    )
    or die("get member DHCP failed: ",
         $session->status_code(), $session->status_detail());

#Modify member DHCP object.
my $option1 = Infoblox::DHCP::Option->new(
    name  => "lease-time",
    value => 50
);
```

```
my $option2 = Infoblox::DHCP::Option->new(
    name  => "domain-name",
    value => "domainname.com",
);

unless (scalar(@object) == 0) {
    my $object = $object[0];

    if ($object) {
        $object->enable_dhcp("true");
        $object->authority("true");
        $object->enable_ddns("true");
        $object->ddns_update_fixed_addresses("true");
        $object->ddns_use_option81("true");
        $object->ddns_server_always_updates("true");
        $object->ddns_generate_hostname("true");
        $object->pxe_lease_time(1000);
        $object->options([$option1, $option2]);
        $object->microsoft_code_page("Cyrillic (ISO-5589-5)");

        #Un-override bootfile
        $object->bootfile(undef);

        #Un-override bootserver
        $object->bootserver(undef);

        #Un-override nextserver
        $object->nextserver(undef);

        #Applying the changes
        $session->modify($object)
            or die("Modify member DHCP failed: ",
                $session->status_code(), $session->status_detail());

    }
     } else {
    print "No grid member DHCP found.";
     }

####PROGRAM ENDS####
```

AUTHOR

Infoblox Inc. http://www.infoblox.com/

SEE ALSO

Infoblox::Grid::Member::DHCP,Infoblox::DHCP::Option,Infoblox::Grid::KerberosKey,Infoblox::Session, Infoblox::Session->get(),Infoblox::Session->modify(), Infoblox::DHCP::Filter::MAC, Infoblox::DHCP::DDNS

COPYRIGHT

Copyright (c) 2017 Infoblox Inc.

Infoblox::DHCP::Range - DHCP Range object.

- NAME
- DESCRIPTION
- CONSTRUCTOR
- SESSION METHODS
  - Infoblox::Session->add( )
  - Infoblox::Session->get( )
  - Infoblox::Session->modify( )
  - Infoblox::Session->remove( )
  - Infoblox::Session->search( )
- METHODS
  - always_update_dns( )
  - bootfile( )
  - bootserver( )
  - comment( )
  - cloud_info( )
  - ddns_domainname( )
  - ddns_generate_hostname( )
  - deny_all_clients( )
  - deny_bootp( )
  - disable( )
  - discovery_basic_poll_setting( )
  - discovery_member( )
  - discover_now_status( )
  - discovery_blackout_setting( )
  - enable_discovery( )
  - enable_immediate_discovery( )
  - email_list( )
  - enable_ddns( )
  - enable_dhcp_thresholds( )
  - enable_ifmap_publishing( )
  - enable_email_warnings( )
  - enable_known_clients_option( )
  - enable_snmp_warnings( )
  - enable_unknown_clients_option( )
  - end_addr( )
  - endpoint_sources( )
  - exclude( )
  - extattrs( )
  - extensible_attributes( )
  - failover_assoc( )
  - filters( )
  - ignore_id( )
  - ignore_mac_addresses( )
  - is_split_scope( )
  - known_clients_option( )
  - lease_scavenge_time( )
  - name( )
  - next_available_ip( )
  - ignore_dhcp_option_list_request( )
  - member( )
  - ms_ad_user_data( )
  - ms_options( )
  - network( )
  - network_view( )
  - nextserver( )
  - options( )
  - logic_filters( )
  - override_ddns_domainname( )
  - override_enable_ifmap_publishing( )
  - override_blackout_setting( )
  - override_ignore_id( )
  - override_lease_scavenge_time( )
  - override_logic_filters( )
  - override_subscribe_settings( )
  - override_update_dns_on_lease_renewal( )
  - port_control_blackout_setting
  - pxe_lease_time( )
  - range_high_water_mark( )
  - range_high_water_mark_reset( )
  - range_low_water_mark( )
  - range_low_water_mark_reset( )

- restart_if_needed( )
- recycle_leases( )
- same_port_control_discovery_blackout( )
- server_association_type( )
- split_member
- split_scope_exclusion_percent
- start_addr( )
- subscribe_settings( )
- template( )
- unknown_clients_option( )
- update_dns_on_lease_renewal( )

- SAMPLE CODE
- AUTHOR
- SEE ALSO
- COPYRIGHT

NAME

Infoblox::DHCP::Range - DHCP Range object.

DESCRIPTION

A DHCP range defines the specified range of IP addresses in a network. A DHCP range should be added for a network so the Infoblox appliance can assign IP addresses within that specified range to DHCP clients. If the client is on a network that is assigned a DHCP range, the device distributes an available IP address from that range to the DHCP client, or to a DHCP relay agent if the request came through an agent. The DHCP range should also be assigned with a device. If devices are in a grid, the particular member serving DHCP for the DHCP range must be specified. If the server is an independent device, this device must be specified as the member that serves the DHCP range.

CONSTRUCTOR

```
 my $dhcp_range = Infoblox::DHCP::Range->new(

     end_addr                        => $ipv4addr,                                  #Required
     network                         => $network,                                   #Optional
     network_view                    => $NetworkView,                               #Optional / Default is
system-defined default network view
     start_addr                      => $ipv4addr,                                  #Required
     always_update_dns               => "true" | "false",                           #Optional / Default is
"false"
     bootfile                        => $filename | undef,                          #Optional / Default is
undefined
     bootserver                      => $ipv4addr | $fqdn | undef,                 #Optional / Default is
undefined
     comment                         => $string,                                    #Optional / Default is
undefined
     ddns_domainname                 => $string | undef,                            #Optional / Default is
undefined
     ddns_generate_hostname          => "true" | "false",                           #Optional / Default is
undefined
     deny_all_clients                => "true" | "false",                           #Optional / Default is
"false"
     deny_bootp                      => "true" | "false" | undef,                   #Optional / Default is
undefined
     disable                         => "true" | "false",                           #Optional / Default is
"false"
     email_list                      => [ "admin\@test.com",...] | undef,           #Optional / Default is
undefined
     enable_ddns                     => "true" | "false",                           #Optional / Default is
undefined
     enable_dhcp_thresholds          => "true" | "false",                           #Optional / Default is
"false"
     enable_email_warnings           => "true" | "false",                           #Optional / Default is
"false"
     enable_ifmap_publishing         => "true" | "false",                           #Optional / Default is
undefined
     enable_known_clients_option     => "true" | "false",                           #Optional / Default is
"false"
     enable_snmp_warnings            => "true" | "false",                           #Optional / Default is
"false"
     enable_unknown_clients_option   => "true" | "false",                           #Optional / Default is
"false"
     exclude                         => [$ExclusionRange1,$ExclusionRange2,...],    #Optional / Default is
empty
     extattrs                        => { $string => $extattr, ... },               #Optional / Default is
undefined
     extensible_attributes           => { $string => $string | $num, $string => [ $string | $num, ... ],
... } #Optional / Default is undefined
```

```
     failover_assoc                  => $string,                                  #Optional / Default is
undefined
     filters                         => [$MAC, $RelayAgent, $Option...],          #Optional / Default is
empty
     ignore_dhcp_option_list_request => "true" | "false" | undef,                 #Optional / Default is
undefined
     ignore_id                       => 'NONE' | 'CLIENT' | 'MACADDR',            #Optional / Default is
'NONE'
     ignore_mac_addresses            => [ "mac_addr", ... ] | undef               #Optional / Default is
undefined
     known_clients_option            => "Allow" | "Deny",                         #Optional / Default is
undefined
     lease_scavenge_time             => $number,                                  #Optional / Default is
-1
     member                          => $Member,                                  #Optional / Default is
undefined
     ms_options                      => [$MSOption1,$MSOption2,...],              #Optional / Default is
empty
     name                            => $string,                                  #Optional / Default is
undefined
     nextserver                      => $ipv4addr | $fqdn | undef,                #Optional / Default is
undefined
     options                         => [$Option1,$Option2,...],                  #Optional / Default is
empty
     logic_filters                   => [ $filter_name | $filter_obj, ...],       #Optional / Default is
undefined
     override_logic_filters          => "true' | 'false",                         #Optional / Default
value is "false"
     override_ddns_domainname        => "true" | "false",                         #Optional / Default
value is "false"
     override_enable_ifmap_publishing => "true" | "false",                        #Optional / Default
value is "false"
     override_lease_scavenge_time    => "true" | "false",                         #Optional / Default
value is "false"
     override_update_dns_on_lease_renewal  => "true" | "false",                   #Optional / Default
value is "false"
     override_ignore_id              => "true" | "false",                         #Optional / Default
value is "false"
     pxe_lease_time                  => $num,                                     #Optional / Default is
undefined
     range_high_water_mark           => $num,                                     #Optional / Default is
95
     range_high_water_mark_reset     => $num,                                     #Optional / Default is
95
     range_low_water_mark            => $num,                                     #Optional / Default is 0
     range_low_water_mark_reset      => $num,                                     #Optional / Default is 0
     recycle_leases                  => "true" | "false",                         #Optional / Default is
undefined
     server_association_type         => "NONE" | "MEMBER" | "FAILOVER" | "MS_SERVER" #Optional / Default is
undefined
     split_member                    => $member,                                  #Optional / Default is
undefined
     split_scope_exclusion_percent   => $num                                      #Optional / Default is
undefined
     template                        => $string,                                  #Optional / Default is
undefined
     unknown_clients_option          => "Allow" | "Deny",                         #Optional / Default is
undefined
     update_dns_on_lease_renewal     => "true" | "false",                         #Optional / Default
value is "false"
     discovery_blackout_setting      => $blackout,                                #Optional
     port_control_blackout_setting   => $blackout,                                #Optional
     override_blackout_setting       => "true" | "false",                         #Optional / Default
value is "false"
     same_port_control_discovery_blackout => "true" | "false",                    #Optional / Default
value is "false"
     subscribe_settings          => $subscribe_settings,                          #Optional / Default is
undefined
     override_subscribe_settings => 'true' | 'false',                             #Optional / Default is
'false'
      );
```

You cannot set both extattrs and extensible_attributes attributes at the same time.

SESSION METHODS

This section describes all the methods in an Infoblox::Session module that you can apply to a DHCP range object.

Infoblox::Session->add( )

Use this method to add a DHCP range object to the Infoblox appliance. See Infoblox::Session->add() for parameters and return values.

Example

```
#Construct an object
my $dhcp_range = Infoblox::DHCP::Range->new(
        end_addr                => "10.0.0.10",
        network                 => "10.0.0.0/8",
        start_addr              => "10.0.0.1",
        bootfile                => "bootfile1.com",
        bootserver              => "someserver.com",
        comment                 => "add range",
        ddns_generate_hostname  => "true",
        deny_all_clients        => "true",
        deny_bootp              => "true",
        disable                 => "true",
        enable_ddns             => "true",
        exclude                 => [$exclusion_range],
        filters                 => [$filter],
        member                  => $memberdhcp,
        network                 => $nview,
        nextserver              => "3.3.3.3",
        options                 => [$option1],
        pxe_lease_time          => "3600",
        recycle_leases          => "true",
);

# Submit for addition
my $response = $session->add( $dhcp_range );
```

Infoblox::Session->get( )

Use this method to retrieve all the matching objects from the Infoblox appliance. See Infoblox::Session->get() for parameters and return values.

'discover_now_status' will not be returned by default, use 'return_methods' to request it if needed.

Key References

```
Apply the following attributes to get a specific DHCP range object:

start_addr             - Optional. Starting IP address of DHCP range within the network.
network_view           - Optional. Network view in which the DHCP range is located. By default, all
network views are searched.
extattrs               - Optional. A hash reference containing extensible attributes.
extensible_attributes  - Optional. A hash reference containing extensible attributes.
failover_assoc         - Optional. The name of the associated failover peer.
member                 - Optional. The member or Microsoft server serving the range. The valid value
is an Infoblox::DHCP::Member, Infoblox::DHCP::MSServer, Infoblox::Grid::Member::DHCP or
Infoblox::Grid::MSServer::DHCP object.
server_association_type - Optional. The server association type for the range.
```

Examples

```
my @retrieved_objs = $session->get(
                     object     => "Infoblox::DHCP::Range",
                     start_addr => "10.0.0.1" );

my @retrieved_objs = $session->get(
                     object       => "Infoblox::DHCP::Range",
                     start_addr   => "10.0.0.1",
                     network_view => "default" );

my @retrieved_objs = $session->get(
                     object               => "Infoblox::DHCP::Range",
                     extensible_attributes => { 'Site' => 'Santa Clara' });
```

Infoblox::Session->modify( )

Use this method to modify a DHCP range object in the Infoblox appliance. See Infoblox::Session->modify() for parameters and return values.

Example

```
# Use method to modify the comment.
$dhcp_range->comment("this is a modified comment");
# Submit modification
my $response = $session->modify( $dhcp_range );
```

Infoblox::Session->remove( )

Use this method to remove a DHCP range object from the Infoblox appliance. See Infoblox::Session->remove() for parameters and return values.

To remove a specific object, first use `get()` or `search()` to retrieve the specific DHCP range object, and then submit this object for removal.

Example

```
# Get the objects with the same name
my @retrieved_objs = $session->get(
    object       => "Infoblox::DHCP::Range",
    start_addr   => "10.0.0.1",
    newtork_view => "default" );
# Find the desired object from the retrieved list.
my $desired_dhcp_range = $retrieved_objs[0];
# Submit for removal
my $response = $session->remove( $desired_dhcp_range );
```

Infoblox::Session->search( )

Use this method to search for DHCP range objects in the Infoblox appliance. See Infoblox::Session->search() for parameters and return values.

'discover_now_status' will not be returned by default, use 'return_methods' to request it if needed.

Key References

```
 Apply the following attributes to search for a DHCP range object:

  network      - Optional. Network in CIDR format (regular expression).
  network_view - Optional. Network view in which the DHCP range is located. By default, all network
views are searched.
  start_addr   - Optional. IPv4 address in dotted decimal notation (regular expression).
  end_addr     - Optional. IPv4 address in dotted decimal notation (regular expression).
  comment      - Optional. Comment in string format (regular expression).
  extattrs     - Optional. A hash reference containing extensible attributes.
  extensible_attributes - Optional. A hash reference containing extensible attributes.
  failover_assoc        - Optional. The name of the associated failover peer.
  member                - Optional. The member or Microsoft server serving the range. The valid value
is an Infoblox::DHCP::Member, Infoblox::DHCP::MSServer, Infoblox::Grid::Member::DHCP or
Infoblox::Grid::MSServer::DHCP object.
  server_association_type - Optional. The server association type for the range.
```

For more information about searching extensible attributes, see Infoblox::Grid::ExtensibleAttributeDef/Searching Extensible Attributes.

Examples

```
# search for DHCP range object with start address having last octet as 1
my @retrieved_objs = $session->search(
    object       => "Infoblox::DHCP::Range",
    network      => "10.0.0.0/.",
    network_view => "default",
    start_addr   => ".*1",
    comment      => ".*comment" );

# search for DHCP range objects defining "Santa Clara" for "Site" extensible attribute
my @retrieved_objs = $session->search(
    object               => "Infoblox::DHCP::Range",
    extensible_attributes => { 'Site' => 'Santa Clara' });
```

METHODS

This section describes all the methods that you can use to configure and retrieve the attribute values of a DHCP range.

always_update_dns( )

Use this method to set or retrieve the always_update_dns flag of a DHCP range.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

This flag only applies if the range is associated with a Microsoft (r) server. If it is set to "true", the DHCP server will always update DNS, regardless of the requests from the DHCP clients. If this flag is set to false, clients will be allowed to update DNS when they request it.

Parameter

Specify "true" to set the always_update_dns flag or "false" to deactivate/unset it. The default value is "false".

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get always_update_dns
my $always_update_dns = $dhcp_range->always_update_dns();
#Modify always_update_dns
$dhcp_range->always_update_dns("false");
```

bootfile( )

Use this method to set or retrieve a bootfile name which is the name of the file the client must download.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

This option is overridden independently from Infoblox::DHCP::Range->bootserver() and Infoblox::DHCP::Range->nextserver().

Parameter

The boot file name in string format with a maximum of 128 bytes. The default value is undefined.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get bootfile
my $bootfile = $dhcp_range->bootfile();
#Modify bootfile
$dhcp_range->bootfile("bootfile2.com");
#Un-override bootfile
$dhcp_range->bootfile(undef);
```

bootserver( )

Use this method to set or retrieve a bootserver name which is the name of the server where the boot file is stored.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

This option is overridden independently from Infoblox::DHCP::Range->bootfile() and Infoblox::DHCP::Range->nextserver().

Parameter

The boot server IP address or name in FQDN (Fully Qualified Domain Name) format. The FQDN consists of the host name followed by the domain name (example: abc.com). A boot server name can have a maximum of 256 bytes. The default value is undefined.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get bootserver
my $bootserver = $dhcp_range->bootserver();
#Modify bootserver
$dhcp_range->bootserver("anotherserver.com");
#Un-override bootserver
$dhcp_range->bootserver(undef);
```

comment( )

Use this method to set or retrieve a descriptive comment.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

Desired comment in string format with a maximum of 256 bytes. The default value is undefined.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get comment
my $comment = $dhcp_range->comment();
#Modify comment
$dhcp_range->comment("Modifying the DHCP range comment");
```

cloud_info( )

Use this method to set or retrieve cloud API related information for the Infoblox::DHCP::Range object.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

The valid value is a Infoblox::Grid::CloudAPI::Info object.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
# Get cloud_info
my $cloud_info = $dhcp_range->cloud_info();
# Modify cloud_info
$dhcp_range->cloud_info($cloud_info_object);
```

ddns_domainname( )

Use this method to set or retrieve the ddns_domainname value of a DHCP Range object.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Setting this method to a defined value implicitly sets the override_ddns_domainname method to "true". Setting the parameter to undefined causes the appliance to use the grid default and automatically resets the override_ddns_domainname attribute to "false".

Note that when ddns_domainname is set to a defined value and override_dddns_domainname is set to "false", the last operation takes precedence. Thus the sequence $object->ddns_domainname("testdomain"); $object->override_ddns_domainname("false"); will set override_ddns_domainname to "false", and the sequence $object->override_ddns_domainname("false"); $object->ddns_domainname("testdomain"); will result in override_ddns_domainname="true".

Parameter

The dynamic DNS domain name in string format.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get ddns_domainname
my $ddns_domainname = $dhcp_range->ddns_domainname();
#Modify ddns_domainname
$dhcp_range->ddns_domainname("test_domain.com");
```

ddns_generate_hostname( )

Use this method to set or retrieve the ddns_generate_hostname flag of a DHCP range.

Enable this method to allow only the DHCP server to generate a hostname.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

Specify "true" to set the ddns_generate_hostname flag or "false" to deactivate/unset it. The default value is undefined.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get ddns_generate_hostname
my $ddns_generate_hostname = $dhcp_range->ddns_generate_hostname();
#Modify ddns_generate_hostname
$dhcp_range->ddns_generate_hostname("false");
```

deny_all_clients( )

Use this method to set or retrieve the deny_all_clients flag of a DHCP range.

Enable this method to deny all lease requests for the DHCP range.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

Specify "true" to set the deny_all_clients flag or "false" to deactivate/unset it. The default value for this field is false.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get deny_all_clients
my $deny_all_clients = $dhcp_range->deny_all_clients();
#Modify deny_all_clients
$dhcp_range->deny_all_clients("false");
```

deny_bootp( )

Use this method to set or retrieve the deny_bootp flag of a DHCP range.

Enable this method to deny all BOOTP requests.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

Specify "true" to set the deny_bootp flag or "false" to deactivate/unset it. The default value is false which indicates that this attribute inherit the upper-level setting.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get deny_bootp
my $deny_bootp = $dhcp_range->deny_bootp();
#Modify deny_bootp
$dhcp_range->deny_bootp("false");
```

disable( )

Use this method to set or retrieve the disable flag of a DHCP range.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

Specify "true" to set the disable flag or "false" to deactivate/unset it. The default value for this field is false.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get disable
my $disable = $dhcp_range->disable();
#Modify disable
$dhcp_range->disable("false");
```

discovery_basic_poll_setting( )

Use this method to set or retrieve the discovery_basic_poll_setting value of a DHCP Range.

Include the specified parameter to set the attribute value.

Parameter

The valid value is object of the Infoblox::Grid::Discovery::BasicPollSettings manpage.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get discovery_basic_poll_setting
my $discovery_basic_poll_setting = $dhcp_range->discovery_basic_poll_setting();
#Modify discovery_basic_poll_setting
$dhcp_range->discovery_basic_poll_setting($poll_setting);
```

discovery_member( )

Use this method to set or retrieve the discovery_member value of a DHCP Range.

Parameter

The discovery member name in string format.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get discovery_member
my $discovery_member = $dhcp_range->discovery_member();
#Modify discovery_member
$dhcp_range->discovery_member("test.localdomain");
```

discover_now_status( )

Use this method to retrieve the current discovery status. This attribute is read-only.

'discover_now_status' will not be returned by default, use 'return_methods' to request it if needed.

Parameter

None.

Returns

The method returns the attribute value that can be 'COMPLETE', 'FAILED', 'NONE', 'PENDING' or 'RUNNING'.

Example

```
#Get discover_now_status
my $discover_now_status = $dhcp_range->discover_now_status();
```

discovery_blackout_setting( )

Use this method to set or retrieve the discovery blackout setting.

Setting this method to a defined value implicitly sets the override_blackout_setting method to "true". If port_control_blackout_setting is undefined, setting the parameter to undefined causes the appliance to use the grid default and automatically resets the override_blackout_setting attribute to "false".

Note that when discovery_blackout_setting is set to a defined value and override_blackout_setting is set to "false", the last operation takes precedence. Thus the sequence $network->discovery_blackout_setting($blackout); $network->override_blackout_setting("false");

will set override_blackout_setting to "false", and the sequence $network->override_blackout_setting("false"); $network->discovery_blackout_setting($blackout); will result in override_blackout_setting="true".

Parameter

Valid value is an Infoblox::Grid::Discovery::Properties::BlackoutSetting object.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get discovery_blackout_setting
my $discovery_blackout_setting = $dhcp_range->discovery_blackout_setting();
#Modify discovery_blackout_setting
$dhcp_range->discovery_blackout_setting($blackout);
```

enable_discovery( )

Use this method to set or retrieve the enable_discovery value of a DHCP Range. This setting overrides the member level settings.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Setting this method to a defined value implicitly sets the override_enable_discovery method to "true". Setting the parameter to undefined causes the appliance to use the default from the upper level and automatically resets the override_enable_discovery attribute to "false".

Enable this flag if you want to set discovery for this object.

Parameter

Specify "true" to set the enable_discovery flag or "false" to deactivate/unset it. The default value is undefined.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get enable_discovery
my $enable_discovery = $dhcp_range->enable_discovery();
#Modify enable_discovery
$dhcp_range->enable_discovery("false");
```

enable_immediate_discovery( )

Use this method to set or retrieve enable_immediate_discovery value of a DHCP Range.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

Specify "true" to set the enable_immediate_discovery flag or "false" to deactivate/unset it. The default value is undefined.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get enable_immediate_discovery
my $enable_immediate_discovery = $dhcp_range->enable_immediate_discovery();
#Modify enable_immediate_discovery
$dhcp_range->enable_immediate_discovery("false");
```

email_list( )

Use this method to set or retrieve the email list to which the appliance sends notifications when DHCP address usage for the grid crosses a threshold.

If you enter email addresses in the email_list, then the appliance sends DHCP threshold alarms to the email addresses.

Parameter

The valid value is an array reference that contains string objects.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get email_list
my @email_list = $range->email_list();
#Modify email_list
$range->email_list( [ "admin\@test.com" ] );
```

enable_ddns( )

Use this method to set or retrieve the enable_ddns flag of a DHCP range.

Enable this method to allow dynamic DNS updates.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

Specify "true" to set the enable_ddns flag or "false" to deactivate/unset it. The default value is undefined.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get enable_ddns
my $enable_ddns = $dhcp_range->enable_ddns();
#Modify enable_ddns
$dhcp_range->enable_ddns("false");
```

enable_dhcp_thresholds( )

Use this method to set or retrieve DHCP thresholds at the DHCP address range level. These thresholds are the high and low watermarks. If DHCP range usage goes above or below these percentage values, the Infoblox appliance generates a syslog message and sends an SNMP trap, if configured.

Enable this flag to set DHCP threshold values.

Parameter

Specify "true" to set the enable_dhcp_thresholds flag or "false" to deactivate/unset it. The default value is undefined.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get enable_dhcp_thresholds
my $enable_dhcp_thresholds = $range->enable_dhcp_thresholds();
#Modify enable_dhcp_thresholds
$range->enable_dhcp_thresholds("false");
```

enable_ifmap_publishing( )

Use this method to set or retrieve the ifmap publishing option of a DHCP Range object. This setting overrides the member level settings.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

Specify "true" to set ifmap publishing option or "false" to deactivate/unset it. The default value is 'false', which indicates that this attribute inherits the member level setting.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
# Get enable_ifmap_publishing
my $enable_ifmap_publishing = $dhcp_range->enable_ifmap_publishing()
# Modify enable_ifmap_publishing
$dhcp_range->enable_ifmap_publishing("true");
# Un-override enable_ifmap_publishing
$dhcp_range->enable_ifmap_publishing(undef);
```

enable_email_warnings( )

Use this method to enable the appliance to send email notifications when DHCP address usage crosses a high or low watermark threshold.

Enable this flag if you want the appliance to send DHCP threshold warnings to the "email_list" provided when DHCP threshold is enabled.

Parameter

Specify "true" to set the enable_email_warnings flag or "false" to deactivate/unset it. The default value is undefined.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get enable_email_warnings
my $enable_email_warnings = $range->enable_email_warnings();
#Modify enable_email_warnings
$range->enable_email_warnings("false");
```

enable_known_clients_option( )

Use this method to set or retrieve the enable_known_clients_option flag of a DHCP range.

You must set the enable_known_clients_option flag to true in order to use the known_clients_option attribute.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

Specify "true" to set the enable_known_clients_option flag or "false" to deactivate/unset it. The default value is "false".

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get enable_known_clients_option
my $enable_known_clients_option = $dhcp_range->enable_known_clients_option();
#Modify enable_known_clients_option
$dhcp_range->enable_known_clients_option("false");
```

enable_snmp_warnings( )

Use this method to enable the appliance to send an SNMP trap when DHCP address usage crosses a watermark threshold.

If the flag is enabled, then the appliance sends SNMP traps when DHCP address usage crosses a watermark threshold when DHCP threshold is enabled.

Parameter

Specify "true" to set the enable_snmp_warnings flag or "false" to deactivate/unset it. The default value is undefined.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get enable_snmp_warnings
my $enable_snmp_warnings = $range->enable_snmp_warnings();
#Modify enable_snmp_warnings
$range->enable_snmp_warnings("false");
```

enable_unknown_clients_option( )

Use this method to set or retrieve the enable_unknown_clients_option flag of a DHCP range.

You must set the enable_unknown_clients_option flag to true in order to use the unknown_clients_option attribute.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

Specify "true" to set the enable_unknown_clients_option flag or "false" to deactivate/unset it. The default value is "false".

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get enable_unknown_clients_option
my $enable_unknown_clients_option = $dhcp_range->enable_unknown_clients_option();
#Modify enable_unknown_clients_option
$dhcp_range->enable_unknown_clients_option("false");
```

end_addr( )

Use this method to set or retrieve the end address of the DHCP range.

An end address is the last IP address in the DHCP range available for the clients.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

Specify the end address of the DHCP range in IPv4 address format. An IPv4 address is a 32-bit number in dotted decimal notation. It consists of four 8-bit groups of decimal digits separated by decimal points (example: 192.168.1.2).

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get end_addr
my $end_addr = $dhcp_range->end_addr();
#Modify end_addr
$dhcp_range->end_addr("10.0.0.20");
```

endpoint_sources( )

Use this method to retrieve the list of endpoints that provide object data. This is a read-only attribute.

Omit the parameter to retrieve the attribute value.

Parameter

None

Returns

The valid return value is an array of Infoblox::CiscoISE::Endpoint objects.

Example

```
#Get endpoint_sources value
my $endpoint_sources = $object->endpoint_sources();
```

exclude( )

Use this method to set or retrieve the exclusion range of IP addresses in the DHCP range which should not be assigned to clients.

If static addresses in the middle of an DHCP range are assigned to certain hosts, those addresses can be excluded from the DHCP range using this method.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

Valid value is an array reference that contains defined Infoblox::DHCP::ExclusionRange objects. The default value is empty.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get exclude
my $ref_exclude = $dhcp_range->exclude();
#Modify exclude
$dhcp_range->exclude([$exl1]); #$exl1 is an Infoblox::DHCP::ExclusionRange object
```

extattrs( )

Use this method to set or retrieve the extensible attributes associated with a DHCP range object.

Parameter

Valid value is a hash reference containing the names of extensible attributes and their associated values (Infoblox::Grid::Extattr objects).

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get extattrs
my $ref_extattrs = $dhcp_range->extattrs();
#Modify extattrs
$dhcp_range->extattrs({ 'Site' => $extattr1, 'Administrator' => $extattr2 });
```

extensible_attributes( )

Use this method to set or retrieve the extensible attributes associated with a DHCP range object

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

For valid values for extensible attributes, see Infoblox::Grid::ExtensibleAttributeDef/Extensible Attribute Values.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get extensible attributes
my $ref_extensible_attributes = $dhcp_range->extensible_attributes();
#Modify extensible attributes
$dhcp_range->extensible_attributes({ 'Site' => 'Santa Clara', 'Administrator' => [ 'Peter', 'Tom' ]
});
```

failover_assoc( )

Use this method to set or retrieve the failover peer association name of the DHCP range.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

If the attribute is set, the server_association_type attribute will also be set to "FAILOVER" if the name is defined, or to "NONE" if the member is also undefined.

Parameter

Specify the failover peer association name in string format. The default value is undefined.

Use an empty string ('') to remove a failover peer association from the DHCP range. Using undef is deprecated and may be changed in the future.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get failover_assoc
my $failover_assoc = $dhcp_range->failover_assoc();
#Modify failover_assoc
$dhcp_range->failover_assoc("peer2");
#Remove failover_assoc (using undef for removal is deprecated)
$dhcp_range->failover_assoc("");
```

filters( )

Use this method to set or retrieve the filters of the DHCP range.

Filters are used to screen the address assignments if the addresses in the DHCP range needs to be assigned to specific hosts.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

Valid value is an array reference that contains defined Infoblox::DHCP::FilterRule::MAC, Infoblox::DHCP::FilterRule::NAC, Infoblox::DHCP::FilterRule::RelayAgent, Infoblox::DHCP::FilterRule::Option, Infoblox::DHCP::FilterRule::Fingerprint objects. The default value is empty.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get filters
my $ref_filters = $dhcp_range->filters();
#Modify filters
$dhcp_range->filters([$exl1]); #$exl1 is an Infoblox::DHCP::FilterRule::MAC,
Infoblox::DHCP::FilterRule::NAC, Infoblox::DHCP::FilterRule::RelayAgent,
Infoblox::DHCP::FilterRule::Option or Infoblox::DHCP::FilterRule::Fingerprint object.
```

ignore_id( )

Use this method to set or retrieve the ignore_id type of a DHCP range object. Indicates whether the appliance will ignore DHCP client IDs or MAC addresses. This setting overrides the upper-level settings.

Parameter

Valid values are 'NONE', 'CLIENT', or 'MACADDR'. The default is 'NONE'.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
# Get attribute value
$value = $dhcp_range->ignore_id();
# Modify attribute value
$dhcp_range->ignore_id('CLIENT');
#Un-override ignore_id
$dhcp_range->ignore_id(undef);
```

ignore_mac_addresses( )

Use this method to set or retrieve the ignore_mac_addresses list of a DHCP range object. A list of MAC addresses the appliance will ignore. This setting overrides the upper-level settings.

If you enter MAC addresses in the ignore_mac_addresses, then the appliance will ignore them. If you enter empty array, then the

appliance will ignore any MAC address.

Parameter

The valid value is an array reference that contains string objects.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
# Get attribute value
$value = $dhcp_range->ignore_mac_addresses();
# Modify attribute value
$dhcp_range->ignore_mac_addresses(["11::11::11::11::11::11"]);
```

is_split_scope( )

Use this method to retrieve the is_split_scope flag of the DHCP range. This flag is "true" when the range is part of a split scope.

Parameter

None

Returns

The method returns the attribute value.

Example

```
#Get is_split_scope
my $is_split_scope = $dhcp_range->is_split_scope();
```

known_clients_option( )

Use this method to set or retrieve the known_clients_option option of a DHCP range.

Set this option to "Allow" or "Deny" to allow or prevent the allocation of an IP address from this pool to any client that has a host declaration.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

Specify "Allow" or "Deny" to set the known_clients_option flag value. The default value is undefined.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get known_clients_option
my $known_clients_option = $dhcp_range->known_clients_option();
#Modify known_clients_option
$dhcp_range->known_clients_option("Deny");
```

lease_scavenge_time( )

Use this method to set or retrieve the lease_scavenge_time value of a DHCP Range object.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Setting this method to a defined value implicitly sets the override_lease_scavenge_time method to "true". Setting the parameter to undefined causes the appliance to use the default from the upper level and automatically resets the override_lease_scavenge_time attribute to "false".

Note that when lease_scavenge_time is set to a defined value and override_lease_scavenge_time is set to "false", the last operation takes precedence. Thus the sequence $object->lease_scavenge_time(604800); $object->override_lease_scavenge_time("false"); will set override_lease_scavenge_time to "false", and the sequence $object->override_lease_scavenge_time("false"); $object->lease_scavenge_time(604800); will result in override_lease_scavenge_time="true".

Parameter

An integer that specifies the period of time, in seconds, that free and backup leases remain in the database before they are automatically deleted. To disable lease scavenging, set the parameter to -1. The minimum positive value must be greater than 86400 seconds (1 day). The default value of this parameter is -1, which means that lease scavenging is disabled.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get lease_scavenge_time
my $lease_scavenge = $dhcp_range->lease_scavenge_time();
#Modify lease_scavenge_time
$dhcp_range->lease_scavenge_time(30 * 24 * 60 * 60);
```

name( )

Use this method to set or retrieve a descriptive name for the range.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

The desired name in string format with a maximum of 256 bytes. The default value is undefined.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get name
my $name = $dhcp_range->name();
#Modify name
$dhcp_range->name("external");
```

next_available_ip( )

Use this method to get the next available IP `address(es)` in the DHCP range object that was retrieved. Note that to use this method, the object MUST be retrieved via a get or search call, or created with an add call.

Parameter

The method accepts the following optional parameters:

```
  requested_num - The number of IP addresses requested. It must be a value between 1 and 20,
inclusive.
  excluded - A reference to an array of addresses to be excluded from the results.
```

Returns

This method returns the next available IP `address(es)` in the range. If requested_num is not specified, a string with the next available address will be returned; otherwise, the method will return a reference to an array of addresses.

Within a DHCP Range object, the next available IP `address(es)` are the first addresses in this range that match all of the following rules:

```
   * Are not part of the passed 'excluded' parameter (if specified).
   * There is no fixed address or host address record matching this IP address
   * There is no unmanaged record matching this IP address
   * The IP address is not contained in any excluded range of this DHCP range
   * There is no active lease matching this IP address (Deleted leases in the recycle bin are not
considered.)
```

Example

```
# Retrieve a particular range
my @retrieved_objs = $session->get(
                        object     => "Infoblox::DHCP::Range",
                        start_addr => "10.0.0.20"
);

$dhcp_range = $retrieved_objs[0];

# Print the next available IP address in the range
print $dhcp_range->next_available_ip() . "\n";
```

```
# This will retrieve the next 10 available addresses and print the first one.
my $addressesref = $dhcp_range->next_available_ip(requested_num => 10);
print @$addressesref[0] . "\n";

# This will retrieve the next 10 available addresses and exclude a couple.
my $addressesref = $dhcp_range->next_available_ip(requested_num => 10, excluded => ["10.0.0.21",
"10.0.0.22"]);
```

ignore_dhcp_option_list_request( )

Use this method to set or retrieve the ignore_dhcp_option_list_request flag of a DHCP shared network object. If this flag is set to false all the defined DHCP options will be returned to the client. This setting overrides the upper-level settings.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

Specify "true" to set the ignore_dhcp_option_list_request flag or "false" to deactivate/unset it. The default value is undefined which indicates that this attribute inherit the upper-level setting..

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
# Get ignore_dhcp_option_list_request
my $ignore_dhcp_option_list_request = $dhcp_range->ignore_dhcp_option_list_request();
# Un-override ignore_dhcp_option_list_request
$dhcp_range->ignore_dhcp_option_list_request(undef);
# Modify ignore_dhcp_option_list_request
$dhcp_range->ignore_dhcp_option_list_request("true");
```

member( )

Use this method to set or retrieve the member or Microsoft server that serves the DHCP range.

The member or Microsoft server that serves DHCP for this IP address range.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

If the attribute is set, the server_association_type attribute will also be set to "MEMBER" if setting member to an Infoblox::DHCP::Member object, to "MS_SERVER" if setting member to an Infoblox::DHCP::MSServer object, or to "NONE" if failover_assoc is also empty.

Parameter

The valid value is a defined Infoblox::DHCP::Member or Infoblox::DHCP::MSServer object. The default value is empty.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get member
my $ref_member = $dhcp_range->member();
#Modify member
$dhcp_range->member($member1); #$member is an Infoblox::DHCP::Member object
```

ms_ad_user_data( )

Use this method to retrieve Microsoft Active Directory users related information. This is a read-only attribute.

Parameter

None

Returns

The valid return value is an Infoblox::Grid::MSServer::AdUser::Data object.

Example

```
#Get ms_ad_user_data
my $ms_ad_user_data = $dhcp_range->ms_ad_user_data();
```

ms_options( )

Use this method to set or retrieve the Microsoft server custom DHCP options that are associated with this DHCP range.

Note: You can set only options that are included in both the Microsoft server and NIOS predefined option definitions. Options that are not common to both are read only and cannot be modified.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

The valid value is an array reference that contains defined Infoblox::DHCP::MSOption objects. The default value is empty.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get options
my $ref_options = $dhcp_range->ms_options();
#Modify options
$dhcp_range->ms_options([$option12]); #$option12 is an Infoblox::DHCP::MSOption object
```

network( )

Use this method to set or retrieve the DHCP network of the address range.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

Specify the address and netmask of network in CIDR format (example: 10.0.0.0/8).

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get network
my $network = $dhcp_range->network();
```

network_view( )

Use this method to set or retrieve the network view of the DHCP range.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

The default value is the "default" network view, which means the DHCP range is in the default network view.

Parameter

Valid value is a defined Infoblox::DHCP::View object. The default network view value is the system-defined default network view object.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get network view
my $nview = $dhcp_range->network_view();
#Modify network view, with an Infoblox::DHCP::View object
$dhcp_range->network_view($nview);
```

nextserver( )

Use this method to set or retrieve the nextserver IP address which is the boot file server where the boot file is stored.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

This option is overridden independently from Infoblox::DHCP::Range->bootfile() and Infoblox::DHCP::Range->bootserver().

Parameter

The next server IP address or name in FQDN (Fully Qualified Domain Name) format. The FQDN consists of the host name followed by the domain name (example: abc.com). A next server name can have a maximum of 256 bytes. The default value is undefined.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get nextserver
my $nextserver = $dhcp_range->nextserver();
#Modify nextserver
$dhcp_range->nextserver("4.4.4.4");
#Un-override nextserver
$dhcp_range->nextserver(undef);
```

options( )

Use this method to set or retrieve the options of the DHCP range.

DHCP options describe network configuration settings and various services available on the network. Setting the DHCP options for the DHCP range overrides the options set at the network level.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

Valid value is an array reference that contains defined Infoblox::DHCP::Option objects. The default value is empty.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get options
my $ref_options = $dhcp_range->options();
#Modify options
$dhcp_range->options([$option12]); #$option12 is an Infoblox::DHCP::Option object
```

logic_filters( )

Use this method to set or retrieve all the logic (ordered) filters associated with the DHCP Range.

Filters are used to screen address assignments in a DHCP Range if the addresses are to be assigned to specific hosts.

Setting this method to a defined value implicitly sets the override_logic_filters method to "true". Setting the parameter to undefined causes the appliance to use the grid default and automatically resets the override_logic_filters attribute to "false".

Note that when logic_filters is set to a defined value and override_dlogic_filters is set to "false", the last operation takes precedence. Thus the sequence $object->logic_filters($filters); $object->override_logic_filters("false"); will set override_logic_filters to "false", and the sequence $object->override_logic_filters("false"); $object->logic_filters($filters); will result in override_logic_filters="true".

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

The valid value is an array reference that contains either the Infoblox::DHCP::Filter::MAC, Infoblox::DHCP::Filter::NAC, Infoblox::DHCP::Filter::Option objects or the names of the corresponding filters. The filters in the list must satisfy the following conditions:

Filter name must be globally unique.
Filters without expressions must be at the end of the list.
The filters should not be disabled.

The default value of the parameter is undefined.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get logic_filters
my $ref_logic_filters = $dhcp_range->logic_filters();
#Modify logic_filters
$dhcp_range->logic_filters(["filter_1","filter_2",$exl1]); #$exl1 is an Infoblox::DHCP::Filter::MAC,
Infoblox::DHCP::Filter::NAC or Infoblox::DHCP::Filter::Option object.
```

override_ddns_domainname( )

The override_ddns_domainname attribute controls whether the ddns_domainname value in the DHCP range is used, instead of the grid default.

The override_ddns_domainname attribute can be specified explicitly. It is also set implicitly when ddns_domainname is set to a defined value.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

Set the parameter to "true" to override the grid-level setting for ddns_domainname. Set the parameter to "false" to inherit the grid-level setting for ddns_domainname.

The default value of this parameter is "false".

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Getting override_ddns_domainname
my $override_ddns_domainname=$dhcp_range->override_ddns_domainname( );
#Modifying override_ddns_domainname
$dhcp_range->override_ddns_domainname("true");
```

override_enable_ifmap_publishing( )

The override_enable_ifmap_publishing controls whether the enable_ifmap_publishing in the DHCP Range is used, instead of the grid default.

The override_enable_ifmap_publishing can be specified explicitly. It is also set implicitly when enable_ifmap_publishing is set to a defined value.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

Set the parameter to "true" to override the grid-level setting for enable_ifmap_publishing. Set the parameter to "false" to inherit the grid-level setting for enable_ifmap_publishing.

The default value of this parameter is "false".

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Getting override_enable_ifmap_publishing
my $override_enable_ifmap_publishing=$dhcp_range->override_enable_ifmap_publishing( );
#Modifying override_enable_ifmap_publishing
$dhcp_range->override_enable_ifmap_publishing("true");
```

override_blackout_setting( )

The override_blackout_setting attribute controls whether the discovery_blackout_setting and port_control_blackout_setting values in the object are used, instead of the grid default.

The override_blackout_setting attribute can be specified explicitly. It is also set implicitly when discovery_blackout_setting or port_control_blackout_setting is set to a defined value.

Parameter

Specify "true" to set the override_blackout_setting flag or "false" to deactivate/unset it. The default value is "false".

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
# Get attribute value
my $value = $dhcp_range->override_blackout_setting();
# Modify attribute value
$dhcp_range->override_blackout_setting('true');
```

override_ignore_id( )

The override_ignore_id controls whether the ignore_id in the DHCP Range is used, instead of the upper-level default.

The override_ignore_id can be specified explicitly. It is also set implicitly when ignore_id is set to a defined value.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

Set the parameter to "true" to override the upper-level setting for ignore_id and ignore_mac_addresses. Set the parameter to "false" to inherit the upper-level setting for ignore_id and ignore_mac_addresses.

The default value of this parameter is "false".

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get attrubute value
my $value=$dhcp_range->override_ignore_id();
#Modifying attribute value
$dhcp_range->override_ignore_id('true');
```

override_lease_scavenge_time( )

The override_lease_scavenge_time attribute controls whether the lease_scavenge_time value in the DHCP range is used, instead of the upper-level default.

The override_lease_scavenge_time attribute can be specified explicitly. It is also set implicitly when lease_scavenge_time is set to a defined value.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

Set the parameter to "true" to override the upper-level setting for lease_scavenge_time. Set the parameter to "false" to inherit the setting for lease_scavenge_time from the upper level.

The default value of this parameter is "false".

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Getting override_lease_scavenge_time
my $override_lease_scavenge_time=$dhcp_range->override_lease_scavenge_time( );
#Modifying override_lease_scavenge_time
$dhcp_range->override_lease_scavenge_time("true");
```

override_logic_filters( )

The override_logic_filters attribute controls whether the logic_filters value in the DHCP Range is used, instead of the Grid default.

The override_logic_filters attribute can be specified explicitly. It is also set implicitly when logic_filters is set to a defined value.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

Set the parameter to "true" to override the Grid-level setting for logic_filters. Set the parameter to "false" to inherit the Grid-level setting for logic_filters.

The default value of this parameter is "false".

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Getting override_logic_filters
my $override_logic_filters=$dhcp_range->override_logic_filters( );
#Modifying override_logic_filters
$dhcp_range->override_logic_filters("true");
```

override_subscribe_settings( )

The override_subscribe_settings attribute controls whether the subscribe_settings method values of the member are used, instead of the Grid default.

The override_subscribe_settings attribute can be specified explicitly. It is also set implicitly when subscribe_settings is set to a defined value.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

Set the parameter to "true" to override the Grid-level setting for subscribe_settings. Set the parameter to "false" to inherit the Grid-level setting for subscribe_settings.

The default value of this parameter is "false".

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get override_subscribe_settings value
my $override_subscribe_settings = $object->override_subscribe_settings( );
#Modify override_subscribe_settings value
$object->override_subscribe_settings("true");
```

override_update_dns_on_lease_renewal( )

The override_update_dns_on_lease_renewal attribute controls whether the update_dns_on_lease_renewal value in the DHCP range is used, instead of the grid default.

The override_update_dns_on_lease_renewal attribute can be specified explicitly. It is also set implicitly when update_dns_on_lease_renewal is set to a defined value.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

Set the parameter to "true" to override the grid-level setting for update_dns_on_lease_renewal. Set the parameter to "false" to inherit the grid-level setting for update_dns_on_lease_renewal.

The default value of this parameter is "false".

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Getting override_update_dns_on_lease_renewal
my $override_update_dns_on_lease_renewal=$dhcp_range->override_update_dns_on_lease_renewal( );
#Modifying override_update_dns_on_lease_renewal
$dhcp_range->override_update_dns_on_lease_renewal("true");
```

port_control_blackout_setting

Use this method to set or retrieve the port control blackout setting.

Setting this method to a defined value implicitly sets the override_blackout_setting method to "true". If discovery_blackout_setting is undefined, setting the parameter to undefined causes the appliance to use the grid default and automatically resets the override_blackout_setting attribute to "false".

Note that when port_control_blackout_setting is set to a defined value and override_blackout_setting is set to "false", the last operation takes precedence. Thus the sequence $network->port_control_blackout_setting($blackout); $network->override_blackout_setting("false"); will set override_blackout_setting to "false", and the sequence $network->override_blackout_setting("false"); $network->port_control_blackout_setting($blackout); will result in override_blackout_setting="true".

Parameter

Valid value is an Infoblox::Grid::Discovery::Properties::BlackoutSetting object.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get port_control_blackout_setting
my $port_control_blackout_setting = $dhcp_range->port_control_blackout_setting();
#Modify port_control_blackout_setting
$dhcp_range->port_control_blackout_setting($blackout);
```

pxe_lease_time( )

Use this method to set or retrieve PXE Lease Time value of the DHCP range.

The Preboot Execution Environment Lease Time value is used by some hosts to boot remotely from a server.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

A 32-bit integer (range from 0 to 4294967295) represents the duration in seconds that the record is cached. Zero indicates that the record should not be cached. The default value is undefined.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get pxe_lease_time
my $pxe_lease_time = $dhcp_range->pxe_lease_time();
#Modify pxe_lease_time
$dhcp_range->pxe_lease_time("7200");
```

range_high_water_mark( )

Use this method to set or retrieve the range_high_water_mark value. If the percentage of allocated addresses exceeds this value, an SNMP trap is sent, unless this is a range served by a Microsoft server.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

A number that specifies the percentage of allocated addresses. The range is from 1 to 100. The high watermark value must be greater than the low watermark value. The default value is 95.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get range_high_water_mark
my $range_high_water_mark = $range->range_high_water_mark();
#Modify range_high_water_mark
$range->range_high_water_mark(80);
```

range_high_water_mark_reset( )

Use this method to set or retrieve the range_high_water_mark_reset value. If the percentage of allocated addresses drops below this value, a corresponding SNMP trap is reset.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

A number that specifies the percentage of allocated addresses. The range is from 1 to 100. The high watermark reset value must be lower than the high watermark value. The default value is 85.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get range_high_water_mark_reset
my $range_high_water_mark_reset = $range->range_high_water_mark_reset();
#Modify range_high_water_mark_reset
$range->range_high_water_mark_reset(70);
```

range_low_water_mark( )

Use this method to set or retrieve the range_low_water_mark value. If the percentage of allocated addresses drops below this watermark, an SNMP trap is sent, unless this is a range served by a Microsoft server.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

A number that specifies the percentage of allocated addresses. The range is from 1 to 100. The low watermark value must be lower than the high watermark value. The default low watermark number is 0.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get range_low_water_mark
my $range_low_water_mark = $range->range_low_water_mark();
#Modify range_low_water_mark
$range->range_low_water_mark(10);
```

range_low_water_mark_reset( )

Use this method to set or retrieve the range_low_water_mark_reset value. If the percentage of allocated addresses exceeds this value, a corresponding SNMP trap is reset.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

A number that specifies the percentage of allocated addresses. The range is from 1 to 100. The low watermark reset value must be higher than the low watermark value. The default low watermark reset number is 0.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get range_low_water_mark_reset
```

```
my $range_low_water_mark_reset = $range->range_low_water_mark_reset();
#Modify range_low_water_mark_reset
$range->range_low_water_mark_reset(10);
```

restart_if_needed( )

Use this method to set the flag that indicates whether DHCP service should be restarted on a member that is serving DHCP for the changed object referenced in a cloud API request.

Parameter

Specify true to restart the DHCP service.

Returns

None

Example

```
# Modify
$network->restart_if_needed('true');
```

recycle_leases( )

Use this method to set or retrieve the recycle_leases flag of the DHCP range.

If the flag is enabled, the leases are kept in recycle bin until one week after expiration. When the flag is disabled, the leases are irrecoverably deleted.

Parameter

Specify "true" to set the recycle_leases flag or "false" to deactivate/unset it. The default value is undefined.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get recycle_leases
my $recycle_leases = $dhcp_range->recycle_leases();
#Modify recycle_leases
$dhcp_range->recycle_leases("false");
```

same_port_control_discovery_blackout( )

Use this method to set or retrieve the flag that enables using the same discovery blackout for port control blackout.

Parameter

Specify 'true' to use same discovery blackout for port control blackout or 'false' to disable it. The default value is 'false'.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
# Get attribute value
my $value = $dhcp_range->same_port_control_discovery_blackout();
# Modify attribute value
$dhcp_range->same_port_control_discovery_blackout('true');
```

server_association_type( )

Use this method to set or retrieve the DHCP range server association type.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

The server_association_type attribute can be specified explicitly. It is also set implicitly when member or failover_assoc are set.

Parameter

Valid values are "NONE", "FAILOVER", "MS_SERVER" and "MEMBER".

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get server_association_type
my $server_association_type = $dhcp_range->server_association_type();
#Modify server_association_type
$dhcp_range->server_association_type("MEMBER");
```

split_member

Use this method to set the address or FQDN of the Microsoft server that is serving the split scope. Note that this method applies only when the range is first added.

Include the specified parameter to set the attribute value.

Parameter

The valid value is a defined Infoblox::DHCP::MSServer object. The default value is empty.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

Example

```
#Set split_member prior to the range insertion
$dhcp_range->split_member($member); #$member is an Infoblox::DHCP::Member object
```

split_scope_exclusion_percent

Use this method to set the percentage for the split scope exclusion. For example, if this is set to 40, then 60% of the range will be assigned to the Microsoft server specified in split_member. Note this method has an effect only when the range is first added.

Include the specified parameter to set the attribute value.

Parameter

An unsigned integer between 1 and 99. The default is undefined.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

Example

```
#Set split_scope_exclusion_percent prior to the range insertion
$dhcp_range->split_scope_exclusion_percent(35);
```

start_addr( )

Use this method to set or retrieve the start address of the DHCP range.

A start address is the first IP address in the DHCP range available for the clients.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

Specify the start address of the DHCP range in IPv4 address format. An IPv4 address is a 32-bit number in dotted decimal notation. It consists of four 8-bit groups of decimal digits separated by decimal points (example: 192.168.1.2).

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get start_addr
my $start_addr = $dhcp_range->start_addr();
#Modify start_addr
$dhcp_range->start_addr("10.0.0.2");
```

subscribe_settings( )

Use this method to set or retrieve the Cisco ISE subscribe settings.

Setting this method to a defined value implicitly sets the override_subscribe_settings method to "true". Setting the parameter to undefined causes the appliance to use the Cisco ISE endpoint default and automatically resets the override_subscribe_settings method to "false".

Note that when subscribe_settings is set to a defined value and override_subscribe_settings is set to "false", the last operation takes precedence. Thus the sequence $object->subscribe_settings($value); $object->override_subscribe_settings("false"); will set override_subscribe_settings to "false", and the sequence $object->override_subscribe_settings("false"); $object->subscribe_settings($value); will result in override_subscribe_settings="true".

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

The valid value is an Infoblox::CiscoISE::SubscribeSetting object.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get subscribe_settings value
my $subscribe_settings = $object->subscribe_settings();
#Modify subscribe_settings value
$object->subscribe_settings($settings);
```

template( )

Use this method to set or retrieve the DHCP range template name for a DHCP Range object.

Template is the name of a DHCP range template which will be used to create the DHCP range.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

The valid name in sting format.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

The template parameter can be used to only add a new DHCP range and cannot be used during object modify or returned during object get.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get template
my $template = $dhcp_range->template();
#Modify template
$dhcp_range->template("Template_Name");
```

unknown_clients_option( )

Use this method to set or retrieve the unknown_clients_option option of a DHCP range.

Set this option to "Allow" or "Deny" to allow or prevent the allocation of an IP address from this pool to any client that has no host declaration.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

Specify "Allow" or "Deny" to set the unknown_clients_option flag value. The default value is undefined.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get unknown_clients_option
my $unknown_clients_option = $dhcp_range->unknown_clients_option();
#Modify unknown_clients_option
$dhcp_range->unknown_clients_option("Deny");
```

update_dns_on_lease_renewal( )

The update_dns_on_lease_renewal attribute controls whether the DHCP server updates DNS when a DHCP lease is renewed.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Setting this method to a defined value implicitly sets the override_update_dns_on_lease_renewal method to "true". Setting the parameter to undefined causes the appliance to use the grid default and automatically resets the override_update_dns_on_lease_renewal attribute to "false".

Note that when update_dns_on_lease_renewal is set to a defined value and override_update_dns_on_lease_renewal is set to "false", the last operation takes precedence. Thus the sequence $object->update_dns_on_lease_renewal("true"); $object->override_update_dns_on_lease_renewal("false"); will set override_update_dns_on_lease_renewal to "false", and the sequence $object->override_update_dns_on_lease_renewal("false"); $object->update_dns_on_lease_renewal("true"); will result in override_update_dns_on_lease_renewal="true".

Parameter

Specify "true" to set the update_dns_on_lease_renewal flag or "false" to deactivate/unset it.

The default value of this parameter is "false".

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get update_dns_on_lease_renewal
my $update_dns_on_lease_renewal=$dhcp_range->update_dns_on_lease_renewal( );
#Modify update_dns_on_lease_renewal
$dhcp_range->update_dns_on_lease_renewal("true");
```

SAMPLE CODE

The following sample code demonstrates the different functions that can be applied to a DHCP range object, such as add, search, modify, and remove. This sample also includes error handling for the operations.

#Preparation prior to a DHCP Range object insertion

```
#PROGRAM STARTS: Include all the modules that will be used
use strict;
use Infoblox;

#Create a session to the Infoblox appliance
my $session = Infoblox::Session->new(
               master   => "192.168.1.2", #appliance host ip
               username => "admin",       #appliance user login
               password => "infoblox"     #appliance password
);
unless ($session) {
       die("Construct session failed: ",
            Infoblox::status_code() . ":" . Infoblox::status_detail());
}
print "Session created successfully\n";

#Create a DHCP Member object
my $memberdhcp = Infoblox::DHCP::Member->new(
    name     => "infoblox.localdomain",
    ipv4addr => "192.168.1.2"
);
unless($memberdhcp) {
       die("Construct member failed: ",
            Infoblox::status_code() . ":" . Infoblox::status_detail());
}
print "DHCP Member object created successfully\n";

#Create the Network object with the member
```

```
my $network = Infoblox::DHCP::Network->new(
    network => "10.0.0.0/8",
    comment => "add network",
    members => [ $memberdhcp ]
);
unless($network) {
       die("Construct Network object failed: ",
            Infoblox::status_code() . ":" . Infoblox::status_detail());
}
print "Network object created successfully\n";

#Add the Network object into the Infoblox appliance through a session
$session->add($network)
    or die("Add Network object failed: ",
            $session->status_code() . ":" . $session->status_detail());
print "Network object added to Infoblox appliance successfully\n";

#Create the Mac Filter object
my $mac_filter = Infoblox::DHCP::Filter::MAC->new(
    "name"    => "filter1",
    "comment" => "5 minutes"
);

unless($mac_filter) {
       die("Construct Mac Filter object failed: ",
            Infoblox::status_code() . ":" . Infoblox::status_detail());
}
print "Mac Filter object created successfully\n";

#Add the Mac Filter object into the Infoblox appliance through a session
$session->add($mac_filter)
    or die("Add Mac Filter object failed: ",
            $session->status_code() . ":" . $session->status_detail());
print "Mac Filter object added to Infoblox appliance successfully\n";

#Construct Custom Option object
my $option1 = Infoblox::DHCP::Option->new(
    type  => "1",
    value => "255.0.0.0", );

#Construct DHCP Mac Filter rule object
my $filter = Infoblox::DHCP::FilterRule::MAC->new(
    filter_name => "filter1",
    permission  => "grant"
);

#Construct exclusion range object
my $exclusion_range = Infoblox::DHCP::ExclusionRange->new(
    start_address => "10.0.0.5",
    end_address   => "10.0.0.7",
    comment       => "Excluded range"
);
```

#Create a DHCP Range object

```
my $dhcp_range = Infoblox::DHCP::Range->new(
        end_addr                => "10.0.0.10",
        network                 => "10.0.0.0/8",
        start_addr              => "10.0.0.1",
        bootfile                => "bootfile1.com",
        bootserver              => "someserver.com",
        comment                 => "add range",
        ddns_generate_hostname  => "true",
        deny_all_clients        => "true",
        deny_bootp              => "true",
        disable                 => "true",
        enable_ddns             => "true",
        exclude                 => [$exclusion_range],
        filters                 => [$filter],
        member                  => $memberdhcp,
        nextserver              => "3.3.3.3",
        options                 => [$option1],
        pxe_lease_time          => "3600",
        recycle_leases          => "true",
);
unless($dhcp_range) {
```

```
        die("Construct DHCP Range object failed: ",
            Infoblox::status_code() . ":" . Infoblox::status_detail());
}
print "DHCP Range object created successfully\n";

#Add the DHCP Range object into the Infoblox appliance through a session
$session->add($dhcp_range)
    or die("Add DHCP Range object failed: ",
            $session->status_code() . ":" . $session->status_detail());
print "DHCP Range object added to the network successfully\n";
```

#Create a DHCP Range object using a template

```
my $dhcp_range_template = Infoblox::DHCP::RangeTemplate->new(
        name                    => "custom_range_template",
        offset                  => "10",
        number_of_addresses     => "10",
);
unless($dhcp_range_template) {
        die("Construct DHCP Range template object failed: ",
            Infoblox::status_code() . ":" . Infoblox::status_detail());
}
print "DHCP Range template object created successfully\n";

#Add the DHCP Range template object into the Infoblox appliance through a session
$session->add($dhcp_range_template)
    or die("Add DHCP Range template object failed: ",
            $session->status_code() . ":" . $session->status_detail());
print "DHCP Range template object added to the network successfully\n";

my $dhcp_range = Infoblox::DHCP::Range->new(
        start_addr              => "10.0.0.11",
        end_addr                => "10.0.0.15",
        network                 => "10.0.0.0/8",
        template                => "custom_range_template"
);
unless($dhcp_range) {
        die("Construct DHCP Range object failed: ",
            Infoblox::status_code() . ":" . Infoblox::status_detail());
}
print "DHCP Range object created successfully\n";

#Add the DHCP Range object into the Infoblox appliance through a session
$session->add($dhcp_range)
    or die("Add DHCP Range object failed: ",
            $session->status_code() . ":" . $session->status_detail());
print "DHCP Range object added to the network successfully\n";
```

#Get and modify a DHCP Range

```
#Get a DHCP Range through the session
my @retrieved_objs = $session->get(
    object     => "Infoblox::DHCP::Range",
    start_addr => "10.0.0.1",
);
my $object = $retrieved_objs[0];
unless ($object) {
        die("Get DHCP Range failed: ",
            $session->status_code() . ":" . $session->status_detail());
}
print "Get DHCP Range object found at least 1 matching entry\n";

#Modify one of the attributes of the specified DHCP Range
$object->comment ("modified comment");

#Un-override bootfile
$object->bootfile(undef);

#Un-override bootserver
$object->bootserver(undef);

#Un-override nextserver
$object->nextserver(undef);

#Apply the changes
$session->modify($object)
    or die("Modify DHCP Range failed: ",
```

```
            $session->status_code() . ":" . $session->status_detail());
print "DHCP Range object modified successfully \n";
```

#Search a DHCP Range

```
# search for DHCP range object with start address having last octet as 1
my @retrieved_objs = $session->search(
    object     => "Infoblox::DHCP::Range",
    network    => "10.0.0.0/.",
    start_addr => ".*1",
    comment    => ".*comment"
);

my $object = $retrieved_objs[0];
unless ($object) {
    die("Search DHCP range failed: ",
        $session->status_code() . ":" . $session->status_detail());
}
print "Search DHCP range object found at least 1 matching entry\n";
```

#Remove a DHCP Range

```
#Get a DHCP Range through the session
my @retrieved_objs = $session->get(
    object     => "Infoblox::DHCP::Range",
    start_addr => "10.0.0.1",
    end_addr   => "10.0.0.10",
);

my $object = $retrieved_objs[0];
unless ($object) {
       die("Get DHCP Range failed: ",
            $session->status_code() . ":" . $session->status_detail());
}
print "Get DHCP Range object found at least 1 matching entry\n";

#submit the object for removal
$session->remove($object)
    or die("Remove DHCP Range failed: ",
            $session->status_code() . ":" . $session->status_detail());
print "DHCP Range object removed successfully \n";

#submit the Range Template object for removal
$session->remove($dhcp_range_template)
    or die("Remove DHCP Range Tempate failed: ",
            $session->status_code() . ":" . $session->status_detail());
print "DHCP Range Templateobject removed successfully \n";

####PROGRAM ENDS####
```

AUTHOR

Infoblox Inc. http://www.infoblox.com/

SEE ALSO

Infoblox::Session, Infoblox::Session->add(), Infoblox::Session->get(), Infoblox::Session->modify(), Infoblox::Session->remove(), Infoblox::Session->search(),Infoblox::DHCP::ViewInfoblox::DHCP::ExclusionRange, Infoblox::DHCP::FilterRule::MAC, Infoblox::DHCP::FilterRule::NAC, Infoblox::DHCP::FilterRule::RelayAgent, Infoblox::DHCP::FilterRule::Option, Infoblox::DHCP::Member, Infoblox::DHCP::Option, the Infoblox::DHCP::RangeTemplate manpage, Infoblox::Grid::MSServer::AdUser::Data

COPYRIGHT

Copyright (c) 2017 Infoblox Inc.

Infoblox::DHCP::RangeTemplate - DHCP RangeTemplate object.

- NAME
- DESCRIPTION
- CONSTRUCTOR
- MODULE METHODS
  - Infoblox::DHCP::NetworkTemplate->range_templates( )
- SESSION METHODS
  - Infoblox::Session->add( )
  - Infoblox::Session->get( )
  - Infoblox::Session->search( )
  - Infoblox::Session->modify( )
  - Infoblox::Session->remove( )
- METHODS
  - bootfile( )
  - bootserver( )
  - comment( )
  - ddns_domainname( )
  - ddns_generate_hostname( )
  - deny_all_clients( )
  - deny_bootp( )
  - enable_ddns( )
  - enable_known_clients_option( )
  - enable_unknown_clients_option( )
  - exclude( )
  - extattrs( )
  - extensible_attributes( )
  - failover_assoc( )
  - filters( )
  - known_clients_option( )
  - lease_scavenge_time( )
  - logic_filters( )
  - member( )
  - name( )
  - nextserver( )
  - number_of_addresses( )
  - offset( )
  - options( )
  - override_ddns_domainname( )
  - override_lease_scavenge_time( )
  - override_logic_filters( )
  - override_update_dns_on_lease_renewal( )
  - pxe_lease_time( )
  - range_high_water_mark( )
  - range_high_water_mark_reset( )
  - range_low_water_mark( )
  - range_low_water_mark_reset( )
  - server_association_type( )
  - recycle_leases( )
  - unknown_clients_option( )
  - update_dns_on_lease_renewal( )
- SAMPLE CODE
- AUTHOR
- SEE ALSO
- COPYRIGHT

NAME

Infoblox::DHCP::RangeTemplate - DHCP RangeTemplate object.

DESCRIPTION

A DHCP range template works as a 'cookie cutter' and can be used to create real DHCP range object in a quick and consistent way. The DHCP range object created from a DHCP range template will inherit most properties defined in range template object. So, most of the DHCP range template properties are the same as the DHCP range object properties.

CONSTRUCTOR

```
my $dhcp_range_template = Infoblox::DHCP::RangeTemplate->new(
```

```
     name                                => $string,
# Required
     number_of_addresses                 => $num,
# Required
     offset                              => $num,
# Required
     bootfile                            => $filename,
# Optional / Default is undefined
     bootserver                          => $ipv4addr | $fqdn | undef,
# Optional / Default is undefined
     comment                             => $string,
# Optional / Default is undefined
     ddns_domainname                     => $string | undef,
# Optional / Default is undefined
     ddns_generate_hostname              => "true" | "false",
# Optional / Default is undefined
     deny_all_clients                    => "true" | "false",
# Optional / Default is "false"
     deny_bootp                          => "true" | "false" | undef,
# Optional / Default is undefined
     enable_ddns                         => "true" | "false",
# Optional / Default is undefined
     enable_known_clients_option         => "true" | "false",
# Optional / Default is "false"
     enable_unknown_clients_option       => "true" | "false",
# Optional / Default is "false"
     exclude                             => [$ExclusionRange1,$ExclusionRange2,...],
# Optional / Default is empty
     extattrs                            => { $string => $extattr, ... },
# Optional / Default is undefined
     extensible_attributes               => { $string => $string | $num, $string => [ $string | $num, ...
], ... } # Optional / Default is undefined
     failover_assoc                      => $string,
# Optional / Default is undefined
     filters                             => [$MAC,$RelayAgent,$Option,...],
# Optional / Default is undefined
     known_clients_option                => "Allow" | "Deny",
# Optional / Default is undefined
     lease_scavenge_time                 => $number,
# Optional / Default is -1
     member                              => $Member,
# Optional / Default is undefined
     nextserver                          => $ipv4addr | $fqdn | undef,
# Optional / Default is undefined
     options                             => [$Option1,$Option2,...],
# Optional / Default is empty
     logic_filters                       => [ $filter_name | $filter_obj, ...],
# Optional / Default is undefined
     override_logic_filters              => "true' | 'false",
# Optional / Default value is "false"
     override_ddns_domainname            => "true" | "false",
# Optional / Default value is "false"
     override_lease_scavenge_time        => "true" | "false",
# Optional / Default value is "false"
     override_update_dns_on_lease_renewal => "true" | "false",
# Optional / Default value is "false"
     pxe_lease_time                      => $num,
# Optional / Default is undefined
     range_high_water_mark               => $num,
# Optional / Default is 95
     range_high_water_mark_reset         => $num,
# Optional / Default is 95
     range_low_water_mark                => $num,
# Optional / Default is 0
     range_low_water_mark_reset          => $num,
# Optional / Default is 0
     recycle_leases                      => "true" | "false",
# Optional / Default is undefined
     server_association_type             => "NONE" | "MEMBER" | "FAILOVER" | "MS_SERVER"
# Optional / Default is undefined
     unknown_clients_option              => "Allow" | "Deny",
# Optional / Default is undefined
     update_dns_on_lease_renewal         => "true" | "false",
# Optional / Default value is "false"
      );
```

You cannot set both extattrs and extensible_attributes attributes at the same time.

MODULE METHODS

The following functions are available to be applied to a DHCP Range Template object.

Infoblox::DHCP::NetworkTemplate->range_templates( )

Use this function to specify a range template at network template on the Infoblox appliance. See Infoblox::DHCP::NetworkTemplate->range_templates() for parameters and return values.

Example

```
#Construct a range template object
my $dhcp_range_template = Infoblox::DHCP::RangeTemplate->new(
            name                    => "custom_range_template",
            number_of_addresses     => "10",
            offset                  => "10",
    );

my $template = Infoblox::DHCP::Template->new(
            name                    => "custom_range_template",
    );

#Configure range template in network template object
my $responce = $network_template->range_templates([$template]);
```

SESSION METHODS

This section describes all the methods in an Infoblox::Session module that can be applied to a DHCP range template object.

Infoblox::Session->add( )

Use this method to add a DHCP range template object to the Infoblox appliance. See Infoblox::Session->add() for parameters and return values.

Example

```
#Construct an object
my $dhcp_range_template = Infoblox::DHCP::RangeTemplate->new(
        name                    => "custom_range_template",
        number_of_addresses     => "10",
        offset                  => "10",
        bootfile                => "bootfile1.com",
        bootserver              => "someserver.com",
        comment                 => "add range",
        ddns_generate_hostname  => "true",
        deny_all_clients        => "true",
        deny_bootp              => "true",
        enable_ddns             => "true",
        exclude                 => [$exclusion_range_template],
        filters                 => [$filter],
        member                  => $memberdhcp,
        nextserver              => "3.3.3.3",
        options                 => [$option1],
        pxe_lease_time          => "3600",
        recycle_leases          => "true",
);

# Submit for addition
my $response = $session->add( $dhcp_range_template );
```

Infoblox::Session->get( )

Use this method to retrieve all the matching objects from the Infoblox appliance. See Infoblox::Session->get() for parameters and return values.

Key References

```
Apply the following attributes to get a specific DHCP range template object:

name - Optional. The name of the DHCP range template.
extattrs     - Optional. A hash reference containing extensible attributes.
extensible_attributes - Optional. A hash reference containing extensible attributes.
server_association_type - Optional. The server association type for the range.
```

Examples

```
my @retrieved_objs = $session->get(
                        object => "Infoblox::DHCP::RangeTemplate",
                        name   => "custom_range_template"
                        );
```

```
my @retrieved_objs = $session->get(
                      object               => "Infoblox::DHCP::RangeTemplate",
                      extensible_attributes => { 'Site' => 'Santa Clara' }
                     );
```

Infoblox::Session->search( )

Use this method to retrieve all the matching objects from the Infoblox appliance. See Infoblox::Session->search() for parameters and return values.

Key References

```
Apply the following attributes to search for a specific DHCP Range Template object:

name    - Optional. The name of the DHCP range template (regular expression).
comment - Optional. The comment of the DHCP range template (regular expression).
extattrs     - Optional. A hash reference containing extensible attributes.
extensible_attributes - Optional. A hash reference containing extensible attributes.
server_association_type - Optional. The server association type for the range.
```

For more information about searching extensible attributes, see Infoblox::Grid::ExtensibleAttributeDef/Searching Extensible Attributes.

Examples

```
my @retrieved_objs = $session->search(
                      object  => "Infoblox::DHCP::RangeTemplate",
                      name    => "custom_range_template"
           comment => ".*",
       );

my @retrieved_objs = $session->search(
                      object               => "Infoblox::DHCP::RangeTemplate",
                      extensible_attributes => { 'Site' => 'Santa Clara' }
       );
```

Infoblox::Session->modify( )

Use this method to modify a DHCP range template object in the Infoblox appliance. See Infoblox::Session->modify() for parameters and return values.

Example

```
# Use method to modify the comment.
$dhcp_range_template->comment("this is a modified comment");
# Submit modification
my $response = $session->modify( $dhcp_range_template );
```

Infoblox::Session->remove( )

Use this method to remove a DHCP range template object from the Infoblox appliance. See Infoblox::Session->remove() for parameters and return values.

To remove a specific object, first use `get()` or `search()` to retrieve the specific DHCP range template object, and then submit this object for removal.

Example

```
# Get the objects with the same name
my @retrieved_objs = $session->get(
    object => "Infoblox::DHCP::RangeTemplate",
    name   => "custom_range_template"
    );

# Find the desired object from the retrieved list.
my $desired_dhcp_range_template = $retrieved_objs[0];
# Submit for removal
my $response = $session->remove( $desired_dhcp_range_template );
```

METHODS

This section describes all the methods that can be used to configure and retrieve the attribute values of a DHCP range template object.

bootfile( )

Use this method to set or retrieve a bootfile name which is the name of the file the client must download.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

This option is overridden independently from Infoblox::DHCP::RangeTemplate->bootserver() and Infoblox::DHCP::RangeTemplate->nextserver().

Parameter

The boot file name in string format with a maximum of 256 bytes. The default value is undefined.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get bootfile
my $bootfile = $dhcp_range_template->bootfile();
#Modify bootfile
$dhcp_range_template->bootfile("bootfile2.com");
#Un-override bootfile
$dhcp_range_template->bootfile(undef);
```

bootserver( )

Use this method to set or retrieve a bootserver name which is the name of the server where the boot file is stored.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

This option is overridden independently from Infoblox::DHCP::RangeTemplate->bootfile() and Infoblox::DHCP::RangeTemplate->nextserver().

Parameter

The boot server IP address or name in FQDN (Fully Qualified Domain Name) format. The FQDN consists of the host name followed by the domain name (example: abc.com). A boot server name can have a maximum of 256 bytes. The default value is undefined.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get bootserver
my $bootserver = $dhcp_range_template->bootserver();
#Modify bootserver
$dhcp_range_template->bootserver("anotherserver.com");
#Un-override bootserver
$dhcp_range_template->bootserver(undef);
```

comment( )

Use this method to set or retrieve a descriptive comment of a DHCP range template.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

Desired comment in string format with a maximum of 256 bytes.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get comment
my $comment = $dhcp_range_template->comment();
#Modify comment
$dhcp_range_template->comment("Modifying the DHCP range template comment");
```

ddns_domainname( )

Use this method to set or retrieve the ddns_domainname value of a DHCP Range template object.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Setting this method to a defined value implicitly sets the override_ddns_domainname method to "true". Setting the parameter to undefined causes the appliance to use the Grid default and automatically resets the override_ddns_domainname attribute to "false".

Note that when ddns_domainname is set to a defined value and override_dddns_domainname is set to "false", the last operation takes precedence. Thus the sequence $object->ddns_domainname("testdomain"); $object->override_ddns_domainname("false"); will set override_ddns_domainname to "false", and the sequence $object->override_ddns_domainname("false"); $object->ddns_domainname("testdomain"); will result in override_ddns_domainname="true".

Parameter

The dynamic DNS domain name in string format.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get ddns_domainname
my $ddns_domainname = $dhcp_range_template->ddns_domainname();
#Modify ddns_domainname
$dhcp_range_template->ddns_domainname("test_domain.com");
```

ddns_generate_hostname( )

Use this method to set or retrieve the ddns_generate_hostname flag of a DHCP range template.

Enable this method to allow only the DHCP server to generate a hostname.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

Specify "true" to set the ddns_generate_hostname flag or "false" to deactivate/unset it. The default value is undefined.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get ddns_generate_hostname
my $ddns_generate_hostname = $dhcp_range_template->ddns_generate_hostname();
#Modify ddns_generate_hostname
$dhcp_range_template->ddns_generate_hostname("false");
```

deny_all_clients( )

Use this method to set or retrieve the deny_all_clients flag of a DHCP range template.

Enable this method to deny all lease requests for the DHCP range created from template.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

Specify "true" to set the deny_all_clients flag or "false" to deactivate/unset it. The default value for this field is false.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get deny_all_clients
my $deny_all_clients = $dhcp_range_template->deny_all_clients();
#Modify deny_all_clients
$dhcp_range_template->deny_all_clients("false");
```

deny_bootp( )

Use this method to set or retrieve the deny_bootp flag of a DHCP range template.

Enable this method to deny all BOOTP requests for DHCP range created from this template.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

Specify "true" to set the deny_bootp flag or "false" to deactivate/unset it. The default value is false which indicates that this attribute inherit the upper-level setting.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get deny_bootp
my $deny_bootp = $dhcp_range_template->deny_bootp();
#Modify deny_bootp
$dhcp_range_template->deny_bootp("false");
```

enable_ddns( )

Use this method to set or retrieve the enable_ddns flag of a DHCP range template.

Enable this method to allow dynamic DNS updates.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

Specify "true" to set the enable_ddns flag or "false" to deactivate/unset it. The default value is undefined.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get enable_ddns
my $enable_ddns = $dhcp_range_template->enable_ddns();
#Modify enable_ddns
$dhcp_range_template->enable_ddns("false");
```

enable_known_clients_option( )

Use this method to set or retrieve the enable_known_clients_option flag of a DHCP range.

You must set the enable_known_clients_option flag to true in order to use the known_clients_option attribute.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

Specify "true" to set the enable_known_clients_option flag or "false" to deactivate/unset it. The default value is "false".

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get enable_known_clients_option
my $enable_known_clients_option = $dhcp_range->enable_known_clients_option();
#Modify enable_known_clients_option
$dhcp_range->enable_known_clients_option("false");
```

enable_unknown_clients_option( )

Use this method to set or retrieve the enable_unknown_clients_option flag of a DHCP range.

You must set the enable_unknown_clients_option flag to true in order to use the unknown_clients_option attribute.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

Specify "true" to set the enable_unknown_clients_option flag or "false" to deactivate/unset it. The default value is "false".

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get enable_unknown_clients_option
my $enable_unknown_clients_option = $dhcp_range->enable_unknown_clients_option();
#Modify enable_unknown_clients_option
$dhcp_range->enable_unknown_clients_option("false");
```

exclude( )

Use this method to set or retrieve the exclusion range template in the DHCP range template.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

Valid value is an array reference that contains defined Infoblox::DHCP::ExclusionRangeTemplate objects.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
    #Get exclude
    my $ref_exclude = $dhcp_range_template->exclude();
    #Modify exclude
    $dhcp_range_template->exclude([$exl1]); #$exl1 is an Infoblox::DHCP::ExclusionRangeTemplate
object
```

extattrs( )

Use this method to set or retrieve the extensible attributes associated with a DHCP Range Template object.

Parameter

Valid value is a hash reference containing the names of extensible attributes and their associated values (Infoblox::Grid::Extattr objects).

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get extattrs
my $ref_extattrs = $dhcp_range_template->extattrs();
#Modify extattrs
$dhcp_range_template->extattrs({ 'Site' => $extattr1, 'Administrator' => $extattr2 });
```

extensible_attributes( )

Use this method to set or retrieve the extensible attributes associated with a DHCP Range Template object.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

For valid values for extensible attributes, see Infoblox::Grid::ExtensibleAttributeDef/Extensible Attribute Values.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get extensible attributes
my $ref_extensible_attributes = $dhcp_range_template->extensible_attributes();
#Modify extensible attributes
$dhcp_range_template->extensible_attributes({ 'Site' => 'Santa Clara', 'Administrator' => [ 'Peter',
'Tom' ] });
```

failover_assoc( )

Use this method to set or retrieve the failover peer association name of the DHCP range template.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

If the attribute is set, the server_association_type attribute will also be set to "FAILOVER" if the name is defined, or to "NONE" if the member is also undefined.

Parameter

Specify the failover peer association name in string format. The default value is undefined.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get failover_assoc
my $failover_assoc = $dhcp_range_template->failover_assoc();
#Modify failover_assoc
$dhcp_range_template->failover_assoc("peer2");
```

filters( )

Use this method to set or retrieve the filters of the DHCP range template.

Filters are used to screen the address assignments if the addresses in the DHCP range needs to be assigned to specific hosts.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

The valid value is an array reference that contains defined Infoblox::DHCP::FilterRule::MAC, Infoblox::DHCP::FilterRule::NAC, Infoblox::DHCP::FilterRule::RelayAgent, Infoblox::DHCP::FilterRule::Fingerprint and Infoblox::DHCP::FilterRule::UserClass objects. The default value is undefined.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get filters
my $ref_filters = $dhcp_range_template->filters();
#Modify filters
$dhcp_range_template->filters([$filter1]); #$filter1 is an Infoblox::DHCP::FilterRule::MAC,
Infoblox::DHCP::FilterRule::MAC,  Infoblox::DHCP::FilterRule::RelayAgent,
Infoblox::DHCP::FilterRule::Fingerprint or Infoblox::DHCP::FilterRule::UserClass object.
```

known_clients_option( )

Use this method to set or retrieve the known_clients_option option of a DHCP range.

Set this option to "Allow" or "Deny" to allow or prevent the allocation of an IP address from this pool to any client that has a host declaration.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

Specify "Allow" or "Deny" to set the known_clients_option flag value. The default value is undefined.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get known_clients_option
my $known_clients_option = $dhcp_range->known_clients_option();
#Modify known_clients_option
$dhcp_range->known_clients_option("Deny");
```

lease_scavenge_time( )

Use this method to set or retrieve the lease_scavenge_time value of a DHCP Range Template object.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Setting this method to a defined value implicitly sets the override_lease_scavenge_time method to "true". Setting the parameter to undefined causes the appliance to use the default from the upper level and automatically resets the override_lease_scavenge_time attribute to "false".

Note that when lease_scavenge_time is set to a defined value and override_lease_scavenge_time is set to "false", the last operation takes precedence. Thus the sequence $object->lease_scavenge_time(604800); $object->override_lease_scavenge_time("false"); will set override_lease_scavenge_time to "false", and the sequence $object->override_lease_scavenge_time("false"); $object->lease_scavenge_time(604800); will result in override_lease_scavenge_time="true".

Parameter

An integer that specifies the period of time, in seconds, that free and backup leases remain in the database before they are automatically deleted. To disable lease scavenging, set the parameter to -1. The minimum positive value must be greater than 86400 seconds (1 day). The default value of this parameter is -1, which means that lease scavenging is disabled.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get lease_scavenge_time
my $lease_scavenge = $dhcp_range_template->lease_scavenge_time();
#Modify lease_scavenge_time
$dhcp_range_template->lease_scavenge_time(30 * 24 * 60 * 60);
```

logic_filters( )

Use this method to set or retrieve all the logic (ordered) filters associated with the DHCP Range Template object.

Filters are used to screen address assignments in a DHCP Range Template if the addresses are to be assigned to specific hosts.

Setting this method to a defined value implicitly sets the override_logic_filters method to "true". Setting the parameter to undefined causes the appliance to use the Grid default and automatically resets the override_logic_filters attribute to "false".

Note that when logic_filters is set to a defined value and override_dlogic_filters is set to "false", the last operation takes precedence. Thus the sequence $object->logic_filters($filters); $object->override_logic_filters("false"); will set override_logic_filters to "false", and the sequence $object->override_logic_filters("false"); $object->logic_filters($filters); will result in override_logic_filters="true".

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

The valid value is an array reference that contains either the Infoblox::DHCP::Filter::MAC, Infoblox::DHCP::Filter::NAC, Infoblox::DHCP::Filter::Option objects or the names of the corresponding filters. The filters in the list must satisfy the following conditions:

Filter name must be globally unique.
Filters without expressions must be at the end of the list.
The filters should not be disabled.

The default value of the parameter is undefined.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get logic_filters
my $ref_logic_filters = $dhcp_range_template->logic_filters();
#Modify logic_filters
$dhcp_range_template->logic_filters(["filter_1","filter_2",$exl1]); #$exl1 is an
Infoblox::DHCP::Filter::MAC, Infoblox::DHCP::Filter::NAC or Infoblox::DHCP::Filter::Option object.
```

member( )

Use this method to set or retrieve the member or Microsoft (r) server of the DHCP range template.

The member or Microsoft server that serves DHCP for this IP address range template.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

If the attribute is set, the server_association_type attribute will also be set to "MEMBER" if you set the member to an Infoblox::DHCP::Member object, to "MS_SERVER" if you set the member to an Infoblox::DHCP::MSServer object, or to "NONE" if failover_assoc is also empty.

Parameter

The valid value is a defined Infoblox::DHCP::Member or Infoblox::DHCP::MSServer object. The default value is undefined.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get member
my $ref_member = $dhcp_range_template->member();
#Modify member
$dhcp_range_template->member($member1); #$member is an Infoblox::DHCP::Member object
```

name( )

Use this method to set or retrieve the description of the name of the DHCP range template.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

Name of the fixed address in string format with a maximum of 64 bytes. The default value is undefined.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get name
my $name = $dhcp_range_template->name();
#Modify name
$dhcp_range_template->name("custom_range_template");
```

nextserver( )

Use this method to set or retrieve the nextserver IP address which is the boot file server where the boot file is stored.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

This option is overridden independently from Infoblox::DHCP::RangeTemplate->bootfile() and Infoblox::DHCP::RangeTemplate->bootserver().

Parameter

The next server IP address or name in FQDN (Fully Qualified Domain Name) format. The FQDN consists of the host name followed by the domain name (example: abc.com). A next server name can have a maximum of 256 bytes. The default value is undefined.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get nextserver
my $nextserver = $dhcp_range_template->nextserver();
#Modify nextserver
$dhcp_range_template->nextserver("4.4.4.4");
#Un-override nextserver
$dhcp_range_template->nextserver(undef);
```

number_of_addresses( )

Use this method to set or retrieve the number of addresses in this DHCP range template

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

The number of addresses in this DHCP range template.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get number_of_addresses
my $number_of_addresses = $dhcp_range_template-> number_of_addresses();
#Modify number_of_addresses
$dhcp_range_template->number_of_addresses("10");
```

offset( )

Use this method to set or retrieve the address offset of this DHCP range template.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

The address offset of this DHCP range template. The offset is calculated from the start address of network template to which the DHCP range template is assigned to. For example, if offset is set to 10 and number_of_addresses is set to 100, and network is set to 10.0.0.0/8, then start address of DHCP range created from this range template will be 10.0.0.10 and end address will be 10.0.0.100.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get offset
my $offset = $dhcp_range_template->offset();
#Modify offset
$dhcp_range_template->offset("10");
```

options( )

Use this method to set or retrieve the options of the DHCP range template.

DHCP options describe network configuration settings and various services available on the network.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

Valid value is an array reference that contains defined Infoblox::DHCP::Option objects. The default value is empty.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get options
my $ref_options = $dhcp_range_template->options();
#Modify options
$dhcp_range_template->options([$option12]); #$option12 is an Infoblox::DHCP::Option object
```

override_ddns_domainname( )

The override_ddns_domainname attribute controls whether the ddns_domainname value in the DHCP range template is used, instead of the Grid default.

The override_ddns_domainname attribute can be specified explicitly. It is also set implicitly when ddns_domainname is set to a defined value.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

Set the parameter to "true" to override the grid-level setting for ddns_domainname. Set the parameter to "false" to inherit the grid-level setting for ddns_domainname.

The default value of this parameter is "false".

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Getting override_ddns_domainname
my $override_ddns_domainname=$dhcp_range_template->override_ddns_domainname( );
#Modifying override_ddns_domainname
$dhcp_range_template->override_ddns_domainname("true");
```

override_lease_scavenge_time( )

The override_lease_scavenge_time attribute controls whether the lease_scavenge_time value in the DHCP range template is used, instead of the upper-level default.

The override_lease_scavenge_time attribute can be specified explicitly. It is also set implicitly when lease_scavenge_time is set to a defined value.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

Set the parameter to "true" to override the upper-level setting for lease_scavenge_time. Set the parameter to "false" to inherit the setting for lease_scavenge_time from the upper level.

The default value of this parameter is "false".

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Getting override_lease_scavenge_time
my $override_lease_scavenge_time=$dhcp_range_template->override_lease_scavenge_time( );
#Modifying override_lease_scavenge_time
$dhcp_range_template->override_lease_scavenge_time("true");
```

override_logic_filters( )

The override_logic_filters attribute controls whether the logic_filters value in the DHCP Range Template is used, instead of the Grid default.

The override_logic_filters attribute can be specified explicitly. It is also set implicitly when logic_filters is set to a defined value.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

Set the parameter to "true" to override the Grid-level setting for logic_filters. Set the parameter to "false" to inherit the Grid-level setting for logic_filters.

The default value of this parameter is "false".

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Getting override_logic_filters
my $override_logic_filters=$dhcp_range_template->override_logic_filters( );
#Modifying override_logic_filters
$dhcp_range_template->override_logic_filters("true");
```

override_update_dns_on_lease_renewal( )

The override_update_dns_on_lease_renewal attribute controls whether the update_dns_on_lease_renewal value in the DHCP range template is used, instead of the Grid default.

The override_update_dns_on_lease_renewal attribute can be specified explicitly. It is also set implicitly when update_dns_on_lease_renewal is set to a defined value.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

Set the parameter to "true" to override the grid-level setting for update_dns_on_lease_renewal. Set the parameter to "false" to inherit the grid-level setting for update_dns_on_lease_renewal.

The default value of this parameter is "false".

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
    #Getting override_update_dns_on_lease_renewal
    my $override_update_dns_on_lease_renewal=$dhcp_range_template-
>override_update_dns_on_lease_renewal( );
    #Modifying override_update_dns_on_lease_renewal
    $dhcp_range_template->override_update_dns_on_lease_renewal("true");
```

pxe_lease_time( )

Use this method to set or retrieve the PXE lease time value of the DHCP range template.

The Preboot Execution Environment lease time value is used by some hosts to boot remotely from a server.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

A 32-bit integer (range from 0 to 4294967295) represents the duration in seconds that the record is cached. Zero indicates that the record should not be cached. The default value is undefined.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get pxe_lease_time
my $pxe_lease_time = $dhcp_range_template->pxe_lease_time();
#Modify pxe_lease_time
$dhcp_range_template->pxe_lease_time("7200");
```

range_high_water_mark( )

Use this method to set or retrieve the range_high_water_mark value. If the percentage of allocated addresses exceeds this value, an

SNMP trap is sent, unless this is a range served by a Microsoft server.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

A number that specifies the percentage of allocated addresses. The range is from 1 to 100. The high watermark value must be greater than the low watermark value. The default value is 95.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get range_high_water_mark
my $dhcp_range_template_high_water_mark = $dhcp_range_template->range_high_water_mark();
#Modify range_high_water_mark
$dhcp_range_template->range_high_water_mark(80);
```

range_high_water_mark_reset( )

Use this method to set or retrieve the range_high_water_mark_reset value. If the percentage of allocated addresses drops below this value, a corresponding SNMP trap is reset.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

A number that specifies the percentage of allocated addresses. The range is from 1 to 100. The high watermark reset value must be lower than the high watermark value. The default value is 95.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get range_high_water_mark_reset
my $dhcp_range_template_high_water_mark_reset = $dhcp_range_template->range_high_water_mark_reset();
#Modify range_high_water_mark_reset
$dhcp_range_template->range_high_water_mark_reset(70);
```

range_low_water_mark( )

Use this method to set or retrieve the range_low_water_mark value. If the percentage of allocated addresses drops below this watermark, an SNMP trap is sent, unless this is a range served by a Microsoft server.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

A number that specifies the percentage of allocated addresses. The range is from 1 to 100. The low watermark value must be lower than the high watermark value. The default low watermark value is 0.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get range_low_water_mark
my $dhcp_range_template_low_water_mark = $dhcp_range_template->range_low_water_mark();
#Modify range_low_water_mark
$dhcp_range_template->range_low_water_mark(10);
```

range_low_water_mark_reset( )

Use this method to set or retrieve the range_low_water_mark_reset value. If the percentage of allocated addresses exceeds this value, a corresponding SNMP trap is reset.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

A number that specifies the percentage of allocated addresses. The range is from 1 to 100. The low watermark reset value must be higher than the low watermark value. The default low watermark reset number is 0.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get range_low_water_mark_reset
my $dhcp_range_template_low_water_mark_reset = $dhcp_range_template->range_low_water_mark_reset();
#Modify range_low_water_mark_reset
$dhcp_range_template->range_low_water_mark_reset(10);
```

server_association_type( )

Use this method to set or retrieve the DHCP range template server association type.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

The server_association_type attribute can be specified explicitly. It is also set implicitly when member or failover_assoc are set.

Parameter

Valid values are "NONE", "FAILOVER", "MS_SERVER" and "MEMBER".

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get server_association_type
my $server_association_type = $dhcp_range_template->server_association_type();
#Modify server_association_type
$dhcp_range_template->server_association_type("MEMBER");
```

recycle_leases( )

Use this method to set or retrieve the recycle_leases flag of the DHCP range template.

If the flag is enabled, the leases are kept in recycle bin until one week after expiration. When the flag is disabled, the leases are irrecoverably deleted.

Parameter

Specify "true" to set the recycle_leases flag or "false" to deactivate/unset it. The default value is undefined.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get recycle_leases
my $recycle_leases = $dhcp_range_template->recycle_leases();
#Modify recycle_leases
$dhcp_range_template->recycle_leases("false");
```

unknown_clients_option( )

Use this method to set or retrieve the unknown_clients_option option of a DHCP range.

Set this option to "Allow" or "Deny" to allow or prevent the allocation of an IP address from this pool to any client that has no host declaration.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

Specify "Allow" or "Deny" to set the unknown_clients_option flag value. The default value is undefined.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get unknown_clients_option
my $unknown_clients_option = $dhcp_range->unknown_clients_option();
#Modify unknown_clients_option
$dhcp_range->unknown_clients_option("Deny");
```

update_dns_on_lease_renewal( )

The update_dns_on_lease_renewal attribute controls whether the DHCP server updates DNS when a DHCP lease is renewed.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Setting this method to a defined value implicitly sets the override_update_dns_on_lease_renewal method to "true". Setting the parameter to undefined causes the appliance to use the Grid default and automatically resets the override_update_dns_on_lease_renewal attribute to "false".

Note that when update_dns_on_lease_renewal is set to a defined value and override_update_dns_on_lease_renewal is set to "false", the last operation takes precedence. Thus the sequence $object->update_dns_on_lease_renewal("true"); $object->override_update_dns_on_lease_renewal("false"); will set override_update_dns_on_lease_renewal to "false", and the sequence $object->override_update_dns_on_lease_renewal("false"); $object->update_dns_on_lease_renewal("true"); will result in override_update_dns_on_lease_renewal="true".

Parameter

Specify "true" to set the update_dns_on_lease_renewal flag or "false" to deactivate/unset it.

The default value of this parameter is "false".

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get update_dns_on_lease_renewal
my $update_dns_on_lease_renewal=$dhcp_range_template->update_dns_on_lease_renewal( );
#Modify update_dns_on_lease_renewal
$dhcp_range_template->update_dns_on_lease_renewal("true");
```

SAMPLE CODE

The following sample code demonstrates the different functions that can be applied to a DHCP range template object such as add, modify, and remove. Also, this sample includes error handling for the operations.

#Preparation prior to a DHCP RangeTemplate object insertion

```
#PROGRAM STARTS: Include all the modules that will be used
use strict;
use Infoblox;

#Create a session to the Infoblox appliance
my $host_ip = "192.168.1.2";
my $session = Infoblox::Session->new(
               master   => $host_ip,      #appliance host ip
               username => "admin",       #appliance user login
               password => "infoblox"     #appliance password
);
unless ($session) {
       die("Construct session failed: ",
            Infoblox::status_code() . ":" . Infoblox::status_detail());
}
print "Session created successfully\n";

#Create a DHCP Member object
my $memberdhcp = Infoblox::DHCP::Member->new(
    name     => "infoblox.localdomain",
    ipv4addr => $host_ip
);
```

```
unless($memberdhcp) {
       die("Construct member failed: ",
            Infoblox::status_code() . ":" . Infoblox::status_detail());
}
print "DHCP Member object created successfully\n";

#Create the Mac Filter object
my $mac_filter = Infoblox::DHCP::Filter::MAC->new(
    name    => "filter1",
    comment => "5 minutes"
);

unless($mac_filter) {
       die("Construct Mac Filter object failed: ",
            Infoblox::status_code() . ":" . Infoblox::status_detail());
}
print "Mac Filter object created successfully\n";

#Add the Mac Filter object into the Infoblox appliance through a session
$session->add($mac_filter)
    or die("Add Mac Filter object failed: ",
            $session->status_code() . ":" . $session->status_detail());

print "Mac Filter object added to Infoblox appliance successfully\n";

#Construct DHCP Mac Filter rule object
my $filter = Infoblox::DHCP::FilterRule::MAC->new(
    filter_name => "filter1",
    permission  => "grant"
);
unless($filter){
   die("Construct filter object failed: ",
        Infoblox::status_code() . ":" . Infoblox::status_detail());
   }
 print "Filter object created successfully\n";

#Construct exclusion range template object
my $exclusion_range_template = Infoblox::DHCP::ExclusionRangeTemplate->new(
    offset              => "15",
    number_of_addresses => "5",
    comment             => "Excluded range"
);
unless($exclusion_range_template){
       die("Construct exclusion range template object failed: ",
                Infoblox::status_code() . ":" . Infoblox::status_detail());
       }
 print "Exclusion range template object created successfully\n";
```

#Create a DHCP RangeTemplate object

```
my $dhcp_range_template = Infoblox::DHCP::RangeTemplate->new(
        name                    => "custom_range_template",
        offset                  => "10",
        number_of_addresses     => "10",
        bootfile                => "bootfile1.com",
        bootserver              => "someserver.com",
        comment                 => "add range",
        ddns_generate_hostname  => "true",
        deny_all_clients        => "true",
        deny_bootp              => "true",
        enable_ddns             => "true",
        exclude                 => [$exclusion_range_template],
        filters                 => [$filter],
        member                  => $memberdhcp,
        nextserver              => "3.3.3.3",
        pxe_lease_time          => "3600",
        recycle_leases          => "true",
);
unless($dhcp_range_template) {
       die("Construct DHCP RangeTemplate object failed: ",
            Infoblox::status_code() . ":" . Infoblox::status_detail());
}
print "DHCP RangeTemplate object created successfully\n";

#Add the DHCP RangeTemplate object into the Infoblox appliance through a session
$session->add($dhcp_range_template)
```

```
    or die("Add DHCP RangeTemplate object failed: ",
            $session->status_code() . ":" . $session->status_detail());

print "DHCP RangeTemplate object added to the network successfully\n";
```

#Search a DHCP RangeTemplate

```
#Search a DHCP RangeTemplate through the session
my @retrieved_objs = $session->search(
    object     => "Infoblox::DHCP::RangeTemplate",
    name       => "custom_range_template",
);
my $object = $retrieved_objs[0];
unless ($object) {
       die("Search DHCP RangeTemplate failed: ",
            $session->status_code() . ":" . $session->status_detail());
}
print "Search DHCP RangeTemplate object found at least 1 matching entry\n";
```

#Get and modify a DHCP RangeTemplate

```
#Get a DHCP RangeTemplate through the session
my @retrieved_objs = $session->get(
    object     => "Infoblox::DHCP::RangeTemplate",
    name       => "custom_range_template",
);
my $object = $retrieved_objs[0];
unless ($object) {
       die("Get DHCP RangeTemplate failed: ",
            $session->status_code() . ":" . $session->status_detail());
}
print "Get DHCP RangeTemplate object found at least 1 matching entry\n";

#Modify one of the attributes of the specified DHCP RangeTemplate
$object->comment("modified comment");

#Un-override bootfile
$object->bootfile(undef);

#Un-override bootserver
$object->bootserver(undef);

#Un-override nextserver
$object->nextserver(undef);

#Apply the changes
$session->modify($object)
    or die("Modify DHCP Range Template failed: ",
            $session->status_code() . ":" . $session->status_detail());

print "DHCP RangeTemplate object modified successfully \n";
```

#Remove a DHCP RangeTemplate

```
#Get a DHCP RangeTemplate through the session
my @retrieved_objs = $session->get(
    object     => "Infoblox::DHCP::RangeTemplate",
    name       => "custom_range_template",
);

my $object = $retrieved_objs[0];
unless ($object) {
       die("Get DHCP Range Template failed: ",
            $session->status_code() . ":" . $session->status_detail());
}
print "Get DHCP RangeTemplate object found at least 1 matching entry\n";

#submit the object for removal
$session->remove($object)
    or die("Remove DHCP Range Template failed: ",
            $session->status_code() . ":" . $session->status_detail());

print "DHCP Range Template object removed successfully \n";

####PROGRAM ENDS####
```

AUTHOR

Infoblox Inc. http://www.infoblox.com/

SEE ALSO

Infoblox::Session, Infoblox::DHCP::NetworkTemplate, Infoblox::DHCP::ExclusionRangeTemplate, Infoblox::DHCP::Template, Infoblox::DHCP::FilterRule::MAC, Infoblox::DHCP::FilterRule::NAC, Infoblox::DHCP::FilterRule::RelayAgent, Infoblox::DHCP::FilterRule::UserClass, Infoblox::DHCP::Member, Infoblox::DHCP::Option

---

COPYRIGHT

Copyright (c) 2017 Infoblox Inc.

Infoblox::DHCP::NetworkTemplate - DHCP Network Template object

- NAME
- DESCRIPTION
- CONSTRUCTOR
- SESSION METHODS
  - Infoblox::Session->add( )
  - Infoblox::Session->get( )
  - Infoblox::Session->search( )
  - Infoblox::Session->modify( )
  - Infoblox::Session->remove( )
- METHODS
  - allow_any_netmask( )
  - authority( )
  - auto_create_reversezone( )
  - bootfile( )
  - bootserver( )
  - comment( )
  - ddns_domainname( )
  - ddns_generate_hostname( )
  - ddns_update_fixed_addresses( )
  - ddns_server_always_updates( )
  - ddns_ttl( )
  - ddns_use_option81( )
  - deny_bootp( )
  - email_list( )
  - enable_ddns( )
  - enable_dhcp_thresholds( )
  - enable_email_warnings( )
  - enable_snmp_warnings( )
  - extattrs( )
  - extensible_attributes( )
  - fixed_address_templates( )
  - high_water_mark( )
  - high_water_mark_reset( )
  - ignore_dhcp_option_list_request( )
  - ipam_email_addresses( )
  - ipam_trap_settings( )
  - ipam_threshold_settings( )
  - lease_scavenge_time( )
  - logic_filters( )
  - low_water_mark( )
  - low_water_mark_reset( )
  - members( )
  - name( )
  - netmask( )
  - nextserver( )
  - options( )
  - override_ddns_domainname( )
  - override_ddns_ttl( )
  - override_ipam_email_addresses( )
  - override_ipam_threshold_settings( )
  - override_ipam_trap_settings( )
  - override_lease_scavenge_time( )
  - override_logic_filters( )
  - override_update_dns_on_lease_renewal( )
  - pxe_lease_time( )
  - range_templates( )
  - update_dns_on_lease_renewal( )
  - recycle_leases( )
  - rir( )
  - rir_organization( )
  - rir_registration_action( )
  - rir_registration_status( )
  - send_rir_request( )
- SAMPLE CODE
- AUTHOR
- SEE ALSO
- COPYRIGHT

NAME

Infoblox::DHCP::NetworkTemplate - DHCP Network Template object

DESCRIPTION

You can use a template to create networks in a quick and consistent way. Networks created from a network template inherit all the properties defined in the network template, except for the comment and netmask if they are defined in the network.

CONSTRUCTOR

```
 my $network_template = Infoblox::DHCP::NetworkTemplate->new(
        name                            => $string,
#Required
        allow_any_netmask               => "true" | "false",
#Optional / Default is "false"
        authority                       => "true" | "false",
#Optional / Default is "false"
        auto_create_reversezone         => "true" | "false"
#Optional / Default is "false"
        bootfile                        => $filename | undef,
#Optional / Default is undefined
        bootserver                      => $ipv4addr | $fqdn | undef,
#Optional / Default is undefined
        comment                         => $string,
#Optional / Default is empty
        ddns_domainname                 => $string | undef,
#Optional / Default is undefined
        ddns_generate_hostname          => "true" | "false" | undef,
#Optional / Default is undefined
        ddns_update_fixed_addresses     => "true" | "false" | undef,
#Optional / Default is undefined
        ddns_server_always_updates      => "true" | "false",
#Optional / Default is "true"
        ddns_ttl                        => $num,
#Optional / Default is 0
        ddns_use_option81               => "true" | "false" | undef,
#Optional / Default is undefined
        deny_bootp                      => "true" | "false" | undef,
#Optional / Default undefined
        email_list                      => [ "admin\@test.com",...],
#Optional / Default is undefined
        enable_ddns                     => "true" | "false" | undef,
#Optional / Default is undefined
        enable_dhcp_thresholds          => "true" | "false",
#Optional / Default is undefined
        enable_email_warnings           => "true" | "false",
#Optional / Default is undefined
        enable_snmp_warnings            => "true" | "false",
#Optional / Default is undefined
        extattrs                        => { $string => $extattr, ... },
#Optional / Default is undefined
        extensible_attributes           => { $string => $string | $num, $string => [ $string | $num, ... ],
... }, #Required if 'rir_organization' attribute is defined

#Otherwise is optional
        fixed_address_templates         => [$FixedAddrTemplate1,$FixedAddrTemplate2,...]
#Optional / Default is empty
        high_water_mark                 => $number,
#Optional / Default is "95"
        high_water_mark_reset           => $number,
#Optional / Default is "95"
        ignore_dhcp_option_list_request => "true" | "false" | undef,
#Optional / Default is undefined
        lease_scavenge_time             => $number,
#Optional / Default is -1
        low_water_mark                  => $number,
#Optional / Default is "0"
        low_water_mark_reset            => $number,
#Optional / Default is "0"
        members                         => [$Member1,$Member2,...],
#Optional / Default is empty
        netmask                         => $netmask,
#Optional / Default is /8
        nextserver                      => $ipv4addr | $fqdn | undef,
#Optional / Default is undefined
        options                         => [$Option1,$Option2,...]
#Optional / Default is empty
        override_ddns_ttl               => "true" | "false",
#Optional / Default value is "false"
        override_ddns_domainname        => "true" | "false",
#Optional / Default value is "false"
        override_lease_scavenge_time    => "true" | "false",
```

```
#Optional / Default value is "false"
        override_update_dns_on_lease_renewal  => "true" | "false",
#Optional / Default value is "false"
        pxe_lease_time                => $num | undef,
#Optional / Default is undefined
        range_templates               => [$RangeTemplate1,$RangeTemplate2,...],
#Optional / Default is empty
        recycle_leases                => "true" | "false" | undef,
#Optional / Default is undefined
        rir_organization              => $rir_org,
#Optional / An Infoblox::Grid::RIR::Organization object
        rir_registration_action       => 'NONE' | 'CREATE',
#Optional / Default is 'NONE'
        rir_registration_status       => 'NOT_REGISTERED' | 'REGISTERED',
#Optional / Default is 'NOT_REGISTERED'
        send_rir_request              => 'true' | 'false',
#Optional / Default is 'false'
        update_dns_on_lease_renewal   => "true" | "false",
#Optional / Default value is "false"
        ipam_trap_settings             => $ipam_trap,
#Optional / An Infoblox::Grid::SNMP::IPAMTrap object
        override_ipam_trap_settings   => "true" | "false",
#Optional / Default value is "false"
        ipam_threshold_settings        => $ipam_threshold,
#Optional / An Infoblox::Grid::SNMP::IPAMThreshold object
        override_ipam_threshold_settings => "true" | "false",
#Optional / Default value is "false"
        ipam_email_addresses          => [$email1, $email2, ...],
#Optional / Default is an empty list
        override_ipam_email_addresses => "true" | "false",
#Optional / Default value is "false"
        logic_filters                 => [ $filter_name | $filter_obj, ...], # Optional / Default is
undefined
        override_logic_filters        => "true" | 'false",                  # Optional / Default value
is "false"
      );
```

You cannot set both extattrs and extensible_attributes attributes at the same time.

SESSION METHODS

This section describes all the methods in an Infoblox::Session module that you can apply to a DHCP Network Template object.

Infoblox::Session->add( )

Use this method to add a network template object to the Infoblox appliance. See Infoblox::Session->add() for parameters and return values.

Example

```
#Construct an object
my $network_template = Infoblox::DHCP::NetworkTemplate->new(
       name             => "custom_network_template",
       allow_any_netmask => "true",
);

# Submit for adding network template
my $response = $session->add( $network_template );
```

Infoblox::Session->get( )

Use this method to retrieve the matching network template objects from the Infoblox appliance. See Infoblox::Session->get() for parameters and return values.

Key References

```
Apply the following attributes to get a specific DHCP Network template object:

 name - Required. The name of network template.
 extattrs     - Optional. A hash reference containing extensible attributes.
```

Example

```
 my  @result_array = $session->get(
      object  => "Infoblox::DHCP::NetworkTemplate" ,
      name    => "custom_network_template"
 );
```

Infoblox::Session->search( )

Use this method to retrieve all the matching network template objects from the Infoblox appliance. See Infoblox::Session->search() for parameters and return values.

Key References

```
Apply the following attributes to search for a specific DHCP Network Template object:

 name    - Optional. The name of the network template (regular
           expression).
 comment - Optional. The comment of the network template (regular
           expression).
 extattrs     - Optional. A hash reference containing extensible attributes.
 extensible_attributes - Optional. A hash reference containing extensible attributes.
```

For more information about searching extensible attributes, see Infoblox::Grid::ExtensibleAttributeDef/Searching Extensible Attributes.

Example

```
my  @result_array = $session->search(
     object  => "Infoblox::DHCP::NetworkTemplate" ,
     name    => "custom_network_template"
 comment => ".*",
);

# search all Network Templates with a given extensible attribute 'Site'
my @retrieved_objs = $session->search(
  object => "Infoblox::DHCP::NetworkTemplate",
  extensible_attributes => { 'Site' => 'Santa Clara' });
```

Infoblox::Session->modify( )

Use this method to modify a network template object in the Infoblox appliance. See Infoblox::Session->modify() for parameters and return values.

Example

```
#Use method to modify enable_ddns method.
$object->enable_ddns("true");
# Submit modification
my $response = $session->modify( $object );
```

Infoblox::Session->remove( )

Use this method to remove an object from the Infoblox appliance. See Infoblox::Session->remove() for parameters and return values.

To remove a specifc object, first use `get()` or `search()` to retrieve the specific object, and then submit this object for removal.

Example

```
# Get the object under the same name
my  @result_array = $session->get(

      object => "Infoblox::DHCP::NetworkTemplate" ,
      name   => "custom_network_template"
            );

# find the desired object from retrieved list.
my $desired_network_template = $result_array[0];

# Submit for removal
my $response = $session->remove( $desired_network_template );
```

METHODS

This section describes all the methods that can be used to configure and retrieve the attribute values of a DHCP Network template object

allow_any_netmask( )

Use this method to set or retrieve the allow_any_netmask parameter of a DHCP network template object. If you set this parameter to "true", then the template allows any netmask. A user must specify a netmask when creating a network using this template. If you set this parameter to false, you must specify the "netmask" property for the network template object.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

Specify "true" to set the allow_any_netmask parameter or specify "false" if you want to specify a particular netmask. The default value is false.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
# Get allow_any_netmask
my $allow_any_netmask = $network_template->allow_any_netmask()
# Modify allow_any_netmask
$network_template->allow_any_netmask("true");
```

authority( )

Use this method to set or retrieve the authority flag of a DHCP network template object. This flag specifies if a DHCP server is authoritative for the network.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

Specify "true" to set the authority flag or "false" to deactivate/unset it. The default value is undefined which indicates that this attribute inherits the upper-level setting.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
# Get authority
my $authority = $network_template->authority()
# Modify authority
$network_template->authority("true");
```

auto_create_reversezone( )

Use this method to set or retrieve the auto_create_reversezone flag of a DHCP network template object. This flag indicates whether the appliance must automatically create the corresponding reverse zone.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

Specify "true" to create a reverse zone automatically or "false" to disable the automatic creation of a reverse zone.The default value for this field is false.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
# Get auto_create_reversezone
my $auto_create_reversezone = $network_template->auto_create_reversezone();
# Modify auto_create_reversezone
$network_template->auto_create_reversezone("true");
```

bootfile( )

Use this method to set or retrieve the bootfile attribute of a DHCP network template object. You can configure the DHCP server to support clients that use the boot file name option in their DHCPREQUEST messages. The default value is undefined which indicates that this attribute inherits the upper-level setting.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

This option is overridden independently from Infoblox::DHCP::NetworkTemplate->bootserver() and Infoblox::DHCP::NetworkTemplate->nextserver().

Parameter

The name, in string format, of the file that the client must download. The default value is undefined.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
# Get bootfile
my $bootfile = $network_template->bootfile();
# Modify bootfile
$network_template->bootfile("bootfile1");
#Un-override bootfile
$network_template->bootfile(undef);
```

bootserver( )

Use this method to set or retrieve the boot server attribute of a DHCP network template object. You can specify the name of the boot server on which the boot file is stored. The default value is undefined which indicates that this attribute inherits the upper-level setting.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

This option is overridden independently from Infoblox::DHCP::NetworkTemplate->bootfile() and Infoblox::DHCP::NetworkTemplate->nextserver().

Parameter

The next server IP address or name in FQDN (Fully Qualified Domain Name) format. The FQDN consists of the host name followed by the domain name (example: abc.com). A next server name can have a maximum of 256 bytes. The default value is undefined.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
# Get bootserver
my $bootserver = $network_template->bootserver();
# Modify bootserver
$network_template->bootserver("abc.domain.com");
#Un-override bootserver
$network_template->bootserver(undef);
```

comment( )

Use this method to set or retrieve a descriptive comment about a DHCP network template object.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

Enter a descriptive comment for the network template in string format with a maximum of 256 bytes.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
# Get comment
my $comment = $network_template->comment();
# Modify comment
$network_template->comment("add a network");
```

ddns_domainname( )

Use this method to set or retrieve the ddns_domainname value of a DHCP Network template object.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Setting this method to a defined value implicitly sets the override_ddns_domainname method to "true". Setting the parameter to undefined causes the appliance to use the grid default and automatically resets the override_ddns_domainname attribute to "false".

Note that when ddns_domainname is set to a defined value and override_dddns_domainname is set to "false", the last operation takes precedence. Thus the sequence $object->ddns_domainname("testdomain"); $object->override_ddns_domainname("false"); will set override_ddns_domainname to "false", and the sequence $object->override_ddns_domainname("false"); $object->ddns_domainname("testdomain"); will result in override_ddns_domainname="true".

Parameter

The dynamic DNS domain name in string format.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get ddns_domainname
my $ddns_domainname = $network_template->ddns_domainname();
#Modify ddns_domainname
$network_template->ddns_domainname("test_domain.com");
```

ddns_generate_hostname( )

Use this method to set or retrieve the ddns_generate_hostname flag of a DHCP network template object. Set this flag to allow only the DHCP server to generate a host name. This setting overrides the upper-level settings.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

Specify "true" to generate hostname or "false" to deactivate/unset it. The default value is undefined which indicates that this attribute inherits the upper-level setting.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
# Get ddns_generate_hostname
my $ddns_generate_hostname = $network_template->ddns_generate_hostname()
# Modify ddns_generate_hostname
$network_template->ddns_generate_hostname("true");
#Un-override
$network_template->ddns_generate_hostname(undef);
```

ddns_update_fixed_addresses( )

Use this method to set or retrieve the ddns_update_fixed_addresses flag of a DHCP network template object. By default, the DHCP server does not update DNS when it allocates a fixed address to a client. You can configure the DHCP server to update the A and PTR records of a client with a fixed address. When this feature is enabled and the DHCP server adds A and PTR records for a fixed address, the DHCP server never discards the records. This setting overrides the upper-level settings.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

Specify "true" to update DNS when it allocates a fixed address to a client or "false" to deactivate/unset it. The default value is undefined which indicates that this attribute inherits the upper-level setting.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
# Get ddns_update_fixed_addresses
my $ddns_update_fixed_addresses= $network_template->ddns_update_fixed_addresses()
# Modify ddns_update_fixed_addresses
$network_template->ddns_update_fixed_addresses("true");
#Un-override
$network_template->ddns_update_fixed_addresses(undef);
```

ddns_server_always_updates( )

Use this method to set or retrieve the ddns_server_always_updates flag of a DHCP network template object. Set this flag to allow the DHCP server only to update DNS, regardless of the requests from DHCP clients.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

Specify "true" to set this flag and allow the DHCP server only to update DNS or "false" to deactivate/unset the flag. The default value is true.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
# Get ddns_server_always_updates
my $ddns_server_always_updates = $network_template->ddns_server_always_updates()
# Modify ddns_server_always_updates
$network_template->ddns_server_always_updates("true");
```

ddns_ttl( )

Use this method to set or retrieve the DNS update Time to Live (TTL) value of a DHCP network template object. The TTL value indicates the number of seconds the Infoblox appliance caches a DDNS update.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Setting this method to a defined value implicitly sets the override_ddns_ttl method to "true". Setting the parameter to undefined causes the appliance to use the grid default and automatically resets the override_ddns_ttl attribute to "false".

Note that when ddns_ttl is set to a defined value and override_dddns_ttl is set to "false", the last operation takes precedence. Thus the sequence $object->ddns_ttl(12); $object->override_ddns_ttl("false"); will set override_ddns_ttl to "false", and the sequence $object->override_ddns_ttl("false"); $object->ddns_ttl(12); will result in override_ddns_ttl="true".

Parameter

A 32-bit integer (range from 0 to 4294967295) that represents the duration, in seconds, that the update is cached. Zero indicates that the update should not be cached. The default value is 0.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get DNS Update ttl
my $dns_ttl = $network_template->ddns_ttl();
#Modify DNS Update ttl
$network_template->ddns_ttl(1200);
```

ddns_use_option81( )

Use this method to set or retrieve the ddns_use_option81 flag of a DHCP network template object. Enable this method to define the settings for option 81 at the network template level. This setting overrides the upper-level settings.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

Specify "true" to enable support for option 81 or "false" to disable support. The default value is undefined which indicates that this attribute inherits the upper-level setting.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
# Get ddns_use_option81
my $ddns_use_option81 = $network_template->ddns_use_option81()
# Modify ddns_use_option81
```

```
$network_template->ddns_use_option81("true");
# un-override ddns_use_option81
$network_template->ddns_use_option81(undef);
```

deny_bootp( )

Use this method to set or retrieve the deny_bootp flag of a DHCP Network object.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

Specify "true" to set the deny_bootp flag or "false" to deactivate/unset it. The default value is undefined which indicates that this attribute inherits the upper-level setting.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get deny_bootp
my $deny_bootp = $network->deny_bootp();
#Modify deny_bootp
$network->deny_bootp("true");
```

email_list( )

Use this method to set or retrieve the e-mail lists to which the appliance sends threshold e-mail messages.

If you enter e-mail addresses in the email_list, then the appliance sends DHCP threshold alarms to the e-mail addresses.

Parameter

The valid value is an array reference that contains string objects.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get email_list
my @email_list = $network_template->email_list();
#Modify email_list
$network_template->email_list( [ "admin\@test.com" ] );
```

enable_ddns( )

Use this method to set or retrieve the dynamic DNS updates flag of a DHCP network template object. The DHCP server can send DDNS updates to DNS servers in the same grid and to external DNS servers. This setting overrides the upper-level settings.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

Specify "true" to set the dynamic DNS updates flag or "false" to deactivate/unset it. The default value is undefined which indicates that this attribute inherits the upper-level setting.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
# Get enable_ddns
my $enable_ddns = $network_template->enable_ddns()
# Modify enable_ddns
$network_template->enable_ddns("true");
# Un-override enable_ddns
$network_template->enable_ddns(undef);
```

enable_dhcp_thresholds( )

Use this method to set or retrieve the DHCP thresholds of a network template object. These thresholds are the high and low watermarks. If DHCP range usage goes above or below these percentage values, the Infoblox appliance generates a syslog message and sends an SNMP notification.

Enable this flag to set DHCP threshold values.

Parameter

Specify "true" to set the enable_dhcp_thresholds flag or "false" to deactivate/unset it. The default value is undefined.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get enable_dhcp_thresholds
my $enable_dhcp_thresholds = $network_template->enable_dhcp_thresholds();
#Modify enable_dhcp_thresholds
$network_template->enable_dhcp_thresholds("false");
```

enable_email_warnings( )

Use this method to send DHCP thresholds warnings through e-mail.

Enable this flag if you want the appliance to send DHCP threshold warnings to the "email_list" provided when DHCP threshold is enabled.

Parameter

+Specify "true" to set the enable_email_warnings flag or "false" to deactivate/unset it. The default value is undefined.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get enable_email_warnings
my $enable_email_warnings = $network_template->enable_email_warnings();
#Modify enable_email_warnings
$network_template->enable_email_warnings("false");
```

enable_snmp_warnings( )

Use this method to send DHCP threshold warnings through SNMP.

If the flag is enabled, then the appliance sends DHCP threshold warnings through SNMP messages when DHCP threshold is enabled.

Parameter

Specify "true" to set the enable_snmp_warnings flag or "false" to deactivate/unset it. The default value is undefined.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get enable_snmp_warnings
my $enable_snmp_warnings = $network_template->enable_snmp_warnings();
#Modify enable_snmp_warnings
$network_template->enable_snmp_warnings("false");
```

extattrs( )

Use this method to set or retrieve the extensible attributes associated with a Network Template object.

Parameter

Valid value is a hash reference containing the names of extensible attributes and their associated values (Infoblox::Grid::Extattr

objects).

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get extattrs
my $ref_extattrs = $network_template->extattrs();
#Modify extattrs
$network_template->extattrs({ 'Site' => $extattr1, 'Administrator' => $extattr2 });
```

extensible_attributes( )

Use this method to set or retrieve the extensible attributes associated with a Network Template object.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

For valid values for extensible attributes, see Infoblox::Grid::ExtensibleAttributeDef/Extensible Attribute Values.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get extensible attributes
my $ref_extensible_attributes = $network_template->extensible_attributes();
#Modify extensible attributes
$network_template->extensible_attributes({ 'Site' => 'Santa Clara', 'Administrator' => [ 'Peter',
'Tom' ] });
```

fixed_address_templates( )

Use this method to set or retrieve the fixed address template assigned to this network template object. When you create a network based on a network template object with a fixed address template, the fixed addresses are created based on the associated fixed address template.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

The valid value is an array reference of Infoblox::DHCP::Template objects.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
# Get fixed_address_templates
my $fixed_address_templates = $network_template->fixed_address_templates();
# Modify fixed_address_templates
my $tmpl = Infoblox::DHCP::Template->new(
   name => "custom_fixed_address_template"
);
$network_template->fixed_address_templates([$tmpl]);
```

high_water_mark( )

Use this method to set the percentage value for DHCP range usage above which the Infoblox appliance generates a syslog message and sends a warning.

When the percent of allocated addresses in a DHCP range exceeds the high_water_mark, the appliance generates a syslog message and sends a warning.

Parameter

The valid value is in string format.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get high_water_mark
my $high_water_mark = $network_template->high_water_mark();
#Modify high_water_mark
$network_template->high_water_mark("80");
```

high_water_mark_reset( )

Use this method to set or retrieve the high_water_mark_reset value. If the percentage of allocated addresses drops below this value, the corresponding SNMP trap is reset.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

A number that specifies the percentage of allocated addresses. The range is from 1 to 100. The high watermark reset value must be lower than the high water mark value. The default value is 95.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get high_water_mark_reset
my $high_water_mark_reset = $network_template->high_water_mark_reset();
#Modify high_water_mark_reset
$network_template->high_water_mark_reset(70);
```

ignore_dhcp_option_list_request( )

Use this method to set or retrieve the ignore_dhcp_option_list_request flag of a DHCP network template object. Set this flag to ignore option lists requested by clients and return all defined options at the network template level.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

Specify "true" to set the ignore_dhcp_option_list_request flag or "false" to deactivate/unset it. The default value is empty.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
# Get ignore_dhcp_option_list_request
my $ignore_dhcp_option_list_request = $network_template->ignore_dhcp_option_list_request();
# Modify ignore_dhcp_option_list_request
$network_template->ignore_dhcp_option_list_request("false");
# Un-override ignore_dhcp_option_list_request
$network_template->ignore_dhcp_option_list_request(undef);
```

ipam_email_addresses( )

Use this method to set or retrieve a list of IPAM Utilization email addresses.

Setting this method to a defined value implicitly sets the override_ipam_email_addresses method to "true". Setting the parameter to undefined causes the appliance to use the default from the upper level and automatically resets the override_ipam_email_addresses attribute to "false".

Note that when ipam_email_addresses is set to a defined value and override_ipam_email_addresses is set to "false", the last operation takes precedence. Thus the sequence $object->ipam_email_addresses(['admin@infoblox.com']); $object->override_ipam_email_addresses("false"); will set override_ipam_email_addresses to "false", and the sequence $object->override_ipam_email_addresses("false"); $object->ipam_email_addresses(['admin@infoblox.com']); will result in override_ipam_email_addresses="true".

Parameter

Valid value is an array reference that contains email addresses in string format.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get ipam_email_addresses
my $ipam_email_addresses = $network_template->ipam_email_addresses();
#Modify ipam_email_addresses
$network_template->ipam_email_addresses($ipam_email_addresses);
```

ipam_trap_settings( )

Use this method to set or retrieve IPAM Utilization SNMP Trap settings.

Setting this method to a defined value implicitly sets the override_ipam_trap_settings method to "true". Setting the parameter to undefined causes the appliance to use the default from the upper level and automatically resets the override_ipam_trap_settings attribute to "false".

Note that when ipam_trap_settings is set to a defined value and override_ipam_trap_settings is set to "false", the last operation takes precedence. Thus the sequence $object->ipam_trap_settings($ipam_trap); $object->override_ipam_trap_settings("false"); will set override_ipam_trap_settings to "false", and the sequence $object->override_ipam_trap_settings("false"); $object->ipam_trap_settings($ipam_trap); will result in override_ipam_trap_settings="true".

Parameter

Valid value is an Infoblox::Grid::SNMP::IPAMTrap object.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get ipam_trap_settings
my $ipam_trap_settings = $network_template->ipam_trap_settings();
#Modify ipam_trap_settings
$network_template->ipam_trap_settings($ipam_trap_settings);
```

ipam_threshold_settings( )

Use this method to set or retrieve IPAM Utilization SNMP Threshold settings.

Setting this method to a defined value implicitly sets the override_ipam_threshold_settings method to "true". Setting the parameter to undefined causes the appliance to use the default from the upper level and automatically resets the override_ipam_threshold_settings attribute to "false".

Note that when ipam_threshold_settings is set to a defined value and override_ipam_threshold_settings is set to "false", the last operation takes precedence. Thus the sequence $object->ipam_threshold_settings($ipam_threshold_settings); $object->override_ipam_trap_settings("false"); will set override_ipam_threshold_settings to "false", and the sequence $object->override_ipam_threshold_settings("false"); $object->ipam_threshold_settings($ipam_threshold_settings); will result in override_ipam_threshold_settings="true".

Parameter

Valid value is an Infoblox::Grid::SNMP::IPAMThreashold object.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get ipam_threshold_settings
my $ipam_threshold_settings = $network_template->ipam_threshold_settings();
#Modify ipam_threshold_settings
$network_template->ipam_threshold_settings($ipam_threshold_settings);
```

lease_scavenge_time( )

Use this method to set or retrieve the lease_scavenge_time value of a Network Template object.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Setting this method to a defined value implicitly sets the override_lease_scavenge_time method to "true". Setting the parameter to undefined causes the appliance to use the default from the upper level and automatically resets the override_lease_scavenge_time attribute to "false".

Note that when lease_scavenge_time is set to a defined value and override_lease_scavenge_time is set to "false", the last operation takes precedence. Thus the sequence $object->lease_scavenge_time(604800); $object->override_lease_scavenge_time("false"); will set override_lease_scavenge_time to "false", and the sequence $object->override_lease_scavenge_time("false"); $object->lease_scavenge_time(604800); will result in override_lease_scavenge_time="true".

Parameter

An integer that specifies the period of time, in seconds, that free and backup leases remain in the database before they are automatically deleted. To disable lease scavenging, set the parameter to -1. The minimum positive value must be greater than 86400 seconds (1 day). The default value of this parameter is -1, which means that lease scavenging is disabled.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get lease_scavenge_time
my $lease_scavenge = $network_template->lease_scavenge_time();
#Modify lease_scavenge_time
$network_template->lease_scavenge_time(30 * 24 * 60 * 60);
```

logic_filters( )

Use this method to set or retrieve all the logic (ordered) filters associated with the DHCP Network Template.

Filters are used to screen address assignments in the DHCP Network Template if the addresses are to be assigned to specific hosts.

Setting this method to a defined value implicitly sets the override_logic_filters method to "true". Setting the parameter to undefined causes the appliance to use the grid default and automatically resets the override_logic_filters attribute to "false".

Note that when logic_filters is set to a defined value and override_dlogic_filters is set to "false", the last operation takes precedence. Thus the sequence $object->logic_filters($filters); $object->override_logic_filters("false"); will set override_logic_filters to "false", and the sequence $object->override_logic_filters("false"); $object->logic_filters($filters); will result in override_logic_filters="true".

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

The valid value is an array reference that contains either the Infoblox::DHCP::Filter::MAC, Infoblox::DHCP::Filter::NAC, Infoblox::DHCP::Filter::Option objects or the names of the corresponding filters. The filters in the list must satisfy the following conditions:

Filter name must be globally unique.
Filters without expressions must be at the end of the list.
The filters should not be disabled.

The default value of the parameter is undefined.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get logic_filters
my $ref_logic_filters = $network_template->logic_filters();
#Modify logic_filters
$network_template->logic_filters(["filter_1","filter_2",$exl1]); #$exl1 is an
Infoblox::DHCP::Filter::MAC, Infoblox::DHCP::Filter::NAC or Infoblox::DHCP::Filter::Option object.
```

low_water_mark( )

Use this method to set the percentage value for DHCP range usage below which the Infoblox appliance generates a syslog message and

sends a warning.

When the percent of allocated addresses in a DHCP range goes below the low_water_mark, the appliance generates a syslog message and sends a warning.

Parameter

The valid value is in string format.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get low_water_mark
my $low_water_mark = $network_template->low_water_mark();
#Modify low_water_mark
$network_template->low_water_mark("10");
```

low_water_mark_reset( )

Use this method to set or retrieve the low_water_mark_reset value. If the percentage of allocated addresses exceeds this value, a corresponding SNMP trap is reset.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

A number that specifies the percentage of allocated addresses. The range is from 1 to 100. The low watermark reset value must be higher than the low watermark value. The default low watermark reset number is 0.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get low_water_mark_reset
my $low_water_mark_reset = $network_template->low_water_mark_reset();
#Modify low_water_mark_reset
$network_template->low_water_mark_reset(10);
```

members( )

Use this method to set or retrieve the member or Microsoft (r) server that serves DHCP for the network created from the network template object.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

The valid value is an array reference that contains Infoblox::DHCP::Member or Infoblox::DHCP::MSServer objects.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
my $member1 = Infoblox::DHCP::Member->new(
    name     => "infoblox.localdomain",
    ipv4addr => "192.168.1.2",
);

# Get members
my $members = $network_template->members();
# Modify members
$network_template->members([$member1]);
```

name( )

Use this method to set or retrieve the description of the name of the DHCP network template.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

Name of the network template in string format with a maximum of 64 bytes.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get name
my $name = $network_template->name();
#Modify name
$network_template->name("custom_network_template");
```

netmask( )

Use this method to set or retrieve the netmask attribute of a DHCP network template object. If "allow_any_netmask" is set to "false", then this attribute must be specified.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

A valid subnet mask. The default value is "255.0.0.0".

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get netmask
my $netmask = $network_template->netmask();
#Set netmask
$nework_template->allow_any_netmask( "false" );
my $netmask = $network_template->netmask("255.0.0.0");
```

nextserver( )

Use this method to set or retrieve the nextserver attribute of a DHCP network template object. You can specify the name and/or IP address of the next server to which the host needs to connect in order to boot. This setting overrides the upper-level settings. The default value is undefined which indicates that this attribute inherits the upper-level setting.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

This option is overridden independently from Infoblox::DHCP::NetworkTemplate->bootfile() and Infoblox::DHCP::NetworkTemplate->bootserver().

Parameter

The next server IP address or name in FQDN (Fully Qualified Domain Name) format. The FQDN consists of the host name followed by the domain name (example: abc.com). A next server name can have a maximum of 256 bytes. The default value is undefined.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
# Get nextserver
my $nextserver = $network_template->nextserver();
# Modify nextserver
$network_template->nextserver("blue.domain.com");
#Un-override nextserver
$network_template->nextserver(undef);
```

options( )

Use this method to set or retrieve the options value of a network template object. DHCP options provide network configuration settings and various services available on the network for DHCP. This setting overrides the upper-level setting. The default value is undefined which indicates that this attribute inherit the upper-level setting.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

The valid value is an array reference that contains Infoblox::DHCP::Option objects.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get options
my $options = $network_template->options();
#Modify options
my $option1 = Infoblox::DHCP::Option->new(
    name  => "lease-time",
    value => 50
);

$network_template->options([$option1]);
```

override_ddns_domainname( )

The override_ddns_domainname attribute controls whether the ddns_domainname value in the DHCP network template object is used, instead of the grid default.

The override_ddns_domainname attribute can be specified explicitly. It is also set implicitly when ddns_domainname is set to a defined value.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

Set the parameter to "true" to override the grid-level setting for ddns_domainname. Set the parameter to "false" to inherit the grid-level setting for ddns_domainname.

The default value of this parameter is "false".

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Getting override_ddns_domainname
my $override_ddns_domainname=$network_template->override_ddns_domainname( );
#Modifying override_ddns_domainname
$network_template->override_ddns_domainname("true");
```

override_ddns_ttl( )

The override_ddns_ttl attribute controls whether the ddns_ttl value in the DHCP network template object is used, instead of the grid default.

The override_ddns_ttl attribute can be specified explicitly. It is also set implicitly when ddns_ttl is set to a defined value.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

Set the parameter to "true" to override the grid-level setting for ddns_ttl. Set the parameter to "false" to inherit the grid-level setting for ddns_ttl.

The default value of this parameter is "false".

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Getting override_ddns_ttl
my $override_ddns_ttl=$network_template->override_ddns_ttl( );
#Modifying override_ddns_ttl
$network_template->override_ddns_ttl("true");
```

override_ipam_email_addresses( )

The override_ipam_email_addresses controls whether the ipam_email_addresses in the DHCP network template is used, instead of the upper-level default.

The override_ipam_email_addresses can be specified explicitly. It is also set implicitly when ipam_email_addresses is set to a defined value.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

Set the parameter to "true" to override the upper-level setting for ipam_email_addresses. Set the parameter to "false" to inherit the upper-level setting for ipam_email_addresses.

The default value of this parameter is "false".

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get attrubute value
my $value=$network_template->override_ipam_email_addresses();
#Modifying attribute value
$network_template->override_ipam_email_addresses('true');
```

override_ipam_threshold_settings( )

The override_ipam_threshold_settings controls whether the ipam_threshold_settings in the DHCP network template is used, instead of the upper-level default.

The override_ipam_threshold_settings can be specified explicitly. It is also set implicitly when ipam_threshold_settings is set to a defined value.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

Set the parameter to "true" to override the upper-level setting for ipam_threshold_settings. Set the parameter to "false" to inherit the upper-level setting for ipam_threshold_settings.

The default value of this parameter is "false".

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get attrubute value
my $value=$network_template->override_ipam_threshold_settings();
#Modifying attribute value
$network_template->override_ipam_threshold_settings('true');
```

override_ipam_trap_settings( )

The override_ipam_trap_settings controls whether the ipam_trap_settings in the DHCP network template is used, instead of the upper-level default.

The override_ipam_trap_settings can be specified explicitly. It is also set implicitly when ipam_trap_settings is set to a defined value.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

Set the parameter to "true" to override the upper-level setting for ipam_trap_settings. Set the parameter to "false" to inherit the upper-level setting for ipam_trap_settings.

The default value of this parameter is "false".

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get attrubute value
my $value=$network_template->override_ipam_trap_settings();
#Modifying attribute value
$network_template->override_ipam_trap_settings('true');
```

override_lease_scavenge_time( )

The override_lease_scavenge_time attribute controls whether the lease_scavenge_time value in the network template is used, instead of the upper-level default.

The override_lease_scavenge_time attribute can be specified explicitly. It is also set implicitly when lease_scavenge_time is set to a defined value.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

Set the parameter to "true" to override the upper-level setting for lease_scavenge_time. Set the parameter to "false" to inherit the setting for lease_scavenge_time from the upper level.

The default value of this parameter is "false".

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Getting override_lease_scavenge_time
my $override_lease_scavenge_time=$network_template->override_lease_scavenge_time( );
#Modifying override_lease_scavenge_time
$network_template->override_lease_scavenge_time("true");
```

override_logic_filters( )

The override_logic_filters attribute controls whether the logic_filters value of the DHCP Network Template is used, instead of the Grid default.

The override_logic_filters attribute can be specified explicitly. It is also set implicitly when logic_filters is set to a defined value.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

Set the parameter to "true" to override the Grid-level setting for logic_filters. Set the parameter to "false" to inherit the Grid-level setting for logic_filters.

The default value of this parameter is "false".

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Getting override_logic_filters
my $override_logic_filters=$network_template->override_logic_filters( );
#Modifying override_logic_filters
$network_template->override_logic_filters("true");
```

override_update_dns_on_lease_renewal( )

The override_update_dns_on_lease_renewal attribute controls whether the update_dns_on_lease_renewal value in the DHCP network template object is used, instead of the grid default.

The override_update_dns_on_lease_renewal attribute can be specified explicitly. It is also set implicitly when update_dns_on_lease_renewal is set to a defined value.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

Set the parameter to "true" to override the grid-level setting for update_dns_on_lease_renewal. Set the parameter to "false" to inherit the grid-level setting for update_dns_on_lease_renewal.

The default value of this parameter is "false".

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Getting override_update_dns_on_lease_renewal
my $override_update_dns_on_lease_renewal=$network_template->override_update_dns_on_lease_renewal(
);
#Modifying override_update_dns_on_lease_renewal
$network_template->override_update_dns_on_lease_renewal("true");
```

pxe_lease_time( )

Use this method to set or retrieve the PXE lease time value of a DHCP network template object. Some hosts use PXE (Preboot Execution Environment) to boot remotely from a server. To better manage your IP resources, set a different lease time for PXE boot requests. You can configure the DHCP server to allocate an IP address with a shorter lease time to hosts that send PXE boot requests, so IP addresses are not leased longer than necessary. This setting overrides the upper-level settings. The default value is undefined, which indicates that this attribute inherits the upper-level setting.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

A 32-bit integer (range from 0 to 4294967295) that represents the duration, in seconds, that the update is cached. Zero indicates that the update should not be cached. The default value is undefined

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get PXE Lease Time
my $pxe_lease_time = $network_template->pxe_lease_time();
#Modify PXE Lease Time
$network_template->pxe_lease_time(1100);
#Un-override PXE Lease Time
$network_template->pxe_lease_time(undef);
```

range_templates( )

Use this method to set or retrieve the IP address range template assigned to this network template object. When you create a network based on a network template object with a range template, the IP address ranges are created based on the associated IP address range template.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

The valid value is an array reference of Infoblox::DHCP::Template objects.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get range_templates
my $range_templates = $network_template->range_templates();
#Modify range_templates
my $tmpl = Infoblox::DHCP::Template->new(
   name => "custom_range_template"
);
$network_template->range_templates([ $tmpl ]);
```

update_dns_on_lease_renewal( )

The update_dns_on_lease_renewal attribute controls whether the DHCP server updates DNS when a DHCP lease is renewed.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Setting this method to a defined value implicitly sets the override_update_dns_on_lease_renewal method to "true". Setting the parameter to undefined causes the appliance to use the grid default and automatically resets the override_update_dns_on_lease_renewal attribute to "false".

Note that when update_dns_on_lease_renewal is set to a defined value and override_update_dns_on_lease_renewal is set to "false", the last operation takes precedence. Thus the sequence $object->update_dns_on_lease_renewal("true"); $object->override_update_dns_on_lease_renewal("false"); will set override_update_dns_on_lease_renewal to "false", and the sequence $object->override_update_dns_on_lease_renewal("false"); $object->update_dns_on_lease_renewal("true"); will result in override_update_dns_on_lease_renewal="true".

Parameter

Specify "true" to set the update_dns_on_lease_renewal flag or "false" to deactivate/unset it.

The default value of this parameter is "false".

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get update_dns_on_lease_renewal
my $update_dns_on_lease_renewal=$network_template->update_dns_on_lease_renewal( );
#Modify update_dns_on_lease_renewal
$network_template->update_dns_on_lease_renewal("true");
```

recycle_leases( )

Use this method to set or retrieve the recycle_leases flag at the network template level.

If the flag is enabled, the leases are kept in the recycle bin until one week after expiration. When the flag is disabled, the leases are irrecoverably deleted.

Parameter

Specify "true" to enable the recycle_leases flag or "false" to disable it. The default value is undefined, which indicates that this attribute inherit the upper-level setting.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get recycle_leases
my $recycle_leases = $network_template->recycle_leases();
#Modify recycle_leases
$network_template->recycle_leases("false");
```

rir( )

Use this method to retrieve the name of the Regional Internet Registry (RIR) that allocated this address space. This is a read-only attribute.

Parameter

None

Returns

The method returns the attribute value that can be 'RIPE' or 'NONE'.

Example

```
#Get rir
my $rir = $network_template->rir();
```

rir_organization( )

Use this method to set or retrieve the Regional Internet Registry (RIR) organization object.

Parameter

An Infoblox::Grid::RIR::Organization object.

You must enable support for RIR/SWIP at the Grid level before you can set attribute values. See Infoblox::Grid->enable_rir_swip() method.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get rir_organization
my $rir_organization = $network_template->rir_organization();
#Modify rir_organization
$network_template->rir_organization($rir_org);
```

rir_registration_action( )

Use this method to set or retrieve the registration action of Regional Internet Registry (RIR).

Parameter

Valid values are 'NONE', 'CREATE'.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get rir_registration_action
my $rir_registration_action = $network_template->rir_registration_action();
#Modify rir_registration_action
$network_template->rir_registration_action('CREATE');
```

rir_registration_status( )

Use this method to set or retrieve the registration status of Regional Internet Registry (RIR).

Parameter

The registration status of Regional Internet Registry (RIR), which can be 'REGISTERED' or 'NOT_REGISTERED'.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get rir_registration_status
my $rir_registration_status = $network_template->rir_registration_status();
#Modify rir_registration_status
$network_template->rir_registration_status('NOT_REGISTERED');
```

send_rir_request( )

Use this method to set or retrieve the flag that indicates whether the appliance should send an API or email request for Regional Internet Registry (RIR).

Parameter

Specify "true" to set the send_rir_request flag or "false" to unset it.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get send_rir_request
my $send_rir_request = $network_template->send_rir_request();
#Modify send_rir_request
$network_template->send_rir_request('true');
```

SAMPLE CODE

The following sample code demonstrates the different functions that can be applied to an object such as add, search, modify, and remove. Also, this sample code includes error handling for the operations.

#Preparation prior to a DHCP network template insertion

```
#PROGRAM STARTS: Include all the modules that will be used

use strict;
use Infoblox;

#Create a session to the Infoblox appliance
my $host_ip = "192.168.1.2";
my $host_name = "infoblox.localdomain";

my $session = Infoblox::Session->new(
    master   => $host_ip,
    username => "admin",
    password => "infoblox"
);
unless ($session) {
   die("Construct session failed: ",
       Infoblox::status_code() . ":" . Infoblox::status_detail());
}
print "Session created successfully\n";

my $memberdhcp = Infoblox::DHCP::Member->new(
            name     => $host_name,
            ipv4addr => $host_ip
           );
```

#Add network template object

```
#Create a DHCP Network template object

my $network_template = Infoblox::DHCP::NetworkTemplate->new(
                               name                    => "custom_network_template",
                               netmask                 => "255.0.0.0",
                               comment                 => "add network template",
                               members                 => [ $memberdhcp ],
                               authority               => "true",
                               enable_ddns             => "true",
                               pxe_lease_time          => "3600",
                               bootserver              => "someserver.com",
                               nextserver              => "3.3.3.3",
                               bootfile                => "bootfile.com",
                               auto_create_reversezone => "true",
                               );

my $response = $session->add($network_template)
       or die("Add Network template failed: ",
             $session->status_code() . ":" . $session->status_detail());

print "Network template added successfully\n";
```

#Get and modify a network template attribute

```
#Get Network template through the session
my  @retrieved_objs = $session->get(
```

```
            object => "Infoblox::DHCP::NetworkTemplate",
            name   => "custom_network_template"
        );

 my $object = $retrieved_objs[0];

 unless ($object) {
     die("Get Network template object failed: ",
         $session->status_code() . ":" . $session->status_detail());
 }
 print "Get Network template found at least 1 matching entry\n";
```

#Search a network template attribute

```
 #Search Network template through the session
 my  @retrieved_objs = $session->search(
            object => "Infoblox::DHCP::NetworkTemplate",
            name   => "custom_network_template"
        );

 my $object = $retrieved_objs[0];

 unless ($object) {
     die("Search Network template object failed: ",
         $session->status_code() . ":" . $session->status_detail());
 }
 print "Search Network template found at least 1 matching entry\n";

 #Modify attributes of the specified Network Template

 $object->enable_ddns("true"); #Apply the changes

 #Un-override bootfile
 $object->bootfile(undef);

 #Un-override bootserver
 $object->bootserver(undef);

 #Un-override nextserver
 $object->nextserver(undef);

 $session->modify($object)
     or die("Modify Network template attribute failed: ",
            $session->status_code() . ":" . $session->status_detail());

 print "Network template object modified successfully \n";
```

#Remove a network template object

```
 #Get Network Template object through the session

  my @retrieved_objs = $session->get(
            object  => "Infoblox::DHCP::NetworkTemplate",
            name    => "custom_network_template"
       );

    my $object = $retrieved_objs[0];

    unless ($object) {
     die("Get Network Template object failed: ",
         $session->status_code() . ":" . $session->status_detail());
     }

 print "Get Network Template object found at least 1 matching entry\n";

 #Submit the object for removal

 $session->remove($object)
     or die("Remove Network Template failed: ",
         $session->status_code() . ":" . $session->status_detail());

 print "Network Template removed successfully \n";

 ####PROGRAM ENDS####
```

AUTHOR

Infoblox Inc. http://www.infoblox.com/

SEE ALSO

Infoblox::Session, Infoblox::DHCP::Member, Infoblox::DHCP::Option, Infoblox::DHCP::Network, Infoblox::DHCP::RangeTemplate, Infoblox::DHCP::FixedAddrTemplate, Infoblox::DHCP::Template, Infoblox::Session->get(), Infoblox::Session->search(), Infoblox::Session->add(), Infoblox::Session->remove(), Infoblox::Session->modify()

COPYRIGHT

Copyright (c) 2017 Infoblox Inc.

Infoblox::DHCP::IPv6Network - DHCP IPv6 Network object

- NAME
- DESCRIPTION
- CONSTRUCTOR
- SESSION METHODS
  - Infoblox::Session->add( )
  - Infoblox::Session->get( )
  - Infoblox::Session->modify( )
  - Infoblox::Session->remove( )
  - Infoblox::Session->search( )
- METHODS
  - auto_create_reversezone( )
  - comment( )
  - cloud_info( )
  - ddns_domainname( )
  - ddns_enable_option_fqdn( )
  - ddns_generate_hostname( )
  - ddns_server_always_updates( )
  - ddns_ttl( )
  - disable( )
  - discovered_bridge_domain( )
  - discovered_tenant( )
  - discovery_basic_poll_setting( )
  - discovery_member( )
  - discover_now_status( )
  - discovery_blackout_setting( )
  - discovery_engine_type( )
  - domain_name( )
  - domain_name_servers( )
  - enable_ddns( )
  - enable_discovery( )
  - enable_ifmap_publishing( )
  - enable_immediate_discovery( )
  - endpoint_sources( )
  - extattrs( )
  - extensible_attributes( )
  - members( )
  - mgm_private( )
  - mgm_private_overridable( )
  - ms_ad_user_data( )
  - network( )
  - network_container( )
  - network_view( )
  - next_available_ip( )
  - next_available_network( )
  - options( )
  - override_ddns_domainname( )
  - override_ddns_enable_option_fqdn( )
  - override_ddns_generate_hostname( )
  - override_ddns_ttl( )
  - override_domain_name( )
  - override_domain_name_servers( )
  - override_blackout_setting( )
  - override_enable_ddns( )
  - override_enable_ifmap_publishing( )
  - override_options( )
  - override_preferred_lifetime( )
  - override_recycle_leases( )
  - override_mgm_private( )
  - override_subscribe_settings( )
  - override_update_dns_on_lease_renewal( )
  - override_valid_lifetime( )
  - preferred_lifetime ( )
  - port_control_blackout_setting
  - recycle_leases( )
  - restart_if_needed( )
  - rir( )
  - rir_organization( )
  - rir_registration_action( )
  - rir_registration_status( )
  - last_rir_registration_update_sent( )
  - last_rir_registration_update_status( )

- same_port_control_discovery_blackout( )
- send_rir_request( )
- delete_reason( )
- shared_network_name( )
- subscribe_settings( )
- template( )
- unmanaged( )
- update_dns_on_lease_renewal( )
- valid_lifetime ( )
- zone_associations( )

- SAMPLE CODE
- AUTHOR
- SEE ALSO
- COPYRIGHT

NAME

Infoblox::DHCP::IPv6Network - DHCP IPv6 Network object

DESCRIPTION

When DHCP services are configured on an appliance, a network that it serves must be defined. After a network is created, you can then create all the subnetworks individually or create a parent network that encompasses all the subnetworks. This object allows the user to create IPv6 networks. For IPv4 networks, see Infoblox::DHCP::Network object.

CONSTRUCTOR

```
  my $network = Infoblox::DHCP::IPv6Network->new(
                network                             => $ipv6_subnet/prefix,             #Required,
when creating an IPv6 network
                auto_create_reversezone             => "true" | "false",                #Optional /
Default is undefined
                comment                             => $string| undef,                  #Optional /
Default is undefined
                ddns_domainname                     => $string| undef,                  #Optional /
Default is undefined
                ddns_enable_option_fqdn             => "true" | "false" ,               #Optional /
Default is "false"
                ddns_generate_hostname              => "true" | "false" | undef,        #Optional /
Default is undefined
                ddns_server_always_updates          => "true" | "false",                #Optional /
Default is "true"
                ddns_ttl                            => $num,                            #Optional /
Default is "0"
                disable                             => "true" | "false",                #Optional /
Default is "false"
                domain_name                         => $string,                         #Optional /
Default is undefined
                domain_name_servers                 => "$ipv6addr,$ipv6addr,...",       #Optional /
Default is undefined
                enable_ddns                         => "true" | "false" | undef,        #Optional /
Default is undefined
                enable_ifmap_publishing             => "true" | "false",                #Optional /
Default is undefined
                extattrs                            => { $string => $extattr, ... },    #Required if
'rir_organization' attribute is defined
                                                                                        Otherwise is
optional
                extensible_attributes               => { $string => $string | $num, $string => [
$string | $num, ... ], ... }, #Required if 'rir_organization' attribute is defined
#Otherwise is optional
                members                             => [ $Member1,$Member2,...],        #Optional /
Default is empty
                mgm_private                         => "true" | "false",                #Optional /
Default is "false"
                network_container                   => $ipv6_subnet/prefix,             #Optional /
Default is "/"
                network_view                        => $NetworkView,                    #Optional /
Default is "default" network view
                options                             => [$Option1,$Option2,...],         #Optional /
Default is empty list
                override_ddns_domainname            => "true" | "false",                #Optional /
Default value is "false"
                override_ddns_enable_option_fqdn    => "true" | "false",                #Optional /
Default value is "false"
                override_ddns_generate_hostname     => "true" | "false",                #Optional /
Default value is "false"
```

```
                override_ddns_ttl                   => "true" | "false",                #Optional /
Default value is "false"
                override_domain_name                => "true" | "false",                #Optional /
Default value is "false"
                override_domain_name_servers        => "true" | "false",                #Optional /
Default value is "false"
                override_mgm_private                => "true" | "false",                #Optional /
Default value is "false"
                override_enable_ddns                => "true" | "false",                #Optional /
Default value is "false"
                override_enable_ifmap_publishing    => "true" | "false",                #Optional /
Default value is "false"
                override_options                    => "true" | "false",                #Optional /
Default value is "false"
                override_preferred_lifetime         => "true" | "false",                #Optional /
Default value is "false"
                override_recycle_leases             => "true" | "false",                #Optional /
Default value is "false"
                override_update_dns_on_lease_renewal => "true" | "false",               #Optional /
Default value is "false"
                override_valid_lifetime             => "true" | "false",                #Optional /
Default value is "false"
                preferred_lifetime                  => $num,                            #Optional /
Default is "27000"
                recycle_leases                      => "true" | "false" | undef,        #Optional /
Default is undefined
                rir_organization                    => $rir_org,                        #Optional /
An Infoblox::Grid::RIR::Organization object
                rir_registration_action             => 'NONE' | 'CREATE' | 'MODIFY' | 'DELETE',  #
Optional / write-only
                rir_registration_status             => 'NOT_REGISTERED' | 'REGISTERED',  #Optional /
Default is 'NOT_REGISTERED'
                send_rir_request                    => 'true' | 'false',                #Optional /
write-only
                delete_reason                       => $string,                         #Optional /
write-only
                                                                                        #Required
only if 'rir_registration_action' is specified as 'DELETE'
                shared_network_name                 => $string| undef,                  #Optional /
Default is undefined
                template                            => $string| undef,                  #Optional /
Default is undefined
                update_dns_on_lease_renewal         => "true" | "false",                #Optional /
Default value is "false"
                valid_lifetime                      => $num,                            #Optional /
Default is "43200"
                zone_associations                   => [$zone1, $zone2, ... ] | undef,  #Optional /
Default is an empty list
                discovery_blackout_setting          => $blackout,                       #Optional
                port_control_blackout_setting       => $blackout,                       #Optional
                override_blackout_setting           => "true" | "false",                #Optional /
Default value is "false"
                same_port_control_discovery_blackout  => "true" | "false",              #Optional /
Default value is "false"
                subscribe_settings          => $subscribe_settings,                     #Optional /
Default is undefined
                override_subscribe_settings => 'true' | 'false',                        #Optional /
Default is 'false'
                discovery_engine_type       => 'NETMRI' | 'NETWORK_INSIGHT' | 'VDISCOVERY' | 'UNKNOWN' |
'NONE', #Optional / Default is 'NONE'
                discovered_bridge_domain    => $string,                                 #Optional /
Default is undefined
                discovered_tenant           => $string,                                 #Optional /
Default is undefined
              );
```

You cannot set both extattrs and extensible_attributes attributes at the same time.

SESSION METHODS

This section describes all the methods in an Infoblox::Session module that can apply to a DHCP IPv6 Network object.

Infoblox::Session->add( )

Use this method to add an IPv6 Network object to the Infoblox appliance. See Infoblox::Session->add() for parameters and return values.

Example

```
#Adding an IPv6 network.
#Construct an object
my $network = Infoblox::DHCP::IPv6Network->new(
```

```
        network => "2001::/64",
);

# Submit for adding an IPv6 network
my $response = $session->add( $network );
```

Infoblox::Session->get( )

Use this method to retrieve all the matching IPv6 Network objects from the Infoblox appliance. See Infoblox::Session->get() for parameters and return values.

'discover_now_status' will not be returned by default, use 'return_methods' to request it if needed.

Key References

```
Apply the following attributes to get a specific DHCP IPv6 network object:

  comment                  - Optional. A comment in string format.
  extattrs                 - Optional. A hash reference containing extensible attributes.
  extensible_attributes    - Optional. A hash reference containing extensible attributes.
  network                  - Optional. The IPv6 address and netmask of an IPv6 network in CIDR
format.
  network_view             - Optional. The network view in which the network is located. By default,
all network views are searched.
  discovered_bridge_domain - Optional. A bridge domain name in string format.
  discovered_tenant        - Optional. A tenant name in string format.
```

Examples

```
my  @result_array = $session->get(
     object       => "Infoblox::DHCP::IPv6Network" ,
     network      => "2001::/64",
     network_view => "default"
);

my  @result_array = $session->get(
     object               => "Infoblox::DHCP::IPv6Network" ,
     extensible_attributes => { 'Site' => 'Santa Clara' }
);

my  @result_array = $session->get(
     object  => "Infoblox::DHCP::IPv6Network" ,
     comment => "my-comment",
);

my @result_array = $session->get(
     object                  => "Infoblox::DHCP::IPv6Network" ,
     discovered_bridge_domain => "domain1",
);

my @result_array = $session->get(
     object           => "Infoblox::DHCP::IPv6Network" ,
     discovered_tenant => "tenant1",
);
```

Infoblox::Session->modify( )

Use this method to modify an IPv6 Network object in the Infoblox appliance. See Infoblox::Session->modify() for parameters and return values.

Example

```
#Use method to modify comment.
$object->comment("This is a new comment");
# Submit modification
my $response = $session->modify( $object );
```

Infoblox::Session->remove( )

Use this method to remove an object from the Infoblox appliance. See Infoblox::Session->remove() for parameters and return values.

To remove a specific object, first use `get()` or `search()` to retrieve the specific object, and then submit this object for removal.

Key References

```
remove_options - Optional. A hash reference containing the following attributes:

+-------------------------------------------------------------------------------------------+
| reason             - Optional. The reason for removing the network in string format with |
```

```
|                       a maximum of 256 bytes.                                                  |
|                       Used when 'rir_organization' attribute has been defined.                 |
| send_rir_request    - Optional. Boolean flag that indicates whether the appliance should       |
|                       send an API or email request for Regional Internet Registry (RIR).       |
|                       Used when 'rir_organization' attribute has been defined.                 |
+------------------------------------------------------------------------------------------------+
```

Example

```
 # Get the object under the same name
my  @result_array = $session->get(

       object        => "Infoblox::DHCP::IPv6Network" ,
       network       => "2001::/64",
       network_view  => "default"
         );

 # find the desired object from the retrieved list.
 my $desired_network = $result_array[0];

 # Submit for removal
 my $response = $session->remove(
     $desired_network,
     remove_options => {'reason' => 'Moved to B3', send_rir_request => 'true'} );
```

Infoblox::Session->search( )

Use this method to search for a DHCP IPv6 Network object in the Infoblox appliance. See Infoblox::Session->search() for parameters and return values.

'discover_now_status' will not be returned by default, use 'return_methods' to request it if needed.

Key References

```
 Apply the following attributes to search for a DHCP IPv6 Network object:

  comment                  - Optional. A comment in string format (regular expression).
  extattrs                 - Optional. A hash reference containing extensible attributes.
  extensible_attributes    - Optional. A hash reference containing extensible attributes.
  network                  - Optional. The IPv6 address and netmask of an IPv6 network in CIDR
format.
  network_view             - Optional. The network view in which the network is located. By default,
all network views are searched.
  discovered_bridge_domain - Optional. A bridge domain name in string format (regular expression).
  discovered_tenant        - Optional. A tenant name in string format (regular expression).
```

For more information about searching extensible attributes, see Infoblox::Grid::ExtensibleAttributeDef/Searching Extensible Attributes.

Examples

```
# search for all DHCP IPv6 Network objects
    my @retrieved_objs = $session->search(
      object       => "Infoblox::DHCP::IPv6Network",
      network      => "2001.*",
      network_view => "default"
    );

# search IPv6 network by comment
    my @retrieved_objs = $session->search(
      object    => "Infoblox::DHCP::IPv6Network",
      comment   => "my-comment",
    );

# search for all DHCP IPv6 Network objects defining "Santa Clara" for "Site" extensible attribute
    my @retrieved_objs = $session->search(
      object               => "Infoblox::DHCP::IPv6Network",
      extensible_attributes => { 'Site' => 'Santa Clara' }
    );

# search network by discovered bridge domain
 my @result_array = $session->search(
       object                   => "Infoblox::DHCP::IPv6Network" ,
       discovered_bridge_domain => "domain.*",
   );

 # search network by discovered tenant
 my @result_array = $session->search(
       object            => "Infoblox::DHCP::IPv6Network" ,
       discovered_tenant => "tenant.*",
   );
```

METHODS

This section describes all the methods that can be used to configure and retrieve the attribute values of a DHCP IPv6 Network object

auto_create_reversezone( )

Use this method to set the auto_create_reversezone flag of a DHCP IPv6 Network object.

Include the specified parameter to set the attribute value. This is a write-only attribute and cannot be retrieved after being set.

Parameter

Specify "true" to set the auto_create_reversezone flag or "false" to deactivate/unset it. The default value for this field is "false".

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

Example

```
# Modify auto_create_reversezone
$network_container->auto_create_reversezone("true");
```

comment( )

Use this method to set or retrieve a descriptive comment of an IPv6 Network object.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

Enter a descriptive comment for the network in string format with a maximum of 256 bytes.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
# Get comment
my $comment = $network->comment();
# Modify comment
$network->comment("add an IPv6 network");
```

cloud_info( )

Use this method to set or retrieve cloud API related information for the Infoblox::DHCP::IPv6Network object.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

The valid value is a Infoblox::Grid::CloudAPI::Info object.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
# Get cloud_info
my $cloud_info = $network_view->cloud_info();
# Modify cloud_info
$network_view->cloud_info($cloud_info_object);
```

ddns_domainname( )

Use this method to set or retrieve the ddns_domainname value of a DHCP IPv6 Network object.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Setting this method to a defined value implicitly sets the override_ddns_domainname method to "true". Setting the parameter to undefined causes the appliance to use the grid default and automatically resets the override_ddns_domainname attribute to "false".

Note that when ddns_domainname is set to a defined value and override_ddns_domainname is set to "false", the last operation takes precedence. Thus the sequence $object->ddns_domainname("testdomain"); $object->override_ddns_domainname("false"); will set override_ddns_domainname to "false", and the sequence $object->override_ddns_domainname("false"); $object->ddns_domainname("testdomain"); will result in override_ddns_domainname="true".

Parameter

The dynamic DNS domain name in string format.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get ddns_domainname
my $ddns_domainname = $network->ddns_domainname();
#Modify ddns_domainname
$network->ddns_domainname("test_domain.com");
```

ddns_enable_option_fqdn( )

Use this method to set or retrieve the ddns_enable_option_fqdn flag of a DHCP IPv6 Network object. This method controls whether the FQDN option sent by the client is to be used, or if the server can automatically generate the FQDN. This setting overrides the upper-level settings.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Setting this method to a defined value implicitly sets the override_ddns_enable_option_fqdn method to "true". Setting the parameter to undefined causes the appliance to use the upper-level setting and automatically resets the override_ddns_enable_option_fqdn attribute to "false".

Note that when ddns_enable_option_fqdn is set to a defined value and override_ddns_enable_option_fqdn is set to "false", the last operation takes precedence. Thus the sequence $object->ddns_enable_option_fqdn("true"); $object->override_ddns_enable_option_fqdn("false"); will set override_ddns_enable_option_fqdn to "false", and the sequence $object->override_ddns_enable_option_fqdn("false"); $object->ddns_enable_option_fqdn("true"); will result in override_ddns_enable_option_fqdn="true".

Parameter

Specify "true" to generate hostname or "false" to deactivate/unset it.The default value for this field is undefined, which indicates that this attribute inherit the member level setting.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
# Get ddns_enable_option_fqdn
my $ddns_enable_option_fqdn = $network->ddns_enable_option_fqdn()
# Modify ddns_enable_option_fqdn
$network->ddns_enable_option_fqdn("true");
#Un-override
$network->ddns_enable_option_fqdn(undef);
```

ddns_generate_hostname( )

Use this method to set or retrieve the ddns_generate_hostname flag of a DHCP IPv6 Network object. Enable this method to allow only the DHCP server for generating hostname. This setting overrides the upper-level settings.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Setting this method to a defined value implicitly sets the override_ddns_generate_hostname method to "true". Setting the parameter to undefined causes the appliance to use the upper-level setting and automatically resets the override_ddns_generate_hostname attribute to "false".

Note that when ddns_generate_hostname is set to a defined value and override_ddns_generate_hostname is set to "false", the last operation takes precedence. Thus the sequence $object->ddns_generate_hostname("true"); $object->override_ddns_generate_hostname("false"); will set override_ddns_generate_hostname to "false", and the sequence $object->override_ddns_generate_hostname("false"); $object->ddns_generate_hostname("true"); will result in override_ddns_generate_hostname="true".

Parameter

Specify "true" to generate a hostname or "false" to deactivate/unset it. The default value for this field is undefined, which indicates that this attribute inherits the member-level setting.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
# Get ddns_generate_hostname
my $ddns_generate_hostname = $network->ddns_generate_hostname()
# Modify ddns_generate_hostname
$network->ddns_generate_hostname("true");
#Un-override
$network->ddns_generate_hostname(undef);
```

ddns_server_always_updates( )

Use this method to set or retrieve the ddns_server_always_updates flag of a DHCP IPv6 Network object. Enable this method to allow only the DHCP server to update DNS, regardless of the requests from the DHCP clients.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

Specify "true" to set the flag that enables the DHCP server to always updates DNS or "false" to deactivate/unset it.The default value for this field is "true".

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
# Get ddns_server_always_updates
my $ddns_server_always_updates = $network->ddns_server_always_updates()
# Modify ddns_server_always_updates
$network->ddns_server_always_updates("true");
```

ddns_ttl( )

Use this method to set or retrieve the DNS update Time to Live (TTL) value of a DHCP IPv6 Network object.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Setting this method to a defined value implicitly sets the override_ddns_ttl method to "true". Setting the parameter to undefined causes the appliance to use the grid default and automatically resets the override_ddns_ttl attribute to "false".

Note that when ddns_ttl is set to a defined value and override_ddns_ttl is set to "false", the last operation takes precedence. Thus the sequence $object->ddns_ttl(12); $object->override_ddns_ttl("false"); will set override_ddns_ttl to "false", and the sequence $object->override_ddns_ttl("false"); $object->ddns_ttl(12); will result in override_ddns_ttl="true".

Parameter

A 32-bit integer (range from 0 to 4294967295) that represents the duration, in seconds, that the update is cached. Zero indicates that the update is not cached. The default value is 0.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get DNS Update ttl
my $dns_ttl = $network->ddns_ttl();
#Modify DNS Update ttl
$network->ddns_ttl(1200);
```

disable( )

Use this method to set or retrieve the disable flag of a DHCP IPv6 Network object.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

Specify "true" to set the disable flag or "false" to deactivate/unset it. The default value for this field is "false".

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
# Get disable
my $disable = $network->disable()
# Modify disable
$network->disable("true");
```

discovered_bridge_domain( )

Use this method to set or retrieve the discovered bridge domain name of a DHCP IPv6 Network object.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

The desired bridge domain name in a string format.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get bridge domain
my $bridge_domain = $network->discovered_bridge_domain();
#Modify bridge domain
$network->discovered_bridge_domain("Domain");
```

discovered_tenant( )

Use this method to set or retrieve the discovered tenant name of a DHCP IPv6 Network object.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

The desired tenant name in a string format.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get tenant
my $tenant = $network->discovered_tenant();
#Modify tenant
$network->discovered_tenant("Tenant");
```

discovery_basic_poll_setting( )

Use this method to set or retrieve the discovery_basic_poll_setting value of a DHCP IPv6 Network object.

Include the specified parameter to set the attribute value.

Parameter

The valid value is object of the Infoblox::Grid::Discovery::BasicPollSettings manpage.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get discovery_basic_poll_setting
my $discovery_basic_poll_setting = $network->discovery_basic_poll_setting();
#Modify discovery_basic_poll_setting
$network->discovery_basic_poll_setting($poll_setting);
```

discovery_member( )

Use this method to set or retrieve the discovery_member value of a DHCP IPv6 Network object.

Parameter

The discovery member name in string format.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get discovery_member
my $discovery_member = $network->discovery_member();
#Modify discovery_member
$network->discovery_member("test.localdomain");
```

discover_now_status( )

Use this method to retrieve the current discovery status. This attribute is read-only.

'discover_now_status' will not be returned by default, use 'return_methods' to request it if needed.

Parameter

None.

Returns

The method returns the attribute value that can be 'COMPLETE', 'FAILED', 'NONE', 'PENDING' or 'RUNNING'.

Example

```
#Get discover_now_status
my $discover_now_status = $network->discover_now_status();
```

discovery_blackout_setting( )

Use this method to set or retrieve the discovery blackout setting.

Setting this method to a defined value implicitly sets the override_blackout_setting method to "true". If port_control_blackout_setting is undefined, setting the parameter to undefined causes the appliance to use the grid default and automatically resets the override_blackout_setting attribute to "false".

Note that when discovery_blackout_setting is set to a defined value and override_blackout_setting is set to "false", the last operation takes precedence. Thus the sequence $network->discovery_blackout_setting($blackout); $network->override_blackout_setting("false"); will set override_blackout_setting to "false", and the sequence $network->override_blackout_setting("false"); $network->discovery_blackout_setting($blackout); will result in override_blackout_setting="true".

Parameter

Valid value is an Infoblox::Grid::Discovery::Properties::BlackoutSetting object.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get discovery_blackout_setting
my $discovery_blackout_setting = $network->discovery_blackout_setting();
#Modify discovery_blackout_setting
$network->discovery_blackout_setting($blackout);
```

discovery_engine_type( )

Use this method to retrieve the DHCP IPv6 Network discovery engine type. This attribute is read-only.

Parameter

None.

Returns

The method returns the attribute value that can be 'NETMRI', 'NETWORK_INSIGHT', 'VDISCOVERY', 'UNKNOWN' or 'NONE'.

Example

```
#Get discovery_engine_type
my $discovery_engine_type = $network->discovery_engine_type();
```

domain_name( )

Use this method to set or retrieve the domain_name value of a DHCP IPv6 Network object.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Setting this method to a defined value implicitly sets the override_domain_name method to "true". Setting the parameter to undefined causes the appliance to use the grid default and automatically resets the override_domain_name attribute to "false".

Note that when domain_name is set to a defined value and override_domain_name is set to "false", the last operation takes precedence. Thus the sequence $object->domain_name("testdomain"); $object->override_domain_name("false"); will set override_domain_name to "false", and the sequence $object->override_domain_name("false"); $object->domain_name("testdomain"); will result in override_domain_name="true".

Parameter

The name of the client's domain for resolving names in DNS in string format.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get domain_name
my $domain_name = $network->domain_name();
#Modify domain_name
$network->domain_name("test_domain.com");
```

domain_name_servers( )

Use this method to set or retrieve the domain_name_servers value of a DHCP IPv6 Network object.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Setting this method to a defined value implicitly sets the override_domain_name_servers method to "true". Setting the parameter to undefined causes the appliance to use the grid default and automatically resets the override_domain_name_servers attribute to "false".

Note that when domain_name_servers is set to a defined value and override_domain_name_servers is set to "false", the last operation takes precedence. Thus the sequence $object->domain_name_servers("2001::1,2001::2"); $object->override_domain_name_servers("false"); will set override_domain_name_servers to "false", and the sequence $object->override_domain_name_servers("false"); $object->domain_name_servers("2001::1,2001::2"); will result in override_domain_name_servers="true".

Parameter

The domain name server addresses in IPv6 address format.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get domain_name_servers
my $domain_name_servers = $network->domain_name_servers();
#Modify domain_name_servers
```

```
$network->domain_name_servers("2001::1,2001::2");
```

enable_ddns( )

Use this method to set or retrieve the dynamic DNS updates flag of a DHCP IPv6 Network object. The DHCP server can send DDNS updates to DNS servers in the same grid and to external DNS servers. This setting overrides the member level settings.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Setting this method to a defined value implicitly sets the override_enable_ddns method to "true". Setting the parameter to undefined causes the appliance to use the upper-level default and automatically resets the override_enable_ddns attribute to "false".

Note that when enable_ddns is set to a defined value and override_enable_ddns is set to "false", the last operation takes precedence. Thus the sequence $object->enable_ddns("true"); $object->override_enable_ddns("false"); will set override_enable_ddns to "false", and the sequence $object->override_enable_ddns("false"); $object->enable_ddns("true"); will result in override_enable_ddns="true".

Parameter

Specify "true" to set the dynamic DNS updates flag or "false" to deactivate/unset it. The default value for this field is undefined, which indicates that this attribute inherit the member level setting.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
# Get enable_ddns
my $enable_ddns = $network->enable_ddns()
# Modify enable_ddns
$network->enable_ddns("true");
# Un-override enable_ddns
$network->enable_ddns(undef);
```

enable_discovery( )

Use this method to set or retrieve the enable_discovery value of a DHCP IPv6 Network object. This setting overrides the member level settings.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Setting this method to a defined value implicitly sets the override_enable_discovery method to "true". Setting the parameter to undefined causes the appliance to use the default from the upper level and automatically resets the override_enable_discovery attribute to "false".

Enable this flag if you want to set discovery for this object.

Parameter

Specify "true" to set the enable_discovery flag or "false" to deactivate/unset it. The default value is undefined.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get enable_discovery
my $enable_discovery = $network->enable_discovery();
#Modify enable_discovery
$network->enable_discovery("false");
```

enable_ifmap_publishing( )

Use this method to set or retrieve the ifmap publishing option of a DHCP IPv6 Network object. This setting overrides the member level settings.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

Specify "true" to set ifmap publishing option or "false" to deactivate/unset it. The default value is 'false', which indicates that this attribute inherits the member level setting.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
# Get enable_ifmap_publishing
my $enable_ifmap_publishing = $network->enable_ifmap_publishing()
# Modify enable_ifmap_publishing
$network->enable_ifmap_publishing("true");
# Un-override enable_ifmap_publishing
$network->enable_ifmap_publishing(undef);
```

enable_immediate_discovery( )

Use this method to set or retrieve enable_immediate_discovery value of a DHCP IPv6 Network object.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

Specify "true" to set the enable_immediate_discovery flag or "false" to deactivate/unset it. The default value is undefined.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get enable_immediate_discovery
my $enable_immediate_discovery = $network->enable_immediate_discovery();
#Modify enable_immediate_discovery
$network->enable_immediate_discovery("false");
```

endpoint_sources( )

Use this method to retrieve the list of endpoints that provide object data. This is a read-only attribute.

Omit the parameter to retrieve the attribute value.

Parameter

None

Returns

The valid return value is an array of Infoblox::CiscoISE::Endpoint objects.

Example

```
#Get endpoint_sources value
my $endpoint_sources = $object->endpoint_sources();
```

extattrs( )

Use this method to set or retrieve the extensible attributes associated with an IPv6 Network object.

Parameter

Valid value is a hash reference containing the names of extensible attributes and their associated values (Infoblox::Grid::Extattr objects).

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get extattrs
my $ref_extattrs = $network->extattrs();
#Modify extattrs
$network->extattrs({ 'Site' => $extattr1, 'Administrator' => $extattr2 });
```

extensible_attributes( )

Use this method to set or retrieve the extensible attributes associated with an IPv6 Network object.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

For valid values for extensible attributes, see Infoblox::Grid::ExtensibleAttributeDef/Extensible Attribute Values.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get extensible attributes
my $ref_extensible_attributes = $network->extensible_attributes();
#Modify extensible attributes
$network->extensible_attributes({ 'Site' => 'Santa Clara', 'Administrator' => [ 'Peter', 'Tom' ] });
```

members( )

Use this method to set or retrieve the member that serves DHCP for an IPv6 Network object.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

The valid value is an array reference that contains Infoblox::DHCP::Member objects.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
my $member1 = Infoblox::DHCP::Member->new(
    name     => "infoblox.localdomain",
    ipv4addr => "192.168.1.2",
);

#Create the Network object with this member
my $network = Infoblox::DHCP::Network->new(
    network => "2001::/64",
    members => [$member1],
);
```

mgm_private( )

Use this method to set or retrieve the flag that controls whether the object should be synced to the MGM or not.

Setting this method to a defined value implicitly sets the override_blackout_setting method to "true". If discovery_blackout_setting is undefined, setting the parameter to undefined causes the appliance to use the grid default and automatically resets the override_blackout_setting attribute to "false".

Note that when mgm_private is set to a defined value and override_mgm_private is set to "false", the last operation takes precedence. Thus the sequence $network->mgm_private("true"); $network->override_mgm_private("false"); will set override_mgm_private to "false", and the sequence $network->override_mgm_private("false"); $network->mgm_private("true"); will result in override_mgm_private="true".

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

Specify 'true' to enable synchronization of the managed Grid with the MGM or 'false' to forbid it. The default value is 'false'.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Note that this member is loaded on on-demand basis, so to retrieve the value the object must have been retrieved from the session first.

Example

```
#Get mgm_private value
my $value = $object->mgm_private();

#Modify mgm_private value
$object->mgm_private('true');
```

mgm_private_overridable( )

Use this method to retrieve the flag that indicates whether the overriding of 'mgm_private' flag is allowed or not. This is a read-only attribute.

Parameter

None

Returns

The method returns the attribute value.

Note that this member is loaded on on-demand basis, so to retrieve the value the object must have been retrieved from the session first.

Example

```
#Get mgm_private_overridable value
my $value = $object->mgm_private_overridable();
```

ms_ad_user_data( )

Use this method to retrieve Microsoft Active Directory users related information. This is a read-only attribute.

Parameter

None

Returns

The valid return value is an Infoblox::Grid::MSServer::AdUser::Data object.

Example

```
#Get ms_ad_user_data
my $ms_ad_user_data = $v6network->ms_ad_user_data();
```

network( )

Use this method to retrieve the network attribute of a DHCP IPv6 Network object.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

Specify the address and netmask of the network in CIDR format (example: 2001::/64).

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get network
my $network = $network->network();
```

network_container( )

Use this method to set or retrieve the network container of a DHCP IPv6 Network object.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

Specify the address and netmask of the network in CIDR format (example: 2001::/64).

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get network container
my $network_container = $network->network_container();
#Modify network container
$network->network_container("2001::/64");
```

network_view( )

Use this method to set or retrieve the network view of the IPv6 Network object.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

The default value is the "default" network view, which means the network is in the default network view.

Parameter

Valid value is a defined Infoblox::DHCP::View object. The default network view value is the system-defined default network view object.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get network view
my $nview = $network->network_view();
#Modify network view, with an Infoblox::DHCP::View object
$network->network_view($nview);
```

next_available_ip( )

Use this method to retrieve the next available IPv6 address(es) in the network. Note that to use this method, the object MUST be retrieved through a get or search call, or created with an add call.

Parameter

The method accepts the following optional parameters:

```
  requested_num - The number of IP addresses requested. It must be a number between 1 and 20,
inclusive.
  excluded - A reference to an array of addresses to be excluded from the results.
```

Returns

This method returns the next available IPv6 address(es) in the network. If requested_num is not specified, a string with the next available address will be returned; otherwise, the method will return a reference to an array of addresses.

Within a DHCP IPv6Network object, the next available IP address(es) are the first addresses in this network that match all of the following rules:

```
   * Are not part of the passed 'excluded' parameter (if specified).
   * There is no AAAA, PTR, fixed address or host address record with this IP address
   * There is no bulk host record with this IP address
   * There is no unmanaged record with this IP address
   * The IP address is not the network or broadcast address of this network
   * The IP address is not in any IPv6 DHCP range within this network (including any exclusion range)
```

Example

```
# Retrieve a particular network
my @retrieved_objs = $session->get(
                          object   => "Infoblox::DHCP::IPv6Network",
                          network  => "10:9::/64"
);

$network = $retrieved_objs[0];

# Print the next available IPv6 address in the network
print $network->next_available_ip() . "\n";

# This will retrieve the next 10 available addresses and print the first one.
my $addressesref = $network->next_available_ip(requested_num => 10);
```

```
print @$addressesref[0] . "\n";

# This will retrieve the next 10 available addresses and exclude two addresses.
my $addressesref = $network->next_available_ip(requested_num => 10, excluded => ["10:9::2",
"10:9::3"]);
```

next_available_network( )

Use this method to retrieve the next available IPv6 network(s). Note that to use this method, the object MUST be retrieved through a get or search call, or it must have been created with an add call.

Parameter

The method accepts the following parameters:

```
  cidr - (required) The CIDR of the requested networks. It must be a number between 3 and 128,
inclusive.
  requested_num - (optional) The number of networks requested.  It must be a number between 1 and 20,
inclusive.
  excluded - (optional) A reference to an array of networks to be excluded from the results.
```

Returns

This method returns the next available IPv6 `network(s)` . If requested_num is not specified, a string with the next available network will be returned; otherwise, the method will return a reference to an array of networks.

A network cannot be returned as the next available network if it cannot be inserted; such as when any of the following conditions exist:

- It's parent network has a DHCP configuration assigned and therefore cannot be converted to a network container.

- It overlaps with an existing network or network container, regardless of whether the existing network has a greater or smaller CIDR than the requested CIDR.

- Its network address or broadcast address is used by a fixed address or a host configured for DHCP.

Example

```
# Retrieve a particular network
my @retrieved_objs = $session->get(
                          object   => "Infoblox::DHCP::IPv6Network",
                          network  => "2001::/64"
);

$network = $retrieved_objs[0];

# Print the next available IPv6 network in the network
print $network->next_available_network(cidr => 68) . "\n";

# This will retrieve the next 10 available networks and print the first one.
my $addressesref = $network->next_available_network(requested_num => 10, cidr => 68);
print @$addressesref[0] . "\n";

# This will retrieve the next 10 available networks and exclude two.
my $addressesref = $network->next_available_network(requested_num => 10, cidr => 72, excluded =>
['2001::100:0:0:0/72','2001::300:0:0:0/72']);
```

options( )

Use this method to set or retrieve the options value of a member DHCP server. DHCP options provide network configuration settings and various services available on the network for DHCP.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Setting this method to a defined value implicitly sets the override_options method to "true". Setting the parameter to undefined causes the appliance to use the upper-level default and automatically resets the override_options attribute to "false".

Note that when the options method is set to a defined value and override_options is set to "false", the last operation takes precedence. Thus the sequence $object->options([$opt]); $object->override_options("false"); will set override_options to "false", and the sequence $object->override_options("false"); $object->options([$opt]); will result in override_options="true".

Parameter

The valid value is an array reference that contains Infoblox::DHCP::Option objects.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get options
my $options = $network->options();
#Modify options
my $option1 = Infoblox::DHCP::Option->new(
    name => "dhcp6.info-refresh-time",
    value => 60
);

$network->options([$option1]);
```

override_ddns_domainname( )

The override_ddns_domainname attribute controls whether the ddns_domainname value in the DHCP IPv6 Network is used, instead of the grid default.

The override_ddns_domainname attribute can be specified explicitly. It is also set implicitly when ddns_domainname is set to a defined value.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

Set the parameter to "true" to override the grid-level setting for ddns_domainname. Set the parameter to "false" to inherit the grid-level setting for ddns_domainname.

The default value of this parameter is "false".

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Getting override_ddns_domainname
my $override_ddns_domainname=$network->override_ddns_domainname( );
#Modifying override_ddns_domainname
$network->override_ddns_domainname("true");
```

override_ddns_enable_option_fqdn( )

The override_ddns_enable_option_fqdn attribute controls whether the ddns_enable_option_fqdn value in the DHCP IPv6 Network is used, instead of the grid default.

The override_ddns_enable_option_fqdn attribute can be specified explicitly. It is also set implicitly when ddns_domainname is set to a defined value.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

Set the parameter to "true" to override the grid-level setting for ddns_enable_option_fqdn. Set the parameter to "false" to inherit the grid-level setting for ddns_enable_option_fqdn.

The default value of this parameter is "false".

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Getting override_ddns_enable_option_fqdn
my $override_ddns_enable_option_fqdn=$network->override_ddns_enable_option_fqdn( );
#Modifying override_ddns_enable_option_fqdn
$network->override_ddns_enable_option_fqdn("true");
```

override_ddns_generate_hostname( )

The override_ddns_generate_hostname attribute controls whether the ddns_generate_hostname value in the DHCP IPv6 Network is used, instead of the grid default.

The override_ddns_generate_hostname attribute can be specified explicitly. It is also set implicitly when ddns_generate_hostname is set to a defined value.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

Set the parameter to "true" to override the grid-level setting for ddns_generate_hostname. Set the parameter to "false" to inherit the grid-level setting for ddns_generate_hostname.

The default value of this parameter is "false".

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Getting override_ddns_generate_hostname
my $override_ddns_generate_hostname=$network->override_ddns_generate_hostname( );
#Modifying override_ddns_generate_hostname
$network->override_ddns_generate_hostname("true");
```

override_ddns_ttl( )

The override_ddns_ttl attribute controls whether the ddns_ttl value in the DHCP IPv6 Network is used, instead of the grid default.

The override_ddns_ttl attribute can be specified explicitly. It is also set implicitly when ddns_ttl is set to a defined value.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

Set the parameter to "true" to override the grid-level setting for ddns_ttl. Set the parameter to "false" to inherit the grid-level setting for ddns_ttl.

The default value of this parameter is "false".

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Getting override_ddns_ttl
my $override_ddns_ttl=$network->override_ddns_ttl( );
#Modifying override_ddns_ttl
$network->override_ddns_ttl("true");
```

override_domain_name( )

The override_domain_name attribute controls whether the domain_name value in the DHCP IPv6 Network is used, instead of the grid default.

The override_domain_name attribute can be specified explicitly. It is also set implicitly when domain_name is set to a defined value.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

Set the parameter to "true" to override the grid-level setting for domain_name. Set the parameter to "false" to inherit the grid-level setting for domain_name.

The default value of this parameter is "false".

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Getting override_domain_name
my $override_domain_name=$network->override_domain_name( );
```

```
#Modifying override_domain_name
$network->override_domain_name("true");
```

override_domain_name_servers( )

The override_domain_name_servers attribute controls whether the domain_name_servers value in the DHCP IPv6 Network is used, instead of the grid default.

The override_domain_name_servers attribute can be specified explicitly. It is also set implicitly when domain_name_servers is set to a defined value.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

Set the parameter to "true" to override the grid-level setting for domain_name_servers. Set the parameter to "false" to inherit the grid-level setting for domain_name_servers.

The default value of this parameter is "false".

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Getting override_domain_name_servers
my $override_domain_name_servers=$network->override_domain_name_servers( );
#Modifying override_domain_name_servers
$network->override_domain_name_servers("true");
```

override_blackout_setting( )

The override_blackout_setting attribute controls whether the discovery_blackout_setting and port_control_blackout_setting values in the object are used, instead of the grid default.

The override_blackout_setting attribute can be specified explicitly. It is also set implicitly when discovery_blackout_setting or port_control_blackout_setting is set to a defined value.

Parameter

Specify "true" to set the override_blackout_setting flag or "false" to deactivate/unset it. The default value is "false".

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
# Get attribute value
my $value = $network->override_blackout_setting();
# Modify attribute value
$network->override_blackout_setting('true');
```

override_enable_ddns( )

The override_enable_ddns attribute controls whether the enable_ddns value in the DHCP IPv6 Network is used, instead of the grid default.

The override_enable_ddns attribute can be specified explicitly. It is also set implicitly when enable_ddns is set to a defined value.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

Set the parameter to "true" to override the grid-level setting for enable_ddns. Set the parameter to "false" to inherit the grid-level setting for enable_ddns.

The default value of this parameter is "false".

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Getting override_enable_ddns
my $override_enable_ddns=$network->override_enable_ddns( );
#Modifying override_enable_ddns
$network->override_enable_ddns("true");
```

override_enable_ifmap_publishing( )

The override_enable_ifmap_publishing controls whether the enable_ifmap_publishing in the DHCP IPv6 Network is used, instead of the grid default.

The override_enable_ifmap_publishing can be specified explicitly. It is also set implicitly when enable_ifmap_publishing is set to a defined value.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

Set the parameter to "true" to override the grid-level setting for enable_ifmap_publishing. Set the parameter to "false" to inherit the grid-level setting for enable_ifmap_publishing.

The default value of this parameter is "false".

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Getting override_enable_ifmap_publishing
my $override_enable_ifmap_publishing=$network->override_enable_ifmap_publishing( );
#Modifying override_enable_ifmap_publishing
$network->override_enable_ifmap_publishing("true");
```

override_options( )

The override_options attribute controls whether the options value in the DHCP IPv6 Network is used, instead of the grid default.

The override_options attribute can be specified explicitly. It is also set implicitly when options is set to a defined value.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

Set the parameter to "true" to override the grid-level setting for options. Set the parameter to "false" to inherit the grid-level setting for options.

The default value of this parameter is "false".

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Getting override_options
my $override_options=$network->override_options( );
#Modifying override_options
$network->override_options("true");
```

override_preferred_lifetime( )

The override_preferred_lifetime attribute controls whether the preferred_lifetime value in the DHCP IPv6 Network is used, instead of the grid default.

The override_preferred_lifetime attribute can be specified explicitly. It is also set implicitly when preferred_lifetime is set to a defined value.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

Set the parameter to "true" to override the grid-level setting for options. Set the parameter to "false" to inherit the grid-level setting for options.

The default value of this parameter is "false".

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Getting override_preferred_lifetime
my $override_preferred_lifetime=$network->override_preferred_lifetime( );
#Modifying override_preferred_lifetime
$network->override_preferred_lifetime("true");
```

override_recycle_leases( )

The override_recycle_leases attribute controls whether the recycle_leases value in the DHCP IPv6 Network is used, instead of the grid default.

The override_recycle_leases attribute can be specified explicitly. It is also set implicitly when recycle_leases is set to a defined value.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

Set the parameter to "true" to override the grid-level setting for options. Set the parameter to "false" to inherit the grid-level setting for options.

The default value of this parameter is "false".

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Getting override_recycle_leases
my $override_recycle_leases=$network->override_recycle_leases( );
#Modifying override_recycle_leases
$network->override_recycle_leases("true");
```

override_mgm_private( )

The override_mgm_private attribute controls whether the mgm_private value in the object is used, instead of the parent-level default.

The override_mgm_private attribute can be specified explicitly. It is also set implicitly when mgm_private is set to a defined value.

Note that setting override_mgm_private to 'false' sets the original 'mgm_private' value to 'false'.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

Set the parameter to 'true' to override the parent-level setting for mgm_private. Set the parameter to 'false' to inherit the parent-level setting for mgm_private.

The default value of this parameter is 'false'.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Note that this member is loaded on on-demand basis, so to retrieve the value the object must have been retrieved from the session first.

Example

```
#Get override_mgm_private value
my $value = $object->override_mgm_private( );

#Modify override_mgm_private value
$object->override_mgm_private('true');
```

override_subscribe_settings( )

The override_subscribe_settings attribute controls whether the subscribe_settings method values of the member are used, instead of the Grid default.

The override_subscribe_settings attribute can be specified explicitly. It is also set implicitly when subscribe_settings is set to a defined value.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

Set the parameter to "true" to override the Grid-level setting for subscribe_settings. Set the parameter to "false" to inherit the Grid-level setting for subscribe_settings.

The default value of this parameter is "false".

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get override_subscribe_settings value
my $override_subscribe_settings = $object->override_subscribe_settings( );
#Modify override_subscribe_settings value
$object->override_subscribe_settings("true");
```

override_update_dns_on_lease_renewal( )

The override_update_dns_on_lease_renewal attribute controls whether the update_dns_on_lease_renewal value in the DHCP IPv6 Network is used, instead of the grid default.

The override_update_dns_on_lease_renewal attribute can be specified explicitly. It is also set implicitly when update_dns_on_lease_renewal is set to a defined value.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

Set the parameter to "true" to override the grid-level setting for update_dns_on_lease_renewal. Set the parameter to "false" to inherit the grid-level setting for update_dns_on_lease_renewal.

The default value of this parameter is "false".

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Getting override_update_dns_on_lease_renewal
my $override_update_dns_on_lease_renewal=$network->override_update_dns_on_lease_renewal( );
#Modifying override_update_dns_on_lease_renewal
$network->override_update_dns_on_lease_renewal("true");
```

override_valid_lifetime( )

The override_valid_lifetime attribute controls whether the valid_lifetime value in the DHCP IPv6 network is used, instead of the grid default.

The override_valid_lifetime attribute can be specified explicitly. It is also set implicitly when valid_lifetime is set to a defined value.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

Set the parameter to "true" to override the grid-level setting for options. Set the parameter to "false" to inherit the grid-level setting for options.

The default value of this parameter is "false".

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Getting override_valid_lifetime
my $override_valid_lifetime=$network->override_valid_lifetime( );
#Modifying override_valid_lifetime
$network->override_valid_lifetime("true");
```

preferred_lifetime ( )

Use this method to set or retrieve the preferred lifetime value of a DHCP IPv6 Network object.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Setting this method to a defined value implicitly sets the override_preferred_lifetime method to "true". Setting the parameter to undefined causes the appliance to use the grid default and automatically resets the override_preferred_lifetime attribute to "false".

Note that when preferred_lifetime is set to a defined value and override_preferred_lifetime is set to "false", the last operation takes precedence. Thus the sequence $object->preferred_lifetime(691200); $object->override_preferred_lifetime("false"); will set override_preferred_lifetime to "false", and the sequence $object->override_preferred_lifetime("false"); $object->preferred_lifetime(691200); will result in override_preferred_lifetime="true".

Parameter

A 32-bit integer (valid range from 1 to 4294967295) that represents the lifetime, in seconds. The default value is 27000.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get Preferred lifetime
my $preferred_lifetime = $network->preferred_lifetime();
#Modify Preferred lifetime
$network->preferred_lifetime(864000);
```

port_control_blackout_setting

Use this method to set or retrieve the port control blackout setting.

Setting this method to a defined value implicitly sets the override_blackout_setting method to "true". If discovery_blackout_setting is undefined, setting the parameter to undefined causes the appliance to use the grid default and automatically resets the override_blackout_setting attribute to "false".

Note that when port_control_blackout_setting is set to a defined value and override_blackout_setting is set to "false", the last operation takes precedence. Thus the sequence $network->port_control_blackout_setting($blackout); $network->override_blackout_setting("false"); will set override_blackout_setting to "false", and the sequence $network->override_blackout_setting("false"); $network->port_control_blackout_setting($blackout); will result in override_blackout_setting="true".

Parameter

Valid value is an Infoblox::Grid::Discovery::Properties::BlackoutSetting object.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get port_control_blackout_setting
my $port_control_blackout_setting = $network->port_control_blackout_setting();
#Modify port_control_blackout_setting
$network->port_control_blackout_setting($blackout);
```

recycle_leases( )

Use this method to set or retrieve the recycle_leases flag at the network level.

If the flag is enabled, the leases are kept in the recycle bin until one week after expiration. When the flag is disabled, the leases are irrecoverably deleted.

Setting this method to a defined value implicitly sets the override_recycle_leases method to "true". Setting the parameter to undefined causes the appliance to use the grid default and automatically resets the override_recycle_leases attribute to "false".

Note that when recycle_leases is set to a defined value and override_recycle_leases is set to "false", the last operation takes precedence. Thus the sequence $object->recycle_leases("true"); $object->override_recycle_leases("false"); will set override_recycle_leases to "false", and the sequence $objct->override_recycle_leases("false"); $object->recycle_leases("true"); will result in override_recycle_leases="true".

Parameter

Specify "true" to enable the recycle_leases flag or "false" to disable it. The default value is undefined.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get recycle_leases
my $recycle_leases = $network->recycle_leases();
#Modify recycle_leases
$network->recycle_leases("false");
```

restart_if_needed( )

Use this method to set the flag that indicates whether DHCP service should be restarted on a member that is serving DHCP for the changed object referenced in a cloud API request.

Parameter

Specify true to restart the DHCP service.

Returns

None

Example

```
# Modify
$network->restart_if_needed('true');
```

rir( )

Use this method to retrieve the name of the Regional Internet Registry (RIR) that allocated this address space. This is a read-only attribute.

Parameter

None

Returns

The method returns the attribute value that can be 'RIPE' or 'NONE'.

Example

```
#Get rir
my $rir = $network->rir();
```

rir_organization( )

Use this method to set or retrieve the Regional Internet Registry (RIR) organization object.

Parameter

An Infoblox::Grid::RIR::Organization object.

You must enable support for RIR/SWIP at the Grid level before you can set attribute values. See Infoblox::Grid->enable_rir_swip() method.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get rir_organization
my $rir_organization = $network->rir_organization();
#Modify rir_organization
$network->rir_organization($rir_org);
```

rir_registration_action( )

Use this method to set the registration action of Regional Internet Registry (RIR). This is a write-only attribute.

Parameter

Valid values are 'NONE', 'CREATE', 'MODIFY' or 'DELETE'.

Returns

The method returns true when the modification succeeds, and returns false when the operation fails.

Example

```
#Set rir_registration_action
$network->rir_registration_action('CREATE');
```

rir_registration_status( )

Use this method to set or retrieve the registration status of Regional Internet Registry (RIR).

Parameter

The registration status of Regional Internet Registry (RIR), which can be 'REGISTERED' or 'NOT_REGISTERED'.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get rir_registration_status
my $rir_registration_status = $network->rir_registration_status();
#Modify rir_registration_status
$network->rir_registration_status('NOT_REGISTERED');
```

last_rir_registration_update_sent( )

Use this method to retrieve the last registration update timestamp of Regional Internet Registry (RIR). This is a read-only attribute.

Parameter

None

Returns

The method returns the attribute value.

Example

```
#Get last_rir_registration_update_sent
my $last_rir_registration_update_sent = $network->last_rir_registration_update_sent();
```

last_rir_registration_update_status( )

Use this method to retrieve the last registration update status of Regional Internet Registry (RIR). This is a read-only attribute.

Parameter

None

Returns

The method returns the attribute value.

Example

```
#Get last_rir_registration_update_status
my $last_rir_registration_update_status = $network->last_rir_registration_update_status();
```

same_port_control_discovery_blackout( )

Use this method to set or retrieve the flag that enables using the same discovery blackout for port control blackout.

Parameter

Specify 'true' to use same discovery blackout for port control blackout or 'false' to disable it. The default value is 'false'.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
# Get attribute value
my $value = $network->same_port_control_discovery_blackout();
# Modify attribute value
$network->same_port_control_discovery_blackout('true');
```

send_rir_request( )

Use this method to set the flag that indicates whether the appliance should send an API or email request for Regional Internet Registry (RIR). This is a write-only attribute.

Parameter

Specify "true" to set the send_rir_request flag or "false" to unset it.

Returns

The method returns true when the modification succeeds, and returns false when the operation fails.

Example

```
#Set send_rir_request
$network->send_rir_request('true');
```

delete_reason( )

Use this method to specify the reason for deleting the Regional Internet Registry (RIR) registration request. This is a write-only attribute. This field is required only if 'rir_registration_action' is specified as 'DELETE'.

Parameter

The reason for deleting the RIR registration request in string format with a maximum of 256 bytes.

Returns

The method returns true when the modification succeeds, and returns false when the operation fails.

Example

```
#Set delete_reason
$network->delete_reason('invalid');
```

shared_network_name( )

Use this method to retrieve the name of the shared network associated with the DHCP IPv6 Network object.

Parameter

none

Returns

The method returns the name of shared network.

Example

```
#Get shared_network_name
my $shared_network_name = $network->shared_network_name();
```

subscribe_settings( )

Use this method to set or retrieve the Cisco ISE subscribe settings.

Setting this method to a defined value implicitly sets the override_subscribe_settings method to "true". Setting the parameter to

undefined causes the appliance to use the Cisco ISE endpoint default and automatically resets the override_subscribe_settings method to "false".

Note that when subscribe_settings is set to a defined value and override_subscribe_settings is set to "false", the last operation takes precedence. Thus the sequence $object->subscribe_settings($value); $object->override_subscribe_settings("false"); will set override_subscribe_settings to "false", and the sequence $object->override_subscribe_settings("false"); $object->subscribe_settings($value); will result in override_subscribe_settings="true".

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

The valid value is an Infoblox::CiscoISE::SubscribeSetting object.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get subscribe_settings value
my $subscribe_settings = $object->subscribe_settings();
#Modify subscribe_settings value
$object->subscribe_settings($settings);
```

template( )

Use this method to set or retrieve the network template name of a DHCP IPv6 Network object.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

The valid name in string format.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get template
my $template = $network->template();
#Modify template
$network->template("Template_Name");
```

unmanaged( )

Use this method to set or retrieve the network status, which indicates whether the network is unmanaged or not.

Parameter

Specify true to mark the network as unmanaged or false as managed.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
# Get network status
$unmanaged = $network->unmanaged();
# Modify network status
$network->unmanaged('true');
```

update_dns_on_lease_renewal( )

The update_dns_on_lease_renewal attribute controls whether the DHCP server updates DNS when a DHCP lease is renewed.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Setting this method to a defined value implicitly sets the override_update_dns_on_lease_renewal method to "true". Setting the parameter to undefined causes the appliance to use the grid default and automatically resets the override_update_dns_on_lease_renewal attribute to

"false".

Note that when update_dns_on_lease_renewal is set to a defined value and override_update_dns_on_lease_renewal is set to "false", the last operation takes precedence. Thus the sequence $object->update_dns_on_lease_renewal("true"); $object->override_update_dns_on_lease_renewal("false"); will set override_update_dns_on_lease_renewal to "false", and the sequence $object->override_update_dns_on_lease_renewal("false"); $object->update_dns_on_lease_renewal("true"); will result in override_update_dns_on_lease_renewal="true".

Parameter

Specify "true" to set the update_dns_on_lease_renewal flag or "false" to deactivate/unset it.

The default value of this parameter is "false".

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get update_dns_on_lease_renewal
my $update_dns_on_lease_renewal=$network->update_dns_on_lease_renewal( );
#Modify update_dns_on_lease_renewal
$network->update_dns_on_lease_renewal("true");
```

valid_lifetime ( )

Use this method to set or retrieve the valid lifetime value of a DHCP IPv6 Network object.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Setting this method to a defined value implicitly sets the override_valid_lifetime method to "true". Setting the parameter to undefined causes the appliance to use the grid default and automatically resets the override_valid_lifetime attribute to "false".

Note that when valid_lifetime is set to a defined value and override_valid_lifetime is set to "false", the last operation takes precedence. Thus the sequence $object->valid_lifetime(691200); $object->override_valid_lifetime("false"); will set override_valid_lifetime to "false", and the sequence $object->override_valid_lifetime("false"); $object->valid_lifetime(691200); will result in override_valid_lifetime="true".

Parameter

A 32-bit integer (valid range from 1 to 4294967295) that represents the valid lifetime, in seconds. The default value is 43200.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get valid lifetime
my $valid_lifetime = $network->valid_lifetime();
#Modify valid lifetime
$network->valid_lifetime(864000);
```

zone_associations( )

Use this method to set or retrieve the associated zone information.

Parameter

The valid value is an array reference that contains Infoblox::DNS::Zone object(s). The default value is an empty list. If the network has a parent, it is possible to use the zone associations of the parent by setting the parameter to undef.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails. If you did not specify a parameter, the method returns the attribute value.

Examples

```
#Getting zone associations
my $zone_associations = $network->zone_associations();
my $zone1 = Infoblox::DNS::Zone->new(
        name      => "test1.com",
);
my $zone2 = Infoblox::DNS::Zone->new(
```

```
            name     => "test2.com",
        );
    #Modifying zone_associations
    $network->zone_associations([$zone1, $zone2]);
    #Clear zone_assocations
    $network->zone_associations([]);
    ##Remove the override for zone_associations
    $network->zone_associations(undef)
```

SAMPLE CODE

The following sample code demonstrates the different functions that can be applied to an object such as add, search, modify, and remove. This sample also includes error handling for the operations.

#Preparation prior to a DHCP IPv6 Network insertion

```
#PROGRAM STARTS: Include all the modules that will be used
use strict;
use Infoblox;

#Create a session to the Infoblox appliance

my $session = Infoblox::Session->new(
    master   => "192.168.1.2",
    username => "admin",
    password => "infoblox"
);
unless ($session) {
   die("Construct session failed: ",
       Infoblox::status_code() . ":" . Infoblox::status_detail());
}
print "Session created successfully\n";
```

#Create a DHCP IPv6 Network object

```
my  $network = Infoblox::DHCP::IPv6Network->new(
              network => "2001::/64",
              comment => "add network",
            );

my $response = $session->add($network)
       or die("Add Network failed: ",
             $session->status_code() . ":" . $session->status_detail());

print "Network added successfully\n";

#Search for a specific Network

#Search the network
my @retrieved_objs = $session->search(
    object  => "Infoblox::DHCP::IPv6Network",
    network => "2001.*"
);
my $object = $retrieved_objs[0];
unless ($object) {
    die("Search Network failed: ",
        $session->status_code() . ":" . $session->status_detail());
}
print "Search Network found at least 1 matching entry\n";
```

#Get and modify a Network attribute

```
#Get IPv6 Network through the session
my @retrieved_objs = $session->get(
           object =>"Infoblox::DHCP::IPv6Network" ,
           network => "2001::"
        );

my $object = $retrieved_objs[0];

unless ($object) {
    die("Get Network object failed: ",
        $session->status_code() . ":" . $session->status_detail());
}
print "Get Network found at least 1 matching entry\n";
```

#Modify one of the attributes of the specified IPv6 Network

```
$object->comment("This is a new comment"); #Apply the changes

$session->modify($object)
    or die("Modify Network attribute failed: ",
            $session->status_code() . ":" . $session->status_detail());

print "Network object modified successfully \n";
```

#Remove an IPv6 Network Object

```
 #Get IPv6 Network object through the session
 my @retrieved_objs = $session->get(
           object  => "Infoblox::DHCP::IPv6Network" ,
           network => "2001::"
       );

   my $object = $retrieved_objs[0];

   unless ($object) {
    die("Get Network object failed: ",
        $session->status_code() . ":" . $session->status_detail());
    }

print "Get Network object found at least 1 matching entry\n";
```

#Submit the object for removal

```
$session->remove($object)
    or die("Remove Network failed: ",
        $session->status_code() . ":" . $session->status_detail());

print "Network removed successfully \n";

####PROGRAM ENDS####
```

AUTHOR

Infoblox Inc. http://www.infoblox.com/

SEE ALSO

Infoblox::Session, Infoblox::DHCP::View, Infoblox::DHCP::IPv6NetworkContainer, Infoblox::DHCP::Network, Infoblox::DHCP::NetworkContainer, Infoblox::Session->get(), Infoblox::Session->search(), Infoblox::Session->add(), Infoblox::Session->remove(), Infoblox::Session->modify(), Infoblox::Grid::MSServer::AdUser::Data

---

COPYRIGHT

Copyright (c) 2017 Infoblox Inc.

Infoblox::Grid::MSServer - Microsoft Grid Server object.

- NAME
- DESCRIPTION
- CONSTRUCTOR
- Infoblox::DNS::Zone Considerations
  - Methods with no effect
  - Applicable methods
- Infoblox::DNS considerations
- Infoblox::Grid::DNS considerations
- SESSION METHODS
  - Infoblox::Session->add( )
  - Infoblox::Session->get( )
  - Infoblox::Session->modify( )
  - Infoblox::Session->remove( )
  - Infoblox::Session->search( )
- METHODS
  - address()
  - ad_domain( )
  - ad_sites()
  - ad_user()
  - comment()
  - disable()
  - login()
  - password()
  - network_view()
  - dns_view()
  - log_destination( )
  - max_connection( )
  - managing_member()
  - override_log_destination( )
  - override_max_connection( )
  - override_rpc_timeout( )
  - read_only()
  - root_ad_domain( )
  - rpc_timeout( )
  - synchronization_interval()
  - logging_mode()
  - version()
  - is_AD()
  - extattrs( )
  - extensible_attributes( )
- SAMPLE CODE
- AUTHOR
- SEE ALSO
- COPYRIGHT

NAME

Infoblox::Grid::MSServer - Microsoft (r) Grid Server object.

DESCRIPTION

A grid Microsoft DNS Server object is a single Microsoft DNS server that is managed by a grid member.

CONSTRUCTOR

```
 my $ms_member = Infoblox::Grid::MSServer->new(
     address                 => $ip_or_fqdn,                                 #Required
     ad_sites                => $ad_sites,                                   #Optional
     ad_user                 => $ad_user,                                    #Optional
     comment                 => $string,                                     #Optional / Default value
is empty
     disable                 => "true" | "false"                             #Optional / Default value
is "false"
     extattrs                => { $string => $extattr, ... },                #Optional / Default is
undefined
     extensible_attributes   => { $string => $string | $num,
                                  $string => [ $string | $num, ... ], ... }, #Optional / Default is
undefined
```

```
     login                    => $string,                                    #Required
     password                 => $string,                                    #Optional / Default value
is empty
     managing_member          => $Member,                                    #Optional / Default value
is empty
     read_only                => "true" | "false",                           #Optional / Default value
is "true"
     synchronization_interval => $num,                                       #Optional / Default value
is 2
     logging_mode             => "minimum" | "normal" | "advanced" | "full", #Optional / Default value
is "normal"
     network_view             => $NetworkView,                               #Optional / Default is
"default" network view
     dns_view                 => $DnsView,                                   #Optional / Default is
"default" view
     log_destination          =>  "SYSLOG" | "MSLOG",                        #Optional / Default is
undefined
     override_log_destination => "true" | "false",                           #Optional / Default value
is "false"
     max_connection           => $num,                                       #Optional / Default is
undefined
     override_max_connection  => "true" | "false",                           #Optional / Default value
is "false"
     rpc_timeout              => $num,                                       #Optional / Default is
undefined
     override_rpc_timeout     => "true" | "false",                           #Optional / Default value
is "false"
 );
```

You cannot set both extattrs and extensible_attributes attributes at the same time.

Infoblox::DNS::Zone Considerations

The following apply to zones assigned to Microsoft DNS servers:

Methods with no effect

The following methods have no effect on a Microsoft DNS server that is assigned to a zone:

allow_transfer()
allow_query( )
allow_update( )
allow_update_forwarding( )
bind_check_names_policy( )
create_underscore_zones( )
disable_forwarding( )
forward_only( )
forward_to( )
import_from( )
override_delegated_ttl( )
override_grid_zone_timer( )
override_grid_email( )
override_serial_number( )
prefix( )
remove_sub_zones( )
soa_negative_ttl( )
shared_record_groups( )

Values set or retrieved using the member that manages the Microsoft DNS server apply only to that grid member if it is assigned to a zone.

Applicable methods

The following methods apply to any grid member or Microsoft DNS server assigned to the zone:

comment( )
delegated_ttl( )
disable( )
email( )
extensible_attributes( )
host_name_restriction_policy( )
locked( )
name( )
ns_group( )
multiple_primaries( )
secondaries( )
soa_default_ttl( )

soa_expire( )
member_soa_mnames( )
soa_refresh( )
soa_retry( )
member_soa_serials( )
stub_from( )
stub_members( )
zone( )

Infoblox::DNS considerations

The following objects cannot be used with a Microsoft DNS Server object or with a Zone object whose primary server is a Microsoft DNS Server object:

Infoblox::DNS::BulkHost
Infoblox::DNS::DnssecTrustedKey
Infoblox::DNS::Host
Infoblox::DNS::SRG
Infoblox::DNS::RootNameServer
Infoblox::DNS::Sortlist
Infoblox::DNS::TSIGKey
Infoblox::DNS::Record

DNAME (only with Windows Server 2008 and later)
DS, RRSIG, DNSKEY, NSEC, NSEC3, NSEC3PARAM (read-only on Windows Server 2008 R2 and later)

Infoblox::Grid::DNS considerations

The following methods can be applied to DNS zones managed by a Microsoft DNS server:

default_ttl( )
email( )
host_name_restriction_policy( )
zone_deletion_double_confirm( )

The following methods have no effect on any zone managed by a Microsoft DNS server:

allow_bulkhost_ddns( )
allow_gss_tsig_zone_updates( )
allow_transfer( )
allow_query( )
allow_recursive_query( )
allow_update( )
bind_check_names_policy( )
blackhole_list( )
bulk_host_name_template( )
bulk_host_name_templates( )
dnssec_enabled( )
dnssec_expired_signatures_enabled( )
`dnssec_ksk_algorithm()`
`dnssec_ksk_rollover_interval()`
`dnssec_ksk_size()`
dnssec_signature_expiration( )
dnssec_validation_enabled( )
dnssec_trusted_keys( )
`dnssec_zsk_algorithm()`
`dnssec_zsk_rollover_interval()`
`dnssec_zsk_size()`
enable_blackhole( )
expire_after( )
forward_only( )
forward_updates( )
forwarders( )
logging_categories( )
logging_facility( )
member_secondary_notify( )
negative_ttl( )
notify_source_port( )
nsgroup_default( )
nsgroups( )
query_source_port( )
record_name_policies( )
recursive_query_list( )
refresh_timer( )

retry_timer( )
rootNS( )
sortlist( )
transfer_excluded_servers( )
transfer_format( )

SESSION METHODS

This section describes all the methods in an Infoblox::Session module that you can apply to a grid member object.

Infoblox::Session->add( )

Use this method to add an object to the Infoblox appliance. See Infoblox::Session->add() for parameters and return values.

Example

```
#Construct an object
my $grid_msserver = Infoblox::Grid::MSServer->new(
                                                  address                  => '10.0.0.1',
                                                  comment                  => 'This is an MS DNS Server',
                                                  disable                  => 'true',
                                                  extensible_attributes    => { Site => 'Main Office'},
                                                  login                    => 'user',
                                                  password                 => 'password',
                                                  managing_member          => 'managing.member.com',
                                                  read_only                => 'false',
                                                  synchronization_interval => 5,
                                                  logging_mode             => 'minimum',
    );
#Submit for addition
my $response = $session->add( $grid_msserver );
```

Infoblox::Session->get( )

Use this method to retrieve all the matching objects from the Infoblox appliance. See Infoblox::Session->get() for parameters and return values.

Key References

```
Apply the following attributes to get a specific grid member object:

 address - Optional. A member address or name in FQDN format.
 comment - Optional. A comment in string format
 read_only - Optional. A boolean value
 managing_member - Optional. A name in FQDN format.
 network_view  -  Optional. The network view served by the member.
 dns_view -  Optional. The DNS view served by the member.
 extattrs - Optional. A hash reference containing extensible attributes.
 extensible_attributes - Optional. A hash reference containing extensible attributes.
```

Example

```
my @retrieved_objs = $session->get(
    object  => "Infoblox::Grid::MSServer",
    address => "10.0.0.1");
```

Infoblox::Session->modify( )

Use this method to modify an object in the Infoblox appliance. See Infoblox::Session->modify() for parameters and return values.

Example

```
# Use this method to modify the comment of a Microsoft DNS server managed by the grid.
$grid_msserver->comment("This is modified object");
# Submit modification
my $response = $session->modify( $grid_msserver );
```

Infoblox::Session->remove( )

Use this method to remove an object from the Infoblox appliance. See Infoblox::Session->remove() for parameters and return values.

To remove a specific object, first use `get()` or `search()` to retrieve the specific object, and then submit this object for removal.

Example

```
#Get the objects with the same name
my @retrieved_objs = $session->get(
```

```
    object => "Infoblox::Grid::MSServer",
    name   => "10.0.0.1");
#Find the desired object from the retrieved list.
my $desired_gridmsserver = $retrieved_objs[0];
# Submit for removal
my $response = $session->remove( $desired_gridmsserver );
```

Infoblox::Session->search( )

Use this method to search for Microsoft DNS Server objects in the Infoblox appliance. See Infoblox::Session->search() for parameters and return values.

Key References

Apply the following attributes to search for a Microsoft DNS Server object

```
address - Optional. A member address or name in FQDN format (regular expression).
comment - Optional. A comment in string format (regular expression).
read_only - Optional. A boolean value
managing_member - Optional. The FQDN of the grid member that manages the Microsoft DNS server?
network_view  -  Optional. The network view served by the member.
dns_view -  Optional. The DNS view served by the member.
extattrs - Optional. A hash reference containing extensible attributes.
extensible_attributes   - Optional. A hash reference that contains extensible attributes.
```

For more information about searching extensible attributes, see Infoblox::Grid::ExtensibleAttributeDef/Searching Extensible Attributes.

Example

```
# search for all matching grid msserver objects
my @retrieved_objs = $session->search(
    object  => "Infoblox::Grid::MSServer",
    extensible_attributes => { Site => 'Local'},
    );
```

METHODS

This section describes all the methods that you can use to configure and retrieve the attribute values of a Microsoft DNS server.

address()

Retrieve or set the IP address or FQDN of the Microsoft DNS server.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

The Microsoft DNS server name in FQDN format or IPV4 format

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get address
my $address = $msserver->address();
```

ad_domain( )

Use this method to retrieve the Active Directory domain name of the Microsoft Server. This is a read-only attribute.

Omit the parameter to retrieve the attribute value.

Parameter

None

Returns

The method returns the attribute value.

Example

```
#Get ad_domin value
my $ad_domain = $object->ad_domain();
```

ad_sites()

Use this method to set or retrieve structure containing Active Directory Sites information.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

The valid value is an Infoblox::Grid::MSServer::AdSites object.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get ad_sites value
my $ad_sites = $msserver->ad_sites();

#Modify ad_sites value
my $res = $msserver->ad_sites($ad_sites);
```

ad_user()

Use this method to set or retrieve the object that contains Active Directory User synchronization information.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

The valid value is an Infoblox::Grid::MSServer::AdUser object.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get ad_user value
my $ad_user = $msserver->ad_user();

#Modify ad_user value
my $res = $msserver->ad_user($ad_user);
```

comment()

Retrieve or set a user-defined string related to this Microsoft DNS server.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

Desired comment in string format with a maximum of 256 bytes.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get comment
my $comment = $msserver->comment();
#Modify comment
$msserver->comment("Modifying the MS DNS server comment");
```

disable()

Retrieve or set the `disable()` flag of this Microsoft DNS server.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

Specify "true" to set the disable flag or "false" to deactivate/unset it. The default value is "false".

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get disable
my $disable = $msserver->disable();
#Modify disable
$msserver->disable("true");
```

login()

Retrieve or set the login name that the grid member uses to connect to the Microsoft DNS server.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

When setting a value that requires a domain name, prepend the domain information followed by a backslash to the username: "domain\username".

Parameter

The valid value is a string that contains the login name.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get login
my $login = $msserver->login();
#Modify login
$msserver->login("newuser");
```

password()

Set the password that the grid member uses to connect to the Microsoft DNS server. This field is used only if the login field for this member is defined. This is a write-only attribute.

This field can not be retrieved.

Parameter

The valid value is a string that contains the password.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

Example

```
#Set the password
$msserver->password("iG&ojG97Y");
```

network_view()

Retrieve or set the name of the network view associated with this member.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

The valid value is a string that contains the name of the network view associated with this member.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get network_view
```

```
my $network_view = $msserver->network_view();
#Modify network_view
$msserver->network_view("external");
```

dns_view()

Retrieve or set the name of the DNS view that applies to this Microsoft DNS server. If this field is set to undefined, the default view is used. The DNS view must be set to a DNS view linked to the Microsoft DNS server network view.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

The valid value is a string that contains the name of the DNS view associated with the Microsoft DNS server

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get dns_view
my $dns_view = $msserver->dns_view();
#Modify dns_view
$msserver->dns_view("external");
```

log_destination( )

Use this method to direct the logging of synchronization messages to the syslog or mslog.

Setting this method to a defined value implicitly sets the override_log_destination method to "true". Setting the parameter to undefined causes the appliance to use the grid default and automatically resets the override_log_destination method to "false".

Note that when log_destination is set to a defined value and override_log_destination is set to "false", the last operation takes precedence. Thus the sequence $object->log_destination("MSLOG"); $object->override_log_destination("false"); will set override_log_destination to "false", and the sequence $object->override_log_destination("false"); $object->log_destination("MSLOG"); will result in override_log_destination="true".

Parameter

Valid values are "SYSLOG" and "MSLOG".

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get log_destination value
my $log_destination = $msserver->log_destination();
#Modify log_destination value
$msserver->log_destination("SYSLOG");
```

max_connection( )

Use this method to set or retrieve the maximum number of connections to a Microsoft server.

Setting this method to a defined value implicitly sets the override_max_connection method to "true". Setting the parameter to undefined causes the appliance to use the grid default and automatically resets the override_max_connection method to "false".

Note that when max_connection is set to a defined value and override_max_connection is set to "false", the last operation takes precedence. Thus the sequence $object->max_connection(20); $object->override_max_connection("false"); will set override_max_connection to "false", and the sequence $object->override_max_connection("false"); $object->max_connection(20); will result in override_max_connection="true".

Parameter

Valid values are integers between 2 and 40 inclusive.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get max_connection value
my $max_connection = $msserver->max_connection();
#Modify max_connection value
$msserver->max_connection(20);
```

managing_member()

Retrieve or set the grid member responsible for the synchronization of this Microsoft DNS server.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

The valid value is a string that contains the name of the grid member.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get managing_member
my $managing_member = $msserver->managing_member();
#Modify managing_member
$msserver->managing_member("example.member.com");
```

override_log_destination( )

The override_log_destination attribute controls whether the log_destination method values of the Microsoft Server are used, instead of the Grid default.

The override_log_destination attribute can be specified explicitly. It is also set implicitly when log_destination is set to a defined value.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

Set the parameter to "true" to override the Grid-level setting for log_destination. Set the parameter to "false" to inherit the Grid-level setting for log_destination. The default value is "false".

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get override_log_destination
my $override_log_destination = $msserver->override_log_destination();
#Modify override_log_destination
$msserver->override_log_destination("true");
```

override_max_connection( )

The override_max_connection attribute controls whether the max_connection method values of the Microsoft Server are used, instead of the Grid default.

The override_max_connection attribute can be specified explicitly. It is also set implicitly when max_connection is set to a defined value.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

Set the parameter to "true" to override the grid-level setting for max_connection. Set the parameter to "false" to inherit the grid-level setting for max_connection. The default value is "false".

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get override_max_connection
my $override_max_connection = $msserver->override_max_connection();
#Modify override_max_connection
$msserver->override_max_connection("true");
```

override_rpc_timeout( )

The override_rpc_timeout attribute controls whether the rpc_timeout method values of the Microsoft Server are used, instead of the Grid default.

The override_rpc_timeout attribute can be specified explicitly. It is also set implicitly when rpc_timeout is set to a defined value.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

Set the parameter to "true" to override the grid-level setting for rpc_timeout. Set the parameter to "false" to inherit the grid-level setting for rpc_timeout. The default value is "false".

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get override_rpc_timeout
my $override_rpc_timeout = $msserver->override_rpc_timeout();
#Modify override_rpc_timeout
$msserver->override_rpc_timeout("true");
```

read_only()

Retrieve or set the synchronization mode of this Microsoft DNS server.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Setting the read_only mode to true synchronizes all data from the Microsoft DNS server to the grid and forbids any change on the grid that could be synchronized to the Microsoft DNS server.

Parameter

Specify "true" to set the disable flag or "false" to deactivate/unset it. The default value is "true".

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get read_only
my $read_only = $msserver->read_only();
#Modify read_only
$msserver->read_only("false");
```

root_ad_domain( )

Use this method to retrieve the Active Directory root domain name of the Microsoft Server. This is a read-only attribute.

Omit the parameter to retrieve the attribute value.

Parameter

None

Returns

The method returns the attribute value.

Example

```
#Get ad_domin value
my $root_ad_domain = $object->root_ad_domain();
```

rpc_timeout( )

Use this method to set or retrieve the timeout value in seconds for RPC connections to Microsoft Server.

Setting this method to a defined value implicitly sets the override_rpc_timeout method to "true". Setting the parameter to undefined causes the appliance to use the grid default and automatically resets the override_rpc_timeout method to "false".

Note that when rpc_timeout is set to a defined value and override_rpc_timeout is set to "false", the last operation takes precedence. Thus the sequence $object->rpc_timeout(10); $object->override_rpc_timeout("false"); will set override_rpc_timeout to "false", and the sequence $object->override_rpc_timeout("false"); $object->rpc_timeout(10); will result in override_rpc_timeout="true".

Parameter

Valid values are integers between 0 and 65535 inclusive.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get rpc_timeout value
my $rpc_timeout = $msserver->rpc_timeout();
#Modify rpc_timeout value
$msserver->rpc_timeout(10);
```

synchronization_interval()

Retrieve or set the synchronization interval of this Microsoft DNS server.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

The synchronization interval controls how often data on the Microsoft DNS server is synchronized with the grid. The value is specified in minutes.

Parameter

The valid value is a number that specifies the synchronization interval in minutes. The default value is 2 minutes.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get synchronization_interval
my $synchronization_interval = $msserver->synchronization_interval();
#Modify synchronization_interval
$msserver->synchronization_interval(5);
```

logging_mode()

Retrieve or set the synchronization logging verbosity of this Microsoft DNS server.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

The valid value is "minimum", "normal", "advanced", or "full". The default value is "normal".

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get logging_mode
my $logging_mode = $msserver->logging_mode();
#Modify logging_mode
$msserver->logging_mode("advanced");
```

version()

Retrieve the version of this Microsoft DNS server. The returned version is a string, such as "Microsoft Windows Server 2003".

Returns

The method returns the attribute value.

Example

```
#Get version
my $version = $msserver->version();
```

is_AD()

Retrieve the Active Directory flag of the Microsoft DNS server. The method returns "true" if the Microsoft DNS server has Active Directory capabilities or "false" if it does not.

Returns

The method returns the attribute value.

Example

```
#Get is_AD
my $is_AD = $msserver->is_AD();
```

extattrs( )

Use this method to set or retrieve the extensible attributes associated with a Microsoft DNS server object.

Parameter

Valid value is a hash reference containing the names of extensible attributes and their associated values (Infoblox::Grid::Extattr objects).

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get extattrs
my $ref_extattrs = $msserver->extattrs();
#Modify extattrs
$msserver->extattrs({ 'Site' => $extattr1, 'Administrator' => $extattr2 });
```

extensible_attributes( )

Use this method to set or retrieve the extensible attributes associated with a Microsoft DNS server.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

For valid values for extensible attributes, see Infoblox::Grid::ExtensibleAttributeDef/Extensible Attribute Values.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get extensible attributes
my $ref_extensible_attributes = $msserver->extensible_attributes();
#Modify extensible attributes
$msserver->extensible_attributes({ 'Site' => 'Santa Clara', 'Administrator' => [ 'Peter', 'Tom' ]
});
```

SAMPLE CODE

The following sample code demonstrates the different functions that can be applied to an object such as add, get, modify, search and remove. This sample also includes error handling for the operations.

#Preparation prior to getting and modifying a grid msserver object

```
use strict;
use Infoblox;
```

```
#refers to Infoblox Appliance IP address
my $host_ip = "192.168.1.2";

#Create a session to the Infoblox appliance

my $session = Infoblox::Session->new(
    master   => $host_ip,
    username => "admin",
    password => "infoblox"
);
unless ($session) {
       die("Construct session failed: ",
               Infoblox::status_code() . ":" . Infoblox::status_detail());
}
print "Session created successfully\n";

#Create a grid member object

my $grid_member = Infoblox::Grid::Member->new(
    name          => "hostname.com",
    ipv4addr      => "3.0.0.2",
    mask          => "255.0.0.0",
    gateway       => "3.0.0.1",
    comment       => "Script Generated"
    );
unless ($grid_member) {
       die("Construct grid member object failed: ",
               Infoblox::status_code() . ":" . Infoblox::status_detail());
}
print "Grid member object created successfully\n";

#Adding the member object to appliance through session
my $response = $session->add( $grid_member );

unless ($response) {
    die("Add grid member failed: ",
           $session->status_code() . ":" . $session->status_detail());
}
print"Grid member added successfully \n";

my $msserver = Infoblox::Grid::MSServer->new(
                                          address                  => '10.0.0.1',
                                          comment                  => 'basic member',
                                          disable                  => 'true',
                                          extensible_attributes    => { Site => 'Local'},
                                          login                    => 'user',
                                          password                 => 'password',
                                          managing_member          => 'hostname.com',
                                          read_only                => 'false',
                                          synchronization_interval => 5,
                                          logging_mode             => 'minimum',
                                         );

unless ($msserver) {
       die("Construct grid MS DNS server object failed: ",
               Infoblox::status_code() . ":" . Infoblox::status_detail());
}
print "Grid MS DNS server object created successfully\n";

#Adding the member object to appliance through session
$response = $session->add( $msserver );

unless ($response) {
    die("Add grid MS DNS server failed: ",
           $session->status_code() . ":" . $session->status_detail());
}
print"Grid MS DNS server added successfully \n";
```

#Search for a grid Microsoft DNS server object

```
 my @retrieved_objs = $session->search(
     object => "Infoblox::Grid::MSServer",
     comment => "basic.*");
 my $object = $retrieved_objs[0];

 unless ($object) {
        die
```

```
            ("Search grid MS DNS server object failed: ",
                $session->status_code() . ":" . $session->status_detail());
 }
 print "Search grid MS DNS server object found at least 1 matching entry\n";
```

#Get and modify the msserver object

```
#Get grid member through session
my @retrieved_objs = $session->get(
    object => "Infoblox::Grid::MSServer",
    address => "10.0.0.1"
    );
my $msserver = $retrieved_objs[0];

unless ($msserver) {
    die("Get grid MS DNS server object failed: ",
           $session->status_code() . ":" . $session->status_detail());
}
print"Get grid MS DNS server successful \n";

# Modify one of the attributes of the selected Microsoft DNS server.

$msserver->comment("Modified MS DNS server object");

#Applying the changes
$session->modify($msserver)
   or die("Modify grid MS DNS server failed",
            $session->status_code() . ":" . $session->status_detail());
print"Grid MS DNS server modified successfully \n";
```

#Remove the created objects object

```
#Get grid member through session
my @retrieved_objs = $session->get(
    object => "Infoblox::Grid::MSServer",
    address => "10.0.0.1"
    );
my $desired_msserver = $retrieved_objs[0];

unless ($desired_msserver) {
    die("Get grid MS DNS server object failed: ",
           $session->status_code() . ":" . $session->status_detail());
}
print"Get grid member successful \n";

#Submit the object for removal
$session->remove( $desired_msserver )
    or die("Remove grid MS DNS server failed",
           $session->status_code() . ":" . $session->status_detail());
print"Grid MS DNS server removed successfully \n";

#Get grid member through session
my @retrieved_objs = $session->get(
    object => "Infoblox::Grid::Member",
    name   => "hostname.com"
    );
my $desired_grid_member = $retrieved_objs[0];

unless ($desired_grid_member) {
    die("Get grid member object failed: ",
           $session->status_code() . ":" . $session->status_detail());
}
print"Get grid member successful \n";

#Submit the object for removal
$session->remove( $desired_grid_member )
    or die("Remove grid member failed",
           $session->status_code() . ":" . $session->status_detail());
print"Grid member removed successfully \n";

####PROGRAM ENDS####
```

AUTHOR

Infoblox Inc. http://www.infoblox.com/

SEE ALSO

Infoblox::Session, Infoblox::Grid::MSServer::AdSites, Infoblox::Grid::MSServer::AdUser.

COPYRIGHT

Copyright (c) 2017 Infoblox Inc.

Infoblox::DHCP::Member - DHCP Member object.

- NAME
- DESCRIPTION
- CONSTRUCTOR
- MODULE METHODS
  - Infoblox::DHCP::Network->members( )
  - Infoblox::DHCP::Range->member( )
- METHODS
  - ipv4addr( )
  - ipv6addr( )
  - name( )
- SAMPLE CODE
- AUTHOR
- SEE ALSO
- COPYRIGHT

NAME

Infoblox::DHCP::Member - DHCP Member object.

DESCRIPTION

A grid member is part of a grid which is a group of two or more Infoblox appliances that share sections of a common, distributed, built-in database. A grid member inherits its settings from the grid. A member is a single Infoblox appliance or an HA pair that provides DHCP services seamlessly across an entire network.

CONSTRUCTOR

```
my $member = Infoblox::DHCP::Member->new(
    ipv4addr => $ipv4addr, #Optional. At least one ipv4addr or ipv6addr is required.
    ipv6addr => $ipv6addr, #Optional. At least one ipv4addr or ipv6addr is required.
    name     => $fqdn,     #Optional
);
```

MODULE METHODS

This section describes all the functions that you can apply to an DHCP Member object.

Infoblox::DHCP::Network->members( )

Use this function to specify the primary member for a network. See Infoblox::DHCP::Network->members() for parameters and return values.

Example

```
my $member1 = Infoblox::DHCP::Member->new(
    name     => "localhost.localdomain",
    ipv4addr => "192.168.1.2",
);

unless($member1) {
     die("Construct member failed: ",
           Infoblox::status_code() . ":" . Infoblox::status_detail());
}
print "Member object created successfully\n";

#Create the Network object with this member
my $network = Infoblox::DHCP::Network->new(
    network => "10.0.0.0/255.0.0.0",
    members => [$member1],
);
```

Infoblox::DHCP::Range->member( )

Use this function to specify the primary member for a DHCP range. See Infoblox::DHCP::Range->members() for parameters and return values.

Example

```
my $member = Infoblox::DHCP::Member->new(
    name     => "localhost.localdomain",
    ipv4addr => "192.168.1.2",
);

unless($member) {
     die("Construct member failed: ",
           Infoblox::status_code() . ":" . Infoblox::status_detail());
}
print "Member object created successfully\n";

#Create the DHCP Range object with this member
my $dhcp_range = Infoblox::DHCP::Range->new(
    network    => "10.0.0.0/8",
    start_addr => "10.0.0.1",
    end_addr   => "10.0.0.10",
    member     => $member,
);
```

METHODS

This section describes all the methods that you can use to set and retrieve the attribute values of a DHCP Member object.

ipv4addr( )

Use this method to set or retrieve the IPv4 address of the member.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

IPv4 address (32 bits) of the member.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get ipv4addr
my $ipv4addr = $member->ipv4addr();
#Modify ipv4addr
$member->ipv4addr("192.168.1.3");
```

ipv6addr( )

Use this method to set or retrieve the IPv6 address of the member.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

IPv6 address of the member.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get ipv6addr
my $ipv6addr = $member->ipv6addr();
#Modify ipv6addr
$member->ipv6addr("fd46:e105:2eac:89b1::1");
```

name( )

Use this method to set or retrieve the name of the member.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

A name in FQDN (Fully Qualified Domain Name) format. The FQDN consists of the host name followed by the domain name (example: abc.com). A host name can have a maximum of 256 characters.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get name
my $name = $member->name();
#Modify name
$member->name("dhcp.infoblox.com");
```

SAMPLE CODE

The following sample code demonstrates the different functions that can be applied to an object, such as add, search, modify, and remove. This sample also includes error handling for the operations.

#Preparation prior to a DHCP Member object insertion

```
#PROGRAM STARTS: Include all the modules that will be used
use strict;
use Infoblox;

#Create a session to the Infoblox appliance

my $session = Infoblox::Session->new(
               master   => "192.168.1.2", #appliance host ip
               username => "admin",       #appliance user login
               password => "infoblox"     #appliance password
);

unless ($session) {
       die("Construct session failed: ",
               Infoblox::status_code() . ":" . Infoblox::status_detail());
}
print "Session created successfully\n";
```

#Create a DHCP Member object

```
my $member1 = Infoblox::DHCP::Member->new(
    name     => "infoblox.localdomain",
    ipv4addr => "192.168.1.2",
);

unless($member1) {
     die("Construct member failed: ",
           Infoblox::status_code() . ":" . Infoblox::status_detail());
}
print "Member object created successfully\n";

#Create the Network object with this member
my $network = Infoblox::DHCP::Network->new(
                    network => "10.0.0.0/255.0.0.0",
                    comment => "add network",
                    members => [ $member1 ]
);

unless($network) {
            die("Construct Network object failed: ",
                  Infoblox::status_code() . ":" . Infoblox::status_detail());
}
print "Network object created successfully\n";

#Add the Network object into the Infoblox appliance through a session
$session->add($network)
     or die("Add Network object failed: ",
           $session->status_code() . ":" . $session->status_detail());
print "DHCP member object added to the network successfully\n";

####PROGRAM ENDS####
```

AUTHOR

Infoblox Inc. http://www.infoblox.com/

SEE ALSO

Infoblox::DHCP::Network, Infoblox::Session->add(),Infoblox::Session

---

COPYRIGHT

Copyright (c) 2017 Infoblox Inc.

Infoblox::DHCP::Fingerprint - DHCP Fingerprint object.

- NAME
- DESCRIPTION
- CONSTRUCTOR
- SESSION METHODS
  - Infoblox::Session->add( )
  - Infoblox::Session->get( )
  - Infoblox::Session->modify( )
  - Infoblox::Session->remove( )
  - Infoblox::Session->search( )
- METHODS
  - comment( )
  - device_class( )
  - name( )
  - disable( )
  - extattrs( )
  - extensible_attributes( )
  - option_sequence( )
  - ipv6_option_sequence( )
  - type( )
  - vendor_id( )
- SAMPLE CODE
- AUTHOR
- SEE ALSO
- COPYRIGHT

NAME

Infoblox::DHCP::Fingerprint - DHCP Fingerprint object.

DESCRIPTION

DHCP Fingerprint object.

CONSTRUCTOR

```
my $fingerprint = Infoblox::DHCP::Fingerprint->new(
   name                  => $string,                              # required
   device_class          => $string,                              # required
   type                  => 'CUSTOM' | 'STANDARD',                # optional, default is 'CUSTOM'
   comment               => $string,                              # optional
   disable               => 'true' | 'false',                     # optional, default is 'false'
   vendor_id             => [$string1, $string2, ...],            # optional
   option_sequence       => [$string1, $string2, ...],            # optional
   ipv6_option_sequence  => [$string1, $string2, ...],            # optional
   extattrs              => { $string => $extattr, ... },         # optional / Default is undefined
   extensible_attributes => $extattrs,                            # optional
);
```

You cannot set both extattrs and extensible_attributes attributes at the same time.

At least one of option_sequence, ipv6_option_sequence or vendor_id must be defined.

SESSION METHODS

This section describes all the methods in an Infoblox::Session module that you can apply to DHCP Fingerprint objects.

Infoblox::Session->add( )

Use this method to add an object to the Infoblox appliance if the type value is 'CUSTOM'. Otherwise this method is not supported. See Infoblox::Session->add() for parameters and return values.

Example

```
#Construct an object
my $fingerprint = Infoblox::DHCP::Fingerprint->new(
   name => 'Infoblox',
   device_class => 'Inc',
```

```
    option_sequence => ['1'],
);

#Submit for addition
my $response = $session->add( $fingerprint );
```

Infoblox::Session->get( )

Use this method to retrieve all the matching objects from the Infoblox appliance. See Infoblox::Session->get() for parameters and return values.

Key References

```
Apply the following attributes to get a specific DHCP Fingerprint object:

 name                       - Optional. The name of the DHPC Fingerprint object.
 device_class               - Optional. The class of DHCP Fingerprint object.
 type                       - Optional. The type of DHCP Fingerprint object.
 comment                    - Optional. The comment.
 extattrs                   - Optional. A hash reference containing extensible attributes.
 extensible_attributes      - Optional. A hash reference containing extensible attributes.
```

Example

```
# Get DHCP Fingerprint object by name
my @retrieved_objs = $session->get(
    object  => 'Infoblox::DHCP::Fingerprint',
    name    => 'Infoblox',
);

# Get all base DHCP Fingerprint objects
my @retrieved_objs = $session->get(
    object  => 'Infoblox::DHCP::Fingerprint',
    type    => 'STANDARD',
);
```

Infoblox::Session->modify( )

Use this method to modify an object in the Infoblox appliance. See Infoblox::Session->modify() for parameters and return values.

Example

```
# Modify comment
$fingerprint->comment("This is modified object");
# Submit modification
my $response = $session->modify( $fingerprint );
```

Infoblox::Session->remove( )

Use this method to remove an object from the Infoblox appliance if the type value is 'CUSTOM'. Otherwise this method is not supported. See Infoblox::Session->remove() for parameters and return values.

To remove a specific object, first use `get()` or `search()` to retrieve the specific object, and then submit this object for removal.

Example

```
# Get the object by name
my @retrieved_objs = $session->get(
    object    => 'Infoblox::DHCP::Fingerprint',
    name      => 'Infoblox',
);
#Find the desired object from the retrieved list.
my $desired_fingerprint = $retrieved_objs[0];
# Submit for removal
my $response = $session->remove( $desired_fingerprint );
```

Infoblox::Session->search( )

Use this method to search for DHCP Fingerprint objects in the Infoblox appliance. See Infoblox::Session->search() for parameters and return values.

Key References

```
Apply the following attributes to search for specific DHCP Fingerprint objects:

  name                       - Optional. The name of the DHPC Fingerprint object (regular
expression).
  device_class               - Optional. The class of DHCP Fingerprint object (regular expression).
  type                       - Optional. The type of DHCP Fingerprint object.
```

```
 comment                    - Optional. The comment (regular expression).
 extattrs                   - Optional. A hash reference containing extensible attributes.
 extensible_attributes      - Optional. A hash reference containing extensible attributes.
```

Example

```
# Search DHCP Fingerprint object by name
my @retrieved_objs = $session->search(
   object  => 'Infoblox::DHCP::Fingerprint',
   name    => 'Infoblox',
);

# Search all base DHCP Fingerprint objects
my @retrieved_objs = $session->search(
   object  => 'Infoblox::DHCP::Fingerprint',
   type    => 'STANDARD',
);
```

METHODS

This section describes all the methods that you can use to configure and retrieve the attribute values of DHCP Fingerprint objects.

comment( )

Use this method to set or retrieve a descriptive comment.

Parameter

Desired comment in string format with a maximum of 256 bytes.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get attribute value
my $value = $fingerprint->comment();
#Modify attribute value
$fingerprint->comment('new comment');
```

device_class( )

Use this method to set or retrieve a class of DHCP Fingerprint object.

Parameter

The class in string format with a maximum of 256 bytes.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get attribute value
my $value = $fingerprint->device_class();
#Modify attribute value
$fingerprint->device_class('Infoblox Inc.');
```

name( )

Use this method to set or retrieve a name of DHCP Fingerprint object.

Parameter

The name in string format with a maximum of 256 bytes.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get attribute value
my $value = $fingerprint->name();
#Modify attribute value
$fingerprint->name('custom fingerprint');
```

disable( )

Use this method to enable or disable a DHCP Fingerprint object.

If a DHCP Fingerprint object is enabled, the option number sequence and vendor_ids for this operating system or device will be included in the list of fingerprints to match incoming lease requests.

Parameter

Specify "true" to disable DHCP Fingerprint object or "false" to enable it. The default value is 'false'.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get attribute value
my $value = $fingerprint->disable();
#Modify attribute value
$fingerprint->disable('false');
```

extattrs( )

Use this method to set or retrieve the extensible attributes associated with a DHCP Fingerprint object.

Parameter

Valid value is a hash reference containing the names of extensible attributes and their associated values (Infoblox::Grid::Extattr objects).

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get extattrs
my $ref_extattrs = $fingerprint->extattrs();
#Modify extattrs
$fingerprint->extattrs({ 'Site' => $extattr1, 'Administrator' => $extattr2 });
```

extensible_attributes( )

Use this method to set or retrieve the extensible attributes associated with DHCP Fingerprint objects.

Parameter

For valid values for extensible attributes, see Infoblox::Grid::ExtensibleAttributeDef/Extensible Attribute Values.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get attribute value
my $value = $fingerprint->extensible_attributes();
#Modify attribute value
$fingerprint->extensible_attributes({ 'Site' => 'Santa Clara', 'Administrator' => [ 'Peter', 'Tom' ]
});
```

option_sequence( )

Use this method to set or retrieve a list (comma separated list) of IPv4 option number sequences of the device or operating system.

At least one of option_sequence, ipv6_option_sequence or vendor_id must be defined.

Parameter

Reference to an array of IPv4 option number sequence (comma separated list) in string format.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get attribute value
my $value = $fingerprint->option_sequence();
#Modify attribute value
$fingerprint->option_sequence(['1,2,3']);
```

ipv6_option_sequence( )

Use this method to set or retrieve a list (comma separated list) of IPv6 option number sequences of the device or operating system.

At least one of option_sequence, ipv6_option_sequence or vendor_id must be defined.

Parameter

Reference to an array of IPv6 option number sequence (comma separated list) in string format.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get attribute value
my $value = $fingerprint->ipv6_option_sequence();
#Modify attribute value
$fingerprint->ipv6_option_sequence(['1,2,3']);
```

type( )

Use this method to set or retrieve the type of the DHCP Fingerprint object.

Parameter

Valid value can be 'STANDARD' or 'CUSTOM'. Standard fingerprints are the fingerprints from fingerbank. Custom fingerprints are user defined fingerprints.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get attribute value
my $value = $fingerprint->type();
#Modify attribute value
$fingerprint->type('CUSTOM');
```

vendor_id( )

Use this method to set or retrieve a list of vendor IDs of the device or operating system.

At least one of option_sequence, ipv6_option_sequence or vendor_id must be defined.

Parameter

Reference to an array of vendor ID in string format.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get attribute value
my $value = $fingerprint->vendor_id();
#Modify attribute value
$fingerprint->vendor_id(['3','4']);
```

SAMPLE CODE

The following sample code demonstrates the different functions that can be applied to an object such as add, get, modify, search and remove. This sample also includes error handling for the operations.

#Preparation prior to getting and modifying DHCP Fingerprint object

```
use strict;
use Infoblox;

#refers to Infoblox Appliance IP address
my $host_ip = "192.168.1.2";

#Create a session to the Infoblox appliance

my $session = Infoblox::Session->new(
    master   => $host_ip,
    username => "admin",
    password => "infoblox"
);
unless ($session) {
       die("Construct session failed: ",
               Infoblox::status_code() . ":" . Infoblox::status_detail());
}
print "Session created successfully\n";
```

#Create DHCP Fingerprint object

```
my $fingerprint = Infoblox::DHCP::Fingerprint->new(
   'name' => 'Infoblox',
   'device_class' => 'Inc',
   'option_sequence' => ['1'],
);

unless ($fingerprint) {
       die("Construct DHCP Fingerprint object failed: ",
               Infoblox::status_code() . ":" . Infoblox::status_detail());
}
print "DHCP Fingerprint object created successfully\n";

my $response = $session->add($fingerprint);

unless ($response) {
    die("Add DHCP Fingerprint object failed: ",
           $session->status_code() . ":" . $session->status_detail());
}
print"DHCP Fingerprint object added successfully \n";
```

#Search for DHCP Fingerprint object

```
my @retrieved_objs = $session->search(
   object   => 'Infoblox::DHCP::Fingerprint',
   name     => 'Infoblox',
);

$fingerprint = $retrieved_objs[0];

unless ($fingerprint) {
        die("Search DHCP Fingerprint object object failed: ",
                $session->status_code() . ":" . $session->status_detail());
}
print "Search DHCP Fingerprint object found at least 1 matching entry\n";
```

#Get and modify DHCP Fingerprint object

```
@retrieved_objs = $session->get(
   object  => 'Infoblox::DHCP::Fingerprint',
   name    => 'Infoblox',
```

```
);

$fingerprint = $retrieved_objs[0];

unless ($fingerprint) {
    die("Get DHCP Fingerprint object failed: ",
            $session->status_code() . ":" . $session->status_detail());
}
print"Get DHCP Fingerprint object successful \n";

$fingerprint->comment("This is modified object");

$session->modify($fingerprint)
   or die("Modify DHCP Fingerprint object failed",
             $session->status_code() . ":" . $session->status_detail());
print "DHCP Fingerprint object modified successfully \n";
```

#Remove DHCP Fingerprint object

```
@retrieved_objs = $session->get(
   object   => 'Infoblox::DHCP::Fingerprint',
   name     => 'Infoblox',
);

$fingerprint = $retrieved_objs[0];

unless ($fingerprint) {
    die("Get DHCP Fingerprint object failed: ",
            $session->status_code() . ":" . $session->status_detail());
}
print"Get DHCP Fingerprint object successful \n";

$session->remove( $fingerprint )
    or die("Remove DHCP Fingerprint object failed",
            $session->status_code() . ":" . $session->status_detail());
print"DHCP Fingerprint object removed successfully \n";

####PROGRAM ENDS####
```

AUTHOR

Infoblox Inc. http://www.infoblox.com/

SEE ALSO

Infoblox::Session

COPYRIGHT

Copyright (c) 2017 Infoblox Inc.

Infoblox::DHCP::Option - DHCP Option object.

- NAME
- DESCRIPTION
- CONSTRUCTOR
- MODULE METHODS
  - Infoblox::Grid::DHCP->options( )
  - Infoblox::Grid::Member::DHCP->options( )
  - Infoblox::DHCP::Network->options( )
  - Infoblox::DHCP::NetworkTemplate->options( )
  - Infoblox::DHCP::SharedNetwork->options( )
  - Infoblox::DHCP::FixedAddr->options( )
  - Infoblox::DHCP::FixedAddrTemplate->options( )
  - Infoblox::DHCP::Range->options( )
  - Infoblox::DHCP::RangeTemplate->options( )
  - Infoblox::DHCP::Filter::Option->option_list( )
- METHODS
  - name( )
  - num( )
  - value( )
  - vendor_class( )
- SAMPLE CODE
- AUTHOR
- SEE ALSO
- COPYRIGHT

NAME

Infoblox::DHCP::Option - DHCP Option object.

DESCRIPTION

An Option sets the value of a DHCP option that has been defined in an option space. DHCP options describe network configuration settings and various services available on the network. These options occur as variable-length fields at the end of DHCP messages.

CONSTRUCTOR

```
 my $option = Infoblox::DHCP::Option->new(
     value        => $num | $string |"true"|"false", #Required
     name         => $string,                        #Optional - One of name or num must be specified /
Default is undefined
     num          => $num,                           #Optional - One of name or num must be specified /
Default is undefined
     vendor_class => $string,                        #Optional / Default Option Space name is "DHCP"
 );
```

Note that in a future NIOS version 'num' will become mandatory and 'name' will become optional.

MODULE METHODS

The following functions can be applied to a DHCP Option object.

Infoblox::Grid::DHCP->options( )

Use this function to specify DHCP options at the grid level on the Infoblox appliance. See Infoblox::Grid::DHCP->options() for parameters and return values.

Example

```
       #Construct a DHCP Option object
       my $option1 = Infoblox::DHCP::Option->new(
                     name  => "lease-time",
                     value => "50"
       );
       # Configure a DHCP option for a grid
       my $response = $Grid_DHCP->options([$option1]);
```

Infoblox::Grid::Member::DHCP->options( )

Use this function to specify DHCP options at the grid member level on the Infoblox appliance. See Infoblox::Grid::Member::DHCP-

>options() for parameters and return values.

Example

```
#Construct a DHCP Option object
my $option2 = Infoblox::DHCP::Option->new(
              name  => "domain-name",
              value => "domainname.com"
);
# Configure a DHCP option for a grid member
my $response = $Grid_Member_DHCP->options([$option2]);
```

Infoblox::DHCP::Network->options( )

Use this function to specify DHCP options at the network level on the Infoblox appliance. See Infoblox::DHCP::Network->options() for parameters and return values.

Example

```
#Construct a DHCP Option object
my $option3 = Infoblox::DHCP::Option->new(
              name  => "broadcast",
              value => "255.0.0.0"
);
# Configure a DHCP option for a network
my $response = $Network_DHCP->options([$option3]);
```

Infoblox::DHCP::NetworkTemplate->options( )

Use this function to specify DHCP options at the network template level on the Infoblox appliance. See Infoblox::DHCP::NetworkTemplate->options() for parameters and return values.

Example

```
#Construct a DHCP Option object
my $option4 = Infoblox::DHCP::Option->new(
              name  => "broadcast",
              value => "2"
);
#Construct another DHCP Option object
my $option5 = Infoblox::DHCP::Option->new(
              name  => "routers",
              value => "1,2"
);
# Configure a DHCP options for a network template
my $response = $Network_Template_DHCP->options([$option4,$option5]);
```

Infoblox::DHCP::SharedNetwork->options( )

Use this function to specify DHCP options at the shared network level on the Infoblox appliance. See Infoblox::DHCP::SharedNetwork->options() for parameters and return values.

Example

```
#Construct a DHCP Option object
my $option6 = Infoblox::DHCP::Option->new(
              name  => "subnet-mask",
              value => "255.255.0.0"
);
# Configure a DHCP option for a shared network
my $response = $Shared_Network_DHCP->options([$option6]);
```

Infoblox::DHCP::FixedAddr->options( )

Use this function to specify DHCP options at the fixed address level on the Infoblox appliance. See Infoblox::DHCP::FixedAddr->options() for parameters and return values.

Example

```
#Construct a DHCP Option object
my $option7 = Infoblox::DHCP::Option->new(
              name  => "host-name",
              value => "host1.com"
);
# Configure a DHCP option for a fixed address
my $response = $Fixed_Address_DHCP->options([$option7]);
```

Infoblox::DHCP::FixedAddrTemplate->options( )

Use this function to specify DHCP options at the fixed address template level on the Infoblox appliance. See Infoblox::DHCP::FixedAddrTemplate->options() for parameters and return values.

Example

```
#Construct a DHCP Option object
my $option8 = Infoblox::DHCP::Option->new(
              name  => "broadcast",
              value => "2"
);
#Construct another DHCP Option object
my $option9 = Infoblox::DHCP::Option->new(
              name  => "routers",
              value => "1,2"
);
# Configure a DHCP options for a fixed address template
my $response = $Fixed_Address_Template_DHCP->options([$option8,$option9]);
```

Infoblox::DHCP::Range->options( )

Use this function to specify DHCP options at the DHCP range level on the Infoblox appliance. See Infoblox::DHCP::Range->options() for parameters and return values.

Example

```
#Construct a DHCP Option object
my $option10 = Infoblox::DHCP::Option->new(
              name  => "swap-server",
              value => "1.1.1.1"
);
# Configure a DHCP option for a DHCP range
my $response = $DHCP_Range->options([$option10]);
```

Infoblox::DHCP::RangeTemplate->options( )

Use this function to specify DHCP options at the range template level on the Infoblox appliance. See Infoblox::DHCP::RangeTemplate->options() for parameters and return values.

Example

```
#Construct a DHCP Option object
my $option11 = Infoblox::DHCP::Option->new(
              name  => "broadcast",
              value => "2"
);
#Construct another DHCP Option object
my $option12 = Infoblox::DHCP::Option->new(
              name  => "routers",
              value => "1,2"
);
# Configure a DHCP options for a range template
my $response = $Range_Template_DHCP->options([$option11,$option12]);
```

Infoblox::DHCP::Filter::Option->option_list( )

Use this function to specify DHCP options at the filter level on the Infoblox appliance. See Infoblox::DHCP::Filter::Option->option_list() for parameters and return values.

Example

```
#Construct a DHCP Option object
my $option13 = Infoblox::DHCP::Option->new(
              name  => "pop-server",
              value => "10.10.10.10"
);
# Configure a DHCP Option for a filter
my $response = $Filter_Option->option_list([$option13]);
```

METHODS

This section describes all the methods that can be used to configure and retrieve the attribute values of a DHCP option.

name( )

Use this method to set or retrieve the name of the DHCP option.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

Specify the name of the DHCP option in string format. Valid values are

```
  "subnet-mask",                              - DHCP Option   1              - ip-address
  "time-offset",                              - DHCP Option   2              - 32-bit signed integer
  "routers",                                  - DHCP Option   3              - array of ip-address |
array of offsets (for templates)
  "time-servers",                             - DHCP Option   4              - array of ip-address
  "ien116-name-servers",                      - DHCP Option   5              - array of ip-address
  "domain-name-servers",                      - DHCP Option   6              - array of ip-address
  "log-servers",                              - DHCP Option   7              - array of ip-address
  "cookie-servers",                           - DHCP Option   8              - array of ip-address
  "lpr-servers",                              - DHCP Option   9              - array of ip-address
  "impress-servers",                          - DHCP Option  10              - array of ip-address
  "resource-location-servers",                - DHCP Option  11              - array of ip-address
  "host-name",                                - DHCP Option  12              - string
  "boot-size",                                - DHCP Option  13              - 16-bit unsigned integer
  "merit-dump",                               - DHCP Option  14              - text
  "domain-name",                              - DHCP Option  15              - text
  "swap-server",                              - DHCP Option  16              - ip-address
  "root-path",                                - DHCP Option  17              - text
  "extensions-path",                          - DHCP Option  18              - text
  "ip-forwarding",                            - DHCP Option  19              - boolean
  "non-local-source-routing",                 - DHCP Option  20              - boolean
  "policy-filter",                            - DHCP Option  21              - array of ip-address |
ip-address
  "max-dgram-reassembly",                     - DHCP Option  22              - 16-bit unsigned integer
  "default-ip-ttl",                           - DHCP Option  23              - 8-bit unsigned integer
  "path-mtu-aging-timeout",                   - DHCP Option  24              - 32-bit unsigned integer
  "path-mtu-plateau-table",                   - DHCP Option  25              - array of 16-bit
unsigned integer
  "interface-mtu",                            - DHCP Option  26              - 16-bit unsigned integer
  "all-subnets-local",                        - DHCP Option  27              - boolean
  "broadcast",                                - DHCP Option  28              - ip-address | offset
(for templates)
  "perform-mask-discovery",                   - DHCP Option  29              - boolean
  "mask-supplier",                            - DHCP Option  30              - boolean
  "router-discovery",                         - DHCP Option  31              - boolean
  "router-solicitation-address",              - DHCP Option  32              - ip-address
  "static-routes",                            - DHCP Option  33              - array of ip-address |
ip-address
  "trailer-encapsulation",                    - DHCP Option  34              - boolean
  "arp-cache-timeout",                        - DHCP Option  35              - 32-bit unsigned integer
  "ieee802-3-encapsulation",                  - DHCP Option  36              - boolean
  "default-tcp-ttl",                          - DHCP Option  37              - 8-bit unsigned integer
  "tcp-keepalive-interval",                   - DHCP Option  38              - 32-bit unsigned integer
  "tcp-keepalive-garbage",                    - DHCP Option  39              - boolean
  "nis-domain",                               - DHCP Option  40              - text
  "nis-servers",                              - DHCP Option  41              - array of ip-address
  "ntp-servers",                              - DHCP Option  42              - array of ip-address
  "vendor-encapsulated-options",              - DHCP Option  43              - string
  "netbios-name-servers",                     - DHCP Option  44              - array of ip-address
  "netbios-dd-server",                        - DHCP Option  45              - array of ip-address
  "netbios-node-type",                        - DHCP Option  46              - 8-bit unsigned
integer(1,2,4,8)
  "netbios-scope",                            - DHCP Option  47              - text
  "font-servers",                             - DHCP Option  48              - array of ip-address
  "x-display-manager",                        - DHCP Option  49              - array of ip-address
  "dhcp-requested-address",                   - DHCP Option  50              - ip-address
  "lease-time",                               - DHCP Option  51              - 32-bit unsigned integer
  "dhcp-option-overload",                     - DHCP Option  52              - 8-bit unsigned integer
  "dhcp-message-type",                        - DHCP Option  53              - 8-bit unsigned integer
  "dhcp-server-identifier",                   - DHCP Option  54              - ip-address
  "dhcp-parameter-request-list",              - DHCP Option  55              - array of 8-bit unsigned
integer
  "dhcp-message",                             - DHCP Option  56              - text
  "dhcp-max-message-size",                    - DHCP Option  57              - 16-bit unsigned integer
  "dhcp-renewal-time",                        - DHCP Option  58              - 32-bit unsigned integer
  "dhcp-rebinding-time",                      - DHCP Option  59              - 32-bit unsigned integer
  "vendor-class-identifier",                  - DHCP Option  60              - string
  "dhcp-client-identifier",                   - DHCP Option  61              - string
  "nwip-domain",                              - DHCP Option  62              - string
  "nwip-suboptions",                          - DHCP Option  63              - string
  "nisplus-domain",                           - DHCP Option  64              - text
  "nisplus-servers",                          - DHCP Option  65              - array of ip-address
  "tftp-server-name",                         - DHCP Option  66              - text
  "bootfile-name",                            - DHCP Option  67              - text
  "mobile-ip-home-agent",                     - DHCP Option  68              - array of ip-address
  "smtp-server",                              - DHCP Option  69              - array of ip-address
  "pop-server",                               - DHCP Option  70              - array of ip-address
  "nntp-server",                              - DHCP Option  71              - array of ip-address
  "www-server",                               - DHCP Option  72              - array of ip-address
  "finger-server",                            - DHCP Option  73              - array of ip-address
  "irc-server",                               - DHCP Option  74              - array of ip-address
  "streettalk-server",                        - DHCP Option  75              - array of ip-address
  "streettalk-directory-assistance-server",   - DHCP Option  76              - array of ip-address
  "user-class",                               - DHCP Option  77              - text
  "slp-directory-agent",                      - DHCP Option  78              - boolean | array of ip-
address
  "slp-service-scope",                        - DHCP Option  79              - boolean | text
```

```
    "option-80",                          - DHCP Option  80             - custom defined
    "fqdn",                               - DHCP Option  81             - string
    "relay-agent-information",            - DHCP Option  82             - string
    "option-83",                          - DHCP Option  83             - custom defined
    "option-84",                          - DHCP Option  84             - custom defined
    "nds-servers",                        - DHCP Option  85             - array of ip-address
    "nds-tree-name",                      - DHCP Option  86             - string
    "nds-context",                        - DHCP Option  87             - string
    "option-88",                          - DHCP Option  88             - custom defined
    "bcms-controller-address",            - DHCP Option  89             - array of ip-address
    "option-90",                          - DHCP Option  90             - custom defined
    "client-last-transaction-time",       - DHCP Option  91             - 32-bit unsigned integer
    "associated-ip",                      - DHCP Option  92             - array of ip-address
    "option-93",                          - DHCP Option  93             - custom defined
    "option-94",                          - DHCP Option  94             - custom defined
    "option-95",                          - DHCP Option  95             - custom defined
    "option-96",                          - DHCP Option  96             - custom defined
    "option-97",                          - DHCP Option  97             - custom defined
    "uap-servers",                        - DHCP Option  98             - text
    "option-99",                          - DHCP Option  99             - custom defined
    "option-100",                         - DHCP Option 100             - custom defined
    "option-101",                         - DHCP Option 101             - custom defined
    "option-102",                         - DHCP Option 102             - custom defined
    "option-103",                         - DHCP Option 103             - custom defined
    "option-104",                         - DHCP Option 104             - custom defined
    "option-105",                         - DHCP Option 105             - custom defined
    "option-106",                         - DHCP Option 106             - custom defined
    "option-107",                         - DHCP Option 107             - custom defined
    "option-108",                         - DHCP Option 108             - custom defined
    "option-109",                         - DHCP Option 109             - custom defined
    "option-110",                         - DHCP Option 110             - custom defined
    "option-111",                         - DHCP Option 111             - custom defined
    "netinfo-server-address",             - DHCP Option 112             - array of ip-address
    "netinfo-server-tag",                 - DHCP Option 113             - text
    "default-url",                        - DHCP Option 114             - text
    "option-115",                         - DHCP Option 115             - custom defined
    "option-116",                         - DHCP Option 116             - custom defined
    "option-117",                         - DHCP Option 117             - custom defined
    "subnet-selection",                   - DHCP Option 118             - string
    "domain-search",                      - DHCP Option 119             - domain-list
    "option-120",                         - DHCP Option 120             - custom defined
    "option-121",                         - DHCP Option 121             - custom defined
    "option-122",                         - DHCP Option 122             - custom defined
    "option-123",                         - DHCP Option 123             - custom defined
    "vivco",                              - DHCP Option 124             - string
    "vivso",                              - DHCP Option 125             - string
    "option-126",                         - DHCP Option 126             - custom defined
    "option-127",                         - DHCP Option 127             - custom defined
    "option-128",                         - DHCP Option 128             - custom defined
    "option-129",                         - DHCP Option 129             - custom defined
    "option-130",                         - DHCP Option 130             - custom defined
    "option-131",                         - DHCP Option 131             - custom defined
    "option-132",                         - DHCP Option 132             - custom defined
    "option-133",                         - DHCP Option 133             - custom defined
    "option-134",                         - DHCP Option 134             - custom defined
    "option-135",                         - DHCP Option 135             - custom defined
    "option-136",                         - DHCP Option 136             - custom defined
    "option-137",                         - DHCP Option 137             - custom defined
    "option-138",                         - DHCP Option 138             - custom defined
    "option-139",                         - DHCP Option 139             - custom defined
    "option-140",                         - DHCP Option 140             - custom defined
    "option-141",                         - DHCP Option 141             - custom defined
    "option-142",                         - DHCP Option 142             - custom defined
    "option-143",                         - DHCP Option 143             - custom defined
    "option-144",                         - DHCP Option 144             - custom defined
    "option-145",                         - DHCP Option 145             - custom defined
    "option-146",                         - DHCP Option 146             - custom defined
    "option-147",                         - DHCP Option 147             - custom defined
    "option-148",                         - DHCP Option 148             - custom defined
    "option-149",                         - DHCP Option 149             - custom defined
    "option-150",                         - DHCP Option 150             - custom defined
    "option-151",                         - DHCP Option 151             - custom defined
    "option-152",                         - DHCP Option 152             - custom defined
    "option-153",                         - DHCP Option 153             - custom defined
    "option-154",                         - DHCP Option 154             - custom defined
    "option-155",                         - DHCP Option 155             - custom defined
    "option-156",                         - DHCP Option 156             - custom defined
    "option-157",                         - DHCP Option 157             - custom defined
    "option-158",                         - DHCP Option 158             - custom defined
    "option-159",                         - DHCP Option 159             - custom defined
    "option-160",                         - DHCP Option 160             - custom defined
    "option-161",                         - DHCP Option 161             - custom defined
    "option-162",                         - DHCP Option 162             - custom defined
    "option-163",                         - DHCP Option 163             - custom defined
    "option-164",                         - DHCP Option 164             - custom defined
    "option-165",                         - DHCP Option 165             - custom defined
    "option-166",                         - DHCP Option 166             - custom defined
    "option-167",                         - DHCP Option 167             - custom defined
    "option-168",                         - DHCP Option 168             - custom defined
    "option-169",                         - DHCP Option 169             - custom defined
    "option-170",                         - DHCP Option 170             - custom defined
    "option-171",                         - DHCP Option 171             - custom defined
    "option-172",                         - DHCP Option 172             - custom defined
    "option-173",                         - DHCP Option 173             - custom defined
    "option-174",                         - DHCP Option 174             - custom defined
    "option-175",                         - DHCP Option 175             - custom defined
    "option-176",                         - DHCP Option 176             - custom defined
```

```
    "option-177",                   - DHCP Option 177               - custom defined
    "option-178",                   - DHCP Option 178               - custom defined
    "option-179",                   - DHCP Option 179               - custom defined
    "option-180",                   - DHCP Option 180               - custom defined
    "option-181",                   - DHCP Option 181               - custom defined
    "option-182",                   - DHCP Option 182               - custom defined
    "option-183",                   - DHCP Option 183               - custom defined
    "option-184",                   - DHCP Option 184               - custom defined
    "option-185",                   - DHCP Option 185               - custom defined
    "option-186",                   - DHCP Option 186               - custom defined
    "option-187",                   - DHCP Option 187               - custom defined
    "option-188",                   - DHCP Option 188               - custom defined
    "option-189",                   - DHCP Option 189               - custom defined
    "option-190",                   - DHCP Option 190               - custom defined
    "option-191",                   - DHCP Option 191               - custom defined
    "option-192",                   - DHCP Option 192               - custom defined
    "option-193",                   - DHCP Option 193               - custom defined
    "option-194",                   - DHCP Option 194               - custom defined
    "option-195",                   - DHCP Option 195               - custom defined
    "option-196",                   - DHCP Option 196               - custom defined
    "option-197",                   - DHCP Option 197               - custom defined
    "option-198",                   - DHCP Option 198               - custom defined
    "option-199",                   - DHCP Option 199               - custom defined
    "option-200",                   - DHCP Option 200               - custom defined
    "option-201",                   - DHCP Option 201               - custom defined
    "option-202",                   - DHCP Option 202               - custom defined
    "option-203",                   - DHCP Option 203               - custom defined
    "option-204",                   - DHCP Option 204               - custom defined
    "option-205",                   - DHCP Option 205               - custom defined
    "option-206",                   - DHCP Option 206               - custom defined
    "option-207",                   - DHCP Option 207               - custom defined
    "option-208",                   - DHCP Option 208               - custom defined
    "option-209",                   - DHCP Option 209               - custom defined
    "option-210",                   - DHCP Option 210               - custom defined
    "option-211",                   - DHCP Option 211               - custom defined
    "option-212",                   - DHCP Option 212               - custom defined
    "option-213",                   - DHCP Option 213               - custom defined
    "option-214",                   - DHCP Option 214               - custom defined
    "option-215",                   - DHCP Option 215               - custom defined
    "option-216",                   - DHCP Option 216               - custom defined
    "option-217",                   - DHCP Option 217               - custom defined
    "option-218",                   - DHCP Option 218               - custom defined
    "option-219",                   - DHCP Option 219               - custom defined
    "option-220",                   - DHCP Option 220               - custom defined
    "option-221",                   - DHCP Option 221               - custom defined
    "option-222",                   - DHCP Option 222               - custom defined
    "option-223",                   - DHCP Option 223               - custom defined
    "option-224",                   - DHCP Option 224               - custom defined
    "option-225",                   - DHCP Option 225               - custom defined
    "option-226",                   - DHCP Option 226               - custom defined
    "option-227",                   - DHCP Option 227               - custom defined
    "option-228",                   - DHCP Option 228               - custom defined
    "option-229",                   - DHCP Option 229               - custom defined
    "option-230",                   - DHCP Option 230               - custom defined
    "option-231",                   - DHCP Option 231               - custom defined
    "option-232",                   - DHCP Option 232               - custom defined
    "option-233",                   - DHCP Option 233               - custom defined
    "option-234",                   - DHCP Option 234               - custom defined
    "option-235",                   - DHCP Option 235               - custom defined
    "option-236",                   - DHCP Option 236               - custom defined
    "option-237",                   - DHCP Option 237               - custom defined
    "option-238",                   - DHCP Option 238               - custom defined
    "option-239",                   - DHCP Option 239               - custom defined
    "option-240",                   - DHCP Option 240               - custom defined
    "option-241",                   - DHCP Option 241               - custom defined
    "option-242",                   - DHCP Option 242               - custom defined
    "option-243",                   - DHCP Option 243               - custom defined
    "option-244",                   - DHCP Option 244               - custom defined
    "option-245",                   - DHCP Option 245               - custom defined
    "option-246",                   - DHCP Option 246               - custom defined
    "option-247",                   - DHCP Option 247               - custom defined
    "option-248",                   - DHCP Option 248               - custom defined
    "option-249",                   - DHCP Option 249               - custom defined
    "option-250",                   - DHCP Option 250               - custom defined
    "option-251",                   - DHCP Option 251               - custom defined
    "option-252",                   - DHCP Option 252               - custom defined
    "option-253",                   - DHCP Option 253               - custom defined
    "option-254",                   - DHCP Option 254               - custom defined
```

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get name
my $name = $option->name();
#Modify name
$option->name("host-name");
```

num( )

Use this method to set or retrieve the numeric value of the DHCP option.

Note that specifying "num" without "name" is supported only for predefined options and custom options with names that haven't been modified.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

Specify the number of the DHCP option. It must be an integer ranging from 1 to 254.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get num
my $num = $option->num();
#Modify num
$option->num("12");
```

value( )

Use this method to set or retrieve the custom value of the DHCP option.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

Specify the data type of the DHCP option. Valid values are

```
  "boolean"                          - Specify "true" or "false" for boolean values.

  "ip-address"                       - Specify an IPv4 address. An IPv4 address is a 32-bit number
in dotted decimal notation.
                                       It consists of four 8-bit groups of decimal digits separated
                                       by decimal points (example: "192.168.1.2").

  "array of ip-address"              - Specify a list of IPv4 addresses, each separated by a comma.
                                       (example: "192.168.1.2, 10.11.12.13, 79.2.3.1")

  "string"                           - Specify a value in ASCII text string or
                                       a list of hexadecimal characters (0-9, a-f, or A-F) without
spaces and separated by colons.

  "text"                             - Specify a value in ASCII text string.

  "8-bit unsigned integer"           - Specify a numeric value between 0 and 255.

  "array of 8-bit unsigned integer"  - Specify a list of 8-bit unsigned integers, each separated by
a comma.

  "16-bit unsigned integer"          - Specify a numeric value between 0 and 65535.

  "array of 16-bit unsigned integer" - Specify a list of 16-bit unsigned integers, each separated by
a comma.

  "32-bit unsigned integer"          - Specify a numeric value between 0 and 4294967295.

  "array of 32-bit unsigned integer" - Specify a list of 32-bit unsigned integers, each separated by
a comma.

  "8-bit signed integer"             - Specify a numeric value between -128 and 127.

  "array of 8-bit signed integer"    - Specify a list of 8-bit signed integers, each separated by a
comma.

  "16-bit signed integer"            - Specify a numeric value between -32768 and 32767.

  "array of 16-bit signed integer"   - Specify a list of 16-bit signed integers, each separated by a
comma.

  "32-bit signed integer"            - Specify a numeric value between -2147483648 and 2147483647.

  "array of 32-bit signed integer"   - Specify a list of 32-bit signed integers, each separated by a
comma.

  "domain-list"                      - Specify a list of domain names, each double-quoted and
separated by a comma.
                                       (example: '"domain1.com","domain2.com","domain3.com"')
```

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get value
my $value = $option->value();
#Modify value
$option->value("abc.com");
```

vendor_class( )

Use this method to set an option space name as the vendor class value of a DHCP option or to retrieve an option space name.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

Specify the defined option space name in string format. The default option space name is "DHCP".

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get vendor_class
my $name = $option->vendor_class();
#Modify vendor_class
$option->vendor_class("DHCP");
```

SAMPLE CODE

The following sample code demonstrates the different functions that can be applied to a DHCP Option object such as add, modify, and remove. Also, this sample includes error handling for the operations.

#Preparation prior to a DHCP Option object insertion

```
#PROGRAM STARTS: Include all the modules that will be used
use strict;
use Infoblox;

#Create a session to the Infoblox appliance
my $session = Infoblox::Session->new(
               master   => "192.168.1.2", #appliance host ip
               username => "admin",       #appliance user login
               password => "infoblox"     #appliance password
);
unless ($session) {
       die("Construct session failed: ",
            Infoblox::status_code() . ":" . Infoblox::status_detail());
}
print "Session created successfully\n";
```

#Create a DHCP Option object

```
my $dhcp_option = Infoblox::DHCP::Option->new(
    name         => "subnet-mask",
    num          => "1",
    value        => "255.0.0.0",
    vendor_class => "DHCP"
);
unless($dhcp_option) {
       die("Construct DHCP option object failed: ",
            Infoblox::status_code() . ":" . Infoblox::status_detail());
}
print "DHCP option object created successfully\n";
```

#Get the Grid DHCP object and add a DHCP option to it

```
#Get the Grid DHCP object
my @retrieved_objs = $session->get(
    object => "Infoblox::Grid::DHCP",
```

```
    grid   => "Infoblox"
);
my $object = $retrieved_objs[0];
unless ($object) {
       die("Get Grid DHCP object failed: ",
            $session->status_code() . ":" . $session->status_detail());
}
print "Get Grid DHCP object found at least 1 matching entry\n";

#Apply the changes to the Grid DHCP object
$object->options([$dhcp_option]);

#Apply the changes
$session->modify($object)
    or die("Modify Grid DHCP object failed: ",
            $session->status_code() . ":" . $session->status_detail());
print "Grid DHCP object with DHCP option updated to Infoblox appliance successfully\n";
```

#Modify DHCP option

```
#Modify an existing option object

#Modifying the name of the DHCP option object.
$dhcp_option->name("host-name");

#Modifying the value of the DHCP option object.
$dhcp_option->value("infoblox.com");

#Modifying the number of the DHCP option object.
$dhcp_option->num("12");

#Apply changes to the Grid DHCP object.
$object->options([$dhcp_option]);

#Update Grid DHCP object through the Infoblox session.
$session->modify($object)
            or die("modify Grid DHCP object failed: ",
                $session->status_code() . ":" . $session->status_detail());
       print "Grid DHCP object with modified DHCP option updated to Infoblox appliance successfully\n";
```

#Remove DHCP option

```
#Apply changes to the Grid DHCP object.
$object->options([]);

#Update Grid DHCP object through the Infoblox session.
$session->modify($object)
            or die("modify Grid DHCP object failed: ",
                $session->status_code() . ":" . $session->status_detail());
       print "Removed DHCP option from Grid DHCP object successfully\n";

####PROGRAM ENDS####
```

AUTHOR

Infoblox Inc. http://www.infoblox.com/

SEE ALSO

Infoblox::Session, Infoblox::Session->get(), Infoblox::Session->modify(),Infoblox::Grid::DHCP, Infoblox::Grid::Member::DHCP, Infoblox::DHCP::Network, Infoblox::DHCP::NetworkTemplate, Infoblox::DHCP::SharedNetwork, Infoblox::DHCP::FixedAddr, Infoblox::DHCP::FixedAddrTemplate, Infoblox::DHCP::Range, Infoblox::DHCP::RangeTemplate, Infoblox::DHCP::Filter::Option

---

COPYRIGHT

Copyright (c) 2017 Infoblox Inc.

Infoblox::DHCP::MAC - DHCP MAC object.

- NAME
- DESCRIPTION
- CONSTRUCTOR
- SESSION METHODS
  - Infoblox::Session->add( )
  - Infoblox::Session->get( )
  - Infoblox::Session->modify( )
  - Infoblox::Session->remove( )
  - Infoblox::Session->search( )
- METHODS
  - authentication_time( )
  - comment( )
  - expired( )
  - expiration_time( )
  - extattrs( )
  - extensible_attributes( )
  - fingerprint( )
  - guest_custom_field1 ( )
  - guest_custom_field2 ( )
  - guest_custom_field3 ( )
  - guest_custom_field4 ( )
  - guest_email ( )
  - guest_first_name ( )
  - guest_last_name ( )
  - guest_middle_name ( )
  - guest_phone ( )
  - filter( )
  - mac( )
  - is_registered_user( )
  - reserved_for_infoblox( )
  - username( )
- SAMPLE CODE
- AUTHOR
- SEE ALSO
- COPYRIGHT

NAME

Infoblox::DHCP::MAC - DHCP MAC object.

DESCRIPTION

A MAC Address is a unique hardware identifier. The Infoblox device can filter address requests by the MAC address of a requesting hosts. The filter instructs the Infoblox device either to grant or deny an address request if the requesting host matches the filter. Also, it is possible configure specific MAC addresses within a filter to expire after a certain amount of time has passed.

CONSTRUCTOR

```
 my $mac = Infoblox::DHCP::MAC->new(
       filter                => $string,        #Required
       mac                   => $mac_addr,      #Required
       authentication_time   => $num,           #Optional / Default is empty
       comment               => $string,        #Optional / Default is empty
       expiration_time       => $num,           #Optional / Default is 0 for never expire
       extattrs              => { $string => $extattr, ... },      #Optional / Default is undefined
       extensible_attributes => { $string => $string | $num, $string => [ $string | $num, ... ], ... },
#Optional / Default is undefined
       guest_custom_field1   => $string,        #Optional / Default is empty
       guest_custom_field2   => $string,        #Optional / Default is empty
       guest_custom_field3   => $string,        #Optional / Default is empty
       guest_custom_field4   => $string,        #Optional / Default is empty
       guest_email           => $string,        #Optional / Default is empty
       guest_first_name      => $string,        #Optional / Default is empty
       guest_last_name       => $string,        #Optional / Default is empty
       guest_middle_name     => $string,        #Optional / Default is empty
       guest_phone           => $string,        #Optional / Default is empty
       is_registered_user    => "true" | "false", #Optional / Default is "true"
       reserved_for_infoblox => $string,        #Optional / Default is "false"
```

```
        username              => $string,        #Optional / Default is empty
);
```

You cannot set both extattrs and extensible_attributes attributes at the same time.

SESSION METHODS

This section describes all the methods in Infoblox::Session module that you can apply to a DHCP MAC object.

Infoblox::Session->add( )

Use this method to add a DHCP MAC object to the Infoblox appliance. See Infoblox::Session->add() for parameters and return values.

Example

```
#Construct a DHCP MAC object
my $mac = Infoblox::DHCP::MAC->new(
            "mac"                  => "00:00:00:00:00:03",
            "filter"               => "new",
            "comment"              => "dhcp mac object",
            "expiration_time"      => "0",
            "username"             => "infoblox_user",
            "authentication_time"  => "0",
            "reserved_for_infoblox" => "For Infoblox Use only",
);
#Submit for addition
my $response = $session->add($mac)
```

Infoblox::Session->get( )

Use this method to retrieve all the matching objects from the Infoblox appliance. See Infoblox::Session->get() for parameters and return values.

Key References

```
Apply the following attributes to get a specific DHCP MAC object:

 mac    - Optional. MAC address of the DHCP MAC object.
 filter - Optional. MAC filter name.
 comment  - Optional. A comment in string format.
 username - Optional. Username of the DHCP MAC object.
 fingerprint - Optional. DHCP fingerprint description.
 extattrs - Optional. A hash reference containing extensible attributes.
 extensible_attributes - Optional. A hash reference containing extensible attributes.
```

Example

```
#Get DHCP MAC object through the session
my @retrieved_objs = $session->get(
    object => "Infoblox::DHCP::MAC",
    mac    => "00:00:00:00:00:03",
    filter => "new",
);

# get all objects with a given extensible attribute
my @retrieved_objs = $session->get(
    object               => "Infoblox::DHCP::MAC",
    extensible_attributes => { 'Site' => 'Santa Clara' }
);
```

Infoblox::Session->modify( )

Use this method to modify a DHCP MAC object in the Infoblox appliance. See Infoblox::Session->modify() for parameters and return values.

Example

```
#Use method to modify the expiration time.
$mac->expiration_time("1234567890");
#Submit modification
my $response = $session->modify( $mac );
```

Infoblox::Session->remove( )

Use this method to remove a DHCP MAC object from the Infoblox appliance. See Infoblox::Session->remove() for parameters and return values.

To remove a specific object, first use `get()` or `search()` to retrieve the specific DHCP MAC object, and then submit this object for removal.

Example

```
#Get DHCP MAC object through the session
my @retrieved_objs = $session->get(
    object     => "Infoblox::DHCP::MAC",
    mac        => "00:00:00:00:00:03",
    filter     => "new",
);
#Find the desired object from the retrieved list.
my $desired_mac = $retrieved_objs[0];
#Submit for removal
my $response = $session->remove( $desired_mac );
```

Infoblox::Session->search( )

Use this method to search for DHCP MAC objects in the Infoblox appliance. See Infoblox::Session->search() for parameters and return values.

Key References

```
Apply the following attributes to search for a specific DHCP MAC Address object:

 comment  - Optional. A comment in string format (regular expression).
 mac      - Optional. MAC address of the DHCP MAC object (regular expression).
 username - Optional. Username of the DHCP MAC object (regular expression).
 filter   - Optional. MAC address filter name (regular expression).
 fingerprint - Optional. DHCP fingerprint description (regular expression).
 extattrs - Optional. A hash reference containing extensible attributes.
 extensible_attributes - Optional. A hash reference containing extensible attributes.
```

For more information about searching extensible attributes, see Infoblox::Grid::ExtensibleAttributeDef/Searching Extensible Attributes.

Example

```
# search for all DHCP MAC objects that are specified in the "new" MAC filter.
my @retrieved_objs = $session->search(
    object => "Infoblox::DHCP::MAC",
    mac    => "00:00:00:00:00.*",
    filter => "new"
);

# search all DHCP MAC associations with the extensible attribute 'Site'
my @retrieved_objs = $session->search(
   object => "Infoblox::DHCP::MAC",
   extensible_attributes => { 'Site' => 'Santa Clara' });
```

METHODS

This section describes all the methods that you can use to set and retrieve the attribute values of a DHCP MAC object.

authentication_time( )

Use this method to set or retrieve the authentication time. Indicates the time when the MAC address was first authenticated.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

The absolute UNIX time (seconds since UNIX epoch) when the address was authenticated. The maximum value can extend upto 4294967295.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get authentication_time
my $authentication_time = $mac->authentication_time();
#Modify authentication_time
$mac->authentication_time("1234567890");
```

comment( )

Use this method to set or retrieve a descriptive comment.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

Desired comment in string format with a maximum of 256 bytes.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get comment
my $comment = $mac->comment();
#Modify comment
$mac->comment("Modified DHCP MAC comment");
```

expired( )

Use this method to retrieve the expired flag of the DHCP MAC object .

Parameter

None

Returns

The method returns the attribute value.

Example

```
#Get expired
my $expired = $mac->expired();
```

expiration_time( )

Use this method to set or retrieve the expiration time of the DHCP MAC object.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

By default, the MAC address never expires; otherwise, it is the absolute UNIX time (seconds since UNIX epoch) when the address expires. The maximum value can extend upto 4294967295.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get expiration_time
my $expiration_time = $mac->expiration_time();
#Modify expiration_time
$mac->expiration_time("1234567890");
```

extattrs( )

Use this method to set or retrieve the extensible attributes associated with a DHCP MAC object.

Parameter

Valid value is a hash reference containing the names of extensible attributes and their associated values (Infoblox::Grid::Extattr objects).

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get extattrs
my $ref_extattrs = $mac->extattrs();
#Modify extattrs
$mac->extattrs({ 'Site' => $extattr1, 'Administrator' => $extattr2 });
```

extensible_attributes( )

Use this method to set or retrieve the extensible attributes associated with a DHCP MAC object.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

For valid values for extensible attributes, see Infoblox::Grid::ExtensibleAttributeDef/Extensible Attribute Values.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get extensible attributes
my $ref_extensible_attributes = $mac->extensible_attributes();
#Modify extensible attributes
$mac->extensible_attributes({ 'Site' => 'Santa Clara', 'Administrator' => [ 'Peter', 'Tom' ] });
```

fingerprint( )

Use this method to retrieve the description of the DHCP fingerprint of a DHCP MAC object. This is a read-only attribute.

Parameter

none

Returns

The method returns the attribute value.

Example

```
#Get fingerprint
my $fingerprint = $mac->fingerprint();
```

guest_custom_field1 ( )

Use this method to set or retrieve the value in Custom Field 1 of the guest registration page of this DHCP MAC object.

Parameter

The valid value is a string. The default value is undefined. The maximum length is 1023 bytes.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Getting guest_custom_field1
my $guest_custom_field1 = $dhcp->guest_custom_field1();
#Modifying guest_custom_field1
$dhcp->guest_custom_field1('Custom value 1');
```

guest_custom_field2 ( )

Use this method to set or retrieve the value in Custom Field 2 of the guest registration page of this DHCP MAC object.

Parameter

The valid value is a string. The default value is undefined. The maximum length is 1023 bytes.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Getting guest_custom_field2
my $guest_custom_field2 = $dhcp->guest_custom_field2();
#Modifying guest_custom_field2
$dhcp->guest_custom_field2('Custom value 2');
```

guest_custom_field3 ( )

Use this method to set or retrieve the value in Custom Field 3 of the guest registration page of this DHCP MAC object.

Parameter

The valid value is a string. The default value is undefined. The maximum length is 1023 bytes.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Getting guest_custom_field3
my $guest_custom_field3 = $dhcp->guest_custom_field3();
#Modifying guest_custom_field3
$dhcp->guest_custom_field3('Custom value 3');
```

guest_custom_field4 ( )

Use this method to set or retrieve the value in Custom Field 4 of the guest registration page of this DHCP MAC object.

Parameter

The valid value is a string. The default value is undefined. The maximum length is 1023 bytes.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Getting guest_custom_field4
my $guest_custom_field4 = $dhcp->guest_custom_field4();
#Modifying guest_custom_field4
$dhcp->guest_custom_field4('Custom value 4');
```

guest_email ( )

Use this method to set or retrieve the email address in the Email field of the guest registration page of this DHCP MAC object.

Parameter

The valid value is a string. The default value is undefined. The maximum length is 128 bytes.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Getting guest_email
my $guest_email = $dhcp->guest_email();
#Modifying guest_email
$dhcp->guest_email('user@domain.com');
```

guest_first_name ( )

Use this method to set or retrieve the entry in the First Name field of the guest registration page of this DHCP MAC object.

Parameter

The valid value is a string. The default value is undefined. The maximum length is 1023 bytes.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Getting guest_first_name
my $guest_first_name = $dhcp->guest_first_name();
#Modifying guest_first_name
$dhcp->guest_first_name('John');
```

guest_last_name ( )

Use this method to set or retrieve the entry in the Last Name field of the guest registration page of this DHCP MAC object.

Parameter

The valid value is a string. The default value is undefined. The maximum length is 1023 bytes.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Getting guest_last_name
my $guest_last_name = $dhcp->guest_last_name();
#Modifying guest_last_name
$dhcp->guest_last_name('Doe');
```

guest_middle_name ( )

Use this method to set or retrieve the entry in the Middle Name field of the guest registration page of this DHCP MAC object.

Parameter

The valid value is a string. The default value is undefined. The maximum length is 1023 bytes.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Getting guest_middle_name
my $guest_middle_name = $dhcp->guest_middle_name();
#Modifying guest_middle_name
$dhcp->guest_middle_name('Robert');
```

guest_phone ( )

Use this method to set or retrieve the entry in the Phone field of the guest registration page of this DHCP MAC object.

Parameter

The valid value is a string. The default value is undefined. The maximum length is 1023 bytes.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Getting guest_phone
my $guest_phone = $dhcp->guest_phone();
#Modifying guest_phone
$dhcp->guest_phone('1-555-555-5555');
```

filter( )

Use this method to set or retrieve the filter value of the DHCP MAC object .

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

Name of the DHCP MAC filter. Maximum length upto 1024 bytes is supported.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get filter
my $filter = $mac->filter();
#Modify filter
$mac->filter("filter1");
```

mac( )

Use this method to set or retrieve the MAC address of the DHCP MAC object .

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

MAC Address of the DHCP MAC object. Format for the hexadecimal strings for MAC addresses and vendor prefixes with colons or dashes. Both of the following formats are acceptable: 11:11:11:11:11:11 and 11-11-11-11-11-11.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get mac
my $mac_value = $mac->mac();
#Modify mac
$mac->mac("00:00:00:11:11:11");
```

is_registered_user( )

Use this method to set or retrieve the is_registered_user flag of the DHCP MAC object. If this flag is "false", then this DHCP MAC object represents a "Guest" registration.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

Specify "true" to set the is_registered_user flag or "false" to deactivate/unset it. The default value for this field is "true".

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
# Get is_registered_user
my $is_registered_user = $mac->is_registered_user()
# Modify is_registered_user
$mac->is_registered_user("true");
```

reserved_for_infoblox( )

Set this flag to reserve the MAC address for an Infoblox appliance.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

Reserved to write comment related to this particular MAC address. Comment cannot exceed 1024 bytes long.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get reserved_for_infoblox
my $reserved_for_infoblox = $mac->reserved_for_infoblox();
#Modify reserved_for_infoblox
$mac->reserved_for_infoblox("reserved for Infoblox");
```

username( )

Use this method to set or retrieve the user name of the DHCP MAC object .

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

User name of the DHCP MAC object. Used for authenticated DHCP purposes. Maximum of 128 characters supported.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get username
my $username = $mac->username();
#Modify username
$mac->username("user_infoblox");
```

SAMPLE CODE

The following sample code demonstrates the different functions that can be applied to an object, such as add, search, modify, and remove. This sample also includes error handling for the operations.

#Preparation prior to a DHCP MAC object insertion

```
#PROGRAM STARTS: Include all the modules that will be used
use strict;
use Infoblox;

#Create a session to the Infoblox appliance

my $session = Infoblox::Session->new(
               master   => "192.168.1.2", #appliance host ip
               username => "admin",       #appliance user login
               password => "infoblox"     #appliance password
);

unless ($session) {
       die("Construct session failed: ",
               Infoblox::status_code() . ":" . Infoblox::status_detail());
}
print "Session created successfully\n";

#Create a DHCP MAC Filter object
my $macfilter = Infoblox::DHCP::Filter::MAC->new ( "name" => "mac_filter");

unless($macfilter) {
     die("Construct mac filter failed: ",
           Infoblox::status_code() . ":" . Infoblox::status_detail());
}
print "MAC Filter object created successfully\n";

#Verify if the DHCP MAC Filter exists in the Infoblox appliance
my $object = $session->get(object => "Infoblox::DHCP::Filter::MAC", name => "mac_filter");
unless ($object) {
   print "mac filter does not exist on server, safe to add the mac filter\n";
   $session->add($macfilter)
      or die
```

```
            ("Add mac filter failed: ",
             $session->status_code() . ":" . $session->status_detail());
}
print "DHCP MAC Filter added successfully\n";
```

#Create a DHCP MAC object

```
my $mac = Infoblox::DHCP::MAC->new(
          "mac"                  => "00:00:00:00:00:01",
          "filter"               => "mac_filter",
          "comment"              => "dhcp mac object",
          "expiration_time"      => "0",
          "username"             => "infoblox_user",
          "authentication_time"  => "0",
          "reserved_for_infoblox" => "For Infoblox Use only",
);

unless($mac) {
          die("Construct MAC object failed: ",
                Infoblox::status_code() . ":" . Infoblox::status_detail());
}
print "MAC Address object created successfully\n";

#Add the MAC Address object into the Infoblox appliance through a session
$session->add($mac)
     or die("Add MAC Address object failed: ",
          $session->status_code() . ":" . $session->status_detail());
print "DHCP MAC Address object added successfully\n";
```

#Search for a specific DHCP MAC Address object

```
#Search for all DHCP MAC address objects
my @retrieved_objs = $session->search(
    object => "Infoblox::DHCP::MAC",
    filter => "mac_filter" );
my $object = $retrieved_objs[0];

unless ($object) {
       die("Search for MAC Address object failed: ",
              $session->status_code() . ":" . $session->status_detail());
}
print "Search MAC Address object found at least 1 matching entry\n";
```

#Get and modify a MAC Address object

```
#Get MAC Address object through the session
my @retrieved_objs = $session->get(
    object => "Infoblox::DHCP::MAC",
    mac    => "00:00:00:00:00:01",
);
my $object = $retrieved_objs[0];
unless ($object) {
    die("Get MAC Address object failed: ",
        $session->status_code() . ":" . $session->status_detail());
}
print "Get MAC Address object found at least 1 matching entry\n";

#Modify one of the attributes of the obtained MAC Address object
$object->expiration_time("1234567890");

#Apply the changes
$session->modify($object)
    or die("Modify MAC Address object failed: ",
          $session->status_code() . ":" . $session->status_detail());
print "MAC Address object modified successfully \n";
```

#Remove a MAC Address object

```
#Get MAC Address object through the session
my @retrieved_objs = $session->get(
    object => "Infoblox::DHCP::MAC",
    mac    => "00:00:00:00:00:01",
    filter => "mac_filter",
);
my $object = $retrieved_objs[0];
unless ($object) {
    die("Get MAC Address object failed: ",
```

```
        $session->status_code() . ":" . $session->status_detail());
}
print "Get MAC Address object found at least 1 matching entry\n";

#Submit the object for removal
$session->remove($object)
    or die("Remove MAC Address object failed: ",
        $session->status_code() . ":" . $session->status_detail());
print "MAC Address object removed successfully \n";

#Submit the MAC Address Filter object for removal
$session->remove($macfilter)
    or die("Remove MAC Address Filter object failed: ",
        $session->status_code() . ":" . $session->status_detail());
print "MAC Address Filter object removed successfully \n";

####PROGRAM ENDS####
```

AUTHOR

Infoblox Inc. http://www.infoblox.com/

SEE ALSO

Infoblox::DHCP::Filter::MAC, Infoblox::DHCP::Filter::RelayAgent, Infoblox::DHCP::Lease,Infoblox::Session->add(), Infoblox::Session->get(), Infoblox::Session->modify(), Infoblox::Session->remove(), Infoblox::Session->search(), Infoblox::Session

---

COPYRIGHT

Copyright (c) 2017 Infoblox Inc.

Infoblox::DHCP::Filter::RelayAgent - DHCP relay agent filter object

- NAME
- DESCRIPTION
- CONSTRUCTOR
- SESSION METHODS
  - Infoblox::Session->add( )
  - Infoblox::Session->get( )
  - Infoblox::Session->modify( )
  - Infoblox::Session->remove( )
  - Infoblox::Session->search( )
- METHODS
  - name( )
  - comment( )
  - circuit_id_name( )
  - circuit_id_substring_length( )
  - circuit_id_substring_offset( )
  - extattrs( )
  - extensible_attributes( )
  - remote_id_name( )
  - remote_id_substring_length( )
  - remote_id_substring_offset( )
  - is_circuit_id( )
  - is_circuit_id_substring( )
  - is_remote_id( )
  - is_remote_id_substring( )
- SAMPLE CODE
- AUTHOR
- SEE ALSO
- COPYRIGHT

NAME

Infoblox::DHCP::Filter::RelayAgent - DHCP relay agent filter object

---

DESCRIPTION

The Infoblox appliance can screen address requests through relay agent filters (DHCP option 82) that assist the agents in forwarding address assignments across the proper circuit. When a relay agent receives the DHCPDISCOVER message, it can add one or two agent IDs in the DHCP option 82 suboption fields to the message. If the agent ID strings match those defined in a relay agent filter applied to a DHCP address range, the Infoblox appliance either assigns addresses from that range or denies the request (based on previously configured parameters; that is, the Grant lease and Deny lease parameters).

CONSTRUCTOR

```
  my $relayagentFilter = Infoblox::DHCP::Filter::RelayAgent->new(
                  name                          => $string | $ipv4addr,
# Required
                  comment                       => $string,
# Optional / Default is empty
                  circuit_id_name               => $string,
# Optional / Default is empty
                  extattrs                      => { $string => $extattr, ... },
# Optional / Default is undefined
                  extensible_attributes         => { $string => $string | $num, $string => [ $string | $num,
... ], ... }, # Optional / Default is undefined
                  remote_id_name                => $string,
# Optional / Default is empty
                  is_circuit_id                 => $string,
# Optional / Default is empty
                  is_remote_id                  => $string,
# Optional / Default is empty
                  is_circuit_id_substring       => 'true' | 'false',
# Optional / Default is 'false'
                  circuit_id_substring_offset => $uint,
# Optional / Default is undefined
                  circuit_id_substring_length => $uint,
# Optional / Default is undefined
                  is_remote_id_substring        => 'true' | 'false',
# Optional / Default is 'false'
                  remote_id_substring_offset  => $uint,
# Optional / Default is undefined
                  remote_id_substring_length  => $uint,
```

```
# Optional / Default is undefined
                );
```

You cannot set both extattrs and extensible_attributes attributes at the same time.

SESSION METHODS

This section describes all the methods in Infoblox::Session module that can be applied to a DHCP relay agent filter object.

Infoblox::Session->add( )

Use this method to add a DHCP relay agent filter object to the Infoblox appliance. See Infoblox::Session->add() for parameters and return values.

Example

```
#Construct an object
my  $relayagentFilter = Infoblox::DHCP::Filter::RelayAgent->new(
       name             => "relayagent_filter",
       circuit_id_name  => "circuit",
       remote_id_name   => "remote"
        );

# Submit for adding a relay agent filter
my $response = $session->add( $relayagentFilter );
```

Infoblox::Session->get( )

Use this method to retrieve all the matching DHCP relay agent filter objects from the Infoblox appliance. See Infoblox::Session->get() for parameters and return values.

Key References

```
Apply the following attributes to get a specific DHCP relay agent filter object:

 name - Required. A relay agent filter name in string format or in ipv4address format.
 extattrs     - Optional. A hash reference containing extensible attributes.
 extensible_attributes - Optional. A hash reference containing extensible attributes.
```

Example

```
my  @retrieved_objs = $session->get(
    object => "Infoblox::DHCP::Filter::RelayAgent" ,
    name   => "relayagent_filter",
);

# get all objects with a given extensible attribute
my @retrieved_objs = $session->get(
    object               => "Infoblox::DHCP::Filter::RelayAgent",
    extensible_attributes => { 'Site' => 'Santa Clara' }
);
```

Infoblox::Session->modify( )

Use this method to modify a DHCP relay agent filter object in the Infoblox appliance. See Infoblox::Session->modify() for parameters and return values.

Example

```
#Use method to modify circuit id name method.
$object->circuit_id_name("diff_Circuit");
# Submit modification
my $response = $session->modify( $object );
```

Infoblox::Session->remove( )

Use this method to remove a DHCP relay agent filter object from the Infoblox appliance. See Infoblox::Session->remove() for parameters and return values.

To remove a specifc object, first use `get()` or `search()` to retrieve the specific object, then submit this object for removal.

Example

```
# Get the object under the same name
my @retrieved_objs = $session->get(
           object  => "Infoblox::DHCP::Filter::RelayAgent" ,
```

```
            "name"   => "relayagent_filter",
        );

    # find the desired object from retrieved list.
    my $desired_obj = $retrieved_objs[0];

    # Submit for removal
    my $response = $session->remove( $desired_obj);
```

Infoblox::Session->search( )

Use this method to search for a DHCP relay agent filter object in the Infoblox appliance. See Infoblox::Session->search() for parameters and return values.

Key References

```
Apply the following attributes to search for a DHCP Network object:

 name - Required. A relay agent filter name in string format (regular expression).
 extattrs     - Optional. A hash reference containing extensible attributes.
 extensible_attributes - Optional. A hash reference containing extensible attributes.
 comment - Optional . A comment in string format (regular expression).
```

For more information about searching extensible attributes, see Infoblox::Grid::ExtensibleAttributeDef/Searching Extensible Attributes.

Example

```
# search for all DHCP relayagent objects
     my @retrieved_objs = $session->search(
               object   => "Infoblox::DHCP::Filter::RelayAgent",
               name     => 'relayagent\..*',
               );

# search all DHCP relay agent associations with the extensible attribute 'Site'
my @retrieved_objs = $session->search(
   object => "Infoblox::DHCP::Filter::RelayAgent",
   extensible_attributes => { 'Site' => 'Santa Clara' });
```

METHODS

This section describes all the methods that can be used to configure and retrieve the attribute values of a DHCP relay agent filter object.

name( )

Use this method to set or retrieve the name of a DHCP relay agent filter object. The name can be an IP address or name of the router acting as relay agent.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

Specify the name in string format or in ipv4addr format. An IPv4 address is a 32-bit number in dotted decimal notation. It consists of four 8-bit groups of decimal digits separated by decimal points (example: 192.168.1.2). The default value for this field is empty.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
# Get name
my $name = $relayagentFilter->name()
# Modify name
$relayagentFilter->name("relay_filter");
```

comment( )

Use this method to set or retrieve the descriptive comment of a DHCP relay agent filter object.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

Desired comment in string format with a maximum of 256 bytes.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get comment
my $comment = $relayagentFilter->comment();
#Modify comment
$relayagentFilter->comment("Modified DHCP relay agent filter object comment");
```

circuit_id_name( )

Use this method to set or retrieve a circuit_id_name of a DHCP relay agent filter object. This filter identifies the circuit between the remote host and the relay agent. For example, the identifier can be the ingress interface number of the circuit access unit, perhaps concatenated with the unit ID number and slot number. Also, the circuit ID can be an ATM virtual circuit ID or cable data virtual circuit ID.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

Specify circuit id name in string format. example: Text String: 001122334455 Binary String: \x00\x11\x22\x33\x44\x55. The default value for this field is empty.

To reset this value, set it to an empty string together with the is_circuit_id flag (anything except "Matches_Value").

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
# Get circuit_id_name
my $circuit_id_name = $relayagentFilter->circuit_id_name();
# Modify circuit_id_name
$relayagentFilter->circuit_id_name("001122334455");
# Set is_circuit_id to "Not_Set"
$relayagentFilter->is_circuit_id("Not_Set");
# Set circuit_id_name to empty string
$relayagentFilter->circuit_id_name("");
```

circuit_id_substring_length( )

Use this method to set or retrive the circuit ID substring length.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

The valid value is an unsigned integer.

Returns

If you specified a parameter, the method returns 'true' when the modification succeeds, and returns 'false' when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#get circuit_id_substring_length value
my $circuit_id_substring_length = $relayagentfilter->circuit_id_substring_length();

#modify circuit_id_substring_length value
$relayagentfilter->circuit_id_substring_length(10);
```

circuit_id_substring_offset( )

Use this method to set or retrive the circuit ID substring offset.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

The valid value is an unsigned integer.

Returns

If you specified a parameter, the method returns 'true' when the modification succeeds, and returns 'false' when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#get circuit_id_substring_offset value
my $circuit_id_substring_offset = $relayagentfilter->circuit_id_substring_offset();

#modify circuit_id_substring_offset value
$relayagentfilter->circuit_id_substring_offset(10);
```

extattrs( )

Use this method to set or retrieve the extensible attributes associated with a DHCP relay agent filter object.

Parameter

Valid value is a hash reference containing the names of extensible attributes and their associated values (Infoblox::Grid::Extattr objects).

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get extattrs
my $ref_extattrs = $relayagentFilter->extattrs();
#Modify extattrs
$relayagentFilter->extattrs({ 'Site' => $extattr1, 'Administrator' => $extattr2 });
```

extensible_attributes( )

Use this method to set or retrieve the extensible attributes associated with a DHCP relay agent filter object.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

For valid values for extensible attributes, see Infoblox::Grid::ExtensibleAttributeDef/Extensible Attribute Values.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get extensible attributes
my $ref_extensible_attributes = $relayagentFilter->extensible_attributes();
#Modify extensible attributes
$relayagentFilter->extensible_attributes({ 'Site' => 'Santa Clara', 'Administrator' => [ 'Peter',
'Tom' ] });
```

remote_id_name( )

Use this method to set or retrieve a remote ID name attribute of a relay agent filter object. This filter identifies the remote host. The remote ID name can represent many different things such as the caller ID telephone number for a dial-up connection, a user name for logging in to the ISP, a modem ID, etc. When the remote ID name is defined on the relay agent, the DHCP server will have a trusted relationship to identify the remote host. The remote ID name is considered as a trusted identifier.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

Specify remote_id_name in string format. example: Text String: 001122334455 Binary String: \x00\x11\x22\x33\x44\x55. The default value for this field is empty.

To reset this value, set it to an empty string together with the is_remote_id flag (anything except "Matches_Value").

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
# Get remote_id_name
my $remote_id_name = $relayagentFilter->remote_id_name();
# Modify remote_id_name
$relayagentFilter->remote_id_name("\x00\x11\x22\x33\x44\x55");
# Set is_remote_id to "Not_Set"
$relayagentFilter->is_remote_id("Not_Set");
# Set remote_id_name to empty string
$relayagentFilter->remote_id_name("");
```

remote_id_substring_length( )

Use this method to set or retrive the remote ID substring length.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

The valid value is an unsigned integer.

Returns

If you specified a parameter, the method returns 'true' when the modification succeeds, and returns 'false' when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#get remote_id_substring_length value
my $remote_id_substring_length = $relayagentfilter->remote_id_substring_length();

#modify remote_id_substring_length value
$relayagentfilter->remote_id_substring_length(10);
```

remote_id_substring_offset( )

Use this method to set or retrive the remote ID substring offset.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

The valid value is an unsigned integer.

Returns

If you specified a parameter, the method returns 'true' when the modification succeeds, and returns 'false' when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#get remote_id_substring_offset value
my $remote_id_substring_offset = $relayagentfilter->remote_id_substring_offset();

#modify remote_id_substring_offset value
$relayagentfilter->remote_id_substring_offset(10);
```

is_circuit_id( )

Use this method to set or retrieve the is_circuit_id flag of a DHCP relay agent filter object. The possible values for the is_circuit_id flag are Any, Not_Set, Matches_Value. The circuit_id value takes effect only if the value is Matches_Value.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

Specify is_circuit_id in string format. example: Text String: Any

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
# Get is_circuit_id
my $is_circuit_id = $relayagentFilter->is_circuit_id();
# Modify is_circuit_id
$relayagentFilter->is_circuit_id("Any");
```

is_circuit_id_substring( )

Use this method to set or retrive the flag that indicates whether the substring of circuit ID, instead of the full circuit ID, is matched.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

Specify 'true' to enable the circuit ID substring match or 'false' to disable it. The default value is 'false'.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
# Get is_circuit_id_substring
my $is_circuit_id_substring = $relayagentFilter->is_circuit_id_substring();
# Modify is_circuit_id_substring
$relayagentFilter->is_circuit_id_substring("true");
```

is_remote_id( )

Use this method to set or retrieve the is_remote_id flag of a DHCP relay agent filter object. The possible values for the is_remote_id flag are Any, Not_Set, Matches_Value. The remote_id value takes effect only if the value is Matches_Value.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

Specify is_remote_id in string format. example: Text String: Not_Set

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
# Get is_remote_id
my $is_remote_id = $relayagentFilter->is_remote_id();
# Modify is_remote_id
$relayagentFilter->is_remote_id("Not_Set");
```

is_remote_id_substring( )

Use this method to set or retrive the flag that indicates whether the substring of the remote ID, instead of the full remote ID, is matched.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

Specify 'true' to enable remote id substring match or 'false' to disable it. The default value is 'false'.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
# Get is_remote_id_substring
my $is_remote_id_substring = $relayagentFilter->is_remote_id_substring();
# Modify is_remote_id_substring
```

```
$relayagentFilter->is_remote_id_substring("true");
```

SAMPLE CODE

The following sample code demonstrates the different functions that can be applied to an object such as add, search, modify, and remove. Also, this sample includes error handling for the operations.

#Preparation prior to a DHCP relay agent filter object insertion

```
#PROGRAM STARTS: Include all the modules that will be used
use strict;
use Infoblox;

#Create a session to the Infoblox appliance

my $session = Infoblox::Session->new(
    master   => "192.168.1.2",
    username => "admin",
    password => "infoblox"
);
unless ($session) {
   die("Construct session failed: ",
       Infoblox::status_code() . ":" . Infoblox::status_detail());
}
print "Session created successfully\n";

#Create a FilterRule relayagent filter object>

my $filter = Infoblox::DHCP::FilterRule::RelayAgent->new(
               filter_name => "relayagent_filter",
               permission => "grant" );
```

#Create a DHCP relay agent filter object

```
my  $relayagentfilter = Infoblox::DHCP::Filter::RelayAgent->new(

       name             => "relayagent_filter",
       circuit_id_name  => "circuit",
       remote_id_name   => "remote"
             );
        my $response = $session->add($relayagentfilter)
       or die("Add DHCP Relay Agent Filter failed: ",
             $session->status_code() . ":" . $session->status_detail());

print "DHCP Relay Agent Filter added successfully\n";
```

#Search for a specific DHCP relay agent filter

```
#Search the DHCP Relay Agent Filter
my @retrieved_objs = $session->search(
               object => "Infoblox::DHCP::Filter::RelayAgent",
               name   => "relayagent.*",
);
my $object = $retrieved_objs[0];
unless ($object) {
    die("Search DHCP Relay Agent Filter  failed: ",
        $session->status_code() . ":" . $session->status_detail());
}

print "Search DHCP relay agent filter found at least 1 matching entry\n";
```

#Get and modify a DHCP relay agent filter attribute

```
#Get  DHCP Relay Agent Filter through the session
my  @retrieved_objs = $session->get(
              object => "Infoblox::DHCP::Filter::RelayAgent" ,
              name   => "relayagent_filter",
       );
my $object = $retrieved_objs[0];
unless ($object) {
    die("Get Network object failed: ",
        $session->status_code() . ":" . $session->status_detail());
}

print "Get DHCP relay agent filter found at least 1 matching entry\n";
```

```
#Modify the attributes of the specified DHCP Relay Agent Filter object
$object->circuit_id_name("diff_Circuit");
$object->remote_id_name("diff_remote");
$session->modify($object)
    or die("Modify DHCP Relay Agent Filter attribute failed: ",
           $session->status_code() . ":" . $session->status_detail());

print "DHCP Relay Agent Filter object modified successfully \n";

#Create the Network object with the member
my $network = Infoblox::DHCP::Network->new(
    network => "10.0.0.0/8",
    comment => "add network",
);
unless($network) {
       die("Construct Network object failed: ",
            Infoblox::status_code() . ":" . Infoblox::status_detail());
}
print "Network object created successfully\n";

#Add the Network object into the Infoblox appliance through a session
$session->add($network)
    or die("Add Network object failed: ",
            $session->status_code() . ":" . $session->status_detail());

print "Network object added to Infoblox appliance successfully\n";
```

#Create a DHCP Range object that uses the relayagent filter

```
my $dhcp_range = Infoblox::DHCP::Range->new(
        end_addr                => "10.0.0.10",
        network                 => "10.0.0.0/8",
        start_addr              => "10.0.0.1",
        authority               => "true",
        filters                 => [$filter],
);
unless($dhcp_range) {
       die("Construct DHCP Range object failed: ",
            Infoblox::status_code() . ":" . Infoblox::status_detail());
}

print "DHCP Range object created successfully\n";

#Add the DHCP Range object into the Infoblox appliance through a session
$session->add($dhcp_range)
    or die("Add DHCP Range object failed: ",
            $session->status_code() . ":" . $session->status_detail());
print "DHCP Range object added to the network successfully\n";

#Remove the Network object into the Infoblox appliance through a session
$session->remove($network)
    or die("Remove Network object failed: ",
            $session->status_code() . ":" . $session->status_detail());
print "Network object removed from Infoblox appliance successfully\n";
```

#Remove a DHCP relay agent filter Object

```
 #Get object through the session
 my @retrieved_objs = $session->get(
         object   => "Infoblox::DHCP::Filter::RelayAgent" ,
         name     => "relayagent_filter",
       );
  my $object = $retrieved_objs[0];    unless ($object) {
    die("Get a DHCP Relay Agent Filterobject failed: ",
        $session->status_code() . ":" . $session->status_detail());
  }

print "Get a DHCP relay agent filter object found at least 1 matching entry\n";

#Submit the object for removal
$session->remove($object)
    or die("Remove a DHCP Relay Agent Filter failed: ",
        $session->status_code() . ":" . $session->status_detail());

print "DHCP relay agent filter removed successfully \n";

####PROGRAM ENDS####
```

AUTHOR

Infoblox Inc. http://www.infoblox.com/

SEE ALSO

Infoblox::Session, Infoblox::DHCP::Network, Infoblox::DHCP::Range, Infoblox::DHCP::FilterRule::RelayAgent, Infoblox::Session->get(), Infoblox::Session->search(), Infoblox::Session->add(), Infoblox::Session->remove(), Infoblox::Session->modify()

COPYRIGHT

Copyright (c) 2017 Infoblox Inc.

Infoblox::DHCP::Lease - DHCP Lease object.

- NAME
- DESCRIPTION
- SESSION METHODS
  - Infoblox::Session->get( )
  - Infoblox::Session->remove( )
  - Infoblox::Session->search( )
- METHODS
  - billing_class( )
  - binding_state( )
  - client_hostname( )
  - cltt( )
  - discovered_data( )
  - discovered_duid( )
  - discovered_name( )
  - discoverer( )
  - ends( )
  - fingerprint( )
  - first_discovered( )
  - hardware( )
  - ip_address( )
  - ipv4addr( )
  - ipv6_duid( )
  - ipv6_iaid( )
  - ipv6_preferred_lifetime( )
  - ipv6_prefix_bits( )
  - is_invalid_mac( )
  - last_discovered( )
  - mac( )
  - ms_ad_user_data( )
  - network( )
  - network_component_description( )
  - network_component_ip( )
  - network_component_name( )
  - network_component_port_description( )
  - network_component_port_name( )
  - network_component_port_number( )
  - network_component_type( )
  - network_view( )
  - netbios( )
  - next_binding_state( )
  - on_commit( )
  - on_expiry( )
  - on_release( )
  - option( )
  - os( )
  - port_duplex( )
  - port_link_status( )
  - port_speed( )
  - port_status( )
  - port_vlan_description( )
  - port_vlan_name( )
  - port_vlan_number( )
  - protocol( )
  - remote_id( )
  - starts( )
  - tsfp( )
  - tstp( )
  - uid( )
  - username( )
  - served_by( )
  - variable( )
  - v_cluster( )
  - v_datacenter( )
  - v_host( )
  - v_name( )
  - v_netadapter( )
  - v_switch( )
  - v_type( )
- SAMPLE CODE
- AUTHOR

- SEE ALSO
- COPYRIGHT

NAME

Infoblox::DHCP::Lease - DHCP Lease object.

DESCRIPTION

A DHCP lease provides information about the lease of an IP address to a DHCP client, such as the time when the Infoblox appliance issued or freed an IP address, the MAC address and host name of the client that received the IP address, and the grid member that supplied the lease. It allows the Infoblox appliance to store and correlate DHCP lease information over the lifetime of a lease.

SESSION METHODS

This section describes all the methods in the Infoblox::Session module that can be applied to a DHCP Lease object.

Infoblox::Session->get( )

Use this method to retrieve all the matching DHCP Lease objects from the Infoblox appliance. See Infoblox::Session->get() for parameters and return values.

Key References

```
 Apply the following attributes to get a specific DHCP Lease object:

 ip_address                          - Optional. IPv4 or IPv6 address of the DHCP lease. Use this,
in place of ipv4addr, even for IPv4 addresses.
 ipv4addr                            - Optional. IPv4 address (32 bits) of the DHCP lease. This
attribute is being deprecated in favor of the ip_address attribute.
 ipv6_duid                           - Optional. IPv6 DUID of the DHCP lease. (For IPv6 leases
only)
 protocol                            - Optional. Specify whether to get IPv4, IPv6, or both leases.
Allowed values are: 'IPV4', 'IPV6', and 'BOTH'.
 start_addr                          - Optional. Start IPv4 or IPv6 address of the DHCP range.
 end_addr                            - Optional. End IPv4 or IPv6 address of the DHCP range.
 network_view                        - Optional. Name of the network view that contains the lease.
 mac                                 - Optional. MAC address of the DHCP lease. (For IPv4 leases
only)
 username                            - Optional. User name associated with the DHCP lease.
 client_hostname                     - Optional. Client hostname of a DHCP lease.
 fingerprint                         - Optional. DHCP fingerprint description.
 remote_id                           - Optional. Relay agent remote ID, a sub-option of DHCP option
82. (For IPv4 leases only)

 discovered_duid                     - Optional. Discovered DHCPv6 UID.
 discovered_name                     - Optional. Discovered name.
 discoverer                          - Optional. Discoverer name.
 first_discovered                    - Optional. Time of the first discovery.
 last_discovered                     - Optional. Time of the last discovery.
 netbios                             - Optional. Discovered NetBIOS name.
 network_component_description       - Optional. Discovered network component description.
 network_component_ip                - Optional. Discovered network component IP address.
 network_component_name              - Optional. Discovered network component name.
 network_component_port_description  - Optional. Discovered network component port description.
 network_component_port_name         - Optional. Discovered network component port name.
 network_component_port_number       - Optional. Discovered network component port number.
 network_component_type              - Optional. Discovered network component type.
 os                                  - Optional. Discovered OS.
 port_duplex                         - Optional. Discovered port duplex settings.
 port_link_status                    - Optional. Discovered port link status.
 port_speed                          - Optional. Discovered port speed settings.
 port_status                         - Optional. Discovered port status settings.
 port_vlan_description               - Optional. Discovered port VLAN description.
 port_vlan_name                      - Optional. Discovered port VLAN name.
 port_vlan_number                    - Optional. Discovered port VLAN number.
 v_cluster                           - Optional. Discovered VMware cluster name. (For IPv4 leases
only)
 v_datacenter                        - Optional. Discovered VMware datacenter name. (For IPv4
leases only)
 v_host                              - Optional. Discovered VMware server name. (For IPv4 leases
only)
 v_name                              - Optional. Discovered VMware host or virtual machine name.
(For IPv4 leases only)
 v_netadapter                        - Optional. Discovered VMware physical network adapter name.
(For IPv4 leases only)
 v_switch                            - Optional. Discovered VMware virtual switch name. (For IPv4
leases only)
 v_type                              - Optional. Discovered VMware entity type (possible values:
"VirtualMachine", "HostSystem", "VirtualCenter").
```

Example

```
#Get a DHCP lease by IP address.
```

```
my @retrieved_objs = $session->get(
                            object   => "Infoblox::DHCP::Lease",
                            ip_address => "10.0.0.10"
                            mac      => "22:22:22:22:22:22",
                            username => "johndoe"
                            );
my @retrieved_objs = $session->get(
                            object   => "Infoblox::DHCP::Lease",
                            ip_address => "2001::1"
                            username => "johndoe"
                            );

#Get DHCP leases in a DHCP range.
my @retrieved_objs = $session->get(
                            object     => "Infoblox::DHCP::Lease" ,
                            start_addr => "10.0.0.1",
                            end_addr   => "10.0.0.10"
                            );
my @retrieved_objs = $session->get(
                            object     => "Infoblox::DHCP::Lease" ,
                            start_addr => "2001::1",
                            end_addr   => "2001::5"
                            );

#Get DHCP leases by v_type:
my @retrieved_objs = $session->get(
  object    => 'Infoblox::DHCP::Lease',
  v_type    => 'VirtualMachine');
```

Infoblox::Session->remove( )

Use this method to remove a DHCP Lease object from the Infoblox appliance. See Infoblox::Session->remove() for parameters and return values.

To remove a specific object, first use `get()` or `search()` to retrieve the specific object, and then submit this object for removal.

Example

```
#Remove a DHCP IPv4 Lease object.
my $object = $session->get(
    object   => "Infoblox::DHCP::Lease",
    ip_address => "10.0.0.10"
    mac      => "22:22:22:22:22:22",
    username => "johndoe"
);
my $response = $session->remove($object);

#Remove a DHCP IPv6 Lease object.
my $object = $session->get(
    object   => "Infoblox::DHCP::Lease",
    ip_address => "2001::1"
    username => "johndoe"
);
my $response = $session->remove($object);
```

Infoblox::Session->search( )

Use this method to search for DHCP Lease objects in the Infoblox appliance. See Infoblox::Session->search() for parameters and return values.

Key References

```
  Apply the following attributes to search for a DHCP Lease object:

  ip_address                          - Optional. IPv4 or IPv6 address of the DHCP lease. Use this,
in place of ipv4addr, even for IPv4 addresses (regular expression).
  ipv4addr                            - Optional. IPv4 address (32 bits) of the DHCP lease. This
attribute is being deprecated in favor of the ip_address attribute (regular exression).
  ipv6_duid                           - Optional. IPv6 DUID of the DHCP lease (regular expression).
(For IPv6 leases only)
  protocol                            - Optional. Specify whether to get IPv4, IPv6, or both
leases. Allowed values are: 'IPV4', 'IPV6', and 'BOTH'.
  mac                                 - Optional. MAC address of the DHCP lease (regular
expression). (For IPv4 leases only)
  network_view                        - Optional. Name of the network view that contains the lease.
  username                            - Optional. User name associated with the DHCP lease (regular
expression).
  start_addr                          - Optional. Start IPv4 or IPv6 address of the DHCP range.
  end_addr                            - Optional. End IPv4 or IPv6 address of the DHCP range.
```

```
  client_hostname                       - Optional. Client hostname of a DHCP lease (regular
expression).
  fingerprint                           - Optional. DHCP fingerprint description (regular
expression).
  remote_id                             - Optional. Relay agent remote ID, a sub-option of DHCP
option 82 (regular expression).

  discovered_duid                       - Optional. Discovered DHCPv6 UID (exact search only).
  discovered_name                       - Optional. Discovered name(regex).
  discoverer                            - Optional. Discoverer name(regex).
  first_discovered                      - Optional. Time of the first discovery (in ISO format, exact
search only).
  last_discovered                       - Optional. Time of the last discovery (in ISO format, exact
search only).
  netbios                               - Optional. Discovered NetBIOS name(regex).
  network_component_description         - Optional. Discovered network component description(regex).
  network_component_ip                  - Optional. Discovered network component IP address(regex).
  network_component_name                - Optional. Discovered network component name(regex).
  network_component_port_description    - Optional. Discovered network component port
description(regex).
  network_component_port_name           - Optional. Discovered network component port name(regex).
  network_component_port_number         - Optional. Discovered network component port number (exact
search only).
  network_component_type                - Optional. Discovered network component type(regex).
  os                                    - Optional. Discovered OS (regex).
  port_duplex                           - Optional. Discovered port duplex settings (exact search
only).
  port_link_status                      - Optional. Discovered port link status (exact search only).
  port_speed                            - Optional. Discovered port speed settings (exact search
only).
  port_status                           - Optional. Discovered port status settings (exact search
only).
  port_vlan_description                 - Optional. Discovered port VLAN description(regex).
  port_vlan_name                        - Optional. Discovered port VLAN name(regex).
  port_vlan_number                      - Optional. Discovered port VLAN number(exact search only).
  v_cluster                             - Optional. Discovered VMware cluster name (regular
expression). (For IPv4 leases only)
  v_datacenter                          - Optional. Discovered VMware datacenter name (regular
expression). (For IPv4 leases only)
  v_host                                - Optional. Discovered VMware server name (regular
expression). (For IPv4 leases only)
  v_name                                - Optional. Discovered VMware host or virtual machine name
(regular expression). (For IPv4 leases only)
  v_netadapter                          - Optional. Discovered VMware physical network adapter name
(regular expression). (For IPv4 leases only)
  v_switch                              - Optional. Discovered VMware virtual switch name (regular
expression). (For IPv4 leases only)
  v_type                                - Optional. Discovered VMware entity type (possible values:
"VirtualMachine", "HostSystem", "VirtualCenter"). (For IPv4 leases only)
```

Example

```
#Search a DHCP lease.
my @result_array = $session->search(
                    object   => "Infoblox::DHCP::Lease",
                    ip_address => "^10\.*",
                    mac      => "22:.*",
                    username => "john.*"
                    );
my @result_array = $session->search(
                    object   => "Infoblox::DHCP::Lease",
                    ip_address => "^ff::.*",
                    username => "john.*"
                    );

# search a DHCP lease by v_datacenter:
my @retrieved_objs = $session->search(
   object       => 'Infoblox::DHCP::Lease',
   v_datacenter => 'test.*');
```

METHODS

This section describes all the methods that can be used to configure and retrieve the attribute value of a DHCP Lease object.

billing_class( )

Use this method to retrieve the billing_class of a DHCP Lease object. This method specifies the class to which this lease is currently billed. This method is for IPv4 leases only.

Parameter

N/A

Returns

The method returns the attribute value.

Example

```
#Get billing_class
my $billing_class = $lease->billing_class();
```

binding_state( )

Use this method to retrieve the binding state for the current lease. The lease state can be one of the following:

```
 Free      : The lease is available for clients to use.
 Active    : The lease is currently in use by a DHCP client.
 Expired   : The lease was in use, but the DHCP client never renewed it, so it is no longer valid.
 Released  : The DHCP client returned the lease to the appliance.
 Abandoned: The Infoblox appliance cannot lease this IP address because the appliance received a response
when it pinged the address.
```

Parameter

N/A

Returns

The method returns the attribute value.

Example

```
#Get binding_state
my $binding_state = $lease->binding_state();
```

client_hostname( )

Use this method to retrieve the client_hostname of a DHCP Lease object. This method specifies the host name that the DHCP client sends to the Infoblox appliance using DHCP option 12.

Parameter

N/A

Returns

The method returns the attribute value.

Example

```
#Get client_hostname
my $client_hostname = $lease->client_hostname();
```

cltt( )

Use this method to retrieve the CLTT (Client Last Transaction Time) value of a DHCP Lease object. This method specifies the time of the last transaction with the DHCP client for this lease.

Parameter

N/A

Returns

The method returns the attribute value.

Example

```
#Get cltt
my $cltt = $lease->cltt();
```

discovered_data( )

Use this method to retrieve the discovered data of a DHCP Lease object. This is a read-only attribute.

Omit the parameter to retrieve the attribute value.

Parameter

None

Returns

The valid return value is an Infoblox::Grid::Discovery::Data object.

Example

```
#Get discovered_data value
my $discovered_data = $object->discovered_data();
```

discovered_duid( )

Use this method to retrieve the discovered DHCPv6 UID (DUID) of a DHCP Lease object.

Parameter

none

Returns

The method returns the discovered name.

Example

```
#Get discovered_duid
my $discovered_duid = $bindptr->discovered_duid();
```

discovered_name( )

Use this method to retrieve the discovered name of a DHCP Lease object.

Parameter

none

Returns

The method returns the discovered name.

Example

```
#Get discovered_name
my $discovered_name = $lease->discovered_name();
```

discoverer( )

Use this method to retrieve the discoverer of a DHCP Lease object.

Parameter

none

Returns

The method returns the discoverer value.

Example

```
#Get discoverer
my $discoverer = $lease->discoverer();
```

ends( )

Use this method to retrieve the end time value of a DHCP Lease object. This method specifies the day, date, and time that a lease ended.

Parameter

N/A

Returns

The method returns the attribute value.

Example

```
#Get ends
my $ends = $lease->ends();
```

fingerprint( )

Use this method to retrieve the descripiton of the DHCP fingerprint of a DHCP Lease object. This is a read-only attribute.

Parameter

none

Returns

The method returns the attribute value.

Example

```
#Get fingerprint
my $fingerprint = $lease->fingerprint();
```

first_discovered( )

Use this method to retrieve the date and time when the DHCP Lease object was first discovered.

Parameter

none

Returns

The method returns the date and time when the DHCP Lease object was first discovered.

Example

```
#Get first_discovered
my $first_discovered = $lease->first_discovered();
```

hardware( )

Use this method to retrieve the hardware type of a DHCP Lease object. This method specifies the MAC address of the network interface on which the lease will be used. This method is for IPv4 leases only.

Parameter

N/A

Returns

The method returns the attribute value.

Example

```
#Get hardware
my $hardware = $lease->hardware();
```

ip_address( )

Use this method to retrieve the IPv4 or IPv6 address that the Infoblox appliance leased to the DHCP client. Use this method, in place of the `ipv4addr()` method, even for IPv4 addresses.

Parameter

N/A

Returns

The method returns the attribute value.

Example

```
#Get ip_address
my $ip_address = $lease->ip_address();
```

ipv4addr( )

Use this method to retrieve the IP address that the Infoblox appliance leased to the DHCP client. This method is being deprecated in favor of the `ip_address()` method. This method is for IPv4 leases only.

Parameter

N/A

Returns

The method returns the attribute value.

Example

```
#Get ipv4addr
my $ipv4addr = $lease->ipv4addr();
```

ipv6_duid( )

Use this method to retrieve the DUID value of an IPv6 address that the Infoblox appliance leased to the DHCP client. This method is for IPv6 leases only.

Parameter

N/A

Returns

The method returns the attribute value.

Example

```
#Get ipv6_duid
my $ipv6_duid = $lease->ipv6_duid();
```

ipv6_iaid( )

Use this method to retrieve the Interface ID value of an IPv6 address that the Infoblox appliance leased to the DHCP client. This method is for IPv6 leases only.

Parameter

N/A

Returns

The method returns the attribute value.

Example

```
#Get ipv6_iaid
my $ipv6_iaid = $lease->ipv6_iaid();
```

ipv6_preferred_lifetime( )

Use this method to retrieve the preferred lifetime value of an IPv6 address that the Infoblox appliance leased to the DHCP client. This method is for IPv6 leases only.

Parameter

N/A

Returns

The method returns the attribute value.

Example

```
#Get ipv6_preferred_lifetime
my $ipv6_preferred_lifetime = $lease->ipv6_preferred_lifetime();
```

ipv6_prefix_bits( )

Use this method to retrieve the prefix bits value of an IPv6 address that the Infoblox appliance leased to the DHCP client. This method is for IPv6 leases only.

Parameter

N/A

Returns

The method returns the attribute value.

Example

```
#Get ipv6_prefix_bits
my $ipv6_prefix_bits = $lease->ipv6_prefix_bits();
```

is_invalid_mac( )

Use this method to retrieve the flag used to indicate whether an invalid MAC address is used. This is a read-only attribute.

Parameter

None

Returns

The method returns the attribute value.

Example

```
#Get is_invalid_mac value
my $is_invalid_mac = $lease->is_invalid_mac();
```

last_discovered( )

Use this method to retrieve the time when this object was last discovered.

The default value for this field is empty.

Parameter

N/A

Returns

The method returns the attribute value.

Example

```
#Get last_discovered
my $last_discovered = $lease->last_discovered();
```

mac( )

Use this method to retrieve the MAC address of the DHCP client that leased the IP address. This method is for IPv4 leases only.

Parameter

N/A

Returns

The method returns the attribute value.

Example

```
#Get mac
my $mac = $lease->mac();
```

ms_ad_user_data( )

Use this method to retrieve Microsoft Active Directory users related information. This is a read-only attribute.

Parameter

None

Returns

The valid return value is an Infoblox::Grid::MSServer::AdUser::Data object.

Example

```
#Get ms_ad_user_data
my $ms_ad_user_data = $lease->ms_ad_user_data();
```

network( )

Use this method to find the network associated with this lease. This property is read-only.

Parameter

N/A

Returns

The method returns the network associated with the lease in the string of the form "network/netmask".

Example

```
#Get network property of the lease
my $network = $lease->network();
```

network_component_description( )

Use this method to retrieve the description of the network device that is connected to the DHCP Lease object.

Parameter

none

Returns

The method returns the network device description.

Example

```
#Get network_component_description
my $network_component_description = $lease->network_component_description();
```

network_component_ip( )

Use this method to retrieve the IP address of the network device that is connected to the DHCP Lease object.

Parameter

none

Returns

The method returns the network device IP address.

Example

```
#Get network_component_ip
my $network_component_ip = $lease->network_component_ip();
```

network_component_name( )

Use this method to retrieve the name of the network device that is connected to the DHCP Lease object.

Parameter

none

Returns

The method returns the network device name.

Example

```
#Get network_component_name
my $network_component_name = $lease->network_component_name();
```

network_component_port_description( )

Use this method to retrieve the description of the network device port that is connected to the DHCP Lease object.

Parameter

none

Returns

The method returns the description of the network device port.

Example

```
#Get network_component_port_description
my $network_component_port_description = $lease->network_component_port_description();
```

network_component_port_name( )

Use this method to retrieve the name of the network device port that is connected to the DHCP Lease object.

Parameter

none

Returns

The method returns the name of the port on the network device.

Example

```
#Get network_component_port_name
my $network_component_port_name = $lease->network_component_port_name();
```

network_component_port_number( )

Use this method to retrieve the number of the network device port that is connected to the DHCP Lease object.

Parameter

none

Returns

The method returns the network device port number.

Example

```
#Get network_component_port_number
my $network_component_port_number = $lease->network_component_port_number();
```

network_component_type( )

Use this method to retrieve the type of the network device that is connected to the DHCP Lease object.

Parameter

none

Returns

The method returns the network device type.

Example

```
#Get network_component_type
my $network_component_type = $lease->network_component_type();
```

network_view( )

Use this method to retrieve the network view of the DHCP lease.

Parameter

N/A

Returns

This method returns the network view of the DHCP lease as Infoblox::DHCP::View object.

Example

```
#Get network view
my $nview = $lease->network_view();
```

netbios( )

Use this method to retrieve the name in the NetBIOS reply that responded to a NetBIOS query.

Returns

The method returns the attribute value.

Example

```
#Get netbios
```

```
my $netbios = $lease->netbios();
```

next_binding_state( )

Use this method to retrieve the subsequent binding state when the current lease expires. This method is for IPv4 leases only.

```
 Free      : The lease is available for clients to use.
 Active    : The lease is currently in use by a DHCP client.
 Expired   : The lease was in use, but the DHCP client never renewed it, so it is no longer valid.
 Released  : The DHCP client returned the lease to the appliance.
 Abandoned: The Infoblox appliance cannot lease this IP address because the appliance received a response
when it pinged the address.
```

Parameter

N/A

Returns

The method returns the attribute value.

Example

```
#Get next_binding_state
my $next_binding_state = $lease->next_binding_state();
```

on_commit( )

Use this method to retrieve the list of commands to be executed upon the granting of the lease.

Parameter

N/A

Returns

The method returns the attribute value.

Example

```
#Get on_commit
my $on_commit = $lease->on_commit();
```

on_expiry( )

Use this method to retrieve the list of commands to be executed upon the expiration of the lease.

Parameter

N/A

Returns

The method returns the attribute value.

Example

```
#Get on_expiry
my $on_expiry = $lease->on_expiry();
```

on_release( )

Use this method to retrieve the list of commands to be executed upon the release of the lease.

Parameter

N/A

Returns

The method returns the attribute value.

Example

```
#Get on_release
my $on_release = $lease->on_release();
```

option( )

Use this method to retrieve the option value of a DHCP Lease object. This method specifies the agent circuit ID and remote ID sent by a DHCP relay agent in DHCP option 82. This method is for IPv4 leases only.

Parameter

N/A

Returns

The method returns the attribute value.

Example

```
#Get option
my $option = $lease->option();
```

os( )

Use this method to retrieve the operating system associated with this DHCP lease.

Returns

The method returns the attribute value.

Example

```
#Get os
my $os = $lease->os();
```

port_duplex( )

Use this method to retrieve the duplex setting of the network device port that is connected to the DHCP Lease object.

Parameter

none

Returns

The method returns the duplex setting of the network device port. The returend value, if any, is one of the following:

```
Full
Half
```

Example

```
#Get port_duplex
my $port_duplex = $lease->port_duplex();
```

port_link_status( )

The method returns the link status of the network device port. The returned value, if any, is one of the following:

Parameter

none

Returns

The method returns the port link status. Returned values, if any, are one of the following:

```
Connected
Not Connected
Unknown
```

Example

```
#Get port_link_status
my $port_link_status = $lease->port_link_status();
```

port_speed( )

Use this method to retrieve the speed of the network device port that is connected to the DHCP Lease object.

Parameter

none

Returns

The method returns the speed value of the network device port. The returned value, if any, is one of the following:

```
10M
100M
1G
10G
100G
Unknown
```

Example

```
#Get port_speed
my $port_speed = $lease->port_speed();
```

port_status( )

Use this method to retrieve the status of the network device port that is connected to the DHCP Lease object.

Parameter

none

Returns

The method returns the status of the network device port. The returned value, if any, is one of the following:

```
Up
Down
Unknown
```

Example

```
#Get port_status
my $port_status = $lease->port_status();
```

port_vlan_description( )

Use this method to retrieve the VLAN description of the network device port that is connected to the DHCP Lease object.

Parameter

none

Returns

The method returns the VLAN description of the network device port.

Example

```
#Get port_vlan_description
my $port_vlan_description = $lease->port_vlan_description();
```

port_vlan_name( )

Use this method to retrieve the VLAN name of the network device port that is connected to the DHCP Lease object.

Parameter

none

Returns

The method returns the VLAN name of the network device port.

Example

```
#Get port_vlan_name
my $port_vlan_name = $lease->port_vlan_name();
```

port_vlan_number( )

Use this method to retrieve the VLAN number of the network device port that is connected to the DHCP Lease object.

Parameter

none

Returns

The method returns the VLAN number of the network device port.

Example

```
#Get port_vlan_number
my $port_vlan_number = $lease->port_vlan_number();
```

protocol( )

Use this method to determine whether the IP address the Infoblox appliance leased an IPv4 or IPv6 address to the client.

Parameter

N/A

Returns

The method returns the attribute value ('IPV4' or 'IPV6').

Example

```
#Get protocol
my $protocol = $lease->protocol();
```

remote_id( )

Use this method to retrieve an agent remote ID (option 82 sub-option). This is a read-only attribute.

Remote ID can be presented either in the ASCII form (e.g. "abcd") or in the form of colon separated HEX values (e.g. 1:2:ab:cd). HEX representation is only used when sub-option value contains unprintable characters. If a remode ID sub-option value is an ASCII string, it's always enclosed in quotes to prevent ambiguous values (e.g. "10:20" - ASCII 5 byte string; 10:20 - HEX 2 byte value). Remote ID can be in ASCII form (e.g. ""abcd"") or in colon-separated HEX form (e.g. "1:2:ab:cd").

```
 ASCII: ASCII representation is used if the remote ID sub-option contains only printable ASCII charaters
(ASCII characters in range "x20-0x7E").
        - The backslash symbol (\) is used as an escape symbol to escape the quote symbol (") in an ASCII
string.
        - Double backslashes (\\) are used to represent the backslash symbol (\) in an ASCII string.
 HEX  : HEX representation is used only when the remote ID sub-option value contains unprintable characters
and is normalized as follows:
        - starting zero is removed from digits: 1, a - Valid, 01, 0a - Invalid;
        - lowercase characters are used for symbols: fa - Valid, FA - Invalid.
```

NIOS does not support the convertion between HEX and ASCII formats. Searches are performed using the exact same format and value as the sub-option is represented.

Query examples assume the following leases are stored in the database:

```
 Number  : option field                                                                  : extracted remote_id field
 Lease01 : agent.remote-id="00152654358700" agent.circuit-id="BX1-PORT-003" : "00152654358700"
 Lease02 : agent.remote-id="Dhcp Relay 10" agent.circuit-id="Port008"       : "Dhcp Relay 10"
 Lease03 : agent.remote-id="00:01:02"                                                : "00:01:02"
 Lease04 : agent.remote-id=0:1:2                                                     : 0:1:2
 Lease05 : agent.remote-id=02:03                                                     : 2:3
 Lease06 : agent.remote-id=10:20                                                     : 10:20
 Lease07 : agent.circuit-id="no-remote-id"                                           :
```

Expected results:

```
 Query                    : Returned leases                                          : Comments
 remote_id=01:02          : None                                                     : EXACT query. No results
are expected
 remote_id="Dhcp Relay 10" : Lease02                                                 : EXACT query for an
ASCII value.
 remote_id=0:1:2          : Lease04                                                  : EXACT query for a HEX
value.
 remote_id=00:01:02       : None                                                     : EXACT query for a HEX
value. No results are expected as the search value is not normalized to the same format used in the
database.
 remote_id~=10            : Lease02, Lease06                                         : REGEX query.
 remote_id~=^".*1         : Lease01, Lease02, Lease03                                : REGEX query. Only ASCII
values are expected due to the starting quote (") in the search value.
 remote_id~=^[^"]*2       : Lease04, Lease05, Lease06                                : REGEX query. Only HEX
values are expected as the starting quote (") is excluded from the search value.
 remote_id=""             : None                                                     : EXACT query. No results
are expected as no leases that contain an empty remote ID value exist in the database.
 remote_id~=""            : Lease01, Lease02, Lease03, Lease04, Lease05, Lease06 : REGEX query. This query
is expected to match any lease that contains remote ID set to any value.
```

NOTE: EXACT queries (without the ~ symbol) are perfomed using the get method. REGEX queries (with the ~ symbol) are perfomed using the search method.

NOTE: Lease07 is not expected to be returned when searching for the remote ID sub-option.

Parameter

None

Returns

The method returns attribute value.

Example

```
#Get remote_id
my $remote_id = $lease->remote_id();
```

starts( )

Use this method to retrieve the start time value of a DHCP Lease object. This method specifies the day, date, and time that the lease started.

Parameter

N/A

Returns

The method returns the attribute value.

Example

```
#Get starts
my $starts = $lease->starts();
```

tsfp( )

Use this method to retrieve the TSFP (Time Sent From Partner) value of a DHCP Lease object. This method specifies the time that the current lease state ends, from the point of view of a remote DHCP failover peer. This method is for IPv4 leases only.

Parameter

N/A

Returns

The method returns the attribute value.

Example

```
#Get tsfp
my $tsfp = $lease->tsfp();
```

tstp( )

Use this method to retrieve the TSTP (Time Sent To Partner) value of a DHCP Lease object. This method specifies the time that the current lease state ends, from the point of view of a local DHCP failover peer. This method is for IPv4 leases only.

Parameter

N/A

Returns

The method returns the attribute value.

Example

```
#Get tstp
my $tstp = $lease->tstp();
```

uid( )

Use this method to retrieve the UID (User ID) value of a DHCP Lease object. This method specifies the client identifier that the DHCP client sends the Infoblox appliance (in DHCP option 61) when it acquires the lease. Not all DHCP clients send a UID. This method is for IPv4 leases only.

Parameter

N/A

Returns

The method returns the attribute value.

Example

```
#Get uid
my $uid = $lease->uid();
```

username( )

Use this method to retrieve the user name that the server has associated with a DHCP Lease object.

Parameter

N/A

Returns

The method returns the attribute value.

Example

```
#Get username
my $username = $lease->username();
```

served_by( )

Use this method to retrieve the IP address of the server that hands out an active lease to a client.

Parameter

N/A

Returns

The method returns the attribute value.

Example

```
#Get served_by
my $served_by = $lease->served_by();
```

variable( )

Use this method to retrieve the variable value of a DHCP Lease object. This method keeps all variables related to the DDNS update of the DHCP lease. The variables can be one of the following:

```
 ddns-text        : The ddns-text variable is used to record the value of the client's TXT identification
record when the
                   interim DDNS update style has been used to update the DNS service for a particular
lease.

 ddns-fwd-name    : The ddns-fwd-name variable records the value of the name used when the client's A
record was updated, if a DDNS
                   update was successfully completed by the server. The server may have used this name
when it updated the
                   client's PTR record as well.

 ddns-client-fqdn : If the server is configured to use the interim DDNS update style, and is also
configured to allow clients
                   to update their own FQDNs, and the client did in fact update its own FQDN, then the
ddns-client-fqdn
                   variable records the name that the client indicated it used. This is also the name that
the server
                   used to update the client's PTR record in this case.

 ddns-rev-name    : If the server successfully updates the client's PTR record, this variable will record
the name that the
                   DHCP server used for the PTR record. The name to which the PTR record points will be
either the
                   ddns-fwd-name or the ddns-client-fqdn.
```

Parameter

N/A

Returns

The method returns the attribute value.

Example

```
#Get variable
```

```
my $variable = $lease->variable();
```

v_cluster( )

Use this method to retrieve the name of the VMware cluster associated with the DHCP Lease object. This method is read-only. This method is for IPv4 leases only.

Parameter

none

Returns

This method returns a string that contains the VMware cluster name. The default value is an empty string.

Example

```
#Get v_cluster
my $v_cluster = $lease->v_cluster();
```

v_datacenter( )

Use this method to retrieve the name of the VMware datacenter associated with the DHCP Lease object. This method is read-only. This method is for IPv4 leases only.

Parameter

none

Returns

This method returns a string that contains the VMware datacenter name. The default value is an empty string.

Example

```
#Get v_datacenter
my $v_datacenter = $lease->v_datacenter();
```

v_host( )

Use this method to retrieve the name of the VMware host associated with the DHCP Lease object. This method is read-only. This method is for IPv4 leases only.

Parameter

none

Returns

This method returns a string that contains the VMware host name. The default value is an empty string.

Example

```
#Get v_host
my $v_host = $lease->v_host();
```

v_name( )

Use this method to retrieve the name of the VMware entity associated with the DHCP Lease object. This method is read-only. This method is for IPv4 leases only.

Parameter

none

Returns

This method returns a string that contains the VMware entity name. The default value is an empty string.

Example

```
#Get v_name
my $v_name = $lease->v_name();
```

v_netadapter( )

Use this method to retrieve the name of the physical network adapter through which the VMware entity is connected to the appliance. This

method is read-only. This method is for IPv4 leases ohly.

Parameter

none

Returns

This method returns a string that contains the network adapter name. The default value is an empty string.

Example

```
#Get v_netadapter
my $v_netadapter = $lease->v_netadapter();
```

v_switch( )

Use this method to retrieve the name of the virtual switch through which the VMware entity is connected to the appliance. This method is read-only. This method is for IPv4 leases only.

Parameter

none

Returns

This method returns a string that contains the virtual switch name. The default value is an empty string.

Example

```
#Get v_switch
my $v_switch = $lease->v_switch();
```

v_type( )

Use this method to retrieve the type of VMware entity associated with the DHCP Lease object. This method is read-only. This method is for IPv4 leases only.

Parameter

none

Returns

This method returns a string that contains the VMware entity name. The default value is an empty string. The returned value, if any, can be one of the following:

```
VirtualMachine
HostSystem
VirtualCenter
```

Example

```
#Get v_type
my $v_type = $lease->v_type();
```

SAMPLE CODE

The following sample code demonstrates the different functions that can be applied to a DHCP Lease object, such as get, search, and remove. Also, this sample code includes error handling for the operations.

#Preparation prior to DHCP Lease operations

```
use strict;
use Infoblox;

#Create a session to the Infoblox appliance
my $host_ip   = "192.168.1.2";
my $host_name = "infoblox.localdomain";
my $session = Infoblox::Session->new(
    master   => $host_ip,       #appliance host ip
    username => "admin",        #appliance user login
    password => "infoblox"      #appliance password
);
unless ($session) {
   die("Construct session failed: ",
       Infoblox::status_code() . ":" . Infoblox::status_detail());
```

```
}
print "Session created successfully.\n";

#Create a member object
 my $member = Infoblox::DHCP::Member->new(
    name     => $host_name,
    ipv4addr => $host_ip,
);

unless($member) {
     die("Construct member failed: ",
           Infoblox::status_code() . ":" . Infoblox::status_detail());
}
print "Member object created successfully\n";

#Create a network object
my $network = Infoblox::DHCP::Network->new(
                            network  => "10.0.0.0/8",
                            comment  => "add network",
                            members  => [$member]
                            );
unless($network){
       die("Construct network object fail: ",
               Infoblox::status_code(). ":" .Infoblox::status_detail());
       }

print"Network object created successfully.\n";

#Verify if the network exists
my $object = $session->get(
               object  => "Infoblox::DHCP::Network",
               network => "10.0.0.0/8"
               );
unless($object){
       print"Network does not exist on server, safe to add the network.\n";
       $session->add($network)
               or die("Add network failed: ",
                       $session->status_code(). ":" .$session->status_detail());
       }

print"Network added successfully.\n";

#Create a dhcp range object
my $dhcp_range = Infoblox::DHCP::Range->new(
    network    => "10.0.0.0/8",
    start_addr => "10.0.0.1",
    end_addr   => "10.0.0.10",
    member     => $member,
);

unless($dhcp_range){
       die("Construct dhcp range object failed: ",
               Infoblox::status_code(). ":" .Infoblox::status_detail());
       }

print "dhcp range object created successfully.\n";

#Add the dhcp range to the Infoblox appliance
$session->add($dhcp_range)
               or die("Add dhcp range failed: ",
                       $session->status_code(). ":" .$session->status_detail());

print "dhcp range added successfully.\n";

my $result = $session->import_data(
       "type" => "leases",
       "path"=> "./API_unit_test_sample_code/import_dhcpd.lease",
       "format" => "ISC",
       "lease_precedence" => "replace-previous"
      );

unless($result){
        die("Import lease failed: ",
          $session->status_code() . ":" . $session->status_detail());
 }

print "Import dhcp lease successfull.\n";
```

#Search for a DHCP Lease object

```
my @retrieved_objs= $session->search(
                    object   => "Infoblox::DHCP::Lease" ,
                    ipv4addr => '^10\..*'
                    );
my $desired_lease = $retrieved_objs[0];
unless($desired_lease){
        die("search lease failed: ",
          $session->status_code() . ":" . $session->status_detail());
 }

print "Search dhcp lease object found at least 1 matching entry\n";
```

#Get a DHCP Lease object

```
#Get a DHCP lease by IP address.
my @retrieved_objs = $session->get(
                         object   => "Infoblox::DHCP::Lease",
                         ipv4addr => "10.0.0.10"
                         );
my $desired_lease = $retrieved_objs[0];
unless($desired_lease){
        die("get lease failed: ",
          $session->status_code() . ":" . $session->status_detail());
 }

print "Get dhcp lease object found at least 1 matching entry\n";
```

#Remove a DHCP Lease object

```
#Submit the object for removal
$session->remove($desired_lease)
    or die("Remove dhcp lease failed: ",
        $session->status_code() . ":" . $session->status_detail());
print "dhcp lease object removed successfully \n";

####PROGRAM ENDS####
```

AUTHOR

Infoblox Inc. http://www.infoblox.com/

SEE ALSO

Infoblox::Session, Infoblox::Session->add(), Infoblox::Session->get(), Infoblox::Session->remove(), Infoblox::Session->search(), Infoblox::DHCP::Network, Infoblox::DHCP::View, Infoblox::DHCP::Range, Infoblox::DHCP::IPv6Network, Infoblox::DHCP::IPv6Range, Infoblox::Grid::MSServer::AdUser::Data, Infoblox::Grid::Discovery::Data

---

COPYRIGHT

Copyright (c) 2017 Infoblox Inc.

Infoblox::DHCP::FilterRule::RelayAgent - DHCP RelayAgent Filter Rule object.

- NAME
- DESCRIPTION
- CONSTRUCTOR
- MODULE METHODS
  - Infoblox::DHCP::Range->filters( )
- METHODS
  - filter_name( )
  - permission( )
- SAMPLE CODE
- AUTHOR
- SEE ALSO
- COPYRIGHT

NAME

Infoblox::DHCP::FilterRule::RelayAgent - DHCP RelayAgent Filter Rule object.

DESCRIPTION

A relay agent adds the circuit ID and Remote ID information to the client request and sends it to DHCP Server. Infoblox appliance uses Relay Agent Filter Rules to grant/deny the lease to a request that comes from those Relay Agents.

CONSTRUCTOR

```
my $ra_filter_rule = Infoblox::DHCP::FilterRule::RelayAgent->new(
      filter_name    => $string,          #Required
      permission     => "grant" | "deny", #Required
);
```

MODULE METHODS

The following functions are available to apply to a DHCP RelayAgent Filter Rule object.

Infoblox::DHCP::Range->filters( )

Use this method to add a DHCP RelayAgent Filter Rule object to the DHCP Range on the Infoblox appliance. See Infoblox::DHCP::Range->filters() for parameters and return values.

Example

```
#Construct a DHCP RelayAgent Filter Rule object
my $ra_filter_rule1 = Infoblox::DHCP::FilterRule::RelayAgent->new(
      filter_name    => "filter1",
      permission     => "grant",
);

#Add RelayAgent Filter Rule to the DHCP Range object.
my $response = $dhcp_range->filters([$ra_filter_rule1]);
```

METHODS

This section describes all the methods that can be used to set and retrieve the attribute values of a DHCP RelayAgent Filter Rule object.

filter_name( )

Use this method to set or retrieve the filter_name of a DHCP RelayAgent Filter Rule object.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

Name of the DHCP RelayAgent filter. Maximum length up to 1024 bytes is supported.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get filter_name
my $filter_name = $ra_filter_rule->filter_name();
#Modify filter_name
$ra_filter_rule->filter_name("filter2");
```

permission( )

Use this method to set or retrieve the permission of a DHCP RelayAgent Filter Rule object.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

Specify the permission in string format. The permission can be either "grant" or "deny".

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get permission
my $permission = $ra_filter_rule->permission();
#Modify permission
$ra_filter_rule->permission("deny");
```

SAMPLE CODE

The following sample code demonstrates the different functions that can be applied to an object such as modify. Also, this sample includes error handling for the operations.

#Preparation prior to a DHCP RelayAgent Filter Rule object insertion

```
#PROGRAM STARTS: Include all the modules that will be used
use strict;
use Infoblox;

#Create a session to the Infoblox appliance

my $session = Infoblox::Session->new(
               master   => "192.168.1.2", #appliance host ip
               username => "admin",       #appliance user login
               password => "infoblox"     #appliance password
);

unless ($session) {
       die("Construct session failed: ",
               Infoblox::status_code() . ":" . Infoblox::status_detail());
}
print "Session created successfully\n";

#Create the Network object
my $network = Infoblox::DHCP::Network->new(
    network => "10.0.0.0/8",
    comment => "add network",
);
unless($network) {
       die("Construct Network object failed: ",
            Infoblox::status_code() . ":" . Infoblox::status_detail());
}
print "Network object created successfully\n";

#Add the Network object into the Infoblox appliance through a session
$session->add($network)
    or die("Add Network object failed: ",
            $session->status_code() . ":" . $session->status_detail());
print "Network object added to Infoblox appliance successfully\n";

#Construct a DHCP RelayAgent Filter object
my $ra_filter = Infoblox::DHCP::Filter::RelayAgent->new(
       name            => "filter1",
```

```
        circuit_id_name => "circuit",
        remote_id_name  => "remote"
);

unless ( $ra_filter ) {
        die("Construct RelayAgent Filter failed: ",
                Infoblox::status_code() . ":" . Infoblox::status_detail() );
}
print "RelayAgent Filter object created successfully\n";

#Add the RelayAgent Filter to the Infoblox appliance through a session
$session->add($ra_filter)
    or die("Add RelayAgent Filter object failed: ",
            $session->status_code() . ":" . $session->status_detail());
print "RelayAgent Filter object added to Infoblox appliance successfully\n";
```

#Create a DHCP RelayAgent Filter Rule object

```
#Construct a DHCP RelayAgent Filter Rule object
my $ra_filter_rule1 = Infoblox::DHCP::FilterRule::RelayAgent->new(
      filter_name   => "filter1",
      permission    => "grant",
);

unless ( $ra_filter_rule1 ) {
        die("Construct RelayAgent Filter Rule failed: ",
                Infoblox::status_code() . ":" . Infoblox::status_detail() );
}
print "RelayAgent Filter Rule object created successfully\n";

#Create the DHCP Range object with this RelayAgent Filter Rule.
my $dhcp_range = Infoblox::DHCP::Range->new(
        end_addr   => "10.0.0.10",
        network    => "10.0.0.0/8",
        start_addr => "10.0.0.1",
        filters    => [ $ra_filter_rule1 ],
);
unless($dhcp_range) {
        die("Construct DHCP Range object failed: ",
            Infoblox::status_code() . ":" . Infoblox::status_detail());
}
print "DHCP Range object created successfully\n";

#Add the DHCP Range object into the Infoblox appliance through a session
$session->add($dhcp_range)
    or die("Add DHCP Range object failed: ",
            $session->status_code() . ":" . $session->status_detail());
print "DHCP Range object added to Infoblox appliance successfully\n";
```

#Get and modify a RelayAgent Filter Rule object

```
#Get RelayAgent Filter Rule object from the DHCP Range object through session
my @retrieved_objs = $session->get(
    object      => "Infoblox::DHCP::Range",
    start_addr  => "10.0.0.1",
);

my $object = $retrieved_objs[0];
my $filter = $object->filters();

my @filters = @{$filter};

my $ra_filterrule = $filters[0];

#Modify one of the attributes of the obtained RelayAgent Filter Rule object
$ra_filterrule->permission("deny");
$object->filters([$ra_filterrule]);

#Apply the changes
$session->modify($object)
    or die("Modify RelayAgent Filter Rule object failed: ",
           $session->status_code() . ":" . $session->status_detail());
print "RelayAgent Filter Rule object modified and added to the DHCP Range object successfully \n";
```

#Remove a RelayAgent Filter Rule object

```
$object->filters([]);
```

```
my $response = $session->modify($object);
unless($response) {
       die("Remove RelayAgent Filter Rule failed: ",
               session->status_code() . ":" . session->status_detail() );
}
print "RelayAgent Filter Rule object removed successfully \n";

#Submit the network object for removal
$session->remove($network)
    or die("Remove Network object failed: ",
        $session->status_code() . ":" . $session->status_detail());
print "Network object removed successfully \n";

#Submit the relay agent filter object for removal
$session->remove($ra_filter)
    or die("Remove RelayAgent Filter object failed: ",
        $session->status_code() . ":" . $session->status_detail());
print "RelayAgent Filter object removed successfully \n";

####PROGRAM ENDS####
```

AUTHOR

Infoblox Inc. http://www.infoblox.com/

SEE ALSO

Infoblox::DHCP::MAC, Infoblox::DHCP::FilterRule::RelayAgent, Infoblox::DHCP::Network, Infoblox::DHCP::Range, Infoblox::Session->add(), Infoblox::Session->get(), Infoblox::Session->modify(), Infoblox::Session->remove(),Infoblox::Session

---

COPYRIGHT

Copyright (c) 2017 Infoblox Inc.

Infoblox::DHCP::Filter::Fingerprint - DHCP Fingerprint Filter object.

- NAME
- DESCRIPTION
- CONSTRUCTOR
- SESSION METHODS
  - Infoblox::Session->add( )
  - Infoblox::Session->get( )
  - Infoblox::Session->modify( )
  - Infoblox::Session->remove( )
  - Infoblox::Session->search( )
- METHODS
  - comment( )
  - extattrs( )
  - extensible_attributes( )
  - fingerprint( )
  - name( )
- SAMPLE CODE
- AUTHOR
- SEE ALSO
- COPYRIGHT

NAME

Infoblox::DHCP::Filter::Fingerprint - DHCP Fingerprint Filter object.

DESCRIPTION

DHCP Fingerprint Filter object.

CONSTRUCTOR

```
 my $filter = Infoblox::DHCP::Filter::Fingerprint->new(
    name            => $string,                                  # required
    fingerprint     => [$fingerprint1, $fingerprint2, ...],      # required, reference to an array of
Infoblox::DHCP::Fingerprint objects
    comment         => $string,                                  # optional
    extattrs        => { $string => $extattr, ... },             # optional / Default is undefined
    extensible_attributes => { $string => $string | $num, $string => [ $string | $num, ... ], ... },
#Optional / Default is undefined
 );
```

You cannot set both extattrs and extensible_attributes attributes at the same time.

SESSION METHODS

This section describes all the methods in an Infoblox::Session module that you can apply to DHCP Fingerprint Filter object.

Infoblox::Session->add( )

Use this method to add an object to the Infoblox appliance. See Infoblox::Session->add() for parameters and return values.

Example

```
#Construct an object
my $filter = Infoblox::DHCP::Filter::Fingerprint->new(
   name        => 'fingerprint_filter',
   fingerprint => [$fingerprint],
);

#Submit for addition
my $response = $session->add( $filter );
```

Infoblox::Session->get( )

Use this method to retrieve all the matching objects from the Infoblox appliance. See Infoblox::Session->get() for parameters and return values.

Key References

```
Apply the following attributes to get a specific DHCP Fingerprint Filter object:

 name       - Optional. The name of DHCP Fingerprint Filter object.
 comment    - Optional. The comment.
 extattrs   - Optional. A hash reference containing extensible attributes.
 extensible_attributes - Optional. A hash reference containing extensible attributes.
```

Example

```
# Get DHCP Fingerprint Filter object by name
my @retrieved_objs = $session->get(
   object    => 'Infoblox::DHCP::Filter::Fingerprint',
   name      => 'fingerprint_filter',
);
```

Infoblox::Session->modify( )

Use this method to modify an object in the Infoblox appliance. See Infoblox::Session->modify() for parameters and return values.

Example

```
# Modify comment
$filter->comment("This is modified object");
# Submit modification
my $response = $session->modify( $filter );
```

Infoblox::Session->remove( )

Use this method to remove an object from the Infoblox appliance. See Infoblox::Session->remove() for parameters and return values.

To remove a specific object, first use `get()` or `search()` to retrieve the specific object, and then submit this object for removal.

Example

```
# Get the object by name
my @retrieved_objs = $session->get(
   object   => 'Infoblox::DHCP::Filter::Fingerprint',
   name     => 'fingerprint_filter',
);
#Find the desired object from the retrieved list.
my $desired_filter = $retrieved_objs[0];
# Submit for removal
my $response = $session->remove( $desired_filter );
```

Infoblox::Session->search( )

Use this method to search for DHCP Fingerprint Filter objects in the Infoblox appliance. See Infoblox::Session->search() for parameters and return values.

Key References

```
Apply the following attributes to search for specific DHCP Fingerprint Filter objects:

 name       - Optional. The name of DHCP Fingerprint Filter object (regular expression).
 comment    - Optional. The comment (regular expression).
 extattrs   - Optional. A hash reference containing extensible attributes.
 extensible_attributes - Optional. A hash reference containing extensible attributes.
```

Example

```
# Search DHCP Fingerprint Filter object by name
my @retrieved_objs = $session->search(
   object    => 'Infoblox::DHCP::Filter::Fingerprint',
   name      => 'fingerprint_filter',
);
```

METHODS

This section describes all the methods that you can use to configure and retrieve the attribute values of DHCP Fingerprint Filter objects.

comment( )

Use this method to set or retrieve a descriptive comment.

Parameter

Desired comment in string format with a maximum of 256 bytes.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get attribute value
my $value = $filter->comment();
#Modify attribute value
$filter->comment('new comment');
```

extattrs( )

Use this method to set or retrieve the extensible attributes associated with a DHCP Fingerprint Filter object.

Parameter

Valid value is a hash reference containing the names of extensible attributes and their associated values (Infoblox::Grid::Extattr objects).

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get extattrs
my $ref_extattrs = $filter->extattrs();
#Modify extattrs
$filter->extattrs({ 'Site' => $extattr1, 'Administrator' => $extattr2 });
```

extensible_attributes( )

Use this method to set or retrieve the extensible attributes associated with a DHCP Fingerprint Filter object.

Parameter

For valid values for extensible attributes, see Infoblox::Grid::ExtensibleAttributeDef/Extensible Attribute Values.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get extensible attributes
my $extensible_attributes = $filter->extensible_attributes();
#Modify extensible attributes
$filter->extensible_attributes({ 'Site' => 'Santa Clara', 'Administrator' => [ 'Peter', 'Tom' ] });
```

fingerprint( )

Use this method to set or retrieve a list of DHCP Fingerprint objects.

Parameter

Reference to an array of Infoblox::DHCP::Fingerprint objects.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get attribute value
my $value = $filter->fingerprint();
#Modify attribute value
$filter->fingerprint([$fingerprint]);
```

name( )

Use this method to set or retrieve the name of a DHCP Fingerprint Filter object.

Parameter

The name in string format with a maximum of 256 bytes.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get attribute value
my $value = $filter->name();
#Modify attribute value
$filter->name('new_fingerprint_filter');
```

SAMPLE CODE

The following sample code demonstrates the different functions that can be applied to an object such as add, get, modify, search and remove. This sample also includes error handling for the operations.

#Preparation prior to getting and modifying DHCP Fingerprint Filter object

```
use strict;
use Infoblox;

#refers to Infoblox Appliance IP address
my $host_ip = "192.168.1.2";

#Create a session to the Infoblox appliance

my $session = Infoblox::Session->new(
    master   => $host_ip,
    username => "admin",
    password => "infoblox"
);
unless ($session) {
       die("Construct session failed: ",
               Infoblox::status_code() . ":" . Infoblox::status_detail());
}
print "Session created successfully\n";
```

#Create DHCP Fingerprint Filter object

```
my $fingerprint = Infoblox::DHCP::Fingerprint->new('name' => 'Microsoft Windows 8');
unless ($fingerprint) {
       die("Construct DHCP Fingerprint object failed: ",
               Infoblox::status_code() . ":" . Infoblox::status_detail());
}
print "DHCP Fingerprint object created successfully\n";

my $filter = Infoblox::DHCP::Filter::Fingerprint->new(
   'name'        => 'fingerprint_filter',
   'fingerprint' => [$fingerprint],
);

unless ($filter) {
       die("Construct DHCP Fingerprint Filter object failed: ",
               Infoblox::status_code() . ":" . Infoblox::status_detail());
}
print "DHCP Fingerprint Filter object created successfully\n";

my $response = $session->add($filter);

unless ($response) {
    die("Add DHCP Fingerprint Filter object failed: ",
           $session->status_code() . ":" . $session->status_detail());
}
print"DHCP Fingerprint Filter object added successfully \n";
```

#Search for DHCP Fingerprint Filter object

```
my @retrieved_objs = $session->search(
```

```
    object  => 'Infoblox::DHCP::Filter::Fingerprint',
    name    => 'fingerprint_filter',
);

$filter = $retrieved_objs[0];

unless ($filter) {
        die("Search DHCP Fingerprint Filter object object failed: ",
                $session->status_code() . ":" . $session->status_detail());
}
print "Search DHCP Fingerprint Filter object found at least 1 matching entry\n";
```

#Get and modify DHCP Fingerprint Filter object

```
@retrieved_objs = $session->get(
    object  => 'Infoblox::DHCP::Filter::Fingerprint',
    name    => 'fingerprint_filter',
);

$filter = $retrieved_objs[0];

unless ($filter) {
    die("Get DHCP Fingerprint Filter object failed: ",
            $session->status_code() . ":" . $session->status_detail());
}
print"Get DHCP Fingerprint Filter object successful \n";

$filter->comment("This is modified object");

$session->modify($filter)
    or die("Modify DHCP Fingerprint Filter object failed",
             $session->status_code() . ":" . $session->status_detail());
print "DHCP Fingerprint Filter object modified successfully \n";
```

#Remove DHCP Fingerprint Filter object

```
@retrieved_objs = $session->get(
    object  => 'Infoblox::DHCP::Filter::Fingerprint',
    name    => 'fingerprint_filter',
);

$filter = $retrieved_objs[0];

unless ($filter) {
    die("Get DHCP Fingerprint Filter object failed: ",
            $session->status_code() . ":" . $session->status_detail());
}
print"Get DHCP Fingerprint Filter object successful \n";

$session->remove( $filter )
    or die("Remove DHCP Fingerprint Filter object failed",
            $session->status_code() . ":" . $session->status_detail());
print"DHCP Fingerprint Filter object removed successfully \n";

####PROGRAM ENDS####
```

AUTHOR

Infoblox Inc. http://www.infoblox.com/

SEE ALSO

Infoblox::Session, Infoblox::DHCP::Fingerprint

COPYRIGHT

Copyright (c) 2017 Infoblox Inc.

NAME

- NAME
- DESCRIPTION
- CONSTRUCTOR
- SESSION METHODS
- METHODS
  - id( )
  - comment( )
  - password( )
  - type( )
  - user( )
- AUTHOR
- SEE ALSO
- COPYRIGHT

NAME

Infoblox::Grid::Discovery::CLICredential

DESCRIPTION

The object provides information about CLI credentials.

CONSTRUCTOR

```
my $object = Infoblox::Grid::Discovery::CLICredential->new(
    'user'     => $string,                                          # optional
    'password' => $string,                                          # optional
    'comment'  => $string,                                          # optional
    'type'     => 'ENABLE_SSH' | 'ENABLE_TELNET' | 'SSH' | 'TELNET',   # required
);
```

SESSION METHODS

The object does not support any session methods.

METHODS

This section describes all the methods that you can use to set or retrieve the attribute values of the object.

id( )

Use this method to retrieve the credential ID. This attribute is read-only.

Parameter

None.

Returns

The method returns the attribute value.

Example

```
# Get attribute value
my $value = $object->id();
```

comment( )

Use this method to set or retrieve the descriptive comment.

Parameter

Desired comment in string format.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
# Get attribute value
my $value = $object->comment();
# Modify attribute value
$object->comment('CLI credentials');
```

password( )

Use this method to set the CLI user password. This attribute is write-only.

Parameter

The CLI password in string format.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

Example

```
# Modify attribute value
$object->password('infoblox');
```

type( )

Use this method to set or retrieve the credential type.

Parameter

The valid value is 'ENABLE_SSH', 'ENABLE_TELNET', 'SSH' or 'TELNET'.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
# Get attribute value
my $value = $object->typ();
# Modify attribute value
$object->type('ENABLE_SSH');
```

user( )

Use this method to set or retrieve the CLI user name.

Parameter

The CLI user name in string format.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
# Get attribute value
my $value = $object->user();
# Modify attribute value
$object->user('admin');
```

AUTHOR

Infoblox Inc. http://www.infoblox.com/

SEE ALSO

Infoblox::Grid::Discovery::Properties

COPYRIGHT

Copyright (c) 2017 Infoblox Inc.

Infoblox::Grid::Discovery::Data - the discovered data.

- NAME
- DESCRIPTION
- CONSTRUCTOR
- SESSION METHODS
- METHODS
  - device_model( )
  - device_port_name( )
  - device_port_type( )
  - device_type( )
  - device_vendor( )
  - discovered_name( )
  - discoverer( )
  - duid( )
  - endpoint_groups( )
  - first_discovered( )
  - iprg_no( )
  - iprg_state( )
  - iprg_type( )
  - last_discovered( )
  - mac_address( )
  - mgmt_ip_address( )
  - netbios( )
  - network_component_description( )
  - network_component_ip( )
  - network_component_model( )
  - network_component_name( )
  - network_component_port_description( )
  - network_component_port_name( )
  - network_component_port_number( )
  - network_component_type( )
  - network_component_vendor( )
  - open_ports( )
  - os( )
  - port_duplex( )
  - port_link_status( )
  - port_speed( )
  - port_status( )
  - port_type( )
  - port_vlan_description( )
  - port_vlan_name( )
  - port_vlan_number( )
  - task_name( )
  - tenant( )
  - network_component_location( )
  - ap_name( )
  - ap_ip_address( )
  - ap_ssid( )
  - bridge_domain( )
  - network_component_contact( )
  - device_location( )
  - device_contact( )
  - v_cluster( )
  - v_datacenter( )
  - v_host( )
  - v_name( )
  - v_netadapter( )
  - v_os( )
  - v_switch( )
  - v_type( )
  - vmi_name( )
  - vmi_id( )
  - vmi_tenant_id( )
  - vlan_port_group( )
  - vswitch_name( )
  - vswitch_id( )
  - vswitch_type( )
  - vswitch_ipv6_enabled( )
  - vport_name( )
  - vport_mac_address( )
  - vport_link_status( )
  - vport_conf_speed( )

- vport_conf_mode( )
- vport_speed( )
- vport_mode( )
- vswitch_segment_type( )
- vswitch_segment_name( )
- vswitch_segment_id( )
- vswitch_segment_port_group( )
- vswitch_available_ports_count( )
- vswitch_tep_type( )
- vswitch_tep_ip ( )
- vswitch_tep_port_group( )
- vswitch_tep_vlan( )
- vswitch_tep_dhcp_server( )
- vswitch_tep_multicast( )
- vmhost_ip_address( )
- vmhost_name( )
- vmhost_mac_address ( )
- vmhost_subnet_cidr ( )
- vmhost_nic_names( )
- cisco_ise_endpoint_profile( )
- cisco_ise_security_group( )
- cisco_ise_session_state( )
- cisco_ise_ssid( )
- cmp_type( )
- vmi_ip_type( )
- vmi_private_address( )
- vmi_is_public_address( )

- SAMPLE CODE
- AUTHOR
- COPYRIGHT

NAME

Infoblox::Grid::Discovery::Data - the discovered data.

DESCRIPTION

The object provides information about the discovered data.

CONSTRUCTOR

```
my $object = Infoblox::Grid::Discovery::Data->new(
    'ap_name'                           => $string,
    'ap_ip_address'                     => $ipaddr,
    'ap_ssid'                           => $string,
    'bridge_domain                      => $string,
    'duid'                              => $string,
    'mac_address'                       => $string,
    'netbios'                           => $string,
    'os'                                => $string,
    'device_model'                      => $string,
    'device_port_name'                  => $string,
    'device_port_type'                  => $string,
    'device_type'                       => $string,
    'device_vendor'                     => $string,
    'discovered_name'                   => $string,
    'discoverer'                        => $string,
    'endpoint_groups'                   => $string,
    'first_discovered'                  => $datetime,
    'last_discovered'                   => $datetime,
    'mgmt_ip_address'                   => $ipaddr,
    'network_component_description'     => $string,
    'network_component_ip'              => $ipaddr,
    'network_component_model'           => $string,
    'network_component_name'            => $string,
    'network_component_port_description' => $string,
    'network_component_port_name'       => $string,
    'network_component_port_number'     => $uint,
    'network_component_type'            => $string,
    'network_component_vendor'          => $string,
    'open_ports'                        => $string,
    'port_duplex'                       => 'Full' | 'Half',
```

```
    'port_link_status'                  => 'Connected' | 'Not Connected' | 'Unknown',
    'port_speed'                        => '100G' | '100M' | '10G' | '10M' | '1G' | 'Unknown',
    'port_status'                       => 'Down' | 'Unknown' | 'Up',
    'port_type'                         => $string,
    'port_vlan_description'             => $string,
    'port_vlan_name'                    => $string,
    'port_vlan_number'                  => $uint,
    'task_name'                         => $string,
    'tenant'                            => $string,
    'network_component_location'        => $string,
    'network_component_contact'         => $string,
    'device_location'                   => $string,
    'device_contact'                    => $string,
    'v_netadapter'                      => $string,
    'v_cluster'                         => $string,
    'v_datacenter'                      => $string,
    'v_name'                            => $string,
    'v_type'                            => 'VirtualCenter' | 'HostSystem' | 'VirtualMachine',
    'v_host'                            => $string,
    'v_os'                              => $string,
    'v_switch'                          => $string,
);
```

SESSION METHODS

The object does not support any session methods.

METHODS

This section describes all the methods that you can use to set or retrieve the attribute values of the discovered data object.

device_model( )

Use this method to set or retrieve the model name of the device in the vendor terminology.

Parameter

The model name of the device in string format.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
# Get attribute value
my $value = $object->device_model();
# Modify attribute value
$object->device_model('model');
```

device_port_name( )

Use this method to set or retrieve the system name of the interface the IP associates with.

Parameter

The system name of the interface in string format.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
# Get attribute value
my $value = $object->device_port_name();
# Modify attribute value
$object->device_port_name('port');
```

device_port_type( )

Use this method to set or retrieve the hardware type of the interface the IP associates with.

Parameter

The hardware type of the interface in string format.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
# Get attribute value
my $value = $object->device_port_type();
# Modify attribute value
$object->device_port_type('type');
```

device_type( )

Use this method to set or retrieve the type of the device in the vendor terminology.

Parameter

The type of the device in string format.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
# Get attribute value
my $value = $object->device_type();
# Modify attribute value
$object->device_type('type');
```

device_vendor( )

Use this method to set or retrieve the vendor name of the device.

Parameter

The vendor name of the device in string format.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
# Get attribute value
my $value = $object->device_vendor();
# Modify attribute value
$object->device_vendor('vendor');
```

discovered_name( )

Use this method to set or retrieve the name of the IP as seen by the discovery station.

Parameter

The discovered name in string format.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
# Get attribute value
```

```
my $value = $object->discovered_name();
# Modify attribute value
$object->discovered_name('name');
```

discoverer( )

Use this method to set or retrieve the name of the discoverer.

Parameter

The name of the discoverer in string format.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
# Get attribute value
my $value = $object->discoverer();
# Modify attribute value
$object->discoverer('discoverer');
```

duid( )

Use this method to set or retrieve the discovered DHCPv6 UID (DUID).

Parameter

The discovered DHCPv6 UID (DUID) in string format.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
# Get attribute value
my $value = $object->duid();
# Modify attribute value
$object->duid('aa:aa');
```

endpoint_groups( )

Use this method to retrieve comma-separated list of discovered endpoint groups. This is a read-only attribute.

Parameter

None

Returns

The method returns the attribute value.

Example

```
# Get attribute value
my $value = $object->endpoint_groups();
```

first_discovered( )

Use this method to set or retrieve the date and time when the object was first discovered.

Parameter

The date and time in the ISO 8601 extended format (e.g. 2008-10-21T10:56:33Z or 2008-10-21T10:56:33.001Z).

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
# Get attribute value
my $value = $object->first_discovered();
# Modify attribute value
$object->first_discovered('2008-10-21T10:56:33Z');
```

iprg_no( )

Use this method to set or retrieve the port redundant group number.

Parameter

Valid value is an unsigned integer which describes port redundant group number.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
# Get attribute value
my $value = $object->iprg_no();
# Modify attribute value
$object->iprg_no(10);
```

iprg_state( )

Use this method to set or retrieve the status for the IP address within port redundant group.

Parameter

Valid value is "VIP", "ACTIVE", "NEGOTIATION" or "STANDBY".

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
# Get attribute value
my $value = $object->iprg_state();
# Modify attribute value
$object->iprg_state('ACTIVE');
```

iprg_type( )

Use this method to set or retrieve the port redundant group type.

Parameter

Valid value is "HSRP" or "VRRP".

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
# Get attribute value
my $value = $object->iprg_type();
# Modify attribute value
$object->iprg_type('HSRP');
```

last_discovered( )

Use this method to set or retrieve the date and time when the object was last discovered.

Parameter

The date and time in the ISO 8601 extended format (e.g. 2008-10-21T10:56:33Z or 2008-10-21T10:56:33.001Z).

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
# Get attribute value
my $value = $object->last_discovered();
# Modify attribute value
$object->last_discovered('2008-10-21T10:56:33Z');
```

mac_address( )

Use this method to set or retrieve the MAC address.

Parameter

The MAC address in string format.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
# Get attribute value
my $value = $object->mac_address();
# Modify attribute value
$object->mac_address('11:11:11:11:11:11');
```

mgmt_ip_address( )

Use this method to set or retrieve the management IP address of the device if the device has more than one IP.

Parameter

The management IP address of the device.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
# Get attribute value
my $value = $object->mgmt_ip_address();
# Modify attribute value
$object->mgmt_ip_address('10.0.0.1');
```

netbios( )

Use this method to set or retrieve the discovered NetBIOS name.

Parameter

The discovered NetBIOS name in string format.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
# Get attribute value
my $value = $object->netbios();
# Modify attribute value
$object->netbios('netbios');
```

network_component_description( )

Use this method to set or retrieve the descriptive string for the network component.

Parameter

The descriptive string for the network component.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
# Get attribute value
my $value = $object->network_component_description();
# Modify attribute value
$object->network_component_description('description');
```

network_component_ip( )

Use this method to set or retrieve the IP Address the network component.

Parameter

The IP Address the network component.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
# Get attribute value
my $value = $object->network_component_ip();
# Modify attribute value
$object->network_component_ip('10.0.0.2');
```

network_component_model( )

Use this method to set or retrieve the model name of the network component.

Parameter

The model name of the network component in string format.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
# Get attribute value
my $value = $object->network_component_model();
# Modify attribute value
$object->network_component_model('model');
```

network_component_name( )

Use this method to set or retrieve the name of the network component.

Parameter

The name of the network component in string format.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
# Get attribute value
my $value = $object->network_component_name();
# Modify attribute value
$object->network_component_name('name');
```

network_component_port_description( )

Use this method to set or retrieve the description of the port on the network component.

Parameter

The description of the port on the network component in string format.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
# Get attribute value
my $value = $object->network_component_port_description();
# Modify attribute value
$object->network_component_port_description('description');
```

network_component_port_name( )

Use this method to set or retrieve the port name on the network component on which the IP was discovered.

Parameter

The port name on the network component in string format.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
# Get attribute value
my $value = $object->network_component_port_name();
# Modify attribute value
$object->network_component_port_name('port');
```

network_component_port_number( )

Use this method to set or retrieve the port number on the network component on which the IP was discovered.

Parameter

The port number on the network component.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
# Get attribute value
my $value = $object->network_component_port_number();
# Modify attribute value
$object->network_component_port_number(10);
```

network_component_type( )

Use this method to set or retrieve the type of network component.

Parameter

The type of network component in string format.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
# Get attribute value
my $value = $object->network_component_type();
# Modify attribute value
$object->network_component_type('type');
```

network_component_vendor( )

Use this method to set or retrieve the vendor name of the network component to which the device is connected.

Parameter

The vendor name of the network component in string format.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
# Get attribute value
my $value = $object->network_component_vendor();
# Modify attribute value
$object->network_component_vendor('vendor');
```

open_ports( )

Use this method to set or retrieve the list of opened ports on the IP address, represented as: "TCP: 21,22,23 UDP: 137,139".

Parameter

The list of opened ports in string format.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
# Get attribute value
my $value = $object->open_ports();
# Modify attribute value
$object->open_ports('TCP: 21,22,23 UDP: 137,139');
```

os( )

Use this method to set or retrieve the name of the operation system by network discovery.

Parameter

The name of the operation system in string format.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
# Get attribute value
my $value = $object->os();
# Modify attribute value
$object->os('linux');
```

port_duplex( )

Use this method to set or retrieve duplex settings on the port on the network component.

Parameter

Valid value is 'Full' or 'Half'.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
# Get attribute value
my $value = $object->port_duplex();
# Modify attribute value
$object->port_duplex('Full');
```

port_link_status( )

Use this method to set or retrieve the link status of the port on the network component.

Parameter

Valid value is 'Connected', 'Not Connected' or 'Unknown'.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
# Get attribute value
my $value = $object->port_link_status();
# Modify attribute value
$object->port_link_status('Connected');
```

port_speed( )

Use this method to set or retrieve speed settings on the port on the network component.

Parameter

Valid value is '10M', '100M', '1G', '10G', '100G' or 'Unknown'.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
# Get attribute value
my $value = $object->port_speed();
# Modify attribute value
$object->port_speed('Unknown');
```

port_status( )

Use this method to set or retrieve the status of the port on the network component.

Parameter

Valid value is 'Up', 'Down' or 'Unknown'.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
# Get attribute value
my $value = $object->port_status();
# Modify attribute value
$object->port_status('Unknown');
```

port_type( )

Use this method to set or retrieve the type of the interface on the network component to which the device is connected.

Parameter

The type of the interface on the network component in string format.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
# Get attribute value
my $value = $object->port_type();
# Modify attribute value
$object->port_type('type');
```

port_vlan_description( )

Use this method to set or retrieve the description of the VLAN on the port on the network component.

Parameter

The description of the VLAN on the port on the network component in string format.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
# Get attribute value
my $value = $object->port_vlan_description();
# Modify attribute value
$object->port_vlan_description('description');
```

port_vlan_name( )

Use this method to set or retrieve the name of the VLAN on the port on the network component.

Parameter

The name of the VLAN on the port on the network component in string format.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
# Get attribute value
my $value = $object->port_vlan_name();
# Modify attribute value
$object->port_vlan_name('name');
```

port_vlan_number( )

Use this method to set or retrieve the number VLANs on the port on the network component.

Parameter

The number of VLANs on the port on the network component.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
# Get attribute value
my $value = $object->port_vlan_number();
# Modify attribute value
$object->port_vlan_number(10);
```

task_name( )

Use this method to retrieve the name of discovery task. This is a read-only attribute.

Parameter

None

Returns

The method returns the attribute value.

Example

```
# Get attribute value
my $value = $object->task_name();
```

tenant( )

Use this method to retrieve the discovered tenant. This is a read-only attribute.

Parameter

None

Returns

The method returns the attribute value.

Example

```
# Get attribute value
my $value = $object->tenant();
```

network_component_location( )

Use this method to retrieve location of the network component on which the IP address was discovered. This is a read-only attribute.

Parameter

None

Returns

The method returns the attribute value.

Example

```
# Get attribute value
my $value = $object->network_component_location();
```

ap_name( )

Use this method to retrieve discovered name of Wireless Access Point. This is a read-only attribute.

Parameter

None

Returns

The method returns the attribute value.

Example

```
# Get attribute value
my $value = $object->ap_name();
```

ap_ip_address( )

Use this method to retrieve discovered IP address of Wireless Access Point. This is a read-only attribute.

Parameter

None

Returns

The method returns the attribute value.

Example

```
# Get attribute value
my $value = $object->ap_ip_address();
```

ap_ssid( )

Use this method to retrieve service set identifier (SSID) associated with Wireless Access Point. This is a read-only attribute.

Parameter

None

Returns

The method returns the attribute value.

Example

```
# Get attribute value
my $value = $object->ap_ssid();
```

bridge_domain( )

Use this method to retrieve the discovered bridge domain. This is a read-only attribute.

Parameter

None

Returns

The method returns the attribute value.

Example

```
# Get attribute value
my $value = $object->bridge_domain();
```

network_component_contact( )

Use this method to retrieve contact information from the network component on which the IP address was discovered. This is a read-only attribute.

Parameter

None

Returns

The method returns the attribute value.

Example

```
# Get attribute value
my $value = $object->network_component_contact();
```

device_location( )

Location of device on which the IP address was discovered. This is a read-only attribute.

Parameter

None

Returns

The method returns the attribute value.

Example

```
# Get attribute value
my $value = $object->device_location();
```

device_contact( )

Contact information from device on which the IP address was discovered. This is a read-only attribute.

Parameter

None

Returns

The method returns the attribute value.

Example

```
# Get attribute value
my $value = $object->device_contact();
```

v_cluster( )

Use this method to set or retrieve the VMware cluster in the entity found.

Parameter

The VMware cluster in string format.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
# Get attribute value
my $value = $object->v_cluster();
# Modify attribute value
$object->v_cluster('cluster');
```

v_datacenter( )

Use this method to set or retrieve the VMware datacenter in the entity found.

Parameter

The VMware datacenter in string format.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
# Get attribute value
my $value = $object->v_datacenter();
# Modify attribute value
$object->v_datacenter('datacenter');
```

v_host( )

Use this method to set or retrieve the VMware host system in the entity found.

Parameter

The VMware host system in string format.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
# Get attribute value
my $value = $object->v_host();
# Modify attribute value
$object->v_host('host');
```

v_name( )

Use this method to set or retrieve the VMware entity name in the entity found.

Parameter

The VMware entity name in string format.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
# Get attribute value
my $value = $object->v_name();
# Modify attribute value
$object->v_name('name');
```

v_netadapter( )

Use this method to set or retrieve the VMware physical adapter in the entity found.

Parameter

The VMware physical adapter in string format.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
# Get attribute value
my $value = $object->v_netadapter();
# Modify attribute value
$object->v_netadapter('netadapter');
```

v_os( )

Use this method to set or retrieve the VMware Operating System in the entity found.

Parameter

The VMware Operating System in string format.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
# Get attribute value
my $value = $object->v_os();
# Modify attribute value
$object->v_os('linux');
```

v_switch( )

Use this method to set or retrieve the VMware virtual switch in the entity found.

Parameter

The VMware virtual switch in string format.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
# Get attribute value
my $value = $object->v_switch();
```

```
# Modify attribute value
$object->v_switch('switch');
```

v_type( )

Use this method to set or retrieve the type of encryption to use.

Parameter

Valid value is 'VirtualCenter', 'HostSystem' or 'VirtualMachine'.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
# Get attribute value
my $value = $object->v_type();
# Modify attribute value
$object->v_type('VirtualMachine');
```

vmi_name( )

Use this method to retrieve the name of the virtual machine. This is a read-only attribute.

Parameter

None

Returns

The method returns the attribute value.

Example

```
# Get attribute value
my $value = $object->vmi_name();
```

vmi_id( )

Use this method to retrieve the ID of the virtual machine. This is a read-only attribute.

Parameter

None

Returns

The method returns the attribute value.

Example

```
# Get attribute value
my $value = $object->vmi_id();
```

vmi_tenant_id( )

Use this method to retrieve the Tenant ID to which the virtual machine belongs. This is a read-only attribute

Parameter

None

Returns

The method returns the attribute value.

Example

```
# Get attribute value
my $value = $object->vmi_tenant_id();
```

vlan_port_group( )

Use this method to retrieve the port group to which the virtual machine belongs. This is a read-only attribute.

Parameter

None

Returns

The method returns the attribute value.

Example

```
# Get attribute value
my $value = $object->vlan_port_group();
```

vswitch_name( )

Use this method to retrieve the name of the virtual switch. This is a read-only attribute.

Parameter

None

Returns

The method returns the attribute value.

Example

```
# Get attribute value
my $value = $object->vswitch_name();
```

vswitch_id( )

Use this method to retrieve the ID of the virtual switch. This is a read-only attribute.

Parameter

None

Returns

The method returns the attribute value.

Example

```
# Get attribute value
my $value = $object->vswitch_id();
```

vswitch_type( )

Use this method to retrieve the type of the virtual switch: Standard or Distributed. 'Unknown' indicates the value is not avialable. This is a read-only attribute.

Parameter

None

Returns

The method returns the attribute value.

Example

```
# Get attribute value
my $value = $object->vswitch_type();
```

vswitch_ipv6_enabled( )

Use this method to retrieve the flag used to indicate if the virtual swich has IPv6 enabled or not. This is a read-only attribute.

Parameter

None

Returns

The method returns the attribute value.

Example

```
# Get attribute value
my $value = $object->vswitch_ipv6_enabled();
```

vport_name( )

Use this method to retrieve the name of the network adapter on the virtual switch to which the virtual machine is connected. This is a read-only attribute.

Parameter

None

Returns

The method returns the attribute value.

Example

```
# Get attribute value
my $value = $object->vport_name();
```

vport_mac_address( )

Use this method to retrieve the MAC address of the network adapter on the virtual switch to which the the virtual machine is connected. This is a read-only attribute.

Parameter

None

Returns

The method returns the attribute value.

Example

```
# Get attribute value
my $value = $object->vport_mac_address();
```

vport_link_status( )

Use this method to retrieve the status of the link of the network adapter on the virtual switch to which the virtual machine is connected. This is a read-only attribute.

Parameter

None

Returns

The method returns the attribute value.

Example

```
# Get attribute value
my $value = $object->vport_link_status();
```

vport_conf_speed( )

Use this method to retrieve the configured speed (in Kb) of the network adapter on the virtual switch to which the virtual machine is connected. Example: 1500Kb. This is a read-only attribute.

Parameter

None

Returns

The method returns the attribute value.

Example

```
# Get attribute value
my $value = $object->vport_conf_speed();
```

vport_conf_mode( )

Use this method to retrieve the configured mode of the network adapter on the virtual switch to which the virtual machine is connected. Valid value is Full-duplex or Half-duplex. 'Unknown' indicates the value is not available. This is a read-only attribute.

Parameter

None

Returns

The method returns the attribute value.

Example

```
# Get attribute value
my $value = $object->vport_conf_mode();
```

vport_speed( )

Use this method to retrieve the actual speed (in Kb) of the network adapter on the virtual switch to which the virtual machine is connected. Example: 1500Kb. This is a read-only attribute.

Parameter

None

Returns

The method returns the attribute value.

Example

```
# Get attribute value
my $value = $object->vport_speed();
```

vport_mode( )

Use this method to retrieve the actual mode of the network adapter on the virtual switch to which the virtual machine is connected. Valid value is Full-duplex or Half-duplex. 'Unknown' indicates the value is not available. This is a read-only attribute.

Parameter

None

Returns

The method returns the attribute value.

Example

```
# Get attribute value
my $value = $object->vport_mode();
```

vswitch_segment_type( )

Use this method to retrieve the type of the network segment to which the current virtual machine/vport is connected. This is a read-only attribute.

Parameter

None

Returns

The method returns the attribute value.

Example

```
# Get attribute value
my $value = $object->vswitch_segment_type();
```

vswitch_segment_name( )

Use this method to retrieve the name of the network segment to which the current virtual machine/vport is connected. This is a read-only attribute.

Parameter

None

Returns

The method returns the attribute value.

Example

```
# Get attribute value
my $value = $object->vswitch_segment_name();
```

vswitch_segment_id( )

Use this method to retrieve the ID of the network segment to which the current virtual machine/vport is connected. This is a read-only attribute.

Parameter

None

Returns

The method returns the attribute value.

Example

```
# Get attribute value
my $value = $object->vswitch_segment_id();
```

vswitch_segment_port_group( )

Use this method to retrieve the port group of the network segment to which the current virtual machine/vport is connected. This is a read-only attribute.

Parameter

None

Returns

The method returns the attribute value.

Example

```
# Get attribute value
my $value = $object->vswitch_segment_port_group();
```

vswitch_available_ports_count( )

Use this method to retrieve the number of available ports reported by the virtual switch to which the virtual machine/vport is connected. This is a read-only attribute.

Parameter

None

Returns

The method returns the attribute value.

Example

```
# Get attribute value
my $value = $object->vswitch_available_ports_count();
```

vswitch_tep_type( )

Use this method to retrieve the type of virtual tunnel endpoint (VTEP) in the virtual switch. This is a read-only attribute.

Parameter

None

Returns

The method returns the attribute value.

Example

```
# Get attribute value
my $value = $object->vswitch_tep_type();
```

vswitch_tep_ip ( )

Use this method to retrieve the IP address of the virtual tunnel endpoint (VTEP) in the virtual switch. This is a read-only attribute.

Parameter

None

Returns

The method returns the attribute value.

Example

```
# Get attribute value
my $value = $object->vswitch_tep_ip ();
```

vswitch_tep_port_group( )

Use this method to retrieve the port group of the virtual tunnel endpoint (VTEP) in the virtual switch. This is a read-only attribute.

Parameter

None

Returns

The method returns the attribute value.

Example

```
# Get attribute value
my $value = $object->vswitch_tep_port_group();
```

vswitch_tep_vlan( )

Use this method to retrieve the VLAN of the virtual tunnel endpoint (VTEP) in the virtual switch. This is a read-only attribute.

Parameter

None

Returns

The method returns the attribute value.

Example

```
# Get attribute value
my $value = $object->vswitch_tep_vlan();
```

vswitch_tep_dhcp_server( )

Use this method to retrieve the DHCP server of the virtual tunnel endpoint (VTEP) in the virtual switch. This is a read-only attribute.

Parameter

None

Returns

The method returns the attribute value.

Example

```
# Get attribute value
my $value = $object->vswitch_tep_dhcp_server();
```

vswitch_tep_multicast( )

Use this method to retrieve the multicast address of the virtual tunnel endpoint (VTEP) in the virtual swtich. This is a read-only attribute.

Parameter

None

Returns

The method returns the attribute value.

Example

```
# Get attribute value
my $value = $object->vswitch_tep_multicast();
```

vmhost_ip_address( )

Use this method to retrieve the IP address of the physical node on which the virtual machine is hosted. This is a read-only attribute.

Parameter

None

Returns

The method returns the attribute value.

Example

```
# Get attribute value
my $value = $object->vmhost_ip_address ();
```

vmhost_name( )

Use this method to retrieve the name of the physical node on which the virtual machine is hosted. This is a read-only attribute.

Parameter

None

Returns

The method returns the attribute value.

Example

```
# Get attribute value
my $value = $object->vmhost_name();
```

vmhost_mac_address ( )

Use this method to retrieve the MAC address of the physical node on which the virtual machine is hosted. This is a read-only attribute.

Parameter

None

Returns

The method returns the attribute value.

Example

```
# Get attribute value
my $value = $object->vmhost_mac_address ();
```

vmhost_subnet_cidr ( )

Use this method to retrieve the CIDR subnet of the physical node on which the virtual machine is hosted. This is a read-only attribute.

Parameter

None

Returns

The method returns the attribute value.

Example

```
# Get attribute value
my $value = $object->vmhost_subnet_cidr ();
```

vmhost_nic_names( )

Use this method to retrieve the list of all physical port names used by the virtual switch on the physical node on which the virtual machine is hosted. Represented as: "eth1,eth2,eth3". This is a read-only attribute.

Parameter

None

Returns

The method returns the attribute value.

Example

```
# Get attribute value
my $value = $object->vmhost_nic_names();
```

cisco_ise_endpoint_profile( )

Use this method to retrieve a name of the endpoint profile created in Cisco ISE. This is a read-only attribute.

Parameter

None

Returns

The method returns the attribute value.

Example

```
# Get attribute value
my $value = $object->cisco_ise_endpoint_profile();
```

cisco_ise_security_group( )

Use this method to retrieve the Cisco ISE security group. This is a read-only attribute.

Parameter

None

Returns

The method returns the attribute value.

Example

```
# Get attribute value
my $value = $object->cisco_ise_security_group();
```

cisco_ise_session_state( )

Use this method to retrieve the Cisco ISE session state. This is a read-only attribute.

Parameter

None

Returns

The method returns the attribute value.

Example

```
# Get attribute value
my $value = $object->cisco_ise_session_state();
```

cisco_ise_ssid( )

Use this method to retrieve the Cisco ISE SSID. This is a read-only attribute.

Parameter

None

Returns

The method returns the attribute value.

Example

```
# Get attribute value
my $value = $object->cisco_ise_ssid();
```

cmp_type( )

Use this method to retrieve the Cloud Management Platform type. This is a read-only attribute.

Parameter

None

Returns

The method returns the attribute value.

Example

```
# Get attribute value
my $value = $object->cmp_type();
```

vmi_ip_type( )

Use this method to retrieve the discovered IP address type. This is a read-only attribute.

Parameter

None

Returns

The method returns the attribute value.

Example

```
# Get attribute value
my $value = $object->vmi_ip_type();
```

vmi_private_address( )

Use this method to retrieve the private IP address of the virtual machine. This is a read-only attribute.

Parameter

None

Returns

The method returns the attribute value.

Example

```
# Get attribute value
my $value = $object->vmi_private_address();
```

vmi_is_public_address( )

Use this method to retrieve the flag used to indicate whether the IP address is a public address or not. This is a read-only attribute.

Parameter

None

Returns

The method returns the attribute value.

Example

```
# Get attribute value
my $value = $object->vmi_is_public_address();
```

SAMPLE CODE

The following sample code demonstrates the different functions that can be applied to a discovered data object.

#Preparation

```
use strict;
use Infoblox;

#Create a session to the Infoblox appliance
my $host_ip   = "192.168.1.2";
my $host_name = "infoblox.localdomain";
my $session = Infoblox::Session->new(
    master   => $host_ip,       #appliance host ip
    username => "admin",        #appliance user login
    password => "infoblox"      #appliance password
);
unless ($session) {
   die("Construct session failed: ".
        Infoblox::status_code() . ":" . Infoblox::status_detail());
}
print "Session created successfully.\n";
```

#Get an Infoblox::Grid::Discovery::Data object

```
my @retrieved_objs = $session->get('object' => 'Infoblox::DHCP::FixedAddr', 'ipv4addr' => '10.0.0.5');
my $fa = $retrieved_objs[0];
unless ($fa) {
   die("Get FixedAddr object failed: ".
       $session->status_code() . ":" . $session->status_detail());
}

my $data = $fa->discovered_data();
unless ($data) {
   die("FixedAddr object doesn't contain the discovered data");
}
```

#Search by discovered data

```
$data = Infoblox::Grid::Discovery::Data->new(
   'netbios' => 'Fixed*',
   'v_type'  => 'VirtualCenter',
);
unless ($data) {
   die("Construct Infoblox::Grid::Discovery::Data object failed: ".
        Infoblox::status_code() . ":" . Infoblox::status_detail());
}

@retrieved_objs = $session->get('object' => 'Infoblox::DHCP::FixedAddr', 'discovered_data' => $data);
$fa = $retrieved_objs[0];
unless ($fa) {
   die("Get FixedAddr object failed: ".
       $session->status_code() . ":" . $session->status_detail());
}

####PROGRAM ENDS####
```

AUTHOR

Infoblox Inc. http://www.infoblox.com/

COPYRIGHT

Copyright (c) 2017 Infoblox Inc.

Infoblox::DHCP::Filter::MAC - DHCP MAC Address Filter object.

- NAME
- DESCRIPTION
- CONSTRUCTOR
- SESSION METHODS
  - Infoblox::Session->add( )
  - Infoblox::Session->get( )
  - Infoblox::Session->modify( )
  - Infoblox::Session->remove( )
  - Infoblox::Session->search( )
- METHODS
  - comment( )
  - default_mac_address_expiration( )
  - disable( )
  - enforce_expiration_times( )
  - extattrs( )
  - extensible_attributes( )
  - lease_time( )
  - name( )
  - option_list( )
  - reserved_for_infoblox( )
- SAMPLE CODE
- AUTHOR
- SEE ALSO
- COPYRIGHT

NAME

Infoblox::DHCP::Filter::MAC - DHCP MAC Address Filter object.

DESCRIPTION

An Infoblox appliance can filter address requests by the MAC address and/or vendor prefix (i.e., the first 6 hexadecimal characters in MAC address) of a requesting host. The filter instructs the infoblox appliance either to grant or deny an address request if the requesting host matches the filter.

CONSTRUCTOR

```
 my $mac_filter = Infoblox::DHCP::Filter::MAC->new(
       name                            => $string,                    #Required
       comment                         => $string,                    #Optional / Default is empty
       default_mac_address_expiration => $num,                        #Optional / Default is 0 for never
expire
       enforce_expiration_times        => "true" | "false",           #Optional / Default is "true"
       extattrs                        => { $string => $extattr, ... },      #Optional / Default is
undefined
       extensible_attributes           => { $string => $string | $num, $string => [ $string | $num, ... ],
... }, #Optional / Default is undefined
       lease_time                      => $time,                      #Optional / Default is empty
       option_list                     => [$Option1, $Option2,...],   #Optional / Default is empty
       reserved_for_infoblox           => $string,                    #Optional / Default is empty
 );
```

You cannot set both extattrs and extensible_attributes attributes at the same time.

SESSION METHODS

This section describes all the methods in Infoblox::Session module that can be applied to a DHCP MAC Address Filter object.

Infoblox::Session->add( )

Use this method to add a DHCP MAC Address Filter object to the Infoblox appliance. See Infoblox::Session->add() for parameters and return values.

Example

```
       #Construct a DHCP MAC Address Filter object
       my $mac_filter = Infoblox::DHCP::Filter::MAC->new (
             name                              => "filter_script",
```

```
        comment                         => "Created through script",
        default_mac_address_expiration => "70",
        enforce_expiration_times        => "true",
        reserved_for_infoblox           => "This is reserved for infoblox",
);
#Submit for addition
my $response = $session->add($mac_filter)
```

Infoblox::Session->get( )

Use this method to retrieve all the matching objects from the Infoblox appliance. See Infoblox::Session->get() for parameters and return values.

Key References

```
Apply the following attributes to get a specific DHCP MAC Address Filter object:

 name - Required. Name of the MAC Address Filter.
 extattrs      - Optional. A hash reference containing extensible attributes.
 extensible_attributes - Optional. A hash reference containing extensible attributes.
```

Example

```
#Get DHCP MAC Address Filter object through the session
my @retrieved_objs = $session->get(
    object => "Infoblox::DHCP::Filter::MAC",
    name   => "filter_script",
);

# get all objects with a given extensible attribute
my @retrieved_objs = $session->get(
    object                => "Infoblox::DHCP::Filter::MAC",
    extensible_attributes => { 'Site' => 'Santa Clara' }
);
```

Infoblox::Session->modify( )

Use this method to modify a DHCP MAC Address Filter object in the Infoblox appliance. See Infoblox::Session->modify() for parameters and return values.

Example

```
#Use method to modify the default_mac_address_expiration
$mac_filter->default_mac_address_expiration("1234");
#Submit modification
my $response = $session->modify( $mac_filter );
```

Infoblox::Session->remove( )

Use this method to remove a DHCP MAC Address Filter object from the Infoblox appliance. See Infoblox::Session->remove() for parameters and return values.

To remove a specific object, first use `get()` or `search()` to retrieve the specific DHCP MAC Address Filter object, then submit this object for removal.

Example

```
#Get DHCP MAC Address Filter object through the session
my @retrieved_objs = $session->get(
    object => "Infoblox::DHCP::Filter::MAC",
    name   => "filter_script",
);
#Find the desired object from the retrieved list.
my $desired_mac_filter = $retrieved_objs[0];
#Submit for removal
my $response = $session->remove( $desired_mac_filter );
```

Infoblox::Session->search( )

Use this method to search for DHCP MAC Address Filter objects in the Infoblox appliance. See Infoblox::Session->search() for parameters and return values.

Key References

```
Apply the following attributes to search for a specific DHCP MAC Address Filter object:

 name - Name of the MAC Address Filter (regular expression).
 comment - Comment string for the MAC Address Filter (regular expression).
```

```
 extattrs     - Optional. A hash reference containing extensible attributes.
 extensible_attributes - Optional. A hash reference containing extensible attributes.

Either name or comment must be specified.
```

For more information about searching extensible attributes, see Infoblox::Grid::ExtensibleAttributeDef/Searching Extensible Attributes.

Example

```
# search for all DHCP MAC Address Filter objects whose name starts with "filter".
my @retrieved_objs = $session->search(
    object  => "Infoblox::DHCP::Filter::MAC",
    name    => "filter.*",
);

# search for all DHCP MAC Address Filter objects whose comment starts with "user1".
my @retrieved_objs = $session->search(
    object  => "Infoblox::DHCP::Filter::MAC",
    comment    => "user1.*",
);

# search all DHCP MAC Address Filter associations with the extensible attribute 'Site'
my @retrieved_objs = $session->search(
   object => "Infoblox::DHCP::Filter::MAC",
   extensible_attributes => { 'Site' => 'Santa Clara' });
```

METHODS

This section describes all the methods that can be used to set and retrieve the attribute values of a DHCP MAC Address Filter object.

comment( )

Use this method to set or retrieve a descriptive comment of a DHCP MAC Address Filter object.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

Desired comment in string format with a maximum of 256 bytes.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get comment
my $comment = $mac_filter->comment();
#Modify comment
$mac_filter->comment("Modified DHCP MAC Address Filter comment");
```

default_mac_address_expiration( )

Use this method to set or retrieve the default mac address expiration time of a DHCP MAC Address Filter object.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

By default, the MAC address filter never expires; otherwise, it is the absolute interval when the MAC address filter expires. The maximum value can extend up to 4294967295 secs. The minimum value is 60 secs (1 min).

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get default_mac_address_expiration
my $default_mac_address_expiration = $mac_filter->default_mac_address_expiration();
#Modify default_mac_address_expiration
$mac_filter->default_mac_address_expiration("1234");
```

disable( )

Use this method to set or retrieve the disable flag of a DHCP MAC Address Filter object.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

Specify "true" to set the disable flag or "false" to deactivate/unset it. The default value for this field is false.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get disable
my $disable = $mac_filter->disable();
#Modify disable
$mac_filter->disable("true");
```

enforce_expiration_times( )

Use this method to set or retrieve the enforce_expiration_times flag of a DHCP MAC Address Filter object.

Determines whether lease requests will be denied from expired MAC Addresses.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

Specify "true" to set the expired flag or "false" to deactivate/unset it. Default value is "true".

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get enforce_expiration_times
my $enforce_expiration_times = $mac_filter->enforce_expiration_times();
#Modify enforce_expiration_times
$mac_filter->enforce_expiration_times("false");
```

extattrs( )

Use this method to set or retrieve the extensible attributes associated with a DHCP MAC Address Filter object.

Parameter

Valid value is a hash reference containing the names of extensible attributes and their associated values (Infoblox::Grid::Extattr objects).

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get extattrs
my $ref_extattrs = $mac_filter->extattrs();
#Modify extattrs
$mac_filter->extattrs({ 'Site' => $extattr1, 'Administrator' => $extattr2 });
```

extensible_attributes( )

Use this method to set or retrieve the extensible attributes associated with a DHCP MAC Address Filter object.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

For valid values for extensible attributes, see Infoblox::Grid::ExtensibleAttributeDef/Extensible Attribute Values.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get extensible attributes
my $ref_extensible_attributes = $mac_filter->extensible_attributes();
#Modify extensible attributes
$mac_filter->extensible_attributes({ 'Site' => 'Santa Clara', 'Administrator' => [ 'Peter', 'Tom' ]
});
```

lease_time( )

Use this method to set or retrieve a lease_time attribute of a DHCP MAC address filter object.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

Enter appropriate values in seconds.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
# Get lease_time
my $lease_time = $mac_filter->lease_time();
# Modify lease_time
$mac_filter->lease_time("7200");
```

name( )

Use this method to set or retrieve the name of a DHCP MAC Address Filter object.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

Name of the DHCP MAC Address filter. Maximum length up to 1024 bytes is supported.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get name
my $name = $mac_filter->name();
#Modify name
$mac_filter->name("filter1");
```

option_list( )

Use this method to set or retrieve the option_list of a DHCP MAC address filter object.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

The valid value is an array reference that contains the Infoblox::DHCP::Option objects. The option list describes filter option configuration settings and various services.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get option_list
my $options = $mac_filter->option_list();
#Modify option_list
$mac_filter->option_list([$option1]);
```

reserved_for_infoblox( )

Set this flag to reserve the MAC address filter for an Infoblox appliance.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

Reserved to write comment related to this particular MAC address filter. The length of comment cannot exceed 1024 bytes long.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get reserved_for_infoblox
my $reserved_for_infoblox = $mac_filter->reserved_for_infoblox();
#Modify reserved_for_infoblox
$mac_filter->reserved_for_infoblox("reserved for Infoblox");
```

SAMPLE CODE

The following sample code demonstrates the different functions that can be applied to an object such as add, search, modify, and remove. Also, this sample includes error handling for the operations.

#Preparation prior to a DHCP MAC Address Filter object insertion

```
#PROGRAM STARTS: Include all the modules that will be used
use strict;
use Infoblox;

#Create a session to the Infoblox appliance

my $session = Infoblox::Session->new(
               master   => "192.168.1.2", #appliance host ip
               username => "admin",       #appliance user login
               password => "infoblox"     #appliance password
);

unless ($session) {
       die("Construct session failed: ",
               Infoblox::status_code() . ":" . Infoblox::status_detail());
}
print "Session created successfully\n";
```

#Create a DHCP MAC Address Filter object

```
#Create a DHCP MAC Address Filter object
my $mac_filter = Infoblox::DHCP::Filter::MAC->new (
       name                          => "script_filter",
       comment                       => "Create through script",
       default_mac_address_expiration => "70",
       enforce_expiration_times      => "true",
       reserved_for_infoblox         => "This is reserved for infoblox",
);

unless($mac_filter) {
     die("Construct mac address filter failed: ",
           Infoblox::status_code() . ":" . Infoblox::status_detail());
}
print "MAC Address Filter object created successfully\n";

#Verify if the DHCP MAC Address Filter exists in the Infoblox appliance
my $object = $session->get(object => "Infoblox::DHCP::Filter::MAC", name => "script_filter");
unless ($object) {
   print "Mac address filter does not exist on server, safe to add the mac address filter\n";
   $session->add($mac_filter)
      or die("Add mac address filter failed: ",
```

```
            $session->status_code() . ":" . $session->status_detail());
}
print "DHCP MAC Address Filter added successfully\n";
```

#Search for a specific DHCP MAC Address Filter object

```
#Search for all DHCP MAC address filter objects
my @retrieved_objs = $session->search(
    object => "Infoblox::DHCP::Filter::MAC",
    name   => ".*" );
my $object = $retrieved_objs[0];

unless ($object) {
       die("Search for MAC Address Filter object failed: ",
              $session->status_code() . ":" . $session->status_detail());
}
print "Search MAC Address Filter object found at least 1 matching entry\n";
```

#Get and modify a MAC Address Filter object

```
#Get MAC Address Filter object through the session
my @retrieved_objs = $session->get(
    object => "Infoblox::DHCP::Filter::MAC",
    name   => "script_filter" );

my $object = $retrieved_objs[0];
unless ($object) {
    die("Get MAC Address Filter object failed: ",
        $session->status_code() . ":" . $session->status_detail());
}
print "Get MAC Address Filter object found at least 1 matching entry\n";

#Modify one of the attributes of the obtained MAC Address Filter object
$object->default_mac_address_expiration("1234");

#Apply the changes
$session->modify($object)
    or die("Modify MAC Address Filter object failed: ",
           $session->status_code() . ":" . $session->status_detail());
print "MAC Address Filter object modified successfully \n";
```

#Remove a MAC Address Filter object

```
#Get MAC Address Filter object through the session
my @retrieved_objs = $session->get(
    object => "Infoblox::DHCP::Filter::MAC",
    name   => "script_filter",
);
my $object = $retrieved_objs[0];
unless ($object) {
    die("Get MAC Address Filter object failed: ",
        $session->status_code() . ":" . $session->status_detail());
}
print "Get MAC Address Filter object found at least 1 matching entry\n";

#Submit the object for removal
$session->remove($object)
    or die("Remove MAC Address Filter object failed: ",
        $session->status_code() . ":" . $session->status_detail());
print "MAC Address Filter object removed successfully \n";

####PROGRAM ENDS####
```

AUTHOR

Infoblox Inc. http://www.infoblox.com/

SEE ALSO

Infoblox::DHCP::MAC, Infoblox::DHCP::Filter::RelayAgent, Infoblox::DHCP::Lease, Infoblox::Session,Infoblox::Session->get(),Infoblox::Session->search(),Infoblox::Session->modify(),Infoblox::Session->remove()

COPYRIGHT

Infoblox::DHCP::Filter::MAC - DHCP MAC Address Filter object.

Copyright (c) 2017 Infoblox Inc.

Infoblox::DHCP::Filter::NAC - DHCP NAC Filter object.

- NAME
- DESCRIPTION
- CONSTRUCTOR
- SESSION METHODS
  - Infoblox::Session->add( )
  - Infoblox::Session->get( )
  - Infoblox::Session->modify( )
  - Infoblox::Session->remove( )
  - Infoblox::Session->search( )
- METHODS
  - comment( )
  - expression( )
  - extattrs( )
  - extensible_attributes( )
  - lease_time( )
  - name( )
  - option_list( )
- SAMPLE CODE
- AUTHOR
- SEE ALSO
- COPYRIGHT

NAME

Infoblox::DHCP::Filter::NAC - DHCP NAC Filter object.

DESCRIPTION

If NAC authentication is configured, the appliance receives authentication responses from NAC authentication servers, and it grants or denies a lease request if the authentication response matches conditions defined by the NAC filters.

CONSTRUCTOR

```
 my $nac_filter = Infoblox::DHCP::Filter::NAC->new(
       name                          => $string,                  #Required
       expression                    => $string,                  #Optional / Default is empty
       extattrs                      => { $string => $extattr, ... },      #Optional / Default is
undefined
       extensible_attributes         => { $string => $string | $num, $string => [ $string | $num, ... ],
... }, #Optional / Default is undefined
       comment                       => $string,                  #Optional / Default is empty
       lease_time                    => $time,                    #Optional / Default is empty
       option_list                   => [$Option1, $Option2,...],  #Optional / Default is empty
 );
```

You cannot set both extattrs and extensible_attributes attributes at the same time.

SESSION METHODS

This section describes all the methods in the Infoblox::Session module that can be applied to a DHCP NAC Filter object.

Infoblox::Session->add( )

Use this method to add a DHCP NAC Filter object to the Infoblox appliance. See Infoblox::Session->add() for parameters and return values.

Example

```
       #Construct a DHCP NAC Filter object
       my $nac_filter = Infoblox::DHCP::Filter::NAC->new (
              name                          => "filter_script",
              comment                       => "Created through script",
              expression                    => 'Sophos.ComplianceState="PartialCompliant"',
       );
       #Submit for addition
       my $response = $session->add($nac_filter)
```

Infoblox::Session->get( )

Use this method to retrieve all the matching objects from the Infoblox appliance. See Infoblox::Session->get() for parameters and return values.

Key References

```
Apply the following attributes to get a specific DHCP NAC Filter object:

 name - Required. Name of the NAC Filter.
 extattrs     - Optional. A hash reference containing extensible attributes.
 extensible_attributes - Optional. A hash reference containing extensible attributes.
```

Example

```
#Get DHCP NAC Filter object through the session
my @retrieved_objs = $session->get(
    object => "Infoblox::DHCP::Filter::NAC",
    name   => "filter_script",
);

# get all objects with a given extensible attribute
my @retrieved_objs = $session->get(
    object               => "Infoblox::DHCP::Filter::NAC",
    extensible_attributes => { 'Site' => 'Santa Clara' }
);
```

Infoblox::Session->modify( )

Use this method to modify a DHCP NAC Filter object in the Infoblox appliance. See Infoblox::Session->modify() for parameters and return values.

Example

```
#Use method to modify the default_nac_address_expiration
$nac_filter->comment("modified comment");
#Submit modification
my $response = $session->modify( $nac_filter );
```

Infoblox::Session->remove( )

Use this method to remove a DHCP NAC Filter object from the Infoblox appliance. See Infoblox::Session->remove() for parameters and return values.

To remove a specific object, first use `get()` or `search()` to retrieve the specific DHCP NAC Filter object, then submit this object for removal.

Example

```
#Get DHCP NAC Filter object through the session
my @retrieved_objs = $session->get(
    object => "Infoblox::DHCP::Filter::NAC",
    name   => "filter_script",
);
#Find the desired object from the retrieved list.
my $desired_nac_filter = $retrieved_objs[0];
#Submit for removal
my $response = $session->remove( $desired_nac_filter );
```

Infoblox::Session->search( )

Use this method to search for DHCP NAC Filter objects in the Infoblox appliance. See Infoblox::Session->search() for parameters and return values.

Key References

```
Apply the following attributes to search for a specific DHCP NAC Filter object:

 name - Name of the NAC filter (regular expression).
 comment - Comment string for the NAC filter (regular expression).
 extattrs     - Optional. A hash reference containing extensible attributes.
 extensible_attributes - Optional. A hash reference that contains extensible attributes.

Either a name or comment must be specified.
```

For more information about searching extensible attributes, see Infoblox::Grid::ExtensibleAttributeDef/Searching Extensible Attributes.

Example

```
# search for all DHCP NAC Filter objects whose name starts with "filter".
my @retrieved_objs = $session->search(
    object  => "Infoblox::DHCP::Filter::NAC",
    name    => "filter.*",
);

# search for all DHCP NAC Filter objects whose comment starts with "user1".
my @retrieved_objs = $session->search(
    object  => "Infoblox::DHCP::Filter::NAC",
    comment    => "user1.*",
);

# search all DHCP NAC Filter associations with the extensible attribute 'Site'
my @retrieved_objs = $session->search(
   object => "Infoblox::DHCP::Filter::NAC",
   extensible_attributes => { 'Site' => 'Santa Clara' });
```

METHODS

This section describes all the methods that can be used to set and retrieve the attribute values of a DHCP NAC Filter object.

comment( )

Use this method to set or retrieve a descriptive comment of a DHCP NAC Filter object.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

Desired comment in string format, with a maximum of 256 bytes.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get comment
my $comment = $nac_filter->comment();
#Modify comment
$nac_filter->comment("Modified DHCP NAC Filter comment");
```

expression( )

Use this method to set or retrieve the conditional expression of a DHCP NAC Filter object.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

An expression in string format, with a maximum of 4096 characters. The expression must comply with the NAC filter expression syntax.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Getting expression
my $expression = $nac_filter->expression( );
#Modifying expression
$nac_filter->expression('Sophos.ComplianceState="PartialCompliant"');
```

extattrs( )

Use this method to set or retrieve the extensible attributes associated with a DHCP NAC Filter object.

Parameter

Valid value is a hash reference containing the names of extensible attributes and their associated values (Infoblox::Grid::Extattr objects).

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get extattrs
my $ref_extattrs = $nac_filter->extattrs();
#Modify extattrs
$nac_filter->extattrs({ 'Site' => $extattr1, 'Administrator' => $extattr2 });
```

extensible_attributes( )

Use this method to set or retrieve the extensible attributes associated with a DHCP NAC Filter object.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

For valid values for extensible attributes, see Infoblox::Grid::ExtensibleAttributeDef/Extensible Attribute Values.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get extensible attributes
my $ref_extensible_attributes = $nac_filter->extensible_attributes();
#Modify extensible attributes
$nac_filter->extensible_attributes({ 'Site' => 'Santa Clara', 'Administrator' => [ 'Peter', 'Tom' ]
});
```

lease_time( )

Use this method to set or retrieve a lease_time attribute of a DHCP NAC filter object.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

Enter appropriate values in seconds.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
# Get lease_time
my $lease_time = $nac_filter->lease_time();
# Modify lease_time
$nac_filter->lease_time("7200");
```

name( )

Use this method to set or retrieve the name of a DHCP NAC Filter object.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

The name of the DHCP NAC filter. The maximum length supported is 1024 bytes.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get name
```

```
my $name = $nac_filter->name();
#Modify name
$nac_filter->name("filter1");
```

option_list( )

Use this method to set or retrieve the option_list of a DHCP NAC filter object.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

The valid value is an array reference that contains the Infoblox::DHCP::Option objects. The option list describes filter option configuration settings and various services.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get option_list
my $options = $nac_filter->option_list();
#Modify option_list
$nac_filter->option_list([$option1]);
```

SAMPLE CODE

The following sample code demonstrates the different functions that can be applied to an object such as add, search, modify, and remove. Also, this sample includes error handling for the operations.

#Preparation prior to a DHCP NAC Filter object insertion

```
#PROGRAM STARTS: Include all the modules that will be used
use strict;
use Infoblox;

#Create a session to the Infoblox appliance

my $session = Infoblox::Session->new(
               master   => "192.168.1.2", #appliance host ip
               username => "admin",       #appliance user login
               password => "infoblox"     #appliance password
);

unless ($session) {
       die("Construct session failed: ",
               Infoblox::status_code() . ":" . Infoblox::status_detail());
}
print "Session created successfully\n";
```

#Create a DHCP NAC Filter object

```
#Create a DHCP NAC Filter object
my $nac_filter = Infoblox::DHCP::Filter::NAC->new (
       name                          => "script_filter",
       comment                       => "Create through script",
       expression                    => 'Sophos.ComplianceState="PartialCompliant"',
);

unless($nac_filter) {
     die("Construct nac filter failed: ",
           Infoblox::status_code() . ":" . Infoblox::status_detail());
}
print "NAC Filter object created successfully\n";

#Verify if the DHCP NAC Filter exists in the Infoblox appliance
my $object = $session->get(object => "Infoblox::DHCP::Filter::NAC", name => "script_filter");
unless ($object) {
   print "nac filter does not exist on server, safe to add the nac filter\n";
   $session->add($nac_filter)
      or die("Add nac filter failed: ",
             $session->status_code() . ":" . $session->status_detail());
}
print "DHCP NAC Filter added successfully\n";
```

#Search for a specific DHCP NAC Filter object

```
#Search for all DHCP NAC filter objects
my @retrieved_objs = $session->search(
    object => "Infoblox::DHCP::Filter::NAC",
    name   => ".*" );
my $object = $retrieved_objs[0];

unless ($object) {
       die("Search for NAC Filter object failed: ",
               $session->status_code() . ":" . $session->status_detail());
}
print "Search NAC Filter object found at least 1 matching entry\n";
```

#Get and modify a NAC Filter object

```
#Get a NAC Filter object through the session
my @retrieved_objs = $session->get(
    object => "Infoblox::DHCP::Filter::NAC",
    name   => "script_filter" );

my $object = $retrieved_objs[0];
unless ($object) {
    die("Get NAC Filter object failed: ",
        $session->status_code() . ":" . $session->status_detail());
}
print "Get NAC Filter object found at least 1 matching entry\n";

#Modify one of the attributes of the obtained NAC Filter object
$object->comment("Modified comment");

#Apply the changes
$session->modify($object)
    or die("Modify NAC Filter object failed: ",
           $session->status_code() . ":" . $session->status_detail());
print "NAC Filter object modified successfully \n";
```

#Remove a NAC Filter object

```
#Get a NAC Filter object through the session
my @retrieved_objs = $session->get(
    object => "Infoblox::DHCP::Filter::NAC",
    name   => "script_filter",
);
my $object = $retrieved_objs[0];
unless ($object) {
    die("Get NAC Filter object failed: ",
        $session->status_code() . ":" . $session->status_detail());
}
print "Get NAC Filter object found at least 1 matching entry\n";

#Submit the object for removal
$session->remove($object)
    or die("Remove NAC Filter object failed: ",
        $session->status_code() . ":" . $session->status_detail());
print "NAC Filter object removed successfully \n";

####PROGRAM ENDS####
```

AUTHOR

Infoblox Inc. http://www.infoblox.com/

SEE ALSO

Infoblox::DHCP::Filter::RelayAgent, Infoblox::DHCP::Lease, Infoblox::Session,Infoblox::Session->get(),Infoblox::Session->search(),Infoblox::Session->modify(),Infoblox::Session->remove()

COPYRIGHT

Copyright (c) 2017 Infoblox Inc.

Infoblox::DHCP::Filter::Option - DHCP filter option object

- NAME
- DESCRIPTION
- CONSTRUCTOR
- SESSION METHODS
  - Infoblox::Session->add( )
  - Infoblox::Session->get( )
  - Infoblox::Session->modify( )
  - Infoblox::Session->remove( )
  - Infoblox::Session->search( )
- METHODS
  - name( )
  - apply_as_class( )
  - boot_file( )
  - boot_server( )
  - comment( )
  - expression( )
  - extattrs( )
  - extensible_attributes( )
  - lease_time( )
  - next_server( )
  - option_list( )
  - option_space( )
  - pxe_lease_time( )
- SAMPLE CODE
- AUTHOR
- SEE ALSO
- COPYRIGHT

NAME

Infoblox::DHCP::Filter::Option - DHCP filter option object

DESCRIPTION

A Filter::Option defines a DHCP option filter. In the ISC DHCP terms, it defines a class of clients that match a particular (option, value) pair. To define an option filter, add Option to the DHCP Filter object.

CONSTRUCTOR

```
  my $optionFilter = Infoblox::DHCP::Filter::Option->new(
                 name                   => $string,                          #Required
                 apply_as_class         => "true" | "false",                 #Optional / Default is "true"
                 comment                => $string,                          #Optional / Default is empty
                 boot_file              => $filename | undef,                #Optional / Default is
undefined
                 boot_server            => $fqdn | undef,                    #Optional / Default is
undefined
                 extattrs               => { $string => $extattr, ... },     #Optional / Default is
undefined
                 extensible_attributes  => { $string => $string | $num, $string => [ $string | $num, ... ],
... }, #Optional / Default is undefined
                 expression             => $string,                          #Optional / Default is empty
                 lease_time             => $time,                            #Optional / Default is empty
                 next_server            => $fqdn | undef,                    #Optional / Default is
undefined
                 option_list            => [$Option1, $Option2,...],         #Optional / Default is empty
                 option_space           => $string,                          #Optional / Default is "DHCP"
                 pxe_lease_time         => $time,                            #Optional / Default is empty
                );
```

You cannot set both extattrs and extensible_attributes attributes at the same time.

SESSION METHODS

This section describes all the methods in Infoblox::Session module that can be applied to a DHCP filter option object.

Infoblox::Session->add( )

Use this method to add a DHCP filter option object to the Infoblox device. See Infoblox::Session->add() for parameters and return values.

Example

```
#Construct an object
my  $optionFilter = Infoblox::DHCP::Filter::Option->new(
       name              => "option_filter",

        );

# Submit for adding a option Filter
my $response = $session->add( $optionFilter );
```

Infoblox::Session->get( )

Use this method to retrieve all the matching DHCP option filter objects from the Infoblox device. See Infoblox::Session->get() for parameters and return values.

Key References

```
Apply the following attributes to get a specific DHCP relayagent filter object:

name - Required. An option filter name in string format.
extattrs              - Optional. A hash reference containing extensible attributes.
extensible_attributes - Optional. A hash reference containing extensible attributes.
```

Example

```
my  @retrieved_objs = $session->get(
    object => "Infoblox::DHCP::Filter::Option" ,
    name   => "option_filter",
);

# get all objects with a given extensible attribute
my @retrieved_objs = $session->get(
    object               => "Infoblox::DHCP::Filter::Option",
    extensible_attributes => { 'Site' => 'Santa Clara' }
);
```

Infoblox::Session->modify( )

Use this method to modify a DHCP option filter object in the Infoblox device. See Infoblox::Session->modify() for parameters and return values.

Example

```
#Use method to modify boot_server method.
$object->boot_server("myboot_server.com");
# Submit modification
my $response = $session->modify( $object );
```

Infoblox::Session->remove( )

Use this method to remove a DHCP option filter object from the Infoblox device. See Infoblox::Session->remove() for parameters and return values.

To remove a specifc object, first use `get()` or `search()` to retrieve the specific object, and then submit this object for removal.

Example

```
# Get the object under the same name
my @retrieved_objs = $session->get(
           object  => "Infoblox::DHCP::Filter::Option" ,
           "name"  => "option_filter",
       );

# find the desired object from retrieved list.
my $desired_obj = $retrieved_objs[0];

# Submit for removal
my $response = $session->remove( $desired_obj);
```

Infoblox::Session->search( )

Use this method to search for DHCP option filter object in the Infoblox device. See Infoblox::Session->search() for parameters and return values.

Key References

```
Apply the following attributes to search for a DHCP Network object:

 name - Required. An option filter name in string format (regular expression).
 comment - Optional . A comment in string format (regular expression).
 extattrs              - Optional. A hash reference containing extensible attributes.
 extensible_attributes - Optional. A hash reference containing extensible attributes.
```

For more information about searching extensible attributes, see Infoblox::Grid::ExtensibleAttributeDef/Searching Extensible Attributes.

Example

```
# search for all DHCP option filter objects
     my @retrieved_objs = $session->search(
    object => "Infoblox::DHCP::Filter::Option",
    name   => "option.*",
);

# search all DHCP option filter associations with the extensible attribute 'Site'
my @retrieved_objs = $session->search(
   object => "Infoblox::DHCP::Filter::Option",
   extensible_attributes => { 'Site' => 'Santa Clara' });
```

METHODS

This section describes all the methods that can be used to configure and retrieve the attribute values of DHCP option filter object

name( )

Use this method to set or retrieve the name of a DHCP option filter object.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

Specify the name in string format. The default value for this field is empty.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
# Get name
my $name = $optionFilter->name();
#Modify name
$optionFilter->name("option_filter");
```

apply_as_class( )

Use this method to set or retrieve the "apply_as_class" flag of a DHCP filter option object. If this flag is set to "true" the filter is treated as global DHCP class, e.g it is written to dhcpd config file even if it is not present in any DHCP range.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

Specify "true" to set the "apply_as_class" flag or "false" to deactivate/unset it. The default value is "true".

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get apply_as_class flag
my $apply_as_class = $optionFilter->apply_as_class();
#Modify apply_as_class flag
$optionFilter->apply_as_class("true");
```

boot_file( )

Use this method to set or retrieve a boot_file attribute of a DHCP filter option object. If this attribute is not set, it will inherit the member level setting.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

The name of the file that the client must download.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
# Get boot_file
my $boot_file = $optionFilter->boot_file();
# Modify boot_file
$optionFilter->boot_file("bootfile1");
#Un-override boot_file
$optionFilter->boot_file(undef);
```

boot_server( )

Use this method to set or retrieve a bootserver attribute of a DHCP filter option object. You can specify the name and/or IP address of the boot server that host needs to boot. If this attribute is not set, it will inherit the member level setting.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

The boot server IP address or name in FQDN (Fully Qualified Domain Name) format. The FQDN consists of the host name followed by the domain name (example: abc.com). A boot server name can have a maximum of 256 bytes. The default value is undefined.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
# Get boot_server
my $bootserver = $optionFilter->bootserver();
# Modify boot_server
$optionFilter->boot_server("abc.domain.com");
#Un-override boot_server
$optionFilter->boot_server(undef);
```

comment( )

Use this method to set or retrieve the descriptive comment of a DHCP filter option object.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

Desired comment in string format with a maximum of 256 bytes.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get comment
my $comment = $optionFilter->comment();
#Modify comment
$optionFilter->comment("Modified DHCP filter option object comment");
```

expression( )

Use this method to set or retrieve the conditional expression of a DHCP filter option object.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

An expression in string format, with a maximum of 32768 characters. The expression must comply with the option filter expression syntax.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Getting expression
my $expression = $optionFilter->expression( );
#Modifying expression
$optionFilter->expression('Sophos.ComplianceState="PartialCompliant"');
```

extattrs( )

Use this method to set or retrieve the extensible attributes associated with a DHCP Filter Option object.

Parameter

Valid value is a hash reference containing the names of extensible attributes and their associated values (Infoblox::Grid::Extattr objects).

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get extattrs
my $ref_extattrs = $optionFilter->extattrs();
#Modify extattrs
$optionFilter->extattrs({ 'Site' => $extattr1, 'Administrator' => $extattr2 });
```

extensible_attributes( )

Use this method to set or retrieve the extensible attributes associated with a DHCP Filter Option object.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

For valid values for extensible attributes, see Infoblox::Grid::ExtensibleAttributeDef/Extensible Attribute Values.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get extensible attributes
my $ref_extensible_attributes = $optionFilter->extensible_attributes();
#Modify extensible attributes
$optionFilter->extensible_attributes({ 'Site' => 'Santa Clara', 'Administrator' => [ 'Peter', 'Tom'
] });
```

lease_time( )

Use this method to set or retrieve a lease_time attribute of a DHCP filter option object.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

Enter appropriate values in seconds.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
# Get lease_time
my $lease_time = $optionFilter->lease_time();
# Modify lease_time
$optionFilter->lease_time("7200");
```

next_server( )

Use this method to set or retrieve a next_server attribute of a DHCP filter option object. You can specify the name and/or IP address of the next server that the host needs to boot. If this attribute is not set, it will inherit the member level setting.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

The next server IP address or name in FQDN (Fully Qualified Domain Name) format. The FQDN consists of the host name followed by the domain name (example: abc.com). The next server name can have a maximum of 256 bytes. The default value is undefined.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
# Get next_server
my $nextserver = $optionFilter->next_server();
# Modify next_server
$optionFilter->next_server("blue.domain.com");
#Un-override next_server
$optionFilter->next_server(undef);
```

option_list( )

Use this method to set or retrieve the option_list of a DHCP filter option object.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

The valid value is an array reference that contains Infoblox::DHCP::Option objects. Option list describe filter option configuration settings and various services.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get option_list
my $options = $optionFilter->option_list();
#Modify option_list
$optionFilter->option_list([$option1]);
```

option_space( )

Use this method to retrieve the option_space of a DHCP filter option object.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

Specify the option space value in string format. The default value of option_space is DHCP.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get option_space
my $options = $optionFilter->option_space();
#Modify option_space
```

```
$optionFilter->option_space("Infoblox_DHCP");
```

pxe_lease_time( )

Use this method to set or retrieve a pxe_lease_time attribute of a DHCP filter option object.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

Enter appropriate values in seconds.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
# Get pxe_lease_time
my $pxe_lease_time = $optionFilter->pxe_lease_time();
# Modify pxe_lease_time
$optionFilter->pxe_lease_time("3600");
```

SAMPLE CODE

The following sample code demonstrates the different functions that can be applied to an object such as add, search, modify, and remove. Also, this sample includes error handling for the operations.

#Preparation prior to a DHCP option filter object insertion

```
#PROGRAM STARTS: Include all the modules that will be used
use strict;
use Infoblox;

#Create a session to the Infoblox device

my $session = Infoblox::Session->new(
    master   => "192.168.1.2",
    username => "admin",
    password => "infoblox"
);
unless ($session) {
   die("Construct session failed: ",
       Infoblox::status_code() . ":" . Infoblox::status_detail());
}
print "Session created successfully\n";
```

#Create a DHCP Filter Option object

```
my  $optionfilter = Infoblox::DHCP::Filter::Option->new(

       name             => "option_filter",
       next_server      => "abc_server.com" ,
       boot_file        => "file1" ,
       boot_server      => "boot_server.com" ,
       lease_time       => "7200",
       pxe_lease_time   => "3600",
              );

unless ($optionfilter) {
   die("Construct DHCP Filter::Option failed: ",
       Infoblox::status_code() . ":" . Infoblox::status_detail());
}
print "DHCP Filter::Option object created successfully\n";
my $response = $session->add($optionfilter)
       or die("Add DHCP Filter Option failed: ",
             $session->status_code() . ":" . $session->status_detail());

print "DHCP Filter Option added successfully\n";
```

#Search for a specific DHCP Filter Option

```
#Search the  DHCP Filter Option
my @retrieved_objs = $session->search(
    object             => "Infoblox::DHCP::Filter::Option",
```

```
    name               => "option.*",
);
my $object = $retrieved_objs[0];
unless ($object) {
    die("Search DHCP Filter Option  failed: ",
        $session->status_code() . ":" . $session->status_detail());
}

print "Search DHCP Filter Option found at least 1 matching entry\n";
```

#Get and modify a DHCP Filter Option attribute

```
#Get  DHCP Filter Option through the session
my  @retrieved_objs = $session->get(

              object      =>"Infoblox::DHCP::Filter::Option" ,
              name        => "option_filter",
                      );

my $object = $retrieved_objs[0];
unless ($object) {
    die("Get option filter object failed: ",
        $session->status_code() . ":" . $session->status_detail());
}

print "Get DHCP Filter Option found at least 1 matching entry\n";

#Modify one of the attributes of the specified DHCP Filter Option attribute

$object->boot_server("another_Server");

#Apply the changes
$session->modify($object)
    or die("Modify DHCP Filter Option attribute failed: ",
           $session->status_code() . ":" . $session->status_detail());

print "DHCP Filter Option object modified successfully \n";
```

#Remove a DHCP Filter Option Object

```
 #Get object through the session
 my @retrieved_objs = $session->get(

           object  => "Infoblox::DHCP::Filter::Option" ,
           "name"  => "option_filter",
                   );

   my $object = $retrieved_objs[0];

   unless ($object) {
    die("Get a DHCP Filter Optionobject failed: ",
        $session->status_code() . ":" . $session->status_detail());
    }

print "Get a DHCP Filter Option object found at least 1 matching entry\n";

#Submit the object for removal
$session->remove($object)
    or die("Remove a DHCP Filter Option failed: ",
        $session->status_code() . ":" . $session->status_detail());

print "DHCP Filter Option removed successfully \n";

####PROGRAM ENDS####
```

AUTHOR

Infoblox Inc. http://www.infoblox.com/

SEE ALSO

Infoblox::Session, Infoblox::Session->get(), Infoblox::Session->search(), Infoblox::Session->add(), Infoblox::Session->remove(), Infoblox::Session->modify(), Infoblox::Grid::Extattr

---

COPYRIGHT

Copyright (c) 2017 Infoblox Inc.

Infoblox::DHCP::MSServer - Microsoft DHCP Server object.

- NAME
- DESCRIPTION
- CONSTRUCTOR
- MODULE METHODS
  - Infoblox::DHCP::FixedAddr->member( ), Infoblox::DHCP::Network->members( ), Infoblox::DHCP::NetworkTemplate->members( ), Infoblox::DHCP::RangeTemplate->member( ), Infoblox::DHCP::Range->member( )
- METHODS
  - address( )
  - name( )
- SAMPLE CODE
- AUTHOR
- SEE ALSO
- COPYRIGHT

NAME

Infoblox::DHCP::MSServer - Microsoft (r) DHCP Server object.

DESCRIPTION

This object is used to reference a Microsoft DHCP server.

CONSTRUCTOR

```
my $member = Infoblox::DHCP::MSServer->new(
    address  => $address, #Required
    name     => $name,    #Optional
);
```

MODULE METHODS

This section describes all the functions that you can apply to a DHCP MSServer object.

Infoblox::DHCP::FixedAddr->member( ), Infoblox::DHCP::Network->members( ), Infoblox::DHCP::NetworkTemplate->members( ), Infoblox::DHCP::RangeTemplate->member( ), Infoblox::DHCP::Range->member( )

Use this function to specify a member for any of the above objects. See their respective documentation pages for parameters and return values.

Example

```
my $member1 = Infoblox::DHCP::MSServer->new(
    address => "10.0.0.1",
);

#Create the Network object with this member
my $network = Infoblox::DHCP::Network->new(
    network => "10.0.0.0/255.0.0.0",
    members => [$member1],
);
```

METHODS

This section describes all the methods that you can use to set and retrieve the attribute values of a DHCP MSServer object.

address( )

Use this method to set or retrieve the address of the member.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

The Microsoft DNS server address in IPV4 format.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get address
my $address = $member->address();
#Modify address
$member->address("192.168.1.3");
```

name( )

Use this method to set or retrieve the name of the member.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

The Microsoft DNS server name in FQDN format.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get name
my $name = $member->name();
#Modify name
$member->name("msserver.com");
```

SAMPLE CODE

The following sample code demonstrates the different functions that can be applied to an object, such as add, search, modify, and remove. This sample also includes error handling for the operations.

#Preparation prior to a DHCP MSServer object insertion

```
#PROGRAM STARTS: Include all the modules that will be used
use strict;
use Infoblox;

#Create a session to the Infoblox appliance

my $session = Infoblox::Session->new(
               master   => "192.168.1.2", #appliance host ip
               username => "admin",       #appliance user login
               password => "infoblox"     #appliance password
);

unless ($session) {
       die("Construct session failed: ",
               Infoblox::status_code() . ":" . Infoblox::status_detail());
}
print "Session created successfully\n";

#Create a grid member object

my $grid_member = Infoblox::Grid::Member->new(
    name          => "hostname.com",
    ipv4addr      => "3.0.0.2",
    mask          => "255.0.0.0",
    gateway       => "3.0.0.1",
    comment       => "Script Generated"
    );
unless ($grid_member) {
       die("Construct grid member object failed: ",
               Infoblox::status_code() . ":" . Infoblox::status_detail());
}
print "Grid member object created successfully\n";

#Adding the member object to appliance through session
```

```
my $response = $session->add( $grid_member );

unless ($response) {
    die("Add grid member failed: ",
            $session->status_code() . ":" . $session->status_detail());
}
print"Grid member added successfully \n";

my $msserver = Infoblox::Grid::MSServer->new(
                                            address                  => '10.0.0.1',
                                            comment                  => 'basic member',
                                            disable                  => 'true',
                                            extensible_attributes    => { Site => 'Local'},
                                            login                    => 'user',
                                            password                 => 'password',
                                            managing_member          => 'hostname.com',
                                            read_only                => 'false',
                                            synchronization_interval => 5,
                                            logging_mode             => 'minimum',
                                           );

unless ($msserver) {
       die("Construct grid MS DNS server object failed: ",
               Infoblox::status_code() . ":" . Infoblox::status_detail());
}
print "Grid MS DNS server object created successfully\n";

#Adding the member object to appliance through session
$response = $session->add( $msserver );

unless ($response) {
    die("Add grid MS DNS server failed: ",
            $session->status_code() . ":" . $session->status_detail());
}
print"Grid MS DNS server added successfully \n";
```

#Create a DHCP MSServer object

```
my $member1 = Infoblox::DHCP::MSServer->new(
    address => "10.0.0.1",
);

unless($member1) {
     die("Construct member failed: ",
            Infoblox::status_code() . ":" . Infoblox::status_detail());
}
print "Member object created successfully\n";

#Create the Network object with this member
my $network = Infoblox::DHCP::Network->new(
                    network => "10.0.0.0/255.0.0.0",
                    comment => "add network",
                    members => [ $member1 ]
);

unless($network) {
             die("Construct Network object failed: ",
                   Infoblox::status_code() . ":" . Infoblox::status_detail());
}
print "Network object created successfully\n";

#Add the Network object into the Infoblox appliance through a session
$session->add($network)
     or die("Add Network object failed: ",
            $session->status_code() . ":" . $session->status_detail());
print "DHCP member object added to the network successfully\n";

####PROGRAM ENDS####
```

AUTHOR

Infoblox Inc. http://www.infoblox.com/

SEE ALSO

Infoblox::DHCP::Network, Infoblox::Session->add(),Infoblox::Session

---

COPYRIGHT

Copyright (c) 2017 Infoblox Inc.

Infoblox::Grid::MSServer::AdUser::Data - the Active Directory User Data object.

- NAME
- DESCRIPTION
- SESSION METHODS
- METHODS
  - active_users_count( )
- AUTHOR
- SEE ALSO
- COPYRIGHT

NAME

Infoblox::Grid::MSServer::AdUser::Data - the Active Directory User Data object.

DESCRIPTION

The object provides information about the Active Directory users.

SESSION METHODS

The object does not support any session methods. It is retrieved as a part of the following objects: Infoblox::IPAM::Address, Infoblox::IPAM::Statistics, Infoblox::DHCP::Network, Infoblox::DHCP::SharedNetwork, Infoblox::DHCP::Range, Infoblox::DHCP::FixedAddr, Infoblox::DHCP::IPv6FixedAddr, Infoblox::DHCP::HostAddr, Infoblox::DNS::Host, Infoblox::DNS::Record::A, Infoblox::DNS::Record::AAAA, Infoblox::DNS::Record::PTR, Infoblox::Grid::Discovery::Device, Infoblox::Grid::CloudAPI::VMAddress.

METHODS

this section describes all the methods that you can use to set or retrieve the attribute values of the object.

active_users_count( )

Use this method to retrieve the number of active users. This is a read-only attribute.

Parameter

None

Returns

The valid return value is an unsigned integer.

Example

```
#get active_users_count
my $active_users_count = $data->active_users_count();
```

AUTHOR

Infoblox Inc. http://www.infoblox.com/

SEE ALSO

Infoblox::IPAM::Address, Infoblox::IPAM::Statistics, Infoblox::DHCP::Network, Infoblox::DHCP::SharedNetwork, Infoblox::DHCP::Range, Infoblox::DHCP::FixedAddr, Infoblox::DHCP::IPv6FixedAddr, Infoblox::DHCP::HostAddr, Infoblox::DNS::Host, Infoblox::DNS::Record::A, Infoblox::DNS::Record::AAAA, Infoblox::DNS::Record::PTR, Infoblox::Grid::Discovery::Device, Infoblox::Grid::CloudAPI::VMAddress.

COPYRIGHT

Copyright (c) 2017 Infoblox Inc.

Infoblox::DHCP::MSOption - Microsoft Server DHCP Option object.

- NAME
- DESCRIPTION
- CONSTRUCTOR
- MODULE METHODS
  - Infoblox::Grid::MSServer::DHCP->ms_options( )
  - Infoblox::DHCP::Range->ms_options( )
  - Infoblox::DHCP::FixedAddr->ms_options( )
- METHODS
  - name( )
  - code( )
  - type( )
  - value( )
  - user_class( )
  - vendor_class( )
- AUTHOR
- SEE ALSO
- COPYRIGHT

NAME

Infoblox::DHCP::MSOption - Microsoft (r) Server DHCP Option object.

DESCRIPTION

DHCP options describe network configuration settings and various services available on the network. These options occur as variable-length fields at the end of DHCP messages.

You can add or change only DHCP options that are in the Microsoft server and NIOS predefined list of options. Options that are not common to both are read-only.

Following are the options that the Microsoft server and the NIOS predefined list of options have in common:

```
 code     name
 ============================================================
   2     'Time Offset'
   4     'Time Server'
   5     'Name Servers'
   7     'Log Servers'
   8     'Cookie Servers'
   9     'LPR Servers'
   10    'Impress Servers'
   11    'Resource Location Servers'
   12    'Host Name'
   13    'Boot File Size'
   14    'Merit Dump File'
   16    'Swap Server'
   17    'Root Path'
   18    'Extensions Path'
   19    'IP Layer Forwarding'
   20    'Nonlocal Source Routing'
   21    'Policy Filter Masks'
   22    'Max DG Reassembly Size'
   23    'Default IP Time-to-live'
   24    'Path MTU Aging Timeout'
   25    'Path MTU Plateau Table'
   26    'MTU Option'
   27    'All subnets are local'
   29    'Perform Mask Discovery'
   30    'Mask Supplier Option'
   31    'Perform Router Discovery'
   32    'Router Solicitation Address'
   33    'Static Route Option'
   34    'Trailer Encapsulation'
   35    'ARP Cache Timeout'
   36    'Ethernet Encapsulation'
   37    'TCP Default Time-to-live'
   38    'Keepalive Interval'
   39    'Keepalive Garbage'
   40    'NIS Domain Name'
   41    'NIS Servers'
   42    'NTP Servers'
   43    'Vendor Specific Info'
   44    'WINS/NBNS Servers'
   45    'NetBIOS over TCP/IP NBDD'
   46    'WINS/NBT Node Type'
   47    'NetBIOS Scope ID'
   48    'X Window System Font'
   49    'X Window System Display'
   64    'NIS+ Domain Name'
   65    'NIS+ Servers'
```

```
   66    'Boot Server Host Name'
   67    'Bootfile Name'
   68    'Mobile IP Home Agents'
   69    'Simple Mail Transport Protocol (SMTP) Servers'
   70    'Post Office Protocol (POP3) Servers'
   71    'Network News Transport Protocol (NNTP) Servers'
   72    'World Wide Web (WWW) Servers'
   73    'Finger Servers'
   74    'Internet Relay Chat (IRC) Servers'
   75    'StreetTalk Servers'
   76    'StreetTalk Directory Assistance (STDA) Servers'
```

CONSTRUCTOR

```
  my $option = Infoblox::DHCP::MSOption->new(
     code         => $int,                              #Required
     value        => $num | $string |"true"|"false",    #Required
     name         => $string,                           #Optional
     type         => $string,                           #Optional
     vendor_class => $string,                           #Optional / If not specified, it uses the default
Microsoft vendor class
     user_class   => $string,                           #Optional / If not specified, it uses the default
Microsoft user class
   );
```

MODULE METHODS

The following functions can be applied to a DHCP Option object.

Infoblox::Grid::MSServer::DHCP->ms_options( )

Use this function to specify DHCP options at the Microsoft server level. See Infoblox::Grid::MSServer::DHCP->ms_options() for parameters and return values.

Example

```
       #Construct a DHCP Option object
       my $opt1 = Infoblox::DHCP::MSOption->new(
                                              name         => 'Time Offset',
                                              code         => 2,
                                              type         => 'long',
                                              value        => 50,
                                              vendor_class => 'DHCP Standard Options',
                                              user_class   => 'Default User Class',
                                             );
       # Configure a DHCP option for a Microsoft DHCP server
       my $response = $MSServer_DHCP->ms_options([$opt1]);
```

Infoblox::DHCP::Range->ms_options( )

Use this function to specify Microsoft DHCP options at the range level. See Infoblox::DHCP::Range->ms_options() for parameters and return values.

Example

```
       my $opt1 = Infoblox::DHCP::MSOption->new(
                                              name         => 'LPR Servers',
                                              code         => 9,
                                              type         => 'array of ip address',
                                              value        => '10.4.1.0,10.4.1.1,10.4.1.2',
                                              vendor_class => 'DHCP Standard Options',
                                              user_class   => 'Default User Class',
                                             );
       # Configure a Microsoft DHCP option for a range
       my $response = $Range->ms_options([$opt1]);
```

Infoblox::DHCP::FixedAddr->ms_options( )

Use this function to specify DHCP options at the fixed address level. See Infoblox::DHCP::FixedAddr->ms_options() for parameters and return values.

Example

```
       my $opt1 = Infoblox::DHCP::MSOption->new(
                                              name         => 'Static Route Option',
                                              code         => 33,
```

```
                                                type          => 'array of ip address pair',
                                                value         => '10.5.1.0 10.5.1.1,10.5.1.2 10.5.1.3',
                                               );
    # Configure a DHCP option for a network
    my $response = $FixedAddr->ms_options([$opt1]);
```

METHODS

This section describes all the methods that can be used to configure and retrieve the attribute values of a DHCP option.

name( )

Use this method to set or retrieve the name of the DHCP option.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

Specify the name of the Microsoft DHCP option in string format.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get name
my $name = $option->name();
#Modify name
$option->name("Static Route Option");
```

code( )

Use this method to set or retrieve the code value of the DHCP option.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

Specify the code of the DHCP option. It must be an integer from 1 to 254.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get code
my $code = $option->code();
#Modify code
$option->code("12");
```

type( )

Use this method to set or retrieve the type value of the DHCP option.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

Specify the type of the DHCP option. It must be one of the following values:

```
'binary'
'byte'
'array of byte'
'array of ip address pair'
'ip address'
'array of ip address'
'long integer'
'array of long integer'
'long'
'array of long'
'array of string'
'array of word'
'encapsulated'
'string'
```

```
'word'
```

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get type
my $type = $option->type();
#Modify type
$option->type("string");
```

value( )

Use this method to set or retrieve the custom value of the DHCP option.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

Specify the data type of the DHCP option. If the type is set to 'array of...', the accepted type is a list of the specified types.

```
  "binary"                          - Binary data.

  "byte"                            - Specify a numeric value between 0 and 255.

  "encapsulated"                    - Encapsulated data.

  "ip address"                      - Specify an IPv4 address.

  "ip address pair"                 - Specify a pair of IPv4 addresses separated by a space.

  "long"                            - Specify a numeric value between 0 and 4294967295.

  "long integer"                    - Specify a numeric value between 0 and 1.84467441 * 10^19

  "string"                          - Specify either a value in ASCII text string or
                                      a list of hexadecimal characters (0-9, a-f, or A-F) without
spaces and separated by colons.

  "word"                            - Specify a numeric value between 0 and 65535.
```

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get value
my $value = $option->value();
#Modify value
$option->value("abc.com");
```

user_class( )

Use this method to set an option space name as the user class value of a DHCP option or to retrieve an option space name.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

Specify the defined option space name in string format. The default user class name is "Default User Class".

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get user_class
my $name = $option->user_class();
#Modify user_class
$option->user_class("Default User Class");
```

vendor_class( )

Use this method to set an option space name as the vendor class value of a DHCP option or to retrieve an option space name.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

Specify the defined option space name in string format. The default vendor space name is "DHCP Standard Options".

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get vendor_class
my $name = $option->vendor_class();
#Modify vendor_class
$option->vendor_class("DHCP Standard Options");
```

AUTHOR

Infoblox Inc. http://www.infoblox.com/

SEE ALSO

Infoblox::Session, Infoblox::Session->get(), Infoblox::Session->modify(),Infoblox::Grid::MSServer::DHCP, Infoblox::DHCP::Range, Infoblox::DHCP::FixedAddr

---

COPYRIGHT

Copyright (c) 2017 Infoblox Inc.

Infoblox::Grid::Discovery::DeviceInterface - the discovery interface.

- NAME
- DESCRIPTION
- SESSION METHODS
  - Infoblox::Session->modify( )
- METHODS
  - admin_status( )
  - admin_status_task_info( )
  - cap_if_admin_status_ind( )
  - cap_if_admin_status_na_reason( )
  - cap_if_description_ind( )
  - cap_if_description_na_reason( )
  - cap_if_net_deprovisioning_ipv4_ind( )
  - cap_if_net_deprovisioning_ipv6_ind( )
  - cap_if_net_deprovisioning_ipv4_na_reason( )
  - cap_if_net_deprovisioning_ipv6_na_reason( )
  - cap_if_net_provisioning_ipv4_ind( )
  - cap_if_net_provisioning_ipv6_ind( )
  - cap_if_net_provisioning_ipv4_na_reason( )
  - cap_if_net_provisioning_ipv6_na_reason( )
  - cap_if_vlan_assignment_ind( )
  - cap_if_vlan_assignment_na_reason( )
  - cap_if_voice_vlan_ind( )
  - cap_if_voice_vlan_na_reason( )
  - description( )
  - description_task_info( )
  - device( )
  - duplex( )
  - discover_now_status( )
  - extattrs( )
  - ifaddr_infos( )
  - index( )
  - last_change( )
  - link_aggregation( )
  - mac( )
  - ms_ad_user_data( )
  - name( )
  - oper_status( )
  - port_fast( )
  - reserved_object( )
  - speed( )
  - trunk_status( )
  - type( )
  - vlan_infos( )
  - vlan_info_task_info( )
  - network_view( )
  - vrf_name( )
  - vrf_description( )
  - vrf_route_distinguisher( )
- AUTHOR
- SEE ALSO
- COPYRIGHT

NAME

Infoblox::Grid::Discovery::DeviceInterface - the discovery interface.

DESCRIPTION

The object provides information about the discovery interface.

SESSION METHODS

This section describes all the methods in Infoblox::Session that you can apply to the object.

Infoblox::Session->modify( )

Use this method to modify an object in the Infoblox appliance. See Infoblox::Session->modify() for parameters and return values.

Example

```
# Use this method to modify extensible attributes
$object->extattrs({ 'Site' => $extattr1, 'Administrator' => $extattr2 });
# Submit modification
my $response = $session->modify( $object );
```

METHODS

This section describes all the methods that you can use to set or retrieve the attribute values of the object.

admin_status( )

Use this method to retrieve the administrative state of the interface. This is a read-only attribute.

Parameter

None

Returns

The method returns the attribute value that can be 'UP' or 'DOWN'.

Example

```
# Get attribute value
my $value = $object->admin_status();
```

admin_status_task_info( )

Use this method to retrieve the configured admin status task information.

Parameter

None

Returns

The method returns the attribute value that can be an Infoblox::Grid::Discovery::Port::Config::AdminStatus object.

Example

```
# Get attribute value
my $value = $object->admin_status_task_info();
```

cap_if_admin_status_ind( )

Use this method to retrieve the flag indicates whether or not modify the admin status of this interface. This is a read-only attribute.

Parameter

None

Returns

The method returns the attribute value.

Example

```
# Get attribute value
my $value = $object->cap_if_admin_status_ind();
```

cap_if_admin_status_na_reason( )

Use this method to retrieve the reason if the edit admin status action is not available. This is a read-only attribute.

Parameter

None

Returns

The method returns the attribute value.

Example

```
# Get attribute value
```

```
my $value = $object->cap_if_admin_status_na_reason();
```

cap_if_description_ind( )

Use this method to retrieve the flag indicates whether or not modify the description of this interface. This is a read-only attribute.

Parameter

None

Returns

The method returns the attribute value.

Example

```
# Get attribute value
my $value = $object->cap_if_description_ind();
```

cap_if_description_na_reason( )

Use this method to retrieve the reason if the edit description action is not available. This is a read-only attribute.

Parameter

None

Returns

The method returns the attribute value.

Example

```
# Get attribute value
my $value = $object->cap_if_description_na_reason();
```

cap_if_net_deprovisioning_ipv4_ind( )

Use this method to retrieve the flag indicates whether or not deprovision the IPv4 network associated to this interface. This is a read-only attribute.

Parameter

None

Returns

The method returns the attribute value.

Example

```
# Get attribute value
my $value = $object->cap_if_net_deprovisioning_ipv4_ind();
```

cap_if_net_deprovisioning_ipv6_ind( )

Use this method to retrieve the flag indicates whether or not deprovision the IPv6 network associated to this interface. This is a read-only attribute.

Parameter

None

Returns

The method returns the attribute value.

Example

```
# Get attribute value
my $value = $object->cap_if_net_deprovisioning_ipv6_ind();
```

cap_if_net_deprovisioning_ipv4_na_reason( )

Use this method to retrieve the reason if the deprovision the IPv4 network from this interface is not available. This is a read-only attribute.

Parameter

None

Returns

The method returns the attribute value.

Example

```
# Get attribute value
my $value = $object->cap_if_net_deprovisioning_ipv4_na_reason();
```

cap_if_net_deprovisioning_ipv6_na_reason( )

Use this method to retrieve the reason if the deprovision the IPv6 network from this interface is not available. This is a read-only attribute.

Parameter

None

Returns

The method returns the attribute value.

Example

```
# Get attribute value
my $value = $object->cap_if_net_deprovisioning_ipv6_na_reason();
```

cap_if_net_provisioning_ipv4_ind( )

Use this method to retrieve the flag indicates whether or not modify the IPv4 network associated to this interface. This is a read-only attribute.

Parameter

None

Returns

The method returns the attribute value.

Example

```
# Get attribute value
my $value = $object->cap_if_net_provisioning_ipv4_ind();
```

cap_if_net_provisioning_ipv6_ind( )

Use this method to retrieve the flag indicates whether or not modify the IPv6 network associated to this interface. This is a read-only attribute.

Parameter

None

Returns

The method returns the attribute value.

Example

```
# Get attribute value
my $value = $object->cap_if_net_provisioning_ipv6_ind();
```

cap_if_net_provisioning_ipv4_na_reason( )

Use this method to retrieve the reason if the IPv4 network provisioning is not available. This is a read-only attribute.

Parameter

None

Returns

The method returns the attribute value.

Example

```
# Get attribute value
my $value = $object->cap_if_net_provisioning_ipv4_na_reason();
```

cap_if_net_provisioning_ipv6_na_reason( )

Use this method to retrieve the reason if the IPv6 network provisioning is not available. This is a read-only attribute.

Parameter

None

Returns

The method returns the attribute value.

Example

```
# Get attribute value
my $value = $object->cap_if_net_provisioning_ipv6_na_reason();
```

cap_if_vlan_assignment_ind( )

Use this method to retrieve the flag indicates whether or not modify the VLAN assignement for this interface. This is a read-only attribute.

Parameter

None

Returns

The method returns the attribute value.

Example

```
# Get attribute value
my $value = $object->cap_if_vlan_assignment_ind();
```

cap_if_vlan_assignment_na_reason( )

Use this method to retrieve the reason if the VLAN assignment action is not available. This is a read-only attribute.

Parameter

None

Returns

The method returns the attribute value.

Example

```
# Get attribute value
my $value = $object->cap_if_vlan_assignment_na_reason();
```

cap_if_voice_vlan_ind( )

Use this method to retrieve the flag indicates whether or not modify the voice VLAN assignement for this interface. This is a read-only attribute.

Parameter

None

Returns

The method returns the attribute value.

Example

```
# Get attribute value
my $value = $object->cap_if_vlan_assignment_ind();
```

cap_if_voice_vlan_na_reason( )

Use this method to retrieve the reason if the voice VLAN assignment action is not available. This is a read-only attribute.

Parameter

None

Returns

The method returns the attribute value.

Example

```
# Get attribute value
my $value = $object->cap_if_voice_vlan_na_reason();
```

description( )

Use this method to retrieve the description of the interface. This is a read-only attribute.

Parameter

None

Returns

The method returns the attribute value.

Example

```
# Get attribute value
my $value = $object->description();
```

description_task_info( )

Use this method to retrieve the configured description task information.

Parameter

None

Returns

The method returns the attribute value that can be an Infoblox::Grid::Discovery::Port::Config::Description object.

Example

```
# Get attribute value
my $value = $object->description_task_info();
```

device( )

Use this method to retrieve the device object. This is a read-only attribute.

Parameter

None

Returns

The method returns the attribute value that can be an Infoblox::Grid::Discovery::Device object.

Example

```
# Get attribute value
my $value = $object->device();
```

duplex( )

Use this method to retrieve the duplex state of the interface. This is a read-only attribute.

Parameter

None

Returns

The method returns the attribute value that can be 'FULL', 'HALF', 'UNKNOWN' or 'UNSUPPORTED'.

Example

```
# Get attribute value
my $value = $object->duplex();
```

discover_now_status( )

Use this method to retrieve the current discovery status. This attribute is read-only. 'discover_now_status' will not be returned by default, use 'return_methods' to request it if needed.

Parameter

None.

Returns

The method returns the attribute value that can be 'COMPLETE', 'FAILED', 'NONE', 'PENDING' or 'RUNNING'.

Example

```
#Get discover_now_status
my $discover_now_status = $object->discover_now_status();
```

extattrs( )

Use this method to set or retrieve the extensible attributes associated with the DeviceInterface object.

Parameter

Valid value is a hash reference containing the names of extensible attributes and their associated values (Infoblox::Grid::Extattr objects).

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get extattrs
my $ref_extattrs = $object->extattrs();
#Modify extattrs
$object->extattrs({ 'Site' => $extattr1, 'Administrator' => $extattr2 });
```

ifaddr_infos( )

Use this method to retrieve a list of IFaddr information associated with the interface. This is a read-only attribute.

Parameter

None

Returns

The method returns the attribute value that can be an array reference that contains Infoblox::Grid::Discovery::IFAddrInfo objects.

Example

```
# Get attribute value
my $value = $object->ifaddr_infos();
```

index( )

Use this method to retrieve the interface index number, as reported by SNMP. This is a read-only attribute.

Parameter

None

Returns

The method returns the attribute value.

Example

```
# Get attribute value
my $value = $object->index();
```

last_change( )

Use this method to retrieve the timestamp of the last detected interface property change. This is a read-only attribute.

Parameter

None

Returns

The method returns the attribute value.

Example

```
# Get attribute value
my $value = $object->last_change();
```

link_aggregation( )

Use this method to retrieve the flag that indicates whether this is a link aggregation interface. This is a read-only attribute.

Parameter

None

Returns

The method returns the attribute value that can be 'true' or 'false'.

Example

```
# Get attribute value
my $value = $object->link_aggregation();
```

mac( )

Use this method to retrieve the MAC address of the interface. This is a read-only attribute.

Parameter

None

Returns

The method returns the attribute value.

Example

```
# Get attribute value
my $value = $object->mac();
```

ms_ad_user_data( )

Use this method to retrieve Microsoft Active Directory users related information. This is a read-only attribute.

Parameter

None

Returns

The valid return value is an Infoblox::Grid::MSServer::AdUser::Data object.

Example

```
#Get ms_ad_user_data
my $ms_ad_user_data = $object->ms_ad_user_data();
```

name( )

Use this method to retrieve the interface system name. This is a read-only attribute.

Parameter

None

Returns

The method returns the attribute value.

Example

```
# Get attribute value
my $value = $object->name();
```

oper_status( )

Use this method to retrieve the operating state of the interface. This is a read-only attribute.

Parameter

None

Returns

The method returns the attribute value that can be 'UP' or 'DOWN'.

Example

```
# Get attribute value
my $value = $object->oper_status();
```

port_fast( )

Use this method to retrieve the port fast status of the interface. This is a read-only attribute.

Parameter

None

Returns

The method returns the attribute value that can be 'ENABLED' or 'DISABLED'.

Example

```
# Get attribute value
my $value = $object->port_fast();
```

reserved_object( )

Use this method to retrieve the object to which this port is reserved. This is a read-only attribute.

Parameter

None

Returns

The method returns the attribute value that can be Infoblox::DHCP::HostAddr, Infoblox::DHCP::IPv6HostAddr, Infoblox::DHCP::FixedAddr, Infoblox::DHCP::IPv6FixedAddr or Infoblox::Grid::Member object.

Example

```
# Get attribute value
my $value = $object->reserved_object();
```

speed( )

Use this method to retrieve the interface speed in bps. This is a read-only attribute.

Parameter

None

Returns

The method returns the attribute value.

Example

```
# Get attribute value
my $value = $object->speed();
```

trunk_status( )

Use this method to retrieve the flag that indicates if the interface is tagged as trunk or not. This is a read-only attribute.

Parameter

None

Returns

The method returns the attribute value that can be 'ON' or 'OFF'.

Example

```
# Get attribute value
my $value = $object->trunk_status();
```

type( )

Use this method to retrieve the interface type. This is a read-only attribute.

Parameter

None

Returns

The method returns the attribute value.

Example

```
# Get attribute value
my $value = $object->type();
```

vlan_infos( )

Use this method to retrieve a list of VLAN Information associated with this interface. This is a read-only attribute.

Parameter

None

Returns

The method returns the attribute value that can be an array reference that contains Infoblox::Grid::Discovery::VLANInfo objects.

Example

```
# Get attribute value
my $value = $object->vlan_infos();
```

vlan_info_task_info( )

Use this method to retrieve the configured VLAN task information.

Parameter

None

Returns

The method returns the attribute value that can be an Infoblox::Grid::Discovery::Port::Config::VlanInfo object.

Example

```
# Get attribute value
my $value = $object->vlan_info_task_info();
```

network_view( )

Use this method to retrieve the name of the network view associated with the interface. This is a read-only attribute.

Parameter

None

Returns

The method returns the attribute value.

Example

```
# Get attribute value
my $value = $object->network_view();
```

vrf_name( )

Use this method to retrieve the name of the VRF associated with the interface, if any. This is a read-only attribute.

Parameter

None

Returns

The method returns the attribute value.

Example

```
# Get attribute value
my $value = $object->vrf_name();
```

vrf_description( )

Use this method to retrieve the VRF description. This is a read-only attribute.

Parameter

None

Returns

The method returns the attribute value.

Example

```
# Get attribute value
my $value = $object->vrf_description();
```

vrf_route_distinguisher( )

Use this method to retrieve the route distinguisher associated with the VRF. This is a read-only attribute.

Parameter

None

Returns

The method returns the attribute value.

Example

```
# Get attribute value
my $value = $object->vrf_route_distinguisher();
```

AUTHOR

Infoblox Inc. http://www.infoblox.com/

SEE ALSO

Infoblox::Grid::Discovery::Device, Infoblox::Grid::Discovery::VLANInfo, Infoblox::Grid::Discovery::IFAddrInfo

---

COPYRIGHT

Copyright (c) 2017 Infoblox Inc.

Infoblox::DHCP::FixedAddrTemplate - DHCP Fixed Address template object.

- NAME
- DESCRIPTION
- CONSTRUCTOR
- MODULE METHODS
  - Infoblox::DHCP::NetworkTemplate-> fixed_address_templates( )
- SESSION METHODS
  - Infoblox::Session->add( )
  - Infoblox::Session->get( )
  - Infoblox::Session->search( )
  - Infoblox::Session->modify( )
  - Infoblox::Session->remove( )
- METHODS
  - bootfile( )
  - bootserver( )
  - comment( )
  - deny_bootp( )
  - enable_ddns( )
  - extattrs( )
  - extensible_attributes( )
  - ddns_domainname( )
  - ddns_hostname( )
  - ignore_dhcp_option_list_request( )
  - logic_filters( )
  - name( )
  - nextserver( )
  - number_of_addresses( )
  - offset( )
  - options( )
  - override_ddns_domainname( )
  - override_logic_filters( )
  - pxe_lease_time( )
- SAMPLE CODE
- AUTHOR
- SEE ALSO
- COPYRIGHT

NAME

Infoblox::DHCP::FixedAddrTemplate - DHCP Fixed Address template object.

DESCRIPTION

A fixed address template works as a 'cookie cutter' and can be used to create a real fixed address object in a quick and consistant way. A Fixed address object created from a fixed address template will inherit most properties defined in fixed address template object so most of the fixed address template properties are the same as the fixed address object properties.

CONSTRUCTOR

```
 my $fixed_addr_template = Infoblox::DHCP::FixedAddrTemplate -> new(
       name                              => $string,                        #Required
       number_of_addresses               => $num | undef,                   #Optional / Default is undefined
       offset                            => $num | undef,                   #Optional / Default is undefined
       bootfile                          => $filename | undef,              #Optional / Default is undefined
       bootserver                        => $ipv4addr | $fqdn | undef,      #Optional / Default is undefined
       comment                           => $string,                        #Optional / Default is empty
       deny_bootp                        => "true" | "false" | undef,       #Optional / Default is undefined
       enable_ddns                       => "true" | "false",               #Optional / Default is false
       ddns_domainname                   => $string | undef,                #Optional / Default is undefined
       ddns_hostname                     => $string | undef,                #Optional / Default is undefined
       extattrs                          => { $string => $extattr, ... },      #Optional / Default is
undefined
       extensible_attributes             => { $string => $string | $num, $string => [ $string | $num, ... ],
... } #Optional / Default is undefined

       ignore_dhcp_option_list_request   => "true" | "false" | undef,       #Optional / Default is false
       nextserver                        => $ipv4addr | $fqdn | undef,      #Optional / Default is undefined
       options                           => [$option1, $option2, ...]       #Optional / Default is empty
```

```
        override_ddns_domainname          => "true" | "false",            #Optional / Default value is
"false"
        pxe_lease_time                    => $time,                       #Optional / Default is empty
        logic_filters                     => [ $filter_name | $filter_obj, ...], # Optional / Default is
undefined
        override_logic_filters            => "true' | 'false",                  # Optional / Default value
is "false"
         );
```

You cannot set both extattrs and extensible_attributes attributes at the same time.

MODULE METHODS

The following functions are available to be applied to a DHCP fixed address template object.

Infoblox::DHCP::NetworkTemplate-> fixed_address_templates( )

Use this function to specify a fixed address template at network template on the Infoblox appliance. See Infoblox::DHCP::NetworkTemplate->fixed_address_templates() for parameters and return values.

Example

```
        #Create the Network Template object with fixed address template
         my $template = Infoblox::DHCP::Template->new(
           name                    => "custom_fa_template"
         );
         my $network_template = Infoblox::DHCP::NetworkTemplate->new(
           name                    => "custom_network_template",
           comment                 => "add network",
           fixed_address_templates => [$template]
         );
```

SESSION METHODS

This section describes all the methods in Infoblox::Session module that can be applied to a DHCP Fixed Address template object.

Infoblox::Session->add( )

Use this method to add a DHCP Fixed Address template object to the Infoblox appliance. See Infoblox::Session->add() for parameters and return values.

Example

```
        #Construct a DHCP Fixed Address template object
        my $fixed_addr_template = Infoblox::DHCP::FixedAddrTemplate->new(
               "name"                  => "custom_fa_template",
               "offset"                => "10",
               "number_of_addresses"   => "1",
               "bootfile"              => "bootfile",
               "bootserver"            => "bootserver.com",
               "comment"               => "add fixed address",
               "deny_bootp"            => "false",
               "enable_ddns"           => "false",
               "nextserver"            => "3.3.3.3",
               "options"               => [ $option1,  $option2 ],
               "pxe_lease_time"        => "3600",
        );

        #Submit for addition
        my $response = $session->add($fixed_addr_template);
```

Infoblox::Session->get( )

Use this method to retrieve all the matching objects from the Infoblox appliance. See Infoblox::Session->get() for parameters and return values.

Key References

```
        Apply the following attributes to get a specific DHCP Fixed Address template object:

        name - Optional. The name of the Fixed address template.
        extattrs     - Optional. A hash reference containing extensible attributes.
        extensible_attributes - Optional. A hash reference containing extensible attributes.
```

Examples

```
#Get DHCP Fixed Address template object through the session
my @retrieved_objs = $session->get(
    object => "Infoblox::DHCP::FixedAddrTemplate",
    name   => "custom_fa_template",
);

my @retrieved_objs = $session->get(
  object               => "Infoblox::DHCP::FixedAddrTemplate",
  extensible_attributes => { 'Site' => 'Santa Clara' }
);
```

Infoblox::Session->search( )

Use this method to retrieve all the matching objects from the Infoblox appliance. See Infoblox::Session->search() for parameters and return values.

Key References

```
Apply the following attributes to search for a DHCP Fixed Address template object:

name    - Optional. The name of the fixed address template (regular expression).
comment - Optional. The comment of the fixed address template (regular expression).
extattrs     - Optional. A hash reference containing extensible attributes.
extensible_attributes - Optional. A hash reference containing extensible attributes.
```

For more information about searching extensible attributes, see Infoblox::Grid::ExtensibleAttributeDef/Searching Extensible Attributes.

Examples

```
#Search DHCP Fixed Address template object through the session
my @retrieved_objs = $session->search(
    object  => "Infoblox::DHCP::FixedAddrTemplate",
    name    => "custom_fa_template",
    comment => ".*",
);

#Search for DHCP Fixed Address template objects defining "Santa Clara" for "Site" extensible attribute
my @retrieved_objs = $session->search(
    object               => "Infoblox::DHCP::FixedAddrTemplate",
    extensible_attributes => { 'Site' => 'Santa Clara' }
);
```

Infoblox::Session->modify( )

Use this method to modify a DHCP Fixed Address template object in the Infoblox appliance. See Infoblox::Session->modify() for parameters and return values.

Example

```
#Use method to modify the pxe_lease_time
$fixed_addr_templateess->pxe_lease_time("1234");
#Submit modification
my $response = $session->modify( $fixed_addr_templateess );
```

Infoblox::Session->remove( )

Use this method to remove a DHCP Fixed Address template object from the Infoblox appliance. See Infoblox::Session->remove() for parameters and return values.

To remove a specific object, first use `get()` to retrieve the specific DHCP Fixed Address template object, and then submit this object for removal.

Example

```
#Get DHCP Fixed Address template object through the session
my @retrieved_objs = $session->get(
    object => "Infoblox::DHCP::FixedAddrTemplate",
    name   => "custom_fa_template",
);
#Find the desired object from the retrieved list.
my $desired_fixed_addr_template = $retrieved_objs[0];
#Submit for removal
my $response = $session->remove( $desired_fixed_addr_template );
```

METHODS

This section describes all the methods that can be used to set and retrieve the attribute values of a DHCP Fixed Address template object.

bootfile( )

Use this method to set or retrieve the bootfile value of a DHCP Fixed Address template object.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

This option is overridden independently from Infoblox::DHCP::FixedAddrTemplate->bootserver() and Infoblox::DHCP::FixedAddrTemplate->nextserver().

Parameter

The name of the file that the client must download. The default value is undefined.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails. If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get bootfile
my $bootfile = $fixed_addr_template->bootfile();
#Modify bootfile
$fixed_addr_template->bootfile("boot_file");
#Un-override bootfile
$fixed_addr_template->bootfile(undef);
```

bootserver( )

Use this method to set or retrieve the bootserver of a DHCP Fixed Address template object.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

This option is overridden independently from Infoblox::DHCP::FixedAddrTemplate->bootfile() and Infoblox::DHCP::FixedAddrTemplate->nextserver().

Parameter

The boot server IP address or name in FQDN (Fully Qualified Domain Name) format. The FQDN consists of the host name followed by the domain name (example: abc.com). A boot server name can have a maximum of 256 bytes. The default value is undefined.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails. If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get bootserver
my $bootserver = $fixed_addr_template->bootserver();
#Modify bootserver
$fixed_addr_template->bootserver("bootserver2.com");
#Un-override bootserver
$fixed_addr_template->bootserver(undef);
```

comment( )

Use this method to set or retrieve the descriptive comment of a DHCP Fixed Address template object.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

Desired comment in string format with a maximum of 256 bytes.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails. If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get comment
my $comment = $fixed_addr_template->comment();
#Modify comment
$fixed_addr_template->comment("Modified DHCP Fixed Address template object comment");
```

deny_bootp( )

Use this method to set or retrieve the deny_bootp flag of a DHCP Fixed Address template object.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

Specify "true" to set the deny_bootp flag or "false" to deactivate/unset it. The default value is undefined which indicates that this attribute inherits the upper-level setting.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails. If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get deny_bootp
my $deny_bootp = $fixed_addr_template->deny_bootp();
#Modify deny_bootp
$fixed_addr_template->deny_bootp("true");
```

enable_ddns( )

Use this method to set or retrieve the enable_ddns flag of a DHCP Fixed Address template object. If enable_ddns value is specified as "true", then the A and PTR records corresponding to the Fixed Address template object will be placed on the DNS server.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

Specify "true" to set the enable_ddns flag or "false" to deactivate/unset it. The default value is "false".

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails. If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get enable_ddns
my $enable_ddns = $fixed_addr_template->enable_ddns();
#Modify enable_ddns
$fixed_addr_template->enable_ddns("true");
```

extattrs( )

Use this method to set or retrieve the extensible attributes associated with a DHCP Fixed Address template object.

Parameter

Valid value is a hash reference containing the names of extensible attributes and their associated values (Infoblox::Grid::Extattr objects).

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get extattrs
my $ref_extattrs = $fixed_addr_template->extattrs();
#Modify extattrs
$fixed_addr_template->extattrs({ 'Site' => $extattr1, 'Administrator' => $extattr2 });
```

extensible_attributes( )

Use this method to set or retrieve the extensible attributes associated with a DHCP Fixed Address template object

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

For valid values for extensible attributes, see Infoblox::Grid::ExtensibleAttributeDef/Extensible Attribute Values.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get extensible attributes
my $ref_extensible_attributes = $fixed_addr_template->extensible_attributes();
#Modify extensible attributes
$fixed_addr_template->extensible_attributes({ 'Site' => 'Santa Clara', 'Administrator' => [ 'Peter',
'Tom' ] });
```

ddns_domainname( )

Use this method to set or retrieve the ddns_domainname value of a DHCP Fixed Address template object.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Setting this method to a defined value implicitly sets the override_ddns_domainname method to "true". Setting the parameter to undefined causes the appliance to use the grid default and automatically resets the override_ddns_domainname attribute to "false".

Note that when ddns_domainname is set to a defined value and override_dddns_domainname is set to "false", the last operation takes precedence. Thus the sequence $object->ddns_domainname("testdomain"); $object->override_ddns_domainname("false"); will set override_ddns_domainname to "false", and the sequence $object->override_ddns_domainname("false"); $object->ddns_domainname("testdomain"); will result in override_ddns_domainname="true".

Parameter

Desired name for the dynamic DNS domain name in string format.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get ddns_domainname
my $ddns_domainname = $fixed_addr->ddns_domainname();
#Modify ddns_domainname
$fixed_addr->ddns_domainname("test_domain.com");
```

ddns_hostname( )

Use this method to set or retrieve the ddns_hostname value of a DHCP Fixed Address template object.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

Desired name for the dynamic DNS host name in string format.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get ddns_hostname
my $ddns_hostname = $fixed_addr->ddns_hostname();
#Modify ddns_hostname
$fixed_addr->ddns_hostname("test_host");
```

ignore_dhcp_option_list_request( )

Use this method to set or retrieve the ignore_dhcp_option_list_request flag of a DHCP fixed address teplate object. If this flag is set to false all the defined DHCP options will be returned to the client. This setting overrides the upper-level settings.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

Specify "true" to set the ignore_dhcp_option_list_request flag or "false" to deactivate it. The default value is false.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
# Get ignore_dhcp_option_list_request
my $ignore_dhcp_option_list_request = $fixed_addr_template->ignore_dhcp_option_list_request();
# Modify ignore_dhcp_option_list_request
$fixed_addr_template->ignore_dhcp_option_list_request("true");
# Un-override ignore_dhcp_option_list_request
$fixed_addr_template->ignore_dhcp_option_list_request(undef);
```

logic_filters( )

Use this method to set or retrieve all the logic (ordered) filters associated with the DHCP Fixed Address Template.

Filters are used to screen address assignments for the DHCP Fixed Address Template if the addresses are to be assigned to specific hosts.

Setting this method to a defined value implicitly sets the override_logic_filters method to "true". Setting the parameter to undefined causes the appliance to use the grid default and automatically resets the override_logic_filters attribute to "false".

Note that when logic_filters is set to a defined value and override_dlogic_filters is set to "false", the last operation takes precedence. Thus the sequence $object->logic_filters($filters); $object->override_logic_filters("false"); will set override_logic_filters to "false", and the sequence $object->override_logic_filters("false"); $object->logic_filters($filters); will result in override_logic_filters="true".

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

The valid value is an array reference that contains either the Infoblox::DHCP::Filter::MAC, Infoblox::DHCP::Filter::NAC, Infoblox::DHCP::Filter::Option objects or the names of the corresponding filters. The filters in the list must satisfy the following conditions:

Filter name must be globally unique.
Filters without expressions must be at the end of the list.
The filters should not be disabled.

The default value of the parameter is undefined.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get logic_filters
my $ref_logic_filters = $fixed_addr_template->logic_filters();
#Modify logic_filters
$fixed_addr_template->logic_filters(["filter_1","filter_2",$exl1]); #$exl1 is an
Infoblox::DHCP::Filter::MAC, Infoblox::DHCP::Filter::NAC or Infoblox::DHCP::Filter::Option object.
```

name( )

Use this method to set or retrieve the name of a DHCP Fixed Address template object.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

Name of the fixed address template in string format with a maximum of 64 characters.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.
If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get name
my $name = $fixed_address_template->name();
#Modify name
$fixed_address_template->name("custom_fa_template");
```

nextserver( )

Use this method to set or retrieve the nextserver value of a DHCP Fixed Address template object.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

This option is overridden independently from Infoblox::DHCP::FixedAddrTemplate->bootfile() and Infoblox::DHCP::FixedAddrTemplate->bootserver().

Parameter

The next server IP address or name in FQDN (Fully Qualified Domain Name) format. The FQDN consists of the host name followed by the domain name (example: abc.com). A next server name can have a maximum of 256 bytes. The default value is undefined.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails. If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get nextserver
my $nextserver = $fixed_addr_template->nextserver();
#Modify nextserver
$fixed_addr_template->nextserver("3.3.3.4");
#Un-override nextserver
$fixed_addr_template->nextserver(undef);
```

number_of_addresses( )

Use this method to set or retrieve the number of addresses in a DHCP Fixed Address template object.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

The number of addresses in this fixed address template.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails. If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get number_of_addresses
my $number_of_addresses = $fixed_address_template-> number_of_addresses();
#Modify number_of_addresses
$fixed_address_template->number_of_addresses("10");
```

offset( )

Use this method to set or retrieve the address offset of a Fixed Address template object.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

The address offset of this fixed address template. The offset is calculated from the start address of network template to which the fixed address template is assigned to. For example, if offset is set to 10 and number_of_addresses is set to 10, and network is set to 10.0.0.0/8, then there will be 10 fixed address object created and start address will be 10.0.0.10 and end address will be 10.0.0.20.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails. If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get offset
my $offset = $fixed_address_template->offset();
#Modify offset
$fixed_address_template->offset("10");
```

options( )

Use this method to set or retrieve the options of a DHCP Fixed Address template object.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

See Infoblox::DHCP::Option for parameters and methods available for this object.

Parameter

Options describe network configuration settings and various services available on the network. These options occur as variable-length fields at the end of DHCP messages.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails. If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get options
my $options = $fixed_addr_template->options();
#Modify options
$fixed_addr_template->options([$option1, $option2]);
```

override_ddns_domainname( )

The override_ddns_domainname attribute controls whether the ddns_domainname value in the DHCP Fixed Address is used, instead of the grid default.

The override_ddns_domainname attribute can be specified explicitly. It is also set implicitly when ddns_domainname is set to a defined value.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

Set the parameter to "true" to override the grid-level setting for ddns_domainname. Set the parameter to "false" to inherit the grid-level setting for ddns_domainname.

The default value of this parameter is "false".

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Getting override_ddns_domainname
my $override_ddns_domainname=$fixed_addr_template->override_ddns_domainname( );
#Modifying override_ddns_domainname
$fixed_addr_template->override_ddns_domainname("true");
```

override_logic_filters( )

The override_logic_filters attribute controls whether the logic_filters value of the DHCP Fixed Address Template is used, instead of the Grid default.

The override_logic_filters attribute can be specified explicitly. It is also set implicitly when logic_filters is set to a defined value.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

Set the parameter to "true" to override the Grid-level setting for logic_filters. Set the parameter to "false" to inherit the Grid-level setting for logic_filters.

The default value of this parameter is "false".

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Getting override_logic_filters
my $override_logic_filters=$fixed_addr_template->override_logic_filters( );
#Modifying override_logic_filters
$fixed_addr_template->override_logic_filters("true");
```

pxe_lease_time( )

Use this method to set or retrieve the pxe_lease_time value of a DHCP Fixed Address template object.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

Enter appropriate values in the Days, Hours, Mins and Secs fields. User can specify the duration of time it takes a host to connect to a boot server, such as a TFTP server, and download the file it needs to boot.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails. If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get pxe_lease_time
my $pxe_lease_time = $fixed_addr_template->pxe_lease_time();
#Modify pxe_lease_time
$fixed_addr_template->pxe_lease_time("369");
```

SAMPLE CODE

The following sample code demonstrates the different functions that can be applied to an object such as add, get, modify, and remove. Also, this sample includes error handling for the operations.

```
#PROGRAM STARTS: Include all the modules that will be used
use strict;
use Infoblox;

#Create a session to the Infoblox appliance

my $session = Infoblox::Session->new(
               master   => "192.168.1.2", #appliance host ip
               username => "admin",       #appliance user login
               password => "infoblox"     #appliance password
);

unless ($session) {
       die("Construct session failed: ",
               Infoblox::status_code() . ":" . Infoblox::status_detail());
}
print "Session created successfully\n";

my $option1 = Infoblox::DHCP::Option->new(
                                          "name"  => "fqdn",
                                          "value" => "mydomain.com"
                                         );
unless($option1) {
     die("Construct option1 object failed: ",
           Infoblox::status_code() . ":" . Infoblox::status_detail());
}
print "option1 object created successfully\n";

my $option2 = Infoblox::DHCP::Option->new(
                                          "name"     => "merit-dump",
                                           "value"   => "sometextdump",
                                         );
unless($option2) {
     die("Construct option2 object failed: ",
           Infoblox::status_code() . ":" . Infoblox::status_detail());
}
print "option2 object created successfully\n";
```

#Create a DHCP Fixed Address template object

```
#Create a DHCP Fixed Address template object
my $fixed_addr_template = Infoblox::DHCP::FixedAddrTemplate->new(
    "name"                  => "custom_fa_template",
    "offset"                => "10",
    "number_of_addresses"   => "1",
    "comment"               => "add fixed address template",
    "enable_ddns"           => "TRUE",
    "ddns_domainname"       => "test_domain.com",
```

```
    "ddns_hostname"          => "test_host",
    "pxe_lease_time"         => "3600",
    "bootserver"             => "someserver.com",
    "nextserver"             => "3.3.3.3",
    "bootfile"               => "bootfile.com",
    "options"                => [ $option1, $option2 ],
);
unless($fixed_addr_template) {
     die("Construct Fixed Address template failed: ",
           Infoblox::status_code() . ":" . Infoblox::status_detail());
}
print "DHCP Fixed Address template object created successfully\n";

#Verify if the DHCP Fixed Address templateexists in the Infoblox appliance
my $object = $session->get(
       object => "Infoblox::DHCP::FixedAddrTemplate",
       name   => "custom_fa_template"
);

unless ($object) {
   print "Fixed Address template does not exist on server, safe to add the fixed address template\n";
   $session->add($fixed_addr_template)
      or die("Add Fixed Address template failed: ",
             $session->status_code() . ":" . $session->status_detail());
}
print "DHCP Fixed Address template added successfully\n";
```

#Search a DHCP Fixed Address template object

```
#Search DHCP Fixed Address template object through the session
my @retrieved_objs = $session->search(
    object => "Infoblox::DHCP::FixedAddrTemplate",
    name   => "custom_fa_template",
);
my $object = $retrieved_objs[0];

unless ($object) {
    die("Search Fixed Address template object failed: ",
        $session->status_code() . ":" . $session->status_detail());
}
print "Search Fixed Address template object found at least 1 matching entry\n";
```

#Get and modify a DHCP Fixed Address template object

```
#Get DHCP Fixed Address template object through the session
my @retrieved_objs = $session->get(
    object => "Infoblox::DHCP::FixedAddrTemplate",
    name   => "custom_fa_template",
);
my $object = $retrieved_objs[0];

unless ($object) {
    die("Get Fixed Address template object failed: ",
        $session->status_code() . ":" . $session->status_detail());
}
print "Get Fixed Address template object found at least 1 matching entry\n";

#Modify one of the attributes of the obtained Fixed Address template object
$object->pxe_lease_time("1234");

#Un-override bootfile
$object->bootfile(undef);

#Un-override bootserver
$object->bootserver(undef);

#Un-override nextserver
$object->nextserver(undef);

#Apply the changes
$session->modify($object)
  or die("Modify Fixed Address template failed: ",
             $session->status_code() . ":" . $session->status_detail());

print "DHCP Fixed Address template added successfully\n";
```

#Create the Network Template object with fixed address template

```
my $template = Infoblox::DHCP::Template->new(
    name => "custom_fa_template",
);

my $network_template = Infoblox::DHCP::NetworkTemplate->new(
    name    => "custom_network_template",
    comment => "add network",
    fixed_address_templates => [$template]
);

unless($network_template) {
       die("Construct Network template object failed: ",
            Infoblox::status_code() . ":" . Infoblox::status_detail());
}

print "Network template object created successfully\n";

#Add the Network template object into the Infoblox appliance through a session
my $resp = $session->add($network_template)
or die("Add Network object failed: ",
            $session->status_code() . ":" . $session->status_detail());

print "Network template object added to Infoblox appliance successfully\n";
```

#Remove a Fixed Address template object

```
#Remove a Network Template object
$session->remove($network_template)
    or die("Remove Network Template template object failed: ",
        $session->status_code() . ":" . $session->status_detail());
print "Network Template object removed successfully \n";

#Get Fixed Address template object through the session
my @retrieved_objs = $session->get(
    object => "Infoblox::DHCP::FixedAddrTemplate",
    name   => "custom_fa_template",
);
my $object = $retrieved_objs[0];
unless ($object) {
    die("Get Fixed Address template object failed: ",
        $session->status_code() . ":" . $session->status_detail());
}
print "Get Fixed Address template object found at least 1 matching entry\n";

#Submit the object for removal
$session->remove($object)
    or die("Remove Fixed Address template object failed: ",
        $session->status_code() . ":" . $session->status_detail());
print "Fixed Address template object removed successfully \n";

####PROGRAM ENDS####
```

AUTHOR

Infoblox Inc. http://www.infoblox.com/

SEE ALSO

Infoblox::Session, Infoblox::Session->get(), Infoblox::Session->add(), Infoblox::Session->remove(), Infoblox::Session->modify(), Infoblox::DHCP::NetworkTemplate, Infoblox::DHCP::RangeTemplate,Infoblox::DHCP::Template,Infoblox::Session, Infoblox::DHCP::Option

COPYRIGHT

Copyright (c) 2017 Infoblox Inc.

Infoblox::DHCP::Template - DHCP Template object.

- NAME
- DESCRIPTION
- CONSTRUCTOR
- MODULE METHODS
  - Infoblox::DHCP::NetworkTemplate->range_templates( )
  - Infoblox::DHCP::NetworkTemplate->fixed_address_templates( )
- METHODS
  - name( )
  - count( )
  - offset( )
- SAMPLE CODE
- AUTHOR
- SEE ALSO
- COPYRIGHT

NAME

Infoblox::DHCP::Template - DHCP Template object.

DESCRIPTION

The DHCP associates Infoblox::DHCP::RangeTemplate->offset(), Infoblox::DHCP::RangeTemplate->number_of_addresses() and Infoblox::DHCP::FixedAddrTemplate->offset(), Infoblox::DHCP::FixedAddrTemplate->number_of_addresses() with Infoblox::DHCP::NetworkTemplate. When optional fields offset/count are set in this object which is passed to Infoblox::DHCP::NetworkTemplate->fixed_address_templates() or Infoblox::DHCP::NetworkTemplate->range_templates() they are used instead of offset/number_of_addresses values in Infoblox::DHCP::RangeTemplate and Infoblox::DHCP::FixedAddrTemplate when creating Infoblox::DHCP::Network based on a template.

CONSTRUCTOR

```
my $template = Infoblox::DHCP::Template->new(
      name                  => $string,       #Required
      count                 => $num,          #Optional
      offset                => $num,          #Optional
);
```

MODULE METHODS

The following functions are available to be applied to a DHCP template object.

Infoblox::DHCP::NetworkTemplate->range_templates( )

Use this function to specify range templates for the DHCP network template object. See Infoblox::DHCP::NetworkTemplate->range_templates() for parameters and return values.

Example

```
        #Construct and add a DHCP Range Template object
        my $range_template = Infoblox::DHCP::RangeTemplate->new(
                name                  => "custom_range_template",
                offset                => "20", # this won't be used if specified in
    Infoblox::DHCP::Template
                number_of_addresses   => "10", # this won't be used if specified in
    Infoblox::DHCP::Template
        );
        $session->add($range_template);

        #Construct a DHCP Template object
        my $template = Infoblox::DHCP::Template->new(
                offset                => "10", # this will be used instead if specified
                count                 => "5", # this will be used instead if specified
                name                  => "custom_range_template"
        );

        # Configure DHCP Template on the DHCP Network template object
        $network_template->range_templates([$template]);
```

Infoblox::DHCP::NetworkTemplate->fixed_address_templates( )

Use this function to specify fixed address templates for the DHCP network template object. See Infoblox::DHCP::NetworkTemplate->fixed_address_templates() for parameters and return values.

Example

```
#Construct and add a DHCP Fixed Address Template object
my $fixed_address_template = Infoblox::DHCP::FixedAddrTemplate->new(
        name                => "custom_fixed_address_template",
        offset              => "20", # this won't be used if specified in
Infoblox::DHCP::Template
        number_of_addresses  => "10", # this won't be used if specified in
Infoblox::DHCP::Template
);
$session->add($fixed_address_template);

#Construct a DHCP Template object
my $template = Infoblox::DHCP::Template->new(
        offset              => "10", # this will be used instead if specified
        count               => "5", # this will be used instead if specified
        name                => "custom_fixed_address_template"
);

# Configure DHCP Template on the DHCP Network template object
$network_template->fixed_address_templates([$template]);
```

METHODS

This section describes all the methods that can be used to configure and retrieve the attribute values of a DHCP range template object.

name( )

Use this method to set or retrieve name of Infoblox::DHCP::RangeTemplate or Infoblox::DHCP::FixedAddrTemplate object assigned to this template object.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

The name of range template or fixed address template object.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get name
my $name = $template->name();
#Modify name
$template->name("custom_range_or_fixed_address_template_name");
```

count( )

Use this method to set or retrieve the number of addresses of the DHCP range or fixed address template. This value will be used instead of Infoblox::DHCP::RangeTemplate->number_of_addresses() or Infoblox::DHCP::FixedAddrTemplate->number_of_addresses() if specified.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

The number of addresses in this template.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get count
my $count = $template->count();
#Modify count
$template->count("10");
```

offset( )

Use this method to set or retrieve the address offset of the DHCP range or fixed address template. This value will be used instead of Infoblox::DHCP::RangeTemplate->offset() or Infoblox::DHCP::FixedAddrTemplate->offset() if specified.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

The address offset of this template.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get offset
my $offset = $template->offset();
#Modify offset
$template->offset("10");
```

SAMPLE CODE

The following sample code demonstrates the different functions that can be applied to a DHCP template object such as add, modify, and remove. Also, this sample includes error handling for the operations.

#Preparation prior to a DHCP Range object insertion

```
#PROGRAM STARTS: Include all the modules that will be used
use strict;
use Infoblox;
#Create a session to the Infoblox appliance
my $session = Infoblox::Session->new(
   master   => "192.168.1.2",
   username => "admin",
   password => "infoblox"
);

unless ($session) {
       die("Construct session failed: ",
            Infoblox::status_code() . ":" . Infoblox::status_detail());
}
print "Session created successfully\n";
```

#Create the DHCP Range template object

```
my $dhcp_range_template = Infoblox::DHCP::RangeTemplate->new(
        name                    => "custom_range_template",
        offset                  => "10",
        number_of_addresses     => "10",
);
unless($dhcp_range_template) {
       die("Construct DHCP Range template object failed: ",
            Infoblox::status_code() . ":" . Infoblox::status_detail());
}
print "DHCP Range template object created successfully\n";

#Add the DHCP Range template object into the Infoblox appliance through a session
$session->add($dhcp_range_template)
    or die("Add DHCP Range template object failed: ",
            $session->status_code() . ":" . $session->status_detail());
print "DHCP Range template object added to Infoblox appliance successfully\n";
```

#Create the Fixed Address template object

```
my $dhcp_fixed_address_template = Infoblox::DHCP::FixedAddrTemplate->new(
        name                    => "custom_fixed_address_template",
        offset                  => "10",
        number_of_addresses     => "10",
);
unless($dhcp_fixed_address_template) {
       die("Construct DHCP Fixed Address template object failed: ",
            Infoblox::status_code() . ":" . Infoblox::status_detail());
```

```
}
print "DHCP Fixed Address template object created successfully\n";

#Add the DHCP Fixed Address Template  object into the Infoblox appliance through a session
$session->add($dhcp_fixed_address_template)
    or die("Add DHCP Fixed Address template object failed: ",
           $session->status_code() . ":" . $session->status_detail());
print "DHCP Fixed Address template object added to Infoblox appliance successfully\n";
```

#Create DHCP template objects

```
#Construct a DHCP Template object
my $tmpl1 = Infoblox::DHCP::Template->new(
         offset               => "10", # this will be used instead if specified
         count                => "5", # this will be used instead if specified
         name                 => "custom_range_template"
);

unless($tmpl1) {
       die("Construct DHCP template object failed: ",
            Infoblox::status_code() . ":" . Infoblox::status_detail());
}
print "DHCP template object created successfully\n";

#Construct a DHCP Template objects
my $tmpl2 = Infoblox::DHCP::Template->new(
         offset               => "20", # this will be used instead if specified
         count                => "3", # this will be used instead if specified
         name                 => "custom_fixed_address_template"
);

unless($tmpl2) {
       die("Construct DHCP template object failed: ",
            Infoblox::status_code() . ":" . Infoblox::status_detail());
}
print "DHCP template object created successfully\n";
```

#Add DHCP template to DHCP Network Template and insert it

```
my $network_template = Infoblox::DHCP::NetworkTemplate->new(
        name                   => "network_template",
        netmask                => "255.0.0.0",
        range_templates        => [ $tmpl1 ],
        fixed_address_templates => [ $tmpl2 ],
);
unless($network_template) {
       die("Construct DHCP Network template object failed: ",
            Infoblox::status_code() . ":" . Infoblox::status_detail());
}
print "DHCP Network template object created successfully\n";

#Add the DHCP Network template object into the Infoblox appliance through a session
$session->add($network_template)
    or die("Add Network template object failed: ",
           $session->status_code() . ":" . $session->status_detail());
print "DHCP Network template object added to Infoblox appliance successfully\n";

####PROGRAM ENDS####
```

AUTHOR

Infoblox Inc. http://www.infoblox.com/

SEE ALSO

Infoblox::Session, Infoblox::DHCP::RangeTemplate, Infoblox::DHCP::FixedAddrTemplate, Infoblox::DHCP::NetworkTemplate, Infoblox::DHCP::Network

COPYRIGHT

Copyright (c) 2017 Infoblox Inc.

Infoblox::Grid::DHCP - Grid DHCP services object.

- NAME
- DESCRIPTION
- CONSTRUCTOR
- SESSION METHODS
  - Infoblox::Session->get( )
  - Infoblox::Session->modify( )
- METHODS
  - authority( )
  - bootfile( )
  - bootserver( )
  - capture_hostname( )
  - ddns_generate_hostname( )
  - ddns_retry_interval( )
  - ddns_server_always_updates( )
  - ddns_ttl( )
  - ddns_domainname( )
  - ddns_update_fixed_addresses( )
  - ddns_use_option81( )
  - deny_bootp( )
  - disable_all_nac_filters( )
  - dns_update_style( )
  - format_log_option_82( )
  - email_list( )
  - enable_ddns( )
  - enable_dhcp_thresholds( )
  - enable_email_warnings( )
  - enable_fingerprint( )
  - enable_gss_tsig( )
  - enable_ifmap( )
  - enable_ifmap_publishing( )
  - enable_hostname_rewrite( )
  - enable_roaming_hosts( )
  - enable_snmp_warnings( )
  - grid( )
  - gss_tsig_keys( )
  - high_water_mark( )
  - high_water_mark_reset( )
  - hostname_rewrite_policy( )
  - ifmap_delete_option( )
  - ifmap_ip_mac_lifetime( )
  - ifmap_port( )
  - ifmap_protocol( )
  - ifmap_url( )
  - ifmap_server_ca_cert( )
  - ignore_id( )
  - ignore_mac_addresses( )
  - ignore_dhcp_option_list_request( )
  - immediate_fa_configuration( )
  - ipv6_capture_hostname( )
  - ipv6_ddns_enable_option_fqdn( )
  - ipv6_ddns_server_always_updates( )
  - ipv6_ddns_domainname( )
  - ipv6_ddns_ttl( )
  - ipv6_default_prefix( )
  - ipv6_dns_update_style( )
  - ipv6_domain_name( )
  - ipv6_domain_name_servers( )
  - ipv6_enable_ddns( )
  - ipv6_enable_gss_tsig( )
  - ipv6_enable_lease_scavenge( )
  - ipv6_remember_expired_client_association( )
  - ipv6_enable_retry_updates( )
  - ipv6_generate_hostname( )
  - ipv6_gss_tsig_keys( )
  - ipv6_kdc_server( )
  - ipv6_lease_scavenge_time( )
  - ipv6_microsoft_code_page( )
  - ipv6_options( )
  - ipv6_prefixes( )
  - ipv6_recycle_leases( )
  - ipv6_retry_updates_interval( )
  - ipv6_txt_record_handling( )
  - ipv6_update_dns_on_lease_renewal( )

- kdc_server( )
- lease_logging_member( )
- lease_per_client_settings( )
- lease_scavenge_time( )
- log_lease_events( )
- logic_filters( )
- low_water_mark( )
- low_water_mark_reset( )
- microsoft_code_page( )
- nextserver( )
- option60_match_rules( )
- options( )
- override_immediate_fa_configuration( )
- ping_count( )
- ping_timeout( )
- preferred_lifetime( )
- protocol_hostname_rewrite_policies( )
- pxe_lease_time( )
- recycle_leases( )
- restart_setting( )
- retry_ddns_updates( )
- syslog_facility( )
- txt_record_handling( )
- update_dns_on_lease_renewal( )
- valid_lifetime( )
- validate_ifmap_ca_cert( )

- SAMPLE CODE
- AUTHOR
- SEE ALSO
- COPYRIGHT

NAME

Infoblox::Grid::DHCP - Grid DHCP services object.

DESCRIPTION

When you configure an Infoblox appliance as a full-featured DHCP server, you can set operational parameters at the grid, member, network, DHCP range and individual IP address levels. This class supports grid-level DHCP services management and network configuration. It implements methods for configuring grid-level DHCP parameters, such as DHCP general properties, ping and DHCP lease settings, Bootstrap Protocol (BOOTP) properties, custom DHCP options, and DHCP logging.

CONSTRUCTOR

The Infoblox::Grid::DHCP object is automatically generated upon the successful installation of the Infoblox appliance. It does not require manual construction.

SESSION METHODS

This section describes all the methods in the Infoblox::Session module that you can apply to a grid DHCP object.

Infoblox::Session->get( )

Use this method to retrieve matching objects from the Infoblox appliance. See Infoblox::Session->get() for parameters and return values.

Key Reference

```
Apply the following attributes to get a specific grid DHCP object:

 grid - Required. A grid name in string format.
```

Example

```
my @retrieved_objs = $session->get(
    object => "Infoblox::Grid::DHCP",
    grid   => "Infoblox"
    );
```

Infoblox::Session->modify( )

Use this method to modify objects in the Infoblox appliance. See Infoblox::Session->modify() for parameters and return values.

Example

```
#Use method to modify authority.
$griddhcp->authority("true");
#Submit modification
my $response = $session->modify( $griddhcp );
```

METHODS

This section describes all the methods that you can use to configure and retrieve the attribute values of the DHCP properties of a grid.

authority( )

Use this method to set or retrieve the grid-level authority flag. This flag specifies whether a DHCP server is authoritative for a domain.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

Specify "true" to set the authority flag or "false" to deactivate/unset it. The default value is "false".

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get authority
my $authority = $griddhcp->authority();
#Modify authority
$griddhcp->authority("true");
```

bootfile( )

Some DHCP clients use BOOTP (bootstrap protocol) or include the boot file name option in their DHCPREQUEST messages. Use this method to set or retrieve a boot file name.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

The name of the file that the client must download.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get bootfile
my $bootfile = $griddhcp->bootfile();
#Modify bootfile
$griddhcp->bootfile("bootfile.txt");
```

bootserver( )

Use this method to set or retrieve the name of the server on which a boot file is stored.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

The boot server IP address or name in FQDN (Fully Qualified Domain Name) format. The FQDN consists of the host name followed by the domain name (example: abc.com). A boot server name can have a maximum of 256 bytes.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get bootserver
my $bootserver = $griddhcp->bootserver();
#Modify bootserver
$griddhcp->bootserver("hostname.com");
```

capture_hostname( )

Use this method to set or retrieve the grid-level capture_hostname flag. Set this flag to capture the hostname and lease time when assigning a fixed address.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

Specify "true" to enable this feature or "false" to disable it. The default value is "false".

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get capture_hostname
my $capture_hostname = $griddhcp->capture_hostname();
#Modify disable
$griddhcp->capture_hostname("true");
```

ddns_generate_hostname( )

Use this method to set or retrieve the grid-level ddns_generate_hostname flag. Set this flag to enable a DHCP server to generate a host name and update DNS with this host name when it receives a DHCP REQUEST message that does not include a host name.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

Specify "true" to enable this feature or "false" to disable it. The default value is "false".

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get ddns_generate_hostname
my $ddns_generate_hostname = $griddhcp->ddns_generate_hostname();
#Modify disable
$griddhcp->ddns_generate_hostname("true");
```

ddns_retry_interval( )

Use this method to set or retrieve the grid-level ddns_retry_interval value. Set the retry interval when the DHCP server makes repeated attempts to send DDNS updates to a DNS server.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

A 32-bit integer (range from 0 to 4294967295) that represents the retry interval in minutes. The default retry interval is five minutes.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get ddns_retry_interval
my $ddns_retry_interval = $griddhcp->ddns_retry_interval();
#Modify ddns_retry_interval
$griddhcp->ddns_retry_interval(10);
```

ddns_server_always_updates( )

Use this method to set or retrieve the grid-level ddns_server_always_updates flag. Set this flag to allow only the DHCP server to update DNS, regardless of requests from DHCP clients.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

Specify "true" to set the ddns_server_always_updates flag or "false" to deactivate/unset it. The default value is "false".

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get ddns_server_always_updates
my $ddns_server_always_updates = $griddhcp->ddns_server_always_updates();
#Modify ddns_server_always_updates
$griddhcp->ddns_server_always_updates("true");
```

ddns_ttl( )

Use this method to set or retrieve the grid-level ddns_ttl value. The DDNS TTL (Dynamic DNS Time To Live) value specifies the number of seconds a DNS record is cached.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

A 32-bit integer (range from 0 to 4294967295) that specifies the number of seconds a DNS record is cached. Default value is half of lease time.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get ddns_ttl
my $ddns_ttl = $griddhcp->ddns_ttl();
#Modify ddns_ttl
$griddhcp->ddns_ttl(1000);
```

ddns_domainname( )

Use this method to set or retrieve the grid-level DDNS domain name value.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

The DDNS domain name in FQDN (Fully Qualified Domain Name) format.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get ddns_domainname
my $ddns_domainname = $griddhcp->ddns_domainname();
#Modify ddns_domainname
$griddhcp->ddns_domainname("domain.com");
```

ddns_update_fixed_addresses( )

Use this method to set or retrieve the grid-level ddns_update_fixed_addresses flag. Set this flag to enable the DHCP server to update the A and PTR records of clients with fixed addresses.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

Specify "true" to enable the DHCP server to update the A and PTR records of clients with fixed addresses, or specify "false" to disable this feature. The default value is "false".

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get ddns_update_fixed_addresses
my $ddns_update_fixed_addresses = $griddhcp->ddns_update_fixed_addresses();
#Modify ddns_update_fixed_addresses
$griddhcp->ddns_update_fixed_addresses("true");
```

ddns_use_option81( )

Use this method to set or retrieve the grid-level ddns_use_option81 flag. Set this flag to enable support for option 81.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

Specify "true" to enable support for option 81 or "false" to disable it. The default value is "false".

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get ddns_use_option81
my $ddns_use_option81 = $griddhcp->ddns_use_option81();
#Modify ddns_use_option81
$griddhcp->ddns_use_option81("true");
```

deny_bootp( )

Use this method to set or retrieve the grid-level deny_bootp flag. Set this flag to enable a BOOTP server to deny BOOTP request.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

Specify "true" to enable this feature or "false" to disable it. The default value is "false".

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get deny_bootp
my $deny_bootp = $griddhcp->deny_bootp();
#Modify deny_bootp
$griddhcp->deny_bootp("true");
```

disable_all_nac_filters( )

Use this method to enable/disable the use of all NAC filters.

Parameter

Specify "true" to disable the use of all NAC filters or "false" to enable it. The default value is "false".

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get disable_all_nac_filters
my $disable_all_nac_filters = $griddhcp->disable_all_nac_filters();
#Modify disable_all_nac_filters
$griddhcp->disable_all_nac_filters("true");
```

dns_update_style( )

Use this method to set or retrieve the update style for dynamic DNS updates (DHCPv4).

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

Valid values are 'INTERIM' and 'STANDARD'.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get dns_update_style value
my $value = $object->dns_update_style();

#Modify dns_update_style value
$object->dns_update_style('STANDARD');
```

format_log_option_82( )

Use this method to set or retrieve the format option for Option 82 logging. The possible values are 'HEX' or 'TEXT'.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

Switch between HEX and human readable format of agent.remote-id/circuit-id in syslog for Option 82. Default is 'HEX'.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get format of option82 logging
my $option82_format = $griddhcp->format_log_option_82();
#Modify format of option82 logging
$griddhcp->format_log_option_82('TEXT');
```

email_list( )

Use this method to specify or retrieve the grid-level email_list value. Specify an e-mail address to which you want the Infoblox appliance to send e-mail notifications when the DHCP address usage for the grid crosses a threshold. You can create a list of several e-mail addresses.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

Array reference with the e-mail values.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get email_list
my $email_list = $griddhcp->email_list();
#Modify email_list
$griddhcp->email_list(["admin\@infoblox.com"]);
```

enable_ddns( )

Use this method to set or retrieve the grid-level enable_ddns flag. Set this flag to enable DHCP servers in a grid to send DDNS updates.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

Specify "true" to enable support for DDNS updates or "false" to disable it. The default value is "false".

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get enable_ddns
```

```
my $enable_ddns= $griddhcp->enable_ddns();
#Modify enable_ddns
$griddhcp->enable_ddns("true");
```

enable_dhcp_thresholds( )

Use this method to set or retrieve the grid-level enable_dhcp_thresholds flag. Thresholds represent the watermarks above or below which address usage in a network is unexpected and might warrant your attention.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

Specify "true" to enable DHCP thresholds or "false" to disable them. The default value is "false".

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get enable_dhcp_thresholds
my $enable_dhcp_thresholds = $griddhcp->enable_dhcp_thresholds();
#Modify enable_dhcp_thresholds
$griddhcp->enable_dhcp_thresholds("true");
```

enable_email_warnings( )

Use this method to set or retrieve the grid-level enable_email_warnings flag. If you enable this feature and DHCP threshold is enabled, the Infoblox appliance sends an e-mail notification to an administrator if DHCP address usage crosses a high or low watermark threshold.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

Specify "true" to set the enable_email_warnings flag or "false" to deactivate/unset it. The default value is "false".

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get enable_email_warnings
my $enable_email_warnings = $griddhcp->enable_email_warnings();
#Modify enable_email_warnings
$griddhcp->enable_email_warnings("true");
```

enable_fingerprint( )

Use this method to set or retrieve the Grid-level enable_fingerprint flag. If you enable this feature, the server will match a fingerprint for incoming lease requests.

Parameter

Specify "true" to set the enable_fingerprint flag or "false" to deactivate/unset it. The default value is "true".

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get enable_fingerprint
my $enable_fingerprint = $griddhcp->enable_fingerprint();
#Modify enable_fingerprint
$griddhcp->enable_fingerprint("false");
```

enable_gss_tsig( )

Use this method to retrieve the grid-level enable_gss_tsig flag. This method returns "true" only if both the gss_tsig_key and kdc_server are specified.

This is for DHCPv4 GSS-TSIG authentication.

Parameter

This is a read-only method. You cannot enter any parameter.

Returns

This method returns the enable_gss_tsig attribute value.

Example

```
#Get enable_gss_tsig
my $enable_gss_tsig = $griddhcp->enable_gss_tsig();
```

enable_ifmap( )

Use this method to enable or disable the IF-MAP DHCP client feature on this grid.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

Boolean to enable or disable IF-MAP client feature. The default value is "false".

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get IF-MAP enable value
my $ifmap_enabled = $griddhcp->enable_ifmap();
#Enable IF-MAP Client feature
$griddhcp->enable_ifmap("true");
```

enable_ifmap_publishing( )

Use this method to enable or disable the IF-MAP publishing feature on this grid.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

Boolean to enable or disable IF-MAP publishing feature. The default value is "true".

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get IF-MAP publishing enable value
my $ifmap_publishing_enabled = $griddhcp->enable_ifmap_publishing();
#Disable IF-MAP publishing feature
$griddhcp->enable_ifmap_publishing("false");
```

enable_hostname_rewrite( )

Use this method to set or retrieve the grid enable_hostname_rewrite flag.

Parameter

Boolean to enable or disable hostname rewrite for the grid. The default value is "false".

Cannot enable the hostname rewrite policy along with the Microsoft code page. See microsoft_code_page().

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get enable_hostname_rewrite
$status = $griddhcp->enable_hostname_rewrite();
#Modify enable_hostname_rewrite
$griddhcp->enable_hostname_rewrite("true");
```

enable_roaming_hosts( )

Use this method to set or retrieve the grid-level enable_roaming_hosts flag. Set this flag to enable DHCP servers in a grid to support roaming hosts.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

Specify "true" to set the enable_roaming_hosts flag or "false" to deactivate/unset it. The default value is "false".

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get enable_roaming_hosts
my $enable_roaming_hosts = $griddhcp->enable_roamin_hosts();
#Modify enable_roaming_hosts
$griddhcp->enable_roaming_hosts("true");
```

enable_snmp_warnings( )

Use this method to set or retrieve the grid-level enable_snmp_warnings flag. When DHCP address usage crosses a watermark threshold and DHCP threshold is enabled, the appliance sends an SNMP trap to trap receivers that you defined for the grid.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

Specify "true" to set the enable_snmp_warnings flag or "false" to deactivate/unset it. The default value is "false".

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get enable_snmp_warnings
my $enable_snmp_warnings = $griddhcp->enable_snmp_warnings();
#Modify enable_snmp_warnings
$griddhcp->enable_snmp_warnings("true");
```

grid( )

Use this method to retrieve the grid that serves DHCP. This specifies a group of Infoblox appliances that are connected together to provide a single point of device administration and service configuration in a secure, highly available environment.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

Text with the name of the grid that provides DHCP service. Default value is "Infoblox".

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#get grid
my $grid = $griddhcp->grid();
```

gss_tsig_keys( )

Use this method to set or retrieve the list of GSS-TSIG keys for a Grid DHCPv4 object.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

The valid value is an array reference that contains Infoblox::Grid::KerberosKey objects.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get the Kerberos keys list
my $kerberos_keys_list = $grid_dhcp->gss_tsig_keys();

#Modify the Kerberos keys list
my $kerberos_key1 = Infoblox::Cluster::KerberosKey->new(
       principal => 'host/appliance.ivan.test1@LADYBUG.LOCAL',
       );
my $kerberos_key2 = Infoblox::Cluster::KerberosKey->new(
       principal => 'host/appliance.ivan.test2@LADYBUG.LOCAL',
       );
$grid_dhcp->gss_tsig_keys([$kerberos_key1, $kerberos_key2]);
```

high_water_mark( )

Use this method to set or retrieve the grid-level high_water_mark value. If the percent of allocated addresses exceeds this watermark, the appliance makes a syslog entry and if enabled, sends an e-mail notification.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

Number with the percentage value. Range is from 1 to 100. The high watermark value must be greater than the low watermark value. Default value is 95.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get high_water_mark
my $high_water_mark = $griddhcp->high_water_mark();
#Modify high_water_mark
$griddhcp->high_water_mark(80);
```

high_water_mark_reset( )

Use this method to set or retrieve the grid-level high_water_mark_reset value. If the percentage of allocated addresses drops below this value, a corresponding SNMP trap is reset.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

Number with the percentage value. Range is from 1 to 100. The high watermark value must be greater than the low watermark value. Default value is 85.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get high_water_mark_reset
my $high_water_mark_reset = $griddhcp->high_water_mark_reset();
#Modify high_water_mark_reset
$griddhcp->high_water_mark_reset(80);
```

hostname_rewrite_policy( )

Use this method to retrieve the name of the default hostname rewrite policy, which is also in the protocol_hostname_rewrite_policies array. This attribute is read-only.

Parameter

None.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get hostname_rewrite_policy
my $name = $griddhcp->hostname_rewrite_policy();
```

ifmap_delete_option( )

Use this method to set or retrieve the IF-MAP delete option. The possible values are 'never', 'always', 'older_than'.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

The IF-MAP delete option. The default value is 'always'.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get IF-MAP delete option
my $delete_option = $griddhcp->ifmap_delete_option();
#Modify IF-MAP delete option
$griddhcp->ifmap_delete_option('never');
```

ifmap_ip_mac_lifetime( )

Use this method to set or retrieve the IF-MAP ip-mac (ip-duid) metadata lifetime. This method applies only if the IF-MAP delete option is set to "older_than".

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

The IF-MAP ip mac lifetime.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get IF-MAP ip mac lifetime
my $port = $griddhcp->ifmap_ip_mac_lifetime();
#Modify IF-MAP ip mac lifetime
$griddhcp->ifmap_ip_mac_lifetime(300);
```

ifmap_port( )

Use this method to set or retrieve the IF-MAP server port used by DHCP IF-MAP clients. This setting is optional and defaults to the port associated with the protocol specified in ifmap_url (eg: for HTTP, the port is 80; for HTTPS, the port is 443).

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

The IF-MAP server port number. The default value is undef.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get IF-MAP server port
my $port = $griddhcp->ifmap_port();
#Modify IF-MAP server port
$griddhcp->ifmap_port(1234);
```

ifmap_protocol( )

Use this method to set or retrieve the IF-MAP server protocol. This setting is required and must be one of 'ifmap_20' or 'ifmap_11'.

Parameter

The IF-MAP server protocol. The default value is 'ifmap_20'.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get IF-MAP server protocol
my $protocol = $griddhcp->ifmap_protocol();
#Modify IF-MAP server protocol
$griddhcp->ifmap_protocol('ifmap_11');
```

ifmap_url( )

Use this method to set or retrieve the IF-MAP server URL used by DHCP IF-MAP clients. This setting is mandatory if IF-MAP is enabled on this grid.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

String for the IF-MAP server URL. The default value is undef.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get IF-MAP server URL
my $url = $griddhcp->ifmap_url();
#Modify IF-MAP server URL
$griddhcp->ifmap_url("https://ifmap.infoblox.com");
```

ifmap_server_ca_cert( )

Use this method to set or retrieve the ifmap server CA certificate.

Parameter

An Infoblox::IFMap::CACertificate object.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute server_ca_cert.

Example

```
# Get server_ca_cert
my $server_ca_cert = $griddhcp->server_ca_cert( );

# Change it
$griddhcp->server_ca_cert($ca_cert);
```

ignore_id( )

Use this method to set or retrieve the grid-level ignore_id type. Indicates whether the appliance will ignore DHCP client IDs or MAC addresses.

Parameter

Valid values are 'NONE', 'CLIENT', or 'MACADDR'. The default is 'NONE'.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
# Get attribute value
$value = $griddhcp->ignore_id();
```

```
# Modify attribute value
$griddhcp->ignore_id('CLIENT');
#Un-override ignore_id
$griddhcp->ignore_id(undef);
```

ignore_mac_addresses( )

Use this method to set or retrieve the grid-level ignore_mac_addresses list. A list of MAC addresses the appliance will ignore.

If you enter MAC addresses in the ignore_mac_addresses, then the appliance will ignore them. If you enter empty array, then the appliance will ignore any MAC address.

Parameter

The valid value is an array reference that contains string objects.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
# Get attribute value
$value = $griddhcp->ignore_mac_addresses();
# Modify attribute value
$griddhcp->ignore_mac_addresses(["11::11::11::11::11::11"]);
```

ignore_dhcp_option_list_request( )

Use this method to set or retrieve the grid-level ignore_dhcp_option_list_request flag. If this flag is set to true all available DHCP options will be returned to the client.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

Specify "true" to set the ignore_dhcp_option_list_request flag or "false" to deactivate/unset it. The default value is "false".

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get ignore_dhcp_option_list_request
my $ignore_dhcp_option_list_request = $griddhcp->ignore_dhcp_option_list_request();
#Modify ignore_dhcp_option_list_request
$griddhcp->ignore_dhcp_option_list_request("true");
```

immediate_fa_configuration( )

Use this method to control whether fixed address configuration takes effect immediately without DHCP service restart.

Setting this method to a defined value implicitly sets the override_immediate_fa_configuration method to "true". Setting the parameter to undefined causes the appliance to use the **Grid|MS Server|...** default and automatically resets the override_immediate_fa_configuration attribute to "false".

Note that when immediate_fa_configuration is set to a defined value and override_immediate_fa_configuration is set to "false", the last operation takes precedence. Thus the sequence $object->immediate_fa_configuration(**VAL**); $object->override_immediate_fa_configuration("false"); will set override_immediate_fa_configuration to "false", and the sequence $object->override_immediate_fa_configuration("false"); $object->immediate_fa_configuration(**VAL**); will result in override_immediate_fa_configuration="true".

Parameter

The valid value is an unsigned integer. Default value is "false".

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get immediate_fa_configuration
my $immediate_fa_configuration = $griddhcp->immediate_fa_configuration();
```

```
#Modify disable
$griddhcp->immediate_fa_configuration("true");
```

ipv6_capture_hostname( )

Use this method to set or retrieve the grid-level ipv6_capture_hostname flag. Set this flag to capture the ipv6 hostname and lease time when assigning a fixed address.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

Specify "true" to enable this feature or "false" to disable it. The default value is "false".

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get ipv6_capture_hostname
my $ipv6_capture_hostname = $griddhcp->ipv6_capture_hostname();
#Modify disable
$griddhcp->ipv6_capture_hostname("true");
```

ipv6_ddns_enable_option_fqdn( )

Use this method to set or retrieve the grid-level ipv6_ddns_enable_option_fqdn flag. This method controls whether the FQDN option sent by the client is to be used, or if the server can automatically generate the FQDN.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

Specify "true" to enable this feature or "false" to disable it. The default value is "false".

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get ipv6_ddns_enable_option_fqdn
my $ipv6_ddns_enable_option_fqdn = $griddhcp->ipv6_ddns_enable_option_fqdn();
#Modify disable
$griddhcp->ipv6_ddns_enable_option_fqdn("true");
```

ipv6_ddns_server_always_updates( )

Use this method to set or retrieve the grid-level ipv6_ddns_server_always_updates flag. Set this flag to "false" to update DNS only if requested by the client.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

Specify "true" to enable this feature or "false" to disable it. The default value is "false".

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get ipv6_ddns_server_always_updates
my $ipv6_ddns_server_always_updates = $griddhcp->ipv6_ddns_server_always_updates();
#Modify ipv6_ddns_server_always_updates
$griddhcp->ipv6_ddns_server_always_updates("true");
```

ipv6_ddns_domainname( )

Use this method to set or retrieve the grid-level IPv6 DDNS domain name value.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

The DDNS domain name in FQDN (Fully Qualified Domain Name) format.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get ipv6_ddns_domainname
my $ipv6_ddns_domainname = $griddhcp->ipv6_ddns_domainname();
#Modify ipv6_ddns_domainname
$griddhcp->ipv6_ddns_domainname("domain.com");
```

ipv6_ddns_ttl( )

Use this method to set or retrieve the grid-level IPv6 DDNS TTL.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

The preferred DDNS TTL value in seconds. The default is undefined.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get ipv6_ddns_ttl
my $ipv6_ddns_ttl = $griddhcp->ipv6_ddns_ttl();
#Modify ipv6_ddns_ttl
$griddhcp->ipv6_ddns_ttl(12345);
```

ipv6_default_prefix( )

Use this method to set or retrieve the grid-level IPv6 default prefix.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

The desired IPv6 prefix. It must be one of those listed in the ipv6_prefixes method.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get ipv6_default_prefix
my $ipv6_default_prefix = $griddhcp->ipv6_default_prefix();
#Modify ipv6_default_prefix
$griddhcp->ipv6_default_prefix("2001::");
```

ipv6_dns_update_style( )

Use this method to set or retrieve the update style for dynamic DNS updates (DHCPv6).

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

Valid values are 'INTERIM' and 'STANDARD'.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get ipv6_dns_update_style value
```

```
my $value = $object->ipv6_dns_update_style();

#Modify ipv6_dns_update_style value
$object->ipv6_dns_update_style('STANDARD');
```

ipv6_domain_name( )

Use this method to set or retrieve the ipv6_domain_name value for a Grid DHCP object.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

The name of the client's domain for resolving names in DNS in string format.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get ipv6_domain_name
my $ipv6_domain_name = $griddhcp->ipv6_domain_name();
#Modify ipv6_domain_name
$griddhcp->ipv6_domain_name("test_domain.com");
```

ipv6_domain_name_servers( )

Use this method to set or retrieve the ipv6_domain_name_servers value of a Grid DHCP object.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

The comma-separated domain name server addresses in IPv6 address format.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get ipv6_domain_name_servers
my $ipv6_domain_name_servers = $griddhcp->ipv6_domain_name_servers();
#Modify ipv6_domain_name_servers
$griddhcp->ipv6_domain_name_servers("2001::1,2001::2");
```

ipv6_enable_ddns( )

Use this method to set or retrieve the grid-level ipv6_enable_ddns flag. Set this flag to enable DHCPv6 dynamic DNS updates.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

Specify "true" to enable this feature or "false" to disable it. The default value is "false".

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get ipv6_enable_ddns
my $ipv6_enable_ddns = $griddhcp->ipv6_enable_ddns();
#Modify ipv6_enable_ddns
$griddhcp->ipv6_enable_ddns("true");
```

ipv6_enable_gss_tsig( )

Use this method to retrieve the grid-level ipv6_enable_gss_tsig flag. This method returns "true" only if both the ipv6_gss_tsig_key and ipv6_kdc_server are specified.

This is for DHCPv6 GSS-TSIG authentication.

Parameter

This is a read-only method. You cannot enter any parameter.

Returns

This method returns the ipv6_enable_gss_tsig attribute value.

Example

```
#Get ipv6_enable_gss_tsig
my $ipv6_enable_gss_tsig = $griddhcp->ipv6_enable_gss_tsig();
```

ipv6_enable_lease_scavenge( )

Use this method to set or retrieve the flag that indicates whether the DHCPv6 lease scavenging is enabled or disabled.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

Specify 'true' to enable DHCPv6 lease scavenging and 'false' to disable it. The default value is false.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get ipv6_enable_lease_scavenge
my $ipv6_enable_lease_scavenge = $griddhcp->ipv6_enable_lease_scavenge();
#Modify ipv6_enable_lease_scavenge
$griddhcp->ipv6_enable_lease_scavenge("true");
```

ipv6_remember_expired_client_association( )

Use this method to set or retrieve the flag that indicates whether the DHCPv6 server can 'remember' the client for a configurable amount of time and lease the same IPv6 address to the client.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

Specify 'true' to enable the DHCPv6 server to 'remember' the client and 'false' to disable it. The default value is false.

Returns

If you specified a parameter, the method returns true when the modification succeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get ipv6_remember_expired_client_association
my $ipv6_remember_expired_client_association = $griddhcp->ipv6_remember_expired_client_association();
#Modify ipv6_remember_expired_client_association
$griddhcp->ipv6_remember_expired_client_association("true");
```

ipv6_enable_retry_updates( )

Use this method to set or retrieve the ipv6_enable_retry_updates flag. If this flag is set to "true", the DHCPv6 server retries failed dynamic DNS updates.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

Specify "true" to enable the DHCP server to retry failed DNS updates or "false" to disable it. The default value is "true".

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get ipv6_enable_retry_updates
my $ipv6_enable_retry_updates = $griddhcp->ipv6_enable_retry_updates();
#Modify ipv6_enable_retry_updates
```

```
$griddhcp->ipv6_enable_retry_updates("true");
```

ipv6_generate_hostname( )

Use this method to set or retrieve the grid-level ipv6_generate_hostname flag. Set this flag to generate the hostname if it is not sent by the client.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

Specify "true" to enable this feature or "false" to disable it. The default value is "false".

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get ipv6_generate_hostname
my $ipv6_generate_hostname = $griddhcp->ipv6_generate_hostname();
#Modify ipv6_generate_hostname
$griddhcp->ipv6_generate_hostname("true");
```

ipv6_gss_tsig_keys( )

Use this method to set or retrieve the list of GSS-TSIG keys for a Grid DHCPv6 object.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

The valid value is an array reference that contains Infoblox::Grid::KerberosKey objects.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get the Kerberos keys list
my $kerberos_keys_list = $grid_dhcp->ipv6_gss_tsig_keys();

#Modify the Kerberos keys list
my $kerberos_key1 = Infoblox::Cluster::KerberosKey->new(
       principal => 'host/appliance.ivan.test1@LADYBUG.LOCAL',
       );
my $kerberos_key2 = Infoblox::Cluster::KerberosKey->new(
       principal => 'host/appliance.ivan.test2@LADYBUG.LOCAL',
       );
$grid_dhcp->ipv6_gss_tsig_keys([$kerberos_key1, $kerberos_key2]);
```

ipv6_kdc_server( )

Use this method to specify or retrieve the IPv4 address or FQDN of the Kerberos server for DHCPv6 GSS-TSIG authentication.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

The Kerberos server IPv4 address or FQDN, with a maximum of 1023 bytes.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get ipv6_kdc_server
my $ipv6_kdc_server = $griddhcp->ipv6_kdc_server();
#Modify ipv6_kdc_server
$griddhcp->ipv6_kdc_server("10.32.0.8");
```

ipv6_lease_scavenge_time( )

Use this method to set or retrieve the Grid-level grace period in seconds to keep an expired lease before it gets deleted by the scavenging process.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

The valid value is an unsigned integer greater than or equal to 21600.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get ipv6_lease_scavenge_time
my $ipv6_lease_scavenge_time = $griddhcp->ipv6_lease_scavenge_time();
#Modify ipv6_lease_scavenge_time
$griddhcp->ipv6_lease_scavenge_time(22000);
```

ipv6_microsoft_code_page( )

Use this method to set or retrieve the grid-level Microsoft client DHCP IPv6 code page value. This value is the hostname translation code page for Microsoft DHCP IPv6 clients.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

Microsoft DHCP client code page in string format. Valid values are:

None

Arabic (1256)

Arabic (ISO-8859-6)

Baltic (1257)

Baltic (775)

Baltic (ISO-8859-4)

Central Europe (1250)

Cyrillic (1251)

Cyrillic (855)

Cyrillic (ISO-8859-5)

Greek (1253)

Greek (737)

Greek (ISO-8859-7)

Hebrew (1255)

Hebrew (862)

Hebrew (ISO-8859-8)

Japanese Shift-JIS (932)

Korean (949)

Latin 1 (ISO-8859-1)

Latin 2 (ISO-8859-2)

Latin 3 (ISO-8859-3)

Latin 9 (ISO-8859-15)

Latin I (1252)

Latin II (852)

Multilingual Latin I (850)

Russian (866)

Simplified Chinese GBK (936)

Thai (874)

Traditional Chinese Big5 (950)

Turkish (1254)

Turkish (857)

Turkish (ISO-8859-9)

US (437)

Vietnam (1258)

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get ipv6_microsoft_code_page
my $ipv6_microsoft_code_page = $griddhcp->ipv6_microsoft_code_page();
#Modify ipv6_microsoft_code_page
$griddhcp->ipv6_microsoft_code_page("Cyrillic (ISO-8859-5)");
```

ipv6_options( )

Use this method to set or retrieve the grid-level IPv6 options.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

The valid value is an array reference that contains Infoblox::DHCP::Option object(s).

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get ipv6_options
my $ipv6_options = $griddhcp->ipv6_options();
#Modify ipv6_options
my $option1 = Infoblox::DHCP::Option->new(
    name   => "dhcp6.rapid-commit",
    value  => "true"
);
$griddhcp->ipv6_options([$option1]);
```

ipv6_prefixes( )

Use this method to set or retrieve the grid-level list of IPv6 prefixes.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

A comma-separated list of IPv6 prefixes.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get ipv6_prefixes
my $ipv6_prefixes = $griddhcp->ipv6_prefixes();
#Modify ipv6_prefixes
$griddhcp->ipv6_prefixes("2001::,2002::");
```

ipv6_recycle_leases( )

Use this method to set or retrieve the ipv6_recycle_leases flag of a grid DHCP server. If you enable this feature, and then delete an IPv6 DHCP range, the appliance stores active leases from this range up to one week after the leases expire.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

Specify "true" to set the ipv6_recycle_leases flag or "false" to deactivate/unset it. The default value is "true".

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get ipv6_recycle_leases
my $ipv6_recycle_leases = $griddhcp->ipv6_recycle_leases();
#Modify ipv6_recycle_leases
$griddhcp->ipv6_recycle_leases("true");
```

ipv6_retry_updates_interval( )

Use this method to set or retrieve the ipv6_retry_updates_interval value. Set the retry interval when the member DHCPv6 server makes repeated attempts to send DDNS updates to a DNS server.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

A 32-bit integer (range from 0 to 4294967295) that represents the retry interval in minutes. The default retry interval is five minutes.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get ipv6_retry_updates_interval
my $ipv6_retry_updates_interval = $griddhcp->ipv6_retry_updates_interval();
#Modify ipv6_retry_updates_interval
$griddhcp->ipv6_retry_updates_interval(10);
```

ipv6_txt_record_handling( )

Use this method to set or retrieve the grid-level TXT record handling value. This value specifies how DHCP should treat TXT records when performing DNS updates.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

TXT record handling value in string format. Valid values are: "IGNORE_CONTENTS", "ISC", "ISC_TRANSITIONAL" and "MS". The default value is "ISC".

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get ipv6_txt_record_handling
my $ipv6_txt_record_handling = $griddhcp->ipv6_txt_record_handling();
#Modify ipv6_txt_record_handling
$griddhcp->ipv6_txt_record_handling("ISC_TRANSITIONAL");
```

ipv6_update_dns_on_lease_renewal( )

The ipv6_update_dns_on_lease_renewal attribute controls whether the DHCP server updates DNS when an IPv6 DHCP lease is renewed.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

Specify "true" to set the ipv6_update_dns_on_lease_renewal flag or "false" to deactivate/unset it.

The default value of this parameter is "false".

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get ipv6_update_dns_on_lease_renewal
my $ipv6_update_dns_on_lease_renewal=$griddhcp->ipv6_update_dns_on_lease_renewal( );
#Modify ipv6_update_dns_on_lease_renewal
$griddhcp->ipv6_update_dns_on_lease_renewal("true");
```

kdc_server( )

Use this method to specify or retrieve the IPv4 address or FQDN of the Kerberos server for DHCPv4 GSS-TSIG authentication.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

The Kerberos server IPv4 address or FQDN, with a maximum of 1023 bytes.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get kdc_server
my $kdc_server = $griddhcp->kdc_server();
#Modify  kdc_server
$griddhcp->kdc_server("10.32.0.7");
```

lease_logging_member( )

Use this method to specify or retrieve the grid member on which you want to store the DHCP lease history log. Infoblox recommends that you dedicate a member other than the master as a logging member. If possible, use this member solely for storing the DHCP lease history log. If you do not select a member, no logging can occur.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

The valid value is an array reference that contains Infoblox::DHCP::Member object(s).

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get lease_logging_member
my $lease_logging_member = $griddhcp->lease_logging_member();
#Modify lease_logging_member
my $dhcpmember = Infoblox::DHCP::Member->new(
    name     => "infoblox.localdomain",
    ipv4addr => "192.168.1.2"
);

$griddhcp->lease_logging_member($dhcpmember);
```

lease_per_client_settings( )

Use this method to set or retrieve the setting that defines how DHCP server will free or release client leases.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

The valid values are 'ONE_LEASE_PER_CLIENT', 'RELEASE_MATCHING_ID' and 'NEVER_RELEASE'.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get lease_per_client_settings value
my $value = $object->lease_per_client_settings();

#Modify lease_per_client_settings value
$object->lease_per_client_settings('NEVER_RELEASE');
```

lease_scavenge_time( )

Use this method to set or retrieve the grid-level lease_scavenge_time value.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

An integer that specifies the period of time, in seconds, that free and backup leases remain in the database before they are automatically deleted. To disable lease scavenging, set the parameter to -1. The minimum positive value must be greater than 86400 seconds (1 day). The default value of this parameter is -1, which means that lease scavenging is disabled.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get lease_scavenge_time
my $lease_scavenge = $griddhcp->lease_scavenge_time();
#Modify lease_scavenge_time
$griddhcp->lease_scavenge_time(30 * 24 * 60 * 60);
```

log_lease_events( )

Use this method to set or retrieve the grid-level log_lease_events flag. This value specifies whether grid members log lease events.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

Specify "true" to set the log_lease_events flag or "false" to deactivate/unset it. The default value is "false".

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get log_lease_events
my $log_lease_events = $griddhcp->log_lease_events();
#Modify log_lease_events
$griddhcp->log_lease_events("true");
```

logic_filters( )

Use this method to set or retrieve all the Grid-level logic (ordered) filters.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

The valid value is an array reference that contains either the Infoblox::DHCP::Filter::MAC, Infoblox::DHCP::Filter::NAC, Infoblox::DHCP::Filter::Option objects or the names of the corresponding filters. The filters in the list must satisfy the following conditions:

Filter name must be globally unique.
Filters without expressions must be at the end of the list.
The filters should not be disabled.

The default value of the parameter is undefined.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get logic_filters
my $ref_logic_filters = $griddhcp->logic_filters();
#Modify logic_filters
$griddhcp->logic_filters(["filter_1","filter_2",$exl1]); #$exl1 is an Infoblox::DHCP::Filter::MAC,
Infoblox::DHCP::Filter::NAC or Infoblox::DHCP::Filter::Option object.
```

low_water_mark( )

Use this method to set or retrieve the grid-level low_water_mark value. If the percent of allocated addresses drops below this watermark, the appliance makes a syslog entry and if enabled, sends an e-mail notification.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

Number with the percentage value. Range is from 0 to 100. The low watermark value must be lower than the high watermark value. The default low watermark number is 0.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get low_water_mark
my $low_water_mark = $griddhcp->low_water_mark();
#Modify low_water_mark
$griddhcp->low_water_mark(10);
```

low_water_mark_reset( )

Use this method to set or retrieve the grid-level low_water_mark_reset value. If the percentage of allocated addresses exceeds this value, a corresponding SNMP trap is reset.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

Number with the percentage value. Range is from 0 to 100. The low watermark reset value must be higher than the low watermark value. The default low watermark reset value is 0.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get low_water_mark_reset
my $low_water_mark_reset = $griddhcp->low_water_mark_reset();
#Modify low_water_mark_reset
$griddhcp->low_water_mark_reset(10);
```

microsoft_code_page( )

Use this method to set or retrieve the grid-level Microsoft client DHCP IPv4 code page value. This value is the hostname translation code page for Microsoft DHCP IPv4 clients.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

Microsoft DHCP client code page in string format. Valid values are the same as those for ipv6_microsoft_code_page. See ipv6_microsoft_code_page().

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get microsoft_code_page
my $microsoft_code_page = $griddhcp->microsoft_code_page();
#Modify microsoft_code_page
```

```
$griddhcp->microsoft_code_page("Cyrillic (ISO-8859-5)");
```

nextserver( )

Use this method to set or retrieve the grid-level "next server" value. This value is the IP address or name of the boot file server on which the boot file is stored.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

The boot file server IP address or name in FQDN format. The FQDN consists of the host name followed by the domain name (example: abc.com). A boot file server name can have a maximum of 256 bytes.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get nextserver
my $nextserver = $griddhcp->nextserver();
#Modify nextserver
$griddhcp->nextserver("1.1.1.1");
```

option60_match_rules( )

Use this method to set or retrieve grid-level DHCP option60 match rules. An Option60MatchRule object allows specifying match rules for Option 60 globally.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

The valid value is an array reference that contains Infoblox::DHCP::Option60MatchRule object(s).

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get option60_match_rules
my $option_matchrule = $griddhcp->option60_match_rules();
#Modify option60_match_rules
my $option60_match_rule = Infoblox::DHCP::Option60MatchRule->new(
                                    "option_space" => "infospace",
                                    "match_value" => "Nortel-i2004-A",
                                    "is_substring" => "true",
                                    "substring_offset" => "8",
                                    "substring_length" => "14"
                                  );

# Configure DHCP Option60MatchRule on the Grid DHCP object
$griddhcp->option60_match_rules([$option60_match_rule]);

#Remove Option60MatchRule object
$griddhcp->option60_match_rules([]);
```

options( )

Use this method to set or retrieve grid-level DHCP options. These options provide network configuration settings and various services available on the network.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

The valid value is an array reference that contains Infoblox::DHCP::Option object(s).

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get options
my $options = $griddhcp->options();
#Modify options
my $option1 = Infoblox::DHCP::Option->new(
    name   => "lease-time",
    value  => 50
);

my $option2 = Infoblox::DHCP::Option->new(
    name    => "routers",
    value   => "1.4.1.1,2.2.2.2",
);

my $option3 = Infoblox::DHCP::Option->new(
    name     => "nameservers",
    value    => "1.1.1.1, 2.2.2.2",
);

my $option4 = Infoblox::DHCP::Option->new(
    name  => "domain-name",
    value => "domainname.com",
);

my $option5 = Infoblox::DHCP::Option->new(
    name     => "broadcast",
    value    => "255.0.0.0",
);

$griddhcp->options([$option1, $option2, $option3, $option4, $option5]);
```

override_immediate_fa_configuration( )

The override_immediate_fa_configuration attribute controls whether the immediate_fa_configuration value in the object is used, instead of the Grid default.

The override_immediate_fa_configuration attribute can be specified explicitly. It is also set implicitly when immediate_fa_configuration is set to a defined value.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

Specify "true" to set override_ttl flag or "false" to disable it. Default value is "false".

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get override_immediate_fa_configuration
my $override_immediate_fa_configuration = $griddhcp->override_immediate_fa_configuration();
#Modify override_immediate_fa_configuration
$griddhcp->override_immediate_fa_configuration("true");
```

ping_count( )

Use this method to set or retrieve the grid-level ping_count value. This value specifies the number of pings that the Infoblox appliance sends to an IP address to verify that it is not in use.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

Number of pings. Range is from 0 to 10, where 0 disables pings. The default value is 1.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get ping_count
my $ping_count = $griddhcp->ping_count();
#Modify ping_count
$griddhcp->ping_count(4);
```

ping_timeout( )

Use this method to set or retrieve the grid-level ping_timeout value. This value indicates the number of seconds the appliance waits for a response to its ping.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

Ping timeout value in seconds. Valid values are 0.1, 0.5, 1, 2, 3, 4 and 5 seconds. The default value is 1.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get ping_timeout
my $ping_timeout = $griddhcp->ping_timeout();
#Modify ping_timeout
$griddhcp->ping_timeout(3);
```

preferred_lifetime( )

Use this method to set or retrieve the grid-level preferred lifetime.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

Preferred lifetime value in seconds. The default value is 27000.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get preferred_lifetime
my $preferred_lifetime = $griddhcp->preferred_lifetime();
#Modify preferred_lifetime
$griddhcp->preferred_lifetime(12345);
```

protocol_hostname_rewrite_policies( )

Use this method to set or retrieve a list of hostname rewrite policies. This attribute is mandatory if enable_hostname_rewrite is "true".

Parameter

Array reference with Infoblox::Grid::HostNameRewritePolicy objects.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get protocol_hostname_rewrite_policies
my $policies = $griddhcp->protocol_hostname_rewrite_policies();

#Modify protocol_hostname_rewrite_policies
push(@$policies, $policy2);
$griddhcp->protocol_hostname_rewrite_policies($policies);
```

pxe_lease_time( )

Use this method to set or retrieve the grid-level pxe_lease_time value. This value is the duration of time it takes a host to connect to a boot server, such as a TFTP server, and download the file it needs to boot.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

A 32-bit integer (range from 0 to 4294967295) that specifies the lease time in seconds. Default value is 43200 seconds.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get pxe_lease_time
my $pxe_lease_time = $griddhcp->pxe_lease_time();
#Modify pxe_lease_time
$griddhcp->pxe_lease_time(200);
```

recycle_leases( )

Use this method to set or retrieve the recycle_leases flag of a grid DHCP server. If you enable this feature, and then delete a DHCP range, the appliance stores active leases from this range up to one week after the leases expire.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

Specify "true" to set the recycle_leases flag or "false" to deactivate/unset it. The default value is "true".

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get recycle_leases
my $recycle_leases = $griddhcp->recycle_leases();
#Modify recycle_leases
$griddhcp->recycle_leases("true");
```

restart_setting( )

Use this method to set or retrive Grid DHCP service restart settings.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

The valid value is an Infoblox::Grid::ServiceRestart object.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get restart_setting
my $restart_setting = $grid_dhcp->restart_setting();
#Modify the retry time
$grid_dhcp->restart_setting($restart_setting);
```

retry_ddns_updates( )

Use this method to set or retrieve the grid-level retry_ddns_updates flag. This value indicates whether the grid DHCP servers make repeated attempts to send DDNS updates to a DNS server.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

Specify "true" to enable the DHCP servers to retry DDNS updates or "false" to disable this feature. The default value is "true".

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get retry_ddns_updates
my $retry_ddns_updates = $griddhcp->retry_ddns_updates();
#Modify retry_ddns_updates
```

```
$griddhcp->retry_ddns_updates("true");
```

syslog_facility( )

Use this method to set or retrieve the grid-level syslog_facility value. The syslog facility is the location on the syslog server to which you want to sort syslog messages.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

Valid values are: "daemon", "local0", "local1", "local2", "local3", "local4", "local5", "local6", "local7". The default value is "daemon".

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get syslog_facility
my $syslog_facility = $griddhcp->syslog_facility();
#Modify syslog_facility
$griddhcp->syslog_facility("local0");
```

txt_record_handling( )

Use this method to set or retrieve the grid-level TXT record handling value. This value specifies how DHCP should treat TXT records when performing DNS updates.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

TXT record handling value in string format. Valid values are: "IGNORE_CONTENTS", "ISC", "ISC_TRANSITIONAL" and "MS". The default value is "ISC".

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get txt_record_handling
my $txt_record_handling = $griddhcp->txt_record_handling();
#Modify txt_record_handling
$griddhcp->txt_record_handling("ISC_TRANSITIONAL");
```

update_dns_on_lease_renewal( )

The update_dns_on_lease_renewal attribute controls whether the DHCP server updates DNS when a DHCP lease is renewed.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

Specify "true" to set the update_dns_on_lease_renewal flag or "false" to deactivate/unset it.

The default value of this parameter is "false".

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get update_dns_on_lease_renewal
my $update_dns_on_lease_renewal=$griddhcp->update_dns_on_lease_renewal( );
#Modify update_dns_on_lease_renewal
$griddhcp->update_dns_on_lease_renewal("true");
```

valid_lifetime( )

Use this method to set or retrieve the valid lifetime value on the Grid level.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

Valid lifetime value in seconds. The default value is 43200.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get valid_lifetime
my $valid_lifetime = $roaming_host->valid_lifetime();
#Modify valid_lifetime
$roaming_host->valid_lifetime(12345);
```

validate_ifmap_ca_cert( )

Use this method to set or retrieve the validate CA certificate value.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

Specify "true" to set the validate_ifmap_ca_cert flag or "false" to deactivate/unset it.

The default value of this parameter is "true".

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get validate_ifmap_ca_cert
my $validate_ifmap_ca_cert = $griddhcp->validate_ifmap_ca_cert();
#Modify validate_ifmap_ca_cert
$griddhcp->validate_ifmap_ca_cert('false');
```

SAMPLE CODE

The following sample code demonstrates different operations that can be applied to a grid DHCP object, such as get and modify an object. This sample code also includes error handling for the operations.

#Preparation prior to get and modify operations on a grid DHCP object

```
#PROGRAM STARTS: Include all the modules that will be used
use strict;
use Infoblox;

#refers to Infoblox Appliance IP address
my $host_ip   = "192.168.1.2";
my $host_name = "infoblox.localdomain";

#Creating a session to appliance.
my $session = Infoblox::Session->new(
    master   => $host_ip,
    username => "admin",
    password => "infoblox"
    )
    or die("Constructor for session failed: ",
               Infoblox::status_code(). ":" .Infoblox::status_detail());

print"Session created successfully.\n";

#Creating a domain-name option object.
my $option = Infoblox::DHCP::Option->new(
                 name   => "domain-name",
                 value  => "domainname.com",
)
or die("Constructor for option failed: ",
               Infoblox::status_code(). ":" .Infoblox::status_detail());

print"dhcp option object created successfully.\n";

#Creating a Member object.
```

```
my $dhcpmember = Infoblox::DHCP::Member->new(
    name     => $host_name,
    ipv4addr => $host_ip
    )
    or die("Constructor for dhcpmember failed: ",
               Infoblox::status_code(). ":" .Infoblox::status_detail());

print"dhcp member object created successfully.\n";

#Creating a forward zone nameserver.
my $ns1 = Infoblox::DNS::Nameserver->new(
    ddns_zone => "sub.test.com",
    name      => "server.dns.com",
    ipv4addr  => "1.1.1.1",
    TSIGname  => "tsig_name",
    TSIGkey   => "ZX1jVJl7C58GT/sc7Q3ucA=="
    )
    or die("Constructor for ns1 failed: ",
               Infoblox::status_code(). ":" .Infoblox::status_detail());

print"forward zone nameserver object created successfully.\n";

#Creating a reverse zone nameserver.
my $ns2 = Infoblox::DNS::Nameserver->new(
    ddns_zone => "10.0.0.0/8",
    name      => "server.dns.com",
    ipv4addr  => "1.1.1.1"
    )
    or die("Constructor for ns2 failed: ",
               Infoblox::status_code(). ":" .Infoblox::status_detail());

print"reverse zone nameserver object created successfully.\n";
```

#Get and modify the grid DHCP object.

```
#Getting the DHCP object from appliance through session.
my @result = $session->get(
    object => "Infoblox::Grid::DHCP",
    grid   => "Infoblox"
    )
    or die("get DHCP failed: ",
               $session->status_code(). ":" .$session->status_detail());
print "get grid DHCP Objcet successful.\n";

unless (scalar(@result) == 0) {
    my $griddhcp = $result[0];
    if ($griddhcp) {

        #Modifying the value of the specified objects.
        $griddhcp->authority("false");
        $griddhcp->enable_ddns("true");
        $griddhcp->bootserver("boot.test.com");
        $griddhcp->ddns_ttl(21600);
        $griddhcp->nextserver("77.77.77.7");
        $griddhcp->pxe_lease_time(43200);
        $griddhcp->bootfile("bootfile");
        $griddhcp->ping_count(3);
        $griddhcp->ping_timeout(2);
        $griddhcp->email_list(["admin\@infoblox.com", "bkp_admin\@infoblox.com"]);
        $griddhcp->options([$option]);
        $griddhcp->lease_logging_member($dhcpmember);
        $griddhcp->microsoft_code_page("Cyrillic (ISO-8859-5)");

        #Applying the changes to appliance through session.
        $session->modify($griddhcp)
            or die("Modify DHCP failed: ",
               $session->status_code(). ":" .$session->status_detail());
                print"Modified Grid DHCP Object successfully.\n";
            }
         } else {
             print "No Grid DHCP object found.";
         }

####PROGRAM ENDS####
```

AUTHOR

Infoblox Inc. http://www.infoblox.com/

Infoblox::Grid::DHCP - Grid DHCP services object.

SEE ALSO

Infoblox::DHCP::Member,Infoblox::DHCP::Option,Infoblox::DNS::Nameserver,Infoblox::Grid::KerberosKey,Infoblox::Session,Infoblox::Session->get(),Infoblox::Session->modify()

COPYRIGHT

Copyright (c) 2017 Infoblox Inc.

Infoblox::DHCP::IPv6Range - DHCP IPv6 Range object.

- NAME
- DESCRIPTION
- CONSTRUCTOR
- SESSION METHODS
  - Infoblox::Session->add( )
  - Infoblox::Session->get( )
  - Infoblox::Session->modify( )
  - Infoblox::Session->remove( )
  - Infoblox::Session->search( )
- METHODS
  - address_type( )
  - comment( )
  - cloud_info( )
  - disable( )
  - discovery_basic_poll_setting( )
  - discovery_member( )
  - discover_now_status( )
  - discovery_blackout_setting( )
  - enable_discovery( )
  - enable_immediate_discovery( )
  - end_addr( )
  - endpoint_sources( )
  - exclude( )
  - extattrs( )
  - extensible_attributes( )
  - ipv6_end_prefix( )
  - ipv6_prefix_bits( )
  - ipv6_start_prefix( )
  - member( )
  - name( )
  - network( )
  - network_view( )
  - next_available_ip( )
  - override_blackout_setting( )
  - override_recycle_leases( )
  - override_subscribe_settings( )
  - port_control_blackout_setting
  - restart_if_needed( )
  - recycle_leases( )
  - same_port_control_discovery_blackout( )
  - server_association_type( )
  - start_addr( )
  - subscribe_settings( )
  - template( )
- SAMPLE CODE
- AUTHOR
- SEE ALSO
- COPYRIGHT

NAME

Infoblox::DHCP::IPv6Range - DHCP IPv6 Range object.

DESCRIPTION

A DHCP IPv6 range defines the specified range of IP addresses in an IPv6 network. A DHCP IPv6 range should be added for an IPv6 network so the Infoblox appliance can assign IP addresses within that specified range to DHCP clients. If the client is on an IPv6 network that is assigned a DHCP IPv6 range, the device distributes an available IP address from that range to the DHCP client, or to a DHCP relay agent if the request came through an agent. The DHCP IPv6 range should also be assigned with a device. If devices are in a grid, the particular member serving DHCP for the DHCP IPv6 range must be specified. If the server is an independent device, this device must be specified as the member that serves the DHCP IPv6 range.

CONSTRUCTOR

```
 my $dhcp_ipv6_range = Infoblox::DHCP::IPv6Range->new(
     network                          => $network,                                  #Required
     ipv6_end_prefix                  => $ipv6addr,                                 #Optional / See
NOTE below
```

```
     ipv6_prefix_bits                    => $num,                                  #Optional / See
NOTE below
     ipv6_start_prefix                   => $ipv6addr,                             #Optional / See
NOTE below
     end_addr                            => $ipv6addr,                             #Optional / See
NOTE below
     start_addr                          => $ipv6addr,                             #Optional / See
NOTE below
     address_type                        => "ADDRESS" | "PREFIX" | "BOTH",         #Optional / Default
is undefined
     comment                             => $string,                               #Optional / Default
is undefined
     disable                             => "true" | "false",                      #Optional / Default
is "false"
     exclude                             => [$ExclusionRange1,$ExclusionRange2,...], #Optional / Default
is empty
     extattrs                            => { $string => $extattr, ... },          #Optional / Default
is undefined
     extensible_attributes               => { $string => $string | $num, $string => [ $string | $num, ...
], ... } #Optional / Default is undefined
     member                              => $Member,                               #Optional / Default
is undefined
     name                                => $string,                               #Optional / Default
is undefined
     network_view                        => $NetworkView,                          #Optional / Default
is system-defined default network view
     override_recycle_leases             => "true" | "false"                       #Optional / Default
is "false"
     recycle_leases                      => "true" | "false",                      #Optional / Default
is "true"
     server_association_type             => "NONE" | "MEMBER"                      #Optional / Default
is undefined
     template                            => $string,                               #Optional / Default
is undefined
     discovery_blackout_setting          => $blackout,                             #Optional
     port_control_blackout_setting       => $blackout,                             #Optional
     override_blackout_setting           => "true" | "false",                      #Optional / Default
value is "false"
     same_port_control_discovery_blackout => "true" | "false",                     #Optional / Default
value is "false"
     subscribe_settings                  => $subscribe_settings,                   #Optional / Default
is undefined
     override_subscribe_settings         => 'true' | 'false',                      #Optional / Default
is 'false'
 );
```

NOTE: one or both of the start_addr/end_addr and ipv6_start_prefix/ipv6_end_prefix/ipv6_prefix_bits parameter sets is required.

You cannot set both extattrs and extensible_attributes attributes at the same time.

SESSION METHODS

This section describes all the methods in an Infoblox::Session module that you can apply to a DHCP IPv6 range object.

Infoblox::Session->add( )

Use this method to add a DHCP IPv6 range object to the Infoblox appliance. See Infoblox::Session->add() for parameters and return values.

Example

```
#Construct an object
my $dhcp_ipv6range = Infoblox::DHCP::IPv6Range->new(
                                                start_addr => "2001:db8:0:c000:0001::",
                                                end_addr   => "2001:db8:0:c000:0021::",
                                                network    => "2001:db8:0:c000::/51",
);

# Submit for addition
my $response = $session->add( $dhcp_ipv6range );
```

Infoblox::Session->get( )

Use this method to retrieve all the matching objects from the Infoblox appliance. See Infoblox::Session->get() for parameters and return values.

'discover_now_status' will not be returned by default, use 'return_methods' to request it if needed.

Key References

```
Apply the following attributes to get a specific DHCP IPv6 range object:

  start_addr             - Optional. Starting IP address of the DHCP IPv6 range within the network.
  end_addr               - Optional. Ending IP address of the DHCP IPv6 range within the network.
  ipv6_start_prefix      - Optional. Starting prefix of the DHCP IPv6 range.
  ipv6_end_prefix        - Optional. Ending prefix of the DHCP IPv6.
  network_view           - Optional. Network view in which the DHCP IPv6 range is located. By
default, all network views are searched.
  extattrs               - Optional. A hash reference containing extensible attributes.
  extensible_attributes  - Optional. A hash reference containing extensible attributes.
  member                 - Optional. The member serving the range. The valid value is an
Infoblox::DHCP::Member object.
  name                   - Optional. The name of the DHCP IPv6 Range.
  network                - Optional. The IPv6 Network the range resides in.
  comment                - Optional. The descriptive comment.
  server_association_type - Optional. The server association type for the range.
```

Examples

```
my @retrieved_objs = $session->get(
                     object     => "Infoblox::DHCP::IPv6Range",
                     start_addr => "2001:db8:0:c000:0001::" );

my @retrieved_objs = $session->get(
                     object       => "Infoblox::DHCP::IPv6Range",
                     start_addr   => "2001:db8:0:c000:0001::",
                     network_view => "default" );

my @retrieved_objs = $session->get(
                     object                => "Infoblox::DHCP::IPv6Range",
                     extensible_attributes => { 'Site' => 'Santa Clara' });
```

Infoblox::Session->modify( )

Use this method to modify a DHCP IPv6 range object in the Infoblox appliance. See Infoblox::Session->modify() for parameters and return values.

Example

```
# Use method to modify the comment.
$dhcp_ipv6range->comment("this is a modified comment");
# Submit modification
my $response = $session->modify( $dhcp_ipv6range );
```

Infoblox::Session->remove( )

Use this method to remove a DHCP IPv6 range object from the Infoblox appliance. See Infoblox::Session->remove() for parameters and return values.

To remove a specific object, first use `get()` or `search()` to retrieve the specific DHCP IPv6 range object, and then submit this object for removal.

Example

```
# Get the objects with the same name
my @retrieved_objs = $session->get(
    object       => "Infoblox::DHCP::IPv6Range",
    start_addr   => "2001:db8:0:c000:0001::",
    newtork_view => "default" );
# Find the desired object from the retrieved list.
my $desired_dhcp_range = $retrieved_objs[0];
# Submit for removal
my $response = $session->remove( $desired_dhcp_range );
```

Infoblox::Session->search( )

Use this method to search for DHCP IPv6 range objects in the Infoblox appliance. See Infoblox::Session->search() for parameters and return values.

'discover_now_status' will not be returned by default, use 'return_methods' to request it if needed.

Key References

```
Apply the following attributes to search for a DHCP IPv6 range object:

  start_addr             - Optional. Starting IP address of the DHCP IPv6 range within the network
(regular expression).
  end_addr               - Optional. Ending IP address of the DHCP IPv6 range within the network
(regular expression).
  ipv6_start_prefix      - Optional. Starting prefix of the DHCP IPv6 range (regular expression).
  ipv6_end_prefix        - Optional. Ending prefix of the DHCP IPv6 (regular expression).
```

```
  network_view           - Optional. Network view in which the DHCP IPv6 range is located. By
default, all network views are searched.
  extattrs               - Optional. A hash reference containing extensible attributes.
  extensible_attributes  - Optional. A hash reference containing extensible attributes.
  member                 - Optional. The member serving the range. The valid value is an
Infoblox::DHCP::Member object.
  name                   - Optional. The name of the DHCP IPv6 Range (regular expression).
  network                - Optional. The IPv6 Network the range resides in (regular expression).
  comment                - Optional. The descriptive comment (regular expression).
  server_association_type - Optional. The server association type for the range.
```

For more information about searching extensible attributes, see Infoblox::Grid::ExtensibleAttributeDef/Searching Extensible Attributes.

Examples

```
# search for DHCP IPv6 range object with start address having last octet as 1
my @retrieved_objs = $session->search(
    object     => "Infoblox::DHCP::IPv6Range",
    network       => "2001:db8:0:c000::/51",
    network_view  => "default",
    start_addr    => "0021",
    comment       => ".*comment" );

# search for DHCP IPv6 range objects defining "Santa Clara" for "Site" extensible attribute
my @retrieved_objs = $session->search(
    object                => "Infoblox::DHCP::IPv6Range",
    extensible_attributes => { 'Site' => 'Santa Clara' });
```

METHODS

This section describes all the methods that you can use to configure and retrieve the attribute values of a DHCP IPv6 range.

address_type( )

Use this method to set or retrieve the address type of a DHCP IPv6 Range object.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

Valid values are "ADDRESS", "PREFIX", or "BOTH". When the address type is "ADDRESS", values for the 'start_addr' and 'end_addr' members are required. When the address type is "PREFIX", values for 'ipv6_start_prefix', 'ipv6_end_prefix', and 'ipv6_prefix_bits' are required. When the address type is "BOTH", values for 'start_addr', 'end_addr', 'ipv6_start_prefix', 'ipv6_end_prefix', and 'ipv6_prefix_bits' are all required.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get address_type
my $address_type = $dhcp_ipv6range->address_type();
#Modify address_type
$dhcp_ipv6range->address_type("PREFIX");
```

comment( )

Use this method to set or retrieve a descriptive comment.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

Desired comment in string format with a maximum of 256 bytes. The default value is undefined.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get comment
my $comment = $dhcp_ipv6range->comment();
```

```
#Modify comment
$dhcp_ipv6range->comment("Modifying the DHCP IPv6 range comment");
```

cloud_info( )

Use this method to set or retrieve cloud API related information for the Infoblox::DHCP::IPv6Range object.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

The valid value is a Infoblox::Grid::CloudAPI::Info object.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
# Get cloud_info
my $cloud_info = $dhcp_ipv6range->cloud_info();
# Modify cloud_info
$dhcp_ipv6range->cloud_info($cloud_info_object);
```

disable( )

Use this method to set or retrieve the disable flag of a DHCP IPv6 range.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

Specify "true" to set the disable flag or "false" to deactivate/unset it. The default value for this field is false.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get disable
my $disable = $dhcp_ipv6range->disable();
#Modify disable
$dhcp_ipv6range->disable("false");
```

discovery_basic_poll_setting( )

Use this method to set or retrieve the discovery_basic_poll_setting value of a DHCP IPv6 range.

Include the specified parameter to set the attribute value.

Parameter

The valid value is object of the Infoblox::Grid::Discovery::BasicPollSettings manpage.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get discovery_basic_poll_setting
my $discovery_basic_poll_setting = $dhcp_ipv6range->discovery_basic_poll_setting();
#Modify discovery_basic_poll_setting
$dhcp_ipv6range->discovery_basic_poll_setting($poll_setting);
```

discovery_member( )

Use this method to set or retrieve the discovery_member value of a DHCP IPv6 range.

Parameter

The discovery member name in string format.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get discovery_member
my $discovery_member = $dhcp_ipv6range->discovery_member();
#Modify discovery_member
$dhcp_ipv6range->discovery_member("test.localdomain");
```

discover_now_status( )

Use this method to retrieve the current discovery status. This attribute is read-only.

'discover_now_status' will not be returned by default, use 'return_methods' to request it if needed.

Parameter

None.

Returns

The method returns the attribute value that can be 'COMPLETE', 'FAILED', 'NONE', 'PENDING' or 'RUNNING'.

Example

```
#Get discover_now_status
my $discover_now_status = $dhcp_ipv6range->discover_now_status();
```

discovery_blackout_setting( )

Use this method to set or retrieve the discovery blackout setting.

Setting this method to a defined value implicitly sets the override_blackout_setting method to "true". If port_control_blackout_setting is undefined, setting the parameter to undefined causes the appliance to use the grid default and automatically resets the override_blackout_setting attribute to "false".

Note that when discovery_blackout_setting is set to a defined value and override_blackout_setting is set to "false", the last operation takes precedence. Thus the sequence $network->discovery_blackout_setting($blackout); $network->override_blackout_setting("false"); will set override_blackout_setting to "false", and the sequence $network->override_blackout_setting("false"); $network->discovery_blackout_setting($blackout); will result in override_blackout_setting="true".

Parameter

Valid value is an Infoblox::Grid::Discovery::Properties::BlackoutSetting object.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get discovery_blackout_setting
my $discovery_blackout_setting = $dhcp_ipv6range->discovery_blackout_setting();
#Modify discovery_blackout_setting
$dhcp_ipv6range->discovery_blackout_setting($blackout);
```

enable_discovery( )

Use this method to set or retrieve the enable_discovery value of a DHCP IPv6 range. This setting overrides the member level settings.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Setting this method to a defined value implicitly sets the override_enable_discovery method to "true". Setting the parameter to undefined causes the appliance to use the default from the upper level and automatically resets the override_enable_discovery attribute to "false".

Enable this flag if you want to set discovery for this object.

Parameter

Specify "true" to set the enable_discovery flag or "false" to deactivate/unset it. The default value is undefined.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get enable_discovery
my $enable_discovery = $dhcp_ipv6range->enable_discovery();
#Modify enable_discovery
$dhcp_ipv6range->enable_discovery("false");
```

enable_immediate_discovery( )

Use this method to set or retrieve enable_immediate_discovery value of a DHCP IPv6 range.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

Specify "true" to set the enable_immediate_discovery flag or "false" to deactivate/unset it. The default value is undefined.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get enable_immediate_discovery
my $enable_immediate_discovery = $dhcp_ipv6range->enable_immediate_discovery();
#Modify enable_immediate_discovery
$dhcp_ipv6range->enable_immediate_discovery("false");
```

end_addr( )

Use this method to set or retrieve the end address of the DHCP IPv6 range.

An end address is the last IP address in the DHCP IPv6 range available for the clients.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

Specify the end address of the DHCP IPv6 range in lowercase.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get end_addr
my $end_addr = $dhcp_ipv6range->end_addr();
#Modify end_addr
$dhcp_ipv6range->end_addr("2001:db8:0:c000:0021::");
```

endpoint_sources( )

Use this method to retrieve the list of endpoints that provide object data. This is a read-only attribute.

Omit the parameter to retrieve the attribute value.

Parameter

None

Returns

The valid return value is an array of Infoblox::CiscoISE::Endpoint objects.

Example

```
#Get endpoint_sources value
my $endpoint_sources = $object->endpoint_sources();
```

exclude( )

Use this method to set or retrieve the exclusion range of IP addresses in the DHCP IPv6 range which should not be assigned to clients.

If static addresses in the middle of a DHCP IPv6 range are assigned to certain hosts, those addresses can be excluded from the DHCP IPv6 range using this method.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

Valid value is an array reference that contains defined Infoblox::DHCP::ExclusionRange objects. The default value is empty.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get exclude
my $ref_exclude = $dhcp_ipv6range->exclude();
#Modify exclude
$dhcp_ipv6range->exclude([$exl1]); #$exl1 is an Infoblox::DHCP::ExclusionRange object
```

extattrs( )

Use this method to set or retrieve the extensible attributes associated with a DHCP IPv6 range object.

Parameter

Valid value is a hash reference containing the names of extensible attributes and their associated values (Infoblox::Grid::Extattr objects).

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get extattrs
my $ref_extattrs = $dhcp_ipv6range->extattrs();
#Modify extattrs
$dhcp_ipv6range->extattrs({ 'Site' => $extattr1, 'Administrator' => $extattr2 });
```

extensible_attributes( )

Use this method to set or retrieve the extensible attributes associated with a DHCP IPv6 range object

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

For valid values for extensible attributes, see Infoblox::Grid::ExtensibleAttributeDef/Extensible Attribute Values.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get extensible attributes
my $ref_extensible_attributes = $dhcp_ipv6range->extensible_attributes();
#Modify extensible attributes
$dhcp_ipv6range->extensible_attributes({ 'Site' => 'Santa Clara', 'Administrator' => [ 'Peter',
'Tom' ] });
```

ipv6_end_prefix( )

Use this method to set or retrieve the end prefix of the DHCP IPv6 range.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

Specify the end prefix of the DHCP IPv6 range in lowercase.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get ipv6_end_prefix
my $ipv6_end_prefix = $dhcp_ipv6range->ipv6_end_prefix();
#Modify ipv6_end_prefix
$dhcp_ipv6range->ipv6_end_prefix("2001:db8:0:c000:0021::");
```

ipv6_prefix_bits( )

Use this method to set or retrieve the prefix bits for the DHCP IPv6 range.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

An unsigned integer between 2 and 128.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get ipv6_prefix_bits
my $ipv6_prefix_bits = $dhcp_ipv6range->ipv6_prefix_bits();
#Modify ipv6_prefix_bits
$dhcp_ipv6range->ipv6_prefix_bits(32);
```

ipv6_start_prefix( )

Use this method to set or retrieve the start prefix of the DHCP IPv6 range.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

Specify the start prefix of the DHCP IPv6 range in lowercase.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get ipv6_start_prefix
my $ipv6_start_prefix = $dhcp_ipv6range->ipv6_start_prefix();
#Modify ipv6_start_prefix
$dhcp_ipv6range->ipv6_start_prefix("2001:db8:0:c000:0011::");
```

member( )

Use this method to set or retrieve the member that serves DHCP for this IP address range.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

If the attribute is set, the server_association_type attribute will also be set: to "MEMBER" if setting member to a defined value, to "NONE" otherwise.

Parameter

The valid value is a defined Infoblox::DHCP::Member object. The default value is empty.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get member
my $ref_member = $dhcp_ipv6range->member();
#Modify member
$dhcp_ipv6range->member($member1); #$member is an Infoblox::DHCP::Member object
```

name( )

Use this method to set or retrieve a descriptive name for the DHCP IPv6 range.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

The desired name in string format with a maximum of 256 bytes. The default value is undefined.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get name
my $name = $dhcp_ipv6range->name();
#Modify name
$dhcp_ipv6range->name("external");
```

network( )

Use this method to set or retrieve the DHCP IPv6 network of the address range.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

The DHCP IPv6 Network.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get network
my $network = $dhcp_ipv6range->network();
#Set the network
$dhcp_ipv6range->network("2001:db8:0:c000::/51");
```

network_view( )

Use this method to set or retrieve the network view of the DHCP IPv6 range.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

The default value is the "default" network view, which means the DHCP IPv6 range is in the default network view.

Parameter

Valid value is a defined Infoblox::DHCP::View object. The default network view value is the system-defined default network view object.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get network view
my $nview = $dhcp_ipv6range->network_view();
#Modify network view, with an Infoblox::DHCP::View object
```

```
$dhcp_ipv6range->network_view($nview);
```

next_available_ip( )

Use this method to get the next available IPv6 address(es) in the DHCP IPv6 range object. Note that to use this method, the object MUST be retrieved via a get or search call, or created with an add call.

Parameter

The method accepts the following optional parameters:

```
  requested_num - The number of IP addresses requested. It must be a value between 1 and 20,
inclusive.
  excluded - A reference to an array of addresses to be excluded from the results.
```

Returns

This method returns the next available IPv6 address(es) in the range. If requested_num is not specified, a string with the next available address will be returned; otherwise, the method will return a reference to an array of addresses.

Within a DHCP Range object, the next available IP address(es) are the first addresses in this range that match all of the following rules:

```
   * Are not part of the passed 'excluded' parameter (if specified).
   * There is no fixed address or host address record with this IP address
   * There is no unmanaged record with this IP address
   * The IP address is not in any excluded range of this IPv6 DHCP range
   * There is no active lease with this IP address (Deleted leases in the recycle bin are not
considered.)
```

Example

```
# Retrieve a particular range
my @retrieved_objs = $session->get(
                        object     => "Infoblox::DHCP::IPv6Range",
                        start_addr => "10:9::"
);

$range = $retrieved_objs[0];

# Print the next available IPv6 address in the range
print $range->next_available_ip() . "\n";

# This will retrieve the next 10 available addresses and print the first one.
my $addressesref = $range->next_available_ip(requested_num => 10);
print @$addressesref[0] . "\n";

# This will retrieve the next 10 available addresses and exclude two.
my $addressesref = $range->next_available_ip(requested_num => 10, excluded => ["10:9::2",
"10:9::3"]);
```

override_blackout_setting( )

The override_blackout_setting attribute controls whether the discovery_blackout_setting and port_control_blackout_setting values in the object are used, instead of the grid default.

The override_blackout_setting attribute can be specified explicitly. It is also set implicitly when discovery_blackout_setting or port_control_blackout_setting is set to a defined value.

Parameter

Specify "true" to set the override_blackout_setting flag or "false" to deactivate/unset it. The default value is "false".

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
# Get attribute value
my $value = $dhcp_range->override_blackout_setting();
# Modify attribute value
$dhcp_range->override_blackout_setting('true');
```

override_recycle_leases( )

The override_recycle_leases attribute controls whether the recycle_leases value in the DHCP IPv6 Range is used, instead of the grid

default.

The override_recycle_leases attribute can be specified explicitly. It is also set implicitly when recycle_leases is set to a defined value.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

Set the parameter to "true" to override the grid-level setting for recycle_leases. Set the parameter to "false" to inherit the grid-level setting for recycle_leases.

The default value of this parameter is "false".

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Getting override_recycle_leases
my $override_recycle_leases=$dhcp_ipv6range->override_recycle_leases( );
#Modifying override_recycle_leases
$dhcp_ipv6range->override_recycle_leases("true");
```

override_subscribe_settings( )

The override_subscribe_settings attribute controls whether the subscribe_settings method values of the member are used, instead of the Grid default.

The override_subscribe_settings attribute can be specified explicitly. It is also set implicitly when subscribe_settings is set to a defined value.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

Set the parameter to "true" to override the Grid-level setting for subscribe_settings. Set the parameter to "false" to inherit the Grid-level setting for subscribe_settings.

The default value of this parameter is "false".

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get override_subscribe_settings value
my $override_subscribe_settings = $object->override_subscribe_settings( );
#Modify override_subscribe_settings value
$object->override_subscribe_settings("true");
```

port_control_blackout_setting

Use this method to set or retrieve the port control blackout setting.

Setting this method to a defined value implicitly sets the override_blackout_setting method to "true". If discovery_blackout_setting is undefined, setting the parameter to undefined causes the appliance to use the grid default and automatically resets the override_blackout_setting attribute to "false".

Note that when port_control_blackout_setting is set to a defined value and override_blackout_setting is set to "false", the last operation takes precedence. Thus the sequence $network->port_control_blackout_setting($blackout); $network->override_blackout_setting("false"); will set override_blackout_setting to "false", and the sequence $network->override_blackout_setting("false"); $network->port_control_blackout_setting($blackout); will result in override_blackout_setting="true".

Parameter

Valid value is an Infoblox::Grid::Discovery::Properties::BlackoutSetting object.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get port_control_blackout_setting
my $port_control_blackout_setting = $dhcp_ipv6range->port_control_blackout_setting();
#Modify port_control_blackout_setting
$dhcp_ipv6range->port_control_blackout_setting($blackout);
```

restart_if_needed( )

Use this method to set the flag that indicates whether DHCP service should be restarted on a member that is serving DHCP for the changed object referenced in a cloud API request.

Parameter

Specify true to restart the DHCP service.

Returns

None

Example

```
# Modify
$network->restart_if_needed('true');
```

recycle_leases( )

Use this method to set or retrieve the recycle_leases flag of the DHCP IPv6 range.

If the flag is enabled, the leases are kept in recycle bin until one week after expiration. When the flag is disabled, the leases are irrecoverably deleted.

Setting this method to a defined value implicitly sets the override_recycle_leases method to "true". Setting the parameter to undefined causes the appliance to use the grid default and automatically resets the override_recycle_leases attribute to "false".

Note that when recycle_leases is set to a defined value and override_recycle_leases is set to "false", the last operation takes precedence. Thus the sequence $object->recycle_leases("true"); $object->override_recycle_leases("false"); will set override_recycle_leases to "false", and the sequence $objct->override_recycle_leases("false"); $object->recycle_leases("true"); will result in override_recycle_leases="true".

Parameter

Specify "true" to set the recycle_leases flag or "false" to deactivate/unset it. The default value is undefined.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get recycle_leases
my $recycle_leases = $dhcp_ipv6range->recycle_leases();
#Modify recycle_leases
$dhcp_ipv6range->recycle_leases("false");
```

same_port_control_discovery_blackout( )

Use this method to set or retrieve the flag that enables using the same discovery blackout for port control blackout.

Parameter

Specify 'true' to use same discovery blackout for port control blackout or 'false' to disable it. The default value is 'false'.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
# Get attribute value
my $value = $dhcp_ipv6range->same_port_control_discovery_blackout();
# Modify attribute value
$dhcp_ipv6range->same_port_control_discovery_blackout('true');
```

server_association_type( )

Use this method to set or retrieve the DHCP IPv6 range server association type.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

The server_association_type attribute can be specified explicitly. It is also set implicitly when member is set.

Parameter

Valid values are "NONE" and "MEMBER".

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get server_association_type
my $server_association_type = $dhcp_ipv6range->server_association_type();
#Modify server_association_type
$dhcp_ipv6range->server_association_type("MEMBER");
```

start_addr( )

Use this method to set or retrieve the start address of the DHCP IPv6 range.

A start address is the first IP address in the DHCP IPv6 range available for the clients.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

Specify the start address of the DHCP IPv6 range in lowercase.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get start_addr
my $start_addr = $dhcp_ipv6range->start_addr();
#Modify start_addr
$dhcp_ipv6range->start_addr("2001:db8:0:c000:0001::");
```

subscribe_settings( )

Use this method to set or retrieve the Cisco ISE subscribe settings.

Setting this method to a defined value implicitly sets the override_subscribe_settings method to "true". Setting the parameter to undefined causes the appliance to use the Cisco ISE endpoint default and automatically resets the override_subscribe_settings method to "false".

Note that when subscribe_settings is set to a defined value and override_subscribe_settings is set to "false", the last operation takes precedence. Thus the sequence $object->subscribe_settings($value); $object->override_subscribe_settings("false"); will set override_subscribe_settings to "false", and the sequence $object->override_subscribe_settings("false"); $object->subscribe_settings($value); will result in override_subscribe_settings="true".

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

The valid value is an Infoblox::CiscoISE::SubscribeSetting object.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get subscribe_settings value
my $subscribe_settings = $object->subscribe_settings();
#Modify subscribe_settings value
$object->subscribe_settings($settings);
```

template( )

Use this method to set or retrieve the DHCP IPv6 range template name for a DHCP IPv6 Range object.

Template is the name of a DHCP IPv6 range template which will be used to create the DHCP IPv6 range.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

The valid name in string format.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

The template parameter can be used to only add a new DHCP IPv6 range and cannot be used during object modify or returned during object get.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get template
my $template = $dhcp_ipv6range->template();
#Modify template
$dhcp_ipv6range->template("Template_Name");
```

SAMPLE CODE

The following sample code demonstrates the different functions that can be applied to a DHCP IPv6 range object, such as add, search, modify, and remove. This sample also includes error handling for the operations.

#Preparation prior to a DHCP IPv6 Range object insertion

```
#PROGRAM STARTS: Include all the modules that will be used
use strict;
use Infoblox;

#Create a session to the Infoblox appliance
my $session = Infoblox::Session->new(
               master   => "192.168.1.2", #appliance host ip
               username => "admin",       #appliance user login
               password => "infoblox"     #appliance password
);
unless ($session) {
       die("Construct session failed: ",
            Infoblox::status_code() . ":" . Infoblox::status_detail());
}
print "Session created successfully\n";

#Create a DHCP Member object
my $memberdhcp = Infoblox::DHCP::Member->new(
    name     => "infoblox.localdomain",
    ipv4addr => "192.168.1.2"
);
unless($memberdhcp) {
       die("Construct member failed: ",
            Infoblox::status_code() . ":" . Infoblox::status_detail());
}
print "DHCP Member object created successfully\n";

#Create the Network object with the member
my $network = Infoblox::DHCP::IPv6Network->new(
    network => "2001:db8:0:a000::/51",
    comment => "add network",
    members => [ $memberdhcp ]
);
unless($network) {
       die("Construct Network object failed: ",
            Infoblox::status_code() . ":" . Infoblox::status_detail());
}
print "Network object created successfully\n";

#Add the Network object into the Infoblox appliance through a session
$session->add($network)
```

```
    or die("Add Network object failed: ",
            $session->status_code() . ":" . $session->status_detail());
print "Network object added to Infoblox appliance successfully\n";

#Construct exclusion range object
my $exclusion_range = Infoblox::DHCP::ExclusionRange->new(
    start_address => "2001:db8:0:a000:0001::1a",
    end_address   => "2001:db8:0:a000:0001::fa",
    comment       => "Excluded range"
);
```

#Create a DHCP IPv6 Range object

```
my $dhcp_range = Infoblox::DHCP::IPv6Range->new(
                                        start_addr => '2001:db8:0:a000:0001::',
                                        end_addr   => '2001:db8:0:a000:0021::',
                                        network    => '2001:db8:0:a000::/51',
                                        comment    => 'Random comment',
                                        disable    => 'true',
                                        exclude => [$exclusion_range],
                                        ipv6_start_prefix       => '2001:db8:0:c001:0001::',
                                        ipv6_end_prefix         => '2001:db8:0:c001:0021::',
                                        ipv6_prefix_bits        => 12,
                                        member                  => $memberdhcp,
                                        name                    => 'randomrange',
                                        override_recycle_leases => 'true',
                                        recycle_leases          => 'false',
                                        server_association_type => 'MEMBER',
);
unless($dhcp_range) {
       die("Construct DHCP IPv6 Range object failed: ",
            Infoblox::status_code() . ":" . Infoblox::status_detail());
}
print "DHCP IPv6 Range object created successfully\n";

#Add the DHCP IPv6 Range object into the Infoblox appliance through a session
$session->add($dhcp_range)
    or die("Add DHCP IPv6 Range object failed: ",
            $session->status_code() . ":" . $session->status_detail());
print "DHCP IPv6 Range object added to the network successfully\n";
```

#Create a DHCP IPv6 Range object using a template

```
my $dhcp_range_template = Infoblox::DHCP::IPv6RangeTemplate->new(
        name                    => "custom_range_template",
        offset                  => "10",
        number_of_addresses     => "90",
);
unless($dhcp_range_template) {
       die("Construct DHCP IPv6 Range template object failed: ",
            Infoblox::status_code() . ":" . Infoblox::status_detail());
}
print "DHCP IPv6 Range template object created successfully\n";

#Add the DHCP IPv6 Range template object into the Infoblox appliance through a session
$session->add($dhcp_range_template)
    or die("Add DHCP IPv6 Range template object failed: ",
            $session->status_code() . ":" . $session->status_detail());
print "DHCP IPv6 Range template object added to the network successfully\n";

my $dhcp_range = Infoblox::DHCP::IPv6Range->new(
        start_addr              => "2001:db8:0:a000:0100::",
        end_addr                => "2001:db8:0:a000:0200::",
        network                 => "2001:db8:0:a000::/51",
        template                => "custom_range_template"
);
unless($dhcp_range) {
       die("Construct DHCP IPv6 Range object failed: ",
            Infoblox::status_code() . ":" . Infoblox::status_detail());
}
print "DHCP IPv6 Range object created successfully\n";

#Add the DHCP IPv6 Range object into the Infoblox appliance through a session
$session->add($dhcp_range)
    or die("Add DHCP IPv6 Range object failed: ",
            $session->status_code() . ":" . $session->status_detail());
```

```
print "DHCP IPv6 Range object added to the network successfully\n";
```

#Get and modify a DHCP IPv6 Range

```
#Get a DHCP IPv6 Range through the session
my @retrieved_objs = $session->get(
    object     => "Infoblox::DHCP::IPv6Range",
    start_addr => "2001:db8:0:a000:0001::",
);
my $object = $retrieved_objs[0];
unless ($object) {
       die("Get DHCP IPv6 Range failed: ",
            $session->status_code() . ":" . $session->status_detail());
}
print "Get DHCP IPv6 Range object found at least 1 matching entry\n";

#Modify one of the attributes of the specified DHCP IPv6 Range
$object->comment ("modified comment");

#Apply the changes
$session->modify($object)
    or die("Modify DHCP IPv6 Range failed: ",
            $session->status_code() . ":" . $session->status_detail());
print "DHCP IPv6 Range object modified successfully \n";
```

#Search a DHCP IPv6 Range

```
# search for DHCP IPv6 range object with start address having last octet as 1
my @retrieved_objs = $session->search(
    object     => "Infoblox::DHCP::IPv6Range",
    network    => "2001:db8:0:a000::/51",
    comment    => "Modified.*"
);

my $object = $retrieved_objs[0];
unless ($object) {
    die("Search DHCP IPv6 range failed: ",
        $session->status_code() . ":" . $session->status_detail());
}
print "Search DHCP IPv6 range object found at least 1 matching entry\n";
```

#Remove a DHCP IPv6 Range

```
#Get a DHCP IPv6 Range through the session
my @retrieved_objs = $session->get(
    object     => "Infoblox::DHCP::IPv6Range",
    start_addr => "2001:db8:0:a000:0001::",
);

my $object = $retrieved_objs[0];
unless ($object) {
       die("Get DHCP IPv6 Range failed: ",
            $session->status_code() . ":" . $session->status_detail());
}
print "Get DHCP IPv6 Range object found at least 1 matching entry\n";

#submit the object for removal
$session->remove($object)
    or die("Remove DHCP IPv6 Range failed: ",
            $session->status_code() . ":" . $session->status_detail());
print "DHCP IPv6 Range object removed successfully \n";

#submit the Range Template object for removal
$session->remove($dhcp_range_template)
    or die("Remove DHCP IPv6 Range Tempate failed: ",
            $session->status_code() . ":" . $session->status_detail());
print "DHCP IPv6 Range Templateobject removed successfully \n";

#Get a DHCP IPv6 Range through the session
my @retrieved_objs = $session->get(
    object     => "Infoblox::DHCP::IPv6Range",
    start_addr => "2001:db8:0:a000:0100::",
);

my $object = $retrieved_objs[0];
unless ($object) {
       die("Get DHCP IPv6 Range failed: ",
```

```
            $session->status_code() . ":" . $session->status_detail());
}
print "Get DHCP IPv6 Range object found at least 1 matching entry\n";

#submit the object for removal
$session->remove($object)
    or die("Remove DHCP IPv6 Range failed: ",
            $session->status_code() . ":" . $session->status_detail());
print "DHCP IPv6 Range object removed successfully \n";

#Remove network

my @retrieved_objs = $session->get(
    object     => "Infoblox::DHCP::IPv6Network",
    network    => "2001:db8:0:a000::/51",
);

my $object = $retrieved_objs[0];
unless ($object) {
       die("Get DHCP IPv6 Network failed: ",
            $session->status_code() . ":" . $session->status_detail());
}
print "Get DHCP IPv6 Network object found at least 1 matching entry\n";

#submit the object for removal
$session->remove($object)
    or die("Remove DHCP IPv6 Network failed: ",
            $session->status_code() . ":" . $session->status_detail());
print "DHCP IPv6 Network object removed successfully \n";

####PROGRAM ENDS####
```

AUTHOR

Infoblox Inc. http://www.infoblox.com/

SEE ALSO

Infoblox::Session, Infoblox::Session->add(), Infoblox::Session->get(), Infoblox::Session->modify(), Infoblox::Session->remove(), Infoblox::Session->search(),Infoblox::DHCP::View, Infoblox::DHCP::ExclusionRange, Infoblox::DHCP::Member, Infoblox::DHCP::Option, Infoblox::DHCP::IPv6RangeTemplate

---

COPYRIGHT

Copyright (c) 2017 Infoblox Inc.

Infoblox::DHCP::IPv6FixedAddrTemplate - DHCP IPv6 Fixed Address Template object

- NAME
- DESCRIPTION
- CONSTRUCTOR
- MODULE METHODS
  - Infoblox::DHCP::NetworkTemplate-> fixed_address_templates( )
- SESSION METHODS
  - Infoblox::Session->add( )
  - Infoblox::Session->get( )
  - Infoblox::Session->search( )
  - Infoblox::Session->modify( )
  - Infoblox::Session->remove( )
- METHODS
  - comment( )
  - domain_name( )
  - domain_name_servers( )
  - extattrs( )
  - extensible_attributes( )
  - name( )
  - number_of_addresses( )
  - offset( )
  - options( )
  - override_domain_name( )
  - override_domain_name_servers( )
  - override_options( )
  - override_preferred_lifetime( )
  - override_valid_lifetime( )
  - preferred_lifetime( )
  - valid_lifetime( )
- SAMPLE CODE
- AUTHOR
- SEE ALSO
- COPYRIGHT

NAME

Infoblox::DHCP::IPv6FixedAddrTemplate - DHCP IPv6 Fixed Address Template object

---

DESCRIPTION

An IPv6 fixed address template contains a set of predefined properties and can be used to create an IPv6 fixed address object in a quick and consistent way. An IPv6 fixed address created from an IPv6 fixed address template inherits the properties defined in the template.

CONSTRUCTOR

```
my $fixed_addr_template = Infoblox::DHCP::IPv6FixedAddrTemplate -> new(
        name                          => $string,                                  #Required
        comment                       => $string,                                  #Optional / Default is
undefined
        domain_name                   => $fqdn,                                    #Optional / Default is
undefined
        domain_name_servers           => "$ipv6addr1,$ipv6addr2,...",              #Optional / Default is
undefined
        extattrs                      => { $string => $extattr, ... },             #Optional / Default is
undefined
        extensible_attributes         => { $string => $string | $num, $string => [ $string | $num, ... ],
... }, #Optional / Default is undefined
        number_of_addresses           => $num | undef,                             #Optional / Default is
undefined
        offset                        => $num | undef,                             #Optional / Default is
undefined
        options                       => [$option1, $option2],                     #Optional / Default is empty
        override_domain_name          => "true" | "false",                         #Optional / Default is
"false"
        override_domain_name_servers => "true" | "false",                          #Optional / Default is
"false"
        override_options              => "true" | "false",                         #Optional / Default is
"false"
        override_preferred_lifetime   => "true" | "false",                         #Optional / Default is
"false"
```

```
        override_valid_lifetime      => "true" | "false",                #Optional / Default is
"false"
        preferred_lifetime           => $num,                            #Optional / Default is 27000
        valid_lifetime               => $num,                            #Optional / Default is 43200
 );
```

You cannot set both extattrs and extensible_attributes attributes at the same time.

MODULE METHODS

The following functions can be applied to a DHCP IPv6 Fixed Address Template object.

Infoblox::DHCP::NetworkTemplate-> fixed_address_templates( )

Use this function to include a fixed address template in a network template on the Infoblox appliance. See Infoblox::DHCP::IPv6NetworkTemplate->fixed_address_templates() for parameters and return values.

Example

```
#Create the IPv6 network template that includes the IPv6 fixed address template
my $template = Infoblox::DHCP::Template->new(
  name                    => "custom_fa_template"
);
my $network_template = Infoblox::DHCP::IPv6NetworkTemplate->new(
  name                    => "custom_network_template",
  comment                 => "add network",
  fixed_address_templates => [$template]
);
```

SESSION METHODS

This section describes all the methods in the Infoblox::Session module that can be applied to a DHCP IPv6 fixed address template object.

Infoblox::Session->add( )

Use this method to add a DHCP IPv6 fixed address template object to the Infoblox appliance. See Infoblox::Session->add() for parameters and return values.

Example

```
#Construct a DHCP IPv6 Fixed Address Template object
my $fixed_addr_template = Infoblox::DHCP::IPv6FixedAddrTemplate->new(
);

#Submit for addition
my $response = $session->add($fixed_addr_template);
```

Infoblox::Session->get( )

Use this method to retrieve all the matching objects from the Infoblox appliance. See Infoblox::Session->get() for parameters and return values.

Key References

```
Apply the following attributes to get a specific DHCP IPv6 Fixed Address Template object:

name - Optional. The name of the fixed address template.
extattrs     - Optional. A hash reference containing extensible attributes.
extensible_attributes - Optional. A hash reference containing extensible attributes.
```

Examples

```
#Get DHCP IPv6 fixed address template object through the session
my @retrieved_objs = $session->get(
    object => "Infoblox::DHCP::IPv6FixedAddrTemplate",
    name   => "custom_fa_template",
);

my @retrieved_objs = $session->get(
  object                => "Infoblox::DHCP::IPv6FixedAddrTemplate",
  extensible_attributes => { 'Site' => 'Santa Clara' }
);
```

Infoblox::Session->search( )

Use this method to retrieve all the matching objects from the Infoblox appliance. See Infoblox::Session->search() for parameters and return values.

Key References

```
Apply the following attributes to search for a DHCP IPv6 Fixed Address Template object:

name    - Optional. The name of the fixed address template (regular expression).
comment - Optional. The comment of the fixed address template (regular expression).
extattrs     - Optional. A hash reference containing extensible attributes.
extensible_attributes - Optional. A hash reference containing extensible attributes.
```

For more information about searching extensible attributes, see Infoblox::Grid::ExtensibleAttributeDef/Searching Extensible Attributes.

Examples

```
#Search for a DHCP IPv6 fixed address template object through the session
my @retrieved_objs = $session->search(
    object  => "Infoblox::DHCP::IPv6FixedAddrTemplate",
    name    => "custom_fa_template",
    comment => ".*",
);

#Search for DHCP IPv6 Fixed Address Template objects that specify "Santa Clara" for the "Site"
extensible attribute
my @retrieved_objs = $session->search(
    object                => "Infoblox::DHCP::IPv6FixedAddrTemplate",
    extensible_attributes => { 'Site' => 'Santa Clara' }
);
```

Infoblox::Session->modify( )

Use this method to modify a DHCP IPv6 fixed address template object in the Infoblox appliance. See Infoblox::Session->modify() for parameters and return values.

Example

```
#Modify the pxe_lease_time
$fixed_addr_templateess->pxe_lease_time("1234");
#Submit modification
my $response = $session->modify( $fixed_addr_templateess );
```

Infoblox::Session->remove( )

Use this method to remove a DHCP IPv6 fixed address template object from the Infoblox appliance. See Infoblox::Session->remove() for parameters and return values.

To remove a specific object, first use `get()` to retrieve the DHCP IPv6 Fixed Address Template object, and then submit it for removal.

Example

```
#Get DHCP IPv6 fixed address template object through the session
my @retrieved_objs = $session->get(
    object => "Infoblox::DHCP::IPv6FixedAddrTemplate",
    name   => "custom_fa_template",
);
#Find the desired object from the retrieved list.
my $desired_fixed_addr_template = $retrieved_objs[0];
#Submit for removal
my $response = $session->remove( $desired_fixed_addr_template );
```

METHODS

This section describes all the methods that can be used to set and retrieve the attribute values of a DHCP IPv6 Fixed Address Template object.

comment( )

Use this method to set or retrieve the descriptive comment of a DHCP IPv6 Fixed Address Template object.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

Desired comment in string format with a maximum of 256 bytes.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails. If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get comment
my $comment = $fixed_addr_template->comment();
#Modify comment
$fixed_addr_template->comment("Modified DHCP Fixed Address template object comment");
```

domain_name( )

Use this method to set or retrieve the domain_name value of a DHCP IPv6 Fixed Address Template object.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Setting this method to a defined value implicitly sets the override_domain_name method to "true". Setting the parameter to undefined causes the appliance to use the grid default and automatically resets the override_domain_name attribute to "false".

Note that when domain_name is set to a defined value and override_domain_name is set to "false", the last operation takes precedence. Thus the sequence $object->domain_name("testdomain"); $object->override_domain_name("false"); will set override_domain_name to "false", and the sequence $object->override_domain_name("false"); $object->domain_name("testdomain"); will result in override_domain_name="true".

Parameter

The client's domain name in string format.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get domain_name
my $domain_name = $fixed_addr_template->domain_name();
#Modify domain_name
$fixed_addr_template->domain_name("test_domain.com");
```

domain_name_servers( )

Use this method to set or retrieve the domain_name_servers value of a DHCP IPv6 fixed address template.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Setting this method to a defined value implicitly sets the override_domain_name_servers method to "true". Setting the parameter to undefined causes the appliance to use the upper-level default and automatically resets the override_domain_name_servers attribute to "false".

Note that when domain_name_servers is set to a defined value and override_domain_name_servers is set to "false", the last operation takes precedence. Thus the sequence $object->domain_name_servers("2001::1,2001::2"); $object->override_domain_name_servers("false"); will set override_domain_name_servers to "false", and the sequence $object->override_domain_name_servers("false"); $object->domain_name_servers("2001::1,2001::2"); will result in override_domain_name_servers="true".

Parameter

Comma-separated list of the IPv6 addresses of the DNS servers.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get domain_name_servers
my $domain_name_servers = $fixed_addr_template->domain_name_servers();
#Modify domain_name_servers
$fixed_addr_template->domain_name_servers("2001::1,2001::2");
```

extattrs( )

Use this method to set or retrieve the extensible attributes associated with a DHCP IPv6 Fixed Address Template object.

Parameter

Valid value is a hash reference containing the names of extensible attributes and their associated values (Infoblox::Grid::Extattr objects).

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get extattrs
my $ref_extattrs = $fixed_addr_template->extattrs();
#Modify extattrs
$fixed_addr_template->extattrs({ 'Site' => $extattr1, 'Administrator' => $extattr2 });
```

extensible_attributes( )

Use this method to set or retrieve the extensible attributes associated with a DHCP IPv6 Fixed Address Template object.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

For valid values for extensible attributes, see Infoblox::Grid::ExtensibleAttributeDef/Extensible Attribute Values.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get extensible attributes
my $ref_extensible_attributes = $fixed_addr_template->extensible_attributes();
#Modify extensible attributes
$fixed_addr_template->extensible_attributes({ 'Site' => 'Santa Clara', 'Administrator' => [ 'Peter',
'Tom' ] });
```

name( )

Use this method to set or retrieve the name of a DHCP IPv6 Fixed Address Template object.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

Name of the IPv6 fixed address template in string format and with a maximum of 64 characters.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.
If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get name
my $name = $fixed_address_template->name();
#Modify name
$fixed_address_template->name("custom_fa_template");
```

number_of_addresses( )

Use this method to set or retrieve the number of addresses in a DHCP IPv6 fixed address template.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

The number of addresses in this fixed address template.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.
If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get number_of_addresses
my $number_of_addresses = $fixed_address_template-> number_of_addresses();
#Modify number_of_addresses
$fixed_address_template->number_of_addresses("10");
```

offset( )

Use this method to set or retrieve the address offset of an IPv6 fixed address template.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

The address offset of this IPv6 fixed address template. The offset is calculated from the start address of the IPv6 network template that includes the fixed address template. For example, in the IPv6 network 2001:0:0:10::/8, if the offset is set to 10 and number_of_addresses is set to 10, then 10 fixed address objects will be created. The start address will be 2001:0:0:10::A and the end address will be 2001:0:0:10::14.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails. If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get offset
my $offset = $fixed_address_template->offset();
#Modify offset
$fixed_address_template->offset("10");
```

options( )

Use this method to set or retrieve the options of a DHCP IPv6 fixed address template object.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

See Infoblox::DHCP::Option for parameters and methods available for this object.

Setting this method to a defined value implicitly sets the override_options method to "true". Setting the parameter to undefined causes the appliance to use the grid default and automatically resets the override_options attribute to "false".

Note that when the options method is set to a defined value and override_options is set to "false", the last operation takes precedence. Thus the sequence $object->options([$opt]); $object->override_options("false"); will set override_options to "false", and the sequence $object->override_options("false"); $object->options([$opt]); will result in override_options="true".

Parameter

Options describe network configuration settings and various services available on the network. These options occur as variable-length fields at the end of DHCP messages.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails. If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get options
my $options = $fixed_addr_template->options();
#Modify options
$fixed_addr_template->options([$option1, $option2]);
```

override_domain_name( )

The override_domain_name attribute controls whether the domain_name value in the DHCP IPv6 fixed address template is used, instead of the upper-level default.

The override_domain_name attribute can be specified explicitly. It is also set implicitly when domain_name is set to a defined value.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

Set the parameter to "true" to override the upper-level setting for domain_name. Set the parameter to "false" to inherit the upper-level setting for domain_name.

The default value of this parameter is "false".

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get override_domain_name
my $override_domain_name=$fixed_addr_template->override_domain_name( );
#Modify override_domain_name
$fixed_addr_template->override_domain_name("true");
```

override_domain_name_servers( )

The override_domain_name_servers attribute controls whether the domain_name_servers value in the DHCP IPv6 fixed address template is used, instead of the upper-level default.

The override_domain_name_servers attribute can be specified explicitly. It is also set implicitly when domain_name_servers is set to a defined value.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

Set the parameter to "true" to override the upper-level setting for domain_name_servers. Set the parameter to "false" to inherit the upper-level setting for domain_name_servers.

The default value of this parameter is "false".

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get override_domain_name_servers
my $override_domain_name_servers=$fixed_addr_template->override_domain_name_servers( );
#Modify override_domain_name_servers
$fixed_addr_template->override_domain_name_servers("true");
```

override_options( )

The override_options attribute controls whether the options in the DHCP IPv6 fixed address template are used, instead of the upper-level default.

The override_options attribute can be specified explicitly. It is also set implicitly when options are set to defined values.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

Set the parameter to "true" to override the upper-level setting for options. Set the parameter to "false" to inherit the upper-level setting for options.

The default value of this parameter is "false".

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get override_options
my $override_options=$fixed_addr_template->override_options( );
#Modify override_options
$fixed_addr_template->override_options("true");
```

override_preferred_lifetime( )

The override_preferred_lifetime attribute controls whether the preferred_lifetime value in the DHCP IPv6 fixed address template is used, instead of the upper-level default.

The override_preferred_lifetime attribute can be specified explicitly. It is also set implicitly when preferred_lifetime is set to a defined value.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

Set the parameter to "true" to override the upper-level setting for options. Set the parameter to "false" to inherit the upper-level setting for options.

The default value of this parameter is "false".

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Getting override_preferred_lifetime
my $override_preferred_lifetime=$fixed_addr_template->override_preferred_lifetime( );
#Modifying override_preferred_lifetime
$fixed_addr_template->override_preferred_lifetime("true");
```

override_valid_lifetime( )

The override_valid_lifetime attribute controls whether the valid_lifetime value in the DHCP IPv6 fixed address template object is used, instead of the upper-level default.

The override_valid_lifetime attribute can be specified explicitly. It is also set implicitly when valid_lifetime is set to a defined value.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

Set the parameter to "true" to override the upper-level setting for valid_lifetime. Set the parameter to "false" to inherit the upper-level setting for valid_lifetime.

The default value of this parameter is "false".

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Getting override_valid_lifetime
my $override_valid_lifetime=$fixed_addr_template->override_valid_lifetime( );
#Modifying override_valid_lifetime
$fixed_addr_template->override_valid_lifetime("true");
```

preferred_lifetime( )

Use this method to set or retrieve the preferred lifetime of the DHCP IPv6 fixed address template.

Setting this method to a defined value implicitly sets the override_preferred_lifetime method to "true". Setting the parameter to undefined causes the appliance to use the upper-level default and automatically resets the override_preferred_lifetime attribute to "false".

Note that when preferred_lifetime is set to a defined value and override_preferred_lifetime is set to "false", the last operation takes precedence. Thus the sequence $object->preferred_lifetime(20000); $object->override_preferred_lifetime("false"); will set override_preferred_lifetime to "false", and the sequence $object->override_preferred_lifetime("false"); $object->preferred_lifetime(20000); will result in override_preferred_lifetime="true".

Parameter

Preferred lifetime value in seconds. The default value is 27000. Note that the preferred lifetime cannot be greater than the valid lifetime.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get preferred_lifetime
my $preferred_lifetime = $fixed_addr_template->preferred_lifetime();
#Modify preferred_lifetime
$fixed_addr_template->preferred_lifetime(12345);
```

valid_lifetime( )

Use this method to set or retrieve the valid lifetime of the DHCP IPv6 fixed address template.

Setting this method to a defined value implicitly sets the override_valid_lifetime method to "true". Setting the parameter to undefined causes the appliance to use the upper-level default and automatically resets the override_valid_lifetime attribute to "false".

Note that when valid_lifetime is set to a defined value and override_valid_lifetime is set to "false", the last operation takes precedence. Thus the sequence $object->valid_lifetime(20000); $object->override_valid_lifetime("false"); will set override_valid_lifetime to "false", and the sequence $object->override_valid_lifetime("false"); $object->valid_lifetime(20000); will result in override_valid_lifetime="true".

Parameter

Valid lifetime value in seconds. The default value is 43200.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get valid_lifetime
my $valid_lifetime = $fixed_addr_template->valid_lifetime();
#Modify valid_lifetime
$fixed_addr_template->valid_lifetime(123456);
```

SAMPLE CODE

The following sample code demonstrates the different functions that can be applied to an object such as add, get, modify, and remove. Also, this sample includes error handling for the operations.

```
#PROGRAM STARTS: Include all the modules that will be used
use strict;
use Infoblox;

#Create a session to the Infoblox appliance
my $session = Infoblox::Session->new(
    master   => "192.168.1.2",
    username => "admin",
    password => "infoblox"
);

unless ($session) {
    die("Construct session failed: ",
        Infoblox::status_code() . ":" . Infoblox::status_detail());
}
print "Session created successfully\n";

my $option1 = Infoblox::DHCP::Option->new(
    "name"  => "dhcp6.fqdn",
    "value" => "mydomain.com"
);
unless($option1) {
    die("Construct option1 object failed: ",
        Infoblox::status_code() . ":" . Infoblox::status_detail());
}
print "option1 object created successfully\n";

my $option2 = Infoblox::DHCP::Option->new(
    "name"     => "dhcp6.info-refresh-time",
    "value"    => "1000",
);
unless($option2) {
    die("Construct option2 object failed: ",
        Infoblox::status_code() . ":" . Infoblox::status_detail());
}
print "option2 object created successfully\n";
```

#Create a DHCP IPv6 Fixed Address Template object

```
#Create a DHCP IPv6 Fixed Address Template object
my $fixed_addr_template = Infoblox::DHCP::IPv6FixedAddrTemplate->new(
    "name"                  => "custom_fa_template",
    "comment"               => "add IPv6 fixed address template",
    "domain_name"           => "test_domain.com",
    "number_of_addresses"   => 1,
    "offset"                => 10,
    "options"               => [ $option1, $option2 ],
    "preferred_lifetime"    => 1000,
    "valid_lifetime"        => 3600,
);
unless($fixed_addr_template) {
    die("Construct IPv6 Fixed Address template failed: ",
        Infoblox::status_code() . ":" . Infoblox::status_detail());
}
print "DHCP IPv6 Fixed Address template object created successfully\n";

#Verify if the DHCP IPv6 Fixed Address template exists in the Infoblox appliance
my $object = $session->get(
    object => "Infoblox::DHCP::IPv6FixedAddrTemplate",
    name   => "custom_fa_template"
);

unless ($object) {
    print "IPv6 Fixed Address template does not exist on server, safe to add the IPv6 fixed address
template\n";
    $session->add($fixed_addr_template)
        or die("Add IPv6 Fixed Address template failed: ",
        $session->status_code() . ":" . $session->status_detail());
}
print "DHCP IPv6 Fixed Address template added successfully\n";
```

#Search for a DHCP IPv6 Fixed Address Template object

```
#Search for a DHCP IPv6 fixed address template object through the session
my @retrieved_objs = $session->search(
    object => "Infoblox::DHCP::IPv6FixedAddrTemplate",
    name   => "custom_fa_template",
);
my $object = $retrieved_objs[0];

unless ($object) {
    die("Search IPv6 Fixed Address template object failed: ",
        $session->status_code() . ":" . $session->status_detail());
}
print "Search IPv6 Fixed Address template object found at least 1 matching entry\n";
```

#Get and modify a DHCP IPv6 Fixed Address Template object

```
#Get DHCP IPv6 fixed address template object through the session
my @retrieved_objs = $session->get(
    object => "Infoblox::DHCP::IPv6FixedAddrTemplate",
    name   => "custom_fa_template",
);
my $object = $retrieved_objs[0];

unless ($object) {
    die("Get IPv6 Fixed Address template object failed: ",
        $session->status_code() . ":" . $session->status_detail());
}
print "Get IPv6 Fixed Address template object found at least 1 matching entry\n";

#Modify one of the attributes of the obtained IPv6 Fixed Address Template object
$object->valid_lifetime(1234);

#Apply the changes
$session->modify($object)
    or die("Modify IPv6 Fixed Address template failed: ",
    $session->status_code() . ":" . $session->status_detail());

print "DHCP IPv6 Fixed Address template added successfully\n";
```

#Create the IPv6 network template object and include the IPv6 fixed address template

```
my $template = Infoblox::DHCP::Template->new(
    name => "custom_fa_template",
);
```

```
my $network_template = Infoblox::DHCP::IPv6NetworkTemplate->new(
    name     => "custom_network_template",
    comment => "add network",
    fixed_address_templates => [$template]
);

unless($network_template) {
    die("Construct Network template object failed: ",
        Infoblox::status_code() . ":" . Infoblox::status_detail());
}

print "Network template object created successfully\n";

#Add the IPv6 network template object to the Infoblox appliance through a session
my $resp = $session->add($network_template)
    or die("Add Network object failed: ",
    $session->status_code() . ":" . $session->status_detail());

print "Network template object added to Infoblox appliance successfully\n";
```

#Remove an IPv6 fixed address template object

```
#Remove an IPv6 network template object
$session->remove($network_template)
    or die("Remove Network Template template object failed: ",
    $session->status_code() . ":" . $session->status_detail());
print "Network Template object removed successfully \n";

#Get an IPv6 Fixed Address Template object through the session
my @retrieved_objs = $session->get(
    object => "Infoblox::DHCP::IPv6FixedAddrTemplate",
    name   => "custom_fa_template",
);
my $object = $retrieved_objs[0];
unless ($object) {
    die("Get IPv6 Fixed Address template object failed: ",
        $session->status_code() . ":" . $session->status_detail());
}
print "Get IPv6 Fixed Address template object found at least 1 matching entry\n";

#Submit the object for removal
$session->remove($object)
    or die("Remove IPv6 Fixed Address template object failed: ",
    $session->status_code() . ":" . $session->status_detail());
print "IPv6 Fixed Address template object removed successfully \n";

####PROGRAM ENDS####
```

AUTHOR

Infoblox Inc. http://www.infoblox.com/

SEE ALSO

Infoblox::Session, Infoblox::Session->get(), Infoblox::Session->add(), Infoblox::Session->remove(), Infoblox::Session->modify(), Infoblox::DHCP::IPv6NetworkTemplate, Infoblox::DHCP::IPv6FixedAddr, Infoblox::DHCP::IPv6RangeTemplate,Infoblox::DHCP::Template,Infoblox::Session, Infoblox::DHCP::Option

COPYRIGHT

Copyright (c) 2017 Infoblox Inc.

Infoblox::DHCP::IPv6NetworkTemplate - DHCP IPv6 Network Template object

- NAME
- DESCRIPTION
- CONSTRUCTOR
- SESSION METHODS
  - Infoblox::Session->add( )
  - Infoblox::Session->get( )
  - Infoblox::Session->search( )
  - Infoblox::Session->modify( )
  - Infoblox::Session->remove( )
- METHODS
  - allow_any_netmask( )
  - auto_create_reversezone( )
  - cidr( )
  - comment( )
  - ddns_domainname( )
  - ddns_enable_option_fqdn( )
  - ddns_generate_hostname( )
  - ddns_server_always_updates( )
  - ddns_ttl( )
  - domain_name( )
  - domain_name_servers( )
  - enable_ddns( )
  - extattrs( )
  - extensible_attributes( )
  - fixed_address_templates( )
  - ipv6prefix( )
  - members( )
  - name( )
  - options( )
  - override_ddns_domainname( )
  - override_ddns_enable_option_fqdn( )
  - override_ddns_generate_hostname( )
  - override_ddns_ttl( )
  - override_domain_name( )
  - override_domain_name_servers( )
  - override_enable_ddns( )
  - override_options( )
  - override_preferred_lifetime( )
  - override_recycle_leases( )
  - override_update_dns_on_lease_renewal( )
  - override_valid_lifetime( )
  - preferred_lifetime( )
  - range_templates( )
  - recycle_leases( )
  - rir( )
  - rir_organization( )
  - rir_registration_action( )
  - rir_registration_status( )
  - send_rir_request( )
  - update_dns_on_lease_renewal( )
  - valid_lifetime( )
- SAMPLE CODE
- AUTHOR
- SEE ALSO
- COPYRIGHT

NAME

Infoblox::DHCP::IPv6NetworkTemplate - DHCP IPv6 Network Template object

---

DESCRIPTION

You can use a template to create IPv6 networks in a quick and consistent way. Networks created from a network template inherit all the properties defined in the network template, except for the comment and cidr if they are defined in the network.

CONSTRUCTOR

```
my $ipv6network_template = Infoblox::DHCP::IPv6NetworkTemplate->new(
```

```
        name                                => $string,
#Required
        allow_any_netmask                   => "true" | "false",
#Optional / Default is "false"
        auto_create_reversezone             => "true" | "false",
#Optional / Default is "false"
        cidr                                => $num,
#Optional / Default is undefined
        comment                             => $string,
#Optional / Default is empty
        ddns_domainname                     => $fqdn,
#Optional / Default is undefined
        ddns_enable_option_fqdn             => "true" | "false" ,
#Optional / Default is "false"
        ddns_generate_hostname              => "true" | "false" | undef,
#Optional / Default is undefined
        ddns_server_always_updates          => "true" | "false",
#Optional / Default is "true"
        ddns_ttl                            => $num,
#Optional / Default is 0
        domain_name                         => $fqdn,
#Optional / Default is undefined
        domain_name_servers                 => "$ipv6addr1,$ipv6addr2,...",
#Optional / Default is undefined
        enable_ddns                         => "true" | "false" | undef,
#Optional / Default is undefined
        extattrs                            => { $string => $extattr, ... },
#Required if 'rir_organization' attribute is defined

#Otherwise is optional
        extensible_attributes               => { $string => $string | $num, $string => [ $string |
$num, ... ], ... },

#Required if 'rir_organization' attribute is defined

#Otherwise is optional

        fixed_address_templates             => [$DhcpTemplate1,$DhcpTemplate2,...],
#Optional / Default is empty
        ipv6prefix                          => $ipv6prefix,
#Optional / Default is undefined
        members                             => [$Member1, $Member2, ...],
#Optional / Default is empty
        options                             => [$Option1, $Option2, ...],
#Optional / Default is empty list
        override_ddns_domainname            => "true" | "false",
#Optional / Default is "false"
        override_ddns_enable_option_fqdn    => "true" | "false",
#Optional / Default is "false"
        override_ddns_generate_hostname     => "true" | "false",
#Optional / Default is "false"
        override_ddns_ttl                   => "true" | "false",
#Optional / Default is "false"
        override_domain_name                => "true" | "false",
#Optional / Default is "false"
        override_domain_name_servers        => "true" | "false",
#Optional / Default is "false"
        override_enable_ddns                => "true" | "false",
#Optional / Default is "false"
        override_options                    => "true" | "false",
#Optional / Default is "false"
        override_preferred_lifetime         => "true" | "false",
#Optional / Default is "false"
        override_recycle_leases             => "true" | "false",
#Optional / Default is "false"
        override_update_dns_on_lease_renewal => "true" | "false",
#Optional / Default is "false"
        override_valid_lifetime             => "true" | "false",
#Optional / Default is "false"
        preferred_lifetime                  => $num,
#Optional / Default is 27000
        range_templates                     => [$RangeTemplate1,$RangeTemplate2,...],
#Optional / Default is empty
        recycle_leases                      => "true" | "false" | undef,
#Optional / Default is undefined
        rir_organization                    => $rir_org,
#Optional / An Infoblox::Grid::RIR::Organization object
        rir_registration_action             => 'NONE' | 'CREATE',
#Optional / Default is 'NONE'
        rir_registration_status             => 'NOT_REGISTERED' | 'REGISTERED',
#Optional / Default is 'NOT_REGISTERED'
        send_rir_request                    => 'true' | 'false',
#Optional / Default is 'false'
        update_dns_on_lease_renewal         => "true" | "false",
#Optional / Default is "false"
```

```
        valid_lifetime                          => $num,
#Optional / Default is 43200
        );
```

You cannot set both extattrs and extensible_attributes attributes at the same time.

SESSION METHODS

This section describes all the methods in an Infoblox::Session module that you can apply to a DHCP IPv6 network template object.

Infoblox::Session->add( )

Use this method to add an IPv6 network template object to the Infoblox appliance. See Infoblox::Session->add() for parameters and return values.

Example

```
#Construct an object
my $ipv6network_template = Infoblox::DHCP::IPv6NetworkTemplate->new(
       name               => "custom_network_template",
       allow_any_netmask => "true",
);

# Submit for addition
my $response = $session->add( $ipv6network_template );
```

Infoblox::Session->get( )

Use this method to retrieve the matching IPv6 network template objects from the Infoblox appliance. See Infoblox::Session->get() for parameters and return values.

Key References

```
Apply the following attributes to get a specific DHCP IPv6 network template object:

 name - Required. The name of network template.
 extattrs     - Optional. A hash reference containing extensible attributes.
```

Example

```
 my  @result_array = $session->get(
      object  => "Infoblox::DHCP::IPv6NetworkTemplate" ,
      name    => "custom_network_template"
 );
```

Infoblox::Session->search( )

Use this method to retrieve all the matching IPv6 network template objects from the Infoblox appliance. See Infoblox::Session->search() for parameters and return values.

Key References

```
Apply the following attributes to search for a specific DHCP IPv6 network template object:

 name     - Optional. The name of the network template (regular
            expression).
 comment - Optional. The comment of the network template (regular
            expression).
 extattrs     - Optional. A hash reference containing extensible attributes.
 extensible_attributes - Optional. A hash reference containing extensible attributes.
```

For more information about searching extensible attributes, see Infoblox::Grid::ExtensibleAttributeDef/Searching Extensible Attributes.

Example

```
 my  @result_array = $session->search(
      object  => "Infoblox::DHCP::IPv6NetworkTemplate" ,
      name    => "custom_network_template"
  comment => ".*",
 );

 # search all Network Templates with a given extensible attribute 'Site'
 my @retrieved_objs = $session->search(
   object => "Infoblox::DHCP::IPv6NetworkTemplate",
   extensible_attributes => { 'Site' => 'Santa Clara' });
```

Infoblox::Session->modify( )

Use this method to modify an IPv6 network template in the Infoblox appliance. See Infoblox::Session->modify() for parameters and return values.

Example

```
#Modify enable_ddns method
$object->enable_ddns("true");
# Submit modification
my $response = $session->modify( $object );
```

Infoblox::Session->remove( )

Use this method to remove an object from the Infoblox appliance. See Infoblox::Session->remove() for parameters and return values.

To remove a specific object, first use `get()` or `search()` to retrieve it, and then submit it for removal.

Example

```
# Get the object under the same name
my  @result_array = $session->get(

      object => "Infoblox::DHCP::IPv6NetworkTemplate" ,
      name   => "custom_network_template"
            );

# Find the desired object in the retrieved list
my $desired_network_template = $result_array[0];

# Submit for removal
my $response = $session->remove( $desired_network_template );
```

METHODS

This section describes all the methods that can be used to configure and retrieve the attribute values of a DHCP IPv6 network template object.

allow_any_netmask( )

Use this method to set or retrieve the allow_any_netmask parameter of a DHCP IPv6 network template object. If you set this parameter to "true", then the template allows any netmask. A user must specify a netmask when creating a network using this template. If you set this parameter to false, you must specify the "netmask" property for the network template object.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

Specify "true" to set the allow_any_netmask parameter or specify "false" if you want to specify a particular netmask. The default value is false.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
# Get allow_any_netmask
my $allow_any_netmask = $ipv6network_template->allow_any_netmask()
# Modify allow_any_netmask
$ipv6network_template->allow_any_netmask("true");
```

auto_create_reversezone( )

Use this method to set or retrieve the auto_create_reversezone flag of a DHCP IPv6 network template object. This flag indicates whether the appliance must automatically create the corresponding reverse zone.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

Specify "true" to create a reverse zone automatically or "false" to disable the automatic creation of a reverse zone.The default value for this field is false.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
# Get auto_create_reversezone
my $auto_create_reversezone = $ipv6network_template->auto_create_reversezone();
# Modify auto_create_reversezone
$ipv6network_template->auto_create_reversezone("true");
```

cidr( )

Use this method to set or retrieve the CIDR for the IPv6 network template object.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

Specify the CIDR as a number from 1 to 128.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get CIDR
my $cidr = $ipv6network_template->cidr();
#Modify CIDR
$ipv6network_template->cidr("68");
```

comment( )

Use this method to set or retrieve a descriptive comment about a DHCP IPv6 network template.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

Enter a descriptive comment for the network template in string format and with a maximum of 256 bytes.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
# Get comment
my $comment = $ipv6network_template->comment();
# Modify comment
$ipv6network_template->comment("add a network");
```

ddns_domainname( )

Use this method to set or retrieve the ddns_domainname value of a DHCP IPv6 Network Tempate object.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Setting this method to a defined value implicitly sets the override_ddns_domainname method to "true". Setting the parameter to undefined causes the appliance to use the grid default and automatically resets the override_ddns_domainname attribute to "false".

Note that when ddns_domainname is set to a defined value and override_ddns_domainname is set to "false", the last operation takes precedence. Thus the sequence $object->ddns_domainname("testdomain"); $object->override_ddns_domainname("false"); will set override_ddns_domainname to "false", and the sequence $object->override_ddns_domainname("false"); $object->ddns_domainname("testdomain"); will result in override_ddns_domainname="true".

Parameter

The dynamic DNS domain name in string format.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get ddns_domainname
my $ddns_domainname = $ipv6network_template->ddns_domainname();
#Modify ddns_domainname
$ipv6network_template->ddns_domainname("test_domain.com");
```

ddns_enable_option_fqdn( )

Use this method to set or retrieve the ddns_enable_option_fqdn flag of a DHCP IPv6 network template object. This method controls whether the FQDN option sent by the client is to be used, or if the server can automatically generate the FQDN. This setting overrides the upper-level settings.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Setting this method to a defined value implicitly sets the override_ddns_enable_option_fqdn method to "true". Setting the parameter to undefined causes the appliance to use the upper-level setting and automatically resets the override_ddns_enable_option_fqdn attribute to "false".

Note that when ddns_enable_option_fqdn is set to a defined value and override_ddns_enable_option_fqdn is set to "false", the last operation takes precedence. Thus the sequence $object->ddns_enable_option_fqdn("true"); $object->override_ddns_enable_option_fqdn("false"); will set override_ddns_enable_option_fqdn to "false", and the sequence $object->override_ddns_enable_option_fqdn("false"); $object->ddns_enable_option_fqdn("true"); will result in override_ddns_enable_option_fqdn="true".

Parameter

Specify "true" to generate the hostname or "false" to deactivate/unset it. The default value is undefined, which indicates that this attribute inherits the upper-level setting.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
# Get ddns_enable_option_fqdn
my $ddns_enable_option_fqdn = $ipv6network_template->ddns_enable_option_fqdn()
# Modify ddns_enable_option_fqdn
$ipv6network_template->ddns_enable_option_fqdn("true");
#Remove override
$network_template->ddns_enable_option_fqdn(undef);
```

ddns_generate_hostname( )

Use this method to set or retrieve the ddns_generate_hostname flag of a DHCP IPv6 network template object. Set this flag to allow only the DHCP server to generate a hostname. This setting overrides the upper-level settings.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Setting this method to a defined value implicitly sets the override_ddns_generate_hostname method to "true". Setting the parameter to undefined causes the appliance to use the upper-level setting and automatically resets the override_ddns_generate_hostname attribute to "false".

Note that when ddns_generate_hostname is set to a defined value and override_ddns_generate_hostname is set to "false", the last operation takes precedence. Thus the sequence $object->ddns_generate_hostname("true"); $object->override_ddns_generate_hostname("false"); will set override_ddns_generate_hostname to "false", and the sequence $object->override_ddns_generate_hostname("false"); $object->ddns_generate_hostname("true"); will result in override_ddns_generate_hostname="true".

Parameter

Specify "true" to generate the hostname or "false" to deactivate/unset it. The default value is undefined, which indicates that this attribute inherits the upper-level setting.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
# Get ddns_generate_hostname
```

```
my $ddns_generate_hostname = $ipv6network_template->ddns_generate_hostname()
# Modify ddns_generate_hostname
$ipv6network_template->ddns_generate_hostname("true");
#Remove override
$network_template->ddns_generate_hostname(undef);
```

ddns_server_always_updates( )

Use this method to set or retrieve the ddns_server_always_updates flag of a DHCP IPv6 network template object. Set this flag to allow the DHCP server only to update DNS, regardless of the requests from DHCP clients.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

Specify "true" to set this flag and allow only the DHCP server to update DNS or "false" to deactivate/unset the flag. The default value is true.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
# Get ddns_server_always_updates
my $ddns_server_always_updates = $ipv6network_template->ddns_server_always_updates()
# Modify ddns_server_always_updates
$ipv6network_template->ddns_server_always_updates("true");
```

ddns_ttl( )

Use this method to set or retrieve the DNS update Time to Live (TTL) value of a DHCP IPv6 network template object. The TTL value indicates the number of seconds the Infoblox appliance caches a DDNS update.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Setting this method to a defined value implicitly sets the override_ddns_ttl method to "true". Setting the parameter to undefined causes the appliance to use the grid default and automatically resets the override_ddns_ttl attribute to "false".

Note that when ddns_ttl is set to a defined value and override_dddns_ttl is set to "false", the last operation takes precedence. Thus the sequence $object->ddns_ttl(12); $object->override_ddns_ttl("false"); will set override_ddns_ttl to "false", and the sequence $object->override_ddns_ttl("false"); $object->ddns_ttl(12); will result in override_ddns_ttl="true".

Parameter

A 32-bit integer, from 0 to 4294967295, that represents the duration, in seconds, that the update is cached. Zero indicates that the update will not be cached. The default value is 0.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get DNS Update TTL
my $dns_ttl = $ipv6network_template->ddns_ttl();
#Modify DNS Update TTL
$ipv6network_template->ddns_ttl(1200);
```

domain_name( )

Use this method to set or retrieve the domain_name value of a DHCP IPv6 network template object.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Setting this method to a defined value implicitly sets the override_domain_name method to "true". Setting the parameter to undefined causes the appliance to use the grid default and automatically resets the override_domain_name attribute to "false".

Note that when domain_name is set to a defined value and override_domain_name is set to "false", the last operation takes precedence. Thus the sequence $object->domain_name("testdomain"); $object->override_domain_name("false"); will set override_domain_name to "false", and the sequence $object->override_domain_name("false"); $object->domain_name("testdomain"); will result in override_domain_name="true".

Parameter

The client's domain name in string format.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get domain_name
my $domain_name = $ipv6network_template->domain_name();
#Modify domain_name
$ipv6network_template->domain_name("test_domain.com");
```

domain_name_servers( )

Use this method to set or retrieve the domain_name_servers value of a DHCP IPv6 network template object.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Setting this method to a defined value implicitly sets the override_domain_name_servers method to "true". Setting the parameter to undefined causes the appliance to use the grid default and automatically resets the override_domain_name_servers attribute to "false".

Note that when domain_name_servers is set to a defined value and override_domain_name_servers is set to "false", the last operation takes precedence. Thus the sequence $object->domain_name_servers("2001::1,2001::2"); $object->override_domain_name_servers("false"); will set override_domain_name_servers to "false", and the sequence $object->override_domain_name_servers("false"); $object->domain_name_servers("2001::1,2001::2"); will result in override_domain_name_servers="true".

Parameter

The domain name server addresses in IPv6 address format.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get domain_name_servers
my $domain_name_servers = $ipv6network_template->domain_name_servers();
#Modify domain_name_servers
$ipv6network_template->domain_name_servers("2001::1,2001::2");
```

enable_ddns( )

Use this method to set or retrieve the dynamic DNS updates flag of a DHCP IPv6 network template object. The DHCP server can send DDNS updates to DNS servers in the same grid and to external DNS servers. This setting overrides the upper-level settings.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Setting this method to a defined value implicitly sets the override_enable_ddns method to "true". Setting the parameter to undefined causes the appliance to use the upper-level default and automatically resets the override_enable_ddns attribute to "false".

Note that when enable_ddns is set to a defined value and override_enable_ddns is set to "false", the last operation takes precedence. Thus the sequence $object->enable_ddns("true"); $object->override_enable_ddns("false"); will set override_enable_ddns to "false", and the sequence $object->override_enable_ddns("false"); $object->enable_ddns("true"); will result in override_enable_ddns="true".

Parameter

Specify "true" to set the dynamic DNS updates flag or "false" to deactivate/unset it. The default value is undefined, which indicates that this attribute inherits the upper-level setting.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
# Get enable_ddns
my $enable_ddns = $ipv6network_template->enable_ddns()
# Modify enable_ddns
$ipv6network_template->enable_ddns("true");
# Remove the enable_ddns override
```

```
$ipv6network_template->enable_ddns(undef);
```

extattrs( )

Use this method to set or retrieve the extensible attributes associated with an IPv6 network template object.

Parameter

Valid value is a hash reference containing the names of extensible attributes and their associated values (Infoblox::Grid::Extattr objects).

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get extattrs
my $ref_extattrs = $ipv6network_template->extattrs();
#Modify extattrs
$ipv6network_template->extattrs({ 'Site' => $extattr1, 'Administrator' => $extattr2 });
```

extensible_attributes( )

Use this method to set or retrieve the extensible attributes associated with an IPv6 network template object.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

For valid values for extensible attributes, see Infoblox::Grid::ExtensibleAttributeDef/Extensible Attribute Values.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get extensible attributes
my $ref_extensible_attributes = $ipv6network_template->extensible_attributes();
#Modify extensible attributes
$ipv6network_template->extensible_attributes({ 'Site' => 'Santa Clara', 'Administrator' => [
'Peter', 'Tom' ] });
```

fixed_address_templates( )

Use this method to set or retrieve the IPv6 fixed address template that is assigned to this IPv6 network template. When you create a network based on a network template that includes a fixed address template, the fixed addresses are created based on the associated fixed address template.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

The valid value is an array reference of Infoblox::DHCP::Template objects.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
# Get fixed_address_templates
my $fixed_address_templates = $ipv6network_template->fixed_address_templates();
# Modify fixed_address_templates
my $tmpl = Infoblox::DHCP::Template->new(
   name => "custom_fixed_address_template"
);
$network_template->fixed_address_templates([$tmpl]);
```

ipv6prefix( )

Use this method to set or retrieve the IPv6 prefix of a DHCP IPv6 network template object. The prefix must belong to the list of global prefixes defined in the Infoblox::Grid::DHCP object.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

IPv6 prefix from the list of global IPv6 prefixes.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails. ls.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get ipv6prefix
my $ipv6prefix = $ipv6network_template->ipv6prefix();
#Modify ipv6prefix
$griddhcp->ipv6prefix("2001::");
```

members( )

Use this method to set or retrieve the member assigned to the IPv6 network template object.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

The valid value is an array reference that contains Infoblox::DHCP::Member objects.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
my $member1 = Infoblox::DHCP::Member->new(
    name     => "infoblox.localdomain",
    ipv4addr => "192.168.1.2",
);

#Create a network template with this member
my $ipv6network_template = Infoblox::DHCP::IPv6NetworkTemplate->new(
    name    => "custom_network_template",
    members => [$member1],
);
```

name( )

Use this method to set or retrieve the name of the DHCP IPv6 network template.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

The network template name in string format and with a maximum of 64 bytes.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get name
my $name = $ipv6network_template->name();
#Modify name
$ipv6network_template->name("custom_network_template");
```

options( )

Use this method to set or retrieve the options value of an IPv6 network template object. DHCP options provide network configuration

settings and various services available on the network for DHCP. This setting overrides the upper-level setting. The default value is undefined which indicates that this attribute inherit the upper-level setting.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Setting this method to a defined value implicitly sets the override_options method to "true". Setting the parameter to undefined causes the appliance to use the upper-level default and automatically resets the override_options attribute to "false".

Note that when the options method is set to a defined value and override_options is set to "false", the last operation takes precedence. Thus the sequence $object->options([$opt]); $object->override_options("false"); will set override_options to "false", and the sequence $object->override_options("false"); $object->options([$opt]); will result in override_options="true".

Parameter

The valid value is an array reference that contains Infoblox::DHCP::Option objects.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get options
my $options = $ipv6network_template->options();
#Modify options
my $option1 = Infoblox::DHCP::Option->new(
    name   => "dhcp6.rapid-commit",
    value  => "true"
);
$ipv6network_template->options([$option1]);
```

override_ddns_domainname( )

The override_ddns_domainname attribute controls whether the ddns_domainname value in the DHCP IPv6 Network Template is used, instead of the grid default.

The override_ddns_domainname attribute can be specified explicitly. It is also set implicitly when ddns_domainname is set to a defined value.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

Set the parameter to "true" to override the grid-level setting for ddns_domainname. Set the parameter to "false" to inherit the grid-level setting for ddns_domainname.

The default value of this parameter is "false".

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Getting override_ddns_domainname
my $override_ddns_domainname=$ipv6network_template->override_ddns_domainname( );
#Modifying override_ddns_domainname
$ipv6network_template->override_ddns_domainname("true");
```

override_ddns_enable_option_fqdn( )

The override_ddns_enable_option_fqdn attribute controls whether the ddns_enable_option_fqdn value in the DHCP IPv6 network template is used, instead of the grid default.

The override_ddns_enable_option_fqdn attribute can be specified explicitly. It is also set implicitly when ddns_domainname is set to a defined value.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

Set the parameter to "true" to override the grid-level setting for ddns_enable_option_fqdn. Set the parameter to "false" to inherit the grid-level setting for ddns_enable_option_fqdn.

The default value of this parameter is "false".

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get override_ddns_enable_option_fqdn
my $override_ddns_enable_option_fqdn=$ipv6network_template->override_ddns_enable_option_fqdn( );
#Modify override_ddns_enable_option_fqdn
$ipv6network_template->override_ddns_enable_option_fqdn("true");
```

override_ddns_generate_hostname( )

The override_ddns_generate_hostname attribute controls whether the ddns_generate_hostname value in the DHCP IPv6 network template is used, instead of the grid default.

The override_ddns_generate_hostname attribute can be specified explicitly. It is also set implicitly when ddns_domainname is set to a defined value.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

Set the parameter to "true" to override the grid-level setting for ddns_generate_hostname. Set the parameter to "false" to inherit the grid-level setting for ddns_generate_hostname.

The default value of this parameter is "false".

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get override_ddns_generate_hostname
my $override_ddns_generate_hostname=$ipv6network_template->override_ddns_generate_hostname( );
#Modify override_ddns_generate_hostname
$ipv6network_template->override_ddns_generate_hostname("true");
```

override_ddns_ttl( )

The override_ddns_ttl attribute controls whether the ddns_ttl value in the DHCP IPv6 network template object is used, instead of the grid default.

The override_ddns_ttl attribute can be specified explicitly. It is also set implicitly when ddns_ttl is set to a defined value.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

Set the parameter to "true" to override the grid-level setting for ddns_ttl. Set the parameter to "false" to inherit the grid-level setting for ddns_ttl.

The default value of this parameter is "false".

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get override_ddns_ttl
my $override_ddns_ttl=$ipv6network_template->override_ddns_ttl( );
#Modify override_ddns_ttl
$ipv6network_template->override_ddns_ttl("true");
```

override_domain_name( )

The override_domain_name attribute controls whether the domain_name value in the DHCP IPv6 network template is used, instead of the grid default.

The override_domain_name attribute can be specified explicitly. It is also set implicitly when domain_name is set to a defined value.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

Set the parameter to "true" to override the grid-level setting for domain_name. Set the parameter to "false" to inherit the grid-level setting for domain_name.

The default value of this parameter is "false".

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Getting override_domain_name
my $override_domain_name=$ipv6network_template->override_domain_name( );
#Modifying override_domain_name
$ipv6network_template->override_domain_name("true");
```

override_domain_name_servers( )

The override_domain_name_servers attribute controls whether the domain_name_servers value in the DHCP IPv6 network template is used, instead of the grid default.

The override_domain_name_servers attribute can be specified explicitly. It is also set implicitly when domain_name_servers is set to a defined value.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

Set the parameter to "true" to override the grid-level setting for domain_name_servers. Set the parameter to "false" to inherit the grid-level setting for domain_name_servers.

The default value of this parameter is "false".

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get override_domain_name_servers
my $override_domain_name_servers=$ipv6network_template->override_domain_name_servers( );
#Modify override_domain_name_servers
$ipv6network_template->override_domain_name_servers("true");
```

override_enable_ddns( )

The override_enable_ddns attribute controls whether the enable_ddns value in the DHCP IPv6 network template is used, instead of the grid default.

The override_enable_ddns attribute can be specified explicitly. It is also set implicitly when enable_ddns is set to a defined value.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

Set the parameter to "true" to override the grid-level setting for enable_ddns. Set the parameter to "false" to inherit the grid-level setting for enable_ddns.

The default value of this parameter is "false".

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get override_enable_ddns
```

```
my $override_enable_ddns=$ipv6network_template->override_enable_ddns( );
#Modify override_enable_ddns
$ipv6network_template->override_enable_ddns("true");
```

override_options( )

The override_options attribute controls whether the options in the DHCP IPv6 network template are used, instead of the grid default.

The override_options attribute can be specified explicitly. It is also set implicitly when options is set to a defined value.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

Set the parameter to "true" to override the grid-level setting for options. Set the parameter to "false" to inherit the grid-level setting for options.

The default value of this parameter is "false".

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get override_options
my $override_options=$ipv6network_template->override_options( );
#Modify override_options
$ipv6network_template->override_options("true");
```

override_preferred_lifetime( )

The override_preferred_lifetime attribute controls whether the preferred_lifetime value in the DHCP IPv6 network template is used, instead of the grid default.

The override_preferred_lifetime attribute can be specified explicitly. It is also set implicitly when preferred_lifetime is set to a defined value.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

Set the parameter to "true" to override the grid-level setting for options. Set the parameter to "false" to inherit the grid-level setting for options.

The default value of this parameter is "false".

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get override_preferred_lifetime
my $override_preferred_lifetime=$ipv6network_template->override_preferred_lifetime( );
#Modify override_preferred_lifetime
$ipv6network_template->override_preferred_lifetime("true");
```

override_recycle_leases( )

The override_recycle_leases attribute controls whether the recycle_leases value in the DHCP IPv6 network template is used, instead of the grid default.

The override_recycle_leases attribute can be specified explicitly. It is also set implicitly when recycle_leases is set to a defined value.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

Set the parameter to "true" to override the grid-level setting for options. Set the parameter to "false" to inherit the grid-level setting for options.

The default value of this parameter is "false".

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get override_recycle_leases
my $override_recycle_leases=$ipv6network_template->override_recycle_leases( );
#Modify override_recycle_leases
$ipv6network_template->override_recycle_leases("true");
```

override_update_dns_on_lease_renewal( )

The override_update_dns_on_lease_renewal attribute controls whether the update_dns_on_lease_renewal value in the DHCP IPv6 network template is used, instead of the grid default.

The override_update_dns_on_lease_renewal attribute can be specified explicitly. It is also set implicitly when update_dns_on_lease_renewal is set to a defined value.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

Set the parameter to "true" to override the grid-level setting for update_dns_on_lease_renewal. Set the parameter to "false" to inherit the grid-level setting for update_dns_on_lease_renewal.

The default value of this parameter is "false".

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get override_update_dns_on_lease_renewal
my $override_update_dns_on_lease_renewal=$ipv6network_template-
>override_update_dns_on_lease_renewal( );
#Modify override_update_dns_on_lease_renewal
$ipv6network_template->override_update_dns_on_lease_renewal("true");
```

override_valid_lifetime( )

The override_valid_lifetime attribute controls whether the valid_lifetime value in the DHCP IPv6 network template object is used, instead of the upper-level default.

The override_valid_lifetime attribute can be specified explicitly. It is also set implicitly when valid_lifetime is set to a defined value.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

Set the parameter to "true" to override the grid-level setting for valid_lifetime. Set the parameter to "false" to inherit the upper-level setting for valid_lifetime.

The default value of this parameter is "false".

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get override_valid_lifetime
my $override_valid_lifetime=$ipv6network_template->override_valid_lifetime( );
#Modify override_valid_lifetime
$ipv6network_template->override_valid_lifetime("true");
```

preferred_lifetime( )

Use this method to set or retrieve the valid lifetime of the DHCP IPv6 network template object.

Setting this method to a defined value implicitly sets the override_preferred_lifetime method to "true". Setting the parameter to undefined

causes the appliance to use the upper-level default and automatically resets the override_preferred_lifetime attribute to "false".

Note that when preferred_lifetime is set to a defined value and override_preferred_lifetime is set to "false", the last operation takes precedence. Thus the sequence $object->preferred_lifetime(20000); $object->override_preferred_lifetime("false"); will set override_preferred_lifetime to "false", and the sequence $object->override_preferred_lifetime("false"); $object->preferred_lifetime(20000); will result in override_preferred_lifetime="true".

Parameter

Preferred lifetime value in seconds. The default value is 27000. Note that the preferred lifetime cannot be greater than the valid lifetime.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get preferred_lifetime
my $preferred_lifetime = $ipv6network_template->preferred_lifetime();
#Modify preferred_lifetime
$ipv6network_template->preferred_lifetime(12345);
```

range_templates( )

Use this method to set or retrieve the IPv6 address range template assigned to this IPv6 network template object. When you create a network based on a network template object with a range template, the IPv6 address ranges are created based on the associated IPv6 address range template.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

The valid value is an array reference of Infoblox::DHCP::Template objects.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get range_templates
my $range_templates = $ipv6network_template->range_templates();
#Modify range_templates
my $tmpl = Infoblox::DHCP::Template->new(
   name => "custom_range_template"
);
$ipv6network_template->range_templates([ $tmpl ]);
```

recycle_leases( )

Use this method to set or retrieve the recycle_leases flag for an IPv6 network template object.

If the flag is enabled, the leases are kept in the recycle bin until one week after expiration. When the flag is disabled, the leases are irrecoverably deleted.

Setting this method to a defined value implicitly sets the override_recycle_leases method to "true". Setting the parameter to undefined causes the appliance to use the grid default and automatically resets the override_recycle_leases attribute to "false".

Note that when recycle_leases is set to a defined value and override_recycle_leases is set to "false", the last operation takes precedence. Thus the sequence $object->recycle_leases("true"); $object->override_recycle_leases("false"); will set override_recycle_leases to "false", and the sequence $objct->override_recycle_leases("false"); $object->recycle_leases("true"); will result in override_recycle_leases="true".

Parameter

Specify "true" to enable the recycle_leases flag or "false" to disable it. The default value is undefined.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get recycle_leases
my $recycle_leases = $ipv6network_template->recycle_leases();
#Modify recycle_leases
$ipv6network_template->recycle_leases("false");
```

rir( )

Use this method to retrieve the name of the Regional Internet Registry (RIR) that allocated this address space. This is a read-only attribute.

Parameter

None

Returns

The method returns the attribute value that can be 'RIPE' or 'NONE'.

Example

```
#Get rir
my $rir = $ipv6network_template->rir();
```

rir_organization( )

Use this method to set or retrieve the Regional Internet Registry (RIR) organization object.

Parameter

An Infoblox::Grid::RIR::Organization object.

You must enable support for RIR/SWIP at the Grid level before you can set attribute values. See Infoblox::Grid->enable_rir_swip() method.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get rir_organization
my $rir_organization = $ipv6network_template->rir_organization();
#Modify rir_organization
$ipv6network_template->rir_organization($rir_org);
```

rir_registration_action( )

Use this method to set or retrieve the registration action of Regional Internet Registry (RIR).

Parameter

Valid values are 'NONE', 'CREATE'.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get rir_registration_action
my $rir_registration_action = $ipv6network_template->rir_registration_action();
#Modify rir_registration_action
$ipv6network_template->rir_registration_action('CREATE');
```

rir_registration_status( )

Use this method to set or retrieve the registration status of Regional Internet Registry (RIR).

Parameter

The registration status of Regional Internet Registry (RIR), which can be 'REGISTERED' or 'NOT_REGISTERED'.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get rir_registration_status
my $rir_registration_status = $ipv6network_template->rir_registration_status();
#Modify rir_registration_status
$ipv6network_template->rir_registration_status('NOT_REGISTERED');
```

send_rir_request( )

Use this method to set or retrieve the flag that indicates whether the appliance should send an API or email request for Regional Internet Registry (RIR).

Parameter

Specify "true" to set the send_rir_request flag or "false" to unset it.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get send_rir_request
my $send_rir_request = $ipv6network_template->send_rir_request();
#Modify send_rir_request
$ipv6network_template->send_rir_request('true');
```

update_dns_on_lease_renewal( )

The update_dns_on_lease_renewal attribute controls whether the DHCP server updates DNS when a DHCP lease is renewed.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Setting this method to a defined value implicitly sets the override_update_dns_on_lease_renewal method to "true". Setting the parameter to undefined causes the appliance to use the grid default and automatically resets the override_update_dns_on_lease_renewal attribute to "false".

Note that when update_dns_on_lease_renewal is set to a defined value and override_update_dns_on_lease_renewal is set to "false", the last operation takes precedence. Thus the sequence $object->update_dns_on_lease_renewal("true"); $object->override_update_dns_on_lease_renewal("false"); will set override_update_dns_on_lease_renewal to "false", and the sequence $object->override_update_dns_on_lease_renewal("false"); $object->update_dns_on_lease_renewal("true"); will result in override_update_dns_on_lease_renewal="true".

Parameter

Specify "true" to set the update_dns_on_lease_renewal flag or "false" to deactivate/unset it.

The default value of this parameter is "false".

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get update_dns_on_lease_renewal
my $update_dns_on_lease_renewal=$ipv6network_template->update_dns_on_lease_renewal( );
#Modify update_dns_on_lease_renewal
$ipv6network_template->update_dns_on_lease_renewal("true");
```

valid_lifetime( )

Use this method to set or retrieve the valid lifetime of the DHCP IPv6 network template object.

Setting this method to a defined value implicitly sets the override_valid_lifetime method to "true". Setting the parameter to undefined causes the appliance to use the upper-level default and automatically resets the override_valid_lifetime attribute to "false".

Note that when valid_lifetime is set to a defined value and override_valid_lifetime is set to "false", the last operation takes precedence. Thus the sequence $object->valid_lifetime(20000); $object->override_valid_lifetime("false"); will set override_valid_lifetime to "false", and the sequence $object->override_valid_lifetime("false"); $object->valid_lifetime(20000); will result in override_valid_lifetime="true".

Parameter

Valid lifetime value in seconds. The default value is 43200.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get valid_lifetime
my $valid_lifetime = $ipv6network_template->valid_lifetime();
#Modify valid_lifetime
$ipv6network_template->valid_lifetime(12345);
```

SAMPLE CODE

The following sample code demonstrates the different functions that can be applied to an object such as add, search, modify, and remove. This sample also includes error handling for the operations.

```
#Preparation prior to a DHCP IPv6 network template object insertion
```

#PROGRAM STARTS: Include all the modules that will be used use strict; use Infoblox; **#Create a session to the Infoblox DNSone appliance** my $session = Infoblox::Session->new(master => "192.168.1.2", username => "admin", password => "infoblox");

```
unless ($session) {
    die("Construct session failed: ",
        Infoblox::status_code() . ":" . Infoblox::status_detail());
}
print "Session created successfully\n";

my $memberdhcp = Infoblox::DHCP::Member->new(
    name     => $host_name,
    ipv4addr => $node_ip
);
```

#Add IPv6 network template object

```
#Create a DHCP network template object

my $network_template = Infoblox::DHCP::IPv6NetworkTemplate->new(
    name                   => "custom_network_template",
    cidr                   => 64,
    comment                => "add IPv6 network template",
    members                => [ $memberdhcp ],
    enable_ddns            => "false",
    valid_lifetime         => 99999,
    auto_create_reversezone => "true",
);

my $response = $session->add($network_template)
    or die("Add Network template failed: ",
    $session->status_code() . ":" . $session->status_detail());

print "Network template added successfully\n";
```

#Get and modify an IPv6 network template attribute

```
#Get Network template through the session
my  @retrieved_objs = $session->get(
    object => "Infoblox::DHCP::IPv6NetworkTemplate",
    name   => "custom_network_template"
);

my $object = $retrieved_objs[0];

unless ($object) {
    die("Get Network template object failed: ",
        $session->status_code() . ":" . $session->status_detail());
}
print "Get Network template found at least 1 matching entry\n";
```

#Search an IPv6 network template attribute

```
#Search Network template through the session
```

```
my  @retrieved_objs = $session->search(
    object => "Infoblox::DHCP::IPv6NetworkTemplate",
    name   => "custom_network_template"
);

my $object = $retrieved_objs[0];

unless ($object) {
    die("Search Network template object failed: ",
        $session->status_code() . ":" . $session->status_detail());
}
print "Search Network template found at least 1 matching entry\n";

#Modify attributes of the specified IPv6 network template

$object->enable_ddns("true"); #Apply the changes

$session->modify($object)
    or die("Modify Network template attribute failed: ",
    $session->status_code() . ":" . $session->status_detail());

print "Network template object modified successfully \n";
```

#Remove an IPv6 network template object

```
#Get IPv6 Network Template object through the session

my @retrieved_objs = $session->get(
    object  => "Infoblox::DHCP::IPv6NetworkTemplate",
    name    => "custom_network_template"
);

my $object = $retrieved_objs[0];

unless ($object) {
    die("Get IPv6 Network Template object failed: ",
        $session->status_code() . ":" . $session->status_detail());
}

print "Get IPv6 Network Template object found at least 1 matching entry\n";

#Submit the object for removal

$session->remove($object)
    or die("Remove IPv6 Network Template failed: ",
    $session->status_code() . ":" . $session->status_detail());

print "IPv6 Network Template removed successfully \n";

####PROGRAM ENDS####
```

AUTHOR

Infoblox Inc. http://www.infoblox.com/

SEE ALSO

Infoblox::Session, Infoblox::DHCP::IPv6Network, Infoblox::Grid::DHCP, Infoblox::DHCP::Template, Infoblox::DHCP::Member, Infoblox::Session->get(), Infoblox::Session->search(), Infoblox::Session->add(), Infoblox::Session->remove(), Infoblox::Session->modify()

COPYRIGHT

Copyright (c) 2017 Infoblox Inc.

Infoblox::DHCP::IPv6FixedAddr - DHCP IPv6 Fixed Address object.

- NAME
- DESCRIPTION
- CONSTRUCTOR
- SESSION METHODS
  - Infoblox::Session->add( )
  - Infoblox::Session->get( )
  - Infoblox::Session->modify( )
  - Infoblox::Session->remove( )
  - Infoblox::Session->search( )
- METHODS
  - address_type( )
  - allow_telnet( )
  - comment( )
  - configure_for_dhcp( )
  - cli_credentials( )
  - cloud_info( )
  - device_description( )
  - device_location( )
  - device_type( )
  - device_vendor( )
  - disable( )
  - discovered_duid( )
  - discovered_name( )
  - discoverer( )
  - discover_now_status( )
  - domain_name( )
  - domain_name_servers( )
  - duid( )
  - extattrs( )
  - extensible_attributes( )
  - first_discovered( )
  - ipv6addr( )
  - ipv6prefix( )
  - ipv6prefix_bits( )
  - last_discovered( )
  - match_client( )
  - ms_ad_user_data( )
  - name( )
  - netbios( )
  - network( )
  - network_component_description( )
  - network_component_ip( )
  - network_component_name( )
  - network_component_port_description( )
  - network_component_port_name( )
  - network_component_port_number( )
  - network_component_type( )
  - network_view( )
  - options( )
  - os( )
  - override_domain_name( )
  - override_domain_name_servers( )
  - override_cli_credentials( )
  - override_options( )
  - override_preferred_lifetime( )
  - override_valid_lifetime( )
  - port_duplex( )
  - port_link_status( )
  - port_speed( )
  - port_status( )
  - port_vlan_description( )
  - port_vlan_name( )
  - port_vlan_number ( )
  - preferred_lifetime( )
  - template( )
  - valid_lifetime( )
- SAMPLE CODE
- AUTHOR
- SEE ALSO
- COPYRIGHT

NAME

Infoblox::DHCP::IPv6FixedAddr - DHCP IPv6 Fixed Address object.

DESCRIPTION

An IPv6 Fixed Address is a specific IP address that a DHCP server always assigns when a lease request comes from a particular MAC address of the client. This is an IPv6 representation of a fixed address.

CONSTRUCTOR

```
 my $ipv6fa = Infoblox::DHCP::IPv6FixedAddr -> new(
       duid                          => $string,                    # Required
       address_type                  => "ADDRESS" | "PREFIX" | "BOTH", # Optional / Default is "ADDRESS"
       comment                       => $string,                    # Optional / Default is empty
       configure_for_dhcp            => "true" | "false",           # Optional / Default is "false"
                                                                    # When both the IP address and DUID
are present, the default value is "true"
                                                                    # Applicable only when adding this
fixed address to a host
       disable                       => "true" | "false",           # Optional / Default is "false"
       domain_name                   => $string,                    # Optional / Default is undefined
       domain_name_servers           => "$ipv6addr1,$ipv6addr2,...", # Optional / Default is undefined
       extattrs                      => { $string => $extattr, ... },#Optional / Default is undefined
       extensible_attributes         => { $string => $string | $num, $string => [ $string | $num, ... ],
... } #Optional / Default is undefined
       ipv6addr                      => $ipv6addr,                  # Optional / one of ipv6addr or
ipv6prefix is required
       ipv6prefix                    => $ipv6addr,                  # Optional / one of ipv6addr or
ipv6prefix is required
       ipv6prefix_bits               => $ipv6addr,                  # Optional / Required if ipv6prefix
is specified
       match_client                  => $string,                    # Optional / Default is "DUID"
                                                                    # Possible values are:
                                                                    #   "DUID", "RESERVED",
       name                          => $string,                    # Optional / Default is undefined
       network                       => $ipv6network,               # Optional / Default is undefined
       network_view                  => $NetworkView,               # Optional / Default is "default"
network view
       options                       => [$option1, $option2, ...]   # Optional / Default is empty
       override_domain_name          => "true" | "false",           # Optional / Default value is
"false"
       override_domain_name_servers  => "true" | "false",           # Optional / Default value is
"false"
       override_options              => "true" | "false",           # Optional / Default value is
"false"
       override_preferred_lifetime   => "true" | "false",           # Optional / Default value is
"false"
       override_valid_lifetime       => "true" | "false",           # Optional / Default value is
"false"
       preferred_lifetime            => $num,                       # Optional / Default is "27000"
       template                      => $string,                    # Optional / Default is undefined
       valid_lifetime                => $num,                       # Optional / Default is 43200
       cli_credentials               => [$cli1, ...],               # Optional
       override_cli_credentials      => "true" | "false",           # Optional / Default is "false"
       reserved_interface            => $interface,                 # Optional
       device_type                   => $string,                    # Optional
       device_vendor                 => $string,                    # Optional
       device_location               => $string,                    # Optional
       device_description            => $string,                    # Optional
       allow_telnet                  => "true" | "false",           # Optional / Default is "false"
 );
```

You cannot set both extattrs and extensible_attributes attributes at the same time.

SESSION METHODS

This section describes all the methods in Infoblox::Session module that you can apply to a DHCP IPv6 Fixed Address object.

Infoblox::Session->add( )

Use this method to add a DHCP IPv6 Fixed Address object to the Infoblox appliance. See Infoblox::Session->add() for parameters and return values.

Example

```
#Construct a DHCP IPv6 Fixed Address object
my $ipv6fa = Infoblox::DHCP::IPv6FixedAddr->new(
       duid                    => "0001",
       comment                 => "add ipv6 fixed address using uid",
       disable                 => "false",
       match_client            => "DUID",
       network                 => "2001::/24",
       network_view            => $nview,
       options                 => [ $option1,  $option2, $option3,  $option4, $option5, $option6 ],
);

#Submit for addition
my $response = $session->add($ipv6fa)
```

Infoblox::Session->get( )

Use this method to retrieve all the matching objects from the Infoblox appliance. See Infoblox::Session->get() for parameters and return values.

'discover_now_status' will not be returned by default, use 'return_methods' to request it if needed.

Key References

```
Apply the following attributes to get a specific DHCP IPv6 Fixed Address object:

  comment                                 - Optional. User-specified comment for the fixed address.
  duid                                    - Optional. DUID of the fixed address.
  ipv6addr                                - Optional. IPv6 address of the fixed address.
  network                                 - Optional. Network address of the fixed address.
  network_view                            - Optional. Network view in which the fixed address is
located. By default, all network views are searched.
  extattrs                                - Optional. A hash reference containing extensible
attributes.
  extensible_attributes                   - Optional. A hash reference containing extensible
attributes.

  discovered_duid                         - Optional. Discovered DHCPv6 UID.
  discovered_name                         - Optional. Discovered name.
  discoverer                              - Optional. Discoverer name.
  first_discovered                        - Optional. Time of the first discovery.
  last_discovered                         - Optional. Time of the last discovery.
  netbios                                 - Optional. Discovered NetBIOS name.
  network_component_description           - Optional. Discovered network component description.
  network_component_ip                    - Optional. Discovered network component IP address.
  network_component_name                  - Optional. Discovered network component name.
  network_component_port_description      - Optional. Discovered network component port description.
  network_component_port_name             - Optional. Discovered network component port name.
  network_component_port_number           - Optional. Discovered network component port number.
  network_component_type                  - Optional. Discovered network component type.
  os                                      - Optional. Discovered OS.
  port_duplex                             - Optional. Discovered port duplex settings.
  port_link_status                        - Optional. Discovered port link status.
  port_speed                              - Optional. Discovered port speed settings.
  port_status                             - Optional. Discovered port status settings.
  port_vlan_description                   - Optional. Discovered port VLAN description.
  port_vlan_name                          - Optional. Discovered port VLAN name.
  port_vlan_number                        - Optional. Discovered port VLAN number.
```

Examples

```
#Get DHCP IPv6 Fixed Address object through the session
my @retrieved_objs = $session->get(
    object => "Infoblox::DHCP::IPv6FixedAddr",
    duid   => "0001",
);
my @retrieved_objs = $session->get(
    object       => "Infoblox::DHCP::IPv6FixedAddr",
    ipv6addr     => "2001::1",
    network_view => "default",
);

my @retrieved_objs = $session->get(
   object                => "Infoblox::DHCP::IPv6FixedAddr",
   extensible_attributes => { 'Site' => 'Santa Clara' }
);
```

Infoblox::Session->modify( )

Use this method to modify a DHCP IPv6 Fixed Address object in the Infoblox appliance. See Infoblox::Session->modify() for parameters and return values.

Example

```
#Use method to modify the preferred_lifetime
$ipv6fa->preferred_lifetime("1234567");
#Submit modification
my $response = $session->modify( $ipv6fa );
```

Infoblox::Session->remove( )

Use this method to remove a DHCP IPv6 Fixed Address object from the Infoblox appliance. See Infoblox::Session->remove() for parameters and return values.

To remove a specific object, first use `get()` or `search()` to retrieve the specific DHCP IPv6 Fixed Address object, and then submit this object for removal.

Example

```
#Get DHCP IPv6 Fixed Address object through the session
my @retrieved_objs = $session->get(
    object       => "Infoblox::DHCP::IPv6FixedAddr",
    duid         => "0001",
    network_view => "default"
);
#Find the desired object from the retrieved list.
my $desired_ipv6fa = $retrieved_objs[0];
#Submit for removal
my $response = $session->remove( $desired_ipv6fa );
```

Infoblox::Session->search( )

Use this method to search for DHCP IPv6 Fixed Address object and matching Host object with MAC address in the Infoblox appliance. See Infoblox::Session->search() for parameters and return values.

'discover_now_status' will not be returned by default, use 'return_methods' to request it if needed.

Key References

```
 Apply the following attributes to search for a specific DHCP IPv6 Fixed Address object:

  comment                                 - Optional. User-specified comment for the fixed address.
  duid                                    - Optional. DUID of the fixed address.
  ipv6addr                                - Optional. IPv6 address of the fixed address.
  network                                 - Optional. Network address of the fixed address.
  network_view                            - Optional. Network view in which the fixed address is
located. By default, all network views are searched.
  discovered_duid                         - Optional. Discovered DHCPv6 UID (exact search only).
  discovered_name                         - Optional. Discovered name(regex).
  discoverer                              - Optional. Discoverer name(regex).
  first_discovered                        - Optional. Time of the first discovery (in ISO format, exact
search only).
  last_discovered                         - Optional. Time of the last discovery (in ISO format, exact
search only).
  netbios                                 - Optional. Discovered NetBIOS name(regex).
  network_component_description           - Optional. Discovered network component description(regex).
  network_component_ip                    - Optional. Discovered network component IP address(regex).
  network_component_name                  - Optional. Discovered network component name(regex).
  network_component_port_description      - Optional. Discovered network component port
description(regex).
  network_component_port_name             - Optional. Discovered network component port name(regex).
  network_component_port_number           - Optional. Discovered network component port number (exact
search only).
  network_component_type                  - Optional. Discovered network component type(regex).
  os                                      - Optional. Discovered OS (regex).
  port_duplex                             - Optional. Discovered port duplex settings (exact search
only).
  port_link_status                        - Optional. Discovered port link status (exact search only).
  port_speed                              - Optional. Discovered port speed settings (exact search
only).
  port_status                             - Optional. Discovered port status settings (exact search
only).
  port_vlan_description                   - Optional. Discovered port VLAN description(regex).
  port_vlan_name                          - Optional. Discovered port VLAN name(regex).
  port_vlan_number                        - Optional. Discovered port VLAN number(exact search only).

  extattrs                                - Optional. A hash reference containing extensible
attributes.
  extensible_attributes                   - Optional. A hash reference containing extensible
attributes.
```

For more information about searching extensible attributes, see Infoblox::Grid::ExtensibleAttributeDef/Searching Extensible Attributes.

Examples

```
 # Search for all DHCP IPv6 Fixed Address objects that are specified in the network "2001::/24" in
the "default" network view.
```

```
my @retrieved_objs = $session->search(
    object       => "Infoblox::DHCP::IPv6FixedAddr",
    duid         => "0001",
    network      => "2001::/24",
    network_view => "default"
);

# Search for all DHCP IPv6 Fixed Address objects with "Santa Clara" for the "Site" extensible
attribute
my @retrieved_objs = $session->search(
    object                => "Infoblox::DHCP::IPv6FixedAddr",
    extensible_attributes => { 'Site' => 'Santa Clara' }
);
```

METHODS

This section describes all the methods that you can use to set and retrieve the attribute values of a DHCP IPv6 Fixed Address object.

address_type( )

Use this method to set or retrieve the address type of a DHCP IPv6 Fixed Address object.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

Valid values are "ADDRESS", "PREFIX", or "BOTH". When the address type is "ADDRESS", a value for the 'ipv6addr' member is required. When the address type is "PREFIX", values for 'ipv6prefix' and 'ipv6prefix_bits' are required. When the address type is "BOTH", values for 'ipv6addr', 'ipv6prefix', and 'ipv6prefix_bits' are all required.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get address_type
my $address_type = $ipv6fa->address_type();
#Modify address_type
$ipv6fa->address_type("PREFIX");
```

allow_telnet( )

Use this method to set or retrieve the flag that enables copying SSH credential to TELNET.

Parameter

Specify 'true' to copy SSH credential to TELNET or 'false' to disable it. The default value is 'false'.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
# Get attribute value
my $value = $ipv6fa->allow_telnet();
# Modify attribute value
$ipv6fa->allow_telnet('true');
```

comment( )

Use this method to set or retrieve the descriptive comment of a DHCP IPv6 Fixed Address object.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

Desired comment in string format with a maximum of 256 bytes.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get comment
my $comment = $ipv6fa->comment();
#Modify comment
$ipv6fa->comment("Modified DHCP IPv6 Fixed Address object comment");
```

configure_for_dhcp( )

Use this method to set or retrieve the configure_for_dhcp flag of a DHCP IPv6 Fixed Address object.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

Specify "true" to set the configure_for_dhcp flag or "false" to deactivate/unset it. The default value is "false".

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get configure_for_dhcp
my $configure_for_dhcp = $ipv6fa->configure_for_dhcp();
#Modify configure_for_dhcp
$ipv6fa->configure_for_dhcp("true");
```

cli_credentials( )

Use this method to set or retrieve the discovery CLI credentials.

Setting this method to a defined value implicitly sets the override_cli_credentials method to "true". Setting the parameter to undefined causes the appliance to use the grid default and automatically resets the override_cli_credentials attribute to "false".

Note that when cli_credentials is set to a defined value and override_cli_credentials is set to "false", the last operation takes precedence. Thus the sequence $ipv6fa->cli_credentials([$cli]); $ipv6fa->override_cli_credentials("false"); will set override_cli_credentials to "false", and the sequence $ipv6fa->override_cli_credentials("false"); $ipv6fa->cli_credentials([$cli]); will result in override_cli_credentials="true".

Parameter

Valid value is an array reference that contains Infoblox::Grid::Discovery::CLICredential objects.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get cli_credentials
my $cli_credentials = $ipv6fa->cli_credentials();
#Modify cli_credentials
$ipv6fa->cli_credentials([$cli1, $cli2]);
```

cloud_info( )

Use this method to retrieve cloud API related information for the Infoblox::DHCP::IPv6FixedAddr object.

Parameter

None

Returns

The method returns the attribute value.

Example

```
# Get cloud_info
my $cloud_info = $object->cloud_info();
```

device_description( )

Use this method to set or retrieve the description of the discovery device.

Parameter

The device description in string format.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get device_description
my $device_description = $ipv6fa->device_description();
#Modify device_description
$ipv6fa->device_description('example');
```

device_location( )

Use this method to set or retrieve the location of the discovery device.

Parameter

The device location in string format.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get device_location
my $device_location = $ipv6fa->device_location();
#Modify device_location
$ipv6fa->device_location('example');
```

device_type( )

Use this method to set or retrieve the type of the discovery device.

Parameter

The device type in string format.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get device_type
my $device_type = $ipv6fa->device_type();
#Modify device_type
$ipv6fa->device_type('example');
```

device_vendor( )

Use this method to set or retrieve the vendor name of the discovery device.

Parameter

The device vendor name in string format.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get device_vendor
```

```
my $device_vendor = $ipv6fa->device_vendor();
#Modify device_vendor
$ipv6fa->device_vendor('example');
```

disable( )

Use this method to set or retrieve the disable flag of a DHCP IPv6 Fixed Address object.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

Specify "true" to set the disable flag or "false" to deactivate/unset it. The default value is "false".

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get disable
my $disable = $ipv6fa->disable();
#Modify disable
$ipv6fa->disable("true");
```

=head2 discovered_data( )

Use this method to retrieve the discovered data of a DHCP IPv6 Fixed Address object. This is a read-only attribute.

Omit the parameter to retrieve the attribute value.

Parameter

None

Returns

The valid return value is an Infoblox::Grid::Discovery::Data object.

Example

```
#Get discovered_data value
my $discovered_data = $object->discovered_data();
```

discovered_duid( )

Use this method to retrieve the discovered DHCPv6 UID (DUID) of a DHCP IPv6 Fixed Address object.

Parameter

none

Returns

The method returns the discovered name.

Example

```
#Get discovered_duid
my $discovered_duid = $ipv6fa->discovered_duid();
```

discovered_name( )

Use this method to retrieve the discovered name of a DHCP IPv6 Fixed Address object.

Parameter

none

Returns

The method returns the discovered name.

Example

```
#Get discovered_name
```

```
my $discovered_name = $ipv6fa->discovered_name();
```

discoverer( )

Use this method to retrieve the discoverer of a DHCP IPv6 Fixed Address object.

Parameter

none

Returns

The method returns the discoverer value.

Example

```
#Get discoverer
my $discoverer = $ipv6fa->discoverer();
```

discover_now_status( )

Use this method to retrieve the current discovery status. This attribute is read-only.

'discover_now_status' will not be returned by default, use 'return_methods' to request it if needed.

Parameter

None.

Returns

The method returns the attribute value that can be 'COMPLETE', 'FAILED', 'NONE', 'PENDING' or 'RUNNING'.

Example

```
#Get discover_now_status
my $discover_now_status = $ipv6fa->discover_now_status();
```

domain_name( )

Use this method to set or retrieve the domain_name value of a DHCP IPv6 fixed address object.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Setting this method to a defined value implicitly sets the override_domain_name method to "true". Setting the parameter to undefined causes the appliance to use the upper-level default and automatically resets the override_domain_name attribute to "false".

Note that when domain_name is set to a defined value and override_domain_name is set to "false", the last operation takes precedence. Thus the sequence $object->domain_name("testdomain"); $object->override_domain_name("false"); will set override_domain_name to "false", and the sequence $object->override_domain_name("false"); $object->domain_name("testdomain"); will result in override_domain_name="true".

Parameter

The client's domain name in string format.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get domain_name
my $domain_name = $ipv6fa->domain_name();
#Modify domain_name
$ipv6fa->domain_name("test_domain.com");
```

domain_name_servers( )

Use this method to set or retrieve the domain_name_servers value of a DHCP IPv6 fixed address object.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Setting this method to a defined value implicitly sets the override_domain_name_servers method to "true". Setting the parameter to undefined causes the appliance to use the upper-level default and automatically resets the override_domain_name_servers attribute to "false".

Note that when domain_name_servers is set to a defined value and override_domain_name_servers is set to "false", the last operation takes precedence. Thus the sequence $object->domain_name_servers("2001::1,2001::2"); $object->override_domain_name_servers("false"); will set override_domain_name_servers to "false", and the sequence $object->override_domain_name_servers("false"); $object->domain_name_servers("2001::1,2001::2"); will result in override_domain_name_servers="true".

Parameter

The comma-separated list of domain name server addresses in IPv6 address format.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get domain_name_servers
my $domain_name_servers = $ipv6fa->domain_name_servers();
#Modify domain_name_servers
$ipv6fa->domain_name_servers("2001::1,2001::2");
```

duid( )

Use this method to set or retrieve the DHCPv6 Unique Identifier (DUID) of the IPv6 Fixed Address object.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

The DUID of the matching client in hexadecimal format.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get duid
my $duid = $ipv6fa->duid();
#Modify duid
$ipv6fa->duid("1c2edfaa");
```

extattrs( )

Use this method to set or retrieve the extensible attributes associated with an IPv6 Fixed Address object.

Parameter

Valid value is a hash reference containing the names of extensible attributes and their associated values (Infoblox::Grid::Extattr objects).

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get extattrs
my $ref_extattrs = $ipv6fa->extattrs();
#Modify extattrs
$ipv6fa->extattrs({ 'Site' => $extattr1, 'Administrator' => $extattr2 });
```

extensible_attributes( )

Use this method to set or retrieve the extensible attributes associated with an IPv6 Fixed Address object.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

For valid values for extensible attributes, see Infoblox::Grid::ExtensibleAttributeDef/Extensible Attribute Values.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get extensible attributes
my $ref_extensible_attributes = $ipv6fa->extensible_attributes();
#Modify extensible attributes
$ipv6fa->extensible_attributes({ 'Site' => 'Santa Clara', 'Administrator' => [ 'Peter', 'Tom' ] });
```

first_discovered( )

Use this method to retrieve the date and time that the DHCP IPv6 Fixed Address object was first discovered.

Parameter

none

Returns

The method returns the date and time that the DHCP IPv6 Fixed Address object was first discovered.

Example

```
#Get first_discovered
my $first_discovered = $ipv6fa->first_discovered();
```

ipv6addr( )

Use this method to set or retrieve the the IPv6 address of the IPv6 Fixed Address object.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

An IPv6 address is a 128-bit number represented as eight groups of four hexadecimal digits (i.e. two octets), each group separated by a `colon(:)` (example: 2001:0db8:85a3:0000:0000:8a2e:0370:7334). Leading zeroes and groups of consecutive zeros within the value can be omitted to produce a more compact representation (example: 2001::1, ::1), and dotted-quad notation can be used to express IPv4-mapped IPv6 addresses (example: ::ffff:192.0.2.128).

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get ipv6addr
my ipv6addr = $ipv6fa->ipv6addr();
#Modify ipv6addr
$ipv6fa->ipv6addr("aaaa:1111:bbbb:2222:cccc:3333:dddd:5555");
```

ipv6prefix( )

Use this method to set or retrieve the IPv6 prefix of a DHCP IPv6 Fixed Address object. The prefix must belong to the list of global prefixes defined in the Infoblox::Grid::DHCP object.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

IPv6 prefix from the list of global IPv6 prefixes.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.
ls.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get ipv6prefix
my $ipv6prefix = $ipv6fa->ipv6prefix();
#Modify ipv6prefix
```

```
$ipv6fa->ipv6prefix("2001::");
```

ipv6prefix_bits( )

Use this method to set or retrieve the prefix bits for the DHCP IPv6 Fixed Address.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

An unsigned integer between 2 and 128.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get ipv6prefix_bits
my $ipv6prefix_bits = $ipv6fa->ipv6prefix_bits();
#Modify ipv6prefix_bits
$ipv6fa->ipv6prefix_bits(32);
```

last_discovered( )

Use this method to retrieve the time this object was last seen by a discovery job.

The default value for this field is empty.

Parameter

N/A

Returns

The method returns the attribute value.

Example

```
#Get last_discovered
my $last_discovered = $ipv6fa->last_discovered();
```

match_client( )

Use this method to set or retrieve the match_client value of a DHCP Fixed Address object.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

```
match_client "DUID": The fixed IP address is leased to the matching DUID.
```

Returns

```
If you specified a parameter, the method returns true when the modification succeeds, and returns
false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.
```

Example

```
#Get match_client
my $match_client = $ipv6fa->match_client();
#Modify match_client
$ipv6fa->match_client("DUID");
```

ms_ad_user_data( )

Use this method to retrieve Microsoft Active Directory users related information. This is a read-only attribute.

Parameter

None

Returns

The valid return value is an Infoblox::Grid::MSServer::AdUser::Data object.

Example

```
#Get ms_ad_user_data
my $ms_ad_user_data = $ipv6fa->ms_ad_user_data();
```

name( )

Use this method to set or retrieve the name of a DHCP IPv6 Fixed Address object. The same name can be used for multiple fixed addresses.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

Specify a name in string format.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get the name
my $name = $ipv6fa->name();
#Modify the name
$ipv6fa->name("pop-server");
```

netbios( )

Use this method to retrieve the name in the NetBIOS reply that responded to a NetBIOS query.

Returns

The method returns the attribute value.

Example

```
#Get NetBIOS
my $netbios = $ipv6fa->netbios();
```

network( )

Use this method to retrieve the network value of a DHCP IPv6 Fixed Address object.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

Specify the address and netmask of the network in CIDR format (example: 2001::/64).

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get network
my $network = $ipv6addr->network();
#Modify network
$ipv6fa->network("2001::/24");
```

network_component_description( )

Use this method to retrieve the description of the network device that is connected to the DHCP IPv6 Fixed Address object.

Parameter

none

Returns

The method returns the network device description.

Example

```
#Get network_component_description
my $network_component_description = $ipv6fa->network_component_description();
```

network_component_ip( )

Use this method to retrieve the IP address of the network device that is connected to the DHCP IPv6 Fixed Address object.

Parameter

none

Returns

The method returns the IP address of the network device.

Example

```
#Get network_component_ip
my $network_component_ip = $ipv6fa->network_component_ip();
```

network_component_name( )

Use this method to retrieve the name of the network device that is connected to the DHCP IPv6 Fixed Address object.

Parameter

none

Returns

The method returns the network device name.

Example

```
#Get network_component_name
my $network_component_name = $ipv6fa->network_component_name();
```

network_component_port_description( )

Use this method to retrieve the description of the network device port that is connected to the DHCP IPv6 Fixed Address object.

Parameter

none

Returns

The method returns the description of the port on the network device.

Example

```
#Get network_component_port_description
my $network_component_port_description = $ipv6fa->network_component_port_description();
```

network_component_port_name( )

Use this method to retrieve the name of the network device port that is connected to the DHCP IPv6 Fixed Address object.

Parameter

none

Returns

The method returns the name of the port on the network device.

Example

```
#Get network_component_port_name
my $network_component_port_name = $ipv6fa->network_component_port_name();
```

network_component_port_number( )

Use this method to retrieve the number of the network device port that is connected to the DHCP IPv6 Fixed Address object.

Parameter

none

Returns

The method returns the network device port number.

Example

```
#Get network_component_port_number
my $network_component_port_number = $ipv6fa->network_component_port_number();
```

network_component_type( )

Use this method to retrieve the type of the network device that is connected to the DHCP IPv6 Fixed Address object.

Parameter

none

Returns

The method returns the network device type.

Example

```
#Get network_component_type
my $network_component_type = $ipv6fa->network_component_type();
```

network_view( )

Use this method to set or retrieve the network view of the DHCP fixed address.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

The default value is the "default" network view, which means the fixed address is in the default network view.

Parameter

Valid value is a defined Infoblox::DHCP::View object. The default network view value is the system-defined default network view object.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get network view
my $nview = $ipv6fa->network_view();
#Modify network view, with an Infoblox::DHCP::View object
$ipv6fa->network_view($nview);
```

options( )

Use this method to set or retrieve the options of a DHCP IPv6 Fixed Address object.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Setting this method to a defined value implicitly sets the override_options method to "true". Setting the parameter to undefined causes the appliance to use the grid default and automatically resets the override_options attribute to "false".

Note that when the options method is set to a defined value and override_options is set to "false", the last operation takes precedence. Thus the sequence $object->options([$opt]); $object->override_options("false"); will set override_options to "false", and the sequence $object->override_options("false"); $object->options([$opt]); will result in override_options="true".

Parameter

The valid value is an array reference that contains Infoblox::DHCP::Option object(s).

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get options
my $options = $ipv6fa->options();
#Modify options
my $option1 = Infoblox::DHCP::Option->new(
    name   => "dhcp6.rapid-commit",
    value  => "true"
);
$ipv6fa->options([$option1);
```

os( )

Use this method to retrieve the operating system associated with this fixed address.

Returns

The method returns the attribute value.

Example

```
#Get OS
my $os = $ipv6fa->os();
```

override_domain_name( )

The override_domain_name attribute controls whether the domain_name value in the DHCP IPv6 fixed address is used, instead of the upper-level default.

The override_domain_name attribute can be specified explicitly. It is also set implicitly when domain_name is set to a defined value.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

Set the parameter to "true" to override the upper-level setting for domain_name. Set the parameter to "false" to inherit the upper-level setting for domain_name.

The default value of this parameter is "false".

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get override_domain_name
my $override_domain_name=$ipv6fa->override_domain_name( );
#Modify override_domain_name
$ipv6fa->override_domain_name("true");
```

override_domain_name_servers( )

The override_domain_name_servers attribute controls whether the domain_name_servers value in the DHCP IPv6 fixed address is used, instead of the upper-level default.

The override_domain_name_servers attribute can be specified explicitly. It is also set implicitly when domain_name_servers is set to a defined value.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

Set the parameter to "true" to override the upper-level setting for domain_name_servers. Set the parameter to "false" to inherit the upper-level setting for domain_name_servers.

The default value of this parameter is "false".

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get override_domain_name_servers
my $override_domain_name_servers=$ipv6fa->override_domain_name_servers( );
```

```
#Modify override_domain_name_servers
$ipv6fa->override_domain_name_servers("true");
```

override_cli_credentials( )

The override_cli_credentials attribute controls whether the cli_credentials value in the object is used, instead of the grid default.

The override_cli_credentials attribute can be specified explicitly. It is also set implicitly when cli_credentials is set to a defined value.

Parameter

Specify "true" to set the override_cli_credentials flag or "false" to deactivate/unset it. The default value is "false".

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
# Get attribute value
my $value = $ipv6fa->override_cli_credentials();
# Modify attribute value
$ipv6fa->override_cli_credentials('true');
```

override_options( )

The override_options attribute controls whether the options in the DHCP IPv6 fixed address are used, instead of the upper-level default.

The override_options attribute can be specified explicitly. It is also set implicitly when options are set to defined values.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

Set the parameter to "true" to override the upper-level setting for valid_lifetime. Set the parameter to "false" to inherit the upper-level setting for valid_lifetime.

The default value of this parameter is "false".

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get override_options
my $override_options=$ipv6fa->override_options( );
#Modify override_options
$ipv6fa->override_options("true");
```

override_preferred_lifetime( )

The override_preferred_lifetime attribute controls whether the preferred_lifetime value in the DHCP IPv6 fixed address is used, instead of the upper-level default.

The override_preferred_lifetime attribute can be specified explicitly. It is also set implicitly when preferred_lifetime is set to a defined value.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

Set the parameter to "true" to override the upper-level setting for valid_lifetime. Set the parameter to "false" to inherit the upper-level setting for valid_lifetime.

The default value of this parameter is "false".

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get override_preferred_lifetime
my $override_preferred_lifetime=$ipv6fa->override_preferred_lifetime( );
#Modify override_preferred_lifetime
$ipv6fa->override_preferred_lifetime("true");
```

override_valid_lifetime( )

The override_valid_lifetime attribute controls whether the valid_lifetime value in the DHCP IPv6 fixed address is used, instead of the upper-level default.

The override_valid_lifetime attribute can be specified explicitly. It is also set implicitly when valid_lifetime is set to a defined value.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

Set the parameter to "true" to override the upper-level setting for valid_lifetime. Set the parameter to "false" to inherit the upper-level setting for valid_lifetime.

The default value of this parameter is "false".

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get override_valid_lifetime
my $override_valid_lifetime=$ipv6fa->override_valid_lifetime( );
#Modify override_valid_lifetime
$ipv6fa->override_valid_lifetime("true");
```

port_duplex( )

Use this method to retrieve the duplex setting of the network device port that is connected to the DHCP IPv6 Fixed Address object.

Parameter

none

Returns

The method returns the port duplex setting. The returned value, if any, is one of the following:

```
Full
Half
```

Example

```
#Get port_duplex
my $port_duplex = $ipv6fa->port_duplex();
```

port_link_status( )

Use this method to retrieve the link status of the network device port that is connected to the DHCP IPv6 Fixed Address object.

Parameter

none

Returns

This method returns the link status of the network device port. The returned value, if any, is one of the following:

```
Connected
Not Connected
Unknown
```

Example

```
#Get port_link_status
my $port_link_status = $ipv6fa->port_link_status();
```

port_speed( )

Use this method to retrieve the speed of the network device port that is connected to the DHCP IPv6 Fixed Address object.

Parameter

none

Returns

The method returns the network device port speed. The returned value, if any, is one of the following:

```
10M
100M
1G
10G
100G
Unknown
```

Example

```
#Get port_speed
my $port_speed = $ipv6fa->port_speed();
```

port_status( )

Use this method to retrieve the status of the network device port that is connected to the DHCP IPv6 Fixed Address object.

Parameter

none

Returns

The method returns the network device port status. The returned value, if any, is one of the following:

```
Up
Down
Unknown
```

Example

```
#Get port_status
my $port_status = $ipv6fa->port_status();
```

port_vlan_description( )

Use this method to retrieve the VLAN description of the network device port that is connected to the DHCP IPv6 Fixed Address object.

Parameter

none

Returns

The method returns the VLAN description of the network device port.

Example

```
#Get port_vlan_description
my $port_vlan_description = $ipv6fa->port_vlan_description();
```

port_vlan_name( )

Use this method to retrieve the VLAN name of the network device port that is connected to the DHCP IPv6 Fixed Address object.

Parameter

none

Returns

The method returns the network device port VLAN name.

Example

```
#Get port_vlan_name
my $port_vlan_name = $ipv6fa->port_vlan_name();
```

port_vlan_number ( )

Use this method to retrieve the VLAN number of the network device port that is connected to the DHCP IPv6 Fixed Address object.

Parameter

none

Returns

The method returns the port VLAN number.

Example

```
#Get port_vlan_number
my $port_vlan_number  = $ipv6fa->port_vlan_number ();
```

preferred_lifetime( )

Use this method to set or retrieve the preferred_lifetime value of a DHCP IPv6 Fixed Address object.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Setting this method to a defined value implicitly sets the override_preferred_lifetime method to "true". Setting the parameter to undefined causes the appliance to use the upper-level default and automatically resets the override_preferred_lifetime attribute to "false".

Note that when preferred_lifetime is set to a defined value and override_preferred_lifetime is set to "false", the last operation takes precedence. Thus the sequence $object->preferred_lifetime(20000); $object->override_preferred_lifetime("false"); will set override_preferred_lifetime to "false", and the sequence $object->override_preferred_lifetime("false"); $object->preferred_lifetime(20000); will result in override_preferred_lifetime="true".

Parameter

A 32-bit integer (valid range from 1 to 4294967295) that represents the lifetime, in seconds. The default value is 27000.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get preferred_lifetime
my $preferred_lifetime = $ipv6fa->preferred_lifetime();
#Modify preferred_lifetime
$ipv6fa->preferred_lifetime("369");
```

template( )

Use this method to set or retrieve the fixed address template name for a DHCP IPv6 Fixed Address object.

Template is the name of a DHCP IPv6 Fixed Address template which will be used to create the DHCP IPv6 Fixed Address.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

The valid name in string format.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

The template parameter can be used to only add a new DHCP IPv6 Fixed Address and cannot be used during object modify or returned during object get.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get template
my $template = $ipv6fa->template();
#Modify template
$ipv6fa->template("Template_Name");
```

valid_lifetime( )

Use this method to set or retrieve the valid_lifetime value of a DHCP IPv6 Fixed Address object.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

A 32-bit integer, from 1 to 4294967295, that represents the lifetime, in seconds. The default value is 43200.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get valid_lifetime
my $valid_lifetime = $ipv6fa->valid_lifetime();
#Modify valid_lifetime
$ipv6fa->valid_lifetime("36900");
```

SAMPLE CODE

The following sample code demonstrates the different functions that can be applied to an object such as add, search, modify, and remove. This sample also includes error handling for the operations.

#Preparation prior to a DHCP IPv6 Fixed Address object insertion

```
#PROGRAM STARTS: Include all the modules that will be used
use strict;
use Infoblox;

#Create a session to the Infoblox appliance

my $session = Infoblox::Session->new(
               master   => "192.168.1.2", #appliance host ip
               username => "admin",       #appliance user login
               password => "infoblox"     #appliance password
);

unless ($session) {
       die("Construct session failed: ",
               Infoblox::status_code() . ":" . Infoblox::status_detail());
}
print "Session created successfully\n";

#Create an IPv6 network object
my $network = Infoblox::DHCP::IPv6Network->new(
                                          "network" => "2001::/64",
                                         );
unless($network) {
     die("Construct IPv6 Network object failed: ",
          Infoblox::status_code() . ":" . Infoblox::status_detail());
}
print "IPv6 Network object created successfully\n";

$session->add($network)
    or die("Add network failed: ",
           $session->status_code() . ":" . $session->status_detail());
print "Network added successfully\n";

my $option1 = Infoblox::DHCP::Option->new(
    "name"  => "dhcp6.fqdn",
    "value" => "mydomain.com"
);
unless($option1) {
    die("Construct option1 object failed: ",
        Infoblox::status_code() . ":" . Infoblox::status_detail());
}
print "option1 object created successfully\n";

my $option2 = Infoblox::DHCP::Option->new(
    "name"     => "dhcp6.info-refresh-time",
    "value"    => "1000",
);
unless($option2) {
    die("Construct option2 object failed: ",
        Infoblox::status_code() . ":" . Infoblox::status_detail());
}
print "option2 object created successfully\n";
```

#Create a DHCP IPv6 Fixed address object

```
#Create a DHCP IPv6 Fixed Address object
my $ipv6fa = Infoblox::DHCP::IPv6FixedAddr->new(
       "ipv6addr"       => "2001::4",
       "network"        => "2001::/64",
       "duid"           => "000000111111",
       "comment"        => "add fixed address",
       "domain_name"    => "foo.com",
       "domain_name_servers" => "2002::1,2002::2",
       "preferred_lifetime" => 1234,
       "override_preferred_lifetime" => "true",
       "valid_lifetime" => 123456,
       "override_valid_lifetime" => "true",
       "options"        => [ $option1 , $option2 ],
       );
unless($ipv6fa) {
     die("Construct IPv6 fixed address failed: ",
           Infoblox::status_code() . ":" . Infoblox::status_detail());
}
print "DHCP IPv6 Fixed address object created successfully\n";

#Verify if the DHCP IPv6 Fixed Address exists in the Infoblox appliance
my $object = $session->get(object => "Infoblox::DHCP::IPv6FixedAddr", duid => "00:00:00:11:11:11");
unless ($object) {
   print "IPv6 fixed address does not exist on server, safe to add the fixed address\n";
   $session->add($ipv6fa)
      or die("Add IPv6 fixed address failed: ",
             $session->status_code() . ":" . $session->status_detail());
}
print "DHCP IPv6 Fixed address added successfully\n";
```

#Create a DHCP IPv6 Fixed address object using a template

```
#Create a DHCP IPv6 Fixed Address template object
my $ipv6fa_template = Infoblox::DHCP::IPv6FixedAddrTemplate->new(
    "name"                  => "custom_fa_template",
    "offset"                => "10",
    "number_of_addresses"   => "1",
    "comment"               => "add fixed address template",
    "valid_lifetime"        => 86400,
);
unless($ipv6fa_template) {
     die("Construct IPv6 fixed address failed: ",
           Infoblox::status_code() . ":" . Infoblox::status_detail());
}
print "DHCP IPv6 Fixed address template object created successfully\n";

$session->add($ipv6fa_template)
      or die("Add IPv6 fixed address template failed: ",
             $session->status_code() . ":" . $session->status_detail());
print "DHCP IPv6 Fixed address template added successfully\n";

#Create a DHCP IPv6 Fixed Address object
my $ipv6fa = Infoblox::DHCP::IPv6FixedAddr->new(
    "network"        => "2001::/64",
    "ipv6addr"       => "2001::2",
    "match_client"   => "DUID",
    "duid"           => "010203040505",
    "template"       => "custom_fa_template"
);
unless($ipv6fa) {
     die("Construct fixed address failed: ",
           Infoblox::status_code() . ":" . Infoblox::status_detail());
}
print "DHCP IPv6 Fixed address object created successfully\n";

$session->add($ipv6fa)
      or die("Add fixed address failed: ",
             $session->status_code() . ":" . $session->status_detail());
print "DHCP IPv6 Fixed address added successfully\n";
```

#Search for a specific DHCP IPv6 Fixed Address object

```
#Search for all DHCP IPv6 Fixed address objects under the network 2001::/64
my @retrieved_objs = $session->search(
    object  => "Infoblox::DHCP::IPv6FixedAddr",
    duid     => "0000001111.*",
```

```
    network => "2001::/64" );
my $object = $retrieved_objs[0];

unless ($object) {
       die("Search for IPv6 Fixed Address object failed: ",
               $session->status_code() . ":" . $session->status_detail());
}
print "Search IPv6 Fixed Address object found at least 1 matching entry\n";
```

#Get and modify a DHCP IPv6 Fixed Address object

```
#Get DHCP IPv6 Fixed Address object through the session
my @retrieved_objs = $session->get(
    object => "Infoblox::DHCP::IPv6FixedAddr",
    duid    => "000000111111",
);
my $object = $retrieved_objs[0];
unless ($object) {
    die("Get IPv6 Fixed Address object failed: ",
        $session->status_code() . ":" . $session->status_detail());
}
print "Get IPv6 Fixed Address object found at least 1 matching entry\n";

#Modify one of the attributes of the obtained IPv6 Fixed Address object
$object->preferred_lifetime("1235");

#Apply the changes
$session->modify($object)
    or die("Modify IPv6 Fixed Address object failed: ",
           $session->status_code() . ":" . $session->status_detail());
print "IPv6 Fixed Address object modified successfully \n";
```

#Remove an IPv6 Fixed Address object

```
#Get IPv6 Fixed Address object through the session
my @retrieved_objs = $session->get(
    object => "Infoblox::DHCP::IPv6FixedAddr",
    duid    => "000000111111",
);
my $object = $retrieved_objs[0];
unless ($object) {
    die("Get IPv6 Fixed Address object failed: ",
        $session->status_code() . ":" . $session->status_detail());
}
print "Get IPv6 Fixed Address object found at least 1 matching entry\n";

#Submit the object for removal
$session->remove($object)
    or die("Remove IPv6 Fixed Address object failed: ",
        $session->status_code() . ":" . $session->status_detail());
print "IPv6 Fixed Address object removed successfully \n";

#Submit the Network object for removal
$session->remove($network)
    or die("Remove Network object failed: ",
        $session->status_code() . ":" . $session->status_detail());
print "Network object removed successfully \n";

#Submit the Fixed Address Template object for removal
$session->remove($ipv6fa_template)
    or die("Remove IPv6 Fixed Address Template object failed: ",
        $session->status_code() . ":" . $session->status_detail());
print "IPv6 Fixed Address Template object removed successfully \n";
####PROGRAM ENDS####
```

AUTHOR

Infoblox Inc. http://www.infoblox.com/

SEE ALSO

Infoblox::DHCP::IPv6Network, Infoblox::DHCP::View, Infoblox::DHCP::IPv6Range, Infoblox::DHCP::IPv6Lease, Infoblox::Session->add(), Infoblox::Session->get(), Infoblox::Session->modify(), Infoblox::Session->remove(), Infoblox::Session->search(), Infoblox::Session, Infoblox::DHCP::IPv6FixedAddrTemplate, Infoblox::Grid::Discovery::Data

---

COPYRIGHT

Copyright (c) 2017 Infoblox Inc.

Infoblox::DHCP::IPv6RangeTemplate - DHCP IPv6 RangeTemplate object.

- NAME
- DESCRIPTION
- CONSTRUCTOR
- MODULE METHODS
  - Infoblox::DHCP::IPv6NetworkTemplate->range_templates( )
- SESSION METHODS
  - Infoblox::Session->add( )
  - Infoblox::Session->get( )
  - Infoblox::Session->search( )
  - Infoblox::Session->modify( )
  - Infoblox::Session->remove( )
- METHODS
  - comment( )
  - exclude( )
  - extattrs( )
  - extensible_attributes( )
  - member( )
  - name( )
  - number_of_addresses( )
  - offset( )
  - override_recycle_leases( )
  - recycle_leases( )
  - server_association_type( )
- SAMPLE CODE
- AUTHOR
- SEE ALSO
- COPYRIGHT

NAME

Infoblox::DHCP::IPv6RangeTemplate - DHCP IPv6 RangeTemplate object.

DESCRIPTION

A DHCP IPv6 range template contains a set of predefined properties and can be used to create DHCP IPv6 ranges in a quick and consistent way. The DHCP IPv6 range created from a DHCP IPv6 range template will inherit the properties defined in the template.

CONSTRUCTOR

```
 my $ipv6_range_template = Infoblox::DHCP::IPv6RangeTemplate->new(
             name                    => $string,                                    #Required
             number_of_addresses     => $num,                                       #Required
             offset                  => $num,                                       #Required
             comment                 => $string,                                    #Optional / Default is
undefined
             exclude                 => [$ExclusionRange1, $ExclusionRange2,...],   #Optional / Default is
empty
             extattrs                => { $string => $extattr, ... },               #Optional / Default is
undefined
             extensible_attributes   => { $string => $string | $num, $string => [ $string | $num, ... ],
... }, #Optional / Default is undefined
             member                  => $Member,                                    #Optional / Default is
undefined
             override_recycle_leases => "true" | "false",                           #Optional / Default is
"false"
             recycle_leases          => "true" | "false" | undef,                   #Optional / Default is
undefined
             server_association_type => "NONE" | "MEMBER",                          #Optional / Default is
"NONE"
  );
```

You cannot set both extattrs and extensible_attributes attributes at the same time.

MODULE METHODS

The following functions can be applied to a DHCP IPv6 range template object.

Infoblox::DHCP::IPv6NetworkTemplate->range_templates( )

Use this function to include a range template in an IPv6 network template on the Infoblox appliance. See Infoblox::DHCP::IPv6NetworkTemplate->range_templates() for parameters and return values.

Example

```
#Construct an IPv6 range template object
my $ipv6_range_template = Infoblox::DHCP::IPv6RangeTemplate->new(
            name                    => "custom_range_template",
            number_of_addresses     => "1000",
            offset                  => "10",
    );

my $template = Infoblox::DHCP::Template->new(
            name                    => "custom_range_template",
    );

#Configure a range template in a network template object
my $responce = $network_template->range_templates([$template]);
```

SESSION METHODS

This section describes all the methods in an Infoblox::Session module that can be applied to a DHCP IPv6 range template object.

Infoblox::Session->add( )

Use this method to add a DHCP IPv6 range template object to the Infoblox appliance. See Infoblox::Session->add() for parameters and return values.

Example

```
#Construct an object
my $ipv6_range_template = Infoblox::DHCP::IPv6RangeTemplate->new(
        name                    => "custom_range_template",
        offset                  => "20",
        number_of_addresses     => "100",
        comment                 => "add range",
        exclude                 => [$exclusion_range_template],
        member                  => $memberdhcp,
        recycle_leases          => "false",
        override_recycle_leases => "true",
        server_association_type => 'MEMBER',
);

# Submit for addition
my $response = $session->add( $ipv6_range_template );
```

Infoblox::Session->get( )

Use this method to retrieve all the matching objects from the Infoblox appliance. See Infoblox::Session->get() for parameters and return values.

Key References

```
Apply the following attributes to get a specific DHCP IPv6 range template object:

name - Optional. The name of the DHCP IPv6 range template.
extattrs     - Optional. A hash reference containing extensible attributes.
extensible_attributes - Optional. A hash reference containing extensible attributes.
server_association_type - Optional. The server association type for the range.
```

Examples

```
my @retrieved_objs = $session->get(
                      object => "Infoblox::DHCP::IPv6RangeTemplate",
                      name   => "custom_range_template"
                      );

my @retrieved_objs = $session->get(
                     object               => "Infoblox::DHCP::IPv6RangeTemplate",
                     extensible_attributes => { 'Site' => 'Santa Clara' }
                    );
```

Infoblox::Session->search( )

Use this method to retrieve all the matching objects from the Infoblox appliance. See Infoblox::Session->search() for parameters and

return values.

Key References

```
Apply the following attributes to search for a specific DHCP IPv6 range template object:

name    - Optional. The name of the DHCP IPv6 range template (regular expression).
comment - Optional. The comment of the DHCP IPv6 range template (regular expression).
extattrs     - Optional. A hash reference containing extensible attributes.
extensible_attributes - Optional. A hash reference containing extensible attributes.
server_association_type - Optional. The server association type for the range.
```

For more information about searching extensible attributes, see Infoblox::Grid::ExtensibleAttributeDef/Searching Extensible Attributes.

Examples

```
my @retrieved_objs = $session->search(
                        object  => "Infoblox::DHCP::IPv6RangeTemplate",
                        name    => "custom_range_template",
                        comment => ".*",
        );

my @retrieved_objs = $session->search(
                        object               => "Infoblox::DHCP::IPv6RangeTemplate",
                        extensible_attributes => { 'Site' => 'Santa Clara' }
        );
```

Infoblox::Session->modify( )

Use this method to modify a DHCP IPv6 range template object in the Infoblox appliance. See Infoblox::Session->modify() for parameters and return values.

Example

```
# Use this method to modify the comment.
$ipv6_range_template->comment("this is a modified comment");
# Submit modification
my $response = $session->modify( $ipv6_range_template );
```

Infoblox::Session->remove( )

Use this method to remove a DHCP IPv6 range template object from the Infoblox appliance. See Infoblox::Session->remove() for parameters and return values.

To remove a specific object, first use `get()` or `search()` to retrieve the specific DHCP IPv6 range template object, and then submit this object for removal.

Example

```
# Get the objects with the same name
my @retrieved_objs = $session->get(
    object => "Infoblox::DHCP::IPv6RangeTemplate",
    name   => "custom_range_template"
    );

# Find the desired object from the retrieved list.
my $desired_ipv6_range_template = $retrieved_objs[0];
# Submit for removal
my $response = $session->remove( $desired_ipv6_range_template );
```

METHODS

This section describes all the methods that can be used to configure and retrieve the attribute values of a DHCP IPv6 range template object.

comment( )

Use this method to set or retrieve a descriptive comment of a DHCP IPv6 range template.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

Desired comment in string format with a maximum of 256 bytes.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get comment
my $comment = $ipv6_range_template->comment();
#Modify comment
$ipv6_range_template->comment("Modifying the DHCP IPv6 range template comment");
```

exclude( )

Use this method to set or retrieve the exclusion range template in the DHCP IPv6 range template.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

Valid value is an array reference that contains defined Infoblox::DHCP::ExclusionRangeTemplate objects.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get exclude
my $ref_exclude = $ipv6_range_template->exclude();
#Modify exclude
$ipv6_range_template->exclude([$exl1]); #$exl1 is an Infoblox::DHCP::ExclusionRangeTemplate
object
```

extattrs( )

Use this method to set or retrieve the extensible attributes associated with a DHCP IPv6 range template object.

Parameter

Valid value is a hash reference containing the names of extensible attributes and their associated values (Infoblox::Grid::Extattr objects).

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get extattrs
my $ref_extattrs = $ipv6_range_template->extattrs();
#Modify extattrs
$ipv6_range_template->extattrs({ 'Site' => $extattr1, 'Administrator' => $extattr2 });
```

extensible_attributes( )

Use this method to set or retrieve the extensible attributes associated with a DHCP IPv6 range template object.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

For valid values for extensible attributes, see Infoblox::Grid::ExtensibleAttributeDef/Extensible Attribute Values.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get extensible attributes
my $ref_extensible_attributes = $ipv6_range_template->extensible_attributes();
#Modify extensible attributes
$ipv6_range_template->extensible_attributes({ 'Site' => 'Santa Clara', 'Administrator' => [ 'Peter',
```

```
    'Tom' ] });
```

member( )

Use this method to set or retrieve the grid member assigned to the DHCP IPv6 range template.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

The valid value is a defined Infoblox::DHCP::Member object. The default value is undefined.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get member
my $ref_member = $ipv6_range_template->member();
#Modify member
$ipv6_range_template->member($member1); #$member is an Infoblox::DHCP::Member object
```

name( )

Use this method to set or retrieve the name of the DHCP IPv6 range template.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

Name of the IPv6 range template in string format with a maximum of 64 bytes. The default value is undefined.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get name
my $name = $ipv6_range_template->name();
#Modify name
$ipv6_range_template->name("custom_range_template");
```

number_of_addresses( )

Use this method to set or retrieve the number of addresses in this DHCP IPv6 range template.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

The number of addresses in this DHCP IPv6 range template.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get number_of_addresses
my $number_of_addresses = $ipv6_range_template-> number_of_addresses();
#Modify number_of_addresses
$ipv6_range_template->number_of_addresses("100");
```

offset( )

Use this method to set or retrieve the address offset of this DHCP IPv6 range template.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

The address offset of this DHCP IPv6 range template. The offset is calculated from the start address of the IPv6 network template that includes the DHCP range template. For example, in the IPv6 network 2010:0:0:10::/64, if the offset is set to 10 and number_of_addresses is set to 100, then the start address of the DHCP IPv6 range created from this range template is 2010:0:0::A and the end address is 2010:0:0::6D.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get offset
my $offset = $ipv6_range_template->offset();
#Modify offset
$ipv6_range_template->offset("10");
```

override_recycle_leases( )

The override_recycle_leases attribute controls whether the recycle_leases value in the DHCP IPv6 range template is used, instead of the grid default.

The override_recycle_leases attribute can be specified explicitly. It is also set implicitly when recycle_leases is set to a defined value.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

Set the parameter to "true" to override the grid-level setting for recycle_leases. Set the parameter to "false" to inherit the grid-level setting for recycle_leases.

The default value of this parameter is "false".

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Getting override_recycle_leases
my $override_recycle_leases=$ipv6_range_template->override_recycle_leases( );
#Modifying override_recycle_leases
$ipv6_range_template->override_recycle_leases("true");
```

recycle_leases( )

Use this method to set or retrieve the recycle_leases flag of the DHCP IPv6 range template.

If the flag is enabled, the leases are kept in the recycle bin until one week after they expire. When the flag is disabled, the leases are irrecoverably deleted.

Setting this method to a defined value implicitly sets the override_recycle_leases method to "true". Setting the parameter to undefined causes the appliance to use the grid default and automatically resets the override_recycle_leases attribute to "false".

Note that when recycle_leases is set to a defined value and override_recycle_leases is set to "false", the last operation takes precedence. Thus the sequence $object->recycle_leases("true"); $object->override_recycle_leases("false"); will set override_recycle_leases to "false", and the sequence $objct->override_recycle_leases("false"); $object->recycle_leases("true"); will result in override_recycle_leases="true".

Parameter

Specify "true" to set the recycle_leases flag or "false" to deactivate/unset it. The default value is undefined.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get recycle_leases
my $recycle_leases = $ipv6_range_template->recycle_leases();
#Modify recycle_leases
$ipv6_range_template->recycle_leases("false");
```

server_association_type( )

Use this method to set or retrieve the DHCP IPv6 range template server association type.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

Valid values are "NONE" and "MEMBER".

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get server_association_type
my $server_association_type = $ipv6_range_template->server_association_type();
#Modify server_association_type
$ipv6_range_template->server_association_type("MEMBER");
```

SAMPLE CODE

The following sample code demonstrates the different functions that can be applied to a DHCP range template object such as add, modify, and remove. Also, this sample includes error handling for the operations.

#Preparation prior to a DHCP IPv6RangeTemplate object insertion

```
#PROGRAM STARTS: Include all the modules that will be used
use strict;
use Infoblox;

#Create a session to the Infoblox appliance
my $session = Infoblox::Session->new(
               master   => "192.168.1.2",      #appliance host ip
               username => "admin",       #appliance user login
               password => "infoblox"     #appliance password
);
unless ($session) {
       die("Construct session failed: ",
            Infoblox::status_code() . ":" . Infoblox::status_detail());
}
print "Session created successfully\n";

#Create a DHCP Member object
my $memberdhcp = Infoblox::DHCP::Member->new(
    name     => "infoblox.localdomain",
    ipv4addr => "192.168.1.2"
);
unless($memberdhcp) {
       die("Construct member failed: ",
            Infoblox::status_code() . ":" . Infoblox::status_detail());
}
print "DHCP Member object created successfully\n";

#Construct exclusion range template object
my $exclusion_range_template = Infoblox::DHCP::ExclusionRangeTemplate->new(
    offset              => "15",
    number_of_addresses => "50",
    comment             => "Excluded range"
);
unless($exclusion_range_template){
       die("Construct exclusion range template object failed: ",
                Infoblox::status_code() . ":" . Infoblox::status_detail());
       }
 print "Exclusion range template object created successfully\n";
```

#Create a DHCP IPv6 range template object

```
my $ipv6_range_template = Infoblox::DHCP::IPv6RangeTemplate->new(
        name                  => "custom_range_template",
        offset                => "20",
        number_of_addresses   => "100",
        comment               => "add range",
```

```
        exclude                 => [$exclusion_range_template],
        member                  => $memberdhcp,
        recycle_leases          => "false",
        override_recycle_leases => "true",
        server_association_type => 'MEMBER',
);
unless($ipv6_range_template) {
       die("Construct DHCP IPv6 range template object failed: ",
            Infoblox::status_code() . ":" . Infoblox::status_detail());
}
print "DHCP IPv6 range template object created successfully\n";

#Add the DHCP IPv6 range template object into the Infoblox appliance through a session
$session->add($ipv6_range_template)
    or die("Add DHCP IPv6 range template object failed: ",
            $session->status_code() . ":" . $session->status_detail());

print "DHCP IPv6 range template object added to the network successfully\n";
```

#Get and modify a DHCP IPv6 range template

```
#Get a DHCP IPv6 range template through the session
my @retrieved_objs = $session->get(
    object     => "Infoblox::DHCP::IPv6RangeTemplate",
    name       => "custom_range_template",
);
my $object = $retrieved_objs[0];
unless ($object) {
       die("Get DHCP IPv6 range template failed: ",
            $session->status_code() . ":" . $session->status_detail());
}
print "Get DHCP IPv6 range template object found at least 1 matching entry\n";

#Modify one of the attributes of the specified DHCP IPv6 range template
$object->comment("modified comment");

#Apply the changes
$session->modify($object)
    or die("Modify DHCP IPv6 range template failed: ",
            $session->status_code() . ":" . $session->status_detail());

print "DHCP IPv6 range template object modified successfully \n";
```

#Remove a DHCP IPv6 range template

```
#Get a DHCP IPv6 range template through the session
my @retrieved_objs = $session->get(
    object     => "Infoblox::DHCP::IPv6RangeTemplate",
    name       => "custom_range_template",
);

my $object = $retrieved_objs[0];
unless ($object) {
       die("Get DHCP IPv6 range template failed: ",
            $session->status_code() . ":" . $session->status_detail());
}
print "Get DHCP IPv6 range template object found at least 1 matching entry\n";

#Submit the object for removal
$session->remove($object)
    or die("Remove DHCP IPv6 range template failed: ",
            $session->status_code() . ":" . $session->status_detail());

print "DHCP IPv6 range template object removed successfully \n";

####PROGRAM ENDS####
```

AUTHOR

Infoblox Inc. http://www.infoblox.com/

SEE ALSO

Infoblox::Session, Infoblox::DHCP::IPv6NetworkTemplate, Infoblox::DHCP::RangeTemplate, Infoblox::DHCP::ExclusionRangeTemplate, Infoblox::DHCP::Template, Infoblox::DHCP::Member

COPYRIGHT

Copyright (c) 2017 Infoblox Inc.

Infoblox::Grid::CloudAPI::Info - Infoblox::Grid::CloudAPI::Info object.

- NAME
- DESCRIPTION
- SESSION METHODS
- METHODS
  - delegated_member( )
  - delegated_root( )
  - delegated_scope( )
  - owned_by_adaptor( )
  - tenant( )
  - usage( )
  - authority_type( )
- AUTHOR
- SEE ALSO
- COPYRIGHT

NAME

Infoblox::Grid::CloudAPI::Info - Infoblox::Grid::CloudAPI::Info object.

DESCRIPTION

This structure aggregates all information related to Cloud objects.

SESSION METHODS

The object does not support any session methods. It is retrieved as part of the objects supporting it.

METHODS

This section describes all the methods that you can use to set or retrieve the attribute values of the object.

delegated_member( )

Use this method to set or retrieve the reference to the Cloud Platform Appliance to which authority of an object is delegated. If the delegated_scope value is SUBTREE or RECLAIMING, this is a read-only attribute.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

None if the attribute is a read-only. Otherwise the valid value is an Infoblox::DHCP::Member object.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

If this is a read-only attribute, the method returns the attribute value.

Example

```
# Get delegated_member
$delegated_member = $cloud_api_info->delegated_member();
# Set delegated_member
$cloud_api_info->delegated_member($member);
```

delegated_root( )

Use this method to retrieve the root of the delegation if delegated_scope is SUBTREE or RECLAIMING. This is a read-only attribute.

Parameter

None

Returns

This method returns the attribute value.

Example

```
# Get delegated_root
$delegated_root = $cloud_api_info->delegated_root();
```

delegated_scope( )

Use this method to retrieve the delegation scope of the object. This is a read-only attribute.

Parameter

None

Returns

This method returns the attribute value.

Example

```
# Get delegated_scope
$delegated_scope = $cloud_api_info->delegated_scope();
```

owned_by_adaptor( )

Use this method to retrieve the information about whether the object was created by the cloud adapter or not.

Parameter

None

Returns

This method returns the attribute value.

Example

```
# Get owned_by_adaptor
$owned_by_adaptor = $cloud_api_info->owned_by_adaptor();
```

tenant( )

Use this method to retrieve the reference to the tenant object associated with the object, if any. This is a read-only attribute.

Parameter

None

Returns

This method returns the attribute value.

Example

```
# Get tenant
$tenant = $cloud_api_info->tenant();
```

usage( )

Use this method to retrieve the the cloud origin of the object. This is a read-only attribute.

Parameter

None

Returns

This method returns the attribute value.

Example

```
# Get usage
$usage = $cloud_api_info->usage();
```

authority_type( )

Use this method to retrieve the authority type of the object. This is a read-only attribute.

Parameter

None

Returns

This method returns the attribute value.

Example

```
# Get authority_type
$authority_type = $cloud_api_info->authority_type();
```

AUTHOR

Infoblox Inc. http://www.infoblox.com/

SEE ALSO

Infoblox::DHCP::Member, Infoblox::Grid::CloudAPI::Tenant

COPYRIGHT

Copyright (c) 2017 Infoblox Inc.

Infoblox::Grid::Discovery::BasicPollSettings - basic polling settings.

- NAME
- DESCRIPTION
- CONSTRUCTOR
- SESSION METHODS
- METHODS
  - auto_arp_refresh_before_switch_port_polling( )
  - cli_collection( )
  - complete_ping_sweep( )
  - device_profile( )
  - netbios_scanning( )
  - port_scanning( )
  - smart_subnet_ping_sweep( )
  - snmp_collection( )
  - switch_port_data_collection_polling( )
  - switch_port_data_collection_polling_interval( )
  - switch_port_data_collection_polling_schedule( )
- AUTHOR
- SEE ALSO
- COPYRIGHT

NAME

Infoblox::Grid::Discovery::BasicPollSettings - basic polling settings.

DESCRIPTION

The object provides information about basic polling settings.

CONSTRUCTOR

```
 my $object = Infoblox::Grid::Discovery::BasicPollSettings->new(
    'auto_arp_refresh_before_switch_port_polling'  => 'true' | 'false',                    # Optional /
Default value is 'false'
    'device_profile'                               => 'true' | 'false',                    # Optional /
Default value is 'false'
    'complete_ping_sweep'                          => 'true' | 'false',                    # Optional /
Default value is 'false'
    'netbios_scanning'                             => 'true' | 'false',                    # Optional /
Default value is 'false'
    'port_scanning'                                => 'true' | 'false',                    # Optional /
Default value is 'false'
    'smart_subnet_ping_sweep'                      => 'true' | 'false',                    # Optional /
Default value is 'false'
    'snmp_collection'                              => 'true' | 'false',                    # Optional /
Default value is 'false'
    'cli_collection'                               => 'true' | 'false',                    # Optional /
Default value is 'false'
    'switch_port_data_collection_polling'          => 'DISABLED' | 'PERIODIC' | 'SCHEDULED',    # Optional
    'switch_port_data_collection_polling_interval' => $uint,                               # Optional
    'switch_port_data_collection_polling_schedule' => $schedule,                           # Optional
 );
```

SESSION METHODS

The object does not support any session methods.

METHODS

This section describes all the methods that you can use to set or retrieve the attribute values of the object.

auto_arp_refresh_before_switch_port_polling( )

Use this method to enable or disable auto ARP refresh before switch port polling.

Parameter

Specify "true" to enable auto ARP refresh before switch port polling or "false" to disable it. Default value is "false".

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
# Get attribute value
my $value = $object->auto_arp_refresh_before_switch_port_polling();
# Modify attribute value
$object->auto_arp_refresh_before_switch_port_polling('true');
```

cli_collection( )

Use this method to disable or enable CLI collection.

Parameter

Specify "true" to enable CLI collection or "false" to disable it. Default value if "false".

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
# Get attribute value
my $value = $object->cli_collection();
# Modify attribute value
$object->cli_collection('true');
```

complete_ping_sweep( )

Use this method to enable or disable ping sweep.

Parameter

Specify "true" to enable ping sweep or "false" to disable it. Default value is "false".

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
# Get attribute value
my $value = $object->complete_ping_sweep();
# Modify attribute value
$object->complete_ping_sweep('true');
```

device_profile( )

Use this method to enable or disable DHCP fingerprinting.

Parameter

Specify "true" to enable DHCP fingerprinting or "false" to disable it. Default value is "false".

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
# Get attribute value
my $value = $object->device_profile();
# Modify attribute value
$object->device_profile('true');
```

netbios_scanning( )

Use this method to enable or disable NETBIOS scanning.

Parameter

Specify "true" to enable NETBIOS scanning or "false" to disable it. Default value is "false".

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
# Get attribute value
my $value = $object->netbios_scanning();
# Modify attribute value
$object->netbios_scanning('true');
```

port_scanning( )

Use this method to enable or disable port scanning.

Parameter

Specify "true" to enable port scanning or "false" to disable it. Default value is "false".

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
# Get attribute value
my $value = $object->port_scanning();
# Modify attribute value
$object->port_scanning('true');
```

smart_subnet_ping_sweep( )

Use this method to enable or disable smart subnet ping sweep.

Parameter

Specify "true" to enable smart subnet ping sweep or "false" to disable it. Default value is "false".

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
# Get attribute value
my $value = $object->smart_subnet_ping_sweep();
# Modify attribute value
$object->smart_subnet_ping_sweep('true');
```

snmp_collection( )

Use this method to enable or disable SNMP collection.

Parameter

Specify "true" to enable SNMP collection or "false" to disable it. Default value is "false".

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
# Get attribute value
my $value = $object->snmp_collection();
# Modify attribute value
$object->snmp_collection('true');
```

switch_port_data_collection_polling( )

Use this method to set or retrieve the polling mode.

Parameter

Valid value is 'DISABLED', 'PERIODIC' or 'SCHEDULED'.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
# Get attribute value
my $value = $object->switch_port_data_collection_polling();
# Modify attribute value
$object->switch_port_data_collection_polling('SCHEDULED');
```

switch_port_data_collection_polling_interval( )

Use this method to set or retrieve the polling interval.

Parameter

Valid value is an unsigned integer.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
# Get attribute value
my $value = $object->switch_port_data_collection_polling_interval();
# Modify attribute value
$object->switch_port_data_collection_polling_interval(100);
```

switch_port_data_collection_polling_schedule( )

Use this method to set or retrieve the polling schedule.

Parameter

Valid value is an Infoblox::Grid::ScheduleSetting object.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
# Get attribute value
my $value = $object->switch_port_data_collection_polling_schedule();
# Modify attribute value
$object->switch_port_data_collection_polling_schedule($schedule);
```

AUTHOR

Infoblox Inc. http://www.infoblox.com/

SEE ALSO

Infoblox::Grid::ScheduleSetting

COPYRIGHT

Copyright (c) 2017 Infoblox Inc.

NAME

- NAME
- DESCRIPTION
- CONSTRUCTOR
- SESSION METHODS
- METHODS
  - blackout_duration( )
  - blackout_schedule( )
  - enable_blackout( )
- AUTHOR
- SEE ALSO
- COPYRIGHT

NAME

Infoblox::Grid::Discovery::Properties::BlackoutSetting

DESCRIPTION

The object provides information about the discovery or port control blackout settings.

CONSTRUCTOR

```
my $object = Infoblox::Grid::Discovery::Properties::BlackoutSetting->new(
    'enable_blackout'   => 'true' | 'false',      # optional
    'blackout_duration' => $uint,                 # optional
    'blackout_schedule' => $schedule,             # optional
);
```

SESSION METHODS

The object does not support any session methods.

METHODS

This section describes all the methods that you can use to set or retrieve the attribute values of the object.

blackout_duration( )

Use this method to set or retrieve the blackout duration.

Parameter

The blackout duration in minutes.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
# Get attribute value
my $value = $object->blackout_duration();
# Modify attribute value
$object->blackout_duration(10);
```

blackout_schedule( )

Use this method to set or retrieve the blackout schedule.

Parameter

Valid value is an Infoblox::Grid::ScheduleSetting object.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
# Get attribute value
my $value = $object->blackout_schedule();
# Modify attribute value
$object->blackout_schedule($schedule);
```

enable_blackout( )

Use this method to enable or disable the Discovery or port control blackout.

Parameter

Specify 'true' to enable the discovery or port control blackout or 'false' to disable it. The default value is 'false'.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
# Get attribute value
my $value = $object->enable_blackout();
# Modify attribute value
$object->enable_blackout('true');
```

AUTHOR

Infoblox Inc. http://www.infoblox.com/

SEE ALSO

Infoblox::Grid::Discovery::Properties

COPYRIGHT

Copyright (c) 2017 Infoblox Inc.

Infoblox::Grid::RIR::Organization - Regional Internet Registry organization object.

- NAME
- DESCRIPTION
- CONSTRUCTOR
- SESSION METHODS
  - Infoblox::Session->add( )
  - Infoblox::Session->get( )
  - Infoblox::Session->modify( )
  - Infoblox::Session->remove( )
  - Infoblox::Session->search( )
- METHODS
  - id( )
  - rir( )
  - extattrs( )
  - extensible_attributes( )
  - name( )
  - maintainer( )
  - password( )
  - sender_email( )
- SAMPLE CODE
- AUTHOR
- SEE ALSO
- COPYRIGHT

NAME

Infoblox::Grid::RIR::Organization - Regional Internet Registry (RIR) organization object.

DESCRIPTION

Regional Internet Registry (RIR) organization object.

CONSTRUCTOR

```
my $rir_org = Infoblox::Grid::RIR::Organization->new(
   id                   => $string,                       # Required
   rir                  => $rir,                          # Required, an Infoblox::Grid::RIR object
   extattrs             => { $string => $extattr, ... },  # Required
   extensible_attributes => $extattrs,                    # Required
   name                 => $string,                       # Required
   maintainer           => $string,                       # Required
   password             => $string,                       # Required, write-only
   sender_email         => $string,                       # Required
);
```

You cannot set both extattrs and extensible_attributes attributes at the same time.

SESSION METHODS

This section describes all the methods in an Infoblox::Session module that you can apply to RIR organization object.

Support for RIR/SWIP must be enabled at the Grid level. See Infoblox::Grid->enable_rir_swip() method.

Infoblox::Session->add( )

Use this method to add an object to the Infoblox appliance. See Infoblox::Session->add() for parameters and return values.

Example

```
#Construct an object
my $rir_org = Infoblox::Grid::RIR::Organization->new(
   id                   => 'ORG_ID',
   name                 => 'RIR organization',
   maintainer           => 'support',
   password             => 'infoblox',
   sender_email         => 'support@infoblox.com',
   rir                  => $rir,
   extensible_attributes' => {'RIPE Registry Source' => 'RIPE',
```

```
                                    'RIPE Country' => 'United States (US)',
                                    'RIPE Admin Contact' => 'CA1-TEST',
                                    'RIPE Technical Contact' => 'CT1-TEST',
                                    'RIPE Email' => 'admin@infoblox.com'},
);


#Submit for addition
my $response = $session->add( $rir_org );
```

Infoblox::Session->get( )

Use this method to retrieve all the matching objects from the Infoblox appliance. See Infoblox::Session->get() for parameters and return values.

Key References

```
Apply the following attributes to get a specific RIR organization object:

 id           - Optional. The RIR Organization ID.
 name         - Optional. The RIR organization name.
 extattrs     - Optional. A hash reference containing extensible attributes.
```

Example

```
my @retrieved_objs = $session->get(
   object => 'Infoblox::Grid::RIR::Organization',
   name   => 'RIR organization',
);
```

Infoblox::Session->modify( )

Use this method to modify an object in the Infoblox appliance. See Infoblox::Session->modify() for parameters and return values.

Example

```
# Use this method to set password of a RIR organization object.
$rir_org->password('infoblox');
# Submit modification
my $response = $session->modify( $rir_org );
```

Infoblox::Session->remove( )

Use this method to remove an object from the Infoblox appliance. See Infoblox::Session->remove() for parameters and return values.

To remove a specific object, first use `get()` or `search()` to retrieve the specific object, and then submit this object for removal.

Example

```
#Get the objects with the same name
my @retrieved_objs = $session->get(
   object => 'Infoblox::Grid::RIR::Organization',
   name   => 'RIR organization',
);
#Find the desired object from the retrieved list.
my $desired_rir_org = $retrieved_objs[0];
# Submit for removal
my $response = $session->remove( $desired_rir_org );
```

Infoblox::Session->search( )

Use this method to search for RIR organization objects in the Infoblox appliance. See Infoblox::Session->search() for parameters and return values.

Key References

```
Apply the following attributes to search a specific RIR organization object:

 id           - Optional. The RIR Organization ID.
 name         - Optional. The RIR organization name.
 extattrs     - Optional. A hash reference containing extensible attributes.
```

Example

```
my @retrieved_objs = $session->search(
   object => 'Infoblox::Grid::RIR::Organization',
   name   => 'RIR organization',
);
```

METHODS

This section describes all the methods that you can use to configure and retrieve attribute values of the RIR organization object.

id( )

Use this method to set or retrieve RIR Organization ID.

Parameter

RIR Organization ID in string format.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get attribute value
my $value = $rir_org->id();
#Modify attribute value
$rir_org->id('ORG-MC1-RIPE');
```

rir( )

Use this method to set or retrieve RIR object which is associated with the organization.

Parameter

An Infoblox::Grid::RIR object.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get attribute value
my $value = $rir_org->rir();
#Modify attribute value
$rir_org->rir($rir);
```

extattrs( )

Use this method to set or retrieve the extensible attributes associated with a RIR organization object.

Parameter

Valid value is a hash reference containing the names of extensible attributes and their associated values (Infoblox::Grid::Extattr objects).

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get extattrs
my $ref_extattrs = $rir_org->extattrs();
#Modify extattrs
$rir_org->extattrs({ 'Site' => $extattr1, 'Administrator' => $extattr2 });
```

extensible_attributes( )

Use this method to set or retrieve the extensible attributes associated with a RIR organization object.

Parameter

For valid values for extensible attributes, see Infoblox::Grid::ExtensibleAttributeDef/Extensible Attribute Values and Infoblox::Grid->enable_rir_swip().

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get attribute value
my $value = $rir_org->extensible_attributes();
#Modify attribute value
$rir_org->extensible_attributes({'RIPE Registry Source' => 'TEST',
                                 'RIPE Country' => 'Canada (CA)',
                                 'RIPE Admin Contact' => 'CA2-TEST',
                                 'RIPE Technical Contact' => 'CT2-TEST',
                                 'RIPE Email' => 'support@infoblox.com'});
```

name( )

Use this method to set or retrieve RIR organization name.

Parameter

RIR organization name in string format.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get attribute value
my $value = $rir_org->name();
#Modify attribute value
$rir_org->name('New RIR organization');
```

maintainer( )

Use this method to set or retrieve the RIR organization maintainer.

Parameter

RIR organization maintainer in string format.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get attribute value
my $value = $rir_org->maintainer();
#Modify attribute value
$rir_org->maintainer('support');
```

password( )

Use this method to set the password for RIR organization's maintainer. This is a write-only attribute.

Parameter

The password for RIR organization's maintainer in string format.

Returns

The method returns true when the modification succeeds, and returns false when the operation fails.

Example

```
#Set attribute value
$rir_org->password('infoblox');
```

sender_email( )

Use this method to set or retrieve the sender email address for RIR organization.

Parameter

The sender email address for RIR organization in string format.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get attribute value
my $value = $rir_org->sender_email();
#Modify attribute value
$rir_org->sender_email('support@infoblox.com');
```

SAMPLE CODE

The following sample code demonstrates the different functions that can be applied to an object such as add, get, modify, search and remove. This sample also includes error handling for the operations.

#Preparation prior to getting and modifying RIR organization object

```
use strict;
use Infoblox;

#refers to Infoblox Appliance IP address
my $host_ip = "192.168.1.2";

#Create a session to the Infoblox appliance

my $session = Infoblox::Session->new(
    master   => $host_ip,
    username => "admin",
    password => "infoblox"
);
unless ($session) {
       die("Construct session failed: ",
               Infoblox::status_code() . ":" . Infoblox::status_detail());
}
print "Session created successfully\n";

my $grid = $self->{'session'}->get('object' => 'Infoblox::Grid');

unless ($grid) {
    die("Get Grid object failed: ",
           $session->status_code() . ":" . $session->status_detail());
}
print"Get Grid successful \n";

$grid->enable_rir_swip('true');

$session->modify($grid)
   or die("Modify Grid failed",
            $session->status_code() . ":" . $session->status_detail());
print "Grid modified successfully \n";
```

#Create RIR organization object

```
my $rir = $self->{'session'}->get('object' => 'Infoblox::Grid::RIR', 'name' => 'RIPE');

unless ($rir) {
    die("Get RIR object failed: ",
           $session->status_code() . ":" . $session->status_detail());
}
print"Get RIR successful \n";

my $rir_org = Infoblox::Grid::RIR::Organization->new(
       'id'            => 'RIR_ORG_ID',
       'name'          => 'RIR organization',
       'maintainer'    => 'support',
       'password'      => 'infoblox',
       'sender_email'  => 'support@infoblox.com',
```

```
        'rir'           => $rir,
        'extensible_attributes' => {'RIPE RIPE Registry Source' => 'RIPE',
                                    'RIPE Country' => 'United States (US)',
                                    'RIPE Admin Contact' => 'CA1-TEST',
                                    'RIPE Technical Contact' => 'CT1-TEST',
                                    'RIPE Email' => 'admin@infoblox.com'},
);

unless ($rir_org) {
    die("Construct RIR organization object failed: ",
            $session->status_code() . ":" . $session->status_detail());
}
print"RIR organization object created successfully\n";

my $response = $session->add($rir_org);

unless ($response) {
    die("Add RIR organization failed: ",
            $session->status_code() . ":" . $session->status_detail());
}
print"RIR organization added successfully \n";
```

#Search for RIR organization object

```
my @retrieved_objs = $session->search(
   object => 'Infoblox::Grid::RIR::Organization',
   id     => 'RIR_ORG_ID',
);

$rir_org = $retrieved_objs[0];

unless ($rir_org) {
        die("Search RIR organization object failed: ",
                $session->status_code() . ":" . $session->status_detail());
}
print "Search RIR organization object found at least 1 matching entry\n";
```

#Get and modify RIR organization object

```
@retrieved_objs = $session->get(
   object => 'Infoblox::Grid::RIR::Organization',
   id     => 'RIR_ORG_ID',
);

$rir_org = $retrieved_objs[0];

unless ($rir_org) {
    die("Get RIR organization object failed: ",
            $session->status_code() . ":" . $session->status_detail());
}
print"Get RIR organization successful \n";

$rir_org->maintainer("admin");

$session->modify($rir_org)
   or die("Modify RIR organization failed",
           $session->status_code() . ":" . $session->status_detail());
print "RIR organization modified successfully \n";
```

#Remove RIR organization object

```
@retrieved_objs = $session->get(
   object => 'Infoblox::Grid::RIR::Organization',
   id     => 'RIR_ORG_ID',
);

$rir_org = $retrieved_objs[0];

unless ($rir_org) {
    die("Get RIR organization object failed: ",
            $session->status_code() . ":" . $session->status_detail());
}
print"Get RIR organization successful \n";

$session->remove( $rir_org )
    or die("Remove RIR organization failed",
```

```
            $session->status_code() . ":" . $session->status_detail());
print"RIR organization removed successfully \n";

####PROGRAM ENDS####
```

AUTHOR

Infoblox Inc. http://www.infoblox.com/

SEE ALSO

Infoblox::Session, Infoblox::Grid::RIR, Infoblox::Grid::ExtensibleAttributeDef/Extensible Attribute Values, Infoblox::Grid->enable_rir_swip()

COPYRIGHT

Copyright (c) 2017 Infoblox Inc.

Infoblox::Grid - Manages the Grid settings.

- NAME
- DESCRIPTION
- CONSTRUCTOR
- SESSION METHODS
  - Infoblox::Session->get( )
  - Infoblox::Session->search( )
  - Infoblox::Session->modify( )
- METHODS
  - admin_access_items( )
  - allow_recursive_deletion( )
  - admin_access_remote_console( )
  - audit_log_format( )
  - alternate_resolver( )
  - consent_banner_setting( )
  - copy_audit_to_syslog( )
  - descendants_action( )
  - deny_mgm_snapshots( )
  - dscp( )
  - enable_login_banner( )
  - enable_lom( )
  - enable_gui_api_for_lan_vip( )
  - enable_member_redirect( )
  - enable_ntp( )
  - enable_recycle_bin( )
  - enable_rir_swip( )
  - enable_snmpv3_query( )
  - enable_snmpv3_traps( )
  - enable_security_access( )
  - external_syslog_server_enable( )
  - http_proxy_server_setting( )
  - informational_banner_setting( )
  - is_grid_visualization_visible( )
  - lcd_input( )
  - login_banner_text( )
  - lom_users( )
  - mgm_strict_delegate_mode( )
  - ms_default_ip_site_link( )
  - ms_enable_invalid_mac( )
  - ms_enable_dhcp_monitoring( )
  - ms_enable_dns_monitoring( )
  - ms_enable_dns_reports_sync( )
  - ms_enable_network_users( )
  - ms_enable_user_sync( )
  - ms_ldap_timeout( )
  - ms_log_destination( )
  - ms_max_connection( )
  - ms_rpc_timeout( )
  - ms_user_default_timeout( )
  - name( )
  - nat_group_list( )
  - notification_email_addr( )
  - ntp_server( )
  - ntp_access_list( )
  - ntp_service_type( )
  - ntp_authentication_key( )
  - ntp_kod( )
  - objects_changes_tracking_setting( )
  - password_chars_to_change( )
  - password_expire_days( )
  - password_expire_enable( )
  - password_force_reset( )
  - password_min_length( )
  - password_num_lower_char( )
  - password_num_numeric_char( )
  - password_num_symbol_char( )
  - password_num_upper_char( )
  - password_reminder_days( )
  - prefer_resolver( )
  - query_comm_string ( )
  - remote_console_access( )
  - restart_banner_setting( )

- restart_status( )
- rpz_hit_rate_interval( )
- rpz_hit_rate_min_query( )
- rpz_hit_rate_max_query( )
- search_domains( )
- secret( )
- service_status( )
- support_access( )
- support_access_info( )
- scheduled_backup( )
- session_timeout( )
- snmp_admin( )
- snmpv3_query_users( )
- syslog_backup_servers( )
- syslog_server( )
- syslog_size( )
- threshold_traps( )
- token_usage_delay( )
- trap_comm_string( )
- trap_notifications( )
- trap_receiver( )
- updates_download_member_config( )
- vpn_port( )

- SAMPLE CODE
- AUTHOR
- SEE ALSO
- COPYRIGHT

NAME

Infoblox::Grid - Manages the Grid settings.

DESCRIPTION

This class manages the database that is distributed among all grid members. The grid object is not intended to be instantiated, it should only be used to modify information.

A grid is a group of two or more Infoblox appliances that share sections of a common, distributed, built-in database and also, are connected together to provide a single point of administration and service configuration in a secure, highly available environment.

CONSTRUCTOR

The Infoblox::Grid object is automatically generated upon the successful installation of the Infoblox appliance. It does not require manual construction.

SESSION METHODS

This section describes all the methods in Infoblox::Session that you can apply to a grid object.

Infoblox::Session->get( )

Use this method to retrieve the existing objects from the Infoblox appliance. See Infoblox::Session->get() for parameters and return values.

Example

```
my $grid = $session->get(
    object => "Infoblox::Grid");
```

Infoblox::Session->search( )

Use this method to search for the existing objects from the Infoblox appliance. See Infoblox::Session->search() for parameters and return values.

Example

```
my $grid = $session->search(
    object => "Infoblox::Grid");
```

Infoblox::Session->modify( )

Use this method to modify objects in the Infoblox appliance. See Infoblox::Session->modify() for parameters and return values.

If you have applied an access control list to an object, you must restart services after you update the access control list for the changes to take effect. See Infoblox::Session->restart() method.

Example

```
# Use method to modify the name
$grid->name("Infoblox2");
# Submit modification
my $response = $session->modify( $grid );
```

METHODS

This section describes all the methods that you can use to configure and retrieve the attribute values of a grid.

admin_access_items( )

Use this method to set or retrieve the security access items.

Parameter

Valid value is an Infoblox::Grid::NamedACL object or an array reference that contains IPv4/IPv6 addresses or networks.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get admin_access_items
my $admin_access_items = $grid->admin_access_items();
#Modify admin_access_items
$grid->admin_access_items(["10.10.10.1", "10.10.2.0/24", "2001::1"]);
#Use Infoblox::Grid::NamedACL
$grid->admin_access_items($nacl);
```

allow_recursive_deletion( )

Use this method to set or retrieve the allow_recursive_deletion property.

Parameter

Valid values are "NOBODY", "SUPERUSERS" and "ALL". Default value is "ALL".

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get allow_recursive_deletion
my $allow_recursive_deletion = $grid->allow_recursive_deletion();
#Modify allow_recursive_deletion
$grid->allow_recursive_deletion("SUPERUSERS");
```

admin_access_remote_console( )

Use this method to set or retrieve the flag that enables or disables access control for remote console access.

Parameter

Valid values are "true" or "false".

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get admin_access_remote_console
my $admin_access_remote_console = $obj->admin_access_remote_console();
```

```
#Modify admin_access_remote_console
$obj->admin_access_remote_console("false");
```

audit_log_format( )

Use this method to set or retrieve the audit log format.

Parameter

Valid values are "detailed" or "brief". Default value is "detailed".

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get audit_log_format
my $audit_log_format = $grid->audit_log_format();
#Modify audit_log_format
$grid->audit_log_format("brief");
```

alternate_resolver( )

Use this method to set or retrieve the alternate resolver.

Parameter

The valid values are Infoblox::DNS::Member object or Infoblox::DNS::Nameserver object.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

Example

```
#Get alternate_resolver
my $alternate = $grid->alternate_resolver();
#Modify alternate_resolver
$grid->alternate_resolver("4.4.4.4");
```

consent_banner_setting( )

Use this method to set or retrieve the Grid consent banner settings.

Parameter

An Infoblox::Grid::ConsentBannerSetting object.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

Example

```
#Enable the consent banner
my $banner = Infoblox::Grid::ConsentBannerSetting->new(enable => 'true', message => 'Consent
message');
$grid->consent_banner_setting($banner);
```

copy_audit_to_syslog( )

Use this method to set or retrieve the flag that enables the appliance to copy audit log messages to syslog.

Parameter

Valid values are "true" or "false".

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get copy_audit_to_syslog
my $copy_audit_to_syslog = $obj->copy_audit_to_syslog();
#Modify copy_audit_to_syslog
$obj->copy_audit_to_syslog("true");
```

descendants_action( )

Use this method to set or retrieve the default actions for extensbile attributes that exist on descendants.

Parameter

Valid value is an Infoblox::Grid::ExtensibleAttributeDef::Descendants object.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get attribute value
my $descendants_action = $grid->descendants_action();
#Modify attribute value
$grid->descendants_action($descendants);
```

deny_mgm_snapshots( )

Use this method to set or retrieve the flag that controls whether the subgrid should send snapshots to MGM or not.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Paramter

Specify 'true' to allow sending snapshots to MGM by subgrid and 'false' to forbid it.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get deny_mgm_snapshots value
my $value = $object->deny_mgm_snapshots();

#Modify deny_mgm_snapshots value
$object->deny_mgm_snapshots('true');
```

dscp( )

Use this method to set or retrieve the DSCP value.

Parameter

Valid values are integers between 0 and 63 inclusive. The default value is 0.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get dscp
my $dscp = $grid->dscp();
#Modify dscp
$grid->dscp(12);
```

enable_login_banner( )

Use this method to enable or disable the login banner.

Parameter

Specify "true" to enable the login banner or "false" to disable it. The default value is "true".

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get enable_login_banner
my $enable_login_banner = $grid->enable_login_banner();
#Modify enable_login_banner
$grid->enable_login_banner("false");
```

enable_lom( )

Use this method to set or retrieve the flag that controls whether the LOM functionality is enabled.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

Specify "true" to set the enable_lom flag or "false" to deactivate/unset it. The default value is "false".

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get enable_lom from the grid
my $lom = $grid->enable_lom();
#Modify enable_lom
$grid->enable_lom("true");
```

enable_gui_api_for_lan_vip( )

Use this method to enable or disable GUI and API access on the LAN/VIP port, in addition to the MGMT port when it is configured.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Paramter

Specify 'true' to enable GUI and API access on the LAN/VIP port or 'false' to disable it.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get enable_gui_api_for_lan_vip
my $enable_gui_api_for_lan_vip = $grid->enable_gui_api_for_lan_vip();

#Modify enable_gui_api_for_lan_vip
$grid->enable_gui_api_for_lan_vip('true');
```

enable_member_redirect( )

Use this method to set or retrieve the flag that specifies if GUI redirection is enabled for members.

Parameter

Valid values are "true" or "false". Default value is "true".

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get enable_member_redirect
my $enable_member_redirect = $obj->enable_member_redirect();
#Modify enable_member_redirect
```

```
$obj->enable_member_redirect("true");
```

enable_ntp( )

Use this method to enable/disable NTP (Network Time Protocol).

Parameter

Specify "true" to enable NTP or "false" to disable it. Default value is "false".

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

Example

```
#Get ntp
my $ntp = $grid->enable_ntp();
#Modify NTP
$grid->enable_ntp ("true");
```

enable_recycle_bin( )

Use this method to enable/disable the recycle bin.

When the recycle bin is enabled, deleted DNS data (Zone,View) and DHCP data (Network,Network container,Shared network,DHCP range,Fixed address,DHCP MAC filter) is moved to the recycle bin and can be restored later.

Parameter

Specify "true" to enable the recycle bin or "false" to disable it. Default value is "true".

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

Example

```
#Get enable_recycle_bin
my $recycle_bin = $grid->enable_recycle_bin();
#Modify enable_recycle_bin
$grid->enable_recycle_bin ("false");
```

enable_rir_swip( )

Use this method to enable or diasble support for RIR/SWIP.

The following specific extensible attribute definitions with the 'RIPE' namespace will be added when support for RIR/SWIP is enabled for the first time:

```
 +-------------------------------------------------------------------------------------------------------------------------------+
 | RIPE Network Name      - The name of the network that contains the RIR assignment, in string format.
|
 |                          Required for Infoblox::DHCP::Network, Infoblox::DHCP::IPv6Network,
Infoblox::DHCP::NetworkContainer,       |
 |                          Infoblox::DHCP::IPv6NetworkContainer, Infoblox::DHCP::NetworkTemplate,
Infoblox::DHCP::IPv6NetworkTemplate  |
 |                          objects if rir_organization member is defined. Otherwise this is optional RIR
attribute for these objects.  |
 | RIPE Description       - The short description related to the object in string format.
|
 |                          Required for Infoblox::DHCP::Network, Infoblox::DHCP::IPv6Network,
Infoblox::DHCP::NetworkContainer,       |
 |                          Infoblox::DHCP::IPv6NetworkContainer, Infoblox::DHCP::NetworkTemplate,
Infoblox::DHCP::IPv6NetworkTemplate  |
 |                          objects if rir_organization member is defined. Otherwise this is optional RIR
attribute for these objects.  |
 |                          Optional for Infoblox::Grid::RIR::Organization object.
|
 | RIPE Country           - The country name with the two-letter ISO 3166 country code in string format,
e.g. 'Afghanistan (AF)'.      |
 |                          Valid values you can see in the next table.
|
 |                          Required for Infoblox::DHCP::Network, Infoblox::DHCP::IPv6Network,
Infoblox::DHCP::NetworkContainer,       |
 |                          Infoblox::DHCP::IPv6NetworkContainer, Infoblox::DHCP::NetworkTemplate,
Infoblox::DHCP::IPv6NetworkTemplate  |
 |                          objects if rir_organization member is defined. Otherwise this is optional RIR
attribute for these objects.  |
 |                          Required for Infoblox::Grid::RIR::Organization object.
|
 | RIPE Admin Contact     - The admin contact in string format.
|
```

| | Required for Infoblox::DHCP::Network, Infoblox::DHCP::IPv6Network, Infoblox::DHCP::NetworkContainer, | | Infoblox::DHCP::IPv6NetworkContainer, Infoblox::DHCP::NetworkTemplate, Infoblox::DHCP::IPv6NetworkTemplate | | objects if rir_organization member is defined. Otherwise this is optional RIR attribute for these objects. | | Required for Infoblox::Grid::RIR::Organization object.

| RIPE Technical Contact - The technical contact in string format.

| | Required for Infoblox::DHCP::Network, Infoblox::DHCP::IPv6Network, Infoblox::DHCP::NetworkContainer, | | Infoblox::DHCP::IPv6NetworkContainer, Infoblox::DHCP::NetworkTemplate, Infoblox::DHCP::IPv6NetworkTemplate | | objects if rir_organization member is defined. Otherwise this is optional RIR attribute for these objects. | | Required for Infoblox::Grid::RIR::Organization object.

| RIPE IPv4 Status - The status of the address range represented by inetnum object that can be 'ALLOCATED PA', | | 'ALLOCATED PI', 'ALLOCATED UNSPECIFIED', 'LIR-PARTITIONED PA', 'LIR-PARTITIONED PI', 'SUB-ALLOCATED PA', | | 'ASSIGNED PA', 'ASSIGNED PI', 'ASSIGNED ANYCAST', 'EARLY-REGISTRATION' or 'NOT-SET'. | | Required for Infoblox::DHCP::Network, Infoblox::DHCP::NetworkContainer and Infoblox::DHCP::NetworkTemplate | | objects if rir_organization member is defined. Otherwise this is optional RIR attribute for these objects. |

| RIPE IPv6 Status - The status of the address range represented by inet6num object that can be 'ALLOCATED-BY-RIR', | | 'ALLOCATED-BY-LIR', 'ASSIGNED', 'ASSIGNED ANYCAST' or 'ASSIGNED PI'.

| | Required for Infoblox::DHCP::IPv6Network, Infoblox::DHCP::IPv6NetworkContainer and | | Infoblox::DHCP::IPv6NetworkTemplate objects if rir_organization member is defined. Otherwise this is | | optional RIR attribute for these objects.

| RIPE Remarks - Some freeform remarks in string format.

| | Required for Infoblox::DHCP::Network, Infoblox::DHCP::IPv6Network, Infoblox::DHCP::NetworkContainer, | | Infoblox::DHCP::IPv6NetworkContainer, Infoblox::DHCP::NetworkTemplate, Infoblox::DHCP::IPv6NetworkTemplate | | objects if rir_organization member is defined. Otherwise this is optional RIR attribute for these objects. | | Optional for Infoblox::Grid::RIR::Organization object.

| RIPE Notify - The Email address to which notifications must be sent for any updates in string format. | | Optional for Infoblox::DHCP::Network, Infoblox::DHCP::IPv6Network, Infoblox::DHCP::NetworkContainer, | | Infoblox::DHCP::IPv6NetworkContainer, Infoblox::DHCP::NetworkTemplate, Infoblox::DHCP::IPv6NetworkTemplate, | | Infoblox::Grid::RIR::Organization objects.

| RIPE Lower Level Maintainer

| | - The identifier of a registerd maintainer object used for hierarchical authorization, in string format. | | Optional for Infoblox::DHCP::Network, Infoblox::DHCP::IPv6Network, Infoblox::DHCP::NetworkContainer, | | Infoblox::DHCP::IPv6NetworkContainer, Infoblox::DHCP::NetworkTemplate, Infoblox::DHCP::IPv6NetworkTemplate | | objects.

| RIPE Reverse Domain Maintainer

| | - The identifier of a registered maintainer object used for reverse domain authorization, in string format. | | Optional for Infoblox::DHCP::Network, Infoblox::DHCP::IPv6Network, Infoblox::DHCP::NetworkContainer, | | Infoblox::DHCP::IPv6NetworkContainer, Infoblox::DHCP::NetworkTemplate, Infoblox::DHCP::IPv6NetworkTemplate | | objects.

| RIPE Routes Maintainer - The identifier of a registered maintainer object that contols the authorization of the creation of route | | and route6 objects, in string format.

| | Optional for Infoblox::DHCP::Network, Infoblox::DHCP::IPv6Network, Infoblox::DHCP::NetworkContainer, | | Infoblox::DHCP::IPv6NetworkContainer, Infoblox::DHCP::NetworkTemplate, Infoblox::DHCP::IPv6NetworkTemplate | | objects.

| RIPE Computer Security Incident Response Team

| | - The point to an irt object representing a Computer Security Incident Response Team (CSIRT) that handles | | security incidents for the address space specified by the inetnum or inet6num object in string format. | | Optional for Infoblox::DHCP::Network, Infoblox::DHCP::IPv6Network, Infoblox::DHCP::NetworkContainer, | | Infoblox::DHCP::IPv6NetworkContainer, Infoblox::DHCP::NetworkTemplate, Infoblox::DHCP::IPv6NetworkTemplate | | objects.

RIPE Registry Source	The registry where the object is registered. This can be 'RIPE' or "TEST". Required for Infoblox::DHCP::Network, Infoblox::DHCP::IPv6Network, Infoblox::DHCP::NetworkContainer, Infoblox::DHCP::IPv6NetworkContainer, Infoblox::DHCP::NetworkTemplate, Infoblox::DHCP::IPv6NetworkTemplate objects if rir_organization member is defined. Otherwise this is optional RIR attribute for these objects. Required for Infoblox::Grid::RIR::Organization object.
RIPE Organization Type	The organization type. This can be 'IANA','RIR','NIR','LIR','WHITEPAGES','DIRECT ASSIGNMENT', or 'OTHER'. Optional for Infoblox::Grid::RIR::Organization object.
RIPE Address	The organization address in string format. Optional for Infoblox::Grid::RIR::Organization object.
RIPE Phone Number	The organization phone number in string format. Optional for Infoblox::Grid::RIR::Organization object.
RIPE Fax Number	The organization fax number in string format. Optional for Infoblox::Grid::RIR::Organization object.
RIPE Email	The organization email address in string format. Required for Infoblox::Grid::RIR::Organization object.
RIPE Abuse Mailbox	The email address to which abuse complaints should be sent in string format. Optional for Infoblox::Grid::RIR::Organization object.
RIPE Reference Notify	The email address to be notified when a reference to the organization object is added or removed, in string format. Optional for Infoblox::Grid::RIR::Organization object.

The following values are valid for 'RIPE Country' attribute:

Code	Country	Value
AF	Afghanistan	Afghanistan (AF)
AX	Aland Islands	Aland Islands (AX)
AL	Albania	Albania (AL)
DZ	Algeria	Algeria (DZ)
AS	American Samoa	American Samoa (AS)
AD	Andorra	Andorra (AD)
AO	Angola	Angola (AO)
AI	Anguilla	Anguilla (AI)
AQ	Antarctica	Antarctica (AQ)
AG	Antigua and Barbuda	Antigua and Barbuda (AG)
AR	Argentina	Argentina (AR)
AM	Armenia	Armenia (AM)
AW	Aruba	Aruba (AW)
AU	Australia	Australia (AU)
AT	Austria	Austria (AT)
AZ	Azerbaijan	Azerbaijan (AZ)
BS	Bahamas	Bahamas (BS)
BH	Bahrain	Bahrain (BH)
BD	Bangladesh	Bangladesh (BD)
BB	Barbados	Barbados (BB)
BY	Belarus	Belarus (BY)
BE	Belgium	Belgium (BE)

BZ	Belize	Belize (BZ)
BJ	Benin	Benin (BJ)
BM	Bermuda	Bermuda (BM)
BT	Bhutan	Bhutan (BT)
BO	Bolivia, Plurinational State of	Bolivia, Plurinational State of (BO)
BQ	Bonaire, Sint Eustatius and Saba	Bonaire, Sint Eustatius and Saba (BQ)
BA	Bosnia and Herzegovina	Bosnia and Herzegovina (BA)
BW	Botswana	Botswana (BW)
BV	Bouvet Island	Bouvet Island (BV)
BR	Brazil	Brazil (BR)
IO	British Indian Ocean Territory	British Indian Ocean Territory (IO)
BN	Brunei Darussalam	Brunei Darussalam (BN)
BG	Bulgaria	Bulgaria (BG)
BF	Burkina Faso	Burkina Faso (BF)
BI	Burundi	Burundi (BI)
KH	Cambodia	Cambodia (KH)
CM	Cameroon	Cameroon (CM)
CA	Canada	Canada (CA)
CV	Cape Verde	Cape Verde (CV)
KY	Cayman Islands	Cayman Islands (KY)
CF	Central African Republic	Central African Republic (CF)
TD	Chad	Chad (TD)
CL	Chile	Chile (CL)
CN	China	China (CN)
CX	Christmas Island	Christmas Island (CX)
CC	Cocos (Keeling) Islands	Cocos (Keeling) Islands (CC)
CO	Colombia	Colombia (CO)
KM	Comoros	Comoros (KM)
CG	Congo	Congo (CG)
CD	Congo, The Democratic Republic of the	Congo, The Democratic Republic of the (CD)
CK	Cook Islands	Cook Islands (CK)
CR	Costa Rica	Costa Rica (CR)
CI	Côte d'Ivoire	Côte d'Ivoire (CI)
HR	Croatia	Croatia (HR)
CU	Cuba	Cuba (CU)
CW	Curaçao	Curaçao (CW)
CY	Cyprus	Cyprus (CY)
CZ	Czech Republic	Czech Republic (CZ)
DK	Denmark	Denmark (DK)
DJ	Djibouti	Djibouti (DJ)
DM	Dominica	Dominica (DM)
DO	Dominican Republic	Dominican Republic (DO)
EC	Ecuador	Ecuador (EC)
EG	Egypt	Egypt (EG)
SV	El Salvador	El Salvador (SV)
GQ	Equatorial Guinea	Equatorial Guinea (GQ)
ER	Eritrea	Eritrea (ER)
EE	Estonia	Estonia (EE)
ET	Ethiopia	Ethiopia (ET)

FK	Falkland Islands (Malvinas)	Falkland Islands (Malvinas) (FK)
FO	Faroe Islands	Faroe Islands (FO)
FJ	Fiji	Fiji (FJ)
FI	Finland	Finland (FI)
FR	France	France (FR)
GF	French Guiana	French Guiana (GF)
PF	French Polynesia	French Polynesia (PF)
TF	French Southern Territories	French Southern Territories (TF)
GA	Gabon	Gabon (GA)
GM	Gambia	Gambia (GM)
GE	Georgia	Georgia (GE)
DE	Germany	Germany (DE)
GH	Ghana	Ghana (GH)
GI	Gibraltar	Gibraltar (GI)
GR	Greece	Greece (GR)
GL	Greenland	Greenland (GL)
GD	Grenada	Grenada (GD)
GP	Guadeloupe	Guadeloupe (GP)
GU	Guam	Guam (GU)
GT	Guatemala	Guatemala (GT)
GG	Guernsey	Guernsey (GG)
GN	Guinea	Guinea (GN)
GW	Guinea-Bissau	Guinea-Bissau (GW)
GY	Guyana	Guyana (GY)
HT	Haiti	Haiti (HT)
HM	Heard Island and McDonald Islands	Heard Island and McDonald Islands (HM)
VA	Holy See (Vatican City State)	Holy See (Vatican City State) (VA)
HN	Honduras	Honduras (HN)
HK	Hong Kong	Hong Kong (HK)
HU	Hungary	Hungary (HU)
IS	Iceland	Iceland (IS)
IN	India	India (IN)
ID	Indonesia	Indonesia (ID)
IR	Iran, Islamic Republic of	Iran, Islamic Republic of (IR)
IQ	Iraq	Iraq (IQ)
IE	Ireland	Ireland (IE)
IM	Isle of Man	Isle of Man (IM)
IL	Israel	Israel (IL)
IT	Italy	Italy (IT)
JM	Jamaica	Jamaica (JM)
JP	Japan	Japan (JP)
JE	Jersey	Jersey (JE)
JO	Jordan	Jordan (JO)
KZ	Kazakhstan	Kazakhstan (KZ)
KE	Kenya	Kenya (KE)
KI	Kiribati	Kiribati (KI)
KP	Korea, Democratic People's Republic of	Korea, Democratic People's Republic of (KP)
KR	Korea, Republic of	Korea, Republic of (KR)

KW	Kuwait	Kuwait (KW)
KG	Kyrgyzstan	Kyrgyzstan (KG)
LA	Lao People's Democratic Republic	Lao People's Democratic Republic (LA)
LV	Latvia	Latvia (LV)
LB	Lebanon	Lebanon (LB)
LS	Lesotho	Lesotho (LS)
LR	Liberia	Liberia (LR)
LY	Libya	Libya (LY)
LI	Liechtenstein	Liechtenstein (LI)
LT	Lithuania	Lithuania (LT)
LU	Luxembourg	Luxembourg (LU)
MO	Macao	Macao (MO)
MK	Macedonia, The Former Yugoslav Republic of (MK)	Macedonia, The Former Yugoslav Republic of (MK)
MG	Madagascar	Madagascar (MG)
MW	Malawi	Malawi (MW)
MY	Malaysia	Malaysia (MY)
MV	Maldives	Maldives (MV)
ML	Mali	Mali (ML)
MT	Malta	Malta (MT)
MH	Marshall Islands	Marshall Islands (MH)
MQ	Martinique	Martinique (MQ)
MR	Mauritania	Mauritania (MR)
MU	Mauritius	Mauritius (MU)
YT	Mayotte	Mayotte (YT)
MX	Mexico	Mexico (MX)
FM	Micronesia, Federated States of	Micronesia, Federated States of (FM)
MD	Moldova, Republic of	Moldova, Republic of (MD)
MC	Monaco	Monaco (MC)
MN	Mongolia	Mongolia (MN)
ME	Montenegro	Montenegro (ME)
MS	Montserrat	Montserrat (MS)
MA	Morocco	Morocco (MA)
MZ	Mozambique	Mozambique (MZ)
MM	Myanmar	Myanmar (MM)
NA	Namibia	Namibia (NA)
NR	Nauru	Nauru (NR)
NP	Nepal	Nepal (NP)
NL	Netherlands	Netherlands (NL)
NC	New Caledonia	New Caledonia (NC)
NZ	New Zealand	New Zealand (NZ)
NI	Nicaragua	Nicaragua (NI)
NE	Niger	Niger (NE)
NG	Nigeria	Nigeria (NG)
NU	Niue	Niue (NU)
NF	Norfolk Island	Norfolk Island (NF)
MP	Northern Mariana Islands	Northern Mariana Islands (MP)
NO	Norway	Norway (NO)
OM	Oman	Oman (OM)
PK	Pakistan	Pakistan (PK)

PW	Palau	Palau (PW)
PS	Palestinian Territory, Occupied	Palestinian Territory, Occupied (PS)
PA	Panama	Panama (PA)
PG	Papua New Guinea	Papua New Guinea (PG)
PY	Paraguay	Paraguay (PY)
PE	Peru	Peru (PE)
PH	Philippines	Philippines (PH)
PN	Pitcairn	Pitcairn (PN)
PL	Poland	Poland (PL)
PT	Portugal	Portugal (PT)
PR	Puerto Rico	Puerto Rico (PR)
QA	Qatar	Qatar (QA)
RE	Réunion	Réunion (RE)
RO	Romania	Romania (RO)
RU	Russian Federation	Russian Federation (RU)
RW	Rwanda	Rwanda (RW)
BL	Saint Barthélemy	Saint Barthélemy (BL)
SH	Saint Helena, Ascension and Tristan da Cunha	Saint Helena, Ascension and Tristan da Cunha (SH)
KN	Saint Kitts and Nevis	Saint Kitts and Nevis (KN)
LC	Saint Lucia	Saint Lucia (LC)
MF	Saint Martin (French part)	Saint Martin (French part) (MF)
PM	Saint Pierre and Miquelon	Saint Pierre and Miquelon (PM)
VC	Saint Vincent and the Grenadines	Saint Vincent and the Grenadines (VC)
WS	Samoa	Samoa (WS)
SM	San Marino	San Marino (SM)
ST	Sao Tome and Principe	Sao Tome and Principe (ST)
SA	Saudi Arabia	Saudi Arabia (SA)
SN	Senegal	Senegal (SN)
RS	Serbia	Serbia (RS)
SC	Seychelles	Seychelles (SC)
SL	Sierra Leone	Sierra Leone (SL)
SG	Singapore	Singapore (SG)
SX	Sint Maarten (Dutch part)	Sint Maarten (Dutch part) (SX)
SK	Slovakia	Slovakia (SK)
SI	Slovenia	Slovenia (SI)
SB	Solomon Islands	Solomon Islands (SB)
SO	Somalia	Somalia (SO)
ZA	South Africa	South Africa (ZA)
GS	South Georgia and the South Sandwich Islands	South Georgia and the South Sandwich Islands (GS)
SS	South Sudan	South Sudan (SS)
ES	Spain	Spain (ES)
LK	Sri Lanka	Sri Lanka (LK)
SD	Sudan	Sudan (SD)
SR	Suriname	Suriname (SR)
SJ	Svalbard and Jan Mayen	Svalbard and Jan Mayen (SJ)
SZ	Swaziland	Swaziland (SZ)
SE	Sweden	Sweden (SE)
CH	Switzerland	Switzerland (CH)

```
|  SY  | Syrian Arab Republic                                | Syrian Arab Republic (SY)
|
|  TW  | Taiwan, Province of China                           | Taiwan, Province of China (TW)
|
|  TJ  | Tajikistan                                          | Tajikistan (TJ)
|
|  TZ  | Tanzania, United Republic of                        | Tanzania, United Republic of (TZ)
|
|  TH  | Thailand                                            | Thailand (TH)
|
|  TL  | Timor-Leste                                         | Timor-Leste (TL)
|
|  TG  | Togo                                                | Togo (TG)
|
|  TK  | Tokelau                                             | Tokelau (TK)
|
|  TO  | Tonga                                               | Tonga (TO)
|
|  TT  | Trinidad and Tobago                                 | Trinidad and Tobago (TT)
|
|  TN  | Tunisia                                             | Tunisia (TN)
|
|  TR  | Turkey                                              | Turkey (TR)
|
|  TM  | Turkmenistan                                        | Turkmenistan (TM)
|
|  TC  | Turks and Caicos Islands                            | Turks and Caicos Islands (TC)
|
|  TV  | Tuvalu                                              | Tuvalu (TV)
|
|  UG  | Uganda                                              | Uganda (UG)
|
|  UA  | Ukraine                                             | Ukraine (UA)
|
|  AE  | United Arab Emirates                                | United Arab Emirates (AE)
|
|  GB  | United Kingdom                                      | United Kingdom (GB)
|
|  US  | United States                                       | United States (US)
|
|  UM  | United States Minor Outlying Islands                | United States Minor Outlying Islands (UM)
|
|  UY  | Uruguay                                             | Uruguay (UY)
|
|  UZ  | Uzbekistan                                          | Uzbekistan (UZ)
|
|  VU  | Vanuatu                                             | Vanuatu (VU)
|
|  VE  | Venezuela, Bolivarian Republic of                   | Venezuela, Bolivarian Republic of (VE)
|
|  VN  | Viet Nam                                            | Viet Nam (VN)
|
|  VG  | Virgin Islands, British                             | Virgin Islands, British (VG)
|
|  VI  | Virgin Islands, U.S.                                | Virgin Islands, U.S. (VI)
|
|  WF  | Wallis and Futuna                                   | Wallis and Futuna (WF)
|
|  EH  | Western Sahara                                      | Western Sahara (EH)
|
|  YE  | Yemen                                               | Yemen (YE)
|
|  ZM  | Zambia                                              | Zambia (ZM)
|
|  ZW  | Zimbabwe                                            | Zimbabwe (ZW)
|
 +------+-----------------------------------------------------+-------------------------------------------
---------+
```

Parameter

Specify "true" to enable the RIR SWIP feature or "false" to disable it. Default value is "false".

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get enable_rir_swip
my $enable_rir_swip = $grid->enable_rir_swip();
#Modify enable_rir_swip
$grid->enable_rir_swip("true");
```

enable_snmpv3_query( )

Use this method to enable or disable SNMPv3 queries.

Parameter

Specify "true" to enable SNMPv3 queries or "false" to disable it.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

Example

```
#Get enable_snmpv3_query
my $enable_snmpv3_query = $grid->enable_snmpv3_query();
#Modify enable_snmpv3_query
$grid->enable_snmpv3_query("false");
```

enable_snmpv3_traps( )

Use this method to enable or disable SNMPv3 traps.

Parameter

Specify "true" to enable SNMPv3 traps or "false" to disable it. Default value is "false".

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

Example

```
#Get enable_snmpv3_traps
my $enable_snmpv3_traps = $grid->enable_snmpv3_traps();
#Modify enable_snmpv3_traps
$grid->enable_snmpv3_traps("true");
```

enable_security_access( )

Use this method to enable or disable the security access.

Parameter

Specify "true" to enable the security access or "false" to disable it. Default value is "false".

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get enable_security_access
my $enable_security_access = $grid->enable_security_access();
#Modify enable_security_access
$grid->enable_security_access("true");
```

external_syslog_server_enable( )

Use this method to set or retrieve the flag that indicates whether external syslog servers are enabled.

The external_syslog_server_enable attribute can be specified explicitly. It is also set implicitly when syslog_server is set to a defined value.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

Specify 'true' to enable external syslog servers and 'false' to disable them.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get external_syslog_server_enable value
my $value = $object->external_syslog_server_enable();

#Modify external_syslog_server_enable value
$object->external_syslog_server_enable('true');
```

http_proxy_server_setting( )

Use this method to set or retreive the Grid HTTP proxy server settings.

Parameter

The valid value is an Infoblox::Grid::HTTPProxyServerSetting object.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

Example

```
#Get http_proxy_server_setting value
my $http_proxy_server_setting = $grid->http_proxy_server_setting();

#Modify http_proxy_server_setting value
$grid->http_proxy_server_setting($http_proxy_setting);
```

informational_banner_setting( )

Use this method to set or retrieve the Grid informational banner settings.

Parameter

An Infoblox::Grid::InformationalBannerSetting object.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

Example

```
#Enable the informational banner
my $banner = Infoblox::Grid::InformationalBannerSetting->new(enable => 'true', message =>
'Informational message', color = > 'GREEN');
$grid->informational_banner_setting($banner);
```

is_grid_visualization_visible( )

Use this method to enable or disable graphical visualization of the Grid. The default value is 'false'.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

Specify 'true' to enable graphical visualization of the Grid or 'false' to disable it.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get is_grid_visualization_visible
my $is_grid_visualization_visible = $grid->is_grid_visualization_visible();

#Modify is_grid_visualization_visible
$grid->is_grid_visualization_visible('true');
```

lcd_input( )

Use this method to enable/disable LCD (Liquid Crystal Display) input. Default value is "true".

Parameter

Specify "true" to enable LCD input or "false" to disable it.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

Example

```
#Get lcd_input
```

```
my $lcd = $grid->lcd_input();
#Modify lcd_input
$grid->lcd_input("false");
```

login_banner_text( )

Use this method to set or retrieve the login banner text.

Parameter

The login banner text.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get login_banner_text
my $login_banner_text = $grid->login_banner_text();
#Modify login_banner_text
$grid->login_banner_text('Your login banner text');
```

lom_users( )

Use this method to set or retrieve the list of users for the LOM functionality.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

Valid value is an array reference that contains Infoblox::Grid::LOM::User objects.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get lom_users from the grid
my $users = $grid->lom_users();
#Modify the lom users
$grid->lom_users([$u1, $u2]);
```

mgm_strict_delegate_mode( )

Use this method to enable/disable strict delegate mode for a grid managed by the Master Grid.

Parameter

Specify "true" to enable strict delegate mode or "false" to disable it. Default value is "false".

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

Example

```
#Get mgm_strict_delegate_mode
my $delegated = $grid->mgm_strict_delegate_mode();
#Modify mgm_strict_delegate_mode
$grid->mgm_strict_delegate_mode ("true");
```

ms_default_ip_site_link( )

Use this method set or retrieve the default IP site link for sites created from NIOS for all Microsoft servers.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

Valid value is a string with the default IP site link name.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get ms_default_ip_site_link value
my $default_ip_site_link = $grid->ms_default_ip_site_link();
#Modify ms_default_ip_site_link value
$grid->ms_default_ip_site_link('Site1');
```

ms_enable_invalid_mac( )

Use this method to allow invalid MAC address synchronization for DHCP leases and fixed addresses.

Parameter

Specify "true" to allow invalid MAC address synchronization or "false" to forbid it. Default value is "false".

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get ms_enable_invalid_mac value
my $enable_invalid_mac = $grid->ms_enable_invalid_mac();
#Modify ms_enable_invalid_mac value
$grid->ms_enable_invalid_mac("true");
```

ms_enable_dhcp_monitoring( )

Use this method to enable or disable the monitoring and control of DHCP service on all Microsoft servers in the Grid.

Parameter

Specify "true" to enable monitoring and control of DHCP service on all Microsoft servers in the Grid and "false" to forbid it. Default value is "false".

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get ms_enable_dhcp_monitoring value
my $enable_dhcp_monitoring = $grid->ms_enable_dhcp_monitoring();
#Modify ms_enable_dhcp_monitoring value
$grid->ms_enable_dhcp_monitoring("true");
```

ms_enable_dns_monitoring( )

Use this method to enable or disable the monitoring and control of DNS service on all Microsoft servers in the Grid.

Parameter

Specify "true" to enable monitoring and control of DNS service on all Microsoft servers in the Grid and "false" to forbid it. Default value is "false".

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get ms_enable_dns_monitoring value
my $enable_dns_monitoring = $grid->ms_enable_dns_monitoring();
#Modify ms_enable_dns_monitoring value
$grid->ms_enable_dns_monitoring("true");
```

ms_enable_dns_reports_sync( )

Use this method to enable or disable the synchronization of DNS reporting data from all Microsoft servers in the Grid.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

Specify "true" to enable the synchronization of DNS reporting data from all Microsoft servers in the Grid. Specify "false" to disable the synchronization of DNS reporting data from all Microsoft servers in the Grid. Default value is "true".

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get ms_enable_dns_reports_sync
my $enable_dns_reports_sync = $grid->ms_enable_dns_reports_sync();
#Modify ms_enable_dns_reports_sync
$grid->ms_enable_dns_reports_sync("true");
```

ms_enable_network_users( )

Use this method to enable or disable Network Users creation.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

Specify "true" to enable network users creation for all Microsoft servers in the Grid and "false" to forbid it. Default value is "false".

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get ms_enable_network_users value
my $enable_network_users = $grid->ms_enable_network_users();
#Modify ms_enable_network_users value
$grid->ms_enable_network_users('true');
```

ms_enable_user_sync( )

Use this method to enable or disable Active Directory users synchronization for all Microsoft servers in the Grid.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

Specify "true" to enable Active Directory users synchronization for all Microsoft servers in the Grid and "false" to forbid it. Default value is "false".

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get ms_enable_user_sync value
my $enable_user_sync = $grid->ms_enable_user_sync();
#Modify ms_enable_user_sync value
$grid->ms_enable_user_sync('true');
```

ms_ldap_timeout( )

Use this method to set or retrieve an LDAP connection timeout interval in seconds for all Microsoft servers.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

Valid value is an unsigned integer. Default value is 10.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get ms_ldap_timeout value
my $ldap_timeout = $grid->ms_ldap_timeout();
#Modify ms_ldap_timeout value
$grid->ms_ldap_timeout(10);
```

ms_log_destination( )

Use this method to direct the logging of synchronization messages to the syslog or mslog.

Parameter

Valid values are "SYSLOG" and "MSLOG".

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get ms_log_destination value
my $log_destination = $grid->ms_log_destination();
#Modify ms_log_destination value
$grid->ms_log_destination("SYSLOG");
```

ms_max_connection( )

Use this method to set or retrieve the maximum number of connections to Microsoft servers.

Parameter

Valid values are integers between 2 and 40 inclusive.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get ms_max_connection value
my $max_connection = $grid->ms_max_connection();
#Modify ms_max_connection value
$grid->ms_max_connection(20);
```

ms_rpc_timeout( )

Use this method to set or retrieve timeout in seconds for RPC connections to all Microsoft servers.

Parameter

Valid values are integers between 0 and 65535 inclusive.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get ms_rpc_timeout value
my $rpc_timeout = $grid->ms_rpc_timeout();
#Modify ms_rpc_timeout value
$grid->ms_rpc_timeout(10);
```

ms_user_default_timeout( )

Use this method to set or retrieve the default timeout in seconds for Active Directory users synchronization for all Microsoft servers.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

The valid value is an unsigned integer.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get ms_user_default_timeout value
my $user_default_timeout = $grid->ms_user_default_timeout();
#Modify ms_user_default_timeout value
$grid->ms_user_default_timeout(10);
```

name( )

Use this method to set or retrieve the grid name.

Parameter

String with the name of the grid.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

Example

```
#Get name
my $name = $grid->name();
#Modify name
$grid->name("InfobloxGrid");
```

nat_group_list( )

Use this method to set or retrieve the NAT (Network Address Translation) group list.

Parameter

Array reference that contains each Infoblox::Grid::NatGroup object. The default value is empty.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

Example

```
#Add a new nat_group_list
my $nat1 = Infoblox::Grid::NatGroup->new( "name" => "natgroup" ,
                                          "comment" => "new NAT group" );

$grid->nat_group_list( [$nat1] );

#To undefined the nat_group_list
$grid->nat_group_list([ ]);
```

notification_email_addr( )

Use this method to set or retrieve the notification e-mail address.

Parameter

Text with the e-mail address of the administrator.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

Example

```
#Get notification_email_addr
my $email = $grid->notification_email_addr();
#Modify notification_email_addr
```

```
$grid->notification_email_addr("admin\@domain.com");
```

ntp_server( )

Use this method to set or retrieve the NTP (Network Time Protocol) server list. Only valid when enable_ntp is set to "true".

Parameter

Array reference that contains Infoblox::Grid::NTPServer objects.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

Example

```
#Add a new ntp_server
my $ntp_server1 = Infoblox::Grid::NTPServer->new(address => "1.1.1.1");
my $ntp_server2 = Infoblox::Grid::NTPServer->new(address => "2.2.2.2");
$grid->ntp_server([$ntp_server1, $ntp_server2]);
```

ntp_access_list( )

Use this method to set or retrieve the NTP (Network Time Protocol) access control list. Only valid when enable_ntp is set to "true".

Parameter

Valid value is an Infoblox::Grid::NamedACL object or an array reference that contains Infoblox::Grid::NTPAccess objects.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

Example

```
#Get ntp_access_list
my @ntp_access_list = $grid->ntp_access_list();
#Modify ntp_access_list
$grid->ntp_access_list([$ntpac1,$ntpac2]);
#To undefined ntp_access_list
$grid->ntp_access_list([ ]);
#Use Infoblox::Grid::NamedACL
$grid->ntp_access_list($nacl);
```

ntp_service_type( )

Use this method to set or retrieve the NTP service type for assigned Named ACL object.

The attribute value is ignored if 'ntp_access_list' attribute is not a Named ACL object.

Parameter

Valid values are "Time" or "Time and NTP control". The default value is "Time".

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get ntp_service_type
my $service = $grid->ntp_service_type();
#Modify ntp_service_type
$grid->ntp_service_type("Time and NTP control");
```

ntp_authentication_key( )

Use this method to set or retrieve the NTP (Network Time Protocol) authentication keys of the grid. Only valid when enable_ntp is set to "true".

Parameter

Valid value is an array reference that contains Infoblox::Grid::NTPKey objects.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

Example

```
#Add a new ntp_authentication_key
my $ntp_key1 = Infoblox::Grid::NTPKey->new(
            key_number => "1111",
            key_type   => "M",
            key_string => "aaaa"
        );

my $ntp_key2 = Infoblox::Grid::NTPKey->new(
            key_number => "2222",
            key_type   => "n",
            key_string => "d3e54352e5548080"
        );

$grid->ntp_authentication_key([$ntp_key1, $ntp_key2]);
#To undefined ntp_authentication_key
$grid->ntp_authentication_key(undef);
```

ntp_kod( )

Use this method to enable Kiss-o'-Death packets on the NTP server.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

Specify "true" to enable Kiss-o'-Death packets on the NTP server or "false" to disable them. The default value is "false".

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get the current value of the ntp_kod parameter
my $ntp_kod = $grid->ntp_kod();
#Set the ntp_kod parameter
$grid->ntp_kod("true");
```

objects_changes_tracking_setting( )

Use this method to set or retrieve the object changes tracking setting.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

The valid value is an Infoblox::Grid::ObjectsChangesTrackingSetting object.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get objects_changes_tracking_setting value
my $value = $object->objects_changes_tracking_setting();

#Construct objects_changes_tracking_setting value
my $objects_changes_tracking_setting = Infoblox::Grid::ObjectsChangesTrackingSetting->new(
    enable => 'true',
);

#Modify objects_changes_tracking_setting value
$object->objects_changes_tracking_setting($objects_changes_tracking_setting);
```

password_chars_to_change( )

Use this method to set or retrieve the minimum required character changes from the previous user password.

Parameter

Valid values are between 0 and 64. The default value is 0.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get password_chars_to_change
my $password_chars_to_change = $obj->password_chars_to_change();
#Modify password_chars_to_change
$obj->password_chars_to_change(6);
```

password_expire_days( )

Use this parameter to set the number of days that a password is valid. This parameter will be ignored if password_expiration_enable is not set to "true".

Parameter

The parameter is a number of days of password validity. The default value is 30.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get the current value of the password_expire_days
my $passw_expiration = $grid->password_expire_days();
#Set the password_expire_days parameter
$grid->password_expire_days(120);
```

password_expire_enable( )

Use this parameter to enable password expiration.

Parameter

Setting the parameter to "true" enables user passwords to expire. Setting the parameter to "false" enables user passwords to remain valid indefinitely. The default value is "false".

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get the current value of the password_expire_enable parameter
my $passw_expiration = $grid->password_expire_enable();
#Set the password_expire_enable parameter
$grid->password_expire_enable("true");
```

password_force_reset( )

Use this parameter to force users to reset their passwords when they first log in.

Parameter

Setting the parameter to "true" forces users to reset their passwords when they first log in. Setting the parameter to "false" allows the new user to keep the admin-defined password. The default value is "false".

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get the current value of the password_force_reset parameter
my $passw_force_reset = $grid->password_force_reset();
#Set the password_force_reset parameter
```

```
$grid->password_force_reset("true");
```

password_min_length( )

Use this method to set or retrieve the minimum user password length.

Parameter

Valid values are between 4 and 64.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get password_min_length
my $password_min_length = $obj->password_min_length();
#Modify password_min_length
$obj->password_min_length(6);
```

password_num_lower_char( )

Use this method to set or retrieve the required minimum number of lowercase characters in a user password.

Parameter

Valid values are between 0 and 64. The default value is 0. Note that the total of password_num_lower_char, password_num_upper_char, password_num_numeric_char and password_num_symbol_char must not exceed password_min_length.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get password_num_lower_char
my $password_num_lower_char = $obj->password_num_lower_char();
#Modify password_num_lower_char
$obj->password_num_lower_char(6);
```

password_num_numeric_char( )

Use this method to set or retrieve the required minimum number of numeric characters in a user password.

Parameter

Valid values are between 0 and 64. The default value is 0. Note that the total of password_num_lower_char, password_num_upper_char, password_num_numeric_char and password_num_symbol_char must not exceed password_min_length.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get password_num_numeric_char
my $password_num_numeric_char = $obj->password_num_numeric_char();
#Modify password_num_numeric_char
$obj->password_num_numeric_char(6);
```

password_num_symbol_char( )

Use this method to set or retrieve the required minimum number of symbol (non-alphanumeric) characters in a user password.

Parameter

Valid values are between 0 and 64. The default value is 0. Note that the total of password_num_lower_char, password_num_upper_char, password_num_numeric_char and password_num_symbol_char must not exceed password_min_length.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get password_num_symbol_char
my $password_num_symbol_char = $obj->password_num_symbol_char();
#Modify password_num_symbol_char
$obj->password_num_symbol_char(6);
```

password_num_upper_char( )

Use this method to set or retrieve the required minimum number of uppercase characters in a user password.

Parameter

Valid values are between 0 and 64. The default value is 0. Note that the total of password_num_lower_char, password_num_upper_char, password_num_numeric_char and password_num_symbol_char must not exceed password_min_length.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get password_num_upper_char
my $password_num_upper_char = $obj->password_num_upper_char();
#Modify password_num_upper_char
$obj->password_num_upper_char(6);
```

password_reminder_days( )

Use this method to set the number of days before a password expires that the appliance starts displaying password change reminders. This parameter will be ignored if password_expiration_enable is "false".

Parameter

The parameter is a number of days before password expiration when the user get a reminder to change password. The default value is 7. Note that password_reminder_days must be lower than password_expire_days.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get the current value of the password_reminder_days parameter
my $passw_reminder= $grid->password_reminder_days();
#Set the password_reminder_days parameter
$grid->password_reminder_days(7);
```

prefer_resolver( )

Use this method to set or retrieve the preferred resolver.

Parameter

The valid values are Infoblox::DNS::Member object or Infoblox::DNS::Nameserver object.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

Example

```
#Get prefer_resolver
my $prefer_resolver = $grid->prefer_resolver();
#Modify prefer_resolver
$grid->prefer_resolver("3.3.3.3");
```

query_comm_string ( )

Use this method to set or retrieve the SNMP (Simple Network Management Protocol) community string that management systems must send together with their queries to the Infoblox appliance. If this is set to undef, SNMP queries will be disabled.

Parameter

Text with the SNMP community string.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

Example

```
#Get query_comm_string
my $query_comm_string = $grid->query_comm_string();
#Modify query_comm_string
$grid->query_comm_string("private");
```

remote_console_access( )

Use this method to enable/disable remote console access.

Parameter

Specify "true" to enable remote console access or "false" to disable it.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

Example

```
#Get remote_console_access
my $remote_console_access = $grid->remote_console_access();
#Modify remote_console_access
$grid->remote_console_access("false");
```

restart_banner_setting( )

Use this method to set or retrieve Grid restart banner setting.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

The valid value is an Infoblox::Grid::RestartBannerSetting object.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get restart_banner_setting
$restart_banner_setting = $grid->restart_banner_setting();

#Modify restart_banner_setting
$grid->restart_banner_setting($banner);
```

restart_status( )

Use this method to retrieve the restart status of the Grid. This is a read-only attribute.

Omit the parameter to retrieve the attribute value.

Parameter

None

Returns

The valid return value is an Infoblox::Grid::ServiceRestart::Status object.

Example

```
#Get restart_status
$restart_status = $grid->restart_status();
```

rpz_hit_rate_interval( )

Use this method to set or retrieve the minimum time interval (in seconds) between changes in RPZ Hit Rate.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

The valid value is an unsigned integer greater than 1 that represents the number of seconds between changes in RPZ Hit Rate.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
# Get attribute value
my $value = $grid->rpz_hit_rate_interval();
# Modify attribute value
$grid->rpz_hit_rate_interval(20);
```

rpz_hit_rate_min_query( )

Use this method to set or retrieve the minimum number of queries between RPZ Hit Rate checks.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

The valid value is an unsigned integer between 0 and 1073741824 that represents the minimum number of queries between changes in RPZ Hit Rate.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
# Get attribute value
my $value = $grid->rpz_hit_rate_min_query();
# Modify attribute value
$grid->rpz_hit_rate_min_query(20);
```

rpz_hit_rate_max_query( )

Use this method to set or retrieve the maximum number of queries between RPZ Hit Rate checks.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

The valid value is an unsigned integer between 1 and 1073741824 that represents the maximum number of queries between changes in RPZ Hit Rate.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
# Get attribute value
my $value = $grid->rpz_hit_rate_max_query();
# Modify attribute value
$grid->rpz_hit_rate_max_query(20);
```

search_domains( )

Use this method to set or retrieve the Search Domain Group, which is a group of domain names that the Infoblox device can add to partial queries that do not specify a domain name. Note that you can set this parameter only when prefer_resolver or alternate_resolver is set.

Parameter

Valid value is a reference to an array of strings.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

Example

```
#Get search_domains
my $search_domains = $grid->search_domains();
#Modify search_domains
$grid->search_domains(["domain1.com", "domain2.com"]);
```

secret( )

Use this method to set the shared secret of the grid. This is a write-only attribute.

Parameter

Text with the shared secret of the grid.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

Example

```
#Modify secret
$grid->secret("shared secret");
```

service_status( )

Use this method to retrieve the overall service status of the Grid. This is a read-only attribute.

Parameter

None

Returns

The valid return values are 'FAILED', 'OFFLINE', 'INACTIVE', 'UNKNOWN', 'WARNING' and 'WORKING'.

Example

```
#Get service_status
$grid->service_status();
```

support_access( )

Use this method to enable/disable support access. Default value is "true".

Parameter

Specify "true" to enable support access or "false" to disable it.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

Example

```
#Get support_access
my $support_access = $grid->support_access();
#Modify support_access
$grid->support_access("false");
```

support_access_info( )

Use this method to retrieve the information string for support access.

Parameter

None.

Returns

The method returns the attribute value.

Example

```
#Get support_access_info
my $support_access_info = $grid->support_access_info();
```

scheduled_backup( )

Use this method to retrieve the scheduled backup configuration.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

Valid value is an Infoblox::Grid::ScheduledBackup object.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

Example

```
#Get scheduled_backup
my $scheduled_backup = $grid->scheduled_backup();
#Modify scheduled_backup
$grid->scheduled_backup($scheduled_backup);
```

session_timeout( )

Use this method to set or retrieve the session timeout.

Parameter

Number that specifies the session timeout for the grid (range is from 60 to 31536000 seconds).

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

Example

```
#Get session_timeout
my $session_timeout = $grid->session_timeout();
#Modify session_timeout
$grid->session_timeout("6000");
```

snmp_admin( )

Use this method to set or retrieve the SNMP admin object.

Parameter

Valid data type is an Infoblox::Grid::SNMP::Admin object.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

Example

```
#Add a SNMP Admin object
my $snmp = Infoblox::Grid::SNMP::Admin->new(
    sysContact  => "Somebody",
    sysLocation => "Infoblox Headquarter",
    sysName     => "noc1",
    sysDescr    => "SunOS noc1 5.6 Generic_105181-03 sun4u"
);

$grid->snmp_admin($snmp);
#To undefined SNMP Admin object
$grid->snmp_admin(undef);
```

snmpv3_query_users( )

Use this method to set or retrieve the SNMPv3 users from which the appliance accepts SNMP queries.

Parameter

Valid value is a reference to the list of the Infoblox::Grid::SNMP::User manpage objects.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

Example

```
#Get snmpv3_query_users field
my $snmpv3_users = $grid->snmpv3_query_users();
my $user = $$snmpv3_users[0];
#Set snmpv3_query_users field
$grid->snmpv3_query_users([$user1, $user2]);
```

syslog_backup_servers( )

Use this method to set or retrieve an array that contains syslog backup server settings for a Grid.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

Valid value is an array reference that contains Infoblox::Grid::SyslogBackupServer objects.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#get syslog_backup_servers value
my $servers = $grid->syslog_backup_servers();

#modify syslog_backup_servers value
$grid->syslog_backup_servers([$server1, $server2]);
```

syslog_server( )

Use this method to set or retrieve the list of syslog servers.

Setting this method to a defined value implicitly sets the external_syslog_server_enable method to 'true'. Setting the parameter to undefined sets the external_syslog_server_enable method to 'false'.

Note that when syslog_server is set to a defined value and external_syslog_server_enable is set to 'false', the last operation takes precedence. Thus the sequence $object->syslog_server([$syslog]); $object->external_syslog_server_enable('false'); will set external_syslog_server_enable to 'false', and the sequence $object->external_syslog_server_enable('false'); $object->syslog_server([$syslog]); will result in external_syslog_server_enable='true'.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

Array reference that contains Infoblox::Grid::SyslogServer objects.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

Example

```
my $syslog1 = Infoblox::Grid::SyslogServer->new(
    ipv4addr => "44.44.44.44",
    severity => "debug",
);

my $syslog2 = Infoblox::Grid::SyslogServer->new(
    ipv4addr => "55.55.55.55",
    severity => "debug",
);

$grid->syslog_server([$syslog1, $syslog2]);
```

syslog_size( )

Use this method to set or retrieve the grid syslog rotation size in megabytes.

Parameter

Valid value is between 10 and 300.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get syslog_size
my $syslog_size = $obj->syslog_size();
#Modify syslog_size
$obj->syslog_size(100);
```

threshold_traps( )

Use this method to retrieve or modify the list of threshold traps. Each element in the list is a reference to the Infoblox::Grid::SNMP::ThresholdTrap object. All the possible threshold traps are in the list. The user can only change the values for each trap or remove traps.

Parameter

Reference to a list of Infoblox::Grid::SNMP::ThresholdTrap objects. The default value is a reference to a list of all threshold traps.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get threshold traps
my $threshold_traps = $grid->threshold_traps();
#Modify threshold traps
my $threshold_traps = $grid->threshold_traps();
foreach(@$threshold_traps){if $_->trap_name() eq 'Rootfs'; $_->trap_reset=60;} #Change value for
'Rootfs' trap
$grid->threshold_traps($threshold_traps);
$session->modify($grid);
```

token_usage_delay( )

Use this method to set or retrieve the amount of time (in minutes) that the one-time security token for a pre-provisioned vNIOS member can stay unused before the appliance logs a warning message. The token_usage_delay has a life time of 60 minutes.

Parameter

The amount of time (in minutes) that the one-time security token for a pre-provisioned vNIOS member can stay unused before the appliance logs a warning message. Valid values must be between 1 and 60. The default value is 10.

Returns

If you specified a parameter,the method returns true when the modification succeeds,and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get token_usage_delay
my $delay = $grid->token_usage_delay();
#Modify token_usage_delay
$grid->token_usage_delay("30");
```

trap_comm_string( )

Use this method to set or retrieve the trap community string. If this is set to undef, SNMP traps will be disabled.

Parameter

Text with the SNMP trap community string.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

Example

```
#Get trap_comm_string
my $trap_comm_string = $grid->trap_comm_string();
#Modify trap_comm_string
$grid->trap_comm_string("public");
```

trap_notifications( )

Use this method to configure trap notifications. Each element in the list is a reference to the Infoblox::Grid::SNMP::TrapNotification object.

Parameter

Reference to a list of Infoblox::Grid::SNMP::TrapNotification objects. The default value is a reference to a list of all trap notifications.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get a reference to trap notifications array
my $trap_notifications = $grid->trap_notifications();
#Modify trap notifications
my $notifications = $grid->trap_notifications();
foreach(@$notifications){if $_->trap_type() eq 'CPU'; $_->enable_email="true";} #Change value for
'CPU' trap
$grid->trap_notifications($notifications);
$session->modify($grid);
```

trap_receiver( )

Use this method to set or retrieve a trap receiver.

Parameter

Array reference that contains IPv4 addresses (example: 192.168.1.2) and the Infoblox::Grid::SNMP::TrapReceiver manpage objects.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

Example

```
#Get trap_receiver
my $trap_receiver = $grid->trap_receiver();
#Modify trap receiver (SNMPv2)
$grid->trap_receiver(["10.0.0.10"]);
#SNMPv3 trap receivers
my $receiver1 = Infoblox::Grid::SNMP::TrapReceiver->new( ip_addr => "19.20.21.22", user => $user);
$grid->trap_receiver([$receiver1]);
```

updates_download_member_config( )

Use this method to set or retrieve the list of updates download member configurations.

Parameter

The valid value is array of Infoblox::Grid::UpdatesDownloadMemberConfig objects.

Returns

If you specified a parameter,the method returns true when the modification succeeds,and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get updates_download_member_config value
my $updates_download_member_config = $grid->updates_download_member_config();

#Modify updates_download_member_config value
$grid->updates_download_member_config([$config1, $config2]);
```

vpn_port( )

Use this method to set or retrieve the VPN port.

Parameter

Number of the Virtual Private Network (VPN) port of the grid. Value must be between 1024 and 63999.

Returns

If you specified a parameter,the method returns true when the modification succeeds,and returns false when the operation fails.

Example

```
#Get port
my $port = $grid->vpn_port();
#Modify vpn_port
$grid->vpn_port("2000");
```

SAMPLE CODE

The following sample code demonstrates different operations that can be applied to an object, such create, modify, and remove an object. This sample code also includes error handling for the operations.

```
#PROGRAM STARTS: Include all the modules that will be used
use strict;
use Infoblox;

#Create a session to the Infoblox device
my $session = Infoblox::Session->new(
    master   => "192.168.1.2",
    username => "admin",
    password => "infoblox"
);
unless ($session) {
   die("Construct session failed: ",
       Infoblox::status_code() . ":" . Infoblox::status_detail());
}
print "Session created successfully\n";
```

#Get and modify the Grid object.

```
#Get a Grid through the session
my $grid = $session->get(
    object => "Infoblox::Grid",
    name   => "Infoblox"
);
unless ($grid) {
    die("Get Grid failed: ",
        $session->status_code() . ":" . $session->status_detail());
}

#Modifying the value of the specified object.
my $nat1 = Infoblox::Grid::NatGroup->new( "name" => "natgroup" ,
                                          "comment" => "new NAT group" );

$grid->nat_group_list( [$nat1] );
$grid->enable_ntp ("true");
my $ntp_server1 = Infoblox::Grid::NTPServer->new(address => "2.2.2.2");
$grid->ntp_server([$ntp_server1]);

my $ntpac1 = Infoblox::Grid::NTPAccess->new(
                                           address => "2.2.2.2",
                                           service => "Time",
                                          );

my $ntpac2 = Infoblox::Grid::NTPAccess->new(
                                           address => "4.4.4.4",
                                           service => "Time and NTP control",
                                          );

$grid->ntp_access_list([$ntpac1,$ntpac2]);
my $ntp_key1 = Infoblox::Grid::NTPKey->new(
           key_number => "1111",
           key_type   => "M",
           key_string => "aaaa"
```

```
        );
my $ntp_key2 = Infoblox::Grid::NTPKey->new(
            key_number => "2222",
            key_type   => "n",
            key_string => "d3e54352e5548080"
        );
$grid->ntp_authentication_key([$ntp_key1, $ntp_key2]);

#Apply the changes
$session->modify($grid)
    or die("Modify Grid failed: ",
            $session->status_code() . ":" . $session->status_detail());
print "Grid object modified successfully \n";

####PROGRAM ENDS####
```

AUTHOR

Infoblox Inc. http://www.infoblox.com/

SEE ALSO

Infoblox::Session, Infoblox::Session->get(), Infoblox::Session->search(), Infoblox::Session->add(), Infoblox::Session->remove(), Infoblox::Session->modify(), Infoblox::Grid::NamedACL, Infoblox::Grid::ExtensibleAttributeDef::Descendants, Infoblox::Grid::SyslogBackupServer , Infoblox::DNS::View, Infoblox::DNS::Zone, Infoblox::Grid::LOM::User, Infoblox::Grid::ServiceRestart::Status, Infoblox::Grid::RestartBannerSetting

COPYRIGHT

Copyright (c) 2017 Infoblox Inc.

Infoblox::DHCP::IPv6NetworkContainer - DHCP IPv6 Network Container object

- NAME
- DESCRIPTION
- CONSTRUCTOR
- SESSION METHODS
  - Infoblox::Session->add( )
  - Infoblox::Session->get( )
  - Infoblox::Session->modify( )
  - Infoblox::Session->remove( )
  - Infoblox::Session->search( )
- METHODS
  - auto_create_reversezone( )
  - comment( )
  - cloud_info( )
  - ddns_domainname( )
  - ddns_enable_option_fqdn( )
  - ddns_generate_hostname( )
  - ddns_server_always_updates( )
  - ddns_ttl( )
  - discovery_basic_poll_setting( )
  - discovery_member( )
  - discover_now_status( )
  - discovery_blackout_setting( )
  - discovery_engine_type( )
  - domain_name( )
  - domain_name_servers( )
  - enable_ddns( )
  - enable_discovery( )
  - enable_immediate_discovery( )
  - endpoint_sources( )
  - extattrs( )
  - extensible_attributes( )
  - mgm_private( )
  - mgm_private_overridable( )
  - ms_ad_user_data( )
  - network( )
  - network_container( )
  - network_view( )
  - next_available_network( )
  - options( )
  - override_ddns_domainname( )
  - override_ddns_enable_option_fqdn( )
  - override_ddns_generate_hostname( )
  - override_ddns_ttl( )
  - override_domain_name( )
  - override_domain_name_servers( )
  - override_blackout_setting( )
  - override_enable_ddns( )
  - override_options( )
  - override_preferred_lifetime( )
  - override_recycle_leases( )
  - override_mgm_private( )
  - override_subscribe_settings( )
  - override_update_dns_on_lease_renewal( )
  - override_valid_lifetime( )
  - preferred_lifetime ( )
  - port_control_blackout_setting
  - recycle_leases( )
  - remove_subnets( )
  - restart_if_needed( )
  - rir( )
  - rir_organization( )
  - rir_registration_action( )
  - rir_registration_status( )
  - last_rir_registration_update_sent( )
  - last_rir_registration_update_status( )
  - same_port_control_discovery_blackout( )
  - send_rir_request( )
  - subscribe_settings( )
  - delete_reason( )
  - unmanaged( )
  - update_dns_on_lease_renewal( )

- valid_lifetime ( )
- zone_associations( )

- SAMPLE CODE
- AUTHOR
- SEE ALSO
- COPYRIGHT

NAME

Infoblox::DHCP::IPv6NetworkContainer - DHCP IPv6 Network Container object

DESCRIPTION

A network can contain child networks. The network that contains the child network is called a network container. This object encapsulates an IPv6 network container object.

CONSTRUCTOR

```
  my $network = Infoblox::DHCP::IPv6NetworkContainer->new(
                network                          => $ipv6_subnet/prefix,               #Required, when
creating a network container
                auto_create_reversezone          => "true" | "false",                  #Optional / Default
is "false"
                comment                          => $string| undef,                    #Optional / Default
is undefined
                remove_subnets                   => "true" | "false",                  #Optional / Default
is undefined
                rir_organization                 => $rir_org,                          #Optional / An
Infoblox::Grid::RIR::Organization object
                rir_registration_action          => 'NONE' | 'CREATE' | 'MODIFY' | 'DELETE',  # Optional /
write-only
                rir_registration_status          => 'NOT_REGISTERED' | 'REGISTERED',   #Optional / Default
is 'NOT_REGISTERED'
                send_rir_request                 => 'true' | 'false',                  #Optional / write-
only
                delete_reason                    => $string,                           #Optional / write-
only
                                                                                       #Required only if
'rir_registration_action' is specified as 'DELETE'
                mgm_private                      => "true" | "false",                  #Optional / Default
is "false"
                network_container                => $ipv6_subnet/prefix,               #Optional / Default
is "/"
                network_view                     => $NetworkView,                      #Optional / Default
is the system-defined default network view
                zone_associations                => [$zone1, $zone2, ... ] | undef,    #Optional / Default
is an empty list
                extattrs                         => { $string => $extattr, ... },      #Required if
'rir_organization' attribute is defined
                                                                                       #Otherwise is
optional
                extattrs                         => { $string => $extattr, ... },      #Required if
'rir_organization' attribute is defined
                                                                                       #Otherwise is
optional
                extensible_attributes            => { $string => $string | $num, $string => [ $string |
$num, ... ], ... } #Required if 'rir_organization' attribute is defined
#Otherwise is optional
                discovery_blackout_setting       => $blackout,                         #Optional
                port_control_blackout_setting    => $blackout,                         #Optional
                override_blackout_setting        => "true" | "false",                  #Optional / Default
value is "false"
                same_port_control_discovery_blackout => "true" | "false",              #Optional / Default
value is "false"

                ddns_domainname                      => $string| undef,                    #Optional /
Default is undefined
                ddns_enable_option_fqdn              => "true" | "false" ,                 #Optional /
Default is "false"
                ddns_generate_hostname               => "true" | "false" | undef,          #Optional /
Default is undefined
                ddns_server_always_updates           => "true" | "false",                  #Optional /
Default is "true"
                ddns_ttl                             => $num,                              #Optional /
Default is "0"
                domain_name                          => $string,                           #Optional /
Default is undefined
                domain_name_servers                  => "$ipv6addr,$ipv6addr,...",         #Optional /
```

```
Default is undefined
                enable_ddns                         => "true" | "false" | undef,          #Optional /
Default is undefined
                options                             => [$Option1,$Option2,...],           #Optional /
Default is empty list
                override_ddns_domainname            => "true" | "false",                  #Optional /
Default value is "false"
                override_ddns_enable_option_fqdn    => "true" | "false",                  #Optional /
Default value is "false"
                override_ddns_generate_hostname     => "true" | "false",                  #Optional /
Default value is "false"
                override_ddns_ttl                   => "true" | "false",                  #Optional /
Default value is "false"
                override_domain_name                => "true" | "false",                  #Optional /
Default value is "false"
                override_domain_name_servers        => "true" | "false",                  #Optional /
Default value is "false"
                override_enable_ddns                => "true" | "false",                  #Optional /
Default value is "false"
                override_mgm_private                => "true" | "false",                  #Optional /
Default value is "false"
                override_options                    => "true" | "false",                  #Optional /
Default value is "false"
                override_preferred_lifetime         => "true" | "false",                  #Optional /
Default value is "false"
                override_recycle_leases             => "true" | "false",                  #Optional /
Default value is "false"
                override_update_dns_on_lease_renewal  => "true" | "false",                #Optional /
Default value is "false"
                override_valid_lifetime             => "true" | "false",                  #Optional /
Default value is "false"
                preferred_lifetime                  => $num,                              #Optional /
Default is "27000"
                recycle_leases                      => "true" | "false" | undef,          #Optional /
Default is undefined
                update_dns_on_lease_renewal         => "true" | "false",                  #Optional /
Default value is "false"
                valid_lifetime                      => $num,                              #Optional /
Default is "43200"
                subscribe_settings          => $subscribe_settings,                       #Optional /
Default is undefined
                override_subscribe_settings => 'true' | 'false',                          #Optional /
Default is 'false'
                discovery_engine_type       => 'NETMRI' | 'NETWORK_INSIGHT' | 'VDISCOVERY' | 'UNKNOWN' |
'NONE', #Optional / Default is 'NONE'
                                );
```

You cannot set both extattrs and extensible_attributes at the same time.

SESSION METHODS

This section describes all the methods in an Infoblox::Session module that you can apply to a DHCP IPv6 Network Container object.

Infoblox::Session->add( )

Use this method to add a DHCP IPv6 Network Container object to the Infoblox appliance. See Infoblox::Session->add() for parameters and return values.

Example

```
#Adding an IPv6 network container.
#Construct an object
my $network_container = Infoblox::DHCP::IPv6NetworkContainer->new(
        network => "2001::/64",
);

# Submit for adding a network container
my $response = $session->add( $network_container );
```

Infoblox::Session->get( )

Use this method to retrieve all the matching DHCP IPv6 Network Container objects from the Infoblox appliance. See Infoblox::Session->get() for parameters and return values.

'discover_now_status' will not be returned by default, use 'return_methods' to request it if needed.

Key References

```
Apply the following attributes to get a specific DHCP IPv6 Network Container object:
```

```
  network               - Optional. The IPv6 address and netmask of a network in CIDR format.
  network_view          - Optional. The network view in which the network is located. By default, all
network views are searched.
  extattrs              - Optional. A hash reference containing extensible attributes.
  extensible_attributes - Optional. A hash reference containing extensible attributes.
  comment               - Optional. A comment in string format.
```

Examples

```
my  @result_array = $session->get(
     object        => "Infoblox::DHCP::IPv6NetworkContainer",
     network       => "2001::/64",
     network_view  => "default"
);

my  @result_array = $session->get(
     object                => "Infoblox::DHCP::IPv6NetworkContainer",
     extensible_attributes => { 'Site' => 'Santa Clara' }
);

my  @result_array = $session->get(
     object  => "Infoblox::DHCP::IPv6NetworkContainer",
     comment => "my-comment",
);
```

Infoblox::Session->modify( )

Use this method to modify a DHCP IPv6 Network Container object in the Infoblox appliance. See Infoblox::Session->modify() for parameters and return values.

Example

```
#Use method to modify comment.
$network_container->comment("modified comment");
# Submit modification
my $response = $session->modify( $network_container );
```

Infoblox::Session->remove( )

Use this method to remove an object from the Infoblox appliance. See Infoblox::Session->remove() for parameters and return values.

To remove a specifc object, first use `get()` or `search()` to retrieve the specific object, and then submit this object for removal.

Key References

```
remove_options - Optional. A hash reference containing the following attributes:

+-------------------------------------------------------------------------------------------+
| reason              - Optional. The reason for removing the network container in string   |
|                       format with a maximum of 256 bytes.                                 |
|                       Used when 'rir_organization' attribute has been defined.            |
| send_rir_request    - Optional. Boolean flag that indicates whether the appliance should |
|                       send an API or email request for Regional Internet Registry (RIR).  |
|                       Used when 'rir_organization' attribute has been defined.            |
+-------------------------------------------------------------------------------------------+
```

Example

```
 # Get the object under the same name
my  @result_array = $session->get(

      object        => "Infoblox::DHCP::IPv6NetworkContainer" ,
      network       => "2001::/64",
      network_view  => "default"
        );

# Find the desired object in the retrieved list.
my $network_container = $result_array[0];

# Submit for removal
my $response = $session->remove(
    $network_container,
    remove_options => {'reason' => 'Moved to B3', send_rir_request => 'true'} );
```

Infoblox::Session->search( )

Use this method to search for a DHCP IPv6 Network Container object in the Infoblox appliance. See Infoblox::Session->search() for parameters and return values.

'discover_now_status' will not be returned by default, use 'return_methods' to request it if needed.

Key References

```
 Apply the following attributes to search for a DHCP IPv6 Network Container object:

  network              - Optional. The IPv6 address and netmask of a network in CIDR format (regular
expression).
  network_view         - Optional. The network view in which the network is located. By default, all
network views are searched.
  extattrs             - Optional. A hash reference containing extensible attributes.
  extensible_attributes - Optional. A hash reference containing extensible attributes.
  comment              - Optional. A comment in string format (regular expression).
```

For more information about searching extensible attributes, see Infoblox::Grid::ExtensibleAttributeDef/Searching Extensible Attributes.

Examples

```
 # search for all DHCP IPv6 Network Container objects
     my @retrieved_objs = $session->search(
       object       => "Infoblox::DHCP::IPv6NetworkContainer",
       network      => "2001.*",
       network_view => "default"
     );

 # search network by comment
     my @retrieved_objs = $session->search(
       object    => "Infoblox::DHCP::IPv6NetworkContainer",
       comment   => "my-comment",
     );

 # search for all DHCP IPv6 Network Container objects defining "Santa Clara" for "Site" extensible
attribute
     my @retrieved_objs = $session->search(
       object                => "Infoblox::DHCP::IPv6NetworkContainer",
       extensible_attributes => { 'Site' => 'Santa Clara' }
     );
```

METHODS

This section describes all the methods that can be used to configure and retrieve the attribute values of a DHCP IPv6 Network Container object.

auto_create_reversezone( )

Use this method to set the auto_create_reversezone flag of a DHCP IPv6 Network Container object.

Include the specified parameter to set the attribute value. This is a write-only attribute and cannot be retrieved after it is set.

Parameter

Specify "true" to set the auto_create_reversezone flag or "false" to deactivate/unset it. The default value for this field is "false".

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

Example

```
 # Modify auto_create_reversezone
 $network_container->auto_create_reversezone("true");
```

comment( )

Use this method to set or retrieve a descriptive comment for a DHCP IPv6 Network Container object.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

Enter a descriptive comment for the network container in string format with a maximum of 256 bytes.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
# Get comment
my $comment = $network_container->comment();
# Modify comment
$network_container->comment("add a network");
```

cloud_info( )

Use this method to set or retrieve cloud API related information for the Infoblox::DHCP::IPv6NetworkContainer object.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

The valid value is a Infoblox::Grid::CloudAPI::Info object.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
# Get cloud_info
my $cloud_info = $network_view->cloud_info();
# Modify cloud_info
$network_view->cloud_info($cloud_info_object);
```

ddns_domainname( )

Use this method to set or retrieve the ddns_domainname value of a DHCP IPv6 Network Container object.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Setting this method to a defined value implicitly sets the override_ddns_domainname method to "true". Setting the parameter to undefined causes the appliance to use the Grid default and automatically resets the override_ddns_domainname attribute to "false".

Note that when ddns_domainname is set to a defined value and override_ddns_domainname is set to "false", the last operation takes precedence. Thus the sequence $object->ddns_domainname("testdomain"); $object->override_ddns_domainname("false"); will set override_ddns_domainname to "false", and the sequence $object->override_ddns_domainname("false"); $object->ddns_domainname("testdomain"); will result in override_ddns_domainname="true".

Parameter

The dynamic DNS domain name in string format.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get ddns_domainname
my $ddns_domainname = $network_container->ddns_domainname();
#Modify ddns_domainname
$network_container->ddns_domainname("test_domain.com");
```

ddns_enable_option_fqdn( )

Use this method to set or retrieve the ddns_enable_option_fqdn flag of a DHCP IPv6 Network Container object. This method controls whether the FQDN option sent by the client is to be used, or if the server can automatically generate the FQDN. This setting overrides the upper-level settings.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Setting this method to a defined value implicitly sets the override_ddns_enable_option_fqdn method to "true". Setting the parameter to undefined causes the appliance to use the upper-level setting and automatically resets the override_ddns_enable_option_fqdn attribute to "false".

Note that when ddns_enable_option_fqdn is set to a defined value and override_ddns_enable_option_fqdn is set to "false", the last operation takes precedence. Thus the sequence $object->ddns_enable_option_fqdn("true"); $object->override_ddns_enable_option_fqdn("false"); will set override_ddns_enable_option_fqdn to "false", and the sequence $object->override_ddns_enable_option_fqdn("false"); $object->ddns_enable_option_fqdn("true"); will result in override_ddns_enable_option_fqdn="true".

Parameter

Specify "true" to generate hostname or "false" to deactivate/unset it. The default value for this field is undefined, which indicates that this attribute inherits the member-level setting.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
# Get ddns_enable_option_fqdn
my $ddns_enable_option_fqdn = $network_container->ddns_enable_option_fqdn()
# Modify ddns_enable_option_fqdn
$network_container->ddns_enable_option_fqdn("true");
#Un-override
$network_container->ddns_enable_option_fqdn(undef);
```

ddns_generate_hostname( )

Use this method to set or retrieve the ddns_generate_hostname flag of a DHCP IPv6 Network Container object. Enable this method to allow only the DHCP server for generating hostname. This setting overrides the upper-level settings.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Setting this method to a defined value implicitly sets the override_ddns_generate_hostname method to "true". Setting the parameter to undefined causes the appliance to use the upper-level setting and automatically resets the override_ddns_generate_hostname attribute to "false".

Note that when ddns_generate_hostname is set to a defined value and override_ddns_generate_hostname is set to "false", the last operation takes precedence. Thus the sequence $object->ddns_generate_hostname("true"); $object->override_ddns_generate_hostname("false"); will set override_ddns_generate_hostname to "false", and the sequence $object->override_ddns_generate_hostname("false"); $object->ddns_generate_hostname("true"); will result in override_ddns_generate_hostname="true".

Parameter

Specify "true" to generate a hostname or "false" to deactivate/unset it. The default value for this field is undefined, which indicates that this attribute inherits the member-level setting.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
# Get ddns_generate_hostname
my $ddns_generate_hostname = $network_container->ddns_generate_hostname()
# Modify ddns_generate_hostname
$network_container->ddns_generate_hostname("true");
#Un-override
$network_container->ddns_generate_hostname(undef);
```

ddns_server_always_updates( )

Use this method to set or retrieve the ddns_server_always_updates flag of a DHCP IPv6 Network Container object. Enable this method to allow only the DHCP server to update DNS, regardless of the requests from the DHCP clients.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

Specify "true" to set the flag that enables the DHCP server to always updates DNS or "false" to deactivate/unset it.The default value for this field is "true".

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
# Get ddns_server_always_updates
my $ddns_server_always_updates = $network_container->ddns_server_always_updates()
# Modify ddns_server_always_updates
```

```
$network_container->ddns_server_always_updates("true");
```

ddns_ttl( )

Use this method to set or retrieve the DNS update Time to Live (TTL) value of a DHCP IPv6 Network Container object.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Setting this method to a defined value implicitly sets the override_ddns_ttl method to "true". Setting the parameter to undefined causes the appliance to use the Grid default and automatically resets the override_ddns_ttl attribute to "false".

Note that when ddns_ttl is set to a defined value and override_ddns_ttl is set to "false", the last operation takes precedence. Thus the sequence $object->ddns_ttl(12); $object->override_ddns_ttl("false"); will set override_ddns_ttl to "false", and the sequence $object->override_ddns_ttl("false"); $object->ddns_ttl(12); will result in override_ddns_ttl="true".

Parameter

A 32-bit integer (range from 0 to 4294967295) that represents the duration, in seconds, the update is cached. Zero indicates that the update is not cached. The default value is 0.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get DNS Update ttl
my $dns_ttl = $network_container->ddns_ttl();
#Modify DNS Update ttl
$network_container->ddns_ttl(1200);
```

discovery_basic_poll_setting( )

Use this method to set or retrieve the discovery_basic_poll_setting value of a DHCP IPv6 Network Container object.

Include the specified parameter to set the attribute value.

Parameter

The valid value is object of the Infoblox::Grid::Discovery::BasicPollSettings manpage.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get discovery_basic_poll_setting
my $discovery_basic_poll_setting = $network->discovery_basic_poll_setting();
#Modify discovery_basic_poll_setting
$network->discovery_basic_poll_setting($poll_setting);
```

discovery_member( )

Use this method to set or retrieve the discovery_member value of a DHCP IPv6 Network Container object.

Parameter

The discovery member name in string format.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get discovery_member
my $discovery_member = $network->discovery_member();
#Modify discovery_member
$network->discovery_member("test.localdomain");
```

discover_now_status( )

Use this method to retrieve the current discovery status. This attribute is read-only.

'discover_now_status' will not be returned by default, use 'return_methods' to request it if needed.

Parameter

None.

Returns

The method returns the attribute value that can be 'COMPLETE', 'FAILED', 'NONE', 'PENDING' or 'RUNNING'.

Example

```
#Get discover_now_status
my $discover_now_status = $network->discover_now_status();
```

discovery_blackout_setting( )

Use this method to set or retrieve the discovery blackout setting.

Setting this method to a defined value implicitly sets the override_blackout_setting method to "true". If port_control_blackout_setting is undefined, setting the parameter to undefined causes the appliance to use the Grid default and automatically resets the override_blackout_setting attribute to "false".

Note that when discovery_blackout_setting is set to a defined value and override_blackout_setting is set to "false", the last operation takes precedence. Thus the sequence $network->discovery_blackout_setting($blackout); $network->override_blackout_setting("false"); will set override_blackout_setting to "false", and the sequence $network->override_blackout_setting("false"); $network->discovery_blackout_setting($blackout); will result in override_blackout_setting="true".

Parameter

Valid value is an Infoblox::Grid::Discovery::Properties::BlackoutSetting object.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get discovery_blackout_setting
my $discovery_blackout_setting = $network->discovery_blackout_setting();
#Modify discovery_blackout_setting
$network->discovery_blackout_setting($blackout);
```

discovery_engine_type( )

Use this method to retrieve the DHCP IPv6 Network Container discovery engine type. This attribute is read-only.

Parameter

None.

Returns

The method returns the attribute value that can be 'NETMRI', 'NETWORK_INSIGHT', 'VDISCOVERY', 'UNKNOWN' or 'NONE'.

Example

```
#Get discovery_engine_type
my $discovery_engine_type = $network_container->discovery_engine_type();
```

domain_name( )

Use this method to set or retrieve the domain_name value of a DHCP IPv6 Network Container object.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Setting this method to a defined value implicitly sets the override_domain_name method to "true". Setting the parameter to undefined causes the appliance to use the Grid default and automatically resets the override_domain_name attribute to "false".

Note that when domain_name is set to a defined value and override_domain_name is set to "false", the last operation takes precedence. Thus the sequence $object->domain_name("testdomain"); $object->override_domain_name("false"); will set override_domain_name to "false", and the sequence $object->override_domain_name("false"); $object->domain_name("testdomain"); will result in override_domain_name="true".

Parameter

The name of the client domain in string format for resolving names in DNS.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get domain_name
my $domain_name = $network_container->domain_name();
#Modify domain_name
$network_container->domain_name("test_domain.com");
```

domain_name_servers( )

Use this method to set or retrieve the domain_name_servers value of a DHCP IPv6 Network Container object.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Setting this method to a defined value implicitly sets the override_domain_name_servers method to "true". Setting the parameter to undefined causes the appliance to use the Grid default and automatically resets the override_domain_name_servers attribute to "false".

Note that when domain_name_servers is set to a defined value and override_domain_name_servers is set to "false", the last operation takes precedence. Thus the sequence $object->domain_name_servers("2001::1,2001::2"); $object->override_domain_name_servers("false"); will set override_domain_name_servers to "false", and the sequence $object->override_domain_name_servers("false"); $object->domain_name_servers("2001::1,2001::2"); will result in override_domain_name_servers="true".

Parameter

The IPv6 addresses of the domain name servers.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get domain_name_servers
my $domain_name_servers = $network_container->domain_name_servers();
#Modify domain_name_servers
$network_container->domain_name_servers("2001::1,2001::2");
```

enable_ddns( )

Use this method to set or retrieve the dynamic DNS updates flag of a DHCP IPv6 Network Container object. The DHCP server can send DDNS updates to DNS servers in the same Grid and to external DNS servers. This setting overrides the member-level settings.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Setting this method to a defined value implicitly sets the override_enable_ddns method to "true". Setting the parameter to undefined causes the appliance to use the upper-level default and automatically resets the override_enable_ddns attribute to "false".

Note that when enable_ddns is set to a defined value and override_enable_ddns is set to "false", the last operation takes precedence. Thus the sequence $object->enable_ddns("true"); $object->override_enable_ddns("false"); will set override_enable_ddns to "false", and the sequence $object->override_enable_ddns("false"); $object->enable_ddns("true"); will result in override_enable_ddns="true".

Parameter

Specify "true" to set the dynamic DNS updates flag or "false" to deactivate/unset it. The default value for this field is undefined, which indicates that this attribute inherits the member-level setting.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
# Get enable_ddns
my $enable_ddns = $network_container->enable_ddns()
# Modify enable_ddns
```

```
$network_container->enable_ddns("true");
# Un-override enable_ddns
$network_container->enable_ddns(undef);
```

enable_discovery( )

Use this method to set or retrieve the enable_discovery value of a DHCP IPv6 Network Container object. This setting overrides the member level settings.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Setting this method to a defined value implicitly sets the override_enable_discovery method to "true". Setting the parameter to undefined causes the appliance to use the default from the upper level and automatically resets the override_enable_discovery attribute to "false".

Enable this flag if you want to set discovery for this object.

Parameter

Specify "true" to set the enable_discovery flag or "false" to deactivate/unset it. The default value is undefined.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get enable_discovery
my $enable_discovery = $network->enable_discovery();
#Modify enable_discovery
$network->enable_discovery("false");
```

enable_immediate_discovery( )

Use this method to set or retrieve enable_immediate_discovery value of a DHCP IPv6 Network Container object.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

Specify "true" to set the enable_immediate_discovery flag or "false" to deactivate/unset it. The default value is undefined.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get enable_immediate_discovery
my $enable_immediate_discovery = $network->enable_immediate_discovery();
#Modify enable_immediate_discovery
$network->enable_immediate_discovery("false");
```

endpoint_sources( )

Use this method to retrieve the list of endpoints that provide object data. This is a read-only attribute.

Omit the parameter to retrieve the attribute value.

Parameter

None

Returns

The valid return value is an array of Infoblox::CiscoISE::Endpoint objects.

Example

```
#Get endpoint_sources value
my $endpoint_sources = $object->endpoint_sources();
```

extattrs( )

Use this method to set or retrieve the extensible attributes associated with a DHCP IPv6 Network Container object.

Parameter

Valid value is a hash reference containing the names of extensible attributes and their associated values (Infoblox::Grid::Extattr objects).

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get extattrs
my $ref_extattrs = $network_container->extattrs();
#Modify extattrs
$network_container->extattrs({ 'Site' => $extattr1, 'Administrator' => $extattr2 });
```

extensible_attributes( )

Use this method to set or retrieve the extensible attributes associated with a DHCP IPv6 Network Container object

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

For valid values for extensible attributes, see Infoblox::Grid::ExtensibleAttributeDef/Extensible Attribute Values.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get extensible attributes
my $ref_extensible_attributes = $network_container->extensible_attributes();
#Modify extensible attributes
$network_container->extensible_attributes({ 'Site' => 'Santa Clara', 'Administrator' => [ 'Peter',
'Tom' ] });
```

mgm_private( )

Use this method to set or retrieve the flag that controls whether the object should be synced to the MGM or not.

Setting this method to a defined value implicitly sets the override_blackout_setting method to "true". If discovery_blackout_setting is undefined, setting the parameter to undefined causes the appliance to use the grid default and automatically resets the override_blackout_setting attribute to "false".

Note that when mgm_private is set to a defined value and override_mgm_private is set to "false", the last operation takes precedence. Thus the sequence $network->mgm_private("true"); $network->override_mgm_private("false"); will set override_mgm_private to "false", and the sequence $network->override_mgm_private("false"); $network->mgm_private("true"); will result in override_mgm_private="true".

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

Specify 'true' to enable synchronization of the managed Grid with the MGM or 'false' to forbid it. The default value is 'false'.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Note that this member is loaded on on-demand basis, so to retrieve the value the object must have been retrieved from the session first.

Example

```
#Get mgm_private value
my $value = $object->mgm_private();

#Modify mgm_private value
$object->mgm_private('true');
```

mgm_private_overridable( )

Use this method to retrieve the flag that indicates whether the overriding of 'mgm_private' flag is allowed or not. This is a read-only attribute.

Parameter

None

Returns

The method returns the attribute value.

Note that this member is loaded on on-demand basis, so to retrieve the value the object must have been retrieved from the session first.

Example

```
#Get mgm_private_overridable value
my $value = $object->mgm_private_overridable();
```

ms_ad_user_data( )

Use this method to retrieve Microsoft Active Directory users related information. This is a read-only attribute.

Parameter

None

Returns

The valid return value is an Infoblox::Grid::MSServer::AdUser::Data object.

Example

```
#Get ms_ad_user_data
my $ms_ad_user_data = $v6network_container->ms_ad_user_data();
```

network( )

Use this method to retrieve the network attribute of a DHCP IPv6 Network Container object.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

Specify the address and netmask of the network in CIDR format (example: 2001::/64).

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get network
my $network = $network_container->network();
```

network_container( )

Use this method to set or retrieve the network container of a DHCP IPv6 Network Container object.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

Specify the address and netmask of the network in CIDR format (example: 2001::/64).

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get network container
```

```
my $network_container = $network->network_container();
#Modify network container
$network->network_container("2001::/64");
```

network_view( )

Use this method to set or retrieve the network view of the DHCP IPv6 network container.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

The default value is the system-defined default network view, which means the network is in the default network view.

Parameter

Valid value is a defined Infoblox::DHCP::View object. The default network view value is the system-defined default network view object.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get network view
my $nview = $network_container->network_view();
#Modify network view, with an Infoblox::DHCP::View object
$network_container->network_view($nview);
```

next_available_network( )

Use this method to retrieve the next available IPv6 `network(s)` in the network container. Note that to use this method, the object MUST be retrieved through a get or search call, or it must have been created with an add call.

Parameter

The method accepts the following parameters:

```
  cidr - (required) The CIDR of the requested networks. It must be a number between 3 and 128,
inclusive.
  requested_num - (optional) The number of networks requested. It must be a number between 1 and 20,
inclusive.
  excluded - (optional) A reference to an array of networks to be excluded from the results.
```

Returns

This method returns the next available IPv6 `network(s)` in the network container. If requested_num is not specified, a string with the next available network will be returned; otherwise, the method will return a reference to an array of networks.

A network cannot be returned as the next available network if it cannot be inserted; such as when any of the following conditions exist:

- It's parent network has a DHCP configuration assigned and therefore cannot be converted to a network container.

- It overlaps with an existing network or network container, regardless of whether the existing network has a greater or smaller CIDR than the requested CIDR.

- Its network address or broadcast address is used by a fixed address or a host configured for DHCP.

Example

```
# Retrieve a particular network container
my @retrieved_objs = $session->get(
                        object   => "Infoblox::DHCP::IPv6NetworkContainer",
                        network  => "2001::/64"
);

$network_container = $retrieved_objs[0];

# Print the next available IPv6 network in the network container
print $network_container->next_available_network(cidr => 68) . "\n";

# This will retrieve the next 10 available networks and print the first one.
my $addressesref = $network_container->next_available_network(requested_num => 10, cidr => 68);
print @$addressesref[0] . "\n";

# This will retrieve the next 10 available networks and exclude two.
my $addressesref = $network_container->next_available_network(requested_num => 10, cidr => 72,
excluded => ['2001::100:0:0:0/72','2001::300:0:0:0/72']);
```

options( )

Use this method to set or retrieve the option values for a member DHCP server. DHCP options provide Network Container configuration settings and various services available for the Network Container.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Setting this method to a defined value implicitly sets the override_options method to "true". Setting the parameter to undefined causes the appliance to use the upper-level default and automatically resets the override_options attribute to "false".

Note that when the options method is set to a defined value and override_options is set to "false", the last operation takes precedence. Thus the sequence $object->options([$opt]); $object->override_options("false"); will set override_options to "false", and the sequence $object->override_options("false"); $object->options([$opt]); will result in override_options="true".

Parameter

The valid value is an array reference that contains Infoblox::DHCP::Option objects.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get options
my $options = $network_container->options();
#Modify options
my $option1 = Infoblox::DHCP::Option->new(
    name => "dhcp6.info-refresh-time",
    value => 60
);

$network_container->options([$option1]);
```

override_ddns_domainname( )

The override_ddns_domainname attribute controls whether the ddns_domainname value in the DHCP IPv6 Network Container is used, instead of the Grid default.

The override_ddns_domainname attribute can be specified explicitly. It is also set implicitly when ddns_domainname is set to a defined value.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

Set the parameter to "true" to override the Grid-level setting for ddns_domainname. Set the parameter to "false" to inherit the Grid-level setting for ddns_domainname.

The default value of this parameter is "false".

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Getting override_ddns_domainname
my $override_ddns_domainname=$network_container->override_ddns_domainname( );
#Modifying override_ddns_domainname
$network_container->override_ddns_domainname("true");
```

override_ddns_enable_option_fqdn( )

The override_ddns_enable_option_fqdn attribute controls whether the ddns_enable_option_fqdn value in the DHCP IPv6 Network Container is used, instead of the Grid default.

The override_ddns_enable_option_fqdn attribute can be specified explicitly. It is also set implicitly when ddns_domainname is set to a defined value.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

Set the parameter to "true" to override the Grid-level setting for ddns_enable_option_fqdn. Set the parameter to "false" to inherit the Grid-level setting for ddns_enable_option_fqdn.

The default value of this parameter is "false".

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Getting override_ddns_enable_option_fqdn
my $override_ddns_enable_option_fqdn=$network_container->override_ddns_enable_option_fqdn( );
#Modifying override_ddns_enable_option_fqdn
$network_container->override_ddns_enable_option_fqdn("true");
```

override_ddns_generate_hostname( )

The override_ddns_generate_hostname attribute controls whether the ddns_generate_hostname value in the DHCP IPv6 Network Container is used, instead of the Grid default.

The override_ddns_generate_hostname attribute can be specified explicitly. It is also set implicitly when ddns_generate_hostname is set to a defined value.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

Set the parameter to "true" to override the Grid-level setting for ddns_generate_hostname. Set the parameter to "false" to inherit the Grid-level setting for ddns_generate_hostname.

The default value of this parameter is "false".

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Getting override_ddns_generate_hostname
my $override_ddns_generate_hostname=$network_container->override_ddns_generate_hostname( );
#Modifying override_ddns_generate_hostname
$network_container->override_ddns_generate_hostname("true");
```

override_ddns_ttl( )

The override_ddns_ttl attribute controls whether the ddns_ttl value in the DHCP IPv6 Network Container is used, instead of the Grid default.

The override_ddns_ttl attribute can be specified explicitly. It is also set implicitly when ddns_ttl is set to a defined value.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

Set the parameter to "true" to override the Grid-level setting for ddns_ttl. Set the parameter to "false" to inherit the Grid-level setting for ddns_ttl.

The default value of this parameter is "false".

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Getting override_ddns_ttl
my $override_ddns_ttl=$network_container->override_ddns_ttl( );
#Modifying override_ddns_ttl
$network_container->override_ddns_ttl("true");
```

override_domain_name( )

The override_domain_name attribute controls whether the domain_name value in the DHCP IPv6 Network Container is used, instead of the Grid default.

The override_domain_name attribute can be specified explicitly. It is also set implicitly when domain_name is set to a defined value.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

Set the parameter to "true" to override the Grid-level setting for domain_name. Set the parameter to "false" to inherit the Grid-level setting for domain_name.

The default value of this parameter is "false".

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Getting override_domain_name
my $override_domain_name=$network_container->override_domain_name( );
#Modifying override_domain_name
$network_container->override_domain_name("true");
```

override_domain_name_servers( )

The override_domain_name_servers attribute controls whether the domain_name_servers value in the DHCP IPv6 Network Container is used, instead of the Grid default.

The override_domain_name_servers attribute can be specified explicitly. It is also set implicitly when domain_name_servers is set to a defined value.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

Set the parameter to "true" to override the Grid-level setting for domain_name_servers. Set the parameter to "false" to inherit the Grid-level setting for domain_name_servers.

The default value of this parameter is "false".

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Getting override_domain_name_servers
my $override_domain_name_servers=$network_container->override_domain_name_servers( );
#Modifying override_domain_name_servers
$network_container->override_domain_name_servers("true");
```

override_blackout_setting( )

The override_blackout_setting attribute controls whether the discovery_blackout_setting and port_control_blackout_setting values in the object are used, instead of the Grid default.

The override_blackout_setting attribute can be specified explicitly. It is also set implicitly when discovery_blackout_setting or port_control_blackout_setting is set to a defined value.

Parameter

Specify "true" to set the override_blackout_setting flag or "false" to deactivate/unset it. The default value is "false".

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
# Get attribute value
my $value = $network->override_blackout_setting();
```

```
# Modify attribute value
$network->override_blackout_setting('true');
```

override_enable_ddns( )

The override_enable_ddns attribute controls whether the enable_ddns value in the DHCP IPv6 Network Container is used, instead of the Grid default.

The override_enable_ddns attribute can be specified explicitly. It is also set implicitly when enable_ddns is set to a defined value.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

Set the parameter to "true" to override the Grid-level setting for enable_ddns. Set the parameter to "false" to inherit the Grid-level setting for enable_ddns.

The default value of this parameter is "false".

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Getting override_enable_ddns
my $override_enable_ddns=$network_container->override_enable_ddns( );
#Modifying override_enable_ddns
$network_container->override_enable_ddns("true");
```

override_options( )

The override_options attribute controls whether the options value in the DHCP IPv6 Network Container is used, instead of the Grid default.

The override_options attribute can be specified explicitly. It is also set implicitly when options is set to a defined value.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

Set the parameter to "true" to override the Grid-level setting for options. Set the parameter to "false" to inherit the Grid-level setting for options.

The default value of this parameter is "false".

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Getting override_options
my $override_options=$network_container->override_options( );
#Modifying override_options
$network_container->override_options("true");
```

override_preferred_lifetime( )

The override_preferred_lifetime attribute controls whether the preferred_lifetime value in the DHCP IPv6 Network Container is used, instead of the Grid default.

The override_preferred_lifetime attribute can be specified explicitly. It is also set implicitly when preferred_lifetime is set to a defined value.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

Set the parameter to "true" to override the Grid-level setting for options. Set the parameter to "false" to inherit the Grid-level setting for options.

The default value of this parameter is "false".

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Getting override_preferred_lifetime
my $override_preferred_lifetime=$network_container->override_preferred_lifetime( );
#Modifying override_preferred_lifetime
$network_container->override_preferred_lifetime("true");
```

override_recycle_leases( )

The override_recycle_leases attribute controls whether the recycle_leases value in the DHCP IPv6 Network Container is used, instead of the Grid default.

The override_recycle_leases attribute can be specified explicitly. It is also set implicitly when recycle_leases is set to a defined value.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

Set the parameter to "true" to override the Grid-level setting for options. Set the parameter to "false" to inherit the Grid-level setting for options.

The default value of this parameter is "false".

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Getting override_recycle_leases
my $override_recycle_leases=$network_container->override_recycle_leases( );
#Modifying override_recycle_leases
$network_container->override_recycle_leases("true");
```

override_mgm_private( )

The override_mgm_private attribute controls whether the mgm_private value in the object is used, instead of the parent-level default.

The override_mgm_private attribute can be specified explicitly. It is also set implicitly when mgm_private is set to a defined value.

Note that setting override_mgm_private to 'false' sets the original 'mgm_private' value to 'false'.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

Set the parameter to 'true' to override the parent-level setting for mgm_private. Set the parameter to 'false' to inherit the parent-level setting for mgm_private.

The default value of this parameter is 'false'.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Note that this member is loaded on on-demand basis, so to retrieve the value the object must have been retrieved from the session first.

Example

```
#Get override_mgm_private value
my $value = $object->override_mgm_private( );

#Modify override_mgm_private value
$object->override_mgm_private('true');
```

override_subscribe_settings( )

The override_subscribe_settings attribute controls whether the subscribe_settings method values of the member are used, instead of the Grid default.

The override_subscribe_settings attribute can be specified explicitly. It is also set implicitly when subscribe_settings is set to a defined value.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

Set the parameter to "true" to override the Grid-level setting for subscribe_settings. Set the parameter to "false" to inherit the Grid-level setting for subscribe_settings.

The default value of this parameter is "false".

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get override_subscribe_settings value
my $override_subscribe_settings = $object->override_subscribe_settings( );
#Modify override_subscribe_settings value
$object->override_subscribe_settings("true");
```

override_update_dns_on_lease_renewal( )

The override_update_dns_on_lease_renewal attribute controls whether the update_dns_on_lease_renewal value in the DHCP IPv6 Network Container is used, instead of the Grid default.

The override_update_dns_on_lease_renewal attribute can be specified explicitly. It is also set implicitly when update_dns_on_lease_renewal is set to a defined value.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

Set the parameter to "true" to override the Grid-level setting for update_dns_on_lease_renewal. Set the parameter to "false" to inherit the Grid-level setting for update_dns_on_lease_renewal.

The default value of this parameter is "false".

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
    #Getting override_update_dns_on_lease_renewal
    my $override_update_dns_on_lease_renewal=$network_container-
>override_update_dns_on_lease_renewal( );
    #Modifying override_update_dns_on_lease_renewal
    $network_container->override_update_dns_on_lease_renewal("true");
```

override_valid_lifetime( )

The override_valid_lifetime attribute controls whether the valid_lifetime value in the DHCP IPv6 Network Container is used, instead of the Grid default.

The override_valid_lifetime attribute can be specified explicitly. It is also set implicitly when valid_lifetime is set to a defined value.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

Set the parameter to "true" to override the Grid-level setting for options. Set the parameter to "false" to inherit the Grid-level setting for options.

The default value of this parameter is "false".

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Getting override_valid_lifetime
my $override_valid_lifetime=$network_container->override_valid_lifetime( );
#Modifying override_valid_lifetime
$network_container->override_valid_lifetime("true");
```

preferred_lifetime ( )

Use this method to set or retrieve the preferred lifetime value of a DHCP IPv6 Network Container object.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Setting this method to a defined value implicitly sets the override_preferred_lifetime method to "true". Setting the parameter to undefined causes the appliance to use the Grid default and automatically resets the override_preferred_lifetime attribute to "false".

Note that when preferred_lifetime is set to a defined value and override_preferred_lifetime is set to "false", the last operation takes precedence. Thus the sequence $object->preferred_lifetime(691200); $object->override_preferred_lifetime("false"); will set override_preferred_lifetime to "false", and the sequence $object->override_preferred_lifetime("false"); $object->preferred_lifetime(691200); will result in override_preferred_lifetime="true".

Parameter

A 32-bit integer (valid range from 1 to 4294967295) that represents the lifetime, in seconds. The default value is 27000.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get Preferred lifetime
my $preferred_lifetime = $network_container->preferred_lifetime();
#Modify Prefered lifetime
$network_container->preferred_lifetime(864000);
```

port_control_blackout_setting

Use this method to set or retrieve the port control blackout setting.

Setting this method to a defined value implicitly sets the override_blackout_setting method to "true". If discovery_blackout_setting is undefined, setting the parameter to undefined causes the appliance to use the Grid default and automatically resets the override_blackout_setting attribute to "false".

Note that when port_control_blackout_setting is set to a defined value and override_blackout_setting is set to "false", the last operation takes precedence. Thus the sequence $network->port_control_blackout_setting($blackout); $network->override_blackout_setting("false"); will set override_blackout_setting to "false", and the sequence $network->override_blackout_setting("false"); $network->port_control_blackout_setting($blackout); will result in override_blackout_setting="true".

Parameter

Valid value is an Infoblox::Grid::Discovery::Properties::BlackoutSetting object.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get port_control_blackout_setting
my $port_control_blackout_setting = $network->port_control_blackout_setting();
#Modify port_control_blackout_setting
$network->port_control_blackout_setting($blackout);
```

recycle_leases( )

Use this method to set or retrieve the recycle_leases flag at the network container level.

If the flag is enabled, the leases are kept in the recycle bin until one week after expiration. When the flag is disabled, the leases are irrecoverably deleted.

Setting this method to a defined value implicitly sets the override_recycle_leases method to "true". Setting the parameter to undefined causes the appliance to use the Grid default and automatically resets the override_recycle_leases attribute to "false".

Note that when recycle_leases is set to a defined value and override_recycle_leases is set to "false", the last operation takes precedence.

Thus the sequence $object->recycle_leases("true"); $object->override_recycle_leases("false"); will set override_recycle_leases to "false", and the sequence $objct->override_recycle_leases("false"); $object->recycle_leases("true"); will result in override_recycle_leases="true".

Parameter

Specify "true" to enable the recycle_leases flag or "false" to disable it. The default value is undefined.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get recycle_leases
my $recycle_leases = $network_container->recycle_leases();
#Modify recycle_leases
$network_container->recycle_leases("false");
```

remove_subnets( )

Use this method to set or retrieve the remove_subnets flag of a DHCP IPv6 Network Container object.

You can set this flag to false, if you want to remove the network, but keep all subnets under it. This flag is used only when removing DHCP IPv6 Network Container objects. It is not retrieved or used for any other operation.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

Specify "false" to keep the subnets, or "true" to remove them. The default value is undefined, which is to remove all subnets.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Set remove_subnets before submitting for removal
$network_container->remove_subnets("false");
#Remove the container of the subnets
$session->remove($network_container);
```

restart_if_needed( )

Use this method to set the flag that indicates whether DHCP service should be restarted on a member that is serving DHCP for the changed object referenced in a cloud API request.

Parameter

Specify true to restart the DHCP service.

Returns

None

Example

```
# Modify
$network->restart_if_needed('true');
```

rir( )

Use this method to retrieve the name of the Regional Internet Registry (RIR) that allocated this address space. This is a read-only attribute.

Parameter

None

Returns

The method returns the attribute value that can be 'RIPE' or 'NONE'.

Example

```
#Get rir
my $rir = $network_container->rir();
```

rir_organization( )

Use this method to set or retrieve the Regional Internet Registry (RIR) organization object.

Parameter

An Infoblox::Grid::RIR::Organization object.

You must enable support for RIR/SWIP at the Grid level before you can set attribute values. See Infoblox::Grid->enable_rir_swip() method.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get rir_organization
my $rir_organization = $network_container->rir_organization();
#Modify rir_organization
$network_container->rir_organization($rir_org);
```

rir_registration_action( )

Use this method to set the registration action of Regional Internet Registry (RIR). This is a write-only attribute.

Parameter

Valid values are 'NONE', 'CREATE', 'MODIFY' or 'DELETE'.

Returns

The method returns true when the modification succeeds, and returns false when the operation fails.

Example

```
#Set rir_registration_action
$network_container->rir_registration_action('CREATE');
```

rir_registration_status( )

Use this method to set or retrieve the registration status of Regional Internet Registry (RIR).

Parameter

The registration status of Regional Internet Registry (RIR), which can be 'REGISTERED' or 'NOT_REGISTERED'.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get rir_registration_status
my $rir_registration_status = $network_container->rir_registration_status();
#Modify rir_registration_status
$network_container->rir_registration_status('NOT_REGISTERED');
```

last_rir_registration_update_sent( )

Use this method to retrieve the last registration update timestamp of Regional Internet Registry (RIR). This is a read-only attribute.

Parameter

None

Returns

The method returns the attribute value.

Example

```
#Get last_rir_registration_update_sent
my $last_rir_registration_update_sent = $network_container->last_rir_registration_update_sent();
```

last_rir_registration_update_status( )

Use this method to retrieve the last registration update status of Regional Internet Registry (RIR). This is a read-only attribute.

Parameter

None

Returns

The method returns the attribute value.

Example

```
#Get last_rir_registration_update_status
my $last_rir_registration_update_status = $network_container->last_rir_registration_update_status();
```

same_port_control_discovery_blackout( )

Use this method to set or retrieve the flag that enables using the same discovery blackout for port control blackout.

Parameter

Specify 'true' to use same discovery blackout for port control blackout or 'false' to disable it. The default value is 'false'.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
# Get attribute value
my $value = $network_container->same_port_control_discovery_blackout();
# Modify attribute value
$network_container->same_port_control_discovery_blackout('true');
```

send_rir_request( )

Use this method to set the flag that indicates whether the appliance should send an API or email request for Regional Internet Registry (RIR). This is a write-only attribute.

Parameter

Specify "true" to set the send_rir_request flag or "false" to unset it.

Returns

The method returns true when the modification succeeds, and returns false when the operation fails.

Example

```
#Set send_rir_request
$network_container->send_rir_request('true');
```

subscribe_settings( )

Use this method to set or retrieve the Cisco ISE subscribe settings.

Setting this method to a defined value implicitly sets the override_subscribe_settings method to "true". Setting the parameter to undefined causes the appliance to use the Cisco ISE endpoint default and automatically resets the override_subscribe_settings method to "false".

Note that when subscribe_settings is set to a defined value and override_subscribe_settings is set to "false", the last operation takes precedence. Thus the sequence $object->subscribe_settings($value); $object->override_subscribe_settings("false"); will set override_subscribe_settings to "false", and the sequence $object->override_subscribe_settings("false"); $object->subscribe_settings($value); will result in override_subscribe_settings="true".

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

The valid value is an Infoblox::CiscoISE::SubscribeSetting object.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get subscribe_settings value
my $subscribe_settings = $object->subscribe_settings();
#Modify subscribe_settings value
$object->subscribe_settings($settings);
```

delete_reason( )

Use this method to specify the reason for deleting the Regional Internet Registry (RIR) registration request. This is a write-only attribute. This field is required only if 'rir_registration_action' is specified as 'DELETE'

Parameter

The reason for deleting the RIR registration request in string format with a maximum of 256 bytes.

Returns

The method returns true when the modification succeeds, and returns false when the operation fails.

Example

```
#Set delete_reason
$network_container->delete_reason('invalid');
```

unmanaged( )

Use this method to set or retrieve the network status, which indicates whether the network is unmanaged or not.

Parameter

Specify true to mark the network as unmanaged or false as managed.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
# Get network status
$unmanaged = $network_container->unmanaged();
# Modify network status
$network_container->unmanaged('true');
```

update_dns_on_lease_renewal( )

The update_dns_on_lease_renewal attribute controls whether the DHCP server updates DNS when a DHCP lease is renewed.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Setting this method to a defined value implicitly sets the override_update_dns_on_lease_renewal method to "true". Setting the parameter to undefined causes the appliance to use the Grid default and automatically resets the override_update_dns_on_lease_renewal attribute to "false".

Note that when update_dns_on_lease_renewal is set to a defined value and override_update_dns_on_lease_renewal is set to "false", the last operation takes precedence. Thus the sequence $object->update_dns_on_lease_renewal("true"); $object->override_update_dns_on_lease_renewal("false"); will set override_update_dns_on_lease_renewal to "false", and the sequence $object->override_update_dns_on_lease_renewal("false"); $object->update_dns_on_lease_renewal("true"); will result in override_update_dns_on_lease_renewal="true".

Parameter

Specify "true" to set the update_dns_on_lease_renewal flag or "false" to deactivate/unset it.

The default value of this parameter is "false".

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get update_dns_on_lease_renewal
my $update_dns_on_lease_renewal=$network_container->update_dns_on_lease_renewal( );
#Modify update_dns_on_lease_renewal
$network_container->update_dns_on_lease_renewal("true");
```

valid_lifetime ( )

Use this method to set or retrieve the valid lifetime value of a DHCP IPv6 Network Container object.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Setting this method to a defined value implicitly sets the override_valid_lifetime method to "true". Setting the parameter to undefined causes the appliance to use the Grid default and automatically resets the override_valid_lifetime attribute to "false".

Note that when valid_lifetime is set to a defined value and override_valid_lifetime is set to "false", the last operation takes precedence. Thus the sequence $object->valid_lifetime(691200); $object->override_valid_lifetime("false"); will set override_valid_lifetime to "false", and the sequence $object->override_valid_lifetime("false"); $object->valid_lifetime(691200); will result in override_valid_lifetime="true".

Parameter

A 32-bit integer (valid range from 1 to 4294967295) that represents the valid lifetime, in seconds. The default value is 43200.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get valid lifetime
my $valid_lifetime = $network_container->valid_lifetime();
#Modify valid lifetime
$network_container->valid_lifetime(864000);
```

zone_associations( )

Use this method to set or retrieve the associated zone information.

Parameter

The valid value is an array reference that contains Infoblox::DNS::Zone object(s). The default value is an empty list. If the network container has a parent, it is possible to use the zone associations of the parent by setting the parameter to undef.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails. If you did not specify a parameter, the method returns the attribute value.

Examples

```
#Getting zone associations
my $zone_associations = $network_container->zone_associations();
my $zone1 = Infoblox::DNS::Zone->new(
      name     => "test1.com",
);
my $zone2 = Infoblox::DNS::Zone->new(
      name     => "test2.com",
   );
#Modifying zone_associations
$network_container->zone_associations([$zone1, $zone2]);
#Clear zone_associations
$network_container->zone_associations([]);
##Remove the override for zone_associations
$network_container->zone_associations(undef)
```

SAMPLE CODE

The following sample code demonstrates the different functions that can be applied to an object such as add, search, modify, and remove. This sample also includes error handling for the operations.

#Preparation prior to a DHCP IPv6 Network Container object insertion

```
#PROGRAM STARTS: Include all the modules that will be used
```

```
use strict;
use Infoblox;

#Create a session to the Infoblox appliance

my $session = Infoblox::Session->new(
    master   => "192.168.1.2",
    username => "admin",
    password => "infoblox"
);
unless ($session) {
   die("Construct session failed: ",
       Infoblox::status_code() . ":" . Infoblox::status_detail());
}
print "Session created successfully\n";
```

#Create a DHCP IPv6 Network Container object

```
my $network_container = Infoblox::DHCP::IPv6NetworkContainer->new(
            network => "2001::/64",
            comment => "add network container",
             );

my $response = $session->add($network_container)
       or die("Add Network Container failed: ",
             $session->status_code() . ":" . $session->status_detail());

print "Network Container added successfully\n";

#Search for a specific Network Container

#Search the network container
my @retrieved_objs = $session->search(
    object  => "Infoblox::DHCP::IPv6NetworkContainer",
    network => "2001.*"
);
my $object = $retrieved_objs[0];
unless ($object) {
    die("Search Network Container failed: ",
        $session->status_code() . ":" . $session->status_detail());
}
print "Search Network Container found at least 1 matching entry\n";
```

#Get and modify an IPv6 Network Container attribute

```
#Get Network Container through the session
my @retrieved_objs = $session->get(
           object =>"Infoblox::DHCP::IPv6NetworkContainer" ,
           network => "2001::/64"
        );

my $object = $retrieved_objs[0];

unless ($object) {
    die("Get Network Container object failed: ",
        $session->status_code() . ":" . $session->status_detail());
}
print "Get Network Container found at least 1 matching entry\n";
```

#Modify one of the attributes of the specified IPv6 network container attribute

```
$object->comment("modified comment"); #Apply the changes

$session->modify($object)
    or die("Modify Network Container attribute failed: ",
           $session->status_code() . ":" . $session->status_detail());

print "Network Container object modified successfully \n";
```

#Remove a Network Container Object

```
 #Get Network Container object through the session
 my @retrieved_objs = $session->get(
           object  => "Infoblox::DHCP::IPv6NetworkContainer" ,
           network => "2001::"
       );
```

```
    my $object = $retrieved_objs[0];

    unless ($object) {
     die("Get Network Container object failed: ",
         $session->status_code() . ":" . $session->status_detail());
     }

 print "Get Network Container object found at least 1 matching entry\n";
```

#Submit the object for removal

```
 $session->remove($object)
     or die("Remove Network Container failed: ",
         $session->status_code() . ":" . $session->status_detail());

 print "Network Container removed successfully \n";
```

####PROGRAM ENDS####

AUTHOR

Infoblox Inc. http://www.infoblox.com/

SEE ALSO

Infoblox::Session, Infoblox::DHCP::View, Infoblox::DHCP::IPv6Network, Infoblox::DHCP::Network, Infoblox::DHCP::NetworkContainer, Infoblox::Session->get(), Infoblox::Session->search(), Infoblox::Session->add(), Infoblox::Session->remove(), Infoblox::Session->modify(), Infoblox::Grid::MSServer::AdUser::Data

---

COPYRIGHT

Copyright (c) 2017 Infoblox Inc.

Infoblox::DHCP::NetworkContainer - DHCP Network Container object

- NAME
- DESCRIPTION
- CONSTRUCTOR
- SESSION METHODS
  - Infoblox::Session->add( )
  - Infoblox::Session->get( )
  - Infoblox::Session->modify( )
  - Infoblox::Session->remove( )
  - Infoblox::Session->search( )
- METHODS
  - authority( )
  - auto_create_reversezone( )
  - bootfile( )
  - bootserver( )
  - comment( )
  - cloud_info( )
  - ddns_domainname( )
  - ddns_generate_hostname( )
  - ddns_server_always_updates( )
  - ddns_ttl( )
  - ddns_update_fixed_addresses( )
  - ddns_use_option81( )
  - deny_bootp( )
  - discovery_basic_poll_setting( )
  - discovery_member( )
  - discover_now_status( )
  - discovery_blackout_setting( )
  - discovery_engine_type( )
  - email_list( )
  - enable_ddns( )
  - enable_dhcp_thresholds( )
  - enable_email_warnings( )
  - enable_discovery( )
  - enable_immediate_discovery( )
  - enable_snmp_warnings( )
  - endpoint_sources( )
  - extattrs( )
  - extensible_attributes( )
  - high_water_mark( )
  - high_water_mark_reset( )
  - ignore_id( )
  - ignore_mac_addresses( )
  - ignore_dhcp_option_list_request( )
  - ipam_email_addresses( )
  - ipam_trap_settings( )
  - ipam_threshold_settings( )
  - mgm_private( )
  - mgm_private_overridable( )
  - ms_ad_user_data( )
  - lease_scavenge_time( )
  - logic_filters( )
  - low_water_mark( )
  - low_water_mark_reset( )
  - network( )
  - override_blackout_setting( )
  - nextserver( )
  - options( )
  - override_ddns_domainname( )
  - override_ddns_ttl( )
  - override_ignore_id( )
  - override_ipam_email_addresses( )
  - override_ipam_threshold_settings( )
  - override_ipam_trap_settings( )
  - override_logic_filters( )
  - override_lease_scavenge_time( )
  - override_mgm_private( )
  - override_subscribe_settings( )
  - override_update_dns_on_lease_renewal( )
  - port_control_blackout_setting
  - pxe_lease_time( )
  - recycle_leases( )

- remove_subnets( )
- network_container( )
- network_view( )
- next_available_network( )
- resize( )
- restart_if_needed( )
- rir( )
- rir_organization( )
- rir_registration_action( )
- rir_registration_status( )
- last_rir_registration_update_sent( )
- last_rir_registration_update_status( )
- same_port_control_discovery_blackout( )
- send_rir_request( )
- subscribe_settings( )
- delete_reason( )
- unmanaged( )
- update_dns_on_lease_renewal( )
- zone_associations( )

- SAMPLE CODE
- AUTHOR
- SEE ALSO
- COPYRIGHT

NAME

Infoblox::DHCP::NetworkContainer - DHCP Network Container object

---

DESCRIPTION

A network can contain child networks. The network that contains the child network is called a network container.

CONSTRUCTOR

```
  my $network = Infoblox::DHCP::NetworkContainer->new(
                network                          => $ipv4_subnet/prefix,               #Required, when
creating a network container
                auto_create_reversezone          => "true" | "false",                  #Optional / Default
is "false"
                comment                          => $string| undef,                    #Optional / Default
is undefined
                remove_subnets                   => "true" | "false",                  #Optional / Default
is undefined
                rir_organization                 => $rir_org,                          #Optional / An
Infoblox::Grid::RIR::Organization object
                rir_registration_action          => 'NONE' | 'CREATE' | 'MODIFY' | 'DELETE',  # Optional /
write-only
                rir_registration_status          => 'NOT_REGISTERED' | 'REGISTERED',   #Optional / Default
is 'NOT_REGISTERED'
                send_rir_request                 => 'true' | 'false',                  #Optional / write-
only
                delete_reason                    => $string,                           #Optional / write-
only
                                                                                       #Required only if
'rir_registration_action' is specified as 'DELETE'
                mgm_private                      => "true" | "false",                  #Optional / Default
is "false"
                network_container                => $ipv4_subnet/prefix,               #Optional / Default
is "/"
                network_view                     => $NetworkView,                      #Optional / Default
is "default" network view
                zone_associations                => [$zone1, $zone2, ... ] | undef,    #Optional / Default
is an empty list
                extattrs                         => { $string => $extattr, ... },      #Optional / Default
is undefined
                extensible_attributes            => { $string => $string | $num, $string => [ $string |
$num, ... ], ... } #Required if 'rir_organization' attribute is defined. Otherwise is optional
                discovery_blackout_setting       => $blackout,                         #Optional
                port_control_blackout_setting    => $blackout,                         #Optional
                override_blackout_setting        => "true" | "false",                  #Optional / Default
value is "false"
                same_port_control_discovery_blackout  => "true" | "false",             #Optional / Default
value is "false"
                ipam_trap_settings                => $ipam_trap,                       #Optional / An
Infoblox::Grid::SNMP::IPAMTrap object
                override_ipam_trap_settings      => "true" | "false",                  #Optional / Default
```

```
value is "false"
                ipam_threshold_settings          => $ipam_threshold,                #Optional / An
Infoblox::Grid::SNMP::IPAMThreshold object
                override_ipam_threshold_settings => "true" | "false",               #Optional / Default
value is "false"
                ipam_email_addresses             => [$email1, $email2, ...],        #Optional / Default
is an empty list
                override_ipam_email_addresses    => "true" | "false",               #Optional / Default
value is "false"

                authority                        => "true" | "false"| undef,        #Optional / Default
is undefined
                bootfile                         => $filename | undef,              #Optional / Default
is undefined
                bootserver                       => $ipv4addr | $fqdn | undef,      #Optional / Default
is undefined
                ddns_domainname                  => $string| undef,                 #Optional / Default
is undefined
                ddns_generate_hostname           => "true" | "false" | undef,       #Optional / Default
is undefined
                ddns_server_always_updates       => "true" | "false",               #Optional / Default
is "true"
                ddns_ttl                         => $num,                           #Optional / Default
is "0"
                ddns_use_option81                => "true" | "false" | undef,       #Optional / Default
is undefined
                ddns_update_fixed_addresses      => "true" | "false" | undef,       #Optional / Default
is undefined
                deny_bootp                       => "true" | "false" | undef,       #Optional / Default
is undefined
                email_list                       => [ "admin\@test.com",...] | undef,  #Optional / Default
is undefined
                enable_ddns                      => "true" | "false" | undef,       #Optional / Default
is undefined
                enable_dhcp_thresholds           => "true" | "false",               #Optional / Default
is "false"
                enable_email_warnings            => "true" | "false",               #Optional / Default
is "false"
                enable_snmp_warnings             => "true" | "false",               #Optional / Default
is "false"
                high_water_mark                  => $number,                        #Optional / Default
is "95"
                high_water_mark_reset            => $number,                        #Optional / Default
is "95"
                ignore_dhcp_option_list_request  => "true" | "false" | undef,       #Optional / Default
is undefined
                ignore_id                        => 'NONE' | 'CLIENT' | 'MACADDR',  #Optional / Default
is 'NONE'
                ignore_mac_addresses             => [ "mac_addr", ... ] | undef     #Optional / Default
is undefined
                lease_scavenge_time              => $number,                        #Optional / Default
is "-1"
                low_water_mark                   => $number,                        #Optional / Default
is "0"
                low_water_mark_reset             => $number,                        #Optional / Default
is "0"
                nextserver                       => $ipv4addr | $fqdn | undef,      #Optional / Default
is undefined
                options                          => [$Option1,$Option2,...],        #Optional / Default
is empty list
                override_ddns_domainname         => "true" | "false",               #Optional / Default
value is "false"
                override_ddns_ttl                => "true" | "false",               #Optional / Default
value is "false"
                override_lease_scavenge_time     => "true" | "false",               #Optional / Default
value is "false"
                override_mgm_private             => "true" | "false",               #Optional / Default
value is "false"
                override_update_dns_on_lease_renewal => "true" | "false",           #Optional / Default
value is "false"
                override_ignore_id               => "true" | "false",               #Optional / Default
value is "false"
                pxe_lease_time                   => $num | undef,                   #Optional / Default
is undefined
                recycle_leases                   => "true" | "false" | undef,       #Optional / Default
is undefined
                update_dns_on_lease_renewal      => "true" | "false",               #Optional / Default
value is "false"
                logic_filters                    => [ $filter_name | $filter_obj, ...], #Optional /
Default is undefined
                override_logic_filters           => "true' | 'false",                #Optional /
Default value is "false"
                subscribe_settings          => $subscribe_settings,                 #Optional /
Default is undefined
```

```
                override_subscribe_settings => 'true' | 'false',                        #Optional /
Default is 'false'
                discovery_engine_type       => 'NETMRI' | 'NETWORK_INSIGHT' | 'VDISCOVERY' | 'UNKNOWN' |
'NONE', #Optional / Default is 'NONE'
                                );
```

You cannot set both extattrs and extensible_attributes at the same time.

SESSION METHODS

This section describes all the methods in an Infoblox::Session module that you can apply to a DHCP Network Container object.

Infoblox::Session->add( )

Use this method to add a DHCP Network Container object to the Infoblox appliance. See Infoblox::Session->add() for parameters and return values.

Example

```
#Adding a network container.
#Construct an object
my $network_container = Infoblox::DHCP::NetworkContainer->new(
      network => "20.0.0.0/8",
);

# Submit for adding a network container
my $response = $session->add( $network_container );
```

Infoblox::Session->get( )

Use this method to retrieve all the matching DHCP Network Container objects from the Infoblox appliance. See Infoblox::Session->get() for parameters and return values.

'discover_now_status' will not be returned by default, use 'return_methods' to request it if needed.

Key References

```
 Apply the following attributes to get a specific DHCP Network Container object:

  network               - Optional. The IP address and netmask of a network in CIDR format.
  network_container     - Optional. The network container, in CIDR format, to which this network
container belongs.
  network_view          - Optional. The network view in which the network is located. By default, all
network views are searched.
  extattrs              - Optional. A hash reference containing extensible attributes.
  extensible_attributes - Optional. A hash reference containing extensible attributes.
  comment               - Optional. A comment in string format.
```

Examples

```
 my  @result_array = $session->get(
      object       => "Infoblox::DHCP::NetworkContainer",
      network      => "20.0.0.0/8",
      network_view  => "default"
 );

 my  @result_array = $session->get(
      object               => "Infoblox::DHCP::NetworkContainer",
      extensible_attributes => { 'Site' => 'Santa Clara' }
 );

 my  @result_array = $session->get(
      object  => "Infoblox::DHCP::NetworkContainer",
      comment => "my-comment",
 );
```

Infoblox::Session->modify( )

Use this method to modify a DHCP Network Container object in the Infoblox appliance. See Infoblox::Session->modify() for parameters and return values.

Example

```
#Use method to modify comment.
$network_container->comment("modified comment");
# Submit modification
my $response = $session->modify( $network_container );
```

Infoblox::Session->remove( )

Use this method to remove an object from the Infoblox appliance. See Infoblox::Session->remove() for parameters and return values.

To remove a specifc object, first use `get()` or `search()` to retrieve the specific object, and then submit this object for removal.

Key References

```
remove_options - Optional. A hash reference containing the following attributes:

+-------------------------------------------------------------------------------------------+
| reason             - Optional. The reason for removing the network container in string    |
|                      format with a maximum of 256 bytes.                                  |
|                      Used when 'rir_organization' attribute has been defined.             |
| send_rir_request   - Optional. Boolean flag that indicates whether the appliance should  |
|                      send an API or email request for Regional Internet Registry (RIR).   |
|                      Used when 'rir_organization' attribute has been defined.             |
+-------------------------------------------------------------------------------------------+
```

Example

```
 # Get the object under the same name
my  @result_array = $session->get(

       object       => "Infoblox::DHCP::NetworkContainer" ,
       network      => "20.0.0.0/8",
       network_view  => "default"
         );

 # Find the desired object in the retrieved list.
 my $network_container = $result_array[0];

 # Submit for removal
 my $response = $session->remove(
     $network_container,
     remove_options => {'reason' => 'Moved to B3', send_rir_request => 'true'} );
```

Infoblox::Session->search( )

Use this method to search for a DHCP Network Container object in the Infoblox appliance. See Infoblox::Session->search() for parameters and return values.

'discover_now_status' will not be returned by default, use 'return_methods' to request it if needed.

Key References

```
 Apply the following attributes to search for a DHCP Network Container object:

  network              - Optional. The IP address and netmask of a network in CIDR format (regular
expression).
  network_container    - Optional. The network container, in CIDR format, to which this network
container belongs.
  network_view         - Optional. The network view in which the network is located. By default, all
network views are searched.
  extattrs             - Optional. A hash reference containing extensible attributes.
  extensible_attributes - Optional. A hash reference containing extensible attributes.
  comment              - Optional. A comment in string format (regular expression).
```

For more information about searching extensible attributes, see Infoblox::Grid::ExtensibleAttributeDef/Searching Extensible Attributes.

Examples

```
 # search for all DHCP Network Container objects
     my @retrieved_objs = $session->search(
       object       => "Infoblox::DHCP::NetworkContainer",
       network      => '20\.0\.0\..*',
       network_view => "default"
     );

 # search network by comment
     my @retrieved_objs = $session->search(
       object    => "Infoblox::DHCP::NetworkContainer",
       comment   => "my-comment",
     );

 # search for all DHCP Network Container objects defining "Santa Clara" for "Site" extensible
attribute
     my @retrieved_objs = $session->search(
       object               => "Infoblox::DHCP::NetworkContainer",
       extensible_attributes => { 'Site' => 'Santa Clara' }
     );
```

METHODS

This section describes all the methods that can be used to configure and retrieve the attribute values of a DHCP Network Container object.

authority( )

Use this method to set or retrieve the authority of a DHCP Network Container object. This setting overrides the upper-level settings.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

Specify "true" to set the authority or "false" to deactivate/unset it. The default value for this field is undefined, which indicates that this attribute inherit the upper-level setting.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
# Get authority
my $authority = $network_container->authority()
# Modify authority
$network_container->authority("true");
```

auto_create_reversezone( )

Use this method to set the auto_create_reversezone flag of a DHCP Network Container object.

Include the specified parameter to set the attribute value. This is a write-only attribute and cannot be retrieved after it is set.

Parameter

Specify "true" to set the auto_create_reversezone flag or "false" to deactivate/unset it. The default value for this field is "false".

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

Example

```
# Modify auto_create_reversezone
$network_container->auto_create_reversezone("true");
```

bootfile( )

Use this method to set or retrieve a bootfile attribute of a DHCP Network Container object. You can configure the DHCP server to support clients that use boot file name option in their DHCPREQUEST messages. This setting overrides the upper-level settings. The default value is undefined, which indicates that this attribute inherits the upper-level setting.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

This option is overridden independently from Infoblox::DHCP::NetworkContainer->bootserver() and Infoblox::DHCP::NetworkContainer->nextserver().

Parameter

The name, in string format, of the file that the client must download. The default value is undefined.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
# Get bootfile
my $bootfile = $network_container->bootfile();
# Modify bootfile
$network_container->bootfile("bootfile1");
#Un-override bootfile
$network_container->bootfile(undef);
```

bootserver( )

Use this method to set or retrieve a bootserver attribute of a DHCP Network Container object. You can specify the name and/or IP address of the boot server that host needs to boot. This setting overrides the upper-level settings. The default value is undefined, which indicates that this attribute inherits the upper-level setting.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

This option is overridden independently from Infoblox::DHCP::NetworkContainer->bootfile() and Infoblox::DHCP::NetworkContainer->nextserver().

Parameter

The boot server IP address or name in FQDN (Fully Qualified Domain Name) format. The FQDN consists of the host name followed by the domain name (example: abc.com). A boot server name can have a maximum of 256 bytes. The default value is undefined.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
# Get bootserver
my $bootserver = $network_container->bootserver();
# Modify bootserver
$network_container->bootserver("abc.domain.com");
#Un-override bootserver
$network_container->bootserver(undef);
```

comment( )

Use this method to set or retrieve a descriptive comment for a DHCP Network Container object.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

Enter a descriptive comment for the network container in string format with a maximum of 256 bytes.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
# Get comment
my $comment = $network_container->comment();
# Modify comment
$network_container->comment("add a network");
```

cloud_info( )

Use this method to set or retrieve cloud API related information for the Infoblox::DHCP::NetworkContainer object.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

The valid value is a Infoblox::Grid::CloudAPI::Info object.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
# Get cloud_info
my $cloud_info = $network_view->cloud_info();
# Modify cloud_info
$network_view->cloud_info($cloud_info_object);
```

ddns_domainname( )

Use this method to set or retrieve the ddns_domainname value of a DHCP Network Container object.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Setting this method to a defined value implicitly sets the override_ddns_domainname method to "true". Setting the parameter to undefined causes the appliance to use the Grid default and automatically resets the override_ddns_domainname attribute to "false".

Note that when ddns_domainname is set to a defined value and override_dddns_domainname is set to "false", the last operation takes precedence. Thus the sequence $object->ddns_domainname("testdomain"); $object->override_ddns_domainname("false"); will set override_ddns_domainname to "false", and the sequence $object->override_ddns_domainname("false"); $object->ddns_domainname("testdomain"); will result in override_ddns_domainname="true".

Parameter

The dynamic DNS domain name in string format.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get ddns_domainname
my $ddns_domainname = $network_container->ddns_domainname();
#Modify ddns_domainname
$network_container->ddns_domainname("test_domain.com");
```

ddns_generate_hostname( )

Use this method to set or retrieve the ddns_generate_hostname flag of a DHCP Network Container object. Enable this method to allow only the DHCP server for generating hostname. This setting overrides the upper-level settings.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

Specify "true" to generate hostname or "false" to deactivate/unset it.The default value for this field is undefined, which indicates that this attribute inherit the member-level setting.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
# Get ddns_generate_hostname
my $ddns_generate_hostname = $network_container->ddns_generate_hostname()
# Modify ddns_generate_hostname
$network_container->ddns_generate_hostname("true");
#Un-override
$network_container->ddns_generate_hostname(undef);
```

ddns_server_always_updates( )

Use this method to set or retrieve the ddns_server_always_updates flag of a DHCP Network Container object. Enable this method to allow only the DHCP server to update DNS regardless of the requests from the DHCP clients.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

Specify "true" to set the flag that enables the DHCP server to always updates DNS or "false" to deactivate/unset it.The default value for this field is "true".

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
# Get ddns_server_always_updates
my $ddns_server_always_updates = $network_container->ddns_server_always_updates()
# Modify ddns_server_always_updates
```

```
$network_container->ddns_server_always_updates("true");
```

ddns_ttl( )

Use this method to set or retrieve the DNS update Time to Live (TTL) value of a DHCP Network Container object.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Setting this method to a defined value implicitly sets the override_ddns_ttl method to "true". Setting the parameter to undefined causes the appliance to use the Grid default and automatically resets the override_ddns_ttl attribute to "false".

Note that when ddns_ttl is set to a defined value and override_dddns_ttl is set to "false", the last operation takes precedence. Thus the sequence $object->ddns_ttl(12); $object->override_ddns_ttl("false"); will set override_ddns_ttl to "false", and the sequence $object->override_ddns_ttl("false"); $object->ddns_ttl(12); will result in override_ddns_ttl="true".

Parameter

A 32-bit integer (range from 0 to 4294967295) that represents the duration, in seconds, the update is cached. Zero indicates that the update is not cached. The default value is 0.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get DNS Update ttl
my $dns_ttl = $network_container->ddns_ttl();
#Modify DNS Update ttl
$network_container->ddns_ttl(1200);
```

ddns_update_fixed_addresses( )

Use this method to set or retrieve the ddns_update_fixed_addresses flag of a DHCP Network Container object. By default, the DHCP server does not update DNS when it allocates a fixed address to a client. You can configure the DHCP server to update the A and PTR records of a client with a fixed address. When this feature is enabled and the DHCP server adds A and PTR records for a fixed address, the DHCP server never discards the records. This setting overrides the upper-level settings.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

Specify "true" to enable the appliance to update DNS when it assigns a fixed address to a client or "false" to disable it. The default value for this field is undefined, which indicates that this attribute inherit the member-level setting.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
# Get ddns_update_fixed_addresses
my $ddns_update_fixed_addresses= $network_container->ddns_update_fixed_addresses()
# Modify ddns_update_fixed_addresses
$network_container->ddns_update_fixed_addresses("true");
#Un-override
$network_container->ddns_update_fixed_addresses(undef);
```

ddns_use_option81( )

Use this method to set or retrieve the ddns_use_option81 flag of a DHCP Network Container object. Enable this method to define the settings for option 81 at the network level. This setting overrides the member-level settings.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

Specify "true" to Configure Support for Option 81 or "false" to deactivate/unset it. The default value for this field is undefined, which indicates that this attribute inherit the member-level setting.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
# Get ddns_use_option81
my $ddns_use_option81 = $network_container->ddns_use_option81()
# Modify ddns_use_option81
$network_container->ddns_use_option81("true");
# un-override ddns_use_option81
$network_container->ddns_use_option81(undef);
```

deny_bootp( )

Use this method to set or retrieve the deny_bootp flag of a DHCP Network Container object.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

Specify "true" to set the deny_bootp flag or "false" to deactivate/unset it. The default value is undefined, which indicates that this attribute inherit the upper-level setting.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get deny_bootp
my $deny_bootp = $network_container->deny_bootp();
#Modify deny_bootp
$network_container->deny_bootp("true");
```

discovery_basic_poll_setting( )

Use this method to set or retrieve the discovery_basic_poll_setting value of a DHCP Network Container object.

Include the specified parameter to set the attribute value.

Parameter

The valid value is object of the Infoblox::Grid::Discovery::BasicPollSettings manpage.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get discovery_basic_poll_setting
my $discovery_basic_poll_setting = $network->discovery_basic_poll_setting();
#Modify discovery_basic_poll_setting
$network->discovery_basic_poll_setting($poll_setting);
```

discovery_member( )

Use this method to set or retrieve the discovery_member value of a DHCP Network Container object.

Parameter

The discovery member name in string format.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get discovery_member
my $discovery_member = $network->discovery_member();
#Modify discovery_member
$network->discovery_member("test.localdomain");
```

discover_now_status( )

Use this method to retrieve the current discovery status. This attribute is read-only.

'discover_now_status' will not be returned by default, use 'return_methods' to request it if needed.

Parameter

None.

Returns

The method returns the attribute value that can be 'COMPLETE', 'FAILED', 'NONE', 'PENDING' or 'RUNNING'.

Example

```
#Get discover_now_status
my $discover_now_status = $network->discover_now_status();
```

discovery_blackout_setting( )

Use this method to set or retrieve the discovery blackout setting.

Setting this method to a defined value implicitly sets the override_blackout_setting method to "true". If port_control_blackout_setting is undefined, setting the parameter to undefined causes the appliance to use the Grid default and automatically resets the override_blackout_setting attribute to "false".

Note that when discovery_blackout_setting is set to a defined value and override_blackout_setting is set to "false", the last operation takes precedence. Thus the sequence $network->discovery_blackout_setting($blackout); $network->override_blackout_setting("false"); will set override_blackout_setting to "false", and the sequence $network->override_blackout_setting("false"); $network->discovery_blackout_setting($blackout); will result in override_blackout_setting="true".

Parameter

Valid value is an Infoblox::Grid::Discovery::Properties::BlackoutSetting object.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get discovery_blackout_setting
my $discovery_blackout_setting = $network->discovery_blackout_setting();
#Modify discovery_blackout_setting
$network->discovery_blackout_setting($blackout);
```

discovery_engine_type( )

Use this method to retrieve the DHCP Network Container container discovery engine type. This attribute is read-only.

Parameter

None.

Returns

The method returns the attribute value that can be 'NETMRI', 'NETWORK_INSIGHT', 'VDISCOVERY', 'UNKNOWN' or 'NONE'.

Example

```
#Get discovery_engine_type
my $discovery_engine_type = $network_container->discovery_engine_type();
```

email_list( )

Use this method to set or retrieve the e-mail lists to which the appliance sends threshold e-mail messages.

If you enter e-mail addresses in the email_list, then the appliance sends DHCP threshold alerts to the e-mail addresses.

Parameter

The valid value is an array reference that contains string objects.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get email_list
my @email_list = $network_container->email_list();
#Modify email_list
$network_container->email_list( [ "admin\@test.com" ] );
```

enable_ddns( )

Use this method to set or retrieve the dynamic DNS updates flag of a DHCP Network Container object. The DHCP server can send DDNS updates to DNS servers in the same grid and to external DNS servers. This setting overrides the member-level settings.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

Specify "true" to set the dynamic DNS updates flag or "false" to deactivate/unset it. The default value for this field is undefined, which indicates that this attribute inherit the member-level setting.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
# Get enable_ddns
my $enable_ddns = $network_container->enable_ddns()
# Modify enable_ddns
$network_container->enable_ddns("true");
# Un-override enable_ddns
$network_container->enable_ddns(undef);
```

enable_dhcp_thresholds( )

Use this method to set or retrieve DHCP thresholds at the network container level. These thresholds are the high and low watermarks. If a DHCP Network Container usage goes above or below these percentage values, the Infoblox appliance generates a syslog message and sends an SNMP notification.

Enable this flag to set DHCP thresholds values.

Parameter

Specify "true" to set the enable_dhcp_thresholds flag or "false" to deactivate/unset it. The default value is undefined.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get enable_dhcp_thresholds
my $enable_dhcp_thresholds = $network_container->enable_dhcp_thresholds();
#Modify enable_dhcp_thresholds
$network_container->enable_dhcp_thresholds("false");
```

enable_email_warnings( )

Use this method to send DHCP threshold warnings through e-mail.

Enable this flag if you want the appliance to send DHCP threshold warnings to the "email_list" provided when DHCP threshold is enabled.

Parameter

Specify "true" to set the enable_email_warnings flag or "false" to deactivate/unset it. The default value is undefined.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get enable_email_warnings
my $enable_email_warnings = $network_container->enable_email_warnings();
#Modify enable_email_warnings
$network_container->enable_email_warnings("false");
```

enable_discovery( )

Use this method to set or retrieve the enable_discovery value of a DHCP Network Container object. This setting overrides the member-level settings.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Setting this method to a defined value implicitly sets the override_enable_discovery method to "true". Setting the parameter to undefined causes the appliance to use the default from the upper level and automatically resets the override_enable_discovery attribute to "false".

Enable this flag if you want to set discovery for this object.

Parameter

Specify "true" to set the enable_discovery flag or "false" to deactivate/unset it. The default value is undefined.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get enable_discovery
my $enable_discovery = $network->enable_discovery();
#Modify enable_discovery
$network->enable_discovery("false");
```

enable_immediate_discovery( )

Use this method to set or retrieve enable_immediate_discovery value of a DHCP Network Container object.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

Specify "true" to set the enable_immediate_discovery flag or "false" to deactivate/unset it. The default value is undefined.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get enable_immediate_discovery
my $enable_immediate_discovery = $network->enable_immediate_discovery();
#Modify enable_immediate_discovery
$network->enable_immediate_discovery("false");
```

enable_snmp_warnings( )

Use this method to send DHCP threshold warnings through SNMP.

If the flag is enabled, then the appliance sends DHCP threshold warnings through SNMP messages when DHCP threshold is enabled.

Parameter

Specify "true" to set the enable_snmp_warnings flag or "false" to deactivate/unset it. The default value is undefined.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get enable_snmp_warnings
my $enable_snmp_warnings = $network_container->enable_snmp_warnings();
```

```
#Modify enable_snmp_warnings
$network_container->enable_snmp_warnings("false");
```

endpoint_sources( )

Use this method to retrieve the list of endpoints that provide object data. This is a read-only attribute.

Omit the parameter to retrieve the attribute value.

Parameter

None

Returns

The valid return value is an array of Infoblox::CiscoISE::Endpoint objects.

Example

```
#Get endpoint_sources value
my $endpoint_sources = $object->endpoint_sources();
```

extattrs( )

Use this method to set or retrieve the extensible attributes associated with a DHCP Network Container object.

Parameter

Valid value is a hash reference containing the names of extensible attributes and their associated values (Infoblox::Grid::Extattr objects).

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get extattrs
my $ref_extattrs = $network_container->extattrs();
#Modify extattrs
$network_container->extattrs({ 'Site' => $extattr1, 'Administrator' => $extattr2 });
```

extensible_attributes( )

Use this method to set or retrieve the extensible attributes associated with a DHCP Network Container object

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

For valid values for extensible attributes, see Infoblox::Grid::ExtensibleAttributeDef/Extensible Attribute Values.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get extensible attributes
my $ref_extensible_attributes = $network_container->extensible_attributes();
#Modify extensible attributes
$network_container->extensible_attributes({ 'Site' => 'Santa Clara', 'Administrator' => [ 'Peter',
'Tom' ] });
```

high_water_mark( )

Use this method to set the percentage value for DHCP Network Container usage above which the Infoblox appliance generates a syslog message and sends a warning.

When the percentage of allocated addresses in a DHCP Network Container exceeds the high watermark, the appliance generates a syslog message and sends a warning. Default value for this option is 95.

Parameter

The valid value is in sting format.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get high_water_mark
my $high_water_mark = $network_container->high_water_mark();
#Modify high_water_mark
$network_container->high_water_mark("80");
```

high_water_mark_reset( )

Use this method to set or retrieve the high_water_mark_reset value. If the percentage of allocated addresses drops below this value, a corresponding SNMP trap is reset.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

A number that specifies the percentage of allocated addresses. The range is from 1 to 100. The high watermark reset value must be lower than the high watermark value. The default value is 95.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get high_water_mark_reset
my $high_water_mark_reset = $network_container->high_water_mark_reset();
#Modify high_water_mark_reset
$network_container->high_water_mark_reset(70);
```

ignore_id( )

Use this method to set or retrieve the ignore_id type of a DHCP Network Container object. Indicates whether the appliance will ignore DHCP client IDs or MAC addresses. This setting overrides the upper-level settings.

Parameter

Valid values are 'NONE', 'CLIENT', or 'MACADDR'. The default is 'NONE'.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
# Get attribute value
$value = $network_container->ignore_id();
# Modify attribute value
$network_container->ignore_id('CLIENT');
#Un-override ignore_id
$network_container->ignore_id(undef);
```

ignore_mac_addresses( )

Use this method to set or retrieve the ignore_mac_addresses list of a DHCP Network Container object. A list of MAC addresses the appliance will ignore. This setting overrides the upper-level settings.

If you enter MAC addresses in the ignore_mac_addresses, then the appliance will ignore them. If you enter empty array, then the appliance will ignore any MAC address.

Parameter

The valid value is an array reference that contains string objects.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
# Get attribute value
$value = $network_container->ignore_mac_addresses();
# Modify attribute value
$network_container->ignore_mac_addresses(["11::11::11::11::11::11"]);
```

ignore_dhcp_option_list_request( )

Use this method to set or retrieve the ignore_dhcp_option_list_request flag of a DHCP Network Container object. If this flag is set to false, the appliance returns all the defined DHCP options to the client. This setting overrides the upper-level settings.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

Specify "true" to set the ignore_dhcp_option_list_request flag or "false" to deactivate/unset it. The default value is undefined, which indicates that this attribute inherits the upper-level setting.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
# Get ignore_dhcp_option_list_request
my $ignore_dhcp_option_list_request = $network_container->ignore_dhcp_option_list_request();
# Modify ignore_dhcp_option_list_request
$network_container->ignore_dhcp_option_list_request("true");
#Do not override ignore_dhcp_option_list_request
$network_container->ignore_dhcp_option_list_request(undef);
```

ipam_email_addresses( )

Use this method to set or retrieve a list of IPAM Utilization email addresses.

Setting this method to a defined value implicitly sets the override_ipam_email_addresses method to "true". Setting the parameter to undefined causes the appliance to use the default from the upper level and automatically resets the override_ipam_email_addresses attribute to "false".

Note that when ipam_email_addresses is set to a defined value and override_ipam_email_addresses is set to "false", the last operation takes precedence. Thus the sequence $object->ipam_email_addresses(['admin@infoblox.com']); $object->override_ipam_email_addresses("false"); will set override_ipam_email_addresses to "false", and the sequence $object->override_ipam_email_addresses("false"); $object->ipam_email_addresses(['admin@infoblox.com']); will result in override_ipam_email_addresses="true".

Parameter

Valid value is an array reference that contains email addresses in string format.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get ipam_email_addresses
my $ipam_email_addresses = $network_container->ipam_email_addresses();
#Modify ipam_email_addresses
$network_container->ipam_email_addresses($ipam_email_addresses);
```

ipam_trap_settings( )

Use this method to set or retrieve IPAM Utilization SNMP Trap settings.

Setting this method to a defined value implicitly sets the override_ipam_trap_settings method to "true". Setting the parameter to undefined causes the appliance to use the default from the upper level and automatically resets the override_ipam_trap_settings attribute to "false".

Note that when ipam_trap_settings is set to a defined value and override_ipam_trap_settings is set to "false", the last operation takes precedence. Thus the sequence $object->ipam_trap_settings($ipam_trap); $object->override_ipam_trap_settings("false"); will set

override_ipam_trap_settings to "false", and the sequence $object->override_ipam_trap_settings("false"); $object->ipam_trap_settings($ipam_trap); will result in override_ipam_trap_settings="true".

Parameter

Valid value is an Infoblox::Grid::SNMP::IPAMTrap object.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get ipam_trap_settings
my $ipam_trap_settings = $network_container->ipam_trap_settings();
#Modify ipam_trap_settings
$network_container->ipam_trap_settings($ipam_trap_settings);
```

ipam_threshold_settings( )

Use this method to set or retrieve IPAM Utilization SNMP Threshold settings.

Setting this method to a defined value implicitly sets the override_ipam_threshold_settings method to "true". Setting the parameter to undefined causes the appliance to use the default from the upper level and automatically resets the override_ipam_threshold_settings attribute to "false".

Note that when ipam_threshold_settings is set to a defined value and override_ipam_threshold_settings is set to "false", the last operation takes precedence. Thus the sequence $object->ipam_threshold_settings($ipam_threshold_settings); $object->override_ipam_trap_settings("false"); will set override_ipam_threshold_settings to "false", and the sequence $object->override_ipam_threshold_settings("false"); $object->ipam_threshold_settings($ipam_threshold_settings); will result in override_ipam_threshold_settings="true".

Parameter

Valid value is an Infoblox::Grid::SNMP::IPAMThreashold object.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get ipam_threshold_settings
my $ipam_threshold_settings = $network_container->ipam_threshold_settings();
#Modify ipam_threshold_settings
$network_container->ipam_threshold_settings($ipam_threshold_settings);
```

mgm_private( )

Use this method to set or retrieve the flag that controls whether the object should be synced to the MGM or not.

Setting this method to a defined value implicitly sets the override_blackout_setting method to "true". If discovery_blackout_setting is undefined, setting the parameter to undefined causes the appliance to use the grid default and automatically resets the override_blackout_setting attribute to "false".

Note that when mgm_private is set to a defined value and override_mgm_private is set to "false", the last operation takes precedence. Thus the sequence $network->mgm_private("true"); $network->override_mgm_private("false"); will set override_mgm_private to "false", and the sequence $network->override_mgm_private("false"); $network->mgm_private("true"); will result in override_mgm_private="true".

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

Specify 'true' to enable synchronization of the managed Grid with the MGM or 'false' to forbid it. The default value is 'false'.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Note that this member is loaded on on-demand basis, so to retrieve the value the object must have been retrieved from the session first.

Example

```
#Get mgm_private value
my $value = $object->mgm_private();

#Modify mgm_private value
$object->mgm_private('true');
```

mgm_private_overridable( )

Use this method to retrieve the flag that indicates whether the overriding of 'mgm_private' flag is allowed or not. This is a read-only attribute.

Parameter

None

Returns

The method returns the attribute value.

Note that this member is loaded on on-demand basis, so to retrieve the value the object must have been retrieved from the session first.

Example

```
#Get mgm_private_overridable value
my $value = $object->mgm_private_overridable();
```

ms_ad_user_data( )

Use this method to retrieve Microsoft Active Directory users related information. This is a read-only attribute.

Parameter

None

Returns

The valid return value is an Infoblox::Grid::MSServer::AdUser::Data object.

Example

```
#Get ms_ad_user_data
my $ms_ad_user_data = $network_container->ms_ad_user_data();
```

lease_scavenge_time( )

Use this method to set or retrieve the lease_scavenge_time value of a DHCP Network Container object.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Setting this method to a defined value implicitly sets the override_lease_scavenge_time method to "true". Setting the parameter to undefined causes the appliance to use the default from the upper level and automatically resets the override_lease_scavenge_time attribute to "false".

Note that when lease_scavenge_time is set to a defined value and override_lease_scavenge_time is set to "false", the last operation takes precedence. Thus the sequence $object->lease_scavenge_time(604800); $object->override_lease_scavenge_time("false"); will set override_lease_scavenge_time to "false", and the sequence $object->override_lease_scavenge_time("false"); $object->lease_scavenge_time(604800); will result in override_lease_scavenge_time="true".

Parameter

An integer that specifies the period of time, in seconds, that free and backup leases remain in the database before they are automatically deleted. To disable lease scavenging, set the parameter to -1. The minimum positive value must be greater than 86400 seconds (1 day). The default value of this parameter is -1, which means that lease scavenging is disabled.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get lease_scavenge_time
my $lease_scavenge = $network_container->lease_scavenge_time();
#Modify lease_scavenge_time
```

```
$network_container->lease_scavenge_time(30 * 24 * 60 * 60);
```

logic_filters( )

Use this method to set or retrieve all the logic (ordered) filters associated with the DHCP Network Container.

Filters are used to screen address assignments in a DHCP Network Container if the addresses are to be assigned to specific hosts.

Setting this method to a defined value implicitly sets the override_logic_filters method to "true". Setting the parameter to undefined causes the appliance to use the Grid default and automatically resets the override_logic_filters attribute to "false".

Note that when logic_filters is set to a defined value and override_dlogic_filters is set to "false", the last operation takes precedence. Thus the sequence $object->logic_filters($filters); $object->override_logic_filters("false"); will set override_logic_filters to "false", and the sequence $object->override_logic_filters("false"); $object->logic_filters($filters); will result in override_logic_filters="true".

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

The valid value is an array reference that contains either the Infoblox::DHCP::Filter::MAC, Infoblox::DHCP::Filter::NAC, Infoblox::DHCP::Filter::Option objects or the names of the corresponding filters. The filters in the list must satisfy the following conditions:

Filter name must be globally unique.
Filters without expressions must be at the end of the list.
The filters should not be disabled.

The default value of the parameter is undefined.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get logic_filters
my $ref_logic_filters = $network_container->logic_filters();
#Modify logic_filters
$network_container->logic_filters(["filter_1","filter_2",$exl1]); #$exl1 is an
Infoblox::DHCP::Filter::MAC, Infoblox::DHCP::Filter::NAC or Infoblox::DHCP::Filter::Option object.
```

low_water_mark( )

Use this method to set the percentage value for DHCP Network Container usage below which the Infoblox appliance generates a syslog message and sends a warning.

When the percentage of allocated addresses in a DHCP Network Container goes below the low watermark, the appliance generates a syslog message and sends a warning. Default value for this option is 0.

Parameter

The valid value is in string format.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get low_water_mark
my $low_water_mark = $network_container->low_water_mark();
#Modify low_water_mark
$network_container->low_water_mark("10");
```

low_water_mark_reset( )

Use this method to set or retrieve the low_water_mark_reset value. If the percentage of allocated addresses exceeds this value, a corresponding SNMP trap is reset.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

A number that specifies the percentage of allocated addresses. The range is from 1 to 100. The low watermark reset value must be

higher than the low watermark value. The default low watermark reset number is 0.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get low_water_mark_reset
my $low_water_mark_reset = $network_container->low_water_mark_reset();
#Modify low_water_mark_reset
$network_container->low_water_mark_reset(10);
```

network( )

Use this method to retrieve the network attribute of a DHCP Network Container object.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

Specify the address and netmask of the network in CIDR format (example: 10.0.0.0/8).

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get network
my $network = $network_container->network();
```

override_blackout_setting( )

The override_blackout_setting attribute controls whether the discovery_blackout_setting and port_control_blackout_setting values in the object are used, instead of the Grid default.

The override_blackout_setting attribute can be specified explicitly. It is also set implicitly when discovery_blackout_setting or port_control_blackout_setting is set to a defined value.

Parameter

Specify "true" to set the override_blackout_setting flag or "false" to deactivate/unset it. The default value is "false".

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
# Get attribute value
my $value = $network->override_blackout_setting();
# Modify attribute value
$network->override_blackout_setting('true');
```

nextserver( )

Use this method to set or retrieve the nextserver attribute of a DHCP Network Container object. You can specify the name and/or IP address of the next server that the host needs to boot. This setting overrides the upper-level settings. The default value is undefined, which indicates that this attribute inherits the upper-level setting.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

This option is overridden independently from Infoblox::DHCP::NetworkContainer->bootfile() and Infoblox::DHCP::NetworkContainer->bootserver().

Parameter

The next server IP address or name in FQDN (Fully Qualified Domain Name) format. The FQDN consists of the host name followed by the domain name (example: abc.com). A next server name can have a maximum of 256 bytes. The default value is undefined.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
# Get nextserver
my $nextserver = $network_container->nextserver();
# Modify nextserver
$network_container->nextserver("blue.domain.com");
#Un-override nextserver
$network_container->nextserver(undef);
```

options( )

Use this method to set or retrieve the options value of a member DHCP server. DHCP options provide network configuration settings and various services available on the network for DHCP.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

The valid value is an array reference that contains Infoblox::DHCP::Option objects.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get options
my $options = $network_container->options();
#Modify options
my $option1 = Infoblox::DHCP::Option->new(
    name => "lease-time",
    value => 50
);

$network_container->options([$option1]);
```

override_ddns_domainname( )

The override_ddns_domainname attribute controls whether the ddns_domainname value in the DHCP Network Container is used, instead of the Grid default.

The override_ddns_domainname attribute can be specified explicitly. It is also set implicitly when ddns_domainname is set to a defined value.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

Set the parameter to "true" to override the upper-level setting for ddns_domainname. Set the parameter to "false" to inherit the upper-level setting for ddns_domainname.

The default value of this parameter is "false".

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Getting override_ddns_domainname
my $override_ddns_domainname=$network_container->override_ddns_domainname( );
#Modifying override_ddns_domainname
$network_container->override_ddns_domainname("true");
```

override_ddns_ttl( )

The override_ddns_ttl attribute controls whether the ddns_ttl value in the DHCP Network Container is used, instead of the Grid default.

The override_ddns_ttl attribute can be specified explicitly. It is also set implicitly when ddns_ttl is set to a defined value.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

Set the parameter to "true" to override the upper-level setting for ddns_ttl. Set the parameter to "false" to inherit the upper-level setting for ddns_ttl.

The default value of this parameter is "false".

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Getting override_ddns_ttl
my $override_ddns_ttl=$network_container->override_ddns_ttl( );
#Modifying override_ddns_ttl
$network_container->override_ddns_ttl("true");
```

override_ignore_id( )

The override_ignore_id controls whether the ignore_id in the DHCP Network Container is used, instead of the upper-level default.

The override_ignore_id can be specified explicitly. It is also set implicitly when ignore_id is set to a defined value.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

Set the parameter to "true" to override the upper-level setting for ignore_id and ignore_mac_addresses. Set the parameter to "false" to inherit the upper-level setting for ignore_id and ignore_mac_addresses.

The default value of this parameter is "false".

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get attrubute value
my $value=$network_container->override_ignore_id();
#Modifying attribute value
$network_container->override_ignore_id('true');
```

override_ipam_email_addresses( )

The override_ipam_email_addresses controls whether the ipam_email_addresses in the DHCP network container is used, instead of the upper-level default.

The override_ipam_email_addresses can be specified explicitly. It is also set implicitly when ipam_email_addresses is set to a defined value.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

Set the parameter to "true" to override the upper-level setting for ipam_email_addresses. Set the parameter to "false" to inherit the upper-level setting for ipam_email_addresses.

The default value of this parameter is "false".

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get attrubute value
my $value=$network_container->override_ipam_email_addresses();
#Modifying attribute value
$network_container->override_ipam_email_addresses('true');
```

override_ipam_threshold_settings( )

The override_ipam_threshold_settings controls whether the ipam_threshold_settings in the DHCP network container is used, instead of the upper-level default.

The override_ipam_threshold_settings can be specified explicitly. It is also set implicitly when ipam_threshold_settings is set to a defined value.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

Set the parameter to "true" to override the upper-level setting for ipam_threshold_settings. Set the parameter to "false" to inherit the upper-level setting for ipam_threshold_settings.

The default value of this parameter is "false".

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get attrubute value
my $value=$network_container->override_ipam_threshold_settings();
#Modifying attribute value
$network_container->override_ipam_threshold_settings('true');
```

override_ipam_trap_settings( )

The override_ipam_trap_settings controls whether the ipam_trap_settings in the DHCP network container is used, instead of the upper-level default.

The override_ipam_trap_settings can be specified explicitly. It is also set implicitly when ipam_trap_settings is set to a defined value.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

Set the parameter to "true" to override the upper-level setting for ipam_trap_settings. Set the parameter to "false" to inherit the upper-level setting for ipam_trap_settings.

The default value of this parameter is "false".

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get attrubute value
my $value=$network_container->override_ipam_trap_settings();
#Modifying attribute value
$network_container->override_ipam_trap_settings('true');
```

override_logic_filters( )

The override_logic_filters attribute controls whether the logic_filters value of the DHCP Network Container is used, instead of the Grid default.

The override_logic_filters attribute can be specified explicitly. It is also set implicitly when logic_filters is set to a defined value.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

Set the parameter to "true" to override the Grid-level setting for logic_filters. Set the parameter to "false" to inherit the Grid-level setting for logic_filters.

The default value of this parameter is "false".

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Getting override_logic_filters
my $override_logic_filters=$network_container->override_logic_filters( );
#Modifying override_logic_filters
$network_container->override_logic_filters("true");
```

override_lease_scavenge_time( )

The override_lease_scavenge_time attribute controls whether the lease_scavenge_time value in the DHCP Network Container is used, instead of the upper-level default.

The override_lease_scavenge_time attribute can be specified explicitly. It is also set implicitly when lease_scavenge_time is set to a defined value.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

Set the parameter to "true" to override the upper-level setting for lease_scavenge_time. Set the parameter to "false" to inherit the setting for lease_scavenge_time from the upper level.

The default value of this parameter is "false".

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Getting override_lease_scavenge_time
my $override_lease_scavenge_time=$network_container->override_lease_scavenge_time( );
#Modifying override_lease_scavenge_time
$network_container->override_lease_scavenge_time("true");
```

override_mgm_private( )

The override_mgm_private attribute controls whether the mgm_private value in the object is used, instead of the parent-level default.

The override_mgm_private attribute can be specified explicitly. It is also set implicitly when mgm_private is set to a defined value.

Note that setting override_mgm_private to 'false' sets the original 'mgm_private' value to 'false'.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

Set the parameter to 'true' to override the parent-level setting for mgm_private. Set the parameter to 'false' to inherit the parent-level setting for mgm_private.

The default value of this parameter is 'false'.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Note that this member is loaded on on-demand basis, so to retrieve the value the object must have been retrieved from the session first.

Example

```
#Get override_mgm_private value
my $value = $object->override_mgm_private( );

#Modify override_mgm_private value
$object->override_mgm_private('true');
```

override_subscribe_settings( )

The override_subscribe_settings attribute controls whether the subscribe_settings method values of the member are used, instead of the Grid default.

The override_subscribe_settings attribute can be specified explicitly. It is also set implicitly when subscribe_settings is set to a defined value.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

Set the parameter to "true" to override the Grid-level setting for subscribe_settings. Set the parameter to "false" to inherit the Grid-level setting for subscribe_settings.

The default value of this parameter is "false".

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get override_subscribe_settings value
my $override_subscribe_settings = $object->override_subscribe_settings( );
#Modify override_subscribe_settings value
$object->override_subscribe_settings("true");
```

override_update_dns_on_lease_renewal( )

The override_update_dns_on_lease_renewal attribute controls whether the update_dns_on_lease_renewal value in the DHCP Network is used, instead of the Grid default.

The override_update_dns_on_lease_renewal attribute can be specified explicitly. It is also set implicitly when update_dns_on_lease_renewal is set to a defined value.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

Set the parameter to "true" to override the upper-level setting for update_dns_on_lease_renewal. Set the parameter to "false" to inherit the upper-level setting for update_dns_on_lease_renewal.

The default value of this parameter is "false".

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Getting override_update_dns_on_lease_renewal
my $override_update_dns_on_lease_renewal=$network->override_update_dns_on_lease_renewal( );
#Modifying override_update_dns_on_lease_renewal
$network->override_update_dns_on_lease_renewal("true");
```

port_control_blackout_setting

Use this method to set or retrieve the port control blackout setting.

Setting this method to a defined value implicitly sets the override_blackout_setting method to "true". If discovery_blackout_setting is undefined, setting the parameter to undefined causes the appliance to use the Grid default and automatically resets the override_blackout_setting attribute to "false".

Note that when port_control_blackout_setting is set to a defined value and override_blackout_setting is set to "false", the last operation takes precedence. Thus the sequence $network->port_control_blackout_setting($blackout); $network->override_blackout_setting("false"); will set override_blackout_setting to "false", and the sequence $network->override_blackout_setting("false"); $network->port_control_blackout_setting($blackout); will result in override_blackout_setting="true".

Parameter

Valid value is an Infoblox::Grid::Discovery::Properties::BlackoutSetting object.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get port_control_blackout_setting
my $port_control_blackout_setting = $network->port_control_blackout_setting();
#Modify port_control_blackout_setting
$network->port_control_blackout_setting($blackout);
```

pxe_lease_time( )

Use this method to set or retrieve the PXE lease time value of a DHCP Network Container object. Some hosts use PXE (Preboot Execution Environment) to boot remotely from a server. To better manage your IP resources, set a different lease time for PXE boot requests. You can configure the DHCP server to allocate an IP address with a shorter lease time to hosts that send PXE boot requests, so IP addresses are not leased longer than necessary. This setting overrides the upper-level settings. The default value is undefined, which indicates that this attribute inherits the upper-level setting.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

A 32-bit integer (range from 0 to 4294967295) that represents the duration, in seconds, that the update is cached. Zero indicates that the update is not cached. The default value is undefined.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get PXE Lease Time
my $pxe_lease_time = $network_container->pxe_lease_time();
#Modify PXE Lease Time
$network_container->pxe_lease_time(1100);
#Un-override PXE Lease Time
$network_container->pxe_lease_time(undef);
```

recycle_leases( )

Use this method to set or retrieve the recycle_leases flag at the network level.

If the flag is enabled, the leases are kept in the recycle bin until one week after expiration. When the flag is disabled, the leases are irrecoverably deleted.

Parameter

Specify "true" to enable the recycle_leases flag or "false" to disable it. The default value is undefined.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get recycle_leases
my $recycle_leases = $network->recycle_leases();
#Modify recycle_leases
$network->recycle_leases("false");
```

remove_subnets( )

Use this method to set or retrieve the remove_subnets flag of a DHCP Network Container object.

You can remove the network, but keep all subnets under it if this flag is set to false. This flag is only intended as an option when removing DHCP Network Container objects. It is not retrieved or used for any other operation.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

Specify "false" to keep the subnets or "true" to remove them. The default value is undefined, which is to remove all subnets.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Set remove_subnets before submitting for removal
$network_container->remove_subnets("false");
#Remove the container keeping the subnets
$session->remove($network_container);
```

network_container( )

Use this method to set or retrieve the network container of a DHCP Network Container object.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

Specify the IP address and netmask of a network in CIDR format.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get network container
my $network_container = $network->network_container();
#Modify network container
$network->network_container("50.0.0.0/8");
```

network_view( )

Use this method to set or retrieve the network view of the DHCP network container.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

The default value is the "default" network view, which means the network is in the default network view.

Parameter

Valid value is a defined Infoblox::DHCP::View object. The default network view value is the system-defined default network view object.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get network view
my $nview = $network_container->network_view();
#Modify network view, with an Infoblox::DHCP::View object
$network_container->network_view($nview);
```

next_available_network( )

Use this method to retrieve the next available `network(s)` in the network container. Note that to use this method, the object MUST be retrieved through a get or search call, or created with an add call.

Parameter

The method accepts the following parameters:

```
  cidr - (required) The CIDR of the requested networks. It must be a value between 1 and 32,
inclusive.
  requested_num - (optional) The number of networks requested. It must be a value between 1 and 20,
inclusive.
  excluded - (optional) aA reference to an array of networks to be excluded from the results.
```

Returns

This method returns the next available `network(s)` in the network container. If requested_num is not specified, a string with the next available network will be returned; otherwise, the method will return a reference to an array of networks.

A network cannot be returned as the next available network if it cannot be inserted; such as when any of the following conditions exist:

- It's parent network has a DHCP configuration assigned and therefore cannot be converted to a network container.

- It overlaps with an existing network or network container, regardless of whether the existing network has a greater or smaller CIDR than the requested CIDR.

- Its network address or broadcast address is used by a fixed address or a host configured for DHCP.

Example

```
# Retrieve a particular network
my @retrieved_objs = $session->get(
                          object   => "Infoblox::DHCP::NetworkContainer",
                          network  => "10.0.0.0/8"
);

$network = $retrieved_objs[0];

# Print the next available IPv6 network in the network container
print $network->next_available_network(cidr => 12) . "\n";

# This will retrieve the next 10 available networks and print the first one.
my $addressesref = $network->next_available_network(requested_num => 10, cidr => 12);
print @$addressesref[0] . "\n";

# This will retrieve the next 10 available networks and exclude two.
my $addressesref = $network->next_available_network(cidr => 12, requested_num => 10, excluded =>
['10.0.0.0/12']);
```

resize( )

Use this method to resize a network container. Note that to use this method, the object MUST be retrieved through a get or search call, or it must have been created with an add call.

Parameter

prefix - Required. The bitmask prefix length after resizing.

auto_create_reversezone - Optional. Boolean flag indicating whether or not to automatically create the reverse zone.

reason - Optional. The reason for resizing the network container in string format with a maximum of 256 bytes. Used when 'rir_organization' attribute has been defined.

send_rir_request - Optional. Boolean flag that indicates whether the appliance should send an API or email request for Regional Internet Registry (RIR). Used when 'rir_organization' attribute has been defined.

Returns

The method returns true when the operation succeeds, and returns false when the operation fails. When an operation fails, the error code and error message are reported in the Session object's `status_code()` and `status_detail()` attributes.

Example

```
#Get Network Container through the session
my @retrieved_objs = $session->get(
           object =>"Infoblox::DHCP::NetworkContainer" ,
           network => "20.0.0.0/8"
        );
my $network_container = $retrieved_objs[0];

# Expand a /24 network container to a /16 network container, inserting
# a corresponding reverese zone in the process.
my $response = $network_container->resize(
      prefix        => "16"
      auto_create_reversezone => "true"
) or die("Resize Network Container failed: ",
         $session->status_code() . ":" . $session->status_detail());
```

restart_if_needed( )

Use this method to set the flag that indicates whether DHCP service should be restarted on a member that is serving DHCP for the changed object referenced in a cloud API request.

Parameter

Specify true to restart the DHCP service.

Returns

None

Example

```
# Modify
$network->restart_if_needed('true');
```

rir( )

Use this method to retrieve the name of the Regional Internet Registry (RIR) that allocated this address space. This is a read-only attribute.

Parameter

None

Returns

The method returns the attribute value that can be 'RIPE' or 'NONE'.

Example

```
#Get rir
my $rir = $network_container->rir();
```

rir_organization( )

Use this method to set or retrieve the Regional Internet Registry (RIR) organization object.

Parameter

An Infoblox::Grid::RIR::Organization object.

You must enable support for RIR/SWIP at the Grid level before you can set attribute values. See Infoblox::Grid->enable_rir_swip() method.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get rir_organization
my $rir_organization = $network_container->rir_organization();
#Modify rir_organization
$network_container->rir_organization($rir_org);
```

rir_registration_action( )

Use this method to set the registration action of Regional Internet Registry (RIR). This is a write-only attribute.

Parameter

Valid values are 'NONE', 'CREATE', 'MODIFY' or 'DELETE'.

Returns

The method returns true when the modification succeeds, and returns false when the operation fails.

Example

```
#Set rir_registration_action
$network_container->rir_registration_action('CREATE');
```

rir_registration_status( )

Use this method to set or retrieve the registration status of Regional Internet Registry (RIR).

Parameter

The registration status of Regional Internet Registry (RIR), which can be 'REGISTERED' or 'NOT_REGISTERED'.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get rir_registration_status
my $rir_registration_status = $network_container->rir_registration_status();
#Modify rir_registration_status
$network_container->rir_registration_status('NOT_REGISTERED');
```

last_rir_registration_update_sent( )

Use this method to retrieve the last registration update timestamp of Regional Internet Registry (RIR). This is a read-only attribute.

Parameter

None

Returns

The method returns the attribute value.

Example

```
#Get last_rir_registration_update_sent
my $last_rir_registration_update_sent = $network_container->last_rir_registration_update_sent();
```

last_rir_registration_update_status( )

Use this method to retrieve the last registration update status of Regional Internet Registry (RIR). This is a read-only attribute.

Parameter

None

Returns

The method returns the attribute value.

Example

```
#Get last_rir_registration_update_status
my $last_rir_registration_update_status = $network_container->last_rir_registration_update_status();
```

same_port_control_discovery_blackout( )

Use this method to set or retrieve the flag that enables using the same discovery blackout for port control blackout.

Parameter

Specify 'true' to use same discovery blackout for port control blackout or 'false' to disable it. The default value is 'false'.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
# Get attribute value
my $value = $network_container->same_port_control_discovery_blackout();
# Modify attribute value
$network_container->same_port_control_discovery_blackout('true');
```

send_rir_request( )

Use this method to set the flag that indicates whether the appliance should send an API or email request for Regional Internet Registry (RIR). This is a write-only attribute.

Parameter

Specify "true" to set the send_rir_request flag or "false" to unset it.

Returns

The method returns true when the modification succeeds, and returns false when the operation fails.

Example

```
#Set send_rir_request
$network_container->send_rir_request('true');
```

subscribe_settings( )

Use this method to set or retrieve the Cisco ISE subscribe settings.

Setting this method to a defined value implicitly sets the override_subscribe_settings method to "true". Setting the parameter to undefined causes the appliance to use the Cisco ISE endpoint default and automatically resets the override_subscribe_settings method to "false".

Note that when subscribe_settings is set to a defined value and override_subscribe_settings is set to "false", the last operation takes precedence. Thus the sequence $object->subscribe_settings($value); $object->override_subscribe_settings("false"); will set override_subscribe_settings to "false", and the sequence $object->override_subscribe_settings("false"); $object->subscribe_settings($value); will result in override_subscribe_settings="true".

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

The valid value is an Infoblox::CiscoISE::SubscribeSetting object.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get subscribe_settings value
my $subscribe_settings = $object->subscribe_settings();
#Modify subscribe_settings value
$object->subscribe_settings($settings);
```

delete_reason( )

Use this method to specify the reason for deleting the Regional Internet Registry (RIR) registration request. This is a write-only attribute. This field is required only if 'rir_registration_action' is specified as 'DELETE'.

Parameter

The reason for deleting the RIR registration request in string format with a maximum of 256 bytes.

Returns

The method returns true when the modification succeeds, and returns false when the operation fails.

Example

```
#Set delete_reason
$network_container->delete_reason('invalid');
```

unmanaged( )

Use this method to set or retrieve the network status, which indicates whether the network is unmanaged or not.

Parameter

Specify true to mark the network as unmanaged or false as managed.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
# Get network status
$unmanaged = $network_container->unmanaged();
# Modify network status
$network_container->unmanaged('true');
```

update_dns_on_lease_renewal( )

The update_dns_on_lease_renewal attribute controls whether the DHCP server updates DNS when a DHCP lease is renewed.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Setting this method to a defined value implicitly sets the override_update_dns_on_lease_renewal method to "true". Setting the parameter to undefined causes the appliance to use the Grid default and automatically resets the override_update_dns_on_lease_renewal attribute to "false".

Note that when update_dns_on_lease_renewal is set to a defined value and override_update_dns_on_lease_renewal is set to "false", the last operation takes precedence. Thus the sequence $object->update_dns_on_lease_renewal("true"); $object->override_update_dns_on_lease_renewal("false"); will set override_update_dns_on_lease_renewal to "false", and the sequence $object->override_update_dns_on_lease_renewal("false"); $object->update_dns_on_lease_renewal("true"); will result in override_update_dns_on_lease_renewal="true".

Parameter

Specify "true" to set the update_dns_on_lease_renewal flag or "false" to deactivate/unset it.

The default value of this parameter is "false".

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get update_dns_on_lease_renewal
my $update_dns_on_lease_renewal=$network_container->update_dns_on_lease_renewal( );
#Modify update_dns_on_lease_renewal
$network_container->update_dns_on_lease_renewal("true");
```

zone_associations( )

Use this method to set or retrieve the associated zone information.

Parameter

The valid value is an array reference that contains Infoblox::DNS::Zone object(s). The default value is an empty list. If the network container has a parent, it is possible to use the zone associations of the parent by setting the parameter to undef.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails. If you did not specify a parameter, the method returns the attribute value.

Examples

```
#Getting zone associations
my $zone_associations = $network_container->zone_associations();
my $zone1 = Infoblox::DNS::Zone->new(
      name     => "test1.com",
);
my $zone2 = Infoblox::DNS::Zone->new(
      name     => "test2.com",
   );
#Modifying zone_associations
$network_container->zone_associations([$zone1, $zone2]);
#Clear zone_assocations
$network_container->zone_associations([]);
##Remove the override for zone_associations
$network_container->zone_associations(undef)
```

SAMPLE CODE

The following sample code demonstrates the different functions that can be applied to an object such as add, search, modify, and remove. This sample also includes error handling for the operations.

#Preparation prior to a DHCP Network Container object insertion

```
#PROGRAM STARTS: Include all the modules that will be used
use strict;
use Infoblox;

#Create a session to the Infoblox appliance

my $session = Infoblox::Session->new(
    master   => "192.168.1.2",
    username => "admin",
```

```
    password => "infoblox"
);
unless ($session) {
   die("Construct session failed: ",
       Infoblox::status_code() . ":" . Infoblox::status_detail());
}
print "Session created successfully\n";
```

#Create a DHCP Network Container object

```
my $network_container = Infoblox::DHCP::NetworkContainer->new(
            network => "10.0.0.0/8",
            comment => "add network container",
             );

my $response = $session->add($network_container)
       or die("Add Network Container failed: ",
             $session->status_code() . ":" . $session->status_detail());

print "Network Container added successfully\n";

#Search for a specific Network Container

#Search the network container
my @retrieved_objs = $session->search(
    object  => "Infoblox::DHCP::NetworkContainer",
    network => '10\.0\.0\..*'
);
my $object = $retrieved_objs[0];
unless ($object) {
    die("Search Network Container failed: ",
        $session->status_code() . ":" . $session->status_detail());
}
print "Search Network Container found at least 1 matching entry\n";
```

#Get and modify a Network Container attribute

```
#Get Network Container through the session
my @retrieved_objs = $session->get(
           object =>"Infoblox::DHCP::NetworkContainer" ,
           network => "10.0.0.0/8"
        );
```

```
my $object = $retrieved_objs[0];
```

```
unless ($object) {
    die("Get Network Container object failed: ",
        $session->status_code() . ":" . $session->status_detail());
}
print "Get Network Container found at least 1 matching entry\n";
```

#Modify one of the attributes of the specified Network Container object

```
$object->comment("modified comment"); #Apply the changes

$session->modify($object)
    or die("Modify Network Container attribute failed: ",
           $session->status_code() . ":" . $session->status_detail());

print "Network Container object modified successfully \n";
```

#Resize a network container

```
#Get Network through the session
my @retrieved_objs = $session->get(
    object =>"Infoblox::DHCP::NetworkContainer" ,
    network => "10.0.0.0/8"
);
my $object = $retrieved_objs[0];

#Resize
my $response = $object->resize(
    prefix => "16"
) or die("Resize Network Container failed: ",
    $session->status_code() . ":" . $session->status_detail());
```

#Remove a Network Container Object

```
 #Get Network Container object through the session
 my @retrieved_objs = $session->get(
           object  => "Infoblox::DHCP::NetworkContainer" ,
           network => "10.0.0.0"
       );

   my $object = $retrieved_objs[0];

   unless ($object) {
    die("Get Network Container object failed: ",
        $session->status_code() . ":" . $session->status_detail());
    }

print "Get Network Container object found at least 1 matching entry\n";
```

#Submit the object for removal

```
$session->remove($object)
    or die("Remove Network Container failed: ",
        $session->status_code() . ":" . $session->status_detail());

print "Network Container removed successfully \n";

####PROGRAM ENDS####
```

AUTHOR

Infoblox Inc. http://www.infoblox.com/

SEE ALSO

Infoblox::Session, Infoblox::DHCP::View, Infoblox::DHCP::Network, Infoblox::Session->get(), Infoblox::Session->search(), Infoblox::Session->add(), Infoblox::Session->remove(), Infoblox::Session->modify(), Infoblox::Grid::MSServer::AdUser::Data, Infoblox::DHCP::Filter::MAC, Infoblox::DHCP::Filter::NAC, Infoblox::DHCP::Filter::Option.

COPYRIGHT

Copyright (c) 2017 Infoblox Inc.

Infoblox::DHCP::IPv6OptionSpace - DHCP IPv6 OptionSpace object.

- NAME
- DESCRIPTION
- CONSTRUCTOR
- SESSION METHODS
  - Infoblox::Session->add( )
  - Infoblox::Session->get( )
  - Infoblox::Session->modify( )
  - Infoblox::Session->remove( )
  - Infoblox::Session->search( )
- METHODS
  - name( )
  - comment( )
  - enterprise_number( )
- SAMPLE CODE
- AUTHOR
- SEE ALSO
- COPYRIGHT

NAME

Infoblox::DHCP::IPv6OptionSpace - DHCP IPv6 OptionSpace object.

DESCRIPTION

An Option Space defines a namespace in which vendor options can be defined. This object is for an IPv6 option space. To define a specific vendor option space, add an option space to DHCP.

CONSTRUCTOR

```
my $ipv6os = Infoblox::DHCP::IPv6OptionSpace->new(
      name              => $string, #Required
      enterprise_number  => $num,    #Required, if this is a vendor option space.  It is optional
otherwise.
      comment           => $string, #Optional, default is undefined
);
```

SESSION METHODS

This section describes all the methods in Infoblox::Session module that can be applied to a DHCP IPv6 OptionSpace object.

Infoblox::Session->add( )

Use this method to add a DHCP IPv6 OptionSpace object to the Infoblox appliance. See Infoblox::Session->add() for parameters and return values.

Example

```
#Construct a DHCP IPV6 OptionSpace object
my $ipv6os = Infoblox::DHCP::IPv6OptionSpace->new(
      name => "infospace",
      comment => "some comment",
      enterprise_number => 123,
);
#Submit for addition
my $response = $session->add($ipv6os)
```

Infoblox::Session->get( )

Use this method to retrieve all the matching objects from the Infoblox appliance. See Infoblox::Session->get() for parameters and return values.

Key References

```
Apply the following attributes to get a specific DHCP IPv6 OptionSpace object:

 name     - Required. Option space name in string format.
 comment  - Optional. Option space comment.
 enterprise_number - Optional. Enterprise number.
```

Example

```
#Get DHCP IPv6 OptionSpace object through the session
my @retrieved_objs = $session->get(
    object => "Infoblox::DHCP::IPv6OptionSpace",
    name   => "infospace",
    enterprise_number => 123,
);
```

Infoblox::Session->modify( )

Use this method to modify a DHCP IPv6 OptionSpace object in the Infoblox appliance. See Infoblox::Session->modify() for parameters and return values.

Example

```
#Use method to modify the name.
$ipv6os->name("info_space");
#Submit modification
my $response = $session->modify( $ipv6os );
```

Infoblox::Session->remove( )

Use this method to remove a DHCP IPv6 OptionSpace object from the Infoblox appliance. See Infoblox::Session->remove() for parameters and return values.

To remove a specific object, first use `get()` to retrieve the specific DHCP IPv6 OptionSpace object, and then submit this object for removal.

Example

```
#Get DHCP IPv6 OptionSpace object through the session
my @retrieved_objs = $session->get(
    object => "Infoblox::DHCP::IPv6OptionSpace",
    name   => "info_space",
    enterprise_number => 123,
);
#Find the desired object from the retrieved list.
my $desired_option_space = $retrieved_objs[0];
#Submit for removal
my $response = $session->remove( $desired_option_space );
```

Infoblox::Session->search( )

Use this method to retrieve all the matching objects from the Infoblox appliance. See Infoblox::Session->search() for parameters and return values.

Key References

```
Apply the following attributes to get a specific DHCP IPv6 OptionSpace object:

 name     - Optional. Option space name in string format.
 comment  - Optional. Option space comment (regular expression).
 enterprise_number - Optional. Enterprise number.
```

Example

```
#Get DHCP IPv6 OptionSpace object through the session
my @retrieved_objs = $session->search(
    object => "Infoblox::DHCP::IPv6OptionSpace",
    comment => ".*comm.*"
);
```

METHODS

This section describes all the methods that can used to set and retrieve the attribute values of a DHCP IPv6 OptionSpace object.

name( )

Use this method to set or retrieve name of IPv6 option space object.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

Desired name in string format with a maximum of 256 bytes.

Returns

If parameter is specified, the method returns true when the modification succeeds, and returns false when the operation fails.

If parameter is not specified, the method returns the attribute value.

Example

```
#Get name
my $name = $ipv6os->name();
#Modify name
$ipv6os->name("info_space");
```

comment( )

Use this method to set or retrieve a comment about the IPv6 option space object.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

Desired comment in string format with a maximum of 256 bytes.

Returns

If the parameter is specified, the method returns true when the modification succeeds, and returns false when the operation fails.

If the parameter is not specified, the method returns the attribute value.

Example

```
#Get comment
my $comment = $ipv6os->comment();
#Modify comment
$ipv6os->comment("my comment");
```

enterprise_number( )

Use this method to set or retrieve the enterprise number for the IPv6 option space object.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

An unsigned integer that represents the enterprise number.

Returns

If the parameter is specified, the method returns true when the modification succeeds, and returns false when the operation fails.

If the parameter is not specified, the method returns the attribute value.

Example

```
#Get enterprise_number
my $enterprise_number = $ipv6os->enterprise_number();
#Modify enterprise_number
$ipv6os->enterprise_number(345);
```

SAMPLE CODE

The following sample code demonstrates the different functions that can be applied to an IPv6OptionSpace object, such as get, add, modify and remove. Also sample includes error handling for the operations.

#Preparation prior to a DHCP IPv6OptionSpace object insertion

```
#PROGRAM STARTS: Include all the modules that will be used
use strict;
use Infoblox;

#Create a session to the Infoblox appliance

my $session = Infoblox::Session->new(
               master   => "192.168.1.2", #appliance host ip
               username => "admin",       #appliance user login
               password => "infoblox"     #appliance password
);
```

```
unless ($session) {
       die("Construct session failed: ",
               Infoblox::status_code() . ":" . Infoblox::status_detail());
}
print "Session created successfully\n";
```

#Create an IPv6OptionSpace object

```
my $ipv6os = Infoblox::DHCP::IPv6OptionSpace->new (
       "name" => "infospace"
);

unless($ipv6os) {
     die("Construct IPv6 option_space filter failed: ",
          Infoblox::status_code() . ":" . Infoblox::status_detail());
}
print "IPv6OptionSpace object created successfully\n";

#Verify if the DHCP IPv6OptionSpace exists in the Infoblox appliance
my $object = $session->get(
       object => "Infoblox::DHCP::IPv6OptionSpace",
       name   => "infospace"
       enterprise_number => 123,
);
unless ($object) {
   print "IPv6 option space does not exist on server, safe to add the option_space\n";
   $session->add($ipv6os)
      or die("Add IPv6 option space failed: ",
             $session->status_code() . ":" . $session->status_detail());
}
print "DHCP IPv6 OptionSpace added successfully\n";
```

#Get and Modify an IPv6OptionSpace object

```
#Get IPv6OptionSpace object through the session
my @retrieved_objs = $session->get(
    object => "Infoblox::DHCP::IPv6OptionSpace",
    name   => "infospace"
);
my $object = $retrieved_objs[0];
unless ($object) {
    die("Get IPv6OptionSpace object failed: ",
        $session->status_code() . ":" . $session->status_detail());
}
print "Get IPv6OptionSpace object found at least 1 matching entry\n";

#Modify name of the obtained IPv6OptionSpace object
$object->name("info_space");

#Apply the changes
$session->modify($object)
    or die("Modify IPv6OptionSpace object failed: ",
           $session->status_code() . ":" . $session->status_detail());
print "IPv6OptionSpace object modified successfully \n";
```

#Remove an IPv6OptionSpace object

```
#Get IPv6OptionSpace object through the session
my @retrieved_objs = $session->get(
    object => "Infoblox::DHCP::IPv6OptionSpace",
    name   => "info_space"
);
my $object = $retrieved_objs[0];
unless ($object) {
    die("Get IPv6OptionSpace object failed: ",
        $session->status_code() . ":" . $session->status_detail());
}
print "Get IPv6OptionSpace object found at least 1 matching entry\n";

#Submit the object for removal
$session->remove($object)
    or die("Remove IPv6OptionSpace object failed: ",
        $session->status_code() . ":" . $session->status_detail());
print "IPv6OptionSpace object removed successfully \n";

####PROGRAM ENDS####
```

AUTHOR

Infoblox Inc. http://www.infoblox.com/

SEE ALSO

Infoblox::DHCP::IPv6OptionDefinition,Infoblox::Session->get(), Infoblox::Session->modify(), Infoblox::Session->remove(), Infoblox::Session

---

COPYRIGHT

Copyright (c) 2017 Infoblox Inc.

Infoblox::DHCP::IPv6OptionDefinition - DHCP IPv6 option definition object

- NAME
- DESCRIPTION
- CONSTRUCTOR
- SESSION METHODS
  - Infoblox::Session->add( )
  - Infoblox::Session->get( )
  - Infoblox::Session->modify( )
  - Infoblox::Session->search( )
  - Infoblox::Session->remove( )
- METHODS
  - name( )
  - code( )
  - space( )
  - type( )
- SAMPLE CODE
- AUTHOR
- SEE ALSO
- COPYRIGHT

NAME

Infoblox::DHCP::IPv6OptionDefinition - DHCP IPv6 option definition object

---

DESCRIPTION

An IPv6OptionDefinition defines an IPv6 DHCP option within a specific IPv6 option space. A custom option can be defined in the predefined DHCP option space or in the user-defined vendor option space. To define an option, add the option definition to the required option space.

CONSTRUCTOR

```
my $ipv6od = Infoblox::DHCP::IPv6OptionDefinition->new(
                name  => $string, #Required
                code  => $num,    #Required
                space => $string, #Optional / Default is DHCP
                type  => $string, #Optional / Default is string
             );
```

SESSION METHODS

This section describes all the methods in an Infoblox::Session module that can be applied to a DHCP IPv6 option definition object.

Infoblox::Session->add( )

Use this method to add an IPv6 option definition object to the Infoblox appliance. See Infoblox::Session->add() for parameters and return values.

Example

```
#Create a DHCP IPv6OptionDefinition object.
my $ipv6od = Infoblox::DHCP::IPv6OptionDefinition->new(
                space => "SUNW",
                name  => "JumpStart-server",
                code  => "6",
                type  => "text"
             );
# Submit for adding
my $response = $session->add( $ipv6od );
```

Infoblox::Session->get( )

Use this method to retrieve all the matching DHCP IPv6 option definition objects from the Infoblox appliance. See Infoblox::Session->get() for parameters and return values.

Key References

```
Apply the following attributes to get a specific DHCP IPv6 option definition object:

 name  - A name for the option definition in string format (regular expression).
```

```
  code  - Option number. Either a name or code is required.
  space - Optional. The option space in which the option is defined. The default is the "DHCPv6"
option space.
```

Example

```
#Getting option by name
my  @result_array = $session->get(
                        object   => "Infoblox::DHCP::IPv6OptionDefinition",
                        space    => "SUNW",
                        name     => "JumpStart-server",
                        );

#Getting option by code
my @result_array = $session->get(
                        object   => "Infoblox::DHCP::IPv6OptionDefinition",
                        space    => "DHCPv6",
                        code     => 176,
                        );
```

Infoblox::Session->modify( )

Use this method to modify a DHCP IPv6 option definition object in the Infoblox appliance. See Infoblox::Session->modify() for parameters and return values.

Example

```
#Use method to modify code
$object->code(20);
# Submit modification
my $response = $session->modify($object);
```

Infoblox::Session->search( )

Use this method to retrieve all the matching DHCP IPv6 option definition objects from the Infoblox appliance. See Infoblox::Session->search() for parameters and return values.

Key References

```
Apply the following attributes to get a specific DHCP IPv6 option definition object:

  name  - A name for the option definition in string format (regular expression).
  code  - Option number. Either a name or code is required.
  space - Optional. The option space in which the option is defined. The default is the "DHCPv6"
option space.
```

Example

```
#Getting option by name
my  @result_array = $session->search(
                        object   => "Infoblox::DHCP::IPv6OptionDefinition",
                        space    => "SUNW",
                        name     => "JumpStart-server",
                        );

#Getting option by code
my @result_array = $session->search(
                        object   => "Infoblox::DHCP::IPv6OptionDefinition",
                        space    => "DHCPv6",
                        code     => 176,
                        );
```

Infoblox::Session->remove( )

Use this method to remove a DHCP IPv6 option definition object from the Infoblox appliance. See Infoblox::Session->remove() for parameters and return values.

To remove a specifc object, first use `get()` to retrieve the specific object, and then submit this object for removal.

Example

```
my  @result_array = $session->get(
                        object   => "Infoblox::DHCP::IPv6OptionDefinition",
                        space    => "SUNW",
                        name     => "JumpStart-server",
                        );

# find the desired object from retrieved list.
```

```
my $desired_obj = $result_array[0];

# Submit for removal
my $response = $session->remove($desired_obj);
```

METHODS

This section describes all the methods that can be used to configure and retrieve the attribute values of a DHCP IPv6 option definition object.

name( )

Use this method to set or retrieve the name of a DHCP IPv6 option definition object.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

Desired name in string format with a maximum of 256 bytes.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
# Get name
my $od_name = $ipv6od->name();
# Modify name
$ipv6od->name("Start-server");
```

code( )

Use this method to set or retrieve the code of a DHCP IPv6 option definition object. An option code number is used to identify the DHCP option.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

Number for the option code. Value must be between 1 and 254. Values 2, 7, 12, 14, 21-14, 27-34, and 39 are predefined. These option codes have corresponding predefined Types. Therefore, you cannot change the predefined Types.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
# Get code
my $od_code = $ipv6od->code();
# Modify code
$ipv6od->code(14);
```

space( )

Use this method to set or retrieve the space of a DHCP IPv6 option definition object.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

Specify the space name in string format.

Returns

If you did not specify a parameter, the method returns the attribute value.

Example

```
# Get space
my $od_space = $ipv6od->space();
# Modify space
$ipv6od->space("SUNW");
```

type( )

Use this method to set or retrieve the data type of the grid DHCP IPv6 option.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

The accepted types are the following:

```
  "boolean"                 - For "true" or "false" boolean values.

  "ip-address"              - For a single IPv6 address. An IPv6 address is a 128-bit number.
                              It consist of eight groups of hexadecimal digits separated
                              by colon (example: "2001:1dfc:2dfb:0:c1:0:a0:a2"). A group of all zeros
                              can be replaced by a double colon (e.g. 2001:1dfc:2dfb::1 is equivalent
                              to 2001:1dfc:2dfb:0:0:0:0:1).

  "array of ip-address"     - For a list of IPv6 addresses, each separated by a comma.
                              (example: "2002:d0:d2::1, 2002:d0:d2::3, 2001::3")

  "string"                  - For value in ASCII text string or
                              a list of hexadecimal characters (0-9, a-f, or A-F) without spaces and
separated by colons.

  "text"                    - For value in ASCII text string.

  "domain-name"             - For domain name in ASCII text format

  "domain-list"             - For a list of domain names, each separated by a comma.

  "8-bit unsigned integer"  - For numeric value between 0 and 255.

  "16-bit unsigned integer" - For numeric value between 0 and 65535.

  "32-bit unsigned integer" - For numeric value between 0 and 4294967295.

  "8-bit signed integer"    - For numeric value between -128 and 127.

  "16-bit signed integer"   - For numeric value between -32768 and 32767.

  "32-bit signed integer"   - For numeric value between -2147483648 and 2147483647.

  "array of 8-bit unsigned integer"  - For a list of numeric values between 0 and 255, each separated
by a comma.

  "array of 16-bit unsigned integer" - For a list of numeric values between 0 and 65535, each
separated by a comma.

  "array of 32-bit unsigned integer" - For a list of numeric values between 0 and 4294967295, each
separated by a comma.

  "array of 8-bit integer"           - For a list of numeric values between -128 and 127, each
separated by a comma.

  "array of 16-bit integer"          - For a list of numeric values between -32768 and 32767, each
separated by a comma.

  "array of 32-bit integer"          - For a list of numeric values between -2147483648 and
2147483647, each separated by a comma.
```

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
# Get type
my $od_type = $ipv6od->type();
# Modify type
$ipv6od->type("string");
```

SAMPLE CODE

The following sample code demonstrates the different functions that can be applied to an object such as add, modify, and remove. Also, this sample includes error handling for the operations.

#Preparation prior to a DHCP Network insertion

```
#PROGRAM STARTS: Include all the modules that will be used
use strict;
```

```
use Infoblox;

#Create a session to the Infoblox appliance

my $session = Infoblox::Session->new(
    master   => "192.168.1.2",
    username => "admin",
    password => "infoblox"
);
unless ($session) {
   die("Construct session failed: ",
       Infoblox::status_code() . ":" . Infoblox::status_detail());
}
print "Session created successfully\n";

my $ipv6od = Infoblox::DHCP::IPv6OptionSpace->new("name" => "SUNW");
my $response = $session->add($ipv6od);
print "Add option space operation success \n" if ($response);
print "Add option space operation failed  \n" if (not $response);
```

#Create a DHCP IPv6 OptionDefinition object.

```
my $ipv6od = Infoblox::DHCP::OptionDefinition->new(
                                space => "SUNW",
                                name  => "JumpStart-server",
                                code  => "6",
                                type  => "text"
                              );
unless ($ipv6od) {
    die("Construct DHCP IPv6OptionDefinition failed: ",
    Infoblox::status_code() . ":" . Infoblox::status_detail());
}
print "DHCP IPv6OptionDefinition object created successfully\n";

#Add the DHCP IPv6OptionDefinition object to Infoblox appliance through a session
my $response = $session->add( $ipv6od )
   or die("Add a DHCP IPv6OptionDefinition object failed: ",
          $session->status_code() . ":" . $session->status_detail());

print "Add a DHCP IPv6OptionDefinition object successfully\n";
```

#Get and modify a DHCP OptionDefinition object attribute

```
#Get Network through the session
my @retrieved_objs = $session->get(
               object   => "Infoblox::DHCP::IPv6OptionDefinition",
               space  => "SUNW",
               name   => "JumpStart-server",
            );

my $object = $retrieved_objs[0];

unless ($object) {
    die("Get DHCP IPv6OptionDefinition object failed: ",
        $session->status_code() . ":" . $session->status_detail());
}
print "Get DHCP IPv6OptionDefinition object found at least 1 matching entry\n";

#Modify one of the attributes of the specified Network attribute

$object->code(20);    #Apply the changes

$session->modify($object)
    or die("Modify DHCP IPv6OptionDefinition attribute failed: ",
           $session->status_code() . ":" . $session->status_detail());

print "DHCP IPv6OptionDefinition object modified successfully \n";
```

#Remove a DHCP IPv6OptionDefinition Object

```
#Get DHCP IPv6OptionDefinition object through the session
my @retrieved_objs = $session->get(
               object =>"Infoblox::DHCP::IPv6OptionDefinition",
               space  => "SUNW",
               name   => "JumpStart-server",
            );
```

```
my $object = $retrieved_objs[0];

unless ($object) {
    die("Get DHCP IPv6OptionDefinition object failed: ",
        $session->status_code() . ":" . $session->status_detail());
}

print "Get DHCP IPv6OptionDefinition object found at least 1 matching entry\n";
```

#Submit the object for removal

```
$session->remove($object)
    or die("Remove Network failed: ",
        $session->status_code() . ":" . $session->status_detail());

print "DHCP IPv6OptionDefinition removed successfully \n";

#Remove option space.

my $response = $session->remove( object => "Infoblox::DHCP::IPv6OptionSpace" ,
                                 name   => "SUNW",
                               );
print "Remove option space success \n" if ($response );
print "Remove option space failed  \n" if ( not $response );

####PROGRAM ENDS####
```

AUTHOR

Infoblox Inc. http://www.infoblox.com/

SEE ALSO

Infoblox::Session, Infoblox::DHCP::IPv6OptionSpace, Infoblox::DHCP::IPv6Option, Infoblox::Session->get(), Infoblox::Session->add(), Infoblox::Session->remove(), Infoblox::Session->modify()

COPYRIGHT

Copyright (c) 2017 Infoblox Inc.

Infoblox::DHCP::ExclusionRange - DHCP ExclusionRange object.

- NAME
- DESCRIPTION
- CONSTRUCTOR
- MODULE METHODS
  - Infoblox::DHCP::Range->exclude( )
- METHODS
  - comment( )
  - end_address( )
  - start_address( )
- SAMPLE CODE
- AUTHOR
- SEE ALSO
- COPYRIGHT

NAME

Infoblox::DHCP::ExclusionRange - DHCP ExclusionRange object.

DESCRIPTION

The DHCP exclusion range describes the range of IP addresses that can be excluded in an address range. If static addresses are assigned to certain hosts in the middle of an address range, those addresses can be excluded from the address range so the DHCP server does not assign those IP addresses to clients.

CONSTRUCTOR

```
my $exclusion_range = Infoblox::DHCP::ExclusionRange->new(
      end_address   => $ipv4addr,      #Required
      start_address => $ipv4addr,      #Required
      comment       => $string,        #Optional / Default is empty
);
```

MODULE METHODS

The following functions are available to apply to a DHCP exclusion range object.

Infoblox::DHCP::Range->exclude( )

Use this function to specify exclusion range for the DHCP range. See Infoblox::DHCP::Range->exclude() for parameters and return values.

Example

```
#Construct a DHCP ExclusionRange object
my $exclusion_range = Infoblox::DHCP::ExclusionRange->new(
         end_address   => "10.0.0.8",
         start_address => "10.0.0.5",
         comment       => "Exclude four IP addresses"
);

# Configure DHCP ExclusionRange on the DHCP Range object
my $response = $dhcp_range->exclude([$exclusion_range]);
```

METHODS

This section describes all the methods that can be used to configure and retrieve the attribute values of a DHCP exclusion range object.

comment( )

Use this method to set or retrieve a descriptive comment of a DHCP exclusion range object.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

Desired comment in string format with a maximum of 256 bytes.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get comment
my $comment = $exclusion_range->comment();
#Modify comment
$exclusion_range->comment("Modifying the DHCP exclusion range comment");
```

end_address( )

Use this method to set or retrieve the end address of a DHCP exclusion range object.

An end address is the last IP address in the DHCP exclusion range which is not available for the clients.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

Specify the end address of the DHCP exclusion range in IPv4 address format. An IPv4 address is a 32-bit number in dotted decimal notation. It consists of four 8-bit groups of decimal digits separated by decimal points (example: 192.168.1.2).

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get end_address
my $end_address = $exclusion_range->end_address();
#Modify end_address
$exclusion_range->end_address("10.0.0.9");
```

start_address( )

Use this method to set or retrieve the start address of a DHCP exclusion range object.

A start address is the first IP address in the DHCP exclusion range which is not available for the clients.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

Specify the start address of the DHCP exclusion range in IPv4 address format. An IPv4 address is a 32-bit number in dotted decimal notation. It consists of four 8-bit groups of decimal digits separated by decimal points (example: 192.168.1.2).

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get start_address
my $start_address = $exclusion_range->start_address();
#Modify start_address
$exclusion_range->start_address("10.0.0.6");
```

SAMPLE CODE

The following sample code demonstrates the different functions that can be applied to a DHCP exclusion range object such as add, modify, and remove. Also, this sample includes error handling for the operations.

#Preparation prior to a DHCP exclusion range object insertion

```
#PROGRAM STARTS: Include all the modules that will be used
use strict;
use Infoblox;

#Create a session to the Infoblox appliance
my $session = Infoblox::Session->new(
              master   => "192.168.1.2", #appliance host ip
                                         #appliance user login
```

```
                username => "admin",
                password => "infoblox"     #appliance password
);
unless ($session) {
       die("Construct session failed: ",
            Infoblox::status_code() . ":" . Infoblox::status_detail());
}
print "Session created successfully\n";

#Create the Network object
my $network = Infoblox::DHCP::Network->new(
    network => "10.0.0.0/8",
    comment => "add network",
);
unless($network) {
       die("Construct Network object failed: ",
            Infoblox::status_code() . ":" . Infoblox::status_detail());
}
print "Network object created successfully\n";

#Add the Network object into the Infoblox appliance through a session
$session->add($network)
    or die("Add Network object failed: ",
            $session->status_code() . ":" . $session->status_detail());
print "Network object added to Infoblox appliance successfully\n";

#Create the DHCP Range object
my $dhcp_range = Infoblox::DHCP::Range->new(
        end_addr   => "10.0.0.10",
        network    => "10.0.0.0/8",
        start_addr => "10.0.0.1",
);
unless($dhcp_range) {
       die("Construct DHCP Range object failed: ",
            Infoblox::status_code() . ":" . Infoblox::status_detail());
}
print "DHCP Range object created successfully\n";

#Add the DHCP Range object into the Infoblox appliance through a session
$session->add($dhcp_range)
    or die("Add DHCP Range object failed: ",
            $session->status_code() . ":" . $session->status_detail());
print "DHCP Range object added to Infoblox appliance successfully\n";
```

#Create a DHCP exclusion range object

```
my $exclusion_range = Infoblox::DHCP::ExclusionRange->new(
         end_address   => "10.0.0.8",
         start_address => "10.0.0.5",
         comment       => "Exclude four IP addresses"
);
unless($exclusion_range) {
       die("Construct DHCP exclusion range object failed: ",
            Infoblox::status_code() . ":" . Infoblox::status_detail());
}
print "DHCP exclusion range object created successfully\n";
```

#Add DHCP exclusion range to DHCP Range

```
#Get the DHCP Range object
my @retrieved_objs = $session->get(
    object     => "Infoblox::DHCP::Range",
    start_addr => "10.0.0.1"
);
my $object = $retrieved_objs[0];
unless ($object) {
       die("Get DHCP Range object failed: ",
            $session->status_code() . ":" . $session->status_detail());
}
print "Get DHCP Range object found at least 1 matching entry\n";

#Apply the changes to the DHCP Range object
$object->exclude([$exclusion_range]);

#Submit the changes to the Session
$session->modify($object)
    or die("Adding DHCP exclusion range to DHCP Range object failed: ",
```

```
            $session->status_code() . ":" . $session->status_detail());
print "DHCP Range with exclusion range updated to Infoblox appliance successfully\n";
```

#Modify DHCP exclusion range

```
#Modifying the end address of the DHCP exclusion range object.
$exclusion_range->end_address("10.0.0.9");

#Modifying the start address of the DHCP exclusion range object.
$exclusion_range->start_address("10.0.0.6");

#Modifying the comment of the DHCP exclusion range object.
$exclusion_range->comment("Modified DHCP Exclusion Range");

#Apply changes to the DHCP Range object.
$object->exclude([$exclusion_range]);

#Update DHCP Range object through the Infoblox session.
$session->modify($object)
  or die("Modify DHCP Range object failed: ",
  $session->status_code() . ":" . $session->status_detail());
print "DHCP Range object with modified DHCP exclusion range updated to Infoblox appliance successfully\n";
```

#Remove DHCP exclusion range

```
#Remove the exclusion range from the DHCP Range object.
$object->exclude([]);

#Update DHCP Range object through the Infoblox session.
$session->modify($object)
  or die("Modify DHCP Range object failed: ",
  $session->status_code() . ":" . $session->status_detail());
print "Removed DHCP exclusion range from DHCP Range successfully\n";
```

####PROGRAM ENDS####

AUTHOR

Infoblox Inc. http://www.infoblox.com/

SEE ALSO

Infoblox::Session, Infoblox::Session->add(),Infoblox::Session->get(),Infoblox::Session->modify(),Infoblox::DHCP::Range,Infoblox::DHCP::Network

---

COPYRIGHT

Copyright (c) 2017 Infoblox Inc.

Infoblox::DHCP::ExclusionRangeTemplate - DHCP ExclusionRangeTemplate object.

- NAME
- DESCRIPTION
- CONSTRUCTOR
- MODULE METHODS
  - Infoblox::DHCP::RangeTemplate->exclude( )
- METHODS
  - comment( )
  - number_of_addresses( )
  - offset( )
- SAMPLE CODE
- AUTHOR
- SEE ALSO
- COPYRIGHT

NAME

Infoblox::DHCP::ExclusionRangeTemplate - DHCP ExclusionRangeTemplate object.

DESCRIPTION

The DHCP exclusion range template describes the range of IP addresses that can be excluded in an address range template. If static IP addresses are assigned to certain hosts in the middle of an address range, those addresses can be excluded from the address range so the DHCP server does not assign those IP addresses to the clients.

CONSTRUCTOR

```
my $exclusion_range_template = Infoblox::DHCP::ExclusionRangeTemplate->new(
      offset               => $num,          #Required
      number_of_addresses  => $num,          #Required
      comment              => $string,       #Optional / Default is empty
);
```

MODULE METHODS

The following functions are available to be applied to a DHCP exclusion range template object.

Infoblox::DHCP::RangeTemplate->exclude( )

Use this function to specify exclusion range for the DHCP range template object. See Infoblox::DHCP::RangeTemplate->exclude() for parameters and return values.

Example

```
#Construct a DHCP ExclusionRangeTemplate object
my $exclusion_range_template = Infoblox::DHCP::ExclusionRangeTemplate->new(
        offset               => "10",
        number_of_addresses  => "5",
        comment              => "Exclude five IP addresses"
);

# Configure DHCP ExclusionRangeTemplate on the DHCP Range template object
my $response = $dhcp_range_template->exclude([$exclusion_range_template]);
```

METHODS

This section describes all the methods that can be used to configure and retrieve the attribute values of a DHCP exclusion range template object.

comment( )

Use this method to set or retrieve a descriptive comment of a DHCP exclusion range template object.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

Desired comment in string format with a maximum of 256 bytes.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get comment
my $comment = $exclusion_range_template->comment();
#Modify comment
$exclusion_range_template->comment("Modifying the DHCP exclusion range template comment");
```

number_of_addresses( )

Use this method to set or retrieve the number of addresses in the DHCP exclusion range template object.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

The number of addresses in this exclusion range template.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get number_of_addresses
my $number_of_addresses = $exclusion_range_template->number_of_addresses();
#Modify number_of_addresses
$exclusion_range_template->number_of_addresses("10");
```

offset( )

Use this method to set or retrieve the address offset of the DHCP exclusion range template object.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

The address offset of this exclusion range template. The offset is calculated from the start address of network template to which the DHCP range template is assigned to. For example, if offset is set to 10 and number_of_addresses is set to 10, and network is set to 10.0.0.0/8, then start address of exclusion range created from this exclusion range template will be 10.0.0.10 and end address will be 10.0.0.20.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get offset
my $offset = $exclusion_range_template->offset();
#Modify offset
$exclusion_range_template->offset("10");
```

SAMPLE CODE

The following sample code demonstrates the different functions that can be applied to a DHCP exclusion range object such as add, modify, and remove. Also, this sample includes error handling for the operations.

#PROGRAM STARTS: Include all the modules that will be used

```
use strict;
use Infoblox;

#Create a session to the Infoblox appliance
my $session = Infoblox::Session->new(
              master   => "192.168.1.2", #appliance host ip
              username => "admin",       #appliance user login
              password => "infoblox"     #appliance password
```

```
);
unless ($session) {
       die("Construct session failed: ",
            Infoblox::status_code() . ":" . Infoblox::status_detail());
}
print "Session created successfully\n";
```

#Create the DHCP Range template object

```
my $dhcp_range_template = Infoblox::DHCP::RangeTemplate->new(
        name                    => "custom_range_template",
        offset                  => "10",
        number_of_addresses     => "10",
        authority               => "TRUE",
        bootfile                => "bootfile1.com",
);
unless($dhcp_range_template) {
       die("Construct DHCP Range template object failed: ",
            Infoblox::status_code() . ":" . Infoblox::status_detail());
}
print "DHCP Range template object created successfully\n";

#Add the DHCP Range object into the Infoblox appliance through a session
$session->add($dhcp_range_template)
    or die("Add DHCP Range template object failed: ",
            $session->status_code() . ":" . $session->status_detail());
print "DHCP Range object added to Infoblox appliance successfully\n";
```

#Create a DHCP exclusion range templat object

```
my $exclusion_range_template = Infoblox::DHCP::ExclusionRangeTemplate->new(
         offset              => "10",
         number_of_addresses => "5",
         comment             => "Exclude five IP addresses"
);
unless($exclusion_range_template) {
       die("Construct DHCP exclusion range template object failed: ",
            Infoblox::status_code() . ":" . Infoblox::status_detail());
}
print "DHCP exclusion range template object created successfully\n";
```

#Add DHCP exclusion range template to DHCP Range template

```
#Get the DHCP Range template object
my @retrieved_objs = $session->get(
    object     => "Infoblox::DHCP::RangeTemplate",
    name       => "custom_range_template"
);
my $object = $retrieved_objs[0];
unless ($object) {
       die("Get DHCP Range template object failed: ",
            $session->status_code() . ":" . $session->status_detail());
}
print "Get DHCP Range template object found at least 1 matching entry\n";

#Apply the changes to the DHCP Range template object
$object->exclude([$exclusion_range_template]);

#Submit the changes to the Session
$session->modify($object)
    or die("Adding DHCP exclusion range to DHCP Range template object failed: ",
            $session->status_code() . ":" . $session->status_detail());
print "DHCP Range template with exclusion range template updated to Infoblox appliance successfully\n";
```

#Remove DHCP exclusion range template

```
#Remove the exclusion range template from the DHCP Range template object.
$object->exclude([]);

#Update DHCP Range template object through the Infoblox session.
$session->modify($object)
  or die("Modify DHCP Range template object failed: ",
  $session->status_code() . ":" . $session->status_detail());
print "Removed DHCP exclusion range template from DHCP Range successfully\n";

####PROGRAM ENDS####
```

AUTHOR

Infoblox Inc. http://www.infoblox.com/

SEE ALSO

Infoblox::Session,Infoblox::DHCP::RangeTemplate,Infoblox::DHCP::NetworkTemplate

---

COPYRIGHT

Copyright (c) 2017 Infoblox Inc.

Infoblox::Grid::MSServer::DHCP - Grid Microsoft Server DHCP object.

- NAME
- DESCRIPTION
- SESSION METHODS
  - Infoblox::Session->get( )
  - Infoblox::Session->search( )
  - Infoblox::Session->modify( )
- METHODS
  - manage_dhcp()
  - enable_dhcp()
  - status()
  - status_detail()
  - status_last_updated()
  - last_sync_ts()
  - last_sync_status()
  - last_sync_detail()
  - login_name( )
  - login_password( )
  - extattrs( )
  - extensible_attributes()
  - address()
  - broadcast_address()
  - enable_ddns()
  - enable_invalid_mac( )
  - enable_monitoring( )
  - ddns_server_always_updates()
  - ms_options( )
  - comment()
  - read_only()
  - name()
  - static_hosts()
  - supports_failover( )
  - dynamic_hosts()
  - dhcp_utilization()
  - dhcp_utilization_status()
  - override_enable_invalid_mac( )
  - override_enable_monitoring( )
  - override_login( )
  - override_synchronization_interval( )
  - synchronization_interval( )
- AUTHOR
- SEE ALSO
- COPYRIGHT

NAME

Infoblox::Grid::MSServer::DHCP - Grid Microsoft (r) Server DHCP object.

DESCRIPTION

You can use Infoblox::Grid::MSServer::DHCP to configure some of the DHCP properties of a Microsoft DHCP server, including enabling/disabling the DHCP service. The service configurations of a grid are not inherited by this object.

SESSION METHODS

This section describes all the methods in an Infoblox::Session module that you can apply to a Microsoft Server DHCP object.

Infoblox::Session->get( )

Use this method to retrieve all the matching objects from the Infoblox appliance. See Infoblox::Session->get() for parameters and return values.

Key References

```
 Apply the following attributes to get a specific grid member object:

  address - Required. A Microsoft server address or name in FQDN (Fully Qualified Domain Name)
format.
```

Example

```
my @retrieved_objs = $session->get(
    object  => "Infoblox::Grid::MSServer::DHCP",
    address => "10.0.0.1");
```

Infoblox::Session->search( )

Use this method to retrieve all the matching objects from the Infoblox appliance. See Infoblox::Session->search() for parameters and return values.

Key References

```
Apply the following attributes to search for a specific grid member object:

  address - Required. A Microsoft server address or name in FQDN (Fully Qualified Domain Name)
format. (Regular expressions are supported)
```

Example

```
my @retrieved_objs = $session->search(
    object  => "Infoblox::Grid::MSServer::DHCP",
    address => "domain.*");
```

Infoblox::Session->modify( )

Use this method to modify an object in the Infoblox appliance. See Infoblox::Session->modify() for parameters and return values.

Example

```
# Use this method to modify a field of a grid Microsoft DHCP server.
$grid_msserver_dhcp->enable_dhcp('true');
# Submit modification
my $response = $session->modify( $grid_msserver_dhcp );
```

METHODS

This section describes all the methods that you can use to configure and retrieve the attribute values of a grid Microsoft DHCP server.

manage_dhcp()

Retrieve or set the managed state of the Microsoft DHCP server. When setting the managed state, use true to enable or false to disable synchronization of the DHCP server. Setting the status does not stop or start the actual DHCP server, but controls whether it is synchronized.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

Specify "true" to set the disable flag or "false" to deactivate/unset it.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get manage_dhcp
my $manage_dhcp = $msserver_dhcp->manage_dhcp();
#Modify manage_dhcp
$msserver_dhcp->manage_dhcp("true");
```

enable_dhcp()

Set the DHCP service status of the Microsoft DHCP server. When setting the service status, use true to enable or false to disable the DHCP service on the Microsoft DHCP server. Setting the service status stops or starts the actual DHCP service. This only applies if `manage_dhcp()` is true.

When setting the service status, the Microsoft DHCP server will be started or stopped at the next synchronization.

Include the specified parameter to set the attribute value. This setting is write-only. To get the actual status of the service, use the `status()` method.

Parameter

Specify "true" to set the disable flag or "false" to deactivate/unset it.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get enable_dhcp
my $enable_dhcp = $msserver_dhcp->enable_dhcp();
#Modify enable_dhcp
$msserver_dhcp->enable_dhcp("true");
```

status()

Retrieve the running status of the DHCP service of the Microsoft DHCP server. The running status is "running", "stopped", or "error". The returned status is the latest known status as of the date returned by the `status_last_updated()` attribute.

Returns

The method returns the attribute value.

Example

```
#Get status
my $status = $msserver_dhcp->status();
```

status_detail()

Retrieve the detailed running status of the DHCP service of the Microsoft DHCP server. The returned status is the latest known status.

Returns

The method returns the attribute value.

Example

```
#Get status
my $status = $msserver_dhcp->status();
```

status_last_updated()

Retrieve the time when the DHCP service status of the Microsoft DHCP server was last updated.

Returns

The method returns the attribute value.

Example

```
#Get status_last_updated
my $status_last_updated = $msserver_dhcp->status_last_updated();
```

last_sync_ts()

Retrieve the timestamp of the end of the last synchronization attempt related to all DHCP data, except leases.

Returns

The method returns the attribute value.

Example

```
#Get last_sync_ts
my $last_sync_ts = $msserver_dhcp->last_sync_ts();
```

last_sync_status()

Retrieve the result of the last synchronization attempt, if last_sync_ts is defined. Valid values are "Ok", "Warning", or "Error".

Returns

The method returns the attribute value.

Example

```
#Get last_sync_status
my $last_sync_status = $msserver_dhcp->last_sync_status();
```

last_sync_detail()

Retrieve the detailed result of the last synchronization attempt, if last_sync_ts is defined.

Returns

The method returns the attribute value.

Example

```
#Get last_sync_detail
my $last_sync_detail = $msserver_dhcp->last_sync_detail();
```

login_name( )

Use this method to set or retrieve the Microsoft Server DHCP login name.

Setting this method to a defined value implicitly sets the override_login method to "true". Setting the parameter to undefined causes the appliance to use the Grid default and automatically resets the override_login attribute to "false".

Note that when login_name is set to a defined value and override_dlogin_name is set to "false", the last operation takes precedence. Thus the sequence $object->login_name('newuser'); $object->override_login("false"); will set override_login to "false", and the sequence $object->override_login("false"); $object->login_name('newuser'); will result in override_login="true".

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

When setting a value that requires a domain name, prepend the domain information followed by a backslash to the username: "domain\username".

Parameter

The valid value is a string that contains the login name.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get login value
my $login = $msserver_dhcp->login_name();
#Modify login value
$msserver_dhcp->login_name("newuser");
```

login_password( )

Use this method to set a password for the Microsoft Server DHCP. This field is used only if the login field for this member is defined. This is a write-only attribute.

This field cannot be retrieved.

Parameter

The valid value is a string that contains the password.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

Example

```
#Set the password
$msserver_dhcp->login_password("iG&ojG97Y");
```

extattrs( )

Use this method to retrieve the extensible attributes associated with a Grid Microsoft DHCP Server object. This attribute is read-only and cannot be set.

Parameter

None.

Returns

The method returns the attribute value.

Example

```
#Get extattrs
my $ref_extattrs = $msserver_dhcp->extattrs();
```

extensible_attributes()

Use this method to retrieve the extensible attributes of the associated Grid Microsoft DHCP Server object.

This method is read-only. Use the Infoblox::Grid::MSServer object to set or modify the extensible attributes of the Grid Microsoft DHCP Server object.

Returns

The method returns the attribute value.

Example

```
#Get extensible_attributes
my $extensible_attributes = $msserver_dhcp->extensible_attributes();
```

address()

Use this method to retrieve the IP address or FQDN of the associated Grid Microsoft DHCP Server object.

This method is read-only. Use the Infoblox::Grid::MSServer object to set or modify the Grid Microsoft DHCP Server object address.

Returns

The method returns the attribute value.

Example

```
#Get address
my $address = $msserver_dhcp->address();
```

broadcast_address()

Use this method to set or retrieve the broadcast address of the associated Grid Microsoft DHCP Server object.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Returns

The Microsoft DHCP server broadcast address in IPV4 format.

Example

```
#Get address
my $address = $msserver_dhcp->broadcast_address();
#Modify broadcast_address
$msserver_dhcp->broadcast_address("10.0.0.1");
```

enable_ddns()

Retrieve or set the dynamic DNS flag of the Microsoft DHCP server.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

Specify "true" to set the disable flag or "false" to deactivate/unset it.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get enable_ddns
my $enable_ddns = $msserver_dhcp->enable_ddns();
#Modify enable_ddns
$msserver_dhcp->enable_ddns("true");
```

enable_invalid_mac( )

Use this method to allow invalid MAC address synchronization in DHCP leases and fixed addresses.

Setting this method to a defined value implicitly sets the override_enable_invalid_mac method to "true". Setting the parameter to undefined causes the appliance to use the grid default and automatically resets the override_enable_invalid_mac method to "false".

Note that when enable_invalid_mac is set to a defined value and override_enable_invalid_mac is set to "false", the last operation takes precedence. Thus the sequence $object->enable_invalid_mac("true"); $object->override_enable_invalid_mac("false"); will set override_enable_invalid_mac to "false", and the sequence $object->override_enable_invalid_mac("false"); $object->enable_invalid_mac("true"); will result in override_enable_invalid_mac="true".

Parameter

Specify "true" to allow invalid MAC address synchronization or "false" to forbid it. Default value is "false".

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get enable_invalid_mac value
my $enable_invalid_mac = $msserver_dhcp->enable_invalid_mac();
#Modify enable_invalid_mac value
$msserver_dhcp->enable_invalid_mac("true");
```

enable_monitoring( )

Use this method to enable or disable the monitoring and control of DHCP service.

Setting this method to a defined value implicitly sets the override_enable_monitoring method to "true". Setting the parameter to undefined causes the appliance to use the grid default and automatically resets the override_enable_monitoring method to "false".

Note that when enable_monitoring is set to a defined value and override_enable_monitoring is set to "false", the last operation takes precedence. Thus the sequence $object->enable_monitoring("true"); $object->override_enable_monitoring("false"); will set override_enable_monitoring to "false", and the sequence $object->override_enable_monitoring("false"); $object->enable_monitoring("true"); will result in override_enable_monitoring="true".

Parameter

Specify "true" to allow invalid mac address synchronization or "false" to forbid it. Default value is "false".

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get enable_monitoring value
my $enable_monitoring = $msserver_dhcp->enable_dhcp_monitoring();
#Modify enable_monitoring value
$msserver_dhcp->enable_monitoring("true");
```

ddns_server_always_updates()

Retrieve or set the "Always update DDNS" flag of the Microsoft DHCP server. Use "true" to always allow dynamic DNS updates, or use "false" to only update when requested by the client. The default value is "false".

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

Specify "true" to set the disable flag or "false" to deactivate/unset it.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get ddns_server_always_updates
my $ddns_server_always_updates = $msserver_dhcp->ddns_server_always_updates();
#Modify ddns_server_always_updates
$msserver_dhcp->ddns_server_always_updates("true");
```

ms_options( )

Use this method to set or retrieve the custom DHCP options that are associated with the specified Microsoft DHCP server.

Note: You can set only options that are included in both the Microsoft server and NIOS predefined option definitions. Options that are not common to both are read-only and cannot be modified.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

The valid value is an array reference that contains defined Infoblox::DHCP::MSOption objects. The default value is empty.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get options
my $ref_options = $msserver_dhcp->ms_options();
#Modify options
$msserver_dhcp->ms_options([$option12]); #$option12 is an Infoblox::DHCP::MSOption object
```

comment()

Use this method to retrieve the comment associated with the Grid Microsoft DHCP Server object.

This method is read-only. Use the Infoblox::Grid::MSServer object to set or modify the Grid Microsoft DHCP Server object comment.

Returns

The method returns the attribute value.

Example

```
#Get comment
my $comment = $msserver_dhcp->comment();
```

read_only()

Use this method to retrieve the read_only flag of the associated Grid Microsoft DHCP Server object.

This method is read-only. Use the Infoblox::Grid::MSServer object to set or modify the Grid Microsoft DHCP Server object read_only status.

Returns

The method returns the attribute value.

Example

```
#Get read_only
my $read_only = $msserver_dhcp->read_only();
```

name()

Use this method to retrieve the name of the associated Grid Microsoft DHCP Server object.

This method is read-only. Use the Infoblox::Grid::MSServer object to set or modify the Grid Microsoft DHCP Server object name.

Returns

The method returns the attribute value.

Example

```
#Get name
my $name = $msserver_dhcp->name();
```

static_hosts()

Use this method to retrieve the number of static addresses (fixed addresses and reservations) associated with the Microsoft server.

Returns

The method returns the attribute value.

Example

```
#Get static_hosts
my $static_hosts = $msserver_dhcp->static_hosts();
```

supports_failover( )

Use this method to retrieve the flag used to indicate whether DHCP supports failover. This is a read-only attribute.

Parameter

None

Returns

The method returns the attribute value.

Example

```
#Get supports_failover value
my $supports_failover = msserver_dhcp->supports_failover();
```

dynamic_hosts()

Use this method to retrieve the number of active leases associated with the Microsoft server.

Returns

The method returns the attribute value.

Example

```
#Get dynamic_hosts
my $dynamic_hosts = $msserver_dhcp->dynamic_hosts();
```

dhcp_utilization()

Use this method to retrieve the DHCP utilization status of the Microsoft server. The returned value is between 0 and 1000. 1000 indicates 100% utilization.

Returns

The method returns the attribute value.

Example

```
#Get dhcp_utilization
my $dhcp_utilization = $msserver_dhcp->dhcp_utilization();
```

dhcp_utilization_status()

Use this method to retrieve the DHCP utilization status of the Microsoft server. The returned value is 'Low', 'Normal', 'High', or 'Full', and depend on the value of the low- and high-water mark parameters.

Returns

The method returns the attribute value.

Example

```
#Get dhcp_utilization_status
my $dhcp_utilization_status = $msserver_dhcp->dhcp_utilization_status();
```

override_enable_invalid_mac( )

The override_enable_invalid_mac attribute controls whether the enable_invalid_mac method values of the Microsoft Server DHCP object are used, instead of the Grid default.

The override_enable_invalid_mac attribute can be specified explicitly. It is also set implicitly when enable_invalid_mac is set to a defined value.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

Set the parameter to "true" to override the grid-level setting for enable_invalid_mac. Set the parameter to "false" to inherit the

grid-level setting for enable_invalid_mac. The default value is "false".

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get override_enable_invalid_mac
my $override_enable_invalid_mac = $msserver_dhcp->override_enable_invalid_mac();
#Modify override_enable_invalid_mac
$msserver_dhcp->override_enable_invalid_mac("true");
```

override_enable_monitoring( )

The override_enable_monitoring attribute controls whether the enable_monitoring method values of the Microsoft Server DHCP object are used, instead of the Grid default.

The override_enable_monitoring attribute can be specified explicitly. It is also set implicitly when enable_monitoring is set to a defined value.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

Set the parameter to "true" to override the grid-level setting for enable_monitoring. Set the parameter to "false" to inherit the grid-level setting for enable_monitoring. The default value is "false".

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get override_enable_monitoring
my $override_enable_monitoring = $msserver_dhcp->override_enable_monitoring();
#Modify override_enable_monitoring
$msserver_dhcp->override_enable_monitoring("true");
```

override_login( )

The override_login attribute controls whether the login_name method values of the Microsoft Server DHCP are used, instead of the Grid default.

The override_login attribute can be specified explicitly. It is also set implicitly when login_name is set to a defined value.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

Set the parameter to "true" to override the Grid-level setting for login_name. Set the parameter to "false" to inherit the Grid-level setting for login_name. The default value is "false".

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get override_login
my $override_login = $msserver_dhcp->override_login();
#Modify override_login
$msserver_dhcp->override_login("true");
```

override_synchronization_interval( )

The override_synchronization_interval attribute controls whether the synchronization_interval method values of the Microsoft Server DHCP are used, instead of the Grid default.

The override_synchronization_interval attribute can be specified explicitly. It is also set implicitly when synchronization_interval is set to a defined value.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

Set the parameter to "true" to override the Grid-level setting for synchronization_interval. Set the parameter to "false" to inherit the Grid-level setting for synchronization_interval. The default value is "false".

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get override_synchronization_interval
my $override_synchronization_interval = $msserver_dhcp->override_synchronization_interval();
#Modify override_synchronization_interval
$msserver_dhcp->override_synchronization_interval("true");
```

synchronization_interval( )

Use this method to set or retrieve the synchronization interval of the Microsoft Server DHCP.

Setting this method to a defined value implicitly sets the override_synchronization_interval method to "true". Setting the parameter to undefined causes the appliance to use the Grid default and automatically resets the override_synchronization_interval attribute to "false".

Note that when synchronization_interval is set to a defined value and override_dsynchronization_interval is set to "false", the last operation takes precedence. Thus the sequence $object->synchronization_interval(10); $object->override_synchronization_interval("false"); will set override_synchronization_interval to "false", and the sequence $object->override_synchronization_interval("false"); $object->synchronization_interval(10); will result in override_synchronization_interval="true".

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

The valid value is an unsigned integer.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get synchronization_interval value
my $synchronization_interval = $msserver_dhcp->synchronization_interval();
#Modify synchronization_interval value
$msserver_dhcp->synchronization_interval(10);
```

AUTHOR

Infoblox Inc. http://www.infoblox.com/

SEE ALSO

Infoblox::Grid::MSServer, Infoblox::DHCP::MSOption

COPYRIGHT

Copyright (c) 2017 Infoblox Inc.

Infoblox::Grid::SNMP::IPAMTrap - IPAM Utilization SNMP Trap object.

- NAME
- DESCRIPTION
- CONSTRUCTOR
- METHODS
  - enable_email_warnings( )
  - enable_snmp_warnings( )
- AUTHOR
- COPYRIGHT

NAME

Infoblox::Grid::SNMP::IPAMTrap - IPAM Utilization SNMP Trap object.

DESCRIPTION

The object provides information about the IPAM utilization SNMP trap.

CONSTRUCTOR

```
my $obj = Infoblox::Grid::SNMP::IPAMTrap->new(
    'enable_email_warnings' => 'true' | 'false',      # optional
    'enable_snmp_warnings'  => 'true' | 'false',      # optional
);
```

METHODS

This section describes all the methods that can be used to retrieve the attribute values of an object.

enable_email_warnings( )

Use this method to enable or disable email notifications.

Returns

Specify 'true' to enable email notifications or 'false' to disable it. The default value is 'true'.

Parameter

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get attrubute value
my $value = $obj->enable_email_warnings();
#Set attribute value
$obj->enable_email_warnings('true');
```

enable_snmp_warnings( )

Use this method to enable or disable SNMP notifications.

Returns

Specify 'true' to enable SNMP notifications or 'false' to disable it. The default value is 'true'.

Parameter

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get attrubute value
my $value = $obj->enable_snmp_warnings();
#Set attribute value
$obj->enable_snmp_warnings('true');
```

AUTHOR

Infoblox Inc. http://www.infoblox.com/

COPYRIGHT

Copyright (c) 2017 Infoblox Inc.

Infoblox::Grid::SNMP::IPAMThreshold - IPAM Utilization SNMP Threshold object.

- NAME
- DESCRIPTION
- CONSTRUCTOR
- METHODS
  - reset_value( )
  - trigger_value( )
- AUTHOR
- COPYRIGHT

NAME

Infoblox::Grid::SNMP::IPAMThreshold - IPAM Utilization SNMP Threshold object.

DESCRIPTION

The object provides information about the IPAM utilization SNMP threshold.

CONSTRUCTOR

```
my $obj = Infoblox::Grid::SNMP::IPAMThreshold->new(
    'reset_value'   => $uint,     # optional
    'trigger_value' => $uint,     # optional
);
```

METHODS

This section describes all the methods that can be used to retrieve the attribute values of an object.

reset_value( )

Use this method to set or retrieve a reset threshold value for the trap.

Returns

Valid value is unsigned integer between 0 and 100 inclusive.

Parameter

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get attrubute value
my $value = $obj->reset_value();
#Set attribute value
$obj->reset_value(55);
```

trigger_value( )

Use this method to set or retrieve a trigger threshold value for the trap.

Returns

Valid value is unsigned integer between 0 and 100 inclusive.

Parameter

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get attrubute value
my $value = $obj->trigger_value();
#Set attribute value
$obj->trigger_value(55);
```

AUTHOR

Infoblox Inc. http://www.infoblox.com/

COPYRIGHT

Copyright (c) 2017 Infoblox Inc.

Infoblox::IPAM::Address - IPAM address object

- NAME
- DESCRIPTION
- CONSTRUCTOR
- SESSION METHODS
  - Infoblox::Session->get( )
  - Infoblox::Session->remove( )
- METHODS
  - conflict_types( )
  - dhcp_client_identifier( )
  - discovered_data( )
  - discovered_name( )
  - discover_now_status( )
  - duid( )
  - discoverer( )
  - extattrs( )
  - extensible_attributes( )
  - fingerprint( )
  - first_discovered( )
  - ip_address( )
  - is_conflict( )
  - is_invalid_mac( )
  - last_discovered( )
  - lease_state( )
  - mac_address( )
  - ms_ad_user_data( )
  - name( )
  - netbios_name( )
  - network_component_description( )
  - network_component_ip( )
  - network_component_name( )
  - network_component_port_description( )
  - network_component_port_name( )
  - network_component_port_number( )
  - network_component_type( )
  - network_view( )
  - objects( )
  - os( )
  - port_duplex( )
  - port_link_status( )
  - port_speed( )
  - port_status( )
  - port_vlan_description( )
  - port_vlan_name( )
  - port_vlan_number( )
  - reserved_port( )
  - status( )
  - types( )
  - usage( )
  - username( )
  - v_cluster( )
  - v_datacenter( )
  - v_host( )
  - v_name( )
  - v_netadapter( )
  - v_switch( )
  - v_type( )
- SAMPLE CODE
- AUTHOR
- SEE ALSO
- COPYRIGHT

NAME

Infoblox::IPAM::Address - IPAM address object

DESCRIPTION

The IPAM address object is used to view IPv4 and IPv6 address management information in an Infoblox appliance.

CONSTRUCTOR

The Infoblox::IPAM::Address object is a read-only object and does not require manual construction.

SESSION METHODS

This section describes all the methods in an Infoblox::Session that can be applied to an IPAM address object.

Infoblox::Session->get( )

Use this method to retrieve the existing objects from an Infoblox appliance. See Infoblox::Session->get() for parameters and return values.

Note that for IPv4 addresses, both used and unused addresses are returned unless 'status' is specified; but IPv6 addresses will only return used addresses.

'discover_now_status' will not be returned by default, use 'return_methods' to request it if needed.

Key Reference

```
 Apply the following attributes to get the IPAM address object(s). Note that at least one of the
following is required: address, start_addr, end_addr, or network.
```

address - Optional. An IPv4 or IPv6 address. Get IPAM information for this address. All other optional arguments are ignored, except for network_view, if 'address' is present.

network - Optional. The IPv4 or IPv6 network, in CIDR notation (e.g 1.0.0.0/8 or 2000::/64), where the IPAM address is located.

network_view - Optional. The network view in which the network is located. The default is the system-defined default network view.

start_addr - Optional. An IPv4 or IPv6 address. Start address of an IPAM address scope. If the start address is not in the specified network, an error is returned.

end_addr - Optional. An IPv4 or IPv6 address. End address of an IPAM address scope. If the end address is not in the specified network, an error is returned.

status - Optional. The filter type of the IPAM address. Valid values are "used", "unused" and "all". Default value is "all". This argument applies to IPv4 addresses only. When the value is returned ", Unmanaged" or ", Conflict" might be appended to the status member when applicable.

type - Optional. This is exactly the same as the 'status' argument, and is being deprecated in favor of the 'status' argument. This argument applies to IPv4 addresses only.

extattrs - Optional. A hash reference containing extensible attributes.

extensible_attributes - Optional. A hash reference containing extensible attributes.

fingerprint - Optional. DHCP fingerprint description.

discovered_duid - Optional. Discovered DHCPv6 UID.

discovered_name - Optional. Discovered name.

discoverer - Optional. Discoverer name.

first_discovered - Optional. Time of the first discovery.

last_discovered - Optional. Time of the last discovery.

netbios - Optional. Discovered NetBIOS name.

network_component_description - Optional. Discovered network component description.

network_component_ip - Optional. Discovered network component IP address.

network_component_name - Optional. Discovered network component name.

network_component_port_description - Optional. Discovered network component port description.

network_component_port_name - Optional. Discovered network component port name.

network_component_port_number - Optional. Discovered network component port number.

network_component_type - Optional. Discovered network component type.

os - Optional. Discovered OS.

port_duplex - Optional. Discovered port duplex settings.

port_link_status - Optional. Discovered port link status.

port_speed - Optional. Discovered port speed settings.

port_status - Optional. Discovered port status settings.

port_vlan_description - Optional. Discovered port VLAN description.

port_vlan_name - Optional. Discovered port VLAN name.

port_vlan_number - Optional. Discovered port VLAN number.

v_cluster - Optional. Discovered VMware cluster name.

v_datacenter - Optional. Discovered VMware datacenter name.

v_host - Optional. Discovered VMware server name.

v_name - Optional. Discovered VMware host or virtual machine name.

v_netadapter - Optional. Discovered VMware physical network adapter name.

v_switch - Optional. Discovered VMware virtual switch name.

v_type - Optional. Discovered VMware entity type (possible values: "VirtualMachine", "HostSystem", "VirtualCenter").

Example

```
# case 1
my @ipam_objects = $session->get( 'object'     => 'Infoblox::IPAM::Address',
                                  'network'  => '10.0.0.0/8');
```

Since there are no start and end addresses specified, the appliance returns a maximum of 2000 IP addresses, which is the maximum page size of an administrator. If the total number of IP addresses in the network is less than 2000, then it returns all IP addresses.

```
# case 2
my @ipam_objects = $session->get( 'object'     => 'Infoblox::IPAM::Address',
                                  'network'    => '10.0.0.0/8',
                                  'start_addr' => '10.0.0.100');
```

This call returns a maximum of 2000 IPAM addresses starting from 10.0.0.100.

```
# case 3
my @ipam_objects = $session->get( 'object'     => 'Infoblox::IPAM::Address',
                                  'network'    => '10.0.0.0/8',
                                  'start_addr' => '10.0.0.100'
                                  'end_addr'   => '10.0.0.200');
```

This call returns 101 IPAM addresses from 10.0.0.100 to 10.0.0.200.

```
# case 4
my @ipam_objects = $session->get( 'object'     => 'Infoblox::IPAM::Address',
                                  'address'    => '10.0.0.10');
```

This call returns the IPAM address object for 10.0.0.10.

```
# case 5
my @ipam_objects = $session->get( 'object'     => 'Infoblox::IPAM::Address',
                                  'network'  => '10.0.0.0/8',
                                  'start_addr' => '10.0.0.100',
                                  'end_addr'  => '10.0.0.200',
                                  'status'          => 'used' );
```

This call returns used IPAM addresses from 10.0.0.100 to 10.0.0.200. Note that the filter is applied to the specified scope. Therefore, the number of returned objects could be less than 101, if only a few IP addresses are used in this scope.

```
# case 6
my @ipam_objects = $session->get( 'object        => 'Infoblox::IPAM::Address',
                                  'network'      => '10.0.0.0/8',
                                  'network_view' => 'my_network_view');
```

Same as case 1, but this uses the 'my_network_view' network view rather than the system-defined default view.

```
# case 7
my @ipam_objects = $session->get( 'object'     => 'Infoblox::IPAM::Address',
                                  'address'    => '10.0.0.5');
```

```
# case 8
my @ipam_objects = $session->get( 'object'     => 'Infoblox::IPAM::Address',
                                  'network'  => '2000::/64');
```

Since there are no start and end addresses specified, the appliance returns a maximum of 2000 used IP addresses, which is the maximum page size of an administrator. If the total number of IP addresses in the network is less than 2000, then it returns all IP addresses.

```
# case 9
my @ipam_objects = $session->get( 'object'     => 'Infoblox::IPAM::Address',
                                  'network'    => '2000::/64',
                                  'start_addr' => '2000::11');
```

This call returns a maximum of 2000 used IPv6 addresses starting from 2000::11.

```
# case 10
my @ipam_objects = $session->get( 'object'     => 'Infoblox::IPAM::Address',
                                  'network'    => '2000::/64',
                                  'start_addr' => '2000::11',
                                  'end_addr'   => '2000::31');
```

This call returns used IPv6addresses from '2000::11' to '2000::31'.

Infoblox::Session->remove( )

Use this method to remove an object from the Infoblox appliance. See Infoblox::Session->remove() for parameters and return values.

To remove a specific object, first use `get()` to retrieve the specific object, and then submit this object for removal.

Example

```
# Get the object under the same name
my @result_array = $session->get(
      object        => "Infoblox::IPAM::Address" ,
      network       => "2001::/64",
);

# find the desired object from the retrieved list.
my $desired_ipam_address = $result_array[0];

# Submit for removal
my $response = $session->remove( $desired_ipam_address );
```

METHODS

This section describes all the methods that can be used to retrieve the attribute values of an IPAM address object.

conflict_types( )

Use this method to retrieve a list of Discovery conflict types. This attribute is read-only.

Parameter

none

Returns

The method returns the attribute value.

Example

```
#Get conflict_types
my $value = $ipam_address->conflict_types();
```

dhcp_client_identifier( )

Use this method to retrieve the DHCP client identifier of an IPAM address object. This method applies to IPv4 addresses only.

Parameter

none

Returns

The method returns the DHCP client identifier attribute value.

Example

```
#Get DHCP client identifier
my $identifier = $ipam_address->dhcp_client_identifier();
```

discovered_data( )

Use this method to retrieve the discovered data of an IPAM address object. This is a read-only attribute.

Omit the parameter to retrieve the attribute value.

Parameter

None

Returns

The valid return value is an Infoblox::Grid::Discovery::Data object.

Example

```
#Get discovered_data value
my $discovered_data = $object->discovered_data();
```

discovered_name( )

Use this method to retrieve the discovered name of an IPAM address object.

Parameter

none

Returns

The method returns the discovered name.

Example

```
#Get discovered_name
my $discovered_name = $ipam_address->discovered_name();
```

discover_now_status( )

Use this method to retrieve the current discovery status. This attribute is read-only.

'discover_now_status' will not be returned by default, use 'return_methods' to request it if needed.

Parameter

None.

Returns

The method returns the attribute value that can be 'COMPLETE', 'FAILED', 'NONE', 'PENDING' or 'RUNNING'.

Example

```
#Get discover_now_status
my $discover_now_status = $ipam_address->discover_now_status();
```

duid( )

Use this method to retrieve the DHCPv6 Unique Identifier (DUID) of an IPAM address object.

Parameter

none

Returns

The method returns the DUID value.

Example

```
#Get mac address
my $duid = $ipam_address->duid();
```

discoverer( )

Use this method to retrieve the discoverer of an IPAM address object.

Parameter

none

Returns

The method returns the discoverer value.

Example

```
#Get discoverer
my $discoverer = $ipam_address->discoverer();
```

extattrs( )

Use this method to retrieve the extensible attributes associated with an IPAM address object. This attribute is read-only and cannot be set.

Parameter

None.

Returns

The method returns the attribute value.

Example

```
#Get extattrs
my $ref_extattrs = $ipam_address->extattrs();
```

extensible_attributes( )

Use this method to retrieve the extensible attributes associated with an IPAM address object.

Parameter

none

Returns

The method returns a reference to a hash containing the extensible attributes.

Example

```
#Get extensible attributes
my $ref_extensible_attributes = $ipam_address->extensible_attributes();
```

fingerprint( )

Use this method to retrieve DHCP fingerprint description of an IPAM address object. This is a read-only attribute.

Parameter

none

Returns

The method returns the attribute value.

Example

```
#Get fingerprint
my $fingerprint = $ipam_address->fingerprint();
```

first_discovered( )

Use this method to retrieve the date and time that the IPAM address object was first discovered.

Parameter

none

Returns

The method returns the date and time that the IPAM address object was fisrt discovered.

Example

```
#Get first_discovered
my $first_discovered = $ipam_address->first_discovered();
```

ip_address( )

Use this method to retrieve the IP addresses of an IPAM address object.

Parameter

none

Returns

The method returns the IP address value. The value can be an IPv4 or IPv6 address.

Example

```
#Get IP address
my $ip_address = $ipam_address->ip_address();
```

is_conflict( )

Use this method to find out whether this IPAM address is in conflict with another IPAM address in the system. This method applies to IPv4 addresses only.

Parameter

none

Returns

The method returns true if the IPAM address is in conflict with another in the system, false otherwise.

Example

```
#Get is_conflict
my $conflict = $ipam_address->is_conflict();
```

is_invalid_mac( )

Use this method to retrieve the flag used to indicate whether an invalid MAC address is used. This is a read-only attribute.

Parameter

None

Returns

The method returns the attribute value.

Example

```
#Get is_invalid_mac value
my $is_invalid_mac = $ipam_address->is_invalid_mac();
```

last_discovered( )

Use this method to retrieve the last discovered time of an IPAM address object.

Parameter

none

Returns

The method returns the last discovered value.

Example

```
#Get last discovered
my $last_discovered = $ipam_address->last_discovered();
```

lease_state( )

Use this method to retrieve the lease state of an IPAM address object. This method applies to IPv4 addresses only.

Parameter

none

Returns

The method returns the lease state value. A valid state is any of the following string values: free, backup, active, expired, released, abandoned, and reset.

Example

```
#Get lease state
my $lease_state = $ipam_address->lease_state();
```

mac_address( )

Use this method to retrieve the MAC address of an IPAM address object.

Parameter

none

Returns

The method returns the MAC address value.

Example

```
#Get mac address
my $mac_address = $ipam_address->mac_address();
```

ms_ad_user_data( )

Use this method to retrieve Microsoft Active Directory users related information. This is a read-only attribute.

Parameter

None

Returns

The valid return value is an Infoblox::Grid::MSServer::AdUser::Data object.

Example

```
#Get ms_ad_user_data
my $ms_ad_user_data = $ipam_address->ms_ad_user_data();
```

name( )

Use this method to retrieve the name of an IPAM address object.

Parameter

none

Returns

The method returns the name value.

Example

```
#Get name
my $name = $ipam_address->name();
```

netbios_name( )

Use this method to retrieve the NETBIOS name of an IPAM address object.

Parameter

none

Returns

The method returns the NETBIOS name value.

Example

```
#Get netbios name
```

```
my $netbios_name = $ipam_address->netbios_name();
```

network_component_description( )

Use this method to retrieve the description of the network device that is connected to the IPAM address object.

Parameter

none

Returns

The method returns the network device description.

Example

```
#Get network_component_description
my $network_component_description = $ipam_address->network_component_description();
```

network_component_ip( )

Use this method to retrieve the IP address of the network device that is connected to the IPAM address object.

Parameter

none

Returns

The method returns the network device IP address.

Example

```
#Get network_component_ip
my $network_component_ip = $ipam_address->network_component_ip();
```

network_component_name( )

Use this method to retrieve the name of the network device that is connected to the IPAM address object.

Parameter

none

Returns

The method returns the network device name.

Example

```
#Get network_component_name
my $network_component_name = $ipam_address->network_component_name();
```

network_component_port_description( )

Use this method to retrieve the description of the network device port that is connected to the IPAM address object.

Parameter

none

Returns

The method returns the network device port description.

Example

```
#Get network_component_port_description
my $network_component_port_description = $ipam_address->network_component_port_description();
```

network_component_port_name( )

Use this method to retrieve the name of the network device port that is connected to the IPAM address object.

Parameter

none

Returns

The method returns the network device port name.

Example

```
#Get network_component_port_name
my $network_component_port_name = $ipam_address->network_component_port_name();
```

network_component_port_number( )

Use this method to retrieve the number of the network device port that is connected to the IPAM address object.

Parameter

none

Returns

The method returns the network device port number.

Example

```
#Get network_component_port_number
my $network_component_port_number = $ipam_address->network_component_port_number();
```

network_component_type( )

Use this method to retrieve the type of the network device that is connected to the IPAM address object.

Parameter

none

Returns

The method returns the network device type.

Example

```
#Get network_component_type
my $network_component_type = $ipam_address->network_component_type();
```

network_view( )

Use this method to retrieve the network view name of an IPAM address object.

Parameter

none

Returns

The method returns the network view name.

Example

```
#Get network view
my $nview_name = $ipam_address->network_view();
```

objects( )

Use this method to retrieve the IPAM address object's associated objects.

Parameter

none

Returns

The method returns a reference to an array of associated objects. Note that this method has to be explicitly requested via Infoblox::Session's return_methods member.

Associated objects can be one or more of the following PAPI objects:

```
Infoblox::DHCP::FixedAddr
Infoblox::DHCP::IPv6FixedAddr
Infoblox::DHCP::Lease
Infoblox::DNS::BulkHost
```

```
Infoblox::DNS::Host
Infoblox::DNS::Record::A
Infoblox::DNS::Record::AAAA
Infoblox::DNS::Record::PTR
```

Example

```
my @retrieved_obj = $session->get(
                                    object     => 'Infoblox::IPAM::Address',
                                    start_addr => '10.0.0.0',
                                    end_addr   => '10.0.0.9',
                                    return_methods => ['+objects'],
                                   );

# Get the associated objects (if any)
my $associated_ref = $retrieved_obj[0]->objects();
```

os( )

Use this method to retrieve the OS of an IPAM address object.

Parameter

none

Returns

The method returns the OS value.

Example

```
#Get os
my $os = $ipam_address->os();
```

port_duplex( )

Use this method to retrieve the duplex setting of the network device port that is connected to the IPAM address object.

Parameter

none

Returns

The method returns the port duplex setting. Returned values, if any, are one of the following:

```
Full
Half
```

Example

```
#Get port_duplex
my $port_duplex = $ipam_address->port_duplex();
```

port_link_status( )

Use this method to retrieve the link status of the network device port that is connected to the IPAM address object.

Parameter

none

Returns

The method returns the port link status of the network device. Returned values, if any, are one of the following:

```
Connected
Not Connected
Unknown
```

Example

```
#Get port_link_status
my $port_link_status = $ipam_address->port_link_status();
```

port_speed( )

Use this method to retrieve the speed of the network device port that is connected to the IPAM address object.

Parameter

none

Returns

The method returns the port speed. Returned values, if any, are one of the following:

```
10M
100M
1G
10G
100G
Unknown
```

Example

```
#Get port_speed
my $port_speed = $ipam_address->port_speed();
```

port_status( )

Use this method to retrieve the status of the network device port that is connected to the IPAM address object.

Parameter

none

Returns

The method returns the network device port status. Returned values, if any, are one of the following:

```
Up
Down
Unknown
```

Example

```
#Get port_status
my $port_status = $ipam_address->port_status();
```

port_vlan_description( )

Use this method to retrieve the VLAN description of the network device port that is connected to the IPAM address object.

Parameter

none

Returns

The method returns the network device port VLAN description.

Example

```
#Get port_vlan_description
my $port_vlan_description = $ipam_address->port_vlan_description();
```

port_vlan_name( )

Use this method to retrieve the VLAN name of the network device port that is connected to the IPAM address object.

Parameter

none

Returns

The method returns the network device port VLAN name.

Example

```
#Get port_vlan_name
my $port_vlan_name = $ipam_address->port_vlan_name();
```

port_vlan_number( )

Use this method to retrieve the VLAN number of the network device port that is connected to the IPAM address object.

Parameter

none

Returns

The method returns the network device port VLAN number.

Example

```
#Get port_vlan_number
my $port_vlan_number = $ipam_address->port_vlan_number();
```

reserved_port( )

Use this method to retrieve the reserved port. This attribute is read-only.

Parameter

none

Returns

The method returns the attribute value.

Example

```
#Get reserved_port
my $value = $ipam_address->reserved_port();
```

status( )

Use this method to retrieve the status of an IPAM address object.

Parameter

none

Returns

The method returns the status value.

Example

```
#Get status
my $status = $ipam_address->status();
```

types( )

Use this method to retrieve the type of an IPAM address object.

Parameter

none

Returns

The method returns the type value. A valid value is a comma-separated list that contains a subset of the following values: A, FA, RESERVATION, RANGE, LEASE, UNMANAGED, HOST, BULKHOST, PTR, NETWORK, and BROADCAST. Note that in the subset of values, "FA" refers to "fixed address".

Example

```
#Get types
my $types= $ipam_address->types();
```

usage( )

Use this method to retrieve the usage of an IPAM address object.

Parameter

none

Returns

The method returns the usage value. A valid value is a comma-separated list that contains a subset of the following values: DNS, DHCP.

Example

```
#Get usage
my $usage= $ipam_address->usage();
```

username( )

Use this method to retrieve the user name of an IPAM address object. This method applies to IPv4 addresses only.

Parameter

none

Returns

The method returns the user name.

Example

```
#Get username
my $username= $ipam_address->username();
```

v_cluster( )

Use this method to retrieve the name of the VMware cluster associated with the IPAM Address object. This method is read-only.

Parameter

none

Returns

This method returns a string that contains the VMware cluster name. The default value is an empty string.

Example

```
#Get v_cluster
my $v_cluster = $ipam_address->v_cluster();
```

v_datacenter( )

Use this method to retrieve the name of the VMware datacenter associated with the IPAM Address object. This method is read-only.

Parameter

none

Returns

This method returns a string that contains the VMware datacenter name. The default value is an empty string.

Example

```
#Get v_datacenter
my $v_datacenter = $ipam_address->v_datacenter();
```

v_host( )

Use this method to retrieve the name of the VMware host associated with the IPAM Address object. This method is read-only.

Parameter

none

Returns

This method returns a string that contains the VMware host name. The default value is an empty string.

Example

```
#Get v_host
my $v_host = $ipam_address->v_host();
```

v_name( )

Use this method to retrieve the name of the VMware entity associated with the IPAM Address object. This method is read-only.

Parameter

none

Returns

This method returns a string that contains the VMware entity name. The default value is an empty string.

Example

```
#Get v_name
my $v_name = $ipam_address->v_name();
```

v_netadapter( )

Use this method to retrieve the name of the physical network adapter through which the VMware entity is connected to the appliance. This method is read-only.

Parameter

none

Returns

This method returns a string that contains the network adapter name. The default value is an empty string.

Example

```
#Get v_netadapter
my $v_netadapter = $ipam_address->v_netadapter();
```

v_switch( )

Use this method to retrieve the name of the virtual switch through which the VMware entity is connected to the appliance. This method is read-only.

Parameter

none

Returns

This method returns a string that contains the virtual switch name. The default value is an empty string.

Example

```
#Get v_switch
my $v_switch = $ipam_address->v_switch();
```

v_type( )

Use this method to retrieve the type of VMware entity associated with the IPAM Address object. This method is read-only.

Parameter

none

Returns

This method returns a string that contains the VMware entity type. The default value is an empty string. The returned value, if any, can be one of the following:

```
VirtualMachine
HostSystem
VirtualCenter
```

Example

```
#Get v_type
my $v_type = $ipam_address->v_type();
```

SAMPLE CODE

The following sample code demonstrates the get session method on an IPAM address object.

```
#PROGRAM STARTS: Include all the modules that will be used
use strict;
use Infoblox;
```

```
my $host_ip = "192.168.1.2";
my $host_name = "localhost";
my $node_ip = "192.168.1.2";

#Create a session to the Infoblox appliance
my $session = Infoblox::Session->new(
               master   => $host_ip,
               username => "admin",
               password => "infoblox"
               );

unless ($session) {
       die("Construct session failed: ",
               Infoblox::status_code() . ":" . Infoblox::status_detail());
}
print "Session created successfully\n";

my $memberdhcp = Infoblox::DHCP::Member->new(
          name     => $host_name,
          ipv4addr => $node_ip,
     );

#Create a large network object
my $network_large = Infoblox::DHCP::Network->new(
                                          "network" => "10.0.0.0/8",
                                          "members" => [$memberdhcp],
                                         );

$session->add($network_large)
   or die("Add network failed: " . $session->status_code() . ": " . $session->status_detail());

#Create another member
my $new_member = Infoblox::Grid::Member->new(
       grid      => "0",
       name      => "foo.com",
       ipv4addr  => "3.0.0.2",
       mask      => "255.255.255.0",
       gateway   => "3.0.0.1",
     );
$session->add($new_member)
   or die("Add grid member failed: " . $session->status_code() . ": " . $session->status_detail());

my $memberdhcp2 = Infoblox::DHCP::Member->new(
          name     => "foo.com",
          ipv4addr => "3.0.0.2",
     );

#Create a network view
my $nview = Infoblox::DHCP::View->new(
                                   "name" => "my_network_view",
                                  );
$session->add($nview)
   or die("Add network view failed: " . $session->status_code() . ": " . $session->status_detail());

#Create a DNS view associated with the network view
my $view = Infoblox::DNS::View->new(
  "name" => "my_dns_view",
  "network_view" => $nview,
);
$session->add($view)
   or die("Add DNS view failed: " . $session->status_code() . ": " . $session->status_detail());

#Create a DNS Zone in a non-default DNS view
my $zone = Infoblox::DNS::Zone->new(
  "name" => "test.com",
  "views" => [$view],
);
$session->add($zone)
   or die("Add DNS Zone failed: " . $session->status_code() . ": " . $session->status_detail());

#Create a small IPv4 network object with a non-default network view
my $network_small = Infoblox::DHCP::Network->new(
                                          "network" => "20.0.0.0/24",
                                          "members" => [$memberdhcp2],
                                          "network_view" => $nview,
                                         );
```

```
$session->add($network_small)
   or die("Add network failed: " . $session->status_code() . ": " . $session->status_detail());

#Create an IPv6 network object with a non-default network view
my $ipv6_network = Infoblox::DHCP::IPv6Network->new(
    "network" => "2001::/64",
    "network_view" => $nview,
);
$session->add($ipv6_network)
   or die("Add IPv6 network failed: " . $session->status_code() . ": " . $session->status_detail());

#Construct an IPv4 DHCP range object
my $dhcp_range = Infoblox::DHCP::Range->new(
        start_addr              => "10.0.0.1",
        end_addr                => "10.0.0.10",
        authority               => "true",
        bootfile                => "bootfile1.com",
        bootserver              => "someserver.com",
        comment                 => "add range",
        ddns_generate_hostname  => "true",
        deny_all_clients        => "true",
        deny_bootp              => "true",
        disable                 => "true",
        enable_ddns             => "true",
        member                  => $memberdhcp,
        nextserver              => "3.3.3.3",
        pxe_lease_time          => "3600",
        recycle_leases          => "true",
);

$session->add($dhcp_range)
   or die("Add DHCP range failed: " . $session->status_code() . ": " . $session->status_detail());

#Construct an IPv4 DHCP fixed address object
my $fixed_addr = Infoblox::DHCP::FixedAddr->new(
      "ipv4addr"       => "10.0.0.15",
      "mac"            => "11:11:11:11:11:11",
      "dhcp_client_identifier" => "",
      "client_identifier_prepend_zero"   => "false",
      "match_client"   => "MAC",
      "network"        => "10.0.0.0/8",
      "bootfile"       => "bootfile",
      "bootserver"     => "bootserver.com",
      "comment"        => "add fixed address",
      "deny_bootp"     => "false",
      "disable"        => "false",
      "dynamic"        => "false",
      "enable_ddns"    => "false",
      "nextserver"     => "3.3.3.3",
      "pxe_lease_time" => "3600",
);

$session->add($fixed_addr)
   or die("Add DHCP range failed: " . $session->status_code() . ": " . $session->status_detail());

#Construct IPv6 Host and AAAA record objects in a non-default DNS view
my $host = Infoblox::DNS::Host->new(
      "name"           => "host1.test.com",
      "ipv6addrs"      => ["2001::1", "2001::2", "2001::5"],
      "views"          => [$view],
);

$session->add($host)
   or die("Add host failed: " . $session->status_code() . ": " . $session->status_detail());

my $aaaa = Infoblox::DNS::Record::AAAA->new(
      "name"           => "myhost1.test.com",
      "ipv6addr"       => "2001::1",
      "views"          => [$view],
);

$session->add($aaaa)
   or die("Add AAAA record failed: " . $session->status_code() . ": " . $session->status_detail());

   # Case 1: Get IPAM address objects using 'network'
   # This should return 2000 IPAM addresses
   my @ipam objects   $session  get
```

```
                    =          ->    ('object'     => 'Infoblox::IPAM::Address',
                                     'network'    => '10.0.0.0/8');
    unless( scalar(@ipam_objects) == 2000) {
        die("Returned IPAM addresses number is not correct");
    }
    print("Returned IPAM addresses number correctly\n");

    # Case 2: Get IPAM address objects using 'start_addr'
    # This should return 2000 IPAM addresses
    my @ipam_objects = $session->get('object'     => 'Infoblox::IPAM::Address',
                                     'network'    => '10.0.0.0/8',
                                     'start_addr' => '10.0.0.10');
    unless( scalar(@ipam_objects) == 2000) {
        die("Returned IPAM addresses number is not correct");
    }
    print("Returned IPAM addresses number correctly\n");

    # Case 3: Get IPAM address objects using 'start_addr' and 'end_addr'
       # This should return the specified number of IPAM addresses
    my @ipam_objects = $session->get('object'     => 'Infoblox::IPAM::Address',
                                     'network'    => '10.0.0.0/8',
                                     'start_addr' => '10.0.0.10',
                                     'end_addr'   => '10.0.0.100');

    unless( scalar(@ipam_objects) == 91 ) {
        die("Returned IPAM addresses number is not correct");
    }
    print("Returned IPAM addresses number correctly\n");

    # Case 4: Get IPAM address objects using 'address'
    # This should return one IPAM address
    my @ipam_objects = $session->get('object'     => 'Infoblox::IPAM::Address',
                                     'address'    => '10.0.0.10');
    unless( scalar(@ipam_objects) == 1 ) {
        die("Returned IPAM addresses number is not correct");
    }
    print("Returned IPAM addresses number correctly\n");

   # Case 5: Get IPAM address objects using a 'start_addr' that is close to the end of the network
   # This should return IPAM addresses from the start_addr to the end of the  network
    my @ipam_objects = $session->get('object'     => 'Infoblox::IPAM::Address',
                                     'network'    => '10.0.0.0/8',
                                     'start_addr' => '10.255.255.100');

    unless( scalar(@ipam_objects) == 156 ) {
        die("Returned IPAM addresses number is not correct");
    }
    print("Returned IPAM addresses number correctly\n");

    # Case 6: Get IPAM address objects using 'start_addr' and 'end_addr'
       # but the total number of IP addresses in this scope is more than 2000.
       # This should return 2000 IPAM addresses starting with the start_addr.
    my @ipam_objects = $session->get('object'     => 'Infoblox::IPAM::Address',
                                     'network'    => '10.0.0.0/8',
                                     'start_addr' => '10.0.0.100',
                                     'end_addr'   => '10.255.255.100');

    unless( scalar(@ipam_objects) == 2000 ) {
        die("Returned IPAM addresses number is not correct");
    }
    print("Returned IPAM addresses number correctly\n");

   # Case 7: Get IPAM address objects from a small network.
   # This should return all IP addresses in this network.
    my @ipam_objects = $session->get('object'       => 'Infoblox::IPAM::Address',
                                     'network'      => '20.0.0.0/24',
                                     'network_view' => 'my_network_view');

    unless( scalar(@ipam_objects) == 256 ) {
        die("Returned IPAM addresses number is not correct");
    }
    print("Returned IPAM addresses number correctly\n");

    # Case 8: Get IPAM address objects using 'type = used'
       # This should return a total of 11 IP addresses in this network
       # which includes 10 in the range and 1 fixed address
    my @ipam_objects = $session->get('object'     => 'Infoblox::IPAM::Address',
                                     'network'    => '10.0.0.0/8',
                                     'start_addr' => '10.0.0.0',
```

```
                                    'end_addr'   => '10.0.0.100',
                                    'type'       => 'used');

unless( scalar(@ipam_objects) == 11 ) {
    die("Returned IPAM addresses number is not correct");
}
print("Returned IPAM addresses number correctly\n");

# Case 9: Get IPAM address objects using 'type = unused'
   # This should return a total of 90 (101 - 11 = 90) IP addresses in this network
   # which excludes 10 in the range and 1 fixed address
my @ipam_objects = $session->get('object'     => 'Infoblox::IPAM::Address',
                                    'network'    => '10.0.0.0/8',
                                    'start_addr' => '10.0.0.0',
                                    'end_addr'   => '10.0.0.100',
                                    'type'       => 'unused');

unless( scalar(@ipam_objects) == 90 ) {
    die("Returned IPAM addresses number is not correct");
}
print("Returned IPAM addresses number correctly\n");

# Case 10: Get IPAM address objects using 'address'
   # This should return one IPAM address object for later tests
my @ipam_objects = $session->get('object'     => 'Infoblox::IPAM::Address',
                                    'address'    => '10.0.0.15');
unless( scalar(@ipam_objects) == 1 ) {
    die("Returned IPAM addresses number is not correct");
}
print("Returned IPAM addresses number correctly\n");

my $ipam_object = $ipam_objects[0];

unless($ipam_object->status() eq "Used") {
    die("Returned IPAM address object's status is wrong");
}
print("Returned IPAM address object's status correctly\n");

unless($ipam_object->usage() eq "DHCP") {
    die("Returned IPAM address object's usage is wrong");
}
print("Returned IPAM address object's usage correctly\n");

unless($ipam_object->mac_address() eq $fixed_addr->mac()) {
    die("Returned IPAM address object's mac address is wrong");
}
print("Returned IPAM address object's mac address correctly\n");

unless($ipam_object->dhcp_client_identifier() eq $fixed_addr->dhcp_client_identifier()) {
    die("Returned IPAM address object's dhcp client identifier is wrong");
}
print("Returned IPAM address object's dhcp client identifier correctly\n");

# Case 11: Get IPv6 IPAM address objects using 'network'
# This should return 3 used IPAM addresses
my @ipam_objects = $session->get('object'     => 'Infoblox::IPAM::Address',
                                    'network'    => '2001::/64',
                                    'network_view' => 'my_network_view');
unless( scalar(@ipam_objects) == 3) {
    die("Returned IPAM addresses number is not correct");
}
print("Returned IPAM addresses number correctly\n");

# Case 12: Get IPv6 IPAM address objects using 'start_addr'
# This should return 2 used IPAM addresses (2001::2, 2001::5)
my @ipam_objects = $session->get('object'     => 'Infoblox::IPAM::Address',
                                    'network'    => '2001::/64',
                                    'start_addr' => '2001::2',
                                    'network_view' => 'my_network_view');
unless( scalar(@ipam_objects) == 2) {
    die("Returned IPAM addresses number is not correct");
}
print("Returned IPAM addresses number correctly\n");

# Case 13: Get IPv6 IPAM address objects using 'start_addr' and 'end_addr'
   # This should return 2 used IPAM addresses
my @ipam_objects = $session->get('object'     => 'Infoblox::IPAM::Address',
                                    'network'    => '2001::/64',
```

```
                                    'start_addr' => '2001::1',
                                    'end_addr'   => '2001::3',
                                    'network_view' => 'my_network_view');
    unless( scalar(@ipam_objects) == 2 ) {
        die("Returned IPAM addresses number is not correct");
    }
    print("Returned IPAM addresses number correctly\n");

    # Case 14: Get IPv6 IPAM address objects using 'address'
    # This should return one IPAM address
    my @ipam_objects = $session->get('object'     => 'Infoblox::IPAM::Address',
                                    'address'    => '2001::5',
                                    'network_view' => 'my_network_view');
    unless( scalar(@ipam_objects) == 1 ) {
        die("Returned IPAM addresses number is not correct");
    }
    print("Returned IPAM addresses number correctly\n");

    my $ipam_object = $ipam_objects[0];

    unless($ipam_object->ip_address() eq "2001::5") {
        die("Returned IPAM address object's address is wrong");
    }
    print("Returned IPAM address object's address correctly\n");

    unless($ipam_object->status() eq "Used") {
        die("Returned IPAM address object's status is wrong");
    }
    print("Returned IPAM address object's status correctly\n");

    unless($ipam_object->usage() eq "DNS") {
        die("Returned IPAM address object's usage is wrong");
    }
    print("Returned IPAM address object's usage correctly\n");

# Remove IPAM Address object
unless($session->remove($ipam_object)) {
        die("Remove IPAM Address object failed");
}
print "Removed IPAM Address correctly\n";

my @retrieved_objs = $session->get(
     object       => "Infoblox::DHCP::Network" ,
     network      => "10.0.0.0/8",
);
my $network = $retrieved_objs[0];

unless($session->remove($network)) {
        die("Remove Network object failed");
}
print "Removed network correctly\n";

@retrieved_objs = $session->get(
     object       => "Infoblox::DHCP::Network" ,
     network      => "20.0.0.0/24",
     network_view => "my_network_view",
);

$network = $retrieved_objs[0];

unless($session->remove($network)) {
        die("Remove Network object failed");
}
print "Removed network correctly\n";

$view = $session->get(
    object  => "Infoblox::DNS::View",
    name    => "my_dns_view",
);
unless($session->remove($view)) {
        die("Remove DNS View object failed");
}
print "Removed DNS view correctly\n";

$nview = $session->get(
    object  => "Infoblox::DHCP::View",
```

```
    name    => "my_network_view",
);
unless($session->remove($nview)) {
        die("Remove Network View object failed");
}
print "Removed network view correctly\n";

$new_member = $session->get(
    object  => "Infoblox::Grid::Member",
    name    => "foo.com",
);
unless($session->remove($new_member)) {
        die("Remove Grid Member object failed");
}
print "Removed grid member correctly\n";

####PROGRAM ENDS####
```

AUTHOR

Infoblox Inc. http://www.infoblox.com/

SEE ALSO

Infoblox::Session, Infoblox::DHCP::Network, Infoblox::DHCP::View, Infoblox::DHCP::FixedAddr, Infoblox::DHCP::Range, Infoblox::Session->get(), Infoblox::Grid::MSServer::AdUser::Data, Infoblox::Grid::Discovery::Data

---

COPYRIGHT

Copyright (c) 2017 Infoblox Inc.

Infoblox::DHCP::SharedNetwork - DHCP Shared Network object

- NAME
- DESCRIPTION
- CONSTRUCTOR
- SESSION METHODS
  - Infoblox::Session->add( )
  - Infoblox::Session->get( )
  - Infoblox::Session->modify( )
  - Infoblox::Session->remove( )
  - Infoblox::Session->search( )
- METHODS
  - authority( )
  - bootfile( )
  - bootserver( )
  - comment( )
  - ddns_generate_hostname( )
  - ddns_server_always_updates( )
  - ddns_update_fixed_addresses( )
  - ddns_ttl( )
  - ddns_use_option81( )
  - deny_bootp( )
  - disable( )
  - enable_ddns( )
  - extattrs( )
  - extensible_attributes( )
  - ignore_id( )
  - ignore_mac_addresses( )
  - ignore_dhcp_option_list_request( )
  - lease_scavenge_time( )
  - ms_ad_user_data( )
  - logic_filters( )
  - name( )
  - networks( )
  - network_view( )
  - nextserver( )
  - options( )
  - override_ddns_ttl( )
  - override_ignore_id( )
  - override_lease_scavenge_time( )
  - override_logic_filters( )
  - override_update_dns_on_lease_renewal( )
  - pxe_lease_time( )
  - update_dns_on_lease_renewal( )
- SAMPLE CODE
- AUTHOR
- SEE ALSO
- COPYRIGHT

NAME

Infoblox::DHCP::SharedNetwork - DHCP Shared Network object

DESCRIPTION

You can create a shared network when two subnets share a particular network segment. Before creating a shared network, you must first create the subnetworks. For example, you must first create the networks 10.32.1.0 and 10.30.0.0 before designating them as a shared network.

CONSTRUCTOR

```
  my $sharednetwork = Infoblox::DHCP::SharedNetwork->new(
                  name                              => $string,                          #Required
                  networks                          => [ $Network1,$Network2,...],       #Required
                  authority                         => "true" | "false" | undef,         #Optional / Default
is undefined
                  bootfile                          => $filename | undef,                #Optional / Default
is undefined
                  bootserver                        => $ipv4addr | $fqdn | undef,
#Optional / Default is undefined
                                                       $string   undef                   #Optional / Default
```

```
                comment                          =>         |      ,
is undefined
                ddns_generate_hostname           => "true" | "false" | undef,          #Optional / Default
is undefined
                ddns_server_always_updates       => "true" | "false",                  #Optional / Default
is "true"
                ddns_update_fixed_addresses      => "true" | "false" | undef,          #Optional / Default
is undefined
                ddns_ttl                         => $num,                              #Optional / Default
is zero
                ddns_use_option81                => "true" | "false" | undef,          #Optional / Default
is undefined
                deny_bootp                       => "true" | "false" | undef,          #Optional / Default
is undefined
                disable                          => "true" | "false",                  #Optional / Default
is "false"
                enable_ddns                      => "true" | "false" | undef,          #Optional / Default
is undefined
                extattrs                         => { $string => $extattr, ... },      #Optional / Default
is undefined
                extensible_attributes            => { $string => $string | $num, $string => [ $string |
$num, ... ], ... }, #Optional / Default is undefined
                ignore_dhcp_option_list_request  => "true" | "false" | undef,          #Optional / Default
is "false"
                ignore_id                        => 'NONE' | 'CLIENT' | 'MACADDR',     #Optional / Default
is 'NONE'
                ignore_mac_addresses             => [ "mac_addr", ... ] | undef        #Optional / Default
is undefined
                lease_scavenge_time              => $number,                           #Optional / Default
is -1
                network_view                     => $NetworkView,                      #Optional / Default
is "default" network view
                nextserver                       => $ipv4addr | $fqdn | undef,
#Optional / Default is undefined
                options                          => [$Option1,$Option2,...],           #Optional / Default
is empty list
                override_ddns_ttl                => "true" | "false",                  #Optional / Default
value is "false"
                override_lease_scavenge_time     => "true" | "false",                  #Optional / Default
value is "false"
                override_update_dns_on_lease_renewal  => "true" | "false",             #Optional / Default
value is "false"
                override_ignore_id               => "true" | "false",                  #Optional / Default
value is "false"
                pxe_lease_time                   => $num | undef,                      #Optional / Default
is undefined
                update_dns_on_lease_renewal      => "true" | "false",                  #Optional / Default
value is "false"
                logic_filters                    => [ $filter_name | $filter_obj, ...], # Optional /
Default is undefined
                override_logic_filters           => "true' | 'false",                    # Optional /
Default value is "false"
              );
```

You cannot set both extattrs and extensible_attributes attributes at the same time.

SESSION METHODS

This section describes all the methods in Infoblox::Session module that you can apply to a DHCP shared network object.

Infoblox::Session->add( )

Use this method to add a shared network object to the Infoblox appliance. See Infoblox::Session->add() for parameters and return values.

Example

```
#Construct an object for network
my $network1 = Infoblox::DHCP::Network->new(
        network => "20.0.0.0/8",
);

#Construct an object shared network
  my $sharednetwork = Infoblox::DHCP::SharedNetwork->new(
        name     => "myname",
        networks => [$network1],
);

# Submit for adding sharednetwork
my $response = $session->add( $sharednetwork );
```

Infoblox::Session->get( )

Use this method to retrieve all the matching shared network objects from the Infoblox appliance. See Infoblox::Session->get() for parameters and return values.

Key References

```
Apply the following attributes to get a specific DHCP shared network object:

  name           - Required.    The name of the shared network in string format.
  network        - Optional.    The network with which the shared network associated.
  network_view   - Optional.    The network view in which the shared network is located. By default,
all network views are searched.
  extattrs       - Optional. A hash reference containing extensible attributes.
  extensible_attributes - Optional. A hash reference containing extensible attributes.
```

Example

```
my  @result_array = $session->get(
      object       => "Infoblox::DHCP::SharedNetwork" ,
      name         => "myname",
      network_view => "default"
);

my  @result_array = $session->get(
      object       => "Infoblox::DHCP::SharedNetwork" ,
      network      => "1.0.0.0/24",
      network_view => "default"
);
```

Infoblox::Session->modify( )

Use this method to modify a shared network object in the Infoblox appliance. See Infoblox::Session->modify() for parameters and return values.

Example

```
#Use method to modify enable_ddns method.
$object->enable_ddns("true");
# Submit modification
my $response = $session->modify( $object );
```

Infoblox::Session->remove( )

Use this method to remove a shared network object from the Infoblox appliance. See Infoblox::Session->remove() for parameters and return values.

To remove a specifc object, first use `get()` or `search()` to retrieve the specific object, and then submit this object for removal.

Example

```
# Get the object under the same name
my  @result_array = $session->get(
      object       => "Infoblox::DHCP::SharedNetwork" ,
      name         => "myname",
      network_view => "default"
 );

# find the desired object from retrieved list.
my $desired_network = $result_array[0];

# Submit for removal
my $response = $session->remove( $desired_network );
```

Infoblox::Session->search( )

Use this method to search for DHCP shared network object in the Infoblox appliance. See Infoblox::Session->search() for parameters and return values.

Key References

```
Apply the following attributes to search for a DHCP shared network object:

  name           - Required. The name of the shared network in string format (regular expression).
  network        - Optional. The network with which the shared network associated.
  network_view   - Optional. The network view in which the shared network is located. By default, all
network views are searched.
  extattrs       - Optional. A hash reference containing extensible attributes.
  extensible_attributes - Optional. A hash reference containing extensible attributes.
```

For more information about searching extensible attributes, see Infoblox::Grid::ExtensibleAttributeDef/Searching Extensible Attributes.

Example

```
# search for all DHCP shared network objects in the "default" network view
    my @retrieved_objs = $session->search(
      object    => "Infoblox::DHCP::SharedNetwork",
      name      => "my.*",
      network_view => "default"
    );

# search all Shared Networks Templates with a given extensible attribute 'Site'
    my @retrieved_objs = $session->search(
      object => "Infoblox::DHCP::SharedNetwork",
      extensible_attributes => { 'Site' => 'Santa Clara' }
    );
```

METHODS

This section describes all the methods that you can use to configure and retrieve the attribute values of DHCP a shared network object

authority( )

Use this method to set or retrieve the authority flag of a shared network object. This setting overrides the member-level settings.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

Specify "true" to set the authority flag or "false" to deactivate/unset it. The default value for this field is undefined, which indicates that this attribute is inherited from the member-level setting.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
# Get authority
my $authority = $sharednetwork->authority()
# Modify authority
$sharednetwork->authority("true");
```

bootfile( )

Use this method to set or retrieve a bootfile attribute of a shared network object. You can configure the DHCP server to support clients that use the boot file name option in their DHCPREQUEST messages. This setting overrides the member-level settings. The default value is undefined, which indicates that this attribute is inherited from the member.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

This option is overridden independently from Infoblox::DHCP::SharedNetwork->bootserver() and Infoblox::DHCP::SharedNetwork->nextserver().

Parameter

The name of the file that the client must download. The file name must be in string format and can have a maximum of 128 characters. The default value is undefined.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
# Get bootfile
my $bootfile = $sharednetwork->bootfile();
# Modify bootfile
$sharednetwork->bootfile("bootfile1");
#Un-override bootfile
$sharednetwork->bootfile(undef);
```

---

bootserver( )

Use this method to set or retrieve the bootserver attribute of a shared network object. You can specify the name and/or IP address of the boot server that the host must access in order to boot. This setting overrides the member-level settings. The default value is undefined, which indicates that this attribute is inherited from the member.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

This option is overridden independently from Infoblox::DHCP::SharedNetwork->bootfile() and Infoblox::DHCP::SharedNetwork->nextserver().

Parameter

The boot server IP address or name in FQDN (Fully Qualified Domain Name) format. The FQDN consists of the host name followed by the domain name (example: abc.com). A boot server name can have a maximum of 256 bytes. The default value is undefined.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
# Get bootserver
my $bootserver = $sharednetwork->bootserver();
# Modify bootserver
$sharednetwork->bootserver("abc.domain.com");
#Un-override bootserver
$sharednetwork->bootserver(undef);
```

comment( )

Use this method to set or retrieve a descriptive comment for a shared network object.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

Enter a descriptive comment for the network in string format with a maximum of 256 bytes.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
# Get comment
my $comment = $sharednetwork->comment();
# Modify comment
$sharednetwork->comment("add a shared network");
```

ddns_generate_hostname( )

Use this method to set or retrieve the ddns_generate_hostname flag of a shared network object. Enable this method to allow only the DHCP server to generate host name. This setting overrides the member-level settings.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

Specify "true" to generate the host name or "false" to deactivate/unset it. The default value for this field is undefined, which indicates that this attribute is inherited from the member-level setting.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
# Get ddns_generate_hostname
my $ddns_generate_hostname = $sharednetwork->ddns_generate_hostname()
# Modify ddns_generate_hostname
$sharednetwork->ddns_generate_hostname("true");
#Un-override
```

```
$sharednetwork->ddns_generate_hostname(undef);
```

ddns_server_always_updates( )

Use this method to set or retrieve the ddns_server_always_updates flag of a shared network object. Enable this method to allow only the DHCP server to update DNS,regardless of the requests from the DHCP clients.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

Specify "true" to allow only the DHCP server to update DNS or "false" to deactivate/unset it. The default value for this field is "true".

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
# Get ddns_server_always_updates
my $ddns_server_always_updates = $sharednetwork->ddns_server_always_updates()
# Modify ddns_server_always_updates
$sharednetwork->ddns_server_always_updates("true");
```

ddns_update_fixed_addresses( )

Use this method to set or retrieve the ddns_update_fixed_addresses flag of a shared network object. By default, the DHCP server does not update DNS when it allocates a fixed address to a client. You can configure the DHCP server to update the A and PTR record of clients with fixed addresses. When you enable this feature and the DHCP server adds A and PTR records for a fixed address, the DHCP server never discards the records. This setting overrides the member-level settings.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

Specify "true" to update DNS when the device allocates a fixed address to a client or "false" to deactivate/unset it. The default value for this field is undefined.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
# Get ddns_update_fixed_addresses
my $ddns_update_fixed_addresses= $sharednetwork->ddns_update_fixed_addresses()
# Modify ddns_update_fixed_addresses
$sharednetwork->ddns_update_fixed_addresses("true");
#Un-override
$sharednetwork->ddns_update_fixed_addresses(undef);
```

ddns_ttl( )

Use this method to set or retrieve the DNS update Time to Live (TTL) value of a shared network object.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Setting this method to a defined value implicitly sets the override_ddns_ttl method to "true". Setting the parameter to undefined causes the appliance to use the grid default and automatically resets the override_ddns_ttl attribute to "false".

Note that when ddns_ttl is set to a defined value and override_dddns_ttl is set to "false", the last operation takes precedence. Thus the sequence $object->ddns_ttl(12); $object->override_ddns_ttl("false"); will set override_ddns_ttl to "false", and the sequence $object->override_ddns_ttl("false"); $object->ddns_ttl(12); will result in override_ddns_ttl="true".

Parameter

A 32-bit integer (range from 0 to 4294967295) that represents the duration,in seconds, that the update is cached. Zero indicates that the update should not be cached. The default value is zero.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get DNS Update ttl
my $dns_ttl = $sharednetwork->ddns_ttl();
#Modify DNS Update ttl
$sharednetwork->ddns_ttl(1200);
```

ddns_use_option81( )

Use this method to set or retrieve the ddns_use_option81 flag of a shared network object. Set this flag to define the settings for option 81 at the network level. This setting overrides the member-level settings.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

Specify "true" to configure support for option 81 or "false" to deactivate/unset it. The default value for this field is undefined.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
# Get ddns_use_option81
my $ddns_use_option81 = $sharednetwork->ddns_use_option81()
# Modify ddns_use_option81
$sharednetwork->ddns_use_option81("true");
# un-override ddns_use_option81
$sharednetwork->ddns_use_option81(undef);
```

deny_bootp( )

Use this method to set or retrieve the deny_bootp flag of a DHCP Shared Network object.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

Specify "true" to set the deny_bootp flag or "false" to deactivate/unset it. The default value is undefined, which indicates that this attribute is inherited from the member-level setting.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get deny_bootp
my $deny_bootp = $sharednetwork->deny_bootp();
#Modify deny_bootp
$sharednetwork->deny_bootp("true");
```

disable( )

Use this method to set or retrieve the disable flag of a shared network object.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

Specify "true" to set the disable flag and disable the shared network or "false" to unset the flag and enable the shared network. The default value for this field is "false".

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
# Get disable
```

```
my $disable = $sharednetwork->disable()
# Modify disable
$sharednetwork->disable("true");
```

enable_ddns( )

Use this method to set or retrieve the dynamic DNS updates flag of a DHCP shared network object. The DHCP server can send DDNS updates to DNS servers in the same grid and to external DNS servers. This setting overrides the member-level settings.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

Specify "true" to set the dynamic DNS updates flag or "false" to deactivate/unset it. The default value for this field is undefined, which indicates that this attribute is inherited from the member-level setting.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
# Get enable_ddns
my $enable_ddns = $sharednetwork->enable_ddns()
# Modify enable_ddns
$sharednetwork->enable_ddns("true");
# Un-override enable_ddns
$sharednetwork->enable_ddns(undef);
```

extattrs( )

Use this method to set or retrieve the extensible attributes associated with a Shared Network object.

Parameter

Valid value is a hash reference containing the names of extensible attributes and their associated values (Infoblox::Grid::Extattr objects).

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get extattrs
my $ref_extattrs = $sharednetwork->extattrs();
#Modify extattrs
$sharednetwork->extattrs({ 'Site' => $extattr1, 'Administrator' => $extattr2 });
```

extensible_attributes( )

Use this method to set or retrieve the extensible attributes associated with a Shared Network object.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

For valid values for extensible attributes, see Infoblox::Grid::ExtensibleAttributeDef/Extensible Attribute Values.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get extensible attributes
my $ref_extensible_attributes = $shared_network->extensible_attributes();
#Modify extensible attributes
$shared_network->extensible_attributes({ 'Site' => 'Santa Clara', 'Administrator' => [ 'Peter',
'Tom' ] });
```

ignore_id( )

Use this method to set or retrieve the ignore_id type of a DHCP shared network object. Indicates whether the appliance will ignore DHCP client IDs or MAC addresses. This setting overrides the upper-level settings.

Parameter

Valid values are 'NONE', 'CLIENT', or 'MACADDR'. The default is 'NONE'.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
# Get attribute value
$value = $shared_network->ignore_id();
# Modify attribute value
$shared_network->ignore_id('CLIENT');
#Un-override ignore_id
$shared_network->ignore_id(undef);
```

ignore_mac_addresses( )

Use this method to set or retrieve the ignore_mac_addresses list of a DHCP shared network object. A list of MAC addresses the appliance will ignore. This setting overrides the upper-level settings.

If you enter MAC addresses in the ignore_mac_addresses, then the appliance will ignore them. If you enter empty array, then the appliance will ignore any MAC address.

Parameter

The valid value is an array reference that contains string objects.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
# Get attribute value
$value = $shared_network->ignore_mac_addresses();
# Modify attribute value
$shared_network->ignore_mac_addresses(["11::11::11::11::11::11"]);
```

ignore_dhcp_option_list_request( )

Use this method to set or retrieve the ignore_dhcp_option_list_request flag of a DHCP shared network object. If this flag is set to true all the defined DHCP options will be returned to the client. This setting overrides the member level settings.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

Specify "true" to set the ignore_dhcp_option_list_request flag or "false" to deactivate/unset it. The default value is false which indicates that this attribute inherit the member level setting..

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
# Get ignore_dhcp_option_list_request
my $ignore_dhcp_option_list_request = $sharednetwork->ignore_dhcp_option_list_request();
# Un-override ignore_dhcp_option_list_request
$sharednetwork->ignore_dhcp_option_list_request(undef);
# Modify ignore_dhcp_option_list_request
$sharednetwork->ignore_dhcp_option_list_request("true");
```

lease_scavenge_time( )

Use this method to set or retrieve the lease_scavenge_time value of a Shared Network object.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Setting this method to a defined value implicitly sets the override_lease_scavenge_time method to "true". Setting the parameter to undefined causes the appliance to use the default from the upper level and automatically resets the override_lease_scavenge_time attribute to "false".

Note that when lease_scavenge_time is set to a defined value and override_lease_scavenge_time is set to "false", the last operation takes precedence. Thus the sequence $object->lease_scavenge_time(604800); $object->override_lease_scavenge_time("false"); will set override_lease_scavenge_time to "false", and the sequence $object->override_lease_scavenge_time("false"); $object->lease_scavenge_time(604800); will result in override_lease_scavenge_time="true".

Parameter

An integer that specifies the period of time, in seconds, that free and backup leases remain in the database before they are automatically deleted. To disable lease scavenging, set the parameter to -1. The minimum positive value must be greater than 86400 seconds (1 day). The default value of this parameter is -1, which means that lease scavenging is disabled.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get lease_scavenge_time
my $lease_scavenge = $sharednetwork->lease_scavenge_time();
#Modify lease_scavenge_time
$sharednetwork->lease_scavenge_time(30 * 24 * 60 * 60);
```

ms_ad_user_data( )

Use this method to retrieve Microsoft Active Directory users related information. This is a read-only attribute.

Parameter

None

Returns

The valid return value is an Infoblox::Grid::MSServer::AdUser::Data object.

Example

```
#Get ms_ad_user_data
my $ms_ad_user_data = $sharednetwork->ms_ad_user_data();
```

logic_filters( )

Use this method to set or retrieve all the logic (ordered) filters associated with the DHCP Shared Network.

Filters are used to screen address assignments in a DHCP Shared Network if the addresses are to be assigned to specific hosts.

Setting this method to a defined value implicitly sets the override_logic_filters method to "true". Setting the parameter to undefined causes the appliance to use the Grid default and automatically resets the override_logic_filters attribute to "false".

Note that when logic_filters is set to a defined value and override_dlogic_filters is set to "false", the last operation takes precedence. Thus the sequence $object->logic_filters($filters); $object->override_logic_filters("false"); will set override_logic_filters to "false", and the sequence $object->override_logic_filters("false"); $object->logic_filters($filters); will result in override_logic_filters="true".

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

The valid value is an array reference that contains either the Infoblox::DHCP::Filter::MAC, Infoblox::DHCP::Filter::NAC, Infoblox::DHCP::Filter::Option objects or the names of the corresponding filters. The filters in the list must satisfy the following conditions:

Filter name must be globally unique.
Filters without expressions must be at the end of the list.
The filters should not be disabled.

The default value of the parameter is undefined.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get logic_filters
my $ref_logic_filters = $sharednetwork->logic_filters();
#Modify logic_filters
$sharednetwork->logic_filters(["filter_1","filter_2",$exl1]); #$exl1 is an
Infoblox::DHCP::Filter::MAC, Infoblox::DHCP::Filter::NAC or Infoblox::DHCP::Filter::Option object.
```

name( )

Use this method to retrieve the name of a shared network object.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

The name of the shared network in string format. A name can have a maximum of 32 characters.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get name
my $sharednetwork = $sharednetwork->name();
```

networks( )

Use this method to retrieve the DHCP networks attribute of a shared network object.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

The valid value is an array reference that contains Infoblox::DHCP::Network objects.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
my $network1 = Infoblox::DHCP::Network->new(
                                    "network" => "20.0.0.0/8",
                                );
my $network2 = Infoblox::DHCP::Network->new(
                                    "network" => "30.0.0.0/8",
                                );

#Get networks
my $sharednetwork = $sharednetwork->networks();
#Modify networks
my $sharednetwork = $sharednetwork->networks([$network1, $network2]);
```

network_view( )

Use this method to set or retrieve the network view of the DHCP shared network.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

The default value is the "default" network view, which means the DHCP shared network is in the default network view.

Parameter

Valid value is a defined Infoblox::DHCP::View object. The default network view value is the system-defined default network view object.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get network view
my $nview = $sharednetwork->network_view();
#Modify network view, with an Infoblox::DHCP::View object
$sharednetwork->network_view($nview);
```

nextserver( )

Use this method to set or retrieve the nextserver attribute of a shared network object. You can specify the name and/or IP address of the next server that the host needs to access in order to boot. This setting overrides the member-level settings. The default value is undefined which indicates that this attribute is inherited from the member-level setting.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

This option is overridden independently from Infoblox::DHCP::SharedNetwork->bootfile() and Infoblox::DHCP::SharedNetwork->bootserver().

Parameter

The next server IP address or name in FQDN (Fully Qualified Domain Name) format. The FQDN consists of the host name followed by the domain name (example: abc.com). A next server name can have a maximum of 256 bytes. The default value is undefined.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
# Get nextserver
my $nextserver = $sharednetwork->nextserver();
# Modify nextserver
$sharednetwork->nextserver("blue.domain.com");
#Un-override nextserver
$sharednetwork->nextserver(undef);
```

options( )

Use this method to set or retrieve the options value of a member DHCP server. DHCP options provide network configuration settings and various services available on the network for DHCP. This setting overrides the member-level setting.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

The valid value is an array reference that contains Infoblox::DHCP::Option objects.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get options
my $options = $sharednetwork->options();
#Modify options
my $option1 = Infoblox::DHCP::Option->new(
    name  => "lease-time",
    value => 50
);

$sharednetwork->options([$option1]);
```

override_ddns_ttl( )

The override_ddns_ttl attribute controls whether the ddns_ttl value in the DHCP shared network object is used, instead of the grid default.

The override_ddns_ttl attribute can be specified explicitly. It is also set implicitly when ddns_ttl is set to a defined value.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

Set the parameter to "true" to override the grid-level setting for ddns_ttl. Set the parameter to "false" to inherit the grid-level

setting for ddns_ttl.

The default value of this parameter is "false".

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Getting override_ddns_ttl
my $override_ddns_ttl=$sharednetwork->override_ddns_ttl( );
#Modifying override_ddns_ttl
$sharednetwork->override_ddns_ttl("true");
```

override_ignore_id( )

The override_ignore_id controls whether the ignore_id in the DHCP shared network is used, instead of the upper-level default.

The override_ignore_id can be specified explicitly. It is also set implicitly when ignore_id is set to a defined value.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

Set the parameter to "true" to override the upper-level setting for ignore_id and ignore_mac_addresses. Set the parameter to "false" to inherit the upper-level setting for ignore_id and ignore_mac_addresses.

The default value of this parameter is "false".

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get attrubute value
my $value=$sharednetwork->override_ignore_id();
#Modifying attribute value
$sharednetwork->override_ignore_id('true');
```

override_lease_scavenge_time( )

The override_lease_scavenge_time attribute controls whether the lease_scavenge_time value in the shared network is used, instead of the upper-level default.

The override_lease_scavenge_time attribute can be specified explicitly. It is also set implicitly when lease_scavenge_time is set to a defined value.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

Set the parameter to "true" to override the upper-level setting for lease_scavenge_time. Set the parameter to "false" to inherit the setting for lease_scavenge_time from the upper level.

The default value of this parameter is "false".

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Getting override_lease_scavenge_time
my $override_lease_scavenge_time=$sharednetwork->override_lease_scavenge_time( );
#Modifying override_lease_scavenge_time
$sharednetwork->override_lease_scavenge_time("true");
```

override_logic_filters( )

The override_logic_filters attribute controls whether the logic_filters value of the DHCP Shared Network is used, instead of the Grid default.

The override_logic_filters attribute can be specified explicitly. It is also set implicitly when logic_filters is set to a defined value.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

Set the parameter to "true" to override the Grid-level setting for logic_filters. Set the parameter to "false" to inherit the Grid-level setting for logic_filters.

The default value of this parameter is "false".

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Getting override_logic_filters
my $override_logic_filters=$sharednetwork->override_logic_filters( );
#Modifying override_logic_filters
$sharednetwork->override_logic_filters("true");
```

override_update_dns_on_lease_renewal( )

The override_update_dns_on_lease_renewal attribute controls whether the update_dns_on_lease_renewal value in the DHCP shared network object is used, instead of the grid default.

The override_update_dns_on_lease_renewal attribute can be specified explicitly. It is also set implicitly when update_dns_on_lease_renewal is set to a defined value.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

Set the parameter to "true" to override the grid-level setting for update_dns_on_lease_renewal. Set the parameter to "false" to inherit the grid-level setting for update_dns_on_lease_renewal.

The default value of this parameter is "false".

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Getting override_update_dns_on_lease_renewal
my $override_update_dns_on_lease_renewal=$sharednetwork->override_update_dns_on_lease_renewal( );
#Modifying override_update_dns_on_lease_renewal
$sharednetwork->override_update_dns_on_lease_renewal("true");
```

pxe_lease_time( )

Use this method to set or retrieve the PXE lease time value of a shared network object. Some hosts use PXE (Preboot Execution Environment) to boot remotely from a server. To better manage your IP resources, set a different lease time for PXE boot requests. You can configure the DHCP server to allocate an IP address with a shorter lease time to hosts that send PXE boot requests, so IP addresses are not leased longer than necessary. This setting overrides the member-level settings. The default value is undefined, which indicates that this attribute is inherited from the member-level setting.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

A 32-bit integer (range from 0 to 4294967295) that represents the duration, in seconds, that the update is cached. Zero indicates that the update is not cached. The default value is undefined

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get PXE Lease Time
```

```
my $pxe_lease_time = $sharednetwork->pxe_lease_time();
#Modify PXE Lease Time
$sharednetwork->pxe_lease_time(1100);
#Un-override PXE Lease Time
$sharednetwork->pxe_lease_time(undef);
```

update_dns_on_lease_renewal( )

The update_dns_on_lease_renewal attribute controls whether the DHCP server updates DNS when a DHCP lease is renewed.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Setting this method to a defined value implicitly sets the override_update_dns_on_lease_renewal method to "true". Setting the parameter to undefined causes the appliance to use the grid default and automatically resets the override_update_dns_on_lease_renewal attribute to "false".

Note that when update_dns_on_lease_renewal is set to a defined value and override_update_dns_on_lease_renewal is set to "false", the last operation takes precedence. Thus the sequence $object->update_dns_on_lease_renewal("true"); $object->override_update_dns_on_lease_renewal("false"); will set override_update_dns_on_lease_renewal to "false", and the sequence $object->override_update_dns_on_lease_renewal("false"); $object->update_dns_on_lease_renewal("true"); will result in override_update_dns_on_lease_renewal="true".

Parameter

Specify "true" to set the update_dns_on_lease_renewal flag or "false" to deactivate/unset it.

The default value of this parameter is "false".

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get update_dns_on_lease_renewal
my $update_dns_on_lease_renewal=$sharednetwork->update_dns_on_lease_renewal( );
#Modify update_dns_on_lease_renewal
$sharednetwork->update_dns_on_lease_renewal("true");
```

SAMPLE CODE

The following sample code demonstrates the different functions that can be applied to an object, such as add, search, modify, and remove. This sample also includes error handling for the operations.

#Preparation prior to a DHCP shared network object insertion

```
#PROGRAM STARTS: Include all the modules that will be used
use strict;
use Infoblox;

#Create a session to the Infoblox appliance

my $session = Infoblox::Session->new(
    master   => "192.168.1.2",
    username => "admin",
    password => "infoblox"
);
unless ($session) {
   die("Construct session failed: ",
       Infoblox::status_code() . ":" . Infoblox::status_detail());
}
print "Session created successfully\n";
```

#Create a DHCP network object

```
my $network1 = Infoblox::DHCP::Network->new(
                                    "network" => "10.0.0.0/8",
                                );
my $network2 = Infoblox::DHCP::Network->new(
                                    "network" => "11.0.0.0/8",
                                );

my $response = $session->add($network1)
or die("Add \"10.0.0.0/8\"Network failed: ",
             $session->status_code() . ":" . $session->status_detail());
```

```
print "Network \"10.0.0.0/8\"added successfully\n";

my $response = $session->add($network2)
       or die("Add  \"11.0.0.0\"Network failed: ",
             $session->status_code() . ":" . $session->status_detail());

print "Network  \"11.0.0.0/8\",added successfully\n";
```

#Create a DHCP shared network object

```
my $sharednetwork1 = Infoblox::DHCP::SharedNetwork->new(
                                        "name"                          => "myname",
                                        "comment"                       => "add shared network",
                                        "networks"                      => [ $network1 , $network2 ],
                                        "disable"                       => "false",
                                        "authority"                     => "true",
                                        "enable_ddns"                   => "true",
                                        "ddns_use_option81"             => "true",
                                        "ddns_generate_hostname"        => "true",
                                        "ddns_server_always_updates"    => "true",
                                        "ddns_update_fixed_addresses"   => "true",
                                        "pxe_lease_time"                => "3600",
                                        "bootserver"                    => "someserver.com",
                                        "nextserver"                    => "3.3.3.3",
                                        "bootfile"                      => "bootfile.com",

                                        );

my $response = $session->add( $sharednetwork1 )
or die("Add Shared Network failed: ",
             $session->status_code() . ":" . $session->status_detail());

print "Shared Network added successfully\n";
```

#Search for a specific shared network object

```
#Search the network
my @retrieved_objs = $session->search(
    object  => "Infoblox::DHCP::SharedNetwork",
    "name"  => "my.*"
);
my $object = $retrieved_objs[0];
unless ($object) {
    die("Search Shared Network failed: ",
        $session->status_code() . ":" . $session->status_detail());
}
print "Search Shared Network found at least 1 matching entry\n";
```

#Get and modify a shared network attribute

```
#GetShared Network through the session
my  @retrieved_objs = $session->get(
           object =>"Infoblox::DHCP::SharedNetwork" ,
           "name" => "myname"
        );

my $object = $retrieved_objs[0];

unless ($object) {
    die("Get Shared Network object failed: ",
        $session->status_code() . ":" . $session->status_detail());
}
print "Get Shared Network found at least 1 matching entry\n";

#Modify attributes of the specified Network
$object->disable("true"); #Apply the changes
my $res = $session->modify($object) ;
if ( !$res)
{
    print "Modify Shared Network attribute failed: ".$session->status_code() . ":" . $session-
>status_detail();
}
else {
print "Shared Network object modified successfully \n";
}

#Un-override bootfile
$object->bootfile(undef);
```

```
#Un-override bootserver
$object->bootserver(undef);

#Un-override nextserver
$object->nextserver(undef);
```

#Remove a shared network object

```
 #Get Network object through the session
 my @retrieved_objs = $session->get(
           object  => "Infoblox::DHCP::SharedNetwork" ,
           name    => "myname"
    );

   my $object = $retrieved_objs[0];

   unless ($object) {
    die("Get Shared Network object failed: ",
        $session->status_code() . ":" . $session->status_detail());
    }

print "Get Shared Network object found at least 1 matching entry\n";

#Submit the shared network object for removal

$session->remove($object)
    or die("Remove Shared Network failed: ",
        $session->status_code() . ":" . $session->status_detail());

print "Shared Network removed successfully \n";

#Submit the network object for removal
my $response = $session->remove($network1)
or die("Remove \"10.0.0.0/8\"Network failed: ",
             $session->status_code() . ":" . $session->status_detail());

print "Network \"10.0.0.0/8\"remove successfully\n";

#Submit the network object for removal
my $response = $session->remove($network2)
       or die("Remove  \"11.0.0.0\"Network failed: ",
             $session->status_code() . ":" . $session->status_detail());

print "Network \"11.0.0.0/8\"remove successfully\

####PROGRAM ENDS####
```

AUTHOR

Infoblox Inc. http://www.infoblox.com/

SEE ALSO

Infoblox::Session, Infoblox::DHCP::Network, Infoblox::DHCP::View, Infoblox::DHCP::Option, Infoblox::Session->get(), Infoblox::Session->search(), Infoblox::Session->add(), Infoblox::Session->remove(), Infoblox::Session->modify(), Infoblox::Grid::MSServer::AdUser::Data

---

COPYRIGHT

Copyright (c) 2017 Infoblox Inc.

Infoblox::DHCP::OptionSpace - DHCP OptionSpace object.

- NAME
- DESCRIPTION
- CONSTRUCTOR
- SESSION METHODS
  - Infoblox::Session->add( )
  - Infoblox::Session->get( )
  - Infoblox::Session->modify( )
  - Infoblox::Session->remove( )
  - Infoblox::Session->search( )
- METHODS
  - name( )
  - comment( )
  - space_type
- SAMPLE CODE
- AUTHOR
- SEE ALSO
- COPYRIGHT

NAME

Infoblox::DHCP::OptionSpace - DHCP OptionSpace object.

DESCRIPTION

An Option Space defines a namespace in which vendor options can be defined. To define a specific vendor option space, add an option space to DHCP.

CONSTRUCTOR

```
my $option_space = Infoblox::DHCP::OptionSpace->new(
      name             => $string,                          #Required
      comment          => $string,                          #Optional, default is undefined
);
```

SESSION METHODS

This section describes all the methods in Infoblox::Session module that can be applied to a DHCP OptionSpace object.

Infoblox::Session->add( )

Use this method to add a DHCP OptionSpace object to the Infoblox appliance. See Infoblox::Session->add() for parameters and return values.

Example

```
#Construct a DHCP OptionSpace object
my $option_space = Infoblox::DHCP::OptionSpace->new(
      name => "infospace",
      comment => "some comment"
);
#Submit for addition
my $response = $session->add($option_space)
```

Infoblox::Session->get( )

Use this method to retrieve all the matching objects from the Infoblox appliance. See Infoblox::Session->get() for parameters and return values.

Key References

```
Apply the following attributes to get a specific DHCP OptionSpace object:

 name     - Required. Option space name in string format.
 comment  - Optional. Option space comment.
```

Example

```
#Get DHCP OptionSpace object through the session
```

```
my @retrieved_objs = $session->get(
    object => "Infoblox::DHCP::OptionSpace",
    name   => "infospace",
);
```

Infoblox::Session->modify( )

Use this method to modify a DHCP OptionSpace object in the Infoblox appliance. See Infoblox::Session->modify() for parameters and return values.

Example

```
#Use method to modify the name.
$option_space->name("info_space");
#Submit modification
my $response = $session->modify( $option_space );
```

Infoblox::Session->remove( )

Use this method to remove a DHCP OptionSpace object from the Infoblox appliance. See Infoblox::Session->remove() for parameters and return values.

To remove a specific object, first use `get()` to retrieve the specific DHCP OptionSpace object, and then submit this object for removal.

Example

```
#Get DHCP OptionSpace object through the session
my @retrieved_objs = $session->get(
    object => "Infoblox::DHCP::OptionSpace",
    name   => "info_space",
);
#Find the desired object from the retrieved list.
my $desired_option_space = $retrieved_objs[0];
#Submit for removal
my $response = $session->remove( $desired_option_space );
```

Infoblox::Session->search( )

Use this method to retrieve all the matching objects from the Infoblox appliance. See Infoblox::Session->search() for parameters and return values.

Key References

```
Apply the following attributes to get a specific DHCP OptionSpace object:

 name     - Optional. Option space name in string format (regular expression).
 comment  - Optional. Option space comment (regular expression).
```

Example

```
#Get DHCP OptionSpace object through the session
my @retrieved_objs = $session->search(
    object => "Infoblox::DHCP::OptionSpace",
    comment => ".*comm.*"
);
```

METHODS

This section describes all the methods that can used to set and retrieve the attribute values of a DHCP OptionSpace object.

name( )

Use this method to set or retrieve name of option space object.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

Desired name in string format with a maximum of 256 bytes.

Returns

If parameter is specified, the method returns true when the modification succeeds, and returns false when the operation fails.

If parameter is not specified, the method returns the attribute value.

Example

```
#Get name
my $name = $option_space->name();
#Modify name
$option_space->name("info_space");
```

comment( )

Use this method to set or retrieve a comment about the option space object.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

Desired comment in string format with a maximum of 256 bytes.

Returns

If the parameter is specified, the method returns true when the modification succeeds, and returns false when the operation fails.

If the parameter is not specified, the method returns the attribute value.

Example

```
#Get comment
my $comment = $option_space->comment();
#Modify comment
$option_space->comment("my comment");
```

space_type

Use this method to retrieve the type of this option space.

Parameter

None.

Returns

The method returns the attribute value, which can be either 'predefined_dhcp' or 'vendor_space'.

Example

```
#Get space_type
my $space_type = $option_space->space_type();
```

SAMPLE CODE

The following sample code demonstrates the different functions that can be applied to an OptionSpace object, such as get, add, modify and remove. In addition, the sample includes error handling for the operations.

#Preparation prior to a DHCP OptionSpace object insertion

```
#PROGRAM STARTS: Include all the modules that will be used
use strict;
use Infoblox;

#Create a session to the Infoblox appliance

my $session = Infoblox::Session->new(
               master   => "192.168.1.2", #appliance host ip
               username => "admin",       #appliance user login
               password => "infoblox"     #appliance password
);

unless ($session) {
       die("Construct session failed: ",
               Infoblox::status_code() . ":" . Infoblox::status_detail());
}
print "Session created successfully\n";
```

#Create an OptionSpace object

```
my $option_space = Infoblox::DHCP::OptionSpace->new (
       "name" => "infospace"
);
```

```
unless($option_space) {
     die("Construct option_space filter failed: ",
           Infoblox::status_code() . ":" . Infoblox::status_detail());
}
print "OptionSpace object created successfully\n";

#Verify if the DHCP OptionSpace exists in the Infoblox appliance
my $object = $session->get(
       object => "Infoblox::DHCP::OptionSpace",
       name   => "infospace"
);
unless ($object) {
   print "option space does not exist on server, safe to add the option_space\n";
   $session->add($option_space)
      or die("Add option space failed: ",
             $session->status_code() . ":" . $session->status_detail());
}
print "DHCP OptionSpace added successfully\n";
```

#Get and Modify an OptionSpace object

```
#Get OptionSpace object through the session
my @retrieved_objs = $session->get(
    object => "Infoblox::DHCP::OptionSpace",
    name   => "infospace"
);
my $object = $retrieved_objs[0];
unless ($object) {
    die("Get OptionSpace object failed: ",
        $session->status_code() . ":" . $session->status_detail());
}
print "Get OptionSpace object found at least 1 matching entry\n";

#Modify name of the obtained OptionSpace object
$object->name("info_space");

#Apply the changes
$session->modify($object)
    or die("Modify OptionSpace object failed: ",
           $session->status_code() . ":" . $session->status_detail());
print "OptionSpace object modified successfully \n";
```

#Remove an OptionSpace object

```
#Get OptionSpace object through the session
my @retrieved_objs = $session->get(
    object => "Infoblox::DHCP::OptionSpace",
    name   => "info_space"
);
my $object = $retrieved_objs[0];
unless ($object) {
    die("Get OptionSpace object failed: ",
        $session->status_code() . ":" . $session->status_detail());
}
print "Get OptionSpace object found at least 1 matching entry\n";

#Submit the object for removal
$session->remove($object)
    or die("Remove OptionSpace object failed: ",
        $session->status_code() . ":" . $session->status_detail());
print "OptionSpace object removed successfully \n";

####PROGRAM ENDS####
```

AUTHOR

Infoblox Inc. http://www.infoblox.com/

SEE ALSO

Infoblox::DHCP::OptionDefinition,Infoblox::Session->get(), Infoblox::Session->modify(), Infoblox::Session->remove(), Infoblox::Session

---

COPYRIGHT

Infoblox::DHCP::OptionSpace - DHCP OptionSpace object.

Copyright (c) 2017 Infoblox Inc.

Infoblox::DHCP::OptionDefinition - DHCP option definition object

- NAME
- DESCRIPTION
- CONSTRUCTOR
- SESSION METHODS
  - Infoblox::Session->add( )
  - Infoblox::Session->get( )
  - Infoblox::Session->modify( )
  - Infoblox::Session->search( )
  - Infoblox::Session->remove( )
- METHODS
  - name( )
  - code( )
  - space( )
  - type( )
- SAMPLE CODE
- AUTHOR
- SEE ALSO
- COPYRIGHT

NAME

Infoblox::DHCP::OptionDefinition - DHCP option definition object

---

DESCRIPTION

An OptionDefinition defines a DHCP option within a specific option space. A custom option can be defined in the predefined DHCP option space or in the user-defined vendor option space. To define an option, add the option definition to the required option space.

CONSTRUCTOR

```
my $option_definition1 = Infoblox::DHCP::OptionDefinition->new(
                                name  => $string, #Required
                                code  => $num,    #Required
                                space => $string, #Optional / Default is DHCP
                                type  => $string, #Optional / Default is string
                               );
```

SESSION METHODS

This section describes all the methods in an Infoblox::Session module that can be applied to a DHCP option definition object.

Infoblox::Session->add( )

Use this method to add a option definition object to the Infoblox appliance. See Infoblox::Session->add() for parameters and return values.

Example

```
#Create a DHCP OptionDefinition object.
my $option_definition1 = Infoblox::DHCP::OptionDefinition->new(
                                    space => "SUNW",
                                    name  => "JumpStart-server",
                                    code  => "6",
                                    type  => "text"
                                    );
# Submit for adding
my $response = $session->add( $option_definition1 );
```

Infoblox::Session->get( )

Use this method to retrieve all the matching DHCP option definition objects from the Infoblox appliance. See Infoblox::Session->get() for parameters and return values.

Key References

```
Apply the following attributes to get a specific DHCP option definition object:

 name - A name for the option definition in string format (regular expression).
```

```
  code - Option number. Either a name or code is required.
  space- Optional. The option space in which the option is defined. The default is the DHCP option
space.
```

Example

```
#Getting option by name
my  @result_array = $session->get(
                        object   => "Infoblox::DHCP::OptionDefinition",
                        space    => "SUNW",
                        name     => "JumpStart-server",
                        );

#Getting option by code
my @result_array = $session->get(
                        object   => "Infoblox::DHCP::OptionDefinition",
                        space    => "DHCP",
                        code     => 176,
                        );
```

Infoblox::Session->modify( )

Use this method to modify a DHCP option definition object in the Infoblox appliance. See Infoblox::Session->modify() for parameters and return values.

Example

```
#Use method to modify code
$object->code(20);
# Submit modification
my $response = $session->modify($object);
```

Infoblox::Session->search( )

Use this method to retrieve all the matching DHCP option definition objects from the Infoblox appliance. See Infoblox::Session->search() for parameters and return values.

Key References

```
Apply the following attributes to get a specific DHCP option definition object:

  name - A name for the option definition in string format (regular expression).
  code - Option number. Either a name or code is required.
  space- Optional. The option space in which the option is defined. The default is the DHCP option
space.
```

Example

```
#Getting option by name
my  @result_array = $session->search(
                        object   => "Infoblox::DHCP::OptionDefinition",
                        space    => "SUNW",
                        name     => "JumpStart-server",
                        );

#Getting option by code
my @result_array = $session->search(
                        object   => "Infoblox::DHCP::OptionDefinition",
                        space    => "DHCP",
                        code     => 176,
                        );
```

Infoblox::Session->remove( )

Use this method to remove a DHCP option definition object from the Infoblox appliance. See Infoblox::Session->remove() for parameters and return values.

To remove a specifc object, first use `get()` to retrieve the specific object, and then submit this object for removal.

Example

```
my  @result_array = $session->get(
                        object   => "Infoblox::DHCP::OptionDefinition",
                        space    => "SUNW",
                        name     => "JumpStart-server",
                        );

# find the desired object from retrieved list.
```

```
my $desired_obj = $result_array[0];

# Submit for removal
my $response = $session->remove($desired_obj);
```

METHODS

This section describes all the methods that can be used to configure and retrieve the attribute values of a DHCP option definition object.

name( )

Use this method to set or retrieve the name of a DHCP option definition object.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

Desired name in string format with a maximum of 256 bytes.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
# Get name
my $option_definitionname = $option_definition1 ->name();
# Modify name
$option_definition1 ->name("Start-server");
```

code( )

Use this method to set or retrieve the code of a DHCP option definition object. An option code number is used to identify the DHCP option.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

Number for the option code. Value must be between 1 and 254. Values 1-79, 81-82, 85-87, 89, 91-92, 98, 112-114, 118-119, 124-125 are predefined. These option codes have corresponding predefined Types. Therefore, you cannot change the predefined Types.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
# Get code
my $option_definitionname = $option_definition1 ->code();
# Modify code
$option_definition1 ->code(14);
```

space( )

Use this method to set or retrieve the space of a DHCP option definition object.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

Specify the space name in string format.

Returns

If you did not specify a parameter, the method returns the attribute value.

Example

```
# Get space
my $option_space = $option_definition1 ->space();
# Modify space
$option_definition1 ->space("SUNW");
```

---

type( )

Use this method to set or retrieve the data type of the grid DHCP option.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

The accepted types are the following:

```
  "boolean"                 - For "true" or "false" boolean values.

  "ip-address"              - For a single IPv4 address. An IPv4 address is a 32-bit number in dotted
decimal notation.
                              It consists of four 8-bit groups of decimal digits separated
                              by decimal points (example: "192.168.1.2").

  "array of ip-address"     - For a list of IPv4 addresses, each separated by a comma.
                              (example: "192.168.1.2, 10.11.12.13, 79.2.3.1")

  "string"                  - For value in ASCII text string or
                              a list of hexadecimal characters (0-9, a-f, or A-F) without spaces and
separated by colons.

  "text"                    - For value in ASCII text string.

  "domain-name"             - For domain name in ASCII text format

  "domain-list"             - For a list of domain names, each separated by a comma.

  "8-bit unsigned integer"  - For numeric value between 0 and 255.

  "16-bit unsigned integer" - For numeric value between 0 and 65535.

  "32-bit unsigned integer" - For numeric value between 0 and 4294967295.

  "8-bit signed integer"    - For numeric value between -128 and 127.

  "16-bit signed integer"   - For numeric value between -32768 and 32767.

  "32-bit signed integer"   - For numeric value between -2147483648 and 2147483647.

  "array of 8-bit unsigned integer"  - For a list of numeric values between 0 and 255, each separated
by a comma.

  "array of 16-bit unsigned integer" - For a list of numeric values between 0 and 65535, each
separated by a comma.

  "array of 32-bit unsigned integer" - For a list of numeric values between 0 and 4294967295, each
separated by a comma.

  "array of 8-bit integer"           - For a list of numeric values between -128 and 127, each
separated by a comma.

  "array of 16-bit integer"          - For a list of numeric values between -32768 and 32767, each
separated by a comma.

  "array of 32-bit integer"          - For a list of numeric values between -2147483648 and
2147483647, each separated by a comma.
```

The following types are valid only for `Microsoft(r)` server options

```
  "binary"

  "encapsulated"

  "array of string"

  "array of 64-bit unsigned integer" - For a list of numeric values between 0 and 1.8446744 * 10^19

  "64-bit unsigned integer"          - For a list of numeric values between 0 and 1.8446744 * 10^19,
each separated by a comma.
```

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
# Get type
my $option_definitiontype = $option_definition1 ->type();
# Modify type
$option_definition1 ->type("string");
```

SAMPLE CODE

The following sample code demonstrates the different functions that can be applied to an object such as add, modify, and remove. Also, this sample includes error handling for the operations.

#Preparation prior to a DHCP Network insertion

```
#PROGRAM STARTS: Include all the modules that will be used
use strict;
use Infoblox;

#Create a session to the Infoblox appliance

my $session = Infoblox::Session->new(
    master   => "192.168.1.2",
    username => "admin",
    password => "infoblox"
);
unless ($session) {
   die("Construct session failed: ",
       Infoblox::status_code() . ":" . Infoblox::status_detail());
}
print "Session created successfully\n";

my $option_space1 = Infoblox::DHCP::OptionSpace->new(
                                                "name" => "SUNW"
                                                );
my $response = $session->add( $option_space1);
print "Add option space operation success \n" if ($response );
print "Add option space operation failed  \n" if ( not $response );
```

#Create a DHCP OptionDefinition object.

```
my $option_definition1 = Infoblox::DHCP::OptionDefinition->new(
                                space => "SUNW",
                                name  => "JumpStart-server",
                                code  => "6",
                                type  => "text"
                              );
        unless ($option_definition1) {
        die("Construct DHCP OptionDefinition failed: ",
       Infoblox::status_code() . ":" . Infoblox::status_detail());
  }
 print "DHCP OptionDefinition object created successfully\n";

 #Add the DHCP OptionDefinition object to Infoblox appliance through a session
       my $response = $session->add( $option_definition1 )
       or die("Add a DHCP  OptionDefinition object failed: ",
             $session->status_code() . ":" . $session->status_detail());

print "Add a DHCP  OptionDefinition object successfully\n";
```

#Get and modify a DHCP OptionDefinition object attribute

```
#Get Network through the session
my  @retrieved_objs = $session->get(
              object   => "Infoblox::DHCP::OptionDefinition",
              space  => "SUNW",
              name   => "JumpStart-server",
            );

my $object = $retrieved_objs[0];

unless ($object) {
    die("Get DHCP OptionDefinition object failed: ",
        $session->status_code() . ":" . $session->status_detail());
}
print "Get DHCP OptionDefinition object found at least 1 matching entry\n";

#Modify one of the attributes of the specified Network attribute

 $object->code(20);    #Apply the changes

$session->modify($object)
    or die("Modify DHCP OptionDefinition attribute failed: ",
           $session->status_code() . ":" . $session->status_detail());
```

```
print "DHCP OptionDefinition object modified successfully \n";
```

#Remove a DHCP OptionDefinition Object

```
 #Get DHCP OptionDefinition object through the session
 my  @retrieved_objs = $session->get(
                  object =>"Infoblox::DHCP::OptionDefinition",
                  space  => "SUNW",
                  name   => "JumpStart-server",
            );

   my $object = $retrieved_objs[0];

   unless ($object) {
    die("Get DHCP OptionDefinition object failed: ",
        $session->status_code() . ":" . $session->status_detail());
    }

print "Get DHCP OptionDefinition object found at least 1 matching entry\n";
```

#Submit the object for removal

```
$session->remove($object)
    or die("Remove Network failed: ",
        $session->status_code() . ":" . $session->status_detail());

print "DHCP OptionDefinition removed successfully \n";

#Remove option space.

my $response = $session->remove( object => "Infoblox::DHCP::OptionSpace" ,
                                 name   => "SUNW",
                               );
print "Remove option space success \n" if ($response );
print "Remove option space failed  \n" if ( not $response );

####PROGRAM ENDS####
```

AUTHOR

Infoblox Inc. http://www.infoblox.com/

SEE ALSO

Infoblox::Session, Infoblox::DHCP::OptionSpace, Infoblox::DHCP::Option, Infoblox::Session->get(), Infoblox::Session->add(), Infoblox::Session->remove(), Infoblox::Session->modify()

COPYRIGHT

Copyright (c) 2017 Infoblox Inc.

Infoblox::DHCP::FilterRule::MAC - DHCP MAC Address Filter Rule object.

- NAME
- DESCRIPTION
- CONSTRUCTOR
- MODULE METHODS
  - Infoblox::DHCP::Range->filters( )
- METHODS
  - filter_name( )
  - permission( )
- SAMPLE CODE
- AUTHOR
- SEE ALSO
- COPYRIGHT

NAME

Infoblox::DHCP::FilterRule::MAC - DHCP MAC Address Filter Rule object.

DESCRIPTION

MAC Address Filter Rule allows a user to grant/deny the lease of an IP defined in DHCP Range.

CONSTRUCTOR

```
my $mac_filter_rule = Infoblox::DHCP::FilterRule::MAC->new(
      filter_name    => $string,          #Required
      permission     => "grant" | "deny", #Required
);
```

MODULE METHODS

The following functions can be applied to a DHCP MAC Filter Rule object.

Infoblox::DHCP::Range->filters( )

Use this method to add a DHCP MAC Address Filter Rule object to the DHCP Range on an Infoblox appliance. See Infoblox::DHCP::Range->filters() for parameters and return values.

Example

```
#Construct a DHCP MAC Address Filter Rule object
my $mac_filter_rule1 = Infoblox::DHCP::FilterRule::MAC->new(
      filter_name    => "filter1",
      permission     => "grant",
);

#Add MAC Address Filter Rule to the DHCP Range object.
my $response = $dhcp_range->filters([$mac_filter_rule1]);
```

METHODS

This section describes all the methods that can be used to set and retrieve the attribute values of a DHCP MAC Address Filter Rule object.

filter_name( )

Use this method to set or retrieve the filter_name of a DHCP MAC Address Filter Rule object.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

Name of the DHCP MAC Address filter. Maximum length up to 1024 bytes is supported.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get filter_name
my $filter_name = $mac_filter_rule->filter_name();
#Modify filter_name
$mac_filter_rule->filter_name("filter2");
```

permission( )

Use this method to set or retrieve the permission of a DHCP MAC Address Filter Rule object.

Permission indicates whether members of this MAC address filter are allowed or denied an IP address from the defined DHCP range.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

Permission value can be either "grant" or "deny".

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get permission
my $permission = $mac_filter_rule->permission();
#Modify permission
$mac_filter_rule->permission("deny");
```

SAMPLE CODE

The following sample code demonstrates different functions that can be applied to an object such as modify. Also, this sample includes error handling for the operations.

#Preparation prior to a DHCP MAC Address Filter Rule object insertion

```
#PROGRAM STARTS: Include all the modules that will be used
use strict;
use Infoblox;

#Create a session to the Infoblox appliance

my $session = Infoblox::Session->new(
               master   => "192.168.1.2", #appliance host ip
               username => "admin",       #appliance user login
               password => "infoblox"     #appliance password
);

unless ($session) {
       die("Construct session failed: ",
               Infoblox::status_code() . ":" . Infoblox::status_detail());
}
print "Session created successfully\n";

#Create the Network object
my $network = Infoblox::DHCP::Network->new(
    network => "10.0.0.0/8",
    comment => "add network",
);
unless($network) {
       die("Construct Network object failed: ",
            Infoblox::status_code() . ":" . Infoblox::status_detail());
}
print "Network object created successfully\n";

#Add the Network object into the Infoblox appliance through a session
$session->add($network)
    or die("Add Network object failed: ",
            $session->status_code() . ":" . $session->status_detail());
print "Network object added to Infoblox appliance successfully\n";

#Construct a DHCP MAC Address Filter object
my $mac_filter = Infoblox::DHCP::Filter::MAC->new(
       name => "filter1",
```

```
);

unless ( $mac_filter ) {
       die("Construct MAC Address Filter failed: ",
               Infoblox::status_code() . ":" . Infoblox::status_detail() );
}
print "MAC Address Filter object created successfully\n";

#Add the MAC Address Filter to the Infoblox appliance through a session
$session->add($mac_filter)
    or die("Add MAC Address Filter object failed: ",
            $session->status_code() . ":" . $session->status_detail());
print "MAC Address Filter object added to Infoblox appliance successfully\n";
```

#Create a DHCP MAC Address Filter Rule object

```
#Construct a DHCP MAC Address Filter Rule object
my $mac_filter_rule1 = Infoblox::DHCP::FilterRule::MAC->new(
      filter_name   => "filter1",
      permission    => "grant",
);

unless ( $mac_filter_rule1 ) {
       die("Construct MAC Address Filter Rule failed: ",
               Infoblox::status_code() . ":" . Infoblox::status_detail() );
}
print "MAC Address Filter Rule object created successfully\n";

#Create the DHCP Range object with this MAC Address Filter Rule.
my $dhcp_range = Infoblox::DHCP::Range->new(
        end_addr   => "10.0.0.10",
        network    => "10.0.0.0/8",
        start_addr => "10.0.0.1",
        filters    => [ $mac_filter_rule1 ],
);
unless($dhcp_range) {
       die("Construct DHCP Range object failed: ",
            Infoblox::status_code() . ":" . Infoblox::status_detail());
}
print "DHCP Range object created successfully\n";

#Add the DHCP Range object into the Infoblox appliance through a session
$session->add($dhcp_range)
    or die("Add DHCP Range object failed: ",
            $session->status_code() . ":" . $session->status_detail());
print "DHCP Range object added to Infoblox appliance successfully\n";
```

#Get and modify a MAC Address Filter Rule object

```
#Get MAC Address Filter Rule object from the DHCP Range object through session
my @retrieved_objs = $session->get(
    object      => "Infoblox::DHCP::Range",
    start_addr  => "10.0.0.1",
);

my $object = $retrieved_objs[0];
my $filter = $object->filters();

my @filters = @{$filter};

my $mac_filterrule = $filters[0];

#Modify one of the attributes of the obtained MAC Address Filter Rule object
$mac_filterrule->permission("deny");
$object->filters([$mac_filterrule]);

#Apply the changes
$session->modify($object)
    or die("Modify MAC Address Filter Rule object failed: ",
           $session->status_code() . ":" . $session->status_detail());
print "MAC Address Filter Rule object modified and added to the DHCP Range object successfully \n";
```

#Remove a MAC Address Filter Rule object

```
$object->filters([]);

my $response = $session->modify($object);
```

```
unless($response) {
       die("Remove MAC Address Filter Rule failed: ",
                session->status_code() . ":" . session->status_detail() );
}
print "MAC Address Filter Rule object removed successfully \n";

#Submit the network object for removal
$session->remove($network)
    or die("Remove Network object failed: ",
        $session->status_code() . ":" . $session->status_detail());
print "Network object removed successfully \n";

#Submit the mac address filter object for removal
$session->remove($mac_filter)
    or die("Remove MAC Address Filter object failed: ",
        $session->status_code() . ":" . $session->status_detail());
print "MAC Address Filter object removed successfully \n";

####PROGRAM ENDS####
```

AUTHOR

Infoblox Inc. http://www.infoblox.com/

SEE ALSO

Infoblox::DHCP::MAC, Infoblox::DHCP::FilterRule::RelayAgent, Infoblox::DHCP::Network, Infoblox::DHCP::Range, Infoblox::Session->add(),Infoblox::Session->modify(), Infoblox::Session->remove(),Infoblox::Session

COPYRIGHT

Copyright (c) 2017 Infoblox Inc.

Infoblox::DHCP::FilterRule::NAC - DHCP NAC Filter Rule object.

- NAME
- DESCRIPTION
- CONSTRUCTOR
- MODULE METHODS
  - Infoblox::DHCP::Range->filters( )
- METHODS
  - filter_name( )
  - permission( )
- SAMPLE CODE
- AUTHOR
- SEE ALSO
- COPYRIGHT

NAME

Infoblox::DHCP::FilterRule::NAC - DHCP NAC Filter Rule object.

DESCRIPTION

A NAC Filter Rule object is used to grant or deny the lease of an IP address in a DHCP range based on whether the authentication response matches the NAC filter.

CONSTRUCTOR

```
my $nac_filter_rule = Infoblox::DHCP::FilterRule::NAC->new(
      filter_name    => $string,          #Required
      permission     => "grant" | "deny", #Required
);
```

MODULE METHODS

The following functions can be applied to a DHCP NAC Filter Rule object.

Infoblox::DHCP::Range->filters( )

Use this method to add a DHCP NAC Filter Rule object to a DHCP range. See Infoblox::DHCP::Range->filters() for parameters and return values.

Example

```
#Construct a DHCP NAC Filter Rule object
my $nac_filter_rule1 = Infoblox::DHCP::FilterRule::NAC->new(
      filter_name    => "filter1",
      permission     => "grant",
);

#Add a NAC Filter Rule to the DHCP Range object.
my $response = $dhcp_range->filters([$nac_filter_rule1]);
```

METHODS

This section describes all the methods that can be used to set and retrieve the attribute values of a DHCP NAC Filter Rule object.

filter_name( )

Use this method to set or retrieve the filter_name of a DHCP NAC Filter Rule object.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

The name of the DHCP NAC filter. The maximum length supported is 1024 bytes.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get filter_name
my $filter_name = $nac_filter_rule->filter_name();
#Modify filter_name
$nac_filter_rule->filter_name("filter2");
```

permission( )

Use this method to set or retrieve the permission of a DHCP NAC Filter Rule object.

The permission indicates whether a DHCP client whose NAC authentication response matches this NAC filter is allowed or denied a lease from the defined DHCP range.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

The permission value can be either "grant" or "deny".

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get permission
my $permission = $nac_filter_rule->permission();
#Modify permission
$nac_filter_rule->permission("deny");
```

SAMPLE CODE

The following sample code demonstrates different functions that can be applied to an object such as modify. Also, this sample includes error handling for the operations.

#Preparation prior to a DHCP NAC Filter Rule object insertion

```
#PROGRAM STARTS: Include all the modules that will be used
use strict;
use Infoblox;

#Create a session to the Infoblox appliance

my $session = Infoblox::Session->new(
               master   => "192.168.1.2", #appliance host ip
               username => "admin",       #appliance user login
               password => "infoblox"     #appliance password
);

unless ($session) {
       die("Construct session failed: ",
               Infoblox::status_code() . ":" . Infoblox::status_detail());
}
print "Session created successfully\n";

#Create the Network object
my $network = Infoblox::DHCP::Network->new(
    network => "10.0.0.0/8",
    comment => "add network",
);
unless($network) {
       die("Construct Network object failed: ",
            Infoblox::status_code() . ":" . Infoblox::status_detail());
}
print "Network object created successfully\n";

#Add the Network object into the Infoblox appliance through a session
$session->add($network)
    or die("Add Network object failed: ",
            $session->status_code() . ":" . $session->status_detail());
print "Network object added to Infoblox appliance successfully\n";
```

```
#Construct a DHCP NAC Filter object
my $nac_filter = Infoblox::DHCP::Filter::NAC->new(
       name => "filter1",
);

unless ( $nac_filter ) {
       die("Construct NAC Filter failed: ",
               Infoblox::status_code() . ":" . Infoblox::status_detail() );
}
print "NAC Filter object created successfully\n";

#Add the NAC Filter to the Infoblox appliance through a session
$session->add($nac_filter)
    or die("Add NAC Filter object failed: ",
            $session->status_code() . ":" . $session->status_detail());
print "NAC Filter object added to Infoblox appliance successfully\n";
```

#Create a DHCP NAC Filter Rule object

```
#Construct a DHCP NAC Filter Rule object
my $nac_filter_rule1 = Infoblox::DHCP::FilterRule::NAC->new(
      filter_name   => "filter1",
      permission    => "grant",
);

unless ( $nac_filter_rule1 ) {
       die("Construct NAC Filter Rule failed: ",
               Infoblox::status_code() . ":" . Infoblox::status_detail() );
}
print "NAC Filter Rule object created successfully\n";

#Create the DHCP Range object with this NAC Filter Rule.
my $dhcp_range = Infoblox::DHCP::Range->new(
        end_addr   => "10.0.0.10",
        network    => "10.0.0.0/8",
        start_addr => "10.0.0.1",
        filters    => [ $nac_filter_rule1 ],
);
unless($dhcp_range) {
       die("Construct DHCP Range object failed: ",
            Infoblox::status_code() . ":" . Infoblox::status_detail());
}
print "DHCP Range object created successfully\n";

#Add the DHCP Range object through a session
$session->add($dhcp_range)
    or die("Add DHCP Range object failed: ",
            $session->status_code() . ":" . $session->status_detail());
print "DHCP Range object added to Infoblox appliance successfully\n";
```

#Get and modify a NAC Filter Rule object

```
#Get NAC Filter Rule object from the DHCP Range object through session
my @retrieved_objs = $session->get(
    object      => "Infoblox::DHCP::Range",
    start_addr  => "10.0.0.1",
);

my $object = $retrieved_objs[0];
my $filter = $object->filters();

my @filters = @{$filter};

my $nac_filterrule = $filters[0];

#Modify one of the attributes of the obtained NAC Filter Rule object
$nac_filterrule->permission("deny");
$object->filters([$nac_filterrule]);

#Apply the changes
$session->modify($object)
    or die("Modify NAC Filter Rule object failed: ",
           $session->status_code() . ":" . $session->status_detail());
print "NAC Filter Rule object modified and added to the DHCP Range object successfully \n";
```

#Remove a NAC Filter Rule object

```
$object->filters([]);

my $response = $session->modify($object);
unless($response) {
       die("Remove NAC Filter Rule failed: ",
               session->status_code() . ":" . session->status_detail() );
}
print "NAC Filter Rule object removed successfully \n";

#Submit the network object for removal
$session->remove($network)
    or die("Remove Network object failed: ",
        $session->status_code() . ":" . $session->status_detail());
print "Network object removed successfully \n";

#Submit the NAC Address Filter object for removal
$session->remove($nac_filter)
    or die("Remove NAC Filter object failed: ",
        $session->status_code() . ":" . $session->status_detail());
print "NAC Filter object removed successfully \n";

####PROGRAM ENDS####
```

AUTHOR

Infoblox Inc. http://www.infoblox.com/

SEE ALSO

Infoblox::DHCP::FilterRule::RelayAgent, Infoblox::DHCP::Network, Infoblox::DHCP::Range, Infoblox::Session->add(),Infoblox::Session->modify(), Infoblox::Session->remove(),Infoblox::Session

COPYRIGHT

Copyright (c) 2017 Infoblox Inc.

Infoblox::DHCP::FilterRule::Option - DHCP filter rule option object.

- NAME
- DESCRIPTION
- CONSTRUCTOR
- MODULE METHODS
  - Infoblox::DHCP::Range->filters( )
- METHODS
  - filter_name( )
  - permission( )
- SAMPLE CODE
- AUTHOR
- SEE ALSO
- COPYRIGHT

NAME

Infoblox::DHCP::FilterRule::Option - DHCP filter rule option object.

DESCRIPTION

A DHCP filter rule option object gives the ability to grant and/or deny a lease of an IP defined in DHCP Range.

CONSTRUCTOR

```
my $FilterRuleOption = Infoblox::DHCP::FilterRule::Option->new(
              filter_name   =>  $string,          #Required
              permission    =>  "grant" | "deny", #Required
          );
```

MODULE METHODS

The following functions are available to apply to a DHCP filter rule option object.

Infoblox::DHCP::Range->filters( )

Use this function to specify filters at the DHCP range level. See details for Infoblox::DHCP::Range objects

Example

```
#Construct DHCP filter rule option object
my $filter = Infoblox::DHCP::FilterRule::Option->new(
    filter_name => "filter1",
    permission  => "grant"
);

# Configure  filters
my $response = $range->filters([$res]);
```

METHODS

This section describes all the methods that can be used to configure and retrieve the attribute values of a DHCP filter rule option object.

filter_name( )

Use this method to set or retrieve the filter name of a DHCP filter rule option object.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

Specify filter name in string format.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
# Get filters from  range
my $resp = $range->filters();
# Modify filters
$range->filters([$res]);
```

permission( )

Use this method to set or retrieve the permission of a DHCP filter rule option object. When selecting Grant lease, you need to assign addresses from the address range to requesting hosts whose user class (as defined on their network adapter) matches an entry in the option filter rules. When selecting Deny lease, you need to refuse an address request from a host whose user class matches an entry in the option filter rules.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

The permission can be either "grant" or "deny". The default value is "grant" lease.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
# Get permission
 $res->permission();
#modify permission value
$res->permission("deny");
```

SAMPLE CODE

The following sample code demonstrates the different functions that can be applied to an object such as add, modify. Also, this sample includes error handling for the operations.

##Preparation prior to a DHCP filter rule option object insertion

```
#PROGRAM STARTS: Include all the modules that will be used
use strict;
use Infoblox;

#Create a session to the Infoblox appliance
my $session = Infoblox::Session->new(
    master   => "192.168.1.2",
    username => "admin",
    password => "infoblox"
);
unless ($session) {
   die("Construct session failed: ",
       Infoblox::status_code() . ":" . Infoblox::status_detail());
}
print "Session created successfully\n";

  #Create a DHCP Member object
   my $memberdhcp = Infoblox::DHCP::Member->new(
    name     => "infoblox.localdomain",
    ipv4addr => "192.168.1.2"
   );
   unless($memberdhcp) {
       die("Construct member failed: ",
            Infoblox::status_code() . ":" . Infoblox::status_detail());
   }
   print "DHCP Member object created successfully\n";

#Create the Network object with the member
my $network = Infoblox::DHCP::Network->new(
    network => "10.0.0.0/8",
    comment => "add network",
    members => [ $memberdhcp ]
);
unless($network) {
       die("Construct Network object failed: ",
            Infoblox::status_code() . ":" . Infoblox::status_detail());
}
```

```
print "Network object created successfully\n";
```

#Add the Network object into the Infoblox appliance through a session

```
$session->add($network)
    or die("Add Network object failed: ",
            $session->status_code() . ":" . $session->status_detail());
print "Network object added to Infoblox appliance successfully\n";

my $opt1 = Infoblox::DHCP::Option->new(
      name => "user-class" ,
      value => "77"
   );
```

#Create the Option Filter object

```
my $option_filter = Infoblox::DHCP::Filter::Option->new(
      name => 'filter1' ,
      option_list => [$opt1],

);

unless($option_filter) {
       die("Construct Option Filter object failed: ",
            Infoblox::status_code() . ":" . Infoblox::status_detail());
}
print "Option Filter object created successfully\n";
```

#Add the filter rule option object into the Infoblox appliance

```
$session->add($option_filter)
    or die("Add option Filter object failed: ",
            $session->status_code() . ":" . $session->status_detail());
print "Option Filter object added to Infoblox appliance successfully\n";
```

#Construct DHCP Filter rule Option object

```
my $filter = Infoblox::DHCP::FilterRule::Option->new(
    filter_name => "filter1",
    permission  => "grant"
);
```

#Create a DHCP Range object

```
my $dhcp_range = Infoblox::DHCP::Range->new(
        end_addr     => "10.0.0.10",
        network      => "10.0.0.0/8",
        start_addr   => "10.0.0.1",
        authority    => "true",
        comment      => "add range",
        filters      => [$filter],
        member       => $memberdhcp,
);
unless($dhcp_range) {
       die("Construct DHCP Range object failed: ",
            Infoblox::status_code() . ":" . Infoblox::status_detail());
}
print "DHCP Range object created successfully\n";

#Add the DHCP Range object into the Infoblox appliance through a session
$session->add($dhcp_range)
    or die("Add DHCP Range object failed: ",
            $session->status_code() . ":" . $session->status_detail());
print "DHCP Range object added to the network successfully\n";
```

#Get and modify the filter rule option object

```
#get  object from session
my @result = $session->get(
    object => "Infoblox::DHCP::Range",
    start_addr => "10.0.0.1"
);
unless (scalar(@result) == 0) {
    my $range = $result[0];

    if ($range) {
    my $resp = $range->filters();
```

```
    my @arr = @{$resp};
    my $res = $arr[0];

        #modify permission value
        $res->permission("deny");
        $range->filters([$res]);
        #update session
        $session->modify($range)
            or die("modify failed: ",
               $session->status_code(), $session->status_detail());

            print "Modify filter permisson successful\n";
                        }
                    } else {
             print "No range found.";
                        }
             $session->remove($option_filter);
             print "Remove filter1 successfull\n";

  ####PROGRAM ENDS####
```

AUTHOR

Infoblox Inc. http://www.infoblox.com/

SEE ALSO

Infoblox::Session, Infoblox::DHCP::Range, Infoblox::DHCP::Network, Infoblox::DHCP::Member, Infoblox::DHCP::Option, Infoblox::Session->get(), Infoblox::Session->add(),

COPYRIGHT

Copyright (c) 2017 Infoblox Inc.

Infoblox::DHCP::FilterRule::Fingerprint - DHCP Fingerprint Filter Rule object.

- NAME
- DESCRIPTION
- CONSTRUCTOR
- MODULE METHODS
  - Infoblox::DHCP::Range->filters( )
  - Infoblox::DHCP::RangeTemplate->filters( )
- METHODS
  - filter_name( )
  - permission( )
- AUTHOR
- SEE ALSO
- COPYRIGHT

NAME

Infoblox::DHCP::FilterRule::Fingerprint - DHCP Fingerprint Filter Rule object.

DESCRIPTION

DHCP Fingerprint Filter Rule object.

CONSTRUCTOR

```
my $filter_rule = Infoblox::DHCP::FilterRule::Fingerprint->new(
      filter_name    => $string,           #Required
      permission     => "grant" | "deny", #Required
);
```

MODULE METHODS

The following functions are available to apply to a DHCP Fingerprint Filter Rule object.

Infoblox::DHCP::Range->filters( )

Use this method to add a DHCP fingerprint Filter Rule object to the DHCP Range on the Infoblox appliance. See Infoblox::DHCP::Range->filters() for parameters and return values.

Example

```
#Construct a DHCP Fingerprint Filter Rule object
my $filter_rule = Infoblox::DHCP::FilterRule::Fingerprint->new(
      filter_name    => "filter",
      permission     => "grant",
);

#Add Fingerprint Filter Rule to the DHCP Range object.
my $response = $dhcp_range->filters([$filter_rule]);
```

Infoblox::DHCP::RangeTemplate->filters( )

Use this method to add a DHCP fingerprint Filter Rule object to the DHCP Range Template on the Infoblox appliance. See Infoblox::DHCP::RangeTemplate->filters() for parameters and return values.

Example

```
#Construct a DHCP Fingerprint Filter Rule object
my $filter_rule = Infoblox::DHCP::FilterRule::Fingerprint->new(
      filter_name    => "filter",
      permission     => "grant",
);

#Add Fingerprint Filter Rule to the DHCP Range Template object.
my $response = $dhcp_range_template->filters([$filter_rule]);
```

METHODS

This section describes all the methods that can be used to set and retrieve the attribute values of a DHCP Fingerprint Filter Rule object.

filter_name( )

Use this method to set or retrieve the filter_name of a DHCP Fingerprint Filter Rule object.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

Name of the DHCP Fingerprint filter. Maximum length up to 1024 bytes is supported.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get filter_name
my $filter_name = $filter_rule->filter_name();
#Modify filter_name
$filter_rule->filter_name("fingerprint filter");
```

permission( )

Use this method to set or retrieve the permission of a DHCP Fingerprint Filter Rule object.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

Specify the permission in string format. The permission can be either "grant" or "deny".

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get permission
my $permission = $filter_rule->permission();
#Modify permission
$filter_rule->permission("deny");
```

AUTHOR

Infoblox Inc. http://www.infoblox.com/

SEE ALSO

Infoblox::DHCP::Fingerprint, Infoblox::DHCP::Filter::Fingerprint, Infoblox::DHCP::Range, Infoblox::DHCP::RangeTemplate

---

COPYRIGHT

Copyright (c) 2017 Infoblox Inc.

Infoblox::DHCP::DDNS - DHCP DDNS object.

- NAME
- DESCRIPTION
- CONSTRUCTOR
- SESSION METHODS
- METHODS
  - dns_ext_zone( )
  - dns_grid_zone( )
  - view( )
  - external_primary( )
  - grid_primary( )
  - zone_match( )
- AUTHOR
- SEE ALSO
- COPYRIGHT

NAME

Infoblox::DHCP::DDNS - DHCP DDNS object.

DESCRIPTION

The object is used to configure Multi-Grid Master preferences for updates from DHCP to Multi-Grid Master zones.

CONSTRUCTOR

```
my $ddns = Infoblox::DHCP::DDNS->new(
    dns_ext_zone      => $string,           # required if zone_match is 'EXTERNAL'
    dns_grid_zone     => $string,           # required if zone_match is 'GRID'
    view              => $string,           # optional / The default value is the name of the default view
    external_primary  => $ipaddr,           # required if zone_match is 'EXTERNAL' or 'ANY_EXTERNAL'
    grid_primary      => $string,           # required if zone_match is 'GRID' or 'ANY_GRID'
    zone_match        => 'ANY_EXTERNAL' | 'ANY_GRID' | 'EXTERNAL' | 'GRID',  # required
);
```

SESSION METHODS

The object doesn't support any session methods.

METHODS

This section describes all the methods that you can use to configure and retrieve a DHCP DDNS object.

dns_ext_zone( )

Use this method to set or retrieve a zone name, in FQDN (Fully Qualified Domain Name) format, that is contained in an external member.

Parameter

A zone name in FQDN format.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
# Get the attribute value
my $value = $ddns->dns_ext_zone();
# Modify the attribute value
$ddns->dns_ext_zone('exmmdns.com');
```

dns_grid_zone( )

Use this method to set or retrieve a zone name, in FQDN (Fully Qualified Domain Name) format, that is contained in a Grid member.

Parameter

A zone name in FQDN format.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
# Get the attribute value
my $value = $ddns->dns_grid_zone();
# Modify the attribute value
$ddns->dns_grid_zone('mmdns.com');
```

view( )

Use this method to set or retrieve the name of the DNS view in which the zone is located.

Parameter

The name of the DNS view in string format. The default value is the name of the default view.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
# Get the attribute value
my $value = $ddns->view();
# Modify the attribute value
$ddns->view('dnsview');
```

external_primary( )

Use this method to set or retrieve the IP address of an external member.

Parameter

The IP address of an external member.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
# Get the attribute value
my $value = $ddns->external_primary();
# Modify the attribute value
$ddns->external_primary('10.0.0.1');
```

grid_primary( )

Use this method to set or retrieve the name of the primary Grid member.

Parameter

The name of the primary Grid member.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
# Get the attribute value
my $value = $ddns->grid_primary();
# Modify the attribute value
$ddns->grid_primary('grid.com');
```

zone_match( )

Use this method to set or retrieve the matching type.

Parameter

The valid value is 'ANY_EXTERNAL', 'ANY_GRID', 'EXTERNAL' or 'GRID'.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
# Get the attribute value
my $value = $ddns->zone_match();
# Modify the attribute value
$ddns->zone_match('GRID');
```

AUTHOR

Infoblox Inc. http://www.infoblox.com/

SEE ALSO

Infoblox::DHCP::View, Infoblox::Grid::Member::DHCP

COPYRIGHT

Copyright (c) 2017 Infoblox Inc.

Infoblox::DNS::Nameserver - DNS Nameserver object

- NAME
- DESCRIPTION
- CONSTRUCTOR
- MODULE METHODS
  - Infoblox::DHCP::View->external_ddns_primaries( )
  - Infoblox::DNS::Zone->delegate_to( )
  - Infoblox::DNS::Zone->forward_to( )
  - Infoblox::DNS::Zone->secondaries( )
  - Infoblox::DNS::Zone->multiple_primaries( )
  - Infoblox::Grid::Member->prefer_resolver( )
  - Infoblox::Grid::Member->alternate_resolver( )
  - Infoblox::Grid::DNS::Nsgroup->multiple_primaries( )
  - Infoblox::Grid::DNS::Nsgroup->secondaries( )
- METHODS
  - ddns_zone( )
  - gss_tsig_domain( )
  - gss_tsig_dns_principal( )
  - ipv4addr( )
  - ipv6addr( )
  - ms_parent_delegated( )
  - name( )
  - stealth( )
  - TSIGalgorithm( )
  - TSIGkey( )
  - TSIGname( )
- SAMPLE CODE
- AUTHOR
- SEE ALSO
- COPYRIGHT

NAME

Infoblox::DNS::Nameserver - DNS Nameserver object

DESCRIPTION

Manipulates a Nameserver object that contains information about the domain name space.

CONSTRUCTOR

```
 my $nameserver = Infoblox::DNS::Nameserver->new(
     ipv4addr  => $ipv4addr,            #Required if $ipv6addr is not present
     ipv6addr  => $ipv6addr,            #Required if $ipv4addr is not present
     name      => $fqdn,                #Required if this name server is not used to represent an external
DDNS server
     ddns_zone => $string,              #Required if this name server is used to represent an external DDNS
server
     gss_tsig_domain  => $string,       #Optional / Default is undefined
     gss_tsig_dns_principal => $string, #Optional / Default is undefined
     stealth   => "true" | "false",     #Optional / Default is "false"
     TSIGkey   => $string | ":2xCOMPAT" #Optional / Default is undefined
     TSIGname  => $string,              #Optional / Default is undefined
     TSIGalgorithm => "HMAC-MD5" | "HMAC-SHA256" ,  #Optional / Default is "HMAC-MD5"
 );
```

MODULE METHODS

The following functions are available to apply to a nameserver object.

Infoblox::DHCP::View->external_ddns_primaries( )

Use this function to specify a nameserver to send DDNS updates to at the network view level.

Example

```
      #Construct an object
        my $nameserver = Infoblox::DNS::Nameserver->new(
          ipv4addr  => "1.2.3.4",
```

```
    TSIGname  => "tsig_name",
    TSIGkey   => "ZX1jVJl7C58GT/sc7Q3uca==",
    ddns_zone => "zone1"
);

#Construct an object
  my $nameserver1 = Infoblox::DNS::Nameserver->new(
    ipv4addr  => "1.2.3.5",
    TSIGname  => "tsig_name",
    TSIGkey   => "ZX1jVJl7C58GT/sc7Q3uca==",
    ddns_zone => "zone2"
);

# Configure Nameserver
my $response = $View->external_ddns_primaries([$nameserver,$nameserver1]);
```

Infoblox::DNS::Zone->delegate_to( )

Use this function to specify a nameserver at the zone level.

Example

```
#Construct an object
  my $nameserver = Infoblox::DNS::Nameserver->new(
    name      => "ns1.test.com",
    ipv4addr  => "4.4.7.7",
);

# Configure Nameserver
my $response = $Zone->delegate_to([$nameserver]);
```

Infoblox::DNS::Zone->forward_to( )

Use this function to specify a nameserver at the zone level.

Example

```
#Construct an object
  my $nameserver = Infoblox::DNS::Nameserver->new(
    name      => "ns2.test.com",
    ipv4addr  => "4.4.6.6",
);

# Configure Nameserver
my $response = $Zone->forward_to([$nameserver]);
```

Infoblox::DNS::Zone->secondaries( )

Use this function to specify a nameserver at the zone level.

Example

```
#Construct an object
  my $nameserver = Infoblox::DNS::Nameserver->new(
    name      => "ns3.test.com",
    ipv4addr  => "4.5.6.4",
    stealth   => "false",
    TSIGname  => "tsig_name",
    TSIGkey   => "ZX1jVJl7C58GT/sc7Q3uca==",
);

#Construct an object
  my $nameserver1 = Infoblox::DNS::Nameserver->new(
    name      => "ns4.test.com",
    ipv6addr  => "2009::546",
    stealth   => "false",
    TSIGname  => "tsig_name",
    TSIGkey   => "ZX1jVJl7C58GT/sc7Q3uca==",
);

# Configure Nameserver
my $response = $Zone->secondaries([$nameserver,$nameserver1]);
```

Infoblox::DNS::Zone->multiple_primaries( )

Use this function to specify nameservers at the zone level.

Example

```
#Construct an object
  my $nameserver = Infoblox::DNS::Nameserver->new(
    name      => "ns4.test.com",
    ipv4addr  => "4.4.5.6",
    stealth   => "false",
    TSIGname  => "tsig_name",
    TSIGkey   => "ZX1jVJl7C58GT/sc7Q3uca==",
);

#Construct an object
  my $nameserver1 = Infoblox::DNS::Nameserver->new(
    name      => "nsv6.test.com",
    ipv6addr  => "2004::223",
    stealth   => "false",
    TSIGname  => "tsig_name",
    TSIGkey   => "ZX1jVJl7C58GT/sc7Q3uca==",
);

# Configure Nameserver
my $response = $Zone->multiple_primaries([$nameserver,$nameserver1]);
```

Infoblox::Grid::Member->prefer_resolver( )

Use this function to specify a nameserver at the grid member level.

Example

```
#Construct an object
  my $nameserver = Infoblox::DNS::Nameserver->new(
    ipv4addr  => "10.7.5.6",
);

# Configure Nameserver
my $response = $grid_member_object->prefer_resolver([$nameserver]);
```

Infoblox::Grid::Member->alternate_resolver( )

Use this function to specify a nameserver at the grid member level.

Example

```
#Construct an object
  my $nameserver = Infoblox::DNS::Nameserver->new(
    ipv4addr  => "10.10.5.6",
);

# Configure Nameserver
my $response = $grid_member_object->alternate_resolver([$nameserver]);
```

Infoblox::Grid::DNS::Nsgroup->multiple_primaries( )

Use this function to specify nameservers at the grid DNS level.

Example

```
#Construct an object
  my $nameserver = Infoblox::DNS::Nameserver->new(
    name      => "ns5.test.com",
    ipv4addr  => "10.10.5.6",
    stealth   => "false",
    TSIGname  => "tsig_name",
    TSIGkey   => "Zx1JVjl7C58gT/sc7Q3uca==",
);

# Configure Nameserver
my $response = $grid_member_object->multiple_primaries([$nameserver]);
```

Infoblox::Grid::DNS::Nsgroup->secondaries( )

Use this function to specify a nameserver at the grid DNS level.

Example

```
#Construct an object
  my $nameserver = Infoblox::DNS::Nameserver->new(
```

```
        name      => "ns5.test.com",
        ipv4addr  => "10.10.5.6",
        stealth   => "false",
        TSIGname  => "tsig_name",
        TSIGkey   => "Zx1JVjl7C58gT/sc7Q3uca==",
    );

    # Configure Nameserver
    my $response = $grid_member_object->secondaries([$nameserver]);
```

METHODS

This section describes all the methods that you can use to configure and retrieve the attribute values of a Nameserver object.

ddns_zone( )

Use this method to set or retrieve the zone name served by the name server.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

Desired ddns_zone for the name server in string format.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
    # Getting ddns_zone
    my $ddns_zone = $nameserver->ddns_zone();
    # Modifying ddns_zone
    $nameserver->ddns_zone("zone2");
```

gss_tsig_domain( )

Use this method to set or retrieve the domain in which GSS-TSIG for dynamic updates is enabled.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

Desired domain name in string format with a maximum of 1023 bytes.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
    # Getting gss_tsig_domain
    my $gss_tsig_domain = $nameserver->gss_tsig_domain();
    # Modifying gss_tsig_domain
    $nameserver->gss_tsig_domain("mydomain");
```

gss_tsig_dns_principal( )

Use this method to set or retrieve the principal name in which GSS-TSIG for dynamic updates is enabled.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

Desired principal name in string format with a maximum of 1023 bytes.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
# Getting gss_tsig_dns_principal
my $gss_tsig_dns_principal = $nameserver->gss_tsig_dns_principal();
# Modifying gss_tsig_dns_principal
$nameserver->gss_tsig_dns_principal("local1/a.localhost@LOCAL");
```

ipv4addr( )

Use this method to set or retrieve the IPv4 address. Required only when ipv6 is not specified.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

An IPv4 address is a 32-bit number in dotted decimal notation. It consists of four 8-bit groups of decimal digits separated by decimal points (example: 192.168.1.2).

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
# Getting ipv4addr
my $ipv4addr = $nameserver->ipv4addr();
# Modifying ipv4addr
$nameserver->ipv4addr("10.0.0.3");
```

ipv6addr( )

Use this method to set or retrieve the IPv6 address. Required only when ipv4 is not specified.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

An IPv6 address is a 128-bit number in colon hexadecimal notation. It consists of eight groups of four hexadecimal digits separated by colons (example: 12ab:0000:0000:0123:4567:89ab:0000:cdef).

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
# Getting ipv6addr
my $ipv6addr = $nameserver->ipv6addr();
# Modifying ipv6addr
$nameserver->ipv6addr("12ab::345");
```

ms_parent_delegated( )

Use this method to retrieve the ms_parent_delegated flag. If the zone is a delegation and the primary server of the parent zone is a Microsoft (r) server, this flag is set to "true" if the FQDN and IP address in the delegation zone's NS record matches the FQDN and IP address of its authoritative name server.

Parameter

None.

Returns

The method returns the attribute value.

Example

```
#Get ms_parent_delegated
my $ms_parent_delegated = $member->ms_parent_delegated();
```

name( )

Use this method to set or retrieve a resolvable domain name for the server.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

Desired name for the name server in FQDN format.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails. If you did not specify a parameter, the method returns the attribute value.

Example

```
# Getting name
my $name = $nameserver->name();
# Modifying name
$nameserver->name("ns2.domain.com");
```

stealth( )

Use this method to make the name server stealth or not. When the attribute stealth is "true", the name server is in stealth mode. This means that its NS record is not published among the zone data, and it does not respond to queries from resolvers and other name servers.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

Valid values are "true" or "false".

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
# Getting stealth
my $stealth = $nameserver->stealth();
# Modifying stealth
$nameserver->stealth("true");
```

TSIGalgorithm( )

Use this method to set or retrieve the algorithm of the TSIG key you want to use.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

Desired algorithm of the TSIG key. Valid values are "HMAC-SHA256" and "HMAC-MD5", which is the default.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
# Getting TSIGalgorithm
my $TSIGalgorithm = $nameserver->TSIGalgorithm();
# Modifying TSIGalgorithm
$nameserver->TSIGalgorithm("HMAC-SHA256");
```

TSIGkey( )

Use this method to set or retrieve the TSIG key. This key must also be present on the external primary server. You can generate a TSIG key, or you can obtain the TSIG key name and key from the external name server either by accessing the appliance yourself or by requesting the appliance administrator to deliver them to you through some out-of-band mechanism. Then type or copy-and-paste that name and key into the appropriate fields.

OR

Use DNS One 2.x TSIG: If you want to use TSIG authentication and the external primary name server is an Infoblox appliance running DNS One 2.x code. The local appliance generates the required TSIG key for authenticating DNS messages to and from appliances running DNS One 2.x code.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

Desired TSIGkey of the name server in string format. Valid values are ":2xCOMPAT" or a TSIG key string.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
# Getting TSIGkey
my $TSIGkey = $nameserver->TSIGkey();
# Modifying TSIGname
$nameserver->TSIGkey("ZX1jVJl7C58GT/sc7Q3uc2==");
```

TSIGname( )

Use this method to set or retrieve the name of the TSIG key you want to use.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

Desired name of the TSIG key in string format.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
# Getting TSIGname
my $TSIGname = $nameserver->TSIGname();
# Modifying TSIGname
$nameserver->TSIGname("tsig2_name");
```

SAMPLE CODE

The following sample code demonstrates the different functions that can be applied to an object such as add, modify. This sample also includes error handling for the operations.

##Preparation prior to a DNS nameserver object insertion

```
#PROGRAM STARTS: Include all the modules that will be used
use strict;
use Infoblox;

#Create a session to the Infoblox appliance
my $session = Infoblox::Session->new(
    master   => "192.168.1.2",
    username => "admin",
    password => "infoblox"
);
unless ($session) {
   die("Construct session failed: ",
       Infoblox::status_code() . ":" . Infoblox::status_detail());
}
print "Session created successfully\n";
```

#Create a nameserver object

```
 my $nameserver1 = Infoblox::DNS::Nameserver->new(
    name     => "ns1.domain.com",
    ipv4addr => "10.4.5.6",
    stealth  => "false",
    TSIGname => "tsig_name",
    TSIGkey  => "ZX1jVJl7C58GT/sc7Q3ucA==",
 );
 unless ($nameserver1){
     die("Construct Nameserver failed: ",
           Infoblox::status_code() . ":" . Infoblox::status_detail());
  }
```

```
print "Nameserver object created successfully\n";
```

 # Create a zone and use $nameserver1

```
#create a zone and use $nameserver1
my $zone = Infoblox::DNS::Zone->new(
   name    => "domain.com",
   multiple_primaries => [$nameserver1],
 );

#put zone into session
$session->add($zone)
  or die("Zone creation failed: ",
           Infoblox::status_code() . ":" . Infoblox::status_detail());

print " Zone creation successful\n";
```

#Get and modify the name server

```
#get Zone object from session
my @result = $session->get(
    object => "Infoblox::DNS::Zone",
    name   => "domain.com"
);

unless (scalar(@result) == 0) {
    my $zone = $result[0];

    if ($zone) {

        #modify ipv4addr value
        $nameserver1->ipv4addr("5.5.5.5");
        $zone->multiple_primaries([$nameserver1]);

        #update session
        $session->modify($zone)
            or die("modify zone domain.com failed: ",
               $session->status_code(), $session->status_detail());
           }
     } else {
         print "No zone found.";
     }
    ####PROGRAM ENDS####
```

AUTHOR

Infoblox Inc. http://www.infoblox.com/

SEE ALSO

Infoblox::Session, Infoblox::Session->add(), Infoblox::Session->get(), Infoblox::Session->modify(),Infoblox::DHCP::View,Infoblox::DNS::Zone, Infoblox::Grid::DHCP, Infoblox::Grid::Member, Infoblox::Grid::DNS::Nsgroup

---

COPYRIGHT

Copyright (c) 2017 Infoblox Inc.

Infoblox::Grid::Member::Discovery::VRF - Object represents the relation between network view and the virtual network membership.

- NAME
- DESCRIPTION
- SESSION METHODS
  - Infoblox::Session->get()
  - Infoblox::Session->search()
- METHODS
  - name()
  - network_view()
  - route_distinguisher()
  - description()
  - device()
- AUTHOR
- SEE ALSO
- COPYRIGHT

NAME

Infoblox::Grid::Member::Discovery::VRF - Object represents the relation between network view and the virtual network membership.

DESCRIPTION

This object is used to represents relation between network view and the virtual network membership. Object is read-only.

SESSION METHODS

Infoblox::Session->get()

Use this method to retrieve all the matching virtual network objects from the Infoblox appliance. See Infoblox::Session->get() for parameters and return values.

Key References

Apply the following attributes to get a specific virtual network objects:

```
name                - Optional. A name in string format.
description         - Optional. A description in string format.
route_distinguisher - Optional. Route distinguisher associated with the VRF in sting format.
network_view        - Optional. Name of the network view associated with the VRF in sting format.
```

Returns

Returns Virtual network membership object.

Example

```
#Get VRF object with the name 'VRF1'
my $vrf_obj = $session->get(
    object  => "Infoblox::Grid::Member::Discovery::VRF" ,
    name    => 'VRF1'
);
```

Infoblox::Session->search()

Use this method to search for a virtual network objects in the Infoblox appliance. See Infoblox::Session->search() for parameters and return values.

Key References

```
Apply the following attributes to search for a virtual network object:

name                - Optional. A name in string format (regular expression).
description         - Optional. A description in string format (regular expression).
route_distinguisher - Optional. Route distinguisher associated with the VRF in sting format.
network_view        - Optional. Name of the network view associated with the VRF in sting format.
```

Returns

Returns Virtual network membership object(s).

Example

```
# Get all VRFs
my @all_vrfs = $session->search(
      object => 'Infoblox::Grid::Member::Discovery::VRF',
      name => '^.*$'
);

# Get vrf by Route distinguisher
my $vrf = $session->search(
      object => 'Infoblox::Grid::Member::Discovery::VRF',
      route_distinguisher => '123:500'
);
```

METHODS

This section describes all the methods that can be used to retrieve values from VRF object.

`name()`

Use this method to retrieve name associated with the VRF. This is a read-only attribute.

Parameter

None

Returns

The method returns the attribute value.

Example

```
#Get VRF name
my $vrf_name = $vrf->name();
```

`network_view()`

Use this method to retrieve name of network view associated with the VRF. This is a read-only attribute.

Parameter

None

Returns

The returned string contains name network view. This is a read-only attribute.

Example

```
#Get VRF network view name
my $vrf_nv = $vrf->network_view();
```

`route_distinguisher()`

Use this method to retrieve route distinguisher associated with the VRF. This is a read-only attribute.

Parameter

None

Returns

The method returns the attribute value.

Example

```
#Get VRF route distinguisher
my $vrf_rd = $vrf->route_distinguisher();
```

`description()`

Use this method to retrieve description associated with the VRF. This is a read-only attribute.

Parameter

None

Returns

The method returns the attribute value.

Example

```
#Get VRF description
my $vrf_desc = $vrf->description();
```

device()

Use this method to retrieve Infoblox::Grid::Discovery::Device object from which this VRF membership configuration was collected. This is a read-only attribute.

Parameter

None

Returns

The valid return value is an Infoblox::Grid::Discovery::Device partial object.

Example

```
#Get VRF device
my $vrf_device = $vrf->device();
```

AUTHOR

Infoblox Inc. http://www.infoblox.com/

SEE ALSO

Infoblox::Grid::Discovery::Device Infoblox::Session, Infoblox::Session->get(), Infoblox::Session->search()

COPYRIGHT

Copyright (c) 2017 Infoblox Inc.

Infoblox::DNS::View - DNS View object

- NAME
- DESCRIPTION
- CONSTRUCTOR
- SESSION METHODS
  - Infoblox::Session->add( )
  - Infoblox::Session->get( )
  - Infoblox::Session->modify( )
  - Infoblox::Session->remove( )
  - Infoblox::Session->search( )
- METHODS
  - blacklist_action( )
  - blacklist_log_query( )
  - blacklist_redirect_addresses( )
  - blacklist_redirect_ttl( )
  - blacklist_rulesets( )
  - cloud_info( )
  - comment( )
  - custom_root_name_servers( )
  - ddns_force_creation_timestamp_update( )
  - ddns_restrict_patterns_list( )
  - ddns_restrict_patterns( )
  - ddns_restrict_static( )
  - ddns_restrict_protected( )
  - ddns_restrict_secure( )
  - ddns_principal_tracking( )
  - ddns_principal_group( )
  - disable( )
  - dns64_groups( )
  - dnssec_enabled( )
  - dnssec_expired_signatures_enabled( )
  - dnssec_validation_enabled( )
  - dnssec_blacklist_enabled()
  - dnssec_negative_trust_anchors( )
  - dnssec_dns64_enabled()
  - dnssec_nxdomain_enabled()
  - dnssec_rpz_enabled()
  - dnssec_trusted_keys( )
  - enable_blacklist( )
  - enable_dns64( )
  - enable_match_recursive_only( )
  - enable_fixed_rrset_order_fqdns( )
  - extattrs( )
  - extensible_attributes( )
  - filter_aaaa( )
  - filter_aaaa_list( )
  - fixed_rrset_order_fqdns( )
  - forwarders( )
  - forward_only( )
  - match_clients( )
  - match_destinations( )
  - max_cache_ttl( )
  - max_ncache_ttl( )
  - lame_ttl( )
  - name( )
  - network_view( )
  - notify_delay( )
  - override_blacklist( )
  - override_ddns_force_creation_timestamp_update( )
  - override_ddns_patterns_restriction( )
  - override_ddns_restrict_static( )
  - override_ddns_restrict_protected( )
  - override_ddns_principal_security( )
  - override_dns64( )
  - override_dnssec( )
  - override_filter_aaaa( )
  - override_fixed_rrset_order_fqdns( )
  - override_forwarders( )
  - override_lame_ttl( )
  - override_max_cache_ttl( )
  - override_max_ncache_ttl( )
  - scavenging_settings( )

- override_scavenging_settings( )
- recursion( )
- rpz_drop_ip_rule_enabled( )
- rpz_drop_ip_rule_min_prefix_length_ipv4( )
- rpz_drop_ip_rule_min_prefix_length_ipv6( )
- rpz_qname_wait_recurse( )
- response_rate_limiting( )
- use_root_name_servers( )
- override_nxdomain_redirect( )
- nxdomain_redirect( )
- nxdomain_redirect_addresses( )
- nxdomain_redirect_addresses_ipv6( )
- nxdomain_redirect_ttl( )
- nxdomain_log_query( )
- nxdomain_rulesets( )
- override_rpz_drop_ip_rule( )
- override_rpz_qname_wait_recurse( )
- override_response_rate_limiting( )
- override_sortlist( )
- sortlist( )

- SAMPLE CODE
- AUTHOR
- SEE ALSO
- COPYRIGHT

NAME

Infoblox::DNS::View - DNS View object

DESCRIPTION

DNS views provide the ability to serve one version of DNS data to one set of clients and another version to another set of clients. With DNS views, the Infoblox appliance can provide a different answer to the same DNS query, depending on the source of the query. When the DNS view is not specified, all objects are automatically associated with the default DNS view.

CONSTRUCTOR

```
 my $view = Infoblox::DNS::View->new(
     name                      => $string,
#Required
     blacklist_action          => "REDIRECT" | "REFUSE",
#Optional / Default is "REFUSE"
     blacklist_log_query       => "true" | "false",
#Optional / Default is "false"
     blacklist_redirect_addresses => [$ipv4addr, ... ],
#Optional / Default is undefined
     blacklist_redirect_ttl    => $num,
#Optional / Default is 60
     blacklist_rulesets        => [ $rulesetname1, $rulesetname2, ...],
#Optional / Default is undefined
     comment                   => $string,
#Optional / Default is empty
     custom_root_name_servers => [$RootNameServer1, $RootNameServer2, ...],
#Optional / Default is undefined
     disable                   => "true" | "false",
#Optional / Default "false"
     dns64_groups              => [$group1, $group2, ... ],
#Optional / Default is undefined
     dnssec_enabled            => "true" | "false" | undef,
#Optional / Default is undefined
     dnssec_expired_signatures_enabled => "true" | "false",
#Optional / Default is "false"
     dnssec_validation_enabled         => "true" | "false",
#Optional / Default is "true"
     dnssec_blacklist_enabled => "true" | "false",
#Optional / Default is "false"
     dnssec_negative_trust_anchors => $strig_array
#Optional / Default value is undefined
     dnssec_dns64_enabled      => "true" | "false",
#Optional / Default is "false"
     dnssec_nxdomain_enabled   => "true" | "false",
#Optional / Default is "false"
     dnssec_rpz_enabled        => "true" | "false",
#Optional / Default is "false"
     dnssec_trusted_keys               => [$key1, $key2, ...]
#Optional / Default is undefined
```

```
     enable_blacklist          => "true" | "false" | undef,
#Optional / Default is undefined
     enable_dns64              => "true" | "false" | undef,
#Optional / Default is undefined
     enable_match_recursive_only => "true" | "false"
#Optional / Default is "false"
     enable_fixed_rrset_order_fqdns => "true" | "false" | undef,
#Optional / Default is undefined
     extattrs                  => { $string => $extattr, ... },
#Optional / Default is undefined
     extensible_attributes => { $string => $string|$num, $string => [ $string|$num, ...], ... },
#Optional / Default is undefined
     filter_aaaa               => "YES" | "NO" | "BREAK_DNSSEC",
#Optional / Default is "NO"
     filter_aaaa_list          => [$ipv4addr | $ipv4_subnet/prefix | ...] | $nacl,
#Optional / Default is undefined
     fixed_rrset_order_fqdns   => [$fqdn1, $fqdn2],
#Optional / Default is undefined
     forwarders                => [ $ipv4addr, ...],
#Optional / Default is undefined
     forward_only              => "true" | "false" | undef,
#Optional / Default is undefined
     match_clients             => [$ipv4addr | $ipv4_subnet/prefix | $ipv6addr |
                                   $ipv6_subnet/prefix | "any" | $TSIGKey ...] | $nacl,
#Optional / Default is undefined
     match_destinations        => [$ipv4addr | $ipv4_subnet/prefix | $ipv6addr |
                                   $ipv6_subnet/prefix | "any" | $TSIGKey ...] | $nacl,
#Optional / Default is undefined
     max_cache_ttl             => $uint,
#Optional / Default is 604800 seconds
     max_ncache_ttl            => $uint,
#Optional / Default is 10800 seconds
     network_view              => $Object,
#Optional / Default is "default" network view
     notify_delay              => $num,
#Optional / Default is undefined
     override_dnssec           => "true"| "false",
#Optional / Default is "false"
     override_blacklist        => "true" | "false",
#Optional / Default is "false"
     override_dns64            => "true" | "false",
#Optional / Default is "false"
     override_filter_aaaa      => "true" | "false",
#Optional / Default is "false"
     override_fixed_rrset_order_fqdns => "true", "false",
#Optional / Default is "false"
     override_forwarders       => "true" | "false",
#Optional / Default is "false"
     override_max_cache_ttl    => "true" | "false",
#Optional / Default is "false"
     override_max_ncache_ttl   => "true" | "false",
#Optional / Default is "false"
     override_nxdomain_redirect => "true"| "false",
#Optional / Default is "false"
     override_rpz_qname_wait_recurse => "true" | "false",
#Optional / Default is "false"
     override_sortlist         => "true"| "false",
#Optional / Default is "false"
     recursion                 => "true" | "false",
#Optional / Default "false"
     rpz_qname_wait_recurse    => "true" | "false",
#Optional / Default is undefined
     scavenging_settings      => $scavenging_settings,
#Optional
     sortlist                  => [ $sortlist1, $sortlist2, ...],
#Optional / Default is undefined
     use_root_name_servers     => "true" | "false",
#Optional / Default "false"
     nxdomain_redirect         => "true" | "false",
#Optional / Default is "false"
     nxdomain_redirect_addresses => [$ipv4addr, ...],
#Optional / Default is undefined
     nxdomain_redirect_addresses_ipv6 => [$ipv6addr, ...],
#Optional / Default is undefined
     nxdomain_redirect_ttl     => $num
#Optional / Default is 60
     nxdomain_log_query        => "true" | "false",
#Optional / Default is "false"
     nxdomain_rulesets         => [$reulesetname1, $reulesetname2, ...]
#Optional / Default is undefined
     lame_ttl                  => $uint,
#Optional / Default is 600
     override_lame_ttl         => "true" | "false",
#Optional / Default is "false"
```

```
     response_rate_limiting   => $response_rate_limiting
#Optional / Default is undefined
     ddns_restrict_patterns_list        => [$string, $string, ...],
#Optional / Default is undefined
     ddns_restrict_patterns             => "true" | "false",
#Optional / Default is 'false'
     ddns_restrict_static               => "true" | "false",
#Optional / Default is 'false'
     ddns_restrict_protected            => "true" | "false",
#Optional / Default is 'false'
     ddns_restrict_secure               => "true" | "false",
#Optional / Default is 'false'
     ddns_principal_tracking            => "true" | "false",
#Optional / Default is 'false'
     ddns_principal_group               => $principal_group,
#Optional / Default is undefined
     ddns_force_creation_timestamp_update     => "true" | "false",
#Optional / Default is 'false'
     override_ddns_patterns_restriction => "true" | "false",
#Optional / Default is 'false'
     override_ddns_restrict_static      => "true" | "false",
#Optional / Default is 'false'
     override_ddns_restrict_protected   => "true" | "false",
#Optional / Default is 'false'
     override_ddns_principal_security   => "true" | "false",
#Optional / Default is 'false'
     override_ddns_force_creation_timestamp_update => "true" | "false",
#Optional / Default is 'false'
     override_scavenging_settings                  => "true" | "false",
#Optional / Default is 'false'
     override_rpz_drop_ip_rule                     => "true" | "false" | undef,
#Optional / Default is "false"
     rpz_drop_ip_rule_enabled                      => "true" | "false" | undef,
#Optional / Default is "false"
     rpz_drop_ip_rule_min_prefix_length_ipv4       => $uint | undef,
#Optional / Default is 29
     rpz_drop_ip_rule_min_prefix_length_ipv6       => $uint | undef,
#Optional / Default is 112
 );
```

You cannot set both extattrs and extensible_attributes attributes at the same time.

SESSION METHODS

This section describes all the methods in Infoblox::Session that you can apply to a DNS view object.

Infoblox::Session->add( )

Use this method to add a DNS view object to the Infoblox appliance. See Infoblox::Session->add() for parameters and return values.

If you have applied an access control list to an object, you must restart services after you update the access control list for the changes to take effect. See Infoblox::Session->restart() method.

Example

```
#create member
my $member1 = Infoblox::DNS::Member->new(
    name     => "member.com",
    ipv4addr => "192.168.1.2",
    lead     => "false",
    stealth  => "false"
);

#create keys
my $key1 = Infoblox::DNS::TSIGKey->new(
    name => "tsig_xfer",
    key  => ":2xCOMPAT"
);

my $tsig_key2 = $session->gen_tsig_key();

my $key2 = Infoblox::DNS::TSIGKey->new(
    name => "key2",
    key  => $tsig_key2
);

#create view object using members
my $firstview = Infoblox::DNS::View->new(
    name              => "internal",
```

```
    match_clients     => ["1.1.1.1", $key1 , "10.0.0.0/8", $key2],
    recursion         => "true",
    disable           => "false",
    comment           => "new Internal view"
    )
    or die(qq(constructor for view internal failed: ),
    join(":", Infoblox::status_code(), Infoblox::status_detail()));

#Add a  View through session
$session->add($firstview);
unless ($firstview) {
         die("add view internal failed: ",
              Infoblox::status_code() . ":" . Infoblox::status_detail());
}
```

Infoblox::Session->get( )

Use this method to retrieve an existing DNS view object from the Infoblox appliance. See Infoblox::Session->get() for parameters and return values.

Key Reference

Apply the following attributes to get a specific DNS view object:

```
name - Optional. A name in string format.
extattrs     - Optional. A hash reference containing extensible attributes.
extensible_attributes - Optional. A hash reference containing extensible attributes.
```

Example

```
#get view object from session
my @result = $session->get(
    object => "Infoblox::DNS::View",
    name   => "internal"
);

my $firstview = $result[0];

# get all DNS view objects with a given extensible attribute
my @retrieved_objs = $session->get(
    object => "Infoblox::DNS::View",
    extensible_attributes => { 'Site' => 'Santa Clara' });
```

Infoblox::Session->modify( )

Use this method to modify a DNS view object in the Infoblox appliance. See Infoblox::Session->modify() for parameters and return values.

If you have applied an access control list to an object, you must restart services after you update the access control list for the changes to take effect. See Infoblox::Session->restart() method.

Example

```
# Use method to modify the comment for view object.
$firstview->comment("new comment internal view");
# Submit modification
my $response = $session->modify($firstview);
```

Infoblox::Session->remove( )

Use this function to submit to remove a DNS view object from the Infoblox appliance. See Infoblox::Session->remove() for parameters and return values.

To remove a specific object, first use `get()` or `search()` to retrieve the specific object, and then submit this object for removal.

Example

#Remove an object with a unique name

```
my @result = $session->search(object => "Infoblox::DNS::View", name => "^internal\$");
my $firstview = $result[0];
#Submit for removal
my $response = $session->remove($firstview);
```

Infoblox::Session->search( )

Use this method to search for an existing DNS view object in the Infoblox appliance. See Infoblox::Session->search() for parameters and

return values.

Key Reference

Apply the following attributes to search for a DNS view object:

```
name    - Optional. A name in string format (regular expression).
extattrs     - Optional. A hash reference containing extensible attributes.
extensible_attributes - Optional. A hash reference containing extensible attributes.
```

For more information about searching extensible attributes, see Infoblox::Grid::ExtensibleAttributeDef/Searching Extensible Attributes.

Example

```
#Search for a DNS view by name
my @result = $session->search(object => "Infoblox::DNS::View", name => "^internal\$");

# search for DNS views with the extensible attribute 'Site'
my @retrieved_objs = $session->search(
   object => "Infoblox::DNS::View",
   extensible_attributes => { 'Site' => 'Santa Clara' });
```

METHODS

This section describes all the methods that you can use to configure and retrieve the attribute value of a DNS view object.

blacklist_action( )

Use this method to specify the action to perform when a domain name matches the pattern defined in a rule that is specified by the blacklist_ruleset method.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

Valid values are "REDIRECT" or "REFUSE". The default value is "REFUSE".

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

Examples

```
#Get blacklist_action
my $blacklist_action = $firstview->blacklist_action();
#Modify blacklist_action
$firstview->blacklist_action("REDIRECT");
```

blacklist_log_query( )

Use this method to set or retrieve the flag that indicates whether blacklist redirection queries are logged.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

Specify "true" to enable logging, or "false" to disable it. The default value is "false".

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get blacklist_log_query
my $blacklist_log_query = $firstview->nxdomain_log_query( );
#Modify blacklist_log_query
$firstview->blacklist_log_query("true");
```

blacklist_redirect_addresses( )

Use this method to specify the IP addresses the appliance includes in the response it sends in place of a blacklisted IP address.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

The valid value is an array of IPv4 addresses. The default value is undefined.

Setting this parameter to ["10.0.0.10", "10.0.0.20"] or [], implicitly sets the override_blacklist_redirect attribute to "true". Setting this parameter to undefined causes the appliance to use the member-level blacklist redirection settings and automatically sets the override_blacklist_redirect attribute to "false" if "enable_blacklist_redirect" is also undefined. But if "enable_blacklist_redirect" contains a value, then the override_blacklist_redirect attribute is set to "true".

Note that when blacklist_redirect_addresses contains a valid array reference and override_blacklist_redirect is set to "false", the last operation takes precedence. Thus the sequence $firstview->blacklist_redirect_addresses(["10.0.0.10", "10.0.0.20"]); $firstview->override_blacklist_redirect("false"); sets override_blacklist_redirect to "false", and the sequence $firstview->override_blacklist_redirect("false"); $firstview->blacklist_redirect_addresses(["10.0.0.10", "10.0.0.20"]); sets override_blacklist_redirect to "true".

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get blacklist_redirect_addresses
my $blacklist_redirect_addresses = $firstview->blacklist_redirect_addresses( );
#Modify blacklist_redirect_addresses
$firstview->blacklist_redirect_addresses(["10.0.0.10", "10.0.0.20"]);
```

blacklist_redirect_ttl( )

Use this method to set or retrieve the TTL value of the synthetic DNS responses that result from blacklist redirection.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

A 32-bit integer, from 0 to 4294967295, that represents the TTL in seconds. The default value is 60.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get blacklist_redirect_ttl
my $blacklist_redirect_ttl = $firstview->blacklist_redirect_ttl( );
#Modify blacklist_redirect_ttl
$firstview->blacklist_redirect_ttl(60);
```

blacklist_rulesets( )

Use this method to specify or retrieve Infoblox::DNS::Ruleset object names assigned at the grid level for blacklist redirection.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

The valid value is an array reference that contains the names of the Infoblox::DNS::Ruleset objects assigned to the grid. The names must be in string format.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get blacklist_rulesets
my $blacklist_rulesets = $firstview->blacklist_rulesets();
#Modify blacklist_rulesets
$firstview->blacklist_rulesets(["ruleset1", "ruleset2"]);
```

cloud_info( )

Use this method to retrieve cloud API related information for the Infoblox::DNS::View object.

Parameter

None

Returns

The method returns the attribute value.

Example

```
# Get cloud_info
my $cloud_info = $object->cloud_info();
```

comment( )

Use this method to set or retrieve the comment.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

Desired comment in string format with a maximum of 256 bytes.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

Example

```
#Get comment
my $comment = $firstview->comment();
#Modify comment
$firstview->comment("modifying the internal view comment");
```

custom_root_name_servers( )

Use this method to set or retrieve the list of customized root name servers. You can either select and use Internet root name servers or specify custom root name servers by providing a host name and IP address to which the Infoblox appliance can send queries.Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

Valid value is an array reference that contains Infoblox::DNS::RootNameServer object(s).

Returns

If you specify a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get custom_root_name_servers
my $rootNS_list = $firstview->custom_root_name_servers();
#Modify the root nameserver list
#Creating RootNameServer objects.
my $rootns1 = Infoblox::DNS::RootNameServer->new(
    ipv4addr  => "4.4.4.4",
    host_name => "rns1.test.com",
    );

my $rootns2 = Infoblox::DNS::RootNameServer->new(
    ipv4addr  => "5.5.5.5",
    host_name => "rns2.test.com",
    );

$firstview->custom_root_name_servers([$rootns1, $rootns2]);
```

ddns_force_creation_timestamp_update( )

Use this method to set or retrieve the flag that indicates whether the creation timestamp of resource records (RR) should be updated when DDNS update happens even if there is no change to the RR.

Setting this parameter to defined value, implicitly sets the override_ddns_force_creation_timestamp_update attribute to "true". Setting this parameter to undefined causes the appliance to use the Grid-level ddns_force_creation_timestamp_update flag and automatically sets the override_ddns_force_creation_timestamp_update attribute to "false".

Note that when ddns_force_creation_timestamp_update contains a value and override_ddns_force_creation_timestamp_update is set to

"false", the last operation takes precedence. Thus the sequence $object->ddns_force_creation_timestamp_update("true"); $object->override_ddns_force_creation_timestamp_update("false"); sets override_ddns_force_creation_timestamp_update to "false", and the sequence $object->override_ddns_force_creation_timestamp_update("false"); $object->ddns_force_creation_timestamp_update("true"); sets override_ddns_force_creation_timestamp_update to "true".

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

Specify 'true' to update the creation timestamp of RR on every DDNS update, and 'false' to update the creation timestamp only when RR changes during a DDNS update. The default is 'false'.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get ddns_force_creation_timestamp_update value
$value = $object->ddns_force_creation_timestamp_update();

#modify ddns_force_creation_timestamp_update value
$object->ddns_force_creation_timestamp_update('true');
```

ddns_restrict_patterns_list( )

Use this method to set or retrieve an unordered list of restriction patterns for an option of to restrict DDNS updates based on FQDN patterns.

Setting this parameter to any list, implicitly sets the override_ddns_patterns_restriction attribute to "true". Setting this parameter to undefined causes the appliance to use the Grid-level DDNS update restrictions and automatically sets the override_ddns_patterns_restriction attribute to "false" if "ddns_restrict_patterns" is also undefined. But if "ddns_restrict_patterns" contains a value, then the override_ddns_patterns_restriction attribute is set to "true".

Note that when ddns_restrict_patterns_list contains a valid array reference and override_ddns_patterns_restriction is set to "false", the last operation takes precedence. Thus the sequence $firstview->ddns_restrict_patterns_list([$pattern1, $pattern2]); $firstview->override_ddns_patterns_restriction("false"); sets override_ddns_patterns_restriction to "false", and the sequence $firstview->override_ddns_patterns_restriction("false"); $firstview->ddns_restrict_patterns_list([$pattern1, $pattern2]); sets override_ddns_patterns_restriction to "true".

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

The valid value is an array of FQDNs in a string format.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#get ddns_restrict_patterns_list value
$ddns_restrict_patterns_list = $firstview->ddns_restrict_patterns_list();

#modify ddns_restrict_patterns_list value
$firstview->ddns_restrict_patterns_list(['foo.com', 'bar.com']);
```

ddns_restrict_patterns( )

Use this method to set or retrieve the flag which indicates whether an option to restrict DDNS update request based on FQDN patterns is enabled or disabled.

Setting this parameter to defined value, implicitly sets the override_ddns_patterns_restriction attribute to "true". Setting this parameter to undefined causes the appliance to use the Grid-level DDNS update restrictions and automatically sets the override_ddns_patterns_restriction attribute to "false" if "ddns_restrict_patterns_list" is also undefined. But if "ddns_restrict_patterns_list" contains a value, then the override_ddns_patterns_restriction attribute is set to "true".

Note that when ddns_restrict_patterns contains a value and override_ddns_patterns_restriction is set to "false", the last operation takes precedence. Thus the sequence $firstview->ddns_restrict_patterns("true"); $firstview->override_ddns_patterns_restriction("false"); sets override_ddns_patterns_restriction to "false", and the sequence $firstview->override_ddns_patterns_restriction("false"); $firstview->ddns_restrict_patterns("true"); sets override_ddns_patterns_restriction to "true".

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

Specify 'true' to enable DDNS update request restriction base on FQDN patterns and 'false to disable it. The default value is 'false'.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#get ddns_restrict_patterns value
$ddns_restrict_patterns = $firstview->ddns_restrict_patterns();

#modify ddns_restrict_patterns value
$firstview->ddns_restrict_patterns('true');
```

ddns_restrict_static( )

Use this method to set or retrieve the flag which indicates whether an option to restrict DDNS update request to resource records which are marked as 'STATIC' is enabled or disabled.

Setting this parameter to defined value, implicitly sets the override_ddns_restrict_static attribute to "true". Setting this parameter to undefined causes the appliance to use the Grid-level ddns_restrict_static flag and automatically sets the override_ddns_restrict_static attribute to "false".

Note that when ddns_restrict_static contains a value and override_ddns_restrict_static is set to "false", the last operation takes precedence. Thus the sequence $firstview->ddns_restrict_static("true"); $firstview->override_ddns_restrict_static("false"); sets override_ddns_restrict_static to "false", and the sequence $firstview->override_ddns_restrict_static("false"); $firstview->ddns_restrict_static("true"); sets override_ddns_restrict_static to "true".

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

Specify 'true' to enable DDNS update requests to resource records which are marked as 'STATIC' and 'false' to disable it. The default value is 'false'.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#get ddns_restrict_static value
$ddns_restrict_static = $firstview->ddns_restrict_static();

#modify ddns_restrict_static value
$firstview->ddns_restrict_static('true');
```

ddns_restrict_protected( )

Use this method to set or retrieve the flag which indicates whether an option to restrict DDNS update request to protected resource records is enabled or disabled.

Setting this parameter to defined value, implicitly sets the override_ddns_restrict_protected attribute to "true". Setting this parameter to undefined causes the appliance to use the Grid-level ddns_restrict_protected flag and automatically sets the override_ddns_restrict_protected attribute to "false".

Note that when ddns_restrict_protected contains a value and override_ddns_restrict_protected is set to "false", the last operation takes precedence. Thus the sequence $firstview->ddns_restrict_protected("true"); $firstview->override_ddns_restrict_protected("false"); sets override_ddns_restrict_protected to "false", and the sequence $firstview->override_ddns_restrict_protected("false"); $firstview->ddns_restrict_protected("true"); sets override_ddns_restrict_protected to "true".

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

Specify 'true' to enable DDNS update requests to protected resource records and 'false' to disable it. The default value is 'false'.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#get ddns_restrict_protected value
$ddns_restrict_protected = $firstview->ddns_restrict_protected();

#modify ddns_restrict_protected value
$firstview->ddns_restrict_protected('true');
```

ddns_restrict_secure( )

Use this method to set or retrieve the flag which indicates whether DDNS update request for principal other than target resource record's principal is restricted.

Setting this parameter to a defined value, implicitly sets the override_ddns_principal_security attribute to "true". Setting this parameter to undefined causes the appliance to use the Grid-level DDNS principal security setting and automatically sets the override_ddns_principal_security attribute to "false" if "ddns_principal_tracking" and "ddns_principal_group" are also undefined. But if "ddns_principal_tracking" or "ddns_principal_group" contains a value, then the override_ddns_principal_security attribute is set to "true".

Note that when ddns_restrict_secure contains a value and override_ddns_principal_security is set to "false", the last operation takes precedence. Thus the sequence $firstview->ddns_restrict_secure("true"); $firstview->override_ddns_principal_security("false"); sets override_ddns_principal_security to "false", and the sequence $firstview->override_ddns_principal_security("false"); $firstview->ddns_restrict_secure("true"); sets override_ddns_principal_security to "true".

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

Specify 'true' to enable DDNS update requests restriction for principal other than target resource record's principal and 'false' to disable it. The default value is 'false'.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#get ddns_restrict_secure value
$ddns_restrict_secure = $firstview->ddns_restrict_secure();

#modify ddns_restrict_secure value
$firstview->ddns_restrict_secure('true');
```

ddns_principal_tracking( )

Use this method to set or retrieve the flag which indicates whether the DDNS principal track is enabled or disabled.

DDNS principal tracking includes putting principal name into resource record affected by update.

Setting this parameter to a defined value, implicitly sets the override_ddns_principal_security attribute to "true". Setting this parameter to undefined causes the appliance to use the Grid-level DDNS principal security setting and automatically sets the override_ddns_principal_security attribute to "false" if "ddns_restrict_secure" and "ddns_principal_group" are also undefined. But if "ddns_restrict_secure" or "ddns_principal_group" contains a value, then the override_ddns_principal_security attribute is set to "true".

Note that when ddns_principal_tracking contains a value and override_ddns_principal_security is set to "false", the last operation takes precedence. Thus the sequence $firstview->ddns_principal_tracking("true"); $firstview->override_ddns_principal_security("false"); sets override_ddns_principal_security to "false", and the sequence $firstview->override_ddns_principal_security("false"); $firstview->ddns_principal_tracking("true"); sets override_ddns_principal_security to "true".

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

Specify 'true' to enable DDNS principal tracking and 'false' to disable it. The default value is 'false'.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#get ddns_principal_tracking value
$ddns_principal_tracking = $firstview->ddns_principal_tracking();

#modify ddns_principal_tracking value
```

```
$firstview->ddns_principal_tracking('true');
```

ddns_principal_group( )

Use this method to set or retrieve the DDNS principal group.

Setting this parameter to a defined value, implicitly sets the override_ddns_principal_security attribute to "true". Setting this parameter to undefined causes the appliance to use the Grid-level DDNS principal security setting and automatically sets the override_ddns_principal_security attribute to "false" if "ddns_restrict_secure" and "ddns_principal_tracking" are also undefined. But if "ddns_restrict_secure" or "ddns_principal_tracking" contains a value, then the override_ddns_principal_security attribute is set to "true".

Note that when ddns_principal_group contains a value and override_ddns_principal_security is set to "false", the last operation takes precedence. Thus the sequence $firstview->ddns_principal_group($principal_group); $firstview->override_ddns_principal_security("false"); sets override_ddns_principal_security to "false", and the sequence $firstview->override_ddns_principal_security("false"); $firstview->ddns_principal_group($principal_group); sets override_ddns_principal_security to "true".

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

The valid value is an Infoblox::DNS::DDNS::PrincipalCluster::Group object.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#get ddns_principal_group value
$ddns_principal_group = $firstview->ddns_principal_group();

#modify ddns_principal_group value
$firstview->ddns_principal_group($principal_group);
```

disable( )

Use this method to set or retrieve the disable flag of a DNS view object.

Include the specific parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

The default value for this field is false; therefore, the DNS view is enabled.

Parameter

Specify "true" to set the disable flag or "false" to deactivate/unset it.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

Example

```
#Get disable
my $disable = $firstview->disable();
#Modify disable
$firstview->disable("true");
```

dns64_groups( )

Use this method to set or retrieve the list of DNS64 synthesis groups associated with this DNS view.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Setting this method to a defined value implicitly sets the override_dns64 method to "true". Setting the parameter to undefined causes the appliance to use the grid DNS default and automatically resets the override_dns64 method to "false".

Note that when dns64_groups is set to a defined value and override_dns64_groups is set to "false", the last operation takes precedence. Thus the sequence $object->dns64_groups([$group1]); $object->override_dns64("false"); will set override_dns64 to "false", and the sequence $object->override_dns64([$group1]); $object->dns64_groups("testdomain"); will result in override_dns64="true".

Parameter

Valid value is an array reference that contains Infoblox::Grid::DNS::DNS64Group objects.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get the list of DNS64 synthesis groups
my $dns64_groups_list = $firstview->dns64_groups();

#Modify it
my $group = $session->get(
                        object => "Infoblox::Grid::DNS::DNS64Group"),
                        name => "groupname",
                        prefix => "groupprefix",
                      );
$firstview->dns64_groups([$group]);
```

dnssec_enabled( )

Use this method to enable/disable the DNS security extension.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

Set the parameter to "true" to enable the DNS security extension. Set it to "false" to disable the feature.

Setting this parameter to "true"/"false" implicitly sets the override_dnssec attribute to "true". Setting the parameter to undefined causes the appliance to use the member-level DNS security settings and automatically resets the override_dnssec attribute to "false".

Note that in the case when dnssec_enabled is set to "true"/"false" and override_dnssec is set to "false" simultaneously, the last operation takes precedence. Thus the sequence $firstview->dnssec_enabled("false"); $firstview->override_dnssec("false"); will set override_dnssec to "false", and the sequence $firstview->override_dnssec("false"); $firstview->dnssec_enabled("false"); will result in override_dnssec="true".

The default value of the parameter is undefined, which means the DNS view inherits the member-level settings.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get dnssec_enabled
my $dnssec_enabled = $firstview->dnssec_enabled();
#Modify dnssec_enabled
$firstview->dnssec_enabled("true");
#Set the value to undefined to inherit the member-level settings.
$firstview->dnssec_enabled(undef);
```

dnssec_expired_signatures_enabled( )

Use this method to control whether DNS accepts expired signatures.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

Set the parameter to "true" to accept expired signatures. Set it to "false" to reject expired signatures.

The default value of the parameter is "false".

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get dnssec_expired_signatures_enabled
my $dnssec_expired_signatures_enabled = $firstview->dnssec_expired_signatures_enabled();
#Modify dnssec_expired_signatures_enabled
$firstview->dnssec_expired_signatures_enabled("true");
```

dnssec_validation_enabled( )

Use this method to enable/disable DNS security validation.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

Set the parameter to "true" to enable the validation. Set the parameter to "false" to disable the validation.

The default value of the parameter is "true".

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get dnssec_validation_enabled
my $dnssec_validation_enabled = $firstview->dnssec_validation_enabled();
#Modify dnssec_validation_enabled
$firstview->dnssec_validation_enabled("false");
```

dnssec_blacklist_enabled()

Use this method to enable and disable blacklist rules for DNSSEC-enabled clients.

Parameter

Set the parameter to "true" to apply blacklist rules for DNSSEC-enabled clients. Set to "false" to disable them.

The default value of the parameter is "false".

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get dnssec_blacklist_enabled
my $dnssec_blacklist_enabled = $firstview->dnssec_blacklist_enabled();
#Modify dnssec_blacklist_enabled
$firstview->dnssec_blacklist_enabled("true");
```

dnssec_negative_trust_anchors( )

Use this method to set or retrieve the list of zone for which the appliance does not perform DNSSEC validation.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

The valid value is an array of strings that contains the list of zone for which the appliance does not perform DNSSEC validation.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get dnssec_negative_trust_anchors value
my $value = $view->dnssec_negative_trust_anchors();
#Modify dnssec_negative_trust_anchors value
$view->dnssec_negative_trust_anchors($zones);
```

dnssec_dns64_enabled()

Use this method to enable and disable DNS64 groups for DNSSEC-enabled clients.

Parameter

Set the parameter to "true" to apply DNS64 groups for DNSSEC-enabled clients. Set to "false" to disable them.

The default value of the parameter is "false".

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get dnssec_dns64_enabled
my $dnssec_dns64_enabled = $grid_dns->dnssec_dns64_enabled();
#Modify dnssec_dns64_enabled
$grid_dns->dnssec_dns64_enabled("true");
```

dnssec_nxdomain_enabled()

Use this method to enable and disable NXDOMAIN rules for DNSSEC-enabled clients.

Parameter

Set the parameter to "true" to apply NXDOMAIN rules for DNSSEC-enabled clients. Set to "false" to disable them.

The default value of the parameter is "false".

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get dnssec_nxdomain_enabled
my $dnssec_nxdomain_enabled = $firstview->dnssec_nxdomain_enabled();
#Modify dnssec_nxdomain_enabled
$firstview->dnssec_nxdomain_enabled("true");
```

dnssec_rpz_enabled()

Use this method to enable and disable RPZ policies for DNSSEC-enabled clients.

Parameter

Set the parameter to "true" to apply RPZ policies for DNSSEC-enabled clients. Set to "false" to disable them.

The default value of the parameter is "false".

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get dnssec_rpz_enabled
my $dnssec_rpz_enabled = $firstview->dnssec_rpz_enabled();
#Modify dnssec_rpz_enabled
$firstview->dnssec_rpz_enabled("true");
```

dnssec_trusted_keys( )

Use this method to specify the list of trusted keys for the DNS security extension.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

Array reference that contains a list of Infoblox::DNS::DnssecTrustedKey objects.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns a reference to a list of the trusted keys.

Example

```
#Get dnssec_trusted_keys
```

```
my $dnssec_trusted_keys = $firstview->dnssec_trusted_keys();
#Modify dnssec_trusted_keys
my $key1=Infoblox::DNS::DnssecTrustedKey->new(
   fqdn => "myzone.com",
   algorithm => "RSAMD5",
   key       => "c2ltcGxlIGtleQo="
   );
$firstview->dnssec_trusted_keys([$key1]);
```

enable_blacklist( )

Use this method to enable or disable a blacklist in a DNS view.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

Specify "true" to set the enable_blacklist flag or "false" to deactivate/unset it. The default is "false".

Setting this parameter to "true" or "false" implicitly sets the override_blacklist attribute to "true". Setting the parameter to undefined causes the appliance to use the member-level blacklist settings and automatically sets the override_blacklist attribute to "false" if "blacklist_redirect_addresses" is also undefined.

Note that when enable_blacklist is set to "true"/"false" and override_blacklist is set to "false", the last operation takes precedence. Thus the sequence $firstview->enable_blacklist("false"); $firstview->override_blacklist("false"); sets override_blacklist to "false", and the sequence $firstview->override_blacklist("false"); $firstview->enable_blacklist("false"); sets override_blacklist to "true".

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Getting enable_blacklist
my $enable_blacklist = $firstview->enable_blacklist( );
#Modifying enable_blacklist
$firstview->enable_blacklist("true");
```

enable_dns64( )

Use this method to enable/disable DNS64 support for this view.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Setting this method to a defined value implicitly sets the override_dns64 method to "true". Setting the parameter to undefined causes the appliance to use the grid DNS default and automatically resets the override_dns64 method to "false".

Note that when enable_dns64 is set to a defined value and override_enable_dns64 is set to "false", the last operation takes precedence. Thus the sequence $object->enable_dns64("true"); $object->override_dns64("false"); will set override_dns64 to "false", and the sequence $object->override_dns64("true"); $object->enable_dns64("testdomain"); will result in override_dns64="true".

Parameter

Specify "true" to set the enable_dns64 flag or "false" to deactivate and unset it. The default value is "false".

Returns

When you specify a parameter and the modification succeeds, the method returns "true". When the operation fails, the method returns "false".

When you do not specify a parameter, the method returns the attribute value.

Example

```
#Get the enable_dns64 flag
my $enable_dns64 = $firstview->enable_dns64();
#Modify the enable_dns64 flag
$firstview->enable_dns64("true");
```

enable_match_recursive_only( )

Use this method to enable or disable the match-recursive-only setting for a DNS View on the member.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

Specify 'true' to enable 'match-recursive-only' setting or 'false' to disable it.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#get enable_match_recursive_only value
my $enable_match_recursive_only = $firstview->enable_match_recursive_only();

#modify enable_match_recursive_only value
$firstview->enable_match_recursive_only('true');
```

enable_fixed_rrset_order_fqdns( )

Use this method to enable or disable fixed RRset order for specified FQDNs.

Setting this method to a defined value implicitly sets the override_fixed_rrset_order_fqdns method to "true".

Note that when enable_fixed_rrset_order_fqdns is set to a defined value and override_fixed_rrset_order_fqdns is set to "false", the last operation takes precedence. Thus the sequence $object->enable_fixed_rrset_order_fqdns("true"); $object->override_fixed_rrset_order_fqdns("false"); will set override_fixed_rrset_order_fqdns to "false", and the sequence $object->override_fixed_rrset_order_fqdns("false"); $object->enable_fixed_rrset_order_fqdns("true"); will result in override_fixed_rrset_order_fqdns="true".

The enable_fixed_rrset_order_fqdns attribute can be specified explicitly. It is also set implicitly when fixed_rrset_order_fqdns is set to a defined value.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

Specify "true" to enable fixed RRset order for specified FQDNs or "false" to disable it.

Returns

When you specify a parameter and the modification succeeds, the method returns "true". When the operation fails, the method returns "false".

When you do not specify a parameter, the method returns the attribute value.

Example

```
#Get enable_fixed_rrset_order_fqdns flag
my $enable_fixed_rrset_order_fqdns = $firstview->enable_fixed_rrset_order_fqdns();
#Modify enable_fixed_rrset_order_fqdns flag
$firstview->enable_fixed_rrset_order_fqdns("true");
```

extattrs( )

Use this method to set or retrieve the extensible attributes associated with a DNS view object.

Parameter

Valid value is a hash reference containing the names of extensible attributes and their associated values (Infoblox::Grid::Extattr objects).

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get extattrs
my $ref_extattrs = $firstview->extattrs();
#Modify extattrs
$firstview->extattrs({ 'Site' => $extattr1, 'Administrator' => $extattr2 });
```

extensible_attributes( )

Use this method to set or retrieve the extensible attributes associated with a DNS view object.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

For valid values for extensible attributes, see Infoblox::Grid::ExtensibleAttributeDef/Extensible Attribute Values.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get extensible attributes
my $ref_extensible_attributes = $firstview->extensible_attributes();
#Modify extensible attributes
$firstview->extensible_attributes({ 'Site' => 'Santa Clara', 'Administrator' => [ 'Peter', 'Tom' ]
});
```

filter_aaaa( )

Use this method to set or retrieve the type of AAAA filtering for this DNS view object.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Specifying values in this parameter, such as "YES", implicitly sets the override_filter_aaaa attribute to "true". Setting this parameter and the "filter_aaaa" attribute to undefined causes the appliance to use the grid-level filter_aaaa settings and automatically sets the override_filter_aaaa attribute to "false". But if the "filter_aaaa" attribute contains a value, then the override_filter_aaaa attribute is set to "true".

Note that when filter_aaaa contains a valid value and override_filter_aaaa is set to "false", the last operation takes precedence. Thus the sequence $firstview->filter_aaaa("YES"); $firstview->override_filter_aaaa("false"); sets override_filter_aaaa to "false", and the sequence $firstview->override_filter_aaaa("false"); $firstview->filter_aaaa("true"); sets override_filter_aaaa to "true".

Parameter

Valid values are "YES", "NO" and "BREAK_DNSSEC". The default value is "NO".

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get the filter aaaa type
my $filter_aaaa = $firstview->filter_aaaa();
#Modify the filtering
$firstview->filter_aaaa("YES");
```

filter_aaaa_list( )

Use this method to set or retrieve the list of IPv4 addresses and networks from which queries are received. AAAA filtering is applied to these addresses. This setting overrides the grid filter_aaaa_list if override_filter_aaaa is set to "true".

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Specifying values in this parameter, such as ["10.0.0.10", "10.0.0.20"] or [], implicitly sets the override_filter_aaaa attribute to "true". Setting this parameter and the "filter_aaaa" attribute to undefined causes the appliance to use the grid-level filter_aaaa settings and automatically sets the override_filter_aaaa attribute to "false". But if the "filter_aaaa" attribute contains a value, then the override_filter_aaaa attribute is set to "true".

Note that when filter_aaaa_list contains a valid array reference and override_filter_aaaa is set to "false", the last operation takes precedence. Thus the sequence $firstview->filter_aaaa_list(["10.0.0.10", "10.0.0.20"]); $firstview->override_filter_aaaa("false"); sets override_filter_aaaa to "false", and the sequence $firstview->override_filter_aaaa("false"); $firstview->filter_aaaa_list(["10.0.0.10", "10.0.0.20"]); sets override_filter_aaaa to "true".

Parameter

A valid value is an Infoblox::Grid::NamedACL object or an array reference that contains IPv4 IP addresses and networks. To return AAAA records for queries received from an IPv4 client, add "!" as the prefix to the IP addresses and networks. Specify "any" to allow AAAA records to be returned for any query received by an IPv4 client.

Returns

When you specify a parameter and the modification succeeds, the method returns "true". When the operation fails, the method returns "false".

When you do not specify a parameter, the method returns the attribute value.

Example

```
#Getting filter_aaaa_list
my $filter_aaaa_list = $firstview->filter_aaaa_list( );
#Modifying filter_aaaa_list
$firstview->filter_aaaa_list(["any"]);
#Modifying filter_aaaa_list
$firstview->filter_aaaa_list(["10.0.0.10", "20.0.0.0/24", "!"."30.0.0.30", "!"."40.10.0.0/24"]);
#Use Grid::DNS level filter_aaaa_list
$firstview->filter_aaaa_list(undef);
$firstview->enable_filter_aaaa(undef);
#Use Infoblox::Grid::NamedACL object
$firstview->filter_aaaa_list($nacl);
```

fixed_rrset_order_fqdns( )

Use this method to set or retrieve configuration for fixed RRSET order for specified FQDNs.

Setting this method to a defined value implicitly sets the enable_fixed_rrset_order_fqdns and override_fixed_rrset_order_fqdns methods to "true". Setting the parameter to undefined causes the appliance to disable fixed RRset order of FQDNs and automatically resets the enable_fixed_rrset_order_fqdns method to "false".

Note that when fixed_rrset_order_fqdns is set to a defined value and enable_fixed_rrset_order_fqdns is set to "false", the last operation takes precedence. Thus the sequence $object->fixed_rrset_order_fqdns($fqdns); $object->enable_fixed_rrset_order_fqdns("false"); will set enable_fixed_rrset_order_fqdns to "false", and the sequence $object->enable_fixed_rrset_order_fqdns("false"); $object->fixed_rrset_order_fqdns($fqdns); will result in enable_fixed_rrset_order_fqdns="true".

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

The valid value is an array reference that contains Infoblox::Grid::DNS::FixedRRSetOrderFQDN objects.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get the fixed_rrset_order_fqdns list
my $fixed_rrset_order_fqdns = $firstview->fixed_rrset_order_fqdns();
#Modify the fixed_rrset_order_fqdns list
$firstview->fixed_rrset_order_fqdns([$fqdn1, $fqdn2]);
```

forwarders( )

Use this method to specify forwarders for the DNS view. A forwarder is essentially a name server to which other name servers first send all of their off-site queries. The forwarder builds up a cache of information, avoiding the need for the other name servers to send queries off-site. This setting overrides the grid-level setting.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

Valid value is an array reference that contains Infoblox::DNS::Member, Infoblox::DNS::Nameserver object(s), or IP addresses.

Specifying values in this parameter, for example ["10.0.0.10", "10.0.0.20"] or [], implicitly sets the override_forwarders attribute to "true". Setting this parameter and the "forward_only" attribute to undefined causes the appliance to use the member-level forwarders setting and automatically sets the override_forwarders attribute to "false". But if the "forward_only" attribute contains a value, then the override_forwarders attribute is set to "true".

Note that when forwarders_redirect_addresses contains a valid array reference and override_forwarders is set to "false", the last operation takes precedence. Thus the sequence $firstview->forwarders(["10.0.0.10", "10.0.0.20"]); $firstview->override_forwarders("false"); sets override_forwarders to "false", and the sequence $firstview->override_forwarders("false"); $firstview->forwarders(["10.0.0.10", "10.0.0.20"]); sets override_forwarders to "true".

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Getting forwarders
```

```
$firstview->forwarders( );
#Modifying forwarders
$firstview->forwarders( [ "10.0.0.10" ] );
```

forward_only( )

Use this method to configure this view to send queries to forwarders only. When the value is "true", the queries are sent to forwarders only, and not to other internal or Internet root servers.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

Specify "true" to use forwarders only or "false" to use forwarders and root servers. The default is "false"

Setting this parameter to "true" or "false" implicitly sets the override_forwarders attribute to "true". Setting the parameter to undefined causes the appliance to use the member-level forwarders settings and automatically sets the override_forwarders attribute to "false" if "forwarders" is also undefined.

Note that when forward_only is set to "true"/"false" and override_forwarders is set to "false", the last operation takes precedence. Thus the sequence $firstview->forward_only("false"); $firstview->override_forwarders("false"); sets override_forwarders to "false", and the sequence $firstview->override_forwarders("false"); $firstview->forward_only("false"); sets override_forwarders to "true".

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get forward_only
$firstview->forward_only( );
#Modify forward_only
$firstview->forward_only("true");
```

match_clients( )

Use this method to set or retrieve the match_clients list. This list specifies IP addresses, networks and/or TSIG keys of the clients that are allowed or denied access to the DNS view.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

Valid value is an Infoblox::Grid::NamedACL object or an array reference that contains IPv4/IPv6 addresses or networks. To deny access to IP addresses and/or networks, add "!" as a prefix to IP addresses and/or networks. Specify "any" to allow access to any client. To allow access to a client authorized by a TSIG key, use Infoblox::DNS::TSIGKey objects.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

Example

```
#Get match_clients
my $clients = $firstview->match_clients();
#Modify match_clients
$firstview->match_clients(["any"]);

#Specifying TSIG keys
my $tsig_key3 = $session->gen_tsig_key();
my $key3 = Infoblox::DNS::TSIGKey->new(
    name => "key3",
    key  => $tsig_key3
);
#Modify match_clients
$firstview->match_clients(["10.0.0.10" , $key3, "20.0.0.0/24"
,"2001::123a","2003::222/64","!"."30.0.0.30" ,
"!"."40.10.0.0/24","!"."2004::125b","!"."2006::34bc"]);
#Use Infoblox::Grid::NamedACL
$firstview->match_clients($nacl);
```

match_destinations( )

Use this method to set or retrieve the match_destinations list. This list specifies IP addresses, networks and/or TSIG keys of the clients that are allowed or denied access to the DNS view.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

Valid value is an Infoblox::Grid::NamedACL object or an array reference that contains IPv4/IPv6 addresses or networks. To deny access to IP addresses and/or networks, add "!" as a prefix to IP addresses and/or networks. Specify "any" to allow access to any client. To allow access to a client authorized by a TSIG key, use Infoblox::DNS::TSIGKey objects.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

Example

```
#Get match_destinations
my $clients = $firstview->match_destinations();
#Modify match_destinations
$firstview->match_destinations(["any"]);

#Specifying TSIG keys
my $tsig_key3 = $session->gen_tsig_key();
my $key3 = Infoblox::DNS::TSIGKey->new(
    name => "key3",
    key  => $tsig_key3
);
#Modify match_destinations
$firstview->match_destinations(["10.0.0.10" , $key3, "20.0.0.0/24"
,"2001::123a","2003::222/64","!"."30.0.0.30" ,
"!"."40.10.0.0/24","!"."2004::125b","!"."2006::34bc"]);
#Use Infoblox::Grid::NamedACL
$firstview->match_destinations($nacl);
```

max_cache_ttl( )

Use this method to set or retrieve the maximum time (in seconds) for which the server will cache positive answers.

Parameter

The maximum time is specified in seconds. The default value is 604800.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get max_cache_ttl
my $max_cache_ttl = $firstview->max_cache_ttl();
#Modify max_cache_ttl
$firstview->max_cache_ttl(400);
```

max_ncache_ttl( )

Use this method to set or retrieve the maximum time (in seconds) for which the server will cache negative (NXDOMAIN) responses.

Parameter

The maximum time is specified in seconds. The default value is 10800. The maximum allowed value is 604800.

Setting this parameter implicitle sets the override_max_ncache_ttl attribute to "true". Setting the parameter to undefined causes the appliance to use the Grid-level settings and automatically sets the override_max_ncache_ttl attribute to "false".

Note that when max_ncache_ttl is set and override_max_ncache_ttl is set to "false", the last operation takes precedence. Thus the sequence $firstview->max_ncache_ttl(10800); $firstview->override_max_ncache_ttl("false"); sets override_max_ncache_ttl to "false", and the sequence $firstview->override_max_ncache_ttl("false"); $firstview->max_ncache_ttl(30600); sets override_max_ncache_ttl to "true".

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get max_ncache_ttl
```

```
my $max_ncache_ttl = $firstview->max_ncache_ttl();
#Modify max_ncache_ttl
$firstview->max_ncache_ttl(400);
```

lame_ttl( )

Use this method to set or retrieve the number of seconds to cache lame delegations or lame servers.

Parameter

The number of seconds to cache lame delegations or lame servers. The default value is 600.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get lame_ttl
my $lame_ttl = $firstview->lame_ttl();
#Modify lame_ttl
$firstview->lame_ttl(700);
```

name( )

Use this method to configure or retrieve the name of a DNS view.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

Name of the DNS view in string format. It has a maximum of 64 bytes.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

Example

```
#Get name
my $name = $firstview->name();
#Modify name
$firstview->name("external");
```

network_view( )

Use this method to set or retrieve the network view object associated with this DNS view object.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

The value should be a reference of defined Infoblox::DHCP::View objects.

Returns

If you specify a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you do not specify a parameter, the method returns the attribute value.

Example

```
#Get network_view
my $useRootNS = $firstview->network_view( );
#Modify network_view
$firstview->use_root_name_servers($network_view);
```

notify_delay( )

Use this method to specify with how many seconds of delay the notify messages are sent to secondaries.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

Valid values are unsigned integer between 5 and 86400, inclusive. Default value is undefined.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Getting notify_delay
my $notify_delay = $firstview->notify_delay( );
#Modifying notify_delay
$firstview->notify_delay(5);
```

override_blacklist( )

The "override_blacklist" attribute controls whether the "enable_blacklist", "blacklist_redirect_addresses", "blacklist_redirect_ttl", "blacklist_log_query", and "blacklist_rulesets" values in the database are used.

The "override_blacklist" attribute can be specified explicitly. It is also set implicitly as follows: "enable_blacklist" => "true"|"false" or "blacklist_redirect_addresses" => ["10.0.0.10", "10.0.0.20"]|[] results in "override_blacklist" => "true", "enable_blacklist" => undef and "blacklist_redirect_addresses" => undef results in "override_blacklist" => "false". If "override_blacklist" is specified explicitly as "true", and "enable_blacklist" => undef or "blacklist_redirect_addresses" => undef, then the explicit value takes precedence. Therefore, "override_blacklist"=>"true", "enable_blacklist"=>undef, and "blacklist_redirect_addresses"=>undef remain unchanged in the database.

Specifying the "blacklist_redirect_ttl", "blacklist_log_query", or "blacklist_rulesets" attributes does not affect the "override_blacklist" value. It is implicitly set only by "enable_blacklist" and "blacklist_redirect_addresses".

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

Set the parameter to "true" to override the member-level settings of the blacklist redirect attributes. Set the parameter to "false" to inherit the member-level settings. The default is "false".

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get override_blacklist
my $override_blacklist=$firstview->override_blacklist( );
#Modify override_blacklist
$firstview->override_blacklist("true");
```

override_ddns_force_creation_timestamp_update( )

The override_ddns_force_creation_timestamp_update attribute controls whether the ddns_force_creation_timestamp_update method value of the DNS View is used, instead of the Grid default.

The override_ddns_force_creation_timestamp_update attribute can be specified explicitly. It is also set implicitly when ddns_force_creation_timestamp_update is set to a defined value.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

Set the parameter to "true" to override the Grid-level setting for ddns_force_creation_timestamp_update. Set the parameter to "false" to inherit the Grid-level setting for ddns_force_creation_timestamp_update.

The default value of this parameter is "false".

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get override_ddns_force_creation_timestamp_update value
my $value = $object->override_ddns_force_creation_timestamp_update();

#Modify override_ddns_force_creation_timestamp_update value
$object->override_ddns_force_creation_timestamp_update("true");
```

override_ddns_patterns_restriction( )

The override_ddns_patterns_restriction attribute controls whether the ddns_restrict_patterns_list and ddns_restrict_patterns methods values of the DNS View are used, instead of the Grid default.

The override_ddns_patterns_restriction attribute can be specified explicitly. It is also set implicitly when ddns_restrict_patterns_list or ddns_restrict_patterns is set to a defined value.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

Set the parameter to "true" to override the Grid-level setting for ddns_restrict_patterns_list and ddns_restrict_patterns. Set the parameter to "false" to inherit the Grid-level setting for ddns_restrict_patterns_list and ddns_restrict_patterns.

The default value of this parameter is "false".

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get override_ddns_patterns_restriction
my $override_ddns_patterns_restriction = $firstview->override_ddns_patterns_restriction();
#Modify override_ddns_patterns_restriction
$firstview->override_ddns_patterns_restriction("true");
```

override_ddns_restrict_static( )

The override_ddns_restrict_static attribute controls whether the ddns_restrict_static method value of the DNS View is used, instead of the Grid default.

The override_ddns_restrict_static attribute can be specified explicitly. It is also set implicitly when ddns_restrict_static is set to a defined value.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

Set the parameter to "true" to override the Grid-level setting for ddns_restrict_static. Set the parameter to "false" to inherit the Grid-level setting for ddns_restrict_static.

The default value of this parameter is "false".

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get override_ddns_restrict_static
my $override_ddns_restrict_static = $firstview->override_ddns_restrict_static();
#Modify override_ddns_restrict_static
$firstview->override_ddns_restrict_static("true");
```

override_ddns_restrict_protected( )

The override_ddns_restrict_protected attribute controls whether the ddns_restrict_protected method value of the DNS View is used, instead of the Grid default.

The override_ddns_restrict_protected attribute can be specified explicitly. It is also set implicitly when ddns_restrict_protected is set to a defined value.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

Set the parameter to "true" to override the Grid-level setting for ddns_restrict_protected. Set the parameter to "false" to inherit the Grid-level setting for ddns_restrict_protected.

The default value of this parameter is "false".

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get override_ddns_restrict_protected
my $override_ddns_restrict_protected = $firstview->override_ddns_restrict_protected();
#Modify override_ddns_restrict_protected
$firstview->override_ddns_restrict_protected("true");
```

override_ddns_principal_security( )

The override_ddns_principal_security attribute controls whether the ddns_restrict_secure, ddns_principal_tracking and ddns_principal_group methods values of the DNS View are used, instead of the Grid default.

The override_ddns_principal_security attribute can be specified explicitly. It is also set implicitly when ddns_restrict_secure, ddns_principal_tracking or ddns_principal_group is set to a defined value.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

Set the parameter to "true" to override the Grid-level setting for ddns_restrict_secure, ddns_principal_tracking and ddns_principal_group. Set the parameter to "false" to inherit the Grid-level setting for ddns_restrict_secure, ddns_principal_tracking and ddns_principal_group.

The default value of this parameter is "false".

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get override_ddns_principal_security
my $override_ddns_principal_security = $firstview->override_ddns_principal_security();
#Modify override_ddns_principal_security
$firstview->override_ddns_principal_security("true");
```

override_dns64( )

The override_dns64 attribute controls whether the DNS64 method values in this DNS view are used, instead of the grid default.

The override_dns64 attribute can be specified explicitly. It is also set implicitly when enable_dns64 or dns64_groups are set to a defined value.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

Set the parameter to "true" to override the grid-level setting for enable_dns64 and dns64_groups. Set the parameter to "false" to inherit the grid-level setting for enable_dns64 and dns64_groups.

The default value of this parameter is "false".

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Getting override_dns64
my $override_dns64=$firstview->override_dns64( );
#Modifying override_dns64
$firstview->override_dns64("true");
```

override_dnssec( )

The override_dnssec attribute controls whether the dnssec_enabled, dnssec_validation_enabled, dnssec_expired_signatures_enabled, and dnssec_trusted_keys values in the database are used.

The override_dnssec attribute can be specified explicitly. It is also established implicitly as follows: "dnssec_enabled" => "true"|"false" results in "override_dnssec" => "true" "dnssec_enabled" => undef results in "override_dnssec" => "false" If the override_dnssec attribute

is specified explicitly as "true", while dnssec_enabled=>undef, the explicit value takes precedence, and override_dnssec=>"true" while dnssec_enabled remains unchanged in the DB.

Supplying the dnssec_validation_enabled, dnssec_expired_signatures_enabled, or dnssec_trusted_keys attributes does not affect the override_dnssec value. It is implicitly affected only by dnssec_enabled.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

Set the parameter to "true" to override the member-level setting of the DNSSEC attributes. Set the parameter to "false" to inherit the grid-member level settings of DNSSec.

The default value of this parameter is "false".

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Getting override_dnssec
my $override_dnssec=$firstview->override_dnssec( );
#Modifying override_dnssec
$firstview->override_dnssec("true");
```

override_filter_aaaa( )

The override_filter_aaaa attribute controls whether the AAAA filter method values of the view are used, instead of the grid default.

The override_filter_aaaa attribute can be specified explicitly. It is also set implicitly when filter_aaaa or filter_aaaa_list are set to a defined value.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

Set the parameter to "true" to override the grid-level setting for filter_aaaa and filter_aaaa_list. Set the parameter to "false" to inherit the grid-level setting for filter_aaaa and filter_aaaa_list.

The default value of this parameter is "false".

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Getting override_filter_aaaa
my $override_filter_aaaa=$firstview->override_filter_aaaa( );
#Modifying override_filter_aaaa
$firstview->override_filter_aaaa("true");
```

override_fixed_rrset_order_fqdns( )

The override_fixed_rrset_order_fqdns controls whether the enable_fixed_rrset_order_fqdns and fixed_rrset_order_fqdns values in the database are used.

The override_fixed_rrset_order_fqdns can be specified explicitly. It is also set implicitly as follows: enable_fixed_rrset_order_fqdns => "true" | "false" or fixed_rrset_order_fqdns => $fqdns results in override_fixed_rrset_order_fqdns => "true";

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

Set the parameter to "true" to override the Grid-leval fixed RRset order settings. Set the parameter to "false" to inherit the Grid-level settings. The default is "false".

Returns

When you specify a parameter and the modification succeeds, the method returns "true". When the operation fails, the method returns "false".

When you do not specify a parameter, the method returns the attribute value.

Example

```
#Get override_fixed_rrset_order_fqdns flag
my $override_fixed_rrset_order_fqdns = $firstview->override_fixed_rrset_order_fqdns();
#Modify override_fixed_rrset_order_fqdns flag
$firstview->override_fixed_rrset_order_fqdns("true");
```

override_forwarders( )

The "override_forwarders" attribute controls whether the "forward_only" and "forwarders" values in the database are used.

The "override_forwarders" attribute can be specified explicitly. It is also set implicitly as follows: "forward_only" => "true"|"false" or "forwarders" => ["10.0.0.10", "10.0.0.20"]|[] results in "override_forwarders" => "true"; "forward_only" => undef and "forwarders" => undef result in "override_forwarders" => "false". If "override_forwarders" is specified explicitly as "true", and "forward_only" => undef or "forwarders" => undef, then the explicit value takes precedence. Therefore, "override_forwarders"=>"true", "forward_only"=>undef, and "forwarders"=>undef remain unchanged in the database.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

Set the parameter to "true" to override the member-level settings of the forwarders attributes. Set the parameter to "false" to inherit the member-level settings. The default is "false".

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get override_forwarders
my $override_forwarders=$firstview->override_forwarders( );
#Modify override_forwarders
$firstview->override_forwarders("true");
```

override_lame_ttl( )

Use this method to set or retrieve the override_lame_ttl attribute that controls whether the lame_ttl method values of the view are used, instead of the Grid default

Parameter

Set the parameter to "true" to override the grid-level setting for lame_ttl. Set the parameter to "false" to inherit the grid-level setting for lame_ttl. The default value is 'false'.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get override_lame_ttl
my $override_lame_ttl = $firstview->override_lame_ttl();
#Modify override_lame_ttl
$firstview->override_lame_ttl('true');
```

override_max_cache_ttl( )

The override_max_cache_ttl attribute controls whether the max_cache_ttl method values of the member are used, instead of the grid default.

The override_max_cache_ttl attribute can be specified explicitly. It is also set implicitly when max_cache_ttl is set to a defined value.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

Set the parameter to "true" to override the grid-level setting for max_cache_ttl. Set the parameter to "false" to inherit the grid-level setting for max_cache_ttl.

The default value of this parameter is "false".

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Getting override_max_cache_ttl
my $override_max_cache_ttl = $firstview->override_max_cache_ttl( );
#Modifying override_max_cache_ttl
$firstview->override_max_cache_ttl("true");
```

override_max_ncache_ttl( )

The override_max_ncache_ttl attribute controls whether the max_ncache_ttl method values of the member are used, instead of the Grid default.

The override_max_ncache_ttl attribute can be specified explicitly. It is also set implicitly when max_ncache_ttl is set to a defined value.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

Set the parameter to "true" to override the grid-level setting for max_ncache_ttl. Set the parameter to "false" to inherit the Grid-level setting for max_ncache_ttl.

The default value of this parameter is "false".

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Getting override_max_ncache_ttl
my $override_max_ncache_ttl = $firstview->override_max_ncache_ttl( );
#Modifying override_max_ncache_ttl
$firstview->override_max_ncache_ttl("true");
```

scavenging_settings( )

Use this method to set or retrieve the view-level scavenging settings.

Setting this parameter implicitle sets the override_scavenging_settings attribute to "true". Setting the parameter to undefined causes the appliance to use the Grid-level settings and automatically sets the override_scavenging_settings attribute to "false".

Note that when scavenging_settings is set and override_scavenging_settings is set to "false", the last operation takes precedence. Thus the sequence $object->scavenging_settings($recl_settings); $object->override_scavenging_settings("false"); sets override_scavenging_settings to "false", and the sequence $object->override_scavenging_settings("false"); $object->scavenging_settings($recl_settings); sets override_scavenging_settings to "true".

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

The valid value is an Infoblox::Grid::DNS::ScavengingSetting object.

Parameter

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get scavenging_settings value
my $scavenging_settings = $object->scavenging_settings();

#Modify scavenging_settings value
$object->scavenging_settings($scavenging_settings);
```

override_scavenging_settings( )

The override_scavenging_settings attribute controls whether the scavenging_settings method values of the DNS View are used, instead of the Grid default.

The override_scavenging_settings attribute can be specified explicitly. It is also set implicitly when scavenging_settings is set to a defined

value.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

Set the parameter to "true" to override the Grid-level setting for scavenging_settings. Set the parameter to "false" to inherit the Grid-level setting for scavenging_settings.

The default value of this parameter is "false".

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get override_scavenging_settings value
my $override_scavenging_settings = $object->override_scavenging_settings( );

#Modify override_scavenging_settings value
$object->override_scavenging_settings("true");
```

recursion( )

Use this method to enable/disable recursion.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

Specify "true" to set the recursion flag or "false" to deactivate/unset it.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

Example

```
#Get recursion
my $recursion = $firstview->recursion();
#Modify recursion
$firstview->recursion("false");
```

rpz_drop_ip_rule_enabled( )

Use this method to set or retreive the flag that indicates whether the DNS server ignores RPZ-IP triggers with large prefix lengths or not.

Setting this method to a defined value implicitly sets the override_rpz_drop_ip_rule method to "true". Setting the parameter to undefined if rpz_drop_ip_rule_min_prefix_length_ipv4 and rpz_drop_ip_rule_min_prefix_length_ipv6 are undefined causes the appliance to use the Grid default and automatically resets the override_rpz_drop_ip_rule attribute to "false".

Note that when rpz_drop_ip_rule_enabled is set to a defined value and override_rpz_drop_ip_rule is set to "false", the last operation takes precedence. Thus the sequence $object->rpz_drop_ip_rule_enabled("true"); $object->override_rpz_drop_ip_rule("false"); will set override_rpz_drop_ip_rule to "false", and the sequence $object->override_rpz_drop_ip_rule("false"); $object->rpz_drop_ip_rule_enabled("true"); will result in override_rpz_drop_ip_rule="true".

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

Specify 'true' to enable ignoring of RPZ-IP triggers with large prefix lengths or 'false' to disable it. The default value is 'false'.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
# Get attribute value
my $value = $firstview->rpz_drop_ip_rule_enabled();
# Modify attribute value
$firstview->rpz_drop_ip_rule_enabled("true");
```

rpz_drop_ip_rule_min_prefix_length_ipv4( )

Use this method to set or retreive minimum prefix length for IPv4 RPZ-IP triggers.

Setting this method to a defined value implicitly sets the override_rpz_drop_ip_rule method to "true". Setting the parameter to undefined if rpz_drop_ip_rule_enabled and rpz_drop_ip_rule_min_prefix_length_ipv6 are undefined causes the appliance to use the Grid default and automatically resets the override_rpz_drop_ip_rule attribute to "false".

Note that when rpz_drop_ip_rule_min_prefix_length_ipv4 is set to a defined value and override_rpz_drop_ip_rule is set to "false", the last operation takes precedence. Thus the sequence $object->rpz_drop_ip_rule_min_prefix_length_ipv4("20"); $object->override_rpz_drop_ip_rule("false"); will set override_rpz_drop_ip_rule to "false", and the sequence $object->override_rpz_drop_ip_rule("false"); $object->rpz_drop_ip_rule_min_prefix_length_ipv4("20"); will result in override_rpz_drop_ip_rule="true".

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

The valid value is an unsigned integer between 1 and 31 that represents prefix length (in bits) for IPv4 RPZ-IP triggers.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
# Get attribute value
my $value = $firstview->rpz_drop_ip_rule_min_prefix_length_ipv4();
# Modify attribute value
$firstview->rpz_drop_ip_rule_min_prefix_length_ipv4("16");
```

rpz_drop_ip_rule_min_prefix_length_ipv6( )

Use this method to set or retreive minimum prefix length for IPv6 RPZ-IP triggers.

Setting this method to a defined value implicitly sets the override_rpz_drop_ip_rule method to "true". Setting the parameter to undefined if rpz_drop_ip_rule_enabled and rpz_drop_ip_rule_min_prefix_length_ipv4 are undefined causes the appliance to use the Grid default and automatically resets the override_rpz_drop_ip_rule attribute to "false".

Note that when rpz_drop_ip_rule_min_prefix_length_ipv6 is set to a defined value and override_rpz_drop_ip_rule is set to "false", the last operation takes precedence. Thus the sequence $object->rpz_drop_ip_rule_min_prefix_length_ipv6("20"); $object->override_rpz_drop_ip_rule("false"); will set override_rpz_drop_ip_rule to "false", and the sequence $object->override_rpz_drop_ip_rule("false"); $object->rpz_drop_ip_rule_min_prefix_length_ipv6("20"); will result in override_rpz_drop_ip_rule="true".

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

The valid value is an unsigned integer between 1 and 127 that represents prefix length (in bits) for IPv6 RPZ-IP triggers.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
# Get attribute value
my $value = $firstview->rpz_drop_ip_rule_min_prefix_length_ipv6();
# Modify attribute value
$firstview->rpz_drop_ip_rule_min_prefix_length_ipv6("120");
```

rpz_qname_wait_recurse( )

Use this method to enable or disable recursive RPZ lookups.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

Specify "true" to enable recursive RPZ lookups or "false" to disable them. The default value is "false".

Note that if both rpz_qname_wait_recurse and override_rpz_qname_wait_recurse are set to some values, the last operation takes precedence. Thus the sequence $firstview->rpz_qname_wait_recurse("true"); $firstview->override_rpz_qname_wait_recurse("false"); results in inheriting rpz_qname_wait_recurse value from the Grid. And the sequence

$firstview->override_rpz_qname_wait_recurse("false"); $firstview->rpz_qname_wait_recurse("true"); sets override_rpz_qname_wait_recurse to "true".

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
# Get attribute value
my $value = $firstview->rpz_qname_wait_recurse();
# Modify attribute value
$firstview->rpz_qname_wait_recurse("true");
```

response_rate_limiting( )

Use this method to set or retrieve response rate limiting settings for the DNS view.

Setting this method to a defined value implicitly sets the override_response_rate_limiting method to "true". Setting the parameter to undefined causes the appliance to use the Member DNS default and automatically resets the override_response_rate_limiting method to "false".

Note that when response_rate_limiting is set to a defined value and override_response_rate_limiting is set to "false", the last operation takes precedence. Thus the sequence $object->response_rate_limiting($response_rate_limiting); $object->override_response_rate_limiting("false"); will set override_response_rate_limiting to "false", and the sequence $object->override_response_rate_limiting("false"); $object->response_rate_limiting($response_rate_limiting); will result in override_response_rate_limiting="true".

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

The valid value is an Infoblox::Grid::DNS::ResponseRateLimiting object.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get response_rate_limiting
my $response_rate_limiting = $firstview->response_rate_limiting();
#Modify response_rate_limiting
$firstview->response_rate_limiting($response_rate_limiting);
```

use_root_name_servers( )

Use this method to enable/disable custom or Internet root name servers. This setting overrides grid root name server settings.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

Specify "true" to enable custom root name servers. Specify "false" to use Internet root name servers.

Returns

If you specify a parameter, the method returns true when the modification succeeds, and returns false when the operation fails. If you do not specify a parameter, the method returns the attribute value.

Example

```
#Get use_root_name_servers
my $useRootNS = $firstview->use_root_name_servers( );
#Modify use_root_name_servers
$firstview->use_root_name_servers("false");
```

override_nxdomain_redirect( )

The "override_nxdomain_redirect" attribute controls whether the "nxdomain_redirect", "nxdomain_redirect_addresses", "nxdomain_redirect_addresses_ipv6", "nxdomain_redirect_ttl", "nxdomain_log_query", and "nxdomain_rulesets" values for this DNS view are used.

The "override_nxdomain_redirect" attribute can be specified explicitly. It is also set implicitly as follows: "nxdomain_redirect" =>

"true"|"false" or "nxdomain_redirect_addresses" => ["10.0.0.10", "10.0.0.20"]|[] or "nxdomain_redirect_addresses" => ["2001:db8::1", "2002:ffff::1"]|[] result in "override_nxdomain_redirect" => "true", "nxdomain_redirect" => undef and "nxdomain_redirect_addresses" => undef and "nxdomain_redirect_addresses_ipv6" => undef result in "override_nxdomain_redirect" => "false". If "override_nxdomain_redirect" is specified explicitly as "true", and "nxdomain_redirect" => undef, "nxdomain_redirect_addresses" => undef and "nxdomain_redirect_addresses_ipv6" => undef, then the explicit value takes precedence; and "override_nxdomain_redirect"=>"true", "nxdomain_redirect", "nxdomain_redirect_addresses", and "nxdomain_redirect_addresses" remain unchanged in the database.

Specifying the "nxdomain_redirect_ttl", "nxdomain_log_query", or "nxdomain_rulesets" attributes does not affect the "override_nxdomain_redirect" value. It is implicitly set only by "nxdomain_redirect".

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

Set the parameter to "true" to override the member-level settings of the NXDOMAIN redirect attributes. Set the parameter to "false" to inherit the member-level settings. The default is "false".

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get override_nxdomain_redirect
my $override_nxdomain_redirect=$firstview->override_nxdomain_redirect( );
#Modify override_nxdomain_redirect
$firstview->override_nxdomain_redirect("true");
```

nxdomain_redirect( )

Use this method to enable or disable NXDOMAIN redirection in a DNS view.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

Specify "true" to set the nxdomain_redirect flag or "false" to deactivate/unset it. The default is "false".

Setting this parameter to "true" or "false" implicitly sets the override_nxdomain_redirect attribute to "true". Setting the parameter to undefined causes the appliance to use the grid-level NXDOMAIN redirection settings and automatically sets the override_nxdomain_redirect attribute to "false" if nxdomain_redirect_addresses or nxdomain_redirect_addresses_ipv6 is also undefined. But if nxdomain_redirect_addresses or nxdomain_redirect_addresses_ipv6 contains a value, then the override_nxdomain_redirect attribute is set to true.

Note that when nxdomain_redirect is set to "true"/"false" and override_nxdomain_redirect is set to "false", the last operation takes precedence. Thus the sequence $firstview->nxdomain_redirect("false"); $firstview->override_nxdomain_redirect("false"); sets override_nxdomain_redirect to "false", and the sequence $firstview->override_nxdomain_redirect("false"); $firstview->nxdomain_redirect("false"); sets override_nxdomain_redirect to "true".

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Getting nxdomain_redirect
my $nxdomain_redirect = $firstview->nxdomain_redirect( );
#Modifying nxdomain_redirect
$firstview->nxdomain_redirect("true");
```

nxdomain_redirect_addresses( )

Use this method to specify the IP addresses the appliance includes in the response it sends in place of an NXDOMAIN response.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

The valid value is an array of IPv4 addresses. The default value is undefined.

Setting this parameter to a defined value implicitly sets the override_nxdomain_redirect attribute to "true". Setting the parameter to undefined causes the appliance to use the grid-level NXDOMAIN redirection settings and automatically sets the override_nxdomain_redirect attribute to "false" if nxdomain_redirect and nxdomain_redirect_addresses_ipv6 is also undefined. But

if nxdomain_redirect or nxdomain_redirect_addresses_ipv6 contains a value, then the override_nxdomain_redirect attribute is set to true.

Note that when nxdomain_redirect_addresses contains a valid array reference and override_nxdomain_redirect is set to "false", the last operation takes precedence. Thus the sequence $firstview->nxdomain_redirect_addresses(["10.0.0.10", "10.0.0.20"]); $firstview->override_nxdomain_redirect("false"); sets override_nxdomain_redirect to "false", and the sequence $firstview->override_nxdomain_redirect("false"); $firstview->nxdomain_redirect_addresses(["10.0.0.10", "10.0.0.20"]); sets override_nxdomain_redirect to "true".

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get nxdomain_redirect_addresses
my $nxdomain_redirect_addresses = $firstview->nxdomain_redirect_addresses( );
#Modify nxdomain_redirect_addresses
$firstview->nxdomain_redirect_addresses(["10.0.0.10", "10.0.0.20"]);
```

nxdomain_redirect_addresses_ipv6( )

Use this method to specify the IP addresses the appliance includes in the response it sends in place of an NXDOMAIN response.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

The valid value is an array of IPv6 addresses. The default value is undefined.

Setting this parameter to a defined value implicitly sets the override_nxdomain_redirect attribute to "true". Setting the parameter to undefined causes the appliance to use the grid-level NXDOMAIN redirection settings and automatically sets the override_nxdomain_redirect attribute to "false" if nxdomain_redirect and nxdomain_redirect_addresses is also undefined. But if nxdomain_redirect or nxdomain_redirect_addresses contains a value, then the override_nxdomain_redirect attribute is set to true.

Note that when nxdomain_redirect_addresses_ipv6 contains a valid array reference and override_nxdomain_redirect is set to "false", the last operation takes precedence. Thus the sequence $firstview->nxdomain_redirect_addresses_ipv6(["2001:db8::1", "2002:ffff::1"]); $firstview->override_nxdomain_redirect("false"); sets override_nxdomain_redirect to "false", and the sequence $firstview->override_nxdomain_redirect("false"); $firstview->nxdomain_redirect_addresses_ipv6(["2001:db8::1", "2002:ffff::1"]); sets override_nxdomain_redirect to "true".

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get nxdomain_redirect_addresses_ipv6
my $nxdomain_redirect_addresses_ipv6 = $firstview->nxdomain_redirect_addresses_ipv6( );
#Modify nxdomain_redirect_addresses_ipv6
$firstview->nxdomain_redirect_addresses_ipv6(["2001:db8::1", "2002:ffff::1"]);
```

nxdomain_redirect_ttl( )

Use this method to set or retrieve the TTL value of the synthetic DNS responses that result from NXDOMAIN redirection.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

A 32-bit integer, from 0 to 4294967295, that represents the TTL in seconds. The default value is 60.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get nxdomain_redirect_ttl
my $nxdomain_redirect_ttl = $firstview->nxdomain_redirect_ttl( );
#Modify nxdomain_redirect_ttl
$firstview->nxdomain_redirect_ttl(60);
```

nxdomain_log_query( )

Use this method to set or retrieve the flag that indicates whether NXDOMAIN redirection queries are logged.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

Specify "true" to enable logging, or "false" to disable it. The default value is "false".

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get nxdomain_log_query
my $nxdomain_log_query = $firstview->nxdomain_log_query( );
#Modify nxdomain_log_query
$firstview->nxdomain_log_query("true");
```

nxdomain_rulesets( )

Use this method to specify or retrieve Infoblox::DNS::Ruleset object names assigned at the grid level for NXDOMAIN redirection.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

The valid value is an array reference that contains the names of the Infoblox::DNS::Ruleset objects assigned to the grid. The names must be in string format.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get nxdomain_rulesets
my $nxdomain_rulesets = $firstview->nxdomain_rulesets();
#Modify nxdomain_rulesets
$firstview->nxdomain_rulesets(["ruleset1", "ruleset2"]);
```

override_rpz_drop_ip_rule( )

The override_rpz_drop_ip_rule attribute controls whether the rpz_drop_ip_rule_enabled, rpz_drop_ip_rule_min_prefix_length_ipv4 and rpz_drop_ip_rule_min_prefix_length_ipv6 method values of the member are used, instead of the Grid default.

The override_rpz_drop_ip_rule attribute can be specified explicitly. It is also set implicitly when one of rpz_drop_ip_rule_enabled, rpz_drop_ip_rule_min_prefix_length_ipv4 or rpz_drop_ip_rule_min_prefix_length_ipv6 methods is set to a defined value.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

Set the parameter to "true" to override the Grid-level setting for rpz_drop_ip_rule_enabled, rpz_drop_ip_rule_min_prefix_length_ipv4 and rpz_drop_ip_rule_min_prefix_length_ipv6 methods. Set the parameter to "false" to inherit the Grid-level setting for rpz_drop_ip_rule_enabled, rpz_drop_ip_rule_min_prefix_length_ipv4 and rpz_drop_ip_rule_min_prefix_length_ipv6 methods.

The default value of this parameter is "false".

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Getting override_rpz_drop_ip_rule
my $override_rpz_drop_ip_rule = $firstview->override_rpz_drop_ip_rule( );
#Modifying override_rpz_drop_ip_rule
$firstview->override_rpz_drop_ip_rule("true");
```

override_rpz_qname_wait_recurse( )

The override_rpz_qname_wait_recurse attribute controls whether the rpz_qname_wait_recurse method value for the member is used, instead of the Grid default.

The "override_rpz_qname_wait_recurse" attribute can be specified explicitly. It is also established implicitly as follows: "rpz_qname_wait_recurse" => "true"|"false" results in "override_rpz_qname_wait_recurse" => "true"; "rpz_qname_wait_recurse" => undef results in "override_rpz_qname_wait_recurse" => "false"; If "override_rpz_qname_wait_recurse" is specified explicitly as "true", while "rpz_qname_wait_recurse"=>undef, the explicit value takes precedence, and "override_rpz_qname_wait_recurse"=>"true" while "rpz_qname_wait_recurse" remains unchanged in the database.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

Set the parameter to "true" to override the Grid-level setting for rpz_qname_wait_recurse. Set the parameter to "false" to inherit the Grid-level setting for rpz_qname_wait_recurse value.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Getting override_rpz_qname_wait_recurse
my $override_rpz_qname_wait_recurse = $firstview->override_rpz_qname_wait_recurse();
#Modyfying override_rpz_qname_wait_recurse
$firstview->override_rpz_qname_wait_recurse("true");
```

override_response_rate_limiting( )

The override_response_rate_limiting attribute controls whether the response_rate_limiting method values of the member are used, instead of the Grid Member default.

The override_response_rate_limiting attribute can be specified explicitly. It is also set implicitly when response_rate_limiting is set to a defined value.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

Set the parameter to "true" to override the member-level setting for response_rate_limiting. Set the parameter to "false" to inherit the member-level setting for response_rate_limiting.

The default value of this parameter is "false".

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get override_response_rate_limiting
my $override_response_rate_limiting = $firstview->override_response_rate_limiting( );
#Modify override_response_rate_limiting
$firstview->override_response_rate_limiting("true");
```

override_sortlist( )

The override_sortlist attribute controls whether the sort list value in this DNS view is used, instead of the member default.

The override_sortlist attribute can be specified explicitly. It is also set implicitly when the sort list is set to a defined value.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

Set the parameter to "true" to override the member-level setting for the sort list. Set the parameter to "false" to inherit the member-level setting for the sort list.

The default value of this parameter is "false".

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Getting override_sortlist
my $override_sortlist=$firstview->override_sortlist( );
#Modifying override_sortlist
$firstview->override_sortlist("true");
```

sortlist( )

Use this method to create a sort list.

A sort list determines the order of addresses in responses made to DNS queries.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Setting this method to a defined value implicitly sets the override_sortlist method to "true". Setting the parameter to undefined causes the appliance to use the member default and automatically resets the override_sortlist attribute to "false".

Note that when the sort list is set to a defined value and override_sortlist is set to "false", the last operation takes precedence. Thus the sequence $object->sortlist([$s1,$s2]); $object->override_sortlist("false"); will set override_sortlist to "false", and the sequence $object->override_sortlist("false"); $object->sortlist([$s1,$s2]); will result in override_sortlist="true".

Parameter

Valid value is an array reference that contains Infoblox::DNS::Sortlist object(s).

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
my $sort1 = Infoblox::DNS::Sortlist->new (
                        "source_ipv4addr" => "1.2.3.4",
                        "match_list"      => [ "10.20.1.0/24" , "10.20.2.0/24"]
                );
$firstview->sortlist( [ $sort1 ] );
```

SAMPLE CODE

The following sample code demonstrates different operations that can be applied to an object, such as create, search, modify, and remove. This sample code also includes error handling for the operations.

#Preparation prior to a DNS view insertion

```
 use strict;
 use Infoblox;

#Create a session to the Infoblox appliance
my $session = Infoblox::Session->new(
    master   => "192.168.1.2",
    username => "admin",
    password => "infoblox"
);
unless ($session) {
   die("Construct session failed: ",
       Infoblox::status_code() . ":" . Infoblox::status_detail());
}
print "Session created successfully\n";
```

#Create a DNS view

```
#create member
my $member1 = Infoblox::DNS::Member->new(
    name     => "member.com",
    ipv4addr => "192.168.1.2",
    lead     => "false",
    stealth  => "false"
);

#create keys
```

```
my $key1 = Infoblox::DNS::TSIGKey->new(
    name => "tsig_xfer",
    key  => ":2xCOMPAT"
);

my $tsig_key2 = $session->gen_tsig_key();

my $key2 = Infoblox::DNS::TSIGKey->new(
    name => "key2",
    key  => $tsig_key2
);

#create View object using members
my $firstview = Infoblox::DNS::View->new(
    name              => "internal",
    match_clients     => ["1.1.1.1", $key1, "10.0.0.0/8", $key2],
    recursion         => "true",
    disable           => "false",
    comment           => "new Internal view"
    );
 unless ($firstview) {
          die("constructor for view internal failed: ",
               Infoblox::status_code() . ":" . Infoblox::status_detail());
     }

#put View into session
$session->add($firstview);
unless ($firstview) {
      die("add view internal failed: ",
      $session->status_code() . ":" . $session->status_detail());
}
```

#Search for a specific view

```
#Searching the internal view
my @result = $session->search(object => "Infoblox::DNS::View", name => "^internal\$");
my $object = $result[0];
unless ($object) {
          die("Search view failed: ",
               $session->status_code() . ":" . $session->status_detail());
     }
print "Search view object found at least 1 matching entry\n";
```

#Get and modify a DNS view

```
my @result = $session->get(
    object => "Infoblox::DNS::View",
    name   => "internal"
);

my $firstview = $result[0];
unless ($firstview) {
         die("Get view failed: ",
           $session->status_code() . ":" . $session->status_detail());
}
print "Get view found at least 1 matching entry\n";
$firstview->comment("new comment internal view");

#Apply the changes
$session->modify($firstview);
unless ($firstview) {
          die("Modify view failed: ",
               $session->status_code() . ":" . $session->status_detail());
     }
print "View object modified successfully \n";
```

#Remove a view

```
my @result = $session->get(
    object => "Infoblox::DNS::View",
    name   => "internal"
);

my $firstview = $result[0];
unless ($firstview) {
         die("Get view failed: ",
```

```
            $session->status_code() . ":" . $session->status_detail());
}
print "Get view found at least 1 matching entry\n";

#submit the object for removal
$session->remove($firstview);
unless ($firstview) {
          die("Remove view failed: ",
            $session->status_code() . ":" . $session->status_detail());
}
print "View object removed successfully \n";

####PROGRAM ENDS####
```

AUTHOR

Infoblox Inc. http://www.infoblox.com/

SEE ALSO

Infoblox::DNS::View, Infoblox::DHCP::View, Infoblox::DNS::Zone, Infoblox::DNS::DnssecTrustedKey, Infoblox::DNS::Ruleset, Infoblox::Session, Infoblox::Session->get(), Infoblox::Session->search(), Infoblox::Session->add(), Infoblox::Session->remove(), Infoblox::Session->modify(), Infoblox::DNS::Sortlist, Infoblox::Grid::NamedACL, Infoblox::Grid::DNS::FixedRRSetOrderFQDN, Infoblox::Grid::DNS::ResponseRateLimiting

COPYRIGHT

Copyright (c) 2017 Infoblox Inc.

Infoblox::DNS::DDNS::PrincipalCluster::Group - A DDNS Principal Cluster Group object.

- NAME
- DESCRIPTION
- CONSTRUCTOR
- SESSION METHODS
  - Infoblox::Session->add( )
  - Infoblox::Session->get( )
  - Infoblox::Session->modify( )
  - Infoblox::Session->remove( )
  - Infoblox::Session->search( )
- MODULE METHODS
  - Infoblox::Grid::DNS->ddns_principal_group( )
  - Infoblox::DNS::Zone->ddns_principal_group( )
  - Infoblox::DNS::View->ddns_principal_group( )
- METHODS
  - clusters( )
  - comment( )
  - name( )
- AUTHOR
- SEE ALSO
- COPYRIGHT

NAME

Infoblox::DNS::DDNS::PrincipalCluster::Group - A DDNS Principal Cluster Group object.

DESCRIPTION

The DDNS Principal Cluster Group object represents a set of DDNS Principal Clusters. A single group can be active at any time.

CONSTRUCTOR

```
my $cluster = Infoblox::DNS::DDNS::PrincipalCluster::Group->new(
    name     => $string,                    # Required
    comment  => $string,                    # Optional / Default is undefined
);
```

SESSION METHODS

This section describes all the methods in a Session module that you can apply to a DDNS Principal Cluster Group object.

Infoblox::Session->add( )

Use this method to add an object to the Infoblox appliance. See Infoblox::Session->add() for parameters and return values.

Example

```
#construct an object
my $object = Infoblox::DNS::DDNS::PrincipalCluster::Group->new(
   name  => 'cluster1',
);

#submit for addition
my $response = $session->add($object);
```

Infoblox::Session->get( )

Use this method to retrieve all the matching objects from the Infoblox appliance. See Infoblox::Session->get() for parameters and return values.

Key Reference

```
Apply the following attribute to get a specific object:

 name    - Optional. A DDNS Principal Cluster Group name in a string format.
 comment - Optional. A DDNS Principal Cluster Group comment.
```

Example

```
my @retrieved_objs = $session->get(
    object => 'Infoblox::DNS::DDNS::PrincipalCluster::Group',
    name   => 'cluster1',
);
```

Infoblox::Session->modify( )

Use this method to modify an object in the Infoblox appliance. See Infoblox::Session->modify() for parameters and return values.

Example

```
#Use method to modify the comment.
$object->comment('this is a modified comment');
#Submit modification
my $response = $session->modify($object);
```

Infoblox::Session->remove( )

Use this method to remove an object from the Infoblox appliance. See Infoblox::Session->remove() for parameters and return values.

To remove a specific object, first use `get()` or `search()` to retrieve the specific object, and then submit this object for removal.

Example

```
#Get the objects with the same name
my @retrieved_objs = $session->get(
    object => 'Infoblox::DNS::DDNS::PrincipalCluster::Group',
    name   => 'cluster1',
);
#Find the desired object from the retrieved list.
my $desired_cluster = $retrieved_objs[0];
#Submit for removal
my $response = $session->remove($desired_cluster);
```

Infoblox::Session->search( )

Use this method to search for objects in the Infoblox appliance. See Infoblox::Session->search() for parameters and return values.

Key Reference

```
Apply the following attribute to get a specific object:

 name    - Optional. A DDNS Principal Cluster Group name in a string format.
 comment - Optional. A DDNS Principal Cluster Group comment.
```

Example

```
my @retrieved_objs = $session->get(
    object => 'Infoblox::DNS::DDNS::PrincipalCluster::Group',
    name   => 'group1',
);
```

MODULE METHODS

Infoblox::Grid::DNS->ddns_principal_group( )

Use this method to set or retrieve the DDNS Principal Cluster Group. See Infoblox::Grid::DNS->ddns_principal_group() for parameters and return values.

Example

```
#get ddns_principal_group value
my $ddns_principal_group = $object->ddns_principal_group();

#modify ddns_principal_group value
$object->ddns_principal_group($group1);
```

Infoblox::DNS::Zone->ddns_principal_group( )

Use this method to set or retrieve the DDNS Principal Cluster Group. See Infoblox::DNS::Zone->ddns_principal_group() for parameters and return values.

Example

```
#get ddns_principal_group value
my $ddns_principal_group = $object->ddns_principal_group();
```

```
#modify ddns_principal_group value
$object->ddns_principal_group($group1);
```

Infoblox::DNS::View->ddns_principal_group( )

Use this method to set or retrieve the DDNS Principal Cluster Group. See Infoblox::DNS::View->ddns_principal_group() for parameters and return values.

Example

```
#get ddns_principal_group value
my $ddns_principal_group = $object->ddns_principal_group();

#modify ddns_principal_group value
$object->ddns_principal_group($group1);
```

METHODS

This section describes all the methods that you can use to set or retrieve the attribute values of the object.

clusters( )

Use this method to retrieve a list of equivalent DDNS principal clusters. This is a read-only attribute.

Omit the parameter to retrieve the attribute value.

Parameter

None

Returns

The valid return value is array of an Infoblox::DNS::DDNS::PrincipalCluster objects.

Example

```
#get clusters value
my $clusters = $object->clusters();
```

comment( )

Use this method to set or retrieve a descriptive comment.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

The valid value is a comment in string format (UTF-8) with a maximum of 256 bytes.

Returns

If you specified a parameter, the method returns 'true' when the modification succeeds, and returns 'false' when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#get comment value
my $comment = $object->comment();

#modify comment value
$object->comment('desired comment');
```

name( )

Use this method to set or retrieve the DDNS Principal Cluster Group name.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

Desired name in string format with a maximum of 256 bytes.

Returns

If you specified a parameter, the method returns 'true' when the modification succeeds, and returns 'false' when the operation

fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#get name value
my $name = $object->name();

#modify name value
$object->name('foo');
```

AUTHOR

Infoblox Inc. http://www.infoblox.com/

SEE ALSO

Infoblox::Session, Infoblox::Session->add(), Infoblox::Session->get(), Infoblox::Session->modify(), Infoblox::Session->remove(), Infoblox::Session->search(), Infoblox::Grid::DNS, Infoblox::Grid::DNS->ddns_principal_group(), Infoblox::DNS::Zone, Infoblox::DNS::Zone->ddns_principal_group(), Infoblox::DNS::View, Infoblox::DNS::View->ddns_principal_group(),

---

COPYRIGHT

Copyright (c) 2017 Infoblox Inc.

Infoblox::Grid::DNS - DNS objects management and configuration

- NAME
- DESCRIPTION
- CONSTRUCTOR
- SESSION METHODS
  - Infoblox::Session->get( )
  - Infoblox::Session->modify( )
- METHODS
  - allow_bulkhost_ddns( )
  - allow_gss_tsig_zone_updates( )
  - allow_query( )
  - allow_recursive_query( )
  - allow_transfer( )
  - allow_update( )
  - anonymize_response_logging( )
  - attack_mitigation( )
  - auto_blackhole( )
  - bind_check_names_policy( )
  - bind_hostname_directive( )
  - blackhole_list( )
  - blacklist_action( )
  - blacklist_log_query( )
  - blacklist_redirect_addresses( )
  - blacklist_redirect_ttl( )
  - blacklist_rulesets( )
  - bulk_host_name_template( )
  - bulk_host_name_templates( )
  - capture_dns_queries_on_all_domains( )
  - client_subnet_domains( )
  - client_subnet_ipv4_prefix_length( )
  - client_subnet_ipv6_prefix_length( )
  - copy_xfer_to_notify( )
  - ddns_force_creation_timestamp_update( )
  - ddns_restrict_patterns_list( )
  - ddns_restrict_patterns( )
  - ddns_restrict_static( )
  - ddns_restrict_protected( )
  - ddns_restrict_secure( )
  - ddns_principal_tracking( )
  - ddns_principal_group( )
  - default_ttl( )
  - disable_edns( )
  - dns_cache_acceleration_ttl( )
  - dns_health_check_anycast_control( )
  - dns_health_check_domain_list( )
  - dns_health_check_interval( )
  - dns_health_check_recursion_flag( )
  - dns_health_check_retries( )
  - dns_health_check_timeout( )
  - dns_query_capture_file_time_limit( )
  - dns64_groups( )
  - dnssec_enabled( )
  - dnssec_enable_ksk_auto_rollover( )
  - dnssec_expired_signatures_enabled( )
  - dnssec_ksk_algorithms( )
  - dnssec_ksk_email_notification_enabled( )
  - dnssec_ksk_rollover_interval()
  - dnssec_ksk_rollover_notification_config( )
  - dnssec_ksk_snmp_notification_enabled( )
  - dnssec_negative_trust_anchors( )
  - dnssec_nsec3_iterations( )
  - dnssec_nsec3_salt_min_length( )
  - dnssec_nsec3_salt_max_length( )
  - dnssec_signature_expiration( )
  - dnssec_validation_enabled( )
  - dnssec_blacklist_enabled()
  - dnssec_dns64_enabled()
  - dnssec_nxdomain_enabled()
  - dnssec_rpz_enabled()
  - dnssec_trusted_keys( )
  - dnssec_zsk_algorithms( )
  - dnssec zsk rollover interval()

- dnssec_zsk_rollover_mechanism( )
- domains_to_capture_dns_queries( )
- dtc_dnssec_mode( )
- dtc_edns_prefer_client_subnet( )
- dtc_topology_ea_list( )
- email( )
- enable_blackhole( )
- enable_blacklist( )
- enable_capture_dns_queries( )
- enable_capture_dns_responses( )
- enable_client_subnet_recursive( )
- enable_client_subnet_forwarding( )
- enable_delete_associated_ptr( )
- enable_dns_health_check( )
- enable_dns64( )
- enable_dtc_dns_fall_through( )
- enable_excluded_domain_names( )
- enable_fixed_rrset_order_fqdns( )
- enable_ftc( )
- enable_gss_tsig( )
- enable_host_rrset_order( )
- enable_hsm_signing( )
- enable_query_rewrite( )
- excluded_domain_names( )
- expire_after( )
- file_transfer_setting( )
- filter_aaaa( )
- filter_aaaa_list( )
- forward_only( )
- forward_updates( )
- forwarders( )
- fixed_rrset_order_fqdns( )
- ftc_expired_record_ttl( )
- ftc_expired_record_timeout( )
- gss_tsig_keys( )
- host_name_restriction_policy( )
- lame_ttl( )
- logging_categories( )
- logging_facility( )
- max_cache_ttl( )
- max_ncache_ttl( )
- max_cached_lifetime( )
- member_secondary_notify( )
- negative_ttl( )
- next_secure_type( )
- notify_delay( )
- nxdomain_redirect( )
- nxdomain_redirect_addresses( )
- nxdomain_redirect_addresses_ipv6( )
- nxdomain_redirect_ttl( )
- nxdomain_log_query( )
- nxdomain_rulesets( )
- notify_source_port( )
- nsgroup_default( )
- nsgroups( )
- preserve_host_rrset_order_on_secondaries( )
- query_source_port( )
- query_rewrite_prefix( )
- query_rewrite_domain_names( )
- scavenging_settings( )
- record_name_policies( )
- recursive_query_list( )
- refresh_timer( )
- resolver_query_timeout( )
- response_rate_limiting( )
- restart_setting( )
- retry_timer( )
- rootNS( )
- serial_query_rate( )
- server_id_directive( )
- sortlist( )
- store_locally( )
- transfer_excluded_servers( )
- transfer_format( )

    - transfers_in( )
    - transfers_out( )
    - transfers_per_ns( )
    - rpz_disable_nsdname_nsip( )
    - rpz_drop_ip_rule_enabled( )
    - rpz_drop_ip_rule_min_prefix_length_ipv4( )
    - rpz_drop_ip_rule_min_prefix_length_ipv6( )
    - rpz_qname_wait_recurse( )
    - zone_deletion_double_confirm( )
- SAMPLE CODE
- AUTHOR
- SEE ALSO
- COPYRIGHT

NAME

Infoblox::Grid::DNS - DNS objects management and configuration

DESCRIPTION

This class supports DNS service management and configuration such as time-to-live (TTL) settings, zone transfers, queries, root name servers, dynamic updates, sort lists, Transaction Signatures (TSIG) for DNS and others, all at the grid level.

The service configurations of a grid are inherited by all members, zones, and networks unless you specifically override them for selected members, zones, and networks. For this reason, it is recommended that you configure services at the grid level before configuring member, zone and network services.

CONSTRUCTOR

The Infoblox::Grid::DNS object is automatically generated upon the successful installation of the Infoblox appliance. It does not require manual construction.

SESSION METHODS

This section describes all the methods in the Infoblox::Session module that you can apply to a grid DNS object.

Infoblox::Session->get( )

Use this method to retrieve the matching grid DNS object from the Infoblox appliance. See Infoblox::Session->get() for parameters and return values.

Key Reference

```
Apply the following attribute to get a specific grid DNS object:

 name - Required. A grid name in string format.
```

Example

```
my @retrieved_objs = $session->get(
    object => "Infoblox::Grid::DNS",
    name   => "Infoblox"
    );
```

Infoblox::Session->modify( )

Use this method to modify a grid DNS object in the Infoblox appliance. See Infoblox::Session->modify() for parameters and return values.

If you have applied an access control list to an object, you must restart services after you update the access control list for the changes to take effect. See Infoblox::Session->restart() method.

Example

```
#Use this method to modify the grid  DNS service.
$grid_dns->allow_recursive_query("true");
#Submit modification
my $response = $session->modify( $grid_dns );
```

METHODS

This section describes all the methods that you can use to configure and retrieve the attribute values of a grid DNS object.

allow_bulkhost_ddns( )

Use this method to set or retrieve the allow_bulkhost_ddns flag of a grid DNS object.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

Specify "true" to set the allow_bulkhost_ddns flag or "false" to deactivate/unset it. The default value is "false".

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get  the allow_bulkhost_ddns flag
my $allow_bulkhost_ddns = $grid_dns->allow_bulkhost_ddns();
#Modify allow bulkhost ddns flag
$grid_dns->allow_bulkhost_ddns("true");
```

allow_gss_tsig_zone_updates( )

Use this method to set or retrieve the allow_gss_tsig_zone_updates flag of a grid.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

Specify "true" to set the flag or "false" to deactivate/unset it. The default value is "false".

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get  the allow_gss_tsig_zone_updates flag
my $allow_gss_tsig_zone_updates = $grid_dns->allow_gss_tsig_zone_updates();
#Modify allow_gss_tsig_zone_updates flag
$grid_dns->allow_gss_tsig_zone_updates("true");
```

allow_query( )

Use this method to set or retrieve the list of allowed/denied queries for a Grid DNS object. The list specifies IP addresses and/or networks from which queries are allowed or denied. The appliance can also use TSIG keys to authenticate the queries.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

Valid value is an Infoblox::Grid::NamedACL object or an array reference that contains IPv4/IPv6 addresses, networks and/or Infoblox::DNS::TSIGKey objects. To deny queries, add "!" as a prefix to addresses and networks. Specify "any" to allow queries from any address. By default, queries are allowed from any address.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get allow query list
my $allow_query_list = $grid_dns->allow_query();
#Modify allow query list
$grid_dns->allow_query(["any"]);

#Specifying TSIG keys
my $tsig_key1 = $session->gen_tsig_key();
my $query_key = Infoblox::DNS::TSIGKey->new(
    name => "query_key1",
    key  => $tsig_key1
);
#Modify allow query list
```

```
 $grid_dns->allow_query(["10.0.0.10" , "20.0.0.0/24" ,"2001::123a","2003::222/64", "!"."30.0.0.30",
$query_key, "!"."40.10.0.0/24","!"."2004::125b","!"."2006::34bc"]);
 #Use Infoblox::Grid::NamedACL
 $grid_dns->allow_query($nacl);
```

allow_recursive_query( )

Use this method to set or retrieve the allow_recursive_query flag of a grid DNS object. If enabled and the Infoblox appliance receives a query for DNS data, it sends a recursive query to the configured internal root servers. If there are no internal root servers configured, the appliance then sends a recursive query to the Internet root servers.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

Specify "true" to set the allow_recursive_query flag or "false" to deactivate/unset it. The default value is "false".

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
 #Get allow recursive query flag
 my $allow_recursive_query = $grid_dns->allow_recursive_query();
 #Modify allow recursive query flag
 $grid_dns->allow_recursive_query("true");
```

allow_transfer( )

Use this method to set or retrieve the list of allowed/denied zone transfers for a Grid DNS object. The list specifies IP addresses and/or networks from which zone transfers are allowed or denied. It is also possible to specify the TSIG keys of clients that are allowed to perform zone transfers. TSIG keys are used to authenticate zone transfer requests and replies. The same key name and key value must be on the primary and secondary name servers for TSIG-authenticated zone transfers to occur.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

Valid value is an Infoblox::Grid::NamedACL object or an array reference that contains IPv4/IPv6 addresses, networks, and/or Infoblox::DNS::TSIGKey objects. To deny zone transfer, add "!" as a prefix to IP addresses and/or networks. Specify "any" to allow transfers from any IP address.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
 #Get allow transfer list
 my $allow_transfer_list = $grid_dns->allow_transfer();
 #Modify allow transfer list
 $grid_dns->allow_transfer(["any"]);

 #Specifying TSIG keys
 my $tsig_key1 = $session->gen_tsig_key();
 my $key1 = Infoblox::DNS::TSIGKey->new(
     name => "key1",
     key  => $tsig_key1
 );
 my $key2 = Infoblox::DNS::TSIGKey->new(
     name => "tsig_xfer",
     key  => ":2xCOMPAT"
     );
 #Modify allow transfer list
 $grid_dns->allow_transfer(["10.0.0.10", "20.0.0.0/24","2004::325f","2007::876c/64","!"."30.0.0.30",
$key1, $key2, "!"."40.10.0.0/24"]);
 #Use Infoblox::Grid::NamedACL
 $grid_dns->allow_transfer($nacl);
```

allow_update( )

Use this method to set or retrieve the list of allowed/denied dynamic updates for a grid DNS object. The list specifies from which IP addresses, networks or TSIG key authenticated hosts dynamic updates are allowed or denied.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

Valid value is an Infoblox::Grid::NamedACL object or an array reference that contains IPv4/IPv6 addresses, networks, and/or Infoblox::DNS::TSIGKey objects. To deny updates, add "!" as a prefix to IP addresses and/or networks. Specify "any" to allow updates from any IP address.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get allow update list
my $allow_update_list = $grid_dns->allow_update();
#Modify allow update list
$grid_dns->allow_update(["any"]);

#Creating a TSIG key object.
my $update_key1 = $session->gen_tsig_key();

my $up_key1 = Infoblox::DNS::TSIGKey->new(
    name => "update_key",
    key  => $update_key1
    );

#Modify allow update list
$grid_dns->allow_update(["10.0.0.10", "20.0.0.0/24","2008::129b","!"."30.0.0.30", $up_key1,
"!"."40.10.0.0/24","!"."2004::76bc/64"]);
#Use Infoblox::Grid::NamedACL
$grid_dns->allow_update($nacl);
```

anonymize_response_logging( )

Use this method to set or retrieve the flag that indicates whether the anonymization of captured DNS responses is enabled or disabled.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

Specify 'true' to enable anonymization of captured DNS response or 'false' to disable it. The default value is 'false'.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get anonymize_response_logging value
my $anonymize_response_logging = $object->anonymize_response_logging();

#Modify anonymize_response_logging value
$object->anonymize_response_logging('true');
```

attack_mitigation( )

Use this method to set or retrieve mitigation settings for DNS attacks.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

The valid value is an Infoblox::Grid::DNS::AttackMitigation object.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get attack_mitigation value
my $attack_mitigation   $grid_dns  attack_mitigation
```

```
                          =          ->                    ();
#Modify attack_mitigation
$grid_dns->attack_mitigation($attack_mitigation);
```

auto_blackhole( )

Use this method to set or retrieve the auto blackhole settings.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

The valid value is an Infoblox::Grid::DNS::AutoBlackHole object.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get auto_blackhole value
my $auto_blackhole = $grid_dns->auto_blackhole();
#Modify auto_blackhole
$grid_dns->auto_blackhole($auto_blackhole);
```

bind_check_names_policy( )

Use this method to set or retrieve the BIND check names policy, which indicates the action the appliance takes when it encounters host names that do not comply with the Strict Hostname Checking policy. This method applies only if the host name restriction policy is set to "Strict Hostname Checking".

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

The allowed values of the parameter are "fail", "warn" or undefined. This indicates the desired BIND behavior in the case of illegal hostnames. The default value is undefined, which means the appliance ignores illegal hostnames.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get bind_check_names_policy value
my $names_policy = $grid_dns->bind_check_names_policy();
#Modify bind_check_names_policy
$grid_dns->bind_check_names_policy("warn");
```

bind_hostname_directive( )

Use this method to set or retrieve the value of BIND 'hostname' directive.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

The valid values are "NONE" and "HOSTNAME". The default value is "NONE".

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get bind_hostname_directive
my $bind_hostname_directive = $grid_dns->bind_hostname_directive();
#Modify bind_hostname_directive
$grid_dns->bind_hostname_directive("HOSTNAME");
```

blackhole_list( )

Use this method to set or retrieve the blackhole list of a grid DNS object. This list specifies the IP addresses and networks from which the DNS queries are blocked.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

A valid value is an Infoblox::Grid::NamedACL object or an array reference that contains IPV4/IPv6 addresses and networks. To allow queries, add "!" as the prefix to the IP addresses and networks. Specify "any" to block DNS queries from any IP address.

Returns

When you specify a parameter and the modification succeeds, the method returns "true". When the operation fails, the method returns "false".

When you do not specify a parameter, the method returns the attribute value.

Example

```
#Get the blackhole list
my $blackhole_list = $grid_dns->blackhole_list();
#Modify the blackhole_list
$grid_dns->blackhole_list(["any"]);
#Modify the blackhole list
$grid_dns->blackhole_list(["10.0.0.10",
"20.0.0.0/24","2010::256/64","!"."2011::23cb/64","!"."30.0.0.30", "!"."40.10.0.0/24"]);
#Modify the blackhole list
$grid_dns->blackhole_list(undef);
#Use Infoblox::Grid::NamedACL
$grid_dns->blackhole_list($nacl);
```

blacklist_action( )

Use this method to specify the action to perform when a domain name matches the pattern defined in a rule that is specified by the blacklist_ruleset method.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

Valid values are "REDIRECT" or "REFUSE". The default value is "REFUSE".

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

Examples

```
#Get blacklist_action
my $blacklist_action = $grid_dns->blacklist_action();
#Modify blacklist_action
$grid_dns->blacklist_action("REDIRECT");
```

blacklist_log_query( )

Use this method to set or retrieve the flag that indicates whether blacklist redirection queries are logged.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

Specify "true" to enable logging, or "false" to disable it. The default value is "false".

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get blacklist_log_query
my $blacklist_log_query = $grid_dns->nxdomain_log_query( );
#Modify blacklist_log_query
$grid_dns->blacklist_log_query("true");
```

blacklist_redirect_addresses( )

Use this method to specify the IP addresses the appliance includes in the response it sends in place of a blacklisted IP address.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

The valid value is an array of IPv4 addresses. The default value is undefined.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get blacklist_redirect_addresses
my $blacklist_redirect_addresses = $grid_dns->blacklist_redirect_addresses( );
#Modify blacklist_redirect_addresses
$grid_dns->blacklist_redirect_addresses(["10.0.0.10", "10.0.0.20"]);
```

blacklist_redirect_ttl( )

Use this method to set or retrieve the TTL value of the synthetic DNS response that result from blacklist redirection.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

A 32-bit integer, from 0 to 4294967295, that represents the TTL in seconds. The default value is 60.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get blacklist_redirect_ttl
my $blacklist_redirect_ttl = $grid_dns->blacklist_redirect_ttl( );
#Modify blacklist_redirect_ttl
$grid_dns->blacklist_redirect_ttl(60);
```

blacklist_rulesets( )

Use this method to specify or retrieve Infoblox::DNS::Ruleset object names assigned at the grid level for blacklist redirection.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

The valid value is an array reference that contains the names of the Infoblox::DNS::Ruleset objects assigned to the grid. The names must be in string format.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get blacklist_rulesets
my $blacklist_rulesets = $grid_dns->blacklist_rulesets();
#Modify blacklist_rulesets
$grid_dns->blacklist_rulesets(["ruleset1", "ruleset2"]);
```

bulk_host_name_template( )

Use this method to set or retrieve the default bulk host name format of a grid DNS object.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

Name of a valid bulk host name format.

For instance, if the prefix of a bulk host is *info*, the IP address range is *213.19.32.133-213.19.32.134*, and the domain name is *infoblox.com*. If the bulk host name template "Three Octets" is set at the grid level, then the bulk host names will be *info-213-19-*

23-133.infoblox.com and *info-213-19-23-134.infoblox.com*.

If the bulk host name template specified is "CustomerTemplate"(*#3*#4), the bulk host names will be *info*023*133.infoblox.com* and *info-023-134.infoblox.com*.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get the bulk host name format
my $grid_template = $grid_dns->bulk_host_name_template();
#Modify bulk host name format
$grid_dns->bulk_host_name_template("Three Octets");
```

bulk_host_name_templates( )

Use this method to set or retrieve the list of bulk host name templates.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

Valid value is an array reference that contains Infoblox::Grid::DNS::BulkHostNameTemplate object(s).

There are four Infoblox predefined bulk host name templates.

Template Name Template Format

"Four Octets" -$1-$2-$3-$4

"Three Octets" -$2-$3-$4

"Two Octets" -$3-$4

"One Octet" -$4

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get bulk_host_name_templates
my $bulk_host_name_templates = $grid_dns->bulk_host_name_templates();

#Modify bulk_host_name_templates

#Constructing four out-of-box bulk host name templates
my $four_octets = Infoblox::Grid::DNS::BulkHostNameTemplate->new(
    template_name   => "Four Octets",
    template_format => '-$1-$2-$3-$4'
);
my $three_octets = Infoblox::Grid::DNS::BulkHostNameTemplate->new(
    template_name   => "Three Octets",
    template_format => '-$2-$3-$4'
);
my $two_octets = Infoblox::Grid::DNS::BulkHostNameTemplate->new(
    template_name   => "Two Octets",
    template_format => '-$3-$4'
);
my $one_octet = Infoblox::Grid::DNS::BulkHostNameTemplate->new(
    template_name   => "One Octet",
    template_format => '-$4'
);

#Creating a new bulk host name templates.
my $custom_template = Infoblox::Grid::DNS::BulkHostNameTemplate->new(
    template_name   => "Custom Template",
    template_format => '-#1-#2-#3-#4'
);

$grid_dns->bulk_host_name_templates([$custom_template, $four_octets, $three_octets, $two_octets,
$one_octet]);
```

capture_dns_queries_on_all_domains( )

Use this method to set or retrieve the flag that indicates whether the capturing of DNS queries for all domains is enabled or disabled.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

Specify 'true' to enable capturing of DNS queries for all domains or 'false' to disable it. The default value is 'false'.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get capture_dns_queries_on_all_domains value
my $capture_dns_queries_on_all_domains = $object->capture_dns_queries_on_all_domains();

#Modify capture_dns_queries_on_all_domains value
$object->capture_dns_queries_on_all_domains('true');
```

client_subnet_domains( )

Use this method to set or retrieve the list of zone domain names that are allowed or forbidden for EDNS client subnet (ECS) recursion.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

The valid value is an array of Infoblox::Grid::DNS::ClientSubnetDomain objects.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get client_subnet_domains value
my $value = $grid_dns->client_subnet_domains();

#Construct Infoblox::Grid::DNS::ClientSubnetDomain object
my $object = Infoblox::Grid::ClientSubnetDomain->new(
    domain     => 'domain.com',
    permission => 'DENY',
);

#Modify client_subnet_domains value
$grid_dns->client_subnet_domains([$object]);
```

client_subnet_ipv4_prefix_length( )

Use this method to set or retrieve the default IPv4 source prefix length used when sending queries with EDNS client subnet option.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

The valid value is an unsigned integer between 1 and 32.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get client_subnet_ipv4_prefix_length value
my $value = $object->client_subnet_ipv4_prefix_length();

#Modify client_subnet_ipv4_prefix_length value
$object->client_subnet_ipv4_prefix_length(30);
```

client_subnet_ipv6_prefix_length( )

Use this method to set or retrieve the default IPv6 source prefix length used when sending queries with EDNS client subnet option.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

The valid value is an unsigned integer between 1 and 128.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get client_subnet_ipv6_prefix_length value
my $value = $object->client_subnet_ipv6_prefix_length();

#Modify client_subnet_ipv6_prefix_length value
$object->client_subnet_ipv6_prefix_length(30);
```

copy_xfer_to_notify( )

Use this method to enable/disable copying of the allowed IPs from zone transfer list into also-notify statement in named.conf.

Parameter

Set the parameter to "true" to enable copying of the allowed IPs from zone transfer list into also-notify statement in named.conf. Set the parameter to "false" to disable it.

The default value of the parameter is "false".

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
# Get attribute value
my $value = $grid_dns->copy_xfer_to_notify();
# Modify attribute value
$grid_dns->copy_xfer_to_notify('true');
```

ddns_force_creation_timestamp_update( )

Use this method to set or retrieve the flag that indicates whether the creation timestamp of resource records (RR) should be updated when DDNS update happens even if there is no change to the RR.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

Specify 'true' to update the creation timestamp of RR on every DDNS update, and 'false' to update the creation timestamp only when RR changes during a DDNS update. The default is 'false'.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get ddns_force_creation_timestamp_update value
$value = $object->ddns_force_creation_timestamp_update();

#modify ddns_force_creation_timestamp_update value
$object->ddns_force_creation_timestamp_update('true');
```

ddns_restrict_patterns_list( )

Use this method to set or retrieve an unordered list of restriction patterns for an option of to restrict DDNS updates based on FQDN patterns.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

The valid value is an array of FQDNs in a string format.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#get ddns_restrict_patterns_list value
$ddns_restrict_patterns_list = $grid_dns->ddns_restrict_patterns_list();

#modify ddns_restrict_patterns_list value
$grid_dns->ddns_restrict_patterns_list(['foo.com', 'bar.com']);
```

ddns_restrict_patterns( )

Use this method to set or retrieve the flag which indicates whether an option to restrict DDNS update request based on FQDN patterns is enabled or disabled.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

Specify 'true' to enable DDNS update request restriction base on FQDN patterns and 'false to disable it. The default value is 'false'.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#get ddns_restrict_patterns value
$ddns_restrict_patterns = $grid_dns->ddns_restrict_patterns();

#modify ddns_restrict_patterns value
$grid_dns->ddns_restrict_patterns('true');
```

ddns_restrict_static( )

Use this method to set or retrieve the flag which indicates whether an option to restrict DDNS update request to resource records which are marked as 'STATIC' is enabled or disabled.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

Specify 'true' to enable DDNS update requests to resource records which are marked as 'STATIC' and 'false' to disable it. The default value is 'false'.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#get ddns_restrict_static value
$ddns_restrict_static = $grid_dns->ddns_restrict_static();

#modify ddns_restrict_static value
$grid_dns->ddns_restrict_static('true');
```

ddns_restrict_protected( )

Use this method to set or retrieve the flag which indicates whether an option to restrict DDNS update request to protected resource records is enabled or disabled.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

Specify 'true' to enable DDNS update requests to protected resource records and 'false' to disable it. The default value is 'false'.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#get ddns_restrict_protected value
$ddns_restrict_protected = $grid_dns->ddns_restrict_protected();

#modify ddns_restrict_protected value
$grid_dns->ddns_restrict_protected('true');
```

ddns_restrict_secure( )

Use this method to set or retrieve the flag which indicates whether DDNS update request for principal other than target resource record's principal is restricted.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

Specify 'true' to enable DDNS update requests restriction for principal other than target resource record's principal and 'false' to disable it. The default value is 'false'.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#get ddns_restrict_secure value
$ddns_restrict_secure = $grid_dns->ddns_restrict_secure();

#modify ddns_restrict_secure value
$grid_dns->ddns_restrict_secure('true');
```

ddns_principal_tracking( )

Use this method to set or retrieve the flag which indicates whether the DDNS principal track is enabled or disabled.

DDNS principal tracking includes putting them into resource record affected by update.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

Specify 'true' to enable DDNS principal tracking and 'false' to disable it. The default value is 'false'.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#get ddns_principal_tracking value
$ddns_principal_tracking = $grid_dns->ddns_principal_tracking();

#modify ddns_principal_tracking value
$grid_dns->ddns_principal_tracking('true');
```

ddns_principal_group( )

Use this method to set or retrieve the DDNS principal group.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

The valid value is an Infoblox::DNS::DDNS::PrincipalCluster::Group object.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#get ddns_principal_group value
$ddns_principal_group = $grid_dns->ddns_principal_group();

#modify ddns_principal_group value
$grid_dns->ddns_principal_group($principal_group);
```

default_ttl( )

Use this method to set or retrieve the default TTL value of a grid DNS object. This interval tells the secondary how long the data can be cached.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

Number for the TTL, in seconds. *Default* value is 1/2 of lease time or 28800 seconds.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get the default TTL
my $default_ttl = $grid_dns->default_ttl();
#Modify the default TTL
$grid_dns->default_ttl(28800);
```

disable_edns( )

Use this method to enable or disable EDNS0 support for queries that require recursive resolution.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

Specify "true" to disable EDNS0 or "false" to enable it. The default value is "false".

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get the double confirmation of zone deletion flag
my $disable_edns = $grid_dns->disable_edns();
#Modify disable_edns flag
$grid_dns->disable_edns("true");
```

dns_cache_acceleration_ttl( )

Use this method to set or retrieve the minimum TTL value, in seconds, that a DNS record must have in order for it to be cached by the DNS Cache Acceleration service.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

An integer from 1 to 65000 that represents the TTL in seconds. The default value is 1.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get dns_cache_acceleration_ttl
my $dns_cache_acceleration_ttl = $grid_dns->dns_cache_acceleration_ttl( );
#Modify dns_cache_acceleration_ttl
$grid_dns->dns_cache_acceleration_ttl(60);
```

dns_health_check_anycast_control( )

Use this method to set or retrieve the flag that indicates whether the anycast failure (BFD session down) is enabled on member failure or not.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

Specify 'true' to enable anycast failure on Member failure, and 'false' to disable it.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get dns_health_check_anycast_control value
my $value = $object->dns_health_check_anycast_control();

#Modify dns_health_check_anycast_control value
$object->dns_health_check_anycast_control('true');
```

dns_health_check_domain_list( )

Use this method to set or retrieve the list of domain names for the DNS health check.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

The valid value is an array reference that contains desired domain names in string format.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get dns_health_check_domain_list value
my $value = $object->dns_health_check_domain_list();

#Modify dns_health_check_domain_list value
$object->dns_health_check_domain_list(['foo.com', 'bar.net']);
```

dns_health_check_interval( )

Use this method to set or retrieve the time interval (in seconds) for DNS health check.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

The valid value is an unsigned integer between 10 and 21600 that represents number of seconds between two health checks.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get dns_health_check_interval value
my $value = $object->dns_health_check_interval();

#Modify dns_health_check_interval value
$object->dns_health_check_interval(2000);
```

dns_health_check_recursion_flag( )

Use this method to set or retrieve the flag that indicates whether the recursive DNS health check is enabled or not.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

Specify 'true' to enable recursive DNS health check queries, and 'false' to disable it.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get dns_health_check_recursion_flag value
my $value = $object->dns_health_check_recursion_flag();

#Modify dns_health_check_recursion_flag value
$object->dns_health_check_recursion_flag('true');
```

dns_health_check_retries( )

Use this method to set or retrieve the number of DNS health check retries.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

The valid value is an unsigned integer between 1 and 10.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get dns_health_check_retries value
my $value = $object->dns_health_check_retries();

#Modify dns_health_check_retries value
$object->dns_health_check_retries(2);
```

dns_health_check_timeout( )

Use this method to set or retrieve the DNS health check timeout interval (in seconds).

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

The valid value is an unsigned integer between 1 and 10.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get dns_health_check_timeout value
my $value = $object->dns_health_check_timeout();

#Modify dns_health_check_timeout value
$object->dns_health_check_timeout(10);
```

dns_query_capture_file_time_limit( )

Use this method to set or retrieve the time limit (in minutes) for the DNS query capture file.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

The valid value is unsigned integer between 1 and 10. The default value is 10.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get dns_query_capture_file_time_limit value
my $dns_query_capture_file_time_limit = $object->dns_query_capture_file_time_limit();

#Modify dns_query_capture_file_time_limit value
$object->dns_query_capture_file_time_limit('5');
```

dns64_groups( )

Use this method to set or retrieve the list of DNS64 synthesis groups.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

Valid value is an array reference that contains Infoblox::Grid::DNS::DNS64Group objects.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get the list of DNS64 synthesis groups
my $dns64_groups_list = $grid_dns->dns64_groups();

#Modify it
my $group = $session->get(
                        object => "Infoblox::Grid::DNS::DNS64Group"),
                        name => "groupname",
                        prefix => "groupprefix",
                      );
$grid_dns->dns64_groups([$group]);
```

dnssec_enabled( )

Use this method to enable/disable the DNS security extension.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

Set the parameter to "true" to enable the DNS security extension. Set the parameter to "false" to disable the feature.

The default value of the parameter is "false".

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get dnssec_enabled
my $dnssec_enabled = $grid_dns->dnssec_enabled();
#Modify dnssec_enabled
$grid_dns->dnssec_enabled("true");
```

dnssec_enable_ksk_auto_rollover( )

Use this method to enable automatic rollover for Key-signing key.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

Specify "true" to enable automatic rollover for Key-signing key or "false" to disable it. The default value is "false".

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get dnssec_enable_ksk_auto_rollover value
my $enable_ksk_auto_rollover = $grid_dns->dnssec_enable_ksk_auto_rollover();
#Modify dnssec_enable_ksk_auto_rollover value
$grid_dns->dnssec_enable_ksk_auto_rollover("true");
```

dnssec_expired_signatures_enabled( )

Use this method to control whether the DNS grid object accepts expired signatures.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

Set the parameter to "true" to accept expired signatures. Set the parameter to "false" to reject expired signatures.

The default value of the parameter is "false".

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get dnssec_expired_signatures_enabled
my $dnssec_expired_signatures_enabled = $grid_dns->dnssec_expired_signatures_enabled();
#Modify dnssec_expired_signatures_enabled
$grid_dns->dnssec_expired_signatures_enabled("true");
```

dnssec_ksk_algorithms( )

Use this method to set or retrieve the DNSSEC key-signing key algorithms of all the zones in the grid.

This property determines which algorithms to use when generating the keys for signing the DNSKEY records.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

Currently there are two major versions of the DNSSEC zone encryption mechanism. The difference between the two is in their implementation of the Next Secure (NSEC) records. The older version, which is referred to as NSEC, is based on RFCs 4033-4035. The newer version, which is referred to as NSEC3, is described in RFC 5155.

The valid value is a list containing Infoblox::DNS::DNSSecKeyAlgorithm objects.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
# Getting algorithm
my $algorithms = $grid_dns->dnssec_ksk_algorithms();

# Modifying algorithm
$grid_dns->dnssec_ksk_algorithm([$alg1, $alg2]);
```

dnssec_ksk_email_notification_enabled( )

Use this method to enable or disable email notifications for KSK related events.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

Specify "true" to enable email notification or "false" to disable it. The default value is "false".

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get dnssec_ksk_email_notification_enabled value
my $value = $grid_dns->dnssec_ksk_email_notification_enabled();
#Modify dnssec_ksk_email_notification_enabled value
$grid_dns->dnssec_ksk_email_notification_enabled("true");
```

dnssec_ksk_rollover_interval()

Use this method to set or retrieve the rollover interval of the KSK keys.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

A positive integer that specifies the number of seconds between KSK rollovers. Normally, this parameter should be larger than the ZSK rollover interval. The DNSSEC key-signing key rollover interval (in seconds) must be between 1 day and seconds to Jan 2, 2038.

The default value of the dnssec_ksk_rollover_interval parameter is 31536000 (1 year).

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
# Getting dnssec_ksk_rollover_interval
my $ksk_rollover = $grid_dns->dnssec_ksk_rollover_interval();

# Modifying dnssec_ksk_rollover_interval
$grid_dns->dnssec_ksk_rollover_interval(86400);
```

dnssec_ksk_rollover_notification_config( )

Use this method to determine the type of events users will be notified.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

The valid values are "NONE", "ALL" and "REQUIRE_MANUAL_INTERVENTION". Default value is undefined.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get dnssec_ksk_rollover_notification_config value
my $value = $grid_dns->dnssec_ksk_rollover_notification_config();
#Modify dnssec_ksk_rollover_notification_config value
$grid_dns->dnssec_ksk_rollover_notification_config("ALL");
```

dnssec_ksk_snmp_notification_enabled( )

Use this method to enable or disable SNMP notifiations for KSK related events.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

Specify "true" to enable SNMP notification or "false" to disable it. The default value is "false".

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get dnssec_ksk_snmp_notification_enabled value
my $value = $grid_dns->dnssec_ksk_snmp_notification_enabled();
#Modify dnssec_ksk_snmp_notification_enabled value
$grid_dns->dnssec_ksk_snmp_notification_enabled("true");
```

dnssec_negative_trust_anchors( )

Use this method to set or retrive the list of zones for which the appliance does not perform DNSSEC validation.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

The valid value is an array of strings that contains the list of zones for which the appliance does not perform DNSSEC validation.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get dnssec_negative_trust_anchors value
my $value = $grid_dns->dnssec_negative_trust_anchors();
#Modify dnssec_negative_trust_anchors value
$grid_dns->dnssec_negative_trust_anchors($zones);
```

dnssec_nsec3_iterations( )

Use this method to define the number of iterations for NSEC3 hashing.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

The valid value are integers between 0 and 65535. Default value is undefined.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get dnssec_nsec3_iterations value
my $iteration = $grid_dns->dnssec_nsec3_iterations();
#Modify dnssec_nsec3_iterations value
$grid_dns->dnssec_nsec3_iterations();
```

dnssec_nsec3_salt_min_length( )

Use this method to define the minimum length for the NSEC3 salt.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

The valid value are integers between 0 and 65535. Default value is undefined.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get dnssec_nsec3_salt_min_length value
my $salt_min_length = $grid_dns->dnssec_nsec3_salt_min_length();
#Modify dnssec_nsec3_salt_min_length value
$grid_dns->dnssec_nsec3_salt_min_length();
```

dnssec_nsec3_salt_max_length( )

Use this method to define the maximum length for the NSEC3 salt.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

The valid value are integers between 0 and 65535. Default value is undefined.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get dnssec_nsec3_salt_max_length value
my $salt_max_length = $grid_dns->dnssec_nsec3_salt_max_length();
#Modify dnssec_nsec3_salt_max_length value
$grid_dns->dnssec_nsec3_salt_max_length();
```

dnssec_signature_expiration( )

Use this method to set or retrieve the expiration time for DNSSEC signatures.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

The signature expiration time in seconds. The minimum value is 86400 (1 day), the maximum value is the number of seconds from the time you set it to January 2038.

The default value of dnssec_signature_expiration parameter is 2592000 seconds (i.e. 30 days).

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
# Getting dnssec_signature_expiration
my $signature_expiration = $grid_dns->dnssec_signature_expiration();

# Modifying dnssec_signature_expiration
$grid_dns->dnssec_signature_expiration(31536000);
```

dnssec_validation_enabled( )

Use this method to enable and disable DNS security validation (for the grid appliances working as clients of secure zones).

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

Set the parameter to "true" to enable the validation. Set to "false" to disable the validation.

The default value of the parameter is "true".

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get dnssec_validation_enabled
my $dnssec_validation_enabled = $grid_dns->dnssec_validation_enabled();
#Modify dnssec_validation_enabled
$grid_dns->dnssec_validation_enabled("false");
```

dnssec_blacklist_enabled()

Use this method to enable and disable blacklist rules for DNSSEC-enabled clients.

Parameter

Set the parameter to "true" to apply blacklist rules for DNSSEC-enabled clients. Set to "false" to disable them.

The default value of the parameter is "false".

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get dnssec_blacklist_enabled
my $dnssec_blacklist_enabled = $grid_dns->dnssec_blacklist_enabled();
#Modify dnssec_blacklist_enabled
$grid_dns->dnssec_blacklist_enabled("true");
```

dnssec_dns64_enabled()

Use this method to enable and disable DNS64 groups for DNSSEC-enabled clients.

Parameter

Set the parameter to "true" to apply DNS64 groups for DNSSEC-enabled clients. Set to "false" to disable them.

The default value of the parameter is "false".

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get dnssec_dns64_enabled
my $dnssec_dns64_enabled = $grid_dns->dnssec_dns64_enabled();
#Modify dnssec_dns64_enabled
$grid_dns->dnssec_dns64_enabled("true");
```

dnssec_nxdomain_enabled()

Use this method to enable and disable NXDOMAIN rules for DNSSEC-enabled clients.

Parameter

Set the parameter to "true" to apply NXDOMAIN rules for DNSSEC-enabled clients. Set to "false" to disable them.

The default value of the parameter is "false".

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get dnssec_nxdomain_enabled
my $dnssec_nxdomain_enabled = $grid_dns->dnssec_nxdomain_enabled();
#Modify dnssec_nxdomain_enabled
$grid_dns->dnssec_nxdomain_enabled("true");
```

dnssec_rpz_enabled()

Use this method to enable and disable RPZ policies for DNSSEC-enabled clients.

Parameter

Set the parameter to "true" to apply RPZ policies for DNSSEC-enabled clients. Set to "false" to disable them.

The default value of the parameter is "false".

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get dnssec_rpz_enabled
my $dnssec_rpz_enabled = $grid_dns->dnssec_rpz_enabled();
#Modify dnssec_rpz_enabled
$grid_dns->dnssec_rpz_enabled("true");
```

dnssec_trusted_keys( )

Use this method to specify the list of trusted keys for the DNSSEC feature.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

Array reference that contains a list of Infoblox::DNS::DnssecTrustedKey objects.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns a reference to a list of the trusted keys.

Example

```
#Get dnssec_trusted_keys
my $dnssec_trusted_keys = $grid_dns->dnssec_trusted_keys();
#Modify dnssec_trusted_keys
my $key1=Infoblox::DNS::DnssecTrustedKey->new(
   fqdn => "myzone.com",
   algorithm => "RSAMD5",
   key       => "c2ltcGxlIGtleQo="
   );
$grid_dns->dnssec_trusted_keys([$key1]);
```

dnssec_zsk_algorithms( )

Use this method to set or retrieve the DNSSEC zone-signing key algorithms of all the zones in the grid.

This property determines which algorithms to use when generating the keys for signing the DNSKEY records.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

Currently there are two major versions of the DNSSEC zone encryption mechanism. The difference between the two is in their implementation of the Next Secure (NSEC) records. The older version, which is referred to as NSEC, is based on RFCs 4033-4035. The newer version, which is referred to as NSEC3, is described in RFC 5155.

The valid value is a list containing Infoblox::DNS::DNSSecKeyAlgorithm objects.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
# Getting algorithms
my $algorithms = $grid_dns->dnssec_zsk_algorithms();

# Modifying algorithms
$grid_dns->dnssec_zsk_algorithm([$alg1, $alg2]);
```

dnssec_zsk_rollover_interval()

Use this method to set or retrieve the zone-signing key rollover interval.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

A positive integer that specifies the number of seconds between ZSK rollovers. Normally, this parameter should be smaller than the KSK rollover interval. The DNSSEC zone-signing key over interval (in seconds) must be between 1 day and the number of seconds from present to Jan 2038.

The default value of the dnssec_zsk_rollover_interval parameter is 2592000 seconds (30 days).

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
# Getting dnssec_zsk_rollover_interval
my $zsk_rollover = $grid_dns->dnssec_zsk_rollover_interval();

# Modifying dnssec_ksk_rollover_interval
$grid_dns->dnssec_zsk_rollover_interval(86400);
```

dnssec_zsk_rollover_mechanism( )

Use this method to set or retrieve the ZSK rollover mechanism type.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

The valid values are "DOUBLE_SIGN" and "PRE_PUBLISH". Default value is undefined.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get dnssec_zsk_rollover_mechanism value
my $value = $grid_dns->dnssec_zsk_rollover_mechanism();
#Modify dnssec_zsk_rollover_mechanism value
$grid_dns->dnssec_zsk_rollover_mechanism("DOUBLE_SIGN");
```

domains_to_capture_dns_queries( )

Use this method to set or retrieve the list of domains to capture DNS queries.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

The valid value is an array reference that contains desired domain names in a string format.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get domains_to_capture_dns_queries value
my $domains_to_capture_dns_queries = $object->domains_to_capture_dns_queries();

#Modify domains_to_capture_dns_queries value
$object->domains_to_capture_dns_queries(['domain1.com', 'domain2.com']);
```

dtc_dnssec_mode( )

Use this method to set or retrieve the DTC DNSSEC operation mode.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

The valid values are 'SIGNED' and 'UNSIGNED'. The default value is 'UNSIGNED'.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get dtc_dnssec_mode value
my $dtc dnssec mode   $object  dtc dnssec mode
```

```
                        =       ->              ();
```

```
#Modify dtc_dnssec_mode value
$object->dtc_dnssec_mode('UNSIGNED');
```

dtc_edns_prefer_client_subnet( )

Use this method to set or retrieve the flag that indicates whether the use of the edns-client-subnet option for DTC is enabled or disabled.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

Specify 'true' to enable use of the edns-client-subnet option for DTC or 'false' to disable it.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get dtc_edns_prefer_client_subnet value
my $dtc_edns_prefer_client_subnet = $object->dtc_edns_prefer_client_subnet();

#Modify dtc_edns_prefer_client_subnet value
$object->dtc_edns_prefer_client_subnet('true');
```

dtc_topology_ea_list( )

Use this method to set or retrieve the DTC topology extensible attribute definition list. When configuring a DTC topology, users may configure classification as either "Geographic" or "Extensible Attributes". Selecting "Extensible Attributes" will replace supported MaxMind database labels (Continent, Country, Subdivision, City) with the names of the selected EA types and provide values of those extensible attributes extracted from DHCP Network Container, Network and Range objects.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

The valid value is an Infoblox::Grid::ExtensibleAttributeDef object name.

Note that the number of extensible attribute definitions is limited to 4. You can select only string, enum, and integer extensible attribute definitions.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get dtc_topology_ea_list value
my $dtc_topology_ea_list = $object->dtc_topology_ea_list();

#Modify dtc_topology_ea_list value
$object->dtc_topology_ea_list(['Site', 'State']);
```

email( )

Use this method to set or retrieve the email address of a grid DNS object.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

String with email address.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get email address
my $email = $grid_dns->email();
```

```
#Modify email address
$grid_dns->email("admin\@infoblox.com");
```

enable_blackhole( )

Use this method to set or retrieve the enable_blackhole flag of a grid DNS object. When enabled, the Infoblox appliance does not respond to DNS queries from clients that are in the configured blackhole_list.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

Specify "true" to set the enable_blackhole flag or "false" to deactivate and unset it. The default value is "false".

Returns

When you specify a parameter and the modification succeeds, the method returns "true". When the operation fails, the method returns "false".

When you do not specify a parameter, the method returns the attribute value.

Example

```
#Get enable_blackhole flag
my $enable_blackhole = $grid_dns->enable_blackhole();
#Modify enable_blackhole recursive query flag
$grid_dns->enable_blackhole("true");
```

enable_blacklist( )

Use this method to enable or disable a blacklist globaly.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

Specify "true" to set the enable_blacklist flag or "false" to deactivate/unset it. The default is "false".

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Getting enable_blacklist
my $enable_blacklist = $grid_dns->enable_blacklist( );
#Modifying enable_blacklist
$grid_dns->enable_blacklist("true");
```

enable_capture_dns_queries( )

Use this method to set or retrieve the flag that indicates whether the capture of DNS queries is enabled or disabled.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

Specify 'true' to enable the capture of DNS queries or 'false' to disable it. The default value is 'false'.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get enable_capture_dns_queries value
my $enable_capture_dns_queries = $object->enable_capture_dns_queries();

#Modify enable_capture_dns_queries value
$object->enable_capture_dns_queries('true');
```

enable_capture_dns_responses( )

Use this method to set or retrieve the flag that indicates whether the capture of DNS responses is enabled or disabled.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

Specify 'true' to enable the capture of DNS responses or 'false' to disable it. The default value is 'false'.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get enable_capture_dns_responses value
my $enable_capture_dns_responses = $object->enable_capture_dns_responses();

#Modify enable_capture_dns_responses value
$object->enable_capture_dns_responses('true');
```

enable_client_subnet_recursive( )

Use this method to set or retrieve the flag that indicates whether adding EDNS client subnet options in recursive resolutions is enabled or not.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

Specify 'true' to enable adding EDNS client subnet options in recursive resolutions or 'false' to disable it. The default value is 'false'.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get enable_client_subnet_recursive value
my $value = $object->enable_client_subnet_recursive();

#Modify enable_client_subnet_recursive value
$object->enable_client_subnet_recursive('true');
```

enable_client_subnet_forwarding( )

Use this method to set or retrieve the flag that indicates whether forwarding EDNS client subnet options to upstream servers is enabled or not.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

Specify 'true' to enable forwarding of EDNS client subnet options to upstream servers or 'false' to disable it. The default value is 'false'.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get enable_client_subnet_forwarding value
my $value = $object->enable_client_subnet_forwarding();

#Modify enable_client_subnet_forwarding value
$object->enable_client_subnet_forwarding('true');
```

enable_delete_associated_ptr( )

Use this method to enable or disable the ability to automatically remove associated PTR records while deleting A or AAAA records.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

Specify "true" to enable the ability to automatically remove associated PTR records while deleting A or AAAA records or "false" to disable it. The default value is "false".

Returns

When you specify a parameter and the modification succeeds, the method returns "true". When the operation fails, the method returns "false".

When you do not specify a parameter, the method returns the attribute value.

Example

```
#Get enable_delete_associated_ptr flag
my $enable_delete_associated_ptr = $grid_dns->enable_delete_associated_ptr();
#Modify enable_delete_associated_ptr flag
$grid_dns->enable_delete_associated_ptr("true");
```

enable_dns_health_check( )

Use this method to set or retrieve the flag that indicates whether the DNS health check is enabled or not.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

Specify 'true' to enable DNS health check, and 'false' to disable it.

Returns

When you specify a parameter and the modification succeeds, the method returns "true". When the operation fails, the method returns "false".

When you do not specify a parameter, the method returns the attribute value.

Example

```
#Get enable_dns_health_check value
my $value = $object->enable_dns_health_check();

#Modify enable_dns_health_check value
$object->enable_dns_health_check('true');
```

enable_dns64( )

Use this method to enable/disable DNS64 support.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

Specify "true" to set the enable_dns64 flag or "false" to deactivate and unset it. The default value is "false".

Returns

When you specify a parameter and the modification succeeds, the method returns "true". When the operation fails, the method returns "false".

When you do not specify a parameter, the method returns the attribute value.

Example

```
#Get enable_dns64 flag
my $enable_dns64 = $grid_dns->enable_dns64();
#Modify enable_dns64 recursive query flag
$grid_dns->enable_dns64("true");
```

enable_dtc_dns_fall_through( )

Use this method to set or retrieve the flag that indicates whether the DNS traffic control (DTC) to DNS fall through is enabled or not.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

Specify 'true' to enable DTC to DNS fall through or 'false' to disable it.

Returns

When you specify a parameter and the modification succeeds, the method returns "true". When the operation fails, the method

returns "false".

When you do not specify a parameter, the method returns the attribute value.

Example

```
#Get enable_dtc_dns_fall_through value
my $value = $grid_dns->enable_dtc_dns_fall_through();

#Modify enable_dtc_dns_fall_through value
$grid_dns->enable_dtc_dns_fall_through('false');
```

enable_excluded_domain_names( )

Use this method to set or retrieve the flag that indicates whether the excluding domain names from captured DNS query and response is enabled or disabled.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

Specify 'true' to enable the excluding domain names from captured DNS query and response or 'false' to disable it. The default value is 'false'.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get enable_excluded_domain_names value
my $enable_excluded_domain_names = $object->enable_excluded_domain_names();

#Modify enable_excluded_domain_names value
$object->enable_excluded_domain_names('true');
```

enable_fixed_rrset_order_fqdns( )

Use this method to enable or disable fixed RRset order for specified FQDNs.

The enable_fixed_rrset_order_fqdns attribute can be specified explicitly. It is also set implicitly when fixed_rrset_order_fqdns is set to a defined value.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

Specify "true" to enable fixed RRset order for specified FQDNs or "false" to disable it.

Returns

When you specify a parameter and the modification succeeds, the method returns "true". When the operation fails, the method returns "false".

When you do not specify a parameter, the method returns the attribute value.

Example

```
#Get enable_fixed_rrset_order_fqdns flag
my $enable_fixed_rrset_order_fqdns = $grid_dns->enable_fixed_rrset_order_fqdns();
#Modify enable_fixed_rrset_order_fqdns flag
$grid_dns->enable_fixed_rrset_order_fqdns("true");
```

enable_ftc( )

Use this method to set or retrieve the flag that indicates whether the Fault Tolerant Caching (FTC) feature is enabled or not.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

Specify 'true' to enable the FTC feature or 'false' to disable it. The default value is 'false'.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get enable_ftc
my $enable_ftc = $grid_dns->enable_ftc();
#Modify enable_ftc
$grid_dns->enable_ftc('true');
```

enable_gss_tsig( )

Use this method to enable/disable the appliance to receive GSS-TSIG authenticated updates from DHCP clients. You must import the keytab file before you enable this feature. See Infoblox::Session->import_data().

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

Specify "true" to enable the the appliance to receive GSS-TSIG authenticated updates or "false" to disable it. The default value is "false".

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get enable_gss_tsig value
$my $enable_gss_tsig = $grid_dns->enable_gss_tsig();

#Modify enable_gss_tsig value

$session->import_data(
     type        => 'keytab',
     path        => '/infoblox/tests/datasets/gss-tsig-gs.keytab',
     service     => 'dns',
     virtual_ip  => '192.168.1.2');

$grid_dns->enable_gss_tsig('true');
```

enable_host_rrset_order( )

Use this method to set or retrieve the grid-level enable_host_rrset_order value.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

Specify "true" to set the enable_host_rrset_order flag or "false" to deactivate and unset it. The default value is "false".

Returns

When you specify a parameter and the modification succeeds, the method returns "true". When the operation fails, the method returns "false".

When you do not specify a parameter, the method returns the attribute value.

Example

```
# Get enable_host_rrset_order
my $host_rrset_order = $grid_dns->enable_host_rrset_order();
# Set enable_host_rrset_order
$grid_dns->enable_host_rrset_order("true");
```

enable_hsm_signing( )

Use this method to enable/disable the HSM for DNSSEC key generation and signing. To enable this feature, an HSM group must be configured with at least one enabled HSM.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

Specify "true" to enable HSM signing or "false" to disable it. The default value is "false".

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get enable_hsm_signing
$grid_dns->enable_hsm_signing( );
#Modify enable_hsm_signing
$grid_dns->enable_hsm_signing("true");
```

enable_query_rewrite( )

Use this method to enable or disable DNS query rewrite.

Parameter

Specify "true" to enable DNS query rewrite or "false" to disable it. The default value is "false".

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get enable_query_rewrite
$grid_dns->enable_query_rewrite( );
#Modify enable_query_rewrite
$grid_dns->enable_query_rewrite("true");
```

excluded_domain_names( )

Use this method to set or retrieve the list of domains that are excluded from DNS response and query capturing.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

The valid value is an array reference that contains desired domain names in a string format.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get excluded_domain_names value
my $excluded_domain_names = $object->excluded_domain_names();

#Modify excluded_domain_names value
$object->excluded_domain_names(['domain1.com', 'domain2.com']);
```

expire_after( )

Use this method to set or retrieve the expiration time of a grid DNS object. If the secondary DNS server fails to contact the primary server for the specified interval, the secondary server stops giving out answers about the zone because the zone data is too old to be useful.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

Number with the seconds for the expiration time. *Default* value is 2419200 seconds (4 weeks).

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get the expiration time
my $expire_after = $grid_dns->expire_after();
#Modify the expiration time
$grid_dns->expire_after(2419200);
```

file_transfer_setting( )

Use this method to set or retrieve the DNS capture file transfer settings.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

The valid value is an Infoblox::Grid::DNS::FileTransferSetting object.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get file_transfer_setting value
my $file_transfer_setting = $object->file_transfer_setting();

#Modify file_transfer_setting value
$object->file_transfer_setting($file_transfer);
```

filter_aaaa( )

Use this method to set or retrieve the type of AAAA filtering for this grid DNS object.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

Valid values are "YES", "NO" and "BREAK_DNSSEC". The default value is "NO".

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get the filter aaaa type
my $filter_aaaa = $grid_dns->filter_aaaa();
#Modify the filtering
$grid_dns->filter_aaaa("YES");
```

filter_aaaa_list( )

Use this method to set or retrieve the list of IPv4 addresses and networks from which queries are received. AAAA filtering is applied to these addresses.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

A valid value is an Infoblox::Grid::NamedACL object or an array reference that contains IPv4 addresses and networks. To return AAAA records for queries received from an IPv4 client, add "!" as the prefix to the IP addresses and networks. Specify "any" to allow AAAA records to be returned for any query received by an IPv4 client.

Returns

When you specify a parameter and the modification succeeds, the method returns "true". When the operation fails, the method returns "false".

When you do not specify a parameter, the method returns the attribute value.

Example

```
#Get the filter_aaaa list
my $filter_aaaa_list = $grid_dns->filter_aaaa_list();
#Modify the filter_aaaa_list
$grid_dns->filter_aaaa_list(["any"]);
#Modify the filter_aaaa list
$grid_dns->filter_aaaa_list(["10.0.0.10", "20.0.0.0/24", "!"."30.0.0.30", "!"."40.10.0.0/24"]);
#Modify the filter_aaaa list
$grid_dns->filter_aaaa_list(undef);
#Use Infoblox::Grid::NamedACL
```

```
$grid_dns->filter_aaaa_list($nacl);
```

forward_only( )

Use this method to set or retrieve the forward only flag of a grid DNS object. On enabling, the Infoblox appliance will send queries to forwarders only, and not to internal or Internet root servers.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

Specify "true" to set the forward_only flag or "false" to deactivate/unset it. The default value is "false".

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get the forward only flag
my $forward_only = $grid_dns->forward_only();
#Modify the forward only
$grid_dns->forward_only("true");
```

forward_updates( )

Use this method to set or retrieve the forward_updates flag of a grid DNS object. This enables a secondary server to receive updates from the DHCP server, and then forward them to the primary DNS server.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

Specify "true" to set the forward_updates flag or "false" to deactivate/unset it. The default value is "false".

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get the forward updates flag
my $forward_updates = $grid_dns->forward_updates();
#Modify the forward updates flag
$grid_dns->forward_updates("true");
```

forwarders( )

Use this method to set or retrieve the forwarders list of a grid DNS object. A forwarder is essentially a name server to which other name servers first send all of their off-site queries. The forwarder builds up a cache of information, avoiding the necessity for the other name servers to send queries off-site.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

The valid value is an array reference that contains IP address(es).

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get the forwarders list
my $forwarders_list = $grid_dns->forwarders();
#Modify the forwarders list
$grid_dns->forwarders(["2.2.2.2", "3.3.3.3"]);
```

fixed_rrset_order_fqdns( )

Use this method to set or retrieve configuration for fixed RRSET order for specified FQDNs.

Setting this method to a defined value implicitly sets the enable_fixed_rrset_order_fqdns method to "true". Setting the parameter to undefined causes the appliance to disable fixed RRset order of FQDNs and automatically resets the enable_fixed_rrset_order_fqdns method to "false".

Note that when fixed_rrset_order_fqdns is set to a defined value and enable_fixed_rrset_order_fqdns is set to "false", the last operation takes precedence. Thus the sequence $object->fixed_rrset_order_fqdns($fqdns); $object->enable_fixed_rrset_order_fqdns("false"); will set enable_fixed_rrset_order_fqdns to "false", and the sequence $object->enable_fixed_rrset_order_fqdns("false"); $object->fixed_rrset_order_fqdns($fqdns); will result in enable_fixed_rrset_order_fqdns="true".

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

The valid value is an array reference that contains Infoblox::Grid::DNS::FixedRRSetOrderFQDN objects.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get the fixed_rrset_order_fqdns list
my $fixed_rrset_order_fqdns = $grid_dns->fixed_rrset_order_fqdns();
#Modify the fixed_rrset_order_fqdns list
$grid_dns->fixed_rrset_order_fqdns([$fqdn1, $fqdn2]);
```

ftc_expired_record_ttl( )

Use this method to set or retrieve the TTL value of the expired Fault Tolerant Caching (FTC) record in DNS responses.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

A 32-bit integer, from 0 to 4294967295, that represents the TTL in seconds. The default value is 5.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get ftc_expired_record_ttl
my $ftc_expired_record_ttl = $grid_dns->ftc_expired_record_ttl( );
#Modify ftc_expired_record_ttl
$grid_dns->ftc_expired_record_ttl(60);
```

ftc_expired_record_timeout( )

Use this method to set or retrieve the timeout value after which the expired Fault Tolerant Caching (FTC) record becomes stale and is no longer valid.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

A 32-bit integer, from 0 to 4294967295, that represents the timeout in seconds. The default value is 86400.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get ftc_expired_record_timeout
my $ftc_expired_record_timeout = $grid_dns->ftc_expired_record_timeout( );
#Modify ftc_expired_record_timeout
$grid_dns->ftc_expired_record_timeout(3600);
```

gss_tsig_keys( )

Use this method to set or retrieve the list of Kerberos keys for a Grid DNS object.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

The valid value is an array reference that contains Infoblox::Grid::KerberosKey objects.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get the Kerberos keys list
my $kerberos_keys_list = $grid_dns->gss_tsig_keys();
#Modify the Kerberos keys list
my $kerberos_key1 = Infoblox::Cluster::KerberosKey->new(
       principal => 'host/appliance.ivan.test1@LADYBUG.LOCAL',
       );
my $kerberos_key2 = Infoblox::Cluster::KerberosKey->new(
       principal => 'host/appliance.ivan.test2@LADYBUG.LOCAL',
       );
$grid_dns->gss_tsig_keys([$kerberos_key1, $kerberos_key2]);
```

host_name_restriction_policy( )

Use this method to set or retrieve the default host name restriction policy of a grid DNS object.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

String with the name of predefined record policies or user-defined record policies. The default value is the predefined record policy "Allow Underscore".

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get host name restriction policy
my $host_name_restriction_policy = $grid_dns->host_name_restriction_policy();
#Modify host name restriction policy
$grid_dns->host_name_restriction_policy("Allow Any");
```

lame_ttl( )

Use this method to set or retrieve the number of seconds to cache lame delegations or lame servers.

Parameter

The number of seconds to cache lame delegations or lame servers. The default value is 600.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get lame_ttl
my $lame_ttl = $grid_dns->lame_ttl();
#Modify lame_ttl
$grid_dns->lame_ttl(700);
```

logging_categories( )

Use this method to set or retrieve the logging categories. It is possible to specify several BIND logging message categories to be captured by syslog.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

Valid value is an array reference that contains one or more of the following values:

"general" - Records the BIND messages that are not specifically classified.

"config" - Records the parsing messages of the configuration file.

"dnssec" - Records the DNSSEC-signed responses.

"network" - Records the network operation messages.

"queries" - Records the query messages.

"query_rewrite" - Records rewritten queries.

"responses" - Records the response messages.

"security" - Records the approved and denied requests.

"xfer-in" - Records zone transfer messages from the remote name servers to the appliance.

"xfer-out" - Records zone transfer messages from the Infoblox appliance to remote name servers.

"update" - Records the dynamic update instances.

"resolver" - Records the DNS resolution instances, including recursive queries from resolvers.

"notify" - Records the asynchronous zone change notification messages.

"lame_servers" - Records bad delegation instances.

"database" - Records BIND internal database processes.

"client" - Records client requests.

"rpz" - Records RPZ messages.

"dtc_gslb" - Records the DTC GSLB activity.

"dtc_health" - Records the DTC health monitoring.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get the logging categories list
my $logging_categories_list = $grid_dns->logging_categories();
#Modify the logging categories list
$grid_dns->logging_categories(["general", "database", "queries"]);
```

logging_facility( )

Use this method to set or retrieve the logging facility of a grid DNS object.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

Text with the facility name. Valid values are: "daemon", "local0", "local1", "local2", "local3", "local4", "local5", "local6", and "local7".

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get the logging facility
my $logging_facility = $grid_dns->logging_facility();
#Modify the logging facility
$grid_dns->logging_facility("daemon");
```

max_cache_ttl( )

Use this method to set or retrieve the maximum time (in seconds) for which the server will cache positive answers.

Parameter

The maximum time is specified in seconds. The default value is 604800.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get max_cache_ttl
my $max_cache_ttl = $grid_dns->max_cache_ttl();
#Modify max_cache_ttl
$grid_dns->max_cache_ttl(400);
```

max_ncache_ttl( )

Use this method to set or retrieve the maximum time (in seconds) for which the server will cache negative (NXDOMAIN) responses.

Parameter

The maximum time is specified in seconds. The default value is 10800. The maximum allowed value is 604800.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get max_ncache_ttl
my $max_ncache_ttl = $grid_dns->max_ncache_ttl();
#Modify max_ncache_ttl
$grid_dns->max_ncache_ttl(400);
```

max_cached_lifetime( )

Use this method to set or retrieve maximum time in seconds a DNS response can be stored in the hardware acceleration cache.

Parameter

Valid values are unsigned integer between 60 and 86400, inclusive. Default value is 86400.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get max_cached_lifetime
my $max_cached_lifetime = $grid_dns->max_cached_lifetime();
#Modify max_cached_lifetime
$grid_dns->max_cached_lifetime(400);
```

member_secondary_notify( )

Use this method to set or retrieve the member_secondary_notify flag of a grid DNS object. When enabled, this feature allows grid members that are authoritative secondary servers to send notification messages to external name servers, if the grid member that is primary for a zone fails or loses connectivity.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

Specify "true" to set member_secondary_notify flag or "false" to deactivate/unset it. The default value is "true".

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get the member secondary notification flag
my $member_secondary_notify = $grid_dns->member_secondary_notify();
#Modify the member secondary notification flag
$grid_dns->member_secondary_notify("false");
```

negative_ttl( )

Use this method to set or retrieve the negative TTL value of a grid DNS object. This interval tells the secondary how long data can be cached for "Does Not Respond" responses.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

Number for the negative TTL, in seconds. *Default* value is 900 seconds.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get the negative ttl value
my $negative_ttl = $grid_dns->negative_ttl();
#Modify the negative ttl value
$grid_dns->negative_ttl(800);
```

next_secure_type( )

Use this method to set or retrieve resource record type.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

Valid value is 'NSEC' or 'NSEC3'.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get the next_secure_type value
my $next_secure_type = $grid_dns->next_secure_type();
#Modify the next_secure_type value
$grid_dns->next_secure_type('NSEC3');
```

notify_delay( )

Use this method to specify with how many seconds of delay the notify messages are sent to secondaries.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

Valid values are unsigned integer between 5 and 86400, inclusive. Default value is 5.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Getting notify_delay
my $notify_delay = $griddns->notify_delay( );
#Modifying notify_delay
$griddns->notify_delay(5);
```

nxdomain_redirect( )

Use this method to enable or disable NXDOMAIN redirection.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

Specify "true" to set the nxdomain_redirect flag or "false" to deactivate/unset it. The default value is "false".

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Getting nxdomain_redirect
my $nxdomain_redirect = $griddns->nxdomain_redirect( );
#Modifying nxdomain_redirect
$griddns->nxdomain_redirect("true");
```

nxdomain_redirect_addresses( )

Use this method to set the NXDOMAIN redirection addresses.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

The valid value is an array of IPv4 addresses. The default value is undefined.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get nxdomain_redirect_addresses
my $nxdomain_redirect_addresses = $griddns->nxdomain_redirect_addresses( );
#Modify nxdomain_redirect_addresses
$griddns->nxdomain_redirect_addresses(["10.0.0.10", "10.0.0.20"]);
```

nxdomain_redirect_addresses_ipv6( )

Use this method to set the NXDOMAIN redirection IPv6 addresses.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

The valid value is an array of IPv6 addresses. The default value is undefined.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get nxdomain_redirect_addresses_ipv6
my $nxdomain_redirect_addresses_ipv6 = $griddns->nxdomain_redirect_addresses_ipv6( );
#Modify nxdomain_redirect_addresses_ipv6
$griddns->nxdomain_redirect_addresses_ipv6(["2001:db8::1", "2002:ffff::1"]);
```

nxdomain_redirect_ttl( )

Use this method to set or retrieve the TTL value of synthetic DNS responses that result from NXDOMAIN redirection.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

A 32-bit integer, from 0 to 4294967295, that represents the TTL in seconds. The default value is 60.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get nxdomain_redirect_ttl
my $nxdomain_redirect_ttl = $griddns->nxdomain_redirect_ttl( );
#Modify nxdomain_redirect_ttl
$griddns->nxdomain_redirect_ttl(60);
```

nxdomain_log_query( )

Use this method to set or retrieve the flag that indicates whether NXDOMAIN redirection queries are logged.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

Specify "true" to enable logging, or "false" to deactivate it. The default value is "false".

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get nxdomain_log_query
my $nxdomain_log_query = $griddns->nxdomain_log_query( );
#Modify nxdomain_log_query
$griddns->nxdomain_log_query("true");
```

nxdomain_rulesets( )

Use this method to specify or retrieve Infoblox::DNS::Ruleset object names assigned at the grid level for NXDOMAIN redirection.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

The valid value is an array reference that contains the names of the Infoblox::DNS::Ruleset objects assigned to the grid. The names must be in string format. The default value is undefined.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get nxdomain_rulesets
my $nxdomain_rulesets = $griddns->nxdomain_rulesets();
#Modify nxdomain_rulesets
$griddns->nxdomain_rulesets(["ruleset1", "ruleset2"]);
```

notify_source_port( )

Use this method to set or retrieve the source port number for notify messages. When requesting zone transfers from the primary server, some secondary DNS servers use the source port number (the primary server used to send the notify message) as the destination port number in the zone transfer request.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

Source port number for notify messages. Valid values are between 1 and 63999. *Default* is picked by BIND.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get the notify source port value
```

```
my $notify_source_port = $grid_dns->notify_source_port();
#Modify the notify source port value
$grid_dns->notify_source_port(77);
```

nsgroup_default( )

Use this method to set or retrieve the default nameserver group.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

Text with the name of the default nameserver group. *Default* value is None.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get the default nameserver group
my $nsgroup_default = $grid_dns->nsgroup_default();
#Modify the default nameserver group
$grid_dns->nsgroup_default("ns_group_A");
```

nsgroups( )

Use this method to set or retrieve the list of nameserver groups. A name server group is a collection of one primary DNS server and one or more secondary DNS servers.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

Valid value is an array reference that contains Infoblox::Grid::DNS::Nsgroup and Infoblox::DNS::Member object(s).

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get the nameserver group
my $nsgroup = $grid_dns->nsgroups();
#Modify the nameserver group
#Creating Member objects.
my $member1 = Infoblox::DNS::Member->new(
    name     => "infoblox.localdomain",
    ipv4addr => "192.168.1.2",
    stealth  => "false"
    );

#Creating a nameserver object.
my $nameserver1 = Infoblox::DNS::Nameserver->new(
    name     => "ns1.test.extra",
    ipv4addr => "4.4.4.4",
    stealth  => "false",
    TSIGname => "tsig_name",
    TSIGkey  => "ZX1jVJl7C58GT/sc7Q3ucA=="
    );

#Creating a nameserver object.
my $nameserver2 = Infoblox::DNS::Nameserver->new(
    name     => "ns2.test.extra",
    ipv4addr => "5.5.5.5",
    stealth  => "false",
    TSIGname => "tsig_name2",
    TSIGkey  => "qwqwqwqwqwqwqwqwqwqA=="
    );

my $nsg1 = Infoblox::Grid::DNS::Nsgroup->new(
    name        => "ns_group_A",
    primary     => $member1,
    secondaries => [$nameserver1, $nameserver2]
```

```
    );

$grid_dns->nsgroups([$nsg1]);
```

preserve_host_rrset_order_on_secondaries( )

Use this method to set or retrieve the grid-level preserve_host_rrset_order_on_secondaries value.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

Specify "true" to set the preserve_host_rrset_order_on_secondaries flag or "false" to unset it. The default value is "false".

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
# Get preserve_host_rrset_order_on_secondaries
my $preserve_order = $griddns->preserve_host_rrset_order_on_secondaries();
# Set preserve_host_rrset_order_on_secondaries
$griddns->preserve_host_rrset_order_on_secondaries("true");
```

query_source_port( )

Use this method to set or retrieve the query source port of a grid DNS object. Specifying a source port number for recursive queries ensures that a firewall will allow the response.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

Source port number for recursive queries . Valid values are between 1 and 63999. *Default* is picked by BIND.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get the query source port
my $query_source_port = $grid_dns->query_source_port();
#Modify the query source port
$grid_dns->query_source_port(99);
```

query_rewrite_prefix( )

Use this method to set or retrieve the domain name prefix for DNS query rewrite.

Parameter

Valid value is the domain name prefix in string format. The default value is undefined.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get query_rewrite_prefix
my $query_rewrite_prefix = $grid_dns->query_rewrite_prefix();
#Modify query_rewrite_prefix
$grid_dns->query_rewrite_prefix("user-ersgcds");
```

query_rewrite_domain_names( )

Use this method to set or retrieve the list of domain names that trigger DNS query rewrite.

Parameter

Valid value is an array reference that contains domain names in string format. The default value is undefined.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get query_rewrite_domain_names
my $query_rewrite_domain_names = $grid_dns->query_rewrite_domain_names();
#Modify query_rewrite_domain_names
$grid_dns->query_rewrite_domain_names(["florida.south", "florida.west"]);
```

scavenging_settings( )

Use this method to set or retrieve the Grid-level scavenging settings.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

The valid value is an Infoblox::Grid::DNS::ScavengingSetting object.

Parameter

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get scavenging_settings value
my $scavenging_settings = $object->scavenging_settings();

#Modify scavenging_settings value
$object->scavenging_settings($scavenging_settings);
```

record_name_policies( )

Use this method to set or retrieve the list of record name policies.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

Valid value is an array reference that contains Infoblox::Grid::RecordNamePolicy object(s). By default there are three predefined policies.

Policy Name Regex Applied

"Allow Any" .+

"Allow Underscore" ^[-a-zA-Z0-9_.]+$

"Strict Hostname Checking" ^[a-zA-Z0-9]$|^[a-zA-Z0-9][-a-zA-Z0-9.]*[a-zA-Z0-9]$

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get record name policies
my $record_name_policies = $grid_dns->record_name_policies();
#Modify record_name_policies

#Creating record name policy objects.
my $recpolicy1 = Infoblox::Grid::RecordNamePolicy->new(
    regex => "*",
    name  => "Local Policy",
    );
my $recpolicy2 = Infoblox::Grid::RecordNamePolicy->new(
    regex => ".+",
    name  => "Allow Any",
    );
```

```
my $recpolicy3 = Infoblox::Grid::RecordNamePolicy->new(
    regex => "^[-a-zA-Z0-9_.]+\$",
    name  => "Allow Underscore",
    );
my $recpolicy4 = Infoblox::Grid::RecordNamePolicy->new(
    regex => "^[a-zA-Z0-9]\$|^[a-zA-Z0-9][-a-zA-Z0-9.]*[a-zA-Z0-9]\$",
    name  => "Strict Hostname Checking",
    );

$grid_dns->record_name_policies([$recpolicy1, $recpolicy2, $recpolicy3, $recpolicy4]);
```

recursive_query_list( )

Use this method to set or retrieve the recursive query list of a Grid DNS object. This list specifies from which IP addresses, networks and TSIG key authenticated clients recursive queries are allowed.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

Valid value is an Infoblox::Grid::NamedACL object or an array reference that contains IPv4/IPv6 addresses, networks, and/or Infoblox::DNS::TSIGKey objects. To deny queries, add "!" as a prefix to IP addresses and/or networks. Specify "any" to allow recursive queries from any IP address.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get the recursive query list
my $recursive_query_list = $grid_dns->recursive_query_list();
#Modify the recursive query list
$grid_dns->recursive_query_list(["any"]);

#Creating a TSIG key object
my $tsig_key2 = $session->gen_tsig_key();

my $key2 = Infoblox::DNS::TSIGKey->new(
    name => "rcl_key",
    key  => $tsig_key2
    );
#Modify the recursive query list
$grid_dns->recursive_query_list(["10.0.0.10", "20.0.0.0/24","2010::256/64", $key2,
"!"."2011::23cb/64","!"."30.0.0.30", "!"."40.10.0.0/24"]);
#Modify the recursive query list
$grid_dns->recursive_query_list(undef);
#Use Infoblox::Grid::NamedACL
$grid_dns->recursive_query_list($nacl);
```

refresh_timer( )

Use this method to set or retrieve the refresh time. This interval tells the secondary how often to send a message to the primary for a zone to check that its data is current, and retrieve fresh data if it is not.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

Number with the seconds for the refresh time. *Default* value is 10800 seconds (3 hours).

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get the refresh time
my $refresh_timer = $grid_dns->refresh_timer();
#Modify the refresh time
$grid_dns->refresh_timer(108);
```

resolver_query_timeout( )

Use this method to set or retrieve recursive query timeout.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

The valid value is an unsigned integer that contains the recursive query time (in seconds) for the resolver.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get resolver_query_timeout
my $resolver_query_timeout = $grid_dns->resolver_query_timeout();
#Modify resolver_query_timeout
$grid_dns->resolver_query_timeout(29);
```

response_rate_limiting( )

Use this method to set or retrieve response rate limiting settings.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

The valid value is an Infoblox::Grid::DNS::ResponseRateLimiting object.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get response_rate_limiting
my $response_rate_limiting = $grid_dns->response_rate_limiting();
#Modify response_rate_limiting
$grid_dns->response_rate_limiting($response_rate_limiting);
```

restart_setting( )

Use this method to set or retrive Grid DNS service restart settings.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

The valid value is an Infoblox::Grid::ServiceRestart object.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get restart_setting
my $restart_setting = $grid_dns->restart_setting();
#Modify the retry time
$grid_dns->restart_setting($restart_setting);
```

retry_timer( )

Use this method to set or retrieve the retry time. This interval tells the secondary how long to wait before attempting to recontact the primary after a connection failure occurs between the two servers.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

Number with the seconds for the retry time. *Default* value is 3600 seconds (1 hour).

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get the retry time
my $retry_timer = $grid_dns->retry_timer();
#Modify the retry time
$grid_dns->retry_timer(360);
```

rootNS( )

Use this method to set or retrieve the list of customized root nameserver(s). You can use Internet root name servers or specify host names and IP addresses of custom root name servers.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

Valid value is an array reference that contains Infoblox::Grid::RootNameServer object(s).

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get the root nameserver list
my $rootNS_list = $grid_dns->rootNS();
#Modify the root nameserver list
#Creating RootNameServer objects.
my $rootns1 = Infoblox::Grid::RootNameServer->new(
    ipv4addr  => "44.44.44.44",
    host_name => "test1.com",
    );

my $rootns2 = Infoblox::Grid::RootNameServer->new(
    ipv4addr  => "55.5.5.5",
    host_name => "test2.com",
    );

$grid_dns->rootNS([$rootns1, $rootns2]);
```

serial_query_rate( )

Use this method to set or retrieve the number of maximum concurrent SOA queries per second for the Grid.

Parameter

Valid values are unsigned integer between 20 and 1000, inclusive. Default value is 20.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
# Get attribute value
my $value = $grid_dns->serial_query_rate();
# Modify attribute value
$grid_dns->serial_query_rate(30);
```

server_id_directive( )

Use this method to set or retrieve the value for the BIND and UNBOUND 'server-id' directive.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

The valid values are "NONE" and "HOSTNAME". The default value is "NONE".

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get server_id_directive
my $server_id_directive = $grid_dns->server_id_directive();
#Modify server_id_directive
$grid_dns->server_id_directive("HOSTNAME");
```

sortlist( )

Use this method to set or retrieve the sort list of a grid DNS object. A sort list sorts the order of addresses in responses made to DNS queries.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

Valid value is an array reference that contains Infoblox::DNS::Sortlist object(s).

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get the sort list
my $sortlist = $grid_dns->sortlist();
#Modify the sort list
#Creating a SortList object.
my $sort1 = Infoblox::DNS::Sortlist->new(
    match_list      => ["10.20.1.0/24", "10.20.2.0/24"],
    source_ipv4addr => "1.2.3.4"
    );

$grid_dns->sortlist([$sort1]);
```

store_locally( )

Use this method to set or retrieve the flag that indicates whether the storing of query capture reports on the appliance is enabled or disabled.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

Specify 'true' to enable storing of query capture reports on the appliance or 'false' to disable it. The default value is 'false'.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get store_locally value
my $store_locally = $object->store_locally();

#Modify store_locally value
$object->store_locally('true');
```

transfer_excluded_servers( )

Use this method to set or retrieve the list of excluded servers for zone transfers of a grid DNS object. This list exclude the specified DNS servers during zone transfers.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

The valid value is an array reference that contains IP addresses of DNS servers.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get the transfer excluded server list
my $transfer_excluded_servers = $grid_dns->transfer_excluded_servers();
#Modify the transfer excluded server list
$grid_dns->transfer_excluded_servers(["10.0.0.10", "20.0.0.0"]);
```

transfer_format( )

Use this method to set or retrieve the zone transfer format. The zone transfer format determines the BIND format for a zone transfer. This provides tracking capabilities for single or multiple transfers and their associated servers.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

Valid value is one of the following:

"many_answers" - Includes as many records as the packet size allows.

"one_answer" - Includes one record per packet.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get the zone transfer format
my $transfer_format = $grid_dns->transfer_format();
#Modify the zone transfer format
$grid_dns->transfer_format("many_answers");
```

transfers_in( )

Use this method to set or retrieve the number of maximum concurrent transfers for the Grid.

Parameter

Valid values are unsigned integer between 10 and 100, inclusive. Default value is 10.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
# Get attribute value
my $value = $grid_dns->transfers_in();
# Modify attribute value
$grid_dns->transfers_in(20);
```

transfers_out( )

Use this method to set or retrieve the number of maximum outbound concurrent zone transfers for the Grid.

Parameter

Valid values are unsigned integers between 1 and 100, inclusive. The default value is 10.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
# Get attribute value
my $value = $grid_dns->transfers_out();
# Modify attribute value
```

```
$grid_dns->transfers_out(20);
```

transfers_per_ns( )

Use this method to set or retrieve the number of maximum concurrent transfers per member for the Grid.

Parameter

Valid values are unsigned integer between 2 and 100, inclusive. Default value is 2.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
# Get attribute value
my $value = $grid_dns->transfers_per_ns();
# Modify attribute value
$grid_dns->transfers_per_ns(3);
```

rpz_disable_nsdname_nsip( )

Use this method to set or retrieve the option to disable NSDNAME and NSIP resource records from RPZ feeds.

Parameter

Specify "true" to enable the filter (to filter out NSDNAME and NSIP resource records). Specify "false" to disable the filter. Valid values are "true" or "false". Default value is "false".

Returns

If you specify a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
# Get attribute value
my $value = $grid_dns->rpz_disable_nsdname_nsip();
# Modify attribute value
$grid_dns->rpz_disable_nsdname_nsip("true");
```

rpz_drop_ip_rule_enabled( )

Use this method to set or retreive the flag that indicates whether the DNS server ignores RPZ-IP triggers with large prefix lengths or not.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

Specify 'true' to enable ignoring of RPZ-IP triggers with large prefix lengths or 'false' to disable it. The default value is 'false'.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
# Get attribute value
my $value = $grid_dns->rpz_drop_ip_rule_enabled();
# Modify attribute value
$grid_dns->rpz_drop_ip_rule_enabled("true");
```

rpz_drop_ip_rule_min_prefix_length_ipv4( )

Use this method to set or retreive minimum prefix length for IPv4 RPZ-IP triggers.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

The valid value is an unsigned integer between 1 and 31 that represents prefix length (in bits) for IPv4 RPZ-IP triggers.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
# Get attribute value
my $value = $grid_dns->rpz_drop_ip_rule_min_prefix_length_ipv4();
# Modify attribute value
$grid_dns->rpz_drop_ip_rule_min_prefix_length_ipv4("16");
```

rpz_drop_ip_rule_min_prefix_length_ipv6( )

Use this method to set or retreive minimum prefix length for IPv6 RPZ-IP triggers.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

The valid value is an unsigned integer between 1 and 127 that represents prefix length (in bits) for IPv6 RPZ-IP triggers.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
# Get attribute value
my $value = $grid_dns->rpz_drop_ip_rule_min_prefix_length_ipv6();
# Modify attribute value
$grid_dns->rpz_drop_ip_rule_min_prefix_length_ipv6("120");
```

rpz_qname_wait_recurse( )

Use this method to enable or disable recursive RPZ lookups.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

Specify "true" to enable recursive RPZ lookups or "false" to disable them. The default value is "false".

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
# Get attribute value
my $value = $grid_dns->rpz_qname_wait_recurse();
# Modify attribute value
$grid_dns->rpz_qname_wait_recurse("true");
```

zone_deletion_double_confirm( )

Use this method to enable or disable double confirmation during zone deletion.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

Specify "true" to enable the zone_deletion_double_confirm flag or "false" to deactivate/unset it. The default value is "true".

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get the double confirmation of zone deletion flag
my $zone_deletion_double_confirm = $grid_dns->zone_deletion_double_confirm();
```

```
#Modify the double confirmation of zone deletion flag
$grid_dns->zone_deletion_double_confirm("false");
```

SAMPLE CODE

The following sample code demonstrates the different functions that can be applied to an object, such as get and modify. This sample code also includes error handling for the operations.

#Preparation prior to getting and modifying a grid DNS object

```
use strict;
use Infoblox;

#Creating a session to appliance.
my $session = Infoblox::Session->new(
    master   => "192.168.1.2",
    username => "admin",
    password => "infoblox"
    );
unless( $session ){
    die("Constructor for session failed:",
    Infoblox::status_code() . ":" . Infoblox::status_detail());
}
print" Session object created successfully \n";

#Creating RootNameServer objects.
my $rootns1 = Infoblox::Grid::RootNameServer->new(
    ipv4addr  => "44.44.44.44",
    host_name => "test1.com",
    );
unless( $rootns1 ){
    die("Constructor for rootnameserver1 failed:",
    Infoblox::status_code() . ":" . Infoblox::status_detail());
}
print" Rootnameserver1 object created successfully \n";

my $rootns2 = Infoblox::Grid::RootNameServer->new(
    ipv4addr  => "55.5.5.5",
    host_name => "test2.com",
    );
unless( $rootns2 ){
    die("Constructor for rootnameserver2 failed:",
    Infoblox::status_code() . ":" . Infoblox::status_detail());
}
print" Rootnameserver2 object created successfully \n";

#Creating Transfer Key objects.
my $key1 = Infoblox::DNS::TSIGKey->new(
    name => "tsig_xfer",
    key  => ":2xCOMPAT"
    );
unless( $key1 ){
    die("Constructor for transfer key \(key1\) failed:",
    Infoblox::status_code() . ":" . Infoblox::status_detail());
}
print" Transfer key object \(key1\) created successfully \n";

my $tsig_key1 = $session->gen_tsig_key();
unless( $tsig_key1 ){
    die("Generate tsig_key1 failed: ",
    $session->status_code() . ":" . $session->status_detail());
}

my $key2 = Infoblox::DNS::TSIGKey->new(
    name => "transfer_key",
    key  => $tsig_key1
    );
unless( $key2 ){
    die("Constructor for transfer key \(key2\) failed:",
    Infoblox::status_code() . ":" . Infoblox::status_detail());
}
print" Transfer key object \(key2\) created successfully \n";

#Creating Update Key object.
my $update_key1 = $session->gen_tsig_key();
```

```
unless( $update_key1 ){
    die("Generate update_key1 failed:",
    $session->status_code() . ":" . $session->status_detail());
}

my $up_key1 = Infoblox::DNS::TSIGKey->new(
    name => "update_key",
    key  => $update_key1
    );
unless( $up_key1 ){
    die("Constructor for update key failed: ",
    Infoblox::status_code() . ":" . Infoblox::status_detail());
}
print" Update key object created successfully \n";

#Creating a SortList object.
my $sort1 = Infoblox::DNS::Sortlist->new(
    match_list      => ["10.20.1.0/24", "10.20.2.0/24"],
    source_ipv4addr => "1.2.3.4"
    );
unless( $sort1 ){
    die("Constructor for sortlist failed: ",
    Infoblox::status_code() . ":" . Infoblox::status_detail());
}
print" Sortlist object created successfully \n";
```

#Get and modify the DNS object

```
#Getting the DNS object from appliance through session.
my @result = $session->get(
    object => "Infoblox::Grid::DNS",
    name   => "Infoblox"
    );
unless( @result ){
    die("Get grid DNS failed: ",
    $session->status_code() . ":" . $session->status_detail());
}
print" Get grid DNS object found at least 1 matching entry \n";

my $grid_dns = $result[0];
if ($grid_dns) {

#Modifying the value of the specified object
   $grid_dns->allow_query(["10.0.0.10", "20.0.0.0/24", "!" . "30.0.0.30", "!" . "40.10.0.0/24"]);
   $grid_dns->allow_recursive_query("true");
   $grid_dns->allow_transfer(["10.0.0.10", "20.0.0.0/24", $key1, "!"."30.0.0.30", "!"."40.10.0.0/24",
$key2]);
   $grid_dns->allow_update(["10.0.0.10", "20.0.0.0/24", "!"."30.0.0.30", "!"."40.10.0.0/24", $up_key1]);
   $grid_dns->default_ttl(28800);
   $grid_dns->expire_after(2419200);
   $grid_dns->forwarders(["2.2.2.2", "3.3.3.3"]);
   $grid_dns->logging_facility("daemon");
   $grid_dns->negative_ttl(900);
   $grid_dns->recursive_query_list(["10.0.0.10", "20.0.0.0/24", "!" . "30.0.0.30", "!" . "40.10.0.0/24"]);
   $grid_dns->refresh_timer(10800);
   $grid_dns->retry_timer(3600);
   $grid_dns->rootNS([$rootns1, $rootns2]);
   $grid_dns->sortlist([$sort1]);
   $grid_dns->transfer_excluded_servers(["10.0.0.10", "20.0.0.0"]);
   $grid_dns->transfer_format("many_answers");

 #Applying the changes
 my $response = $session->modify($grid_dns);
 unless( $response ){
    die("Modify grid DNS failed: ",
    $session->status_code() . ":" . $session->status_detail());
 }
 print" Modify grid DNS successful \n";
  }
  else {
   print "No grid DNS object found.";
  }

####PROGRAM ENDS####
```

AUTHOR

Infoblox Inc. http://www.infoblox.com/

SEE ALSO

Infoblox::DNS::Sortlist, Infoblox::DNS::TSIGKey, Infoblox::DNS::DnssecTrustedKey, Infoblox::Grid::DNS::Nsgroup, Infoblox::Grid::RootNameServer, Infoblox::DNS::Ruleset, Infoblox::Session->get(), Infoblox::Session->modify(), Infoblox::Session, Infoblox::Grid::NamedACL, Infoblox::DNS::DNSSecKeyAlgorithm, Infoblox::Grid::DNS::FixedRRSetOrderFQDN, Infoblox::Grid::DNS::ResponseRateLimiting, Infoblox::Grid::DNS::FileTransferSetting, Infoblox::Grid::DNS::ClientSubnetDomain

---

COPYRIGHT

Copyright (c) 2017 Infoblox Inc.

Infoblox::DNS::DDNS::PrincipalCluster - A DDNS Principal Cluster object.

- NAME
- DESCRIPTION
- CONSTRUCTOR
- SESSION METHODS
  - Infoblox::Session->add( )
  - Infoblox::Session->get( )
  - Infoblox::Session->modify( )
  - Infoblox::Session->remove( )
  - Infoblox::Session->search( )
- MODULE METHODS
  - Infoblox::DNS::DDNS::PrincipalCluster::Group->clusters( )
- METHODS
  - comment( )
  - group( )
  - name( )
  - principals( )
- AUTHOR
- SEE ALSO
- COPYRIGHT

NAME

Infoblox::DNS::DDNS::PrincipalCluster - A DDNS Principal Cluster object.

DESCRIPTION

The DDNS Principal Cluster object represents a set of principals such that any principal in a DDNS Principal Cluster can update records created by any other principal in the same cluster.

CONSTRUCTOR

```
my $cluster = Infoblox::DNS::DDNS::PrincipalCluster->new(
    name       => $string,                    # Required
    group      => $string,                    # Required
    comment    => $string,                    # Optional / Default is undefined
    principals => [$string1, $string2, ...], # Optional / Default is undefined
);
```

SESSION METHODS

This section describes all the methods in a Session module that you can apply to a DDNS Principal Cluster object.

Infoblox::Session->add( )

Use this method to add an object to the Infoblox appliance. See Infoblox::Session->add() for parameters and return values.

Example

```
#create DDNS principal cluster group

my $cluster_group = Infoblox::DNS::DDNS::PrincipalCluster::Group->new(
    name => 'cluster_group',
);

#submit for addition
my $response = $session->add($cluster_group);

#construct an object
my $object = Infoblox::DNS::DDNS::PrincipalCluster->new(
   name  => 'cluster1',
   group => 'cluster_group',
);

#submit for addition
my $response = $session->add($object);
```

Infoblox::Session->get( )

Use this method to retrieve all the matching objects from the Infoblox appliance. See Infoblox::Session->get() for parameters and return values.

Key Reference

```
Apply the following attribute to get a specific object:

 name    - Optional. A DDNS Principal cluster name in a string format.
 comment - Optional. A DDNS Principal cluster comment.
 group   - Optional. A DDNS Principal cluster group name.
```

Example

```
my @retrieved_objs = $session->get(
    object => 'Infoblox::DNS::DDNS::PrincipalCluster',
    name   => 'cluster1',
    group  => 'cluster_group',
);
```

Infoblox::Session->modify( )

Use this method to modify an object in the Infoblox appliance. See Infoblox::Session->modify() for parameters and return values.

Example

```
#Use method to modify the comment.
$object->comment('this is a modified comment');
#Submit modification
my $response = $session->modify($object);
```

Infoblox::Session->remove( )

Use this method to remove an object from the Infoblox appliance. See Infoblox::Session->remove() for parameters and return values.

To remove a specific object, first use `get()` or `search()` to retrieve the specific object, and then submit this object for removal.

Example

```
#Get the objects with the same name
my @retrieved_objs = $session->get(
    object => 'Infoblox::DNS::DDNS::PrincipalCluster',
    name   => 'cluster1',
);
#Find the desired object from the retrieved list.
my $desired_cluster = $retrieved_objs[0];
#Submit for removal
my $response = $session->remove($desired_cluster);
```

Infoblox::Session->search( )

Use this method to search for objects in the Infoblox appliance. See Infoblox::Session->search() for parameters and return values.

Key Reference

```
Apply the following attribute to get a specific object:

 name    - Optional. A DDNS Principal Cluster name in a string format.
 comment - Optional. A DDNS Principal Cluster comment.
 group   - Optional. A DDNS Principal cluster group name.
```

Example

```
my @retrieved_objs = $session->get(
    object => 'Infoblox::DNS::DDNS::PrincipalCluster',
    name   => 'group11',
);
```

MODULE METHODS

Infoblox::DNS::DDNS::PrincipalCluster::Group->clusters( )

Use this method to retrieve a list of DDNS Principal Clusters. See Infoblox::DNS::DDNS::PrincipalCluster::Group->clusters() for parameters and return values.

Example

```
#get clusters value
```

```
my $clusters = $object->clusters();
```

METHODS

This section describes all the methods that you can use to set or retrieve the attribute values of the object.

comment( )

Use this method to set or retrieve a descriptive comment.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

The valid value is a comment in string format (UTF-8) with a maximum of 256 bytes.

Returns

If you specified a parameter, the method returns 'true' when the modification succeeds, and returns 'false' when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#get comment value
my $comment = $object->comment();

#modify comment value
$object->comment('desired comment');
```

group( )

Use this method to set or retrieve the DDNS Principal Cluster group the cluster associated with.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

The valid value is a DDNS Principal Cluster group name in string format (UTF-8) with a maximum of 256 bytes.

Returns

If you specified a parameter, the method returns 'true' when the modification succeeds, and returns 'false' when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#get group value
my $group = $object->group();

#modify group value
$object->group('desired_group');
```

name( )

Use this method to set or retrieve the DDNS Principal Cluster name.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

Desired name in string format with a maximum of 256 bytes.

Returns

If you specified a parameter, the method returns 'true' when the modification succeeds, and returns 'false' when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#get name value
my $name = $object->name();
```

```
#modify name value
$object->name('foo');
```

principals( )

Use this method to set or retrieve a list of equivalent principals.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

The valid value is an array of desired principal names in string format.

Returns

If you specified a parameter, the method returns 'true' when the modification succeeds, and returns 'false' when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#get principals value
my $principals = $object->principals();

#modify principals value
$object->principals(['foo', 'bar']);
```

AUTHOR

Infoblox Inc. http://www.infoblox.com/

SEE ALSO

Infoblox::Session, Infoblox::Session->add(), Infoblox::Session->get(), Infoblox::Session->modify(), Infoblox::Session->remove(), Infoblox::Session->search(), Infoblox::DNS::DDNS::PrincipalCluster::Group, Infoblox::DNS::DDNS::PrincipalCluster::Group->clusters().

COPYRIGHT

Copyright (c) 2017 Infoblox Inc.

Infoblox::Grid::ScheduledTask - scheduled task object.

- NAME
- DESCRIPTION
- CONSTRUCTOR
- SESSION METHODS
  - Infoblox::Session->get( )
  - Infoblox::Session->modify( )
  - Infoblox::Session->remove( )
  - Infoblox::Session->search( )
- METHODS
  - approval_status( )
  - approver( )
  - approver_comment( )
  - changed_objects( )
  - dependent_tasks( )
  - execution_details( )
  - execution_details_type( )
  - execution_status( )
  - execution_time( )
  - member( )
  - is_network_insight_task( )
  - re_execute_task( )
  - scheduled_time( )
  - submit_time( )
  - submitter_comment( )
  - submitter( )
  - task_id( )
  - ticket_number( )
  - task_type( )
  - time_zone( )
  - predecessor_task( )
- SAMPLE CODE
- SCHEDULED DATE AND TIME FORMAT
- AUTHOR
- SEE ALSO
- COPYRIGHT

NAME

Infoblox::Grid::ScheduledTask - scheduled task object.

DESCRIPTION

A scheduled task object is used to retrieve information about scheduled tasks. Scheduled tasks are commands that are scheduled by the Infoblox::Session methods, Infoblox::Session->add(), Infoblox::Session->modify() and Infoblox::Session->remove(), and are called with the scheduled_at parameter.

CONSTRUCTOR

The Infoblox::Grid::ScheduledTask object is automatically generated after the Infoblox::Session methods, Infoblox::Session->add(), Infoblox::Session->modify() and Infoblox::Session->remove(), are called with the scheduled_at parameter. It does not require manual construction.

SESSION METHODS

This section describes all the methods in the Infoblox::Session module that you can apply to a scheduled task object.

Infoblox::Session->get( )

Use this method to retrieve all the matching objects from the Infoblox appliance. See Infoblox::Session->get() for parameters and return values.

Key References

Apply the following attributes to get a specific scheduled task object:

member - The Grid member name.

scheduled_time - The time when the task is scheduled to occur.

submit_time - The time when the scheduled task was submitted.

submitter - The name of the user who submitted the task (regular expression).

task_id - The identifier of a scheduled task.

action - The type of action performed. When used, the search returns ScheduledTask objects that have ScheduledTask::ChangedObject objects in the changed_objects list with matching 'action' attributes. Possible values are:

```
Add
Modify
Delete
Network Discovery
Restart Services
```

object_name - The name of the changed object (regular expression). When used, the search returns ScheduledTask objects that have ScheduledTask::ChangedObject objects in the changed_objects list with matching 'object_name' attirbutes.

object_type - The type of the changed object (regular expression). When used, the search returns ScheduledTask objects that have ScheduledTask::ChangedObject objects in the changed_objects list with matching 'object_type' attributes. Possible values are:

```
A Record
AAAA Record
Authoritative Zone
Bulk Host
CNAME Record
Delegated Zone
DHCP Range
DNAME Record
DNS View
Fixed Address
Forward Zone
Host Record
IPv4 Network
IPv4 Network Container
IPv6 Network
IPv6 Network Container
MX Record
NS Record
PTR Record
Reservation
Roaming Host
Shared A Record
Shared AAAA Record
Shared MX Record
Shared Network
Shared Record Group
Shared SRV Record
Shared TXT Record
SRV Record
Stub Zone
TXT Record
```

Example

```
my @retrieved_objs = $session->get(
    object     => "Infoblox::Grid::ScheduledTask",
    task_id    => "34" );
```

Infoblox::Session->modify( )

Use this method to modify an object in the Infoblox appliance. See Infoblox::Session->modify() for parameters and return values.

Example

```
# Use this method to modify the object.
$task->scheduled_time( "2008-10-21T10:56:33Z" );
# Submit modification
my $response = $session->modify( $task );
```

Infoblox::Session->remove( )

Use this method to remove an object from the Infoblox appliance. See Infoblox::Session->remove() for parameters and return values.

To remove a specific object, use `get()` or `search()` to retrieve the specific object first, and then submit this object for removal.

Example

```
# Get the ScheduledTask objects
my @retrieved_objs = $session->get(
    object     => "Infoblox::Grid::ScheduledTask",
    submitter  => "admin" );
# find the desired object from the retrieved list.
```

```
my $task = $retrieved_objs[0];
# Submit for removal
my $response = $session->remove( $task );
```

Infoblox::Session->search( )

Use this method to search for scheduled task objects in the Infoblox appliance. See Infoblox::Session->search() for parameters and return values.

Key References

The search can be performed on any combination of the following attributes:

member - The Grid member name.

scheduled_time - The time when the task is scheduled to occur. This method supports searching with limit parameters (see below).

submit_time - The time when the scheduled task was submitted. This method supports searching with limit parameters (see below).

submitter - The name of the user who submitted the task (regular expression).

task_id - The identifier of a scheduled task.

action - The type of action performed. When used, the search returns ScheduledTask objects that have ScheduledTask::ChangedObject objects in the changed_objects list with matching 'action' attributes. Possible values are:

```
Add
Modify
Delete
Network Discovery
Restart Services
```

object_name - The name of the changed object (regular expression). When used, the search returns ScheduledTask objects that have ScheduledTask::ChangedObject objects in the changed_objects list with matching 'object_name' attirbutes.

object_type - The type of the changed object (regular expression). When used, the search returns ScheduledTask objects that have ScheduledTask::ChangedObject objects in the changed_objects list with matching 'object_type' attributes. Possible values are:

```
A Record
AAAA Record
Authoritative Zone
Bulk Host
CNAME Record
Delegated Zone
DHCP Range
DNAME Record
DNS View
Fixed Address
Forward Zone
Host Record
IPv4 Network
IPv4 Network Container
IPv6 Network
IPv6 Network Container
MX Record
NS Record
PTR Record
Reservation
Roaming Host
Shared A Record
Shared AAAA Record
Shared MX Record
Shared Network
Shared Record Group
Shared SRV Record
Shared TXT Record
SRV Record
Stub Zone
TXT Record
```

See Infoblox::Session/search with limit parameters for more information on this functionality.

Example

```
# search for all ScheduledTask objects
my @retrieved_objs = $session->search(
    object  => "Infoblox::Grid::ScheduledTask",
    scheduled_time => ">=< 2008-10-21T10:56:33Z,2008-11-21T10:56:33Z");
```

METHODS

This section describes all the methods that you can use to set and retrieve the attribute values of a ScheduledTask object.

approval_status( )

Use this method to set or retrieve the approval status if approvals are required for this task.

Parameter

Valid values are 'NONE', 'APPROVED', 'PENDING' and 'REJECTED'.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get approval_status value
my $approval_status = $task->approval_status();

#Modify approval_status value
$task->approval_status('APPROVED');
```

approver( )

Use this method to retrieve the username of the approver. This is a read-only attribute.

Parameter

none

Returns

The method returns the attribute value.

Example

```
#Get approver value
my $approver = $task->approver();
```

approver_comment( )

Use this method to set or retrieve the approver comment.

Parameter

The approver comment in string format.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get approver_comment value
my $approver_comment = $task->approver_comment();

#Modify approver_comment value
$task->approver_comment('ok');
```

changed_objects( )

Use this method to retrieve information about the objects that are scheduled to be changed.

Parameter

none

Returns

The method returns a reference to an array of Infoblox::Grid::ScheduledTask::ChangedObject objects.

Example

```
#Get changed_objects
my $changed_objects = $task->changed_objects();
my $obj = $changed_objects->[0];
```

dependent_tasks( )

Use this method to retrieve an array of of tasks dependent on this one. This is a read-only attribute.

Parameter

None

Returns

The method returns a reference to an array of Infoblox::Grid::ScheduledTask objects.

Example

```
#Get array of ScheduledTask objects
my @dependent_tasks = $task->dependent_tasks();
```

execution_details( )

Use this method to retrieve the list of details generated by the execution. This is a read-only field.

Parameters

None

Returns

The method returns the array of strings that contains details generated by the execution.

Example

```
#Get execution_details value
my $details = $task->execution_details();
```

execution_details_type( )

Use this method to retrieve the type of details generated by the execution after the task has been completed. This is a read-only field.

Parameters

None

Returns

The method returns the attribute value that can be 'NONE' or 'WARNING'.

Example

```
#Get execution_details_type value
my $details = $task->execution_details_type();
```

execution_status( )

Use this method to retrieve the status of the task. This is a read-only attribute.

Parameter

none

Returns

The method returns the attribute value that can be 'COMPLETED', 'FAILED' or 'PENDING'.

Example

```
#Get execution_status
my $execution_status = $task->execution_status();
```

execution_time( )

Use this method to retrieve the execution time. This is a read-only attribute.

Parameter

none

Returns

The method returns the attribute value.

Example

```
#Get execution_time
my $execution_time = $task->execution_time();
```

member( )

Use this method to retrieve the Grid member that has created this task. This is a read-only attribute.

Parameter

None

Returns

The method returns the attribute value.

Example

```
#Get member
my $member = $task->member();
```

is_network_insight_task( )

Use this method to retrieve the flag that indicates if this task is related to network insight. This is a read-only attribute.

Parameter

none

Returns

This method returns the attribute value.

Example

```
#Get value
$value = $task->is_network_insight_task();
```

re_execute_task( )

Use this method to set failed discovery task to be re-executed.

If this flag is set to 'true' a failed Network Insight discovery task will be re-executed, a new task will be created cloned from this one and restarted.

Parameter

Boolean to set failed netmri task to be re-executed. Only "true" value makes sense.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
# Get value
$value = $task->re_execute_task();

# Set value
$task->re_execute_task('true');
```

scheduled_time( )

Use this method to set or retrieve the date and time the task is scheduled to occur.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

Scheduled date and time in ISO 8601 extended format (e.g., 2008-10-21T10:56:33Z or 2008-10-21T10:56:33.001Z).

Use case insensitive value "now" to execute a scheduled task immediately.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get scheduled_time value
my $scheduled_time = $task->scheduled_time();

#Modify scheduled_time value
$task->scheduled_time("2008-10-21T10:56:33Z");
# Submit modification
$session->modify($task);

# Do it now
$task->scheduled_time("Now");
# Submit modification
$session->modify($task);
```

submit_time( )

Use this method to retrieve the time the scheduled task was submitted (when appropriate Session method was called with parameter scheduled_time).

Parameter

none

Returns

The method returns the submitted date and time in ISO 8601 extended format (e.g., 2008-10-21T10:56:00Z or 2008-10-21T10:56:00.001Z).

Example

```
#Get submit_time
my $submit_time = $task->submit_time();
```

submitter_comment( )

Use this method to set or retrieve the submitter comment.

Parameter

The submitter comment in string format.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get submitter_comment value
my $submitter_comment = $task->submitter_comment();

#Modify submitter_comment value
$task->submitter_comment('ok');
```

submitter( )

Use this method to retrieve the name of the user who submitted the task.

Parameter

none

Returns

The method returns the user name in string format.

Example

```
#Get submitter
my $submitter = $task->submitter();
```

task_id( )

Use this method to retrieve the identifier of a scheduled task.

Parameter

none

Returns

The method returns the identifier in numeric format.

Example

```
#Get task_id
my $task_id = $task->task_id();
```

ticket_number( )

Use this method to set or retrieve the ticket number.

Parameter

The ticket number in string format.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get ticket_number value
my $ticket_number = $task->ticket_number();

#Modify ticket_number value
$task->ticket_number('ok');
```

task_type( )

Use this method to retrieve the scheduled task type. This is a read-only attribute.

Parameter

none

Returns

The method returns the attribute value. Valid return value is "OBJECT_CHANGE" and "PORT_CONTROL".

Example

```
#Get task_type value
my $task_type = $task->task_type();
```

time_zone( )

Use this method to set or retrieve the time zone.

Parameter

The UTC string that represents the time zone. For example "(UTC - 6:00) Central Time (US and Canada)". The default value is "(UTC) Coordinated Universal Time".

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get attribute value
my $value = $schedule_setting->time_zone();

#Modify attribute value
$schedule_setting->time_zone("(UTC - 6:00) Central Time (US and Canada)");
```

predecessor_task( )

Use this method to retrieve the predecessor task. This is a read-only attribute.

Parameter

none

Returns

The method returns an Infoblox::Grid::ScheduledTask object.

Example

```
#Get predecessor_task value
my $predecessor_task = $task->predecessor_task();
```

SAMPLE CODE

The following sample code demonstrates the different functions that can be applied to an object, such as add, search, modify, and remove. This sample also includes error handling for the operations.

#Preparation prior to a ScheduledTask object insertion

```
#PROGRAM STARTS: Include all the modules that will be used
use strict;
use Infoblox;

#Create a session to the Infoblox appliance

my $session = Infoblox::Session->new(
               master   => "192.168.1.2", #appliance host ip
               username => "admin",       #appliance user login
               password => "infoblox"     #appliance password
               );

unless ($session) {
       die("Construct session failed: ",
               Infoblox::status_code() . ":" . Infoblox::status_detail());
}
print "Session created successfully\n";

# Add a network
my $network = Infoblox::DHCP::Network->new(
    network => "10.0.0.0/24",
);
unless ($network) {
    die("Construct Network failed: ",
        Infoblox::status_code() . ":" . Infoblox::status_detail());
}
print "Network created successfully\n";

$session->add($network)
    or die("Add Network object failed: ",
    $session->status_code() . ":" . $session->status_detail());
print "Network object added to server successfully\n";
```

#Create a ScheduledTask object

```
# Create a fixed address
my $fixed_addr = Infoblox::DHCP::FixedAddr->new(
    ipv4addr    => '10.0.0.3',
    mac         => '11:22:33:44:55:66'
);
unless ($fixed_addr) {
    die("Construct FixedAddr failed: ",
        Infoblox::status_code() . ":" . Infoblox::status_detail());
}
print "FixedAddr created successfully\n";

# Schedule the addition of a fixed address
$session->add($fixed_addr, scheduled_at => "2020-01-01T14:52:00Z")
    or die("Schedule fixed address add failed: ",
    $session->status_code() . ":" . $session->status_detail());
print "Fixed address add scheduled successfully\n";
```

#Search for a ScheduledTask object

```
my @retrieved_objs = $session->search(
    object    => "Infoblox::Grid::ScheduledTask",
    submitter => "admin"
);
my $object = $retrieved_objs[0];

unless ($object) {
    die("Search for a ScheduledTask object failed: ",
        $session->status_code() . ":" . $session->status_detail());
}
print "Search for a ScheduledTask object found at least 1 matching entry\n";
```

#Get and modify a ScheduledTask object

```
#Get the ScheduledTask object from Infoblox appliance through a session
my @retrieved_objs = $session->get(
    object         => "Infoblox::Grid::ScheduledTask",
    scheduled_time => "2020-01-01T14:52:33Z"
);
my $object = $retrieved_objs[0];

unless ($object) {
    die("Get ScheduledTask object failed: ",
        $session->status_code() . ":" . $session->status_detail());
}
print "Get ScheduledTask object found at least 1 matching entry\n";

#Modify the ScheduledTask object
$object->scheduled_time("2020-02-01T14:52:33Z");

#Apply the changes.
$session->modify($object)
    or die("Modify ScheduledTask object failed: ",
    $session->status_code() . ":" . $session->status_detail());
print "ScheduledTask object modified successfully \n";
```

#Remove a ScheduledTask object

```
#Get the ScheduledTask object through the session
my @retrieved_objs = $session->get(
    object => "Infoblox::Grid::ScheduledTask",
    action => "Add"
);
my $object = $retrieved_objs[0];
unless ($object) {
    die("Get ScheduledTask object failed: ",
        $session->status_code() . ":" . $session->status_detail());
}
print "Get ScheduledTask object found at least 1 matching entry\n";

#Submit the object for removal
$session->remove($object)
    or die("Remove ScheduledTask object failed: ",
    $session->status_code() . ":" . $session->status_detail());
print "ScheduledTask object removed successfully \n";
```

#Cleanup

```
#Get the Network object through the session
my @retrieved_objs = $session->get(
    object  => "Infoblox::DHCP::Network",
    network => "10.0.0.0/24"
);
my $object = $retrieved_objs[0];
unless ($object) {
    die("Get Network object failed: ",
        $session->status_code() . ":" . $session->status_detail());
}
print "Get Network object found at least 1 matching entry\n";

#Submit the object for removal
$session->remove($object)
    or die("Remove Network object failed: ",
    $session->status_code() . ":" . $session->status_detail());
print "Network object removed successfully \n";
```

```
####PROGRAM ENDS####
```

SCHEDULED DATE AND TIME FORMAT

Scheduled date and time are represented in ISO 8601 extended format (e.g., 2008-10-21T10:56:00Z or 2008-10-21T10:56:00.001Z) with some restrictions:

- Maximal value for year field is 2037.

When the scheduling parameter is used, changes to objects do not occur immediately. Instead, the Infoblox::Grid::ScheduledTask object is created, and then executed at the time specified in this parameter.

AUTHOR

Infoblox Inc. http://www.infoblox.com/

SEE ALSO

Infoblox::Grid::ScheduledTask, Infoblox::Grid::ScheduledTask::ChangedObject, Infoblox::Session->add(), Infoblox::Session->get(), Infoblox::Session->modify(), Infoblox::Session->remove(), Infoblox::Session->search(),Infoblox::Session

COPYRIGHT

Copyright (c) 2017 Infoblox Inc.

Infoblox::Grid::Member::DNS - Grid Member DNS object.

- NAME
- DESCRIPTION
- CONSTRUCTOR
- SESSION METHODS
  - Infoblox::Session->get( )
  - Infoblox::Session->modify( )
- METHODS
  - anonymize_response_logging( )
  - allow_gss_tsig_zone_updates( )
  - allow_query( )
  - allow_recursive_query( )
  - allow_transfer( )
  - allow_update( )
  - attack_mitigation( )
  - auto_blackhole( )
  - auto_create_a_and_ptr_for_lan2( )
  - auto_create_a_and_ipv6ptr_for_lan2( )
  - auto_sort_views( )
  - blackhole_list( )
  - blacklist_action( )
  - blacklist_log_query( )
  - blacklist_redirect_addresses( )
  - blacklist_redirect_ttl( )
  - blacklist_rulesets( )
  - bind_check_names_policy( )
  - bind_hostname_directive( )
  - bind_hostname_directive_fqdn( )
  - capture_dns_queries_on_all_domains( )
  - copy_xfer_to_notify( )
  - custom_root_name_servers( )
  - disable_edns( )
  - dns_cache_acceleration_ttl( )
  - dns_cache_acceleration_status( )
  - dns_health_check_anycast_control( )
  - dns_health_check_domain_list( )
  - dns_health_check_interval( )
  - dns_health_check_recursion_flag( )
  - dns_health_check_retries( )
  - dns_health_check_timeout( )
  - dns_ip_list( )
  - dns64_groups( )
  - dnssec_enabled( )
  - dnssec_expired_signatures_enabled( )
  - dnssec_validation_enabled( )
  - dnssec_blacklist_enabled()
  - dnssec_negative_trust_anchors( )
  - dnssec_dns64_enabled()
  - dnssec_nxdomain_enabled()
  - dnssec_rpz_enabled()
  - dnssec_trusted_keys( )
  - dns_notify_transfer_source_interface( )
  - dns_query_capture_file_time_limit( )
  - dns_query_source_interface( )
  - dns_view_address_settings( )
  - domains_to_capture_dns_queries( )
  - dtc_edns_prefer_client_subnet( )
  - dtc_health_source_interface( )
  - enable_blackhole( )
  - enable_blacklist( )
  - enable_capture_dns_queries( )
  - enable_capture_dns_responses( )
  - enable_dns( )
  - enable_dns_cache_acceleration( )
  - enable_dns_health_check( )
  - enable_dns64( )
  - enable_excluded_domain_names( )
  - enable_fixed_rrset_order_fqdns( )
  - enable_ftc( )
  - enable_gss_tsig( )
  - enable_query_rewrite( )
  - excluded_domain_names( )

- extattrs( )
- extensible_attributes( )
- facility( )
- file_transfer_setting( )
- filter_aaaa( )
- filter_aaaa_list( )
- fixed_rrset_order_fqdns( )
- forwarders( )
- forward_only( )
- forward_updates( )
- ftc_expired_record_ttl( )
- ftc_expired_record_timeout( )
- glue_record_addresses( )
- gss_tsig_keys( )
- host_name_restriction_policy( )
- ipv6_glue_record_addresses( )
- is_unbound_capable( )
- lame_ttl( )
- logging_categories( )
- max_cache_ttl( )
- max_ncache_ttl( )
- max_cached_lifetime( )
- minimal_resp( )
- name( )
- notify_delay( )
- notify_source_port( )
- override_attack_mitigation( )
- override_auto_blackhole( )
- override_bind_hostname_directive( )
- override_server_id_directive( )
- override_capture_dns_queries_on_all_domains( )
- override_copy_xfer_to_notify( )
- override_disable_edns( )
- override_dns64( )
- override_dns_health_check( )
- override_filter_aaaa( )
- override_dns_cache_acceleration_ttl( )
- override_ftc( )
- override_gss_tsig_keys( )
- override_enable_capture_dns( )
- override_enable_excluded_domain_names( )
- override_enable_gss_tsig( )
- override_max_cache_ttl( )
- override_max_ncache_ttl( )
- override_max_cached_lifetime( )
- override_nxdomain_redirect( )
- override_lame_ttl( )
- override_rpz_qname_wait_recurse( )
- override_transfers_in( )
- override_transfers_out( )
- override_transfers_per_ns( )
- override_serial_query_rate( )
- override_query_rewrite( )
- nxdomain_redirect( )
- nxdomain_redirect_addresses( )
- nxdomain_redirect_addresses_ipv6( )
- nxdomain_redirect_ttl( )
- nxdomain_log_query( )
- nxdomain_rulesets( )
- override_blacklist( )
- override_dnssec( )
- override_dtc_edns_prefer_client_subnet( )
- override_forwarders( )
- override_fixed_rrset_order_fqdns( )
- query_source_port( )
- recursive_client_limit( )
- recursive_query_list( )
- resolver_query_timeout( )
- response_rate_limiting( )
- recursive_resolver( )
- serial_query_rate( )
- server_id_directive( )
- server_id_directive_string( )
- skip_in_grid_rpz_queries( )

- sortlist( )
- store_locally( )
- transfer_excluded_servers( )
- transfer_format( )
- transfers_in( )
- transfers_out( )
- transfers_per_ns( )
- unbound_logging_level( )
- use_lan_ipv6_port( )
- use_lan2_port( )
- use_lan2_ipv6_port( )
- use_mgmt_port( )
- use_mgmt_ipv6_port( )
- use_root_name_servers( )
- rpz_disable_nsdname_nsip( )
- rpz_drop_ip_rule_enabled( )
- rpz_drop_ip_rule_min_prefix_length_ipv4( )
- rpz_drop_ip_rule_min_prefix_length_ipv6( )
- rpz_qname_wait_recurse( )
- override_resolver_query_timeout( )
- override_response_rate_limiting( )
- override_rpz_disable_nsdname_nsip( )
- override_rpz_drop_ip_rule( )
- use_lan_port( )
- views( )

- SAMPLE CODE
- AUTHOR
- SEE ALSO
- COPYRIGHT

NAME

Infoblox::Grid::Member::DNS - Grid Member DNS object.

DESCRIPTION

You can configure the DNS properties of a grid member, including enabling/disabling DNS services and other DNS service parameters. The service configurations of a grid are inherited by all members. For this reason, it is recommended that you configure services at the grid level before configuring member services. When you configure DNS services at the grid level, all grid members inherit the grid-level configuration settings, unless you specifically override them for selected members.

CONSTRUCTOR

The Infoblox::Grid::Member::DNS object is automatically generated upon the successful installation of the Infoblox appliance. It does not require manual construction.

SESSION METHODS

This section describes all the methods in the Infoblox::Session module that you can apply to a grid member DNS object.

Infoblox::Session->get( )

Use this method to retrieve all the matching objects from the Infoblox appliance. See Infoblox::Session->get() for parameters and return values.

Key Reference

Apply the following attributes to get a specific grid member DNS object:

```
name - Optional.   Host name in string format
ipv4addr - Optional.   IPv4 address of the member
ipv6addr - Optional.   IPv6 address of the member
extattrs     - Optional. A hash reference containing extensible attributes.
extensible_attributes - Optional. A hash reference containing extensible attributes.
```

Example

```
my $object   = $session->get(
   object    => "Infoblox::Grid::Member::DNS",
   name      => "infoblox.localdomain",
    );

# get all Grid::Member::DNS objects with a given extensible attribute
```

```
my @retrieved_objs = $session->get(
    object => "Infoblox::Grid::Member::DNS",
    extensible_attributes => { 'Site' => 'Santa Clara' });
```

Infoblox::Session->modify( )

Use this method to modify an object in the Infoblox appliance. See Infoblox::Session->modify() for parameters and return values.

If you have applied an access control list to an object, you must restart services after you update the access control list for the changes to take effect. See Infoblox::Session->restart() method.

Example

```
#Use this method to modify the enable_dns  property.
$object->enable_dns("false");
#Submit modification
my $response = $session->modify( $object );
```

METHODS

This section describes all the methods that you can use to configure and retrieve the attribute values of the DNS properties of a grid member.

anonymize_response_logging( )

Use this method to set or retrieve the flag that indicates whether the anonymization of captured DNS responses is enabled or disabled.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

Specify 'true' to enable anonymization of captured DNS responses or 'false' to disable it. The default value is 'false'.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get anonymize_response_logging value
my $anonymize_response_logging = $object->anonymize_response_logging();

#Modify anonymize_response_logging value
$object->anonymize_response_logging('true');
```

allow_gss_tsig_zone_updates( )

Use this method to enable/disable GSS-TSIG zone updates for this member. This setting overrides the grid GSS-TSIG zone update settings.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

Specify "true" to allow GSS-TSIG zone updates for this member or "false" to disallow them. The default value is "false".

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails. If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get allow_gss_tsig_zone_updates
my $allow_gss_tsig_zone_updates = $memberdns->allow_gss_tsig_zone_updates( );
#Modify allow_gss_tsig_zone_updates
$memberdns->allow_gss_tsig_zone_updates("true");
```

allow_query( )

Use this method to add or retrieve an allow_query list.

Select "allow query" to permit queries from the specified IPv4 or IPv6 addresses and networks. The list specifies IP `address(es)` and/or `network(s)` from which queries are allowed or denied. The appliance can also use Transaction Signature (TSIG) keys to authenticate the

queries. This setting overrides the grid query settings.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

Valid value is an Infoblox::Grid::NamedACL object or an array reference that contains IPv4/IPv6 addresses, networks and/or Infoblox::DNS::TSIGKey object(s). To deny queries, add "!" as a prefix to `address(es)` and network(s). Specify "any" to allow queries from any address. By default, queries are allowed from any address. Undefine the value to use the Grid::DNS level allow_query list.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails. If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get allow_query
$memberdns->allow_query();
#Modify allow_query
$memberdns->allow_query( ["any"] );

#Specifying TSIG keys
my $tsig_key1 = $session->gen_tsig_key();
my $query_key = Infoblox::DNS::TSIGKey->new(
  name => "query_key1",
  key  => $tsig_key1
);
#Modify allow_query
$memberdns->allow_query( [ "10.0.0.10" , "20.0.0.0/24"
,"2001::123a","2003::222/64","!"."30.0.0.30" , "!"."40.10.0.0/24","!"."2004::125b","!"."2006::34bc"]
);

#Modify allow_query to use Grid level settings
$memberdns->allow_query(undef);
#Use Infoblox::Grid::NamedACL
$memberdns->allow_query($nacl);
```

allow_recursive_query( )

Use this method to enable/disable responses to recursive queries. This setting overrides grid recursive query settings.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

Specify "true" to allow responses to recursive queries or "false" to reject recursive queries. The default value is "false". Undefine to use the Grid::DNS level allow_recursive_query setting. Note: You should also undefine the recursive_query_list list to use the Grid::DNS level settings.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails. If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get allow_recursive_query
my $allow_recursive_query = $memberdns->allow_recursive_query( );
#Modify allow_recursive_query
$memberdns->allow_recursive_query("true");
```

allow_transfer( )

Use this method to allow or reject zone transfers from specified IPv4 or IPv6 addresses and networks or to allow transfers from hosts authenticated by Transaction signature (TSIG) key. This setting overrides the grid zone transfer settings.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

Valid value is an Infoblox::Grid::NamedACL object or an array reference that contains IPv4/IPv6 addresses, networks and/or Infoblox::DNS::TSIGKey object(s). To reject zone transfers, add '!' as a prefix to IP addresses and networks. Specify "any" to allow any transfer, or undefine to use the Grid::DNS level allow_transfer list.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Getting allow_transfer
my $memberdns = $object->allow_transfer( );
#Modifying allow_transfer
$memberdns->allow_transfer(["any"]);

#Specifying TSIG keys
my $tsig_key1 = $session->gen_tsig_key();
my $key1 = Infoblox::DNS::TSIGKey->new(
  name => "key1",
  key  => $tsig_key1
);
my $key2 = Infoblox::DNS::TSIGKey->new(
  name => "tsig_xfer",
  key  => ":2xCOMPAT"
);
#Modifying allow_transfer
$memberdns->allow_transfer(["10.0.0.10", $key1, "20.0.0.0/24", $key2,
"2004::325f","2007::876c/64","!"."30.0.0.30", "!"."40.10.0.0/24"]);

#Use Grid::DNS level allow_transfer list
$memberdns->allow_transfer(undef);

#Use Infoblox::Grid::NamedACL
$memberdns->allow_transfer($nacl);
```

allow_update( )

Use this method to allow or reject dynamic updates from specified IPv4 or IPv6 addresses, networks or from host authenticated by TSIG key. This setting overrides grid update settings.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

Valid value is an Infoblox::Grid::NamedACL object or an array reference that contains IPv4/IPv6 addresses, networks and/or Infoblox::DNS::TSIGKey object(s). To reject updates, add '!' as a prefix to IP addresses and networks. Specify "any" to allow any update, or undefine to use the Grid::DNS level allow_update list.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Getting allow_update
$memberdns->allow_update();
#Modifying allow_update
$memberdns->allow_update(["any"]);

#Creating TSIG Key object.
my $update_key1 = $session->gen_tsig_key();
my $up_key1 = Infoblox::DNS::TSIGKey->new(
  name => "update_key",
  key  => $update_key1
  );
#Modifying allow_update
$memberdns->allow_update([ $up_key1, "10.0.0.10", "20.0.0.0/24","2008::129b","!"."30.0.0.30",
"!"."40.10.0.0/24","!"."2004::76bc/64"]);

#Use Grid::DNS level allow_update list
$memberdns->allow_update(undef);
$memberdns->update_keys(undef);
#Use Infoblox::Grid::NamedACL
$memberdns->allow_update($nacl);
```

attack_mitigation( )

Use this method to set or retrieve DNS attack mitigation settings for the member.

Setting this method to a defined value implicitly sets the override_attack_mitigation method to "true". Setting the parameter to undefined causes the appliance to use the Grid DNS default and automatically resets the override_attack_mitigation method to "false".

Note that when attack_mitigation is set to a defined value and override_attack_mitigation is set to "false", the last operation takes

precedence. Thus the sequence $object->attack_mitigation($attack_mitigation); $object->override_attack_mitigation("false"); will set override_attack_mitigation to "false", and the sequence $object->override_attack_mitigation("false"); $object->attack_mitigation($attack_mitigation); will result in override_attack_mitigation="true".

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

The valid value is an Infoblox::Grid::DNS::AttackMitigation object.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get attack_mitigation value
my $attack_mitigation = $memberdns->attack_mitigation();
#Modify attack_mitigation
$memberdns->attack_mitigation($attack_mitigation);
```

auto_blackhole( )

Use this method to set or retrieve the auto blackhole settings for the member.

Setting this method to a defined value implicitly sets the override_auto_blackhole method to "true". Setting the parameter to undefined causes the appliance to use the Grid DNS default and automatically resets the override_auto_blackhole method to "false".

Note that when auto_blackhole is set to a defined value and override_auto_blackhole is set to "false", the last operation takes precedence. Thus the sequence $object->auto_blackhole($auto_blackhole); $object->override_auto_blackhole("false"); will set override_auto_blackhole to "false", and the sequence $object->override_auto_blackhole("false"); $object->auto_blackhole($auto_blackhole); will result in override_auto_blackhole="true".

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

The valid value is an Infoblox::Grid::DNS::AutoBlackHole object.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get auto_blackhole value
my $auto_blackhole = $memberdns->auto_blackhole();
#Modify auto_blackhole
$memberdns->auto_blackhole($auto_blackhole);
```

auto_create_a_and_ptr_for_lan2( )

Use this method to enable/disable the auto-generation of A and PTR records for the LAN2 IP address, if DNS service is enabled on LAN2.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

Specify "true" to enable the auto-generation of A and PTR records for the LAN2 IP address or "false" to disable it. Default is "true".

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Getting auto_create_a_and_ptr_for_lan2
my $auto_create_a_and_ptr_for_lan2 = $memberdns->auto_create_a_and_ptr_for_lan2( );
#Modifying auto_create_a_and_ptr_for_lan2
$memberdns->auto_create_a_and_ptr_for_lan2("true");
```

auto_create_a_and_ipv6ptr_for_lan2( )

Use this method to enable/disable the auto-generation of AAAA and IPv6 PTR records for the LAN2 IP address, if DNS service is enabled on LAN2.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

Specify "true" to enable the auto-generation of AAAA and IPv6 PTR records for the LAN2 IP address or "false" to disable it. Default is "true".

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Getting auto_create_a_and_ipv6ptr_for_lan2
my $auto_create_a_and_ipv6ptr_for_lan2 = $memberdns->auto_create_a_and_ipv6ptr_for_lan2( );
#Modifying auto_create_a_and_ipv6ptr_for_lan2
$memberdns->auto_create_a_and_ipv6ptr_for_lan2("true");
```

auto_sort_views( )

Use this method to enable/disable a grid member to automatically sort DNS views. The order of the DNS views determines the order in which the appliance checks the match lists.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

Specify "true" to enable the automatic sorting of DNS views or "false" to disable it. Default is "false"

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Getting auto_sort_views
$memberdns->auto_sort_views( );
#Modifying auto_sort_views
$memberdns->auto_sort_views("true");
```

blackhole_list( )

Use this method to set or retrieve the list of IPv4 or IPv6 addresses and networks from which DNS queries are blocked. This setting overrides the grid blackhole_list.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

A valid value is an Infoblox::Grid::NamedACL object or an array reference that contains IPv4/IPv6 addresses and networks. To allow queries, add '!' as the prefix to IP addresses and networks. Specify "any" to block queries or undefine the usage of Grid::DNS level blackhole_list. Note: You should also undefine the enable_blackhole list to use Grid::DNS level settings.

Returns

When you specify a parameter and the modification succeeds, the method returns "true". When the operation fails, the method returns "false".

When you do not specify a parameter, the method returns the attribute value.

Example

```
    #Getting blackhole_list
    my $blackhole_list = $memberdns->blackhole_list( );
    #Modifying blackhole_list
    $memberdns->blackhole_list(["any"]);
    #Modifying blackhole_list
    $memberdns->blackhole_list(["10.0.0.10",
"20.0.0.0/24","2010::256/64","!"."2011::23cb/64","!"."30.0.0.30", "!"."40.10.0.0/24"]);
    #Use Grid::DNS level blackhole_list
    $memberdns->blackhole_list(undef);
    $memberdns->enable_blackhole(undef);
```

```
#Use Infoblox::Grid::NamedACL
$memberdns->blackhole_list($nacl);
```

blacklist_action( )

Use this method to specify the action to perform when a domain name matches the pattern defined in a rule that is specified by the blacklist_ruleset method.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

Valid values are "REDIRECT" or "REFUSE". The default value is "REFUSE".

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

Examples

```
#Get blacklist_action
my $blacklist_action = $memberdns->blacklist_action();
#Modify blacklist_action
$memberdns->blacklist_action("REDIRECT");
```

blacklist_log_query( )

Use this method to set or retrieve the flag that indicates whether blacklist redirection queries are logged.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

Specify "true" to enable logging, or "false" to disable it. The default value is "false".

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get blacklist_log_query
my $blacklist_log_query = $memberdns->nxdomain_log_query( );
#Modify blacklist_log_query
$memberdns->blacklist_log_query("true");
```

blacklist_redirect_addresses( )

Use this method to specify the IP addresses the appliance includes in the response it sends in place of a blacklisted IP address.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

The valid value is an array of IPv4 addresses. The default value is undefined.

Setting this parameter to ["10.0.0.10", "10.0.0.20"] or [], implicitly sets the override_blacklist_redirect attribute to "true". Setting this parameter to undefined causes the appliance to use the grid-level blacklist redirection settings and automatically sets the override_blacklist_redirect attribute to "false" if "enable_blacklist_redirect" is also undefined. But if "enable_blacklist_redirect" contains a value, then the override_blacklist_redirect attribute is set to "true".

Note that when blacklist_redirect_addresses contains a valid array reference and override_blacklist_redirect is set to "false", the last operation takes precedence. Thus the sequence $memberdns->blacklist_redirect_addresses(["10.0.0.10", "10.0.0.20"]); $memberdns->override_blacklist_redirect("false"); sets override_blacklist_redirect to "false", and the sequence $memberdns->override_blacklist_redirect("false"); $memberdns->blacklist_redirect_addresses(["10.0.0.10", "10.0.0.20"]); sets override_blacklist_redirect to "true".

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get blacklist_redirect_addresses
```

```
my $blacklist_redirect_addresses = $memberdns->blacklist_redirect_addresses( );
#Modify blacklist_redirect_addresses
$memberdns->blacklist_redirect_addresses(["10.0.0.10", "10.0.0.20"]);
```

blacklist_redirect_ttl( )

Use this method to set or retrieve the TTL value of the synthetic DNS responses that result from blacklist redirection.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

A 32-bit integer, from 0 to 4294967295, that represents the TTL in seconds. The default value is 60.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get blacklist_redirect_ttl
my $blacklist_redirect_ttl = $memberdns->blacklist_redirect_ttl( );
#Modify blacklist_redirect_ttl
$memberdns->blacklist_redirect_ttl(60);
```

blacklist_rulesets( )

Use this method to specify or retrieve Infoblox::DNS::Ruleset object names assigned at the grid level for blacklist redirection.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

The valid value is an array reference that contains the names of the Infoblox::DNS::Ruleset objects assigned to the grid. The names must be in string format.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get blacklist_rulesets
my $blacklist_rulesets = $memberdns->blacklist_rulesets();
#Modify blacklist_rulesets
$memberdns->blacklist_rulesets(["ruleset1", "ruleset2"]);
```

bind_check_names_policy( )

Use this method to set or retrieve the BIND check names policy, which indicates the action the appliance takes when it encounters host names that do not comply with the Strict Hostname Checking policy. This method applies only if the host name restriction policy is set to "Strict Hostname Checking".

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

The allowed values of the parameter are "fail", "warn" or undefined. This indicates the desired BIND behavior in the case of illegal hostnames. The default value is undefined, which means the appliance ignores illegal hostnames.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get bind_check_names_policy value
my $names_policy = $memberdns->bind_check_names_policy();
#Modify bind_check_names_policy
$memberdns->bind_check_names_policy("warn");
```

bind_hostname_directive( )

Use this method to set or retrieve the value of BIND 'hostname' directive.

Setting this method to a defined value implicitly sets the override_bind_hostname_directive method to "true". Setting the parameter to undefined causes the appliance to use the Grid DNS default and automatically resets the override_bind_hostname_directive method to "false".

Note that when bind_hostname_directive is set to a defined value and override_bind_hostname_directive is set to "false", the last operation takes precedence. Thus the sequence $object->bind_hostname_directive("HOSTNAME"); $object->override_bind_hostname_directive("false"); will set override_bind_hostname_directive to "false", and the sequence $object->override_bind_hostname_directive("false"); $object->bind_hostname_directive("true"); will result in override_bind_hostname_directive="true".

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

The valid values are "NONE", "HOSTNAME" and "USER_DEFINED". The default value is "NONE".

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get bind_hostname_directive
my $bind_hostname_directive = $memberdns->bind_hostname_directive();
#Modify bind_hostname_directive
$memberdns->bind_hostname_directive("HOSTNAME");
```

bind_hostname_directive_fqdn( )

Use this method to set or retrieve the value of BIND user-defined hostname for 'hostname' directive.

Note that to enable user-defined 'hostname' the bind_hostname_directive should be set to "USER_DEFINED".

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

The valid value is Fully-Qualified Domain Name (FQDN) in a string format. The default value is undefined.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get bind_hostname_directive_fqdn
my $bind_hostname_directive_fqdn = $memberdns->bind_hostname_directive_fqdn();
#Modify bind_hostname_directive_fqdn
$memberdns->bind_hostname_directive_fqdn("foo-bar.com");
```

capture_dns_queries_on_all_domains( )

Use this method to set or retrieve the flag that indicates whether the capturing of DNS queries for all domains is enabled or disabled.

Setting this method to a defined value implicitly sets the override_capture_dns_queries_on_all_domains method to "true". Setting the parameter to undefined causes the appliance to use the Grid DNS default and automatically resets the override_capture_dns_queries_on_all_domains method to "false".

Note that when capture_dns_queries_on_all_domains is set to a defined value and override_capture_dns_queries_on_all_domains is set to "false", the last operation takes precedence. Thus the sequence $object->capture_dns_queries_on_all_domains("true"); $object->override_capture_dns_queries_on_all_domains("false"); will set override_capture_dns_queries_on_all_domains to "false", and the sequence $object->override_capture_dns_queries_on_all_domains("false"); $object->capture_dns_queries_on_all_domains("true"); will result in override_capture_dns_queries_on_all_domains="true".

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

Specify 'true' to enable capturing of DNS queries for all domains or 'false' to disable it. The default value is 'false'.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get capture_dns_queries_on_all_domains value
my $capture_dns_queries_on_all_domains = $object->capture_dns_queries_on_all_domains();

#Modify capture_dns_queries_on_all_domains value
$object->capture_dns_queries_on_all_domains('true');
```

copy_xfer_to_notify( )

Use this method to enable/disable copying of the allowed IPs from zone transfer list into also-notify statement in named.conf.

Parameter

Set the parameter to "true" to enable copying of the allowed IPs from zone transfer list into also-notify statement in named.conf. Set the parameter to "false" to disable it.

The default value of the parameter is "false".

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
# Get attribute value
my $value = $memberdns->copy_xfer_to_notify();
# Modify attribute value
$memberdns->copy_xfer_to_notify('true');
```

custom_root_name_servers( )

Use this method to set or retrieve the list of customized root nameserver(s). You can either select and use Internet root name servers or specify custom root name servers by providing a host name and IP address to which the Infoblox appliance can send queries. Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

Valid value is an array reference that contains Infoblox::DNS::RootNameServer object(s).

Returns

If you specify a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get the root nameserver list
my $rootNS_list = $memberdns->custom_root_name_servers();
#Modify the root nameserver list
#Creating RootNameServer objects.
my $rootns1 = Infoblox::DNS::RootNameServer->new(
    ipv4addr  => "4.4.4.4",
    host_name => "rns1.test.com",
    );

my $rootns2 = Infoblox::DNS::RootNameServer->new(
    ipv4addr  => "5.5.5.5",
    host_name => "rns2.test.com",
    );

$memberdns->custom_root_name_servers([$rootns1, $rootns2]);
```

disable_edns( )

Use this method to disable or enable EDNS0 support for queries that require recursive resolution on Grid members.

Setting this method to a defined value implicitly sets the override_disable_edns method to "true". Setting the parameter to undefined causes the appliance to use the grid DNS default and automatically resets the override_disable_edns method to "false".

Note that when disable_edns is set to a defined value and override_disable_edns is set to "false", the last operation takes precedence.

Thus the sequence $object->disable_edns("true"); $object->override_disable_edns("false"); will set override_disable_edns to "false", and the sequence $object->override_disable_edns("false"); $object->disable_edns("true"); will result in override_disable_edns="true".

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

Specify "true" to disable EDNS0 or "false" to enable it. The default value is "false".

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get disable_edns flag
my $disable_edns = $memberdns->disable_edns();
#Modify disable_edns flag
$memberdns->disable_edns("true");
```

dns_cache_acceleration_ttl( )

Use this method to set or retrieve the minimum TTL value, in seconds, that a DNS record must have in order for it to be cached by the DNS Cache Acceleration service.

Setting this method to a defined value implicitly sets the override_dns_cache_acceleration_ttl method to "true". Setting the parameter to undefined causes the appliance to use the grid DNS default and automatically resets the override_dns_cache_acceleration_ttl method to "false".

Note that when dns_cache_acceleration_ttl is set to a defined value and override_dns_cache_acceleration_ttl is set to "false", the last operation takes precedence. Thus the sequence $object->dns_cache_acceleration_ttl(60); $object->override_dns_cache_acceleration_ttl("false"); will set override_dns_cache_acceleration_ttl to "false", and the sequence $object->override_dns_cache_acceleration_ttl("false"); $object->dns_cache_acceleration_ttl(30); will result in override_dns_cache_acceleration_ttl="true".

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

An integer from 1 to 65000 that represents the TTL in seconds. The default value is 1.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get dns_cache_acceleration_ttl
my $dns_cache_acceleration_ttl = $member_dns->dns_cache_acceleration_ttl( );
#Modify dns_cache_acceleration_ttl
$member_dns->dns_cache_acceleration_ttl(60);
```

dns_cache_acceleration_status( )

Use this method to retrieve the DNS cache acceleration status.

Parameter

None.

Returns

The method returns the attribute value, which is one of the following: "Failed", "Inactive", "Offline", "Unknown", "Warning", or "Working".

Example

```
#Get dns_cache_acceleration_status
my $dns_cache_acceleration_status = $member_dns->dns_cache_acceleration_status( );
```

dns_health_check_anycast_control( )

Use this method to set or retrieve the flag that indicates whether the anycast failure (BFD session down) is enabled on Member failure or not.

Setting this parameter to defined value, implicitly sets the override_dns_health_check attribute to "true". Setting this parameter to undefined causes the appliance to use the Grid-level DNS health check settings and automatically sets the override_dns_health_check attribute to "false" if following fields are undefined:

```
dns_health_check_domain_list
dns_health_check_recursion_flag
dns_health_check_interval
dns_health_check_retries
dns_health_check_timeout
enable_dns_health_check
```

But if any of the fields above contains a value, then the override_dns_health_check attribute is set to "true".

Note that when dns_health_check_anycast_control contains a value and override_dns_health_check is set to "false", the last operation takes precedence. Thus the sequence $zone->dns_health_check_anycast_control("true"); $zone->override_dns_health_check("false"); sets override_dns_health_check to "false", and the sequence $zone->override_dns_health_check("false"); $zone->dns_health_check_anycast_control("true"); sets override_dns_health_check to "true".

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

Specify 'true' to enable anycast failure on Member failure, and 'false' to disable it.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get dns_health_check_anycast_control value
my $value = $object->dns_health_check_anycast_control();

#Modify dns_health_check_anycast_control value
$object->dns_health_check_anycast_control('true');
```

dns_health_check_domain_list( )

Use this method to set or retrieve the list of domain names for the DNS health check.

Setting this parameter to defined value, implicitly sets the override_dns_health_check attribute to "true". Setting this parameter to undefined causes the appliance to use the Grid-level DNS health check settings and automatically sets the override_dns_health_check attribute to "false" if following fields are undefined:

```
dns_health_check_recursion_flag
dns_health_check_anycast_control
dns_health_check_interval
dns_health_check_retries
dns_health_check_timeout
enable_dns_health_check
```

But if any of the fields above contains a value, then the override_dns_health_check attribute is set to "true".

Note that when dns_health_check_anycast_control contains a value and override_dns_health_check is set to "false", the last operation takes precedence. Thus the sequence $zone->dns_health_check_domain_list(['domain1.com', 'domain2.com']); $zone->override_dns_health_check("false"); sets override_dns_health_check to "false", and the sequence $zone->override_dns_health_check("false"); $zone->dns_health_check_domain_list(['domain1.com', 'domain2.com']); sets override_dns_health_check to "true".

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

The valid value is an array reference that contains desired domain names in string format.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get dns_health_check_domain_list value
my $value = $object->dns_health_check_domain_list();

#Modify dns_health_check_domain_list value
$object->dns_health_check_domain_list(['foo.com', 'bar.net']);
```

dns_health_check_interval( )

Use this method to set or retrieve the time interval (in seconds) for DNS health check.

Setting this parameter to defined value, implicitly sets the override_dns_health_check attribute to "true". Setting this parameter to undefined causes the appliance to use the Grid-level DNS health check settings and automatically sets the override_dns_health_check attribute to "false" if following fields are undefined:

```
dns_health_check_domain_list
dns_health_check_recursion_flag
dns_health_check_anycast_control
dns_health_check_retries
dns_health_check_timeout
enable_dns_health_check
```

But if any of the fields above contains a value, then the override_dns_health_check attribute is set to "true".

Note that when dns_health_check_interval contains a value and override_dns_health_check is set to "false", the last operation takes precedence. Thus the sequence $zone->dns_health_check_interval("500"); $zone->override_dns_health_check("false"); sets override_dns_health_check to "false", and the sequence $zone->override_dns_health_check("false"); $zone->dns_health_check_interval("500"); sets override_dns_health_check to "true".

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

The valid value is an unsigned integer between 10 and 21600 that represents number of seconds between two health checks.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get dns_health_check_interval value
my $value = $object->dns_health_check_interval();

#Modify dns_health_check_interval value
$object->dns_health_check_interval(2000);
```

dns_health_check_recursion_flag( )

Use this method to set or retrieve the flag that indicates whether the recursive DNS health check is enabled or not.

Setting this parameter to defined value, implicitly sets the override_dns_health_check attribute to "true". Setting this parameter to undefined causes the appliance to use the Grid-level DNS health check settings and automatically sets the override_dns_health_check attribute to "false" if following fields are undefined:

```
dns_health_check_domain_list
dns_health_check_anycast_control
dns_health_check_interval
dns_health_check_retries
dns_health_check_timeout
enable_dns_health_check
```

But if any of the fields above contains a value, then the override_dns_health_check attribute is set to "true".

Note that when dns_health_check_recursion_flag contains a value and override_dns_health_check is set to "false", the last operation takes precedence. Thus the sequence $zone->dns_health_check_recursion_flag("true"); $zone->override_dns_health_check("false"); sets override_dns_health_check to "false", and the sequence $zone->override_dns_health_check("false"); $zone->dns_health_check_recursion_flag("true"); sets override_dns_health_check to "true".

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

Specify 'true' to enable recursive DNS health check queries, and 'false' to disable it.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get dns_health_check_recursion_flag value
my $value = $object->dns_health_check_recursion_flag();
```

```
#Modify dns_health_check_recursion_flag value
$object->dns_health_check_recursion_flag('true');
```

dns_health_check_retries( )

Use this method to set or retrieve the number of DNS health check retries.

Setting this parameter to defined value, implicitly sets the override_dns_health_check attribute to "true". Setting this parameter to undefined causes the appliance to use the Grid-level DNS health check settings and automatically sets the override_dns_health_check attribute to "false" if following fields are undefined:

```
dns_health_check_domain_list
dns_health_check_recursion_flag
dns_health_check_anycast_control
dns_health_check_interval
dns_health_check_timeout
enable_dns_health_check
```

But if any of the fields above contains a value, then the override_dns_health_check attribute is set to "true".

Note that when dns_health_check_retries contains a value and override_dns_health_check is set to "false", the last operation takes precedence. Thus the sequence $zone->dns_health_check_retries("4"); $zone->override_dns_health_check("false"); sets override_dns_health_check to "false", and the sequence $zone->override_dns_health_check("false"); $zone->dns_health_check_retries("4"); sets override_dns_health_check to "true".

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

The valid value is an unsigned integer between 1 and 10.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get dns_health_check_retries value
my $value = $object->dns_health_check_retries();

#Modify dns_health_check_retries value
$object->dns_health_check_retries(2);
```

dns_health_check_timeout( )

Use this method to set or retrieve the DNS health check timeout interval (in seconds).

Setting this parameter to defined value, implicitly sets the override_dns_health_check attribute to "true". Setting this parameter to undefined causes the appliance to use the Grid-level DNS health check settings and automatically sets the override_dns_health_check attribute to "false" if following fields are undefined:

```
dns_health_check_domain_list
dns_health_check_recursion_flag
dns_health_check_anycast_control
dns_health_check_interval
dns_health_check_retries
enable_dns_health_check
```

But if any of the fields above contains a value, then the override_dns_health_check attribute is set to "true".

Note that when dns_health_check_timeout contains a value and override_dns_health_check is set to "false", the last operation takes precedence. Thus the sequence $zone->dns_health_check_timeout("4"); $zone->override_dns_health_check("false"); sets override_dns_health_check to "false", and the sequence $zone->override_dns_health_check("false"); $zone->dns_health_check_timeout("4"); sets override_dns_health_check to "true".

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

The valid value is an unsigned integer between 1 and 10.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get dns_health_check_timeout value
my $value = $object->dns_health_check_timeout();

#Modify dns_health_check_timeout value
$object->dns_health_check_timeout(10);
```

dns_ip_list( )

Use this method to add or retrieve a list of additional IP addresses on which DNS is enabled for a grid member.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

Valid value is an Infoblox::Grid::Member::DNS::IP object.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get dns_ip_list value
my $value = $object->dns_ip_list();

#Modify dns_ip_list

#Construct Infoblox::Grid::Member::DNS::IP object

my $ip = Infoblox::Grid::Member::DNS::IP->new(
    member_virtual_ip => '10.0.0.10',
);

$object->dns_ip_list([$ip]);
```

dns64_groups( )

Use this method to set or retrieve the list of DNS64 synthesis groups associated with this member.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Setting this method to a defined value implicitly sets the override_dns64 method to "true". Setting the parameter to undefined causes the appliance to use the grid DNS default and automatically resets the override_dns64 method to "false".

Note that when dns64_groups is set to a defined value and override_dns64_groups is set to "false", the last operation takes precedence. Thus the sequence $object->dns64_groups([$group1]); $object->override_dns64("false"); will set override_dns64 to "false", and the sequence $object->override_dns64("false"); $object->dns64_groups([$group1]); will result in override_dns64="true".

Parameter

Valid value is an array reference that contains Infoblox::Grid::DNS::DNS64Group objects.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get the list of DNS64 synthesis groups
my $dns64_groups_list = $memberdns->dns64_groups();

#Modify it
my $group = $session->get(
                          object => "Infoblox::Grid::DNS::DNS64Group"),
                          name => "groupname",
                          prefix => "groupprefix",
                        );
$memberdns->dns64_groups([$group]);
```

dnssec_enabled( )

Use this method to enable/disable the DNSSEC feature.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

Set the parameter to "true" to enable the DNSSEC feature. Set the parameter to "false" to disable the feature.

Setting this parameter to "true" or "false" implicitly sets the override_dnssec attribute to "true". Setting the parameter to undefined causes the appliance to use the grid-level DNSSEC settings and automatically sets the override_dnssec attribute to "false".

Note that in the case when dnssec_enabled is set to "true"/"false" and override_dnssec is set to "false" simultaneously, the last operation takes precedence. Thus the sequence $grid_member->dnssec_enabled("false"); $grid_member->override_dnssec("false"); will set override_dnssec to "false", and the sequence $grid_member->override_dnssec("false"); $grid_member->dnssec_enabled("false"); will result in override_dnssec="true".

The default value of the parameter is "false".

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get dnssec_enabled
my $dnssec_enabled = $memberdns->dnssec_enabled();
#Modify dnssec_enabled
$memberdns->dnssec_enabled("true");
#Set the value to undefined to inherit grid level settings.
$memberdns->dnssec_enabled(undef);
```

dnssec_expired_signatures_enabled( )

Use this method to control whether the DNS member accepts expired signatures.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

Set the parameter to "true" to accept expired signatures. Set it to "false" to reject expired signatures.

The default value of the parameter is "false".

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get dnssec_expired_signatures_enabled
my $dnssec_expired_signatures_enabled = $memberdns->dnssec_expired_signatures_enabled();
#Modify dnssec_expired_signatures_enabled
$memberdns->dnssec_expired_signatures_enabled("true");
```

dnssec_validation_enabled( )

Use this method to enable or disable DNS security validation.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

Set the parameter to "true" to enable the validation. Set it to "false" to disable the validation.

The default value of the parameter is "true".

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get dnssec_validation_enabled
my $dnssec_validation_enabled = $memberdns->dnssec_validation_enabled();
#Modify dnssec_validation_enabled
$memberdns->dnssec_validation_enabled("false");
```

dnssec_blacklist_enabled()

Use this method to enable and disable blacklist rules for DNSSEC-enabled clients.

Parameter

Set the parameter to "true" to apply blacklist rules for DNSSEC-enabled clients. Set to "false" to disable them.

The default value of the parameter is "false".

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get dnssec_blacklist_enabled
my $dnssec_blacklist_enabled = $memberdns->dnssec_blacklist_enabled();
#Modify dnssec_blacklist_enabled
$memberdns->dnssec_blacklist_enabled("true");
```

dnssec_negative_trust_anchors( )

Use this method to set or retrieve the list of zones for which the appliance does not perform DNSSEC validation.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

The valid value is an array of strings that contains the lsit of zones for which the appliance does not perform DNSSEC validation.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get dnssec_negative_trust_anchors value
my $value = $member_dns->dnssec_negative_trust_anchors();
#Modify dnssec_negative_trust_anchors value
$member_dns->dnssec_negative_trust_anchors($zones);
```

dnssec_dns64_enabled()

Use this method to enable and disable DNS64 groups for DNSSEC-enabled clients.

Parameter

Set the parameter to "true" to apply DNS64 groups for DNSSEC-enabled clients. Set to "false" to disable them.

The default value of the parameter is "false".

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get dnssec_dns64_enabled
my $dnssec_dns64_enabled = $member_dns->dnssec_dns64_enabled();
#Modify dnssec_dns64_enabled
$member_dns->dnssec_dns64_enabled("true");
```

dnssec_nxdomain_enabled()

Use this method to enable and disable NXDOMAIN rules for DNSSEC-enabled clients.

Parameter

Set the parameter to "true" to apply NXDOMAIN rules for DNSSEC-enabled clients. Set to "false" to disable them.

The default value of the parameter is "false".

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get dnssec_nxdomain_enabled
my $dnssec_nxdomain_enabled = $memberdns->dnssec_nxdomain_enabled();
#Modify dnssec_nxdomain_enabled
$memberdns->dnssec_nxdomain_enabled("true");
```

dnssec_rpz_enabled()

Use this method to enable and disable RPZ policies for DNSSEC-enabled clients.

Parameter

Set the parameter to "true" to apply RPZ policies for DNSSEC-enabled clients. Set to "false" to disable them.

The default value of the parameter is "false".

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get dnssec_rpz_enabled
my $dnssec_rpz_enabled = $memberdns->dnssec_rpz_enabled();
#Modify dnssec_rpz_enabled
$memberdns->dnssec_rpz_enabled("true");
```

dnssec_trusted_keys( )

Use this method to specify the list of trusted keys for the DNSSEC feature.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

Array reference that contains a list of Infoblox::DNS::DnssecTrustedKey objects.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns a reference to a list of the trusted keys.

Example

```
#Get dnssec_trusted_keys
my $dnssec_trusted_keys = $memberdns->dnssec_trusted_keys();
#Modify dnssec_trusted_keys
my $key1=Infoblox::DNS::DnssecTrustedKey->new(
   fqdn => "myzone.com",
   algorithm => "RSAMD5",
   key       => "c2ltcGxlIGtleQo="
   );
$memberdns->dnssec_trusted_keys([$key1]);
```

dns_notify_transfer_source_interface( )

Use this method to specify which IP address is used as the source for DDNS notify and transfer operations.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

Valid values are "VIP", "MGMT", "LAN2", "Any" or an IPV4 address. Default is "VIP".

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Getting dns_source_interface
my $dns_source_interface = $memberdns->dns_source_interface( );
#Modifying dns_source_interface
$memberdns->dns_source_interface("VIP");
```

dns_query_capture_file_time_limit( )

Use this method to set or retrieve the DNS query capture file time (in minutes) limit.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

The valid value is unsigned integer between 1 and 10. The default value is 10.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get dns_query_capture_file_time_limit value
my $dns_query_capture_file_time_limit = $object->dns_query_capture_file_time_limit();

#Modify dns_query_capture_file_time_limit value
$object->dns_query_capture_file_time_limit('5');
```

dns_query_source_interface( )

Use this method to specify which IP address is used as the source for DDNS query operations.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

Valid values are "VIP", "MGMT", "LAN2", "Any" or an IPV4 address. Default is "VIP".

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Getting dns_source_interface
my $dns_source_interface = $memberdns->dns_source_interface( );
#Modifying dns_source_interface
$memberdns->dns_source_interface("VIP");
```

dns_view_address_settings( )

Use this method to set or retrieve the list of address settings for Grid member views.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

The valid value is an array of Infoblox::Grid::Member::DNS::ViewAddressSetting objects.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get dns_view_address_settings value
my $value = $object->dns_view_address_settings();

#Construct view address setting
my $view_address_setting = Infoblox::Grid::Member::DNS::ViewAddressSetting->new('view_name' =>
'view1');
```

```
#Modify dns_view_address_settings value
$object->dns_view_address_settings([$view_address_setting]);
```

domains_to_capture_dns_queries( )

Use this method to set or retrieve the list of domains to capture DNS queries.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

The valid value is an array reference that contains desired domain names in a string format.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get domains_to_capture_dns_queries value
my $domains_to_capture_dns_queries = $object->domains_to_capture_dns_queries();

#Modify domains_to_capture_dns_queries value
$object->domains_to_capture_dns_queries(['domain1.com', 'domain2.com']);
```

dtc_edns_prefer_client_subnet( )

Use this method to set or retrieve the flag that indicates whether the use of the edns-client-subnet option for DTC is enabled or disabled.

Setting this method to a defined value implicitly sets the override_dtc_edns_prefer_client_subnet method to "true". Setting the parameter to undefined causes the appliance to use the Grid DNS default and automatically resets the override_dtc_edns_prefer_client_subnet method to "false".

Note that when dtc_edns_prefer_client_subnet is set to a defined value and override_dtc_edns_prefer_client_subnet is set to "false", the last operation takes precedence. Thus the sequence $object->dtc_edns_prefer_client_subnet("true"); $object->override_dtc_edns_prefer_client_subnet("false"); will set override_dtc_edns_prefer_client_subnet to "false", and the sequence $object->override_dtc_edns_prefer_client_subnet("false"); $object->dtc_edns_prefer_client_subnet("true"); will result in override_dtc_edns_prefer_client_subnet="true".

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

Specify 'true' to enable use of the edns-client-subnet option for DTC or 'false' to disable it.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get dtc_edns_prefer_client_subnet value
my $dtc_edns_prefer_client_subnet = $object->dtc_edns_prefer_client_subnet();

#Modify dtc_edns_prefer_client_subnet value
$object->dtc_edns_prefer_client_subnet('true');
```

dtc_health_source_interface( )

Use this method to set or retrieve the source address for the health checks.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

The valid value is one of the following: VIP, MGMT, LAN2, ANY or IPv4 / IPv6 address.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Getting dtc_health_source_interface
my $dtc_source_interface = $memberdns->dtc_health_source_interface();
#Modifying dtc_health_source_interface
$memberdns->dtc_health_source_interface('VIP');
```

enable_blackhole( )

Use this method to enable or disable the blocking of DNS queries. This setting overrides the grid enable_blackhole settings.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

Specify "true" to block DNS queries or "false" to disable blocking DNS queries. The default value is "false".

Returns

When you specify a parameter and the modification succeeds, the method returns "true". When the operation fails, the method returns "false".

When you do not specify a parameter, the method returns the attribute value.

Example

```
#Get enable_blackhole
my $enable_blackhole = $object->enable_blackhole( );
#Modify enable_blackhole
$memberdns->enable_blackhole("true");
```

enable_blacklist( )

Use this method to enable or disable a blacklist on the grid member.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

Specify "true" to set the enable_blacklist flag or "false" to deactivate/unset it. The default is "false".

Setting this parameter to "true" or "false" implicitly sets the override_blacklist attribute to "true". Setting the parameter to undefined causes the appliance to use the grid-level blacklist settings and automatically sets the override_blacklist attribute to "false" if "blacklist_redirect_addresses" is also undefined.

Note that when enable_blacklist is set to "true"/"false" and override_blacklist is set to "false", the last operation takes precedence. Thus the sequence $memberdns->enable_blacklist("false"); $memberdns->override_blacklist("false"); sets override_blacklist to "false", and the sequence $memberdns->override_blacklist("false"); $memberdns->enable_blacklist("false"); sets override_blacklist to "true".

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Getting enable_blacklist
my $enable_blacklist = $memberdns->enable_blacklist( );
#Modifying enable_blacklist
$memberdns->enable_blacklist("true");
```

enable_capture_dns_queries( )

Use this method to set or retrieve the flag that indicates whether the capture of DNS queries is enabled or disabled.

Setting this method to a defined value implicitly sets the override_enable_capture_dns method to "true". Setting the parameter to undefined unless enable_capture_dns_responses is defined causes the appliance to use the Grid DNS default and automatically resets the override_enable_capture_dns method to "false".

Note that when enable_capture_dns_queries is set to a defined value and override_enable_capture_dns is set to "false", the last operation takes precedence. Thus the sequence $object->enable_capture_dns_queries("true"); $object->override_enable_capture_dns("false"); will set override_enable_capture_dns to "false", and the sequence $object->override_enable_capture_dns("false"); $object->enable_capture_dns_queries("true"); will result in override_enable_capture_dns="true".

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

Specify 'true' to enable the capture of DNS queries or 'false' to disable it. The default value is 'false'.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get enable_capture_dns_queries value
my $enable_capture_dns_queries = $object->enable_capture_dns_queries();

#Modify enable_capture_dns_queries value
$object->enable_capture_dns_queries('true');
```

enable_capture_dns_responses( )

Use this method to set or retrieve the flag that indicates whether the capture of DNS responses is enabled or disabled.

Setting this method to a defined value implicitly sets the override_enable_capture_dns method to "true". Setting the parameter to undefined unless enable_capture_dns_queries is defined causes the appliance to use the Grid DNS default and automatically resets the override_enable_capture_dns method to "false".

Note that when enable_capture_dns_responses is set to a defined value and override_enable_capture_dns is set to "false", the last operation takes precedence. Thus the sequence $object->enable_capture_dns_responses("true"); $object->override_enable_capture_dns("false"); will set override_enable_capture_dns to "false", and the sequence $object->override_enable_capture_dns("false"); $object->enable_capture_dns_responses("true"); will result in override_enable_capture_dns="true".

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

Specify 'true' to enable the capture of DNS responses or 'false' to disable it. The default value is 'false'.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get enable_capture_dns_responses value
my $enable_capture_dns_responses = $object->enable_capture_dns_responses();

#Modify enable_capture_dns_responses value
$object->enable_capture_dns_responses('true');
```

enable_dns( )

Use this method to enable/disable the DNS service of a member.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

Specify "true" to enable DNS service or "false" to disable it. The default value is "true".

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Getting enable_dns
$memberdns->enable_dns( );
#Modifying enable_dns
$memberdns->enable_dns("true");
```

enable_dns_cache_acceleration( )

Use this method to enable/disable the DNS Cache Acceleration service of a member.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

Specify "true" to enable the DNS Cache Acceleration service or "false" to disable it. The default value is "false".

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Getting enable_dns_cache_acceleration
$memberdns->enable_dns_cache_acceleration( );
#Modifying enable_dns_cache_acceleration
$memberdns->enable_dns_cache_acceleration("true");
```

enable_dns_health_check( )

Use this method to set or retrieve the flag that indicates whether the DNS health check is enabled or not.

Setting this parameter to defined value, implicitly sets the override_dns_health_check attribute to "true". Setting this parameter to undefined causes the appliance to use the Grid-level DNS health check settings and automatically sets the override_dns_health_check attribute to "false" if following fields are undefined:

```
dns_health_check_domain_list
dns_health_check_recursion_flag
dns_health_check_anycast_control
dns_health_check_interval
dns_health_check_retries
dns_health_check_timeout
```

But if any of the fields above contains a value, then the override_dns_health_check attribute is set to "true".

Note that when enable_dns_health_check contains a value and override_dns_health_check is set to "false", the last operation takes precedence. Thus the sequence $zone->enable_dns_health_check("true"); $zone->override_dns_health_check("false"); sets override_dns_health_check to "false", and the sequence $zone->override_dns_health_check("false"); $zone->enable_dns_health_check("true"); sets override_dns_health_check to "true".

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

Specify 'true' to enable DNS health check, and 'false' to disable it.

Returns

When you specify a parameter and the modification succeeds, the method returns "true". When the operation fails, the method returns "false".

When you do not specify a parameter, the method returns the attribute value.

Example

```
#Get enable_dns_health_check value
my $value = $object->enable_dns_health_check();

#Modify enable_dns_health_check value
$object->enable_dns_health_check('true');
```

enable_dns64( )

Use this method to enable/disable DNS64 support for this member.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Setting this method to a defined value implicitly sets the override_dns64 method to "true". Setting the parameter to undefined causes the appliance to use the grid DNS default and automatically resets the override_dns64 method to "false".

Note that when enable_dns64 is set to a defined value and override_enable_dns64 is set to "false", the last operation takes precedence. Thus the sequence $object->enable_dns64("true"); $object->override_dns64("false"); will set override_dns64 to "false", and the sequence $object->override_dns64("true"); $object->enable_dns64("testdomain"); will result in override_dns64="true".

Parameter

Specify "true" to set the enable_dns64 flag or "false" to deactivate and unset it. The default value is "false".

Returns

When you specify a parameter and the modification succeeds, the method returns "true". When the operation fails, the method

returns "false".

When you do not specify a parameter, the method returns the attribute value.

Example

```
#Get the enable_dns64 flag
my $enable_dns64 = $memberdns->enable_dns64();
#Modify the enable_dns64 flag
$memberdns->enable_dns64("true");
```

enable_excluded_domain_names( )

Use this method to set or retrieve the flag that indicates whether the excluding domain names captured DNS query and response is enabled or disabled..

Setting this method to a defined value implicitly sets the override_enable_excluded_domain_names method to "true". Setting the parameter to undefined causes the appliance to use the Grid DNS default and automatically resets the override_enable_excluded_domain_names method to "false".

Note that when enable_excluded_domain_names is set to a defined value and override_enable_excluded_domain_names is set to "false", the last operation takes precedence. Thus the sequence $object->enable_excluded_domain_names("true"); $object->override_enable_excluded_domain_names("false"); will set override_enable_excluded_domain_names to "false", and the sequence $object->override_enable_excluded_domain_names("false"); $object->enable_excluded_domain_names("true"); will result in override_enable_excluded_domain_names="true".

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

Specify 'true' to enable the excluding domain names from captured DNS query and response or 'false' to disable it. The default value is 'false'.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get enable_excluded_domain_names value
my $enable_excluded_domain_names = $object->enable_excluded_domain_names();

#Modify enable_excluded_domain_names value
$object->enable_excluded_domain_names('true');
```

enable_fixed_rrset_order_fqdns( )

Use this method to enable or disable fixed RRset order for specified FQDNs.

Setting this method to a defined value implicitly sets the override_fixed_rrset_order_fqdns method to "true".

Note that when enable_fixed_rrset_order_fqdns is set to a defined value and override_fixed_rrset_order_fqdns is set to "false", the last operation takes precedence. Thus the sequence $object->enable_fixed_rrset_order_fqdns("true"); $object->override_fixed_rrset_order_fqdns("false"); will set override_fixed_rrset_order_fqdns to "false", and the sequence $object->override_fixed_rrset_order_fqdns("false"); $object->enable_fixed_rrset_order_fqdns("true"); will result in override_fixed_rrset_order_fqdns="true".

The enable_fixed_rrset_order_fqdns attribute can be specified explicitly. It is also set implicitly when fixed_rrset_order_fqdns is set to a defined value.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

Specify "true" to enable fixed RRset order for specified FQDNs or "false" to disable it.

Returns

When you specify a parameter and the modification succeeds, the method returns "true". When the operation fails, the method returns "false".

When you do not specify a parameter, the method returns the attribute value.

Example

```
#Get enable_fixed_rrset_order_fqdns flag
```

```
my $enable_fixed_rrset_order_fqdns = $member_dns->enable_fixed_rrset_order_fqdns();
#Modify enable_fixed_rrset_order_fqdns flag
$member_dns->enable_fixed_rrset_order_fqdns("true");
```

enable_ftc( )

Use this method to set or retrieve the flag that indicates whether the Fault Tolerant Caching (FTC) feature is enabled or not.

Setting this method to a defined value implicitly sets the override_ftc method to "true". Setting the parameter to undefined if ftc_expired_record_ttl and ftc_expired_record_timeout are also undefined causes the appliance to use the Grid DNS default and automatically resets the override_ftc method to "false".

Note that when enable_ftc is set to a defined value and override_ftc is set to "false", the last operation takes precedence. Thus the sequence $object->enable_ftc("true"); $object->override_ftc("false"); will set override_ftc to "false", and the sequence $object->override_ftc("false"); $object->enable_ftc("true"); will result in override_ftc="true".

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

Specify 'true' to enable FTC or 'false' to disable it. The default value is 'false'.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get enable_ftc
my $enable_ftc = $memberdns->enable_ftc();
#Modify enable_ftc
$memberdns->enable_ftc('true');
```

enable_gss_tsig( )

Use this method to enable/disable the appliance to receive GSS-TSIG authenticated updates from DHCP clients. You must import the keytab file before you enable this feature. See Infoblox::Session->import_data().

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

Specify "true" to enable the the appliance to receive GSS-TSIG authenticated updates or "false" to disable it. The default value is "false".

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Getting enable_gss_tsig
$my $enable_gss_tsig = $memberdns->enable_gss_tsig( );
#Modifying enable_gss_tsig
$session->import_data(
     type        => 'keytab',
     path        => '/infoblox/tests/datasets/gss-tsig-gs.keytab',
     service     => 'dns',
     virtual_ip  => '192.168.1.2');
$memberdns->enable_gss_tsig("true");
```

enable_query_rewrite( )

Use this method to enable or disable DNS query rewrite for this member.

Setting this method to a defined value implicitly sets the override_query_rewrite method to "true". Setting the parameter to undefined causes the appliance to use the grid-level enable_query_rewrite value and automatically resets the override_query_rewrite method to "false".

Note that when enable_query_rewrite is set to a defined value and override_query_rewrite is set to "false", the last operation takes precedence. Thus the sequence $object->enable_query_rewrite("true"); $object->override_query_rewrite("false"); will set override_query_rewrite to "false", and the sequence $object->override_query_rewrite("true"); $object->enable_query_rewrite("testdomain"); will result in override_query_rewrite="true".

Parameter
Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get enable_query_rewrite
$memberdns->enable_query_rewrite( );
#Modify enable_query_rewrite
$memberdns->enable_query_rewrite("true");
```

excluded_domain_names( )

Use this method to set or retrieve the list of domains that are excluded from DNS response and query capturing.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

The valid value is an array reference that contains desired domain names in a string format.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get excluded_domain_names value
my $excluded_domain_names = $object->excluded_domain_names();

#Modify excluded_domain_names value
$object->excluded_domain_names(['domain1.com', 'domain2.com']);
```

extattrs( )

Use this method to set or retrieve the extensible attributes associated with a Grid Member DNS object.

Parameter

Valid value is a hash reference containing the names of extensible attributes and their associated values (Infoblox::Grid::Extattr objects).

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get extattrs
my $ref_extattrs = $memberdns->extattrs();
#Modify extattrs
$memberdns->extattrs({ 'Site' => $extattr1, 'Administrator' => $extattr2 });
```

extensible_attributes( )

Use this method to set or retrieve the extensible attributes associated with a Grid Member DNS object.

This method is read-only. Use the Infoblox::Grid::Member object to set or modify the grid member extensible attributes.

Returns

The method returns the attribute value. Attempting to modify the extensible attributes produces an error.

Example

```
#Get extensible attributes
my $ref_extensible_attributes = $memberdns->extensible_attributes();
```

facility( )

Use this method to specify a syslog facility. This is the location on the syslog server to which you want to sort the DNS logging messages.

This setting overrides the grid logging facility settings.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

Values are: "daemon", "local0", "local1", "local2", "local3", "local4", "local5", "local6", and "local7".

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get facility
$memberdns->facility( );
#Modify facility
$memberdns->facility("local0");
```

file_transfer_setting( )

Use this method to set or retrieve the DNS capture file transfer settings.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

The valid value is an Infoblox::Grid::DNS::FileTransferSetting object.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get file_transfer_setting value
my $file_transfer_setting = $object->file_transfer_setting();

#Modify file_transfer_setting value
$object->file_transfer_setting($file_transfer);
```

filter_aaaa( )

Use this method to set or retrieve the type of AAAA filtering for this member DNS object.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Specifying values in this parameter, such as "YES", implicitly sets the override_filter_aaaa attribute to "true". Setting this parameter and the "filter_aaaa" attribute to undefined causes the appliance to use the grid-level filter_aaaa settings and automatically sets the override_filter_aaaa attribute to "false". But if the "filter_aaaa" attribute contains a value, then the override_filter_aaaa attribute is set to "true".

Note that when filter_aaaa contains a valid value and override_filter_aaaa is set to "false", the last operation takes precedence. Thus the sequence $memberdns->filter_aaaa("YES"); $memberdns->override_filter_aaaa("false"); sets override_filter_aaaa to "false", and the sequence $memberdns->override_filter_aaaa("false"); $memberdns->filter_aaaa("true"); sets override_filter_aaaa to "true".

Parameter

Valid values are "YES", "NO" and "BREAK_DNSSEC". The default value is "NO".

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get the filter aaaa type
my $filter_aaaa = $memberdns->filter_aaaa();
#Modify the filtering
$memberdns->filter_aaaa("YES");
```

filter_aaaa_list( )

Use this method to set or retrieve the list of IPv4 addresses and networks from which queries are received. AAAA filtering is applied to these addresses. This setting overrides the grid filter_aaaa_list if override_filter_aaaa is set to "true".

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Specifying values in this parameter, such as ["10.0.0.10", "10.0.0.20"] or [], implicitly sets the override_filter_aaaa attribute to "true". Setting this parameter and the "filter_aaaa" attribute to undefined causes the appliance to use the grid-level filter_aaaa settings and automatically sets the override_filter_aaaa attribute to "false". But if the "filter_aaaa" attribute contains a value, then the override_filter_aaaa attribute is set to "true".

Note that when filter_aaaa_list contains a valid array reference and override_filter_aaaa is set to "false", the last operation takes precedence. Thus the sequence $memberdns->filter_aaaa_list(["10.0.0.10", "10.0.0.20"]); $memberdns->override_filter_aaaa("false"); sets override_filter_aaaa to "false", and the sequence $memberdns->override_filter_aaaa("false"); $memberdns->filter_aaaa_list(["10.0.0.10", "10.0.0.20"]); sets override_filter_aaaa to "true".

Parameter

A valid value is an Infoblox::Grid::NamedACL object or an array reference that contains IPv4 addresses and networks. To return AAAA records for queries received from an IPv4 client, add "!" as the prefix to the IP addresses and networks. Specify "any" to allow AAAA records to be returned for any query received by an IPv4 client.

Returns

When you specify a parameter and the modification succeeds, the method returns "true". When the operation fails, the method returns "false".

When you do not specify a parameter, the method returns the attribute value.

Example

```
#Getting filter_aaaa_list
my $filter_aaaa_list = $memberdns->filter_aaaa_list( );
#Modifying filter_aaaa_list
$memberdns->filter_aaaa_list(["any"]);
#Modifying filter_aaaa_list
$memberdns->filter_aaaa_list(["10.0.0.10", "20.0.0.0/24", "!"."30.0.0.30", "!"."40.10.0.0/24"]);
#Use Grid::DNS level filter_aaaa_list
$memberdns->filter_aaaa_list(undef);
$memberdns->enable_filter_aaaa(undef);
#Use Infoblox::Grid::NamedACL
$memberdns->filter_aaaa_list($nacl);
```

fixed_rrset_order_fqdns( )

Use this method to set or retrieve configuration for fixed RRSET order for specified FQDNs.

Setting this method to a defined value implicitly sets the enable_fixed_rrset_order_fqdns and override_fixed_rrset_order_fqdns methods to "true". Setting the parameter to undefined causes the appliance to disable fixed RRset order of FQDNs and automatically resets the enable_fixed_rrset_order_fqdns method to "false".

Note that when fixed_rrset_order_fqdns is set to a defined value and enable_fixed_rrset_order_fqdns is set to "false", the last operation takes precedence. Thus the sequence $object->fixed_rrset_order_fqdns($fqdns); $object->enable_fixed_rrset_order_fqdns("false"); will set enable_fixed_rrset_order_fqdns to "false", and the sequence $object->enable_fixed_rrset_order_fqdns("false"); $object->fixed_rrset_order_fqdns($fqdns); will result in enable_fixed_rrset_order_fqdns="true".

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

The valid value is an array reference that contains Infoblox::Grid::DNS::FixedRRSetOrderFQDN objects.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get the fixed_rrset_order_fqdns list
my $fixed_rrset_order_fqdns = $member_dns->fixed_rrset_order_fqdns();
#Modify the fixed_rrset_order_fqdns list
$member_dns->fixed_rrset_order_fqdns([$fqdn1, $fqdn2]);
```

forwarders( )

Use this method to specify forwarders for the member. A forwarder is essentially a name server to which other name servers first send all

of their off-site queries. The forwarder builds up a cache of information, avoiding the need for the other name servers to send queries off-site. This setting overrides the grid-level setting.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

Valid value is an array reference that contains Infoblox::DNS::Member, Infoblox::DNS::Nameserver object(s), or IP addresses.

Specifying values in this parameter, such as ["10.0.0.10", "10.0.0.20"] or [], implicitly sets the override_forwarders attribute to "true". Setting this parameter and the "forward_only" attribute to undefined causes the appliance to use the grid-level forwarders settings and automatically sets the override_forwarders attribute to "false". But if the "forward_only" attribute contains a value, then the override_forwarders attribute is set to "true".

Note that when forwarders_redirect_addresses contains a valid array reference and override_forwarders is set to "false", the last operation takes precedence. Thus the sequence $memberdns->forwarders(["10.0.0.10", "10.0.0.20"]); $memberdns->override_forwarders("false"); sets override_forwarders to "false", and the sequence $memberdns->override_forwarders("false"); $memberdns->forwarders(["10.0.0.10", "10.0.0.20"]); sets override_forwarders to "true".

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Getting forwarders
$memberdns->forwarders( );
#Modifying forwarders
$memberdns->forwarders( [ "10.0.0.10" ] );
```

forward_only( )

Use this method to configure this member to send queries to forwarders only. When the value is "true", the member sends queries to forwarders only, and not to other internal or Internet root servers.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

Specify "true" to enable forward only or "false" to desable it. Default is "false"

Setting this parameter to "true" or "false" implicitly sets the override_forwarders attribute to "true". Setting the parameter to undefined causes the appliance to use the grid-level forwarders settings and automatically sets the override_forwarders attribute to "false", if "forwarders" is also undefined.

Note that when forward_only is set to "true"/"false" and override_forwarders is set to "false", the last operation takes precedence. Thus the sequence $memberdns->forward_only("false"); $memberdns->override_forwarders("false"); sets override_forwarders to "false", and the sequence $memberdns->override_forwarders("false"); $memberdns->forward_only("false"); sets override_forwarders to "true".

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Getting forward_only
$memberdns->forward_only( );
#Modifying forward_only
$memberdns->forward_only("true");
```

forward_updates( )

Use this method to allow/disallow secondary servers to forward updates to the DNS server. This setting overrides grid update settings.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

Specify "true" to enable forward updates or "false" desable it. Default is "false"

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Getting forward_updates
$memberdns->forward_updates( );
#Modifying forward_only
$memberdns->forward_updates("true");
```

ftc_expired_record_ttl( )

Use this method to set or retrieve the TTL value of the expired Fault Tolerant Caching (FTC) record in the DNS responses.

Setting this method to a defined value implicitly sets the override_ftc method to "true". Setting the parameter to undefined if enable_ftc and ftc_expired_record_timeout are also undefined causes the appliance to use the Grid DNS default and automatically resets the override_ftc method to "false".

Note that when ftc_expired_record_ttl is set to a defined value and override_ftc is set to "false", the last operation takes precedence. Thus the sequence $object->ftc_expired_record_ttl("true"); $object->override_ftc("false"); will set override_ftc to "false", and the sequence $object->override_ftc("false"); $object->ftc_expired_record_ttl("true"); will result in override_ftc="true".

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

A 32-bit integer, from 0 to 4294967295, that represents the TTL in seconds. The default value is 5.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get ftc_expired_record_ttl
my $ftc_expired_record_ttl = $memberdns->ftc_expired_record_ttl( );
#Modify ftc_expired_record_ttl
$memberdns->ftc_expired_record_ttl(60);
```

ftc_expired_record_timeout( )

Use this method to set or retrieve the timeout value after which the expired Fault Tolerant Caching (FTC) record becomes stale and is no longer valid.

Setting this method to a defined value implicitly sets the override_ftc method to "true". Setting the parameter to undefined if ftc_expired_record_ttl and enable_ftc are also undefined causes the appliance to use the Grid DNS default and automatically resets the override_ftc method to "false".

Note that when ftc_expired_record_timeout is set to a defined value and override_ftc is set to "false", the last operation takes precedence. Thus the sequence $object->ftc_expired_record_timeout("true"); $object->override_ftc("false"); will set override_ftc to "false", and the sequence $object->override_ftc("false"); $object->ftc_expired_record_timeout("true"); will result in override_ftc="true".

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

A 32-bit integer, from 0 to 4294967295, that represents the timeout in seconds. The default value is 86400.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get ftc_expired_record_timeout
my $ftc_expired_record_timeout = $memberdns->ftc_expired_record_timeout( );
#Modify ftc_expired_record_timeout
$memberdns->ftc_expired_record_timeout(3600);
```

glue_record_addresses( )

Use this method to specify which IP address the appliance uses when it generates the glue record. It can use the IP address of the appliance, the NAT IP address, or an IP address that you define.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

Valid value is an array reference that contains Infoblox::DNS::GlueRecordAddr object(s).

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
my $glue_record_addr1 = Infoblox::DNS::GlueRecordAddr->new(
    view                => "default",  #can only be one of existing views
    glue_record_address => "NAT",
);
$memberdns->glue_record_addresses([$glue_record_addr1]);
```

gss_tsig_keys( )

Use this method to set or retrieve the list of GSS-TSIG keys for a member DNS object.

Setting this method to a defined value implicitly sets the override_gss_tsig_keys method to "true". Setting the parameter to undefined causes the appliance to use the grid default and automatically resets the override_gss_tsig_keys method to "false".

Note that when gss_tsig_keys is set to a defined value and override_gss_tsig_keys is set to "false", the last operation takes precedence. Thus the sequence $object->gss_tsig_keys($keys); $object->override_gss_tsig_keys("false"); will set override_gss_tsig_keys to "false", and the sequence $object->override_gss_tsig_keys("false"); $object->gss_tsig_keys($keys); will result in override_gss_tsig_keys="true".

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

The valid value is an array reference that contains Infoblox::Grid::KerberosKey objects.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get the Kerberos keys list
my $kerberos_keys_list = $memberdns->gss_tsig_keys();
#Modify the Kerberos keys list
my $kerberos_key1 = Infoblox::Cluster::KerberosKey->new(
       principal => 'host/appliance.ivan.test1@LADYBUG.LOCAL',
       );
my $kerberos_key2 = Infoblox::Cluster::KerberosKey->new(
       principal => 'host/appliance.ivan.test2@LADYBUG.LOCAL',
       );
$memberdns->gss_tsig_keys([$kerberos_key1, $kerberos_key2]);
```

host_name_restriction_policy( )

Use this method to set or retrieve the member-level host name restriction policy that overrides the grid-level settings.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

Valid value is the name of record name policy defined at the grid level via Infoblox::Grid::RecordNamePolicy

There are three predefined policies:

Policy Name Regex Applied

"Allow Any" .+

"Allow Underscore" ^[-a-zA-Z0-9_.]+$

"Strict Hostname Checking" ^[a-zA-Z0-9]$|^[a-zA-Z0-9][-a-zA-Z0-9.]*[a-zA-Z0-9]$

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Getting host_name_restriction_policy
$memberdns->host_name_restriction_policy( );
#Modifying host_name_restriction_policy
$memberdns->host_name_restriction_policy("Allow Any");
#Un-override host_name_restriction_policy
$memberdns->host_name_restriction_policy(undef);
```

ipv6_glue_record_addresses( )

Use this method to specify which IPv6 address the appliance uses when it generates the glue record. It can use the IPv6 address of the appliance, the NAT IPv6 address, or an IPv6 address that you define.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

Valid value is an array of Infoblox::DNS::GlueRecordAddr objects.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
my $v6_glue_record_addr1 = Infoblox::DNS::GlueRecordAddr->new(
    view                => "default",  #can only be one of existing views
    glue_record_address => "2001:db8::1",
);

$memberdns->ipv6_glue_record_addresses([$v6_glue_record_addr1]);
```

is_unbound_capable( )

Use this method to retrieve the flag that indicates whether Member DNS supports unbound recursive resolver or not. This is a read-only attribute.

Omit the parameter to retrieve the attribute value.

Parameter

None

Returns

The method returns the attribute value.

Example

```
#Get is_unbound_capable
my $is_unbound_capable = $memberdns->is_unbound_capable();
```

lame_ttl( )

Use this method to set or retrieve the number of seconds to cache lame delegations or lame servers.

Parameter

The number of seconds to cache lame delegations or lame servers. The default value is 600.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get lame_ttl
my $lame_ttl = $memberdns->lame_ttl();
#Modify lame_ttl
$memberdns->lame_ttl(700);
```

logging_categories( )

Use this method to set logging categories for this member. This setting overrides grid logging category settings.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

Valid value is an array reference that contains one or more of the following values:

"general": Records the BIND messages that are not specifically classified.

"config": Records the configuration file parsing messages.

"dnssec": Records the DNSSEC-signed responses.

"network": Records the network operation messages.

"queries": Records the query messages.

"responses": Records the response messages.

"security": Records the approved and denied requests.

"xfer-in": Records zone transfer messages from the remote name servers to the appliance.

"xfer-out": Records zone transfer messages from the Infoblox appliance to remote name servers.

"update": Records the dynamic update instances.

"resolver": Records the DNS resolution instances, including recursive queries from resolvers.

"notify": Records the asynchronous zone change notification messages.

"lame_servers": Records bad delegation instances.

"database": Records BIND's internal database processes.

"client": Records client requests.

"rpz": Records RPZ messages.

"dtc_gslb": Records DTC GSLB activity.

"dtc_health": Records DTC health monitoring.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Getting logging_categories
$memberdns->logging_categories( );
#Modifying logging_categories
$memberdns->logging_categories( [ "general", "database" , "queries" ] );
```

max_cache_ttl( )

Use this method to set or retrieve the maximum time (in seconds) for which the server will cache positive answers.

Parameter

The maximum time is specified in seconds. The default value is 604800.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get max_cache_ttl
my $max_cache_ttl = $memberdns->max_cache_ttl();
#Modify max_cache_ttl
$memberdns->max_cache_ttl(400);
```

max_ncache_ttl( )

Use this method to set or retrieve the maximum time (in seconds) for which the server will cache negative (NXDOMAIN) responses. This method applies for the defined member configuration.

Parameter

The maximum time is specified in seconds. The default value is 10800. The maximum allowed value is 604800.

Setting this parameter implicitle sets the override_max_ncache_ttl attribute to "true". Setting the parameter to undefined causes the appliance to use the Grid-level settings and automatically sets the override_max_ncache_ttl attribute to "false".

Note that when max_ncache_ttl is set and override_max_ncache_ttl is set to "false", the last operation takes precedence. Thus the sequence $firstview->max_ncache_ttl(10800); $firstview->override_max_ncache_ttl("false"); sets override_max_ncache_ttl to "false", and the sequence $firstview->override_max_ncache_ttl("false"); $firstview->max_ncache_ttl(30600); sets override_max_ncache_ttl to "true".

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get max_ncache_ttl
my $max_ncache_ttl = $memberdns->max_ncache_ttl();
#Modify max_ncache_ttl
$memberdns->max_ncache_ttl(400);
```

max_cached_lifetime( )

Use this method to set or retrieve maximum time in seconds a DNS response can be stored in the hardware acceleration cache.

Parameter

Valid values are unsigned integer between 60 and 86400, inclusive. Default value is 86400.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get max_cached_lifetime
my $max_cached_lifetime = $memberdns->max_cached_lifetime();
#Modify max_cached_lifetime
$memberdns->max_cached_lifetime(400);
```

minimal_resp( )

Use this method to enable/disable the ability to return a minimal amount of data in response to a query. This capability speeds up the DNS services provided by the appliance.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

Specify "true" to enable minimal responses or "false" to desable it. Default is "true"

Returns

If you specified a parameter, the method returns 1 when the modification succeeds, and returns other values when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Getting minimal_resp
$memberdns->minimal_resp( );
#Modifying minimal_resp
$memberdns->minimal_resp("true");
```

name( )

Use this method to retrieve the grid name.

Parameter

None.

Returns

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get name
my $name = $memberdns->name();
```

notify_delay( )

Use this method to specify with how many seconds of delay the notify messages are sent to secondaries.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

Valid values are unsigned integer between 5 and 86400, inclusive. Default value is undefined.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Getting notify_delay
my $notify_delay = $memberdns->notify_delay( );
#Modifying notify_delay
$memberdns->notify_delay(5);
```

notify_source_port( )

Use this method to specify the source port for notify messages. When requesting zone transfers from the primary server, some secondary DNS servers use the source port number (the primary server used to send the notify message) as the destination port number in the zone transfer request. This setting overrides grid static source port settings.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

Source port number for notify messages. Valid values are between 1 and 63999. The default is picked by BIND.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Getting notify_source_port
$memberdns->notify_source_port( );
#Modifying notify_source_port
$memberdns->notify_source_port("2");
```

override_attack_mitigation( )

The override_attack_mitigation attribute controls whether the attack_mitigation method values of the member are used, instead of the Grid default.

The override_attack_mitigation attribute can be specified explicitly. It is also set implicitly when attack_mitigation is set to a defined value.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

Set the parameter to "true" to override the Grid-level setting for attack_mitigation. Set the parameter to "false" to inherit the Grid-level setting for attack_mitigation.

The default value of this parameter is "false".

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Getting override_attack_mitigation
my $override_attack_mitigation = $memberdns->override_attack_mitigation( );
#Modifying override_attack_mitigation
$memberdns->override_attack_mitigation("true");
```

override_auto_blackhole( )

The override_auto_blackhole attribute controls whether the auto_blackhole method values of the member are used, instead of the Grid default.

The override_auto_blackhole attribute can be specified explicitly. It is also set implicitly when auto_blackhole is set to a defined value.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

Set the parameter to "true" to override the Grid-level setting for auto_blackhole. Set the parameter to "false" to inherit the Grid-level setting for auto_blackhole.

The default value of this parameter is "false".

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Getting override_auto_blackhole
my $override_auto_blackhole = $memberdns->override_auto_blackhole( );
#Modifying override_auto_blackhole
$memberdns->override_auto_blackhole("true");
```

override_bind_hostname_directive( )

The override_bind_hostname_directive attribute controls whether the bind_hostname_directive method values of the member are used, instead of the Grid default.

The override_bind_hostname_directive attribute can be specified explicitly. It is also set implicitly when bind_hostname_directive is set to a defined value.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

Set the parameter to "true" to override the Grid-level setting for bind_hostname_directive. Set the parameter to "false" to inherit the Grid-level setting for bind_hostname_directive.

The default value of this parameter is "false".

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Getting override_bind_hostname_directive
my $override_bind_hostname_directive = $memberdns->override_bind_hostname_directive( );
#Modifying override_bind_hostname_directive
$memberdns->override_bind_hostname_directive("true");
```

override_server_id_directive( )

The override_server_id_directive attribute controls whether the server_id_directive method values of the member are used, instead of the Grid default.

The override_server_id_directive attribute can be specified explicitly. It is also set implicitly when server_id_directive is set to a defined

value.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

Set the parameter to "true" to override the Grid-level setting for server_id_directive. Set the parameter to "false" to inherit the Grid-level setting for server_id_directive.

The default value of this parameter is "false".

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Getting override_server_id_directive
my $override_server_id_directive = $memberdns->override_server_id_directive( );
#Modifying override_server_id_directive
$memberdns->override_server_id_directive("true");
```

override_capture_dns_queries_on_all_domains( )

The override_capture_dns_queries_on_all_domains attribute controls whether the capture_dns_queries_on_all_domains method values of the member are used, instead of the Grid default.

The override_capture_dns_queries_on_all_domains attribute can be specified explicitly. It is also set implicitly when capture_dns_queries_on_all_domains is set to a defined value.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

Set the parameter to "true" to override the Grid-level setting for capture_dns_queries_on_all_domains. Set the parameter to "false" to inherit the Grid-level setting for capture_dns_queries_on_all_domains.

The default value of this parameter is "false".

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
 #Get override_capture_dns_queries_on_all_domains value
 my $override_capture_dns_queries_on_all_domains = $object-
>override_capture_dns_queries_on_all_domains();

 #Modify override_capture_dns_queries_on_all_domains value
 $object->override_capture_dns_queries_on_all_domains("true");
```

override_copy_xfer_to_notify( )

The override_copy_xfer_to_notify attribute controls whether the copy_xfer_to_notify method values of the member are used, instead of the grid default.

The override_copy_xfer_to_notify attribute can be specified explicitly. It is also set implicitly when copy_xfer_to_notify is set to a defined value.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

Set the parameter to "true" to override the grid-level setting for copy_xfer_to_notify. Set the parameter to "false" to inherit the grid-level setting for copy_xfer_to_notify.

The default value of this parameter is "false".

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Getting override_copy_xfer_to_notify
my $override_copy_xfer_to_notify = $memberdns->override_copy_xfer_to_notify( );
#Modifying override_copy_xfer_to_notify
$memberdns->override_copy_xfer_to_notify("true");
```

override_disable_edns( )

The override_disable_edns attribute controls whether the disable_edns method values of the member are used, instead of the Grid default.

The override_disable_edns attribute can be specified explicitly. It is also set implicitly when disable_edns is set to a defined value.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

Set the parameter to "true" to override the grid-level setting for disable_edns. Set the parameter to "false" to inherit the grid-level setting for disable_edns. The default value is "false".

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get override_disable_edns
my $override_disable_edns = $memberdns->override_disable_edns();
#Modify override_disable_edns
$memberdns->override_disable_edns("true");
```

override_dns64( )

The override_dns64 attribute controls whether the DNS64 method values of the member are used, instead of the grid default.

The override_dns64 attribute can be specified explicitly. It is also set implicitly when enable_dns64 or dns64_groups are set to a defined value.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

Set the parameter to "true" to override the grid-level setting for enable_dns64 and dns64_groups. Set the parameter to "false" to inherit the grid-level setting for enable_dns64 and dns64_groups.

The default value of this parameter is "false".

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Getting override_dns64
my $override_dns64=$memberdns->override_dns64( );
#Modifying override_dns64
$memberdns->override_dns64("true");
```

override_dns_health_check( )

The override_dns_health_check attribute controls whether the DNS health check settings Member are used, instead of the Grid default.

The override_dns_health_check attribute can be specified explicitly. It is also set implicitly when following fields are set to a defined value:

```
dns_health_check_domain_list
dns_health_check_recursion_flag
dns_health_check_anycast_control
dns_health_check_interval
dns_health_check_retries
dns_health_check_timeout
enable_dns_health_check
```

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

Specify 'true' to override the Grid-level DNS health check setting or 'false' to inherit the Grid-level DNS health check setting.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get override_dns_health_check value
my $value = $object->override_dns_health_check();

#Modify override_dns_health_check value
$object->override_dns_health_check('true');
```

override_filter_aaaa( )

The override_filter_aaaa attribute controls whether the AAAA filter method values of the member are used, instead of the grid default.

The override_filter_aaaa attribute can be specified explicitly. It is also set implicitly when filter_aaaa or filter_aaaa_list are set to a defined value.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

Set the parameter to "true" to override the grid-level setting for filter_aaaa and filter_aaaa_list. Set the parameter to "false" to inherit the grid-level setting for filter_aaaa and filter_aaaa_list.

The default value of this parameter is "false".

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Getting override_filter_aaaa
my $override_filter_aaaa=$memberdns->override_filter_aaaa( );
#Modifying override_filter_aaaa
$memberdns->override_filter_aaaa("true");
```

override_dns_cache_acceleration_ttl( )

The override_dns_cache_acceleration_ttl attribute controls whether the dns_cache_acceleration_ttl method values of the member are used, instead of the grid default.

The override_dns_cache_acceleration_ttl attribute can be specified explicitly. It is also set implicitly when dns_cache_acceleration_ttl is set to a defined value.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

Set the parameter to "true" to override the grid-level setting for dns_cache_acceleration_ttl. Set the parameter to "false" to inherit the grid-level setting for dns_cache_acceleration_ttl.

The default value of this parameter is "false".

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Getting override_dns_cache_acceleration_ttl
my $override_dns_cache_acceleration_ttl=$memberdns->override_dns_cache_acceleration_ttl( );
#Modifying override_dns_cache_acceleration_ttl
$memberdns->override_dns_cache_acceleration_ttl("true");
```

override_ftc( )

The override_ftc attribute controls whether the values of the Fault Tolerant Caching (FTC) feature are used for the member, instead of the

Grid default.

The override_ftc attribute can be specified explicitly. It is also set implicitly when the following fields are set to a defined value:

```
enable_ftc
ftc_expired_record_ttl
ftc_expired_record_timeout
```

Parameter

Set the parameter to "true" to override the Grid-level setting for FTC feature. Set the parameter to "false" to inherit the Grid-level setting for FTC. The default value is "false".

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get override_ftc
my $override_ftc = $memberdns->override_ftc();
#Modify override_ftc
$memberdns->override_ftc('true');
```

override_gss_tsig_keys( )

The override_gss_tsig_keys attribute controls whether the gss_tsig_keys method values for the member are used, instead of the Grid default.

The override_gss_tsig_keys attribute can be specified explicitly. It is also set implicitly when gss_tsig_keys is set to a defined value.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

Set the parameter to "true" to override GSS-TSIG keys. Set the parameter to "false" to inherit the grid-level GSS-TSIG keys.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Getting override_gss_tsig_keys
my $override_gss_tsig_keys = $memberdns->override_gss_tsig_keys();
#Modyfying override_gss_tsig_keys
$memberdns->override_gss_tsig_keys("true");
```

override_enable_capture_dns( )

The override_enable_capture_dns attribute controls whether the enable_capture_dns_queries and enable_capture_dns_responses method values of the member are used, instead of the Grid default.

The override_enable_capture_dns attribute can be specified explicitly. It is also set implicitly when enable_capture_dns_queries or enable_capture_dns_responses is set to a defined value.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

Set the parameter to "true" to override the Grid-level setting for enable_capture_dns_queries and enable_capture_dns_responses. Set the parameter to "false" to inherit the Grid-level setting for enable_capture_dns_queries and enable_capture_dns_responses.

The default value of this parameter is "false".

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get override_enable_capture_dns value
my $override_enable_capture_dns = $object->override_enable_capture_dns();
```

```
#Modify override_enable_capture_dns value
$object->override_enable_capture_dns("true");
```

override_enable_excluded_domain_names( )

The override_enable_excluded_domain_names attribute controls whether the capture_dns_queries_on_all_domains method values of the member are used, instead of the Grid default.

The override_enable_excluded_domain_names attribute can be specified explicitly. It is also set implicitly when capture_dns_queries_on_all_domains is set to a defined value.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

Set the parameter to "true" to override the Grid-level setting for capture_dns_queries_on_all_domains. Set the parameter to "false" to inherit the Grid-level setting for capture_dns_queries_on_all_domains.

The default value of this parameter is "false".

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get override_enable_excluded_domain_names value
my $override_enable_excluded_domain_names = $object->override_enable_excluded_domain_names();

#Modify override_enable_excluded_domain_names value
$object->override_enable_excluded_domain_names("true");
```

override_enable_gss_tsig( )

The override_enable_gss_tsig attribute controls whether the enable_gss_tsig method value for the member is used, instead of the Grid default.

The override_enable_gss_tsig attribute can be specified explicitly. It is also set implicitly when enable_gss_tsig is set to a defined value.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

Set the parameter to "true" to override the grid-level setting for enable_gss_tsig. Set the parameter to "false" to inherit the grid-level setting for enable_gss_tsig value.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Getting override_enable_gss_tsig
my $override_enable_gss_tsig = $memberdns->override_enable_gss_tsig();
#Modyfying override_enable_gss_tsig
$memberdns->override_enable_gss_tsig("true");
```

override_max_cache_ttl( )

The override_max_cache_ttl attribute controls whether the max_cache_ttl method values of the member are used, instead of the grid default.

The override_max_cache_ttl attribute can be specified explicitly. It is also set implicitly when max_cache_ttl is set to a defined value.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

Set the parameter to "true" to override the grid-level setting for max_cache_ttl. Set the parameter to "false" to inherit the grid-level setting for max_cache_ttl.

The default value of this parameter is "false".

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Getting override_max_cache_ttl
my $override_max_cache_ttl = $memberdns->override_max_cache_ttl( );
#Modifying override_max_cache_ttl
$memberdns->override_max_cache_ttl("true");
```

override_max_ncache_ttl( )

The override_max_ncache_ttl attribute controls whether the max_ncache_ttl method values of the member are used, instead of the Grid default.

The override_max_ncache_ttl attribute can be specified explicitly. It is also set implicitly when max_ncache_ttl is set to a defined value.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

Set the parameter to "true" to override the grid-level setting for max_ncache_ttl. Set the parameter to "false" to inherit the Grid-level setting for max_ncache_ttl.

The default value of this parameter is "false".

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Getting override_max_ncache_ttl
my $override_max_ncache_ttl = $memberdns->override_max_ncache_ttl( );
#Modifying override_max_ncache_ttl
$memberdns->override_max_ncache_ttl("true");
```

override_max_cached_lifetime( )

The override_max_cached_lifetime attribute controls whether the max_cached_lifetime method values of the member are used, instead of the grid default.

The override_max_cached_lifetime attribute can be specified explicitly. It is also set implicitly when max_cached_lifetime is set to a defined value.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

Set the parameter to "true" to override the grid-level setting for max_cached_lifetime. Set the parameter to "false" to inherit the grid-level setting for max_cached_lifetime.

The default value of this parameter is "false".

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Getting override_max_cached_lifetime
my $override_max_cached_lifetime = $memberdns->override_max_cached_lifetime( );
#Modifying override_max_cached_lifetime
$memberdns->override_max_cached_lifetime("true");
```

override_nxdomain_redirect( )

The "override_nxdomain_redirect" attribute controls whether the "nxdomain_redirect", "nxdomain_redirect_addresses", "nxdomain_redirect_addresses_ipv6", "nxdomain_redirect_ttl", "nxdomain_log_query", and "nxdomain_rulesets" values for this member are used. Note that these values can be overridden at the DNS view level as well.

The "override_nxdomain_redirect" attribute can be specified explicitly. It is also set implicitly as follows: "nxdomain_redirect" => "true"|"false" or "nxdomain_redirect_addresses" => ["10.0.0.10", "10.0.0.20"]|[] or "nxdomain_redirect_addresses" => ["2001:db8::1",

"2002:ffff::1"]|[] result in "override_nxdomain_redirect" => "true", "nxdomain_redirect" => undef and "nxdomain_redirect_addresses" => undef and "nxdomain_redirect_addresses_ipv6" => undef result in "override_nxdomain_redirect" => "false". If "override_nxdomain_redirect" is specified explicitly as "true", and "nxdomain_redirect" => undef, "nxdomain_redirect_addresses" => undef and "nxdomain_redirect_addresses_ipv6" => undef, then the explicit value takes precedence; and "override_nxdomain_redirect"=>"true", "nxdomain_redirect", "nxdomain_redirect_addresses", and "nxdomain_redirect_addresses" remain unchanged in the database.

Specifying the "nxdomain_redirect_ttl", "nxdomain_log_query", or "nxdomain_rulesets" attributes does not affect the "override_nxdomain_redirect" value.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

Set the parameter to "true" to override the grid-level setting of the NXDOMAIN redirection attributes. Set the parameter to "false" to inherit the grid-level settings.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get override_nxdomain_redirect
my $override_nxdomain_redirect=$memberdns->override_nxdomain_redirect( );
#Modify override_nxdomain_redirect
$memberdns->override_nxdomain_redirect("true");
```

override_lame_ttl( )

Use this method to set or retrieve the override_lame_ttl attribute that controls whether the lame_ttl method values of the member are used, instead of the Grid default

Parameter

Set the parameter to "true" to override the grid-level setting for lame_ttl. Set the parameter to "false" to inherit the grid-level setting for lame_ttl. The default value is 'false'.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get override_lame_ttl
my $override_lame_ttl = $memberdns->override_lame_ttl();
#Modify override_lame_ttl
$memberdns->override_lame_ttl('true');
```

override_rpz_qname_wait_recurse( )

The override_rpz_qname_wait_recurse attribute controls whether the rpz_qname_wait_recurse method value for the member is used, instead of the Grid default.

The "override_rpz_qname_wait_recurse" attribute can be specified explicitly. It is also established implicitly as follows: "rpz_qname_wait_recurse" => "true"|"false" results in "override_rpz_qname_wait_recurse" => "true"; "rpz_qname_wait_recurse" => undef results in "override_rpz_qname_wait_recurse" => "false"; If "override_rpz_qname_wait_recurse" is specified explicitly as "true", while "rpz_qname_wait_recurse"=>undef, the explicit value takes precedence, and "override_rpz_qname_wait_recurse"=>"true" while "rpz_qname_wait_recurse" remains unchanged in the database.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

Set the parameter to "true" to override the Grid-level setting for rpz_qname_wait_recurse. Set the parameter to "false" to inherit the Grid-level setting for rpz_qname_wait_recurse value.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Getting override_rpz_qname_wait_recurse
my $override_rpz_qname_wait_recurse = $memberdns->override_rpz_qname_wait_recurse();
#Modyfying override_rpz_qname_wait_recurse
$memberdns->override_rpz_qname_wait_recurse("true");
```

override_transfers_in( )

Use this method to set or retrieve the override_transfers_in attribute that controls whether the transfers_in method values of the member are used, instead of the Grid default.

Parameter

Set the parameter to "true" to override the grid-level setting for transfers_in. Set the parameter to "false" to inherit the grid-level setting for transfers_in. The default value is 'false'.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get override_transfers_in
my $override_transfers_in = $memberdns->override_transfers_in();
#Modify override_transfers_in
$memberdns->override_transfers_in('true');
```

override_transfers_out( )

Use this method to set or retrieve the override_transfers_out attribute that controls whether the transfers_out method values of the member are used, instead of the Grid default.

Parameter

Set the parameter to "true" to override the grid-level setting for transfers_out. Set the parameter to "false" to inherit the grid-level setting for transfers_out. The default value is '

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get override_transfers_out
my $override_transfers_out = $memberdns->override_transfers_out();
#Modify override_transfers_out
$memberdns->override_transfers_out('true');
```

override_transfers_per_ns( )

Use this method to set or retrieve the transfers_per_ns attribute that controls whether the transfers_per_ns method values of the member are used, instead of the Grid default.

Parameter

Set the parameter to "true" to override the grid-level setting for transfers_per_ns. Set the parameter to "false" to inherit the grid-level setting for transfers_per_ns. The default value is 'false'.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get override_transfers_per_ns
my $override_transfers_per_ns = $memberdns->override_transfers_per_ns();
#Modify override_transfers_per_ns
$memberdns->override_transfers_per_ns('true');
```

override_serial_query_rate( )

Use this method to set or retrieve the override_serial_query_rate attribute that controls whether the serial_query_rate method values of the member are used, instead of the Grid default.

Parameter

Set the parameter to "true" to override the grid-level setting for serial_query_rate. Set the parameter to "false" to inherit the grid-level setting for serial_query_rate. The default value is 'false'.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get override_serial_query_rate
my $override_serial_query_rate = $memberdns->override_serial_query_rate();
#Modify override_serial_query_rate
$memberdns->override_serial_query_rate('true');
```

override_query_rewrite( )

Use this method to set or retrieve the override_query_rewrite that controls whether the enable_query_rewrite method values of the member are used, instead of the grid-level enable_query_rewrite value.

Parameter

Set the parameter to "true" to override the grid-level DNS query rewrite settings. Set the parameter to "false" to inherit the grid-level DNS query rewrite settings. The default is "false".

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get override_query_rewrite
my $override_query_rewrite = $memberdns->override_query_rewrite();
#Modify override_query_rewrite
$memberdns->override_query_rewrite("true");
```

nxdomain_redirect( )

Use this method to enable or disable NXDOMAIN redirection.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

Specify "true" to set the nxdomain_redirect flag or "false" to deactivate/unset it. The default is "false".

Setting this parameter to "true" or "false" implicitly sets the override_nxdomain_redirect attribute to "true". Setting the parameter to undefined causes the appliance to use the grid-level NXDOMAIN redirection settings and automatically sets the override_nxdomain_redirect attribute to "false" if nxdomain_redirect_addresses or nxdomain_redirect_addresses_ipv6 is also undefined. But if nxdomain_redirect_addresses or nxdomain_redirect_addresses_ipv6 contains a value, then the override_nxdomain_redirect attribute is set to true.

Note that when nxdomain_redirect is set to "true"/"false" and override_nxdomain_redirect is set to "false", the last operation takes precedence. Thus the sequence $grid_member->nxdomain_redirect("false"); $grid_member->override_nxdomain_redirect("false"); sets override_nxdomain_redirect to "false", and the sequence $grid_member->override_nxdomain_redirect("false"); $grid_member->nxdomain_redirect("false"); sets override_nxdomain_redirect to "true".

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Getting nxdomain_redirect
my $nxdomain_redirect = $grid_member->nxdomain_redirect( );
#Modifying nxdomain_redirect
$grid_member->nxdomain_redirect("true");
```

nxdomain_redirect_addresses( )

Use this method to set the NXDOMAIN redirection addresses.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

The valid value is an array of IPv4 addresses. The default value is undefined.

Setting this parameter to a defined value implicitly sets the override_nxdomain_redirect attribute to "true". Setting the parameter to undefined causes the appliance to use the grid-level NXDOMAIN redirection settings and automatically sets the override_nxdomain_redirect attribute to "false" if nxdomain_redirect and nxdomain_redirect_addresses_ipv6 is also undefined. But if nxdomain_redirect or nxdomain_redirect_addresses_ipv6 contains a value, then the override_nxdomain_redirect attribute is set to true.

Note that when nxdomain_redirect_addresses contains a valid array reference and override_nxdomain_redirect is set to "false", the last operation takes precedence. Thus the sequence $grid_member->nxdomain_redirect_addresses(["10.0.0.10", "10.0.0.20"]); $grid_member->override_nxdomain_redirect("false"); sets override_nxdomain_redirect to "false", and the sequence $grid_member->override_nxdomain_redirect("false"); $grid_member->nxdomain_redirect_addresses(["10.0.0.10", "10.0.0.20"]); sets override_nxdomain_redirect to "true".

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get nxdomain_redirect_addresses
my $nxdomain_redirect_addresses = $grid_member->nxdomain_redirect_addresses( );
#Modify nxdomain_redirect_addresses
$grid_member->nxdomain_redirect_addresses(["10.0.0.10", "10.0.0.20"]);
```

nxdomain_redirect_addresses_ipv6( )

Use this method to set the NXDOMAIN redirection IPv6 addresses.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

The valid value is an array of IPv6 addresses. The default value is undefined.

Setting this parameter to a defined value implicitly sets the override_nxdomain_redirect attribute to "true". Setting the parameter to undefined causes the appliance to use the grid-level NXDOMAIN redirection settings and automatically sets the override_nxdomain_redirect attribute to "false" if nxdomain_redirect and nxdomain_redirect_addresses is also undefined. But if nxdomain_redirect or nxdomain_redirect_addresses contains a value, then the override_nxdomain_redirect attribute is set to true.

Note that when nxdomain_redirect_addresses_ipv6 contains a valid array reference and override_nxdomain_redirect is set to "false", the last operation takes precedence. Thus the sequence $grid_member->nxdomain_redirect_addresses_ipv6(["2001:db8::1", "2002:ffff::1"]); $grid_member->override_nxdomain_redirect("false"); sets override_nxdomain_redirect to "false", and the sequence $grid_member->override_nxdomain_redirect("false"); $grid_member->nxdomain_redirect_addresses_ipv6(["2001:db8::1", "2002:ffff::1"]); sets override_nxdomain_redirect to "true".

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get nxdomain_redirect_addresses_ipv6
my $nxdomain_redirect_addresses_ipv6 = $grid_member->nxdomain_redirect_addresses_ipv6( );
#Modify nxdomain_redirect_addresses_ipv6
$grid_member->nxdomain_redirect_addresses_ipv6(["2001:db8::1", "2002:ffff::1"]);
```

nxdomain_redirect_ttl( )

Use this method to set or retrieve the TTL value of synthetic DNS responses that result from NXDOMAIN redirection.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

A 32-bit integer, from 0 to 4294967295, that represents the TTL in seconds. The default value is 60.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get nxdomain_redirect_ttl
my $nxdomain_redirect_ttl = $grid_member->nxdomain_redirect_ttl( );
#Modify nxdomain_redirect_ttl
$grid_member->nxdomain_redirect_ttl(60);
```

nxdomain_log_query( )

Use this method to set or retrieve the flag that indicates whether NXDOMAIN redirection queries are logged.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

Specify "true" to enable logging, or "false" to deactivate it. The default value is "false".

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get nxdomain_log_query
my $nxdomain_log_query = $grid_member->nxdomain_log_query( );
#Modify nxdomain_log_query
$grid_member->nxdomain_log_query("true");
```

nxdomain_rulesets( )

Use this method to specify or retrieve Infoblox::DNS::Ruleset object names assigned at the grid level for NXDOMAIN redirection.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

The valid value is an array reference that contains the names of the Infoblox::DNS::Ruleset objects assigned to the grid. The names must be in string format. The default value is undefined.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get nxdomain_rulesets
my $nxdomain_rulesets = $grid_member->nxdomain_rulesets();
#Modify nxdomain_rulesets
$grid_member->nxdomain_rulesets(["ruleset1", "ruleset2"]);
```

override_blacklist( )

The "override_blacklist" attribute controls whether the "enable_blacklist", "blacklist_redirect_addresses", "blacklist_redirect_ttl", "blacklist_log_query", and "blacklist_rulesets" values in the database are used.

The "override_blacklist" attribute can be specified explicitly. It is also set implicitly as follows: "enable_blacklist" => "true"|"false" or "blacklist_redirect_addresses" => ["10.0.0.10", "10.0.0.20"]|[] results in "override_blacklist" => "true"; "enable_blacklist" => undef and "blacklist_redirect_addresses" => undef result in "override_blacklist" => "false". If "override_blacklist" is specified explicitly as "true", and "enable_blacklist" => undef or "blacklist_redirect_addresses" => undef, then the explicit value takes precedence. Therefore, "override_blacklist"=>"true", "enable_blacklist"=>undef, and "blacklist_redirect_addresses"=>undef remain unchanged in the database.

Specifying the "blacklist_redirect_ttl", "blacklist_log_query", or "blacklist_rulesets" attributes does not affect the "override_blacklist" value. It is implicitly set only by "enable_blacklist" and "blacklist_redirect_addresses".

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

Set the parameter to "true" to override the grid-level settings of the blacklist redirect attributes. Set the parameter to "false" to inherit the grid-level settings. The default is "false".

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get override_blacklist
my $override_blacklist=$memberdns->override_blacklist( );
#Modify override_blacklist
$memberdns->override_blacklist("true");
```

override_dnssec( )

The "override_dnssec" attribute controls whether the "dnssec_enabled", "dnssec_validation_enabled", "dnssec_expired_signatures_enabled", and "dnssec_trusted_keys" values in the database are used. Note that these values can be overridden at the DNS view level as well.

The "override_dnssec" attribute can be specified explicitly. It is also established implicitly as follows: "dnssec_enabled" => "true"|"false" results in "override_dnssec" => "true" "dnssec_enabled" => undef results in "override_dnssec" => "false" If "override_dnssec" is specified explicitly as "true", while "dnssec_enabled"=>undef, the explicit value takes precedence, and "override_dnssec"=>"true" while "dnssec_enabled" remains unchanged in the database.

Supplying the "dnssec_validation_enabled", "dnssec_expired_signatures_enabled", or "dnssec_trusted_keys" attributes does not affect the "override_dnssec" value. It is implicitly affected only by "dnssec_enabled".

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

Set the parameter to "true" to override the grid-level setting of DNSSEC attributes. Set the parameter to "false" to inherit the grid-level settings.

The default value of this parameter is "false".

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Getting override_dnssec
my $override_dnssec=$memberdns->override_dnssec( );
#Modifying override_dnssec
$memberdns->override_dnssec("true");
```

override_dtc_edns_prefer_client_subnet( )

The override_dtc_edns_prefer_client_subnet attribute controls whether the dtc_edns_prefer_client_subnet method values of the member are used, instead of the Grid default.

The override_dtc_edns_prefer_client_subnet attribute can be specified explicitly. It is also set implicitly when dtc_edns_prefer_client_subnet is set to a defined value.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

Set the parameter to "true" to override the grid-level setting for dtc_edns_prefer_client_subnet. Set the parameter to "false" to inherit the Grid-level setting for dtc_edns_prefer_client_subnet. The default value is "false".

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get override_dtc_edns_prefer_client_subnet
my $override_dtc_edns_prefer_client_subnet = $memberdns->override_dtc_edns_prefer_client_subnet();
#Modify override_dtc_edns_prefer_client_subnet
$memberdns->override_dtc_edns_prefer_client_subnet("true");
```

override_forwarders( )

The "override_forwarders" attribute controls whether the "forward_only" and "forwarders" values in the database are used.

The "override_forwarders" attribute can be specified explicitly. It is also set implicitly as follows: "forward_only" => "true"|"false" or "forwarders" => ["10.0.0.10", "10.0.0.20"]|[] results in "override_forwarders" => "true"; "forward_only" => undef and "forwarders" => undef result in "override_forwarders" => "false". If "override_forwarders" is specified explicitly as "true", and "forward_only" => undef or "forwarders" => undef, then the explicit value takes precedence. Therefore, "override_forwarders"=>"true", "forward_only"=>undef, and "forwarders"=>undef remain unchanged in the database.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

Set the parameter to "true" to override the grid-level settings of the forwarders attributes. Set the parameter to "false" to inherit the grid-level settings. The default is "false".

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get override_forwarders
my $override_forwarders=$memberdns->override_forwarders( );
#Modify override_forwarders
$memberdns->override_forwarders("true");
```

override_fixed_rrset_order_fqdns( )

The override_fixed_rrset_order_fqdns controls whether the enable_fixed_rrset_order_fqdns and fixed_rrset_order_fqdns values in the database are used. Note that these values can be overridden at the DNS View level as well.

The override_fixed_rrset_order_fqdns can be specified explicitly. It is also set implicitly as follows: enable_fixed_rrset_order_fqdns => "true" | "false" or fixed_rrset_order_fqdns => $fqdns results in override_fixed_rrset_order_fqdns => "true";

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

Set the parameter to "true" to override the Grid-level fixed RRset order settings. Set the parameter to "false" to inherit the Grid-level settings. The default is "false".

Returns

When you specify a parameter and the modification succeeds, the method returns "true". When the operation fails, the method returns "false".

When you do not specify a parameter, the method returns the attribute value.

Example

```
#Get override_fixed_rrset_order_fqdns flag
my $override_fixed_rrset_order_fqdns = $member_dns->override_fixed_rrset_order_fqdns();
#Modify override_fixed_rrset_order_fqdns flag
$member_dns->override_fixed_rrset_order_fqdns("true");
```

query_source_port( )

Use this method to specify the source port for queries. Specifying a source port number for recursive queries ensures that a firewall will allow the response. This setting overrides grid static source port settings.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

Source port number for queries. Valid values are between 1 and 63999. The default is picked by BIND.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Getting query_source_port
$memberdns->query_source_port( );
#Modifying notify source port
```

```
$memberdns->query_source_port("2");
```

recursive_client_limit( )

Use this method to set a limit on the number of concurrent recursive clients.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

Maximum number of concurrent recursive clients. Range is from 0 to 4294967295

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Getting recursive_client_limit
$memberdns->recursive_client_limit( );
#Modifying recursive_client_limit
$memberdns->recursive_client_limit("10");
```

recursive_query_list( )

Use this method to set or retrieve the list of IPv4 or IPv6 addresses, networks or hosts authenticated by Transaction signature (TSIG) key from which recursive queries are allowed or denied. This setting overrides grid recursive query list settings.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

Valid value is an Infoblox::Grid::NamedACL object or an array reference that contains IPv4/IPv6 addresses, networks and/or Infoblox::DNS::TSIGKey objects. To deny queries, add '!' as a prefix to IP addresses and networks. Specify "any" to allow any recursive query, or undefine to use the Grid::DNS level recursive_query_list list. Note: You should also undefine the allow_recursive_query list to use Grid::DNS level settings.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Getting recursive_query_list
$memberdns->recursive_query_list( );
#Modifying recursive_query_list
$memberdns->recursive_query_list(["any"]);

#Creating TSIG key object
my $tsig_key2 = $session->gen_tsig_key();
my $key2 = Infoblox::DNS::TSIGKey->new(
   name => "rcl_key",
   key  => $tsig_key2
);
#Modifying recursive_query_list
$memberdns->recursive_query_list(["10.0.0.10", $key2,
"20.0.0.0/24","2010::256/64","!"."2011::23cb/64","!"."30.0.0.30", "!"."40.10.0.0/24"]);

#Use Grid::DNS level recursive_query_list
$memberdns->recursive_query_list(undef);
$memberdns->allow_recursive_query(undef);

#Use Infoblox::Grid::NamedACL
$memberdns->recursive_query_list($nacl);
```

resolver_query_timeout( )

Use this method to set or retrieve recursive query timeout for the member.

Setting this method to a defined value implicitly sets the override_resolver_query_timeout method to "true". Setting the parameter to undefined causes the appliance to use the Grid DNS default and automatically resets the override_resolver_query_timeout method to "false".

Note that when resolver_query_timeout is set to a defined value and override_resolver_query_timeout is set to "false", the last operation

takes precedence. Thus the sequence $object->resolver_query_timeout(29); $object->override_resolver_query_timeout("false"); will set override_resolver_query_timeout to "false", and the sequence $object->override_resolver_query_timeout("false"); $object->resolver_query_timeout(29); will result in override_resolver_query_timeout="true".

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

The valid value is an unsigned integer that contains the recursive query time (in seconds) for the resolver.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get resolver_query_timeout
my $resolver_query_timeout = $memberdns->resolver_query_timeout();
#Modify resolver_query_timeout
$memberdns->resolver_query_timeout(29);
```

response_rate_limiting( )

Use this method to set or retrieve response rate limiting settings for the member.

Setting this method to a defined value implicitly sets the override_response_rate_limiting method to "true". Setting the parameter to undefined causes the appliance to use the Grid DNS default and automatically resets the override_response_rate_limiting method to "false".

Note that when response_rate_limiting is set to a defined value and override_response_rate_limiting is set to "false", the last operation takes precedence. Thus the sequence $object->response_rate_limiting($response_rate_limiting); $object->override_response_rate_limiting("false"); will set override_response_rate_limiting to "false", and the sequence $object->override_response_rate_limiting("false"); $object->response_rate_limiting($response_rate_limiting); will result in override_response_rate_limiting="true".

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

The valid value is an Infoblox::Grid::DNS::ResponseRateLimiting object.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get response_rate_limiting
my $response_rate_limiting = $memberdns->response_rate_limiting();
#Modify response_rate_limiting
$memberdns->response_rate_limiting($response_rate_limiting);
```

recursive_resolver( )

Use this method to set or retrieve the recursive resolver for Member DNS.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

The valid value is 'BIND' or 'UNBOUND'. The default value is 'BIND'.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get recursive_resolver
my $recursive_resolver = $memberdns->recursive_resolver();

#Modify recursive_resolver
$memberdns->recursive_resolver('UNBOUND');
```

serial_query_rate( )

Use this method to set or retrieve the number of maximum concurrent SOA queries per second for the Member.

Parameter

Valid values are unsigned integer between 20 and 1000, inclusive. Default value is 20.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
# Get attribute value
my $value = $memberdns->serial_query_rate();
# Modify attribute value
$memberdns->serial_query_rate(30);
```

server_id_directive( )

Use this method to set or retrieve the value for the BIND and UNBOUND 'server-id' directive.

Setting this method to a defined value implicitly sets the override_server_id_directive method to "true". Setting the parameter to undefined causes the appliance to use the Grid DNS default and automatically resets the override_server_id_directive method to "false".

Note that when server_id_directive is set to a defined value and override_server_id_directive is set to "false", the last operation takes precedence. Thus the sequence $object->server_id_directive("HOSTNAME"); $object->override_server_id_directive("false"); will set override_server_id_directive to "false", and the sequence $object->override_server_id_directive("false"); $object->server_id_directive("true"); will result in override_server_id_directive="true".

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

The valid values are "NONE", "HOSTNAME" and "USER_DEFINED". The default value is "NONE".

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get server_id_directive
my $server_id_directive = $memberdns->server_id_directive();
#Modify server_id_directive
$memberdns->server_id_directive("HOSTNAME");
```

server_id_directive_string( )

Use this method to set or retrieve the value for the BIND and UNBOUND user-defined server ID for 'server-id' directive.

Note that to enable user-defined 'server-id' the server_id_directive should be set to "USER_DEFINED".

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

The valid value is a desired server ID in a string format. The default value is undefined.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get server_id_directive_string
my $server_id_directive_string = $memberdns->server_id_directive_string();
#Modify server_id_directive_string
$memberdns->server_id_directive_string("test-server-id-1");
```

skip_in_grid_rpz_queries( )

Use this method to set or retrieve the flag that indicates whether the RPZ rules are applied to queries originating from this member received by other members of the Grid or not.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

Specify 'true' to allow other members to recieve RPZ queries by other members and 'false' to forbid it.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get skip_in_grid_rpz_queries value
my $value = $object->skip_in_grid_rpz_queries();

#Modify skip_in_grid_rpz_queries value
$memberdns->skip_in_grid_rpz_queries('false');
```

sortlist( )

Use this method to create a sort list.

A sort list determines the order of addresses in responses made to DNS queries. This setting overrides grid sort list settings.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

Valid value is an array reference that contains Infoblox::DNS::Sortlist object(s).

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
my $sort1 = Infoblox::DNS::Sortlist->new (
                        "source_ipv4addr" => "1.2.3.4",
                        "match_list"      => [ "10.20.1.0/24" , "10.20.2.0/24"]
                );
$memberdns_object->sortlist( [ $sort1 ] );
```

store_locally( )

Use this method to set or retrieve the flag that indicates whether the storing of query capture reports on the appliance is enabled or disabled.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

Specify 'true' to enable storing of query capture reports on the appliance or 'false' to disable it. The default value is 'false'.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get store_locally value
my $store_locally = $object->store_locally();

#Modify store_locally value
$object->store_locally('true');
```

transfer_excluded_servers( )

Use this method to exclude specified DNS servers during zone transfers. This setting overrides grid zone transfer settings.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

The valid value is an array reference that contains IP addresses of the DNS server.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#This means that it will set zone transfer format to Many answers except these servers.
$memberdns->transfer_format("many_answers");
$memberdns->transfer_excluded_servers(["10.0.0.10", "20.0.0.0"]);
```

transfer_format( )

Use this method to set or retrieve the BIND format for a zone transfer. This provides tracking capabilities for single or multiple transfers and their associated servers. This setting overrides grid zone transfer format settings.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

Valid value is one of the following values:

`"Many Answers" (Secondaries run BIND 8/9)`: includes as many records as the packet size allows.

`"Many Answers" Except for these servers`: includes as many records as the packet size allows.

`"One Answer" (Secondaries run BIND 4)`: includes one record per packet.

`"One Answer" Except for these servers`: includes one record per packet.

If you specified servers in the transfer_excluded_servers method, the Many Answers or One Answer zone format (depending on transfer_format method setting) will allow zone transfers, except from the specified servers.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Getting transfer_format
$memberdns->transfer_format( );
#Modifying transfer_format
$memberdns->transfer_format("many_answers");
```

transfers_in( )

Use this method to set or retrieve the number of maximum concurrent transfers for the Member.

Parameter

Valid values are unsigned integer between 10 and 100, inclusive. Default value is 10.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
# Get attribute value
my $value = $memberdns->transfers_in();
# Modify attribute value
$memberdns->transfers_in(20);
```

transfers_out( )

Use this method to set or retrieve the number of maximum outbound concurrent zone transfers for the member.

Parameter

Valid values are unsigned integers between 1 and 100, inclusive. The default value is 10.

Specifying value in this parameter sets the override_transfers_out attribute to "true". Setting this parameter attribute to undefined causes a member to use the grid-level transfers_out value and automatically sets the override_transfers_out attribute to "false".

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
# Get attribute value
my $value = $memberdns->transfers_out();
# Modify attribute value
$memberdns->transfers_out(20);
```

transfers_per_ns( )

Use this method to set or retrieve the number of maximum concurrent transfers per member for the Member.

Parameter

Valid values are unsigned integer between 2 and 100, inclusive. Default value is 2.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
# Get attribute value
my $value = $memberdns->transfers_per_ns();
# Modify attribute value
$memberdns->transfers_per_ns(3);
```

unbound_logging_level( )

Use this method to set or retrieve the logging level for the unbound recursive resolver.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

The valid value is 'ERRORS_ONLY', 'OPERATIONS', 'DETAILED_OPERATIONS', 'QUERY', 'ALGORITHM' or 'CACHE_MISSES'.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get unbound_logging_level
my $unbound_logging_level = $memberdns->unbound_logging_level();

#Modify unbound_logging_level
$memberdns->unbound_logging_level('ALGORITHM');
```

use_lan_ipv6_port( )

Use this method to enable/disable DNS service on the IPv6 LAN port.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

Specify "true" to enable DNS service on the IPv6 LAN port or "false" to disable it. Default is "false"

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Getting use_lan_ipv6_port
$memberdns->use_lan_ipv6_port( );
#Modifying use_lan_ipv6_port
$memberdns->use_lan_ipv6_port("true");
```

use_lan2_port( )

Use this method to enable/disable DNS service on the LAN2 port.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

Specify "true" to enable DNS service on the LAN2 port or "false" to disable it. Default is "false"

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Getting use_lan2_port
$memberdns->use_lan2_port( );
#Modifying use_lan2_port
$memberdns->use_lan2_port("true");
```

use_lan2_ipv6_port( )

Use this method to enable/disable DNS service on the IPv6 LAN2 port.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

Specify "true" to enable DNS service on the IPv6 LAN2 port or "false" to disable it. Default is "false"

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Getting use_lan2_ipv6_port
$memberdns->use_lan2_ipv6_port( );
#Modifying use_lan2_ipv6_port
$memberdns->use_lan2_ipv6_port("true");
```

use_mgmt_port( )

Use this method to enable/disable DNS services on the MGMT port.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

Specify "true" to enable DNS services on the MGMT port or "false" to disable it. Default is "false"

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Getting use_mgmt_port
$memberdns->use_mgmt_port( );
#Modifying enable_dns
$memberdns->use_mgmt_port("true");
```

use_mgmt_ipv6_port( )

Use this method to enable/disable DNS services on the IPv6 MGMT port.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

Specify "true" to enable DNS services on the IPv6 MGMT port or "false" to disable it. Default is "false"

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Getting use_mgmt_ipv6_port
$memberdns->use_mgmt_ipv6_port( );
#Modifying enable_dns
$memberdns->use_mgmt_ipv6_port("true");
```

use_root_name_servers( )

Use this method to enable/disable custom or Internet root name servers. This setting overrides the grid root name server settings. Specify "False" to indicate that the member overrides the grid root server setting and uses Internet root name servers. Specify "True" to indicate that the member overrides the grid root server setting and uses custom root name servers. Specify "undef" to indicate that the member does not override the grid root server settings.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

Specify "true" to allow custom root name servers. Specify "false" to use Internet root name servers. Set it to undef to use the grid root name server settings.

Returns

If you specify a parameter, the method returns true when the modification succeeds, and returns false when the operation fails. If you do not specify a parameter, the method returns the attribute value.

Example

```
#Get use_root_name_servers
my $useRootNS = $memberdns->use_root_name_servers( );
#Modify use_root_name_servers
$memberdns->use_root_name_servers("false");
#Use grid root server settings
$memberdns->use_root_name_servers(undef);
```

rpz_disable_nsdname_nsip( )

Use this method to disable NSDNAME and NSIP resource records from RPZ feeds at member level.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

Specify "true" to enable the filter (to filter out NSDNAME and NSIP resource records). Specify "false" to disable the filter. The default value is "false".

Note that if both rpz_disable_nsdname_nsip and override_rpz_disable_nsdname_nsip are set to some value, the last operation takes precedence. Thus the sequence $grid_member->rpz_disable_nsdname_nsip("true"); $grid_member->override_rpz_disable_nsdname_nsip("false"); results in inheriting rpz_disable_nsdname_nsip value from the Grid. And the sequence $grid_member->override_rpz_disable_nsdname_nsip("false"); $grid_member->rpz_disable_nsdname_nsip("true"); sets override_rpz_disable_nsdname_nsip to "true".

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Getting rpz_disable_nsdname_nsip
$my $rpz_filter = $memberdns->rpz_disable_nsdname_nsip( );
#Modifying rpz_disable_nsdname_nsip
$memberdns->rpz_disable_nsdname_nsip("true");
```

rpz_drop_ip_rule_enabled( )

Use this method to set or retreive the flag that indicates whether the DNS server ignores RPZ-IP triggers with large prefix lengths or not.

Setting this method to a defined value implicitly sets the override_rpz_drop_ip_rule method to "true". Setting the parameter to undefined if rpz_drop_ip_rule_min_prefix_length_ipv4 and rpz_drop_ip_rule_min_prefix_length_ipv6 are undefined causes the appliance to use the Grid default and automatically resets the override_rpz_drop_ip_rule attribute to "false".

Note that when rpz_drop_ip_rule_enabled is set to a defined value and override_rpz_drop_ip_rule is set to "false", the last operation takes precedence. Thus the sequence $object->rpz_drop_ip_rule_enabled("true"); $object->override_rpz_drop_ip_rule("false"); will set override_rpz_drop_ip_rule to "false", and the sequence $object->override_rpz_drop_ip_rule("false"); $object->rpz_drop_ip_rule_enabled("true"); will result in override_rpz_drop_ip_rule="true".

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

Specify 'true' to enable ignoring of RPZ-IP triggers with large prefix lengths or 'false' to disable it. The default value is 'false'.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
# Get attribute value
my $value = $memberdns->rpz_drop_ip_rule_enabled();
# Modify attribute value
$memberdns->rpz_drop_ip_rule_enabled("true");
```

rpz_drop_ip_rule_min_prefix_length_ipv4( )

Use this method to set or retreive minimum prefix length for IPv4 RPZ-IP triggers.

Setting this method to a defined value implicitly sets the override_rpz_drop_ip_rule method to "true". Setting the parameter to undefined if rpz_drop_ip_rule_enabled and rpz_drop_ip_rule_min_prefix_length_ipv6 are undefined causes the appliance to use the Grid default and automatically resets the override_rpz_drop_ip_rule attribute to "false".

Note that when rpz_drop_ip_rule_min_prefix_length_ipv4 is set to a defined value and override_rpz_drop_ip_rule is set to "false", the last operation takes precedence. Thus the sequence $object->rpz_drop_ip_rule_min_prefix_length_ipv4("20"); $object->override_rpz_drop_ip_rule("false"); will set override_rpz_drop_ip_rule to "false", and the sequence $object->override_rpz_drop_ip_rule("false"); $object->rpz_drop_ip_rule_min_prefix_length_ipv4("20"); will result in override_rpz_drop_ip_rule="true".

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

The valid value is an unsigned integer between 1 and 31 that represents prefix length (in bits) for IPv4 RPZ-IP triggers.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
# Get attribute value
my $value = $member_dns->rpz_drop_ip_rule_min_prefix_length_ipv4();
# Modify attribute value
$member_dns->rpz_drop_ip_rule_min_prefix_length_ipv4("16");
```

rpz_drop_ip_rule_min_prefix_length_ipv6( )

Use this method to set or retreive minimum prefix length for IPv6 RPZ-IP triggers.

Setting this method to a defined value implicitly sets the override_rpz_drop_ip_rule method to "true". Setting the parameter to undefined if rpz_drop_ip_rule_enabled and rpz_drop_ip_rule_min_prefix_length_ipv4 are undefined causes the appliance to use the Grid default and automatically resets the override_rpz_drop_ip_rule attribute to "false".

Note that when rpz_drop_ip_rule_min_prefix_length_ipv6 is set to a defined value and override_rpz_drop_ip_rule is set to "false", the last operation takes precedence. Thus the sequence $object->rpz_drop_ip_rule_min_prefix_length_ipv6("20"); $object->override_rpz_drop_ip_rule("false"); will set override_rpz_drop_ip_rule to "false", and the sequence $object->override_rpz_drop_ip_rule("false"); $object->rpz_drop_ip_rule_min_prefix_length_ipv6("20"); will result in override_rpz_drop_ip_rule="true".

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

The valid value is an unsigned integer between 1 and 127 that represents prefix length (in bits) for IPv6 RPZ-IP triggers.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
# Get attribute value
my $value = $member_dns->rpz_drop_ip_rule_min_prefix_length_ipv6();
# Modify attribute value
$member_dns->rpz_drop_ip_rule_min_prefix_length_ipv6("120");
```

rpz_qname_wait_recurse( )

Use this method to enable or disable recursive RPZ lookups.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

Specify "true" to enable recursive RPZ lookups or "false" to disable them. The default value is "false".

Note that if both rpz_qname_wait_recurse and override_rpz_qname_wait_recurse are set to some values, the last operation takes precedence. Thus the sequence $grid_member->rpz_qname_wait_recurse("true"); $grid_member->override_rpz_qname_wait_recurse("false"); results in inheriting rpz_qname_wait_recurse value from the Grid. And the sequence $grid_member->override_rpz_qname_wait_recurse("false"); $grid_member->rpz_qname_wait_recurse("true"); sets override_rpz_qname_wait_recurse to "true".

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
# Get attribute value
my $value = $grid_member->rpz_qname_wait_recurse();
# Modify attribute value
$grid_member->rpz_qname_wait_recurse("true");
```

override_resolver_query_timeout( )

The override_resolver_query_timeout attribute controls whether the resolver_query_timeout method values of the member are used, instead of the Grid default.

The override_resolver_query_timeout attribute can be specified explicitly. It is also set implicitly when resolver_query_timeout is set to a defined value.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

Set the parameter to "true" to override the Grid-level setting for resolver_query_timeout. Set the parameter to "false" to inherit the Grid-level setting for resolver_query_timeout.

The default value of this parameter is "false".

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Getting override_resolver_query_timeout
my $override_resolver_query_timeout = $memberdns->override_resolver_query_timeout( );
#Modifying override_resolver_query_timeout
$memberdns->override_resolver_query_timeout("true");
```

override_response_rate_limiting( )

The override_response_rate_limiting attribute controls whether the response_rate_limiting method values of the member are used, instead of the Grid default.

The override_response_rate_limiting attribute can be specified explicitly. It is also set implicitly when response_rate_limiting is set to a defined value.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

Set the parameter to "true" to override the Grid-level setting for response_rate_limiting. Set the parameter to "false" to inherit the Grid-level setting for response_rate_limiting.

The default value of this parameter is "false".

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Getting override_response_rate_limiting
my $override_response_rate_limiting = $memberdns->override_response_rate_limiting( );
#Modifying override_response_rate_limiting
$memberdns->override_response_rate_limiting("true");
```

override_rpz_disable_nsdname_nsip( )

The override_rpz_disable_nsdname_nsip attribute controls whether the rpz_disable_nsdname_nsip method value for the member is used, instead of the Grid default.

The "override_rpz_disable_nsdname_nsip" attribute can be specified explicitly. It is also established implicitly as follows: "rpz_disable_nsdname_nsip" => "true"|"false" results in "override_rpz_disable_nsdname_nsip" => "true" "rpz_disable_nsdname_nsip" => undef results in "override_rpz_disable_nsdname_nsip" => "false" If "override_rpz_disable_nsdname_nsip" is specified explicitly as "true", while "rpz_disable_nsdname_nsip"=>undef, the explicit value takes precedence, and "override_rpz_disable_nsdname_nsip"=>"true" while "rpz_disable_nsdname_nsip" remains unchanged in the database.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

Set the parameter to "true" to override the Grid-level setting for rpz_disable_nsdname_nsip. Set the parameter to "false" to inherit the grid-level setting for rpz_disable_nsdname_nsip value.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Getting override_rpz_disable_nsdname_nsip
my $override_rpz_filter = $memberdns->override_rpz_disable_nsdname_nsip();
#Modyfying override_rpz_disable_nsdname_nsip
$memberdns->override_disable_nsdname_nsip("true");
```

override_rpz_drop_ip_rule( )

The override_rpz_drop_ip_rule attribute controls whether the rpz_drop_ip_rule_enabled, rpz_drop_ip_rule_min_prefix_length_ipv4 and rpz_drop_ip_rule_min_prefix_length_ipv6 method values of the member are used, instead of the Grid default.

The override_rpz_drop_ip_rule attribute can be specified explicitly. It is also set implicitly when one of rpz_drop_ip_rule_enabled, rpz_drop_ip_rule_min_prefix_length_ipv4 or rpz_drop_ip_rule_min_prefix_length_ipv6 methods is set to a defined value.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

Set the parameter to "true" to override the Grid-level setting for rpz_drop_ip_rule_enabled, rpz_drop_ip_rule_min_prefix_length_ipv4 and rpz_drop_ip_rule_min_prefix_length_ipv6 methods. Set the parameter to "false" to inherit the Grid-level setting for rpz_drop_ip_rule_enabled, rpz_drop_ip_rule_min_prefix_length_ipv4 and rpz_drop_ip_rule_min_prefix_length_ipv6 methods.

The default value of this parameter is "false".

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Getting override_rpz_drop_ip_rule
my $override_rpz_drop_ip_rule = $memberdns->override_rpz_drop_ip_rule( );
#Modifying override_rpz_drop_ip_rule
$memberdns->override_rpz_drop_ip_rule("true");
```

use_lan_port( )

Use this method to set or retrieve the status of the use of DNS services on the IPv4 LAN1 port.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

Specify true to enable DNS services on the IPv4 LAN1 port of false to disable it.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Getting use_lan_port
my $use_lan_port = $memberdns->use_lan_port();
#Modyfying use_lan_port
$memberdns->use_lan_port("true");
```

views( )

Use this method to specify or retrieve the view of this member.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value. The default value is the "default" view, which means the DNS member object is located under the default view.

Parameter

Valid value is an array reference that contains the names of the DNS views associated with the member. The names must be in string format.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get views
my $ref_views = $object->views();
#Modify DNS views, list of DNS view names in string format
$object->views(["view1","view2","view3"]);
```

SAMPLE CODE

The following sample code demonstrates the different functions that can be applied to an object,such as modify and get. This sample also includes error handling for the operations.

#Preparation prior to member DNS object operations

```
#PROGRAM STARTS: Include all the modules that will be used

use strict;
use Infoblox;

#Create a session to the Infoblox appliance
my $session = Infoblox::Session->new(
    master   => "192.168.1.2",
```

```
    username => "admin",
    password => "infoblox"
);
unless ($session) {
   die("Construct session failed: ",
       Infoblox::status_code() . ":" . Infoblox::status_detail());
}
print "Session created successfully\n";

# Get and modify the member DNS property object. Considering the id grid member named infoblox.localdomain
was previously created.
my $object = $session->get(
    object => "Infoblox::Grid::Member::DNS",
    name   => "infoblox.localdomain"
);

 unless ($object) {
      die("get member DNS failed: ",
      $session->status_code() . ":" . $session->status_detail());
  }
 print "member DNS get  successfull\n";

#Modifying the value of the specified object
$object->enable_dns("true");
$object->forward_only("false");
$object->minimal_resp("true");
$object->use_mgmt_port("true");
$object->notify_source_port(53);
$object->query_source_port(53);

 #Apply the changes
 $session->modify($object)
  or die("modify member DNS failed: ",
      $session->status_code() . ":" . $session->status_detail());

print "DNS member object modified successfully \n";

####PROGRAM ENDS####
```

AUTHOR

Infoblox Inc. http://www.infoblox.com/

SEE ALSO

Infoblox::DNS::View, Infoblox::Session, Infoblox::Grid::Member::DNS, Infoblox::DNS::DnssecTrustedKey, Infoblox::DNS::Ruleset, Infoblox::Session->get(), Infoblox::Session->modify(), Infoblox::Grid::NamedACL, Infoblox::Grid::DNS::AutoBlackHole, Infoblox::Grid::DNS::AttackMitigation, Infoblox::Grid::DNS::ResponseRateLimiting, Infoblox::Grid::Member::DNS::ViewAddressSetting, Infoblox::Grid::DNS::FileTransferSetting

COPYRIGHT

Copyright (c) 2017 Infoblox Inc.

Infoblox::DNS::FireEyeRuleMapping - FireEye Rule Mapping object

- NAME
- DESCRIPTION
- CONSTRUCTOR
- MODULE METHODS
  - Infoblox::DNS::Zone->fireeye_rule_mapping( )
  - apt_override( )
  - fireeye_alert_mapping( )
  - substituted_domain_name( )
- AUTHOR
- SEE ALSO
- COPYRIGHT

NAME

Infoblox::DNS::FireEyeRuleMapping - FireEye Rule Mapping object

---

DESCRIPTION

The rule map object is used to configure rule mapping for FireEye devices and Response Policy Zones.

CONSTRUCTOR

```
my $rulemapping = Infoblox::DNS::FireEye::Rulemapping->new(
    fireeye_alert_mapping => [$am1, am2, ...], #Optional
    apt_override => "PASSTHRU" | "NXDOMAIN" | "NODATA" | "SUBSTITUTE" | "NOOVERRIDE", #Optional
    substituted_domain_name => $fqdn, #Optional
);
```

MODULE METHODS

The following functions are available to apply to a FireEye Alert Map object.

Infoblox::DNS::Zone->fireeye_rule_mapping( )

Use this function to specify the alert map for this rule mapping. See Infoblox::DNS::Zone->fireeye_rule_mapping() for parameters and return values.

Example

```
my $am1 = Infoblox::DNS::FireEye::AlertMap->new(
                                                alert_type => 'INFECTION_MATCH',
                                                rpz_rule => 'PASSTHRU',
                                                lifetime => 0,
                                               );

unless($am1) {
     die("Construct member failed: ",
           Infoblox::status_code() . ":" . Infoblox::status_detail());
}

my $am2 = Infoblox::DNS::FireEye::AlertMap->new(
                                                alert_type => 'WEB_INFECTION',
                                                rpz_rule => 'NONE',
                                                lifetime => 1234,
                                               );

unless($am2) {
     die("Construct member failed: ",
           Infoblox::status_code() . ":" . Infoblox::status_detail());
}

my $rm = Infoblox::DNS::FireEye::RuleMapping->new(
                                                  fireeye_alert_mapping => [$am1, $am2],
                                                  apt_override => 'SUBSTITUTE',
                                                  substituted_domain_name => 'somedomain.com'
                                                 );
$rpz_zone->rpz_type('FIREEYE');
$rpz_zone->fireeye_rule_mapping($rm);
```

```
my $response = $session->modify($rpz_zone)
unless($response) {
     die("Modify zone failed: ",
           session->status_code() . ":" . session->status_detail());
}
print "Zone modified successfully\n";
```

apt_override( )

Use this method to set or retrieve the APT override.

Parameter

The valid value is one of "PASSTHRU", "NXDOMAIN", "NODATA", "SUBSTITUTE" or "NOOVERRIDE".

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get apt_override
my $apt_override = $rm->apt_override();
#Modify apt_override
$rm->apt_override("NOOVERRIDE");
```

fireeye_alert_mapping( )

Use this method to set or retrieve the FireEye alert mapping.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Returns

An array of the Infoblox::DNS::FireEye::AlertMap manpage objects.

Example

```
#Get fireeye_alert_mapping
my $fireeye_alert_mapping = $zone->fireeye_alert_mapping();
#Modify fireeye_alert_mapping
$rm->fireeye_alert_mapping([$am1, $am2]);
```

substituted_domain_name( )

Use this method to set or retrieve the domain name to be substituted, this is applicable only when apt_override is set to "SUBSTITUTE".

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

The domain name to use.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get substituted_domain_name
my $substituted_domain_name = $rm->substituted_domain_name();
#Modify substituted_domain_name
$rm->substituted_domain_name("somedomain.com");
```

AUTHOR

Infoblox Inc. http://www.infoblox.com/

SEE ALSO

Infoblox::DNS::Zone, Infoblox::DNS::Zone

COPYRIGHT

Copyright (c) 2017 Infoblox Inc.

Infoblox::DNS::FireEyeAlertMap - FireEye Alert Map object

- NAME
- DESCRIPTION
- CONSTRUCTOR
- MODULE METHODS
  - Infoblox::DNS::FireEye::RuleMapping->fireeye_alert_mapping( )
- METHODS
  - alert_type( )
  - rpz_rule( )
  - lifetime( )
- AUTHOR
- SEE ALSO
- COPYRIGHT

NAME

Infoblox::DNS::FireEyeAlertMap - FireEye Alert Map object

DESCRIPTION

The alert map object is used to configure FireEye alerts to Response Policy Zone rules.

CONSTRUCTOR

```
 my $alertmap = Infoblox::DNS::FireEye::AlertMap->new(
     alert_type => "INFECTION_MATCH" | "WEB_INFECTION" | "MALWARE_OBJECT" | "DOMAIN_MATCH" |
"MALWARE_CALLBACK", # Required
     rpz_rule   => "PASSTHRU" | "NXDOMAIN" | "NODATA" | "SUBSTITUTE" | "NONE", # Required
     lifetime   => $uint, # Required
 );
```

MODULE METHODS

The following functions are available to apply to a FireEye Alert Map object.

Infoblox::DNS::FireEye::RuleMapping->fireeye_alert_mapping( )

Use this function to specify the alert map for this rule mapping. See Infoblox::DNS::FireEye::RuleMapping->fireeye_alert_mapping() for parameters, return values and code samples.

METHODS

This section describes all the methods that you can use to set and retrieve the attribute values of a DNS Member object.

alert_type( )

Use this method to set or retrieve the alert type.

Parameter

The valid value is one of "INFECTION_MATCH", "WEB_INFECTION", "MALWARE_OBJECT", "DOMAIN_MATCH" or "MALWARE_CALLBACK".

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get alert_type
my $alert_type = $am->alert_type();
#Modify alert_type
$am->alert_type("INFECTION_MATCH");
```

rpz_rule( )

Use this method to set or retrieve the rpz rule.

Parameter

The valid value is one of "PASSTHRU", "NXDOMAIN", "NODATA", "SUBSTITUTE" or "NONE".

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get rpz_rule
my $rpz_rule = $am->rpz_rule();
#Modify rpz_rule
$am->rpz_rule("PASSTHRU");
```

lifetime( )

Use this method to set or retrieve the expiration lifetime for this alert type.

Parameter

The valid value is an unsigned integer, which specifies the amount of seconds this alert type will live for. 0 means the alert will never expire.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get lifetime
my $lifetime = $am->lifetime();
#Modify lifetime
$am->lifetime(1234);
```

AUTHOR

Infoblox Inc. http://www.infoblox.com/

SEE ALSO

Infoblox::DNS::Zone, Infoblox::DNS::Zone

COPYRIGHT

Copyright (c) 2017 Infoblox Inc.

Infoblox::DHCP::HostAddr - DHCP Host Address object.

- NAME
- DESCRIPTION
- CONSTRUCTOR
- SESSION METHODS
  - Infoblox::DNS::Host->ipv4addrs( )
  - Infoblox::Session->get( )
  - Infoblox::Session->modify( )
  - Infoblox::Session->search( )
- METHODS
  - bootfile( )
  - bootserver( )
  - configure_for_dhcp( )
  - deny_bootp( )
  - discovered_data( )
  - discovered_mac( )
  - discovered_name( )
  - discoverer( )
  - discover_now_status( )
  - enable_pxe_lease_time( )
  - first_discovered( )
  - host( )
  - ignore_client_requested_options( )
  - ipv4addr( )
  - is_invalid_mac( )
  - last_discovered( )
  - last_queried( )
  - logic_filters( )
  - mac( )
  - match_client( )
  - ms_ad_user_data( )
  - netbios( )
  - network( )
  - network_component_description( )
  - network_component_ip( )
  - network_component_name( )
  - network_component_port_description( )
  - network_component_port_name( )
  - network_component_port_number( )
  - network_component_type( )
  - nextserver( )
  - options( )
  - os( )
  - override_bootfile( )
  - override_bootserver( )
  - override_deny_bootp( )
  - override_ignore_client_requested_option( )
  - override_logic_filters( )
  - override_nextserver( )
  - override_options( )
  - override_pxe_lease_time( )
  - port_duplex( )
  - port_link_status( )
  - port_speed( )
  - port_status( )
  - port_vlan_description( )
  - port_vlan_name( )
  - port_vlan_number( )
  - pxe_lease_time( )
  - reserved_interface( )
  - v_cluster( )
  - v_datacenter( )
  - v_host( )
  - v_name( )
  - v_netadapter( )
  - v_switch( )
  - v_type( )
  - use_for_ea_inheritance( )
- SAMPLE CODE
- AUTHOR
- SEE ALSO

- COPYRIGHT

NAME

Infoblox::DHCP::HostAddr - DHCP Host Address object.

DESCRIPTION

A Host Address is an object used to specify addresses in a DNS Host object.

CONSTRUCTOR

```
my $host_addr = Infoblox::DHCP::HostAddr -> new(
      ipv4addr                          => $ipv4addr,                  #Required
      bootfile                          => $filename,                  #Optional / Default is undefined
      bootserver                        => $ipv4addr | $fqdn,          #Optional / Default is undefined
      configure_for_dhcp                => "true" | "false",           #Optional / Default is "false"
      deny_bootp                        => "true" | "false",           #Optional / Default is "false"
      dhcp_client_identifier            => $string,                    #Optional / Default is empty
      enable_pxe_lease_time             => "true" | "false",           #Optional / Default is "false"
      ignore_client_requested_options   => "true" | "false",           #Optional / Default is "false"
      mac                               => $mac_addr,                  #Optional / Default is empty
      match_client                      => $string,                    #Optional / Default is "MAC"
                                                                       #Possible values are:
                                                                       #
"MAC","CLIENT_IDENTIFIER","RESERVED",
                                                                       # "CIRCUIT_ID", "REMOTE_ID"
      nextserver                        => $ipv4addr | $fqdn | undef,  #Optional / Default is undefined
      options                           => [$option1, $option2, ...]   #Optional / Default is empty
      override_bootfile                 => "true" | "false",           #Optional / Default is "false"
      override_bootserver               => "true" | "false",           #Optional / Default is "false"
      override_deny_bootp               => "true" | "false",           #Optional / Default is "false"
      override_ignore_client_requested_options => "true" | "false",    #Optional / Default is "false"
      override_nextserver               => "true" | "false",           #Optional / Default is "false"
      override_options                  => "true" | "false",           #Optional / Default is "false"
      override_pxe_lease_time           => "true" | "false",           #Optional / Default is "false"
      pxe_lease_time                    => $time,                      #Optional / Default is empty
      use_for_ea_inheritance            => "true" | "false",           #Optional / Default is "false"
      reserved_interface                => $interface,                 #Optional
      logic_filters                     => [ $filter_name | $filter_obj, ...], # Optional / Default is
undefined
      override_logic_filters            => "true' | 'false",                  # Optional / Default value is
"false"
 );
```

SESSION METHODS

This section describes all the methods in Infoblox::Session module that you can apply to a DHCP Host Address object.

Infoblox::DNS::Host->ipv4addrs( )

Use this function to specify the addresses associated with the Host record. See Infoblox::DNS::Host->ipv4addrs() for parameters and return values.

Example

```
      #Construct a Host Address object
      my $h1 = Infoblox::DHCP::HostAddr->new(
                  ipv4addr  => "10.0.0.1"
      );

      # Configure an address in the Host record object
      $host->ipv4addrs([$h1]);
```

Infoblox::Session->get( )

Use this method to retrieve all the matching objects from the Infoblox appliance. See Infoblox::Session->get() for parameters and return values.

'discover_now_status' will not be returned by default, use 'return_methods' to request it if needed.

Key References

```
    Apply the following attributes to get a specific DHCP Host Address object:

     ipv4addr                - Optional. IPV4 address of the host address.
     mac                     - Optional. MAC address of the host address.
     host                    - Optional, the name of the host to which this address belongs.

     discovered_mac                          - Optional. Discovered MAC address.
     discovered_name                         - Optional. Discovered name.
     discoverer                              - Optional. Discoverer name.
     first_discovered                        - Optional. Time of the first discovery.
     last_discovered                         - Optional. Time of the last discovery.
     netbios                                 - Optional. Discovered NetBIOS name.
     network_component_description           - Optional. Discovered network component description.
     network_component_ip                    - Optional. Discovered network component IP address.
     network_component_name                  - Optional. Discovered network component name.
     network_component_port_description      - Optional. Discovered network component port description.
     network_component_port_name             - Optional. Discovered network component port name.
     network_component_port_number           - Optional. Discovered network component port number.
     network_component_type                  - Optional. Discovered network component type.
     os                                      - Optional. Discovered OS.
     port_duplex                             - Optional. Discovered port duplex settings.
     port_link_status                        - Optional. Discovered port link status.
     port_speed                              - Optional. Discovered port speed settings.
     port_status                             - Optional. Discovered port status settings.
     port_vlan_description                   - Optional. Discovered port VLAN description.
     port_vlan_name                          - Optional. Discovered port VLAN name.
     port_vlan_number                        - Optional. Discovered port VLAN number.
     v_cluster                               - Optional. Discovered VMware cluster name.
     v_datacenter                            - Optional. Discovered VMware datacenter name.
     v_host                                  - Optional. Discovered VMware server name.
     v_name                                  - Optional. Discovered VMware host or virtual machine name.
     v_netadapter                            - Optional. Discovered VMware physical network adapter name.
     v_switch                                - Optional. Discovered VMware virtual switch name.
     v_type                                  - Optional. Discovered VMware entity type (possible values:
"VirtualMachine", "HostSystem", "VirtualCenter").
```

Examples

```
#Get DHCP Host Address object through the session
my @retrieved_objs = $session->get(
    object => "Infoblox::DHCP::HostAddr",
    mac    => "11:11:11:11:11.11",
);

# get HostAddr by v_type:
my @retrieved_objs = $session->get(
   object    => 'Infoblox::DHCP::HostAddr',
   v_type    => 'VirtualMachine');
```

Infoblox::Session->modify( )

Use this method to modify a DHCP Host Address object in the Infoblox appliance. See Infoblox::Session->modify() for parameters and return values.

Example

```
#Use method to modify the pxe_lease_time
$host_address->pxe_lease_time("1234");
#Submit modification
my $response = $session->modify( $host_address );
```

Infoblox::Session->search( )

Use this method to search for DHCP Host Address object. See Infoblox::Session->search() for parameters and return values.

'discover_now_status' will not be returned by default, use 'return_methods' to request it if needed.

Key References

```
    Apply the following attributes to search for a specific DHCP Host Address object:

     ipv4addr                - Optional. IPV4 address of the host address (regular expression).
     mac                     - Optional. MAC address of the host address (regular expression).
     host                    - Optional, the name of the host to which this address belongs (regular
expression).

     discovered_mac                          - Optional. Discovered MAC address.
     discovered_name                         - Optional. Discovered name (regular expression).
     discoverer                              - Optional. Discoverer name (regular expression).
     first_discovered                        - Optional. Time of the first discovery.
     last_discovered                         - Optional. Time of the last discovery.
     netbios                                 - Optional. Discovered NetBIOS name (regular expression).
     network_component_description           - Optional. Discovered network component description (regular
expression).
     network_component_ip                    - Optional. Discovered network component IP address (regular
expression).
     network_component_name                  - Optional. Discovered network component name (regular
```

```
expression).
  network_component_port_description    - Optional. Discovered network component port description
(regular expression).
  network_component_port_name           - Optional. Discovered network component port name (regular
expression).
  network_component_port_number         - Optional. Discovered network component port number.
  network_component_type                - Optional. Discovered network component type (regular
expression).
  os                                    - Optional. Discovered OS (regular expression).
  port_duplex                           - Optional. Discovered port duplex settings.
  port_link_status                      - Optional. Discovered port link status.
  port_speed                            - Optional. Discovered port speed settings.
  port_status                           - Optional. Discovered port status settings.
  port_vlan_description                 - Optional. Discovered port VLAN description (regular
expression).
  port_vlan_name                        - Optional. Discovered port VLAN name (regular expression).
  port_vlan_number                      - Optional. Discovered port VLAN number.
  v_cluster                             - Optional. Discovered VMware cluster name (regular
expression).
  v_datacenter                          - Optional. Discovered VMware datacenter name (regular
expression).
  v_host                                - Optional. Discovered VMware server name (regular
expression).
  v_name                                - Optional. Discovered VMware host or virtual machine name
(regular expression).
  v_netadapter                          - Optional. Discovered VMware physical network adapter name
(regular expression).
  v_switch                              - Optional. Discovered VMware virtual switch name (regular
expression).
  v_type                                - Optional. Discovered VMware entity type (possible values:
"VirtualMachine", "HostSystem", "VirtualCenter").
```

Examples

```
# Search DHCP Host Address object through the session
my @retrieved_objs = $session->search(
    object       => "Infoblox::DHCP::HostAddr",
    mac          => "11:11:11:11:11.*",
);

# search HostAddr by v_datacenter:
my @retrieved_objs = $session->search(
   object                => 'Infoblox::DHCP::HostAddr',
   v_datacenter => 'test.*');
```

METHODS

This section describes all the methods that you can use to set and retrieve the attribute values of a DHCP Host Address object.

bootfile( )

Use this method to set or retrieve the bootfile value of a DHCP Host Address object.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Note that when bootfile is set to a defined value and override_bootfile is set to "false", the last operation takes precedence. Thus the sequence $object->bootfile("boot_file"); $object->override_bootfile("false"); will set override_bootfile to "false", and the sequence $object->override_bootfile("false"); $object->bootfile("boot_file"); will set override_bootfile to "true".

Parameter

The name of the file that the client must download. The default value is undefined.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get bootfile
my $bootfile = $host_addr->bootfile();
#Modify bootfile
$host_addr->bootfile("boot_file");
```

bootserver( )

Use this method to set or retrieve the bootserver of a DHCP Host Address object.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Note that when bootserver is set to a defined value and override_bootserver is set to "false", the last operation takes precedence. Thus

the sequence $object->bootserver("bootserver2.com"); $object->override_bootserver("false"); will set override_bootserver to "false", and the sequence $object->override_bootserver("false"); $object->bootserver("bootserver2.com"); will set override_bootserver to "true".

Parameter

The boot server IP address or FQDN (Fully Qualified Domain Name). The FQDN consists of the host name followed by the domain name (example: abc.com). A boot server name can have a maximum of 256 bytes. The default value is undefined.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get bootserver
my $bootserver = $host_addr->bootserver();
#Modify bootserver
$host_addr->bootserver("bootserver2.com");
#Un-override bootserver
$host_addr->bootserver(undef);
```

configure_for_dhcp( )

Use this method to set or retrieve the configure_for_dhcp flag of a DHCP Host Address object.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

Specify "true" to set the configure_for_dhcp flag or "false" to deactivate/unset it. The default value is "false".

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get configure_for_dhcp
my $configure_for_dhcp = $host_addr->configure_for_dhcp();
#Modify configure_for_dhcp
$host_addr->configure_for_dhcp("true");
```

deny_bootp( )

Use this method to set or retrieve the deny_bootp flag of a DHCP Host Address object.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Note that when deny_bootp is set to a defined value and override_deny_bootp is set to "false", the last operation takes precedence. Thus the sequence $object->deny_bootp("true"); $object->override_deny_bootp("false"); will set override_deny_bootp to "false", and the sequence $object->override_deny_bootp("false"); $object->deny_bootp("true"); will result in override_deny_bootp="true".

Parameter

Specify "true" to set the deny_bootp flag or "false" to deactivate/unset it. The default value is undefined, which indicates that this attribute inherits the upper-level setting.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get deny_bootp
my $deny_bootp = $host_addr->deny_bootp();
#Modify deny_bootp
$host_addr->deny_bootp("true");
```

discovered_data( )

Use this method to retrieve the discovered data of a DHCP Host Address object. This is a read-only attribute.

Omit the parameter to retrieve the attribute value.

Parameter

None

Returns

The valid return value is an Infoblox::Grid::Discovery::Data object.

Example

```
#Get discovered_data value
my $discovered_data = $object->discovered_data();
```

discovered_mac( )

Use this method to retrieve the discovered MAC address of a DHCP Host Address object.

Parameter

none

Returns

The method returns the discovered MAC address.

Example

```
#Get discovered_mac
my $discovered_mac = $host_addr->discovered_mac();
```

discovered_name( )

Use this method to retrieve the discovered name of a DHCP Host Address object.

Parameter

none

Returns

The method returns the discovered name.

Example

```
#Get discovered_name
my $discovered_name = $host_addr->discovered_name();
```

discoverer( )

Use this method to retrieve the discoverer of a DHCP Host Address object.

Parameter

none

Returns

The method returns the discoverer value.

Example

```
#Get discoverer
my $discoverer = $host_addr->discoverer();
```

discover_now_status( )

Use this method to retrieve the current discovery status. This attribute is read-only.

'discover_now_status' will not be returned by default, use 'return_methods' to request it if needed.

Parameter

None.

Returns

The method returns the attribute value that can be 'COMPLETE', 'FAILED', 'NONE', 'PENDING' or 'RUNNING'.

Example

```
#Get discover_now_status
my $discover_now_status = $host_addr->discover_now_status();
```

enable_pxe_lease_time( )

Use this method to set or retrieve the enable_pxe_lease_time flag of a DHCP Host Address object.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

Specify "true" to set the enable_pxe_lease_time flag or "false" to deactivate/unset it. The default value is "false".

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get enable_pxe_lease_time
my $enable_pxe_lease_time = $host_addr->enable_pxe_lease_time();
#Modify enable_pxe_lease_time
$host_addr->enable_pxe_lease_time("true");
```

first_discovered( )

Use this method to retrieve the date and time that the DHCP Host Address object was first discovered.

Parameter

none

Returns

The method returns the date and time that the DHCP Host Address object was first discovered.

Example

```
#Get first_discovered
my $first_discovered = $host_addr->first_discovered();
```

host( )

Use this method to retrieve the name of the Host record this address belongs to.

Parameter

none

Returns

The method returns the host value.

Example

```
#Get host
my $host = $host_addr->host();
```

ignore_client_requested_options( )

Use this method to set or retrieve the ignore_client_requested_option flag of a DHCP Host address object. If this flag is set to false, all the defined DHCP options are returned to the client.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Note that when ignore_client_requested_options is set to a defined value and override_ignore_client_requested_options is set to "false", the last operation takes precedence. Thus the sequence $object->ignore_client_requested_options("true"); $object->override_ignore_client_requested_options("false"); will set override_ignore_client_requested_options to "false", and the sequence $object->override_ignore_client_requested_options("false"); $object->ignore_client_requested_options("true"); will set override_ignore_client_requested_options to "true".

Parameter

Specify "true" to set the ignore_client_requested_option flag or "false" to deactivate/unset it. The default value is false, which

indicates that this attribute inherits the member-level setting.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
# Get ignore_client_requested_option
my $ignore_client_requested_option = $host_addr->ignore_client_requested_options();
# Un-override ignore_client_requested_option
$host_addr->ignore_client_requested_options(undef);
# Modify ignore_client_requested_option
$host_addr->ignore_client_requested_options("true");
```

ipv4addr( )

Use this method to set or retrieve the ipv4addr value of a DHCP Host Address object.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

An IPv4 address is a 32-bit number in dotted decimal notation. It consists of four 8-bit groups of decimal digits separated by decimal points (example: 192.168.1.2).

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get ipv4addr
my $ipv4addr = $host_addr->ipv4addr();
#Modify ipv4addr
$host_addr->ipv4addr("10.0.0.5");
```

is_invalid_mac( )

Use this method to retrieve the flag used to indicate whether an invalid MAC address is used. This is a read-only attribute.

Parameter

None

Returns

The method returns the attribute value.

Example

```
#Get is_invalid_mac value
my $is_invalid_mac = $host_addr->is_invalid_mac();
```

last_discovered( )

Use this method to retrieve the time this object was last seen by a discovery job.

The default value for this field is empty.

Parameter

N/A

Returns

The method returns the attribute value.

Example

```
#Get last_discovered
my $last_discovered = $host_addr->last_discovered();
```

last_queried( )

Use this method to retrieve the time when the associated record was last queried. This is a read-only attribute.

Parameter

None

Returns

The method returns the attribute value. The number of seconds that have elapsed since January 1st, 1970 UTC.

Example

```
#Get last_queried
my $last_queried = $host_addr->last_queried();
```

logic_filters( )

Use this method to set or retrieve all the logic (ordered) filters associated with the DHCP Host Address.

Filters are used to screen address assignments for the DHCP Host Address if the addresses are to be assigned to specific hosts.

Setting this method to a defined value implicitly sets the override_logic_filters method to "true". Setting the parameter to undefined causes the appliance to use the grid default and automatically resets the override_logic_filters attribute to "false".

Note that when logic_filters is set to a defined value and override_dlogic_filters is set to "false", the last operation takes precedence. Thus the sequence $object->logic_filters($filters); $object->override_logic_filters("false"); will set override_logic_filters to "false", and the sequence $object->override_logic_filters("false"); $object->logic_filters($filters); will result in override_logic_filters="true".

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

The valid value is an array reference that contains either the Infoblox::DHCP::Filter::MAC, Infoblox::DHCP::Filter::NAC, Infoblox::DHCP::Filter::Option objects or the names of the corresponding filters. The filters in the list must satisfy the following conditions:

Filter name must be globally unique.
Filters without expressions must be at the end of the list.
The filters should not be disabled.

The default value of the parameter is undefined.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get logic_filters
my $ref_logic_filters = $host_addr->logic_filters();
#Modify logic_filters
$host_addr->logic_filters(["filter_1","filter_2",$exl1]); #$exl1 is an Infoblox::DHCP::Filter::MAC,
Infoblox::DHCP::Filter::NAC or Infoblox::DHCP::Filter::Option object.
```

mac( )

Use this method to set or retrieve the MAC address value of a DHCP Host Address object.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

MAC Address of the DHCP Host address object. The format for MAC addresses and vendor prefixes are hexadecimal strings separated by colons or dashes. The following formats are acceptable: 11:11:11:11:11:11 and 11-11-11-11-11-11.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get mac
my $mac = $host_addr->mac();
#Modify mac
$host_addr->mac("11::11::11::11::11::12");
```

match_client( )

Use this method to set or retrieve the match_client value of a DHCP Host Address object.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

```
    match_client "MAC": The fixed IP address is leased to the matching MAC address.

    match_client "CLIENT_IDENTIFIER": The fixed IP address is leased to the matching DHCP client
identifier

    match_client "RESERVED": The fixed IP address is reserved for later use with a MAC address that
only has zeros.

    match_client "CIRCUIT_ID": The fixed IP address is leased to the DHCP client with a matching
circuit ID. Note that
                  the "agent_circuit_id" field must be set in this case.

    match_client "REMOTE_ID": The fixed IP address is leased to the DHCP client with a matching
remote ID. Note that
                  the "agent_remote_id" field must be set in this case.
```

Returns

```
    If you specified a parameter, the method returns true when the modification succeeds, and returns
false when the operation fails.

    If you did not specify a parameter, the method returns the attribute value.
```

Example

```
#Get match_client
my $match_client = $host_addr->match_client();
#Modify match_client
$host_addr->match_client("CLIENT_IDENTIFIER");
```

ms_ad_user_data( )

Use this method to retrieve Microsoft Active Directory users related information. This is a read-only attribute.

Parameter

None

Returns

The valid return value is an Infoblox::Grid::MSServer::AdUser::Data object.

Example

```
#Get ms_ad_user_data value
my $value = $object->ms_ad_user_data();
```

netbios( )

Use this method to retrieve the name in the NetBIOS reply that responded to a NetBIOS query.

Returns

The method returns the attribute value.

Example

```
#Get netbios
my $netbios = $host_addr->netbios();
```

network( )

Use this method to retrieve the network value of a DHCP Host Address object. This is a read-only attribute.

Omit the parameter to retrieve the attribute value.

Parameter

None

Returns

The method returns the attribute value.

Example

```
#Get network
my $network = $host_addr->network();
```

network_component_description( )

Use this method to retrieve the description of the network device that is connected to the DHCP Host Address object.

Parameter

none

Returns

The method returns the network device description.

Example

```
#Get network_component_description
my $network_component_description = $host_addr->network_component_description();
```

network_component_ip( )

Use this method to retrieve the IP address of the network device that is connected to the DHCP Host Address object.

Parameter

none

Returns

The method returns the IP address of the network device.

Example

```
#Get network_component_ip
my $network_component_ip = $host_addr->network_component_ip();
```

network_component_name( )

Use this method to retrieve the name of the network device that is connected to the DHCP Host Address object.

Parameter

none

Returns

The method returns the network device name.

Example

```
#Get network_component_name
my $network_component_name = $host_addr->network_component_name();
```

network_component_port_description( )

Use this method to retrieve the description of the network device port that is connected to the DHCP Host Address object.

Parameter

none

Returns

The method returns the description of the port on the network device.

Example

```
#Get network_component_port_description
my $network_component_port_description = $host_addr->network_component_port_description();
```

network_component_port_name( )

Use this method to retrieve the name of the network device port that is connected to the DHCP Host Address object.

Parameter

none

Returns

The method returns the name of the port on the network device.

Example

```
#Get network_component_port_name
my $network_component_port_name = $host_addr->network_component_port_name();
```

network_component_port_number( )

Use this method to retrieve the number of the network device port that is connected to the DHCP Host Address object.

Parameter

none

Returns

The method returns the network device port number.

Example

```
#Get network_component_port_number
my $network_component_port_number = $host_addr->network_component_port_number();
```

network_component_type( )

Use this method to retrieve the type of the network device that is connected to DHCP Host Address object.

Parameter

none

Returns

The method returns the network device type.

Example

```
#Get network_component_type
my $network_component_type = $host_addr->network_component_type();
```

nextserver( )

Use this method to set or retrieve the nextserver value of a DHCP Host Address object.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Note that when nextserver is set to a defined value and override_nextserver is set to "false", the last operation takes precedence. Thus the sequence $object->nextserver("3.3.3.4"); $object->override_nextserver("false"); will set override_nextserver to "false", and the sequence $object->override_nextserver("false"); $object->nextserver("3.3.3.4"); will set override_nextserver to "true".

Parameter

The next server IP address or FQDN (Fully Qualified Domain Name). The FQDN consists of the host name followed by the domain name (example: abc.com). A next server name can have a maximum of 256 bytes. The default value is undefined.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get nextserver
my $nextserver = $host_addr->nextserver();
#Modify nextserver
$host_addr->nextserver("3.3.3.4");
#Un-override nextserver
$host_addr->nextserver(undef);
```

options( )

Use this method to set or retrieve the options of a DHCP Host Address object.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

See Infoblox::DHCP::Option for parameters and methods available for this object.

Note that when options is set to a defined value and override_options is set to "false", the last operation takes precedence. Thus the sequence $object->options([$o1,$o2]); $object->override_options("false"); will set override_options to "false", and the sequence $object->override_options("false"); $object->options([$o1,$o2]); will set override_options to "true".

Parameter

Options describe network configuration settings and various services available on the network. These options occur as variable-length fields at the end of DHCP messages.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get options
my $options = $host_addr->options();
#Modify options
$host_addr->options([$option1, $option2]);
```

os( )

Use this method to retrieve the operating system associated with this host address.

Returns

The method returns the attribute value.

Example

```
#Get os
my $os = $host_addr->os();
```

override_bootfile( )

The override_bootfile attribute controls whether the bootfile value in the member is used, instead of the grid default.

The override_bootfile attribute can be specified explicitly. It is also set implicitly when bootfile is set to a defined value.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

Specify "true" to set the override_bootfile flag or "false" to deactivate/unset it. The default value is "false".

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get override_bootfile
my $override = $host_addr->override_bootfile();
#Modify the override
$host_addr->override_bootfile("true");
```

override_bootserver( )

The override_bootserver attribute controls whether the bootserver value in the member is used, instead of the grid default.

The override_bootserver attribute can be specified explicitly. It is also set implicitly when bootserver is set to a defined value.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

Specify "true" to set the override_bootserver flag or "false" to deactivate/unset it. The default value is "false".

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get override_bootserver
my $override = $host_addr->override_bootserver();
#Modify the override
$host_addr->override_bootserver("true");
```

override_deny_bootp( )

The override_deny_bootp attribute controls whether the deny_bootp value in the member is used, instead of the grid default.

The override_deny_bootp attribute can be specified explicitly. It is also set implicitly when deny_bootp is set to a defined value.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

Specify "true" to set the override_deny_bootp flag or "false" to deactivate/unset it. The default value is "false".

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get override_deny_bootp
my $override = $host_addr->override_deny_bootp();
#Modify the override
$host_addr->override_deny_bootp("true");
```

override_ignore_client_requested_option( )

The override_ignore_client_requested_option attribute controls whether the ignore_client_requested_option value in the member is used, instead of the grid default.

The override_ignore_client_requested_option attribute can be specified explicitly. It is also set implicitly when ignore_client_requested_option is set to a defined value.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

Specify "true" to set the override_ignore_client_requested_option flag or "false" to deactivate/unset it. The default value is "false".

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get override_ignore_client_requested_option
my $override = $host_addr->override_ignore_client_requested_option();
#Modify the override
$host_addr->override_ignore_client_requested_option("true");
```

override_logic_filters( )

The override_logic_filters attribute controls whether the logic_filters value of the DHCP Host Address is used, instead of the Grid default.

The override_logic_filters attribute can be specified explicitly. It is also set implicitly when logic_filters is set to a defined value.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

Set the parameter to "true" to override the Grid-level setting for logic_filters. Set the parameter to "false" to inherit the Grid-level setting for logic_filters.

The default value of this parameter is "false".

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Getting override_logic_filters
my $override_logic_filters=$host_addr->override_logic_filters( );
#Modifying override_logic_filters
$host_addr->override_logic_filters("true");
```

override_nextserver( )

The override_nextserver attribute controls whether the nextserver value in the member is used, instead of the grid default.

The override_nextserver attribute can be specified explicitly. It is also set implicitly when nextserver is set to a defined value.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

Specify "true" to set the override_nextserver flag or "false" to deactivate/unset it. The default value is "false".

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get override_nextserver
my $override = $host_addr->override_nextserver();
#Modify the override
$host_addr->override_nextserver("true");
```

override_options( )

The override_options attribute controls whether the options value in the member is used, instead of the upper-level default.

The override_options attribute can be specified explicitly. It is also set implicitly when options is set to a defined value.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

Specify "true" to set the override_options flag or "false" to deactivate/unset it. The default value is "false".

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get override_options
my $override = $host_addr->override_options();
#Modify the override
$host_addr->override_options("true");
```

override_pxe_lease_time( )

The override_pxe_lease_time attribute controls whether the pxe_lease_time value in the member is used, instead of the grid default.

The override_pxe_lease_time attribute can be specified explicitly. It is also set implicitly when pxe_lease_time is set to a defined value.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

Specify "true" to set the override_pxe_lease_time flag or "false" to deactivate/unset it. The default value is "false".

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get override_pxe_lease_time
my $override = $host_addr->override_pxe_lease_time();
#Modify the override
$host_addr->override_pxe_lease_time("true");
```

port_duplex( )

Use this method to retrieve the duplex setting of the network device port that is connected to the DHCP Host Address object.

Parameter

none

Returns

The method returns the port duplex setting. The returned value, if any, is one of the following:

```
Full
Half
```

Example

```
#Get port_duplex
my $port_duplex = $host_addr->port_duplex();
```

port_link_status( )

Use this method to retrieve the link status of the network device port that is connected to the DHCP Host Address object.

Parameter

none

Returns

The method returns the network device port link status. The returned value, if any, is one of the following:

```
Connected
Not Connected
Unknown
```

Example

```
#Get port_link_status
my $port_link_status = $host_addr->port_link_status();
```

port_speed( )

Use this method to retrieve the speed of the network device port that is connected to the DHCP Host Address object.

Parameter

none

Returns

The method returns the network device port speed. The returned value, if any, is one of the following:

```
10M
100M
1G
10G
100G
Unknown
```

Example

```
#Get port_speed
my $port_speed = $host_addr->port_speed();
```

port_status( )

Use this method to retrieve the status of the network device port that is connected to the DHCP Host Address object.

Parameter

none

Returns

The method returns the network device port status. The returned value, if any, is one of the following:

```
Up
Down
Unknown
```

Example

```
#Get port_status
my $port_status = $host_addr->port_status();
```

port_vlan_description( )

Use this method to retrieve the VLAN description of the network device port that is connected to the DHCP Host Address object.

Parameter

none

Returns

The method returns the VLAN description of the network device port.

Example

```
#Get port_vlan_description
my $port_vlan_description = $host_addr->port_vlan_description();
```

port_vlan_name( )

Use this method to retrieve the VLAN name of the network device port that is connected to the DHCP Host Address object.

Parameter

none

Returns

The method returns the network device port VLAN name.

Example

```
#Get port_vlan_name
my $port_vlan_name = $host_addr->port_vlan_name();
```

port_vlan_number( )

Use this method to retrieve the VLAN number of the network device port that is connected to the DHCP Host Address object.

Parameter

none

Returns

The method returns the port VLAN number.

Example

```
#Get port_vlan_number
my $port_vlan_number = $host_addr->port_vlan_number();
```

pxe_lease_time( )

Use this method to set or retrieve the pxe_lease_time value of a DHCP Host Address object.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Note that when pxe_lease_time is set to a defined value and override_pxe_lease_time is set to "false", the last operation takes precedence. Thus the sequence $object->pxe_lease_time("369"); $object->override_pxe_lease_time("false"); will set override_pxe_lease_time to "false", and the sequence $object->override_pxe_lease_time("false"); $object->pxe_lease_time("369"); will set override_pxe_lease_time to "true".

Parameter

Enter appropriate values in the Days, Hours, Mins and Secs fields. User can specify the duration of time it takes a host to connect to a boot server, such as a TFTP server, and download the file it needs to boot.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get pxe_lease_time
my $pxe_lease_time = $host_addr->pxe_lease_time();
#Modify pxe_lease_time
$host_addr->pxe_lease_time("369");
```

reserved_interface( )

Use this method to set or retrieve the reserved discovery interface.

Parameter

Valid value is an Infoblox::Grid::Discovery::DeviceInterface object.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get reserved_interface
my $reserved_interface = $host_addr->reserved_interface();
#Modify reserved_interface
$host_addr->reserved_interface($interface);
```

v_cluster( )

Use this method to retrieve the name of the VMware cluster associated with the DHCP Host Address object. This method is read-only.

Parameter

none

Returns

This method returns a string that contains the VMware cluster name. The default value is an empty string.

Example

```
#Get v_cluster
my $v_cluster = $host_addr->v_cluster();
```

v_datacenter( )

Use this method to retrieve the name of the VMware datacenter associated with the DHCP Host Address object. This method is read-only.

Parameter

none

Returns

This method returns a string that contains the VMware datacenter name. The default value is an empty string.

Example

```
#Get v_datacenter
my $v_datacenter = $host_addr->v_datacenter();
```

v_host( )

Use this method to retrieve the name of the VMware host associated with the DHCP Host Address object. This method is read-only.

Parameter

none

Returns

This method returns a string that contains the VMware host name. The default value is an empty string.

Example

```
#Get v_host
my $v_host = $host_addr->v_host();
```

v_name( )

Use this method to retrieve the name of the VMware entity associated with the DHCP Host Address object. This method is read-only.

Parameter

none

Returns

This method returns a string that contains the VMware entity name. The default value is an empty string.

Example

```
#Get v_name
my $v_name = $host_addr->v_name();
```

v_netadapter( )

Use this method to retrieve the name of the physical network adapter through which the VMware entity is connected to the appliance. This method is read-only.

Parameter

none

Returns

This method returns a string that contains the network adapter name. The default value is an empty string.

Example

```
#Get v_netadapter
my $v_netadapter = $host_addr->v_netadapter();
```

v_switch( )

Use this method to retrieve the name of the virtual switch through which the VMware entity is connected to the appliance. This method is read-only.

Parameter

none

Returns

This method returns a string that contains the virtual switch name. The default value is an empty string.

Example

```
#Get v_switch
my $v_switch = $host_addr->v_switch();
```

v_type( )

Use this method to retrieve the type of VMware entity associated with the DHCP Host Address object. This method is read-only.

Parameter

none

Returns

This method returns a string that contains the VMware entity type. The default value is an empty string. The returned value, if any, can be one of the following:

```
VirtualMachine
```

```
HostSystem
VirtualCenter
```

Example

```
#Get v_type
my $v_type = $host_addr->v_type();
```

use_for_ea_inheritance( )

Use this method to set or retrieve the flag that indicates if this host address is used for inheritable extensible attributes.

Parameter

Specify "true" to enable the flag or "false" to disable it. The default value is "false".

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get use_for_ea_inheritance
my $use_for_ea_inheritance = $host_addr->use_for_ea_inheritance();
#Modify use_for_ea_inheritance
$host_addr->use_for_ea_inheritance("true");
```

SAMPLE CODE

The following sample code demonstrates the different functions that can be applied to an object such as search and modify. This sample also includes error handling for the operations.

#Preparation prior to a DHCP IPv4 Host Address object insertion

```
#PROGRAM STARTS: Include all the modules that will be used, note the hostaddress option
use strict;
use Infoblox qw (:hostaddress);

#Create a session to the Infoblox appliance

my $session = Infoblox::Session->new(
              master   => "192.168.1.2", #appliance host ip
              username => "admin",       #appliance user login
              password => "infoblox"     #appliance password
);

unless ($session) {
       die("Construct session failed: ",
              Infoblox::status_code() . ":" . Infoblox::status_detail());
}
print "Session created successfully\n";

#Create an IPv4 network object
my $network = Infoblox::DHCP::Network->new(
                                          "network" => "10.0.0.0/8",
                                         );
unless($network) {
     die("Construct IPv4 Network object failed: ",
           Infoblox::status_code() . ":" . Infoblox::status_detail());
}
print "IPv4 Network object created successfully\n";

$session->add($network)
    or die("Add network failed: ",
           $session->status_code() . ":" . $session->status_detail());
print "Network added successfully\n";

#Create a zone object
my $zone = Infoblox::DNS::Zone->new(
                                    name => "test.com",
                                   );
unless($zone) {
     die("Construct Zone object failed: ",
           Infoblox::status_code() . ":" . Infoblox::status_detail());
}
```

```
print "Zone object created successfully\n";

$session->add($zone)
    or die("Add zone failed: ",
           $session->status_code() . ":" . $session->status_detail());
print "Zone added successfully\n";

my $option1 = Infoblox::DHCP::Option->new(
                                          name  => "routers",
                                          value => "10.0.0.100,10.0.0.200"
                                         );

unless($option1) {
    die("Construct option1 object failed: ",
        Infoblox::status_code() . ":" . Infoblox::status_detail());
}
print "option1 object created successfully\n";

my $option2 = Infoblox::DHCP::Option->new(
                                          name  => "nameservers",
                                          value => "1.1.1.1,2.2.2.2,3.3.3.3"
                                         );
unless($option2) {
    die("Construct option2 object failed: ",
        Infoblox::status_code() . ":" . Infoblox::status_detail());
}
print "option2 object created successfully\n";
```

#Create a DHCP IPv4 Host address object

```
#Create a DHCP IPv4 Host Address object
my $host_addr = Infoblox::DHCP::HostAddr->new(
       "ipv4addr"                => "10.0.0.5",
       "mac"                     => "00:00:00:11:11:11",
       "pxe_lease_time"          => 1234,
       "override_pxe_lease_time" => "true",
       "options"                 => [ $option1 , $option2 ],
       );
unless($host_addr) {
     die("Construct IPv4 host address failed: ",
           Infoblox::status_code() . ":" . Infoblox::status_detail());
}
print "DHCP IPv4 Host address object created successfully\n";

my $host = Infoblox::DNS::Host->new(
                                    name         => "host.test.com",
                                    ipv4addrs    => [$host_addr],
                                   );

unless($host) {
     die("Construct IPv4 host record failed: ",
           Infoblox::status_code() . ":" . Infoblox::status_detail());
}

$session->add($host)
    or die("Add host failed: ",
           $session->status_code() . ":" . $session->status_detail());
print "Host added successfully\n";
```

#Get and modify a DHCP IPv4 Host Address object

```
#Get DHCP IPv4 Host Address object through the session
my @retrieved_objs = $session->get(
    object => "Infoblox::DHCP::HostAddr",
    mac    => "00:00:00:11:11:11",
);
my $object = $retrieved_objs[0];
unless ($object) {
    die("Get IPv4 Host Address object failed: ",
        $session->status_code() . ":" . $session->status_detail());
}
print "Get IPv4 Host Address object found at least 1 matching entry\n";

#Modify one of the attributes of the obtained IPv4 Host Address object
$object->pxe_lease_time(1235);
```

```
#Apply the changes
$session->modify($object)
    or die("Modify IPv4 Host Address object failed: ",
           $session->status_code() . ":" . $session->status_detail());
print "IPv4 Host Address object modified successfully \n";

#Verify that this was changed also when retrieving this via the host record
my @retrieved_objs = $session->get(
    object => "Infoblox::DNS::Host",
    name => "host.test.com",
);
my $object = $retrieved_objs[0];
unless ($object) {
    die("Get Host record failed: ",
        $session->status_code() . ":" . $session->status_detail());
}
print "Get host record object found at least 1 matching entry\n";

my $hostaddress = @{$object->ipv4addrs()}[0];

unless ($hostaddress->pxe_lease_time() == 1235) {
    die("The modification of host address has failed: ",
        $session->status_code() . ":" . $session->status_detail());
}
print "The change to preferred lifetime has succeeded correctly\n";

#The host address retrieved this way can also be modified without submitting the full host record
$hostaddress->pxe_lease_time(1111);

#Apply the changes
$session->modify($hostaddress)
    or die("Modify IPv4 Host Address object failed: ",
           $session->status_code() . ":" . $session->status_detail());
print "IPv4 Host Address object modified successfully \n";

#Verify the change was submitted successfully
@retrieved_objs = $session->get(
    object => "Infoblox::DHCP::HostAddr",
    mac    => "00:00:00:11:11:11",
);
$object = $retrieved_objs[0];
unless ($object) {
    die("Get IPv4 Host Address object failed: ",
        $session->status_code() . ":" . $session->status_detail());
}
print "Get IPv4 Host Address object found at least 1 matching entry\n";

unless ($hostaddress->pxe_lease_time() == 1111) {
    die("The modification of host address has failed: ",
        $session->status_code() . ":" . $session->status_detail());
}

#Submit the Host object for removal
$session->remove($host)
    or die("Remove Host object failed: ",
        $session->status_code() . ":" . $session->status_detail());
print "Host object removed successfully \n";

#Submit the Zone object for removal
$session->remove($zone)
    or die("Remove Zone object failed: ",
        $session->status_code() . ":" . $session->status_detail());
print "Zone object removed successfully \n";

#Submit the Network object for removal
$session->remove($network)
    or die("Remove Network object failed: ",
        $session->status_code() . ":" . $session->status_detail());
print "Network object removed successfully \n";
```

AUTHOR

Infoblox Inc. http://www.infoblox.com/

SEE ALSO

Infoblox::DHCP::Network, Infoblox::DHCP::View, Infoblox::DHCP::Range, Infoblox::DHCP::Lease, Infoblox::Session->add(), Infoblox::Session->get(), Infoblox::Session->modify(), Infoblox::Session->remove(), Infoblox::Session->search(), Infoblox::Session, Infoblox::Grid::MSServer::AdUser::Data, Infoblox::Grid::Discovery::Data

---

COPYRIGHT

Copyright (c) 2017 Infoblox Inc.

Infoblox::DHCP::IPv6HostAddr - DHCP IPv6 Host Address object.

- NAME
- DESCRIPTION
- CONSTRUCTOR
- SESSION METHODS
  - Infoblox::DNS::Host->ipv6addrs( )
  - Infoblox::Session->get( )
  - Infoblox::Session->modify( )
  - Infoblox::Session->search( )
- METHODS
  - address_type( )
  - configure_for_dhcp( )
  - discovered_data( )
  - discovered_duid( )
  - discovered_name( )
  - discoverer( )
  - discover_now_status( )
  - domain_name( )
  - domain_name_servers( )
  - duid( )
  - first_discovered( )
  - ipv6addr( )
  - ipv6prefix( )
  - ipv6prefix_bits( )
  - last_discovered( )
  - last_queried( )
  - match_client( )
  - ms_ad_user_data( )
  - name( )
  - netbios( )
  - network( )
  - network_component_description( )
  - network_component_ip( )
  - network_component_name( )
  - network_component_port_description( )
  - network_component_port_name( )
  - network_component_port_number( )
  - network_component_type( )
  - options( )
  - os( )
  - override_domain_name( )
  - override_domain_name_servers( )
  - override_options( )
  - override_preferred_lifetime( )
  - override_valid_lifetime( )
  - port_duplex( )
  - port_link_status( )
  - port_speed( )
  - port_status( )
  - port_vlan_description( )
  - port_vlan_name( )
  - port_vlan_number ( )
  - preferred_lifetime( )
  - reserved_interface( )
  - valid_lifetime( )
  - use_for_ea_inheritance( )
- SAMPLE CODE
- AUTHOR
- SEE ALSO
- COPYRIGHT

NAME

Infoblox::DHCP::IPv6HostAddr - DHCP IPv6 Host Address object.

DESCRIPTION

An IPv6 Host Address is an object used to specify IPv6 addresses in a DNS Host object.

CONSTRUCTOR

```
 my $host_addr = Infoblox::DHCP::IPv6HostAddr -> new(
       duid                          => $string,                       # Required
       address_type                  => "ADDRESS" | "PREFIX" | "BOTH", # Optional / Default is "ADDRESS"
       configure_for_dhcp            => "true" | "false",              # Optional / Default is "false"
                                                                       # When both the IP address and DUID
are present, the default value is "true"
                                                                       # Applicable only when adding this
host address to a host
       domain_name                   => $string,                       # Optional / Default is undefined
       domain_name_servers           => "$ipv6addr1,$ipv6addr2,...",   # Optional / Default is undefined
       ipv6addr                      => $ipv6addr,                     # Optional / one of ipv6addr or
ipv6prefix is required
       ipv6prefix                    => $ipv6addr,                     # Optional / one of ipv6addr or
ipv6prefix is required
       ipv6prefix_bits               => $ipv6addr,                     # Optional / Required if ipv6prefix
is specified
       match_client                  => $string,                       # Optional / Default is "DUID"
                                                                       # Possible values are:
                                                                       #  "DUID", "RESERVED",
       options                       => [$option1, $option2, ...]      # Optional / Default is empty
       override_domain_name          => "true" | "false",              # Optional / Default value is
"false"
       override_domain_name_servers  => "true" | "false",              # Optional / Default value is
"false"
       override_options              => "true" | "false",              # Optional / Default value is
"false"
       override_preferred_lifetime   => "true" | "false",              # Optional / Default value is
"false"
       override_valid_lifetime       => "true" | "false",              # Optional / Default value is
"false"
       preferred_lifetime            => $num,                          # Optional / Default is "27000"
       valid_lifetime                => $num,                          # Optional / Default is 43200
       use_for_ea_inheritance        => "true" | "false",              # Optional / Default value is
"false"
       reserved_interface            => $interface,                    # Optional
 );
```

SESSION METHODS

This section describes all the methods in Infoblox::Session module that you can apply to a DHCP IPv6 Host Address object.

Infoblox::DNS::Host->ipv6addrs( )

Use this function to specify the addresses associated with the Host record. See Infoblox::DNS::Host->ipv6addrs() for parameters and return values.

Example

```
       #Construct a Host Address object
       my $h1 = Infoblox::DHCP::HostAddr->new(
                                          duid     => '0cc084d303010003',
                                          ipv6addr => '2001::4',
       );

       # Configure an address in the Host record object
       $host->ipv6addrs([$h1]);
```

Infoblox::Session->get( )

Use this method to retrieve all the matching objects from the Infoblox appliance. See Infoblox::Session->get() for parameters and return values.

'discover_now_status' will not be returned by default, use 'return_methods' to request it if needed.

Key References

```
       Apply the following attributes to get a specific DHCP IPv6 Host Address object:

        comment                              - Optional. User-specified comment for the host address.
        duid                                 - Optional. DUID of the host address.
        ipv6addr                             - Optional. IPv6 address of the host address.
        network                              - Optional. Network address of the host address.
        network_view                         - Optional. Network view in which the host address is
      located. By default, all network views are searched.
        extensible_attributes                - Optional. A hash reference containing extensible
      attributes.

        discovered_duid                      - Optional. Discovered DHCPv6 UID.
```

```
  discovered_name                       - Optional. Discovered name.
  discoverer                            - Optional. Discoverer name.
  first_discovered                      - Optional. Time of the first discovery.
  last_discovered                       - Optional. Time of the last discovery.
  netbios                               - Optional. Discovered NetBIOS name.
  network_component_description         - Optional. Discovered network component description.
  network_component_ip                  - Optional. Discovered network component IP address.
  network_component_name                - Optional. Discovered network component name.
  network_component_port_description    - Optional. Discovered network component port description.
  network_component_port_name           - Optional. Discovered network component port name.
  network_component_port_number         - Optional. Discovered network component port number.
  network_component_type                - Optional. Discovered network component type.
  os                                    - Optional. Discovered OS.
  port_duplex                           - Optional. Discovered port duplex settings.
  port_link_status                      - Optional. Discovered port link status.
  port_speed                            - Optional. Discovered port speed settings.
  port_status                           - Optional. Discovered port status settings.
  port_vlan_description                 - Optional. Discovered port VLAN description.
  port_vlan_name                        - Optional. Discovered port VLAN name.
  port_vlan_number                      - Optional. Discovered port VLAN number.
```

Examples

```
#Get DHCP IPv6 Host Address object through the session
my @retrieved_objs = $session->get(
    object => "Infoblox::DHCP::IPv6HostAddr",
    duid   => "0001",
);
my @retrieved_objs = $session->get(
    object       => "Infoblox::DHCP::IPv6HostAddr",
    ipv6addr     => "2001::1",
);
```

Infoblox::Session->modify( )

Use this method to modify a DHCP IPv6 Host Address object in the Infoblox appliance. See Infoblox::Session->modify() for parameters and return values.

Example

```
#Use method to modify the preferred_lifetime
$ipv6ha->preferred_lifetime("1234567");
#Submit modification
my $response = $session->modify( $ipv6ha );
```

Infoblox::Session->search( )

Use this method to search for DHCP IPv6 Host Address object and matching Host object with MAC address in the Infoblox appliance. See Infoblox::Session->search() for parameters and return values.

'discover_now_status' will not be returned by default, use 'return_methods' to request it if needed.

Key References

```
  Apply the following attributes to search for a specific DHCP IPv6 Host Address object:

  comment                               - Optional. User-specified comment for the host address.
  duid                                  - Optional. DUID of the host address.
  ipv6addr                              - Optional. IPv6 address of the host address.
  network                               - Optional. Network address of the host address.
  network_view                          - Optional. Network view in which the host address is
located. By default, all network views are searched.

  discovered_duid                       - Optional. Discovered DHCPv6 UID.
  discovered_name                       - Optional. Discovered name (regular expression).
  discoverer                            - Optional. Discoverer name (regular expression).
  first_discovered                      - Optional. Time of the first discovery.
  last_discovered                       - Optional. Time of the last discovery.
  netbios                               - Optional. Discovered NetBIOS name (regular expression).
  network_component_description         - Optional. Discovered network component description (regular
expression).
  network_component_ip                  - Optional. Discovered network component IP address (regular
expression).
  network_component_name                - Optional. Discovered network component name (regular
expression).
  network_component_port_description    - Optional. Discovered network component port description
(regular expression).
  network_component_port_name           - Optional. Discovered network component port name (regular
expression).
  network_component_port_number         - Optional. Discovered network component port number.
  network_component_type                - Optional. Discovered network component type (regular
expression).
  os                                    - Optional. Discovered OS  (regular expression).
  port_duplex                           - Optional. Discovered port duplex settings.
  port_link_status                      - Optional. Discovered port link status.
  port_speed                            - Optional. Discovered port speed settings
  port_status                           - Optional. Discovered port status settings.
```

```
  port_vlan_description                - Optional. Discovered port VLAN description (regular
expression).
  port_vlan_name                       - Optional. Discovered port VLAN name (regular expression).
  port_vlan_number                     - Optional. Discovered port VLAN numbe.
```

Examples

```
 # Search for all DHCP IPv6 Host Address objects that are specified in the network "2001::/24" in the
"default" network view.
 my @retrieved_objs = $session->search(
     object       => "Infoblox::DHCP::IPv6HostAddr",
     duid         => "0001",
 );
```

METHODS

This section describes all the methods that you can use to set and retrieve the attribute values of a DHCP IPv6 Host Address object.

address_type( )

Use this method to set or retrieve the address type of a DHCP IPv6 Host Address object.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

Valid values are "ADDRESS", "PREFIX", or "BOTH". When the address type is "ADDRESS", a value for the 'ipv6addr' member is required. When the address type is "PREFIX", values for 'ipv6prefix' and 'ipv6prefix_bits' are required. When the address type is "BOTH", values for 'ipv6addr', 'ipv6prefix', and 'ipv6prefix_bits' are all required.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get address_type
my $address_type = $ipv6ha->address_type();
#Modify address_type
$ipv6ha->address_type("PREFIX");
```

configure_for_dhcp( )

Use this method to set or retrieve the configure_for_dhcp flag of a DHCP IPv6 Host Address object.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

Specify "true" to set the configure_for_dhcp flag or "false" to deactivate/unset it. The default value is "false".

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get configure_for_dhcp
my $configure_for_dhcp = $ipv6ha->configure_for_dhcp();
#Modify configure_for_dhcp
$ipv6ha->configure_for_dhcp("true");
```

discovered_data( )

Use this method to retrieve the discovered data of a DHCP IPv6 Host Address object. This is a read-only attribute.

Omit the parameter to retrieve the attribute value.

Parameter

None

Returns

The valid return value is an Infoblox::Grid::Discovery::Data object.

Example

```
#Get discovered_data value
my $discovered_data = $object->discovered_data();
```

discovered_duid( )

Use this method to retrieve the discovered DHCPv6 UID (DUID) of a DHCP IPv6 Host Address object.

Parameter

none

Returns

The method returns the discovered name.

Example

```
#Get discovered_duid
my $discovered_duid = $ipv6ha->discovered_duid();
```

discovered_name( )

Use this method to retrieve the discovered name of a DHCP IPv6 Host Address object.

Parameter

none

Returns

The method returns the discovered name.

Example

```
#Get discovered_name
my $discovered_name = $ipv6ha->discovered_name();
```

discoverer( )

Use this method to retrieve the discoverer of a DHCP IPv6 Host Address object.

Parameter

none

Returns

The method returns the discoverer value.

Example

```
#Get discoverer
my $discoverer = $ipv6ha->discoverer();
```

discover_now_status( )

Use this method to retrieve the current discovery status. This attribute is read-only.

'discover_now_status' will not be returned by default, use 'return_methods' to request it if needed.

Parameter

None.

Returns

The method returns the attribute value that can be 'COMPLETE', 'FAILED', 'NONE', 'PENDING' or 'RUNNING'.

Example

```
#Get discover_now_status
my $discover_now_status = $ipv6ha->discover_now_status();
```

domain_name( )

Use this method to set or retrieve the domain_name value of a DHCP IPv6 host address object.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Setting this method to a defined value implicitly sets the override_domain_name method to "true". Setting the parameter to undefined causes the appliance to use the upper-level default and automatically resets the override_domain_name attribute to "false".

Note that when domain_name is set to a defined value and override_domain_name is set to "false", the last operation takes precedence. Thus the sequence $object->domain_name("testdomain"); $object->override_domain_name("false"); will set override_domain_name to "false", and the sequence $object->override_domain_name("false"); $object->domain_name("testdomain"); will set override_domain_name to "true".

Parameter

The client's domain name in string format.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get domain_name
my $domain_name = $host_addr->domain_name();
#Modify domain_name
$host_addr->domain_name("test_domain.com");
```

domain_name_servers( )

Use this method to set or retrieve the domain_name_servers value of a DHCP IPv6 host address object.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Setting this method to a defined value implicitly sets the override_domain_name_servers method to "true". Setting the parameter to undefined causes the appliance to use the upper-level default and automatically resets the override_domain_name_servers attribute to "false".

Note that when domain_name_servers is set to a defined value and override_domain_name_servers is set to "false", the last operation takes precedence. Thus the sequence $object->domain_name_servers("2001::1,2001::2"); $object->override_domain_name_servers("false"); will set override_domain_name_servers to "false", and the sequence $object->override_domain_name_servers("false"); $object->domain_name_servers("2001::1,2001::2"); will set in override_domain_name_servers to "true".

Parameter

The comma-separated list of domain name server addresses in IPv6 address format.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get domain_name_servers
my $domain_name_servers = $host_addr->domain_name_servers();
#Modify domain_name_servers
$host_addr->domain_name_servers("2001::1,2001::2");
```

duid( )

Use this method to set or retrieve the DHCPv6 Unique Identifier (DUID) of the IPv6 Host Address object.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

The DUID of the matching client in hexadecimal format.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get duid
my $duid = $ipv6ha->duid();
#Modify duid
$ipv6ha->duid("1c2edfaa");
```

first_discovered( )

Use this method to retrieve the date and time that the DHCP IPv6 Host Address object was first discovered.

Parameter

none

Returns

The method returns the date and time that the DHCP IPv6 Host Address object was first discovered.

Example

```
#Get first_discovered
my $first_discovered = $ipv6ha->first_discovered();
```

ipv6addr( )

Use this method to set or retrieve the the IPv6 address of the IPv6 Host Address object.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

An IPv6 address is a 128-bit number represented as eight groups of four hexadecimal digits (i.e. two octets), with each group separated by a `colon(:)` (example: 2001:0db8:85a3:0000:0000:8a2e:0370:7334). Leading zeroes and groups of consecutive zeros within the value can be omitted to produce a more compact representation (example: 2001::1, ::1), and dotted-quad notations can be used to express IPv4-mapped IPv6 addresses (example: ::ffff:192.0.2.128).

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get ipv6addr
my ipv6addr = $ipv6ha->ipv6addr();
#Modify ipv6addr
$ipv6ha->ipv6addr("aaaa:1111:bbbb:2222:cccc:3333:dddd:5555");
```

ipv6prefix( )

Use this method to set or retrieve the IPv6 prefix of a DHCP IPv6 Host Address object. The prefix must belong to the list of global prefixes defined in the Infoblox::Grid::DHCP object.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

IPv6 prefix from the list of global IPv6 prefixes.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails. ls.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get ipv6prefix
my $ipv6prefix = $ipv6ha->ipv6prefix();
#Modify ipv6prefix
$ipv6ha->ipv6prefix("2001::");
```

ipv6prefix_bits( )

Use this method to set or retrieve the prefix bits for the DHCP IPv6 Host Address.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

An unsigned integer between 2 and 128.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get ipv6prefix_bits
my $ipv6prefix_bits = $ipv6ha->ipv6prefix_bits();
#Modify ipv6prefix_bits
$ipv6ha->ipv6prefix_bits(32);
```

last_discovered( )

Use this method to retrieve the time this object was last seen by a discovery job.

The default value for this field is empty.

Parameter

N/A

Returns

The method returns the attribute value.

Example

```
#Get last_discovered
my $last_discovered = $ipv6ha->last_discovered();
```

last_queried( )

Use this method to retrieve the time when the associated record was last queried. This is a read-only attribute.

Parameter

None

Returns

The method returns the attribute value. The number of seconds that have elapsed since January 1st, 1970 UTC.

Example

```
#Get last_queried
my $last_queried = $ipv6ha->last_queried();
```

match_client( )

Use this method to set or retrieve the match_client value of a DHCP Host Address object.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

```
match_client "DUID": The fixed IP address is leased to the matching DUID.
```

Returns

```
If you specified a parameter, the method returns true when the modification succeeds, and returns
false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.
```

Example

```
#Get match_client
my $match_client = $ipv6ha->match_client();
#Modify match_client
$ipv6ha->match_client("DUID");
```

ms_ad_user_data( )

Use this method to retrieve Microsoft Active Directory users related information. This is a read-only attribute.

Parameter

None

Returns

The valid return value is an Infoblox::Grid::MSServer::AdUser::Data object.

Example

```
#Get ms_ad_user_data
my $ms_ad_user_data = $host_addr->ms_ad_user_data();
```

name( )

Use this method to set or retrieve the name of a DHCP IPv6 Host Address object. The same name can be used for multiple host addresses.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

Specify a name in string format.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get the name
my $name = $ipv6ha->name();
#Modify the name
$ipv6ha->name("pop-server");
```

netbios( )

Use this method to retrieve the name in the NetBIOS reply that responded to a NetBIOS query.

Returns

The method returns the attribute value.

Example

```
#Get NetBIOS
my $netbios = $ipv6ha->netbios();
```

network( )

Use this method to retrieve the network value of a DHCP IPv6 Host Address object. This is a read-only attribute.

Omit the parameter to retrieve the attribute value.

Parameter

None

Returns

The method returns the attribute value.

Example

```
#Get network value
my $value = $object->network();
```

network_component_description( )

Use this method to retrieve the description of the network device that is connected to the DHCP IPv6 Host Address object.

Parameter

none

Returns

The method returns the network device description.

Example

```
#Get network_component_description
my $network_component_description = $ipv6ha->network_component_description();
```

network_component_ip( )

Use this method to retrieve the IP address of the network device that is connected to the DHCP IPv6 Host Address object.

Parameter

none

Returns

The method returns the IP address of the network device.

Example

```
#Get network_component_ip
my $network_component_ip = $ipv6ha->network_component_ip();
```

network_component_name( )

Use this method to retrieve the name of the network device that is connected to the DHCP IPv6 Host Address object.

Parameter

none

Returns

The method returns the network device name.

Example

```
#Get network_component_name
my $network_component_name = $ipv6ha->network_component_name();
```

network_component_port_description( )

Use this method to retrieve the description of the network device port that is connected to the DHCP IPv6 Host Address object.

Parameter

none

Returns

The method returns the description of the port on the network device.

Example

```
#Get network_component_port_description
my $network_component_port_description = $ipv6ha->network_component_port_description();
```

network_component_port_name( )

Use this method to retrieve the name of the network device port that is connected to the DHCP IPv6 Host Address object.

Parameter

none

Returns

The method returns the name of the port on the network device.

Example

```
#Get network_component_port_name
my $network_component_port_name = $ipv6ha->network_component_port_name();
```

network_component_port_number( )

Use this method to retrieve the number of the network device port that is connected to the DHCP IPv6 Host Address object.

Parameter

none

Returns

The method returns the network device port number.

Example

```
#Get network_component_port_number
my $network_component_port_number = $ipv6ha->network_component_port_number();
```

network_component_type( )

Use this method to retrieve the type of the network device that is connected to the DHCP IPv6 Host Address object.

Parameter

none

Returns

The method returns the network device type.

Example

```
#Get network_component_type
my $network_component_type = $ipv6ha->network_component_type();
```

options( )

Use this method to set or retrieve the options of a DHCP IPv6 Host Address object.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Setting this method to a defined value implicitly sets the override_options method to "true". Setting the parameter to undefined causes the appliance to use the grid default and automatically resets the override_options attribute to "false".

Note that when the options method is set to a defined value and override_options is set to "false", the last operation takes precedence. Thus the sequence $object->options([$opt]); $object->override_options("false"); will set override_options to "false", and the sequence $object->override_options("false"); $object->options([$opt]); will set override_options to "true".

Parameter

The valid value is an array reference that contains Infoblox::DHCP::Option object(s).

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get options
my $options = $ipv6ha->options();
#Modify options
my $option1 = Infoblox::DHCP::Option->new(
    name   => "dhcp6.rapid-commit",
    value  => "true"
);
$ipv6ha->options([$option1);
```

os( )

Use this method to retrieve the operating system associated with this host address.

Returns

The method returns the attribute value.

Example

```
#Get OS
my $os = $ipv6ha->os();
```

override_domain_name( )

The override_domain_name attribute controls whether the domain_name value in the DHCP IPv6 host address is used, instead of the upper-level default.

The override_domain_name attribute can be specified explicitly. It is also set implicitly when domain_name is set to a defined value.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

Set the parameter to "true" to override the upper-level setting for domain_name. Set the parameter to "false" to inherit the upper-level setting for domain_name.

The default value of this parameter is "false".

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get override_domain_name
my $override_domain_name=$ipv6ha->override_domain_name( );
#Modify override_domain_name
$ipv6ha->override_domain_name("true");
```

override_domain_name_servers( )

The override_domain_name_servers attribute controls whether the domain_name_servers value in the DHCP IPv6 host address is used, instead of the upper-level default.

The override_domain_name_servers attribute can be specified explicitly. It is also set implicitly when domain_name_servers is set to a defined value.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

Set the parameter to "true" to override the upper-level setting for domain_name_servers. Set the parameter to "false" to inherit the upper-level setting for domain_name_servers.

The default value of this parameter is "false".

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get override_domain_name_servers
my $override_domain_name_servers=$ipv6ha->override_domain_name_servers( );
#Modify override_domain_name_servers
$ipv6ha->override_domain_name_servers("true");
```

override_options( )

The override_options attribute controls whether the options in the DHCP IPv6 host address are used, instead of the upper-level default.

The override_options attribute can be specified explicitly. It is also set implicitly when options are set to defined values.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

Set the parameter to "true" to override the upper-level setting for valid_lifetime. Set the parameter to "false" to inherit the upper-level setting for valid_lifetime.

The default value of this parameter is "false".

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get override_options
my $override_options=$host_addr->override_options( );
#Modify override_options
$host_addr->override_options("true");
```

override_preferred_lifetime( )

The override_preferred_lifetime attribute controls whether the preferred_lifetime value in the DHCP IPv6 host address is used, instead of the upper-level default.

The override_preferred_lifetime attribute can be specified explicitly. It is also set implicitly when preferred_lifetime is set to a defined value.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

Set the parameter to "true" to override the upper-level setting for valid_lifetime. Set the parameter to "false" to inherit the upper-level setting for valid_lifetime.

The default value of this parameter is "false".

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get override_preferred_lifetime
my $override_preferred_lifetime=$host_addr->override_preferred_lifetime( );
#Modify override_preferred_lifetime
$host_addr->override_preferred_lifetime("true");
```

override_valid_lifetime( )

The override_valid_lifetime attribute controls whether the valid_lifetime value in the DHCP IPv6 host address is used, instead of the upper-level default.

The override_valid_lifetime attribute can be specified explicitly. It is also set implicitly when valid_lifetime is set to a defined value.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

Set the parameter to "true" to override the upper-level setting for valid_lifetime. Set the parameter to "false" to inherit the upper-level setting for valid_lifetime.

The default value of this parameter is "false".

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get override_valid_lifetime
my $override_valid_lifetime=$host_addr->override_valid_lifetime( );
#Modify override_valid_lifetime
$host_addr->override_valid_lifetime("true");
```

port_duplex( )

Use this method to retrieve the duplex setting of the network device port that is connected to the DHCP IPv6 Host Address object.

Parameter

none

Returns

The method returns the port duplex setting. The returned value, if any, is one of the following:

```
Full
Half
```

Example

```
#Get port_duplex
my $port_duplex = $ipv6ha->port_duplex();
```

port_link_status( )

Use this method to retrieve the link status of the network device port that is connected to the DHCP IPv6 Host Address object.

Parameter

none

Returns

This method returns the link status of the network device port. The returned value, if any, is one of the following:

```
Connected
Not Connected
Unknown
```

Example

```
#Get port_link_status
my $port_link_status = $ipv6ha->port_link_status();
```

port_speed( )

Use this method to retrieve the speed of the network device port that is connected to the DHCP IPv6 Host Address object.

Parameter

none

Returns

The method returns the network device port speed. The returned value, if any, is one of the following:

```
10M
100M
1G
10G
100G
Unknown
```

Example

```
#Get port_speed
my $port_speed = $ipv6ha->port_speed();
```

port_status( )

Use this method to retrieve the status of the network device port that is connected to the DHCP IPv6 Host Address object.

Parameter

none

Returns

The method returns the network device port status. The returned value, if any, is one of the following:

```
Up
Down
Unknown
```

Example

```
#Get port_status
my $port_status = $ipv6ha->port_status();
```

port_vlan_description( )

Use this method to retrieve the VLAN description of the network device port that is connected to the DHCP IPv6 Host Address object.

Parameter

none

Returns

The method returns the VLAN description of the network device port.

Example

```
#Get port_vlan_description
my $port_vlan_description = $ipv6ha->port_vlan_description();
```

port_vlan_name( )

Use this method to retrieve the VLAN name of the network device port that is connected to the DHCP IPv6 Host Address object.

Parameter

none

Returns

The method returns the network device port VLAN name.

Example

```
#Get port_vlan_name
my $port_vlan_name = $ipv6ha->port_vlan_name();
```

port_vlan_number ( )

Use this method to retrieve the VLAN number of the network device port that is connected to the DHCP IPv6 Host Address object.

Parameter

none

Returns

The method returns the port VLAN number.

Example

```
#Get port_vlan_number
my $port_vlan_number  = $ipv6ha->port_vlan_number ();
```

preferred_lifetime( )

Use this method to set or retrieve the preferred_lifetime value of a DHCP IPv6 Host Address object.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Setting this method to a defined value implicitly sets the override_preferred_lifetime method to "true". Setting the parameter to undefined causes the appliance to use the upper-level default and automatically resets the override_preferred_lifetime attribute to "false".

Note that when preferred_lifetime is set to a defined value and override_preferred_lifetime is set to "false", the last operation takes precedence. Thus the sequence $object->preferred_lifetime(20000); $object->override_preferred_lifetime("false"); will set override_preferred_lifetime to "false", and the sequence $object->override_preferred_lifetime("false"); $object->preferred_lifetime(20000); will set override_preferred_lifetime to "true".

Parameter

A 32-bit integer (valid range from 1 to 4294967295) that represents the lifetime, in seconds. The default value is 27000.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get preferred_lifetime
my $preferred_lifetime = $ipv6ha->preferred_lifetime();
#Modify preferred_lifetime
$ipv6ha->preferred_lifetime("369");
```

reserved_interface( )

Use this method to set or retrieve the reserved discovery interface.

Parameter

Valid value is an Infoblox::Grid::Discovery::DeviceInterface object.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get reserved_interface
my $reserved_interface = $ipv6ha->reserved_interface();
#Modify reserved_interface
$ipv6ha->reserved_interface($interface);
```

valid_lifetime( )

Use this method to set or retrieve the valid_lifetime value of a DHCP IPv6 Host Address object.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

A 32-bit integer, from 1 to 4294967295, that represents the lifetime, in seconds. The default value is 43200.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get valid_lifetime
my $valid_lifetime = $ipv6ha->valid_lifetime();
#Modify valid_lifetime
$ipv6ha->valid_lifetime("36900");
```

use_for_ea_inheritance( )

Use this method to set or retrieve the flag that indicates if this host address is used for inheritable extensible attributes.

Parameter

Specify "true" to enable the flag or "false" to disable it. The default value is "false".

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get use_for_ea_inheritance
my $use_for_ea_inheritance = $ipv6ha->use_for_ea_inheritance();
#Modify use_for_ea_inheritance
$ipv6ha->use_for_ea_inheritance("true");
```

SAMPLE CODE

The following sample code demonstrates the different functions that can be applied to an object such as search and modify. This sample also includes error handling for the operations.

#Preparation prior to a DHCP IPv6 Host Address object insertion

```
#PROGRAM STARTS: Include all the modules that will be used, note the hostaddress option
use strict;
use Infoblox qw (:hostaddress);

#Create a session to the Infoblox appliance
```

```
my $session = Infoblox::Session->new(
              master   => "192.168.1.2", #appliance host ip
              username => "admin",       #appliance user login
              password => "infoblox"     #appliance password
);

unless ($session) {
       die("Construct session failed: ",
              Infoblox::status_code() . ":" . Infoblox::status_detail());
}
print "Session created successfully\n";

#Create an IPv6 network object
my $network = Infoblox::DHCP::IPv6Network->new(
                                          "network" => "2001::/64",
                                         );
unless($network) {
     die("Construct IPv6 Network object failed: ",
           Infoblox::status_code() . ":" . Infoblox::status_detail());
}
print "IPv6 Network object created successfully\n";

$session->add($network)
    or die("Add network failed: ",
           $session->status_code() . ":" . $session->status_detail());
print "Network added successfully\n";

#Create a zone object
my $zone = Infoblox::DNS::Zone->new(
                                    name => "test.com",
                                   );
unless($zone) {
     die("Construct Zone object failed: ",
           Infoblox::status_code() . ":" . Infoblox::status_detail());
}
print "Zone object created successfully\n";

$session->add($zone)
    or die("Add zone failed: ",
           $session->status_code() . ":" . $session->status_detail());
print "Zone added successfully\n";

my $option1 = Infoblox::DHCP::Option->new(
    "name"  => "dhcp6.fqdn",
    "value" => "mydomain.com"
);
unless($option1) {
    die("Construct option1 object failed: ",
        Infoblox::status_code() . ":" . Infoblox::status_detail());
}
print "option1 object created successfully\n";

my $option2 = Infoblox::DHCP::Option->new(
    "name"     => "dhcp6.info-refresh-time",
    "value"    => "1000",
);
unless($option2) {
    die("Construct option2 object failed: ",
        Infoblox::status_code() . ":" . Infoblox::status_detail());
}
print "option2 object created successfully\n";
```

#Create a DHCP IPv6 Host address object

```
#Create a DHCP IPv6 Host Address object
my $host_addr = Infoblox::DHCP::IPv6HostAddr->new(
       "ipv6addr"                    => "2001::4",
       "duid"                        => "00:00:00:11:11:11",
       "domain_name"                 => "foo.com",
       "domain_name_servers"         => "2002::1,2002::2",
       "preferred_lifetime"          => 1234,
       "override_preferred_lifetime" => "true",
       "valid_lifetime"              => 123456,
       "override_valid_lifetime"     => "true",
       "options"                     => [ $option1 , $option2 ],
       );
```

```
unless($host_addr) {
     die("Construct IPv6 host address failed: ",
           Infoblox::status_code() . ":" . Infoblox::status_detail());
}
print "DHCP IPv6 Host address object created successfully\n";

my $host = Infoblox::DNS::Host->new(
                                    name         => "host.test.com",
                                    ipv6addrs    => [$host_addr],
                                   );

unless($host) {
     die("Construct IPv6 host record failed: ",
           Infoblox::status_code() . ":" . Infoblox::status_detail());
}

$session->add($host)
    or die("Add host failed: ",
           $session->status_code() . ":" . $session->status_detail());
print "Host added successfully\n";
```

#Get and modify a DHCP IPv6 Host Address object

```
#Get DHCP IPv6 Host Address object through the session
my @retrieved_objs = $session->get(
    object => "Infoblox::DHCP::IPv6HostAddr",
    duid    => "00:00:00:11:11:11",
);
my $object = $retrieved_objs[0];
unless ($object) {
    die("Get IPv6 Host Address object failed: ",
        $session->status_code() . ":" . $session->status_detail());
}
print "Get IPv6 Host Address object found at least 1 matching entry\n";

#Modify one of the attributes of the obtained IPv6 Host Address object
$object->preferred_lifetime(1235);

#Apply the changes
$session->modify($object)
    or die("Modify IPv6 Host Address object failed: ",
           $session->status_code() . ":" . $session->status_detail());
print "IPv6 Host Address object modified successfully \n";

#Verify that this was changed also when retrieving this via the host record
my @retrieved_objs = $session->get(
    object => "Infoblox::DNS::Host",
    name => "host.test.com",
);
my $object = $retrieved_objs[0];
unless ($object) {
    die("Get Host record failed: ",
        $session->status_code() . ":" . $session->status_detail());
}
print "Get host record object found at least 1 matching entry\n";

my $hostaddress = @{$object->ipv6addrs()}[0];

unless ($hostaddress->preferred_lifetime() == 1235) {
    die("The modification of host address has failed: ",
        $session->status_code() . ":" . $session->status_detail());
}
print "The change to preferred lifetime has succeeded correctly\n";

#The host address retrieved this way can also be modified without submitting the full host record
$hostaddress->preferred_lifetime(1111);

#Apply the changes
$session->modify($hostaddress)
    or die("Modify IPv6 Host Address object failed: ",
           $session->status_code() . ":" . $session->status_detail());
print "IPv6 Host Address object modified successfully \n";

#Verify the change was submitted successfully
@retrieved_objs = $session->get(
    object => "Infoblox::DHCP::IPv6HostAddr",
    duid    => "00:00:00:11:11:11",
```

```
);
$object = $retrieved_objs[0];
unless ($object) {
    die("Get IPv6 Host Address object failed: ",
        $session->status_code() . ":" . $session->status_detail());
}
print "Get IPv6 Host Address object found at least 1 matching entry\n";

unless ($hostaddress->preferred_lifetime() == 1111) {
    die("The modification of host address has failed: ",
        $session->status_code() . ":" . $session->status_detail());
}

#Submit the Host object for removal
$session->remove($host)
    or die("Remove Host object failed: ",
        $session->status_code() . ":" . $session->status_detail());
print "Host object removed successfully \n";

#Submit the Zone object for removal
$session->remove($zone)
    or die("Remove Zone object failed: ",
        $session->status_code() . ":" . $session->status_detail());
print "Zone object removed successfully \n";

#Submit the Network object for removal
$session->remove($network)
    or die("Remove Network object failed: ",
        $session->status_code() . ":" . $session->status_detail());
print "Network object removed successfully \n";
```

AUTHOR

Infoblox Inc. http://www.infoblox.com/

SEE ALSO

Infoblox::DHCP::IPv6Network, Infoblox::DHCP::View, Infoblox::DHCP::IPv6Range, Infoblox::Session->get(), Infoblox::Session->modify(), Infoblox::Session->search(), Infoblox::Session, Infoblox::Grid::Discovery::Data, Infoblox::Grid::MSServer::AdUser::Data

---

COPYRIGHT

Copyright (c) 2017 Infoblox Inc.

Infoblox::Grid::DNS::Nsgroup - Creates a name server group.

- NAME
- DESCRIPTION
- CONSTRUCTOR
- SESSION METHODS
  - Infoblox::Session->add( )
  - Infoblox::Session->get( )
  - Infoblox::Session->modify( )
  - Infoblox::Session->remove( )
  - Infoblox::Session->search( )
- MODULE METHODS
  - Infoblox::Grid::DNS->nsgroups( )
- METHODS
  - name( )
  - comment( )
  - extensible_attributes( )
  - is_multimaster( )
  - multiple_primaries( )
  - secondaries( )
- SAMPLE CODE
- AUTHOR
- SEE ALSO
- COPYRIGHT

NAME

Infoblox::Grid::DNS::Nsgroup - Creates a name server group.

DESCRIPTION

A name server group is a collection of one and more primary DNS servers and one or more secondary DNS servers. Grouping a commonly used set of primary and secondary DNS servers together simplifies zone creation, allowing you to specify a single name server group instead of specifying multiple name servers individually.

CONSTRUCTOR

```
 my $nsg1 = Infoblox::Grid::DNS::Nsgroup->new(
     name         => $string,                                                          #Required
     comment      => $string,                                                          #Optional
     extattrs              => { $string => $extattr, ... },                            #Optional
/ Default is undefined
     extensible_attributes => { $string => $string|$num, $string => [ $string|$num, ...], ... },  #Optional
/ Default is undefined
     is_multimaster       => "true" | "false",                                        #Optional
/ Default is "false"
     multiple_primaries   => [$Member1, ...] | [$Nameserver1, ...] | [$MSServer1, ...],  #Optional
/ Default is undefined
     secondaries          => [$Member1, $Member2, $Nameserver1, $Nameserver2, ...]     #Optional
/ List of Infoblox::DNS::Member object(s) and/or Infoblox::DNS::Nameserver object(s).
 );
```

You cannot set both extattrs and extensible_attributes attributes at the same time.

SESSION METHODS

This section describes all the methods in the Infoblox::Session module that you can apply to an Nsgroup object.

Infoblox::Session->add( )

Use this method to add an object to the Infoblox appliance. See Infoblox::Session->add() for parameters and return values.

Example

```
#Creating Member objects.
my $member1 = Infoblox::DNS::Member->new(
    name     => "infoblox.localdomain",
    ipv4addr => "192.168.1.2",
    stealth  => "false"
    );
```

```
my $member2 = Infoblox::DNS::Member->new(
    name     => "member2.test.com",
    ipv4addr => "2.2.2.2",
    stealth  => "false"
    );

#Creating a nameserver object.
my $nameserver1 = Infoblox::DNS::Nameserver->new(
    name     => "ns1.test.extra",
    ipv4addr => "4.4.4.4",
    stealth  => "false",
    TSIGname => "tsig_name",
    TSIGkey  => "ZX1jVJl7C58GT/sc7Q3ucA=="
    );

#Creating a nameserver object.
my $nameserver2 = Infoblox::DNS::Nameserver->new(
    name     => "ns2.test.extra",
    ipv4addr => "5.5.5.5",
    stealth  => "false",
    TSIGname => "tsig_name2",
    TSIGkey  => "qwqwqwqwqwqwqwqwqwqA=="
    );

#Creating a nameserver object.
my $nameserver3 = Infoblox::DNS::Nameserver->new(
    name     => "ns3.test.extra",
    ipv6addr => "12ab::34a",
    stealth  => "false",
    TSIGname => "tsig_name2",
    TSIGkey  => "qwqwqwqwqwqwqwqwqwqA=="
    );

#Construct an object
my $nsg1 = Infoblox::Grid::DNS::Nsgroup->new(
    name        => "ns_group_A",
    multiple_primaries => [$member1],
    secondaries => [$member2, $nameserver1, $nameserver2, $nameserver3]
    );

#Submit for addition
my $response = $session->add( $nsg1 );
```

Infoblox::Session->get( )

Use this method to retrieve all the matching objects from the Infoblox appliance. See Infoblox::Session->get() for parameters and return values.

Key References

```
Apply the following attributes to get a specific Nsgroup object:

 name - Required. A name server group name in string format.
 extattrs      - Optional. A hash reference containing extensible attributes.
 extensible_attributes - Optional. A hash reference containing extensible attributes.
```

Examples

```
my @retrieved_objs = $session->get(
    object => "Infoblox::Grid::DNS::Nsgroup",
    name   => "ns_group_A",
    );

my @retrieved_objs = $session->get(
    object => "Infoblox::Grid::DNS::Nsgroup",
    extensible_attributes => { 'Site' => 'Santa Clara' });
```

Infoblox::Session->modify( )

Use this method to modify an object in the Infoblox appliance. See Infoblox::Session->modify() for parameters and return values.

Example

```
#Use this method to modify the comment.
$nsg1->comment("this is a modified comment");
#Submit modification
my $response = $session->modify( $nsg1 );
```

Infoblox::Session->remove( )

Use this method to remove an object from the Infoblox appliance. See Infoblox::Session->remove() for parameters and return values.

To remove a specific object, first use `get()` or `search()` to retrieve the specific object, and then submit this object for removal.

Example

```
#Get the objects with the same name
my @retrieved_objs = $session->get(
    object => "Infoblox::Grid::DNS::Nsgroup",
    name   => "ns_group_A",
     );
#Find the desired object from the retrieved list.
my $desired_nsgroup= $retrieved_objs[0];
#Submit for removal
my $response = $session->remove( $desired_nsgroup );
```

Infoblox::Session->search( )

Use this method to search for Nsgroup objects in the Infoblox appliance. See Infoblox::Session->search() for parameters and return values.

Key References

```
Apply the following attributes to search for an Nsgroup object:

 name - Required. A host name in string format (regular expression).
 extattrs     - Optional. A hash reference containing extensible attributes.
 extensible_attributes - Optional. A hash reference containing extensible attributes.
```

For more information about searching extensible attributes, see Infoblox::Grid::ExtensibleAttributeDef/Searching Extensible Attributes.

Examples

```
# Search for all Nsgroup objects
my @retrieved_objs = $session->search(
    object => "Infoblox::Grid::DNS::Nsgroup",
    name   => ".*",
    );

# search for all Nsgroup objects with "Santa Clara" as the value for the "Site" extensible attribute
my @retrieved_objs = $session->get(
    object => "Infoblox::Grid::DNS::Nsgroup",
    extensible_attributes => { 'Site' => 'Santa Clara' });
```

MODULE METHODS

This section describes all the methods that you can appy to an Nsgroup object.

Infoblox::Grid::DNS->nsgroups( )

Use this function to specify a name server group at the grid level on the Infoblox appliance. See Infoblox::Grid::DNS->nsgroups() for parameters and return values.

Example

```
#Creating Member objects.

my $member1 = Infoblox::DNS::Member->new(
    name     => "infoblox.localdomain",
    ipv4addr => "192.168.1.2",
    stealth  => "false"
    );

my $member2 = Infoblox::DNS::Member->new(
    name     => "member2.test.com",
    ipv4addr => "2.2.2.2",
    stealth  => "false"
    );

#Creating a nameserver object.
my $nameserver1 = Infoblox::DNS::Nameserver->new(
    name     => "ns1.test.extra",
    ipv4addr => "4.4.4.4",
```

```
    stealth  => "false",
    TSIGname => "tsig_name",
    TSIGkey  => "ZX1jVJl7C58GT/sc7Q3ucA=="
    );

#Creating a nameserver object.
my $nameserver2 = Infoblox::DNS::Nameserver->new(
    name     => "ns2.test.extra",
    ipv4addr => "5.5.5.5",
    stealth  => "false",
    TSIGname => "tsig_name2",
    TSIGkey  => "qwqwqwqwqwqwqwqwqwqA=="
    );

#Creating a nameserver object.
my $nameserver3 = Infoblox::DNS::Nameserver->new(
    name     => "ns3.test.extra",
    ipv6addr => "12ab::34a",
    stealth  => "false",
    TSIGname => "tsig_name2",
    TSIGkey  => "qwqwqwqwqwqwqwqwqwqA=="
    );

my $nsg1 = Infoblox::Grid::DNS::Nsgroup->new(
    name        => "ns_group_A",
    multiple_primaries => [$member1],
    secondaries => [$member2, $nameserver1, $nameserver2,$nameserver3]
    );

my @result_array = $session->get( "object" => "Infoblox::Grid::DNS" ,  "grid" => "Infoblox" );

 my $dns = $result_array[0];
 $dns->nsgroups([$nsg1]);
```

METHODS

This section describes all the methods that you can use to configure and retrieve the attribute values of an Nsgroup object

name( )

Use this method to set or retrieve the name server group, providing a meaningful reference for the set of servers.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

Name of the nameserver group in string format.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
 #Getting name
  my $name = $nsg1->name();
  #Modifying name
  $nsg1->name("ns_group_2");
```

comment( )

Use this method to set or retrieve the comment field of the Nsgroup object. You can enter additional information about the NS group in this field.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

Comment of the nameserver group in string format.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Getting comment
my $comment = $nsg1->comment();
#Modifying comment
$nsg1->comment("Some comment");
```

extensible_attributes( )

Use this method to set or retrieve the extensible attributes associated with an Nsgroup object.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

For valid values for extensible attributes, see Infoblox::Grid::ExtensibleAttributeDef/Extensible Attribute Values.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get extensible attributes
my $ref_extensible_attributes = $nsg1->extensible_attributes();
#Modify extensible attributes
$nsg1->extensible_attributes({ 'Site' => 'Santa Clara', 'Administrator' => [ 'Peter', 'Tom' ] });
```

is_multimaster( )

Use this method to set or retrieve the is_multimaster flag.

Parameter

Specify "true" to set the is_multimaster flag or "false" to deactivate/unset it. The default value is "false".

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
# Get the attribute value
my $attr = $nsg1->is_multimaster();
# Modify the attribute value
$nsg1->is_multimaster('true');
```

multiple_primaries( )

Use this method to set or retrieve Grid primary servers or external primary servers of the name server group.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

The valid value is an array reference that contains Infoblox::DNS::Member objects for Grid primary servers or Infoblox::DNS::Nameserver objects for external primary servers.

To set two and more Infoblox::DNS::Member objects you should set the is_multimaster flag to "true".

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#For grid primary:

my $member1 = Infoblox::DNS::Member->new(
    name     => "infoblox.localdomain",
    ipv4addr => "192.168.1.2",
```

```
    stealth  => "true"
);

$nsg1->multiple_primaries([$member1]);

#For external primary:

my $nameserver1 = Infoblox::DNS::Nameserver->new(
    name     => "ns1.domain.extra",
    ipv4addr => "5.5.5.5",
    stealth  => "false",
    TSIGname => "tsig_name",
    TSIGkey  => "ZX1jVJl7C58GT/sc7Q3ucA=="
);

my $nameserver2 = Infoblox::DNS::Nameserver->new(
    name     => "ns2.domain.extra",
    ipv4addr => "6.6.6.6",
    stealth  => "false",
    TSIGname => "tsig_name2",
    TSIGkey  => "qwqwqwqwqwqwqwqwqwqA=="
);

#Creating a nameserver object.
my $nameserver3 = Infoblox::DNS::Nameserver->new(
    name     => "ns3.test.extra",
    ipv6addr => "12ab::34a",
    stealth  => "false",
    TSIGname => "tsig_name2",
    TSIGkey  => "qwqwqwqwqwqwqwqwqwqA=="
    );

$nsg1->multiple_primaries([$nameserver1, $nameserver2,$nameserver3]);
```

secondaries( )

Use this method to set or retrieve the grid secondary server or external secondary servers of the nameserver group.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

The valid value is an array reference that contains Infoblox::DNS::Member objects for grid secondary servers and Infoblox::DNS::Nameserver objects for external secondary servers.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails. If you did not specify a parameter, the method returns the attribute value.

Example

```
my $member2 = Infoblox::DNS::Member->new(
    name     => "member2.domain.com",
    ipv4addr => "2.2.2.2",
    stealth  => "false"
);

my $nameserver1 = Infoblox::DNS::Nameserver->new(
    name     => "ns1.domain.extra",
    ipv4addr => "4.4.4.4",
    stealth  => "false",
    TSIGname => "tsig_name",
    TSIGkey  => "ZX1jVJl7C58GT/sc7Q3ucA=="
);

my $nameserver2 = Infoblox::DNS::Nameserver->new(
    name     => "ns2.domain.extra",
    ipv4addr => "5.5.5.5",
    stealth  => "false",
    TSIGname => "tsig_name2",
    TSIGkey  => "qwqwqwqwqwqwqwqwqwqA=="
);

#Creating a nameserver object.
my $nameserver3 = Infoblox::DNS::Nameserver->new(
```

```
            name     => "ns3.test.extra",
            ipv6addr => "12ab::34a",
            stealth  => "false",
            TSIGname => "tsig_name2",
            TSIGkey  => "qwqwqwqwqwqwqwqwqwqA=="
            );

      $nsg1->secondaries([$member2, $nameserver1, $nameserver2,$nameserver3]);
```

SAMPLE CODE

The following sample code demonstrates the different functions that can be applied to an object, such as modify and remove. This sample also includes error handling for the operations.

#Preparation prior to an Nsgroup object insertion

```
#PROGRAM STARTS: Include all the modules that will be used
use strict;
use Infoblox;

#Create a session to the Infoblox device
my $session = Infoblox::Session->new(
    master   => "192.168.1.2",
    username => "admin",
    password => "infoblox"
);
unless ($session) {
   die("Construct session failed: ",
       Infoblox::status_code() . ":" . Infoblox::status_detail());
}
print "Session created successfully\n";
```

#Create an Nsgroup object

```
#Creating a member object.
my $member1 = Infoblox::DNS::Member->new(
    name     => "infoblox.localdomain",
    ipv4addr => "192.168.1.2",
    stealth  => "false"
);

#Creating a nsgroup object.
my $nsg1 = Infoblox::Grid::DNS::Nsgroup->new(
    name    => "ns_group_1",
    multiple_primaries => [$member1]
);

my $nsg1_A = Infoblox::Grid::DNS::Nsgroup->new(
    name    => "ns_group_A",
    multiple_primaries => [$member1]
);

unless ($nsg1) {
   die("Construct Nsgroup failed: ",
       Infoblox::status_code() . ":" . Infoblox::status_detail());
}
print "Nsgroup object created successfully\n";
```

#Get and add the Nsgroup object

```
my @result_array = $session->get( "object" => "Infoblox::Grid::DNS" ,  "grid" => "Infoblox" );

if( defined @result_array  and scalar( @result_array ) > 0 )
{
       my $object = $result_array[0];
       if( ref( $object ) eq "Infoblox::Grid::DNS" )
       {

               $object->nsgroups( [ $nsg1 ,$nsg1_A] );

               my $response = $session->modify( $object );
               print "Add Nsgroup to grid dns properties success.\n" if ( $response );
                     }

 unless ($object) {
    die("Add Nsgroup to grid dns properties failed: ",
```

```
        $session->status_code() . ":" . $session->status_detail());
 }
  }
```

#Modify the Nsgroup object

```
my @result_array = $session->get( "object" => "Infoblox::Grid::DNS" ,  "grid" => "Infoblox" );

if( defined @result_array  and scalar( @result_array ) > 0 )
{
       my $object = $result_array[0];
       if( ref( $object ) eq "Infoblox::Grid::DNS" )
       {

          $nsg1_A->name("ns_group_modified");

         $object->nsgroups( [ $nsg1 ,$nsg1_A] );

               my $response = $session->modify( $object );
               print "Modify Nsgroup success.\n" if ( $response );
                        }

 unless ($object) {
    die("Modify NSgroup failed: ",
        $session->status_code() . ":" . $session->status_detail());
 }
  }
```

#Remove a Nsgroup object

```
my @result_array = $session->get( "object" => "Infoblox::Grid::DNS" ,  "grid" => "Infoblox" );

if( defined @result_array  and scalar( @result_array ) > 0 )
{
       my $object = $result_array[0];
       if( ref( $object ) eq "Infoblox::Grid::DNS" )
       {

               $object->nsgroups( [ ] );

               my $response = $session->modify( $object );
               print "Remove Nsgroup success.\n" if ( $response );
                        }

 unless ($object) {
    die("Remove Nsgroup failed: ",
        $session->status_code() . ":" . $session->status_detail());
 }
  }

####PROGRAM ENDS####
```

AUTHOR

Infoblox Inc. http://www.infoblox.com/

SEE ALSO

Infoblox::Session, Infoblox::Session->get(),Infoblox::Session->modify(),Infoblox::Grid::DNS,
Infoblox::DNS::Member,Infoblox::DNS::Nameserver

COPYRIGHT

Copyright (c) 2017 Infoblox Inc.

Infoblox::DNS::TSIGKey - DNS TSIGKey object

- NAME
- DESCRIPTION
- CONSTRUCTOR
- MODULE METHODS
  - Infoblox::Grid::DNS->allow_query( )
  - Infoblox::Grid::Member::DNS->allow_query( )
  - Infoblox::DNS::Zone->allow_query( )
  - Infoblox::Grid::DNS->allow_transfer( )
  - Infoblox::Grid::Member::DNS->allow_transfer( )
  - Infoblox::DNS::Zone->allow_transfer( )
  - Infoblox::Grid::DNS->allow_update( )
  - Infoblox::Grid::Member::DNS->allow_update( )
  - Infoblox::DNS::Zone->allow_update( )
  - Infoblox::Grid::DNS->recursive_query_list( )
  - Infoblox::Grid::Member::DNS->recursive_query_list( )
- METHODS
  - algorithm( )
  - key( )
  - name( )
- SAMPLE CODE
- AUTHOR
- SEE ALSO
- COPYRIGHT

NAME

Infoblox::DNS::TSIGKey - DNS TSIGKey object

DESCRIPTION

On a device you configure as a secondary serer for a zone, you can associate a TSIG key for each primary server to which the secondary server requests zone transfers. On the device you configure as a primary server for a zone, you can set a TSIG key at the grid, member, or zone level. When the secondary server requests zone transfers, it must send a specific key in its requests to the primary server. When the primary server responds to the requests, it can have a set of TSIG keys from which it can draw when responding. As long as the primary server can find the same TSIG key that the secondary sends it, it can verify the authenticity of the requests it receives and authenticate the responses it sends.

TSIG is based on DNS specifications RFC 2845, TSIG and RFC 2930, TKEY which provides authentication protocol enhancements to secure Server-Server transactions.

CONSTRUCTOR

```
my $tsig_key = Infoblox::DNS::TSIGKey->new(
   key  => $string | ":2xCOMPAT",  #Optional / Default is empty
   name => $string,                #Optional / Default is empty
 );
```

MODULE METHODS

This section describes all the functions that you can apply to a TSIGKey object.

Infoblox::Grid::DNS->allow_query( )

Use this function to specify a TSIG key for a Grid DNS object. This key is used to authenticate clients from which DNS queries are allowed.

Example

```
## Construct the object
my $tsig_key = Infoblox::DNS::TSIGKey->new(
    key  => "Zx1jVJl7C58GT/sc7Q3ucA==",
    name => "test_key"
);
#Adding a TSIG key to the allow_query list of the Grid DNS object
#Retrieving the list
my $allow_query_ref= $Grid_DNS->allow_query();
push @$allow_query_ref, $tsig_key;
#Modifying the object;
```

```
my $response = $Grid_DNS->allow_query($allow_query_ref);
```

Infoblox::Grid::Member::DNS->allow_query( )

Use this function to specify a TSIG key for a Grid Member DNS object. This key is used to authenticate clients from which DNS queries are allowed.

Example

```
##Construct the object
my $tsig_key = Infoblox::DNS::TSIGKey->new(
    key  => "ZX1jVjl7C58GT/sc7Q3ucA==",
    name => "test_key"
);
#Adding a TSIG key to the allow_query list of the Grid Member DNS object
#Retrieving the list
my $allow_query_ref= $Grid_Member_DNS->allow_query();
push @$allow_query_ref, $tsig_key;
#Modifying the object;
my $response = $Grid_Member_DNS->allow_query($allow_query_ref);
```

Infoblox::DNS::Zone->allow_query( )

Use this function to specify a TSIG key for a Zone object. This key is used to authenticate clients from which DNS queries are allowed.

Example

```
## Construct the object
my $tsig_key = Infoblox::DNS::TSIGKey->new(
    key  => "ZX1jVJl7c58GT/sc7Q3ucA==",
    name => "test_key"
);
#Adding a TSIG key to the allow_query list of a zone
#Retrieving the list
my $allow_query_ref= $zone->allow_query();
push @$allow_query_ref, $tsig_key;
#Modifying the object;
my $response = $zone->allow_query($allow_query_ref);
```

Infoblox::Grid::DNS->allow_transfer( )

Use this function to specify a TSIG key for a Grid DNS object. This key is used during zone transfers from the primary server to the secondary servers.

Example

```
## Construct the object
my $tsig_key = Infoblox::DNS::TSIGKey->new(
    key  => "Zx1jVJl7C58GT/sc7Q3ucA==",
    name => "test_key"
);
#Adding a TSIG key to the allow_transfer list of the Grid DNS object
#Retrieving the list
my $allow_transfer_ref= $Grid_DNS->allow_transfer();
push @$allow_transfer_ref, $tsig_key;
#Modifying the object;
my $response = $Grid_DNS->allow_transfer($allow_transfer_ref);
```

Infoblox::Grid::Member::DNS->allow_transfer( )

Use this function to specify a TSIGKey for a grid member object. This key is used during zone transfers from the primary server to the secondary servers.

Example

```
## Construct the object
my $tsig_key = Infoblox::DNS::TSIGKey->new(
    key  => "ZX1jVjl7C58GT/sc7Q3ucA==",
    name => "test_key"
);
#Adding a TSIG key to the allow_transfer list of the Grid Member DNS object
#Retrieving the list
my $allow_transfer_ref= $Grid_Member_DNS->allow_transfer();
push @$allow_transfer_ref, $tsig_key;
#Modifying the object;
my $response = $Grid_Member_DNS->allow_transfer($allow_transfer_ref);
```

Infoblox::DNS::Zone->allow_transfer( )

Use this function to specify a TSIGKey for a Zone object. This key is used during zone transfers from the primary server to the secondary servers.

Example

```
## Construct the object
 my $tsig_key = Infoblox::DNS::TSIGKey->new(
   key  => "ZX1jVJl7c58GT/sc7Q3ucA==",
   name => "test_key"
 );
 #Adding a TSIG key to the allow_transfer list of a zone
 #Retrieving the list
 my $allow_transfer_ref= $zone->allow_transfer();
 push @$allow_transfer_ref, $tsig_key;
 #Modifying the object;
 my $response = $zone->allow_transfer($allow_transfer_ref);
```

Infoblox::Grid::DNS->allow_update( )

Use this function to specify a TSIGKey for a grid DNS object. This key is used during dynamic DNS updates.

Example

```
## Construct the object
 my $tsig_key = Infoblox::DNS::TSIGKey->new(
   key  => "ZX1jVJl7C58gT/sc7Q3ucA==",
   name => "test_key"
 );
 #Adding a TSIG key to the allow_update list of the Grid DNS object
 #Retrieving the list
 my $allow_update_ref= $Grid_DNS->allow_update();
 push @$allow_update_ref, $tsig_key;
 #Modifying the object;
 my $response = $Grid_DNS->allow_update($allow_transfer_ref);
```

Infoblox::Grid::Member::DNS->allow_update( )

Use this function to specify a TSIGKey for a grid member object. This key is used during dynamic DNS updates.

Example

```
## Construct the object
my $tsig_key = Infoblox::DNS::TSIGKey->new(
    key  => "ZX1jVJl7C58Gt/sc7Q3ucA==",
    name => "test_key"
);
# Configure a TSIG key for a Grid Member DNS object
#Retrieving the list
my $allow_update_ref= $Grid_Member_DNS->allow_update();
push @$allow_update_ref, $tsig_key;
#Modifying the object;
my $response = $Grid_Member_DNS->allow_update($allow_transfer_ref);
```

Infoblox::DNS::Zone->allow_update( )

Use this function to specify a TSIGKey for a Zone object. This key is used during dynamic DNS updates.

Example

```
## Construct the object
 my $tsig_key = Infoblox::DNS::TSIGKey->new(
   key  => "ZX1jVJl7C58GT/sc7q3ucA==",
   name => "test_key"
 );
 # Configure TSIGKey on a Zone
 #Retrieving the list
 my $allow_update_ref= $zone->allow_update();
 push @$allow_update_ref, $tsig_key;
 #Modifying the object;
 my $response = $zone->allow_update($allow_transfer_ref);
```

Infoblox::Grid::DNS->recursive_query_list( )

Use this function to specify a TSIG key for a Grid DNS object. This key is used to authenticate clients from which the grid is allowed to receive recursive queries.

Example

```
## Construct the object
 my $tsig_key = Infoblox::DNS::TSIGKey->new(
   key  => "ZX1jVJl7C58gT/sc7Q3ucA==",
   name => "test_key"
 );
 #Adding a TSIG key to the recursive_query_list list of the Grid DNS object
 #Retrieving the list
 my $recursive_query_list_ref= $Grid_DNS->recursive_query_list();
 push @$recursive_query_list_ref, $tsig_key;
 #Modifying the object;
 my $response = $Grid_DNS->recursive_query_list($allow_transfer_ref);
```

Infoblox::Grid::Member::DNS->recursive_query_list( )

Use this function to specify a TSIG key for a Grid Member DNS object. This key is used to authenticate clients from which recursive queries are allowed.

Example

Example

```
## Construct the object
my $tsig_key = Infoblox::DNS::TSIGKey->new(
    key  => "ZX1jVJl7C58Gt/sc7Q3ucA==",
    name => "test_key"
);

# Configure a TSIG key for a Grid Member DNS object
#Retrieving the list
my $recursive_query_list_ref= $Grid_Member_DNS->recursive_query_list();
push @$recursive_query_list_ref, $tsig_key;
#Modifying the object;
my $response = $Grid_Member_DNS->recursive_query_list($allow_transfer_ref);
```

METHODS

This section describes all the methods that you can use to configure and retrieve the attribute values of a TSIGKey object.

algorithm( )

Use this method to set or retrieve the algorithm of the TSIG key you want to use.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

Desired algorithm of the TSIG key. Valid values are "HMAC-SHA256" and "HMAC-MD5", which is the default.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
# Getting Algorithm
my $alg = $algorithmserver->algorithm();
# Modifying algorithm
$tsig_key->algorithm("HMAC-SHA256");
```

key( )

Use this method to add or retrieve a TSIGKey. To use an existing TSIG key, type or paste the key in the Key field. The TSIG key string is a text string with a key generated by an encryption algorithm

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

For TSIG zone transfers, valid values for a key are `":2xCOMPAT"` or a TSIG key string. `":2xCOMPAT"` means you are using Infoblox DNSone 2.x TSIG. If the key is `":2xCOMPAT"`, the name can only be `"tsig_xfer"`. For TSIG dynamic updates, the valid value a for key is a TSIG key string.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
# Getting key
my $key = $tsig_key->key();
# Modifying key
$tsig_key->key("ZX1jVJl7C58GT/sc7Q3ucB==");
```

name( )

Use this method to set or retrieve the name of the TSIGKey.

Enter a meaningful name for the key, such as a zone name or the name of the remote name server with which the local server authenticates zone transfer requests and replies. This name must match the name of the TSIG key on the other name server that use it to authenticate zone transfers with the local server.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

Text with the TSIG key name.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
# Getting name
my $name = $tsig_key->name();
# Modifying name
$tsig_key->name("tsig_key1");
```

SAMPLE CODE

The following sample code demonstrates the different functions that can be applied to an object, such as add and modify. This sample also includes error handling for the operations.

##Preparation prior to a DNS TSIGKey object insertion

```
#PROGRAM STARTS: Include all the modules that will be used
use strict;
use Infoblox;

#Create a session to the Infoblox device
my $session = Infoblox::Session->new(
    master   => "192.168.1.2",
    username => "admin",
    password => "infoblox"
);
unless ($session) {
   die("Construct session failed: ",
       Infoblox::status_code() . ":" . Infoblox::status_detail());
}
print "Session created successfully\n";
```

#Create a TSIGKey object

```
 my $tsig_key = Infoblox::DNS::TSIGKey->new(
    key  => "ZX1jVJl7C58GT/sc7Q3ucA==",
    name => "test_key"
  );

unless($tsig_key) {
     die("Construct TSIGKey failed: ",
           Infoblox::status_code() . ":" . Infoblox::status_detail());
}
print "TSIGKey object created successfully\n";

#Get the member DNS object and add a TSIGKey object to it

my @result_array = $session->get(
```

```
    object => "Infoblox::Grid::Member::DNS",
    name   => "infoblox.localdomain"
);

unless (scalar(@result_array) == 0) {
    my $memberdns = $result_array[0];
    if ($memberdns) {

       #Add a TSIG key to the Grid Member DNS object allow_transfer list.
       my $allow_transfer_ref=$memberdns->allow_transfer();
       push @$allow_transfer_ref, $tsig_key;
       $memberdns->allow_transfer($allow_transfer_ref)
            or die("modify member failed: ",
                   Infoblox::status_code() . ":" . Infoblox::status_detail());
        print "TSIGKey added to Member DNS object successfully\n";

    #Update the Grid Member DNS object through the Infoblox session.
        $session->modify($memberdns)
            or die("modify session failed: ",
                   $session->status_code() . ":" . $session->status_detail());
        print "Member DNS object with TSIGKey updated to Infoblox device successfully\n";

        #Modify existing TSIGKey object
        #Modifying the value of the specified object.

        $tsig_key->name("another_test_key");
        print "Modify TSIG name value\n";

        #Replace the key in the allow_transfer list
        my $allow_transfer_ref2=$memberdns->allow_transfer();

        if(ref($allow_transfer_ref2) eq 'ARRAY'){
            map { if((ref($_) eq 'Infoblox::DNS::TSIGKey')&&($_->name() eq 'test_key')){ $_=$tsig_key;} }
@$allow_transfer_ref2;
        }

        $memberdns->allow_transfer($allow_transfer_ref2)
            or die("modify member failed: ",
                   Infoblox::status_code() . ":" . Infoblox::status_detail());
        print "TSIG name updated to Member DNS object successfully\n";

       #Update the Grid Member DNS object through the Infoblox session.
        $session->modify($memberdns)
            or die("modify session failed: ",
                $session->status_code() . ":" . $session->status_detail());
                print "Member DNS object with TSIG name updated to Infoblox device successfully\n";
             }
         } else {
             print "No member found with the specified name.";
         }

 ####PROGRAM ENDS####
```

AUTHOR

Infoblox Inc. http://www.infoblox.com/

SEE ALSO

Infoblox::Session,Infoblox::Session->get(), Infoblox::Session->modify(),Infoblox::Grid::Member::DNS, Infoblox::DNS::Zone, Infoblox::Grid::DNS

COPYRIGHT

Copyright (c) 2017 Infoblox Inc.

Infoblox::Grid::Member - Grid Member object.

- NAME
- DESCRIPTION
- CONSTRUCTOR
- SESSION METHODS
  - Infoblox::Session->add( )
  - Infoblox::Session->get( )
  - Infoblox::Session->modify( )
  - Infoblox::Session->remove( )
  - Infoblox::Session->search( )
- METHODS
  - active_position( )
  - additional_ip_list( )
  - allow_syslog_clients( )
  - alternate_resolver( )
  - bgp_as( )
  - comment( )
  - config_addr_type( )
  - dscp( )
  - enable_ha( )
  - enable_lom( )
  - enable_member_redirect( )
  - enable_ntp( )
  - enable_snmpv3_query( )
  - enable_snmpv3_traps( )
  - enable_ro_api_access( )
  - enable_syslog_proxy( )
  - external_ntp_servers_enabled( )
  - exclude_grid_master_as_ntp_server( )
  - extattrs( )
  - extensible_attributes( )
  - gateway( )
  - ha_node2_port_duplex( )
  - ha_node2_port_speed( )
  - ha_port_duplex( )
  - ha_port_speed( )
  - ha_reserved_interface( )
  - paid_nios( )
  - hwplatform( )
  - hwid( )
  - is_dscp_capable( )
  - ipv4addr( )
  - ipv6addr( )
  - ipv6_cidr( )
  - ipv6_enable_auto_config( )
  - ipv6_gateway( )
  - ipv6_static_routes( )
  - ipv6_vlan_id( )
  - lan_reserved_interface( )
  - lan1_dscp( )
  - lan1_ipv6_dscp( )
  - lan2_dscp( )
  - lan2_gateway( )
  - lan2_ipv4addr( )
  - lan2_ipv6addr( )
  - lan2_ipv6_cidr( )
  - lan2_ipv6_dscp( )
  - lan2_ipv6_enable( )
  - lan2_ipv6_enable_auto_config( )
  - lan2_ipv6_gateway( )
  - lan2_ipv6_vlan_id( )
  - lan2_mask( )
  - lan2_node2_port_duplex( )
  - lan2_port( )
  - lan2_node2_port_speed( )
  - lan2_port_duplex( )
  - lan2_port_speed( )
  - lan2_reserved_interface( )
  - lan2_router_id( )
  - lan2_vlan_id( )
  - lan_node2_port_duplex( )
  - lan_node2_port_speed( )

- lan_port_duplex( )
- lan_port_speed( )
- lcd_input( )
- lcd_autodim( )
- lcd_brightness( )
- lcd_hwident( )
- lcd_version( )
- lom_gateway( )
- lom_ipv4addr( )
- lom_is_capable( )
- lom_mask( )
- lom_users( )
- mask( )
- master_candidate( )
- member_service_communication( )
- mgmt_dscp( )
- mgmt_gateway( )
- mgmt_lan( )
- mgmt_ipv6addr( )
- mgmt_ipv6_cidr( )
- mgmt_ipv6_dscp( )
- mgmt_ipv6_enable_auto_config( )
- mgmt_ipv6_gateway( )
- mgmt_mask( )
- mgmt_node2_port_duplex( )
- mgmt_node2_port_speed( )
- mgmt_port( )
- mgmt_port_duplex( )
- mgmt_port_speed( )
- mgmt_reserved_interface( )
- mmdb_ea_build_time( )
- mmdb_geoip_build_time( )
- name( )
- nat_enabled( )
- nat_group( )
- nat_ip_address( )
- nic_failover_enabled( )
- nic_failover_enable_primary( )
- node1_ha( )
- node1_ha_reserved_interface( )
- node1_paid_nios( )
- node1_hwplatform( )
- node1_hwid( )
- node1_lan( )
- node1_lan_ipv6addr( )
- node1_lan_reserved_interface( )
- node1_lan2_reserved_interface( )
- node1_lcd_autodim( )
- node1_lcd_brightness( )
- node1_lcd_hwident( )
- node1_lcd_version( )
- node1_lom_gateway( )
- node1_lom_is_capable( )
- node1_lom_ipv4addr( )
- node1_lom_mask( )
- node1_mgmt_dscp( )
- node1_mgmt_gateway( )
- node1_mgmt_ipv6addr( )
- node1_mgmt_ipv6_cidr( )
- node1_mgmt_ipv6_dscp( )
- node1_mgmt_ipv6_enable_auto_config( )
- node1_mgmt_ipv6_gateway( )
- node1_mgmt_mask( )
- node1_mgmt_lan( )
- node1_mgmt_port( )
- node1_mgmt_reserved_interface( )
- node1_nat( )
- node1_service_status( )
- vpn_mtu( )
- node1_vpn_on_mgmt( )
- node1_hwtype( )
- node2_ha( )
- node2_ha_reserved_interface( )
- node2_paid_nios( )

  - node2_hwplatform( )
  - node2_hwid( )
  - node2_lan( )
  - node2_lan_ipv6addr( )
  - node2_lan_reserved_interface( )
  - node2_lan2_reserved_interface( )
  - node2_lcd_autodim( )
  - node2_lcd_brightness( )
  - node2_lcd_hwident( )
  - node2_lcd_version( )
  - node2_lom_gateway( )
  - node2_lom_is_capable( )
  - node2_lom_ipv4addr( )
  - node2_lom_mask( )
  - node2_mgmt_dscp( )
  - node2_mgmt_gateway( )
  - node2_mgmt_mask( )
  - node2_mgmt_lan( )
  - node2_mgmt_port( )
  - node2_mgmt_reserved_interface( )
  - node2_mgmt_ipv6_dscp( )
  - node2_nat( )
  - node2_service_status( )
  - node2_vpn_on_mgmt( )
  - node2_hwtype( )
  - notification_email_addr( )
  - ntp_access_list( )
- ntp_kod( )
  - ntp_service_type( )
  - ntp_authentication_key( )
  - ntp_server( )
  - ospf_list( )
  - override_enable_lom( )
  - override_lom_users( )
  - override_dscp( )
  - override_enable_member_redirect( )
  - override_lan1_dscp( )
  - override_lan1_ipv6_dscp( )
  - override_lan2_dscp( )
  - override_lan2_ipv6_dscp( )
  - override_mgmt_dscp( )
  - override_mgmt_ipv6_dscp( )
  - override_node1_mgmt_dscp( )
  - override_node1_mgmt_ipv6_dscp( )
  - override_node2_mgmt_dscp( )
  - override_node2_mgmt_ipv6_dscp( )
  - override_ntp_kod( )
  - override_syslog_backup_servers( )
  - override_threshold_traps( )
  - override_trap_notifications( )
  - passive_ha_arp_enabled( )
  - platform( )
  - prefer_resolver( )
  - pre_provisioning( )
  - preserve_if_owns_delegation( )
  - query_comm_string( )
  - remote_console_access( )
  - router_ID( )
  - search_domains( )
  - service_status( )
  - service_type_configuration( )
  - snmp_admin( )
  - snmpv3_query_users( )
  - static_routes( )
  - support_access( )
  - support_access_info( )
  - syslog_proxy_tcp_port( )
  - syslog_proxy_udp_port( )
  - syslog_size( )
  - syslog_backup_servers( )
  - syslog_server( )
  - time_zone( )
  - threshold_traps( )
  - traffic_capture( )

- trap_comm_string( )
- trap_notifications( )
- trap_receiver( )
- upgrade_group( )
- use_v4_vrrp( )
- vpn_on_mgmt( )
- vlan_id( )
- create_token( )
- read_token( )

- SAMPLE CODE
- AUTHOR
- SEE ALSO
- COPYRIGHT

NAME

Infoblox::Grid::Member - Grid Member object.

DESCRIPTION

A grid member is a single Infoblox appliance or an HA pair of Infoblox appliances that belong to a grid.

CONSTRUCTOR

```
 my $grid_member = Infoblox::Grid::Member->new(
     gateway                 => $ipv4addr,                                   #Required
     ipv4addr                => $ipv4addr,                                   #Required
     mask                    => $netmask,                                    #Required
     name                    => $fqdn,                                       #Required
     additional_ip_list      => [$Interface1, $ipv4addr2, ...],              #Optional / Default
value is empty
     allow_syslog_clients    => [$ipv4addr | $ipv4_subnet/prefix, ...] | $nacl | undef, #Optional / Default
value is undefined
     bgp_as                  => [$AS1, ...],                                 #Optional / Default
value is empty
     alternate_resolver      => $Nameserver | $Member | $ipv4addr | undef,    #Optional / Default
value is undefined
     comment                 => $string,                                     #Optional / Default
value is empty
     config_addr_type        => "IPv4", "IPv6", "BOTH"                       #Optional / Default
value is "IPv4"
     dscp                    => $uint,
#Optional / Default value is 0
     enable_member_redirect  => "true" | "false",                            #Optional / Default
value is "false"
     enable_ha               => "true" | "false",                            #Optional / Default
value is "false"
     enable_lom              => "true" | "false",                            #Optional / Default
value is undefined
     enable_ntp              => "true" | "false",                            #Optional / Default
value is "false"
     enable_ro_api_access    => "true" | "false",                            #Optional / Default
value is "false"
     enable_syslog_proxy     => "true" | "false",                            #Optional / Default
value is "false"
     exclude_grid_master_as_ntp_server=> "true" | "false",                   #Optional / Default
value is "false"
     extattrs                => { $string => $extattr, ... },                #Optional / Default is
undefined
     extensible_attributes   => { $string => $string | $num, $string => [ $string | $num, ... ], ... }
#Optional / Default is undefined
     external_ntp_servers_enabled  => "true" | "false",                      #Optional / Default
value is "false"
     ha_node2_port_duplex    => "Half" | "Full" | undef,                     #Optional / Default
value is undefined
     ha_node2_port_speed     => 10 | 100 | 1000 | undef,                     #Optional / Default
value is undefined
     ha_port_duplex          => "Half" | "Full" | undef,                     #Optional / Default
value is undefined
     ha_port_speed           => 10 | 100 | 1000 | undef,                     #Optional / Default
value is undefined
     ha_reserved_interface   => $interface,                                  #Optional
     ipv6addr                => $ipv6addr,                                   #Optional / Default
value is undefined
     ipv6_cidr               => $cidr,                                       #Optional / Default
value is undefined
     ipv6_gateway            => $ipv6addr,                                   #Optional / Default
```

```
  value is undefined
      ipv6_enable_auto_config => "true" | "false",                    #Optional / Default
  value is "false"
      ipv6_static_routes      => [$Route1, $Route2, ...],             #Optional / Default
  value is empty
      ipv6_vlan_id            => $uint,                               #Optional / Default
  value is undefined
      lan1_dscp               => $uint,                               #Optional / Default
  value is undefined
      lan1_ipv6_dscp          => $uint,                               #Optional / Default
  value is undefined
      lan2_dscp               => $uint,                               #Optional / Default
  value is undefined
      lan2_gateway            => $ipv4addr,                           #Optional / Default
  value is undefined
      lan2_ipv4addr           => $ipv4addr,                           #Optional / Default
  value is undefined
      lan2_ipv6addr           => $ipv6addr,                           #Optional / Default
  value is undefined
      lan2_ipv6_cidr          => $cidr,                               #Optional / Default
  value is undefined
      lan2_ipv6_dscp          => $uint,                               #Optional / Default
  value is undefined
      lan2_ipv6_gateway       => $ipv6addr,                           #Optional / Default
  value is undefined
      lan2_ipv6_enable        => "true" | "false",                    #Optional / Default
  value is "false"
      lan2_ipv6_enable_auto_config => "true" | "false",               #Optional / Default
  value is "false"
      lan2_ipv6_vlan_id       => $uint,                               #Optional / Default
  value is undefined
      lan2_mask               => $netmask,                            #Optional / Default
  value is undefined
      lan2_node2_port_duplex  => "Half" | "Full" | undef,             #Optional / Default
  value is undefined
      lan2_node2_port_speed   => 10 | 100 | 1000 | undef,             #Optional / Default
  value is undefined
      lan2_port_duplex        => "Half" | "Full" | undef,             #Optional / Default
  value is undefined
      lan2_port_speed         => 10 | 100 | 1000 | undef,             #Optional / Default
  value is undefined
      lan2_port               => "true" | "false",                    #Optional / Default
  value is "false"
      lan2_reserved_interface => $interface,                          #Optional
      lan2_router_id          => $num,                                #Optional / Default
  value is undefined
      lan2_vlan_id            => $uint,                               #Optional / Default
  value is undefined
      lan_node2_port_duplex   => "Half" | "Full" | undef,             #Optional / Default
  value is undefined
      lan_node2_port_speed    => 10 | 100 | 1000 | undef,             #Optional / Default
  value is undefined
      lan_port_duplex         => "Half" | "Full" | undef,             #Optional / Default
  value is undefined
      lan_port_speed          => 10 | 100 | 1000 | undef,             #Optional / Default
  value is undefined
      lan_reserved_interface  => $interface,                          #Optional
      lcd_input               => "true" | "false" | undef,            #Optional / Default
  value is undefined
      lcd_autodim             => $num,                                #Optional / Default
  value is 30
      lcd_brightness          => $num,                                #Optional / Default
  value is 7
      lcd_hwident             => "true" | "false",                    #Optional / Default
  value is "false"
      lom_gateway             => $ipv4addr,
  #Optional / Default value is undefined
      lom_ipv4addr            => $ipv4addr,
  #Optional / Default value is undefined
      lom_mask                => $netmask,
  #Optional / Default value is undefined
      lom_users               => [$user1, $user2, ...],
  #Optional / Default value is undefined
      node1_lom_gateway       => $ipv4addr,
  #Optional / Default value is undefined
      node1_lom_ipv4addr      => $ipv4addr,
  #Optional / Default value is undefined
      node1_lom_mask          => $netmask,
  #Optional / Default value is undefined
      node2_lom_gateway       => $ipv4addr,
  #Optional / Default value is undefined
      node2_lom_ipv4addr      => $ipv4addr,
  #Optional / Default value is undefined
      node2_lom_mask          => $netmask,
```

```
#Optional / Default value is undefined
     master_candidate         => "true" | "false",                          #Optional / Default
value is undefined
     member_service_communication => [$c1, $c2, ...],                       #Optional / Default
value is undefined
     mgmt_dscp                => $uint,                                     #Optional / Default
value is undefined
     mgmt_gateway             => $ipv4addr,                                 #Optional / Default
value is undefined
     mgmt_lan                 => $ipv4addr,                                 #Optional / Default
value is undefined
     mgmt_ipv6addr            => $ipv6addr,                                 #Optional / Default
value is undefined
     mgmt_ipv6_cidr           => $cidr,                                     #Optional / Default
value is undefined
     mgmt_ipv6_dscp           => $uint,                                     #Optional / Default
value is undefined
     mgmt_ipv6_gateway        => $ipv6addr,                                 #Optional / Default
value is undefined
     mgmt_ipv6_enable_auto_config => "true" | "false",                      #Optional / Default
value is "false"
     mgmt_mask                => $netmask,                                  #Optional / Default
value is undefined
     mgmt_node2_port_duplex   => "Half" | "Full" | undef,                   #Optional / Default
value is undefined
     mgmt_node2_port_speed    => 10 | 100 | 1000 | undef,                   #Optional / Default
value is undefined
     mgmt_port_duplex         => "Half" | "Full" | undef,                   #Optional / Default
value is undefined
     mgmt_port_speed          => 10 | 100 | 1000 | undef,                   #Optional / Default
value is undefined
     mgmt_port                => "true" | "false",                          #Optional / Default
value is "false"
     mgmt_reserved_interface => $interface,                                 #Optional
     nat_enabled              => "true" | "false",                          #Optional / Default
value is "false"
     nat_group                => $string | undef,                           #Optional / Default
value is empty
     nat_ip_address           => $ipv4addr | undef,                         #Optional / Default
value is undefined

     nic_failover_enabled         => "true" | "false",                         #Optional / Default
value is "false"
     nic_failover_enable_primary => "true" | "false",                          #Optional / Default
value is "false"
     node1_ha                 => $ipv4addr,                                 #Optional / Default
value is undefined
     node1_ha_reserved_interface => $interface,                             #Optional
     node1_hwtype             => "IB-RSP" | ""                              #Optional / Default
value is ""
     node1_lan                => $ipv4addr,                                 #Optional / Default
value is undefined
     node1_lan_ipv6addr       => $ipv6addr,                                 #Optional / Default
value is undefined
     node1_lan_reserved_interface  => $interface,                           #Optional
     node1_lan2_reserved_interface => $interface,                           #Optional
     node1_lcd_autodim        => $num,                                      #Optional / Default
value is undefined
     node1_lcd_brightness     => $num,                                      #Optional / Default
value is undefined
     node1_lcd_hwident        => "true" | "false",                          #Optional / Default
value is undefined
     node1_mgmt_dscp          => $uint,                                     #Optional / Default
value is undefined
     node1_mgmt_gateway       => $ipv4addr,                                 #Optional / Default
value is undefined
     node1_mgmt_lan           => $ipv4addr,                                 #Optional / Default
value is undefined
     node1_mgmt_ipv6addr      => $ipv6addr,                                 #Optional / Default
value is undefined
     node1_mgmt_ipv6_cidr     => $cidr,                                     #Optional / Default
value is undefined
     node1_mgmt_ipv6_dscp     => $uint,                                     #Optional / Default
value is undefined
     node1_mgmt_ipv6_gateway => $ipv6addr,                                  #Optional / Default
value is undefined
     node1_mgmt_ipv6_enable_auto_config => "true" | "false",                #Optional / Default
value is "false"
     node1_mgmt_mask          => $netmask,                                  #Optional / Default
value is undefined
     node1_mgmt_port          => "true" | "false",                          #Optional / Default
value is undefined
     node1_mgmt_reserved_interface => $interface,                           #Optional
```

```
     node1_nat               => $ipv4addr | undef,                                  #Optional / Default
value is undefined
     node1_vpn_on_mgmt       => "true" | "false",                                   #Optional / Default
value is undefined
     node2_ha                => $ipv4addr,                                          #Optional / Default
value is undefined
     node2_ha_reserved_interface => $interface,                                     #Optional
     node2_hwtype            => ""                                                  #Optional / Default
value is ""
     node2_lan               => $ipv4addr,                                          #Optional / Default
value is undefined
     node2_lan_ipv6addr      => $ipv6addr,                                          #Optional / Default
value is undefined
     node2_lan_reserved_interface  => $interface,                                   #Optional
     node2_lan2_reserved_interface => $interface,                                   #Optional
     node2_mgmt_dscp         => $uint,                                              #Optional / Default
value is undefined
     node2_mgmt_ipv6addr     => $ipv6addr,                                          #Optional / Default
value is undefined
     node2_mgmt_ipv6_cidr    => $cidr,                                              #Optional / Default
value is undefined
     node2_mgmt_ipv6_dscp    => $uint,                                              #Optional / Default
value is undefined
     node2_mgmt_ipv6_gateway => $ipv6addr,                                          #Optional / Default
value is undefined
     node2_mgmt_ipv6_enable_auto_config => "true" | "false",                        #Optional / Default
value is "false"
     node2_mgmt_reserved_interface      => $interface,                              #Optional
     node2_lcd_autodim       => $num,                                               #Optional / Default
value is undefined
     node2_lcd_brightness    => $num,                                               #Optional / Default
value is undefined
     node2_lcd_hwident       => "true" | "false",                                   #Optional / Default
value is undefined
     node2_mgmt_gateway      => $ipv4addr,                                          #Optional / Default
value is undefined
     node2_mgmt_lan          => $ipv4addr,                                          #Optional / Default
value is undefined
     node2_mgmt_mask         => $netmask,                                           #Optional / Default
value is undefined
     node2_mgmt_port         => "true" | "false",                                   #Optional / Default
value is undefined
     node2_nat               => $ipv4addr | undef,                                  #Optional / Default
value is undefined
     node2_vpn_on_mgmt       => "true" | "false",                                   #Optional / Default
value is undefined
     notification_email_addr => $email,                                             #Optional / Default
value is undefined
     ntp_access_list         => [$ntpaccess1, $ntpaccess2, ... ] | $nacl | undef,  #Optional / Default
value is undefined
     ntp_service_type        => "Time" | "Time and NTP control",                    #Optional / Default
value is "Time"
     ntp_authentication_key  => [$NTPKey1, $NTPKey2, ...],                          #Optional / Default
value is undefined
     ntp_server              => [$ntp_server1, $ntp_server2] | undef,               #Optional / Default
value is undefined
     ntp_kod                 => "true" | "false",                                   #Optional / Default
value is undefined
     override_ntp_kod        => "true" | "false",                                   #Optional / Default
value is undefined
     ospf_list               => [$OSPF1, $OSPF2, ...],                              #Optional / Default
value is empty
     override_dscp           => "true" | "false",
#Optional / Default value is "false"
     override_enable_member_redirect  => "true" | "false",                          #Optional / Default
value is "false"
     override_lan1_dscp            => "true" | "false",                             #Optional / Default
value is "false"
     override_lan1_ipv6_dscp       => "true" | "false",                             #Optional / Default
value is "false"
     override_lan2_dscp            => "true" | "false",                             #Optional / Default
value is "false"
     override_lan2_ipv6_dscp       => "true" | "false",                             #Optional / Default
value is "false"
     override_mgmt_dscp            => "true" | "false",                             #Optional / Default
value is "false"
     override_mgmt_ipv6_dscp       => "true" | "false",                             #Optional / Default
value is "false"
     override_node1_mgmt_dscp      => "true" | "false",                             #Optional / Default
value is "false"
     override_node1_mgmt_ipv6_dscp => "true" | "false",                             #Optional / Default
value is "false"
     override_node2_mgmt_dscp      => "true" | "false",                             #Optional / Default
value is "false"
```

```
     override_node2_mgmt_ipv6_dscp => "true" | "false",                    #Optional / Default
value is "false"
     override_syslog_backup_servers => "true" | "false",                   #Optional / Default
value is "false"
     override_threshold_traps      => "true" | "false",                    #Optional / Default
value is "false"
     override_trap_notifications   => "true" | "false",                    #Optional / Default
value is "false"
     override_enable_lom      => "true" | "false",
#Optional / Default value is undefined
     override_lom_users       => "true" | "false",                         #Optional / Default
value is undefined
     passive_ha_arp_enabled   => "true" | "false",                         #Optional / Default
value is "false"
     platform                 => "Infoblox" | "Cisco" | "Riverbed" | "VNIOS",      #Optional / Default
value is "Infoblox"
     prefer_resolver          => $Nameserver | $Member | $ipv4addr | undef,        #Optional / Default
value is undefined
     query_comm_string        => $string | undef,                          #Optional / Default
value is undefined
     remote_console_access    => "true" | "false" | undef,                 #Optional / Default
value is undefined
     router_ID                => $num,                                     #Optional / Default
value is undefined
     service_type_configuration => "ALL_V4", "ALL_V6", "CUSTOM"            #Optional / Default
value depends from the member type.
     snmp_admin               => $Admin | undef,                           #Optional / Default
value is undefined
     static_routes            => [$Route1, $Route2, ...],                  #Optional / Default
value is empty
     support_access           => "true" | "false" | undef,                 #Optional / Default
value is undefined
     syslog_proxy_tcp_port    => $num,                                     #Optional / Default
value is 514
     syslog_proxy_udp_port    => $num,                                     #Optional / Default
value is 514
     syslog_server            => [$SyslogServer, ...] | undef,             #Optional / Default
value is undefined
     syslog_backup_servers    => [$SyslogBackupServer, ...] | undef,       #Optional / Default
value is undefined
     syslog_size              => $num,                                     #Optional / Default
value is 300
     time_zone                => $TimeZone | undef,                        #Optional / Default
value is undefined
     threshold_traps          => [$trap1, $trap2,...] | undef,             #Optional / Default
value is a list of all the traps
     trap_comm_string         => $string | undef,                          #Optional / Default
value is undefined
     trap_notifications       => [$notification1, $notification2,...] | undef,     #Optional / Default
value is a list of all the notifications
     trap_receiver            => [$ipv4addr, ...] | undef,                 #Optional / Default
value is undefined
     upgrade_group            => $string,                                  #Optional / Default
value is undefined
     vlan_id                  => $uint,                                    #Optional / Default
value is undefined
     vpn_mtu                  => $num,                                     #Optional / Default
value is 1450
     vpn_on_mgmt              => "true" | "false",                         #Optional / Default
value is "false"
     use_v4_vrrp              => "true" | "false",                         #Optional / Default
value depends from the member type.
     pre_provisioning         => $pp,                                      #optional / Default
value is undef. $pp is an Infoblox::Grid:Member::PreProvisioning object
     preserve_if_owns_delegation => "true" | "false"                       #Optional / Default
value is "false"
      );
```

You cannot set both extattrs and extensible_attributes attributes at the same time.

SESSION METHODS

This section describes all the methods in an Infoblox::Session module that you can apply to a grid member object.

Infoblox::Session->add( )

Use this method to add an object to the Infoblox appliance. See Infoblox::Session->add() for parameters and return values.

If you have applied an access control list to an object, you must restart services after you update the access control list for the changes to take effect. See Infoblox::Session->restart() method.

Example

```
#Construct an object
my $grid_member = Infoblox::Grid::Member->new(
    name                    => "hostname.com",
    ipv4addr                => "3.0.0.2",
    mask                    => "255.0.0.0",
    gateway                 => "3.0.0.1",
    );
#Submit for addition
my $response = $session->add( $grid_member );
```

Infoblox::Session->get( )

Use this method to retrieve all the matching objects from the Infoblox appliance. See Infoblox::Session->get() for parameters and return values.

Key References

```
Apply the following attributes to get a specific grid member object:

 name - Optional. A name in FQDN (Fully Qualified Domain Name) format.
 ipv4addr  - Optional. The IPv4 address of the member.
 ipv6addr  - Optional. The IPv6 address of the member.
 extattrs  - Optional. A hash reference containing extensible attributes.
```

Example

```
my @retrieved_objs = $session->get(
    object => "Infoblox::Grid::Member",
    name   => "hostname.com");
```

Infoblox::Session->modify( )

Use this method to modify an object in the Infoblox appliance. See Infoblox::Session->modify() for parameters and return values.

If you have applied an access control list to an object, you must restart services after you update the access control list for the changes to take effect. See Infoblox::Session->restart() method.

Example

```
# Use this method to modify comment of grid member.
$grid_member->comment("This is modified object");
# Submit modification
my $response = $session->modify( $grid_member );
```

Infoblox::Session->remove( )

Use this method to remove an object from the Infoblox appliance. See Infoblox::Session->remove() for parameters and return values.

To remove a specific object, first use `get()` or `search()` to retrieve the specific object, and then submit this object for removal.

Example

```
#Get the objects with the same name
my @retrieved_objs = $session->get(
    object => "Infoblox::Grid::Member",
    name   => "hostname.com");
#Find the desired object from the retrieved list.
my $desired_gridmember = $retrieved_objs[0];
# Submit for removal
my $response = $session->remove( $desired_gridmember );
```

Infoblox::Session->search( )

Use this method to search for grid member objects in the Infoblox appliance. See Infoblox::Session->search() for parameters and return values.

Key References

```
Apply the following attributes to search for a grid member object:

 name    - Optional. A name in FQDN (Fully Qualified Domain Name) format (regular expression).
 comment - Optional. A comment in string format (regular expression).
 extattrs     - Optional. A hash reference containing extensible attributes.
 extensible_attributes   - Optional. A hash reference containing extensible attributes.
 ipv4addr  - Optional. The IPv4 address of the member.
 ipv6addr  - Optional. The IPv6 address of the member.
```

For more information about searching extensible attributes, see Infoblox::Grid::ExtensibleAttributeDef/Searching Extensible Attributes.

Example

```
# search for all grid member objects
my @retrieved_objs = $session->search(
    object => "Infoblox::Grid::Member",
    name   => ".*\\.com");
```

METHODS

This section describes all the methods that you can use to configure and retrieve the attribute values of a grid member.

active_position( )

Use this method to retrieve the active server of a grid member.

Parameter

None

Returns

The method returns the attribute value.

Example

```
#Get active server
my $active_server = $grid_member->active_position();
```

additional_ip_list( )

Use this method to set or retrieve the additional_ip_list of a grid member. This list contains additional interface information that can be used on the member level.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Any object in the list with interface set to "LAN_HA" and ipv6 parameters will be mapped to the toplevel ipv6 methods (ipv6addr, ipv6_cidr, ipv6_enable_auto_config and ipv6_gateway). If such an object is already in the list and is removed, the values of those methods will be cleared.

Parameter

Valid value is an array reference that contains Infoblox::Grid::Member::Interface `object(s)` or IPv4 addresses in string format.

Note that Infoblox::Grid::Member::Interface `object(s)` with interface set to "MGMT" are not supported.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get additional ip list
my $additional_ip_list = $grid_member->additional_ip_list();
#Modify additional ip list
my $interface_obj = Infoblox::Grid::Member::Interface->new(
        subnet_mask => "255.255.255.255",
        anycast     => "false",
        comment     => "advanced ip config",
        ipv4addr    => "4.3.2.1",
        interface   => "LOOPBACK",
);
$grid_member->additional_ip_list([$interface_obj]);
```

allow_syslog_clients( )

Use this method to set or retrieve the allow_syslog_clients list of a grid member. This list specifies the clients where the syslog messages are logged. This is required only when enable_syslog_proxy is set to "true".

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

Valid value is an Infoblox::Grid::NamedACL object or an array reference that contains IPv4/IPv6 addresses or networks. To undefine allow_syslog_clients, you must disable the enable_syslog_proxy flag.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get allow syslog clients
my $allow_syslog_clients = $grid_member->allow_syslog_clients();
#Modify allow syslog clients
$grid_member->allow_syslog_clients(["10.10.10.1", "10.10.2.0/24"]);
#Use Infoblox::Grid::NamedACL
$grid_member->allow_syslog_clients($nacl);
```

alternate_resolver( )

Use this method to set or retrieve the alternate_resolver of a grid member. The alternate resolver receives queries from the appliance if it does not receive a response from the preferred name server first.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

The valid values are Infoblox::DNS::Member object or Infoblox::DNS::Nameserver object.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get alternate resolver
my $alternate_resolver = $grid_member->alternate_resolver();
#Modify alternate resolver
my $member1 = Infoblox::DNS::Member->new(
    name           => $host_name,
    ipv4addr       => $host_ip,
    stealth        => "false",
    lead           => "false",
    grid_replicate => "false"
);
$grid_member->alternate_resolver($member1);
#Undef alternate resolver
$grid_member->alternate_resolver(undef);
```

bgp_as( )

Use this method to set or retrieve the BGP configuration for anycast, of a grid member.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

The valid value is an array reference that contains Infoblox::Grid::Member::BGP::AS object(s). Currently, only one Autonomous System (AS) object per grid member is supported.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get AS list
my $as = $grid_member->bgp_as();
#Modify AS list
my $as1 = Infoblox::Grid::Member::BGP::AS->new(
    as             => 269,
    keepalive      => 4,
    holddown       => 16,
    neighbors      => [ $neighbor ]
);
```

```
$grid_member->bgp_as([$as1]);
```

comment( )

Use this method to set or retrieve a descriptive comment.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

Desired comment in string format with a maximum of 256 bytes.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get comment
my $comment = $grid_member->comment();
#Modify comment
$grid_member->comment("Modified grid member");
```

config_addr_type( )

Use this method to set or retrieve the configuration type for the address.

Parameter

The valid value is one of the following: "IPv4", "IPv6" or "BOTH".

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get config_addr_type
my $config_addr_type = $grid_member->config_addr_type();
#Modify config_addr_type
$grid_member->config_addr_type("IPv4");
```

dscp( )

Use this method to set or retrieve the DSCP value.

Setting this method to a defined value implicitly sets the override_dscp method to "true". Setting the parameter to undefined causes the appliance to use the grid default and automatically resets the override_dscp attribute to "false".

Note that when dscp is set to a defined value and override_ddscp is set to "false", the last operation takes precedence. Thus the sequence $object->dscp(11); $object->override_dscp("false"); will set override_dscp to "false", and the sequence $object->override_dscp("false"); $object->dscp(11); will result in override_dscp="true".

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

Valid values are integers between 0 and 63 inclusive. The default value is 0.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get the DSCP value from the member
my $users = $grid_member->dscp();
#Modify the DSCP value
$grid_member->dscp(11);
```

enable_ha( )

Use this method to set or retrieve the HA state of a member. It indicates whether a member is a single appliance or an HA pair.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

Specify "true" to set the enable_ha flag or "false" to deactivate/unset it. The default value is "false". If set to "false", then all the "node1_xxxx" and "node2_xxxx" attributes but node1_service_status and node1_hwtype are disabled, and "get" does not return those attributes.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get enable_ha from the member
my $enable_ha = $grid_member->enable_ha();
#Modify enable_ha
$grid_member->enable_ha("true");
```

enable_lom( )

Use this method to set or retrieve the flag that controls whether the LOM functionality is enabled.

Setting this method to a defined value implicitly sets the override_enable_lom method to "true". Setting the parameter to undefined causes the appliance to use the grid default and automatically resets the override_enable_lom attribute to "false".

Note that when enable_lom is set to a defined value and override_denable_lom is set to "false", the last operation takes precedence. Thus the sequence $object->enable_lom("true"); $object->override_enable_lom("false"); will set override_enable_lom to "false", and the sequence $object->override_enable_lom("false"); $object->enable_lom("true"); will result in override_enable_lom="true".

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

Specify "true" to set the enable_lom flag or "false" to deactivate/unset it. The default value is "false".

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get enable_lom from the member
my $lom = $grid_member->enable_lom();
#Modify enable_lom
$grid_member->enable_lom("true");
```

enable_member_redirect( )

Use this method to set or retrieve the flag that controls whether the member will redirect GUI connections to the grid master.

Setting this method to a defined value implicitly sets the override_enable_member_redirect method to "true". Setting the parameter to undefined causes the appliance to use the grid default and automatically resets the override_enable_member_redirect attribute to "false".

Note that when enable_member_redirect is set to a defined value and override_denable_member_redirect is set to "false", the last operation takes precedence. Thus the sequence $object->enable_member_redirect("true"); $object->override_enable_member_redirect("false"); will set override_enable_member_redirect to "false", and the sequence $object->override_enable_member_redirect("false"); $object->enable_member_redirect("true"); will result in override_enable_member_redirect="true".

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

Specify "true" to set the enable_member_redirect flag or "false" to deactivate/unset it. The default value is "false".

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get enable_member_redirect from the member
my $redirect = $grid_member->enable_member_redirect();
#Modify type of member
$grid_member->enable_member_redirect("true");
```

enable_ntp( )

Use this method to set or retrieve the enable_ntp flag of a grid member. Network Time Protocol (NTP) is a protocol designed to synchronize the clocks of computers over a network. This method enables/disables NTP service on the grid member.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

Specify "true" to set the enable_ntp flag or "false" to deactivate/unset it. The default value is "false".

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get enable ntp flag
my $enable_ntp = $grid_member->enable_ntp();
#Modify enable ntp flag
$grid_member->enable_ntp("true");
```

enable_snmpv3_query( )

Use this method to enable or disable SNMPv3 queries.

Parameter

Specify "true" to enable SNMPv3 queries or "false" to disable it.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

Example

```
#Get enable_snmpv3_query
my $enable_snmpv3_query = $gridmember->enable_snmpv3_query();
#Modify enable_snmpv3_query
$gridmember->enable_snmpv3_query("false");
```

enable_snmpv3_traps( )

Use this method to enable or disable SNMPv3 traps.

Parameter

Specify "true" to enable SNMPv3 traps or "false" to disable it. Default value is "false".

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

Example

```
#Get enable_snmpv3_traps
my $enable_snmpv3_traps = $gridmember->enable_snmpv3_traps();
#Modify enable_snmpv3_traps
$gridmember->enable_snmpv3_traps("true");
```

enable_ro_api_access( )

Use this method to enable or disable read-only API access for the Grid Master candidate.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

Specify "true" to allow read-only API access or "false" to forbid it. The default value is "false".

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get enable syslog proxy flag
my $enable_ro_api_access = $grid_member->enable_ro_api_access();
#Modify enable syslog proxy flag
$grid_member->enable_ro_api_access("true");
```

enable_syslog_proxy( )

Use this method to set or retrieve the enable_syslog_proxy flag of a grid member. When enabled, the member receives syslog messages from specified devices, such as syslog servers and routers, and then forwards these messages to an external syslog server.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

Specify "true" to set the enable_syslog_proxy flag or "false" to deactivate/unset it. The default value is "false".

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get enable syslog proxy flag
my $enable_syslog_proxy = $grid_member->enable_syslog_proxy();
#Modify enable syslog proxy flag
$grid_member->enable_syslog_proxy("true");
```

external_ntp_servers_enabled( )

Use this method to set or retrieve the external_ntp_servers_enabled flag of a grid member.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

When this flag is set to "false", the member does not use the configured list of NTP (Network Time Protocol) servers.

When this flag is set to "true", the member uses the configured list of NTP servers. If no NTP servers are configured on the member, it uses the NTP servers configured on the grid.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get external_ntp_servers_enabled flag
my $external_ntp_servers_enabled= $grid_member->external_ntp_servers_enabled();
#Modify external_ntp_servers_enabled flag
$grid_member->external_ntp_servers_enabled("true");
```

exclude_grid_master_as_ntp_server( )

Use this method to set or retrieve the exclude_grid_master_as_ntp_server flag of a grid member.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

When this flag is set to "true", the member does not use the grid master as an NTP (Network Time Protocol) time source. The default value of this parameter is "false", which means that by default the Grid Master is used as one of the time sources for Grid Member.

Note that setting this flag to "true" may have a detrimental effect on the database integrity of the member.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get exclude_grid_master_as_ntp_server flag
my $exclude_grid_master_as_ntp_server= $grid_member->exclude_grid_master_as_ntp_server();
#Modify exclude_grid_master_as_ntp_server flag
$grid_member->exclude_grid_master_as_ntp_server("true");
```

extattrs( )

Use this method to set or retrieve the extensible attributes associated with a Grid Member object.

Parameter

Valid value is a hash reference containing the names of extensible attributes and their associated values (Infoblox::Grid::Extattr objects).

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get extattrs
my $ref_extattrs = $grid_member->extattrs();
#Modify extattrs
$grid_member->extattrs({ 'Site' => $extattr1, 'Administrator' => $extattr2 });
```

extensible_attributes( )

Use this method to set or retrieve the extensible attributes associated with a Grid Member object

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

For valid values for extensible attributes, see Infoblox::Grid::ExtensibleAttributeDef/Extensible Attribute Values.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get extensible attributes
my $ref_extensible_attributes = $member->extensible_attributes();
#Modify extensible attributes
$member->extensible_attributes({ 'Site' => 'Santa Clara', 'Administrator' => [ 'Peter', 'Tom' ] });
```

gateway( )

Use this method to set or retrieve the gateway IP address of a grid member.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

An IPv4 address is a 32-bit number in dotted decimal notation. It consists of four 8-bit groups of decimal digits separated by decimal points (example: 192.168.1.2).

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get gateway
my $gateway = $grid_member->gateway();
#Modify gateway
$grid_member->gateway("3.0.0.1");
```

ha_node2_port_duplex( )

Use this method to specify or retrieve the duplex setting of the HA port on node 2 of an HA grid member. Note: Use this method in combination with ha_node2_port_speed. If any of these attributes are undefined, the default values for duplex and speed settings are used.

Parameter

Valid values are "Half" or "Full". The default value is undefined.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get ha_node2_port_duplex
my $ha_node2_port_duplex = $obj->ha_node2_port_duplex();
#Modify ha_node2_port_duplex
$obj->ha_node2_port_duplex("Half");
```

ha_node2_port_speed( )

Use this method to set or retrieve the connection speed of the HA port on node #2 of an HA grid member. Note: Use this method in combination with ha_node2_port_duplex. If any of these attributes are undefined, the default values for duplex and speed settings are used.

Parameter

Valid values are 10, 100 or 1000. The default value is undefined.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get ha_node2_port_speed
my $ha_node2_port_speed = $obj->ha_node2_port_speed();
#Modify ha_node2_port_speed
$obj->ha_node2_port_speed(10);
```

ha_port_duplex( )

Use this method to specify or retrieve the duplex setting of the HA port on node 1 of an HA grid member. Note: Use this method in combination with ha_port_speed. If any of these attributes are undefined, the default values for duplex and speed settings are used.

Parameter

Valid values are "Half" or "Full". The default value is undefined.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get ha_port_duplex
my $ha_port_duplex = $obj->ha_port_duplex();
#Modify ha_port_duplex
$obj->ha_port_duplex("Half");
```

ha_port_speed( )

Use this method to set or retrieve the connection speed of the HA port on node 1 of an HA grid member. Note: Use this method in combination with ha_port_duplex. If any of these attributes are undefined, the default values for duplex and speed settings are used.

Parameter

Valid values are 10, 100 or 1000. The default value is undefined.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get ha_port_speed
my $ha_port_speed = $obj->ha_port_speed();
#Modify ha_port_speed
$obj->ha_port_speed(10);
```

ha_reserved_interface( )

Use this method to set or retrieve the discovery reserved interface of the HA port on node 1 of an HA grid member.

Parameter

Valid value is an Infoblox::Grid::Discovery::DeviceInterface object.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get attribute value
my $attr = $grid_member->ha_reserved_interface();
#Modify attribute value
$grid_member->ha_reserved_interface($interface);
```

paid_nios( )

Use this method to retrieve whether the grid member is Paid NIOS.

Parameter

None

Returns

The method returns the attribute value.

Example

```
#Get Paid NIOS status
my $paid_nios = $grid_member->paid_nios();
```

hwplatform( )

Use this method to retrieve the hardware platform of the member appliance.

Parameter

None

Returns

The method returns the attribute value.

Example

```
#Get hardware platform
my $hwplatform = $grid_member->hwplatform();
```

hwid( )

Use this method to retrieve the hardware ID of the member appliance.

Parameter

None

Returns

The method returns the attribute value.

Example

```
#Get hardware ID
my $hwid = $grid_member->hwid();
```

is_dscp_capable( )

Use this method to retrieve the flag that describes whether a Grid member supports DSCP.

Parameter

The method retruns "true" if the member supports DSCP, and "false" otherwise.

Returns

The method returns the attribute value.

Example

```
#Get is_dscp_capable
my $is_dscp_capable = $grid_member->is_dscp_capable();
```

ipv4addr( )

Use this method to set or retrieve the IPv4 address of a grid member.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

An IPv4 address is a 32-bit number in dotted decimal notation. It consists of four 8-bit groups of decimal digits separated by decimal points (example: 192.168.1.2).

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get ipv4addr
my $ipv4addr = $grid_member->ipv4addr();
#Modify ipv4addr
$grid_member->ipv4addr("3.0.0.3");
```

ipv6addr( )

Use this method to set or retrieve the LAN IPv6 address of the grid member.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Setting any of the ipv6 methods in the member (ipv6addr, ipv6_cidr, ipv6_enable_auto_config and ipv6_gateway) is equivalent to creating an Infoblox::Grid::Member::Interface object with those fields set and interface set to "LAN_HA", if there is already such an object it will be modified, if not it will be created. See the additional_ip_list section for more information.

Parameter

An IPv6 address is a 128-bit number represented as eight groups of four hexadecimal digits (i.e. two octets), each group separated by a `colon(:)` (example: 2001:0db8:85a3:0000:0000:8a2e:0370:7334). Leading zeroes and groups of consecutive zeros within the value can be omitted to produce a more compact representation (example: 2001::1, ::1).

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get LAN2 IPv6 address
my $ipv6addr = $grid_member->ipv6addr();
#Modify LAN2 IPv6 address
$grid_member->ipv6addr("2001::2");
```

ipv6_cidr( )

Use this method to set or retrieve the CIDR of the grid member. Required only when ipv6addr is specified.

See ipv6addr for more information on how this impacts additional_ip_list.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

Specify the netmask in CIDR format.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get LAN2 cidr
my $cidr = $grid_member->cidr();
#Modify LAN2 cidr
$grid_member->cidr("64");
```

ipv6_enable_auto_config( )

Use this method to enable/disable IPv6 auto-configuration of the grid member.

See ipv6addr for more information on how this impacts additional_ip_list.

When IPv6 auto-configuration is enabled, a gateway is not needed since it will be automatically discovered.

Parameter

Specify "true" to enable IPv6 auto-configuration or "false" to disable it. Default value is "false".

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

Example

```
#Get IPv6 auto-configuration
my $ipv6_autoconfig_enabled = $grid_member->ipv6_enable_auto_config();
#Modify IPv6 auto-configuration
$grid_member->ipv6_enable_auto_config("true");
```

ipv6_gateway( )

Use this method to set or retrieve the LAN IPv6 gateway of the grid member.

See ipv6addr for more information on how this impacts additional_ip_list.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

An IPv6 address is a 128-bit number represented as eight groups of four hexadecimal digits (i.e. two octets), each group separated by a `colon(:)` (example: 2001:0db8:85a3:0000:0000:8a2e:0370:7334). Leading zeroes and groups of consecutive zeros within the value can be omitted to produce a more compact representation (example: 2001::1, ::1).

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get LAN2 IPv6 gateway
my $ipv6_gateway = $grid_member->ipv6_gateway();
#Modify LAN2 IPv6 gateway
$grid_member->ipv6_gateway("2001::1");
```

ipv6_static_routes( )

Use this method to set or retrieve the IPv6 static routes list of a grid member. This list contains the static routes of a grid member.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

Valid value is an array reference that contains Infoblox::Grid::Member::IPv6StaticRoute object(s).

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get static routes list
my $ipv6_static_routes = $grid_member->ipv6_static_routes();
#Modify static routes list
my $route = Infoblox::Grid::Member::IPv6StaticRoute->new(
        "address" => "2002::5",
        "cidr"    => 64
        "gateway" => "2002::1",
);
$grid_member->ipv6_static_routes([$route]);
```

ipv6_vlan_id( )

Use this method to set or retrieve the IPv6 VLAN ID of a grid member.

Parameter

Valid values are unsigned integers between 1 and 4095, inclusive. The default value is undefined.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get attribute value
my $attr = $grid_member->ipv6_vlan_id();
#Modify attribute value
$grid_member->ipv6_vlan_id(10);
```

lan_reserved_interface( )

Use this method to set or retrieve the discovery reserved interface of the LAN port on a single grid member or on node 1 of an HA grid member.

Parameter

Valid value is an Infoblox::Grid::Discovery::DeviceInterface object.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get attribute value
my $attr = $grid_member->lan_reserved_interface();
#Modify attribute value
$grid_member->lan_reserved_interface($interface);
```

lan1_dscp( )

Use this method to set or retrieve the DSCP value of the IPv4 LAN1 port.

Setting this method to a defined value implicitly sets the override_lan1_dscp method to "true". Setting the parameter to undefined causes the appliance to use the grid default and automatically resets the override_lan1_dscp attribute to "false".

Note that when dscp is set to a defined value and override_lan1_dscp is set to "false", the last operation takes precedence. Thus the sequence $object->lan1_dscp(11); $object->override_lan1_dscp("false"); will set override_lan1_dscp to "false", and the sequence $object->override_lan1_dscp("false"); $object->lan1_dscp(11); will result in override_lan1_dscp="true".

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

Valid values are integers between 0 and 63, inclusive. The default value is undefined.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get attribute value
my $value = $grid_member->lan1_dscp();
#Modify attribute value
$grid_member->lan1_dscp(11);
```

lan1_ipv6_dscp( )

Use this method to set or retrieve the DSCP value of the IPv6 LAN1 port.

Setting this method to a defined value implicitly sets the override_lan1_ipv6_dscp method to "true". Setting the parameter to undefined causes the appliance to use the grid default and automatically resets the override_lan1_ipv6_dscp attribute to "false".

Note that when dscp is set to a defined value and override_lan1_ipv6_dscp is set to "false", the last operation takes precedence. Thus the sequence $object->lan1_ipv6_dscp(11); $object->override_lan1_ipv6_dscp("false"); will set override_lan1_ipv6_dscp to "false", and the sequence $object->override_lan1_ipv6_dscp("false"); $object->lan1_ipv6_dscp(11); will result in override_lan1_ipv6_dscp="true".

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

Valid values are integers between 0 and 63, inclusive. The default value is undefined.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get attribute value
my $value = $grid_member->lan1_ipv6_dscp();
#Modify attribute value
$grid_member->lan1_ipv6_dscp(11);
```

lan2_dscp( )

Use this method to set or retrieve the DSCP value of the IPv4 LAN2 port.

Setting this method to a defined value implicitly sets the override_lan1_dscp method to "true". Setting the parameter to undefined causes the appliance to use the grid default and automatically resets the override_lan1_dscp attribute to "false".

Note that when dscp is set to a defined value and override_lan2_dscp is set to "false", the last operation takes precedence. Thus the sequence $object->lan2_dscp(11); $object->override_lan2_dscp("false"); will set override_lan2_dscp to "false", and the sequence $object->override_lan2_dscp("false"); $object->lan2_dscp(11); will result in override_lan2_dscp="true".

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

Valid values are integers between 0 and 63, inclusive. The default value is undefined.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get attribute value
my $value = $grid_member->lan2_dscp();
#Modify attribute value
$grid_member->lan2_dscp(11);
```

lan2_gateway( )

Use this method to set or retrieve the gateway IP address of the LAN2 port. Note that to configure the LAN2 port, the lan2_port flag must

be set to 'true'.

lan2_ipv4addr, lan2_mask, lan2_gateway attributes are required if one of them is defined.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

An IPv4 address is a 32-bit number in dotted decimal notation. It consists of four 8-bit groups of decimal digits separated by decimal points (example: 192.168.1.2).

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get LAN2 gateway
my $lan2_gateway = $grid_member->lan2_gateway();
#Modify LAN2 gateway
$grid_member->lan2_gateway("4.0.0.1");
```

lan2_ipv4addr( )

Use this method to set or retrieve the LAN IP address of the LAN2 port. Note that to configure the LAN2 port, the lan2_port flag must be set to 'true'.

lan2_gateway, lan2_ipv4addr, lan2_mask attributes are required if one of them is defined.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

An IPv4 address is a 32-bit number in dotted decimal notation. It consists of four 8-bit groups of decimal digits separated by decimal points (example: 192.168.1.2).

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get LAN2 IPv4 address
my $lan2_ipv4addr = $grid_member->lan2_ipv4addr();
#Modify LAN2 IPv4 address
$grid_member->lan2_ipv4addr("4.0.0.2");
```

lan2_ipv6addr( )

Use this method to set or retrieve the LAN IPv6 address of the LAN2 port. Note that to configure the LAN2 port, the lan2_port flag must be set to 'true'.

lan2_ipv6addr, lan2_ipv6_cidr, lan2_ipv6_gateway attributes are required if one of them is defined.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

An IPv6 address is a 128-bit number represented as eight groups of four hexadecimal digits (i.e. two octets), each group separated by a `colon(:)` (example: 2001:0db8:85a3:0000:0000:8a2e:0370:7334). Leading zeroes and groups of consecutive zeros within the value can be omitted to produce a more compact representation (example: 2001::1, ::1).

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get LAN2 IPv6 address
my $lan2_ipv6addr = $grid_member->lan2_ipv6addr();
#Modify LAN2 IPv6 address
$grid_member->lan2_ipv6addr("2001::2");
```

lan2_ipv6_cidr( )

Use this method to set or retrieve the CIDR of the LAN2 port. Note that to configure the LAN2 port, the lan2_port flag must be set to 'true'.

lan2_ipv6addr, lan2_ipv6_cidr, lan2_ipv6_gateway attributes are required if one of them is defined.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

Specify the netmask in CIDR format.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get LAN2 cidr
my $lan2_cidr = $grid_member->lan2_cidr();
#Modify LAN2 cidr
$grid_member->lan2_cidr("64");
```

lan2_ipv6_dscp( )

Use this method to set or retrieve the DSCP value of the IPv6 LAN2 port.

Setting this method to a defined value implicitly sets the override_lan2_ipv6_dscp method to "true". Setting the parameter to undefined causes the appliance to use the grid default and automatically resets the override_lan2_ipv6_dscp attribute to "false".

Note that when dscp is set to a defined value and override_lan2_ipv6_dscp is set to "false", the last operation takes precedence. Thus the sequence $object->lan2_ipv6_dscp(11); $object->override_lan2_ipv6_dscp("false"); will set override_lan1_ipv6_dscp to "false", and the sequence $object->override_lan2_ipv6_dscp("false"); $object->lan2_ipv6_dscp(11); will result in override_lan2_ipv6_dscp="true".

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

Valid values are integers between 0 and 63, inclusive. The default value is undefined.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get attribute value
my $value = $grid_member->lan2_ipv6_dscp();
#Modify attribute value
$grid_member->lan2_ipv6_dscp(11);
```

lan2_ipv6_enable( )

Use this method to enable/disable IPv6 configuration of the LAN2 interface.

Parameter

Specify "true" to enable IPv6 configuration or "false" to disable it. Default value is "false".

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get attribute value
my $attr = $grid_member->lan2_ipv6_enable();
#Modify attribute value
$grid_member->lan2_ipv6_enable("true");
```

lan2_ipv6_enable_auto_config( )

Use this method to enable/disable IPv6 auto-configuration of the LAN2 interface.

When IPv6 auto-configuration is enabled, a gateway is not needed since it will be automatically discovered.

Parameter

Specify "true" to enable IPv6 auto-configuration or "false" to disable it. Default value is "false".

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get IPv6 auto-configuration
my $ipv6_autoconfig_enabled = $grid_member->lan2_ipv6_enable_auto_config();
#Modify IPv6 auto-configuration
$grid_member->lan2_ipv6_enable_auto_config("true");
```

lan2_ipv6_gateway( )

Use this method to set or retrieve the LAN IPv6 gateway of the LAN2 port. Note that to configure the LAN2 port, the lan2_port flag must be set to 'true'.

lan2_ipv6addr, lan2_ipv6_cidr, lan2_ipv6_gateway attributes are required if one of them is defined.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

An IPv6 address is a 128-bit number represented as eight groups of four hexadecimal digits (i.e. two octets), each group separated by a `colon(:)` (example: 2001:0db8:85a3:0000:0000:8a2e:0370:7334). Leading zeroes and groups of consecutive zeros within the value can be omitted to produce a more compact representation (example: 2001::1, ::1).

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get LAN2 IPv6 gateway
my $lan2_ipv6_gateway = $grid_member->lan2_ipv6_gateway();
#Modify LAN2 IPv6 gateway
$grid_member->lan2_ipv6_gateway("2001::1");
```

lan2_ipv6_vlan_id( )

Use this method to set or retrieve the VLAN ID of the IPv6 LAN2 port. Note that to configure the LAN2 port, the lan2_port flag must be set to 'true'.

Parameter

Valid values are unsigned integers between 1 and 4095, inclusive. The default value is undefined.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get attribute value
my $attr = $grid_member->lan2_ipv6_vlan_id();
#Modify attribute value
$grid_member->lan2_ipv6_vlan_id(10);
```

lan2_mask( )

Use this method to set or retrieve the netmask of the LAN2 port. Note that to configure the LAN2 port, the lan2_port flag must be set to 'true'.

lan2_ipv4addr, lan2_mask, lan2_gateway attributes are required if one of them is defined.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

Netmask address is a 32-bit number in dotted decimal notation. It consists of four 8-bit groups of decimal digits separated by decimal points (example: 255.255.0.0).

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get LAN2 mask
my $lan2_mask = $grid_member->lan2_mask();
#Modify LAN2 mask
$grid_member->lan2_mask("255.255.0.0");
```

lan2_node2_port_duplex( )

Use this method to specify or retrieve the duplex setting of the LAN2 port on node 2 of an HA grid member. Note: Use this method in combination with lan2_node2_port_speed. If any of these attributes are undefined, the default values for duplex and speed settings are used.

Parameter

Valid values are "Half" or "Full". The default value is undefined.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get lan2_node2_port_duplex
my $lan2_node2_port_duplex = $obj->lan2_node2_port_duplex();
#Modify lan2_node2_port_duplex
$obj->lan2_node2_port_duplex("Half");
```

lan2_port( )

Use this method to set or retrieve the LAN2 port flag of a single grid member or node 1 of an HA grid member.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

Specify "true" to set the lan2_port flag or "false" to deactivate/unset it. The default value is "false".

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get LAN2 port flag
my $lan2_port = $grid_member->lan2_port();
#Modify LAN2 port flag
$grid_member->lan2_port("true");
```

lan2_node2_port_speed( )

Use this method to set or retrieve the connection speed of the LAN2 port on node 2 of an HA grid member. Note: Use this method in combination with lan2_node2_port_duplex. If any of these attributes are undefined, the default values for duplex and speed settings are used.

Parameter

Valid values are 10, 100 or 1000. The default value is undefined.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get lan2_node2_port_speed
my $lan2_node2_port_speed = $obj->lan2_node2_port_speed();
#Modify lan2_node2_port_speed
$obj->lan2_node2_port_speed(10);
```

lan2_port_duplex( )

Use this method to specify or retrieve the duplex setting of the LAN2 port on a single grid member or on node 1 of an HA grid member. Note: Use this method in combination with lan2_port_speed. If any of these attributes are undefined, the default values for duplex and speed settings are used.

Parameter

Valid values are "Half" or "Full". The default value is undefined.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get lan2_port_duplex
my $lan2_port_duplex = $obj->lan2_port_duplex();
#Modify lan2_port_duplex
$obj->lan2_port_duplex("Half");
```

lan2_port_speed( )

Use this method to set or retrieve the connection speed of the LAN2 port on a single grid member or on node 1 of an HA grid member. Note: Use this method in combination with lan2_port_duplex. If any of these attributes are undefined, the default values for duplex and speed settings are used.

Parameter

Valid values are 10, 100 or 1000. The default value is undefined.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get lan2_port_speed
my $lan2_port_speed = $obj->lan2_port_speed();
#Modify lan2_port_speed
$obj->lan2_port_speed(10);
```

lan2_reserved_interface( )

Use this method to set or retrieve the discovery reserved interface of the LAN2 port on a single grid member or on node 1 of an HA grid member.

Parameter

Valid value is an Infoblox::Grid::Discovery::DeviceInterface object.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get attribute value
my $attr = $grid_member->lan2_reserved_interface();
#Modify attribute value
$grid_member->lan2_reserved_interface($interface);
```

lan2_router_id( )

Use this method to set or retrieve the LAN2 virtual router ID of a grid member. Required only when the member is an HA pair.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

A number with the LAN2 router identification. Valid values are from 1 through 255.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get LAN2 virtual router ID
my $lan2_router_id = $grid_member->lan2_router_id();
#Modify LAN2 virtual router ID
$grid_member->lan2_router_id("123");
```

lan2_vlan_id( )

Use this method to set or retrieve the VLAN ID of the IPv4 LAN2 port. Note that to configure the LAN2 port, the lan2_port flag must be set to 'true'.

Parameter

Valid values are unsigned integers between 1 and 4095, inclusive. The default value is undefined.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get attribute value
my $attr = $grid_member->lan2_vlan_id();
#Modify attribute value
$grid_member->lan2_vlan_id(10);
```

lan_node2_port_duplex( )

Use this method to specify or retrieve the duplex setting of the LAN port on node 2 of an HA grid member. Note: Use this method in combination with lan_node2_port_speed. If any of these attributes are undefined, the default values for duplex and speed settings are used.

Parameter

Valid values are "Half" or "Full". The default value is undefined.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get lan_node2_port_duplex
my $lan_node2_port_duplex = $obj->lan_node2_port_duplex();
#Modify lan_node2_port_duplex
$obj->lan_node2_port_duplex("Half");
```

lan_node2_port_speed( )

Use this method to set or retrieve the connection speed of the LAN port on node 2 of an HA grid member. Note: Use this method in combination with lan_node2_port_duplex. If any of these attributes are undefined, the default values for duplex and speed settings are used.

Parameter

Valid values are 10, 100 or 1000. The default value is undefined.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get lan_node2_port_speed
my $lan_node2_port_speed = $obj->lan_node2_port_speed();
#Modify lan_node2_port_speed
$obj->lan_node2_port_speed(10);
```

lan_port_duplex( )

Use this method to specify or retrieve the duplex setting of the LAN port on a single grid member or on node 1 of an HA grid member.. Note: Use this method in combination with lan_port_speed. If any of these attributes are undefined, the default values for duplex and speed settings are used.

Parameter

Valid values are "Half" or "Full". The default value is undefined.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get lan_port_duplex
my $lan_port_duplex = $obj->lan_port_duplex();
#Modify lan_port_duplex
$obj->lan_port_duplex("Half");
```

lan_port_speed( )

Use this method to set or retrieve the connection speed of the LAN port on a single grid member or on node 1 of an HA grid member.. Note: Use this method in combination with lan_port_duplex. If any of these attributes are undefined, the default values for duplex and speed settings are used.

Parameter

Valid values are 10, 100 or 1000. The default value is undefined.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get lan_port_speed
my $lan_port_speed = $obj->lan_port_speed();
#Modify lan_port_speed
$obj->lan_port_speed(10);
```

lcd_input( )

Use this method to set or retrieve the lcd_input flag of a grid member. This flag enables/disables the Liquid Crystal Display (LCD) input buttons on the front panel of the appliance.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

Specify "true" to set the lcd_input flag or "false" to deactivate/unset it. The default value is undefined.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get lcd input flag
```

```
my $lcd_input = $grid_member->lcd_input();
#Modify lcd input flag
$grid_member->lcd_input("true");
#Undef lcd input flag
$grid_member->lcd_input(undef);
```

lcd_autodim( )

Use this method to set or retrieve the number of seconds before the Liquid Crystal Display (LCD) on the front panel of the appliance automatically dims.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

The number of seconds before the LCD automatically dims. Valid values are between 5 and 3600. The default value is 30.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get lcd_autodim
my $lcd_autodim = $grid_member->lcd_autodim();
#Modify the autodim value
$grid_member->lcd_autodim(60);
```

lcd_brightness( )

Use this method to set or retrieve the brightness level of the LCD on the front panel of the appliance.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

Valid values are between 1 and 10. The default value is 7.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get lcd_brightness
my $lcd_brightness = $grid_member->lcd_brightness();
#Modify the brightness value
$grid_member->lcd_brightness(3);
```

lcd_hwident( )

Use this method to set or retrieve whether the grid member's hardware ID identification functionality is active.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

Specify "true" to set the lcd_hwident flag or "false" to deactivate/unset it. The default value is "false".

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get the lcd_hwident status
my $lcd_hwident = $grid_member->lcd_hwident();
#Modify the lcd_hwident status
$grid_member->lcd_hwident("true");
```

lcd_version( )

Use this method to retrieve the version of the Liquid Crystal Display (LCD) of the appliance.

Parameter

None.

Returns

The method returns the attribute value.

Example

```
#Get lcd_version
my $lcd_version = $grid_member->lcd_version();
```

lom_gateway( )

Use this method to set or retrieve the gateway IP address of a grid member for the LOM functionality.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

An IPv4 address is a 32-bit number in dotted decimal notation. It consists of four 8-bit groups of decimal digits separated by decimal points (example: 192.168.1.2).

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get lom_gateway
my $lom_gateway = $grid_member->lom_gateway();
#Modify lom_gateway
$grid_member->lom_gateway("3.0.0.1");
```

lom_ipv4addr( )

Use this method to set or retrieve the IPv4 address of a grid member for the LOM functionality.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

An IPv4 address is a 32-bit number in dotted decimal notation. It consists of four 8-bit groups of decimal digits separated by decimal points (example: 192.168.1.2).

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get lom_ipv4addr
my $lom_ipv4addr = $grid_member->lom_ipv4addr();
#Modify lom_ipv4addr
$grid_member->lom_ipv4addr("3.0.0.3");
```

lom_is_capable( )

Use this method to retrieve a flag describing if a grid member is capable of the LOM functionality.

Parameter

The method returns "true" if the member is capable of LOM operation, "false" otherwise.

Returns

The method returns the attribute value.

Example

```
#Get lom_is_capable
my $lom_is_capable = $grid_member->lom_is_capable();
```

lom_mask( )

Use this method to set or retrieve the netmask of a grid member for the LOM functionality.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

Netmask address is a 32-bit number in dotted decimal notation. It consists of four 8-bit groups of decimal digits separated by decimal points (example: 255.255.0.0).

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get lom_mask
my $lom_mask = $grid_member->lom_mask();
#Modify lom_mask
$grid_member->lom_mask("255.255.0.0");
```

lom_users( )

Use this method to set or retrieve the list of users for the LOM functionality.

Setting this method to a defined value implicitly sets the override_lom_users method to "true". Setting the parameter to undefined causes the appliance to use the grid default and automatically resets the override_lom_users attribute to "false".

Note that when lom_users is set to a defined value and override_dlom_users is set to "false", the last operation takes precedence. Thus the sequence $object->lom_users([$user1,$user2]); $object->override_lom_users("false"); will set override_lom_users to "false", and the sequence $object->override_lom_users("false"); $object->lom_users([$user1,$user2]); will result in override_lom_users="true".

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

Valid value is an array reference that contains Infoblox::Grid::LOM::User objects.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get lom_users from the member
my $users = $grid_member->lom_users();
#Modify the lom users
$grid_member->lom_users([$u1, $u2]);
```

mask( )

Use this method to set or retrieve the netmask of a grid member.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

Netmask address is a 32-bit number in dotted decimal notation. It consists of four 8-bit groups of decimal digits separated by decimal points (example: 255.255.0.0).

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get mask
my $mask = $grid_member->mask();
#Modify mask
$grid_member->mask("255.255.0.0");
```

master_candidate( )

Use this method to set or retrieve the master_candidate flag of a grid member. This flag enables the grid member to assume the role of grid master as a disaster recovery measure.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

Specify "true" to set the master_candidate flag or "false" to deactivate/unset it. The default value is "false".

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get master candidate flag
my $master_candidate = $grid_member->master_candidate();
#Modify master candidate flag
$grid_member->master_candidate("true");
```

member_service_communication( )

Use this method to set or retrieve the communication type for various services.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

The valid value is an array reference that contains Infoblox::Grid::Member::Communication objects.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
# Get the attribute value
my $attr = $grid_member->member_service_communication();
# Modify the attribute value
$grid_member->member_service_communication([$c1, $c2, $c3]);
```

mgmt_dscp( )

Use this method to set or retrieve the DSCP value of the IPv4 MGMT port.

Setting this method to a defined value implicitly sets the override_mgmt_dscp method to "true". Setting the parameter to undefined causes the appliance to use the grid default and automatically resets the override_mgmt_dscp attribute to "false".

Note that when dscp is set to a defined value and override_mgmt_dscp is set to "false", the last operation takes precedence. Thus the sequence $object->mgmt_dscp(11); $object->override_mgmt_dscp("false"); will set override_mgmt_dscp to "false", and the sequence $object->override_mgmt_dscp("false"); $object->mgmt_dscp(11); will result in override_mgmt_dscp="true".

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

Valid values are integers between 0 and 63, inclusive. The default value is undefined.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get attribute value
my $value = $grid_member->mgmt_dscp();
#Modify attribute value
$grid_member->mgmt_dscp(11);
```

mgmt_gateway( )

Use this method to set or retrieve the gateway IP address of the MGMT port.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

An IPv4 address is a 32-bit number in dotted decimal notation. It consists of four 8-bit groups of decimal digits separated by decimal points (example: 192.168.1.2).

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get mgmt gateway
my $mgmt_gateway = $grid_member->mgmt_gateway();
#Modify mgmt gateway
$grid_member->mgmt_gateway("4.0.0.1");
```

mgmt_lan( )

Use this method to set or retrieve the LAN IP address of the MGMT port.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

An IPv4 address is a 32-bit number in dotted decimal notation. It consists of four 8-bit groups of decimal digits separated by decimal points (example: 192.168.1.2).

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get mgmt lan
my $mgmt_lan = $grid_member->mgmt_lan();
#Modify mgmt lan
$grid_member->mgmt_lan("4.0.0.2");
```

mgmt_ipv6addr( )

Use this method to set or retrieve the LAN IPv6 address of the MGMT port.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

An IPv6 address is a 128-bit number represented as eight groups of four hexadecimal digits (i.e. two octets), each group separated by a `colon(:)` (example: 2001:0db8:85a3:0000:0000:8a2e:0370:7334). Leading zeroes and groups of consecutive zeros within the value can be omitted to produce a more compact representation (example: 2001::1, ::1).

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get MGMT IPv6 address
my $mgmt_ipv6addr = $grid_member->mgmt_ipv6addr();
#Modify MGMT IPv6 address
$grid_member->mgmt_ipv6addr("2001::2");
```

mgmt_ipv6_cidr( )

Use this method to set or retrieve the CIDR of the MGMT port. Required only when mgmt_ipv6addr is specified.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

Specify the netmask in CIDR format.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get MGMT cidr
my $mgmt_cidr = $grid_member->mgmt_cidr();
#Modify MGMT cidr
$grid_member->mgmt_cidr("64");
```

mgmt_ipv6_dscp( )

Use this method to set or retrieve the DSCP value of the IPv6 MGMT port.

Setting this method to a defined value implicitly sets the override_mgmt_ipv6_dscp method to "true". Setting the parameter to undefined causes the appliance to use the grid default and automatically resets the override_mgmt_ipv6_dscp attribute to "false".

Note that when dscp is set to a defined value and override_mgmt_ipv6_dscp is set to "false", the last operation takes precedence. Thus the sequence $object->mgmt_ipv6_dscp(11); $object->override_mgmt_ipv6_dscp("false"); will set override_mgmt_ipv6_dscp to "false", and the sequence $object->override_mgmt_ipv6_dscp("false"); $object->mgmt_ipv6_dscp(11); will result in override_mgmt_ipv6_dscp="true".

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

Valid values are integers between 0 and 63, inclusive. The default value is undefined.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get attribute value
my $value = $grid_member->mgmt_ipv6_dscp();
#Modify attribute value
$grid_member->mgmt_ipv6_dscp(11);
```

mgmt_ipv6_enable_auto_config( )

Use this method to enable/disable IPv6 auto-configuration of the MGMT interface.

When IPv6 auto-configuration is enabled, a gateway is not needed since it will be automatically discovered.

Parameter

Specify "true" to enable IPv6 auto-configuration or "false" to disable it. Default value is "false".

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

Example

```
#Get IPv6 auto-configuration
my $ipv6_autoconfig_enabled = $grid_member->mgmt_ipv6_enable_auto_config();
#Modify IPv6 auto-configuration
$grid_member->mgmt_ipv6_enable_auto_config("true");
```

mgmt_ipv6_gateway( )

Use this method to set or retrieve the LAN IPv6 gateway of the MGMT port.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

An IPv6 address is a 128-bit number represented as eight groups of four hexadecimal digits (i.e. two octets), each group separated by a `colon(:)` (example: 2001:0db8:85a3:0000:0000:8a2e:0370:7334). Leading zeroes and groups of consecutive zeros within the value can be omitted to produce a more compact representation (example: 2001::1, ::1).

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get MGMT IPv6 gateway
my $mgmt_ipv6_gateway = $grid_member->mgmt_ipv6_gateway();
#Modify MGMT IPv6 gateway
$grid_member->mgmt_ipv6_gateway("2001::1");
```

mgmt_mask( )

Use this method to set or retrieve the netmask of the MGMT port.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

Netmask address is a 32-bit number in dotted decimal notation. It consists of four 8-bit groups of decimal digits separated by decimal points (example: 255.255.0.0).

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get mgmt mask
my $mgmt_mask = $grid_member->mgmt_mask();
#Modify mgmt mask
$grid_member->mgmt_mask("255.255.0.0");
```

mgmt_node2_port_duplex( )

Use this method to specify the duplex setting of the MGMT port on node 2 of an HA pair. Note: Use this method in combination with mgmt_node2_port_speed. If any of these attributes are undefined, the default values for duplex and speed settings are used.

Parameter

Valid values are "Half" or "Full". The default value is undefined.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get mgmt_node2_port_duplex
my $mgmt_node2_port_duplex = $obj->mgmt_node2_port_duplex();
#Modify mgmt_node2_port_duplex
$obj->mgmt_node2_port_duplex("Half");
```

mgmt_node2_port_speed( )

Use this method to set the connection speed of the MGMT port on node 2 of an HA pair. Note: Use this method in combination with mgmt_node2_port_duplex. If any of these attributes are undefined, the default values for duplex and speed settings are used.

Parameter

Valid values are 10, 100 or 1000. The default value is undefined.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get mgmt_node2_port_speed
my $mgmt_node2_port_speed = $obj->mgmt_node2_port_speed();
#Modify mgmt_node2_port_speed
```

```
$obj->mgmt_node2_port_speed(10);
```

mgmt_port( )

Use this method to set or retrieve the MGMT port flag of a grid member. You can restrict administrative access to an Infoblox appliance by connecting the MGMT port to a subnet containing only management systems.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Note that node1_mgmt_port, node2_mgmt_port and mgmt_port share the value; setting any of them will have the same effect.

Parameter

Specify "true" to set the mgmt_port flag or "false" to deactivate/unset it. The default value is "false". To disable the mgmt_port flag, you must disable the "node1_mgmt_port" and "node2_mgmt_port" flags as well.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get MGMT port flag
my $mgmt_port = $grid_member->mgmt_port();
#Modify MGMT port flag
$grid_member->mgmt_port("true");
```

mgmt_port_duplex( )

Use this method to specify the duplex setting of the MGMT port on a single grid member or on node 1 of an HA grid member. Note: Use this method in combination with mgmt_port_speed. If any of these attributes are undefined, the default values for duplex and speed settings are used.

Parameter

Valid values are "Half" or "Full". The default value is undefined.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get mgmt_port_duplex
my $mgmt_port_duplex = $obj->mgmt_port_duplex();
#Modify mgmt_port_duplex
$obj->mgmt_port_duplex("Half");
```

mgmt_port_speed( )

Use this method to specify the connection speed of the MGMT port on a single grid member or on node 1 of an HA grid member. Note: Use this method in combination with mgmt_port_duplex. If any of these attributes are undefined, the default values for duplex and speed settings are used.

Parameter

Valid values are 10, 100 or 1000. The default value is undefined.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get mgmt_port_speed
my $mgmt_port_speed = $obj->mgmt_port_speed();
#Modify mgmt_port_speed
$obj->mgmt_port_speed(10);
```

mgmt_reserved_interface( )

Use this method to set or retrieve the discovery reserved interface of the MGMT port on a single grid member or on node 1 of an HA grid

member.

Parameter

Valid value is an Infoblox::Grid::Discovery::DeviceInterface object.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get attribute value
my $attr = $grid_member->mgmt_reserved_interface();
#Modify attribute value
$grid_member->mgmt_reserved_interface($interface);
```

mmdb_ea_build_time( )

Use this method to retrieve the extensible attributes MaxMind database build time. This is a read-only attribute.

Parameter

None

Returns

The method returns the attribute value. The number of seconds that have elapsed since January 1, 1970 UTC.

Example

```
#Get mmdb_ea_build_time value
my $value = $object->mmdb_ea_build_time();
```

mmdb_geoip_build_time( )

Use this method to retrieve the GeoIP MaxMind database build time. This is a read-only attribute.

Parameter

None

Returns

The method returns the attribute value. The number of seconds that have elapsed since January 1, 1970 UTC.

Example

```
#Get mmdb_ea_build_time value
my $value = $object->mmdb_ea_build_time();
```

name( )

Use this method to set or retrieve the host name of a grid member.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

The host name in FQDN (Fully Qualified Domain Name) format of a grid member. The FQDN consists of the host name followed by the domain name (example: abc.com). A host name can have a maximum of 256 bytes.

Note that name matching is not case-sensitive, and that names are inserted in lowercase.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get name
my $name = $grid_member->name();
#Modify name
$grid_member->name("hostname2.com");
```

nat_enabled( )

Use this method to set or retrieve the NAT compatibility setting.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

Specify "true" to enable the NAT compatibility setting or "false" to disable it.

This method will also be automatically set to "true" or "false" when "nat_group", "nat_ip_address", "node1_nat" and "node2_nat" change. If any of them is set, nat_enabled will be set to "true"; if all of them are undefined, it will be set to "false".

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get nat group
my $nat_group = $grid_member->nat_group();
#Modify nat group
$grid_member->nat_group("nat_group");
```

nat_group( )

Use this method to set or retrieve the NAT group of a grid member. Required only when NAT compatibility is enabled.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

Text with the NAT (Network Address Translation) group name with a maximum of 256 bytes. Default value is empty.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get nat group
my $nat_group = $grid_member->nat_group();
#Modify nat group
$grid_member->nat_group("nat_group");
```

nat_ip_address( )

Use this method to set or retrieve the NAT IP address of a grid member. Required only when NAT compatibility is enabled.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

An IPv4 address is a 32-bit number in dotted decimal notation. It consists of four 8-bit groups of decimal digits separated by decimal points (example: 192.168.1.2).

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get nat IP address
my $nat_ip_address = $grid_member->nat_ip_address();
#Modify nat ip address
$grid_member->nat_ip_address("5.5.5.5");
```

nic_failover_enabled( )

Use this method to set or retrieve the nic_failover_enabled flag of a grid member. This is required only when the member is a single appliance. To enable this flag, the lan2_port flag must also be set to 'true'.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

Specify "true" to set the nic_failover_enabled flag or "false" to deactivate/unset it. The default value is "false".

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get NIC failover enabled flag
my $nic_failover_enabled = $grid_member->nic_failover_enabled();
#Modify NIC failover enabled flag
$grid_member->nic_failover_enabled("true");
```

nic_failover_enable_primary( )

Use this method to set or retrieve the flag that indicates whether to prefer LAN1 when it's available or not.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

Specify "true" to nic_failover_enable_primary flag or "false" to deactivate/unset it. The default value is "false".

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get nic_failover_enable_primary value
my $value = $object->nic_failover_enable_primary();

#Modify nic_failover_enable_primary value
$object->nic_failover_enable_primary("true");
```

node1_ha( )

Use this method to set or retrieve the HA IP address of node 1. Required only when the member is an HA pair.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

An IPv4 address is a 32-bit number in dotted decimal notation. It consists of four 8-bit groups of decimal digits separated by decimal points (example: 192.168.1.2).

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get node1 ha IP address
my $node1_ha = $grid_member->node1_ha();
#Modify node1 ha IP address
$grid_member->node1_ha("3.0.0.4");
```

node1_ha_reserved_interface( )

Use this method to set or retrieve the discovery reserved interface of the HA port on node 1 of an HA grid member.

Parameter

Valid value is an Infoblox::Grid::Discovery::DeviceInterface object.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get attribute value
my $attr = $grid_member->node1_ha_reserved_interface();
#Modify attribute value
$grid_member->node1_ha_reserved_interface($interface);
```

node1_paid_nios( )

Use this method to retrieve whether the node 1 of an HA grid member is Paid NIOS.

Parameter

None

Returns

The method returns the attribute value.

Example

```
#Get Paid NIOS status
my $node1_paid_nios = $grid_member->node1_paid_nios();
```

node1_hwplatform( )

Use this method to retrieve the hardware platform of node 1 of an HA grid member.

Parameter

None

Returns

The method returns the attribute value.

Example

```
#Get hardware platform
my $node1_hwplatform = $grid_member->node1_hwplatform();
```

node1_hwid( )

Use this method to retrieve the hardware ID of node 1 of an HA grid member.

Parameter

None

Returns

The method returns the attribute value.

Example

```
#Get node1 hardware ID
my $node1_hwid = $grid_member->node1_hwid();
```

node1_lan( )

Use this method to set or retrieve the LAN IPv4 address of node 1 of an HA grid member. Required only when the member is an HA pair.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

An IPv4 address is a 32-bit number in dotted decimal notation. It consists of four 8-bit groups of decimal digits separated by decimal points (example: 192.168.1.2).

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get LAN IP address of node1
my $node1_lan = $grid_member->node1_lan();
#Modify LAN IP address of node1
$grid_member->node1_lan("3.0.0.9");
```

node1_lan_ipv6addr( )

Use this method to set or retrieve the LAN IPv6 address of node 1 of an HA grid member. Required only when the member is an HA pair.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

An IPv6 address is a 128-bit number represented as eight groups of four hexadecimal digits (i.e. two octets), each group separated by a `colon(:)` (example: 2001:0db8:85a3:0000:0000:8a2e:0370:7334). Leading zeroes and groups of consecutive zeros within the value can be omitted to produce a more compact representation (example: 2001::1, ::1).

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get LAN IPv6 address of node1
my $node1_lan_ipv6addr = $grid_member->node1_lan_ipv6addr();
#Modify LAN IPv6 address of node1
$grid_member->node1_lan_ipv6addr("2001::1");
```

node1_lan_reserved_interface( )

Use this method to set or retrieve the discovery reserved interface of the LAN port on node 1 of an HA grid member.

Parameter

Valid value is an Infoblox::Grid::Discovery::DeviceInterface object.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get attribute value
my $attr = $grid_member->node1_lan_reserved_interface();
#Modify attribute value
$grid_member->node1_lan_reserved_interface($interface);
```

node1_lan2_reserved_interface( )

Use this method to set or retrieve the discovery reserved interface of the LAN2 port on node 1 of an HA grid member.

Parameter

Valid value is an Infoblox::Grid::Discovery::DeviceInterface object.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get attribute value
my $attr = $grid_member->node1_lan2_reserved_interface();
#Modify attribute value
$grid_member->node1_lan2_reserved_interface($interface);
```

node1_lcd_autodim( )

Use this method to set or retrieve the number of seconds before the LCD on the front panel of node 1 of an HA grid member automatically dims.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

The number of seconds. Valid values are between 5 and 3600. The default value is 30.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get lcd_autodim on node 1
my $node1_lcd_autodim = $grid_member->node1_lcd_autodim();
#Modify the autodim value
$grid_member->node1_lcd_autodim(60);
```

node1_lcd_brightness( )

Use this method to set or retrieve the brightness level of the LCD on the front panel of node 1 of an HA grid member.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

Valid values are between 1 and 10. The default value is 7.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get lcd_brightness on node 1
my $node1_lcd_brightness = $grid_member->node1_lcd_brightness();
#Modify the brightness value
$grid_member->node1_lcd_brightness(3);
```

node1_lcd_hwident( )

Use this method to set or retrieve whether the node 1 of an HA grid member's hardware ID identification functionality is active.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

Specify "true" to set the lcd_hwident flag or "false" to deactivate/unset it. The default value is undefined.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get the lcd_hwident status
my $node1_lcd_hwident = $grid_member->node1_lcd_hwident();
#Modify the lcd_hwident status
$grid_member->node1_lcd_hwident("true");
```

node1_lcd_version( )

Use this method to retrieve the version of the Liquid Crystal Display (LCD) of the node 1 of an HA grid member.

Parameter

None.

Returns

The method returns the attribute value.

Example

```
#Get node1_lcd_version
my $node1_lcd_version = $grid_member->node1_lcd_version();
```

node1_lom_gateway( )

Use this method to set or retrieve the gateway IP address on node 1 of an HA grid member for the LOM functionality.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

An IPv4 address is a 32-bit number in dotted decimal notation. It consists of four 8-bit groups of decimal digits separated by decimal points (example: 192.168.1.2).

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get node1_lom_gateway
my $node1_lom_gateway = $grid_member->node1_lom_gateway();
#Modify node1_lom_gateway
$grid_member->node1_lom_gateway("3.0.0.1");
```

node1_lom_is_capable( )

Use this method to retrieve a flag describing if node 1 of an HA grid member is capable of the LOM functionality.

Parameter

The method returns "true" if the node is capable of LOM operation, "false" otherwise.

Returns

The method returns the attribute value.

Example

```
#Get node1_lom_is_capable
my $node1_lom_is_capable = $grid_member->node1_lom_is_capable();
```

node1_lom_ipv4addr( )

Use this method to set or retrieve the IPv4 address on node 1 of an HA grid member for the LOM functionality.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

An IPv4 address is a 32-bit number in dotted decimal notation. It consists of four 8-bit groups of decimal digits separated by decimal points (example: 192.168.1.2).

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get node1_lom_ipv4addr
my $node1_lom_ipv4addr = $grid_member->node1_lom_ipv4addr();
#Modify node1_lom_ipv4addr
$grid_member->node1_lom_ipv4addr("3.0.0.3");
```

node1_lom_mask( )

Use this method to set or retrieve the netmask on node 1 of an HA grid member for the LOM functionality.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

Netmask address is a 32-bit number in dotted decimal notation. It consists of four 8-bit groups of decimal digits separated by decimal points (example: 255.255.0.0).

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get node1_lom_mask
my $node1_lom_mask = $grid_member->node1_lom_mask();
#Modify node1_lom_mask
$grid_member->node1_lom_mask("255.255.0.0");
```

node1_mgmt_dscp( )

Use this method to set or retrieve the DSCP value of the IPv4 MGMT port on node 1 of an HA grid member.

Setting this method to a defined value implicitly sets the override_node1_mgmt_dscp method to "true". Setting the parameter to undefined causes the appliance to use the grid default and automatically resets the override_node1_mgmt_dscp attribute to "false".

Note that when dscp is set to a defined value and override_node1_mgmt_dscp is set to "false", the last operation takes precedence. Thus the sequence $object->node1_mgmt_dscp(11); $object->override_node1_mgmt_dscp("false"); will set override_node1_mgmt_dscp to "false", and the sequence $object->override_node1_mgmt_dscp("false"); $object->node1_mgmt_dscp(11); will result in override_node1_mgmt_dscp="true".

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

Valid values are integers between 0 and 63, inclusive. The default value is undefined.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get attribute value
my $value = $grid_member->node1_mgmt_dscp();
#Modify attribute value
$grid_member->node1_mgmt_dscp(11);
```

node1_mgmt_gateway( )

Use this method to set or retrieve the gateway IP address of the MGMT port on node 1 of an HA grid member. Required only when the MGMT port is enabled.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

An IPv4 address is a 32-bit number in dotted decimal notation. It consists of four 8-bit groups of decimal digits separated by decimal points (example: 192.168.1.2).

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get gateway of MGMT port of node1
my $node1_mgmt_gateway = $grid_member->node1_mgmt_gateway();
#Modify gateway of MGMT port of node1
$grid_member->node1_mgmt_gateway("6.0.0.1");
```

node1_mgmt_ipv6addr( )

Use this method to set or retrieve the LAN IPv6 address of the MGMT port on node 1 of an HA grid member.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

An IPv6 address is a 128-bit number represented as eight groups of four hexadecimal digits (i.e. two octets), each group separated by a `colon(:)` (example: 2001:0db8:85a3:0000:0000:8a2e:0370:7334). Leading zeroes and groups of consecutive zeros within the value can be omitted to produce a more compact representation (example: 2001::1, ::1).

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get MGMT IPv6 address of node 1
my $node1_mgmt_ipv6addr = $grid_member->node1_mgmt_ipv6addr();
#Modify MGMT IPv6 address of node 1
$grid_member->node1_mgmt_ipv6addr("2001::2");
```

node1_mgmt_ipv6_cidr( )

Use this method to set or retrieve the CIDR of the MGMT port on node 1 of an HA grid member. Required only when node1_mgmt_ipv6addr is specified.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

Specify the netmask in CIDR format.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get MGMT cidr of node 1
my $node1_mgmt_cidr = $grid_member->node1_mgmt_cidr();
#Modify MGMT cidr of node 1
$grid_member->node1_mgmt_cidr("64");
```

node1_mgmt_ipv6_dscp( )

Use this method to set or retrieve the DSCP value of the IPv6 MGMT port on node 1 of an HA grid member.

Setting this method to a defined value implicitly sets the override_node1_mgmt_ipv6_dscp method to "true". Setting the parameter to undefined causes the appliance to use the grid default and automatically resets the override_node1_mgmt_ipv6_dscp attribute to "false".

Note that when dscp is set to a defined value and override_node1_mgmt_ipv6_dscp is set to "false", the last operation takes precedence. Thus the sequence $object->node1_mgmt_ipv6_dscp(11); $object->override_node1_mgmt_ipv6_dscp("false"); will set override_node1_mgmt_ipv6_dscp to "false", and the sequence $object->override_node1_mgmt_ipv6_dscp("false"); $object->node1_mgmt_ipv6_dscp(11); will result in override_node1_mgmt_ipv6_dscp="true".

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

Valid values are integers between 0 and 63, inclusive. The default value is undefined.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get attribute value
my $value = $grid_member->node1_mgmt_ipv6_dscp();
#Modify attribute value
$grid_member->node1_mgmt_ipv6_dscp(11);
```

node1_mgmt_ipv6_enable_auto_config( )

Use this method to enable/disable IPv6 auto-configuration of the MGMT interface.

When IPv6 auto-configuration is enabled, a gateway is not needed since it will be automatically discovered.

Parameter

Specify "true" to enable IPv6 auto-configuration or "false" to disable it. Default value is "false".

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

Example

```
#Get IPv6 auto-configuration of node 1
my $ipv6_autoconfig_enabled = $grid_member->node1_mgmt_ipv6_enable_auto_config();
#Modify IPv6 auto-configuration of node 1
$grid_member->node1_mgmt_ipv6_enable_auto_config("true");
```

node1_mgmt_ipv6_gateway( )

Use this method to set or retrieve the LAN IPv6 gateway of the MGMT port on node 1 of an HA grid member.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

An IPv6 address is a 128-bit number represented as eight groups of four hexadecimal digits (i.e. two octets), each group separated by a `colon(:)` (example: 2001:0db8:85a3:0000:0000:8a2e:0370:7334). Leading zeroes and groups of consecutive zeros within the value can be omitted to produce a more compact representation (example: 2001::1, ::1).

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get MGMT IPv6 gateway of node 1
my $node1_mgmt_ipv6_gateway = $grid_member->node1_mgmt_ipv6_gateway();
#Modify MGMT IPv6 gateway of node 1
$grid_member->node1_mgmt_ipv6_gateway("2001::1");
```

node1_mgmt_mask( )

Use this method to set or retrieve the netmask of the MGMT port on node 1 of an HA grid member. Required only when the MGMT port is enabled.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

Netmask address is a 32-bit number in dotted decimal notation. It consists of four 8-bit groups of decimal digits separated by decimal points (example: 255.255.0.0).

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get netmask of MGMT port of node 1
my $node1_mgmt_mask = $grid_member->node1_mgmt_mask();
#Modify netmask of MGMT port of node 1
$grid_member->node1_mgmt_mask("255.255.0.0");
```

node1_mgmt_lan( )

Use this method to set or retrieve the LAN IP address of the MGMT port on node 1 of an HA grid member. Required only when the MGMT port is enabled.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

An IPv4 address is a 32-bit number in dotted decimal notation. It consists of four 8-bit groups of decimal digits separated by decimal points (example: 192.168.1.2).

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get LAN IP address of MGMT port on node 1
my $node1_mgmt_lan = $grid_member->node1_mgmt_lan();
#Modify LAN IP address of MGMT port on node 1
$grid_member->node1_mgmt_lan("6.0.0.5");
```

node1_mgmt_port( )

Use this method to set or retrieve the MGMT port flag of an HA grid member. Required only when the MGMT port is enabled.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Note that node1_mgmt_port, node2_mgmt_port and mgmt_port share the value; setting any of them will have the same effect.

Parameter

Specify "true" to set the node1_mgmt_port flag or "false" to deactivate/unset it. The default value is undefined. To disable node1_mgmt_port flag we also need to disable "mgmt_port" and "node2_mgmt_port" flags.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get node 1 MGMT port flag
my $node1_mgmt_port = $grid_member->node1_mgmt_port();
#Modify node 1 MGMT port flag
$grid_member->node1_mgmt_port("true");
```

node1_mgmt_reserved_interface( )

Use this method to set or retrieve the discovery reserved interface of the MGMT port on node 1 of an HA grid member.

Parameter

Valid value is an Infoblox::Grid::Discovery::DeviceInterface object.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get attribute value
my $attr = $grid_member->node1_mgmt_reserved_interface();
#Modify attribute value
$grid_member->node1_mgmt_reserved_interface($interface);
```

node1_nat( )

Use this method to set or retrieve the Network Address Translation (NAT) address of node 1 of an HA grid member. Required only when NAT compatibility is enabled.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

An IPv4 address is a 32-bit number in dotted decimal notation. It consists of four 8-bit groups of decimal digits separated by decimal points (example: 192.168.1.2).

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get Network Address Translation (NAT) address of node 1
my $node1_nat = $grid_member->node1_nat();
#Modify Network Address Translation (NAT) address of node 1
$grid_member->node1_nat("5.2.2.1");
```

node1_service_status( )

Use this method to retrieve the service status list of a grid member.

Returns

On retrieval, the member returns an array reference that contains Infoblox::Grid::Member::ServiceStatus objects.

Example

```
#Get the service status list
my $services_list = $grid_member->node1_service_status();
```

vpn_mtu( )

Use this method to set or retrieve the VPN maximum transmission unit (MTU).

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

Specify the value to set the vpn_mtu value.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get VPN MTU
my $vpn_mtu = $grid_member->vpn_mtu();
#Modify VPN MTU
$grid_member->vpn_mtu(600);
```

node1_vpn_on_mgmt( )

Use this method to set or retrieve the VPN on MGMT port flag of an HA grid member. Valid only when the MGMT port is enabled and the member is not a master candidate. This flag enables VPN services on the MGMT port of an HA grid member.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Note that node1_vpn_on_mgmt, node2_vpn_on_mgmt and vpn_on_mgmt share the value; setting any of them will have the same effect.

Parameter

Specify "true" to set the node1_vpn_on_mgmt flag or "false" to deactivate/unset it. The default value is undefined.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get node 1 VPN on MGMT port flag
my $node1_vpn_on_mgmt = $grid_member->node1_vpn_on_mgmt();
#Modify node 1 VPN on MGMT port flag
$grid_member->node1_vpn_on_mgmt("false");
```

node1_hwtype( )

Use this method to retrieve the hardware type of node 1 of an HA grid member.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

The actual hardware type is populated when the physical node joins the grid. The default value is "".

Returns

The method returns the attribute value.

Example

```
#Get node1 hardware type
```

```
my $node1_hwtype = $grid_member->node1_hwtype();
```

node2_ha( )

Use this method to set or retrieve the HA IP address of node 2 of an HA grid member.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

An IPv4 address is a 32-bit number in dotted decimal notation. It consists of four 8-bit groups of decimal digits separated by decimal points (example: 192.168.1.2).

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get node 2 HA IP address
my $node2_ha = $grid_member->node2_ha();
#Modify node 2 HA IP address
$grid_member->node2_ha("3.0.0.6");
```

node2_ha_reserved_interface( )

Use this method to set or retrieve the discovery reserved interface of the HA port on node 2 of an HA grid member.

Parameter

Valid value is an Infoblox::Grid::Discovery::DeviceInterface object.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get attribute value
my $attr = $grid_member->node2_ha_reserved_interface();
#Modify attribute value
$grid_member->node2_ha_reserved_interface($interface);
```

node2_paid_nios( )

Use this method to retrieve whether the node 2 of an HA grid member is Paid NIOS.

Parameter

None

Returns

The method returns the attribute value.

Example

```
#Get Paid NIOS status
my $node2_paid_nios = $grid_member->node2_paid_nios();
```

node2_hwplatform( )

Use this method to retrieve the hardware platform of node 2 of an HA grid member.

Parameter

None

Returns

The method returns the attribute value.

Example

```
#Get hardware platform
my $node2_hwplatform = $grid_member->node2_hwplatform();
```

node2_hwid( )

Use this method to retrieve the hardware ID of node 2 of an HA grid member.

Parameter

None

Returns

The method returns the attribute value.

Example

```
#Get node2 hardware ID
my $node2_hwid = $grid_member->node2_hwid();
```

node2_lan( )

Use this method to set or retrieve the LAN IP address of node 2 of an HA grid member.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

An IPv4 address is a 32-bit number in dotted decimal notation. It consists of four 8-bit groups of decimal digits separated by decimal points (example: 192.168.1.2).

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get LAN IP address of node 2
my $node2_lan = $grid_member->node2_lan();
#Modify LAN IP address of node 2
$grid_member->node2_lan("3.0.0.5");
```

node2_lan_ipv6addr( )

Use this method to set or retrieve the LAN IPv6 address of node 2 of an HA grid member. Required only when the member is an HA pair.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

An IPv6 address is a 128-bit number represented as eight groups of four hexadecimal digits (i.e. two octets), each group separated by a `colon(:)` (example: 2001:0db8:85a3:0000:0000:8a2e:0370:7334). Leading zeroes and groups of consecutive zeros within the value can be omitted to produce a more compact representation (example: 2001::1, ::1).

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get LAN IPv6 address of node2
my $node2_lan_ipv6addr = $grid_member->node2_lan_ipv6addr();
#Modify LAN IPv6 address of node2
$grid_member->node2_lan_ipv6addr("2001::1");
```

node2_lan_reserved_interface( )

Use this method to set or retrieve the discovery reserved interface of the LAN port on node 2 of an HA grid member.

Parameter

Valid value is an Infoblox::Grid::Discovery::DeviceInterface object.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get attribute value
my $attr = $grid_member->node2_lan_reserved_interface();
#Modify attribute value
$grid_member->node2_lan_reserved_interface($interface);
```

node2_lan2_reserved_interface( )

Use this method to set or retrieve the discovery reserved interface of the LAN2 port on node 2 of an HA grid member.

Parameter

Valid value is an Infoblox::Grid::Discovery::DeviceInterface object.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get attribute value
my $attr = $grid_member->node2_lan2_reserved_interface();
#Modify attribute value
$grid_member->node2_lan2_reserved_interface($interface);
```

node2_lcd_autodim( )

Use this method to set or retrieve the number of seconds before the LCD on the front panel of node 2 of an HA grid member automatically dims.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

The number of seconds. Valid values are between 5 and 3600. The default value is 30.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get lcd_autodim on node 2
my $node2_lcd_autodim = $grid_member->node2_lcd_autodim();
#Modify the autodim value
$grid_member->node2_lcd_autodim(60);
```

node2_lcd_brightness( )

Use this method to set or retrieve the brightness level of the LCD on the front panel of node 2 of an HA grid member.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

Valid values are between 1 and 10. The default value is 7.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get lcd_brightness on node 2
my $node2_lcd_brightness = $grid_member->node2_lcd_brightness();
#Modify the brightness value
$grid_member->node2_lcd_brightness(3);
```

node2_lcd_hwident( )

Use this method to set or retrieve whether the node 2 of an HA grid member's hardware ID identification functionality is active.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

Specify "true" to set the node2_lcd_hwident flag or "false" to deactivate/unset it. The default value is undefined.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get the lcd_hwident status
my $node2_lcd_hwident = $grid_member->node2_lcd_hwident();
#Modify the lcd_hwident status
$grid_member->node2_lcd_hwident("true");
```

node2_lcd_version( )

Use this method to retrieve the version of the Liquid Crystal Display (LCD) of the node 2 of an HA grid member.

Parameter

None.

Returns

The method returns the attribute value.

Example

```
#Get node2_lcd_version
my $node2_lcd_version = $grid_member->node2_lcd_version();
```

node2_lom_gateway( )

Use this method to set or retrieve the gateway IP address on node 2 of an HA grid member for the LOM functionality.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

An IPv4 address is a 32-bit number in dotted decimal notation. It consists of four 8-bit groups of decimal digits separated by decimal points (example: 192.168.1.1).

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get node2_lom_gateway
my $node2_lom_gateway = $grid_member->node2_lom_gateway();
#Modify node2_lom_gateway
$grid_member->node2_lom_gateway("3.0.0.1");
```

node2_lom_is_capable( )

Use this method to retrieve a flag describing if node 1 of an HA grid member is capable of the LOM functionality.

Parameter

The method returns "true" if the node is capable of LOM operation, "false" otherwise.

Returns

The method returns the attribute value.

Example

```
#Get node2 lom is capable
```

```
my $node2_lom_is_capable = $grid_member->node2_lom_is_capable();
```

node2_lom_ipv4addr( )

Use this method to set or retrieve the IPv4 address on node 2 of an HA grid member for the LOM functionality.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

An IPv4 address is a 32-bit number in dotted decimal notation. It consists of four 8-bit groups of decimal digits separated by decimal points (example: 192.168.1.2).

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get node2_lom_ipv4addr
my $node2_lom_ipv4addr = $grid_member->node2_lom_ipv4addr();
#Modify node2_lom_ipv4addr
$grid_member->node2_lom_ipv4addr("3.0.0.3");
```

node2_lom_mask( )

Use this method to set or retrieve the netmask on node 2 of an HA grid member for the LOM functionality.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

Netmask address is a 32-bit number in dotted decimal notation. It consists of four 8-bit groups of decimal digits separated by decimal points (example: 255.255.0.0).

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get node2_lom_mask
my $node2_lom_mask = $grid_member->node2_lom_mask();
#Modify node2_lom_mask
$grid_member->node2_lom_mask("255.255.0.0");
```

node2_mgmt_dscp( )

Use this method to set or retrieve the DSCP value of the IPv4 MGMT port on node 2 of an HA grid member.

Setting this method to a defined value implicitly sets the override_node2_mgmt_dscp method to "true". Setting the parameter to undefined causes the appliance to use the grid default and automatically resets the override_node2_mgmt_dscp attribute to "false".

Note that when dscp is set to a defined value and override_node2_mgmt_dscp is set to "false", the last operation takes precedence. Thus the sequence $object->node2_mgmt_dscp(11); $object->override_node2_mgmt_dscp("false"); will set override_node2_mgmt_dscp to "false", and the sequence $object->override_node2_mgmt_dscp("false"); $object->node2_mgmt_dscp(11); will result in override_node2_mgmt_dscp="true".

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

Valid values are integers between 0 and 63, inclusive. The default value is undefined.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get attribute value
my $value = $grid_member->node2_mgmt_dscp();
#Modify attribute value
```

```
$grid_member->node2_mgmt_dscp(11);
```

node2_mgmt_gateway( )

Use this method to set or retrieve the gateway IP address of the MGMT port on node 2 of an HA grid member. Required only when the MGMT port is enabled.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

An IPv4 address is a 32-bit number in dotted decimal notation. It consists of four 8-bit groups of decimal digits separated by decimal points (example: 192.168.1.2).

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get gateway of MGMT port of node 2
my $node2_mgmt_gateway = $grid_member->node2_mgmt_gateway();
#Modify gateway of MGMT port of node 2
$grid_member->node2_mgmt_gateway("7.0.0.1");
```

node2_mgmt_mask( )

Use this method to set or retrieve the netmask of the MGMT port on node 2 of an HA grid member. Required only when the MGMT port is enabled.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

Netmask address is a 32-bit number in dotted decimal notation. It consists of four 8-bit groups of decimal digits separated by decimal points (example: 255.255.0.0).

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get netmask of MGMT port of node 2
my $node2_mgmt_mask = $grid_member->node2_mgmt_mask();
#Modify netmask of MGMT port of node 2
$grid_member->node2_mgmt_mask("255.255.0.0");
```

node2_mgmt_lan( )

Use this method to set or retrieve the LAN IP address of the MGMT port on node 2 of an HA grid member. Required only when the MGMT port is enabled.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

An IPv4 address is a 32-bit number in dotted decimal notation. It consists of four 8-bit groups of decimal digits separated by decimal points (example: 192.168.1.2).

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get LAN IP address of MGMT port of node 2
my $node2_mgmt_lan = $grid_member->node2_mgmt_lan();
#Modify LAN IP address of MGMT port of node 2
$grid_member->node2_mgmt_lan("7.0.0.5");
```

node2_mgmt_port( )

Use this method to set or retrieve the MGMT port flag of an HA grid member. Required only when the MGMT port is enabled.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Note that node1_mgmt_port, node2_mgmt_port and mgmt_port share the value: setting any of them will have the same effect.

Parameter

Specify "true" to set the node2_mgmt_port flag or "false" to deactivate/unset it. The default value is undefined. To disable node2_mgmt_port flag, you must disable the "mgmt_port" and "node1_mgmt_port" flags as well.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get MGMT port flag of node 2
my $node2_mgmt_port = $grid_member->node2_mgmt_port();
#Modify MGMT port flag of node 2
$grid_member->node2_mgmt_port("true");
```

node2_mgmt_reserved_interface( )

Use this method to set or retrieve the discovery reserved interface of the MGMT port on node 2 of an HA grid member.

Parameter

Valid value is an Infoblox::Grid::Discovery::DeviceInterface object.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get attribute value
my $attr = $grid_member->node2_mgmt_reserved_interface();
#Modify attribute value
$grid_member->node2_mgmt_reserved_interface($interface);
```

node2_mgmt_ipv6_dscp( )

Use this method to set or retrieve the DSCP value of the IPv6 MGMT port on node 2 of an HA grid member.

Setting this method to a defined value implicitly sets the override_node2_mgmt_ipv6_dscp method to "true". Setting the parameter to undefined causes the appliance to use the grid default and automatically resets the override_node2_mgmt_ipv6_dscp attribute to "false".

Note that when dscp is set to a defined value and override_node2_mgmt_ipv6_dscp is set to "false", the last operation takes precedence. Thus the sequence $object->node2_mgmt_ipv6_dscp(11); $object->override_node2_mgmt_ipv6_dscp("false"); will set override_node2_mgmt_ipv6_dscp to "false", and the sequence $object->override_node2_mgmt_ipv6_dscp("false"); $object->node2_mgmt_ipv6_dscp(11); will result in override_node2_mgmt_ipv6_dscp="true".

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

Valid values are integers between 0 and 63, inclusive. The default value is undefined.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get attribute value
my $value = $grid_member->node2_mgmt_ipv6_dscp();
#Modify attribute value
$grid_member->node2_mgmt_ipv6_dscp(11);
```

node2_nat( )

Use this method to set or retrieve the Network Address Translation (NAT) address of node 2. Required only when NAT compatibility is enabled.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

An IPv4 address is a 32-bit number in dotted decimal notation. It consists of four 8-bit groups of decimal digits separated by decimal points (example: 192.168.1.2).

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get Network Address Translation (NAT) address of node2
my $node2_nat = $grid_member->node2_nat();
#Modify Network Address Translation (NAT) address of node2
$grid_member->node2_nat("5.4.4.6");
```

node2_service_status( )

Use this method to retrieve the service status list of a grid member.

Returns

On retrieval, the member returns an array reference that contains Infoblox::Grid::Member::ServiceStatus objects.

Example

```
#Get the service status list
my $services_list = $grid_member->node2_service_status();
```

node2_vpn_on_mgmt( )

Use this method to set or retrieve the VPN on MGMT port flag of an HA grid member. Valid only when the MGMT port is enabled and the member is not a master candidate. This flag enables VPN services on the MGMT port of an HA grid member.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Note that node1_vpn_on_mgmt, node2_vpn_on_mgmt and vpn_on_mgmt share the value: setting any of them will have the same effect.

Parameter

Specify "true" to set the node2_vpn_on_mgmt flag or "false" to deactivate/unset it. The default value is undefined.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get node 2 VPN on MGMT port flag
my $node2_vpn_on_mgmt = $grid_member->node2_vpn_on_mgmt();
#Modify node 2 VPN on MGMT port flag
$grid_member->node2_vpn_on_mgmt("false");
```

node2_hwtype( )

Use this method to retrieve the hardware type of node 2 of an HA grid member.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

The actual hardware type is populated when the physical node joins the grid. The default value is "".

Returns

The method returns the attribute value.

Example

```
#Get node2 hardware type
```

```
my $node2_hwtype = $grid_member->node2_hwtype();
```

notification_email_addr( )

Use this method to set or retrieve the notification email address of a grid member.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

The email address. Default value is undefined.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get notification email address
my $notification_email_addr = $grid_member->notification_email_addr();
#Modify notification email address
$grid_member->notification_email_addr("admin\@infoblox.com");
```

ntp_access_list( )

Use this method to set or retrieve the Network Time Protocols (NTP) access control list of a grid member. Valid only when the enable_ntp flag is set to "true".

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

Valid value is an Infoblox::Grid::NamedACL object or an array reference that contains Infoblox::Grid::NTPAccess objects .

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get NTP access control list
my $ntp_access_list = $grid_member->ntp_access_list();
#Modify NTP access control list
$grid_member->ntp_access_list([$ntpac1,$ntpac2]);
#Undef NTP access control list
$grid_member->ntp_access_list(undef);
#Use Infoblox::Grid::NamedACL
$grid_member->ntp_access_list($nacl);
```

---

ntp_kod( )

Use this method to enable Kiss-o'-Death packets on the NTP server.

Setting this method to a defined value implicitly sets the override_ntp_kod method to "true". Setting the parameter to undefined causes the appliance to use the Grid default and automatically resets the override_ntp_kod attribute to "false".

Note that when ntp_kod is set to a defined value and override_ntp_kod is set to "false", the last operation takes precedence. Thus the sequence $object->ntp_kod("true"); $object->override_ntp_kod("false"); will set override_ntp_kod to "false", and the sequence $object->override_ntp_kod("false"); $object->ntp_kod("true"); will result in override_ntp_kod="true".

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

Specify "true" to enable Kiss-o'-Death packets on the NTP server or "false" to disable them. The default value is "false".

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get the current value of the ntp_kod parameter
my $ntp_kod = $grid->ntp_kod();
#Set the ntp_kod parameter
$grid->ntp_kod("true");
```

ntp_service_type( )

Use this method to set or retrieve the NTP service type for assigned Named ACL object.

The attribute value is ignored if 'ntp_access_list' attribute is not a Named ACL object.

Parameter

Valid values are "Time" or "Time and NTP control". The default value is "Time".

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get ntp_service_type
my $service = $grid_member->ntp_service_type();
#Modify ntp_service_type
$grid_member->ntp_service_type("Time and NTP control");
```

ntp_authentication_key( )

Use this method to set or retrieve the Network Time Protocol (NTP) authentication `key(s)` of a grid member. Valid only when the enable_ntp flag is set to "true".

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

Valid value is an array reference that contains Infoblox::Grid::NTPKey object(s).

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get NTP authentication key
my $ntp_authentication_key = $grid_member->ntp_authentication_key();
#Modify NTP authentication key
my $ntp_key1 = Infoblox::Grid::NTPKey->new(
    key_number => "1111",
    key_type   => "M",
    key_string => "aaaa"
);
my $ntp_key2 = Infoblox::Grid::NTPKey->new(
    key_number => "2222",
    key_type   => "n",
    key_string => "d3e54352e5548080"
);
$grid_member->ntp_authentication_key([$ntp_key1, $ntp_key2]);
```

ntp_server( )

Use this method to set or retrieve the NTP server list.

Parameter

Array reference that contains Infoblox::Grid::NTPServer objects or undef.

Specifying a reference to an empty array while the "exclude_grid_master_as_ntp_server" attribute is set to "false" sets the Grid Master as the only NTP source for the given Member. Setting the value of this parameter to undef disables the feature.

NOTE: Currently, if you need to specify an external NTP server with authentication, the NTP keys that you use must be known to the Infoblox::Grid object. See the following example.

NOTE: Setting this to a list also sets external_ntp_servers_enabled to 'true'. Setting this to undef also sets external_ntp_servers_enabled to 'false'.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

Example

```
#Get the ntp_server list for the given member
my $ntp_server_list=$grid_member->ntp_server();

#Modify ntp_server property
my $ntp_server1 = Infoblox::Grid::NTPServer->new(address => "1.1.1.1");
my $ntp_server2 = Infoblox::Grid::NTPServer->new(address => "2.2.2.2");
$grid_member->ntp_server([$ntp_server1,$ntp_server2]);

#Setting grid master as the only NTP source
my $session=Infoblox::Session->new(
    master   => "127.0.0.1",
    username => "admin",
    password => "infoblox"
   );
die(Infoblox::status_code().":".Infoblox::status_detail()) unless $session;

my $grid_member=$session->get(
                              object => "Infoblox::Grid::Member",
                              name   => "infoblox.localdomain"
                             );
die($session->status_code()." : ".$session->status_detail()) unless (ref($grid_member) eq
"Infoblox::Grid::Member");

$grid_member->ntp_server([]);
$grid_member->exclude_grid_master_as_ntp_server("false");
$session->modify($grid_member) or die($session->status_code()." : ".$session->status_detail());

#Adding external NTP servers with authentication
my $session=Infoblox::Session->new(
    master   => "127.0.0.1",
    username => "admin",
    password => "infoblox"
   );
die(Infoblox::status_code().":".Infoblox::status_detail()) unless $session;
my $grid_member=$session->get(
                              object => "Infoblox::Grid::Member",
                              name   => "infoblox.localdomain"
                             );
die($session->status_code()." : ".$session->status_detail()) unless (ref($grid_member) eq
"Infoblox::Grid::Member");

my $grid=$session->get(
                        object => "Infoblox::Grid",
                        name   => "Infoblox"
                      );
die($session->status_code()." : ".$session->status_detail()) unless (ref($grid) eq
"Infoblox::Grid");

#It is assumed in the following that the grid has NTP enabled
die("Grid must have ntp enabled) unless ($grid->enable_ntp() eq "true");

my $ntp_key_1= Infoblox::Grid::NTPKey->new(
           key_number => "101",
           key_type   => "n",
           key_string => "d3e54352e5548080"
       );
$grid->ntp_authentication_key([$ntp_key_1]);
$session->modify($grid) or die(($session->status_code()." : ".$session->status_detail());

my $ntp_server=Infoblox::Grid::NTPServer->new(
                                              address => "1.1.1.1"
                                              authentication => "true"
                                              ntp_key=>$ntp_key_1
                                             );

$grid_member->ntp_server([$ntp_server]);
$session->modify($grid_member) or die(($session->status_code()." : ".$session->status_detail());
```

ospf_list( )

Use this method to set or retrieve the OSPF area configuration (for anycast) list of a grid member.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

Valid value is an array reference that contains Infoblox::Grid::Member::OSPF object(s).

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get OSPF list
my $ospf_list = $grid_member->ospf_list();
#Modify OSPF list
my $ospf1 = Infoblox::Grid::Member::OSPF->new(
        area_id             => "1",
        interface           => "LAN_HA",
        authentication_type => "none",
);

$grid_member->ospf_list([$ospf1]);
```

override_enable_lom( )

The override_enable_lom attribute controls whether the enable_lom value in the member is used, instead of the grid default.

The override_enable_lom attribute can be specified explicitly. It is also set implicitly when enable_lom is set to a defined value.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

Specify "true" to set the override_enable_lom flag or "false" to deactivate/unset it. The default value is "false".

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get override_enable_lom from the member
my $override = $grid_member->override_enable_lom();
#Modify override_enable_lom
$grid_member->override_enable_lom("true");
```

override_lom_users( )

The override_lom_users attribute controls whether the lom_users value in the member is used, instead of the grid default.

The override_lom_users attribute can be specified explicitly. It is also set implicitly when lom_users is set to a defined value.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

Specify "true" to set the override_lom_users flag or "false" to deactivate/unset it. The default value is "false".

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get override_lom_users from the member
my $override = $grid_member->override_lom_users();
#Modify lom_users
$grid_member->override_lom_users("true");
```

override_dscp( )

The override_dscp attribute contols whether the member DSCP value is used, instead of the Grid default.

The override_dscp attribute can be specified explicitly. It is also set implicitly when DSCP is set to a defined value.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

Specify "true" to set the override_dscp flag or "false" to deactivate/unset it. The default value is "false".

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get override_dscp from the member
my $override = $grid_member->override_dscp();
#Modify override_dscp
$grid_member->override_dscp("true");
```

override_enable_member_redirect( )

The override_enable_member_redirect attribute controls whether the enable_member_redirect value in the member is used, instead of the grid default.

The override_enable_member_redirect attribute can be specified explicitly. It is also set implicitly when enable_member_redirect is set to a defined value.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

Specify "true" to set the override_enable_member_redirect flag or "false" to deactivate/unset it. The default value is "false".

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get override_enable_member_redirect from the member
my $override = $grid_member->override_enable_member_redirect();
#Modify enable_member_redirect
$grid_member->override_enable_member_redirect("true");
```

override_lan1_dscp( )

The override_lan1_dscp attribute contols whether the DSCP value of the IPv4 LAN1 port is used, instead of the Grid default.

The override_lan1_dscp attribute can be specified explicitly. It is also set implicitly when DSCP is set to a defined value.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

Specify "true" to set the override_lan1_dscp flag or "false" to deactivate/unset it. The default value is "false".

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get override_lan1_dscp from the member
my $override = $grid_member->override_lan1_dscp();
#Modify override_lan1_dscp
$grid_member->override_lan1_dscp("true");
```

override_lan1_ipv6_dscp( )

The override_lan1_ipv6_dscp attribute contols whether the DSCP value of the IPv6 LAN1 port is used, instead of the Grid default.

The override_lan1_ipv6_dscp attribute can be specified explicitly. It is also set implicitly when DSCP is set to a defined value.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

Specify "true" to set the override_lan1_ipv6_dscp flag or "false" to deactivate/unset it. The default value is "false".

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get override_lan1_ipv6_dscp from the member
my $override = $grid_member->override_lan1_ipv6_dscp();
#Modify override_lan1_ipv6_dscp
$grid_member->override_lan1_ipv6_dscp("true");
```

override_lan2_dscp( )

The override_lan2_dscp attribute contols whether the DSCP value of the IPv4 LAN2 port is used, instead of the Grid default.

The override_lan2_dscp attribute can be specified explicitly. It is also set implicitly when DSCP is set to a defined value.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

Specify "true" to set the override_lan2_dscp flag or "false" to deactivate/unset it. The default value is "false".

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get override_lan2_dscp from the member
my $override = $grid_member->override_lan2_dscp();
#Modify override_lan2_dscp
$grid_member->override_lan2_dscp("true");
```

override_lan2_ipv6_dscp( )

The override_lan2_ipv6_dscp attribute contols whether the DSCP value of the IPv6 LAN2 port is used, instead of the Grid default.

The override_lan2_ipv6_dscp attribute can be specified explicitly. It is also set implicitly when DSCP is set to a defined value.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

Specify "true" to set the override_lan2_ipv6_dscp flag or "false" to deactivate/unset it. The default value is "false".

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get override_lan2_ipv6_dscp from the member
my $override = $grid_member->override_lan2_ipv6_dscp();
#Modify override_lan2_ipv6_dscp
$grid_member->override_lan2_ipv6_dscp("true");
```

override_mgmt_dscp( )

The override_mgmt_dscp attribute contols whether the DSCP value of the IPv4 MGMT port is used, instead of the Grid default.

The override_mgmt_dscp attribute can be specified explicitly. It is also set implicitly when DSCP is set to a defined value.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

Specify "true" to set the override_mgmt_dscp flag or "false" to deactivate/unset it. The default value is "false".

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get override_mgmt_dscp from the member
my $override = $grid_member->override_mgmt_dscp();
#Modify override_mgmt_dscp
$grid_member->override_mgmt_dscp("true");
```

override_mgmt_ipv6_dscp( )

The override_mgmt_ipv6_dscp attribute contols whether the DSCP value of the IPv6 MGMT port is used, instead of the Grid default.

The override_mgmt_ipv6_dscp attribute can be specified explicitly. It is also set implicitly when DSCP is set to a defined value.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

Specify "true" to set the override_mgmt_ipv6_dscp flag or "false" to deactivate/unset it. The default value is "false".

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get override_mgmt_ipv6_dscp from the member
my $override = $grid_member->override_mgmt_ipv6_dscp();
#Modify override_mgmt_ipv6_dscp
$grid_member->override_mgmt_ipv6_dscp("true");
```

override_node1_mgmt_dscp( )

The override_node1_mgmt_dscp attribute contols whether the DSCP value of the IPv4 MGMT port on node 1 of an HA grid member is used, instead of the Grid default.

The override_node1_mgmt_dscp attribute can be specified explicitly. It is also set implicitly when DSCP is set to a defined value.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

Specify "true" to set the override_node1_mgmt_dscp flag or "false" to deactivate/unset it. The default value is "false".

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get override_node1_mgmt_dscp from the member
my $override = $grid_member->override_node1_mgmt_dscp();
#Modify override_node1_mgmt_dscp
$grid_member->override_node1_mgmt_dscp("true");
```

override_node1_mgmt_ipv6_dscp( )

The override_node1_mgmt_ipv6_dscp attribute contols whether the DSCP value of the IPv6 MGMT port on node 1 of an HA grid member is used, instead of the Grid default.

The override_node1_mgmt_ipv6_dscp attribute can be specified explicitly. It is also set implicitly when DSCP is set to a defined value.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

Specify "true" to set the override_node1_mgmt_ipv6_dscp flag or "false" to deactivate/unset it. The default value is "false".

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get override_node1_mgmt_ipv6_dscp from the member
my $override = $grid_member->override_node1_mgmt_ipv6_dscp();
#Modify override_node1_mgmt_ipv6_dscp
$grid_member->override_node1_mgmt_ipv6_dscp("true");
```

override_node2_mgmt_dscp( )

The override_node2_mgmt_dscp attribute contols whether the DSCP value of the IPv4 MGMT port on node 2 of an HA grid member is used, instead of the Grid default.

The override_node2_mgmt_dscp attribute can be specified explicitly. It is also set implicitly when DSCP is set to a defined value.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

Specify "true" to set the override_node2_mgmt_dscp flag or "false" to deactivate/unset it. The default value is "false".

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get override_node2_mgmt_dscp from the member
my $override = $grid_member->override_node2_mgmt_dscp();
#Modify override_node2_mgmt_dscp
$grid_member->override_node2_mgmt_dscp("true");
```

override_node2_mgmt_ipv6_dscp( )

The override_node2_mgmt_ipv6_dscp attribute contols whether the DSCP value of the IPv6 MGMT port on node 2 of an HA grid member is used, instead of the Grid default.

The override_node2_mgmt_ipv6_dscp attribute can be specified explicitly. It is also set implicitly when DSCP is set to a defined value.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

Specify "true" to set the override_node2_mgmt_ipv6_dscp flag or "false" to deactivate/unset it. The default value is "false".

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get override_node2_mgmt_ipv6_dscp from the member
my $override = $grid_member->override_node2_mgmt_ipv6_dscp();
#Modify override_node2_mgmt_ipv6_dscp
$grid_member->override_node2_mgmt_ipv6_dscp("true");
```

override_ntp_kod( )

The override_ntp_kod attribute controls whether the ntp_kod value in the member is used, instead of the Grid default.

The override_ntp_kod attribute can be specified explicitly. It is also set implicitly when ntp_kod is set to a defined value.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

Specify "true" to set the override_ntp_kod flag or "false" to deactivate it. The default value is "false".

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get override_ntp_kod from the member
my $override = $grid_member->override_ntp_kod();
#Modify override_ntp_kod
$grid_member->override_ntp_kod("true");
```

override_syslog_backup_servers( )

The override_syslog_backup_servers attribute controls whether the syslog_backup_servers value for the Member is used, instead of the Grid default.

The override_syslog_backup_servers attribute can be specified explicitly. It is also set implicitly when syslog_backup_servers is set to a defined value.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

Specify 'true' to set the override_syslog_backup_servers flag or 'false' to deactivate/unset it. The default value is 'false'.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#get override_syslog_backup_servers of member
my $override_syslog_backup_servers = $grid_member->override_syslog_backup_servers();

#modify override_syslog_backup_servers
$grid_member->override_syslog_backup_servers('true');
```

override_threshold_traps( )

The override_threshold_traps attribute controls whether the threshold_traps value in the member is used, instead of the grid default.

The override_threshold_traps attribute can be specified explicitly. It is also set implicitly when threshold_traps is set to a defined value.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

Specify "true" to set the override_threshold_traps flag or "false" to deactivate/unset it. The default value is "false".

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get override_threshold_traps of member
my $override_threshold_traps = $grid_member->override_threshold_traps();
#Set override_threshold_traps
$grid_member->override_threshold_traps("true");
```

override_trap_notifications( )

The override_trap_notifications attribute controls whether the trap_notifications value in the member is used, instead of the grid default.

The override_trap_notifications attribute can be specified explicitly. It is also set implicitly when trap_notifications is set to a defined value.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

Specify "true" to set the override_trap_notifications flag or "false" to deactivate/unset it. The default value is "false".

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get override_trap_notifications of member
my $override_trap_notifications = $grid_member->override_trap_notifications();
#Set override_trap_notifications
$grid_member->override_trap_notifications("true");
```

passive_ha_arp_enabled( )

Use this method to set or retrieve the ARP protocol setting on the passive node of an HA pair. If you do not specify a value, the default value is "false". You can only set this value to "true" if the member is an HA pair.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

Specify "true" to set the passive_ha_arp_enabled flag or "false" to deactivate/unset it. The default value is "false".

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get member passive_ha_arp_enabled
my $passive_ha_arp_enabled = $grid_member->passive_ha_arp_enabled();
#Modify member passive_ha_arp_enabled
$grid_member->passive_ha_arp_enabled("true");
```

platform( )

Use this method to set or retrieve the platform type of a grid member. If you do not specify a value, the default value is "Infoblox".

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

Valid value is one of either "Infoblox", "Cisco", "Riverbed", or "VNIOS".

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get member platform
my $platform = $grid_member->platform();
#Modify node 2 hardware type
$grid_member->platform("Cisco");
```

prefer_resolver( )

Use this method to set or retrieve the preferred name server of the grid member. The grid member sends queries to this name server first.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

Valid values are Infoblox::DNS::Nameserver object and Infoblox::DNS::Member object.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get preferred resolver
```

```
my $prefer_resolver = $grid_member->prefer_resolver();
#Modify preferred resolver
my $member1 = Infoblox::DNS::Member->new(
    name           => $host_name,
    ipv4addr       => $host_ip,
    stealth        => "false",
    lead           => "false",
    grid_replicate => "false"
);
$grid_member->prefer_resolver($member1);
#Undef prefer resolver
$grid_member->prefer_resolver(undef);
```

pre_provisioning( )

Use this method to set or retrieve the pre-provisioned Grid member.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

Valid value is an Infoblox::Grid::Member::PreProvisioning object.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get PreProvisioning grid Member
my $pp = $grid_member->pre_provisioning();
#Modify PreProvisioning grid Member
my $hw = Infoblox::Grid::Member::PreProvisioning::Hardware->new(
   hwtype          => 'IB-4010',
   hwmodel         => 'Rev1',
);

my $pp = Infoblox::Grid::Member::PreProvisioning->new(
   hardware_info   => [$hw],
   license         => ['dns', 'dhcp'],
);
$grid_member->pre_provisioning($pp);
```

preserve_if_owns_delegation( )

Use this method to set or retrieve the preserve_if_owns_delegation attribute. If you do not specify a value, the default value is "false".

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

Specify "true" to set the preserve_if_owns_delegation flag or "false" to deactivate/unset it. The default value is "false".

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get member preserve_if_owns_delegation
my $preserve_if_owns_delegation  = $grid_member->preserve_if_owns_delegation ();
#Modify member preserve_if_owns_delegation
$grid_member->preserve_if_owns_delegation ("true");
```

query_comm_string( )

Use this method to set or retrieve the Simple Network Management Protocol (SNMP) query community string that the SNMP management system must send together with its queries to the grid member. An SNMP community string is a text string that acts as a password. It is used to authenticate messages that are sent between the management station (the SNMP manager) and the grid member.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

Text with the Simple Network Management Protocol (SNMP) query community string with a maximum of 50 bytes. Default value is undefined.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get query community string
my $query_comm_string = $grid_member->query_comm_string();
#Modify query community string
$grid_member->query_comm_string("test");
#Undef query communication string
$grid_member->query_comm_string(undef);
```

remote_console_access( )

Use this method to set or retrieve the remote_console support access flag of a grid member. When you enable remote console access, a client can access the device using a Secure Shell (SSH) connection.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

Specify "true" to set the remote_console_access flag or "false" to deactivate/unset it. The default value is undefined.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get remote console access flag
my $remote_console_access = $grid_member->remote_console_access();
#Modify remote console access flag
$grid_member->remote_console_access("true");
#Undef remote console access flag
$grid_member->remote_console_access(undef);
```

router_ID( )

Use this method to set or retrieve the virtual router ID of an HA grid member. Required only when the member is an HA pair.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

Number with the router identification. Default value is undefined. Valid values should be between 1 and 255

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get router ID
my $router_ID = $grid_member->router_ID();
#Modify router ID
$grid_member->router_ID("34");
```

search_domains( )

Use this method to set or retrieve the Search Domain Group, which is a group of domain names that the Infoblox device can add to partial queries that do not specify a domain name. Note that you can set this parameter only when prefer_resolver or alternate_resolver is set.

Parameter

Valid value is a reference to an array of strings.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

Example

```
#Get search_domains
my $search_domains = $grid_member->search_domains();
#Modify search_domains
$grid_member->search_domains(["domain1.com", "domain2.com"]);
```

service_status( )

Use this method to retrieve the service status list of a grid member.

Returns

On retrieval, the member returns an array reference that contains Infoblox::Grid::Member::ServiceStatus objects.

Example

```
#Get the service status list
my $services_list = $grid_member->service_status();
```

service_type_configuration( )

Use this method to set or retrieve the configuration option for the member service type.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

The valid value is one of the following: "ALL_V4", "ALL_V6" or "CUSTOM".

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
# Get the value of service_type_configuration
my $service_type_configuration = $grid_member->service_type_configuration();
# Modify service_type_configuration
$grid_member->service_type_configuration("ALL_V6");
```

snmp_admin( )

Use this method to set or retrieve the Simple Network Management Protocol (SNMP) admin object of a grid member.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

Valid value is an Infoblox::Grid::SNMP::Admin object.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get SNMP admin
my $snmp_admin = $grid_member->snmp_admin();
#Modify SNMP admin
my $snmp_admin_obj = Infoblox::Grid::SNMP::Admin->new(
    sysContact_node1  => "Somebody",
    sysDescr_node1    => "SunOS noc1 5.6 Generic_105181-03 sun4u",
    sysLocation_node1 => "Infoblox Headquarter",
    sysName_node1     => "noc1"
);
$grid_member->snmp_admin($snmp_admin_obj);
#Undef snmp admin
$grid_member->snmp_admin(undef);
```

snmpv3_query_users( )

Use this method to set or retrieve the SNMPv3 user setting at the grid level.

Parameter

Valid value is a reference to the list of the Infoblox::Grid::SNMP::User manpage objects.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

Example

```
#Get snmpv3_query_users field
my $snmpv3_users = $gridmember->snmpv3_query_users();
my $user = $$snmpv3_users[0];
#Set snmpv3_query_users field
$gridmember->snmpv3_query_users([$user1, $user2]);
```

static_routes( )

Use this method to set or retrieve the static routes list of a grid member. This list contains the static routes of a grid member.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

Valid value is an array reference that contains Infoblox::Grid::Member::StaticRoute object(s).

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get static routes list
my $static_routes = $grid_member->static_routes();
#Modify static routes list
my $route = Infoblox::Grid::Member::StaticRoute->new(
   "network" => "192.168.2.0/24",
   "gateway" => "192.168.1.1",
);
$grid_member->static_routes([$route]);
```

support_access( )

Use this method to set or retrieve the support access flag of a grid member. This flag enables a Secure Shell (SSH) daemon that only Infoblox Technical Support can access to perform troubleshooting at the grid member level.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

Specify "true" to set the remote_console_access flag or "false" to deactivate/unset it. The default value is undefined.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get support access flag
my $support_access = $grid_member->support_access();
#Modify support access flag
$grid_member->support_access("true");
#Undef support access
$grid_member->support_access(undef);
```

support_access_info( )

Use this method to retrieve the information string for support access.

Parameter

None.

Returns

The method returns the attribute value.

Example

```
#Get support_access_info
my $support_access_info = $grid_member->support_access_info();
```

syslog_proxy_tcp_port( )

Use this method to set or retrieve the Transmission Control Protocol (TCP) listening port for syslog proxy.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

TCP port number. Default value is 514. Valid values should be between 1 and 65535.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get syslog proxy tcp port
my $syslog_proxy_tcp_port = $grid_member->syslog_proxy_tcp_port();
#Modify syslog proxy tcp port
$grid_member->syslog_proxy_tcp_port(514);
```

syslog_proxy_udp_port( )

Use this method to set or retrieve the User Datagram Protocol (UDP) listening port for syslog proxy.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

UDP port number. Default value is 514. Valid values should be between 1 and 65535.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get syslog proxy udp port
my $syslog_proxy_udp_port = $grid_member->syslog_proxy_udp_port();
#Modify syslog proxy udp port
$grid_member->syslog_proxy_udp_port(514);
```

syslog_size( )

Use this method to set or retrieve the grid member syslog rotation size in megabytes.

Parameter

Valid value is between 10 and 300.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get syslog_size
my $syslog_size = $obj->syslog_size();
#Modify syslog_size
$obj->syslog_size(100);
```

syslog_backup_servers( )

Use this method to set or retrieve an array that contains syslog backup server settings for a Grid member.

Setting this method to a defined value implicitly sets the override_syslog_backup_servers method to 'true'. Setting the parameter to undefined causes the appliance to use the Grid default and automatically resets the override_syslog_backup_servers attribute to 'false'.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

Valid value is an array reference that contains Infoblox::Grid::SyslogBackupServer objects.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#get syslog_backup_servers value
my $servers = $grid_member->syslog_backup_servers();

#modify syslog_backup_servers value
$grid_member->syslog_backup_servers([$server1, $server2]);
```

syslog_server( )

Use this method to set or retrieve the syslog server of a grid member.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

Valid value is an array reference that contains Infoblox::Grid::SyslogServer object(s).

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get syslog server
my $syslog_server = $grid_member->syslog_server();
#Modify syslog server
my $syslog1 = Infoblox::Grid::SyslogServer->new(
    ipv4addr => "44.44.44.44",
    severity => "debug",
);
my $syslog2 = Infoblox::Grid::SyslogServer->new(
    ipv4addr => "55.55.55.55",
    severity => "debug",
);

$grid_member->syslog_server([$syslog1, $syslog2]);
#Undef syslog server
$grid_member->syslog_server(undef);
```

time_zone( )

Use this method to set or retrieve the time zone of the grid member.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

The UTC string that represents the time zone. For example "(UTC - 5:00) Eastern Time (US and Canada)". Default value is undefined.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get time zone
my $time_zone = $grid_member->time_zone();
#Modify time zone
```

```
$grid_member->time_zone("(UTC - 5:00) Eastern Time (US and Canada)");
#Undef time zone
$grid_member->time_zone(undef);
```

threshold_traps( )

Use this method to retrieve the reference to the list of threshold traps. Each element in the list is a reference to the Infoblox::Grid::SNMP::ThresholdTrap object. If the member inherits traps from the grid, the threshold_traps value is undefined.

Setting this method to a defined value implicitly sets the override_threshold_traps method to "true". Setting the parameter to undefined causes the appliance to use the grid default and automatically resets the override_threshold_traps attribute to "false".

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

List of Infoblox::Grid::SNMP::ThresholdTrap objects.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get threshold traps
my $threshold_traps = $member->threshold_traps();
#Modify threshold traps
my $threshold_traps = $member->threshold_traps();
foreach(@$threshold_traps){if $_->trap_name() eq 'Rootfs'; $_->trap_reset=60;} #Change value for
'Rootfs' trap
$grid->threshold_traps($threshold_traps);
$session->modify($member);
```

traffic_capture( )

Use this method to control traffic capture for the specified member. Note that when you use this method, the object MUST be retrieved through a get or search call, or created with an add call.

Parameter

The method accepts an Infoblox::Grid::Member::Capture::Control object as a parameter.

Returns

When passing a capture control object as a parameter, the method returns true if the specified traffic control operation succeeds. It returns false when the operation fails.

If invoked without a parameter, the method returns a Infoblox::Grid::Member::Capture::Status object that describes the current traffic capture status.

Please see the status and control objects for usage examples.

trap_comm_string( )

Use this method to set or retrieve the Simple Network Management Protocol (SNMP) trap community string of the grid member. An SNMP trap community string is a text string that acts as a password. It is used to authenticate messages that are sent between the grid member and the SNMP management systems.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

Text with the Simple Network Management Protocol (SNMP) trap community, with a maximum of 50 bytes. Default value is undefined. To undefine trap_comm_string we also need to undefine "trap_receiver" attribute.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get trap community string
my $trap_comm_string = $grid_member->trap_comm_string();
#Modify trap community string
```

```
$grid_member->trap_comm_string("trap_string");
```

trap_notifications( )

Use this method to configure trap notifications. Each element in the list is a reference to the Infoblox::Grid::SNMP::TrapNotification object. If the trap_notifications value is undefined the member inherits trap notifications from the grid.

Explicitly setting this method to a defined value implicitly sets the override_trap_notifications method to "true". Setting the parameter to undefined causes the appliance to use the grid default and automatically resets the override_trap_notifications attribute to "false".

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

Reference to a list of Infoblox::Grid::SNMP::TrapNotification objects. The default value is a reference to a list of all trap notifications.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get a reference to a trap notifications array
my $trap_notifications = $member->trap_notifications();
#Modify trap notifications
my $notifications = $member->trap_notifications();
foreach(@$notifications){if $_->trap_type() eq 'CPU'; $_->enable_email="true";} #Change value for
'CPU' trap
$member->trap_notifications($notifications);
$session->modify($member);
```

trap_receiver( )

Use this method to set or retrieve the Simple Network Management Protocol (SNMP) trap receivers of a grid member.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

Valid value is an array reference that contains IPv4 addresses and the Infoblox::Grid::SNMP::TrapReceiver manpage objects. Default value is undefined. To undefine trap_receiver, you must also undefine the "trap_comm_string" attribute.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get trap receiver
my $trap_receiver = $grid_member->trap_receiver();
#Modify trap receiver (SNMPv2)
$grid_member->trap_receiver(["10.0.0.10"]);
#SNMPv3 trap receivers
my $receiver1 = Infoblox::Grid::SNMP::TrapReceiver->new( ip_addr => "19.20.21.22", user => $user);
$gridmember->trap_receiver([$receiver1]);
```

upgrade_group( )

Use this method to set or retrieve the name of the upgrade group to which this Grid member belongs.

Parameter

Valid value is a string representing the upgrade group name.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get upgrade_group
my $upgrade group   $obj  upgrade group
```

```
                    =     ->              ();
#Modify upgrade_group
$obj->upgrade_group("group2");
```

use_v4_vrrp( )

Use this method to switch between VRRPv4 and VRRPv6.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

Specify "true" to use VRRPv4 or "false" to use VRRPv6.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get use_v4_vrrp
my $use_v4_vrrp = $obj->use_v4_vrrp();
#Modify use_v4_vrrp
$obj->use_v4_vrrp("false");
```

vpn_on_mgmt( )

Use this method to set or retrieve the VPN services on the MGMT port of a grid member that is a single appliance.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Note that node1_vpn_on_mgmt, node2_vpn_on_mgmt and vpn_on_mgmt share the value; setting any of them will have the same effect.

Parameter

Specify "true" to set the vpn_on_mgmt flag or "false" to deactivate/unset it. The default value is "false".

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get VPN on MGMT port flag
my $vpn_on_mgmt = $grid_member->vpn_on_mgmt();
#Modify VPN on MGMT port flag
$grid_member->vpn_on_mgmt("false");
```

vlan_id( )

Use this method to set or retrieve the VLAN ID of a grid member.

Parameter

Valid value is an unsigned integer between 1 and 4095 inclusive. The default value is undefined.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get attribute value
my $attr = $grid_member->vlan_id();
#Modify attribute value
$grid_member->vlan_id(10);
```

create_token( )

Use this function to create the authorization token string for each physical node of the pre-provisioned member.

Parameter

None

Returns

The method returns a reference to an array of Infoblox::Grid::Member::PNodeToken objects.

Example

```
my $tokens = $grid_member->create_token();
```

read_token( )

Use this function to read the authorization token string for each physical node of the pre-provisioned member.

Parameter

None

Returns

The method returns a reference to an array of Infoblox::Grid::Member::PNodeToken objects. If tokens were not created previously, or have already expired, an empty array will be returned.

Example

```
my $tokens = $grid_member->read_token();
```

SAMPLE CODE

The following sample code demonstrates the different functions that can be applied to an object such as add, get, modify, search and remove. This sample also includes error handling for the operations.

#Preparation prior to getting and modifying a grid member object

```
use strict;
use Infoblox;

#refers to Infoblox Appliance IP address
my $host_ip = "192.168.1.2";

#Create a session to the Infoblox appliance

my $session = Infoblox::Session->new(
    master   => $host_ip,
    username => "admin",
    password => "infoblox"
);
unless ($session) {
       die("Construct session failed: ",
               Infoblox::status_code() . ":" . Infoblox::status_detail());
}
print "Session created successfully\n";

my $sysserv1 = Infoblox::Grid::SyslogServer->new(
    ipv4addr => "7.7.7.7"
);
unless ($sysserv1) {
       die("Construct sysserv1 failed: ",
               Infoblox::status_code() . ":" . Infoblox::status_detail());
}
print "Sysserv1 created successfully\n";
```

#Create a grid member object

```
my $grid_member = Infoblox::Grid::Member->new(
    name          => "hostname.com",
    ipv4addr      => "3.0.0.2",
    mask          => "255.0.0.0",
    gateway       => "3.0.0.1",
    syslog_server => [$sysserv1],
    comment       => "Script Generated"
    );
unless ($grid_member) {
       die("Construct grid member object failed: ",
               Infoblox::status_code() . ":" . Infoblox::status_detail());
}
```

```
print "Grid member object created successfully\n";

#Adding the member object to appliance through session
my $response = $session->add( $grid_member );

unless ($response) {
    die("Add grid member failed: ",
            $session->status_code() . ":" . $session->status_detail());
}
print"Grid member added successfully \n";
```

#Search for a grid member object

```
 my @retrieved_objs = $session->search(
     object => "Infoblox::Grid::Member",
     name   => ".*\\.com");
 my $object = $retrieved_objs[0];

 unless ($object) {
        die("Search grid member object failed: ",
                $session->status_code() . ":" . $session->status_detail());
 }
 print "Search grid member object found at least 1 matching entry\n";
```

#Get and modify the member object

```
#Get grid member through session
my @retrieved_objs = $session->get(
    object => "Infoblox::Grid::Member",
    name   => "hostname.com"
    );
my $grid_member = $retrieved_objs[0];

unless ($grid_member) {
    die("Get grid member object failed: ",
            $session->status_code() . ":" . $session->status_detail());
}
print"Get grid member successful \n";

# Modify one of the attributes of selected grid member.

$grid_member->comment("Modified member object");

#Applying the changes
$session->modify($grid_member)
   or die("Modify grid member failed",
            $session->status_code() . ":" . $session->status_detail());
print"Grid member modified successfully \n";
```

#Remove the grid member object

```
#Get grid member through session
my @retrieved_objs = $session->get(
    object => "Infoblox::Grid::Member",
    name   => "hostname.com"
    );
my $desired_grid_member = $retrieved_objs[0];

unless ($desired_grid_member) {
    die("Get grid member object failed: ",
            $session->status_code() . ":" . $session->status_detail());
}
print"Get grid member successful \n";

#Submit the object for removal
$session->remove( $desired_grid_member )
    or die("Remove grid member failed",
            $session->status_code() . ":" . $session->status_detail());
print"Grid member removed successfully \n";

####PROGRAM ENDS####
```

AUTHOR

Infoblox Inc. http://www.infoblox.com/

SEE ALSO

Infoblox::Grid, Infoblox::Grid::NTPKey, Infoblox::Grid::SNMP::Admin, Infoblox::Grid::SyslogServer, Infoblox::Session, Infoblox::Grid::NamedACL, Infoblox::Grid::SyslogBackupServer Infoblox::Grid::Member::OSPF,Infoblox::Grid::LOM::User, Infoblox::DNS::Member, Infoblox::DNS::Nameserver,

---

COPYRIGHT

Copyright (c) 2017 Infoblox Inc.

Infoblox::DNS::Record::NS - DNS NS record object

- NAME
- DESCRIPTION
- CONSTRUCTOR
- SESSION METHODS
  - Infoblox::Session->add( )
  - Infoblox::Session->get( )
  - Infoblox::Session->modify( )
  - Infoblox::Session->remove( )
  - Infoblox::Session->search( )
- METHODS
  - cloud_info( )
  - creator( )
  - dns_name( )
  - ms_delegation_name( )
  - last_queried( )
  - name( )
  - nameserver( )
  - addresses( )
  - views( )
- SAMPLE CODE
- AUTHOR
- SEE ALSO
- COPYRIGHT

NAME

Infoblox::DNS::Record::NS - DNS NS record object

DESCRIPTION

An NS record identifies an authoritative DNS server for a domain. Each authoritative DNS server must have an NS record. The appliance automatically creates an NS record when you assign a grid member as the primary server for a zone. You can manually create NS records for other zones.

Note that you must specify only one DNS view for the attribute "views".

CONSTRUCTOR

```
my $ns = Infoblox::DNS::Record::NS->new(
    name                => $name,                   #Required
    nameserver          => $nameserver,             #Required
    addresses           => [$ns1, $ns2, ...],       #Required
    ms_delegation_name  => $name,                   #Optional
    views               => [$View1]                 #Optional / Default is "default" view
);
```

SESSION METHODS

This section describes all the methods in Infoblox::Session module that you can apply to a DNS NS Record object.

Infoblox::Session->add( )

Use this method to add an object to the Infoblox appliance. See Infoblox::Session->add() for parameters and return values.

Example

```
#Construct an object
my $nsrecord = Infoblox::DNS::Record::NS->new(
    nameserver => "ns1.server.isp.org",
    name       => "domain.com",
    addresses  => [$ns1, $ns2],
);
# Submit for addition
my $response = $session->add( $nsrecord );
```

Infoblox::Session->get( )

Use this method to retrieve all the matching objects from the Infoblox appliance. See Infoblox::Session->get() for parameters and return values.

Key References

```
Apply the following attributes to get a specific DNS NS Record object:

 name        - Optional.  A domain name in FQDN (Fully Qualified Domain Name) format.
 nameserver  - Optional.  A domain name in FQDN (Fully Qualified Domain Name) format.
 view        - Optional.  The DNS view in which the NS record is located. By default, all DNS views
are searched.
```

Example

```
 my @retrieved_objs = $session->get(
    object    => "Infoblox::DNS::Record::NS",
    name      => "domain.com",
    view      => "default" );
```

Infoblox::Session->modify( )

Use this method to modify an object in the Infoblox appliance. See Infoblox::Session->modify() for parameters and return values.

Example

```
#Use method to modify the addresses.
$nsrecord->addresses([$ns3]);
# Submit modification
my $response = $session->modify( $nsrecord );
```

Infoblox::Session->remove( )

Use this method to remove an object from the Infoblox appliance. See Infoblox::Session->remove() for parameters and return values.

To remove a specific object, first use `get()` or `search()` to retrieve the specific object, and then submit this object for removal.

Example

```
# Get the object under the same name
my @retrieved_objs = $session->get(
    object    => "Infoblox::DNS::Record::NS",
    name      => "domain.com",
    view      => "default" );
# Find the desired object from the retrieved list.
my $desired_ns = $retrieved_objs[0];
# Submit for removal
my $response = $session->remove( $desired_ns );
```

Infoblox::Session->search( )

Use this method to search for DNS NS Record objects in the Infoblox appliance. See Infoblox::Session->search() for parameters and return values.

Key References

```
Apply the following attributes to search for a DNS NS Record object:

  name          - Optional. A domain name in FQDN (Fully Qualified Domain Name) format (regular
expression).
  nameserver    - Optional. A domain name in FQDN (Fully Qualified Domain Name) format (regular
expression).
  view          - Optional. The DNS view in which the NS record is located. By default, all DNS views
are searched.
```

Example

```
# search for all DNS NS objects that match "domain.com" in the default DNS view
    my @retrieved_objs = $session->search(
    object => "Infoblox::DNS::Record::NS",
    name   => 'domain\.com',
    view   => "default" );
```

METHODS

This section describes all the methods that you can use to configure and retrieve the attribute values of an NS record.

cloud_info( )

Use this method to retrieve cloud API related information for the Infoblox::DNS::Record::NS object.

Parameter

None

Returns

The method returns the attribute value.

Example

```
# Get cloud_info
my $cloud_info = $object->cloud_info();
```

creator( )

Use this method to retrieve the NS record creator. This is a read-only attribute.

Parameter

None

Returns

The method returns the attribute value.

Example

```
# Get attribute value
my $value = $object->creator();
```

dns_name( )

Use this method to retrieve the domain name, in punycode format, of the zone to be redirected. This is a read-only attribute.

Parameter

None

Returns

The method returns the attribute value.

Example

```
# Get attribute value
my $value = $nsrecord->dns_name();
```

ms_delegation_name( )

Use this method to set or retrieve the ms_delegation_name field of the NS record. This method can be used if the primary server of the zone is a Microsoft (r) server. If this NS record is for a subzone zone, it will be the subzone name.

Parameter

A subzone name.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Getting ms_delegation_name
my $ms_delegation_name = $nsrecord->ms_delegation_name();
# Modifying ms_delegation_name (the full FQDN of the zone is "sub.test.com")
$nsrecord->ms_delegation_name("sub");
```

last_queried( )

Use this method to retrieve the time when the associated record was last queried. This is a read-only attribute.

Parameter

None

Returns

The method returns the attribute value. The number of seconds that have elapsed since January 1st, 1970 UTC.

Example

```
#Get last_queried
my $last_queried = $nsrecord->last_queried();
```

name( )

Use this method to set or retrieve the domain name of the zone to be redirected.

The attribute value can be in unicode format.

Parameter

A domain name in FQDN format.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Getting name
my $name = $nsrecord->name();
# Modifying name
$nsrecord->name("domain.com");
```

nameserver( )

Use this method to set or retrieve the domain name of an authoritative server for the redirected zone.

Parameter

A domain name in FQDN format.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Getting nameserver
my $nameserver = $nsrecord->nameserver();
# Modifying nameserver
$nsrecord->nameserver("ns1.server.isp.org");
```

addresses( )

Use this method to set or retrieve the IP addresses of the name servers associated with the NS record.

Parameter

The valid value is an array reference that contains Infoblox::DNS::Nameserver::Address objects.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Getting addresses
my $nslist = $nsrecord->addresses();
# Modifying addresses
$nsrecord->addresses([$ns1, $ns2]);
```

views( )

Use this method to specify or retrieve the DNS view of the NS record.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

The default value is the 'default' view, which means the NS record is associated with a zone in the system-defined default view.

Parameter

Array reference of defined Infoblox::DNS::View objects.

Note that the array size must be 1.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get views
my $ref_views = $nsrecord->views();
#Modify views, list of Infoblox::DNS::View objects
$nsrecord->views([$view1]);
```

SAMPLE CODE

The following sample code demonstrates the different functions that can be applied to an object, such as add, search, modify, and remove. This sample also includes error handling for the operations.

#Preparation prior to a DNS NS record insertion

```
#PROGRAM STARTS: Include all the modules that will be used
use strict;
use Infoblox;

#Create a session to the Infoblox appliance
my $session = Infoblox::Session->new(
    master   => "192.168.1.2",
    username => "admin",
    password => "infoblox"
);
unless ($session) {
   die("Construct session failed: ",
       Infoblox::status_code() . ":" . Infoblox::status_detail());
}
print "Session created successfully\n";

#Create the zone prior to NS record insertion
my $zone = Infoblox::DNS::Zone->new(name => "domain.com");
unless ($zone) {
   die("Construct zone failed: ",
     Infoblox::status_code() . ":" . Infoblox::status_detail());
}
print "Zone object created successfully\n";

#Verify if the zone exists
my $object = $session->get(object => "Infoblox::DNS::Zone", name => "domain.com");
unless ($object) {
   print "Zone does not exist on server, safe to add the zone\n";
   $session->add($zone)
      or die("Add zone failed: ",
             $session->status_code() . ":" . $session->status_detail());
}
print "Zone added successfully\n";
```

#Create a DNS NS record

```
my $ns1 = Infoblox::DNS::Nameserver::Address->new(
                                                  address  => '10.0.0.1',
                                            auto_create_ptr => 'false'
                                            );

my $ns2 = Infoblox::DNS::Nameserver::Address->new(
                                            address         => '10.0.0.2',
                                            );

 #Construct a DNS NS object
  my $nsrecord = Infoblox::DNS::Record::NS->new(
```

```
    nameserver => "ns1.server.isp.org",
    name       => "domain.com",
    addresses  => [$ns1, $ns2],
  );

 unless ($nsrecord) {
   die("Construct DNS record NS failed: ",
       Infoblox::status_code() . ":" . Infoblox::status_detail());
 }
 print "DNS NS object created successfully\n"; #Add the DNS NS record object to Infoblox appliance through
a session
 $session->add($nsrecord)
    or die("Add record NS failed: ",
           $session->status_code() . ":" . $session->status_detail());
 print "DNS NS object added to server successfully\n";
```

#Search for a specific DNS NS record

```
 #Search all NS records that match "domain.com"
 my @retrieved_objs = $session->search(
    object     => "Infoblox::DNS::Record::NS",
    nameserver => "ns1.server.isp.org",
    name       => 'domain\.com'
 );
 my $object = $retrieved_objs[0];
 unless ($object) {
    die("Search record NS failed: ",
        $session->status_code() . ":" . $session->status_detail());
 }
 print "Search DNS NS object found at least 1 matching entry\n";
```

#Get and modify a DNS NS record

```
 #Get NS record through the session
 my  @retrieved_objs = $session->get(
    object     => "Infoblox::DNS::Record::NS",
    nameserver => "ns1.server.isp.org",
    name       => "domain.com",
 );

 my $object = $retrieved_objs[0];
 unless ($object) {
    die("Get record NS failed: ",
        $session->status_code() . ":" . $session->status_detail());
 }
 print "Get DNS NS object found at least 1 matching entry\n";

 #Modify one of the attributes of the specified NS record
 $object->addresses([$ns1]);

 #Apply the changes
 $session->modify($object)
    or die("Modify record NS failed: ",
           $session->status_code() . ":" . $session->status_detail());
 print "DNS NS object modified successfully \n";
```

#Remove a DNS NS record

```
  #Get NS record through the session
  my @retrieved_objs = $session->get(
     object     => "Infoblox::DNS::Record::NS",
     nameserver => "ns1.server.isp.org",
     name       => "domain.com"
   );
    my $object = $retrieved_objs[0];
    unless ($object) {
     die("Get record NS failed: ",
         $session->status_code() . ":" . $session->status_detail());
   }
    print "Get DNS NS object found at least 1 matching entry\n";

 #Submit the object for removal
 $session->remove($object)
    or die("Remove record NS failed: ",
        $session->status_code() . ":" . $session->status_detail());
 print "DNS NS object removed successfully \n";
```

```
####PROGRAM ENDS####
```

AUTHOR

Infoblox Inc. http://www.infoblox.com/

SEE ALSO

Infoblox::DNS::View, Infoblox::DNS::Zone, Infoblox::Session, Infoblox::Session->get(), Infoblox::Session->search(), Infoblox::Session->add(), Infoblox::Session->remove(), Infoblox::Session->modify()

COPYRIGHT

Copyright (c) 2017 Infoblox Inc.

Infoblox::DNS::Record::DS - DS Record object.

- NAME
- DESCRIPTION
- CONSTRUCTOR
- SESSION METHODS
  - Infoblox::Session->get( )
  - Infoblox::Session->remove( )
  - Infoblox::Session->search( )
- METHODS
  - algorithm( )
  - cloud_info( )
  - creator( )
  - digest( )
  - digest_type( )
  - dns_name( )
  - key_tag( )
  - last_queried( )
  - name( )
  - ttl( )
  - view( )
  - zone( )
- SAMPLE CODE
- AUTHOR
- SEE ALSO
- COPYRIGHT

NAME

Infoblox::DNS::Record::DS - DS Record object.

DESCRIPTION

The DS key record is a part of the DNS security extension records. It is used to authorize the DNSKEY records of the child zone and thus to establish the DNSSEC chain of trust.

The DS resource record is defined in RFC 4034.

The DS resource records are automatically generated upon the signing of the child zone of an authoritative zone residing on the appliance. You can also upload the external DS records via Infoblox::Session->import_data(). It is not possible to add, remove or modify automatically generated DS records. However, the records imported with Infoblox::Session->import_data() method can be deleted.

CONSTRUCTOR

The Infoblox::DNS::Record::DS object is a read-only object and does not require manual construction.

SESSION METHODS

This section describes all the methods in an Infoblox::Session that can be applied to an DS Record object.

Infoblox::Session->get( )

Use this method to retrieve the existing objects from an Infoblox appliance. See Infoblox::Session->get() for parameters and return values.

Key Reference

```
 Apply the following attributes to get the DS Record object(s):

     algorithm       - Optional. The encryption algorithm of the corresponding DNSKEY record.
                       The allowed values are
'1','3','5','6','7','RSAMD5','RSASHA1','DSA','NSEC3RSASHA1','NSEC3DSA','RSASHA256','RSASHA512','NSEC3RSASHA256'
 and 'NSEC3RSASHA512'.
     digest          - Optional. The digest of the DNSKEY RR.
     digest_type     - Optional. Identifies the algorithm used to construct the digest. Allowed values are
'SHA1', 'SHA256', 1, 2.
     key_tag         - Optional. Key-tag to identify the public key.
     name            - Optional. FQDN of the DS record.
     ttl             - Optional. TTL value of the record.
     view            - Optional. The DNS view in which the record is located. By default, all DNS views are
searched. But if you omit this attribute and specify a zone, the appliance searches the 'default' view only.
     zone            - Optional. A zone name in FQDN format.
```

Example

```
 # get all the DS records for the zone 'domain.com' in the default DNS view
 my @retrieved_objs = $session->get(
                      object => "Infoblox::DNS::Record::DS",
                      name   => "domain.com",
                      view   => "default" );
```

```
# get all the DS records in the "domain.com" zone of the default DNS view
# with digest type "SHA1" and TTL of 1 day:
my @retrieved_objs = $session->get(
                     object     => "Infoblox::DNS::Record::DS",
                     name       => "domain.com",
                     digest_type => "SHA1",
                     ttl        =>  86400,
                     view       => "default" );
```

Infoblox::Session->remove( )

Use this method to submit an object for removal from the Infoblox appliance. See Infoblox::Session->remove() for parameters and return values.

To remove a specific object, first use `get()` or `search()` to retrieve the object, and then submit it for removal.

Example

```
# Get the objects with the same name
my @retrieved_objs = $session->get(
    object => "Infoblox::DNS::Record::DS",
    name   => "domain.com",
    view   => "default" );
# Find the desired object from the retrieved list.
my $desired_ds = $retrieved_objs[0];
# Submit for removal
my $response = $session->remove( $desired_ds );
```

Infoblox::Session->search( )

Use this method to retrieve the existing objects from an Infoblox appliance. See Infoblox::Session->search() for parameters and return values.

Key Reference

```
Apply the following attributes to search the DS Record object(s):
    algorithm       - Optional. The encryption algorithm of the corresponding DNSKEY record.
                      The allowed values are
'1','3','5','6','7','RSAMD5','RSASHA1','DSA','NSEC3RSASHA1','NSEC3DSA','RSASHA256','RSASHA512','NSEC3RSASHA256'
and 'NSEC3RSASHA512'.
    digest          - Optional. Digest of the DNSKEY RR (regular expression).
    digest_type     - Optional. Identifies the algorithm used to construct the digest. Allowed values are
'SHA1', 'SHA256', 1, 2.
    key_tag         - Optional. Key-tag to identify the public key.
    name            - Optional. FQDN of the DS record (regular expression).
    ttl             - Optional. TTL value of the record.
    view            - Optional. The DNS view in which the record is located. By default, all DNS views are
searched. But if you omit this attribute and specify a zone, the appliance searches the 'default' view only.
    zone            - Optional. A zone name in FQDN format.
```

Example

```
# search the DS records in the default DNS view by zone name.
my @retrieved_objs = $session->search(
                     object => "Infoblox::DNS::Record::DS",
                     name   => ".*[.]com",
                     view   => "default" );

# search for all DS records in the "domain.com" zone of the default DNS view
# Note that the 'default' view is assumed implicitly here
my @retrieved_objs = $session->search(
    object => "Infoblox::DNS::Record::DS",
    zone   => "domain.com",
   );
```

METHODS

This section describes all the methods that can be used to retrieve the attribute values of a DS Record object.

algorithm( )

Use this method to retrieve the public key encryption algorithm of the DNSKEY record of the given DS record.

Parameter

none

Returns

The method returns the public key encryption algorithm attribute value. Returned values are one of the following:

RSAMD5
DSA
RSASHA1
NSEC3DSA
NSEC3RSASHA1

RSASHA256
RSASHA512
Example

```
#Get the public key encryption algorithm.
my $algorithm = $ds_record->algorithm();
```

cloud_info( )

Use this method to retrieve cloud API related information for the Infoblox::DNS::Record::DS object.

Parameter

None

Returns

The method returns the attribute value.

Example

```
# Get cloud_info
my $cloud_info = $object->cloud_info();
```

creator( )

Use this method to retrieve the DS record creator. This is a read-only attribute.

Parameter

None

Returns

The method returns the attribute value.

Example

```
# Get attribute value
my $value = $object->creator();
```

digest( )

Use this method to retrieve the digest of the DNSKEY resource record that is stored in a DS Record object.

Parameter

none

Returns

The method returns the digest of the corresponding DNSKEY RR attribute value.

Example

```
#Get the digest of the DNSKEY RR
my $digest = $ds_record->digest();
```

digest_type( )

Use this method to retrieve the algorithm that is used to construct the digest stored in the digest field.

Parameter

none

Returns

The method returns the algorithm that is used to construct the digest attribute value.

Example

```
#Get the algorithm used to construct the digest.
my $digest_type = $ds_record->digest_type();
```

dns_name( )

Use this method to retrieve the FQDN, in punycode format, of the DNSKEY record referred by the DS Record object. This is a read-only attribute.

Parameter

None

Returns

The method returns the attribute value.

Example

```
# Get attribute value
my $value = $ds_record->dns_name();
```

key_tag( )

Use this method to retrieve the key tag of the DNSKEY RR that is referred by the given DS Record object.

Parameter

none

Returns

The method returns the key tag attribute value. The key tag is used to identify the public keys that belong to the different DS records. The returned value is an unsigned 16 bit integer.

Example

```
#Get key-tag
my $key_tag = $ds_record->key_tag();
```

last_queried( )

Use this method to retrieve the time when the associated record was last queried. This is a read-only attribute.

Parameter

None

Returns

The method returns the attribute value. The number of seconds that have elapsed since January 1st, 1970 UTC.

Example

```
#Get last_queried
my $last_queried = $ds_record->last_queried();
```

name( )

Use this method to retrieve the FQDN of the DNSKEY record referred by the DS Record object.

The attribute value can be in unicode format.

Parameter

none

Returns

The method returns the FQDN of the DNSKEY record that corresponds to the DS record.

Example

```
#Get  FQDN of the DS record
my $name = $ds_record->name();
```

ttl( )

Use this method to retrieve the Time to Live (TTL) value of a DS Record object.

Parameter

none

Returns

The method returns the TTL attribute value. The returned parameter is a 32-bit integer (range from 0 to 4294967295) that represents the duration, in seconds, that the record is cached. Zero indicates that the record should not be cached.

Example

```
#Get  TTL
my $ttl = $ds_record->ttl();
```

view( )

Use this method to retrieve the DNS view object that contains the DS Record object.

Parameter

none

Returns

The method returns the Infoblox::DNS::View object that contains the DS record.

Example

```
#Get  Ref to the view object
my $view = $ds_record->view();
```

zone( )

Use this method to retrieve the zone name of a DS record.

Parameter

none

Returns

Returns the zone name in FQDN format.

Example

```
# Get zone
my $zone = $ds_record->zone();
```

SAMPLE CODE

The following sample code demonstrates the session methods on a DS Record object.

```
#PROGRAM STARTS: Include all the modules that will be used
use strict;
use Infoblox;

#Create a session to the Infoblox device
       my $session = Infoblox::Session->new(
           master   => "192.168.1.2",
           username => "admin",
           password => "infoblox"
       );
       unless ($session) {
          die("Construct session failed: ",
              $session->status_code() . ":" . $session->status_detail());
       }
       print "Session created successfully\n";

       #Enable DNSSEC in the default view

       my $default_view=$session->get(
                                    object=> "Infoblox::DNS::View",
                                    name => "default"
                                   );
       unless($default_view) {
           die("Getting the default view failed:",
              Infoblox::status_code() . ":" . Infoblox::status_detail());
       }
       print "Got the default view successfully\n";

       $default_view->dnssec_enabled("true")
           or die("Changing the dnssec_enabled in the default view failed:",
                    Infoblox::status_code() . ":" . Infoblox::status_detail());
       $session->modify($default_view)
           or die("Changing the dnssec_enabled in the default view failed:",
                    Infoblox::status_code() . ":" . Infoblox::status_detail());

       #Create the zone

       print "Creating Member primary server for the zone\n";
       my $primary=Infoblox::DNS::Member->new(
                                           ipv4addr => "192.168.1.2",
                                           name => "infoblox.localdomain"
                                          );
       unless($primary) {
          die("Unable to create primary server object: ",
              Infoblox::status_code() . ":" . Infoblox::status_detail());
       }

       my $zone = Infoblox::DNS::Zone->new(
```

```
                                        name => "domain.com",
                                        primary => $primary
                                      );
unless ($zone) {
   die("Construct zone failed: ",
       Infoblox::status_code() . ":" . Infoblox::status_detail());
}
print "Zone object created successfully\n";

#Verify if the zone exists
my $object = $session->get(object => "Infoblox::DNS::Zone", name => "domain.com");
unless ($object) {
   print "Zone does not exist on server, safe to add the zone\n";
   $session->add($zone)
      or die("Add zone failed: ",
             $session->status_code() . ":" . $session->status_detail());
}
print "Zone added successfully\n";

#Retrieving zone back from the server in order to sign it
$zone = $session->get(object => "Infoblox::DNS::Zone", name => "domain.com");
unless($zone) {
   die("Retrieving zone back failed: ",
        Infoblox::status_code( ). ":". Infoblox::status_detail( ));
}
print "Zone retrieved for signing successfully.\n";

$zone->dnssec_ksk_algorithm("NSEC3RSASHA1") &&
$zone->dnssec_zsk_algorithm("NSEC3RSASHA1") &&
$zone->dnssec_ksk_size(640) &&
$zone->dnssec_zsk_size(640)
  or die("Changing the zone DNSSEC setting failed: ",
         Infoblox::status_code() . ":" . Infoblox::status_detail());
$session->modify($zone)
  or die("Modifying dnssec values in zone failed: ",
         Infoblox::status_code() . ":" . Infoblox::status_detail());
print "Zone modified successfully\n";

#Signing the zone
$zone->dnssec_signed("true")
   or die("Signing of the zone failed: ",
          Infoblox::status_code() . ":" . Infoblox::status_detail());
       print "Zone signed successfully\n";

# In order to have DS records in the parent zone we need to have a signed subzone
my $subzone = Infoblox::DNS::Zone->new(
                                        name => "subzone.domain.com",
                                        primary => $primary
                                      );
unless ($subzone) {
   die("Construct subzone failed: ",
       Infoblox::status_code() . ":" . Infoblox::status_detail());
}
print "Zone object created successfully\n";

#Verify if the subzone exists
my $object = $session->get(object => "Infoblox::DNS::Zone", name => "subzone.domain.com");
unless ($object) {
   print "Subzone does not exist on server, safe to add the subzone\n";
   $session->add($subzone)
      or die("Add subzone failed: ",
             $session->status_code() . ":" . $session->status_detail());
}
print "Subzone added successfully\n";

#Retrieving subzone back from the server in order to sign it
$subzone = $session->get(object => "Infoblox::DNS::Zone", name => "subzone.domain.com");
unless($subzone) {
   die("Retrieving subzone back failed: ",
        Infoblox::status_code( ). ":". Infoblox::status_detail( ));
}
print "Subzone retrieved for signing successfully.\n";

$subzone->dnssec_ksk_algorithm("NSEC3DSA") &&
$subzone->dnssec_zsk_algorithm("NSEC3DSA") &&
$subzone->dnssec_ksk_size(640) &&
$subzone->dnssec_zsk_size(640)
  or die("Changing the subzone DNSSEC setting failed: ",
         Infoblox::status_code() . ":" . Infoblox::status_detail());
$session->modify($subzone)
  or die("Modifying dnssec values in subzone failed: ",
         Infoblox::status_code() . ":" . Infoblox::status_detail());
```

```
print "Subzone modified successfully\n";

#Signing the subzone
$subzone->dnssec_signed("true")
   or die("Signing of the subzone failed: ",
            Infoblox::status_code() . ":" . Infoblox::status_detail());
        print "Subzone signed successfully\n";

#Getting the DNSKEY record for subzone
my $ksk_dnskey=$session->get(
                             object => "Infoblox::DNS::Record::DNSKEY",
                             flags  => 257,
                             name   => "subzone.domain.com",
                             view   => "default"
                            );
     unless($ksk_dnskey) {
  die("Getting ksk_dnskey failed: ",
      Infoblox::status_code() . ":" . Infoblox::status_detail());
      }
      print "Got Key-signing key DNSKEY record successfully\n";
```

#Retrieving the DS record corresponding to the given DNSKEY

```
my $ds_record=$session->get(
                             object => "Infoblox::DNS::Record::DS",
                             name   => "subzone.domain.com",
                             key_tag => $ksk_dnskey->key_tag(),
                             view   => "default"
                            );
unless($ds_record) {
   die("Getting DS record corresponding to ksk_dnskey failed: ",
       Infoblox::status_code() . ":" . Infoblox::status_detail());
}
print "Got the DS record corresponding to public key with key_tag ".$ksk_dnskey->key_tag()." successfully\n";
print "Digest value: ".$ds_record->digest()."\n";
```

#Searching for DS object

```
my @retrieved_objs=$session->search(
                                    object => "Infoblox::DNS::Record::DS",
                                    name   => ".*com",
                                    view   => "default"
                                   );
     unless(@retrieved_objs>0) {
   die("Searching for DS objects failed: ",
       Infoblox::status_code() . ":" . Infoblox::status_detail());
     }
     print scalar(@retrieved_objs)." DS objects successfully found by search\n";
```

#Removing the created zones and cleaning up the view

```
$session->remove($subzone)
  or die("Unable to remove the subzone: ",
           Infoblox::status_code() . ":" . Infoblox::status_detail());
print "Subzone removed successfully\n";

$session->remove($zone)
  or die("Unable to remove the zone: ",
           Infoblox::status_code() . ":" . Infoblox::status_detail());
print "Zone removed successfully\n";

$default_view->dnssec_enabled("false")
&& $default_view->override_dnssec("false")
&& $session->modify($default_view)
 or die("Restoring dnssec_enabled value in the default view failed: ",
        Infoblox::status_code() . ":" . Infoblox::status_detail());

####PROGRAM ENDS####
```

AUTHOR

Infoblox Inc. http://www.infoblox.com/

SEE ALSO

Infoblox::Session, Infoblox::Session->get(), Infoblox::Session->search(), Infoblox::Session->remove(), Infoblox::DNS::Record::DNSKEY, Infoblox::DNS::View, Infoblox::DNS::Zone

---

COPYRIGHT

Copyright (c) 2017 Infoblox Inc.

Infoblox::DNS::Record::RRSIG - RRSIG Record object

- NAME
- DESCRIPTION
- CONSTRUCTOR
- SESSION METHODS
  - Infoblox::Session->get( )
  - Infoblox::Session->search( )
- METHODS
  - algorithm( )
  - cloud_info( )
  - creator( )
  - dns_name( )
  - dns_signer_name( )
  - expiration( )
  - inception( )
  - key_tag( )
  - labels( )
  - name( )
  - original_ttl( )
  - signature( )
  - signer_name( )
  - ttl( )
  - type_covered( )
  - view( )
  - zone( )
- SAMPLE CODE
- AUTHOR
- SEE ALSO
- COPYRIGHT

NAME

Infoblox::DNS::Record::RRSIG - RRSIG Record object

DESCRIPTION

RRSIG records are one of the resource records in DNSSEC. These records store digital signatures of resource record sets (RRsets). The digital signatures are used to authenticate the data that is in the signed RRsets.

RRSIG resource records are defined in RFC 4034.

RRSIG records are automatically generated upon the signing of an authoritative zone. The RRSIG record-object is read-only.

CONSTRUCTOR

The Infoblox::DNS::Record::RRSIG object is a read-only object and does not require manual construction.

SESSION METHODS

This section describes all the methods in an Infoblox::Session that can be applied to an RRSIG record object.

Infoblox::Session->get( )

Use this method to retrieve the existing objects from an Infoblox appliance. See Infoblox::Session->get() for parameters and return values.

Key Reference

```
    Apply the following attributes to get the RRSIG Record object(s):

     algorithm       - Optional. The public key encryption algorithm. The allowed values are
'1','3','5','6','7','RSAMD5','RSASHA1','DSA','NSEC3RSASHA1','NSEC3DSA','RSASHA256','RSASHA512','NSEC3RSASHA256'
 and 'NSEC3RSASHA512'.
     expiration      - Optional. Signature expiration time, in seconds, since 1 January 1970 00:00:00 UTC.
     name            - Optional. The name of the corresponding RRset.
     inception       - Optional. Signature inception time, in seconds, since 1 January 1970 00:00:00 UTC.
     key_tag         - Optional. Key tag identifying the corresponding public key in the DNSKEY record.
     labels          - Optional. Specifies the number of labels in the original RRSIG RR owner name.
     original_ttl    - Optional. TTL of the covered RRset as it appears in the authoritative zone.
     signature       - Optional. Cryptographic signature that covers the RRSIG RDATA.
     signer_name     - Optional. Domain name of the zone that contains the signed RRset.
     ttl             - Optional. TTL value of the record itself.
     type_covered    - Optional. Identifies the type of the RRset that is covered by this RRSIG record.
     view            - Optional. The DNS view in which the record is located. By default, all DNS views are
searched. But if you omit this attribute and specify a zone, the appliance searches the 'default' view only.
     zone            - Optional. A zone name in FQDN format.
```

Example

```
# get all the RRSIG records for the zone 'domain.com' in the default DNS view.
my @retrieved_objs = $session->get(
                     object => "Infoblox::DNS::Record::RRSIG",
                     signer_name   => "domain.com",
                     view   => "default" );

# get the RRSIG record that corresponds to the 'test.domain.com' A record of the default DNS view
my $retrieved_obj = $session->get(
                     object    => "Infoblox::DNS::Record::RRSIG",
                     name      => "test.domain.com",
                     type_covered => "A",
                     view      => "default" );
```

Infoblox::Session->search( )

Use this method to retrieve the existing objects from an Infoblox appliance. See Infoblox::Session->search() for parameters and return values.

Key Reference

```
 Apply the following attributes to search the RRSIG Record object(s):

     algorithm       - Optional. The public key encryption algorithm. The allowed values are
'1','3','5','6','7','RSAMD5','RSASHA1','DSA','NSEC3RSASHA1','NSEC3DSA','RSASHA256','RSASHA512','NSEC3RSASHA256'
 and 'NSEC3RSASHA512'.
     expiration      - Optional. Signature expiration time, specified as seconds since 1 January 1970
00:00:00 UTC. This method supports searching with limit parameters (see below).
     name            - Optional. The name of the corresponding RRset (regular expression).
     inception       - Optional. Signature inception time, specified as seconds since 1 January 1970 00:00:00
UTC. This method supports searching with limit parameters (see below).
     key_tag         - Optional. Key tag that identifies the corresponding public key in the DNSKEY record.
     labels          - Optional. Specifies the number of labels in the original RRSIG RR owner name.
     original_ttl    - Optional. TTL of the covered RRset as it appears in the authoritative zone.
     signature       - Optional. Cryptographic signature that covers the RRSIG RDATA (regular expression).
     signer_name     - Optional. Domain name of the zone that contains the signed RRset (regular expression).
     ttl             - Optional. TTL value of the record itself.
     type_covered    - Optional. Identifies the type of the RRset that is covered by this RRSIG record
(regular expression).
     view            - Optional. The DNS view in which the record is located. By default, all DNS views are
searched. But if you omit this attribute and specify a zone, the appliance searches the 'default' view only.
     zone            - Optional. A zone name in FQDN format.
```

See Infoblox::Session/search with limit parameters for more information on this functionality.

Example

```
# get all the RRSIG records for the zone matching the regular expression.
my @retrieved_objs = $session->search(
                     object => "Infoblox::DNS::Record::RRSIG",
                     signer_name   => ".*[.]com",);

# get all the RRSIG records with inception time between 12:00 September 1, 2009 and 12:00 September 10,
2009, expiring no later then January 1, 2010.
my @retrieved_objs = $session->search(
                     object           => "Infoblox::DNS::Record::RRSIG",
                     inception        => ">=< 1254398400,1255176000",
                     expiration       => "<= 1264982400");

# search for all RRSIG records in the "domain.com" zone of the default DNS view
# Note that the 'default' DNS view is assumed implicitly here
my @retrieved_objs = $session->search(
    object => "Infoblox::DNS::Record::RRSIG",
    zone   => "domain.com",
   );
```

METHODS

This section describes all the methods that can be used to retrieve the attribute values of an RRSIG Record object.

algorithm( )

Use this method to retrieve the value of the public key encryption algorithm used to create the RRSIG signature.

Parameter

none

Returns

The method returns the public key encryption algorithm. Returned values are one of the following:

RSAMD5
DSA
RSASHA1
NSEC3DSA
NSEC3RSASHA1
RSASHA256

RSASHA512
Example

```
#Get the public key encryption algorithm.
my $algorithm = $rrsig->algorithm();
```

cloud_info( )

Use this method to retrieve cloud API related information for the Infoblox::DNS::Record::RRSIG object.

Parameter

None

Returns

The method returns the attribute value.

Example

```
# Get cloud_info
my $cloud_info = $object->cloud_info();
```

creator( )

Use this method to retrieve the RRSIG record creator. This is a read-only attribute.

Parameter

None

Returns

The method returns the attribute value.

Example

```
# Get attribute value
my $value = $object->creator();
```

dns_name( )

Use this method to retrieve the FQDN, in punycode format, of the RRset covered by the RRSIG record. This is a read-only attribute.

Parameter

None

Returns

The method returns the attribute value.

Example

```
# Get attribute value
my $value = $rrsig->dns_name();
```

dns_signer_name( )

Use this method to retrieve the domain name, in punycode format, of the zone that contains the signed RRset. This is a read-only attribute.

Parameter

None

Returns

The method returns the attribute value.

Example

```
# Get attribute value
my $value = $bind_a->dns_signer_name();
```

expiration( )

Use this method to retrieve the expiration time of an RRSIG record.

Parameter

none

Returns

The method returns the RRSIG record expiration time. The returned value is in UNIX timestamp format; that is, it is measured in seconds since 00:00:00 January 1, 1970 UTC.

Example

```
#Get the RRSIG expiration time
my $expiration = $rrsig->expiration();
```

inception( )

Use this method to retrieve the RRSIG inception time. This is the time when the record was created.

Parameter

none

Returns

The method returns the RRSIG record inception time. The returned value is in UNIX timestamp format; that is, it is measured in seconds since 00:00:00 January 1, 1970 UTC.

Example

```
#Get the inception time of the RRSIG record
my $inception = $rrsig->inception();
```

key_tag( )

Use this method to retrieve the key tag of the public key that correspods to the RRSIG Record object.

Parameter

none

Returns

The method returns the key tag of the public key that corresponds to the private key used to encrypt RRSIG data. The key tag is a 16-bit integer key fingerprint that is used to distinguish one key from another when multiple keys are associated with the same zone.

Example

```
#Get the key tag of the public key
my $key_tag = $rrsig->key_tag();
```

labels( )

Use this method to retrieve the number of labels in the name of the RRset signed with the RRSIG object.

Parameter

none

Returns

The method returns the number of labels in the original RRSIG RR owner name. Usually this number is the same as the number of labels in the 'name' field.

Example

```
#Get the number of labels in the original RRSIG RR owner name.
my $labels = $rrsig->labels();
```

name( )

Use this method to retrieve the FQDN of the RRset covered by the RRSIG record.

The attribute value can be in unicode format.

Parameter

none

Returns

The method returns the name attribute of the RRSIG object. This is a name of the RRset covered by the RRSIG record.

Example

```
#Get the name attribute
my $fqdn = $rrsig->name();
```

original_ttl( )

Use this method to retrieve the TTL of the covered RRset as it appears in the authoritative zone of an RRSIG record object.

Parameter

none

Returns

The method returns the TTL of the covered RRset as it appears in the authoritative zone attribute value.

Example

```
#Get the TTL of the covered RRset
my $original_ttl = $rrsig->original_ttl();
```

signature( )

Use this method to retrieve the cryptographic signature that covers the RRSIG RDATA of an RRSIG Record object.

Parameter

none

Returns

The method returns the cryptographic signature that covers the RRSIG RDATA attribute value.

Example

```
#Get the signature that covers the RRSIG RDATA
my $signature = $rrsig->signature();
```

signer_name( )

Use this method to retrieve the domain name of the zone that contains the signed RRset.

The attribute value can be in unicode format.

Parameter

none

Returns

The method returns the domain name of the zone that contain the signed RRset in FQDN format.

Example

```
#Get the signer name
my $signer_name = $rrsig->signer_name();
```

ttl( )

Use this method to retrieve the Time to Live (TTL) value of an RRSIG Record object.

Parameter

none

Returns

The method returns the TTLattribute value. The returned parameter is a 32-bit integer (range from 0 to 4294967295) that represents the duration in seconds that the record is cached. Zero indicates that the record should not be cached.

Example

```
#Get the TTL of the RRSIG record
my $ttl = $rrsig->ttl();
```

type_covered( )

Use this method to retrieve the type of the RRset that is covered by this RRSIG record.

Parameter

none

Returns

The method returns the type of the RRset that is covered by this RRSIG record attribute value.

Example

```
#Get the type of the RRset that is covered by this RRSIG record
```

```
my $type_covered = $rrsig->type_covered();
```

view( )

Use this method to retrieve the DNS view object that contains the RRSIG Record object.

Parameter

none

Returns

The method returns the Infoblox::DNS::View object that contains the RRSIG record.

Example

```
#Get the DNS view
my $view = $rrsig->view();
```

zone( )

Use this method to retrieve the zone name of an RRSIG record.

Parameter

none

Returns

Returns the zone name of the zone that contains RRSIG record. The name is in FQDN format.

Example

```
# Get zone
my $zone = $rrsig->zone();
```

SAMPLE CODE

The following sample code demonstrates the session methods on an RRSIG record object.

```
#PROGRAM STARTS: Include all the modules that will be used
use strict;
use Infoblox;

#Create a session to the Infoblox appliance
       my $session = Infoblox::Session->new(
           master   => "192.168.1.2",
           username => "admin",
           password => "infoblox"
       );
       unless ($session) {
          die("Construct session failed: ",
              $session->status_code() . ":" . $session->status_detail());
       }
       print "Session created successfully\n";

       #Enable DNSSEC in the default view

       my $default_view=$session->get(
                                      object=> "Infoblox::DNS::View",
                                      name => "default"
                                     );
       unless($default_view) {
           die("Getting the default view failed:",
              Infoblox::status_code() . ":" . Infoblox::status_detail());
       }
       print "Got the default view successfully\n";

       $default_view->dnssec_enabled("true")
           or die("Changing the dnssec_enabled in the default view failed:",
                  Infoblox::status_code() . ":" . Infoblox::status_detail());
       $session->modify($default_view)
           or die("Changing the dnssec_enabled in the default view failed:",
                  Infoblox::status_code() . ":" . Infoblox::status_detail());
```

#Creating a signed zone and populationg it with an A record

```
       print "Creating Member primary server for the zone\n";
       my $primary=Infoblox::DNS::Member->new(
                                              ipv4addr => "192.168.1.2",
                                              name => "infoblox.localdomain"
                                             );
```

```
unless($primary) {
   die("Unable to create primary server object: ",
       Infoblox::status_code() . ":" . Infoblox::status_detail());
}

my $zone = Infoblox::DNS::Zone->new(
                                    name => "domain.com",
                                    primary => $primary
                                   );
unless ($zone) {
   die("Construct zone failed: ",
       Infoblox::status_code() . ":" . Infoblox::status_detail());
}
print "Zone object created successfully\n";

#Verify if the zone exists
my $object = $session->get(object => "Infoblox::DNS::Zone", name => "domain.com");
unless ($object) {
   print "Zone does not exist on server, safe to add the zone\n";
   $session->add($zone)
      or die("Add zone failed: ",
             $session->status_code() . ":" . $session->status_detail());
}
print "Zone added successfully\n";

#Adding A record to the zone to demonstrate the corresponding RRSIG record
my $a_record=Infoblox::DNS::Record::A->new(
                                           ipv4addr => "10.9.8.7",
                                           name => "recorda.domain.com"
                                          );
unless($a_record) {
   die("Creating A record failed : ",
      Infoblox::status_code() . ":" . Infoblox::status_detail());
}

$object = $session->get( object => "Infoblox::DNS::Record::A", name => "recorda.domain.com" );
unless ($object) {
   print "A record does not exist on the server, safe to add the A record\n";
   $session->add($a_record)
     or die("Adding A record failed: ",
            Infoblox::status_code() . ":" . Infoblox::status_detail());
}
print "A record added to the zone successfully\n";

#Retrieving zone back from the server in order to sign it
$zone = $session->get(object => "Infoblox::DNS::Zone", name => "domain.com");
unless($zone) {
   die("Retrieving zone back failed: ",
        Infoblox::status_code( ). ":". Infoblox::status_detail( ));
}
print "Zone retrieved for signing successfully.\n";

$zone->dnssec_ksk_algorithm("NSEC3RSASHA1") &&
$zone->dnssec_zsk_algorithm("NSEC3RSASHA1") &&
$zone->dnssec_ksk_size(640) &&
$zone->dnssec_zsk_size(640)
  or die("Changing the zone DNSSEC setting failed: ",
         Infoblox::status_code() . ":" . Infoblox::status_detail());
$session->modify($zone)
  or die("Modifying dnssec values in zone failed: ",
         Infoblox::status_code() . ":" . Infoblox::status_detail());
print "Zone modified successfully\n";

#Signing the zone
$zone->dnssec_signed("true")
   or die("Signing of the zone failed: ",
           Infoblox::status_code() . ":" . Infoblox::status_detail());
       print "Zone signed successfully\n";
```

#Getting the RRSIG record corresponding to A record

```
my $rrsig_record_a=$session->get(
                            object => "Infoblox::DNS::Record::RRSIG",
                            name   => "recorda.domain.com",
                            signer_name => "domain.com",
                            type_covered   => "A",
                            view   => "default"
                           );
     unless($rrsig_record_a) {
  die("Getting RRSIG corresponding to the A record failed: ",
      Infoblox::status_code() . ":" . Infoblox::status_detail());
      }
```

```
            print "Got RRSIG record successfully\n";
```

#Searching for RRSIG objects using regular expressions

```
      my @retrieved_objs=$session->search(
                                          object => "Infoblox::DNS::Record::RRSIG",
                                          name   => ".*com",
                                          type_covered => "A|MX",
                                          view   => "default"
                                         );
          unless(@retrieved_objs>0) {
        die("Searching for RRSIG objects failed: ",
            Infoblox::status_code() . ":" . Infoblox::status_detail());
          }
          print "Search for the RRSIG objects successful, ".scalar(@retrieved_objs)." objects found\n";

      #Searching for RRSIG objects created today using inception time interval search
      @retrieved_objs=$session->search(
                                          object => "Infoblox::DNS::Record::RRSIG",
                                          name   => ".*com",
                                          inception => ">=< " . (time() -86400) . "," . time(),
                                          view   => "default"
                                         );
          unless(@retrieved_objs>0) {
        die("Searching for RRSIG objects failed: ",
            Infoblox::status_code() . ":" . Infoblox::status_detail());
          }
          print "Search for the RRSIG objects successful, ".scalar(@retrieved_objs)." objects found\n";
```

#Removing the created zone and cleaning up the view

```
      $session->remove($zone)
        or die("Unable to remove the zone: ",
                Infoblox::status_code() . ":" . Infoblox::status_detail());
      print "Zone removed successfully\n";

      $default_view->dnssec_enabled("false")
      && $default_view->override_dnssec("false")
      && $session->modify($default_view)
       or die("Restoring dnssec_enabled value in the default view failed: ",
              Infoblox::status_code() . ":" . Infoblox::status_detail());

 ####PROGRAM ENDS####
```

AUTHOR

Infoblox Inc. http://www.infoblox.com/

SEE ALSO

Infoblox::Session, Infoblox::Session->get(), Infoblox::Session->search(), Infoblox::DNS::Record::DNSKEY, Infoblox::DNS::View, Infoblox::DNS::Zone

---

COPYRIGHT

Copyright (c) 2017 Infoblox Inc.

Infoblox::DNS::Record::DNSKEY - DNSKEY Record object.

- NAME
- DESCRIPTION
- CONSTRUCTOR
- SESSION METHODS
  - Infoblox::Session->get( )
  - Infoblox::Session->search( )
- METHODS
  - algorithm( )
  - creator( )
  - dns_name( )
  - flags( )
  - key_tag( )
  - name( )
  - public_key( )
  - ttl( )
  - view( )
  - zone( )
- SAMPLE CODE
- AUTHOR
- SEE ALSO
- COPYRIGHT

NAME

Infoblox::DNS::Record::DNSKEY - DNSKEY Record object.

DESCRIPTION

The DNSKEY resource record stores public keys for the DNSSEC authentication process. The DNSKEY records are generated automatically when the corresponding authoritative zone is signed. The the DNSKEY resource record object is read-only.

The DNSKEY resource record is defined in RFC 4034.

CONSTRUCTOR

The Infoblox::DNS::Record::DNSKEY object is a read-only object and does not require manual construction.

SESSION METHODS

This section describes all the methods in an Infoblox::Session that can be applied to a DNSKEY Record object.

Infoblox::Session->get( )

Use this method to retrieve the existing objects from an Infoblox appliance. See Infoblox::Session->get() for parameters and return values.

Key Reference

```
 Apply the following attributes to get the DNSKEY Record object(s):

     algorithm - Optional. The public key encryption algorithm. The allowed values are
'1','3','5','6','7','RSAMD5','RSASHA1','DSA','NSEC3RSASHA1','NSEC3DSA','RSASHA256','RSASHA512','NSEC3RSASHA256'
 and 'NSEC3RSASHA512'.
     flags     - Optional. Flags field to identify the type of the key.
     key_tag   - Optional. Key tag to identify the public key.
     name      - Optional. FQDN of the DNSKEY record.
     public_key- Optional. Public key in the base-64 encoding.
     ttl       - Optional. TTL of the DNSKEY record.
     view      - Optional. The DNS view in which the record is located. By default, all DNS views are
searched. But if you omit this attribute and specify a zone, the appliance searches the 'default' view only.
     zone      - Optional. A zone name in FQDN format.
```

Example

```
# get all the DNSKEY records for the zone 'domain.com' in the default view.
my @retrieved_objs = $session->get(
                     object => "Infoblox::DNS::Record::DNSKEY",
                     name   => "domain.com",
                     view   => "default" );

# get all the DNSKEY records in the "domain.com" zone of the default view
# with algorithm field having the value "RSASHA1" and "flags" having value 257
my @retrieved_objs = $session->get(
                     object    => "Infoblox::DNS::Record::DNSKEY",
                     name      => "domain.com",
                     algorithm => "RSASHA1",
                     flags     => 257,
                     view      => "default" );
```

Infoblox::Session->search( )

Use this method to retrieve the existing objects from an Infoblox appliance. See Infoblox::Session->search() for parameters and return values.

Key Reference

```
Apply the following attributes to search the DNSKEY Record object(s):

    algorithm - Optional. The public key encryption algorithm. The allowed values are
'1','3','5','6','7','RSAMD5','RSASHA1','DSA','NSEC3RSASHA1','NSEC3DSA','RSASHA256','RSASHA512','NSEC3RSASHA256'
 and 'NSEC3RSASHA512'.
    flags     - Optional. Flags field to identify the type of the key.
    key_tag   - Optional. Key tag to identify the public key.
    name      - Optional. FQDN of the DNSKEY record (regular expression).
    public_key- Optional. Public key in the base-64 encoding (regular expression).
    ttl       - Optional. TTL of the DNSKEY record.
    view      - Optional. The DNS view in which the record is located. By default, all DNS views are
searched. But if you omit this attribute and specify a zone, the appliance searches the 'default' view only.
    zone      - Optional. A zone name in FQDN format.
```

Example

```
# search the DNSKEY records in the default view by zone name.
my @retrieved_objs = $session->search(
                     object => "Infoblox::DNS::Record::DNSKEY",
                     name   => ".*[.]com",
                     view   => "default" );

# search for all DNSKEY records in the "domain.com" zone of the default view
# Note that the 'default' view is assumed implicitly here
my @retrieved_objs = $session->search(
    object => "Infoblox::DNS::Record::DNSKEY",
    zone   => "domain.com",
   );
```

METHODS

This section describes all the methods that can be used to retrieve the attribute values of a DNSKEY Record object.

algorithm( )

Use this method to retrieve the public key encryption algorithm of a DNSKEY Record object.

Parameter

none

Returns

The method returns the public key encryption algorithm attribute value. Returned values are one of the following:

RSAMD5
DSA
RSASHA1
NSEC3DSA
NSEC3RSASHA1
RSASHA256
RSASHA512

Example

```
#Get the public key encryption algorithm.
my $algorithm = $dnskey->algorithm();
```

creator( )

Use this method to retrieve the DNSKEY record creator. This is a read-only attribute.

Parameter

None

Returns

The method returns the attribute value.

Example

```
# Get attribute value
my $value = $dnskey->creator();
```

dns_name( )

Use this method to retrieve the FQDN, in punycode format, of the zone that owns the DNSKEY record. This is a read-only attribute.

Parameter

None

Returns

The method returns the attribute value.

Example

```
# Get attribute value
my $value = $dnskey->dns_name();
```

flags( )

Use this method to retrieve the flags of a DNSKEY Record object.

Parameter

none

Returns

The method returns the flags attribute value. The flags field is a 16-bit unsigned integer. Currently, only two bits of this value are used--the least significant bit and bit 7. The other bits are reserved for future use and must be zero. If bit 7 is set to 1, the key is a DNS zone key. Otherwise, the key is not a zone key and cannot be used to verify zone data. The least significant bit indicates "secure entry point property". If it is not zero, the key is a key signing key (KSK type). Otherwise, the key type is ZSK.

Example

```
#Getting flags
my $flags = $dnskey->flags();
```

key_tag( )

Use this method to retrieve the key tag identifying the public key of a DNSKEY Record object.

Parameter

none

Returns

The method returns the key tag of the public key value. A key tag is a 16-bit integer key fingerprint. It is used to distinguish one key from another when multiple keys are associated with the same zone.

Example

```
#Get the key tag of the public key
my $key_tag = $dnskey->key_tag();
```

name( )

Use this method to retrieve the FQDN of the zone that owns the DNSKEY record.

The attribute value can be in unicode format.

Parameter

none

Returns

The method returns the FQDN of the zone that contains the given DNSKEY record.

Example

```
#Get the FQDN of the DNSKEY record
my $name = $dnskey->name();
```

public_key( )

Use this method to retrieve the public key. The format of the returned value depend on the key algorithm.

Parameter

none

Returns

The method returns the public key. The exact format of the returned value depends on the algorithm of the key.

Example

```
#Get public key.
my $public_key = $dnskey->public_key();
```

ttl( )

Use this method to retrieve the Time to Live (TTL) value of a DNSKEY Record object.

Parameter

none

Returns

The method returns the TTL attribute value. The returned parameter is a 32-bit integer (range from 0 to 4294967295) that represents the duration, in seconds, that the record is cached. Zero indicates that the record should not be cached.

Example

```
#Get the TTL value
my $ttl = $dnskey->ttl();
```

view( )

Use this method to retrieve the DNS View object that contains the DNSKEY Record object.

Parameter

none

Returns

The method returns the Infoblox::DNS::View object that contains the DNSKEY record.

Example

```
#Get the view
my $view = $dnskey->view();
```

zone( )

Use this method to retrieve the zone name of a DNSKEY record.

Parameter

none

Returns

The method returns the name of the zone that contains the given DNSKEY record. The returned value is a string in FQDN format.

Example

```
# Get zone
my $zone = $dnskey->zone();
```

SAMPLE CODE

The following sample code demonstrates the session methods on a DNSKEY Record object.

```
#PROGRAM STARTS: Include all the modules that will be used
use strict;
use Infoblox;

#Create a session to the Infoblox device
       my $session = Infoblox::Session->new(
           master   => "192.168.1.2",
           username => "admin",
           password => "infoblox"
       );
       unless ($session) {
          die("Construct session failed: ",
              $session->status_code() . ":" . $session->status_detail());
       }
       print "Session created successfully\n";

       #Enable DNSSEC in the default view

       my $default_view=$session->get(
                                      object=> "Infoblox::DNS::View",
                                      name => "default"
                                     );
       unless($default_view) {
           die("Getting the default view failed:",
```

```
        Infoblox::status_code() . ":" . Infoblox::status_detail());
}
print "Got the default view successfully\n";

$default_view->dnssec_enabled("true")
    or die("Changing the dnssec_enabled in the default view failed:",
           Infoblox::status_code() . ":" . Infoblox::status_detail());
$session->modify($default_view)
    or die("Changing the dnssec_enabled in the default view failed:",
           Infoblox::status_code() . ":" . Infoblox::status_detail());

#Create the zone

print "Creating Member primary server for the zone\n";
my $primary=Infoblox::DNS::Member->new(
                                        ipv4addr => "192.168.1.2",
                                        name => "infoblox.localdomain"
                                      );
unless($primary) {
   die("Unable to create primary server object: ",
        Infoblox::status_code() . ":" . Infoblox::status_detail());
}

my $zone = Infoblox::DNS::Zone->new(
                                     name => "domain.com",
                                     primary => $primary
                                   );
unless ($zone) {
   die("Construct zone failed: ",
        Infoblox::status_code() . ":" . Infoblox::status_detail());
}
print "Zone object created successfully\n";

#Verify if the zone exists
my $object = $session->get(object => "Infoblox::DNS::Zone", name => "domain.com");
unless ($object) {
   print "Zone does not exist on server, safe to add the zone\n";
   $session->add($zone)
      or die("Add zone failed: ",
             $session->status_code() . ":" . $session->status_detail());
}
print "Zone added successfully\n";

#Retrieving zone back from the server in order to sign it
$zone = $session->get(object => "Infoblox::DNS::Zone", name => "domain.com");
unless($zone) {
   die("Retrieving zone back failed: ",
         Infoblox::status_code( ). ":". Infoblox::status_detail( ));
}
print "Zone retrieved for signing successfully.\n";

$zone->dnssec_ksk_algorithm("NSEC3RSASHA1") &&
$zone->dnssec_zsk_algorithm("NSEC3RSASHA1") &&
$zone->dnssec_ksk_size(640) &&
$zone->dnssec_zsk_size(640)
  or die("Changing the zone DNSSEC setting failed: ",
         Infoblox::status_code() . ":" . Infoblox::status_detail());
$session->modify($zone)
  or die("Modifying dnssec values in zone failed: ",
         Infoblox::status_code() . ":" . Infoblox::status_detail());
print "Zone modified successfully\n";

#Signing the zone
$zone->dnssec_signed("true")
   or die("Signing of the zone failed: ",
           Infoblox::status_code() . ":" . Infoblox::status_detail());
       print "Zone signed successfully\n";
```

#Getting the DNSKEY record

```
my $ksk_dnskey=$session->get(
                             object => "Infoblox::DNS::Record::DNSKEY",
                             flags  => 257,
                             name   => "domain.com",
                             view   => "default"
                            );
     unless($ksk_dnskey) {
 die("Getting ksk_dnskey failed: ",
     Infoblox::status_code() . ":" . Infoblox::status_detail());
     }
     print "Got Key-signing key DNSKEY record successfully\n";
     print "Key value: ".$ksk_dnskey->public_key()."\n";
```

```
my $zsk_dnskey=$session->get(
                                object => "Infoblox::DNS::Record::DNSKEY",
                                flags  => 256,
                                name   => "domain.com",
                                view   => "default"
                               );
unless($zsk_dnskey) {
   die("Getting zsk_dnskey failed: ",
       Infoblox::status_code() . ":" . Infoblox::status_detail());
}
print "Got Zone-signing key DNSKEY record successfully\n";
print "Key value: ".$zsk_dnskey->public_key()."\n";
```

#Searching for DNSKEY object

```
my @retrieved_objs=$session->search(
                                      object => "Infoblox::DNS::Record::DNSKEY",
                                      name   => ".*com",
                                      view   => "default"
                                    );
     unless(@retrieved_objs>0) {
   die("Searching for DNSKEY objects failed: ",
       Infoblox::status_code() . ":" . Infoblox::status_detail());
     }

#Removing the created zone and cleaning up the view
$session->remove($zone)
  or die("Unable to remove the zone: ",
         Infoblox::status_code() . ":" . Infoblox::status_detail());
print "Zone removed successfully\n";

$default_view->dnssec_enabled("false")
&& $default_view->override_dnssec("false")
&& $session->modify($default_view)
 or die("Restoring dnssec_enabled value in the default view failed: ",
        Infoblox::status_code() . ":" . Infoblox::status_detail());

####PROGRAM ENDS####
```

AUTHOR

Infoblox Inc. http://www.infoblox.com/

SEE ALSO

Infoblox::Session, Infoblox::Session->get(), Infoblox::Session->search(), Infoblox::DNS::Record::DS, Infoblox::DNS::Record::RRSIG, Infoblox::DNS::View, Infoblox::DNS::Zone

---

COPYRIGHT

Copyright (c) 2017 Infoblox Inc.

Infoblox::DTC::LBDN - A DTC Load Balanced Domain Name object.

- NAME
- DESCRIPTION
- CONSTRUCTOR
- SESSION METHODS
  - Infoblox::Session->add( )
  - Infoblox::Session->get( )
  - Infoblox::Session->modify( )
  - Infoblox::Session->remove( )
  - Infoblox::Session->search( )
- METHODS
  - auth_zones( )
  - comment( )
  - disable( )
  - extattrs( )
  - extensible_attributes( )
  - health( )
  - lb_method( )
  - name( )
  - override_ttl( )
  - patterns( )
  - persistence( )
  - pools( )
  - priority( )
  - topology( )
  - ttl( )
  - types( )
- AUTHOR
- SEE ALSO
- COPYRIGHT

NAME

Infoblox::DTC::LBDN - A DTC Load Balanced Domain Name object.

DESCRIPTION

A DTC Load Balanced Domain Name object.

CONSTRUCTOR

```
 my $lbdn = Infoblox::DTC::LBDN->new(
    name                 => $string,
#Required
    lb_method            => 'GLOBAL_AVAILABILITY' | 'RATIO' | 'ROUND_ROBIN' | 'TOPOLOGY',
#Required
    patterns             => [$pattern1, $pattern2, ...],
#Optional / Default is undefined
    disable              => 'true' | 'false',
#Optional / Default is 'false'
    comment              => $string,
#Optional / Default is undefined
    persistence          => $uint,
#Optional / Default is undefined
    topology             => $topology_obj,
#Optional / Default is undefined
    ttl                  => $uint,
#Optional / Default is undefined
    types                => ['A', 'AAAA', 'NAPTR', 'CNAME'],
#Optional / Default is ['A', 'AAAA']
    override_ttl         => 'true' | 'false',
#Optional / Default is 'false'
    pools                => [$pool_link_obj1, $pool_link_obj2, ...],
#Optional / Default is undefined
    priority             => $uint,
#Optional / Default is 1
    auth_zones           => [$auth_zone1, $auth_zone2, ...],
#Optional / Default is undefined
    extattrs             => { $string => $extattr, ... },
#Optional / Default is undefined
    extensible_attributes => { $string => $string | $num, $string => [ $string | $num, ... ], ... }
#Optional / Default is undefined
```

```
);

You cannot set both extattrs and extensible_attributes attributes at the same time.
```

SESSION METHODS

This section describes all the methods in a Session module that you can apply to a DTC LBDN object.

Infoblox::Session->add( )

Use this method to add an object to the Infoblox appliance. See Infoblox::Session->add() for parameters and return values.

Example

```
#construct an object
my $lbdn = Infoblox::DTC::LBDN->new(
   name      => 'load_bal',
   lb_method => 'GLOBAL_AVAILABILITY',
);

#submit for addition
my $response = $session->add($lbdn);
```

Infoblox::Session->get( )

Use this method to retrieve all the matching objects from the Infoblox appliance. See Infoblox::Session->get() for parameters and return values.

Key Reference

```
Apply the following attribute to get a specific object:

 name                 - Optional. A DTC LBDN name in a string format.
 comment              - Optional. A DTC LBDN comment.
 fqdn                 - Optional. A fully qualified domain name (FQDN).
 extattrs             - Optional. A hash reference containing extensible attributes.
 extensible_attributes - Optional. A hash reference that contains extensible attributes.
 status_member        - Optional. A member that provides the object health status. The default is
Grid.
```

Example

```
my @retrieved_objs = $session->get(
    object => 'Infoblox::DTC::LBDN',
    name   => 'load_bal',
    fqdn   => 'lb.com'
);

my @retrieved_objs = $session->get(
    object => 'Infoblox::DTC::LBDN',
    extensible_attributes => { 'Site' => 'Santa Clara' }
);
```

Infoblox::Session->modify( )

Use this method to modify an object in the Infoblox appliance. See Infoblox::Session->modify() for parameters and return values.

Example

```
#Use method to modify the comment.
$lbdn->comment('this is a modified comment');
#Submit modification
my $response = $session->modify($lbdn);
```

Infoblox::Session->remove( )

Use this method to remove an object from the Infoblox appliance. See Infoblox::Session->remove() for parameters and return values.

To remove a specific object, first use `get()` or `search()` to retrieve the specific object, and then submit this object for removal.

Example

```
#Get the objects with the same name
my @retrieved_objs = $session->get(
    object => 'Infoblox::DTC::LBDN',
    name   => 'load_bal',
);
```

```
#Find the desired object from the retrieved list.
my $desired_lbdn = $retrieved_objs[0];
#Submit for removal
my $response = $session->remove($desired_lbdn);
```

Infoblox::Session->search( )

Use this method to search for objects in the Infoblox appliance. See Infoblox::Session->search() for parameters and return values.

Key Reference

```
Apply the following attribute to get a specific object:

 name                  - Optional. A DTC LBDN name in a string format.
 comment               - Optional. A DTC LBDN comment.
 fqdn                  - Optional. A fully qualified domain name (FQDN).
 extattrs              - Optional. A hash reference containing extensible attributes.
 extensible_attributes - Optional. A hash reference that contains extensible attributes.
 status_member         - Optional. A member that provides the object health status. The default is
Grid.
```

Example

```
my @retrieved_objs = $session->search(
    object => 'Infoblox::DTC::LBDN',
    name   => 'load_b.*',
    fqdn   => 'lb.*'
);

my @retrieved_objs = $session->search(
    object => 'Infoblox::DTC::LBDN',
    extensible_attributes => { 'Site' => 'Santa Clara' }
);
```

METHODS

This section describes all the methods that you can use to set or retrieve the attribute values of the object.

auth_zones( )

Use this method to set or retrieve a list of authoritative zones the DTC LBDN is linked to.

Note that two different DTC LBDNs cannot be linked to the same zone if their patterns overlap.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

The valid value is an array containing Infoblox::DNS::Zone objects.

Returns

If you specified a parameter, the method returns 'true' when the modification succeeds, and returns 'false' when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#get auth_zones value
my @auth_zones = $lbdn->auth_zones();

#modify auth_zones value
$lbdn->auth_zones([$zone1, $zone2]);
```

comment( )

Use this method to set or retrieve a descriptive comment.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

Desired comment in string format (UTF-8) with a maximum of 256 bytes.

Returns

If you specified a parameter, the method returns 'true' when the modification succeeds, and returns 'false' when the operation

fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#get comment value
my $comment = $lbdn->comment();

#modify comment value
$lbdn->comment('desired comment');
```

disable( )

Use this method to enable or disable a DTC LBDN object.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

Specify 'true' to disable an DTC LBDN object or 'false' to enable it.

Returns

If you specified a parameter, the method returns 'true' when the modification succeeds, and returns 'false' when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#get disable value
my $disable = $lbdn->disable();

#modify disable value
$lbdn->disable('true');
```

extattrs( )

Use this method to set or retrieve the extensible attributes associated with a DTC LBDN object.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

Valid value is a hash reference containing the names of extensible attributes and their associated values (Infoblox::Grid::Extattr objects).

Returns

If you specified a parameter, the method returns 'true' when the modification succeeds, and returns 'false' when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#get extattrs value
my $ref_extattrs = $lbdn->extattrs();

#Modify extattrs
$lbdn->extattrs({ 'Site' => $extattr1, 'Administrator' => $extattr2 });
```

extensible_attributes( )

Use this method to set or retrieve the extensible attributes associated with a DTC LBDN object.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

For valid values for extensible attributes, see Infoblox::Grid::ExtensibleAttributeDef/Extensible Attribute Values.

Returns

If you specified a parameter, the method returns 'true' when the modification succeeds, and returns 'false' when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get extensible attributes
my $ref_extensible_attributes = $lbdn->extensible_attributes();

#Modify extensible attributes
$lbdn->extensible_attributes({'Site' => 'Santa Clara', 'Administrator' => ['Peter', 'Tom']});
```

health( )

Use this method to retrieve DTC LBDN health information. This is a read-only attribute.

Parameter

None

Returns

The valid return value is an Infoblox::DTC::Health object.

Example

```
#get health value
my $health = $lbdn->health();
```

lb_method( )

Use this method to set or retrieve a DTC LBDN load balancing method.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

The valid values are 'GLOBAL_AVAILABILITY', 'RATIO', 'ROUND_ROBIN' and 'TOPOLOGY'.

Returns

If you specified a parameter, the method returns 'true' when the modification succeeds, and returns 'false' when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#get lb_method value
my $lb_method = $lbdn->lb_method();

#modify lb_method value
$lbdn->lb_method('RATIO');
```

name( )

Use this method to set or retrieve the DTC LBDN name.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

The valid value is a desired name in a string format.

Returns

If you specified a parameter, the method returns 'true' when the modification succeeds, and returns 'false' when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#get name value
my $name = $lbdn->name();

#modify name value
$lbdn->name('load_bal1');
```

override_ttl( )

The override_ttl attribute controls whether the TTL value in the object is used, instead of the Grid default.

The override_ttl attribute can be specified explicitly. It is also set implicitly when ttl is set to a defined value.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

Specify 'true' to set override_ttl flag or 'false' to disable it. Default value is 'false'.

Returns

If you specified a parameter, the method returns 'true' when the modification succeeds, and returns 'false' when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#get override_ttl value
my $override_ttl = $lbdn->override_ttl();

#modify override_ttl value
$lbdn->override_ttl('true');
```

patterns( )

Use this method to set or retrieve an array of DNS FQDN patterns. The patterns are used to match DNS query names processed by the DTC LBDN.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

The valid value is an array of FQDN patterns in string format.

Returns

If you specified a parameter, the method returns 'true' when the modification succeeds, and returns 'false' when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#get patterns value
my @patterns = $lbdn->patterns();

#modify patterns value
$lbdn->patterns(['???'.domain.com', '*.test.com', 'test.domain.com']);
```

persistence( )

Use this method to set or retrieve the maximum time (in seconds) for which client specific DTC LBDN responses are cached.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

The valid value is an unsigned integer. Zero specifies no caching.

Returns

If you specified a parameter, the method returns 'true' when the modification succeeds, and returns 'false' when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#get persistence value
my $persistence = $lbdn->persistence();

#modify persistence value
$lbdn->persistence(5);
```

pools( )

Use this method to set or retrieve the list of pools the DTC LBDN is linked to.

Note that a DTC LBDN linked to a zone must contain at least one enabled pool if lb_method is set to anything other than 'TOPOLOGY'.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

The valid value is an array of Infoblox::DTC::Pool::Link objects.

Returns

If you specified a parameter, the method returns 'true' when the modification succeeds, and returns 'false' when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#get pools value
my @pools = $lbdn->pools();

#modify pools value
$lbdn->pools([$pool_link1, $pool_link2]);
```

priority( )

Use this method to set or retrieve the LBDN pattern match priority for "overlapping" DTC LBDN objects.

LBDNs are "overlapping" if they are simultaneously assigned to a zone and have patterns that can match the same FQDN. The matching LBDN with highest priority (lowest ordinal) will be used.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

The valid value is an unsigned integer between 1 and 3. The default is 3.

Returns

If you specified a parameter, the method returns 'true' when the modification succeeds, and returns 'false' when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get priority value
my $priority = $object->priority();

#Modify priority value
$object->priority(3);
```

topology( )

Use this method to set or retrieve topology rules for the 'TOPOLOGY' load balancing method.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

The valid value is an Infoblox::DTC::Topology object.

Returns

If you specified a parameter, the method returns 'true' when the modification succeeds, and returns 'false' when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#get topology value
my $topology = $lbdn->topology();

#modify topology value
$lbdn->topology($topology);
```

ttl( )

Use this method to set a TTL value.

Setting this method to a defined value implicitly sets the override_ttl method to 'true'. Setting the parameter to undefined causes the appliance to use the Grid default and automatically resets the override_ttl attribute to 'false'.

Note that when ttl is set to a defined value and override_ttl is set to 'false', the last operation takes precedence. Thus the sequence $object->ttl(10); $object->override_ttl('false'); will set override_ttl to 'false', and the sequence $object->override_ttl('false'); $object->ttl(10); will result in override_ttl='true'.

Parameter

The valid value is an unsigned integer.

Returns

If you specified a parameter, the method returns 'true' when the modification succeeds, and returns 'false' when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#get ttl value
my $ttl = $lbdn->ttl();

#modify ttl value
$lbdn->ttl(10);
```

types( )

Use this method to set or retrieve the list of record types supported by LBDN.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

The valid valid is an array reference that contains any of the following values: 'A', 'AAA', 'NAPTR', 'CNAME'.

Returns

If you specified a parameter, the method returns 'true' when the modification succeeds, and returns 'false' when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#get types value
my $types = $lbdn->types();

#modify types value
$lbdn->types(['NAPTR']);
```

AUTHOR

Infoblox Inc. http://www.infoblox.com/

SEE ALSO

Infoblox::Session, Infoblox::Session->add(), Infoblox::Session->get(), Infoblox::Session->modify(), Infoblox::Session->remove(), Infoblox::Session->search(), Infoblox::Grid::Extattr, Infoblox::Grid::ExtensibleAttributeDef/Extensible Attribute Values, Infoblox::DTC::Health, Infoblox::DTC::Pool::Link, Infoblox::DTC::Topology.

COPYRIGHT

Copyright (c) 2017 Infoblox Inc.

Infoblox::DNS::Nameserver::Address - NS Name Server object.

- NAME
- DESCRIPTION
- CONSTRUCTOR
- MODULE METHODS
  - Infoblox::DNS::Record::NS->nameservers( )
- METHODS
  - address( )
  - auto_create_ptr( )
- AUTHOR
- SEE ALSO
- COPYRIGHT

NAME

Infoblox::DNS::Nameserver::Address - NS Name Server object.

DESCRIPTION

The NS Name Server object provides IP address information for the name server associated with an NS record.

CONSTRUCTOR

```
my $rootns = Infoblox::DNS::Nameserver::Address->new(
             address => $address,                  #Required
             auto_create_ptr => "true" | "false"   #Optional / Default "true"
);
```

MODULE METHODS

You can apply the following functions to an NS Name Server object:

Infoblox::DNS::Record::NS->nameservers( )

Use this function to specify the IP addresses associated with the NS record. See Infoblox::DNS::Record::NS->nameservers() for parameters and return values.

Example

```
#Construct an NS Name Server object
my $ns1 = Infoblox::DNS::Nameserver::Address->new(
              address  => "10.0.0.1"
);

# Configure a name server address on the NS record object
$ns_record->nameservers([$ns1]);
```

METHODS

This section describes all the methods you can use to configure and retrieve the attribute values of an NS Name Server object.

address( )

Use this method to set or retrieve the IPv4 address of the name server.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

The IPv4 address (32 bits) of the name server.

Returns

If you specify a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you do not specify a parameter, the method returns the attribute value.

Example

```
#Get the address
my $addr = $ns->address();
#Modify the address
$ns->address("10.0.0.1");
```

auto_create_ptr( )

Use this method to set or retrieve the auto_create_ptr flag of the name server. When this flag is set to true, PTR records are automatically created for NS records created in a reverse zone.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

Valid values are "true" or "false".

Returns

If you specify a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you do not specify a parameter, the method returns the attribute value.

Example

```
#Get auto_create_ptr
my $aptr = $ns->auto_create_ptr();
#Modify auto_create_ptr
$rootns->auto_create_ptr("false");
```

AUTHOR

Infoblox Inc. http://www.infoblox.com/

SEE ALSO

Infoblox::Session, Infoblox::DNS::Record::NS

COPYRIGHT

Copyright (c) 2017 Infoblox Inc.

Infoblox::DNS::Record::NSEC3 - NSEC3 Record object.

- NAME
- DESCRIPTION
- CONSTRUCTOR
- SESSION METHODS
  - Infoblox::Session->get( )
  - Infoblox::Session->search( )
- METHODS
  - cloud_info( )
  - creator( )
  - flags( )
  - hash_algorithm( )
  - iterations( )
  - name( )
  - next_hash_owner( )
  - salt( )
  - ttl( )
  - types( )
  - view( )
  - zone( )
- SAMPLE CODE
- AUTHOR
- SEE ALSO
- COPYRIGHT

NAME

Infoblox::DNS::Record::NSEC3 - NSEC3 Record object.

DESCRIPTION

The NSEC3 resource record is a part of DNS security extension (DNSSEC) authentication mechanism. It is used for authenticated denial of existence of DNS Resource record sets in the DNSSEC signed authoritative zones. The functionality that NSEC3 records provide is similar to the functionality of NSEC records, but NSEC3 includes cryptographic protection against zone enumeration.

The NSEC3 resource record is desribed in RFC 5155.

All the NSEC3 records are automatically generated during signing of the corresponding zone.

CONSTRUCTOR

The Infoblox::DNS::Record::NSEC3 object is a read-only object and does not require manual construction.

SESSION METHODS

This section describes all the methods in an Infoblox::Session that can be applied to an NSEC3 Record object.

Infoblox::Session->get( )

Use this method to retrieve the existing objects from an Infoblox appliance. See Infoblox::Session->get() for parameters and return values.

Key Reference

```
 Apply the following attributes to get the NSEC3 Record object(s):

     flags           - Optional. Integer containing 8 one-bit flags that can be used to indicate
different processing.
     hash_algorithm  - Optional. Integer representing the cryptographic hash algorithm used to
construct the hash-value.
     iterations      - Optional. Number of additional times the hash function has been performed.
     name            - Optional. FQDN of the NSEC3 record (contains zone name and the hashed record
name).
     next_hash_owner - Optional. Next hashed owner name in hash order.
     salt            - Optional. Salt value.
     ttl             - Optional. TTL value.
     types           - Optional. Identifies the RRSet types that exist at the original owner name of
the NSEC3 RR.
     view            - Optional. The DNS view in which the record is located. By default, all DNS
views are searched. But if you omit this attribute and specify a zone, the appliance searches the
'default' view only.
     zone            - Optional. A zone name in FQDN format.
```

Example

```
# get all the NSEC3 records corresponding to 'A' records in the default DNS view.
my @retrieved_objs = $session->get(
                     object => "Infoblox::DNS::Record::NSEC3",
                     types  => "A",
                     view   => "default" );
```

Infoblox::Session->search( )

Use this method to retrieve the existing objects from an Infoblox appliance. See Infoblox::Session->search() for parameters and return values.

Key Reference

```
Apply the following attributes to search the NSEC3 Record object(s):

     flags           - Optional. Integer containing 8 one-bit flags that can be used to indicate
different processing.
     hash_algorithm  - Optional. Integer representing the cryptographic hash algorithm used to
construct the hash-value.
     iterations      - Optional. Number of additional times the hash function has been performed.
     name            - Optional. FQDN of the NSEC3 record (contains zone name and the hashed record
name). Regular expression
     next_hash_owner - Optional. Next hashed owner name in hash order (regular expression).
     salt            - Optional. Salt value (regular expression).
     ttl             - Optional. TTL value.
     types           - Optional. Identifies the RRSet types that exist at the original owner name of
the NSEC3 RR (regular expression).
     view            - Optional. The DNS view in which the record is located. By default, all DNS
views are searched. But if you omit this attribute and specify a zone, the appliance searches the
'default' view only.
     zone            - Optional. A zone name in FQDN format.
```

Example

```
# get all the NSEC3 records in the zone test.com.
my @retrieved_objs = $session->get(
                     object => "Infoblox::DNS::Record::NSEC3",
                     name   => ".*test[.]com",
                     );

# search for all NSEC3 records in the "domain.com" zone of the default DNS view
# Note that the 'default' view is assumed implicitly here
my @retrieved_objs = $session->search(
    object => "Infoblox::DNS::Record::NSEC3",
    zone   => "domain.com",
   );
```

METHODS

This section describes all the methods that can be used to retrieve the attribute values of an NSEC3 Record object.

cloud_info( )

Use this method to retrieve cloud API related information for the Infoblox::DNS::Record::NSEC3 object.

Parameter

None

Returns

The method returns the attribute value.

Example

```
# Get cloud_info
my $cloud_info = $object->cloud_info();
```

creator( )

Use this method to retrieve the NSEC3 record creator. This is a read-only attribute.

Parameter

None

Returns

The method returns the attribute value.

Example

```
# Get attribute value
my $value = $object->creator();
```

flags( )

Use this method to retrieve the flags field of an NSEC3 Record object.

Parameter

none

Returns

The method returns the integer that contains 8 one-bit flags that can be used to indicate different processing. Currently only the least significant bit of the flags field is meaningful. It represents the opt-out flag, which is used to determine the processing of non-secure delegations.

Example

```
#Get the flags field.
my $flags = $nsec3->flags();
```

hash_algorithm( )

Use this method to retrieve the cryptographic hash algorithm used to construct the hash value of an NSEC3 Record object.

Parameter

none

Returns

The method returns an integer that corresponds to the cryptographic hash algorithm used to construct the hash value.

Example

```
#Get hash algorithm
my $hash_algorithm = $nsec3->hash_algorithm();
```

iterations( )

Use this method to retrieve the number of iterations of the hash function performed to obtain the hashed name.

Parameter

none

Returns

The method returns the number of additional times the hash function has been performed to compute the hash value. This is an integer in the range 0 through 65535, inclusive.

Example

```
#Get the iterations parameter
my $iterations = $nsec3->iterations();
```

name( )

Use this method to retrieve the name of NSEC3 record in FQDN format. The name of NSEC3 record consists of the hashed owner name prepended to the name of the zone, containing NSEC3 record.

Parameter

none

Returns

The method returns the name of NSEC3 record.

Example

```
#Get the NSEC3 record name
my $name = $nsec3->name();
```

next_hash_owner( )

Use this method to retrieve the next hashed owner name. This is a hash of an owner name that immediately follows the owner name of the given NSEC3 record in the hash order.

Parameter

none

Returns

The method returns the next hashed owner name. It is a string in base-32 encoding.

Example

```
#Get the next hashed owner name
my $next_hash_owner = $nsec3->next_hash_owner();
```

salt( )

Use this method to retrieve the salt value of a NSEC3 record. The salt is prepended to the original owner name before hashing and on each of the hash iterations in order to defend against pre-calculated dictionary attacks.

Parameter

none

Returns

The method returns a salt value. The returned value is a string containing a sequence of hexadecimal digits representing the salt or '-' if the salt is not used.

Example

```
#Get the salt
my $salt = $nsec3->salt();
```

ttl( )

Use this method to retrieve the Time to Live (TTL) value of a NSEC3 Record object.

Parameter

none

Returns

The method returns the TTLattribute value. The returned parameter is a 32-bit integer (range from 0 to 4294967295) that represents the duration in seconds that the record is cached. Zero indicates that the record should not be cached.

Example

```
#Get TTL
my $ttl = $nsec3->ttl();
```

types( )

Use this method to retrieve the types field of an NSEC3 record. Types field identifies the RRSet types that exist at the original owner name of the NSEC3 RR.

Parameter

none

Returns

The method returns the RRSet types that exist at the original owner name of the NSEC3 RR attribute value. The returned value is a string containing space separated RR types mnemonics, e.g. "NS SOA RRSIG DNSKEY NSEC3PARAM".

Example

```
#Get the types field value
my $types = $nsec3->types();
```

view( )

Use this method to retrieve the DNS view object that contains the NSEC3 Record object.

Parameter

none

Returns

The method returns the Infoblox::DNS::View object that contains the NSEC3 record.

Example

```
#Get the DNS view
my $view = $nsec3->view();
```

zone( )

Use this method to retrieve the zone name of a NSEC3 record.

Parameter

none

Returns

Returns the zone name of the zone that contains the NSEC3 record. The zone name is in FQDN format.

Example

```
# Get zone
my $zone = $nsec3->zone();
```

SAMPLE CODE

The following sample code demonstrates the session methods on a NSEC3 Record object.

```
#PROGRAM STARTS: Include all the modules that will be used
use strict;
use Infoblox;

#Create a session to the Infoblox device
       my $session = Infoblox::Session->new(
           master   => "192.168.1.2",
           username => "admin",
           password => "infoblox"
       );
       unless ($session) {
          die("Construct session failed: ",
              $session->status_code() . ":" . $session->status_detail());
       }
       print "Session created successfully\n";

       #Enable DNSSEC in the default view

       my $default_view=$session->get(
                                      object=> "Infoblox::DNS::View",
                                      name => "default"
                                     );
       unless($default_view) {
           die("Getting the default view failed:",
              Infoblox::status_code() . ":" . Infoblox::status_detail());
       }
       print "Got the default view successfully\n";

       $default_view->dnssec_enabled("true")
           or die("Changing the dnssec_enabled in the default view failed:",
                    Infoblox::status_code() . ":" . Infoblox::status_detail());
       $session->modify($default_view)
           or die("Changing the dnssec_enabled in the default view failed:",
                    Infoblox::status_code() . ":" . Infoblox::status_detail());
```

#Create a signed zone and populate it with an A record

```
       print "Creating Member primary server for the zone\n";
       my $primary=Infoblox::DNS::Member->new(
                                              ipv4addr => "192.168.1.2",
                                              name => "infoblox.localdomain",
```

```
                                        );
    unless($primary) {
       die("Unable to create primary server object: ",
           Infoblox::status_code() . ":" . Infoblox::status_detail());
    }

    my $zone = Infoblox::DNS::Zone->new(
                                        name => "domain.com",
                                        primary => $primary
                                      );
    unless ($zone) {
       die("Construct zone failed: ",
           Infoblox::status_code() . ":" . Infoblox::status_detail());
    }
    print "Zone object created successfully\n";

    #Verify if the zone exists
    my $object = $session->get(object => "Infoblox::DNS::Zone", name => "domain.com");
    unless ($object) {
       print "Zone does not exist on server, safe to add the zone\n";
       $session->add($zone)
          or die("Add zone failed: ",
                 $session->status_code() . ":" . $session->status_detail());
    }
    print "Zone added successfully\n";

    #Adding A record to the zone to demonstrate the corresponding NSEC3 record
    my $a_record=Infoblox::DNS::Record::A->new(
                                               ipv4addr => "10.9.8.7",
                                               name => "recorda.domain.com"
                                              );
    unless($a_record) {
       die("Creating A record failed : ",
          Infoblox::status_code() . ":" . Infoblox::status_detail());
    }

    $object = $session->get( object => "Infoblox::DNS::Record::A", name => "recorda.domain.com" );
    unless ($object) {
       print "A record does not exist on the server, safe to add the A record\n";
       $session->add($a_record)
         or die("Adding A record failed: ",
                Infoblox::status_code() . ":" . Infoblox::status_detail());
    }
    print "A record added to the zone successfully\n";

    #Retrieving zone back from the server in order to sign it
    $zone = $session->get(object => "Infoblox::DNS::Zone", name => "domain.com");
    unless($zone) {
       die("Retrieving zone back failed: ",
            Infoblox::status_code( ). ":". Infoblox::status_detail( ));
    }
    print "Zone retrieved for signing successfully.\n";

    $zone->dnssec_ksk_algorithm("NSEC3RSASHA1") &&
    $zone->dnssec_zsk_algorithm("NSEC3RSASHA1") &&
    $zone->dnssec_ksk_size(640) &&
    $zone->dnssec_zsk_size(640)
      or die("Changing the zone DNSSEC setting failed: ",
             Infoblox::status_code() . ":" . Infoblox::status_detail());
    $session->modify($zone)
      or die("Modifying dnssec values in zone failed: ",
             Infoblox::status_code() . ":" . Infoblox::status_detail());
    print "Zone modified successfully\n";

    #Signing the zone
    $zone->dnssec_signed("true")
       or die("Signing of the zone failed: ",
              Infoblox::status_code() . ":" . Infoblox::status_detail());
           print "Zone signed successfully\n";
```

#Getting the NSEC3 record corresponding to the A record

```
    my $nsec3_record_a=$session->get(
                                object => "Infoblox::DNS::Record::NSEC3",
                                types     => "A RRSIG",
                                view   => "default"
```

```
                                    );
        unless($nsec3_record_a) {
    die("Getting NSEC3 corresponding to the A record failed: ",
        Infoblox::status_code() . ":" . Infoblox::status_detail());
        }
        print "Got NSEC3 record successfully, name value: ".$nsec3_record_a->name()."\n";
```

#Searching for NSEC3 objects using regular expressions

```
    my @retrieved_objs=$session->search(
                                        object => "Infoblox::DNS::Record::NSEC3",
                                        name   => ".*domain[.]com",
                                        types  => "A|MX|SOA",
                                        view   => "default"
                                      );
      unless(@retrieved_objs>0) {
    die("Searching for NSEC3 objects failed: ",
        Infoblox::status_code() . ":" . Infoblox::status_detail());
      }
      print "Search for the NSEC3 objects successful, ".scalar(@retrieved_objs)." objects found\n";
```

#Removing the created zone and cleaning up the view

```
    $session->remove($zone)
      or die("Unable to remove the zone: ",
             Infoblox::status_code() . ":" . Infoblox::status_detail());
    print "Zone removed successfully\n";

    $default_view->dnssec_enabled("false")
    && $default_view->override_dnssec("false")
    && $session->modify($default_view)
     or die("Restoring dnssec_enabled value in the default view failed: ",
            Infoblox::status_code() . ":" . Infoblox::status_detail());
```

####PROGRAM ENDS####

AUTHOR

Infoblox Inc. http://www.infoblox.com/

SEE ALSO

Infoblox::Session, Infoblox::Session->get(), Infoblox::Session->search(), Infoblox::DNS::Record::NSEC, Infoblox::DNS::Record::NSEC3PARAM, Infoblox::DNS::View, Infoblox::DNS::Zone

---

COPYRIGHT

Copyright (c) 2017 Infoblox Inc.

Infoblox::DNS::NSEC3PARAM - NSEC3PARAM Record object

- NAME
- DESCRIPTION
- CONSTRUCTOR
- SESSION METHODS
  - Infoblox::Session->get( )
  - Infoblox::Session->search( )
- METHODS
  - cloud_info( )
  - creator( )
  - dns_name( )
  - flags( )
  - hash_algorithm( )
  - iterations( )
  - name( )
  - salt( )
  - ttl( )
  - view( )
  - zone( )
- SAMPLE CODE
- AUTHOR
- SEE ALSO
- COPYRIGHT

NAME

Infoblox::DNS::NSEC3PARAM - NSEC3PARAM Record object

DESCRIPTION

The NSEC3PARAM resource record is one of the resource records of the DNS security extension (DNSSEC) authentication mechanism. The NSEC3PARAM RR contains the NSEC3 parameters (hash algorithm, flags, iterations, and salt) needed by authoritative servers to calculate hashed owner names for NSEC3 records. The NSEC3PARAM record is present at the apex of the DNSSEC-signed authoritative zone.

The NSEC3PARAM resource record is desribed in RFC 5155.

The NSEC3PARAM record is generated automatically upon the signing of the corresponding zone. It is a read-only object.

CONSTRUCTOR

The Infoblox::DNS::NSEC3PARAM object is a read-only object and does not require manual construction.

SESSION METHODS

This section describes all the methods in an Infoblox::Session that can be applied to an NSEC3PARAM Record object.

Infoblox::Session->get( )

Use this method to retrieve the existing objects from an Infoblox appliance. See Infoblox::Session->get() for parameters and return values.

Key Reference

```
 Apply the following attributes to get the NSEC3PARAM Record object(s):

     flags           - Optional. Integer containing 8 one-bit flags that can be used to indicate
different processing.
     hash_algorithm  - Optional. Integer representing the cryptographic hash algorithm.
     iterations      - Optional. Number of additional times the hash function has been performed.
     name            - Optional. FQDN of the NSEC3PARAM record owner (zone name).
     salt            - Optional. Salt value.
     ttl             - Optional. TTL of the record.
     view            - Optional. The DNS view in which the record is located. By default, all DNS
views are searched. But if you omit this attribute and specify a zone, the appliance searches the
'default' view only.
     zone            - Optional. A zone name in FQDN format.
```

Example

```
#get the NSEC3PARAM record for the zone 'test.com'
my $nsec3param=$session->get(
                          object => "Infoblox::DNS::Record::NSEC3PARAM",
```

```
                                name   => "test.com"
                              );

#get all the NSEC3PARAM records for the default DNS view
my @retrieved_objs=$session->get(
                                    object => "Infoblox::DNS::Record::NSEC3PARAM",
                                    view   => "default"
                                  );
```

Infoblox::Session->search( )

Use this method to retrieve the existing objects from an Infoblox appliance. See Infoblox::Session->search() for parameters and return values.

Key Reference

```
Apply the following attributes to search the NSEC3PARAM Record object(s):

     flags           - Optional. Integer containing 8 one-bit flags that can be used to indicate
different processing.
     hash_algorithm  - Optional. Integer representing the cryptographic hash algorithm.
     iterations      - Optional. Number of additional times the hash function has been performed.
     name            - Optional. FQDN of the NSEC3PARAM record owner (zone name). Regular expression.
     salt            - Optional. Salt value (regular expression).
     ttl             - Optional. TTL of the record.
     view            - Optional. The DNS view in which the record is located. By default, all DNS
views are searched. But if you omit this attribute and specify a zone, the appliance searches the
'default' view only.
     zone            - Optional. A zone name in FQDN format.
```

Example

```
# search for all NSEC3PARAM records in the "domain.com" zone of the default DNS view
# Note that the 'default' DNS view is assumed implicitly here
my @retrieved_objs = $session->search(
    object => "Infoblox::DNS::Record::NSEC3PARAM",
    zone   => "domain.com",
   );
```

METHODS

This section describes all the methods that can be used to retrieve the attribute values of an NSEC3PARAM Record object.

cloud_info( )

Use this method to retrieve cloud API related information for the Infoblox::DNS::NSEC3PARAM object.

Parameter

None

Returns

The method returns the attribute value.

Example

```
# Get cloud_info
my $cloud_info = $object->cloud_info();
```

creator( )

Use this method to retrieve the NSEC3PARAM record creator. This is a read-only attribute.

Parameter

None

Returns

The method returns the attribute value.

Example

```
# Get attribute value
my $value = $object->creator();
```

dns_name( )

Use this method to retrieve the owner name, in punycode format, of the NSEC3PARAM record. This is a read-only attribute.

Parameter

None

Returns

The method returns the attribute value.

Example

```
# Get attribute value
my $value = $nsec3param->dns_name();
```

flags( )

Use this method to retrieve the flags field of an NSEC3PARAM Record object.

Parameter

none

Returns

The method returns the integer that contains 8 one-bit flags that can be used to indicate different processing. Currently only the least significant bit of the flags field is meaningful. It represents the opt-out flag, which is used to determine the processing of non-secure delegations.

Example

```
#Get the flags field.
my $flags = $nsec3param->flags();
```

hash_algorithm( )

Use this method to retrieve the cryptographic hash algorithm used to construct the hash value of the corresponding NSEC3 Record objects.

Parameter

none

Returns

The method returns an integer that corresponds to the cryptographic hash algorithm that is used to construct the hash value.

Example

```
#Get hash algorithm
my $hash_algorithm = $nsec3param->hash_algorithm();
```

iterations( )

Use this method to retrieve the number of iterations of the hash function performed to obtain the hashed name.

Parameter

none

Returns

The method returns the number of additional times the hash function has been performed to compute the hash value. This is an integer in the range 0 through 65535, inclusive.

Example

```
#Get the iterations parameter
my $iterations = $nsec3param->iterations();
```

name( )

Use this method to retrieve the owner name of the NSEC3PARAM record in FQDN format. The name value of the NSEC3PARAM record is the same as the name of the zone that contains this record.

The attribute value can be in unicode format.

Parameter

none

Returns

The method returns the name of the NSEC3PARAM record. This name is a zone name in FQDN format.

Example

```
#Get the NSEC3PARAM name field
my $name = $nsec3param->name();
```

salt( )

Use this method to retrieve the salt value of an NSEC3PARAM record. The salt is used to calculate the hashed name of NSEC3 resource records to defend against pre-calculated dictionary attacks.

Parameter

none

Returns

The method returns a salt value. The returned value is a string containing a sequence of hexadecimal digits representing the salt or '-' if the salt is not used.

Example

```
#Get the salt
my $salt = $nsec3param->salt();
```

ttl( )

Use this method to retrieve the Time to Live (TTL) value of an NSEC3PARAM Record object.

Parameter

none

Returns

The method returns the TTL attribute value. The returned parameter is a 32-bit integer (range from 0 to 4294967295) that represents the duration, in seconds, that the record is cached. Zero indicates that the record should not be cached.

Example

```
#Get TTL
my $ttl = $nsec3param->ttl();
```

view( )

Use this method to retrieve the DNS view object that contains the NSEC3PARAM Record object.

Parameter

none

Returns

The method returns the Infoblox::DNS::View object that contains the NSEC3PARAM record.

Example

```
#Get the view
my $view = $nsec3param->view();
```

zone( )

Use this method to retrieve the zone name of an NSEC3PARAM record.

Parameter

none

Returns

Returns the name of the zone that contains the given NSEC3PARAM record.

Example

```
# Get zone
my $zone = $nsec3param->zone();
```

SAMPLE CODE

The following sample code demonstrates the session methods on an NSEC3PARAM Record object.

```
#PROGRAM STARTS: Include all the modules that will be used
use strict;
use Infoblox;

#Create a session to the Infoblox appliance
       my $session = Infoblox::Session->new(
           master   => $host_ip,
           username => "admin",
           password => "infoblox"
       );
       unless ($session) {
          die("Construct session failed: ",
              $session->status_code() . ":" . $session->status_detail());
       }
       print "Session created successfully\n";

       #Enable DNSSEC in the default view

       my $default_view=$session->get(
                                       object=> "Infoblox::DNS::View",
                                       name => "default"
                                      );
       unless($default_view) {
           die("Getting the default view failed:",
              Infoblox::status_code() . ":" . Infoblox::status_detail());
       }
       print "Got the default view successfully\n";

       $default_view->dnssec_enabled("true")
           or die("Changing the dnssec_enabled in the default view failed:",
                    Infoblox::status_code() . ":" . Infoblox::status_detail());
       $session->modify($default_view)
           or die("Changing the dnssec_enabled in the default view failed:",
                    Infoblox::status_code() . ":" . Infoblox::status_detail());
```

#Create a DNSSEC signed zone

```
       print "Creating Member primary server for the zone\n";
       my $primary=Infoblox::DNS::Member->new(
                                           ipv4addr => $node_ip,
                                           name => $host_name
                                         );
       unless($primary) {
          die("Unable to create primary server object: ",
              Infoblox::status_code() . ":" . Infoblox::status_detail());
       }

       my $zone = Infoblox::DNS::Zone->new(
                                           name => "domain.com",
                                           primary => $primary
                                          );
       unless ($zone) {
          die("Construct zone failed: ",
              Infoblox::status_code() . ":" . Infoblox::status_detail());
       }
       print "Zone object created successfully\n";

       #Verify if the zone exists
       my $object = $session->get(object => "Infoblox::DNS::Zone", name => "domain.com");
       unless ($object) {
          print "Zone does not exist on server, safe to add the zone\n";
          $session->add($zone)
             or die("Add zone failed: ",
                    $session->status_code() . ":" . $session->status_detail());
       }
       print "Zone added successfully\n";
```

```
        #Retrieving zone back from the server in order to sign it
        $zone = $session->get(object => "Infoblox::DNS::Zone", name => "domain.com");
        unless($zone) {
           die("Retrieving zone back failed: ",
                Infoblox::status_code( ). ":". Infoblox::status_detail( ));
        }
        print "Zone retrieved for signing successfully.\n";

        $zone->dnssec_ksk_algorithm("NSEC3RSASHA1") &&
        $zone->dnssec_zsk_algorithm("NSEC3RSASHA1") &&
        $zone->dnssec_ksk_size(640) &&
        $zone->dnssec_zsk_size(640)
          or die("Changing the zone DNSSEC setting failed: ",
                 Infoblox::status_code() . ":" . Infoblox::status_detail());
        $session->modify($zone)
          or die("Modifying dnssec values in zone failed: ",
                 Infoblox::status_code() . ":" . Infoblox::status_detail());
        print "Zone modified successfully\n";

        #Signing the zone
        $zone->dnssec_signed("true")
           or die("Signing of the zone failed: ",
                  Infoblox::status_code() . ":" . Infoblox::status_detail());
               print "Zone signed successfully\n";
```

#Getting the NSEC3PARAM record by zone name

```
       my $nsec3param=$session->get(
                                   object => "Infoblox::DNS::Record::NSEC3PARAM",
                                   name   => "domain.com",
                                   view   => "default"
                                  );
            unless($nsec3param) {
         die("Getting NSEC3PARAM record failed: ",
            Infoblox::status_code() . ":" . Infoblox::status_detail());
            }
            print "Got NSEC3PARAM record successfully.\n";
```

#Searching for NSEC3PARAM objects using regular expressions

```
      my @retrieved_objs=$session->search(
                                          object => "Infoblox::DNS::Record::NSEC3PARAM",
                                          name   => ".*com",
                                          salt   => ".*",
                                          iterations => 100,
                                          view   => "default"
                                         );
           unless(@retrieved_objs>0) {
         die("Searching for NSEC3PARAM objects failed: ",
            Infoblox::status_code() . ":" . Infoblox::status_detail());
           }
           print "Search for the NSEC3PARAM objects successful, ".scalar(@retrieved_objs)." objects
found\n";
```

#Removing the created zone and cleaning up the view

```
      $session->remove($zone)
        or die("Unable to remove the zone: ",
               Infoblox::status_code() . ":" . Infoblox::status_detail());
      print "Zone removed successfully\n";

      $default_view->dnssec_enabled("false")
      && $default_view->override_dnssec("false")
      && $session->modify($default_view)
       or die("Restoring dnssec_enabled value in the default view failed: ",
              Infoblox::status_code() . ":" . Infoblox::status_detail());

 ####PROGRAM ENDS####
```

AUTHOR

Infoblox Inc. http://www.infoblox.com/

SEE ALSO

Infoblox::Session, Infoblox::Session->get(), Infoblox::Session->search(), Infoblox::DNS::Record::NSEC3, Infoblox::DNS::View, Infoblox::DNS::Zone

---

COPYRIGHT

Copyright (c) 2017 Infoblox Inc.

Infoblox::DNS::Record::NSEC - NSEC Record object

- NAME
- DESCRIPTION
- CONSTRUCTOR
- SESSION METHODS
  - Infoblox::Session->get( )
  - Infoblox::Session->search( )
- METHODS
  - cloud_info( )
  - creator( )
  - dns_name( )
  - dns_next_domain_name( )
  - name( )
  - next_domain_name( )
  - ttl( )
  - types( )
  - view( )
  - zone( )
- SAMPLE CODE
- AUTHOR
- SEE ALSO
- COPYRIGHT

NAME

Infoblox::DNS::Record::NSEC - NSEC Record object

DESCRIPTION

The NSEC resource record is one of the resource records included in the DNS security extension mechanism (DNSSEC). This record is used to provide authenticated denial of existence of a resource record in responce to a resolver query.

The NSEC record is defined in RFC 4034.

The NSEC records are automatically generated upon the signing of an authoritative zone. The NSEC Record object is read-only.

CONSTRUCTOR

The Infoblox::DNS::Record::NSEC object is a read-only object and does not require manual construction.

SESSION METHODS

This section describes all the methods in an Infoblox::Session that can be applied to an NSEC Record object.

Infoblox::Session->get( )

Use this method to retrieve the existing objects from an Infoblox appliance. See Infoblox::Session->get() for parameters and return values.

Key Reference

```
 Apply the following attributes to get the NSEC Record object(s):

     name             - Optional. FQDN of the NSEC record (.e., FQDN of the owning RRset).
     next_domain_name - Optional. Next owner name (in the canonical ordering of the zone) that has
authoritative data
     ttl               - Optional. TTL value for the record
     types            - Optional. Identifies the RRset types that exists at the NSEC RR's owner name
     view             - Optional. The DNS view in which the record is located. By default, all DNS
views are searched. But if you omit this attribute and specify a zone, the appliance searches the
'default' view only.
     zone             - Optional. A zone name in FQDN format.
```

Example

```
   #get an NSEC record by name, DNS view name and record type
   my $nsec=$session->get(
                          object => "Infoblox::DNS::Record::NSEC",
                          name   => "a_record.test.com",
                          types  => "A",
                          view   => "default"
```

```
                              );
```

Infoblox::Session->search( )

Use this method to retrieve the existing objects from an Infoblox appliance. See Infoblox::Session->search() for parameters and return values.

Key Reference

```
 Apply the following attributes to search the NSEC Record object(s):

     name            - Optional. FQDN of the NSEC record. Regular expression.
     next_domain_name - Optional. Next owner name (in the canonical ordering of the zone) that has
authoritative data (regular expression).
     ttl              - Optional. TTL value for the record.
     types           - Optional. Identifies the RRset types that exist at the NSEC RR's owner name
(regular expression)
     view            - Optional. The DNS view in which the record is located. By default, all DNS
views are searched. But if you omit this attribute and specify a zone, the appliance searches the
'default' view only.
     zone            - Optional. A zone name in FQDN format.
```

Example

```
   #search for all NSEC records that match the regular expression for name and record type
   my @retrieved_objs=$session->search(
                          object => "Infoblox::DNS::Record::NSEC",
                          name   => "rec.*[.]test[.]com",
                          types  => "A|MX",
                        );

# search for all NSEC records in the "domain.com" zone of the default DNS view
# Note that the 'default' view is assumed implicitly here
my @retrieved_objs = $session->search(
    object => "Infoblox::DNS::Record::NSEC",
    zone   => "domain.com",
   );
```

METHODS

This section describes all the methods that can be used to retrieve the attribute values of an NSEC Record object.

cloud_info( )

Use this method to retrieve cloud API related information for the Infoblox::DNS::Record::NSEC object.

Parameter

None

Returns

The method returns the attribute value.

Example

```
# Get cloud_info
my $cloud_info = $object->cloud_info();
```

creator( )

Use this method to retrieve the NSEC record creator. This is a read-only attribute.

Parameter

None

Returns

The method returns the attribute value.

Example

```
# Get attribute value
my $value = $object->creator();
```

dns_name( )

Use this method to retrieve the FQDN, in punycode format, of the NSEC record. This is a read-only attribute.

Parameter

None

Returns

The method returns the attribute value.

Example

```
# Get attribute value
my $value = $nsec->dns_name();
```

dns_next_domain_name( )

Use this method to retrieve the next owner name in punycode format. This is a read-only attribute.

Parameter

None

Returns

The method returns the attribute value.

Example

```
# Get attribute value
my $value = $nsec->dns_next_domain_name();
```

name( )

Use this method to retrieve the FQDN of the NSEC record (which is the same as the name of the owning record).

The attribute value can be in unicode format.

Parameter

none

Returns

The method returns the FQDN of the NSEC record attribute value.

Example

```
#Get the FQDN of the NSEC record
my $name = $nsec->name();
```

next_domain_name( )

Use this method to retrieve the next owner name, in the canonical order of the zone, that has authoritative data.

The attribute value can be in unicode format.

Parameter

none

Returns

The method returns the next owner name (in the canonical ordering of the zone) that has authoritative data. The returned data is a string in FQDN format.

Example

```
#Get  next owner name (in the canonical ordering of the zone) that has authoritative data
my $next_domain_name = $nsec->next_domain_name();
```

ttl( )

Use this method to retrieve the Time to Live (TTL) value of an NSEC record object.

Parameter

none

Returns

The method returns the TTL attribute value. The returned parameter is a 32-bit integer (range from 0 to 4294967295) that represents the duration, in seconds, that the record is cached. Zero indicates that the record should not be cached.

Example

```
#Get TTL
my $ttl = $nsec->ttl();
```

types( )

Use this method to retrieve the types field of an NSEC record. This field identifies the RRSet types that exist at the original owner name of the NSEC RR.

Parameter

none

Returns

The method returns the RRSet types that exist at the owner name of the NSEC RR attribute value. The returned value is a string containing RR type mnemonics separated by a space; for example, "NS SOA RRSIG DNSKEY".

Example

```
#Get the types field value
my $types = $nsec->types();
```

view( )

Use this method to retrieve the DNS view object that contains the NSEC Record object.

Parameter

none

Returns

The method returns the Infoblox::DNS::View object that contains the NSEC record.

Example

```
#Get the view
my $view = $nsec->view();
```

zone( )

Use this method to retrieve the zone name of an NSEC record.

Parameter

none

Returns

Returns the zone name in FQDN format.

Example

```
# Get zone
my $zone = $nsec->zone();
```

SAMPLE CODE

The following sample code demonstrates the session methods on an NSEC record object.

```
#PROGRAM STARTS: Include all the modules that will be used
use strict;
use Infoblox;

#Create a session to the Infoblox appliance
       my $session = Infoblox::Session->new(
           master   => "192.168.1.2",
           username => "admin",
           password => "infoblox"
       );
```

```
unless ($session) {
   die("Construct session failed: ",
       $session->status_code() . ":" . $session->status_detail());
}
print "Session created successfully\n";

#Enable DNSSEC in the default view

my $default_view=$session->get(
                                object=> "Infoblox::DNS::View",
                                name => "default"
                               );
unless($default_view) {
    die("Getting the default view failed:",
       Infoblox::status_code() . ":" . Infoblox::status_detail());
}
print "Got the default view successfully\n";

$default_view->dnssec_enabled("true")
    or die("Changing the dnssec_enabled in the default view failed:",
            Infoblox::status_code() . ":" . Infoblox::status_detail());
$session->modify($default_view)
    or die("Changing the dnssec_enabled in the default view failed:",
            Infoblox::status_code() . ":" . Infoblox::status_detail());
```

#Create a signed zone and populate it with an A record to demonstrate NSEC record

```
print "Creating Member primary server for the zone\n";
my $primary=Infoblox::DNS::Member->new(
                                       ipv4addr => "192.168.1.2",
                                       name => "infoblox.localdomain",
                                      );
unless($primary) {
   die("Unable to create primary server object: ",
       Infoblox::status_code() . ":" . Infoblox::status_detail());
}

my $zone = Infoblox::DNS::Zone->new(
                                    name => "domain.com",
                                    primary => $primary
                                   );
unless ($zone) {
   die("Construct zone failed: ",
       Infoblox::status_code() . ":" . Infoblox::status_detail());
}
print "Zone object created successfully\n";

#Verify if the zone exists
my $object = $session->get(object => "Infoblox::DNS::Zone", name => "domain.com");
unless ($object) {
   print "Zone does not exist on server, safe to add the zone\n";
   $session->add($zone)
      or die("Add zone failed: ",
             $session->status_code() . ":" . $session->status_detail());
}
print "Zone added successfully\n";

#Adding A record to the zone to demonstrate the corresponding NSEC record
my $a_record=Infoblox::DNS::Record::A->new(
                                           ipv4addr => "10.9.8.7",
                                           name => "recorda.domain.com"
                                          );
unless($a_record) {
   die("Creating A record failed : ",
      Infoblox::status_code() . ":" . Infoblox::status_detail());
}

$object = $session->get( object => "Infoblox::DNS::Record::A", name => "recorda.domain.com" );
unless ($object) {
   print "A record does not exist on the server, safe to add the A record\n";
   $session->add($a_record)
     or die("Adding A record failed: ",
            Infoblox::status_code() . ":" . Infoblox::status_detail());
}
print "A record added to the zone successfully\n";
```

```
        #Retrieving zone back from the server in order to sign it
        $zone = $session->get(object => "Infoblox::DNS::Zone", name => "domain.com");
        unless($zone) {
           die("Retrieving zone back failed: ",
                Infoblox::status_code( ). ":". Infoblox::status_detail( ));
        }
        print "Zone retrieved for signing successfully.\n";

        $zone->dnssec_ksk_algorithm("RSASHA1") &&
        $zone->dnssec_zsk_algorithm("RSASHA1") &&
        $zone->dnssec_ksk_size(640) &&
        $zone->dnssec_zsk_size(640)
          or die("Changing the zone DNSSEC setting failed: ",
                 Infoblox::status_code() . ":" . Infoblox::status_detail());
        $session->modify($zone)
          or die("Modifying dnssec values in zone failed: ",
                 Infoblox::status_code() . ":" . Infoblox::status_detail());
        print "Zone modified successfully\n";

        #Signing the zone
        $zone->dnssec_signed("true")
           or die("Signing of the zone failed: ",
                  Infoblox::status_code() . ":" . Infoblox::status_detail());
              print "Zone signed successfully\n";
```

#Getting the NSEC record corresponding to the A record

```
       my $nsec_record_a=$session->get(
                                  object => "Infoblox::DNS::Record::NSEC",
                                  name   => "recorda.domain.com",
                                  view   => "default"
                                 );
            unless($nsec_record_a) {
         die("Getting NSEC corresponding to the A record failed: ",
            Infoblox::status_code() . ":" . Infoblox::status_detail());
            }
            print "Got NSEC record successfully, next_domain_name value: ".$nsec_record_a-
>next_domain_name()."\n";
```

#Searching for NSEC objects using regular expressions

```
      my @retrieved_objs=$session->search(
                                       object => "Infoblox::DNS::Record::NSEC",
                                       name   => ".*domain[.]com",
                                       types  => "A|MX|SOA",
                                       view   => "default"
                                      );
          unless(@retrieved_objs>0) {
         die("Searching for NSEC objects failed: ",
            Infoblox::status_code() . ":" . Infoblox::status_detail());
          }
          print "Search for the NSEC objects successful, ".scalar(@retrieved_objs)." objects found\n";
```

#Removing the created zone and cleaning up the view

```
      $session->remove($zone)
        or die("Unable to remove the zone: ",
               Infoblox::status_code() . ":" . Infoblox::status_detail());
      print "Zone removed successfully\n";

      $default_view->dnssec_enabled("false")
      && $default_view->override_dnssec("false")
      && $session->modify($default_view)
       or die("Restoring dnssec_enabled value in the default view failed: ",
              Infoblox::status_code() . ":" . Infoblox::status_detail());

 ####PROGRAM ENDS####
```

AUTHOR

Infoblox Inc. http://www.infoblox.com/

SEE ALSO

Infoblox::Session, Infoblox::Session->get(), Infoblox::Session->search(), Infoblox::DNS::Record::NSEC3, Infoblox::DNS::Record::NSEC3PARAM, Infoblox::DNS::View, Infoblox::DNS::Zone

---

COPYRIGHT

Copyright (c) 2017 Infoblox Inc.

Infoblox::DNS::RPZRecord::AIpAddress - Response Policy Zone Rule object.

- NAME
- DESCRIPTION
- CONSTRUCTOR
- SESSION METHODS
  - Infoblox::Session->add( )
  - Infoblox::Session->get( )
  - Infoblox::Session->modify( )
  - Infoblox::Session->remove( )
  - Infoblox::Session->search( )
- METHODS
  - comment( )
  - disable( )
  - extattrs( )
  - extensible_attributes( )
  - ipv4addr( )
  - name( )
  - ttl( )
  - views( )
  - zone( )
  - rp_zone( )
- SAMPLE CODE
- AUTHOR
- SEE ALSO
- COPYRIGHT

NAME

Infoblox::DNS::RPZRecord::AIpAddress - Response Policy Zone (RPZ) Substitute (IPv4 Address) Rule object.

DESCRIPTION

An RPZ AIpAddress is an Substitute (IPv4 Address) Rule that maps an IP address represented by a host name to a substitute IPv4 address. To define a specific address-to-address mapping, add an Substitute (IPv4 Address) Rule to a previously defined Response Policy Zone.

This record represents the substitution rule for IP trigger policy. It matches IP addresses that would otherwise appear in A record in the "answer" section of DNS response.

You should use this object to create IP address substitution rules instead usage Infoblox::DNS::RPZRecord::CNAME::IpAddress object.

Note that you must specify only one view for the attribute "views".

CONSTRUCTOR

```
my $rpz_aip = Infoblox::DNS::RPZRecord::AIpAddress->new (
    ipv4addr => $ipv4addr,          #Required
    name     => $fqdn,              #Required
    rp_zone  => $rp_zone            #Required
    comment  => $string,            #Optional / Default is empty
    disable  => "true" | "false"    #Optional / Default "false"
    extattrs              => { $string => $extattr, ... },      #Optional / Default is undefined
    extensible_attributes => { $string => $string | $num, $string => [ $string | $num, ... ], ... },
#Optional / Default is undefined
    last_discovered  => $string,    #Optional / Default is empty
    ttl      => $num | undef,       #Optional / Default is undefined
    views    => [$View1],           #Optional / Default is "default" view
);
```

You cannot set both extattrs and extensible_attributes attributes at the same time.

SESSION METHODS

This section describes all the methods in Infoblox::Session module that you can apply to a Substitute (IPv4 Address) Rule object.

Infoblox::Session->add( )

Use this method to add an object to the Infoblox appliance. See Infoblox::Session->add() for parameters and return values.

Example

```
#Construct an object
my $rpz_aip = Infoblox::DNS::RPZRecord::AIpAddress->new (
    ipv4addr => "1.1.1.1",
    name     => "10.6.0.0/16.rpz.com",
    rp_zone  => response_policy_zone1,
    comment  => "this is a demo rule",
    disable  => "true",
    ttl      => "33",
    views    => [$view1]  );
# Submit for addition
my $response = $session->add( $rpz_aip );
```

Infoblox::Session->get( )

Use this method to retrieve all the matching objects from the Infoblox appliance. See Infoblox::Session->get() for parameters and return values.

Key References

```
Apply the following attributes to get a specific Substitute (IPv4 Address) Rule object:

  name - Optional. A host name in string format.
  view - Optional. The DNS view in which the RPZ Substitute (IPv4 Address) Rule is located. By
default, all DNS views are searched. But if you omit this attribute and specify a zone, the appliance
searches the 'default' view only.
  zone - Optional. A zone name in FQDN format.
  ipv4addr - Optional. The address of the host record.
  extattrs - Optional. A hash reference containing extensible attributes.
  extensible_attributes - Optional. A hash reference containing extensible attributes.
```

Example

```
my @retrieved_objs = $session->get(
    object => "Infoblox::DNS::RPZRecord::AIpAddress",
    name   => "1.2.3.4.rpz.com",
    view   => "default" );

#get all Substitute (IPv4 Address) Rule recods with the extensible attribute 'Site'
my @retrieved_objs = $session->get(
   object => "Infoblox::DNS::RPZRecord::AIpAddress",
   extensible_attributes => { 'Site' => 'Santa Clara' });
```

Infoblox::Session->modify( )

Use this method to modify an object in the Infoblox appliance. See Infoblox::Session->modify() for parameters and return values.

Example

```
# Use method to modify the comment.
$rpz_aip->comment("this is a modified comment");
# Submit modification
my $response = $session->modify( $rpz_aip );
```

Infoblox::Session->remove( )

Use this method to submit the removal of an object from the Infoblox appliance. See Infoblox::Session->remove() for parameters and return values.

To remove a specifc object, first use `get()` or `search()` to retrieve the specific object, and then submit this object for removal.

Example

```
# Get the objects with the same name
my @retrieved_objs = $session->get(
    object => "Infoblox::DNS::RPZRecord::AIpAddress",
    name   => "bind_a.rpz.com",
    view   => "default" );
# Find the desired object from the retrieved list.
my $desired_a = $retrieved_objs[0];
# Submit for removal
my $response = $session->remove( $desired_a );
```

Infoblox::Session->search( )

Use this method to search for Substitute (IPv4 Address) Rule objects in the Infoblox appliance. See Infoblox::Session->search() for para meters and return values.

Key References

```
 Apply the following attributes to search for a Substitute (IPv4 Address) Rule object:

  name     - Optional. A host name in string format (regular expression).
  view     - Optional. The DNS view in which the RPZ Substitute (IPv4 Address) Rule is located. By
default, all DNS views are searched. But if you omit this attribute and specify a zone, the appliance
searches the 'default' view only.
  zone     - Optional. A zone name in FQDN format.
  comment  - Optional. A comment in string format (regular expression).
  ipv4addr - Optional. The address of the host record (regular expression).
  extattrs - Optional. A hash reference containing extensible attributes.
  extensible_attributes - Optional. A hash reference containing extensible attributes.
```

For more information about searching extensible attributes, see Infoblox::Grid::ExtensibleAttributeDef/Searching Extensible Attributes.

Example

```
# search for all Substitute (IPv4 Address) Rule objects that match "rpz.com" in the default DNS view
my @retrieved_objs = $session->search(
    object => "Infoblox::DNS::RPZRecord::AIpAddress",
    name   => 'rpz\.com',
    view   => "default" );

# search for all Substitute (IPv4 Address) Rules in the "rpz.com" zone of the default view
my @retrieved_objs = $session->search(
    object => "Infoblox::DNS::RPZRecord::AIpAddress",
    zone   => "rpz.com",
    view   => "default" );

#get all Substitute (IPv4 Address) Rules with the extensible attribute 'Site'
my @retrieved_objs = $session->search(
  object => "Infoblox::DNS::RPZRecord::AIpAddress",
  extensible_attributes => { 'Site' => 'Santa Clara' });
```

METHODS

This section describes all the methods that you can use to configure and retrieve the attribute values of an RPZ Substitute (IPv4 Address) Rule.

comment( )

Use this method to set or retrieve the descriptive comment.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

Desired comment in string format with a maximum of 256 bytes.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get comment
my $comment = $rpz_aip->comment();
#Modify comment
$rpz_aip->comment("Modifying the Substitute (IPv4 Address) Rule comment");
```

disable( )

Use this method to set or retrieve the disable flag of a DNS record.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

The default value for this field is false. The DNS record is enabled.

Parameter

Specify "true" to set the disable flag or "false" to deactivate/unset it.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get disable
my $disable = $rpz_aip->disable();
#Modify disable
$rpz_aip->disable("true");
```

extattrs( )

Use this method to set or retrieve the extensible attributes associated with a Substitute (IPv4 Address) Rule object.

Parameter

Valid value is a hash reference containing the names of extensible attributes and their associated values (Infoblox::Grid::Extattr objects).

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get extattrs
my $ref_extattrs = $rpz_aip->extattrs();
#Modify extattrs
$rpz_aip->extattrs({ 'Site' => $extattr1, 'Administrator' => $extattr2 });
```

extensible_attributes( )

Use this method to set or retrieve the extensible attributes associated with a Substitute (IPv4 Address) Rule.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

For valid values for extensible attributes, see Infoblox::Grid::ExtensibleAttributeDef/Extensible Attribute Values.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get extensible attributes
my $ref_extensible_attributes = $rpz_aip->extensible_attributes();
#Modify extensible attributes
$rpz_aip->extensible_attributes({ 'Site' => 'Santa Clara', 'Administrator' => [ 'Peter', 'Tom' ] });
```

ipv4addr( )

Use this method to set or retrieve the the IPv4 address.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

An IPv4 address is a 32-bit number in dotted decimal notation. It consists of four 8-bit groups of decimal digits separated by decimal points (example: 192.168.1.2).

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get ipv4addr
my $ipv4addr = $rpz_aip->ipv4addr();
#Modify ipv4addr
$rpz_aip->ipv4addr("2.2.2.2");
```

name( )

Use this method to set or retrieve the host name.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

Hostname in FQDN (Fully Qualified Domain Name) format. The FQDN consists of the network address in CIDR notation or host ip address followed by the domain name (example: 10.0.0.0/8.rpz.com, 1.2.3.4.rpz.com).

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get name
my $name = $rpz_aip->name();
#Modify name
$rpz_aip->name("1.2.3.4.rpz.com");
```

ttl( )

Use this method to set or retrieve the Time to Live (TTL) value.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

The default value is undefined which indicates that the record inherits the TTL value of the zone.

Specify a TTL value to override the TTL value at the zone level.

Parameter

A 32-bit integer (range from 0 to 4294967295) that represents the duration in seconds that the record is cached. Zero indicates that the record should not be cached.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get ttl
my $ttl = $rpz_aip->ttl();
#Modify ttl
$rpz_aip->ttl(1800);
#Un-override ttl
$rpz_aip->ttl(undef);
```

views( )

Use this method to set or retrieve the view of the RPZ Substitute (IPv4 Address) Rule.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

The default value is the "default" view, which means the RPZ Substitute (IPv4 Address) Rule is in the default view.

Parameter

An array reference of defined Infoblox::DNS::View objects.

Note that the array size must be 1.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get views
my $ref_views = $rpz_aip->views();
#Modify views, an array of Infoblox::DNS::View objects
```

```
$rpz_aip->views([$view1]);
```

zone( )

Use this method to retrieve the zone name of a Substitute (IPv4 Address) Rule. This method is read-only and cannot be set.

Parameter

None

Returns

Returns the attribute value.

Example

```
# Get zone
my $zone = $rpz_aip->zone();
```

rp_zone( )

Use this method to set or retrieve the zone object of a Substitute (IPv4 Address) Rule.

Parameter

An Infoblox::DNS::Zone object.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
# Get rp_zone
my $rp_zone = $rpz_aipa->rp_zone();
#Modify rp_zone, reference of Infoblox::DNS::Zone object
$rpz_aipa->rp_zone($response_policy_zone);
```

SAMPLE CODE

The following sample code demonstrates the different functions that can be applied to an object, such as add, search, modify, and remove. This sample also includes error handling for the operations.

#Preparation prior to a Substitute (IPv4 Address) Rule insertion

```
#PROGRAM STARTS: Include all the modules that will be used
use strict;
use Infoblox;

#Create a session to the Infoblox Appliance
my $session = Infoblox::Session->new(
    master   => "192.168.1.2",
    username => "admin",
    password => "infoblox"
);
unless ($session) {
   die("Construct session failed: ",
       $session->status_code() . ":" . $session->status_detail());
}
print "Session created successfully\n";

#Create the RPZ zone prior to an RPZ Substitute (IPv4 Address) Rule insertion
my $zone = Infoblox::DNS::Zone->new(name => "rpz.com",
                                    rpz_policy => "GIVEN");
unless ($zone) {
die("Construct zone failed: ",
    Infoblox::status_code() . ":" . Infoblox::status_detail());
}
print "Zone object created successfully\n";

#Verify if the zone exists
my $object = $session->get(object => "Infoblox::DNS::Zone", name => "rpz.com");
unless ($object) {
print "Zone does not exist on server, safe to add the zone\n";
$session->add($zone)
```

```
    or die("Add zone failed: ",
           $session->status_code() . ":" . $session->status_detail());
}
```

#Create a Substitute (IPv4 Address) Rule

```
my $view1 = Infoblox::DNS::View->new(
 name      => "default" );

#Construct a Substitute (IPv4 Address) Rule object
my $ip_address = Infoblox::DNS::RPZRecord::AIpAddress->new(
 name      => "10.6.0.0/16.rpz.com",
 ipv4addr => "111.222.11.22",
 comment   => "this is a demo A ip address record",
 ttl       => "33",
 views     => [$view1],
 rp_zone   => $zone);
unless ($ip_address) {
die("Construct DNS record CNAME failed: ",
    Infoblox::status_code() . ":" . Infoblox::status_detail());
}
print "DNS RPZ CNAME object created successfully\n";

#Add the DNS RPZ CNAMEIpRecord record object to the Infoblox appliance through a session
$session->add($ip_address)
 or die("Add record cname failed: ",
        $session->status_code() . ":" . $session->status_detail());
print "DNS RPZ CNAME object added to server successfully\n";
```

#Search for a specific DNS RPZ CNAMEIpRecord record

```
#Search all CNAMEIpRecord records that match "rpz.com"
my @retrieved_objs = $session->search(
 object => "Infoblox::DNS::RPZRecord::AIpAddress",
 name   => 'rpz\.com'
);
my $object = $retrieved_objs[0];
unless ($object) {
 die("Search Substitute (IPv4 Address) Rule failed: ",
     $session->status_code() . ":" . $session->status_detail());
}
print "Search Substitute (IPv4 Address) Rule object found at least 1 matching entry\n";

#Search all Substitute (IPv4 Address) Rules that start with "bind" and end with ".rpz.com"
my @retrieved_objs = $session->search(
 object => "Infoblox::DNS::RPZRecord::AIpAddress",
 name   => '^10\.6.*\.rpz\.com'
);
my $object = $retrieved_objs[0];
unless ($object) {
 die("Search Substitute (IPv4 Address) Rule failed: ",
     $session->status_code() . ":" . $session->status_detail());
}
print "Search Substitute (IPv4 Address) Rule object using regexp found at least 1 matching entry\n";
```

#Get and modify a Substitute (IPv4 Address) Rule

```
#Get Substitute (IPv4 Address) Rule through the session
my @retrieved_objs = $session->get(
 object => "Infoblox::DNS::RPZRecord::AIpAddress",
 name   => "10.6.0.0/16.rpz.com",
 view   => "default"
);
my $object = $retrieved_objs[0];
unless ($object) {
 die("Get Substitute (IPv4 Address) Rule failed: ",
     $session->status_code() . ":" . $session->status_detail());
}
print "Get Substitute (IPv4 Address) Rule object found at least 1 matching entry\n";

#Modify one of the attributes of the specified Substitute (IPv4 Address) Rule
$object->disable("true");

#Apply the changes
$session->modify($object)
 or die("Modify Substitute (IPv4 Address) Rule failed: ",
        $session->status_code() . ":" . $session->status_detail());
```

```
print "Substitute (IPv4 Address) Rule object modified successfully \n";
```

#Remove a Substitute (IPv4 Address) Rule

```
#Get Substitute (IPv4 Address) Rule through the session
my @retrieved_objs = $session->get(
 object => "Infoblox::DNS::RPZRecord::AIpAddress",
 name   => "10.6.0.0/16.rpz.com",
 view   => "default"
);
$object = $retrieved_objs[0];
unless ($object) {
 die("Get Substitute (IPv4 Address) Rule failed: ",
     $session->status_code() . ":" . $session->status_detail());
}
print "Get Substitute (IPv4 Address) Rule object found at least 1 matching entry\n";

#Submit the object for removal
$session->remove( $object )
or die("Remove Substitute (IPv4 Address) Rule failed: ",
     $session->status_code() . ":" . $session->status_detail());
print "Substitute (IPv4 Address) Rule object removed successfully \n";

####PROGRAM ENDS####
```

AUTHOR

Infoblox Inc. http://www.infoblox.com/

SEE ALSO

Infoblox::DNS::View, Infoblox::DNS::Zone, Infoblox::Session, Infoblox::Session->get(), Infoblox::Session->search(), Infoblox::Session->add(), Infoblox::Session->remove(), Infoblox::Session->modify()

COPYRIGHT

Copyright (c) 2017 Infoblox Inc.

Infoblox::DNS::RPZRecord::AAAAIpAddress - Response Policy Zone Rule object.

- NAME
- DESCRIPTION
- CONSTRUCTOR
- SESSION METHODS
  - Infoblox::Session->add( )
  - Infoblox::Session->get( )
  - Infoblox::Session->modify( )
  - Infoblox::Session->remove( )
  - Infoblox::Session->search( )
- METHODS
  - comment( )
  - disable( )
  - extattrs( )
  - extensible_attributes( )
  - ipv6addr( )
  - name( )
  - ttl( )
  - views( )
  - zone( )
  - rp_zone( )
- SAMPLE CODE
- AUTHOR
- SEE ALSO
- COPYRIGHT

NAME

Infoblox::DNS::RPZRecord::AAAAIpAddress - Response Policy Zone (RPZ) Substitute (IPv6 Address) Rule object.

DESCRIPTION

An RPZ Substitute (IPv6 Address) Rule maps an IP address represented by a host name to a substitute IPv6 address. To define a specific address-to-address mapping, add an RPZ Substitute (IPv6 Address) Rule to a previously defined Response Policy Zone.

This record represents the substitution rule for IP trigger policy. It matches IP addresses that would otherwise appear in AAAA record in the "answer" section of DNS response.

You should use this object to create IP address substitution rules instead usage Infoblox::DNS::RPZRecord::CNAME::IpAddress object.

Note that you must specify only one view for the attribute "views".

CONSTRUCTOR

```
 my $rpz_4aip = Infoblox::DNS::RPZRecord::AAAAIpAddress->new (
     ipv6addr => $ipv6addr,          #Required
     name     => $fqdn,              #Required
     rp_zone  => $rp_zone,           #Required
     comment  => $string,            #Optional / Default is empty
     extattrs              => { $string => $extattr, ... },      #Optional / Default is undefined
     extensible_attributes => { $string => $string | $num, $string => [ $string | $num, ... ], ... },
#Optional / Default is undefined
     disable  => "true" | "false"    #Optional / Default "false"
     ttl      => $num | undef,       #Optional / Default is undefined
     views    => [$View1],           #Optional / Default is "default" view
 );
```

You cannot set both extattrs and extensible_attributes attributes at the same time.

SESSION METHODS

This section describes all the methods in Infoblox::Session module that you can apply to a Substitute (IPv6 Address) Rule object.

Infoblox::Session->add( )

Use this method to add an RPZ Substitute (IPv6 Address) Rule object to the Infoblox appliance. See Infoblox::Session->add() for parameters and return values.

Example

```
#Construct an object
my $rpz_4aip = Infoblox::DNS::RPZRecord::AAAAIpAddress->new(
    name     => "2011::/16.rpz.com",
    comment  => "add a Substitute (IPv6 Address) Rule aaaa1.rpz.com",
    ipv6addr => "aaaa:1111:bbbb:2222:cccc:3333:dddd:4444",
    rp_zone  => $response_policy_zone,
    disable  => "true",
    ttl      => "33",
    views    => [$view1]  );
# Submit for addition
my $response = $session->add( $rpz_4aip );
```

Infoblox::Session->get( )

Use this method to retrieve all the matching objects from the Infoblox appliance. See Infoblox::Session->get() for parameters and return values.

Key References

```
Apply the following attributes to get a specific Substitute (IPv6 Address) Rule object:

name                                - Optional. A host name in string format.
view                                - Optional. The DNS view in which the host record is located.
By default, all DNS views are searched. But if you omit this attribute and specify a zone, the
appliance searches the 'default' view only.
zone                                - Optional. A zone name in FQDN format.
ipv6addr                            - Optional. The address of the RPZ Substitute (IPv6 Address)
Rule.
extattrs                            - Optional. A hash reference containing extensible attributes.
extensible_attributes               - Optional. A hash reference containing extensible attributes.
```

Example

```
my @retrieved_objs = $session->get(
                     object => "Infoblox::DNS::RPZRecord::AAAAIpAddress",
                     name   => "2001::1.rpz.com",
                     view   => "default" );

# get all the Substitute (IPv6 Address) Rules in the "rpz.com" zone of the default view
my @retrieved_objs = $session->get(
                     object => "Infoblox::DNS::RPZRecord::AAAAIpAddress",
                     zone   => "rpz.com",
                     view   => "default" );

#get all Substitute (IPv6 Address) Rule recods with the extensible attribute 'Site'
my @retrieved_objs = $session->get(
   object => "Infoblox::DNS::RPZRecord::AAAAIpAddress",
   extensible_attributes => { 'Site' => 'Santa Clara' });
```

Infoblox::Session->modify( )

Use this method to modify an RPZ Substitute (IPv6 Address) Rule object in the Infoblox appliance. See Infoblox::Session->modify() for parameters and return values.

Example

```
# Use method to modify the comment.
$rpz_4aip->comment("this is a modified comment");
# Submit modification
my $response = $session->modify( $rpz_4aip );
```

Infoblox::Session->remove( )

Use this method to remove an RPZ Substitute (IPv6 Address) Rule object from the Infoblox appliance. See Infoblox::Session->remove() for parameters and return values.

To remove a specifc object, first use `get()` or `search()` to retrieve the specific RPZ Substitute (IPv6 Address) Rule object, and then submit this object for removal.

Example

```
# Get the objects with the same name
my @retrieved_objs = $session->get(
    object => "Infoblox::DNS::RPZRecord::AAAAIpAddress",
    name   => "aaaa1.rpz.com",
    view   => "default" );
# Find the desired object from the retrieved list.
my $desired_aaaa = $retrieved_objs[0];
```

```
# Submit for removal
my $response = $session->remove( $desired_aaaa );
```

Infoblox::Session->search( )

Use this method to search for Substitute (IPv6 Address) Rule objects in the Infoblox appliance. See Infoblox::Session->search() for parameters and return values.

Key References

```
Apply the following attributes, to search for a Substitute (IPv6 Address) Rule object:

  name                                  - Optional. A host name in string format (regular
expression).
  view                                  - Optional. The DNS view in which the RPZ Substitute (IPv6
Address) Rule is located. By default, all DNS views are searched. But if you omit this attribute and
specify a zone, the appliance searches the 'default' view only.
  zone                                  - Optional. A zone name in FQDN format.
  comment                               - Optional. Comment in string format (regular expression).
  ipv6addr                              - Optional. The address of the RPZ Substitute (IPv6 Address)
Rule (regular expression).
  extattrs                              - Optional. A hash reference containing extensible
attributes.
  extensible_attributes                 - Optional. A hash reference containing extensible
attributes.
```

For more information about searching extensible attributes, see Infoblox::Grid::ExtensibleAttributeDef/Searching Extensible Attributes.

Example

```
# search for all Substitute (IPv6 Address) Rule objects that match "rpz.com" in the default DNS view
my @retrieved_objs = $session->search(
    object  => "Infoblox::DNS::RPZRecord::AAAAIpAddress",
    name    => 'rpz\.com',
    view    => "default",
    comment => ".*modified comment" );

# search for all Substitute (IPv6 Address) Rules in zone "rpz.com" of default view
my @retrieved_objs = $session->search(
                     object => "Infoblox::DNS::RPZRecord::AAAAIpAddress",
                     zone   => "rpz.com",
                     view   => "default" );

# search all Substitute (IPv6 Address) Rule recods with the extensible attribute 'Site'
my @retrieved_objs = $session->search(
   object => "Infoblox::DNS::RPZRecord::AAAAIpAddress",
   extensible_attributes => { 'Site' => 'Santa Clara' });
```

METHODS

This section describes all the methods that you can use to configure and retrieve the attribute values of an RPZ Substitute (IPv6 Address) Rule.

comment( )

Use this method to set or retrieve the descriptive comment.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

Desired comment in string format with a maximum of 256 bytes.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get comment
my $comment = $rpz_4aip->comment();
#Modify comment
$rpz_4aip->comment("Modifying the Substitute (IPv6 Address) Rule comment");
```

disable( )

Use this method to set or retrieve the disable flag of a DNS record.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

The default value for this field is false. The DNS record is enabled.

Parameter

Specify "true" to set the disable flag or "false" to deactivate/unset it.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get disable
my $disable = $rpz_4aip->disable();
#Modify disable
$rpz_4aip->disable("true");
```

extattrs( )

Use this method to set or retrieve the extensible attributes associated with a Substitute (IPv6 Address) Rule object.

Parameter

Valid value is a hash reference containing the names of extensible attributes and their associated values (Infoblox::Grid::Extattr objects).

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get extattrs
my $ref_extattrs = $rpz_4aip->extattrs();
#Modify extattrs
$rpz_4aip->extattrs({ 'Site' => $extattr1, 'Administrator' => $extattr2 });
```

extensible_attributes( )

Use this method to set or retrieve the extensible attributes associated with a Substitute (IPv6 Address) Rule.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

For valid values for extensible attributes, see Infoblox::Grid::ExtensibleAttributeDef/Extensible Attribute Values.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get extensible attributes
my $ref_extensible_attributes = $rpz_4aip->extensible_attributes();
#Modify extensible attributes
$rpz_4aip->extensible_attributes({ 'Site' => 'Santa Clara', 'Administrator' => [ 'Peter', 'Tom' ]
});
```

ipv6addr( )

Use this method to set or retrieve the the IPv6 address.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

An IPv6 address is a 128-bit number in colon hexadecimal notation. It consists of eight 16-bit groups of hexadecimal digits separated by colons (example: 12ab:0000:0000:0123:4567:89ab:0000:cdef).

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get ipv6addr
my ipv6addr = $rpz_4aip->ipv6addr();
#Modify ipv6addr
$rpz_4aip->ipv6addr("aaaa:1111:bbbb:2222:cccc:3333:dddd:5555");
```

name( )

Use this method to set or retrieve the host name.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

Hostname in FQDN (Fully Qualified Domain Name) format. The FQDN consists of the network address in CIDR notation or host ip address followed by the domain name (example: 2001::/16.rpz.com, 2001:2002::2203.rpz.com).

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get name
my $name = $rpz_4aip->name();
#Modify name
$rpz_4aip-> name("2001::/64.rpz.com");
```

ttl( )

Use this method to set or retrieve the Time to Live (TTL) value.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

The default value is undefined which indicates that the record inherits the TTL value of the zone.

Specify a TTL value to override the TTL value at the zone level.

Parameter

A 32-bit integer (range from 0 to 4294967295) that represents the duration in seconds that the record is cached. Zero indicates that the record should not be cached.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get ttl
my $ttl = $rpz_4aip->ttl();
#Modify ttl
$rpz_4aip->ttl(1800);
#Un-override ttl
$rpz_4aip->ttl(undef);
```

views( )

Use this method to set or retrieve the view of the RPZ Substitute (IPv6 Address) Rule.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

The default value is the "default" view, which means the RPZ Substitute (IPv6 Address) Rule is located under the default view.

Parameter

An array reference of defined Infoblox::DNS::View objects.

Note that the array size must be 1.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get views
my $ref_views = $rpz_4aip->views();
#Modify views, an array of Infoblox::DNS::View objects
$rpz_4aip->views([$view1]);
```

zone( )

Use this method to retrieve the zone name of a Substitute (IPv6 Address) Rule. This method is read-only and cannot be set.

Parameter

None

Returns

Returns the attribute value.

Example

```
# Get zone
my $zone = $rpz_4aip->zone();
```

rp_zone( )

Use this method to set or retrieve the zone object of a Substitute (IPv6 Address) Rule.

Parameter

An Infoblox::DNS::Zone object.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
# Get rp_zone
my $rp_zone = $rpz_4aip->rp_zone();
#Modify rp_zone, reference of Infoblox::DNS::Zone object
$rpz_4aip->rp_zone($response_policy_zone);
```

SAMPLE CODE

The following sample code demonstrates the different functions that can be applied to a Substitute (IPv6 Address) Rule object, such as add, search, modify, and remove. This sample also includes error handling for the operations.

#Preparation prior to a Substitute (IPv6 Address) Rule insertion

```
#PROGRAM STARTS: Include all the modules that will be used

use strict;
use Infoblox;

#Create a session to the Infoblox Appliance
my $session = Infoblox::Session->new(
    master   => "192.168.1.2", #appliance host ip
    username => "admin",       #appliance user login
    password => "infoblox"     #appliance password
);

unless ($session) {
   die("Construct session failed: ",
       Infoblox::status_code() . ":" . Infoblox::status_detail());
}

#Create the Response Policy zone prior to an RPZ Substitute (IPv6 Address) Rule insertion
```

```
my $zone = Infoblox::DNS::Zone->new(name => "rpz.com");

unless ($zone) {
   die("Construct zone failed: ",
       Infoblox::status_code() . ":" . Infoblox::status_detail());
}

print "Zone object created successfully\n";

#Verify if the zone exists
my $object = $session->get(object => "Infoblox::DNS::Zone",
                           name => "rpz.com");
unless ($object) {
   print "Zone does not exist on server, safe to add the zone\n";
   $session->add($zone)
      or die("Add zone failed: ",
             $session->status_code() . ":" . $session->status_detail());
}
print "Zone added successfully\n";

#Create the zone prior to a CNAME record insertion
my $zone = Infoblox::DNS::Zone->new(name => "rpz.com",
                                    rpz_policy => "GIVEN");
unless ($zone) {
die("Construct zone failed: ",
    Infoblox::status_code() . ":" . Infoblox::status_detail());
}
print "Zone object created successfully\n";

#Verify if the zone exists
my $object = $session->get(object => "Infoblox::DNS::Zone", name => "rpz.com");
unless ($object) {
print "Zone does not exist on server, safe to add the zone\n";
$session->add($zone)
   or die("Add zone failed: ",
          $session->status_code() . ":" . $session->status_detail());
}
```

#Create a Substitute (IPv6 Address) Rule

```
#Construct a Substitute (IPv6 Address) Rule object

my $rpz_4aip = Infoblox::DNS::RPZRecord::AAAAIpAddress->new(
 name     => "2001::/64.rpz.com",
 comment  => "add a Substitute (IPv6 Address) Rule aaaa1.rpz.com",
 ipv6addr => "aaaa:1111:bbbb:2222:cccc:3333:dddd:4444",
 rp_zone  => $zone,
);

unless ($rpz_4aip) {
die("Construct Substitute (IPv6 Address) Rule failed: ",
    Infoblox::status_code() . ":" . Infoblox::status_detail());
}
print "Substitute (IPv6 Address) Rule object created successfully\n";

#Add the Substitute (IPv6 Address) Rule object to Infoblox Appliance through a session
$session->add($rpz_4aip)

 or die("Add Substitute (IPv6 Address) Rule failed: ",
        $session->status_code() . ":" . $session->status_detail());
  print "Substitute (IPv6 Address) Rule object added to server successfully\n";
```

#Search for a specific Substitute (IPv6 Address) Rule

```
#Search all RPZ Substitute (IPv6 Address) Rules that match "rpz.com"
my @retrieved_objs = $session->search(
 object => "Infoblox::DNS::RPZRecord::AAAAIpAddress",
 name   => 'rpz\.com'
);
my $object = $retrieved_objs[0];
unless ($object) {
 die("Search Substitute (IPv6 Address) Rule failed: ",
     $session->status_code() . ":" . $session->status_detail());
}
print "Search Substitute (IPv6 Address) Rule object found at least 1 matching entry\n";

#Search all RPZ Substitute (IPv6 Address) Rules that start with "add" in the comment
```

```
my @retrieved_objs = $session->search(
 object    => "Infoblox::DNS::RPZRecord::AAAAIpAddress",
 comment   => "add.*",
);
my $object = $retrieved_objs[0];
unless ($object) {
 die("Search Substitute (IPv6 Address) Rule failed: ",
     $session->status_code() . ":" . $session->status_detail());
}
print "Search Substitute (IPv6 Address) Rule object found at least 1 matching entry\n";

#Search all RPZ Substitute (IPv6 Address) Rules that start with "aaaa1" and end with ".com"
my @retrieved_objs = $session->search(
 object => "Infoblox::DNS::RPZRecord::AAAAIpAddress",
 name   => '2001.*\.com',
);
my $object = $retrieved_objs[0];
unless ($object) {
 die("Search Substitute (IPv6 Address) Rule failed: ",
     $session->status_code() . ":" . $session->status_detail());
}
print "Search Substitute (IPv6 Address) Rule object using regexp found at least 1 matching entry\n";
```

#Get and modify a Substitute (IPv6 Address) Rule

```
#Get RPZ Substitute (IPv6 Address) Rule through the session
my @retrieved_objs = $session->get(
 object => "Infoblox::DNS::RPZRecord::AAAAIpAddress",
 name   => "2001::/64.rpz.com"
);
my $object = $retrieved_objs[0];

unless ($object) {
 die("Get Substitute (IPv6 Address) Rule failed: ",
     $session->status_code() . ":" . $session->status_detail());
}

#Modify one of the attributes of the specified RPZ Substitute (IPv6 Address) Rule
$object->ipv6addr("aaaa:1111:bbbb:2222:cccc:3333:dddd:eeee");

#Applying the changes
$session->modify($object)

 or die("Modify Substitute (IPv6 Address) Rule failed: ",
        $session->status_code() . ":" . $session->status_detail());
  print "Substitute (IPv6 Address) Rule object modified successfully \n";
```

#Remove a Substitute (IPv6 Address) Rule.

```
#Get RPZ Substitute (IPv6 Address) Rule through the session
my @retrieved_objs = $session->get(
 object   => "Infoblox::DNS::RPZRecord::AAAAIpAddress",
 name     => "2001::/64.rpz.com"
);
my $object = $retrieved_objs[0];
unless ($object) {
 die("Get Substitute (IPv6 Address) Rule failed: ",
     $session->status_code() . ":" . $session->status_detail());
}

print "Get Substitute (IPv6 Address) Rule object found at least 1 matching entry\n";

#Submit the object for removal
$session->remove($object)
 or die("Remove Substitute (IPv6 Address) Rule failed: ",
        $session->status_code() . ":" . $session->status_detail());
print "Substitute (IPv6 Address) Rule object removed successfully \n";

####PROGRAM ENDS####
```

AUTHOR

Infoblox Inc. http://www.infoblox.com/

SEE ALSO

Infoblox::DNS::View, Infoblox::DNS::Zone, Infoblox::Session, Infoblox::Session->get(), Infoblox::Session->search(), Infoblox::Session->add(), Infoblox::Session->remove(), Infoblox::Session->modify()

COPYRIGHT

Copyright (c) 2017 Infoblox Inc.

Infoblox::DNS::Ruleset::NxdomainRule - Rule of Ruleset object

- NAME
- DESCRIPTION
- CONSTRUCTOR
- MODULE METHODS
  - Infoblox::DNS::Ruleset->nxdomain_rules( )
- METHODS
  - pattern( )
  - action( )
- AUTHOR
- SEE ALSO
- COPYRIGHT

NAME

Infoblox::DNS::Ruleset::NxdomainRule - Rule of Ruleset object

DESCRIPTION

Represents a rule that is used to match domain names.

CONSTRUCTOR

```
my $rule = Infoblox::DNS::Ruleset::NxdomainRule->new(
             pattern => $string,                    #Required
             action => $string,                     #Required
);
```

MODULE METHODS

The following functions can be applied to an Infoblox::DNS::Ruleset::NxdomainRule object.

Infoblox::DNS::Ruleset->nxdomain_rules( )

Use this method to add a list of Infoblox::DNS::Ruleset::NxdomainRule objects to the Ruleset object of an Infoblox appliance. See Infoblox::DNS::Ruleset->`nxdomain_rules()` for parameters and return values.

Example

#Construct a rule my $rule1 = Infoblox::DNS::Ruleset::NxdomainRule->new(pattern => "www.a.zone.com", action => "PASS");

Configure a rule for the Ruleset object $ruleset->nxdomain_rules([$rule1]);

METHODS

This section describes all the methods you can use to configure and retrieve the attribute values of a Rule object.

pattern( )

Use this method to specify the pattern that is used to match domain names.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

The pattern in string format, with a maximum of 1024 characters.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get pattern
my $pattern = $ruleset_item->pattern();
#Modify pattern
$ruleset_item->pattern("^*$");
```

action( )

Use this method to specify the action to perform when a domain name matches the pattern defined in this Infoblox::DNS::Ruleset::NxdomainRule object.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

The valid value is "PASS", "REDIRECT" or "MODIFY".

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get action
my $action = $ruleset_item->action();
#Modify action
$ruleset_item->action("PASS");
```

AUTHOR

Infoblox Inc. http://www.infoblox.com/

SEE ALSO

Infoblox::Session, Infoblox::DNS::Ruleset

COPYRIGHT

Copyright (c) 2017 Infoblox Inc.

Infoblox::DNS::Ruleset - Ruleset object

- NAME
- DESCRIPTION
- CONSTRUCTOR
- SESSION METHODS
  - Infoblox::Session->add( )
  - Infoblox::Session->get( )
  - Infoblox::Session->modify( )
  - Infoblox::Session->remove( )
  - Infoblox::Session->search( )
- METHODS
  - name( )
  - type( )
  - comment( )
  - disabled( )
  - nxdomain_rules( )
- SAMPLE CODE
- AUTHOR
- SEE ALSO
- COPYRIGHT

NAME

Infoblox::DNS::Ruleset - Ruleset object

DESCRIPTION

Represents a Ruleset object, which is a collection of rules that is used to match domain names.

CONSTRUCTOR

See Infoblox::DNS::Ruleset::Rule for details.

```
 my $rule1 = Infoblox::DNS::Ruleset::NxdomainRule->new(
     pattern => $string,                                          #Optional
     action => $string,                                           #Optional
 );
 my $rule2 = Infoblox::DNS::Ruleset::NxdomainRule->new(
     pattern => $string,                                          #Optional
     action => $string,                                           #Optional
 );
 my $ruleset = Infoblox::DNS::Ruleset->new (
     name => $string,                                             #Required
     type => <"NXDOMAIN" | "BLACKLIST">                           #Required
     comment => $string,                                          #Optional
     disabled => "true" | "false",                                #Optional
     nxdomain_rules => [$rule1, $rule2]                           #Optional, valid only when
type is "NXDOMAIN"
 );
```

SESSION METHODS

This section describes all the methods in the Infoblox::Session module that you can apply to a Ruleset object.

Infoblox::Session->add( )

Use this method to add an object to the Infoblox appliance. See Infoblox::Session->add() for parameters and return values.

Example

```
#Create Rule 1 object
my $rule1 = Infoblox::DNS::Ruleset::NxdomainRule->new(
    pattern => "www.a.zone.com",
    action => 'PASS' );

#Create Rule 2 object
my $rule2 = Infoblox::DNS::Ruleset::NxdomainRule->new(
    pattern=> "www.b.zone.com",
    action => 'REDIRECT' );
```

```
my $ruleset = Infoblox::DNS::Ruleset->new (
   name => $ruleset_name,
   type => "NXDOMAIN",
   comment=> "This is a demo ruleset",
   disabled=> "false",
   nxdomain_rules=> [$rule1, $rule2]);

 #Submit for addition
 my $response = $session->add( $nsg1 );
```

Infoblox::Session->get( )

Use this method to retrieve all the matching objects from the Infoblox appliance. See Infoblox::Session->get() for parameters and return values.

Key References

```
Apply the following attributes to get a specific Ruleset object:

name - Required. The name of the ruleset in string format.
disabled - Optional. Indicates if the ruleset is disabled. The valid value is either "true" or
"false".
comment - Optional. The comment that was entered for the ruleset.
```

Examples

```
my @retrieved_objs = $session->get(
    object => "Infoblox::DNS::Ruleset",
    name   => "Ruleset 1",
    );

my @retrieved_objs = $session->get(
    object => "Infoblox::DNS::Ruleset",
    disabled   => "false",
    );

my @retrieved_objs = $session->get(
    object => "Infoblox::DNS::Ruleset",
    comment   => "Ruleset comment",
    );
```

Infoblox::Session->modify( )

Use this method to modify an object in the Infoblox appliance. See Infoblox::Session->modify() for parameters and return values.

Example

```
#Modify the comment
$ruleset->comment("this is a modified comment");
#Submit the modification
my $response = $session->modify( $ruleset );
```

Infoblox::Session->remove( )

Use this method to remove an object from the Infoblox appliance. See Infoblox::Session->remove() for parameters and return values.

To remove a specific object, first use `get()` or `search()` to retrieve the object, and then submit it for removal.

Example

#Get the objects with the same name my @retrieved_objs = $session->get(object => "Infoblox::DNS::Ruleset", name => "Ruleset 1",); #Find the desired object on the retrieved list my $ruleset= $retrieved_objs[0]; #Submit the object for removal my $response = $session->remove($ruleset);

Infoblox::Session->search( )

Use this method to search for Ruleset objects in the Infoblox appliance. See Infoblox::Session->search() for parameters and return values.

Key References

Apply the following attributes to search for a Ruleset object:

```
name - Required. The ruleset name in string format (regular expression).
disabled - Optional. Indicates if the ruleset is disabled. The valid value is either "true" or
"false".
comment - Optional. The comment that was entered for the ruleset (regular expression).
```

Examples

```
# Search for all Ruleset objects
my @retrieved_objs = $session->search(
    object => "Infoblox::DNS::Ruleset",
    name   => ".*",
    );
```

Search for all Ruleset objects with "false" as the value for the "disabled" attribute my @retrieved_objs = $session->get(object => "Infoblox::DNS::Ruleset", disabled => "false");

Search for all Ruleset objects with "Ruleset Comm" as the value for the "comment" attribute my @retrieved_objs = $session->get(object => "Infoblox::DNS::Ruleset", comment => "Ruleset Comm");

METHODS

This section describes all the methods that you can use to configure and retrieve the attribute values of a Ruleset object.

name( )

Use this method to set or retrieve the name of this Ruleset object.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

The ruleset name in string format, with a maximum of 256 characters.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get name
my $name = $ruleset->name();
#Modify name
$ruleset->name("rulset");
```

type( )

Use this method to set or retrieve the type of this Ruleset object.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

The ruleset type. Supported values are "NXDOMAIN" or "BLACKLIST".

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get type
my $type = $ruleset->type();
#Modify type
$ruleset->type("NXDOMAIN");
```

comment( )

Use this method to set or retrieve a descriptive comment about the Ruleset object.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

Comment in string format, with a maximum of 256 characters.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get comment
my $comment = $ruleset->comment();
#Modify comment
$ruleset->comment("This is Default Ruleset");
```

disabled( )

Use this method to set or retrieve the flag that indicates if the Ruleset object is disabled.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

Specify "true" to set the disabled flag or "false" to deactivate/unset it. The default value is "false".

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get disabled
my $disabled = $ruleset->disabled();
#Modify disabled
$ruleset->disabled("true");
```

nxdomain_rules( )

Use this method to set or retrieve a list of Infoblox::DNS::Ruleset::NxdomainRule objects assigned to this Ruleset object. Rules can be set only when the ruleset type is set to "NXDOMAIN".

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

The valid value is an array reference that contains a list of Infoblox::DNS::Ruleset::NxdomainRule objects.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Getting rules
my $rules = $ruleset->nxdomain_rules( );
#Modifying rules
$ruleset->nxdomain_rules([$rule1]);
```

SAMPLE CODE

The following sample code demonstrates the different functions that can be applied to an object, such as modify and remove. This sample also includes error handling for the operations.

#Preparation prior to a Ruleset object insertion

```
#PROGRAM STARTS: Include all the modules that will be used
use strict;
use Infoblox;

#Create a session to the Infoblox appliance
my $session = Infoblox::Session->new(
    master   => "192.168.1.2",
    username => "admin",
    password => "infoblox"
);
unless ($session) {
   die("Construct session failed: ",
       Infoblox::status_code() . ":" . Infoblox::status_detail());
}
```

```
print "Session created successfully\n";
```

#Create a Ruleset object

```
my $rule1 = Infoblox::DNS::Ruleset::NxdomainRule->new(
    pattern => "www.a.zone.com",
    action => 'PASS' );
unless ($rule1) {
   die("Construct Rule 1 failed: ",
       Infoblox::status_code() . ":" . Infoblox::status_detail());
}

my $rule2 = Infoblox::DNS::Ruleset::NxdomainRule->new(
    pattern=> "www.b.zone.com",
    action => 'REDIRECT' );
unless ($rule2) {
   die("Construct Rule 2 failed: ",
       Infoblox::status_code() . ":" . Infoblox::status_detail());
}

my $ruleset = Infoblox::DNS::Ruleset->new (
    name => "Ruleset 1",
    type => "NXDOMAIN",
    comment=> "This is demo ruleset",
    disabled=> "false",
    nxdomain_rules=> [$rule1, $rule2]);

unless ($ruleset) {
   die("Construct Ruleset failed: ",
       Infoblox::status_code() . ":" . Infoblox::status_detail());
}
print "Ruleset object created successfully\n";
```

#Add the Ruleset object

```
my $resp = $session->add($ruleset);
unless ($resp == 1) {
   die("Add Ruleset failed: ",
       Infoblox::status_code() . ":" . Infoblox::status_detail());
}
```

#Modify the Ruleset object

```
my @result_array = $session->get(
       object => "Infoblox::DNS::Ruleset",
       name => "Ruleset 1");
if( defined @result_array  and scalar( @result_array ) > 0 ){
       my $ruleset = $result_array[0];
       $ruleset->name("Ruleset 2");

       my $response = $session->modify( $ruleset );
       print "Modify the Ruleset object success.\n" if ( $response );
        }
```

#Remove a Ruleset object

```
my @result_array = $session->get( "object" => "Infoblox::DNS::Ruleset" ,  "name" => "Ruleset 2" );

if( defined @result_array  and scalar( @result_array ) > 0 ){
       my $ruleset = $result_array[0];
       if( ref( $ruleset ) eq "Infoblox::DNS::Ruleset" )
       {

               my $response = $session->remove($ruleset);
               print "Remove Ruleset success.\n" if ( $response );

       }

       unless ($ruleset) {
               die("Remove Ruleset failed: ",
                       $session->status_code() . ":" . $session->status_detail());
       }
        }

####PROGRAM ENDS####
```

AUTHOR

Infoblox Inc. http://www.infoblox.com/

SEE ALSO

Infoblox::Session, Infoblox::Session->get(),Infoblox::Session->remove(),Infoblox::Session->modify(),Infoblox::DNS,Infoblox::DNS::Ruleset::NxdomainRule

---

COPYRIGHT

Copyright (c) 2017 Infoblox Inc.

Infoblox::DNS::SRG - Create a shared record group.

- NAME
- DESCRIPTION
- CONSTRUCTOR
- SESSION METHODS
  - Infoblox::Session->add( )
  - Infoblox::Session->get( )
  - Infoblox::Session->modify( )
  - Infoblox::Session->remove( )
  - Infoblox::Session->search( )
- METHODS
  - comment( )
  - extattrs( )
  - extensible_attributes( )
  - name( )
  - override_record_name_policy( )
  - record_name_policy( )
  - zone_association( )
- SAMPLE CODE
- AUTHOR
- SEE ALSO
- COPYRIGHT

NAME

Infoblox::DNS::SRG - Create a shared record group.

DESCRIPTION

A shared record group (SRG) is created to contain DNS shared records and share them between different zones. For example, if a group of DNS records needs to be in three different zones, you can include the records in a shared record group and assign the group to the three zones. For more information about shared record groups and shared records, please refer to Infoblox Administrator Guide.

CONSTRUCTOR

```
 my $srg = Infoblox::DNS::SRG->new(
     name                          => $string,               #Required
     comment                       => $string,               #Optional / Default is undefined
     extattrs                      => { $string => $extattr, ... },      #Optional / Default is undefined
     extensible_attributes => { $string => $string|$num, $string => [ $string|$num, ...], ... }, #Optional
/ Default is undefined
     override_record_name_policy => "true" | "false",      #Optional / Default is "false"
     record_name_policy            => $string,               #Optional / Required when
override_record_name_policy is set to "true". Default is undefined
     zone_association              => [$Zone1, $Zone2, ...]  #Optional / Default is undefined
 );
```

You cannot set both extattrs and extensible_attributes attributes at the same time.

SESSION METHODS

This section describes all the methods in Infoblox::Session module that you can apply to a DNS SRG object.

Infoblox::Session->add( )

Use this method to add an SRG object to the Infoblox appliance. See Infoblox::Session->add() for parameters and return values.

Example

```
#Construct an object
my $srg = Infoblox::DNS::SRG->new (
    name     => "test_srg",
    comment  => "this is a demo SRG object",
    );
# Submit for addition
my $response = $session->add( $srg );
```

Infoblox::Session->get( )

Use this method to retrieve all the matching objects from the Infoblox appliance. See Infoblox::Session->get() for parameters and return values.

Key References

```
Apply the following attributes to get a specific DNS SRG object:

 name - Required. A SRG name in string format.
 extattrs     - Optional. A hash reference containing extensible attributes.
 extensible_attributes - Optional. A hash reference containing extensible attributes.
```

Example

```
my @retrieved_objs = $session->get(
    object => "Infoblox::DNS::SRG",
    name   => "test_srg");

# get all shared record group objects with a given extensible attribute
my @retrieved_objs = $session->get(
    object => "Infoblox::DNS::SRG",
    extensible_attributes => { 'Site' => 'Santa Clara' });
```

Infoblox::Session->modify( )

Use this method to modify an object in the Infoblox appliance. See Infoblox::Session->modify() for parameters and return values.

Example

```
# Use method to modify the comment.
$srg->comment("this is a modified comment");
# Submit modification
my $response = $session->modify( $srg );
```

Infoblox::Session->remove( )

Use this method to submit to remove an object from the Infoblox appliance. See Infoblox::Session->remove() for parameters and return values.

To remove a specific object, first use `get()` or `search()` to retrieve the specific object, and then submit this object for removal.

Example

```
# Get the objects with the same name
my @retrieved_objs = $session->get(
    object => "Infoblox::DNS::SRG",
    name   => "test_srg");
# Find the desired object from the retrieved list.
my $desired_srg = $retrieved_objs[0];
# Submit for removal
my $response = $session->remove( $desired_srg );
```

Infoblox::Session->search( )

Use this method to search for DNS SRG record objects in the Infoblox appliance. See Infoblox::Session->search() for parameters and return values.

Key References

```
Apply the following attributes to search for a DNS SRG record object:

 name - Required. A SRG name in string format (regular expression).
 comment - Optional. A comment in string format (regular expression).
 extattrs     - Optional. A hash reference containing extensible attributes.
 extensible_attributes - Optional. A hash reference containing extensible attributes.
```

For more information about searching extensible attributes, see Infoblox::Grid::ExtensibleAttributeDef/Searching Extensible Attributes.

Example

```
# search for all DNS SRG objects with name containing 'test'
my @retrieved_objs = $session->search(
    object => "Infoblox::DNS::SRG",
    name   => ".*test.*");

# search all DNS SRG objects with the extensible attribute 'Site'
my @retrieved_objs = $session->search(
   object => "Infoblox::DNS::SRG",
   extensible_attributes => { 'Site' => 'Santa Clara' });
```

METHODS

This section describes all the methods that you can use to configure and retrieve the attribute values of a DNS SRG object.

comment( )

Use this method to set or retrieve the descriptive comment of a DNS SRG object.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

Desired comment in string format with a maximum of 256 bytes.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get comment
my $comment = $srg->comment();
#Modify comment
$srg->comment("Modifying the DNS SRG comment");
```

extattrs( )

Use this method to set or retrieve the extensible attributes associated with a DNS SRG object.

Parameter

Valid value is a hash reference containing the names of extensible attributes and their associated values (Infoblox::Grid::Extattr objects).

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get extattrs
my $ref_extattrs = $srg->extattrs();
#Modify extattrs
$srg->extattrs({ 'Site' => $extattr1, 'Administrator' => $extattr2 });
```

extensible_attributes( )

Use this method to set or retrieve the extensible attributes associated with a DNS SRG object.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

For valid values for extensible attributes, see Infoblox::Grid::ExtensibleAttributeDef/Extensible Attribute Values.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get extensible attributes
my $ref_extensible_attributes = $srg->extensible_attributes();
#Modify extensible attributes
$srg->extensible_attributes({ 'Site' => 'Santa Clara', 'Administrator' => [ 'Peter', 'Tom' ] });
```

name( )

Use this method to set or retrieve the shared record group name of a DNS SRG object.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

The name of shared record group.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get name
my $name = $srg->name();
#Modify name
$srg->name("test_srg");
```

override_record_name_policy( )

Use this method to set or retrieve the override record name policy flag of a DNS SRG object.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

The default value for this field is false, which means this SRG will use default record name policy setting defined at grid level.

Parameter

Specify "true" to set the override_record_name_policy flag or "false" to deactivate/unset it.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get override_record_name_policy
my $override = $srg->doverride_record_name_policy();
#Modify override_record_name_policy
$srg->override_record_name_policy("true");
```

record_name_policy( )

Use this method to set or retrieve the record name policy of a DNS SRG object.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

The name of the record name policy defined at the grid level. See Infoblox::Grid::DNS for more information about record name policies.

This property is required only if override_record_name_policy is set to "true", which means SRG object will use its own record name policy.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get record name policy
my $policy = $srg->record_name_policy();
#Modify record name policy
$srg->record_name_policy("Allow Any");
```

zone_association( )

Use this method to set or retrieve the zone association of a DNS SRG object. All zone objects in a association list are associated with the SRG object, which means that all shared records in this SRG will be shared among these zones. For more information about SRG and zone association, please refer to Infoblox Administrator's Guide.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

Array reference of defined Infoblox::DNS::Zone objects.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get zone association
my $zone_associations = $srg->zone_association();
#Modify zone associations, list of Infoblox::DNS::Zone objects
$srg->zone_association([$zone1,$zone2,$zone3]);
```

SAMPLE CODE

The following sample code demonstrates the different functions that can be applied to an object such as add, search, modify, and remove. This sample also includes error handling for the operations.

#Preparation prior to a DNS SRG object insertion

```
#PROGRAM STARTS: Include all the modules that will be used
use strict;
use Infoblox;

#Create a session to the Infoblox Appliance
my $session = Infoblox::Session->new(
    master   => "192.168.1.2",
    username => "admin",
    password => "infoblox"
);
unless ($session) {
   die("Construct session failed: ",
       $session->status_code() . ":" . $session->status_detail());
}
print "Session created successfully\n";

#Create the zone prior to an SRG object insertion
my $zone = Infoblox::DNS::Zone->new(name => "domain.com");
unless ($zone) {
   die("Construct zone failed: ",
       Infoblox::status_code() . ":" . Infoblox::status_detail());
}
print "Zone object created successfully\n";

#Verify if the zone exists
my $object = $session->get(object => "Infoblox::DNS::Zone", name => "domain.com");
unless ($object) {
   print "Zone does not exist on server, safe to add the zone\n";
   $session->add($zone)
      or die("Add zone failed: ",
             $session->status_code() . ":" . $session->status_detail());
}
print "Zone added successfully\n";
```

#Create a DNS SRG object

```
#Construct a DNS SRG object
my $srg = Infoblox::DNS::SRG->new(
    name     => "test_srg",
    comment  => "this is a demo srg object",
    zone_association=> [ $zone ],
    override_record_name_policy => "true",
    record_name_policy=>"Allow Underscore"
);
unless ($srg) {
   die("Construct DNS SRG failed: ",
       Infoblox::status_code() . ":" . Infoblox::status_detail());
}
print "DNS SRG object created successfully\n";

#Add the DNS SRG record object to Infoblox Appliance through a session
$session->add($srg)
    or die("Add SRG object failed: ",
           $session->status_code() . ":" . $session->status_detail());
```

```
print "DNS SRG object added to server successfully\n";
```

#Search for a specific DNS SRG record

```
#Search all SRG records with name containing 'test'
my @retrieved_objs = $session->search(
    object => "Infoblox::DNS::SRG",
    name   => ".*test.*"
);
my $object = $retrieved_objs[0];
unless ($object) {
    die("Search SRG failed: ",
        $session->status_code() . ":" . $session->status_detail());
}
print "Search DNS SRG object found at least 1 matching entry\n";
```

#Get and modify a DNS SRG record

```
#Get SRG object through the session
my @retrieved_objs = $session->get(
    object => "Infoblox::DNS::SRG",
    name   => "test_srg"
);
my $object = $retrieved_objs[0];
unless ($object) {
    die("Get SRG failed: ",
        $session->status_code() . ":" . $session->status_detail());
}
print "Get DNS SRG object found at least 1 matching entry\n";

#Modify one of the attributes of the specified SRG object
$object->comment("Updated comment.");

#Apply the changes
$session->modify($object)
    or die("Modify SRG object failed: ",
           $session->status_code() . ":" . $session->status_detail());
print "DNS SRG object modified successfully \n";
```

#Remove a DNS SRG object

```
#Get SRG object through the session
my @retrieved_objs = $session->get(
    object => "Infoblox::DNS::SRG",
    name   => "test_srg"
);
my $object = $retrieved_objs[0];
unless ($object) {
    die("Get SRG object failed: ",
        $session->status_code() . ":" . $session->status_detail());
}
print "Get SRG object found at least 1 matching entry\n";

#Submit the object for removal
$session->remove($object)
    or die("Remove SRG object failed: ",
        $session->status_code() . ":" . $session->status_detail());
print "DNS SRG object removed successfully \n";

####PROGRAM ENDS####
```

AUTHOR

Infoblox Inc. http://www.infoblox.com/

SEE ALSO

Infoblox::DNS::Zone, Infoblox::Grid::DNS, Infoblox::Grid::RecordNamePolicy, Infoblox::Session, Infoblox::Session->get(), Infoblox::Session->search(), Infoblox::Session->add(), Infoblox::Session->remove(), Infoblox::Session->modify()

COPYRIGHT

Copyright (c) 2017 Infoblox Inc.

Infoblox::Grid::RecordNamePolicy - Manages the DNS record name policies defined at the grid level.

- NAME
- DESCRIPTION
- CONSTRUCTOR
- MODULE METHODS
  - Infoblox::Grid::DNS->record_name_policies( )
- METHODS
  - name( )
  - regex( )
- SAMPLE CODE
- AUTHOR
- SEE ALSO
- COPYRIGHT

NAME

Infoblox::Grid::RecordNamePolicy - Manages the DNS record name policies defined at the grid level.

DESCRIPTION

Record name policy defines the DNS record name format restrictions at the grid level.

CONSTRUCTOR

```
my $policy = Infoblox::Grid::RecordNamePolicy->new(
             name  => $string, #Required
             regex => $regex   #Required
);
```

MODULE METHODS

The following functions are available to apply to the record name policy object.

Infoblox::Grid::DNS->record_name_policies( )

Use this function to specify a record name policy at the grid level in the Infoblox appliance.

Example

```
#Create a RecordNamePolicy object.
my $policy = Infoblox::Grid::RecordNamePolicy->new(
                 name  => "test",
                 regex => "^[a-zA-Z0-9]+\$",
               );

#Configure record name policy on the Infoblox grid object
my $response = $grid_dns->record_name_policies([$policy]);
```

METHODS

This section describes all the methods that can be used to configure and retrieve the attributes values of a record name policy object.

name( )

Use this method to set or retrieve the record name policy name.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

String with the name of the record name policy.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get name
my $name = $policy->name();
#Modify name
$policy->name("new_name");
```

regex( )

Use this method to set or retrieve the record name policy in regular expression format.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

The record name policy in regular expression format. It should follow POSIX regular expression rules.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get regex
my $regex = $policy->regex();
#Modify regex
$policy->regex("^[a-zA-Z0-9]*\$");
```

SAMPLE CODE

The following sample code demonstrates different operations that can be applied to a record name policy object, such as create, modify, and remove. This sample code also includes error handling for the operations.

```
#PROGRAM STARTS: Include all the modules that will be used
use strict;
use Infoblox;

#Create a session to the Infoblox appliance
my $session = Infoblox::Session->new(
    master   => "192.168.1.2",
    username => "admin",
    password => "infoblox"
);
unless ($session) {
   die("Construct session failed: ",
       Infoblox::status_code() . ":" . Infoblox::status_detail());
}
print "Session created successfully\n";
```

#Create a record name policy object

```
my $policy1 = Infoblox::Grid::RecordNamePolicy->new(
    name  => "Allow Any",
    regex => ".+",
    );
unless ($policy1) {
     die("Construct record name policy1 object failed: ",
           Infoblox::status_code() . ":" . Infoblox::status_detail());
}
print "Record name policy1 object created successfully\n";

my $policy2 = Infoblox::Grid::RecordNamePolicy->new(
      name  => "Allow Underscore",
      regex => "^[-a-zA-Z0-9_.]+\$",
    );

unless ($policy2) {
     die("Construct record name policy1 object failed: ",
           Infoblox::status_code() . ":" . Infoblox::status_detail());
}
print "Record name policy2 object created successfully\n";

my $policy3 = Infoblox::Grid::RecordNamePolicy->new(
      name  => "Strict Hostname Checking",
```

```
      regex => "^[a-zA-Z0-9]\$|^[a-zA-Z0-9][-a-zA-Z0-9.]*[a-zA-Z0-9]\$",
    );

unless ($policy3) {
     die("Construct record name policy3 object failed: ",
           Infoblox::status_code() . ":" . Infoblox::status_detail());
}
print "Record name policy3 object created successfully\n";

my $policy4 = Infoblox::Grid::RecordNamePolicy->new(
    name  => "test",
    regex => "^[a-zA-Z0-9]+\$",
    );
unless ($policy4) {
     die("Construct record name policy3 object failed: ",
           Infoblox::status_code() . ":" . Infoblox::status_detail());
}
print "Record name policy4 object created successfully\n";
```

#Get the Member DNS object to add a record name policy object to it

```
my $grid_dns = $session->get( "object" => "Infoblox::Grid::DNS", "grid" => "Infoblox");

unless ($grid_dns) {
         die("Get Grid DNS failed: ",
             $session->status_code() . ":" . $session->status_detail());
 }
$grid_dns->record_name_policies([$policy1,$policy2,$policy3,$policy4]);
$session->modify($grid_dns)
      or die("modify Grid DNS failed:" ,
      $session->status_code(), $session->status_detail());
print "Modify grid DNS successful for record name policy object\n";

####PROGRAM ENDS####
```

AUTHOR

Infoblox Inc. http://www.infoblox.com/

SEE ALSO

Infoblox::Grid::DNS, Infoblox::Session, Infoblox::Session->get(), Infoblox::Session->modify()

COPYRIGHT

Copyright (c) 2017 Infoblox Inc.

Infoblox::DNS::RootNameServer - Custom root name server object.

- NAME
- DESCRIPTION
- CONSTRUCTOR
- MODULE METHODS
  - Infoblox::DNS::View->custom_root_name_servers( )
  - Infoblox::Grid::Member::DNS->custom_root_name_servers( )
- METHODS
  - host_name( )
  - ipv4addr( )
  - ipv6addr( )
- SAMPLE CODE
- AUTHOR
- SEE ALSO
- COPYRIGHT

NAME

Infoblox::DNS::RootNameServer - Custom root name server object.

DESCRIPTION

The custom root name server provides host name and IP address information of the internal root name server. When the Infoblox appliance receives a recursive query for DNS data that it does not have, it uses the internal root name server specified instead of using the Internet root name server.

CONSTRUCTOR

```
my $rootns = Infoblox::DNS::RootNameServer->new(
             host_name => $fqdn,      #Required
             ipv4addr  => $ipv4addr   #Required if $ipv6addr is not present
             ipv6addr  => $ipv4addr   #Required if $ipv4addr is not present
);
```

MODULE METHODS

You can apply the following functions to a custom root name server object:

Infoblox::DNS::View->custom_root_name_servers( )

Use this function to specify custom root name servers at the view level on the Infoblox appliance. See Infoblox::DNS::View->custom_root_name_servers() for parameters and return values.

Example

```
#Construct a custom root name server object
my $rootns1 = Infoblox::DNS::RootNameServer->new(
             host_name => "rns1.test.com",
             ipv4addr  => "4.4.4.4"
);

#Construct a custom root name server object
my $rootns2 = Infoblox::DNS::RootNameServer->new(
             host_name => "rns3.test.com",
             ipv6addr  => "6666::"
);

# Configure custom root name server on the View object
my $response = $View->custom_root_name_servers([$rootns1, $rootns2]);
```

Infoblox::Grid::Member::DNS->custom_root_name_servers( )

Use this function to specify custom root name servers at the member level on the Infoblox appliance. See Infoblox::Grid::Member::DNS->custom_root_name_servers() for parameters and return values.

Example

```
#Construct a custom root name server object
my $rootns1 = Infoblox::DNS::RootNameServer->new(
```

```
                host_name => "rns1.test.com",
                ipv4addr  => "4.4.4.4"
);

#Construct a custom root name server object
my $rootns2 = Infoblox::DNS::RootNameServer->new(
                host_name => "rns3.test.com",
                ipv6addr  => "6666::"
);

# Configure custom root name server on the member DNS object
my $response = $Member_DNS->custom_root_name_servers([$rootns1, $rootns2]);
```

METHODS

This section describes all the methods you can use to configure and retrieve the attribute values of a custom root name server.

host_name( )

Use this method to set or retrieve the host name of the custom root name server.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

Specify the host name in the FQDN (Fully Qualified Domain Name) format. The FQDN consists of the host name followed by the domain name (example: abc.com). A host name can have a maximum of 256 characters.

Returns

If you specify a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you do not specify a parameter, the method returns the attribute value.

Example

```
#Get host name
my $host_name = $rootns->host_name();
#Modify host name
$rootns->host_name("qa.infoblox.com");
```

ipv4addr( )

Use this method to set or retrieve the IPv4 address of the custom root name server. It is required only when IPv6 is not specified.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

An IPv4 address is a 32-bit number in dotted decimal notation. It consists of four 8-bit groups of decimal digits separated by decimal points (example: 192. 168.1.2).

Returns

If you specify a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you do not specify a parameter, the method returns the attribute value.

Example

```
#Get ipv4addr
my $ipv4addr = $rootns->ipv4addr();
#Modify ipv4addr
$rootns->ipv4addr("2.2.2.2");
```

ipv6addr( )

Use this method to set or retrieve the IPv6 address of the custom root name server. It is required only when IPv4 is not specified.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

An IPv6 address is a 128-bit number in colon hexadecimal notation. It consists of eight groups of four hexadecimal digits separated by colons (example: 12ab:0000:0000:0123:4567:89ab:0000:cdef).

Returns

If you specify a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you do not specify a parameter, the method returns the attribute value.

Example

```
# Getting ipv6addr
my $ipv6addr = $rootns->ipv6addr();
# Modifying ipv6addr
$rootns->ipv6addr("12ab::345");
```

SAMPLE CODE

The following sample code demonstrates the different functions that can be applied to a custom root name server object such as add, modify, and remove. Also, this sample includes error handling for the operations.

#Preparation prior to a custom root name server object insertion

```
#PROGRAM STARTS: Include all the modules that will be used
use strict;
use Infoblox;

#Create a session to the Infoblox appliance
my $session = Infoblox::Session->new(
               master   => "192.168.1.2", #appliance host ip
               username => "admin",       #appliance user login
               password => "infoblox"     #appliance password
);
unless ($session) {
       die("Construct session failed: ",
            Infoblox::status_code() . ":" . Infoblox::status_detail());
}
print "Session created successfully\n";
```

#Create a custom root name server object

```
my $rootns = Infoblox::DNS::RootNameServer->new(
             host_name => "rns1.test.com",
             ipv4addr  => "4.4.4.4"
);
unless($rootns) {
       die("Construct custom root name server object failed: ",
            Infoblox::status_code() . ":" . Infoblox::status_detail());
}
print "Root name server object created successfully\n";
```

#Get the Member DNS object and add custom root name server to it

```
#Get the Member DNS object
my @retrieved_objs = $session->get(
    object => "Infoblox::Grid::Member::DNS",
    name   => "infoblox.localdomain"
);
my $object = $retrieved_objs[0];
unless ($object) {
       die("Get Member DNS object failed: ",
            $session->status_code() . ":" . $session->status_detail());
}
print "Get Member DNS object found at least 1 matching entry\n";

#Apply the changes to the Grid DNS object
$object->custom_root_name_servers([$rootns]);

#Apply the changes
$session->modify($object)
    or die("Modify Member DNS object failed: ",
            $session->status_code() . ":" . $session->status_detail());
print "Member DNS object with custom root name server updated to Infoblox appliance successfully\n";
```

#Modify custom root name server

```
#Modify existing custom root name server object

#Modifying the host name of the custom root name server object.
$rootns->host_name("eng.infoblox.com");

#Modifying the IPV4 address of the custom root name server object.
```

```
$rootns->ipv4addr("3.3.3.3");

#Apply changes to the Member DNS object.
$object->custom_root_name_servers([$rootns]);

#Update Member DNS object through the Infoblox session.
$session->modify($object)
            or die("modify Member DNS object failed: ",
                $session->status_code() . ":" . $session->status_detail());
       print "Member DNS object with modified custom root name server updated to Infoblox appliance
successfully\n";
```

#Remove custom root name server

```
#Apply changes to the Member DNS object to use default internet root name server.
$object->custom_root_name_servers(undef);

#Update Member DNS object through the Infoblox session.
$session->modify($object)
            or die("Remove custom root name server from Member DNS object failed: ",
                $session->status_code() . ":" . $session->status_detail());
       print "Removed custom root name server from Member DNS object successfully\n";

####PROGRAM ENDS####
```

AUTHOR

Infoblox Inc. http://www.infoblox.com/

SEE ALSO

Infoblox::Session, Infoblox::Grid::Member::DNS, Infoblox::DNS::View

---

COPYRIGHT

Copyright (c) 2017 Infoblox Inc.

Infoblox::Grid::DNS::DNS64Group - DNS64 Synthesis Group object

- NAME
- DESCRIPTION
- CONSTRUCTOR
- SESSION METHODS
  - Infoblox::Session->add( )
  - Infoblox::Session->get( )
  - Infoblox::Session->modify( )
  - Infoblox::Session->remove( )
  - Infoblox::Session->search( )
- METHODS
  - comment( )
  - clients( )
  - disable( )
  - dnssec_dns64_enabled( )
  - exclude( )
  - extattrs( )
  - extensible_attributes( )
  - name( )
  - mapped( )
  - prefix( )
- SAMPLE CODE
- AUTHOR
- SEE ALSO
- COPYRIGHT

NAME

Infoblox::Grid::DNS::DNS64Group - DNS64 Synthesis Group object

---

DESCRIPTION

A DNS64 synthesis group is used to configure the DNS64 feature.

CONSTRUCTOR

```
 my $group = Infoblox::Grid::DNS::DNS64Group->new(
     name                  => $string,
#Required
     prefix                => $ipv6prefix,
#Required
     clients               => [ $ipv4addr | $ipv4_subnet/prefix | $ipv6addr | $ipv6_subnet/prefix ,... ] |
$nacl, #Optional / Default is an empty list
     comment               => $string,
#Optional
     disable               => "true" | "false"
#Optional / Default is "false"
     dnssec_dns64_enabled  => "true" | "false"
#Optional / Default is "false"
     exclude               => [ $ipv4addr | $ipv4_subnet/prefix | $ipv6addr | $ipv6_subnet/prefix ,... ] |
$nacl, #Optional / Default is an empty list
     extattrs              => { $string => $extattr, ... },
#Optional / Default is undefined
     extensible_attributes => { $string => $string|$num, $string => [ $string|$num, ...], ... },
#Optional / Default is undefined
     mapped                => [ $ipv4addr | $ipv4_subnet/prefix | $ipv6addr | $ipv6_subnet/prefix ,... ] |
$nacl, #Optional / Default is an empty list
 );
```

You cannot set both extattrs and extensible_attributes attributes at the same time.

SESSION METHODS

This section describes all the methods in the Infoblox::Session module that you can apply to a DNS64 synthesis group object.

Infoblox::Session->add( )

Use this method to add an object to the Infoblox appliance. See Infoblox::Session->add() for parameters and return values.

If you have applied an access control list to an object, you must restart services after you update the access control list for the changes to

take effect. See Infoblox::Session->restart() method.

Example

```
#Construct an object
my $group = Infoblox::Grid::DNS::DNS64Group->new(
    name        => "dns64group_A",
    prefix      => "64:FF9B::/96",
    );

#Submit for addition
my $response = $session->add( $group );
```

Infoblox::Session->get( )

Use this method to retrieve all the matching objects from the Infoblox appliance. See Infoblox::Session->get() for parameters and return values.

Key References

```
Apply the following attributes to get a specific DNS64 synthesis group object:

 name                - Optional. Group name in string format.
 prefix              - Optional. An IPv6 prefix.
 extattrs            - Optional. A hash reference containing extensible attributes.
 extensible_attributes - Optional. A hash reference containing extensible attributes.
 comment             - Optional. User-specified comment for the group.
```

Examples

```
my @retrieved_objs = $session->get(
    object => "Infoblox::Grid::DNS::DNS64Group",
    name   => "dns64group_A",
    );

my @retrieved_objs = $session->get(
    object => "Infoblox::Grid::DNS::DNS64Group",
    extensible_attributes => { 'Site' => 'Santa Clara' });
```

Infoblox::Session->modify( )

Use this method to modify an object in the Infoblox appliance. See Infoblox::Session->modify() for parameters and return values.

If you have applied an access control list to an object, you must restart services after you update the access control list for the changes to take effect. See Infoblox::Session->restart() method.

Example

```
#Use this method to modify the comment.
$group->comment("this is a modified comment");
#Submit modification
my $response = $session->modify( $group );
```

Infoblox::Session->remove( )

Use this method to remove an object from the Infoblox appliance. See Infoblox::Session->remove() for parameters and return values.

To remove a specific object, first use `get()` or `search()` to retrieve the specific object, and then submit this object for removal.

Example

```
#Get the objects with the same name
my @retrieved_objs = $session->get(
    object => "Infoblox::Grid::DNS::DNS64Group",
    name   => "dns64group_A",
     );
#Find the desired object on the retrieved list
my $desired_nsgroup= $retrieved_objs[0];
#Submit for removal
my $response = $session->remove( $desired_nsgroup );
```

Infoblox::Session->search( )

Use this method to search for DNS64 synthesis group objects in the Infoblox appliance. See Infoblox::Session->search() for parameters and return values.

Key References

```
Apply the following attributes to search for a DNS64 synthesis group object:

 name                  - Optional. Group name in string format. (regular expression)
 prefix                - Optional. An IPv6 prefix. (regular expression)
 extattrs              - Optional. A hash reference containing extensible attributes.
 extensible_attributes - Optional. A hash reference containing extensible attributes.
 comment               - Optional. User-specified comment for the group. (regular expression)
```

For more information about searching extensible attributes, see Infoblox::Grid::ExtensibleAttributeDef/Searching Extensible Attributes.

Examples

```
# Search for all DNS64 synthesis group objects
my @retrieved_objs = $session->search(
    object => "Infoblox::Grid::DNS::DNS64Group",
    name   => ".*",
    );

# Search for all DNS64 synthesis group objects with "Santa Clara" as the value for the "Site"
extensible attribute
my @retrieved_objs = $session->get(
    object => "Infoblox::Grid::DNS::DNS64Group",
    extensible_attributes => { 'Site' => 'Santa Clara' });
```

METHODS

This section describes all the methods that you can use to configure and retrieve the attribute values of a DNS64 synthesis group object.

comment( )

Use this method to set or retrieve the comment field of the DNS64 synthesis group object.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

Comment of the group in string format.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Getting comment
my $comment = $group->comment();
#Modifying comment
$group->comment("Some comment");
```

clients( )

Use this method to set or retrieve the clients ACL list of a DNS64 synthesis group object. This is the list of clients for which synthesized records are returned. The default is "any".

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

The valid value is an Infoblox::Grid::NamedACL object or an array reference that contains IPv4/IPv6 addresses and networks. To exclude addresses and networks, add "!" as a prefix to the IP addresses and networks. The default value is an empty list.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Getting clients
my $clients = $group->clients();
#Modifying clients
$group->clients(["2001::333/64", "!2001::1234", "20.0.0.0/8"]);
#Use Infoblox::Grid::NamedACL
$group->clients($nacl);
```

disable( )

Use this method to set or retrieve the disable flag of a DNS64 synthesis group object.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

Specify "true" to set the disable flag or "false" to deactivate/unset it. The default value is "false".

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get disable
my $disable = $fixed_addr->disable();
#Modify disable
$fixed_addr->disable("true");
```

dnssec_dns64_enabled( )

Use this method to set or retrieve the dnssec_dns64_enabled flag of a DNS64 synthesis group object.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

Specify "true" to set the dnssec_dns64_enabled flag or "false" to deactivate/unset it. The default value is "false".

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get dnssec_dns64_enabled
my $dnssec_dns64_enabled = $group->dnssec_dns64_enabled();
#Modify dnssec_dns64_enabled
$group->dnssec_dns64_enabled('true');
```

exclude( )

Use this method to set or retrieve the exclude ACL list of a DNS64 synthesis group object. AAAA records which match this ACL are treated as non-existent for the purposes of synthesis. By default, the ACL is empty and no records match.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

The valid value is an Infoblox::Grid::NamedACL object or an array reference that contains IPv4/IPv6 addresses and networks. To exclude addresses and networks, add "!" as a prefix to the IP addresses and networks. The default value is an empty list.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Getting exclude
my $exclude = $group->exclude();
#Modifying exclude
$group->exclude(["2001::333/64", "!2001::1234"]);
#Use Infoblox::Grid::NamedACL
$group->exclude($nacl);
```

extattrs( )

Use this method to set or retrieve the extensible attributes associated with a DNS64 synthesis group object.

Parameter

Valid value is a hash reference containing the names of extensible attributes and their associated values (Infoblox::Grid::Extattr objects).

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get extattrs
my $ref_extattrs = $group->extattrs();
#Modify extattrs
$group->extattrs({ 'Site' => $extattr1, 'Administrator' => $extattr2 });
```

extensible_attributes( )

Use this method to set or retrieve the extensible attributes associated with a DNS64 synthesis group object.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

For valid values for extensible attributes, see Infoblox::Grid::ExtensibleAttributeDef/Extensible Attribute Values.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get extensible attributes
my $ref_extensible_attributes = $group->extensible_attributes();
#Modify extensible attributes
$group->extensible_attributes({ 'Site' => 'Santa Clara', 'Administrator' => [ 'Peter', 'Tom' ] });
```

name( )

Use this method to set or retrieve the DNS64 synthesis group name.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

Name of the group in string format.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Getting name
my $name = $group->name();
#Modifying name
$group->name("ns_group_2");
```

mapped( )

Use this method to set or retrieve the mapped ACL list of a DNS64 synthesis group object. The appliance generates AAAA records only for A records that match this ACL. If the ACL is empty, as it is by default, then all records will match.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

The valid value is an Infoblox::Grid::NamedACL object or an array reference that contains IPv4/IPv6 addresses and networks. To deny addresses and networks, add "!" as a prefix to the IP addresses and networks. The default value is an empty list.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Getting mapped
my $mapped = $group->mapped();
#Modifying mapped
$group->mapped(["40.10.0.0/24", "!40.10.3.4]);
#Use Infoblox::Grid::NamedACL
$group->mapped($nacl);
```

prefix( )

Use this method to set or retrieve the DNS64 synthesis group prefix. The prefix is an IPv6 subnet prefix with a length of /32, /40, /48, /56, /64, or /96.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

Prefix of the group in string format.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Getting prefix
my $prefix = $group->prefix();
#Modifying prefix
$group->prefix("64:FF9B::/96");
```

SAMPLE CODE

The following sample code demonstrates the different functions that can be applied to an object, such as modify and remove. This sample also includes error handling for the operations.

#Preparation prior to the addition of a DNS64 synthesis group object

```
#PROGRAM STARTS: Include all the modules that will be used
use strict;
use Infoblox;

#Create a session to the Infoblox device
my $session = Infoblox::Session->new(
                                     master   => "192.168.1.2",
                                     username => "admin",
                                     password => "infoblox"
                                    );
unless ($session) {
    die("Construct session failed: ",
        Infoblox::status_code() . ":" . Infoblox::status_detail());
}
print "Session created successfully\n";
```

#Create a DNS64 synthesis group object and add it to the appliance

```
my $group = Infoblox::Grid::DNS::DNS64Group->new(
                                                 name => 'testgroup',
                                                 prefix => '64:FF9B::/96',
                                                 clients => ["2001::333/64", "!2001::1234", "20.0.0.0/8"],
                                                 comment => 'some comment',
                                                );

unless ($group) {
    die("Construct DNS64 synthesis group failed: ",
        Infoblox::status_code() . ":" . Infoblox::status_detail());
}
print "DNS64 synthesis group object created successfully\n";

my $response = $session->add( $group );
unless ($response) {
    die("Adding DNS64 synthesis group failed: ",
        Infoblox::status_code() . ":" . Infoblox::status_detail());
```

```
}
print "Add DNS64 synthesis group success.\n" if ( $response );
```

#Add the DNS64 synthesis group object to the grid and enable it

```
my @result_array = $session->get( "object" => "Infoblox::Grid::DNS" ,  "grid" => "Infoblox" );

if ( defined @result_array  and scalar( @result_array ) > 0 ) {
    my $object = $result_array[0];
    if ( ref( $object ) eq "Infoblox::Grid::DNS" ) {
        $object->dns64_groups( [ $group ] );
        $object->enable_dns64("true");

        $response = $session->modify( $object );
        print "Add DNS64 synthesis group to grid dns properties success.\n" if ( $response );
             }

    unless ($object) {
        die("Add DNS64 synthesis group to grid dns properties failed: ",
            $session->status_code() . ":" . $session->status_detail());
    }
     }
```

#Modify the DNS64 synthesis group object

```
@result_array = $session->get( "object" => "Infoblox::Grid::DNS::DNS64Group" ,  name => "testgroup" );

if ( defined @result_array  and scalar( @result_array ) > 0 ) {
    my $object = $result_array[0];
    if ( ref( $object ) eq "Infoblox::Grid::DNS::DNS64Group" ) {

        $object->clients(["2001::333/64", "!2001::1234"]);

        $response = $session->modify( $object );
        print "Modify DNS64 synthesis group success.\n" if ( $response );
             }

    unless ($object) {
        die("Modify Group failed: ",
            $session->status_code() . ":" . $session->status_detail());
    }
     }
```

#Remove a DNS64 synthesis group object

#First let's remove it from the grid configuration

```
@result_array = $session->get( "object" => "Infoblox::Grid::DNS" ,  "grid" => "Infoblox" );

if ( defined @result_array  and scalar( @result_array ) > 0 ) {
    my $object = $result_array[0];
    if ( ref( $object ) eq "Infoblox::Grid::DNS" ) {

        $object->dns64_groups( [ ] );
        $object->enable_dns64( "false" );

        $response = $session->modify( $object );
        print "Remove DNS64 synthesis group from the grid success.\n" if ( $response );
             }

    unless ($object) {
        die("Remove DNS64 synthesis group failed: ",
            $session->status_code() . ":" . $session->status_detail());
    }
     }
```

#And now remove the group

```
@result_array = $session->get( "object" => "Infoblox::Grid::DNS::DNS64Group" ,  name => "testgroup" );

if ( defined @result_array  and scalar( @result_array ) > 0 ) {
    my $object = $result_array[0];
    if ( ref( $object ) eq "Infoblox::Grid::DNS::DNS64Group" ) {

        $response = $session->remove( $object );
        print "Remove DNS64 synthesis group success.\n" if ( $response );
```

```
            }

    unless ($object) {
        die("Remove DNS64 synthesis group failed: ",
            $session->status_code() . ":" . $session->status_detail());
    }
     }

 ####PROGRAM ENDS####
```

AUTHOR

Infoblox Inc. http://www.infoblox.com/

SEE ALSO

Infoblox::Session,Infoblox::Session->get(),Infoblox::Session->modify(),Infoblox::Session->search(),Infoblox::Session->remove()Infoblox::Grid::DNS, Infoblox::Grid::Member::DNS, Infoblox::DNS::View, Infoblox::Grid::NamedACL

COPYRIGHT

Copyright (c) 2017 Infoblox Inc.

Infoblox::Grid::NamedACL - Named ACL object.

- NAME
- DESCRIPTION
- CONSTRUCTOR
- SESSION METHODS
  - Infoblox::Session->get( )
  - Infoblox::Session->modify( )
  - Infoblox::Session->remove( )
  - Infoblox::Session->search( )
- MODULE METHODS
- METHODS
  - access_list( )
  - comment( )
  - exploded_access_list( )
  - extattrs( )
  - extensible_attributes( )
  - name( )
- SAMPLE CODE
- AUTHOR
- SEE ALSO
- COPYRIGHT

NAME

Infoblox::Grid::NamedACL - Named ACL object.

DESCRIPTION

Named ACL object represents a named list of access control entries.

CONSTRUCTOR

```
 my $nacl = Infoblox::Grid::NamedACL->new(
    'name'                  => $string,
# Required
    'comment'               => $string,
# Optional
    'access_list'           => [ $ipv4addr | $ipv4_subnet/prefix | $ipv6addr |
# Optional
                                 $ipv6_subnet/prefix | $TSIGKey | $NamedACL, ... ],
    'extattrs'              => { $string => $extattr, ... },
# Optional / Default is undefined
    'extensible_attributes' => { $string => $string | $num, $string => [ $string | $num, ... ], ... },
# Optional
 );
```

You cannot set both extattrs and extensible_attributes attributes at the same time.

SESSION METHODS

This section describes all the methods in the Infoblox::Session module that you can apply to a Named ACL object.

Infoblox::Session->get( )

Use this method to retrieve all the matching objects from the Infoblox appliance. See Infoblox::Session->get() for parameters and return values.

Key References

```
Apply the following attributes to get a specific Named ACL object:

 name    - Optional. The name of Named ACL object.
 comment - Optional. The comment in string format.
 extattrs- Optional. A hash reference containing extensible attributes.
```

Example

```
my @retrieved_objs = $session->get(
   object => 'Infoblox::Grid::NamedACL',
   name   => 'nacl',
```

```
);
```

Infoblox::Session->modify( )

Use this method to modify an object in the Infoblox appliance. See Infoblox::Session->modify() for parameters and return values.

If you have applied an access control list to an object, you must restart services after you update the access control list for the changes to take effect. See Infoblox::Session->restart() method.

Example

```
# Use this method to modify comment of Named ACL object.
$nacl->comment('new comment');
# Submit modification
my $response = $session->modify( $nacl );
```

Infoblox::Session->remove( )

Use this method to remove an object from the Infoblox appliance. See Infoblox::Session->remove() for parameters and return values.

To remove a specific object, first use `get()` or `search()` to retrieve the specific object, and then submit this object for removal.

Example

```
# Get the objects with the same name
my @retrieved_objs = $session->get(
    object => "Infoblox::Grid::NamedACL",
    name   => "nacl");
# Find the desired object from the retrieved list.
my $desired_nacl = $retrieved_objs[0];
# Submit for removal
my $response = $session->remove( $desired_nacl );
```

Infoblox::Session->search( )

Use this method to search for Named ACL objects in the Infoblox appliance. See Infoblox::Session->search() for parameters and return values.

Key References

```
Apply the following attributes to search for Named ACL objects:

 name    - Optional. The name of Named ACL object (regular expression).
 comment - Optional. The comment in string format (regular expression).
 extattrs- Optional. A hash reference containing extensible attributes.
```

Example

```
my @retrieved_objs = $session->search(
   object => 'Infoblox::Grid::NamedACL',
   name   => 'nacl',
);
```

MODULE METHODS

Retrieved Named ACL objects in an access list are considered 'partial' objects. See Infoblox::Session->fill_partial_object() for more information on how it can be converted to a 'full' API object.

METHODS

This section describes all the methods that you can use to configure and retrieve the attribute values of Named ACL object.

access_list( )

Use this method to set or retrieve an access list of Named ACL object.

Parameter

The valid value is an array reference that contains IPv4/IPv6 addresses, networks, Infoblox::DNS::TSIGKey objects and/or Infoblox::Grid::NamedACL objects.

To deny queries from certain IP addresses and networks, add "!" as a prefix to the IP addresses and networks.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
# Get attribute value
my $value = $nacl->access_list();
# Modify attribute value
$nacl->access_list(['10.0.0.1', $tsigkey, $nacl]);
```

comment( )

Use this method to set or retrieve a descriptive comment about Named ACL object.

Parameter

Desired comment in string format with a maximum of 256 bytes.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
# Get attribute value
my $value = $nacl->comment();
# Modify attribute value
$nacl->comment('modified comment');
```

exploded_access_list( )

Use this method to retrieve the exploded access list of the Named ACL object. This is a read-only attribute.

Parameter

None.

Returns

The method returns the attribute value.

The valid value is an array reference that contains IPv4/IPv6 addresses, networks and/or Infoblox::DNS::TSIGKey objects.

Example

```
# Get attribute value
my $value = $nacl->exploded_access_list();
```

extattrs( )

Use this method to set or retrieve the extensible attributes associated with a Named ACL object.

Parameter

Valid value is a hash reference containing the names of extensible attributes and their associated values (Infoblox::Grid::Extattr objects).

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get extattrs
my $ref_extattrs = $nacl->extattrs();
#Modify extattrs
$nacl->extattrs({ 'Site' => $extattr1, 'Administrator' => $extattr2 });
```

extensible_attributes( )

Use this method to set or retrieve the extensible attributes associated with Named ACL object.

Parameter

For valid values for extensible attributes, see Infoblox::Grid::ExtensibleAttributeDef/Extensible Attribute Values.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
# Get attribute value
my $value = $nacl->extensible_attributes();
# Modify attribute value
$nacl->extensible_attributes({ 'Site' => 'Santa Clara', 'Administrator' => [ 'Peter', 'Tom' ] });
```

name( )

Use this method to set or retrieve the name of the Named ACL object.

Parameter

The name of the Named ACL object in string format.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
# Get attribute value
my $value = $nacl->name();
# Modify attribute value
$nacl->name('new_nacl');
```

SAMPLE CODE

The following sample code demonstrates the different functions that can be applied to an object such as add, get, modify, search and remove. This sample also includes error handling for the operations.

#Preparation prior to getting and modifying an object

```
use strict;
use Infoblox;

#refers to Infoblox Appliance IP address
my $host_ip = "192.168.1.2";

#Create a session to the Infoblox appliance

my $session = Infoblox::Session->new(
    master   => $host_ip,
    username => "admin",
    password => "infoblox"
);
unless ($session) {
       die("Construct session failed: ",
               Infoblox::status_code() . ":" . Infoblox::status_detail());
}
print "Session created successfully\n";
```

#Create an object

```
my $nacl = Infoblox::Grid::NamedACL->new(
    name          => "NACL",
    access_list   => ['10.0.0.1', '20.0.0.0/24'],
    comment       => "Script Generated",
    );
unless ($nacl) {
       die("Construct object failed: ",
               Infoblox::status_code() . ":" . Infoblox::status_detail());
}
print "The object created successfully\n";

#Adding the object to appliance through session
my $response = $session->add( $nacl );
```

```
unless ($response) {
    die("Add object failed: ",
           $session->status_code() . ":" . $session->status_detail());
}
print"The object was added successfully \n";
```

#Search for the object

```
 my @retrieved_objs = $session->search(
     object => "Infoblox::Grid::NamedACL",
     name   => ".*");
 my $object = $retrieved_objs[0];

 unless ($object) {
        die("Search the object failed: ",
                $session->status_code() . ":" . $session->status_detail());
 }
 print "Search the object found at least 1 matching entry\n";
```

#Get and modify the object

```
#Get the object through session
my @retrieved_objs = $session->get(
    object => "Infoblox::Grid::NamedACL",
    name   => "NACL"
    );
my $nacl = $retrieved_objs[0];

unless ($nacl) {
    die("Get the object failed: ",
           $session->status_code() . ":" . $session->status_detail());
}
print"Retrieved the object successfully \n";

# Modify one of the attributes of selected object.

$nacl->comment("Modified object");

#Applying the changes
$session->modify($nacl)
   or die("Modify the object failed",
            $session->status_code() . ":" . $session->status_detail());
print"The object was modified successfully \n";
```

#Remove the object

```
#Get the object through session
my @retrieved_objs = $session->get(
    object => "Infoblox::Grid::NamedACL",
    name   => "NACL"
    );
my $desired_nacl = $retrieved_objs[0];

unless ($desired_nacl) {
    die("Get the object failed: ",
           $session->status_code() . ":" . $session->status_detail());
}
print"Retrieved the object successfully \n";

#Submit the object for removal
$session->remove( $desired_nacl )
    or die("Remove the object failed",
           $session->status_code() . ":" . $session->status_detail());
print"The object was removed successfully \n";

####PROGRAM ENDS####
```

AUTHOR

Infoblox Inc. http://www.infoblox.com/

SEE ALSO

Infoblox::Session, Infoblox::DNS::TSIGKey, Infoblox::Grid::ExtensibleAttributeDef

---

COPYRIGHT

Copyright (c) 2017 Infoblox Inc.

Infoblox::Grid::DNS::FixedRRSetOrderFQDN - A fixed RRset order FQDN object.

- NAME
- DESCRIPTION
- CONSTRUCTOR
- SESSION METHODS
- METHODS
  - fqdn( )
  - record_type( )
- AUTHOR
- SEE ALSO
- COPYRIGHT

NAME

Infoblox::Grid::DNS::FixedRRSetOrderFQDN - A fixed RRset order FQDN object.

DESCRIPTION

A fixed RRset order FQDN contains information about the fixed RRset configuration items.

By default, when a client queries a domain name, the DNS caching appliance returns the A and AAAA records of the domain name in the cyclic order. However, this default behavior can be overridden if you have enabled and configured (at the Grid level) fixed RRset order for hosts that have multiple addresses. When you override the default behavior and preserve the fixed RRset order for cached DNS responses, the DNS caching appliance returns A and AAAA records associated with domain names in the order they were received from an upstream server. You can preserve the RRset order for the cached DNS responses and specify the fixed RRset order for A, AAAA, or both A and AAAA records at the Grid level and override at the member and DNS view levels.

Note that configuring fixed RRset order for specific FQDNs might significantly affect the performance of the DNS caching appliance.

CONSTRUCTOR

```
my $rrset_order = Infoblox::Grid::DNS::FixedRRSetOrderFQDN->new(
    fqdn        => $string,               #Required
    record_type => 'A' | 'AAAA' | 'BOTH', #Optional / Default is 'A'
);
```

SESSION METHODS

The object does not support any session methods. It's used for fixed RRset order configuration for Infoblox::Grid::DNS, Infoblox::Grid::Member::DNS, Infoblox::DNS::View objects.

METHODS

This section describes all the methods that you can use to set or retrieve the attribute values of the object.

fqdn( )

Use this method to set or retrieve the FQDN of the fixed RRset configuration item.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

The valid value is a string containing desired FQDN.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get fqdn value
my $fqdn = $order_fqdn->fqdn();
#Modify fqdn
$order_fqdn->fqdn('domain.com');
```

record_type( )

Use this method to set or retrieve the returned record type for the specified FQDN in the fixed RRset configuration.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

The valid values are 'A', 'AAAA' and 'BOTH'. The default value is 'A'.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get record_type value
my $record_type = $order_fqdn->record_type();
#Modify record_type
$order_fqdn->record_type('AAAA');
```

AUTHOR

Infoblox Inc. http://www.infoblox.com/

SEE ALSO

Infoblox::Grid::DNS, Infoblox::Grid::Member::DNS, Infoblox::DNS::View

---

COPYRIGHT

Copyright (c) 2017 Infoblox Inc.

Infoblox::Grid::DNS::ScavengingSetting - A DNS scavenging settings object.

- NAME
- DESCRIPTION
- CONSTRUCTOR
- SESSION METHODS
- MODULE METHODS
  - Infoblox::Grid::DNS->scavenging_settings( )
  - Infoblox::DNS::View->scavenging_settings( )
  - Infoblox::DNS::Zone->scavenging_settings( )
- METHODS
  - ea_expression_list( )
  - enable_scavenging( )
  - enable_recurrent_scavenging( )
  - enable_rr_last_queried( )
  - enable_zone_last_queried( )
  - enable_auto_reclamation( )
  - reclaim_associated_records( )
  - expression_list( )
  - scavenging_schedule( )
- AUTHOR
- SEE ALSO
- COPYRIGHT

NAME

Infoblox::Grid::DNS::ScavengingSetting - A DNS scavenging settings object.

DESCRIPTION

The DNS scavenging settings object provides information about DNS records scavenging configuration such as the conditions under which records can be reclaimed and periodicity of the scavening operations.

CONSTRUCTOR

```
my $reclamation_setting = Infoblox::Grid::DNS::ScavengingSetting->new(
    ea_expression_list          => [$ea_expr1, $ea_expr2, ...], # Optional / Default is undefined
    enable_auto_reclamation     => 'true' | 'false',            # Optional / Default is 'false'
    enable_recurrent_scavenging => 'true' | 'false',            # Optional / Default is 'false'
    enable_rr_last_queried      => 'true' | 'false',            # Optional / Default is 'false'
    enable_scavenging           => 'true' | 'false',            # Optional / Default is 'false'
    enable_zone_last_queried    => 'true' | 'false',            # Optional / Default is 'false'
    expression_list             => [$expr1, $expr2, ...],       # Optional / Default is undefined
    reclaim_associated_records  => 'true' | 'false',            # Optional / Default is 'false'
    scavenging_schedule         => $sched_setting,              # Optional / Default is undefined
);
```

SESSION METHODS

The object does not support any session methods.

MODULE METHODS

Infoblox::Grid::DNS->scavenging_settings( )

Use this method to set or retrieve the Grid-level scavenging settings. See Infoblox::Grid::DNS->scavenging_settings() for parameters and return values.

Example

```
#Get scavenging_settings value
my $scavenging_settings = $object->scavenging_settings();

#Modify scavenging_settings value
$object->scavenging_settings($scavenging_settings);
```

Infoblox::DNS::View->scavenging_settings( )

Use this method to set or retrieve the DNS view-level scavenging settings. See Infoblox::DNS::View->scavenging_settings() for parameters and return values.

Example

```
#Get scavenging_settings value
my $scavenging_settings = $object->scavenging_settings();

#Modify scavenging_settings value
$object->scavenging_settings($scavenging_settings);
```

Infoblox::DNS::Zone->scavenging_settings( )

Use this method to set or retrieve the DNS zone-level scavenging settings. See Infoblox::DNS::Zone->scavenging_settings() for parameters and return values.

Example

```
#Get scavenging_settings value
my $scavenging_settings = $object->scavenging_settings();

#Modify scavenging_settings value
$object->scavenging_settings($scavenging_settings);
```

METHODS

This section describes all the methods that you can use to configure and retrieve the attribute values of a DNS scavenging settings object.

ea_expression_list( )

Use this method to set or retrieve the extensible attributes expression list.

The particular record is treated as reclaimable if the extensible attribute expression condition evaluates to 'true' and scavenging is not manually disabled on the given record. Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

The valid value is an array of Infoblox::Grid::EAExpressionOp objects.

The following is a table of allowed expression operand parameters:

op1_type	op1	op2_type	op2	op
FIELD	Infoblox::Grid::ExtensibleAttributeDef	STRING	$value	EQ, NOT_EQ
	Infoblox::Grid::ExtensibleAttributeDef	undef	undef	EXISTS, NOT_EXISTS
LIST	undef	undef	undef	AND, OR, ENDLIST

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get ea_expression_list value
my $ea_expression_list = $object->ea_expression_list();

#Construct expression (Site == LA OR Site == CA)

my $ead = Infoblox::Grid::ExtensibleAttributeDef->new(
    name => 'Site',
);

my $start_list = Infoblox::Grid::EAExpressionOp->new(
    op1_type => 'LIST',
    op       => 'OR',
);
```

```
my $ea_op1 = Infoblox::Grid::EAExpressionOp->new(
    op1_type => 'FIELD',
    op1      => $ead,
    op2_type => 'STRING',
    op2      => 'LA',
    op       => 'EQ',
);

my $ea_op2 = Infoblox::Grid::EAExpressionOp->new(
    op1_type => 'FIELD',
    op1      => $ead,
    op2_type => 'STRING',
    op2      => 'CA',
    op       => 'EQ',
);

my $end_list = Infoblox::Grid::EAExpressionOp->new(
    op1_type => 'LIST',
    op       => 'ENDLIST',
);

#Modify ea_expression_list value
$object->ea_expression_list([$start_list, $ea_op1, $ea_op2, $end_list]);
```

enable_scavenging( )

Use this method to set or retrieve the flag that indicates whether the resource record scavenging is enabled or not.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

Specify 'true' to enable the resource record reclamation or 'false' to disable it. The default value is 'false'.

Parameter

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get enable_scavenging value
my $enable_scavenging = $object->enable_scavenging();

#Modify enable_scavenging value
$object->enable_scavenging('true');
```

enable_recurrent_scavenging( )

Use this method to set or retrieve the flag that indicates whether the recurrent resource record scavenging is enabled or not.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

Specify 'true' to enable the recurrent resource record scavenging or 'false' to disable it. The default value is 'false'.

Parameter

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get enable_recurrent_scavenging value
my $enable_recurrent_scavenging = $object->enable_recurrent_scavenging();

#Modify enable_recurrent_scavenging value
$object->enable_recurrent_scavenging('true');
```

enable_rr_last_queried( )

Use this method to set or retrieve the flag that indicates whether the resource record last queried monitoring in affected zones is enabled or not.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

Specify 'true' to enable the resource record last queried monitoring or 'false' to disable it. The default value is 'false'.

Parameter

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get enable_rr_last_queried value
my $enable_rr_last_queried = $object->enable_rr_last_queried();

#Modify enable_rr_last_queried value
$object->enable_rr_last_queried('true');
```

enable_zone_last_queried( )

Use this method to set or retrieve the flag that indicates whether the last queried monitoring for affected zones is enabled or not.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

Specify 'true' to enable the last queried monitoring for affected zones or 'false' to disable it. The default value is 'false'.

Parameter

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get enable_zone_last_queried value
my $enable_zone_last_queried = $object->enable_zone_last_queried();

#Modify enable_zone_last_queried value
$object->enable_zone_last_queried('true');
```

enable_auto_reclamation( )

Use this method to set or retrieve the flag that indicates whether the automatic resource record reclamation is enabled or not.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

Specify 'true' to enable the automatic resource record reclamation or 'false' to disable it. The default value is 'false'.

Parameter

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get enable_auto_reclamation value
my $enable_auto_reclamation = $object->enable_auto_reclamation();

#Modify enable_auto_reclamation value
$object->enable_auto_reclamation('true');
```

reclaim_associated_records( )

Use this method to set or retrieve the flag that indicates whether the associated resource record reclamation is enabled or not.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

Specify 'true' to enable the associated resource record reclamation or 'false' to disable it. The default value is 'false'.

Parameter

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

G #Get reclaim_associated_records value my $reclaim_associated_records = $object->reclaim_associated_records();

```
#Modify reclaim_associated_records value
$object->reclaim_associated_records('true');
```

expression_list( )

Use this method to set or retrieve the expression list.

The particular record is treated as reclaimable if expression condition evaluates to 'true' for given record if reclamation hasn't been manually disabled on a given resource record.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

The valid value is an array of Infoblox::Grid::ExpressionOp objects.

The following is a table of allowed expression operand parameters:

```
  +----------+-------------------------------------------------------------+----------+-------------------
------------------------------+----------------------------------+
  | op1_type | op1                                                         | op2_type | op2
|  op                          |
  +----------+-------------------------------------------------------------+----------+-------------------
------------------------------+----------------------------------+
  |          | rtype    - the resource record type                         | STRING   | A, AAAA, CNAME,
DNAME, MX, NAPTR, PTR, SRV, TXT | EQ, NOT_EQ                     |
  |          | ctime    - the resource record creation time                | STRING   | $uint
| GT (in days)                 |
  |          | qtime    - the resource record last queried time            | STRING   | $uint
| GT (in days)                 |
  | FIELD    | stime    - the resource record last seen time               | STRING   | $uint
| GT (in days)                 |
  |          | rcreator - the resource record creator                      | STRING   | STATIC, DYNAMIC
| EQ                           |
  |          | assocr   - the resource record associated record            | undef    | undef
| EXISTS, NOT_EXISTS           |
  +----------+-------------------------------------------------------------+----------+-------------------
------------------------------+----------------------------------+
  | LIST     | undef                                                       | undef    | undef
| AND, OR, ENDLIST             |
  +----------+-------------------------------------------------------------+----------+-------------------
------------------------------+----------------------------------+
```

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get expression_list value
my $expression_list = $object->expression_list();

#Construct expression (rcreator == DYNAMIC AND ctime > 10)

my $start_list = Infoblox::Grid::ExpressionOp->new(
    op1_type => 'LIST',
    op       => 'AND',
);

my $op1 = Infoblox::Grid::ExpressionOp->new(
    op1_type => 'FIELD',
    op1      => 'rcreator',
    op2_type => 'STRING',
    op2      => 'DYNAMIC',
    op       => 'EQ',
);

my $op2 = Infoblox::Grid::ExpressionOp->new(
    op1_type => 'FIELD',
    op1      => 'ctime',
    op2_type => 'STRING',
    op2      => '10',
```

```
    op       => 'GT',
);

my $end_list = Infoblox::Grid::ExpressionOp->new(
    op1_type => 'LIST',
    op       => 'ENDLIST',
);

#Modify expression_list value
$object->expression_list([$start_list, $op1, $op2, $end_list]);
```

scavenging_schedule( )

Use this method to set or retrieve the scavenging schedule settings.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

The valid value is an Infoblox::Grid::ScheduleSetting object.

Parameter

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get scavenging_schedule value
my $scavenging_schedule = $object->scavenging_schedule();

#Modify scavenging_schedule value
$object->scavenging_schedule($schedule);
```

AUTHOR

Infoblox Inc. http://www.infoblox.com/

SEE ALSO

Infoblox::Grid::DNS, Infoblox::Grid::DNS->scavenging_settings(), Infoblox::DNS::View, Infoblox::DNS::View->scavenging_settings(), Infoblox::DNS::Zone, Infoblox::DNS::Zone->scavenging_settings(), Infoblox::Grid::ScheduleSetting, Infoblox::Grid::ExpressionOp, Infoblox::Grid::EAExpressionOp,

---

COPYRIGHT

Copyright (c) 2017 Infoblox Inc.

Infoblox::Grid::DNS::ResponseRateLimiting - a DNS Response Rate Limiting object.

- NAME
- DESCRIPTION
- CONSTRUCTOR
- SESSION METHODS
- MODULE METHODS
  - Infoblox::Grid::DNS->response_rate_limiting( )
  - Infoblox::Grid::Member::DNS->response_rate_limiting( )
  - Infoblox::DNS::View->response_rate_limiting( )
- METHODS
- enable_rrl( )
- log_only( )
- responses_per_second( )
- slip( )
- window( )
- AUTHOR
- SEE ALSO
- COPYRIGHT

NAME

Infoblox::Grid::DNS::ResponseRateLimiting - a DNS Response Rate Limiting object.

DESCRIPTION

The DNS Response Rate Limiting object provides information about DNS response rate limiting configuration.

CONSTRUCTOR

```
my $response_rate_limiting = Infoblox::Grid::DNS::ResponseRateLimiting->new(
        enable_rrl           => 'true' | 'false', # Optional
        log_only             => 'true' | 'false', # Optional
        responses_per_second => $uint,            # Optional
        window               => $uint,            # Optional
        slip                 => $uint,            # Optional
);
```

SESSION METHODS

The object does not support any session methods.

MODULE METHODS

The following functions can be applied to a DNS Response Rate Limiting object.

Infoblox::Grid::DNS->response_rate_limiting( )

+Use this method to set or retrieve DNS response rate limiting settings. See Infoblox::Grid::DNS->response_rate_limiting() for parameters and return values.

Example

```
#Get response_rate_limiting value
my $response_rate_limiting = $grid_dns->response_rate_limiting();
#Modify bind_check_response_rate_limiting
$grid_dns->response_rate_limiting($response_rate_limiting);
```

Infoblox::Grid::Member::DNS->response_rate_limiting( )

Use this method to set or retrieve DNS response rate limiting settings on the member. See Infoblox::Grid::Member::DNS->response_rate_limiting() for parameters and return values.

Example

```
#Get response_rate_limiting value
my $response_rate_limiting = $member_dns->response_rate_limiting();
#Modify response_rate_limiting
$member_dns->response_rate_limiting($response_rate_limiting);
```

Infoblox::DNS::View->response_rate_limiting( )

Use this method to set or retrieve DNS response rate limiting settings for the DNS view. See Infoblox::DNS::View->response_rate_limiting() for parameters and return values.

Example

```
#Get response_rate_limiting value
my $response_rate_limiting = $dns_view->response_rate_limiting();
#Modify response_rate_limiting
$dns_view->response_rate_limiting($response_rate_limiting);
```

METHODS

---

enable_rrl( )

Use this method to enabel or disable response rate limiting.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

Specify 'true' to enable response rate limiting or 'false' to disable it.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get enable_rrl value
my $enable_rrl = $response_rate_limiting->enable_rrl();
#Modify enable_rrl
$response_rate_limiting->enable_rrl('true');
```

---

log_only( )

Use this method to enable or disable logging for response rate limiting without dropping any requests.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

Specify 'true' to enable logging for response rate limiting without dropping any requests or 'false' to disable it.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get log_only value
my $log_only = $response_rate_limiting->log_only();
#Modify log_only
$response_rate_limiting->log_only('true');
```

---

responses_per_second( )

Use this method to set or retrieve the number of responses per client per second.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

The valid value is an unsigned integer.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get responses_per_second value
my $responses_per_second = $response_rate_limiting->responses_per_second();
#Modify responses_per_second
$response_rate_limiting->responses_per_second(50);
```

---

slip( )

Use this method to set or retrieve the response rate limiting slip.

Note that if a slip does not equal 0, instead of dropping the request, a truncated response is sent to each n-th rate-limited UDP request.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

The valid value is an unsigned integer.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get slip value
my $slip = $response_rate_limiting->slip();
#Modify slip
$response_rate_limiting->slip(50);
```

---

window( )

Use this method to set or retrieve the time interval in seconds over which responses are tracked.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

The valid value is an unsigned integer.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get window value
my $window = $response_rate_limiting->window();
#Modify window
$response_rate_limiting->window(10);
```

AUTHOR

Infoblox Inc. http://www.infoblox.com/

SEE ALSO

Infoblox::Grid::DNS, Infoblox::Grid::DNS->response_rate_limiting(), Infoblox::Grid::Member::DNS, Infoblox::Grid::Member::DNS->response_rate_limiting(), Infoblox::DNS::View->response_rate_limiting()

COPYRIGHT

Copyright (c) 2017 Infoblox Inc.

Infoblox::DNS::Sortlist - DNS Sortlist Object.

- NAME
- DESCRIPTION
- CONSTRUCTOR
- MODULE METHODS
  - Infoblox::Grid::DNS->sortlist( )
  - Infoblox::Grid::Member::DNS->sortlist( )
- METHODS
  - match_list( )
  - source_ipv4addr( )
  - source_ipv6addr( )
  - comment( )
- SAMPLE CODE
- AUTHOR
- SEE ALSO
- COPYRIGHT

NAME

Infoblox::DNS::Sortlist - DNS Sortlist Object.

DESCRIPTION

A sortlist sorts the order of addresses in responses made to DNS queries.

CONSTRUCTOR

```
 my $sort_list = Infoblox::DNS::Sortlist->new(
     source_ipv4addr => $ipv4addr | $ipv4_subnet/prefix
#Either source_ipv4addr or source_ipv6addr is required
     source_ipv6addr => $ipv6addr | $ipv6_subnet/prefix
#Either source_ipv4addr or source_ipv6addr is required
     match_list      => [ $ipv4addr | $ipv4_subnet/prefix | $ipv6addr |  $ipv6_subnet/prefix ...] | undef
#Optional / Default is undefined
     comment         => $string| undef,                    #Optional / Default is undefined
 );
```

MODULE METHODS

The following functions are available to apply to an Sortlist object.

Infoblox::Grid::DNS->sortlist( )

Use this function to specify a sortlist at the grid level on the Infoblox appliance. See Infoblox::Grid::DNS->sortlist() for parameters and return values.

Example

```
# Construct the object
my $sortlist = Infoblox::DNS::Sortlist->new(
    match_list      => ["10.20.1.0/24", "10.20.2.0/24"],
    source_ipv4addr => "1.2.3.4"
);
# Configure sortlist on the grid DNS object
my $response = $Grid_DNS->sortlist([$sortlist]);
```

Infoblox::Grid::Member::DNS->sortlist( )

Use this function to specify a sortlist at the grid member level. See Infoblox::Grid::Member::DNS->sortlist() for parameters and return values.

Example

```
# Construct the object
my $sortlist = Infoblox::DNS::Sortlist->new(
    match_list      => ["10.20.1.0/24", "10.20.2.0/24"],
    source_ipv4addr => "1.2.3.4"
);
# Configure sortlist on a Grid Member DNS object
```

```
my $response = $Grid_Member_DNS->sortlist([$sortlist]);
```

METHODS

This section describes all the methods that you can use to configure and retrieve the attribute values of a sortlist object.

match_list( )

Use this method to set or retrieve the match list of a sortlist.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

Array reference which contains IP addresses and/or networks.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get match list.
my $ref_matchlist = $sort_list->match_list();
#Modify match list.
$sort_list->match_list(["10.0.0.10", "20.0.0.0/24", "2001:db8:1234::/48", "2001:db8::1428:57ab"]);
```

source_ipv4addr( )

Use this method to set or retrieve the source IPv4 address of a sortlist object.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

Address or network of querier which will be preferred by the resolver as the result of a query. To specify "ANY" querier, please use "0.0.0.0/0".

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get ipv4addrs
my $source_ipv4addr = $sort_list->source_ipv4addr();
#Modify ipv4addrs
$sort_list->source_ipv4addr("10.20.1.0/24");
```

source_ipv6addr( )

Use this method to set or retrieve the source IPv6 address of a sortlist object.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

Address or network of querier which will be preferred by the resolver as the result of a query.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get ipv6addrs
my $source_ipv6addr = $sort_list->source_ipv6addr();
#Modify ipv6addrs
$sort_list->source_ipv6addr("2001:25d8:44:55::/64");
```

comment( )

Use this method to set or retrieve a descriptive comment of a sortlist object.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

Enter a descriptive comment for the sortlist in string format with a maximum of 256 bytes.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
# Get comment
my $comment = $sort_list->comment();
# Modify comment
$sort_list->comment("test comment");
```

SAMPLE CODE

The following sample code demonstrates the different functions that can be applied to an object, such as add, modify. This sample also includes error handling for the operations.

#Preparation prior to a DNS sortlist object insertion

```
#PROGRAM STARTS: Include all the modules that will be used
use strict;
use Infoblox;

#Create a session to the Infoblox appliance

my $session = Infoblox::Session->new(
           master   => "192.168.1.2",
           username => "admin",
           password => "infoblox"
);

unless ($session) {
   die("Construc session failed: ",
       Infoblox::status_code() . ":" . Infoblox::status_detail());
}
print "Session created successfully\n";
```

#Create a Sortlist object

```
my $sort1 = Infoblox::DNS::Sortlist->new(
    match_list      => ["10.20.1.0/24", "10.20.2.0/24"],
    source_ipv4addr => "1.2.3.4"
);

unless($sort1) {
     die("Construct sort list failed: ",
           Infoblox::status_code() . ":" . Infoblox::status_detail());
}
print "First sort list object (IPv4) created successfully\n";
```

#Create another sortlist to demonstrate IPv6

```
my $sort2 = Infoblox::DNS::Sortlist->new(
    match_list      => ["2001:db8:1234::/48", "2001:db8:345::/64", "192.168.1.1"],
    source_ipv6addr => "2001:db8::1428:57ab"
);

unless($sort2) {
     die("Construct sort list failed: ",
           Infoblox::status_code() . ":" . Infoblox::status_detail());
}
print "Second sort list object (IPv6) created successfully\n";
```

#Get the Member DNS object to add sortlist object to it

```
my @result_array = $session->get(
    object => "Infoblox::Grid::Member::DNS",
```

```
    name    => $host_name
);

unless (scalar(@result_array) == 0) {
    my $memberdns = $result_array[0];

    if ($memberdns) {

        #Apply changes to the Member object.
        $memberdns->sortlist([$sort1, $sort2])
            or die("modify member failed: ",
                   $session->status_code() . ":" . $session->status_detail());
        print "sort lists added to Member DNS object successfully\n";

        #Update member DNS object through the Infoblox session.
        $session->modify($memberdns)
            or die("modify session failed: ",
                   $session->status_code() . ":" . $session->status_detail());
        print "Member DNS object with sortlists updated to Infoblox device successfully\n";

        #Modify existing sortlist objects

        #Modifying the value of the specified object.
        $sort1->source_ipv4addr("10.20.1.0/24");
        print "Modify first sortlist value\n";

        #Modifying the value of the specified object.
        $sort1->match_list(["10.0.0.10", "20.0.0.0/24", "30.0.0.30", "40.10.0.0/24"]);

        #Modifying the value of the specified object.
        $sort2->source_ipv6addr("2001:db8::1428:0/112");
        print "Modify second sortlist value\n";

        #Apply changes to the member object.
        $memberdns->sortlist([$sort1, $sort2])
            or die("modify member failed: ",
                   Infoblox::status_code() . ":" . Infoblox::status_detail());
        print "sort list updated to Member DNS object successfully\n";

        #Update member DNS object through the Infoblox session.
        $session->modify($memberdns)
            or die("modify session failed: ",
                $session->status_code() . ":" . $session->status_detail());
                print "Member DNS object with sortlist updated to Infoblox device successfully\n";
            }
         } else {
             print "No member found with the specified name.";
         }

 ####PROGRAM ENDS####
```

AUTHOR

Infoblox Inc. http://www.infoblox.com/

SEE ALSO

Infoblox::Session,Infoblox::Session->get(), Infoblox::Session->modify(),Infoblox::Grid::Member::DNS->sortlist(), Infoblox::Grid::DNS->sortlist()

COPYRIGHT

Copyright (c) 2017 Infoblox Inc.

Infoblox::DNS::DnssecTrustedKey - DNSSEC trusted key object.

- NAME
- DESCRIPTION
- CONSTRUCTOR
- MODULE METHODS
  - Infoblox::Grid::DNS->dnssec_trusted_keys( )
  - Infoblox::Grid::Member::DNS->dnssec_trusted_keys( )
  - Infoblox::DNS::View->dnssec_trusted_keys( )
- METHODS
  - algorithm( )
  - fqdn( )
  - key( )
  - secure_entry_point()
- SAMPLE CODE
- AUTHOR
- SEE ALSO
- COPYRIGHT

NAME

Infoblox::DNS::DnssecTrustedKey - DNSSEC trusted key object.

DESCRIPTION

For recursively derived signed data, the DNS server can perform validation. In order to be secure, this validation relies on a set of trust anchors. These trust anchors and their associated trusted keys can be specified via the Infoblox::DNS::DnssecTrustedKey objects.

DNSSEC is based on DNS specifications RFC 4033, RFC 4034, and RFC 4035, which provide authentication protocol enhancements to secure DNS transactions.

CONSTRUCTOR

```
 my $dnssec_trusted_key=Infoblox::DNS::DnssecTrustedKey->new(
                    algorithm             =>
"RSAMD5"|"DSA"|"RSASHA1"|"NSEC3DSA"|"NSEC3RSASHA1"|"RSASHA256"|"RSASHA512"|"NSEC3RSASHA256"|"NSEC3RSASHA512"|1|3|5|6|7,
  #Required
                    fqdn                  => $string,                                  #Required
                    key                   => $string,                                  #Required
                    secure_entry_point    => "true"|"false"                            #Optional / Default is "true"
                        );
```

MODULE METHODS

This section describes all the functions that you can apply to a DnssecTrustedKey object.

Infoblox::Grid::DNS->dnssec_trusted_keys( )

Use this function to specify a list of DNSSEC keys for a grid object.

Example

```
      ## Construct the trusted key object. When trying to implement this example replace with the actual public key
     of the signed zone.
      my $dnssec_key = Infoblox::DNS::DnssecTrustedKey->new(
          algorithm => "RSAMD5",
          fqdn => "example.com",
          key  => "Zx1jVJl7C58GT/sc7Q3ucA==",
          secure_entry_point => "false"
      );
      # Configure dnssec_trusted_keys on the grid DNS object
      my $response = $Grid_DNS->dnssec_trusted_keys([$dnssec_key]);
```

Infoblox::Grid::Member::DNS->dnssec_trusted_keys( )

Use this function to specify a list of DNSSEC keys for a grid member object.

Example

```
      ## Construct the trusted key object
      my $dnssec_key = Infoblox::DNS::DnssecTrustedKey->new(
          algorithm => "RSAMD5",
          fqdn => "example.com",
          key  => "Zx1jVJl7C58GT/sc7Q3ucA==",
          secure_entry_point => "false"
      );
      # Configure dnssec_trusted_keys on the grid DNS object
      my $response = $Grid Member DNS->dnssec trusted keys([$dnssec key]);
```

Infoblox::DNS::View->dnssec_trusted_keys( )

Use this function to specify a list of DNSSEC keys for a DNS View object.

Example

```
## Construct the trusted key object
my $dnssec_key = Infoblox::DNS::DnssecTrustedKey->new(
    algorithm => "RSAMD5",
    fqdn => "example.com",
    key  => "Zx1jVJl7C58GT/sc7Q3ucA==",
    secure_entry_point => "false"
);
# Configure dnssec_trusted_keys on the grid DNS object
my $response = $DNS_View->dnssec_trusted_keys([$dnssec_key]);
```

METHODS

algorithm( )

Use this method to set or retrieve a DNSSEC encryption algorithm.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

The public-key encryption algorithm. This can be either an integer or mnemonic. The following table shows the correlation between the integers and mnemonics for the algorithms supported by the appliance:

Value	Algorithm	Mnemonic
1	RSA/MD5	RSAMD5
3	DSA	DSA
5	RSA/SHA-1	RSASHA1
6	NSEC3DSA	NSEC3DSA
7	NSEC3RSA/SHA-1	NSEC3RSASHA1

For algorithms not listed above, only the mnemonic representation is supported.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
# Getting algorithm
my $algorithm = $dnssec_key->algorithm();
# Modifying algorithm
$dnssec_key->algorithm("DSA");
```

fqdn( )

Use this method to set or retrieve the FQDN of the zone to which the key belongs.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

FQDN of the zone to which the trusted_key pertains, in DNS label order.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
# Getting fqdn
my $zone = $dnssec_key->fqdn();
# Modifying algorithm
$dnssec_key->fqdn("zone.example.com");
```

key( )

Use this method to add or retrieve the DNSSEC public key for the zone.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

Base-64 encoding of the key.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
# Getting key
my $key = $dnssec_key->key();
# Modifying key
$dnssec_key->key("AQPdWbrGbVv1eDhNgRhpJMPonJfA3reyEo82ekwRnjbX7+uBxB11BqL7LAB7/C+eb0vCtI53FwMhkkNkTmA6bI8B");
```

secure_entry_point()

Use this method to set or retrieve the secure entry point value for the field.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

Set to "true" to enable the secure entry point, set to "false" to disable it. Default value of this parameter is "true".

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
# Getting secure_entry_point
my $sep = $dnssec_key->secure_entry_point();
# Modifying secure_entry_point
$dnssec_key->secure_entry_point("false");
```

SAMPLE CODE

The following sample code demonstrates the different functions that can be applied to an object, such as add and modify. This sample also includes error handling for the operations.

##Preparation prior to a DnssecTrustedKey object insertion

```
#PROGRAM STARTS: Include all the modules that will be used
use strict;
use Infoblox;

#Create a session to the Infoblox  appliance
my $session = Infoblox::Session->new(
    master   => "192.168.1.2",
    username => "admin",
    password => "infoblox"
);
unless ($session) {
   die("Construct session failed: ",
       Infoblox::status_code() . ":" . Infoblox::status_detail());
}
print "Session created successfully\n";
```

#Create a DnssecTrustedKey object

```
 my $dnssec_key = Infoblox::DNS::DnssecTrustedKey->new(
    key  => "ZX1jVJl7C58GT/sc7Q3ucA==",
    fqdn => "myzone.example.com",
    algorithm => "RSASHA1"
  );

unless($dnssec_key) {
     die("Construct DnssecTrustedKey failed: ",
           Infoblox::status_code() . ":" . Infoblox::status_detail());
}
print "DnssecTrustedKey object created successfully\n";
```

#Get the member DNS object and add a DnssecTrustedKey object to it

```
my @result_array = $session->get(
    object => "Infoblox::Grid::Member::DNS",
    name   => "infoblox.localdomain"
);

unless (scalar(@result_array) == 0) {
    my $memberdns = $result_array[0];

    if ($memberdns) {

        #Apply changes to the Member object.
```

```
                $memberdns->dnssec_trusted_keys([$dnssec_key])
            or die("modify member failed: ",
                   Infoblox::status_code() . ":" . Infoblox::status_detail());
        print "DnssecTrustedKey added to Member DNS object successfully\n";

  #Update member DNS object through the Infoblox session.
        $session->modify($memberdns)
            or die("modify session failed: ",
                   $session->status_code() . ":" . $session->status_detail());
        print "Member DNS object with DnssecTrustedKey updated to Infoblox device successfully\n";

         #Modify existing DnssecTrustedKey object

        #Modifying the value of the specified object.
        $dnssec_key->fqdn("another.example.com");
        print "Modify DnssecTrustedKey fqdn value\n";

        #Apply changes to the member object.
        $memberdns->transfer_keys([$dnssec_key])
            or die("modify member failed: ",
                   Infoblox::status_code() . ":" . Infoblox::status_detail());
        print "DnssecTrustedKey fqdn updated to Member DNS object successfully\n";

        #Update member DNS object through the Infoblox session.
        $session->modify($memberdns)
            or die("modify session failed: ",
                $session->status_code() . ":" . $session->status_detail());
                print "Member DNS object with DnssecTrustedKey fqdn updated to Infoblox device successfully\n";
             }
         } else {
             print "No member found with the specified name.";
         }

 ####PROGRAM ENDS####
```

AUTHOR

Infoblox Inc. http://www.infoblox.com/

SEE ALSO

Infoblox::Grid::Member::DNS, Infoblox::DNS::View, Infoblox::Grid::DNS

COPYRIGHT

Copyright (c) 2017 Infoblox Inc.

Infoblox::Grid::DNS::Nsgroup::DelegationMember - A delegation nameservers group.

- NAME
- DESCRIPTION
- CONSTRUCTOR
- SESSION METHODS
  - Infoblox::Session->add( )
  - Infoblox::Session->get( )
  - Infoblox::Session->modify( )
  - Infoblox::Session->remove( )
  - Infoblox::Session->search( )
- METHODS
  - comment( )
  - extattrs( )
  - extensible_attributes( )
  - delegation_servers( )
  - name( )
- SAMPLE CODE
- AUTHOR
- SEE ALSO
- COPYRIGHT

NAME

Infoblox::Grid::DNS::Nsgroup::DelegationMember - A delegation nameservers group.

DESCRIPTION

When you configure a name server group, you can now create a set of external name servers as a delegation name server group and assign it to delegated zones. Specifying a single delegation name server group instead of configuring multiple name servers individually for delegated zones can significantly reduce configuration efforts.

CONSTRUCTOR

```
 my $object = Infoblox::Grid::DNS::Nsgroup::DelegationMember->new(
     name                  => $string,
#Required
     delegation_servers    => [$nameserver1, $nameserver1, ...],
#Required
     comment               => $string,
#Optional / Default is undefined
     extattrs              => { $string => $extattr, ... },
#Optional / Default is undefined
     extensible_attributes => {$string => $string | $num, $string => [ $string | $num, ... ], ... }
#Optional / Default is undefined
 );
```

You cannot set both extattrs and extensible_attributes attributes at the same time.

SESSION METHODS

This section describes all the methods in an Infoblox::Session module that you can apply to a delegation members group object.

Infoblox::Session->add( )

Use this method to add the object to the Infoblox appliance. See Infoblox::Session->add() for parameters and return values.

Example

```
#Construct an object
my $object = Infoblox::Grid::DNS::Nsgroup::DelegationMember->new(
    name               => 'dlggroup',
    delegation_servers => [$nameserver1, $nameserver2],
);

#Submit for addition
my $response = $session->add($object);
```

Infoblox::Session->get( )

Use this method to retrieve all the matching objects from the Infoblox appliance. See Infoblox::Session->get() for parameters and return values.

Key Reference

```
Apply the following attributes to get a specific delegation member group object:

 comment               - Optional. The delegation member group comment in a string format.
 name                  - Optional. The delegation member group name in a string format.
 extattrs              - Optional. A hash reference containing extensible attributes.
 extensible_attributes - Optional. A hash reference containing extensible attributes.
```

For more information about searching extensible attributes, see Infoblox::Grid::ExtensibleAttributeDef/Searching Extensible Attributes.

Example

```
my @retrieved_objs = $session->get(
    object => 'Infoblox::Grid::DNS::Nsgroup::DelegationMember',
    name   => 'dlggroup',
);
```

Infoblox::Session->modify( )

Use this method to modify the object in the Infoblox appliance. See Infoblox::Session->modify() for parameters and return values.

Example

```
#Modify comment value
$object->comment('this is a modified comment');

#Submit modification
my $response = $session->modify($object);
```

Infoblox::Session->remove( )

Use this method to remove the object from the Infoblox appliance. See Infoblox::Session->remove() for parameters and return values.

To remove a specific object, first use `get()` or `search()` to retrieve the specific DHCP range object, and then submit this object for removal.

Example

```
#Get the objects
my @retrieved_objs = $session->get(
    object => 'Infoblox::Grid::DNS::Nsgroup::DelegationMember',
    name   => 'dlggroup',
);

#Find the desired object from the retrieved list
my $desired_object = $retrieved_objs[0];

#Submit for removal
my $response = $session->remove($desired_object);
```

Infoblox::Session->search( )

Use this method to search for objects in the Infoblox appliance. See Infoblox::Session->search() for parameters and return values.

Key References

```
Apply the following attributes to search for the delegation member group object:

 comment               - Optional. The delegation member group comment in a string format (regexp).
 name                  - Optional. The delegation member group name in a string format (regexp).
 extattrs              - Optional. A hash reference containing extensible attributes.
 extensible_attributes - Optional. A hash reference containing extensible attributes.
```

For more information about searching extensible attributes, see Infoblox::Grid::ExtensibleAttributeDef/Searching Extensible Attributes.

Example

```
# search for objects
my @retrieved_objs = $session->search(
    object  => 'Infoblox::Grid::DNS::Nsgroup::DelegationMember',
    name    => '^dlg.*',
    comment => '.*comment',
);
```

```
# search for objects defining 'Santa Clara' for 'Site' extensible attribute
my @retrieved_objs = $session->search(
    object               => 'Infoblox::Grid::DNS::Nsgroup::DelegationMember',
    extensible_attributes => {'Site' => 'Santa Clara'},
);
```

METHODS

This section describes all the methods that you can use to configure and retrieve the attribute values of a delegation member group.

comment( )

Use this method to set or retrieve the delegation member group comment.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

The valid value is a desired comment in a string format.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get comment value
my $comment = $object->comment();

#Modify comment value
$object->comment('dlg comment');
```

extattrs( )

Use this method to set or retrieve the extensible attributes associated with a delegation member group object.

Parameter

Valid value is a hash reference that contains the names of extensible attributes and their associated values (Infoblox::Grid::Extattr objects).

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get extattrs value
my $extattrs = $object->extattrs();

#Modify extattrs value
$object->extattrs({'Site' => $extattr1, 'Administrator' => $extattr2});
```

extensible_attributes( )

Use this method to set or retrieve the extensible attributes associated with a delegation member group object.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

For valid values for extensible attributes, see Infoblox::Grid::ExtensibleAttributeDef/Extensible Attribute Values.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get extensible attributes value
my $extensible_attributes = $object->extensible_attributes();
```

```
#Modify extensible attributes
$object->extensible_attributes({'Site' => 'Santa Clara', 'Administrator' => ['Peter', 'Tom']});
```

delegation_servers( )

Use this method to set or retrieve the list of delegation servers.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

The valid value is an array of Infoblox::DNS::Nameserver objects.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get delegation_servers value
my $delegation_servers = $object->delegation_servers();

#Modify delegation_servers value

#Create nameserver object
my $nameserver = Infoblox::DNS::Nameserver->new(
    name     => 'ns1.domain1.extra',
    ipv4addr => '5.5.5.5',
);

#Set delegation_servers value
$object->delegation_servers([$nameserver]);
```

name( )

Use this method to set or retrieve the delegation member group name.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

The valid value is a desired name in a string format.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get name value
my $name = $object->name();

#Modify name value
$object->name('dlgg1');
```

SAMPLE CODE

The following sample code demonstrates the different functions that can be applied to an object, such as modify and remove. This sample also includes error handling for the operations.

#Preparation prior to an Delegation Member Nsgroup object insertion

```
#PROGRAM STARTS: Include all the modules that will be used
use strict;
use Infoblox;

my ($session, $result);

#Create a session to the Infoblox device
$session = Infoblox::Session->new(
    master   => "192.168.1.2",
    username => "admin",
    password => "infoblox"
);
```

```
unless ($session) {
   die("Construct session failed: ",
       Infoblox::status_code() . ":" . Infoblox::status_detail());
}
print "Session created successfully\n";
```

#Create an Delegation Member Nsgroup object

```
#Creating an external nameserver
my $nameserver = Infoblox::DNS::Nameserver->new(
    name      => "ns1.domain0.external",
    ipv4addr => "5.5.5.5",
);

#Creating a delegation nsgroup object.
my $dlg_nsg1 = Infoblox::Grid::DNS::Nsgroup::DelegationMember->new(
    name               => "dlg_group_1",
    delegation_servers => [$nameserver],
);

unless ($dlg_nsg1) {
   die("Construct Delegation Member Nsgroup failed: ",
       Infoblox::status_code() . ":" . Infoblox::status_detail());
}

print "Delegation Member Nsgroup object constructed successfully\n";
```

#Add the Nsgroup object

```
$result = $session->add($dlg_nsg1);

unless ($result) {
    die("Add Delegation Member Nsgroup to session failed: ",
        $session->status_code() . ":" . $session->status_detail());
}

print "Delegation Member Nsgroup object created successfully\n";
```

#Search and Modify the Nsgroup object

```
my @result_array = $session->search(
    object => "Infoblox::Grid::DNS::Nsgroup::DelegationMember",
    name   => "dlg_.*",
);

my $object = $result_array[0];

unless ($object) {
   die("Search for Delegation Member Nsgroup failed: ",
       $session->status_code() . ":" . $session->status_detail());
}

$result = $object->comment("delegation member");

unless ($result) {
    die("Modify Delegation Member Nsgroup failed: ",
        Infoblox::status_code() . ":" . Infoblox::status_detail());
}

$result = $session->modify($object);

unless ($result) {
    die("Submit modification to the session failed: ",
        $session->status_code() . ":" . $session->status_detail());
}

print "Delegation Member Nsgroup object modified successfully\n";
```

#Remove a Nsgroup object

```
my @result_array = $session->get(
    object => "Infoblox::Grid::DNS::Nsgroup::DelegationMember",
    name   => "dlg_group_1",
);

my $object = $result_array[0];
```

```
unless ($object) {
   die("Get Delegation Member Nsgroup failed: ",
       $session->status_code() . ":" . $session->status_detail());
}

$result = $session->remove($object);

unless ($result) {
    die("Remove Nsgroup failed: ",
        $session->status_code() . ":" . $session->status_detail());
}

print "Delegation Member Nsgroup object removed successfully\n";

####PROGRAM ENDS####
```

AUTHOR

Infoblox Inc. http://www.infoblox.com/

SEE ALSO

Infoblox::Session, Infoblox::Session->add(), Infoblox::Session->get(), Infoblox::Session->modify(), Infoblox::Session->remove(), Infoblox::Session->search(), Infoblox::DNS::Nameserver, Infoblox::Grid::ExtensibleAttributeDef/Extensible Attribute Values, Infoblox::Grid::Extattr

---

COPYRIGHT

Copyright (c) 2017 Infoblox Inc.

Infoblox::DNS::DNSSecKey - DNSSEC key object.

- NAME
- DESCRIPTION
- SESSION METHODS
- METHODS
  - algorithm( )
  - next_event_date( )
  - public_key( )
  - status( )
  - tag( )
  - type( )
- AUTHOR
- SEE ALSO
- COPYRIGHT

NAME

Infoblox::DNS::DNSSecKey - DNSSEC key object.

DESCRIPTION

The DNSSEC object contains information about keys available in authoritative zones.

SESSION METHODS

The object does not support any session methods.

METHODS

This section describes all the methods that you can use to set or retrieve the attribute values of the object.

algorithm( )

Use this method to retrieve the public-key encryption algorithm. This is a read-only attribute.

Parameter

None

Returns

The return value is the public-key encryption algorithm in string format.

Example

```
#Get algorithm value
my $value = $dnssec_key->algorithm();
```

next_event_date( )

Use this method to retrieve the next event date. The event date can be a date for the rollover of an ACTIVE key or a PUBLISHED key, or the removal of a ROLLED key. This is a read-only attribute.

Parameter

None

Returns

The date of next event in ISO 8601 extended format (e.g., 2008-10-21T10:56:00Z or 2008-10-21T10:56:00.001Z).

Example

```
#Get next_event_date value
my $value = $dnssec_key->next_event_date();
```

public_key( )

Use this method to retrieve the Base-64 encoding of the public key. This is a read-only attribute.

Parameter

None

Returns

The return value is a Base-64 encoded public key in string format.

Example

```
#Get public_key value
my $value = $dnssec_key->public_key();
```

status( )

Use this method to retrieve key status for the authoritative zone. This is a read-only attribute.

Parameter

None

Returns

The valid return values are "ACTIVE", "PUBLISHED" and "ROLLED".

Example

```
#Get status value
my $value = $dnssec_key->status();
```

tag( )

Use this method to retrieve the key tag for the authoritative zone. This is a read-only attribute.

Parameter

None

Returns

The return value is an integer between 0 and 65535.

Example

```
#Get tag value
my $value = $dnssec_key->tag();
```

type( )

Use this method to retrieve the type of key. This is a read-only attribute.

Parameter

None

Returns

The valid return values are "KSK" and "ZSK".

Example

```
#Get type value
my $value = $dnssec_key->type();
```

AUTHOR

Infoblox Inc. http://www.infoblox.com/

SEE ALSO

ISO 8601 extended format

COPYRIGHT

Copyright (c) 2017 Infoblox Inc.

Infoblox::DNS::DNSSecKey - DNSSEC key object.

Infoblox::DNS::DNSSecKeyAlgorithm - DNSSEC key algorithm object.

- NAME
- DESCRIPTION
- CONSTRUCTOR
- MODULE METHODS
  - Infoblox::Grid::DNS->dnssec_ksk_algorithms( )
  - Infoblox::Grid::DNS->dnssec_zsk_algorithms( )
  - Infoblox::Grid::Member::DNS->dnssec_ksk_algorithms( )
  - Infoblox::Grid::Member::DNS->dnssec_zsk_algorithms( )
- SESSION METHODS
- METHODS
  - algorithm( )
  - size( )
- AUTHOR
- SEE ALSO
- COPYRIGHT

NAME

Infoblox::DNS::DNSSecKeyAlgorithm - DNSSEC key algorithm object.

DESCRIPTION

The DNSSEC key algorithm structure provides for configuring algorithms for Key-signing and Zone-signing keys. Multiple algorithms can be used at the same time per Grid or per zone.

CONSTRUCTOR

```
 #Constructor for an DNSSEC key algorithm object
  my $algorithm = Infoblox::DNS::DNSSecKeyAlgorithm->new(
     algorithm => "RSAMD5" | "DSA" | "RSASHA1" | "RSASHA256" | "RSASHA512" ,  #Optional / Default is
undefined
     size      => $num,                                                       #Optional / Default is
undefined
   );
```

MODULE METHODS

The following functions are available to be applied to an DNSSecKeyAlgorithm object.

Infoblox::Grid::DNS->dnssec_ksk_algorithms( )

Use this function to specify a DNSSEC key signing key algorithm at the grid level on the Infoblox appliance. See Infoblox::Grid::DNS->dnssec_ksk_algorithms() for parameters and return values.

Example

```
#Create a DNSSEC key algorithm object.
my $algorithm = Infoblox::DNS::DNSSecKeyAlgorithm->new(
    algorithm => 'RSASHA256',
    size      => 512,
);

#Configure DNSSEC key signing key algorithms on the Infoblox grid object
my $response = $grid->dnssec_ksk_algorithms([$algorithm]);
```

Infoblox::Grid::DNS->dnssec_zsk_algorithms( )

Use this function to specify a DNSSEC zone signing key algorithm at the grid level on the Infoblox appliance. See Infoblox::Grid::DNS->dnssec_zsk_algorithms() for parameters and return values.

Example

```
#Create a DNSSEC key algorithm object.
my $algorithm = Infoblox::DNS::DNSSecKeyAlgorithm->new(
    algorithm => 'RSASHA512',
    size      => 1024,
);

#Configure DNSSEC zone signing key algorithms on the Infoblox grid object
```

```
my $response = $grid->dnssec_zsk_algorithms([$algorithm]);
```

Infoblox::Grid::Member::DNS->dnssec_ksk_algorithms( )

Use this function to specify a DNSSEC key signing key at the member level on the Infoblox appliance. See Infoblox::Grid::Member::DNS->dnssec_ksk_algorithms() for parameters and return values.

Example

```
#Create a DNSSEC key algorithm object.
my $algorithm = Infoblox::DNS::DNSSecKeyAlgorithm->new(
    algorithm => 'RSASHA512',
    size      => 1024,
);

#Configure DNSSEC key signing key algorithms on the Infoblox member object
my $response = $member->dnssec_ksk_algorithms([$algorithm]);
```

Infoblox::Grid::Member::DNS->dnssec_zsk_algorithms( )

Use this function to specify a DNSSEC zone signing key at the member level on the Infoblox appliance. See Infoblox::Grid::Member::DNS->dnssec_zsk_algorithms() for parameters and return values.

Example

```
#Create a DNSSEC key algorithm object.
my $algorithm = Infoblox::DNS::DNSSecKeyAlgorithm->new(
    algorithm => 'RSASHA512',
    size      => 1024,
);

#Configure DNSSEC zone signing key algorithms on the Infoblox member object
my $response = $member->dnssec_zsk_algorithms([$algorithm]);
```

SESSION METHODS

The object does not support any session methods.

METHODS

this section describes all the methods that you can use to set or retrieve the attribute values of the object.

algorithm( )

Use this method to set or retrieve signing key algorithm.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

Valid value is 'RSAMD5', 'DSA', 'RSASHA1', 'RSASHA256' or 'RSASHA512'.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get algorithm value
my $algorithm = $dnssec_key_algorithm->algorithm();
#Modify algorithm value
$dnssec_key_algorithm->algorithm('RSASHA512');
```

size( )

Use this method to set or retrieve signing key size, in bits.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

The DNSSEC key-signing key possible size depends on the algorithm. Use the following table to determine the possible values for key sizes:

```
  Algorithm    Key size     Additional restrictions
NSEC
  RSA/MD5      512-4096
  DSA/SHA1     512-1024     Key size must be divisible by 64
  RSA/SHA1     512-4096
  RSA256/SHA2  512-4096
  RSA512/SHA2  1024-4096
NSEC3
  DSA/SHA1     512-1024     Key size must be divisible by 64
  RSA/SHA1     512-4096
  RSA256/SHA2  512-4096
  RSA512/SHA2  1024-4096
```

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get size value
my $size = $dnssec_key_algorithm->size();
#Modify size value
$dnssec_key_algorithm->size(1024);
```

AUTHOR

Infoblox Inc. http://www.infoblox.com/

SEE ALSO

Infoblox::Grid::DNS, Infoblox::Grid::Member::DNS, Infoblox::Grid::DNS->dnssec_ksk_algorithms(), Infoblox::Grid::DNS->dnssec_zsk_algorithms(), Infoblox::Grid::Member::DNS->dnssec_ksk_algorithms(), Infoblox::Grid::Member::DNS->dnssec_zsk_algorithms()

COPYRIGHT

Copyright (c) 2017 Infoblox Inc.

Infoblox::Grid::DNS::Nsgroup::ForwardingMember - A forwarding member group.

- NAME
- DESCRIPTION
- CONSTRUCTOR
- SESSION METHODS
  - Infoblox::Session->add( )
  - Infoblox::Session->get( )
  - Infoblox::Session->modify( )
  - Infoblox::Session->remove( )
  - Infoblox::Session->search( )
- METHODS
  - comment( )
  - extattrs( )
  - extensible_attributes( )
  - forwarding_servers( )
  - name( )
- SAMPLE CODE
- AUTHOR
- SEE ALSO
- COPYRIGHT

NAME

Infoblox::Grid::DNS::Nsgroup::ForwardingMember - A forwarding member group.

DESCRIPTION

The forwarding member group provides forwarding servers configuration for forward zones.

CONSTRUCTOR

```
 my $object = Infoblox::Grid::DNS::Nsgroup::ForwardingMember->new(
     name                  => $string,
#Required
     forwarding_servers    => [$dnsmember1, $dnsmember2, ...],
#Required
     comment               => $string,
#Optional / Default is undefined
     extattrs              => { $string => $extattr, ... },
#Optional / Default is undefined
     extensible_attributes => {$string => $string | $num, $string => [ $string | $num, ... ], ... }
#Optional / Default is undefined
 );
```

You cannot set both extattrs and extensible_attributes attributes at the same time.

SESSION METHODS

This section describes all the methods in an Infoblox::Session module that you can apply to a forwarding members group object.

Infoblox::Session->add( )

Use this method to add the object to the Infoblox appliance. See Infoblox::Session->add() for parameters and return values.

Example

```
#Construct an object
my $object = Infoblox::Grid::DNS::Nsgroup::ForwardingMember->new(
    name             => 'fwdgroup',
    forwarding_servers => [$dnsmember1, $dnsmember2],
);

#Submit for addition
my $response = $session->add($object);
```

Infoblox::Session->get( )

Use this method to retrieve all the matching objects from the Infoblox appliance. See Infoblox::Session->get() for parameters and return values.

Key Reference

```
Apply the following attributes to get a specific forwarding member group object:

 comment               - Optional. The forwarding member group comment in a string format.
 name                  - Optional. The forwarding member group name in a string format.
 extattrs              - Optional. A hash reference containing extensible attributes.
 extensible_attributes - Optional. A hash reference containing extensible attributes.
```

For more information about searching extensible attributes, see Infoblox::Grid::ExtensibleAttributeDef/Searching Extensible Attributes.

Example

```
my @retrieved_objs = $session->get(
    object => 'Infoblox::Grid::DNS::Nsgroup::ForwardingMember',
    name   => 'fwdgroup',
);

# search for objects defining 'Santa Clara' for 'Site' extensible attribute
my @retrieved_objs = $session->search(
    object               => 'Infoblox::Grid::DNS::Nsgroup::ForwardingMember',
    extensible_attributes => {'Site' => 'Santa Clara'},
);
```

Infoblox::Session->modify( )

Use this method to modify the object in the Infoblox appliance. See Infoblox::Session->modify() for parameters and return values.

Example

```
#Modify comment value
$object->comment('this is a modified comment');

#Submit modification
my $response = $session->modify($object);
```

Infoblox::Session->remove( )

Use this method to remove the object from the Infoblox appliance. See Infoblox::Session->remove() for parameters and return values.

To remove a specific object, first use `get()` or `search()` to retrieve the specific DHCP range object, and then submit this object for removal.

Example

```
#Get the objects
my @retrieved_objs = $session->get(
    object => 'Infoblox::Grid::DNS::Nsgroup::ForwardingMember',
    name   => 'fwdgroup',
);

#Find the desired object from the retrieved list
my $desired_object = $retrieved_objs[0];

#Submit for removal
my $response = $session->remove($desired_object);
```

Infoblox::Session->search( )

Use this method to search for objects in the Infoblox appliance. See Infoblox::Session->search() for parameters and return values.

Key References

```
Apply the following attributes to search for the forwarding member group object:

 comment               - Optional. The forwarding member group comment in a string format (regexp).
 name                  - Optional. The forwarding member group name in a string format (regexp).
 extattrs              - Optional. A hash reference containing extensible attributes.
 extensible_attributes - Optional. A hash reference containing extensible attributes.
```

For more information about searching extensible attributes, see Infoblox::Grid::ExtensibleAttributeDef/Searching Extensible Attributes.

Examples

```
# search for objects
my @retrieved_objs = $session->search(
    object  => 'Infoblox::Grid::DNS::Nsgroup::ForwardingMember',
```

```
    name    => '^fwd.*',
    comment => '.*comment',
);

# search for objects defining 'Santa Clara' for 'Site' extensible attribute
my @retrieved_objs = $session->search(
    object               => 'Infoblox::Grid::DNS::Nsgroup::ForwardingMember',
    extensible_attributes => {'Site' => 'Santa Clara'},
);
```

METHODS

This section describes all the methods that you can use to configure and retrieve the attribute values of a forwarding member group.

comment( )

Use this method to set or retrieve the forwarding member group comment.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

The valid value is a desired comment in a string format.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get comment value
my $comment = $object->comment();

#Modify comment value
$object->comment('fwd comment');
```

extattrs( )

Use this method to set or retrieve the extensible attributes associated with a forwarding member group object.

Parameter

Valid value is a hash reference that contains the names of extensible attributes and their associated values (Infoblox::Grid::Extattr objects).

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get extattrs value
my $extattrs = $object->extattrs();

#Modify extattrs value
$object->extattrs({'Site' => $extattr1, 'Administrator' => $extattr2});
```

extensible_attributes( )

Use this method to set or retrieve the extensible attributes associated with a forwarding member group object.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

For valid values for extensible attributes, see Infoblox::Grid::ExtensibleAttributeDef/Extensible Attribute Values.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get extensible attributes value
my $extensible_attributes = $object->extensible_attributes();
#Modify extensible attributes
$object->extensible_attributes({'Site' => 'Santa Clara', 'Administrator' => ['Peter', 'Tom']});
```

forwarding_servers( )

Use this method to set or retrieve the list of forwarding servers.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

The valid value is an array of Infoblox::DNS::Member objects.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get forwarding_servers value
my $forwarding_servers = $object->forwarding_servers();

#Modify forwarding_servers value

#Create nameserver object
my $nameserver = Infoblox::DNS::Nameserver->new(
    name     => 'ns1.domain1.extra',
    ipv4addr => '5.5.5.5',
);

#Create dns member object
my $dnsmember = Infoblox::DNS::Member->new(
    ipv4addr   => '1.1.1.1',
    forward_to => [$nameserver],
);

#Set forwarding_servers value
$object->forwarding_servers([$dnsmember]);
```

name( )

Use this method to set or retrieve the forwarding member group name.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

The valid value is a desired name in a string format.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get name value
my $name = $object->name();

#Modify name value
$object->name('fwdg1');
```

SAMPLE CODE

The following sample code demonstrates the different functions that can be applied to an object, such as modify and remove. This sample also includes error handling for the operations.

#Preparation prior to an Forwarding Member Nsgroup object insertion

```
#PROGRAM STARTS: Include all the modules that will be used
use strict;
```

```
use Infoblox;

my ($session, $result);

#Create a session to the Infoblox device
$session = Infoblox::Session->new(
    master   => "192.168.1.2",
    username => "admin",
    password => "infoblox"
);
unless ($session) {
   die("Construct session failed: ",
       Infoblox::status_code() . ":" . Infoblox::status_detail());
}
print "Session created successfully\n";
```

#Create an Forwarding Member Nsgroup object

```
#Creating an external nameserver
my $nameserver = Infoblox::DNS::Nameserver->new(
    name     => "ns1.domain0.external",
    ipv4addr => "5.5.5.5",
);

#Creating a forwarding member object.
my $fwd_member = Infoblox::DNS::Member->new(
    name       => "infoblox.localdomain",
    ipv4addr   => "192.168.1.2",
    forward_to =>  [$nameserver],
);

#Creating a forwarding nsgroup object.
my $fwd_nsg1 = Infoblox::Grid::DNS::Nsgroup::ForwardingMember->new(
    name               => "fwd_group_1",
    forwarding_servers => [$fwd_member],
);

unless ($fwd_nsg1) {
   die("Construct Forwarding Member Nsgroup failed: ",
       Infoblox::status_code() . ":" . Infoblox::status_detail());
}

print "Forwarding Member Nsgroup object constructed successfully\n";
```

#Add the Nsgroup object

```
$result = $session->add($fwd_nsg1);

unless ($result) {
    die("Add Forwarding Member Nsgroup to session failed: ",
        $session->status_code() . ":" . $session->status_detail());
}

print "Forwarding Member Nsgroup object created successfully\n";
```

#Search and Modify the Nsgroup object

```
my @result_array = $session->search(
    object => "Infoblox::Grid::DNS::Nsgroup::ForwardingMember",
    name   => "fwd_.*",
);

my $object = $result_array[0];

unless ($object) {
   die("Search for Forwarding Member Nsgroup failed: ",
       $session->status_code() . ":" . $session->status_detail());
}

$result = $object->comment("forwarding member");

unless ($result) {
    die("Modify Forwarding Member Nsgroup failed: ",
        Infoblox::status_code() . ":" . Infoblox::status_detail());
}

$result = $session->modify($object);
```

```
unless ($result) {
    die("Submit modification to the session failed: ",
        $session->status_code() . ":" . $session->status_detail());
}

print "Forwarding Member Nsgroup object modified successfully\n";
```

#Remove a Nsgroup object

```
my @result_array = $session->get(
    object => "Infoblox::Grid::DNS::Nsgroup::ForwardingMember",
    name   => "fwd_group_1",
);

my $object = $result_array[0];

unless ($object) {
   die("Get Forwarding Member Nsgroup failed: ",
       $session->status_code() . ":" . $session->status_detail());
}

$result = $session->remove($object);

unless ($result) {
    die("Remove Nsgroup failed: ",
        $session->status_code() . ":" . $session->status_detail());
}

print "Forwarding Member Nsgroup object removed successfully\n";

####PROGRAM ENDS####
```

AUTHOR

Infoblox Inc. http://www.infoblox.com/

SEE ALSO

Infoblox::Session, Infoblox::Session->add(), Infoblox::Session->get(), Infoblox::Session->modify(), Infoblox::Session->remove(), Infoblox::Session->search(), Infoblox::DNS::Member, Infoblox::Grid::ExtensibleAttributeDef/Extensible Attribute Values, Infoblox::Grid::Extattr

---

COPYRIGHT

Copyright (c) 2017 Infoblox Inc.

Infoblox::DNS::Member - DNS Member object

- NAME
- DESCRIPTION
- CONSTRUCTOR
- MODULE METHODS
  - Infoblox::DNS::Zone->multiple_primaries( )
  - Infoblox::DNS::Zone->secondary( )
  - Infoblox::Grid::DNS::Nsgroup->multiple_primaries( )
  - Infoblox::Grid::DNS::Nsgroup->secondary( )
- METHODS
  - forward_to( )
  - forwarders_only( )
  - override_forwarders( )
  - grid_replicate( )
  - ipv4addr( )
  - ipv6addr( )
  - lead( )
  - ms_parent_delegated( )
  - name( )
  - override_preferred_primaries( )
  - preferred_primaries( )
  - stealth( )
- SAMPLE CODE
- AUTHOR
- SEE ALSO
- COPYRIGHT

NAME

Infoblox::DNS::Member - DNS Member object

DESCRIPTION

A grid member is part of a grid which is a group of two or more Infoblox appliances that share sections of a common, distributed, built-in database. A grid member inherits its settings from the grid. A member is a single Infoblox appliance or an HA pair that provides DNS services seamlessly across an entire network.

CONSTRUCTOR

```
my $member = Infoblox::DNS::Member->new(
    ipv4addr       => $ipv4addr,        #Required if $ipv6addr is not present
    ipv6addr       => $ipv6addr,        #Required if $ipv4addr is not present
    grid_replicate => "true" | "false", #Optional / Default is "true"
    lead           => "true" | "false", #Optional / Default is "false"
    name           => $fqdn,            #Optional / Default is  hostname of the matching ipv4addr
    stealth        => "true" | "false"  #Optional / Default is "false"
    preferred_primaries          => [$pp1, $pp2, ...], #Optional / Default is undefined
    override_preferred_primaries => "true" | "false"   #Optional / Default is "false"
    forward_to          => [$ns1, $ns2, ...],  #Optional
    forwarders_only     => "true" | "false"    #Optional / Default is "false"
    override_forwarders => "true" | "false"    #Optional / Default is "false"
);
```

MODULE METHODS

The following functions are available to apply to an DNS Member object.

Infoblox::DNS::Zone->multiple_primaries( )

Use this function to specify primary members to a zone. See Infoblox::DNS::Zone->multiple_primaries() for parameters and return values.

Example

```
my $member1 = Infoblox::DNS::Member->new(
    name           => "localhost.localdomain",
    ipv4addr       => "192.168.1.2",
    stealth        => "false",
```

```
    lead           => "false",
    grid_replicate => "false"
);

unless($member1) {
     die("Construct member failed: ",
           Infoblox::status_code() . ":" . Infoblox::status_detail());
}
print "Member object created successfully\n";

#Create the Zone object with this member
my $firstzone = Infoblox::DNS::Zone->new(
    name        => "infoblox.com",
    multiple_primaries => [$member1],
);

my $response = $session->add($firstzone)
unless($response) {
     die("Add zone failed: ",
           session->status_code() . ":" . session->status_detail());
}
print "DNS member object added to the zone successfully\n";
```

Infoblox::DNS::Zone->secondary( )

Use this function to specify secondary member to a zone. See Infoblox::DNS::Zone->secondary() for parameters and return values.

Example

```
my $member1 = Infoblox::DNS::Member->new(
    name           => "localhost.localdomain",
    ipv4addr       => "192.168.1.2",
    stealth        => "false",
    lead           => "false",
    grid_replicate => "false"
);

unless($member1) {
     die("Construct member failed: ",
           Infoblox::status_code() . ":" . Infoblox::status_detail());
}
print "Member object created successfully\n";

my $member2 = Infoblox::DNS::Member->new(
    name           => "localhost1.localdomain",
    ipv4addr       => "192.168.1.3",
    stealth        => "false",
    lead           => "false",
    grid_replicate => "false"
);

unless($member2) {
     die("Construct member failed: ",
           Infoblox::status_code() . ":" . Infoblox::status_detail());
}
print "Member object created successfully\n";

#Create the Zone object with this member
my $firstzone = Infoblox::DNS::Zone->new(
    name        => "infoblox.com",
    multiple_primaries => [$member1],
    secondaries => [ $member2 ] ,
);

my $response = $session->add($firstzone)
unless($response) {
     die("Add zone failed: ",
           session->status_code() . ":" . session->status_detail());
}

print "DNS member object added to the zone successfully\n";
```

Infoblox::Grid::DNS::Nsgroup->multiple_primaries( )

Use this function to specify primary members to a NS group. See Infoblox::Grid::DNS::Nsgroup->multiple_primaries() for parameters and

return values.

Example

```
my $member1 = Infoblox::DNS::Member->new(
    name           => "localhost.localdomain",
    ipv4addr       => "192.168.1.2",
    stealth        => "false",
    lead           => "false",
    grid_replicate => "false"
);

unless($member1) {
     die("Construct member failed: ",
           Infoblox::status_code() . ":" . Infoblox::status_detail());
}
print "Member object created successfully\n";

my $nsgroup1 = Infoblox::Grid::DNS::Nsgroup->new (
   name      => "ns_group1",
   multiple_primaries => [$member1]
);
```

Infoblox::Grid::DNS::Nsgroup->secondary( )

Use this function to specify secondary member to a NS group. See Infoblox::Grid::DNS::Nsgroup->secondary() for parameters and return values.

Example

```
my $member1 = Infoblox::DNS::Member->new(
    name           => "localhost.localdomain",
    ipv4addr       => "192.168.1.2",
    stealth        => "false",
    lead           => "false",
    grid_replicate => "false"
);

unless($member1) {
     die("Construct member failed: ",
           Infoblox::status_code() . ":" . Infoblox::status_detail());
}
print "Member object created successfully\n";

my $member2 = Infoblox::DNS::Member->new(
    name           => "localhost1.localdomain",
    ipv4addr       => "192.168.1.3",
    stealth        => "false",
    lead           => "false",
    grid_replicate => "false"
);

unless($member2) {
     die("Construct member failed: ",
           Infoblox::status_code() . ":" . Infoblox::status_detail());
}
print "Member object created successfully\n";

my $nsgroup1 = Infoblox::Grid::DNS::Nsgroup->new (
   name        => "ns_group1",
   multiple_primaries => [$member1],
   secondaries => [ $member2 ],
);
```

METHODS

This section describes all the methods that you can use to set and retrieve the attribute values of a DNS Member object.

forward_to( )

Use this method to set or retrieve a list of DNS forwarders. You can use this attribute only for DNS members in forward-mapping zones.

Parameter

The valid value is an array reference that contains Infoblox::DNS::Nameserver objects.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get forward_to
my $forward_to = $member->forward_to();
#Modify forward_to
my $nameserver = Infoblox::DNS::Nameserver->new(
        name     => "ns1.forwd.com",
        ipv4addr => "11.11.11.11",
   );

$zone->forward_to([$nameserver]);
```

forwarders_only( )

Use this method to set the forwarders_only flag. You can use this attribute only for DNS members in forward-mapping zones.

Parameter

Specify "true" to use only DNS forwarders or "false" to disable it. The default value is 'false'.

The server writes 'forward only' into the DNS configuration file for the DNS member.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get forwarders_only
my $forwarders_only = $member->forwarders_only();
#Modify forwarders_only
$member->forwarders_only("true");
```

override_forwarders( )

Use this method to set the override_forwarders flag. You can use this attribute only for DNS members in forward-mapping zones.

Parameter

Specify "true" to override DNS forwarders or "false" to use zone default DNS forwarders. The default value is 'false'.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get override_forwarders
my $override_forwarders = $member->override_forwarders();
#Modify override_forwarders
$member->override_forwarders("true");
```

grid_replicate( )

Use this method to set or retrieve the grid_replicate flag, which indicates whether grid members that are primary and secondary servers use database replication or zone transfers for zone updates.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

Specify "true" to use ID grid replication or "false" to use DNS zone transfers.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get grid replicate
my $grid_replicate = $member->grid_replicate();
#Modify grid replicate
$member->grid_replicate("true");
```

ipv4addr( )

Use this method to set or retrieve the IPv4 address of the member.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

IPv4 address (32 bits) of the member.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get ipv4Addr
my $ipv4Addr = $member->ipv4Addr();
#Modify ipv4Addr
$member->ipv4Addr("192.168.1.3");
```

ipv6addr( )

Use this method to set or retrieve the IPv6 address if available. Required only when ipv4 is not specified.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

An IPv6 address is a 128-bit number in colon hexadecimal notation. It consists of eight groups of four hexadecimal digits separated by colons (example: 12ab:0000:0000:0123:4567:89ab:0000:cdef).

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
# Getting ipv6addr
my $ipv6addr = $member->ipv6addr();
# Modifying ipv6addr
$member->ipv6addr("12ab::345");
```

lead( )

Use this method to set or retrieve the "lead" flag.

When a primary server is external to a grid whose members are secondary servers, you can designate one member as a lead secondary. The primary server sends zone transfers to the lead secondary, which distributes the zone data to the other secondary servers in the grid using zone transfers (not the ID grid data replication mechanism).

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

Specify "true" to set lead flag or "false" to deactivate/unset it.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get lead
```

```
my $lead = $member->lead();
#Modify lead
$member->lead("true");
```

ms_parent_delegated( )

Use this method to retrieve the ms_parent_delegated flag. If the zone is a delegation and the primary server of the parent zone is a Microsoft (r) server, this flag is set to "true" if the FQDN and IP address in the delegation zone's NS record matches the FQDN and IP address of its authoritative name server.

Parameter

None.

Returns

The method returns the attribute value.

Example

```
#Get ms_parent_delegated
my $ms_parent_delegated = $member->ms_parent_delegated();
```

name( )

Use this method to set or retrieve the name of the member.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

A name in FQDN (Fully Qualified Domain Name) format. The FQDN consists of the host name followed by the domain name (example: host.domain.com). A host name can have a maximum of 256 characters.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get name
my $name = $member->name();
#Modify name
$member->name("dns.infoblox.com");
```

override_preferred_primaries( )

Use this method to set or retrieve the override_preferred_primaries attribute that controls whether the preferred_primaries method values of the member are used.

Parameter

Set the parameter to "true" to use preferred_primaries method values. Set the parameter to "false", and a primary preference list is determined automatically. The default value is "false".

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
# Get the attribute value
my $value = $member->override_preferred_primaries();
# Modify the attribute value
$member->override_preferred_primaries("false");
```

preferred_primaries( )

Use this method to set or retrieve a primary preference list for the member.

Parameter

A reference to an array of Grid member names only or Infoblox::DNS::Nameserver objects.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
# Get the attribute value
$value = $member->preferred_primaries();
# Modify the attribute value
$member->preferred_primaries(['test.com', 'example.com']);
$member->preferred_primaries([$nameserver1, $nameserver2]);
```

stealth( )

Use this method to set or retrieve the "stealth" flag, which indicates whether this member is in stealth mode. When a member is in stealth mode, its NS record is hidden from DNS queries; it does not respond to queries from resolvers and other name servers.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

Specify "true" to set the stealth flag or "false" to deactivate/unset it.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get stealth
my $stealth = $member->stealth();
#Modify stealth
$member->stealth("true");
```

SAMPLE CODE

The following sample code demonstrates the different functions that can be applied to an object, such as add, search, modify, and remove. This sample also includes error handling for the operations.

#Preparation prior to a DNS Member object insertion

```
#PROGRAM STARTS: Include all the modules that will be used
use strict;
use Infoblox;

#Create a session to the Infoblox appliance

my $host_ip   = "192.168.1.2";
my $host_name = "infoblox.localdomain";

my $session = Infoblox::Session->new(
               master   => $host_ip, #appliance host ip
               username => "admin",       #appliance user login
               password => "infoblox"     #appliance password
);

unless ($session) {
       die("Construct session failed: ",
               Infoblox::status_code() . ":" . Infoblox::status_detail());
}
print "Session created successfully\n";
```

#Create a DNS Member object

```
my $member1 = Infoblox::DNS::Member->new(
    name           => $host_name,
    ipv4addr       => $host_ip,
    stealth        => "false",
    lead           => "false",
    grid_replicate => "false"
);

unless($member1) {
```

```
      die("Construct member failed: ",
            Infoblox::status_code() . ":" . Infoblox::status_detail());
}
print "Member object created successfully\n";

my $default_view = Infoblox::DNS::View->new(name => "default");

#Create the Zone object with this member
my $firstzone = Infoblox::DNS::Zone->new(
    name         => "infoblox.com",
    views        => [$default_view],
    email        => "admin\@infoblox.com",
    comment      => "add a zone infoblox.com",
    multiple_primaries => [$member1],
);

unless($firstzone) {
            die("Construct Zone object failed: ",
                  Infoblox::status_code() . ":" . Infoblox::status_detail());
}
print "Zone object created successfully\n";

#Add the Zone object into the Infoblox appliance through a session
$session->add($firstzone)
     or die("Add Zone object failed: ",
           $session->status_code() . ":" . $session->status_detail());
print "DNS member object added to the zone successfully\n";

#Modify a DNS member

my @result = $session->search(object => "Infoblox::DNS::Zone",
                              name   => "infoblox.com");

 unless (scalar(@result) == 0) {
     $firstzone = $result[0];

     if ($firstzone) {

          #modify grid replicate value of member
          $member1->grid_replicate("true");

          #update zone
          $firstzone->multiple_primaries([$member1]);

          #update session
          $session->modify($firstzone)
                 or die("Modify zone object failed: ",
                        $session->status_code() . ":" . $session->status_detail());
          print "DNS member object updated and added to the zone successfully\n";
                }
            } else {
                print "No zone found with the specified name";
            }

####PROGRAM ENDS####
```

AUTHOR

Infoblox Inc. http://www.infoblox.com/

SEE ALSO

Infoblox::DNS::TSIGKey, Infoblox::Session->add(),Infoblox::Session->modify(),Infoblox::Session->search(),Infoblox::Session, Infoblox::DNS::Nameserver

COPYRIGHT

Copyright (c) 2017 Infoblox Inc.

Infoblox::DNS::OrderedResponsePolicyZones - Ordered Response Policy Zones object.

- NAME
- DESCRIPTION
- CONSTRUCTOR
- SESSION METHODS
  - Infoblox::Session->add( )
  - Infoblox::Session->get( )
  - Infoblox::Session->modify( )
- METHODS
  - view( )
  - rp_zones( )
- SAMPLE CODE
- AUTHOR
- SEE ALSO
- COPYRIGHT

NAME

Infoblox::DNS::OrderedResponsePolicyZones - Ordered Response Policy Zones object.

DESCRIPTION

An object type to set or retrieve an ordered list of Response Policy Zones in a DNS view.

Server will reject zones that are disabled or zones without primary name server assigned.

CONSTRUCTOR

```
 my $ordered_rpz = Infoblox::DNS::OrderedResponsePolicyZones->new(
                        view                    => $view,
#Required
                        rp_zones                => [$response_policy_zone1, $response_policy_zone2, ...],
#Required
 );
```

SESSION METHODS

This section describes all the methods in the Infoblox::Session module that you can apply to an Ordered Response Policy Zones object.

Infoblox::Session->add( )

Use this method to add an object to the Infoblox appliance. See Infoblox::Session->add() for parameters and return values.

No object is added in this method, but the Response Policy Zones for the specified DNS View assume the specified order.

Example

```
       #Construct an object
       my $ordered_rpz = Infoblox::DNS::OrderedResponsePolicyZones->new(
                                                                  view => $view,
                                                                  rp_zones  => [$response_policy_zone1,
      $response_policy_zone2]
                                                                 );
       #Submit for addition
       my $response = $session->add( $ordered_rpz );
```

Infoblox::Session->get( )

Use this method to retrieve all the matching objects from the Infoblox appliance. See Infoblox::Session->get() for parameters and return values.

Key References

```
       Apply the following attributes to get a specific Response Policy Zones object:

       view - Required. An Infoblox::DNS::View object.
```

Examples

```
       my @retrieved_objs = $session->get(
```

```
        object   => "Infoblox::DNS::OrderedResponsePolicyZones",
        view  => $view
    );
```

Infoblox::Session->modify( )

Use this method to modify an object in the Infoblox appliance. See Infoblox::Session->modify() for parameters and return values.

No object is modified in this method, but the Response Policy Zones for the specified DNS View assume the specified order.

Example

```
#Use this method to modify the order of Response Policy Zones.
$ordered_rpz->rp_zones([$rpz4, $rpz2, $rpz3, $rpz1]);
#Submit modification
my $response = $session->modify( $ordered_rpz );
```

METHODS

Use the following methods to access the attributes of an Infoblox::DNS::OrderedResponsePolicyZones object. Specify a parameter to set the attribute's value, or omit the parameter to get the attribute's value.

view( )

Use this method to set or retrieve the DNS View object.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

An Infoblox::DNS::View object.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Getting view
my $view = $ordered_rpz->view();
#Modifying view
$ordered_rpz->view($view1);
```

rp_zones( )

Use this method to set or retrieve an ordered list of Response Policy Zones object references.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

An array reference of defined Infoblox::DNS::Zone objects.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Getting zones list reference
my $zone_list_ref = $ordered_rpz->rp_zones();
#Getting zones list
my @zone_list = @{$oz->rp_zones()};
#Get Zone from list
my $zone = $zone_list[0];
#Modifying zones
$ordered_rpz->rp_zones([$rpz1, $rpz2, @rpz3]);
```

SAMPLE CODE

The following sample code demonstrates the different functions that can be applied to an object, such as add, search, modify, and remove. This sample also includes error handling for the operations.

#Preparation prior to the Response Policy Zone reordering

```
#PROGRAM STARTS: Include all the modules that will be used
use strict;
use Infoblox;

#Create a session to the Infoblox appliance
my $host_ip =  "192.168.1.2";
my $session = Infoblox::Session->new(
    master   => $host_ip,
    username => "admin",
    password => "infoblox"
    );
unless($session){
        die("Constructor for session failed: ",
               Infoblox::status_code(). ":" . Infoblox::status_detail());
}
print "Session created successfully.\n";

my $memberns1 = Infoblox::DNS::Member->new(
   name     => $host_name,
   ipv4addr => $node_ip,
);

my $rpz_zone = Infoblox::DNS::Zone->new (
   name     => 'rpz.com',
   comment  => "this is a demo zone 1",
   disable  => "false",
   rpz_policy => "GIVEN",
   primary => $memberns1,
);

$session->add($rpz_zone)
   or die("Add zone failed: ",
              $session->status_code(). ":" .$session->status_detail());

print"Response policy zone added successfully.\n";

$rpz_zone = Infoblox::DNS::Zone->new (
   name     => 'rpz1.com',
   comment  => "this is a demo zone 2",
   disable  => "false",
   rpz_policy => "GIVEN",
   primary => $memberns1,
);

$session->add($rpz_zone)
   or die("Add zone failed: ",
              $session->status_code(). ":" .$session->status_detail());

print"Response policy zone added successfully.\n";

$rpz_zone = Infoblox::DNS::Zone->new (
   name     => 'rpz2.com',
   comment  => "this is a demo zone 3",
   disable  => "false",
   rpz_policy => "GIVEN",
   primary => $memberns1,
);

$session->add($rpz_zone)
   or die("Add zone failed: ",
              $session->status_code(). ":" .$session->status_detail());

print"Response policy zone added successfully.\n";
```

#Get the current Response Policy Zones order.

```
my $orpz = $session->get(
    object  => "Infoblox::DNS::OrderedResponsePolicyZones",
    view => "default",
);

unless($orpz){
      die("Get Response Policy Zones failed: ",
               $session->status_code() . ":" . $session->status_detail());
```

```
      }

print "Get Response Policy Zones object found at least 1 matching entry\n";

my ($z1, $z2, $z3) = @{$orpz->rp_zones()};

print "The current order of the Response Policy Zones is : " . $z1->comment() . "," .
              $z2->comment() . "," . $z3->comment() . "\n";
```

#Modify the Response Policy Zones order.

```
$orpz->rp_zones([$z3, $z2, $z1]);

#Apply the change
$session->modify($orpz)
     or die("Modify Response Policy Zones failed: ",
              $session->status_code(). ":" .$session->status_detail());

print "Response Policy Zones modified successfully.\n";

$orpz = $session->get(
   object  => "Infoblox::DNS::OrderedResponsePolicyZones",
   view => "default",
);

unless($orpz){
     die("Get Response Policy Zones failed: ",
              $session->status_code() . ":" . $session->status_detail());
     }

print "Get Response Policy Zones object found at least 1 matching entry\n";

($z1, $z2, $z3) = @{$orpz->rp_zones()};

print "The current order of the Response Policy Zones is : " . $z1->comment() . "," .
              $z2->comment() . "," . $z3->comment() . "\n";

####PROGRAM ENDS####
```

AUTHOR

Infoblox Inc. http://www.infoblox.com

SEE ALSO

Infoblox::DNS::View, Infoblox::DNS::Zone, Infoblox::Session, Infoblox::Session->add(), Infoblox::Session->get(), Infoblox::Session->modify()

COPYRIGHT

Copyright (c) 2017 Infoblox Inc.

Infoblox::DNS::Member::SoaSerial - a per-master SOA serial information.

- NAME
- DESCRIPTION
- CONSTRUCTOR
- SESSION METHODS
- METHODS
  - grid_member( )
  - serial( )
  - ms_server
- AUTHOR
- SEE ALSO
- COPYRIGHT

NAME

Infoblox::DNS::Member::SoaSerial - a per-master SOA serial information.

DESCRIPTION

The object contains a per-master SOA serial information.

CONSTRUCTOR

The object cannot be created because all object attributes are read-only.

SESSION METHODS

The object doesn't support any session methods.

METHODS

This section describes all the methods that you can use to configure and retrieve the attribute values of the object.

grid_member( )

Use this method to retrieve the primary server name for the zone. This is a read-only attribute.

Parameter

None.

Returns

The method returns the attribute value.

Example

```
# Get the attribute value
my $attr = $soa_serial->grid_member();
```

serial( )

Use this method to retrieve the SOA serial number. This is a read-only attribute.

Parameter

None.

Returns

The method returns the attribute value.

Example

```
# Get the attribute value
my $attr = $soa_serial->serial();
```

ms_server

Use this method to retrieve the IPv4 address or FQDN of the Microsoft (r) Server for the zone. This is a read-only attribute.

Parameter

None.

Returns

The method returns the attribute value.

Example

```
# Get the attribute value
my $attr = $soa_serial->ms_server();
```

AUTHOR

Infoblox Inc. http://www.infoblox.com/

SEE ALSO

Infoblox::DNS::Zone

COPYRIGHT

Copyright (c) 2017 Infoblox Inc.

Infoblox::DNS::Member::SoaMname - a per-master SOA MNAME information.

- NAME
- DESCRIPTION
- CONSTRUCTOR
- SESSION METHODS
- METHODS
  - dns_mname( )
  - grid_member( )
  - mname( )
  - ms_server( )
- AUTHOR
- SEE ALSO
- COPYRIGHT

NAME

Infoblox::DNS::Member::SoaMname - a per-master SOA MNAME information.

DESCRIPTION

The object contains a per-master SOA MNAME Information.

CONSTRUCTOR

```
my $soa_mname = Infoblox::DNS::Member::SoaMname->new(
   grid_member => $string,       # Required if ms_server is not specified
   ms_server   => $string,       # Required if grid_member is not specified
   mname       => $string,       # Optional
);
```

You cannot set both grid_member and ms_server attributes at the same time.

SESSION METHODS

The object doesn't support any session methods.

METHODS

This section describes all the methods that you can use to configure and retrieve the attribute values of the object.

dns_mname( )

Use this method to retrieve the master's SOA MNAME in punycode format. This is a read-only attribute.

Parameter

None

Returns

The method returns the attribute value.

Example

```
# Get attribute value
my $value = $soa_mname->dns_mname();
```

grid_member( )

Use this method to set or retrieve the primary server name for the zone.

Parameter

The primary server name in string format.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
# Get the attribute value
my $attr = $soa_mname->grid_member();
# Modify the attribute value
$soa_mname->grid_member('member.com');
```

mname( )

Use this method to set or retrieve the master's SOA MNAME.

Parameter

The master's SOA MNAME in string format.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
# Get the attribute value
my $attr = $soa_mname->mname();
# Modify the attribute value
$soa_mname->mname('dummy-mname.com');
```

ms_server( )

Use this method to set or retrieve the IPv4 address or FQDN of the Microsoft (r) Server for the zone.

Parameter

Valid value is an IPv4 address (32 bits) or FQDN of the Microsoft (r) Server.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
# Get the attribute value
my $attr = $soa_mname->ms_server();
# Modify the attribute value
$soa_mname->ms_server('10.0.0.1');
```

AUTHOR

Infoblox Inc. http://www.infoblox.com/

SEE ALSO

Infoblox::DNS::Zone

COPYRIGHT

Copyright (c) 2017 Infoblox Inc.

Infoblox::Grid::MSServer::DCNSRecordCreation - An Infoblox Active Directory Domain Controller object

- NAME
- DESCRIPTION
- CONSTRUCTOR
- SESSION METHODS
- MODULE METHODS
  - Infoblox::DNS::Zone->ms_dc_ns_record_creation( )
- METHODS
  - address( )
  - comment( )
- AUTHOR
- SEE ALSO
- COPYRIGHT

NAME

Infoblox::Grid::MSServer::DCNSRecordCreation - An Infoblox Active Directory Domain Controller object

---

DESCRIPTION

The Active Directory Domain Controller object represents the Active Directory Domain Controller which is allowed for NS Record creation in a DNS Zone.

CONSTRUCTOR

```
my $dcncrecordcreation = Infoblox::Grid::MSServer::DCNSRecordCreation->new(
    address => $ipv4addr | $ipv6addr, #Required
    comment => $string,               #Optional / Default is undefined
);
```

SESSION METHODS

The object does not support any session methods. It is retrieved as a part of the DNS Zone object. It also can be retrieved as a result of Infoblox::Session->get_ms_dc_ns_record_creation_list() function.

MODULE METHODS

The following functions can be applied to a AD Domain Controller object.

Infoblox::DNS::Zone->ms_dc_ns_record_creation( )

Use this method to set or retrieve the list of Active Directory Domain Controllers which are allowed to create NS Record for the DNS Zone. See Infoblox::DNS::Zone->ms_dc_ns_record_creation() for parameters and return values.

Example

```
#Get ms_dc_ns_record_creation
my $ms_dc_ns_record_creation = $zone->ms_dc_ns_record_creation();

#Modify ms_dc_ns_record_creation
$zone->ms_dc_ns_record_creation([$msdcns1, $msdcns1]);
```

METHODS

this section describes all the methods that you can use to set or retrieve the attribute values of the object.

address( )

Use this method to set or retrieve the address of a Domain Controller.

Parameter

The valid value is a desired IPv4 or IPv6 address in a string format.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get address value
my $address = $object->address();

#Set address value
$object->address('10.0.0.1');
```

comment( )

Use this method to set or retrieve the descriptive comment.

Parameter

The valid value is a desired comment in a string format.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get comment value
my $address = $object->comment();

#Set comment value
$object->comment('desired-comment');
```

AUTHOR

Infoblox Inc. http://www.infoblox.com/

SEE ALSO

Infoblox::DNS::Zone->ms_dc_ns_record_creation(), Infoblox::Session->get_ms_dc_ns_record_creation_list()

COPYRIGHT

Copyright (c) 2017 Infoblox Inc.

Infoblox::DNS::MSServer - Microsoft DNS Server object

- NAME
- DESCRIPTION
- CONSTRUCTOR
- MODULE METHODS
  - Infoblox::DNS::Zone->multiple_primaries( )
  - Infoblox::DNS::Zone->secondaries( )
  - Infoblox::DNS::Zone->stub_members( )
- METHODS
  - address( )
  - ipv4addr( )
  - ms_parent_delegated( )
  - name( )
  - stealth( )
- AUTHOR
- SEE ALSO
- COPYRIGHT

NAME

Infoblox::DNS::MSServer - Microsoft (r) DNS Server object

DESCRIPTION

A Microsoft DNS Server object is a Windows-based server that is running the DNS service and is synchronized with an Infoblox grid. As such, the Microsoft DNS server shares its DNS data with the Infoblox DNS servers in a grid.

CONSTRUCTOR

```
my $msserver = Infoblox::DNS::MSServer->new(
    address        => $string,           #Required
    ipv4addr       => $string,           #Required
    name           => $string,           #Required
    stealth        => "true" | "false",  #Optional
);
```

MODULE METHODS

The following functions can be applied to a Microsoft DNS Server object.

Infoblox::DNS::Zone->multiple_primaries( )

Use any of these functions to assign a Microsoft DNS server as the primary server of a zone. See Infoblox::DNS::Zone->multiple_primaries() for parameters and return values.

Example

```
my $msserver1 = Infoblox::DNS::MSServer->new(
    address        => "10.0.0.1",
    ipv4addr       => "10.0.0.1",
    name           => "hostname.com",
);

unless($msserver1) {
     die("Construct msserver failed: ",
           Infoblox::status_code() . ":" . Infoblox::status_detail());
}
print "Msserver object created successfully\n";

#Create the Zone object with this Microsoft DNS server
my $firstzone = Infoblox::DNS::Zone->new(
    name        => "infoblox.com",
    multiple_primaries => [$msserver1],
);

my $response = $session->add($firstzone)
unless($response) {
     die("Add zone failed: ",
```

```
            session->status_code() . ":" . session->status_detail());
}
print "DNS MS Server object added to the zone successfully\n";
```

Infoblox::DNS::Zone->secondaries( )

Use this function to specify an Microsoft DNS server as the secondary server to a zone. See Infoblox::DNS::Zone->secondaries() for parameters and return values.

Example

```
my $msserver1 = Infoblox::DNS::MSServer->new(
    address        => "10.0.0.1",
    ipv4addr       => "10.0.0.1",
    name           => "hostname.com",
);

unless($msserver1) {
     die("Construct msserver failed: ",
           Infoblox::status_code() . ":" . Infoblox::status_detail());
}
print "Msserver object created successfully\n";

#Create the Zone object with this member
my $firstzone = Infoblox::DNS::Zone->new(
    name        => "infoblox.com",
    multiple_primaries => [$member1],
    secondaries => [ $msserver2 ] ,
);

my $response = $session->add($firstzone)
unless($response) {
     die("Add zone failed: ",
           session->status_code() . ":" . session->status_detail());
}

print "DNS MS server member object added to the zone successfully\n";
```

Infoblox::DNS::Zone->stub_members( )

Use this function to assign a Microsoft DNS server to a stub zone. See Infoblox::DNS::Zone->stub_members() for parameters and return values.

Example

```
my $msserver1 = Infoblox::DNS::MSServer->new(
    address        => "10.0.0.1",
    ipv4addr       => "10.0.0.1",
    name           => "hostname.com",
);

unless($msserver1) {
     die("Construct msserver failed: ",
           Infoblox::status_code() . ":" . Infoblox::status_detail());
}
print "Msserver object created successfully\n";

#Create the Zone object with this member
my $firstzone = Infoblox::DNS::Zone->new(
    name        => "infoblox.com",
    stub_members => [ $msserver2 ] ,
);

my $response = $session->add($firstzone)
unless($response) {
     die("Add zone failed: ",
           session->status_code() . ":" . session->status_detail());
}

print "DNS MS server member object added to the zone successfully\n";
```

METHODS

This section describes all the methods that you can use to set and retrieve the attribute values of a DNS Member object.

address( )

Use this method to set or retrieve the IPv4 address or FQDN of the member.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

IPv4 address (32 bits) or FQDN of the member.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get address
my $address = $member->address();
#Modify address
$member->address("192.168.1.3");
```

ipv4addr( )

Use this method to set or retrieve the member IPv4 address that is published in the glue A record when setting this Microsoft DNS server as a name server.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

IPv4 address (32 bits) of the member.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get ipv4addr
my $ipv4addr = $member->ipv4addr();
#Modify ipv4addr
$member->ipv4addr("192.168.1.3");
```

ms_parent_delegated( )

Use this method to retrieve the ms_parent_delegated flag. If the zone is a delegation and the primary server of the parent zone is a Microsoft server, this flag is set to "true" if the FQDN and IP address in the delegation zone's NS record matches the FQDN and IP address of its authoritative name server.

Parameter

None.

Returns

The method returns the attribute value.

Example

```
#Get ms_parent_delegated
my $ms_parent_delegated = $member->ms_parent_delegated();
```

name( )

Use this method to set or retrieve the member FQDN that is used when setting this Microsoft DNS server as a name server.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

The member's FQDN.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get name
my $name = $member->name();
#Modify name
$member->name("hostname.com");
```

stealth( )

Use this method to set or retrieve the "stealth" flag, which indicates whether this name server is in stealth or normal mode.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

Specify "true" to set the stealth flag or "false" to deactivate/unset it.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get stealth
my $stealth = $member->stealth();
#Modify stealth
$member->stealth("true");
```

AUTHOR

Infoblox Inc. http://www.infoblox.com/

SEE ALSO

Infoblox::DNS::TSIGKey, Infoblox::Session->add(),Infoblox::Session->modify(),Infoblox::Session->search(),Infoblox::Session

---

COPYRIGHT

Copyright (c) 2017 Infoblox Inc.

Infoblox::Grid::DNS::Nsgroup::StubMember - A stub member group.

- NAME
- DESCRIPTION
- CONSTRUCTOR
- SESSION METHODS
  - Infoblox::Session->add( )
  - Infoblox::Session->get( )
  - Infoblox::Session->modify( )
  - Infoblox::Session->remove( )
  - Infoblox::Session->search( )
- METHODS
  - comment( )
  - extattrs( )
  - extensible_attributes( )
  - stub_members( )
  - name( )
- SAMPLE CODE
- AUTHOR
- SEE ALSO
- COPYRIGHT

NAME

Infoblox::Grid::DNS::Nsgroup::StubMember - A stub member group.

DESCRIPTION

The stub member group provides stub servers configuration for stub zones.

CONSTRUCTOR

```
 my $object = Infoblox::Grid::DNS::Nsgroup::StubMember->new(
     name                  => $string,
#Required
     stub_members          => [$grid_member1, $grid_member2, ...],
#Required
     comment               => $string,
#Optional / Default is undefined
     extattrs              => { $string => $extattr, ... },
#Optional / Default is undefined
     extensible_attributes => {$string => $string | $num, $string => [ $string | $num, ... ], ... }
#Optional / Default is undefined
 );
```

You cannot set both extattrs and extensible_attributes attributes at the same time.

SESSION METHODS

This section describes all the methods in an Infoblox::Session module that you can apply to a stub members group object.

Infoblox::Session->add( )

Use this method to add the object to the Infoblox appliance. See Infoblox::Session->add() for parameters and return values.

Example

```
#Construct an object
my $object = Infoblox::Grid::DNS::Nsgroup::StubMember->new(
    name         => 'stubgroup',
    stub_members => ['member1.com', 'member2.com'],
);

#Submit for addition
my $response = $session->add($object);
```

Infoblox::Session->get( )

Use this method to retrieve all the matching objects from the Infoblox appliance. See Infoblox::Session->get() for parameters and return values.

Key Reference

```
Apply the following attributes to get a specific stub member group object:

 comment               - Optional. The stub member group comment in a string format.
 name                  - Optional. The stub member group name in a string format.
 extattrs              - Optional. A hash reference containing extensible attributes.
 extensible_attributes - Optional. A hash reference containing extensible attributes.
```

For more information about searching extensible attributes, see Infoblox::Grid::ExtensibleAttributeDef/Searching Extensible Attributes.

Example

```
my @retrieved_objs = $session->get(
    object => 'Infoblox::Grid::DNS::Nsgroup::StubMember',
    name   => 'stubgroup',
);

my @retrieved_objs = $session->get(
    object               => 'Infoblox::Grid::DNS::Nsgroup::StubMember',
    extensible_attributes => {'Site' => 'Santa Clara'},
);
```

Infoblox::Session->modify( )

Use this method to modify the object in the Infoblox appliance. See Infoblox::Session->modify() for parameters and return values.

Example

```
#Modify comment value
$object->comment('this is a modified comment');

#Submit modification
my $response = $session->modify($object);
```

Infoblox::Session->remove( )

Use this method to remove the object from the Infoblox appliance. See Infoblox::Session->remove() for parameters and return values.

To remove a specific object, first use `get()` or `search()` to retrieve the specific DHCP range object, and then submit this object for removal.

Example

```
#Get the objects
my @retrieved_objs = $session->get(
    object => 'Infoblox::Grid::DNS::Nsgroup::StubMember',
    name   => 'stubgroup',
);

#Find the desired object from the retrieved list
my $desired_object = $retrieved_objs[0];

#Submit for removal
my $response = $session->remove($desired_object);
```

Infoblox::Session->search( )

Use this method to search for objects in the Infoblox appliance. See Infoblox::Session->search() for parameters and return values.

Key Reference

```
Apply the following attributes to search for the stub member group object:

 comment               - Optional. The stub member group comment in a string format (regexp).
 name                  - Optional. The stub member group name in a string format (regexp).
 extattrs              - Optional. A hash reference containing extensible attributes.
 extensible_attributes - Optional. A hash reference containing extensible attributes.
```

For more information about searching extensible attributes, see Infoblox::Grid::ExtensibleAttributeDef/Searching Extensible Attributes.

Example

```
# search for objects
my @retrieved_objs = $session->search(
    object  => 'Infoblox::Grid::DNS::Nsgroup::StubMember',
    name    => '^stub.*',
```

```
    comment => '.*comment',
);

# search for objects defining 'Santa Clara' for 'Site' extensible attribute
my @retrieved_objs = $session->search(
    object               => 'Infoblox::Grid::DNS::Nsgroup::StubMember',
    extensible_attributes => {'Site' => 'Santa Clara'},
);
```

METHODS

This section describes all the methods that you can use to configure and retrieve the attribute values of a stub member group.

comment( )

Use this method to set or retrieve the stub member group comment.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

The valid value is a desired comment in a string format.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get comment value
my $comment = $object->comment();

#Modify comment value
$object->comment('stub comment');
```

extattrs( )

Use this method to set or retrieve the extensible attributes associated with a stub member group object.

Parameter

Valid value is a hash reference that contains the names of extensible attributes and their associated values (Infoblox::Grid::Extattr objects).

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get extattrs value
my $extattrs = $object->extattrs();

#Modify extattrs value
$object->extattrs({'Site' => $extattr1, 'Administrator' => $extattr2});
```

extensible_attributes( )

Use this method to set or retrieve the extensible attributes associated with a stub member group object.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

For valid values for extensible attributes, see Infoblox::Grid::ExtensibleAttributeDef/Extensible Attribute Values.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get extensible attributes value
my $extensible_attributes = $object->extensible_attributes();
#Modify extensible attributes
$object->extensible_attributes({'Site' => 'Santa Clara', 'Administrator' => ['Peter', 'Tom']});
```

stub_members( )

Use this method to set or retrieve the list of stub members.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

The valid value is an array of Grid Member host names in a string format.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get stub_members value
my $stub_members = $object->stub_members();

#Modify stub_members value

#Set stub_members value
$object->stub_members(['member1.com', 'member2.com']);
```

name( )

Use this method to set or retrieve the stub member group name.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

The valid value is a desired name in a string format.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get name value
my $name = $object->name();

#Modify name value
$object->name('stubg1');
```

SAMPLE CODE

The following sample code demonstrates the different functions that can be applied to an object, such as modify and remove. This sample also includes error handling for the operations.

#Preparation prior to an Stub Member Nsgroup object insertion

```
#PROGRAM STARTS: Include all the modules that will be used
use strict;
use Infoblox;

my ($session, $result);

#Create a session to the Infoblox device
$session = Infoblox::Session->new(
    master   => "192.168.1.2",
    username => "admin",
    password => "infoblox"
);
unless ($session) {
   die("Construct session failed: ",
       Infoblox::status_code() . ":" . Infoblox::status_detail());
}
```

```
print "Session created successfully\n";
```

#Create an Stub Member Nsgroup object

```
#Creating a forwarding nsgroup object.
my $stub_nsg1 = Infoblox::Grid::DNS::Nsgroup::StubMember->new(
    name         => "stub_group_1",
    stub_members => ["infoblox.localdomain"],
);

unless ($stub_nsg1) {
   die("Construct Stub Member Nsgroup failed: ",
       Infoblox::status_code() . ":" . Infoblox::status_detail());
}

print "Stub Member Nsgroup object constructed successfully\n";
```

#Add the Nsgroup object

```
$result = $session->add($stub_nsg1);

unless ($result) {
    die("Add Stub Member Nsgroup to session failed: ",
        $session->status_code() . ":" . $session->status_detail());
}

print "Stub Member Nsgroup object created successfully\n";
```

#Search and Modify the Nsgroup object

```
my @result_array = $session->search(
    object => "Infoblox::Grid::DNS::Nsgroup::StubMember",
    name   => "stub_.*",
);

my $object = $result_array[0];

unless ($object) {
   die("Search for Stub Member Nsgroup failed: ",
       $session->status_code() . ":" . $session->status_detail());
}

$result = $object->comment("stub member");

unless ($result) {
    die("Modify Stub Member Nsgroup failed: ",
        Infoblox::status_code() . ":" . Infoblox::status_detail());
}

$result = $session->modify($object);

unless ($result) {
    die("Submit modification to the session failed: ",
        $session->status_code() . ":" . $session->status_detail());
}

print "Stub Member Nsgroup object modified successfully\n";
```

#Remove a Nsgroup object

```
my @result_array = $session->get(
    object => "Infoblox::Grid::DNS::Nsgroup::StubMember",
    name   => "stub_group_1",
);

my $object = $result_array[0];

unless ($object) {
   die("Get Stub Member Nsgroup failed: ",
       $session->status_code() . ":" . $session->status_detail());
}

$result = $session->remove($object);

unless ($result) {
    die("Remove Nsgroup failed: ",
        $session->status_code() . ":" . $session->status_detail());
```

```
}

print "Stub Member Nsgroup object removed successfully\n";

####PROGRAM ENDS####
```

AUTHOR

Infoblox Inc. http://www.infoblox.com/

SEE ALSO

Infoblox::Session, Infoblox::Session->add(), Infoblox::Session->get(), Infoblox::Session->modify(), Infoblox::Session->remove(), Infoblox::Session->search(), Infoblox::Grid::ExtensibleAttributeDef/Extensible Attribute Values, Infoblox::Grid::Extattr

COPYRIGHT

Copyright (c) 2017 Infoblox Inc.

Infoblox::DTC::Query::Parameters - A DTC query call parameters object.

- NAME
- DESCRIPTION
- CONSTRUCTOR
- SESSION METHODS
- MODULE METHODS
- Infoblox::DTC->query( )
- METHODS
  - address( )
  - lbdn( )
  - member( )
  - qname( )
  - type( )
- AUTHOR
- SEE ALSO
- COPYRIGHT

NAME

Infoblox::DTC::Query::Parameters - A DTC query call parameters object.

DESCRIPTION

A DTC query call parameters object.

CONSTRUCTOR

```
my $params = Infoblox::DTC::Query::Parameters->new(
    address => $string,                          # Required
    qname   => $string,                          # Required
    type    => 'A' | 'AAAA' | 'NAPTR' | 'CNAME', # Required
    lbdn    => $lbdn,                            # Required
    member  => $string,                          # Required
);
```

SESSION METHODS

The object does not support any session methods. It's used as a parameters for a DTC query call.

MODULE METHODS

Infoblox::DTC->query( )

Use this method to perform a DTC query on a given member.

Parameter

Valid value is an Infoblox::DTC::Query::Parameters object.

Returns

The method returns an Infoblox::DTC::Query::Response when operation succeeds, and returns 'false' when the operation fails.

Example

```
#perform DTC query
my $response = $dtc->query($params);
```

METHODS

This section describes all the methods that you can use to set or retrieve the attribute values of the object.

address( )

Use this method to set or retrieve an ostensible IPv4 or IPv6 source address.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

The valid value is a string that contains an IPv4 or IPv6 address.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#get address value
my $address = $params->address();

#modify address value
$params->address('192.168.1.0');
```

lbdn( )

Use this method to set or retrieve a specific LBDN object for querying.

Note that LBDN objects must be first retrieved using the `get()` or `search()` operation.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

The valid value is an Infoblox::DTC::LBDN object.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#get lbdn value
my $lbdn = $params->lbdn();

#modify lbdn value
$params->lbdn($lbdn);
```

member( )

Use this method to set or retrieve a member name for querying.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

The valid value is a string that contains the member name.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#get member value
my $member = $params->member();

#modify member value
$params->member('localhost.localdomain');
```

qname( )

Use this method to set or retrieve a fully-qualified DNS query name.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

The valid value is a string, which contains a query name in FQDN format.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#get qname value
my $qname = $params->qname();

#modify qname value
$params->qname($fqdn);
```

type( )

Use this method to set or retrieve the type of desired results.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

The valid values are 'A', 'AAAA', 'CNAME' and 'NAPTR'.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#get type value
my $type = $params->type();

#modify type value
$params->type('AAAA');
```

AUTHOR

Infoblox Inc. http://www.infoblox.com/

SEE ALSO

Infoblox::DTC->query()

COPYRIGHT

Copyright (c) 2017 Infoblox Inc.

Infoblox::DTC::Query::Response - A DTC query call response object.

- NAME
- DESCRIPTION
- SESSION METHODS
- MODULE METHODS
- Infoblox::DTC->query( )
- METHODS
  - records( )
- AUTHOR
- SEE ALSO
- COPYRIGHT

NAME

Infoblox::DTC::Query::Response - A DTC query call response object.

DESCRIPTION

A DTC query call response object.

SESSION METHODS

The object does not support any session methods. It is retrieved as a result of a DTC query call.

MODULE METHODS

Infoblox::DTC->query( )

Use this method to perform a DTC query on a member.

Parameter

The valid value is an Infoblox::DTC::Query::Parameters object.

Returns

The method returns an Infoblox::DTC::Query::Response when operation succeeds, and returns 'false' when the operation fails.

Example

```
#perform DTC query
my $response = $dtc->query($params);
```

METHODS

This section describes all the methods that you can use to set or retrieve the attribute values of the object.

records( )

Use this method to retrieve an array of query call records. This is a read-only attribute.

Parameter

None

Returns

The valid return value is an array of Infoblox::DTC::Query::Result objects.

Example

```
#get records value
my @records = $response->records();
```

AUTHOR

Infoblox Inc. http://www.infoblox.com/

SEE ALSO

Infoblox::DTC->query()

COPYRIGHT

Copyright (c) 2017 Infoblox Inc.

Infoblox::DTC::Server - A DTC Server object.

- NAME
- DESCRIPTION
- CONSTRUCTOR
- SESSION METHODS
  - Infoblox::Session->add( )
  - Infoblox::Session->get( )
  - Infoblox::Session->modify( )
  - Infoblox::Session->remove( )
  - Infoblox::Session->search( )
- METHODS
  - auto_create_host_record( )
  - comment( )
  - disable( )
  - extattrs( )
  - extensible_attributes( )
  - health( )
  - host( )
  - monitors( )
  - name( )
  - sni_hostname( )
  - override_sni_hostname( )
- AUTHOR
- SEE ALSO
- COPYRIGHT

NAME

Infoblox::DTC::Server - A DTC Server object.

DESCRIPTION

A DTC Server object.

CONSTRUCTOR

```
 my $server = Infoblox::DTC::Server->new(
    name                    => $string,
#Required
    host                    => $string,
#Reqiured
    auto_create_host_record => 'true' | 'false',
#Optional Default is 'false'
    comment                 => $string,
#Optional / Default is undefined
    disable                 => 'true' | 'false',
#Optional / Default is 'false'
    sni_hostname            => $fqdn,
#Optional / Default is undefined
    override_sni_hostname   => 'true' | 'false',
#Optional / Default is 'false'
    monitors                => [$dtc_server_monitor1, $dtc_server_monitor2, ...],
#Optional / Default is empty list
    extattrs                => { $string => $extattr, ... },
#Optional / Default is undefined
    extensible_attributes   => { $string => $string | $num, $string => [ $string | $num, ... ], ... }
#Optional / Default is undefined
 );
```

You cannot set both extattrs and extensible_attributes attributes at the same time.

SESSION METHODS

This section describes all the methods in a Session module that you can apply to a DTC Server object.

Infoblox::Session->add( )

Use this method to add an object to the Infoblox appliance. See Infoblox::Session->add() for parameters and return values.

Example

```
#construct an object
my $server = Infoblox::DTC::Server->new(
   name  => 'server1',
   host  => '192.168.1.2',
);

#submit for addition
my $response = $session->add($server);
```

Infoblox::Session->get( )

Use this method to retrieve all the matching objects from the Infoblox appliance. See Infoblox::Session->get() for parameters and return values.

Key Reference

```
Apply the following attribute to get a specific object:

 name                  - Optional. A DTC Server name in a string format.
 host                  - Optional. A DTC Server host address or FQDN.
 comment               - Optional. A DTC Server comment.
 extattrs              - Optional. A hash reference containing extensible attributes.
 extensible_attributes - Optional. A hash reference that contains extensible attributes.
 pool                  - Optional. A DTC Pool which is linked with a DTC Server.
 status_member         - Optional. A member that provides the object health status. The default is
Grid.
 sni_hostname          - Optional. A hostname for Server Network Indication (SNI) to be used with
the HTTPS monitor.
```

Example

```
my @retrieved_objs = $session->get(
    object => 'Infoblox::DTC::Server',
    name   => 'server1',
    host   => '192.168.1.2'
);

my @retrieved_objs = $session->get(
    object => 'Infoblox::DTC::Server',
    extensible_attributes => { 'Site' => 'Santa Clara' }
);
```

Infoblox::Session->modify( )

Use this method to modify an object in the Infoblox appliance. See Infoblox::Session->modify() for parameters and return values.

Example

```
#Use method to modify the comment.
$server->comment('this is a modified comment');
#Submit modification
my $response = $session->modify($server);
```

Infoblox::Session->remove( )

Use this method to remove an object from the Infoblox appliance. See Infoblox::Session->remove() for parameters and return values.

To remove a specific object, first use `get()` or `search()` to retrieve the specific object, and then submit this object for removal.

Example

```
#Get the objects with the same name
my @retrieved_objs = $session->get(
    object => 'Infoblox::DTC::Server',
    name   => 'server1',
);
#Find the desired object from the retrieved list.
my $desired_server = $retrieved_objs[0];
#Submit for removal
my $response = $session->remove($desired_server);
```

Infoblox::Session->search( )

Use this method to search for objects in the Infoblox appliance. See Infoblox::Session->search() for parameters and return values.

Key Reference

```
Apply the following attribute to get a specific object:
```

```
  name                  - Optional. A DTC Server name in a string format.
  host                  - Optional. A DTC Server host address or FQDN.
  comment               - Optional. A DTC Server comment.
  extattrs              - Optional. A hash reference containing extensible attributes.
  extensible_attributes - Optional. A hash reference that contains extensible attributes.
  pool                  - Optional. A DTC Pool which is linked with a DTC Server.
  status_member         - Optional. A member that provides the object health status. The default is
Grid.
  sni_hostname          - Optional. A hostname for Server Network Indication (SNI) to be used with
the HTTPS monitor.
```

Example

```
my @retrieved_objs = $session->get(
    object => 'Infoblox::DTC::Server',
    name   => 'server1',
    host   => '192.168.1.2'
);

my @retrieved_objs = $session->get(
    object => 'Infoblox::DTC::Server',
    extensible_attributes => { 'Site' => 'Santa Clara' }
);
```

METHODS

This section describes all the methods that you can use to set or retrieve the attribute values of the object.

auto_create_host_record( )

Use this method to set or retrieve the flag that indicates whether the generation of resource records that represents the host is allowed or not.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

Specify 'true' to allow host record generation and 'false' to forbid it. The default value is 'true'.

Returns

If you specified a parameter, the method returns 'true' when the modification succeeds, and returns 'false' when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#get auto_create_host_record value
my $auto_create_host_record = $server->auto_create_host_record();

#modify auto_create_host_record value
$server->auto_create_host_record('false');
```

comment( )

Use this method to set or retrieve a descriptive comment.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

The valid value is a comment in string format (UTF-8) with a maximum of 256 bytes.

Returns

If you specified a parameter, the method returns 'true' when the modification succeeds, and returns 'false' when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#get comment value
my $comment = $server->comment();

#modify comment value
$server->comment('desired comment');
```

disable( )

Use this method to enable or disable a DTC Server.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

Specify 'true' to disable a DTC Server or 'false' to enable it. The default value is 'false'.

Returns

If you specified a parameter, the method returns 'true' when the modification succeeds, and returns 'false' when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#get disable value
my $disable = $server->disable();

#modify disable value
$server->disable('true');
```

extattrs( )

Use this method to set or retrieve the extensible attributes associated with a DTC Server object.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

Valid value is a hash reference containing the names of extensible attributes and their associated values (Infoblox::Grid::Extattr objects).

Returns

If you specified a parameter, the method returns 'true' when the modification succeeds, and returns 'false' when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#get extattrs value
my $ref_extattrs = $server->extattrs();

#Modify extattrs
$server->extattrs({ 'Site' => $extattr1, 'Administrator' => $extattr2 });
```

extensible_attributes( )

Use this method to set or retrieve the extensible attributes associated with a DTC Server object.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

For valid values for extensible attributes, see Infoblox::Grid::ExtensibleAttributeDef/Extensible Attribute Values.

Returns

If you specified a parameter, the method returns 'true' when the modification succeeds, and returns 'false' when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get extensible attributes
my $ref_extensible_attributes = $server->extensible_attributes();

#Modify extensible attributes
$server->extensible_attributes({'Site' => 'Santa Clara', 'Administrator' => ['Peter', 'Tom']});
```

health( )

Use this method to retrieve a DTC Server health information. This is a read-only attribute.

Parameter

None

Returns

The valid return value is an Infoblox::DTC::Health object.

Example

```
#get health value
my $health = $server->health();
```

host( )

Use this method to set or retrieve an address or FQDN of a DTC Server.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

The valid value is a string containing an address or FQDN of a DTC server.

Returns

If you specified a parameter, the method returns 'true' when the modification succeeds, and returns 'false' when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#get host value
my $host = $server->host();

#modify host value
$server->host('192.168.1.2');
```

monitors( )

Use this method to set or retrieve the list of DTC health monitors for the DTC Server.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameters

The valid value is an array reference that contains Infoblox::DTC::Server::Monitor objects.

Returns

If you specified a parameter, the method returns 'true' when the modification succeeds, and returns 'false' when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#get monitors value
my $monitors = $server->monitors();

#retrieve TCP monitor
my $tcp_monitor = $session->get(
    object => 'Infoblox::DTC::Monitor::TCP',
    name   => 'tcp_monitor',
);

#construct DTC server monitor
my $monitor = Infoblox::DTC::Server::Monitor->new(
    monitor => $tcp_monitor,
    host    => '192.168.1.10',
);

#modify monitors value
$server->monitors([$monitor]);
```

name( )

Use this method to set or retrieve the DTC Server name.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

Desired name in string format with a maximum of 256 bytes.

Returns

If you specified a parameter, the method returns 'true' when the modification succeeds, and returns 'false' when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#get name value
my $name = $server->name();

#modify name value
$server->name('server_name');
```

sni_hostname( )

Use this method to set or retrieve the hostname for Server Network Indication (SNI) to be used with the HTTPS monitor.

Setting this method to a defined value implicitly sets the override_sni_hostname method to 'true'. Setting the parameter to undefined causes the HTTPS monitor to use DTC server's hostname with SNI option and automatically resets the override_sni_hostname method to 'false'.

Note that when sni_hostname is set to a defined value and override_sni_hostname is set to 'false', the last operation takes precedence. Thus the sequence $object->sni_hostname('sni-host.infoblox.localdomain'); $object->override_sni_hostname('false'); will set override_sni_hostname to 'false', and the sequence $object->override_sni_hostname('false'); $object->sni_hostname('sni-host.infoblox.localdomain'); will result in override_sni_hostname='true'.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

The valid value is the FQDN for the desired host in string format.

Returns

If you specified a parameter, the method returns 'true' when the modification succeeds, and returns 'false' when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#get sni_hostname value
my $sni_hostname = $server->sni_hostname();

#modify sni_hostname value
$server->sni_hostname('sni-host.infoblox.localdomain');
```

override_sni_hostname( )

The override_sni_hostname attribute controls whether the sni_hostname method values of the DTC server are used, instead of the DTC server name.

The override_sni_hostname attribute can be specified explicitly. It is also set implicitly when sni_hostname is set to a defined value.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

Set the parameter to 'true' to allow usage of the sni_hostname. Set the parameter to 'false' to use the DTC server name with the SNI option.

The default value of this parameter is 'false'.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get override_sni_hostname value
my $override_sni_hostname = $server->override_sni_hostname();

#Modify override_sni_hostname value
$server->override_sni_hostname('true');
```

AUTHOR

Infoblox Inc. http://www.infoblox.com/

SEE ALSO

Infoblox::Session, Infoblox::Session->add(), Infoblox::Session->get(), Infoblox::Session->modify(), Infoblox::Session->remove(), Infoblox::Session->search(), Infoblox::Grid::Extattr, Infoblox::Grid::ExtensibleAttributeDef/Extensible Attribute Values, Infoblox::DTC::Health.

COPYRIGHT

Copyright (c) 2017 Infoblox Inc.

Infoblox::DTC::Record::NAPTR - A DTC NAPTR record object.

- NAME
- DESCRIPTION
- CONSTRUCTOR
- SESSION METHODS
  - Infoblox::Session->add( )
  - Infoblox::Session->get( )
  - Infoblox::Session->modify( )
  - Infoblox::Session->remove( )
  - Infoblox::Session->search( )
- METHODS
  - comment( )
  - disable( )
  - dtc_server( )
  - flags( )
  - order( )
  - preference( )
  - regexp( )
  - replacement( )
  - services( )
  - ttl( )
- AUTHOR
- SEE ALSO
- COPYRIGHT

NAME

Infoblox::DTC::Record::NAPTR - A DTC NAPTR record object.

DESCRIPTION

A DTC NAPTR object represents a DNS Traffic Control Naming Authority Pointer (DTC NAPTR) resource record. This resource record specifies a regular expression-based rewrite rule that, when applied to an existing string, produces a new domain name or URI.

CONSTRUCTOR

```
my $dtc_naptr = Infoblox::DTC::Record::NAPTR->new(
    'dtc_server'  => $dtc_server,      #Required
    'order'       => $num,             #Required
    'preference'  => $num,             #Required
    'replacement' => $string           #Required
    'comment'     => $string,          #Optional
    'disable'     => 'true' | 'false', #Optional / Default is 'false'
    'flags'       => $string,          #Optional
    'regexp'      => $string,          #Optional
    'services'    => $string,          #Optional
    'ttl'         => $uint | undef,    #Optional / Default is undefined
);
```

SESSION METHODS

This section describes all the methods in an Infoblox::Session module that can be applied to a DTC NAPTR record object.

Infoblox::Session->add( )

Use this method to add an object to the Infoblox appliance. See Infoblox::Session->add() for parameters and return values.

Example

```
#Construct an object
my $dtc_naptr = Infoblox::DTC::Record::NAPTR->new(
    'comment'     => 'Sample DTC NAPTR record',
    'disable'     => 'false',
    'dtc_server'  => $dtc_server,
    'flags'       => 'U',
    'order'       => 100,
    'preference'  => 10,
    'regexp'      => '!http://my[.](.*)!1!i',
```

```
    'replacement' => 'domain2.com',
    'services'    => 'http+E2U',
    'ttl'         => 1024,
);

# Submit for addition
my $response = $session->add($dtc_naptr);
```

Infoblox::Session->get( )

Use this method to retrieve the existing objects from an Infoblox appliance. See Infoblox::Session->get() for parameters and return values.

Key Reference

```
Apply the following attributes to get specific DTC NAPTR record objects:

    dtc_server      - Required. DTC Server object the DTC NAPTR record is associated with.
    comment         - Optional. Descriptive comment in string format.
    flags           - Optional. Flags in string format.
    order           - Optional. Unsigned integer that represent DTC NAPTR order.
    preference      - Optional. Unsigned integer that represent DTC NAPTR preference.
    replacement     - Optional. Replacement field in string format.
    services        - Optional. Services field in string format.
```

Example

```
my @retrieved_objs = $session->get(
    object     => 'Infoblox::DTC::Record::NAPTR',
    dtc_server => $dtc_server,
);

my @retrieved_objs = $session->get(
   object     => 'Infoblox::DTC::Record::NAPTR',
   dtc_server => $dtc_server,
   order      => 1,
);
```

Infoblox::Session->modify( )

Use this method to modify an object in the Infoblox appliance. See Infoblox::Session->modify() for parameters and return values.

Example

```
# Use this method to modify the comment.
$dtc_naptr->comment('this is a modified comment');

# Submit the modification
my $response = $session->modify($dtc_naptr);
```

Infoblox::Session->remove( )

Use this method to submit the removal of an object from the Infoblox appliance. See Infoblox::Session->remove() for parameters and return values.

To remove a specific object, first use `get()` or `search()` to retrieve the object, and then submit it for removal.

Example

```
# Get the objects associated with DTC Server $dtc_server
my @retrieved_objs = $session->get(
    object     => 'Infoblox::DTC::Record::NAPTR',
    dtc_server => $dtc_server,
);

# Find the desired object in the retrieved list.
my $desired_rec = $retrieved_objs[0];

# Submit for removal
my $response = $session->remove($desired_rec);
```

Infoblox::Session->search( )

Use this method to retrieve the existing objects from an Infoblox appliance. See Infoblox::Session->search() for parameters and return values.

Key Reference

```
Apply the following attributes to search the DTC NAPTR record objects:

    dtc_server      - Required. DTC Server object the DTC NAPTR record is associated with (exact).
    comment         - Optional. Descriptive comment in string format (regexp).
    flags           - Optional. Flags in string format (regexp).
    order           - Optional. Unsigned integer that represent DTC NAPTR order (exact).
    preference      - Optional. Unsigned integer that represent DTC NAPTR preference (exact).
    replacement     - Optional. Replacement field in string format (regexp).
    services        - Optional. Services field in string format (regexp).
```

Example

```
# Get the objects associated with DTC Server $dtc_server
my @retrieved_objs = $session->search(
    object     => 'Infoblox::DTC::Record::NAPTR',
    dtc_server => $dtc_server,
);

# Get the objects associated with DTC Server $dtc_server with comment that starts from 'NAPTR'
my @retrieved_objs = $session->get(
    object     => 'Infoblox::DTC::Record::NAPTR',
    dtc_server => $dtc_server,
    comment    => '^NAPTR.*',
);
```

METHODS

This section describes all the methods that can be used to retrieve the attribute values of a DTC NAPTR record object.

comment( )

Use this method to set or retrieve the descriptive comment of a DTC NAPTR record object.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

Desired comment in string format with a maximum of 256 characters.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get comment value
my $comment = $dtc_naptr->comment();
#Modify comment value
$dtc_naptr->comment('This is a new comment');
```

disable( )

Use this method to set or retrieve the disable flag of a DTC NAPTR record.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

Specify 'true' to set the disable flag or 'false' to deactivate/unset it.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get disable value
my $disable = $dtc_naptr->disable();
#Modify disable value
$dtc_naptr->disable('true');
```

dtc_server( )

Use this method to set or retrieve the DTC Server the DTC NAPTR record associated with.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Note that once the DTC record has been created the DTC Server value cannot be changed.

Parameter

The valid value is an Infoblox::DTC::Server object.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get the dtc_server field
my $dtc_server = $dtc_naptr->dtc_server();

#Modify dtc_server value
$dtc_naptr->dtc_server($server1);
```

flags( )

Use this method to retrieve the flags that control the interpretation of the fields for a DTC NAPTR record object.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

Currently supported values for the flags field are 'U', 'S', 'P' and 'A'.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get flags value
my $flags = $dtc_naptr->flags();
#Modify flags value
$dtc_naptr->flags('U');
```

order( )

Use this method to retrieve the order parameter of a DTC NAPTR record. This parameter specifies the order in which the DTC NAPTR rules are applied when multiple rules are present.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

The order parameter is a 16-bit unsigned integer.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get order value
my $order = $dtc_naptr->order();
#Modify order value
$dtc_naptr->order(200);
```

preference( )

Use this method to retrieve the preference field of a DTC NAPTR record. The preference field determines the order DTC NAPTR records are processed when multiple records with the same order parameter are present.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

The preference parameter is a 16-bit integer.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get preference value
my $preference = $dtc_naptr->preference();
#Modify preference value
$dtc_naptr->preference(300);
```

regexp( )

Use this method to specify the regular expression-based rewriting rule of a DTC NAPTR record. This should be a POSIX compliant regular expression, including the substitution rule and flags. Refer to RFC 2915 for the field syntax details.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

The valid parameter is a POSIX compliant regular expression or substitution expression.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get regexp value
my $regexp = $dtc_naptr->regexp();
#Modify regexp value
$dtc_naptr->regexp('!http://(.*)!1!i');
```

replacement( )

Use this method to retrieve the replacement field of a DTC NAPTR record object. For non-terminal DTC NAPTR records, this field specifies the next domain name to look up.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

The attribute value can be in unicode format.

Parameter

The desired replacement value in a Fully-Qualified Domain Name (FQDN) format.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get replacement value
my $replacement = $dtc_naptr->replacement();
#Modify replacement value
$dtc_naptr->replacement('__tcp__.domain.com');
```

services( )

Use this field to specify services. The services field contains protocol and service identifiers, for example 'http+E2U' or 'SIPS+D2T'.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

The services field in string format.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get services value
my $services = $dtc_naptr->services();
#Modify services value
$dtc_naptr->services('http+E2U');
```

ttl( )

Use this method to set or retrieve the Time to Live (TTL) value.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

The default value is undefined which indicates that the record inherits the TTL value of the DTC server pool.

Specify a TTL value to override the TTL value at the DTC server pool level.

Parameter

The valid value is an unsigned integer between 0 and 4294967295 that represents the duration (in seconds) that the record is cached. Zero indicates that the record should not be cached.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get ttl value
my $ttl = $dtc_naptr->ttl();

#Modify ttl value
$dtc_naptr->ttl(1800);

#inherit ttl value
$dtc_naptr->ttl(undef);
```

AUTHOR

Infoblox Inc. http://www.infoblox.com/

SEE ALSO

Infoblox::Session, Infoblox::Session->add(), Infoblox::Session->get(), Infoblox::Session->modify(), Infoblox::Session->remove(), Infoblox::Session->search(), Infoblox::DTC::Server

---

COPYRIGHT

Copyright (c) 2017 Infoblox Inc.

Infoblox::DTC::Record::A - A DTC A record object.

- NAME
- DESCRIPTION
- CONSTRUCTOR
- SESSION METHODS
  - Infoblox::Session->add( )
  - Infoblox::Session->get( )
  - Infoblox::Session->modify( )
  - Infoblox::Session->remove( )
  - Infoblox::Session->search( )
- METHODS
  - auto_created( )
  - comment( )
  - disable( )
  - dtc_server( )
  - ipv4addr( )
  - ttl( )
- AUTHOR
- SEE ALSO
- COPYRIGHT

NAME

Infoblox::DTC::Record::A - A DTC A record object.

DESCRIPTION

A DTC A object represents a DNS Traffic Control Address (DTC A) resource record. DTC A record maps a load balancing domain name to an IPv4 address.

CONSTRUCTOR

```
my $object = Infoblox::DTC::Record::A->new(
    'dtc_server'  => $dtc_server,      #Required
    'ipv4addr'    => $ipv4addr,        #Required
    'comment'     => $string,          #Optional
    'disable'     => 'true' | 'false', #Optional / Default is 'false'
    'ttl'         => $uint | undef,    #Optional / Default is undefined
);
```

SESSION METHODS

This section describes all the methods in an Infoblox::Session module that can be applied to a DTC A record object.

Infoblox::Session->add( )

Use this method to add an object to the Infoblox appliance. See Infoblox::Session->add() for parameters and return values.

Example

```
#Construct an object
my $object = Infoblox::DTC::Record::A->new(
    'comment'     => 'Sample DTC A record',
    'disable'     => 'false',
    'dtc_server'  => $dtc_server,
    'ipv4addr'    => '10.0.0.1',
    'ttl'         => 1024,
);

# Submit for addition
my $response = $session->add($object);
```

Infoblox::Session->get( )

Use this method to retrieve the existing objects from an Infoblox appliance. See Infoblox::Session->get() for parameters and return values.

Key Reference

```
Apply the following attributes to get specific DTC A record objects:

    dtc_server - Required. DTC Server object the DTC A record is associated with (exact).
    comment    - Optional. Descriptive comment in string format (regexp).
    ipv4addr   - Optional. DTC A record IPv4 address in string format (regexp).
```

Example

```
my @retrieved_objs = $session->get(
    object     => 'Infoblox::DTC::Record::A',
    dtc_server => $dtc_server,
);

my @retrieved_objs = $session->get(
   object     => 'Infoblox::DTC::Record::A',
   dtc_server => $dtc_server,
   ipv4addr   => '10.0.0.1',
);
```

Infoblox::Session->modify( )

Use this method to modify an object in the Infoblox appliance. See Infoblox::Session->modify() for parameters and return values.

Example

```
# Use this method to modify the comment.
$object->comment('this is a modified comment');

# Submit the modification
my $response = $session->modify($object);
```

Infoblox::Session->remove( )

Use this method to submit the removal of an object from the Infoblox appliance. See Infoblox::Session->remove() for parameters and return values.

To remove a specific object, first use `get()` or `search()` to retrieve the object, and then submit it for removal.

Example

```
# Get the objects associated with DTC Server $dtc_server
my @retrieved_objs = $session->get(
    object     => 'Infoblox::DTC::Record::A',
    dtc_server => $dtc_server,
);

# Find the desired object in the retrieved list.
my $desired_rec = $retrieved_objs[0];

# Submit for removal
my $response = $session->remove($desired_rec);
```

Infoblox::Session->search( )

Use this method to retrieve the existing objects from an Infoblox appliance. See Infoblox::Session->search() for parameters and return values.

Key Reference

```
Apply the following attributes to search the DTC A record objects:

    dtc_server - Required. DTC Server object the DTC A record is associated with.
    comment    - Optional. Descriptive comment in string format.
    ipv4addr   - Optional. DTC A record address.
```

Example

```
# Get the objects associated with DTC Server $dtc_server
my @retrieved_objs = $session->search(
    object     => 'Infoblox::DTC::Record::A',
    dtc_server => $dtc_server,
);

# Get the objects associated with DTC Server $dtc_server with comment that starts from 'A'
my @retrieved_objs = $session->get(
    object     => 'Infoblox::DTC::Record::A',
    dtc_server => $dtc_server,
    comment    => '^A.*',
```

```
);
```

METHODS

This section describes all the methods that can be used to retrieve the attribute values of a DTC A record object.

auto_created( )

Use this method to retrieve the flag that indicates whether this record was automatically created by NIOS or not. This is a read-only attribute.

Parameter

None

Returns

The method returns the attribute value.

Example

```
#Get auto_created value
my $auto_created = $object->auto_created();
```

comment( )

Use this method to set or retrieve the descriptive comment of a DTC A record object.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

Desired comment in string format with a maximum of 256 characters.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get comment value
my $comment = $object->comment();

#Modify comment value
$object->comment('This is a new comment');
```

disable( )

Use this method to set or retrieve the disable flag of a DTC A record.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

Specify 'true' to set the disable flag or 'false' to deactivate/unset it.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get disable value
my $disable = $object->disable();

#Modify disable value
$object->disable('true');
```

dtc_server( )

Use this method to set or retrieve the DTC Server the DTC A record is associated with.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Note that once the DTC record was created, the DTC Server value cannot be changed.

Parameter

The valid value is an Infoblox::DTC::Server object.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get the dtc_server field
my $dtc_server = $object->dtc_server();

#Modify dtc_server value
$object->dtc_server($server1);
```

ipv4addr( )

Use this method to set or retrieve the IPv4 address that is mapped to a domain name.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

The valid value is desired IPv4 address in string format.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get ipv4addr value
my $ipv4addr = $object->ipv4addr();

#Modify ipv4addr value
$object->ipv4addr('10.0.0.10');
```

ttl( )

Use this method to set or retrieve the Time to Live (TTL) value.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

The default value is undefined which indicates that the record inherits the TTL value of the DTC server pool.

Specify a TTL value to override the TTL value at the DTC server pool level.

Parameter

The valid value is an unsigned integer between 0 and 4294967295 that represents the duration (in seconds) that the record is cached. Zero indicates that the record should not be cached.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get ttl value
my $ttl = $object->ttl();

#Modify ttl value
$object->ttl(1800);

#inherit ttl value
$object->ttl(undef);
```

AUTHOR

Infoblox Inc. http://www.infoblox.com/

SEE ALSO

Infoblox::Session, Infoblox::Session->add(), Infoblox::Session->get(), Infoblox::Session->modify(), Infoblox::Session->remove(), Infoblox::Session->search(), Infoblox::DTC::Server

---

COPYRIGHT

Copyright (c) 2017 Infoblox Inc.

Infoblox::DTC::Record::AAAA - A DTC AAAA record object.

- NAME
- DESCRIPTION
- CONSTRUCTOR
- SESSION METHODS
  - Infoblox::Session->add( )
  - Infoblox::Session->get( )
  - Infoblox::Session->modify( )
  - Infoblox::Session->remove( )
  - Infoblox::Session->search( )
- METHODS
  - auto_created( )
  - comment( )
  - disable( )
  - dtc_server( )
  - ipv6addr( )
  - ttl( )
- AUTHOR
- SEE ALSO
- COPYRIGHT

NAME

Infoblox::DTC::Record::AAAA - A DTC AAAA record object.

DESCRIPTION

A DTC AAAA object represents a DNS Traffic Control IPv6 Address (DTC AAAA) resource record. DTC AAAA record maps a load balancing domain name to an IPv6 address.

CONSTRUCTOR

```
my $object = Infoblox::DTC::Record::AAAA->new(
    'dtc_server'  => $dtc_server,      #Required
    'ipv6addr'    => $ipv6addr,        #Required
    'comment'     => $string,          #Optional
    'disable'     => 'true' | 'false', #Optional / Default is 'false'
    'ttl'         => $uint | undef,    #Optional / Default is undefined
);
```

SESSION METHODS

This section describes all the methods in an Infoblox::Session module that can be applied to a DTC AAAA record object.

Infoblox::Session->add( )

Use this method to add an object to the Infoblox appliance. See Infoblox::Session->add() for parameters and return values.

Example

```
#Construct an object
my $object = Infoblox::DTC::Record::AAAA->new(
    'comment'     => 'Sample DTC AAAA record',
    'disable'     => 'false',
    'dtc_server'  => $dtc_server,
    'ipv6addr'    => '2001:db8::1',
    'ttl'         => 1024,
);

# Submit for addition
my $response = $session->add($object);
```

Infoblox::Session->get( )

Use this method to retrieve the existing objects from an Infoblox appliance. See Infoblox::Session->get() for parameters and return values.

Key Reference

```
Apply the following attributes to get specific DTC AAAA record objects:

    dtc_server - Required. DTC Server object the DTC AAAA record is associated with.
    comment    - Optional. Descriptive comment in string format.
    ipv6addr   - Optional. DTC AAAA record IPv6 address in string format.
```

Example

```
my @retrieved_objs = $session->get(
    object     => 'Infoblox::DTC::Record::AAAA',
    dtc_server => $dtc_server,
);

my @retrieved_objs = $session->get(
   object     => 'Infoblox::DTC::Record::AAAA',
   dtc_server => $dtc_server,
   ipv6addr   => '2001:db8::1',
);
```

Infoblox::Session->modify( )

Use this method to modify an object in the Infoblox appliance. See Infoblox::Session->modify() for parameters and return values.

Example

```
# Use this method to modify the comment.
$object->comment('this is a modified comment');

# Submit the modification
my $response = $session->modify($object);
```

Infoblox::Session->remove( )

Use this method to submit the removal of an object from the Infoblox appliance. See Infoblox::Session->remove() for parameters and return values.

To remove a specific object, first use `get()` or `search()` to retrieve the object, and then submit it for removal.

Example

```
# Get the objects associated with DTC Server $dtc_server
my @retrieved_objs = $session->get(
    object     => 'Infoblox::DTC::Record::AAAA',
    dtc_server => $dtc_server,
);

# Find the desired object in the retrieved list.
my $desired_rec = $retrieved_objs[0];

# Submit for removal
my $response = $session->remove($desired_rec);
```

Infoblox::Session->search( )

Use this method to retrieve the existing objects from an Infoblox appliance. See Infoblox::Session->search() for parameters and return values.

Key Reference

```
Apply the following attributes to search the DTC AAAA record objects:

    dtc_server - Required. DTC Server object the DTC AAAA record is associated with (exact).
    comment    - Optional. Descriptive comment in string format (regexp).
    ipv6addr   - Optional. DTC AAAA record address (regexp).
```

Example

```
# Get the objects associated with DTC Server $dtc_server
my @retrieved_objs = $session->search(
    object     => 'Infoblox::DTC::Record::AAAA',
    dtc_server => $dtc_server,
);

# Get the objects associated with DTC Server $dtc_server with comment that starts from 'A'
my @retrieved_objs = $session->get(
    object     => 'Infoblox::DTC::Record::AAAA',
    dtc_server => $dtc_server,
    comment    => '^A.*',
```

```
);
```

METHODS

This section describes all the methods that can be used to retrieve the attribute values of a DTC AAAA record object.

auto_created( )

Use this method to retrieve the flag that indicates whether this record was automatically created by NIOS or not. This is a read-only attribute.

Parameter

None

Returns

The method returns the attribute value.

Example

```
#Get auto_created value
my $auto_created = $object->auto_created();
```

comment( )

Use this method to set or retrieve the descriptive comment of a DTC AAAA record object.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

Desired comment in string format with a maximum of 256 characters.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get comment value
my $comment = $object->comment();

#Modify comment value
$object->comment('This is a new comment');
```

disable( )

Use this method to set or retrieve the disable flag of a DTC AAAA record.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

Specify 'true' to set the disable flag or 'false' to deactivate/unset it.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get disable value
my $disable = $object->disable();

#Modify disable value
$object->disable('true');
```

dtc_server( )

Use this method to set or retrieve the DTC Server the DTC AAAA record is associated with.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Note that once the DTC record was created, the DTC Server value cannot be changed.

Parameter

The valid value is an Infoblox::DTC::Server object.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get the dtc_server field
my $dtc_server = $object->dtc_server();

#Modify dtc_server value
$object->dtc_server($server1);
```

ipv6addr( )

Use this method to set or retrieve the IPv6 address that is mapped to a domain name.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

The valid value is desired IPv6 address in string format.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get ipv6addr value
my $ipv6addr = $object->ipv6addr();

#Modify ipv6addr value
$object->ipv6addr('2001:db8::1');
```

ttl( )

Use this method to set or retrieve the Time to Live (TTL) value.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

The default value is undefined which indicates that the record inherits the TTL value of the DTC server pool.

Specify a TTL value to override the TTL value at the DTC server pool level.

Parameter

The valid value is an unsigned integer between 0 and 4294967295 that represents the duration (in seconds) that the record is cached. Zero indicates that the record should not be cached.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get ttl value
my $ttl = $object->ttl();

#Modify ttl value
$object->ttl(1800);

#inherit ttl value
$object->ttl(undef);
```

AUTHOR

Infoblox::DTC::Record::AAAA - A DTC AAAA record object.

Infoblox Inc. http://www.infoblox.com/

SEE ALSO

Infoblox::Session, Infoblox::Session->add(), Infoblox::Session->get(), Infoblox::Session->modify(), Infoblox::Session->remove(), Infoblox::Session->search(), Infoblox::DTC::Server

---

COPYRIGHT

Copyright (c) 2017 Infoblox Inc.

Infoblox::DTC::Record::CNAME - A DTC CNAME record object.

- NAME
- DESCRIPTION
- CONSTRUCTOR
- SESSION METHODS
  - Infoblox::Session->add( )
  - Infoblox::Session->get( )
  - Infoblox::Session->modify( )
  - Infoblox::Session->remove( )
  - Infoblox::Session->search( )
- METHODS
  - auto_created( )
  - comment( )
  - disable( )
  - dtc_server( )
  - canonical( )
  - dns_canonical( )
  - ttl( )
- AUTHOR
- SEE ALSO
- COPYRIGHT

NAME

Infoblox::DTC::Record::CNAME - A DTC CNAME record object.

DESCRIPTION

A DTC CNAME object represents a DNS Traffic Control Canonical Name (DTC CNAME) resource record. DTC CNAME record maps domain name alias to its canonical domain name.

CONSTRUCTOR

```
my $object = Infoblox::DTC::Record::CNAME->new(
    'dtc_server' => $dtc_server,       #Required
    'canonical'  => $fqdn,             #Required
    'comment'    => $string,           #Optional
    'disable'    => 'true' | 'false', #Optional / Default is 'false'
    'ttl'        => $uint | undef,     #Optional / Default is undefined
);
```

SESSION METHODS

This section describes all the methods in an Infoblox::Session module that can be applied to a DTC CNAME record object.

Infoblox::Session->add( )

Use this method to add an object to the Infoblox appliance. See Infoblox::Session->add() for parameters and return values.

Example

```
#Construct an object
my $object = Infoblox::DTC::Record::CNAME->new(
    'comment'    => 'Sample DTC CNAME record',
    'disable'    => 'false',
    'dtc_server' => $dtc_server,
    'canonical'  => 'dtc.localdomain',
    'ttl'        => 1024,
);

# Submit for addition
my $response = $session->add($object);
```

Infoblox::Session->get( )

Use this method to retrieve the existing objects from an Infoblox appliance. See Infoblox::Session->get() for parameters and return values.

Key Reference

```
Apply the following attributes to get specific DTC CNAME record objects:

    dtc_server - Required. DTC Server object the DTC CNAME record is associated with.
    comment    - Optional. Descriptive comment in string format.
    canonical  - Optional. DTC CNAME record canonical name in string format.
```

Example

```
my @retrieved_objs = $session->get(
    object     => 'Infoblox::DTC::Record::CNAME',
    dtc_server => $dtc_server,
);

my @retrieved_objs = $session->get(
   object     => 'Infoblox::DTC::Record::CNAME',
   dtc_server => $dtc_server,
   canonical  => 'dtc.localdomain',
);
```

Infoblox::Session->modify( )

Use this method to modify an object in the Infoblox appliance. See Infoblox::Session->modify() for parameters and return values.

Example

```
# Use this method to modify the comment.
$object->comment('this is a modified comment');

# Submit the modification
my $response = $session->modify($object);
```

Infoblox::Session->remove( )

Use this method to submit the removal of an object from the Infoblox appliance. See Infoblox::Session->remove() for parameters and return values.

To remove a specific object, first use `get()` or `search()` to retrieve the object, and then submit it for removal.

Example

```
# Get the objects associated with DTC Server $dtc_server
my @retrieved_objs = $session->get(
    object     => 'Infoblox::DTC::Record::CNAME',
    dtc_server => $dtc_server,
);

# Find the desired object in the retrieved list.
my $desired_rec = $retrieved_objs[0];

# Submit for removal
my $response = $session->remove($desired_rec);
```

Infoblox::Session->search( )

Use this method to retrieve the existing objects from an Infoblox appliance. See Infoblox::Session->search() for parameters and return values.

Key Reference

```
Apply the following attributes to search the DTC CNAME record objects:

    dtc_server - Required. DTC Server object the DTC CNAME record is associated with (exact).
    comment    - Optional. Descriptive comment in string format (regexp).
    canonical  - Optional. DTC CNAME canonical name (regexp).
```

Example

```
# Get the objects associated with DTC Server $dtc_server
my @retrieved_objs = $session->search(
    object     => 'Infoblox::DTC::Record::CNAME',
    dtc_server => $dtc_server,
);

# Get the objects associated with DTC Server $dtc_server with comment that starts from 'A'
my @retrieved_objs = $session->get(
    object     => 'Infoblox::DTC::Record::CNAME',
    dtc_server => $dtc_server,
```

```
        comment     => '^A.*',
);
```

METHODS

This section describes all the methods that can be used to retrieve the attribute values of a DTC CNAME record object.

auto_created( )

Use this method to retrieve the flag that indicates whether this record was automatically created by NIOS or not. This is a read-only attribute.

Parameter

None

Returns

The method returns the attribute value.

Example

```
#Get auto_created value
my $auto_created = $object->auto_created();
```

comment( )

Use this method to set or retrieve the descriptive comment of a DTC CNAME record object.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

Desired comment in string format with a maximum of 256 characters.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get comment value
my $comment = $object->comment();

#Modify comment value
$object->comment('This is a new comment');
```

disable( )

Use this method to set or retrieve the disable flag of a DTC CNAME record.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

Specify 'true' to set the disable flag or 'false' to deactivate/unset it.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get disable value
my $disable = $object->disable();

#Modify disable value
$object->disable('true');
```

dtc_server( )

Use this method to set or retrieve the DTC Server the DTC CNAME record associated with.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Note that once the DTC record was created, the DTC Server value cannot be changed.

Parameter

The valid value is an Infoblox::DTC::Server object.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get the dtc_server field
my $dtc_server = $object->dtc_server();

#Modify dtc_server value
$object->dtc_server($server1);
```

canonical( )

Use this method to set or retrieve the canonical name value.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

The valid value is desired canonical name in string format. Note that canonical name should be in Fully-Qualified Domain Name (FQDN) format.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get canonical value
my $canonical = $object->canonical();

#Modify canonical value
$object->canonical('dtc.localdomain');
```

dns_canonical( )

Use this method to retrieve the canonical name as served by DNS protocol. This is a read-only attribute.

Omit the parameter to retrieve the attribute value.

Parameter

None

Returns

The method returns the attribute value.

Example

```
#Get dns_canonical value
my $dns_canonical = $object->dns_canonical();
```

ttl( )

Use this method to set or retrieve the Time to Live (TTL) value.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

The default value is undefined which indicates that the record inherits the TTL value of the DTC server pool.

Specify a TTL value to override the TTL value at the DTC server pool level.

Parameter

The valid value is an unsigned integer between 0 and 4294967295 that represents the duration (in seconds) that the record is

cached. Zero indicates that the record should not be cached.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get ttl value
my $ttl = $object->ttl();

#Modify ttl value
$object->ttl(1800);

#inherit ttl value
$object->ttl(undef);
```

AUTHOR

Infoblox Inc. http://www.infoblox.com/

SEE ALSO

Infoblox::Session, Infoblox::Session->add(), Infoblox::Session->get(), Infoblox::Session->modify(), Infoblox::Session->remove(), Infoblox::Session->search(), Infoblox::DTC::Server

---

COPYRIGHT

Copyright (c) 2017 Infoblox Inc.

Infoblox::Grid::X509Certificate - An X509Certificate object.

- NAME
- DESCRIPTION
- CONSTRUCTOR
- SESSION METHODS
  - Infoblox::Session->get( )
  - Infoblox::Session->remove( )
  - Infoblox::Session->search( )
- METHODS
  - issuer( )
  - serial( )
  - distinguished_name( )
  - valid_not_before( )
  - valid_not_after( )
- AUTHOR
- SEE ALSO
- COPYRIGHT

NAME

Infoblox::Grid::X509Certificate - An X509Certificate object.

DESCRIPTION

An X509Certificate object.

CONSTRUCTOR

SESSION METHODS

This section describes all the methods in an Infoblox::Session module that you can apply to an X509Certificate object.

Infoblox::Session->get( )

Use this method to retrieve all the matching objects from the Infoblox appliance. See Infoblox::Session->get() for parameters and return values.

Key References

```
 Apply the following attributes to get for an X509Certificate object:

  serial              - Optional. The certificate serial number in hex format.
  issuer              - Optional. The name of the issuer.
  distinguished_name  - Optional. The certificate subject name.
  valid_not_after     - Optional. String that contains a desired value of the certificate expiration
date in ISO 8601 format.
  valid_not_before    - Optional. String that contains a desired value of the certificate start date
in ISO 8601 format.
```

Example

```
 # Retrieve X509Certificate objects with specify certificate subject name
 my @retrieved_objs = $session->get(
        object            => 'Infoblox::Grid::X509Certificate',
        distinguished_name => 'CN="10.34.1.106",OU="Engineering",O="Infoblox Inc",L="Santa
Clara",ST="California",C="US"',
 );
```

Infoblox::Session->remove( )

Use this method to remove an object from the Infoblox appliance. See Infoblox::Session->remove() for parameters and return values.

To remove a specific object, first use `get()` or `search()` to retrieve the specific object, and then submit this object for removal.

Example

```
 # Get X509Certificate object
 my @retrieved_objs = $session->get(
        object            => 'Infoblox::Grid::X509Certificate',
        distinguished_name => 'CN="10.34.1.106",OU="Engineering",O="Infoblox Inc",L="Santa
```

```
Clara",ST="California",C="US"',
 );

 # Find the desired object from the retrieved list
 my $x509cert = $retrieved_objs[0];

 # Submit for removal
 my $response = $session->remove( $x509cert );
```

Infoblox::Session->search( )

Use this method to search for objects in the Infoblox appliance. See Infoblox::Session->search() for parameters and return values.

Key References

```
 Apply the following attributes to search for an X509Certificate object:

  serial              - Optional. The certificate serial number in hex format.
  issuer              - Optional. The name of the issuer.
  distinguished_name  - Optional. The certificate subject name.
  valid_not_after     - Optional. String that contains a desired value of the certificate expiration
date in ISO 8601 format.
  valid_not_before    - Optional. String that contains a desired value of the certificate start date
in ISO 8601 format.
```

Example

```
 # Retrieve X509Certificate objects with specify certificate subject name
 my @retrieved_objs = $session->search(
        object             => 'Infoblox::Grid::X509Certificate',
        distinguished_name => 'CN="10.34.1.106",OU="Engineering",O="Infoblox Inc",L="Santa
Clara",ST="California",C="US"',
 );
```

METHODS

This section describes all the methods that you can use to configure and retrieve the attribute values of a X509Certificate object.

issuer( )

Use this method to retrieve the issuer subject name. This method does not support any argument because the attribute is read-only and is set by the server.

Parameter

None

Returns

The method returns the attribute value.

Example

```
 # Get attribute value
 my $issuer = $x509cert->issuer();
```

serial( )

Use this method to retrieve the certificate serial number in hex format. This method does not support any argument because the attribute is read-only and is set by the server.

Parameter

None

Returns

The method returns the attribute value.

Example

```
 # Get attribute value
 my $serial = $x509cert->serial();
```

distinguished_name( )

Use this method to retrieve the certificate subject name. This method does not support any argument because the attribute is read-only and is set by the server.

Parameter

None

Returns

The method returns the attribute value.

Example

```
# Get attribute value
my $distinguished_name = $x509cert->distinguished_name();
```

valid_not_before( )

Use this method to retrieve the date before the certificate becomes invalid in ISO 8601 format. This method does not support any argument because the attribute is read-only and is set by the server.

Parameter

None

Returns

The method returns the attribute value.

Example

```
# Get attribute value
my $valid_not_before = $x509cert->valid_not_before();
```

valid_not_after( )

Use this method to retrieve the date after the certificate becomes invalid in ISO 8601 format. This method does not support any argument because the attribute is read-only and is set by the server.

Parameter

None

Returns

The method returns the attribute value.

Example

```
# Get attribute value
my $valid_not_after = $x509cert->valid_not_after();
```

AUTHOR

Infoblox Inc. http://www.infoblox.com/

SEE ALSO

Infoblox::Session

COPYRIGHT

Copyright (c) 2017 Infoblox Inc.

Infoblox::DTC - A DTC functions.

- NAME
- DESCRIPTION
- CONSTRUCTOR
- SESSION METHODS
- METHODS
  - query( )
  - import_maxminddb( )
  - add_certificate( )
- SEE ALSO
- AUTHOR
- COPYRIGHT

NAME

Infoblox::DTC - A DTC functions.

DESCRIPTION

The object provides access to DTC functions.

CONSTRUCTOR

```
my $dtc = Infoblox::DTC->new(
    'session' => $session,       # Required
);
```

SESSION METHODS

The object does not support any session methods.

METHODS

query( )

Use this method to perform a DTC query on a given member.

Parameter

Valid value is an Infoblox::DTC::Query::Parameters object.

Returns

The method returns an Infoblox::DTC::Query::Response object when operation succeeds, and returns 'false' when the operation fails.

Example

```
#perform DTC query
my $response = $dtc->query($params);
```

import_maxminddb( )

Use this method to upload the MaxMind database.

Key Reference

```
Apply the following attribute to upload a MaxMind Database:

    path - Required. The path to a tar archive that contains the MaxMind database.
```

Returns

The method returns "true" when an upload of a MaxMind Database has been performed successfully, and returns "false" when the operation fails.

Example

```
#upload MaxMind DB
my $res = $dtc->import_maxminddb(path => $path_to_maxminddb);
```

add_certificate( )

Use this method to upload a DTC certificate.

Key Reference

```
Apply the following attribute to upload a DTC certificate:
   path - Required. The path to the DTC certificate file.
```

Returns

The method returns 'true' when the upload of a DTC certificate has been performed successfully, and returns 'false' when the operation fails.

Example

```
#add certificate with a path given
my $res = $dtc->add_certificate(path => $path_to_cert);
```

SEE ALSO

Infoblox::DTC::Query::Parameters.

AUTHOR

Infoblox Inc. http://www.infoblox.com/

COPYRIGHT

Copyright (c) 2017 Infoblox Inc.

Infoblox::DTC::Pool - a DTC Pool object.

- NAME
- DESCRIPTION
- CONSTRUCTOR
- SESSION METHODS
  - Infoblox::Session->add( )
  - Infoblox::Session->get( )
  - Infoblox::Session->modify( )
  - Infoblox::Session->remove( )
  - Infoblox::Session->search( )
- METHODS
  - availability( )
  - comment( )
  - dynamic_ratio_alternate( )
  - dynamic_ratio_preferred( )
  - disable( )
  - extattrs( )
  - extensible_attributes( )
  - health( )
  - lb_alternate_method( )
  - lb_alternate_topology( )
  - lb_preferred_method( )
  - lb_preferred_topology( )
  - monitors( )
  - name( )
  - override_ttl( )
  - quorum( )
  - servers( )
  - ttl( )
- AUTHOR
- SEE ALSO
- COPYRIGHT

NAME

Infoblox::DTC::Pool - a DTC Pool object.

DESCRIPTION

A DTC Pool object.

CONSTRUCTOR

```
 my $pool = Infoblox::DTC::Pool->new(
    name                    => $string,,
#Required
    lb_preferred_method     => 'ALL_AVAILABLE' | 'GLOBAL_AVAILABILITY' | 'RATIO' |
                               'ROUND_ROBIN' | 'TOPOLOGY' | 'DYNAMIC_RATIO',
#Required
    disable                 => 'true' | 'false',
#Optional / Default is 'false'
    comment                 => $string,
#Optional / Default is undefined
    lb_preferred_topology   => $topology,
#Optional / Default is undefined
    lb_alternate_method     => 'ALL_AVAILABLE' | 'GLOBAL_AVAILABILITY' | 'RATIO' |
                               'ROUND_ROBIN' | 'TOPOLOGY' | 'DYNAMIC_RATIO',
#Optional / Default is undefined
    lb_alternate_topology   => $topology,
#Optional / Default is undefined
    availability            => 'ANY' | 'QUORUM' | 'ALL',
#Optional / Default is undefined
    quorum                  => $uint,
#Optional / Default is undefined
    ttl                     => $uint,
#Optional / Default is undefined
    override_ttl            => 'true' | 'false',
#Optional / Default is 'false'
    monitors                => [$monitor1, $monitor2, ...],
#Optional / Default is empty list
    servers                 => [$server1, $server2, ...],
#Optional / Default is empty list
```

```
    dynamic_ratio_preferred => $ratio_setting,
#Optional / Default is undefined
    dynamic_ratio_alternate => $ratio_setting,
#Optional / Default is undefined
    extattrs                => { $string => $extattr, ... },
#Optional / Default is undefined
    extensible_attributes   => { $string => $string | $num, $string => [ $string | $num, ... ], ... }
#Optional / Default is undefined
 );
```

You cannot set both extattrs and extensible_attributes attributes at the same time.

SESSION METHODS

This section describes all the methods in a Session module that you can apply to a DTC Pool object

Infoblox::Session->add( )

Use this method to add an object to the Infoblox appliance. See Infoblox::Session->add() for parameters and return values.

Example

```
#construct an object
my $pool = Infoblox::DTC::Pool->new(
   name  => 'pool1',
   lb_preferred_method => 'RATIO',
);

#submit for addition
my $response = $session->add($pool);
```

Infoblox::Session->get( )

Use this method to retrieve all the matching objects from the Infoblox appliance. See Infoblox::Session->get() for parameters and return values.

Key Reference

```
Apply the following attribute to get a specific object:

 name                - Optional. A DTC Pool name in a string format.
 comment             - Optional. A DTC Pool comment.
 extattrs            - Optional. A hash reference containing extensible attributes.
 extensible_attributes - Optional. A hash reference that contains extensible attributes.
 status_member       - Optional. A member which provides object health status, default is Grid.
```

Example

```
my @retrieved_objs = $session->get(
    object => 'Infoblox::DTC::Pool',
    name   => 'pool1',
);

my @retrieved_objs = $session->get(
    object => 'Infoblox::DTC::Pool',
    extensible_attributes => { 'Site' => 'Santa Clara' }
);
```

Infoblox::Session->modify( )

Use this method to modify an object in the Infoblox appliance. See Infoblox::Session->modify() for parameters and return values.

Example

```
#Use method to modify the comment.
$pool->comment('this is a modified comment');
#Submit modification
my $response = $session->modify($pool);
```

Infoblox::Session->remove( )

Use this method to remove an object from the Infoblox appliance. See Infoblox::Session->remove() for parameters and return values.

To remove a specific object, first use `get()` or `search()` to retrieve the specific object, and then submit this object for removal.

Example

```
#Get the objects with the same name
my @retrieved_objs = $session->get(
    object => 'Infoblox::DTC::Pool',
    name   => 'pool1',
);
#Find the desired object from the retrieved list.
my $desired_pool = $retrieved_objs[0];
#Submit for removal
my $response = $session->remove($desired_pool);
```

Infoblox::Session->search( )

Use this method to search for objects in the Infoblox appliance. See Infoblox::Session->search() for parameters and return values.

Key Reference

```
Apply the following attribute to get a specific object:

 name                 - Optional. A DTC Pool name in a string format.
 comment              - Optional. A DTC Pool comment.
 extattrs             - Optional. A hash reference containing extensible attributes.
 extensible_attributes - Optional. A hash reference that contains extensible attributes.
 status_member        - Optional. A member which provides object health status, default is Grid.
```

Example

```
my @retrieved_objs = $session->get(
    object => 'Infoblox::DTC::Pool',
    name   => 'pool1',
);

my @retrieved_objs = $session->get(
    object => 'Infoblox::DTC::Pool',
    extensible_attributes => { 'Site' => 'Santa Clara' }
);
```

METHODS

This section describes all the methods that you can use to set or retrieve the attribute values of the object.

availability( )

Use this method to set or retrieve an DTC Pool resources availability status.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

The valid values are 'ALL', 'ANY' and 'QUORUM'.

Returns

If you specified a parameter, the method returns 'true' when the modification succeeds, and returns 'false' when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#get availability value
my $availability = $pool->availability();

#modify availability value
$pool->availability('ANY');
```

comment( )

Use this method to set or retrieve a descriptive comment.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

The valid value is a comment in string format (UTF-8) with a maximum of 256 bytes.

Returns

If you specified a parameter, the method returns 'true' when the modification succeeds, and returns 'false' when the operation

fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#get comment value
my $comment = $pool->comment();

#modify comment value
$pool->comment('desired comment');
```

dynamic_ratio_alternate( )

Use this method to set or retrieve the alternate dynamic ratio load balancing settings.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

The valid value is an Infoblox::DTC::Pool::DynamicRatioSetting object.

Returns

If you specified a parameter, the method returns 'true' when the modification succeeds, and returns 'false' when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#get dynamic_ratio_alternate value
my $dynamic_ratio_alternate = $pool->dynamic_ratio_alternate();

#Construct dynamic_ratio_alternate value
my $dynamic_ratio = Infoblox::DTC::Pool::DynamicRatioSetting->new(
    method                => 'ROUND_TRIP_DELAY',
    invert_monitor_metric => 'true',
);

#modify dynamic_ratio_alternate value
$pool->dynamic_ratio_alternate($dynamic_ratio);
```

dynamic_ratio_preferred( )

Use this method to set or retrieve the preferred dynamic ratio load balancing settings.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

The valid value is an Infoblox::DTC::Pool::DynamicRatioSetting object.

Returns

If you specified a parameter, the method returns 'true' when the modification succeeds, and returns 'false' when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#get dynamic_ratio_preferred value
my $dynamic_ratio_preferred = $pool->dynamic_ratio_preferred();

#Construct dynamic_ratio_preferred value
my $dynamic_ratio = Infoblox::DTC::Pool::DynamicRatioSetting->new(
    method                => 'ROUND_TRIP_DELAY',
    invert_monitor_metric => 'true',
);

#modify dynamic_ratio_preferred value
$pool->dynamic_ratio_preferred($dynamic_ratio);
```

disable( )

Use this method to enable or disable a DTC Pool.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

Specify 'true' to disable a DTC Pool or 'false' to enable it. The default value is 'false'.

Returns

If you specified a parameter, the method returns 'true' when the modification succeeds, and returns 'false' when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#get disable value
my $disable = $pool->disable();

#modify disable value
$pool->disable('true');
```

extattrs( )

Use this method to set or retrieve the extensible attributes associated with a DTC Pool object.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

Valid value is a hash reference containing the names of extensible attributes and their associated values (Infoblox::Grid::Extattr objects).

Returns

If you specified a parameter, the method returns 'true' when the modification succeeds, and returns 'false' when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#get extattrs value
my $ref_extattrs = $pool->extattrs();

#Modify extattrs
$pool->extattrs({ 'Site' => $extattr1, 'Administrator' => $extattr2 });
```

extensible_attributes( )

Use this method to set or retrieve the extensible attributes associated with a DTC Pool object.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

For valid values for extensible attributes, see Infoblox::Grid::ExtensibleAttributeDef/Extensible Attribute Values.

Returns

If you specified a parameter, the method returns 'true' when the modification succeeds, and returns 'false' when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get extensible attributes
my $ref_extensible_attributes = $pool->extensible_attributes();

#Modify extensible attributes
$pool->extensible_attributes({'Site' => 'Santa Clara', 'Administrator' => ['Peter', 'Tom']});
```

health( )

Use this method to retrieve a DTC Pool health information. This is a read-only attribute.

Parameter

None

Returns

The valid return value is an Infoblox::DTC::Health object.

Example

```
#get health value
my $health = $pool->health();
```

lb_alternate_method( )

Use this method to set or retrieve the alternate load balancing method for the DTC Pool object.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

The valid values are 'ALL_AVAILABLE', 'DYNAMIC_RATIO', 'NONE', 'GLOBAL_AVAILABILITY', 'RATIO', 'ROUND_ROBIN' and 'TOPOLOGY'.

Returns

If you specified a parameter, the method returns 'true' when the modification succeeds, and returns 'false' when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#get lb_alternate_method value
my $lb_alternate_method = $pool->lb_alternate_method();

#modify lb_alternate_method value
$pool->lb_alternate_method('RATIO');
```

lb_alternate_topology( )

Use this method to set or retrieve topology rules for the alternate 'TOPOLOGY' load balancing method for the DTC Pool object.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Not that this field will be retrieved from the appliance only if alternate load balancing method is 'TOPOLOGY'.

Parameter

The valid value is an Infoblox::DTC::Topology object.

Returns

If you specified a parameter, the method returns 'true' when the modification succeeds, and returns 'false' when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#get lb_alternate_topology value
my $lb_alternate_topology = $pool->lb_alternate_topology();

#modify lb_alternate_topology value
$pool->lb_alternate_topology($lb_alternate_topology);
```

lb_preferred_method( )

Use this method to set or retrieve the preferred load balancing method for the DTC Pool object.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

The valid values are 'ALL_AVAILABLE', 'DYNAMIC_RATIO', 'GLOBAL_AVAILABILITY', 'RATIO', 'ROUND_ROBIN' and 'TOPOLOGY'.

Returns

If you specified a parameter, the method returns 'true' when the modification succeeds, and returns 'false' when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#get lb_preferred_method value
my $lb_preferred_method = $pool->lb_preferred_method();

#modify lb_preferred_method value
$pool->lb_preferred_method('RATIO');
```

lb_preferred_topology( )

Use this method to set or retrieve topology rules for the preferred 'TOPOLOGY' load balancing method for the DTC Pool object.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

The valid value is an Infoblox::DTC::Topology object.

Returns

If you specified a parameter, the method returns 'true' when the modification succeeds, and returns 'false' when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Not that this field will be retrieved from the appliance only if preferred load balancing method is 'TOPOLOGY'.

Example

```
#get lb_preferred_topology value
my $lb_preferred_topology = $pool->lb_preferred_topology();

#modify lb_preferred_topology value
$pool->lb_preferred_topology($lb_preferred_topology);
```

monitors( )

Use this method to set or retrieve health monitors related to a DTC Pool object.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

The valid value is an array which contains following objects: Infoblox::DTC::Monitor::TCP, Infoblox::DTC::Monitor::ICMP, Infoblox::DTC::Monitor::SIP, Infoblox::DTC::Monitor::HTTP, Infoblox::DTC::Monitor::PDP.

Returns

If you specified a parameter, the method returns 'true' when the modification succeeds, and returns 'false' when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#get monitors value
my @monitors = $pool->monitors();

#modify monitors value
$pool->monitors([$monitor1, $monitor2]);
```

name( )

Use this method to set or retrieve a DTC Pool name.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

The valid value is a desired name in a string format.

Returns

If you specified a parameter, the method returns 'true' when the modification succeeds, and returns 'false' when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#get name value
my $name = $pool->name();

#modify name value
$pool->name('pool1');
```

override_ttl( )

The override_ttl attribute controls whether the TTL value in the object is used, instead of the Grid default.

The override_ttl attribute can be specified explicitly. It is also set implicitly when ttl is set to a defined value.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

Specify 'true' to set override_ttl flag or 'false' to disable it. Default value is 'false'.

Returns

If you specified a parameter, the method returns 'true' when the modification succeeds, and returns 'false' when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#get override_ttl value
my $override_ttl = $pool->override_ttl();

#modify override_ttl value
$pool->override_ttl('true');
```

quorum( )

Use this method to set or retrieve the number of monitors that must report the resource as 'up' for 'QUORUM' availability mode to be available.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

The valid value is an unsigned integer.

Returns

If you specified a parameter, the method returns 'true' when the modification succeeds, and returns 'false' when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#get quorum value
my $quorum = $pool->quorum();

#modify quorum value
$pool->quorum(20);
```

servers( )

Use this method to set or retrieve a list of DTC Servers linked to a DTC Pool.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

The valid value is an array of Infoblox::DTC::Server::Link objects.

Returns

If you specified a parameter, the method returns 'true' when the modification succeeds, and returns 'false' when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#get servers value
my @servers = $pool->servers();

#modify servers value
$pool->servers([$server1, $server2]);
```

ttl( )

Use this method to set a TTL value.

Setting this method to a defined value implicitly sets the override_ttl method to 'true'. Setting the parameter to undefined causes the appliance to use the Grid default and automatically resets the override_ttl attribute to 'false'.

Note that when ttl is set to a defined value and override_ttl is set to 'false', the last operation takes precedence. Thus the sequence $object->ttl(10); $object->override_ttl('false'); will set override_ttl to 'false', and the sequence $object->override_ttl('false'); $object->ttl(10); will result in override_ttl='true'.

Parameter

The valid value is an unsigned integer.

Returns

If you specified a parameter, the method returns 'true' when the modification succeeds, and returns 'false' when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#get ttl value
my $ttl = $pool->ttl();

#modify ttl value
$pool->ttl(10);
```

AUTHOR

Infoblox Inc. http://www.infoblox.com/

SEE ALSO

Infoblox::Session, Infoblox::Session->add(), Infoblox::Session->get(), Infoblox::Session->modify(), Infoblox::Session->remove(), Infoblox::Session->search(), Infoblox::Grid::Extattr, Infoblox::Grid::ExtensibleAttributeDef/Extensible Attribute Values, Infoblox::DTC::Health, Infoblox::DTC::Pool::DynamicRatioSetting, Infoblox::DTC::Pool::Link, Infoblox::DTC::Server::Link, Infoblox::DTC::Topology, Infoblox::DTC::Monitor::TCP, Infoblox::DTC::Monitor::ICMP, Infoblox::DTC::Mo nitor::SIP, Infoblox::DTC::Monitor::HTTP, Infoblox::DTC::Monitor::PDP.

COPYRIGHT

Copyright (c) 2017 Infoblox Inc.

Infoblox::DTC::Health - A DTC health information object.

- NAME
- DESCRIPTION
- SESSION METHODS
- METHODS
  - availability( )
  - description( )
  - enabled_state( )
- AUTHOR
- SEE ALSO
- COPYRIGHT

NAME

Infoblox::DTC::Health - A DTC health information object.

DESCRIPTION

The DTC health object provides information about the health status for a specified object.

SESSION METHODS

The object does not support any session methods. It is retrieved as a part of the following objects: Infoblox::DTC::Pool, Infoblox::DTC::Server, Infoblox::DTC::LBDN.

METHODS

This section describes all the methods that you can use to set or retrieve the attribute values of the object.

availability( )

Use this method to retrieve the color status for availability. This is a read-only attribute.

Parameter

None

Returns

The valid return values are 'NONE', 'GREEN', 'YELLOW', 'RED', 'BLUE' and 'GRAY'.

Example

```
#get availability value
my $availability = $health->availability();
```

description( )

Use this method to retrieve the textual description of the object's status. This is a read-only attribute.

Parameter

None

Returns

The method returns the attribute value.

Example

```
#get description value
my $description = $health->description();
```

enabled_state( )

Use this method to retrieve the enabled state of the object. This is a read-only attribute.

Parameter

None

Returns

The valid return values are 'NONE', 'ENABLED', 'DISABLED' and 'DISABLED_BY_PARENT'.

Example

```
#get enabled_state value
my $enabled_state = $health->enabled_state();
```

AUTHOR

Infoblox Inc. http://www.infoblox.com/

SEE ALSO

Infoblox::DTC::Pool, Infoblox::DTC::Server, Infoblox::DTC::LBDN.

COPYRIGHT

Copyright (c) 2017 Infoblox Inc.

Infoblox::DTC::Pool::Link - An DTC Pool Link object.

- NAME
- DESCRIPTION
- CONSTRUCTOR
- SESSION METHODS
- METHODS
  - dtc_pool( )
  - ratio( )
- AUTHOR
- SEE ALSO
- COPYRIGHT

NAME

Infoblox::DTC::Pool::Link - An DTC Pool Link object.

DESCRIPTION

An DTC Pool Link object.

CONSTRUCTOR

```
my $pool_link = Infoblox::DTC::Pool::Link->new(
    dtc_pool => $pool, #Optional / Default is undefined
    ratio    => $uint, #Optional / Default is undefined
);
```

SESSION METHODS

The object does not support any session methods.

METHODS

This section describes all the methods that you can use to set or retrieve the attribute values of the object.

dtc_pool( )

Use this method to set or retrieve the associated DTC Pool.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

The valid value is an Infoblox::DTC::Pool object.

Returns

If you specified a parameter, the method returns 'true' when the modification succeeds, and returns 'false' when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#get dtc_pool value
my $pool = $pool_link->dtc_pool();

#modify dtc_pool value
$pool_link->dtc_pool($pool);
```

ratio( )

Use this method to set or retrieve the weight of a DTC Pool.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

The valid value is an unsigned integer.

Returns

If you specified a parameter, the method returns 'true' when the modification succeeds, and returns 'false' when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#get ratio value
my $ratio = $pool_link->ratio();

#modify ratio value
$pool_link->ratio(70);
```

AUTHOR

Infoblox Inc. http://www.infoblox.com/

SEE ALSO

Infoblox::DTC::Pool.

COPYRIGHT

Copyright (c) 2017 Infoblox Inc.

Infoblox::DTC::Topology - A DTC Topology object.

- NAME
- DESCRIPTION
- CONSTRUCTOR
- SESSION METHODS
  - Infoblox::Session->add( )
  - Infoblox::Session->get( )
  - Infoblox::Session->modify( )
  - Infoblox::Session->remove( )
  - Infoblox::Session->search( )
- METHODS
  - comment( )
  - extattrs( )
  - extensible_attributes( )
  - name( )
  - rules( )
- AUTHOR
- SEE ALSO
- COPYRIGHT

NAME

Infoblox::DTC::Topology - A DTC Topology object.

DESCRIPTION

A DTC Topology is a named list of ordered DTC topology rules. These rules cannot be shared. Topology rules map client IPs to pools or resources.

CONSTRUCTOR

```
 my $topology = Infoblox::DTC::Topology->new(
    name                  => $string,
#Required
    comment               => $string,
#Optional / Default is undefined
    rules                 => [$rule1, $rule2, ...],
#Optional / Default is undefined
    extattrs              => { $string => $extattr, ... },
#Optional / Default is undefined
    extensible_attributes => { $string => $string | $num, $string => [ $string | $num, ... ], ... }
#Optional / Default is undefined
 );
```

You cannot set both extattrs and extensible_attributes attributes at the same time.

SESSION METHODS

This section describes all the methods in a Session module that you can apply to a Topology object.

Infoblox::Session->add( )

Use this method to add an object to the Infoblox appliance. See Infoblox::Session->add() for parameters and return values.

Example

```
#construct an object
my $topology = Infoblox::DTC::Topology->new(
   name  => 'topology',
   rules => [$rule1, $rule2, ...],
);

#submit for addition
my $response = $session->add($topology);
```

Infoblox::Session->get( )

Use this method to retrieve all the matching objects from the Infoblox appliance. See Infoblox::Session->get() for parameters and return values.

Key Reference

```
Apply the following attribute to get a specific object:

 name                  - Optional. A DTC Topology name in a string format.
 comment               - Optional. A DTC Topology comment.
 extattrs              - Optional. A hash reference containing extensible attributes.
 extensible_attributes - Optional. A hash reference that contains extensible attributes.
```

Example

```
my @retrieved_objs = $session->get(
    object => 'Infoblox::DTC::Topology',
    name   => 'topology',
);

my @retrieved_objs = $session->get(
    object => 'Infoblox::DTC::Topology',
    extensible_attributes => { 'Site' => 'Santa Clara' }
);
```

Infoblox::Session->modify( )

Use this method to modify an object in the Infoblox appliance. See Infoblox::Session->modify() for parameters and return values.

Example

```
#Use method to modify the comment
$topology->comment('this is a modified comment');
#Submit modification
my $response = $session->modify($topology);
```

Infoblox::Session->remove( )

Use this method to remove an object from the Infoblox appliance. See Infoblox::Session->remove() for parameters and return values.

To remove a specific object, first use `get()` or `search()` to retrieve the specific object, and then submit this object for removal.

Example

```
#Get the objects with the same name
my @retrieved_objs = $session->get(
    object => 'Infoblox::DTC::Topology',
    name   => 'topology',
);
#Find the desired object from the retrieved list.
my $desired_topology = $retrieved_objs[0];
#Submit for removal
my $response = $session->remove($desired_topology);
```

Infoblox::Session->search( )

Use this method to search for objects in the Infoblox appliance. See Infoblox::Session->search() for parameters and return values.

Key Reference

```
Apply the following attribute to get a specific object:

 name                  - Optional. A DTC Topology name in a string format.
 comment               - Optional. A DTC Topology comment.
 extattrs              - Optional. A hash reference containing extensible attributes.
 extensible_attributes - Optional. A hash reference that contains extensible attributes.
```

Example

```
my @retrieved_objs = $session->search(
    object => 'Infoblox::DTC::Topology',
    name   => 'top.*',
);

my @retrieved_objs = $session->search(
    object => 'Infoblox::DTC::Topology',
    extensible_attributes => { 'Site' => 'Santa Clara' }
);
```

METHODS

This section describes all the methods that you can use to set or retrieve the attribute values of the object.

comment( )

Use this method to set or retrieve a descriptive comment.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

Desired comment in a string format (UTF-8) with a maximum of 256 bytes.

Returns

If you specified a parameter, the method returns 'true' when the modification succeeds, and returns 'false' when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#get comment value
my $comment = $topology->comment();

#modify comment value
$topology->comment('desired comment');
```

extattrs( )

Use this method to set or retrieve the extensible attributes associated with a DTC Topology object.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

Valid value is a hash reference containing the names of extensible attributes and their associated values (Infoblox::Grid::Extattr objects).

Returns

If you specified a parameter, the method returns 'true' when the modification succeeds, and returns 'false' when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#get extattrs value
my $ref_extattrs = $topology->extattrs();

#Modify extattrs
$topology->extattrs({ 'Site' => $extattr1, 'Administrator' => $extattr2 });
```

extensible_attributes( )

Use this method to set or retrieve the extensible attributes associated with a DTC Topology object.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

For valid values for extensible attributes, see Infoblox::Grid::ExtensibleAttributeDef/Extensible Attribute Values.

Returns

If you specified a parameter, the method returns 'true' when the modification succeeds, and returns 'false' when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get extensible attributes
my $ref_extensible_attributes = $topology->extensible_attributes();

#Modify extensible attributes
$topology->extensible_attributes({'Site' => 'Santa Clara', 'Administrator' => ['Peter', 'Tom']});
```

name( )

Use this method to set or retrieve topology name.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

The valid value is a topology list name in a string format.

Returns

If you specified a parameter, the method returns 'true' when the modification succeeds, and returns 'false' when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#get name value
my $name = $topology->name();

#modify name value
$topology->name('topology1');
```

rules( )

Use this method to set or retrieve a list of topology rules.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

The valid value is an Infoblox::DTC::Topology::Rule object.

Returns

If you specified a parameter, the method returns 'true' when the modification succeeds, and returns 'false' when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#get rules value
my $rules = $topology->rules();

#modify rules value
$topology->rules([$rule1, $rule2]);
```

AUTHOR

Infoblox Inc. http://www.infoblox.com/

SEE ALSO

Infoblox::Session, Infoblox::Session->add(), Infoblox::Session->get(), Infoblox::Session->modify(), Infoblox::Session->remove(), Infoblox::Session->search(), Infoblox::Grid::Extattr, Infoblox::Grid::ExtensibleAttributeDef/Extensible Attribute Values, Infoblox::DTC::Topology::Rule.

COPYRIGHT

Copyright (c) 2017 Infoblox Inc.

Infoblox::DTC::Monitor::TCP - A DTC TCP Health Monitor object.

- NAME
- DESCRIPTION
- CONSTRUCTOR
- SESSION METHODS
  - Infoblox::Session->add( )
  - Infoblox::Session->get( )
  - Infoblox::Session->modify( )
  - Infoblox::Session->remove( )
  - Infoblox::Session->search( )
- METHODS
  - comment( )
  - extattrs( )
  - extensible_attributes( )
  - interval( )
  - name( )
  - port( )
  - retry_down( )
  - retry_up( )
  - timeout( )
- AUTHOR
- SEE ALSO
- COPYRIGHT

NAME

Infoblox::DTC::Monitor::TCP - A DTC TCP Health Monitor object.

DESCRIPTION

A DTC TCP Health Monitor object.

CONSTRUCTOR

```
 my $tcp_monitor = Infoblox::DTC::Monitor::TCP->new(
    name                  => $string,
#Required
    comment               => $string,
#Optional / Default is undefined
    interval              => $uint,
#Optional / Default is undefined
    port                  => $uint,
#Required
    retry_down            => $uint,
#Optional / Default is undefined
    retry_up              => $uint,
#Optional / Default is undefined
    timeout               => $uint,
#Optional / Default is undefined
    extattrs              => { $string => $extattr, ... },
#Optional / Default is undefined
    extensible_attributes => { $string => $string | $num, $string => [ $string | $num, ... ], ... }
#Optional / Default is undefined
 );
```

You cannot set both extattrs and extensible_attributes attributes at the same time.

SESSION METHODS

This section describes all the methods in a Session module that you can apply to a DTC TCP Monitor object.

Infoblox::Session->add( )

Use this method to add an object to the Infoblox appliance. See Infoblox::Session->add() for parameters and return values.

Example

```
#construct an object
my $tcp_monitor = Infoblox::DTC::Monitor::TCP->new(
   name  => 'tcp_monitor1',
```

```
    port  => 15,
);

#submit for addition
my $response = $session->add($tcp_monitor);
```

Infoblox::Session->get( )

Use this method to retrieve all the matching objects from the Infoblox appliance. See Infoblox::Session->get() for parameters and return values.

Key Reference

```
Apply the following attribute to get a specific object:

 name                - Optional. A DTC TCP Monitor name in a string format.
 comment             - Optional. A DTC TCP Monitor comment.
 extattrs            - Optional. A hash reference containing extensible attributes.
 extensible_attributes - Optional. A hash reference that contains extensible attributes.
```

Example

```
my @retrieved_objs = $session->get(
    object => 'Infoblox::DTC::Monitor::TCP',
    name   => 'tcp_monitor1',
);

my @retrieved_objs = $session->get(
    object => 'Infoblox::DTC::Monitor::TCP',
    extensible_attributes => { 'Site' => 'Santa Clara' }
);
```

Infoblox::Session->modify( )

Use this method to modify an object in the Infoblox appliance. See Infoblox::Session->modify() for parameters and return values.

Example

```
#Use method to modify the comment.
$tcp_monitor->comment('this is a modified comment');
#Submit modification
my $response = $session->modify($tcp_monitor);
```

Infoblox::Session->remove( )

Use this method to remove an object from the Infoblox appliance. See Infoblox::Session->remove() for parameters and return values.

To remove a specific object, first use `get()` or `search()` to retrieve the specific object, and then submit this object for removal.

Example

```
#Get the objects with the same name
my @retrieved_objs = $session->get(
    object => 'Infoblox::DTC::Monitor::TCP',
    name   => 'tcp_monitor1',
);
#Find the desired object from the retrieved list.
my $desired_tcp_monitor = $retrieved_objs[0];
#Submit for removal
my $response = $session->remove($desired_tcp_monitor);
```

Infoblox::Session->search( )

Use this method to search for objects in the Infoblox appliance. See Infoblox::Session->search() for parameters and return values.

Key Reference

```
Apply the following attribute to get a specific object:

 name                - Optional. A DTC TCP Monitor name in a string format.
 comment             - Optional. A DTC TCP Monitor comment.
 extattrs            - Optional. A hash reference containing extensible attributes.
 extensible_attributes - Optional. A hash reference that contains extensible attributes.
```

Example

```
my @retrieved_objs = $session->search(
    object => 'Infoblox::DTC::Monitor::TCP',
```

```
    name   => 'tcp_monitor1',
);

my @retrieved_objs = $session->search(
    object => 'Infoblox::DTC::Monitor::TCP',
    extensible_attributes => { 'Site' => 'Santa Clara' }
);
```

METHODS

This section describes all the methods that you can use to set or retrieve the attribute values of the object.

comment( )

Use this method to set or retrieve a DTC TCP Health Monitor object comment.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

The valid value is a comment in string format (UTF-8) with a maximum of 256 bytes.

Returns

If you specified a parameter, the method returns 'true' when the modification succeeds, and returns 'false' when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#get comment value
my $comment = $tcp_monitor->comment();

#modify comment value
$tcp_monitor->comment('desired comment');
```

extattrs( )

Use this method to set or retrieve the extensible attributes associated with a DTC TCP Health Monitor object.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

Valid value is a hash reference containing the names of extensible attributes and their associated values (Infoblox::Grid::Extattr objects).

Returns

If you specified a parameter, the method returns 'true' when the modification succeeds, and returns 'false' when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#get extattrs value
my $ref_extattrs = $tcp_monitor->extattrs();

#Modify extattrs
$tcp_monitor->extattrs({ 'Site' => $extattr1, 'Administrator' => $extattr2 });
```

extensible_attributes( )

Use this method to set or retrieve the extensible attributes associated with a DTC TCP Health Monitor object.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

For valid values for extensible attributes, see Infoblox::Grid::ExtensibleAttributeDef/Extensible Attribute Values.

Returns

If you specified a parameter, the method returns 'true' when the modification succeeds, and returns 'false' when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get extensible attributes
my $ref_extensible_attributes = $tcp_monitor->extensible_attributes();

#Modify extensible attributes
$tcp_monitor->extensible_attributes({'Site' => 'Santa Clara', 'Administrator' => ['Peter', 'Tom']});
```

interval( )

Use this method to set or retrieve the time interval for the TCP health check.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

The valid value is an unsigned integer.

Returns

If you specified a parameter, the method returns 'true' when the modification succeeds, and returns 'false' when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#get interval value
my $interval = $tcp_monitor->interval();

#modify interval value
$tcp_monitor->interval(10);
```

name( )

Use this method to set or retrieve a DTC TCP Health Monitor name.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

The valid value is a desired name in a string format.

Returns

If you specified a parameter, the method returns 'true' when the modification succeeds, and returns 'false' when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#get name value
my $name = $tcp_monitor->name();

#modify name value
$tcp_monitor->name('tcp_monitor1');
```

port( )

Use this method to set or retrieve the port value for TCP requests.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

The valid value is an unsigned integer between 1 and 65535.

Returns

If you specified a parameter, the method returns 'true' when the modification succeeds, and returns 'false' when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#get port value
my $port = $tcp_monitor->port();

#modify port value
$tcp_monitor->port(6030);
```

retry_down( )

Use this method to set or retrieve the number of times the server appears offline after it was online so it is treated as dead.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

The valid value is an unsigned integer between 1 and 10.

Returns

If you specified a parameter, the method returns 'true' when the modification succeeds, and returns 'false' when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#get retry_down value
my $retry_down = $tcp_monitor->retry_down();

#modify retry_down value
$tcp_monitor->retry_down(3);
```

retry_up( )

Use this method to set or retrieve the number of times the server appears online after it was offline so it is treated as alive.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

The valid value is an unsigned integer between 1 and 10.

Returns

If you specified a parameter, the method returns 'true' when the modification succeeds, and returns 'false' when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#get retry_up value
my $retry_up = $tcp_monitor->retry_up();

#modify retry_up value
$tcp_monitor->retry_up(3);
```

timeout( )

Use this method to set or retrieve the value of a timeout for TCP health check.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

The valid value is an unsigned integer between 1 and 15.

Returns

If you specified a parameter, the method returns 'true' when the modification succeeds, and returns 'false' when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#get timeout value
my $timeout = $tcp_monitor->timeout();

#modify timeout value
```

```
$tcp_monitor->timeout(7);
```

AUTHOR

Infoblox Inc. http://www.infoblox.com/

SEE ALSO

Infoblox::Session, Infoblox::Session->add(), Infoblox::Session->get(), Infoblox::Session->modify(), Infoblox::Session->remove(), Infoblox::Session->search(), Infoblox::Grid::Extattr, Infoblox::Grid::ExtensibleAttributeDef/Extensible Attribute Values.

COPYRIGHT

Copyright (c) 2017 Infoblox Inc.

Infoblox::DTC::Monitor::ICMP - A DTC ICMP Health Monitor object.

- NAME
- DESCRIPTION
- CONSTRUCTOR
- SESSION METHODS
  - Infoblox::Session->add( )
  - Infoblox::Session->get( )
  - Infoblox::Session->modify( )
  - Infoblox::Session->remove( )
  - Infoblox::Session->search( )
- METHODS
  - comment( )
  - extattrs( )
  - extensible_attributes( )
  - interval( )
  - name( )
  - retry_down( )
  - retry_up( )
  - timeout( )
- AUTHOR
- SEE ALSO
- COPYRIGHT

NAME

Infoblox::DTC::Monitor::ICMP - A DTC ICMP Health Monitor object.

DESCRIPTION

A DTC ICMP Health Monitor object.

CONSTRUCTOR

```
 my $icmp_monitor = Infoblox::DTC::Monitor::ICMP->new(
    name                  => $string,
#Required
    comment               => $string,
#Optional / Default is undefined
    interval              => $uint,
#Optional / Default is undefined
    retry_down            => $uint,
#Optional / Default is undefined
    retry_up              => $uint,
#Optional / Default is undefined
    timeout               => $uint,
#Optional / Default is undefined
    extattrs              => { $string => $extattr, ... },
#Optional / Default is undefined
    extensible_attributes => { $string => $string | $num, $string => [ $string | $num, ... ], ... }
#Optional / Default is undefined
 );
```

You cannot set both extattrs and extensible_attributes attributes at the same time.

SESSION METHODS

This section describes all the methods in a Session module that you can apply to a DTC ICMP Monitor object

Infoblox::Session->add( )

Use this method to add an object to the Infoblox appliance. See Infoblox::Session->add() for parameters and return values.

Example

```
#construct an object
my $icmp_monitor = Infoblox::DTC::Monitor::ICMP->new(
   name  => 'icmp_monitor1',
);

#submit for addition
```

```
my $response = $session->add($icmp_monitor);
```

Infoblox::Session->get( )

Use this method to retrieve all the matching objects from the Infoblox appliance. See Infoblox::Session->get() for parameters and return values.

Key Reference

```
Apply the following attribute to get a specific object:

 name                  - Optional. A DTC ICMP Monitor name in a string format.
 comment               - Optional. A DTC ICMP Monitor comment.
 extattrs              - Optional. A hash reference containing extensible attributes.
 extensible_attributes - Optional. A hash reference that contains extensible attributes.
```

Example

```
my @retrieved_objs = $session->get(
    object => 'Infoblox::DTC::Monitor::ICMP',
    name   => 'icmp_monitor1',
);

my @retrieved_objs = $session->get(
    object => 'Infoblox::DTC::Monitor::ICMP',
    extensible_attributes => { 'Site' => 'Santa Clara' }
);
```

Infoblox::Session->modify( )

Use this method to modify an object in the Infoblox appliance. See Infoblox::Session->modify() for parameters and return values.

Example

```
#Use method to modify the comment.
$icmp_monitor->comment('this is a modified comment');
#Submit modification
my $response = $session->modify($icmp_monitor);
```

Infoblox::Session->remove( )

Use this method to remove an object from the Infoblox appliance. See Infoblox::Session->remove() for parameters and return values.

To remove a specific object, first use `get()` or `search()` to retrieve the specific object, and then submit this object for removal.

Example

```
#Get the objects with the same name
my @retrieved_objs = $session->get(
    object => 'Infoblox::DTC::Monitor::ICMP',
    name   => 'icmp_monitor1',
);
#Find the desired object from the retrieved list.
my $desired_icmp_monitor = $retrieved_objs[0];
#Submit for removal
my $response = $session->remove($desired_icmp_monitor);
```

Infoblox::Session->search( )

Use this method to search for objects in the Infoblox appliance. See Infoblox::Session->search() for parameters and return values.

Key Reference

```
Apply the following attribute to get a specific object:

 name                  - Optional. A DTC ICMP Monitor name in a string format.
 comment               - Optional. A DTC ICMP Monitor comment.
 extattrs              - Optional. A hash reference containing extensible attributes.
 extensible_attributes - Optional. A hash reference that contains extensible attributes.
```

Example

```
my @retrieved_objs = $session->search(
    object => 'Infoblox::DTC::Monitor::ICMP',
    name   => 'icmp_monitor1',
);

my @retrieved_objs = $session->search(
```

```
        object => 'Infoblox::DTC::Monitor::ICMP',
        extensible_attributes => { 'Site' => 'Santa Clara' }
    );
```

METHODS

This section describes all the methods that you can use to set or retrieve the attribute values of the object.

comment( )

Use this method to set or retrieve a DTC ICMP Health Monitor object comment.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

The valid value is a comment in string format (UTF-8) with a maximum of 256 bytes.

Returns

If you specified a parameter, the method returns 'true' when the modification succeeds, and returns 'false' when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#get comment value
my $comment = $icmp_monitor->comment();

#modify comment value
$icmp_monitor->comment('desired comment');
```

extattrs( )

Use this method to set or retrieve the extensible attributes associated with a DTC ICMP Health Monitor object.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

Valid value is a hash reference containing the names of extensible attributes and their associated values (Infoblox::Grid::Extattr objects).

Returns

If you specified a parameter, the method returns 'true' when the modification succeeds, and returns 'false' when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#get extattrs value
my $ref_extattrs = $icmp_monitor->extattrs();

#Modify extattrs
$icmp_monitor->extattrs({ 'Site' => $extattr1, 'Administrator' => $extattr2 });
```

extensible_attributes( )

Use this method to set or retrieve the extensible attributes associated with a DTC ICMP Health Monitor object.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

For valid values for extensible attributes, see Infoblox::Grid::ExtensibleAttributeDef/Extensible Attribute Values.

Returns

If you specified a parameter, the method returns 'true' when the modification succeeds, and returns 'false' when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get extensible attributes
my $ref_extensible_attributes = $icmp_monitor->extensible_attributes();

#Modify extensible attributes
$icmp_monitor->extensible_attributes({'Site' => 'Santa Clara', 'Administrator' => ['Peter',
'Tom']});
```

interval( )

Use this method to set or retrieve the time interval for the ICMP health check.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

The valid value is an unsigned integer.

Returns

If you specified a parameter, the method returns 'true' when the modification succeeds, and returns 'false' when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#get interval value
my $interval = $icmp_monitor->interval();

#modify interval value
$icmp_monitor->interval(10);
```

name( )

Use this method to set or retrieve a DTC ICMP Health Monitor name.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

The valid value is a desired name in a string format.

Returns

If you specified a parameter, the method returns 'true' when the modification succeeds, and returns 'false' when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#get name value
my $name = $icmp_monitor->name();

#modify name value
$icmp_monitor->name('icmp_monitor1');
```

retry_down( )

Use this method to set or retrieve the number of times the server appears offline after it was online so it is treated as dead.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

The valid value is an unsigned integer between 1 and 10.

Returns

If you specified a parameter, the method returns 'true' when the modification succeeds, and returns 'false' when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#get retry_down value
my $retry_down = $icmp_monitor->retry_down();
```

```
#modify retry_down value
$icmp_monitor->retry_down(3);
```

retry_up( )

Use this method to set or retrieve the number of times the server appears online after it was offline so it is treated as alive.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

The valid value is an unsigned integer between 1 and 10.

Returns

If you specified a parameter, the method returns 'true' when the modification succeeds, and returns 'false' when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#get retry_up value
my $retry_up = $icmp_monitor->retry_up();

#modify retry_up value
$icmp_monitor->retry_up(3);
```

timeout( )

Use this method to set or retrieve the value of a timeout for ICMP health check.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

The valid value is an unsigned integer between 1 and 15.

Returns

If you specified a parameter, the method returns 'true' when the modification succeeds, and returns 'false' when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#get timeout value
my $timeout = $icmp_monitor->timeout();

#modify timeout value
$icmp_monitor->timeout(7);
```

AUTHOR

Infoblox Inc. http://www.infoblox.com/

SEE ALSO

Infoblox::Session, Infoblox::Session->add(), Infoblox::Session->get(), Infoblox::Session->modify(), Infoblox::Session->remove(), Infoblox::Session->search(), Infoblox::Grid::Extattr, Infoblox::Grid::ExtensibleAttributeDef/Extensible Attribute Values.

COPYRIGHT

Copyright (c) 2017 Infoblox Inc.

Infoblox::DTC::Monitor::SIP - A DTC SIP Health Monitor object.

- NAME
- DESCRIPTION
- CONSTRUCTOR
- SESSION METHODS
  - Infoblox::Session->add( )
  - Infoblox::Session->get( )
  - Infoblox::Session->modify( )
  - Infoblox::Session->remove( )
  - Infoblox::Session->search( )
- METHODS
  - ciphers( )
  - client_cert( )
  - comment( )
  - extattrs( )
  - extensible_attributes( )
  - interval( )
  - name( )
  - port( )
  - request( )
  - result( )
  - result_code( )
  - retry_down( )
  - retry_up( )
  - timeout( )
  - transport( )
  - validate_cert( )
- AUTHOR
- SEE ALSO
- COPYRIGHT

NAME

Infoblox::DTC::Monitor::SIP - A DTC SIP Health Monitor object.

DESCRIPTION

The DTC SIP Health Monitor determines the health of a SIP servers such as SIP Proxies and Session Border Controllers, and SIP gateways by issuing SIP options to a server and examining the response provided by the server. The service is considered available if the received response matches the expected response.

CONSTRUCTOR

```
 my $sip_monitor = Infoblox::DTC::Monitor::SIP->new(
    name                => $string,
#Required
    ciphers             => [$cipher1, $cipher2, ...],
#Optional / Default is undefined
    client_cert         => $certificate
#Optional / Default is undefined
    comment             => $string,
#Optional / Default is undefined
    interval            => $uint,
#Optional / Default is undefined
    port                => $uint,
#Optional / Default is undefined
    request             => $string,
#Optional / Default is undefined
    result              => 'ANY' | 'CODE_IS' | 'CODE_IS_NOT',
#Optional / Default is undefined
    result_code         => $uint,
#Optional / Default is undefined
    retry_down          => $uint,
#Optional / Default is undefined
    retry_up            => $uint,
#Optional / Default is undefined
    timeout             => $uint,
#Optional / Default is undefined
    transport           => 'TCP' | 'UDP' | 'TLS' | 'SIPS',
#Optional / Default is undefined
    validate_cert       => 'true' | 'false',
#Optional / Default is 'false'
```

```
    extattrs              => { $string => $extattr, ... },
#Optional / Default is undefined
    extensible_attributes => { $string => $string | $num, $string => [ $string | $num, ... ], ... }
#Optional / Default is undefined
 );
```

You cannot set both extattrs and extensible_attributes attributes at the same time.

SESSION METHODS

This section describes all the methods in a Session module that you can apply to a DTC SIP Monitor object

Infoblox::Session->add( )

Use this method to add an object to the Infoblox appliance. See Infoblox::Session->add() for parameters and return values.

Example

```
#construct an object
my $sip_monitor = Infoblox::DTC::Monitor::SIP->new(
   name  => 'sip_monitor1',
);

#submit for addition
my $response = $session->add($sip_monitor);
```

Infoblox::Session->get( )

Use this method to retrieve all the matching objects from the Infoblox appliance. See Infoblox::Session->get() for parameters and return values.

Key Reference

```
Apply the following attribute to get a specific object:

 name                  - Optional. A DTC SIP Monitor name in a string format.
 comment               - Optional. A DTC SIP Monitor comment.
 extattrs              - Optional. A hash reference containing extensible attributes.
 extensible_attributes - Optional. A hash reference that contains extensible attributes.
```

Example

```
my @retrieved_objs = $session->get(
    object => 'Infoblox::DTC::Monitor::SIP',
    name   => 'sip_monitor1',
);

my @retrieved_objs = $session->get(
    object => 'Infoblox::DTC::Monitor::SIP',
    extensible_attributes => { 'Site' => 'Santa Clara' }
);
```

Infoblox::Session->modify( )

Use this method to modify an object in the Infoblox appliance. See Infoblox::Session->modify() for parameters and return values.

Example

```
#Use method to modify the comment.
$sip_monitor->comment('this is a modified comment');
#Submit modification
my $response = $session->modify($sip_monitor);
```

Infoblox::Session->remove( )

Use this method to remove an object from the Infoblox appliance. See Infoblox::Session->remove() for parameters and return values.

To remove a specific object, first use `get()` or `search()` to retrieve the specific object, and then submit this object for removal.

Example

```
#Get the objects with the same name
my @retrieved_objs = $session->get(
    object => 'Infoblox::DTC::Monitor::SIP',
    name   => 'sip_monitor1',
);
#Find the desired object from the retrieved list.
```

```
my $desired_sip_monitor = $retrieved_objs[0];
#Submit for removal
my $response = $session->remove($desired_sip_monitor);
```

Infoblox::Session->search( )

Use this method to search for objects in the Infoblox appliance. See Infoblox::Session->search() for parameters and return values.

Key Reference

```
Apply the following attribute to get a specific object:

 name                 - Optional. A DTC SIP Monitor name in a string format.
 comment              - Optional. A DTC SIP Monitor Descriptive comment.
 extattrs             - Optional. A hash reference containing extensible attributes.
 extensible_attributes - Optional. A hash reference that contains extensible attributes.
```

Example

```
my @retrieved_objs = $session->search(
    object => 'Infoblox::DTC::Monitor::SIP',
    name   => 'sip_monitor1',
);

my @retrieved_objs = $session->search(
    object => 'Infoblox::DTC::Monitor::SIP',
    extensible_attributes => { 'Site' => 'Santa Clara' }
);
```

METHODS

This section describes all the methods that you can use to set or retrieve the attribute values of the object.

ciphers( )

Use this method to set or retrieve a list of ciphers for a secure TLS or SIPS connection.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

The valid value is an array of ciphers in a string format. Ciphers use an OpenSSL syntax. Empty array is equivalent to 'ALL' in an OpenSSL notation.

Returns

If you specified a parameter, the method returns 'true' when the modification succeeds, and returns 'false' when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#get ciphers value
my @ciphers = $sip_monitor->ciphers();

#modify ciphers value
$sip_monitor->ciphers(['RC4-MD5', 'KRB5-RC4-MD5']);
```

client_cert( )

Use this method to set or retrieve a client certificate, supplied in a secure TLS or SIPS mode if present.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

The valid value is an Infoblox::DTC::Certificate object.

Returns

If you specified a parameter, the method returns 'true' when the modification succeeds, and returns 'false' when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#get client_cert value
my $cert = $sip_monitor->client_cert();

#modify client_cert value
$sip_monitor->client_cert($cert);
```

comment( )

Use this method to set or retrieve the comment for a DTC SIP Health Monitor object.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

The valid value is a comment in string format (UTF-8) with a maximum of 256 bytes.

Returns

If you specified a parameter, the method returns 'true' when the modification succeeds, and returns 'false' when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#get comment value
my $comment = $sip_monitor->comment();

#modify comment value
$sip_monitor->comment('desired comment');
```

extattrs( )

Use this method to set or retrieve the extensible attributes associated with a DTC SIP Health Monitor object.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

Valid value is a hash reference containing the names of extensible attributes and their associated values (Infoblox::Grid::Extattr objects).

Returns

If you specified a parameter, the method returns 'true' when the modification succeeds, and returns 'false' when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#get extattrs value
my $ref_extattrs = $sip_monitor->extattrs();

#Modify extattrs
$sip_monitor->extattrs({ 'Site' => $extattr1, 'Administrator' => $extattr2 });
```

extensible_attributes( )

Use this method to set or retrieve the extensible attributes associated with a DTC SIP Health Monitor object.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

For valid values for extensible attributes, see Infoblox::Grid::ExtensibleAttributeDef/Extensible Attribute Values.

Returns

If you specified a parameter, the method returns 'true' when the modification succeeds, and returns 'false' when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get extensible attributes
my $ref_extensible_attributes = $sip_monitor->extensible_attributes();
```

```
#Modify extensible attributes
$sip_monitor->extensible_attributes({'Site' => 'Santa Clara', 'Administrator' => ['Peter', 'Tom']});
```

interval( )

Use this method to set or retrieve the time interval for the SIP health check.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

The valid value is an unsigned integer.

Returns

If you specified a parameter, the method returns 'true' when the modification succeeds, and returns 'false' when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#get interval value
my $interval = $sip_monitor->interval();

#modify interval value
$sip_monitor->interval(10);
```

name( )

Use this method to set or retrieve a DTC SIP Health Monitor name.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

The valid value is a desired name in a string format.

Returns

If you specified a parameter, the method returns 'true' when the modification succeeds, and returns 'false' when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#get name value
my $name = $sip_monitor->name();

#modify name value
$sip_monitor->name('sip_monitor1');
```

port( )

Use this method to set or retrieve the port value for SIP or SIPS requests.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

The valid value is an unsigned integer between 1 and 65535.

Returns

If you specified a parameter, the method returns 'true' when the modification succeeds, and returns 'false' when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#get port value
my $port = $sip_monitor->port();

#modify port value
$sip_monitor->port(8080);
```

request( )

Use this method to set or retrieve an SIP request to send.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

The valid value is a string of 1024 characters maximum.

Returns

If you specified a parameter, the method returns 'true' when the modification succeeds, and returns 'false' when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#get request value
my $request = $sip_monitor->request();

#modify request value
$sip_monitor->request('GET /');
```

result( )

Use this method to set or retrieve the type of expected result.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

The valid values are 'ANY', 'CODE_IS' and 'CODE_IS_NOT'.

Returns

If you specified a parameter, the method returns 'true' when the modification succeeds, and returns 'false' when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#get result value
my $result = $sip_monitor->result();

#modify result value
$sip_monitor->result('CODE_IS');
```

result_code( )

Use this method to set or retrieve the expected return code.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

The valid value is an unsigned integer between 0 and 999.

Returns

If you specified a parameter, the method returns 'true' when the modification succeeds, and returns 'false' when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#get result_code value
my $result_code = $sip_monitor->result_code();

#modify result_code value
$sip_monitor->result_code(300);
```

retry_down( )

Use this method to set or retrieve the number of times the server appears offline after it was online so it is treated as dead.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

The valid value is an unsigned integer between 1 and 10.

Returns

If you specified a parameter, the method returns 'true' when the modification succeeds, and returns 'false' when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#get retry_down value
my $retry_down = $sip_monitor->retry_down();

#modify retry_down value
$sip_monitor->retry_down(3);
```

retry_up( )

Use this method to set or retrieve the number of times the server appears online after it was offline so it is treated as alive.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

The valid value is an unsigned integer between 1 and 10.

Returns

If you specified a parameter, the method returns 'true' when the modification succeeds, and returns 'false' when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#get retry_up value
my $retry_up = $sip_monitor->retry_up();

#modify retry_up value
$sip_monitor->retry_up(3);
```

timeout( )

Use this method to set or retrieve the value of a timeout for SIP health check.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

The valid value is an unsigned integer between 1 and 15.

Returns

If you specified a parameter, the method returns 'true' when the modification succeeds, and returns 'false' when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#get timeout value
my $timeout = $sip_monitor->timeout();

#modify timeout value
$sip_monitor->timeout(7);
```

transport( )

Use this method of set or retrieve the type of transport layer protocol being used for SIP health check.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

The valid values are 'TCP', 'UDP', 'SIPS' and 'TLS'.

Returns

If you specified a parameter, the method returns 'true' when the modification succeeds, and returns 'false' when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#get transport value
my $transport = $sip_monitor->transport();

#modify transport value
$sip_monitor->transport('UDP');
```

validate_cert( )

Use this method to set or retrieve the flag that indicates whether the validation of the remote server's certificate is enabled.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

Specify 'true' to enable remote server's certificate validation and 'false' to disable it. The default is 'true'.

Returns

If you specified a parameter, the method returns 'true' when the modification succeeds, and returns 'false' when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#get validate_cert value
my $validate_cert = $http_monitor->validate_cert();

#modify validate_cert value
$http_monitor->validate_cert('false');
```

AUTHOR

Infoblox Inc. http://www.infoblox.com/

SEE ALSO

Infoblox::Session, Infoblox::Session->add(), Infoblox::Session->get(), Infoblox::Session->modify(), Infoblox::Session->remove(), Infoblox::Session->search(), Infoblox::Grid::Extattr, Infoblox::Grid::ExtensibleAttributeDef/Extensible Attribute Values, Infoblox::DTC::Certificate.

COPYRIGHT

Copyright (c) 2017 Infoblox Inc.

Infoblox::DTC::Monitor::HTTP - An DTC HTTP Health Monitor object.

- NAME
- DESCRIPTION
- CONSTRUCTOR
- SESSION METHODS
  - Infoblox::Session->add( )
  - Infoblox::Session->get( )
  - Infoblox::Session->modify( )
  - Infoblox::Session->remove( )
  - Infoblox::Session->search( )
- METHODS
  - ciphers( )
  - client_cert( )
  - comment( )
  - content_check( )
  - content_check_input( )
  - content_check_regex( )
  - content_check_op( )
  - content_extract_group( )
  - content_extract_type( )
  - content_extract_value( )
  - enable_sni( )
  - extattrs( )
  - extensible_attributes( )
  - interval( )
  - name( )
  - port( )
  - request( )
  - result( )
  - result_code( )
  - retry_down( )
  - retry_up( )
  - secure( )
  - timeout( )
  - validate_cert( )
- AUTHOR
- SEE ALSO
- COPYRIGHT

NAME

Infoblox::DTC::Monitor::HTTP - An DTC HTTP Health Monitor object.

DESCRIPTION

The DTC HTTP Health Monitor validates the health of a HTTP service by first sending a specific HTTP message to a server and then examining the response received from the server. The validation is successful if the received response matches the expected message.

CONSTRUCTOR

```
 my $http_monitor = Infoblox::DTC::Monitor::HTTP->new(
    name                 => $string,
#Required
    ciphers              => [$cipher1, $cipher2, ...],
#Optional / Default is undefined
    client_cert          => $certificate
#Optional / Default is undefined
    comment              => $string,
#Optional / Default is undefined
    content_check        => 'NONE' | 'MATCH' | 'EXTRACT',
#Optional / Default is 'NONE'
    content_check_input  => 'HEADERS' | 'ALL' | 'BODY',
#Optional / Default is 'ALL'
    content_check_op     => 'EQ' | 'NEQ' | 'LEQ' | 'GEQ',
#Optional / Default is undefined
    content_check_regex  => $string,
#Optional / Default is undefined
    content_extract_group => $uint,
#Optional / Default is 0
    content_extract_type  => 'STRING' | 'INTEGER',
#Optional / Default is 'STRING'
```

```
    content_extract_value => $string,
#Optional / Default is undefined
    enable_sni            => 'true' | 'false',
#Optional / Default is 'false'
    extattrs              => { $string => $extattr, ... },
#Optional / Default is undefined
    extensible_attributes => { $string => $string | $num, $string => [ $string | $num, ... ], ... }
#Optional / Default is undefined
    interval              => $uint,
#Optional / Default is undefined
    port                  => $uint,
#Optional / Default is undefined
    request               => $string,
#Optional / Default is undefined
    result                => 'ANY' | 'CODE_IS' | 'CODE_IS_NOT',
#Optional / Default is undefined
    result_code           => $uint,
#Optional / Default is undefined
    retry_down            => $uint,
#Optional / Default is undefined
    retry_up              => $uint,
#Optional / Default is undefined
    secure                => 'true' | 'false',
#Optional / Default is 'false'
    timeout               => $uint,
#Optional / Default is undefined
    validate_cert         => 'true' | 'false',
#Optional / Default is 'true'
 );
```

You cannot set both extattrs and extensible_attributes attributes at the same time.

SESSION METHODS

This section describes all the methods in a Session module that you can apply to a DTC HTTP Monitor object.

Infoblox::Session->add( )

Use this method to add an object to the Infoblox appliance. See Infoblox::Session->add() for parameters and return values.

Example

```
#construct an object
my $http_monitor = Infoblox::DTC::Monitor::HTTP->new(
   name  => 'http_monitor1',
);

#submit for addition
my $response = $session->add($http_monitor);
```

Infoblox::Session->get( )

Use this method to retrieve all the matching objects from the Infoblox appliance. See Infoblox::Session->get() for parameters and return values.

Key Reference

```
Apply the following attribute to get a specific object:

 name                  - Optional. A DTC HTTP Monitor name in a string format.
 comment               - Optional. A DTC HTTP Monitor comment.
 extattrs              - Optional. A hash reference containing extensible attributes.
 extensible_attributes - Optional. A hash reference that contains extensible attributes.
```

Example

```
my @retrieved_objs = $session->get(
    object => 'Infoblox::DTC::Monitor::HTTP',
    name   => 'http_monitor1',
);

my @retrieved_objs = $session->get(
    object => 'Infoblox::DTC::Monitor::HTTP',
    extensible_attributes => { 'Site' => 'Santa Clara' }
);
```

Infoblox::Session->modify( )

Use this method to modify an object in the Infoblox appliance. See Infoblox::Session->modify() for parameters and return values.

Example

```
#Use method to modify the comment.
$http_monitor->comment('this is a modified comment');
#Submit modification
my $response = $session->modify($http_monitor);
```

Infoblox::Session->remove( )

Use this method to remove an object from the Infoblox appliance. See Infoblox::Session->remove() for parameters and return values.

To remove a specific object, first use `get()` or `search()` to retrieve the specific object, and then submit this object for removal.

Example

```
#Get the objects with the same name
my @retrieved_objs = $session->get(
    object => 'Infoblox::DTC::Monitor::HTTP',
    name   => 'http_monitor1',
);
#Find the desired object from the retrieved list.
my $desired_http_monitor = $retrieved_objs[0];
#Submit for removal
my $response = $session->remove($desired_http_monitor);
```

Infoblox::Session->search( )

Use this method to search for objects in the Infoblox appliance. See Infoblox::Session->search() for parameters and return values.

Key Reference

```
Apply the following attribute to get a specific object:

 name                  - Optional. A DTC HTTP Monitor name in a string format.
 comment               - Optional. A DTC HTTP Monitor Descriptive comment.
 extattrs              - Optional. A hash reference containing extensible attributes.
 extensible_attributes - Optional. A hash reference that contains extensible attributes.
```

Example

```
my @retrieved_objs = $session->search(
    object => 'Infoblox::DTC::Monitor::HTTP',
    name   => 'http_monitor1',
);

my @retrieved_objs = $session->search(
    object => 'Infoblox::DTC::Monitor::HTTP',
    extensible_attributes => { 'Site' => 'Santa Clara' }
);
```

METHODS

This section describes all the methods that you can use to set or retrieve the attribute values of the object.

ciphers( )

Use this method to set or retrieve a list of ciphers for a secure HTTP connection.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

The valid value is an array of ciphers in a string format. Ciphers use an OpenSSL syntax. Empty array is equivalent to 'ALL' in the OpenSSL notation.

Returns

If you specified a parameter, the method returns 'true' when the modification succeeds, and returns 'false' when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#get ciphers value
```

```
my @ciphers = $http_monitor->ciphers();

#modify ciphers value
$http_monitor->ciphers(['RC4-MD5', 'KRB5-RC4-MD5']);
```

client_cert( )

Use this method to set or retrieve a client certificate, supplied in a secure HTTP mode if present.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

The valid value is an Infoblox::DTC::Certificate object.

Returns

If you specified a parameter, the method returns 'true' when the modification succeeds, and returns 'false' when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#get client_cert value
my $cert = $http_monitor->client_cert();

#modify client_cert value
$http_monitor->client_cert($cert);
```

comment( )

Use this method to set or retrieve a DTC HTTP Health Monitor object comment.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

The valid value is a comment in string format (UTF-8) with a maximum of 256 bytes.

Returns

If you specified a parameter, the method returns 'true' when the modification succeeds, and returns 'false' when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#get comment value
my $comment = $http_monitor->comment();

#modify comment value
$http_monitor->comment('desired comment');
```

content_check( )

Use this method to set or retrieve content check type.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

The valid values are 'NONE', 'MATCH' and 'EXTRACT'. The default is 'NONE'.

Returns

If you specified a parameter, the method returns 'true' when the modification succeeds, and returns 'false' when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#get content_check value
my $content_check = $http_monitor->content_check();

#modify content_check value
$http_monitor->content_check('EXTRACT');
```

content_check_input( )

Use this method to set or retrieve a portion of reponse to use as input for content check.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

The valid values are 'HEADERS', 'ALL' and 'BODY'. The default is 'ALL'.

Returns

If you specified a parameter, the method returns 'true' when the modification succeeds, and returns 'false' when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#get content_check_input value
my $content_check_input = $http_monitor->content_check_input();

#modify content_check_input value
$http_monitor->content_check_input('BODY');
```

content_check_regex( )

Use this method to set or retrieve a content check regular expression.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

The valid value is a desired regular expression in a string format.

Returns

If you specified a parameter, the method returns 'true' when the modification succeeds, and returns 'false' when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#get content_check_regex value
my $content_check_regex = $http_monitor->content_check_regex();

#modify content_check_regex value
$http_monitor->content_check_regex('SQL Error');
```

content_check_op( )

Use this method to set or retrieve a content check success criteria operator.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

The valid values are 'EQ' and 'NEQ' for 'MATCH' content check, and 'EQ', 'NEQ', 'LEQ' and 'GEQ' for 'EXTRACT' content check type.

Returns

If you specified a parameter, the method returns 'true' when the modification succeeds, and returns 'false' when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#get content_check_op value
my $content_check_op = $http_monitor->content_check_op();

#modify content_check_op value
$http_monitor->content_check_op('EQ');
```

content_extract_group( )

Use this method to set or retrieve a content extraction sub-expression to extract.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

The valid value is an unsigned integer between 0 and 8. The default is 0.

Returns

If you specified a parameter, the method returns 'true' when the modification succeeds, and returns 'false' when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#get content_extract_group value
my $content_extract_group = $http_monitor->content_extract_group();

#modify content_extract_group value
$http_monitor->content_extract_group(3);
```

content_extract_type( )

Use this method to set or retrieve a content extraction expected type for the extracted data.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

The valid values are 'INTEGER' and 'STRING'. The default is 'STRING'.

Returns

If you specified a parameter, the method returns 'true' when the modification succeeds, and returns 'false' when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#get content_extract_type value
my $content_extract_type = $http_monitor->content_extract_type();

#modify content_extract_type value
$http_monitor->content_extract_type('INTEGER');
```

content_extract_value( )

Use this method to set or retrieve a content extraction value to compare with extracted result.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

The valid value is a desired extraction value in string format. The default is undefined.

Returns

If you specified a parameter, the method returns 'true' when the modification succeeds, and returns 'false' when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#get content_extract_value value
my $content_extract_value = $http_monitor->content_extract_value();

#modify content_extract_value value
$http_monitor->content_extract_value('1');
```

enable_sni( )

Use this method to set or retrieve the flag that indicates whether the Server Name Indication (SNI) for HTTPS monitor is enabled.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

Specify 'true' to enable SNI for HTTPS monitor and 'false' to disable it. The default is 'false'.

Returns

If you specified a parameter, the method returns 'true' when the modification succeeds, and returns 'false' when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#get enable_sni value
my $enable_sni = $http_monitor->enable_sni();

#modify enable_sni value
$http_monitor->enable_sni('true');
```

extattrs( )

Use this method to set or retrieve the extensible attributes associated with a DTC HTTP Health Monitor object.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

Valid value is a hash reference containing the names of extensible attributes and their associated values (Infoblox::Grid::Extattr objects).

Returns

If you specified a parameter, the method returns 'true' when the modification succeeds, and returns 'false' when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#get extattrs value
my $ref_extattrs = $http_monitor->extattrs();

#Modify extattrs
$http_monitor->extattrs({ 'Site' => $extattr1, 'Administrator' => $extattr2 });
```

extensible_attributes( )

Use this method to set or retrieve the extensible attributes associated with a DTC HTTP Health Monitor object.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

For valid values for extensible attributes, see Infoblox::Grid::ExtensibleAttributeDef/Extensible Attribute Values.

Returns

If you specified a parameter, the method returns 'true' when the modification succeeds, and returns 'false' when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get extensible attributes
my $ref_extensible_attributes = $http_monitor->extensible_attributes();

#Modify extensible attributes
$http_monitor->extensible_attributes({'Site' => 'Santa Clara', 'Administrator' => ['Peter',
'Tom']});
```

interval( )

Use this method to set or retrieve the interval for the HTTP health check.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

The valid value is an unsigned integer.

Returns

If you specified a parameter, the method returns 'true' when the modification succeeds, and returns 'false' when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#get interval value
my $interval = $http_monitor->interval();

#modify interval value
$http_monitor->interval(10);
```

name( )

Use this method to set or retrieve a DTC HTTP Health Monitor name.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

The valid value is a desired name in a string format.

Returns

If you specified a parameter, the method returns 'true' when the modification succeeds, and returns 'false' when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#get name value
my $name = $http_monitor->name();

#modify name value
$http_monitor->name('http_monitor1');
```

port( )

Use this method to set or retrieve the port value for HTTP or HTTPS requests.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

The valid value is an unsigned integer between 1 and 65535.

Returns

If you specified a parameter, the method returns 'true' when the modification succeeds, and returns 'false' when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#get port value
my $port = $http_monitor->port();

#modify port value
$http_monitor->port(8080);
```

request( )

Use this method to set or retrieve an HTTP request to send.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

The valid value is a string up to 1024 characters.

Returns

If you specified a parameter, the method returns 'true' when the modification succeeds, and returns 'false' when the operation

fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#get request value
my $request = $http_monitor->request();

#modify request value
$http_monitor->request('GET /');
```

result( )

Use this method to set or retrieve the type of expected result.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

The valid values are 'ANY', 'CODE_IS' and 'CODE_IS_NOT'.

Returns

If you specified a parameter, the method returns 'true' when the modification succeeds, and returns 'false' when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#get result value
my $result = $http_monitor->result();

#modify result value
$http_monitor->result('CODE_IS');
```

result_code( )

Use this method to set or retrieve the expected return code.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

The valid value is an unsigned integer between 0 and 999.

Returns

If you specified a parameter, the method returns 'true' when the modification succeeds, and returns 'false' when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#get result_code value
my $result_code = $http_monitor->result_code();

#modify result_code value
$http_monitor->result_code(300);
```

retry_down( )

Use this method to set or retrieve the number of times the server appears offline after it was online so it is treated as as dead.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

The valid value is an unsigned integer between 1 and 10.

Returns

If you specified a parameter, the method returns 'true' when the modification succeeds, and returns 'false' when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#get retry_down value
my $retry_down = $http_monitor->retry_down();

#modify retry_down value
$http_monitor->retry_down(3);
```

retry_up( )

Use this method to set or retrieve the number of times the server appears online after it was offline so it is treated as alive.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

The valid value is an unsigned integer between 1 and 10.

Returns

If you specified a parameter, the method returns 'true' when the modification succeeds, and returns 'false' when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#get retry_up value
my $retry_up = $http_monitor->retry_up();

#modify retry_up value
$http_monitor->retry_up(3);
```

secure( )

Use this method to disable or enable a secure connection.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

Specify 'true' to disable a secure connection or 'false' to enable it. The default value is 'false'.

Returns

If you specified a parameter, the method returns 'true' when the modification succeeds, and returns 'false' when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#get secure value
my $secure = $http_monitor->secure();

#modify secure value
$http_monitor->secure('true');
```

timeout( )

Use this method to set or retrieve the value of a timeout for HTTP health check.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

The valid value is an unsigned integer between 1 and 15.

Returns

If you specified a parameter, the method returns 'true' when the modification succeeds, and returns 'false' when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#get timeout value
my $timeout = $http_monitor->timeout();
```

```
#modify timeout value
$http_monitor->timeout(7);
```

validate_cert( )

Use this method to set or retrieve the flag that indicates whether the validation of the remote server's certificate is enabled.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

Specify 'true' to enable remote server's certificate validation and 'false' to disable it. The default is 'true'.

Returns

If you specified a parameter, the method returns 'true' when the modification succeeds, and returns 'false' when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#get validate_cert value
my $validate_cert = $http_monitor->validate_cert();

#modify validate_cert value
$http_monitor->validate_cert('false');
```

AUTHOR

Infoblox Inc. http://www.infoblox.com/

SEE ALSO

Infoblox::Session, Infoblox::Session->add(), Infoblox::Session->get(), Infoblox::Session->modify(), Infoblox::Session->remove(), Infoblox::Session->search(), Infoblox::Grid::Extattr, Infoblox::Grid::ExtensibleAttributeDef/Extensible Attribute Values, Infoblox::DTC::Certificate.

COPYRIGHT

Copyright (c) 2017 Infoblox Inc.

Infoblox::DTC::Monitor::PDP - A DTC PDP Health Monitor object.

- NAME
- DESCRIPTION
- CONSTRUCTOR
- SESSION METHODS
  - Infoblox::Session->add( )
  - Infoblox::Session->get( )
  - Infoblox::Session->modify( )
  - Infoblox::Session->remove( )
  - Infoblox::Session->search( )
- METHODS
  - comment( )
  - extattrs( )
  - extensible_attributes( )
  - interval( )
  - name( )
  - port( )
  - retry_down( )
  - retry_up( )
  - timeout( )
- AUTHOR
- SEE ALSO
- COPYRIGHT

NAME

Infoblox::DTC::Monitor::PDP - A DTC PDP Health Monitor object.

DESCRIPTION

A DTC PDP Health Monitor object.

CONSTRUCTOR

```
 my $pdp_monitor = Infoblox::DTC::Monitor::PDP->new(
    name                  => $string,
#Required
    comment               => $string,
#Optional / Default is undefined
    interval              => $uint,
#Optional / Default is undefined
    port                  => $uint,
#Optional / Default is undefined
    retry_down            => $uint,
#Optional / Default is undefined
    retry_up              => $uint,
#Optional / Default is undefined
    timeout               => $uint,
#Optional / Default is undefined
    extattrs              => { $string => $extattr, ... },
#Optional / Default is undefined
    extensible_attributes => { $string => $string | $num, $string => [ $string | $num, ... ], ... }
#Optional / Default is undefined
 );
```

You cannot set both extattrs and extensible_attributes attributes at the same time.

SESSION METHODS

This section describes all the methods in a Session module that you can apply to a DTC PDP Monitor object.

Infoblox::Session->add( )

Use this method to add an object to the Infoblox appliance. See Infoblox::Session->add() for parameters and return values.

Example

```
#construct an object
my $pdp_monitor = Infoblox::DTC::Monitor::PDP->new(
   name  => 'pdp_monitor1',
```

```
);

#submit for addition
my $response = $session->add($pdp_monitor);
```

Infoblox::Session->get( )

Use this method to retrieve all the matching objects from the Infoblox appliance. See Infoblox::Session->get() for parameters and return values.

Key Reference

```
Apply the following attribute to get a specific object:

 name                  - Optional. A DTC PDP Monitor name in a string format.
 comment               - Optional. A DTC PDP Monitor comment.
 extattrs              - Optional. A hash reference containing extensible attributes.
 extensible_attributes - Optional. A hash reference that contains extensible attributes.
```

Example

```
my @retrieved_objs = $session->get(
    object => 'Infoblox::DTC::Monitor::PDP',
    name   => 'pdp_monitor1',
);

my @retrieved_objs = $session->get(
    object => 'Infoblox::DTC::Monitor::PDP',
    extensible_attributes => { 'Site' => 'Santa Clara' }
);
```

Infoblox::Session->modify( )

Use this method to modify an object in the Infoblox appliance. See Infoblox::Session->modify() for parameters and return values.

Example

```
#Use method to modify the comment.
$pdp_monitor->comment('this is a modified comment');
#Submit modification
my $response = $session->modify($pdp_monitor);
```

Infoblox::Session->remove( )

Use this method to remove an object from the Infoblox appliance. See Infoblox::Session->remove() for parameters and return values.

To remove a specific object, first use `get()` or `search()` to retrieve the specific object, and then submit this object for removal.

Example

```
#Get the objects with the same name
my @retrieved_objs = $session->get(
    object => 'Infoblox::DTC::Monitor::PDP',
    name   => 'pdp_monitor1',
);
#Find the desired object from the retrieved list.
my $desired_pdp_monitor = $retrieved_objs[0];
#Submit for removal
my $response = $session->remove($desired_pdp_monitor);
```

Infoblox::Session->search( )

Use this method to search for objects in the Infoblox appliance. See Infoblox::Session->search() for parameters and return values.

Key Reference

```
Apply the following attribute to get a specific object:

 name                  - Optional. A DTC PDP Monitor name in a string format.
 comment               - Optional. A DTC PDP Monitor comment.
 extattrs              - Optional. A hash reference containing extensible attributes.
 extensible_attributes - Optional. A hash reference that contains extensible attributes.
```

Example

```
my @retrieved_objs = $session->search(
    object => 'Infoblox::DTC::Monitor::PDP',
    name   => 'pdp_monitor1',
```

```
);

my @retrieved_objs = $session->search(
    object => 'Infoblox::DTC::Monitor::PDP',
    extensible_attributes => { 'Site' => 'Santa Clara' }
);
```

METHODS

This section describes all the methods that you can use to set or retrieve the attribute values of the object.

comment( )

Use this method to set or retrieve a DTC PDP Health Monitor object comment.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

The valid value is a comment in string format (UTF-8) with a maximum of 256 bytes.

Returns

If you specified a parameter, the method returns 'true' when the modification succeeds, and returns 'false' when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#get comment value
my $comment = $pdp_monitor->comment();

#modify comment value
$pdp_monitor->comment('desired comment');
```

extattrs( )

Use this method to set or retrieve the extensible attributes associated with a DTC PDP Health Monitor object.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

Valid value is a hash reference containing the names of extensible attributes and their associated values (Infoblox::Grid::Extattr objects).

Returns

If you specified a parameter, the method returns 'true' when the modification succeeds, and returns 'false' when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#get extattrs value
my $ref_extattrs = $pdp_monitor->extattrs();

#Modify extattrs
$pdp_monitor->extattrs({ 'Site' => $extattr1, 'Administrator' => $extattr2 });
```

extensible_attributes( )

Use this method to set or retrieve the extensible attributes associated with a DTC PDP Health Monitor object.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

For valid values for extensible attributes, see Infoblox::Grid::ExtensibleAttributeDef/Extensible Attribute Values.

Returns

If you specified a parameter, the method returns 'true' when the modification succeeds, and returns 'false' when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get extensible attributes
my $ref_extensible_attributes = $pdp_monitor->extensible_attributes();

#Modify extensible attributes
$pdp_monitor->extensible_attributes({'Site' => 'Santa Clara', 'Administrator' => ['Peter', 'Tom']});
```

interval( )

Use this method to set or retrieve the time interval for the PDP health check.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

The valid value is an unsigned integer.

Returns

If you specified a parameter, the method returns 'true' when the modification succeeds, and returns 'false' when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#get interval value
my $interval = $pdp_monitor->interval();

#modify interval value
$pdp_monitor->interval(10);
```

name( )

Use this method to set or retrieve a DTC PDP Health Monitor name.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

The valid value is a desired name in a string format.

Returns

If you specified a parameter, the method returns 'true' when the modification succeeds, and returns 'false' when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#get name value
my $name = $pdp_monitor->name();

#modify name value
$pdp_monitor->name('pdp_monitor1');
```

port( )

Use this method to set or retrieve the port value for PDP requests.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

The valid value is an unsigned integer between 1 and 65535.

Returns

If you specified a parameter, the method returns 'true' when the modification succeeds, and returns 'false' when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#get port value
my $port = $pdp_monitor->port();

#modify port value
$pdp_monitor->port(6030);
```

retry_down( )

Use this method to set or retrieve the number of times the server appears offline after it was online so it is treated as dead.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

The valid value is an unsigned integer between 1 and 10.

Returns

If you specified a parameter, the method returns 'true' when the modification succeeds, and returns 'false' when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#get retry_down value
my $retry_down = $pdp_monitor->retry_down();

#modify retry_down value
$pdp_monitor->retry_down(3);
```

retry_up( )

Use this method to set or retrieve the number of times the server appears online after it was offline so it is treated as alive.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

The valid value is an unsigned integer between 1 and 10.

Returns

If you specified a parameter, the method returns 'true' when the modification succeeds, and returns 'false' when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#get retry_up value
my $retry_up = $pdp_monitor->retry_up();

#modify retry_up value
$pdp_monitor->retry_up(3);
```

timeout( )

Use this method to set or retrieve the value of a timeout for PDP health check.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

The valid value is an unsigned integer between 1 and 15.

Returns

If you specified a parameter, the method returns 'true' when the modification succeeds, and returns 'false' when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#get timeout value
my $timeout = $pdp_monitor->timeout();

#modify timeout value
```

```
$pdp_monitor->timeout(7);
```

AUTHOR

Infoblox Inc. http://www.infoblox.com/

SEE ALSO

Infoblox::Session, Infoblox::Session->add(), Infoblox::Session->get(), Infoblox::Session->modify(), Infoblox::Session->remove(), Infoblox::Session->search(), Infoblox::Grid::Extattr, Infoblox::Grid::ExtensibleAttributeDef/Extensible Attribute Values.

COPYRIGHT

Copyright (c) 2017 Infoblox Inc.

Infoblox::DTC::Monitor::SNMP - A DTC SNMP Health Monitor object.

- NAME
- DESCRIPTION
- CONSTRUCTOR
- SESSION METHODS
  - Infoblox::Session->add( )
  - Infoblox::Session->get( )
  - Infoblox::Session->modify( )
  - Infoblox::Session->remove( )
  - Infoblox::Session->search( )
- METHODS
  - comment( )
  - community( )
  - context( )
  - engine_id( )
  - extattrs( )
  - extensible_attributes( )
  - interval( )
  - name( )
  - oids( )
  - port( )
  - retry_down( )
  - retry_up( )
  - timeout( )
  - user( )
  - version( )
- AUTHOR
- SEE ALSO
- COPYRIGHT

NAME

Infoblox::DTC::Monitor::SNMP - A DTC SNMP Health Monitor object.

DESCRIPTION

The DTC SNMP Health Monitor determines the health of SNMP servers such as SNMP Proxies and Session Border Controllers, and SNMP gateways by issuing SNMP options to a server and examining the response provided by the server. The service is considered available if the received response matches the expected response.

CONSTRUCTOR

```
 my $snmp_monitor = Infoblox::DTC::Monitor::SNMP->new(
    name                => $string,
#Required
    comment             => $string,
#Optional / Default is undefined
    community           => $string,
#Optional / Default is 'public'
    interval            => $uint,
#Optional / Default is 5
    oids                => [$oid1, $oid2, ...],
#Optional / Default is empty list
    port                => $uint,
#Optional / Default is 161
    retry_down          => $uint,
#Optional / Default is 1
    retry_up            => $uint,
#Optional / Default is 1
    timeout             => $uint,
#Optional / Default is 15
    version             => 'V1' | 'V2C' | 'V3',
#Optional / Default is 'V2C'
    context             => $string,
#Optional / Default is undefined
    engine_id           => $string,
#Optional / Default is undefined
    user                => $string,
#Optional / Default is undefined
    extattrs            => { $string => $extattr, ... },
#Optional / Default is undefined
    extensible_attributes => { $string => $string | $num, $string => [ $string | $num, ... ], ... }
```

```
#Optional / Default is undefined
 );
```

You cannot set both extattrs and extensible_attributes attributes at the same time.

SESSION METHODS

This section describes all the methods in a Session module that you can apply to a DTC SNMP Monitor object

Infoblox::Session->add( )

Use this method to add an object to the Infoblox appliance. See Infoblox::Session->add() for parameters and return values.

Example

```
#construct an object
my $snmp_monitor = Infoblox::DTC::Monitor::SNMP->new(
   name  => 'snmp_monitor1',
);

#submit for addition
my $response = $session->add($snmp_monitor);
```

Infoblox::Session->get( )

Use this method to retrieve all the matching objects from the Infoblox appliance. See Infoblox::Session->get() for parameters and return values.

Key Reference

```
Apply the following attribute to get a specific object:

 name                  - Optional. A DTC SNMP Monitor name in string format.
 comment               - Optional. A DTC SNMP Monitor descriptive comment.
 engine_id             - Optional. A DTC SNMPv3 Monitor engine identifier.
 context               - Optional. A DTC SNMPv3 Monitor context.
 extattrs              - Optional. A hash reference containing extensible attributes.
 extensible_attributes - Optional. A hash reference that contains extensible attributes.
```

Example

```
my @retrieved_objs = $session->get(
    object => 'Infoblox::DTC::Monitor::SNMP',
    name   => 'snmp_monitor1',
);

my @retrieved_objs = $session->get(
    object => 'Infoblox::DTC::Monitor::SNMP',
    extensible_attributes => { 'Site' => 'Santa Clara' }
);
```

Infoblox::Session->modify( )

Use this method to modify an object in the Infoblox appliance. See Infoblox::Session->modify() for parameters and return values.

Example

```
#Use method to modify the comment.
$snmp_monitor->comment('this is a modified comment');
#Submit modification
my $response = $session->modify($snmp_monitor);
```

Infoblox::Session->remove( )

Use this method to remove an object from the Infoblox appliance. See Infoblox::Session->remove() for parameters and return values.

To remove a specific object, first use `get()` or `search()` to retrieve the specific object, and then submit this object for removal.

Example

```
#Get the objects with the same name
my @retrieved_objs = $session->get(
    object => 'Infoblox::DTC::Monitor::SNMP',
    name   => 'snmp_monitor1',
);
#Find the desired object from the retrieved list.
my $desired_snmp_monitor = $retrieved_objs[0];
```

```
#Submit for removal
my $response = $session->remove($desired_snmp_monitor);
```

Infoblox::Session->search( )

Use this method to search for objects in the Infoblox appliance. See Infoblox::Session->search() for parameters and return values.

Key Reference

```
Apply the following attribute to get a specific object:

 name                 - Optional. A DTC SNMP Monitor name in string format (regexp).
 comment              - Optional. A DTC SNMP Monitor descriptive comment (regexp).
 engine_id            - Optional. A DTC SNMPv3 Monitor engine identifier (regexp).
 context              - Optional. A DTC SNMPv3 Monitor context (regexp).
 extattrs             - Optional. A hash reference containing extensible attributes.
 extensible_attributes - Optional. A hash reference that contains extensible attributes.
```

Example

```
my @retrieved_objs = $session->search(
    object => 'Infoblox::DTC::Monitor::SNMP',
    name   => 'snmp_monitor1',
);

my @retrieved_objs = $session->search(
    object => 'Infoblox::DTC::Monitor::SNMP',
    extensible_attributes => { 'Site' => 'Santa Clara' }
);
```

METHODS

This section describes all the methods that you can use to set or retrieve the attribute values of the object.

comment( )

Use this method to set or retrieve the comment for a DTC SNMP Health Monitor object.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

The valid value is a comment in string format (UTF-8) with a maximum of 256 bytes.

Returns

If you specified a parameter, the method returns 'true' when the modification succeeds, and returns 'false' when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#get comment value
my $comment = $snmp_monitor->comment();

#modify comment value
$snmp_monitor->comment('desired comment');
```

community( )

Use this method to set or retrieve the SNMP community string for a SNMP authentication.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

The valid value is a comment in string format (UTF-8) with a maximum of 256 bytes.

Returns

If you specified a parameter, the method returns 'true' when the modification succeeds, and returns 'false' when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#get community value
my $community = $snmp_monitor->community();

#modify community value
$snmp_monitor->community('desired community');
```

context( )

Use this method to set or retrive the SNMPv3 context.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

The valid value is a context in string format (UTF-8) with a maximum of 1023 characters.

Returns

If you specified a parameter, the method returns 'true' when the modification succeeds, and returns 'false' when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#get context value
my $context = $snmp_monitor->context();

#modify context value
$snmp_monitor->context('desired_context');
```

engine_id( )

Use this method to set or retrive the SNMPv3 engine identifier.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

The valid value is an engine identifier in string format (UTF-8) with a maximum of 1023 characters.

Returns

If you specified a parameter, the method returns 'true' when the modification succeeds, and returns 'false' when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#get engine identifiervalue
my $engine identifier= $snmp_monitor->engine_id();

#modify engine identifiervalue
$snmp_monitor->engine_id('desired_engine_id');
```

extattrs( )

Use this method to set or retrieve the extensible attributes associated with a DTC SNMP Health Monitor object.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

Valid value is a hash reference containing the names of extensible attributes and their associated values (Infoblox::Grid::Extattr objects).

Returns

If you specified a parameter, the method returns 'true' when the modification succeeds, and returns 'false' when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#get extattrs value
my $ref_extattrs = $snmp_monitor->extattrs();
```

```
#Modify extattrs
$snmp_monitor->extattrs({ 'Site' => $extattr1, 'Administrator' => $extattr2 });
```

extensible_attributes( )

Use this method to set or retrieve the extensible attributes associated with a DTC SNMP Health Monitor object.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

For valid values for extensible attributes, see Infoblox::Grid::ExtensibleAttributeDef/Extensible Attribute Values.

Returns

If you specified a parameter, the method returns 'true' when the modification succeeds, and returns 'false' when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get extensible attributes
my $ref_extensible_attributes = $snmp_monitor->extensible_attributes();

#Modify extensible attributes
$snmp_monitor->extensible_attributes({'Site' => 'Santa Clara', 'Administrator' => ['Peter',
'Tom']});
```

interval( )

Use this method to set or retrieve the time interval for the SNMP health check.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

The valid value is an unsigned integer.

Returns

If you specified a parameter, the method returns 'true' when the modification succeeds, and returns 'false' when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#get interval value
my $interval = $snmp_monitor->interval();

#modify interval value
$snmp_monitor->interval(10);
```

name( )

Use this method to set or retrieve a DTC SNMP Health Monitor name.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

The valid value is a desired name in string format.

Returns

If you specified a parameter, the method returns 'true' when the modification succeeds, and returns 'false' when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#get name value
my $name = $snmp_monitor->name();

#modify name value
$snmp_monitor->name('snmp_monitor1');
```

oids( )

Use this method to set or retrieve the list of OIDs for SNMP monitoring.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

The valid value is an array of an Infoblox::DTC::Monitor::SNMP::OID objects.

Returns

If you specified a parameter, the method returns 'true' when the modification succeeds, and returns 'false' when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#get oids value
my $oids = $snmp_monitor->oids();

#modify oids value
$snmp_monitor->oids([$oid1, $oid2]);
```

port( )

Use this method to set or retrieve the port value for SNMP or SNMPS requests.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

The valid value is an unsigned integer between 1 and 65535.

Returns

If you specified a parameter, the method returns 'true' when the modification succeeds, and returns 'false' when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#get port value
my $port = $snmp_monitor->port();

#modify port value
$snmp_monitor->port(8080);
```

retry_down( )

Use this method to set or retrieve the number of times the server appears offline after it was online so it is treated as unavailable.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

The valid value is an unsigned integer between 1 and 10.

Returns

If you specified a parameter, the method returns 'true' when the modification succeeds, and returns 'false' when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#get retry_down value
my $retry_down = $snmp_monitor->retry_down();

#modify retry_down value
$snmp_monitor->retry_down(3);
```

retry_up( )

Use this method to set or retrieve the number of times the server appears online after it was offline so it is treated as alive.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

The valid value is an unsigned integer between 1 and 10.

Returns

If you specified a parameter, the method returns 'true' when the modification succeeds, and returns 'false' when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#get retry_up value
my $retry_up = $snmp_monitor->retry_up();

#modify retry_up value
$snmp_monitor->retry_up(3);
```

timeout( )

Use this method to set or retrieve the value of a timeout for SNMP health check.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

The valid value is an unsigned integer between 1 and 15.

Returns

If you specified a parameter, the method returns 'true' when the modification succeeds, and returns 'false' when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#get timeout value
my $timeout = $snmp_monitor->timeout();

#modify timeout value
$snmp_monitor->timeout(7);
```

user( )

Use this method to set or retrieve the SNMPv3 user setting.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

The valid value is an Infoblox::Grid::SNMP::User object name.

Returns

If you specified a parameter, the method returns 'true' when the modification succeeds, and returns 'false' when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#get user value
my $user = $snmp_monitor->user();

#modify user value
$snmp_monitor->user('user1');
```

version( )

Use this method to set or retrieve the SNMP protocol version for the SNMP health check.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

The valid values are 'V1', 'V2C' and 'V3'.

Returns

If you specified a parameter, the method returns 'true' when the modification succeeds, and returns 'false' when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#get version value
my $version = $snmp_monitor->version();

#modify version value
$snmp_monitor->version('V1');
```

AUTHOR

Infoblox Inc. http://www.infoblox.com/

SEE ALSO

Infoblox::Session, Infoblox::Session->add(), Infoblox::Session->get(), Infoblox::Session->modify(), Infoblox::Session->remove(), Infoblox::Session->search(), Infoblox::Grid::Extattr, Infoblox::Grid::ExtensibleAttributeDef/Extensible Attribute Values, Infoblox::DTC::Monitor::SNMP::OID, Infoblox::Grid::SNMP::User

---

COPYRIGHT

Copyright (c) 2017 Infoblox Inc.

Infoblox::DTC::Certificate - a DTC Certificate object.

- NAME
- DESCRIPTION
- CONSTRUCTOR
- SESSION METHODS
  - Infoblox::Session->get( )
  - Infoblox::Session->search( )
- MODULE METHODS
  - Infoblox::DTC->add_certificate( )
- METHODS
  - certificate( )
  - in_use( )
- AUTHOR
- SEE ALSO
- COPYRIGHT

NAME

Infoblox::DTC::Certificate - a DTC Certificate object.

DESCRIPTION

A DTC Certificate object.

CONSTRUCTOR

SESSION METHODS

This section describes all the methods in a Session module that you can apply to a DTC Certificate object.

Infoblox::Session->get( )

Use this method to retrieve all the matching objects from the Infoblox appliance. See Infoblox::Session->get() for parameters and return values.

Key Reference

```
Apply the following attribute to get a specific object:

  distinguished_name - Optional. A X509Certificate Distinguished name (DN).
  issuer             - Optional. A X509Certificate issuer.
  serial             - Optional. A X509Certificate serial number.
  valid_not_after    - Optional. A X509Certificate expiry date.
  valid_not_before   - Optional. A X509Certificate validity start date.
```

Example

```
my @retrieved_objs = $session->get(
    object => 'Infoblox::DTC::Certificate',
    serial => 'd80b0e8c47df5082',
);
```

Infoblox::Session->search( )

Use this method to search for objects in the Infoblox appliance. See Infoblox::Session->search() for parameters and return values.

Key Reference

```
Apply the following attribute to get a specific object:

  distinguished_name - Optional. A X509Certificate Distinguished name (DN).
  issuer             - Optional. A X509Certificate issuer.
  serial             - Optional. A X509Certificate serial number.
  valid_not_after    - Optional. A X509Certificate expiry date.
  valid_not_before   - Optional. A X509Certificate validity start date.
```

Example

```
my @retrieved_objs = $session->search(
    object => 'Infoblox::DTC::Certificate',
```

```
        serial => 'd80b0e8c47df5082',
);
```

MODULE METHODS

Infoblox::DTC->add_certificate( )

Use this method to upload a DTC certificate.

Key Reference

```
Apply the following attribute to upload a DTC certificate:

   path - Required. The path to the file that contains the DTC certificate.
```

Returns

The method returns 'true' when the upload of a MaxMind Database has been performed successfully, and returns 'false' when the operation fails.

Example

```
#add certificate with a path given
my $res = $dtc->add_certificate(path => $path_to_cert);
```

METHODS

This section describes all the methods that you can use to set or retrieve the attribute values of the object.

certificate( )

Use this method to retrieve an X509Certificate object. This is a read-only attribute.

Parameter

None

Returns

The valid return value is an Infoblox::Grid::X509Certificate object.

Example

```
#get certificate value
my $certificate = $dtc_certificate->certificate();
```

in_use( )

Use this method to retrieve the certificate status. This is a read-only attribute.

Parameter

None

Returns

The method returns the attribute value.

Example

```
#get in_use value
my $in_use = $dtc_certificate->in_use();
```

AUTHOR

Infoblox Inc. http://www.infoblox.com/

SEE ALSO

Infoblox::Session, Infoblox::Session->get(), Infoblox::Session->search(), Infoblox::DTC->add_certificate()

COPYRIGHT

Infoblox::DTC::Certificate - a DTC Certificate object.

Copyright (c) 2017 Infoblox Inc.

Infoblox::DTC::Monitor::SNMP::OID - a DTC SNMP Monitor OID object.

- NAME
- DESCRIPTION
- CONSTRUCTOR
- SESSION METHODS
- MODULE METHODS
  - Infoblox::DTC::Monitor::SNMP->oids( )
- METHODS
  - comment( )
  - condition( )
  - first( )
  - last( )
  - oid( )
  - type( )
- AUTHOR
- SEE ALSO
- COPYRIGHT

NAME

Infoblox::DTC::Monitor::SNMP::OID - a DTC SNMP Monitor OID object.

DESCRIPTION

SNMP is stateless request-response protocol for monitoring and managing devices on network. SNMP agent (software running on monitored server) exposes management data in form of variables. Each variable associated with a uniq id called OID.

CONSTRUCTOR

```
my $snmp_oid = Infoblox::DTC::Monitor::SNMP::OID->new(
    oid       => $string,                                    # Required
    comment   => $string,                                    # Optional / Default is undefined
    type      => 'STRING' | 'INTEGER',                       # Optional / Default is 'STRING'
    condition => 'ANY' | 'EXACT' | 'LEQ' | 'GEQ' | 'RANGE', # Optional / Default is 'ANY'
    first     => $string,                                    # Optional / Default is undefined
    last      => $string,                                    # Optional / Default is undefined
);
```

SESSION METHODS

The object does not support any session methods.

MODULE METHODS

The following functions can be applied to a DTC SNMP Monitor OID object.

Infoblox::DTC::Monitor::SNMP->oids( )

Use this method to set or retrieve the list of OIDs for SNMP monitoring. See Infoblox::DTC::Monitor::SNMP->oids() for parameters and return values.

Example

```
#Get snmp_oid value
my $snmp_oid = $snmp_monitor->snmp_oid();
#Modify bind_check_snmp_oid
$snmp_monitor->snmp_oid([$snmp_oids]);
```

METHODS

comment( )

Use this method to set or retrieve the comment for a DTC SNMP Health Monitor OID object.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

The valid value is a comment in string format (UTF-8) with a maximum of 256 bytes.

Returns

If you specified a parameter, the method returns 'true' when the modification succeeds, and returns 'false' when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#get comment value
my $comment = $snmp_oid->comment();

#modify comment value
$snmp_oid->comment('desired comment');
```

condition( )

Use this method to set or retrieve the condition for validating the result of a SNMP health check.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

The following conditions can be applied to result check:

'ANY' accepts any response.

'EXACT' accepts result equal to 'first'

'LEQ' accepts result which is less than 'first'

'GEQ' accepts result which is greater than 'first'

'RANGE' accepts result value of which is between 'first' and 'last'

The default value is 'ANY'.

Returns

If you specified a parameter, the method returns 'true' when the modification succeeds, and returns 'false' when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#get condition value
my $condition = $snmp_oid->condition();

#modify condition value
$snmp_oid->condition('RANGE');
```

first( )

Use this method to set or retrieve the condition's first term to match against the SNMP health check result.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

The valid value is a string or integer both in string format (UTF-8) with a maximum of 256 bytes.

Returns

If you specified a parameter, the method returns 'true' when the modification succeeds, and returns 'false' when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#get first value
my $first = $snmp_oid->first();

#modify first value
$snmp_oid->first('first_value');
```

last( )

Use this method to set or retrieve the condition's second term to match against the SNMP health check result with 'RANGE' condition.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

The valid value is a string or integer both in string format (UTF-8) with a maximum of 256 bytes.

Returns

If you specified a parameter, the method returns 'true' when the modification succeeds, and returns 'false' when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#get last value
my $last = $snmp_oid->last();

#modify last value
$snmp_oid->last('1000');
```

oid( )

Use this method to set or retrieve the SNMP OID value for DTC SNMP monitor health checks.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

The valid value is an OID in string format (UTF-8) with a maximum of 256 bytes.

Returns

If you specified a parameter, the method returns 'true' when the modification succeeds, and returns 'false' when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#get oid value
my $oid = $snmp_oid->oid();

#modify oid value
$snmp_oid->oid('sampleoid.1.1.1.1.2);
```

type( )

Use this method to set or retrieve the condition's value type for DTC SNMP monitor health check results.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

The valid values are 'STRING' and 'INTEGER'.

Returns

If you specified a parameter, the method returns 'true' when the modification succeeds, and returns 'false' when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#get type value
my $type = $snmp_oid->type();

#modify type value
$snmp_oid->type('INTEGER');
```

AUTHOR

Infoblox::DTC::Monitor::SNMP::OID - a DTC SNMP Monitor OID object.

Infoblox Inc. http://www.infoblox.com/

SEE ALSO

Infoblox::DTC::Monitor::SNMP, Infoblox::DTC::Monitor::SNMP->oids()

COPYRIGHT

Copyright (c) 2017 Infoblox Inc.

Infoblox::Grid::SNMP::User - Grid SNMP User object

- NAME
- DESCRIPTION
- CONSTRUCTOR
- SESSION METHODS
  - Infoblox::Session->get( )
  - Infoblox::Session->modify( )
  - Infoblox::Session->search( )
  - Infoblox::Session->remove( )
- METHODS
  - auth_password( )
  - auth_protocol( )
  - comment( )
  - disable( )
  - engine_id( )
  - extattrs( )
  - name( )
  - privacy_password( )
  - privacy_protocol( )
- AUTHOR
- SEE ALSO
- COPYRIGHT

NAME

Infoblox::Grid::SNMP::User - Grid SNMP User object

DESCRIPTION

The Grid SNMP User object is used to set, get, search, modify, or remove an SNMPv3 user. An SNMPv3 user is a managment system that sends queries and receives traps from the appliance.

CONSTRUCTOR

```
#Constructor for a Grid SNMP User object
my $snmp_user = Infoblox::Grid::SNMP::User->new(
        name                => $string,                 #Required
        auth_protocol       => "none"| "md5" | "sha",   #Required
        privacy_protocol    => "none"| "des" | "aes",   #Required
        auth_password       => $string,                 #Optional. Required for MD5/SHA authentication
        comment             => $string,                 #Optional / Default is undefined
        disable             => "true" | "false",        #Optional / Default is "false"
        privacy_password    => $string,                 #Optional. Required for MD5/SHA authentication
        extattrs            => { $string => $extattr, ... },      #Optional / Default is undefined
 );
```

SESSION METHODS

This section describes all the methods in Infoblox::Session that you can apply to a Grid SNMP User object.

Infoblox::Session->get( )

Use this method to retrieve existing objects from the Infoblox appliance. See Infoblox::Session->get() for parameters and return values.

Key Reference

```
Apply the following attributes to get a Grid SNMP User object:

name     -  user name in string format
comment  -  comment string
extattrs - Optional. A hash reference containing extensible attributes.
```

Example

```
my @retrieved_objs = $session->get(
        object      => "Infoblox::Grid::SNMP::User",
        name        => "testuser" );
```

Infoblox::Session->modify( )

Use this method to modify objects in the Infoblox appliance. See Infoblox::Session->modify() for parameters and return values.

Example

```
# Use this method to modify the comment
$snmp_user->comment("This is a modified comment");
# Submit modification
my $response = $session->modify( $snmp_user );
```

Infoblox::Session->search( )

Use this method to search for Grid SNMP User objects in the Infoblox appliance. See Infoblox::Session->search() for parameters and return values.

Key References

```
Apply the following attributes to search for a Grid SNMP User object:

 name -  user name (regular expression)
 comment - comment string (regular expression)
 extattrs - Optional. A hash reference containing extensible attributes.
```

Example

```
# Search for all Grid SNMP User objects that start with "test"
my @retrieved_objs = $session->search(
    object      => "Infoblox::Grid::SNMP::User",
    name        => "test.*",
);
```

Infoblox::Session->remove( )

Use this method to remove an object from the Infoblox appliance. See Infoblox::Session->remove() for parameters and return values.

To remove a specific object, first use `get()` or `search()` to retrieve the object, and then submit it for removal.

Example

```
# Get Grid SNMP User objects with the same starting name
my @retrieved_objs = $session->get(
    object      => "Infoblox::Grid::SNMP::User",
    name        => "testuser" );
# find the desired object from the retrieved list.
my $desired_user = $retrieved_objs[0];
# Submit for removal
my $response = $session->remove( $desired_user );
```

METHODS

This section describes all the methods that you can use to configure and retrieve the attribute values of an SNMPv3 user.

auth_password( )

Use this method to provide an authentication password for the user. This is a write-only attribute.

Parameter

Authentication password of the user in string format.

Returns

If a parameter is specified, the method returns true when the modification succeeds, and returns false when the operation fails.

Example

```
#Set the password
$snmp_user->auth_password("mysecret");
```

auth_protocol( )

Use this method to specify the authentication protocol of the user.

Parameter

Possible values for the authentication protocol are:

```
none
```

```
md5
sha
```

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get auth_protocol
my $proto = $snmp_user->auth_protocol();
#Set the protocol
$snmp_user->auth_protocol("md5");
```

comment( )

Use this method to provide a comment for the SNMPv3 User object.

Parameter

Comment in string format with a maximum of 256 bytes.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get comment
my $comment = $snmp_user->comment();
#Set the comment
$snmp_user->comment("This is a sample comment");
```

disable( )

Use this method to configure or retrieve the "disable" flag.

The default value is false; that is, the user is enabled.

Parameter

Specify "true" to set the disable flag or "false" to deactivate/unset it. The default value is "false".

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get disable flag
my $disabled = $snmp_user->disable();
#Set the disable flag
$snmp_user->disable("true");
```

engine_id( )

Use this method to retrieve the SNMP engine ID.

This field is read-only.

Parameter

None

Returns

When called without parameters, the method returns the attribute value. Calling this method with a parameter results in an error.

Example

```
#Get engine_id
my $engine_id = $snmp_user->engine_id();
```

extattrs( )

Use this method to set or retrieve the extensible attributes associated with a SNMP user object.

Parameter

Valid value is a hash reference containing the names of extensible attributes and their associated values (Infoblox::Grid::Extattr objects).

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get extattrs
my $ref_extattrs = $snmp_user->extattrs();
#Modify extattrs
$snmp_user->extattrs({ 'Site' => $extattr1, 'Administrator' => $extattr2 });
```

name( )

Use this method to set or retrieve the name of an SNMPv3 user.

Parameter

Text with the name of the user.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get name
my $name = $snmp_user->name();
#Modify name
$snmp_user->name("test_user");
```

privacy_password( )

Use this method to provide a password for the privacy protocol. This is a write-only attribute.

Parameter

Password in string format.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

Example

```
#Set the password
$snmp_user->privacy_password("mysecret");
```

privacy_protocol( )

Use this method to specify encryption protocol for the SNMP packets.

Parameter

Possible values for the encryption protocol are:

```
none
des
aes
```

The values are case insensitive.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get privacy_protocol
my $proto = $snmp_user->privacy_protocol();
#Set the protocol
$snmp_user->privacy_protocol("aes");
```

AUTHOR

Infoblox Inc. http://www.infoblox.com/

SEE ALSO

Infoblox::Grid::SNMP::TrapReceiver,Infoblox::Session, Infoblox::Session->get(), Infoblox::Session->modify()

COPYRIGHT

Copyright (c) 2017 Infoblox Inc.

Infoblox::DTC::Pool::DynamicRatioSetting - An DTC Pool dynamic ratio setting object.

- NAME
- DESCRIPTION
- CONSTRUCTOR
- SESSION METHODS
- MODULE METHODS
  - Infoblox::DTC::Pool->dynamic_ratio_preferred( )
  - Infoblox::DTC::Pool->dynamic_ratio_alternate( )
- METHODS
  - method( )
  - monitor( )
  - monitor_metric( )
  - monitor_weighing( )
  - invert_monitor_metric( )
- AUTHOR
- SEE ALSO
- COPYRIGHT

NAME

Infoblox::DTC::Pool::DynamicRatioSetting - An DTC Pool dynamic ratio setting object.

DESCRIPTION

The DTC Pool dynamic ratio setting object provides settings for dynamic ratio load balancing if you later select preferred or alternate as the load balancing method.

CONSTRUCTOR

```
my $object = Infoblox::DTC::Pool::DynamicRatioSetting->new(
    method                => 'MONITOR' | 'ROUND_TRIP_DELAY', # Optional
    monitor               => $dtc_monitor,                   # Optional
    monitor_metric        => $string,                        # Optional
    monitor_weighing      => 'RATIO' | 'PRIORITY',           # Optional
    invert_monitor_metric => 'true' | 'false',               # Optional
);
```

SESSION METHODS

The object does not support any session methods.

MODULE METHODS

The following functions can be applied to the object.

Infoblox::DTC::Pool->dynamic_ratio_preferred( )

Use this method to set or retrieve the preferred dynamic ratio load balancing settings. See Infoblox::DTC::Pool->dynamic_ratio_preferred() for parameters and return values.

Example

```
#get dynamic_ratio_preferred value
my $dynamic_ratio_preferred = $pool->dynamic_ratio_preferred();

#Construct dynamic_ratio_preferred value
my $dynamic_ratio = Infoblox::DTC::Pool::DynamicRatioSetting->new(
    method                => 'ROUND_TRIP_DELAY',
    invert_monitor_metric => 'true',
);

#modify dynamic_ratio_preferred value
$pool->dynamic_ratio_preferred($dynamic_ratio);
```

Infoblox::DTC::Pool->dynamic_ratio_alternate( )

Use this method to set or retrieve the alternate dynamic ratio load balancing settings. See Infoblox::DTC::Pool->dynamic_ratio_alternate() for parameters and return values.

Example

```
#get dynamic_ratio_alternate value
my $dynamic_ratio_alternate = $pool->dynamic_ratio_alternate();

#modify dynamic_ratio_alternate value
$pool->dynamic_ratio_alternate($dynamic_ratio);
```

METHODS

This section describes all the methods that you can use to set or retrieve the attribute values of the object.

method( )

Use this method to set or retrieve the dynamic ratio load balancing method.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

The valid values are 'MONITOR' and 'ROUND_TRIP_DELAY'.

Returns

If you specified a parameter, the method returns 'true' when the modification succeeds, and returns 'false' when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get method value
my $method = $object->method();

#Modify method value
$object->method('MONITOR');
```

monitor( )

Use this method to set or retrieve the DTC monitor that will be used for the dynamic ratio load balancing.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

The valid value is an Infoblox::DTC::Monitor::TCP, Infoblox::DTC::Monitor::ICMP, Infoblox::DTC::Monitor::SIP, Infoblox::DTC::Monitor::HTTP, or Infoblox::DTC::Monitor::PDP objects.

Note that you must retrieve the object from the server first, or set the name of the monitor.

Returns

If you specified a parameter, the method returns 'true' when the modification succeeds, and returns 'false' when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get monitor value
my $monitor = $object->monitor();

#Construct Infoblox::DTC::Monitor::SNMP object (should be an existing object)
my $snmp_monitor = Infoblox::DTC::Monitor::SNMP->new(name => 'snmp1');

#Modify monitor value
$object->monitor($snmp_monitor);
```

monitor_metric( )

Use this method to set or retrieve the DTC monitor metric that will be used for dynamic weighing.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

The valid value is a desired metric in string format.

Returns

If you specified a parameter, the method returns 'true' when the modification succeeds, and returns 'false' when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get monitor_metric value
my $monitor_metric = $object->monitor_metric();

#Modify monitor_metric value
$object->monitor_metric('222');
```

monitor_weighing( )

Use this method to set or retrieve the weighing settings for the dynamic ratio load balancing servers.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

The valid values are 'PRIORITY' and 'RATIO'. 'PRIORITY' means that all clients will be forwarded to the least loaded server. 'RATIO' means that distribution will be calculated based on dynamic weights.

Returns

If you specified a parameter, the method returns 'true' when the modification succeeds, and returns 'false' when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get monitor_weighing value
my $monitor_weighing = $object->monitor_weighing();

#Modify monitor_weighing value
$object->monitor_weighing('RATIO');
```

invert_monitor_metric( )

Use this method to set or retrieve the flag that indicates whether the inverted value of the monitor metric is used.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Specify 'true' to force the server to use the inverted monitor metric and 'false' to use the direct metric value. The default is 'false'.

Returns

If you specified a parameter, the method returns 'true' when the modification succeeds, and returns 'false' when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get invert_monitor_metric value
my $invert_monitor_metric = $object->invert_monitor_metric();

#Modify invert_monitor_metric value
$object->invert_monitor_metric('true');
```

AUTHOR

Infoblox Inc. http://www.infoblox.com/

SEE ALSO

Infoblox::DTC::Pool, Infoblox::DTC::Pool->dynamic_ratio_preferred(), Infoblox::DTC::Pool->dynamic_ratio_alternate(), Infoblox::DTC::Monitor::TCP, Infoblox::DTC::Monitor::ICMP, Infoblox::DTC::Monitor::SIP, Infoblox::DTC::Monitor::HTTP, Infoblox::DTC::Monitor::PDP

COPYRIGHT

Infoblox::DTC::Pool::DynamicRatioSetting - An DTC Pool dynamic ratio setting object.

Copyright (c) 2017 Infoblox Inc.

Infoblox::DTC::Server::Link - An DTC Server Link object.

- NAME
- DESCRIPTION
- CONSTRUCTOR
- SESSION METHODS
- METHODS
  - server( )
  - ratio( )
- AUTHOR
- SEE ALSO
- COPYRIGHT

NAME

Infoblox::DTC::Server::Link - An DTC Server Link object.

DESCRIPTION

An DTC Pool Link object.

CONSTRUCTOR

```
my $server_link = Infoblox::DTC::Server::Link->new(
    server   => $server, #Optional / Default is undefined
    ratio    => $uint,   #Optional / Default is undefined
);
```

SESSION METHODS

The object does not support any session methods.

METHODS

This section describes all the methods that you can use to set or retrieve the attribute values of the object.

server( )

Use this method to set or retrieve a DTC Server to link with.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

The valid value is an Infoblox::DTC::Server object.

Returns

If you specified a parameter, the method returns 'true' when the modification succeeds, and returns 'false' when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#get dtc_server value
my $server = $server_link->server();

#modify dtc_server value
$server_link->dtc_server($server);
```

ratio( )

Use this method to set or retrieve the weight of a DTC Server.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

The valid value is an unsigned integer.

Returns

If you specified a parameter, the method returns 'true' when the modification succeeds, and returns 'false' when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#get ratio value
my $ratio = $server_link->ratio();

#modify ratio value
$server_link->ratio(70);
```

AUTHOR

Infoblox Inc. http://www.infoblox.com/

SEE ALSO

Infoblox::DTC::Server.

COPYRIGHT

Copyright (c) 2017 Infoblox Inc.

Infoblox::DTC::Query::Result - A DTC Query result object.

- NAME
- DESCRIPTION
- SESSION METHODS
- METHODS
  - override_ttl( )
  - ttl( )
  - rdata( )
  - type( )
- AUTHOR
- SEE ALSO
- COPYRIGHT

NAME

Infoblox::DTC::Query::Result - A DTC Query result object.

DESCRIPTION

A DTC Query function call result object.

SESSION METHODS

The object does not support any session methods. It is retrieved as a part of the DTC Query response object.

METHODS

override_ttl( )

Use this method to retrieve the attribute, which controls whether the TTL value of the object is used, instead of the Grid default. This is a read-only attribute.

Parameter

None

Returns

The method returns the attribute value, which can be 'true' or 'false'.

Example

```
#get override_ttl value
my $override_ttl = $result->override_ttl();
```

ttl( )

Use this method to retrieve a TTL value. This is a read-only attribute.

Parameter

None

Returns

The valid return value is an unsigned integer.

Example

```
#get ttl value
my $ttl = $result->ttl();
```

rdata( )

Use this method to retrieve DNS record data. This is a read-only attribute.

Parameter

None

Returns

The method returns the attribute value.

Example

```
#get rdata value
my $rdata = $result->rdata();
```

type( )

Use this method to retrieve the DNS record type. This is a read-only attribute.

Parameter

None

Returns

The valid return values are 'A', 'AAAA', 'CNAME' and 'NAPTR'.

Example

```
#get type value
my $type = $result->type();
```

AUTHOR

Infoblox Inc. http://www.infoblox.com/

SEE ALSO

Infoblox::DTC->query(), Infoblox::DTC::Response.

COPYRIGHT

Copyright (c) 2017 Infoblox Inc.

Infoblox::DTC::Server::Monitor - A DTC Server Monitor object.

- NAME
- DESCRIPTION
- CONSTRUCTOR
- SESSION METHODS
- MODULE METHODS
  - Infoblox::DTC::Server->monitors( )
- METHODS
  - host( )
  - monitor( )
- AUTHOR
- SEE ALSO
- COPYRIGHT

NAME

Infoblox::DTC::Server::Monitor - A DTC Server Monitor object.

DESCRIPTION

The DTC Server Monitor object is used to configure DTC Health Monitor for the particular DTC Server.

CONSTRUCTOR

```
 my $server_monitor = Infoblox::DTC::Server::Monitor->new(
     monitor => $tcp_monitor | $icmp_monitor | $snmp_monitor | $sip_monitor | $pdp_monitor | $http_monitor,
#Required
     host    => $fqdn | $ipaddr,
#Optional / Default is undefined
```

SESSION METHODS

The object does not support any session methods.

MODULE METHODS

This section describes all the functions that you can apply to an DTC Server Monitor object.

Infoblox::DTC::Server->monitors( )

Use this method to set or retrieve the list of DTC health monitors for the DTC Server. See Infoblox::DTC::Server->monitors() for parameters and return values.

Example

```
#get monitors value
my $monitors = $server->monitors();

#modify monitors value

#retrieve TCP monitor
my $tcp_monitor = $session->get(
    object => 'Infoblox::DTC::Monitor::TCP',
    name   => 'tcp_monitor',
);

#construct DTC server monitor
my $monitor = Infoblox::DTC::Server::Monitor->new(
    monitor => $tcp_monitor,
    host    => '192.168.1.10',
);

#modify monitors value
$server->monitors([$monitor]);
```

METHODS

This section describes all the methods that you can use to set or retrieve the attribute values of the object.

host( )

Use this method to set or retrieve the address that is a subject of the DTC health monitoring.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

The valid value is an IP address or FQDN in string format.

Returns

If you specified a parameter, the method returns 'true' when the modification succeeds, and returns 'false' when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get host value
my $value = $object->host();

#Modify host value
$object->host('1.1.1.1');
```

monitor( )

Use this method to set or retrieve the DTC Health Monitor used to monitor the DTC Server.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

The valid value is one of following objects: Infoblox::DTC::Monitor::HTTP, Infoblox::DTC::Monitor::SIP, Infoblox::DTC::Monitor::PDP, Infoblox::DTC::Monitor::SNMP, Infoblox::DTC::Monitor::TCP or Infoblox::DTC::Monitor::ICMP.

Note that yo must first retrieve the Monitor object from the session.

Returns

If you specified a parameter, the method returns 'true' when the modification succeeds, and returns 'false' when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get monitor value
my $value = $object->monitor();

#Retrieve TCP monitor
my $tcp_monitor = $session->get(
    object => 'Infoblox::DTC::Monitor::TCP',
    name   => 'tcp_monitor',
);

#Modify monitor value
$object->monitor($tcp_monitor);
```

AUTHOR

Infoblox Inc. http://www.infoblox.com/

SEE ALSO

Infoblox::DTC::Server, Infoblox::DTC::Server->monitors(), Infoblox::DTC::Monitor::HTTP, Infoblox::DTC::Monitor::SIP, Infoblox::DTC::Monitor::PDP, Infoblox::DTC::Monitor::SNMP, Infoblox::DTC::Monitor::TCP, Infoblox::DTC::Monitor::ICMP.

COPYRIGHT

Copyright (c) 2017 Infoblox Inc.

Infoblox::DTC::Topology::Rule - A DTC Topology Rule object.

- NAME
- DESCRIPTION
- CONSTRUCTOR
- SESSION METHODS
  - Infoblox::Session->get( )
  - Infoblox::Session->modify( )
  - Infoblox::Session->search( )
- METHODS
  - destination_link( )
  - destination_type( )
  - sources( )
  - topology( )
  - valid( )
- AUTHOR
- SEE ALSO
- COPYRIGHT

NAME

Infoblox::DTC::Topology::Rule - A DTC Topology Rule object.

DESCRIPTION

Topology Rule describes the behavior of a load balancer used for a matching condition contained in the rules for a 'TOPOLOGY' load balancing method.

CONSTRUCTOR

```
my $rule = Infoblox::DTC::Topology::Rule->new(
   sources          => [$source1, $source2, ...], #Optional / default is undefined
   destination_type => 'POOL' | 'SERVER',         #Optional / default is undefined
   destination_link => $link_object               #Optional / default is undefined
);
```

SESSION METHODS

This section describes all the methods in a Session module that you can apply to a Topology Rule object. Note that to add a rule to the Infoblox appliance, you must add it to the existing Topology object. To remove a rule from the Infoblox appliance, you must remove it from the existing Topology.

Infoblox::Session->get( )

Use this method to retrieve all the matching objects from the Infoblox appliance. See Infoblox::Session->get() for parameters and return values.

Key Reference

```
Apply the following attribute to get a specific object:

 topology         - Optional. A DTC Topology the rule belongs to.
 destination_type - Optional. The type of the destination link.
 destination_link - Optional. The destination link value.
 valid            - Optional. The flag that indicates whether a rule is valid or not.
```

Example

```
my @retrieved_objs = $session->get(
    object => 'Infoblox::DTC::Topology::Rule',
    destination_type => 'POOL',
);
```

Infoblox::Session->modify( )

Use this method to modify an object in the Infoblox appliance. See Infoblox::Session->modify() for parameters and return values.

Example

```
#Use method to modify the comment
```

```
$rule->comment('modified_rule1');
#Submit modification
my $response = $session->modify($rule);
```

Infoblox::Session->search( )

Use this method to search for objects in the Infoblox appliance. See Infoblox::Session->search() for parameters and return values.

Key Reference

```
Apply the following attribute to get a specific object:

 topology         - Optional. A DTC Topology the rule belongs to.
 destination_type - Optional. The type of the destination link.
 destination_link - Optional. The destination link value.
 valid            - Optional. The flag that indicates whether a rule is valid or not.
```

Example

```
my @retrieved_objs = $session->search(
    object => 'Infoblox::DTC::Topology::Rule',
    valid  => 'true',
);
```

METHODS

This section describes all the methods that you can use to set or retrieve the attribute values of the object.

destination_link( )

Use this method to set or retrieve a DTC Topology Rule destination link.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

The valid value is an Infoblox::DTC::Pool or Infoblox::DTC::Server object.

Returns

If you specified a parameter, the method returns 'true' when the modification succeeds, and returns 'false' when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#get destination_link value
my $destination_link = $rule->destination_link();

#modify destination_link value
$rule->destination_link($pool_link);
```

destination_type( )

Use this method to set or retrieve the destination type for a DTC Topology rule.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

The valid values are 'POOL' and 'SERVER'.

Returns

If you specified a parameter, the method returns 'true' when the modification succeeds, and returns 'false' when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#get destination_type value
my $destination_type = $rule->destination_type();

#modify destination_type value
$rule->destination_type('SERVER');
```

sources( )

Use this method to set or retrieve conditions for matching sources.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

The valid value is an array of Infoblox::DTC::Topology::Rule::Source objects.

Returns

If you specified a parameter, the method returns 'true' when the modification succeeds, and returns 'false' when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#get sources value
my @sources = $rule->sources();

#modify sources value
$rule->sources([$source1, $source2]);
```

topology( )

Use this method to retrieve the DTC Topoloyg to which a rule belongs. This is a read-only attribute.

Parameter

None

Returns

The valid return value is an partial Infoblox::DTC::Topology object.

Example

```
#get sources value
my $topology = $rule->topology();
```

valid( )

Use this method to retrieve the status of GeoIP labels. This is a read-only attribute.

Parameter

None

Returns

The method returns 'true' if GeoIP labels exist or 'false' if they do not.

Example

```
#get sources value
my $valid = $rule->valid();
```

AUTHOR

Infoblox Inc. http://www.infoblox.com/

SEE ALSO

Infoblox::Session, Infoblox::Session->get(), Infoblox::Session->modify(), Infoblox::Session->search(), Infoblox::DTC::Pool, Infoblox::DTC::Server, Infoblox::DTC::Topology::Rule::Source, Infoblox::DTC::Topology.

COPYRIGHT

Copyright (c) 2017 Infoblox Inc.

Infoblox::DTC::Topology::Rule::Source - A DTC Topology Rule Source object.

- NAME
- DESCRIPTION
- CONSTRUCTOR
- SESSION METHODS
- METHODS
  - source_op( )
  - source_type( )
  - source_value( )
- AUTHOR
- SEE ALSO
- COPYRIGHT

NAME

Infoblox::DTC::Topology::Rule::Source - A DTC Topology Rule Source object.

DESCRIPTION

A DTC Topology Rule Source object.

CONSTRUCTOR

```
 my $rule_source = Infoblox::DTC::Topology::Rule::Source->new(
    source_type  => 'SUBNET' | 'CONTINENT' | 'COUNTRY' | 'SUBDIVISION' |
                    'CITY' | 'EA0' | 'EA1' | 'EA2' | 'EA3',                    #Required
    source_value => $string,                                                    #Required
    source_op    => 'IS' | 'IS_NOT',                                            #Optional / Default is
'IS'
 );
```

SESSION METHODS

The object does not support any session methods. It's retrieved as a part of an Infoblox::DTC::Topology::Rule, and used as a matching conditions for a DTC load balancer.

METHODS

This section describes all the methods that you can use to set or retrieve the attribute values of the object.

source_op( )

Use this method to set or retrieve the operation used to match the value.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

The valid values are 'IS' and 'IS_NOT'.

Returns

If you specified a parameter, the method returns 'true' when the modification succeeds, and returns 'false' when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#get source_op value
my $source_op = $source->source_op();

#modify source_op value
$source->source_op('IS');
```

source_type( )

Use this method to set or retrieve the source type.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

The valid values are 'SUBNET', 'CONTINENT', 'COUNTRY', 'CITY', 'SUBDIVISION', 'EA0', 'EA1', 'EA2' and 'EA3'.

Returns

If you specified a parameter, the method returns 'true' when the modification succeeds, and returns 'false' when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#get source_type value
my $source_type = $source->source_type();

#modify source_type value
$source->source_type('COUNTRY');
```

source_value( )

Use this method to set or retrieve the source value.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

The valid value is a string that contains the desired source value.

Returns

If you specified a parameter, the method returns 'true' when the modification succeeds, and returns 'false' when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#get source_value value
my $source_value = $source->source_value();

#modify source_value value
$source->source_value('United States');
```

AUTHOR

Infoblox Inc. http://www.infoblox.com/

SEE ALSO

Infoblox::DTC::Topology::Rule

COPYRIGHT

Copyright (c) 2017 Infoblox Inc.

Infoblox::DXL::Endpoint::Broker - A Data Exchange Layer endpoint broker object.

- NAME
- DESCRIPTION
- CONSTRUCTOR
- SESSION METHODS
- MODULE METHODS
  - Infoblox::DXL::Endpoint->brokers( )
- METHODS
  - address( )
  - host_name( )
  - port( )
  - unique_id( )
- AUTHOR
- SEE ALSO
- COPYRIGHT

NAME

Infoblox::DXL::Endpoint::Broker - A Data Exchange Layer (DXL) endpoint broker object.

DESCRIPTION

DXL brokers are installed on managed systems for routing messages between connected clients. The network of brokers tracks active consumers and dynamically adjusts the message routing as needed. When a client requests a service or when an update is broadcasted, brokers relay these messages.

The DXL endpoint broker is used to configure DXL broker for a particular endpoint.

CONSTRUCTOR

```
my $object = Infoblox::DXL::Endpoint::Broker->new(
    host_name => $fqdn,    #Required
    address   => $ipaddr, #Optional / Default is undefined
    port      => $uint,    #Optional / Default is 8883
    unique_id => $string, #Optional / Default is undefined
);
```

SESSION METHODS

The object does not support any session methods.

MODULE METHODS

This section describes all the functions that you can apply to a DXL Broker object.

Infoblox::DXL::Endpoint->brokers( )

Use this method to set or retrieve the list of DXL brokers for the DXL endpoint. See Infoblox::DXL::Endpoint->brokers() for parameters and return values.

Example

```
#Get brokers value
my $brokers = $object->brokers();

#Construct Infoblox::DXL::Endpoint::Broker object
my $broker = Infoblox::DXL::Endpoint::Broker->new(
    host_name => 'dxl-br.localdomain',
);

#Modify brokers value
$object->brokers([$broker]);
```

METHODS

This section describes all the methods that you can use to set or retrieve the attribute values of the object.

address( )

Use this method to set or retrieve the DXL broker IP address.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

The valid value is an IP address in string format.

Returns

If you specified a parameter, the method returns 'true' when the modification succeeds, and returns 'false' when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get address value
my $value = $object->address();

#Modify address value
$object->address('1.1.1.1');
```

host_name( )

Use this method to set or retrieve the host name for the DXL broker.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

The valid value is a desired FQDN in string format.

Returns

If you specified a parameter, the method returns 'true' when the modification succeeds, and returns 'false' when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get host_name value
my $value = $object->host_name();

#Modify host_name value
$object->host_name('dxl-broker.localdomain');
```

port( )

Use this method to set or retrieve the port number for the DXL broker communication.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

The valid value is an unsigned integer between 1 and 65535. The default value is 8883.

Returns

If you specified a parameter, the method returns 'true' when the modification succeeds, and returns 'false' when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get port value
my $value = $object->port();

#Modify port value
$object->port('1312');
```

unique_id( )

Use this method to set or retrieve the unique ID for the DXL broker.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

The valid value is a desired ID in string format.

Returns

If you specified a parameter, the method returns 'true' when the modification succeeds, and returns 'false' when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get unique_id value
my $value = $object->unique_id();

#Modify unique_id value
$object->unique_id('5d73b77f-8c4b-4ae0-b437-febd12facfd4');
```

AUTHOR

Infoblox Inc. http://www.infoblox.com/

SEE ALSO

Infoblox::DXL::Endpoint, Infoblox::DXL::Endpoint->brokers().

COPYRIGHT

Copyright (c) 2017 Infoblox Inc.

Infoblox::Notification::REST::TemplateInstance - A REST API template instance.

- NAME
- DESCRIPTION
- CONSTRUCTOR
- SESSION METHODS
- MODULE METHODS
  - Infoblox::Notification::Rule->template_instance( )
  - Infoblox::Notification::REST::Endpoint->template_instance( )
- METHODS
  - template( )
  - parameters( )
- AUTHOR
- SEE ALSO
- COPYRIGHT

NAME

Infoblox::Notification::REST::TemplateInstance - A REST API template instance.

DESCRIPTION

The REST API template instance is used to configure REST API template setting for notification rule and REST API endpoint.

CONSTRUCTOR

```
my $templ_instance = Infoblox::Notification::REST::TemplateInstance->new(
    template   => $string,                  #Required
    parameters => [$param1, $param2, ...], #Optional / Default is empty list
);
```

SESSION METHODS

The object does not have any session methods.

MODULE METHODS

This section describes all the functions that you can apply to REST API template instance object.

Infoblox::Notification::Rule->template_instance( )

Use this method to set or retrieve the REST API template instance. See Infoblox::Notification::Rule->template_instance() for parameters and return values.

Example

```
#Get template_instance value
my $template_instance = $object->template_instance();

#Modify template_instance value
$object->template_instance($instance);
```

Infoblox::Notification::REST::Endpoint->template_instance( )

Use this method to set or retrieve the REST API template instance. See Infoblox::Notification::REST::Endpoint->template_instance() for parameters and return values.

Example

```
#Get template_instance value
my $template_instance = $object->template_instance();

#Modify template_instance value
$object->template_instance($instance);
```

METHODS

This section describes all the methods that you can use to configure and retrieve the attribute values of a REST API template instance

object.

template( )

Use this method to set or retrieve the REST API template name.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

The valid value is an Infoblox::Notification::REST::Template object name.

Returns

If you have specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get template value
my $template = $object->template();

#Modify template value
$object->template('template1');
```

parameters( )

Use this method to set or retrieve the list of template parameters.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

The valid value is array of Infoblox::Notification::REST::TemplateParameter objects.

Returns

If you have specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get parameters value
my $parameters = $object->parameters();

#Modify parameters value
$object->parameters([$param1, $param2]);
```

AUTHOR

Infoblox Inc. http://www.infoblox.com/

SEE ALSO

Infoblox::Notification::REST::Template, Infoblox::Notification::REST::TemplateParameter

---

COPYRIGHT

Copyright (c) 2017 Infoblox Inc.

Infoblox::DXL::Endpoint - A Data Exchange Layer endpoint object.

- NAME
- DESCRIPTION
- CONSTRUCTOR
- SESSION METHODS
  - Infoblox::Session->add( )
  - Infoblox::Session->get( )
  - Infoblox::Session->modify( )
  - Infoblox::Session->remove( )
  - Infoblox::Session->search( )
- METHODS
  - brokers( )
  - client_certificate( )
  - client_certificate_subject( )
  - client_certificate_valid_from( )
  - client_certificate_valid_to( )
  - comment( )
  - disable( )
  - extattrs( )
  - extensible_attributes( )
  - name( )
  - outbound_members( )
  - outbound_member_type( )
  - log_level( )
  - template_instance( )
  - vendor_identifier( )
  - wapi_user_name( )
  - wapi_user_password( )
- AUTHOR
- SEE ALSO
- COPYRIGHT

NAME

Infoblox::DXL::Endpoint - A Data Exchange Layer (DXL) endpoint object.

DESCRIPTION

The DXL endpoint object provides configuration for third party DXL server integration.

CONSTRUCTOR

```
 my $endpoint = Infoblox::DXL::Endpoint->new(
     name                  => $string,                                                                  #
Required
     outbound_member_type  => 'GM' | 'MEMBER',                                                          #
Required
     brokers               => [$endpoint_dxl_broker1, ...],                                             #
Required
     client_certificate    => $path,                                                                    #
Required
     comment               => $string,                                                                  #
Optional / Default is undefined
     disable               => 'true' | 'false',                                                         #
Optional / Default is 'false'
     extattrs              => { $string => $extattr, ... },                                             #
Optional / Default is undefined
     extensible_attributes => {$string => $string | $num, $string => [ $string | $num, ... ], ... } #
Optional / Default is undefined
     log_level             => 'WARNING' | 'DEBUG' | 'INFO' | 'ERROR',                                   #
Optional / Default is 'WARNING'
     outbound_members      => [$string, $string, ...],                                                  #
Optional / Default is empty list
     template_instance     => $template_instance                                                        #
Optional / Default is undefined
     vendor_identifier     => $string,                                                                  #
Optional / Default is undefined
     wapi_user_name        => $string,                                                                  #
Optional / Default is undefined
     wapi_user_password    => $string,                                                                  #
Optional / Default is undefined
 );
```

You cannot set both extattrs and extensible_attributes attributes at the same time.

SESSION METHODS

This section describes all the methods in an Infoblox::Session module that you can apply to a DXL endpoint object.

Infoblox::Session->add( )

Use this method to add the object to the Infoblox appliance. See Infoblox::Session->add() for parameters and return values.

Example

```
#Construct Infoblox::DXL::Endpoint::Broker object
my $broker = Infoblox::DXL::Endpoint::Broker->new(
    host_name => 'dxl-br.localdomain',
);

#Construct an object
my $object = Infoblox::DXL::Endpoint->new(
    name                 => 'dxl_ep',
    outbound_member_type => 'GM',
    brokers              => [$broker],
    client_certificate   => '/path/to/certificate.pem',
);

#Submit for addition
my $response = $session->add($object);
```

Infoblox::Session->get( )

Use this method to retrieve all the matching objects from the Infoblox appliance. See Infoblox::Session->get() for parameters and return values.

Key References

```
Apply the following attributes to get a specific DXL endpoint object:

 comment                  - Optional. The DXL endpoint comment in string format.
 name                     - Optional. The DXL endpoint name in string format.
 vendor_identifier        - Optional. The DXL endpoint vendor identifier in string format.
 outbound_member_type     - Optional. The DXL endpoint outbound member type.
 extattrs                 - Optional. A hash reference containing Infoblox::Grid::Extattr object.
 extensible_attributes    - Optional. A hash reference containing extensible attributes.
```

Examples

```
my @retrieved_objs = $session->get(
    object => 'Infoblox::DXL::Endpoint',
    name   => 'dxl_ep',
);

my @retrieved_objs = $session->get(
    object               => 'Infoblox::DXL::Endpoint',
    extensible_attributes => {'Site' => 'Santa Clara'},
);
```

Infoblox::Session->modify( )

Use this method to modify the object in the Infoblox appliance. See Infoblox::Session->modify() for parameters and return values.

Example

```
#Modify comment value
$object->comment('this is a modified comment');

#Submit modification
my $response = $session->modify($object);
```

Infoblox::Session->remove( )

Use this method to remove the object from the Infoblox appliance. See Infoblox::Session->remove() for parameters and return values.

To remove a specific object, first use `get()` or `search()` to retrieve the specific DXL endpoint object, and then submit this object for removal.

Example

```
#Get the objects
my @retrieved_objs = $session->get(
    object => 'Infoblox::DXL::Endpoint',
    name   => 'dxl_ep',
);

#Find the desired object from the retrieved list
my $desired_object = $retrieved_objs[0];

#Submit for removal
my $response = $session->remove($desired_object);
```

Infoblox::Session->search( )

Use this method to search for objects in the Infoblox appliance. See Infoblox::Session->search() for parameters and return values.

Key References

```
Apply the following attributes to search for the DXL endpoint object:

 comment                    - Optional. The DXL endpoint comment in string format (regexp).
 name                       - Optional. The DXL endpoint name in string format (regexp).
 vendor_identifier          - Optional. The DXL endpoint vendor identifier in string format
(regexp).
 outbound_member_type       - Optional. The DXL endpoint outbound member type.
 extattrs                   - Optional. A hash reference containing Infoblox::Grid::Extattr object.
 extensible_attributes      - Optional. A hash reference containing extensible attributes.
```

For more information about searching extensible attributes, see Infoblox::Grid::ExtensibleAttributeDef/Searching Extensible Attributes.

Examples

```
# search for objects
my @retrieved_objs = $session->search(
    object            => 'Infoblox::DXL::Endpoint',
    name              => 'dxl_.*',
    vendor_identifier => 'Carbon',
);

# search for objects defining 'Santa Clara' for 'Site' extensible attribute
my @retrieved_objs = $session->search(
    object                => 'Infoblox::DHCP::Range',
    extensible_attributes => {'Site' => 'Santa Clara'},
);
```

METHODS

This section describes all the methods that you can use to configure and retrieve the attribute values of a DXL endpoint.

brokers( )

Use this method to set or retrieve the list of DXL brokers for the DXL endpoint.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

The valid value is an array reference that contains Infoblox::DXL::Endpoint::Broker objects.

Returns

If you have specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get brokers value
my $brokers = $object->brokers();

#Construct Infoblox::DXL::Endpoint::Broker object
my $broker = Infoblox::DXL::Endpoint::Broker->new(
    host_name => 'dxl-br.localdomain',
);

#Modify brokers value
$object->brokers([$broker]);
```

client_certificate( )

Use this method to set the path to a client certificate file. This is a write-only attribute.

Include the specified parameter to set the attribute value.

Parameter

The valid value is a path to a certificate in string format.

Returns

If you have specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

Example

```
#Modify client_certificate value
$object->client_certificate('/path/to/cert.pem');
```

client_certificate_subject( )

Use this method to retrieve the client certificate subject. This is a read-only attrbiute.

Omit the parameter to retrieve the attribute value.

Parameter

None

Returns

The method returns the attribute value.

Example

```
#Get client_certificate_subject value
my $client_certificate_subject = $object->client_certificate_subject();
```

client_certificate_valid_from( )

Use this method to retrieve the time from which the client certificate is valid. This is a read-only attrbiute.

Omit the parameter to retrieve the attribute value.

Parameter

None

Returns

The method returns the attribute value. The number of seconds that have elapsed since January 1st, 1970 UTC.

Example

```
#Get client_certificate_valid_from value
my $client_certificate_valid_from = $object->client_certificate_valid_from();
```

client_certificate_valid_to( )

Use this method to retrieve the expiration time for the client certificate. This is a read-only attrbiute.

Omit the parameter to retrieve the attribute value.

Parameter

None

Returns

The method returns the attribute value. The number of seconds that have elapsed since January 1st, 1970 UTC.

Example

```
#Get client_certificate_valid_to value
my $client_certificate_valid_to = $object->client_certificate_valid_to();
```

comment( )

Use this method to set or retrieve the DXL endpoint comment.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

The valid value is a desired comment in string format.

Returns

If you have specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get comment value
my $comment = $object->comment();

#Modify comment value
$object->comment('modified comment');
```

disable( )

Use this method to set or retrieve the flag that indicates whether the DXL endpoint is enabled or disabled.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

Specify 'true' to disable the DXL endpoint or 'false' to enable it. The default value is 'true'.

Returns

If you have specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get disable value
my $disable = $object->disable();

#Modify disable value
$object->disable('false');
```

extattrs( )

Use this method to set or retrieve the extensible attributes associated with a DXL endpoint object.

Parameter

Valid value is a hash reference that contains the names of extensible attributes and their associated values (Infoblox::Grid::Extattr objects).

Returns

If you have specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get extattrs value
my $extattrs = $object->extattrs();

#Modify extattrs value
$object->extattrs({'Site' => $extattr1, 'Administrator' => $extattr2});
```

extensible_attributes( )

Use this method to set or retrieve the extensible attributes associated with a DXL endpoint object.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

For valid values for extensible attributes, see Infoblox::Grid::ExtensibleAttributeDef/Extensible Attribute Values.

Returns

If you have specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get extensible attributes value
my $extensible_attributes = $object->extensible_attributes();
#Modify extensible attributes
$object->extensible_attributes({'Site' => 'Santa Clara', 'Administrator' => ['Peter', 'Tom']});
```

name( )

Use this method to set or retrieve the DXL endpoint name.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

The valid value is a desired name in string format.

Returns

If you have specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get name value
my $name = $object->name();

#Modify name value
$object->name('modified_name');
```

outbound_members( )

Use this method to set or retrieve the list of DXL endpoint members that are responsible for event generation.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

The valid value is an array that contains Grid member names in string format.

Returns

If you have specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get outbound_members value
my $outbound_members = $object->outbound_members();

#Modify outbound_members value
$object->outbound_members(['member1', 'member2']);
```

outbound_member_type( )

Use this method to set or retrieve the DXL endpoint member that is responsible for event generation.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

The valid values are 'GM' and 'MEMBER'.

Returns

If you have specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get outbound_member_type value
my $outbound_member_type = $object->outbound_member_type();

#Modify outbound_member_type value
$object->outbound_member_type('MEMBER');
```

log_level( )

Use this method to set or retrieve the DXL endpoint logging level.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

The valid values are 'DEBUG', 'ERROR', 'INFO', and 'WARNING'.

Returns

If you have specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get log_level value
my $value = $object->log_level();

#Modify log_level value
$object->log_level('INFO');
```

template_instance( )

Use this method to set or retrieve the DXL endpoint template instance.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Note that you cannot change the 'parameters' value of the template instance object for a DXL endpoint.

Parameter

The valid value is an Infoblox::Notification::REST::TemplateInstance object.

Returns

If you have specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get template_instance value
my $value = $object->template_instance();

#Modify template_instance value
$object->template_instance($instance);
```

vendor_identifier( )

Use this method to set or retrieve the DXL endpoint vendor identifier.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

The valid value is a desired vendor identifier in string format.

Returns

If you have specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get vendor_identifier value
my $vendor_identifier = $object->vendor_identifier();

#Modify vendor_identifier value
$object->vendor_identifier('CarbonBlack');
```

wapi_user_name( )

Use this method to set or retrieve the user name for WAPI integration.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

The valid value is a desired user name in string format.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get wapi_user_name value
my $value = $object->wapi_user_name();

#Modify wapi_user_name value
$object->wapi_user_name('admin');
```

wapi_user_password( )

Use this method to set the user password for WAPI integration. This is a write-only attribute.

Include the specified parameter to set the attribute value.

Parameter

The valid value is a desired user password in string format.

Returns

The method returns true when the modification succeeds, and returns false when the operation fails.

Example

```
#Modify wapi_user_password value
$object->wapi_user_password('infoblox');
```

AUTHOR

Infoblox Inc. http://www.infoblox.com/

SEE ALSO

Infoblox::Session, Infoblox::Session->add(), Infoblox::Session->get(), Infoblox::Session->modify(), Infoblox::Session->remove(), Infoblox::Session->search(), Infoblox::Grid::Extattr, Infoblox::Grid::ExtensibleAttributeDef/Searching Extensible Attributes, Infoblox::Notification::REST::TemplateInstance, Infoblox::DXL::Endpoint::Broker.

COPYRIGHT

Copyright (c) 2017 Infoblox Inc.

Infoblox::Grid::ConsentBannerSetting - The consent banner setting.

- NAME
- DESCRIPTION
- CONSTRUCTOR
- MODULE METHODS
  - Infoblox::Grid->consent_banner_setting( )
- METHODS
  - enable( )
  - message( )
- AUTHOR
- SEE ALSO
- COPYRIGHT

NAME

Infoblox::Grid::ConsentBannerSetting - The consent banner setting.

DESCRIPTION

The consent banner setting object controls the banner displayed on the login page of Grid Manager.

CONSTRUCTOR

```
my $consent = Infoblox::Grid::ConsentBannerSetting->new(
    enable    => "true" | "false", #Mandatory
    message   => $string,          #Optional / Default is empty
);
```

MODULE METHODS

The following functions are available to apply to a consent banner setting object.

Infoblox::Grid->consent_banner_setting( )

Use this function to specify the consent banner settings for the Grid. See Infoblox::Grid->consent_banner_setting() for parameters and return values.

Example

```
 my $grid = $session->get(object => 'Infoblox::Grid');

 my $banner = Infoblox::Grid::ConsentBannerSetting->new(enable => 'true', message => 'Consent
message');
 $grid->consent_banner_setting($banner);
 my $response = $session->modify($grid);
 unless($response) {
      die("Modify grid failed: ",
            session->status_code() . ":" . session->status_detail());
 }
 print "Grid banner setting modified successfully\n";
```

METHODS

This section describes all the methods that you can use to set and retrieve the attribute values of a consent banner setting object.

enable( )

Use this method to set the enable flag.

Parameter

Specify "true" to enable the banner or "false" to disable it.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get enable
my $enable = $banner->enable();
#Modify enable
$banner->enable("true");
```

message( )

Use this method to set the message that will be displayed on the login page of Grid Manager.

Parameter

A string containing the message.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get message
my $message = $banner->message();
#Modify message
$banner->message("Test message");
```

AUTHOR

Infoblox Inc. http://www.infoblox.com/

SEE ALSO

Infoblox::Grid

COPYRIGHT

Copyright (c) 2017 Infoblox Inc.

Infoblox::Grid::ExtensibleAttributeDef::Descendants - an action for the extensible attributes that exist on descendants.

- NAME
- DESCRIPTION
- CONSTRUCTOR
- SESSION METHODS
- METHODS
  - option_delete_ea( )
  - option_with_ea( )
  - option_without_ea( )
- AUTHOR
- COPYRIGHT

NAME

Infoblox::Grid::ExtensibleAttributeDef::Descendants - an action for the extensible attributes that exist on descendants.

DESCRIPTION

An object that provides the action for extensible attributes that exist on descendants.

CONSTRUCTOR

```
my $descendants = Infoblox::Grid::ExtensibleAttributeDef::Descendants->new(
    option_without_ea => "INHERIT" | "NOT_INHERIT",             # optional
    option_with_ea    => "RETAIN" | "CONVERT" | "INHERIT",      # optional
    option_delete_ea  => "RETAIN" | "REMOVE",                   # optional
);
```

SESSION METHODS

The object does not support any session methods.

METHODS

This section describes all the methods that you can use to configure and retrieve attribute values of the extensible attributes that exist on descendants based on the corresponding action.

option_delete_ea( )

Use this method to set or retrieve the action for objects that contain inheritable extensible attributes if a parent object is deleted.

Parameter

Valid value is "RETAIN" or "REMOVE".

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get attribute value
my $value = $descendants->option_delete_ea();
#Modify attribute value
$descendants->option_delete_ea('RETAIN');
```

option_with_ea( )

Use this method to set or retrieve the action for objects that contain inheritable extensible attributes.

Parameter

Valid value is "RETAIN", "CONVERT" or "INHERIT".

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get attribute value
my $value = $descendants->option_with_ea();
#Modify attribute value
$descendants->option_with_ea('INHERIT');
```

option_without_ea( )

Use this method to set or retrieve the action for objects that do not contain inheritable extensible attributes.

Parameter

Valid value is "INHERIT" or "NOT_INHERIT".

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get attribute value
my $value = $descendants->option_without_ea();
#Modify attribute value
$descendants->option_without_ea('NOT_INHERIT');
```

AUTHOR

Infoblox Inc. http://www.infoblox.com/

COPYRIGHT

Copyright (c) 2017 Infoblox Inc.

Infoblox::Grid::HTTPProxyServerSetting - An Infoblox Grid HTTP Proxy Server Setting object.

- NAME
- DESCRIPTION
- CONSTRUCTOR
- SESSION METHODS
- MODULE METHODS
  - Infoblox::Grid->http_proxy_server_setting( )
- METHODS
  - address( )
  - certificate( )
  - comment( )
  - enable_content_inspection( )
  - enable_username_and_password( )
  - enable_proxy( )
  - password( )
  - port( )
  - username( )
  - verify_cname( )
- AUTHOR
- SEE ALSO
- COPYRIGHT

NAME

Infoblox::Grid::HTTPProxyServerSetting - An Infoblox Grid HTTP Proxy Server Setting object.

DESCRIPTION

The Grid HTTP Proxy Server Setting object provides information about HTTP Proxy Server configuration.

CONSTRUCTOR

```
my $http_proxy_setting = Infoblox::Grid::HTTPProxyServerSetting->new(
    address                     => $fqdn | $ipv4addr | $ipv6addr, # Required
    port                        => $uint,                         # Required
    enable_username_and_password => 'true' | 'false',             # Optional / Default is 'false'
    enable_proxy                => 'true' | 'false',              # Optional / Default is 'false'
    enable_content_inspection   => 'true' | 'false',              # Optional / Default is 'false'
    verify_cname                => 'true' | 'false',              # Optional / Default is 'false'
    comment                     => $string,                       # Optional / Default is undefined
    username                    => $string,                       # Optional / Default is undefined
    password                    => $string,                       # Optional / Default is undefined
    certificate                 => $string,                       # Optional / Default is undefined
);
```

SESSION METHODS

The object does not support any session methods.

MODULE METHODS

The following functions are available to be applied to a HTTP Proxy Server object.

Infoblox::Grid->http_proxy_server_setting( )

Use this method to set or retreive the Grid HTTP proxy server settings. See Infoblox::Grid->http_proxy_server_setting() for parameters andreturn values.

Example

```
#Get http_proxy_server_setting value
my $http_proxy_server_setting = $grid->http_proxy_server_setting();

#Modify http_proxy_server_setting value
$grid->http_proxy_server_setting($http_proxy_setting);
```

METHODS

This section describes all the methods that you can use to configure and retrieve the attribute values of a HTTP Proxy Server Setting object.

address( )

Use this method to set or retrieve the address of the HTTP Proxy Server.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

The valid value is a string that contains Fully-Qualified Domain Name (FQDN), IPv4 address or IPv6 address.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get address value
my $address = $obj->address();

#Modify address value
$obj->address('10.0.0.1');
```

certificate( )

Use this method to set the path to a CA Certificate used in a content inspection by a HTTP Proxy Server. This is a write-only attribute.

Include the specified parameter to set the attribute value.

Parameter

The valid value is a string which contains a path to certificate.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

Example

```
#Modify certificateibute value
$obj->certificate('/path/to/cert.pem');
```

comment( )

Use this method to set or retrieve the descriptive comment for HTTP Proxy Server configuration.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

The valid value is a desired comment in a string format.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get comment value
my $comment = $obj->attr();

#Modify comment value
$obj->comment('desired comment');
```

enable_content_inspection( )

Use this method to set or retrieve the flag that indicates whether the HTTPS packets content inspection by HTTP Proxy Server is enabled or disabled.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

Specify 'true' to enable HTTPS packets content inspection or 'false' to disable it. The default value is 'false'.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get enable_content_inspection value
my $enable_content_inspection = $obj->enable_content_inspection();

#Modify enable_content_inspection value
$obj->enable_content_inspection('true');
```

enable_username_and_password( )

Use this method to set or retrieve the flag that indicates whether the use of username and password for the connectivity of HTTP proxy server is enabled or not.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

Specify 'true' to use username and password for testing the connectivity of HTTP proxy server or 'false' to disallow it. The default value is 'false'.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get enable_username_and_password value
my $enable_username_and_password = $obj->enable_username_and_password();

#Modify enable_username_and_password value
$obj->enable_username_and_password('true');
```

enable_proxy( )

Use this method to set or retrieve the flag that indicates whether HTTP Proxy Server is enabled or disabled.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

Specify 'true' to enable HTTP Proxy Server or 'false' to disable it. The default value is 'false'.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get enable_proxy value
my $enable_proxy = $obj->enable_proxy();

#Modify enable_proxy value
$obj->enable_proxy('true');
```

password( )

Use this method to set the HTTP Proxy password. This is a write-only attribute.

Include the specified parameter to set the attribute value.

Parameter

The valid value is a desired password in a string format.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

Example

```
#Modify password value
$obj->password('pass');
```

port( )

Use this method to set or retrieve the HTTP Proxy Server port it listens to.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

The valid value is an unsigned integer between 0 and 65535.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get port value
my $port = $obj->port();

#Modify port value
$obj->port(10000);
```

username( )

Use this method to set or retrieve the HTTP Proxy user name.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

The valid value is a desired user name in a string format.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get username value
my $username = $obj->username();

#Modify username value
$obj->username('user1');
```

verify_cname( )

Use this method to set or retrieve the flag which indicates whether the CNAME record query verification is enabled or not.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

Specify 'true' to enable CNAME record query verification or 'false' to disable it. The default value is 'false'.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get verify_cname value
my $verify_cname = $obj->verify_cname();

#Modify verify_cname value
$obj->verify_cname('true');
```

AUTHOR

Infoblox Inc. http://www.infoblox.com/

SEE ALSO

Infoblox::Grid, Infoblox::Grid->http_proxy_server_setting()

COPYRIGHT

Copyright (c) 2017 Infoblox Inc.

Infoblox::Grid::InformationalBannerSetting - The informational banner setting.

- NAME
- DESCRIPTION
- CONSTRUCTOR
- MODULE METHODS
  - Infoblox::Grid->informational_banner_setting( )
- METHODS
  - enable( )
  - color( )
  - message( )
- AUTHOR
- SEE ALSO
- COPYRIGHT

NAME

Infoblox::Grid::InformationalBannerSetting - The informational banner setting.

DESCRIPTION

The informational banner setting object that controls the banner displayed on the login page of Grid Manager.

CONSTRUCTOR

```
 my $color = "BLACK" | "BLUE" | "BROWN" | "CYAN" | "GREEN" | "MAGENTA" | "ORANGE" | "PURPLE" | "RED" |
"YELLOW";

 my $informational = Infoblox::Grid::InformationalBannerSetting->new(
     enable    => "true" | "false", #Mandatory
     color     => $color,           #Mandatory
     message   => $string,          #Optional / Default is empty
 );
```

MODULE METHODS

The following functions are available to apply to an informational banner setting object.

Infoblox::Grid->informational_banner_setting( )

Use this function to specify the informational banner settings for the Grid. See Infoblox::Grid->informational_banner_setting() for parameters and return values.

Example

```
      my $grid = $session->get(object => 'Infoblox::Grid');

      my $banner = Infoblox::Grid::InformationalBannerSetting->new(enable => 'true', color => 'BLUE',
    message => 'Informational message');
      $grid->informational_banner_setting($banner);
      my $response = $session->modify($grid);
      unless($response) {
           die("Modify grid failed: ",
                 session->status_code() . ":" . session->status_detail());
      }
      print "Grid banner setting modified successfully\n";
```

METHODS

This section describes all the methods that you can use to set and retrieve the attribute values of an informational banner setting object.

enable( )

Use this method to set the enable flag.

Parameter

Specify "true" to enable the banner or "false" to disable it.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get enable
my $enable = $banner->enable();
#Modify enable
$banner->enable("true");
```

color( )

Use this method to set the color of the message that will be displayed on the login page of Grid Manager.

Parameter

A string containing the color. Banner colors can be one of the following: 'BLACK', 'BLUE', 'BROWN', 'CYAN', 'GREEN', 'MAGENTA', 'ORANGE', 'PURPLE', 'RED', 'YELLOW'.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get color
my $color = $banner->color();
#Modify color
$banner->color("RED");
```

message( )

Use this method to set the message that will be displayed on the login page of Grid Manager.

Parameter

A string containing the message.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get message
my $message = $banner->message();
#Modify message
$banner->message("Test message");
```

AUTHOR

Infoblox Inc. http://www.infoblox.com/

SEE ALSO

Infoblox::Grid

COPYRIGHT

Copyright (c) 2017 Infoblox Inc.

Infoblox::Grid::LOM::User - User information for the Lights Out Management functionality.

- NAME
- DESCRIPTION
- CONSTRUCTOR
- MODULE METHODS
  - Infoblox::Grid::Member->bgp_as( )
- METHODS
  - disable( )
  - name( )
  - password( )
  - role( )
- AUTHOR
- SEE ALSO
- COPYRIGHT

NAME

Infoblox::Grid::LOM::User - User information for the Lights Out Management functionality.

DESCRIPTION

An Infoblox::Grid::LOM::User object represents a user for the LOM functionality.

CONSTRUCTOR

```
my $user    = Infoblox::Grid::LOM::User->new(
     name     => $string,              # Required
     password => $string,              # Required
     role     => "OPERATOR" | "USER", # Required
     disable  => "true" | "false",     # Required
);
```

MODULE METHODS

The following functions can be applied to a LOM User object.

Infoblox::Grid::Member->bgp_as( )

Use this function to specify BGP Autonomous Systems for the grid member. See Infoblox::Grid::Member->bgp_as() for parameters and return values.

Example

```
#Construct a LOM User
my $lomuser = Infoblox::Grid::LOM::User->new(
     name     => 'username',
     password => 'password',
     role     => 'OPERATOR',
     disable  => 'false,
);


# Configure the LOM user on the grid object
$grid->lom_users([$lomuser]);
```

METHODS

This section describes all the methods that can be used to configure and retrieve the attribute values of an Infoblox::Grid::LOM::User object.

disable( )

Use this method to set or retrieve the disable flag for the user.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

Specify "true" to set the disable flag or "false" to deactivate/unset it.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get the disable
my $disable = $lom_user->disable();
#Modify the disable
$lom_user->disable('false');
```

name( )

Use this method to set or retrieve the name of the user.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

The name of the user in string format.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get the name
my $name = $lom_user->name();
#Modify the name
$lom_user->name('username');
```

password( )

Use this method to set the password the user will use to authenticate. This is a write-only attribute.

Include the specified parameter to set the attribute value.

Parameter

The password for the user in string format.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

Example

```
#Modify the password
$lom_user->password('password');
```

role( )

Use this method to set or retrieve the role of the user.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

Valid values are "OPERATOR" and "USER".

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get the role
my $role = $lom_user->role();
#Modify the role
$lom_user->role('OPERATOR');
```

Infoblox::Grid::LOM::User - User information for the Lights Out Management functionality.

AUTHOR

Infoblox Inc. http://www.infoblox.com/

SEE ALSO

Infoblox::Grid::Member, Infoblox::Grid::Member::BGP::Neighbor

COPYRIGHT

Copyright (c) 2017 Infoblox Inc.

Infoblox::Grid::NatGroup - NAT Group object.

- NAME
- DESCRIPTION
- CONSTRUCTOR
- MODULE METHODS
  - Infoblox::Grid->nat_group_list( )
- METHODS
  - comment( )
  - name( )
- SAMPLE CODE
- AUTHOR
- SEE ALSO
- COPYRIGHT

NAME

Infoblox::Grid::NatGroup - NAT Group object.

DESCRIPTION

NAT Groups are necessary if the Grid master is behind a NAT device and there memberies behind both side of the NAT device. Any member on the same side as the master go into the same NAT group as the master and use their interface address for Grid communication with each other. Grid members on the other side of that NAT device do not go into the same NAT group as the master and use master's NAT address for Grid communication.

CONSTRUCTOR

```
#Constructor for an NAT Group object
my $nat_group = Infoblox::Grid::NatGroup->new (
        name    => $string      #Required
        comment => $string      #Optional / Default is empty
    );
```

MODULE METHODS

The following functions are available to be applied to an NatGroup object

Infoblox::Grid->nat_group_list( )

Use this function to specify a NatGroup. See Infoblox::Grid->nat_group_list() for parameters and return values.

Example

```
# Retrieve the list of NatGroup objects from Grid object.
my @retrieved_objs = $session->get(
    object => "Infoblox::Grid",
    name   => "Infoblox" );

my $Grid_obj = $retrieved_objs[0];

# construct a NatGroup object
my $nat_group = Infoblox::Grid::NatGroup->new (
        name => "my_nat_group",
        comment => "inside NAT device");

# Modify Nat Group list.
$Grid_obj->nat_group_list([$nat_group]);
```

METHODS

This section describes all the methods that you can use to configure and retrieve the attribute value of a NatGroup.

comment( )

Use this method to set or retrieve the descriptive comment.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

Desired comment in string format with a maximum of 256 bytes.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get comment
my $comment = $nat_group->comment();
#Modify comment
$nat_group->comment("new comment");
```

name( )

Use this method to set or retrieve a NAT group name

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

Desired name in string format with a maximum of 256 bytes.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

Example

```
#Get NAT group name
my $nat_group_name = $nat_group->name();
#Modify NAT group name
$nat_group->name("new_group_name");
```

SAMPLE CODE

The following sample code demonstrates different operations that can be applied to an object such as create, modify, and remove an object. This sample code also includes error handling for the operations.

```
#PROGRAM STARTS: Include all the modules that will be used
use strict;
use Infoblox;

#Create a session to the Infoblox appliance
my $session = Infoblox::Session->new(
    master   => "192.168.1.2",
    username => "admin",
    password => "infoblox"
);
unless ($session) {
   die("Construct session failed: ",
       Infoblox::status_code() . ":" . Infoblox::status_detail());
}
print "Session created successfully\n";
```

#Add a NAT Group object

```
my @result_array = $session->get( "object" => "Infoblox::Grid", "name" => "Infoblox");
if( scalar( @result_array ) > 0 )
 {
        my $Grid_obj = $result_array[0];
        unless ($Grid_obj) {
                die("Get Grid object failed: ",
        $session->status_code() . ":" . $session->status_detail());
        }
         # construct a NatGroup object
         my $nat_group = Infoblox::Grid::NatGroup->new (
              name => "my_nat_group",
              comment => "inside NAT device");

         # Add Nat Group to Grid object.
         $Grid_obj->nat_group_list([$nat_group]);
```

```
          $session->modify($Grid_obj);
          print "NAT Group added to Grid object sucessfully\n";
           }
```

#Modify existing NAT Group object

```
 @result_array = $session->get( "object" => "Infoblox::Grid", "name" => "Infoblox");
 if( scalar( @result_array ) > 0 )
  {
         my $Grid_obj = $result_array[0];
         unless ($Grid_obj) {
                 die("Get Grid object failed: ",
         $session->status_code() . ":" . $session->status_detail());
         }

          # get Nat Group from Grid object.
          my $nat_groups_list = $Grid_obj->nat_group_list();
          my $nat_group = @{$nat_groups_list}[0];

          #modify the NAT group name
          $nat_group->name("new_nat_group_name");

          # Add Nat Group to Grid object.
          $Grid_obj->nat_group_list([$nat_group]);

          $session->modify($Grid_obj);
          print "NAT Group modified on Grid object sucessfully\n";
           }
```

#Remove existing NAT Group object

```
 @result_array = $session->get( "object" => "Infoblox::Grid", "name" => "Infoblox");
 if( scalar( @result_array ) > 0 )
  {
         my $Grid_obj = $result_array[0];
         unless ($Grid_obj) {
                 die("Get Grid object failed: ",
         $session->status_code() . ":" . $session->status_detail());
         }

          # empty Nat Group list on Grid object.
          $Grid_obj->nat_group_list([]);

          $session->modify($Grid_obj);
          print "NAT Group removed on Grid object sucessfully\n";
           }

 ####PROGRAM ENDS####
```

AUTHOR

Infoblox Inc. http://www.infoblox.com/

SEE ALSO

Infoblox::Session, Infoblox::Session->get(), Infoblox::Session->modify()

COPYRIGHT

Copyright (c) 2017 Infoblox Inc.

Infoblox::Grid::NTPServer - Grid Network Time Protocol server object.

- NAME
- DESCRIPTION
- CONSTRUCTOR
- MODULE METHODS
  - Infoblox::Grid->ntp_server( )
- METHODS
  - address( )
  - authentication( )
  - burst( )
  - iburst( )
  - ntp_key( )
- SAMPLE CODE
- AUTHOR
- SEE ALSO
- COPYRIGHT

NAME

Infoblox::Grid::NTPServer - Grid Network Time Protocol (NTP) server object.

DESCRIPTION

The Grid NTP server object is used to synchronize the date and time for an Infoblox appliance.

CONSTRUCTOR

```
#Constructor for an Grid NTP server object
 my $ntp_server = Infoblox::Grid::NTPServer->new(
    address        =>  $ipv4addr | $ipv6addr | $fqdn    #Required
    authentication => "true" | "false",                 #Optional / Default "false"
    burst          => "true" | "false",                 #Optional / Default "true"
    iburst         => "true" | "false",                 #Optional / Default "true"
    ntp_key        => [$NTPKey1, $NTPKey2,...]          #Optional / Default is undefined
   );
```

This constructed object can be used only when NTP is set to "true" at the grid level.

MODULE METHODS

This section describes all the functions that can be applied to a NTP server object.

Infoblox::Grid->ntp_server( )

Use this function to specify the NTP server for the grid.

Example

```
#Create a NTPKey object.
my $ntp_key1 = Infoblox::Grid::NTPKey->new(
   key_number => 1111,
   key_type   => "M",
   key_string => "aaaa"
);

#Create a NTPServer object.
 my $ntp_server = Infoblox::Grid::NTPServer->new(
    address        => "3.3.3.3",
    authentication => "true",
    ntp_key        => $ntp_key1
);

my $grid = $session->get(
   object  => "Infoblox::Grid",
   name    => "Infoblox"
 );

 unless ($grid) {
     die("get Grid failed:" ,
```

```
            $session->status_code(), $session->status_detail());
 }

$grid->enable_ntp("true");
$grid->ntp_authentication_key([$ntp_key1]);
$grid->ntp_server([$ntp_server]);

#Applying the changes to appliance through session.
$session->modify($grid)
      or die("modify Grid failed:" ,
       $session->status_code(), $session->status_detail());
```

METHODS

This section describes all the methods that can be used to configure and retrieve the attribute value of a NTP server object.

address( )

Use this method to set or retrieve the IPv4 or IPv6 address or FQDN of the external NTP server.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

Valid values are IPv4 or IPv6 address or FQDN of the external NTP server.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get address
my $address = $ntp_server->address();
#Modify address
$ntp_server->address("4.4.4.4");
```

authentication( )

Use this method to enable or disable the authentication for an NTP server.

Parameter

Specify "true" to enable authentication or "false" to disable it. Default value is "false".

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get authentication
my $authentication = $ntp_server->authentication();
#Modify authentication
$ntp_server->authentication("false");
```

burst( )

Use this method to enable or disable BURST operating mode. In BURST operating mode, when the external server is reachable and a valid source of synchronization is available, NTP sends a burst of 8 packets with a 2 second interval between packets.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

Specify "true" to eanble the BURST operating mode or "false" to disable it. The default value is "true".

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get burst
my $burst = $ntp_server->burst();
#Modify burst
$ntp_server->burst("false");
```

iburst( )

Use this method to enable or disable IBURST operating mode. In IBURST operating mode, when the external server is unreachable, NTP server sends a burst of 8 packets with a 2 second interval between packets.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

Specify "true" to eanble the IBURST operating mode or "false" to disable it. The default value is "true".

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get iburst
my $iburst = $ntp_server->iburst();
#Modify iburst
$ntp_server->iburst("false");
```

ntp_key( )

Use this method to set or retrieve the ntp_key used for authentication.

Parameter

Valid value is an array reference that contains Infoblox::Grid::NTPKey objects.

See Infoblox::Grid::NTPKey for parameters and return values.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Create a NTPKey object.
my $ntp_key2 = Infoblox::Grid::NTPKey->new(
  key_number => 2222,
  key_type   => "n",
  key_string =>"d3e54352e5548080" );

#Modify NTPServer object
my $ntp_server = Infoblox::Grid::NTPServer->new(
   address        => "3.3.3.3",
   authentication => "true",
   ntp_key        => $ntp_key2
);
```

SAMPLE CODE

The following sample code demonstrates different operations that can be applied to an object such as create, modify, and remove an object. Also this sample code also includes error handling for the operations.

```
#PROGRAM STARTS: Include all the modules that will be used
 use strict;
 use Infoblox;

 #Create a session to the Infoblox appliance
 my $session = Infoblox::Session->new(
    master   => "192.168.1.2",
    username => "admin",
    password => "infoblox"
 );
```

```
  unless ($session) {
    die("Construct session failed: ",
        Infoblox::status_code() . ":" . Infoblox::status_detail());
  }
  print "Session created successfully\n";
```

#Adding an NTP server object #Get the grid object. my $result = $session->get(object => "Infoblox::Grid", name => "Infoblox"); unless ($result) { die("get Grid failed:" , $session->status_code(), $session->status_detail()); }

```
  #Create a NTPKey object.
  my $ntp_key1 = Infoblox::Grid::NTPKey->new(
      key_number => 3333,
      key_type   => "S",
      key_string => "a7cb86a4cba80101"
  );

  #Create a NTPServer object.
  my $ntp_server = Infoblox::Grid::NTPServer->new(
      address        => "3.3.3.3",
      authentication => "true",
      ntp_key        => $ntp_key1
  );
   #Modifying the value of the ntp_authentication_key and ntp_server methods, from the grid object.
  $result->enable_ntp("true");
  $result->ntp_authentication_key([$ntp_key1]);
  $result->ntp_server([$ntp_server]);

  #Applying the changes to appliance through session.
  $session->modify($result)
       or die("modify Grid failed:" ,
        $session->status_code(), $session->status_detail());
   print "\n  Modify Grid successful \n";
```

#Modifying an NTP server object #Create a NTPServer object. my $ntp_server = Infoblox::Grid::NTPServer->new(address => "3.3.3.3", authentication => "false",);

```
 $result->ntp_server([$ntp_server]);

 #Applying the changes to appliance through session.
 $session->modify($result)
      or die("modify Grid failed:" ,
       $session->status_code(), $session->status_detail());
 print "\n  Modify grid successful for NTP server object\n";
```

#Removing an NTP server object #Get the grid object. my $result = $session->get(object => "Infoblox::Grid", name => "Infoblox"); unless ($result) { die("get Grid failed:" , $session->status_code(), $session->status_detail()); }

```
 $result->enable_ntp("false");
 $result->ntp_server([]);
 #Applying the changes to appliance through session.
 $session->modify($result)
      or die("modify Grid failed:" ,
       $session->status_code(), $session->status_detail());
 print "\n  Modify grid successful for NTP server object\n";

 ####PROGRAM ENDS####
```

AUTHOR

Infoblox Inc. http://www.infoblox.com/

SEE ALSO

Infoblox::Session, Infoblox::Grid::NTPKey, Infoblox::Grid,Infoblox::Session->get(), Infoblox::Session->modify()

COPYRIGHT

Copyright (c) 2017 Infoblox Inc.

Infoblox::Grid::NTPAccess - Grid Network Time Protocol access object.

- NAME
- DESCRIPTION
- CONSTRUCTOR
- MODULE METHODS
  - Infoblox::Grid->ntp_access_list( )
  - Infoblox::Grid::Member->ntp_access_list( )
- METHODS
  - address( )
  - service( )
- AUTHOR
- SEE ALSO
- COPYRIGHT

NAME

Infoblox::Grid::NTPAccess - Grid Network Time Protocol (NTP) access object.

DESCRIPTION

The Grid NTP access object is used to specify the access list for the NTP service.

CONSTRUCTOR

```
#Constructor for an Grid NTP access object
 my $ntp_access = Infoblox::Grid::NTPAccess->new(
    address         =>  $ipv4addr | "Any"   #Required
    service         => "Time" | "Time and NTP control"          #Optional / Default "Time"
   );
```

This constructed object can be used only when NTP is set to "true" at the grid level.

MODULE METHODS

This section describes all the functions that can be applied to an NTP access object.

Infoblox::Grid->ntp_access_list( )

Infoblox::Grid::Member->ntp_access_list( )

Use this function to specify the NTP access for the grid/member.

Example

```
# Create NTP access objects
my $ntpac1 = Infoblox::Grid::NTPAccess->new(
                                            address => "2.2.2.2",
                                            service => "Time",
                                           );

my $ntpac2 = Infoblox::Grid::NTPAccess->new(
                                            address => "4.4.4.4",
                                            service => "Time and NTP control",
                                           );


$grid->ntp_access_list([$ntpac1,$ntpac2]);

my $grid = $session->get(
   object  => "Infoblox::Grid",
   name    => "Infoblox"
 );

 unless ($grid) {
     die("get Grid failed:" ,
          $session->status_code(), $session->status_detail());
 }

$grid->ntp_access_list([$ntpac1,$ntpac2]);
```

```
#Applying the changes to appliance through session.
$session->modify($grid)
     or die("modify Grid failed:" ,
      $session->status_code(), $session->status_detail());
```

METHODS

This section describes all the methods that can be used to configure and retrieve the attribute value of an NTP access object.

address( )

Use this method to set or retrieve the IPv4 address of the external NTP client. Use "Any" to apply this permission to all clients.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

Valid values are an IPv4 address of the external NTP client, or "Any".

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get address
my $address = $ntp_access->address();
#Modify address
$ntp_access->address("4.4.4.4");
```

service( )

Use this method to specify to which service this NTP Access object applies.

Parameter

Valid values are "Time" or "Time and NTP control".

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get service
my $service = $ntp_access->service();
#Modify service
$ntp_access->service("Time and NTP control");
```

AUTHOR

Infoblox Inc. http://www.infoblox.com/

SEE ALSO

Infoblox::Session, Infoblox::Grid::Member, Infoblox::Grid,Infoblox::Session->get(), Infoblox::Session->modify()

COPYRIGHT

Copyright (c) 2017 Infoblox Inc.

Infoblox::Grid::NTPKey - Grid Network Time Protocol key object.

- NAME
- DESCRIPTION
- CONSTRUCTOR
- MODULE METHODS
  - Infoblox::Grid->ntp_authentication_key( )
  - Infoblox::Grid::Member->ntp_authentication_key( )
- METHODS
  - key_number( )
  - key_string( )
  - key_type( )
- SAMPLE CODE
- AUTHOR
- SEE ALSO
- COPYRIGHT

NAME

Infoblox::Grid::NTPKey - Grid Network Time Protocol (NTP) key object.

DESCRIPTION

Grid NTPKey object is used by NTP server to authenticate the clients.

CONSTRUCTOR

```
#Constructor for an Grid NTPKey object
 my $ntp_key = Infoblox::Grid::NTPKey->new(
    key_number    =>  $num,                  #Required
    key_string    =>  $string,               #Required
    key_type      =>  "M" | "S" | "A" | "N"  #Optional / Default is "M"
  );
```

MODULE METHODS

The following functions are available to be applied to an NTPKey object.

Infoblox::Grid->ntp_authentication_key( )

Use this function to specify a NTP key at the grid level on the Infoblox appliance. See Infoblox::Grid->ntp_authentication_key() for parameters and return values.

Example

```
#Create a NTPKey object.
my $ntp_key = Infoblox::Grid::NTPKey->new(
                  key_number => 1111,
                  key_type   => "M",
                  key_string => "aaaa"
                );

#Configure NTPKey on the Infoblox grid object
my $responce = $grid->ntp_authentication_key()([$ntp_key]);
```

Infoblox::Grid::Member->ntp_authentication_key( )

Use this function to specify a NTP key at the member level on the Infoblox appliance. See Infoblox::Grid::Member->ntp_authentication_key() for parameters and return values.

Example

```
 #Create a NTPKey object.
  my $ntp_key = Infoblox::Grid::NTPKey->new(
                  key_number => 1111,
                  key_type   => "M",
                  key_string => "aaaa"
                );

#Configure NTPKey on the Infoblox member object
```

```
my $responce = $member->ntp_authentication_key()([$ntp_key]);
```

METHODS

This section describes all the methods that you can used to configure and retrieve the attribute value of a NTPKey object.

key_number( )

Use this method to set or retrieve the key number of the NTPKey object.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

A positive integer number. Value must be between 1 and 65534.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get key_number
my $number = $ntp_key->key_number();
#Modify key_number
$ntp_key->key_number("3333");
```

key_string( )

Use this method to set or retrieve the key string of the NTPKey object.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

Desired key_string in string format defined in the key_type.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get key_string
my $string = $ntp_key->key_string();
#Modify key_string
$ntp_key->key_string("d3e54352e5548080");
```

key_type( )

Use this method to set or retrieve the key type of the NTPKey object.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

Valid values are :"M","S","N","A".The default value is "M".

```
 "M" - For  Message Digest (MD5) in ASCII format and the key_string must be a 1-31 character ASCII
using
          MD5;
 "S" - For Data Encryption Standard (DES) in hex format and key_string must be a 64-bit hexadecimal
         number in DES. The high order 7 bits of each octet form the 56-bit key, and the low order
bit of each
        octet is given a value,  so that the octet maintains odd parity. You must specify leading
zeros so the key
        is exactly 16 hexadecimal digits long and maintains odd parity;

 "A" - For Data Encryption Standard (DES) in ASCII format and the key_string must be a 1-8 character
          ASCII using DES;
 "N" - For Data Encryption Standard (DES) in NTP format and the key_string must be a 64-bit
hexadecimal in
          NTP format. It is the same as the "S" format, but the bits in each octet have been rotated
```

```
one bit right,
          so the parity bit is in the high order bit of the octet. You must specify leading zeros and
odd parity must
          be maintained.
```

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get key_type
my $type = $ntp_key->key_type();
#Modify key_type
$ntp_key->ntp_key("N");
```

SAMPLE CODE

The following sample code demonstrates different operations that can be applied to an object such as create, modify, and remove an object. Also this sample code also includes error handling for the operations.

```
#PROGRAM STARTS: Include all the modules that will be used
 use strict;
 use Infoblox;

 #Create a session to the Infoblox appliance
 my $session = Infoblox::Session->new(
    master   => "192.168.1.2",
    username => "admin",
    password => "infoblox"
 );
 unless ($session) {
   die("Construct session failed: ",
       Infoblox::status_code() . ":" . Infoblox::status_detail());
 }
 print "Session created successfully\n";
```

#Adding an NTPKey object

```
#Create a NTPKey object.
my $ntp_key = Infoblox::Grid::NTPKey->new(
   key_number => 65533,
   key_type   => "M",
   key_string => "aaaa"
);

unless($ntp_key) {
       die("Construct NTP Key object failed: ",
            Infoblox::status_code() . ":" . Infoblox::status_detail());
}
print "NTP Key object created successfully\n";

#Create a NTPServer object.
 my $ntp_server = Infoblox::Grid::NTPServer->new(
    address        => "3.3.3.3",
);

unless($ntp_server) {
       die("Construct NTP server object failed: ",
            Infoblox::status_code() . ":" . Infoblox::status_detail());
}
print "NTP Server object created successfully\n";

my @grid_objs = $session->get(
   object  => "Infoblox::Grid",
   name    => "Infoblox"
 );

 my $grid = $grid_objs[0];
 unless ($grid) {
       die("Get Grid object failed: ",
            $session->status_code() . ":" . $session->status_detail());
  }
  print "Get Grid object found at least 1 matching entry\n";
```

```
$grid->enable_ntp("true");
$grid->ntp_authentication_key([$ntp_key]);
$grid->ntp_server([$ntp_server]);

#Applying the changes to appliance through session.
$session->modify($grid)
     or die("modify Grid failed:" ,
      $session->status_code(), $session->status_detail());
print "Grid modified successfully for adding NTPKey object\n";
```

#Modifying an NTPKey object

```
my $ntp_key = Infoblox::Grid::NTPKey->new(
   key_number => 65532,
   key_type   => "M",
   key_string => "aaaa"
);

my @grid_objs = $session->get(
           object => "Infoblox::Grid",
           name   => "Infoblox"
       );

my $grid = $grid_objs[0];
unless ($grid) {
       die("Get Grid object failed: ",
            $session->status_code() . ":" . $session->status_detail());
}
print "Get Grid object found at least 1 matching entry\n";

$grid->ntp_authentication_key([$ntp_key]);

#Applying the changes to appliance through session.
$session->modify($grid)
     or die("modify Grid failed:" ,
      $session->status_code(), $session->status_detail());
print "Grid modified successfully for modifying NTPKey object\n";
```

#Removing an NTPKey object

```
#Get Infoblox grid object
my @grid_objs = $session->get(
           object => "Infoblox::Grid",
           name   => "Infoblox"
       );

my $grid = $grid_objs[0];
unless ($grid) {
       die("Get Grid object failed: ",
            $session->status_code() . ":" . $session->status_detail());
}
print "Get Grid object found at least 1 matching entry\n";

$grid->ntp_authentication_key([]);
#Applying the changes to appliance through session.
$session->modify($grid)
    or die("modify Grid failed for removing NTPKey object:" ,
     $session->status_code(), $session->status_detail());
print "Grid modified successfully for removing NTPKey object\n";
####PROGRAM ENDS####
```

AUTHOR

Infoblox Inc. http://www.infoblox.com/

SEE ALSO

Infoblox::Session, Infoblox::Grid::NTPServer, Infoblox::Grid,Infoblox::Session->get(), Infoblox::Session->modify()

COPYRIGHT

Copyright (c) 2017 Infoblox Inc.

Infoblox::Grid::ObjectsChangesTrackingSetting - the object changes tracking setting.

- NAME
- DESCRIPTION
- CONSTRUCTOR
- MODULE METHODS
  - Infoblox::Grid->objects_changes_tracking_setting( )
- METHODS
  - enable( )
  - enable_completion( )
  - max_objs_to_track( )
  - max_time_to_track( )
  - state( )
- AUTHOR
- SEE ALSO
- COPYRIGHT

NAME

Infoblox::Grid::ObjectsChangesTrackingSetting - the object changes tracking setting.

DESCRIPTION

The object changes tracking setting object is used to display the object changes state and to enable or disable the object changes tracking feature.

CONSTRUCTOR

```
my $object_changes = Infoblox::Grid::ObjectsChangesTrackingSetting->new(
    enable => 'true' | 'false', #Optional / Default is 'false'
);
```

MODULE METHODS

+The following functions can be applied to an object changes tracking setting object.

Infoblox::Grid->objects_changes_tracking_setting( )

Use this function to specify the object changes tracking settings for the Grid. See Infoblox::Grid/objects_changes_tracking_setting() for parameters and return values.

Example

```
#Get objects_changes_tracking_setting value
my $value = $object->objects_changes_tracking_setting();

#Construct objects_changes_tracking_setting value
my $objects_changes_tracking_setting = Infoblox::Grid::ObjectsChangesTrackingSetting->new(
    enable => 'true',
);

#Modify objects_changes_tracking_setting value
$object->objects_changes_tracking_setting($objects_changes_tracking_setting);
```

METHODS

This section describes all the methods that you can use to set and retrieve the attribute values of an object changes tracking setting object.

enable( )

Use this method to set or retrieve the flag that indicates whether the object changes tracking feature is enabled or disabled.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

Specify 'true' to enable the object changes tracking feature or 'false' to disable it.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get enable value
my $value = $object->enable();

#Modify enable value
$object->enable('true');
```

enable_completion( )

Use this method to retrieve the object changes tracking completion percentage. This is a read-only attribute.

Omit the parameter to retrieve the attribute value.

Parameter

None

Returns

The valid return value is an unsigned integer between 0 and 100.

Example

```
#Get enable_completion value
my $value = $object->enable_completion();
```

max_objs_to_track( )

Use this method to set or retrieve the maximum number of objects retained for change tracking.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

The valid value is an unsigned integer.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get max_objs_to_track value
my $value = $object->max_objs_to_track();

#Modify max_objs_to_track value
$object->max_objs_to_track(10);
```

max_time_to_track( )

Use this method to set or retrieve the maximum time period (in seconds) to track the deleted object changes.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

The valid value is an unsigned integer that represents time period (in seconds).

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get max_time_to_track value
my $value = $object->max_time_to_track();

#Modify max_time_to_track value
```

```
$object->max_time_to_track(10);
```

state( )

Use this method to retrieve the object changes tracking state. This is a read-only attribute.

Omit the parameter to retrieve the attribute value.

Parameter

None

Returns

The valid values are 'ENABLING', 'ENABLED', 'DISABLED' and 'ENABLING_ERROR'.

Example

```
#Get state value
my $value = $object->state();
```

AUTHOR

Infoblox Inc. http://www.infoblox.com/

SEE ALSO

Infoblox::Grid, Infoblox::Grid/objects_changes_tracking_setting()

COPYRIGHT

Copyright (c) 2017 Infoblox Inc.

Infoblox::Grid::RestartBannerSetting - The restart banner setting.

- NAME
- DESCRIPTION
- CONSTRUCTOR
- MODULE METHODS
  - Infoblox::Grid->restart_banner_setting( )
- METHODS
  - enable( )
  - enable_double_confirmation( )
- AUTHOR
- SEE ALSO
- COPYRIGHT

NAME

Infoblox::Grid::RestartBannerSetting - The restart banner setting.

DESCRIPTION

The Restart Banner Setting object controls the banner displayed for Grid restart.

CONSTRUCTOR

```
my $restart = Infoblox::Grid::RestartBannerSetting->new(
    enable                    => "true" | "false", #Mandatory
    enable_double_confirmation => "true" | "false", #Optional / Default is "false"
);
```

MODULE METHODS

The following functions are available to apply to a restart banner setting object.

Infoblox::Grid->restart_banner_setting( )

Use this function to specify the restart banner settings for the Grid. See Infoblox::Grid->restart_banner_setting() for parameters and return values.

Example

```
my $grid = $session->get(object => 'Infoblox::Grid');

my $banner = Infoblox::Grid::RestartBannerSetting->new(enable => 'true');
$grid->restart_banner_setting($banner);
my $response = $session->modify($grid);
unless($response) {
     die("Modify grid failed: ",
           session->status_code() . ":" . session->status_detail());
}
print "Grid banner setting modified successfully\n";
```

METHODS

This section describes all the methods that you can use to set and retrieve the attribute values of a Restart Banner Setting object.

enable( )

Use this method to set the enable flag.

Parameter

Specify "true" to enable the banner or "false" to disable it.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get enable
my $enable = $banner->enable();
#Modify enable
$banner->enable("true");
```

enable_double_confirmation( )

Use this method to set the enable_double_confirmation flag. If this flag is set to "true" the user is required to input name before restarting services.

Parameter

Specify "true" to enable double confirmation or "false" to disable it.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get enable_double_confirmation
my $enable_double_confirmation = $banner->enable_double_confirmation();
#Modify enable_double_confirmation
$banner->enable_double_confirmation("true");
```

AUTHOR

Infoblox Inc. http://www.infoblox.com/

SEE ALSO

Infoblox::Grid

COPYRIGHT

Copyright (c) 2017 Infoblox Inc.

Infoblox::Grid::ServiceRestart::Status - A Grid Service Restart Status object.

- NAME
- DESCRIPTION
- SESSION METHODS
  - Infoblox::Session->get( )
  - Infoblox::Session->search( )
- MODULE METHODS
  - Infoblox::Grid::ServiceRestart::Group->status( )
  - Infoblox::Grid->restart_status( )
- METHODS
  - failures( )
  - finished( )
  - parent( )
  - grouped( )
  - needed_restart( )
  - no_restart( )
  - pending( )
  - pending_restart( )
  - processing( )
  - restarting( )
  - success( )
  - timeouts( )
- AUTHOR
- SEE ALSO
- COPYRIGHT

NAME

Infoblox::Grid::ServiceRestart::Status - A Grid Service Restart Status object.

DESCRIPTION

The Grid Service Restart Status object provides information and statistics about restart service routine of the Grid or Restart Service Group.

SESSION METHODS

This section describes all the methods in a Session module that you can apply to a Grid Service Restart Status object.

Infoblox::Session->get( )

Use this method to retrieve all the matching objects from the Infoblox appliance. See Infoblox::Session->get() for parameters and return values.

Key Reference

```
Apply the following attribute to get a specific object:

parent  - Optional. A Grid Service Restart Group or Grid object.
grouped - Optional. A grouping type of the status which can be either 'GRID' or 'GROUP'.
```

Example

```
my @retrieved_objs = $session->get(
    object  => 'Infoblox::Grid::ServiceRestart::Status',
    parent  => $test_group,
    grouped => 'GROUP',

);
```

Infoblox::Session->search( )

Use this method to search for objects in the Infoblox appliance. See Infoblox::Session->search() for parameters and return values.

Key Reference

```
Apply the following attribute to search for a specific object:

parent  - Optional. A Grid Service Restart Group or Grid object.
grouped - Optional. A grouping type of the status which can be either 'GRID' or 'GROUP'.
```

Example

```
my @retrieved_objs = $session->get(
    object  => 'Infoblox::Grid::ServiceRestart::Status',
    parent  => $test_group,
    grouped => 'GROUP',

);
```

MODULE METHODS

The following functions are available to apply to a Grid Service Restart Status object.

Infoblox::Grid::ServiceRestart::Group->status( )

Use this method to retrieve the Grid Service Restart restart status. This is a read-only attribute. See Infoblox::Grid::ServiceRestart::Group->status() for parameters and return values.

Example

```
#Get status
my $status = $svcrst_group->status();
```

Infoblox::Grid->restart_status( )

Use this method to retrieve the Grid restart status. This is a read-only attribute. See Infoblox::Grid->restart_status() for parameters and return values.

Example

```
#Get status
my $status = $grid->restart_status();
```

METHODS

This section describes all the methods that you can use to set or retrieve the attribute values of the object.

failures( )

Use this method to retrieve the number of failed service restart requests. This is a read-only attribute.

Omit the parameter to retrieve the attribute value.

Parameter

None

Returns

The method returns attribute value.

Example

```
#Get failures
my $failures = $svcrst_status->failures();
```

finished( )

Use this method to retrieve the number of completed service restart requests. This is a read-only attribute.

Omit the parameter to retrieve the attribute value.

Parameter

None

Returns

The method returns attribute value.

Example

```
#Get finished
my $finished = $svcrst_status->finished();
```

parent( )

Use this method to retrieve the Grid or Grid Service Restart Group objects. This is a read-only attribute.

Omit the parameter to retrieve the attribute value.

Parameter

None

Returns

The valid return value can be either Infoblox::Grid or Infoblox::Grid::ServiceRestart::Group object.

Example

```
#Get group
my $group = $svcrst_status->group();
```

grouped( )

Use this method to retrieve the grouping type of the status which can be either 'GRID' or 'GROUP'. This is a read-only attribute.

Omit the parameter to retrieve the attribute value.

Parameter

None

Returns

The method returns attribute value.

Example

```
#Get failures
my $failures = $svcrst_status->failures();
```

needed_restart( )

Use this method to retrieve the number of created but unprocessed restart requests. This is a read-only attribute.

Omit the parameter to retrieve the attribute value.

Parameter

None

Returns

The method returns attribute value.

Example

```
#Get no_restart
my $needed_restart = $svcrst_status->needed_restart();
```

no_restart( )

Use this method to retrieve the number of service restarts that are not required. This is a read-only attribute.

Omit the parameter to retrieve the attribute value.

Parameter

None

Returns

The method returns attribute value.

Example

```
#Get no_restart
my $no_restart = $svcrst_status->no_restart();
```

pending( )

Use this method to retrieve the number of pending service restart requests. This is a read-only attribute.

Omit the parameter to retrieve the attribute value.

Parameter

None

Returns

The method returns attribute value.

Example

```
#Get pending
my $pending = $svcrst_status->pending();
```

pending_restart( )

Use this method to retrieve the number of required force restart requests that are still pending. This is a read-only attribute.

Omit the parameter to retrieve the attribute value.

Parameter

None

Returns

The method returns attribute value.

Example

```
#Get pending_restart
my $pending_restart = $svcrst_status->pending_restart();
```

processing( )

Use this method to retrieve the number of pending restart requests that are not required and not forced. This is a read-only attribute.

Omit the parameter to retrieve the attribute value.

Parameter

None

Returns

The method returns attribute value.

Example

```
#Get processing
my $processing = $svcrst_status->processing();
```

restarting( )

Use this method to retrieve the number of restarting services. This is a read-only attribute.

Omit the parameter to retrieve the attribute value.

Parameter

None

Returns

The method returns attribute value.

Example

```
#Get restarting
my $restarting = $svcrst_status->restarting();
```

success( )

Use this method to retrieve the number of successful service restart requests. This is a read-only attribute.

Omit the parameter to retrieve the attribute value.

Parameter

None

Returns

The method returns attribute value.

Example

```
#Get success
my $success = $svcrst_status->success();
```

timeouts( )

Use this method to retrieve the number of timeouts for service restart requests. This is a read-only attribute.

Omit the parameter to retrieve the attribute value.

Parameter

None

Returns

The method returns attribute value.

Example

```
#Get timeouts
my $timeouts = $svcrst_status->timeouts();
```

AUTHOR

Infoblox Inc. http://www.infoblox.com/

SEE ALSO

Infoblox::Session, Infoblox::Session->get(), Infoblox::Session->search(), Infoblox::Grid::ServiceRestart::Group, Infoblox::Grid::ServiceRestart::Group->status(), Infoblox::Grid, Infoblox::Grid->restart_status().

COPYRIGHT

Copyright (c) 2017 Infoblox Inc.

Infoblox::Grid::ScheduledBackup - Scheduled Backup object.

- NAME
- DESCRIPTION
- CONSTRUCTOR
- MODULE METHODS
  - Infoblox::Grid->scheduled_backup( )
- METHODS
  - backup_type( )
  - keep_local_copy( )
  - backup_frequency( )
  - weekday( )
  - execute( )
  - hour_of_day( )
  - minute_of_hour( )
  - nios_data( )
  - discovery_data( )
  - splunk_app_data( )
  - user( )
  - password( )
  - backup_server( )
  - path( )
  - disabled( )
  - source( )
  - status( )
  - operation( )
  - restore_type( )
  - restore_host( )
  - restore_username( )
  - restore_password( )
  - restore_directory( )
- SAMPLE CODE
- AUTHOR
- SEE ALSO
- COPYRIGHT

NAME

Infoblox::Grid::ScheduledBackup - Scheduled Backup object.

DESCRIPTION

The Scheduled Backup object is used to schedule system backups.

CONSTRUCTOR

```
 #Constructor for a ScheduledBackup object
 my $backup = Infoblox::Grid::ScheduledBackup->new (
         backup_type => "LOCAL" | "TFTP" | "FTP" | "SCP",    #Optional / Default is "LOCAL"
         keep_local_copy  => "true" | "false",               #Optional / Default is "false"
         backup_frequency => "Weekly" | "Daily" | "Hourly", #Optional / Default is "Daily"
         weekday => "0" | "1" | "2" | "3" | "4" | "5" | "6" | "7" | "Sunday" | "Monday" | "Tuesday" |
"Wednesday" | "Thursday" | "Friday" | "Saturday"  #Optional / Default is "0"
         hour_of_day      => $num,                           #Optional / Default is 3
         minute_of_hour   => $num,                           #Optional / Default is 0
         user             => $string,                        #Optional / Default is undefined
         password         => $string,                        #Optional / Default is undefined
         backup_server    => $string,                        #Optional / Default is undefined
         path             => $string,                        #Optional / Default is undefined
         disabled         => "true" | "false",               #Optional / Default is "false"

         source             => "DB" | "REPORTING"            #Optional / Default is "DB"
         status             => "IDLE" | "TRIGGERED" | "IN_PROGRESS" | "FINISHED" | "FAILED", #Optional /
Default is "IDLE"
         operation_state    => 'TRIGGERED',                  #Optional / Default is undefined
         operation          => "NONE" | "BACKUP" | "RESTORE", #Optional / Default is "NONE"
         restore_type       => "FTP" | "SCP",                #Optional / Default is "FTP"
         restore_host       => $string,                      #Optional / Default is undefined
         restore_username   => $string,                      #Optional / Default is undefined
         restore_password   => $string,                      #Optional / Default is undefined
         restore_directory  => $string,                      #Optional / Default is undefined
         nios_data          => 'true' | 'false',             #Optional / If any of nios_data and
discovery_data is not
                                                             #defined, nios_data is 'true'.
```

```
        discovery_data    => 'true' | 'false',              #Optional / If any of nios_data and
discovery_data is not
                                                            #defined, discovery_data is 'false'.
        splunk_app_data   => 'true' | 'false',              #Optional / Default is 'false'
         );
```

MODULE METHODS

The following functions can be applied to a ScheduledBackup object.

Infoblox::Grid->scheduled_backup( )

Use this function to specify a ScheduledBackup object. See Infoblox::Grid->scheduled_backup() for parameters and return values.

Example

```
# Retrieve the list of ScheduledBackup objects from the Grid object.
my @retrieved_objs = $session->get(
    object => "Infoblox::Grid",
    name   => "Infoblox"
);

my $Grid_obj = $retrieved_objs[0];

# construct a ScheduledBackup object
my $backup = Infoblox::Grid::ScheduledBackup->new (
    backup_type => "LOCAL",
    backup_frequency => "Weekly",
    weekday => "0", # "0" is the same as "Sunday"
    hour_of_day => "4",
    minute_of_hour => "0",
    user  => "username",
    password => "user-password",
    backup_server  => "my-backup-server",
    path => "/tmp/infoblox/database.tar.gz",
    disabled => "false",
);

# Modify the scheduled backup list
$Grid_obj->scheduled_backup($backup);
```

METHODS

This section describes all the methods that you can use to configure and retrieve the attribute values of a ScheduledBackup object.

backup_type( )

Use this method to set or retrieve the destination of the backup files. You can back up system files to a local directory, a TFTP server, an FTP server, or an SSH server that supports SCP.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

Valid values are "LOCAL", "TFTP", "FTP" or "SCP". The default value is "LOCAL".

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get backup_type
my $backup_type = $backup->backup_type();
#Modify backup_type
$backup->backup_type("FTP");
```

keep_local_copy( )

Use this method to control whether a local backup is created in addition to a remote one. This option works only if you also specify a remote backup type (TFTP, FTP, or SCP). You cannot enable this option using the "LOCAL" backup type.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

Valid values are "true" and "false". The default value is "false".

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get keep_local_copy
my $keep_local_copy = $backup->keep_local_copy();
#Modify keep_local_copy
$backup->backup_type("true");
```

backup_frequency( )

Use this method to set or retrieve the frequency of backups.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

Valid values are "Weekly", "Daily" or "Hourly". The default value is "Daily".

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

Example

```
#Get backup frequency
my $backup_backup_frequency = $backup->backup_frequency();
#Modify backup frequency
$backup->backup_frequency("Weekly");
```

weekday( )

Use this method to set or retrieve the day of the week when the backup is performed. This is required only when backup_frequency is set to "Weekly".

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

Valid values are "0", "1", "2", "3", "4", "5", "6", "7", "Sunday", "Monday", "Tuesday", "Wednesday", "Thursday", "Friday" or "Saturday". "0" means "Sunday", "1" - "Monday". etc. The default value is "6" (Saturday).

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get weekday
my $weekday = $backup->weekday();
#Modify weekday
$backup->weekday("0");
```

execute( )

Use this method to set the state for Reporting scheduled backup / restore operation. This is a write-only attribute.

Parameter

The valid value is 'TRIGGER'. The default is undefined.

Returns

None

Example

```
#Modify operation_state value
$object->execute('TRIGGER');
```

hour_of_day( )

Use this method to set or retrieve the hour when the backup is performed.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

A number from 0 through 23. The default value is 3 (3:00 A.M.).

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get hour_of_day
my $hour_of_day = $backup->hour_of_day();
#Modify hour_of_day
$backup->hour_of_day("4");
```

minute_of_hour( )

Use this method to set or retrieve the minute of the hour when the backup is performed.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

A number from 0 through 59. The default value is 0.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get minute_of_hour
my $minute_of_hour = $backup->minute_of_hour();
#Modify minute_of_hour
$backup->minute_of_hour("5");
```

nios_data( )

Use this method to enable or disable the backup and restore of NIOS data.

If any of nios_data and discovery_data is not defined, nios_data is 'true'.

Parameter

Set the parameter to "true" to enable backup/restore of NIOS data. Set the parameter to "false" to disable the object. The default value of this parameter is "false".

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get nios_data
my $nios_data = $backup->nios_data();
#Modify nios_data
$backup->nios_data("true");
```

discovery_data( )

Use this method to enable or disable the backup and restore of Network Automation data.

If any of nios_data and discovery_data is not defined, discovery_data is 'false'.

Parameter

Set the parameter to "true" to enable backup/restore of Network Automation data. Set the parameter to "false" to disable the object. The default value of this parameter is "false".

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get discovery_data
my $nios_data = $backup->discovery_data();
#Modify discovery_data
$backup->discovery_data("true");
```

splunk_app_data( )

Use this method to enable or disable the backup and restore of Splunk application data.

Parameter

Set the parameter to "true" to enable backup/restore of Splunk application data. Set the parameter to "false" to disable the object. The default value of this parameter is "false".

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get splunk_app_data
my $nios_data = $backup->splunk_app_data();
#Modify splunk_app_data
$backup->splunk_app_data("true");
```

user( )

Use this method to set or retrieve the user name on the backup server. This is required only when backup_type is set to "FTP" or "SCP".

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

The user name in string format with a maximum of 1024 bytes.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get user
my $user = $backup->user();
#Modify user
$backup->user("admin1");
```

password( )

Use this method to set the user password on the backup server. This is required only when backup_type is set to "FTP" or "SCP". This is a write-only attribute.

Parameter

The password in string format with a maximum of 64 bytes.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

Example

```
#Modify password
$backup->password("userpassword");
```

backup_server( )

Use this method to set or retrieve the IP address of the backup server. This is required only when backup_type is set to "FTP", "TFTP" or "SCP".

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

The IP address of the backup server.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get backup_server
my $backup_server = $backup->backup_server();
#Modify backup_server
$backup->backup_server("my-backup-server");
```

path( )

Use this method to set or retrieve the directory path to the backup file stored on the server. This is required only when backup_type is set to "FTP" or "SCP".

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

Desired path in string format with a maximum of 1024 bytes.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get path
my $path = $backup->path();
#Modify path
$backup->path("/tmp/infoblox/database.tar.gz");
```

disabled( )

Use this method to disable the ScheduledBackup object.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

Set the parameter to "true" to disable the ScheduledBackup object. Set the parameter to "false" to enable the object. The default value of this parameter is "false".

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get disabled
my $disabled = $backup->disabled();
#Modify disabled
$backup->disabled("true");
```

source( )

Use thi method to set or retrieve the source of ScheduledBackup object.

Parameter

Can be set to "DB" to operate with Grid ScheduledBackup object or to "REPORTING" to operate with Reporting ScheduledBackup object.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get source
my $source = $backup->source();
#Modify source
$backup->source("REPORTING");
```

status( )

Use this method to set or retrieve the status of the Reporting ScheduledBackup object.

Parameter

Either "TRIGGERED" or "IN_PROGRESS" to identify that Reporting backup/restore is running. Status "FINISHED" means Reporting backup/restore succeeded. Status "FAILED" means Reporting backup/restore failed. The default value of this parameter is "IDLE".

Set status to "TRIGGERED" to run a Reporting manual backup. Use this for a Reporting manual backup only.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get status
my $status = $backup->status();
#Modify status
$backup->status("TRIGGERED");
```

operation( )

Use this method to set or retrieve the operation of the Reporting ScheduledBackup object.

Parameter

Specify "BACKUP" to indicate that a Reporting backup must be performed, or specify "RESTORE" to indicate that a Reporting restore must be performed. The default value is "NONE".

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get operation
my $operation = $backup->operation();
#Modify operation
$backup->operation("RESTORE");
```

restore_type( )

Use this method to set or retrieve the destination of the restore files when the operation is set to "RESTORE". You can back up system files to an FTP server, or an SSH server that supports SCP.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

Valid values are "FTP" or "SCP". The default value is "FTP".

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get restore_type
my $restore_type = $backup->restore_type();
#Modify restore_type
$backup->restore_type("FTP");
```

restore_host( )

Use this method to set or retrieve the IP address of the restore server when the operation is set to "RESTORE".

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

The IP address of the restore server.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get backup_server
my $restore_host = $backup->restore_host();
#Modify restore_host
$backup->restore_host("my-restore-server");
```

restore_username( )

Use this method to set or retrieve the user name that is used to log in to the restore server. This is required only when "RESTORE" operation is performed.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

The user name in string format with a maximum of 1024 bytes.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get user
my $restore_username = $backup->restore_username();
#Modify user
$backup->restore_username("admin1");
```

restore_password( )

Use this method to set the user password that is used to log in to the restore server. This is required only when "RESTORE" operation is performed. This is a write-only attribute.

Parameter

The restore_password in string format with a maximum of 64 bytes.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

Example

```
#Modify restore_password
$backup->restore_password("userpassword");
```

restore_directory( )

Use this method to set or retrieve the directory path to the restored file on the server. This is required only when "RESTORE" operation is performed.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

Desired path in string format with a maximum of 1024 bytes.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get restore_directory
my $restore_directory = $backup->restore_directory();
#Modify restore_directory
$backup->restore_directory("/tmp");
```

SAMPLE CODE

The following sample code demonstrates different operations that can be applied to an object such as create, modify, and remove an object. This sample code also includes error handling for the operations.

```
#PROGRAM STARTS: Include all the modules that will be used
use strict;
use Infoblox;

#Create a session to the Infoblox appliance
my $session = Infoblox::Session->new(
    master   => "192.168.1.2",
    username => "admin",
    password => "infoblox"
);
unless ($session) {
   die("Construct session failed: ",
       Infoblox::status_code() . ":" . Infoblox::status_detail());
}
print "Session created successfully\n";
```

#Add a ScheduledBackup object

```
my @result_array = $session->get( "object" => "Infoblox::Grid", "name" => "Infoblox");
if( scalar( @result_array ) > 0 )
 {
        my $Grid_obj = $result_array[0];
        unless ($Grid_obj) {
                die("Get Grid object failed: ",
        $session->status_code() . ":" . $session->status_detail());
        }
        # construct a ScheduledBackup object
        my $backup = Infoblox::Grid::ScheduledBackup->new (
            backup_type => "LOCAL",
            backup_frequency => "Weekly",
            weekday => "0", # "0" is the same as "Sunday"
            hour_of_day => "4",
            minute_of_hour => "0",
            user  => "username",
            password => "user-password",
            backup_server  => "10.0.3.4",
            path => "/tmp/infoblox/database.tar.gz",
            disabled => "false",
        );

        # Add ScheduledBackup to Grid object.
        $Grid_obj->scheduled_backup($backup);

        $session->modify($Grid_obj)
            or die("Modify Grid failed: ",$session->status_code(). ":" .$session->status_detail());
        print "ScheduledBackup added to Grid object sucessfully\n";
         } else {
             print "No Grid object found.";
         }
```

#Modify an existing ScheduledBackup object

```
@result_array = $session->get( "object" => "Infoblox::Grid", "name" => "Infoblox");
if( scalar( @result_array ) > 0 )
```

```
{
        my $Grid_obj = $result_array[0];
        unless ($Grid_obj) {
                die("Get Grid object failed: ",
        $session->status_code() . ":" . $session->status_detail());
        }

         # get ScheduledBackup from Grid object.
         my $backup= $Grid_obj->scheduled_backup();

         #modify the backup frequency
         $backup->backup_frequency("Daily");

         # Add ScheduledBackup to Grid object.
         $Grid_obj->scheduled_backup($backup);

         $session->modify($Grid_obj)
            or die("Modify Grid failed: ",$session->status_code(). ":" .$session->status_detail());
         print "ScheduledBackup modified on Grid object sucessfully\n";
          } else {
              print "No Grid object found.";
          }
```

#Remove an existing ScheduledBackup object

```
@result_array = $session->get( "object" => "Infoblox::Grid", "name" => "Infoblox");
if( scalar( @result_array ) > 0 )
 {
        my $Grid_obj = $result_array[0];
        unless ($Grid_obj) {
                die("Get Grid object failed: ",
        $session->status_code() . ":" . $session->status_detail());
        }

         # empty ScheduledBackup list on Grid object.
         $Grid_obj->scheduled_backup([]);

         $session->modify($Grid_obj)
            or die("Modify Grid failed: ",$session->status_code(). ":" .$session->status_detail());
         print "ScheduledBackup removed on Grid object sucessfully\n";
          } else {
              print "No Grid object found.";
          }

####PROGRAM ENDS####
```

AUTHOR

Infoblox Inc. http://www.infoblox.com/

SEE ALSO

Infoblox::Grid, Infoblox::Grid->scheduled_backup()

COPYRIGHT

Copyright (c) 2017 Infoblox Inc.

Infoblox::Grid::SNMP::Admin - Manages a SNMP Admin object.

- NAME
- DESCRIPTION
- CONSTRUCTOR
- MODULE METHODS
  - Infoblox::Grid->snmp_admin( )
  - Infoblox::Grid::Member->snmp_admin( )
- METHODS
  - sysContact( )
  - sysContact_node1( )
  - sysContact_node2( )
  - sysDescr( )
  - sysDescr_node1( )
  - sysDescr_node2( )
  - sysLocation( )
  - sysLocation_node1( )
  - sysLocation_node2( )
  - sysName( )
  - sysName_node1( )
  - sysName_node2( )
- SAMPLE CODE
- AUTHOR
- SEE ALSO
- COPYRIGHT

NAME

Infoblox::Grid::SNMP::Admin - Manages a SNMP Admin object.

DESCRIPTION

SNMP (Simple Network Management Protocol) is a protocol that administrators use to manage network devices and monitor their processes. An SNMP-managed appliance, such as an Infoblox appliance, has an SNMP agent that collects data and stores them as objects in MIBs (Management Information Bases). You can configure an Infoblox appliance as an SNMP-managed appliance. Infoblox appliances support SNMP versions 1, 2, and 3. You can enter values for sysContact, sysLocation, sysName, sysDescr managed objects in MIB-II, the standard MIB defined in RFC 1213. Once these values are set on the appliance, the administrators can send queries for these values from management systems that are allowed to send queries to the appliance.

CONSTRUCTOR

For SNMP Admin objects used as part of an HA pair

```
my $snmp = Infoblox::Grid::SNMP::Admin->new(
    sysContact_node1  => $string,      #Optional / Default is empty
    sysDescr_node1    => $string,      #Optional / Default is empty
    sysLocation_node1 => $string,  #Optional / Default is empty
    sysName_node1     => $string,  #Optional / Default is hostname of appliance
    sysContact_node2  => $string,      #Optional / Default is empty
    sysDescr_node2    => $string,      #Optional / Default is empty
    sysLocation_node2 => $string,  #Optional / Default is empty
    sysName_node2     => $string,  #Optional / Default is hostname of appliance
 );
```

otherwise

```
my $snmp = Infoblox::Grid::SNMP::Admin->new(
    sysContact  => $string,  #Optional / Default is empty
    sysDescr    => $string,  #Optional / Default is empty
    sysLocation => $string,  #Optional / Default is empty
    sysName     => $string,  #Optional / Default is hostname of appliance
 );
```

When the SNMP Admin object is not used as part of an HA pair, sysContact/sysContact_node1, sysDescr/sysDescr_node1, ... are equivalent. The _node2 fields will be empty.

MODULE METHODS

The following functions are available to apply to an SNMP admin object.

Infoblox::Grid->snmp_admin( )

Use this function to specify an SNMP admin at the grid level. See Infoblox::Grid->snmp_admin() for parameters and return values.

Example

```
#Construct an object
 my $snmp = Infoblox::Grid::SNMP::Admin->new(
   sysContact  => "Somebody",
   sysDescr    => "SunOS noc1 5.6 Generic_105181-03 sun4u",
   sysLocation => "Infoblox Headquarter",
   sysName     => "noc1"
);

#Configure snmp at grid level
my $response = $grid->snmp_admin($snmp);
```

Infoblox::Grid::Member->snmp_admin( )

Use this function to specify an SNMP admin at the member level. See Infoblox::Grid::Member->snmp_admin() for parameters and return values.

Example

```
#Construct an object
  my $snmp = Infoblox::Grid::SNMP::Admin->new(
    sysContact_node1  => "Somebody",
    sysDescr_node1    => "SunOS noc1 5.6 Generic_105181-03 sun4u",
    sysLocation_node1 => "Infoblox Headquarter",
    sysName_node1     => "noc1"
);

# Configure snmp at member level
$member->snmp_admin($snmp);
```

METHODS

This section describes all the methods that you can use to configure and retrieve the attribute values of an SNMP admin object.

sysContact( )

Use this method to set or retrieve the name of the contact person for the Simple Network Management Protocol (SNMP) agent host.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

The name of the contact person in string format. The maximum length of the name is 50 characters.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get sysContact
my $sysContact = $snmp->sysContact();
#Modify sysContact
$snmp->sysContact("Infoblox");
```

sysContact_node1( )

Use this method to set or retrieve the name of the contact person for the Simple Network Management Protocol (SNMP) agent host on node 1 of an HA pair.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

The name of the contact person in string format. The maximum length of the name is 50 characters.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get sysContact_node1
my $sysContact_node1 = $snmp->sysContact_node1();
#Modify sysContact_node1
$snmp->sysContact_node1("Infoblox");
```

sysContact_node2( )

Use this method to set or retrieve the name of the contact person for the Simple Network Management Protocol (SNMP) agent host on node 2 of an HA pair.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

The name of the contact person in string format. The maximum length of the name is 50 characters.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get sysContact_node2
my $sysContact_node2 = $snmp->sysContact_node2();
#Modify sysContact_node2
$snmp->sysContact_node2("Infoblox");
```

sysDescr( )

Use this method to set or retrieve the name and version ID of the system's hardware type, the software Operational System (OS), and networking software that the appliance is running.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

Useful information about the hardware and software described in string format. The maximum length of the sysDescr is 50 characters.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get sysDescr
my $sysDescr = $snmp->sysDescr();
#Modify sysDescr
$snmp->sysDescr("SunOS noc2 5.6 Generic_105181-03 sun5u");
```

sysDescr_node1( )

Use this method to set or retrieve the name and version ID of the system's hardware type, the software Operational System (OS), and networking software that the appliance is running on node 1 of an HA pair.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

Useful information about the hardware and software described in string format. The maximum length of the sysDescr_node1 is 50 characters.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get sysDescr_node1
my $sysDescr_node1 = $snmp->sysDescr_node1();
#Modify sysDescr_node1
$snmp->sysDescr_node1("SunOS noc2 5.6 Generic_105181-03 sun5u");
```

sysDescr_node2( )

Use this method to set or retrieve the name and version ID of the system's hardware type, the software Operational System (OS), and networking software that the appliance is running on node 2 of an HA pair.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

Useful information about the hardware and software described in string format. The maximum length of the sysDescr_node2 is 50 characters.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get sysDescr_node2
my $sysDescr_node2 = $snmp->sysDescr_node2();
#Modify sysDescr_node2
$snmp->sysDescr_node2("SunOS noc2 5.6 Generic_105181-03 sun5u");
```

sysLocation( )

Use this method to set or retrieve the physical location of the appliance.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

The physical location of the appliance in string format. The maximum length of the sysLocation is 50 characters.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails. If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get sysLocation
my $sysLocation = $snmp->sysLocation();
#Modify sysLocation
$snmp->sysLocation("Infoblox Headquarter5");
```

sysLocation_node1( )

Use this method to set or retrieve the physical location of the appliance on node 1 of an HA pair.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

The physical location of the appliance in string format. The maximum length of the sysLocation_node1 is 50 characters.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails. If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get sysLocation_node1
my $sysLocation_node1 = $snmp->sysLocation_node1();
#Modify sysLocation_node1
$snmp->sysLocation_node1("Infoblox Headquarter5");
```

sysLocation_node2( )

Use this method to set or retrieve the physical location of the appliance on node 2 of an HA pair.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

The physical location of the appliance in string format. The maximum length of the sysLocation_node2 is 50 characters.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails. If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get sysLocation_node2
my $sysLocation_node2 = $snmp->sysLocation_node2();
#Modify sysLocation_node2
$snmp->sysLocation_node2("Infoblox Headquarter5");
```

sysName( )

Use this method to set or retrieve the fully qualified domain name (FQDN) of the appliance.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

The fully qualified domain name (FQDN) of the appliance in string format. The maximum length of the sysName is 50 characters. The default value of sysName is the hostname of the Infoblox appliance.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get sysName
my $sysName = $snmp->sysName();
#Modify sysName
$snmp->sysName("Infobloxnoc2");
```

sysName_node1( )

Use this method to set or retrieve the fully qualified domain name (FQDN) of the appliance on node 1 of an HA pair.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

The fully qualified domain name (FQDN) of the appliance in string format. The maximum length of the sysName_node1 is 50 characters. The default value of sysName_node1 is the hostname of the Infoblox appliance.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get sysName_node1
my $sysName_node1 = $snmp->sysName_node1();
#Modify sysName_node1
$snmp->sysName_node1("Infobloxnoc2");
```

sysName_node2( )

Use this method to set or retrieve the fully qualified domain name (FQDN) of the appliance on node 2 of an HA pair.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

The fully qualified domain name (FQDN) of the appliance in string format. The maximum length of the sysName_node2 is 50 characters. The default value of sysName_node2 is the hostname of the Infoblox appliance.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get sysName_node2
my $sysName_node2 = $snmp->sysName_node2();
#Modify sysName_node2
$snmp->sysName_node2("Infobloxnoc2");
```

SAMPLE CODE

The following sample code demonstrates the different functions that can be applied to an object such as add, modify and remove. This sample also includes error handling for the operations.

#Preparation prior to an SNMP admin object insertion

```
#PROGRAM STARTS: Include all the modules that will be used
use strict;
use Infoblox;

#Create a session to the Infoblox appliance
my $session = Infoblox::Session->new(
    master   => "192.168.1.2",
    username => "admin",
    password => "infoblox"
);
unless ($session) {
   die("Construct session failed: ",
       Infoblox::status_code() . ":" . Infoblox::status_detail());
}
print "Session created successfully\n";
```

#Creating an SNMP admin object

```
my $snmp = Infoblox::Grid::SNMP::Admin->new(
    sysContact  => "Somebody",
    sysDescr    => "SunOS noc1 5.6 Generic_105181-03 sun4u",
    sysLocation => "Infoblox Headquarter",
    sysName     => "noc1"
    );
unless($snmp) {
     die("Construct a SNMP Admin failed: ",
           Infoblox::status_code() . ":" . Infoblox::status_detail());
}
print "SNMP Admin object created successfully\n";
```

#Getting the grid object from the appliance through session

```
 my @result = $session->get(
    object => "Infoblox::Grid",
    name   => "Infoblox"
    );

my $grid = $result[0];
unless ($grid) {
       die("Get Grid object failed: ",
            $session->status_code() . ":" . $session->status_detail());
}
print "Get Grid object found at least 1 matching entry\n";

      #Modifying the value of the specified object.
        $grid->snmp_admin($snmp);  #Applying the changes.
        $session->modify($grid);
        print "SNMP Admin modified successfully\n";
```

#Modify SNMP admin attribute sysName

```
 #Modify existing attribute sysName
$snmp->sysName("noc2.infoblox.com");

 #Apply changes to the Grid object.
$grid->snmp_admin($snmp);

 #Update Grid object through the Infoblox session.
$session->modify($grid)
```

```
            or die("modify Grid object failed: ",
                $session->status_code() . ":" . $session->status_detail());
        print "Modified SNMP Admin sysName of Grid object successful\n";
```

#Remove SNMP admin attribute from Grid object

```
  #Apply changes to the Grid object to Un-def SNMP Admin object.
  $grid->snmp_admin(undef);

  #Update Grid object through the Infoblox session.
  $session->modify($grid)
            or die("Remove SNMP Admin from Grid object failed: ",
                $session->status_code() . ":" . $session->status_detail());
        print "Removed SNMP Admin from Grid object successful\n";

 ####PROGRAM ENDS####
```

AUTHOR

Infoblox Inc. http://www.infoblox.com/

SEE ALSO

Infoblox::Session,Infoblox::Session->get(), Infoblox::Session->modify(), Infoblox::Grid

---

COPYRIGHT

Copyright (c) 2017 Infoblox Inc.

Infoblox::Grid::SyslogBackupServer - Syslog Backup Server object.

- NAME
- DESCRIPTION
- CONSTRUCTOR
- MODULE METHODS
  - Infoblox::Grid->syslog_backup_servers( )
  - Infoblox::Grid::Member->syslog_backup_servers( )
- METHODS
  - address( )
  - directory_path( )
  - enable( )
  - password( )
  - port( )
  - protocol( )
  - username( )
- AUTHOR
- SEE ALSO
- COPYRIGHT

NAME

Infoblox::Grid::SyslogBackupServer - Syslog Backup Server object.

DESCRIPTION

The Syslog Backup Server object contains configuration for external servers that are used for replication of rotated log files through SCP or FTP.

CONSTRUCTOR

```
my $syslog_backup_server = Infoblox::Grid::SyslogBackupServer->new(
    address        => $ipv4addr | $ipv6addr, # Optional
    directory_path => $string,               # Optional
    enable         => 'true' | 'false',      # Optional
    password       => $string,               # Optional
    port           => $uint,                 # Optional
    protocol       => 'SCP' | 'FTP',         # Optional
    username       => $string,               # Optional
);
```

MODULE METHODS

The following functions are available to be applied to a syslog backup server object.

Infoblox::Grid->syslog_backup_servers( )

Use this method to set or retrieve an array that contains Grid-level syslog backup server settings. See Infoblox::Grid->syslog_backup_servers() for parameters and return values.

Example

```
#get syslog_backup_servers value
my $servers = $grid->syslog_backup_servers();

#modify syslog_backup_servers value
$grid->syslog_backup_servers([$server1, $server2]);
```

Infoblox::Grid::Member->syslog_backup_servers( )

Use this method to set or retrieve an array that contains syslog backup server settings for a Grid member. See Infoblox::Grid::Member->syslog_backup_servers() for parameters and return values.

Example

```
#get syslog_backup_servers value
my $servers = $grid_member->syslog_backup_servers();

#modify syslog_backup_servers value
$grid_member->syslog_backup_servers([$server1, $server2]);
```

METHODS

address( )

Use this method to set or retrieve the IPv4 or IPv6 address of a syslog backup server.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

An IPv4 address is a 32-bit number in dotted decimal notation. It consists of four 8-bit groups of decimal digits separated by decimal points (example: 192.168.1.2).

An IPv6 address is a 128-bit number represented as eight groups of four hexadecimal digits (i.e. two octets), with each group separated by a `colon(:)` (example: 2001:0db8:85a3:0000:0000:8a2e:0370:7334). Leading zeroes and groups of consecutive zeros within the value can be omitted to produce a more compact representation (example: 2001::1, ::1).

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#get address value
my $address = $server->address();

#modify address value
$server->address('192.168.1.5');
```

directory_path( )

Use this method to set or retrieve the path for rotated syslog file replication.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

The valid value is a desired path in a string format.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#get directory_path value
my $directory_path = $server->directory_path();

#modify directory_path value
$server->directory_path('/path/to/directory');
```

enable( )

Use this method to enable or disable the syslog backup server.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

Specify 'true' to enable syslog backup server or 'false' to disable it. The default value is 'true'.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#get enable value
my $enable = $server->enable();

#modify enable value
```

```
$server->enable('true');
```

password( )

Use this method to set the syslog backup server password. This is a write-only attribute.

Include the specified parameter to set the attribute value.

Parameter

The valid value is a desired password in a string format.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

Example

```
#modify password value
$server->password('infoblox');
```

port( )

Use this method to set or retrieve the transport layer port for connection to the syslog backup server.

Include the specified parameter to set the attribute value.

Parameter

The valid value is an unsigned integer between 0 and 65535.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#get port value
my $port = $server->port();

#modify port value
$server->port(22);
```

protocol( )

Use this method to set or retireve the transport layer protocol for the syslog backup server connection.

Include the specified parameter to set the attribute value.

Parameter

The valid values are 'FTP' and 'SCP'.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#get protocol value
my $protocol = $server->protocol();

#modify protocol value
$server->protocol('SCP');
```

username( )

Use this method to set or retrieve the username for the syslog backup server.

Include the specified parameter to set the attribute value.

Parameter

Valid value is a desired username in string format.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#get username value
my $username = $server->username();

#modify username value
$server->username('admin');
```

AUTHOR

Infoblox Inc. http://www.infoblox.com/

SEE ALSO

Infoblox::Grid, Infoblox::Grid::Member

COPYRIGHT

Copyright (c) 2017 Infoblox Inc.

Infoblox::Grid::SyslogServer - Syslog server object.

- NAME
- DESCRIPTION
- CONSTRUCTOR
- MODULE METHODS
  - Infoblox::Grid->syslog_server( )
  - Infoblox::Grid::Member->syslog_server( )
- METHODS
  - address( )
  - category_list( )
  - certificate( )
  - connection_type( )
  - local_interface( )
  - message_source( )
  - only_category_list( )
  - port( )
  - severity( )
- SAMPLE CODE
- AUTHOR
- SEE ALSO
- COPYRIGHT

NAME

Infoblox::Grid::SyslogServer - Syslog server object.

DESCRIPTION

Syslog is a widely used mechanism for logging system events. Infoblox appliances generate syslog messages which can be viewed through the system log viewer and downloaded to a directory on your management station. In addition, the Infoblox appliance can be configured to send the messages to one or more external syslog servers for later analysis. Syslog messages provide information about appliance operations and processes. Also, the appliance can include audit log messages and specific BIND messages among the messages sent to the syslog server.

CONSTRUCTOR

```
 my $syslog_server = Infoblox::Grid::SyslogServer->new(
        address             => $address,                          #Required
        connection_type     => "tcp" | "udp" | "stcp",            #Optional / Default is "udp"
        local_interface     => "any" | "mgmt" | "lan",            #Optional / Default is "any"
        message_source      => "any" | "internal" | "external",   #Optional / Default is "any"
        port                => $num,                              #Optional / Default is 514 for TCP/UDP,
6514 for STCP
        severity            => "debug" | "info" | "notice" | "warning" | "err" | "crit" | "alert" | "emerg",
#Optional / Default is "debug"
        certificate         => $fname,  #Optional / Default is undefined

 );
```

MODULE METHODS

The following functions are available to be applied to a syslog server object.

Infoblox::Grid->syslog_server( )

Use this function to specify a syslog server at the grid level on the Infoblox appliance. See Infoblox::Grid->syslog_server() for parameters and return values.

Example

```
       #Construct a syslog server object
       my $syslog_server1 = Infoblox::Grid::SyslogServer->new(
                               address => "1.1.1.1",
                             );

       #Configure syslog server on the Infoblox grid object
       my $responce = $grid->syslog_server([$syslog_server1]);
```

Infoblox::Grid::Member->syslog_server( )

Use this function to specify a syslog server at the member level on the Infoblox appliance. See Infoblox::Grid::Member->syslog_server() for parameters and return values.

Example

```
#Construct a syslog server object
my $syslog_server2 = Infoblox::Grid::SyslogServer->new(
                       address => "2.2.2.2",
                       severity => "debug",
                     );

#Configure syslog server on the grid member object
my $responce = $grid_member->syslog_server([$syslog_server2]);
```

METHODS

This section describes all the methods that can be used to configure and retrieve the attribute value of a syslog server object.

address( )

Use this method to set or retrieve the IPv4 or IPv6 address of a syslog server.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

An IPv4 address is a 32-bit number in dotted decimal notation. It consists of four 8-bit groups of decimal digits separated by decimal points (example: 192.168.1.2).

An IPv6 address is a 128-bit number represented as eight groups of four hexadecimal digits (i.e. two octets), with each group separated by a `colon(:)` (example: 2001:0db8:85a3:0000:0000:8a2e:0370:7334). Leading zeroes and groups of consecutive zeros within the value can be omitted to produce a more compact representation (example: 2001::1, ::1).

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#get address
my $address = $syslog_server->address();
#Modify address
$syslog_server->address("3.3.3.3");
```

category_list( )

Use this method to set or retrieve the list of syslog categories for forwarding.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

The valid value is a list containing these values: 'NON_CATEGORIZED', 'ATP', 'DNS_CLIENT', 'DNS_CONFIG', 'DNS_DATABASE', 'DNS_DNSSEC', 'DNS_GENERAL', 'DNS_LAME_SERVERS', 'DNS_NETWORK', 'DNS_NOTIFY', 'DNS_QUERIES', 'DNS_QUERY_REWRITE', 'DNS_RESOLVER', 'DNS_RESPONSES', 'DNS_RPZ', 'DNS_SECURITY', 'DNS_UPDATE', 'DNS_UPDATE_SECURITY', 'DNS_XFER_IN', 'DNS_XFER_OUT', 'DNS_UNBOUND', 'DTC_HEALTHD', 'DTC_IDNSD', 'DHCPD', 'NTP', 'FTPD', 'TFTPD', 'CLOUD_API', 'MS_DNS_SERVER', 'MS_CONNECT_STATUS', 'MS_DNS_ZONE', 'MS_DHCP_SERVER', 'MS_DHCP_LEASE', 'MS_DHCP_CLEAR_LEASE', 'MS_SITES', 'MS_AD_USERS', 'AUTH_COMMON', 'AUTH_NON_SYSTEM', 'AUTH_UI_API', 'AUTH_ACTIVE_DIRECTORY', 'AUTH_RADIUS', 'AUTH_TACACS', 'AUTH_LDAP', 'OUTBOUND_API'.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get category_list
my $category_list = $syslog_server->category_list();

#Modify category_list
$syslog_server->category_list(['DNS_CLIENT']);
```

certificate( )

Use this method to set a path to the file that contains a client certificate. This is a write-only attribute.

Note that the certificate must be set in order to establish an STCP connection.

Parameter

The valid value is a string that contains the path to the client certificate file.

Returns

The method returns true when the modification succeeds, and returns false when the operation fails.

Example

```
#Modify certificate
$syslog_server->certificate('/path/to/certificate.pem');
```

connection_type( )

Use this method to set or retrieve the connection type to the external syslog server. This method specifies whether the appliance uses TCP or UDP to connect to the external syslog server.

Note that the certificate must be set in order to establish an STCP connection.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

Valid values are: `"tcp", "udp", "stcp"`. The default value is "udp".

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#get connection_type
my $connection_type = $syslog_server->connection_type();
#Modify connection_type
$syslog_server->connection_type("tcp");
```

local_interface( )

Use this method to set or retrieve the interface through which the appliance sends syslog messages to the syslog server.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

Valid values are: `"any", "mgmt", "lan"`. The default value is "any".

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#get local_interface
my $local_interface = $syslog_server->local_interface();
#Modify local_interface
$syslog_server->local_interface("lan");
```

message_source( )

Use this method to set or retrieve the message source value of the syslog server. This method specifies which syslog messages the appliance sends to the external syslog server:

```
 Any     : The appliance sends both internal and external syslog messages.
 Internal: The appliance sends syslog messages that it generates.
 External: The appliance sends syslog messages that it receives from other appliances, such as syslog
servers and routers.
```

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

Valid values are: "any", "internal", "external". The default value is "any".

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#get message_source
my $message_source = $syslog_server->message_source();
#Modify message_source
$syslog_server->message_source("internal");
```

only_category_list( )

Use this method to enable or disable forwarding of syslog messages that belong to the list of categories.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

Specify "true" to enable forwarding or "false" to disable it. The default value is "false".

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get only_category_list
my $only_category_list = $syslog_server->only_category_list();

#Modify only_category_list
$syslog_server->only_category_list("true");
```

port( )

Use this method to set or retrieve the destination port number.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

Valid values are between 1 and 65535. The default value is 514.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#get port
my $port = $syslog_server->port();
#Modify port
$syslog_server->port("515");
```

severity( )

Use this method to set or retrieve the severity level for the syslog server. When you set a severity level, grid members send messages for that severity level plus all messages for all severity levels above it. The lowest severity level is "debug", and the highest severity level is "emerg".

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

Valid values are: "debug", "info", "notice", "warning", "err", "crit", "alert", "emerg". The default value is "debug".

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#get severity
my $severity = $syslog_server->severity();
#Modify severity
$syslog_server->severity("info");
```

SAMPLE CODE

The following sample code demonstrates the different functions that can be applied to a syslog server object such as add, modify, and remove. Also, this sample code includes error handling for the operations.

#Preparation prior to a syslog server object insertion

```
use strict;
use Infoblox;

#Create a session to the Infoblox appliance
my $session = Infoblox::Session->new(
    master   => "192.168.1.2",   #appliance host ip
    username => "admin",         #appliance user login
    password => "infoblox"       #appliance password
);
unless ($session) {
   die("Construct session failed: ",
       Infoblox::status_code() . ":" . Infoblox::status_detail());
}
print "Session created successfully.\n";
```

#Create a syslog server object

```
my $syslog_server = Infoblox::Grid::SyslogServer->new(
                    address => "1.1.1.1",
                    severity => "debug"
                   );

unless($syslog_server) {
       die("Construct syslog server object failed: ",
            Infoblox::status_code() . ":" . Infoblox::status_detail());
}
print "syslog server object created successfully\n";
```

#Get Infoblox grid object and add syslog server to it

```
#Get Infoblox grid object
my @grid_objs = $session->get(
           object => "Infoblox::Grid",
           name   => "Infoblox"
       );

my $grid = $grid_objs[0];
unless ($grid) {
       die("Get Grid object failed: ",
            $session->status_code() . ":" . $session->status_detail());
}
print "Get Grid object found at least 1 matching entry\n";

#Apply the changes to the Grid object
$grid->syslog_server([$syslog_server]);

#Apply the changes
$session->modify($grid)
    or die("Modify Grid object failed: ",
            $session->status_code() . ":" . $session->status_detail());

print "Grid object with syslog server updated to Infoblox appliance successfully\n";
```

#Modify syslog server object

```
#Get Infoblox grid object
my @grid_objs = $session->get(
           object => "Infoblox::Grid",
```

```
            name   => "Infoblox"
        );

 unless(scalar(@grid_objs) == 0){
        my $grid = $grid_objs[0];
        if($grid){
                my $syslog_obj = $grid->syslog_server();
                my @sys_servers = @{$syslog_obj};
                my $sys_server = $sys_servers[0];

                #Modifying the address of the syslog server object.
                $sys_server->address("2.2.2.2");

                #Modifying the connection_type of the syslog server object.
                $sys_server->connection_type("tcp");

                #Modifying the local_interface of the syslog server object.
                $sys_server->local_interface("lan");

                #Modifying the message_source of the syslog server object.
                $sys_server->message_source("external");

                #Modifying the port of the syslog server object.
                $sys_server->port("516");

                #Modifying the severity of the syslog server object.
                $sys_server->severity("warning");

                #Apply changes to the Grid object.
                $grid->syslog_server([$sys_server]);

                #Update Grid object through the Infoblox session.
                $session->modify($grid)
                             or die("modify Grid object failed: ",
                                 $session->status_code() . ":" . $session->status_detail());

                print "Grid object with modified syslog server updated to Infoblox appliance
successfully\n";
                        }
                 }
```

#Remove syslog server

```
 #Get Infoblox grid object
 my @grid_objs = $session->get(
            object => "Infoblox::Grid",
            name   => "Infoblox"
        );

 my $grid = $grid_objs[0];
 unless ($grid) {
        die("Get Grid object failed: ",
             $session->status_code() . ":" . $session->status_detail());
 }
 print "Get Grid object found at least 1 matching entry\n";

 #Remove syslog servers from the Grid object.
 $grid->syslog_server(undef);

 #Update Grid object through the Infoblox session.
 $session->modify($grid)
             or die("Remove syslog server from Grid object failed: ",
                 $session->status_code() . ":" . $session->status_detail());

 print "Removed syslog server from Grid object successfully\n";

 ####PROGRAM ENDS####
```

AUTHOR

Infoblox Inc. http://www.infoblox.com/

SEE ALSO

Infoblox::Session,Infoblox::Grid,Infoblox::Grid::Member

COPYRIGHT

Copyright (c) 2017 Infoblox Inc.

Infoblox::Grid::SNMP::TrapReceiver - Grid SNMP TrapReceiver object.

- NAME
- DESCRIPTION
- CONSTRUCTOR
- METHODS
  - address()
  - user()
- AUTHOR
- SEE ALSO
- COPYRIGHT

NAME

Infoblox::Grid::SNMP::TrapReceiver - Grid SNMP TrapReceiver object.

DESCRIPTION

The Grid SNMP TrapReceiver object is used to set or retrieve the IP address and user name of a trap receiver.

CONSTRUCTOR

```
my $Infoblox::Grid::SNMP::TrapReceiver->new(
    address => $ip_address,   #Required
    user => $snmpv3user       #Required
);
```

METHODS

This section describes all the methods that you can use to configure and retrieve the attribute values of an SNMPv3 TrapReceiver object.

address()

Use this method to set or retrieve the IP address of the trap receiver.

Parameter

IPv4 address of the trap receiver

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get trap receiver address
my $ip=$trap_receiver->address();
#Set trap receiver address
$trap_receiver->address("192.168.1.2");
```

user()

Use this method to set or retrieve the user name for the trap receiver.

Parameter

A reference to the Infoblox::Grid::SNMP::User object or the user name in string format.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get trap receiver user
my $user=$trap_receiver->user();
#Set trap receiver user
$trap_receiver->user($user);
```

Infoblox::Grid::SNMP::TrapReceiver - Grid SNMP TrapReceiver object.

AUTHOR

Infoblox Inc. http://www.infoblox.com/

SEE ALSO

Infoblox::Grid::SNMP::User

COPYRIGHT

Copyright (c) 2017 Infoblox Inc.

Infoblox::Grid::UpdatesDownloadMemberConfig - An Updates Download Member Configuration object.

- NAME
- DESCRIPTION
- CONSTRUCTOR
- SESSION METHODS
- MODULE METHODS
  - Infoblox::Grid->updates_download_member_config( )
- METHODS
  - interface( )
  - is_online( )
  - member( )
- AUTHOR
- SEE ALSO
- COPYRIGHT

NAME

Infoblox::Grid::UpdatesDownloadMemberConfig - An Updates Download Member Configuration object.

DESCRIPTION

The Updates Download Member Configuration object provides information and settings for configuring Member that is responsible for updates download.

CONSTRUCTOR

```
my $http_proxy_setting = Infoblox::Grid::UpdatesDownloadMemberConfig->new(
    member    => $string | undef,                  #Required
    interface => 'ANY' | 'VIP' | 'LAN2' | 'MGMT', #Required
);
```

SESSION METHODS

The object does not support any session methods.

MODULE METHODS

The following functions are available to be applied to a Updates Download Member Configuration object.

Infoblox::Grid->updates_download_member_config( )

Use this method to set or retrieve the list of updates download member configurations. See Infoblox::Grid->updates_download_member_config() for parameters andreturn values.

Example

```
#Get updates_download_member_config value
my $updates_download_member_config = $grid->updates_download_member_config();

#Modify updates_download_member_config value
$grid->updates_download_member_config([$conf1, $conf2]);
```

METHODS

This section describes all the methods that you can use to configure and retrieve the attribute values of a Updates Download Member Configuration object.

interface( )

Use this method to set or retrieve the source interface for updates download requests.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

The valid values are 'ANY', 'VIP', 'LAN2' and 'MGMT'.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get interface value
my $interface = $obj->interface();

#Modify interface value
$obj->interface('ANY');
```

is_online( )

Use this method to retrieve the flag that indicates whether the updates download member is on-line or not.

Omit the parameter to retrieve the attribute value.

Parameter

None

Returns

The method returns the attribute value.

Example

```
#Get is_online value
my $is_online = $obj->is_online();
```

member( )

Use this method to set or retrieve the updates download member name.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

The valid value is a desired member's name in a string format or undef for Grid Master.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get member value
my $member = $obj->member();

#Modify member value
$obj->member('desired-member.com');

#Modify member value to Grid Master
$obj->member(undef);
```

AUTHOR

Infoblox Inc. http://www.infoblox.com/

SEE ALSO

Infoblox::Grid, Infoblox::Grid->updates_download_member_config()

COPYRIGHT

Copyright (c) 2017 Infoblox Inc.

Infoblox::Grid::Admin::AdAuthService - Active Directory Authentication Service object

- NAME
- DESCRIPTION
- CONSTRUCTOR
- SESSION METHODS
  - Infoblox::Session->add( )
  - Infoblox::Session->get( )
  - Infoblox::Session->modify( )
  - Infoblox::Session->remove( )
- METHODS
  - ad_auth_server_list( )
  - ad_domain( )
  - additional_search_paths( )
  - comment( )
  - disabled( )
  - disable_default_search_path( )
  - name( )
  - nested_group_querying( )
  - timeout( )
- SAMPLE CODE
- AUTHOR
- SEE ALSO
- COPYRIGHT

NAME

Infoblox::Grid::Admin::AdAuthService - Active Directory Authentication Service object

---

DESCRIPTION

This object allows you to specify an Active Directory (AD) authentication method and the AD authentication servers that Infoblox uses to authenticate administrators.

CONSTRUCTOR

```
my $obj = Infoblox::Grid::Admin::AdAuthService->new(
    ad_auth_server_list  => [$ad_server1, $ad_server2, ...],      #Required
    ad_domain            => $string,                              #Required
    additional_search_paths => [$string, ...],                    #Optional / Default is undefined
    disable_default_search_path => "true" | "false",              #Optional / Default is "false"
    name                 => $string,                              #Required
    nested_group_querying => "true" | "false",                    #Optional / Default is "false"
    comment              => $string,                              #Optional / Default is empty
    disabled             => "true" | "false",                     #Optional / Default is "false"
    timeout              => $num,                                 #Optional / Default is 5
);
```

SESSION METHODS

This section describes all the methods in an Infoblox::Session module that you can apply to an object.

Infoblox::Session->add( )

Use this method to add an object to the Infoblox appliance. See Infoblox::Session->add() for parameters and return values.

Example

```
# Construct an object
my $ad_auth_server1 = Infoblox::Grid::Admin::AdAuthServer->new(
    name       => "1.2.3.4",
    port       => 636,
    comment    => "domain controller in room 1"
    disabled   => "false",
    encryption => "SSL",
    mgmt_port  => "false",
);

# Construct another object
```

```
my $ad_auth_server2 = Infoblox::Grid::Admin::AdAuthServer->new(
    name       => "foo.com",
    port       => 389,
    comment    => "domain controller in room 2"
    disabled   => "false",
    encryption => "None",
    mgmt_port  => "false",
);

# Construct an object
my $obj = Infoblox::Grid::Admin::AdAuthService->new(
   ad_auth_server_list  => [$ad_auth_server1, $ad_auth_server2],
   ad_domain            => "authdhcptest.infoblox.com",
   comment              => "test comment",
   disabled             => "false",
   name                 => "Test AD Auth",
   timeout              => 333,
   nested_group_querying => "true",
   disable_default_search_path => "false",
   additional_search_paths => ["OU=OU_1","OU=OU_2,OU=OU_3"],
);
# Submit for addition
my $response = $session->add( $obj );
```

Infoblox::Session->get( )

Use this method to retrieve all the matching objects from the Infoblox appliance. See Infoblox::Session->get() for parameters and return values.

Key References

Apply the following attributes to get a specific object:

name - Required. AD authentication service name

Example

```
my @retrieved_objs = $session->get(
  object => "Infoblox::Grid::Admin::AdAuthService",
    name => "Test AD Auth",
);
```

Infoblox::Session->modify( )

Use this method to modify an object in the Infoblox appliance. See Infoblox::Session->modify() for parameters and return values.

Example

```
# Modify the comment
$obj->comment("this is a modified comment");
# Submit modification
my $response = $session->modify( $obj);
```

Infoblox::Session->remove( )

Use this method to remove an object from the Infoblox appliance. See Infoblox::Session->remove() for parameters and return values.

To remove a specific object, first use `get()` to retrieve the specific object, and then submit this object for removal.

Example

```
#Get the objects with the same name
my @retrieved_objs = $session->get(
  object => "Infoblox::Grid::Admin::AdAuthService",
    name => "Test AD Auth",
);
#Find the desired object from the retrieved list.
my $desired_obj = $retrieved_objs[0];
# Submit for removal
my $response = $session->remove( $desired_obj );
```

METHODS

Use the following methods to access the attributes of an Infoblox::Grid::Admin::AdAuthService object. Specify a parameter to set the attribute's value, or omit the parameter to get the attribute's value.

ad_auth_server_list( )

Use this method to set or retrieve the AD authentication server list.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

Valid value is an array reference that contains a list of Infoblox::Grid::Admin::AdAuthServer objects that describe the AD servers to which the appliance connects.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get ad_auth_server_list
my $val = $obj->ad_auth_server_list();
#Modify ad_auth_server_list
$obj->ad_auth_server_list([ $adauthserver1 ]);
```

ad_domain( )

Use this method to set or retrieve the Active Directory Domain in which the user credentials are located.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

Valid Active Directory Domain name

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get ad_domain
my $val = $obj->ad_domain();
#Modify ad_domain
$obj->ad_domain("authdhcptest.infoblox.com");
```

additional_search_paths( )

Use this method to set or retrieve an unordered list of additional search paths for nested group querying.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

The valid value is an array of search paths in a string format.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#get additional_search_paths value
$additional_search_paths = $obj->additional_search_paths();

#modify additional_search_paths value
$obj->additional_search_paths(['OU=OU_1', 'OU=OU_2,OU=OU_3']);
```

comment( )

Use this method to set or retrieve comments about the service.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

Comments in string format with a maximum of 256 bytes.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get comment
my $val = $obj->comment();
#Modify comment
$obj->comment("head office");
```

disabled( )

Use this method to disable the Active Directory authentication service.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

Set the parameter to "true" to disable the service. Set the parameter to "false" to enable the service. The default value is "false".

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get disabled
my $val = $obj->disabled();
#Modify disabled
$obj->disabled("true");
```

disable_default_search_path( )

Use this method to set or retrieve the flag that indicates whether default search path is used or not.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Paramter

Specify 'true' to exclude default search path or 'false' otherwise. The default value is 'false'.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get disable_default_search_path value
my $val = $obj->disable_default_search_path();
#Modify disable_default_search_path value
$obj->disable_default_search_path("true");
```

name( )

Use this method to set or retrieve the name of the AD authentication service.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

The AD authentication service name.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get name
my $val = $obj->name();
#Modify name
$obj->name("AD service");
```

nested_group_querying( )

Use this method to set or retrieve the flag that indicates whether nested group querying is enabled or not.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Paramter

Specify 'true' to enable nested group querying or 'false' to disable it. The default value is 'false'.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get nested_group_querying value
my $val = $obj->nested_group_querying();
#Modify nested_group_querying value
$obj->nested_group_querying("true");
```

timeout( )

Use this method to set or retrieve the timeout value, which is the number of seconds that the appliance waits for a response from the AD server.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

The number of seconds that the Infoblox appliance waits to connect to the server. The default value is 5.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get timeout
my $val = $obj->timeout();
#Modify timeout
$obj->timeout(22);
```

SAMPLE CODE

The following sample code demonstrates the different functions that can be applied to an Active Directory Authentication Service object, such as add, get, modify, and remove. This sample also includes error handling for the operations.

#Preparation prior to an AD Auth Service object insertion

```
use strict;
use Infoblox;

#Create a session to the Infoblox appliance
my $session = Infoblox::Session->new(
               master   => "192.168.1.2", #appliance host ip
               username => "admin",       #appliance user login
               password => "infoblox"     #appliance password
);
unless ($session) {
       die("Construct session failed: ",
            Infoblox::status_code() . ":" . Infoblox::status_detail());
}
print "Session created successfully\n";
```

#Create an AD Auth Service object

```
# Construct an object
```

```
my $ad_auth_server1 = Infoblox::Grid::Admin::AdAuthServer->new(
    name       => "1.2.3.4",
    port       => 636,
    comment    => "domain controller in room 1",
    disabled   => "false",
    encryption => "None",
    mgmt_port  => "false",
);

# Construct another object
my $ad_auth_server2 = Infoblox::Grid::Admin::AdAuthServer->new(
    name       => "foo.com",
    port       => 389,
    comment    => "domain controller in room 2",
    disabled   => "false",
    encryption => "SSL",
    mgmt_port  => "false",
);

#Create an AD Auth Service object
my $ad_service = Infoblox::Grid::Admin::AdAuthService->new(
    ad_auth_server_list => [ $ad_auth_server1, $ad_auth_server2 ],
    ad_domain           => "authdhcptest.infoblox.com",
    comment             => "test comment",
    disabled            => "false",
    name                => "Test AD Auth",
    timeout             => 333,
);
unless($ad_service) {
       die("Construct AD Auth Service object failed: ",
            Infoblox::status_code() . ":" . Infoblox::status_detail());
}
print "AD Auth Service object created successfully\n";

#Add the AD Auth Service object into the Infoblox appliance through a session
$session->add($ad_service)
    or die("Add AD Auth Service object failed: ",
            $session->status_code() . ":" . $session->status_detail());
print "AD Auth Service object added successfully\n";
```

#Get and modify an AD Auth Service

```
#Get an AD Auth Service through the session
my @retrieved_objs = $session->get(
    object     => "Infoblox::Grid::Admin::AdAuthService",
    name => "Test AD Auth",
);
my $object = $retrieved_objs[0];
unless ($object) {
       die("Get AD Auth Service failed: ",
            $session->status_code() . ":" . $session->status_detail());
}
print "Get AD Auth Service object found at least 1 matching entry\n";

#Modify one of the attributes of the specified AD Auth Service
$object->comment ("modified comment");

#Apply the changes
$session->modify($object)
    or die("Modify AD Auth Service failed: ",
            $session->status_code() . ":" . $session->status_detail());
print "AD Auth Service object modified successfully \n";
```

#Remove an AD Auth Service

```
#Get an AD Auth Service through the session
my @retrieved_objs = $session->get(
    object     => "Infoblox::Grid::Admin::AdAuthService",
    name       => "Test AD Auth"
);

my $object = $retrieved_objs[0];
unless ($object) {
       die("Get AD Auth Service failed: ",
            $session->status_code() . ":" . $session->status_detail());
}
print "Get AD Auth Service object found at least 1 matching entry\n";
```

```
#submit the object for removal
$session->remove($object)
    or die("Remove AD Auth Service failed: ",
            $session->status_code() . ":" . $session->status_detail());
print "AD Auth Service object removed successfully \n";

####PROGRAM ENDS####
```

AUTHOR

Infoblox Inc. http://www.infoblox.com

SEE ALSO

Infoblox::Grid::Admin::AdAuthServer, Infoblox::Session, Infoblox::Session->add(), Infoblox::Session->get(), Infoblox::Session->modify(), Infoblox::Session->remove()

COPYRIGHT

Copyright (c) 2017 Infoblox Inc.

Infoblox::Grid::Admin::AdAuthServer - Active Directory Authentication Server object

- NAME
- DESCRIPTION
- CONSTRUCTOR
- SESSION METHODS
  - Infoblox::Grid::Admin::AdAuthService->ad_auth_server_list()
- METHODS
  - comment( )
  - disabled( )
  - encryption( )
  - name( )
  - port( )
  - mgmt_port( )
- AUTHOR
- SEE ALSO
- COPYRIGHT

NAME

Infoblox::Grid::Admin::AdAuthServer - Active Directory Authentication Server object

DESCRIPTION

Represents an AD authentication server (i.e., domain controller) that is used to authenticate administrators.

CONSTRUCTOR

```
my $ad_auth_server = Infoblox::Grid::Admin::AdAuthServer->new(
    name        => $fqdn | $ipv4addr,    # Required
    port        => $num,                 # Required
    comment     => $string,              # Optional / Default is empty
    disabled    => "true" | "false",     # Optional / Default is "false"
    encryption  => "SSL" | "None",       # Optional / Default is "None"
    mgmt_port   => "true" | "false",     # Optional / Default is "false"
);
```

SESSION METHODS

This section describes all the methods in an Infoblox::Session module that you can apply to an object.

Infoblox::Grid::Admin::AdAuthService->ad_auth_server_list()

Use this method to specify AD authentication servers that are used to authenticate administrators.

Example

```
# Construct an object
my $ad_auth_server1 = Infoblox::Grid::Admin::AdAuthServer->new(
    name        => "1.2.3.4",
    port        => 636,
    comment     => "domain controller in room 1"
    disabled    => "false",
    encryption  => "SSL",
    mgmt_port   => "false",
);

# Construct another object
my $ad_auth_server2 = Infoblox::Grid::Admin::AdAuthServer->new(
    name        => "foo.com",
    port        => 389,
    comment     => "domain controller in room 2"
    disabled    => "false",
    encryption  => "None",
    mgmt_port   => "false",
);

# Configure AdAuthServer
my $response = $ad_auth_service->ad_auth_server_list([$ad_auth_server1, $ad_auth_server2]);
```

METHODS

Use the following methods to access the attributes of an Infoblox::Grid::Admin::AdAuthServer object. Specify a parameter to set the attribute's value, or omit the parameter to get the attribute's value.

comment( )

Use this method to set or retrieve comments about the server.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

Comments in string format with a maximum of 256 bytes.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get comment
my $val = $obj->comment();
#Modify comment
$obj->comment("head office");
```

disabled( )

Use this method to disable the Active Directory authentication service.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

Set the parameter to "true" to disable the service. Set the parameter to "false" to enable the service. The default value is "false".

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get disabled
my $val = $obj->disabled();
#Modify disabled
$obj->disabled("true");
```

encryption( )

Use this method to set or retrieve the encryption flag.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

Specify "SSL" to use SSL encryption or "None" to use no encryption.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get encryption
my $val = $obj->encryption();
#Modify encryption
$obj->encryption("SSL");
```

name( )

Use this method to set or retrieve the FQDN or IPv4 address of the AD authentication server.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

The FQDN or IPv4 address of the AD authentication server in string format.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get name
my $val = $obj->name();
#Modify name
$obj->name("1.2.3.5");
```

port( )

Use this method to set or retrieve the port number on the AD server to which the appliance sends authentication requests.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

TCP port of the AD authentication server

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get port
my $val = $obj->port();
#Modify port
$obj->port(389);
```

mgmt_port( )

Use this method to set or retrieve the MGMT port flag of the AD server. Set this flag to "true" to restrict the appliance to connecting to the AD server through the management port only.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

```
Specify "true" to set the mgmt_port flag or "false" to deactivate/unset it. The default value is
"false".
```

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get mgmt_port
my $val = $obj->mgmt_port();
#Modify mgmt_port
$obj->mgmt_port('false');
```

AUTHOR

Infoblox Inc. http://www.infoblox.com

SEE ALSO

Infoblox::Grid::Admin::AdAuthService, Infoblox::Session, Infoblox::Session->add(), Infoblox::Session->get(), Infoblox::Session->modify(), Infoblox::Session->remove()

COPYRIGHT

Copyright (c) 2017 Infoblox Inc.

Infoblox::Grid::Admin::Group - Admin Group object.

- NAME
- DESCRIPTION
- CONSTRUCTOR
- SESSION METHODS
  - Infoblox::Session->add( )
  - Infoblox::Session->get( )
  - Infoblox::Session->modify( )
  - Infoblox::Session->remove( )
  - Infoblox::Session->search( )
- METHODS
  - access_method( )
  - comment( )
  - disabled( )
  - enable_restricted_user_access( )
  - extattrs( )
  - extensible_attributes( )
  - email_addresses( )
  - name( )
  - superuser( )
  - roles( )
  - user_access( )
- SAMPLE CODE
- AUTHOR
- SEE ALSO
- COPYRIGHT

NAME

Infoblox::Grid::Admin::Group - Admin Group object.

DESCRIPTION

An Admin Group object creates and manages a local admin group on the Infoblox appliance. The privileges and properties that are set for the group apply to all the admin accounts that are assigned to the group.

CONSTRUCTOR

```
 my $group = Infoblox::Grid::Admin::Group->new(
     name                          => $string,
# Required
     access_method                 => ["GUI"|"API"|"TAXII",...],
# Optional / Default is ["GUI", "API", "TAXII"]
     comment                       => $string,
# Optional / Default is undefined
     disabled                      => "true" | "false",
# Optional / Default is "false"
     enable_restricted_user_access => "true" | "false",
# Optional / Default is "false"
     extattrs                      => { $string => $extattr, ... },
# Optional / Default is undefined
     extensible_attributes         => { $string => $string | $num, $string => [ $string | $num, ... ], ...
} # Optional / Default is undefined
     email_addresses               => [ 'admin@test.com',...],
# Optional / Default is undefined
     superuser                     => "true" | "false",
# Optional / Default is "false"
     roles                         => [$string, ...]
# Optional / Default is an empty array reference
     user_access                   => [$ipaddr, $netaddr, ...]
# Optional / Default is empty list
 );
```

You cannot set both extattrs and extensible_attributes attributes at the same time.

SESSION METHODS

This section describes all the methods in the Infoblox::Session module that you can apply to an Admin Group object.

Infoblox::Session->add( )

Use this method to add an object to the Infoblox appliance. See Infoblox::Session->add() for parameters and return values.

Example

```
#Construct the Group object
my $group = Infoblox::Grid::Admin::Group->new(
    name                => "testgroup",
    access_method       => ["GUI", "API"],
    comment             => "test group",
    disabled            => "true",
    superuser           => "false"
);
# Submit for addition
my $response = $session->add( $group );
```

Infoblox::Session->get( )

Use this method to retrieve all the matching objects from the Infoblox appliance. See Infoblox::Session->get() for parameters and return values.

Key References

```
Apply the following attributes to get a specific Group object:

 name      - Required. Group name in string format.
 extensible_attributes - Optional. A hash reference containing extensible attributes.
 extattrs  - Optional. A hash reference containing extensible attributes.
```

Example

```
my @retrieved_objs = $session->get(
    object     => "Infoblox::Grid::Admin::Group",
    name       => "testgroup" );
```

Infoblox::Session->modify( )

Use this method to modify an object in the Infoblox appliance. See Infoblox::Session->modify() for parameters and return values.

Example

```
# Use this method to modify the comment.
$group->comment("This is a modified comment");
# Submit modification
my $response = $session->modify( $group );
```

Infoblox::Session->remove( )

Use this method to remove an object from the Infoblox appliance. See Infoblox::Session->remove() for parameters and return values.

To remove a specific object, use `get()` or `search()` to retrieve the specific object first, and then submit this object for removal.

Example

```
# Get the group objects with the same starting name
my @retrieved_objs = $session->get(
    object     => "Infoblox::Grid::Admin::Group",
    name       => "testgroup" );
# find the desired object from the retrieved list.
my $desired_group = $retrieved_objs[0];
# Submit for removal
my $response = $session->remove( $desired_group );
```

Infoblox::Session->search( )

Use this method to search for Admin Group objects in the Infoblox appliance. See Infoblox::Session->search() for parameters and return values.

Key References

```
Apply the following attributes to search for a specific Group object:

 name                            - Optional. Group name in string format (regular expression).
 roles                           - Optional. An array of role names in string format.
 ca_certificate_issuer           - Optional. The name of the CA certificate issuer in string
format.
 client_certificate_serial_number - Optional. The serial number of the CA certificate in string
format. This is unique for a particular issuer.
 extensible_attributes           - Optional. A hash reference containing extensible attributes.
 extattrs                        - Optional. A hash reference containing extensible attributes.
```

For more information about searching extensible attributes, see Infoblox::Grid::ExtensibleAttributeDef/Searching Extensible Attributes.

Example

```
# search for all Group objects that start with "test"
my @retrieved_objs = $session->search(
    object => "Infoblox::Grid::Admin::Group",
    name   => "test.*" );

# search for all Group objects that start with "test"
my @retrieved_objs = $session->search(
    object => "Infoblox::Grid::Admin::Group",
    roles   => ["DHCP Admin", "AAA Admin"]);
```

METHODS

This section describes all the methods that you can use to set and retrieve the attribute values of a Group object.

access_method( )

Use this method to set or retrieve a group's access methods. Access methods specify whether a group can use the GUI and the API to access the appliance or to send Taxii messages to the appliance. Valid value is an array reference that contains the literal string "GUI", "TAXII" and/or "API". The default value is ["GUI", "API", "TAXII"].

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

Array reference of the literal string "GUI", "TAXII" and/or "API". It indicates whether a group has permission to access the appliance through the GUI and/or API and/or to send Taxii messages to the appliance.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get access methods
my $ref_access_method = $group->access_method();
#Modify access methods
$group->access_method(["GUI"]);
```

comment( )

Use this method to set or retrieve the comment.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

Pertinent information about the admin group, such as location or department. Comment in string format with a maximum of 256 bytes. The default value is undefined.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get comment
my $comment = $group->comment();
#Modify comment
$group->comment("This is the modified comment for admin group");
```

disabled( )

Use this method to set or retrieve the "disabled" flag.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

The default value for this field is false. The admin group is enabled.

Parameter

Specify "true" to set the disable flag or "false" to deactivate/unset it.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get disabled
my $disabled = $group->disabled();
#Modify disabled
$group->disabled("true");
```

enable_restricted_user_access( )

Use this method to set or retrieve the flag that indicates whether the restrictions will be applied on an admin connector level for a given user or not.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

Specify "true" to enable restricted access on admin connector level and "false" to disable restrictions. The default value is "false".

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get enable_restricted_user_access
my $enable_restricted_user_access = $group->enable_restricted_user_access();

#Modify enable_restricted_user_access
$group->enable_restricted_user_access("true");
```

extattrs( )

Use this method to set or retrieve the extensible attributes associated with a Group object.

Parameter

Valid value is a hash reference containing the names of extensible attributes and their associated values (Infoblox::Grid::Extattr objects).

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get extattrs
my $ref_extattrs = $group->extattrs();
#Modify extattrs
$group->extattrs({ 'Site' => $extattr1, 'Administrator' => $extattr2 });
```

extensible_attributes( )

Use this method to set or retrieve the extensible attributes associated with a Group object.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

For valid values for extensible attributes, see Infoblox::Grid::ExtensibleAttributeDef/Extensible Attribute Values.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get extensible attributes
my $ref_extensible_attributes = $group->extensible_attributes();
#Modify extensible attributes
$group->extensible_attributes({ 'Site' => 'Santa Clara', 'Administrator' => [ 'Peter', 'Tom' ] });
```

email_addresses( )

Use this method to set or retrieve the email addresses for the Admin Group.

Parameter

The valid value is an array reference that contains emails in string format.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get email_addresses
my @email_addresses = $rgroup->email_addresses();
#Modify addresses
$group->addresses( [ 'admin@test.com' ] );
```

name( )

Use this method to set or retrieve the name of an admin group.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

Text with the name of the admin group.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get name
my $name = $group->name();
```

superuser( )

Use this method to set or retrieve the "superuser" flag. A superuser group can perform all operations on the appliance, and can view and configure all types of data.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

The default value for this field is "false" which means that this is not a superuser admin group. The admins in the group have limited privileges.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get super user flag
my $disabled = $group->superuser();
#Modify super user
$group->superuser("true");
```

roles( )

Use this method to set or retrieve the role names associated with this admin group.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

The default value for this field is an empty array. A valid value is an array reference with literal strings containing valid role names. The corresponding role objects must exist in the database.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get roles
my $ref_roles = $group->roles();
#Modify roles
$group->roles(["role1", "role2"]);
#Remove roles
$group->roles([]);
```

user_access( )

Use this method to set or retrieve the access control items for a given group.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

Valid value is an Infoblox::Grid::NamedACL object or an array reference that contains IPv4/IPv6 addresses or networks.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get user_access
my $user_access = $group->user_access();

#Modify user_access
$group->user_access(['10.0.0.1', '2001:db8::/64']);
```

SAMPLE CODE

The following sample code demonstrates the different functions that can be applied to an object, such as add, search, modify, and remove. This sample also includes error handling for the operations.

#Preparation prior to an Admin Group object insertion

```
#PROGRAM STARTS: Include all the modules that will be used
use strict;
use Infoblox;

#Create a session to the Infoblox appliance

my $session = Infoblox::Session->new(
               master   => "192.168.1.2", #appliance host ip
               username => "admin",       #appliance user login
               password => "infoblox"     #appliance password
               );

unless ($session) {
       die("Construct session failed: ",
               Infoblox::status_code() . ":" . Infoblox::status_detail());
}
print "Session created successfully\n";

my $role = Infoblox::Grid::Admin::Role->new(
    name                 => "testrole",
    comment              => "test admin role",
);

unless ($role) {
       die("Construct role object failed: ",
```

```
                Infoblox::status_code() . ":" . Infoblox::status_detail());
}
print "Admin Role object created successfully\n";

#Add the Admin Role object to the Infoblox appliance through a session
$session->add($role)
       or die("Add Admin Role object failed: ",
                        $session->status_code() . ":" . $session->status_detail());
print "Admin Role object added to server successfully\n";
```

#Create an Admin Group object

```
my $group = Infoblox::Grid::Admin::Group->new(
    name                => "testgroup",
    comment             => "test admin group",
    disabled            => "false",
    superuser           => "false",
    access_method       => ["GUI", "API"],
    roles               => ["testrole"]
);

unless ($group) {
       die("Construct group object failed: ",
                Infoblox::status_code() . ":" . Infoblox::status_detail());
}
print "Admin Group object created successfully\n";

#Add the Admin Group object to the Infoblox appliance through a session
$session->add($group)
       or die("Add Admin Group object failed: ",
                        $session->status_code() . ":" . $session->status_detail());
print "Admin Group object added to server successfully\n";
```

#Search for an Admin Group

```
my @retrieved_objs = $session->search(
               object => "Infoblox::Grid::Admin::Group",
               name   => "test.*"
               );
my $object = $retrieved_objs[0];

unless ($object) {
       die("Search Admin Group object failed: ",
                $session->status_code() . ":" . $session->status_detail());
}
print "Search Admin Group object found at least 1 matching entry\n";
```

#Get and modify an Admin Group object

```
#Get the Admin Group object from Infoblox appliance through a session
my @retrieved_objs = $session->get(
    object => "Infoblox::Grid::Admin::Group",
    name   => "testgroup"
);
my $object = $retrieved_objs[0];

unless ($object) {
       die("Get Admin Group object failed: ",
                $session->status_code() . ":" . $session->status_detail());
}
print "Get Admin Group object found at least 1 matching entry\n";

#Modify the Admin Group object
$object->name("testgroup_renamed");
$object->access_method(["API"]);
$object->comment("modified test group");
$object->disabled("false");
$object->superuser("false"); # no real change

#Apply the changes.
$session->modify($object)
       or die("Modify Admin Group object failed: ",
                $session->status_code() . ":" . $session->status_detail());
print "Admin Group object modified successfully \n";
```

#Remove an Admin Group object

```
#Get the Admin Group object through the session
my @retrieved_objs = $session->get(
    object => "Infoblox::Grid::Admin::Group",
    name   => "testgroup_renamed"
);
my $object = $retrieved_objs[0];
unless ($object) {
    die("Get Admin Group object failed: ",
        $session->status_code() . ":" . $session->status_detail());
}
print "Get Admin Group object found at least 1 matching entry\n";

#Submit the object for removal
$session->remove($object)
       or die("Remove Admin Group object failed: ",
                $session->status_code() . ":" . $session->status_detail());
print "Admin Group object removed successfully \n";
```

#Clean up the Admin Role object

```
$session->remove($role)
       or die("Remove Admin Role object failed: ",
                $session->status_code() . ":" . $session->status_detail());
print "Admin Role object removed successfully \n";

####PROGRAM ENDS####
```

AUTHOR

Infoblox Inc. http://www.infoblox.com/

SEE ALSO

Infoblox::Grid::Admin::Role, Infoblox::Grid::Admin::User, Infoblox::Session->add(), Infoblox::Session->get(), Infoblox::Session->modify(), Infoblox::Session->remove(), Infoblox::Session->search(),Infoblox::Session

---

COPYRIGHT

Copyright (c) 2017 Infoblox Inc.

Infoblox::Grid::Admin::LocalUserAuthService - Local User Authentication Service object

- NAME
- DESCRIPTION
- MODULE METHODS
  - Infoblox::Grid::Admin::AuthPolicy->auth_services( )
- SESSION METHODS
  - Infoblox::Session->get( )
- METHODS
  - comment( )
  - disabled( )
  - name( )
- AUTHOR
- SEE ALSO
- COPYRIGHT

NAME

Infoblox::Grid::Admin::LocalUserAuthService - Local User Authentication Service object

DESCRIPTION

This object represents a local authentication service for authenticating users against the local database. All methods are read-only.

MODULE METHODS

The following functions can be applied to an Infoblox::Grid::Admin::LocalUserAuthService object.

Infoblox::Grid::Admin::AuthPolicy->auth_services( )

Use this function to set or retrieve a reference to an array of authentication services.

Example

```
#Retrieve the local user authentication service
my $local = $session->get(object => "Infoblox::Grid::Admin::LocalUserAuthService");

#Set it as the authentication policy
$auth_policy->auth_services([$local]);
```

SESSION METHODS

This section describes all the methods in the Infoblox::Session module that you can apply to a Local User Authentication Service object.

Infoblox::Session->get( )

Use this method to retrieve the local user authentication service from the Infoblox appliance. See Infoblox::Session->get() for parameters and return values.

Key Reference

No parameters are required.

Example

```
my $local_user_auth_service = $session->get(
    object => "Infoblox::Grid::Admin::LocalUserAuthService",
    );
```

METHODS

This section describes all the methods that you can use to retrieve the attribute values of a local user authentication service.

comment( )

Use this method to retrieve the descriptive comment.

Returns

The method returns the attribute value.

Example

```
#Get comment
my $comment = $local_user_auth_service->comment();
```

disabled( )

Use this method to retrieve the disabled flag for the local user authentication service.

Returns

The method returns the attribute value.

Example

```
#Get disabled
my $disabled = $local_user_auth_service->disabled();
```

name( )

Use this method to retrieve the name of the local user authentication service.

Returns

The method returns the attribute value.

Example

```
#Get name
my $name = $local_user_auth_service->name();
```

AUTHOR

Infoblox Inc. http://www.infoblox.com/

SEE ALSO

Infoblox::Grid::Admin::AuthPolicy/>, Infoblox::Session->get()

COPYRIGHT

Copyright (c) 2017 Infoblox Inc.

Infoblox::Grid::Admin::TACACSPlusAuthService - TACACS+ Authentication Service object

- NAME
- DESCRIPTION
- CONSTRUCTOR
- SESSION METHODS
  - Infoblox::Session->add( )
  - Infoblox::Session->get( )
  - Infoblox::Session->modify( )
  - Infoblox::Session->remove( )
- METHODS
  - acct_retries( )
  - acct_timeout( )
  - auth_retries( )
  - auth_timeout( )
  - comment( )
  - disabled( )
  - name( )
  - tacacsplus_servers( )
- SAMPLE CODE
- AUTHOR
- SEE ALSO
- COPYRIGHT

NAME

Infoblox::Grid::Admin::TACACSPlusAuthService - TACACS+ Authentication Service object

---

DESCRIPTION

Represents a group of TACACS+ servers that are used to authenticate administrators.

CONSTRUCTOR

```
my $tacacs_auth_server = Infoblox::Grid::Admin::TACACSPlusAuthService->new(
    name                => $string,                             #Required
    tacacsplus_servers  => [$TACACSPlusAuthServer1, ...],       #Required
    acct_retries        => $num,                                #Optional / Default is 0
    acct_timeout        => $num,                                #Optional / Default is 1000
    auth_retries        => $num,                                #Optional / Default is 0
    auth_timeout        => $num,                                #Optional / Default is 5000
    comment             => $string,                             #Optional / Default is empty
    disabled            => "true" | "false"                     #Optional / Default is false
);
```

SESSION METHODS

This section describes all the methods in the Infoblox::Session module that you can apply to a TACACS+ Authentication Service object.

Infoblox::Session->add( )

Use this method to add an object to the Infoblox appliance. See Infoblox::Session->add() for parameters and return values.

Example

```
#Construct an object
my $service = Infoblox::Grid::Admin::TACACSPlusAuthService->new(
                                                    name           => 'some.name.com',
                                                    tacacsplus_servers => [$tcs1, $tcs2]
                                                   );
#Submit for addition
my $response = $session->add( $service );
```

Infoblox::Session->get( )

Use this method to retrieve all the matching objects from the Infoblox appliance. See Infoblox::Session->get() for parameters and return values.

Key References

```
Apply the following attributes to get a specific TACACS+ Authentication Service object:

 name - Required. TACACS+ Authentication Service name.
```

Examples

```
my @retrieved_objs = $session->get(
    object => "Infoblox::Grid::Admin::TACACSPlusAuthService",
    name   => "some.name.com"
);
```

Infoblox::Session->modify( )

Use this method to modify an object in the Infoblox appliance. See Infoblox::Session->modify() for parameters and return values.

Example

```
#Use this method to modify the comment.
$service->comment("this is a modified comment");
#Submit modification
my $response = $session->modify( $service );
```

Infoblox::Session->remove( )

Use this method to remove an object from the Infoblox appliance. See Infoblox::Session->remove() for parameters and return values.

To remove a specific object, first use `get()` or `search()` to retrieve the specific object, and then submit this object for removal.

Example

```
#Get the objects with the same name
my @retrieved_objs = $session->get(
    object => "Infoblox::Grid::Admin::TACACSPlusAuthService",
    name   => "some.name.com"
);

#Find the desired object on the retrieved list.
my $desired_service = $retrieved_objs[0];
#Submit for removal
my $response = $session->remove( $desired_service );
```

METHODS

Use the following methods to access the attributes of an Infoblox::Grid::Admin::TACACSPlusAuthService object. Specify a parameter to set the attribute's value, or omit the parameter to get the attribute's value.

acct_retries( )

Use this method to set or retrieve the value that specifies how many times the appliance must resend the TACACS+ accounting packet if the appliance does not receive a response.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

An unsigned integer between 0 and 5, inclusive.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Getting acct_retries
my $acct_retries = $tacacsplus_service->acct_retries( );
#Modifying acct_retries
$tacacsplus_service->acct_retries("5");
```

acct_timeout( )

Use this method to set or retrieve the value, in milliseconds, that specifies how long the appliance must wait for accounting response packets.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

An unsigned integer between 1000 and 30000, inclusive.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get acct_timeout
my $acct_timeout = $tacacsplus_service->acct_timeout( );
#Modify acct_timeout
$tacacsplus_service->acct_timeout("5000");
```

auth_retries( )

Use this method to set or retrieve the number of times NIOS retries authentication and authorization if there is no response or a server error occurrs.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

An unsigned integer between 0 and 5, inclusive.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get auth_retries
my $auth_retries = $tacacsplus_service->auth_retries( );
#Modify auth_retries
$tacacsplus_service->auth_retries("5");
```

auth_timeout( )

Use this method to set or retrieve the number of milliseconds NIOS waits to complete admin authentication and authorization by the TACACS+ server.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

An unsigned integer between 5000 and 60000, inclusive.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get auth_timeout
my $auth_timeout = $tacacsplus_service->auth_timeout( );
#Modify auth_timeout
$tacacsplus_service->auth_timeout("5000");
```

comment( )

Use this method to set or retrieve a descriptive comment about the TACACS+ Authentication Service object.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

The comment in string format, with a maximum of 256 characters.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get comment
my $comment = $tacacsplus_service->comment( );
#Modify comment
$tacacsplus_service->comment("TACACS+ Authentication Service for NAC authentication");
```

disabled( )

Use this method to set or retrieve the disable flag of the TACACS+ Authentication Service object.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

Specify "true" to set the disable flag or "false" to deactivate/unset it. The default value is "false".

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get disable
my $disable = $tacacsplus_service->disabled( );
#Modify disabled
$tacacsplus_service->disabled("true");
```

name( )

Use this method to set or retrieve the name of the TACACS+ Authentication Service object.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

The name in string format, with a maximum of 64 characters.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get name
my $name = $tacacsplus_service->name( );
#Modify name
$tacacsplus_service->name("TACACS+ Authentication Service");
```

tacacsplus_servers( )

Use this method to set or retrieve a list of TACACS+ servers assigned to this TACACS+ service.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

An array reference that contains a list of Infoblox::Grid::Admin::TACACSPlusAuthServer objects.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get tacacsplus_servers
my $tacacsplus_servers = $tacacsplus_service->tacacsplus_servers( );
#Modify tacacsplus_servers
$tacacsplus_service->tacacsplus_servers([$tacacsplus_server1]);
```

SAMPLE CODE

The following sample code demonstrates the different functions that can be applied to an object, such as add, search, modify, and remove. This sample also includes error handling for the operations.

#Preparation prior to a TACACS+ Authentication Service object insertion

```
#PROGRAM STARTS: Include all the modules that will be used
use strict;
use Infoblox;

#Create a session to the Infoblox appliance
my $host_ip =  "192.168.1.2";
my $session = Infoblox::Session->new(
    master   => $host_ip,
    username => "admin",
    password => "infoblox"
    );
unless($session){
        die("Constructor for session failed: ",
               Infoblox::status_code(). ":" . Infoblox::status_detail());
}
print "Session created successfully.\n";

my $tcs1 = Infoblox::Grid::Admin::TACACSPlusAuthServer->new(
                                                        comment  => 'server 1',
                                                        fqdn_or_ip => '10.0.1.1',
                                                        shared_secret => 'secret1',

auth_type => 'ASCII',
                                                      );

my $tcs2 = Infoblox::Grid::Admin::TACACSPlusAuthServer->new(
                                                        comment  => 'server 2',
                                                        fqdn_or_ip => 'domain.com',
                                                        shared_secret => 'secret2',

auth_type => 'ASCII',
                                                     );
```

#Create the TACACS+ Authentication Service.

```
my $service = Infoblox::Grid::Admin::TACACSPlusAuthService->new(
                                                            acct_retries   => 1,
                                                            auth_retries   => 1,
                                                            disabled       => 'true',
                                                            acct_timeout   => 1100,
                                                            auth_timeout   => 5000,
                                                            name           => 'some.name.com',
                                                            tacacsplus_servers => [$tcs1, $tcs2]
                                                          );

unless($service){
       die("Construct service object failed: ",
               Infoblox::status_code(). ":" .Infoblox::status_detail());
       }
print "Service object created successfully.\n";

$session->add($service)
    or die("Add TACACS+ service failed: ",
               $session->status_code(). ":" .$session->status_detail());

print"Service added successfully.\n";
```

#Get a specific service.

```
my @search_result = $session->get(
    object => "Infoblox::Grid::Admin::TACACSPlusAuthService",
    name   => "some.name.com"
);

my $search_obj = $search_result[0];
unless($search_obj){
       die("Search service failed: ",
                $session->status_code() . ":" . $session->status_detail());
       }
```

```
print "Search service object found at least 1 matching entry\n";
```

#Get and modify a TACACS+ service.

```
$search_obj->comment("this is a modified comment.");

#Apply the change
    $session->modify($search_obj)
        or die("Modify service failed: ",
               $session->status_code(). ":" .$session->status_detail());

 print "Service object modified successfully.\n";
```

#Remove a service.

```
my @search_result = $session->get(
    object => "Infoblox::Grid::Admin::TACACSPlusAuthService",
    name   => "some.name.com"
);

my $search_obj = $search_result[0];
unless($search_obj){
       die("Search service failed: ",
                $session->status_code() . ":" . $session->status_detail());
       }

print "Search service object found at least 1 matching entry\n";

$session->remove($search_obj)
    or die("Remove service failed: ",
            $session->status_code(). ":" .$session->status_detail());

print "Remove service successfull\n";

####PROGRAM ENDS####
```

AUTHOR

Infoblox Inc. http://www.infoblox.com

SEE ALSO

Infoblox::Grid::Admin::TACACSPlusAuthServer, Infoblox::Session, Infoblox::Session->add(), Infoblox::Session->get(), Infoblox::Session->modify(), Infoblox::Session->remove()

COPYRIGHT

Copyright (c) 2017 Infoblox Inc.

Infoblox::LDAP::AuthService - An LDAP Authentication Service object.

- NAME
- DESCRIPTION
- CONSTRUCTOR
- SESSION METHODS
  - Infoblox::Session->add( )
  - Infoblox::Session->get( )
  - Infoblox::Session->modify( )
  - Infoblox::Session->remove( )
  - Infoblox::Session->search( )
- METHODS
  - comment( )
  - disable( )
  - ea_mapping( )
  - mode( )
  - name( )
  - ldap_group_attribute( )
  - ldap_group_authentication_type( )
  - ldap_user_attribute( )
  - recovery_interval( )
  - retries( )
  - search_scope( )
  - servers( )
  - timeout( )
- SAMPLE CODE
- AUTHOR
- SEE ALSO
- COPYRIGHT

NAME

Infoblox::LDAP::AuthService - An LDAP Authentication Service object.

DESCRIPTION

The LDAP Authentication Service.

CONSTRUCTOR

```
 my $service = Infoblox::LDAP::AuthService->new(
     name                          => $string,                                  # Required
     servers                       => [$server1, $server2, ...],                # Required, These are
Infoblox::LDAP::Server objects
     timeout                       => $uint,                                    # Optional, Default is 5
     retries                       => $uint,                                    # Optional, Default is 5
     recovery_interval             => $uint,                                    # Optional, Default is 30
     mode                          => 'ORDERED_LIST'| 'ROUND_ROBIN',            # Optional, Default is
'ORDERED_LIST'
     ldap_user_attribute           => $string,                                  # Required
     ldap_group_attribute          => $string,                                  # Optional, Default is 'memberOf'
     search_scope                  => BASE' | 'ONELEVEL' | 'SUBTREE',          # Optional, Default is 'ONELEVEL'
     ea_mapping                    => [$ea_mapping1, $ea_mapping, ...],         # Optional, These are
Infoblox::LDAP::EA_Mapping objects
     comment                       => $string,                                  # Optional
     disable                       => 'true' | 'false',                         # Optional, Default is 'false'
     ldap_group_authentication_type => 'GROUP_ATTRIBUTE' | 'POSIX_GROUP',       # Optional, Default is
'GROUP_ATTRIBUTE'
 );
```

SESSION METHODS

This section describes all the methods in an Infoblox::Session module that you can apply to an LDAP Authentication Service object.

Infoblox::Session->add( )

Use this method to add an object to the Infoblox appliance. See Infoblox::Session->add() for parameters and return values.

Example

```
#Construct an object
my $service = Infoblox::LDAP::AuthService->new(
   name                => 'LDAPAuthService',
   servers             => [$server],
   ldap_user_attribute => 'cn',
);

#Submit for addition
my $response = $session->add( $service );
```

Infoblox::Session->get( )

Use this method to retrieve all the matching objects from the Infoblox appliance. See Infoblox::Session->get() for parameters and return values.

Key References

```
Apply the following attributes to get a specific LDAP Authentication Service object:

 name         - Optional. The LDAP Authentication Service name.
 comment      - Optional. The comment.
 search_scope - Optional. The starting point of the LDAP search.
 mode         - Optional. The LDAP authentication mode.
```

Example

```
my @retrieved_objs = $session->get(
   object => 'Infoblox::LDAP::AuthService',
   name   => 'LDAPAuthService',
);
```

Infoblox::Session->modify( )

Use this method to modify an object in the Infoblox appliance. See Infoblox::Session->modify() for parameters and return values.

Example

```
# Use this method to modify comment of a LDAP Authentication Service.
$service->comment("This is modified object");
# Submit modification
my $response = $session->modify( $service );
```

Infoblox::Session->remove( )

Use this method to remove an object from the Infoblox appliance. See Infoblox::Session->remove() for parameters and return values.

To remove a specific object, first use `get()` or `search()` to retrieve the specific object, and then submit this object for removal.

Example

```
#Get the objects with the same name
my @retrieved_objs = $session->get(
   object => 'Infoblox::LDAP::AuthService',
   name   => 'LDAPAuthService',
)
#Find the desired object from the retrieved list.
my $desired_device = $retrieved_objs[0];
# Submit for removal
my $response = $session->remove( $desired_device );
```

Infoblox::Session->search( )

Use this method to search for device objects in the Infoblox appliance. See Infoblox::Session->search() for parameters and return values.

Key References

```
Apply the following attributes to search a specific LDAP Authentication Service object:

 name         - Optional. The LDAP Authentication Service name.
 comment      - Optional. The comment.
 search_scope - Optional. The starting point of the LDAP search.
 mode         - Optional. The LDAP authentication mode.
```

Example

```
my @retrieved_objs = $session->search(
   object => 'Infoblox::LDAP::AuthService',
```

```
    name    => 'LDAPAuthService',
);
```

METHODS

This section describes all the methods that you can use to configure and retrieve the attribute values of an LDAP Authentication Service object.

comment( )

Use this method to set or retrieve a descriptive comment.

Parameter

A desired comment in string format with a maximum of 256 bytes.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get attribute value
my $value = $service->comment();
#Modify attribute value
$service->comment('LDAP Authentication Service');
```

disable( )

Use this method to enable or disable an LDAP Authentication Service.

Parameter

Specify "true" to disable an LDAP Authentication Service or "false" to enable it. The default value is 'false'.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get attribute value
my $value = $service->disable();
#Modify attribute value
$service->disable('true');
```

ea_mapping( )

Use this method to set or retrieve mapping of LDAP fields to extensible attributes.

Parameter

An array of an Infoblox::LDAP::EA_Mapping objects.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get attribute value
my $value = $service->ea_mapping();
#Modify attribute value
$service->ea_mapping([$ea_mapping1, $ea_mapping2]);
```

mode( )

Use this method to set or retrieve an LDAP authentication mode.

Parameter

Valid values are 'ORDERED_LIST' and 'ROUND_ROBIN'. The default value is 'ORDERED_LIST'.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get attribute value
my $value = $service->mode();
#Modify attribute value
$service->mode('ROUND_ROBIN');
```

name( )

Use this method to set or retrieve an LDAP Authentication Service name.

Parameter

An LDAP Authentication Service name in string format.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get attribute value
my $value = $service->name();
#Modify attribute value
$service->name('NewLDAPAuthService');
```

ldap_group_attribute( )

Use this method to set or retrieve the name of the LDAP attribute that defines group membership.

Parameter

A name of the LDAP attribute in string format. The default value is 'memberOf'.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get attribute value
my $value = $service->ldap_group_attribute();
#Modify attribute value
$service->ldap_group_attribute('memberOf');
```

ldap_group_authentication_type( )

Use this method to set or retrieve the LDAP group authentication type.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

The valid values are 'GROUP_ATTRIBUTE' and 'POSIX_GROUP'.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get ldap_group_authentication_type value
my $value = $service->ldap_group_authentication_type();
#Modify ldap_group_authentication_type value
$service->ldap_group_authentication_type('POSIX_GROUP');
```

ldap_user_attribute( )

Use this method to set or retrieve an LDAP userid attribute that is used for search.

Parameter

An LDAP userid attribute in string format.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get attribute value
my $value = $service->ldap_user_attribute();
#Modify attribute value
$service->ldap_user_attribute('cn');
```

recovery_interval( )

Use this method to set or retrieve the period of time to wait before trying to contact an LDAP server that has been marked as 'down'.

Parameter

The period of time in seconds. The default value is 30.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get attribute value
my $value = $service->recovery_interval();
#Modify attribute value
$service->recovery_interval(40);
```

retries( )

Use this method to set or retrieve maximum number of authentication attempts.

Parameter

Maximum number of authentication attempts. The default value is 5.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get attribute value
my $value = $service->retries();
#Modify attribute value
$service->retries(4);
```

search_scope( )

Use this method to set or retrieve the starting point of an LDAP search.

Parameter

Valid values are 'BASE', 'ONELEVEL' and 'SUBTREE'. The default value is 'ONELEVEL'.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get attribute value
my $value = $service->search_scope();
#Modify attribute value
$service->search_scope('ONELEVEL');
```

servers( )

Use this method to set or retrieve the list of LDAP servers used for authentication.

Parameter

An array of an Infoblox::LDAP::Server objects.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get attribute value
my $value = $service->servers();
#Modify attribute value
$service->servers([$server1, $server2]);
```

timeout( )

Use this method to set or retrieve timeout for authentication.

Parameter

Timout in seconds. The default value is 5.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get attribute value
my $value = $service->timeout();
#Modify attribute value
$service->timeout(4);
```

SAMPLE CODE

The following sample code demonstrates the different functions that can be applied to an object such as add, get, modify, search and remove. This sample also includes error handling for the operations.

#Preparation prior to getting and modifying a LDAP Authentication Service object

```
use strict;
use Infoblox;

#refers to Infoblox Appliance IP address
my $host_ip = "192.168.1.2";

#Create a session to the Infoblox appliance

my $session = Infoblox::Session->new(
    master   => $host_ip,
    username => "admin",
    password => "infoblox"
);
unless ($session) {
       die("Construct session failed: ",
               Infoblox::status_code() . ":" . Infoblox::status_detail());
}
print "Session created successfully\n";
```

#Create a LDAP Authentication Service object

```
my $server = Infoblox::LDAP::Server->new(
    'address' => '10.0.0.1',
```

```
    'base_dn' => 'dc=infoblox,dc=com',
);

unless ($server) {
       die("Construct LDAP server object failed: ",
               Infoblox::status_code() . ":" . Infoblox::status_detail());
}
print "LDAP server object created successfully\n";

my $service = Infoblox::LDAP::AuthService->new(
    'name'                => 'papi-ldap-service',
    'servers'             => [$server],
    'ldap_user_attribute' => 'cn',
);

unless ($service) {
       die("Construct LDAP Authentication Service object failed: ",
               Infoblox::status_code() . ":" . Infoblox::status_detail());
}
print "LDAP Authentication Service object created successfully\n";

my $response = $session->add($service);

unless ($response) {
    die("Add LDAP Authentication Service failed: ",
          $session->status_code() . ":" . $session->status_detail());
}
print "LDAP Authentication Service added successfully \n";
```

#Search for a LDAP Authentication Service object

```
my @retrieved_objs = $session->search(
   object => 'Infoblox::LDAP::AuthService',
   name   => 'papi-ldap-service',
);

$service = $retrieved_objs[0];

unless ($service) {
        die("Search LDAP Authentication Service object failed: ",
                $session->status_code() . ":" . $session->status_detail());
}
print "Search LDAP Authentication Service object found at least 1 matching entry\n";
```

#Get and modify the LDAP Authentication Service object

```
@retrieved_objs = $session->get(
   object => 'Infoblox::LDAP::AuthService',
   name   => 'papi-ldap-service',
);

$service = $retrieved_objs[0];

unless ($service) {
    die("Get LDAP Authentication Service object failed: ",
          $session->status_code() . ":" . $session->status_detail());
}
print"Get LDAP Authentication Service successful \n";

$service->comment("This is modified object");

$session->modify($service)
   or die("Modify LDAP Authentication Service failed",
           $session->status_code() . ":" . $session->status_detail());
print "LDAP Authentication Service modified successfully \n";
```

#Remove the LDAP Authentication Service object

```
@retrieved_objs = $session->get(
   object => 'Infoblox::LDAP::AuthService',
   name   => 'papi-ldap-service',
);

$service = $retrieved_objs[0];

unless ($service) {
```

```
    die("Get LDAP Authentication Service object failed: ",
            $session->status_code() . ":" . $session->status_detail());
}
print"Get LDAP Authentication Service successful \n";

$session->remove( $service )
    or die("Remove LDAP Authentication Service failed",
            $session->status_code() . ":" . $session->status_detail());
print "LDAP Authentication Service removed successfully \n";

####PROGRAM ENDS####
```

AUTHOR

Infoblox Inc. http://www.infoblox.com/

SEE ALSO

Infoblox::LDAP::Server, Infoblox::LDAP::EA_Mapping

COPYRIGHT

Copyright (c) 2017 Infoblox Inc.

Infoblox::Grid::Admin::CertificateAuthService - a Certificate Authentication Service.

- NAME
- DESCRIPTION
- CONSTRUCTOR
- SESSION METHODS
  - Infoblox::Session->add( )
  - Infoblox::Session->get( )
  - Infoblox::Session->modify( )
  - Infoblox::Session->remove( )
  - Infoblox::Session->search( )
- METHODS
  - auto_populate_login( )
  - ca_certificates( )
  - comment( )
  - disabled( )
  - enable_password_request( )
  - enable_remote_lookup( )
  - max_retries( )
  - name( )
  - ocsp_check( )
  - ocsp_responders( )
  - recovery_interval( )
  - remote_lookup_password( )
  - remote_lookup_service( )
  - remote_lookup_username( )
  - response_timeout( )
  - trust_model( )
  - user_match_type( )
- SAMPLE CODE
- AUTHOR
- SEE ALSO
- COPYRIGHT

NAME

Infoblox::Grid::Admin::CertificateAuthService - a Certificate Authentication Service.

DESCRIPTION

An Infoblox::Grid::Admin::CertificateAuthService object represents a Certificate Authentication Service.

CONSTRUCTOR

```
 my $auth_service = Infoblox::Grid::Admin::CertificateAuthService->new(
     name                  => $string,                                  # Required
     ca_certificates       => [$certificate1, ...],                     # Required
     ocsp_responders       => [$responder1, ...],                       # Required unless ocsp_check is "DISABLED"
or "AIA_ONLY"
     disabled              => "true" | "false",                         # Optional / Default value is "false"
     comment               => $string,                                  # Optional / Default value is empty
     trust_model           => "DIRECT" | "DELEGATED",                   # Optional / Default value is "DIRECT"
     recovery_interval     => $uint,                                    # Optional / Default value is 30
     max_retries           => $uint,                                    # Optional / Default value is 0
     response_timeout      => $uint,                                    # Optional / Default value is 1000
     auto_populate_login   => "SERIAL_NUMBER" | "S_DN_CN" | "S_DN_EMAIL" | "SAN_UPN" | "SAN_EMAIL" |
"AD_SUBJECT_ISSUER" # Optional / Default value is "S_DN_CN"
     enable_password_request => "true" | "false",                       # Optional / Default value is "true"
     user_match_type       => "DIRECT_MATCH" | "AUTO_MATCH",            # Optional / Default value is "AUTO_MATCH"
     ocsp_check            => "MANUAL" | "AIA_ONLY" | "AIA_AND_MANUAL" | "DISABLED", # Optional / Default
value is "MANUAL"
     enable_remote_lookup  => "true" | "false",                         # Optional / Default value is "false"
     remote_lookup_service => $auth_server,                             # Optional / Default value is undefined
     remote_lookup_username => $string,                                 # Optional / Default value is undefined
     remote_lookup_password => $string,                                 # Optional / Default value is undefined
 );
```

SESSION METHODS

This section describes all the methods in an Infoblox::Session module that you can apply to a Certificate Authentication Service object.

Infoblox::Session->add( )

Use this method to add an object to the Infoblox appliance. See Infoblox::Session->add() for parameters and return values.

Example

```
#Construct an object
my $auth_service = Infoblox::Grid::Admin::CertificateAuthService->new(
    name            => 'cas',
    ca_certificates => [$ca1, $ca2],
    ocsp_responders => [$responder],

#Submit for addition
my $response = $session->add($auth_service);
```

Infoblox::Session->get( )

Use this method to retrieve all the matching objects from the Infoblox appliance. See Infoblox::Session->get() for parameters and return values.

Key References

```
Apply the following attributes to get a specific Certificate Authentication Service object:

name                   - Optional. The name of the Certificate Authentication Service in string
format.
disabled               - Optional. The boolean flag that shows whether the Certificate
Authentication Service is enabled or disabled.
user_match_type        - Optional. The Certificate Authentication Service user match type in enum
format.
ocsp_check             - Optional. The Certificate Authentication Service ocsp check in enum format.
remote_lookup_username - Optional. The Certificate Authentication Service remote lookup username in
string format.
```

Example

```
my @retrieved_objs = $session->get(
     object => 'Infoblox::Grid::Admin::CertificateAuthService',
     name   => 'ocsp',
);

my @retrieved_objs = $session->get(
     object   => 'Infoblox::Grid::Admin::CertificateAuthService',
     disabled => 'false',
);
```

Infoblox::Session->modify( )

Use this method to modify an object on the Infoblox appliance. See Infoblox::Session->modify() for parameters and return values.

Example

```
# Use this method to modify the comment of a Certificate Authentication Service object
$auth_service->comment('This is modified Certificate Authentication Service object');

# Submit modification
my $response = $session->modify($auth_service);
```

Infoblox::Session->remove( )

Use this method to remove an object from the Infoblox appliance. See Infoblox::Session->remove() for parameters and return values.

To remove a specific object, first use `get()` to retrieve the object, and then submit it for removal.

Example

```
#Get the objects with the specified name.
my @retrieved_objs = $session->get(
     object => 'Infoblox::Grid::Admin::CertificateAuthService',
     name   => 'ocsp',
);

#Find the desired object on the retrieved list.
my $desired_auth_service = $retrieved_objs[0];

# Submit for removal
my $response = $session->remove($desired_auth_service);
```

Infoblox::Session->search( )

Use this method to retrieve all the matching objects from the Infoblox appliance. See Infoblox::Session->search() for parameters and return values.

Key References

```
 Apply the following attributes to get a specific Certificate Authentication Service object:

 name                   - Optional. The name of the Certificate Authentication Service in string
format.
 disabled               - Optional. The boolean flag that shows whether the Certificate
Authentication Service is enabled or disabled.
 user_match_type        - Optional. The Certificate Authentication Service user match type in enum
format.
 ocsp_check             - Optional. The Certificate Authentication Service ocsp check in enum format.
 remote_lookup_username - Optional. The Certificate Authentication Service remote lookup username in
string format.
```

Example

```
my @retrieved_objs = $session->search(
     object => 'Infoblox::Grid::Admin::CertificateAuthService',
     name   => 'ocsp',
);

my @retrieved_objs = $session->search(
     object   => 'Infoblox::Grid::Admin::CertificateAuthService',
     disabled => 'false',
);
```

METHODS

This section describes all the methods that you can use to configure and retrieve the attribute values of a Certificate Authentication Service object.

auto_populate_login( )

Use this method to set or retrieve a parameter that specifies the client certificate value to auto-populate the NIOS login name field.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

The method returns either of these attribute values: "SERIAL_NUMBER", "S_DN_CN", "S_DN_EMAIL", "SAN_UPN", "SAN_EMAIL" or "AD_SUBJECT_ISSUER". The default value is "S_DN_CN".

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get auto_populate_login
my $auto_populate_login = $auth_service->auto_populate_login();

#Set auto_populate_login
$auth_service->auto_populate_login('SERIAL_NUMBER');
```

ca_certificates( )

Use this method to set or retrieve the list of CA certificates in the Certificate Authentication Service object.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

Valid value is an array reference that contains Infoblox::Grid::CACertificate object(s).

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get ca_certificates value
my $ca_certificates = $auth_service->ca_certificates();

#Modify ca_certificates

#Retrieve the existing certificate
my $certificate = $session->get(
    object => 'Infoblox::Grid::CACertificate',
    serial => 'e76227f314c0f3db',
);

#Modify ca_certificates value
$auth_service->ca_certificates([$certificate]);
```

comment( )

Use this method to set or retrieve a descriptive comment.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

Desired comment in string format with a maximum of 256 bytes.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get comment
my $comment = $auth_service->comment();

#Modify comment
$auth_service->comment("Modified comment");
```

disabled( )

Use this method to set or retrieve the disabled flag for the Certificate Authentication Service.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

Specify "true" to disable the Certificate Authentication Service or "false" to enable it. Default value is "false".

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value as "true" or "false".

Example

```
#Get disabled flag
my $disabled = $auth_service->disabled();

#Modify disabled flag
$auth_service->disabled("true");
```

enable_password_request( )

Use this method to set or retrieve the flag that shows whether the username/password authentication together with certificate based is enabled or disabled.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

Specify "true" to enable the username/password authentication together with certificate based or "false" to disable it. Default value is "true".

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value as "true" or "false".

Example

```
#Get enable_password_request flag
my $enable_password_request = $auth_service->enable_password_request();

#Modify enable_password_request flag
$auth_service->enable_password_request("false");
```

enable_remote_lookup( )

Use this method to set or retrieve the flag that shows whether the lookup for users group membership info on remote services is enabled or disabled.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

Specify "true" to enable the lookup for users group membership info on remote services or "false" to disable it. Default value is "false".

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value as "true" or "false".

Example

```
#Get enable_remote_lookup flag
my $enable_remote_lookup = $auth_service->enable_remote_lookup();

#Modify enable_remote_lookup flag
$auth_service->enable_remote_lookup("true");
```

max_retries( )

Use this method to set or retrieve the number of validation retries before the appliance moves on to the next OCSP responder.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

An unsigned integer. Default value is 0.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get max_retries
my $max_retries = $auth_service->max_retries();

#Modify max_retries
$auth_service->max_retries(1);
```

name( )

Use this method to set or retrieve the Certificate Authentication Service name.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

The Certificate Authentication Service name in string format. The default value is "Cert auth service".

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get name
my $name = $auth_service->name();
```

```
#Modify comment
$auth_service->name("New cert group");
```

ocsp_check( )

Use this method to set or retrieve the source of OCSP settings.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

The name of the parameter can be "MANUAL", "AIA_ONLY", "AIA_AND_MANUAL" or "DISABLED". The default value is "MANUAL".

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get ocsp_check
my $ocsp_check = $auth_service->ocsp_check();

#Set ocsp_check
$auth_service->ocsp_check('DISABLED');
```

ocsp_responders( )

Use this method to set or retrieve the list of OCSP responders in the Certificate Authentication Service object.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

The valid value is an array reference that contains Infoblox::OCSP::Responder object(s).

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get ocsp_responders
my $responder_list = $auth_service->ocsp_responders();

#Modify ocsp_responders
my $responder1 = Infoblox::OCSP::Responder->new(
     fqdn_or_ip  => 'domain.com',
     certificate => '/tmp/cert1.pem',
);

my $responder2 = Infoblox::OCSP::Responder->new(
     fqdn_or_ip  => '192.168.1.10',
     certificate => '/tmp/cert2.pem',
);

$auth_service->ocsp_responders([$responder1, $responder2]);
```

recovery_interval( )

Use this method to set or retrieve the specified period of time the appliance waits before retrying a responder that has been offline. The value must be between 1 and 600 seconds.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

An unsigned integer. Default value is 30.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get recovery_interval
my $recovery_interval = $auth_service->recovery_interval();

#Modify recovery_interval
$auth_service->recovery_interval(5);
```

remote_lookup_password( )

Use this method to set or retrieve the password for a service account.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

The password for service account in string format.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get remote_lookup_password
my $remote_lookup_password = $auth_service->remote_lookup_password();

#Set remote_lookup_password
$auth_service->remote_lookup_password('infoblox');
```

remote_lookup_service( )

Use this method to set or retrieve the service that will be used for remote lookup.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

Valid value is a reference that contains Infoblox::Grid::Admin::AdAuthServer object.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get remote_lookup_service
my $remote_lookup_service = $auth_service->remote_lookup_service();

#Modify remote_lookup_service
my $remote_lookup_service1 = Infoblox::Grid::Admin::AdAuthServer->new(
   name => "1.2.3.4",
   port => 15623,
   encryption => "SSL"
);

$auth_service->remote_lookup_service($remote_lookup_service1);
```

remote_lookup_username( )

Use this method to set or retrieve the username for a service account.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

The username for a service account in string format.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get remote_lookup_username
my $remote_lookup_username = $auth_service->remote_lookup_username();

#Set remote_lookup_username
$auth_service->remote_lookup_username('admin');
```

response_timeout( )

Use this method to set or retrieve validation timeout period in milliseconds, must be between 1000 and 60000.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

An unsigned integer. Default value is 1000.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get response_timeout
my $response_timeout = $auth_service->response_timeout();

#Modify response_timeout
$auth_service->response_timeout(7000);
```

trust_model( )

Use this method to set or retrieve a parameter that specifies how the appliance performs OCSP requests.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

The name of parameter is either "DIRECT" or "DELEGATED". Default value is "DIRECT".

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get trust_model
my $trust_model = $auth_service->trust_model();

#Set trust_model
$auth_service->trust_model('DELEGATED');
```

user_match_type( )

Use this method to set or retrieve a parameter that specifies how to search for a particular user: if the parameter is equal to "DIRECT_MATCH" the method searches for only local users with manually assigned certificates, else if the parameter is equal to "AUTO_MATCH" the method populates username from certificate and searches against effective Auth Policies.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

The valid value is "DIRECT_MATCH" or "AUTO_MATCH". The default value is "DIRECT_MATCH".

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get user_match_type
my $user_match_type = $auth_service->user_match_type();

#Set user_match_type
```

```
$auth_service->user_match_type('AUTO_MATCH');
```

SAMPLE CODE

The following sample code demonstrates the different functions that can be applied to an object such as add, get, modify and remove. This sample also includes error handling for the operations.

#Preparation prior to creating and modifying a Certificate Authentication Service object

```
use strict;
use Infoblox;

#refers to Infoblox Appliance IP address
my $host_ip = "192.168.1.2";

#Create a session to the Infoblox appliance

my $session = Infoblox::Session->new(
    master   => $host_ip,
    username => "admin",
    password => "infoblox"
);
unless ($session) {
       die("Construct session failed: ",
               Infoblox::status_code() . ":" . Infoblox::status_detail());
}
print "Session created successfully\n";
```

#Create an OCSP responder

```
my $responder = Infoblox::OCSP::Responder->new(
    address     => 'domain.com',
    certificate => '/tmp/cert.pem',
    disabled => 'true',
);

unless ($responder) {
       die("Construct OCSP responder object failed: ",
               Infoblox::status_code() . ":" . Infoblox::status_detail());
}
print "OCSP responder object created successfully\n";
```

#Create a Certificate Authentication Service object

```
my $auth_service = Infoblox::Grid::Admin::CertificateAuthService->new(
    name            => 'cas',
    ca_certificates => [$ca1, $ca2],
    ocsp_responders => [$responder],
    disabled        => 'true',
);

unless ($auth_service) {
       die("Construct Certificate Authentication Service object failed: ",
               Infoblox::status_code() . ":" . Infoblox::status_detail());
}
print "Certificate Authentication Service object created successfully\n";

#Add the Certificate Authentication Service object to an appliance through session
my $response = $session->add($auth_service);

unless ($response) {
    die("Add Certificate Authentication Service object failed: ",
           $session->status_code() . ":" . $session->status_detail());
}
print "Certificate Authentication Service object added successfully \n";
```

#Get and modify the Certificate Authentication Service object

```
#Get a Certificate Authentication Service object through session
my @retrieved_objs = $session->get(
    object   => "Infoblox::Grid::Admin::CertificateAuthService",
    name     => "cas",
    );
my $auth_service = $retrieved_objs[0];

unless ($auth_service) {
```

```
    die("Get a Certificate Authentication Service object failed: ",
           $session->status_code() . ":" . $session->status_detail());
}
print"Get a Certificate Authentication Service object successful \n";

# Modify one of the attributes of the selected Certificate Authentication Service object.

$auth_service->comment("new comment");

#Applying the changes
$session->modify($auth_service)
   or die("Modify Certificate Authentication Service object failed",
            $session->status_code() . ":" . $session->status_detail());
print "Certificate Authentication Service object modified successfully \n";
```

#Remove the Certificate Authentication Service object

```
#Get a Certificate Authentication Service object through session
my @retrieved_objs = $session->get(
    object => "Infoblox::Grid::Admin::CertificateAuthService",
    name   => "cas",
    );
my $desired_auth_service = $retrieved_objs[0];

unless ($desired_auth_service) {
    die("Get a Certificate Authentication Service object failed: ",
           $session->status_code() . ":" . $session->status_detail());
}
print "Get a Certificate Authentication Service object successful \n";

#Submit the object for removal
$session->remove($desired_auth_service)
    or die("Remove Certificate Authentication Service object failed",
           $session->status_code() . ":" . $session->status_detail());
print "Certificate Authentication Service object removed successfully \n";

####PROGRAM ENDS####
```

AUTHOR

Infoblox Inc. http://www.infoblox.com/

SEE ALSO

Infoblox::OCSP::Responder

COPYRIGHT

Copyright (c) 2017 Infoblox Inc.

Infoblox::Grid::Admin::RadiusAuthService - RADIUS Authentication Service object

- NAME
- DESCRIPTION
- CONSTRUCTOR
- SESSION METHODS
  - Infoblox::Session->add( )
  - Infoblox::Session->get( )
  - Infoblox::Session->modify( )
  - Infoblox::Session->remove( )
- METHODS
  - acct_retries( )
  - acct_timeout( )
  - auth_retries( )
  - auth_timeout( )
  - cache_ttl( )
  - comment( )
  - disabled( )
  - enable_cache( )
  - name( )
  - selection_mode( )
  - radius_servers( )
  - recovery_interval( )
- SAMPLE CODE
- AUTHOR
- SEE ALSO
- COPYRIGHT

NAME

Infoblox::Grid::Admin::RadiusAuthService - RADIUS Authentication Service object

DESCRIPTION

Represents a group of RADIUS servers that are used to authenticate administrators and DHCP clients.

CONSTRUCTOR

```
my $radius_auth_server = Infoblox::Grid::Admin::RadiusAuthService->new(
    acct_retries          => $num,                                  #Optional / Default is 1000
    acct_timeout          => $num,                                  #Required
    auth_retries          => $num,                                  #Optional / Default is 5
    auth_timeout          => $num,                                  #Required
    cache_ttl             => $num,                                  #Optional / Default is 3600
    comment               => $string,                               #Optional / Default is empty
    disabled              => "true" | "false"                       #Optional / Default is false
    enable_cache          => "true" | "false"                       #Optional / Default is false
    name                  => $string,                               #Required
    radius_servers        => [$RadiusAuthServer1, ...],             #Required
    recovery_ttl          => $num,                                  #Optional / Default is 30
    selection_mode        => "HUNT_GROUP" | "ROUND_ROBIN",          #Optional / Default is "HUNT_GROUP"
);
```

SESSION METHODS

This section describes all the methods in the Infoblox::Session module that you can apply to a RADIUS Auth object.

Infoblox::Session->add( )

Use this method to add an object to the Infoblox appliance. See Infoblox::Session->add() for parameters and return values.

Example

```
#Construct an object
my $service = Infoblox::Grid::Admin::RadiusAuthService->new(
                                                             name           => 'some.name.com',
                                                             radius_servers => [$ads1, $ads2]
                                                            );
#Submit for addition
```

```
my $response = $session->add( $service );
```

Infoblox::Session->get( )

Use this method to retrieve all the matching objects from the Infoblox appliance. See Infoblox::Session->get() for parameters and return values.

Key References

```
Apply the following attributes to get a specific RADIUS Auth object:

 name - Required. RADIUS Authentication Service name.
```

Examples

```
my @retrieved_objs = $session->get(
    object => "Infoblox::Grid::Admin::RadiusAuthService",
    name   => "some.name.com"
);
```

Infoblox::Session->modify( )

Use this method to modify an object in the Infoblox appliance. See Infoblox::Session->modify() for parameters and return values.

Example

```
#Use this method to modify the comment.
$service->comment("this is a modified comment");
#Submit modification
my $response = $session->modify( $service );
```

Infoblox::Session->remove( )

Use this method to remove an object from the Infoblox appliance. See Infoblox::Session->remove() for parameters and return values.

To remove a specific object, first use `get()` or `search()` to retrieve the specific object, and then submit this object for removal.

Example

```
#Get the objects with the same name
my @retrieved_objs = $session->get(
    object => "Infoblox::Grid::Admin::RadiusAuthService",
    name   => "some.name.com"
);

#Find the desired object from the retrieved list.
my $desired_service = $retrieved_objs[0];
#Submit for removal
my $response = $session->remove( $desired_service );
```

METHODS

Use the following methods to access the attributes of an Infoblox::Grid::Admin::RadiusAuthService object. Specify a parameter to set the attribute's value, or omit the parameter to get the attribute's value.

acct_retries( )

Use this method to set or retrieve the value that specifies how many times the appliance must resend the RADIUS accounting packet if the appliance does not receive a response.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

An unsigned integer between 1 and 10, inclusive.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Getting acct_retries
my $acct_retries = $radius_service->acct_retries( );
#Modifying acct_retries
```

```
$radius_service->acct_retries("5");
```

acct_timeout( )

Use this method to set or retrieve the value, in milliseconds, that specifies how long the appliance must wait for authentication response packets.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

An unsigned integer between 1000 and 10000, inclusive.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Getting acct_timeout
my $acct_timeout = $radius_service->acct_timeout( );
#Modifying acct_timeout
$radius_service->acct_timeout("5000");
```

auth_retries( )

Use this method to set or retrieve the value of how many times to retry if there is no response for the RADIUS authentication packet.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

An unsigned integer between 1 and 10, inclusive.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Getting auth_retries
my $auth_retries = $radius_service->auth_retries( );
#Modifying auth_retries
$radius_service->auth_retries("5");
```

auth_timeout( )

Use this method to set or retrieve the value in milliseconds to wait for authentication response packets.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

An unsigned integer between 1000 and 10000, inclusive.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Getting auth_timeout
my $auth_timeout = $radius_service->auth_timeout( );
#Modifying auth_timeout
$radius_service->auth_timeout("5000");
```

cache_ttl( )

Use this method to set or retrieve the TTL for the authentication response in the authentication cache.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

An unsigned integer between 1 and 259200, inclusive.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Getting cache_ttl
my $cache_ttl = $radius_service->cache_ttl( );
#Modifying cache_ttl
$radius_service->cache_ttl("3600");
```

comment( )

Use this method to set or retrieve a descriptive comment about the RADIUS Authentication Service object.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

The comment in string format, with a maximum of 256 characters.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get comment
my $comment = $radius_service->comment( );
#Modify comment
$radius_service->comment("RADIUS Authentication Service for NAC authentication");
```

disabled( )

Use this method to set or retrieve the disable flag of the RADIUS Authentication Service object.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

Specify "true" to set the disable flag or "false" to deactivate/unset it. The default value is "false".

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get disable
my $disable = $radius_service->disabled( );
#Modify disabled
$radius_service->disabled("true");
```

enable_cache( )

Use this method to set or retrieve the flag that indicates if the authentication cache is enabled on the RADIUS Authentication Service object.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

Specify "true" to set the disable flag or "false" to deactivate/unset it. The default value is "false".

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get enable_cache
my $enable_cache = $radius_service->enable_cache( );
#Modify enable_cache
$radius_service->enable_cache("true");
```

name( )

Use this method to set or retrieve the name of the RADIUS Authentication Service object.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

The name in string format, with a maximum of 64 characters.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Getting name
my $name = $radius_service->name( );
#Modifying name
$radius_service->name("RADIUS Authentication Service");
```

selection_mode( )

Use this method to set or retrieve the process the appliance uses to select to which RADIUS server in the RADIUS Authentication Service it sends RADIUS requests.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

Valid values are "HUNT_GROUP" (ordered list) or "ROUND_ROBIN". The default is "HUNT_GROUP".

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Getting selection_mode
my $selection_mode = $radius_service->selection_mode( );
#Modifying selection_mode
$radius_service->selection_mode("HUNT_GROUP");
```

radius_servers( )

Use this method to set or retrieve a list of RADIUS servers assigned to this RADIUS Service.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

An array reference that contains a list of Infoblox::Grid::Admin::RadiusAuthServer objects.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Getting radius_servers
my $radius_servers = $radius_service->radius_servers( );
#Modifying radius_servers
$radius_service->radius_servers([$radius_server1]);
```

recovery_interval( )

Use this method to set or retrieve the time in seconds that the appliance waits before re-enabling a RADIUS server marked as DOWN.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

An unsigned integer between 1 and 600, inclusive.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Getting recovery_interval
my $recovery_interval = $radius_service->recovery_interval( );
#Modifying recovery_interval
$radius_service->recovery_interval("30");
```

SAMPLE CODE

The following sample code demonstrates the different functions that can be applied to an object, such as add, search, modify, and remove. This sample also includes error handling for the operations.

#Preparation prior to a Radius Auth Service object insertion

```
#PROGRAM STARTS: Include all the modules that will be used
use strict;
use Infoblox;

#Create a session to the Infoblox appliance
my $host_ip =  "192.168.1.2";
my $session = Infoblox::Session->new(
    master   => $host_ip,
    username => "admin",
    password => "infoblox"
    );
unless($session){
        die("Constructor for session failed: ",
               Infoblox::status_code(). ":" . Infoblox::status_detail());
}
print "Session created successfully.\n";

my $ads1 = Infoblox::Grid::Admin::RadiusAuthServer->new(
                                                        comment  => 'server 1',
                                                        fqdn_or_ip => '10.0.1.1',
                                                        shared_secret => 'secret1',
                                                       );

my $ads2 = Infoblox::Grid::Admin::RadiusAuthServer->new(
                                                        comment  => 'server 2',
                                                        fqdn_or_ip => 'domain.com',
                                                        shared_secret => 'secret2',
                                                      );
```

#Create the RADIUS Auth Service.

```
my $service = Infoblox::Grid::Admin::RadiusAuthService->new(
                                                            acct_retries   => 2000,
                                                            auth_retries   => 9,
                                                            cache_ttl      => 500,
                                                            disabled       => 'true',
                                                            enable_cache   => 'true',
                                                            recovery_ttl   => 50,
                                                            selection_mode => 'ROUND_ROBIN',
                                                            acct_timeout   => 1100,
                                                            auth_timeout   => 1200,
                                                            name           => 'some.name.com',
                                                            radius_servers => [$ads1, $ads2]
                                                           );

unless($service){
       die("Construct service object failed: ",
               Infoblox::status_code(). ":" .Infoblox::status_detail());
```

```
        }
print "test.com zone object created successfully.\n";

$session->add($service)
    or die("Add zone service failed: ",
                $session->status_code(). ":" .$session->status_detail());

print"Service added successfully.\n";
```

#Get a specific service

```
my @search_result = $session->get(
    object => "Infoblox::Grid::Admin::RadiusAuthService",
    name   => "some.name.com"
);

my $search_obj = $search_result[0];
unless($search_obj){
        die("Search service failed: ",
                 $session->status_code() . ":" . $session->status_detail());
        }

print "Search service object found at least 1 matching entry\n";
```

#Get and modify a RADIUS service

```
$search_obj->comment("this is a modified comment.");

#Apply the change
    $session->modify($search_obj)
        or die("Modify service failed: ",
                $session->status_code(). ":" .$session->status_detail());

 print "Service object modified successfully.\n";
```

#Remove a Service

```
my @search_result = $session->get(
    object => "Infoblox::Grid::Admin::RadiusAuthService",
    name   => "some.name.com"
);

my $search_obj = $search_result[0];
unless($search_obj){
        die("Search service failed: ",
                 $session->status_code() . ":" . $session->status_detail());
        }

print "Search service object found at least 1 matching entry\n";

$session->remove($search_obj)
    or die("Remove service failed: ",
             $session->status_code(). ":" .$session->status_detail());

print "Remove service successfull\n";

####PROGRAM ENDS####
```

AUTHOR

Infoblox Inc. http://www.infoblox.com

SEE ALSO

Infoblox::Grid::Admin::RadiusAuthServer, Infoblox::Session, Infoblox::Session->add(), Infoblox::Session->get(), Infoblox::Session->modify(), Infoblox::Session->remove()

COPYRIGHT

Copyright (c) 2017 Infoblox Inc.

Infoblox::Grid::CACertificate - CA certificate description.

- NAME
- DESCRIPTION
- SESSION METHODS
  - Infoblox::Session->get( )
  - Infoblox::Session->remove( )
  - Infoblox::Session->search( )
- METHODS
  - distinguished_name( )
  - issuer( )
  - serial( )
  - used_by( )
- AUTHOR
- COPYRIGHT

NAME

Infoblox::Grid::CACertificate - CA certificate description.

DESCRIPTION

An Infoblox::Grid::CACertificate object represents a CA certificate description.

SESSION METHODS

This section describes all the methods in an Infoblox::Session module that you can apply to an Infoblox::Grid::CACertificate object.

Infoblox::Session->get( )

Use this method to retrieve all the matching objects from the Infoblox appliance. See Infoblox::Session->get() for parameters and return values.

Example

```
my @retrieved_objs = $session->get(
     object => 'Infoblox::Grid::CACertificate',
     serial => 'serial',
);

my @retrieved_objs = $session->get(
     object  => 'Infoblox::Grid::CACertificate',
     issuer => 'issuer',
);

my @retrieved_objs = $session->get(
     object   => 'Infoblox::Grid::CACertificate',
     distinguished_name => 'distinguished_name',
);
```

Infoblox::Session->remove( )

Use this method to remove an object from the Infoblox appliance. See Infoblox::Session->remove() for parameters and return values.

To remove a specific object, first use `get()` to retrieve the object, and then submit it for removal.

Example

```
#Get the objects with the specified serial.
my @retrieved_objs = $session->get(
     object => 'Infoblox::Grid::CACertificate',
     serial => 'serial',
);

#Find the desired object on the retrieved list.
my $desired_ca_cert = $retrieved_objs[0];

# Submit for removal
my $response = $session->remove($desired_ca_cert);
```

Infoblox::Session->search( )

Use this method to retrieve all the matching objects from the Infoblox appliance. See Infoblox::Session->search() for parameters and return values.

Example

```
my @retrieved_objs = $session->search(
     object => 'Infoblox::Grid::CACertificate',
     serial => 'serial',
);

my @retrieved_objs = $session->search(
     object  => 'Infoblox::Grid::CACertificate',
     issuer => 'issuer',
);

my @retrieved_objs = $session->search(
     object   => 'Infoblox::Grid::CACertificate',
     distinguished_name => 'distinguished_name',
);
```

METHODS

This section describes all the methods that you can use to configure and retrieve the attribute values of an Infoblox::Grid::CACertificate object.

distinguished_name( )

Use this method to retrieve certificate subject name. This attribute is read-only and cannot be set.

Parameter

None.

Returns

The method returns the attribute value.

Example

```
#Get distinguished_name
my $distinguished_name = $ca_cert->distinguished_name();
```

issuer( )

Use this method to retrieve issuer subject name. This attribute is read-only and cannot be set.

Parameter

None.

Returns

The method returns the attribute value.

Example

```
#Get issuer
my $issuer = $ca_cert->issuer();
```

serial( )

Use this method to retrieve Certificate serial number in hex format. This attribute is read-only and cannot be set.

Parameter

None.

Returns

The method returns the attribute value.

Example

```
#Get serial
my $serial = $ca_cert->serial();
```

used_by( )

Use this method to retrieve information about usages of the certificate. This attribute is read-only and cannot be set.

Parameter

None.

Returns

The method returns the attribute value. If the returned value is equal to SSL/TLS - all certificates in CA store can be used for HTTPS and if the returned value is equal to CAS - additionally certificate can be assigned to Certificate Authentication Group.

Example

```
#Get used_by
my $used_by = $ca_cert->used_by();
```

AUTHOR

Infoblox Inc. http://www.infoblox.com/

COPYRIGHT

Copyright (c) 2017 Infoblox Inc.

Infoblox::OCSP::Responder - an OCSP authentication responder.

- NAME
- DESCRIPTION
- CONSTRUCTOR
- MODULE METHODS
  - Infoblox::Grid::Admin::CertificateAuthService->ocsp_responders( )
- METHODS
  - certificate( )
  - comment( )
  - disabled( )
  - address( )
  - port( )
- AUTHOR
- SEE ALSO
- COPYRIGHT

NAME

Infoblox::OCSP::Responder - an OCSP authentication responder.

DESCRIPTION

An Infoblox::OCSP::Responder object represents an OCSP authentication responder.

CONSTRUCTOR

```
my $responder = Infoblox::OCSP::Responder->new(
     address      => $string,              # Required
     certificate  => $string,              # Required
     port         => $uint,                # Optional / Default value is 80
     comment      => $string,              # Optional
     disabled     => "true" | "false",     # Optional / Default value is "false"
);
```

MODULE METHODS

The following functions can be applied to an OCSP authentication responder object.

Infoblox::Grid::Admin::CertificateAuthService->ocsp_responders( )

Use this function to add or retrieve the OCSP authentication responder object in the Certificate Authentication Service object. See Infoblox::Grid::Admin::CertificateAuthService->ocsp_responders() for parameters and return values.

Example

```
#Get ocsp_responders
my $responder_list = $auth_service->ocsp_responders();

#Modify ocsp_responders
my $responder1 = Infoblox::OCSP::Responder->new(
     fqdn_or_ip  => 'domain.com',
     certificate => '/tmp/cert1.pem',
);

my $responder2 = Infoblox::OCSP::Responder->new(
     fqdn_or_ip  => '192.168.1.10',
     certificate => '/tmp/cert2.pem',
);

$auth_service->ocsp_responders([$responder1, $responder2]);
```

METHODS

This section describes all the methods that can be used to configure and retrieve the attribute values of an Infoblox::OCSP::Responder object.

certificate( )

Use this method to set the OCSP responder certificate. This attribute is write-only and cannot be retrieved.

Parameter

String with a reference to a file.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

Example

```
#Set certificate
$responder->certificate('/tmp/cer.pem');
```

comment( )

Use this method to set or retrieve a descriptive comment.

Parameter

Desired comment in string format with a maximum of 256 bytes.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get comment
my $comment = $responder->comment();

#Modify comment
$responder->comment("Modified comment");
```

disabled( )

Use this method to set or retrieve the disabled flag for the OCSP authentication responder.

Parameter

Specify "true" to disable the OCSP authentication respondert or "false" to enable it. Default value is "false".

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value as "true" or "false".

Example

```
#Get disabled flag
my $disabled = $responder->disabled();

#Modify disabled flag
$responder->disabled("true");
```

address( )

Use this method to set or retrieve the IP address or the domain name for the OCSP authentication responder.

Parameter

String with the IP address or the domain name for the OCSP authentication responder.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get address
my $address = $responder->address();

#Modify address
```

```
$responder->address("192.168.1.2");
```

port( )

Use this method to set or retrieve the TCP Port on which the OCSP client communicates with the OCSP responder.

Parameter

An unsigned integer. Default value is 80.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get port
my $port = $responder->port();

#Modify port
$responder->port(90);
```

AUTHOR

Infoblox Inc. http://www.infoblox.com/

SEE ALSO

Infoblox::Grid::Admin::CertificateAuthService

COPYRIGHT

Copyright (c) 2017 Infoblox Inc.

Infoblox::Grid::Admin::Role - Admin Role object.

- NAME
- DESCRIPTION
- CONSTRUCTOR
- SESSION METHODS
  - Infoblox::Session->add( )
  - Infoblox::Session->get( )
  - Infoblox::Session->modify( )
  - Infoblox::Session->remove( )
  - Infoblox::Session->search( )
- METHODS
  - comment( )
  - disabled( )
  - extattrs( )
  - extensible_attributes( )
  - name( )
- SAMPLE CODE
- AUTHOR
- SEE ALSO
- COPYRIGHT

NAME

Infoblox::Grid::Admin::Role - Admin Role object.

DESCRIPTION

An Admin Role object creates and manages a local admin role on the Infoblox appliance. A Role object is used to aggregate a set of permissions (represented by Permission objects).

CONSTRUCTOR

```
 my $role = Infoblox::Grid::Admin::Role->new(
     name                  => $string,
#Required
     comment               => $string,
#Optional / Default is undefined
     disabled              => "true" | "false",
#Optional / Default is "false"
     extattrs              => { $string => $extattr, ... },
#Optional / Default is undefined
     extensible_attributes => { $string => $string | $num, $string => [ $string | $num, ... ], ... }
#Optional / Default is undefined
 );
```

You cannot set both extattrs and extensible_attributes attributes at the same time.

SESSION METHODS

This section describes all the methods in Infoblox::Session module that you can apply to an Admin Role object.

Infoblox::Session->add( )

Use this method to add an object to the Infoblox appliance. See Infoblox::Session->add() for parameters and return values.

Example

```
#Construct the Role object
my $role = Infoblox::Grid::Admin::Role->new(
    name                  => "testrole",
    comment               => "test role",
);
# Submit for addition
my $response = $session->add( $role );
```

Infoblox::Session->get( )

Use this method to retrieve all the matching objects from the Infoblox appliance. See Infoblox::Session->get() for parameters and return values.

Key References

```
Apply the following attributes to get a specific Role object:

 name      - Required. Role name in string format.
 extattrs  - Optional. A hash reference containing extensible attributes.
 extensible_attributes - Optional. A hash reference containing extensible attributes.
```

Example

```
my @retrieved_objs = $session->get(
    object     => "Infoblox::Grid::Admin::Role",
    name       => "testrole" );
```

Infoblox::Session->modify( )

Use this method to modify an object in the Infoblox appliance. See Infoblox::Session->modify() for parameters and return values.

Example

```
# Use this method to modify the comment.
$role->comment("This is a modified comment");
# Submit modification
my $response = $session->modify( $role );
```

Infoblox::Session->remove( )

Use this method to remove an object from the Infoblox appliance. See Infoblox::Session->remove() for parameters and return values.

To remove a specific object, use `get()` or `search()` to retrieve the specific object first, and then submit this object for removal.

Example

```
# Get the role objects with the same starting name
my @retrieved_objs = $session->get(
    object     => "Infoblox::Grid::Admin::Role",
    name       => "testrole" );
# find the desired object from the retrieved list.
my $desired_role = $retrieved_objs[0];
# Submit for removal
my $response = $session->remove( $desired_role );
```

Infoblox::Session->search( )

Use this method to search for Admin Role objects in the Infoblox appliance. See Infoblox::Session->search() for parameters and return values.

Key References

```
Apply the following attributes to search for a specific Role object:

 name      - Required. Role name in string format (regular expression).
 extattrs  - Optional. A hash reference containing extensible attributes.
 extensible_attributes - Optional. A hash reference containing extensible attributes.
```

For more information about searching extensible attributes, see Infoblox::Grid::ExtensibleAttributeDef/Searching Extensible Attributes.

Example

```
# search for all Role objects that start with "test"
my @retrieved_objs = $session->search(
    object => "Infoblox::Grid::Admin::Role",
    name   => "test.*" );
```

METHODS

This section describes all the methods that you can use to set and retrieve the attribute values of a Role object.

comment( )

Use this method to set or retrieve the comment.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

Pertinent information about the admin role, such as its purpose. Comment in string format with a maximum of 256 bytes. The

default value is undefined.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get comment
my $comment = $role->comment();
#Modify comment
$role->comment("This is the modified comment for admin role");
```

disabled( )

Use this method to set or retrieve the "disabled" flag.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

The default value for this field is false. The role is enabled.

Parameter

Specify "true" to set the disable flag or "false" to deactivate/unset it.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get disabled
my $disabled = $role->disabled();
#Modify disabled
$role->disabled("true");
```

extattrs( )

Use this method to set or retrieve the extensible attributes associated with a Role object.

Parameter

Valid value is a hash reference containing the names of extensible attributes and their associated values (Infoblox::Grid::Extattr objects).

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get extattrs
my $ref_extattrs = $role->extattrs();
#Modify extattrs
$role->extattrs({ 'Site' => $extattr1, 'Administrator' => $extattr2 });
```

extensible_attributes( )

Use this method to set or retrieve the extensible attributes associated with a Role object.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

For valid values for extensible attributes, see Infoblox::Grid::ExtensibleAttributeDef/Extensible Attribute Values.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get extensible attributes
my $ref_extensible_attributes = $role->extensible_attributes();
#Modify extensible attributes
$role->extensible_attributes({ 'Site' => 'Santa Clara', 'Administrator' => [ 'Peter', 'Tom' ] });
```

name( )

Use this method to set or retrieve the name of an admin role.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

Text with the name of the admin role with a maximum of 64 characters.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get name
my $name = $role->name();
```

SAMPLE CODE

The following sample code demonstrates the different functions that can be applied to an object, such as add, search, modify, and remove. This sample also includes error handling for the operations.

#Preparation prior to an Admin Role object insertion

```
#PROGRAM STARTS: Include all the modules that will be used
use strict;
use Infoblox;

#Create a session to the Infoblox appliance

my $session = Infoblox::Session->new(
               master   => "192.168.1.2", #appliance host ip
               username => "admin",       #appliance user login
               password => "infoblox"     #appliance password
               );

unless ($session) {
       die("Construct session failed: ",
               Infoblox::status_code() . ":" . Infoblox::status_detail());
}
print "Session created successfully\n";
```

#Create an Admin Role object

```
my $role = Infoblox::Grid::Admin::Role->new(
    name                => "testrole",
    comment             => "test admin role",
);

unless ($role) {
       die("Construct role object failed: ",
               Infoblox::status_code() . ":" . Infoblox::status_detail());
}
print "Admin Role object created successfully\n";

#Add the Admin Role object to the Infoblox appliance through a session
$session->add($role)
       or die("Add Admin Role object failed: ",
                       $session->status_code() . ":" . $session->status_detail());
print "Admin Role object added to server successfully\n";
```

#Search for an Admin Role

```
my @retrieved_objs = $session->search(
               object => "Infoblox::Grid::Admin::Role",
               name   => "test.*"
               );
```

```
my $object = $retrieved_objs[0];

unless ($object) {
       die("Search Admin Role object failed: ",
               $session->status_code() . ":" . $session->status_detail());
}
print "Search Admin Role object found at least 1 matching entry\n";
```

#Get and modify an Admin Role object

```
#Get the Admin Role object from Infoblox appliance through a session
my @retrieved_objs = $session->get(
    object => "Infoblox::Grid::Admin::Role",
    name   => "testrole"
);
my $object = $retrieved_objs[0];

unless ($object) {
       die("Get Admin Role object failed: ",
               $session->status_code() . ":" . $session->status_detail());
}
print "Get Admin Role object found at least 1 matching entry\n";

#Modify the Admin Role object
$object->name("testrole_renamed");
$object->comment("this is a new comment");

#Apply the changes.
$session->modify($object)
       or die("Modify Admin Role object failed: ",
               $session->status_code() . ":" . $session->status_detail());
print "Admin Role object modified successfully \n";

#Accessor methods
print "name: " . $object->name() . "\n";
print "comment: " . $object->comment() . "\n";
```

#Remove an Admin Role object

```
#Get the Admin Role object through the session
my @retrieved_objs = $session->get(
    object => "Infoblox::Grid::Admin::Role",
    name   => "testrole_renamed"
);
my $object = $retrieved_objs[0];
unless ($object) {
    die("Get Admin Role object failed: ",
        $session->status_code() . ":" . $session->status_detail());
}
print "Get Admin Role object found at least 1 matching entry\n";

#Submit the object for removal
$session->remove($object)
       or die("Remove Admin Role object failed: ",
               $session->status_code() . ":" . $session->status_detail());
print "Admin Role object removed successfully \n";

####PROGRAM ENDS####
```

AUTHOR

Infoblox Inc. http://www.infoblox.com/

SEE ALSO

Infoblox::Grid::Admin::Group, Infoblox::Grid::Admin::Permission, Infoblox::Grid::Admin::User, Infoblox::Session->add(), Infoblox::Session->get(), Infoblox::Session->modify(), Infoblox::Session->remove(), Infoblox::Session->search(),Infoblox::Session

---

COPYRIGHT

Copyright (c) 2017 Infoblox Inc.

Infoblox::Grid::Admin::User - User Admin object.

- NAME
- DESCRIPTION
- CONSTRUCTOR
- SESSION METHODS
  - Infoblox::Session->add( )
  - Infoblox::Session->get( )
  - Infoblox::Session->modify( )
  - Infoblox::Session->remove( )
  - Infoblox::Session->search( )
- METHODS
  - admin_group( )
  - auth_type( )
  - ca_certificate_issuer( )
  - client_certificate_serial_number( )
  - comment( )
  - disabled( )
  - email( )
  - enable_certificate_authentication( )
  - extattrs( )
  - extensible_attributes( )
  - name( )
  - password( )
  - roles( )
  - time_zone( )
- SAMPLE CODE
- AUTHOR
- SEE ALSO
- COPYRIGHT

NAME

Infoblox::Grid::Admin::User - User Admin object.

DESCRIPTION

An admin account provides access to the Infoblox appliance. An admin account inherits the privileges and properties of the group to which it belongs.

CONSTRUCTOR

```
 my $user = Infoblox::Grid::Admin::User->new(
     admin_group                       => $string,
# Required
     auth_type                         => 'LOCAL' | 'REMOTE',
# Optional / Default is 'LOCAL'
     name                              => $string,
# Required
     password                          => $string,
# Required
     ca_certificate_issuer             => $string,
# Optional / Default is undefined
     client_certificate_serial_number  => $string,
# Optional / Default is undefined
     comment                           => $string,
# Optional / Default is undefined
     disabled                          => "true" | "false",
# Optional / Default is "false"
     enable_certificate_authentication => "true" | "false",
# Optional / Default is undefined
     extattrs                          => { $string => $extattr, ... },
# Optional / Default is undefined
     extensible_attributes             => { $string => $string | $num, $string => [ $string | $num, ... ],
... } # Optional / Default is undefined
     email                             => $email,
# Optional / Default is undefined
     time_zone                         => $TimeZone | undef
# Optional / Default value is undefined
 );
```

You cannot set both extattrs and extensible_attributes attributes at the same time.

SESSION METHODS

This section describes all the methods in the Infoblox::Session module that you can apply to an Admin User object.

Infoblox::Session->add( )

Use this method to add an object to the Infoblox appliance. See Infoblox::Session->add() for parameters and return values.

Example

```
#Construct the user object
my $user = Infoblox::Grid::Admin::User->new(
    admin_group => "admin-group",
    name        => "testuser",
    password    => "infoblox",
    comment     => "test user account"
    disabled    => "true",
    email       => "testuser\@test.com",
);
# Submit for addition
my $response = $session->add( $user );
```

Infoblox::Session->get( )

Use this method to retrieve all the matching objects from the Infoblox appliance. See Infoblox::Session->get() for parameters and return values.

Key References

```
Apply the following attributes to get a specific User object:

 name                            - Optional. Optional. The name of the user in string format.
 extattrs                        - Optional. A hash reference containing extensible attributes.
 extensible_attributes           - Optional. A hash reference containing extensible attributes.
 role                            - Optional. A role in string format or undef.
 ca_certificate_issuer           - Optional. The name of the CA certificate issuer in string
format.
 client_certificate_serial_number - Optional. The serial number of the client certificate in string
format.
```

When searching by role, admins assigned to the specified role, regardless of group, will be retrieved. If undef is passed as a get parameter for the role attribute, users that have no assigned roles, regardless of admin group, will be retrieved.

Note that name and role cannot be specified at the same time.

Example

```
my @retrieved_objs = $session->get(
    object      => "Infoblox::Grid::Admin::User",
    name        => "testuser" );
```

Infoblox::Session->modify( )

Use this method to modify an object in the Infoblox appliance. See Infoblox::Session->modify() for parameters and return values.

Example

```
# Use method to modify the comment.
$user->comment("This is a modified comment");
# Submit modification
my $response = $session->modify( $user );
```

Infoblox::Session->remove( )

Use this method to remove an object from the Infoblox appliance. See Infoblox::Session->remove() for parameters and return values.

To remove a specific object, use `get()` or `search()` to retrieve the specific object first, and then submit this object for removal.

Example

```
# Get the user objects with the same starting name
my @retrieved_objs = $session->get(
    object      => "Infoblox::Grid::Admin::User",
    name        => "testuser" );
# find the desired object from the retrieved list.
my $desired_user = $retrieved_objs[0];
# Submit for removal
my $response = $session->remove( $desired_user );
```

Infoblox::Session->search( )

Use this method to search for Admin User objects in the Infoblox appliance. See Infoblox::Session->search() for parameters and return values.

Key References

```
 Apply the following attributes to search for a specific User object:

  name                              - Optional. The name of the user in string format (regular
expression).
  admin_group                       - Optional. The name of the admin group in string format.
  extattrs                          - Optional. A hash reference containing extensible attributes.
  extensible_attributes             - Optional. A hash reference containing extensible attributes.
  role                              - Optional. A role in string format or undef.
  ca_certificate_issuer             - Optional. The name of the CA certificate issuer in string
format.
  client_certificate_serial_number - Optional. The serial number of the client certificate in string
format (regular expression).
```

When searching by role, the matching users will be users member of a group that has the specified role. If undef is passed as a search parameter for role, matching users will be ones belonging to groups with no roles assigned.

Note that name and role cannot be specified at the same time.

For more information about searching extensible attributes, see Infoblox::Grid::ExtensibleAttributeDef/Searching Extensible Attributes.

Example

```
# search for all User objects that start with "test"
my @retrieved_objs = $session->search(
    object      => "Infoblox::Grid::Admin::User",
    name        => "test.*",
    admin_group => "admin-group" );
```

METHODS

This section describes all the methods that you can use to set and retrieve the attribute values of a User object.

admin_group( )

Use this method to set or retrieve the admin_group.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

Admin group to which the user belongs. A user can belong to only one admin group at a time.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get admin group
my $admin_group = $user->admin_group();
#Modify name
$user->name("admin-group");
```

auth_type( )

Use this method to set or retrieve the user authentication type.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

The valid values are 'LOCAL' and 'REMOTE'. The default value is 'LOCAL'.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get auth_type value
my $auth_type = $user->auth_type();
#Modify auth_type value
$user->auth_type('REMOTE');
```

ca_certificate_issuer( )

Use this method to set or retrieve the name of the CA certificate issuer that is used for user lookup during authentication.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

The name of the CA certificate issuer that is used for user lookup during authentication in string format.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get ca_certificate_issuer
my $ca_certificate_issuer = $user->ca_certificate_issuer();

#Modify ca_certificate_issuer
$user->ca_certificate_issuer("issuer");
```

client_certificate_serial_number( )

Use this method to set or retrieve a parameter that is used in pair with the name of the CA certificate issuer.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

The parameter in string format that is used in pair with the name of the CA certificate issuer. Serial number can be either integer or hex. Serial number is unique for a particular issuer.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get client_certificate_serial_number
my $client_certificate_serial_number = $user->client_certificate_serial_number();

#Modify client_certificate_serial_number
$user->client_certificate_serial_number("11223344");
```

comment( )

Use this method to set or retrieve a comment.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

Pertinent information about the administrator, such as location or department. Comment in string format with a maximum of 256 bytes.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get comment
my $comment = $user->comment();
#Modify comment
$user->comment("This is the modified comment for admin user");
```

disabled( )

Use this method to set or retrieve the "disabled" flag.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

The default value for this field is false. The admin is enabled.

Parameter

Specify "true" to set the disable flag or "false" to deactivate/unset it. The default value is "false".

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get disabled
my $disabled = $user->disabled();
#Modify disabled
$user->disabled("true");
```

email( )

Use this method to set or retrieve the e-mail address of an administrator.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

The e-mail address of an administrator. The default value is undefined.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get email
my $email = $user->email();
#Modify email
$user->email("usertest\@infoblox.com");
```

enable_certificate_authentication( )

Use this method to set or retrieve the flag that indicates whether the user is allowed to log in only with the certificate. Regular username/password authentication will be disabled for this user.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

Specify 'true' to enable certificate-only user authentification and 'false' to disable it. The default value is 'false'.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get enable_certificate_authentication
my $value = $object->enable_certificate_authentication();

#Modify enable_certificate_authentication
$object->enable_certificate_authentication('true');
```

extattrs( )

Use this method to set or retrieve the extensible attributes associated with an User object.

Parameter

Valid value is a hash reference containing the names of extensible attributes and their associated values (Infoblox::Grid::Extattr objects).

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get extattrs
my $ref_extattrs = $user->extattrs();
#Modify extattrs
$user->extattrs({ 'Site' => $extattr1, 'Administrator' => $extattr2 });
```

extensible_attributes( )

Use this method to set or retrieve the extensible attributes associated with a User object.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

For valid values for extensible attributes, see Infoblox::Grid::ExtensibleAttributeDef/Extensible Attribute Values.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get extensible attributes
my $ref_extensible_attributes = $user->extensible_attributes();
#Modify extensible attributes
$user->extensible_attributes({ 'Site' => 'Santa Clara', 'Administrator' => [ 'Peter', 'Tom' ] });
```

name( )

Use this method to set or retrieve the name of an administrator.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

Text with the name of the admin.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get name
my $name = $user->name();
#Modify name
$user->name("test_user");
```

password( )

Use this method to set the password of an administrator. This is a write-only attribute.

Parameter

Password for the administrator to use when logging in.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

Example

```
#Modify password
$user->password("infobloxone");
```

roles( )

Use this method to retrieve the list of roles assigned to the admin's group.

Parameter

None

Returns

The method returns a reference to an array of strings of the role names.

Example

```
#Get the roles
my $roles = $user->roles();
```

time_zone( )

Use this method to set or retrieve the time zone of the administrator.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

The UTC string that represents the time zone. For example "(UTC - 6:00) Central Time (US and Canada)". Default value is undefined.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get time zone
my $time_zone = $user->time_zone();
#Modify time zone
$user->time_zone("(UTC - 6:00) Central Time (US and Canada)");
#Undef time zone
$user->time_zone(undef);
```

SAMPLE CODE

The following sample code demonstrates the different functions that can be applied to an object, such as add, search, modify, and remove. This sample also includes error handling for the operations.

#Preparation prior to an Admin User object insertion

```
#PROGRAM STARTS: Include all the modules that will be used
use strict;
use Infoblox;

#Create a session to the Infoblox appliance

my $session = Infoblox::Session->new(
               master   => "192.168.1.2", #appliance host ip
               username => "admin",       #appliance user login
               password => "infoblox"     #appliance password
               );

unless ($session) {
       die("Construct session failed: ",
               Infoblox::status_code() . ":" . Infoblox::status_detail());
}
print "Session created successfully\n";
```

#Create an Admin User object

```
my $user = Infoblox::Grid::Admin::User->new(
    name        => "testuser",
    password    => "infoblox",
    admin_group => "admin-group",
    email       => "testuser\@test.com",
    disabled    => "false",
```

```
    comment     => "test user account"
);

unless ($user) {
       die("Construct user object failed: ",
               Infoblox::status_code() . ":" . Infoblox::status_detail());
}
print "User object created successfully\n";

#Add the user object to the Infoblox appliance through a session
$session->add($user)
       or die("Add User object failed: ",
                       $session->status_code() . ":" . $session->status_detail());
print "Admin User object added to server successfully\n";
```

#Search for all users

```
my @retrieved_objs = $session->search(
               object      => "Infoblox::Grid::Admin::User",
               name        => "test.*",
               admin_group => "admin-group"
             );
my $object = $retrieved_objs[0];

unless ($object) {
       die("Search User object failed: ",
               $session->status_code() . ":" . $session->status_detail());
}
print "Search Admin User object found at least 1 matching entry\n";
```

#Get and modify a User object

```
#Get the user object from Infoblox appliance through a session
my @retrieved_objs = $session->get(
            object      => "Infoblox::Grid::Admin::User",
            name        => "testuser",
);
my $object = $retrieved_objs[0];

unless ($object) {
       die("Get User object failed: ",
               $session->status_code() . ":" . $session->status_detail());
}
print "Get Admin User object found at least 1 matching entry\n";

#Modify the password of the User object
$object->password("infoblox123");

#Apply the changes.
$session->modify($object)
       or die("Modify User object failed: ",
               $session->status_code() . ":" . $session->status_detail());
print "User object modified successfully \n";
```

#Remove the User object

```
#Get the User object through the session
my @retrieved_objs = $session->get(
            object      => "Infoblox::Grid::Admin::User",
            name        => "testuser",
);
my $object = $retrieved_objs[0];
unless ($object) {
    die("Get User object failed: ",
        $session->status_code() . ":" . $session->status_detail());
}
print "Get User object found at least 1 matching entry\n";

#Submit the object for removal
$session->remove($object)
       or die("Remove User object failed: ",
               $session->status_code() . ":" . $session->status_detail());
print "Admin User object removed successfully \n";

####PROGRAM ENDS####
```

Infoblox::Grid::Admin::User - User Admin object.

AUTHOR

Infoblox Inc. http://www.infoblox.com/

SEE ALSO

Infoblox::Grid::Admin::Group,Infoblox::Session->add(), Infoblox::Session->get(), Infoblox::Session->modify(), Infoblox::Session->remove(), Infoblox::Session->search(), Infoblox::Session

---

COPYRIGHT

Copyright (c) 2017 Infoblox Inc.

Infoblox::Grid::Admin::AuthPolicy - Authentication Policy object

- NAME
- DESCRIPTION
- SESSION METHODS
  - Infoblox::Session->get( )
  - Infoblox::Session->modify( )
- METHODS
  - admin_groups( )
  - auth_services( )
  - default_group( )
  - usage_type( )
- SAMPLE CODE
- AUTHOR
- SEE ALSO
- COPYRIGHT

NAME

Infoblox::Grid::Admin::AuthPolicy - Authentication Policy object

---

DESCRIPTION

This object represents the grid authentication policy.

SESSION METHODS

This section describes all the methods in the Infoblox::Session module that you can apply to the Grid Authentication Policy object.

Infoblox::Session->get( )

Use this method to retrieve the grid authentication policy from the Infoblox appliance. See Infoblox::Session->get() for parameters and return values.

Key Reference

No parameters are required.

Example

```
my $auth_policy = $session->get(
        object => "Infoblox::Grid::Admin::AuthPolicy",
    );
```

Infoblox::Session->modify( )

Use this method to modify the grid authentication policy on the Infoblox appliance. See Infoblox::Session->modify() for parameters and return values.

Example

```
#Changing the default group
$auth_policy->default_group($group);
#Submit modification
my $response = $session->modify( $auth_policy );
```

METHODS

This section describes all the methods that you can use to set or retrieve the attribute values of the grid authentication policy.

admin_groups( )

Use this method to set or retrieve the list of local admin groups that are mapped to remote administration groups. The default value is an empty list.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

The valid value is an array reference that contains Infoblox::Grid::Admin::Group objects.

Returns

If you specified a parameter, the method returns "true" when the modification succeeds, and returns "false" when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get the admin groups list
my $admin_groups = $auth_policy->admin_groups();

#Modify the admin_groups
$auth_policy->admin_groups([$admingroup1,$admingroup2]);
```

auth_services( )

Use this method to set or retrieve the list of authentication services that grid members use to authenticate admins. The default is a list that contains the LocalUserAuthService object.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

The valid value is an array reference that contains an ordered list of Infoblox::Grid::Admin::AdAuthService, Infoblox::Grid::Admin::LocalUserAuthService, Infoblox::Grid::Admin::TACACSPlusAuthService, Infoblox::LDAP::AuthService, Infoblox::Grid::Admin::CertificateAuthService, Infoblox::Grid::Admin::RadiusAuthService objects.

Returns

If you specified a parameter, the method returns "true" when the modification succeeds, and returns "false" when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get the services list
my $auth_services = $auth_policy->auth_services();
#Modify the auth_services
$auth_policy->auth_services([$radius1, $local, $ad2]);
```

default_group( )

Use this method to set or retrieve the default admin group that provides authentication in case no valid group is found. The default value is none (represented by undef).

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

The valid value is an Infoblox::Grid::Admin::Group object.

Returns

If you specified a parameter, the method returns "true" when the modification succeeds, and returns "false" when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get the default group
my $default_group = $auth_policy->default_group();

#Modify the default_group
$auth_policy->default_group($group2);
```

usage_type( )

Use this method to set or retrieve remote policies usage.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

Specify 'FULL' to define the list of remote users, their passwords and their group ownership, and specify 'AUTH_ONLY' to validate passwords of local users only. The default value is 'FULL'.

Returns

If you specified a parameter, the method returns "true" when the modification succeeds, and returns "false" when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get usage_type value
my $usage_type = $auth_policy->usage_type();

#Modify usage_type value
$auth_policy->usage_type('AUTH_ONLY');
```

SAMPLE CODE

The following sample code demonstrates the different functions that can be applied to an object, such as get and modify. This sample code also includes error handling for the operations.

#Preparation prior to getting and modifying an auth policy object

```
use strict;
use Infoblox;

#Creating a session to appliance.
my $session = Infoblox::Session->new(
    master   => "192.168.1.2",
    username => "admin",
    password => "infoblox"
    );
unless( $session ){
    die("Constructor for session failed:",
    Infoblox::status_code() . ":" . Infoblox::status_detail());
}
print" Session object created successfully \n";
```

#Retrieving the auth policy object

```
my $policy = $session->get(
                          object => 'Infoblox::Grid::Admin::AuthPolicy',
                         ) || die("Retrieval of the auth policy failed: ",
                Infoblox::status_code() . ":" . Infoblox::status_detail());
```

#Retrieving the local user auth service

```
my $local_auth = $session->get(
                               object => 'Infoblox::Grid::Admin::LocalUserAuthService',
                              ) || die("Retrieval of the local user auth service failed: ",
                Infoblox::status_code() . ":" . Infoblox::status_detail());
```

#Creation of a Radius auth server/service pair

```
my $ads = Infoblox::Grid::Admin::RadiusAuthServer->new(
                                                        fqdn_or_ip => '10.1.0.1',
                                                        shared_secret => 'secret',
                                                       ) ||
  die("Couldn't create the auth server: ",
      Infoblox::status_code() . ":" . Infoblox::status_detail());

my $service = Infoblox::Grid::Admin::RadiusAuthService->new(
                                                             acct_timeout => 1100,
                                                             auth_timeout => 1100,
                                                             name => 'some.name.com',
                                                             radius_servers => [$ads]
                                                            ) ||
  die("Couldn't create the auth service: ",
      Infoblox::status_code() . ":" . Infoblox::status_detail());

$session->add($service) || die("Couldn't add the auth service: ",
        Infoblox::status_code() . ":" . Infoblox::status_detail());
```

#Retrieving the auth service created above

```
my $radius_auth = $session->get(
```

```
                                object => 'Infoblox::Grid::Admin::RadiusAuthService',
                                name   => 'some.name.com',
                            ) || die("Retrieval of the radius auth service failed: ",
                        Infoblox::status_code() . ":" . Infoblox::status_detail());
```

#Modify the authpolicy

```
$policy->auth_services([$radius_auth,$local_auth]);

$session->modify($policy) || die("Policy modify failed: ",
        Infoblox::status_code() . ":" . Infoblox::status_detail());

####PROGRAM ENDS####
```

AUTHOR

Infoblox Inc. http://www.infoblox.com/

SEE ALSO

Infoblox::Grid::Admin::Group, Infoblox::Grid::Admin::AdAuthService, Infoblox::Grid::Admin::TACACSPlusAuthService, Infoblox::Grid::Admin::LocalUserAuthService, Infoblox::Grid::Admin::RadiusAuthService, Infoblox::LDAP::AuthService, Infoblox::Grid::Admin::CertificateAuthService, Infoblox::Session->get(), Infoblox::Session->get()

COPYRIGHT

Copyright (c) 2017 Infoblox Inc.

Infoblox::Grid::Admin::RadiusAuthServer - RADIUS Authentication Server object

- NAME
- DESCRIPTION
- CONSTRUCTOR
- SESSION METHODS
  - Infoblox::Grid::Admin::RadiusAuthService->radius_servers()
- METHODS
  - acct_enabled( )
  - acct_port( )
  - auth_port( )
  - auth_type( )
  - comment( )
  - disabled( )
  - fqdn_or_ip( )
  - shared_secret( )
  - use_mgmt_port( )
- AUTHOR
- SEE ALSO
- COPYRIGHT

NAME

Infoblox::Grid::Admin::RadiusAuthServer - RADIUS Authentication Server object

DESCRIPTION

Represents a RADIUS server that is used to authenticate administrators and DHCP clients.

CONSTRUCTOR

```
my $radius_auth_server = Infoblox::Grid::Admin::RadiusAuthServer->new(
    acct_enabled     => "true" | "false"      #Optional / Default is false
    acct_port        => $num,                 #Optional / Default is 1813
    auth_port        => $num,                 #Optional / Default is 1812
    auth_type        => "PAP" | "CHAP",       #Optional / Default is "PAP"
    comment          => $string,              #Optional / Default is empty
    disabled         => "true" | "false"      #Optional / Default is false
    fqdn_or_ip       => $fqdn | $ipv4addr     #Required
    shared_secret    => $string,              #Required
    use_mgmt_port    => "true" | "false"      #Optional / Default is false
);
```

SESSION METHODS

This section describes all the methods in an Infoblox::Session module that you can apply to an object.

Infoblox::Grid::Admin::RadiusAuthService->radius_servers()

Use this method to add a list of RADIUS Server objects to the RADIUS Service object on an Infoblox appliance. See Infoblox::Grid::Admin::RadiusAuthService for parameters and return values.

Example

```
# Construct an object
my $radius_auth_server1 = Infoblox::Grid::Admin::RadiusAuthServer->new(
                                                    fqdn_or_ip    => '10.0.1.1',
                                                    shared_secret => 'secret',
);

# Construct another object
my $radius_auth_server2 = Infoblox::Grid::Admin::RadiusAuthServer->new(
                                                    fqdn_or_ip    => '10.0.2.1',
                                                    shared_secret => 'secret',
);

# Configure RadiusAuthServer
my $response = $radius_auth_service->radius_servers([$radius_auth_server1, $radius_auth_server2]);
```

METHODS

Use the following methods to access the attributes of an Infoblox::Grid::Admin::RadiusAuthServer object. Specify a parameter to set the attribute's value, or omit the parameter to get the attribute's value.

acct_enabled( )

Use this method to set or retrieve the flag that indicates if the RADIUS server is enabled for accounting packets.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

Specify "true" to set the disable flag or "false" to deactivate/unset it. The default value is "false".

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get acct_enabled
my $acct_enabled = $radius_server->acct_enabled();
#Modify acct_enabled
$radius_server->acct_enabled("true");
```

acct_port( )

Use this method to set or retrieve the port number used by RADIUS server for accounting packets.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

An unsigned integer between 1 and 65535, inclusive.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Getting acct_port
my $acct_port = $radius_server->acct_port( );
#Modifying acct_port
$radius_server->acct_port("1813");
```

auth_port( )

Use this method to set or retrieve the port number used by the RADIUS server for accounting packets.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

An unsigned integer between 1 and 65535, inclusive.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Getting auth_port
my $auth_port = $radius_server->auth_port( );
#Modifying auth_port
$radius_server->auth_port("1812");
```

auth_type( )

Use this method to set or retrieve the authentication type used by the RADIUS client and server.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

The valid value is either "PAP" or "CHAP", the default is "PAP".

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Getting auth_type
my $auth_type = $radius_server->auth_type( );
#Modifying auth_type
$radius_server->auth_type("PAP");
```

comment( )

Use this method to set or retrieve a descriptive comment about the RADIUS server.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

The comment in string format, with a maximum of 256 characters.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get comment
my $comment = $radius_server->comment();
#Modify comment
$radius_server->comment("NAC Authentication RADIUS server");
```

disabled( )

Use this method to set or retrieve the disable flag of the RADIUS server.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

Specify "true" to set the disable flag or "false" to deactivate/unset it. The default value is "false".

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get disabled
my $disable = $radius_server->disabled();
#Modify disabled
$radius_server->disabled("true");
```

fqdn_or_ip( )

Use this method to set or retrieve the FQDN or IP address of the RADIUS server.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

A string in FQDN (Fully Qualified Domain Name) format, or an IP address.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get fqdn_or_ip
my $fqdn_or_ip = $radius_server->fqdn_or_ip();
#Modify fqdn_or_ip
$radius_server->fqdn_or_ip("10.0.0.1");
```

shared_secret( )

Use this method to set the shared secret used by the RADIUS client and RADIUS server. This is a write-only attribute.

Include the specified parameter to set the attribute value.

Parameter

The shared secret in string format, with a maximum of 128 characters.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

Example

```
#Modify shared_secret
$radius_server->shared_secret("secret");
```

use_mgmt_port( )

Use this method to set or retrieve the flag that indicates if the appliance connects to the RADIUS server through the MGMT network interface.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

Specify "true" to set the flag or "false" to deactivate/unset it. The default value is "false".

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get use_mgmt_port
my $use_mgmt_port = $radius_server->use_mgmt_port();
#Modify use_mgmt_port
$radius_server->use_mgmt_port("true");
```

AUTHOR

Infoblox Inc. http://www.infoblox.com

SEE ALSO

Infoblox::Grid::Admin::RadiusAuthService, Infoblox::Session, Infoblox::Session->add(), Infoblox::Session->get(), Infoblox::Session->modify(), Infoblox::Session->remove()

COPYRIGHT

Copyright (c) 2017 Infoblox Inc.

Infoblox::Grid::Admin::Permission - Admin Permission object.

- NAME
- DESCRIPTION
- CONSTRUCTOR
- SESSION METHODS
  - Infoblox::Session->add( )
  - Infoblox::Session->get( )
  - Infoblox::Session->modify( )
  - Infoblox::Session->remove( )
  - Infoblox::Session->search( )
- METHODS
  - admin_group( )
  - role( )
  - permission( )
  - resource_type( )
  - resource_object( )
- SAMPLE CODE
- AUTHOR
- SEE ALSO
- COPYRIGHT

NAME

Infoblox::Grid::Admin::Permission - Admin Permission object.

DESCRIPTION

An Admin Permission object is used to set the permissions for an admin to access various resources.

CONSTRUCTOR

```
 my $group = Infoblox::Grid::Admin::Permission->new(
     admin_group          => $string,                                    #exactly
one of admin_group and role Required
     role                 => $string,                                    #exactly
one of admin_group and role Required
     permission           => "read" | "write" | "deny",                  #Required
     resource_type        => $string,
#Optional, if resource_object is provided
     resource_object      => $Object,
#Optional, if resource_type is provided
 );
```

SESSION METHODS

This section describes all the methods in the Infoblox::Session module that you can apply to an Admin Permission object.

Infoblox::Session->add( )

Use this method to add an object to the Infoblox appliance. See Infoblox::Session->add() for parameters and return values.

Example

```
       #Construct the Group object
       my $permission = Infoblox::Grid::Admin::Permission->new(
                 admin_group => "test_group",
                 resource_type => "All Zones",
                 permission => "write",
       );
       # Submit for addition
       my $response = $session->add( $permission );
```

Infoblox::Session->get( )

Use this method to retrieve all the matching objects from the Infoblox appliance. See Infoblox::Session->get() for parameters and return values.

Key References

```
Apply the following attributes to get a specific Group object:

 admin_group      - Optional. Group name in string format.
 role             - Optional. Role name in string format.
 permission       - Optional. Permission in string format.
 resource_type    - Optional. Resource type in string format.
 resource_object  - Optional. Resource Object reference.

NOTE: You cannot specify both an admin_group and role.
```

Example

```
my @retrieved_objs = $session->get(
    object     => "Infoblox::Grid::Admin::Permission",
    admin_group   => "test_group",
    resource_type => "All Members");

or

my @retrieved_objs = $session->get(
    object     => "Infoblox::Grid::Admin::Permission",
    role          => "test_role",
    resource_type => "All Members");
```

Infoblox::Session->modify( )

Use this method to modify an object in the Infoblox appliance. See Infoblox::Session->modify() for parameters and return values.

Example

```
# Use this method to modify the permission.
$permission->permission("read");
# Submit modification
my $response = $session->modify( $permission );
```

Infoblox::Session->remove( )

Use this method to remove an object from the Infoblox appliance. See Infoblox::Session->remove() for parameters and return values.

To remove a specific object, use `get()` or `search()` to retrieve the specific object first, and then submit this object for removal.

Example

```
# Get the permission objects with the same group name
my @retrieved_objs = $session->get(
    object     => "Infoblox::Grid::Admin::Permission",
    admin_group   => "test_group",
    resource_type => "All Members");
# find the desired object from the retrieved list.
my $desired_perm = $retrieved_objs[0];
# Submit for removal
my $response = $session->remove( $desired_perm );

or

# Get the permission objects with the same role name
my @retrieved_objs = $session->get(
    object     => "Infoblox::Grid::Admin::Permission",
    role          => "test_role",
    resource_type => "All Members");
# find the desired object from the retrieved list.
my $desired_perm = $retrieved_objs[0];
# Submit for removal
my $response = $session->remove( $desired_perm );
```

Infoblox::Session->search( )

Use this method to search for Admin Permission objects in the Infoblox appliance. See Infoblox::Session->search() for parameters and return values.

Key References

```
Apply the following attributes to search for a specific Group object:

 admin_group          - Optional. Group name in string format.
 role                 - Optional. Role name in string format.
 permission           - Optional. Permission in string format.
 resource_object      - Optional. Resource Object reference.
 resource_type        - Optional. Resource Type in string format.

NOTE: You cannot specify both an admin group and role.
```

Example

```
# search for all permission objects of admin group "test_group"
my @retrieved_objs = $session->search(
    object => "Infoblox::Grid::Admin::Permission",
    admin_group   => "test_group" );

or

# search for all permission objects of admin role "test_role"
my @retrieved_objs = $session->search(
    object => "Infoblox::Grid::Admin::Permission",
    role   => "test_role" );
```

METHODS

This section describes all the methods that you can use to set and retrieve the attribute values of a Group object.

admin_group( )

Use this method to set or retrieve the admin group information.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

The admin group name for the specified object.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get admin group name
my $name = $permission->admin_group();
#Modify admin group
$permission->admin_group("test-group1");
```

role( )

Use this method to set or retrieve the role information.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

The admin role name for the specified object.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get admin role name
my $name = $permission->role();
#Modify admin role
$permission->role("test-role1");
```

permission( )

Use this method to set or retrieve the admin group permission.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

The admin group permission for the specified object. Permission values are "write", "read" and "deny".

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get permission
my $perm = $permission->permission();
#Modify permission
$permission->permission("deny");
```

resource_type( )

Use this method to set or retrieve the resource type. Resource type represents all resources of the specific type.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

The resource type for which permission is set. Sets permissions for all resources of the resource_type specified.

If resource_object is also provided, it sets the permission for all resources of the specified type under the resource_object.

The allowed values are:

```
"All Zones"
"All DNS Views"
"All DNS64 Synthesis Groups"
"All Members"
"All MS Server Members"
"All Microsoft Active Directory Domains"
"All Microsoft Superscopes"
"All Shared Record Groups"
"All Networks" (deprecated in favor of "All IPv4 Networks")
"All IPv4 Networks"
"All IPv6 Networks"
"All Network Views"
"All Shared Networks" (deprecated in favor of "All IPv4 DHCP Shared Networks")
"All IPv4 DHCP Shared Networks"
"All IPv6 DHCP Shared Networks"
"All Mac Address Filters"
"All Ranges" (deprecated in favor of "All IPv4 Ranges")
"All IPv4 DHCP Ranges" (deprecated in favor of "All IPv4 Ranges")
"All IPv4 Ranges"
"All IPv6 DHCP Ranges"  (deprecated in favor of "All IPv6 Ranges")
"All IPv6 Ranges"
"All Fixed Addresses/Reservations" (deprecated in favor of "All IPv4 DHCP Fixed
Addresses/Reservations")
"All IPv4 DHCP Fixed Addresses/Reservations"
"All IPv6 DHCP Fixed Addresses"
"All Roaming Hosts"
"All Rulesets"
"All Templates" (deprecated in favor of "All IPv4 DHCP Templates")
"All IPv4 DHCP Templates"
"All IPv6 DHCP Templates"
"All IPv4 Host Addresses"
"All IPv6 Host Addresses"
"All A Records"
"All AAAA Records"
"All Alias Records"
"All BULKHOST Records"
"All CNAME Records"
"All DNAME Records"
"All HOST Records"
"All MX Records"
"All NAPTR Records"
"All PTR Records"
"All SRV Records"
"All TLSA Records"
"All TXT Records"
"All Shared A Records"
"All Shared AAAA Records"
"All Shared MX Records"
"All Shared SRV Records"
"All Shared TXT Records"
"All Shared CNAME Records"
"DNS Scavenging"
"Grid DHCP properties"
"Grid DNS properties"
"Lease History Access" (deprecated in favor of "All IPv4 DHCP Lease History")
"All IPv4 DHCP Lease History"
"All IPv6 DHCP Lease History"
"Grid File Distribution Properties"
"Grid AAA Properties"
"All AAA External Services"
"Network Discovery"
"Restart"
"All HSM Groups"
"All Certificate Auth Services"
"All CA Certificates"
"All CA Certificates"
"All Response Policy Zones"
"All Response Policy Rules"
```

```
"All CSV Import Tasks"
"All Dashboard Tasks"
"All DHCP Fingerprints"
"All Named ACLs"
"All Kerberos Keys"
"All BFD Templates"
"Grid Security configuration"
"Port Control"
"All DTC LBDNs"
"All DTC LBDN Records"
"All DTC Pools"
"All DTC Servers"
"All DTC Monitors"
"All DTC Topologies"
"All DTC GeoIP"
"All DTC Certificates"
"Grid Reporting properties"
"Reporting dashboard"
"Reporting search"
"Allow adding A/AAAA records with empty hostname"
"Allow deleted objects information tracking"
"All Parental Control accounting AVPs"
"Parental Control Properties"
"All Parental Control Subscriber Sites"
"Scheduling"
```

The "Restart" resource_type is supported only when resource_object is an Infoblox::Grid::Member::DHCP or Infoblox::Grid::Member::DNS object.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get resource_type
my $resource_type = $permission->resource_type();
#Modify resource_type
$permission->resource_type("All Networks");
```

resource_object( )

Use this method to set or retrieve a resource object. Resource object is a reference to a single object.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

The resource object for which permission is set. Sets the permission for the specified resource object only.

If resource_type is also provided, it sets the permission for all resources of the specified type under the resource_object.

The value must be a reference of a defined object of any of the following types:

```
Infoblox::DHCP::Filter::MAC
Infoblox::DHCP::FixedAddr
Infoblox::DHCP::FixedAddrTemplate
Infoblox::DHCP::IPv6FixedAddr
Infoblox::DHCP::IPv6Network
Infoblox::DHCP::IPv6NetworkContainer
Infoblox::DHCP::IPv6Range
Infoblox::DHCP::IPv6SharedNetwork
Infoblox::DHCP::Network
Infoblox::DHCP::NetworkContainer
Infoblox::DHCP::NetworkTemplate
Infoblox::DHCP::Range
Infoblox::DHCP::RangeTemplate
Infoblox::DHCP::RoamingHost
Infoblox::DHCP::SharedNetwork
Infoblox::DHCP::View
Infoblox::DNS::BulkHost
Infoblox::DNS::Host
Infoblox::DNS::Record::A
Infoblox::DNS::Record::AAAA
Infoblox::DNS::Record::Alias
Infoblox::DNS::Record::CNAME
Infoblox::DNS::Record::DNAME
Infoblox::DNS::Record::MX
Infoblox::DNS::Record::PTR
Infoblox::DNS::Record::SRV
Infoblox::DNS::Record::TLSA
Infoblox::DNS::Record::TXT
Infoblox::DNS::Ruleset
Infoblox::DNS::SharedRecord::A
Infoblox::DNS::SharedRecord::AAAA
Infoblox::DNS::SharedRecord::MX
Infoblox::DNS::SharedRecord::SRV
Infoblox::DNS::SharedRecord::TXT
```

```
Infoblox::DNS::SharedRecord::CNAME
Infoblox::DNS::SRG
Infoblox::DNS::View
Infoblox::DNS::Zone
Infoblox::Grid::DNS::DNS64Group
Infoblox::Grid::FileDistributionDir
Infoblox::Grid::Member
Infoblox::Grid::Member::FileDistribution
Infoblox::Grid::MSServer
Infoblox::Grid::MSServer::AdSites::Domain
Infoblox::Grid::MSSuperscope
Infoblox::Grid::Member::ThreatProtection
Infoblox::Grid::Discovery::Device
```

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get resource_obejct
my $resource_object = $permission->resource_object();
#Modify resource_object
$object = Infoblox::DNS::Zone->new(
                name => "zone.com"
                );
$session->add($object);
$permission->resource_object($zone);
```

SAMPLE CODE

The following sample code demonstrates the different functions that can be applied to an object, such as add, search, modify, and remove. This sample also includes error handling for the operations.

#Preparation prior to an Admin Permission object insertion

```
#PROGRAM STARTS: Include all the modules that will be used
use strict;
use Infoblox;

#Create a session to the Infoblox appliance

my $session = Infoblox::Session->new(
               master   => "192.168.1.2", #appliance host ip
               username => "admin",       #appliance user login
               password => "infoblox"     #appliance password
               );

unless ($session) {
       die("Construct session failed: ",
               Infoblox::status_code() . ":" . Infoblox::status_detail());
}
print "Session created successfully\n";

#Create an admin group

my $group = Infoblox::Grid::Admin::Group->new(
    name                => "test_group",
    disabled            => "false",
    superuser           => "false"

);

unless ($group) {
       die("Construct user object failed: ",
               Infoblox::status_code() . ":" . Infoblox::status_detail());
}
print "Admin Group object created successfully\n";

#Add the Admin Group object to the Infoblox appliance through a session
$session->add($group)
       or die("Add Admin Group object failed: ",
                       $session->status_code() . ":" . $session->status_detail());
print "Admin Group object added to server successfully\n";

# Create an Admin Role

my $role = Infoblox::Grid::Admin::Role->new(
    name                => "test_role",
```

```
    comment              => "test admin role",
);

unless ($role) {
       die("Construct role object failed: ",
               Infoblox::status_code() . ":" . Infoblox::status_detail());
}
print "Admin Role object created successfully\n";

#Add the Admin Role object to the Infoblox appliance through a session
$session->add($role)
       or die("Add Admin Role object failed: ",
                       $session->status_code() . ":" . $session->status_detail());
print "Admin Role object added to server successfully\n";

#Create a zone
my $zone = Infoblox::DNS::Zone->new(
               name => "zone.com"
);

unless ($zone) {
       die("Construct user object failed: ",
               Infoblox::status_code() . ":" . Infoblox::status_detail());
}

#Add the Zone object to the Infoblox appliance through a session
$session->add($zone)
       or die("Add Zone object failed: ",
                       $session->status_code() . ":" . $session->status_detail());
print "Zone object added to server successfully\n";

#Create a shared record group (SRG)
my $srg = Infoblox::DNS::SRG->new(
               name => "mysrg"
);

unless ($srg) {
       die("Construct user object failed: ",
               Infoblox::status_code() . ":" . Infoblox::status_detail());
}

#Add the SRG object to the Infoblox appliance through a session
$session->add($srg)
       or die("Add SRG object failed: ",
                       $session->status_code() . ":" . $session->status_detail());
print "SRG object added to server successfully\n";
```

#Create an Admin Permission object based on Admin Group

```
#Create an admin permission object
my $group_permission = Infoblox::Grid::Admin::Permission->new(
   admin_group     => $group->name(),
   resource_object => $zone,  # This can be omitted if don't care about
                              # which zone
   resource_type => "All A Records",
   permission => "write");

unless ($group_permission) {
       die("Construct Admin Permission object failed: ",
               Infoblox::status_code() . ":" . Infoblox::status_detail());
}
print "Admin Permission object created successfully\n";

#Add the Admin Permission object to the Infoblox appliance through a session
$session->add($group_permission)
       or die("Add Admin Permission object failed: ",
                       $session->status_code() . ":" . $session->status_detail());
print "Admin Permission object added to server successfully\n";

#Create an admin permission object
my $scheduling_permission = Infoblox::Grid::Admin::Permission->new(
     admin_group => $group->name(),
     resource_type => "Scheduling",
     permission => "write");

unless ($scheduling_permission) {
       die("Construct Admin Permission object failed: ",
               Infoblox::status_code() . ":" . Infoblox::status_detail());
```

```
}
print "Admin Permission object created successfully\n";

#Add the Admin Permission object to the Infoblox appliance through a session
$session->add($scheduling_permission)
       or die("Add Admin Permission object failed: ",
                        $session->status_code() . ":" . $session->status_detail());
print "Admin Permission object added to server successfully\n";
```

#Create an Admin Permission object based on Admin Role

```
#Create an admin permission object
my $role_permission = Infoblox::Grid::Admin::Permission->new(
   role            => $role->name(),
   resource_object => $srg,  # This can be omitted if don't care about
                             # which shared record group
   resource_type => "All Shared A Records",
   permission => "write");

unless ($role_permission) {
       die("Construct user object failed: ",
               Infoblox::status_code() . ":" . Infoblox::status_detail());
}
print "Admin Permission object created successfully\n";

#Add the Admin Permission object to the Infoblox appliance through a session
$session->add($role_permission)
       or die("Add Admin Permission object failed: ",
                        $session->status_code() . ":" . $session->status_detail());
print "Admin Permission object added to server successfully\n";
```

#Search for an Admin Permission object based on Admin Group

```
my @retrieved_objs = $session->search(
               object => "Infoblox::Grid::Admin::Permission",
               admin_group   => "test_group"
               );
my $object = $retrieved_objs[0];

unless ($object) {
       die("Search Admin Permission object based on Admin Group failed: ",
               $session->status_code() . ":" . $session->status_detail());
}
print "Search Admin Permission object based on Admin Group found at least 1 matching entry\n";
```

#Search for an Admin Permission object based on Admin Role

```
my @retrieved_objs = $session->search(
               object => "Infoblox::Grid::Admin::Permission",
               role   => "test_role"
               );
my $object = $retrieved_objs[0];

unless ($object) {
       die("Search Admin Permission object based on Admin Role failed: ",
               $session->status_code() . ":" . $session->status_detail());
}
print "Search Admin Permission object based on Admin Role found at least 1 matching entry\n";
```

#Get and modify an Admin Permission object based on Admin Group

```
#Get the Admin Group object from the Infoblox appliance through a session
my @retrieved_objs = $session->get(
    object => "Infoblox::Grid::Admin::Permission",
    admin_group   => "test_group",
    resource_object => $zone,
    resource_type => "All A Records",
);
my $object = $retrieved_objs[0];

unless ($object) {
       die("Get Admin Permission object based on Admin Group failed: ",
               $session->status_code() . ":" . $session->status_detail());
}
print "Get Admin Permission object found at least 1 matching entry\n";

#Modify the permission of the Admin Permission object
```

```
$object->permission("read");

#Apply the changes.
$session->modify($object)
       or die("Modify Admin Permission object failed: ",
                $session->status_code() . ":" . $session->status_detail());
print "Admin Permission object modified successfully \n";
```

#Get Admin Permission object based on Admin Role

```
#Get the Admin Group object from the Infoblox appliance through a session
my @retrieved_objs = $session->get(
    object => "Infoblox::Grid::Admin::Permission",
    role             => "test_role",
    resource_object => $srg,
    resource_type => "All Shared A Records",
);
my $object = $retrieved_objs[0];

unless ($object) {
       die("Get Admin Permission object based on Admin Role failed: ",
                $session->status_code() . ":" . $session->status_detail());
}
print "Get Admin Permission object found at least 1 matching entry\n";
```

#Remove an Admin Permission object

```
#Get the Admin Permission object through the session
my @retrieved_objs = $session->get(
    object => "Infoblox::Grid::Admin::Permission",
    admin_group    => "test_group",
    resource_object => $zone,
    resource_type => "All A Records",
);
my $object = $retrieved_objs[0];
unless ($object) {
    die("Get Admin Permission object failed: ",
        $session->status_code() . ":" . $session->status_detail());
}
print "Get Admin Permission object found at least 1 matching entry\n";

#Submit the object for removal
$session->remove($object)
       or die("Remove Admin Permission object failed: ",
                $session->status_code() . ":" . $session->status_detail());
print "Admin Permission object removed successfully \n";

# We can remove the other permissions in the same way

#Get the Admin Permission object for Shared Resource Records through the session
@retrieved_objs = $session->get(
    object => "Infoblox::Grid::Admin::Permission",
    role    => "test_role",
    resource_object => $srg,    # Omit this if it was omitted during creation
    resource_type => "All Shared A Records",
);
$object = $retrieved_objs[0];
unless ($object) {
    die("Get Admin Permission object for Shared RR failed: ",
        $session->status_code() . ":" . $session->status_detail());
}
print "Get Admin Permission object found at least 1 matching entry\n";

#Submit the object for removal
$session->remove($object)
       or die("Remove Admin Permission object for Shared RR failed: ",
                $session->status_code() . ":" . $session->status_detail());
print "Admin Permission object for Shared RR removed successfully \n";
```

#Clean up

```
$session->remove($role)
       or die("Remove Admin Role object failed: ",
                $session->status_code() . ":" . $session->status_detail());
print "Admin Role object removed successfully \n";

$session->remove($group)
```

```
        or die("Remove Admin Group object failed: ",
                $session->status_code() . ":" . $session->status_detail());
print "Admin Group object removed successfully \n";

$session->remove($zone)
        or die("Remove Zone object failed: ",
                $session->status_code() . ":" . $session->status_detail());
print "Zone object removed successfully \n";

$session->remove($srg)
        or die("Remove Shared Record Group object failed: ",
                $session->status_code() . ":" . $session->status_detail());
print "Shared Record Group object removed successfully \n";

####PROGRAM ENDS####
```

AUTHOR

Infoblox Inc. http://www.infoblox.com/

SEE ALSO

Infoblox::Grid::Admin::Role, Infoblox::Grid::Admin::Group, Infoblox::Grid::Admin::User, Infoblox::Session->add(), Infoblox::Session->get(), Infoblox::Session->modify(), Infoblox::Session->remove(), Infoblox::Session->search(), Infoblox::Session

---

COPYRIGHT

Copyright (c) 2017 Infoblox Inc.

Infoblox::Grid::Admin::TACACSPlusAuthServer - TACACS+ Authentication Server object

- NAME
- DESCRIPTION
- CONSTRUCTOR
- SESSION METHODS
  - Infoblox::Grid::Admin::TACACSPlusAuthService->tacacsplus_servers()
- METHODS
  - acct_enabled( )
  - auth_type( )
  - comment( )
  - disabled( )
  - fqdn_or_ip( )
  - port( )
  - shared_secret( )
  - use_mgmt_port( )
- AUTHOR
- SEE ALSO
- COPYRIGHT

NAME

Infoblox::Grid::Admin::TACACSPlusAuthServer - TACACS+ Authentication Server object

---

DESCRIPTION

Represents a TACACS+ server that is used to authenticate administrators.

CONSTRUCTOR

```
my $tacacs_auth_server = Infoblox::Grid::Admin::TACACSPlusAuthServer->new(
    fqdn_or_ip       => $fqdn | $ipv4addr         #Required
    shared_secret    => $string,                  #Required
    acct_enabled     => "true" | "false"          #Optional / Default is false
    auth_type        => "PAP" | "CHAP" | "ASCII", #Optional / Default is "CHAP"
    comment          => $string,                  #Optional / Default is empty
    disabled         => "true" | "false"          #Optional / Default is false
    use_mgmt_port    => "true" | "false"          #Optional / Default is false
    port             => $num,                     #Optional / Default is 49
);
```

SESSION METHODS

This section describes all the methods in an Infoblox::Session module that you can apply to an object.

Infoblox::Grid::Admin::TACACSPlusAuthService->tacacsplus_servers()

Use this method to add a list of TACACS+ Server objects to the TACACS+ Service object on an Infoblox appliance. See Infoblox::Grid::Admin::TACACSPlusAuthService for parameters and return values.

Example

```
# Construct an object
my $tacacs_auth_server1 = Infoblox::Grid::Admin::TACACSPlusAuthServer->new(
                                                    fqdn_or_ip    => '10.0.1.1',
                                                    shared_secret => 'secret',
);

# Construct another object
my $tacacs_auth_server2 = Infoblox::Grid::Admin::TACACSPlusAuthServer->new(
                                                    fqdn_or_ip    => '10.0.2.1',
                                                    shared_secret => 'secret',
);

# Configure TACACSPlusAuthServer
my $response = $tacacs_auth_service->tacacs_servers([$tacacs_auth_server1, $tacacs_auth_server2]);
```

METHODS

Use the following methods to access the attributes of an Infoblox::Grid::Admin::TACACSPlusAuthServer object. Specify a parameter to set the attribute's value, or omit the parameter to get the attribute's value.

acct_enabled( )

Use this method to set or retrieve the flag that indicates if the TACACS+ server is enabled for accounting packets.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

Specify "true" to set the disable flag or "false" to deactivate/unset it. The default value is "false".

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get acct_enabled
my $acct_enabled = $tacacs_server->acct_enabled();
#Modify acct_enabled
$tacacs_server->acct_enabled("true");
```

auth_type( )

Use this method to set or retrieve the authentication type used by the TACACS+ client and server.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

The valid value is either "PAP", "ASCII" or "CHAP". The default is "CHAP".

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get auth_type
my $auth_type = $tacacs_server->auth_type( );
#Modifying auth_type
$tacacs_server->auth_type("PAP");
```

comment( )

Use this method to set or retrieve a descriptive comment about the TACACS+ server.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

The comment in string format, with a maximum of 256 characters.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get comment
my $comment = $tacacs_server->comment();
#Modify comment
$tacacs_server->comment("NAC Authentication TACACS+ server");
```

disabled( )

Use this method to set or retrieve the disable flag of the TACACS+ server.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

Specify "true" to set the disable flag or "false" to deactivate/unset it. The default value is "false".

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get disabled
my $disable = $tacacs_server->disabled();
#Modify disabled
$tacacs_server->disabled("true");
```

fqdn_or_ip( )

Use this method to set or retrieve the FQDN or IP address of the TACACS+ server.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

A string in FQDN (Fully Qualified Domain Name) format, or an IP address.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get fqdn_or_ip
my $fqdn_or_ip = $tacacs_server->fqdn_or_ip();
#Modify fqdn_or_ip
$tacacs_server->fqdn_or_ip("10.0.0.1");
```

port( )

Use this method to set or retrieve the destination port number on the TACACS+ server.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

An unsigned integer between 1 and 65535, inclusive.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get port
my $port = $tacacs_server->port( );
#Modifying port
$tacacs_server->port("49");
```

shared_secret( )

Use this method to set the shared secret that the NIOS appliance and the TACACS+ server use to encrypt and decrypt their messages. This is a write-only attribute.

Include the specified parameter to set the attribute value.

Parameter

The shared secret in string format, with a maximum of 64 characters.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

Example

```
#Modify shared_secret
$tacacs_server->shared_secret("secret");
```

use_mgmt_port( )

Use this method to set or retrieve the flag that indicates if the appliance connects to the TACACS+ server through the MGMT network interface.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

Specify "true" to set the flag or "false" to deactivate/unset it. The default value is "false".

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get use_mgmt_port
my $use_mgmt_port = $tacacs_server_server->use_mgmt_port();
#Modify use_mgmt_port
$tacacs_server->use_mgmt_port("true");
```

AUTHOR

Infoblox Inc. http://www.infoblox.com

SEE ALSO

Infoblox::Grid::Admin::TACACSPlusAuthService, Infoblox::Session, Infoblox::Session->add(), Infoblox::Session->get(), Infoblox::Session->modify(), Infoblox::Session->remove()

COPYRIGHT

Copyright (c) 2017 Infoblox Inc.

Infoblox::Grid::CloudAPI::User - Infoblox::Grid::CloudAPI::User object.

- NAME
- DESCRIPTION
- CONSTRUCTOR
- SESSION METHODS
- MODULE METHODS
  - Infoblox::Grid::CloudAPI->allowed_api_admins( )
- METHODS
  - is_remote( )
  - local_admin( )
  - remote_admin( )
- AUTHOR
- SEE ALSO
- COPYRIGHT

NAME

Infoblox::Grid::CloudAPI::User - Infoblox::Grid::CloudAPI::User object.

DESCRIPTION

This object defines an entry in ACL for Cloud API Users.

CONSTRUCTOR

SESSION METHODS

The object does not support any session methods. It is retrieved as a part of the Grid CloudAPI object.

MODULE METHODS

This section describes all the functions that you can apply to an CloudAPI User object.

Infoblox::Grid::CloudAPI->allowed_api_admins( )

Use this method to set or retrieve the list of administrators who can perform cloud API requests on the Grid Master. See Infoblox::Grid::CloudAPI->allowed_api_admins()

Example

```
my $cloud_user = Infoblox::Grid::CloudAPI::User->new(
    is_remote => "false",
    remote_admin => "",
    local_admin => $local_admin_user
);

unless($cloud_user) {
     die("Construct member failed: ",
           Infoblox::status_code() . ":" . Infoblox::status_detail());
}
print "User object created successfully\n";

# Get CloudAPI object
my $cloud_api = $session->get( object => "Infoblox::Grid::CloudAPI" );

# Modify CloudAPI object
my $cloud_api->allowed_api_admins([$cloud_user]);
```

METHODS

This section describes all the methods that you can use to set or retrieve the attribute values of the object.

is_remote( )

Use this method to set or retrieve the user remote_admin status.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

Specify "true" to set user status to remote_admin or "false" to disable it.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
# Get is_remote
$is_remote = $user->is_remote()
# Modify is_remote
$user->is_remote('true')
```

local_admin( )

Use this method to set or retrieve a local administrator allowed to perform Cloud Platform API requests. Required if remote is false, should be none otherwise.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

The valid value is an Infoblox::Grid::Admin::User object.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
# Get local_admin
$local_admin = $user->local_admin()
# Modify local_admin
$user->local_admin($admin_user)
```

remote_admin( )

Use this method to set or retrieve a username matching a remotely authenticated administrator allowed to perform Cloud Platform API requests. Required if remote is true, should be none otherwise.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

The valid value is a string containing a remotely authenticated administrator name.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
# Get remote_admin
$remote_admin = $user->remote_admin()
# Modify remote_admin
$user->remote_admin("remote_admin")
```

AUTHOR

Infoblox Inc. http://www.infoblox.com/

SEE ALSO

Infoblox::Grid::Admin::User

COPYRIGHT

Copyright (c) 2017 Infoblox Inc.

Infoblox::Grid::CloudAPI::Tenant - Infoblox::Grid::CloudAPI::Tenant object.

- NAME
- DESCRIPTION
- CONSTRUCTOR
- SESSION METHODS
  - Infoblox::Session->get( )
  - Infoblox::Session->modify( )
  - Infoblox::Session->search( )
- METHODS
  - comment( )
  - first_seen( )
  - id( )
  - last_seen( )
  - name( )
  - network_count( )
  - vm_count( )
  - unmanaged( )
  - first_discovered_timestamp( )
  - last_discovered_timestamp( )
- AUTHOR
- SEE ALSO
- COPYRIGHT

NAME

Infoblox::Grid::CloudAPI::Tenant - Infoblox::Grid::CloudAPI::Tenant object.

DESCRIPTION

This object represents an administrative concept in Cloud Management Platforms that associates all the network elements (networks, zones, VMs, IPs, ...) that are related and constitutive of the tenant.

CONSTRUCTOR

SESSION METHODS

This section describes all the methods in an Infoblox::Session module that you can apply to a Grid member object.

Infoblox::Session->get( )

Use this method to retrieve all the matching objects from the Infoblox appliance. See Infoblox::Session->get() for parameters and return values.

Key Reference

```
Apply the following attribute to get a specific object:

  comment       - Optional. Descriptive comments.
  id            - Optional. Unique ID associated with the tenant. Set at creation time only.
  name          - Required. Name of the tenant.
  unmanaged     - Optional. Flag to indicate if the tenant is unmanaged. Valid value is 'true' or
'false'.
```

Example

```
$session->get(object => "Infoblox::Grid::CloudAPI::Tenant", comment => "comment");
```

Infoblox::Session->modify( )

Use this method to modify an object in the Infoblox appliance. See Infoblox::Session->modify() for parameters and return values.

Example

```
$session->modify($tenant_object);
```

Infoblox::Session->search( )

Use this method to search for objects in the Infoblox appliance. See Infoblox::Session->search() for parameters and return values.

Key Reference

```
Apply the following attribute to get a specific object:

   comment       - Optional. Descriptive comments (regular expression).
   id            - Optional. Unique ID associated with the tenant. Set at creation time only (regular
expression).
   name          - Required. Name of the tenant (regular expression).
   unmanaged     - Optional. Flag to indicate if the tenant is unmanaged. Valid value is 'true' or
'false'.
```

Example

```
$session->get(object => "Infoblox::Grid::CloudAPI::Tenant", comment => "comment");
```

METHODS

This section describes all the methods that you can use to set or retrieve the attribute values of the object.

comment( )

Use this method to set or retrieve a descriptive comment.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

The valid value is a comment in string format (UTF8) with a maximum of 256 bytes.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
# Get comment
$comment = $tenant->comment();
# Modify comment
$tenant->comment("New comment");
```

first_seen( )

Use this method to retrieve the date and time the first time the tenant was seen on the system. This is a read-only attribute.

Parameter

None

Returns

The valid return value is number of seconds that have elapsed since January 1st, 1970 UTC.

Example

```
# Get first_seen
$first_seen = $tenant->first_seen();
```

id( )

Use this method to retrieve a unique ID associated with the tenant. Set at creation time only. This is a read-only attribute.

Parameter

None

Returns

The method returns attribute value.

Example

```
# Get id
$id = $tenant->id();
```

last_seen( )

Use this method to retrieve the date and time the last time the tenant was seen on the system. This is a read-only attribute.

Parameter

None

Returns

The valid return value is number of seconds that have elapsed since January 1st, 1970 UTC.

Example

```
# Get last_seen
$last_seen = $tenant->last_seen();
```

name( )

Use this method to set or retrieve the name of the tenant.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

The valid value is a comment in string format (UTF8) with a maximum of 64 bytes.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
# Get name
$name = $tenant->name();
# Modify name
$tenant->name("New name");
```

network_count( )

Use this method to retrieve the number of networks associated with the tenant. This is a read-only attribute.

Parameter

None

Returns

The method returns attribute value.

Example

```
# Get network_count
$network_count = $tenant->network_count();
```

vm_count( )

Use this method to retrieve the number of VMs associated with the tenant. This is a read-only attribute.

Parameter

None

Returns

The method returns attribute value.

Example

```
# Get vm_count
$vm_count = $tenant->vm_count();
```

unmanaged( )

Use this method to retrieve the boolean flag used to indicate if the tenant is unmanaged or not. This is a read-only attribute

Parameter

None

Returns

The method returns attribute value. The valid return value is "true" or "false".

Example

```
# Get unmanaged
$unmanaged = $tenant->unmanaged();
```

first_discovered_timestamp( )

Use this method to retrieve the date and time the tenant was first seen on the system. This is a read-only attribute.

Parameter

None

Returns

The valid return value is the number of seconds that have elapsed since January 1st, 1970 UTC.

Example

```
# Get first_discovered_timestamp
$first_discovered_timestamp = $tenant->first_discovered_timestamp();
```

last_discovered_timestamp( )

Use this method to retrieve the date and time the tenant was last seen on the system. This is a read-only attribute.

Parameter

None

Returns

The valid return value is the number of seconds that have elapsed since January 1st, 1970 UTC.

Example

```
# Get last_discovered_timestamp
$last_discovered_timestamp = $tenant->last_discovered_timestamp();
```

AUTHOR

Infoblox Inc. http://www.infoblox.com/

SEE ALSO

Infoblox::Session->get(), Infoblox::Session->search(), Infoblox::Session->modify()

COPYRIGHT

Copyright (c) 2017 Infoblox Inc.

Infoblox::Grid::CloudAPI - Infoblox::Grid::CloudAPI object.

- NAME
- DESCRIPTION
- SESSION METHODS
  - Infoblox::Session->get( )
  - Infoblox::Session->modify( )
- METHODS
  - allow_api_admins( )
  - allowed_api_admins( )
  - enable_recycle_bin( )
- AUTHOR
- SEE ALSO
- COPYRIGHT

NAME

Infoblox::Grid::CloudAPI - Infoblox::Grid::CloudAPI object.

DESCRIPTION

This object represents the cloud Grid.

SESSION METHODS

This section describes all the methods in an Infoblox::Session module that you can apply to a Grid CloudAPI object.

Infoblox::Session->get( )

Use this method to retrieve all the matching objects from the Infoblox appliance. See Infoblox::Session->get() for parameters and return values.

Key Reference

```
Apply the following attribute to get a specific object:

  allow_api_admins   - Optional. The definition of administrators who can perform cloud API requests
on the Grid Master. The valid value is NONE, ALL or LIST.
  allowed_api_admins   - Optional. The list of administrators who can perform cloud API requests on
the Cloud Platform Appliance.
```

Example

```
my @retrieved_objs = $session->get(
    object => "Infoblox::Grid::CloudAPI"
);
```

Infoblox::Session->modify( )

Use this method to modify an object in the Infoblox appliance. See Infoblox::Session->modify() for parameters and return values.

Example

```
$cloud_api->allow_api_admins("ALL");
$session->modify($cloud_api);
```

METHODS

This section describes all the methods that you can use to set or retrieve the attribute values of the object.

allow_api_admins( )

Use this method to set or retrieve the definition of administrators who can perform cloud API requests on the Grid Master. The valid value is NONE (no administrator), ALL (all administrators) or LIST (administrators on the ACL).

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

The valid value is one of the following: NONE, ALL or LIST. The default is ALL.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
# Get allow_api_admins
$allow_api_admins = $cloud_api->allow_api_admins();
# Modify allow_api_admin
$cloud_api->allow_api_admins("ALL");
```

allowed_api_admins( )

Use this method to set or retrieve the list of administrators who can perform cloud API requests on the Grid Master.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

The valid value is an array containing Infoblox::Grid::CloudAPI::User objects.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
# Get allowed_api_admins
$allowed_api_admins = $cloud_api->allowed_api_admins();
# Modify allowed_api_admins
$cloud_api->allowed_api_admins([$cloud_user1, $cloud_user2]);
```

enable_recycle_bin( )

Use this method to enable or disable the recycle bin for deleted cloud objects on the Grid Master regardless of whether certificate is enabled or not.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

Specify "true" to enable recycle or "false" to disable it.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
# Get enable_recycle_bin
$enabled_recycle_bins = $cloud_api->enable_recycle_bin();
# Modify enable_recycle_bin
$cloud_api->enable_recycle_bin("true");
```

AUTHOR

Infoblox Inc. http://www.infoblox.com/

SEE ALSO

Infoblox::Session, Infoblox::Session->get(), Infoblox::Session->modify(), Infoblox::Grid::CloudAPI::User

---

COPYRIGHT

Copyright (c) 2017 Infoblox Inc.

Infoblox::Grid::DataCollection::Cluster - A data collection cluster object.

- NAME
- DESCRIPTION
- SESSION METHODS
  - Infoblox::Session->get( )
  - Infoblox::Session->modify( )
  - Infoblox::Session->search( )
- METHODS
  - blacklisted_clusters( )
  - comment( )
  - datacollector_vms( )
  - disable( )
  - enable_registration( )
  - last_activation_time( )
  - name( )
  - registration_time( )
  - unique_id( )
- AUTHOR
- SEE ALSO
- COPYRIGHT

NAME

Infoblox::Grid::DataCollection::Cluster - A data collection cluster object.

DESCRIPTION

The data collection cluster object provides information and configuration for data collection feature.

SESSION METHODS

This section describes all the methods in the Infoblox::Session module that you can apply to a data collection cluster object.

Infoblox::Session->get( )

Use this method to retrieve all the matching objects from the Infoblox appliance. See Infoblox::Session->get() for parameters and return values.

Key Reference

Apply the following attributes to get a specific data collection cluster object:

```
comment   - Optional. The data collection cluster comment.
name      - Optional. The data collection cluster name.
unique_id - Optional. The data collection cluster unique hexadecimal identifier.
```

Example

```
#Get cluster with name 'cluster1'
my $object = $session->get(
    object => 'Infoblox::Grid::DataCollection::Cluster',
    name   => 'cluster1',
);
```

Infoblox::Session->modify( )

Use this method to modify an object in the Infoblox appliance. See Infoblox::Session->modify() for parameters and return values.

Example

```
#Modify disable value
$object->disable('true');

#Submit modification
my $response = $session->modify($object);
```

Infoblox::Session->search( )

Use this method to retrieve all the matching objects from the Infoblox appliance. See Infoblox::Session->search() for parameters and return values.

Key Reference

Apply the following attributes to search for a data collection cluster object:

```
comment   - Optional. The data collection cluster comment (regexp);
name      - Optional. The data collection cluster name (regexp);
unique_id - Optional. The data collection cluster unique hexadecimal identifier (regexp).
```

Example

```
#Search for cluster that matches /cluster.*/ pattern
my @objects = $session->search(
    object => 'Infoblox::Grid::DataCollection::Cluster',
    name   => 'cluster.*',
);
```

METHODS

This section describes all the methods that can be used to configure and retrieve the attribute values of a data collection cluster object.

blacklisted_clusters( )

Use this method to set or retrieve the list of clusters that are prevented from activation.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

The valid value is an array of Infoblox::Grid::DataCollection::BlacklistedCluster objects.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get blacklisted_clusters value
my $blacklisted_clusters = $object->blacklisted_clusters();

#Modify blacklisted_clusters value
$object->blacklisted_clusters([$cluster1, $cluster2]);
```

comment( )

Use this method to set or retrieve the data collection cluster comment.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

The valid value is a desired comment in a string format.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get comment value
my $comment = $object->comment();

#Modify comment value
$object->comment('cluster comment');
```

datacollector_vms( )

Use this method to retrieve the list of data collector virtual machines. This is a read-only attribute.

Omit the parameter to retrieve the attribute value.

Parameter

None

Returns

The valid return value is an array of Infoblox::Grid::DataCollection::IPInfo objects.

Example

```
#Get datacollector_vms value
my $datacollector_vms = $object->datacollector_vms();
```

disable( )

Use this method to set or retrieve the flag that indicates whether the data collection cluster is enabled or disabled.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

Specify 'true' to enable the data collection cluster or 'false' to disable/deactivate it. The default value is 'false'.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get disable value
my $disable = $object->disable();

#Modify disable value
$object->disable('true');
```

enable_registration( )

Use this method to set or retrieve the flag that indicates whether the data collection cluster registration is enabled or disabled.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

Specify 'true' to enable the data collection cluster registration or 'false' to prevent it from registration. The default value is 'false'.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get enable_registration value
my $enable_registration = $object->enable_registration();

#Modify enable_registration value
$object->enable_registration('true');
```

last_activation_time( )

Use this method to retrieve the last activation time for the data collection cluster.

Omit the parameter to retrieve the attribute value.

Parameter

None

Returns

The method returns the attribute value, the number of seconds that have elapsed since January 1st, 1970 UTC.

Example

```
#Get last_activation_time value
my $last_activation_time = $object->last_activation_time();
```

name( )

Use this method to set or retrieve the name of the data collection cluster. This is a read-only attribute.

Omit the parameter to retrieve the attribute value.

Parameter

None

Returns

The method returns the attribute value.

Example

```
#Get name value
my $name = $object->name();
```

registration_time( )

Use this method to retrieve the time the data collection cluster was registered. This is a read-only attribute.

Omit the parameter to retrieve the attribute value.

Parameter

None

Returns

The method returns the attribute value, the number of seconds that have elapsed since January 1st, 1970 UTC.

Example

```
#Get registration_time value
my $registration_time = $object->registration_time();
```

unique_id( )

Use this method to set or retrieve the unique identifier for the data collection cluster.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

The valid value is a desired hexadecimal identifier in a string format.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get unique_id value
my $unique_id = $object->unique_id();

#Modify unique_id value
$object->unique_id('ABCD');
```

AUTHOR

Infoblox Inc. http://www.infoblox.com/

SEE ALSO

Infoblox::Grid::DataCollection::BlacklistedCluster, Infoblox::Grid::DataCollection::IPInfo,

---

COPYRIGHT

Copyright (c) 2017 Infoblox Inc.

Infoblox::Grid::DataCollection::BlacklistedCluster - A Data Collection blacklisted cluster object.

- NAME
- DESCRIPTION
- CONSTRUCTOR
- SESSION METHODS
- MODULE METHODS
  - Infoblox::Grid::DataCollection::Cluster->blacklisted_clusters( )
- METHODS
  - comment( )
  - name( )
  - unique_id( )
- AUTHOR
- SEE ALSO
- COPYRIGHT

NAME

Infoblox::Grid::DataCollection::BlacklistedCluster - A Data Collection blacklisted cluster object.

DESCRIPTION

The Data Collection blacklisted cluster object is used to configure Data Collection cluster blacklist.

CONSTRUCTOR

```
my $blacklisted_cluster = Infoblox::Grid::DataCollection::BlacklistedCluster->new(
    unique_id => $string, #Required
    comment   => $string, #Optional / Default is undefined
    name      => $string, #Optional / Default is undefined
);
```

SESSION METHODS

The object does not support any session methods.

MODULE METHODS

The following functions are available to be applied to a Data Collection blacklisted cluster object.

Infoblox::Grid::DataCollection::Cluster->blacklisted_clusters( )

Use this method to set or retrieve the list of clusters which are prevented from activation. See Infoblox::Grid::DataCollection::Cluster->blacklisted_clusters() for parameters and return values.

Example

```
#Get blacklisted_clusters value
my $blacklisted_clusters = $object->blacklisted_clusters();

#Modify blacklisted_clusters value
$object->blacklisted_clusters([$cluster1, $cluster2]);
```

METHODS

This section describes all the methods that can be used to configure and retrieve the attribute values of a Data Collection blacklisted cluster object.

comment( )

Use this method to set or retrieve the blacklisted cluster comment.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

The valid value is a desired cluster comment in string format.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get comment value
my $comment = $object->comment();

#Modify comment value
$object->comment('cluster1 comment');
```

name( )

Use this method to retrieve the blacklisted cluster name. This is a read-only attribute.

Omit the parameter to retrieve the attribute value.

Parameter

None

Returns

The method returns the attribute value.

Example

```
#Get name value
my $name = $object->name();
```

unique_id( )

Use this method to set or retrieve the blacklisted cluster unique hexadecimal identifier.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

The valid value is a desired cluster unique hexadecimal identifier in string format.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get unique_id value
my $unique_id = $object->unique_id();

#Modify unique_id value
$object->unique_id('ABCD');
```

AUTHOR

Infoblox Inc. http://www.infoblox.com/

SEE ALSO

Infoblox::Grid::DataCollection::Cluster, Infoblox::Grid::DataCollection::Cluster->blacklisted_clusters()

COPYRIGHT

Copyright (c) 2017 Infoblox Inc.

Infoblox::Grid::DataCollection::IPInfo - A data collection IP information object.

- NAME
- DESCRIPTION
- SESSION METHODS
- MODULE METHODS
  - Infoblox::Grid::DataCollection::Cluster->datacollector_vms( )
- METHODS
  - ip_address( )
  - nat_enabled( )
  - nat_group( )
  - nat_ip( )
- AUTHOR
- SEE ALSO
- COPYRIGHT

NAME

Infoblox::Grid::DataCollection::IPInfo - A data collection IP information object.

DESCRIPTION

The Data Collection IP information object is used to configure the list of data collector virtual machines in the data collection cluster.

SESSION METHODS

The object does not support any session methods.

MODULE METHODS

The following functions are available to be applied to a data collection IP information object.

Infoblox::Grid::DataCollection::Cluster->datacollector_vms( )

Use this method to retrieve the list of data collector virtual machines. This is a read-only attribute. See Infoblox::Grid::DataCollection::Cluster->datacollector_vms() for parameters and return values.

Example

```
#Get datacollector_vms value
my $datacollector_vms = $object->datacollector_vms();
```

METHODS

This section describes all the methods that can be used to retrieve the attribute values of a data collection IP information.

Note that this is a read-only object.

ip_address( )

Use this method to retrieve the virtual machine IP address. This is a read-only attribute.

Omit the parameter to retrieve the attribute value.

Parameter

None

Returns

The method returns the attribute value.

Example

```
#Get ip_address value
my $ip_address = $object->ip_address();
```

nat_enabled( )

Use this method or retrieve the flag that indicates whether the NAT translation of the virtual machine IP address is enabled or disabled. This is a read-only attribute.

Omit the parameter to retrieve the attribute value.

Parameter

None

Returns

The method returns the attribute value.

Example

```
#Get enable_nat value
my $enable_nat = $object->enable_nat();
```

nat_group( )

Use this method to retrieve the virtual machine NAT group. This is a read-only attribute.

Omit the parameter to retrieve the attribute value.

Parameter

None

Returns

The method returns the attribute value.

Example

```
#Get nat_group value
my $nat_group = $object->nat_group();
```

nat_ip( )

Use this method to retrieve the virtual machine external IP address. This is a read-only attribute.

Omit the parameter to retrieve the attribute value.

Parameter

None

Returns

The method returns the attribute value.

Example

```
#Get nat_ip value
my $nat_ip = $object->nat_ip();
```

AUTHOR

Infoblox Inc. http://www.infoblox.com/

SEE ALSO

Infoblox::Grid::DataCollection::Cluster, Infoblox::Grid::DataCollection::Cluster->datacollector_vms()

COPYRIGHT

Copyright (c) 2017 Infoblox Inc.

Infoblox::Grid::KerberosKey - Grid kerberos key object.

- NAME
- DESCRIPTION
- CONSTRUCTOR
- MODULE METHODS
  - Infoblox::Grid::DNS->gss_tsig_keys( )
  - Infoblox::Grid::DHCP->gss_tsig_keys( )
  - Infoblox::Grid::Member::DNS->gss_tsig_keys( )
  - Infoblox::Grid::Member::DHCP->gss_tsig_keys( )
- SESSION METHODS
  - Infoblox::Session->get( )
  - Infoblox::Session->remove( )
  - Infoblox::Session->search( )
- METHODS
  - enctype( )
  - domain( )
  - in_use( )
  - principal( )
  - version( )
  - upload_timestamp( )
- SAMPLE CODE
- AUTHOR
- SEE ALSO
- COPYRIGHT

NAME

Infoblox::Grid::KerberosKey - Grid kerberos key object.

DESCRIPTION

This object is used to manage kerberos keys for GSS-TSIG.

CONSTRUCTOR

```
#Constructor for an Grid KerberosKey object
 my $key = Infoblox::Grid::KerberosKey->new(
    principal => $string,  #Required.
    domain    => $string,  #Optional / Default is undefined
    version   => $num,     #Optional / Default is undefined
  );
```

MODULE METHODS

The following functions are available to be applied to an KerberosKey object.

Infoblox::Grid::DNS->gss_tsig_keys( )

Use this function to specify a Kerberos key at the grid level on the Infoblox appliance. See Infoblox::Grid::DNS->gss_tsig_keys() for parameters and return values.

Example

```
#Create a Kerberos object.
my $key = Infoblox::Grid::KerberosKey->new(
                  principal => 'host/appliance.ivan.test@LADYBUG.LOCAL',
                  domain => 'JACKIE.LOCAL',
                  version => '3',
);

#Configure kerberos key on the Infoblox grid object
my $response = $grid->gss_tsig_keys([$key]);
```

Infoblox::Grid::DHCP->gss_tsig_keys( )

Use this function to specify a Kerberos key at the grid level on the Infoblox appliance. See Infoblox::Grid::DHCP->gss_tsig_keys() for parameters and return values.

Example

```
#Create a Kerberos object.
my $key = Infoblox::Grid::KerberosKey->new(
                principal => 'host/appliance.ivan.test@LADYBUG.LOCAL',
                domain => 'JACKIE.LOCAL',
                version => '3',
              );

#Configure kerberos key on the Infoblox grid object
my $response = $grid->gss_tsig_keys([$key]);
```

Infoblox::Grid::Member::DNS->gss_tsig_keys( )

Use this function to specify a Kerberos key at the member level on the Infoblox appliance. See Infoblox::Grid::Member::DNS->gss_tsig_keys() for parameters and return values.

Example

```
#Create a KerberosKey object.
my $key = Infoblox::Grid::KerberosKey->new(
                principal => 'host/appliance.ivan.test@LADYBUG.LOCAL',
                domain => 'JACKIE.LOCAL',
                version => '3',
);

#Configure KerberosKey on the Infoblox member object
my $response = $member->gss_tsig_keys([$key]);
```

Infoblox::Grid::Member::DHCP->gss_tsig_keys( )

Use this function to specify a Kerberos key at the member level on the Infoblox appliance. See Infoblox::Grid::Member::DHCP->gss_tsig_keys() for parameters and return values.

Example

```
 #Create a KerberosKey object.
  my $key = Infoblox::Grid::KerberosKey->new(
                principal => 'host/appliance.ivan.test@LADYBUG.LOCAL',
                domain => 'JACKIE.LOCAL',
                version => '3',
              );

#Configure KerberosKey on the Infoblox member object
my $response = $member->gss_tsig_keys([$key]);
```

SESSION METHODS

This section describes all the methods in an Infoblox::Session module that you can apply to a Kerberos key object.

Infoblox::Session->get( )

Use this method to retrieve all the matching objects from the Infoblox appliance. See Infoblox::Session->get() for parameters and return values.

Key References

```
Apply the following attributes to get a specific Kerberos key object:

 principal - Required. The principal of the Kerberos key object you want to retrieve.
```

Example

```
my @retrieved_objs = $session->get(
    object    => "Infoblox::Grid::KerberosKey",
    principal => "my-principal");
```

Infoblox::Session->remove( )

Use this method to remove a Kerberos key object from the Infoblox appliance. See Infoblox::Session->remove() for parameters and return values.

To remove a specific object, first use `get()` or `search()` to retrieve the specific Kerberos key object, and then submit this object for removal.

Example

```
# Get the objects with the same principal
my @retrieved_objs = $session->get(
    object    => "Infoblox::Grid::KerberosKey",
    principal => "my-principal");
# Find the desired object from the retrieved list.
my $desired_kerberos_key = $retrieved_objs[0];
# Submit for removal
my $response = $session->remove( $desired_kerberos_key );
```

Infoblox::Session->search( )

Use this method to search for Kerberos key objects in the Infoblox appliance. See Infoblox::Session->search() for parameters and return values.

Key References

```
Apply the following attributes to search for a Kerberos key object:

 principal - Required. The principal of the Kerberos key object you want to retrieve (regular
expression is supported).
```

Example

```
# search for all grid member objects
my @retrieved_objs = $session->search(
    object => "Infoblox::Grid::KerberosKey",
    principal => ".*");
```

METHODS

This section describes all the methods that you can used to configure and retrieve the attribute value of a KerberosKey object.

enctype( )

Use this method to retrieve the encryption type for the KerberosKey object. This is a read-only attribute.

Parameter

None

Returns

The method returns the attribute value.

Example

```
#Get enctype value
my $enctype = $key->enctype();
```

domain( )

Use this method to set or retrieve the domain of the KerberosKey object.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

Desired domain in string format with a maximum of 1023 bytes.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get domain
my $domain = $key->domain();
#Modify domain
$key->domain("my-domain");
```

in_use( )

Use this method to retrieve the flag used to indicate whether the key has been assigned to a member. This is a read-only attribute.

Parameter

None

Returns

The return value is "true" if KerberosKey object has been assigned to a member and "false" if it hasn't.

Example

```
#Get in_use value
my $in_use = $key->in_use();
```

principal( )

Use this method to set or retrieve the principal string of the KerberosKey object.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

Desired principal in string format with a maximum of 1023 bytes.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get principal
my $string = $key->principal();
#Modify principal
$key->principal("my-principal");
```

version( )

Use this method to set or retrieve the version number of the KerberosKey object.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

Enter an unsigned integer between 0 and 4294967295.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get version
my $version = $key->version();
#Modify version
$key->version("5");
```

upload_timestamp( )

Use this method to retrieve the timestamp when the KerberosKey object was uploaded. This is a read-only attribute.

Parameter

None

Returns

The return value is the date and time in the ISO 8601 extended format (e.g. 2008-10-21T10:56:33Z or 2008-10-21T10:56:33.001Z).

Example

```
#Get upload_timestamp value
my $upload_timestamp = $key->upload_timestamp();
```

SAMPLE CODE

The following sample code demonstrates different operations that can be applied to an object such as create, get, and remove an object. Also this sample code also includes error handling for the operations.

```
#PROGRAM STARTS: Include all the modules that will be used
 use strict;
 use Infoblox;

 #Create a session to the Infoblox appliance
 my $session = Infoblox::Session->new(
    master   => "192.168.1.2",
    username => "admin",
    password => "infoblox"
 );
 unless ($session) {
   die("Construct session failed: ",
       Infoblox::status_code() . ":" . Infoblox::status_detail());
 }
 print "Session created successfully\n";
```

#import kerberos key object

```
my $result = $session->import_data(
    type       => "keytab",
    path       => "/infoblox/tests/datasets/dnsone.keytab",
    service => 'dhcp',
   );

unless($result) {
       die("import kerberos key object failed: ",
            Infoblox::status_code() . ":" . Infoblox::status_detail());
}
print "import kerberos key object successfully\n";
```

#retrieve kerberos key object

```
my @keys = $session->get(
   object    => "Infoblox::Grid::KerberosKey",
   principal => 'host/appliance.ivan.test@LADYBUG.LOCAL',
 );

 unless ($key) {
       die("Get KerberosKey object failed: ",
            $session->status_code() . ":" . $session->status_detail());
  }
  print "Get KerberosKey object found at least 1 matching entry\n";

my $grid_dhcp = $session->get(
   object  => 'Infoblox::Grid::DHCP',
   cluster => 'Infoblox',
   );
unless ($grid_dhcp) {
       die("Get Grid DHCP object failed: ",
            $session->status_code() . ":" . $session->status_detail());
}

$grid_dhcp->gss_tsig_keys([$key]);
$grid_dhcp->kdc_server('test.com');

#Applying the changes to appliance through session.
$session->modify($grid_dhcp)
     or die("modify Grid DHCP properties failed:" ,
      $session->status_code(), $session->status_detail());
print "Grid DHCP properties modified successfully for assigning KerberosKey object\n";
```

#Removing an KerberosKey object

```
# remove the reference to kerberos key
$grid_dhcp->gss_tsig_keys(undef);
$grid_dhcp->kdc_server(undef);

#Applying the changes to appliance through session.
$session->modify($grid_dhcp)
     or die("modify Grid DHCP properties failed:" ,
      $session->status_code(), $session->status_detail());
print "Grid DHCP properties modified successfully for unassigning KerberosKey object\n";
```

```
$session->remove(object => 'Infoblox::Grid::KerberosKey',
                 principal => 'host/appliance.ivan.test@LADYBUG.LOCAL')
     or die("remove KerberosKey object failed:" ,
      $session->status_code(), $session->status_detail());
print "remove KerberosKey object successfully\n";

####PROGRAM ENDS####
```

AUTHOR

Infoblox Inc. http://www.infoblox.com/

SEE ALSO

Infoblox::Session, Infoblox::Grid::DNS, Infoblox::Grid::DHCP, Infoblox::Grid::Member::DNS, Infoblox::Grid::Member::DHCP, Infoblox::Grid::DNS->gss_tsig_keys(), Infoblox::Grid::DHCP->gss_tsig_keys(), Infoblox::Grid::Member::DNS->gss_tsig_keys(), Infoblox::Grid::Member::DHCP->gss_tsig_keys(), Infoblox::Session->get(), Infoblox::Session->modify(),Infoblox::Session->remove()

COPYRIGHT

Copyright (c) 2017 Infoblox Inc.

Infoblox::IFMap::CACertificate - IF-MAP CA Certificate object.

- NAME
- DESCRIPTION
- CONSTRUCTOR
- SESSION METHODS
  - Infoblox::Session->add( )
  - Infoblox::Session->get( )
  - Infoblox::Session->remove( )
  - Infoblox::Session->search( )
- METHODS
  - certificate( )
  - subject( )
  - issuer( )
  - valid_not_before( )
  - valid_not_after( )
- SAMPLE CODE
- AUTHOR
- SEE ALSO
- COPYRIGHT

NAME

Infoblox::IFMap::CACertificate - IF-MAP CA Certificate object.

DESCRIPTION

You can configure the Infoblox appliance as a metadata access point (MAP) and implement the IF-MAP protocol defined by the Trusted Computing Group (TCG, www.trustedcomputinggroup.com). This module provides an interface for configuring CA certificates that the IF-MAP server uses to authenticate IF-MAP clients.

CONSTRUCTOR

```
my $ca_cert=Infoblox::IFMap::CACertificate->new(
    certificate      => $string, #Required
);
```

SESSION METHODS

This section describes all the methods in an Infoblox::Session module that you can apply to an IF-MAP CA Certificate object.

Infoblox::Session->add( )

Use this method to add a new CA certificate for the IF-MAP server. See Infoblox::Session->add( ) for parameters and return values.

Example

```
# Read the certificate contents from the file
open CERTFILE "<cert.pem" or die "Unable to open certificate file";
my $pem_cert = join('', <CERTFILE>);

# Create a new CA certificate object
my $cacert = Infoblox::IFMap::CACertificate->new(
certificate => $pem_cert
);

# Add the CA certificate
my $response = $session->add( $cacert );
```

Infoblox::Session->get( )

Use this method to retrieve the CA certificates configured for the IF-MAP server. See Infoblox::Session->get( ) for parameters and return values.

Key References

Apply the following attributes to get a specific IF-MAP CA certificate object:

```
subject                - Optional. String that contains a certificate subject.
issuer                 - Optional. String that contains a certificate issuer.
```

```
 valid_not_after        - Optional. String that contains a desired value of the certificate
expiration date in ISO 8601 format.
 valid_not_before       - Optional. String that contains a desired value of the certificate start
date in ISO 8601 format.
```

Example

```
# Retrieve CA certificate with the subject = "/CN=verisign.com/OU=Verisign"
my @retrieved_objs = $session->get(
object => "Infoblox::IFMap::CACertificate",
subject => "/CN=verisign.com/OU=Verisign"
);
```

Infoblox::Session->remove( )

Use this method to remove a CA certificate.

Examples

```
# Get the CA certificate
my @retrieved_objs = $session->get(
object => "Infoblox::IFMap::CACertificate",
subject   => "/CN=verisign.com/OU=Verisign"
);

# Find the desired object from the retrieved list.
my $desired_cert = $retrieved_objs[0];

# Submit for removal
my $response = $session->remove( $desired_cert );
```

Infoblox::Session->search( )

Use this method to search for the CA certificates configured for the IF-MAP server. You can specify the following attributes in a search: subject, issuer, valid_not_before, and valid_not_after.

Key References

Apply the following attributes to search for a specific IF-MAP CA certificate object:

```
 subject                - Optional. Regex for the certificate subject.
 issuer                 - Optional. Regex for the certificate issuer.
 valid_not_after        - Optional. String that contains the desired certificate expiration date in
ISO 8601 format. This method supports searching with limit parameters  (see below).
 valid_not_before       - Optional. String that contains the desired certificate start date in ISO
8601 format. This method supports searching with limit parameters  (see below).
```

See Infoblox::Session/search with limit parameters for more information on this functionality.

Examples

```
# Search for IF-MAP CA certificates issued by Verisign
# and with valid not before time in 2010
my @retrieved_objs = $session->search(
object => "Infoblox::IFMap::CACertificate",
issuer => ".*verisign.*",
valid_not_before => ">=< 2010-01-01T00:00:00Z,2011-01-01T00:00:00Z",
);
```

METHODS

certificate( )

Use this method to retrieve or set the value of the certificate attribute. If no argument is passed, the method returns the current attribute value. The value must be a single certificate in PEM format.

Parameter

The valid value is a string that contains the certificate in PEM format. This is a required attribute.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Examples

```
# Get certificate
```

```
my $pem_cert = $cert_object->certificate( );

# Set certificate
open CERTFILE "<cert.pem" or die "Unable to open certificate file";
my $pem_cert = join('', <CERTFILE>);
$cert_object->certificate( $pem_cert );
```

subject( )

Use this method to retrieve the value of the subject attribute. This method does not support any argument because the attribute is read-only and is set by the server. It has a value only when it reads an existing certificate from the Infoblox::IFMap::CACertificate object retrieved from the appliance.

Parameter

N/A.

Returns

The method returns the attribute value.

This is a read-only attribute. If you specify a parameter, the method returns an error.

Examples

```
# Get subject
my $subject = $certificate->subject( );
```

issuer( )

Use this method to retrieve the value of the =issuer= attribute. This method does not support any argument because the attribute is read-only and is set by the server. It has a value only when it reads an existing certificate from the =Infoblox::IFMap::CACertificate= object.

Parameter

N/A.

Returns

The method returns the attribute value.

This is a read-only attribute. If you specify a parameter, the method returns an error.

Examples

```
# Get subject
my $issuer = $certificate->issuer( );
```

valid_not_before( )

Use this method to retrieve the value of the valid_not_before attribute. This method does not support any argument because the attribute is read-only and is set by the server. It has a value only when it reads an existing certificate from the Infoblox::IFMap::CACertificate object. The returned value is the date before the certificate becomes invalid in ISO 8601 format.

Parameter

N/A.

Returns

The method returns the attribute value.

This is a read-only attribute. If you specify a parameter, the method returns an error.

Examples

```
# Get valid_not_before attribute
my $valid_not_before = $certificate->valid_not_before( );
```

valid_not_after( )

Use this method to retrieve the value of the valid_not_after attribute. This method does not support any argument because it is read-only and is set by the server. It has a value only when it reads an existing certificate from the Infoblox::IFMap::CACertificate object. The returned value is the date after the certificate becomes invalid in the ISO 8601 format.

Parameter

N/A.

Returns

The method returns the attribute value.

This is a read-only attribute. If you specify a parameter, the method returns an error.

Examples

```
# Get operation
my $valid_not_after = $certificate->valid_not_after( );
```

SAMPLE CODE

The following sample code demonstrates the different functions that can be applied to an object, such as add, search and remove. This sample also includes error handling for the operations.

#Preparations

```
#PROGRAM STARTS: Include all the modules that will be used
use strict;
use Infoblox;
#Create a session to the Infoblox appliance
my $session = Infoblox::Session->new(
               master   => "192.168.1.2", #appliance host ip
               username => "admin",       #appliance user login
               password => "infoblox"     #appliance password
);
unless ($session) {
       die("Construct session failed: ",
            Infoblox::status_code( ) . ":" . Infoblox::status_detail( ));
}
print "Session created successfully\n";

#Get a certificate from the external file
local *CERTFILE;
open CERTFILE, "<API_unit_test_sample_code/ca_cert.pem" or die "Unable to open certificate file";
my $pem_cert = join('', <CERTFILE>);
close(CERTFILE);
```

#Create CA certificate object

```
my $ca_certificate= Infoblox::IFMap::CACertificate->new(
                                                          certificate => $pem_cert
                                                        );
unless($ca_certificate) {
       die("Construct CA certificate object failed: ",
               Infoblox::status_code( ) . ":" . Infoblox::status_detail( ));
}
print "IFMap CA Certificate object created successfully\n";
```

#Add the CA Certificate object to the Infoblox appliance through a session

```
$session->add($ca_certificate)
    or die("Add IFMap CA certificate object failed: ",
                        $session->status_code( ) . ":" . $session->status_detail( ));
print "IFMap CA Certificate object added to server successfully\n";

#Get the IFMap CA Certificate object using subject attribute
my @retrieved_objs=$session->get(
                                  object => "Infoblox::IFMap::CACertificate",
                                  subject => 'CN="Test CA 2",OU="Engineering",O="Infoblox",L="Santa
Clara",ST="CA",C="US"'
                                );
my $object = $retrieved_objs[0];
unless($object){
    die("Get IFMap CA Certificate object failed: ",
                $session->status_code( ) . ":" . $session->status_detail( ));
}
print "Get IFMap CA Certificate object found at least 1 matching entry\n";
```

#Search for IFMap CA certificate objects

```
my @retrieved_objs=$session->search(
                                  object => "Infoblox::IFMap::CACertificate",
                                  subject => '.*Engineering.*',
```

```
                                    valid_not_before => '>=< 2010-01-01T08:00:00Z,2010-10-01T00:00:00Z'
                                );
 my $object = $retrieved_objs[0];
 unless($object){
     die("Search for IFMap CA Certificate object failed: ",
                 $session->status_code( ) . ":" . $session->status_detail( ));
 }
 print "Searching IFMap CA Certificate object found at least 1 matching entry\n";
```

#Remove IFMap CA Certificate object

```
 #Get IFMap CA Certificate object
 my @retrieved_objs=$session->get(
                                    object => "Infoblox::IFMap::CACertificate",
                                    subject => 'CN="Test CA 2",OU="Engineering",O="Infoblox",L="Santa
Clara",ST="CA",C="US"'
                                );
 my $object = $retrieved_objs[0];
 unless($object){
     die("Get IFMap CA certificate object failed: ",
                 $session->status_code( ) . ":" . $session->status_detail( ));
 }
 print "Get IFMap CA certificate object found at least 1 matching entry\n";

 #submit the object for removal
 $session->remove($object)
     or die("Remove IFMap CA certificate failed: ",
             $session->status_code( ) . ":" . $session->status_detail( ));
 print "IFMap CA certificate object removed successfully \n";
```

####PROGRAM ENDS####

AUTHOR

Infoblox Inc. http://www.infoblox.com/

SEE ALSO

Infoblox::Session,Infoblox::Session->add( ),Infoblox::Session->get( ),Infoblox::Session->remove( ),Infoblox::Session->search( )

COPYRIGHT

Copyright (c) 2017 Infoblox Inc.

Infoblox::DHCP::Option60MatchRule - DHCP Option60MatchRule object.

- NAME
- DESCRIPTION
- CONSTRUCTOR
- MODULE METHODS
  - Infoblox::Grid::DHCP->option60_match_rules( )
  - Infoblox::Grid::Member::DHCP->option60_match_rules( )
- METHODS
  - match_value( )
  - option_space( )
  - is_substring( )
  - substring_offset( )
  - substring_length( )
- SAMPLE CODE
- AUTHOR
- SEE ALSO
- COPYRIGHT

NAME

Infoblox::DHCP::Option60MatchRule - DHCP Option60MatchRule object.

DESCRIPTION

An Option60MatchRule object allows specifying match rules for Option 60 globally.

CONSTRUCTOR

```
my $option60_match_rule = Infoblox::DHCP::Option60MatchRule->new(
                            match_value      => $string,          #Required
                            option_space     => $string,          #Required
                            is_substring     => "true" | "false", #Optional / Default is undefined
                            substring_offset => $num,             #Optional / Default is undefined
                            substring_length => $num              #Optional / Default is undefined
                          );
```

MODULE METHODS

The following functions are available to be applied to a DHCP Option60MatchRule object.

Infoblox::Grid::DHCP->option60_match_rules( )

Use this function to specify match rule for option60 at grid level. See Infoblox::Grid::DHCP->option60_match_rules() for parameters and return values.

Example

```
#Construct a DHCP Option60MatchRule object
my $option60_match_rule = Infoblox::DHCP::Option60MatchRule->new(
                                        "option_space" => "infospace",
                                        "match_value" => "Nortel-i2004-A",
                                        "is_substring" => "true",
                                        "substring_offset" => "8",
                                        "substring_length" => "14"
                                      );

# Configure DHCP Option60MatchRule on the Grid DHCP object
my $response = $grid_dhcp->option60_match_rules([$option60_match_rule]);
```

Infoblox::Grid::Member::DHCP->option60_match_rules( )

Use this function to specify match rule for option60 at member level. See Infoblox::Grid::Member::DHCP->option60_match_rules() for parameters and return values.

Example

```
#Construct a DHCP Option60MatchRule object
my $option60_match_rule = Infoblox::DHCP::Option60MatchRule->new(
                                        "option_space" => "infospace",
```

```
                                        "match_value" => "Nortel-i2004-A",
                                        "is_substring" => "true",
                                        "substring_offset" => "10",
                                        "substring_length" => "24"
                                      );

# Configure DHCP Option60MatchRule on the Grid Member DHCP object
my $response = $grid_member_dhcp->option60_match_rules([$option60_match_rule]);
```

METHODS

This section describes all the methods that can be used to configure and retrieve the attribute values of a DHCP Option60MatchRule object.

match_value( )

Use this method to set or retrieve the match value of a DHCP Option60MatchRule object.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

Desired match value in string format.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
# Get match_value
my $match_value = $option60_match_rule->match_value();
# Modify match_value
$option60_match_rule->match_value("SUNW");
```

option_space( )

Use this method to set or retrieve the option space of a DHCP Option60MatchRule object.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

Valid value is an string that contains defined Infoblox::DHCP::OptionSpace object. See Infoblox::DHCP::OptionSpace for parameters and return values.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
# Get option_space
my $match_value = $option60_match_rule->option_space();
# Modify option_space
$option60_match_rule->option_space("new_option");
```

is_substring( )

Use this method to set or retrieve the is_substring flag of a DHCP Option60MatchRule object.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

Specify "true" to set the is_substring flag or "false" to deactivate/unset it. The default value for this field is undefined.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
# Get is_substring
my $is_substring = $option60_match_rule->is_substring();
# Modify is_substring
$option60_match_rule->is_substring("true");
```

substring_offset( )

Use this method to set or retrieve the substring_offset of a DHCP Option60MatchRule object.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

The substring_offset for the DHCP Option60MatchRule object (range from 0 to 65535). The default value is undefined.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
# Get substring_offset
my $substring_offset = $option60_match_rule->substring_offset();
# Modify substring_offset
$option60_match_rule->substring_offset("10");
```

substring_length( )

Use this method to set or retrieve the substring_length of a DHCP Option60MatchRule object.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

The substring_length for the DHCP Option60MatchRule object (range from 0 to 65535). The default value is undefined.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
# Get substring_length
my $substring_length = $option60_match_rule->substring_length();
# Modify substring_length
$option60_match_rule->substring_length("23");
```

SAMPLE CODE

The following sample code demonstrates the different functions that can be applied to an object such as add, and remove. Also, this sample includes error handling for the operations.

#Preparation prior to a DHCP Option60MatchRule insertion

```
#PROGRAM STARTS: Include all the modules that will be used
use strict;
use Infoblox;

#Create a session to the Infoblox appliance

my $session = Infoblox::Session->new(
    master   => "192.168.1.2",
    username => "admin",
    password => "infoblox"
);
unless ($session) {
   die("Construct session failed: ",
       Infoblox::status_code() . ":" . Infoblox::status_detail());
}
print "Session created successfully\n";

my $option space1   Infoblox::DHCP::OptionSpace  new
```

```
                    =                                   ->   (
                                                        "name" => "infospace"
                                                        );
my $response = $session->add( $option_space1);
print "Add option space operation success \n" if ($response );
print "Add option space operation failed  \n" if ( not $response );

#Create a DHCP OptionDefinition object.>
my $option60_match_rule = Infoblox::DHCP::Option60MatchRule->new(
                                                        "option_space" => "infospace",
                                                        "match_value" => "Nortel-i2004-A",
                                                        "is_substring" => "true",
                                                        "substring_offset" => "0",
                                                        "substring_length" => "14"
                                                        );
unless ($option60_match_rule) {
     die("Construct DHCP Option60MatchRule failed: ",
       Infoblox::status_code() . ":" . Infoblox::status_detail());
}

#Get on Grid DHCP object to add Option60MatchRule object>
my @result = $session->get(
    object => "Infoblox::Grid::DHCP",
    grid   => "Infoblox"
    )
    or die("get DHCP failed: ",
                $session->status_code(). ":" .$session->status_detail());
print "get grid DHCP Objcet successful.\n";

unless (scalar(@result) == 0) {
my $griddhcp = $result[0];
if ($griddhcp) {

 $griddhcp->option60_match_rules([$option60_match_rule]);

  #Applying the changes to appliance through session.
  $session->modify($griddhcp)
      or die("Modify DHCP failed: ",
         $session->status_code(). ":" .$session->status_detail());
  print"Modified Grid DHCP Object successfully.\n";
    }
    } else {
      print "No Grid DHCP object found.";
   }

#To remove Option60MatchRule object in Grid DHCP object>
my @result = $session->get(
    object => "Infoblox::Grid::DHCP",
    grid   => "Infoblox"
    )
    or die("get DHCP failed: ",
                $session->status_code(). ":" .$session->status_detail());
print "get grid DHCP Objcet successful.\n";

unless (scalar(@result) == 0) {
my $griddhcp = $result[0];
if ($griddhcp) {

$griddhcp->option60_match_rules(undef);

  #Applying the changes to appliance through session.
  $session->modify($griddhcp)
      or die("Modify DHCP failed: ",
         $session->status_code(). ":" .$session->status_detail());
  print"Modified Grid DHCP Object successfully to remove Option60MatchRule object.\n";
  }
    } else {
      print "No Grid DHCP object found.";
   }

#Get OptionSpace object through the session
my @retrieved_objs = $session->get(
    object => "Infoblox::DHCP::OptionSpace",
    name   => "infospace"
);
my $object = $retrieved_objs[0];
unless  $object
```

```
        (       ) {
    die("Get OptionSpace object failed: ",
        $session->status_code() . ":" . $session->status_detail());
}
print "Get OptionSpace object found at least 1 matching entry\n";

#Submit the object for removal
$session->remove($object)
    or die("Remove OptionSpace object failed: ",
        $session->status_code() . ":" . $session->status_detail());
print "OptionSpace object removed successfully \n";

####PROGRAM ENDS####
```

AUTHOR

Infoblox Inc. http://www.infoblox.com/

SEE ALSO

Infoblox::Session,Infoblox::Grid::DHCP, Infoblox::Grid::Member::DHCP,Infoblox::DHCP::OptionSpace, Infoblox::Session->get(), Infoblox::Session->add(), Infoblox::Session->remove(), Infoblox::Session->modify()

COPYRIGHT

Copyright (c) 2017 Infoblox Inc.

Infoblox::Grid::HostNameRewritePolicy - a structure for storing hostname rewrite policies.

- NAME
- DESCRIPTION
- CONSTRUCTOR
- MODULE METHODS
  - Infoblox::Grid::DHCP->protocol_hostname_rewrite_policies( )
- METHODS
  - is_grid_default( )
  - name( )
  - replacement_character( )
  - pre_defined( )
  - valid_characters( )
- AUTHOR
- SEE ALSO
- COPYRIGHT

NAME

Infoblox::Grid::HostNameRewritePolicy - a structure for storing hostname rewrite policies.

DESCRIPTION

An Infoblox::Grid::HostNameRewritePolicy object represents a hostname rewrite policy.

CONSTRUCTOR

```
my $policy = Infoblox::Grid::HostNameRewritePolicy->new(
     name                => $string,           # Required
     valid_characters    => $string,           # Required
     replacement_character => $string,         # Required
     is_grid_default     => "true" | "false",  # Optional / Default value is "false"
);
```

MODULE METHODS

The following functions can be applied to an Infoblox::Grid::HostNameRewritePolicy object.

Infoblox::Grid::DHCP->protocol_hostname_rewrite_policies( )

Use this function to add or retrieve hostname rewrite policies. See Infoblox::Grid::DHCP->protocol_hostname_rewrite_policies() for parameters and return values.

Example

```
#Get Grid DHCP object
my @retrieved_objs = $session->get(
       object => "Infoblox::Grid::DHCP",
       grid   => "Infoblox"
);
my $griddhcp = $retrieved_objs[0];

#Modify protocol_hostname_rewrite_policies
my $policies = $griddhcp->protocol_hostname_rewrite_policies();
push(@$policies, $policy);
$griddhcp->protocol_hostname_rewrite_policies($policies);
```

METHODS

This section describes all the methods that can be used to configure and retrieve the attribute values of an Infoblox::Grid::HostNameRewritePolicy object.

is_grid_default( )

Use this method to set or retrieve the grid's default hostname rewrite policy.

Parameter

Specify "true" to set a policy as the grid default hostname rewrite policy or "false" to disable it. Default value is "false".

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get is_grid_default
my $is_grid_default = $policy->is_grid_default();
#Modify is_grid_default
$policy->is_grid_default('true');
```

name( )

Use this method to set or retrieve the policy name.

Parameter

String with policy name.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get name
my $name = $policy->name();
#Modify name
$policy->name('new_policy');
```

replacement_character( )

Use this method to set or retrieve the replacement character.

Parameter

The replacement character.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get replacement_character
my $replacement_character = $policy->replacement_character();
#Modify name
$policy->replacement_character('-');
```

pre_defined( )

Use this method to retrieve a predefined hostname rewrite policy. This attribute is read-only.

Parameter

None.

Returns

The method returns the attribute value.

Example

```
#Get pre_defined
my $pre_defined = $policy->pre_defined();
```

valid_characters( )

Use this method to set or retrieve permitted characters for a hostname.

Parameter

String with permitted characters for a hostname.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get valid_characters
my $valid_characters = $policy->valid_characters();
#Modify valid_characters
$policy->valid_characters('A-Z0-9_');
```

AUTHOR

Infoblox Inc. http://www.infoblox.com/

SEE ALSO

Infoblox::Grid::DHCP, Infoblox::Grid::Member::DHCP

COPYRIGHT

Copyright (c) 2017 Infoblox Inc.

Infoblox::Grid::ServiceRestart - a Grid Service Restart setting object.

- NAME
- DESCRIPTION
- CONSTRUCTOR
- SESSION METHODS
- MODULE METHODS
  - Infoblox::Grid::DNS->restart_setting( )
  - Infoblox::Grid::DHCP->restart_setting( )
- METHODS
  - delay( )
  - restart_offline( )
  - timeout( )
- AUTHOR
- SEE ALSO
- COPYRIGHT

NAME

Infoblox::Grid::ServiceRestart - a Grid Service Restart setting object.

DESCRIPTION

The Grid Service Restart setting object provides information about service restart configurations.

CONSTRUCTOR

```
my $service_restart = Infoblox::Grid::ServiceRestart->new(
   delay           => $uint,              # Optional / Default is 10
   restart_offline => 'true' | 'false', # Optional / Default is 'true'
   timeout         => $int,               # Optional / Default is 60
```

SESSION METHODS

The object does not support any session methods.

MODULE METHODS

The following functions are available to apply to a Grid Service Restart object.

Infoblox::Grid::DNS->restart_setting( )

Use this method to set or retrive Grid DNS service restart settings. See Infoblox::Grid::DNS->restart_setting() for parameters and return values.

Example

```
#Get restart_setting
my $restart_setting = $grid_dns->restart_setting();
#Modify the retry time
$grid_dns->restart_setting($restart_setting);
```

Infoblox::Grid::DHCP->restart_setting( )

Use this method to set or retrive Grid DHCP service restart settings. See Infoblox::Grid::DHCP->restart_setting() for parameters and return values.

Example

```
#Get restart_setting
my $restart_setting = $grid_dns->restart_setting();
#Modify the retry time
$grid_dns->restart_setting($restart_setting);
```

METHODS

delay( )

Use this method to set or retrieve restart delay in seconds between Grid Service Groups.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

The valid value is an unsigned integer. The default value is 10.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get delay
my $delay = $svcrst->delay();
#Modify delay
$svcrsts->delay(50);
```

restart_offline( )

Use this method to enable or disable the restart of offline Grid Members.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

Specify 'true' to enable the restart of offline Grid Services or 'false' to disable it. The default value is 'true'.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get restart_offline
my $restart_offline = $svcrst->restart_offline();
#Modify restart_offline
$svcrsts->restart_offline('false');
```

timeout( )

Use this method to set or retrieve timeout interval in seconds for Grid Service Groups restart.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

The valid value is an integer greater or equal to -1 (where -1 means infinite). The default value is 10.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get timeout
my $timeout = $svcrst->timeout();
#Modify timeout
$svcrst->timeout(50);
```

AUTHOR

Infoblox Inc. http://www.infoblox.com/

SEE ALSO

Infoblox::Grid::DNS, Infoblox::Grid::DNS->restart_setting(), Infoblox::Grid::DHCP, Infoblox::Grid::DHCP->restart_setting()

COPYRIGHT

Infoblox::Grid::ServiceRestart - a Grid Service Restart setting object.

Copyright (c) 2017 Infoblox Inc.

Infoblox::Discovery::NetworkDeprovisionInfo - Network deprovision information.

- NAME
- DESCRIPTION
- CONSTRUCTOR
- SESSION METHODS
- METHODS
  - interface( )
  - network( )
  - network_view( )
- AUTHOR
- COPYRIGHT

NAME

Infoblox::Discovery::NetworkDeprovisionInfo - Network deprovision information.

DESCRIPTION

The object provides information about network deprovision.

CONSTRUCTOR

```
my $object = Infoblox::Discovery::NetworkDeprovisionInfo->new(
    'interface'          => $interface,       # Required
    'network'            => $network,         # Required
    'network_view'       => $address/cidr,    # Optional, default value is the default network view
);
```

SESSION METHODS

The object does not support any session methods.

METHODS

interface( )

Use this method to set or retrieve the interface object.

Parameter

Valid value is an Infoblox::Grid::Discovery::DeviceInterface object.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get interface value
$value = $deprovision_info->interface();

#Set interface value
$deprovision_info->interface($interface);
```

network( )

Use this method to set or retrieve the network address.

Parameter

Valid value is a string representing the address and netmask of the network in CIDR format (example: 10.0.0.0/8).

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get attribute value
my $value = $deprovision_info->network();

#Set attribute value
$deprovision_info->network("10.0.0.0/8");
```

network_view( )

Use this method to set or retrieve the network view name.

Parameter

Valid value is a string representing network view name.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get attribute value
my $value = $deprovision_info->network_view();

#Set attribute value
$deprovision_info->network_view($view);
```

AUTHOR

Infoblox Inc. http://www.infoblox.com/

COPYRIGHT

Copyright (c) 2017 Infoblox Inc.

Infoblox::Grid::Discovery::DiagnosticsStatus - The discovery diagnostic status.

- NAME
- DESCRIPTION
- SESSION METHODS
  - Infoblox::Session->discovery_diagnostics_status( )
- METHODS
  - end( )
  - id( )
  - start( )
  - status( )
  - text( )
- AUTHOR
- SEE ALSO
- COPYRIGHT

NAME

Infoblox::Grid::Discovery::DiagnosticsStatus - The discovery diagnostic status.

DESCRIPTION

The object provides information about the discovery diagnostic status.

SESSION METHODS

This section describes all the methods in Infoblox::Session that you can apply to the object.

Infoblox::Session->discovery_diagnostics_status( )

Use this method to retrieve the discovery diagnostic status.

Key References

```
Apply the following attributes to get a specific object:

  id     - Required. The discovery diagnostic status ID.
  start  - Optional. The start position of the text.
```

Returns

The method returns an Infoblox::Grid::Discovery::DiagnosticsStatus object when the operation succeeds, and returns false when the operation fails.

Example

```
my $id = $session->discovery_diagnostics('address' => '10.0.0.1');
my $object = $session->discovery_diagnostics_status('id' => $id);
```

METHODS

This section describes all the methods that you can use to set or retrieve the attribute values of the object.

end( )

Use this method to retrieve the end position of text. This is a read-only attribute.

Parameter

None

Returns

The method returns the attribute value.

Example

```
# Get attribute value
my $value = $object->end();
```

id( )

Use this method to retrieve the discovery diagnostic status ID. This is a read-only attribute.

Parameter

None

Returns

The method returns the attribute value.

Example

```
# Get attribute value
my $value = $object->id();
```

start( )

Use this method to retrieve the start position of the text. This is a read-only attribute.

Parameter

None

Returns

The method returns the attribute value.

Example

```
# Get attribute value
my $value = $object->start();
```

status( )

Use this method to retrieve the status. This is a read-only attribute.

Parameter

None

Returns

The method returns the attribute value that can be 'COMPLETED', 'FAILED' or 'INPROGRESS'.

Example

```
# Get attribute value
my $value = $object->status();
```

text( )

Use this method to retrieve the text. This is a read-only attribute.

Parameter

None

Returns

The method returns the attribute value.

Example

```
# Get attribute value
my $value = $object->text();
```

AUTHOR

Infoblox Inc. http://www.infoblox.com/

SEE ALSO

Infoblox::Session

Infoblox::Grid::Discovery::DiagnosticsStatus - The discovery diagnostic status.

COPYRIGHT

Copyright (c) 2017 Infoblox Inc.

Infoblox::Grid::Discovery::ConversionAttributes - conversion attributes.

- NAME
- DESCRIPTION
- SESSION METHODS
- METHODS
  - configure_for_dhcp()
  - configure_for_dns()
  - disabled()
  - comment()
  - zone()
- AUTHOR
- SEE ALSO
- COPYRIGHT

NAME

Infoblox::Grid::Discovery::ConversionAttributes - conversion attributes.

DESCRIPTION

The object represents the list of attributes for discovery data conversion.

SESSION METHODS

The object does not support any session methods.

METHODS

configure_for_dhcp()

Use this method to set the configure_for_dhcp flag of a DHCP Host Address object.

Parameter

Specify "true" to set the configure_for_dhcp flag or "false" to deactivate/unset it. The default value is "false".

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Set value
my $value = $conversion_attributes->configure_for_dhcp('true');
```

configure_for_dns()

Use this method to set the configure_for_dns flag of a DNS host.

Parameter

Specify "true" to set the configure_for_dns flag or "false" to deactivate/unset it. The default value is "false".

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Set value
my $value = $conversion_attributes->configure_for_dns('true');
```

disabled()

Use this method to set or retrieve the flag that indicates whether the target object should be disabled or not.

Parameter

Specify "true" to set the disable flag or "false" to deactivate/unset it. The default value is "false".

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Set value
my $value = $conversion_attributes->disable('true');
```

comment()

Use this method to set or retrieve a descriptive comment.

Parameter

Desired comment in string format with a maximum of 256 bytes. The default value is undefined.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Set value
my $value = $conversion_attributes->comment('test comment');
```

zone()

Use this method to set the authoritative zone for DNS record.

Parameter

The valid value is an Infoblox::DNS::Zone object.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
my $zone = $discovery->get('object' => 'Infoblox::DNS::Zone', 'name' => 'papi.com');
#Set value
my $value = $conversion_attributes->zone($zone);
```

AUTHOR

Infoblox Inc. http://www.infoblox.com/

SEE ALSO

Infoblox::Grid::Discovery, Infoblox::DNS::Zone.

COPYRIGHT

Copyright (c) 2017 Infoblox Inc.

Infoblox::Grid::Discovery::DiscoveryDataConversionResult - discovery data conversion result.

- NAME
- DESCRIPTION
- SESSION METHODS
- METHODS
  - object()
  - address()
  - status()
  - message()
- AUTHOR
- SEE ALSO
- COPYRIGHT

NAME

Infoblox::Grid::Discovery::DiscoveryDataConversionResult - discovery data conversion result.

DESCRIPTION

The object represents the result of the discovery data conversion. This is a read-only object.

SESSION METHODS

The object does not support any session methods.

METHODS

object()

Use this method to retrieve the reference to the converted object.

Parameter

None

Returns

The method returns the attribute value.

Example

```
#Get value
my $object = $conversion_result->object();
```

address()

Use this method to retrieve the reference to the Infoblox::IPAM::Address object that has been converted.

Parameter

None

Returns

The method returns the attribute value.

Example

```
#Get value
my $address = $conversion_result->address();
```

status()

Use this method to retrieve the status of the conversion operation.

Parameter

None

Returns

The method returns the attribute value.

Example

```
#Get value
my $status = $conversion_result->status();
```

message()

Use this method to retrieve the detailed status of the conversion operation. This could contain either the error or success message.

Parameter

None

Returns

The method returns the attribute value.

Example

```
#Get value
my $message = $conversion_result->message();
```

AUTHOR

Infoblox Inc. http://www.infoblox.com/

SEE ALSO

Infoblox::Grid::Discovery, Infoblox::IPAM::Address.

COPYRIGHT

Copyright (c) 2017 Infoblox Inc.

Infoblox::Grid::Discovery::Device - the discovery device.

- NAME
- DESCRIPTION
- SESSION METHODS
  - Infoblox::Session->get( )
  - Infoblox::Session->modify( )
  - Infoblox::Session->search( )
- METHODS
  - address( )
  - address_object( )
  - cap_admin_status_ind( )
  - cap_admin_status_na_reason( )
  - cap_description_ind( )
  - cap_description_na_reason( )
  - cap_net_deprovisioning_ind( )
  - cap_net_deprovisioning_na_reason( )
  - cap_net_provisioning_ind( )
  - cap_net_provisioning_na_reason( )
  - cap_net_vlan_provisioning_ind( )
  - cap_net_vlan_provisioning_na_reason( )
  - cap_vlan_assignment_ind( )
  - cap_vlan_assignment_na_reason( )
  - cap_voice_vlan_ind( )
  - cap_voice_vlan_na_reason( )
  - chassis_serial_number( )
  - class( )
  - description( )
  - discover_now_status( )
  - extattrs( )
  - interfaces( )
  - location( )
  - model( )
  - ms_ad_user_data( )
  - name( )
  - neighbors( )
  - network( )
  - network_view( )
  - networks( )
  - os_version( )
  - port_stats( )
  - vendor( )
  - vlan_infos( )
- AUTHOR
- SEE ALSO
- COPYRIGHT

NAME

Infoblox::Grid::Discovery::Device - the discovery device.

DESCRIPTION

The object provides information about the discovery device.

SESSION METHODS

This section describes all the methods in Infoblox::Session that you can apply to the object.

Infoblox::Session->get( )

Use this method to retrieve all the matching objects from the Infoblox appliance. See Infoblox::Session->get() for parameters and return values.

Note that you should use 'return_methods' attribute to retrieve 'networks', 'interfaces', 'cap_admin_status_ind', 'cap_admin_status_na_reason', 'cap_description_ind', 'cap_description_na_reason', 'cap_net_deprovisioning_ind', 'cap_net_deprovisioning_na_reason', 'cap_net_provisioning_ind', 'cap_net_provisioning_na_reason', 'cap_net_vlan_provisioning_ind', 'cap_net_vlan_provisioning_na_reason', 'cap_vlan_assignment_ind', 'cap_vlan_assignment_na_reason', 'cap_voice_vlan_ind', 'cap_voice_vlan_na_reason' and 'neighbors' members of the device.

Key References

```
Apply the following attributes to get a specific object:

 address     - Optional. The IP address of the device.
 chassis_serial_number - Optional. The device chassis serial number.
 name        - Optional. The device name.
 model       - Optional. The device model.
 class       - Optional. The device class.
 location    - Optional. The device location.
 description - Optional. The device description.
 discovery_member - Optional. The name of the discovery member.
```

Example

```
# without networks', 'interfaces' and 'neighbors' members
my @retrieved_objs = $session->get(
    'object' => 'Infoblox::Grid::Discovery::Device',
    'name'   => 'device',
);

# with networks', 'interfaces' and 'neighbors' members
my @retrieved_objs = $session->get(
    'object' => 'Infoblox::Grid::Discovery::Device',
    'return_methods' => ['+networks', '+interfaces', '+neighbors'],
    'name'   => 'device',
);
```

Infoblox::Session->modify( )

Use this method to modify an object in the Infoblox appliance. See Infoblox::Session->modify() for parameters and return values.

Example

```
# Use this method to modify extensible attributes
$object->extattrs({ 'Site' => $extattr1, 'Administrator' => $extattr2 });
# Submit modification
my $response = $session->modify( $object );
```

Infoblox::Session->search( )

Use this method to search for objects in the Infoblox appliance. See Infoblox::Session->search() for parameters and return values.

Note that you should use 'return_methods' attribute to retrieve 'networks', 'interfaces', 'cap_admin_status_ind', 'cap_admin_status_na_reason', 'cap_description_ind', 'cap_description_na_reason', 'cap_net_deprovisioning_ind', 'cap_net_deprovisioning_na_reason', 'cap_net_provisioning_ind', 'cap_net_provisioning_na_reason', 'cap_net_vlan_provisioning_ind', 'cap_net_vlan_provisioning_na_reason', 'cap_vlan_assignment_ind', 'cap_vlan_assignment_na_reason', 'cap_voice_vlan_ind', 'cap_voice_vlan_na_reason' and 'neighbors' members of the device.

Key References

```
Apply the following attributes to search for a object:

 address     - Optional. The IP address of the device (regular expression).
 chassis_serial_number - Optional. The device chassis serial number (regular expression).
 name        - Optional. The device name (regular expression).
 model       - Optional. The device model (regular expression).
 class       - Optional. The device class (regular expression).
 location    - Optional. The device location (regular expression).
 description - Optional. The device description (regular expression).
 discovery_member - Optional. The host name of the discovery member.
```

Example

```
# without networks', 'interfaces' and 'neighbors' members
my @retrieved_objs = $session->search(
    'object' => 'Infoblox::Grid::Discovery::Device',
    'name'   => 'device',
);

# with networks', 'interfaces' and 'neighbors' members
my @retrieved_objs = $session->session(
    'object' => 'Infoblox::Grid::Discovery::Device',
    'return_methods' => ['+networks', '+interfaces', '+neighbors'],
    'name'   => 'device',
);
```

METHODS

This section describes all the methods that you can use to set or retrieve the attribute values of the object.

address( )

Use this method to retrieve the IP address of the device. This is a read-only attribute.

Parameter

None

Returns

The method returns the attribute value.

Example

```
# Get attribute value
my $value = $object->address();
```

address_object( )

Use this method to retrieve the IP address object of the device. This is a read-only attribute.

Parameter

None

Returns

The method returns the attribute value that can be an Infoblox::IPAM::Address object.

Example

```
# Get attribute value
my $value = $object->address_object();
```

cap_admin_status_ind( )

Use this method to retrieve the flag indicates whether or not modify the admin status of an interface in this device. This is a read-only attribute.

Parameter

None

Returns

The method returns the attribute value. Note that this method has to be explicitly requested via Infoblox::Session's return_methods member.

Example

```
# Get attribute value
my $value = $object->cap_admin_status_ind();
```

cap_admin_status_na_reason( )

Use this method to retrieve the reason if the edit admin status action is not available. This is a read-only attribute.

Parameter

None

Returns

The method returns the attribute value. Note that this method has to be explicitly requested via Infoblox::Session's return_methods member.

Example

```
# Get attribute value
my $value = $object->cap_admin_status_na_reason();
```

cap_description_ind( )

Use this method to retrieve the flag indicates whether or not modify the description of an interface in this device. This is a read-only attribute.

Parameter

None

Returns

The method returns the attribute value. Note that this method has to be explicitly requested via Infoblox::Session's return_methods member.

Example

```
# Get attribute value
my $value = $object->cap_description_ind();
```

cap_description_na_reason( )

Use this method to retrieve the reason if the description edit action is not available. This is a read-only attribute.

Parameter

None

Returns

The method returns the attribute value. Note that this method has to be explicitly requested via Infoblox::Session's return_methods member.

Example

```
# Get attribute value
my $value = $object->cap_description_na_reason();
```

cap_net_deprovisioning_ind( )

Use this method to retrieve the flag indicates whether or not deprovision a network from interfaces of this device. This is a read-only attribute.

Parameter

None

Returns

The method returns the attribute value. Note that this method has to be explicitly requested via Infoblox::Session's return_methods member.

Example

```
# Get attribute value
my $value = $object->cap_net_deprovisioning_ind();
```

cap_net_deprovisioning_na_reason( )

Use this method to retrieve the reason if the deprovision a network from interfaces of this device is not available. This is a read-only attribute.

Parameter

None

Returns

The method returns the attribute value. Note that this method has to be explicitly requested via Infoblox::Session's return_methods member.

Example

```
# Get attribute value
my $value = $object->cap_net_deprovisioning_na_reason();
```

cap_net_provisioning_ind( )

Use this method to retrieve the flag indicates whether or not modify the network associated to an interface of this device. This is a read-only attribute.

Parameter

None

Returns

The method returns the attribute value. Note that this method has to be explicitly requested via Infoblox::Session's return_methods member.

Example

```
# Get attribute value
my $value = $object->cap_net_provisioning_ind();
```

cap_net_provisioning_na_reason( )

Use this method to retrieve the reason if the network provisioning is not available. This is a read-only attribute.

Parameter

None

Returns

The method returns the attribute value. Note that this method has to be explicitly requested via Infoblox::Session's return_methods member.

Example

```
# Get attribute value
my $value = $object->cap_net_provisioning_na_reason();
```

cap_net_vlan_provisioning_ind( )

Use this method to retrieve the flag indicates whether or not create a VLAN and then provision a network in to this interface. This is a read-only attribute.

Parameter

None

Returns

The method returns the attribute value. Note that this method has to be explicitly requested via Infoblox::Session's return_methods member.

Example

```
# Get attribute value
my $value = $object->cap_net_vlan_provisioning_ind();
```

cap_net_vlan_provisioning_na_reason( )

Use this method to retrieve the reason if the network provisioning on VLAN is not available. This is a read-only attribute.

Parameter

None

Returns

The method returns the attribute value. Note that this method has to be explicitly requested via Infoblox::Session's return_methods member.

Example

```
# Get attribute value
my $value = $object->cap_net_vlan_provisioning_na_reason();
```

cap_vlan_assignment_ind( )

Use this method to retrieve the flag indicates whether or not modify the VLAN assignement of an interface in this device. This is a read-only attribute.

Parameter

None

Returns

The method returns the attribute value. Note that this method has to be explicitly requested via Infoblox::Session's return_methods member.

Example

```
# Get attribute value
my $value = $object->cap_vlan_assignment_ind();
```

cap_vlan_assignment_na_reason( )

Use this method to retrieve the reason if the vlan assignment action is not available. This is a read-only attribute.

Parameter

None

Returns

The method returns the attribute value. Note that this method has to be explicitly requested via Infoblox::Session's return_methods member.

Example

```
# Get attribute value
my $value = $object->cap_vlan_assignment_na_reason();
```

cap_voice_vlan_ind( )

Use this method to retrieve the flag indicates whether or not modify the voice vlan assignement of an interface in this device. This is a read-only attribute.

Parameter

None

Returns

The method returns the attribute value. Note that this method has to be explicitly requested via Infoblox::Session's return_methods member.

Example

```
# Get attribute value
my $value = $object->cap_voice_vlan_ind();
```

cap_voice_vlan_na_reason( )

Use this method to retrieve the reason if the voice vlan assignment action is not available. This is a read-only attribute.

Parameter

None

Returns

The method returns the attribute value. Note that this method has to be explicitly requested via Infoblox::Session's return_methods member.

Example

```
# Get attribute value
my $value = $object->cap_voice_vlan_na_reason();
```

chassis_serial_number( )

Use this method to retrieve the device chassis serial number. This is a read-only attribute.

Parameter

None

Returns

The method returns the attribute value.

Example

```
# Get attribute value
my $value = $object->chassis_serial_number();
```

class( )

Use this method to retrieve the device class. This is a read-only attribute.

Parameter

None

Returns

The method returns the attribute value.

Example

```
# Get attribute value
my $value = $object->class();
```

description( )

Use this method to retrieve the device description. This is a read-only attribute.

Parameter

None

Returns

The method returns the attribute value.

Example

```
# Get attribute value
my $value = $object->description();
```

discover_now_status( )

Use this method to retrieve the current discovery status. This attribute is read-only.

Parameter

None.

Returns

The method returns the attribute value that can be 'COMPLETE', 'FAILED', 'NONE', 'PENDING' or 'RUNNING'.

Example

```
#Get discover_now_status
my $discover_now_status = $object->discover_now_status();
```

extattrs( )

Use this method to set or retrieve the extensible attributes associated with the Device object.

Parameter

Valid value is a hash reference containing the names of extensible attributes and their associated values (Infoblox::Grid::Extattr objects).

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get extattrs
my $ref_extattrs = $object->extattrs();
#Modify extattrs
$object->extattrs({ 'Site' => $extattr1, 'Administrator' => $extattr2 });
```

interfaces( )

Use this method to retrieve a list of the device interfaces. This is a read-only attribute.

Parameter

None

Returns

The method returns the attribute value that can be an array reference that contains Infoblox::Grid::Discovery::DeviceInterface objects. Note that this method has to be explicitly requested via Infoblox::Session's return_methods member.

Example

```
# Get attribute value
my $value = $object->interfaces();
```

location( )

Use this method to retrieve the device location. This is a read-only attribute.

Parameter

None

Returns

The method returns the attribute value.

Example

```
# Get attribute value
my $value = $object->location();
```

model( )

Use this method to retrieve the device model name. This is a read-only attribute.

Parameter

None

Returns

The method returns the attribute value.

Example

```
# Get attribute value
my $value = $object->model();
```

ms_ad_user_data( )

Use this method to retrieve Microsoft Active Directory users related information. This is a read-only attribute.

Parameter

None

Returns

The valid return value is an Infoblox::Grid::MSServer::AdUser::Data object.

Example

```
#Get ms_ad_user_data
my $ms_ad_user_data = $object->ms_ad_user_data();
```

name( )

Use this method to retrieve the device name. This is a read-only attribute.

Parameter

None

Returns

The method returns the attribute value.

Example

```
# Get attribute value
```

```
my $value = $object->name();
```

neighbors( )

Use this method to retrieve a list of the device neighbors. This is a read-only attribute.

Parameter

None

Returns

The method returns the attribute value that can be an array reference that contains Infoblox::Grid::Discovery::DeviceNeighbor objects. Note that this method has to be explicitly requested via Infoblox::Session's return_methods member.

Example

```
# Get attribute value
my $value = $object->neighbors();
```

network( )

Use this method to retrieve the network object. This is a read-only attribute.

Parameter

None

Returns

The method returns the attribute value that can be an Infoblox::DHCP::Network or Infoblox::DHCP::IPv6Network object.

Example

```
# Get attribute value
my $value = $object->network();
```

network_view( )

Use this method to retrieve the network view name. This is a read-only attribute.

Parameter

None

Returns

The method returns the attribute value.

Example

```
# Get attribute value
my $value = $object->network_view();
```

networks( )

Use this method to retrieve a list of network objects. This is a read-only attribute.

Parameter

None

Returns

The method returns the attribute value that can be an array reference that contains Infoblox::DHCP::Network and Infoblox::DHCP::IPv6Network objects. Note that this method has to be explicitly requested via Infoblox::Session's return_methods member.

Example

```
# Get attribute value
my $value = $object->networks();
```

os_version( )

Use this method to retrieve the Operating System version running on the device. This is a read-only attribute.

Parameter

None

Returns

The method returns the attribute value.

Example

```
# Get attribute value
my $value = $object->os_version();
```

port_stats( )

Use this method to retrieve a list of device port statistics. This is a read-only attribute.

Parameter

None

Returns

The method returns the attribute value that can be an arrary reference that contains Infoblox::Grid::Discovery::Device::PortStatistics objects.

Example

```
# Get attribute value
my $value = $object->port_stats();
```

vendor( )

Use this method to retrieve the device vendor name. This is a read-only attribute.

Parameter

None

Returns

The method returns the attribute value.

Example

```
# Get attribute value
my $value = $object->vendor();
```

vlan_infos( )

Use this method to retrieve a list of VLAN information. This is a read-only attribute.

Parameter

None

Returns

The method returns the attribute value that can be an arrary reference that contains Infoblox::Grid::Discovery::VLANInfo objects.

Example

```
# Get attribute value
my $value = $object->vlan_infos();
```

AUTHOR

Infoblox Inc. http://www.infoblox.com/

SEE ALSO

Infoblox::Session, Infoblox::DHCP::Network, Infoblox::DHCP::IPv6Network, Infoblox::Grid::Discovery::DeviceNeighbor, Infoblox::Grid::Discovery::DeviceInterface, Infoblox::IPAM::Address, Infoblox::Grid::MSServer::AdUser::Data

---

COPYRIGHT

Infoblox::Grid::Discovery::Device - the discovery device.

Copyright (c) 2017 Infoblox Inc.

Infoblox::Grid::Discovery::DeviceSupportInfoResponse - the device support info response.

- NAME
- DESCRIPTION
- SESSION METHODS
- METHODS
  - device_data_collection_status()
  - device_support_info()
- AUTHOR
- SEE ALSO
- COPYRIGHT

NAME

Infoblox::Grid::Discovery::DeviceSupportInfoResponse - the device support info response.

DESCRIPTION

The object is created as a result of Infoblox::Grid::Discovery->get_device_support_info() function call and cannot be added or removed manually.

SESSION METHODS

The object does not support any session methods.

METHODS

This section describes all the methods that you can use to retrieve the attribute values of a device support info response.

device_data_collection_status()

Use this method to retrieve the list of Discovery device data collection statuses. This is a read-only attribute.

Parameter

None

Returns

The method returns the attribute value, that is an array of Infoblox::Grid::Discovery::DeviceDataCollectionStatus objects.

Example

```
#Get value
my $device_data_collection_status = $device_support_info_response->device_data_collection_status();
```

device_support_info()

Use this method to retrieve the list of Discovery device support infos. This is a read-only attribute.

Parameter

None

Returns

The method returns the attribute value, that is an array of Infoblox::Grid::Discovery::DeviceSupportInfo objects.

Example

```
#Get value
my $device_support_info = $device_support_info_response->device_support_info();
```

AUTHOR

Infoblox Inc. http://www.infoblox.com/

SEE ALSO

Infoblox::Grid::Discovery, Infoblox::Grid::Discovery::DeviceDataCollectionStatus, Infoblox::Grid::Discovery::DeviceSupportInfo.

Infoblox::Grid::Discovery::DeviceSupportInfoResponse - the device support info response.

COPYRIGHT

Copyright (c) 2017 Infoblox Inc.

Infoblox::Grid::Discovery::JobProcessDetails - job process details object.

- NAME
- DESCRIPTION
- SESSION METHODS
- METHODS
  - end_line( )
  - status( )
  - stream( )
- AUTHOR
- SEE ALSO
- COPYRIGHT

NAME

Infoblox::Grid::Discovery::JobProcessDetails - job process details object.

DESCRIPTION

The object provides discovery job processing logs related to a given task.

SESSION METHODS

The object does not support any session methods.

METHODS

end_line( )

Use this method to retrieve the ending line of the returned stream. This is a read-only attribute.

Parameter

None

Returns

The method returns the attribute value.

Example

```
#Get value
my $value = $process_details->end_line();
```

status( )

Use this method to retrieve the indicator of session status. This is a read-only attribute.

Parameter

None

Returns

The method returns the attribute value that can be "INPROGRESS", "COMPLETED" and "FAILED".

Example

```
#Get value
my $value = $process_details->status();
```

stream( )

Use this method to retrieve the job process log stream. This is a read-only attribute.

Parameter

None

Returns

The method returns the attribute value.

Example

```
#Get value
my $value = $process_details->stream();
```

AUTHOR

Infoblox Inc. http://www.infoblox.com/

SEE ALSO

Infoblox::Grid::Discovery

COPYRIGHT

Copyright (c) 2017 Infoblox Inc.

Infoblox::Grid::Discovery::VLANInfo - VLAN information associated with the discovery interface.

- NAME
- DESCRIPTION
- CONSTRUCTOR
- SESSION METHODS
- METHODS
  - id( )
  - name( )
- AUTHOR
- COPYRIGHT

NAME

Infoblox::Grid::Discovery::VLANInfo - VLAN information associated with the discovery interface.

DESCRIPTION

The object provides VLAN information associated with the discovery interface.

CONSTRUCTOR

```
my $object = Infoblox::Grid::Discovery::VLANInfo->new(
    'id'   => $string,            # Optional
    'name' => $string,            # Optional
);
```

SESSION METHODS

The object does not support any session methods.

METHODS

This section describes all the methods that you can use to set or retrieve the attribute values of the object.

id( )

Use this method to set or retrieve the VLAN ID.

Parameter

The VLAN ID in string format.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
# Get attribute value
my $value = $object->id();
# Modify attribute value
$object->id('23');
```

name( )

Use this method to set or retrieve the VLAN name.

Parameter

The VLAN name in string format.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
# Get attribute value
my $value = $object->name();
# Modify attribute value
$object->name('vlan');
```

AUTHOR

Infoblox Inc. http://www.infoblox.com/

COPYRIGHT

Copyright (c) 2017 Infoblox Inc.

Infoblox::Grid::ScheduleSetting - a schedule setting object.

- NAME
- DESCRIPTION
- CONSTRUCTOR
- SESSION METHODS
- METHODS
  - day_of_month( )
  - disable( )
  - every( )
  - frequency( )
  - hour_of_day( )
  - month( )
  - year( )
  - minutes_past_hour( )
  - repeat( )
  - time_zone( )
  - weekdays( )
- AUTHOR
- SEE ALSO
- COPYRIGHT

NAME

Infoblox::Grid::ScheduleSetting - a schedule setting object.

DESCRIPTION

An object that provides the ability to set the schedule for a discovery task.

CONSTRUCTOR

```
my $schedule_setting = Infoblox::Grid::ScheduleSetting->new(
   frequency           => "DAILY" | "HOURLY" | "MONTHLY" | "WEEKLY",      # optional
   day_of_month        => $uint,                                          # optional
   every               => $uint,                                          # optional
   weekdays            => ["MONDAY", "THURSDAY", .... ],                  # optional
   hour_of_day         => $uint,                                          # optional
   minutes_past_hour   => $uint,                                          # optional
   month               => $uint,                                          # optional
   year                => $uint,                                          # optional
   repeat              => "ONCE" | "RECUR",                               # optional
   disable             => "true" | "false",                               # optional
   time_zone           => $time_zone,                                     # optional
);
```

SESSION METHODS

The object does not support any session methods.

METHODS

This section describes all the methods that you can use to configure and retrieve attribute values of the scheduling object.

day_of_month( )

Use this method to set or retrieve the scheduled day of month when the discovery is triggered.

Parameter

Valid value is an unsigned integer between 1-31.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get attribute value
my $value = $schedule_setting->day_of_month();
#Modify attribute value
$schedule_setting->day_of_month(10);
```

disable( )

Use this method to enable or disable scheduling of a discovery task.

Parameter

Specify "true" to disable scheduling or "false" to enable it. The default value is "false".

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get attribute value
my $value = $schedule_setting->disable();
#Modify attribute value
$schedule_setting->disable("true");
```

every( )

Use this method to set or retrieve the recurrence pattern of a recurring discovery.

Parameter

Valid value is an unsigned integer.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get attribute value
my $value = $schedule_setting->every();
#Modify attribute value
$schedule_setting->every(2);
```

frequency( )

Use this method ot set or retrieve the recurrence pattern of a recurring discovery.

Parameter

Valid value is "DAILY", "HOURLY", "MONTHLY" or "WEEKLY".

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get attribute value
my $value = $schedule_setting->frequency();
#Modify attribute value
$schedule_setting->frequency("MONTHLY");
```

hour_of_day( )

Use this method to set or retrieve the hour, based on the 24-hour clock notation, when the discovery should start. For example, use 18 for 18:00.

Parameter

Valid value is an unsigned integer between 0-23.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get attribute value
my $value = $schedule_setting->hour_of_day();
#Modify attribute value
$schedule_setting->hour_of_day(5);
```

month( )

Use this method to set or retrieve the month of the year when the discovery should start. For example, use 1 for January, 2 for February, and so on.

Parameter

Valid value is an unsigned integer between 1 and 12.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get attribute value
my $value = $schedule_setting->month();
#Modify attribute value
$schedule_setting->month(10);
```

year( )

Use this method to set or retrieve the year when the discovery should start.

Parameter

Valid value is an unsigned integer.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get attribute value
my $value = $schedule_setting->year();
#Modify attribute value
$schedule_setting->year(2015);
```

minutes_past_hour( )

Use this method to set or retrieve the minute, based on the 24-hour clock notation, when the discovery should start. For example, use 59 for 00:59.

Parameter

Valid value is an unsigned integer between 0-59.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get attribute value
my $value = $schedule_setting->minutes_past_hour();
#Modify attribute value
$schedule_setting->minutes_past_hour(10);
```

repeat( )

Use this method to set or retrieve the flag that indicates if the discovery is scheduled for once or recurring.

Parameter

Valid value is "ONCE" or "RECUR".

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get attribute value
my $value = $schedule_setting->repeat();
#Modify attribute value
$schedule_setting->repeat("RECUR");
```

time_zone( )

Use this method to set or retrieve the time zone.

Parameter

The UTC string that represents the time zone. For example "(UTC - 6:00) Central Time (US and Canada)". The default value is "(UTC) Coordinated Universal Time".

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get attribute value
my $value = $schedule_setting->time_zone();
#Modify attribute value
$schedule_setting->time_zone("(UTC - 6:00) Central Time (US and Canada)");
```

weekdays( )

Use this method to set or retrieve the days of the week when the discovery is triggered.

Parameter

Valid value is a reference to an array that can contain the following values: 'SUNDAY', 'MONDAY', 'TUESDAY', 'WEDNESDAY', 'THURSDAY', 'FRIDAY' and 'SATURDAY'.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get attribute value
my $value = $schedule_setting->weekdays();
#Modify attribute value
$schedule_setting->weekdays(["MONDAY", "FRIDAY"]);
```

AUTHOR

Infoblox Inc. http://www.infoblox.com/

SEE ALSO

Infoblox::IPAM::DiscoveryTask

COPYRIGHT

Copyright (c) 2017 Infoblox Inc.

Infoblox::Grid::Discovery::Properties - Grid discovery properties.

- NAME
- DESCRIPTION
- CONSTRUCTOR
- SESSION METHODS
  - Infoblox::Session->get( )
  - Infoblox::Session->modify( )
- METHODS
  - advanced_polling_settings( )
  - auto_conversion_settings( )
  - basic_polling_settings( )
  - enable_auto_conversion( )
  - enable_auto_updates( )
  - cli_credentials( )
  - discovery_blackout_setting( )
  - ignore_conflict_duration( )
  - ports( )
  - port_control_blackout_setting( )
  - same_port_control_discovery_blackout( )
  - snmp3_credentials( )
  - snmp_credentials( )
  - test_credential( )
  - unmanaged_ips_limit( )
  - unmanaged_ips_timeout( )
  - vrf_mapping_rules( )
  - vrf_mapping_policy( )
- AUTHOR
- SEE ALSO
- COPYRIGHT

NAME

Infoblox::Grid::Discovery::Properties - Grid discovery properties.

DESCRIPTION

The object provides information about Grid discovery properties.

CONSTRUCTOR

```
my $object = Infoblox::Grid::Discovery::Properties->new(
    'advanced_polling_settings'      => $adv,                 # Required
    'auto_conversion_settings'       => [$auto_conversion1, ...], # Optional
    'basic_polling_settings'         => $basic,               # Required
    'enable_auto_conversion'         => 'true' | 'false'      # Optional
    'enable_auto_updates'            => 'true' | 'false'      # Optional
    'snmp3_credentials'              => [$snmp3_1, ...],      # Optional
    'snmp_credentials'               => [$snmp_1, ...],       # Optional
    'cli_credentials'                => [$cli1, ...],         # Optional
    'discovery_blackout_setting'     => $blackout,            # Optional
    'port_control_blackout_setting' => $blackout,             # Optional
    'ports'                          => [$port1, ...],        # Optional
    'ignore_conflict_duration'       => $uint,                # Optional
    'same_port_control_discovery_blackout' => 'true' | 'false' # Optional
    'unmanaged_ips_limit'            => $uint,                  # Optional
    'unmanaged_ips_timeout'          => $uint,                  # Optional
    'vrf_mapping_rules'              => [$vrf_rule_1, ...],     # Optional
    'vrf_mapping_policy'             => 'NONE' | 'RULE_BASED' | 'RULE_AND_INTERNAL_BASED', # Optional
);
```

SESSION METHODS

This section describes all the methods in Infoblox::Session that you can apply to a object.

Infoblox::Session->get( )

Use this method to retrieve the existing object from the Infoblox appliance. See Infoblox::Session->get() for parameters and return values.

Example

```
my $object = $session->get('object' => 'Infoblox::Grid::Discovery::Properties');
```

Infoblox::Session->modify( )

Use this method to modify object in the Infoblox appliance. See Infoblox::Session->modify() for parameters and return values.

Example

```
my $port = Infoblox::Grid::Discovery::Port->new(
    'port' => 40,
    'type' => 'UDP',
);

# Use method to modify ports
$object->ports([$port]);
# Submit modification
my $response = $session->modify( $object );
```

METHODS

This section describes all the methods that you can use to set or retrieve the attribute values of the object.

advanced_polling_settings( )

Use this method to set or retrieve discovery advanced poll settings.

Parameter

Valid value is an Infoblox::Grid::Discovery::AdvancedPollSettings object.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
# Get attribute value
my $value = $object->advanced_polling_settings();
# Modify attribute value
$object->advanced_polling_settings($adv);
```

auto_conversion_settings( )

Use this method to set or retrieve a list of automatic conversion settings.

Parameter

Valid value is an array reference that contains Infoblox::Grid::Discovery::AutoConversionSetting objects.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
# Get attribute value
my $value = $object->auto_conversion_settings();
# Modify attribute value
$object->auto_conversion_settings([$auto_conversion_settings1, $auto_conversion_settings2]);
```

basic_polling_settings( )

Use this method to set or retrieve discovery basic poll settings.

Parameter

Valid value is an Infoblox::Grid::Discovery::BasicPollSettings object.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
# Get attribute value
my $value = $object->basic_polling_settings();
# Modify attribute value
$object->basic_polling_settings($basic);
```

enable_auto_conversion( )

Use this method to set or retrieve the flag that enables automatic conversion of discovered data.

Parameter

Specify 'true' to enable automatic conversion of discovery data or 'false' to disable it. The default value is 'false'.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
# Get attribute value
my $value = $object->enable_auto_conversion();
# Modify attribute value
$object->enable_auto_conversion('true');
```

enable_auto_updates( )

Use this method to set or retrieve the flag that enables updating discovered data for managed objects.

Parameter

Specify 'true' to enable updating discovered data for managed objects or 'false' to disable it. The default value is 'true'.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
# Get attribute value
my $value = $object->enable_auto_updates();
# Modify attribute value
$object->enable_auto_updates('true');
```

cli_credentials( )

Use this method to set or retrieve the discovery CLI credentials.

Parameter

Valid value is an array reference that contains Infoblox::Grid::Discovery::CLICredential objects.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
# Get attribute value
my $value = $object->cli_credentials();
# Modify attribute value
$object->cli_credentials([$cli1, $cli2]);
```

discovery_blackout_setting( )

Use this method to set or retrieve the discovery blackout setting.

Parameter

Valid value is an Infoblox::Grid::Discovery::Properties::BlackoutSetting object.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
# Get attribute value
my $value = $object->discovery_blackout_setting();
# Modify attribute value
$object->discovery_blackout_setting($blackout);
```

ignore_conflict_duration( )

Use this method to set or retrieve the ignore discovery conflict duration.

Parameter

The ignore discovery conflict duration in seconds.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
# Get attribute value
my $value = $object->ignore_conflict_duration();
# Modify attribute value
$object->ignore_conflict_duration(3600);
```

ports( )

Use this method to set or retrieve ports to scan.

Parameter

Valid value is an array reference that contains Infoblox::Grid::Discovery::Port objects.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
# Get attribute value
my $value = $object->ports();
# Modify attribute value
$object->ports([$port1, $port2]);
```

port_control_blackout_setting( )

Use this method to set or retrieve the port control blackout setting.

Parameter

Valid value is an Infoblox::Grid::Discovery::Properties::BlackoutSetting object.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
# Get attribute value
my $value = $object->port_control_blackout_setting();
# Modify attribute value
$object->port_control_blackout_setting($blackout);
```

same_port_control_discovery_blackout( )

Use this method to set or retrieve the flag that enables using the same discovery blackout for port control blackout.

Parameter

Specify 'true' to use same discovery blackout for port control blackout or 'false' to disable it. The default value is 'false'.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
# Get attribute value
my $value = $object->same_port_control_discovery_blackout();
# Modify attribute value
$object->same_port_control_discovery_blackout('true');
```

snmp3_credentials( )

Use this method to set or retrieve a list of discovery SNMPv3 credentials.

Parameter

Valid value is an array reference that contains Infoblox::Grid::Discovery::SNMP3Credential objects.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
# Get attribute value
my $value = $object->snmp3_credentials();
# Modify attribute value
$object->snmp3_credentials([$snmp3_1, $snmp3_2]);
```

snmp_credentials( )

Use this method to set or retrieve a list of discovery SNMP v1 and v2 credentials>.

Parameter

Valid value is an array reference that contains Infoblox::Grid::Discovery::SNMPCredential objects.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
# Get attribute value
my $value = $object->snmp_credentials();
# Modify attribute value
$object->snmp_credentials([$snmp1, $snmp2]);
```

test_credential( )

Use this method to test credentials.

Note that this method does not perform automatic polling for the operational status. It returns a snapshot of the status at the time when you call the method, which may not be the final status. To get the latest status, the client script needs to provide polling logic for this method until the operation status is 'INPROGRESS' by calling the method with start parameter assigned to the end field in the previous call result. For more details, see the example below.

Parameter

```
id              - Optional. The test credential ID.
```

The following members are supported when id is defined:

```
start           - Optional. The start position of the text.
```

The following members are supported when id is not defined:

```
 view             - Optional. The network view name. The default value is the system-defined default
network view.

 type             - Optional if one of the following is defined: cli_credentials, snmp_credential, or
snmp3_credential. Otherwise, this member is required. The valid value is 'CLI', 'SNMP' or 'SNMP3'.

 device           - Required if address is not defined. The Infoblox::Grid::Discovery::Device object.

 member           - Optional. The discovery member name.

 address          - Required if device is not defined. The IP address of the device.

 cli_credentials  - Optional. At least one of the following must be defined: cli_credentials,
snmp_credential, or snmp3_credential. The array reference that contains
Infoblox::Grid::Discovery::CLICredential objects.

 snmp_credential  - Optional. At least one of the following must be defined: cli_credentials,
snmp_credential, or snmp3_credential. The Infoblox::Grid::Discovery::SNMPCredential object.

 snmp3_credential - Optional. At least one of the following must be defined: cli_credentials,
snmp_credential, or snmp3_credential. The Infoblox::Grid::Discovery::SNMP3Credential object.
```

Returns

The method returns an Infoblox::Grid::Discovery::TestCredential object when the operation succeeds, and returns false when the operation fails.

Example

```
# Get a test credential ID
my $result = $object->test_credential(
   'address'         => '10.0.0.1',
   'snmp_credential' => $snmp,
);

# Get detailed information by id
if (ref($result) eq 'Infoblox::Grid::Discovery::TestCredential') {
    $result = $object->test_credential('id' => $result->id());
}

# check for the 'COMPLETE' or 'FAILED' status of test credential
while (ref($result) eq 'Infoblox::Grid::Discovery::TestCredential' &&
       $result->status() eq 'INPROGRESS') {
    $result = $grid->test_credential('id' => $result->id(), 'start' => $result->end());
}
```

unmanaged_ips_limit( )

Use this method to set or retrieve the limit value for discovered unmanaged IP addresses. This value determines how often users are notified when new unmanaged IP addresses are discovered in a particular network.

Parameter

The valid value is an unsigned integer. The minimun value is 1.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
# Get attribute value
my $value = $object->unmanaged_ips_limit();
# Modify attribute value
$object->unmanaged_ips_limit(30);
```

unmanaged_ips_timeout( )

Use this method to set or retrieve the timeout value (in seconds) between notifications about new unmanaged IP addresses in a particular network.

Parameter

The valid value is an unsigned integer. The minimum value is 60 (one minute), and the maximum value is the number of seconds from the time you set it until January 2038.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
# Get attribute value
my $value = $object->unmanaged_ips_timeout();
# Modify attribute value
$object->unmanaged_ips_timeout(300);
```

vrf_mapping_rules( )

Use this method to set or retrieve a list of VRF mapping rules.

Parameter

Valid value is an array reference that contains Infoblox::Grid::Discovery::VRFMappingRule objects.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
# Get attribute value
my $value = $object->vrf_mapping_rules();
# Modify attribute value
$object->vrf_mapping_rules([$vrf_mapping_rule1, $vrf_mapping_rule2]);
```

vrf_mapping_policy( )

Use this method to set or retrieve the VRF mapping policy.

Parameter

The policy type used to define the behavior of the VRF mapping. Valid value is one of the following policy types: 'NONE', 'RULE_BASED', 'RULE_AND_INTERNAL_BASED'.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
# Get attribute value
my $value = $object->vrf_mapping_policy();
# Modify attribute value
$object->vrf_mapping_policy('RULE_AND_INTERNAL_BASED');
```

AUTHOR

Infoblox Inc. http://www.infoblox.com/

SEE ALSO

Infoblox::Session, Infoblox::Grid::Discovery::SNMPCredential, Infoblox::Grid::Discovery::SNMP3Credential, Infoblox::Grid::Discovery::Port, Infoblox::Grid::Discovery::CLICredential, Infoblox::Grid::Discovery::Properties::BlackoutSetting, Infoblox::Grid::Discovery::VRFMappingRule, Infoblox::Grid::Discovery::AutoConversionSetting

---

COPYRIGHT

Copyright (c) 2017 Infoblox Inc.

Infoblox::Grid::Discovery - Grid discovery functions.

- NAME
- DESCRIPTION
- CONSTRUCTOR
- SESSION METHODS
- METHODS
  - clear_network_port_assignment( )
  - control_ip_address( )
  - control_switch_port( )
  - diagnostics( )
  - diagnostics_status( )
  - discovery_data_conversion( )
  - get_device_support_info( )
  - get_job_devices( )
  - get_job_process_details( )
  - provision_network_dhcp_relay( )
  - provision_network_port( )
  - start_discovery( )
- SEE ALSO
- AUTHOR
- COPYRIGHT

NAME

Infoblox::Grid::Discovery - Grid discovery functions.

DESCRIPTION

The object provides access to the Grid discovery and Network Insight functions.

CONSTRUCTOR

```
my $discovery = Infoblox::Grid::Discovery->new(
    'session' => $session,      # Required
);
```

SESSION METHODS

The object does not support any session methods.

METHODS

clear_network_port_assignment( )

Use this method to clear network provision information on given router or switch-router device interfaces for given networks.

Note that this function is always executed as part of scheduled task. Also the function goes through approval process if it is applicable.

Parameter

network_deprovision_info - Required. An array reference that contains Infoblox::Grid::Discovery::NetworkDeprovisionInfo objects.
scheduled_at - Required. Scheduled time in ISO 8601 extended format for date and time (e.g. 2008-10-21T10:56:00Z or 2008-10-21T10:56:00.001Z) or 'now' which will set scheduled date and time to the current date and time.

Returns

The method returns true when a scheduled task has been created successfully, and returns false when the operation fails.

Example

```
 my $res = $discovery->clear_network_port_assignment('scheduled_at' => $scheduled_time,
'network_deprovision_info' => [$deprovision_info1, $deprovision_info2]);
```

control_ip_address( )

Use this method to control multiple IP addresses.

Parameter

addresses - Required. An array reference that contains IP addresses. network_view - Optional. The network view name. exclude - Optional. The flag for exclude. Valid value is 'true' or 'false'.

Returns

The method returns true when the modification succeeds, and returns false when the operation fails.

Example

```
my $res = $discovery->control_ip_address('addresses' => ['10.0.0.1', '10.0.0.2']);
```

control_switch_port( )

Use this method to configure the interface of a given switch device.

Note that this function is always executed as part of scheduled task. Also the function goes through approval process if it is applicable. This function is used for both IPAM IP address object port configuration and device interface port control.

Parameter

port_config - Required. An array reference that contains Infoblox::Grid::Discovery::Port::Control objects. scheduled_at - Required. Scheduled time in ISO 8601 extended format for date and time (e.g. 2008-10-21T10:56:00Z or 2008-10-21T10:56:00.001Z) or 'now' which will set scheduled date and time to the current date and time.

Returns

The method returns "true" when a scheduled task has been created successfully, and returns "false" when the operation fails.

Example

```
my $res = $discovery->control_switch_port('scheduled_at' => $scheduled_time, 'port_configs' =>
[$config1, $config2]);
```

diagnostics( )

Use this method to execute discovery diagnostics for the IP address.

Parameter

address - Required. The IP address. community_string - Optional. The communitry string. force_test - Optional. The flag for force test. Valid value is 'true' or 'false'. discovery_member - Optional. The discovery member name. network_view - Optional. The network view name. debug_snmp - Optional. The flag for enable/disable SNMP debug (enabled by default). Valid value is 'true' or 'false'.

Returns

The method returns a discovery diagnostic status ID in string format when the operations succeeds, and returns false when teh operation fails.

Example

```
my $id = $discovery->diagnostics('address' => '10.0.0.1');
```

diagnostics_status( )

Use this method to retrieve the discovery diagnostic status.

Parameter

id - Required. The discovery diagnostic status ID. start - Optional. The start position of the text.

Returns

The method returns an Infoblox::Grid::Discovery::DiagnosticsStatus object when the operation succeeds, and returns false when the operation fails.

Example

```
my $object = $discovery->diagnostics_status('id' => $id);
```

discovery_data_conversion( )

Use this method to convert discovery data to specified objects.

Parameter

type - Required. The object type of the converted object. Valid value is one of the following object types: A_RECORD,

AAAA_RECORD, PTR_RECORD, HOST_RECORD, FIXED_ADDRESS, IPV6_FIXED_ADDRESS. addresses - Required. An array reference that contains Infoblox::IPAM::Address objects retrieved from the appliance. attributes - Optional. An Infoblox::Grid::Discovery::ConversionAttributes object. extensible_attributes - Optional. The list of extensible attributes associated with the created objects. Valid value is a hash reference containing the names of extensible attributes and their associated values (Infoblox::Grid::Extattr objects).

Returns

The method returns an array reference that contains Infoblox::Grid::Discovery::DiscoveryDataConversionResult objects if the operation succeeds.

Example

```
my $address = $discovery->get('object' => 'Infoblox::IPAM::Address', 'ip_address' => '10.0.0.31');
my $zone = $discovery->get('object' => 'Infoblox::DNS::Zone', 'name' => 'papi.com');
my $attrs = Infoblox::Grid::Discovery::ConversionAttributes->new(
    'configure_for_dns' => 'true',
    'configure_for_dhcp' => 'true',
    'comment' => 'convert',
    'disabled' => 'false',
    'zone' => $zone);
my $results = $discovery->discovery_data_conversion(
    'type' => 'HOST_RECORD',
    'attributes' => $attrs,
    'addresses' => [$address],
    'extensible_attributes' => {'Site' => Infoblox::Grid::Extattr->new('value' => 'test.com')});
```

get_device_support_info( )

Use this method to retrieve such information about device as device support statuses and device support info.

Parameter

device - Required. An Infoblox::Grid::Discovery::Device object.

Returns

The method returns an Infoblox::Grid::Discovery::DeviceSupportInfoResponse object.

Example

```
my $res = $discovery->get_device_support_info('device' => $device);
```

get_job_devices( )

Use this method to get the list of devices on wich the network insight job is processing.

This function should be called only if the task is related to network insight port control job.

Parameter

task - Required. An Infoblox::Grid::ScheduledTask object or a task ID.

Returns

The method returns an array reference that contains Infoblox::Grid::Discovery::Device object if operation succeeds, and returns "false" when the operation fails.

Example

```
# Get value by task_id
my $devices = $discovery->get_job_devices('task' => $task_id);

# Get value by scheduled_task object
my $devices = $discovery->get_job_devices('task' => $scheduled_task);
```

get_job_process_details( )

Use this method to get a network insight job processing logs related to a given task.

This function should be called only if the task is related to network insight port control job.

Parameter

device - Required. An Infoblox::Grid::Discovery::Device object. task - Required. An Infoblox::Grid::ScheduledTask object or a task ID. start_line - Optional. Valid value is unsigned integer representing starting line of a log stream. Default value is 0.

Returns

The method returns a Infoblox::Grid::Discovery::JobProcessDetails object if operation succeeds, and returns "false" if operation fails.

Example

```
 # Get job prcess details
 my $job_process_details = $discovery->get_job_process_details('device' => $device, 'task' =>
$scheduled_task, 'start_line' => 10);
```

provision_network_dhcp_relay( )

Use this method to provision only DHCP relay information for given devices.

Note that this function is always executed as part of scheduled task. Also the function goes through approval process if it is applicable.

Parameter

interfaces - Required. An array reference that contains Infoblox::Grid::Discovery::DeviceInterface objects. network - Required. The IP address and netmask of a network in CIDR format. scheduled_at - Required. Scheduled time in ISO 8601 extended format for date and time (e.g. 2008-10-21T10:56:00Z or 2008-10-21T10:56:00.001Z) or 'now' which will set scheduled date and time to the current date and time. enable_dhcp_relay - Optional. Enables DHCP relay. Valid value is 'true' and 'false'. Default value is 'false'. network_view - Optional. The network view name.

Returns

The method returns 'true' when a scheduled task has been created successfully, and returns 'false' when the operation fails.

Example

```
 my $res = $discovery->provision_network_dhcp_relay('scheduled_at' => 'now', 'interfaces' =>
\@interfaces, 'network' => $network);
```

provision_network_port( )

Use this method to provision the network with interface and assign default router ip address on a device.

Note that this function is always executed as part of scheduled task. Also the function goes through approval process if it is applicable.

Parameter

device - Optional. An Infoblox::Grid::Discovery::Device object. At least one of device or interface is required. interface - Optional. An Infoblox::Grid::Discovery::DeviceInterface object. At least one of device or interface is required. network - Required. The IP address and netmask of a network in CIDR format to operate on. router_ip - Required. Router IP address to operate on. scheduled_at - Required. Scheduled time in ISO 8601 extended format for date and time (e.g. 2008-10-21T10:56:00Z or 2008-10-21T10:56:00.001Z) or 'now' which will set scheduled date and time to the current date and time. enable_dhcp_relay - Optional. Enables DHCP relay. Valid value is 'true' and 'false'. Default value is 'false'. network_view - Optional. An Infoblox::DHCP::View object. vlan_info - Optional. An Infoblox::Grid::Discovery::VLANInfo object.

Returns

The method returns 'true' when a scheduled task has been created successfully, and returns 'false' when the operation fails.

Example

```
 my $res = $discovery->provision_network_port('scheduled_at' => '2008-10-21T10:56:00Z', 'device' =>
$device, 'network' => '10.0.0.0/8', router_ip => '10.0.0.1');
```

start_discovery( )

Use this method to start discovery on specified objects.

Parameter

objects - Required. An array reference that contains Infoblox::IPAM::Address, Infoblox::DHCP::Network, Infoblox::DHCP::IPv6Network, Infoblox::DHCP::NetworkContainer and Infoblox::DHCP::IPv6NetworkContainer objects that have been retrieved from the appliance.

Returns

The method returns true when the modification succeeds, and returns false when the operation fails.

Example

```
 my @networks = $discovery->get('object' => 'Infoblox::DHCP::Network');
 my $res = $discovery->start_discovery('objects' => \@networks);
```

Infoblox::Grid::Discovery - Grid discovery functions.

SEE ALSO

Infoblox::Grid::Discovery::DiagnosticsStatus, Infoblox::IPAM::Address, Infoblox::DHCP::Network, Infoblox::DHCP::IPv6Network, Infoblox::DHCP::NetworkContainer, Infoblox::DHCP::IPv6NetworkContainer, Infoblox::Grid::Discovery::NetworkDeprovisionInfo, Infoblox::Grid::Discovery::Port::Control, Infoblox::Grid::Discovery::Device, Infoblox::Grid::Discovery::JobProcessDetails, Infoblox::Grid::Discovery::DeviceInterface, Infoblox::Grid::Discovery::VLANInfo, Infoblox::Grid::Discovery::DiscoveryDataConversionResult, Infoblox::Grid::Discovery::ConversionAttributes, Infoblox::Grid::Discovery::DeviceSupportInfoResponse.

AUTHOR

Infoblox Inc. http://www.infoblox.com/

COPYRIGHT

Copyright (c) 2017 Infoblox Inc.

Infoblox::Grid::Discovery::DeviceNeighbor - the neighbor associated with the device.

- NAME
- DESCRIPTION
- SESSION METHODS
- METHODS
  - address( )
  - address_object( )
  - device( )
  - interface( )
  - mac( )
  - name( )
  - vlan_infos( )
- AUTHOR
- SEE ALSO
- COPYRIGHT

NAME

Infoblox::Grid::Discovery::DeviceNeighbor - the neighbor associated with the device.

DESCRIPTION

The object provides information about the neighbor associated with the device.

SESSION METHODS

The object does not support any session methods.

METHODS

This section describes all the methods that you can use to set or retrieve the attribute values of the object.

address( )

Use this method to retrieve the neighbor IP address. This is a read-only attribute.

Parameter

None

Returns

The method returns the attribute value.

Example

```
# Get attribute value
my $value = $object->address();
```

address_object( )

Use this method to retrieve the IP Address object. This is a read-only attribute.

Parameter

None

Returns

The method returns the attribute value that can be an Infoblox::IPAM::Address object.

Example

```
# Get attribute value
my $value = $object->address_object();
```

device( )

Use this method to retrieve the device object. This is a read-only attribute.

Parameter

None

Returns

The method returns the attribute value that can be an Infoblox::Grid::Discovery::Device object.

Example

```
# Get attribute value
my $value = $object->device();
```

interface( )

Use this method to retrieve the interface object. This is a read-only attribute.

Parameter

None

Returns

The method returns the attribute value that can be an Infoblox::Grid::Discovery::DeviceInterface object.

Example

```
# Get attribute value
my $value = $object->interface();
```

mac( )

Use this method to retrieve the MAC address of the neighbor. This is a read-only attribute.

Parameter

None

Returns

The method returns the attribute value.

Example

```
# Get attribute value
my $value = $object->mac();
```

name( )

Use this method to retrieve the neighbor name. This is a read-only attribute.

Parameter

None

Returns

The method returns the attribute value.

Example

```
# Get attribute value
my $value = $object->name();
```

vlan_infos( )

Use this method to retrieve a list of VLAN Information associated with this neighbor. This is a read-only attribute.

Parameter

None

Returns

The method returns the attribute value that can be an array reference that contains Infoblox::Grid::Discovery::VLANInfo objects.

Example

```
# Get attribute value
my $value = $object->vlan_infos();
```

AUTHOR

Infoblox Inc. http://www.infoblox.com/

SEE ALSO

Infoblox::IPAM::Address, Infoblox::Grid::Discovery::Device, Infoblox::Grid::Discovery::DeviceInterface

---

COPYRIGHT

Copyright (c) 2017 Infoblox Inc.

Infoblox::Grid::Discovery::Device::PortStatistics - the device port statistics object.

- NAME
- DESCRIPTION
- SESSION METHODS
- METHODS
  - admin_down_oper_down_count( )
  - admin_up_oper_down_count( )
  - admin_up_oper_up_count( )
  - interfaces_count( )
- AUTHOR
- SEE ALSO
- COPYRIGHT

NAME

Infoblox::Grid::Discovery::Device::PortStatistics - the device port statistics object.

DESCRIPTION

The object provides information about device port statistics.

SESSION METHODS

The object does not support any session methods.

METHODS

admin_down_oper_down_count( )

Use this method to retrieve the total number of interfaces which have administrative state "DOWN" and operating state "DOWN". This is a read-only attribute.

Parameter

None

Returns

The method returns the attribute value.

Example

```
# Get attribute value
my $value = $stats->admin_down_oper_down_count();
```

admin_up_oper_down_count( )

Use this method to retrieve the total number of interfaces which have administrative state "UP" and operating state "DOWN". This is a read-only attribute.

Parameter

None

Returns

The method returns the attribute value.

Example

```
# Get attribute value
my $value = $stats->admin_up_oper_down_count();
```

admin_up_oper_up_count( )

Use this method to retrieve the total number of interfaces which have administrative state "UP" and operating state "UP". This is a read-only attribute.

Parameter

None

Returns

The method returns the attribute value.

Example

```
# Get attribute value
my $value = $stats->admin_up_oper_up_count();
```

interfaces_count( )

Use this method to retrieve the total number of interfaces available on device. This is a read-only attribute.

Parameter

None

Returns

The method returns the attribute value.

Example

```
# Get attribute value
my $value = $stats->interfaces_count();
```

AUTHOR

Infoblox Inc. http://www.infoblox.com/

SEE ALSO

Infoblox::Grid::Discovery::Device

COPYRIGHT

Copyright (c) 2017 Infoblox Inc.

Infoblox::Grid::Discovery::DeviceSupportInfo - the device support info.

- NAME
- DESCRIPTION
- SESSION METHODS
- MODULE METHODS
  - Infoblox::Grid::Discovery::DeviceSupportInfoResponse->device_support_info( )
- METHODS
  - available()
  - function()
  - supported()
  - value()
- AUTHOR
- SEE ALSO
- COPYRIGHT

NAME

Infoblox::Grid::Discovery::DeviceSupportInfo - the device support info.

DESCRIPTION

The object provides information about the device support info. This is a read-only object.

SESSION METHODS

The object does not support any session methods.

MODULE METHODS

This section describes all the functions that you can apply to a Device Support Info object.

Infoblox::Grid::Discovery::DeviceSupportInfoResponse->device_support_info( )

Use this method to retrieve the Device support infos. See Infoblox::Grid::Discovery::DeviceSupportInfoResponse->device_support_info() for parameters and return values.

Example

```
#Get value
my $device_support_info = $device_support_info_response->device_support_info();
```

METHODS

This section describes all the methods that you can use to retrieve the attribute values of a device support info.

available()

Use this method to determine whether the function is available for the device or not. This is a read-only attribute.

Parameter

None

Returns

The method returns the attribute value.

Example

```
#Get value
my $available = $device_support_info->available();
```

function()

Use this method to retrieve the function of the device. This is a read-only attribute.

Parameter

None

Returns

The method returns the attribute value.

Example

```
#Get value
my $function = $device_support_info->function();
```

supported()

Use this method to determine whether the function is supported by the device or not. This is a read-only attribute.

Parameter

None

Returns

The method returns the attribute value.

Example

```
#Get value
my $supported = $device_support_info->supported();
```

value()

Use this method to retrieve the discovery member indicator knowing that a given device supports the given type of data collection. This is a read-only attribute.

Parameter

None

Returns

The method returns the attribute value.

Example

```
#Get value
my $value = $device_support_info->value();
```

AUTHOR

Infoblox Inc. http://www.infoblox.com/

SEE ALSO

Infoblox::Grid::Discovery::DeviceSupportInfoResponse, Infoblox::Grid::Discovery::DeviceDataCollectionStatus.

COPYRIGHT

Copyright (c) 2017 Infoblox Inc.

Infoblox::Grid::Discovery::Port::Config::AdminStatus - admin status of port configuration.

- NAME
- DESCRIPTION
- SESSION METHODS
- METHODS
  - status( )
  - task_details( )
- AUTHOR
- SEE ALSO
- COPYRIGHT

NAME

Infoblox::Grid::Discovery::Port::Config::AdminStatus - admin status of port configuration.

DESCRIPTION

The object provides information about the admin status of port configuration.

SESSION METHODS

The object does not support any session methods.

METHODS

This section describes all the methods that you can use to retrieve the attribute values of the object.

status( )

Use this method to retrieve the configured admin status. This attribute is read-only.

Parameter

None

Returns

The method returns the attribute value that can be 'UP' or 'DOWN'.

Example

```
# Get attribute value
my $value = $object->status();
```

task_details( )

Use this method to retrieve task details of the port control. This attribute is read-only.

Parameter

None

Returns

The method returns the attribute value that can be an Infoblox::Grid::Discovery::Port::Control::TaskDetails object.

Example

```
# Get attribute value
my $value = $object->task_details();
```

AUTHOR

Infoblox Inc. http://www.infoblox.com/

SEE ALSO

Infoblox::Grid::Discovery::Port::Control::TaskDetails, Infoblox::Grid::Discovery::DeviceInterface

Infoblox::Grid::Discovery::Port::Config::AdminStatus - admin status of port configuration.

COPYRIGHT

Copyright (c) 2017 Infoblox Inc.

Infoblox::Grid::Discovery::Port::Config::Description - port configuration description.

- NAME
- DESCRIPTION
- SESSION METHODS
- METHODS
  - description( )
  - task_details( )
- AUTHOR
- SEE ALSO
- COPYRIGHT

NAME

Infoblox::Grid::Discovery::Port::Config::Description - port configuration description.

DESCRIPTION

The object provides information about the port configuration description.

SESSION METHODS

The object does not support any session methods.

METHODS

This section describes all the methods that you can use to retrieve the attribute values of the object.

description( )

Use this method to retrieve the configured description. This attribute is read-only.

Parameter

None

Returns

The method returns the attribute value.

Example

```
# Get attribute value
my $value = $object->description();
```

task_details( )

Use this method to retrieve task details of the port control. This attribute is read-only.

Parameter

None

Returns

The method returns the attribute value that can be an Infoblox::Grid::Discovery::Port::Control::TaskDetails object.

Example

```
# Get attribute value
my $value = $object->task_details();
```

AUTHOR

Infoblox Inc. http://www.infoblox.com/

SEE ALSO

Infoblox::Grid::Discovery::Port::Control::TaskDetails, Infoblox::Grid::Discovery::DeviceInterface

Infoblox::Grid::Discovery::Port::Config::Description - port configuration description.

COPYRIGHT

Copyright (c) 2017 Infoblox Inc.

Infoblox::Grid::Discovery::IFAddrInfo - IFAddr information associated with the discovery interface.

- NAME
- DESCRIPTION
- CONSTRUCTOR
- SESSION METHODS
- METHODS
  - address( )
  - address_object( )
  - network( )
  - network_str( )
- AUTHOR
- SEE ALSO
- COPYRIGHT

NAME

Infoblox::Grid::Discovery::IFAddrInfo - IFAddr information associated with the discovery interface.

DESCRIPTION

The object provides IFAddr information associated with the discovery interface.

CONSTRUCTOR

```
my $object = Infoblox::Grid::Discovery::IFAddrInfo->new(
    'address'        => $ipaddr,          # Required
    'address_object' => $addr_obj,        # Optional
    'network'        => $network,         # Optional
    'network_str'    => $address/cidr,    # Optional
);
```

SESSION METHODS

The object does not support any session methods.

METHODS

This section describes all the methods that you can use to set or retrieve the attribute values of the object.

address( )

Use this method to set or retrieve the IP address of the discovery interface.

Parameter

The IP address in string format.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
# Get attribute value
my $value = $object->address();
# Modify attribute value
$object->address('10.0.0.1');
```

address_object( )

Use this method to set or retrieve the IP address object of the discovery interface.

Parameter

Valid value is an Infoblox::IPAM::Address object.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
# Get attribute value
my $value = $object->address_object();
# Modify attribute value
$object->address_object($addr_obj);
```

network( )

Use this method to set or retrieve the network object of the discovery interface.

Parameter

Valid value is an Infoblox::DHCP::Network or Infoblox::DHCP::IPv6Network object.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
# Get attribute value
my $value = $object->network();
# Modify attribute value
$object->network($network);
```

network_str( )

Use this method to set or retrieve the network address.

Parameter

The network address in address/CIDR format.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
# Get attribute value
my $value = $object->network_str();
# Modify attribute value
$object->network_str('10.0.0.0/8');
```

AUTHOR

Infoblox Inc. http://www.infoblox.com/

SEE ALSO

Infoblox::DHCP::Network, Infoblox::DHCP::IPv6Network, Infoblox::IPAM::Address

---

COPYRIGHT

Copyright (c) 2017 Infoblox Inc.

Infoblox::Grid::Discovery::Port::Config::VlanInfo - VLAN information of port configuration.

- NAME
- DESCRIPTION
- SESSION METHODS
- METHODS
  - data_vlan_info( )
  - task_details( )
  - voice_vlan_info( )
- AUTHOR
- SEE ALSO
- COPYRIGHT

NAME

Infoblox::Grid::Discovery::Port::Config::VlanInfo - VLAN information of port configuration.

DESCRIPTION

The object provides information about the port configuration VLAN.

SESSION METHODS

The object does not support any session methods.

METHODS

This section describes all the methods that you can use to retrieve the attribute values of the object.

data_vlan_info( )

Use this method to retrieve information about data VLAN. This attribute is read-only.

Parameter

None

Returns

The method returns the attribute value that can be an Infoblox::Grid::Discovery::VLANInfo object.

Example

```
# Get attribute value
my $value = $object->data_vlan_info();
```

task_details( )

Use this method to retrieve task details of the port control. This attribute is read-only.

Parameter

None

Returns

The method returns the attribute value that can be an Infoblox::Grid::Discovery::Port::Control::TaskDetails object.

Example

```
# Get attribute value
my $value = $object->task_details();
```

voice_vlan_info( )

Use this method to retrieve information about voice VLAN. This attribute is read-only.

Parameter

None

Returns

The method returns the attribute value that can be an Infoblox::Grid::Discovery::VLANInfo object.

Example

```
# Get attribute value
my $value = $object->voice_vlan_info();
```

AUTHOR

Infoblox Inc. http://www.infoblox.com/

SEE ALSO

Infoblox::Grid::Discovery::Port::Control::TaskDetails, Infoblox::Grid::Discovery::DeviceInterface, Infoblox::Grid::Discovery::VLANInfo

---

COPYRIGHT

Copyright (c) 2017 Infoblox Inc.

Infoblox::Grid::Discovery::DeviceDataCollectionStatus - the device data collection status.

- NAME
- DESCRIPTION
- SESSION METHODS
- MODULE METHODS
  - Infoblox::Grid::Discovery::DeviceSupportInfoResponse->device_data_collection_status( )
- METHODS
  - data_source()
  - end_time()
- AUTHOR
- SEE ALSO
- COPYRIGHT

NAME

Infoblox::Grid::Discovery::DeviceDataCollectionStatus - the device data collection status.

DESCRIPTION

The object provides information about the device data collection status. This is a read-only object.

SESSION METHODS

The object does not support any session methods.

MODULE METHODS

This section describes all the functions that you can apply to a Device Data Collection Status object.

Infoblox::Grid::Discovery::DeviceSupportInfoResponse->device_data_collection_status( )

Use this method to retrieve the Device data collection statuses. See Infoblox::Grid::Discovery::DeviceSupportInfoResponse->device_data_collection_status() for parameters and return values.

Example

```
#Get value
my $device_data_collection_status = $device_support_info_response->device_data_collection_status();
```

METHODS

This section describes all the methods that you can use to retrieve the attribute values of a device data collection status.

data_source()

Use this method to retrieve the source from which device support information is collected. This is a read-only attribute.

Parameter

None

Returns

The method returns the attribute value.

Example

```
#Get value
my $data_source = $device_data_collection_status->data_source();
```

end_time()

Use this method to retrieve the time when the most recent collection from the data sources was completed. This is a read-only attribute.

Parameter

None

Returns

The method returns the attribute value. The number of seconds that have elapsed since January 1st, 1970 UTC.

Example

```
#Get value
my $end_time = $device_data_collection_status->end_time();
```

AUTHOR

Infoblox Inc. http://www.infoblox.com/

SEE ALSO

Infoblox::Grid::Discovery::DeviceSupportInfoResponse, Infoblox::Grid::Discovery::DeviceSupportInfo.

COPYRIGHT

Copyright (c) 2017 Infoblox Inc.

Infoblox::Grid::Discovery::Port::Control::TaskDetails - port control task details.

- NAME
- DESCRIPTION
- SESSION METHODS
- METHODS
  - id( )
  - is_synchronized( )
  - status( )
- AUTHOR
- SEE ALSO
- COPYRIGHT

NAME

Infoblox::Grid::Discovery::Port::Control::TaskDetails - port control task details.

DESCRIPTION

The object provides information about port control task details.

SESSION METHODS

The object does not support any session methods.

METHODS

This section describes all the methods that you can use to retrieve the attribute values of the object.

id( )

Use this method to retrieve the task ID. This attribute is read-only.

Parameter

None

Returns

The method returns the attribute value.

Example

```
# Get attribute value
my $value = $object->id();
```

is_synchronized( )

Use this method to retrieve the flag indicates if the task completed and matched with current discovered value. This attribute is read-only.

Parameter

None

Returns

The method returns the attribute value.

Example

```
# Get attribute value
my $value = $object->is_synchronized();
```

status( )

Use this method to retrieve the task status. This attribute is read-only.

Parameter

None

Returns

The method returns the attribute value.

Example

```
# Get attribute value
my $value = $object->status();
```

AUTHOR

Infoblox Inc. http://www.infoblox.com/

SEE ALSO

Infoblox::Grid::Discovery::Port::Config::AdminStatus, Infoblox::Grid::Discovery::Port::Config::Description, Infoblox::Grid::Discovery::Port::Config::VlanInfo

---

COPYRIGHT

Copyright (c) 2017 Infoblox Inc.

Infoblox::Grid::Discovery::AdvancedPollSettings - advanced polling settings.

- NAME
- DESCRIPTION
- CONSTRUCTOR
- SESSION METHODS
  - arp_aggregate_limit( )
  - arp_cache_refresh_interval( )
  - dhcp_router_as_seed( )
  - disable_discovery_outside_ipam( )
  - enable_purge_expired_endhost_data( )
  - ping_retries( )
  - ping_sweep_interval( )
  - ping_timeout( )
  - polling_authenticate_snmpv1_only( )
  - purge_expired_device_data( )
  - purge_expired_endhost_data( )
  - route_limit( )
  - tcp_scan_technique( )
- AUTHOR
- COPYRIGHT

NAME

Infoblox::Grid::Discovery::AdvancedPollSettings - advanced polling settings.

DESCRIPTION

The object provides information about advanced polling settings.

CONSTRUCTOR

```
my $object = Infoblox::Grid::Discovery::AdvancedPollSettings->new(
    'arp_aggregate_limit'               => $uint,                  # Optional
    'arp_cache_refresh_interval'        => $uint,                  # Optional
    'dhcp_router_as_seed'               => 'true' | 'false',       # Optional / Default is 'false'
    'disable_discovery_outside_ipam'    => 'true' | 'false',       # Optional / Default is 'false'
    'enable_purge_expired_endhost_data' => 'true' | 'false',       # Optional / Default is 'false'
    'ping_retries'                      => $uint,                  # Optional
    'ping_sweep_interval'               => $uint,                  # Optional
    'ping_timeout'                      => $uint,                  # Optional
    'polling_authenticate_snmpv1_only'  => 'true' | 'false',       # Optional / Default is 'false'
    'purge_expired_device_data'         => $uint,                  # Optional
    'purge_expired_endhost_data'        => $uint,                  # Optional
    'route_limit'                       => $uint,                  # Optional
    'tcp_scan_technique'                => 'CONNECT' | 'SYN',      # Optional
);
```

SESSION METHODS

The object does not support any session methods.

arp_aggregate_limit( )

Use this method to set or retrieve the ARP aggregate limit.

Parameter

Valid value is an unsigned integer.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
# Get attribute value
my $value = $object->arp_aggregate_limit();
# Modify attribute value
```

```
$object->arp_aggregate_limit(10);
```

arp_cache_refresh_interval( )

Use this method to set or retrieve the refresh interval for ARP cache in second.

Parameter

Valid value is an unsigned integer.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
# Get attribute value
my $value = $object->arp_cache_refresh_interval();
# Modify attribute value
$object->arp_cache_refresh_interval(5);
```

dhcp_router_as_seed( )

Use this method to enable or disable the flag that controls using DHCP routers as seed routers for discovery.

Parameter

Specify "true" to enable the flag or "false" to disable it. Default value is "false".

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
# Get attribute value
my $value = $object->dhcp_router_as_seed();
# Modify attribute value
$object->dhcp_router_as_seed('true');
```

disable_discovery_outside_ipam( )

Use this method to enable or disable the discovery of networks that are not in IPAM.

Parameter

Specify "true" to disable the discovery of networks that are not in IPAM or "false" to enable it. Default value is "false".

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
# Get attribute value
my $value = $object->disable_discovery_outside_ipam();
# Modify attribute value
$object->disable_discovery_outside_ipam('true');
```

enable_purge_expired_endhost_data( )

Use this method to enable or disable the purge of expired end host data.

Parameter

Specify "true" to enable the purge of expired end host data or "false" to disable it. Default value is "false".

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
# Get attribute value
my $value = $object->enable_purge_expired_endhost_data();
# Modify attribute value
$object->enable_purge_expired_endhost_data('true');
```

ping_retries( )

Use this method to set or retrieve ping retries.

Parameter

Valid value is an unsigned integer.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
# Get attribute value
my $value = $object->ping_retries();
# Modify attribute value
$object->ping_retries(3);
```

ping_sweep_interval( )

Use this method to set or retrieve the hourly wait interval between ping sweeps for individual discovery ranges.

Parameter

Valid value is an unsigned integer.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
# Get attribute value
my $value = $object->ping_sweep_interval();
# Modify attribute value
$object->ping_sweep_interval(1);
```

ping_timeout( )

Use this method to set or retrieve the ping timeout in seconds.

Parameter

Valid value is an unsigned integer.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
# Get attribute value
my $value = $object->ping_timeout();
# Modify attribute value
$object->ping_timeout(10);
```

polling_authenticate_snmpv1_only( )

Use this method to enable or disable polling and authenticating using SNMPv1 only.

Parameter

Specify "true" to enable polling and authenticating using SNMPv1 only or "false" to disable it. Default value is "false".

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
# Get attribute value
my $value = $object->polling_authenticate_snmpv1_only();
# Modify attribute value
$object->polling_authenticate_snmpv1_only('true');
```

purge_expired_device_data( )

Use this method to set or retrieve the number of days a device remains in database after it is no longer found in network.

Parameter

Valid value is an unsigned integer.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
# Get attribute value
my $value = $object->purge_expired_device_data();
# Modify attribute value
$object->purge_expired_device_data(5);
```

purge_expired_endhost_data( )

Use this method to set or retrieve the number of days an end host remains in database after it is no longer found in network.

Parameter

Valid value is an unsigned integer.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
# Get attribute value
my $value = $object->purge_expired_endhost_data();
# Modify attribute value
$object->purge_expired_endhost_data(5);
```

route_limit( )

Use this method to set or retrieve the route limit.

Parameter

Valid value is an unsigned integer.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
# Get attribute value
my $value = $object->route_limit();
# Modify attribute value
$object->route_limit(10);
```

tcp_scan_technique( )

Use this method to set or retrieve the TCP scan method.

Parameter

Valid value is 'CONNECT' or 'SYN'.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
# Get attribute value
my $value = $object->tcp_scan_technique();
# Modify attribute value
$object->tcp_scan_technique('SYN');
```

AUTHOR

Infoblox Inc. http://www.infoblox.com/

COPYRIGHT

Copyright (c) 2017 Infoblox Inc.

Infoblox::Grid::Discovery::AutoConversionSetting - Auto conversion setting.

- NAME
- DESCRIPTION
- CONSTRUCTOR
- SESSION METHODS
- METHODS
  - comment( )
  - condition( )
  - format( )
  - network_view( )
  - type( )
- AUTHOR
- COPYRIGHT

NAME

Infoblox::Grid::Discovery::AutoConversionSetting - Auto conversion setting.

DESCRIPTION

The object contains settings for automatic conversion of discovered data to managed objects.

CONSTRUCTOR

```
my $object = Infoblox::Grid::Discovery::AutoConversionSetting->new(
    'comment'           => $string,     # Optional
    'condition'         => $string,     # Optional
    'format'            => $string,     # Required
    'network_view'      => $string,     # Required
    'type'              => 'A_AND_PTR_RECORD' | 'HOST_RECORD' | 'FIXED_ADDRESS', # Required
);
```

SESSION METHODS

The object does not support any session methods.

METHODS

This section describes all the methods that you can use to set or retrieve the attribute values of the object.

comment( )

Use this method to set or retrieve the comment for the automatic conversion setting.

Parameter

The comment in string format.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
# Get attribute value
my $value = $object->comment();
# Modify attribute value
$object->comment('comment');
```

condition( )

Use this method to set or retrieve the condition for automatic conversion settings.

Parameter

The string contains the condition for automatic conversion settings.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
# Get attribute value
my $value = $object->condition();
# Modify attribute value
$object->condition('${discovered_name} == "unknown"');
```

format( )

Use this method to set or retrieve the format of the template for conversion.

Parameter

Template string used to generate host names.

Note that using double quotes may lead to interpolation of variables. Instead, use single quotes or encapsulated '$' characters.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
# Get attribute value
my $value = $object->format();
# Modify attribute value
$object->format('${discovered_name}.test.com');
```

network_view( )

Use this method to set or retrieve the name of the network view which contains discovered data for convert.

Parameter

The name of the network view.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
# Get attribute value
my $value = $object->network_view();
# Modify attribute value
$object->network_view('default');
```

type( )

Use this method to set or retrieve the type of the converted object.

Parameter

The object type used to define the converted object. Valid value is one of the following object types: 'A_AND_PTR_RECORD', 'HOST_RECORD', 'FIXED_ADDRESS'.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
# Get attribute value
my $value = $object->type();
# Modify attribute value
$object->type('FIXED_ADDRESS');
```

AUTHOR

Infoblox Inc. http://www.infoblox.com/

COPYRIGHT

Copyright (c) 2017 Infoblox Inc.

Infoblox::Grid::Discovery::Port - the discovery port structure.

- NAME
- DESCRIPTION
- CONSTRUCTOR
- SESSION METHODS
- METHODS
  - comment( )
  - port( )
  - type( )
- AUTHOR
- COPYRIGHT

NAME

Infoblox::Grid::Discovery::Port - the discovery port structure.

DESCRIPTION

The object provides information about the discovery port.

CONSTRUCTOR

```
my $object = Infoblox::Grid::Discovery::Port->new(
    'comment' => $string,            # Optional
    'port'    => $uint,              # Required
    'type'    => 'TCP' | 'UDP',      # Required
);
```

SESSION METHODS

The object does not support any session methods.

METHODS

This section describes all the methods that you can use to set or retrieve the attribute values of the object.

comment( )

Use this method to set or retrieve the comment.

Parameter

The comment in string format.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
# Get attribute value
my $value = $object->comment();
# Modify attribute value
$object->comment('comment');
```

port( )

Use this method to set or retrieve the port number.

Parameter

The port number.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
# Get attribute value
my $value = $object->port();
# Modify attribute value
$object->port(56);
```

type( )

Use this method to set or retrieve the port type.

Parameter

Valid value is 'TCP' or 'UDP'.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
# Get attribute value
my $value = $object->type();
# Modify attribute value
$object->type('UDP');
```

AUTHOR

Infoblox Inc. http://www.infoblox.com/

COPYRIGHT

Copyright (c) 2017 Infoblox Inc.

Infoblox::Grid::Discovery::SNMP3Credential - SNMPv3 credential.

- NAME
- DESCRIPTION
- CONSTRUCTOR
- SESSION METHODS
- METHODS
  - authentication_password( )
  - authentication_protocol( )
  - comment( )
  - privacy_password( )
  - privacy_protocol( )
  - user( )
- AUTHOR
- COPYRIGHT

NAME

Infoblox::Grid::Discovery::SNMP3Credential - SNMPv3 credential.

DESCRIPTION

The object provides information about SNMPv3 credential.

CONSTRUCTOR

```
 my $object = Infoblox::Grid::Discovery::SNMP3Credential->new(
    'authentication_password' => $string,                          # Required if authentication_protocol is
MD5 or SHA
    'authentication_protocol' => 'MD5' | 'NONE' | 'SHA',           # Requred
    'comment'                 => $string,                          # Optional
    'privacy_password'        => $string,                          # Required if privacy_protocol is 'AES',
'DES' or '3DES'
    'privacy_protocol'        => 'AES' | 'DES' | '3DES' | 'NONE', # Required
    'user'                    => $string,                          # Required
 );
```

SESSION METHODS

The object does not support any session methods.

METHODS

This section describes all the methods that you can use to set or retrieve the attribute values of the object.

authentication_password( )

Use this method to set the authentication password if the authentication protocol is MD5 or SHA. This is a write-only attribute.

Parameter

The authentication password in string format.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

Example

```
 # Modify attribute value
 $object->authentication_password('infoblox');
```

authentication_protocol( )

Use this method to set or retrieve the authentication protocol.

Parameter

Valid value is 'MD5', 'NONE' or 'SHA'.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
# Get attribute value
my $value = $object->authentication_protocol();
# Modify attribute value
$object->authentication_protocol('MD5');
```

comment( )

Use this method to set or retrieve the comment on SNMPv3 user.

Parameter

The comment in string format.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
# Get attribute value
my $value = $object->comment();
# Modify attribute value
$object->comment('SNMPv3');
```

privacy_password( )

Use this method to set the privacy password if the privacy protocol is 'AES', 'DES' or '3DES'. This is a write-only attribute.

Parameter

The privacy password in string format.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

Example

```
# Modify attribute value
$object->privacy_password('infoblox');
```

privacy_protocol( )

Use this method to set or retrieve the privacy protocol.

Parameter

Valid value is 'AES', 'DES', '3DES' or 'NONE'.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
# Get attribute value
my $value = $object->privacy_protocol();
# Modify attribute value
$object->privacy_protocol('3DES');
```

user( )

Use this method to set or retrieve the SNMPv3 user name.

Parameter

Tthe SNMPv3 user name in string format.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
# Get attribute value
my $value = $object->user();
# Modify attribute value
$object->user('admin');
```

AUTHOR

Infoblox Inc. http://www.infoblox.com/

COPYRIGHT

Copyright (c) 2017 Infoblox Inc.

Infoblox::Grid::Discovery::SNMPCredential - SNMPv1 and SNMPv2 credential.

- NAME
- DESCRIPTION
- CONSTRUCTOR
- SESSION METHODS
- METHODS
  - comment( )
  - community_string( )
- AUTHOR
- COPYRIGHT

NAME

Infoblox::Grid::Discovery::SNMPCredential - SNMPv1 and SNMPv2 credential.

DESCRIPTION

The object provides information about SNMPv1 and SNMPv2 credential.

CONSTRUCTOR

```
my $object = Infoblox::Grid::Discovery::SNMPCredential->new(
    'comment'          => $string,    # Optional
    'community_string' => $string,    # Required
);
```

SESSION METHODS

The object does not support any session methods.

METHODS

This section describes all the methods that you can use to set or retrieve the attribute values of the object.

comment( )

Use this method to set or retrieve the comment on SNMPv1 and SNMPv2 user.

Parameter

The comment in string format.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
# Get attribute value
my $value = $object->comment();
# Modify attribute value
$object->comment('comment');
```

community_string( )

Use this method to set or retrieve the public community string.

Parameter

The public community string.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
# Get attribute value
my $value = $object->community_string();
# Modify attribute value
$object->community_string('string');
```

AUTHOR

Infoblox Inc. http://www.infoblox.com/

COPYRIGHT

Copyright (c) 2017 Infoblox Inc.

NAME

- NAME
- DESCRIPTION
- SESSION METHODS
- METHODS
  - end( )
  - id( )
  - status( )
  - text( )
- AUTHOR
- SEE ALSO
- COPYRIGHT

NAME

Infoblox::Grid::Discovery::TestCredential

DESCRIPTION

The object provides information about results of the test credential.

SESSION METHODS

The object does not support any session methods.

METHODS

This section describes all the methods that you can use to set or retrieve the attribute values of the object.

end( )

Use this method to retrieve the end position of the returned text. This attribute is read-only.

Parameter

None.

Returns

The method returns the attribute value.

Example

```
# Get attribute value
my $value = $object->end();
```

id( )

Use this method to retrieve the test credential ID. This attribute is read-only.

Parameter

None.

Returns

The method returns the attribute value.

Example

```
# Get attribute value
my $value = $object->id();
```

status( )

Use this method to retrieve the test credential status. This attribute is read-only.

Parameter

None.

Returns

The method returns the attribute value that can be 'COMPLETED', 'FAILED' or 'INPROGRESS'.

Example

```
# Get attribute value
my $value = $object->status();
```

text( )

Use this method to retrieve the portion of the text from start (inclusive) to end (exclusive). This attribute is read-only.

Parameter

None.

Returns

The method returns the attribute value.

Example

```
# Get attribute value
my $value = $object->text();
```

AUTHOR

Infoblox Inc. http://www.infoblox.com/

SEE ALSO

Infoblox::Grid::Discovery::Properties

COPYRIGHT

Copyright (c) 2017 Infoblox Inc.

Infoblox::Grid::Discovery::VRFMappingRule - VRF mapping rule.

- NAME
- DESCRIPTION
- CONSTRUCTOR
- SESSION METHODS
- METHODS
  - network_view( )
  - criteria( )
  - comment( )
- AUTHOR
- COPYRIGHT

NAME

Infoblox::Grid::Discovery::VRFMappingRule - VRF mapping rule.

DESCRIPTION

The object provides mapping of regular expressions that represent the VRF name and network view.

CONSTRUCTOR

```
my $object = Infoblox::Grid::Discovery::VRFMappingRule->new(
    'network_view'      => $string,     # Required
    'criteria'          => $string,     # Required
    'comment'           => $string,     # Optional
);
```

SESSION METHODS

The object does not support any session methods.

METHODS

This section describes all the methods that you can use to set or retrieve the attribute values of the object.

network_view( )

Use this method to set or retrieve the name of the network view associated with the VRF mapping rule.

Parameter

The name of the network view associated with the VRF mapping rule.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
# Get attribute value
my $value = $object->network_view();
# Modify attribute value
$object->network_view('default');
```

criteria( )

Use this method to set or retrieve the criteria of the VRF mapping rule.

Parameter

Extended POSIX regular expression matching the VRF name. The regular expression string should contain the leading caret anchor ^ and the trailing dollar anchor $ symbols.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
# Get attribute value
my $value = $object->criteria();
# Modify attribute value
$object->criteria('.*+Red\d');
```

comment( )

Use this method to set or retrieve the comment for the VRF mapping rule.

Parameter

The comment in string format.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
# Get attribute value
my $value = $object->comment();
# Modify attribute value
$object->comment('comment');
```

AUTHOR

Infoblox Inc. http://www.infoblox.com/

COPYRIGHT

Copyright (c) 2017 Infoblox Inc.

Infoblox::Grid::Discovery::StatusInfo - the discovery status information.

- NAME
- DESCRIPTION
- CONSTRUCTOR
- SESSION METHODS
- METHODS
  - message( )
  - status( )
  - timestamp( )
- AUTHOR
- COPYRIGHT

NAME

Infoblox::Grid::Discovery::StatusInfo - the discovery status information.

DESCRIPTION

The object provides information about the discovery status.

CONSTRUCTOR

```
my $object = Infoblox::Grid::Discovery::StatusInfo->new(
    'message'   => $string,                                  # Required
    'status'    => 'OK' | 'ERROR' | 'NON_REACHABLE',         # Required
    'timestamp' => $epoch,                                   # Required
);
```

SESSION METHODS

The object does not support any session methods.

METHODS

This section describes all the methods that you can use to set or retrieve the attribute values of the object.

message( )

Use this method to set or retrieve the detailed message.

Parameter

The detailed message in string format.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
# Get attribute value
my $value = $object->message();
# Modify attribute value
$object->message('text');
```

status( )

Use this method to set or retrieve the status of the device.

Parameter

Valid value is 'OK', 'ERROR' or 'NON_REACHABLE'.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
# Get attribute value
my $value = $object->status();
# Modify attribute value
$object->status('OK');
```

timestamp( )

Use this method to set or retrieve the timestamp when status was generated.

Parameter

The date and time in the ISO 8601 extended format (e.g. 2008-10-21T10:56:33Z or 2008-10-21T10:56:33.001Z).

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
# Get attribute value
my $value = $object->timestamp();
# Modify attribute value
$object->timestamp('2008-10-21T10:56:33Z');
```

AUTHOR

Infoblox Inc. http://www.infoblox.com/

COPYRIGHT

Copyright (c) 2017 Infoblox Inc.

Infoblox::Grid::DNS::AttackMitigation - a DNS Attack Mitigation object.

- NAME
- DESCRIPTION
- CONSTRUCTOR
- SESSION METHODS
- MODULE METHODS
  - Infoblox::Grid::DNS->attack_mitigation( )
  - Infoblox::Grid::Member::DNS->attack_mitigation( )
- METHODS
  - detect_chr( )
  - detect_chr_grace( )
  - detect_nxdomain_responses( )
  - detect_udp_drop( )
  - interval( )
  - mitigate_nxdomain_lru( )
- AUTHOR
- SEE ALSO
- COPYRIGHT

NAME

Infoblox::Grid::DNS::AttackMitigation - a DNS Attack Mitigation object.

DESCRIPTION

The DNS Attack Mitigation object provides information about mitigation configuration for DNS attacks.

CONSTRUCTOR

```
my $attack_mitigation = Infoblox::Grid::DNS::AttackMitigation->new(
    detect_chr                => $attack_detect,    # Optional
    detect_chr_grace          => $uint,             # Optional
    detect_nxdomain_responses => $attack_detect,    # Optional
    detect_udp_drop           => $attack_detect,    # Optional
    interval                  => $uint,             # Optional
    mitigate_nxdomain_lru     => 'true' | 'false', # Optional
);
```

SESSION METHODS

The object does not support any session methods.

MODULE METHODS

The following functions can be applied to a DNS Attack Mitigation object.

Infoblox::Grid::DNS->attack_mitigation( )

Use this method to set or retrieve mitigation settings for DNS attacks. See Infoblox::Grid::DNS->attack_mitigation() for parameters and return values.

Example

```
#Get attack_mitigation value
my $attack_mitigation = $grid_dns->attack_mitigation();
#Modify attack_mitigation
$grid_dns->attack_mitigation($attack_mitigation);
```

Infoblox::Grid::Member::DNS->attack_mitigation( )

Use this method to set or retrieve mitigation settings for DNS attacks on the member. See Infoblox::Grid::Member::DNS->attack_mitigation() for parameters and return values.

Example

```
#Get attack_mitigation value
my $attack_mitigation = $member_dns->attack_mitigation();
#Modify attack_mitigation
```

```
$member_dns->attack_mitigation($attack_mitigation);
```

METHODS

detect_chr( )

Use this method to set or retrieve configuration for the Cache Hit Ratio (CHR) changes detection.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

The valid value is an Infoblox::Grid::DNS::AttackDetect object.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get detect_chr value
my $detect_chr = $attack_mitigation->detect_chr();
#Modify detect_chr
$attack_mitigation->detect_chr($detect_chr);
```

detect_chr_grace( )

Use this method to set or retrieve the cache usage (in percentage) when Cache Hit Ratio (CHR) detection starts.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

The valid value is an unsigned integer between 0 and 100.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get detect_chr_grace value
my $detect_chr_grace = $attack_mitigation->detect_chr_grace();
#Modify detect_chr_grace
$attack_mitigation->detect_chr_grace(50);
```

detect_nxdomain_responses( )

Use this method to set or retrieve configuration for the recursive queries NXDOMAIN responses detection.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

The valid value is an Infoblox::Grid::DNS::AttackDetect object.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get detect_nxdomain_responses value
my $detect_nxdomain_responses = $attack_mitigation->detect_nxdomain_responses();
#Modify detect_nxdomain_responses
$attack_mitigation->detect_nxdomain_responses($detect_nxdomain_responses);
```

detect_udp_drop( )

Use this method to set or retrieve configuration for the UDP packet drop rate detection.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

The valid value is an Infoblox::Grid::DNS::AttackDetect object.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get detect_udp_drop value
my $detect_udp_drop = $attack_mitigation->detect_udp_drop();
#Modify detect_udp_drop
$attack_mitigation->detect_udp_drop($detect_udp_drop);
```

interval( )

Use this method to set or retrieve the minimum time interval (in seconds) between changes in attack status.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

The valid value is an unsigned integer.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get interval value
my $interval = $attack_mitigation->interval();
#Modify interval
$attack_mitigation->interval(50);
```

mitigate_nxdomain_lru( )

Use this method to enable or disable impact mitigation of NXDOMAIN responses in cache by tracking NXDOMAIN entries.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

Specify 'true' to enable impact mitigation of NXDOMAIN responses, or 'false' to disable it. The default value is 'false'.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get mitigate_nxdomain_lru value
my $mitigate_nxdomain_lru = $attack_mitigation->mitigate_nxdomain_lru();
#Modify mitigate_nxdomain_lru
$attack_mitigation->mitigate_nxdomain_lru('true');
```

AUTHOR

Infoblox Inc. http://www.infoblox.com/

SEE ALSO

Infoblox::Grid::DNS, Infoblox::Grid::DNS->attack_mitigation(), Infoblox::Grid::Member::DNS, Infoblox::Grid::Member::DNS->attack_mitigation(), Infoblox::Grid::DNS::AttackDetect,

---

COPYRIGHT

Infoblox::Grid::DNS::AttackMitigation - a DNS Attack Mitigation object.

Copyright (c) 2017 Infoblox Inc.

Infoblox::Grid::DNS::AutoBlackHole - a DNS Auto Blackhole object.

- NAME
- DESCRIPTION
- CONSTRUCTOR
- SESSION METHODS
- MODULE METHODS
  - Infoblox::Grid::DNS->auto_blackhole( )
  - Infoblox::Grid::Member::DNS->auto_blackhole( )
- METHODS
  - enable_fetches_per_server( )
  - enable_fetches_per_zone( )
  - enable_holddown( )
  - fetches_per_server( )
  - fetches_per_zone( )
  - fps_freq( )
  - holddown( )
  - holddown_threshold( )
  - holddown_timeout( )
- AUTHOR
- SEE ALSO
- COPYRIGHT

NAME

Infoblox::Grid::DNS::AutoBlackHole - a DNS Auto Blackhole object.

DESCRIPTION

The DNS Auto Blackhole object provides information about DNS auto blackhole configuration.

CONSTRUCTOR

```
my $auto_blackhole = Infoblox::Grid::DNS::AutoBlackHole->new(
        enable_fetches_per_server => 'true' | 'false', # Optional
        enable_fetches_per_zone   => 'true' | 'false', # Optional
        enable_holddown           => 'true' | 'false', # Optional
        fetches_per_server        => $uint,            # Optional
        fetches_per_zone          => $uint,            # Optional
        fps_freq                  => $uint,            # Optional
        holddown                  => $uint,            # Optional
        holddown_timeout          => $uint,            # Optional
        holddown_threshold        => $uint,            # Optional
);
```

SESSION METHODS

The object does not support any session methods.

MODULE METHODS

The following functions can be applied to a DNS Auto Blackhole object.

Infoblox::Grid::DNS->auto_blackhole( )

Use this method to set or retrieve the auto blackhole settings. See Infoblox::Grid::DNS->auto_blackhole() for parameters and return values.

Example

```
#Get auto_blackhole value
my $auto_blackhole = $grid_dns->auto_blackhole();
#Modify bind_check_auto_blackhole
$grid_dns->auto_blackhole($auto_blackhole);
```

Infoblox::Grid::Member::DNS->auto_blackhole( )

Use this method to set or retrieve the auto blackhole settings on the member. See Infoblox::Grid::Member::DNS->auto_blackhole() for

paparameters and return values.

Example

```
#Get auto_blackhole value
my $auto_blackhole = $member_dns->auto_blackhole();
#Modify auto_blackhole
$member_dns->auto_blackhole($auto_blackhole);
```

METHODS

enable_fetches_per_server( )

Use this method to enable or disable the maximum number of concurrent recursive queries sent to each upstream DNS server.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

Specify 'true' to enable the maximum number of concurrent recursive queries sent to each upstream DNS server or 'false" to disable it.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get enable_fetches_per_server value
my $enable_fetches_per_server = $auto_blackhole->enable_fetches_per_server();
#Modify enable_fetches_per_server
$auto_blackhole->enable_fetches_per_server('true');
```

enable_fetches_per_zone( )

Use this method to enable or disable the maximum number of concurrent recursive queries sent to each DNS zone.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

Specify 'true' to enable the maximum number of concurrent recursive queries sent to each DNS zone or 'false" to disable it.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get enable_fetches_per_zone value
my $enable_fetches_per_zone = $auto_blackhole->enable_fetches_per_zone();
#Modify enable_fetches_per_zone
$auto_blackhole->enable_fetches_per_zone('true');
```

enable_holddown( )

Use this method to enable or disable the holddown for non-responsive servers for a specified time interval.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

Specify 'true' to enable the holddown for non-responsive servers or 'false' to disable it.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get enable_holddown value
my $enable_holddown = $auto_blackhole->enable_holddown();
```

```
#Modify enable_holddown
$auto_blackhole->enable_holddown('true');
```

fetches_per_server( )

Use this method to set or retrieve the maximum number of concurrent recursive queries for each DNS server.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

The valid value is an unsigned integer.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get fetches_per_server value
my $fetches_per_server = $auto_blackhole->fetches_per_server();
#Modify fetches_per_server
$auto_blackhole->fetches_per_server(2000);
```

fetches_per_zone( )

Use this method to set or retrieve the maximum number of concurrent recursive queries for each DNS zone.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

The valid value is an unsigned integer.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get fetches_per_zone value
my $fetches_per_zone = $auto_blackhole->fetches_per_zone();
#Modify fetches_per_zone
$auto_blackhole->fetches_per_zone(2000);
```

fps_freq( )

Use this method to set or retrieve the number of recursive queries that determines the recalculation of the timeout ratio for each DNS server.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

The valid value is an unsigned integer.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get fps_freq value
my $fps_freq = $auto_blackhole->fps_freq();
#Modify fps_freq
$auto_blackhole->fps_freq(2000);
```

holddown( )

Use this method to set or retrieve the holddown duration for non-responsive servers.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

The valid value is an unsigned integer.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get holddown value
my $holddown = $auto_blackhole->holddown();
#Modify holddown
$auto_blackhole->holddown(100);
```

holddown_threshold( )

Use this method to set or retrieve the number of consecutive timeouts before holding down a non-responsive server.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

The valid value is an unsigned integer.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get holddown_threshold value
my $holddown_threshold = $auto_blackhole->holddown_threshold();
#Modify holddown_threshold
$auto_blackhole->holddown_threshold(100);
```

holddown_timeout( )

Use this method to set or retrieve the minimum time (in seconds) that needs to be passed before a timeout occurs. Note that only these timeouts are counted towards the number of consecutive timeouts.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

The valid value is an unsigned integer.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get holddown_timeout value
my $holddown_timeout = $auto_blackhole->holddown_timeout();
#Modify holddown_timeout
$auto_blackhole->holddown_timeout(100);
```

AUTHOR

Infoblox Inc. http://www.infoblox.com/

SEE ALSO

Infoblox::Grid::DNS, Infoblox::Grid::DNS->auto_blackhole(), Infoblox::Grid::Member::DNS, Infoblox::Grid::Member::DNS->auto_blackhole()

COPYRIGHT

Copyright (c) 2017 Infoblox Inc.

Infoblox::Grid::DNS::AutoBlackHole - a DNS Auto Blackhole object.

Infoblox::Grid::DNS::BulkHostNameTemplate - Manages the DNS bulk host name formats defined at the grid level.

- NAME
- DESCRIPTION
- CONSTRUCTOR
- MODULE METHODS
  - Infoblox::Grid::DNS->bulk_host_name_templates( )
- METHODS
  - template_name( )
  - template_format( )
- SAMPLE CODE
- AUTHOR
- SEE ALSO
- COPYRIGHT

NAME

Infoblox::Grid::DNS::BulkHostNameTemplate - Manages the DNS bulk host name formats defined at the grid level.

DESCRIPTION

Bulk host name template defines the format based on which the host names are generated from a bulk host..

CONSTRUCTOR

```
my $bulk_host_name_template = Infoblox::Grid::DNS::BulkHostNameTemplate->new(
    template_name   => $string,    #Required
    template_format => $string     #Required
);
```

MODULE METHODS

The following functions are available to apply to the bulk host name template object.

Infoblox::Grid::DNS->bulk_host_name_templates( )

Use this function to specify a bulk host name template at the grid level in the Infoblox appliance.

Note that we can't remove or modify default bulk host name templates we can only add and modify our custom templates to the current bulk host name templates array.

Example

```
#Create a BulkHostNameTemplate object.
my $custom_template = Infoblox::Grid::DNS::BulkHostNameTemplate->new(
   template_name   => "test",
   template_format => "-#1-#2-#3-#4",
);

#Retrieve bulk host name templates array
my $bhnt = $grid_dns->bull_host_name_templates();

#Configure bulk host name template on the Infoblox grid object
push @$bhnt, $custom_template;
my $response = $grid_dns->bulk_host_name_templates($bhnt);
```

METHODS

This section describes all the methods that can be used to configure and retrieve the attributes values of a bulk host name template object.

template_name( )

Use this method to set or retrieve the name of bulk host name template.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

String with the name of the bulk host name template.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get template_name
my $template_name = $bulk_host_name_template->template_name();
#Modify template_name
$template_name->template_name("new_name");
```

template_format( )

Use this method to set or retrieve the format of bulk host name template.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

The format of bulk host name template. It should follow certain rules (please use Administration Guide as reference).

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get template_format
my $template_format = $bulk_host_name_template->template_format();
#Modify template_format
$bulk_host_name_template>template_format("-#1-#2-#3-#4");
```

SAMPLE CODE

The following sample code demonstrates different operations that can be applied to a bulk host name template object such as create, modify, and remove. Also, this sample code includes error handling for the operations.

```
#PROGRAM STARTS: Include all the modules that will be used
use strict;
use Infoblox;

#Create a session to the Infoblox appliance
my $session = Infoblox::Session->new(
    master   => "192.168.1.2",
    username => "admin",
    password => "infoblox"
);
unless ($session) {
    die( "Construct session failed: ",
        Infoblox::status_code() . ":" . Infoblox::status_detail() );
}
print "Session created successfully\n";

#Create a new bulk host name template object

my $my_template = Infoblox::Grid::DNS::BulkHostNameTemplate->new(
    template_name   => "My Template",
    template_format => "#4",
);
unless ($my_template) {
    die(
        "Construct bulk host name template object my_template failed: ",
        Infoblox::status_code() . ":" . Infoblox::status_detail()
    );
}
print "Bulk host name template object my_template created successfully\n";

#Get the Member DNS object to add a bulk host name template object to it

my $grid_dns =
  $session->get( "object" => "Infoblox::Grid::DNS", "grid" => "Infoblox" );
```

```
unless ($grid_dns) {
    die( "Get Grid DNS failed: ",
        $session->status_code() . ":" . $session->status_detail() );
}

my $bhnt = $grid_dns->bulk_host_name_templates();

#Add new bulk host name template to Grid DNS bulk host name templates

unshift(@$bhnt, $my_template);
$grid_dns->bulk_host_name_templates($bhnt);

$session->modify($grid_dns)
  or die( "modify Grid DNS failed:",
    $session->status_code(), $session->status_detail() );
print "Modify grid DNS successful for bulk host name template object\n";

####PROGRAM ENDS####
```

AUTHOR

Infoblox Inc. http://www.infoblox.com/

SEE ALSO

Infoblox::Grid::DNS, Infoblox::Session, Infoblox::Session->get(), Infoblox::Session->modify()

COPYRIGHT

Copyright (c) 2017 Infoblox Inc.

Infoblox::Grid::DNS::ClientSubnetDomain - a DNS client subnet domain object.

- NAME
- DESCRIPTION
- CONSTRUCTOR
- SESSION METHODS
- MODULE METHODS
  - Infoblox::Grid::DNS->client_subnet_domains( )
- METHODS
  - domain( )
  - permission( )
- AUTHOR
- SEE ALSO
- COPYRIGHT

NAME

Infoblox::Grid::DNS::ClientSubnetDomain - a DNS client subnet domain object.

DESCRIPTION

The DNS client subnet domain represent the zone domain name that is allowed or forbidden for the EDNS client subnet (ECS) recursion.

CONSTRUCTOR

```
my $object = Infoblox::Grid::DNS::ClientSubnetDomain->new(
    'domain'     => $fqdn,            # Required
    'permission' => 'ALLOW' | 'DENY', # Optional / Default is 'ALLOW'
);
```

SESSION METHODS

The object does not support any session methods.

MODULE METHODS

The following functions can be applied to a DNS client subnet domain object.

Infoblox::Grid::DNS->client_subnet_domains( )

Use this method to set or retrieve the list of zone domain names that are allowed or forbidden for EDNS client subnet (ECS) recursion.

Example

```
#Get client_subnet_domains value
my $value = $grid_dns->client_subnet_domains();

#Construct Infoblox::Grid::DNS::ClientSubnetDomain object
my $object = Infoblox::Grid::ClientSubnetDomain->new(
    domain     => 'domain.com',
    permission => 'DENY',
);

#Modify client_subnet_domains value
$grid_dns->client_subnet_domains([$object]);
```

METHODS

This section describes all the methods that you can use to configure and retrieve the attribute values of the object.

domain( )

Use this method to set or retrieve the client subnet zone domain name.

Parameter

The valid value is a Fully-Qualified Domain Name (FQDN) that represents the desired zone domain name.

Returns

The method returns the attribute value.

Example

```
#Get domain value
my $value = $object->domain();

#Modify domain value
$object->domain('test-domain.com');
```

permission( )

Use this method to set or retrieve the client subnet domain ECS permission.

Parameter

The valid values are 'ALLOW' and 'DENY'. The default value is 'ALLOW'.

Returns

The method returns the attribute value.

Example

```
#Get permission value
my $value = $object->permission();

#Modify permission value
$object->permission('DENY');
```

AUTHOR

Infoblox Inc. http://www.infoblox.com/

SEE ALSO

Infoblox::Grid::DNS, Infoblox::Grid::DNS->client_subnet_domains(),

---

COPYRIGHT

Copyright (c) 2017 Infoblox Inc.

Infoblox::Grid::DNS::FileTransferSetting - A DNS traffic capture file transfer setting object.

- NAME
- DESCRIPTION
- CONSTRUCTOR
- SESSION METHODS
- MODULE METHODS
  - Infoblox::Grid::DNS->file_transfer_setting( )
  - Infoblox::Grid::Member::DNS->file_transfer_setting( )
- METHODS
  - directory( )
  - host( )
  - password( )
  - type( )
  - username( )
- AUTHOR
- SEE ALSO
- COPYRIGHT

NAME

Infoblox::Grid::DNS::FileTransferSetting - A DNS traffic capture file transfer setting object.

DESCRIPTION

The DNS traffic capture file transfer setting object provides information and configuration for captured DNS traffic transfer to a remote server.

CONSTRUCTOR

```
my $transfer_setting = Infoblox::Grid::DNS::FileTransferSetting->new(
    type      => 'NONE' | 'FTP' | 'SCP', # Required
    directory => $string,                # Optional / default is undefined
    host      => $string,                # Optional / default is undefined
    username  => $string,                # Optional / default is undefined
    password  => $string,                # Optional / default is undefined
);
```

SESSION METHODS

The object does not support any session methods.

MODULE METHODS

The following functions are available to be applied to a DNS traffic capture file transfer setting object.

Infoblox::Grid::DNS->file_transfer_setting( )

Use this method to set or retrieve the DNS capture file transfer settings. See Infoblox::Grid::DNS->file_transfer_setting() for parameters and return values.

Example

```
#Get override_enable_excluded_domain_names value
my $override_enable_excluded_domain_names = $object->override_enable_excluded_domain_names();

#Modify override_enable_excluded_domain_names value
$object->override_enable_excluded_domain_names("true");
```

Infoblox::Grid::Member::DNS->file_transfer_setting( )

Use this method to set or retrieve the DNS capture file transfer settings. See Infoblox::Grid::Member::DNS->file_transfer_setting() for parameters and return values.

Example

```
#Get override_enable_excluded_domain_names value
my $override_enable_excluded_domain_names = $object->override_enable_excluded_domain_names();
```

```
#Modify override_enable_excluded_domain_names value
$object->override_enable_excluded_domain_names("true");
```

METHODS

This section describes all the methods that can be used to configure and retrieve the attribute values of a DNS traffic capture file transfer setting object.

directory( )

Use this method to set or retrieve the directory to save the captured DNS queries and responses.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

The valid value is a desired path in a string format.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get directory value
my $directory = $object->directory();

#Modify directory value
$object->directory('/path/to/dir');
```

host( )

Use this method to set or retrieve the hostname for the DNS capture transfer destination server.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

The valid value is Fully-Qualified Domain Name (FQDN) of the DNS capture transfer destination server.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get host value
my $host = $object->host();

#Modify host value
$object->host('transfer.com');
```

password( )

Use this method to set the password to access the destination server directory. This is a write-only attribute.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

The valid value is a desired password in a string format.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Modify password value
$object->password('pass');
```

type( )

Use this method to set or retrieve the transfer protocol used to capture DNS queries and responses.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

The valid value are 'FTP', 'SCP' and 'NONE'.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get type value
my $type = $object->type();

#Modify type value
$object->type('SCP');
```

username( )

Use this method to set or retrieve the username to access the destination server directory.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

The valid value is a desired username in a string format.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get username value
my $username = $object->username();

#Modify username value
$object->username('admin');
```

AUTHOR

Infoblox Inc. http://www.infoblox.com/

SEE ALSO

Infoblox::Grid::DNS, Infoblox::Grid::DNS->file_transfer_setting(), Infoblox::Grid::Member::DNS, Infoblox::Grid::Member::DNS->file_transfer_setting()

COPYRIGHT

Copyright (c) 2017 Infoblox Inc.

Infoblox::Grid::RootNameServer - Custom root name server object.

- NAME
- DESCRIPTION
- CONSTRUCTOR
- MODULE METHODS
  - Infoblox::Grid::DNS->rootNS( )
- METHODS
  - host_name( )
  - ipv4addr( )
  - ipv6addr( )
- SAMPLE CODE
- AUTHOR
- SEE ALSO
- COPYRIGHT

NAME

Infoblox::Grid::RootNameServer - Custom root name server object.

DESCRIPTION

The custom root name server provides host name and IP address information of internal root name server. When the Infoblox appliance receives a recursive query for DNS data that it does not have, it will use the internal root name server specified, instead of using the internet root name server.

CONSTRUCTOR

```
my $rootns = Infoblox::Grid::RootNameServer->new(
             host_name => $fqdn,      #Required
             ipv4addr  => $ipv4addr   #Required if $ipv6addr is not present
             ipv6addr  => $ipv6addr   #Required if $ipv4addr is not present
);
```

MODULE METHODS

The following functions are available to be applied to a custom root name server object.

Infoblox::Grid::DNS->rootNS( )

Use this function to specify custom root name servers at the grid level on Infoblox appliance. See Infoblox::Grid::DNS->rootNS() for parameters and return values.

Example

```
#Construct a custom root name server object
my $rootns1 = Infoblox::Grid::RootNameServer->new(
             host_name => "qe.infoblox.com",
             ipv4addr  => "1.1.1.1"
);

#Construct a custom root name server object
my $rootns2 = Infoblox::Grid::RootNameServer->new(
             host_name => "doc.infoblox.com",
             ipv6addr  => "2006::132"
);

# Configure custom root name server on the grid DNS object
my $response = $Grid_DNS->rootNS([$rootns1, $rootns2]);
```

METHODS

This section describes all the methods that can be used to configure and retrieve the attribute values of a custom root name server.

host_name( )

Use this method to set or retrieve the host name of the custom root name server.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

Specify the host name in FQDN (Fully Qualified Domain Name) format. The FQDN consists of the host name followed by the domain name (example: abc.com). A host name can have a maximum of 256 characters.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get host name
my $host_name = $rootns->host_name();
#Modify host name
$rootns->host_name("qa.infoblox.com");
```

ipv4addr( )

Use this method to set or retrieve the IPv4 address of the custom root name server. Required only when ipv6 is not specified.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

An IPv4 address is a 32-bit number in dotted decimal notation. It consists of four 8-bit groups of decimal digits separated by decimal points (example: 192. 168.1.2).

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get ipv4addr
my $ipv4addr = $rootns->ipv4addr();
#Modify ipv4addr
$rootns->ipv4addr("2.2.2.2");
```

ipv6addr( )

Use this method to set or retrieve the IPv6 address of the custom root name server. Required only when ipv4 is not specified.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

An IPv6 address is a 128-bit number in colon hexadecimal notation. It consists of eight groups of four hexadecimal digits separated by colons (example: 12ab:0000:0000:0123:4567:89ab:0000:cdef).

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
# Getting ipv6addr
my $ipv6addr = $rootns->ipv6addr();
# Modifying ipv6addr
$rootns->ipv6addr("12ab::345");
```

SAMPLE CODE

The following sample code demonstrates the different functions that can be applied to a custom root name server object such as add, modify, and remove. Also, this sample includes error handling for the operations.

#Preparation prior to a custom root name server object insertion

```
#PROGRAM STARTS: Include all the modules that will be used
use strict;
use Infoblox;
```

```
#Create a session to the Infoblox appliance
my $session = Infoblox::Session->new(
              master   => "192.168.1.2", #appliance host ip
              username => "admin",       #appliance user login
              password => "infoblox"     #appliance password
);
unless ($session) {
       die("Construct session failed: ",
            Infoblox::status_code() . ":" . Infoblox::status_detail());
}
print "Session created successfully\n";
```

#Create a custom root name server object

```
my $rootns = Infoblox::Grid::RootNameServer->new(
             host_name => "qe.infoblox.com",
             ipv4addr  => "1.1.1.1"
);
unless($rootns) {
       die("Construct custom root name server object failed: ",
            Infoblox::status_code() . ":" . Infoblox::status_detail());
}
print "Root name server object created successfully\n";
```

#Get the Grid DNS object and add custom root name server to it

```
#Get the Grid DNS object
my @retrieved_objs = $session->get(
    object => "Infoblox::Grid::DNS",
    grid   => "Infoblox"
);
my $object = $retrieved_objs[0];
unless ($object) {
       die("Get Grid DNS object failed: ",
            $session->status_code() . ":" . $session->status_detail());
}
print "Get Grid DNS object found at least 1 matching entry\n";

#Apply the changes to the Grid DNS object
$object->rootNS([$rootns]);

#Apply the changes
$session->modify($object)
    or die("Modify Grid DNS object failed: ",
            $session->status_code() . ":" . $session->status_detail());
print "Grid DNS object with custom root name server updated to Infoblox appliance successfully\n";
```

#Modify custom root name server

```
#Modify existing custom root name server object

#Modifying the host name of the custom root name server object.
$rootns->host_name("eng.infoblox.com");

#Modifying the IPV4 address of the custom root name server object.
$rootns->ipv4addr("3.3.3.3");

#Apply changes to the Grid DNS object.
$object->rootNS([$rootns]);

#Update Grid DNS object through the Infoblox session.
$session->modify($object)
            or die("modify Grid DNS object failed: ",
                $session->status_code() . ":" . $session->status_detail());
       print "Grid DNS object with modified custom root name server updated to Infoblox appliance
successfully\n";
```

#Remove custom root name server

```
#Apply changes to the Grid DNS object to use default internet root name server.
$object->rootNS(undef);

#Update Grid DNS object through the Infoblox session.
$session->modify($object)
            or die("Remove custom root name server from Grid DNS object failed: ",
                $session->status_code() . ":" . $session->status_detail());
       print "Removed custom root name server from Grid DNS object successfully\n";
```

```
####PROGRAM ENDS####
```

AUTHOR

Infoblox Inc. http://www.infoblox.com/

SEE ALSO

Infoblox::Session, Infoblox::Grid::DNS

COPYRIGHT

Copyright (c) 2017 Infoblox Inc.

Infoblox::Grid::DNS::AttackDetect - a DNS Attack Detect object.

- NAME
- DESCRIPTION
- CONSTRUCTOR
- SESSION METHODS
- MODULE METHODS
  - Infoblox::Grid::DNS::AttackMitigation->detect_chr( )
  - Infoblox::Grid::DNS::AttackMitigation->detect_nxdomain_responses( )
  - Infoblox::Grid::DNS::AttackMitigation->detect_udp_drop( )
- METHODS
  - enabled( )
  - high( )
  - interval_max( )
  - interval_min( )
  - interval_time( )
  - low( )
- AUTHOR
- SEE ALSO
- COPYRIGHT

NAME

Infoblox::Grid::DNS::AttackDetect - a DNS Attack Detect object.

DESCRIPTION

The DNS Attack Detect object provides information about DNS attack detection settings.

CONSTRUCTOR

```
my $attack_detect = Infoblox::Grid::DNS::AttackDetect->new(
    enable        => 'true' | 'false', # Optional
    high          => $uint,            # Optional
    interval_max  => $uint,            # Optional
    interval_min  => $uint,            # Optional
    interval_time => $uint,            # Optional
    low           => $uint,            # Optional
);
```

SESSION METHODS

The object does not support any session methods.

MODULE METHODS

Infoblox::Grid::DNS::AttackMitigation->detect_chr( )

Use this method to set or retrieve configuration for the Cache Hit Ratio (CHR) changes detection. See Infoblox::Grid::DNS::AttackMitigation->detect_chr() for parameters and return values.

Example

```
#Get detect_chr value
my $detect_chr = $attack_mitigation->detect_chr();
#Modify detect_chr
$attack_mitigation->detect_chr($detect_chr);
```

Infoblox::Grid::DNS::AttackMitigation->detect_nxdomain_responses( )

Use this method to set or retrieve configuration for the recursive queries NXDOMAIN responses detection. See Infoblox::Grid::DNS::AttackMitigation->detect_nxdomain_responses() for parameters and return values.

Example

```
#Get detect_nxdomain_responses value
my $detect_nxdomain_responses = $attack_mitigation->detect_nxdomain_responses();
#Modify detect_nxdomain_responses
$attack_mitigation->detect_nxdomain_responses($detect_nxdomain_responses);
```

Infoblox::Grid::DNS::AttackMitigation->detect_udp_drop( )

Use this method to set or retrieve configuration for the UDP packet drop rate detection. See Infoblox::Grid::DNS::AttackMitigation->detect_udp_drop() for parameters and return values.

Example

```
#Get detect_udp_drop value
my $detect_udp_drop = $attack_mitigation->detect_udp_drop();
#Modify detect_udp_drop
$attack_mitigation->detect_udp_drop($detect_udp_drop);
```

METHODS

enabled( )

Use this method to enable or disable DNS attack detection.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

Specify 'true' to enable DNS attack detection or 'false' to disable it. The default value is 'false'.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get enable value
my $enable = $attack_detect->enable();
#Modify enable
$attack_detect->enable('true');
```

high( )

Use this method to set or retrieve the high threshold value (in percentage) for starting DNS attack detection.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

The valid value is an unsigned integer between 0 and 100.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get high value
my $high = $attack_detect->high();
#Modify high
$attack_detect->high(90);
```

interval_max( )

Use this method to set or retrieve the maximum number of events that have occurred before processing DNS attack detection.

Note that detection processing occurs each time when interval_max events have occurred or at least interval_min events have occurred and at least interval_time seconds have elapsed.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

The valid value is an unsigned integer.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get interval_max value
my $interval_max = $attack_detect->interval_max();
#Modify interval_max
$attack_detect->interval_max(100000);
```

interval_min( )

Use this method to set or retrieve the minimum number of events that have occurred before processing DNS attack detection.

Note that detection processing occurs each time when interval_max events have occurred or at least interval_min events have occurred and at least interval_time seconds have elapsed.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

The valid value is an unsigned integer.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get interval_min value
my $interval_min = $attack_detect->interval_min();
#Modify interval_min
$attack_detect->interval_min(1000);
```

interval_time( )

Use this method to set or retrieve the time interval between detection processing.

Note that detection processing occurs each time when interval_max events have occurred or at least interval_min events have occurred and at least interval_time seconds have elapsed.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

The valid value is an unsigned integer.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get interval_time value
my $interval_time = $attack_detect->interval_time();
#Modify interval_time
$attack_detect->interval_time(10);
```

low( )

Use this method to set or retrieve the low threshold value (in percentage) for starting DNS attack detection.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

The valid value is an unsigned integer between 0 and 100.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get low value
my $low = $attack_detect->low();
#Modify low
$attack_detect->low(40);
```

AUTHOR

Infoblox Inc. http://www.infoblox.com/

SEE ALSO

the Infoblox::Grid::DNS::AttackMitigation manpage, Infoblox::Grid::DNS::AttackMitigation->detect_chr(), Infoblox::Grid::DNS::AttackMitigation->detect_nxdomain_responses(), Infoblox::Grid::DNS::AttackMitigation->detect_udp_drop()

COPYRIGHT

Copyright (c) 2017 Infoblox Inc.

Infoblox::Grid::EAExpressionOp - A extensible attribute expression operand object.

- NAME
- DESCRIPTION
- CONSTRUCTOR
- SESSION METHODS
- MODULE METHODS
  - Infoblox::Grid::DNS::ReclamationSetting->ea_expression_list( )
- METHODS
  - op( )
  - op1( )
  - op1_type( )
  - op2( )
  - op2_type( )
- AUTHOR
- SEE ALSO
- COPYRIGHT

NAME

Infoblox::Grid::EAExpressionOp - A extensible attribute expression operand object.

DESCRIPTION

The extensible attribute expression operand object is used to build an extensible attribute expression lists. The allowed values for the expression operand object depend on the object they appear to be a part of.

CONSTRUCTOR

```
 my $expression = Infoblox::Grid::EAExpressionOp->new(
     op       => 'AND'| 'ENDLIST'| 'EQ'| 'EXISTS'| 'NOT_EQ'| 'NOT_EXISTS'| 'OR', # Required
     op1      => $ead,                                                          # Optional / Default is
undefined
     op2      => $string,                                                       # Optional / Default is
undefined
     op1_type => 'FIELD' | 'LIST' | 'STRING',                                   # Optional / Default is
undefined
     op2_type => 'FIELD' | 'LIST' | 'STRING',                                   # Optional / Default is
undefined
 );
```

SESSION METHODS

The object does not support any session methods.

MODULE METHODS

Infoblox::Grid::DNS::ReclamationSetting->ea_expression_list( )

Use this method to set or retrieve the ea_expression list. See Infoblox::Grid::DNS::ReclamationSetting->ea_expression_list() for parameters and return values.

Example

```
#Get ea_expression_list value
my $ea_expression_list = $object->ea_expression_list();

#Modify ea_expression_list value
$object->ea_expression_list([$start_list, $op1, $op2, $end_list]);
```

METHODS

op( )

Use this method to set or retrieve the operation name.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

The valid values are 'AND', 'ENDLIST', 'EQ', 'EXISTS', 'NOT_EQ', 'NOT_EXISTS' and 'OR'.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get op value
my $op = $object->op();

#Modify op value
$object->op('AND');
```

op1( )

Use this method to set or retrieve the first operand value.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

The valid value is an Infoblox::Grid::ExtensibleAttributeDef object.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get op1 value
my $op1 = $object->op1();

#Modify op1 value
$object->op1($ead);
```

op1_type( )

Use this method to set or retrieve the first operand type.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

The valid values are 'FIELD', 'LIST' and 'STRING'.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get op1_type value
my $op1_type = $object->op1_type();

#Modify op1_type value
$object->op1_type('LIST');
```

op2( )

Use this method to set or retrieve the second operand value.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

The valid value is a string that contains the second operand value.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get op2 value
my $op2 = $object->op2();

#Modify op2 value
$object->op2('OP2_VAL');
```

op2_type( )

Use this method to set or retrieve the second operand type.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

The valid values are 'FIELD', 'LIST' and 'STRING'.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get op2_type value
my $op2_type = $object->op2_type();

#Modify op2_type value
$object->op2_type('STRING');
```

AUTHOR

Infoblox Inc. http://www.infoblox.com/

SEE ALSO

Infoblox::Grid::DNS::ReclamationSetting, Infoblox::Grid::DNS::ReclamationSetting->ea_expression_list(), Infoblox::Grid::ExtensibleAttributeDef,

---

COPYRIGHT

Copyright (c) 2017 Infoblox Inc.

Infoblox::Grid::ExpressionOp - An expression operand object.

- NAME
- DESCRIPTION
- CONSTRUCTOR
- SESSION METHODS
- MODULE METHODS
  - Infoblox::Grid::DNS::ReclamationSetting->expression_list( )
- METHODS
  - op( )
  - op1( )
  - op1_type( )
  - op2( )
  - op2_type( )
- AUTHOR
- SEE ALSO
- COPYRIGHT

NAME

Infoblox::Grid::ExpressionOp - An expression operand object.

DESCRIPTION

The expression operand object is used to build expression lists. The allowed values for the expression operand object depend on the object they appear to be a part of.

CONSTRUCTOR

```
 my $expression = Infoblox::Grid::ExpressionOp->new(
     op       => 'AND'| 'OR'| 'GT'| 'GE'| 'LT'| 'LE'| 'EQ'| 'NOT_EQ'| 'ENDLIST'| 'EXISTS'| 'MATCH_IP'|
'MATCH_RANGE'| 'MATCH_CIDR', # Required
     op1      => $string,
# Optional / Default is undefined
     op2      => $string,
# Optional / Default is undefined
     op1_type => 'FIELD' | 'LIST' | 'STRING',
# Optional / Default is undefined
     op2_type => 'FIELD' | 'LIST' | 'STRING',
# Optional / Default is undefined
 );
```

SESSION METHODS

The object does not support any session methods.

MODULE METHODS

Infoblox::Grid::DNS::ReclamationSetting->expression_list( )

Use this method to set or retrieve the expression list. See Infoblox::Grid::DNS::ReclamationSetting->expression_list() for parameters and return values.

Example

```
#Get expression_list value
my $expression_list = $object->expression_list();

#Modify expression_list value
$object->expression_list([$start_list, $op1, $op2, $end_list]);
```

METHODS

op( )

Use this method to set or retrieve the operation name.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

The valid values are 'AND', 'ENDLIST', 'GT', 'LT', 'LE', 'GE', 'MATCH_IP', 'MATCH_RANGE', 'MATCH_CIDR', 'EQ', 'EXISTS', 'NOT_EQ', 'NOT_EXISTS' and 'OR'.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get op value
my $op = $object->op();

#Modify op value
$object->op('LE');
```

op1( )

Use this method to set or retrieve the first operand value.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

The valid value is a string that contains the first operand value.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get op1 value
my $op1 = $object->op1();

#Modify op1 value
$object->op1('OP1_VAL');
```

op1_type( )

Use this method to set or retrieve the first operand type.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

The valid values are 'FIELD', 'LIST' and 'STRING'.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get op1_type value
my $op1_type = $object->op1_type();

#Modify op1_type value
$object->op1_type('LIST');
```

op2( )

Use this method to set or retrieve the second operand value.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

The valid value is a string that contains the second operand value.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get op2 value
my $op2 = $object->op2();

#Modify op2 value
$object->op2('OP2_VAL');
```

op2_type( )

Use this method to set or retrieve the second operand type.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

The valid values are 'FIELD', 'LIST' and 'STRING'.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get op2_type value
my $op2_type = $object->op2_type();

#Modify op2_type value
$object->op2_type('STRING');
```

AUTHOR

Infoblox Inc. http://www.infoblox.com/

SEE ALSO

Infoblox::Grid::DNS::ReclamationSetting, Infoblox::Grid::DNS::ReclamationSetting->expression_list(),

---

COPYRIGHT

Copyright (c) 2017 Infoblox Inc.

Infoblox::Grid::ExtensibleAttributeDef::ListValue - Extensible Attribute Definition List Value object.

- NAME
- DESCRIPTION
- CONSTRUCTOR
- METHODS
  - value( )
- AUTHOR
- SEE ALSO
- COPYRIGHT

NAME

Infoblox::Grid::ExtensibleAttributeDef::ListValue - Extensible Attribute Definition List Value object.

DESCRIPTION

The Extensible Attribute Definition List Value object is used to store values for an extensible attribute object defined as the "list" type.

CONSTRUCTOR

```
my $ext_attr_def = Infoblox::Grid::ExtensibleAttributeDef::ListValue->new(
    value                => $string,                                          #Required
);
```

METHODS

This section describes all the methods that you can use to set and retrieve the values of an Extensible Attribute Definition List Value object.

value( )

Use this method to set or retrieve the value.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

Parameter is a string.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get value
my $value = $ext_attr_def_value->value();
#Modify value
$ext_attr_def_value->value($value);
```

AUTHOR

Infoblox Inc. http://www.infoblox.com/

SEE ALSO

Infoblox::Grid::ExtensibleAttributeDef

COPYRIGHT

Copyright (c) 2017 Infoblox Inc.

Infoblox::Grid::Member::FileDistribution - Grid Member FileDistribution object.

- NAME
- DESCRIPTION
- CONSTRUCTOR
- SESSION METHODS
  - Infoblox::Session->get( )
  - Infoblox::Session->modify( )
- METHODS
  - allow_uploads( )
  - name( )
  - enable_tftp( )
  - tftp_port( )
  - tftp_allow_access( )
  - enable_http( )
  - enable_http_acl( )
  - http_allow_access( )
  - enable_ftp( )
  - ftp_port( )
  - ftp_allow_access( )
  - ftp_login_banner( )
  - ftp_enable_passive_mode( )
  - ftp_enable_file_listing( )
  - override_allow_uploads( )
- SAMPLE CODE
- AUTHOR
- SEE ALSO
- COPYRIGHT

NAME

Infoblox::Grid::Member::FileDistribution - Grid Member FileDistribution object.

DESCRIPTION

You can configure the file distribution properties of a grid member. You can enable and disable TFTP, HTTP, and FTP services, and set their operational parameters.

CONSTRUCTOR

The Infoblox::Grid::Member::FileDistribution object is automatically generated upon the successful installation of the Infoblox appliance. It does not require manual construction.

SESSION METHODS

This section describes all the methods in the Infoblox::Session module that you can apply to a grid member FileDistribution object.

Infoblox::Session->get( )

Use this method to retrieve all the matching objects from the Infoblox appliance. See Infoblox::Session->get() for parameters and return values.

Key Reference

Apply the following attributes to get a specific grid member FileDistribution object:

name - Required. Host name in string format

Example

```
my $object   = $session->get(
   object    => "Infoblox::Grid::Member::FileDistribution",
   name      => "infoblox.localdomain",
    );
```

Infoblox::Session->modify( )

Use this method to modify an object in the Infoblox appliance. See Infoblox::Session->modify() for parameters and return values.

If you have applied an access control list to an object, you must restart services after you update the access control list for the changes to take effect. See Infoblox::Session->restart() method.

Example

```
#Use method to modify the enable_tftp Property.
$object->enable_tftp("true");
#Submit modification
my $response = $session->modify( $object );
```

METHODS

This section describes all the methods that you can use to configure and retrieve the attribute values of the file distribution properties of a grid member.

allow_uploads( )

Use this method to enable or disable file uploads for a member.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Setting this method to a defined value implicitly sets the override_allow_uploads method to "true". Setting the parameter to undefined causes the appliance to use the grid default and automatically resets the override_allow_uploads attribute to "false".

Note that when allow_uploads is set to a defined value and override_dallow_uploads is set to "false", the last operation takes precedence. Thus the sequence $object->allow_uploads("true"); $object->override_allow_uploads("false"); will set override_allow_uploads to "false", and the sequence $object->override_allow_uploads("false"); $object->allow_uploads("true"); will result in override_allow_uploads="true".

Parameter

Specify "true" to enable file uploads or "false" to disable it. The default value is "false".

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Getting allow_uploads
$file_distribution->allow_uploads( );
#Modifying allow_uploads
$file_distribution->allow_uploads("true");
```

name( )

Use this method to retrieve the grid name. This is a required argument.

Parameter

None.

Returns

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get name
my $name = $file_distribution->name();
```

enable_tftp( )

Use this method to enable or disable the TFTP service of a member.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

Specify "true" to enable the TFTP service or "false" to disable it. The default value is "false".

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Getting enable_tftp
$file_distribution->enable_tftp( );
#Modifying enable_tftp
$file_distribution->enable_tftp("true");
```

tftp_port( )

Use this method to specify the port to listen on for the TFTP file distribution service.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

Port number for TFTP file distribution service. Valid values are between 1 and 63999. The default is 69.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Getting tftp_port
$file_distribution->tftp_port( );
#Modifying tftp_port
$file_distribution->tftp_port("1069");
```

tftp_allow_access( )

Use this method to allow TFTP file distribution access from specified IPv4 addresses and networks.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

Valid value is an Infoblox::Grid::NamedACL object or an array reference that contains IPv4 addresses and networks. To reject TFTP file distribution access, add '!' as a prefix to IP addresses and networks. Specify "any" to allow any address or set to an empty array to clear the list.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails. If you did not specify a parameter, the method returns the attribute value.

Example

```
#Getting tftp_allow_access
my $file_distribution = $file_distribution->tftp_allow_access( );
#Modifying tftp_allow_access
$file_distribution->tftp_allow_access(["any"]);
#Modifying tftp_allow_access
$file_distribution->tftp_allow_access(["10.0.0.10", "20.0.0.0/24", "!"."30.0.0.30",
"!"."40.10.0.0/24"]);
#Modifying tftp_allow_access
$file_distribution->tftp_allow_access([]);
#Use Infoblox::Grid::NamedACL
$file_distribution->tftp_allow_access($nacl);
```

enable_http( )

Use this method to enable or disable the HTTP file distribution service of a member.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

Specify "true" to enable HTTP file distribution service or "false" to disable it. The default value is "false".

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Getting enable_http
$file_distribution->enable_http( );
#Modifying enable_http
$file_distribution->enable_http("true");
```

enable_http_acl( )

Use this method to enable or disable the ACLs for the HTTP file distribution service of a member.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

Specify "true" to enable ACLs for the HTTP file distribution service or "false" to disable it. The default value is "false".

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Getting enable_http_acl
$file_distribution->enable_http_acl( );
#Modifying enable_http_acl
$file_distribution->enable_http_acl("true");
```

http_allow_access( )

Use this method to allow HTTP file distribution access from specified IPv4 addresses and networks. enable_http and enable_http_acl must be set to 'true' in order for this http_allow_access to be usable. Set to an empty array to clear the list.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

Valid value is an Infoblox::Grid::NamedACL object or an array reference that contains IPv4 addresses and networks.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails. If you did not specify a parameter, the method returns the attribute value.

Example

```
#Getting http_allow_access
my $file_distribution = $file_distribution->http_allow_access( );
#Modifying http_allow_access
$file_distribution->http_allow_access(["10.0.0.10", "20.0.0.0/24", "30.0.0.30", "40.10.0.0/24"]);
#Modifying http_allow_access
$file_distribution->http_allow_access([]);
#Use Infoblox::Grid::NamedACL
$file_distribution->http_allow_access($nacl);
```

enable_ftp( )

Use this method to enable or disable the FTP service of a member.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

Specify "true" to enable the FTP service or "false" to disable it. The default value is "false".

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Getting enable_ftp
$file_distribution->enable_ftp( );
#Modifying enable_ftp
$file_distribution->enable_ftp("true");
```

ftp_port( )

Use this method to specify the port to listen on for the FTP file distribution service.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

Port number for the FTP file distribution service. Valid values are between 1 and 63999. The default is 21.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Getting ftp_port
$file_distribution->ftp_port( );
#Modifying ftp_port
$file_distribution->ftp_port("1022");
```

ftp_allow_access( )

Use this method to allow FTP file distribution access from specified IPv4 addresses and networks.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

Valid value is an Infoblox::Grid::NamedACL object or an array reference that contains IPv4 addresses and networks. To reject FTP file distribution access, add '!' as a prefix to IP addresses and networks. Specify "any" to allow any address or set to an empty array to clear the list.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails. If you did not specify a parameter, the method returns the attribute value.

Example

```
#Getting ftp_allow_access
my $file_distribution = $file_distribution->ftp_allow_access( );
#Modifying ftp_allow_access
$file_distribution->ftp_allow_access(["any"]);
#Modifying ftp_allow_access
$file_distribution->ftp_allow_access(["10.0.0.10", "20.0.0.0/24", "!"."30.0.0.30",
"!"."40.10.0.0/24"]);
#Modifying ftp_allow_access
$file_distribution->ftp_allow_access([]);
#Use Infoblox::Grid::NamedACL
$file_distribution->ftp_allow_access($nacl);
```

ftp_login_banner( )

Use this method to specify the login banner for the FTP file distribution service.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

The login banner for the FTP file distribution service. Valid values are strings of up to 256 characters.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Getting ftp_login_banner
$file_distribution->ftp_login_banner( );
#Modifying ftp_login_banner
$file_distribution->ftp_login_banner("This is my new FTP login banner");
```

ftp_enable_passive_mode( )

Use this method to enable or disable the FTP service in passive mode.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

Specify "true" to enable the FTP service in passive mode or "false" to disable it. The default value is "false".

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Getting ftp_enable_passive_mode
$file_distribution->ftp_enable_passive_mode( );
#Modifying ftp_enable_passive_mode
$file_distribution->ftp_enable_passive_mode("true");
```

ftp_enable_file_listing( )

Use this method to enable or disable file listing for the FTP service.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

Specify "true" to enable file listing or "false" to disable it. The default value is "false".

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Getting ftp_enable_file_listing
$file_distribution->ftp_enable_file_listing( );
#Modifying ftp_enable_file_listing
$file_distribution->ftp_enable_file_listing("true");
```

override_allow_uploads( )

The override_allow_uploads attribute controls whether the allow_uploads value in the member is used, instead of the grid default.

The override_allow_uploads attribute can be specified explicitly. It is also set implicitly when allow_uploads is set to a defined value.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

Specify "true" to set the override_allow_uploads flag or "false" to deactivate/unset it. The default value is "false".

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get override_allow_uploads for the member
my $type = $file_distribution->override_allow_uploads();
#Modify override_allow_uploads for the member
$file_distribution->override_allow_uploads("true");
```

SAMPLE CODE

The following sample code demonstrates the different functions that can be applied to an object, such as modify and get. This sample also includes error handling for the operations.

#Preparation prior to member FileDistribution object operations

```
#PROGRAM STARTS: Include all the modules that will be used
```

```
use strict;
use Infoblox;

#Create a session to the Infoblox appliance
my $session = Infoblox::Session->new(
    master   => "192.168.1.2",
    username => "admin",
    password => "infoblox"
);
unless ($session) {
   die("Construct session failed: ",
       Infoblox::status_code() . ":" . Infoblox::status_detail());
}
print "Session created successfully\n";
```

#Get and modify a Member FileDistribution object

```
# Get and modify the member FileDistribution property object. Note that the grid member named
infoblox.localdomain was previously created.
my $object = $session->get(
    object => "Infoblox::Grid::Member::FileDistribution",
    name   => "infoblox.localdomain"
);

 unless ($object) {
      die("get member FileDistribution failed: ",
      $session->status_code() . ":" . $session->status_detail());
  }
 print "member FileDistribution get successful\n";

#Modifying the value of the specified object
$object->enable_tftp("true");
$object->tftp_allow_access(["any"]);
$object->enable_http("true");
$object->enable_http_acl("true");
$object->http_allow_access(["10.0.0.10", "20.0.0.0/24"]);
$object->enable_ftp("true");
$object->ftp_allow_access(["10.0.0.10", "20.0.0.0/24", "!"."30.0.0.30", "!"."40.10.0.0/24"]);
$object->tftp_port(1069);
$object->ftp_port(1022);
$object->ftp_login_banner("My new banner");
$object->ftp_enable_passive_mode("true");

 #Apply the changes
 $session->modify($object)
  or die("modify member FileDistribution failed: ",
      $session->status_code() . ":" . $session->status_detail());

print "FileDistribution member object modified successfully \n";

####PROGRAM ENDS####
```

AUTHOR

Infoblox Inc. http://www.infoblox.com/

SEE ALSO

Infoblox::Grid::FileDistributionDir, Infoblox::Session, Infoblox::Session->get(), Infoblox::Session->modify(), Infoblox::Grid::NamedACL

COPYRIGHT

Copyright (c) 2017 Infoblox Inc.

Infoblox::Grid::SmartFolder::GroupBy - Smart Folder Group By object

- NAME
- DESCRIPTION
- CONSTRUCTOR
- MODULE METHODS
  - Infoblox::Grid::GlobalSmartFolder->group_bys( )
  - Infoblox::Grid::PersonalSmartFolder->group_bys( )
- METHODS
  - value( )
  - value_type( )
  - enable_grouping( )
- SAMPLE CODE
- AUTHOR
- SEE ALSO
- COPYRIGHT

NAME

Infoblox::Grid::SmartFolder::GroupBy - Smart Folder Group By object

---

DESCRIPTION

Defines grouping information in a global or personal smart folder.

CONSTRUCTOR

```
my $group_by = Infoblox::Grid::SmartFolder::GroupBy->new(
    value                => $string,            #Required
    enable_grouping      => "true" | "false",   #Required
    value_type           => "EXTATTR" | "NORMAL", #Optional / Default is "EXTATTR"
);
```

MODULE METHODS

The following functions can be applied to a GroupBy object.

Infoblox::Grid::GlobalSmartFolder->group_bys( )

Use this function to specify a list of "group by" items to store in a global smart folder.

Example

```
#Construct an object
my $group_by_state = Infoblox::Grid::SmartFolder::GroupBy->new(
                                                    enable_grouping => 'true',
                                                    value => "State",
                                                    value_type => "EXTATTR",
                                                   );

my $group_by_vcluster = Infoblox::Grid::SmartFolder::GroupBy->new(
                                                  enable_grouping => 'true',
                                                  value => "v_cluster",
                                                  value_type => "NORMAL",
                                                 );
# Configure group_by items
my $response = $GlobalSmartFolder->group_bys([$group_by_state,$group_by_vcluster]);
```

Infoblox::Grid::PersonalSmartFolder->group_bys( )

Use this function to specify a list of "group by" items to store in a personal smart folder.

Example

```
#Construct an object
my $group_by_state = Infoblox::Grid::SmartFolder::GroupBy->new(
                                                    enable_grouping => 'true',
                                                    value => "State",
                                                    value_type => "EXTATTR",
                                                   );
```

```
my $group_by_vcluster = Infoblox::Grid::SmartFolder::GroupBy->new(
                                                        enable_grouping => 'true',
                                                        value => "v_cluster",
                                                        value_type => "NORMAL",
                                                       );
# Configure group_by items
my $response = $PersonalSmartFolder->group_bys([$group_by_state,$group_by_vcluster]);
```

METHODS

This section describes all the methods that you can use to configure and retrieve the attribute values of a SmartFolder::GroupBy object.

value( )

Use this method to specify the name of the attribute (when value_type is "EXTATTR" or "NORMAL"). The possible values for the value attribute are v_datacenter, v_cluster, v_host, v_entity_type, port_vlan_name, network_component_name, network_component_port_name, and any valid extensible attribute in the appliance.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

Valid values are strings corresponding to extensible attributes, object types, or object names.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
# Getting value
my $value = $group_by->value();
# Modifying value
$group_by->value("Country");
```

value_type( )

Use this method to set or retrieve the value type.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

"EXTATTR" and "NORMAL" are supported.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
# Getting value_type
my $value_type = $group_by->value_type();
# Modifying value_type
$group_by->value_type("EXTATTR");
```

enable_grouping( )

Use this method to set or retrieve the enable grouping attribute.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

Specify "true" to set the enable_grouping flag or "false" to deactivate/unset it. "group by" rules where enable_grouping is set to "false" will be ignored when retrieving the corresponding smart folder.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
# Getting enable_grouping
my $enable_grouping = $group_by->enable_grouping();
# Modifying enable_grouping
$group_by->enable_grouping("true");
```

SAMPLE CODE

##Preparation prior to a smart folder group by object insertion

```
#PROGRAM STARTS: Include all the modules that will be used
use strict;
use Infoblox;

#Create a session to the Infoblox appliance
my $session = Infoblox::Session->new(
    master   => "192.168.1.2",
    username => "admin",
    password => "infoblox"
);
unless ($session) {
   die("Construct session failed: ",
       Infoblox::status_code() . ":" . Infoblox::status_detail());
}
print "Session created successfully\n";
```

#Create a smart folder query item object

```
my $group_by_state = Infoblox::Grid::SmartFolder::GroupBy->new(
                                                        enable_grouping => 'true',
                                                        value => "State",
                                                        value_type => "EXTATTR",
                                                       );
unless ($group_by_state){
     die("Construct smart folder group_by item failed: ",
           Infoblox::status_code() . ":" . Infoblox::status_detail());
}

print "SmartFolder::GroupBy object created successfully\n";
```

#Create a personal smart folder and use $group_by

```
 #create a personal smart folder and use $query_item
 my $smart_folder = Infoblox::Grid::PersonalSmartFolder->new(
    name         => "my_personal_folder",
    group_bys    => [$group_by_state],
  );

#put smart folder into session
$session->add($smart_folder)
  or die("Smart folder creation failed: ",
           Infoblox::status_code() . ":" . Infoblox::status_detail());

print "Smart folder creation successful\n";
```

#Clean up

```
my @result = $session->get(
    object => "Infoblox::Grid::PersonalSmartFolder",
    name   => "my_personal_folder"
);

unless (scalar(@result) == 0) {
    $session->remove($result[0]);
}

####PROGRAM ENDS####
```

AUTHOR

Infoblox Inc. http://www.infoblox.com/

SEE ALSO

Infoblox::Session, Infoblox::Session->add(), Infoblox::Session->get(), Infoblox::Session->modify(),Infoblox::Grid::GlobalSmartFolder,Infoblox::Grid::PersonalSmartFolder

---

COPYRIGHT

Copyright (c) 2017 Infoblox Inc.

Infoblox::Grid::SmartFolder::QueryItem - Smart Folder Query Item object

- NAME
- DESCRIPTION
- CONSTRUCTOR
- MODULE METHODS
  - Infoblox::Grid::GlobalSmartFolder->query_items( )
- METHODS
  - is_extensible_attribute( )
  - name( )
  - operator( )
  - value( )
- SAMPLE CODE
- AUTHOR
- SEE ALSO
- COPYRIGHT

NAME

Infoblox::Grid::SmartFolder::QueryItem - Smart Folder Query Item object

---

DESCRIPTION

Manipulates a Smart Folder Query Item object that contains information about a query to be stored in a global or personal smart folder.

CONSTRUCTOR

```
 my $query_item = Infoblox::Grid::SmartFolder::QueryItem->new(
     name                   => $string,    #Required
     is_extensible_attribute => $string,    #Optional / Default is "true"
     operator               => "eq" | "ne" | "lt" | "gt" | "leq" | "geq" | "begins with" | "does not begin
with" | "ends with" | "does not end with" | "has value" | "does not have value" | "within" | "on" | "today"
| "yesterday" | "this week" | "this month",    #Required
     value                  => $string | $num | $boolean | type of extensible attribute definition,
#Optional / Default is undefined
 );
```

MODULE METHODS

The following functions can be applied to a query item object.

Infoblox::Grid::GlobalSmartFolder->query_items( )

Use this function to specify a list of query items to store in a global smart folder.

Example

```
#Construct an object
  my $query_item1 = Infoblox::Grid::SmartFolder::QueryItem->new(
    name                   => "Building",
    is_extensible_attibute => "true",
    operator               => "eq",
    value                  => "office1"
);

#Construct an object
  my $query_item2 = Infoblox::Grid::SmartFolder::QueryItem->new(
    name                   => "address",
    is_extensible_attibute => "false",
    operator               => "ne",
    value                  => "10.0.0.1"
);

# Configure query items
my $response = $GlobalSmartFolder->query_items([$query_item1,$query_item2]);
```

METHODS

This section describes all the methods that you can use to configure and retrieve the attribute values of a SmartFolder::QueryItem object.

is_extensible_attribute( )

Use this method to indicate whether this smart folder query item is an extensible or not. When the attribute is_extensible_attribute is "true", the query item is an extensible attribute. When it is "false", it is a normal attribute.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

Valid values are "true" or "false".

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
# Getting is_extensible_attribute
my $is_extensible_attribute = $query_item->is_extensible_attribute();
# Modifying is_extensible_attribute
$query_item->is_extensible_attribute("true");
```

name( )

Use this method to set or retrieve the name of the smart folder query item. The possible values for the name attribute are

```
address
circuit_id
comment
disabled
duid
is_leaf_network
last_discovered_timestamp
locked
mac_address
name
network_component_name
network_component_port_name
network_view
port_vlan_name
record_type
remote_id
server_association_type
service
service_status
type
unmanaged
v_cluster
v_datacenter
v_entity_type
v_host
was_conflict
zone_func
zone_type
```

or any valid extensible attribute definition name present in the system.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

Desired name for the smart folder query item.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails. If you did not specify a parameter, the method returns the attribute value.

Example

```
# Getting name
my $name = $query_item->name();
# Modifying name
$query_item->name("Region");
```

operator( )

Use this method to set or retrieve the operator used in the query.

Parameter

Valid values are any of the following strings: "eq", "ne", "lt", "gt", "geq", "leq", "begins with", "does not begin with", "ends with", "does not end with", "has value", "does not have value", "within", "on", "today", "yesterday", "this week", "this month" and "inheritance state equals".

"has_value" and "does not have value" apply to extensible attribute values only.

For valid values for 'value' parameter for the "within" operator, please see the description of the 'value' method.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
# Getting operator
my $operator = $query_item->operator();
# Modifying operator
$query_item->operator("today");
```

value( )

Use this method to set or retrieve the value used in the query.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

Desired value in string format.

For the "within" operator, the following format is supported: n [d/w/m], where n is the number of days/weeks/months depending on the unit following it. The unit is required. For example, 4d (4 days), 1w (1 week), or 7m (7 months)

For the following query items, a boolean value 'true' or 'false' will be expected: "disabled", "is_leaf_network", "locked".

For a "type" query item, a valid value is one of:

```
'All'
'AllNetwork'
'AllTemplate'
'DhcpFailoverAssoc'
'DhcpFilter'
'DHCPObjects'
'DhcpRange'
'DhcpRangeTemplate'
'DNSObjects'
'FixedAddress'
'FixedAddressTemplate'
'Host'
'IPAMObjects'
'IPv4DHCPObjects'
'IPv6DhcpFilter'
'IPv6DHCPObjects'
'IPv6DhcpRange'
'IPv6DhcpRangeTemplate'
'IPv6FixedAddress'
'IPv6FixedAddressTemplate'
'IPv6Lease'
'IPv6Network'
'IPv6NetworkTemplate'
'IPv6OptionSpace'
'IPv6SharedNetwork'
'MacFilterAddress'
'Member'
'Network'
'NetworkTemplate'
'Network/Zone/Range/Member'
'NsGroup'
'OptionFilterMatchRule'
'OptionSpace'
'Reservation'
'ResourceRecord'
'RoamingHost'
'SharedNetwork'
'SharedRecordGroup'
'View'
'Zone'
```

For a "record_type" query item a valid value is one of:

```
'A'
'AAAA'
'All types'
'CNAME'
'DNAME'
'MX'
```

```
'PTR'
'SRV'
'TXT'
```

For a "service_status" query item a valid value is one of:

```
'FAILED'
'INACTIVE'
'OFFLINE'
'WARNING'
'WORKING'
```

For a "zone_func" query item a valid value is one of:

```
'Forward-mapping Zone'
'IPv4 Reverse-mapping Zone'
'IPv6 Reverse-mapping Zone'
```

For a "zone_type" query item a valid value is one of:

```
'Authoritative'
'Delegation'.
'Forward'
'Stub'
```

For a "v_type" query item a valid value is one of:

```
'HostSystem'
'VirtualCenter'
'VirtualMachine'
```

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
# Getting value
my $value = $query_item->value();
# Modifying value
$query_item->value("Santa Clara");
```

SAMPLE CODE

The following sample code demonstrates the different functions that can be applied to an object such as add, modify. This sample also includes error handling for the operations.

##Preparation prior to a smart folder query item object insertion

```
#PROGRAM STARTS: Include all the modules that will be used
use strict;
use Infoblox;

#Create a session to the Infoblox appliance
my $session = Infoblox::Session->new(
    master   => "192.168.1.2",
    username => "admin",
    password => "infoblox"
);
unless ($session) {
   die("Construct session failed: ",
       Infoblox::status_code() . ":" . Infoblox::status_detail());
}
print "Session created successfully\n";
```

#Create a smart folder query item object

```
my $query_item = Infoblox::Grid::SmartFolder::QueryItem->new(
    name                      => "Country",
    is_extensible_attribute   => "true",
    operator                  => "eq",
    value                     => "Canada",
);
unless ($query_item){
     die("Construct smart folder query item failed: ",
           Infoblox::status_code() . ":" . Infoblox::status_detail());
}
```

```
print "SmartFolder::QueryItem object created successfully\n";
```

#Create a personal smart folder and use $query_item

```
#create a personal smart folder and use $query_item
my $smart_folder = Infoblox::Grid::PersonalSmartFolder->new(
   name         => "my_personal_folder",
   query_items  => [$query_item],
 );

#put smart folder into session
$session->add($smart_folder)
  or die("Smart folder creation failed: ",
           Infoblox::status_code() . ":" . Infoblox::status_detail());

print "Smart folder creation successful\n";
```

#Get and modify the query items

```
#get the personal smart folder object from session
my @result = $session->get(
    object => "Infoblox::Grid::PersonalSmartFolder",
    name   => "my_personal_folder"
);

unless (scalar(@result) == 0) {
    my $smart_folder = $result[0];

    if ($smart_folder) {

      #modify query_items
      $query_item->value("France");
      $smart_folder->query_items([$query_item]);

      #update session
      $session->modify($smart_folder)
        or die("modify smart_folder my_personal_folder failed: ",
            $session->status_code(), $session->status_detail());
          }
       } else {
         print "No smart folder found.";
       }
```

#Clean up

```
@result = $session->get(
    object => "Infoblox::Grid::PersonalSmartFolder",
    name   => "my_personal_folder"
);

unless (scalar(@result) == 0) {
    $session->remove($result[0]);
}

####PROGRAM ENDS####
```

AUTHOR

Infoblox Inc. http://www.infoblox.com/

SEE ALSO

Infoblox::Session, Infoblox::Session->add(), Infoblox::Session->get(), Infoblox::Session->modify(),Infoblox::Grid::GlobalSmartFolder,Infoblox::Grid::PersonalSmartFolder

---

COPYRIGHT

Copyright (c) 2017 Infoblox Inc.

Infoblox::Grid::SmartFolderChildren - Smart Folder retrieval item.

- NAME
- DESCRIPTION
- CONSTRUCTOR
- SESSION METHODS
  - Infoblox::Session->search( )
- METHODS
  - is_matching_object( )
  - value( )
  - value_type( )
  - comment( )
  - extattrs( )
  - extensible_attributes( )
- SAMPLE CODE
- AUTHOR
- SEE ALSO
- COPYRIGHT

NAME

Infoblox::Grid::SmartFolderChildren - Smart Folder retrieval item.

DESCRIPTION

This module helps read the objects that are associated with either a smart folder (global or personal) or a set of queries that users can make without saving a smart folder object on the appliance.

This interface can be used for both "filtering" and "grouping" the results of smart folder associated objects.

CONSTRUCTOR

The object cannot be constructed. It can only be returned as part of a search.

SESSION METHODS

This section describes all the methods in the Infoblox::Session module that you can apply to a personal smart folder object.

Infoblox::Session->search( )

Use this method to search for SmartFolderChildren objects in the Infoblox appliance. See Infoblox::Session->search() for parameters and return values.

Key References

```
 Apply the following attributes to search for a specific SmartFolderChildren object:

  query_items                  - Required. A list of Infoblox::Grid::SmartFolder::QueryItem objects.
  smart_folder_name            - Required. PersonalSmartFolder/GlobalSmartFolder name in string
format.
  smart_folder_type            - Optional. "Global" / "Personal", "Global" if not specified.
  smart_folder_group_bys       - Optional. A list of Infoblox::Grid::SmartFolder::GroupBy objects.
  smart_folder_group_by_values - Optional. A list of Infoblox::Grid::SmartFolder::GroupByValue
objects.

 Both "smart_folder_name" and "query_items" are marked "Required" but they cannot be specified
together. Note the following guidelines:

 - "query_items" cannot be specified when "smart_folder_name" is specified because if
"smart_folder_name" is specified, all the properties of a query can be derived from that smart
folder.

 - "query_items" can be specified with "smart_folder_group_bys". In this case, the results are
grouped by specified group_bys attribute and a list of intermediate objects are returned.

 - When only "smart_folder_name" is specified, it runs like a query using all properties from this
folder and returning the results accordingly.

 - When "smart_folder_group_bys" is specified with "smart_folder_name", the query is used from the
specified smart folder; however, the results are shown from the level listed in
"smart_folder_group_bys".

 - "smart_folder_group_bys" by itself cannot be an argument since it does not provide sufficient
input.
```

```
 - "smart_folder_group_by_values" by itself cannot be an argument since it does not provide
sufficient input.

 - For the non-saved smart folder query, the "smart_folder_group_by_values" can be used to specify
the level of results to return. "smart_folder_group_bys" must also be specified in this case.
```

Example

```
  Please see the 'SAMPLE CODE' section below.
```

METHODS

This section describes all the methods that you can use to retrieve the attribute values of a SmartFolderChildren object.

is_matching_object( )

Use this method to retrieve the is_matching_object flag.

Parameter

The flag will be "true" if a matching object is returned inside the value(); if not, the attribute `value()` will contain the name of the intermediate object.

value( )

Use this method to retrieve the value of the object.

Parameter

This attribute will contain either an actual object matching the query (if is_matching_object is "true") or a string describing the name of the intermediate object.

Intermediate objects are the extensible attribute value objects for which the grouping was requested. Matching objects are the DB objects that matched the query specified.

value_type( )

Use this method to retrieve the type of the returned value.

Parameter

If is_matching_object is "false", this will contain one of: "STRING" / "INTEGER" / "DATE" / "BOOLEAN".

comment( )

Use this method to retrieve the comment.

Parameter

This attribute will return the value of the `comment()` attribute of the associated object, if `is_matching_object()` is "true". It will return undef if is_matching_object is "false".

extattrs( )

Use this method to retrieve the extensible attributes. This attribute is read-only and cannot be set.

Parameter

This attribute will return the value of the `extattrs()` attribute of the associated object (if `is_matching_object()` is "true"). It will return undef if is_matching_object is "true"). It will return undef if is_matching_object is "false".

extensible_attributes( )

Use this method to retrieve the extensible attributes.

Parameter

This attribute will return the value of the `extensible_attributes()` attribute of the associated object (if `is_matching_object()` is "true"). It will return undef if is_matching_object is "false".

SAMPLE CODE

```
USE-CASES & EXAMPLES:

#########################################################################
```

```
# Case 1 : Fetching objects based on the query defined for an existing folder
##############################################################################

my @retrieved_objs = $session->search(
            object =>"Infoblox::Grid::SmartFolderChildren" ,
            smart_folder_name => "global_smart_folder_1"
          );

my $object = $retrieved_objs[0];

if ($object->is_matching_object())
{
   print $object->comment();
   # $object->extensible_attributes() will contain the object's
   # extensible attributes
   #
   # This will be the actual matching object
   # $object->value();
}
else
{
   # Name of the intermediate group, use this to fetch another level if needed
   # This will be "USA", "Canada" etc
   # if the grouping was on "Country" for example
   print $object->value();
   print $object->value_type();
}

##############################################################################
# Case 2 : Fetching objects based on the smart folder and a given group_by values
##############################################################################

my $groupbyvalue1 = Infoblox::Grid::SmartFolder::GroupByValue->new(
          name => "Country",
          value => "USA",
);
my $groupbyvalue2 = Infoblox::Grid::SmartFolder::GroupByValue->new(
          name => "State",
          value => "CA",
);

my  @retrieved_objs = $session->search(
      object =>"Infoblox::Grid::SmartFolderChildren" ,
      smart_folder_name => "global_smart_folder_1",
      smart_folder_group_by_values => [$groupbyvalue1, $groupbyvalue2],
         );

my $object = $retrieved_objs[0];
if ($object->is_matching_object())
{
   print $object->comment();
   # $object->extensible_attributes() will contain the object's
   # extensible attributes
   #
   # This will be the actual matching object
   # $object->value();
}
else
{
   # Name of the intermediate group, use this to fetch another level if needed
   print $object->value();
   print $object->value_type();
}

##############################################################################
# Case 3 : Fetching objects based on the not-saved smart folder query, no grouping
##############################################################################

my  @retrieved_objs = $session->search(
      object =>"Infoblox::Grid::SmartFolderChildren" ,
      #This query will only return intermediate objects
      query_items => [queryitem1, queryitem2],
         );

my $object = $retrieved_objs[0];
if ($object->is_matching_object())
{
   print $object->comment();
   # $object->extensible_attributes() will contain the object's
   # extensible attributes
   #
   # This will be the actual matching object
```

```
    # $object->value();

}
else
{
    # This shall never happen
}

#########################################################################
# Case 4 : Fetching objects based on the not-saved smart folder query & with top level grouping
#########################################################################

my  @retrieved_objs = $session->search(
      object =>"Infoblox::Grid::SmartFolderChildren" ,
      query_items => [queryitem1, queryitem2],
      smart_folder_group_bys => [$groupby1, $groupby2],
         );

my $object = $retrieved_objs[0];
if ($object->is_matching_object())
{
    # This shall never happen
}
else
{
    # Name of the intermediate group, use this to fetch another level if needed
    print $object->value();
    print $object->value_type();
}

#########################################################################
# Case 5 : Fetching objects based on the not-saved smart folder query & with second level grouping
#########################################################################

my  @retrieved_objs = $session->search(
      object =>"Infoblox::Grid::SmartFolderChildren" ,
      query_items => [$queryitem1, $queryitem2],
      smart_folder_group_bys => [$groupby1, $groupby2],
      smart_folder_group_by_values => [$groupbyvalue1],
         );

my $object = $retrieved_objs[0];
if ($object->is_matching_object())
{
    # This shall never happen
}
else
{
    # Name of the intermediate group, use this to fetch another level if needed
    # The intermediate objects grouped by $groupby2 are returned in this case
    print $object->value();
    print $object->value_type();
}

#########################################################################
# Case 6 : Fetching objects based on the defined SF query & grouping by a given rule "value"
#########################################################################

my $groupbyvalue1 = Infoblox::Grid::SmartFolder::GroupByValue->new(
          name => "Country",
          value => "USA",
);

my  @retrieved_objs = $session->search(
      object =>"Infoblox::Grid::SmartFolderChildren" ,
      smart_folder_name => "global_smart_folder_1"
      smart_folder_group_by_values => [$groupbyvalue1],
   );

my $object = $retrieved_objs[0];

if ($object->is_matching_object())
{
    print $object->comment();
    # $object->extensible_attributes() will contain the object's
    # extensible attributes
    #
    # This will be the actual matching object
    # $object->value();
}
else
```

```
{
   # Name of the intermediate group, use this to fetch another level if needed
   print $object->value();
   print $object->value_type();
}

##########################################################################
# Case 7 : Fetching next levels of data from result of previous one (NESTED example)
##########################################################################

# EXAMPLE: Get objects for Country USA-> State CA-> DB Objects.

#Add Personal Smart Folder with 2 levels of grouping
#Use this smart folder to fetch intermediate objects from first level (groups)
#Use the result of this first level and fetch second level of intermediate objects (groups)
#Use the result of this second level and fetch the final / third level of DB protocol objects

my $groupby1 = Infoblox::Grid::SmartFolder::GroupBy->new(
            enable_grouping => True,
            value => "Country",
            value_type => "EXTATTR",
);
my $groupby2 = Infoblox::Grid::SmartFolder::GroupBy->new(
            enable_grouping => True,
            value => "State",
            value_type => "EXTATTR",
);

my  $smartfolder = Infoblox::Grid::PersonalSmartFolder->new(
             name => "personal_smart_folder_1",
             comment => "personal smart folder 1 comment",
             enable_grouping => True,
             group_bys => [$groupby1, $groupby2],
             query_items => [],
              );
my $response = $session->add($smartfolder)

my  @retrieved_objs = $session->search(
      object =>"Infoblox::Grid::SmartFolderChildren" ,
      smart_folder_name => "personal_smart_folder_1",
      smart_folder_type => "Personal",
   );

my $object = $retrieved_objs[0];

if ($object->is_matching_object())
{
        # This will not happen since intermediate objects are returned in this case
}
else
{
   # Name of the intermediate group, use this to fetch another level if needed
   my $intermediate_group_value = $object->value();
        if ($intermediate_group_value eq "USA")
        {
            my $groupbyvalue_usa = Infoblox::Grid::SmartFolder::GroupByValue->new(
                name => "Country",
                value => "USA",
            );
            my  @retrieved_objs = $session->search(
                object =>"Infoblox::Grid::SmartFolderChildren" ,
                smart_folder_name => "personal_smart_folder_1"
                smart_folder_group_by_values => [$groupbyvalue_usa],
            );

            my $object = $retrieved_objs[0];

            my $intermediate2_group_value = $object->value();
            if ($intermediate2_group_value eq "CA")
            {
                  my $groupbyvalue_ca = Infoblox::Grid::SmartFolder::GroupByValue->new(
                      name => "State",
                      value => "CA",
                  );
                  my  @retrieved_objs = $session->search(
                      object =>"Infoblox::Grid::SmartFolderChildren" ,
                      smart_folder_name => "personal_smart_folder_1"
                      smart_folder_type => "Personal",
```

```
        smart_folder_group_by_values => [$groupbyvalue_ca],
);

my $object = $retrieved_objs[0];

 if ($object->comment() eq "I am a network object")
 {
     # $object here represents the Database object
 }
            }
  }

####PROGRAM ENDS####
```

AUTHOR

Infoblox Inc. http://www.infoblox.com/

SEE ALSO

Infoblox::Grid::PersonalSmartFolder, Infoblox::Session->add(), Infoblox::Session->fetch_smart_folder_objects(), Infoblox::Session->get(), Infoblox::Session->modify(), Infoblox::Session->remove(), Infoblox::Session->search(),Infoblox::Session

---

COPYRIGHT

Copyright (c) 2017 Infoblox Inc.

Infoblox::Grid::PersonalSmartFolder - Personal Smart Folder object.

- NAME
- DESCRIPTION
- CONSTRUCTOR
- SESSION METHODS
  - Infoblox::Session->add( )
  - Infoblox::Session->get( )
  - Infoblox::Session->modify( )
  - Infoblox::Session->remove( )
  - Infoblox::Session->search( )
- METHODS
  - comment( )
  - group_bys( )
  - name( )
  - query_items( )
- SAMPLE CODE
- AUTHOR
- SEE ALSO
- COPYRIGHT

NAME

Infoblox::Grid::PersonalSmartFolder - Personal Smart Folder object.

DESCRIPTION

A Personal Smart Folder object is used for data organization.

CONSTRUCTOR

```
 my $group = Infoblox::Grid::PersonalSmartFolder->new(
     name                => $string,                                          #Required
     comment             => $string,                                          #Optional
/ Default is undefined
     group_bys           => [$GroupbyItem1, $GroupbyItem2],                   #Optional
/ Default is undefined
     query_items         => [$QueryItem1, $QueryItem2],                       #Optional
/ Default is undefined
 );
```

SESSION METHODS

This section describes all the methods in the Infoblox::Session module that you can apply to a Personal Smart Folder object.

Infoblox::Session->add( )

Use this method to add an object to the Infoblox appliance. See Infoblox::Session->add() for parameters and return values.

Example

```
#Construct the QueryItem in-memory object
my $query_item1 = Infoblox::Grid::SmartFolder::QueryItem->new(
    name                => "location",
    operator            => "eq",
    value               => "Santa Clara",
);

#Construct the PersonalSmartFolder object
my $group_by_state = Infoblox::Grid::SmartFolder::GroupBy->new(
                                                              enable_grouping => 'true',
                                                              value => "State",
                                                              value_type => "EXTATTR",
                                                             );

my $smart_folder = Infoblox::Grid::PersonalSmartFolder->new(
    name                => "smartfolder1",
    comment             => "personal smart folder",
    group_bys           => [$group_by_state],
    query_items         => [$query_item1],
```

```
);
# Submit for addition
my $response = $session->add( $smart_folder );
```

Infoblox::Session->get( )

Use this method to retrieve all the matching objects from the Infoblox appliance. See Infoblox::Session->get() for parameters and return values.

Key References

```
Apply the following attributes to get a specific PersonalSmartFolder object:

 name      - Required. PersonalSmartFolder name in string format.
 comment   - Optional. Comment in string format.
```

Example

```
my @retrieved_objs = $session->get(
    object     => "Infoblox::Grid::PersonalSmartFolder",
    name       => "smartfolder1" );
```

Infoblox::Session->modify( )

Use this method to modify an object in the Infoblox appliance. See Infoblox::Session->modify() for parameters and return values.

Example

```
# Use this method to modify the comment.
$smart_folder->comment("This is a modified comment");
# Submit modification
my $response = $session->modify( $smart_folder );
```

Infoblox::Session->remove( )

Use this method to remove an object from the Infoblox appliance. See Infoblox::Session->remove() for parameters and return values.

To remove a specific object, use `get()` or `search()` to retrieve the specific object first, and then submit this object for removal.

Example

```
# Get the PersonalSmartFolder objects using a name
my @retrieved_objs = $session->get(
    object     => "Infoblox::Grid::PersonalSmartFolder",
    name       => "smartfolder1" );
# find the desired object from the retrieved list.
my $desired_smart_folder = $retrieved_objs[0];
# Submit for removal
my $response = $session->remove( $desired_smart_folder );
```

Infoblox::Session->search( )

Use this method to search for Personal Smart Folder objects in the Infoblox appliance. See Infoblox::Session->search() for parameters and return values.

Key References

```
Apply the following attributes to search for a specific PersonalSmartFolder object:

 name      - Optional. PersonalSmartFolder name in string format (regular expression).
 comment   - Optional. Comment in string format (regular expression).
```

Example

```
# search for all PersonalSmartFolder objects that start with "smartfolder"
my @retrieved_objs = $session->search(
    object => "Infoblox::Grid::PersonalSmartFolder",
    name   => "smartfolder.*" );
```

METHODS

This section describes all the methods that you can use to set and retrieve the attribute values of a PersonalSmartFolder object.

comment( )

Use this method to set or retrieve the comment.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

Pertinent information about the personal smart folder. Comment in string format with a maximum of 256 bytes. The default value is undefined.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get comment
my $comment = $smart_folder->comment();
#Modify comment
$smart_folder->comment("This is the modified comment for smartfolder1");
```

group_bys( )

Use this method to set or retrieve the group_bys value.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

The valid value is an array reference that contains Infoblox::Grid::SmartFolder::GroupBy object(s). The default value is undefined.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
my $group_by_state = Infoblox::Grid::SmartFolder::GroupBy->new(
                                                              enable_grouping => 'true',
                                                              value => "State",
                                                              value_type => "EXTATTR",
                                                             );

#Get group_by value
my $group_by = $smart_folder->group_by();
#Modify group_by value
$smart_folder->group_bys([$group_by_state]);
```

name( )

Use this method to set or retrieve the name of a personal smart folder.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

Text with the name of the personal smart folder.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get name
my $name = $smart_folder->name();
```

query_items( )

Use this method to set or retrieve the query items. If query items are not specified when you `add()` a smart folder, the server automatically assigns a default query item for the smart folder. This default query item is "type=Network/Zone/Range/Member". This ensures that objects are included in searches for these 4 types, by default. When you `modify()` a smart folder, the server keeps the query items provided by the client and does not add the default items.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

The valid value is an array reference that contains Infoblox::Grid::SmartFolder::QueryItem object(s). The default value is undefined.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get query items
my $query_items = $smart_folder->query_items();

#Modify query items
my $query_item1 = Infoblox::Grid::SmartFolder::QueryItem->new(
    name                => "location",
    operator            => "eq",
    value               => "Santa Clara",
);
$smart_folder->query_items([$query_items1]);
```

SAMPLE CODE

The following sample code demonstrates the different functions that can be applied to an object, such as add, search, modify, and remove. This sample also includes error handling for the operations.

#Preparation prior to an PersonalSmartFolder object insertion

```
#PROGRAM STARTS: Include all the modules that will be used
use strict;
use Infoblox;

#Create a session to the Infoblox appliance

my $session = Infoblox::Session->new(
              master   => "192.168.1.2", #appliance host ip
              username => "admin",       #appliance user login
              password => "infoblox"     #appliance password
              );

unless ($session) {
       die("Construct session failed: ",
              Infoblox::status_code() . ":" . Infoblox::status_detail());
}
print "Session created successfully\n";

# Add a network with extensible attributes

my %extensible_attributes=(
    'Region' => 'Santa Clara County',
    'Country' => 'USA',
    'State' => 'California',
    'Site' => 'Santa Clara',
    'Building' => 'HQ',
    'VLAN' => '1',
);

my $network = Infoblox::DHCP::Network->new(
    network => "10.0.0.0/24",
    extensible_attributes => \%extensible_attributes,
);
unless ($network) {
    die("Construct Network failed: ",
        Infoblox::status_code() . ":" . Infoblox::status_detail());
}
print "Network created successfully\n";

$session->add($network)
    or die("Add Network object failed: ",
    $session->status_code() . ":" . $session->status_detail());
print "Network object added to server successfully\n";
```

#Create a PersonalSmartFolder object

```
my $query_item = Infoblox::Grid::SmartFolder::QueryItem->new(
    name                    => "Country",
    is_extensible_attribute => "true",
    operator                => "eq",
    value                   => "USA",
);

unless ($query_item) {
    die("Construct query item object failed: ",
        Infoblox::status_code() . ":" . Infoblox::status_detail());
}
print "Smart folder query item object created successfully\n";

my $smart_folder = Infoblox::Grid::PersonalSmartFolder->new(
    name                    => "my_folder_personal",
    comment                 => "my personal folder",
    query_items             => [$query_item],
);

unless ($smart_folder) {
    die("Construct personal smart folder object failed: ",
        Infoblox::status_code() . ":" . Infoblox::status_detail());
}
print "Smart folder object created successfully\n";

#Add the PersonalSmartFolder object to the Infoblox appliance through a session
$session->add($smart_folder)
    or die("Add PersonalSmartFolder object failed: ",
    $session->status_code() . ":" . $session->status_detail());
print "PersonalSmartFolder object added to server successfully\n";
```

#Search for a PersonalSmartFolder

```
my @retrieved_objs = $session->search(
    object => "Infoblox::Grid::PersonalSmartFolder",
    name   => "my_folder.*"
);
my $object = $retrieved_objs[0];

unless ($object) {
    die("Search for a PersonalSmartFolder object failed: ",
        $session->status_code() . ":" . $session->status_detail());
}
print "Search for a PersonalSmartFolder object found at least 1 matching entry\n";
```

#Get and modify a PersonalSmartFolder object

```
#Get the PersonalSmartFolder object from Infoblox appliance through a session
my @retrieved_objs = $session->get(
    object => "Infoblox::Grid::PersonalSmartFolder",
    name   => "my_folder_personal"
);
my $object = $retrieved_objs[0];

unless ($object) {
    die("Get PersonalSmartFolder object failed: ",
        $session->status_code() . ":" . $session->status_detail());
}
print "Get PersonalSmartFolder object found at least 1 matching entry\n";

#Modify the PersonalSmartFolder object
$object->name("my_folder_personal_renamed");
$object->comment("modified personal smart folder");

#Apply the changes.
$session->modify($object)
    or die("Modify PersonalSmartFolder object failed: ",
    $session->status_code() . ":" . $session->status_detail());
print "PersonalSmartFolder object modified successfully \n";
```

#Remove a PersonalSmartFolder object

```
#Get the PersonalSmartFolder object through the session
my @retrieved_objs = $session->get(
    object => "Infoblox::Grid::PersonalSmartFolder",
    name   => "my_folder_personal_renamed"
```

```
);
my $object = $retrieved_objs[0];
unless ($object) {
    die("Get PersonalSmartFolder object failed: ",
        $session->status_code() . ":" . $session->status_detail());
}
print "Get PersonalSmartFolder object found at least 1 matching entry\n";

#Submit the object for removal
$session->remove($object)
    or die("Remove PersonalSmartFolder object failed: ",
    $session->status_code() . ":" . $session->status_detail());
print "PersonalSmartFolder object removed successfully \n";
```

#Cleanup

```
#Get the Network object through the session
my @retrieved_objs = $session->get(
    object  => "Infoblox::DHCP::Network",
    network => "10.0.0.0/24"
);
my $object = $retrieved_objs[0];
unless ($object) {
    die("Get Network object failed: ",
        $session->status_code() . ":" . $session->status_detail());
}
print "Get Network object found at least 1 matching entry\n";

#Submit the object for removal
$session->remove($object)
    or die("Remove Network object failed: ",
    $session->status_code() . ":" . $session->status_detail());
print "Network object removed successfully \n";

####PROGRAM ENDS####
```

AUTHOR

Infoblox Inc. http://www.infoblox.com/

SEE ALSO

Infoblox::Grid::SmartFolderChildren, Infoblox::Grid::PersonalSmartFolder, Infoblox::Session->add(), Infoblox::Session->get(), Infoblox::Session->modify(), Infoblox::Session->remove(), Infoblox::Session->search(),Infoblox::Session

---

COPYRIGHT

Copyright (c) 2017 Infoblox Inc.

Infoblox::Grid::HSM::Thales::Group - Thales HSM Group object

- NAME
- DESCRIPTION
- CONSTRUCTOR
- SESSION METHODS
  - Infoblox::Session->add( )
  - Infoblox::Session->get( )
  - Infoblox::Session->modify( )
  - Infoblox::Session->remove( )
- METHODS
  - card_name( )
  - comment( )
  - key_server_ip( )
  - key_server_port( )
  - name( )
  - pass_phrase( )
  - protection( )
  - thales_hsm( )
  - status( )
- SAMPLE CODE
- AUTHOR
- SEE ALSO
- COPYRIGHT

NAME

Infoblox::Grid::HSM::Thales::Group - Thales HSM Group object

DESCRIPTION

Hardware Security Modules (HSMs) are used for secure private key storage, and key generation and zone-signing offloading.

CONSTRUCTOR

```
 my $hsm_group = Infoblox::Grid::HSM::Thales::Group->new(
     key_server_ip           => $ipv4addr,                                   #Required
     name                    => $string,                                     #Required
     thales_hsm              => [$thales1, ...],                             #Required
     card_name               => $string,                                     #Required if
"protection" is "SOFTCARD"
     pass_phrase             => $string,                                     #Required if
"protection" is "SOFTCARD"
     comment                 => $string,                                     #Optional / Default
value is empty
     key_server_port         => $num,                                        #Optional / Default
value is 9004
     protection              => "MODULE" | "SOFTCARD" ,                      #Optional / Default
value is Module
 );
```

SESSION METHODS

This section describes all the methods in an Infoblox::Session module that you can apply to an HSM Group object.

Infoblox::Session->add( )

Use this method to add an object to the Infoblox appliance. See Infoblox::Session->add() for parameters and return values.

Example

```
#Construct an object
my $hsm_group = Infoblox::Grid::HSM::Thales::Group->new(
    name                    => "HSM Group",
    key_server_ip           => "10.0.0.1",
    key_server_port         => 9004,
    comment                 => "A new HSM Group ",
    thales_hsm              => [$thales1]
    );

#Submit for addition
```

```
my $response = $session->add($hsm_group);
```

Infoblox::Session->get( )

Use this method to retrieve all the matching objects from the Infoblox appliance. See Infoblox::Session->get() for parameters and return values.

Example

```
my @retrieved_objs = $session->get(
    object => "Infoblox::Grid::HSM::Thales::Group",
    key_server_ip   => "10.0.0.1");
```

Infoblox::Session->modify( )

Use this method to modify an object in the Infoblox appliance. See Infoblox::Session->modify() for parameters and return values.

Example

```
# Use this method to modify the comment of an HSM Group object.
$hsm_group->comment("This is modified HSM Group object");
# Submit modification
my $response = $session->modify($hsm_group );
```

Infoblox::Session->remove( )

Use this method to remove an object from the Infoblox appliance. See Infoblox::Session->remove() for parameters and return values.

To remove a specific object, first use `get()` to retrieve the object, and then submit it for removal.

Example

```
#Get the objects with the specified key_server_ip.
my @retrieved_objs = $session->get(
    object => "Infoblox::Grid::HSM::Thales::Group",
    key_server_ip   => "10.0.0.1");

#Find the desired object on the retrieved list.
my $desired_hsmgroup = $retrieved_objs[0];

# Submit for removal
my $response = $session->remove($desired_hsmgroup);
```

METHODS

This section describes all the methods that you can use to configure and retrieve the attribute values of an HSM Group object.

card_name( )

Use this method to set or retrieve the name of the softcard used by the HSM Group object.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

Desired name for the softcard in string format.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get softcard name
my $card_name = $hsm_group->card_name();
#Modify softcard name
$hsm_group->card_name("Softcard 1")
```

comment( )

Use this method to set or retrieve a descriptive comment.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

Desired comment in string format with a maximum of 256 bytes.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get comment
my $comment = $hsm_group->comment();
#Modify comment
$hsm_group->comment("Modified HSM Group");
```

key_server_ip( )

Use this method to set or retrieve the key file server IP address of an HSM Group object.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

The IPv4 address of the key file server.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get key server IP address
my $key_server_ip = $hsm_group->key_server_ip();
#Modify key server IP address
$hsm_group->key_server_ip("10.0.0.5");
```

key_server_port( )

Use this method to set or retrieve the port number used by the key file server configured in the HSM Group object.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

Valid value is an unsigned integer ranging from 0 to 65535.The default value is 9004.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get key server port
my $key_server_port = $hsm_group->key_server_port();
#Modify key server port
$hsm_group->key_server_port(9012);
```

name( )

Use this method to set or retrieve the name of the HSM Group object.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

Desired name for the HSM Group object in string format.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get name
my $name = $hsm_group->name();
#Modify name
$hsm_group->name("HSM GROUP 2")
```

pass_phrase( )

Use this method to set the passphrase that is used to unlock the key store. This is a write-only attribute.

Include the specified parameter to set the attribute value.

Parameter

The valid value is a string, which cannot include a tab and carriage return.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

Example

```
#Modify passphrase
$hsm_group->pass_phrase("pass1234");
```

protection( )

Use this method to set or retrieve the protection type of an HSM Group object.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

The valid values are "MODULE" and "SOFTCARD".The default value is "MODULE".

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get protection
my $protection = $hsm_group->protection();
#Modify protection
$grid_member->protection("SOFTCARD");
```

thales_hsm( )

Use this method to set or retrieve the list of Thales servers in an HSM Group object.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

The valid value is an array reference that contains Infoblox::Grid::HSM::Thales object(s).

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get Thales HSM list
my $thales_hsm = $hsm_group->thales_hsm();
#Modify Thales HSM list
my $thales_hsm1 = Infoblox::Grid::HSM::Thales->new(
    remote_ip              => "20.0.0.1",
    keyhash                => "a123",
);

my $thales_hsm2 = Infoblox::Grid::HSM::Thales->new(
    remote_ip              => "20.0.0.5",
    keyhash                => "b456",
```

```
);

$hsm_group->thales_server([$thales_hsm1,$thales_hsm2]);
```

status( )

Use this method to retrieve the status of the remote file server used in the HSM Group object.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Returns

The method returns the attribute value of "UP" or "DOWN" .

Example

```
#Get status
my $status = $hsm_group->status();
```

SAMPLE CODE

The following sample code demonstrates the different functions that can be applied to an object such as add, get, modify and remove. This sample also includes error handling for the operations.

#Preparation prior to getting and modifying a grid member object

```
use strict;
use Infoblox;

#refers to Infoblox Appliance IP address
my $host_ip = "192.168.1.2";

#Create a session to the Infoblox appliance

my $session = Infoblox::Session->new(
    master   => $host_ip,
    username => "admin",
    password => "infoblox"
);
unless ($session) {
       die("Construct session failed: ",
               Infoblox::status_code() . ":" . Infoblox::status_detail());
}
print "Session created successfully\n";
```

#Create an HSM Group object

```
my $thales_server = Infoblox::Grid::HSM::Thales->new(
    remote_ip     => "20.0.0.1",
    keyhash       => "1234hash",
    remote_port   => 9012,
    );
unless ($thales_server) {
       die("Construct thales server object failed: ",
               Infoblox::status_code() . ":" . Infoblox::status_detail());
}
print "Thales server object created successfully\n";

my $hsm_group = Infoblox::Grid::HSM::Thales::Group->new(
    key_server_ip     => "20.0.0.1",
    key_server_port   => 9007,
    );
unless ($hsm_group) {
       die("Construct HSM Group object failed: ",
               Infoblox::status_code() . ":" . Infoblox::status_detail());
}
print "HSM Group object created successfully\n";

#Add the HSM object to appliance through session
my $response = $session->add( $hsm_group );

unless ($response) {
    die("Add HSM Group failed: ",
           $session->status_code() . ":" . $session->status_detail());
}
print"HSM Group added successfully \n";
```

#Get and modify the HSM Group object

```
#Get HSM Group through session
my @retrieved_objs = $session->get(
    object => "Infoblox::Grid::HSM::Thales::Group",
    key_server_ip     => "20.0.0.1",
    );
my $hsm_group = $retrieved_objs[0];

unless ($hsm_group) {
    die("Get HSM Group object failed: ",
           $session->status_code() . ":" . $session->status_detail());
}
print"Get HSM Group successful \n";

# Modify one of the attributes of the selected HSM Group.

$hsm group->comment("Modified hsm group object");

#Applying the changes
$session->modify($hsm_group)
   or die("Modify HSM Group failed",
            $session->status_code() . ":" . $session->status_detail());
print"HSM Group modified successfully \n";
```

#Remove the grid member object

```
#Get HSM Group through session
my @retrieved_objs = $session->get(
    object => "Infoblox::Grid::HSM::Thales::Group",
    key_server_ip     => "20.0.0.1"
    );
my $desired_hsm_group = $retrieved_objs[0];

unless ($desired_hsm_group) {
    die("Get HSM Group object failed: ",
           $session->status_code() . ":" . $session->status_detail());
}
print"Get HSM Group successful \n";

#Submit the object for removal
$session->remove( $desired_hsm_group )
    or die("Remove HSM Group failed",
           $session->status_code() . ":" . $session->status_detail());
print"HSM Group removed successfully \n";

####PROGRAM ENDS####
```

AUTHOR

Infoblox Inc. http://www.infoblox.com/

SEE ALSO

Infoblox::Grid::HSM::Thales

COPYRIGHT

Copyright (c) 2017 Infoblox Inc.

Infoblox::Grid::HSM::SafeNet::Group - SafeNet HSM Group object

- NAME
- DESCRIPTION
- CONSTRUCTOR
- SESSION METHODS
  - Infoblox::Session->add( )
  - Infoblox::Session->get( )
  - Infoblox::Session->modify( )
  - Infoblox::Session->remove( )
  - Infoblox::Session->search( )
- METHODS
  - comment( )
  - name( )
  - passphrase( )
  - hsm_servers( )
  - hsm_version( )
  - serial_number( )
  - status( )
- SAMPLE CODE
- AUTHOR
- SEE ALSO
- COPYRIGHT

NAME

Infoblox::Grid::HSM::SafeNet::Group - SafeNet HSM (Hardware Security Module) Group object

---

DESCRIPTION

Hardware Security Modules (HSMs) are used for secure private key storage, and key generation and zone-signing offloading.

CONSTRUCTOR

```
 my $hsm_group = Infoblox::Grid::HSM::SafeNet::Group->new(
     name                    => $string,                                  #Required
     passphrase              => $string,                                  #Required
     hsm_servers             => [$safenet1, ...],                         #Required
     hsm_version             => "LunaSA_4" | "LunaSA_5" | "LunaSA_6",     #Required
     comment                 => $string,                                  #Optional / Default
value is empty
 );
```

SESSION METHODS

This section describes all the methods in an Infoblox::Session module that you can apply to a SafeNet HSM Group object.

Infoblox::Session->add( )

Use this method to add an object to the Infoblox appliance. See Infoblox::Session->add() for parameters and return values.

Example

```
#Construct an object
my $hsm_group = Infoblox::Grid::HSM::SafeNet::Group->new(
    name                    => "SafeNetGroup",
    passphrase              => "PassPhrase",
    comment                 => "This is a test SafeNet HSM Group",
    hsm_servers             => [$safenet1],
    hsm_version             => "LunaSA_4",
    );

#Submit for addition
my $response = $session->add($hsm_group);
```

Infoblox::Session->get( )

Use this method to retrieve all the matching objects from the Infoblox appliance. See Infoblox::Session->get() for parameters and return values.

Example

```
my @retrieved_objs = $session->get(
    object => "Infoblox::Grid::HSM::SafeNet::Group",
    name   => "SafeNetGroup");
```

Infoblox::Session->modify( )

Use this method to modify an object in the Infoblox appliance. See Infoblox::Session->modify() for parameters and return values.

Example

```
# Use this method to modify the comment of a SafeNet HSM Group object.
$hsm_group->comment("This is modified SafeNet HSM Group object");

# Submit modification
my $response = $session->modify($hsm_group);
```

Infoblox::Session->remove( )

Use this method to remove an object from the Infoblox appliance. See Infoblox::Session->remove() for parameters and return values.

To remove a specific object, first use `get()` to retrieve the object, and then submit it for removal.

Example

```
#Get the objects with the specified name.
my @retrieved_objs = $session->get(
    object => "Infoblox::Grid::HSM::SafeNet::Group",
    name   => "SafeNetGroup");

#Find the desired object on the retrieved list.
my $desired_hsmgroup = $retrieved_objs[0];

# Submit for removal
my $response = $session->remove($desired_hsmgroup);
```

Infoblox::Session->search( )

Use this method to retrieve all the matching objects from the Infoblox appliance. See Infoblox::Session->search() for parameters and return values.

Example

```
my @retrieved_objs = $session->search(
    object => "Infoblox::Grid::HSM::SafeNet::Group",
    name   => "SafeNetGroup");
```

METHODS

This section describes all the methods that you can use to configure and retrieve the attribute values of an HSM Group object.

comment( )

Use this method to set or retrieve a descriptive comment.

Parameter

Desired comment in string format with a maximum of 256 bytes.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get comment
my $comment = $hsm_group->comment();

#Modify comment
$hsm_group->comment("Modified comment");
```

name( )

Use this method to set or retrieve the name of the SafeNet HSM Group object.

Parameter

Desired name for the SafeNet HSM Group object in string format.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get name
my $name = $hsm_group->name();

#Modify name
$hsm_group->name("SafeNetGroup2")
```

passphrase( )

Use this method to set the passphrase that is used to unlock the key store. This is a write-only attribute.

Parameter

The valid value is a string, which cannot include a tab and carriage return.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

Example

```
#Modify passphrase
$hsm_group->passphrase("pass1234");
```

hsm_servers( )

Use this method to set or retrieve the list of SafeNet servers in a SafeNet HSM Group object.

Parameter

The valid value is an array reference that contains Infoblox::Grid::HSM::SafeNet object(s).

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get SafeNet HSM servers list
my $hsm_servers = $hsm_group->hsm_servers();

#Modify SafeNet HSM servers list
my $safenet_server1 = Infoblox::Grid::HSM::SafeNet->new(
   name               => 'server1',
   partition_serial   => 123,
   server_certificate => '/tmp/safenet/cert1.pem',
);
my $safenet_server2 = Infoblox::Grid::HSM::SafeNet->new(
   name               => 'server2',
   partition_serial   => 456,
   server_certificate => '/tmp/safenet/cert2.pem',
);

$hsm_group->hsm_servers([$safenet_server1,$safenet_server2]);
```

hsm_version( )

Use this method to set or retrieve the HSM version of the SafeNet HSM Group.

Parameter

The value can be either "LunaSA_4", "LunaSA_5" or "LunaSA_6".

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value of "LunaSA_4", "LunaSA_5" or "LunaSA_6".

Example

```
#Get hsm_version
my $hsm_version = $hsm_group->hsm_version();

#Modify hsm_version
$hsm_version->hsm_version("LunaSA_4");
```

serial_number( )

Use this method to retrieve the serial number of the SafeNet HSM Group. This attribute is read-only and cannot be modified.

Parameter

None

Returns

The method returns the attribute value.

Example

```
#Get serial_number
my $serial_number = $hsm_group->serial_number();
```

status( )

Use this method to retrieve the status of the SafeNet HSM Group. This attribute is read-only and cannot be modified.

Parameter

None

Returns

The method returns the attribute value of "UP" or "DOWN".

Example

```
#Get status
my $status = $hsm_group->status();
```

SAMPLE CODE

The following sample code demonstrates the different functions that can be applied to an object such as add, get, modify and remove. This sample also includes error handling for the operations.

#Preparation prior to getting and modifying a grid member object

```
use strict;
use Infoblox;

#refers to Infoblox Appliance IP address
my $host_ip = "192.168.1.2";

#Create a session to the Infoblox appliance

my $session = Infoblox::Session->new(
    master   => $host_ip,
    username => "admin",
    password => "infoblox"
);
unless ($session) {
       die("Construct session failed: ",
               Infoblox::status_code() . ":" . Infoblox::status_detail());
}
print "Session created successfully\n";
```

#Create a SafeNet HSM Group object

```
my $safenet_server = Infoblox::Grid::HSM::SafeNet->new(
```

```
    name                => "20.0.0.1",
    partition_serial    => 1234,
    server_certificate  => '/tmp/safenet/cert.pem',
    );
unless (safenet_server) {
       die("Construct safenet server object failed: ",
               Infoblox::status_code() . ":" . Infoblox::status_detail());
}
print "SafeNet server object created successfully\n";

my $hsm_group = Infoblox::Grid::HSM::SafeNet::Group->new(
    name         => "SafeNet Group",
    passphrase   => 'pass',
    hsm_servers  => [safenet_server],
    hsm_version  => "LunaSA_4",
    );
unless ($hsm_group) {
       die("Construct SafeNet HSM Group object failed: ",
               Infoblox::status_code() . ":" . Infoblox::status_detail());
}
print "SafeNet HSM Group object created successfully\n";

#Add the SafeNet HSM object to appliance through session
my $response = $session->add($hsm_group);

unless ($response) {
    die("Add SafeNet HSM Group failed: ",
           $session->status_code() . ":" . $session->status_detail());
}
print "SafeNet HSM Group added successfully \n";
```

#Get and modify the SafeNet HSM Group object

```
#Get SafeNet HSM Group through session
my @retrieved_objs = $session->get(
    object   => "Infoblox::Grid::HSM::SafeNet::Group",
    name     => "SafeNet Group",
    );
my $hsm_group = $retrieved_objs[0];

unless ($hsm_group) {
    die("Get SafeNet HSM Group object failed: ",
           $session->status_code() . ":" . $session->status_detail());
}
print"Get SafeNet HSM Group successful \n";

# Modify one of the attributes of the selected SafeNet HSM Group.

$hsm group->comment("Modified hsm group object");

#Applying the changes
$session->modify($hsm_group)
   or die("Modify SafeNet HSM Group failed",
            $session->status_code() . ":" . $session->status_detail());
print "SafeNet HSM Group modified successfully \n";
```

#Remove the grid member object

```
#Get SafeNet HSM Group through session
my @retrieved_objs = $session->get(
    object => "Infoblox::Grid::HSM::SafeNet::Group",
    name   => "SafeNet Group",
    );
my $desired_hsm_group = $retrieved_objs[0];

unless ($desired_hsm_group) {
    die("Get SafeNet HSM Group object failed: ",
           $session->status_code() . ":" . $session->status_detail());
}
print "Get SafeNet HSM Group successful \n";

#Submit the object for removal
$session->remove($desired_hsm_group)
    or die("Remove SafeNet HSM Group failed",
           $session->status_code() . ":" . $session->status_detail());
print "SafeNet HSM Group removed successfully \n";
```

```
####PROGRAM ENDS####
```

AUTHOR

Infoblox Inc. http://www.infoblox.com/

SEE ALSO

Infoblox::Grid::HSM::SafeNet

COPYRIGHT

Copyright (c) 2017 Infoblox Inc.

Infoblox::Grid::HSM::SafeNet - SafeNet HSM Server object

- NAME
- DESCRIPTION
- CONSTRUCTOR
- MODULE METHODS
  - Infoblox::Grid::HSM::SafeNet::Group->hsm_servers( )
- METHODS
  - disabled( )
  - is_fips_compliant( )
  - name( )
  - partition_capacity( )
  - partition_id( )
  - partition_serial( )
  - server_certificate( )
  - status( )
- AUTHOR
- SEE ALSO
- COPYRIGHT

NAME

Infoblox::Grid::HSM::SafeNet - SafeNet HSM (Hardware Security Module) Server object

DESCRIPTION

An Infoblox::Grid::HSM::SafeNet object represents a server configured in the SafeNet HSM group.

CONSTRUCTOR

```
my $hsm_server = Infoblox::Grid::HSM::SafeNet->new(
    name                => $string,                 # Required
    partition_serial    => $uint,                   # Required
    server_certificate  => $string,                 # Required
    disabled            => "true" | "false",        # Optional / Default value is "false"
);
```

MODULE METHODS

The following functions can be applied to a SafeNet HSM Server object.

Infoblox::Grid::HSM::SafeNet::Group->hsm_servers( )

Use this function to add or retrieve SafeNet HSM servers in the SafeNet HSM group. See Infoblox::Grid::HSM::SafeNet::Group->hsm_servers() for parameters and return values.

Example

```
#Construct a SafeNet HSM Group
my $hsm_group = Infoblox::Grid::HSM::SafeNet::Group->new(
    name          => 'SafeNetGroup',
    passphrase    => 'PassPhrase',
    hsm_servers   => [],
    hsm_version   => "LunaSA_4",
);

# Configure the SafeNet HSM server in the SafeNet HSM Group object.
my $response = $hsm_group->hsm_servers([$hsm_server]);
```

METHODS

This section describes all the methods that can be used to configure and retrieve the attribute values of an Infoblox::Grid::HSM::SafeNet object.

disabled( )

Use this method to set or retrieve the disabled setting.

Parameter

Specify "true" to disable the SafeNet HSM Server object or "false" to enable it. Default value is "false".

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value as "true" or "false".

Example

```
#Get disabled flag
my $disabled = $hsm_server->disabled();

#Modify disabled flag
$hsm_server->disabled("true");
```

is_fips_compliant( )

Use this method to retrieve the FIPS compliant mode status. This attribute is read-only and cannot be modified.

Parameter

None

Returns

The method returns the attribute value as "true" or "false".

Example

```
#Get is_fips_compliant flag
my $is_fips_compliant = $hsm_server->is_fips_compliant();
```

name( )

Use this method to set or retrieve the IP address or the domain name for the HSM.

Parameter

String with the IP address or the domain name for the HSM.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get name
my $name = $hsm_server->name();

#Modify name
$hsm_server->name("SafeNet");
```

partition_capacity( )

Use this method to retrieve the partition capacity percentage used. This attribute is read-only and cannot be modified.

Parameter

None

Returns

The method returns the attribute value.

Example

```
#Get partition_capacity
my $partition_capacity = $hsm_server->partition_capacity();
```

partition_id( )

Use this method to retrieve the partition label on the HSM. This attribute is read-only and cannot be modified.

Parameter

None

Returns

The method returns the attribute value.

Example

```
#Get partition_id
my $partition_id = $hsm_server->partition_id();
```

partition_serial( )

Use this method to set or retrieve the serial number of the partition on the HSM.

Parameter

An unsigned integer.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get partition_serial
my $partition_serial = $hsm_server->partition_serial();

#Modify partition_serial
$hsm_server->partition_serial(12345);
```

server_certificate( )

Use this method to generate the server certificate of the SafeNet HSM server. This attribute is write-only and cannot be retrieved.

Parameter

String with the content of the server certificate or a reference to a file.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

Example

```
#Set server_certificate as string
$hsm_server->server_certificate($cert);

#Set server_certificate as reference to a file
$hsm_server->server_certificate('/tmp/safenet/cert.pem');
```

status( )

Use this method to retrieve the status of the SafeNet HSM server. This attribute is read-only and cannot be modified.

Parameter

None

Returns

The method returns the attribute value as "UP" or "DOWN".

Example

```
#Get status
my $status = $hsm_server->status();
```

AUTHOR

Infoblox Inc. http://www.infoblox.com/

SEE ALSO

Infoblox::Grid::HSM::SafeNet::Group

COPYRIGHT

Copyright (c) 2017 Infoblox Inc.

Infoblox::Grid::HSM::Thales - Thales HSM Server object

- NAME
- DESCRIPTION
- CONSTRUCTOR
- MODULE METHODS
  - Infoblox::Grid::HSM::Thales::Group->thales_hsm( )
- METHODS
  - disabled( )
  - keyhash( )
  - remote_ip( )
  - remote_port( )
  - remote_esn( )
  - status( )
- AUTHOR
- SEE ALSO
- COPYRIGHT

NAME

Infoblox::Grid::HSM::Thales - Thales HSM (Hardware Security Module) Server object

---

DESCRIPTION

An Infoblox::Grid::HSM::Thales object represents a server configured in the HSM group.

CONSTRUCTOR

```
my $hsm_server = Infoblox::Grid::HSM::Thales->new(
    remote_ip       => $ipv4addr,                   # Required
    keyhash         => $string,                     # Required
    disabled        => "true" | "false",            # Optional / Default value is "false"
    remote_port     => $num,                        # Optional / Default value is 9004
);
```

MODULE METHODS

The following functions can be applied to an HSM Server object.

Infoblox::Grid::HSM::Thales::Group->thales_hsm( )

Use this function to add or retrieve HSM servers in the HSM group. See Infoblox::Grid::HSM::Thales::Group->thales_hsm() for parameters and return values.

Example

```
#Construct an HSM Group
my $hsm_group = Infoblox::Grid::HSM::Thales::Group->new(
    name                    => "HSM Group",
    key_server_ip           => "10.0.0.1",
    key_server_port         => 9004,
    comment                 => "A new HSM Group ",
    thales_hsm              => []
);

# Configure the HSM server in the HSM Group object.
my $response = $hsm_group->thales_hsm([$thales_server]);
```

METHODS

This section describes all the methods that can be used to configure and retrieve the attribute values of an Infoblox::Grid::HSM::Thales object.

disabled( )

Use this method to set or retrieve the disabled setting.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

Specify "true" to disable the HSM Server object or "false" to enable it. Default value is "false".

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get disabled flag
my $disabled = $thales_server->disabled();
#Modify disabled flag
$thales_server->disabled("true");
```

keyhash( )

Use this method to set or retrieve the key hash used to authenticate the HSM.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

Desired key hash in string format.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get key hash
my $key = $thales_server->keyhash();
#Modify key hash
$thales_server->keyhash("123hashxyz");
```

remote_ip( )

Use this method to set or retrieve the IP address of the Thales HSM server.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

The IPv4 address of the Thales HSM server.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get the Thales HSM server IP address
my $remote_ip = $thales_server->remote_ip();
#Modify the Thales HSM server IP address
$thales_server->remote_ip("20.0.0.9");
```

remote_port( )

Use this method to set or retrieve the remote port setting of the Thales HSM server.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

Valid values are between 1 and 65535.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get remote port
my $remote_port = $thales_server->remote_port();
#Modify remote port
$thales_server->remote_port(9012);
```

remote_esn( )

Use this method to retrieve the Electronic Serial Number (ESN) of the Thales HSM server.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Returns

The method returns the attribute value.

Example

```
#Get remote ESN
my $remote_esn= $thales_server->remote_esn();
```

status( )

Use this method to retrieve the status of the Thales HSM server.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Returns

The method returns the attribute value as "UP" or "DOWN".

Example

```
#Get status
my $status= $thales_server->status();
```

AUTHOR

Infoblox Inc. http://www.infoblox.com/

SEE ALSO

Infoblox::Grid::HSM::Thales::Group

COPYRIGHT

Copyright (c) 2017 Infoblox Inc.

Infoblox::Grid::LicenseSubPool - a license subpool object.

- NAME
- DESCRIPTION
- CONSTRUCTOR
- SESSION METHODS
- MODULE METHODS
  - Infoblox::Grid::LicensePool->subpools( )
- METHODS
  - expiration( )
  - installed( )
  - key( )
- AUTHOR
- SEE ALSO
- COPYRIGHT

NAME

Infoblox::Grid::LicenseSubPool - a license subpool object.

DESCRIPTION

The license subpool object shows the license installments that have the same expiry date.

CONSTRUCTOR

The Infoblox::Grid::LicenseSubPool object is a read-only object and does not require manual construction.

SESSION METHODS

The object does not support any session methods.

MODULE METHODS

Infoblox::Grid::LicensePool->subpools( )

Use this method to retrieve the license pool subpools. For parameters and return values see Infoblox::Grid::LicensePool->subpools.

Example

```
#Get subpools value
my $value = $object->subpools();
```

METHODS

This section describes all the methods that you can use to set or retrieve the attribute values of a license subpool object.

expiration( )

Use this method to retrieve the license subpool expiration timestamp. This is a read-only attribute.

Parameter

None

Returns

The method returns the number of seconds elapsed since January 1, 1970, in UTC format.

Example

```
#Get expiration value
my $expiration = $object->expiration();
```

installed( )

Use this method to retrive the total number of dynamic licenses allowed for this license sub pool. This is a read-only attribute.

Parameter

None

Returns

The method returns the attribute value.

Example

```
#Get installed value
my $value = $object->installed();
```

key( )

Use this method to retrieve the license string for the license subpool. This is a read-only attribute.

Parameter

None

Returns

The method returns the attribute value.

Example

```
#Get key value
my $value = $object->key();
```

AUTHOR

Infoblox Inc. http://www.infoblox.com/

SEE ALSO

Infoblox::Session, Infoblox::Grid::LicensePool, Infoblox::Grid::LicensePool->subpools,

---

COPYRIGHT

Copyright (c) 2017 Infoblox Inc.

Infoblox::Grid::LicensePool - License Pool object.

- NAME
- DESCRIPTION
- CONSTRUCTOR
- SESSION METHODS
  - Infoblox::Session->get( )
  - Infoblox::Session->search( )
- METHODS
  - assigned( )
  - expiration( )
  - expiration_status( )
  - installed( )
  - key( )
  - limit( )
  - limit_context( )
  - model( )
  - subpools( )
  - temp_assigned( )
  - type( )
- SAMPLE CODE
- AUTHOR
- SEE ALSO
- COPYRIGHT

NAME

Infoblox::Grid::LicensePool - License Pool object.

DESCRIPTION

A license pool is a container associated with the Grid and holds dynamic licenses for a specific feature. Licenses in the pool can be dynamically allocated to and deallocated from Grid members. When not in use, dynamic licenses are released back to the pool for future allocation. There is no expiration for dynamic licenses.

CONSTRUCTOR

The Infoblox::Grid::LicensePool object is a read-only object and does not require manual construction.

SESSION METHODS

This section describes all the methods in the Infoblox::Session module that you can apply to a License Pool object.

Infoblox::Session->get( )

Use this method to retrieve the existing objects from an Infoblox appliance. See Infoblox::Session->get() for parameters and return values.

Example

```
my $retrieved_objs = $session->get(object => "Infoblox::Grid::LicensePool");
```

Infoblox::Session->search( )

Use this method to retrieve the existing objects from an Infoblox appliance. See Infoblox::Session->search() for parameters and return values.

Key Reference

```
Apply the following attributes to search the License Pool object(s):

    "type"  - Optional. The license type.
    "limit" - Optional. The limitation of dynamic license.
    "model" - Optional. The supported vNIOS virtual applaince model.
    "key"   - Optional. The license string.
```

Example

```
my @retrieved_objs = $session->search(
    object => "Infoblox::Grid::LicensePool",
```

```
        type    => "DNS"
);
```

METHODS

This section describes all the methods that you can use to set or retrieve the attribute values of a License Pool object.

assigned( )

Use this method to retrieve the number of dynamic licenses allocated to vNIOS appliances. This is a read-only attribute.

Parameter

None

Returns

The method returns the attribute value.

Example

```
#Get assigned value
my $value = $object->assigned();
```

expiration( )

Use this method to retrieve the license pool expiration timestamp. This is a read-only attribute.

Parameter

None

Returns

The method returns the number of seconds elapsed since January 1, 1970, in UTC format.

Example

```
#Get expiration value
my $expiration = $object->expiration();
```

expiration_status( )

Use this method to retrieve the license pool expiration status. This is a read-only attribute.

Parameter

None

Returns

The method returns the attribute value.

Example

```
#Get expiration_status value
my $expiration_status = $object->expiration_status();
```

installed( )

Use this method to retrieve the total number of dynamic licenses allowed for this license pool. This is a read-only attribute.

Parameter

None

Returns

The method returns the attribute value.

Example

```
#Get installed value
my $value = $object->installed();
```

key( )

Use this method to retrieve the license string for the license pool. This is a read-only attribute.

Parameter

None

Returns

The method returns the attribute value.

Example

```
#Get key value
my $value = $object->key();
```

limit( )

Use this method to retrieve the total number of available dynamic licenses from the license pool. This is a read-only attribute.

Parameter

None

Returns

The method returns the attribute value.

Example

```
#Get limit value
my $value = $object->limit();
```

limit_context( )

Use this method to retrieve the context of the license limit. This is a read-only attribute.

Parameter

None

Returns

The method returns the attribute value.

Example

```
#Get limit_context value
my $limit_context = $object->limit_context();
```

model( )

Use this method to retrieve the supported vNIOS appliance model. This is a read-only attribute.

Parameter

None

Returns

The method returns the attribute value.

Example

```
#Get model value
my $value = $object->model();
```

subpools( )

Use this method to retrieve dynamic licenses in sub pools. This is a read-only attribute.

Parameter

None

Returns

The method returns the attribute value. An array of Infoblox::Grid::LicenseSubPool objects.

Example

```
#Get subpools value
my $value = $object->subpools();
```

temp_assigned( )

Use this method to retrieve the total number of temporary dynamic licenses allocated to vNIOS appliances. This is a read-only attribute.

Parameter

None

Returns

The method returns the attribute value.

Example

```
#Get temp_assigned value
my $value = $object->temp_assigned();
```

type( )

Use this method to retrieve the license type. This is a read-only attribute.

Parameter

None

Returns

The method returns the attribute value.

Example

```
#Get type value
my $value = $object->type();
```

SAMPLE CODE

The following sample code demonstrates different operations that can be applied to a License Pool object. This sample code also includes error handling for the operations.

#Preparation prior License Pool fetching

```
#PROGRAM STARTS: Include all the modules that will be used
use strict;
use Infoblox;

#Create a session to the Infoblox device
my $session = Infoblox::Session->new(
    master   => "192.168.1.2",
    username => "admin",
    password => "infoblox"
);

unless ($session) {
   die("Construct session failed: ",
       Infoblox::status_code() . ":" . Infoblox::status_detail());
}
print "Session created successfully\n";
```

#Fetching the License Pool objects

```
my $retrieved_objs=$session->get(object=> "Infoblox::Grid::LicensePool");

unless($retrieved_objs) {
    die("Getting the license pools failed:",
       Infoblox::status_code() . ":" . Infoblox::status_detail());
}
print "Got the license pools successfully\n";
####PROGRAM ENDS####
```

AUTHOR

Infoblox Inc. http://www.infoblox.com/

SEE ALSO

Infoblox::Session, Infoblox::Grid::LicenseSubPool

COPYRIGHT

Copyright (c) 2017 Infoblox Inc.

Infoblox::Grid::Member::BGP::Neighbor - BGP neighbor object

- NAME
- DESCRIPTION
- CONSTRUCTOR
- MODULE METHODS
  - Infoblox::Grid::Member::BGP::AS->neighbors( )
- METHODS
  - enable_multihop( )
  - enable_bfd( )
  - interface( )
  - multihop_ttl( )
  - neighbor_ip( )
  - remote_as( )
  - authentication_mode( )
  - bfd_template( )
  - bgp_neighbor_pass( )
  - comment( )
- AUTHOR
- SEE ALSO
- COPYRIGHT

NAME

Infoblox::Grid::Member::BGP::Neighbor - BGP (Border Gateway Protocol) neighbor object

DESCRIPTION

An Infoblox::BGP::Neighbor object represents the BGP neighbor that is configured in an Autonomous System (AS). BGP neighbors are configured at the grid member level.

CONSTRUCTOR

```
my $neighbor = Infoblox::Grid::Member::BGP::Neighbor->new(
    neighbor_ip          => $ipv4addr | $ipv6addr, # Required
    remote_as            => $num,                  # Required
    bgp_neighbor_pass    => $string,               # Required if  authentication_mode set to "MD5"
    authentication_mode  => "NONE" | "MD5",        # Optional / default is "NONE"
    bfd_template         => $string,               # Optional / default is undefined
    comment              => $string,               # Optional / default is undefined
    enable_bfd           => "true" | "false",      # Optional / default is "false"
    enable_multihop      => "true" | "false",      # Optional / default is "false"
    interface            => $string,               # Optional / default is "LAN_HA"
    multihop_ttl         => $uint | undef,         # Optional / default is undefined
);
```

MODULE METHODS

The following functions can be applied to a BGP AS object.

Infoblox::Grid::Member::BGP::AS->neighbors( )

Use this function to specify BGP Autonomous Systems for the grid member. See Infoblox::Grid::Member::BGP::AS->neighbors() for parameters and return values.

Example

```
#Construct a BGP neighbor
my $neighbor = Infoblox::Grid::Member::BGP::Neighbor->new(
    neighbor_ip           => "172.16.0.1",
    remote_as             => 1234,
    authentication_mode   => "NONE",
    interface             => "LAN_HA",
);

#Construct a BGP AS
my $autonomous_system = Infoblox::Grid::Member::BGP::AS->new(
    as          => 269,
    keepalive       => 4,
```

```
    holddown        => 16,
    neighbors       => [ $neighbor ]
);

# Configure BGP on the Grid Member object
my $response = $Grid_Member->bgp_as([$as]);
```

METHODS

This section describes all the methods that can be used to configure and retrieve the attribute values of an Infoblox::BGP::AS object.

enable_multihop( )

Use this method to set or retrieve the flag that indicates whether the multihop option is enabled or not.

By default, BGP connects to external peers residing on directly connected networks. Usually the ttl in IP header (hop-limit IPv6) is set to 1 to avoid BGP packets crossing networks. EBGP multihop option allows connection to peers which have one or more non-BGP routers in between.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

Specify 'true' to enable multihop option and 'false' to disable it. The default value is 'false'.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get enable_multihop value
my $enable_multihop = $object->enable_multihop();
#Modify enable_multihop value
$object->enable_multihop("true");
```

enable_bfd( )

Use this method to set or retrieve the flag that indicates whether the Bidirectional Forwarding Detection (BFD) fallover is enabled or not.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

Specify 'true' to enable BFD fallover and false to disable it. The default value is 'false'.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get enable_bfd value
my $enable_bfd = $object->enable_bfd();
#Modify enable_bfd value
$object->enable_bfd('true');
```

interface( )

Use this method to set or retrieve the interface that sends BGP advertisement information.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

Specify the interface in string format. The only supported value is "LAN_HA".

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get interface
my $interface = $neighbor->interface();
#Modify interface
$neighbor->interface("LAN_HA");
```

multihop_ttl( )

Use this method to set or retrieve the TTL setting for multihop option.

By default, BGP connects to external peers residing on directly connected networks. Usually the ttl in IP header (hop-limit IPv6) is set to 1 to avoid BGP packets crossing networks. EBGP multihop option allows connection to peers which have one or more non-BGP routers in between.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

The valid value is an unsigned integer between 1 and 255. The default value is undefined.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get multihop_ttl value
my $multihop_ttl = $object->multihop_ttl();
#Modify multihop_ttl value
$object->multihop_ttl(200);
```

neighbor_ip( )

Use this method to set or retrieve the IP address of the BGP neighbor.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

The IPv4 or IPv6 address of the BGP neighbor.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get neighbor IP address
my $neighbor_ip = $neighbor->neighbor_ip();
#Modify neighbor IP address
$neighbor->neighbor_ip("192.168.1.2");
$neighbor->neighbor_ip("2001::1");
```

remote_as( )

Use this method to set or retrieve the remote AS number of the BGP neighbor.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

An AS number is a 16-bit number from 1 to 65535.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get neighbor remote AS number
my $remote_as = $neighbor->remote_as();
#Modify remote AS number
```

```
$neighbor->remote_as(269);
```

authentication_mode( )

Use this method to set or retrieve the attribute about whether BGP authentication has been set for each neighbor.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

Set parameter to "MD5" to enable BGP authentication for a definite neighbor.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get neighbor enable authentication
my $remote_as = $neighbor->authentication_mode();
#Modify enable authentication
$neighbor->authentication_mode("MD5");
```

bfd_template( )

Use this method to set or retrieve the Bidirectional Forwarding Detection (BFD) template name.

BFD template is used to configure advanced BFD settings such as timer intervals, authentication.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

Desired BFD template in string format. Default value is undefined.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get bfd_template value
my $bfd_template = $object->bfd_template();
#Modify bfd_template value
$object->bfd_template("template1");
```

bgp_neighbor_pass( )

Use this method to set the password for a BGP neighbor. This is required only if authentication_mode is set to "MD5". When the password is entered, the value is preserved even if authentication_mode is changed to "NONE". This is a write-only attribute.

Parameter

The bgp_neighbor_pass in string format.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

Example

```
#Modify authentication password per neighbor
$neighbor->bgp_neighbor_pass("authpassword");
```

comment( )

Use this method to set or retrieve a descriptive comment.

Parameter

Desired comment in string format with a maximum of 256 bytes.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get neighbor comment
my $comment = $neighbor->comment();
#Modify comment
$neighbor->comment("my comment");
```

AUTHOR

Infoblox Inc. http://www.infoblox.com/

SEE ALSO

L<Infoblox::Grid::Member::BGP::AS|Infoblox::Grid::Member::BGP::AS>

COPYRIGHT

Copyright (c) 2017 Infoblox Inc.

Infoblox::Grid::Member::Interface - Interface member object.

- NAME
- DESCRIPTION
- CONSTRUCTOR
- MODULE METHODS
  - Infoblox::Grid::Member->additional_ip_list( )
  - Infoblox::Grid::Member::DNS->additional_ip_list( )
- METHODS
  - anycast( )
  - cidr( )
  - comment( )
  - dscp( )
  - enable_bgp( )
  - enable_ospf( )
  - enable_ipv6_auto_config( )
  - gateway( )
  - interface( )
  - ipv4addr( )
  - ipv6addr( )
  - subnet_mask( )
  - override_dscp( )
  - vlan_id( )
- SAMPLE CODE
- AUTHOR
- SEE ALSO
- COPYRIGHT

NAME

Infoblox::Grid::Member::Interface - Interface member object.

DESCRIPTION

An Interface member object represents additional interface information that can be used at the member level. A single IPv6 address may be added to each member by using an Interface object.

CONSTRUCTOR

```
my  $interface = Infoblox::Grid::Member::Interface->new(
    anycast     => "true" | "false",                          #Optional / Default is "false"
    cidr        => $prefix,                                   #Optional / Default is undefined
    comment     => $string,                                   #Optional / Default is undefined
    enable_bgp  => "true" | "false",                          #Optional / Default is "false"
    enable_ospf => "true" | "false",                          #Optional / Default is "false"
    enable_ipv6_auto_config => "true" | "false",              #Optional / Default is "false"
    gateway     => $ipv4addr | $ipv6addr,                     #Optional / Default is undefined
    interface   => "LAN_HA" | "LOOPBACK" | "MGMT" | "LAN2",   #Optional / Default is "LOOPBACK"
    ipv4addr    => $ipv4addr,                                 #Optional / Default is undefined
    ipv6addr    => $ipv6addr,                                 #Optional / Default is undefined
    subnet_mask => $netmask                                   #Optional / Default is undefined
    dscp          => $uint,                                   #Optional / Default is undefined
    override_dscp => "true" | "false",                        #Optional / Default is "false"
    vlan_id       => $uint,                                   #Optional / Default is undefined
);
```

MODULE METHODS

The following functions are available to be applied to an Interface object.

Infoblox::Grid::Member->additional_ip_list( )

Use this function to specify Interface for the Grid Member. See Infoblox::Grid::Member->additional_ip_list() for parameters and return values.

Example 1

```
#Construct an Interface object
my $interface1 = Infoblox::Grid::Member::Interface->new(
```

```
    ipv4addr    => "10.2.3.4",
    anycast     => "true",
    interface   => "LOOPBACK",
    subnet_mask => "255.255.255.255",
    comment     => "this is a loopback interface"
);

# Configure Interface on the Grid Member object
my $response = $Grid_Member->additional_ip_list([$interface1]);
```

Example 2

```
#Construct an IPv6 Interface object for member addition
my $interface6 = Infoblox::Grid::Member::Interface->new(
    ipv6addr    => "2001::1234",
    gateway     => "2001::1111",
    anycast     => "false",
    interface   => "LAN_HA",
    cidr        => "64",
    comment     => "this is an IPv6 address"
);

# Configure Interface on the Grid Member object
my $response = $Grid_Member->additional_ip_list([$interface6]);
```

Example 3

```
#Construct an IPv6 Interface object for member addition with IPv6 auto-configuration turned on
my $interface6 = Infoblox::Grid::Member::Interface->new(
    ipv6addr    => "2001::1234",
    enable_ipv6_auto_config  => "true",
    anycast     => "false",
    interface   => "LAN_HA",
    cidr        => "64",
    comment     => "this is an IPv6 address with auto-configuration turned on"
);

# Configure Interface on the Grid Member object
my $response = $Grid_Member->additional_ip_list([$interface6]);
```

Infoblox::Grid::Member::DNS->additional_ip_list( )

Use this function to specify Interface for the Grid Member DNS. See Infoblox::Grid::Member::DNS->additional_ip_list() for parameters and return values.

Example

```
#Construct an Interface object
my $interface2 = Infoblox::Grid::Member::Interface->new(
    ipv4addr    => "11.22.33.44",
    anycast     => "true",
    interface   => "LOOPBACK",
    subnet_mask => "255.0.0.0",
    comment     => "this is a loopback interface"
);

# Configure Interface on the Grid Member DNS object
my $response = $Grid_Member_DNS->additional_ip_list([$interface2]);
```

METHODS

This section describes all the methods that can be used to configure and retrieve the attribute values of an Interface object.

anycast( )

Use this method to enable/disable anycast of the Interface object.

When anycast is enabled the Interface is served for anycast usage.

Parameter

Specify "true" to enable anycast or "false" to disable it. Default value is "false".

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

Example

```
#Get anycast
my $anycast = $interface->anycast();
#Modify anycast
$interface->anycast("false");
```

cidr( )

Use this method to set or retrieve the CIDR netmask format of the Inteface object. Required only when ipv6addr is specified.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

Specify the netmask in CIDR format (example: "8" should be specified for netmask 255.0.0.0).

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get cidr
my $cidr = $interface->cidr();
#Modify cidr
$interface->cidr("16");
```

comment( )

Use this method to set or retrieve a descriptive comment of Interface object.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

Desired comment in string format with a maximum of 512 bytes. Default value is undefined.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get comment
my $comment = $interface->comment();
#Modify comment
$interface->comment("Modifying the Interface comment");
```

dscp( )

Use this method to set or retrieve the DSCP value of the Interface object.

Setting this method to a defined value implicitly sets the override_dscp method to "true". Setting the parameter to undefined causes the appliance to use the member default and automatically resets the override_dscp attribute to "false".

Note that when dscp is set to a defined value and override_dscp is set to "false", the last operation takes precedence. Thus the sequence $object->dscp(11); $object->override_dscp("false"); will set override_dscp to "false", and the sequence $object->override_dscp("false"); $object->dscp(11); will result in override_dscp="true".

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

Valid values are integers between 0 and 63, inclusive. The default value is undefined.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get attribute value
```

```
my $value = $interface->dscp();
#Modify attribute value
$interface->dscp(11);
```

enable_bgp( )

Use this method to set or retrieve the BGP advertisement setting for this interface.

Parameter

Specify "true" to enable BGP advertisements on this interface or "false" to disable them. Enabling advertisements requires the anycast attribute to be set to "true".

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

Example

```
#Get enable_bgp
my $enable_bgp = $interface->enable_bgp();
#Modify enable_bgp
$interface->enable_bgp("true");
```

enable_ospf( )

Use this method to set or retrieve the OSPF advertisement setting for this interface.

Parameter

Specify "true" to enable OSPF advertisement on this interface or "false" to disable it. Enabling advertisement requires the anycast attribute to be set to "true".

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

Example

```
#Get enable_ospf
my $enable_ospf = $interface->enable_ospf();
#Modify enable_ospf
$interface->enable_ospf("true");
```

enable_ipv6_auto_config( )

Use this method to enable/disable IPv6 auto-configuration of the Interface object.

When IPv6 auto-configuration is enabled, a gateway is not needed since it will be automatically discovered.

Parameter

Specify "true" to enable IPv6 auto-configuration or "false" to disable it. Default value is "false".

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

Example

```
#Get IPv6 auto-configuration
my $ipv6_autoconfig_enabled = $interface->enable_ipv6_auto_config();
#Modify IPv6 auto-configuration
$interface->enable_ipv6_auto_config("true");
```

gateway( )

Use this method to set or retrieve the gateway of the Interface object.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

Specify the gateway of the Interface in IPv4 address format or IPV6 address format. An IPv4 address is a 32-bit number in dotted decimal notation. It consists of four 8-bit groups of decimal digits separated by decimal points (example: 192.168.1.2). An IPv6 address is a 128-bit number in colon hexadecimal notation. It consists of eight 16-bit groups of hexadecimal digits separated by colons (example: 12ab:0000:0000:0123:4567:89ab:0000:cdef).

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get gateway
my $gateway = $interface->gateway();
#Modify gateway
$interface->gateway("10.1.1.1");
```

interface( )

Use this method to set or retrieve the interface type of this Interface object.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

Specify the interface type in string format. Valid values are "LOOPBACK", "LAN_HA", "MGMT" or "LAN2". Default value is "LOOPBACK".

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get interface
my $interface = $interface->interface();
#Modify interface
$interface->interface("LAN2");
```

ipv4addr( )

Use this method to set or retrieve the IPV4 address used for the Interface object.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

An IPv4 address is a 32-bit number in dotted decimal notation. It consists of four 8-bit groups of decimal digits separated by decimal points (example: 192.168.1.2).

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get ipv4addr
my $ipv4addr = $interface->ipv4addr();
#Modify ipv4addr
$interface->ipv4addr("2.2.2.2");
```

ipv6addr( )

Use this method to set or retrieve the IPV6 address used for the Interface object.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

An IPv6 address is a 128-bit number in colon hexadecimal notation. It consists of eight 16-bit groups of hexadecimal digits separated by colons (example: 12ab:0000:0000:0123:4567:89ab:0000:cdef). IPv6 gateway, and an CIDR are used in conjunctions with ipv6addr ( ).

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get ipv6addr
my $ipv6addr = $interface->ipv6addr();
#Modify ipv6addr
$interface->ipv6addr("aaaa:1111:bbbb:2222:cccc:3333:dddd:5555");
```

subnet_mask( )

Use this method to set or retrieve the subnet mask of the Interface object.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

Specify the subnet mask of the Interface in IPv4 address format. An IPv4 address is a 32-bit number in dotted decimal notation. It consists of four 8-bit groups of decimal digits separated by decimal points (example: 255.0.0.0).

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get subnet_mask
my $ipv4_subnet_mask = $interface->subnet_mask();
#Modify subnet_mask
$interface->subnet_mask("255.255.255.255");
```

override_dscp( )

The override_dscp attribute contols whether the DSCP value is used, instead of the Member default.

The override_dscp attribute can be specified explicitly. It is also set implicitly when DSCP is set to a defined value.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

Specify "true" to set the override_dscp flag or "false" to deactivate/unset it. The default value is "false".

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get override_dscp from the member
my $override = $interface->override_dscp();
#Modify override_dscp
$interface->override_dscp("true");
```

vlan_id( )

Use this method to set or retrieve the VLAN ID of the Interface object.

Parameter

Valid values are unsigned integers between 1 and 4095, inclusive. The default value is undefined.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get attribute value
my $attr = $interface->vlan_id();
#Modify attribute value
$interface->vlan_id(10);
```

SAMPLE CODE

The following sample code demonstrates the different functions that can be applied to an Interface object such as add, modify, and remove. Also, this sample includes error handling for the operations.

#Preparation prior to Interface object insertion

```
use strict;
use warnings;
use Infoblox;

my $host_name = "infoblox.localdomain";
my $host_ip   = "192.168.1.2";
#Creating a session to Infoblox appliance.
my $session = Infoblox::Session->new(
    master   => $host_ip,
    username => "admin",
    password => "infoblox"
);

unless ($session) {
    die(qq(constructor for session failed: ),
        join(":", Infoblox::status_code(), Infoblox::status_detail()));
}
print "Session created successfully \n";
```

#Create an Interface object

```
#Creating Interface object.
my $interface1 = Infoblox::Grid::Member::Interface->new(
    ipv4addr     => "10.2.3.4",
    anycast      => "false",
    interface    => "LOOPBACK",
    subnet_mask  => "255.255.255.255",
   ) or die(qq(Constructor for interface1 failed: ),
   join(":", Infoblox::status_code(), Infoblox::status_detail()));
print "Interface object constructed successfully \n";
```

#Add Interface object to Grid Member

```
#Getting the member object from appliance through session.
my @result = $session->get(
    object => "Infoblox::Grid::Member",
    name   => $host_name,
    ) or die(qq(Get Grid Member failed: ),
    join(":", $session->status_code(), $session->status_detail()));
print "Get Grid Member successful\n";

my $result = $result[0];
if ($result) {
    #Modifying the value of the Interface method from the member object.
    $result->additional_ip_list([$interface1]);
    #Applying the changes to appliance through session.
    $session->modify($result)
        or die(qq(Modify Grid Member failed: ),
        join(":", $session->status_code(), $session->status_detail()));
}
print "Interface object added to Grid Member successfully\n";
```

#Modify Interface object

```
#Modifying the comment of the Interface object.
$interface1->comment("Modified Comment");

#Apply changes to the Grid Member object.
$result->additional_ip_list([$interface1]);

#Update Grid Member object through the Infoblox session.
$session->modify($result)
  or die("Modify Grid Member object failed: ",
  $session->status_code() . ":" . $session->status_detail());
print "Grid Member object with modified Interface updated to Infoblox appliance successfully\n";
```

#Remove Interface object

```
#Remove the Interface from the Grid Member object.
$result->additional_ip_list([]);

#Update Grid Member object through the Infoblox session.
```

```
$session->modify($result)
  or die("Modify Grid Member object failed: ",
  $session->status_code() . ":" . $session->status_detail());
print "Removed Interface object from Grid Member successfully\n";

####PROGRAM ENDS####
```

AUTHOR

Infoblox Inc. http://www.infoblox.com/

SEE ALSO

Infoblox::Grid::Member, Infoblox::Session , Infoblox::Grid::Member::DNS

COPYRIGHT

Copyright (c) 2017 Infoblox Inc.

Infoblox::Grid::Member::BGP::AS - BGP object.

- NAME
- DESCRIPTION
- CONSTRUCTOR
- MODULE METHODS
  - Infoblox::Grid::Member->bgp_as( )
- METHODS
  - as( )
  - keepalive( )
  - holddown( )
  - neighbors( )
- AUTHOR
- SEE ALSO
- COPYRIGHT

NAME

Infoblox::Grid::Member::BGP::AS - BGP (Border Gateway Protocol) Autonomous System (AS) object.

DESCRIPTION

An Infoblox::BGP::AS object represents a BGP Autonomous System configured at the grid member level.

CONSTRUCTOR

```
my $bgp_as = Infoblox::Grid::Member::BGP::AS->new(
    as              => $num,                          # Optional / Default value is undefined
    keepalive       => $num,                          # Optional / Default value is 4
    holddown        => $num,                          # Optional / Default value is 16
    neighbors       => [ $neighbor1, $neighbor2, ... ], # Optional / Default value is empty
);
```

MODULE METHODS

The following functions can be applied to a BGP AS object.

Infoblox::Grid::Member->bgp_as( )

Use this function to specify BGP Autonomous Systems for the grid member. See Infoblox::Grid::Member->bgp_as() for parameters and return values.

Example

```
#Construct a BGP neighbor
my $neighbor = Infoblox::Grid::Member::BGP::Neighbor->new(
    neighbor_ip          => "172.16.0.1",
    remote_as            => 1234,
    interface            => "LAN_HA",
);

#Construct a BGP AS
my $autonomous_system = Infoblox::Grid::Member::BGP::AS->new(
    as          => 269,
    keepalive       => 4,
    holddown        => 16,
    neighbors       => [ $neighbor ]
);

# Configure BGP on the Grid Member object
my $response = $Grid_Member->bgp_as([$as]);
```

METHODS

This section describes all the methods that can be used to configure and retrieve the attribute values of an Infoblox::BGP::AS object.

as( )

Use this method to set or retrieve the number of this AS.

A valid AS number is required to advertise to neighbors defined in this AS.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

An AS number is a 16-bit integer from 1 to 65535.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get AS number
my $as_number = $autonomous_system->as();
#Modify AS number
$autonomous_system->as(300);
```

keepalive( )

Use this method to set or retrieve the AS keepalive timer.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

The number of seconds for the keepalive timer. The valid value is from 1 to 21845 seconds.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get AS keepalive timer
my $keepalive = $autonomous_system->keepalive();
#Modify AS keepalive timer
$autonomous_system->keepalive(3600);
```

holddown( )

Use this method to set or retrieve the AS hold down timer.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

The number of seconds for the hold down timer. The valid value is from 3 to 65535 seconds.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get Autonomous System hold down timer
my $holddown = $autonomous_system->holddown();
#Modify Autonomous System hold down timer
$autonomous_system->holddown(3600);
```

neighbors( )

Use this method to set or retrieve the BGP neighbors for the Infoblox::BGP::AS object.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

The valid value is an array reference that contains Infoblox::Grid::Member::BGP::Neighbor object(s).

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get AS neighbor list
my $neighbors = $autonomous_system->neighbors();
#Modify AS neighbors
$autonomous_system->neighbors( [ $neighbor1, $neighbor2 ] );
```

AUTHOR

Infoblox Inc. http://www.infoblox.com/

SEE ALSO

Infoblox::Grid::Member, Infoblox::Grid::Member::BGP::Neighbor

---

COPYRIGHT

Copyright (c) 2017 Infoblox Inc.

Infoblox::Grid::Member::IPv6StaticRoute - Static route member object

- NAME
- DESCRIPTION
- CONSTRUCTOR
- MODULE METHODS
  - Infoblox::Grid::Member->ipv6_static_routes( )
- METHODS
  - address( )
  - cidr( )
  - gateway( )
- AUTHOR
- SEE ALSO
- COPYRIGHT

NAME

Infoblox::Grid::Member::IPv6StaticRoute - Static route member object

DESCRIPTION

An IPv6StaticRoute object provides IPv6 static route information for a grid member.

CONSTRUCTOR

```
my $route = Infoblox::Grid::Member::IPv6StaticRoute->new(
        "address" => $ip_address,        #Required
        "gateway" => $ip_address,        #Required
        "cidr"    => $cidr,                      #Required
);
```

MODULE METHODS

The following functions can be applied to an IPv6staticroute object.

Infoblox::Grid::Member->ipv6_static_routes( )

Use this function to specify an IPv6staticroute object for the grid member. See Infoblox::Grid::Member->ipv6_static_routes() for parameters and return values.

Example

```
#Get static routes list
my $ipv6_static_routes = $grid_member->ipv6_static_routes();
#Modify static routes list
my $route = Infoblox::Grid::Member::IPv6StaticRoute->new(
        "address" => "2002::5",
        "cidr"    => 64
        "gateway" => "2002::1",
);
$grid_member->ipv6_static_routes([$route]);
```

METHODS

This section describes all the methods that can be used to configure and retrieve the attribute values of an IPv6 static route object.

address( )

Use this method to set or retrieve the IPv6 address of an IPv6 static route object.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

An IPv6 address is a 128-bit number represented as eight groups of four hexadecimal digits (i.e. two octets), each group separated by a `colon(:)` (example: 2001:0db8:85a3:0000:0000:8a2e:0370:7334). Leading zeroes and groups of consecutive zeros within the value can be omitted to produce a more compact representation (example: 2001::1, ::1).

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get the address
my $address= $route->address();
#Modify the address
$route->address("2002::5");
```

cidr( )

Use this method to set or retrieve the cidr of the IPv6 static route object.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

Specify the netmask in CIDR format.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get the cidr
my $cidr= $route->cidr();
#Modify the cidr
$route->cidr(64);
```

gateway( )

Use this method to set or retrieve the gateway of the IPv6 static route object.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

An IPv6 address is a 128-bit number represented as eight groups of four hexadecimal digits (i.e. two octets), each group separated by a `colon(:)` (example: 2001:0db8:85a3:0000:0000:8a2e:0370:7334). Leading zeroes and groups of consecutive zeros within the value can be omitted to produce a more compact representation (example: 2001::1, ::1).

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get the gateway address
my $gateway= $route->gateway();
#Modify the gateway address
$route->gateway("2002::1");
```

AUTHOR

Infoblox Inc. http://www.infoblox.com/

SEE ALSO

Infoblox::Grid::Member

COPYRIGHT

Copyright (c) 2017 Infoblox Inc.

Grid::Member::Communication - Grid::Member::Communication object.

- NAME
- DESCRIPTION
- MODULE METHODS
  - Infoblox::Grid::Member->member_service_communication( )
- METHODS
  - option( )
  - service( )
  - type( )
- AUTHOR
- SEE ALSO
- COPYRIGHT

NAME

Grid::Member::Communication - Grid::Member::Communication object.

DESCRIPTION

The structure stores information regarding service, transport type and preference.

MODULE METHODS

The following functions can be applied to a communication object.

Infoblox::Grid::Member->member_service_communication( )

Use this function to retrieve a reference to an array of Infoblox::Grid::Member::Communication objects that represents the communication settings for an Infoblox appliance.

Example

```
#Retrieve the member object
my $member = $session->get(object => 'Infoblox::Grid::Member', name => 'hostname.com');

# Access its communication settings
my @communication = @{$member->member_service_communication()};
print $communications[0]->type();
```

METHODS

This section describes all the methods that you can use to set or retrieve the attribute values of the object.

option( )

Use this method to retrieve the flag when the service is either forced to use one type of transport or have a preference for the transport type. This is a read-only attribute.

Parameter

None

Returns

The method returns the attribute value, which can be either "FORCE" or "PREFER".

Example

```
#Get option
my $option = $comm->option();
```

service( )

Use this method to set or retrieve services for which communication type can be configured.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

The valid value is one of the following: "GRID", "GRID_BACKUP", "NTP", "REPORTING", "REPORTING_BACKUP", "OCSP", "MAIL",

"AD".

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
# Get service
my $service = $comm->service();
# Modifying service
$comm->service("GRID");
```

type( )

Use this method to set or retrieve the transport type.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

The valid value is IPv4 or IPv6.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
# Get type
my $type = $comm->type();
# Modifying type
$comm->type("IPv4");
```

AUTHOR

Infoblox Inc. http://www.infoblox.com/

SEE ALSO

Infoblox::Grid::Member/>

COPYRIGHT

Copyright (c) 2017 Infoblox Inc.

Infoblox::Grid::Member::ServiceStatus - Service Status object.

- NAME
- DESCRIPTION
- MODULE METHODS
  - Infoblox::Grid::Member->service_status( ), Infoblox::Grid::Member->node1_service_status( ), Infoblox::Grid::Member->node2_service_status( )
- METHODS
  - description( )
  - status( )
  - service( )
- AUTHOR
- SEE ALSO
- COPYRIGHT

NAME

Infoblox::Grid::Member::ServiceStatus - Service Status object.

DESCRIPTION

This object is used to report the service status on an Infoblox appliance. All methods are to be considered read-only.

MODULE METHODS

The following functions can be applied to a ServiceStatus object.

Infoblox::Grid::Member->service_status( ), Infoblox::Grid::Member->node1_service_status( ), Infoblox::Grid::Member->node2_service_status( )

Use this function to retrieve a reference to an array of Infoblox::Grid::Member::ServiceStatus objects that represents the service status for an Infoblox appliance.

Example

```
#Retrieve the member object
my $member = $session->get(object => 'Infoblox::Grid::Member', name => 'hostname.com');

# Access its service status
my @services = @{$member->service_status()};
print $services[0]->status();
```

METHODS

This section describes all the methods that you can use to retrieve the attribute value of an appliance's service status.

description( )

Use this method to retrieve the description of the current ServiceStatus object.

Returns

The method returns the attribute value, which depends on the value of the 'service' method:

```
service value (for service_status) returned by this method
==========================================================
DHCP - DHCP service status
DNS - DNS service status
NTP - NTP service status
TFTP -  Percentage of the file distribution storage space that is used
HTTP_FILE_DIST - Percentage of the file distribution storage space that is used
FTP - Percentage of the file distribution storage space that is used
DNS_CACHE_ACCELERATION - DNS Cache Acceleration service status.

service value (for node1/2_service_status) returned by this method
===================================================================
NODE_STATUS - Node status
CPU1_TEMP - CPU1 temperature in Celsius
CPU2_TEMP - CPU2 temperature in Celsius
DISK_USAGE - Disk usage percentage
ENET_LAN - LAN1 interface IP
ENET_LAN2 - LAN2 interface IP
ENET_HA - HA interface IP
ENET_MGMT - MGMT interface IP
```

```
LCD - State of the LCD process (running/not running)
MEMORY - Memory usage percentage
REPLICATION - HA replication state
DB_OBJECT - Database capacity usage percentage
RAID_DISK1 - Disk status and type
RAID_DISK2 - Disk status and type
RAID_DISK3 - Disk status and type
RAID_DISK4 - Disk status and type
FAN1 - Fan 1 speed
FAN2 - Fan 2 speed
FAN3 - Fan 3 speed
POWER_SUPPLY - Power status
NTP_SYNC - Indicates if there are "reachable" NTP associations
TEMP_SYS - System temperature in Celsius
RAID_BATTERY - RAID battery status
CPU_USAGE - CPU Usage
BGP - BGP service status
OSPF - OSPF service status
OSPF6 - OSPFv6 service status
SWAP_USAGE - Swap Usage
```

Example

```
#Get description
my $description = $service_status->description();
```

status( )

Use this method to retrieve the service status.

Returns

The method returns the attribute value. Valid values are "Failed", "Warning", "Working", and "Inactive"

Example

```
#Get status
my $status = $service_status->status();
```

service( )

Use this method to retrieve the service identifier.

Returns

The method returns the attribute value. Possible values when the object is retrieved via service_status are:

ANALYTICS, ATP, CAPTIVE_PORTAL, CLOUD_API, DHCP, DISCOVERY, DNS, DNS_CACHE_ACCELERATION, FTP, HSM, HTTP_FILE_DIST, NTP, REPORTING, TAXII, TFTP

Possible values when the object is retrieved via node1/2_service_status are:

AUTH_NAMED, BGP, CORE_FILES, CPU1_TEMP, CPU2_TEMP, CPU_USAGE, CPU_USAGE, DB_OBJECT, DISCOVERY_CAPACITY, DISK_SIZE, DISK_USAGE, ENET_HA, ENET_LAN, ENET_LAN2, ENET_MGMT, EXTERNAL_STORAGE, FAN1, FAN2, FAN3, FAN4, FAN5, FAN6, FAN7, FAN8, JOIN_STATUS, LCD, MEMORY, MGM_SERVICE, NETWORK_CAPACITY, NODE_STATUS, NTP_SYNC, OSPF, OSPF6, PASSIVE_HA_CONNECTIVITY, POWER1, POWER2, POWER3, POWER4, POWER_SUPPLY, RAID_BATTERY, RAID_DISK1, RAID_DISK2, RAID_DISK3, RAID_DISK4, RAID_DISK5, RAID_DISK6, RAID_DISK7, RAID_DISK8, RAID_SUMMARY, REPLICATION, SFP_HA, SFP_LAN, SFP_LAN2, SFP_MGMT, SNIC_CHIP_TEMP, SNIC_CORE_UTIL, SNIC_PCB_TEMP, SNIC_UTIL, SUBGRID_CONN, SWAP_USAGE, SYS_TEMP, VPN_CERT

Example

```
#Get service
my $service = $service_status->service();
```

AUTHOR

Infoblox Inc. http://www.infoblox.com/

SEE ALSO

Infoblox::Grid::Member/>

COPYRIGHT

Copyright (c) 2017 Infoblox Inc.

Infoblox::Grid::Member::OSPF - OSPF member object.

- NAME
- DESCRIPTION
- CONSTRUCTOR
- MODULE METHODS
  - Infoblox::Grid::Member->ospf_list( )
- METHODS
  - area_id( )
  - area_type( )
  - authentication_key( )
  - authentication_type( )
  - bfd_template( )
  - comment( )
  - cost( )
  - dead_interval( )
  - enable_bfd( )
  - hello_interval( )
  - interface( )
  - is_ipv4( )
  - key_id( )
  - retransmit_interval( )
  - transmit_delay( )
- SAMPLE CODE
- AUTHOR
- SEE ALSO
- COPYRIGHT

NAME

Infoblox::Grid::Member::OSPF - OSPF (Open Shortest Path First) member object.

DESCRIPTION

A OSPF member represents the configuration for a OSPF area used at the grid member level.

CONSTRUCTOR

```
my  $ospf = Infoblox::Grid::Member::OSPF->new(
    area_id             => $num | $ipv4addr,                        #Required
    interface           => "LAN_HA" | $ipaddr                       #Required
    area_type           => "standard" | "stub" | "nssa",            #Optional / Default is "standard"
    authentication_key  => $string | undef,                         #Optional / Default is undefined
    authentication_type => "none" | "simple" | "message-digest",    #Optional / Default is "none"
    bfd_template        => $string | undef,                         #Optional / Default is undefined
    enable_bfd          => "true" | "false"                         #Optional / Default is "false"
    comment             => $string | undef,                         #Optional / Default is undefined
    cost                => $num | "AUTO",                           #Optional / Default is "AUTO"
    dead_interval       => $num,                                    #Optional / Default is 40
    hello_interval      => $num,                                    #Optional / Default is 10
    key_id              => $num | undef,                            #Optional / Default is 1
    retransmit_interval => $num,                                    #Optional / Default is 5
    transmit_delay      => $num                                     #Optional / Default is 1
    is_ipv4             => "true" | "false"                         #Optional / Default is "true"
);
```

MODULE METHODS

The following functions are available to be applied to a OSPF object.

Infoblox::Grid::Member->ospf_list( )

Use this function to specify OSPF for the Grid Member. See Infoblox::Grid::Member->ospf_list() for parameters and return values.

Example

```
#Construct a OSPF object
my $ospf = Infoblox::Grid::Member::OSPF->new(
    area_id             => "20",
```

```
    area_type           => "standard",
    interface           => "LAN_HA",
    authentication_type => "simple",
    authentication_key  => "infoblox"
);

# Configure OSPF on the Grid Member object
my $response = $Grid_Member->ospf_list([$ospf]);
```

METHODS

This section describes all the methods that can be used to configure and retrieve the attribute values of a OSPF object.

area_id( )

Use this method to set or retrieve the area id value of the OSPF object.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

Specify the area id in a 32-bit integer (range from 0 to 4294967295) format or in IPv4 address format. An IPv4 address is a 32-bit number in dotted decimal notation. It consists of four 8-bit groups of decimal digits separated by decimal points (example: 192.168.1.2).

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get area_id
my $area_id = $ospf->area_id();
#Modify area_id
$ospf->area_id(1);
```

area_type( )

Use this method to set or retrieve the OSPF area type.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

Specify the area type in string format. Valid values are "standard", "stub" OR "nssa"(Not-so-stubby). Default value is "standard".

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get area_type
my $area_type = $ospf->area_type();
#Modify area_type
$ospf->area_type("stub");
```

authentication_key( )

Use this method to set the authentication password to use for OSPF. This is a write-only attribute. The authentication key is valid only when authentication type is "simple" or "message-digest".

Include the specified parameter to set the attribute value.

Parameter

Specify the authentication key in string format. The authentication key can have a maximum of 8 bytes if authentication type is "simple" or it can have a maximum of 16 bytes if the authentication type is "message-digest". Default value is undefined.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

Example

```
#Modify authentication_key
$ospf->authentication_key("infoblox");
```

authentication_type( )

Use this method to set or retrieve the authentication type used for the OSPF advertisement.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

Specify the authentication type in string format. Valid values are "none", "simple" or "message-digest". Default value is "none".

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get authentication_type
my $authentication_type = $ospf->authentication_type();
#Modify authentication_type
$ospf->authentication_type("message-digest");
```

bfd_template( )

Use this method to set or retrieve the Bidirectional Forwarding Detection (BFD) template name.

BFD template is used to configure advanced BFD settings such as timer intervals, authentication.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

Desired BFD template in string format. Default value is undefined.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get bfd_template value
my $bfd_template = $object->bfd_template();
#Modify bfd_template value
$object->bfd_template("template1");
```

comment( )

Use this method to set or retrieve a descriptive comment of the OSPF configuration.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

Desired comment in string format with a maximum of 256 bytes. Default value is undefined.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get comment
my $comment = $ospf->comment();
#Modify comment
$ospf->comment("Modifying the OSPF comment");
```

cost( )

Use this method to set or retrieve the cost metric associated with the OSPF advertisement. If "AUTO" is provided, cost metric will be generated automatically.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

Specify the cost metric in a 16-bit integer (range from 0 to 65535) format or in string format as "AUTO". Default value is "AUTO".

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get cost
my $cost = $ospf->cost();
#Modify cost
$ospf->cost(1);
```

dead_interval( )

Use this method to set or retrieve the dead interval value of OSPF. The dead interval describes the time taken in seconds to wait before declaring that the device is unavailable and down.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

Specify the dead interval in a 16-bit integer (range from 0 to 65535) format that represents the duration in seconds. Default value is 40.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get dead_interval
my $dead_interval = $ospf->dead_interval();
#Modify dead_interval
$ospf->dead_interval(10);
```

enable_bfd( )

Use this method to set or retrieve the flag that indicates whether the Bidirectional Forwarding Detection (BFD) fallover is enabled or not.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

Specify 'true' to enable BFD fallover and false to disable it. The default value is 'false'.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get enable_bfd value
my $enable_bfd = $object->enable_bfd();
#Modify enable_bfd value
$object->enable_bfd('true');
```

hello_interval( )

Use this method to set or retrieve the hello interval value of OSPF. The hello interval specifies how often to send OSPF hello advertisement, in seconds.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

Specify the hello interval in a 16-bit integer (range from 0 to 65535) format that represents the duration in seconds. Default value is 10.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get hello_interval
my $hello_interval = $ospf->hello_interval();
#Modify hello_interval
$ospf->hello_interval(20);
```

interface( )

Use this method to set or retrieve the interface that send out OSPF advertisement information.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

Valid value is "LAN_HA" or an IPv4 or IPv6 address of the VLAN interface.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get interface
my $interface = $ospf->interface();
#Modify interface
$ospf->interface("LAN_HA");
```

is_ipv4( )

Use this method to set or retrieve the OSPF protocol version.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

Specify "true" if the IPv4 version of OSPF is used, or "false" if the IPv6 version of OSPF is used. Default value is "true".

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get protocol version
my $ospf_is_ipv4 = $ospf->is_ipv4();
#Modify protocol version
$ospf->is_ipv4("true");
$ospf->is_ipv4("false");
```

key_id( )

Use this method to set or retrieve the hash key identifier to use for "message-digest" authentication. The hash key identifier is valid only when authentication type is "message-digest".

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

Specify the hash key identifier in a 8-bit integer (range from 0 to 255) format. Default value is 1.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get key_id
my $key_id = $ospf->key_id();
#Modify key_id
$ospf->key_id(100);
```

retransmit_interval( )

Use this method to set or retrieve the retransmit interval time of OSPF. The retransmit interval describes the time taken in seconds to wait before retransmitting OSPF advertisement.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

Specify the retransmit interval in a 16-bit integer (range from 0 to 65535) format that represents the duration in seconds. Default value is 5.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get retransmit_interval
my $retransmit_interval = $ospf->retransmit_interval();
#Modify retransmit_interval
$ospf->retransmit_interval(50);
```

transmit_delay( )

Use this method to set or retrieve the transmit delay value of OSPF. The transmit delay describes the time taken in seconds to wait before sending an advertisement.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

Specify the transmit delay in a 16-bit integer (range from 0 to 65535) format that represents the duration in seconds. Default value is 1.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get transmit_delay
my $transmit_delay = $ospf->transmit_delay();
#Modify transmit_delay
$ospf->transmit_delay(60);
```

SAMPLE CODE

The following sample code demonstrates the different functions that can be applied to an OSPF object such as add, modify, and remove. Also, this sample includes error handling for the operations.

#Preparation prior to OSPF object insertion

```
use strict;
use warnings;
use Infoblox;

my $host_name = "infoblox.localdomain";
my $host_ip = "192.168.1.2";
#Creating a session to Infoblox appliance.
my $session = Infoblox::Session->new(
    master   => $host_ip,
    username => "admin",
    password => "infoblox"
);
```

```
unless ($session) {
    die(qq(constructor for session failed: ),
        join(":", Infoblox::status_code(), Infoblox::status_detail()));
}
print "Session created successfully \n";
```

#Create a grid member OSPF object

```
#Creating OSPF object.
my $ospf1 = Infoblox::Grid::Member::OSPF->new(
    area_id             => "20",
    area_type           => "standard",
    interface           => "LAN_HA",
    authentication_type => "simple",
    authentication_key  => "infoblox"
   ) or die(qq(Constructor for ospf1 failed: ),
   join(":", Infoblox::status_code(), Infoblox::status_detail()));
print "OSPF object constructed successfully \n";
```

#Add OSPF object to Grid Member

```
#Getting the member object from appliance through session.
my @result = $session->get(
    object => "Infoblox::Grid::Member",
    name   => $host_name,
    ) or die(qq(Get Grid Member failed: ),
    join(":", $session->status_code(), $session->status_detail()));
print "Get Grid Member successful\n";

my $result = $result[0];
if ($result) {
    #Modifying the value of the ospf method from the member object.
    $result->ospf_list([$ospf1]);
    #Applying the changes to appliance through session.
    $session->modify($result)
        or die(qq(Modify Grid Member failed: ),
        join(":", $session->status_code(), $session->status_detail()));
}
print "OSPF object added to Grid Member successfully\n";
```

#Modify OSPF object

```
#Modifying the area id of the OSPF object.
$ospf1->area_id("50");

#Apply changes to the Grid Member object.
$result->ospf_list([$ospf1]);

#Update Grid Member object through the Infoblox session.
$session->modify($result)
  or die("Modify Grid Member object failed: ",
  $session->status_code() . ":" . $session->status_detail());
print "Grid Member object with modified OSPF updated to Infoblox appliance successfully\n";
```

#Remove OSPF object

```
#Remove the OSPF from the Grid Member object.
$result->ospf_list([]);

#Update Grid Member object through the Infoblox session.
$session->modify($result)
  or die("Modify Grid Member object failed: ",
  $session->status_code() . ":" . $session->status_detail());
print "Removed OSPF object from Grid Member successfully\n";

####PROGRAM ENDS####
```

AUTHOR

Infoblox Inc. http://www.infoblox.com/

SEE ALSO

Infoblox::Grid::Member, Infoblox::Session, Infoblox::Session->get(), Infoblox::Session->modify()

COPYRIGHT

Copyright (c) 2017 Infoblox Inc.

Infoblox::Grid::Member::PreProvisioning - Pre-Provisioned member object.

- NAME
- DESCRIPTION
- CONSTRUCTOR
- METHODS
  - hardware_info( )
  - licenses( )
- AUTHOR
- SEE ALSO
- COPYRIGHT

NAME

Infoblox::Grid::Member::PreProvisioning - Pre-Provisioned member object.

DESCRIPTION

An Infoblox::Grid::Member::PreProvisioning object represents pre-provisioned data for a Grid member.

CONSTRUCTOR

```
my $pp = Infoblox::Grid::Member::PreProvisioning->new(
    hardware_info   => [$hw1, $hw2, ...],                  # Optional / an array of
Infoblox::Grid::Member::PreProvisioning::Hardware objects
    licenses        => [$lic1, $lic2, ...],                # Optional / Default value is undefined
);
```

METHODS

This section describes all the methods that can be used to configure and retrieve the attribute values of an Infoblox::Grid::Member::PreProvisioning object.

hardware_info( )

Use this method to set or retrieve the hardware information.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

Valid value is an array of Infoblox::Grid::Member::PreProvisioning::Hardware objects.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get get hardware_info()
my $hw = $pp->hardware_info();
#Modify hardware_info
$pp->hardware_info([$hw1, $hw2]);
```

licenses( )

Use this method to set or retrieve pre-provisioning licenses.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

Valid value is an array of valid licenses for each IB model type for the pre-provisioinig feature.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get get license()
my $lics = $pp->license();
#Modify hardwary_info
$pp->license(['dns', 'dhcp']);
```

AUTHOR

Infoblox Inc. http://www.infoblox.com/

SEE ALSO

Infoblox::Grid::Member, Infoblox::Grid::Member::PreProvisioning::Hardware

---

COPYRIGHT

Copyright (c) 2017 Infoblox Inc.

Infoblox::Grid::Member::StaticRoute - Static route member object

- NAME
- DESCRIPTION
- CONSTRUCTOR
- MODULE METHODS
  - Infoblox::Grid::Member->static_routes( )
- METHODS
  - network( )
  - gateway( )
- SAMPLE CODE
- AUTHOR
- SEE ALSO
- COPYRIGHT

NAME

Infoblox::Grid::Member::StaticRoute - Static route member object

DESCRIPTION

A StaticRoute object provides static route information for a grid member.

CONSTRUCTOR

```
my $route = Infoblox::Grid::Member::StaticRoute->new(
        "network" => $network,          #Required
        "gateway" => $ip_address,       #Required
);
```

MODULE METHODS

The following functions can be applied to a StaticRoute object.

Infoblox::Grid::Member->static_routes( )

Use this function to specify a StaticRoute object for the grid member. See Infoblox::Grid::Member->static_routes() for parameters and return values.

Example

```
#Get static routes list
my $static_routes = $grid_member->static_routes();
#Modify static routes list
my $route = Infoblox::Grid::Member::StaticRoute->new(
        "network" => "192.168.2.0/24",
        "gateway" => "192.168.1.1",
);
$grid_member->static_routes([$route]);
```

METHODS

This section describes all the methods that can be used to configure and retrieve the attribute values of a StaticRoute object.

network( )

Use this method to set or retrieve the network address of a StaticRoute object.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

Specify the address and netmask of network in CIDR format (example: 192.168.2.0/24).

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get network address
my $network= $route->network();
#Modify network address
$route-->network("192.168.2.0/24");
```

gateway( )

Use this method to set or retrieve the gateway of the StaticRoute object.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

Specify the gateway address in IPv4 address format. An IPv4 address is a 32-bit number in dotted decimal notation. It consists of four 8-bit groups of decimal digits separated by decimal points (example: 192.168.1.2).

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get gateway address
my $gateway= $route->gateway();
#Modify gateway address
$route-->gateway("192.168.2.0");
```

SAMPLE CODE

The following sample code demonstrates the different functions that can be applied to a StaticRoute object such as add, modify, and remove. Also, this sample includes error handling for the operations.

#Preparation prior to a StaticRoute object insertion

```
use strict;
use warnings;
use Infoblox;

my $host_name = "infoblox.localdomain";
my $host_ip   = "192.168.1.2";
#Creating a session to the Infoblox appliance.
my $session = Infoblox::Session->new(
    master   => $host_ip,
    username => "admin",
    password => "infoblox"
);

unless ($session) {
    die(qq(constructor for session failed: ),
        join(":", Infoblox::status_code(), Infoblox::status_detail()));
}
print "Session created successfully \n";
```

#Create a StaticRoute object

```
#Creating a StaticRoute object.
my $route = Infoblox::Grid::Member::StaticRoute->new(
       "network" => "192.168.2.0/24",
       "gateway" => "192.168.1.1",
) or die(qq(Constructor for StaticRoute failed: ),
   join(":", Infoblox::status_code(), Infoblox::status_detail()));
print "StaticRoute object constructed successfully \n";
```

#Add a StaticRoute object to the static routes list of the grid member

```
#Getting the Grid Member object from the appliance through a session.
my @result = $session->get(
    object => "Infoblox::Grid::Member",
    name   => $host_name,
    ) or die(qq(Get Grid Member failed: ),
    join(":", $session->status_code(), $session->status_detail()));
print "Get Grid Member successful\n";
```

```
my $result = $result[0];
if ($result) {
    #Modifying the value of the StaticRoute method from the member object.
    $result->static_routes([$route]);
    #Applying the changes to appliance through session.
    $session->modify($result)
        or die(qq(Modify Grid Member failed: ),
        join(":", $session->status_code(), $session->status_detail()));
}
print "StaticRoute object added to Grid Member successfully\n";
```

#Remove the static routes list

```
#Remove the StaticRoute object from the Grid Member object.
$result->static_routes([]);

#Update the Grid Member object through the Infoblox session.
$session->modify($result)
  or die("Modify Grid Member object failed: ",
  $session->status_code() . ":" . $session->status_detail());
print "Removed StaticRoute object from Grid Member successfully\n";

####PROGRAM ENDS####
```

AUTHOR

Infoblox Inc. http://www.infoblox.com/

SEE ALSO

Infoblox::Grid::Member

COPYRIGHT

Copyright (c) 2017 Infoblox Inc.

Infoblox::Grid::Member::Capture::Control - Traffic capture control object

- NAME
- DESCRIPTION
- CONSTRUCTOR
- MODULE METHODS
  - Infoblox::Grid::Member->traffic_capture( )
- METHODS
  - action( )
  - interface( )
  - seconds_to_run( )
- SAMPLE CODE
- AUTHOR
- SEE ALSO
- COPYRIGHT

NAME

Infoblox::Grid::Member::Capture::Control - Traffic capture control object

---

DESCRIPTION

You can control traffic capture for a grid member via the settings in this object.

CONSTRUCTOR

```
my $control = Infoblox::Grid::Member::Capture::Control->new(
              action         => "START" | "STOP",                          # Required
              interface      => "ALL" | "HA" | "LAN1" | "LAN2" | "MGMT",# Required
              seconds_to_run => $num,                                      # Required
);
```

MODULE METHODS

The following functions are available to be applied to a traffic capture control object.

Infoblox::Grid::Member->traffic_capture( )

Use this function to control traffic capture for the Grid Member. See Infoblox::Grid::Member->traffic_capture() for parameters and return values. Note that the member object must be retrieved from the appliance in order for the traffic capture method to be available.

METHODS

This section describes all the methods that you can use to control traffic capture for a grid member.

action( )

Use this method to select the traffic capture action.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

Valid values are "START" or "STOP".

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Setting the action
$control->action("START");
```

interface( )

Use this method to specify the interface on which you capture traffic.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

Valid values are "ALL", "HA", "LAN", "LAN2" or "MGMT".

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Choosing the interface
$control->interface("ALL");
```

seconds_to_run( )

Use this method to select the number of seconds for which you want to run the traffic capture.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

Valid value is an integer specifying the number of seconds.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Setting the capture interfal
$control->seconds_to_run(5);
```

SAMPLE CODE

The following sample code demonstrates the different functions that can be applied to a capture control object. This sample also includes error handling for the operations.

#Preparation prior to traffic capture object operations

```
#PROGRAM STARTS: Include all the modules that will be used

use strict;
use Infoblox;

#Create a session to the Infoblox appliance
my $session = Infoblox::Session->new(
    master   => "192.168.1.2",
    username => "admin",
    password => "infoblox"
);
unless ($session) {
   die("Construct session failed: ",
       Infoblox::status_code() . ":" . Infoblox::status_detail());
}
print "Session created successfully\n";
```

#Start traffic capture for a member

```
# Get the grid member object.
my $object = $session->get(
    object => "Infoblox::Grid::Member",
    name => "infoblox.localdomain",
);

unless ($object) {
      die("get grid member failed: ",
      $session->status_code() . ":" . $session->status_detail());
}
print "grid member get successful\n";
```

```
my $capture = Infoblox::Grid::Member::Capture::Control->new(
                                                            action => 'START',
                                                            interface => 'ALL',
                                                            seconds_to_run => 2,
                                                           );
unless ($object->traffic_capture($capture)) {
      die("traffic capture failed: ",
      $session->status_code() . ":" . $session->status_detail());
}

print "traffic capture started\n";

####PROGRAM ENDS####
```

AUTHOR

Infoblox Inc. http://www.infoblox.com/

SEE ALSO

Infoblox::Session, Infoblox::Grid::Member

COPYRIGHT

Copyright (c) 2017 Infoblox Inc.

Infoblox::Grid::Member::Capture::Status - Traffic capture status object

- NAME
- DESCRIPTION
- MODULE METHODS
  - Infoblox::Grid::Member->traffic_capture( )
- METHODS
  - file_exists( )
  - file_size( )
  - status( )
- SAMPLE CODE
- AUTHOR
- SEE ALSO
- COPYRIGHT

NAME

Infoblox::Grid::Member::Capture::Status - Traffic capture status object

---

DESCRIPTION

You can retrieve the traffic capture status for a grid member via this object.

MODULE METHODS

The following functions are available to be applied to a traffic capture status object.

Infoblox::Grid::Member->traffic_capture( )

Use this function to retrieve the traffic capture status for the Grid Member. See Infoblox::Grid::Member->traffic_capture() for parameters and return values. Note that the member object must be retrieved from the appliance in order for the traffic capture method to be available.

METHODS

This section describes all the methods that explain a grid member's traffic capture status. All members are read-only.

file_exists( )

Use this method to retrieve a traffic capture file if it exists for the member. If the file exists, it can be downloaded using the export_data method in Infoblox::Session.

Returns

The method returns the attribute value, which can be "true" or "false".

Example

```
#Get file_exists
$exists = capstatus->file_exists();
```

file_size( )

Use this method to retrieve the size of the traffic capture file for the member.

Returns

The method returns the attribute value, which is the file size in bytes.

Example

```
#Get the file_size
$size = capstatus->file_size();
```

status( )

Use this method to retrieve the status of the capture operation for the grid member.

Returns

The method returns the attribute value, which can be "RUNNING", "STOPPED" or "UNKNOWN".

Example

```
#Get the status
$status = capstatus->status();
```

SAMPLE CODE

The following sample code demonstrates the different functions that can be applied to a capture status object. This sample also includes error handling for the operations.

#Preparation prior to traffic capture object operations

```
#PROGRAM STARTS: Include all the modules that will be used

use strict;
use Infoblox;

#Create a session to the Infoblox appliance
my $session = Infoblox::Session->new(
    master   => "192.168.1.2",
    username => "admin",
    password => "infoblox"
);
unless ($session) {
   die("Construct session failed: ",
       Infoblox::status_code() . ":" . Infoblox::status_detail());
}
print "Session created successfully\n";
```

#Start traffic capture for a member

```
# Get the grid member object.
my $object = $session->get(
    object => "Infoblox::Grid::Member",
    name => "infoblox.localdomain",
);

unless ($object) {
      die("get grid member failed: ",
      $session->status_code() . ":" . $session->status_detail());
}
print "grid member get successful\n";

my $capture = Infoblox::Grid::Member::Capture::Status->new(
                                                            action => 'START',
                                                            interface => 'ALL',
                                                            seconds_to_run => 2,
                                                           );
unless ($object->traffic_capture($capture)) {
      die("traffic capture failed: ",
      $session->status_code() . ":" . $session->status_detail());
}

print "traffic capture started\n";

my $capstatus = $object->traffic_capture();

print "The capture is " . $capstatus->status() . "\n";

####PROGRAM ENDS####
```

AUTHOR

Infoblox Inc. http://www.infoblox.com/

SEE ALSO

Infoblox::Session, Infoblox::Grid::Member

COPYRIGHT

Copyright (c) 2017 Infoblox Inc.

Infoblox::Grid::Member::PNodeToken - Physical Node Token object.

- NAME
- DESCRIPTION
- CONSTRUCTOR
- METHODS
  - Infoblox::Grid::Member->physical_oid( )
  - Infoblox::Grid::Member->token_exp_date( )
  - Infoblox::Grid::Member->token( )
- SAMPLE CODE
- AUTHOR
- SEE ALSO
- COPYRIGHT

NAME

Infoblox::Grid::Member::PNodeToken - Physical Node Token object.

DESCRIPTION

This object is used as a permission token for a pre-provisioned member to automatically join the Grid.

CONSTRUCTOR

This object actually have no constructor, it is only returned by create_token and read_token function calls of Infoblox::Grid::Member class.

METHODS

This section describes all the methods in the Infoblox::Session module that you can apply to a License object.

Infoblox::Grid::Member->physical_oid( )

Use this method to retrieve the OID of the physical node for which the token is created.

Parameter

None

Returns

The method returns the attribute value, which is an integer number.

Example

```
# get the physical_oid value for the token
$pnode_token_obj->physical_oid()
```

Infoblox::Grid::Member->token_exp_date( )

Use this method to retrieve the date and time when the token will expire. The value is presented as a Unix timestamp.

Parameter

None

Returns

The method returns the attribute value, which is an integer number.

Example

```
# get the token_exp_date value for the token
$pnode_token_obj->token_exp_date()
```

Infoblox::Grid::Member->token( )

Use this method to retrieve the value (in text string) of the physical node token.

Parameter

None

Returns

The method returns the attribute value, which is a string.

Example

```
# get the token value itself
$pnode_token_obj->token()
```

SAMPLE CODE

The following sample code demonstrates how to create a token for the newly created pre-provisioned member.

```
#!/usr/bin/perl

use strict;
use Infoblox;

# get session for all subsequent operations
my $session = Infoblox::Session->new(master => 'localhost', username => 'admin', password => 'infoblox');

# create an object for the Grid member
my $grid_member = Infoblox::Grid::Member->new(
     name                    => "hostname.com",
     ipv4addr                => "3.0.0.2",
     mask                    => "255.0.0.0",
     gateway                 => "3.0.0.1",
     platform                => "VNIOS",
);
# add member
$session->add($grid_member);

# create pre-provisioning info objects
my $hw = Infoblox::Grid::Member::PreProvisioning::Hardware->new(
    hwtype          => 'IB-VNIOS',
    hwmodel         => 'CP-V1400',
);
my $pp = Infoblox::Grid::Member::PreProvisioning->new(
    hardware_info   => [$hw],
    licenses        => ['dns', 'dhcp'],
);
$grid_member->pre_provisioning($pp);
# ...and add pre-provisioning info to the Grid member
$session->modify($grid_member);

# Create tokens for the Grid member
# (actually the only token, as the member is not a HA-pair)
my $tokens_ref = $grid_member->create_token();

# Read all tokens for the grid member.
# Actually, we can skip the read_token call,
# as we already have tokens from previous
# create_token call in this example
# $tokens_ref = $grid_member->read_token();

if (defined $tokens_ref) {
   my @tokens = @$tokens_ref;
   # Iterate through all tokens in the array and
   # print their values.
   for (my $i = 0; $i < scalar @tokens; $i++) {
      print "Token $i:\n";
      print "physical_oid: " . $tokens[$i]->physical_oid(). "\n";
      print "token_exp_date: " . $tokens[$i]->token_exp_date() . "\n";
      print "token: " . $tokens[$i]->token() . "\n";
   }
}
```

AUTHOR

Infoblox Inc. http://www.infoblox.com/

SEE ALSO

Infoblox::Grid::Member

COPYRIGHT

Copyright (c) 2017 Infoblox Inc.

Infoblox::Grid::Member::CaptivePortal::File - Captive Portal File object

- NAME
- DESCRIPTION
- MODULE METHODS
  - Infoblox::Grid::Member::CaptivePortal->files( )
- METHODS
  - name( )
  - type( )
- AUTHOR
- SEE ALSO
- COPYRIGHT

NAME

Infoblox::Grid::Member::CaptivePortal::File - Captive Portal File object

DESCRIPTION

This object is used to represent files uploaded to the captive portal. All methods are read-only.

MODULE METHODS

The following functions can be applied to a CaptivePortal::File object.

Infoblox::Grid::Member::CaptivePortal->files( )

Use this function to retrieve a reference to an array of Infoblox::Grid::Member::CaptivePortal::File objects that represent the files uploaded to the captive portal.

Example

```
#Retrieve the captive portal object
my $member = $session->get(object => 'Infoblox::Grid::Member::CaptivePortal', name =>
'cportal.com');

# Access its files
my @files = @{$member->files()};
print $files[0]->name();
```

METHODS

This section describes all the methods that you can use to retrieve the attribute value of a file uploaded to a captive portal.

name( )

Use this method to retrieve the name of the uploaded file.

Returns

The method returns the attribute value.

Example

```
#Get name
my $name = $service_status->name();
```

type( )

Use this method to retrieve the type of the uploaded file.

Returns

The method returns the attribute value. Valid values are "Logo", "Footer", "Header", and "AUP"

Example

```
#Get type
my $type = $service_type->type();
```

AUTHOR

Infoblox Inc. http://www.infoblox.com/

SEE ALSO

Infoblox::Grid::Member::CaptivePortal/>

COPYRIGHT

Copyright (c) 2017 Infoblox Inc.

Infoblox::Grid::Member::CaptivePortal - Captive Portal object.

- NAME
- DESCRIPTION
- CONSTRUCTOR
- SESSION METHODS
  - Infoblox::Session->get( )
  - Infoblox::Session->modify( )
- METHODS
  - authn_server_group ( )
  - company_name ( )
  - enable_syslog_auth_failure ( )
  - enable_syslog_auth_success ( )
  - enable_user_type ( )
  - encryption ( )
  - files ( )
  - guest_custom_field1_name ( )
  - guest_custom_field1_required ( )
  - guest_custom_field2_name ( )
  - guest_custom_field2_required ( )
  - guest_custom_field3_name ( )
  - guest_custom_field3_required ( )
  - guest_custom_field4_name ( )
  - guest_custom_field4_required ( )
  - guest_email_required ( )
  - guest_first_name_required ( )
  - guest_last_name_required ( )
  - guest_middle_name_required ( )
  - guest_phone_required ( )
  - helpdesk_message ( )
  - listen_address ( )
  - name ( )
  - network_view ( )
  - port ( )
  - service_enabled ( )
  - syslog_auth_failure_level ( )
  - syslog_auth_success_level ( )
  - welcome_message ( )
- SAMPLE CODE
- AUTHOR
- SEE ALSO
- COPYRIGHT

NAME

Infoblox::Grid::Member::CaptivePortal - Captive Portal object.

DESCRIPTION

This object represents the captive portal configuration.

CONSTRUCTOR

The Infoblox::Grid::Member::CaptivePortal object is automatically generated upon the successful installation of the Infoblox appliance. It does not require manual construction.

SESSION METHODS

This section describes all the methods in the Infoblox::Session module that you can apply to a Captive Portal object.

Infoblox::Session->get( )

Use this method to retrieve matching objects from the Infoblox appliance. See Infoblox::Session->get() for parameters and return values.

Key Reference

```
Apply the following attribute to get a specific Captive Portal object:

 name - Required. A domain name in string format.
```

Example

```
my @retrieved objs   $session  get
```

```
                =          ->    (
object => "Infoblox::Grid::Member::CaptivePortal",
name   => "infoblox.localdomain"
);
```

Infoblox::Session->modify( )

Use this method to modify objects in the Infoblox appliance. See Infoblox::Session->modify() for parameters and return values.

Example

```
#Use this method to modify the welcome message.
$captive_portal->welcome_message("New welcome message");
#Submit modification
my $response = $session->modify( $captive_portal );
```

METHODS

This section describes all the methods that you can use to configure and retrieve the attribute values of the captive portal.

authn_server_group ( )

Use this method to set or retrieve the authentication server group assigned to this captive portal.

Parameter

The valid values are an Infoblox::Grid::Admin::AdAuthService, an Infoblox::Grid::Admin::RadiusAuthService or an Infoblox::LDAP::AuthService object. The default value is undefined.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Getting authn_server_group
my $authn_server_group = $captive_portal->authn_server_group();
#Modifying authn_server_group
$captive_portal->authn_server_group($adservice);
```

company_name ( )

Use this method to set or retrieve the company name that appears in the guest registration page.

Parameter

The valid value is a string. The default value is undefined.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Getting company_name
my $company_name = $captive_portal->company_company_name();
#Modifying company_name
$captive_portal->company_name('Company Name');
```

enable_syslog_auth_failure ( )

Use this method to set or retrieve the flag that controls whether authentication failures are logged to syslog.

Parameter

Specify "true" to enable logging or "false" to disable it. The default value is "false".

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Getting enable_syslog_auth_failure
```

```
my $enable_syslog_auth_failure = $captive_portal->enable_syslog_auth_failure();
#Modifying enable_syslog_auth_failure
$captive_portal->enable_syslog_auth_failure('true');
```

enable_syslog_auth_success ( )

Use this method to set or retrieve the flag that controls whether successful authentications are logged to syslog.

Parameter

Specify "true" to enable logging or "false" to disable it. The default value is "true".

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Getting enable_syslog_auth_success
my $enable_syslog_auth_success = $captive_portal->enable_syslog_auth_success();
#Modifying enable_syslog_auth_success
$captive_portal->enable_syslog_auth_success('false');
```

enable_user_type ( )

Use this method to set or retrieve the type of user to be enabled for the captive portal.

Parameter

Valid values are "Authenticated", "Guest" or "Both". The default value is "Authenticated".

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Getting enable_user_type
my $enable_user_type = $captive_portal->enable_user_type();
#Modifying enable_user_type
$captive_portal->enable_user_type('Both');
```

encryption ( )

Use this method to set or retrieve the encryption the captive portal uses.

Parameter

Valid values are 'None' or 'SSL'. The default value is 'None'.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Getting encryption
my $encryption = $captive_portal->encryption();
#Modifying encryption
$captive_portal->encryption('SSL');
```

files ( )

Use this method to set or retrieve the list of files associated with the captive portal.

Parameter

The valid value is an array reference that contains Infoblox::Grid::Member::CaptivePortal::File object(s). New files must be added using Infoblox::Session->import_data(). However, if a file is removed from this list, it is also removed from the captive portal.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Getting files
my $files = $captive_portal->files();
#Modifying files
$captive_portal->files([$logo]);
```

guest_custom_field1_name ( )

Use this method to set or retrieve the name of the custom field that you are adding to the guest registration page.

Parameter

The valid value is a string. The default value is undefined. The maximum length is 200 bytes.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Getting guest_custom_field1_name
my $guest_custom_field1_name = $captive_portal->guest_custom_field1_guest_custom_field1_name();
#Modifying guest_custom_field1_name
$captive_portal->guest_custom_field1_name('Extra field 1');
```

guest_custom_field1_required ( )

Use this method to set or retrieve the flag that controls whether the custom field is required.

Parameter

Specify "true" to set the flag or "false" to disable it. The default value is "false".

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Getting guest_custom_field1_required
my $guest_custom_field1_required = $captive_portal->guest_custom_field1_required();
#Modifying guest_custom_field1_required
$captive_portal->guest_custom_field1_required('true');
```

guest_custom_field2_name ( )

Use this method to set or retrieve the name of the custom field that you are adding to the guest registration page.

Parameter

The valid value is a string. The default value is undefined. The maximum length is 200 bytes.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Getting guest_custom_field2_name
my $guest_custom_field2_name = $captive_portal->guest_custom_field2_guest_custom_field2_name();
#Modifying guest_custom_field2_name
$captive_portal->guest_custom_field2_name('Extra field 2');
```

guest_custom_field2_required ( )

Use this method to set or retrieve the flag that controls whether the custom field is required.

Parameter

Specify "true" to set the flag or "false" to disable it. The default value is "false".

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Getting guest_custom_field2_required
my $guest_custom_field2_required = $captive_portal->guest_custom_field2_required();
#Modifying guest_custom_field2_required
$captive_portal->guest_custom_field2_required('true');
```

guest_custom_field3_name ( )

Use this method to set or retrieve the name of the custom field that you are adding to the guest registration page.

Parameter

The valid value is a string. The default value is undefined. The maximum length is 200 bytes.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Getting guest_custom_field3_name
my $guest_custom_field3_name = $captive_portal->guest_custom_field3_guest_custom_field3_name();
#Modifying guest_custom_field3_name
$captive_portal->guest_custom_field3_name('Extra field 3');
```

guest_custom_field3_required ( )

Use this method to set or retrieve the flag that controls whether the custom field is required.

Parameter

Specify "true" to set the flag or "false" to disable it. The default value is "false".

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Getting guest_custom_field3_required
my $guest_custom_field3_required = $captive_portal->guest_custom_field3_required();
#Modifying guest_custom_field3_required
$captive_portal->guest_custom_field3_required('true');
```

guest_custom_field4_name ( )

Use this method to set or retrieve the name of the custom field that you are adding to the guest registration page.

Parameter

The valid value is a string. The default value is undefined. The maximum length is 200 bytes.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Getting guest_custom_field4_name
my $guest_custom_field4_name = $captive_portal->guest_custom_field4_guest_custom_field4_name();
#Modifying guest_custom_field4_name
$captive_portal->guest_custom_field4_name('Extra field 4');
```

guest_custom_field4_required ( )

Use this method to set or retrieve the flag that controls whether the custom field is required.

Parameter

Specify "true" to set the flag or "false" to disable it. The default value is "false".

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Getting guest_custom_field4_required
my $guest_custom_field4_required = $captive_portal->guest_custom_field4_required();
#Modifying guest_custom_field4_required
$captive_portal->guest_custom_field4_required('true');
```

guest_email_required ( )

Use this method to set or retrieve the flag that controls whether the email address of the guest is required.

Parameter

Specify "true" to set this flag or "false" to disable it. The default value is "true".

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Getting guest_email_required
my $guest_email_required = $captive_portal->guest_email_required();
#Modifying guest_email_required
$captive_portal->guest_email_required('false');
```

guest_first_name_required ( )

Use this method to set or retrieve the flag that controls whether the first name of the guest is required.

Parameter

Specify "true" to enable or "false" to disable it. The default value is "true".

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Getting guest_first_name_required
my $guest_first_name_required = $captive_portal->guest_first_name_required();
#Modifying guest_first_name_required
$captive_portal->guest_first_name_required('false');
```

guest_last_name_required ( )

Use this method to set or retrieve the flag that controls whether the last name of the guest is required.

Parameter

Specify "true" to set the flag or "false" to disable it. The default value is "true".

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Getting guest_last_name_required
my $guest_last_name_required = $captive_portal->guest_last_name_required();
#Modifying guest_last_name_required
$captive_portal->guest_last_name_required('false');
```

guest_middle_name_required ( )

Use this method to set or retrieve the flag that controls whether the middle name of the guest is required.

Parameter

Specify "true" to set the flag or "false" to disable it. The default value is "false".

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Getting guest_middle_name_required
my $guest_middle_name_required = $captive_portal->guest_middle_name_required();
#Modifying guest_middle_name_required
$captive_portal->guest_middle_name_required('true');
```

guest_phone_required ( )

Use this method to set or retrieve the flag that controls whether the phone number of the guest is required.

Parameter

Specify "true" to set this flag or "false" to disable it. The default value is "true".

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Getting guest_phone_required
my $guest_phone_required = $captive_portal->guest_phone_required();
#Modifying guest_phone_required
$captive_portal->guest_phone_required('false');
```

helpdesk_message ( )

Use this method to set or retrieve the helpdesk message that appears in the guest registration page.

Parameter

The valid value is a string. The default value is undefined.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Getting helpdesk_message
my $helpdesk_message = $captive_portal->helpdesk_message();
#Modifying helpdesk_message
$captive_portal->helpdesk_message('In case of problems, please contact us at 555-555-5555');
```

listen_address ( )

Use this method to set or retrieve the IP address on which the captive portal listens.

Parameter

The valid values are 'VIP', 'LAN2' or an IPv4 address. The default value is 'VIP'.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Getting listen_address
my $listen_address = $captive_portal->listen_address();
#Modifying listen_address
$captive_portal->listen_address('VIP');
```

name ( )

Use this method to retrieve the hostname of the grid member that hosts the captive portal.

Returns

The method returns the attribute value.

Example

```
#Getting name
my $name = $captive_portal->name();
```

network_view ( )

Use this method to set or retrieve the network view of the captive portal.

Parameter

The valid value is a network view name. The default is the system default network view.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Getting network_view
my $network_view = $captive_portal->network_view();
#Modifying network_view
$captive_portal->network_view('otherview');
```

port ( )

Use this method to set or retrieve the TCP port used by the Captive Portal service. The port is required when the Captive Portal service is enabled.

Parameter

Valid values are between 1 and 63999. The default value is 4433. Please note that setting the port number to 80 or 443 might impact performance.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Getting the port
my $port = $captive_portal->port();
#Modifying the port
$captive_portal->port(4457);
```

service_enabled ( )

Use this method to set or retrieve the state of the captive portal service.

Parameter

Specify "true" to enable the service or "false" to disable it. The default value is "false".

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Getting service_enabled
my $service_enabled = $captive_portal->service_enabled();
#Modifying service_enabled
$captive_portal->service_enabled('false');
```

syslog_auth_failure_level ( )

Use this method to set or retrieve the syslog level at which authentication failures are logged.

Parameter

Valid values are "debug", "info", "notice", "warning", "err", "crit", "alert" or "emerg" . The default value is "info".

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Getting syslog_auth_failure_level
my $syslog_auth_failure_level = $captive_portal->syslog_auth_failure_level();
#Modifying syslog_auth_failure_level
$captive_portal->syslog_auth_failure_level('notice');
```

syslog_auth_success_level ( )

Use this method to set or retrieve the syslog level at which successful authentications are logged.

Parameter

Valid values are "debug", "info", "notice", "warning", "err", "crit", "alert" or "emerg" . The default value is "info".

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Getting syslog_auth_success_level
my $syslog_auth_success_level = $captive_portal->syslog_auth_success_level();
#Modifying syslog_auth_success_level
$captive_portal->syslog_auth_success_level('warning');
```

welcome_message ( )

Use this method to set or retrieve the welcome message that appears in the guest registration page.

Parameter

The valid value is a string. The default value is undefined.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Getting welcome_message
my $welcome_message = $captive_portal->welcome_message();
#Modifying welcome_message
$captive_portal->welcome_message('Welcome to our network!');
```

SAMPLE CODE

The following sample code demonstrates different operations that can be applied to a captive portal object, such as get and modify an object. This sample code also includes error handling for the operations.

#Preparation prior to operations on a captive portal object

```
#PROGRAM STARTS: Include all the modules that will be used
use strict;
use Infoblox;

#Create a session to the Infoblox appliance
my $session = Infoblox::Session->new(
    master   => "192.168.1.2",
    username => "admin",
    password => "infoblox"
);
unless ($session) {
   die("Construct session failed: ",
       Infoblox::status_code() . ":" . Infoblox::status_detail());
}
print "Session created successfully\n";

# Upload a new AUP to the captive portal, from now on
# the files() method will contain an Infoblox::Grid::Member::CaptivePortal::File
# object referencing this.
$session->import_data(
                      type     => 'captive_portal_file',
```

```
                        path    => 'aup.txt',
                        member  => '192.168.1.2',
                        cp_type => 'aup'
    )
    or die("file import failed: ",
            $session->status_code(), $session->status_detail());

    my $ads1 = Infoblox::Grid::Admin::RadiusAuthServer->new(
                                                            comment  => 'server 1',
                                                            fqdn_or_ip => '10.0.1.1',
                                                            shared_secret => 'secret1',
                                                           );

    my $ads2 = Infoblox::Grid::Admin::RadiusAuthServer->new(
                                                            comment  => 'server 2',
                                                            fqdn_or_ip => 'domain.com',
                                                            shared_secret => 'secret2',
                                                          );
```

#Create the RADIUS Auth Service.

```
    my $service = Infoblox::Grid::Admin::RadiusAuthService->new(
                                                                acct_retries   => 2000,
                                                                auth_retries   => 9,
                                                                cache_ttl      => 500,
                                                                disabled       => 'true',
                                                                enable_cache   => 'true',
                                                                recovery_ttl   => 50,
                                                                selection_mode => 'ROUND_ROBIN',
                                                                acct_timeout   => 1100,
                                                                auth_timeout   => 1200,
                                                                name           => 'some.name.com',
                                                                radius_servers => [$ads1, $ads2]
                                                              );

    unless($service){
           die("Construct service object failed: ",
                   Infoblox::status_code(). ":" .Infoblox::status_detail());
           }
    print "service object object created successfully.\n";

    $session->add($service)
        or die("Add service object failed: ",
                   $session->status_code(). ":" .$session->status_detail());

    print"Service object added successfully.\n";
```

#Get and modify the captive portal property object.

```
    #Get the captive portal object. Considering the id grid member named infoblox.localdomain was previously
created.
    my @object = $session->get(
        object => "Infoblox::Grid::Member::CaptivePortal",
        name   => "infoblox.localdomain"
        )
        or die("get captive portal failed: ",
             $session->status_code(), $session->status_detail());

    #Modify captive portal object.
    unless (scalar(@object) == 0) {
        my $object = $object[0];

        if ($object) {
            $object->authn_server_group($service);
            $object->guest_phone_required('false');
            $object->guest_custom_field1_name('Custom field 1');
            $object->guest_custom_field1_required('true');
            $object->guest_custom_field2_name('Custom field 2');
            $object->guest_custom_field2_required('true');

            # This will remove the AUP uploaded above
            $object->files([]);

            #Applying the changes
            $session->modify($object)
                or die("Modify captive portal failed: ",
                     $session->status_code(), $session->status_detail());

        }
```

```
    } else {
   print "No grid captive portal found.";
    }

####PROGRAM ENDS####
```

AUTHOR

Infoblox Inc. http://www.infoblox.com/

SEE ALSO

Infoblox::Grid::Member::CaptivePortal::File,Infoblox::Grid::Admin::AdAuthService,Infoblox::Grid::Admin::RadiusAuthService,Infoblox::Session,
Infoblox::Session->get(),Infoblox::Session->modify(),Infoblox::LDAP::AuthService

---

COPYRIGHT

Copyright (c) 2017 Infoblox Inc.

Infoblox::Grid::Member::DiscoveryProperties - Grid member discovery properties.

- NAME
- DESCRIPTION
- CONSTRUCTOR
- SESSION METHODS
  - Infoblox::Session->get( )
  - Infoblox::Session->modify( )
  - Infoblox::Session->search( )
- METHODS
  - sdn_configs( )
  - default_seed_routers( )
  - enable_service( )
  - cli_credentials( )
  - member( )
  - override_snmp3_credentials( )
  - override_cli_credentials( )
  - override_snmp_credentials( )
  - role( )
  - seed_routers( )
  - snmp3_credentials( )
  - snmp_credentials( )
  - test_credential( )
  - scan_interfaces( )
- AUTHOR
- SEE ALSO
- COPYRIGHT

NAME

Infoblox::Grid::Member::DiscoveryProperties - Grid member discovery properties.

DESCRIPTION

The object provides information about Grid member discovery properties.

CONSTRUCTOR

```
my $object = Infoblox::Grid::Member::DiscoveryProperties->new(
    'member'                    => $string,                     # Optional
    'enable_service'            => 'true' | 'false',            # Optional
    'scan_interfaces'           => [$si1, ...],                 # Optional
    'role'                      => 'DNM' | 'DNP' | 'NONE',      # Optional
    'seed_routers'              => [$seed1, ...],               # Optional
    'snmp3_credentials'         => [$snmp3, ...],               # Optional
    'override_snmp3_credentials' => 'true' | 'false',           # Optional
    'snmp_credentials'          => [$snmp, ...],                # Optional
    'override_snmp_credentials'  => 'true' | 'false',           # Optional
    'cli_credentials'           => [$cli1, ...],                # Optional
    'override_cli_credentials'   => 'true' | 'false',           # Optional
    'sdn_configs'               => [$conf1, ...],               # Optional
);
```

SESSION METHODS

This section describes all the methods in Infoblox::Session that you can apply to the object.

Infoblox::Session->get( )

Use this method to retrieve all the matching objects from the Infoblox appliance. See Infoblox::Session->get() for parameters and return values.

Key References

```
Apply the following attributes to get a specific object:

 member - Optional. The member name.
```

Example

```
my @retrieved_objs = $session->get(
    'object' => 'Infoblox::Grid::Member::DiscoveryProperties',
    'member' => 'member.com',
);
```

Infoblox::Session->modify( )

Use this method to modify an object in the Infoblox appliance. See Infoblox::Session->modify() for parameters and return values.

Example

```
# Use this method to modify role.
$object->role("DNP");
# Submit modification
my $response = $session->modify( $object );
```

Infoblox::Session->search( )

Use this method to search for objects in the Infoblox appliance. See Infoblox::Session->search() for parameters and return values.

Key References

```
Apply the following attributes to search for a object:

 member - Optional. The member name.
```

Example

```
# search for all objects
my @retrieved_objs = $session->search(
    'object' => 'Infoblox::Grid::Member::DiscoveryProperties',
    'member' => 'member.com',
);
```

METHODS

This section describes all the methods that you can use to set or retrieve the attribute values of the object.

sdn_configs( )

Use this method to set or retrieve the list of SDN configurations for a given probe Member.

Parameter

Valid value is an array reference that contains Infoblox::Grid::Member::Discovery::SDNConfig objects.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
# Get attribute value
my $value = $object->sdn_configs();
# Construct SDN config
my $conf = Infoblox::Grid::Member::Discovery::SDNConfig->new(
    'sdn_type'       => 'CISCO_APIC',
    'addresses'      => ['apic.aci.com'],
    'network_view'   => 'default',
    'protocol'       => 'HTTPS',
    'handle'         => 'ACIConf1',
    'password'       => 'infoblox',
    'username'       => 'admin',
    'ca_certificate' => $cert,
    'comment'        => 'test_sdn_config',
);
$object->sdn_configs([$conf]);
```

default_seed_routers( )

Use this method to retrieve a list of default seed routers. This is a read-only attribute.

Parameter

None

Returns

The method returns the attribute value.

Example

```
# Get attribute value
my $value = $object->default_seed_routers();
```

enable_service( )

Use this method to enable or disable the discovery service.

Parameter

Specify "true" to enable the discovery service or "false" to disable it. Default value is "false".

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
# Get attribute value
my $value = $object->enable_service();
# Modify attribute value
$object->enable_service('true');
```

cli_credentials( )

Use this method to set or retrieve the Discovery CLI credentials.

Setting this method to a defined value implicitly sets the override_cli_credentials method to "true". Setting the parameter to undefined causes the appliance to use the grid default and automatically resets the override_cli_credentials attribute to "false".

Note that when cli_credentials is set to a defined value and override_cli_credentials is set to "false", the last operation takes precedence. Thus the sequence $object->cli_credentials([$snmp]); $object->override_cli_credentials("false"); will set override_cli_credentials to "false", and the sequence $object->override_cli_credentials("false"); $object->cli_credentials([$snmp]); will result in override_cli_credentials="true".

Parameter

Valid value is an array reference that contains Infoblox::Grid::Discovery::CLICredential objects.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
# Get attribute value
my $value = $object->cli_credentials();
# Modify attribute value
$object->cli_credentials([$cli1, $cli2]);
```

member( )

Use this method to set or retrieve the member name.

Parameter

The member name in string format.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
# Get attribute value
my $value = $object->member();
```

```
# Modify attribute value
$object->member('infoblox.com');
```

override_snmp3_credentials( )

The override_snmp3_credentials attribute controls whether the snmp3_credentials value in the member is used, instead of the grid default.

The override_snmp3_credentials attribute can be specified explicitly. It is also set implicitly when snmp3_credentials is set to a defined value.

Parameter

Specify "true" to set the override_snmp3_credentials flag or "false" to deactivate/unset it. The default value is "false".

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
# Get attribute value
my $value = $object->override_snmp3_credentials();
# Modify attribute value
$object->override_snmp3_credentials('true');
```

override_cli_credentials( )

The override_cli_credentials attribute controls whether the cli_credentials value in the member is used, instead of the grid default.

The override_cli_credentials attribute can be specified explicitly. It is also set implicitly when cli_credentials is set to a defined value.

Parameter

Specify "true" to set the override_cli_credentials flag or "false" to deactivate/unset it. The default value is "false".

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
# Get attribute value
my $value = $object->override_cli_credentials();
# Modify attribute value
$object->override_cli_credentials('true');
```

override_snmp_credentials( )

The override_snmp_credentials attribute controls whether the snmp_credentials value in the member is used, instead of the grid default.

The override_snmp_credentials attribute can be specified explicitly. It is also set implicitly when snmp_credentials is set to a defined value.

Parameter

Specify "true" to set the override_snmp_credentials flag or "false" to deactivate/unset it. The default value is "false".

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
# Get attribute value
my $value = $object->override_snmp_credentials();
# Modify attribute value
$object->override_snmp_credentials('true');
```

role( )

Use this method to set or retrieve the role of the discovery member.

Parameter

Valid value is 'DNM', 'DNP' or 'NONE'.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
# Get attribute value
my $value = $object->role();
# Modify attribute value
$object->role('NONE');
```

seed_routers( )

Use this method to set or retrieve seed routers.

Parameter

Valid value is an array reference that contains Infoblox::Grid::Discovery::SeedRouter objects.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
# Get attribute value
my $value = $object->seed_routers();
# Modify attribute value
$object->seed_routers([$seed1, $seed2]);
```

snmp3_credentials( )

Use this method to set or retrieve discovery SNMPv3 credentials.

Setting this method to a defined value implicitly sets the override_snmp3_credentials method to "true". Setting the parameter to undefined causes the appliance to use the grid default and automatically resets the override_snmp3_credentials attribute to "false".

Note that when snmp3_credentials is set to a defined value and override_snmp3_credentials is set to "false", the last operation takes precedence. Thus the sequence $object->snmp3_credentials([$snmp3]); $object->override_snmp3_credentials("false"); will set override_snmp3_credentials to "false", and the sequence $object->override_snmp3_credentials("false"); $object->snmp3_credentials([$snmp3]); will result in override_snmp3_credentials="true".

Parameter

Valid value is an array reference that contains Infoblox::Grid::Discovery::SNMP3Credential objects.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
# Get attribute value
my $value = $object->snmp3_credentials();
# Modify attribute value
$object->snmp3_credentials([$snmp3]);
```

snmp_credentials( )

Use this method to set or retrieve SNMPv1 and SNMPv2 credentials.

Setting this method to a defined value implicitly sets the override_snmp_credentials method to "true". Setting the parameter to undefined causes the appliance to use the grid default and automatically resets the override_snmp_credentials attribute to "false".

Note that when snmp_credentials is set to a defined value and override_snmp_credentials is set to "false", the last operation takes precedence. Thus the sequence $object->snmp_credentials([$snmp]); $object->override_snmp_credentials("false"); will set override_snmp_credentials to "false", and the sequence $object->override_snmp_credentials("false"); $object->snmp_credentials([$snmp]); will result in override_snmp_credentials="true".

Parameter

Valid value is an array reference that contains Infoblox::Grid::Discovery::SNMPCredential objects.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
# Get attribute value
my $value = $object->snmp_credentials();
# Modify attribute value
$object->snmp_credentials([$snmp]);
```

test_credential( )

Use this method to test credentials.

Parameter

```
id               - Optional. The test credential ID.
```

The following members are supported when id is defined:

```
start            - Optional. The start position of the text.
```

The following members are supported when id is not defined:

```
view             - Optional. The network view name. The default value is the system-defined default
network view.

type             - Optional if one of the following is defined: cli_credentials, snmp_credential or
snmp3_credential. Otherwise, this member is required. The valid value is 'CLI', 'SNMP' or 'SNMP3'.

device           - Required if address is not defined. The Infoblox::Grid::Discovery::Device object.

member           - Optional. The discovery member name.

address          - Required if device is not defined. The IP address of the device.

cli_credentials  - Optional. At least one of the following must be defined: cli_credentials,
snmp_credential, or  snmp3_credential. The array reference that contains
Infoblox::Grid::Discovery::CLICredential objects.

snmp_credential  - Optional. At least one of the following must be defined: cli_credentials,
snmp_credential, or  snmp3_credential. The Infoblox::Grid::Discovery::SNMPCredential object.

snmp3_credential - Optional. At least one of the following must be defined: cli_credentials,
snmp_credential, or snmp3_credential. The Infoblox::Grid::Discovery::SNMP3Credential object.
```

Returns

The method returns an Infoblox::Grid::Discovery::TestCredential object when the operation succeeds, and returns false when the operation fails.

Example

```
# Get a test credential ID
my $result = $object->test_credential(
   'address'         => '10.0.0.1',
   'snmp_credential' => $snmp,
);

# Get detailed information by id
if (ref($result) eq 'Infoblox::Grid::Discovery::TestCredential') {
    $result = $object->test_credential('id' => $result->id());
}
```

scan_interfaces( )

Use this method to set or retrieve the scan interface configuration for a given probe member.

Parameter

Valid value is an array reference that contains Infoblox::Grid::Member::Discovery::ScanInterface objects.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
# Get attribute value
my $value = $object->scan_interfaces();
# Modify attribute value
$object->scan_interfaces([$interface1, $interface2]);
```

AUTHOR

Infoblox Inc. http://www.infoblox.com/

SEE ALSO

Infoblox::Session, Infoblox::Grid::Member::Discovery::SDNConfig, Infoblox::Grid::Discovery::SNMPCredential, Infoblox::Grid::Discovery::SNMP3Credential, Infoblox::Grid::Discovery::SeedRouter, Infoblox::Grid::Discovery::TestCredential, Infoblox::Grid::Discovery::CLICredential, Infoblox::Grid::Member::Discovery::ScanInterface

---

COPYRIGHT

Copyright (c) 2017 Infoblox Inc.

Infoblox::Grid::Member::Discovery::SDNConfig - The SDN controller configuration structure.

- NAME
- DESCRIPTION
- CONSTRUCTOR
- MODULE METHODS
  - Infoblox::Grid::Member::DiscoveryProperties->sdn_configs( )
- SESSION METHODS
- METHODS
  - sdn_type( )
  - addresses( )
  - network_view( )
  - protocol( )
  - handle( )
  - password( )
  - username( )
  - api_key( )
  - on_prem( )
  - use_global_proxy( )
  - ca_certificate( )
  - comment( )
  - uuid( )
  - network_interface_type( )
  - network_interface_virtual_ip( )
- AUTHOR
- SEE ALSO
- COPYRIGHT

NAME

Infoblox::Grid::Member::Discovery::SDNConfig - The SDN controller configuration structure.

DESCRIPTION

The object provides information about the SDN controller configuration.

CONSTRUCTOR

```
my $object = Infoblox::Grid::Member::Discovery::SDNConfig->new(
    'sdn_type'                    => 'CISCO_APIC' | 'MERAKI', # Required
    'addresses'                   => ARRAY[$ipaddr | $fqdn],  # Required
    'network_view'                => $string,                 # Optional
    'protocol'                    => 'HTTP' | 'HTTPS',        # Optional
    'handle'                      => $string,                 # Required
    'password'                    => $string,                 # Optional
    'username'                    => $string,                 # Optional
    'api_key'                     => $string,                 # Optional
    'on_prem'                     => $bool,                   # Optional
    'use_global_proxy'            => $bool,                   # Optional
    'ca_certificate'              => $ca_cert,                # Optional
    'comment'                     => $string,                 # Optional
    'uuid'                        => $string,                 # Optional
    'network_interface_type'      => 'LAN1', 'LAN2', 'MGMT'   # Optional
    'network_interface_virtual_ip' => $vip                    # Optional
);
```

MODULE METHODS

The following functions can be applied to a SDN controller configuration structure.

Infoblox::Grid::Member::DiscoveryProperties->sdn_configs( )

Use this method to set or retrieve the SDN controller configurations configured for the Grid member discovery properties. See Infoblox::Grid::Member::DiscoveryProperties->sdn_configs() for parameters and return values.

Example

```
#Get sdn_configs value
my $sdn_configs = $discovery_properties->sdn_configs();
```

```
#Modify sdn_configs
$discovery_properties->sdn_configs([$sdn_config1, $sdn_config2]);
```

SESSION METHODS

The object does not support any session methods.

METHODS

This section describes all the methods that you can use to set or retrieve the attribute values of the object.

sdn_type( )

Use this method to set or retrieve SDN controller type.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

The valid values are 'CISCO_APIC' and 'MERAKI'.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
# Get attribute value
my $value = $object->sdn_type();
# Modify attribute value
my $result = $object->sdn_type('CISCO_APIC');
```

addresses( )

Use this method to set or retrieve SDN controller address or FQDN.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

The desired SDN address or Fully-Qualified Domain Name (FQDN) in a string format as array.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
# Get attribute value
my $value = $object->addresses();
# Modify attribute value
my $result = $object->addresses(['10.11.0.1']);
```

network_view( )

Use this method to set or retrieve the network view associated with SDN controller.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

The valid value is a network view name that is associated with a SDN controller in a string format.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
# Get attribute value
my $value = $object->network_view();
```

```
# Modify attribute value
my $result = $object->network_view('papi_network_view');
```

protocol( )

Use this method to set or retrieve the connection protocol for the SDN controller.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

The valid values are 'HTTP' and 'HTTPS'.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
# Get attribute value
my $value = $object->protocol();
# Modify attribute value
$object->protocol('HTTP');
```

handle( )

Use this method to set or retrieve SDN controller unique configuration handle.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

The valid value is a desired SDN controller unique configuration handle in a string format.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
# Get attribute value
my $value = $object->handle();
# Modify attribute value
$object->handle('handle');
```

password( )

Use this method to set SDN controller login password. This is a write-only attribute.

Include the specified parameter to set the attribute value.

Parameter

SDN controller login password.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

Example

```
# Modify attribute value
$object->password('password');
```

username( )

Use this method to set or retrieve SDN controller login name.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

The valid value is a desired SDN controller login name in a string format.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
# Get attribute value
my $value = $object->username();
# Modify attribute value
$object->username('username');
```

api_key( )

Use this method to set or retrieve SDN controller api key.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

The valid value is a desired SDN controller api key in a string format.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
# Get attribute value
my $value = $object->api_key();
# Modify attribute value
$object->api_key('api_key');
```

on_prem( )

Use this method to set or retrieve SDN controller on-prem value. True if controller is on-premises. Otherwise we consider it is in cloud.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

The valid value is a desired SDN controller on-prem in a boolean format.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
# Get attribute value
my $value = $object->on_prem();
# Modify attribute value
$object->on_prem(true);
```

use_global_proxy( )

Use this method to set or retrieve SDN controller use_global_proxy value. Use global proxy settings to access SDN controller.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

The valid value is a desired SDN controller use_global_proxy value in a boolean format.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
# Get attribute value
my $value = $object->use_global_proxy();
# Modify attribute value
```

```
$object->use_global_proxy(false);
```

ca_certificate( )

Use this method to set or retrieve the CA certificate.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

The valid value is an Infoblox::Grid::CACertificate object.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
# Get attribute value
my $value = $object->ca_certificate();
# Modify attribute value
my $result = $object->ca_certificate($ca_cert);
```

comment( )

Use this method to set or retrieve the descriptive comment for the SDN controller configuration.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

The valid value is a desired comment in a string format.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
# Get attribute value
my $value = $object->comment();
# Modify attribute value
$object->comment('SDN configuration');
```

uuid( )

Use this method to set or retrieve SDN controller configuration UUID.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

The valid value is a desired SDN controller uuid in a string format.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
# Get attribute value
my $value = $object->uuid();
# Modify attribute value
$object->uuid('ABCDEF');
```

network_interface_type( )

Use this method to set or retrieve SDN controller network interface type.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

The valid values are 'LAN1', 'LAN2', 'MGMT'.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
# Get attribute value
my $value = $object->network_interface_type();
# Modify attribute value
$object->network_interface_type('LAN1');
```

network_interface_virtual_ip( )

Use this method to set or retrieve SDN controller network interface virtual ip.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

The valid value is a desired SDN controller interface virtual ip in a string format in case network interface type is VLAN.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
# Get attribute value
my $value = $object->network_interface_virtual_ip();
# Modify attribute value
$object->network_interface_virtual_ip(.);
```

AUTHOR

Infoblox Inc. http://www.infoblox.com/

SEE ALSO

Infoblox::Grid::Member::DiscoveryProperties->sdn_configs(), Infoblox::Grid::Member::DiscoveryProperties, Infoblox::Grid::CACertificate

---

COPYRIGHT

Copyright (c) 2019 Infoblox Inc.

Infoblox::Grid::Discovery::SeedRouter - The seed router structure.

- NAME
- DESCRIPTION
- CONSTRUCTOR
- SESSION METHODS
- METHODS
  - address( )
  - network_view( )
  - comment( )
- AUTHOR
- COPYRIGHT

NAME

Infoblox::Grid::Discovery::SeedRouter - The seed router structure.

DESCRIPTION

The object provides information about the seed router.

CONSTRUCTOR

```
my $object = Infoblox::Grid::Discovery::SeedRouter->new(
    'address'      => $ipaddr,       # Required
    'network_view' => $string,       # Optional
    'comment'      => $string,       # Optional
);
```

SESSION METHODS

The object does not support any session methods.

METHODS

This section describes all the methods that you can use to set or retrieve the attribute values of the object.

address( )

Use this method to set or retrieve the IP address of the seed router.

Parameter

The IP address in string format.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
# Get attribute value
my $value = $object->address();
# Modify attribute value
$object->address('10.0.0.1');
```

network_view( )

Use this method to set or retrieve the network view associated with the seed router.

Parameter

The name of the network view

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
# Get attribute value
my $value = $object->network_view();
# Modify attribute value
$object->network_view('papi_network_view');
```

comment( )

Use this method to set or retrieve the description of the seed router.

Parameter

The description of the seed router in string format.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
# Get attribute value
my $value = $object->comment();
# Modify attribute value
$object->comment('comment');
```

AUTHOR

Infoblox Inc. http://www.infoblox.com/

COPYRIGHT

Copyright (c) 2017 Infoblox Inc.

Infoblox::Grid::Member::Discovery::ScanInterface - The scan interfaces structure.

- NAME
- DESCRIPTION
- CONSTRUCTOR
- MODULE METHODS
  - Infoblox::Grid::Member::DiscoveryProperties->scan_interfaces( )
- SESSION METHODS
- METHODS
  - scan_interface_type( )
  - network_view( )
  - vlan_id( )
- AUTHOR
- COPYRIGHT

NAME

Infoblox::Grid::Member::Discovery::ScanInterface - The scan interfaces structure.

DESCRIPTION

The object provides information about the scan interfaces.

CONSTRUCTOR

```
my $object = Infoblox::Grid::Member::Discovery::ScanInterface->new(
    'scan_interface_type' => (LAN1|LAN2|MGMT), # Required
    'network_view' => $string,         # Required
    'vlan_id' => $uint,                # Optional
);
```

MODULE METHODS

The following functions can be applied to a Discovery Properties object.

Infoblox::Grid::Member::DiscoveryProperties->scan_interfaces( )

Use this method to set or retrieve the scan interfaces configured for the Grid member discovery properties. See Infoblox::Grid::Member::DiscoveryProperties->scan_interfaces() for parameters and return values.

Example

```
#Get scan_interfaces value
my $scan_interfaces = $discovery_properties->scan_interfaces();
#Modify scan_interfaces
$discovery_properties->scan_interfaces($scan_interfaces);
```

SESSION METHODS

The object does not support any session methods.

METHODS

This section describes all the methods that you can use to set or retrieve the attribute values of the object.

scan_interface_type( )

Use this method to set or retrieve the probing interface type associated with the discovery probe member.

Parameter

The name of the probing interface (LAN1, LAN2, MGMT).

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

The method returns the value of scan_interface_type attribute.

Example

```
# Get attribute value
my $value = $object->scan_interface_type();
# Modify attribute value
my $result = $object->scan_interface_type('LAN1');
```

network_view( )

Use this method to set or retrieve the name of the network view associated with the interface.

Parameter

The name of the network view associated with the interface.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

The method returns the name of the network view associated with the interface.

Example

```
# Get attribute value
my $value = $object->network_view();
# Modify attribute value
my $result = $object->('papi_network_view');
```

vlan_id( )

Use this method to set or retrieve information for the VLAN tag of the probing network. This method is allowed only for the LAN1 and LAN2 interfaces.

Parameter

VLAN tag of the probing network.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
# Get attribute value
my $value = $object->vlan_id();
# Modify attribute value
$object->vlan_id(22);
```

AUTHOR

Infoblox Inc. http://www.infoblox.com/

COPYRIGHT

Copyright (c) 2017 Infoblox Inc.

Infoblox::Grid::Member::DNS::IP - a Grid member DNS IP.

- NAME
- DESCRIPTION
- CONSTRUCTOR
- SESSION METHODS
- MODULE METHODS
  - Infoblox::Grid::Member::DNS->dns_ip_list( )
- METHODS
  - member_virtual_ip( )
- AUTHOR
- SEE ALSO
- COPYRIGHT

NAME

Infoblox::Grid::Member::DNS::IP - a Grid member DNS IP.

DESCRIPTION

The DNS IP object is used to configure additional DNS-enabled interfaces for Grid member.

CONSTRUCTOR

```
my $object = Infoblox::Grid::Member::DNS::IP->new(
    member_virtual_ip => $Interface | $ipv4addr | $ipv6addr, #Required
);
```

SESSION METHODS

The object does not have any session methods.

MODULE METHODS

This section describes all the functions that you can apply to DNS IP object.

Infoblox::Grid::Member::DNS->dns_ip_list( )

Use this method to add or retrieve a list of additional IP addresses on which DNS is enabled for a grid member. See Infoblox::Grid::Member::DNS->dns_ip_list() for parameters and return values.

Example

```
#Get dns_ip_list value
my $value = $object->dns_ip_list();

#Modify dns_ip_list

#Construct Infoblox::Grid::Member::DNS::IP object

my $ip = Infoblox::Grid::Member::DNS::IP->new(
    member_virtual_ip => '10.0.0.10',
);

$object->dns_ip_list([$ip]);
```

METHODS

This section describes all the methods that you can use to configure and retrieve the attribute values of a DNS IP object.

member_virtual_ip( )

Use this method to set or retrieve the member's additional interface or interface IP address.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

The valid value is an Infoblox::Grid::Member::Interface object or IPv4 address or IPv6 address.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get member_virtual_ip value
my $member_virtual_ip = $object->member_virtual_ip();

#Modify member_virtual_ip value
$object->member_virtual_ip('10.0.0.10');
```

AUTHOR

Infoblox Inc. http://www.infoblox.com/

SEE ALSO

Infoblox::Grid::Member::DNS, Infoblox::Grid::Member::DNS->dns_ip_list(), Infoblox::Grid::Member::Interface,

---

COPYRIGHT

Copyright (c) 2016 Infoblox Inc.

IInfoblox::Grid::Member::DNS::ViewAddressSetting - The Grid member DNS view address settings.

- NAME
- DESCRIPTION
- CONSTRUCTOR
- MODULE METHODS
  - Infoblox::Grid::Member::DNS->dns_view_address_settings( )
- METHODS
  - enable_notify_source_port( )
  - enable_query_source_port( )
  - notify_delay( )
  - notify_source_port( )
  - notify_transfer_source_interface( )
  - query_source_interface( )
  - override_notify_delay( )
  - override_source_ports( )
  - query_source_port( )
  - view_name( )
- AUTHOR
- SEE ALSO
- COPYRIGHT

NAME

IInfoblox::Grid::Member::DNS::ViewAddressSetting - The Grid member DNS view address settings.

DESCRIPTION

The object provides information about the settings for the query source address and port, zone transfer request address, notify address and port as well as notify delay.

CONSTRUCTOR

```
 my $object = IInfoblox::Grid::Member::DNS::ViewAddressSetting->new(
     enable_notify_source_port        => 'true' | 'false',                          #Optional / Default is
'false'
     enable_query_source_port         => 'true' | 'false',                          #Optional / Default is
'false'
     notify_delay                     => $uint,                                     #Optional / Default is
5
     notify_source_port               => $uint,                                     #Optional / Default is
undefined
     notify_transfer_source_interface => 'VIP' | 'MGMT' | 'LAN2' | 'Any' | $ipaddr, #Optional / Default is
'Any'
     override_notify_delay            => 'true' | 'false',                          #Optional / Default is
'false'
     override_source_port             => 'true' | 'false',                          #Optional / Default is
'false'
     query_source_interface           => 'VIP' | 'MGMT' | 'LAN2' | 'Any' | $ipaddr, #Optional / Default is
'Any'
     query_source_port                => $uint,                                     #Optional / Default is
undefined
     view_name                        => $string,                                   #Required
 );
```

MODULE METHODS

You can apply the following functions to a DNS view address settings object.

Infoblox::Grid::Member::DNS->dns_view_address_settings( )

Use this method to set or retrieve the list of address settings for Grid member views. See Infoblox::Grid::Member::DNS/dns_view_address_settings() for parameters and return values.

Example

```
#Get dns_view_address_settings value
my $value = $object->dns_view_address_settings();

#Construct view address setting
```

```
my $view_address_setting = Infoblox::Grid::Member::DNS::ViewAddressSetting->new('view_name' =>
'view1');

#Modify dns_view_address_settings value
$object->dns_view_address_settings([$view_address_setting]);
```

METHODS

This section describes all the methods that you can use to set and retrieve the attribute values for a DNS view address settings object.

enable_notify_source_port( )

Use this method to set or retrieve the flag that indicates whether the notify source port setting is enabled.

Setting this method to a defined value implicitly sets the override_source_ports method to 'true'. Setting the parameter to undefined (if enable_query_source_port, query_source_port, notify_source_port are undefined) causes the appliance to use the Grid member default and automatically resets the override_source_ports method to 'false'.

Note that when enable_notify_source_port is set to a defined value and override_source_ports is set to 'false', the last operation takes precedence. Thus the sequence $object->enable_notify_source_port('true'); $object->override_source_ports('false'); will set override_source_ports to 'false', and the sequence $object->override_source_ports('false'); $object->enable_notify_source_port('true'); will result in override_source_ports='true'.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

Specify 'true' to enable notify source port setting, and 'false' to disable it.

Returns

When you specify a parameter and the modification succeeds, the method returns 'true'. When the operation fails, the method returns 'false'.

When you do not specify a parameter, the method returns the attribute value.

Example

```
#Get enable_notify_source_port value
my $value = $object->enable_notify_source_port();

#Modify enable_notify_source_port value
$object->enable_notify_source_port('true');
```

enable_query_source_port( )

Use this method to set or retrieve the flag that indicates whether the query source port setting is enabled.

Setting this method to a defined value implicitly sets the override_source_ports method to 'true'. Setting the parameter to undefined (if enable_notify_source_port, query_source_port, notify_source_port are undefined) causes the appliance to use the Grid member default and automatically resets the override_source_ports method to 'false'.

Note that when enable_query_source_port is set to a defined value and override_source_ports is set to 'false', the last operation takes precedence. Thus the sequence $object->enable_query_source_port('true'); $object->override_source_ports('false'); will set override_source_ports to 'false', and the sequence $object->override_source_ports('false'); $object->enable_query_source_port('true'); will result in override_source_ports='true'.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

Specify 'true' to enable query source port setting, and 'false' to disable it.

Returns

When you specify a parameter and the modification succeeds, the method returns 'true'. When the operation fails, the method returns 'false'.

When you do not specify a parameter, the method returns the attribute value.

Example

```
#Get enable_query_source_port value
my $value = $object->enable_query_source_port();

#Modify enable_query_source_port value
$object->enable_query_source_port('true');
```

notify_delay( )

Use this method to set or retrieve the minimum delay (in seconds) between a set of NOTIFY messages.

Setting this method to a defined value implicitly sets the override_notify_delay method to 'true'. Setting the parameter to undefined causes the appliance to use the Grid DNS default and automatically resets the override_notify_delay method to 'false'.

Note that when notify_delay is set to a defined value and override_notify_delay is set to 'false', the last operation takes precedence. Thus the sequence $object->notify_delay('true'); $object->override_notify_delay('false'); will set override_notify_delay to 'false', and the sequence $object->override_notify_delay('false'); $object->notify_delay('true'); will result in override_notify_delay='true'.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

The valid value is an unsigned integer between 5 and 86400. The default value is 5.

Returns

When you specify a parameter and the modification succeeds, the method returns 'true'. When the operation fails, the method returns 'false'.

When you do not specify a parameter, the method returns the attribute value.

Example

```
#Get notify_delay value
my $value = $object->notify_delay();

#Modify notify_delay value
$object->notify_delay(10000);
```

notify_source_port( )

Use this method to set or retrieve the notify source port number.

Setting this method to a defined value implicitly sets the override_source_ports method to 'true'. Setting the parameter to undefined (if enable_query_source_port, enable_notify_source_port, query_source_port are undefined) causes the appliance to use the Grid member default and automatically resets the override_source_ports method to 'false'.

Note that when notify_source_port is set to a defined value and override_source_ports is set to 'false', the last operation takes precedence. Thus the sequence $object->notify_source_port('true'); $object->override_source_ports('false'); will set override_source_ports to 'false', and the sequence $object->override_source_ports('false'); $object->notify_source_port('true'); will result in override_source_ports='true'.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

The valid value is an unsigned integer between 1 and 63999.

Returns

When you specify a parameter and the modification succeeds, the method returns 'true'. When the operation fails, the method returns 'false'.

When you do not specify a parameter, the method returns the attribute value.

Example

```
#Get notify_source_port value
my $value = $object->notify_source_port();

#Modify notify_source_port value
$object->notify_source_port(10000);
```

notify_transfer_source_interface( )

Use this method to specify which IP address is used as the source for notify and zone transfer operations.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

Valid values are 'VIP', 'MGMT', 'LAN2', 'Any' or an IP address. The default is 'VIP'.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get notify_transfer_source_interface value
my $value = $object->notify_transfer_source_interface();

#Modify notify_transfer_source_interface value
$object->notify_transfer_source_interface('Any');
```

query_source_interface( )

Use this method to specify which IP address is used as the source for query operations.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

Valid values are 'VIP', 'MGMT', 'LAN2', 'Any' or an IP address. Default is 'VIP'.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get query_source_interface value
my $value = $object->query_source_interface();

#Modify query_source_interface value
$object->query_source_interface('Any');
```

override_notify_delay( )

The override_notify_delay attribute controls whether the notify_delay method values of the member are used, instead of the Grid member default.

The override_notify_delay attribute can be specified explicitly. It is also set implicitly when notify_delay is set to a defined value.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

Set the parameter to 'true' to override the member-level setting for notify_delay. Set the parameter to 'false' to inherit the member-level setting for notify_delay.

The default value of this parameter is 'false'.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get override_notify_delay value
my $value = $object->override_notify_delay();

#Modify override_notify_delay value
$object->override_notify_delay('true');
```

override_source_ports( )

The override_source_ports attribute controls whether the values of enable_notify_source_port, enable_query_source_port, query_source_port and notify_source_port for the member are used, instead of the Grid member default.

The override_source_ports attribute can be specified explicitly. It is also set implicitly when any of enable_notify_source_port, enable_query_source_port, query_source_port, notify_source_port is set to a defined value.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

Set the parameter to true to override the member-level setting for enable_notify_source_port, enable_query_source_port,

query_source_port, notify_source_port. Set the parameter to 'false' to inherit the member-level setting for source_ports.

The default value of this parameter is 'false'.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get override_source_ports value
my $value = $object->override_source_ports();

#Modify override_source_ports value
$object->override_source_ports('true');
```

query_source_port( )

Use this method to set or retrieve the query source port number.

Setting this method to a defined value implicitly sets the override_source_ports method to 'true'. Setting the parameter to undefined (if enable_query_source_port, enable_notify_source_port, notify_source_port are undefined) causes the appliance to use the Grid member default and automatically resets the override_source_ports method to 'false'.

Note that when query_source_port is set to a defined value and override_source_ports is set to 'false', the last operation takes precedence. Thus the sequence $object->query_source_port('true'); $object->override_source_ports('false'); will set override_source_ports to 'false', and the sequence $object->override_source_ports('false'); $object->query_source_port('true'); will result in override_source_ports='true'.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

The valid value is an unsigned integer between 1 and 63999.

Returns

When you specify a parameter and the modification succeeds, the method returns 'true'. When the operation fails, the method returns 'false'.

When you do not specify a parameter, the method returns the attribute value.

Example

```
#Get query_source_port value
my $value = $object->query_source_port();

#Modify query_source_port value
$object->query_source_port(10000);
```

view_name( )

Use this method to set or retrieve the DNS view name.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

The valid value is a desired DNS view name in a string format.

Returns

When you specify a parameter and the modification succeeds, the method returns 'true'. When the operation fails, the method returns 'false'.

When you do not specify a parameter, the method returns the attribute value.

Example

```
#Get view_name value
my $value = $object->view_name();

#Modify view_name value
$object->view_name('view1');
```

AUTHOR

Infoblox Inc. http://www.infoblox.com/

SEE ALSO

Infoblox::Grid::Member::DNS, Infoblox::Grid::Member::DNS/dns_view_address_settings(),

---

COPYRIGHT

Copyright (c) 2017 Infoblox Inc.

Infoblox::DNS::GlueRecordAddr - Infoblox DNS Glue Record Object.

- NAME
- DESCRIPTION
- CONSTRUCTOR
- MODULE METHODS
  - Infoblox::Grid::Member::DNS->glue_record_addresses( )
- METHODS
  - attach_empty_recursive_view( )
  - glue_record_address( )
  - view( )
- SAMPLE CODE
- AUTHOR
- SEE ALSO
- COPYRIGHT

NAME

Infoblox::DNS::GlueRecordAddr - Infoblox DNS Glue Record Object.

DESCRIPTION

The GlueRecordAddr allows you to specify which interface address the appliance uses to generate the glue record for each view.

CONSTRUCTOR

The Infoblox::DNS::GlueRecordAddr object is automatically generated upon the successful creation of the Infoblox::DNS::View object. It does not require manual construction.

MODULE METHODS

This section describes all the functions that you can apply to a GlueRecordAddr object.

Infoblox::Grid::Member::DNS->glue_record_addresses( )

Use this function to specify a glue record address for a view at the grid member level. See Infoblox::Grid::Member::DNS->glue_record_addresses() for parameters and return values.

Example

```
# Retrieve the list of GlueRecordAddr objects from Member DNS object.
my @retrieved_objs = $session->get(
    object => "Infoblox::Grid::Member::DNS",
    name   => "infoblox.localdomain" );

#select the desired GlueRecordAddr for the view
my $Grid_Member_DNS = $retrieved_objs[0];
my $ref_all_glue_record_addr = $Grid_Member_DNS->glue_record_addresses();
my $glue_record_addr = @$ref_all_glue_record_addr[0];

# Modify Glue Record Address for the view to use NAT address.
$glue_record_addr->glue_record_address("NAT");

#Save modification to Member DNS object.
my $response = $Grid_Member_DNS->glue_record_addresses($ref_all_glue_record_addr);
```

METHODS

This section describes all the methods that you can use to configure and retrieve the attribute values of a GlueRecordAddr object.

attach_empty_recursive_view( )

Use this method to set or retrieve the attach_empty_recursive_view flag of glue record object. If the attach_empty_recursive_view flag is set to "true", then empty view with recursion enabled will be written into the conf file.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

Specify "true" to set the attach_empty_recursive_view flag or "false" to deactivate/unset it.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get attach_empty_recursive_view.
my $attach_empty_recursive_view = $glue_record_addr->attach_empty_recursive_view();
#Modify attach_empty_recursive_view.
$glue_record_addr->attach_empty_recursive_view("true");
```

glue_record_address( )

Use this method to set or retrieve the address the appliance uses to generate the glue record.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

String which specifies the interface address to use. Valid values are "interface", "NAT", or specify an IPv4 or IPv6 address.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get glue record address.
my $glue_record_address = $glue_record_addr->glue_record_address();
#Modify glue record address.
$glue_record_addr->glue_record_address("NAT");
```

view( )

Use this method to retrieve the view of this Glue Record object

Parameter

None.

Returns

The method returns the attribute value.

Example

```
#Get view
my $glue_record_view = $glue_record_addr->view();
```

SAMPLE CODE

The following sample code demonstrates the different functions that can be applied to an object, such as get and modify. This sample also includes error handling for the operations.

#Preparation prior to a DNS GlueRecordAddr object modification

```
#PROGRAM STARTS: Include all the modules that will be used
use strict;
use Infoblox;

#Create a session to the Infoblox appliance

my $session = Infoblox::Session->new(
           master   => "192.168.1.2",
           username => "admin",
           password => "infoblox"
);

unless ($session) {
   die("Construct session failed: ",
        Infoblox::status_code() . ":" . Infoblox::status_detail());
}
print "Session created successfully\n";
```

```
# Retrieve the list of GlueRecordAddr objects from Member DNS object.
my @retrieved_objs = $session->get(
    object => "Infoblox::Grid::Member::DNS",
    name   => "infoblox.localdomain" );

my $Grid_Member_DNS = $retrieved_objs[0];
unless ($Grid_Member_DNS) {
   die("Retrieve Grid Member DNS object failed: ",
       $session->status_code() . ":" . $session->status_detail());
}
print "Grid Member DNS object retrieve successfully\n";
```

#Modify the Glue Record address

```
my $ref_all_glue_record_addr = $Grid_Member_DNS->glue_record_addresses();

# Select the desired GlueRecordAddr for the view
my $glue_record_addr = @$ref_all_glue_record_addr[0];
unless ($glue_record_addr) {
   die("Glue Record Address not found on Grid Member DNS object: ",
       $session->status_code() . ":" . $session->status_detail());
}
print "Glue Record Address found on Grid Member DNS object\n";

# Modify Glue Record Address for the view to use a custom defined address.
$glue_record_addr->glue_record_address("interface")
    or die("modify glue record failed: ",
        Infoblox::status_code() . ":" . Infoblox::status_detail());
print "Glue Record Address modified successfully\n";

# Save modification to Member DNS object.
my $response = $Grid_Member_DNS->glue_record_addresses($ref_all_glue_record_addr);
unless($response) {
    die("Modify glue record on Member DNS failed: ",
        Infoblox::status_code() . ":" . Infoblox::status_detail());
}
print "Glue Record Address updated to Grid Member DNS object successfully\n";

#submit changes to Infoblox appliance
$session->modify($Grid_Member_DNS)
   or die("Modify Member DNS object on Infoblox appliance failed: ",
        Infoblox::status_code() . ":" . Infoblox::status_detail());
print "Grid Member DNS modified on Infoblox appliance successfully\n";

####PROGRAM ENDS####
```

AUTHOR

Infoblox Inc. http://www.infoblox.com/

SEE ALSO

Infoblox::Session->get(), Infoblox::Session->modify(),Infoblox::Grid::Member::DNS->glue_record_addresses()

COPYRIGHT

Copyright (c) 2017 Infoblox Inc.

Infoblox::Grid::FileDistributionDir - Grid FileDistributionDir object.

- NAME
- DESCRIPTION
- CONSTRUCTOR
- SESSION METHODS
  - Infoblox::Session->add( )
  - Infoblox::Session->get( )
  - Infoblox::Session->modify( )
  - Infoblox::Session->remove( )
  - Infoblox::Session->search( )
- METHODS
  - name( )
- SAMPLE CODE
- AUTHOR
- SEE ALSO
- COPYRIGHT

NAME

Infoblox::Grid::FileDistributionDir - Grid FileDistributionDir object.

DESCRIPTION

The FileDistributionDir object is used to add, get, modify, and remove directories that store files for a file distribution service.

CONSTRUCTOR

```
my $dir = Infoblox::Grid::FileDistributionDir->new(
    name        => $string,             #Required
);
```

SESSION METHODS

This section describes all the methods in the Infoblox::Session module that you can apply to a FileDistributionDir object.

Infoblox::Session->add( )

Use this method to add an object to the Infoblox appliance. See Infoblox::Session->add() for parameters and return values.

Example

```
#Construct the file distribution directory object
my $dir = Infoblox::Grid::FileDistributionDir->new(
    name        => "mydir",
);
# Submit for addition
my $response = $session->add( $dir );
```

Infoblox::Session->get( )

Use this method to retrieve all the matching objects from the Infoblox appliance. See Infoblox::Session->get() for parameters and return values.

Key References

```
Apply the following attributes to get a specific FileDistributionDir object:

 name          - Required. User name in string format.
```

Example

```
my @retrieved_objs = $session->get(
    object      => "Infoblox::Grid::FileDistributionDir",
    name        => "mydir" );
```

Infoblox::Session->modify( )

Use this method to modify an object in the Infoblox appliance. See Infoblox::Session->modify() for parameters and return values.

Example

```
# Use method to rename a file distribution directory.
$dir->name("mynewdir");
# Submit modification
my $response = $session->modify( $dir );
```

Infoblox::Session->remove( )

Use this method to remove an object from the Infoblox appliance. See Infoblox::Session->remove() for parameters and return values.

To remove a specific object, use `get()` or `search()` to retrieve the specific object first, and then submit this object for removal.

Example

```
# Get the user objects with the same starting name
my @retrieved_objs = $session->get(
    object      => "Infoblox::Grid::FileDistributionDir",
    name        => "mydir" );
# find the desired object from the retrieved list.
my $dir = $retrieved_objs[0];
# Submit for removal
my $response = $session->remove( $dir );
```

Infoblox::Session->search( )

Use this method to search for FileDistributionDir objects in the Infoblox appliance. See Infoblox::Session->search() for parameters and return values.

Key References

```
Apply the following attributes to search for a specific FileDistributionDir object:

 name          - Required. Directory name in string format. Only the exact match is supported (that
is, exactly the same as 'get' method).
```

Example

```
# search for all FileDistributionDir objects with name "mydir"
my @retrieved_objs = $session->search(
    object      => "Infoblox::Grid::FileDistributionDir",
    name        => "mydir");
```

METHODS

This section describes all the methods that you can use to set and retrieve the attribute values of a FileDistributionDir object.

name( )

Use this method to set or retrieve the name of a file distribution directory.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

Text with the name of the file distribution directory. Enter the complete path to the file distribution directory, including all existing intermediate directories.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get name of the file distribution directory
my $name = $file_distribution->name();
#Modify name of the file distribution directory (i.e. rename the directory)
$file_distribution->name("mynewdir");
#Create a new subdirectory 'subdir' under 'mynewdir' ('mynewdir' must exist)
$file_distribution->name("mynewdir/subdir");
```

SAMPLE CODE

The following sample code demonstrates the different functions that can be applied to an object, such as add, search, modify, and remove. This sample also includes error handling for the operations.

#Preparation prior to a FileDistributionDir object insertion

```
#PROGRAM STARTS: Include all the modules that will be used
use strict;
use Infoblox;

#Create a session to the Infoblox appliance

my $session = Infoblox::Session->new(
               master   => "192.168.1.2", #appliance host ip
               username => "admin",       #appliance user login
               password => "infoblox"     #appliance password
               );

unless ($session) {
       die("Construct session failed: ",
               Infoblox::status_code() . ":" . Infoblox::status_detail());
}
print "Session created successfully\n";
```

#Create a FileDistributionDir object

```
my $dir = Infoblox::Grid::FileDistributionDir->new(
    name        => "mydir",
);

unless ($dir) {
       die("Construct file distribution directory object failed: ",
               Infoblox::status_code() . ":" . Infoblox::status_detail());
}
print "FileDistributionDir object created successfully\n";

#Add the file distribution dir object to the Infoblox appliance through a session
$session->add($dir)
       or die("Add FileDistributionDir object failed: ",
               $session->status_code() . ":" . $session->status_detail());
print "FileDistributionDir object added to server successfully\n";
```

#Search for a file distribution directory

```
my @retrieved_objs = $session->search(
               object      => "Infoblox::Grid::FileDistributionDir",
               name        => "mydir"
              );
my $object = $retrieved_objs[0];

unless ($object) {
       die("Search FileDistributionDir object failed: ",
               $session->status_code() . ":" . $session->status_detail());
}
print "Search FileDistributionDir object found at least 1 matching entry\n";
```

#Get and modify a FileDistributionDir object

```
#Get the file distribution dir object from Infoblox appliance through a session
my @retrieved_objs = $session->get(
            object      => "Infoblox::Grid::FileDistributionDir",
            name        => "mydir",
);
my $object = $retrieved_objs[0];

unless ($object) {
       die("Get FileDistributionDir object failed: ",
               $session->status_code() . ":" . $session->status_detail());
}
print "Get FileDistributionDir object found at least 1 matching entry\n";

#Modify the name of the FileDistributionDir object (that is, rename the directory)
$object->name("mynewdir");

#Apply the changes.
$session->modify($object)
       or die("Modify User object failed: ",
               $session->status_code() . ":" . $session->status_detail());
print "FileDistributionDir object modified successfully \n";
```

#Remove a FileDistributionDir object

```
#Get a FileDistributionDir object through the session
my @retrieved_objs = $session->get(
            object      => "Infoblox::Grid::FileDistributionDir",
            name        => "mynewdir",
);
my $object = $retrieved_objs[0];
unless ($object) {
    die("Get FileDistributionDir object failed: ",
        $session->status_code() . ":" . $session->status_detail());
}
print "Get FileDistributionDir object found at least 1 matching entry\n";

#Submit an object for removal
$session->remove($object)
       or die("Remove FileDistributionDir object failed: ",
                $session->status_code() . ":" . $session->status_detail());
print "FileDistributionDir object removed successfully \n";

####PROGRAM ENDS####
```

AUTHOR

Infoblox Inc. http://www.infoblox.com/

SEE ALSO

Infoblox::Grid::Member::FileDistribution, Infoblox::Session->add(), Infoblox::Session->get(), Infoblox::Session->modify(), Infoblox::Session->remove(), Infoblox::Session->search(), Infoblox::Session

---

COPYRIGHT

Copyright (c) 2017 Infoblox Inc.

Infoblox::Grid::Member::PreProvisioning::Hardware - PreProvisioning hardware info object.

- NAME
- DESCRIPTION
- CONSTRUCTOR
- METHODS
  - hwtype( )
  - hwmodel( )
- AUTHOR
- SEE ALSO
- COPYRIGHT

NAME

Infoblox::Grid::Member::PreProvisioning::Hardware - PreProvisioning hardware info object.

DESCRIPTION

An Infoblox::Grid::Member::PreProvisioning::Hardware object represents pre-provisioned data for the hardware model.

CONSTRUCTOR

```
 my $hw = Infoblox::Grid::Member::PreProvisioning->new(
     hwtype          => $string,                  # Optional / one of IB-100,IB-810, IB-820, IB-1410, IB-1420, IB-2210, IB-FLEX,
                                                  # IB-2220, IB-4010, IB-4030, PT-1400, PT-2200, PT-4000, IB-RSP2, IB-VM, IB-VNIOS
     hwmodel         => $string,                  # Optional / one of:
                                                  # - for IB-4010 are Rev1, Rev2;
                                                  # - for IB-4030 are Rev1, Rev2;
                                                  # - for PT-4000 is Rev2;
                                                  # - for IB-RSP2 is [TBD];
                                                  # - for IB-VNIOS are IB-VM-100, IB-VM-810, IB-VM-820, IB-VM-RSP, IB-VM-1410, IB-VM-1420, IB-VM-2210, IB-VM-2220, IB-VM-4010;
 );
```

METHODS

This section describes all the methods that can be used to configure and retrieve the attribute values of an Infoblox::Grid::Member::PreProvisioning::Hardware object.

hwtype( )

Use this method to set or retrieve the hardware type.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

Valid value is a valid hardware type string.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get get hwtype()
my $hwt = $hw->hwtype();
#Modify hwtype
$hw->hwtype('PT-4000');
```

hwmodel( )

Use this method to set or retrieve the hardware model.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

Valid value is a valid hardware model string.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get get hwmodel()
my $hwm = $hw->hwmodel();
#Modify hwmodel
$hw->hwmodel('Rev2');
```

AUTHOR

Infoblox Inc. http://www.infoblox.com/

SEE ALSO

Infoblox::Grid::Member, Infoblox::Grid::Member::PreProvisioning

---

COPYRIGHT

Copyright (c) 2017 Infoblox Inc.

Infoblox::Grid::Member::Taxii::RPZConfig - A Taxii Member RPZ Configuration object.

- NAME
- DESCRIPTION
- CONSTRUCTOR
- MODULE METHODS
  - Infoblox::Grid::Member::Taxii->taxii_rpz_config( )
- METHODS
  - collection_name( )
  - zone( )
- AUTHOR
- SEE ALSO
- COPYRIGHT

NAME

Infoblox::Grid::Member::Taxii::RPZConfig - A Taxii Member RPZ Configuration object.

DESCRIPTION

The Taxii Member RPZ Configuration object provides configuration for RPZ rule creation through the Taxii protocol. Each Grid member running the Taxii server can support a separate set of response policy zones. These zones will be accessible via a user configurable name for the STIX collection.

CONSTRUCTOR

```
my $rpz_config = Infoblox::Grid::Member::Taxii::RPZConfig->new(
    collection_name => $string, #Required
    zone            => $rpzone, #Required
);
```

MODULE METHODS

Infoblox::Grid::Member::Taxii->taxii_rpz_config( )

Use this method to set or retrieve the Taxii service RPZ configuration list. See Infoblox::Grid::Member::Taxii->taxii_rpz_config() for parameters and return values.

Example

```
#Get taxii_rpz_config value
my $taxii_rpz_config = $object->taxii_rpz_config();

#Modify taxii_rpz_config value
$object->taxii_rpz_config([$conf1, $conf2]);
```

METHODS

collection_name( )

Use this method to set or retrieve the STIX collection name.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

The valid value is a desired collection name in sstring format.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get collection_name value
my $collection_name = $object->collection_name();

#Modify collection_name value
```

```
$object->collection_name('collection1');
```

zone( )

Use this method to set or retrieve the response policy zone that can be modified through the Taxii protocol.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

The valid value is an Infoblox::DNS::Zone object.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get zone value
my $zone = $object->zone();

#Modify zone value
$object->zone($rp_zone);
```

AUTHOR

Infoblox Inc. http://www.infoblox.com/

SEE ALSO

Infoblox::Grid::Member::Taxii, Infoblox::Grid::Member::Taxii->taxii_rpz_config(), Infoblox::DNS::Zone

---

COPYRIGHT

Copyright (c) 2017 Infoblox Inc.

Infoblox::Grid::Member::Taxii - A Taxii Member object.

- NAME
- DESCRIPTION
- SESSION METHODS
  - Infoblox::Session->get( )
  - Infoblox::Session->modify( )
  - Infoblox::Session->search( )
- METHODS
  - enable_service( )
  - ipv4addr( )
  - ipv6addr( )
  - name( )
  - taxii_rpz_config( )
- AUTHOR
- SEE ALSO
- COPYRIGHT

NAME

Infoblox::Grid::Member::Taxii - A Taxii Member object.

DESCRIPTION

The Taxii Member object provides information on Taxii service configuration such as the start/stop flag and RPZ (Response Policy Zone) configuration.

SESSION METHODS

This section describes all the methods in an Infoblox::Session module that you can apply to a Taxii Member object.

Infoblox::Session->get( )

Use this method to retrieve all the matching objects from the Infoblox appliance. See Infoblox::Session->get() for parameters and return values.

Key Reference

```
Apply the following attribute to get a specific object:

  ipv4addr - Optional. IPv4 Member address.
  ipv6addr - Optional. IPv6 Member address.
  name     - Optional. The host name of the parent member.
```

Example

```
#Get Taxii Member object with name 'taxii_member.com'
my @taxii_member = $session->get(
    object => 'Infoblox::Grid::Member::Taxii',
    name   => 'taxii_member.com',
);
```

Infoblox::Session->modify( )

Use this method to modify an object in the Infoblox appliance. See Infoblox::Session->modify() for parameters and return values.

Example

```
#Modify enable_service value
$taxii_member->enable_service('true');

#Submit modification
$session->modify($taxii_member);
```

Infoblox::Session->search( )

Use this method to search for objects in the Infoblox appliance. See Infoblox::Session->search() for parameters and return values.

Key Reference

```
Apply the following attribute to search a specific object:
```

```
  ipv4addr - Optional. IPv4 Member address (regular expression).
  ipv6addr - Optional. IPv6 Member address (regular expression).
  name     - Optional. The host name of the parent member.
```

Example

```
#Search for Taxii Member object with name starting with 'taxii'
my @taxii_member = $session->get(
    object => 'Infoblox::Grid::Member::Taxii',
    name   => 'taxii.*',
);
```

METHODS

This section describes all the methods that you can use to set or retrieve the attribute values of the object.

enable_service( )

Use this method to set or retrieve the flag that indicates whether the Taxii service is running on the given member or not.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

Specify 'true' to start/enable Taxii service or 'false' to stop/deactivate it.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get enable_service value
my $enable_service = $object->enable_service();

#Modify enable_service value
$object->enable_service('true');
```

ipv4addr( )

Use this method to retrieve the IPv4 address of the Taxii member. This is a read-only attribute.

Omit the parameter to retrieve the attribute value.

Parameter

None

Returns

The method returns the attribute value.

Example

```
#Get ipv4addr value
my $ipv4addr = $object->ipv4addr();
```

ipv6addr( )

Use this method to retrieve the IPv6 address of the Taxii member. This is a read-only attribute.

Omit the parameter to retrieve the attribute value.

Parameter

None

Returns

The method returns the attribute value.

Example

```
#Get ipv6addr value
my $ipv6addr = $object->ipv6addr();
```

name( )

Use this method to retrieve the name of the Taxii member. This is a read-only attribute.

Omit the parameter to retrieve the attribute value.

Parameter

None

Returns

The method returns the attribute value.

Example

```
#Get name value
my $name = $object->name();
```

taxii_rpz_config( )

Use this method to set or retrieve the Taxii service RPZ configuration list.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

The valid value is an array of Infoblox::Grid::Member::Taxii::RPZConfig.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get taxii_rpz_config value
my $taxii_rpz_config = $object->taxii_rpz_config();

#Modify taxii_rpz_config value
$object->taxii_rpz_config([$conf1, $conf2]);
```

AUTHOR

Infoblox Inc. http://www.infoblox.com/

SEE ALSO

Infoblox::Session, Infoblox::Session->get(), Infoblox::Session->modify(), Infoblox::Session->search(), Infoblox::Grid::Member::Taxii::RPZConfig

---

COPYRIGHT

Copyright (c) 2017 Infoblox Inc.

Infoblox::Grid::ThreatProtection::Ruleset - A Grid Threat Protection Ruleset object.

- NAME
- DESCRIPTION
- SESSION METHODS
  - Infoblox::Session->get( )
  - Infoblox::Session->search( )
  - Infoblox::Session->modify( )
  - Infoblox::Session->remove( )
- METHODS
  - add_type( )
  - added_time( )
  - comment( )
  - do_not_delete( )
  - is_factory_reset_enabled( )
  - used_by( )
  - version( )
- AUTHOR
- SEE ALSO
- COPYRIGHT

NAME

Infoblox::Grid::ThreatProtection::Ruleset - A Grid Threat Protection Ruleset object.

DESCRIPTION

This object represents the Grid Threat Protection ruleset.

SESSION METHODS

This section describes all the methods in an Infoblox::Session module that you can apply to a Grid ruleset object.

Infoblox::Session->get( )

Use this method to retrieve all the ruleset objects from the Infoblox appliance. See Infoblox::Session->get() for parameters and return values.

Example

```
my @retrieved_objs = $session->get(
    'object' => 'Infoblox::Grid::ThreatProtection::Ruleset',
);
```

Infoblox::Session->search( )

Use this method to retrieve all the ruleset objects from the Infoblox appliance. See Infoblox::Session->search() for parameters and return values.

Example

```
my @retrieved_objs = $session->search(
    'object' => 'Infoblox::Grid::ThreatProtection::Ruleset',
);
```

Infoblox::Session->modify( )

Use this method to modify an object in the Infoblox appliance. See Infoblox::Session->modify() for parameters and return values.

If you have applied an access control list to an object, you must restart services after you update the access control list for the changes to take effect. See Infoblox::Session->restart() method.

Example

```
# Use this method to modify comments for the Grid member.
$ruleset->comment("This is modified object");
# Submit modification
my $response = $session->modify( $ruleset );
```

Infoblox::Session->remove( )

Use this method to remove an object from the Infoblox appliance. See Infoblox::Session->remove() for parameters and return values.

To remove a specific object, first use `get()` or `search()` to retrieve the specific object, and then submit this object for removal.

Example

```
# Submit for removal
my $response = $session->remove( $desired_ruleset );
```

METHODS

This section describes all the methods that you can use to set or retrieve the attribute values of the struct.

add_type( )

Use this method to retrieve how the ruleset was added. This is a read-only attribute.

Parameter

None

Returns

The method returns the attribute value, which is either "AUTOMATIC" or "MANUAL".

Example

```
# Get attribute value
my $value = $object->add_type();
```

added_time( )

Use this method to retrieve the time when the ruleset was added. This is a read-only attribute.

Parameter

None

Returns

The method returns the attribute value, the number of seconds that have elapsed since January 1st, 1970 UTC.

Example

```
# Get attribute value
my $value = $object->added_time();
```

comment( )

Use this method to set or retrieve a descriptive comment.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

Desired comment in string format with a maximum of 256 bytes.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get comment
my $comment = $object->comment();
#Modify comment
$object->comment("Modified comment");
```

do_not_delete( )

Use this method to set or retrieve the do_not_delete attribute of the ruleset. If set to "true", the ruleset will not be deleted during a ruleset upgrade.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

None

Returns

The method returns the attribute value, which is "true" or "false".

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get do_not_delete from the ruleset
my $do_not_delete = $object->do_not_delete();
#Modify do_not_delete
$object->do_not_delete("true");
```

is_factory_reset_enabled( )

Use this method to retrieve the flag that indicates whether the factory reset of this object is enabled or disabled. This is a read-only attribute.

Parameter

None

Returns

The method returns the attribute value.

Example

```
# Get attribute value
my $value = $object->is_factory_reset_enabled();
```

used_by( )

Use this method to retrieve the users of the ruleset. This is a read-only attribute.

Parameter

None

Returns

The method returns the attribute value, which is a list of strings.

Example

```
# Get attribute value
my $value = $object->used_by();
```

version( )

Use this method to retrieve the ruleset version. This is a read-only attribute.

Parameter

None

Returns

The method returns the attribute value.

Example

```
# Get attribute value
my $value = $object->version();
```

AUTHOR

Infoblox Inc. http://www.infoblox.com/

SEE ALSO

Infoblox::Grid::ThreatProtection, Infoblox::Grid::Member::ThreatProtection

---

COPYRIGHT

Copyright (c) 2017 Infoblox Inc.

Infoblox::Grid::ThreatProtection::NATRule - a NAT Threat Protection Rule object.

- NAME
- DESCRIPTION
- CONSTRUCTOR
- SESSION METHODS
- MODULE METHODS
  - Infoblox::Grid::ThreatProtection->nat_rules( )
  - Infoblox::Grid::Member::ThreatProtection->nat_rules( )
- METHODS
  - address( )
  - cidr( )
  - end_address( )
  - nat_ports( )
  - network( )
  - rule_type( )
  - start_address( )
- AUTHOR
- SEE ALSO
- COPYRIGHT

NAME

Infoblox::Grid::ThreatProtection::NATRule - a NAT Threat Protection Rule object.

DESCRIPTION

The NAT Threat Protection Rule object provides information about NAT rules configured for the threat protection NAT mapping feature.

CONSTRUCTOR

```
my $nat_rule = Infoblox::Grid::ThreatProtection::NATRule->new(
    rule_type     => 'ADDRESS' | 'NETWORK' | 'RANGE', #Required
    address       => $ipaddress,                      #Required if rule_type is 'ADDRESS'
    network       => $ipaddress,                      #Required if rule_type is 'NETWORK'
    cidr          => $uint,                           #Required if rule_type is 'NETWORK'
    start_address => $ipaddress,                      #Required if rule_type is 'RANGE'
    end_address   => $ipaddress,                      #Required if rule_type is 'RANGE'
    nat_ports     => $nat_ports,                      #Optional
);
```

SESSION METHODS

The object does not support any session methods.

MODULE METHODS

The following functions are available to apply to a NAT Port object.

Infoblox::Grid::ThreatProtection->nat_rules( )

Use this method to set or retrieve the list of rules configured for the threat protection NAT mapping feature. See Infoblox::Grid::ThreatProtection->nat_rules() for parameters and return values.

Example

```
#Get nat_rules
my $nat_rules = $nat_rule->nat_rules();

#Modify nat_rules
$nat_rule->nat_rules($nat_rules);
```

Infoblox::Grid::Member::ThreatProtection->nat_rules( )

Use this method to set or retrieve the list of rules configured for the threat protection NAT mapping feature. See Infoblox::Grid::Member::ThreatProtection->nat_rules() for parameters and return values.

Example

```
#Get nat_rules
my $nat_rules = $nat_rule->nat_rules();

#Modify nat_rules
$nat_rule->nat_rules($nat_rules);
```

METHODS

This section describes all the methods that you can use to configure and retrieve the attribute values of a Threat Protection NAT Port.

address( )

Use this method to set or retrieve the IP address for the threat protection NAT mapping rule.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

The valid value is a desired IP address in string format.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get attribute value
my $value = $object->address();

#Modify attribute value
$object->address('1.1.1.1');
```

cidr( )

Use this method to set or retrieve the network CIDR for the threat protection NAT mapping rule.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

The valid value is an unsigned integer.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get attribute value
my $value = $object->cidr();

#Modify attribute value
$object->cidr(8);
```

end_address( )

Use this method to set or retrieve the end address for the range of the threat protection NAT mapping rule.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

The valid value is a desired IP address in string format.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get attribute value
my $value = $object->end_address();
```

```
#Modify attribute value
$object->end_address('1.1.0.255');
```

nat_ports( )

Use this method to set or retrieve the NAT port configuration for the threat protection NAT mapping rule.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

The valid value is an array of Infoblox::Grid::ThreatProtection::NATPort objects.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get attribute value
my $value = $object->nat_ports();

#Modify attribute value
$object->nat_ports($nat_ports);
```

network( )

Use this method to set or retrieve the network address for the threat protection NAT mapping rule.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

The valid value is an unsigned integer.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get attribute value
my $value = $object->network();

#Modify attribute value
$object->network('10.0.0.0');
```

rule_type( )

Use this method to set or retrieve the rule type for the threat protection NAT mapping rule.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

The valid values are 'ADDRESS', 'RANGE', 'NETWORK'.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get attribute value
my $value = $object->rule_type();

#Modify attribute value
$object->rule_type('RANGE');
```

start_address( )

Use this method to set or retrieve the start address of the range for the threat protection NAT mapping rule.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

The valid value is a desired IP address in string format.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get attribute value
my $value = $object->start_address();

#Modify attribute value
$object->start_address('1.1.0.255');
```

AUTHOR

Infoblox Inc. http://www.infoblox.com/

SEE ALSO

Infoblox::Grid::ThreatProtection::NATPort, Infoblox::Grid::ThreatProtection->nat_rules(), Infoblox::Grid::ThreatProtection, Infoblox::Grid::Member::ThreatProtection->nat_rules(), Infoblox::Grid::Member::ThreatProtection.

COPYRIGHT

Copyright (c) 2017 Infoblox Inc.

Infoblox::Grid::ThreatProtection::Profile - Threat Protection profile object.

- NAME
- DESCRIPTION
- CONSTRUCTOR
- SESSION METHODS
  - Infoblox::Session->add( )
  - Infoblox::Session->get( )
  - Infoblox::Session->modify( )
  - Infoblox::Session->remove( )
  - Infoblox::Session->search( )
- METHODS
  - comment( )
  - current_ruleset( )
  - disable_multiple_dns_tcp_request( )
  - events_per_second_per_rule( )
  - extattrs( )
  - extensible_attributes( )
  - members( )
  - name( )
  - override_events_per_second_per_rule( )
  - override_current_ruleset( )
  - override_disable_multiple_dns_tcp_request( )
  - source( )
- SAMPLE CODE
- AUTHOR
- SEE ALSO
- COPYRIGHT

NAME

Infoblox::Grid::ThreatProtection::Profile - Threat Protection profile object.

DESCRIPTION

The Threat Protection profile object facilitates configuring groups of Threat Protection members that have similar traffic properties. A member can be either associated with a Threat Protection profile or inherit the ruleset from the Grid or override the ruleset individually at the member level.

CONSTRUCTOR

```
 my $object = Infoblox::Grid::ThreatProtection::Profile->new(
     name                                    => $string,
#Required
     comment                                 => $string,
#Optional / Default is undefined
     current_ruleset                         => $ruleset,
#Optional / Default is undefined
     disable_multiple_dns_tcp_request        => 'true' | 'false',
#Optional / Default is 'true'
     events_per_second_per_rule              => $uint,
#Optional / Default is undefined
     extattrs                                => { $string => $extattr, ... },
#Optional / Default is undefined
     extensible_attributes                   => {$string => $string | $num, $string => [ $string | $num,
... ], ... }, #Optional / Default is undefined
     members                                 => [$host_name | ... ],
#Optional / Default is empty list
     override_current_ruleset                => 'true' | 'false',
#Optional / Default is 'false'
     override_disable_multiple_dns_tcp_request => 'true' | 'false',
#Optional / Default is 'false'
     override_events_per_second_per_rule     => 'true' | 'false',
#Optional / Default is 'false'
     source                                  => $tp_member | $tp_profile,
#Optional / Default is undefined
 );
```

You cannot set both extattrs and extensible_attributes attributes at the same time.

SESSION METHODS

The following functions can be applied to a Infoblox::Grid::ThreatProtection::Profile object.

Infoblox::Session->add( )

Use this method to add the object to the Infoblox appliance. See Infoblox::Session->add() for parameters and return values.

Example

```
#Construct object
my $object = Infoblox::Grid::ThreatProtection::Profile->new(
    name => 'profile1',
);

#Submit object for addition
$session->add($object);
```

Infoblox::Session->get( )

Use this method to retrieve all the matching objects from the Infoblox appliance. See Infoblox::Session->get() for parameters and return values.

Key References

```
Apply the following attributes to get a specific Threat Protection profile object:

    name                  - Optional. The Threat Protection profile name.
    comment               - Optional. The Threat Protection profile comment.
    extattrs              - Optional. A hash reference containing Infoblox::Grid::Extattr object.
    extensible_attributes - Optional. A hash reference containing extensible attributes.
```

Example

```
my @retrieved_objs = $session->get(
    'object' => 'Infoblox::Grid::ThreatProtection::Profile',
    'name'   => 'profile1',
);
```

Infoblox::Session->modify( )

Use this method to modify an object in the Infoblox appliance. See Infoblox::Session->modify() for parameters and return values.

Example

```
#Modify events_per_second_per_rule value
$object->events_per_second_per_rule('2');

#Submit modification
my $response = $session->modify($object);
```

Infoblox::Session->remove( )

Use this method to remove the object from the Infoblox appliance. See Infoblox::Session->remove() for parameters and return values.

To remove a specific object, first use `get()` or `search()` to retrieve the specific DHCP range object, and then submit this object for removal.

Example

```
#Get the objects
my @retrieved_objs = $session->get(
    object => 'Infoblox::Grid::ThreatProtection::Profile',
    name   => 'profile1',
);

#Find the desired object from the retrieved list
my $desired_object = $retrieved_objs[0];

#Submit for removal
my $response = $session->remove($desired_object);
```

Infoblox::Session->search( )

Use this method to search for objects in the Infoblox appliance. See Infoblox::Session->search() for parameters and return values.

Key References

```
Apply the following attributes to search for the Threat Protection profile object:
```

```
name                  - Optional. The Threat Protection profile name (regular expression).
comment               - Optional. The Threat Protection profile comment (regular expression).
extattrs              - Optional. A hash reference containing Infoblox::Grid::Extattr object.
extensible_attributes - Optional. A hash reference containing extensible attributes.
```

For more information about searching extensible attributes, see Infoblox::Grid::ExtensibleAttributeDef/Searching Extensible Attributes.

Examples

```
# search for objects
my @retrieved_objs = $session->search(
    object      => 'Infoblox::Grid::ThreatProtection::Profile',
    comment     => '.*comment',
);

# search for objects defining 'Santa Clara' for 'Site' extensible attribute
my @retrieved_objs = $session->search(
    object               => 'Infoblox::Grid::ThreatProtection::Profile',
    extensible_attributes => {'Site' => 'Santa Clara'},
);
```

METHODS

This object provides information about the Threat Protection profile settings.

comment( )

Use this method to set or retrieve the Threat Protection profile descriptive comment.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

Valid value is a desired comment in a string format.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get comment value
my $value = $object->comment();

#Modify comment value
$object->comment('new comment');
```

current_ruleset( )

Use this method to set or retrieve the current Threat Protection profile ruleset.

Setting this method to a defined value implicitly sets the override_current_ruleset method to 'true'. Setting the parameter to undefined causes the appliance to use the Grid default and automatically resets the override_current_ruleset attribute to 'false'.

Note that when current_ruleset is set to a defined value and override_dcurrent_ruleset is set to 'false', the last operation takes precedence. Thus the sequence $object->current_ruleset($rs); $object->override_current_ruleset('false'); will set override_current_ruleset to 'false', and the sequence $object->override_current_ruleset('false'); $object->current_ruleset($rs); will result in override_current_ruleset='true'.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

Valid value is an Infoblox::Grid::ThreatProtection::Ruleset object.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get current_ruleset value
my $value = $object->current_ruleset();
```

```
#Modify current_ruleset value
$object->current_ruleset($rs);
```

disable_multiple_dns_tcp_request( )

Use this method to disable multiple BIND responses via TCP connection.

Setting this method to a defined value implicitly sets the override_disable_multiple_dns_tcp_request method to 'true'. Setting the parameter to undefined causes the appliance to use the Grid default and automatically resets the override_disable_multiple_dns_tcp_request attribute to 'false'.

Note that when config is set to a defined value and override_disable_multiple_dns_tcp_request is set to 'false', the last operation takes precedence. Thus the sequence $object->disable_multiple_dns_tcp_request($value); $object->override_disable_multiple_dns_tcp_request('false'); will set override_disable_multiple_dns_tcp_request to 'false', and the sequence $object->override_disable_multiple_dns_tcp_request('false'); $object->disable_multiple_dns_tcp_request($value); will result in override_disable_multiple_dns_tcp_request='true'.

Parameter

Set the parameter to 'true' to disable multiple TCP requests. Set the parameter to 'false' to enable multiple TCP requests.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
# Get attribute value
my $value = $object->disable_multiple_dns_tcp_request();
# Modify attribute value
$object->disable_multiple_dns_tcp_request('true');
```

events_per_second_per_rule( )

Use this method to set or retrieve the number of events logged per second per rule.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

The number of events logged per second per rule in string format.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get events_per_second_per_rule value
my $value = $object->events_per_second_per_rule();

#Modify events_per_second_per_rule value
$object->events_per_second_per_rule('2');
```

extattrs( )

Use this method to set or retrieve the extensible attributes that are associated with a Threat Protection profile object.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

Valid value is a hash reference that contains the names of extensible attributes and their associated values (Infoblox::Grid::Extattr objects).

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get extattrs value
```

```
my $value = $object->extattrs();

#Modify extattrs value
$object->extattrs({'Site' => $extattr1, 'Administrator' => $extattr2});
```

extensible_attributes( )

Use this method to set or retrieve the extensible attributes that are associated with a Threat Protection profile object.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

For valid values for extensible attributes, see Infoblox::Grid::ExtensibleAttributeDef/Extensible Attribute Values.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get extensible attributes value
my $value = $object->extensible_attributes();

#Modify extensible attributes value
$object->extensible_attributes({'Site' => 'Santa Clara', 'Administrator' => ['Peter', 'Tom']});
```

members( )

Use this method to set or retrieve the list of members that are using the Threat Protection profile.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

The valid value is an array reference that contains desired Threat Protection member names in a string format.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get members value
my $value = $object->members();

#Modify members value
$object->members(['infoblox.localdomain']);
```

name( )

Use this method to retrieve a Threat Protection profile name.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

The valid value is a desired Threat Protection profile name in a string format.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get name value
my $value = $object->name();

#Modify name value
$object->name('new_name');
```

override_events_per_second_per_rule( )

Use this method to set or retrieve the override flag for the number of events logged per second per rule.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

Specify 'true' to set the override_events_per_second_per_rule flag or 'false' to deactivate/unset it. The default value is 'false'.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get override_events_per_second_per_rule value
my $value = $object->override_events_per_second_per_rule();

#Modify override_events_per_second_per_rule value
$object->override_events_per_second_per_rule('true');
```

override_current_ruleset( )

The override_current_ruleset attribute controls whether the current ruleset value is used for Threat Protection profile instead of the Grid default.

The override_current_ruleset attribute can be specified explicitly. It is also set implicitly when current_ruleset is set to a defined value.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

Specify 'true' to set the override_current_ruleset flag or 'false' to deactivate/unset it. The default value is 'false'.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get override_current_ruleset value
my $override = $object->override_current_ruleset();

#Modify override_current_ruleset
$object->override_current_ruleset('true');
```

override_disable_multiple_dns_tcp_request( )

The override_disable_multiple_dns_tcp_request attribute controls whether the Threat Protection profile disable_multiple_dns_tcp_request value is used instead of the Grid default.

The override_disable_multiple_dns_tcp_request attribute can be specified explicitly. It is also set implicitly when disable_multiple_dns_tcp_request is set to a defined value.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

Specify 'true' to set the override_disable_multiple_dns_tcp_request flag or 'false' to unset it. The default value is 'false'.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get override_disable_multiple_dns_tcp_request value
my $value = $object->override_disable_multiple_dns_tcp_request();

#Modify override_disable_multiple_dns_tcp_request value
$object->override_disable_multiple_dns_tcp_request('true');
```

source( )

Use this method to set the source for cloning the profile settings. This is a write-only attribute and can be used only on create operation.

Include the specified parameter to set the attribute value.

Parameter

The valid value is an Infoblox::Grid::Member::ThreatProtection or Infoblox::Grid::ThreatProtection::Profile object reference.

Returns

The method returns true when the modification succeeds, and returns false when the operation fails.

Example

```
#Modify source value
$object->source($profile1);
```

SAMPLE CODE

The following sample code demonstrates the different functions that can be applied to an object, such as add, search, modify, and remove. This sample also includes error handling for the operations.

#Preparation prior to a Threat Protection profile insertion

```
#PROGRAM STARTS: Include all the modules that will be used
use strict;
use Infoblox;

#Create a session to the Infoblox appliance

my $session = Infoblox::Session->new(
               master   => "192.168.1.2", #appliance host ip
               username => "admin",       #appliance user login
               password => "infoblox"     #appliance password
               );

unless ($session) {
       die("Construct session failed: ",
               Infoblox::status_code() . ":" . Infoblox::status_detail());
}
print "Session created successfully\n";
```

#Create a Threat Protection profile

```
my $atp_profile = Infoblox::Grid::ThreatProtection::Profile->new(
               name    => "atp_profile1",
               comment => "add TP profile",
               );

unless ($atp_profile) {
       die("Construct Threat Protection profile failed: ",
               Infoblox::status_code() . ":" . Infoblox::status_detail());
}

#Add the Threat Protection profile object to the Infoblox appliance through a session
$session->add($atp_profile)
       or die("Add Threat Protection profile failed: ",
                       $session->status_code() . ":" . $session->status_detail());
print "Threat Protection profile object added to server successfully\n";
```

#Search for a specific Threat Protection profile

```
#Search all Threat Protection profiles that matches "atp"
my @retrieved_objs = $session->search(
               object => "Infoblox::Grid::ThreatProtection::Profile",
               name   => 'atp'
               );
my $object = $retrieved_objs[0];

unless ($object) {
       die("Search Threat Protection profile failed: ",
               $session->status_code() . ":" . $session->status_detail());
}
print "Search Threat Protection profile object found at least 1 matching entry\n";
```

#Get and modify a Threat Protection profile

```
#Get a Threat Protection profile through the session
my @retrieved_objs = $session->get(
               object => "Infoblox::Grid::ThreatProtection::Profile",
               name   => "atp_profile1"
               );
my $object = $retrieved_objs[0];

unless ($object) {
       die("Get Threat Protection profile failed: ",
               $session->status_code() . ":" . $session->status_detail());
}
print "Get Threat Protection profile object found at least 1 matching entry\n";

#Modify one of the attributes of the specified Threat Protection profile
$object->comment("this is new comment");

#Apply the changes
$session->modify($object)
       or die("Modify Threat Protection profile failed: ",
               $session->status_code() . ":" . $session->status_detail());
print "Threat Protection profile object modified successfully \n";
```

#Remove a Threat Protection profile

```
#Get Threat Protection profile through the session
my @retrieved_objs = $session->get(
    object => "Infoblox::Grid::ThreatProtection::Profile",
    name   => "atp_profile1"
);
my $object = $retrieved_objs[0];
unless ($object) {
    die("Get Threat Protection profile failed: ",
        $session->status_code() . ":" . $session->status_detail());
}
print "Get Threat Protection profile object found at least 1 matching entry\n";

#Submit the object for removal
$session->remove($object)
       or die("Remove Threat Protection profile failed: ",
               $session->status_code() . ":" . $session->status_detail());
print "Threat Protection profile object removed successfully \n";

####PROGRAM ENDS####
```

AUTHOR

Infoblox Inc. http://www.infoblox.com/

SEE ALSO

Infoblox::Session, Infoblox::Session->add(), Infoblox::Session->get(), Infoblox::Session->modify(), Infoblox::Session->remove(), Infoblox::Session->search(), Infoblox::Grid::ThreatProtection::Ruleset, Infoblox::Grid::Member::ThreatProtection, Infoblox::Grid::ThreatProtection::Profile,

---

COPYRIGHT

Copyright (c) 2017 Infoblox Inc.

Infoblox::Grid::ThreatProtection - Grid Threat Protection object.

- NAME
- DESCRIPTION
- SESSION METHODS
  - Infoblox::Session->get( )
  - Infoblox::Session->modify( )
- METHODS
  - current_ruleset( )
  - enable_nat_rules( )
  - events_per_second_per_rule( )
  - enable_auto_download( )
  - enable_scheduled_download( )
  - rule_update_policy( )
  - last_checked_for_update( )
  - last_rule_update_version( )
  - nat_rules( )
  - scheduled_download( )
  - test_threat_protection_server_connectivity( )
  - disable_multiple_dns_tcp_request( )
- AUTHOR
- SEE ALSO
- COPYRIGHT

NAME

Infoblox::Grid::ThreatProtection - Grid Threat Protection object.

DESCRIPTION

This object provides information about the Grid Threat protection settings.

SESSION METHODS

The following functions can be applied to a Grid::ThreatProtection object.

Infoblox::Session->get( )

Use this method to retrieve all the matching objects from the Infoblox appliance. See Infoblox::Session->get() for parameters and return values.

Key References

```
Apply the following attributes to get a specific Grid Threat Protection object
 object - Required. It must be set to "Infoblox::Grid::ThreatProtection".
```

Example

```
my @retrieved_objs = $session->get(
    'object' => 'Infoblox::Grid::ThreatProtection',
);
```

Infoblox::Session->modify( )

Use this method to modify an object in the Infoblox appliance. See Infoblox::Session->modify() for parameters and return values.

Example

```
# Use this method to modify some of the attributes of the retrieved_objs object.
$retrieved_objs->events_per_second_per_rule('2');
$retrieved_objs->rule_update_policy('MANUAL');
# Submit modification
my $response = $session->modify( $retrieved_objs );
```

METHODS

This section describes all the methods that you can use to configure and retrieve values of the Threat Protection object.

current_ruleset( )

Use this method to set or retrieve the current Grid ruleset.

Parameter

Valid value is an Infoblox::Grid::ThreatProtection::Ruleset object.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get attribute value
my $value = $obj->current_ruleset();
#Modify attribute value
$obj->current_ruleset($rs);
```

enable_nat_rules( )

Use this method to enable or disable the NAT Threat Protection feature.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Paramter

Specify 'true' to enable NAT mapping support for threat protection or 'false' to disable it. The default value is 'false'.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get attribute value
my $value = $object->enable_nat_rules();

#Modify attribute value
$object->enable_nat_rules('true');
```

events_per_second_per_rule( )

Use this method to set or retrieve the number of events logged per second per rule.

Parameter

The number of events logged per second per rule in string format.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
# Get attribute value
my $value = $object->events_per_second_per_rule();
# Modify attribute value
$object->events_per_second_per_rule('2');
```

enable_auto_download( )

Use this method to enable or disable the auto download service.

Parameter

Specify 'true' to enable the auto download service or 'false' to disable it. The default value is 'false'.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get attribute value
my $value = $obj->enable_auto_download();
#Modify attribute value
$obj->enable_auto_download('true');
```

enable_scheduled_download( )

Use this method to enable or disable the scheduled download. The default frequency is once every 24 hours if it is disabled.

Parameter

Specify 'true' to enable the scheduled download or 'false' to disable it. The default value is 'false'.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get attribute value
my $value = $obj->enable_scheduled_download();
#Modify attribute value
$obj->enable_scheduled_download('true');
```

rule_update_policy( )

Use this method to set or retrieve the updated rule policy.

Parameter

Valid value is 'AUTOMATIC' or 'MANUAL'.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
# Get attribute value
my $value = $object->rule_update_policy();
# Modify attribute value
$object->rule_update_policy('MANUAL');
```

last_checked_for_update( )

Use this method to retrieve the time when the Grid last checked for updates.

Parameter

None

Returns

The method returns the attribute value, the number of seconds that have elapsed since January 1st, 1970 UTC.

Example

```
# Get attribute value
my $value = $object->last_checked_for_update();
```

last_rule_update_version( )

Use this method to retrieve the last rule update version. This is a read-only attribute.

Parameter

None

Returns

The method returns the attribute value.

Example

```
# Get attribute value
my $value = $object->last_rule_update_version();
```

nat_rules( )

Use this method to set or retrieve the list of NAT mapping rules for threat protection.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

The valid value is an array of Infoblox::Grid::ThreatProtection::NATRule objects.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get attribute value
my $value = $obj->nat_rules();
#Modify attribute value
$obj->nat_rules($nat_rules);
```

scheduled_download( )

Use this method to set or retrieve the schedule setting for automatic rule update.

Parameter

Valid value is an Infoblox::Grid::ScheduleSetting object.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get attribute value
my $value = $obj->scheduled_download();
#Modify attribute value
$obj->scheduled_download($schedule);
```

test_threat_protection_server_connectivity( )

Use this method ot test the connectivity with the Infoblox threat protection server.

Key Reference

None

Returns

The method returns an Infoblox::Grid::TestResult object if connectivity test occurs and 0 otherwise.

Example

```
my $res = $obj->test_threat_protection_server_connectivity();
```

disable_multiple_dns_tcp_request( )

Use this method to disable multiple BIND responses via TCP connection.

Parameter

Set the parameter to "true" to disable multiple TCP requests. Set the parameter to "false" to enable multiple TCP requests.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
# Get attribute value
my $value = $object->disable_multiple_dns_tcp_request();
# Modify attribute value
$object->disable_multiple_dns_tcp_request('true');
```

AUTHOR

Infoblox Inc. http://www.infoblox.com/

SEE ALSO

Infoblox::Session, Infoblox::Grid::Member::ThreatProtection, Infoblox::Grid::ScheduleSetting, Infoblox::Grid::ThreatProtection::NATRule

COPYRIGHT

Copyright (c) 2017 Infoblox Inc.

Infoblox::Grid::ThreatProtection::RuleConfig - The rule configuration.

- NAME
- DESCRIPTION
- CONSTRUCTOR
- SESSION METHODS
- METHODS
  - action( )
  - log_severity( )
  - params( )
- AUTHOR
- SEE ALSO
- COPYRIGHT

NAME

Infoblox::Grid::ThreatProtection::RuleConfig - The rule configuration.

DESCRIPTION

This is an in-memory struct.

CONSTRUCTOR

```
my $object = Infoblox::Grid::ThreatProtection::RuleConfig->new(
    'action'          => 'ALERT' | 'PASS' | 'DROP',                              # optional
    'log_severity'    => 'CRITICAL' | 'MAJOR' | 'WARNING' | 'INFORMATIONAL',     # optional
    'params'          => [$param1, $param2, ...],                                # required
);
```

SESSION METHODS

The object does not support any session methods.

METHODS

This section describes all the methods that you can use to set or retrieve the attribute values of the struct.

action( )

Use this method to set or retrieve the action of the rule.

Parameter

Valid value is 'ALERT', 'PASS' or 'DROP'.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
# Get attribute value
my $value = $object->action();
# Modify attribute value
$object->action('DROP');
```

log_severity( )

Use this method to set or retrieve the severity for the rule log.

Parameter

Valid value is 'CRITICAL', 'MAJOR', 'WARNING' or 'INFORMATIONAL'.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
# Get attribute value
my $value = $object->log_severity();
# Modify attribute value
$object->log_severity('WARNING');
```

params( )

Use this method to set or retrieve rule parameters.

Parameter

Valid value is an array that contains Infoblox::Grid::ThreatProtection::RuleParam structs.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
# Get attribute value
my $value = $object->params();

# create expected RuleParam struct
my $param = Infoblox::Grid::ThreatProtection::RuleParam->new(
    'name' => 'SRC_IP',
    'value' => 'abcd::/64',
);

# Modify attribute value
$object->params([$param]);
```

AUTHOR

Infoblox Inc. http://www.infoblox.com/

SEE ALSO

Infoblox::Grid::ThreatProtection::Rule, Infoblox::Grid::ThreatProtection::RuleParam

---

COPYRIGHT

Copyright (c) 2017 Infoblox Inc.

Infoblox::Grid::Member::ThreatProtection - Member Threat Protection object.

- NAME
- DESCRIPTION
- SESSION METHODS
  - Infoblox::Session->get( )
  - Infoblox::Session->modify( )
  - Infoblox::Session->search( )
- METHODS
  - current_ruleset( )
  - enable_nat_rules( )
  - enable_service( )
  - events_per_second_per_rule( )
  - grid_member( )
  - hardware_model( )
  - hardware_type( )
  - override_events_per_second_per_rule( )
  - disable_multiple_dns_tcp_request( )
  - nat_rules( )
  - override_current_ruleset( )
  - override_disable_multiple_dns_tcp_request( )
  - override_enable_nat_rules( )
  - profile( )
- AUTHOR
- SEE ALSO
- COPYRIGHT

NAME

Infoblox::Grid::Member::ThreatProtection - Member Threat Protection object.

DESCRIPTION

This object provides information about the member Threat protection settings.

SESSION METHODS

The following functions can be applied to a Grid::Member::ThreatProtection object.

Infoblox::Session->get( )

Use this method to retrieve all the matching objects from the Infoblox appliance. See Infoblox::Session->get() for parameters and return values.

Key References

```
Apply the following attributes to get a specific Member Threat Protection object.

 grid_member    - Optional. The Grid member name.
 hardware_model - Optional. The Grid member hardware model name.
 hardware_type  - Optional. The Grid member hardware type.
 profile        - Optional. The Threat Protection profile object.
```

Example

```
my @retrieved_objs = $session->get(
    'object'      => 'Infoblox::Grid::Member::ThreatProtection',
    'grid_member' => 'host.com',
);
```

Infoblox::Session->modify( )

Use this method to modify an object in the Infoblox appliance. See Infoblox::Session->modify() for parameters and return values.

Example

```
# Use this method to modify attributes of an member object.
$member->enable_service('true');
$member->override_events_per_second_per_rule('true');
$member->events_per_second_per_rule('2');
$member->override_disable_multiple_dns_tcp_request('true');
```

```
$member->disable_multiple_dns_tcp_request('true');

# Submit modification
my $response = $session->modify( $member );
```

Infoblox::Session->search( )

Use this method to retrieve all the matching objects from the Infoblox appliance. See Infoblox::Session->search() for parameters and return values.

Key References

```
Apply the following attributes to search a specific Member Threat Protection object.

 grid_member    - Optional. The Grid member name.
 hardware_model - Optional. The Grid member hardware model name (regular expression).
 hardware_type  - Optional. The Grid member hardware type (regular expression).
 profile        - Optional. The Threat Protection profile object.
```

Example

```
my @retrieved_objs = $session->search(
    'object'      => 'Infoblox::Grid::Member::ThreatProtection',
    'grid_member' => 'host.com',
);
```

METHODS

This object provides information about the member threat protection settings.

current_ruleset( )

Use this method to set or retrieve the current member ruleset.

Setting this method to a defined value implicitly sets the override_current_ruleset method to "true". Setting the parameter to undefined causes the appliance to use the grid default and automatically resets the override_current_ruleset attribute to "false".

Note that when current_ruleset is set to a defined value and override_dcurrent_ruleset is set to "false", the last operation takes precedence. Thus the sequence $object->current_ruleset($rs); $object->override_current_ruleset("false"); will set override_current_ruleset to "false", and the sequence $object->override_current_ruleset("false"); $object->current_ruleset($rs); will result in override_current_ruleset="true".

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

Valid value is an Infoblox::Grid::ThreatProtection::Ruleset object.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get attribute value
my $value = $obj->current_ruleset();
#Modify attribute value
$obj->current_ruleset($rs);
```

enable_nat_rules( )

Use this method to enable or disable NAT (Network Address Translation) mapping for threat protection at the member level.

Setting this method to a defined value implicitly sets the override_enable_nat_rules method to "true". Setting the parameter to undefined causes the appliance to use the Grid default and automatically resets the override_enable_nat_rules attribute to "false".

Note that when enable_nat_rules is set to a defined value and override_denable_nat_rules is set to "false", the last operation takes precedence. Thus the sequence $object->enable_nat_rules($nat_rules); $object->override_enable_nat_rules("false"); will set override_enable_nat_rules to "false", and the sequence $object->override_enable_nat_rules("false"); $object->enable_nat_rules($nat_rules); will result in override_enable_nat_rules="true".

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Paramter

Specify 'true' to enable NAT mapping for threat protection or 'false' to disable it. The default value is 'false.'

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get attribute value
my $value = $object->enable_nat_rules();

#Modify attribute value
$object->enable_nat_rules('true');
```

enable_service( )

Use this method to enable or disable the Threat protection service on the member.

Parameter

Set the parameter to "true" to enable Threat protection service on the member. Set the parameter to "false" to disable the service.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
# Get attribute value
my $value = $object->enable_service();
# Modify attribute value
$object->enable_service('true');
```

events_per_second_per_rule( )

Use this method to set or retrieve the number of events logged per second per rule.

Parameter

The number of events logged per second per rule in string format.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
# Get attribute value
my $value = $object->events_per_second_per_rule();
# Modify attribute value
$object->events_per_second_per_rule('2');
```

grid_member( )

Use this method to retrieve a Grid member name. This is a read-only attribute.

Parameter

None

Returns

The method returns the attribute value.

Example

```
# Get attribute value
my $value = $object->grid_member();
```

hardware_model( )

Use this method to retrieve the hardware model name of the Grid member. This is a read-only attribute.

Parameter

None

Returns

The method returns the attribute value.

Example

```
#Get hardware_model value
my $value = $object->hardware_model();
```

hardware_type( )

Use this method to retrieve the type of the Grid member hardware. This is a read-only attribute.

Parameter

None

Returns

The method returns the attribute value.

Example

```
#Get hardware_type value
my $value = $object->hardware_type();
```

override_events_per_second_per_rule( )

Use this method to set or retrieve the override flag for the number of events logged per second per rule.

Parameter

Specify "true" to set the override_events_per_second_per_rule flag or "false" to deactivate/unset it. The default value is "false".

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
# Get attribute value
my $value = $object->override_events_per_second_per_rule();
# Modify attribute value
$object->override_events_per_second_per_rule('true');
```

disable_multiple_dns_tcp_request( )

Use this method to disable multiple BIND responses via TCP connection.

Setting this method to a defined value implicitly sets the override_disable_multiple_dns_tcp_request method to "true". Setting the parameter to undefined causes the appliance to use the Grid default and automatically resets the override_disable_multiple_dns_tcp_request attribute to "false".

Note that when config is set to a defined value and override_disable_multiple_dns_tcp_request is set to "false", the last operation takes precedence. Thus the sequence $object->disable_multiple_dns_tcp_request($value); $object->override_disable_multiple_dns_tcp_request("false"); will set override_disable_multiple_dns_tcp_request to "false", and the sequence $object->override_disable_multiple_dns_tcp_request("false"); $object->disable_multiple_dns_tcp_request($value); will result in override_disable_multiple_dns_tcp_request="true".

Parameter

Set the parameter to "true" to disable multiple TCP requests. Set the parameter to "false" to enable multiple TCP requests.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
# Get attribute value
```

```
my $value = $object->disable_multiple_dns_tcp_request();
# Modify attribute value
$object->disable_multiple_dns_tcp_request('true');
```

nat_rules( )

Use this method to set or retrieve the list of NAT mapping rules cofigured for the member.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

The valid value is an array of Infoblox::Grid::ThreatProtection::NATRule objects.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get attribute value
my $value = $obj->nat_rules();
#Modify attribute value
$obj->nat_rules($nat_rules);
```

override_current_ruleset( )

The override_current_ruleset attribute controls whether the member's current ruleset value is used, instead of the Grid default.

The override_current_ruleset attribute can be specified explicitly. It is also set implicitly when current_ruleset is set to a defined value.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

Specify "true" to set the override_current_ruleset flag or "false" to deactivate/unset it. The default value is "false".

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get override_current_ruleset from the member
my $override = $object->override_current_ruleset();
#Modify override_current_ruleset
$object->override_current_ruleset("true");
```

override_disable_multiple_dns_tcp_request( )

Use this method to set or retrieve the override flag for disabling multiple BIND responses via TCP connection.

Parameter

Specify "true" to set the override_disable_multiple_dns_tcp_request flag or "false" to unset it. The default value is "false".

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
# Get attribute value
my $value = $object->override_disable_multiple_dns_tcp_request();
# Modify attribute value
$object->override_disable_multiple_dns_tcp_request('true');
```

override_enable_nat_rules( )

The override_enable_nat_rules attribute controls whether the member's current enable_nat_rules value is used, instead of the Grid default.

The override_enable_nat_rules attribute can be specified explicitly. It is also set implicitly when enable_nat_rules is set to a defined value.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

Specify "true" to set the override_enable_nat_rules flag or "false" to deactivate/unset it. The default value is "false".

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get override_enable_nat_rules from the member
my $override = $object->override_enable_nat_rules();
#Modify override_enable_nat_rules
$object->override_enable_nat_rules("true");
```

profile( )

Use this method to set or retrieve the Threat Protection profile that is used to group this member.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

The valid value is an Infoblox::Grid::ThreatProtection::Profile object.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get profile value
my $value = $object->profile();

#Modify profile value
$object->profile($profile);
```

AUTHOR

Infoblox Inc. http://www.infoblox.com/

SEE ALSO

Infoblox::Session, Infoblox::Session->get(), Infoblox::Session->modify(), Infoblox::Session->search(), Infoblox::Grid::ThreatProtection::NATRule, Infoblox::Grid::ThreatProtection::Profile, Infoblox::Grid::ThreatProtection,

---

COPYRIGHT

Copyright (c) 2017 Infoblox Inc.

Infoblox::Grid::ThreatProtection::Rule - Grid Threat Protection Rule object.

- NAME
- DESCRIPTION
- CONSTRUCTOR
- SESSION METHODS
  - Infoblox::Session->add( )
  - Infoblox::Session->get( )
  - Infoblox::Session->modify( )
  - Infoblox::Session->remove( )
  - Infoblox::Session->ruleset( )
  - Infoblox::Session->search( )
- METHODS
  - allowed_actions( )
  - category( )
  - comment( )
  - config( )
  - description( )
  - disable( )
  - is_factory_reset_enabled( )
  - name( )
  - sid( )
  - template( )
  - type( )
- AUTHOR
- SEE ALSO
- COPYRIGHT

NAME

Infoblox::Grid::ThreatProtection::Rule - Grid Threat Protection Rule object.

DESCRIPTION

This object provides information about the Grid Threat protection rule settings.

CONSTRUCTOR

```
my $object = Infoblox::Grid::ThreatProtection::Rule->new(
    'comment'     => $string                                    # Optional
    'disable'     => 'true' | 'false',                          # Optional / Default is 'false'
    'config'      => $config,                                   # Required
    'template'    => $template,                                 # Required
);
```

SESSION METHODS

The following functions can be applied to a Grid::ThreatProtection::Rule object.

Infoblox::Session->add( )

Use this method to add an object to the Infoblox appliance. See Infoblox::Session->add() for parameters and return values.

Example

```
    #Get Rule template Object
    my $template = $session->search('object' => 'Infoblox::Grid::ThreatProtection::RuleTemplate', 'name'
  => 'Whitelist source IPv6 address');
    my $config = $template->default_config();
    my $params = $config->params();
    $params->[0]->value('2001::1');
    #Construct an object
    my $custom_rule = Infoblox::Grid::ThreatProtection::Rule->new(
                      'template'    => $template,
                      'comment'     => 'Test Custom Rule',
                      'config'      => $config,
                      'disable'     => 'false');

    #Submit for addition
    my $response = $session->add( $custom_rule );
```

Infoblox::Session->get( )

Use this method to retrieve all the matching objects from the Infoblox appliance. See Infoblox::Session->get() for parameters and return values.

Key References

```
Apply the following attributes to get a specific Grid Threat Protection::Rule object.
 sid    - Optional. The Rule ID.
 name   - Optional. The name of Threat Protection Rule.
```

Example

```
my @retrieved_objs = $session->get(
    'object' => 'Infoblox::Grid::ThreatProtection::Rule',
    'name' => 'Whitelist source IPv6 address',
);
```

Infoblox::Session->modify( )

Use this method to modify an object in the Infoblox appliance. See Infoblox::Session->modify() for parameters and return values.

Example

```
my $modify_rule_template = $session->search('object' =>
'Infoblox::Grid::ThreatProtection::RuleTemplate', 'name' => 'Whitelist source IP address');
my $config = $modify_rule_template->default_config();
$conf->action('PASS');
# Use this method to modify attributes of object.
$retrieved_objs->comment('Comment of Custom Rule');
$retrieved_objs->disable('true');
$retrieved_objs->config($config);
$retrieved_objs->template($modify_rule_template);
# Submit modification
my $response = $session->modify( $retrieved_objs );
```

Infoblox::Session->remove( )

Use this method to remove an object from the Infoblox appliance. See Infoblox::Session->remove() for parameters and return values.

To remove a specific object, first use `get()` or `search()` to retrieve the specific object, and then submit this object for removal.

Example

```
#Get the objects
my @retrieved_objs = $session->get(
    'object' => 'Infoblox::Grid::ThreatProtection::Rule',
    'name' => 'Whitelist source IPv6 address',
);
#Find the desired object from the retrieved list.
my $desired_object = $retrieved_objs[0];
# Submit for removal
my $response = $session->remove( $desired_object );
```

Infoblox::Session->ruleset( )

Use this method to retrieve the ruleset. See Infoblox::Session->add() for parameters and return values.

Example

```
#Get the object
my $rule = $self->{'session'}->search(
    'object' => 'Infoblox::Grid::ThreatProtection::Rule',
    'sid' => '111111',
);

# Retrieve ruleset
my $ruleset = $rule->ruleset();
```

Infoblox::Session->search( )

Use this method to search for objects in the Infoblox appliance. See Infoblox::Session->search() for parameters and return values.

Key References

```
Apply the following attributes to get a specific Grid Threat Protection::Rule object.
```

```
sid    - Optional. The Rule ID.
name   - Optional. The name of Threat Protection Rule.
```

Example

```
my @retrieved_objs = $session->search(
    'object' => 'Infoblox::Grid::ThreatProtection::Rule',
    'name' = 'Whitelist source IP network';
);
```

METHODS

This section describes all the methods that you can use to configure and retrieve values of the Threat Protection Rule.

allowed_actions( )

Use this method to retrieve the allowed actions for the rule. This is a read-only attribute.

Parameter

None

Returns

The method returns the attribute value.

Example

```
# Get attribute value
my $value = $object->allowed_actions();
```

category( )

Use this method to retrieve reference to the rule category. This is a read-only attribute.

Parameter

None

Returns

The method returns the attribute value.

Example

```
# Get attribute value
my $value = $object->category();
```

comment( )

Use this method to set or retrieve user editable comments.

Parameter

Comments in string format.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
# Get attribute value
my $value = $object->comment();
# Modify attribute value
$object->comment('New comment');
```

config( )

Use this method to set or retrieve information about the rule configuration.

This method is required for creating custom rules. The server will set all undefined members of the object to include the 'params' array from the template.

Parameter

Valid value is reference on Infoblox::Grid::ThreatProtection::RuleConfig object.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
# Get attribute value
my $value = $object->config();
$value->log_severity('INFORMATIONAL');
# Modify attribute value
$object->config($value);
```

description( )

Use this method to retrieve the description for the rule. This is a read-only attribute.

Parameter

None

Returns

The method returns the attribute value.

Example

```
# Get attribute value
my $value = $object->description();
```

disable( )

Use this method to set or indicate whether the rule is enabled or disabled for the Grid.

Parameter

Valid value is "true" or "false". The default value is "false".

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
# Get attribute value
my $value = $object->disable();
# Modify attribute value
$object->disable('true');
```

is_factory_reset_enabled( )

Use this method to retrieve the flag that indicates whether the factory reset of this object is enabled or disabled. This is a read-only attribute.

Parameter

None

Returns

The method returns the attribute value.

Example

```
# Get attribute value
my $value = $object->is_factory_reset_enabled();
```

name( )

Use this method to retrieve the rule name. This is a read-only attribute.

Parameter

None

Returns

The method returns the attribute value.

Example

```
# Get attribute value
my $value = $object->name();
```

sid( )

Use this method to retrieve the Rule ID. This is a read-only attribute.

Parameter

None

Returns

The method returns the attribute value.

Example

```
# Get attribute value
my $value = $object->sid();
```

template( )

Use this method to set or retrieve reference to the template object.

Parameter

Valid value is reference on Infoblox::Grid::ThreatProtection::RuleTemplate object.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
# Get attribute value
my $value = $object->template();
# Get Rule Template Object.
my $modify_rule_template = $session->search('object' =>
'Infoblox::Grid::ThreatProtection::RuleTemplate', 'name' => 'Whitelist source IP network');
$object->template($modify_rule_template);
# Submit modification
my $response = $session->modify( $object );
```

type( )

Use this method to retrieve the rule type. This is a read-only attribute.

Parameter

None

Returns

The method returns the attribute value.

Example

```
# Get attribute value
my $value = $object->type();
```

AUTHOR

Infoblox Inc. http://www.infoblox.com/

SEE ALSO

Infoblox::Session, Infoblox::Grid::ThreatProtection, Infoblox::Grid::Member::ThreatProtection::Rule, Infoblox::Grid::ThreatProtection::RuleConfig, Infoblox::Grid::ThreatProtection::RuleParam

---

COPYRIGHT

Copyright (c) 2017 Infoblox Inc.

Infoblox::Grid::ThreatProtection::RuleParam - A Grid Threat Protection Rule parameter object.

- NAME
- DESCRIPTION
- CONSTRUCTOR
- SESSION METHODS
- MODULE METHODS
  - Infoblox::Grid::ThreatProtection::RuleConfig->params( )
- METHODS
  - description( )
  - min( )
  - max( )
  - name( )
  - read_only( )
  - syntax( )
  - value( )
- AUTHOR
- SEE ALSO
- COPYRIGHT

NAME

Infoblox::Grid::ThreatProtection::RuleParam - A Grid Threat Protection Rule parameter object.

DESCRIPTION

The Grid Threat Protection Rule parameter object represents information about Threat Protection Rule Config parameters.

CONSTRUCTOR

```
my $object = Infoblox::Grid::ThreatProtection::RuleParam->new(
    'name'  => $string,     # Required
    'value' => $string,     # Required
);
```

SESSION METHODS

The object does not support any session methods.

MODULE METHODS

This section describes all the functions that you can apply to an ThreatProtection RuleParam object.

Infoblox::Grid::ThreatProtection::RuleConfig->params( )

Use this method to set or retrieve rule parameters. See Infoblox::Grid::ThreatProtection::RuleConfig->params()

Example

```
my $rule_param = Infoblox::Grid::ThreatProtection::RuleParam->new(
  'name'  => 'SRC_IP',
  'value' => 'abcd::',
);

my $rule_config = Infoblox::Grid::ThreatProtection::RuleConfig->new(
  'action'              => 'PASS',
  'log_severity'        => 'MAJOR',
  'params'              => [$rule_param]);
```

METHODS

This section describes all the methods that you can use to set or retrieve the attribute values of the struct.

description( )

Use this method to retrieve the parameter description. This is a read-only attribute.

Parameter

None

Returns

The method returns the attribute value.

Example

```
# Get attribute value
my $value = $object->description();
```

min( )

Use this method to retrieve the minimum value of the parameter if the parameter syntax is UNSIGNED_INT. This is a read-only attribute.

Parameter

None

Returns

The method returns the attribute value.

Example

```
# Get attribute value
my $value = $object->min();
```

max( )

Use this method to retrieve the maximum value of the parameter if the parameter syntax is UNSIGNED_INT. This is a read-only attribute.

Parameter

None

Returns

The method returns the attribute value.

Example

```
# Get attribute value
my $value = $object->max();
```

name( )

Use this method to set or retrieve the parameter name.

Parameter

The parameter name in string format.

Returns

The method returns the attribute value.

Example

```
# Get attribute value
my $value = $object->name();
# Modify attribute value
$object->name('SRC_IP');
```

read_only( )

Use this method to indicate if the parameter is read-only. This is a read-only attribute.

Parameter

None

Returns

The method returns the attribute value.

Example

```
# Get attribute value
my $value = $object->read_only();
```

syntax( )

Use this method to retrieve the parameter syntax. This is a read-only attribute.

Parameter

None

Returns

The method returns the attribute value.

Example

```
# Get attribute value
my $value = $object->syntax();
```

value( )

Use this method to set or retrieve the parameter value.

Parameter

The parameter value in string format.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
# Get attribute value
my $value = $object->value();
# Modify attribute value
$object->value('15');
```

AUTHOR

Infoblox Inc. http://www.infoblox.com/

SEE ALSO

Infoblox::Grid::ThreatProtection::Rule, Infoblox::Grid::ThreatProtection::RuleConfig

---

COPYRIGHT

Copyright (c) 2017 Infoblox Inc.

Infoblox::Grid::MSServer::AdSites - the Active Directory Site object.

- NAME
- DESCRIPTION
- SESSION METHODS
- METHODS
  - default_ip_site_link( )
  - last_sync_ts( )
  - last_sync_detail( )
  - last_sync_status( )
  - ldap_timeout( )
  - ldap_auth_port( )
  - ldap_encryption( )
  - login_name( )
  - login_password( )
  - managed( )
  - override_default_ip_site_link( )
  - override_login( )
  - override_ldap_timeout( )
  - override_synchronization_interval( )
  - read_only( )
  - synchronization_interval( )
- AUTHOR
- SEE ALSO
- COPYRIGHT

NAME

Infoblox::Grid::MSServer::AdSites - the Active Directory Site object.

DESCRIPTION

The object provides information about the Active Directory Site.

SESSION METHODS

The object does not support any session methods. It is retrieved as a part of the Microsoft Server object.

METHODS

this section describes all the methods that you can use to set or retrieve the attribute values of the object.

default_ip_site_link( )

Use this method to set or retrieve the default IP site link name.

Setting this method to a defined value implicitly sets the override_default_ip_site_link method to "true". Setting the parameter to undefined causes the appliance to use the Grid default and automatically resets the override_default_ip_site_link attribute to "false".

Note that when default_ip_site_link is set to a defined value and override_ddefault_ip_site_link is set to "false", the last operation takes precedence. Thus the sequence $object->default_ip_site_link('LAX'); $object->override_default_ip_site_link("false"); will set override_default_ip_site_link to "false", and the sequence $object->override_default_ip_site_link("false"); $object->default_ip_site_link('LAX'); will result in override_default_ip_site_link="true".

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

Valid value is a string with the default IP site link name.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get default_ip_site_link value
my $default_ip_site_link = $ad_sites->default_ip_site_link();
#Modify default_ip_site_link value
$ad sites  default ip site link
```

```
                ->                      ('Site1');
```

last_sync_ts( )

Use this method to retrieve a timestamp of the last LDAP synchronization attempt. This is a read-only attribute.

Parameter

None

Returns

The method returns the attribute value.

Example

```
#Get last_sync_ts value
my $last_sync_ts = $ad_sites->last_sync_ts();
```

last_sync_detail( )

Use this method to retrieve a string containing details of the last LDAP synchronization attempt. This is a read-only attribute.

Parameter

None

Returns

The method returns the attribute value.

Example

```
#Get last_sync_detail value
my $last_sync_detail = $ad_sites->last_sync_detail();
```

last_sync_status( )

Use this method to retrieve a string containing status of the last LDAP synchronization attempt. This is a read-only attribute.

Parameter

None

Returns

The method returns the attribute value that can be 'OK', 'WARNING' or 'ERROR'.

Example

```
#Get last_sync_status value
my $last_sync_status = $ad_sites->last_sync_status();
```

ldap_timeout( )

Use this method to set or retrieve an LDAP connection timeout interval in seconds for the Microsoft Server.

Setting this method to a defined value implicitly sets the override_ldap_timeout method to "true". Setting the parameter to undefined causes the appliance to use the Grid default and automatically resets the override_ldap_timeout attribute to "false".

Note that when ldap_timeout is set to a defined value and override_dldap_timeout is set to "false", the last operation takes precedence. Thus the sequence $object->ldap_timeout(10); $object->override_ldap_timeout("false"); will set override_ldap_timeout to "false", and the sequence $object->override_ldap_timeout("false"); $object->ldap_timeout(10); will result in override_ldap_timeout="true".

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

Valid value is an unsigned integer.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get ldap_timeout value
my $ldap_timeout = $ad_sites->ldap_timeout();
#Modify ldap_timeout value
$ad_sites->ldap_timeout(10);
```

ldap_auth_port( )

Use this method to set or retrieve an LDAP TCP connection port for the Microsoft Server.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

Valid values are unsinged integers between 0 and 65535.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get ldap_auth_port value
my $ldap_auth_port = $ad_sites->ldap_auth_port();
#Modify ldap_auth_port value
$ad_sites->ldap_auth_port(636);
```

ldap_encryption( )

Use this method to set or retrieve an encryption for the LDAP connection for the Microsoft Server.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

Valid value is 'SSL' or 'NONE'.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get ldap_encryption value
my $ldap_encryption = $ad_sites->ldap_encryption();
#Modify ldap_encryption value
$ad_sites->ldap_encryption('SSL');
```

login_name( )

Use this method to set or retrieve the Microsoft Server login name.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

When setting a value that requires a domain name, prepend the domain information followed by a backslash to the username: "domain\username".

Parameter

The valid value is a string that contains the login name.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get login value
my $login = $ad_sites->login();
#Modify login value
$ad_sites->login("newuser");
```

login_password( )

Use this method to set a password for the Microsoft Server. This field is used only if the login field for this member is defined. This is a write-only attribute.

This field cannot be retrieved.

Parameter

The valid value is a string that contains the password.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

Example

```
#Set the password
$ad_sites->password("iG&ojG97Y");
```

managed( )

Use this method to set or retrieve the flag that indicates whether the sites of the Microsoft Server are to be synchronized with the assigned managing member or not.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

Specify "true" to set the disable flag or "false" to deactivate/unset it. The default value is "false".

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get managed value
$managed = $ad_sites->managed();
#Modify managed value
$ad_sites->managed("true");
```

override_default_ip_site_link( )

The override_default_ip_site_link attribute controls whether the default_ip_site_link method values of the Microsoft Server are used, instead of the Grid default.

The override_default_ip_site_link attribute can be specified explicitly. It is also set implicitly when default_ip_site_link is set to a defined value.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

Set the parameter to "true" to override the Grid-level setting for default_ip_site_link. Set the parameter to "false" to inherit the Grid-level setting for default_ip_site_link. The default value is "false".

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get override_default_ip_site_link
my $override_default_ip_site_link = $ad_sites->override_default_ip_site_link();
#Modify override_default_ip_site_link
$ad_sites->override_default_ip_site_link("true");
```

override_login( )

The override_login attribute controls whether the login_name method values of the Microsoft Server are used, instead of the Grid default.

The override_login attribute can be specified explicitly. It is also set implicitly when login_name is set to a defined value.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

Set the parameter to "true" to override the Grid-level setting for login_name. Set the parameter to "false" to inherit the Grid-level setting for login_name. The default value is "false".

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get override_login
my $override_login = $ad_sites->override_login();
#Modify override_login
$ad_sites->override_login("true");
```

override_ldap_timeout( )

The override_ldap_timeout attribute controls whether the ldap_timeout method values of the Microsoft Server are used, instead of the Grid default.

The override_ldap_timeout attribute can be specified explicitly. It is also set implicitly when ldap_timeout is set to a defined value.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

Set the parameter to "true" to override the Grid-level setting for ldap_timeout. Set the parameter to "false" to inherit the Grid-level setting for ldap_timeout. The default value is "false".

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get override_ldap_timeout
my $override_ldap_timeout = $ad_sites->override_ldap_timeout();
#Modify override_ldap_timeout
$ad_sites->override_ldap_timeout("true");
```

override_synchronization_interval( )

The override_synchronization_interval attribute controls whether the synchronization_interval method values of the Microsoft Server are used, instead of the Grid default.

The override_synchronization_interval attribute can be specified explicitly. It is also set implicitly when synchronization_interval is set to a defined value.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

Set the parameter to "true" to override the Grid-level setting for synchronization_interval. Set the parameter to "false" to inherit the Grid-level setting for synchronization_interval. The default value is "false".

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get override_synchronization_interval
my $override_synchronization_interval = $ad_sites->override_synchronization_interval();
#Modify override_synchronization_interval
$ad_sites->override_synchronization_interval("true");
```

read_only( )

Use this method to enable or disable read-only synchronization of the sites for the Active Directory Domain.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

Specify "true" to set the disable flag or "false" to deactivate/unset it.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get read_only value
$read_only = $ad_sites->read_only();
#Modify read_only value
$ad_sites->read_only("true");
```

synchronization_interval( )

Use this method to set or retrieve the synchronization interval of the sites for the Active Directory Domain.

Setting this method to a defined value implicitly sets the override_synchronization_interval method to "true". Setting the parameter to undefined causes the appliance to use the Grid default and automatically resets the override_synchronization_interval attribute to "false".

Note that when synchronization_interval is set to a defined value and override_dsynchronization_interval is set to "false", the last operation takes precedence. Thus the sequence $object->synchronization_interval(10); $object->override_synchronization_interval("false"); will set override_synchronization_interval to "false", and the sequence $object->override_synchronization_interval("false"); $object->synchronization_interval(10); will result in override_synchronization_interval="true".

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

Valid value is an unsigned integer.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get synchronization_interval value
my $synchronization_interval = $ad_sites->synchronization_interval();
#Modify synchronization_interval value
$ad_sites->synchronization_interval(10);
```

AUTHOR

Infoblox Inc. http://www.infoblox.com/

SEE ALSO

Infoblox::Grid::MSServer

COPYRIGHT

Copyright (c) 2017 Infoblox Inc.

Infoblox::Grid::MSServer::AdUser - the Active Directory User object.

- NAME
- DESCRIPTION
- CONSTRUCTOR
- SESSION METHODS
- MODULE METHODS
  - Infoblox::Grid::MSServer->ad_user( )
- METHODS
  - enable_user_sync( )
  - last_sync_time( )
  - last_sync_detail( )
  - last_sync_status( )
  - login_name( )
  - login_password( )
  - override_enable_user_sync( )
  - override_login( )
  - override_synchronization_interval( )
  - synchronization_interval( )
- AUTHOR
- SEE ALSO
- COPYRIGHT

NAME

Infoblox::Grid::MSServer::AdUser - the Active Directory User object.

DESCRIPTION

The object provides information about synchronization of the Active Directory users.

CONSTRUCTOR

```
my $ad_user = Infoblox::Grid::MSServer::AdUser->new(
   enable_user_sync                => "true" | "false", # Optional / Default is 'true'
   login_name                      => $string,          # Optional / Default is undefined
   login_password                  => $string,          # Optional / Default is undefined
   synchronization_interval        => $uint,            # Optional / Default is undefined
   override_login                  => "true" | "false", # Optional / Default is 'false'
   override_enable_user_sync       => "true" | "false", # Optional / Default is 'false'
   override_synchronization_interval => "true" | "false", # Optional / Default is 'false'
```

SESSION METHODS

The object does not support any session methods. It is retrieved as a part of the Microsoft Server object.

MODULE METHODS

The following functions can be applied to a Active Directory User object.

Infoblox::Grid::MSServer->ad_user( )

Use this method to set or retrieve structure containing Active Directory User synchronization information. See Infoblox::Grid::MSServer->ad_user() for parameters and return values.

Example

```
#Get ad_user value
my $ad_user = $msserver->ad_user();
#Modify ad_user value
my $res = $msserver->ad_user($ad_user);
```

METHODS

this section describes all the methods that you can use to set or retrieve the attribute values of the object.

enable_user_sync( )

Use this method to enable or disable the Active Directory user information synchronization.

Setting this method to a defined value implicitly sets the override_enable_user_sync method to "true". Setting the parameter to undefined causes the appliance to use the grid default and automatically resets the override_enable_user_sync method to "false".

Note that when enable_user_sync is set to a defined value and override_enable_user_sync is set to "false", the last operation takes precedence. Thus the sequence $object->enable_user_sync("true"); $object->override_enable_user_sync("false"); will set override_enable_user_sync to "false", and the sequence $object->override_enable_user_sync("false"); $object->enable_user_sync("true"); will result in override_enable_user_sync="true".

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

Specify "true" to enable Active Directory user information synchronization or "false" to forbid it. Default value is "false".

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get enable_user_sync value
my $enable_user_sync = $ad_user->enable_user_sync();
#Modify enable_user_sync value
$ad_user->enable_user_sync("true");
```

last_sync_time( )

Use this method to retrieve the timestamp of the last Active Directory user information synchronization attempt. This is a read-only attribute.

Parameter

None

Returns

The method returns the attribute value.

Example

```
#Get last_sync_time value
my $last_sync_time = $ad_user->last_sync_time();
```

last_sync_detail( )

Use this method to retrieve a string containing details of the last Active Directory user information synchronization attempt. This is a read-only attribute.

Parameter

None

Returns

The method returns the attribute value.

Example

```
#Get last_sync_detail value
my $last_sync_detail = $ad_user->last_sync_detail();
```

last_sync_status( )

Use this method to retrieve a string containing status of the last Active Directory user information synchronization attempt. This is a read-only attribute.

Parameter

None

Returns

The method returns the attribute value that can be 'OK', 'WARNING' or 'ERROR'.

Example

```
#Get last_sync_status value
my $last_sync_status = $ad_user->last_sync_status();
```

login_name( )

Use this method to set or retrieve the Active Directory user login name.

Setting this method to a defined value implicitly sets the override_login method to "true". Setting the parameter to undefined causes the appliance to use the Grid default and automatically resets the override_login attribute to "false".

Note that when login_name is set to a defined value and override_dlogin_name is set to "false", the last operation takes precedence. Thus the sequence $object->login_name('newuser'); $object->override_login("false"); will set override_login to "false", and the sequence $object->override_login("false"); $object->login_name('newuser'); will result in override_login="true".

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

When setting a value that requires a domain name, prepend the domain information followed by a backslash to the username: "domain\username".

Parameter

The valid value is a string that contains the login name.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get login value
my $login = $ad_user->login_name();
#Modify login value
$ad_user->login_name("newuser");
```

login_password( )

Use this method to set a password for the Active Directory user. This field is used only if the login field for this member is defined. This is a write-only attribute.

This field cannot be retrieved.

Parameter

The valid value is a string that contains the password.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

Example

```
#Set the password
$ad_user->login_password("iG&ojG97Y");
```

override_enable_user_sync( )

The override_enable_user_sync attribute controls whether the enable_user_sync method values of the Active Directory user are used, instead of the Grid default.

The override_enable_user_sync attribute can be specified explicitly. It is also set implicitly when enable_user_sync is set to a defined value.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

Set the parameter to "true" to override the Grid-level setting for enable_user_sync. Set the parameter to "false" to inherit the Grid-level setting for enable_user_sync. The default value is "false".

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get override_enable_user_sync
my $override_enable_user_sync = $ad_user->override_enable_user_sync();
#Modify override_enable_user_sync
$ad_user->override_enable_user_sync("true");
```

override_login( )

The override_login attribute controls whether the login_name method values of the Active Directory user are used, instead of the Grid default.

The override_login attribute can be specified explicitly. It is also set implicitly when login_name is set to a defined value.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

Set the parameter to "true" to override the Grid-level setting for login_name. Set the parameter to "false" to inherit the Grid-level setting for login_name. The default value is "false".

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get override_login
my $override_login = $ad_user->override_login();
#Modify override_login
$ad_user->override_login("true");
```

override_synchronization_interval( )

The override_synchronization_interval attribute controls whether the synchronization_interval method values of the Active Directory user are used, instead of the Grid default.

The override_synchronization_interval attribute can be specified explicitly. It is also set implicitly when synchronization_interval is set to a defined value.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

Set the parameter to "true" to override the Grid-level setting for synchronization_interval. Set the parameter to "false" to inherit the Grid-level setting for synchronization_interval. The default value is "false".

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get override_synchronization_interval
my $override_synchronization_interval = $ad_user->override_synchronization_interval();
#Modify override_synchronization_interval
$ad_user->override_synchronization_interval("true");
```

synchronization_interval( )

Use this method to set or retrieve the synchronization interval of the Active Directory user.

Setting this method to a defined value implicitly sets the override_synchronization_interval method to "true". Setting the parameter to undefined causes the appliance to use the Grid default and automatically resets the override_synchronization_interval attribute to "false".

Note that when synchronization_interval is set to a defined value and override_dsynchronization_interval is set to "false", the last operation takes precedence. Thus the sequence $object->synchronization_interval(10); $object->override_synchronization_interval("false"); will set override_synchronization_interval to "false", and the sequence $object->override_synchronization_interval("false"); $object->synchronization_interval(10); will result in override_synchronization_interval="true".

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

The valid value is an unsigned integer.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get synchronization_interval value
my $synchronization_interval = $ad_user->synchronization_interval();
#Modify synchronization_interval value
$ad_user->synchronization_interval(10);
```

AUTHOR

Infoblox Inc. http://www.infoblox.com/

SEE ALSO

Infoblox::Grid::MSServer

COPYRIGHT

Copyright (c) 2017 Infoblox Inc.

Infoblox::Grid::MSServer::AdSites::Domain - the Active Directory Domain object.

- NAME
- DESCRIPTION
- SESSION METHODS
  - Infoblox::Session->get( )
  - Infoblox::Session->search( )
- METHODS
  - ms_sync_master_name( )
  - ea_definition( )
  - network_view( )
  - netbios( )
  - name( )
  - read_only( )
- AUTHOR
- SEE ALSO
- COPYRIGHT

NAME

Infoblox::Grid::MSServer::AdSites::Domain - the Active Directory Domain object.

DESCRIPTION

The object provides information about the Active Directory Domain.

SESSION METHODS

Infoblox::Session->get( )

Use this method to retrieve all the matching objects from the Infoblox appliance. See Infoblox::Session->get() for parameters and return values.

Key Reference

```
Apply the following attribute to get a specific object:

  name          - Optional. The Active Directory Domain name in DNS form.
  netbios       - Optional. The Active Directory Domain name in NetBIOS form.
  network_view   - Optional. The network view which the Active Directory Domain is associated with.
  ea_definition - Optional. Extensible attribute definition that is linked with this Active
Directory Domain.
```

Example

```
my @retrieved_objs = $session->get(
   name          => 'domain.com',
   netbios       => 'DOMAIN',
   network_view  => 'external',
   ea_definition => 'DOMAIN.external Site',
);
```

Infoblox::Session->search( )

Use this method to search for objects in the Infoblox appliance. See Infoblox::Session->search() for parameters and return values.

Key Reference

```
Apply the following attribute to get a specific object:

  ea_definition - Optional. Extensible attribute definition that is linked with this Active
Directory Domain.
  name          - Optional. The Active Directory Domain name in DNS form.
  netbios       - Optional. The Active Directory Domain name in NetBIOS form.
  network_view   - Optional. The network view which the Active Directory Domain is associated with.
```

Example

```
my @retrieved_objs = $session->search(
   netbios => 'DOMAIN*',
);
```

METHODS

ms_sync_master_name( )

Use this method to retrieve IP address or FQDN of the managing master Microsoft Server. This is a read-only attribute.

Parameter

None

Returns

The method returns the attribute value.

Example

```
#Get ms_sync_master_name value
my $sync_master_name = $domain->ms_sync_master_name();
```

ea_definition( )

Use this method to retrieve extensible attribute definition that is linked with the Active Directory Domain. This is a read-only attribute.

Parameter

None

Returns

The method returns the attribute value.

Example

```
#Get ea_definition value
my $ea_definition = $domain->ea_definition();
```

network_view( )

Use this method to retrieve the network view which the Active Directory Domain is associated with. This is a read-only attribute.

Parameter

None

Returns

The method returns the attribute value.

Example

```
#Get network_view value
my $network_view = $domain->network_view();
```

netbios( )

Use this method to retrieve the Active Directory Domain name in NetBIOS form. This is a read-only attribute.

Parameter

None

Returns

The method returns the attribute value.

Example

```
#Get netbios value
my $netbios = $domain->netbios();
```

name( )

Use this method to retrieve the Active Directory Domain name in DNS form. This is a read-only attribute.

Parameter

None

Returns

The method returns the attribute value.

Example

```
#Get name value
my $name = $domain->name();
```

read_only( )

Use this method to retrieve the flag that indicates if this Active Directory Domain is synchronized in Read-Only or Read/Write mode. This is a read-only attribute.

Parameter

None

Returns

The valid return value is "true" or "false".

Example

```
#Get read_only value
my $read_only = $domain->read_only();
```

AUTHOR

Infoblox Inc. http://www.infoblox.com/

SEE ALSO

Infoblox::Session, Infoblox::Session->get(), Infoblox::Session->search()

COPYRIGHT

Copyright (c) 2017 Infoblox Inc.

Infoblox::IPAM::Statistics - Manages IPAM statistics.

- NAME
- DESCRIPTION
- CONSTRUCTOR
- SESSION METHODS
  - Infoblox::Session->get( )
  - Infoblox::Session->search( )
- METHODS
  - utilization( )
  - conflict_count( )
  - ms_ad_user_data( )
  - unmanaged_count( )
  - utilization_update( )
- SAMPLE CODE
- AUTHOR
- SEE ALSO
- COPYRIGHT

NAME

Infoblox::IPAM::Statistics - Manages IPAM statistics.

DESCRIPTION

The IPAM statistics object is used to view the IPAM statistics of the network or network container in an Infoblox appliance.

CONSTRUCTOR

The Infoblox::IPAM::Statistics object does not require manual construction.

SESSION METHODS

This section describes all the methods in an Infoblox::Session that can be applied to a IPAM statistics object.

Infoblox::Session->get( )

Use this method to retrieve the existing objects from an Infoblox appliance. See Infoblox::Session->get() for parameters and return values.

Key Reference

```
 Apply the following attributes to get the IPAM statistics:

 network      - Required. An IPv4 address in dotted decimal notation followed by the network prefix
(e.g 1.0.0.0/8).
 network_view - Optional. The name of the network view in which the network object is located. By
default, all network views are searched.
```

Example

```
my $ipam_stats = $session->get(
         object  => "Infoblox::IPAM::Statistics",
         network => "1.0.0.0/8",
         network_view => "my_network_view");
```

Infoblox::Session->search( )

Use this method to search for an IPAM statistics object from an Infoblox appliance. See Infoblox::Session->search() for parameters and return values.

Key Reference

```
 Apply the following attributes to get the IPAM statistics:

 network      - Required. An IPv4 address in dotted decimal notation followed by the network prefix
(e.g 1.0.0.0/8) (regular expression).
 network_view - Optional. The name of the network view in which the network object is located. By
default, all network views are searched.
```

Example

```
my $ipam_stats = $session->search(
         object  => "Infoblox::IPAM::Statistics",
         network => '1\.0\.0\..*',
         network_view => "my_network_view");
```

METHODS

This section describes all the methods that can be used to retrieve the attribute values of a IPAM statistics object.

utilization( )

Use this method to retrieve the network utilization.

Parameter

none

Returns

The method returns the network utilization in percentage.

Example

```
#Get utilization
my $utilization = $ipam_stats->utilization();
```

conflict_count( )

Use this method to retrieve the number of conflicts discovered via network discovery. This method is only valid for a Network object.

Parameter

none

Returns

The method returns the number of discovered conflicts. For a NetworkContainer object, the return value is undefined.

Example

```
#Get conflict_count
my $conflict_count = $ipam_stats->conflict_count();
```

ms_ad_user_data( )

Use this method to retrieve Microsoft Active Directory users related information. This is a read-only attribute.

Parameter

None

Returns

The valid return value is an Infoblox::Grid::MSServer::AdUser::Data object.

Example

```
#Get ms_ad_user_data
my $ms_ad_user_data = $ipam_stats->ms_ad_user_data();
```

unmanaged_count( )

Use this method to retrieve the number of unmanaged IP addresses as discovered by network discovery. This method is only valid for a Network object.

Parameter

none

Returns

The method returns the number of unmanaged IP addresses. For a NetworkContainer object, the return value is undefined.

Example

```
#Get unmanaged_count
my $unmanaged_count = $ipam_stats->unmanaged_count();
```

utilization_update( )

Use this method to retrieve the time that the utilization statistics were updated last. This method is only valid for a Network object.

Parameter

none

Returns

The method returns a timestamp value in the ISO 8601 extended format for date and time (e.g. 2008-10-21T10:56:33Z or 2008-10-21T10:56:33.001Z). For a NetworkContainer object, the return value is undefined.

Example

```
#Get utilization_update
my $utilization_update = $ipam_stats->utilization_update();
```

SAMPLE CODE

The following sample code demonstrates the get and search session methods on an IPAM statistics object. Also, this sample code also includes error handling for the operations.

```
#PROGRAM STARTS: Include all the modules that will be used
use strict;
use Infoblox;
```

#Preparation prior to getting IPAM statistics

```
#Create a session to the Infoblox appliance
my $session = Infoblox::Session->new(
    master   => "192.168.1.2",
    username => "admin",
    password => "infoblox"
);
unless ($session) {
   die("Construct session failed: ",
       Infoblox::status_code() . ":" . Infoblox::status_detail());
}
print "Session created successfully\n";

my $member = Infoblox::DHCP::Member->new(
    name     => "infoblox.localdomain",
    ipv4addr => "192.168.1.2",
);

unless($member) {
     die("Construct member failed: ",
           Infoblox::status_code() . ":" . Infoblox::status_detail());
}
print "Member object created successfully\n";

#Create a Network View object
my $network_view = Infoblox::DHCP::View->new(
                   name => "my_network_view",
);
unless($network_view) {
            die("Construct Network View object failed: ",
                  Infoblox::status_code() . ":" . Infoblox::status_detail());
}
print "Network View object created successfully\n";

#Add the Network View object into the Infoblox appliance through a session
$session->add($network_view)
     or die("Add Network View object failed: ",
           $session->status_code() . ":" . $session->status_detail());
print "Network View added successfully\n";

#Create the Network object with this member
my $network = Infoblox::DHCP::Network->new(
                   network => "1.0.0.0/255.0.0.0",
                   members => [ $member ],
                   network_view => $network_view,
);
unless($network) {
```

```
            die("Construct Network object failed: ",
                  Infoblox::status_code() . ":" . Infoblox::status_detail());
}
print "Network object created successfully\n";

#Add the Network object into the Infoblox appliance through a session
$session->add($network)
     or die("Add Network object failed: ",
           $session->status_code() . ":" . $session->status_detail());
print "Network added successfully\n";

#Create the DHCP Range object with this member
my $dhcp_range = Infoblox::DHCP::Range->new(
    network    => "1.0.0.0/8",
    start_addr => "1.0.0.1",
    end_addr   => "1.0.0.10",
    member     => $member,
    network_view => $network_view,
);
unless($dhcp_range) {
     die("Construct DHCP Range failed: ",
           Infoblox::status_code() . ":" . Infoblox::status_detail());
}
print "DHCP range object created successfully\n";

$session->add($dhcp_range)
     or die("Add DHCP range object failed: ",
           $session->status_code() . ":" . $session->status_detail());
print "DHCP range added successfully\n";

#Create the Fixed Address object
my $fixed_address = Infoblox::DHCP::FixedAddr->new(
                                              "network" => "1.0.0.0/8",
                                              "ipv4addr" => "1.0.0.2",
                                              "mac" => "22:22:22:22:22:22",
                                              "network_view" => $network_view,
                                              );
unless($fixed_address) {
     die("Construct DHCP Fixed address failed: ",
           Infoblox::status_code() . ":" . Infoblox::status_detail());
}
print "DHCP Fixed address object created successfully\n";

$session->add($fixed_address)
     or die("Add DHCP Fixed address object failed: ",
           $session->status_code() . ":" . $session->status_detail());
print "DHCP Fixed address added successfully\n";
```

#Get the IPAM statistics

```
 my @result_array = $session->get(
          object  => "Infoblox::IPAM::Statistics",
          network => "1.0.0.0/8",
          network_view => "my_network_view");

 my $ipam_stats = $result_array[0];
 unless ($ipam_stats) {
      die("Get IPAM statistics failed: ",
            $session->status_code() . ":" . $session->status_detail());
 }
 print "Get IPAM Statistics successful\n";

 print "Utilization for '1.0.0.0/8': ", $ipam_stats->utilization(), "\n";
```

#Search for the IPAM statistics

```
 my @result_array = $session->search(
          object  => "Infoblox::IPAM::Statistics",
          network => '1\.0\.0\..*',
          network_view => "my_network_view");

 my $ipam_stats = $result_array[0];
 unless ($ipam_stats) {
      die("Get IPAM statistics failed: ",
            $session->status_code() . ":" . $session->status_detail());
 }
```

```
print "Get IPAM Statistics successful\n";

print "Utilization for '1.0.0.0/8': ", $ipam_stats->utilization(), "\n";
```

#Clean up

```
# Remove the IPAM Statistics object that was just retrieved
$session->remove($ipam_stats);

# Retrieve and remove the Network View object
my $network_view = $session->get(
        object => "Infoblox::DHCP::View",
        name => "my_network_view",
);
unless ($network_view) {
     die("Get Network View failed: ",
           $session->status_code() . ":" . $session->status_detail());
}
$session->remove($network_view);

####PROGRAM ENDS####
```

AUTHOR

Infoblox Inc. http://www.infoblox.com/

SEE ALSO

Infoblox::Session, Infoblox::DHCP::Network,Infoblox::DHCP::View,Infoblox::DHCP::FixedAddr,Infoblox::DHCP::Range,Infoblox::Session->add(),Infoblox::Session->get(), Infoblox::Grid::MSServer::AdUser::Data

---

COPYRIGHT

Copyright (c) 2017 Infoblox Inc.

Infoblox::Grid::CloudAPI::VMAddress - Infoblox::Grid::CloudAPI::VMAddress object.

- NAME
- DESCRIPTION
- CONSTRUCTOR
- SESSION METHODS
  - Infoblox::Session->get( )
  - Infoblox::Session->search( )
  - address( )
  - associated_objects( )
  - dns_names( )
  - is_ipv4( )
  - mac_address( )
  - ms_ad_user_data( )
  - network( )
  - network_view( )
  - port_id( )
  - tenant( )
  - tenant_name( )
  - vm_id( )
  - vm_name( )
- AUTHOR
- SEE ALSO
- COPYRIGHT

NAME

Infoblox::Grid::CloudAPI::VMAddress - Infoblox::Grid::CloudAPI::VMAddress object.

DESCRIPTION

Class that represents Tenant.

CONSTRUCTOR

SESSION METHODS

This section describes all the methods in an Infoblox::Session module that you can apply to a Grid member object.

Infoblox::Session->get( )

Use this method to retrieve all the matching objects from the Infoblox appliance. See Infoblox::Session->get() for parameters and return values.

Key Reference

```
Apply the following attribute to get a specific object:

  address            - Optional. IP address of the interface.
  mac_address        - Optional. MAC address of the interface.
  tenant_name        - Optional. Name of the tenant associated with the VM.
  vm_id              - Optional. UUID of the VMirtual Machine.
  vm_name            - Optional. Name of the Virtual Machine.
```

Example

```
$session->get(object => "Infoblox::Grid::CloudAPI::VMAddress", vm_id => $vm_id);
```

Infoblox::Session->search( )

Use this method to search for objects in the Infoblox appliance. See Infoblox::Session->search() for parameters and return values.

Key Reference

```
Apply the following attribute to get a specific object:

  address            - Optional. IP address of the interface.
  mac_address        - Optional. MAC address of the interface.
  tenant_name        - Optional. Name of the tenant associated with the VM.
  vm_id              - Optional. UUID of the VMirtual Machine.
  vm_name            - Optional. Name of the Virtual Machine.
```

Example

```
$session->search(object => "Infoblox::Grid::CloudAPI::VMAddress", vm_name => "vm name");
```

address( )

Use this method to retrieve the IP address of the interface. This is a read-only attribute.

Parameter

None

Returns

The method returns attribute value.

Example

```
# Get address
$address = $vmaddress->address();
```

associated_objects( )

Use this method to retrieve the list of references to the object (Host, Fixed Address, RR, ...) that define this IP. This is a read-only attribute.

Parameter

None

Returns

The method returns attribute value.

Example

```
# Get associated_objects
@associated_objects = $vmaddress->associated_objects();
```

dns_names( )

Use this method to retrieve the list of all FQDN associated with the IP address. This is a read-only attribute.

Parameter

None

Returns

The method returns attribute value.

Example

```
# Get dns_names
@dns_names = $vmaddress->dns_names();
```

is_ipv4( )

Use this method to retrieve the address type whether the address is IPv4 or IPv6. This is a read-only attribute.

Parameter

None

Returns

The method returns attribute value.

Example

```
# Get is_ipv4
$is_ipv4 = $vmaddress->is_ipv4();
```

mac_address( )

Use this method to retrieve the MAC address of the interface. This is a read-only attribute.

Parameter

None

Returns

The method returns attribute value.

Example

```
# Get mac_address
$mac_address = $vmaddress->mac_address();
```

ms_ad_user_data( )

Use this method to retrieve Microsoft Active Directory users related information. This is a read-only attribute.

Parameter

None

Returns

The valid return value is an Infoblox::Grid::MSServer::AdUser::Data object.

Example

```
#Get ms_ad_user_data
my $ms_ad_user_data = $vmaddress->ms_ad_user_data();
```

network( )

Use this method to retrieve the network address / CIDR that contains the IP. This is a read-only attribute.

Parameter

None

Returns

The method returns attribute value.

Example

```
# Get network
$network = $vmaddress->network();
```

network_view( )

Use this method to retrieve the network view name that contains the IP. This is a read-only attribute.

Parameter

None

Returns

The method returns attribute value.

Example

```
# Get network_view
$network_view = $vmaddress->network_view();
```

port_id( )

Use this method to retrieve the port identifier of the interface. This is a read-only attribute.

Parameter

None

Returns

The method returns attribute value.

Example

```
# Get port_id
$port_id = $vmaddress->port_id();
```

tenant( )

Use this method to retrieve a Infoblox::Grid::CloudAPI::Tenant object. This is a read-only attribute.

Parameter

None

Returns

The method returns attribute value.

Example

```
# Get tenant
$tenant = $vmaddress->tenant();
```

tenant_name( )

Use this method to retrieve the tenant name. This is a read-only attribute.

Parameter

None

Returns

The method returns attribute value.

Example

```
# Get tenant_name
$tenant_name = $vmaddress->tenant_name();
```

vm_id( )

Use this method to retrieve the UUID of the VMirtual Machine. This is a read-only attribute.

Parameter

None

Returns

The method returns attribute value.

Example

```
# Get vm_id
$vm_id = $vmaddress->vm_id();
```

vm_name( )

Use this method to retrieve the name of the Virtual Machine. This is a read-only attribute.

Parameter

None

Returns

The method returns attribute value.

Example

```
# Get vm_name
$vm_name = $vmaddress->vm_name();
```

AUTHOR

Infoblox Inc. http://www.infoblox.com/

SEE ALSO

Infoblox::Session->get(), Infoblox::Session->search(), Infoblox::Grid::CloudAPI::Tenant, Infoblox::Grid::MSServer::AdUser::Data

COPYRIGHT

Copyright (c) 2017 Infoblox Inc.

Infoblox::Grid::ParentalControl::SubscriberSite - a Parental Control subscriber site object

- NAME
- DESCRIPTION
- CONSTRUCTOR
- SESSION METHODS
  - Infoblox::Session->add( )
  - Infoblox::Session->get( )
  - Infoblox::Session->modify( )
  - Infoblox::Session->remove( )
  - Infoblox::Session->search( )
- METHODS
  - comment( )
  - blocking_ipv4_vip1( )
  - blocking_ipv4_vip2( )
  - blocking_ipv6_vip1( )
  - blocking_ipv6_vip2( )
  - extattrs( )
  - extensible_attributes( )
  - maximum_subscribers( )
  - members( )
  - msps( )
  - name( )
  - nas_gateways( )
  - nas_port( )
- AUTHOR
- SEE ALSO
- COPYRIGHT

NAME

Infoblox::Grid::ParentalControl::SubscriberSite - a Parental Control subscriber site object

---

DESCRIPTION

CONSTRUCTOR

```
 my $subscriber_site = Infoblox::Grid::ParentalControl::SubscriberSite->new(
    name                  => $string,                                                  #
Required
    maximum_subscribers   => $uint,                                                    #
Required
    nas_port              => $uint,                                                    #
Required
    comment               => $string,                                                  #
Optional / Default is undefined
    members               => [$site_member1, $site_member2, ...],                      #
Optional / Default is empty list
    msps                  => [$msp1, $msp2, ...],                                      #
Optional / Default is empty list
    nas_gateways          => [$gateway1, $gateway2, ...],                              #
Optional / Default is empty list
    blocking_ipv4_vip1    => $ipv4addr,                                                #
Optional / Default is undefined
    blocking_ipv6_vip1    => $ipv6addr,                                                #
Optional / Default is undefined
    blocking_ipv4_vip2    => $ipv4addr,                                                #
Optional / Default is undefined
    blocking_ipv6_vip2    => $ipv6addr,                                                #
Optional / Default is undefined
    extattrs              => { $string => $extattr, ... },                             #
Optional / Default is undefined
    extensible_attributes => {$string => $string | $num, $string => [ $string | $num, ... ], ... } #
Optional / Default is undefined
 );
```

You cannot set both extattrs and extensible_attributes attributes at the same time.

SESSION METHODS

The following functions can be applied to a Infoblox::Grid::ParentalControl::SubscriberSite object.

Infoblox::Session->add( )

Use this method to add an object to the Infoblox appliance. See Infoblox::Session->add() for parameters and return values.

Example

```
#Construct an object
my $object = Infoblox::Grid::ParentalControl::SubscriberSite->new(
   name                => 'site1',
   maximum_subscribers => '20000',
   nas_port            => '1919',
   comment             => 'site1 comment',
);

#Submit for addition
my $response = $session->add($object);
```

Infoblox::Session->get( )

Use this method to retrieve all the matching objects from the Infoblox appliance. See Infoblox::Session->get() for parameters and return values.

Key References

```
 Apply the following attributes to get a specific Infoblox::Grid::ParentalControl::SubscriberSite
object.
  name                - Optional. The Parental Control subscriber site name.
  comment             - Optional. The Parental Control subscriber site comment.
  extattrs            - Optional. A hash reference containing Infoblox::Grid::Extattr object.
  extensible_attributes - Optional. A hash reference containing extensible attributes.
  blocking_ipv4_vip1    - Optional. The Parental Control primary VIP IPv4 address of the blocking
server.
  blocking_ipv6_vip1    - Optional. The Parental Control primary VIP IPv6 address of the blocking
server.
  blocking_ipv4_vip2    - Optional. The Parental Control secondary VIP IPv4 address of the blocking
server.
  blocking_ipv6_vip2    - Optional. The Parental Control secondary VIP IPv6 address of the blocking
server.
```

Example

```
# search for the object which name is 'test.com'
my @retrieved_objs = $session->get(
    'object' => 'Infoblox::Grid::ParentalControl::SubscriberSite',
    'name'   => 'test.com',
);
```

Infoblox::Session->modify( )

Use this method to modify an object in the Infoblox appliance. See Infoblox::Session->modify() for parameters and return values.

Example

```
# modify object object
$object->comment('comment change');

# Submit modification
my $response = $session->modify($object);
```

Infoblox::Session->remove( )

Use this method to remove an object from the Infoblox appliance. See Infoblox::Session->remove() for parameters and return values.

To remove a specific object, first use `get()` or `search()` to retrieve the specific object, and then submit this object for removal.

Example

```
#Get the objects with the same name
my @retrieved_objs = $session->get(
    object => 'Infoblox::Grid::ParentalControl::SubscriberSite',
    name   => 'site1');

#Find the desired object from the retrieved list.
my $desired_object = $retrieved_objs[0];

# Submit for removal
my $response = $session->remove($desired_object);
```

Infoblox::Session->search( )

Use this method to search for objects in the Infoblox appliance. See Infoblox::Session->search() for parameters and return values.

Key References

```
 Apply the following attributes to search for a specific
Infoblox::Grid::ParentalControl::SubscriberSite object.
  name                  - Optional. The Parental Control subscriber site name (regexp).
  comment               - Optional. The Parental Control subscriber site comment (regexp).
  extattrs              - Optional. A hash reference containing Infoblox::Grid::Extattr object.
  extensible_attributes - Optional. A hash reference containing extensible attributes.
  blocking_ipv4_vip1    - Optional. The Parental Control primary VIP IPv4 address of the blocking
server (regexp).
  blocking_ipv6_vip1    - Optional. The Parental Control primary VIP IPv6 address of the blocking
server (regexp).
  blocking_ipv4_vip2    - Optional. The Parental Control secondary VIP IPv4 address of the blocking
server (regexp).
  blocking_ipv6_vip2    - Optional. The Parental Control secondary VIP IPv6 address of the blocking
server (regexp).
```

Example

```
 # search for objects which name contains test and primary IPv6 address of the blocking server
contains '2:'
 my @retrieved_objs = $session->search(
     'object'             => 'Infoblox::Grid::ParentalControl::SubscriberSite',
     'name'               => 'test.*',
     'blocking_ipv6_vip1' => '2:.*',
 );

 # search for objects defining 'Santa Clara' for 'Site' extensible attribute
 my @retrieved_objs = $session->search(
     object               => 'Infoblox::Grid::ParentalControl::SubscriberSite',
     extensible_attributes => {'Site' => 'Santa Clara'},
 );
```

METHODS

This section describes all the methods that can be used to configure and retrieve the attribute values of a Parental Control subscriber site object.

comment( )

Use this method to set or retrieve Parental Control subscriber site comment.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

The valid value is desired comment in string format.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
 #Get comment
 my $value = $object->comment();

 #Modify comment
 $object->comment('new comment');
```

blocking_ipv4_vip1( )

Use this method to set or retrieve the primary IPv4 address of the blocking server.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

The valid value is a desired IPv4 address in a string format.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get blocking_ipv4_vip1 value
my $blocking_ipv4_vip1 = $object->blocking_ipv4_vip1();

#Modify blocking_ipv4_vip1 value
$object->blocking_ipv4_vip1('10.0.10.1');
```

blocking_ipv4_vip2( )

Use this method to set or retrieve the secondary IPv4 address of the blocking server.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

The valid value is a desired IPv4 address in a string format.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get blocking_ipv4_vip2 value
my $blocking_ipv4_vip2 = $object->blocking_ipv4_vip2();

#Modify blocking_ipv4_vip2 value
$object->blocking_ipv4_vip2('10.0.10.1');
```

blocking_ipv6_vip1( )

Use this method to set or retrieve the primary IPv6 address of the blocking server.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

The valid value is a desired IPv6 address in a string format.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get blocking_ipv6_vip1 value
my $blocking_ipv6_vip1 = $object->blocking_ipv6_vip1();

#Modify blocking_ipv6_vip1 value
$object->blocking_ipv6_vip1('2001:db8::1');
```

blocking_ipv6_vip2( )

Use this method to set or retrieve the secondary IPv6 address of the blocking server.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

The valid value is a desired IPv6 address in a string format.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get blocking_ipv6_vip2 value
my $blocking_ipv6_vip2 = $object->blocking_ipv6_vip2();

#Modify blocking_ipv6_vip2 value
$object->blocking_ipv6_vip2('2002:db8::1');
```

extattrs( )

Use this method to set or retrieve the extensible attributes associated with a Parental Control subscriber site object.

Parameter

Valid value is a hash reference that contains the names of extensible attributes and their associated values (Infoblox::Grid::Extattr objects).

Returns

If you have specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get extattrs value
my $extattrs = $object->extattrs();

#Modify extattrs value
$object->extattrs({'Site' => $extattr1, 'Administrator' => $extattr2});
```

extensible_attributes( )

Use this method to set or retrieve the extensible attributes associated with a Parental Control subscriber site object.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

For valid values for extensible attributes, see Infoblox::Grid::ExtensibleAttributeDef/Extensible Attribute Values.

Returns

If you have specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get extensible attributes value
my $extensible_attributes = $object->extensible_attributes();
#Modify extensible attributes
$object->extensible_attributes({'Site' => 'Santa Clara', 'Administrator' => ['Peter', 'Tom']});
```

maximum_subscribers( )

Use this method to set or retrieve the port number to reach the collector.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

The valid value is an unsigned integer between 10 000 and 10 000 000.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get maximum_subscribers value
my $maximum_subscribers = $object->maximum_subscribers();

#Modify maximum_subscribers value
$object->maximum_subscribers('20000');
```

members( )

Use this method to set or retrieve the Grid members associatied with subscriber site.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

The valid value is an array of Infoblox::Grid::ParentalControl::SiteMember.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Construct first member
my $m1 = Infoblox::Grid::ParentalControl::SiteMember->new(
    name => 'infoblox.localdomain',
    type => 'NONE',
);

#Construct second member
my $m2 = Infoblox::Grid::ParentalControl::SiteMember->new(
    name => 'imc-collector',
    type => 'COLLECTOR',
);

#Get members value
my $members = $object->members();

#Modify members value
$object->members([$m1, $m2]);
```

msps( )

Use this method to set or retrieve the list of Parental Control managed service providers.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

The valid value is an array of Infoblox::Grid::ParentalControl::MSP objects.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Exmaple

```
#Get msps value
my $value = $object->msps();

#Construct Parental Control MSP
my $msp = Infoblox::Grid::ParentalControl::MSP->new(address => '10.0.10.10');

#Modify msps value
$object->msps([$msp]);
```

name( )

Use this method to set or retrieve a name of the Parental Control subscriber site.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

The valid value is a desired name in string format.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Exmaple

```
#Get name value
my $value = $object->name();
```

```
#Modify name value
$object->name('test2');
```

nas_gateways( )

Use this method to set or retrieve the NAS gateways associated with subscriber site.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

The valid value is an array of Infoblox::Grid::ParentalControl::NAS::Gateway.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get nas_gateways value
my $nas_gateways = $object->nas_gateways();

#Construct first gateway
my $gw1 = Infoblox::Grid::ParentalControl::NAS::Gateway->new(
    name          => 'gateway-1',
    address       => '10.1.1.1',
    shared_secret => 'secret',
);

#Construct first gateway
my $gw2 = Infoblox::Grid::ParentalControl::NAS::Gateway->new(
    name          => 'gateway-2',
    address       => '2001:db8::1',
    shared_secret => 'secret',
    send_ack      => 'true',
);

#Modify nas_gateways value
$object->nas_gateways([$gw1, $gw2]);
```

nas_port( )

Use this method to set or retrieve the port number to reach the collector.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

The valid value is an unsigned integer between 1 and 65535.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get nas_port value
my $nas_port = $object->nas_port();

#Modify nas_port value
$object->nas_port('1919');
```

AUTHOR

Infoblox Inc. http://www.infoblox.com/

SEE ALSO

Infoblox::Session, Infoblox::Session->add(), Infoblox::Session->get(), Infoblox::Session->remove(), Infoblox::Session->search(), Infoblox::Session->modify(), Infoblox::Grid::ParentalControl::SiteMember, Infoblox::Grid::ParentalControl::MSP, Infoblox::Grid::ParentalControl::NAS::Gateway

---

COPYRIGHT

Copyright (c) 2017 Infoblox Inc.

Infoblox::Grid::ParentalControl::SiteMember - A Parental Control site member object.

- NAME
- DESCRIPTION
- CONSTRUCTOR
- MODULE METHODS
  - Infoblox::Grid::ParentalControl::SubscriberSite->members( )
- METHODS
  - name( )
  - type( )
- AUTHOR
- SEE ALSO
- COPYRIGHT

NAME

Infoblox::Grid::ParentalControl::SiteMember - A Parental Control site member object.

DESCRIPTION

The Parental Control site member object associate particular Grid member with it's role in process of collecting and consuming the authentication information and the Parental Control site the feature is deployed on.

CONSTRUCTOR

```
my $site_member = Infoblox::Grid::ParentalControl::SiteMember->new(
    name => $string,                                  #Required
    type => 'COLLECTOR' | 'NONE' | 'BOTH' | 'CLIENT', #Optional / Default is 'NONE'
);
```

MODULE METHODS

The following functions are available to be applied to a Parental Control site member object.

Infoblox::Grid::ParentalControl::SubscriberSite->members( )

Use this method to set or retrieve the Grid members associatied with subscriber site.

See Infoblox::Grid::ParentalControl::SubscriberSite->members() for parameters and return values.

Example

```
#Get members value
my $members = $object->members();

#Construct first member
my $m1 = Infoblox::Grid::ParentalControl::SiteMember->new(
    name => 'infoblox.localdomain',
    type => 'NONE',
);

#Construct second member
my $m2 = Infoblox::Grid::ParentalControl::SiteMember->new(
    name => 'imc-collector',
    type => 'COLLECTOR',
);

#Modify members value
$object->members([$m1, $m2]);
```

METHODS

This section describes all the methods that can be used to configure and retrieve the attribute values of a Parental Control site member object.

name( )

Use this method to set or retrieve the name of the member.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

The valid value is a desired name in a string format.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get name value
my $name = $object->name();

#Modify name value
$object->name('my_collector');
```

type( )

Use this method to set or retrieve the type of the member.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

The valid values are 'BOTH', 'COLLECTOR', 'CLIENT' and 'NONE'. The default value is 'NONE'.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get type value
my $type = $object->type();

#Modify type value
$object->type('CLIENT');
```

AUTHOR

Infoblox Inc. http://www.infoblox.com/

SEE ALSO

Infoblox::Grid::ParentalControl::SubscriberSite, Infoblox::Grid::ParentalControl::SubscriberSite->members(),

---

COPYRIGHT

Copyright (c) 2017 Infoblox Inc.

Infoblox::Grid::ParentalControl::MSP - A Parental Control managed service provider object.

- NAME
- DESCRIPTION
- CONSTRUCTOR
- MODULE METHODS
  - Infoblox::Grid::ParentalControl::SubscriberSite->msps( )
- METHODS
  - address( )
- AUTHOR
- SEE ALSO
- COPYRIGHT

NAME

Infoblox::Grid::ParentalControl::MSP - A Parental Control managed service provider object.

DESCRIPTION

The Parental Control managed service provider object is used to configure list of service providers that are managed by the Parental Control subscriber site.

CONSTRUCTOR

```
my $nas_gateway = Infoblox::Grid::ParentalControl::MSP->new(
    address => $ipv4addr, # Required
);
```

MODULE METHODS

The following functions are available to be applied to a Parental Control managed service provider object.

Infoblox::Grid::ParentalControl::SubscriberSite->msps( )

Use this method to set or retrieve the list of Parental Control managed service providers.

See Infoblox::Grid::ParentalControl::SubscriberSite->msps() for parameters and return values.

Example

```
#Get msps value
my $value = $object->msps();

#Construct Parental Control MSP
my $msp = Infoblox::Grid::ParentalControl::MSP->new(
    address => '10.0.10.10',
);

#Modify msps value
$object->msps([$msp]);
```

METHODS

This section describes all the methods that can be used to configure and retrieve the attribute values of a Parental Control managed service provider object.

address( )

Use this method to set or retrieve the managed service provider IP address.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

The valid value is desired IPv4 address in a string format.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get address value
my $address = $object->address();

#Modify address value
$object->address('10.0.0.10');
```

AUTHOR

Infoblox Inc. http://www.infoblox.com/

SEE ALSO

Infoblox::Grid::ParentalControl::SubscriberSite, Infoblox::Grid::ParentalControl::SubscriberSite->msps(),

---

COPYRIGHT

Copyright (c) 2017 Infoblox Inc.

Infoblox::Grid::ParentalControl::NAS::Gateway - A Parental Control network accounting server gateway object.

- NAME
- DESCRIPTION
- CONSTRUCTOR
- MODULE METHODS
  - Infoblox::Grid::ParentalControl::SubscriberSite->nas_gateways( )
- METHODS
  - address( )
  - comment( )
  - message_rate( )
  - name( )
  - send_ack( )
  - shared_secret( )
- AUTHOR
- SEE ALSO
- COPYRIGHT

NAME

Infoblox::Grid::ParentalControl::NAS::Gateway - A Parental Control network accounting server gateway object.

DESCRIPTION

The Parental Control network accounting server (NAS) gateway object provides configuration for transmission of authentication protocol accounting infromation to the NIOS accounting collector.

CONSTRUCTOR

```
my $nas_gateway = Infoblox::Grid::ParentalControl::NAS::Gateway->new(
    name          => $string,                    # Required
    address       => $ipv4addr,                  # Required
    shared_secret => $string,                    # Optional
    send_ack      => 'true' | 'false',           # Optional / Default is 'false'
    comment       => $string,                    # Optional / Default is undefined
);
```

MODULE METHODS

The following functions are available to be applied to a Parental Control network accounting server gateway object.

Infoblox::Grid::ParentalControl::SubscriberSite->nas_gateways( )

Use this method to set or retrieve the NAS gateways associated with subscriber site.

See Infoblox::Grid::ParentalControl::SubscriberSite->nas_gateways() for parameters and return values.

Example

```
#Get nas_gateways value
my $nas_gateways = $object->nas_gateways();

#Construct first gateway
my $gw1 = Infoblox::Grid::ParentalControl::NAS::Gateway->new(
    name          => 'gateway-1',
    address       => '10.1.1.1',
    shared_secret => 'secret',
);

#Construct first gateway
my $gw2 = Infoblox::Grid::ParentalControl::NAS::Gateway->new(
    name          => 'gateway-2',
    address       => '2001:db8::1',
    shared_secret => 'secret',
    send_ack      => 'true',
);

#Modify nas_gateways value
$object->nas_gateways([$gateway1, $gateway2]);
```

METHODS

This section describes all the methods that can be used to configure and retrieve the attribute values of a Parental Control network accounting server gateway object.

address( )

Use this method to set or retrieve the NAS gateway IP address.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

The valid value is desired IPv4 address in a string format.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get address value
my $address = $object->address();

#Modify address value
$object->address('10.0.0.10');
```

comment( )

Use this method to set or retrieve the NAS gateway comment.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

The valid value is a desired comment in a string format.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get comment value
my $comment = $object->comment();

#Modify comment value
$object->comment('nas gateway comment');
```

message_rate( )

Use this method to retrieve the vendor specific accounting messages rate. This is a read-only attribute.

Omit the parameter to retrieve the attribute value.

Parameter

None

Returns

The method returns the attribute value.

Example

```
#Get message_rate value
my $message_rate = $object->message_rate();
```

name( )

Use this method to set or retrieve the NAS gateway object name.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

The valid value is a desired name in a string format.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get name value
my $name = $object->name();

#Modify name value
$object->name('accounting_log_server');
```

send_ack( )

Use this method to set or retrieve the flag that indicates whether the acknowledgement of the accounting packets is enabled or disabled.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

Specify 'true' to enable acknowledgement of the accounting packets or 'false' to disable it. The default value is 'false'.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get send_ack value
my $send_ack = $object->send_ack();

#Modify send_ack value
$object->send_ack('true');
```

shared_secret( )

Use this method to set or retrieve the MD5 phrase for the accounting data. This is a write-only attribute.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

The valid value is a desired MD5 phrase in a string format.

Returns

The method returns true when the modification succeeds, and returns false when the operation fails.

Example

```
#Modify shared_secret value
$object->shared_secret('secret');
```

AUTHOR

Infoblox Inc. http://www.infoblox.com/

SEE ALSO

Infoblox::Grid::ParentalControl::SubscriberSite, Infoblox::Grid::ParentalControl::SubscriberSite->nas_gateways(),

---

COPYRIGHT

Copyright (c) 2017 Infoblox Inc.

Infoblox::Grid::RIR - Regional Internet Registry object.

- NAME
- DESCRIPTION
- CONSTRUCTOR
- SESSION METHODS
  - Infoblox::Session->get( )
  - Infoblox::Session->modify( )
  - Infoblox::Session->search( )
- METHODS
  - communication_mode( )
  - name( )
  - email( )
  - url( )
  - override_email( )
  - override_url( )
- AUTHOR
- SEE ALSO
- COPYRIGHT

NAME

Infoblox::Grid::RIR - Regional Internet Registry (RIR) object.

DESCRIPTION

Regional Internet Registry (RIR) object.

CONSTRUCTOR

```
my $rir = Infoblox::Grid::RIR->new(
   communication_mode    => 'API' | 'EMAIL' | 'NONE', # Required
   name                  => 'RIPE',                   # Optional, Default is 'RIPE'
   email                 => $string,                  # Optional
   override_email        => 'true' | 'false',         # Optional, Default is 'false'
   url                   => $string,                  # Optional
   override_url          => 'true' | 'false',         # Optional, Default is 'false'
);
```

SESSION METHODS

This section describes all the methods in an Infoblox::Session module that you can apply to a RIR object.

Support for RIR/SWIP must be enabled at the Grid level. See Infoblox::Grid->enable_rir_swip() method.

Infoblox::Session->get( )

Use this method to retrieve all the matching objects from the Infoblox appliance. See Infoblox::Session->get() for parameters and return values.

Key References

```
Apply the following attributes to get a specific RIR object:

 name    - Optional. The RIR name.
```

Example

```
my @retrieved_objs = $session->get(
   object => 'Infoblox::Grid::RIR',
   name   => 'RIPE',
);
```

Infoblox::Session->modify( )

Use this method to modify an object in the Infoblox appliance. See Infoblox::Session->modify() for parameters and return values.

Example

```
# Use this method to modify email of a RIR object.
```

```
$rir->email('support@infoblox.com');
# Submit modification
my $response = $session->modify( $rir );
```

Infoblox::Session->search( )

Use this method to search for RIR objects in the Infoblox appliance. See Infoblox::Session->search() for parameters and return values.

Key References

```
Apply the following attributes to search a specific RIR object:

 name         - Optional. The RIR name.
```

Example

```
my @retrieved_objs = $session->search(
   object => 'Infoblox::Grid::RIR',
   name   => 'RIPE',
);
```

METHODS

This section describes all the methods that you can use to configure and retrieve the attribute values of a RIR object.

communication_mode( )

Use this method to set or retrieve a communication mode for RIR.

Parameter

Valid values are 'API', 'EMAIL' and 'NONE'. The default value is 'API'.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get attribute value
my $value = $rir->communication_mode();
#Modify attribute value
$rir->communication_mode('EMAIL');
```

name( )

Use this method to set or retrieve a name of RIR.

Parameter

Valid value is 'RIPE'. The default value is 'RIPE'.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get attribute value
my $value = $rir->name();
#Modify attribute value
$rir->name('RIPE');
```

email( )

Use this method to set or retrieve an email address for RIR.

Parameter

Email address in string format.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get attribute value
my $value = $rir->email();
#Modify attribute value
$rir->email('support@infoblox.com');
```

url( )

Use this method to set or retrieve the Web API URL for RIR.

Parameter

Web API URL in string format.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get attribute value
my $value = $rir->url();
#Modify attribute value
$rir->url('http://www.infoblox.com');
```

override_email( )

Use this method to set or retrieve the override_email flag of RIR.

Parameter

Specify "true" to override the predefined email address or "false" to use the predefined email address. The default value is "false".

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get attribute value
my $value = $rir->override_email();
#Modify attribute value
$rir->override_email('true');
```

override_url( )

Use this method to set or retrieve the override_url flag of RIR.

Parameter

Specify "true" to override the predefined Web API URL or "false" to use the predefined Web API URL. The default value is "false".

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get attribute value
my $value = $rir->override_url();
#Modify attribute value
$rir->override_url('true');
```

AUTHOR

Infoblox Inc. http://www.infoblox.com/

SEE ALSO

Infoblox::Session, Infoblox::Grid->enable_rir_swip()

COPYRIGHT

Copyright (c) 2017 Infoblox Inc.

Infoblox::Grid::ScheduledTask::ChangedObject - Scheduled Task Changed Object.

- NAME
- DESCRIPTION
- CONSTRUCTOR
- MODULE METHODS
  - Infoblox::Grid::ScheduledTask->changed_objects( )
- METHODS
  - action( )
  - changed_properties( )
  - object_name( )
  - object_type( )
- SAMPLE CODE
- AUTHOR
- SEE ALSO
- COPYRIGHT

NAME

Infoblox::Grid::ScheduledTask::ChangedObject - Scheduled Task Changed Object.

DESCRIPTION

A Scheduled Task Changed Object is used to retrieve information about objects with changes that were scheduled in Infoblox::Grid::ScheduledTask.

CONSTRUCTOR

The Infoblox::Grid::ScheduledTask::ChangedObject is a read-only object and does not require manual construction.

MODULE METHODS

The following functions can be applied to a ChangedObject object.

Infoblox::Grid::ScheduledTask->changed_objects( )

Use this function to retrieve the changed objects list. See Infoblox::Grid::ScheduledTask->changed_objects() for parameters and return values.

Example

```
# Retrieve the list of ScheduledTask objects
my @retrieved_objs = $session->get(
    object => "Infoblox::Grid::ScheduledTask",
    submitter => "admin" );

my $task = $retrieved_objs[0];

# get ChangedObject
my $changed_objects = $task->changed_objects();
my $obj = $changed_objects->[0];
```

METHODS

This section describes all the methods that you can use to retrieve the attribute values of a ChangedObject object.

action( )

Use this method to retrieve the operation performed on the objects.

Parameter

none

Returns

The method returns the operation in string format. Possible values are:

"Network Discovery" for network_discovery_control( ) service,

"Restart Services" for restart( ) services,

"Add", "Modify", "Delete" for all other objects.

Example

```
#Get action
my $action = $changed_object->action();
```

changed_properties( )

Use this method to retrieve the list of changed properties in the object.

Parameter

none

Returns

The method returns a reference to an array of strings. Each string in the array describes the changes. The descriptions are similiar to the entries in the Change Audit Log column in the GUI.

Example

```
#Get changed_properties value
my $changed_properties = $changed_object->changed_properties();
for (@$changed_properties) {
    print $_,"\n";
}
```

object_name( )

Use this method to retrieve the name of the changed object.

Parameter

none

Returns

The method returns the object name in string format.

Example

```
#Get object_name
my $object_name = $changed_object->object_name();
```

object_type( )

Use this method to retrieve the type of the changed object.

Parameter

none

Returns

The method returns an object type in string format, such as "A Record" or "Shared TXT Record".

Example

```
#Get object_type
my $object_type = $changed_object->object_type();
```

SAMPLE CODE

The following sample code demonstrates the different functions that can be applied to an object, such as add, search, modify, and remove. This sample also includes error handling for the operations.

#Preparation prior to a ScheduledTask object insertion

```
#PROGRAM STARTS: Include all the modules that will be used
use strict;
use Infoblox;

#Create a session to the Infoblox appliance
```

```
my $session = Infoblox::Session->new(
               master   => "192.168.1.2", #appliance host ip
               username => "admin",       #appliance user login
               password => "infoblox"     #appliance password
               );

unless ($session) {
       die("Construct session failed: ",
               Infoblox::status_code() . ":" . Infoblox::status_detail());
}
print "Session created successfully\n";

my $network = Infoblox::DHCP::Network->new(
    network => "10.0.0.0/24",
);
unless ($network) {
    die("Construct Network failed: ",
        Infoblox::status_code() . ":" . Infoblox::status_detail());
}
print "Network created successfully\n";

$session->add($network)
    or die("Add Network object failed: ",
    $session->status_code() . ":" . $session->status_detail());
print "Network object added to server successfully\n";
```

#Create a ScheduledTask object

```
# Create a fixed address
my $fixed_addr = Infoblox::DHCP::FixedAddr->new(
    ipv4addr    => '10.0.0.3',
    mac         => '11:22:33:44:55:66'
);
unless ($fixed_addr) {
    die("Construct FixedAddr failed: ",
        Infoblox::status_code() . ":" . Infoblox::status_detail());
}
print "FixedAddr created successfully\n";

# Schedule the addition of a fixed address
$session->add($fixed_addr, scheduled_at => "2020-01-01T14:52:00Z")
    or die("Schedule fixed address add failed: ",
    $session->status_code() . ":" . $session->status_detail());
print "Fixed address add scheduled successfully\n";
```

#Search for a ScheduledTask

```
my @retrieved_objs = $session->search(
    object    => "Infoblox::Grid::ScheduledTask",
    submitter => "admin"
);
my $object = $retrieved_objs[0];

unless ($object) {
    die("Search for a ScheduledTask object failed: ",
        $session->status_code() . ":" . $session->status_detail());
}
print "Search for a ScheduledTask object found at least 1 matching entry\n";
my $objs = $object->changed_objects;
my $changed_obj = $objs->[0];
my $action = $changed_obj->action();
my $object_name= $changed_obj->object_name();
my $object_type= $changed_obj->object_type();
my $changed_properties= $changed_obj->changed_properties();
my $property1 = $changed_properties->[0];
```

#Get and modify a ScheduledTask object

```
#Get the ScheduledTask object from Infoblox appliance through a session
my @retrieved_objs = $session->get(
    object         => "Infoblox::Grid::ScheduledTask",
    scheduled_time => "2020-01-01T14:52:00Z"
);
my $object = $retrieved_objs[0];

unless ($object) {
```

```
    die("Get ScheduledTask object failed: ",
        $session->status_code() . ":" . $session->status_detail());
}
print "Get ScheduledTask object found at least 1 matching entry\n";

#Modify the ScheduledTask object
$object->scheduled_time("2020-02-01T14:52:00Z");

#Apply the changes.
$session->modify($object)
    or die("Modify ScheduledTask object failed: ",
    $session->status_code() . ":" . $session->status_detail());
print "ScheduledTask object modified successfully \n";
```

#Remove a ScheduledTask object

```
#Get the ScheduledTask object through the session
my @retrieved_objs = $session->get(
    object => "Infoblox::Grid::ScheduledTask",
    action => "Add"
);
my $object = $retrieved_objs[0];
unless ($object) {
    die("Get ScheduledTask object failed: ",
        $session->status_code() . ":" . $session->status_detail());
}
print "Get ScheduledTask object found at least 1 matching entry\n";

#Submit the object for removal
$session->remove($object)
    or die("Remove ScheduledTask object failed: ",
    $session->status_code() . ":" . $session->status_detail());
print "ScheduledTask object removed successfully \n";
```

#Cleanup

```
#Get the Network object through the session
my @retrieved_objs = $session->get(
    object  => "Infoblox::DHCP::Network",
    network => "10.0.0.0/24"
);
my $object = $retrieved_objs[0];
unless ($object) {
    die("Get Network object failed: ",
        $session->status_code() . ":" . $session->status_detail());
}
print "Get Network object found at least 1 matching entry\n";

#Submit the object for removal
$session->remove($object)
    or die("Remove Network object failed: ",
    $session->status_code() . ":" . $session->status_detail());
print "Network object removed successfully \n";

####PROGRAM ENDS####
```

AUTHOR

Infoblox Inc. http://www.infoblox.com/

SEE ALSO

Infoblox::Grid::ScheduledTask, Infoblox::Grid::ScheduledTask->changed_objects()

COPYRIGHT

Copyright (c) 2017 Infoblox Inc.

Infoblox::IPAM::DiscoveryTask - Network discovery configuration

- NAME
- DESCRIPTION
- CONSTRUCTOR
- SESSION METHODS
  - Infoblox::Session->get( )
  - Infoblox::Session->modify( )
- METHODS
  - disable_ip_scanning( )
  - disable_vmware_scanning( )
  - discovery_task_oid( )
  - member( )
  - mode( )
  - merge_data( )
  - networks( )
  - network_view( )
  - ping_retries( )
  - ping_timeout( )
  - scheduled_run( )
  - state( )
  - status( )
  - tcp_ports( )
  - tcp_scan_technique( )
  - update_metadata( )
  - vservers( )
  - v_network_view( )
- SAMPLE CODE
- AUTHOR
- SEE ALSO
- COPYRIGHT

NAME

Infoblox::IPAM::DiscoveryTask - Network discovery configuration

DESCRIPTION

This class represents the condiguration of network discovery jobs. Configuration parameters in this class have control over the behavior of network discovery jobs.

CONSTRUCTOR

Both current and scheduled Infoblox::IPAM::DiscoveryTask objects are automatically generated upon the successful installation of the Infoblox appliance. You do not need to create it.

SESSION METHODS

This section describes all the methods in the Infoblox::Session module that you can apply to a discovery task object.

Infoblox::Session->get( )

Use this method to retrieve matching objects from the Infoblox appliance. See Infoblox::Session->get() for parameters and return values.

Key Reference

```
Apply the following attributes to get a specific IPAM network discovery task object:

 object - Required. It should be set to "Infoblox::IPAM::DiscoveryTask".
 discovery_task_oid - Optional. The discovery task identifier that can be "current" or "scheduled".
```

Example

```
my @retrieved_objs = $session->get(
    object => "Infoblox::IPAM::DiscoveryTask",
    discovery_task_oid => "current",
    );
```

Infoblox::Session->modify( )

Use this method to modify objects in the Infoblox appliance. See Infoblox::Session->modify() for parameters and return values.

Example

```
#Use this method to modify the ping_timeout.
$discovery_task->ping_timeout("500");
#Submit modification
my $response = $session->modify( $discovery_task);
```

METHODS

This section describes all the methods that you can use to configure and retrieve the attribute values of the network discovery task.

disable_ip_scanning( )

Use this method to enable or disable IP discovery. This is a boolean value.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

Set the disable_ip_scanning attribute to "false" to enable IP discovery. Set the attribute to "true" to disable IP discovery. The default value is "false".

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get disable_ip_scanning attribute
my $disable_ip_scanning = $discovery_task->disable_ip_scanning();
#Modify disable_ip_scanning attribute
$discovery_task->disable_ip_scanning("true");
```

disable_vmware_scanning( )

Use this method to enable or disable VM discovery. This is a boolean value.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

Set the disable_vmware_scanning attribute to "false" to enable VM discovery. Set the attribute to "true" to disable VM discovery. The default value is "true".

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get disable_vmware_scanning attribute
my $disable_vmware_scanning = $discovery_task->disable_vmware_scanning();
#Modify disable_vmware_scanning attribute
$discovery_task->disable_vmware_scanning("true");
```

discovery_task_oid( )

Use this method to retrieve the discovery task identifier. This attribute is read-only and cannot be set.

Parameter

None.

Returns

The method returns the attribute value that can be "current" or "scheduled".

Example

```
#Get discovery_task_oid attribute
my $discovery_task_oid = $discovery_task->discovery_task_oid();
```

member( )

Use this method to set or retrieve the grid member that will run the discovery. Member is required for network discovery task object.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

Valid value is a defined Infoblox::DHCP::Member object. The default value is empty.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get member
my $member = $discovery_task->member();
#Modify member
$discovery_task->member($member1); #$member1 is an Infoblox::DHCP::Member object
```

mode( )

Use this method to set or retrieve the network discovery mode.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

Network discovery mode. Valid values when setting the mode are "FULL", "ICMP", "NETBIOS" and "TCP". Default value is "FULL".

When retrieving the network discovery mode, you can get any value listed above plus the value "CSV" if a CSV discovery job is running.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get mode
my $mode = $discovery_task->mode();
#Modify mode
$discovery_task->mode("ICMP");
```

merge_data( )

Use this method to set or retrieve the merge data flag that a network discovery uses.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

The flag indicates whether the GM sorter replaces or merges the discovered data with existing data. The valid values are "true" and "false". The default value is "true".

Returns

If you specified a parameter, the method returns "true" when the modification succeeds, and it returns "false" when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get merge_data
my $merge_data = $discovery_task->merge_data();
#Modify merge_data
$discovery_task->merge_data("false");
```

networks( )

Use this method to retrieve the DHCP networks on which network discovery jobs search for IP addresses.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

The valid value is an array reference that contains Infoblox::DHCP::Network objects. Networks must contain at least one Infoblox::DHCP::Network object.

Note: The discovery task object returned from the appliance may or may not contain Infoblox::DHCP::Network objects. Networks must contain at least one Infoblox::DHCP::Network object when calling modify

function.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
my $network1 = Infoblox::DHCP::Network->new(
                                        "network" => "10.0.0.0/24",
                                );
#Get networks
my $networks = $discovery_task->networks();
#Modify networks
$discovery_task->networks([$network1]);
```

network_view( )

Use this method to set or retrieve the network view of a discovery task.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

The default value is the default network view.

Parameter

The valid value is a defined Infoblox::DHCP::View object. The default value is the system-defined network view.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get network view
my $nview = $discovery_task->network_view();
#Modify network view, with an Infoblox::DHCP::View object
$discovery_task->network_view($nview);
```

ping_retries( )

Use this method to set or retrieve the network discovery ping retry times.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

Network discovery ping retry times. Minimium value is 1 and maximum value is 5. Default value is 2.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get ping_retries
my $ping_retries = $discovery_task->ping_retries();
#Modify ping_retries
$discovery_task->ping_retries("4");
```

ping_timeout( )

Use this method to set or retrieve the network discovery ping timeout.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

Network discovery ping timeout. Minimium value is 1 ms and maximum value is 4000 ms. Default value is 1000 ms.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get ping_timeout
my $ping_timeout = $discovery_task->ping_timeout();
#Modify ping_timeout
$discovery_task->ping_timeout("500");
```

scheduled_run( )

Use this method to set or retrieve the schedule of a discovery task.

Parameter

Valid value is a Infoblox::Grid::ScheduleSetting object. The default value is undef.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get scheduled_run
my $scheduled_run = $discovery_task->scheduled_run();
#Modify scheduled_run
$discovery_task->scheduled_run($schedule_setting);
```

state( )

Use this method to retrieve the network discovery state. The discovery state is read-only and cannot be set.

Parameter

None

Returns

Return the state of network discovery.

Example

```
#Get state
my $state = $discovery_task->state();
```

status( )

Use this method to retrieve the network discovery status. The discovery status is read-only and cannot be set.

Parameter

None

Returns

Return the status of network discovery.

Example

```
#Get status
my $status = $discovery_task->status();
```

tcp_ports( )

Use this method to set or retrieve the TCP ports that the network discovery jobs use.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

The valid value is an array reference that contains Infoblox::IPAM::TCPPort objects. Default TCP ports are:

```
ftp               21/tcp      # File Transfer [Control]
ssh               22/tcp      # Secure Shell Login
```

```
telnet            23/tcp     #
smtp              25/tcp     # Simple Mail Transfer
finger            79/tcp     #
http              80/tcp     # World Wide Web HTTP
kerberos-sec      88/tcp     # Kerberos (v5)
pop3              110/tcp    # PostOffice V.3
rpcbind           111/tcp    # portmapper, rpcbind
auth              113/tcp    # ident, tap, Authentication Service
msrpc             135/tcp    # Microsoft RPC services
netbios-ns        137/tcp    # NETBIOS Name Service
netbios-dgm       138/tcp    # NETBIOS Datagram Service
netbios-ssn       139/tcp    # NETBIOS Session Service
imap              143/tcp    # Interim Mail Access Protocol v2
https             443/tcp    # secure http (SSL)
microsoft-ds      445/tcp    # SMB directly over IP
login             513/tcp    # BSD rlogind(8)
shell             514/tcp    # BSD rshd(8)
printer           515/tcp    # spooler (lpd)
klogin            543/tcp    # Kerberos (v4/v5)
kerberos          750/tcp    # kdc Kerberos (v4)
kdm               1024/tcp   # K Display Manager (KDE version of xdm)
NFS-or-IIS        1025/tcp   # IIS, NFS, or listener RFS remote_file_sharing
```

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get tcp_ports
my $ports = $discovery_task->tcp_ports();

#Modify tcp_ports
my $port = Infoblox::IPAM::TCPPort->new(
       number => '333',
       comment => 'test_port');
push @$ports , $port;

$discovery_task->tcp_ports($ports);
```

tcp_scan_technique( )

Use this method to set or retrieve the TCP scan technique that the network discovery jobs use.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

Network discovery TCP scan technique. The scan techniques are identical to Nmap techniques. Valid values are "SYN" and "CONNECT". The default value is "SYN".

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get tcp_scan_technique
my $tcp_scan_technique= $discovery_task->tcp_scan_technique();
#Modify tcp_scan_technique
$discovery_task->tcp_scan_technique("CONNECT");
```

update_metadata( )

Use this method to set or retrieve the updated metadata flag that the network discovery jobs use.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

The flag indicates whether the appliance updates the metadata of the discovered objects. The valid values are "true" and "false". The default value is "true". If the value is "false", only the unmanaged objects are updated.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get update_metadata
my $update_metadata = $discovery_task->update_metadata();
#Modify update_metadata
$discovery_task->update_metadata("false");
```

vservers( )

Use this method to set or retrieve the list of VMware vSphere servers for VM discovery.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

The valid value is an array reference that contains the Infoblox::IPAM::DiscoveryTask::VServer objects. The default value is a reference to an empty list.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get vservers
my $vservers = $discovery_task->vservers();
#Modify vservers
my $vserver1 = Infoblox::IPAM::DiscoveryTask::VServer->new(
       fqdn_or_ip => "vcenter.com",
       username => "test1",
       password => "test1",
   );
my $vserver2 = Infoblox::IPAM::DiscoveryTask::VServer->new(
       fqdn_or_ip => "10.2.3.4",
       username => "test2",
       password => "test2",
   );
$discovery_task->vservers([$vserver1,$vserver2]);
```

v_network_view( )

Use this method to set or retrieve the network view of a VM discovery.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

The default value is the default network view.

Parameter

The valid value is a defined Infoblox::DHCP::View object. The default value is the system-defined network view.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get network view
my $nview = $discovery_task->v_network_view();
#Modify network view, with an Infoblox::DHCP::View object
$discovery_task->v_network_view($nview);
```

SAMPLE CODE

The following sample code demonstrates different operations that can be applied to a network discovery task object, such as getting and modifying an object. This sample code also includes error handling for the operations.

#Preparation prior to the get and modify operations on a network discovery task object

```
#PROGRAM STARTS: Include all the modules that will be used
use strict;
use Infoblox;

#refers to Infoblox Appliance IP address
```

```
my $host_ip   = "192.168.1.2";
my $host_name = "infoblox.localdomain";

#Create a session to the Infoblox appliance
my $session = Infoblox::Session->new(
    master   => $host_ip,
    username => "admin",
    password => "infoblox"
    )
    or die("Constructor for session failed: ",
               Infoblox::status_code(). ":" .Infoblox::status_detail());

print"Session created successfully.\n";

my  @result_array = $session->get(
      object  => "Infoblox::IPAM::DiscoveryTask" ,
);

my $discovery_task = $result_array[0];
unless (ref($discovery_task) eq 'Infoblox::IPAM::DiscoveryTask') {
       die("Get discovery task failed");
}
print "Get discovery task successfully\n";

#Create the Network object
my $network = Infoblox::DHCP::Network->new(
       network => "10.0.0.0/24",
);

$session->add($network)
   or die("Add network failed: " .
              $session->status_code() . ":" . $session->status_detail());
#modify networks
$discovery_task->networks([$network]);

# modify ping_timeout
$discovery_task->ping_timeout("500");

# modify tcp_ports
my $ports = $discovery_task->tcp_ports();

my $port = Infoblox::IPAM::TCPPort->new(
       number => '333',
       comment => 'test_port');
push @$ports , $port;

$discovery_task->tcp_ports($ports);

# submit changes
$session->modify( $discovery_task )
       or die("Modify discovery task failed: " .
              $session->status_code() . ":" . $session->status_detail());
print "Modify discovery task successfully\n";

# clean up
# remove network
$session->remove( $network)
   or die("Remove network failed: " .
         $session->status_code() . ":" . $session->status_detail());

####### Program Ends ############################
```

AUTHOR

Infoblox Inc. http://www.infoblox.com/

SEE ALSO

Infoblox::IPAM::DiscoveryTask::VServer,Infoblox::DHCP::Member,Infoblox::DHCP::Network,Infoblox::IPAM::TCPPort,Infoblox::Session,Infoblox::Session->get(),Infoblox::Session->modify(),Infoblox::Grid::ScheduleSetting

COPYRIGHT

Copyright (c) 2017 Infoblox Inc.

Infoblox::Grid::ServiceRestart::Group::Schedule - A Grid Service Restart Group Schedule object.

- NAME
- DESCRIPTION
- CONSTRUCTOR
- MODULE METHODS
  - Infoblox::Grid::ServiceRestart::Group->recurring_schedule( )
- METHODS
  - services( )
  - mode( )
  - force( )
  - schedule( )
- AUTHOR
- SEE ALSO
- COPYRIGHT

NAME

Infoblox::Grid::ServiceRestart::Group::Schedule - A Grid Service Restart Group Schedule object.

DESCRIPTION

The Grid Service Restart Group Schedule object provides information about restart service schedule settings for Service Restart Group.

CONSTRUCTOR

```
my $svcrst_schedule = Infoblox::Grid::ServiceRestart::Group::Schedule->new(
   schedule => $schedule,                                  # Required
   services => $array_of_services,                         # Optional / Default is ['ALL']
   mode     => 'GROUPED' | 'SEQUENTIAL' | 'SIMULTANEOUS', # Optional / Default is 'GROUPED'
   force    => 'true' | 'false',                           # Optional / Default is 'false'
);
```

MODULE METHODS

Infoblox::Grid::ServiceRestart::Group->recurring_schedule( )

Use this method to set or retrieve the Grid Service Restart Group Schedule. See Infoblox::Grid::ServiceRestart::Group->recurring_schedule() for parameters and return values.

Example

```
#Get recurring_schedule
my $recurring_schedule = $svcrst_group->recurring_schedule();
#Modify recurring_schedule
$svcrst_group->recurring_schedule($sched);
```

METHODS

This section describes all the methods that you can use to set or retrieve the attribute values of the object.

services( )

Use this method to set or retrieve the list of services for a scheduled restart for a particular Grid Service Restart Group.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

The valid value is an array which contains following values: 'ALL', 'DNS', 'DHCP', 'DHCPV4', 'DHCPV6'.

Returns

If you specified a parameter, the method returns 'true' when the modification succeeds, and returns 'false' when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get services
my $services = $svcrst_schedule->services();
#Modify services
$svcrst_schedule->services('DHCP');
```

mode( )

Use this method to set or retrieve the service restart mode for a scheduled restart for a particular Grid Service Restart Group.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

The valid values are 'GROUPED', 'SEQUENTIAL' and 'SIMULTANEOUS'.

Returns

If you specified a parameter, the method returns 'true' when the modification succeeds, and returns 'false' when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get mode
my $mode = $svcrst_schedule->mode();
#Modify mode
$svcrst_schedule->mode('SEQUENTIAL');
```

force( )

Use this method to set or retrieve the flag that indicates whether a forced scheduled restart will be performed for a particular Grid Service Restart Group.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

Specify 'true' to set the flag, and 'false' to unset/deactivate it. The default value is 'false'.

Returns

If you specified a parameter, the method returns 'true' when the modification succeeds, and returns 'false' when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get force
my $force = $svcrst_schedule->force();
#Modify force
$svcrst_schedule->force('true');
```

schedule( )

Use this method to set or retrieve the schedule settings for a particular Grid Service Restart Group.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

The valid value is an Infoblox::Grid::ScheduleSetting object.

Returns

If you specified a parameter, the method returns 'true' when the modification succeeds, and returns 'false' when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get schedule
my $schedule = $svcrst_schedule->schedule();
#Modify schedule
$svcrst_schedule->schedule($sched);
```

Infoblox::Grid::ServiceRestart::Group::Schedule - A Grid Service Restart Group Schedule object.

AUTHOR

Infoblox Inc. http://www.infoblox.com/

SEE ALSO

Infoblox::Grid::ServiceRestart::Group, Infoblox::Grid::ServiceRestart::Group->recurring_schedule(), Infoblox::Grid::ScheduleSetting.

COPYRIGHT

Copyright (c) 2017 Infoblox Inc.

Infoblox::Grid::ServiceRestart::Request - A Grid Service Restart Request object.

- NAME
- DESCRIPTION
- SESSION METHODS
  - Infoblox::Session->get( )
  - Infoblox::Session->search( )
- MODULE METHODS
  - Infoblox::Grid::ServiceRestart::Group->requests( )
- METHODS
  - error( )
  - forced( )
  - group( )
  - last_updated_time( )
  - member( )
  - needed( )
  - order( )
  - result( )
  - service( )
  - state( )
- AUTHOR
- SEE ALSO
- COPYRIGHT

NAME

Infoblox::Grid::ServiceRestart::Request - A Grid Service Restart Request object.

DESCRIPTION

The Grid Service Restart Request object provides information and statistics about restart service routine of the Restart Service Group.

SESSION METHODS

This section describes all the methods in a Session module that you can apply to a Grid Service Restart Request object.

Infoblox::Session->get( )

Use this method to retrieve all the matching objects from the Infoblox appliance. See Infoblox::Session->get() for parameters and return values.

Key Reference

```
Apply the following attribute to get a specific object:

group  - Optional. A Grid Service Restart Group object.
member - Optional. A Grid Member name or object.
```

Example

```
my @retrieved_objs = $session->get(
    object => 'Infoblox::Grid::ServiceRestart::Request',
    group  => $group,
);
```

Infoblox::Session->search( )

Use this method to search for objects in the Infoblox appliance. See Infoblox::Session->search() for parameters and return values.

Key Reference

```
Apply the following attribute to search for a specific object:

group  - A Grid Service Restart Group object.
member - A Grid Member name or object.
```

Example

```
my @retrieved_objs = $session->search(
    object => 'Infoblox::Grid::ServiceRestart::Request',
    group  => $group,
```

```
);
```

MODULE METHODS

The following functions are available to apply to a Grid Service Restart Request object.

Infoblox::Grid::ServiceRestart::Group->requests( )

Use this method to retrieve restart requests for the Grid Service Restart Group. This is a read-only attribute. See Infoblox::Grid::ServiceRestart::Group->requests() for parameters and return values.

Example

```
#Get requests
my $requests = $svcrst_group->requests();
```

METHODS

This section describes all the methods that you can use to set or retrieve the attribute values of the object.

error( )

Use this method to retrieve error messages for failed restart requests. This is a read-only attribute.

Omit the parameter to retrieve the attribute value.

Parameter

None

Returns

The method returns the attribute value.

Example

```
#Get error
my $error = $svcrst_request->error();
```

forced( )

Use this method to retrieve the flag that indicates whether this is force restart request or not. This is a read-only attribute.

Omit the parameter to retrieve the attribute value.

Parameter

None

Returns

The method returns the attribute value

Example

```
#Get forced
my $forced = $svcrst_request->forced();
```

group( )

Use this method to retrieve the Grid Service Restart Group object that is associated with the request. This is a read-only attribute.

Omit the parameter to retrieve the attribute value.

Parameter

None

Returns

The valid return value is an Infoblox::Grid::ServiceRestart::Group object.

Example

```
#Get group
my $group = $svcrst_request->group();
```

last_updated_time( )

Use this method to retrieve the time when the request status has changed. This is a read-only attribute.

Omit the parameter to retrieve the attribute value.

Parameter

None

Returns

The method returns the number of seconds that have elapsed since January 1st, 1970 UTC.

Example

```
#Get last_updated_time
my $last_updated_time = $svcrst_request->last_updated_time();
```

member( )

Use this method to retrieve the Grid Member name for the restart. This is a read-only attribute.

Omit the parameter to retrieve the attribute value.

Parameter

None

Returns

The method returns the attribute value.

Example

```
#Get member
my $member = $svcrst_request->member();
```

needed( )

Use this method to retrieve the value that indicates whether a restart is required or not. This is a read-only attribute.

Omit the parameter to retrieve the attribute value.

Parameter

None

Returns

The valid return values are 'UNKNOWN', 'CHECKING', 'YES', 'NO' and 'FAILURE'.

Example

```
#Get needed
my $needed = $svcrst_request->needed();
```

order( )

Use this method to retrieve the restart order for the Grid Service Restart Request. This is a read-only attribute.

Omit the parameter to retrieve the attribute value.

Parameter

None

Returns

The method returns attribute value.

Example

```
#Get order
my $order = $svcrst_request->order();
```

result( )

Use this method to retrieve the result of a service restart for a particular Grid Service Restart Request. This is a read-only attribute.

Omit the parameter to retrieve the attribute value.

Parameter

None

Returns

The valid return values are 'SUCCESS', 'TIMEOUT', 'FAILURE' and 'NORESTART'.

Example

```
#Get result
my $result = $svcrst_request->result();
```

service( )

Use this method to retrieve the service to restart for a particular Grid Service Restart Request. This is a read-only attribute.

Omit the parameter to retrieve the attribute value.

Parameter

None

Returns

The valid return values are 'DNS', 'DHCP', 'DHCPV4' and 'DHCPV6'.

Example

```
#Get service
my $service = $svcrst_request->service();
```

state( )

Use this method to retrieve the state of a particular Grid Service Restart Request. This is a read-only attribute.

Omit the parameter to retrieve the attribute value.

Parameter

None

Returns

The valid return values are 'NEW', 'QUEUED', 'PROCESSING' and 'FINISHED'.

Example

```
#Get state
my $state = $svcrst_request->state();
```

AUTHOR

Infoblox Inc. http://www.infoblox.com/

SEE ALSO

Infoblox::Session, Infoblox::Session->get(), Infoblox::Session->search(), Infoblox::Grid::ServiceRestart::Group, Infoblox::Grid::ServiceRestart::Group->requests().

COPYRIGHT

Copyright (c) 2017 Infoblox Inc.

Infoblox::Grid::ServiceRestart::Group - a Grid Service Restart Group object.

- NAME
- DESCRIPTION
- CONSTRUCTOR
- SESSION METHODS
  - Infoblox::Session->add( )
  - Infoblox::Session->get( )
  - Infoblox::Session->modify( )
  - Infoblox::Session->remove( )
  - Infoblox::Session->search( )
- METHODS
  - comment( )
  - extattrs( )
  - extensible_attributes( )
  - is_default( )
  - last_updated_time( )
  - members( )
  - mode( )
  - name( )
  - position( )
  - recurring_schedule( )
  - requests( )
  - service( )
  - status( )
- AUTHOR
- SEE ALSO
- COPYRIGHT

NAME

Infoblox::Grid::ServiceRestart::Group - a Grid Service Restart Group object.

DESCRIPTION

The Grid Service Restart Group object provides information about restart of particular services and members, their order and periodicity.

CONSTRUCTOR

```
 my $svcrst_group = Infoblox::Grid::ServiceRestart::Group->new(
    name                  => $string,                                                  #
Required
    service               => 'DHCP' | 'DNS',                                           #
Required
    comment               => $string,                                                  #
Optional / Default is undefined
    mode                  => 'SIMULTANEOUS' | 'SEQUENTIAL',                            #
Optional / Default is
                                                                                       #
'SEQUENTIAL' for default restart groups /                                              #
'SIMULTANEOUS' for non-default restart groups
    members               => $string_array,                                            #
Optional / Default is undefined
    recurring_schedule    => $svcrst_schedule,                                         #
Optional / Default is undefined
    extattrs              => { $string => $extattr, ... },                             #
Optional / Default is undefined
    extensible_attributes => { $string => $string | $num, $string => [ $string | $num, ... ], ... } #
Optional / Default is undefined
 );
```

SESSION METHODS

This section describes all the methods in a Session module that you can apply to a Grid Service Restart Group object.

Infoblox::Session->add( )

Use this method to add an object to the Infoblox appliance. See Infoblox::Session->add() for parameters and return values.

Example

```
#Construct an object
my $svcrst_group = Infoblox::Grid::ServiceRestart::Group->new(
   name    => 'test_group',
   service => 'DHCP',
);

#Submit for addition
my $response = $session->add($svcrst_group);
```

Infoblox::Session->get( )

Use this method to retrieve all the matching objects from the Infoblox appliance. See Infoblox::Session->get() for parameters and return values.

Key Reference

```
Apply the following attribute to get a specific object:

  comment               - Optional. A Grid Service Restart Group comment.
  extattrs              - Optional. A hash reference containing extensible attributes.
  extensible_attributes - Optional. A hash reference that contains extensible attributes.
  is_default            - Optional. Flag which indicates whether the Grid Service Restart Group is a
default restart group or not.
  name                  - Optional. A Grid Service Restart Group name.
  position              - Optional. A Grid Service Restart Group position in Restart Service Order
object.
  service               - Optional. A Grid Service Restart Group service name.
```

Example

```
my @retrieved_objs = $session->get(
    object => 'Infoblox::Grid::ServiceRestart::Group',
    name   => 'test_group',
);

my @retrieved_objs = $session->get(
    object => 'Infoblox::Grid::ServiceRestart::Group',
    extensible_attributes => { 'Site' => 'Santa Clara' }
);
```

Infoblox::Session->modify( )

Use this method to modify an object in the Infoblox appliance. See Infoblox::Session->modify() for parameters and return values.

Example

```
#Use method to modify the comment.
$svcrst_group->comment('this is a modified comment');
#Submit modification
my $response = $session->modify($svcrst_group);
```

Infoblox::Session->remove( )

Use this method to remove an object from the Infoblox appliance. See Infoblox::Session->remove() for parameters and return values.

To remove a specific object, first use `get()` or `search()` to retrieve the specific object, and then submit this object for removal.

Example

```
#Get the objects with the same name
my @retrieved_objs = $session->get(
    object => 'Infoblox::Grid::ServiceRestart::Group',
    name   => 'svcrst_group1',
);
#Find the desired object from the retrieved list.
my $desired_svcrst_group = $retrieved_objs[0];
#Submit for removal
my $response = $session->remove($desired_svcrst_group);
```

Infoblox::Session->search( )

Use this method to search for objects in the Infoblox appliance. See Infoblox::Session->search() for parameters and return values.

Key Reference

```
Apply the following attribute to get a specific object:

  comment               - Optional. A Grid Service Restart Group comment.
  extattrs              - Optional. A hash reference containing extensible attributes.
```

```
  extensible_attributes - Optional. A hash reference that contains extensible attributes.
  is_default            - Optional. Flag which indicates whether the Grid Service Restart Group is a
default restart group or not.
  name                  - Optional. A Grid Service Restart Group name.
  position              - Optional. A Grid Service Restart Group position in Restart Service Order
object.
  service               - Optional. A Grid Service Restart Group service name.
```

Example

```
my @retrieved_objs = $session->search(
    object => 'Infoblox::Grid::ServiceRestart::Group',
    name   => 'svcrst.*',
);

my @retrieved_objs = $session->search(
    object => 'Infoblox::Grid::ServiceRestart::Group',
    extensible_attributes => { 'Site' => 'Santa Clara' }
);
```

METHODS

This section describes all the methods that you can use to set or retrieve the attribute values of the object.

comment( )

Use this method to set or retrieve Grid Service Restart Group object comment.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

The valid value is a desired comment in string format.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get comment
my $comment = $svcrst_group->comment();
#Modify comment
$svcrst_group->comment(50);
```

extattrs( )

Use this method to set or retrieve the extensible attributes associated with a Grid Service Restart Group object.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

Valid value is a hash reference containing the names of extensible attributes and their associated values (Infoblox::Grid::Extattr objects).

Returns

If you specified a parameter, the method returns 'true' when the modification succeeds, and returns 'false' when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#get extattrs value
my $ref_extattrs = $svcrst_group->extattrs();

#Modify extattrs
$svcrst_group->extattrs({ 'Site' => $extattr1, 'Administrator' => $extattr2 });
```

extensible_attributes( )

Use this method to set or retrieve the extensible attributes associated with a Grid Service Restart Group object.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

For valid values for extensible attributes, see Infoblox::Grid::ExtensibleAttributeDef/Extensible Attribute Values.

Returns

If you specified a parameter, the method returns 'true' when the modification succeeds, and returns 'false' when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get extensible attributes
my $ref_extensible_attributes = $svcrst_group->extensible_attributes();

#Modify extensible attributes
$svcrst_group->extensible_attributes({'Site' => 'Santa Clara', 'Administrator' => ['Peter',
'Tom']});
```

is_default( )

Use this method to retrieve the flag which indicates whether the Grid Service Restart Group is a default group or not. This is a read-only attribute.

Omit the parameter to retrieve the attribute value.

Parameter

None

Returns

The method returns attribute value.

Example

```
#Get is_default
my $is_default = $svcrst_group->is_default();
```

last_updated_time( )

Use this method to retrieve time of the last restart of the Grid Service Restart Group. This is the read-only attribute.

Omit the parameter tor retrieve the attribute value.

Parameter

None

Returns

The method returns the number of seconds that have elapsed since January 1st, 1970 UTC.

Example

```
#Get last_updated_time
my $last_updated_time = $svcrst_group->last_updated_time()
```

members( )

Use this method to set or retrieve Grid Members associated with the Grid Service Restart Group.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

The valid value is an array reference of desired Grid Members names.

Returns

If you specified a parameter, the method returns 'true' when the modification succeeds, and returns 'false' when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get members
my $members = $svcrst_group->members();
#Modify members
$svcrst_group->members(['foo.com', 'bar.com']);
```

mode( )

Use this method to set or retrieve the restart mode of Grid Members in the Grid Service Restart Group.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Note that if the Service Restart Group is a default one, the default value is 'SEQUENTIAL', 'SIMULTANEOUS' otherwise.

Parameter

The valid values are 'SEQUENTIAL' and 'SIMULTANEOUS'.

Returns

If you specified a parameter, the method returns 'true' when the modification succeeds, and returns 'false' when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get mode
my $mode = $svcrst_group->mode();
#Modify mode
$svcrst_group->mode('SEQUENTIAL');
```

name( )

Use this method to set or retrieve the Grid Service Restart Group name.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

The valid value is a desired name in a string format.

Returns

If you specified a parameter, the method returns 'true' when the modification succeeds, and returns 'false' when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get name
my $name = $svcrst_group->name();
#Modify name
$svcrst_group->name('new_name');
```

position( )

Use this method to retrieve the restart position of the Grid Service Restart Group. This is a read-only attribute.

Omit the parameter to retrieve the attribute value.

Parameter

None

Returns

The method returns the attribute value.

Example

```
#Get position
my $position = $svcrst_group->position();
```

recurring_schedule( )

Use this method to set or retrieve the recurring schedule of the Grid Service Restart Group.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

The valid value is an Infoblox::Grid::ServiceRestart::Group::Schedule object.

Returns

If you specified a parameter, the method returns 'true' when the modification succeeds, and returns 'false' when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get recurring_schedule
my $recurring_schedule = $svcrst_group->recurring_schedule();
#Modify recurring_schedule
$svcrst_group->recurring_schedule($svcrst_sched);
```

requests( )

Use this method to retrieve the array of Grid Service Restart Requests. This is a read-only attribute.

Omit the parameter to retrieve the attribute value.

Parameter

None

Returns

The valid return value is an array of Infoblox::Grid::ServiceRestart::Request objects.

Example

```
#Get requests
my $requests = $svcrst_group->requests();
```

service( )

Use this method to set or retrieve the service name for the Grid Service Restart Group.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

The valid value is 'DHCP' or 'DNS'.

Returns

If you specified a parameter, the method returns 'true' when the modification succeeds, and returns 'false' when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get service
my $service = $svcrst_group->service();
#Modify service
$svcrst_group->service('DHCP');
```

status( )

Use this method to retrieve the Grid Service Restart Group restart status. This is a read-only attribute.

Omit the parameter to retrieve the attribute value.

Parameter

None

Returns

The valid return value is an Infoblox::Grid::ServiceRestart::Status object.

Example

```
#Get status
my $status = $svcrst_group->status();
```

AUTHOR

Infoblox Inc. http://www.infoblox.com/

SEE ALSO

Infoblox::Session, Infoblox::Session->add(), Infoblox::Session->get(), Infoblox::Session->modify(), Infoblox::Session->remove(), Infoblox::Session->search(), Infoblox::Grid::Extattr, Infoblox::Grid::ExtensibleAttributeDef/Extensible Attribute Values, Infoblox::Grid::ServiceRestart::Group::Schedule, Infoblox::Grid::ServiceRestart::Request, Infoblox::Grid::ServiceRestart::Status.

COPYRIGHT

Copyright (c) 2017 Infoblox Inc.

Infoblox::Grid::GlobalSmartFolder - Global Smart Folder object.

- NAME
- DESCRIPTION
- CONSTRUCTOR
- SESSION METHODS
  - Infoblox::Session->add( )
  - Infoblox::Session->get( )
  - Infoblox::Session->modify( )
  - Infoblox::Session->remove( )
  - Infoblox::Session->search( )
- METHODS
  - comment( )
  - group_bys( )
  - name( )
  - query_items( )
- SAMPLE CODE
- AUTHOR
- SEE ALSO
- COPYRIGHT

NAME

Infoblox::Grid::GlobalSmartFolder - Global Smart Folder object.

DESCRIPTION

A Global Smart Folder object is used for data organization.

CONSTRUCTOR

```
 my $group = Infoblox::Grid::GlobalSmartFolder->new(
     name                => $string,                                    #Required
     comment             => $string,                                    #Optional
/ Default is undefined
     group_bys           => [$GroupbyItem1, $GroupbyItem2],             #Optional
/ Default is undefined
     query_items         => [$QueryItem1, $QueryItem2],                 #Optional
/ Default is undefined
 );
```

SESSION METHODS

This section describes all the methods in the Infoblox::Session module that you can apply to a Global Smart Folder object.

Infoblox::Session->add( )

Use this method to add an object to the Infoblox appliance. See Infoblox::Session->add() for parameters and return values.

Example

```
#Construct the QueryItem in-memory object
my $query_item1 = Infoblox::Grid::SmartFolder::QueryItem->new(
    name                => "location",
    operator            => "eq",
    value               => "Santa Clara",
);

#Construct the GlobalSmartFolder object
my $group_by_state = Infoblox::Grid::SmartFolder::GroupBy->new(
                                                    enable_grouping => 'true',
                                                    value => "State",
                                                    value_type => "EXTATTR",
                                                   );

my $smart_folder = Infoblox::Grid::GlobalSmartFolder->new(
    name                => "smartfolder1",
    comment             => "global smart folder",
    group_bys           => [$group_by_state],
    query_items         => [$query_item1],
```

```
);
# Submit for addition
my $response = $session->add( $smart_folder );
```

Infoblox::Session->get( )

Use this method to retrieve all the matching objects from the Infoblox appliance. See Infoblox::Session->get() for parameters and return values.

Key References

```
Apply the following attributes to get a specific GlobalSmartFolder object:

 name      - Required. GlobalSmartFolder name in string format.
 comment   - Optional. Comment in string format.
```

Example

```
my @retrieved_objs = $session->get(
    object     => "Infoblox::Grid::GlobalSmartFolder",
    name       => "smartfolder1" );
```

Infoblox::Session->modify( )

Use this method to modify an object in the Infoblox appliance. See Infoblox::Session->modify() for parameters and return values.

Example

```
# Use this method to modify the comment.
$smart_folder->comment("This is a modified comment");
# Submit modification
my $response = $session->modify( $smart_folder );
```

Infoblox::Session->remove( )

Use this method to remove an object from the Infoblox appliance. See Infoblox::Session->remove() for parameters and return values.

To remove a specific object, use `get()` or `search()` to retrieve the specific object first, and then submit this object for removal.

Example

```
# Get the GlobalSmartFolder objects using a name
my @retrieved_objs = $session->get(
    object     => "Infoblox::Grid::GlobalSmartFolder",
    name       => "smartfolder1" );
# find the desired object from the retrieved list.
my $desired_smart_folder = $retrieved_objs[0];
# Submit for removal
my $response = $session->remove( $desired_smart_folder );
```

Infoblox::Session->search( )

Use this method to search for Global Smart Folder objects in the Infoblox appliance. See Infoblox::Session->search() for parameters and return values.

Key References

```
Apply the following attributes to search for a specific GlobalSmartFolder object:

 name      - Optional. GlobalSmartFolder name in string format (regular expression).
 comment   - Optional. Comment in string format (regular expression).
```

Example

```
# search for all GlobalSmartFolder objects that start with "smartfolder"
my @retrieved_objs = $session->search(
    object => "Infoblox::Grid::GlobalSmartFolder",
    name   => "smartfolder.*" );
```

METHODS

This section describes all the methods that you can use to set and retrieve the attribute values of a GlobalSmartFolder object.

comment( )

Use this method to set or retrieve the comment.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

Pertinent information about the global smart folder. Comment in string format with a maximum of 256 bytes. The default value is undefined.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get comment
my $comment = $smart_folder->comment();
#Modify comment
$smart_folder->comment("This is the modified comment for smartfolder1");
```

group_bys( )

Use this method to set or retrieve the group_bys value.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

The valid value is an array reference that contains Infoblox::Grid::SmartFolder::GroupBy object(s). The default value is undefined.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
my $group_by_state = Infoblox::Grid::SmartFolder::GroupBy->new(
                                                    enable_grouping => 'true',
                                                    value => "State",
                                                    value_type => "EXTATTR",
                                                   );
#Get group_by value
my $group_by = $smart_folder->group_bys();
#Modify group_bys value
$smart_folder->group_bys([$group_by_state]);
```

name( )

Use this method to set or retrieve the name of a global smart folder.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

Text with the name of the global smart folder.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get name
my $name = $smart_folder->name();
```

query_items( )

Use this method to set or retrieve the query items. If query items are not specified when you `add()` a smart folder, the server automatically assigns a default query item for the smart folder. This default query item is "type=Network/Zone/Range/Member". This ensures that objects are included in searches for these 4 types, by default. When you `modify()` a smart folder, the server keeps the query items provided by the client and does not add the default items.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

The valid value is an array reference that contains Infoblox::Grid::SmartFolder::QueryItem object(s). The default value is undefined.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get query items
my $query_items = $smart_folder->query_items();

#Modify query items
my $query_item1 = Infoblox::Grid::SmartFolder::QueryItem->new(
    name                => "location",
    operator            => "eq",
    value               => "Santa Clara",
);
$smart_folder->query_items([$query_items1]);
```

SAMPLE CODE

The following sample code demonstrates the different functions that can be applied to an object, such as add, search, modify, and remove. This sample also includes error handling for the operations.

#Preparation prior to an GlobalSmartFolder object insertion

```
#PROGRAM STARTS: Include all the modules that will be used
use strict;
use Infoblox;

#Create a session to the Infoblox appliance

my $session = Infoblox::Session->new(
              master   => "192.168.1.2", #appliance host ip
              username => "admin",       #appliance user login
              password => "infoblox"     #appliance password
              );

unless ($session) {
       die("Construct session failed: ",
              Infoblox::status_code() . ":" . Infoblox::status_detail());
}
print "Session created successfully\n";

# Add a network with extensible attributes

my %extensible_attributes=(
    'Region' => 'Santa Clara County',
    'Country' => 'USA',
    'State' => 'California',
    'Site' => 'Santa Clara',
    'Building' => 'HQ',
    'VLAN' => '1',
    );

my $network = Infoblox::DHCP::Network->new(
    network => "10.0.0.0/24",
    extensible_attributes => \%extensible_attributes,
    );
unless ($network) {
  die("Construct Network failed: ",
      Infoblox::status_code() . ":" . Infoblox::status_detail());
}
print "Network created successfully\n";

$session->add($network)
  or die("Add Network object failed: ",
      $session->status_code() . ":" . $session->status_detail());
print "Network object added to server successfully\n";
```

#Create a GlobalSmartFolder object

```
my $query_item = Infoblox::Grid::SmartFolder::QueryItem->new(
    name                    => "Country",
    is_extensible_attribute => "true",
    operator                => "eq",
    value                   => "USA",
    );

unless ($query_item) {
  die("Construct query item object failed: ",
      Infoblox::status_code() . ":" . Infoblox::status_detail());
}
print "Smart folder query item object created successfully\n";

my $group_by_country = Infoblox::Grid::SmartFolder::GroupBy->new(
                                                        enable_grouping => 'true',
                                                        value => "Country",
                                                        value_type => "EXTATTR",
                                                       );

my $smart_folder = Infoblox::Grid::GlobalSmartFolder->new(
    name                    => "my_folder_global",
    comment                 => "my global folder",
    query_items             => [$query_item],
    group_bys               => [$group_by_country],
    );

unless ($smart_folder) {
  die("Construct global smart folder object failed: ",
      Infoblox::status_code() . ":" . Infoblox::status_detail());
}
print "Smart folder object created successfully\n";

#Add the GlobalSmartFolder object to the Infoblox appliance through a session
$session->add($smart_folder)
  or die("Add GlobalSmartFolder object failed: ",
      $session->status_code() . ":" . $session->status_detail());
print "GlobalSmartFolder object added to server successfully\n";
```

#Search for a GlobalSmartFolder

```
my @retrieved_objs = $session->search(
    object => "Infoblox::Grid::GlobalSmartFolder",
    name   => "my_folder.*"
    );
my $object = $retrieved_objs[0];

unless ($object) {
  die("Search for a GlobalSmartFolder object failed: ",
      $session->status_code() . ":" . $session->status_detail());
}
print "Search for a GlobalSmartFolder object found at least 1 matching entry\n";
```

#Get and modify a GlobalSmartFolder object

```
#Get the GlobalSmartFolder object from Infoblox appliance through a session
my @retrieved_objs = $session->get(
    object => "Infoblox::Grid::GlobalSmartFolder",
    name   => "my_folder_global"
    );
my $object = $retrieved_objs[0];

unless ($object) {
  die("Get GlobalSmartFolder object failed: ",
      $session->status_code() . ":" . $session->status_detail());
}
print "Get GlobalSmartFolder object found at least 1 matching entry\n";

#Modify the GlobalSmartFolder object
$object->name("my_folder_global_renamed");
$object->comment("modified global smart folder");

#Apply the changes.
$session->modify($object)
  or die("Modify GlobalSmartFolder object failed: ",
      $session->status_code() . ":" . $session->status_detail());
print "GlobalSmartFolder object modified successfully \n";
```

#Remove a GlobalSmartFolder object

```
#Get the GlobalSmartFolder object through the session
my @retrieved_objs = $session->get(
    object => "Infoblox::Grid::GlobalSmartFolder",
    name   => "my_folder_global_renamed"
    );
my $object = $retrieved_objs[0];
unless ($object) {
  die("Get GlobalSmartFolder object failed: ",
      $session->status_code() . ":" . $session->status_detail());
}
print "Get GlobalSmartFolder object found at least 1 matching entry\n";

#Submit the object for removal
$session->remove($object)
  or die("Remove GlobalSmartFolder object failed: ",
      $session->status_code() . ":" . $session->status_detail());
print "GlobalSmartFolder object removed successfully \n";
```

#Cleanup

```
#Get the Network object through the session
my @retrieved_objs = $session->get(
    object  => "Infoblox::DHCP::Network",
    network => "10.0.0.0/24"
    );
my $object = $retrieved_objs[0];
unless ($object) {
  die("Get Network object failed: ",
      $session->status_code() . ":" . $session->status_detail());
}
print "Get Network object found at least 1 matching entry\n";

#Submit the object for removal
$session->remove($object)
  or die("Remove Network object failed: ",
      $session->status_code() . ":" . $session->status_detail());
print "Network object removed successfully \n";

####PROGRAM ENDS####
```

AUTHOR

Infoblox Inc. http://www.infoblox.com/

SEE ALSO

Infoblox::Grid::SmartFolderChildren, Infoblox::Grid::PersonalSmartFolder, Infoblox::Session->add(), Infoblox::Session->get(), Infoblox::Session->modify(), Infoblox::Session->remove(), Infoblox::Session->search(),Infoblox::Session

---

COPYRIGHT

Copyright (c) 2017 Infoblox Inc.

Infoblox::Grid::ThreatAnalytics - Grid Threat Analytics object.

- NAME
- DESCRIPTION
- SESSION METHODS
  - Infoblox::Session->get( )
  - Infoblox::Session->modify( )
- METHODS
  - current_moduleset( )
  - dns_tunnel_black_list_rpz_zones( )
  - download_threat_analytics_moduleset_update( )
  - enable_auto_download( )
  - enable_scheduled_download( )
  - last_checked_for_update( )
  - last_module_update_time( )
  - last_module_update_version( )
  - last_whitelist_update_time( )
  - last_whitelist_update_version( )
  - module_update_policy( )
  - scheduled_download( )
  - test_threat_analytics_server_connectivity( )
  - update_threat_analytics_moduleset( )
- AUTHOR
- SEE ALSO
- COPYRIGHT

NAME

Infoblox::Grid::ThreatAnalytics - Grid Threat Analytics object.

DESCRIPTION

The object provides information about Grid Threat Analytics settings.

SESSION METHODS

The following functions can be applied to a Infoblox::Grid::ThreatAnalytics object.

Infoblox::Session->get( )

Use this method to retrieve all the matching objects from the Infoblox appliance. See Infoblox::Session->get() for parameters and return values.

Key References

```
Apply the following attributes to get a specific Grid Threat Analytics object
 object - Required. It must be set to "Infoblox::Grid::ThreatAnalytics".
```

Example

```
my @retrieved_objs = $session->get(
    'object' => 'Infoblox::Grid::ThreatAnalytics',
);
```

Infoblox::Session->modify( )

Use this method to modify an object in the Infoblox appliance. See Infoblox::Session->modify() for parameters and return values.

Example

```
# Use this method to modify some of the attributes of the retrieved_objs object.
$retrieved_objs->dns_tunnel_black_list_rpz_zones([$rpz]);
# Submit modification
my $response = $session->modify($retrieved_objs);
```

METHODS

This section describes all the methods that you can use to configure and retrieve values of the Grid Threat Analytics object.

current_moduleset( )

Use this method to set or retrieve the current threat analytics model set

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

The valid value is an Infoblox::Grid::ThreatAnalytics::ModuleSet object.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get attribute value
my $current_moduleset = $obj->current_moduleset();
#Modify attribute value
$obj->current_moduleset($current_moduleset);
```

dns_tunnel_black_list_rpz_zones( )

Use this method to set or retrieve the list of Response Policy Zones for DNS tunnelling requests.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

The valid value is an array reference that contains Infoblox::DNS::Zone objects.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get attribute value
my $rpz_zones = $obj->dns_tunnel_black_list_rpz_zones();
#Modify attribute value
$obj->dns_tunnel_black_list_rpz_zones([$rpz_zone]);
```

download_threat_analytics_moduleset_update( )

Use this method to download and install the latest threar analytics module set update, available from the Infoblox threat analytics download server.

Key Reference

None

Returns

The method returns 1 when the operation succeeds, and 0 when the operation fails.

Example

```
my $res = $obj->download_threat_analytics_moduleset_update();
```

enable_auto_download( )

Use this method to set or retrieve the flag that indicates whether the automatic threat analytics module set download is enabled or disabled.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

Specify 'true' to enable the automatic download of the threat analytics module set or 'false' to disable it. The default value is 'false'.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get enable_auto_download value
my $enable_auto_download = $obj->enable_auto_download();
#Modify enable_auto_download value
$obj->enable_auto_download('false');
```

enable_scheduled_download( )

Use this method to set or retrieve the flag that indicates whether the scheduled download of the threat analytics module set is enabled or disabled.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

Specify 'true' to enable the scheduled download of the threat analytics module set or 'false' to disable it. The default value is 'false'.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get enable_scheduled_download value
my $enable_scheduled_download = $obj->enable_scheduled_download();
#Modify enable_scheduled_download value
$obj->enable_scheduled_download('false');
```

last_checked_for_update( )

Use this method to retrieve the last checked for update time for the analytics module set.

Omit the parameter to retrieve the attribute value.

Parameter

None

Returns

The method returns the number of seconds that have elapsed since January 1st, 1970 UTC.

Example

```
#Get last_checked_for_update value
my $last_checked_for_update = $obj->last_checked_for_update();
```

last_module_update_time( )

Use this method to retrieve the last update time for the threat analytics module set.

Omit the parameter to retrieve the attribute value.

Parameter

None

Returns

The method returns the number of seconds that have elapsed since January 1st, 1970 UTC.

Example

```
#Get last_module_update_time value
my $last_module_update_time = $obj->last_module_update_time();
```

last_module_update_version( )

Use this method to retrieve the version number of the last updated threat analytics module set.

Omit the parameter to retrieve the attribute value.

Parameter

None

Returns

The method returns the attribute value.

Example

```
#Get last_module_update_version value
my $last_module_update_version = $obj->last_module_update_version();
```

last_whitelist_update_time( )

Use this method to retrieve the last update time for the threat analytics whitelist.

Omit the parameter to retrieve the attribute value.

Parameter

None

Returns

The method returns the number of seconds that have elapsed since January 1st, 1970 UTC.

Example

```
#Get last_whitelist_update_time value
my $last_whitelist_update_time = $obj->last_whitelist_update_time();
```

last_whitelist_update_version( )

Use this method to retrieve the version number of the last updated threat analytics whitelist.

Omit the parameter to retrieve the attribute value.

Parameter

None

Returns

The method returns the attribute value.

Example

```
#Get last_whitelist_update_version value
my $last_whitelist_update_version = $obj->last_whitelist_update_version();
```

module_update_policy( )

Use this method to set or retrieve the updated policy for the threat analytics module set.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

The valid values are 'AUTOMATIC' and 'MANUAL'.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get module_update_policy value
my $module_update_policy = $obj->module_update_policy();
#Modify module_update_policy value
$obj->module_update_policy('MANUAL');
```

scheduled_download( )

Use this method to set or retrieve the schedule settings for the threat analytics module set download.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

The valid value is an Infoblox::Grid::ScheduleSetting object.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get scheduled_download value
my $scheduled_download = $obj->scheduled_download();
#Modify scheduled_download value
$obj->scheduled_download($sched_download);
```

test_threat_analytics_server_connectivity( )

Use this method ot test the connectivity with the Infoblox threat analytics download server.

Key Reference

None

Returns

The method returns an Infoblox::Grid::TestResult object if connectivity test occurs and 0 otherwise.

Example

```
my $res = $obj->test_threat_analytics_server_connectivity();
```

update_threat_analytics_moduleset( )

Use this method to update the threat analytics module set

Note that to update the threat analytics module set, the threat analytics update file should be uploaded via Infoblox::Session->import_data() method

Key Reference

```
mode - Optional. Specify 'TEST' to check uploaded module set or 'EXECUTE' to apply updates to the
current module set. The default value is 'EXECUTE'.
```

Returns

The method returns 1 when the operation succeeds, and 0 when the operation fails.

Example

```
my $res = $obj->update_threat_analytics_moduleset(mode => 'TEST');
```

AUTHOR

Infoblox Inc. http://www.infoblox.com/

SEE ALSO

Infoblox::Session, Infoblox::Session->get(), Infoblox::Session->modify(), Infoblox::DNS::Zone, Infoblox::Grid::ScheduleSetting, Infoblox::Grid::ThreatAnalytics::ModuleSet

---

COPYRIGHT

Copyright (c) 2017 Infoblox Inc.

Infoblox::Grid::ThreatAnalytics::ModuleSet - A Grid Threat Analytics moduleset object.

- NAME
- DESCRIPTION
- SESSION METHODS
  - Infoblox::Session->get( )
  - Infoblox::Session->search( )
- METHODS
  - version( )
- AUTHOR
- SEE ALSO
- COPYRIGHT

NAME

Infoblox::Grid::ThreatAnalytics::ModuleSet - A Grid Threat Analytics moduleset object.

DESCRIPTION

The Grid Threat Analytics moduleset represents an uploaded Apache Spark Module.

SESSION METHODS

The following functions can be applied to a Threat Analytics moduleset object.

Infoblox::Session->get( )

Use this method to retrieve all the matching objects from the Infoblox appliance. See Infoblox::Session->get() for parameters and return values.

Key References

```
Apply the following attributes to get a specific Threat Analytics moduleset object.

 version - Optional. The Threat Analytics moduleset version.
```

Example

```
my @retrieved_objs = $session->get(
    'object' => 'Infoblox::Grid::ThreatAnalytics::ModuleSet',
    'version' => '1.0',
);
```

Infoblox::Session->search( )

Use this method to search for objects in the Infoblox appliance. See Infoblox::Session->search() for parameters and return values.

Key References

```
Apply the following attributes to get a specific Threat Analytics moduleset object.

 version - Optional. The Threat Analytics moduleset version.
```

Example

```
my @retrieved_objs = $session->get(
    'object' => 'Infoblox::Grid::ThreatAnalytics::ModuleSet',
    'version' => '1.*',
);
```

METHODS

version( )

Use this method to retrieve the Threat Analytics moduleset version.

Omit the parameter to retrieve the attribute value.

Parameter

None

Returns

The method returns the attribute value.

Example

```
#Get version
my $version = $object->version();
```

AUTHOR

Infoblox Inc. http://www.infoblox.com/

SEE ALSO

Infoblox::Session, Infoblox::Session->get(), Infoblox::Session->search()

COPYRIGHT

Copyright (c) 2017 Infoblox Inc.

Infoblox::Grid::TestResult - A result object returned by various test functions.

- NAME
- DESCRIPTION
- SESSION METHODS
- METHODS
  - error_message( )
  - error_messages( )
  - overall_status( )
- AUTHOR
- SEE ALSO
- COPYRIGHT

NAME

Infoblox::Grid::TestResult - A result object returned by various test functions.

DESCRIPTION

The result object represents the evaluation status for various test operations (eg Threat Analytics server connectivity, Threat Protection server connectivity).

SESSION METHODS

The object does not support any session methods.

METHODS

This section describes all the methods that you can use to retrieve the attribute values of a test result object.

error_message( )

Use this method to retrieve the single error messages returned by the test operation. This is a read-only attribute.

Parameter

None

Returns

The method returns the attribute value, that is an error message in string format.

Example

```
#Get error_message value
my $error_message = $object->error_message();
```

error_messages( )

Use this method to retrieve the list of error messages returned by the test operation. This is a read-only attribute.

Parameter

None

Returns

The method returns an array reference that contains error messages in a string format.

Example

```
#Get error_messages value
my $error_messages = $object->error_messages();
```

overall_status( )

Use this method to retrieve the overall status returned by the test operation. This is a read-only attribute.

Parameter

None

Returns

The method returns the attribute value.

Example

```
#Get overall_status value
my $overall_status = $object->overall_status();
```

AUTHOR

Infoblox Inc. http://www.infoblox.com/

SEE ALSO

Infoblox::Grid::ThreatProtection->test_threat_protection_server_connectivity(), Infoblox::Grid::ThreatAnalytics->test_threat_analytics_server_connectivity(),

---

COPYRIGHT

Copyright (c) 2017 Infoblox Inc.

Infoblox::DNS::RPZRecord::CNAME - DNS Response Policy Zone CNAME record object.

- NAME
- DESCRIPTION
- CONSTRUCTOR
- SESSION METHODS
  - Infoblox::Session->add( )
  - Infoblox::Session->get( )
  - Infoblox::Session->modify( )
  - Infoblox::Session->remove( )
  - Infoblox::Session->search( )
- METHODS
  - canonical( )
  - comment( )
  - disable( )
  - extattrs( )
  - extensible_attributes( )
  - name( )
  - ttl( )
  - views( )
  - zone( )
  - rp_zone( )
- SAMPLE CODE
- AUTHOR
- SEE ALSO
- COPYRIGHT

NAME

Infoblox::DNS::RPZRecord::CNAME - DNS Response Policy Zone (RPZ) CNAME record object.

DESCRIPTION

An RPZ CNAME record represents different RPZ rules, depending on the value of the canonical name. The intention of this object is to support QNAME Trigger policy. The QNAME policy trigger applies to requested domain names (QNAME). This record represents Passthru Domain Name Rule, Block Domain Name (No Such Domain) Rule, Block Domain Name (No Data) Rule and Substitute (Domain Name) Rule.

If canonical name is empty, it is a Block Domain Name (No Such Domain) Rule.

If canonical name is asterisk, it is a Block Domain Name (No Data) Rule.

If canonical name is the same as record name, it is a Passthru Domain Name Rule. If name of object starts with wildcard you must specify special value 'infoblox-passthru' in canonical name in order to create Wildcard Passthru Domain Name Rule, for more details please see the Infoblox Administrator Guide.

If canonical name is not Block Domain Name (No Such Domain) Rule, Block Domain Name (No Data) Rule, or Passthru Domain Name Rule, it is a substitution rule.

Note that you must specify only one view for the attribute "views".

CONSTRUCTOR

```
  my $bind_cname =  Infoblox::DNS::RPZRecord::CNAME->new (
      canonical => $fqdn,                  #Required
      name      => $fqdn,                  #Required
      rp_zone   => $rp_zone,               #Required
      comment   => $string,                #Optional / Default is empty
      disable   => "true"|"false"          #Optional / Default "false"
      extattrs              => { $string => $extattr, ... },      #Optional / Default is undefined
      extensible_attributes => { $string => $string|$num, $string => [ $string|$num, ...], ... }, #Optional
/ Default is undefined
      ttl       => $num|undef,             #Optional / Default is undefined
      views     => [$View1],               #Optional / Default is "default" view
   );
```

You cannot set both extattrs and extensible_attributes attributes at the same time.

SESSION METHODS

This section describes all the methods in Infoblox::Session module that you can apply to a RPZ CNAME record object.

Infoblox::Session->add( )

Use this method to add an object to the Infoblox appliance. See Infoblox::Session->add() for parameters and return values.

Example

```
#Construct an object
my $bind_cname = Infoblox::DNS::RPZRecord::CNAME->new (
    name      => "bind_cname.rpz.com",
    canonical => "cname.domain.com",
    rp_zone   => response_policy_zone1,
    comment   => "this is a demo bind_cname record",
    disable   => "true",
    ttl       => "33",
    views     => [$view1]  );
# Submit for addition
my $response = $session->add( $bind_cname );
```

Infoblox::Session->get( )

Use this method to retrieve all the matching objects from the Infoblox appliance. See Infoblox::Session->get() for parameters and return values.

Key References

```
Apply the following attributes to get a specific DNS RPZ CNAME object:

 name - Optional. A host name in string format.
 view - Optional. The DNS view in which the RPZ CNAME record is located. By default, all DNS views
are searched. But if you omit this attribute and specify a zone, the appliance searches the 'default'
view only.
 zone - Optional. A zone name in FQDN format.
 canonical - Optional. A canonical name.
 extattrs  - Optional. A hash reference containing extensible attributes.
 extensible_attributes - Optional. A hash reference containing extensible attributes.
```

Example

```
my @retrieved_objs = $session->get(
    object => "Infoblox::DNS::RPZRecord::CNAME",
    name   => "bind_cname.rpz.com",
    view   => "default" );

# get all DNS RPZ CNAME record in zone "rpz.com" of default view
my @retrieved_objs = $session->get(
    object => "Infoblox::DNS::RPZRecord::CNAME",
    zone   => "rpz.com",
    view   => "default" );

# get DNS RPZ CNAME record with canonical name "cname.rpz.com" of default view
my @retrieved_objs = $session->get(
    object => "Infoblox::DNS::RPZRecord::CNAME",
    canonical => "cname.domain.com",
    view   => "default" );

# get all DNS RPZ CNAME record with a given extensible attribute
my @retrieved_objs = $session->get(
    object => "Infoblox::DNS::RPZRecord::CNAME",
    extensible_attributes => { 'Site' => 'Santa Clara' });
```

Infoblox::Session->modify( )

Use this method to modify an object in the Infoblox appliance. See Infoblox::Session->modify() for parameters and return values.

Example

```
# Use method to modify the comment.
$bind_cname->comment("this is a modified comment");
# Submit modification
my $response = $session->modify( $bind_cname );
```

Infoblox::Session->remove( )

Use this method to remove an object from the Infoblox appliance. See Infoblox::Session->remove() for parameters and return values.

To remove a specific object, first use `get()` or `search()` to retrieve the specific object, and then submit this object for removal.

Example

```
#Get the objects with the same name
my @retrieved_objs = $session->get(
  object => "Infoblox::DNS::RPZRecord::CNAME",
  name   => "bind_cname.rpz.com",
  view   => "default");
#Find the desired object from the retrieved list.
my $desired_cname = $retrieved_objs[0];
# Submit for removal
my $response = $session->remove( $desired_cname );
```

Infoblox::Session->search( )

Use this method to search for DNS RPZ CNAME record objects in the Infoblox appliance. See Infoblox::Session->search() for parameters and return values.

Key References

```
Apply the following attributes to search for a specific DNS RPZ CNAME object.

  name    - Optional. A host name in string format (regular expression).
  view    - Optional. The DNS view in which the RPZ CNAME record is located. By default, all DNS
views are searched. But if you omit this attribute and specify a zone, the appliance searches the
'default' view only.
  zone    - Optional. A zone name in FQDN format.
  canonical - Optional. A canonical name (regular expression).
  comment - Optional. A comment in string format (regular expression).
  extattrs     - Optional. A hash reference containing extensible attributes.
  extensible_attributes - Optional. A hash reference containing extensible attributes.
```

For more information about searching extensible attributes, see Infoblox::Grid::ExtensibleAttributeDef/Searching Extensible Attributes.

Example

```
# search for all DNS RPZ CNAME objects that match "rpz.com" in the default DNS view
my @retrieved_objs = $session->search(
    object  => "Infoblox::DNS::RPZRecord::CNAME",
    name    => 'rpz\.com',
    view    => "default",
    comment => "this is a modified comment" );

# search for all DNS RPZ CNAME record in zone "rpz.com" of default view
my @retrieved_objs = $session->search(
    object => "Infoblox::DNS::RPZRecord::CNAME",
    zone   => "rpz\.com",
    view   => "default" );

# search DNS RPZ CNAME record with canonical name "cname.rpz.com" of default view
my @retrieved_objs = $session->search(
    object => "Infoblox::DNS::RPZRecord::CNAME",
    canonical => "cname\.domain\.com",
    view   => "default" );

# search all DNS RPZ CNAME recods with the extensible attribute 'Site'
my @retrieved_objs = $session->search(
   object => "Infoblox::DNS::RPZRecord::CNAME",
   extensible_attributes => { 'Site' => 'Santa Clara' });
```

METHODS

This section describes all the methods that you can use to configure and retrieve the attribute values of a RPZ CNAME record.

canonical( )

Use this method to set or retrieve the canonical name.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

Canonical name can be empty, asterisk or in FQDN (Fully Qualified Domain Name) format. The FQDN consists of the hostname followed by the domain name (example: abc.com). A hostname can have a maximum of 256 bytes. If name of object starts with '*.' you must specify special value 'infoblox-passthru' in order to create Passthru Domain Name Rule.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get canonical
my $canonical = $bind_cname->canonical();
#Modify canonical name
$bind_cname->canonical("bind_cname.domain.com");
```

comment( )

Use this method to set or retrieve the descriptive comment.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

Desired comment in string format with a maximum of 256 bytes.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get comment
my $comment = $bind_cname->comment();
#Modify comment
$bind_cname->comment("Modifying the DNS RPZ CNAME comment");
```

disable( )

Use this method to set or retrieve the disable flag of a DNS record.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

The default value for this field is false. The DNS record is enabled.

Parameter

Specify "true" to set the disable flag or "false" to deactivate/unset it.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get disable
my $disable = $bind_cname->disable();
#Modify disable
$bind_cname->disable("true");
```

extattrs( )

Use this method to set or retrieve the extensible attributes associated with a DNS RPZ CNAME record object.

Parameter

Valid value is a hash reference containing the names of extensible attributes and their associated values (Infoblox::Grid::Extattr objects).

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get extattrs
my $ref_extattrs = $bind_cname->extattrs();
#Modify extattrs
```

```
$bind_cname->extattrs({ 'Site' => $extattr1, 'Administrator' => $extattr2 });
```

extensible_attributes( )

Use this method to set or retrieve the extensible attributes associated with a DNS RPZ CNAME record.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

For valid values for extensible attributes, see Infoblox::Grid::ExtensibleAttributeDef/Extensible Attribute Values.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get extensible attributes
my $ref_extensible_attributes = $bind_cname->extensible_attributes();
#Modify extensible attributes
$bind_cname->extensible_attributes({ 'Site' => 'Santa Clara', 'Administrator' => [ 'Peter', 'Tom' ]
});
```

name( )

Use this method to set or retrieve the host name.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

A name in FQDN (Fully Qualified Domain Name) format. The FQDN consists of the host_name followed by the domain name (example: abc.com). A host_name can have a maximum of 256 bytes.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get name
my $name = $bind_cname->name();
#Modify name
$bind_cname->name("bind_cname2.rpz.com");
```

ttl( )

Use this method to set or retrieve the Time to Live (TTL) value.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

The default value is undefined which indicates that the record inherits the TTL value of the zone.

Specify a TTL value to override the TTL value at the zone level.

Parameter

A 32-bit integer (range from 0 to 4294967295) that represents the duration, in seconds, that the record is cached. Zero indicates that the record should not be cached.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get ttl
my $ttl = $bind_cname->ttl();
#Modify ttl
$bind_cname->ttl(1800);
#Un-override ttl
$bind_cname->ttl(undef);
```

views( )

Use this method to set or retrieve the view of the RPZ CNAME record.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

The default value is the "default" view, which means the RPZ CNAME record is located under the default view.

Parameter

An array reference of defined Infoblox::DNS::View objects.

Note that the array size must be 1.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get views
my $ref_views = $bind_cname->views();
#Modify views, an array of Infoblox::DNS::View objects
$bind_cname->views([$view1]);
```

zone( )

Use this method to retrieve the zone name of a DNS RPZ CNAME record. This method is read-only and cannot be set.

Parameter

None

Returns

Returns the attribute value.

Example

```
# Get zone
my $zone = $bind_cname->zone();
```

rp_zone( )

Use this method to set or retrieve the zone object of a DNS RPZ CNAME record.

Parameter

An Infoblox::DNS::Zone object.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
# Get rp_zone
my $rp_zone = $rpz_cname->rp_zone();
#Modify rp_zone, reference of Infoblox::DNS::Zone object
$rpz_cname->rp_zone($response_policy_zone);
```

SAMPLE CODE

The following sample code demonstrates the different functions that can be applied to an object, such as add, search, modify, and remove. This sample code also includes error handling for the operations.

#Preparation prior to a DNS RPZ CNAME record insertion

```
#PROGRAM STARTS: Include all the modules that will be used
use strict;
use Infoblox;

#Create a session to the Infoblox appliance
```

```
my $session = Infoblox::Session->new(
    master   => "192.168.1.2",
    username => "admin",
    password => "infoblox"
);
unless ($session) {
   die("Construct session failed: ",
       Infoblox::status_code() . ":" . Infoblox::status_detail());
}
print "Session created successfully\n";

#Create the zone prior to a RPZ CNAME record insertion
my $zone = Infoblox::DNS::Zone->new(name => "rpz.com",
                                    rpz_policy => "GIVEN");
unless ($zone) {
die("Construct zone failed: ",
    Infoblox::status_code() . ":" . Infoblox::status_detail());
}
print "Zone object created successfully\n";

#Verify if the zone exists
my $object = $session->get(object => "Infoblox::DNS::Zone", name => "rpz.com");
unless ($object) {
print "Zone does not exist on server, safe to add the zone\n";
$session->add($zone)
   or die("Add zone failed: ",
          $session->status_code() . ":" . $session->status_detail());
}
```

#Create a DNS RPZ CNAME record

```
my $view1 = Infoblox::DNS::View->new(
 name      => "default" );

#Construct a DNS RPZ CNAME object
my $bind_cname = Infoblox::DNS::RPZRecord::CNAME->new(
 name      => "bind_cname.rpz.com",
 canonical => "cname.domain.com",
 comment   => "this is a demo bind_cname record",
 ttl       => "33",
 views     => [$view1],
 rp_zone   => $zone);
unless ($bind_cname) {
die("Construct DNS record RPZ CNAME failed: ",
    Infoblox::status_code() . ":" . Infoblox::status_detail());
}
print "DNS RPZ CNAME object created successfully\n";

#Add the DNS RPZ CNAME record object to the Infoblox appliance through a session
$session->add($bind_cname)
 or die("Add record cname failed: ",
        $session->status_code() . ":" . $session->status_detail());
print "DNS RPZ CNAME object added to server successfully\n";
```

#Search for a specific DNS RPZ CNAME record

```
#Search all RPZ CNAME records that match "rpz.com"
my @retrieved_objs = $session->search(
 object => "Infoblox::DNS::RPZRecord::CNAME",
 name   => 'rpz\.com'
);
my $object = $retrieved_objs[0];
unless ($object) {
 die("Search record RPZ CNAME failed: ",
     $session->status_code() . ":" . $session->status_detail());
}
print "Search DNS RPZ CNAME object found at least 1 matching entry\n";

#Search all RPZ CNAME records that start with "bind" and end with ".rpz.com"
my @retrieved_objs = $session->search(
 object => "Infoblox::DNS::RPZRecord::CNAME",
 name   => '^bind.*\.rpz\.com'
);
my $object = $retrieved_objs[0];
unless ($object) {
 die("Search record RPZ CNAME failed: ",
```

```
      $session->status_code() . ":" . $session->status_detail());
}
print "Search DNS RPZ CNAME object using regexp found at least 1 matching entry\n";
```

#Get and modify a DNS RPZ CNAME record

```
#Get CNAME record through the session
my @retrieved_objs = $session->get(
 object => "Infoblox::DNS::RPZRecord::CNAME",
 name   => "bind_cname.rpz.com",
 view   => "default"
);
my $object = $retrieved_objs[0];
unless ($object) {
 die("Get record RPZ CNAME failed: ",
      $session->status_code() . ":" . $session->status_detail());
}
print "Get DNS RPZ CNAME object found at least 1 matching entry\n";

#Modify one of the attributes of the specified RPZ CNAME record
$object->disable("true");

#Apply the changes
$session->modify($object)
 or die("Modify record RPZ CNAME failed: ",
        $session->status_code() . ":" . $session->status_detail());
print "DNS RPZ CNAME object modified successfully \n";
```

#Remove a DNS RPZ CNAME record

```
#Get CNAME record through the session
my @retrieved_objs = $session->get(
 object => "Infoblox::DNS::RPZRecord::CNAME",
 name   => "bind_cname.rpz.com",
 view   => "default"
);
$object = $retrieved_objs[0];
unless ($object) {
 die("Get record RPZ CNAME failed: ",
      $session->status_code() . ":" . $session->status_detail());
}
print "Get DNS RPZ CNAME object found at least 1 matching entry\n";

#Submit the object for removal
$session->remove( $object )
or die("Remove record RPZ CNAME failed: ",
      $session->status_code() . ":" . $session->status_detail());
print "DNS RPZ CNAME object removed successfully \n";

####PROGRAM ENDS####
```

AUTHOR

Infoblox Inc. http://www.infoblox.com/

SEE ALSO

Infoblox::DNS::View, Infoblox::DNS::Zone, Infoblox::Session, Infoblox::Session->get(), Infoblox::Session->search(), Infoblox::Session->add(), Infoblox::Session->remove(), Infoblox::Session->modify()

COPYRIGHT

Copyright (c) 2017 Infoblox Inc.

Infoblox::Grid::ThreatProtection::NATPort - a NAT Threat Protection Port object.

- NAME
- DESCRIPTION
- CONSTRUCTION
- SESSION METHODS
- MODULE METHODS
  - Infoblox::Grid::ThreatProtection::NATRule->nat_ports( )
- METHODS
  - block_size( )
  - end_port( )
  - start_port( )
- AUTHOR
- SEE ALSO
- COPYRIGHT

NAME

Infoblox::Grid::ThreatProtection::NATPort - a NAT Threat Protection Port object.

DESCRIPTION

The Threat Protection NAT Port object provides information about the port blocks configured for NAT mapping.

CONSTRUCTION

```
my $nat_port = Infoblox::Grid::ThreatProtection::NATPort->new(
   start_port => $uint, #Required
   end_port   => $uint, #Required
   block_size => $uint, #Required
);
```

SESSION METHODS

The object does not support any session methods.

MODULE METHODS

The following functions are available to apply to a NAT Port object.

Infoblox::Grid::ThreatProtection::NATRule->nat_ports( )

Use this method to set or retrieve NAT Port configuration for the NAT Threat Protection Rule. See Infoblox::Grid::ThreatProtection::NATRule->nat_ports() for parameters and return values.

Example

```
#Get nat_ports
my $nat_ports = $nat_rule->nat_ports();

#Modify nat_ports
$nat_rule->nat_ports($nat_ports);
```

METHODS

This section describes all the methods that you can use to configure and retrieve the attribute values of a Threat Protection NAT Port.

block_size( )

Use this method to set or retrieve the block size for the NAT Port configuration object.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

The valid value is an unsigned integer.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get attribute value
my $value = $object->block_size();

#Modify attribute value
$object->block_size(20);
```

end_port( )

Use this method to set or retrieve the end port value for the NAT port configuration object.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

The valid value is an unsigned integer.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get attribute value
my $value = $object->end_port();

#Modify attribute value
$object->end_port(10);
```

start_port( )

Use this method to set or retrieve the start port value for the NAT port configuration object.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

The valid value is an unsigned integer.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get attribute value
my $value = $object->start_port();

#Modify attribute value
$object->start_port(5);
```

AUTHOR

Infoblox Inc. http://www.infoblox.com/

SEE ALSO

Infoblox::Grid::ThreatProtection::NATRule, Infoblox::Grid::ThreatProtection::NATRule->nat_ports()

COPYRIGHT

Copyright (c) 2017 Infoblox Inc.

Infoblox::Grid::ThreatProtection::RuleTemplate - Grid Threat Protection Rule Template object.

- NAME
- DESCRIPTION
- CONSTRUCTOR
- SESSION METHODS
  - Infoblox::Session->get( )
  - Infoblox::Session->search( )
- METHODS
  - allowed_actions( )
  - category( )
  - default_config( )
  - description( )
  - name( )
  - ruleset( )
  - sid( )
  - versions( )
- AUTHOR
- SEE ALSO
- COPYRIGHT

NAME

Infoblox::Grid::ThreatProtection::RuleTemplate - Grid Threat Protection Rule Template object.

DESCRIPTION

This object provides information about the Grid Threat protection rule template settings.

CONSTRUCTOR

```
my $object = Infoblox::Grid::ThreatProtection::RuleTemplate->new(
   'name' => $string,        # Required
);
```

SESSION METHODS

The following functions can be applied to a Grid::ThreatProtection::RuleTemplate object.

Infoblox::Session->get( )

Use this method to retrieve all the matching objects from the Infoblox appliance. See Infoblox::Session->get() for parameters and return values.

Key References

```
Apply the following attributes to get a specific Grid Threat Protection RuleTemplate object.
 sid    - Optional. The Rule ID.
 name   - Optional. The name of the Threat protection rule template.
```

Example

```
my @retrieved_objs = $session->get(
    'object' => 'Infoblox::Grid::ThreatProtection::RuleTemplate',
    'name' => 'Whitelist source IP network',
);
```

Infoblox::Session->search( )

Use this method to search for objects in the Infoblox appliance. See Infoblox::Session->search() for parameters and return values.

Key References

```
Apply the following attributes to get a specific Grid Threat Protection::RuleTemplate object.
 sid     - Optional. The Rule ID.
 name    - Optional. The name of the Threat protection rule template.
 ruleset - Optional. An
L<Infoblox::Grid::ThreatProtection::Ruleset|Infoblox::Grid::ThreatProtection::Ruleset> object
representing a reference to the ruleset object.
```

Example

```
my @retrieved_objs = $session->search(
    'object' => 'Infoblox::Grid::ThreatProtection::RuleTemplate',
    'name' => 'Whitelist source IP network',
);
```

METHODS

allowed_actions( )

Use this method to retrieve allowed actions for the rule template. This is a read-only attribute.

Parameter

None

Returns

The method returns the attribute value.

Example

```
# Get attribute value
my $value = $object->allowed_actions();
```

category( )

Use this method to retrieve reference to the rule category. This is a read-only attribute.

Parameter

None

Returns

The method returns the attribute value.

Example

```
# Get attribute value
my $value = $object->category();
```

default_config( )

Use this method to retrieve default rule configuration information. This is a read-only attribute.

Parameter

None

Returns

The method returns the attribute value.

Example

```
# Get attribute value
my $value = $object->default_config();
```

description( )

Use this method to retrieve the description for the rule template. This is a read-only attribute.

Parameter

None

Returns

The method returns the attribute value.

Example

```
# Get attribute value
my $value = $object->description();
```

name( )

Use this method to set or retrieve the name for the rule template. This is a read-only attribute if the object was retrieved from the server.

Parameter

The name in string format.

Returns

The method returns the attribute value.

Example

```
# Get attribute value
my $value = $object->name();
# Modify attribute value
$object->name('Whitelist source IP network');
```

ruleset( )

Use this method to retrieve a reference to the ruleset object. This is a read-only attribute.

Parameter

None

Returns

An Infoblox::Grid::ThreatProtection::Ruleset object.

Example

```
#Get ruleset value
my $value = $object->ruleset();
```

sid( )

Use this method to retrieve the Rule ID. This is a read-only attribute.

Parameter

None

Returns

The method returns the attribute value.

Example

```
# Get attribute value
my $value = $object->sid();
```

versions( )

Use this method to retrieve information about the rule version. This is a read-only attribute.

Parameter

None

Returns

The method returns the attribute value.

Example

```
# Get attribute value
my $value = $object->versions();
```

AUTHOR

Infoblox Inc. http://www.infoblox.com/

SEE ALSO

Infoblox::Session, Infoblox::Grid::ThreatProtection, Infoblox::Grid::ThreatProtection::Rule, Infoblox::Grid::ThreatProtection::RuleConfig, Infoblox::Grid::ThreatProtection::RuleParam, Infoblox::Grid::ThreatProtection::Ruleset

---

COPYRIGHT

Copyright (c) 2017 Infoblox Inc.

Infoblox::Grid::Member::ThreatProtection::Rule - Member Threat Protection Rule object.

- NAME
- DESCRIPTION
- SESSION METHODS
  - Infoblox::Session->get( )
  - Infoblox::Session->modify( )
  - Infoblox::Session->search( )
- METHODS
  - config( )
  - disable( )
  - member( )
  - override_config( )
  - override_disable( )
  - rule( )
  - sid( )
- AUTHOR
- SEE ALSO
- COPYRIGHT

NAME

Infoblox::Grid::Member::ThreatProtection::Rule - Member Threat Protection Rule object.

DESCRIPTION

This object provides information about the member Threat protection rule settings.

SESSION METHODS

The following functions can be applied to a Grid::Member::ThreatProtection::Rule object.

Infoblox::Session->get( )

Use this method to retrieve all the matching objects from the Infoblox appliance. See Infoblox::Session->get() for parameters and return values.

Key References

```
Apply the following attributes to get a specific Member Threat Protection::Rule object.
 sid    - Optional. The Rule ID.
 rule   - Optional. The Threat Protection Rule object.
 member - Optional. The Member Threat Protection object.
```

Example

```
my @retrieved_objs = $session->get(
    'object' => 'Infoblox::Grid::Member::ThreatProtection::Rule',
    'sid' => '211111'
);
```

Infoblox::Session->modify( )

Use this method to modify an object in the Infoblox appliance. See Infoblox::Session->modify() for parameters and return values.

Example

```
 my $rule = $session->search('object' => 'Infoblox::Grid::ThreatProtection::Rule', 'sid' =>
'211111');
 my $member = $session->get('object' => 'Infoblox::Grid::Member::ThreatProtection');
 my $member_rule = $session->search('object' => 'Infoblox::Grid::Member::ThreatProtection::Rule',
'member' => $member, 'rule' => $rule);

 my $conf = $member_rule->config();
 $conf->action('PASS');

 # Use this method to modify attributes of object.
 $member_rule->disable('true');
 $member_rule->override_disable('true');
 $member_rule->config($conf);
 $member_rule->override_config('true');
```

```
# Submit modification
my $response = $session->modify( $member_rule );
```

Infoblox::Session->search( )

Use this method to search for objects in the Infoblox appliance. See Infoblox::Session->search() for parameters and return values.

Key References

```
Apply the following attributes to get a specific Grid Threat Protection::Rule object.
 sid    - Optional. The Rule ID.
 rule   - Optional. The Threat Protection Rule object.
 member - Optional. The Member Threat Protection object.
```

Example

```
my $rule = $session->search('object' => 'Infoblox::Grid::ThreatProtection::Rule', 'sid' =>
'211111');
my $member = $session->get('object' => 'Infoblox::Grid::Member::ThreatProtection');
my $member_rule = $session->search('object' => 'Infoblox::Grid::Member::ThreatProtection::Rule',
    'member' => $member,
    'rule' => $rule,
);
```

METHODS

config( )

Use this method to set or retrieve rule configuration information.

Setting this method to a defined value implicitly sets the override_config method to "true". Setting the parameter to undefined causes the appliance to use the Grid default and automatically resets the override_config attribute to "false".

Note that when config is set to a defined value and override_config is set to "false", the last operation takes precedence. Thus the sequence $object->config($value); $object->override_config("false"); will set override_config to "false", and the sequence $object->override_config("false"); $object->config($value); will result in override_config="true".

Parameter

Valid value is reference on Infoblox::Grid::ThreatProtection::RuleConfig object.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
# Get attribute value
my $value = $object->config();
$value->log_severity('INFORMATIONAL');
# Modify attribute value
$object->config($value);
```

disable( )

Use this method to set or indicate whether the rule is enabled or disabled for the member.

Setting this method to a defined value implicitly sets the override_disable method to "true". Setting the parameter to undefined causes the appliance to use the Grid default and automatically resets the override_disable attribute to "false".

Note that when disable is set to a defined value and override_disable is set to "false", the last operation takes precedence. Thus the sequence $object->disable('true'); $object->override_disable("false"); will set override_disable to "false", and the sequence $object->override_disable("false"); $object->disable('true'); will result in override_disable="true".

Parameter

Valid value is "true" or "false".

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
# Get attribute value
my $value = $object->disable();
# Modify attribute value
$object->disable(true);
```

member( )

Use this method to retrieve reference to the member object. This is a read-only attribute.

Parameter

None

Returns

The method returns the attribute value.

Example

```
# Get attribute value
my $value = $object->member();
```

override_config( )

Use this method to set or retrieve the override flag for the rule configuration.

Parameter

Valid value is "true" or "false".

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
# Get attribute value
my $value = $object->override_config();
# Modify attribute value
$object->override_config(true);
```

override_disable( )

Use this method to set or retrieve the override flag for the disabled field.

Parameter

Valid value is "true" or "false".

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
# Get attribute value
my $value = $object->override_disable();
# Modify attribute value
$object->override_disable(true);
```

rule( )

Use this method to retrieve reference to the rule object. This is a read-only attribute.

Parameter

None

Returns

The method returns the attribute value.

Example

```
# Get attribute value
my $value = $object->rule();
```

sid( )

Use this method to retrieve the Rule ID. This is a read-only attribute.

Parameter

None

Returns

The method returns the attribute value.

Example

```
# Get attribute value
my $value = $object->sid();
```

AUTHOR

Infoblox Inc. http://www.infoblox.com/

SEE ALSO

Infoblox::Session, Infoblox::Grid::Member::ThreatProtection, Infoblox::Grid::ThreatProtection::Rule, Infoblox::Grid::ThreatProtection::RuleConfig, Infoblox::Grid::ThreatProtection::RuleParam

---

COPYRIGHT

Copyright (c) 2017 Infoblox Inc.

Infoblox::Grid::ThreatProtection::Statistics - Threat protection statistics.

- NAME
- DESCRIPTION
- SESSION METHODS
  - Infoblox::Session->get( )
  - Infoblox::Session->search( )
- METHODS
  - stat_infos( )
  - grid_member( )
- AUTHOR
- SEE ALSO
- COPYRIGHT

NAME

Infoblox::Grid::ThreatProtection::Statistics - Threat protection statistics.

DESCRIPTION

This object provides information about Threat protection statistics.

SESSION METHODS

The following functions can be applied to a Grid::ThreatProtection::Statistics object.

Infoblox::Session->get( )

Use this method to retrieve all the matching objects from the Infoblox appliance. See Infoblox::Session->get() for parameters and return values.

Key References

```
Apply the following attributes to get a specific Threat Protection Statistics object.
 grid_member   - Optional. The Grid member name.
```

Example

```
my @retrieved_objs = $session->get(
    'object'      => 'Infoblox::Grid::ThreatProtection::Statistics',
    'grid_member' => 'member.com',
);
```

Infoblox::Session->search( )

Use this method to search for objects in the Infoblox appliance. See Infoblox::Session->search() for parameters and return values.

Key References

```
Apply the following attributes to get a specific Threat Protection Statistics object.
 grid_member   - Optional. The Grid member name.
```

Example

```
my @retrieved_objs = $session->search(
    'object'      => 'Infoblox::Grid::ThreatProtection::Statistics',
    'grid_member' => 'member.com',
);
```

METHODS

stat_infos( )

Use this method to retrieve event statistics. This is a read-only attribute.

Parameter

None

Returns

The method returns the attribute value that can be an array reference of Infoblox::Grid::ThreatProtection::StatInfo objects.

Example

```
# Get attribute value
my $value = $object->stat_infos();
```

grid_member( )

Use this method to retrieve a Grid member name. This is a read-only attribute.

Parameter

None

Returns

The method returns the attribute value.

Example

```
# Get attribute value
my $value = $object->grid_member();
```

AUTHOR

Infoblox Inc. http://www.infoblox.com/

SEE ALSO

Infoblox::Session, Infoblox::Grid::ThreatProtection, Infoblox::Grid::ThreatProtection::Rule, Infoblox::Grid::ThreatProtection::StatInfo

---

COPYRIGHT

Copyright (c) 2017 Infoblox Inc.

Infoblox::Grid::ThreatProtection::StatInfo - Statistics information.

- NAME
- DESCRIPTION
- SESSION METHODS
- METHODS
  - critical( )
  - informational( )
  - major( )
  - timestamp( )
  - warning( )
- AUTHOR
- SEE ALSO
- COPYRIGHT

NAME

Infoblox::Grid::ThreatProtection::StatInfo - Statistics information.

DESCRIPTION

This is an in-memory struct.

SESSION METHODS

The object does not support any session methods.

METHODS

This section describes all the methods that you can use to set or retrieve the attribute values of the struct.

critical( )

Use this method to retrieve the number of critical events. This is a read-only attribute.

Parameter

None

Returns

The method returns the attribute value.

Example

```
# Get attribute value
my $value = $object->critical();
```

informational( )

Use this method to retrieve the number of informational events. This is a read-only attribute.

Parameter

None

Returns

The method returns the attribute value.

Example

```
# Get attribute value
my $value = $object->informational();
```

major( )

Use this method to retrieve the number of major events. This is a read-only attribute.

Parameter

None

Returns

The method returns the attribute value.

Example

```
# Get attribute value
my $value = $object->major();
```

timestamp( )

Use this method to retrieve the timestamp when the following data is being collected. This is a read-only attribute.

Parameter

None

Returns

The method returns the attribute value.

Example

```
# Get attribute value
my $value = $object->timestamp();
```

warning( )

Use this method to retrieve the number of warning events. This is a read-only attribute.

Parameter

None

Returns

The method returns the attribute value.

Example

```
# Get attribute value
my $value = $object->warning();
```

AUTHOR

Infoblox Inc. http://www.infoblox.com/

SEE ALSO

Infoblox::Grid::ThreatProtection::Rule, Infoblox::Grid::ThreatProtection::Statistics

---

COPYRIGHT

Copyright (c) 2017 Infoblox Inc.

Infoblox::Grid::UpgradeSchedule - Information about the Grid's upgrade schedule.

- NAME
- DESCRIPTION
- CONSTRUCTOR
- SESSION METHODS
  - Infoblox::Session->get( )
  - Infoblox::Session->modify( )
- METHODS
  - active( )
  - groups( )
  - start_time( )
  - time_zone( )
- AUTHOR
- SEE ALSO
- COPYRIGHT

NAME

Infoblox::Grid::UpgradeSchedule - Information about the Grid's upgrade schedule.

DESCRIPTION

The Upgrade Schedule object is used to set and view the upgrade schedule of an Infoblox Grid.

CONSTRUCTOR

The Infoblox::Grid::UpgradeSchedule object is automatically generated. It does not require manual construction.

SESSION METHODS

This section describes all the methods in an Infoblox::Session that can be applied to an Upgrade Schedule object.

Infoblox::Session->get( )

Use this method to retrieve the existing objects from an Infoblox appliance. See Infoblox::Session->get() for parameters and return values.

Example

```
my $schedule = $session->get(object => "Infoblox::Grid::UpgradeSchedule");
```

Infoblox::Session->modify( )

Use this method to modify objects in the Infoblox appliance. See Infoblox::Session->modify() for parameters and return values.

Example

```
# Submit modification
my $response = $session->modify( $schedule );
```

METHODS

This section describes all the methods that can be used to set or retrieve the attribute values of an Upgrade Schedule object.

active( )

Use this method to set or retrieve the "active" flag.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

Specify "true" to set the active flag or "false" to deactivate/unset it. The default value is "false".

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get active
my $active = $schedule->active();
#Modify active
$schedule->active("true");
```

groups( )

Use this method to set or retrieve the list of groups affected by this upgrade schedule.

Parameter

Array reference that contains each Infoblox::Grid::UpgradeGroup object.

Note that the members of this list must have their upgrade_time member set in order for the modification to succeed. This means that it is possible to retrieve an upgrade schedule but not be allowed to modify it without setting the upgrade_time values for the groups.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

Example

```
#Get the group list
my $grouplist = $schedule->groups();
#Modify it
$grid->groups([$ug1, $ug2, $ug3 ]);
```

start_time( )

Use this method to set or retrieve the start time of the upgrade.

Parameter

The start time of the upgrade in ISO 8601 extended format (e.g. 2008-10-21T10:56:00Z or 2008-10-21T10:56:00.001Z).

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get start_time
my $start_time = $schedule->start_time();
#Modify start_time
$schedule->start_time("2020-12-21T11:22:33Z");
```

time_zone( )

Use this method to set or retrieve the time zone of the upgrade schedule.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

The UTC string that represents the time zone. For example "(UTC - 6:00) Central Time (US and Canada)". Default value is undefined.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get time_zone
my $time_zone = $schedule->time_zone();
#Modify time_zone
$schedule->time_zone("(UTC - 6:00) Central Time (US and Canada)");
```

AUTHOR

Infoblox Inc. http://www.infoblox.com/

SEE ALSO

Infoblox::Session, Infoblox::Session->get()

COPYRIGHT

Copyright (c) 2017 Infoblox Inc.

Infoblox::Grid::DistributionSchedule - Information about the Grid's distribution schedule.

- NAME
- DESCRIPTION
- CONSTRUCTOR
- SESSION METHODS
  - Infoblox::Session->get( )
  - Infoblox::Session->modify( )
- METHODS
  - active( )
  - groups( )
  - start_time( )
  - time_zone( )
- AUTHOR
- SEE ALSO
- COPYRIGHT

NAME

Infoblox::Grid::DistributionSchedule - Information about the Grid's distribution schedule.

DESCRIPTION

The Distribution Schedule object is used to set and view the distribution schedule of an Infoblox Grid.

CONSTRUCTOR

The Infoblox::Grid::DistributionSchedule object is automatically generated. It does not require manual construction.

SESSION METHODS

This section describes all the methods in an Infoblox::Session that can be applied to a Distribution Schedule object.

Infoblox::Session->get( )

Use this method to retrieve the existing objects from an Infoblox appliance. See Infoblox::Session->get() for parameters and return values.

Example

```
my $schedule = $session->get(object => "Infoblox::Grid::DistributionSchedule");
```

Infoblox::Session->modify( )

Use this method to modify objects in the Infoblox appliance. See Infoblox::Session->modify() for parameters and return values.

Example

```
# Submit modification
my $response = $session->modify( $schedule );
```

METHODS

This section describes all the methods that can be used to set or retrieve the attribute values of a Distribution Schedule object.

active( )

Use this method to set or retrieve the "active" flag.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

Specify "true" to set the active flag or "false" to deactivate/unset it. The default value is "false".

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get active
my $active = $schedule->active();
#Modify active
$schedule->active("true");
```

groups( )

Use this method to set or retrieve the list of groups affected by this distribution schedule.

Parameter

Array reference that contains each Infoblox::Grid::Upgradegroup object.

Note that the members of this list must have their distribution_time parameter set in order for the modification to succeed. This means that it is possible to retrieve a distribution schedule but not be allowed to modify it without setting the distribution_time values for the groups.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

Example

```
#Get the group list
my $grouplist = $schedule->groups();
#Modify it
$grid->groups([$ug1, $ug2, $ug3 ]);
```

start_time( )

Use this method to set or retrieve the start time of the distribution.

Parameter

The start time of the distribution in ISO 8601 extended format (e.g. 2008-10-21T10:56:00Z or 2008-10-21T10:56:00.001Z).

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get start_time
my $start_time = $schedule->start_time();
#Modify start_time
$schedule->start_time("2020-12-21T11:22:33Z");
```

time_zone( )

Use this method to set or retrieve the time zone of the distribution schedule.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

The UTC string that represents the time zone. For example "(UTC - 6:00) Central Time (US and Canada)". Default value is undefined.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get time_zone
my $time_zone = $schedule->time_zone();
#Modify time_zone
$schedule->time_zone("(UTC - 6:00) Central Time (US and Canada)");
```

AUTHOR

Infoblox Inc. http://www.infoblox.com/

SEE ALSO

Infoblox::Session, Infoblox::Session->get()

COPYRIGHT

Copyright (c) 2017 Infoblox Inc.

Infoblox::Grid::UpgradeGroup::Member - Upgrade Group Member object

- NAME
- DESCRIPTION
- CONSTRUCTOR
- MODULE METHODS
  - Infoblox::Grid::UpgradeGroup->members( )
- METHODS
  - member( )
  - time_zone( )
- AUTHOR
- SEE ALSO
- COPYRIGHT

NAME

Infoblox::Grid::UpgradeGroup::Member - Upgrade Group Member object

DESCRIPTION

This object is used to represent Upgrade Group Grid members.

CONSTRUCTOR

```
my $member = Infoblox::Grid::UpgradeGroup::Member->new(
    member          => $string, # Required, name of the member
);
```

MODULE METHODS

The following functions are available to apply to an Upgrade Group Member object.

Infoblox::Grid::UpgradeGroup->members( )

Use this function to specify the members that belong to this upgrade group. See Infoblox::Grid::UpgradeGroup->members() for parameters and return values.

Example

```
my $member1 = Infoblox::Grid::UpgradeGroup::Member->new(
                                                        member => 'hostname.com',
                                                       );
unless($member1) {
     die("Construct member failed: ",
           Infoblox::status_code() . ":" . Infoblox::status_detail());
}
print "Member object created successfully\n";

#Create the upgrade group object with this member
my $sgu = Infoblox::Grid::UpgradeGroup->new(
                                            name                          => "groupname1",
                                            comment                       => "group name 1 comment",
                                            distribution_policy           => "sequentially",
                                            upgrade_policy                => "simultaneously",
                                            distribution_dependent_group  => 'distgroup1',
                                            upgrade_dependent_group       => 'distgroup2',
                                            distribution_time             => '2020-12-22T11:22:33Z',
                                            upgrade_time                  => '2021-12-22T11:22:33Z',
                                            members                       => [$member1],
                                            );

my $response = $session->add($sgu)
unless($response) {
     die("Add upgrade group failed: ",
           session->status_code() . ":" . session->status_detail());
}
print "Upgrade group created successfully\n";
```

METHODS

This section describes all the methods that you can use to set and retrieve the attribute values of an Upgrade Group Member object.

member( )

Use this method to set or retrieve the name of the member.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

The name of the grid member.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get member
my $member = $member->member();
#Modify member
$member->member("192.168.1.3");
```

time_zone( )

Use this method to retrieve the time zone of the member.

Parameter

none

Returns

The method returns the attribute value.

Example

```
#Get time_zone
my $time_zone = $member->time_zone();
```

AUTHOR

Infoblox Inc. http://www.infoblox.com/

SEE ALSO

Infoblox::Grid::UpgradeGroup

COPYRIGHT

Copyright (c) 2017 Infoblox Inc.

Infoblox::Grid::UpgradeGroup - Upgrade Group object.

- NAME
- DESCRIPTION
- CONSTRUCTOR
- SESSION METHODS
  - Infoblox::Session->add( )
  - Infoblox::Session->get( )
  - Infoblox::Session->modify( )
  - Infoblox::Session->remove( )
  - Infoblox::Session->search( )
  - Infoblox::Grid::UpgradeSchedule->groups( )
  - Infoblox::Grid::DistributionSchedule->groups( )
- METHODS
  - comment( )
  - distribution_policy( )
  - distribution_dependent_group( )
  - distribution_time( )
  - members( )
  - name( )
  - time_zone( )
  - upgrade_dependent_group( )
  - upgrade_policy( )
  - upgrade_time( )
- SAMPLE CODE
- AUTHOR
- SEE ALSO
- COPYRIGHT

NAME

Infoblox::Grid::UpgradeGroup - Upgrade Group object.

DESCRIPTION

This object enables the configuration of upgrade groups for the Infoblox Grid.

CONSTRUCTOR

```
 my $group = Infoblox::Grid::UpgradeGroup->new(
         name                          => $string, #Required
         comment                       => $string, #Optional / Default is undefined
         distribution_policy           => "simultaneously" | "sequentially", #Optional / Default is
simultaneously
         upgrade_policy                => "simultaneously" | "sequentially", #Optional / Default is
simultaneously
         distribution_dependent_group  => $string, #Optional / Default is undefined
         upgrade_dependent_group       => $string, #Optional / Default is undefined
         distribution_time             => $timestamp, #See below
         upgrade_time                  => $timestamp, #See below
         members                       => [$m1, $m2, ...], #Optional / Default is undefined
 );
```

SESSION METHODS

This section describes all the methods in the Infoblox::Session module that you can apply to an Upgrade Group object.

Infoblox::Session->add( )

Use this method to add an object to the Infoblox appliance. See Infoblox::Session->add() for parameters and return values.

Example

```
       my $member1 = Infoblox::Grid::UpgradeGroup::Member->new(
                                                              member => '192.168.1.2',
                                                             );
       unless($member1) {
            die("Construct member failed: ",
                  Infoblox::status_code() . ":" . Infoblox::status_detail());
```

```
}
print "Member object created successfully\n";

#Create the upgrade group object with this member
my $sgu = Infoblox::Grid::UpgradeGroup->new(
                                            name                          => "groupname1",
                                            comment                       => "group name 1 comment",
                                            distribution_policy           => "sequentially",
                                            upgrade_policy                => "simultaneously",
                                            distribution_dependent_group  => 'distgroup1',
                                            upgrade_dependent_group       => 'distgroup2',
                                            distribution_time             => '2020-12-22T11:22:33Z',
                                            upgrade_time                  => '2021-12-22T11:22:33Z',
                                            members                       => [$member1],
                                            );

my $response = $session->add($sgu)
unless($response) {
     die("Add upgrade group failed: ",
           session->status_code() . ":" . session->status_detail());
}
print "Upgrade group created successfully\n";
```

Infoblox::Session->get( )

Use this method to retrieve all the matching objects from the Infoblox appliance. See Infoblox::Session->get() for parameters and return values.

Key References

```
Apply the following attributes to get a specific Upgrade Group object:

 name          - Optional. Group name in string format.
 comment       - Optional. Comment in string format.
```

Example

```
my @retrieved_objs = $session->get(
    object      => "Infoblox::Grid::UpgradeGroup",
    name        => "testgroup" );
```

Infoblox::Session->modify( )

Use this method to modify an object in the Infoblox appliance. See Infoblox::Session->modify() for parameters and return values.

Example

```
# Use this method to modify the comment.
$upgradegroup->comment("This is a modified comment");
# Submit modification
my $response = $session->modify( $upgradegroup );
```

Infoblox::Session->remove( )

Use this method to remove an object from the Infoblox appliance. See Infoblox::Session->remove() for parameters and return values.

To remove a specific object, use `get()` or `search()` to retrieve the specific object first, and then submit this object for removal.

Example

```
# Get the group object
my @retrieved_objs = $session->get(
    object      => "Infoblox::Grid::UpgradeGroup",
    name        => "testgroup" );
# Find the desired object from the retrieved list.
my $desired_group = $retrieved_objs[0];
# Submit for removal
my $response = $session->remove( $desired_group );
```

Infoblox::Session->search( )

Use this method to search for Upgrade Group objects in the Infoblox appliance. See Infoblox::Session->search() for parameters and return values.

Key References

```
Apply the following attributes to search for a specific Upgrade Group object:
```

```
 name          - Optional. Group name in string format (regular expression).
 comment       - Optional. Comment in string format (regular expression).
```

Example

```
# Search for all Upgrade Group objects that start with "test"
my @retrieved_objs = $session->search(
    object      => "Infoblox::Grid::UpgradeGroup",
    name        => "test.*",
);
```

Infoblox::Grid::UpgradeSchedule->groups( )

Use this function to specify the groups that belong to this upgrade schedule. See Infoblox::Grid::UpgradeSchedule->groups() for parameters and return values.

Infoblox::Grid::DistributionSchedule->groups( )

Use this function to specify the groups that belong to this distribution schedule. See Infoblox::Grid::DistributionSchedule->groups() for parameters and return values.

METHODS

This section describes all the methods that you can use to set and retrieve the attribute values of an upgrade group object.

comment( )

Use this method to set or retrieve the comment.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

Comment in string format with a maximum of 256 bytes. The default value is undefined.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get comment
my $comment = $upgradegroup->comment();
#Modify comment
$upgradegroup->comment("descriptive comment");
```

distribution_policy( )

Use this method to set or retrieve the distribution policy of this Upgrade Group.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

The distribution policy. Valid values are "sequentially" and "simultaneously". The default is "simultaneously".

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get distribution_policy
my $distribution_policy = $upgradegroup->distribution_policy();
#Modify distribution_policy
$upgradegroup->distribution_policy("sequentially");
```

distribution_dependent_group( )

Use this method to set or retrieve the distribution dependent group.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

The name of the distribution dependent group.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get distribution_dependent_group
my $distribution_dependent_group = $upgradegroup->distribution_dependent_group();
#Modify distribution_dependent_group
$upgradegroup->distribution_dependent_group("group2");
```

distribution_time( )

Use this method to set or retrieve the distribution time.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Note this member is inserted only if the Upgrade Group is submitted for modification as part of an Infoblox::Grid::DistributionSchedule update, this member will not be changed if the Upgrade Group is submitted for modification as a stand-alone object.

Note that if distribution_dependent_group is set, distribution_time will be ignored.

Parameter

The start time of the distribution in ISO 8601 extended format (e.g. 2008-10-21T10:56:00Z or 2008-10-21T10:56:00.001Z).

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get distribution_time
my $distribution_time = $upgradegroup->distribution_time();
#Modify distribution_time
$upgradegroup->distribution_time("2008-10-21T10:56:00.001Z");
```

members( )

Use this method to set or retrieve the members of this upgrade group.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

Array reference that contains Infoblox::Grid::UpgradeGroup::Member object. The default value is empty.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get members
my $members = $upgradegroup->members();
#Modify members
$upgradegroup->members([$ug1, $ug2, $ug3]);
```

name( )

Use this method to set or retrieve the name of this upgrade group.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

The name in string format.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get name
my $name = $upgradegroup->name();
#Modify name
$upgradegroup->name("group2");
```

time_zone( )

Use this method to retrieve the time_zone for this upgrade group.

Parameter

None.

Returns

The method returns the attribute value.

Example

```
#Get time_zone
my $time_zone = $upgradegroup->time_zone();
```

upgrade_dependent_group( )

Use this method to set or retrieve the upgrade dependent group.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

The name of the upgrade dependent group.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get upgrade_dependent_group
my $upgrade_dependent_group = $upgradegroup->upgrade_dependent_group();
#Modify upgrade_dependent_group
$upgradegroup->upgrade_dependent_group("group2");
```

upgrade_policy( )

Use this method to set or retrieve the upgrade_policy.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

The upgrade policy. Valid values are "sequentially" and "simultaneously". The default is "simultaneously".

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get upgrade_policy
my $upgrade_policy = $upgradegroup->upgrade_policy();
#Modify upgrade_policy
$upgradegroup->upgrade_policy("sequentially");
```

upgrade_time( )

Use this method to set or retrieve the upgrade time.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Note this member is inserted only if the Upgrade Group is submitted for modification as part of an Infoblox::Grid::UpgradeSchedule update, this member will not be changed if the Upgrade Group is submitted for modification as a stand-alone object.

Parameter

The start time of the upgrade in ISO 8601 extended format (e.g. 2008-10-21T10:56:00Z or 2008-10-21T10:56:00.001Z).

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get upgrade_time
my $upgrade_time = $upgradegroup->upgrade_time();
#Modify upgrade_time
$upgradegroup->upgrade_time("2008-10-21T10:56:00.001Z");
```

SAMPLE CODE

The following sample code demonstrates the different functions that can be applied to an object, such as add, search, modify, and remove. This sample also includes error handling for the operations.

#Preparation prior to an Upgrade Group insertion

```
#PROGRAM STARTS: Include all the modules that will be used
use strict;
use Infoblox;

#Create a session to the Infoblox appliance

my $session = Infoblox::Session->new(
               master   => "127.0.0.1", #appliance host ip
               username => "admin",       #appliance user login
               password => "infoblox"     #appliance password
               );

unless ($session) {
       die("Construct session failed: ",
               Infoblox::status_code() . ":" . Infoblox::status_detail());
}
print "Session created successfully\n";
```

#Create a few Upgrade Group objects

```
my $group1 = Infoblox::Grid::UpgradeGroup->new(
                                          name => "group1",
                                         );

unless ($group1) {
       die("Construct group object failed: ",
               Infoblox::status_code() . ":" . Infoblox::status_detail());
}
print "Group object created successfully\n";

my $group2 = Infoblox::Grid::UpgradeGroup->new(
                                          name => "group2",
                                         );

$session->add($group1)
       or die("Add Upgrade Group object failed: ",
                       $session->status_code() . ":" . $session->status_detail());
print "Upgrade Group object added to server successfully\n";

unless ($group2) {
       die("Construct group object failed: ",
               Infoblox::status_code() . ":" . Infoblox::status_detail());
}
print "Group object created successfully\n";

$session->add($group2)
```

```
        or die("Add Upgrade Group object failed: ",
                        $session->status_code() . ":" . $session->status_detail());
print "Upgrade Group object added to server successfully\n";

# Note that this has to be a valid Grid member name
my $member = Infoblox::Grid::UpgradeGroup::Member->new(
                                                        member      => 'grid_member.com',
                                                       );

my $group3 = Infoblox::Grid::UpgradeGroup->new(
                                               name                          => "group3",
                                               comment                       => "descriptive comment",
                                               distribution_policy           => "sequentially",
                                               upgrade_policy                => "simultaneously",
                                               distribution_dependent_group  => 'group1',
                                               upgrade_dependent_group       => 'group2',
                                               distribution_time             => '2020-12-22T11:22:33Z',
                                               upgrade_time                  => '2021-12-22T11:22:33Z',
                                               members                       => [$member],
                                              );

unless ($group3) {
        die("Construct group object failed: ",
                Infoblox::status_code() . ":" . Infoblox::status_detail());
}
print "Group object created successfully\n";

#Add the upgrade group object to the Infoblox appliance through a session
$session->add($group3)
        or die("Add Upgrade Group object failed: ",
                        $session->status_code() . ":" . $session->status_detail());
print "Upgrade Group object added to server successfully\n";
```

#Search for all groups

```
my @retrieved_objs = $session->search(
                object      => "Infoblox::Grid::UpgradeGroup",
                name        => "group.*",
               );
my $object = $retrieved_objs[0];

unless ($object) {
        die("Search Group object failed: ",
                $session->status_code() . ":" . $session->status_detail());
}
print "Search Upgrade Group objects found at least 1 matching entry\n";
```

#Get and modify a Group object

```
#Get the object from an Infoblox appliance through a session
my @retrieved_objs = $session->get(
            object      => "Infoblox::Grid::UpgradeGroup",
            name        => "group3",
);
my $object = $retrieved_objs[0];

unless ($object) {
        die("Get Group object failed: ",
                $session->status_code() . ":" . $session->status_detail());
}
print "Get Upgrade Group object found at least 1 matching entry\n";

#Modify the upgrade policy of the Group object
$object->upgrade_policy("sequentially");

#Apply the changes.
$session->modify($object)
        or die("Modify Group object failed: ",
                $session->status_code() . ":" . $session->status_detail());
print "Group object modified successfully \n";
```

#Remove the Group object

```
#Get the Group object through the session
my @retrieved_objs = $session->get(
            object      => "Infoblox::Grid::UpgradeGroup",
            name        => "group3",
```

```
);
my $object = $retrieved_objs[0];
unless ($object) {
    die("Get Group object failed: ",
        $session->status_code() . ":" . $session->status_detail());
}
print "Get Group object found at least 1 matching entry\n";

#Submit the object for removal
$session->remove($object)
       or die("Remove Group object failed: ",
                $session->status_code() . ":" . $session->status_detail());
print "Upgrade Group object removed successfully \n";
```

#Remove the other created objects

```
$session->remove($group2)
       or die("Remove Group object failed: ",
                $session->status_code() . ":" . $session->status_detail());

$session->remove($group1)
       or die("Remove Group object failed: ",
                $session->status_code() . ":" . $session->status_detail());

####PROGRAM ENDS####
```

AUTHOR

Infoblox Inc. http://www.infoblox.com/

SEE ALSO

Infoblox::Grid::UpgradeGroup::Member,Infoblox::Session->add(), Infoblox::Session->get(), Infoblox::Session->modify(), Infoblox::Session->remove(), Infoblox::Session->search(), Infoblox::Session

---

COPYRIGHT

Copyright (c) 2017 Infoblox Inc.

Infoblox::IPAM::DiscoveryTask::VServer - VMWare discovery server object.

- NAME
- DESCRIPTION
- CONSTRUCTOR
- MODULE METHODS
  - Infoblox::IPAM::DiscoveryTask->vservers( )
- METHODS
  - disable( )
  - encryption( )
  - fqdn_or_ip( )
  - password( )
  - port( )
  - username( )
- AUTHOR
- SEE ALSO
- COPYRIGHT

NAME

Infoblox::IPAM::DiscoveryTask::VServer - VMWare discovery server object.

DESCRIPTION

This object contains information about the vSphere servers on which the VM discovery is performed.

CONSTRUCTOR

```
my $vserver = Infoblox::IPAM::DiscoveryTask::VServer->new(
        fqdn_or_ip    => $string,           #Required.
        password      => $string,           #Required.
        username      => $string,           #Required.
        disable       => "true" | "false",  #Optional. Default is "false".
        encryption    => "true" | "false",  #Optional. Default is "true".
        port          => $num,              #Optional. Default is 443.
);
```

MODULE METHODS

You can apply the following functions to a VMware vSphere server.

Infoblox::IPAM::DiscoveryTask->vservers( )

Use this function to specify the vSphere server on which the VM discovery is performed. See Infoblox::IPAM::DiscoveryTask->vservers( ) for parameters and returned values.

Example

```
    my $vserver1 = Infoblox::IPAM::DiscoveryTask::VServer->new(
          fqdn_or_ip => "vcenter.com",
          username => "test1",
          password => "test1",
       );
    my $vserver2 = Infoblox::IPAM::DiscoveryTask::VServer->new(
          fqdn_or_ip => "10.2.3.4",
          username => "test2",
          password => "test2",
       );
     #Modifying vserver
     $discovery_task->vservers([$vserver1, $vserver2]);
```

METHODS

This section describes all the methods that can be used to retrieve the attribute values of a vSphere server.

disable( )

Use this method to set or retrieve the disable flag of a vSphere server.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

Specify "true" to disable the vSphere server or "false" to enable the server. The default value is "false".

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get disable flag
my $disable = $vserver->disable();
#Modify disable
$vserver->disable("true");
```

encryption( )

Use this method to set or retrieve the encryption flag of a vSphere server.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

Specify "true" to enable encryption or specify "false" to disable encryption on the vSphere server. The default value is "true".

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get encryption
my $encryption = $vserver->encryption();
#Modify encryption
$vserver->encryption("false");
```

fqdn_or_ip( )

Use this method to retrieve the FQDN (Fully Qualified Domain Name) or IP address of the vShpere server.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

The string that contains the FQDN or the IP address of the vSphere server.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get fqdn_or_ip
my $fqdn_or_ip = $vserver->fqdn_or_ip();
#Modify fqdn_or_ip
$vserver->fqdn_or_ip("vdiscovery.example.com");
```

password( )

Use this method to set the user password on the vSphere server. This is a write-only attribute.

Parameter

String containing password.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

Example

```
#Modify password
$vserver->password("infobloxx");
```

port( )

Use this method to retrieve the connection port that the vSphere server uses.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

Valid values are between 1 and 65535. The default value is 443.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get port
my $port = $vserver->port();
#Modify port
$vserver->port(444);
```

username( )

Use this method to set or retrieve the user name on the VSphere server.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

The username in the string format.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get username
my $username = $vserver->username();
#Modify username
$vserver->username("someuser");
```

AUTHOR

Infoblox Inc. http://www.infoblox.com/

SEE ALSO

Infoblox::IPAM::DiscoveryTask, Infoblox::IPAM::DiscoveryTask->vservers()

COPYRIGHT

Copyright (c) 2017 Infoblox Inc.

Infoblox::IPAM::TCPPort - Network discovery TCP port

- NAME
- DESCRIPTION
- CONSTRUCTOR
- SESSION METHODS
- METHODS
  - comment( )
  - number( )
- SAMPLE CODE
- AUTHOR
- SEE ALSO
- COPYRIGHT

NAME

Infoblox::IPAM::TCPPort - Network discovery TCP port

DESCRIPTION

This class represents the TCP port that network discovery jobs use to find IP addresses.

CONSTRUCTOR

```
my $tcp_port = Infoblox::IPAM::TCPPort->new(
     number             => $string,             #Required.
     comment            => $string,             #Optional.
);
```

SESSION METHODS

This object does not have a session method. You must use it with Infoblox::IPAM::DiscoveryTask class.

METHODS

This section describes all the methods that you can use to configure and retrieve the attribute values of a TCP port.

comment( )

Use this method to set or retrieve a descriptive comment for a TCP port.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

Enter a descriptive comment for the TCP port in string format.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
# Get comment
my $comment = $tcp_port->comment();
# Modify comment
$tcp_port->comment("new port");
```

number( )

Use this method to set or retrieve the TCP port number.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

Enter a number the TCP port.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
# Get port number
my $port_number = $tcp_port->number();
# Modify port number
$tcp_port->number("222");
```

SAMPLE CODE

Please see sample code in Infoblox::IPAM::DiscoveryTask.

AUTHOR

Infoblox Inc. http://www.infoblox.com/

SEE ALSO

Infoblox::IPAM::DiscoveryTask

COPYRIGHT

Copyright (c) 2017 Infoblox Inc.

Infoblox::LDAP::EA_Mapping - An LDAP Extensible Attribute Mapping object.

- NAME
- DESCRIPTION
- CONSTRUCTOR
- SESSION METHODS
- METHODS
  - attr_name( )
  - ext_attr_def( )
- AUTHOR
- SEE ALSO
- COPYRIGHT

NAME

Infoblox::LDAP::EA_Mapping - An LDAP Extensible Attribute Mapping object.

DESCRIPTION

Mapping of LDAP field to Extensible Attribute.

CONSTRUCTOR

```
my $ea_mapping = Infoblox::LDAP::EA_Mapping->new(
   attr_name    => $string,
   ext_attr_def => $attr_def,                    # An Infoblox::Grid::ExtensibleAttributeDef object
);
```

SESSION METHODS

The LDAP Extensible Attribute Mapping object does not support any session methods.

METHODS

This section describes all the methods that you can use to configure and retrieve the attribute values of an LDAP Extensible Attribute Mapping object.

attr_name( )

Use this method to set or retrieve the name of an LDAP attribute.

Parameter

The name of an LDAP attribute in string format.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get attribute value
my $value = $ea_mapping->attr_name();
#Modify attribute value
$ea_mapping->attr_name('homephone');
```

ext_attr_def( )

Use this method to set or retrieve the definition of an extensible attribute.

Parameter

An Infoblox::Grid::ExtensibleAttributeDef object.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get attribute value
my $value = $ea_mapping->ext_attr_def();
#Modify attribute value
$ea_mapping->ext_attr_def($attr_def);
```

AUTHOR

Infoblox Inc. http://www.infoblox.com/

SEE ALSO

Infoblox::LDAP::AuthService, Infoblox::Grid::ExtensibleAttributeDef

---

COPYRIGHT

Copyright (c) 2017 Infoblox Inc.

Infoblox::LDAP::Server - An LDAP Server object.

- NAME
- DESCRIPTION
- CONSTRUCTOR
- SESSION METHODS
- METHODS
  - address( )
  - authentication_type( )
  - base_dn( )
  - bind_password( )
  - bind_user_dn( )
  - comment( )
  - encryption( )
  - disable( )
  - port( )
  - use_mgmt_port( )
  - version( )
- AUTHOR
- SEE ALSO
- COPYRIGHT

NAME

Infoblox::LDAP::Server - An LDAP Server object.

DESCRIPTION

The LDAP Server is used for authentication.

CONSTRUCTOR

```
my $server = Infoblox::LDAP::Server->new(
    address             => $address_or_fqdn,              # Required
    base_dn             => $string,                       # Required
    bind_user_dn        => $string,                       # Required if authentication_type is
'AUTHENTICATED'
    bind_password       => $string,                       # Required if authentication_type is
'AUTHENTICATED', write-only
    port                => $uint,                         # Optional, Default is 636
    version             => 'V2' | 'V3',                   # Optional, Default is 'V3'
    comment             => $string,                       # Optional
    authentication_type => 'ANONYMOUS' | 'AUTHENTICATED', # Optional, Default is 'ANONYMOUS'
    encryption          => 'NONE' | 'SSL',                # Optional, Default is 'SSL'
    use_mgmt_port       => 'true' | 'false',              # Optional, Default is 'false'
    disable             => 'true' | 'false',              # Optional, Default is 'false'
);
```

SESSION METHODS

The LDAP Server object does not support any session methods.

METHODS

This section describes all the methods that you can use to configure and retrieve the attribute values of an LDAP server object.

address( )

Use this method to set or retrieve an IP address or FQDN of an LDAP server.

Parameter

An IP address or FQDN of an LDAP server in string format.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get attribute value
my $value = $server->address();
#Modify attribute value
$server->address('10.0.0.1');
```

authentication_type( )

Use this method to set or retrieve an authentication type.

Parameter

Valid values are 'ANONYMOUS' and 'AUTHENTICATED'. The default value is 'ANONYMOUS'.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get attribute value
my $value = $server->authentication_type();
#Modify attribute value
$server->authentication_type('AUTHENTICATED');
```

base_dn( )

Use this method to set or retrieve a base DN for the LDAP server.

Parameter

A base DN in string format.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get attribute value
my $value = $server->base_dn();
#Modify attribute value
$server->base_dn('dc=infoblox,dc=com');
```

bind_password( )

Use this method to set a user password for authentication. This attribute is required if authentication_type attribute value is 'AUTHENTICATED'. This is a write-only attribute.

Parameter

An user password in string format.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

Example

```
#Modify attribute value
$server->bind_password('password');
```

bind_user_dn( )

Use this method to set or retrieve an user DN for authentication. This attribute is required if authentication_type attribute value is 'AUTHENTICATED'.

Parameter

An user DN in string format.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get attribute value
my $value = $server->bind_user_dn();
#Modify attribute value
$server->bind_user_dn('cn=user,dc=infoblox,dc=com');
```

comment( )

Use this method to set or retrieve a descriptive comment.

Parameter

A desired comment in string format with a maximum of 256 bytes.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get attribute value
my $value = $server->comment();
#Modify attribute value
$server->comment('LDAP object');
```

encryption( )

Use this method to set or retrieve an encryption type.

Parameter

Valid values are 'NONE' and 'SSL'. The default value is 'SSL'.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get attribute value
my $value = $server->encryption();
#Modify attribute value
$server->encryption('SSL');
```

disable( )

Use this method to enable or disable an LDAP server.

Parameter

Specify "true" to disable an LDAP server or "false" to enable it. The default value is 'false'.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get attribute value
my $value = $server->disable();
#Modify attribute value
$server->disable('true');
```

port( )

Use this method to set or retrieve a port number.

Parameter

A port number. The default value is 636.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get attribute value
my $value = $server->port();
#Modify attribute value
$server->port(389);
```

use_mgmt_port( )

Use this method to set or retrieve the flag that allows connections through the management interface.

Parameter

Specify "true" to allow or "false" to deny it. The default value is 'false'.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get attribute value
my $value = $server->use_mgmt_port();
#Modify attribute value
$server->use_mgmt_port('true');
```

version( )

Use this method to set or retrieve an LDAP server version.

Parameter

Valid values are 'V2' and 'V3'. The default value is 'V3'.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get attribute value
my $value = $server->version();
#Modify attribute value
$server->version('V2');
```

AUTHOR

Infoblox Inc. http://www.infoblox.com/

SEE ALSO

Infoblox::LDAP::AuthService

COPYRIGHT

Copyright (c) 2017 Infoblox Inc.

Infoblox::Notification::REST::Endpoint - A REST API endpoint object.

- NAME
- DESCRIPTION
- CONSTRUCTOR
- SESSION METHODS
  - Infoblox::Session->add( )
  - Infoblox::Session->get( )
  - Infoblox::Session->modify( )
  - Infoblox::Session->remove( )
  - Infoblox::Session->search( )
- METHODS
  - client_certificate( )
  - client_certificate_subject( )
  - client_certificate_valid_from( )
  - client_certificate_valid_to( )
  - comment( )
  - disable( )
  - extattrs( )
  - extensible_attributes( )
  - name( )
  - outbound_members( )
  - outbound_member_type( )
  - log_level( )
  - password( )
  - server_cert_validation( )
  - template_instance( )
  - timeout( )
  - uri( )
  - username( )
  - vendor_identifier( )
  - wapi_user_name( )
  - wapi_user_password( )
- AUTHOR
- SEE ALSO
- COPYRIGHT

NAME

Infoblox::Notification::REST::Endpoint - A REST API endpoint object.

DESCRIPTION

The REST API endpoint object provides configuration for third party REST API servers integration.

CONSTRUCTOR

```
 my $endpoint = Infoblox::Notification::REST::Endpoint->new(
     name                  => $string,                                                  #
Required
     uri                   => $string,                                                  #
Required
     outbound_member_type  => 'GM' | 'MEMBER',                                          #
Required
     client_certificate    => $path,                                                    #
Optional / Default is undefined
     comment               => $string,                                                  #
Optional / Default is undefined
     disable               => 'true' | 'false',                                         #
Optional / Default is 'true'
     extattrs              => { $string => $extattr, ... },                             #
Optional / Default is undefined
     extensible_attributes => {$string => $string | $num, $string => [ $string | $num, ... ], ... } #
Optional / Default is undefined
     log_level             => 'WARNING' | 'DEBUG' | 'INFO' | 'ERROR',                   #
Optional / Default is 'WARNING'
     outbound_members      => [$string, $string, ...],                                  #
Optional / Default is empty list
     password              => $string,                                                  #
Optional / Default is undefined
     template_instance     => $template_instance                                        #
Optional / Default is undefined
     timeout               => $uint,                                                    #
```

```
Optional / Default is 0
     username               => $string,                                                  #
Optional / Default is undefined
     vendor_identifier      => $string,                                                  #
Optional / Default is undefined
     server_cert_validation => 'CA_CERT' | 'CA_CERT_NO_HOSTNAME' | 'NO_VALIDATION',      #
Optional / Default is 'CA_CERT'
     wapi_user_name         => $string,                                                  #
Optional / Default is undefined
     wapi_user_password     => $string,                                                  #
Optional / Default is undefined

 );
```

You cannot set both extattrs and extensible_attributes attributes at the same time.

SESSION METHODS

This section describes all the methods in an Infoblox::Session module that you can apply to a REST API endpoint object.

Infoblox::Session->add( )

Use this method to add the object to the Infoblox appliance. See Infoblox::Session->add() for parameters and return values.

Example

```
#Construct an object
my $object = Infoblox::Notification::REST::Endpoint->new(
    uri                  => 'http://10.0.0.1/office',
    name                 => 'restapi_endpoint',
    outbound_member_type => 'GM',
);

#Submit for addition
my $response = $session->add($object);
```

Infoblox::Session->get( )

Use this method to retrieve all the matching objects from the Infoblox appliance. See Infoblox::Session->get() for parameters and return values.

Key References

```
Apply the following attributes to get a specific REST API endpoint object:

 comment                    - Optional. The REST API endpoint comment in string format.
 name                       - Optional. The REST API endpoint name in string format.
 uri                        - Optional. The REST API endpoint URI in string format.
 vendor_identifier          - Optional. The REST API endpoint vendor identifier in string format.
 extattrs                   - Optional. A hash reference containing Infoblox::Grid::Extattr object.
 extensible_attributes      - Optional. A hash reference containing extensible attributes.
```

Examples

```
my @retrieved_objs = $session->get(
    object => 'Infoblox::Notification::REST::Endpoint',
    uri    => 'http://10.0.0.1/office',
);

my @retrieved_objs = $session->get(
    object               => 'Infoblox::Notification::REST::Endpoint',
    extensible_attributes => {'Site' => 'Santa Clara'},
);
```

Infoblox::Session->modify( )

Use this method to modify the object in the Infoblox appliance. See Infoblox::Session->modify() for parameters and return values.

Example

```
#Modify comment value
$object->comment('this is a modified comment');

#Submit modification
my $response = $session->modify($object);
```

Infoblox::Session->remove( )

Use this method to remove the object from the Infoblox appliance. See Infoblox::Session->remove() for parameters and return values.

To remove a specific object, first use `get()` or `search()` to retrieve the specific REST API endpoint object, and then submit this object for removal.

Example

```
#Get the objects
my @retrieved_objs = $session->get(
    object       => 'Infoblox::Notification::REST::Endpoint',
    uri          => 'http://10.0.0.1/office',
);

#Find the desired object from the retrieved list
my $desired_object = $retrieved_objs[0];

#Submit for removal
my $response = $session->remove($desired_object);
```

Infoblox::Session->search( )

Use this method to search for objects in the Infoblox appliance. See Infoblox::Session->search() for parameters and return values.

Key References

```
Apply the following attributes to search for the REST API endpoint object:

 comment                   - Optional. The REST API endpoint comment in string format (regexp).
 name                      - Optional. The REST API endpoint name in string format (regexp).
 uri                       - Optional. The REST API endpoint URI in string format (regexp).
 vendor_identifier         - Optional. The REST API endpoint vendor identifier in string format
(regexp).
 extattrs                  - Optional. A hash reference containing Infoblox::Grid::Extattr object.
 extensible_attributes     - Optional. A hash reference containing extensible attributes.
```

For more information about searching extensible attributes, see Infoblox::Grid::ExtensibleAttributeDef/Searching Extensible Attributes.

Examples

```
# search for objects
my @retrieved_objs = $session->search(
    object           => 'Infoblox::Notification::REST::Endpoint',
    uri              => 'http.*',
    vendor_identifier => 'Carbon',
);

# search for objects defining 'Santa Clara' for 'Site' extensible attribute
my @retrieved_objs = $session->search(
    object                => 'Infoblox::DHCP::Range',
    extensible_attributes => {'Site' => 'Santa Clara'},
);
```

METHODS

This section describes all the methods that you can use to configure and retrieve the attribute values of a REST API endpoint.

client_certificate( )

Use this method to set the path to a client certificate file. This is a write-only attribute.

Include the specified parameter to set the attribute value.

Parameter

The valid value is a path to a certificate in string format.

Returns

If you have specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

Example

```
#Modify client_certificate value
$object->client_certificate('/path/to/cert.pem');
```

client_certificate_subject( )

Use this method to retrieve the client certificate subject. This is a read-only attrbiute.

Omit the parameter to retrieve the attribute value.

Parameter

None

Returns

The method returns the attribute value.

Example

```
#Get client_certificate_subject value
my $client_certificate_subject = $object->client_certificate_subject();
```

client_certificate_valid_from( )

Use this method to retrieve the time from which the client certificate is valid. This is a read-only attrbiute.

Omit the parameter to retrieve the attribute value.

Parameter

None

Returns

The method returns the attribute value. The number of seconds that have elapsed since January 1st, 1970 UTC.

Example

```
#Get client_certificate_valid_from value
my $client_certificate_valid_from = $object->client_certificate_valid_from();
```

client_certificate_valid_to( )

Use this method to retrieve the bulk client certificate expiration time. This is a read-only attrbiute.

Omit the parameter to retrieve the attribute value.

Parameter

None

Returns

The method returns the attribute value. The number of seconds that have elapsed since January 1st, 1970 UTC.

Example

```
#Get client_certificate_valid_to value
my $client_certificate_valid_to = $object->client_certificate_valid_to();
```

comment( )

Use this method to set or retrieve the REST API endpoint comment.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

The valid value is a desired comment in string format.

Returns

If you have specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get comment value
my $comment = $object->comment();

#Modify comment value
```

```
$object->comment('modified comment');
```

disable( )

Use this method to set or retrieve the flag that indicates whether the REST API endpoint is enabled or disabled.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

Specify 'true' to disable the REST API endpoint or 'false' to enable it. The default value is 'true'.

Returns

If you have specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get disable value
my $disable = $object->disable();

#Modify disable value
$object->disable('false');
```

extattrs( )

Use this method to set or retrieve the extensible attributes associated with a REST API endpoint object.

Parameter

Valid value is a hash reference that contains the names of extensible attributes and their associated values (Infoblox::Grid::Extattr objects).

Returns

If you have specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get extattrs value
my $extattrs = $object->extattrs();

#Modify extattrs value
$object->extattrs({'Site' => $extattr1, 'Administrator' => $extattr2});
```

extensible_attributes( )

Use this method to set or retrieve the extensible attributes associated with a REST API endpoint object.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

For valid values for extensible attributes, see Infoblox::Grid::ExtensibleAttributeDef/Extensible Attribute Values.

Returns

If you have specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get extensible attributes value
my $extensible_attributes = $object->extensible_attributes();
#Modify extensible attributes
$object->extensible_attributes({'Site' => 'Santa Clara', 'Administrator' => ['Peter', 'Tom']});
```

name( )

Use this method to set or retrieve the REST API endpoint name.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

The valid value is a desired name in string format.

Note that this field will default to URI value if no value is specified.

Returns

If you have specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get name value
my $name = $object->name();

#Modify name value
$object->name('modified_name');
```

outbound_members( )

Use this method to set or retrieve the list of REST API endpoint members which are responsible for event generation.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

The valid value is an array that contains Grid member names in a string format.

Returns

If you have specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get outbound_members value
my $outbound_members = $object->outbound_members();

#Modify outbound_members value
$object->outbound_members(['member1', 'member2']);
```

outbound_member_type( )

Use this method to set or retrieve the REST API endpoint member which is responsible for event generation.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

The valid values are 'GM' and 'MEMBER'.

Returns

If you have specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get outbound_member_type value
my $outbound_member_type = $object->outbound_member_type();

#Modify outbound_member_type value
$object->outbound_member_type('MEMBER');
```

log_level( )

Use this method to set or retrieve the REST API endpoint logging level.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

The valid values are 'DEBUG', 'ERROR', 'INFO', and 'WARNING'.

Returns

If you have specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get log_level value
my $value = $object->log_level();

#Modify log_level value
$object->log_level('INFO');
```

password( )

Use this method to set or retrieve the REST API endpoint auth password. This is a write-only attribute.

Include the specified parameter to set the attribute value.

Parameter

The valid value is a desired password in string format.

Returns

The method returns true when the modification succeeds, and returns false when the operation fails.

Example

```
#Modify password value
$object->password('usr1');
```

server_cert_validation( )

Use this method to set or retrieve the server certificate validation type.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

The valid values are 'CA_CERT', 'CA_CERT_NO_HOSTNAME' and 'NO_VALIDATION'. The default value is 'CA_CERT'.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get server_cert_validation value
my $value = $object->server_cert_validation();

#Modify server_cert_validation value
$object->server_cert_validation('NO_VALIDATION');
```

template_instance( )

Use this method to set or retrieve the REST API endpoint REST API template instance.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Note that 'parameters' value of a template instance object cannot be changed for REST API endpoint.

Parameter

The valid value is an Infoblox::Notification::REST::TemplateInstance object.

Returns

If you have specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get template_instance value
my $value = $object->template_instance();

#Modify template_instance value
$object->template_instance($instance);
```

timeout( )

Use this method to set or retrieve the REST API endpoint management session timeout.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

The valid value is an unsigned integer that represents the number of seconds for the management session timeout.

Returns

If you have specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get timeout value
my $value = $object->timeout();

#Modify timeout value
$object->timeout(100);
```

uri( )

Use this method to set or retrieve the REST API endpoint URI.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

The valid value is a desired URI in string format which should be valid URL.

Returns

If you have specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get uri value
my $uri = $object->uri();

#Modify uri value
$object->uri('http://10.0.0.1/new_uri');
```

username( )

Use this method to set or retrieve the REST API endpoint auth username.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

The valid value is a desired username in string format.

Returns

If you have specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get username value
my $username = $object->username();

#Modify username value
$object->username('usr1');
```

vendor_identifier( )

Use this method to set or retrieve the REST API endpoint vendor identifier.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

The valid value is a desired vendor identifier in string format.

Returns

If you have specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get vendor_identifier value
my $vendor_identifier = $object->vendor_identifier();

#Modify vendor_identifier value
$object->vendor_identifier('CarbonBlack');
```

wapi_user_name( )

Use this method to set or retrieve the user name for WAPI integration.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

The valid value is a desired user name in string format.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get wapi_user_name value
my $value = $object->wapi_user_name();

#Modify wapi_user_name value
$object->wapi_user_name('admin');
```

wapi_user_password( )

Use this method to set the user password for WAPI integration. This is a write-only attribute.

Include the specified parameter to set the attribute value.

Parameter

The valid value is a desired user password in string format.

Returns

The method returns true when the modification succeeds, and returns false when the operation fails.

Example

```
#Modify wapi_user_password value
$object->wapi_user_password('infoblox');
```

AUTHOR

Infoblox Inc. http://www.infoblox.com/

SEE ALSO

Infoblox::Session, Infoblox::Session->add(), Infoblox::Session->get(), Infoblox::Session->modify(), Infoblox::Session->remove(), Infoblox::Session->search(), Infoblox::Grid::Extattr, Infoblox::Grid::ExtensibleAttributeDef/Searching Extensible Attributes,

---

COPYRIGHT

Copyright (c) 2017 Infoblox Inc.

Infoblox::Notification::REST::Template - REST API template object.

- NAME
- DESCRIPTION
- CONSTRUCTOR
- SESSION METHODS
  - Infoblox::Session->get( )
  - Infoblox::Session->modify( )
  - Infoblox::Session->remove( )
  - Infoblox::Session->search( )
- METHODS
  - action_name( )
  - added_on( )
  - comment( )
  - content( )
  - event_type( )
  - name( )
  - outbound_type( )
  - template_type( )
  - vendor_identifier( )
- AUTHOR
- SEE ALSO
- COPYRIGHT

NAME

Infoblox::Notification::REST::Template - REST API template object.

DESCRIPTION

The REST API template object used to define particular template configuration.

CONSTRUCTOR

```
my $templ = Infoblox::Notification::REST::Template->new(
    name          => $string,        #Required
    content       => $string,        #Required
    comment       => $string,        #Optional / Default is undefined
    outbound_type => 'REST' | 'DXL', #Optional / Default is 'REST'
);
```

SESSION METHODS

This section describes all the methods in an Infoblox::Session module that you can apply to a REST API template object.

Note that template object does not support 'add' method. Use Infoblox::Session->import_data() of a 'restapi_template' type instead.

Infoblox::Session->get( )

Use this method to retrieve all the matching objects from the Infoblox appliance. See Infoblox::Session->get() for parameters and return values.

Key References

```
Apply the following attributes to get a specific REST API template object:

 comment       - Optional. The REST API template comment in string format.
 name          - Optional. The REST API template name in string format.
 action_name   - Optional. The REST API template action name in string format.
 outbound_type - Optional. The template type.
```

Examples

```
my @retrieved_objs = $session->get(
    object => 'Infoblox::Notification::REST::Template',
    name   => 'template1',
);
```

Infoblox::Session->modify( )

Use this method to modify the object in the Infoblox appliance. See Infoblox::Session->modify() for parameters and return values.

Example

```
#Modify comment value
$object->comment('this is a modified comment');

#Submit modification
my $response = $session->modify($object);
```

Infoblox::Session->remove( )

Use this method to remove the object from the Infoblox appliance. See Infoblox::Session->remove() for parameters and return values.

To remove a specific object, first use `get()` or `search()` to retrieve the specific REST API template object, and then submit this object for removal.

Example

```
#Get the objects
my @retrieved_objs = $session->get(
    object => 'Infoblox::Notification::REST::Template',
    name   => 'template1',
);

#Find the desired object from the retrieved list
my $desired_object = $retrieved_objs[0];

#Submit for removal
my $response = $session->remove($desired_object);
```

Infoblox::Session->search( )

Use this method to search for objects in the Infoblox appliance. See Infoblox::Session->search() for parameters and return values.

Key References

```
Apply the following attributes to search for the REST API template object:

 comment       - Optional. The REST API template comment in string format (regexp).
 name          - Optional. The REST API template name in string format (regexp).
 action_name   - Optional. The REST API template action name in string format (regexp).
 outbound_type - Optional. The template type.
```

Examples

```
# search for objects
my @retrieved_objs = $session->search(
    object => 'Infoblox::Notification::REST::Template',
    name   => 'templ.*',
);
```

METHODS

This section describes all the methods that you can use to configure and retrieve the attribute values of a REST API template instance object.

action_name( )

Use this method to retrieve the template action name. This is a read-only attribute.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

None

Returns

The method returns the attribute value.

Example

```
#Get action_name value
my $action_name = $object->action_name();
```

added_on( )

Use this method to retrieve the timestamp the template was added. This is a read-only attribute.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

None

Returns

The method returns the attribute value, the number of seconds that have elapsed since January 1st, 1970.

Example

```
#Get added_on value
my $added_on = $object->added_on();
```

comment( )

Use this method to set or retrieve the descriptive comment.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

The valid value is a desired comment in string format.

Returns

If you have specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get comment value
my $comment = $object->comment();

#Modify comment value
$object->comment('new comment');
```

content( )

Use this method to set or retrieve the template content.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

The valid value is an well formed JSON string.

Returns

If you have specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get content value
my $content = $object->content();

#Modify content value
$object->content($content);
```

event_type( )

Use this method to retrieve the template event type. This is a read-only attribute.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

None

Returns

The method returns the attribute value.

Example

```
#Get event_type value
my $event_type = $object->event_type();
```

name( )

Use this method to set or retrieve the template name.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

The valid value is a desired name in string format.

Returns

If you have specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get name value
my $name = $object->name();

#Modify name value
$object->name('new_name');
```

outbound_type( )

Use this method to set or retrieve the outbound type for the template.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

The valid value is 'REST' or 'DXL'. The default value is 'REST'.

Returns

If you have specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get outbound_type value
my $outbound_type = $object->outbound_type();

#Modify outbound_type value
$object->outbound_type('DXL');
```

template_type( )

Use this method to retrieve the template type. This is a read-only attribute.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

None

Returns

The method returns the attribute value.

Example

```
#Get template_type value
my $template_type = $object->template_type();
```

vendor_identifier( )

Use this method to retrieve the template vendor identifier. This is a read-only attribute.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

None

Returns

The method returns the attribute value.

Example

```
#Get vendor_identifier value
my $vendor_identifier = $object->vendor_identifier();
```

AUTHOR

Infoblox Inc. http://www.infoblox.com/

SEE ALSO

Infoblox::Session, Infoblox::Session->get(), Infoblox::Session->modify(), Infoblox::Session->remove(), Infoblox::Session->search(), Infoblox::Session->import_data()

COPYRIGHT

Copyright (c) 2017 Infoblox Inc.

Infoblox::Notification::REST::TemplateParameter - A REST API template parameter.

- NAME
- DESCRIPTION
- CONSTRUCTOR
- SESSION METHODS
- MODULE METHODS
  - Infoblox::Notification::REST::TemplateInstance->parameters( )
- METHODS
  - default_value( )
  - name( )
  - syntax( )
  - value( )
- AUTHOR
- SEE ALSO
- COPYRIGHT

NAME

Infoblox::Notification::REST::TemplateParameter - A REST API template parameter.

DESCRIPTION

The REST API template parameter is used to configure REST API template instance variables.

CONSTRUCTOR

```
my $templ_parameter = Infoblox::Notification::REST::TemplateParameter->new(
    name          => $string,                 # Required
    syntax        => 'STR' | 'INT' | 'BOOL', # Required
    value         => $string,                 # Optional / Default is undefined
);
```

SESSION METHODS

The object does not have any session methods.

MODULE METHODS

This section describes all the functions that you can apply to REST API template parameter object.

Infoblox::Notification::REST::TemplateInstance->parameters( )

Use this method to set or retrieve the list of REST API template parameters. See Infoblox::Notification::REST::TemplateInstance->parameters() for parameters and return values.

Example

```
#Get parameters value
my $parameters = $object->parameters();

#Modify parameters value
$object->parameters([[$param1, $param2]);
```

METHODS

This section describes all the methods that you can use to configure and retrieve the attribute values of a REST API template parameter object.

default_value( )

Use this method to retrieve the template parameter default value. This is a read-only attribute.

Omit the parameter to retrieve the attribute value.

Parameter

None

Returns

The method returns the attribute value.

Example

```
#Get default_value value
my $default_value = $object->default_value();
```

name( )

Use this method to set or retrieve the template parameter name.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

The valid value is desired name in string format.

Returns

If you have specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get name value
my $name = $object->name();

#Modify name value
$object->name('param1');
```

syntax( )

Use this method to set or retrieve the template parameter syntax.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

The valid values are 'BOOL', 'STR' and 'INT'.

Returns

If you have specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get syntax value
my $syntax = $object->syntax();

#Modify syntax value
$object->syntax('INT');
```

value( )

Use this method to set or retrieve the template parameter value.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

The desired value in string format.

Returns

If you have specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get value value
my $value = $object->value();

#Modify value value
$object->value('val');
```

AUTHOR

Infoblox Inc. http://www.infoblox.com/

SEE ALSO

Infoblox::Notification::REST::TemplateInstance, Infoblox::Notification::REST::TemplateInstance->parameters()

COPYRIGHT

Copyright (c) 2017 Infoblox Inc.

Infoblox::Notification::RuleExpressionOp - A notification rule expression operand object.

- NAME
- DESCRIPTION
- CONSTRUCTOR
- SESSION METHODS
- MODULE METHODS
  - Infoblox::Notification::Rule->expression_list( )
- METHODS
  - op( )
  - op1( )
  - op1_type( )
  - op2( )
  - op2_type( )
- AUTHOR
- SEE ALSO
- COPYRIGHT

NAME

Infoblox::Notification::RuleExpressionOp - A notification rule expression operand object.

DESCRIPTION

The expression operand object is used to build expression lists. The allowed values for the expression operand object depend on the object they appear to be a part of.

CONSTRUCTOR

```
 my $expression = Infoblox::Notification::RuleExpressionOp->new(
     op       => 'AND' | 'OR' | 'ENDLIST' | 'EQ' | 'REGEX' |
                 | 'MATCH_CIDR' | 'MATCH_RANGE' | 'LE' | 'GE' | 'CONTAINED_IN'            # Required
     op1      => 'ADDRESS_TYPE' | 'DHCP_FINGERPRINT' |
                 'DHCP_LEASE_STATE' | 'DHCP_IP_ADDRESS' | 'DISABLE' |
                 'DNS_RPZ_ACTION_POLICY' | 'DNS_RPZ_NAME' | 'DNS_RPZ_RULE_NAME' |
                 'DNS_RPZ_TYPE' | 'DUID' | 'HOST' | 'IPV4_ADDRESS' |
                 'IPV6_ADDRESS' | 'IPV6_PREFIX' | 'IPV6_PREFIX_BITS' | 'MAC' |
                 'NAME' | 'NETWORK' | 'NETWORK_VIEW' | 'SECURITY_ADP_RULE_MESSAGE' |
                 'SECURITY_ADP_RULE_SEVERITY' | 'SECURITY_ADP_SID' |
                 'SERVER_ASSOC_TYPE_IPV4', 'SERVER_ASSOC_TYPE_IPV6' | 'SOURCE_IP',        # Optional /
Default is undefined
     op2      => $string,                                                                 # Optional /
Default is undefined
     op1_type => 'FIELD' | 'LIST',                                                        # Optional /
Default is undefined
     op2_type => 'STRING',                                                                # Optional /
Default is undefined
 );
```

SESSION METHODS

The object does not support any session methods.

MODULE METHODS

Infoblox::Notification::Rule->expression_list( )

Use this method to set or retrieve the notification rule condition expression. See Infoblox::Notification::Rule->expression_list() for parameters and return values.

Example

```
#Get expression_list value
my $expression_list = $object->expression_list();

#Modify expression_list value
$object->expression_list([$start_list, $op1, $op2, $end_list]);
```

METHODS

op( )

Use this method to set or retrieve the operation name.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

The valid values are 'AND', 'OR', 'ENDLIST', 'EQ', 'NOT_EQ', 'LE', 'REGEX', 'GE', 'CONTAINED_IN', 'MATCH_CIDR' and 'MATCH_RANGE'.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get op value
my $op = $object->op();

#Modify op value
$object->op('OR');
```

op1( )

Use this method to set or retrieve the first operand value.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

The valid values are 'ADDRESS_TYPE', 'DHCP_FINGERPRINT' , 'DHCP_LEASE_STATE', 'DHCP_IP_ADDRESS', 'DISABLE' , 'DNS_RPZ_ACTION_POLICY', 'DNS_RPZ_NAME', 'DNS_RPZ_RULE_NAME' , 'DNS_RPZ_TYPE', 'DUID', 'HOST', 'IPV4_ADDRESS' , 'IPV6_ADDRESS', 'IPV6_PREFIX', 'IPV6_PREFIX_BITS', 'MAC' , 'NAME', 'NETWORK', 'NETWORK_VIEW', 'SECURITY_ADP_RULE_MESSAGE' , 'SECURITY_ADP_RULE_SEVERITY', 'SECURITY_ADP_SID' , 'SERVER_ASSOC_TYPE_IPV4', 'SERVER_ASSOC_TYPE_IPV6', 'SOURCE_IP'.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get op1 value
my $op1 = $object->op1();

#Modify op1 value
$object->op1('SOURCE_IP');
```

op1_type( )

Use this method to set or retrieve the first operand type.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

The valid values are 'FIELD', 'LIST' and 'STRING'.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get op1_type value
my $op1_type = $object->op1_type();

#Modify op1_type value
$object->op1_type('LIST');
```

op2( )

Use this method to set or retrieve the second operand value.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

The valid value is a string that contains the second operand value.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get op2 value
my $op2 = $object->op2();

#Modify op2 value
$object->op2('OP2_VAL');
```

op2_type( )

Use this method to set or retrieve the second operand type.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

The valid value is 'STRING'.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get op2_type value
my $op2_type = $object->op2_type();

#Modify op2_type value
$object->op2_type('STRING');
```

AUTHOR

Infoblox Inc. http://www.infoblox.com/

SEE ALSO

Infoblox::Notification::Rule, Infoblox::Notification::Rule->expression_list(),

---

COPYRIGHT

Copyright (c) 2017 Infoblox Inc.

Infoblox::Grid::IPValidationInfo - IP Validation information

- NAME
- DESCRIPTION
- CONSTRUCTOR
- MODULE METHODS
  - Infoblox::Session->add( ) and Infoblox::Session->modify( )
- METHODS
  - address( )
  - parent( )
- SAMPLE CODE
- AUTHOR
- SEE ALSO
- COPYRIGHT

NAME

Infoblox::Grid::IPValidationInfo - IP Validation information

DESCRIPTION

When submitting add or modify requests that contain IPs returned by the next_available_ip functionality, this object can be passed to ensure that these IPs are unique and that concurrent additions and modifications containing the same IPs will fail.

CONSTRUCTOR

```
my $member = Infoblox::Grid::IPValidationInfo->new(
    address        => $ipaddr,         #Required
    parent         => $obj,            #Required
);
```

MODULE METHODS

The following functions are available for an IP validation info object.

Infoblox::Session->add( ) and Infoblox::Session->modify( )

The object can be passed in the ip_validation option. For usage information, see the sample code section.

METHODS

This section describes all the methods that you can use to set and retrieve the attribute values of an IP validation info object.

address( )

Use this method to set or retrieve the address.

Parameter

The valid value is an IPv4 or IPv6 address.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get address
my $address = $vip->address();
#Modify address
$vip->address('10.0.0.1');
```

parent( )

Use this method to set the parent object for the validation. This is the object from which the next available IP was retrieved, and it must be an object retrieved from the server.

Parameter

The valid value is an Infoblox::DHCP::Range, Infoblox::DHCP::IPv6Range, Infoblox::DHCP::Network, Infoblox::DHCP::IPv6Network

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get parent
my $net = $member->vip();
#Modify vip
$member->vip($net2);
```

SAMPLE CODE

The following sample code demonstrates the different functions that can be applied to an object, such as add, search, modify, and remove. This sample also includes error handling for the operations.

#Preparation prior to the insertion

```
#PROGRAM STARTS: Include all the modules that will be used
use strict;
use Infoblox;

#Create a session to the Infoblox appliance

my $host_ip   = "192.168.1.2";
my $host_name = "infoblox.localdomain";

my $session = Infoblox::Session->new(
              master   => $host_ip,      #appliance host ip
              username => "admin",       #appliance user login
              password => "infoblox"     #appliance password
);

unless ($session) {
       die("Construct session failed: ",
              Infoblox::status_code() . ":" . Infoblox::status_detail());
}
print "Session created successfully\n";
```

#Create a DNS Zone object

```
my $default_view = Infoblox::DNS::View->new(name => "default");

my $memberns1 = Infoblox::DNS::Member->new(
    name     => "infoblox.localdomain",
    ipv4addr => $host_ip,
    lead     => "false",
    stealth  => "false"
);

#To add zone test.com in the default view
my $firstzone = Infoblox::DNS::Zone->new(
    name         => "test.com",
    views        => [$default_view],
    email        => "admin\@infoblox.com",
    comment      => "add a zone test.com",
    primary      => $memberns1,
    );

unless($firstzone){
       die("Construct test.com zone object failed: ",
              Infoblox::status_code(). ":" .Infoblox::status_detail());
       }
print "test.com zone object created successfully.\n";

$session->add($firstzone)
    or die("Add zone for test.com failed: ",
              $session->status_code(). ":" .$session->status_detail());

print"Zone test.com added successfully.\n";
```

#Create a DHCP Network

```
my $memberdhcp = Infoblox::DHCP::Member->new(
            name     => "infoblox.localdomain",
            ipv4addr => $host_ip,
           );

my  $network = Infoblox::DHCP::Network->new(
            network => "10.0.0.0/8",
            comment => "add network",
            members => [$memberdhcp],
             );
unless($network){
       die("Construct test.com zone object failed: ",
               Infoblox::status_code(). ":" .Infoblox::status_detail());
       }
print "Network created successfully\n";

my $response = $session->add($network)
       or die("Add Network failed: ",
             $session->status_code() . ":" . $session->status_detail());

print "Network added successfully\n";
```

#Fetch a next available IP from the network

```
my $nip = $network->next_available_ip();
```

#Create a validation object for this IP

```
my $vip = Infoblox::Grid::IPValidationInfo->new(
   parent  => $network,
   address => $nip,
);
```

#Add a DNS A record with this IP and validation

```
#Construct a DNS A object
my $bind_a = Infoblox::DNS::Record::A->new(
    name     => "bind_a.test.com",
    comment  => " this is a demo bind_a record ",
    ipv4addr => $nip,
);
unless ($bind_a) {
   die("Construct DNS record A failed: ",
       Infoblox::status_code() . ":" . Infoblox::status_detail());
}
print "DNS A object created successfully\n";

#Add the DNS A record object to Infoblox Appliance through a session
$session->add($bind_a, ip_validation => [$vip])
    or die("Add record A failed: ",
            $session->status_code() . ":" . $session->status_detail());
print "DNS A object added to server successfully\n";
```

#Cleanup

```
$session->remove($bind_a)
    or die("Remove record A failed: ",
        $session->status_code() . ":" . $session->status_detail());

$session->remove($network)
    or die("Remove network failed: ",
        $session->status_code() . ":" . $session->status_detail());

$session->remove($firstzone)
    or die("Remove zone failed: ",
        $session->status_code() . ":" . $session->status_detail());

####PROGRAM ENDS####
```

AUTHOR

Infoblox Inc. http://www.infoblox.com/

SEE ALSO

Infoblox::Session

COPYRIGHT

Copyright (c) 2017 Infoblox Inc.

Infoblox::Cursor - Manages paged searches.

- NAME
- DESCRIPTION
- CONSTRUCTOR
- METHODS
  - fetch_size( )
  - other parameters
  - Supported object types
- SAMPLE CODE
- AUTHOR
- SEE ALSO
- COPYRIGHT

NAME

Infoblox::Cursor - Manages paged searches.

DESCRIPTION

This class enables users to conduct database searches returning large datasets in an object-by-object manner.

CONSTRUCTOR

```
my $cursor=$session->new_cursor(
                                fetch_size     => 200,
                                object         => "Infoblox::DNS::AllRecords",
                                return_methods => ['name'],
                                name           => ".*"
                               );
```

METHODS

This section describes all the methods that you can use when creating a cursor used for paged searches.

fetch_size( )

This method is used to set the amount of records returned from the back-end to the client for every request. Larger values will cause less requests to be sent to the back-end and more memory used on the client-side; smaller values will cause more requests.

The default value is 100.

other parameters

Other parameters for this constructor are any valid search parameters as described in Infoblox::Session->search()

Supported object types

Paged search is supported for the following object types:

Infoblox::DNS::AllRecords
Infoblox::DNS::Zone
Infoblox::DHCP::FixedAddr
Infoblox::DHCP::Network
Infoblox::DNS::Host
Infoblox::DNS::Record::A
Infoblox::DNS::Record::AAAA
Infoblox::DNS::Record::PTR
Infoblox::DNS::Record::TXT
Infoblox::DNS::Record::MX
Infoblox::DNS::Record::SRV
Infoblox::DNS::Record::CNAME
Infoblox::DNS::Record::DNAME

SAMPLE CODE

The following sample code demonstrates how to execute a paged search.

```
#PROGRAM STARTS: Include all the modules that will be used
```

```
$cursor=$session->new_cursor(
                             fetch_size => 100,
                             object => "Infoblox::DNS::AllRecords",
                             name   => '.*',
                            );

while ($current_record = $cursor->fetch()) {
    # $current_record will contain a matching record object
}
```

AUTHOR

Infoblox Inc. http://www.infoblox.com/

SEE ALSO

Infoblox::Session, Infoblox::Session->search()

COPYRIGHT

Copyright (c) 2017 Infoblox Inc.

Infoblox::Grid::Member::QueryFQDNParameter - A FQDN query parameters.

- NAME
- DESCRIPTION
- CONSTRUCTOR
- MODULE METHODS
  - Infoblox::Session->query_fqdn_on_member( )
- METHODS
  - fqdn( )
  - member( )
  - name_server( )
  - record_type( )
  - recursive_query( )
- AUTHOR
- SEE ALSO
- COPYRIGHT

NAME

Infoblox::Grid::Member::QueryFQDNParameter - A FQDN query parameters.

DESCRIPTION

A FQDN query call parameter objects. This object is intended to use only as Infoblox::Session->query_fqdn_on_member() parameter.

CONSTRUCTOR

```
 my $params = Infoblox::Grid::Member::QueryFQDNParameter->new(
     fqdn            => $string,                                          # Required
     member          => $string,                                          # Optional / Default is GM
     name_server     => $ipv4addr,                                        # Optional / Default is local
nameserver
     record_type     => 'ANY' | 'A' | 'AAAA' | 'CNAME' | 'DNAME' | 'MX' |
                        'NAPTR' | 'NS' | 'PTR' | 'SRV' | 'TXT' | 'AXFR'   # Optional / Default is 'ANY'
     recursive_query => 'true' | 'false',                                 # Optional / Default is 'true'
 );
```

MODULE METHODS

Infoblox::Session->query_fqdn_on_member( )

Use this method to query FQDN on the specified member.

This method forces Grid Master to execute RPC call to invoke dog on the member for a specific FQDN.

See Infoblox::Session->query_fqdn_on_member() for parameters and return values.

Example

```
#Construct parameters object
my $query_fqdn = Infoblox::Grid::Member::QueryFQDNParameter->new(
    fqdn        => 'bar.com',
    member      => 'member1',
    name_server => '5.5.5.5',
    record_type => 'A',
);

#Query
my $response = $session->query_fqdn_on_member($query_fqdn);
```

METHODS

This section describes all the methods that you can use to set or retrieve the attribute values of the object.

fqdn( )

Use this method to set or retrieve a fully-qualified DNS query name.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

The valid value is a FQDN to query in string format.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get fqdn value
my $fqdn = $object->fqdn();

#Modify fqdn value
$object->fqdn($fqdn);
```

member( )

Use this method to set or retrieve a member name to invoke dig on.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

The valid value is a string that contains the Grid Member host name. Default is undefined, which causes dig invokation on Grid Master.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get member value
my $member = $object->member();

#Modify member value
$object->member('localhost.localdomain');
```

name_server( )

Use this method to set or retrieve a nameserver to query.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

The valid value is a string that contains an IPv4 address.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get name_server value
my $value = $object->name_server();

#Modify name_server value
$object->name_server('5.5.5.5');
```

record_type( )

Use this method to set or retrieve the resource record type to query.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

The valid values are 'ANY', 'A', 'AAAA', 'CNAME', 'DNAME', 'MX', 'NAPTR', 'NS', 'PTR', 'SRV', 'TXT' and 'AXFR'. The default value is 'ANY'.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get record_type value
my $value = $object->record_type();

#Modify record_type value
$object->record_type('AAAA');
```

recursive_query( )

Use this method to set or retrieve the recursive query flag.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

Specify 'true' to set recursive query flag and 'false' to unset/deactivate it. The default value is 'true'.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get recursive_query value
my $value = $object->recursive_query();

#Modify recursive_query value
$object->recursive_query('false');
```

AUTHOR

Infoblox Inc. http://www.infoblox.com/

SEE ALSO

Infoblox::Session->query_fqdn_on_member()

COPYRIGHT

Copyright (c) 2017 Infoblox Inc.

Infoblox::Grid::Member::QueryFQDNResponse - A FQDN query response.

- NAME
- DESCRIPTION
- METHODS
  - result( )
  - result_text( )
  - dig_started( )
  - source_address( )
- AUTHOR
- SEE ALSO
- COPYRIGHT

NAME

Infoblox::Grid::Member::QueryFQDNResponse - A FQDN query response.

DESCRIPTION

A FQDN query call response objects. This object is returned from Infoblox::Session->query_fqdn_on_member() function call.

METHODS

This section describes all the methods that you can use to set or retrieve the attribute values of the object.

Note that this is a read-only object.

result( )

Use this method to set or retrieve a dig request return code.

Returns

The method returns the attribute value, which can be 'NOERROR', 'FORMERR', 'SERVFAIL', 'NXDOMAIN', 'NOTIMP', 'REFUSED' or 'INTERNAL_ERROR'.

Example

```
#Get result value
my $result = $object->result();
```

result_text( )

Use this method to set or retrieve a dig result multilined text.

Returns

The method returns the attribute value, a dig result text in string format.

Example

```
#Get result_text value
my $result_text = $object->result_text();
```

dig_started( )

Use this method to set or retrieve a timestamp since dig started.

Returns

The method returns the attribute value, the number of seconds that have elapsed since January 1st, 1970.

Example

```
#Get dig_started value
my $dig_started = $object->dig_started();
```

source_address( )

Use this method to set or retrieve a source IP address of the query.

Returns

The method returns the attribute value, the IP address in string format.

Example

```
#Get source_address value
my $source_address = $object->source_address();
```

AUTHOR

Infoblox Inc. http://www.infoblox.com/

SEE ALSO

Infoblox::Session->query_fqdn_on_member()

COPYRIGHT

Copyright (c) 2017 Infoblox Inc.

Infoblox::DHCP::FailOver - DHCP Failover object.

- NAME
- DESCRIPTION
- CONSTRUCTOR
- SESSION METHODS
  - Infoblox::Session->add( )
  - Infoblox::Session->get( )
  - Infoblox::Session->modify( )
  - Infoblox::Session->remove( )
  - Infoblox::Session->search( )
- METHODS
  - association_type( )
  - ms_association_mode( )
  - ms_enable_authentication( )
  - ms_enable_switchover_interval( )
  - ms_failover_mode( )
  - ms_failover_partner( )
  - ms_hotstandby_partner_role( )
  - ms_is_conflict( )
  - ms_server( )
  - ms_shared_secret( )
  - ms_state( )
  - ms_switchover_interval( )
  - ms_previous_state( )
  - name( )
  - primary( )
  - secondary( )
  - comment( )
  - extattrs( )
  - extensible_attributes( )
  - load_balance_split( )
  - max_load_balance_delay( )
  - max_client_lead_time( )
  - max_unacked_updates( )
  - max_response_delay( )
  - primary_state( )
  - recycle_leases( )
  - secondary_state( )
- SAMPLE CODE
- AUTHOR
- SEE ALSO
- COPYRIGHT

NAME

Infoblox::DHCP::FailOver - DHCP Failover object.

DESCRIPTION

DHCP failover is a protocol designed to allow a backup DHCP server to take over for a main server if the main server is taken off the network for any reason. DHCP failover can be used to configure two DHCP servers to operate as a redundant pair.

CONSTRUCTOR

```
 my $dhcp_failover = Infoblox::DHCP::FailOver->new(
     name                   => $string,                              #Required
     primary                => $ipv4addr | $Member                   #Required
     secondary              => $ipv4addr | $Member                   #Required
     comment                => $string,                              #Optional / Default is empty
     extattrs               => { $string => $extattr, ... },         #Optional / Default is undefined
     extensible_attributes  => { $string => $string | $num, $string => [ $string | $num, ... ], ... },
#Optional / Default is undefined
     load_balance_split     => $num,                                 #Optional / Default value is 128
     max_load_balance_delay => $num,                                 #Optional / Default value is 3
     max_client_lead_time   => $num,                                 #Optional / Default value is 3600
     max_unacked_updates    => $num,                                 #Optional / Default value is 10
     max_response_delay     => $num,                                 #Optional / Default value is 60
     recycle_leases         => "true" | "false" | undef,             #Optional / Default value is undefined
 );
```

You cannot set both extattrs and extensible_attributes attributes at the same time.

SESSION METHODS

This section describes all the methods in Infoblox::Session module that can be applied to a DHCP failover object.

Infoblox::Session->add( )

Use this method to add a DHCP failover object to the Infoblox appliance. See Infoblox::Session->add() for parameters and return values.

Example

```
#Create a DHCP Member object
my $memberdhcp = Infoblox::DHCP::Member->new(
      name     => "infoblox.localdomain",
      ipv4addr => "192.168.1.2"
);

#Construct an object
my $dhcp_failover = Infoblox::DHCP::FailOver->new(
    name                  => "peer1",
    primary               => $memberdhcp,
    secondary             => "1.1.1.1",
    load_balance_split    => 128,
    max_load_balance_delay => 3,
    max_client_lead_time  => 3600,
    max_unacked_updates   => 10,
    max_response_delay    => 60,
    recycle_leases        => "false",
);

# Submit for addition
my $response = $session->add( $dhcp_failover );
```

Infoblox::Session->get( )

Use this method to retrieve all the matching DHCP failover objects from the Infoblox appliance. See Infoblox::Session->get() for parameters and return values.

Key References

```
Apply the following attributes to get a specific DHCP failover object:

name - Required. Name of a DHCP failover in string format.
extattrs     - Optional. A hash reference containing extensible attributes.
```

Example

```
my @retrieved_objs = $session->get(
                     object     => "Infoblox::DHCP::FailOver",
                     name       => "peer1" );
```

Infoblox::Session->modify( )

Use this method to modify a DHCP failover object in the Infoblox appliance. See Infoblox::Session->modify() for parameters and return values.

Example

```
# Use method to modify the load_balance_split.
$dhcp_failover->load_balance_split(130);
# Submit modification
my $response = $session->modify( $dhcp_failover );
```

Infoblox::Session->remove( )

Use this method to remove a DHCP failover object from the Infoblox appliance. See Infoblox::Session->remove() for parameters and return values.

To remove a specific object, first use `get()` or `search()` to retrieve the specific DHCP failover object, and then submit this object for removal.

Example

```
# Get the objects with the same name
my @retrieved objs   $session  get
```

```
                    =         ->    (
    object  => "Infoblox::DHCP::FailOver",
    name    => "peer1" );
# Find the desired object from the retrieved list.
my $desired_dhcp_failover = $retrieved_objs[0];
# Submit for removal
my $response = $session->remove( $desired_dhcp_failover );
```

Infoblox::Session->search( )

Use this method to search for DHCP failover objects in the Infoblox appliance. See Infoblox::Session->search() for parameters and return values.

Key References

Apply the following attributes, to search for a DHCP failover object:

```
name    - Optional. The name of a DHCP failover object. Regular expression supported.
comment - Optional. The comment of the DHCP failover object. Regular expression supported.
extattrs      - Optional. A hash reference containing extensible attributes.
extensible_attributes - Optional. A hash reference containing extensible attributes.
```

For more information about searching extensible attributes, see Infoblox::Grid::ExtensibleAttributeDef/Searching Extensible Attributes.

Example

```
# search for DHCP failover object with name as peer1
my @retrieved_objs = $session->search(
    object  => "Infoblox::DHCP::FailOver",
    name    => "pe.*" );

# search all DHCP failover associations with the extensible attribute 'Site'
my @retrieved_objs = $session->search(
   object => "Infoblox::DHCP::FailOver",
   extensible_attributes => { 'Site' => 'Santa Clara' });
```

METHODS

This section describes all the methods that can be used to configure and retrieve the attribute values of a DHCP failover object.

association_type( )

Use this method to retrieve the value indicating whether the failover assocation is Microsoft or Grid based. This is a read-only attribute.

Parameter

None

Returns

The return value can be "MS" or "GRID".

Example

```
#Get association type
my $association_type = $failover->association_type();
```

ms_association_mode( )

Use this method to retrieve the value that indicates whether the failover association is read-write or read-only. This is a read-only attribute.

Parameter

None

Returns

The valid return values are "RO" and "RW".

Example

```
#Get association_mode
my $association_mode = $failover->ms_association_mode();
```

ms_enable_authentication( )

Use this method to enable or disable authentication for the failover association.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

Specify "true" to enable authentication for the failover association or "false" to disable it. The default value is "false".

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get ms_enable_authentication value
my $enable_authentication_value = $failover->ms_enable_authentication();
#Modify ms_enable_authentication value
$failover->ms_enable_authentication("true");
```

ms_enable_switchover_interval( )

Use this method to enable or disable the switchover interval.

The ms_enable_switchover_interval attribute can be specified explicitly. It is also set implicitly when ms_switchover_interval is set to a defined value.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

Specify "true" to enable the switchover interval or "false" to disable it. The default value is "false".

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get ms_enable_switchover_interval value
my $enable_switchover_interval = $failover->ms_enable_switchover_interval();
#Modify ms_enable_switchover_interval value
$failover->ms_enable_switchover_interval("true");
```

ms_failover_mode( )

Use this method to set or retrieve the mode for the failover association.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

The valid parameters are "LOADBALANCE" and "HOTSTANDBY".

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

```
=item B<Example>

#Get ms_failover_mode
my $failover_mode = $failover->ms_failover_mode();
#Modify ms_failover_mode
$failover->ms_failover_mode("HOTSTANDBY");
```

ms_failover_partner( )

Use this method to set or retrieve the failover partner defined in the association for the Microsoft Server.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Note that the failover partner must be retrieved from the server settings and cannot be changed after being written.

Parameter

The valid parameter is an Infoblox::Grid::MSServer object.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get ms_failover_partner
my $failover_partner = $failover->ms_failover_partner();
#Modify ms_failover_partner
$failover->ms_failover_partner($failover_partner);
```

ms_hotstandby_partner_role( )

Use this method to set or retrieve the partner role in the case of HotStandby.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

The valid values are "ACTIVE" and "PASSIVE".

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get ms_hotstandby_partner_role
my $partner_role = $failover->ms_hotstandby_partner_role();
#Modify ms_hotstandby_partner_role
$failover->ms_hotstandby_partner_role("ACTIVE");

=back
```

ms_is_conflict( )

Use this method to retrieve the flag used to indicate whether the matching Microsfot failover association (if any) is in synchronization (False) or not (True). The method returns False if there is no matching failover association. This is a read-only attribute.

Parameter

None

Returns

The valid return values are "true" and "false".

Example

```
#Get ms_is_conflict value
my $is_conflict = $failover->ms_is_conflict();
```

ms_server( )

Use this method to set or retrieve the primary Microsoft Server object.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Note that the Microsoft server must be retrieved from the server settings and cannot be changed after being written.

Parameter

The valid parameter is an Infoblox::Grid::MSServer object.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get ms_server
my $server = $failover->ms_server();
#Modify ms_failover_partner
$failover->ms_server($server);
```

ms_shared_secret( )

Use this method to set the shared secret for the failover association authentication. This is a write-only attribute.

Parameter

The valid value is a string that contains the shared secret for the failover association authentication.

Returns

The method returns true when the modification succeeds, and returns false when the operation fails.

Example

```
#Set ms_shared_secret value
$failover->ms_shared_secret($shared_secret);
```

ms_state( )

Use this method to retrieve the failover association state. This is a read-only attribute.

Parameter

None

Returns

The valid return values are "NO_STATE", "INIT", "STARTUP", "NORMAL", "COMMUNICATION_INIT", "PARTNER_DOWN", "POTENTIAL_CONFLICT", "CONFLICT_DONE", "RESOLUTION_INIT", "RECOVER", "RECOVER_DONE", "RECOVER_WAIT".

Example

```
#Get ms_state value
my $state = $failover->ms_state();
```

ms_switchover_interval( )

Use this method to set or retrieve the time (in seconds) that DHCPv4 server will wait before transitioning the server from the COMMUNICATION-INT state to PARTNER-DOWN state.

Setting this method to a defined value implicitly sets the ms_enable_switchover_interval method to "true". Setting the parameter to undefined causes the appliance to use the grid default and automatically resets the ms_enable_switchover_interval method to "false".

Note that when ms_switchover_interval is set to a defined value and ms_enable_switchover_interval is set to "false", the last operation takes precedence. Thus the sequence $object->ms_switchover_interval(20); $object->ms_enable_switchover_interval("false"); will set ms_enable_switchover_interval to "false", and the sequence $object->ms_enable_switchover_interval("false"); $object->ms_switchover_interval(20); will result in ms_enable_switchover_interval="true".

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

The valid value are integers between 1 and 65535.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get ms_switchover_interval value
my $so_interval = $failover->ms_switchover_interval();
#Modify ms_switchover_interval value
$failover->ms_switchover_interval(1000);
```

ms_previous_state( )

Use this method to retrieve the previous failover association state. This is a read-only attribute.

Parameter

None

Returns

The valid return values are "NO_STATE", "INIT", "STARTUP", "NORMAL", "COMMUNICATION_INIT", "PARTNER_DOWN", "POTENTIAL_CONFLICT", "CONFLICT_DONE", "RESOLUTION_INIT", "RECOVER", "RECOVER_DONE", "RECOVER_WAIT".

Example

```
#Get ms_previous_state value
my $previous_state = $failover->ms_previous_state();
```

name( )

Use this method to set or retrieve the name of a DHCP failover object.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

Specify the name in string format.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get name
my $name = $dhcp_failover->name();
#Modify name
$dhcp_failover->name("peer2");
```

primary( )

Use this method to set or retrieve primary server of a DHCP failover object.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

Specify external primary server in IPv4 address format or grid primary server in a defined Infoblox::DHCP::Member object. An IPv4 address is a 32-bit number in dotted decimal notation. It consists of four 8-bit groups of decimal digits separated by decimal points (example: 192.168.1.2).

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get primary
my $primary = $dhcp_failover->primary();
#Modify primary
$dhcp_failover->primary($member); #$member is a Infoblox::DHCP::Member object
```

secondary( )

Use this method to set or retrieve secondary server of a DHCP failover object.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

Specify external secondary server in IPv4 address format or grid secondary server in a defined Infoblox::DHCP::Member object. An IPv4 address is a 32-bit number in dotted decimal notation. It consists of four 8-bit groups of decimal digits separated by decimal points (example: 192.168.1.2).

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get secondary
my $secondary = $dhcp_failover->secondary();
#Modify secondary
$dhcp_failover->secondary("2.2.2.2");
```

comment( )

Use this method to set or retrieve a descriptive comment about a DHCP failover object.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

Enter a descriptive comment for the network in string format with a maximum of 256 bytes.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
# Get comment
my $comment = $dhcp_failover->comment();
# Modify comment
$dhcp_failover->comment("add a network");
```

extattrs( )

Use this method to set or retrieve the extensible attributes associated with a DHCP failover object.

Parameter

Valid value is a hash reference containing the names of extensible attributes and their associated values (Infoblox::Grid::Extattr objects).

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get extattrs
my $ref_extattrs = $dhcp_failover->extattrs();
#Modify extattrs
$dhcp_failover->extattrs({ 'Site' => $extattr1, 'Administrator' => $extattr2 });
```

extensible_attributes( )

Use this method to set or retrieve the extensible attributes associated with a DHCP failover object.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

For valid values for extensible attributes, see Infoblox::Grid::ExtensibleAttributeDef/Extensible Attribute Values.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get extensible attributes
my $ref_extensible_attributes = $dhcp_failover->extensible_attributes();
#Modify extensible attributes
$dhcp_failover->extensible_attributes({ 'Site' => 'Santa Clara', 'Administrator' => [ 'Peter', 'Tom'
] });
```

load_balance_split( )

Use this method to set or retrieve a load balancing split value of a DHCP failover object.

Load balancing split determines which server handles IP address requests. This parameter is specified on the primary only.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

Specify the value of the maximum load balancing delay in a 8-bit integer format (range from 0 to 255). The default value is 128.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get load_balance_split
my $load_balance_split = $dhcp_failover->load_balance_split();
#Modify load_balance_split
$dhcp_failover->load_balance_split(200);
```

max_load_balance_delay( )

Use this method to set or retrieve the maximum load balancing delay value of a DHCP failover object.

Maximum load balancing delay specifies the cutoff after which load balancing is disabled. The cutoff is based on the number of seconds since the client sent its first DHCPDISCOVER or DHCPREQUEST message. For instance, if one of the failover peers gets into a state where it is responding to failover messages, but not responding to some client requests, the other failover peer will take over its client load automatically as the clients retry.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

Specify the value of the maximum load balancing delay in a 32-bit integer format (range from 0 to 4294967295) that represents the duration in seconds. The default value is 3.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get max_load_balance_delay
my $max_load_balance_delay = $dhcp_failover->max_load_balance_delay();
#Modify max_load_balance_delay
$dhcp_failover->max_load_balance_delay("500");
```

max_client_lead_time( )

Use this method to set or retrieve the maximum client lead time value of a DHCP failover object.

Maximum client lead time value specifies the length of time that a failover peer can renew a lease without contacting the other peer. The larger the number, the longer it takes for the peer server to recover IP addresses after moving to the Partner Down mode. The smaller the number, the more load your servers experience when they are not communicating. This is specified on the primary server only.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

Specify the value of the maximum client lead time in a 32-bit integer format (range from 0 to 4294967295) that represents the duration in seconds. The default value is 3600.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get max_client_lead_time
my $max_client_lead_time = $dhcp_failover->max_client_lead_time();
```

```
#Modify max_client_lead_time
$dhcp_failover->max_client_lead_time(600);
```

max_unacked_updates( )

Use this method to set or retrieve the maximum number of unacked updates value of a DHCP failover object.

Maximum number of unacked updates value specifies how many update messages the server can send before it should receive an ACK from the failover peer. If no ACK is received after these messages are sent, failover occurs.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

Specify the value of the maximum number of unacked updates in a 32-bit integer format (range from 0 to 4294967295) that represents the number of messages. The default value is 10.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get max_unacked_updates
my $max_unacked_updates = $dhcp_failover->max_unacked_updates();
#Modify max_unacked_updates
$dhcp_failover->max_unacked_updates(700);
```

max_response_delay( )

Use this method to set or retrieve the maximum response delay value of a DHCP failover object.

Maximum response delay value before failover specifies how many seconds can transpire before the primary server assumes its peer (the secondary server) is not sending messages due to failure.

Include the specified parameter to set the attribute value. Omit the parameter to retrieve the attribute value.

Parameter

Specify the value of the maximum response delay in a 32-bit integer format (range from 0 to 4294967295) that represents the duration in seconds. The default value is 60.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get max_response_delay
my $max_response_delay = $dhcp_failover->max_response_delay();
#Modify max_response_delay
$dhcp_failover->max_response_delay(800);
```

primary_state( )

Use this method to retrieve the primary server status of a DHCP failover object.

Parameter

None

Returns

The method returns the attribute value.

Example

```
#Get primary_state
my $primary_state = $dhcp_failover->primary_state();
```

recycle_leases( )

Use this method to set or retrieve the recycle leases flag of a DHCP failover object.

If the flag is enabled, the leases are kept in recycle bin until one week after expiration and when the flag is disabled, the leases are irrecoverably deleted.

Parameter

Specify "true" to set the recycle_leases flag or "false" to deactivate/unset it. The default value for this field is undefined.

Returns

If you specified a parameter, the method returns true when the modification succeeds, and returns false when the operation fails.

If you did not specify a parameter, the method returns the attribute value.

Example

```
#Get recycle_leases
my $recycle_leases = $dhcp_failover->recycle_leases();
#Modify recycle_leases
$dhcp_failover->recycle_leases("true");
#Un-override recycle_leases
$dhcp_failover->recycle_leases(undef);
```

secondary_state( )

Use this method to retrieve the secondary server status of a DHCP failover object.

Parameter

None

Returns

The method returns the attribute value.

Example

```
#Get secondary_state
my $secondary_state = $dhcp_failover->secondary_state();
```

SAMPLE CODE

The following sample code demonstrates the different functions that can be applied to a DHCP failover object such as add, search, modify, and remove. Also, this sample includes error handling for the operations.

#Preparation prior to a DHCP failover object insertion

```
#PROGRAM STARTS: Include all the modules that will be used
use strict;
use Infoblox;

#Create a session to the Infoblox appliance
my $session = Infoblox::Session->new(
               master   => "192.168.1.2", #appliance host ip
               username => "admin",       #appliance user login
               password => "infoblox"     #appliance password
);
unless ($session) {
       die("Construct session failed: ",
            Infoblox::status_code() . ":" . Infoblox::status_detail());
}
print "Session created successfully\n";

#Create a DHCP Member object
my $memberdhcp = Infoblox::DHCP::Member->new(
    name     => "infoblox.localdomain",
    ipv4addr => "192.168.1.2"
);
unless($memberdhcp) {
       die("Construct member failed: ",
            Infoblox::status_code() . ":" . Infoblox::status_detail());
}
print "DHCP Member object created successfully\n";

#Create the Network object with member
my $network = Infoblox::DHCP::Network->new(
    network => "10.0.0.0/8",
    comment => "add network",
```

```
    members => [ $memberdhcp ]
);
unless($network) {
       die("Construct Network object failed: ",
            Infoblox::status_code() . ":" . Infoblox::status_detail());
}
print "Network object created successfully\n";

#Add the Network object into the Infoblox appliance through a session
$session->add($network)
    or die("Add Network object failed: ",
            $session->status_code() . ":" . $session->status_detail());
print "Network object added to Infoblox appliance successfully\n";
```

#Create a DHCP failover object

```
my $dhcp_failover = Infoblox::DHCP::FailOver->new(
    name                  => "peer1",
    primary               => $memberdhcp,
    secondary             => "1.1.1.1",
    load_balance_split    => 128,
    max_load_balance_delay => 3,
    max_client_lead_time  => 3600,
    max_unacked_updates   => 10,
    max_response_delay    => 60,
    recycle_leases        => "true",
);
unless($dhcp_failover) {
       die("Construct DHCP failover object failed: ",
            Infoblox::status_code() . ":" . Infoblox::status_detail());
}
print "DHCP failover object created successfully\n";

#Add a DHCP failover object into the Infoblox appliance through a session
$session->add($dhcp_failover)
    or die("Add DHCP failover object failed: ",
            $session->status_code() . ":" . $session->status_detail());
print "DHCP failover object added to the network successfully\n";
```

#Get and modify a DHCP failover

```
#Get a DHCP failover through the session
my @retrieved_objs = $session->get(
    object  => "Infoblox::DHCP::FailOver",
    name    => "peer1",
);
my $object = $retrieved_objs[0];
unless ($object) {
       die("Get DHCP failover failed: ",
            $session->status_code() . ":" . $session->status_detail());
}
print "Get DHCP failover object found at least 1 matching entry\n";

#Modify one of the attributes of the specified DHCP failover
$object->max_client_lead_time(3500);

#Apply the changes
$session->modify($object)
    or die("Modify DHCP failover failed: ",
            $session->status_code() . ":" . $session->status_detail());
print "DHCP failover object modified successfully \n";
```

#Search a DHCP failover

```
# search for DHCP failover object with name starting with letter "pe"
my @retrieved_objs = $session->search(
    object => "Infoblox::DHCP::FailOver",
    name   => "pe.*"
);

my $object = $retrieved_objs[0];
unless ($object) {
    die("Search DHCP failover failed: ",
        $session->status_code() . ":" . $session->status_detail());
}
print "Search DHCP failover object found at least 1 matching entry\n";
```

#Add a DHCP range with failover

```
#Construct the DHCP range with failover peer1
my $dhcp_range = Infoblox::DHCP::Range->new(
        end_addr       => "10.0.0.10",
        network        => "10.0.0.0/8",
        start_addr     => "10.0.0.1",
        failover_assoc => "peer1"
);
unless($dhcp_range) {
       die("Construct DHCP Range object with failover failed: ",
            Infoblox::status_code() . ":" . Infoblox::status_detail());
}
print "DHCP Range object with failover peer1 created successfully\n";

#Add the DHCP Range object with failover into the Infoblox appliance through a session
$session->add($dhcp_range)
    or die("Add DHCP Range object with failover failed: ",
            $session->status_code() . ":" . $session->status_detail());
print "DHCP Range object with failover added to the network successfully\n";

#submit the network object for removal
$session->remove($network)
    or die("Remove Network failed: ",
            $session->status_code() . ":" . $session->status_detail());
print "Network removed successfully \n";
```

#Remove a DHCP failover

```
#Get a DHCP failover through the session
my @retrieved_objs = $session->get(
    object => "Infoblox::DHCP::FailOver",
    name   => "peer1"
);

my $object = $retrieved_objs[0];
unless ($object) {
       die("Get DHCP failover failed: ",
            $session->status_code() . ":" . $session->status_detail());
}
print "Get DHCP failover object found at least 1 matching entry\n";

#submit the object for removal
$session->remove($object)
    or die("Remove DHCP failover failed: ",
            $session->status_code() . ":" . $session->status_detail());
print "DHCP failover object removed successfully \n";

####PROGRAM ENDS####
```

AUTHOR

Infoblox Inc. http://www.infoblox.com/

SEE ALSO

Infoblox::Session, Infoblox::Session->get(), Infoblox::Session->search(), Infoblox::Session->add(), Infoblox::Session->remove(), Infoblox::Session->modify(), Infoblox::DHCP::Member

---

COPYRIGHT

Copyright (c) 2017 Infoblox Inc.

Infoblox::Grid::CSVImportStatus - CSV import status object

- NAME
- DESCRIPTION
- MODULE METHODS
  - Infoblox::Session->get( )
- METHODS
  - admin( )
  - end_time( )
  - file_name( )
  - import_id( )
  - lines_failed( )
  - lines_processed( )
  - lines_total( )
  - lines_warning( )
  - start_time( )
  - status( )
- SAMPLE CODE
- AUTHOR
- SEE ALSO
- COPYRIGHT

NAME

Infoblox::Grid::CSVImportStatus - CSV import status object

DESCRIPTION

This object is used to represent the status of the current CSV import task. All methods are read-only.

MODULE METHODS

The following functions can be applied to a Grid::CSVImportStatus object.

Infoblox::Session->get( )

Use this method to retrieve matching objects from the Infoblox appliance. See Infoblox::Session->get() for parameters and return values.

Key Reference

```
Apply the following attributes to get a specific CSV import status object

 object    - Required. It must be set to "Infoblox::Grid::CSVImportStatus".
 import_id - Optional. ID of the import task that the status object is describing. If not specified,
             the status of the first import task is returned.
```

Example

```
my $import_id = $session->import_data(
                                      type => 'csv',
                                      path => "/tmp/zone.csv",
                                      separator   => 'semicolon',
                                     );

unless($import_id){
    die("Import failed: ",
        Infoblox::status_code(). ":" .Infoblox::status_detail());
}

my @retrieved_objs = $session->get(
    object    => "Infoblox::Grid::CSVImportStatus",
    import_id => $import_id,
    );
```

METHODS

This section describes all the methods that you can use to retrieve the status of a CSV import operation.

admin( )

Use this method to retrieve the name of the administrator that started the import.

Returns

The method returns the attribute value.

Example

```
#Get admin
my $admin = $service_status->admin();
```

end_time( )

Use this method to retrieve the time the CSV import ended.

Returns

The method returns the attribute value.

Example

```
#Get end_time
my $end_time = $service_status->end_time();
```

file_name( )

Use this method to retrieve the name of the file used for the CSV import.

Returns

The method returns the attribute value.

Example

```
#Get file_name
my $file_name = $service_status->file_name();
```

import_id( )

Use this method to retrieve the ID of the import task that the status object is describing.

Returns

The method returns the attribute value.

Example

```
#Get import_id
my $import_id = $service_status->import_id();
```

lines_failed( )

Use this method to retrieve the number of lines that had failures in the CSV file used for the current CSV import.

Returns

The method returns the attribute value.

Example

```
#Get lines_failed
my $lines_failed = $service_status->lines_failed();
```

lines_processed( )

Use this method to retrieve the number of lines processed in the CSV file used for the current CSV import.

Returns

The method returns the attribute value.

Example

```
#Get lines_processed
my $lines_processed = $service_status->lines_processed();
```

lines_total( )

Use this method to retrieve the total number of lines read from the CSV file during the current CSV import.

Returns

The method returns the attribute value.

Example

```
#Get lines_total
my $lines_total = $service_status->lines_total();
```

lines_warning( )

Use this method to retrieve the number of lines that had warnings in the CSV file used for the current CSV import.

Returns

The method returns the attribute value.

Example

```
#Get lines_warning
my $lines_warning = $service_status->lines_warning();
```

start_time( )

Use this method to retrieve the time the CSV import started.

Returns

The method returns the attribute value.

Example

```
#Get start_time
my $start_time = $service_status->start_time();
```

status( )

Use this method to retrieve the status of the current CSV import. The returned status is one of the following: 'Uploaded', 'Pending', 'Running', 'Failed', 'Stopped' or 'Completed'.

Returns

The method returns the attribute value.

Example

```
#Get status
my $status = $service_status->status();
```

SAMPLE CODE

The following sample code demonstrates how to export / import data by using CSV files.

```
#PROGRAM STARTS: Include all the modules that will be used
use strict;
use Infoblox;

#Create a session to the Infoblox appliance
my $host_ip =  "192.168.1.2";
my $session = Infoblox::Session->new(
                                     master   => $host_ip,
                                     username => "admin",
                                     password => "infoblox"
                                    );
unless($session){
    die("Constructor for session failed: ",
        Infoblox::status_code(). ":" . Infoblox::status_detail());
}
print "Session created successfully.\n";

#Create a DNS zone
my $memberns1 = Infoblox::DNS::Member->new(
                                           name     => "infoblox.localdomain",
                                           ipv4addr => $host_ip,
```

```
                                          lead     => "false",
                                          stealth  => "false"
                                         );

#Add the zone
my $firstzone = Infoblox::DNS::Zone->new(
                                         name        => "test.com",
                                         email       => "admin\@infoblox.com",
                                         comment     => "add a zone test.com",
                                         primary     => $memberns1,
                                        );

unless($firstzone){
    die("Construct test.com zone object failed: ",
        Infoblox::status_code(). ":" .Infoblox::status_detail());
}
print "test.com zone object created successfully.\n";

$session->add($firstzone)
  or die("Add zone for test.com failed: ",
         $session->status_code(). ":" .$session->status_detail());

print"Zone test.com added successfully.\n";

#Export the newly created zone
$session->export_data(
                      type        => 'csv',
                      path        => '/tmp/zone.csv',
                      object_type => 'Infoblox::DNS::Zone',
                      separator   => 'semicolon',

                      # search parameters
                      name        => '.*test.com',
                                          )
                         or die("Export zone for test.com failed: ",
                                $session->status_code(). ":" .$session->status_detail());

# Wait for the export to complete
sleep(5);

print "The zone was exported\n";

# Remove the zone from the database
$session->remove($firstzone)
  or die("Remove zone for test.com failed: ",
         $session->status_code(). ":" .$session->status_detail());

print "The zone was removed\n";

# Import the zone by using the CSV file that was exported

my $import_id = $session->import_data(
                                      type => 'csv',
                                      path => "/tmp/zone.csv",
                                      separator   => 'semicolon',
                                     );

unless($import_id){
    die("Import failed: ",
        Infoblox::status_code(). ":" .Infoblox::status_detail());
}

# Wait for the import to complete
sleep(5);

my $status = $session->get(
                           object  => 'Infoblox::Grid::CSVImportStatus',
                          );

unless($status){
    die("Failed to fetch the import status: ",
        Infoblox::status_code(). ":" .Infoblox::status_detail());
}

print "The zone was re-imported\n";

# The code should be cycling to sleep, fetch and wait until the import is finished.
unless($status->status() eq 'Completed'){
    die("The import took too long to complete: " . $status->status());
```

```
}

unless($status->lines_failed() == 0){
    die("There were some import failures: " . $status->lines_failed());
}

print "The zone re-import was successful\n";

# The zone was re-imported. If we import the file again without merging,
# an error occurs.

$import_id = $session->import_data(
                                    type => 'csv',
                                    path => "/tmp/zone.csv",
                                    separator   => 'semicolon',
                                  );

unless($import_id){
    die("Import failed: ",
        Infoblox::status_code(). ":" .Infoblox::status_detail());
}

# Wait for the import to complete
sleep(5);

print "The zone re-re-import was completed\n";

my $status = $session->get(
                            object  => 'Infoblox::Grid::CSVImportStatus',
                           );

unless($status){
    die("Failed to fetch the import status: ",
        Infoblox::status_code(). ":" .Infoblox::status_detail());
}

# The code should be cycling to sleep, fetch and wait until the import is finished.
unless($status->status() eq 'Completed'){
    die("The import took too long to complete: " . $status->status());
}

if($status->lines_failed() == 0){
    die("There were no import failures: " . $status->lines_failed());
}

print "The zone re-re-import correctly failed\n";

# Review the  failure
$session->export_data(
                      type        => 'csv_error_log',
                      path        => '/tmp/error_log.txt',
                      import_id   => $import_id,
                     )
  or die("Export error log failed: ",
         $session->status_code(). ":" .$session->status_detail());

# The failure is in this file
print "Error log for the failed import\n-------\n";
print `cat /tmp/error_log.txt`;

#Remove the zone
$session->remove(
                 object => "Infoblox::DNS::Zone",
                 name   => "test.com",
                )
  or die("Remove zone test.com failed: ",
         $session->status_code(). ":" .$session->status_detail());

print "The zone was removed\n";

####PROGRAM ENDS####
```

AUTHOR

Infoblox Inc. http://www.infoblox.com/

SEE ALSO

Infoblox::Session,Infoblox::Session->get()

COPYRIGHT

Copyright (c) 2017 Infoblox Inc.

Infoblox unites networking and security to deliver unmatched performance and protection. Trusted by Fortune 100 companies and emerging innovators, we provide real-time visibility and control over who and what connects to your network, so your organization runs faster and stops threats earlier.

Corporate Headquarters
2390 Mission College Blvd, Ste. 501
Santa Clara, CA 95054
+1.408.986.4000
www.infoblox.com

© 2023 Infoblox, Inc. All rights reserved. Infoblox logo, and other marks appearing herein are property of Infoblox, Inc.
All other marks are the property of their respective owner(s).